axiom™

The 30 Year Horizon

Manuel Bronstein *William Burge* *Timothy Daly*
James Davenport *Michael Dewar* *Martin Dunstan*
Albrecht Fortenbacher *Patrizia Gianni* *Johannes Grabmeier*
Jocelyn Guidry *Richard Jenks* *Larry Lambe*
Michael Monagan *Scott Morrison* *William Sit*
Jonathan Steinbach *Robert Sutor* *Barry Trager*
Stephen Watt *Jim Wen* *Clifton Williamson*

Volume 10.5: Axiom Algebra: Numerics

Portions Copyright (c) 2005 Timothy Daly

The Blue Bayou image Copyright (c) 2004 Jocelyn Guidry

Portions Copyright (c) 2004 Martin Dunstan
Portions Copyright (c) 2007 Alfredo Portes
Portions Copyright (c) 2007 Arthur Ralfs
Portions Copyright (c) 2005 Timothy Daly

Portions Copyright (c) 1991-2002,
The Numerical ALgorithms Group Ltd.
All rights reserved.

This book and the Axiom software is licensed as follows:

Redistribution and use in source and binary forms, with or
without modification, are permitted provided that the following
conditions are
met:

- Redistributions of source code must retain the above
  copyright notice, this list of conditions and the
  following disclaimer.

- Redistributions in binary form must reproduce the above
  copyright notice, this list of conditions and the
  following disclaimer in the documentation and/or other
  materials provided with the distribution.

- Neither the name of The Numerical ALgorithms Group Ltd.
  nor the names of its contributors may be used to endorse
  or promote products derived from this software without
  specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND
CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES,
INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF
MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR
CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING,
BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR
SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS
INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY,
WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING
NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
SUCH DAMAGE.

Inclusion of names in the list of credits is based on historical information and is as accurate as possible. Inclusion of names does not in any way imply an endorsement but represents historical influence on Axiom development.

Michael Albaugh
Cyril Alberga
Roy Adler
Christian Aistleitner
Richard Anderson
George Andrews
S.J. Atkins
Jeremy Avigad
Henry Baker
Martin Baker
Stephen Balzac
Yurij Baransky
David R. Barton
Thomas Baruchel
Gerald Baumgartner
Gilbert Baumslag
Michael Becker
Nelson H. F. Beebe
Jay Belanger
David Bindel
Fred Blair
Vladimir Bondarenko
Mark Botch
Raoul Bourquin
Alexandre Bouyer
Karen Braman
Wolfgang Brehm
Peter A. Broadbery
Martin Brock
Manuel Bronstein
Stephen Buchwald
Florian Bundschuh
Luanne Burns
William Burge
Ralph Byers
Quentin Carpent
Robert Cavines
Bruce Char
Ondrej Certik
Tzu-Yi Chen
Bobby Cheng
Cheekai Chin
David V. Chudnovsky
Gregory V. Chudnovsky
Mark Clements
James Cloos
Jia Zhao Cong
Josh Cohen
Christophe Conil
Don Coppersmith
George Corliss
Robert Corless
Gary Cornell
Meino Cramer
Jeremy Du Croz
David Cyganski
Nathaniel Daly
Timothy Daly Sr.
Timothy Daly Jr.
James H. Davenport
David Day
James Demmel
Didier Deshommes
Michael Dewar
Inderjit Dhillon
Jack Dongarra
Jean Della Dora
Gabriel Dos Reis
Claire DiCrescendo
Sam Dooley
Zlatko Drmac
Lionel Ducos
Iain Duff
Lee Duhem
Martin Dunstan
Brian Dupee
Dominique Duval
Robert Edwards
Heow Eide-Goodman
Lars Erickson
Mark Fahey
Richard Fateman
Bertfried Fauser
Stuart Feldman
John Fletcher
Brian Ford
Albrecht Fortenbacher
George Frances
Constantine Frangos
Timothy Freeman
Korrinn Fu
Marc Gaetano
Rudiger Gebauer
Van de Geijn
Kathy Gerber
Patricia Gianni
Gustavo Goertkin
Samantha Goldrich
Holger Gollan
Teresa Gomez-Diaz
Laureano Gonzalez-Vega
Stephen Gortler
Johannes Grabmeier
Matt Grayson
Klaus Ebbe Grue
James Griesmer
Vladimir Grinberg
Oswald Gschnitzer
Ming Gu
Jocelyn Guidry
Gaetan Hache
Steve Hague
Satoshi Hamaguchi
Sven Hammarling
Mike Hansen
Richard Hanson
Richard Harke
Bill Hart
Vilya Harvey
Martin Hassner
Arthur S. Hathaway
Dan Hatton
Waldek Hebisch
Karl Hegbloom
Ralf Hemmecke
Henderson
Antoine Hersen
Nicholas J. Higham
Roger House
Gernot Hueber
Pietro Iglio
Alejandro Jakubi
Richard Jenks
Bo Kagstrom
William Kahan
Kyriakos Kalorkoti
Kai Kaminski
Grant Keady
Wilfrid Kendall
Tony Kennedy
David Kincaid

Ted Kosan
Paul Kosinski
Igor Kozachenko
Fred Krogh
Klaus Kusche
Bernhard Kutzler
Tim Lahey
Larry Lambe
Kaj Laurson
Charles Lawson
George L. Legendre
Franz Lehner
Frederic Lehobey
Michel Levaud
Howard Levy
J. Lewis
Ren-Cang Li
Rudiger Loos
Craig Lucas
Michael Lucks
Richard Luczak
Camm Maguire
Francois Maltey
Osni Marques
Alasdair McAndrew
Bob McElrath
Michael McGettrick
Edi Meier
Ian Meikle
David Mentre
Victor S. Miller
Gerard Milmeister
Mohammed Mobarak
H. Michael Moeller
Michael Monagan
Marc Moreno-Maza
Scott Morrison
Joel Moses
Mark Murray
William Naylor
Patrice Naudin
C. Andrew Neff
John Nelder
Godfrey Nolan
Arthur Norman
Jinzhong Niu
Michael O'Connor
Summat Oemrawsingh
Kostas Oikonomou
Humberto Ortiz-Zuazaga
Julian A. Padget
Bill Page
David Parnas
Susan Pelzel
Michel Petitot
Didier Pinchon
Ayal Pinkus
Frederick H. Pitts
Jose Alfredo Portes
E. Quintana-Orti
Gregorio Quintana-Orti
Beresford Parlett
A. Petitet
Peter Poromaa
Claude Quitte
Arthur C. Ralfs
Norman Ramsey
Anatoly Raportirenko
Guilherme Reis
Huan Ren
Albert D. Rich
Michael Richardson
Jason Riedy
Renaud Rioboo
Jean Rivlin
Nicolas Robidoux
Simon Robinson
Raymond Rogers
Michael Rothstein
Martin Rubey
Jeff Rutter
Philip Santas
Alfred Scheerhorn
William Schelter
Gerhard Schneider
Martin Schoenert
Marshall Schor
Frithjof Schulze
Fritz Schwarz
Steven Segletes
V. Sima
Nick Simicich
William Sit
Elena Smirnova
Jacob Nyffeler Smith
Ken Stanley
Jonathan Steinbach
Fabio Stumbo
Christine Sundaresan
Robert Sutor
Moss E. Sweedler
Eugene Surowitz
Max Tegmark
T. Doug Telford
James Thatcher
Laurent Thery
Balbir Thomas
Mike Thomas
Dylan Thurston
Francoise Tisseur
Steve Toleque
Raymond Toy
Barry Trager
Themos T. Tsikas
Gregory Vanuxem
Kresimir Veselic
Christof Voemel
Bernhard Wall
Stephen Watt
Jaap Weel
Juergen Weiss
M. Weller
Mark Wegman
James Wen
Thorsten Werther
Michael Wester
R. Clint Whaley
James T. Wheeler
John M. Wiley
Berhard Will
Clifton J. Williamson
Stephen Wilson
Shmuel Winograd
Robert Wisbauer
Sandra Wityak
Waldemar Wiwianka
Knut Wolf
Yanyang Xiao
Liu Xiaojun
Clifford Yapp
David Yun
Qian Yun
Vadim Zhytnikov
Richard Zippel
Evelyn Zoernack
Bruno Zuercher
Dan Zwillinger

Contents

1 Numerical Analysis **1**

2 The Quality of Computed Solutions by Sven Hammarling **3**
2.1 Introduction . . . 3
2.2 Floating Point Numbers and IEEE Arithmetic . . . 4
2.2.1 Example 2.1 (Floating point numbers) . . . 4
2.3 Why Worry about Computed Solutions? . . . 6
2.3.1 Example 3.1 (Cancellation) . . . 6
2.3.2 Example 3.2 (Sample variance [High02],Section 1.9) . . . 7
2.3.3 Example 3.3 (Hypotenuse) . . . 8
2.3.4 Example 3.4 (Modulus of a complex number) . . . 9
2.3.5 Example 3.5 (Sample mean)[High98] . . . 10
2.4 Condition, Stability and Error Analysis . . . 11
2.4.1 Condition . . . 11
2.4.2 Stability . . . 18
2.4.3 Error Analysis . . . 23
2.5 Floating Point Error Analysis . . . 27
2.5.1 Example 5.1 (Floating point numbers) . . . 27
2.5.2 Example 5.2 (Product of values) . . . 28
2.5.3 Example 5.3 (Sum of values) . . . 29
2.5.4 Example 5.4 (Difference of two squares) . . . 29
2.5.5 Example 5.5 (The need for pivoting) . . . 31
2.6 Posing the Mathematical Problem . . . 34
2.7 Error Bounds and Software . . . 34
2.8 Other Approaches . . . 38
2.8.1 Example 8.1 (Cancellation and interval arithmetic) . . . 39
2.9 Summary . . . 39

3 Chapter Overview **41**

4 Optimizations **45**

5 Algebra Cover Code **47**
5.1 package BLAS1 BlasLevelOne . . . 47
5.1.1 Axiom Package unit tests . . . 47
5.1.2 Axiom help page . . . 81
5.1.3 BlasLevelOne (BLAS1 Spad code) . . . 82
5.2 lsame . . . 89
5.2.1 fortran code . . . 89
5.2.2 lisp code . . . 91
5.3 xerbla . . . 92
5.3.1 fortran code . . . 92
5.3.2 lisp code . . . 93

6 BLAS Level 1 Real Vector Operations **95**
6.1 sasum - the sum of the absolute values . . . 95
6.1.1 Axiom unit tests . . . 95
6.1.2 Axiom help page . . . 95
6.1.3 fortran code . . . 96
6.1.4 lisp code . . . 97
6.2 saxpy - constant times a vector plus a vector . . . 99
6.2.1 Axiom unit tests . . . 99
6.2.2 Axiom help page . . . 99
6.2.3 fortran code . . . 100
6.2.4 lisp code . . . 101
6.3 scabs1 - abs(real(z))+abs(imag(z)) of a complex number . . . 103
6.3.1 Axiom unit tests . . . 103
6.3.2 Axiom help page . . . 104
6.3.3 fortran code . . . 104
6.3.4 lisp code . . . 105
6.4 scasum - abs(real(z))+abs(imag(z)) of a complex vector . . . 105
6.4.1 Axiom unit tests . . . 105
6.4.2 Axiom help page . . . 106
6.4.3 fortran code . . . 106
6.4.4 lisp code . . . 107
6.5 scnrm2 - Euclidean norm of a vector . . . 109
6.5.1 Axiom unit tests . . . 109
6.5.2 Axiom help page . . . 109
6.5.3 fortran code . . . 110
6.5.4 lisp code . . . 111
6.6 scopy - copy a vector x to a vector y . . . 112
6.6.1 Axiom unit tests . . . 112
6.6.2 Axiom help page . . . 113
6.6.3 fortran code . . . 113
6.6.4 lisp code . . . 115
6.7 sdot - compute the dot product of two vectors . . . 117
6.7.1 Axiom unit tests . . . 117
6.7.2 Axiom help page . . . 117

6.7.3 fortran code . . . 118
6.7.4 lisp code . . . 119
6.8 sdsdot - compute the inner product of two vectors . . . 121
6.8.1 Axiom unit tests . . . 121
6.8.2 Axiom help page . . . 121
6.8.3 fortran code . . . 122
6.8.4 lisp code . . . 124
6.9 snrm2 - Euclidean norm of a vector $\sqrt{x'x}$ . . . 126
6.9.1 Axiom unit tests . . . 126
6.9.2 Axiom help page . . . 126
6.9.3 fortran code . . . 127
6.9.4 lisp code . . . 128
6.10 srot - applies a plane rotation . . . 129
6.10.1 Axiom unit tests . . . 129
6.10.2 Axiom help page . . . 130
6.10.3 fortran code . . . 130
6.10.4 lisp code . . . 131
6.11 srotg - construct givens plane rotation . . . 133
6.11.1 Axiom unit tests . . . 133
6.11.2 Axiom help page . . . 133
6.11.3 fortran code . . . 134
6.11.4 lisp code . . . 135
6.12 srotm - apply a modified givens transformation . . . 135
6.12.1 Axiom unit tests . . . 135
6.12.2 Axiom help page . . . 136
6.12.3 fortran code . . . 137
6.12.4 lisp code . . . 140
6.13 srotmg - apply a modified givens transformation . . . 143
6.13.1 Axiom unit tests . . . 143
6.13.2 Axiom help page . . . 143
6.13.3 fortran code . . . 144
6.13.4 lisp code . . . 148
6.14 sscal - scales a vector by a constant . . . 150
6.14.1 Axiom unit tests . . . 150
6.14.2 Axiom help page . . . 150
6.14.3 fortran code . . . 151
6.14.4 lisp code . . . 152
6.15 sswap - interchange two vectors . . . 154
6.15.1 Axiom unit tests . . . 154
6.15.2 Axiom help page . . . 154
6.15.3 fortran code . . . 155
6.15.4 lisp code . . . 156

7 BLAS Level 1 Double Vector Operations **159**
7.1 dasum - sum of the absolute values . . . 159
7.1.1 Axiom unit tests . . . 159
7.1.2 Axiom help page . . . 164
7.1.3 fortran code . . . 168
7.1.4 fortran unit test . . . 169
7.1.5 lisp code . . . 172
7.2 daxpy - constant time vector plus a vector . . . 172
7.2.1 Axiom unit tests . . . 172
7.2.2 Axiom help page . . . 176
7.2.3 fortran code . . . 180
7.2.4 fortran unit test . . . 181
7.2.5 lisp code . . . 184
7.3 dcabs1 - abs(real(z))+abs(imag(z)) . . . 185
7.3.1 Axiom unit tests . . . 185
7.3.2 Axiom help page . . . 187
7.3.3 fortran code . . . 188
7.3.4 fortran unit test . . . 189
7.3.5 lisp code . . . 190
7.4 dcopy - copy a vector x to a vector y . . . 190
7.4.1 Axiom unit tests . . . 190
7.4.2 Axiom help page . . . 194
7.4.3 fortran code . . . 196
7.4.4 fortran unit test . . . 198
7.4.5 lisp code . . . 200
7.5 ddot - dot product of two vectors . . . 202
7.5.1 Axiom unit tests . . . 202
7.5.2 Axiom help page . . . 203
7.5.3 fortran code . . . 205
7.5.4 fortran unit test . . . 207
7.5.5 lisp code . . . 209
7.6 dnrm2 - Euclidean norm of a vector . . . 210
7.6.1 Axiom unit tests . . . 210
7.6.2 Axiom help page . . . 211
7.6.3 fortran code . . . 212
7.6.4 fortran unit test . . . 214
7.6.5 lisp code . . . 215
7.7 drot - applies a plane rotation . . . 216
7.7.1 Axiom unit tests . . . 216
7.7.2 Axiom help page . . . 221
7.7.3 fortran code . . . 223
7.7.4 fortran unit test . . . 225
7.7.5 lisp code . . . 230
7.8 drotg - constructs givens plane rotation . . . 233
7.8.1 Axiom unit tests . . . 233
7.8.2 Axiom help page . . . 235

7.8.3 fortran code . . . 239
7.8.4 fortran unit test . . . 240
7.8.5 lisp code . . . 244
7.9 drotm - Apply the modified givens transformation to DX^T . . . 246
7.9.1 Axiom unit tests . . . 246
7.9.2 Axiom help page . . . 246
7.9.3 fortran code . . . 248
7.9.4 lisp code . . . 250
7.10 drotmg - Construct the modified givens transformation matrix . . . 253
7.10.1 Axiom unit tests . . . 253
7.10.2 Axiom help page . . . 253
7.10.3 fortran code . . . 255
7.10.4 lisp code . . . 258
7.11 dscal - scale a vector by a constant . . . 260
7.11.1 Axiom unit tests . . . 260
7.11.2 Axiom help page . . . 262
7.11.3 fortran code . . . 264
7.11.4 fortran unit test . . . 265
7.11.5 lisp code . . . 267
7.12 dsdot - compute the inner product of two vectors . . . 268
7.12.1 Axiom unit tests . . . 268
7.12.2 Axiom help page . . . 269
7.12.3 fortran code . . . 270
7.12.4 lisp code . . . 271
7.13 dswap - interchanges two vectors . . . 272
7.13.1 Axiom unit tests . . . 272
7.13.2 Axiom help page . . . 274
7.13.3 fortran code . . . 276
7.13.4 fortran unit test . . . 278
7.13.5 lisp code . . . 280
7.14 dtrsv - solve $Ax = b$ or $A^T x = b$ . . . 281
7.14.1 Axiom unit tests . . . 281
7.14.2 Axiom help page . . . 281
7.14.3 fortran code . . . 283
7.14.4 lisp code . . . 287
7.15 dzasum - sum of the dcabs1 of a complex vector . . . 295
7.15.1 Axiom unit tests . . . 295
7.15.2 Axiom help page . . . 296
7.15.3 fortran code . . . 297
7.15.4 fortran unit test . . . 298
7.15.5 lisp code . . . 300
7.16 dznrm2 - Euclidean norm of a vector = $\sqrt{x^H * x}$ . . . 301
7.16.1 Axiom unit tests . . . 301
7.16.2 Axiom help page . . . 302
7.16.3 fortran code . . . 304
7.16.4 fortran unit test . . . 306

7.16.5 lisp code . . . 307
7.17 icamax - maximum index function . . . 309
7.17.1 Axiom unit tests . . . 309
7.17.2 Axiom help page . . . 310
7.17.3 fortran code . . . 312
7.17.4 fortran unit test . . . 314
7.17.5 lisp code . . . 316
7.18 idamax - index of element having max. absolute value . . . 317
7.18.1 Axiom unit tests . . . 317
7.18.2 Axiom help page . . . 319
7.18.3 fortran code . . . 322
7.18.4 fortran unit test . . . 324
7.18.5 lisp code . . . 325
7.19 isamax - index of element having max. absolute value . . . 326
7.19.1 Axiom unit tests . . . 326
7.19.2 Axiom help page . . . 328
7.19.3 fortran code . . . 331
7.19.4 fortran unit test . . . 332
7.19.5 lisp code . . . 334
7.20 izamax . . . 335
7.20.1 Axiom unit tests . . . 335
7.20.2 Axiom help page . . . 336
7.20.3 fortran code . . . 338
7.20.4 fortran unit test . . . 339
7.20.5 lisp code . . . 341

8 BLAS Level 1 Complex Vector Operations 345
8.1 caxpy - constant times a vector plus a vector . . . 345
8.1.1 Axiom unit tests . . . 345
8.1.2 Axiom help page . . . 345
8.1.3 fortran code . . . 346
8.1.4 lisp code . . . 347
8.2 ccopy - copies a vector x to a vector y . . . 349
8.2.1 Axiom unit tests . . . 349
8.2.2 Axiom help page . . . 349
8.2.3 fortran code . . . 350
8.2.4 lisp code . . . 351
8.3 cdotc - dot product of two complex vectors $X^H Y$ . . . 352
8.3.1 Axiom unit tests . . . 352
8.3.2 Axiom help page . . . 352
8.3.3 fortran code . . . 353
8.3.4 lisp code . . . 354
8.4 cdotu - dot product of two complex vectors $X^T Y$ . . . 355
8.4.1 Axiom unit tests . . . 355
8.4.2 Axiom help page . . . 355
8.4.3 fortran code . . . 356

8.4.4 lisp code . . . 357
8.5 crotg - determines a complex Givens rotation . . . 358
8.5.1 Axiom unit tests . . . 358
8.5.2 Axiom help page . . . 359
8.5.3 fortran code . . . 359
8.5.4 lisp code . . . 360
8.6 cscal - scales a vector by a constant . . . 361
8.6.1 Axiom unit tests . . . 361
8.6.2 Axiom help page . . . 362
8.6.3 fortran code . . . 362
8.6.4 lisp code . . . 363
8.7 csrot - appies a plane rotation . . . 364
8.7.1 Axiom unit tests . . . 364
8.7.2 Axiom help page . . . 364
8.7.3 fortran code . . . 366
8.7.4 lisp code . . . 367
8.8 csscal - scales a complex vector by a real constant . . . 368
8.8.1 Axiom unit tests . . . 368
8.8.2 Axiom help page . . . 369
8.8.3 fortran code . . . 369
8.8.4 lisp code . . . 370
8.9 cswap - interchanges two vectors . . . 371
8.9.1 Axiom unit tests . . . 371
8.9.2 Axiom help page . . . 372
8.9.3 fortran code . . . 372
8.9.4 lisp code . . . 374

9 BLAS Level 1 Complex16 Vector Operations 377
9.1 zaxpy - constant times a vector plus a vector . . . 377
9.1.1 Axiom unit tests . . . 377
9.1.2 Axiom help page . . . 381
9.1.3 fortran code . . . 384
9.1.4 fortran unit test . . . 385
9.1.5 lisp code . . . 391
9.2 zcopy - copies a vector x to a vector y . . . 394
9.2.1 Axiom unit tests . . . 394
9.2.2 Axiom help page . . . 394
9.2.3 fortran code . . . 395
9.2.4 lisp code . . . 396
9.3 zdotc - dot product of two complex vectors $X^H Y$ . . . 397
9.3.1 Axiom unit tests . . . 397
9.3.2 Axiom help page . . . 398
9.3.3 fortran code . . . 399
9.3.4 lisp code . . . 401
9.4 zdotu - forms the dot product of two complex vectors $X^T Y$ . . . 402
9.4.1 Axiom unit tests . . . 402

9.4.2 Axiom help page . . . 402
9.4.3 fortran code . . . 404
9.4.4 lisp code . . . 405
9.5 zdrot - applies a plane rotation . . . 406
9.5.1 Axiom unit tests . . . 406
9.5.2 Axiom help page . . . 407
9.5.3 fortran code . . . 408
9.5.4 lisp code . . . 409
9.6 zdscal - scales a vector by a constant . . . 411
9.6.1 Axiom unit tests . . . 411
9.6.2 Axiom help page . . . 411
9.6.3 fortran code . . . 412
9.6.4 lisp code . . . 413
9.7 zrotg - determines a double complex Givens rotation . . . 414
9.7.1 Axiom unit tests . . . 414
9.7.2 Axiom help page . . . 414
9.7.3 fortran code . . . 417
9.7.4 lisp code . . . 418
9.8 zscal - scales a vector by a constant . . . 418
9.8.1 Axiom unit tests . . . 418
9.8.2 Axiom help page . . . 419
9.8.3 fortran code . . . 420
9.8.4 lisp code . . . 421
9.9 zswap - interchanges two vectors . . . 422
9.9.1 Axiom unit tests . . . 422
9.9.2 Axiom help page . . . 422
9.9.3 fortran code . . . 423
9.9.4 lisp code . . . 425

10 BLAS Level 2 Real Matrix-Vector Operations 427
10.1 sgbmv - $\alpha Ax + \beta y$ or $\alpha A^T x + \beta y$ . . . 427
10.1.1 Axiom unit tests . . . 427
10.1.2 Axiom help page . . . 427
10.1.3 fortran code . . . 430
10.1.4 lisp code . . . 434
10.2 sgemv - matrix-vector $\alpha Ax + \beta y$ or $\alpha A^T x + \beta y$ . . . 439
10.2.1 Axiom unit tests . . . 439
10.2.2 Axiom help page . . . 439
10.2.3 fortran code . . . 441
10.2.4 lisp code . . . 445
10.3 sger - rank 1 $\alpha xy^T + A$ . . . 449
10.3.1 Axiom unit tests . . . 449
10.3.2 Axiom help page . . . 449
10.3.3 fortran code . . . 451
10.3.4 lisp code . . . 453
10.4 ssbmv - $\alpha Ax + \beta y$ . . . 455

10.4.1 Axiom unit tests . . . 455
10.4.2 Axiom help page . . . 456
10.4.3 fortran code . . . 458
10.4.4 lisp code . . . 462
10.5 sspmv - matrix-vector $\alpha Ax + \beta y$ . . . 468
10.5.1 Axiom unit tests . . . 468
10.5.2 Axiom help page . . . 468
10.5.3 fortran code . . . 470
10.5.4 lisp code . . . 474
10.6 sspr - rank 1 $\alpha xx^T + A$ . . . 480
10.6.1 Axiom unit tests . . . 480
10.6.2 Axiom help page . . . 480
10.6.3 fortran code . . . 482
10.6.4 lisp code . . . 485
10.7 sspr2 - rank 2 $\alpha xy^T + \alpha yx^T + A$ . . . 488
10.7.1 Axiom unit tests . . . 488
10.7.2 Axiom help page . . . 488
10.7.3 fortran code . . . 490
10.7.4 lisp code . . . 493
10.8 ssymv - matrix-vector $\alpha Ax + \beta y$ . . . 498
10.8.1 Axiom unit tests . . . 498
10.8.2 Axiom help page . . . 498
10.8.3 fortran code . . . 500
10.8.4 lisp code . . . 504
10.9 ssyr - symmetric rank 1 $\alpha xx^T + A$ . . . 509
10.9.1 Axiom unit tests . . . 509
10.9.2 Axiom help page . . . 509
10.9.3 fortran code . . . 511
10.9.4 lisp code . . . 514
10.10 ssyr2 - $\alpha xy^T + \alpha yx^T + A$ . . . 517
10.10.1 Axiom unit tests . . . 517
10.10.2 Axiom help page . . . 517
10.10.3 fortran code . . . 519
10.10.4 lisp code . . . 523
10.11 stbmv - banded matrix-vector $x = A * x$ or $x = A^T x$ . . . 527
10.11.1 Axiom unit tests . . . 527
10.11.2 Axiom help page . . . 527
10.11.3 fortran code . . . 530
10.11.4 lisp code . . . 534
10.12 stbsv - banded solve system of equations $Ax = b$ or $A^T x = b$ . . . 542
10.12.1 Axiom unit tests . . . 542
10.12.2 Axiom help page . . . 542
10.12.3 fortran code . . . 545
10.12.4 lisp code . . . 550
10.13 stpmv - matrix-vector $x = A * x$ or $x = A^T x$ . . . 557
10.13.1 Axiom unit tests . . . 557

10.13.2 Axiom help page . . . 557
10.13.3 fortran code . . . 559
10.13.4 lisp code . . . 564
10.14 stpsv - solve systems of equations $Ax = b$ or $A^T x = b$ . . . 571
10.14.1 Axiom unit tests . . . 571
10.14.2 Axiom help page . . . 571
10.14.3 fortran code . . . 573
10.14.4 lisp code . . . 578
10.15 strmv - matrix-vector $x = Ax$ or $x = A^T x$ . . . 585
10.15.1 Axiom unit tests . . . 585
10.15.2 Axiom help page . . . 585
10.15.3 fortran code . . . 587
10.15.4 lisp code . . . 591
10.16 strsv - solve systems of equations $Ax = b$ or $A^T x = b$ . . . 597
10.16.1 Axiom unit tests . . . 597
10.16.2 Axiom help page . . . 598
10.16.3 fortran code . . . 600
10.16.4 lisp code . . . 604

11 BLAS Level 2 Double Matrix-Vector Operations 611
11.1 dgbmv - band matrix $\alpha Ax + \beta y$ or $\alpha A^T x + \beta y$ . . . 611
11.1.1 Axiom unit tests . . . 611
11.1.2 Axiom help page . . . 611
11.1.3 fortran code . . . 614
11.1.4 lisp code . . . 618
11.2 dgemv - $\alpha Ax + \beta y$ or $\alpha A^T x + \beta y$ . . . 624
11.2.1 Axiom unit tests . . . 624
11.2.2 Axiom help page . . . 624
11.2.3 fortran code . . . 626
11.2.4 lisp code . . . 630
11.3 dger - rank 1 $\alpha xy^T + a$ . . . 635
11.3.1 Axiom unit tests . . . 635
11.3.2 Axiom help page . . . 635
11.3.3 fortran code . . . 637
11.3.4 lisp code . . . 639
11.4 dsbmv - banded $\alpha Ax + \beta y$ . . . 641
11.4.1 Axiom unit tests . . . 641
11.4.2 Axiom help page . . . 641
11.4.3 fortran code . . . 644
11.4.4 lisp code . . . 648
11.5 dspmv - matrix-vector operation $\alpha Ax + \beta y$ . . . 655
11.5.1 Axiom unit tests . . . 655
11.5.2 Axiom help page . . . 655
11.5.3 fortran code . . . 657
11.5.4 lisp code . . . 661
11.6 dspr - symmetric rank 1 $\alpha xx^T + A$ . . . 667

11.6.1 Axiom unit tests . . . 667
11.6.2 Axiom help page . . . 668
11.6.3 fortran code . . . 669
11.6.4 lisp code . . . 672
11.7 dspr2 - symmetric rank 2 $\alpha xy^T + \alpha yx^T + A$ . . . 676
11.7.1 Axiom unit tests . . . 676
11.7.2 Axiom help page . . . 677
11.7.3 fortran code . . . 679
11.7.4 lisp code . . . 682
11.8 dsymv - matrix-vector operation $\alpha Ax + \beta y$ . . . 688
11.8.1 Axiom unit tests . . . 688
11.8.2 Axiom help page . . . 688
11.8.3 fortran code . . . 690
11.8.4 lisp code . . . 694
11.9 dsyr - rank 1 $\alpha xx^T + A$ . . . 700
11.9.1 Axiom unit tests . . . 700
11.9.2 Axiom help page . . . 700
11.9.3 fortran code . . . 702
11.9.4 lisp code . . . 705
11.10 dsyr2 - symmetric rank 2 $\alpha xy^T + \alpha yx^T + A$ . . . 709
11.10.1 Axiom unit tests . . . 709
11.10.2 Axiom help page . . . 709
11.10.3 fortran code . . . 711
11.10.4 lisp code . . . 714
11.11 dtbmv - triangular banded matrix $x = Ax$ or $x = A^T x$ . . . 720
11.11.1 Axiom unit tests . . . 720
11.11.2 Axiom help page . . . 720
11.11.3 fortran code . . . 723
11.11.4 lisp code . . . 727
11.12 dtbsv - triangular banded matrix $Ax = b$ or $A^T x = b$ . . . 736
11.12.1 Axiom unit tests . . . 736
11.12.2 Axiom help page . . . 737
11.12.3 fortran code . . . 740
11.12.4 lisp code . . . 744
11.13 dtpmv - matrix-vector $x = Ax$ or $x = A^T x$ . . . 753
11.13.1 Axiom unit tests . . . 753
11.13.2 Axiom help page . . . 754
11.13.3 fortran code . . . 756
11.13.4 lisp code . . . 760
11.14 dtpsv - solve systems of equations $Ax = b$ or $A^T x = b$ . . . 769
11.14.1 Axiom unit tests . . . 769
11.14.2 Axiom help page . . . 770
11.14.3 fortran code . . . 772
11.14.4 lisp code . . . 776
11.15 dtrmv - matrix-vector $x = Ax$ or $x = A^T x$ . . . 785
11.15.1 Axiom unit tests . . . 785

11.15.2 Axiom help page . . . 786
11.15.3 fortran code . . . 788
11.15.4 lisp code . . . 792
11.16 dtrsv - solve system of equations $Ax = b$ or $A^{'} = b$ . . . 800
11.16.1 Axiom unit tests . . . 800
11.16.2 Axiom help page . . . 800
11.16.3 fortran code . . . 802
11.16.4 lisp code . . . 806

12 BLAS Level 2 Complex Matrix-Vector Operations 815
12.1 cgbmv - band mat-vec $\alpha Ax + \beta y$ or $\alpha A^T + \beta y$ or $\alpha A^H x + \beta y$ . . . 815
12.1.1 Axiom unit tests . . . 815
12.1.2 Axiom help page . . . 815
12.1.3 fortran code . . . 818
12.1.4 lisp code . . . 822
12.2 cgemv - matrix-vector $\alpha Ax + \beta y$ or $\alpha A^T + \beta y$ or $\alpha A^H x + \beta y$ . . . 829
12.2.1 Axiom unit tests . . . 829
12.2.2 Axiom help page . . . 830
12.2.3 fortran code . . . 832
12.2.4 lisp code . . . 836
12.3 cgerc - rank 1 $\alpha xy^H + A$ . . . 841
12.3.1 Axiom unit tests . . . 841
12.3.2 Axiom help page . . . 841
12.3.3 fortran code . . . 843
12.3.4 lisp code . . . 845
12.4 cgeru - rank 1 $\alpha xy^T + A$ . . . 847
12.4.1 Axiom unit tests . . . 847
12.4.2 Axiom help page . . . 848
12.4.3 fortran code . . . 849
12.4.4 lisp code . . . 851
12.5 chbmv - hermitian band matrix-vector $\alpha Ax + \beta y$ . . . 854
12.5.1 Axiom unit tests . . . 854
12.5.2 Axiom help page . . . 854
12.5.3 fortran code . . . 857
12.5.4 lisp code . . . 861
12.6 chemv - hermitian matrix-vector $\alpha Ax + \beta y$ . . . 867
12.6.1 Axiom unit tests . . . 867
12.6.2 Axiom help page . . . 867
12.6.3 fortran code . . . 869
12.6.4 lisp code . . . 873
12.7 cher - hermitian rank 1 $\alpha xx^H + A$ . . . 879
12.7.1 Axiom unit tests . . . 879
12.7.2 Axiom help page . . . 879
12.7.3 fortran code . . . 881
12.7.4 lisp code . . . 884
12.8 cher2 - hermitian rank 2 $\alpha xy^H + conjg(\alpha)yx^H + A$ . . . 889

12.8.1 Axiom unit tests . . . 889
12.8.2 Axiom help page . . . 890
12.8.3 fortran code . . . 892
12.8.4 lisp code . . . 895
12.9 chpmv - hermitian matrix-vector $\alpha Ax + \beta y$ . . . 902
12.9.1 Axiom unit tests . . . 902
12.9.2 Axiom help page . . . 902
12.9.3 fortran code . . . 904
12.9.4 lisp code . . . 908
12.10 chpr - hermitian $\alpha xx^H + A$ . . . 914
12.10.1 Axiom unit tests . . . 914
12.10.2 Axiom help page . . . 915
12.10.3 fortran code . . . 916
12.10.4 lisp code . . . 920
12.11 chpr2 - hermitian rank 2 $\alpha xy^H + conjg(\alpha)yx^H + A$ . . . 925
12.11.1 Axiom unit tests . . . 925
12.11.2 Axiom help page . . . 926
12.11.3 fortran code . . . 928
12.11.4 lisp code . . . 931
12.12 ctbmv - banded matrix-vector Ax or A^Tx or A^Hx . . . 938
12.12.1 Axiom unit tests . . . 938
12.12.2 Axiom help page . . . 939
12.12.3 fortran code . . . 942
12.12.4 lisp code . . . 947
12.13 ctbsv - banded solved systems of equations $Ax = b$ or $A^Tx = b$ or $A^Hx = b$ . . . 958
12.13.1 Axiom unit tests . . . 958
12.13.2 Axiom help page . . . 958
12.13.3 fortran code . . . 961
12.13.4 lisp code . . . 966
12.14 ctpmv - packed matrix-vector Ax or A^Tx or A^Hx . . . 978
12.14.1 Axiom unit tests . . . 978
12.14.2 Axiom help page . . . 978
12.14.3 fortran code . . . 980
12.14.4 lisp code . . . 985
12.15 ctpsv - solve system of equations $Ax = b$ or $A^Tx = b$ or $A^Hx = b$ . . . 996
12.15.1 Axiom unit tests . . . 996
12.15.2 Axiom help page . . . 996
12.15.3 fortran code . . . 998
12.15.4 lisp code . . . 1003
12.16 ctrmv - matrix-vector Ax or A^Tx or A^Hx . . . 1014
12.16.1 Axiom unit tests . . . 1014
12.16.2 Axiom help page . . . 1014
12.16.3 fortran code . . . 1016
12.16.4 lisp code . . . 1021
12.17 ctrsv - solve systems of equations $Ax = b$ or $A^Tx = b$ or $A^Hx = b$ . . . 1031
12.17.1 Axiom unit tests . . . 1031

12.17.2 Axiom help page . . . 1031
12.17.3 fortran code . . . 1033
12.17.4 lisp code . . . 1038

13 BLAS Level 2 Complex16 Matrix-Vector Operations 1049
13.1 zgbmv - banded $\alpha Ax + \beta y$ or $\alpha A^T x + \beta y$ or $\alpha A^H x + \beta y$ . . . 1049
13.1.1 Axiom unit tests . . . 1049
13.1.2 Axiom help page . . . 1049
13.1.3 fortran code . . . 1052
13.1.4 lisp code . . . 1056
13.2 zgemv - matrix-vector $\alpha Ax + \beta y$ or $\alpha A^T x + \beta y$ or $\alpha A^H x + \beta y$ . . . 1063
13.2.1 Axiom unit tests . . . 1063
13.2.2 Axiom help page . . . 1064
13.2.3 fortran code . . . 1066
13.2.4 lisp code . . . 1070
13.3 zgerc - rank 1 $\alpha xy^H + A$ . . . 1075
13.3.1 Axiom unit tests . . . 1075
13.3.2 Axiom help page . . . 1076
13.3.3 fortran code . . . 1077
13.3.4 lisp code . . . 1079
13.4 zgeru - rank 1 $\alpha xy^T + A$ . . . 1082
13.4.1 Axiom unit tests . . . 1082
13.4.2 Axiom help page . . . 1082
13.4.3 fortran code . . . 1084
13.4.4 lisp code . . . 1086
13.5 zhbmv - hermitian band matrix-vector $\alpha Ax + \beta y$ . . . 1089
13.5.1 Axiom unit tests . . . 1089
13.5.2 Axiom help page . . . 1089
13.5.3 fortran code . . . 1092
13.5.4 lisp code . . . 1096
13.6 zhemv - hermitian matrix-vector $\alpha Ax + \beta y$ . . . 1102
13.6.1 Axiom unit tests . . . 1102
13.6.2 Axiom help page . . . 1103
13.6.3 fortran code . . . 1105
13.6.4 lisp code . . . 1109
13.7 zher - hermitian rank 1 $\alpha xx^H + A$ . . . 1115
13.7.1 Axiom unit tests . . . 1115
13.7.2 Axiom help page . . . 1115
13.7.3 fortran code . . . 1117
13.7.4 lisp code . . . 1120
13.8 zher2 - hermitian rank 2 $\alpha xy^H + conjg(\alpha)yx^H + A$ . . . 1127
13.8.1 Axiom unit tests . . . 1127
13.8.2 Axiom help page . . . 1127
13.8.3 fortran code . . . 1129
13.8.4 lisp code . . . 1133
13.9 zhpmv - hermitian matrix-vector $\alpha Ax + \beta y$ . . . 1141

13.9.1 Axiom unit tests . . . 1141
13.9.2 Axiom help page . . . 1142
13.9.3 fortran code . . . 1144
13.9.4 lisp code . . . 1148
13.10 zhpr - hermitian rank 1 $\alpha x x^H + A$ . . . 1154
13.10.1 Axiom unit tests . . . 1154
13.10.2 Axiom help page . . . 1155
13.10.3 fortran code . . . 1156
13.10.4 lisp code . . . 1160
13.11 zhpr2 - hermitian rank 2 $\alpha x y^H + conjg(\alpha) y x^H + A$ . . . 1167
13.11.1 Axiom unit tests . . . 1167
13.11.2 Axiom help page . . . 1167
13.11.3 fortran code . . . 1169
13.11.4 lisp code . . . 1173
13.12 ztbmv - band matrix-vector Ax or $A^T x$ or $A^H x$ . . . 1182
13.12.1 Axiom unit tests . . . 1182
13.12.2 Axiom help page . . . 1182
13.12.3 fortran code . . . 1185
13.12.4 lisp code . . . 1190
13.13 ztbsv - solve equations $Ax = b$ or $A^T x = b$ or $A^H x = b$ . . . 1202
13.13.1 Axiom unit tests . . . 1202
13.13.2 Axiom help page . . . 1203
13.13.3 fortran code . . . 1205
13.13.4 lisp code . . . 1211
13.14 ztpmv - matrix-vector Ax or $A^T x$ or $A^H x$ . . . 1223
13.14.1 Axiom unit tests . . . 1223
13.14.2 Axiom help page . . . 1223
13.14.3 fortran code . . . 1225
13.14.4 lisp code . . . 1230
13.15 ztpsv - solves systems of equations $Ax = b$ or $A^T x = b$ or $A^H x = b$ . . . 1242
13.15.1 Axiom unit tests . . . 1242
13.15.2 Axiom help page . . . 1242
13.15.3 fortran code . . . 1245
13.15.4 lisp code . . . 1250
13.16 ztrmv - triangular matrix-vector Ax or $A^T x$ or $A^H x$ . . . 1262
13.16.1 Axiom unit tests . . . 1262
13.16.2 Axiom help page . . . 1262
13.16.3 fortran code . . . 1264
13.16.4 lisp code . . . 1269
13.17 ztrsv - triangular solve system of equations $Ax = b$ or $A^T x = b$ or $A^H x = b$ . . . 1280
13.17.1 Axiom unit tests . . . 1280
13.17.2 Axiom help page . . . 1280
13.17.3 fortran code . . . 1282
13.17.4 lisp code . . . 1287

14 BLAS Level 3 Real Matrix-Matrix Operations **1299**
14.1 sgemm - $\alpha op(A)op(B) + \beta C$, $op(X) = X$ or $op(X) = X^T$ . . . 1299
14.1.1 Axiom unit tests . . . 1299
14.1.2 Axiom help page . . . 1299
14.1.3 fortran code . . . 1302
14.1.4 lisp code . . . 1306
14.2 ssymm - matrix-matrix $\alpha AB + \beta C$ or $\alpha BA + \beta C$ . . . 1311
14.2.1 Axiom unit tests . . . 1311
14.2.2 Axiom help page . . . 1312
14.2.3 fortran code . . . 1315
14.2.4 lisp code . . . 1318
14.3 ssyr2k - rank2k $\alpha AB^T + \alpha BA^T + \beta C$ or $\alpha A^T B + \alpha B^T A + \beta C$ . . . 1324
14.3.1 Axiom unit tests . . . 1324
14.3.2 Axiom help page . . . 1325
14.3.3 fortran code . . . 1328
14.3.4 lisp code . . . 1332
14.4 ssyrk - symmetric rank k $\alpha AA^T + \beta C$ or $\alpha A^T A + \beta C$ . . . 1338
14.4.1 Axiom unit tests . . . 1338
14.4.2 Axiom help page . . . 1339
14.4.3 fortran code . . . 1341
14.4.4 lisp code . . . 1345
14.5 strmm - $\alpha op(A)B$ or $\alpha Bop(A)$, $op(A) = A$ or $op(A) = A^T$ . . . 1351
14.5.1 Axiom unit tests . . . 1351
14.5.2 Axiom help page . . . 1352
14.5.3 fortran code . . . 1354
14.5.4 lisp code . . . 1359
14.6 strsm - $op(A)X = \alpha B$ or $Xop(A) = \alpha B$, $op(A) = A$ or $op(A) = A^T$ . . . 1368
14.6.1 Axiom unit tests . . . 1368
14.6.2 Axiom help page . . . 1368
14.6.3 fortran code . . . 1371
14.6.4 lisp code . . . 1376

15 BLAS Level 3 Double Matrix-Matrix Operations **1387**
15.1 dgemm - $\alpha op(A)op(B) + \beta C$, $op(X) = X$ or $op(X) = X^T$ . . . 1387
15.1.1 Axiom unit tests . . . 1387
15.1.2 Axiom help page . . . 1387
15.1.3 fortran code . . . 1390
15.1.4 lisp code . . . 1394
15.2 dsymm - $\alpha AB + \beta C$ or $\alpha BA + \beta C$ . . . 1400
15.2.1 Axiom unit tests . . . 1400
15.2.2 Axiom help page . . . 1401
15.2.3 fortran code . . . 1404
15.2.4 lisp code . . . 1408
15.3 dsyr2k - $\alpha AB^T + \alpha BA^T + \beta C$ or $\alpha A^T B + \alpha B^T A + \beta C$ . . . 1415
15.3.1 Axiom unit tests . . . 1415
15.3.2 Axiom help page . . . 1415

15.3.3 fortran code . . . 1418
15.3.4 lisp code . . . 1422
15.4 dsyrk - symmetric rank k $\alpha AA^T + \beta C$ or $\alpha A^T A + \beta C$ . . . 1430
15.4.1 Axiom unit tests . . . 1430
15.4.2 Axiom help page . . . 1430
15.4.3 fortran code . . . 1433
15.4.4 lisp code . . . 1437
15.5 dtrmm - $\alpha op(A)B$ or $\alpha Bop(A)$, $op(A) = A$ or $op(X) = A^T$ . . . 1444
15.5.1 Axiom unit tests . . . 1444
15.5.2 Axiom help page . . . 1444
15.5.3 fortran code . . . 1447
15.5.4 lisp code . . . 1452
15.6 dtrsm - $op(A)X = \alpha B$ or $Xop(A) = \alpha B$, $op(A) = A$ or $op(A) = A^T$ . . . 1462
15.6.1 Axiom unit tests . . . 1462
15.6.2 Axiom help page . . . 1462
15.6.3 fortran code . . . 1465
15.6.4 lisp code . . . 1470

16 BLAS Level 3 Complex Matrix-Matrix Operations 1483
16.1 cgemm - $\alpha op(A)op(B) + \beta C$, $op(X) = X$ or $op(X) = X^T$ or $op(X) = X^H$ . . . 1483
16.1.1 Axiom unit tests . . . 1483
16.1.2 Axiom help page . . . 1483
16.1.3 fortran code . . . 1486
16.1.4 lisp code . . . 1492
16.2 chemm - hermitian $\alpha AB + \beta C$ or $\alpha BA + \beta C$ . . . 1502
16.2.1 Axiom unit tests . . . 1502
16.2.2 Axiom help page . . . 1502
16.2.3 fortran code . . . 1505
16.2.4 lisp code . . . 1509
16.3 cher2k - hermitian rank 2 $\alpha AB^H + conjg(\alpha)BA^H + \beta C$ or $\alpha A^H B + conjg(\alpha)B^H A + \beta C$ . . . 1516
16.3.1 Axiom unit tests . . . 1516
16.3.2 Axiom help page . . . 1516
16.3.3 fortran code . . . 1519
16.3.4 lisp code . . . 1524
16.4 cherk - hermitian rank k $\alpha AA^H + \beta C$ or $\alpha A^H A + \beta C$ . . . 1535
16.4.1 Axiom unit tests . . . 1535
16.4.2 Axiom help page . . . 1535
16.4.3 fortran code . . . 1538
16.4.4 lisp code . . . 1542
16.5 csymm - symmetric $\alpha AB + \beta C$ or $\alpha BA + \beta C$ . . . 1552
16.5.1 Axiom unit tests . . . 1552
16.5.2 Axiom help page . . . 1552
16.5.3 fortran code . . . 1555
16.5.4 lisp code . . . 1559
16.6 csyr2k - symmetric rank 2k $\alpha AB^T + \alpha BA^T + \beta C$ or $\alpha A^T B + \alpha B^T A + \beta C$ . . . 1566

16.6.1 Axiom unit tests . . . 1566
16.6.2 Axiom help page . . . 1566
16.6.3 fortran code . . . 1569
16.6.4 lisp code . . . 1573
16.7 csyrk - symmetric $\alpha AA^T + \beta C$ or $\alpha A^T A + \beta C$ . . . 1580
16.7.1 Axiom unit tests . . . 1580
16.7.2 Axiom help page . . . 1581
16.7.3 fortran code . . . 1583
16.7.4 lisp code . . . 1587
16.8 ctrmm - $\alpha op(A)B$ or $\alpha Bop(A)$, $op(A) = A$ or $op(A) = A^T$ or $op(A) = A^H$ . . . 1594
16.8.1 Axiom unit tests . . . 1594
16.8.2 Axiom help page . . . 1594
16.8.3 fortran code . . . 1597
16.8.4 lisp code . . . 1602
16.9 ctrsm - triangular $op(A)X = \alpha B$ or $Xop(A) = \alpha B$, $op(A) = A$ or $op(A) = A^T$ or $op(A) = A^H$ . . . 1615
16.9.1 Axiom unit tests . . . 1615
16.9.2 Axiom help page . . . 1615
16.9.3 fortran code . . . 1618
16.9.4 lisp code . . . 1624

17 BLAS Level 3 Complex16 Matrix-Matrix Operations 1639
17.1 zgemm - $\alpha op(A)op(B) + \beta C$, $op(X) = X$ or $op(X) = X^T$ or $op(X) = X^H$ . . . 1639
17.1.1 Axiom unit tests . . . 1639
17.1.2 Axiom help page . . . 1639
17.1.3 fortran code . . . 1642
17.1.4 lisp code . . . 1648
17.2 zhemm - hermitian $\alpha AB + \beta C$ or $\alpha BA + \beta C$ . . . 1659
17.2.1 Axiom unit tests . . . 1659
17.2.2 Axiom help page . . . 1659
17.2.3 fortran code . . . 1662
17.2.4 lisp code . . . 1666
17.3 zher2k - hermitian rank 2k $\alpha AB^H + conjg(\alpha)BA^H + \beta C$ or $\alpha A^H B + conjg(\alpha)B^H A + \beta C$ . . . 1673
17.3.1 Axiom unit tests . . . 1673
17.3.2 Axiom help page . . . 1674
17.3.3 fortran code . . . 1677
17.3.4 lisp code . . . 1682
17.4 zherk - hermitian rank k $\alpha AA^H + \beta C$ or $\alpha A^H A + \beta C$ . . . 1694
17.4.1 Axiom unit tests . . . 1694
17.4.2 Axiom help page . . . 1694
17.4.3 fortran code . . . 1697
17.4.4 lisp code . . . 1702
17.5 zsymm - symmetric $\alpha AB + \beta C$ or $\alpha BA + \beta C$ . . . 1712
17.5.1 Axiom unit tests . . . 1712
17.5.2 Axiom help page . . . 1713

17.5.3 fortran code . . . 1716
17.5.4 lisp code . . . 1719
17.6 zsyr2k - symmetric rank 2k $\alpha AB^T + \alpha BA^T + \beta C$ or $\alpha A^T B + \alpha B^T A + \beta C$. 1727
17.6.1 Axiom unit tests . . . 1727
17.6.2 Axiom help page . . . 1727
17.6.3 fortran code . . . 1730
17.6.4 lisp code . . . 1734
17.7 zsyrk - symmetric rank k $\alpha AA^T + \beta C$ or $\alpha A^T A + \beta C$ . . . 1742
17.7.1 Axiom unit tests . . . 1742
17.7.2 Axiom help page . . . 1742
17.7.3 fortran code . . . 1745
17.7.4 lisp code . . . 1749
17.8 ztrmm - triangular $\alpha op(A)B$ or $\alpha Bop(A)$, $op(A) = A$ or $op(A) = A^T$ or $op(A) = A^H$ . . . 1755
17.8.1 Axiom unit tests . . . 1755
17.8.2 Axiom help page . . . 1756
17.8.3 fortran code . . . 1758
17.8.4 lisp code . . . 1764
17.9 ztrsm - triangular $op(A)X = \alpha B$ $Xop(A) = \alpha B$, $op(A) = A$ or $op(A) = A^T$ or $op(A) = A^H$ . . . 1776
17.9.1 Axiom unit tests . . . 1776
17.9.2 Axiom help page . . . 1776
17.9.3 fortran code . . . 1779
17.9.4 lisp code . . . 1785

18 LAPACK Concepts 1799
18.1 Tridiagonal matrices [High86] . . . 1799
18.2 Condition Number [High86] . . . 1799
18.3 Euclidean Norm [Ahre15] . . . 1800

19 LAPACK Overview 1801
19.1 General Matrix . . . 1803
19.2 General Band Matrix . . . 1804
19.3 General Tridiagonal Matrix . . . 1804
19.4 Symmetric/Hermitian Positive Definite . . . 1804
19.5 Symmetric/Hermitian Positive Definite (packed storage) . . . 1804
19.6 Symmetric/Hermitian Positive Definite Band . . . 1805
19.7 Symmetric/Hermitian Positive Definite Tridiagonal . . . 1805
19.8 Symmetric/Hermitian Indefinite Matrix . . . 1805
19.9 Complex Symmetric Matrix . . . 1805
19.10 Symmetric/Hermitian Indefinite Matrix (packed storage) . . . 1806
19.11 Complex Symmetric (packed storage) Matrix . . . 1806
19.12 Triangular Matrix . . . 1806
19.13 Triangular Matrix (packed storage) . . . 1806
19.14 Triangular Band Matrix . . . 1807

20 LAPACK - General Matrices, Linear Solve, Double **1809**
20.1 dgels . . . 1809
20.1.1 Axiom unit tests . . . 1809
20.1.2 Axiom help page . . . 1809
20.1.3 fortran code . . . 1812
20.1.4 lisp code . . . 1819
20.2 dgelsd . . . 1829
20.2.1 Axiom unit tests . . . 1829
20.2.2 Axiom help page . . . 1830
20.2.3 fortran code . . . 1833
20.2.4 lisp code . . . 1841
20.3 dgelss . . . 1865
20.3.1 Axiom unit tests . . . 1865
20.3.2 Axiom help page . . . 1865
20.3.3 fortran code . . . 1868
20.3.4 lisp code . . . 1879
20.4 dgelsx - DEPRECATED. Use dgelsy . . . 1907
20.5 dgelsy . . . 1907
20.5.1 Axiom unit tests . . . 1907
20.5.2 Axiom help page . . . 1907
20.5.3 fortran code . . . 1910
20.5.4 lisp code . . . 1916
20.6 dgesv . . . 1926
20.6.1 Axiom unit tests . . . 1926
20.6.2 Axiom help page . . . 1927
20.6.3 fortran code . . . 1929
20.6.4 lisp code . . . 1930
20.7 dgesvx . . . 1930
20.7.1 Axiom unit tests . . . 1930
20.7.2 Axiom help page . . . 1931
20.7.3 fortran code . . . 1936
20.7.4 lisp code . . . 1941
20.8 dgesvxx . . . 1951
20.8.1 Axiom unit tests . . . 1951
20.8.2 Axiom help page . . . 1951
20.8.3 fortran code . . . 1960
20.8.4 lisp code . . . 1965
20.9 dsgesv . . . 1973
20.9.1 Axiom unit tests . . . 1973
20.9.2 Axiom help page . . . 1974
20.9.3 fortran code . . . 1977
20.9.4 lisp code . . . 1982

21 LAPACK - General Matrices, Linear Solve, Real **1989**
21.1 sgels . . . 1989
21.1.1 Axiom unit tests . . . 1989
21.1.2 Axiom help page . . . 1989
21.1.3 fortran code . . . 1992
21.1.4 lisp code . . . 1999
21.2 sgelsd . . . 2009
21.2.1 Axiom unit tests . . . 2009
21.2.2 Axiom help page . . . 2010
21.2.3 fortran code . . . 2013
21.2.4 lisp code . . . 2021
21.3 sgelss . . . 2044
21.3.1 Axiom unit tests . . . 2044
21.3.2 Axiom help page . . . 2045
21.3.3 fortran code . . . 2047
21.3.4 lisp code . . . 2059
21.4 sgelsx - DEPRECATED use sgelsy . . . 2086
21.5 sgelsy . . . 2086
21.5.1 Axiom unit tests . . . 2086
21.5.2 Axiom help page . . . 2086
21.5.3 fortran code . . . 2089
21.5.4 lisp code . . . 2095
21.6 sgesv . . . 2106
21.6.1 Axiom unit tests . . . 2106
21.6.2 Axiom help page . . . 2106
21.6.3 fortran code . . . 2108
21.6.4 lisp code . . . 2109
21.7 sgesvx . . . 2111
21.7.1 Axiom unit tests . . . 2111
21.7.2 Axiom help page . . . 2111
21.7.3 fortran code . . . 2117
21.7.4 lisp code . . . 2122
21.8 sgesvxx . . . 2132
21.8.1 Axiom unit tests . . . 2132
21.8.2 Axiom help page . . . 2132
21.8.3 fortran code . . . 2141
21.8.4 lisp code . . . 2146

22 LAPACK - General Matrices, Linear Solve, Complex **2155**
22.1 cgels . . . 2155
22.1.1 Axiom unit tests . . . 2155
22.1.2 Axiom help page . . . 2155
22.1.3 fortran code . . . 2158
22.1.4 lisp code . . . 2165
22.2 cgelsd . . . 2175
22.2.1 Axiom unit tests . . . 2175

22.2.2 Axiom help page . . . 2176
22.2.3 fortran code . . . 2179
22.2.4 lisp code . . . 2188
22.3 cgelss . . . 2211
22.3.1 Axiom unit tests . . . 2211
22.3.2 Axiom help page . . . 2211
22.3.3 fortran code . . . 2214
22.3.4 lisp code . . . 2226
22.4 cgelsx - DEPRECATED use cgelsy . . . 2254
22.5 cgelsy . . . 2254
22.5.1 Axiom unit tests . . . 2254
22.5.2 Axiom help page . . . 2255
22.5.3 fortran code . . . 2258
22.5.4 lisp code . . . 2263
22.6 cgesv . . . 2274
22.6.1 Axiom unit tests . . . 2274
22.6.2 Axiom help page . . . 2274
22.6.3 fortran code . . . 2276
22.6.4 lisp code . . . 2277
22.7 cgesvx . . . 2279
22.7.1 Axiom unit tests . . . 2279
22.7.2 Axiom help page . . . 2279
22.7.3 fortran code . . . 2285
22.7.4 lisp code . . . 2290
22.8 cgesvxx . . . 2300
22.8.1 Axiom unit tests . . . 2300
22.8.2 Axiom help page [Demm05] . . . 2300
22.8.3 fortran code . . . 2309
22.8.4 lisp code . . . 2314

23 LAPACK - General Matrices, Linear Solve, Complex16 2323
23.1 zcgesv . . . 2323
23.1.1 Axiom unit tests . . . 2323
23.1.2 Axiom help page . . . 2323
23.1.3 fortran code . . . 2326
23.1.4 lisp code . . . 2332
23.2 zgels . . . 2339
23.2.1 Axiom unit tests . . . 2339
23.2.2 Axiom help page . . . 2340
23.2.3 fortran code . . . 2342
23.2.4 lisp code . . . 2349
23.3 zgelsd . . . 2360
23.3.1 Axiom unit tests . . . 2360
23.3.2 Axiom help page . . . 2360
23.3.3 fortran code . . . 2363
23.3.4 lisp code . . . 2372

23.4 zgelss . . . 2395
23.4.1 Axiom unit tests . . . 2395
23.4.2 Axiom help page . . . 2395
23.4.3 fortran code . . . 2398
23.4.4 lisp code . . . 2410
23.5 zgelsx - DEPRECATED use zgelsy . . . 2438
23.6 zgelsy . . . 2438
23.6.1 Axiom unit tests . . . 2438
23.6.2 Axiom help page . . . 2438
23.6.3 fortran code . . . 2441
23.6.4 lisp code . . . 2447
23.7 zgesv . . . 2458
23.7.1 Axiom unit tests . . . 2458
23.7.2 Axiom help page . . . 2458
23.7.3 fortran code . . . 2460
23.7.4 lisp code . . . 2461
23.8 zgesvx . . . 2463
23.8.1 Axiom unit tests . . . 2463
23.8.2 Axiom help page . . . 2463
23.8.3 fortran code . . . 2468
23.8.4 lisp code . . . 2474
23.9 zgesvxx . . . 2484
23.9.1 Axiom unit tests . . . 2484
23.9.2 Axiom help page [Demm05] . . . 2484
23.9.3 fortran code . . . 2493
23.9.4 lisp code . . . 2498

24 LAPACK - General Matrices, Eigenvalue , Double 2507
24.1 dgees . . . 2507
24.1.1 Axiom unit tests . . . 2507
24.1.2 Axiom help page . . . 2507
24.1.3 fortran code . . . 2511
24.1.4 lisp code . . . 2517
24.2 dgeesx . . . 2517
24.2.1 Axiom unit tests . . . 2517
24.2.2 Axiom help page . . . 2518
24.2.3 fortran code . . . 2522
24.2.4 lisp code . . . 2530
24.3 dgeev . . . 2530
24.3.1 Axiom unit tests . . . 2530
24.3.2 Axiom help page . . . 2530
24.3.3 fortran code . . . 2533
24.3.4 lisp code . . . 2540
24.4 dgeevx . . . 2540
24.4.1 Axiom unit tests . . . 2540
24.4.2 Axiom help page . . . 2541

24.4.3 fortran code . . . 2545
24.4.4 lisp code . . . 2553
24.5 dgegs - DEPRECATED use dgges . . . 2553
24.6 dgegv - DEPRECATED use dggev . . . 2553
24.7 dgges . . . 2553
24.7.1 Axiom unit tests . . . 2553
24.7.2 Axiom help page . . . 2554
24.7.3 fortran code . . . 2558
24.7.4 lisp code . . . 2566
24.8 dgges3 . . . 2567
24.8.1 Axiom unit tests . . . 2567
24.8.2 Axiom help page . . . 2567
24.8.3 fortran code . . . 2571
24.8.4 lisp code . . . 2579
24.9 dggesx . . . 2580
24.9.1 Axiom unit tests . . . 2580
24.9.2 Axiom help page . . . 2580
24.9.3 fortran code . . . 2585
24.9.4 lisp code . . . 2595
24.10 dggev . . . 2595
24.10.1 Axiom unit tests . . . 2595
24.10.2 Axiom help page . . . 2595
24.10.3 fortran code . . . 2599
24.10.4 lisp code . . . 2606
24.11 dggev3 . . . 2621
24.11.1 Axiom unit tests . . . 2621
24.11.2 Axiom help page . . . 2621
24.11.3 fortran code . . . 2625
24.11.4 lisp code . . . 2632
24.12 dggevx . . . 2648
24.12.1 Axiom unit tests . . . 2648
24.12.2 Axiom help page . . . 2648
24.12.3 fortran code . . . 2654
24.12.4 lisp code . . . 2664

25 LAPACK - General Matrices, Eigenvalue , Real 2685
25.1 sgees . . . 2685
25.1.1 Axiom unit tests . . . 2685
25.1.2 Axiom help page . . . 2685
25.1.3 fortran code . . . 2689
25.1.4 lisp code . . . 2695
25.2 sgeesx . . . 2695
25.2.1 Axiom unit tests . . . 2695
25.2.2 Axiom help page . . . 2696
25.2.3 fortran code . . . 2700
25.2.4 lisp code . . . 2708

25.3 sgeev . . . 2708
25.3.1 Axiom unit tests . . . 2708
25.3.2 Axiom help page . . . 2708
25.3.3 fortran code . . . 2711
25.3.4 lisp code . . . 2718
25.4 sgeevx . . . 2733
25.4.1 Axiom unit tests . . . 2733
25.4.2 Axiom help page . . . 2733
25.4.3 fortran code . . . 2738
25.4.4 lisp code . . . 2746
25.5 sgegs - DEPRECATED use sgges . . . 2764
25.6 sgegv - DEPRECATED use sggev . . . 2764
25.7 sgges . . . 2764
25.7.1 Axiom unit tests . . . 2764
25.7.2 Axiom help page . . . 2764
25.7.3 fortran code . . . 2769
25.7.4 lisp code . . . 2777
25.8 sgges3 . . . 2777
25.8.1 Axiom unit tests . . . 2777
25.8.2 Axiom help page . . . 2777
25.8.3 fortran code . . . 2782
25.8.4 lisp code . . . 2790
25.9 sggesx . . . 2790
25.9.1 Axiom unit tests . . . 2790
25.9.2 Axiom help page . . . 2790
25.9.3 fortran code . . . 2796
25.9.4 lisp code . . . 2805
25.10 sggev . . . 2805
25.10.1 Axiom unit tests . . . 2805
25.10.2 Axiom help page . . . 2806
25.10.3 fortran code . . . 2809
25.10.4 lisp code . . . 2817
25.11 sggev3 . . . 2831
25.11.1 Axiom unit tests . . . 2831
25.11.2 Axiom help page . . . 2831
25.11.3 fortran code . . . 2835
25.11.4 lisp code . . . 2842
25.12 sggevx . . . 2857
25.12.1 Axiom unit tests . . . 2857
25.12.2 Axiom help page . . . 2858
25.12.3 fortran code . . . 2864
25.12.4 lisp code . . . 2874

26 LAPACK - General Matrices, Eigenvalue , Complex **2893**
26.1 cgees . . . 2893
26.1.1 Axiom unit tests . . . 2893
26.1.2 Axiom help page . . . 2893
26.1.3 fortran code . . . 2896
26.1.4 lisp code . . . 2901
26.2 cgeesx . . . 2901
26.2.1 Axiom unit tests . . . 2901
26.2.2 Axiom help page . . . 2902
26.2.3 fortran code . . . 2905
26.2.4 lisp code . . . 2910
26.3 cgeev . . . 2911
26.3.1 Axiom unit tests . . . 2911
26.3.2 Axiom help page . . . 2911
26.3.3 fortran code . . . 2914
26.3.4 lisp code . . . 2920
26.4 cgeevx . . . 2933
26.4.1 Axiom unit tests . . . 2933
26.4.2 Axiom help page . . . 2933
26.4.3 fortran code . . . 2938
26.4.4 lisp code . . . 2945
26.5 cgegs - DEPRECATED use cgges . . . 2961
26.6 cgegv - DEPRECATED use cggev . . . 2961
26.7 cgges . . . 2961
26.7.1 Axiom unit tests . . . 2961
26.7.2 Axiom help page . . . 2961
26.7.3 fortran code . . . 2965
26.7.4 lisp code . . . 2972
26.8 cgges3 . . . 2972
26.8.1 Axiom unit tests . . . 2972
26.8.2 Axiom help page . . . 2973
26.8.3 fortran code . . . 2977
26.8.4 lisp code . . . 2983
26.9 cggesx . . . 2984
26.9.1 Axiom unit tests . . . 2984
26.9.2 Axiom help page . . . 2984
26.9.3 fortran code . . . 2989
26.9.4 lisp code . . . 2997
26.10 cggev . . . 2997
26.10.1 Axiom unit tests . . . 2997
26.10.2 Axiom help page . . . 2998
26.10.3 fortran code . . . 3001
26.10.4 lisp code . . . 3008
26.11 cggev3 . . . 3021
26.11.1 Axiom unit tests . . . 3021
26.11.2 Axiom help page . . . 3022

26.11.3 fortran code . . . 3025
26.11.4 lisp code . . . 3032
26.12 cggevx . . . 3046
26.12.1 Axiom unit tests . . . 3046
26.12.2 Axiom help page . . . 3047
26.12.3 fortran code . . . 3052
26.12.4 lisp code . . . 3061

27 LAPACK - General Matrices, Eigenvalue , Complex16 3081
27.1 zgees . . . 3081
27.1.1 Axiom unit tests . . . 3081
27.1.2 Axiom help page . . . 3081
27.1.3 fortran code . . . 3084
27.1.4 lisp code . . . 3089
27.2 zgeesx . . . 3089
27.2.1 Axiom unit tests . . . 3089
27.2.2 Axiom help page . . . 3090
27.2.3 fortran code . . . 3093
27.2.4 lisp code . . . 3098
27.3 zgeev . . . 3099
27.3.1 Axiom unit tests . . . 3099
27.3.2 Axiom help page . . . 3099
27.3.3 fortran code . . . 3102
27.3.4 lisp code . . . 3108
27.4 zgeevx . . . 3124
27.4.1 Axiom unit tests . . . 3124
27.4.2 Axiom help page . . . 3124
27.4.3 fortran code . . . 3128
27.4.4 lisp code . . . 3136
27.5 zgegs - DEPRECATED use zgges . . . 3152
27.6 zgegv - DEPRECATED use zggev . . . 3152
27.7 zgges . . . 3152
27.7.1 Axiom unit tests . . . 3152
27.7.2 Axiom help page . . . 3152
27.7.3 fortran code . . . 3156
27.7.4 lisp code . . . 3163
27.8 zgges3 . . . 3163
27.8.1 Axiom unit tests . . . 3163
27.8.2 Axiom help page . . . 3164
27.8.3 fortran code . . . 3168
27.8.4 lisp code . . . 3174
27.9 zggesx . . . 3175
27.9.1 Axiom unit tests . . . 3175
27.9.2 Axiom help page . . . 3175
27.9.3 fortran code . . . 3180
27.9.4 lisp code . . . 3188

27.10zggev . . . 3188
27.10.1 Axiom unit tests . . . 3188
27.10.2 Axiom help page . . . 3189
27.10.3 fortran code . . . 3192
27.10.4 lisp code . . . 3199
27.11zggev3 . . . 3212
27.11.1 Axiom unit tests . . . 3212
27.11.2 Axiom help page . . . 3212
27.11.3 fortran code . . . 3216
27.11.4 lisp code . . . 3223
27.12zggevx . . . 3237
27.12.1 Axiom unit tests . . . 3237
27.12.2 Axiom help page . . . 3237
27.12.3 fortran code . . . 3243
27.12.4 lisp code . . . 3252

28 LAPACK - General Matrices, Singular Value, Double 3271
28.1 dgejsv . . . 3271
28.1.1 Axiom unit tests . . . 3271
28.1.2 Axiom help page . . . 3271
28.1.3 fortran code . . . 3279
28.1.4 lisp code . . . 3305
28.2 dgesdd . . . 3372
28.2.1 Axiom unit tests . . . 3372
28.2.2 Axiom help page . . . 3372
28.2.3 fortran code . . . 3376
28.2.4 lisp code . . . 3400
28.3 dgesvd . . . 3400
28.3.1 Axiom unit tests . . . 3400
28.3.2 Axiom help page . . . 3401
28.3.3 fortran code . . . 3404
28.3.4 lisp code . . . 3470
28.4 dgesvdx . . . 3492
28.4.1 Axiom unit tests . . . 3492
28.4.2 Axiom help page . . . 3492
28.4.3 fortran code . . . 3496
28.4.4 lisp code . . . 3507

29 LAPACK - General Matrices, Singular Value, Real 3531
29.1 sgejsv . . . 3531
29.1.1 Axiom unit tests . . . 3531
29.1.2 Axiom help page . . . 3531
29.1.3 fortran code . . . 3539
29.1.4 lisp code . . . 3565
29.2 sgesdd . . . 3631
29.2.1 Axiom unit tests . . . 3631

29.2.2 Axiom help page . . . 3632
29.2.3 fortran code . . . 3635
29.2.4 lisp code . . . 3659
29.3 sgesvd . . . 3732
29.3.1 Axiom unit tests . . . 3732
29.3.2 Axiom help page . . . 3733
29.3.3 fortran code . . . 3736
29.3.4 lisp code . . . 3802
29.4 sgesvdx . . . 3933
29.4.1 Axiom unit tests . . . 3933
29.4.2 Axiom help page . . . 3934
29.4.3 fortran code . . . 3937
29.4.4 lisp code . . . 3948

30 LAPACK - General Matrices, Singular Value, Complex 3971
30.1 cgejsv . . . 3971
30.1.1 Axiom unit tests . . . 3971
30.1.2 Axiom help page . . . 3971
30.1.3 fortran code . . . 3980
30.1.4 lisp code . . . 4007
30.2 cgesdd . . . 4074
30.2.1 Axiom unit tests . . . 4074
30.2.2 Axiom help page . . . 4074
30.2.3 fortran code . . . 4077
30.2.4 lisp code . . . 4114
30.3 cgesvd . . . 4223
30.3.1 Axiom unit tests . . . 4223
30.3.2 Axiom help page . . . 4223
30.3.3 fortran code . . . 4227
30.3.4 lisp code . . . 4296
30.4 cgesvdx . . . 4424
30.4.1 Axiom unit tests . . . 4424
30.4.2 Axiom help page . . . 4425
30.4.3 fortran code . . . 4429
30.4.4 lisp code . . . 4440

31 LAPACK - General Matrices, Singular Value, Complex16 4465
31.1 zgejsv . . . 4465
31.1.1 Axiom unit tests . . . 4465
31.1.2 Axiom help page . . . 4465
31.1.3 fortran code . . . 4473
31.1.4 lisp code . . . 4501
31.2 zgesdd . . . 4568
31.2.1 Axiom unit tests . . . 4568
31.2.2 Axiom help page . . . 4568
31.2.3 fortran code . . . 4571

31.2.4 lisp code . . . 4608
31.3 zgesvd . . . 4711
31.3.1 Axiom unit tests . . . 4711
31.3.2 Axiom help page . . . 4711
31.3.3 fortran code . . . 4715
31.3.4 lisp code . . . 4784
31.4 zgesvdx . . . 4912
31.4.1 Axiom unit tests . . . 4912
31.4.2 Axiom help page . . . 4913
31.4.3 fortran code . . . 4916
31.4.4 lisp code . . . 4928

32 LAPACK - General Matrices, Comp. Routines, Double 4953
32.1 dgebak . . . 4953
32.1.1 Axiom unit tests . . . 4953
32.1.2 Axiom help page . . . 4953
32.1.3 fortran code . . . 4955
32.1.4 lisp code . . . 4958
32.2 dgebal . . . 4958
32.2.1 Axiom unit tests . . . 4958
32.2.2 Axiom help page . . . 4959
32.2.3 fortran code . . . 4961
32.2.4 lisp code . . . 4966
32.3 dgebd2 . . . 4966
32.3.1 Axiom unit tests . . . 4966
32.3.2 Axiom help page . . . 4967
32.3.3 fortran code . . . 4969
32.3.4 lisp code . . . 4972
32.4 dgebrd . . . 4973
32.4.1 Axiom unit tests . . . 4973
32.4.2 Axiom help page . . . 4973
32.4.3 fortran code . . . 4976
32.4.4 lisp code . . . 4979
32.5 dgecon . . . 4980
32.5.1 Axiom unit tests . . . 4980
32.5.2 Axiom help page . . . 4980
32.5.3 fortran code . . . 4982
32.5.4 lisp code . . . 4985
32.6 dgeequ . . . 4988
32.6.1 Axiom unit tests . . . 4988
32.6.2 Axiom help page . . . 4989
32.6.3 fortran code . . . 4991
32.6.4 lisp code . . . 4994
32.7 dgeequb . . . 4998
32.7.1 Axiom unit tests . . . 4998
32.7.2 Axiom help page . . . 4998

32.7.3 fortran code . . . 5000
32.7.4 lisp code . . . 5004
32.8 dgehd2 . . . 5008
32.8.1 Axiom unit tests . . . 5008
32.8.2 Axiom help page . . . 5008
32.8.3 fortran code . . . 5010
32.8.4 lisp code . . . 5012
32.9 dgehrd . . . 5012
32.9.1 Axiom unit tests . . . 5012
32.9.2 Axiom help page . . . 5013
32.9.3 fortran code . . . 5015
32.9.4 lisp code . . . 5019
32.10 dgelq2 . . . 5019
32.10.1 Axiom unit tests . . . 5019
32.10.2 Axiom help page . . . 5020
32.10.3 fortran code . . . 5021
32.10.4 lisp code . . . 5023
32.11 dgelqf . . . 5023
32.11.1 Axiom unit tests . . . 5023
32.11.2 Axiom help page . . . 5024
32.11.3 fortran code . . . 5025
32.11.4 lisp code . . . 5028
32.12 dgemqrt . . . 5029
32.12.1 Axiom unit tests . . . 5029
32.12.2 Axiom help page . . . 5029
32.12.3 fortran code . . . 5031
32.12.4 lisp code . . . 5034
32.13 dgeql2 . . . 5039
32.13.1 Axiom unit tests . . . 5039
32.13.2 Axiom help page . . . 5039
32.13.3 fortran code . . . 5041
32.13.4 lisp code . . . 5042
32.14 dgeqlf . . . 5045
32.14.1 Axiom unit tests . . . 5045
32.14.2 Axiom help page . . . 5045
32.14.3 fortran code . . . 5047
32.14.4 lisp code . . . 5050
32.15 dgeqp3 . . . 5054
32.15.1 Axiom unit tests . . . 5054
32.15.2 Axiom help page . . . 5055
32.15.3 fortran code . . . 5057
32.15.4 lisp code . . . 5061
32.16 dgeqpf - DEPRECATED use dgeqp3 . . . 5067
32.17 dgeqr2 . . . 5067
32.17.1 Axiom unit tests . . . 5067
32.17.2 Axiom help page . . . 5068

32.17.3 fortran code . . . 5069
32.17.4 lisp code . . . 5071
32.18 dgeqr2p . . . 5073
32.18.1 Axiom unit tests . . . 5073
32.18.2 Axiom help page [Demm08] . . . 5073
32.18.3 fortran code . . . 5075
32.18.4 lisp code . . . 5077
32.19 dgeqrf . . . 5078
32.19.1 Axiom unit tests . . . 5078
32.19.2 Axiom help page . . . 5079
32.19.3 fortran code . . . 5081
32.19.4 lisp code . . . 5084
32.20 dgeqrfp . . . 5088
32.20.1 Axiom unit tests . . . 5088
32.20.2 Axiom help page [Demm08] . . . 5088
32.20.3 fortran code . . . 5090
32.20.4 lisp code . . . 5093
32.21 dgeqrt . . . 5097
32.21.1 Axiom unit tests . . . 5097
32.21.2 Axiom help page . . . 5097
32.21.3 fortran code . . . 5099
32.21.4 lisp code . . . 5101
32.22 dgeqrt2 . . . 5103
32.22.1 Axiom unit tests . . . 5103
32.22.2 Axiom help page . . . 5104
32.22.3 fortran code . . . 5105
32.22.4 lisp code . . . 5108
32.23 dgeqrt3 . . . 5111
32.23.1 Axiom unit tests . . . 5111
32.23.2 Axiom help page [Elmr00] . . . 5111
32.23.3 fortran code . . . 5114
32.23.4 lisp code . . . 5116
32.24 dgerfs . . . 5121
32.24.1 Axiom unit tests . . . 5121
32.24.2 Axiom help page . . . 5122
32.24.3 fortran code . . . 5124
32.24.4 lisp code . . . 5129
32.25 dgerfsx . . . 5137
32.25.1 Axiom unit tests . . . 5137
32.25.2 Axiom help page [Demm05] . . . 5138
32.25.3 fortran code . . . 5144
32.25.4 lisp code . . . 5151
32.26 dgerq2 . . . 5151
32.26.1 Axiom unit tests . . . 5151
32.26.2 Axiom help page . . . 5152
32.26.3 fortran code . . . 5153

32.26.4 lisp code . . . 5155
32.27 dgerqf . . . 5157
32.27.1 Axiom unit tests . . . 5157
32.27.2 Axiom help page . . . 5157
32.27.3 fortran code . . . 5159
32.27.4 lisp code . . . 5162
32.28 dgesvj . . . 5166
32.28.1 Axiom unit tests . . . 5166
32.28.2 Axiom help page . . . 5167
32.28.3 fortran code . . . 5171
32.28.4 lisp code . . . 5197
32.29 dgetf2 . . . 5267
32.29.1 Axiom unit tests . . . 5267
32.29.2 Axiom help page . . . 5267
32.29.3 fortran code . . . 5269
32.29.4 lisp code . . . 5271
32.30 dgetrf . . . 5271
32.30.1 Axiom unit tests . . . 5271
32.30.2 Axiom help page . . . 5272
32.30.3 fortran code . . . 5273
32.30.4 lisp code . . . 5276
32.31 dgetrf2 . . . 5279
32.31.1 Axiom unit tests . . . 5279
32.31.2 Axiom help page . . . 5279
32.31.3 fortran code . . . 5281
32.31.4 lisp code . . . 5285
32.32 dgetri . . . 5288
32.32.1 Axiom unit tests . . . 5288
32.32.2 Axiom help page . . . 5288
32.32.3 fortran code . . . 5290
32.32.4 lisp code . . . 5293
32.33 dgetrs . . . 5298
32.33.1 Axiom unit tests . . . 5298
32.33.2 Axiom help page . . . 5298
32.33.3 fortran code . . . 5300
32.33.4 lisp code . . . 5302
32.34 dhgeqz . . . 5302
32.34.1 Axiom unit tests . . . 5302
32.34.2 Axiom help page [Mole71] . . . 5303
32.34.3 fortran code . . . 5307
32.34.4 lisp code . . . 5329
32.35 dla_geamv . . . 5374
32.35.1 Axiom unit tests . . . 5374
32.35.2 Axiom help page . . . 5374
32.35.3 fortran code . . . 5377
32.35.4 lisp code . . . 5381

32.36dla_gercond . . . 5387
32.36.1 Axiom unit tests . . . 5387
32.36.2 Axiom help page . . . 5388
32.36.3 fortran code . . . 5390
32.36.4 lisp code . . . 5393
32.37dla_gerfsx_extended . . . 5400
32.37.1 Axiom unit tests . . . 5400
32.37.2 Axiom help page [Demm05] . . . 5400
32.37.3 fortran code . . . 5406
32.37.4 lisp code . . . 5412
32.38dla_gerpvgrw . . . 5420
32.38.1 Axiom unit tests . . . 5420
32.38.2 Axiom help page . . . 5421
32.38.3 fortran code . . . 5422
32.38.4 lisp code . . . 5423
32.39dtgevc . . . 5424
32.39.1 Axiom unit tests . . . 5424
32.39.2 Axiom help page . . . 5425
32.39.3 fortran code . . . 5429
32.39.4 lisp code . . . 5448
32.40dtgexc . . . 5485
32.40.1 Axiom unit tests . . . 5485
32.40.2 Axiom help page . . . 5486
32.40.3 fortran code . . . 5489
32.40.4 lisp code . . . 5495
32.41zgesvj . . . 5504
32.41.1 Axiom unit tests . . . 5504
32.41.2 Axiom help page . . . 5504
32.41.3 fortran code . . . 5509
32.41.4 lisp code . . . 5531

33 LAPACK - General Matrices, Comp. Routines, Real 5581
33.1 sgebak . . . 5581
33.1.1 Axiom unit tests . . . 5581
33.1.2 Axiom help page . . . 5581
33.1.3 fortran code . . . 5583
33.1.4 lisp code . . . 5586
33.2 sgebal . . . 5591
33.2.1 Axiom unit tests . . . 5591
33.2.2 Axiom help page . . . 5591
33.2.3 fortran code . . . 5594
33.2.4 lisp code . . . 5599
33.3 sgebd2 . . . 5605
33.3.1 Axiom unit tests . . . 5605
33.3.2 Axiom help page . . . 5605
33.3.3 fortran code . . . 5608

33.3.4 lisp code . . . 5611
33.4 sgebrd . . . 5616
33.4.1 Axiom unit tests . . . 5616
33.4.2 Axiom help page . . . 5616
33.4.3 fortran code . . . 5619
33.4.4 lisp code . . . 5622
33.5 sgecon . . . 5628
33.5.1 Axiom unit tests . . . 5628
33.5.2 Axiom help page . . . 5628
33.5.3 fortran code . . . 5630
33.5.4 lisp code . . . 5633
33.6 sgeequ . . . 5637
33.6.1 Axiom unit tests . . . 5637
33.6.2 Axiom help page . . . 5637
33.6.3 fortran code . . . 5639
33.6.4 lisp code . . . 5642
33.7 sgeequb . . . 5646
33.7.1 Axiom unit tests . . . 5646
33.7.2 Axiom help page . . . 5646
33.7.3 fortran code . . . 5648
33.7.4 lisp code . . . 5652
33.8 sgehd2 . . . 5656
33.8.1 Axiom unit tests . . . 5656
33.8.2 Axiom help page . . . 5656
33.8.3 fortran code . . . 5658
33.8.4 lisp code . . . 5660
33.9 sgehrd . . . 5662
33.9.1 Axiom unit tests . . . 5662
33.9.2 Axiom help page . . . 5663
33.9.3 fortran code . . . 5665
33.9.4 lisp code . . . 5669
33.10 sgelq2 . . . 5675
33.10.1 Axiom unit tests . . . 5675
33.10.2 Axiom help page . . . 5676
33.10.3 fortran code . . . 5677
33.10.4 lisp code . . . 5679
33.11 sgelqf . . . 5681
33.11.1 Axiom unit tests . . . 5681
33.11.2 Axiom help page . . . 5681
33.11.3 fortran code . . . 5683
33.11.4 lisp code . . . 5686
33.12 sgemqrt . . . 5690
33.12.1 Axiom unit tests . . . 5690
33.12.2 Axiom help page . . . 5690
33.12.3 fortran code . . . 5693
33.12.4 lisp code . . . 5695

33.13sgeql2 . . . 5700
33.13.1 Axiom unit tests . . . 5700
33.13.2 Axiom help page . . . 5701
33.13.3 fortran code . . . 5702
33.13.4 lisp code . . . 5704
33.14sgeqlf . . . 5706
33.14.1 Axiom unit tests . . . 5706
33.14.2 Axiom help page . . . 5706
33.14.3 fortran code . . . 5708
33.14.4 lisp code . . . 5711
33.15sgeqp3 . . . 5716
33.15.1 Axiom unit tests . . . 5716
33.15.2 Axiom help page . . . 5716
33.15.3 fortran code . . . 5718
33.15.4 lisp code . . . 5722
33.16sgeqpf - DEPRECATED use sgeqp3 . . . 5729
33.17sgeqr2 . . . 5729
33.17.1 Axiom unit tests . . . 5729
33.17.2 Axiom help page . . . 5729
33.17.3 fortran code . . . 5731
33.17.4 lisp code . . . 5732
33.18sgeqr2p . . . 5734
33.18.1 Axiom unit tests . . . 5734
33.18.2 Axiom help page [Demm08] . . . 5735
33.18.3 fortran code . . . 5736
33.18.4 lisp code . . . 5738
33.19sgeqrf . . . 5740
33.19.1 Axiom unit tests . . . 5740
33.19.2 Axiom help page . . . 5740
33.19.3 fortran code . . . 5742
33.19.4 lisp code . . . 5745
33.20sgeqrfp . . . 5749
33.20.1 Axiom unit tests . . . 5749
33.20.2 Axiom help page [Demm08] . . . 5749
33.20.3 fortran code . . . 5751
33.20.4 lisp code . . . 5754
33.21sgeqrt . . . 5758
33.21.1 Axiom unit tests . . . 5758
33.21.2 Axiom help page . . . 5758
33.21.3 fortran code . . . 5760
33.21.4 lisp code . . . 5762
33.22sgeqrt2 . . . 5765
33.22.1 Axiom unit tests . . . 5765
33.22.2 Axiom help page . . . 5765
33.22.3 fortran code . . . 5767
33.22.4 lisp code . . . 5769

33.23sgeqrt3 . . . 5772
33.23.1 Axiom unit tests . . . 5772
33.23.2 Axiom help page [Elmr00] . . . 5773
33.23.3 fortran code . . . 5774
33.23.4 lisp code . . . 5777
33.24sgerfs . . . 5782
33.24.1 Axiom unit tests . . . 5782
33.24.2 Axiom help page . . . 5782
33.24.3 fortran code . . . 5785
33.24.4 lisp code . . . 5790
33.25sgerfsx . . . 5798
33.25.1 Axiom unit tests . . . 5798
33.25.2 Axiom help page [Demm05] . . . 5799
33.25.3 fortran code . . . 5805
33.25.4 lisp code . . . 5812
33.26sgerq2 . . . 5812
33.26.1 Axiom unit tests . . . 5812
33.26.2 Axiom help page . . . 5812
33.26.3 fortran code . . . 5814
33.26.4 lisp code . . . 5815
33.27sgerqf . . . 5818
33.27.1 Axiom unit tests . . . 5818
33.27.2 Axiom help page . . . 5818
33.27.3 fortran code . . . 5820
33.27.4 lisp code . . . 5823
33.28sgesvj . . . 5827
33.28.1 Axiom unit tests . . . 5827
33.28.2 Axiom help page . . . 5828
33.28.3 fortran code . . . 5832
33.28.4 lisp code . . . 5858
33.29sgetf2 . . . 5927
33.29.1 Axiom unit tests . . . 5927
33.29.2 Axiom help page . . . 5928
33.29.3 fortran code . . . 5929
33.29.4 lisp code . . . 5932
33.30sgetrf . . . 5934
33.30.1 Axiom unit tests . . . 5934
33.30.2 Axiom help page . . . 5935
33.30.3 fortran code . . . 5936
33.30.4 lisp code . . . 5939
33.31sgetrf2 . . . 5942
33.31.1 Axiom unit tests . . . 5942
33.31.2 Axiom help page . . . 5943
33.31.3 fortran code . . . 5944
33.31.4 lisp code . . . 5948
33.32sgetri . . . 5951

33.32.1 Axiom unit tests . . . 5951
33.32.2 Axiom help page . . . 5952
33.32.3 fortran code . . . 5953
33.32.4 lisp code . . . 5956
33.33sgetrs . . . 5961
33.33.1 Axiom unit tests . . . 5961
33.33.2 Axiom help page . . . 5961
33.33.3 fortran code . . . 5963
33.33.4 lisp code . . . 5965
33.34shgeqz . . . 5968
33.34.1 Axiom unit tests . . . 5968
33.34.2 Axiom help page [Mole71] . . . 5969
33.34.3 fortran code . . . 5973
33.34.4 lisp code . . . 5995
33.35sla_geamv . . . 6039
33.35.1 Axiom unit tests . . . 6039
33.35.2 Axiom help page . . . 6040
33.35.3 fortran code . . . 6042
33.35.4 lisp code . . . 6047
33.36sla_gercond . . . 6053
33.36.1 Axiom unit tests . . . 6053
33.36.2 Axiom help page . . . 6053
33.36.3 fortran code . . . 6055
33.36.4 lisp code . . . 6059
33.37sla_gerfsx_extended . . . 6065
33.37.1 Axiom unit tests . . . 6065
33.37.2 Axiom help page [Demm05] . . . 6066
33.37.3 fortran code . . . 6072
33.37.4 lisp code . . . 6078
33.38sla_gerpvgrw . . . 6086
33.38.1 Axiom unit tests . . . 6086
33.38.2 Axiom help page . . . 6086
33.38.3 fortran code . . . 6088
33.38.4 lisp code . . . 6089
33.39stgevc . . . 6090
33.39.1 Axiom unit tests . . . 6090
33.39.2 Axiom help page . . . 6090
33.39.3 fortran code . . . 6095
33.39.4 lisp code . . . 6114
33.40stgexc . . . 6150
33.40.1 Axiom unit tests . . . 6150
33.40.2 Axiom help page . . . 6151
33.40.3 fortran code . . . 6153
33.40.4 lisp code . . . 6160

34 LAPACK - General Matrices, Comp. Routines, Complex 6171
34.1 cgebak . . . 6171
34.1.1 Axiom unit tests . . . 6171
34.1.2 Axiom help page . . . 6171
34.1.3 fortran code . . . 6173
34.1.4 lisp code . . . 6176
34.2 cgebal . . . 6181
34.2.1 Axiom unit tests . . . 6181
34.2.2 Axiom help page . . . 6181
34.2.3 fortran code . . . 6184
34.2.4 lisp code . . . 6189
34.3 cgebd2 . . . 6196
34.3.1 Axiom unit tests . . . 6196
34.3.2 Axiom help page . . . 6196
34.3.3 fortran code . . . 6199
34.3.4 lisp code . . . 6202
34.4 cgebrd . . . 6207
34.4.1 Axiom unit tests . . . 6207
34.4.2 Axiom help page . . . 6208
34.4.3 fortran code . . . 6211
34.4.4 lisp code . . . 6214
34.5 cgecon . . . 6220
34.5.1 Axiom unit tests . . . 6220
34.5.2 Axiom help page . . . 6220
34.5.3 fortran code . . . 6222
34.5.4 lisp code . . . 6225
34.6 cgeequ . . . 6229
34.6.1 Axiom unit tests . . . 6229
34.6.2 Axiom help page . . . 6229
34.6.3 fortran code . . . 6231
34.6.4 lisp code . . . 6235
34.7 cgeequb . . . 6238
34.7.1 Axiom unit tests . . . 6238
34.7.2 Axiom help page . . . 6239
34.7.3 fortran code . . . 6241
34.7.4 lisp code . . . 6245
34.8 cgehd2 . . . 6249
34.8.1 Axiom unit tests . . . 6249
34.8.2 Axiom help page . . . 6249
34.8.3 fortran code . . . 6251
34.8.4 lisp code . . . 6253
34.9 cgehrd . . . 6255
34.9.1 Axiom unit tests . . . 6255
34.9.2 Axiom help page . . . 6256
34.9.3 fortran code . . . 6258
34.9.4 lisp code . . . 6262

34.10cgelq2 . . . 6269
34.10.1 Axiom unit tests . . . 6269
34.10.2 Axiom help page . . . 6269
34.10.3 fortran code . . . 6271
34.10.4 lisp code . . . 6272
34.11cgelqf . . . 6275
34.11.1 Axiom unit tests . . . 6275
34.11.2 Axiom help page . . . 6275
34.11.3 fortran code . . . 6277
34.11.4 lisp code . . . 6280
34.12cgemqrt . . . 6284
34.12.1 Axiom unit tests . . . 6284
34.12.2 Axiom help page . . . 6284
34.12.3 fortran code . . . 6286
34.12.4 lisp code . . . 6289
34.13cgeql2 . . . 6294
34.13.1 Axiom unit tests . . . 6294
34.13.2 Axiom help page . . . 6294
34.13.3 fortran code . . . 6296
34.13.4 lisp code . . . 6298
34.14cgeqlf . . . 6300
34.14.1 Axiom unit tests . . . 6300
34.14.2 Axiom help page . . . 6300
34.14.3 fortran code . . . 6302
34.14.4 lisp code . . . 6305
34.15cgeqp3 . . . 6309
34.15.1 Axiom unit tests . . . 6309
34.15.2 Axiom help page . . . 6310
34.15.3 fortran code . . . 6312
34.15.4 lisp code . . . 6316
34.16cgeqpf - DEPRECATED use cgeqp3 . . . 6322
34.17cgeqr2 . . . 6322
34.17.1 Axiom unit tests . . . 6322
34.17.2 Axiom help page . . . 6323
34.17.3 fortran code . . . 6324
34.17.4 lisp code . . . 6326
34.18cgeqr2p . . . 6328
34.18.1 Axiom unit tests . . . 6328
34.18.2 Axiom help page [Demm08] . . . 6328
34.18.3 fortran code . . . 6330
34.18.4 lisp code . . . 6332
34.19cgeqrf . . . 6334
34.19.1 Axiom unit tests . . . 6334
34.19.2 Axiom help page . . . 6334
34.19.3 fortran code . . . 6336
34.19.4 lisp code . . . 6339

34.20cgeqrfp . . . 6343
34.20.1 Axiom unit tests . . . 6343
34.20.2 Axiom help page [Demm08] . . . 6343
34.20.3 fortran code . . . 6345
34.20.4 lisp code . . . 6348
34.21cgeqrt . . . 6352
34.21.1 Axiom unit tests . . . 6352
34.21.2 Axiom help page . . . 6352
34.21.3 fortran code . . . 6354
34.21.4 lisp code . . . 6356
34.22cgeqrt2 . . . 6358
34.22.1 Axiom unit tests . . . 6358
34.22.2 Axiom help page . . . 6359
34.22.3 fortran code . . . 6361
34.22.4 lisp code . . . 6363
34.23cgeqrt3 . . . 6366
34.23.1 Axiom unit tests . . . 6366
34.23.2 Axiom help page [Elmr00] . . . 6367
34.23.3 fortran code . . . 6368
34.23.4 lisp code . . . 6371
34.24cgerfs . . . 6376
34.24.1 Axiom unit tests . . . 6376
34.24.2 Axiom help page . . . 6377
34.24.3 fortran code . . . 6379
34.24.4 lisp code . . . 6384
34.25cgerfsx . . . 6393
34.25.1 Axiom unit tests . . . 6393
34.25.2 Axiom help page [Demm05] . . . 6393
34.25.3 fortran code . . . 6399
34.25.4 lisp code . . . 6406
34.26cgerq2 . . . 6406
34.26.1 Axiom unit tests . . . 6406
34.26.2 Axiom help page . . . 6407
34.26.3 fortran code . . . 6408
34.26.4 lisp code . . . 6410
34.27cgerqf . . . 6413
34.27.1 Axiom unit tests . . . 6413
34.27.2 Axiom help page . . . 6413
34.27.3 fortran code . . . 6415
34.27.4 lisp code . . . 6418
34.28cgesvj . . . 6422
34.28.1 Axiom unit tests . . . 6422
34.28.2 Axiom help page . . . 6422
34.28.3 fortran code . . . 6427
34.28.4 lisp code . . . 6449
34.29cgetf2 . . . 6498

34.29.1 Axiom unit tests . . . 6498
34.29.2 Axiom help page . . . 6498
34.29.3 fortran code . . . 6500
34.29.4 lisp code . . . 6502
34.30cgetrf . . . 6505
34.30.1 Axiom unit tests . . . 6505
34.30.2 Axiom help page . . . 6506
34.30.3 fortran code . . . 6507
34.30.4 lisp code . . . 6510
34.31cgetrf2 . . . 6513
34.31.1 Axiom unit tests . . . 6513
34.31.2 Axiom help page . . . 6514
34.31.3 fortran code . . . 6515
34.31.4 lisp code . . . 6519
34.32cgetri . . . 6522
34.32.1 Axiom unit tests . . . 6522
34.32.2 Axiom help page . . . 6523
34.32.3 fortran code . . . 6524
34.32.4 lisp code . . . 6527
34.33cgetrs . . . 6532
34.33.1 Axiom unit tests . . . 6532
34.33.2 Axiom help page . . . 6533
34.33.3 fortran code . . . 6534
34.33.4 lisp code . . . 6536
34.34chgeqz . . . 6539
34.34.1 Axiom unit tests . . . 6539
34.34.2 Axiom help page [Mole71] . . . 6540
34.34.3 fortran code . . . 6544
34.34.4 lisp code . . . 6556
34.35cla_geamv . . . 6582
34.35.1 Axiom unit tests . . . 6582
34.35.2 Axiom help page . . . 6582
34.35.3 fortran code . . . 6585
34.35.4 lisp code . . . 6590
34.36cla_gercond_c . . . 6596
34.36.1 Axiom unit tests . . . 6596
34.36.2 Axiom help page . . . 6596
34.36.3 fortran code . . . 6598
34.36.4 lisp code . . . 6602
34.37cla_gercond_x . . . 6608
34.37.1 Axiom unit tests . . . 6608
34.37.2 Axiom help page . . . 6608
34.37.3 fortran code . . . 6610
34.37.4 lisp code . . . 6613
34.38cla_gerfsx_extended . . . 6618
34.38.1 Axiom unit tests . . . 6618

34.38.2 Axiom help page [Demm05] . . . 6618
34.38.3 fortran code . . . 6624
34.38.4 lisp code . . . 6631
34.39 cla_gerpvgrw . . . 6639
34.39.1 Axiom unit tests . . . 6639
34.39.2 Axiom help page . . . 6639
34.39.3 fortran code . . . 6641
34.39.4 lisp code . . . 6642
34.40 ctgevc . . . 6644
34.40.1 Axiom unit tests . . . 6644
34.40.2 Axiom help page . . . 6644
34.40.3 fortran code . . . 6647
34.40.4 lisp code . . . 6658
34.41 ctgexc . . . 6674
34.41.1 Axiom unit tests . . . 6674
34.41.2 Axiom help page . . . 6674
34.41.3 fortran code . . . 6677
34.41.4 lisp code . . . 6679

35 LAPACK - General Matrices, Comp. Routines, Complex16 6683
35.1 zgebak . . . 6683
35.1.1 Axiom unit tests . . . 6683
35.1.2 Axiom help page . . . 6683
35.1.3 fortran code . . . 6685
35.1.4 lisp code . . . 6688
35.2 zgebal . . . 6688
35.2.1 Axiom unit tests . . . 6688
35.2.2 Axiom help page . . . 6689
35.2.3 fortran code . . . 6691
35.2.4 lisp code . . . 6696
35.3 zgebd2 . . . 6696
35.3.1 Axiom unit tests . . . 6696
35.3.2 Axiom help page . . . 6697
35.3.3 fortran code . . . 6699
35.3.4 lisp code . . . 6702
35.4 zgebrd . . . 6708
35.4.1 Axiom unit tests . . . 6708
35.4.2 Axiom help page . . . 6708
35.4.3 fortran code . . . 6712
35.4.4 lisp code . . . 6715
35.5 zgecon . . . 6720
35.5.1 Axiom unit tests . . . 6720
35.5.2 Axiom help page . . . 6721
35.5.3 fortran code . . . 6722
35.5.4 lisp code . . . 6725
35.6 zgeequ . . . 6729

35.6.1 Axiom unit tests . . . 6729
35.6.2 Axiom help page . . . 6730
35.6.3 fortran code . . . 6732
35.6.4 lisp code . . . 6735
35.7 zgeequb . . . 6739
35.7.1 Axiom unit tests . . . 6739
35.7.2 Axiom help page . . . 6739
35.7.3 fortran code . . . 6741
35.7.4 lisp code . . . 6745
35.8 zgehd2 . . . 6750
35.8.1 Axiom unit tests . . . 6750
35.8.2 Axiom help page . . . 6750
35.8.3 fortran code . . . 6752
35.8.4 lisp code . . . 6754
35.9 zgehrd . . . 6754
35.9.1 Axiom unit tests . . . 6754
35.9.2 Axiom help page . . . 6754
35.9.3 fortran code . . . 6757
35.9.4 lisp code . . . 6761
35.10 zgelq2 . . . 6761
35.10.1 Axiom unit tests . . . 6761
35.10.2 Axiom help page . . . 6761
35.10.3 fortran code . . . 6763
35.10.4 lisp code . . . 6765
35.11 zgelqf . . . 6767
35.11.1 Axiom unit tests . . . 6767
35.11.2 Axiom help page . . . 6767
35.11.3 fortran code . . . 6769
35.11.4 lisp code . . . 6772
35.12 zgemqrt . . . 6776
35.12.1 Axiom unit tests . . . 6776
35.12.2 Axiom help page . . . 6776
35.12.3 fortran code . . . 6779
35.12.4 lisp code . . . 6782
35.13 zgeql2 . . . 6786
35.13.1 Axiom unit tests . . . 6786
35.13.2 Axiom help page . . . 6787
35.13.3 fortran code . . . 6789
35.13.4 lisp code . . . 6790
35.14 zgeqlf . . . 6792
35.14.1 Axiom unit tests . . . 6792
35.14.2 Axiom help page . . . 6793
35.14.3 fortran code . . . 6795
35.14.4 lisp code . . . 6798
35.15 zgeqp3 . . . 6802
35.15.1 Axiom unit tests . . . 6802

35.15.2 Axiom help page . . . 6802
35.15.3 fortran code . . . 6804
35.15.4 lisp code . . . 6809
35.16zgeqpf - DEPRECATED use zgeqp3 . . . 6815
35.17zgeqr2 . . . 6815
35.17.1 Axiom unit tests . . . 6815
35.17.2 Axiom help page . . . 6815
35.17.3 fortran code . . . 6817
35.17.4 lisp code . . . 6819
35.18zgeqr2p . . . 6821
35.18.1 Axiom unit tests . . . 6821
35.18.2 Axiom help page [Demm08] . . . 6821
35.18.3 fortran code . . . 6823
35.18.4 lisp code . . . 6824
35.19zgeqrf . . . 6826
35.19.1 Axiom unit tests . . . 6826
35.19.2 Axiom help page . . . 6827
35.19.3 fortran code . . . 6829
35.19.4 lisp code . . . 6831
35.20zgeqrfp . . . 6835
35.20.1 Axiom unit tests . . . 6835
35.20.2 Axiom help page [Demm08] . . . 6836
35.20.3 fortran code . . . 6838
35.20.4 lisp code . . . 6840
35.21zgeqrt . . . 6844
35.21.1 Axiom unit tests . . . 6844
35.21.2 Axiom help page . . . 6845
35.21.3 fortran code . . . 6847
35.21.4 lisp code . . . 6848
35.22zgeqrt2 . . . 6851
35.22.1 Axiom unit tests . . . 6851
35.22.2 Axiom help page . . . 6852
35.22.3 fortran code . . . 6853
35.22.4 lisp code . . . 6856
35.23zgeqrt3 . . . 6859
35.23.1 Axiom unit tests . . . 6859
35.23.2 Axiom help page [Elmr00] . . . 6859
35.23.3 fortran code . . . 6861
35.23.4 lisp code . . . 6864
35.24zgerfs . . . 6869
35.24.1 Axiom unit tests . . . 6869
35.24.2 Axiom help page . . . 6870
35.24.3 fortran code . . . 6872
35.24.4 lisp code . . . 6878
35.25zgerfsx . . . 6886
35.25.1 Axiom unit tests . . . 6886

35.25.2 Axiom help page [Demm05] . . . 6886
35.25.3 fortran code . . . 6893
35.25.4 lisp code . . . 6899
35.26zgerq2 . . . 6900
35.26.1 Axiom unit tests . . . 6900
35.26.2 Axiom help page . . . 6900
35.26.3 fortran code . . . 6902
35.26.4 lisp code . . . 6903
35.27zgerqf . . . 6906
35.27.1 Axiom unit tests . . . 6906
35.27.2 Axiom help page . . . 6906
35.27.3 fortran code . . . 6908
35.27.4 lisp code . . . 6911
35.28zgetf2 . . . 6915
35.28.1 Axiom unit tests . . . 6915
35.28.2 Axiom help page . . . 6916
35.28.3 fortran code . . . 6917
35.28.4 lisp code . . . 6920
35.29zgetrf . . . 6922
35.29.1 Axiom unit tests . . . 6922
35.29.2 Axiom help page . . . 6923
35.29.3 fortran code . . . 6924
35.29.4 lisp code . . . 6927
35.30zgetrf2 . . . 6931
35.30.1 Axiom unit tests . . . 6931
35.30.2 Axiom help page . . . 6931
35.30.3 fortran code . . . 6933
35.30.4 lisp code . . . 6936
35.31zgetri . . . 6940
35.31.1 Axiom unit tests . . . 6940
35.31.2 Axiom help page . . . 6940
35.31.3 fortran code . . . 6942
35.31.4 lisp code . . . 6945
35.32zgetrs . . . 6950
35.32.1 Axiom unit tests . . . 6950
35.32.2 Axiom help page . . . 6950
35.32.3 fortran code . . . 6952
35.32.4 lisp code . . . 6954
35.33zhgeqz . . . 6957
35.33.1 Axiom unit tests . . . 6957
35.33.2 Axiom help page [Mole71] . . . 6957
35.33.3 fortran code . . . 6961
35.33.4 lisp code . . . 6973
35.34zla_geamv . . . 7000
35.34.1 Axiom unit tests . . . 7000
35.34.2 Axiom help page . . . 7000

35.34.3 fortran code . . . 7003
35.34.4 lisp code . . . 7007
35.35 zla_gercond_c . . . 7014
35.35.1 Axiom unit tests . . . 7014
35.35.2 Axiom help page . . . 7014
35.35.3 fortran code . . . 7016
35.35.4 lisp code . . . 7020
35.36 zla_gercond_x . . . 7025
35.36.1 Axiom unit tests . . . 7025
35.36.2 Axiom help page . . . 7026
35.36.3 fortran code . . . 7028
35.36.4 lisp code . . . 7031
35.37 zla_gerfsx_extended . . . 7036
35.37.1 Axiom unit tests . . . 7036
35.37.2 Axiom help page [Demm05] . . . 7036
35.37.3 fortran code . . . 7042
35.37.4 lisp code . . . 7049
35.38 zla_gerpvgrw . . . 7057
35.38.1 Axiom unit tests . . . 7057
35.38.2 Axiom help page . . . 7057
35.38.3 fortran code . . . 7059
35.38.4 lisp code . . . 7060
35.39 ztgevc . . . 7062
35.39.1 Axiom unit tests . . . 7062
35.39.2 Axiom help page . . . 7062
35.39.3 fortran code . . . 7065
35.39.4 lisp code . . . 7076
35.40 ztgexc . . . 7092
35.40.1 Axiom unit tests . . . 7092
35.40.2 Axiom help page . . . 7092
35.40.3 fortran code . . . 7095
35.40.4 lisp code . . . 7097

36 LAPACK - General Matrices, Auxilliary Routines, Double 7101
36.1 dgesc2 . . . 7101
36.1.1 Axiom unit tests . . . 7101
36.1.2 Axiom help page . . . 7101
36.1.3 fortran code . . . 7103
36.1.4 lisp code . . . 7105
36.2 dgetc2 . . . 7107
36.2.1 Axiom unit tests . . . 7107
36.2.2 Axiom help page . . . 7108
36.2.3 fortran code . . . 7109
36.2.4 lisp code . . . 7112
36.3 dlange . . . 7115
36.3.1 Axiom unit tests . . . 7115

36.3.2 Axiom help page . . . 7115
36.3.3 fortran code . . . 7117
36.3.4 lisp code . . . 7119
36.4 dlaqge . . . 7119
36.4.1 Axiom unit tests . . . 7119
36.4.2 Axiom help page . . . 7119
36.4.3 fortran code . . . 7121
36.4.4 lisp code . . . 7123
36.5 dtgex2 . . . 7126
36.5.1 Axiom unit tests . . . 7126
36.5.2 Axiom help page . . . 7126
36.5.3 fortran code . . . 7129
36.5.4 lisp code . . . 7139

37 LAPACK - General Matrices, Auxilliary Routines, Real 7169
37.1 sgesc2 . . . 7169
37.1.1 Axiom unit tests . . . 7169
37.1.2 Axiom help page . . . 7169
37.1.3 fortran code . . . 7171
37.1.4 lisp code . . . 7173
37.2 sgetc2 . . . 7175
37.2.1 Axiom unit tests . . . 7175
37.2.2 Axiom help page . . . 7176
37.2.3 fortran code . . . 7177
37.2.4 lisp code . . . 7180
37.3 slange . . . 7183
37.3.1 Axiom unit tests . . . 7183
37.3.2 Axiom help page . . . 7183
37.3.3 fortran code . . . 7185
37.3.4 lisp code . . . 7187
37.4 slaqge . . . 7190
37.4.1 Axiom unit tests . . . 7190
37.4.2 Axiom help page . . . 7190
37.4.3 fortran code . . . 7192
37.4.4 lisp code . . . 7194
37.5 stgex2 . . . 7196
37.5.1 Axiom unit tests . . . 7196
37.5.2 Axiom help page . . . 7197
37.5.3 fortran code . . . 7200
37.5.4 lisp code . . . 7209

38 LAPACK - General Matrices, Auxilliary Routines, Complex 7235
38.1 cgesc2 . . . 7235
38.1.1 Axiom unit tests . . . 7235
38.1.2 Axiom help page . . . 7235
38.1.3 fortran code . . . 7237

38.1.4 lisp code . . . 7239
38.2 cgetc2 . . . 7242
38.2.1 Axiom unit tests . . . 7242
38.2.2 Axiom help page . . . 7242
38.2.3 fortran code . . . 7243
38.2.4 lisp code . . . 7246
38.3 clange . . . 7249
38.3.1 Axiom unit tests . . . 7249
38.3.2 Axiom help page . . . 7249
38.3.3 fortran code . . . 7251
38.3.4 lisp code . . . 7253
38.4 claqge . . . 7256
38.4.1 Axiom unit tests . . . 7256
38.4.2 Axiom help page . . . 7257
38.4.3 fortran code . . . 7259
38.4.4 lisp code . . . 7261
38.5 ctgex2 . . . 7263
38.5.1 Axiom unit tests . . . 7263
38.5.2 Axiom help page . . . 7263
38.5.3 fortran code . . . 7266
38.5.4 lisp code . . . 7269

39 LAPACK - General Matrices, Auxilliary Routines, Complex16 7277
39.1 zgesc2 . . . 7277
39.1.1 Axiom unit tests . . . 7277
39.1.2 Axiom help page . . . 7277
39.1.3 fortran code . . . 7279
39.1.4 lisp code . . . 7281
39.2 zgetc2 . . . 7284
39.2.1 Axiom unit tests . . . 7284
39.2.2 Axiom help page . . . 7284
39.2.3 fortran code . . . 7285
39.2.4 lisp code . . . 7288
39.3 zlange . . . 7291
39.3.1 Axiom unit tests . . . 7291
39.3.2 Axiom help page . . . 7291
39.3.3 fortran code . . . 7293
39.3.4 lisp code . . . 7295
39.4 zlaqge . . . 7295
39.4.1 Axiom unit tests . . . 7295
39.4.2 Axiom help page . . . 7296
39.4.3 fortran code . . . 7297
39.4.4 lisp code . . . 7300
39.5 ztgex2 . . . 7302
39.5.1 Axiom unit tests . . . 7302
39.5.2 Axiom help page . . . 7302

39.5.3 fortran code . . . 7305
39.5.4 lisp code . . . 7308

40 LAPACK - General Band Matrix, Linear Solve, Double 7315
40.1 dgbsv . . . 7315
40.1.1 Axiom unit tests . . . 7315
40.1.2 Axiom help page . . . 7315
40.1.3 fortran code . . . 7318
40.1.4 lisp code . . . 7319
40.2 dgbsvx . . . 7321
40.2.1 Axiom unit tests . . . 7321
40.2.2 Axiom help page . . . 7321
40.2.3 fortran code . . . 7327
40.2.4 lisp code . . . 7333
40.3 dgbsvxx . . . 7344
40.3.1 Axiom unit tests . . . 7344
40.3.2 Axiom help page [Demm05] . . . 7344
40.3.3 fortran code . . . 7353
40.3.4 lisp code . . . 7358

41 LAPACK - General Band Matrix, Linear Solve, Real 7369
41.1 sgbsv . . . 7369
41.1.1 Axiom unit tests . . . 7369
41.1.2 Axiom help page . . . 7369
41.1.3 fortran code . . . 7372
41.1.4 lisp code . . . 7373
41.2 sgbsvx . . . 7375
41.2.1 Axiom unit tests . . . 7375
41.2.2 Axiom help page . . . 7375
41.2.3 fortran code . . . 7381
41.2.4 lisp code . . . 7387
41.3 sgbsvxx . . . 7398
41.3.1 Axiom unit tests . . . 7398
41.3.2 Axiom help page [Demm05] . . . 7398
41.3.3 fortran code . . . 7407
41.3.4 lisp code . . . 7412

42 LAPACK - General Band Matrix, Linear Solve, Complex 7423
42.1 cgbsv . . . 7423
42.1.1 Axiom unit tests . . . 7423
42.1.2 Axiom help page . . . 7423
42.1.3 fortran code . . . 7426
42.1.4 lisp code . . . 7427
42.2 cgbsvx . . . 7429
42.2.1 Axiom unit tests . . . 7429
42.2.2 Axiom help page . . . 7429

42.2.3 fortran code . . . 7435
42.2.4 lisp code . . . 7441
42.3 cgbsvxx . . . 7452
42.3.1 Axiom unit tests . . . 7452
42.3.2 Axiom help page . . . 7452
42.3.3 fortran code . . . 7461
42.3.4 lisp code . . . 7466

43 LAPACK - General Band Matrix, Linear Solve, Complex16 7477
43.1 zgbsv . . . 7477
43.1.1 Axiom unit tests . . . 7477
43.1.2 Axiom help page . . . 7477
43.1.3 fortran code . . . 7480
43.1.4 lisp code . . . 7481
43.2 zgbsvx . . . 7483
43.2.1 Axiom unit tests . . . 7483
43.2.2 Axiom help page . . . 7483
43.2.3 fortran code . . . 7489
43.2.4 lisp code . . . 7495
43.3 zgbsvxx . . . 7506
43.3.1 Axiom unit tests . . . 7506
43.3.2 Axiom help page [Demm05] . . . 7506
43.3.3 fortran code . . . 7515
43.3.4 lisp code . . . 7520

44 LAPACK - General Band Matrix, Comp. Routines, Double 7531
44.1 dgbbrd . . . 7531
44.1.1 Axiom unit tests . . . 7531
44.1.2 Axiom help page . . . 7531
44.1.3 fortran code . . . 7534
44.1.4 lisp code . . . 7541
44.2 dgbcon . . . 7556
44.2.1 Axiom unit tests . . . 7556
44.2.2 Axiom help page . . . 7556
44.2.3 fortran code . . . 7558
44.2.4 lisp code . . . 7562
44.3 dgbequ . . . 7567
44.3.1 Axiom unit tests . . . 7567
44.3.2 Axiom help page . . . 7567
44.3.3 fortran code . . . 7569
44.3.4 lisp code . . . 7573
44.4 dgbequb . . . 7577
44.4.1 Axiom unit tests . . . 7577
44.4.2 Axiom help page . . . 7578
44.4.3 fortran code . . . 7580
44.4.4 lisp code . . . 7584

44.5 dgbrfs . . . 7588
44.5.1 Axiom unit tests . . . 7588
44.5.2 Axiom help page . . . 7588
44.5.3 fortran code . . . 7591
44.5.4 lisp code . . . 7597
44.6 dgbrfsx . . . 7606
44.6.1 Axiom unit tests . . . 7606
44.6.2 Axiom help page [Demm05] . . . 7606
44.6.3 fortran code . . . 7613
44.6.4 lisp code . . . 7620
44.7 dgbtf2 . . . 7620
44.7.1 Axiom unit tests . . . 7620
44.7.2 Axiom help page . . . 7620
44.7.3 fortran code . . . 7622
44.7.4 lisp code . . . 7625
44.8 dgbtrf . . . 7628
44.8.1 Axiom unit tests . . . 7628
44.8.2 Axiom help page . . . 7629
44.8.3 fortran code . . . 7631
44.8.4 lisp code . . . 7638
44.9 dgbtrs . . . 7651
44.9.1 Axiom unit tests . . . 7651
44.9.2 Axiom help page . . . 7652
44.9.3 fortran code . . . 7654
44.9.4 lisp code . . . 7656
44.10 dggbak . . . 7661
44.10.1 Axiom unit tests . . . 7661
44.10.2 Axiom help page . . . 7661
44.10.3 fortran code . . . 7663
44.10.4 lisp code . . . 7666
44.11 dggbal . . . 7672
44.11.1 Axiom unit tests . . . 7672
44.11.2 Axiom help page . . . 7672
44.11.3 fortran code . . . 7675
44.11.4 lisp code . . . 7683
44.12 dla_gbamv . . . 7699
44.12.1 Axiom unit tests . . . 7699
44.12.2 Axiom help page . . . 7699
44.12.3 fortran code . . . 7702
44.12.4 lisp code . . . 7707
44.13 dla_gbrcond . . . 7714
44.13.1 Axiom unit tests . . . 7714
44.13.2 Axiom help page . . . 7714
44.13.3 fortran code . . . 7717
44.13.4 lisp code . . . 7721
44.14 dla_gbrfsx_extended . . . 7729

44.14.1 Axiom unit tests . . . 7729
44.14.2 Axiom help page [Demm05] . . . 7729
44.14.3 fortran code . . . 7735
44.14.4 lisp code . . . 7742
44.15 dla_gbrpvgrw . . . 7750
44.15.1 Axiom unit tests . . . 7750
44.15.2 Axiom help page . . . 7750
44.15.3 fortran code . . . 7752
44.15.4 lisp code . . . 7753
44.16 dorgbr . . . 7755
44.16.1 Axiom unit tests . . . 7755
44.16.2 Axiom help page . . . 7755
44.16.3 fortran code . . . 7758
44.16.4 lisp code . . . 7761

45 LAPACK - General Band Matrix, Comp. Routines, Real 7763
45.1 sgbbrd . . . 7763
45.1.1 Axiom unit tests . . . 7763
45.1.2 Axiom help page . . . 7763
45.1.3 fortran code . . . 7766
45.1.4 lisp code . . . 7773
45.2 sgbcon . . . 7788
45.2.1 Axiom unit tests . . . 7788
45.2.2 Axiom help page . . . 7788
45.2.3 fortran code . . . 7790
45.2.4 lisp code . . . 7794
45.3 sgbequ . . . 7799
45.3.1 Axiom unit tests . . . 7799
45.3.2 Axiom help page . . . 7799
45.3.3 fortran code . . . 7801
45.3.4 lisp code . . . 7805
45.4 sgbequb . . . 7809
45.4.1 Axiom unit tests . . . 7809
45.4.2 Axiom help page . . . 7809
45.4.3 fortran code . . . 7812
45.4.4 lisp code . . . 7815
45.5 sgbrfs . . . 7820
45.5.1 Axiom unit tests . . . 7820
45.5.2 Axiom help page . . . 7820
45.5.3 fortran code . . . 7823
45.5.4 lisp code . . . 7829
45.6 sgbrfsx . . . 7837
45.6.1 Axiom unit tests . . . 7837
45.6.2 Axiom help page [Demm05] . . . 7838
45.6.3 fortran code . . . 7845
45.6.4 lisp code . . . 7851

45.7 sgbtf2 . . . 7852
45.7.1 Axiom unit tests . . . 7852
45.7.2 Axiom help page . . . 7852
45.7.3 fortran code . . . 7854
45.7.4 lisp code . . . 7857
45.8 sgbtrf . . . 7860
45.8.1 Axiom unit tests . . . 7860
45.8.2 Axiom help page . . . 7860
45.8.3 fortran code . . . 7862
45.8.4 lisp code . . . 7870
45.9 sgbtrs . . . 7883
45.9.1 Axiom unit tests . . . 7883
45.9.2 Axiom help page . . . 7883
45.9.3 fortran code . . . 7885
45.9.4 lisp code . . . 7888
45.10 sggbak . . . 7892
45.10.1 Axiom unit tests . . . 7892
45.10.2 Axiom help page . . . 7893
45.10.3 fortran code . . . 7895
45.10.4 lisp code . . . 7898
45.11 sggbal . . . 7904
45.11.1 Axiom unit tests . . . 7904
45.11.2 Axiom help page . . . 7904
45.11.3 fortran code . . . 7906
45.11.4 lisp code . . . 7914
45.12 sla_gbamv . . . 7930
45.12.1 Axiom unit tests . . . 7930
45.12.2 Axiom help page . . . 7931
45.12.3 fortran code . . . 7934
45.12.4 lisp code . . . 7938
45.13 sla_gbrcond . . . 7946
45.13.1 Axiom unit tests . . . 7946
45.13.2 Axiom help page . . . 7946
45.13.3 fortran code . . . 7948
45.13.4 lisp code . . . 7952
45.14 sla_gbrfsx_extended . . . 7960
45.14.1 Axiom unit tests . . . 7960
45.14.2 Axiom help page [Demm05] . . . 7961
45.14.3 fortran code . . . 7967
45.14.4 lisp code . . . 7973
45.15 sla_gbrpvgrw . . . 7981
45.15.1 Axiom unit tests . . . 7981
45.15.2 Axiom help page . . . 7982
45.15.3 fortran code . . . 7983
45.15.4 lisp code . . . 7984
45.16 sorgbr . . . 7986

45.16.1 Axiom unit tests . . . 7986
45.16.2 Axiom help page . . . 7987
45.16.3 fortran code . . . 7989
45.16.4 lisp code . . . 7993

46 LAPACK - General Band Matrix, Comp. Routines, Complex 7999
46.1 cgbbrd . . . 7999
46.1.1 Axiom unit tests . . . 7999
46.1.2 Axiom help page . . . 7999
46.1.3 fortran code . . . 8002
46.1.4 lisp code . . . 8010
46.2 cgbcon . . . 8010
46.2.1 Axiom unit tests . . . 8010
46.2.2 Axiom help page . . . 8011
46.2.3 fortran code . . . 8013
46.2.4 lisp code . . . 8016
46.3 cgbequ . . . 8022
46.3.1 Axiom unit tests . . . 8022
46.3.2 Axiom help page . . . 8022
46.3.3 fortran code . . . 8024
46.3.4 lisp code . . . 8028
46.4 cgbequb . . . 8032
46.4.1 Axiom unit tests . . . 8032
46.4.2 Axiom help page . . . 8033
46.4.3 fortran code . . . 8035
46.4.4 lisp code . . . 8039
46.5 cgbrfs . . . 8044
46.5.1 Axiom unit tests . . . 8044
46.5.2 Axiom help page . . . 8044
46.5.3 fortran code . . . 8047
46.5.4 lisp code . . . 8052
46.6 cgbrfsx . . . 8061
46.6.1 Axiom unit tests . . . 8061
46.6.2 Axiom help page . . . 8062
46.6.3 fortran code . . . 8069
46.6.4 lisp code . . . 8075
46.7 cgbtf2 . . . 8076
46.7.1 Axiom unit tests . . . 8076
46.7.2 Axiom help page . . . 8076
46.7.3 fortran code . . . 8078
46.7.4 lisp code . . . 8081
46.8 cgbtrf . . . 8084
46.8.1 Axiom unit tests . . . 8084
46.8.2 Axiom help page . . . 8085
46.8.3 fortran code . . . 8087
46.8.4 lisp code . . . 8094

46.9 cgbtrs . . . 8107
46.9.1 Axiom unit tests . . . 8107
46.9.2 Axiom help page . . . 8108
46.9.3 fortran code . . . 8110
46.9.4 lisp code . . . 8113
46.10 cggbak . . . 8119
46.10.1 Axiom unit tests . . . 8119
46.10.2 Axiom help page . . . 8119
46.10.3 fortran code . . . 8121
46.10.4 lisp code . . . 8125
46.11 cggbal . . . 8130
46.11.1 Axiom unit tests . . . 8130
46.11.2 Axiom help page . . . 8131
46.11.3 fortran code . . . 8133
46.11.4 lisp code . . . 8141
46.12 cla_gbamv . . . 8158
46.12.1 Axiom unit tests . . . 8158
46.12.2 Axiom help page . . . 8159
46.12.3 fortran code . . . 8162
46.12.4 lisp code . . . 8167
46.13 cla_gbrcond_c . . . 8174
46.13.1 Axiom unit tests . . . 8174
46.13.2 Axiom help page . . . 8175
46.13.3 fortran code . . . 8177
46.13.4 lisp code . . . 8181
46.14 cla_gbrcond_x . . . 8188
46.14.1 Axiom unit tests . . . 8188
46.14.2 Axiom help page . . . 8188
46.14.3 fortran code . . . 8190
46.14.4 lisp code . . . 8194
46.15 cla_gbrfsx_extended . . . 8200
46.15.1 Axiom unit tests . . . 8200
46.15.2 Axiom help page [Demm05] . . . 8200
46.15.3 fortran code . . . 8206
46.15.4 lisp code . . . 8213
46.16 cla_gbrpvgrw . . . 8221
46.16.1 Axiom unit tests . . . 8221
46.16.2 Axiom help page . . . 8222
46.16.3 fortran code . . . 8223
46.16.4 lisp code . . . 8225
46.17 cungbr . . . 8227
46.17.1 Axiom unit tests . . . 8227
46.17.2 Axiom help page . . . 8227
46.17.3 fortran code . . . 8229
46.17.4 lisp code . . . 8233

47 LAPACK - General Band Matrix, Comp. Routines, Complex16 8239
47.1 zgbbrd . . . 8239
47.1.1 Axiom unit tests . . . 8239
47.1.2 Axiom help page . . . 8239
47.1.3 fortran code . . . 8242
47.1.4 lisp code . . . 8250
47.2 zgbcon . . . 8266
47.2.1 Axiom unit tests . . . 8266
47.2.2 Axiom help page . . . 8266
47.2.3 fortran code . . . 8268
47.2.4 lisp code . . . 8272
47.3 zgbequ . . . 8277
47.3.1 Axiom unit tests . . . 8277
47.3.2 Axiom help page . . . 8277
47.3.3 fortran code . . . 8280
47.3.4 lisp code . . . 8283
47.4 zgbequb . . . 8288
47.4.1 Axiom unit tests . . . 8288
47.4.2 Axiom help page . . . 8288
47.4.3 fortran code . . . 8290
47.4.4 lisp code . . . 8294
47.5 zgbrfs . . . 8299
47.5.1 Axiom unit tests . . . 8299
47.5.2 Axiom help page . . . 8299
47.5.3 fortran code . . . 8302
47.5.4 lisp code . . . 8308
47.6 zgbrfsx . . . 8317
47.6.1 Axiom unit tests . . . 8317
47.6.2 Axiom help page [Demm05] . . . 8317
47.6.3 fortran code . . . 8324
47.6.4 lisp code . . . 8331
47.7 zgbtf2 . . . 8331
47.7.1 Axiom unit tests . . . 8331
47.7.2 Axiom help page . . . 8331
47.7.3 fortran code . . . 8334
47.7.4 lisp code . . . 8336
47.8 zgbtrf . . . 8340
47.8.1 Axiom unit tests . . . 8340
47.8.2 Axiom help page . . . 8340
47.8.3 fortran code . . . 8342
47.8.4 lisp code . . . 8350
47.9 zgbtrs . . . 8363
47.9.1 Axiom unit tests . . . 8363
47.9.2 Axiom help page . . . 8363
47.9.3 fortran code . . . 8365
47.9.4 lisp code . . . 8368

47.10zggbak . . . 8374
47.10.1 Axiom unit tests . . . 8374
47.10.2 Axiom help page . . . 8375
47.10.3 fortran code . . . 8377
47.10.4 lisp code . . . 8380
47.11zggbal . . . 8386
47.11.1 Axiom unit tests . . . 8386
47.11.2 Axiom help page . . . 8386
47.11.3 fortran code . . . 8389
47.11.4 lisp code . . . 8397
47.12zla_gbamv . . . 8414
47.12.1 Axiom unit tests . . . 8414
47.12.2 Axiom help page . . . 8414
47.12.3 fortran code . . . 8417
47.12.4 lisp code . . . 8422
47.13zla_gbrcond_c . . . 8430
47.13.1 Axiom unit tests . . . 8430
47.13.2 Axiom help page . . . 8430
47.13.3 fortran code . . . 8432
47.13.4 lisp code . . . 8436
47.14zla_gbrcond_x . . . 8443
47.14.1 Axiom unit tests . . . 8443
47.14.2 Axiom help page . . . 8444
47.14.3 fortran code . . . 8446
47.14.4 lisp code . . . 8449
47.15zla_gbrfsx_extended . . . 8455
47.15.1 Axiom unit tests . . . 8455
47.15.2 Axiom help page [Demm05] . . . 8455
47.15.3 fortran code . . . 8462
47.15.4 lisp code . . . 8468
47.16zla_gbrpvgrw . . . 8477
47.16.1 Axiom unit tests . . . 8477
47.16.2 Axiom help page . . . 8477
47.16.3 fortran code . . . 8479
47.16.4 lisp code . . . 8480
47.17zungbr . . . 8482
47.17.1 Axiom unit tests . . . 8482
47.17.2 Axiom help page . . . 8483
47.17.3 fortran code . . . 8485
47.17.4 lisp code . . . 8489

48 LAPACK - General Band Matrix, Auxilliary Routines, Double 8495
48.1 dlangb . . . 8495
48.1.1 Axiom unit tests . . . 8495
48.1.2 Axiom help page . . . 8495
48.1.3 fortran code . . . 8497

48.1.4 lisp code . . . 8499
48.2 dlaqgb . . . 8504
48.2.1 Axiom unit tests . . . 8504
48.2.2 Axiom help page . . . 8504
48.2.3 fortran code . . . 8507
48.2.4 lisp code . . . 8509

49 LAPACK - General Band Matrix, Auxilliary Routines, Real 8513
49.1 slangb . . . 8513
49.1.1 Axiom unit tests . . . 8513
49.1.2 Axiom help page . . . 8513
49.1.3 fortran code . . . 8515
49.1.4 lisp code . . . 8517
49.2 slaqgb . . . 8522
49.2.1 Axiom unit tests . . . 8522
49.2.2 Axiom help page . . . 8522
49.2.3 fortran code . . . 8524
49.2.4 lisp code . . . 8526

50 LAPACK - General Band Matrix, Auxilliary Routines, Complex 8531
50.1 clangb . . . 8531
50.1.1 Axiom unit tests . . . 8531
50.1.2 Axiom help page . . . 8531
50.1.3 fortran code . . . 8533
50.1.4 lisp code . . . 8535
50.2 claqgb . . . 8540
50.2.1 Axiom unit tests . . . 8540
50.2.2 Axiom help page . . . 8540
50.2.3 fortran code . . . 8543
50.2.4 lisp code . . . 8545

51 LAPACK - General Band Matrix, Auxilliary Routines, Complex16 8549
51.1 zlangb . . . 8549
51.1.1 Axiom unit tests . . . 8549
51.1.2 Axiom help page . . . 8549
51.1.3 fortran code . . . 8551
51.1.4 lisp code . . . 8553
51.2 zlaqgb . . . 8558
51.2.1 Axiom unit tests . . . 8558
51.2.2 Axiom help page . . . 8558
51.2.3 fortran code . . . 8561
51.2.4 lisp code . . . 8563

52 LAPACK - Symmetric Matrix, Linear Solve, Double **8567**
52.1 dsysv . . . 8567
52.1.1 Axiom unit tests . . . 8567
52.1.2 Axiom help page . . . 8567
52.1.3 fortran code . . . 8570
52.1.4 lisp code . . . 8572
52.2 dsysv_rook . . . 8575
52.2.1 Axiom unit tests . . . 8575
52.2.2 Axiom help page . . . 8575
52.2.3 fortran code . . . 8578
52.2.4 lisp code . . . 8580
52.3 dsysvx . . . 8583
52.3.1 Axiom unit tests . . . 8583
52.3.2 Axiom help page . . . 8583
52.3.3 fortran code . . . 8587
52.3.4 lisp code . . . 8590

53 LAPACK - Symmetric Matrix, Linear Solve, Real **8597**
53.1 ssysv . . . 8597
53.1.1 Axiom unit tests . . . 8597
53.1.2 Axiom help page . . . 8597
53.1.3 fortran code . . . 8600
53.1.4 lisp code . . . 8602
53.2 ssysv_rook . . . 8605
53.2.1 Axiom unit tests . . . 8605
53.2.2 Axiom help page . . . 8605
53.2.3 fortran code . . . 8608
53.2.4 lisp code . . . 8610
53.3 ssysvx . . . 8613
53.3.1 Axiom unit tests . . . 8613
53.3.2 Axiom help page . . . 8613
53.3.3 fortran code . . . 8617
53.3.4 lisp code . . . 8620

54 LAPACK - Symmetric Matrix, Linear Solve, Complex **8625**
54.1 csysv . . . 8625
54.1.1 Axiom unit tests . . . 8625
54.1.2 Axiom help page . . . 8625
54.1.3 fortran code . . . 8628
54.1.4 lisp code . . . 8630
54.2 csysv_rook . . . 8633
54.2.1 Axiom unit tests . . . 8633
54.2.2 Axiom help page . . . 8633
54.2.3 fortran code . . . 8636
54.2.4 lisp code . . . 8638
54.3 csysvx . . . 8641

54.3.1 Axiom unit tests . . . 8641
54.3.2 Axiom help page . . . 8641
54.3.3 fortran code . . . 8645
54.3.4 lisp code . . . 8648
54.4 csysvxx . . . 8653
54.4.1 Axiom unit tests . . . 8653
54.4.2 Axiom help page . . . 8653
54.4.3 fortran code . . . 8661
54.4.4 lisp code . . . 8665

55 LAPACK - Symmetric Matrix, Linear Solve, Complex16 8673
55.1 zsysv . . . 8673
55.1.1 Axiom unit tests . . . 8673
55.1.2 Axiom help page . . . 8673
55.1.3 fortran code . . . 8676
55.1.4 lisp code . . . 8678
55.2 zsysv_rook . . . 8681
55.2.1 Axiom unit tests . . . 8681
55.2.2 Axiom help page . . . 8681
55.2.3 fortran code . . . 8684
55.2.4 lisp code . . . 8686
55.3 zsysvx . . . 8689
55.3.1 Axiom unit tests . . . 8689
55.3.2 Axiom help page . . . 8689
55.3.3 fortran code . . . 8693
55.3.4 lisp code . . . 8696
55.4 zsysvxx . . . 8701
55.4.1 Axiom unit tests . . . 8701
55.4.2 Axiom help page [Demm05] . . . 8701
55.4.3 fortran code . . . 8709
55.4.4 lisp code . . . 8714

56 LAPACK - Symmetric Matrix, Eigenvalue, Double 8721
56.1 dsyev . . . 8721
56.1.1 Axiom unit tests . . . 8721
56.1.2 Axiom help page . . . 8721
56.1.3 fortran code . . . 8723
56.1.4 lisp code . . . 8727
56.2 dsyevd . . . 8731
56.2.1 Axiom unit tests . . . 8731
56.2.2 Axiom help page . . . 8732
56.2.3 fortran code . . . 8734
56.2.4 lisp code . . . 8738
56.3 dsyevr . . . 8743
56.3.1 Axiom unit tests . . . 8743
56.3.2 Axiom help page . . . 8743

56.3.3 fortran code . . . 8748
56.3.4 lisp code . . . 8755
56.4 dsyevx . . . 8766
56.4.1 Axiom unit tests . . . 8766
56.4.2 Axiom help page . . . 8767
56.4.3 fortran code . . . 8770
56.4.4 lisp code . . . 8777
56.5 dsygv . . . 8787
56.5.1 Axiom unit tests . . . 8787
56.5.2 Axiom help page . . . 8788
56.5.3 fortran code . . . 8790
56.5.4 lisp code . . . 8793
56.6 dsygvd . . . 8797
56.6.1 Axiom unit tests . . . 8797
56.6.2 Axiom help page . . . 8798
56.6.3 fortran code . . . 8801
56.6.4 lisp code . . . 8804
56.7 dsygvx . . . 8808
56.7.1 Axiom unit tests . . . 8808
56.7.2 Axiom help page . . . 8809
56.7.3 fortran code . . . 8813
56.7.4 lisp code . . . 8817

57 LAPACK - Symmetric Matrix, Eigenvalue, Real 8823
57.1 ssyev . . . 8823
57.1.1 Axiom unit tests . . . 8823
57.1.2 Axiom help page . . . 8823
57.1.3 fortran code . . . 8825
57.1.4 lisp code . . . 8829
57.2 ssyevd . . . 8833
57.2.1 Axiom unit tests . . . 8833
57.2.2 Axiom help page . . . 8834
57.2.3 fortran code . . . 8836
57.2.4 lisp code . . . 8840
57.3 ssyevr . . . 8845
57.3.1 Axiom unit tests . . . 8845
57.3.2 Axiom help page . . . 8845
57.3.3 fortran code . . . 8850
57.3.4 lisp code . . . 8857
57.4 ssyevx . . . 8868
57.4.1 Axiom unit tests . . . 8868
57.4.2 Axiom help page . . . 8869
57.4.3 fortran code . . . 8872
57.4.4 lisp code . . . 8878
57.5 ssygv . . . 8889
57.5.1 Axiom unit tests . . . 8889

57.5.2 Axiom help page . . . 8889
57.5.3 fortran code . . . 8892
57.5.4 lisp code . . . 8895
57.6 ssygvd . . . 8899
57.6.1 Axiom unit tests . . . 8899
57.6.2 Axiom help page . . . 8899
57.6.3 fortran code . . . 8902
57.6.4 lisp code . . . 8906
57.7 ssygvx . . . 8910
57.7.1 Axiom unit tests . . . 8910
57.7.2 Axiom help page . . . 8910
57.7.3 fortran code . . . 8914
57.7.4 lisp code . . . 8918

58 LAPACK - Symmetric Matrix, Comp. Routines, Double 8925
58.1 dla_syamv . . . 8925
58.1.1 Axiom unit tests . . . 8925
58.1.2 Axiom help page . . . 8925
58.1.3 fortran code . . . 8928
58.1.4 lisp code . . . 8933
58.2 dla_syrcond . . . 8941
58.2.1 Axiom unit tests . . . 8941
58.2.2 Axiom help page . . . 8941
58.2.3 fortran code . . . 8943
58.2.4 lisp code . . . 8947
58.3 dla_syrfsx_extended . . . 8955
58.3.1 Axiom unit tests . . . 8955
58.3.2 Axiom help page [Demm05] . . . 8955
58.3.3 fortran code . . . 8961
58.3.4 lisp code . . . 8968
58.4 dla_syrpvgrw . . . 8976
58.4.1 Axiom unit tests . . . 8976
58.4.2 Axiom help page . . . 8976
58.4.3 fortran code . . . 8978
58.4.4 lisp code . . . 8982
58.5 dlasyf . . . 8990
58.5.1 Axiom unit tests . . . 8990
58.5.2 Axiom help page . . . 8990
58.5.3 fortran code . . . 8992
58.5.4 lisp code . . . 9006
58.6 dlasyf_rook . . . 9023
58.6.1 Axiom unit tests . . . 9023
58.6.2 Axiom help page . . . 9024
58.6.3 fortran code . . . 9026
58.6.4 lisp code . . . 9041
58.7 dsycon . . . 9063

58.7.1 Axiom unit tests . . . 9063
58.7.2 Axiom help page . . . 9063
58.7.3 fortran code . . . 9065
58.7.4 lisp code . . . 9068
58.8 dsycon_rook . . . 9070
58.8.1 Axiom unit tests . . . 9070
58.8.2 Axiom help page . . . 9071
58.8.3 fortran code . . . 9072
58.8.4 lisp code . . . 9075
58.9 dsyconv . . . 9078
58.9.1 Axiom unit tests . . . 9078
58.9.2 Axiom help page . . . 9078
58.9.3 fortran code . . . 9080
58.9.4 lisp code . . . 9085
58.10 dsyequb . . . 9092
58.10.1 Axiom unit tests . . . 9092
58.10.2 Axiom help page . . . 9093
58.10.3 fortran code . . . 9095
58.10.4 lisp code . . . 9099
58.11 dsygs2 . . . 9108
58.11.1 Axiom unit tests . . . 9108
58.11.2 Axiom help page . . . 9109
58.11.3 fortran code . . . 9110
58.11.4 lisp code . . . 9114
58.12 dsygst . . . 9121
58.12.1 Axiom unit tests . . . 9121
58.12.2 Axiom help page . . . 9122
58.12.3 fortran code . . . 9124
58.12.4 lisp code . . . 9128
58.13 dsyrfs . . . 9140
58.13.1 Axiom unit tests . . . 9140
58.13.2 Axiom help page . . . 9140
58.13.3 fortran code . . . 9143
58.13.4 lisp code . . . 9148
58.14 dsyrfsx . . . 9157
58.14.1 Axiom unit tests . . . 9157
58.14.2 Axiom help page [Demm05] . . . 9157
58.14.3 fortran code . . . 9163
58.14.4 lisp code . . . 9170
58.15 dsytd2 . . . 9170
58.15.1 Axiom unit tests . . . 9170
58.15.2 Axiom help page . . . 9170
58.15.3 fortran code . . . 9173
58.15.4 lisp code . . . 9176
58.16 dsytf2 . . . 9181
58.16.1 Axiom unit tests . . . 9181

58.16.2 Axiom help page . . . 9182
58.16.3 fortran code . . . 9184
58.16.4 lisp code . . . 9193
58.17 dsytf2_rook . . . 9204
58.17.1 Axiom unit tests . . . 9204
58.17.2 Axiom help page . . . 9204
58.17.3 fortran code . . . 9207
58.17.4 lisp code . . . 9211
58.18 dsytrd . . . 9225
58.18.1 Axiom unit tests . . . 9225
58.18.2 Axiom help page . . . 9225
58.18.3 fortran code . . . 9228
58.18.4 lisp code . . . 9232
58.19 dsytrf . . . 9237
58.19.1 Axiom unit tests . . . 9237
58.19.2 Axiom help page . . . 9238
58.19.3 fortran code . . . 9241
58.19.4 lisp code . . . 9244
58.20 dsytrf_rook . . . 9249
58.20.1 Axiom unit tests . . . 9249
58.20.2 Axiom help page . . . 9249
58.20.3 fortran code . . . 9252
58.20.4 lisp code . . . 9256
58.21 dsytri . . . 9260
58.21.1 Axiom unit tests . . . 9260
58.21.2 Axiom help page . . . 9260
58.21.3 fortran code . . . 9262
58.21.4 lisp code . . . 9268
58.22 dsytri2 . . . 9278
58.22.1 Axiom unit tests . . . 9278
58.22.2 Axiom help page . . . 9278
58.22.3 fortran code . . . 9280
58.22.4 lisp code . . . 9282
58.23 dsytri2x . . . 9284
58.23.1 Axiom unit tests . . . 9284
58.23.2 Axiom help page . . . 9284
58.23.3 fortran code . . . 9286
58.23.4 lisp code . . . 9295
58.24 dsytri_rook . . . 9317
58.24.1 Axiom unit tests . . . 9317
58.24.2 Axiom help page . . . 9317
58.24.3 fortran code . . . 9319
58.24.4 lisp code . . . 9325
58.25 dsytrs . . . 9338
58.25.1 Axiom unit tests . . . 9338
58.25.2 Axiom help page . . . 9338

58.25.3 fortran code . . . 9340
58.25.4 lisp code . . . 9347
58.26 dsytrs2 . . . 9358
58.26.1 Axiom unit tests . . . 9358
58.26.2 Axiom help page . . . 9358
58.26.3 fortran code . . . 9360
58.26.4 lisp code . . . 9365
58.27 dsytrs_rook . . . 9374
58.27.1 Axiom unit tests . . . 9374
58.27.2 Axiom help page . . . 9374
58.27.3 fortran code . . . 9376
58.27.4 lisp code . . . 9383
58.28 dtgsyl . . . 9395
58.28.1 Axiom unit tests . . . 9395
58.28.2 Axiom help page . . . 9396
58.28.3 fortran code . . . 9400
58.28.4 lisp code . . . 9408
58.29 dtrsyl . . . 9427
58.29.1 Axiom unit tests . . . 9427
58.29.2 Axiom help page . . . 9427
58.29.3 fortran code . . . 9430
58.29.4 lisp code . . . 9447

59 LAPACK - Symmetric Matrix, Comp. Routines, Real 9503
59.1 sla_syamv . . . 9503
59.1.1 Axiom unit tests . . . 9503
59.1.2 Axiom help page . . . 9503
59.1.3 fortran code . . . 9506
59.1.4 lisp code . . . 9511
59.2 sla_syrcond . . . 9519
59.2.1 Axiom unit tests . . . 9519
59.2.2 Axiom help page . . . 9519
59.2.3 fortran code . . . 9521
59.2.4 lisp code . . . 9525
59.3 sla_syrfsx_extended . . . 9532
59.3.1 Axiom unit tests . . . 9532
59.3.2 Axiom help page [Demm05] . . . 9533
59.3.3 fortran code . . . 9539
59.3.4 lisp code . . . 9545
59.4 sla_syrpvgrw . . . 9554
59.4.1 Axiom unit tests . . . 9554
59.4.2 Axiom help page . . . 9554
59.4.3 fortran code . . . 9556
59.4.4 lisp code . . . 9560
59.5 slasyf . . . 9567
59.5.1 Axiom unit tests . . . 9567

59.5.2 Axiom help page . . . 9568
59.5.3 fortran code . . . 9570
59.5.4 lisp code . . . 9583
59.6 slasyf_rook . . . 9601
59.6.1 Axiom unit tests . . . 9601
59.6.2 Axiom help page . . . 9601
59.6.3 fortran code . . . 9604
59.6.4 lisp code . . . 9618
59.7 ssycon . . . 9641
59.7.1 Axiom unit tests . . . 9641
59.7.2 Axiom help page . . . 9641
59.7.3 fortran code . . . 9643
59.7.4 lisp code . . . 9645
59.8 ssycon_rook . . . 9648
59.8.1 Axiom unit tests . . . 9648
59.8.2 Axiom help page . . . 9648
59.8.3 fortran code . . . 9650
59.8.4 lisp code . . . 9653
59.9 ssyconv . . . 9655
59.9.1 Axiom unit tests . . . 9655
59.9.2 Axiom help page . . . 9656
59.9.3 fortran code . . . 9657
59.9.4 lisp code . . . 9662
59.10 ssyequb . . . 9670
59.10.1 Axiom unit tests . . . 9670
59.10.2 Axiom help page . . . 9670
59.10.3 fortran code . . . 9672
59.10.4 lisp code . . . 9676
59.11 ssygs2 . . . 9686
59.11.1 Axiom unit tests . . . 9686
59.11.2 Axiom help page . . . 9686
59.11.3 fortran code . . . 9688
59.11.4 lisp code . . . 9691
59.12 ssygst . . . 9699
59.12.1 Axiom unit tests . . . 9699
59.12.2 Axiom help page . . . 9699
59.12.3 fortran code . . . 9701
59.12.4 lisp code . . . 9705
59.13 ssyrfs . . . 9717
59.13.1 Axiom unit tests . . . 9717
59.13.2 Axiom help page . . . 9718
59.13.3 fortran code . . . 9720
59.13.4 lisp code . . . 9726
59.14 ssyrfsx . . . 9734
59.14.1 Axiom unit tests . . . 9734
59.14.2 Axiom help page [Demm05] . . . 9735

59.14.3 fortran code . . . 9741
59.14.4 lisp code . . . 9747
59.15 ssytd2 . . . 9748
59.15.1 Axiom unit tests . . . 9748
59.15.2 Axiom help page . . . 9748
59.15.3 fortran code . . . 9751
59.15.4 lisp code . . . 9754
59.16 ssytf2 . . . 9759
59.16.1 Axiom unit tests . . . 9759
59.16.2 Axiom help page . . . 9759
59.16.3 fortran code . . . 9762
59.16.4 lisp code . . . 9771
59.17 ssytf2_rook . . . 9781
59.17.1 Axiom unit tests . . . 9781
59.17.2 Axiom help page . . . 9782
59.17.3 fortran code . . . 9784
59.17.4 lisp code . . . 9797
59.18 ssytrd . . . 9811
59.18.1 Axiom unit tests . . . 9811
59.18.2 Axiom help page . . . 9811
59.18.3 fortran code . . . 9814
59.18.4 lisp code . . . 9818
59.19 ssytrf . . . 9823
59.19.1 Axiom unit tests . . . 9823
59.19.2 Axiom help page . . . 9824
59.19.3 fortran code . . . 9827
59.19.4 lisp code . . . 9830
59.20 ssytrf_rook . . . 9835
59.20.1 Axiom unit tests . . . 9835
59.20.2 Axiom help page . . . 9835
59.20.3 fortran code . . . 9838
59.20.4 lisp code . . . 9842
59.21 ssytri . . . 9846
59.21.1 Axiom unit tests . . . 9846
59.21.2 Axiom help page . . . 9846
59.21.3 fortran code . . . 9848
59.21.4 lisp code . . . 9854
59.22 ssytri2 . . . 9864
59.22.1 Axiom unit tests . . . 9864
59.22.2 Axiom help page . . . 9864
59.22.3 fortran code . . . 9866
59.22.4 lisp code . . . 9867
59.23 ssytri2x . . . 9870
59.23.1 Axiom unit tests . . . 9870
59.23.2 Axiom help page . . . 9870
59.23.3 fortran code . . . 9872

59.23.4 lisp code . . . 9881
59.24 ssytri_rook . . . 9903
59.24.1 Axiom unit tests . . . 9903
59.24.2 Axiom help page . . . 9903
59.24.3 fortran code . . . 9905
59.24.4 lisp code . . . 9911
59.25 ssytrs . . . 9924
59.25.1 Axiom unit tests . . . 9924
59.25.2 Axiom help page . . . 9924
59.25.3 fortran code . . . 9926
59.25.4 lisp code . . . 9933
59.26 ssytrs2 . . . 9944
59.26.1 Axiom unit tests . . . 9944
59.26.2 Axiom help page . . . 9944
59.26.3 fortran code . . . 9946
59.26.4 lisp code . . . 9951
59.27 ssytrs_rook . . . 9960
59.27.1 Axiom unit tests . . . 9960
59.27.2 Axiom help page . . . 9960
59.27.3 fortran code . . . 9962
59.27.4 lisp code . . . 9969
59.28 stgsyl . . . 9981
59.28.1 Axiom unit tests . . . 9981
59.28.2 Axiom help page . . . 9982
59.28.3 fortran code . . . 9986
59.28.4 lisp code . . . 9994
59.29 strsyl . . . 10011
59.29.1 Axiom unit tests . . . 10011
59.29.2 Axiom help page . . . 10011
59.29.3 fortran code . . . 10013
59.29.4 lisp code . . . 10030

60 LAPACK - Symmetric Matrix, Comp. Routines, Complex 10085
60.1 cla_syamv . . . 10085
60.1.1 Axiom unit tests . . . 10085
60.1.2 Axiom help page . . . 10085
60.1.3 fortran code . . . 10088
60.1.4 lisp code . . . 10093
60.2 cla_syrcond_c . . . 10101
60.2.1 Axiom unit tests . . . 10101
60.2.2 Axiom help page . . . 10102
60.2.3 fortran code . . . 10104
60.2.4 lisp code . . . 10108
60.3 cla_syrcond_x . . . 10114
60.3.1 Axiom unit tests . . . 10114
60.3.2 Axiom help page . . . 10114

60.3.3 fortran code . . . 10116
60.3.4 lisp code . . . 10120
60.4 cla_syrfsx_extended . . . 10125
60.4.1 Axiom unit tests . . . 10125
60.4.2 Axiom help page [Demm05] . . . 10125
60.4.3 fortran code . . . 10131
60.4.4 lisp code . . . 10138
60.5 cla_syrpvgrw . . . 10147
60.5.1 Axiom unit tests . . . 10147
60.5.2 Axiom help page . . . 10147
60.5.3 fortran code . . . 10149
60.5.4 lisp code . . . 10153
60.6 clasyf . . . 10161
60.6.1 Axiom unit tests . . . 10161
60.6.2 Axiom help page . . . 10161
60.6.3 fortran code . . . 10164
60.6.4 lisp code . . . 10177
60.7 clasyf_rook . . . 10196
60.7.1 Axiom unit tests . . . 10196
60.7.2 Axiom help page . . . 10196
60.7.3 fortran code . . . 10199
60.7.4 lisp code . . . 10213
60.8 csycon . . . 10238
60.8.1 Axiom unit tests . . . 10238
60.8.2 Axiom help page . . . 10238
60.8.3 fortran code . . . 10240
60.8.4 lisp code . . . 10242
60.9 csycon_rook . . . 10245
60.9.1 Axiom unit tests . . . 10245
60.9.2 Axiom help page . . . 10245
60.9.3 fortran code . . . 10247
60.9.4 lisp code . . . 10249
60.10 csyconv . . . 10252
60.10.1 Axiom unit tests . . . 10252
60.10.2 Axiom help page . . . 10252
60.10.3 fortran code . . . 10254
60.10.4 lisp code . . . 10259
60.11 csyequb . . . 10267
60.11.1 Axiom unit tests . . . 10267
60.11.2 Axiom help page . . . 10267
60.11.3 fortran code . . . 10269
60.11.4 lisp code . . . 10273
60.12 csyrfs . . . 10283
60.12.1 Axiom unit tests . . . 10283
60.12.2 Axiom help page . . . 10284
60.12.3 fortran code . . . 10286

60.12.4 lisp code . . . 10292
60.13 csyrfsx . . . 10300
60.13.1 Axiom unit tests . . . 10300
60.13.2 Axiom help page [Demm05] . . . 10301
60.13.3 fortran code . . . 10307
60.13.4 lisp code . . . 10313
60.14 csytf2 . . . 10313
60.14.1 Axiom unit tests . . . 10313
60.14.2 Axiom help page . . . 10314
60.14.3 fortran code . . . 10317
60.14.4 lisp code . . . 10325
60.15 csytf2_rook . . . 10337
60.15.1 Axiom unit tests . . . 10337
60.15.2 Axiom help page . . . 10337
60.15.3 fortran code . . . 10340
60.15.4 lisp code . . . 10353
60.16 csytrf . . . 10368
60.16.1 Axiom unit tests . . . 10368
60.16.2 Axiom help page . . . 10368
60.16.3 fortran code . . . 10371
60.16.4 lisp code . . . 10375
60.17 csytrf_rook . . . 10379
60.17.1 Axiom unit tests . . . 10379
60.17.2 Axiom help page . . . 10380
60.17.3 fortran code . . . 10383
60.17.4 lisp code . . . 10386
60.18 csytri . . . 10391
60.18.1 Axiom unit tests . . . 10391
60.18.2 Axiom help page . . . 10391
60.18.3 fortran code . . . 10393
60.18.4 lisp code . . . 10398
60.19 csytri2 . . . 10408
60.19.1 Axiom unit tests . . . 10408
60.19.2 Axiom help page . . . 10409
60.19.3 fortran code . . . 10411
60.19.4 lisp code . . . 10412
60.20 csytri2x . . . 10415
60.20.1 Axiom unit tests . . . 10415
60.20.2 Axiom help page . . . 10415
60.20.3 fortran code . . . 10417
60.20.4 lisp code . . . 10426
60.21 csytri_rook . . . 10448
60.21.1 Axiom unit tests . . . 10448
60.21.2 Axiom help page . . . 10448
60.21.3 fortran code . . . 10450
60.21.4 lisp code . . . 10456

60.22csytrs . . . 10469
60.22.1 Axiom unit tests . . . 10469
60.22.2 Axiom help page . . . 10470
60.22.3 fortran code . . . 10471
60.22.4 lisp code . . . 10478
60.23csytrs2 . . . 10489
60.23.1 Axiom unit tests . . . 10489
60.23.2 Axiom help page . . . 10490
60.23.3 fortran code . . . 10492
60.23.4 lisp code . . . 10496
60.24csytrs_rook . . . 10505
60.24.1 Axiom unit tests . . . 10505
60.24.2 Axiom help page . . . 10506
60.24.3 fortran code . . . 10507
60.24.4 lisp code . . . 10514
60.25ctgsyl . . . 10527
60.25.1 Axiom unit tests . . . 10527
60.25.2 Axiom help page . . . 10528
60.25.3 fortran code . . . 10532
60.25.4 lisp code . . . 10540
60.26ctrsyl . . . 10557
60.26.1 Axiom unit tests . . . 10557
60.26.2 Axiom help page . . . 10557
60.26.3 fortran code . . . 10559
60.26.4 lisp code . . . 10566

61 LAPACK - Symmetric Matrix, Comp. Routines, Complex16 10577
61.1 zla_syamv . . . 10577
61.1.1 Axiom unit tests . . . 10577
61.1.2 Axiom help page . . . 10577
61.1.3 fortran code . . . 10580
61.1.4 lisp code . . . 10585
61.2 zla_syrcond_c . . . 10593
61.2.1 Axiom unit tests . . . 10593
61.2.2 Axiom help page . . . 10594
61.2.3 fortran code . . . 10595
61.2.4 lisp code . . . 10599
61.3 zla_syrcond_x . . . 10606
61.3.1 Axiom unit tests . . . 10606
61.3.2 Axiom help page . . . 10606
61.3.3 fortran code . . . 10608
61.3.4 lisp code . . . 10612
61.4 zla_syrfsx_extended . . . 10617
61.4.1 Axiom unit tests . . . 10617
61.4.2 Axiom help page [Demm05] . . . 10617
61.4.3 fortran code . . . 10624

61.4.4 lisp code . . . 10630
61.5 zla_syrpvgrw . . . 10639
61.5.1 Axiom unit tests . . . 10639
61.5.2 Axiom help page . . . 10639
61.5.3 fortran code . . . 10641
61.5.4 lisp code . . . 10645
61.6 zlasyf . . . 10653
61.6.1 Axiom unit tests . . . 10653
61.6.2 Axiom help page . . . 10654
61.6.3 fortran code . . . 10656
61.6.4 lisp code . . . 10669
61.7 zlasyf_rook . . . 10670
61.7.1 Axiom unit tests . . . 10670
61.7.2 Axiom help page . . . 10670
61.7.3 fortran code . . . 10673
61.7.4 lisp code . . . 10687
61.8 zsycon . . . 10687
61.8.1 Axiom unit tests . . . 10687
61.8.2 Axiom help page . . . 10688
61.8.3 fortran code . . . 10689
61.8.4 lisp code . . . 10692
61.9 zsycon_rook . . . 10695
61.9.1 Axiom unit tests . . . 10695
61.9.2 Axiom help page . . . 10695
61.9.3 fortran code . . . 10697
61.9.4 lisp code . . . 10699
61.10 zsyconv . . . 10702
61.10.1 Axiom unit tests . . . 10702
61.10.2 Axiom help page . . . 10702
61.10.3 fortran code . . . 10704
61.10.4 lisp code . . . 10709
61.11 zsyequb . . . 10717
61.11.1 Axiom unit tests . . . 10717
61.11.2 Axiom help page . . . 10717
61.11.3 fortran code . . . 10719
61.11.4 lisp code . . . 10723
61.12 zsyrfs . . . 10733
61.12.1 Axiom unit tests . . . 10733
61.12.2 Axiom help page . . . 10734
61.12.3 fortran code . . . 10736
61.12.4 lisp code . . . 10742
61.13 zsyrfsx . . . 10750
61.13.1 Axiom unit tests . . . 10750
61.13.2 Axiom help page [Demm05] . . . 10751
61.13.3 fortran code . . . 10757
61.13.4 lisp code . . . 10763

61.14zsytf2 . . . 10764
61.14.1 Axiom unit tests . . . 10764
61.14.2 Axiom help page . . . 10764
61.14.3 fortran code . . . 10767
61.14.4 lisp code . . . 10775
61.15zsytf2_rook . . . 10776
61.15.1 Axiom unit tests . . . 10776
61.15.2 Axiom help page . . . 10776
61.15.3 fortran code . . . 10779
61.15.4 lisp code . . . 10783
61.16zsytrf . . . 10783
61.16.1 Axiom unit tests . . . 10783
61.16.2 Axiom help page . . . 10783
61.16.3 fortran code . . . 10786
61.16.4 lisp code . . . 10790
61.17zsytrf_rook . . . 10794
61.17.1 Axiom unit tests . . . 10794
61.17.2 Axiom help page . . . 10795
61.17.3 fortran code . . . 10798
61.17.4 lisp code . . . 10801
61.18zsytri . . . 10806
61.18.1 Axiom unit tests . . . 10806
61.18.2 Axiom help page . . . 10806
61.18.3 fortran code . . . 10808
61.18.4 lisp code . . . 10813
61.19zsytri2 . . . 10824
61.19.1 Axiom unit tests . . . 10824
61.19.2 Axiom help page . . . 10824
61.19.3 fortran code . . . 10826
61.19.4 lisp code . . . 10828
61.20zsytri2x . . . 10830
61.20.1 Axiom unit tests . . . 10830
61.20.2 Axiom help page . . . 10830
61.20.3 fortran code . . . 10832
61.20.4 lisp code . . . 10841
61.21zsytri_rook . . . 10863
61.21.1 Axiom unit tests . . . 10863
61.21.2 Axiom help page . . . 10863
61.21.3 fortran code . . . 10865
61.21.4 lisp code . . . 10871
61.22zsytrs . . . 10885
61.22.1 Axiom unit tests . . . 10885
61.22.2 Axiom help page . . . 10885
61.22.3 fortran code . . . 10887
61.22.4 lisp code . . . 10893
61.23zsytrs2 . . . 10905

61.23.1 Axiom unit tests . . . 10905
61.23.2 Axiom help page . . . 10905
61.23.3 fortran code . . . 10907
61.23.4 lisp code . . . 10912
61.24 zsytrs_rook . . . 10921
61.24.1 Axiom unit tests . . . 10921
61.24.2 Axiom help page . . . 10921
61.24.3 fortran code . . . 10923
61.24.4 lisp code . . . 10930
61.25 ztgsyl . . . 10943
61.25.1 Axiom unit tests . . . 10943
61.25.2 Axiom help page . . . 10943
61.25.3 fortran code . . . 10947
61.25.4 lisp code . . . 10955
61.26 ztrsyl . . . 10972
61.26.1 Axiom unit tests . . . 10972
61.26.2 Axiom help page . . . 10973
61.26.3 fortran code . . . 10975
61.26.4 lisp code . . . 10981

62 LAPACK - Symmetric Matrix, Auxilliary Routines, Double 10993
62.1 dlansy . . . 10993
62.1.1 Axiom unit tests . . . 10993
62.1.2 Axiom help page . . . 10993
62.1.3 fortran code . . . 10995
62.1.4 lisp code . . . 10998
62.2 dlaqsy . . . 11003
62.2.1 Axiom unit tests . . . 11003
62.2.2 Axiom help page . . . 11003
62.2.3 fortran code . . . 11005
62.2.4 lisp code . . . 11007
62.3 dlasy2 . . . 11009
62.3.1 Axiom unit tests . . . 11009
62.3.2 Axiom help page . . . 11009
62.3.3 fortran code . . . 11012
62.3.4 lisp code . . . 11018
62.4 dsyswapr . . . 11019
62.4.1 Axiom unit tests . . . 11019
62.4.2 Axiom help page . . . 11019
62.4.3 fortran code . . . 11020
62.4.4 lisp code . . . 11022
62.5 dtgsy2 . . . 11023
62.5.1 Axiom unit tests . . . 11023
62.5.2 Axiom help page . . . 11023
62.5.3 fortran code . . . 11027
62.5.4 lisp code . . . 11043

63 LAPACK - Symmetric Matrix, Auxilliary Routines, Real **11083**
63.1 slansy . . . 11083
63.1.1 Axiom unit tests . . . 11083
63.1.2 Axiom help page . . . 11083
63.1.3 fortran code . . . 11085
63.1.4 lisp code . . . 11088
63.2 slaqsy . . . 11093
63.2.1 Axiom unit tests . . . 11093
63.2.2 Axiom help page . . . 11093
63.2.3 fortran code . . . 11095
63.2.4 lisp code . . . 11097
63.3 slasy2 . . . 11099
63.3.1 Axiom unit tests . . . 11099
63.3.2 Axiom help page . . . 11099
63.3.3 fortran code . . . 11101
63.3.4 lisp code . . . 11108
63.4 ssyswapr . . . 11120
63.4.1 Axiom unit tests . . . 11120
63.4.2 Axiom help page . . . 11120
63.4.3 fortran code . . . 11121
63.4.4 lisp code . . . 11123
63.5 stgsy2 . . . 11124
63.5.1 Axiom unit tests . . . 11124
63.5.2 Axiom help page . . . 11124
63.5.3 fortran code . . . 11128
63.5.4 lisp code . . . 11144

64 LAPACK - Symmetric Matrix, Auxilliary Routines, Complex **11185**
64.1 claesy . . . 11185
64.1.1 Axiom unit tests . . . 11185
64.1.2 Axiom help page . . . 11185
64.1.3 fortran code . . . 11187
64.1.4 lisp code . . . 11189
64.2 clansy . . . 11191
64.2.1 Axiom unit tests . . . 11191
64.2.2 Axiom help page . . . 11191
64.2.3 fortran code . . . 11193
64.2.4 lisp code . . . 11196
64.3 claqsy . . . 11201
64.3.1 Axiom unit tests . . . 11201
64.3.2 Axiom help page . . . 11201
64.3.3 fortran code . . . 11203
64.3.4 lisp code . . . 11205
64.4 csymv . . . 11207
64.4.1 Axiom unit tests . . . 11207
64.4.2 Axiom help page . . . 11208

64.4.3 fortran code . . . 11210
64.4.4 lisp code . . . 11214
64.5 csyr . . . 11219
64.5.1 Axiom unit tests . . . 11219
64.5.2 Axiom help page . . . 11220
64.5.3 fortran code . . . 11222
64.5.4 lisp code . . . 11225
64.6 csyswapr . . . 11228
64.6.1 Axiom unit tests . . . 11228
64.6.2 Axiom help page . . . 11229
64.6.3 fortran code . . . 11230
64.6.4 lisp code . . . 11232
64.7 ctgsy2 . . . 11232
64.7.1 Axiom unit tests . . . 11232
64.7.2 Axiom help page . . . 11233
64.7.3 fortran code . . . 11236
64.7.4 lisp code . . . 11241

65 LAPACK - Symmetric Matrix, Auxilliary Routines, Complex16 11251
65.1 zlaesy . . . 11251
65.1.1 Axiom unit tests . . . 11251
65.1.2 Axiom help page . . . 11251
65.1.3 fortran code . . . 11253
65.1.4 lisp code . . . 11255
65.2 zlansy . . . 11257
65.2.1 Axiom unit tests . . . 11257
65.2.2 Axiom help page . . . 11257
65.2.3 fortran code . . . 11259
65.2.4 lisp code . . . 11262
65.3 zlaqsy . . . 11267
65.3.1 Axiom unit tests . . . 11267
65.3.2 Axiom help page . . . 11268
65.3.3 fortran code . . . 11269
65.3.4 lisp code . . . 11271
65.4 zsymv . . . 11273
65.4.1 Axiom unit tests . . . 11273
65.4.2 Axiom help page . . . 11274
65.4.3 fortran code . . . 11276
65.4.4 lisp code . . . 11280
65.5 zsyr . . . 11285
65.5.1 Axiom unit tests . . . 11285
65.5.2 Axiom help page . . . 11286
65.5.3 fortran code . . . 11288
65.5.4 lisp code . . . 11291
65.6 zsyswapr . . . 11294
65.6.1 Axiom unit tests . . . 11294

65.6.2 Axiom help page . . . 11295
65.6.3 fortran code . . . 11296
65.6.4 lisp code . . . 11298
65.7 ztgsy2 . . . 11298
65.7.1 Axiom unit tests . . . 11298
65.7.2 Axiom help page . . . 11299
65.7.3 fortran code . . . 11302
65.7.4 lisp code . . . 11307

66 LAPACK - Positive Definite Matrix, Linear Solve, Double 11317
66.1 dposv . . . 11317
66.1.1 Axiom unit tests . . . 11317
66.1.2 Axiom help page . . . 11317
66.1.3 fortran code . . . 11319
66.1.4 lisp code . . . 11321
66.2 dposvx . . . 11323
66.2.1 Axiom unit tests . . . 11323
66.2.2 Axiom help page . . . 11323
66.2.3 fortran code . . . 11328
66.2.4 lisp code . . . 11332
66.3 dposvxx . . . 11338
66.3.1 Axiom unit tests . . . 11338
66.3.2 Axiom help page [Demm05] . . . 11338
66.3.3 fortran code . . . 11346
66.3.4 lisp code . . . 11350
66.4 dsposv . . . 11356
66.4.1 Axiom unit tests . . . 11356
66.4.2 Axiom help page . . . 11356
66.4.3 fortran code . . . 11359
66.4.4 lisp code . . . 11364

67 LAPACK - Positive Definite Matrix, Linear Solve, Real 11373
67.1 sposv . . . 11373
67.1.1 Axiom unit tests . . . 11373
67.1.2 Axiom help page . . . 11373
67.1.3 fortran code . . . 11375
67.1.4 lisp code . . . 11377
67.2 sposvx . . . 11379
67.2.1 Axiom unit tests . . . 11379
67.2.2 Axiom help page . . . 11379
67.2.3 fortran code . . . 11384
67.2.4 lisp code . . . 11388
67.3 sposvxx . . . 11394
67.3.1 Axiom unit tests . . . 11394
67.3.2 Axiom help page [Demm05] . . . 11394
67.3.3 fortran code . . . 11402

67.3.4 lisp code 11406

68 LAPACK - Positive Definite Matrix, Linear Solve, Complex 11413
68.1 cposv 11413
68.1.1 Axiom unit tests 11413
68.1.2 Axiom help page 11413
68.1.3 fortran code 11415
68.1.4 lisp code 11417
68.2 cposvx 11419
68.2.1 Axiom unit tests 11419
68.2.2 Axiom help page 11419
68.2.3 fortran code 11424
68.2.4 lisp code 11428
68.3 cposvxx 11434
68.3.1 Axiom unit tests 11434
68.3.2 Axiom help page [Demm05] 11434
68.3.3 fortran code 11442
68.3.4 lisp code 11446

69 LAPACK - Positive Definite Matrix, Linear Solve, Complex16 11453
69.1 zcposv 11453
69.1.1 Axiom unit tests 11453
69.1.2 Axiom help page 11453
69.1.3 fortran code 11457
69.1.4 lisp code 11462
69.2 zposv 11470
69.2.1 Axiom unit tests 11470
69.2.2 Axiom help page 11470
69.2.3 fortran code 11472
69.2.4 lisp code 11474
69.3 zposvx 11475
69.3.1 Axiom unit tests 11475
69.3.2 Axiom help page 11476
69.3.3 fortran code 11480
69.3.4 lisp code 11484
69.4 zposvxx 11491
69.4.1 Axiom unit tests 11491
69.4.2 Axiom help page [Demm05] 11491
69.4.3 fortran code 11499
69.4.4 lisp code 11503

70 LAPACK - Positive Definite Matrix, Comp. Routines, Double 11511
70.1 dla_porcond 11511
70.1.1 Axiom unit tests 11511
70.1.2 Axiom help page 11511
70.1.3 fortran code 11513

70.1.4 lisp code . . . 11517
70.2 dla_porfsx_extended . . . 11524
70.2.1 Axiom unit tests . . . 11524
70.2.2 Axiom help page [Demm05] . . . 11525
70.2.3 fortran code . . . 11531
70.2.4 lisp code . . . 11537
70.3 dla_porpvgrw . . . 11545
70.3.1 Axiom unit tests . . . 11545
70.3.2 Axiom help page . . . 11545
70.3.3 fortran code . . . 11547
70.3.4 lisp code . . . 11549
70.4 dpocon . . . 11552
70.4.1 Axiom unit tests . . . 11552
70.4.2 Axiom help page . . . 11553
70.4.3 fortran code . . . 11554
70.4.4 lisp code . . . 11557
70.5 dpoequ . . . 11561
70.5.1 Axiom unit tests . . . 11561
70.5.2 Axiom help page . . . 11561
70.5.3 fortran code . . . 11563
70.5.4 lisp code . . . 11565
70.6 dpoequb . . . 11567
70.6.1 Axiom unit tests . . . 11567
70.6.2 Axiom help page . . . 11567
70.6.3 fortran code . . . 11569
70.6.4 lisp code . . . 11571
70.7 dporfs . . . 11573
70.7.1 Axiom unit tests . . . 11573
70.7.2 Axiom help page . . . 11573
70.7.3 fortran code . . . 11576
70.7.4 lisp code . . . 11581
70.8 dporfsx . . . 11589
70.8.1 Axiom unit tests . . . 11589
70.8.2 Axiom help page citeDemm05 . . . 11590
70.8.3 fortran code . . . 11596
70.8.4 lisp code . . . 11602
70.9 dpotf2 . . . 11603
70.9.1 Axiom unit tests . . . 11603
70.9.2 Axiom help page . . . 11603
70.9.3 fortran code . . . 11605
70.9.4 lisp code . . . 11607
70.10 dpotrf . . . 11611
70.10.1 Axiom unit tests . . . 11611
70.10.2 Axiom help page . . . 11612
70.10.3 fortran code . . . 11613
70.10.4 lisp code . . . 11616

70.11 dpotrf2 . . . 11621
70.11.1 Axiom unit tests . . . 11621
70.11.2 Axiom help page . . . 11621
70.11.3 fortran code . . . 11623
70.11.4 lisp code . . . 11626
70.12 dpotri . . . 11630
70.12.1 Axiom unit tests . . . 11630
70.12.2 Axiom help page . . . 11630
70.12.3 fortran code . . . 11631
70.12.4 lisp code . . . 11633
70.13 dpotrs . . . 11634
70.13.1 Axiom unit tests . . . 11634
70.13.2 Axiom help page . . . 11635
70.13.3 fortran code . . . 11636
70.13.4 lisp code . . . 11638

71 LAPACK - Positive Definite Matrix, Comp. Routines, Real 11643
71.1 sla_porcond . . . 11643
71.1.1 Axiom unit tests . . . 11643
71.1.2 Axiom help page . . . 11643
71.1.3 fortran code . . . 11645
71.1.4 lisp code . . . 11649
71.2 sla_porfsx_extended . . . 11656
71.2.1 Axiom unit tests . . . 11656
71.2.2 Axiom help page [Demm05] . . . 11657
71.2.3 fortran code . . . 11663
71.2.4 lisp code . . . 11669
71.3 sla_porpvgrw . . . 11677
71.3.1 Axiom unit tests . . . 11677
71.3.2 Axiom help page . . . 11677
71.3.3 fortran code . . . 11678
71.3.4 lisp code . . . 11681
71.4 spocon . . . 11684
71.4.1 Axiom unit tests . . . 11684
71.4.2 Axiom help page . . . 11684
71.4.3 fortran code . . . 11686
71.4.4 lisp code . . . 11689
71.5 spoequ . . . 11693
71.5.1 Axiom unit tests . . . 11693
71.5.2 Axiom help page . . . 11693
71.5.3 fortran code . . . 11695
71.5.4 lisp code . . . 11697
71.6 spoequb . . . 11698
71.6.1 Axiom unit tests . . . 11698
71.6.2 Axiom help page . . . 11699
71.6.3 fortran code . . . 11700

71.6.4 lisp code . . . 11703
71.7 sporfs . . . 11705
71.7.1 Axiom unit tests . . . 11705
71.7.2 Axiom help page . . . 11705
71.7.3 fortran code . . . 11707
71.7.4 lisp code . . . 11713
71.8 sporfsx . . . 11721
71.8.1 Axiom unit tests . . . 11721
71.8.2 Axiom help page [Demm05] . . . 11722
71.8.3 fortran code . . . 11728
71.8.4 lisp code . . . 11734
71.9 spotf2 . . . 11734
71.9.1 Axiom unit tests . . . 11734
71.9.2 Axiom help page . . . 11735
71.9.3 fortran code . . . 11736
71.9.4 lisp code . . . 11739
71.10 spotrf . . . 11743
71.10.1 Axiom unit tests . . . 11743
71.10.2 Axiom help page . . . 11743
71.10.3 fortran code . . . 11744
71.10.4 lisp code . . . 11747
71.11 spotrf2 . . . 11753
71.11.1 Axiom unit tests . . . 11753
71.11.2 Axiom help page . . . 11753
71.11.3 fortran code . . . 11755
71.11.4 lisp code . . . 11757
71.12 spotri . . . 11761
71.12.1 Axiom unit tests . . . 11761
71.12.2 Axiom help page . . . 11761
71.12.3 fortran code . . . 11763
71.12.4 lisp code . . . 11764
71.13 spotrs . . . 11766
71.13.1 Axiom unit tests . . . 11766
71.13.2 Axiom help page . . . 11766
71.13.3 fortran code . . . 11768
71.13.4 lisp code . . . 11770

72 LAPACK - Positive Definite Matrix, Comp. Routines, Complex 11773
72.1 cla_porcond_c . . . 11773
72.1.1 Axiom unit tests . . . 11773
72.1.2 Axiom help page . . . 11773
72.1.3 fortran code . . . 11775
72.1.4 lisp code . . . 11779
72.2 cla_porcond_x . . . 11785
72.2.1 Axiom unit tests . . . 11785
72.2.2 Axiom help page . . . 11786

72.2.3 fortran code . . . 11787
72.2.4 lisp code . . . 11791
72.3 cla_porfsx_extended . . . 11796
72.3.1 Axiom unit tests . . . 11796
72.3.2 Axiom help page [Demm05] . . . 11797
72.3.3 fortran code . . . 11803
72.3.4 lisp code . . . 11809
72.4 cla_porpvgrw . . . 11817
72.4.1 Axiom unit tests . . . 11817
72.4.2 Axiom help page . . . 11817
72.4.3 fortran code . . . 11819
72.4.4 lisp code . . . 11821
72.5 cpocon . . . 11825
72.5.1 Axiom unit tests . . . 11825
72.5.2 Axiom help page . . . 11825
72.5.3 fortran code . . . 11827
72.5.4 lisp code . . . 11830
72.6 cpoequ . . . 11833
72.6.1 Axiom unit tests . . . 11833
72.6.2 Axiom help page . . . 11834
72.6.3 fortran code . . . 11835
72.6.4 lisp code . . . 11837
72.7 cpoequb . . . 11839
72.7.1 Axiom unit tests . . . 11839
72.7.2 Axiom help page . . . 11840
72.7.3 fortran code . . . 11841
72.7.4 lisp code . . . 11843
72.8 cporfs . . . 11845
72.8.1 Axiom unit tests . . . 11845
72.8.2 Axiom help page . . . 11846
72.8.3 fortran code . . . 11848
72.8.4 lisp code . . . 11854
72.9 cporfsx . . . 11862
72.9.1 Axiom unit tests . . . 11862
72.9.2 Axiom help page [Demm05] . . . 11863
72.9.3 fortran code . . . 11869
72.9.4 lisp code . . . 11875
72.10 cpotf2 . . . 11875
72.10.1 Axiom unit tests . . . 11875
72.10.2 Axiom help page . . . 11876
72.10.3 fortran code . . . 11877
72.10.4 lisp code . . . 11880
72.11 cpotrf . . . 11885
72.11.1 Axiom unit tests . . . 11885
72.11.2 Axiom help page . . . 11885
72.11.3 fortran code . . . 11886

72.11.4 lisp code . . . 11889
72.12 cpotrf2 . . . 11895
72.12.1 Axiom unit tests . . . 11895
72.12.2 Axiom help page . . . 11895
72.12.3 fortran code . . . 11897
72.12.4 lisp code . . . 11900
72.13 cpotri . . . 11903
72.13.1 Axiom unit tests . . . 11903
72.13.2 Axiom help page . . . 11904
72.13.3 fortran code . . . 11905
72.13.4 lisp code . . . 11906
72.14 cpotrs . . . 11908
72.14.1 Axiom unit tests . . . 11908
72.14.2 Axiom help page . . . 11908
72.14.3 fortran code . . . 11910
72.14.4 lisp code . . . 11912

73 LAPACK - Positive Definite Matrix, Comp. Routines, Complex16 11915
73.1 zla_porcond_c . . . 11915
73.1.1 Axiom unit tests . . . 11915
73.1.2 Axiom help page . . . 11915
73.1.3 fortran code . . . 11917
73.1.4 lisp code . . . 11921
73.2 zla_porcond_x . . . 11928
73.2.1 Axiom unit tests . . . 11928
73.2.2 Axiom help page . . . 11928
73.2.3 fortran code . . . 11930
73.2.4 lisp code . . . 11933
73.3 zla_porfsx_extended . . . 11939
73.3.1 Axiom unit tests . . . 11939
73.3.2 Axiom help page [Demm05] . . . 11939
73.3.3 fortran code . . . 11945
73.3.4 lisp code . . . 11951
73.4 zla_porpvgrw . . . 11959
73.4.1 Axiom unit tests . . . 11959
73.4.2 Axiom help page . . . 11960
73.4.3 fortran code . . . 11961
73.4.4 lisp code . . . 11964
73.5 zpocon . . . 11967
73.5.1 Axiom unit tests . . . 11967
73.5.2 Axiom help page . . . 11968
73.5.3 fortran code . . . 11969
73.5.4 lisp code . . . 11972
73.6 zpoequ . . . 11976
73.6.1 Axiom unit tests . . . 11976
73.6.2 Axiom help page . . . 11976

73.6.3 fortran code . . . 11978
73.6.4 lisp code . . . 11980
73.7 zpoequb . . . 11982
73.7.1 Axiom unit tests . . . 11982
73.7.2 Axiom help page . . . 11982
73.7.3 fortran code . . . 11984
73.7.4 lisp code . . . 11986
73.8 zporfs . . . 11988
73.8.1 Axiom unit tests . . . 11988
73.8.2 Axiom help page . . . 11988
73.8.3 fortran code . . . 11991
73.8.4 lisp code . . . 11996
73.9 zporfsx . . . 12005
73.9.1 Axiom unit tests . . . 12005
73.9.2 Axiom help page [Demm05] . . . 12005
73.9.3 fortran code . . . 12012
73.9.4 lisp code . . . 12018
73.10 zpotf2 . . . 12018
73.10.1 Axiom unit tests . . . 12018
73.10.2 Axiom help page . . . 12018
73.10.3 fortran code . . . 12020
73.10.4 lisp code . . . 12023
73.11 zpotrf . . . 12027
73.11.1 Axiom unit tests . . . 12027
73.11.2 Axiom help page . . . 12028
73.11.3 fortran code . . . 12029
73.11.4 lisp code . . . 12032
73.12 zpotrf2 . . . 12037
73.12.1 Axiom unit tests . . . 12037
73.12.2 Axiom help page . . . 12038
73.12.3 fortran code . . . 12039
73.12.4 lisp code . . . 12042
73.13 zpotri . . . 12046
73.13.1 Axiom unit tests . . . 12046
73.13.2 Axiom help page . . . 12047
73.13.3 fortran code . . . 12048
73.13.4 lisp code . . . 12049
73.14 zpotrs . . . 12051
73.14.1 Axiom unit tests . . . 12051
73.14.2 Axiom help page . . . 12051
73.14.3 fortran code . . . 12053
73.14.4 lisp code . . . 12055

74 LAPACK - General Tridiagonal Matrix, Linear Solve, Double 12059
74.1 dgtsv . . . 12059
74.1.1 Axiom unit tests . . . 12059
74.1.2 Axiom help page . . . 12059
74.1.3 fortran code . . . 12061
74.1.4 lisp code . . . 12066
74.2 dgtsvx . . . 12074
74.2.1 Axiom unit tests . . . 12074
74.2.2 Axiom help page . . . 12074
74.2.3 fortran code . . . 12079
74.2.4 lisp code . . . 12081

75 LAPACK - General Tridiagonal Matrix, Linear Solve, Real 12087
75.1 sgtsv . . . 12087
75.1.1 Axiom unit tests . . . 12087
75.1.2 Axiom help page . . . 12087
75.1.3 fortran code . . . 12089
75.1.4 lisp code . . . 12094
75.2 sgtsvx . . . 12102
75.2.1 Axiom unit tests . . . 12102
75.2.2 Axiom help page . . . 12102
75.2.3 fortran code . . . 12107
75.2.4 lisp code . . . 12109

76 LAPACK - General Tridiagonal Matrix, Linear Solve, Complex 12115
76.1 cgtsv . . . 12115
76.1.1 Axiom unit tests . . . 12115
76.1.2 Axiom help page . . . 12115
76.1.3 fortran code . . . 12117
76.1.4 lisp code . . . 12120
76.2 cgtsvx . . . 12124
76.2.1 Axiom unit tests . . . 12124
76.2.2 Axiom help page . . . 12124
76.2.3 fortran code . . . 12128
76.2.4 lisp code . . . 12131

77 LAPACK - General Tridiagonal Matrix, Linear Solve, Complex16 12137
77.1 zgtsv . . . 12137
77.1.1 Axiom unit tests . . . 12137
77.1.2 Axiom help page . . . 12137
77.1.3 fortran code . . . 12139
77.1.4 lisp code . . . 12142
77.2 zgtsvx . . . 12146
77.2.1 Axiom unit tests . . . 12146
77.2.2 Axiom help page . . . 12146
77.2.3 fortran code . . . 12150

77.2.4 lisp code . . . 12153

78 LAPACK - General Tridiagonal Matrix, Comp. Routines, Double 12159
78.1 dgtcon . . . 12159
78.1.1 Axiom unit tests . . . 12159
78.1.2 Axiom help page . . . 12159
78.1.3 fortran code . . . 12161
78.1.4 lisp code . . . 12164
78.2 dgtrfs . . . 12167
78.2.1 Axiom unit tests . . . 12167
78.2.2 Axiom help page . . . 12167
78.2.3 fortran code . . . 12170
78.2.4 lisp code . . . 12175
78.3 dgttrf . . . 12185
78.3.1 Axiom unit tests . . . 12185
78.3.2 Axiom help page . . . 12186
78.3.3 fortran code . . . 12187
78.3.4 lisp code . . . 12190
78.4 dgttrs . . . 12193
78.4.1 Axiom unit tests . . . 12193
78.4.2 Axiom help page . . . 12193
78.4.3 fortran code . . . 12195
78.4.4 lisp code . . . 12197
78.5 dgtts2 . . . 12200
78.5.1 Axiom unit tests . . . 12200
78.5.2 Axiom help page . . . 12200
78.5.3 fortran code . . . 12202
78.5.4 lisp code . . . 12205

79 LAPACK - General Tridiagonal Matrix, Comp. Routines, Real 12213
79.1 sgtcon . . . 12213
79.1.1 Axiom unit tests . . . 12213
79.1.2 Axiom help page . . . 12213
79.1.3 fortran code . . . 12215
79.1.4 lisp code . . . 12218
79.2 sgtrfs . . . 12221
79.2.1 Axiom unit tests . . . 12221
79.2.2 Axiom help page . . . 12221
79.2.3 fortran code . . . 12224
79.2.4 lisp code . . . 12229
79.3 sgttrf . . . 12239
79.3.1 Axiom unit tests . . . 12239
79.3.2 Axiom help page . . . 12240
79.3.3 fortran code . . . 12241
79.3.4 lisp code . . . 12244
79.4 sgttrs . . . 12247

79.4.1 Axiom unit tests 12247
79.4.2 Axiom help page 12247
79.4.3 fortran code 12249
79.4.4 lisp code 12251
79.5 sgtts2 12254
79.5.1 Axiom unit tests 12254
79.5.2 Axiom help page 12254
79.5.3 fortran code 12256
79.5.4 lisp code 12259

80 LAPACK - General Tridiagonal Matrix, Comp. Routines, Complex 12267
80.1 cgtcon 12267
80.1.1 Axiom unit tests 12267
80.1.2 Axiom help page 12267
80.1.3 fortran code 12269
80.1.4 lisp code 12272
80.2 cgtrfs 12275
80.2.1 Axiom unit tests 12275
80.2.2 Axiom help page 12275
80.2.3 fortran code 12278
80.2.4 lisp code 12284
80.3 cgttrf 12294
80.3.1 Axiom unit tests 12294
80.3.2 Axiom help page 12294
80.3.3 fortran code 12296
80.3.4 lisp code 12299
80.4 cgttrs 12302
80.4.1 Axiom unit tests 12302
80.4.2 Axiom help page 12302
80.4.3 fortran code 12304
80.4.4 lisp code 12306
80.5 cgtts2 12309
80.5.1 Axiom unit tests 12309
80.5.2 Axiom help page 12309
80.5.3 fortran code 12311
80.5.4 lisp code 12316

81 LAPACK - General Tridiagonal Matrix, Comp. Routines, Complex16 12327
81.1 zgtcon 12327
81.1.1 Axiom unit tests 12327
81.1.2 Axiom help page 12327
81.1.3 fortran code 12329
81.1.4 lisp code 12332
81.2 zgtrfs 12335
81.2.1 Axiom unit tests 12335
81.2.2 Axiom help page 12335

81.2.3 fortran code . . . 12338
81.2.4 lisp code . . . 12344
81.3 zgttrf . . . 12354
81.3.1 Axiom unit tests . . . 12354
81.3.2 Axiom help page . . . 12354
81.3.3 fortran code . . . 12356
81.3.4 lisp code . . . 12359
81.4 zgttrs . . . 12362
81.4.1 Axiom unit tests . . . 12362
81.4.2 Axiom help page . . . 12363
81.4.3 fortran code . . . 12364
81.4.4 lisp code . . . 12366
81.5 zgtts2 . . . 12369
81.5.1 Axiom unit tests . . . 12369
81.5.2 Axiom help page . . . 12369
81.5.3 fortran code . . . 12371
81.5.4 lisp code . . . 12376

82 LAPACK - Positive Definite Tridiagonal Matrix, Linear Solve, Double 12387
82.1 dptsv . . . 12387
82.1.1 Axiom unit tests . . . 12387
82.1.2 Axiom help page . . . 12387
82.1.3 fortran code . . . 12389
82.1.4 lisp code . . . 12390
82.2 dptsvx . . . 12392
82.2.1 Axiom unit tests . . . 12392
82.2.2 Axiom help page . . . 12392
82.2.3 fortran code . . . 12395
82.2.4 lisp code . . . 12398

83 LAPACK - Positive Definite Tridiagonal Matrix, Linear Solve, Real 12401
83.1 sptsv . . . 12401
83.1.1 Axiom unit tests . . . 12401
83.1.2 Axiom help page . . . 12401
83.1.3 fortran code . . . 12403
83.1.4 lisp code . . . 12404
83.2 sptsvx . . . 12406
83.2.1 Axiom unit tests . . . 12406
83.2.2 Axiom help page . . . 12406
83.2.3 fortran code . . . 12409
83.2.4 lisp code . . . 12412

84 LAPACK - Positive Definite Tridiagonal Matrix, Linear Solve, Complex 12415
84.1 cptsv . . . 12415
84.1.1 Axiom unit tests . . . 12415
84.1.2 Axiom help page . . . 12415
84.1.3 fortran code . . . 12417
84.1.4 lisp code . . . 12418
84.2 cptsvx . . . 12420
84.2.1 Axiom unit tests . . . 12420
84.2.2 Axiom help page . . . 12420
84.2.3 fortran code . . . 12423
84.2.4 lisp code . . . 12426

85 LAPACK - Positive Definite Tridiagonal Matrix, Linear Solve, Complex 12431
85.1 zptsv . . . 12431
85.1.1 Axiom unit tests . . . 12431
85.1.2 Axiom help page . . . 12431
85.1.3 fortran code . . . 12433
85.1.4 lisp code . . . 12434
85.2 zptsvx . . . 12436
85.2.1 Axiom unit tests . . . 12436
85.2.2 Axiom help page . . . 12436
85.2.3 fortran code . . . 12440
85.2.4 lisp code . . . 12442

86 LAPACK - Positive Definite Tridiagonal Matrix, Comp. Routines, Doub 12447
86.1 dptcon . . . 12447
86.1.1 Axiom unit tests . . . 12447
86.1.2 Axiom help page . . . 12447
86.1.3 fortran code . . . 12449
86.1.4 lisp code . . . 12451
86.2 dpteqr . . . 12453
86.2.1 Axiom unit tests . . . 12453
86.2.2 Axiom help page . . . 12454
86.2.3 fortran code . . . 12456
86.2.4 lisp code . . . 12458
86.3 dptrfs . . . 12461
86.3.1 Axiom unit tests . . . 12461
86.3.2 Axiom help page . . . 12461
86.3.3 fortran code . . . 12464
86.3.4 lisp code . . . 12468
86.4 dpttrf . . . 12475
86.4.1 Axiom unit tests . . . 12475
86.4.2 Axiom help page . . . 12475
86.4.3 fortran code . . . 12477
86.4.4 lisp code . . . 12479
86.5 dpttrs . . . 12482

86.5.1 Axiom unit tests . . . 12482
86.5.2 Axiom help page . . . 12482
86.5.3 fortran code . . . 12484
86.5.4 lisp code . . . 12486
86.6 dptts2 . . . 12487
86.6.1 Axiom unit tests . . . 12487
86.6.2 Axiom help page . . . 12488
86.6.3 fortran code . . . 12489
86.6.4 lisp code . . . 12491

87 LAPACK - Positive Definite Tridiagonal Matrix, Comp. Routines, Real 12493
87.1 sptcon . . . 12493
87.1.1 Axiom unit tests . . . 12493
87.1.2 Axiom help page . . . 12493
87.1.3 fortran code . . . 12495
87.1.4 lisp code . . . 12497
87.2 spteqr . . . 12499
87.2.1 Axiom unit tests . . . 12499
87.2.2 Axiom help page . . . 12500
87.2.3 fortran code . . . 12502
87.2.4 lisp code . . . 12504
87.3 sptrfs . . . 12507
87.3.1 Axiom unit tests . . . 12507
87.3.2 Axiom help page . . . 12507
87.3.3 fortran code . . . 12510
87.3.4 lisp code . . . 12514
87.4 spttrf . . . 12521
87.4.1 Axiom unit tests . . . 12521
87.4.2 Axiom help page . . . 12521
87.4.3 fortran code . . . 12523
87.4.4 lisp code . . . 12525
87.5 spttrs . . . 12528
87.5.1 Axiom unit tests . . . 12528
87.5.2 Axiom help page . . . 12528
87.5.3 fortran code . . . 12530
87.5.4 lisp code . . . 12532
87.6 sptts2 . . . 12533
87.6.1 Axiom unit tests . . . 12533
87.6.2 Axiom help page . . . 12534
87.6.3 fortran code . . . 12535
87.6.4 lisp code . . . 12537

88 LAPACK - Positive Definite Tridiagonal Matrix, Comp. Routines, Complex 12539
88.1 cptcon . . . 12539
88.1.1 Axiom unit tests . . . 12539
88.1.2 Axiom help page . . . 12539
88.1.3 fortran code . . . 12541
88.1.4 lisp code . . . 12543
88.2 cpteqr . . . 12545
88.2.1 Axiom unit tests . . . 12545
88.2.2 Axiom help page . . . 12546
88.2.3 fortran code . . . 12548
88.2.4 lisp code . . . 12550
88.3 cptrfs . . . 12553
88.3.1 Axiom unit tests . . . 12553
88.3.2 Axiom help page . . . 12553
88.3.3 fortran code . . . 12556
88.3.4 lisp code . . . 12562
88.4 cpttrf . . . 12572
88.4.1 Axiom unit tests . . . 12572
88.4.2 Axiom help page . . . 12573
88.4.3 fortran code . . . 12574
88.4.4 lisp code . . . 12577
88.5 cpttrs . . . 12580
88.5.1 Axiom unit tests . . . 12580
88.5.2 Axiom help page . . . 12581
88.5.3 fortran code . . . 12582
88.5.4 lisp code . . . 12584
88.6 cptts2 . . . 12587
88.6.1 Axiom unit tests . . . 12587
88.6.2 Axiom help page . . . 12587
88.6.3 fortran code . . . 12589
88.6.4 lisp code . . . 12591

89 LAPACK - Positive Definite Tridiagonal Matrix, Comp. Routines, Complex16 12597
89.1 zptcon . . . 12597
89.1.1 Axiom unit tests . . . 12597
89.1.2 Axiom help page . . . 12597
89.1.3 fortran code . . . 12599
89.1.4 lisp code . . . 12601
89.2 zpteqr . . . 12603
89.2.1 Axiom unit tests . . . 12603
89.2.2 Axiom help page . . . 12604
89.2.3 fortran code . . . 12606
89.2.4 lisp code . . . 12608
89.3 zptrfs . . . 12611

89.3.1 Axiom unit tests . . . 12611
89.3.2 Axiom help page . . . 12612
89.3.3 fortran code . . . 12614
89.3.4 lisp code . . . 12620
89.4 zpttrf . . . 12630
89.4.1 Axiom unit tests . . . 12630
89.4.2 Axiom help page . . . 12631
89.4.3 fortran code . . . 12632
89.4.4 lisp code . . . 12635
89.5 zpttrs . . . 12639
89.5.1 Axiom unit tests . . . 12639
89.5.2 Axiom help page . . . 12639
89.5.3 fortran code . . . 12641
89.5.4 lisp code . . . 12643
89.6 zptts2 . . . 12645
89.6.1 Axiom unit tests . . . 12645
89.6.2 Axiom help page . . . 12645
89.6.3 fortran code . . . 12647
89.6.4 lisp code . . . 12650

90 LAPACK - Other Auxilliary Routines 12655
90.1 disnan . . . 12655
90.1.1 Axiom unit tests . . . 12655
90.1.2 Axiom help page . . . 12655
90.1.3 fortran code . . . 12656
90.1.4 lisp code . . . 12657
90.2 dlabad . . . 12657
90.2.1 Axiom unit tests . . . 12657
90.2.2 Axiom help page . . . 12657
90.2.3 fortran code . . . 12658
90.2.4 lisp code . . . 12659
90.3 dlacpy . . . 12659
90.3.1 Axiom unit tests . . . 12659
90.3.2 Axiom help page . . . 12660
90.3.3 fortran code . . . 12661
90.3.4 lisp code . . . 12662
90.4 dladiv . . . 12663
90.4.1 Axiom unit tests . . . 12663
90.4.2 Axiom help page . . . 12663
90.4.3 fortran code . . . 12664
90.4.4 lisp code . . . 12668
90.5 dlae2 . . . 12668
90.5.1 Axiom unit tests . . . 12668
90.5.2 Axiom help page . . . 12668
90.5.3 fortran code . . . 12670
90.5.4 lisp code . . . 12672

90.6 dlaebz . . . 12673
90.6.1 Axiom unit tests . . . 12673
90.6.2 Axiom help page . . . 12673
90.6.3 fortran code . . . 12678
90.6.4 lisp code . . . 12685
90.7 dlaev2 . . . 12695
90.7.1 Axiom unit tests . . . 12695
90.7.2 Axiom help page . . . 12696
90.7.3 fortran code . . . 12697
90.7.4 lisp code . . . 12700
90.8 dlagts . . . 12701
90.8.1 Axiom unit tests . . . 12701
90.8.2 Axiom help page . . . 12702
90.8.3 fortran code . . . 12704
90.8.4 lisp code . . . 12709
90.9 dlaisnan . . . 12716
90.9.1 Axiom unit tests . . . 12716
90.9.2 Axiom help page . . . 12716
90.9.3 fortran code . . . 12717
90.9.4 lisp code . . . 12718
90.10 dlamc1 . . . 12718
90.10.1 Axiom unit tests . . . 12718
90.10.2 Axiom help page . . . 12718
90.10.3 fortran code . . . 12719
90.10.4 lisp code . . . 12719
90.11 dlamc2 . . . 12719
90.11.1 Axiom unit tests . . . 12719
90.11.2 Axiom help page . . . 12720
90.11.3 fortran code . . . 12721
90.11.4 lisp code . . . 12721
90.12 dlamc3 . . . 12721
90.12.1 Axiom unit tests . . . 12721
90.12.2 Axiom help page . . . 12722
90.12.3 fortran code . . . 12722
90.12.4 lisp code . . . 12723
90.13 dlamch . . . 12723
90.13.1 Axiom unit tests . . . 12723
90.13.2 Axiom help page . . . 12723
90.13.3 fortran code . . . 12724
90.13.4 lisp code . . . 12725
90.14 dlaneg . . . 12725
90.14.1 Axiom unit tests . . . 12725
90.14.2 Axiom help page . . . 12725
90.14.3 fortran code . . . 12727
90.14.4 lisp code . . . 12729
90.15 dlanst . . . 12733

90.15.1 Axiom unit tests . . . 12733
90.15.2 Axiom help page . . . 12733
90.15.3 fortran code . . . 12735
90.15.4 lisp code . . . 12737
90.16 dlapy2 . . . 12737
90.16.1 Axiom unit tests . . . 12737
90.16.2 Axiom help page . . . 12737
90.16.3 fortran code . . . 12738
90.16.4 lisp code . . . 12739
90.17 dlapy3 . . . 12739
90.17.1 Axiom unit tests . . . 12739
90.17.2 Axiom help page . . . 12740
90.17.3 fortran code . . . 12741
90.17.4 lisp code . . . 12742
90.18 dlarnv . . . 12742
90.18.1 Axiom unit tests . . . 12742
90.18.2 Axiom help page . . . 12742
90.18.3 fortran code . . . 12744
90.18.4 lisp code . . . 12745
90.19 dlarra . . . 12747
90.19.1 Axiom unit tests . . . 12747
90.19.2 Axiom help page . . . 12748
90.19.3 fortran code . . . 12749
90.19.4 lisp code . . . 12751
90.20 dlarrb . . . 12753
90.20.1 Axiom unit tests . . . 12753
90.20.2 Axiom help page . . . 12753
90.20.3 fortran code . . . 12756
90.20.4 lisp code . . . 12760
90.21 dlarrc . . . 12765
90.21.1 Axiom unit tests . . . 12765
90.21.2 Axiom help page . . . 12765
90.21.3 fortran code . . . 12767
90.21.4 lisp code . . . 12769
90.22 dlarrd . . . 12772
90.22.1 Axiom unit tests . . . 12772
90.22.2 Axiom help page . . . 12772
90.22.3 fortran code . . . 12777
90.22.4 lisp code . . . 12788
90.23 dlarre . . . 12802
90.23.1 Axiom unit tests . . . 12802
90.23.2 Axiom help page . . . 12803
90.23.3 fortran code . . . 12807
90.23.4 lisp code . . . 12819
90.24 dlarrf . . . 12837
90.24.1 Axiom unit tests . . . 12837

90.24.2 Axiom help page . . . 12837
90.24.3 fortran code . . . 12840
90.24.4 lisp code . . . 12846
90.25 dlarrj . . . 12854
90.25.1 Axiom unit tests . . . 12854
90.25.2 Axiom help page . . . 12854
90.25.3 fortran code . . . 12856
90.25.4 lisp code . . . 12860
90.26 dlarrk . . . 12866
90.26.1 Axiom unit tests . . . 12866
90.26.2 Axiom help page . . . 12866
90.26.3 fortran code . . . 12868
90.26.4 lisp code . . . 12871
90.27 dlarrr . . . 12873
90.27.1 Axiom unit tests . . . 12873
90.27.2 Axiom help page . . . 12873
90.27.3 fortran code . . . 12874
90.27.4 lisp code . . . 12877
90.28 dlartg . . . 12879
90.28.1 Axiom unit tests . . . 12879
90.28.2 Axiom help page . . . 12879
90.28.3 fortran code . . . 12880
90.28.4 lisp code . . . 12883
90.29 dlartgp . . . 12883
90.29.1 Axiom unit tests . . . 12883
90.29.2 Axiom help page . . . 12883
90.29.3 fortran code . . . 12885
90.29.4 lisp code . . . 12887
90.30 dlaruv . . . 12889
90.30.1 Axiom unit tests . . . 12889
90.30.2 Axiom help page . . . 12889
90.30.3 fortran code . . . 12891
90.30.4 lisp code . . . 12898
90.31 dlas2 . . . 12906
90.31.1 Axiom unit tests . . . 12906
90.31.2 Axiom help page . . . 12906
90.31.3 fortran code . . . 12908
90.31.4 lisp code . . . 12910
90.32 dlascl . . . 12910
90.32.1 Axiom unit tests . . . 12910
90.32.2 Axiom help page . . . 12910
90.32.3 fortran code . . . 12912
90.32.4 lisp code . . . 12917
90.33 dlasd0 . . . 12918
90.33.1 Axiom unit tests . . . 12918
90.33.2 Axiom help page . . . 12918

90.33.3 fortran code . . . 12920
90.33.4 lisp code . . . 12924
90.34 dlasd1 . . . 12924
90.34.1 Axiom unit tests . . . 12924
90.34.2 Axiom help page . . . 12924
90.34.3 fortran code . . . 12927
90.34.4 lisp code . . . 12930
90.35 dlasd2 . . . 12930
90.35.1 Axiom unit tests . . . 12930
90.35.2 Axiom help page . . . 12931
90.35.3 fortran code . . . 12935
90.35.4 lisp code . . . 12942
90.36 dlasd3 . . . 12942
90.36.1 Axiom unit tests . . . 12942
90.36.2 Axiom help page . . . 12943
90.36.3 fortran code . . . 12946
90.36.4 lisp code . . . 12951
90.37 dlasd4 . . . 12951
90.37.1 Axiom unit tests . . . 12951
90.37.2 Axiom help page . . . 12952
90.37.3 fortran code . . . 12954
90.37.4 lisp code . . . 12972
90.38 dlasd5 . . . 12972
90.38.1 Axiom unit tests . . . 12972
90.38.2 Axiom help page . . . 12973
90.38.3 fortran code . . . 12974
90.38.4 lisp code . . . 12977
90.39 dlasd6 . . . 12977
90.39.1 Axiom unit tests . . . 12977
90.39.2 Axiom help page . . . 12978
90.39.3 fortran code . . . 12982
90.39.4 lisp code . . . 12985
90.40 dlasd7 . . . 12985
90.40.1 Axiom unit tests . . . 12985
90.40.2 Axiom help page . . . 12986
90.40.3 fortran code . . . 12990
90.40.4 lisp code . . . 12996
90.41 dlasd8 . . . 12996
90.41.1 Axiom unit tests . . . 12996
90.41.2 Axiom help page . . . 12997
90.41.3 fortran code . . . 12999
90.41.4 lisp code . . . 13003
90.42 dlasda . . . 13003
90.42.1 Axiom unit tests . . . 13003
90.42.2 Axiom help page . . . 13003
90.42.3 fortran code . . . 13007

90.42.4 lisp code . . . 13012
90.43dlasdq . . . 13013
90.43.1 Axiom unit tests . . . 13013
90.43.2 Axiom help page . . . 13013
90.43.3 fortran code . . . 13016
90.43.4 lisp code . . . 13020
90.44dlasdt . . . 13020
90.44.1 Axiom unit tests . . . 13020
90.44.2 Axiom help page . . . 13021
90.44.3 fortran code . . . 13022
90.44.4 lisp code . . . 13024
90.45dlaset . . . 13024
90.45.1 Axiom unit tests . . . 13024
90.45.2 Axiom help page . . . 13024
90.45.3 fortran code . . . 13026
90.45.4 lisp code . . . 13027
90.46dlasr . . . 13028
90.46.1 Axiom unit tests . . . 13028
90.46.2 Axiom help page . . . 13028
90.46.3 fortran code . . . 13031
90.46.4 lisp code . . . 13036
90.47dlassq . . . 13036
90.47.1 Axiom unit tests . . . 13036
90.47.2 Axiom help page . . . 13037
90.47.3 fortran code . . . 13038
90.47.4 lisp code . . . 13039
90.48dlasv2 . . . 13040
90.48.1 Axiom unit tests . . . 13040
90.48.2 Axiom help page . . . 13040
90.48.3 fortran code . . . 13042
90.48.4 lisp code . . . 13046
90.49dsecnd . . . 13046
90.49.1 Axiom unit tests . . . 13046
90.49.2 Axiom help page . . . 13047
90.49.3 fortran code . . . 13047
90.49.4 lisp code . . . 13047
90.50ieeeck . . . 13048
90.50.1 Axiom unit tests . . . 13048
90.50.2 Axiom help page . . . 13048
90.50.3 fortran code . . . 13049
90.50.4 lisp code . . . 13052
90.51iladlc . . . 13052
90.51.1 Axiom unit tests . . . 13052
90.51.2 Axiom help page . . . 13052
90.51.3 fortran code . . . 13053
90.51.4 lisp code . . . 13054

90.52iladlr . . . 13055
90.52.1 Axiom unit tests . . . 13055
90.52.2 Axiom help page . . . 13056
90.52.3 fortran code . . . 13057
90.52.4 lisp code . . . 13058
90.53ilaenv . . . 13059
90.53.1 Axiom unit tests . . . 13059
90.53.2 Axiom help page . . . 13060
90.53.3 fortran code . . . 13061
90.53.4 lisp code . . . 13071
90.54ilaver . . . 13072
90.54.1 Axiom unit tests . . . 13072
90.54.2 Axiom help page . . . 13072
90.54.3 fortran code . . . 13073
90.54.4 lisp code . . . 13073
90.55iparmq . . . 13074
90.55.1 Axiom unit tests . . . 13074
90.55.2 Axiom help page . . . 13074
90.55.3 fortran code . . . 13078
90.55.4 lisp code . . . 13082
90.56lsame . . . 13085
90.56.1 Axiom unit tests . . . 13085
90.56.2 Axiom help page . . . 13086
90.56.3 fortran code . . . 13086
90.56.4 lisp code . . . 13087
90.57lsamen . . . 13088
90.57.1 Axiom unit tests . . . 13088
90.57.2 Axiom help page . . . 13089
90.57.3 fortran code . . . 13090
90.57.4 lisp code . . . 13091
90.58second . . . 13092
90.58.1 Axiom unit tests . . . 13092
90.58.2 Axiom help page . . . 13092
90.58.3 fortran code . . . 13093
90.58.4 lisp code . . . 13093
90.59sisnan . . . 13093
90.59.1 Axiom unit tests . . . 13093
90.59.2 Axiom help page . . . 13093
90.59.3 fortran code . . . 13094
90.59.4 lisp code . . . 13095
90.60slabad . . . 13095
90.60.1 Axiom unit tests . . . 13095
90.60.2 Axiom help page . . . 13096
90.60.3 fortran code . . . 13097
90.60.4 lisp code . . . 13098
90.61slacpy . . . 13098

90.61.1 Axiom unit tests . . . 13098
90.61.2 Axiom help page . . . 13099
90.61.3 fortran code . . . 13100
90.61.4 lisp code . . . 13101
90.62sladiv . . . 13103
90.62.1 Axiom unit tests . . . 13103
90.62.2 Axiom help page . . . 13103
90.62.3 fortran code . . . 13105
90.62.4 lisp code . . . 13108
90.63slae2 . . . 13112
90.63.1 Axiom unit tests . . . 13112
90.63.2 Axiom help page . . . 13112
90.63.3 fortran code . . . 13114
90.63.4 lisp code . . . 13115
90.64slaebz . . . 13117
90.64.1 Axiom unit tests . . . 13117
90.64.2 Axiom help page . . . 13117
90.64.3 fortran code . . . 13122
90.64.4 lisp code . . . 13129
90.65slaev2 . . . 13139
90.65.1 Axiom unit tests . . . 13139
90.65.2 Axiom help page . . . 13139
90.65.3 fortran code . . . 13141
90.65.4 lisp code . . . 13144
90.66slag2d . . . 13145
90.66.1 Axiom unit tests . . . 13145
90.66.2 Axiom help page . . . 13145
90.66.3 fortran code . . . 13147
90.66.4 lisp code . . . 13148
90.67slagts . . . 13149
90.67.1 Axiom unit tests . . . 13149
90.67.2 Axiom help page . . . 13149
90.67.3 fortran code . . . 13152
90.67.4 lisp code . . . 13156
90.68slaisnan . . . 13163
90.68.1 Axiom unit tests . . . 13163
90.68.2 Axiom help page . . . 13163
90.68.3 fortran code . . . 13164
90.68.4 lisp code . . . 13165
90.69slamc1 . . . 13166
90.69.1 Axiom unit tests . . . 13166
90.69.2 Axiom help page . . . 13166
90.69.3 fortran code . . . 13167
90.69.4 lisp code . . . 13167
90.70slamc2 . . . 13167
90.70.1 Axiom unit tests . . . 13167

90.70.2 Axiom help page . . . 13168
90.70.3 fortran code . . . 13169
90.70.4 lisp code . . . 13169
90.71 slamc3 . . . 13169
90.71.1 Axiom unit tests . . . 13169
90.71.2 Axiom help page . . . 13169
90.71.3 fortran code . . . 13170
90.71.4 lisp code . . . 13170
90.72 slamch . . . 13171
90.72.1 Axiom unit tests . . . 13171
90.72.2 Axiom help page . . . 13171
90.72.3 fortran code . . . 13172
90.72.4 lisp code . . . 13172
90.73 slaneg . . . 13172
90.73.1 Axiom unit tests . . . 13172
90.73.2 Axiom help page . . . 13173
90.73.3 fortran code . . . 13174
90.73.4 lisp code . . . 13177
90.74 slanst . . . 13180
90.74.1 Axiom unit tests . . . 13180
90.74.2 Axiom help page . . . 13180
90.74.3 fortran code . . . 13182
90.74.4 lisp code . . . 13184
90.75 slapy2 . . . 13187
90.75.1 Axiom unit tests . . . 13187
90.75.2 Axiom help page . . . 13187
90.75.3 fortran code . . . 13188
90.75.4 lisp code . . . 13189
90.76 slapy3 . . . 13190
90.76.1 Axiom unit tests . . . 13190
90.76.2 Axiom help page . . . 13190
90.76.3 fortran code . . . 13191
90.76.4 lisp code . . . 13192
90.77 slarnv . . . 13193
90.77.1 Axiom unit tests . . . 13193
90.77.2 Axiom help page . . . 13193
90.77.3 fortran code . . . 13195
90.77.4 lisp code . . . 13196
90.78 slarra . . . 13198
90.78.1 Axiom unit tests . . . 13198
90.78.2 Axiom help page . . . 13199
90.78.3 fortran code . . . 13200
90.78.4 lisp code . . . 13202
90.79 slarrb . . . 13204
90.79.1 Axiom unit tests . . . 13204
90.79.2 Axiom help page . . . 13204

90.79.3 fortran code . . . 13207
90.79.4 lisp code . . . 13211
90.80 slarrc . . . 13216
90.80.1 Axiom unit tests . . . 13216
90.80.2 Axiom help page . . . 13216
90.80.3 fortran code . . . 13218
90.80.4 lisp code . . . 13220
90.81 slarrd . . . 13223
90.81.1 Axiom unit tests . . . 13223
90.81.2 Axiom help page . . . 13223
90.81.3 fortran code . . . 13228
90.81.4 lisp code . . . 13239
90.82 slarre . . . 13253
90.82.1 Axiom unit tests . . . 13253
90.82.2 Axiom help page . . . 13254
90.82.3 fortran code . . . 13258
90.82.4 lisp code . . . 13270
90.83 slarrf . . . 13287
90.83.1 Axiom unit tests . . . 13287
90.83.2 Axiom help page . . . 13288
90.83.3 fortran code . . . 13290
90.83.4 lisp code . . . 13296
90.84 slarrj . . . 13304
90.84.1 Axiom unit tests . . . 13304
90.84.2 Axiom help page . . . 13305
90.84.3 fortran code . . . 13307
90.84.4 lisp code . . . 13311
90.85 slarrk . . . 13317
90.85.1 Axiom unit tests . . . 13317
90.85.2 Axiom help page . . . 13317
90.85.3 fortran code . . . 13319
90.85.4 lisp code . . . 13321
90.86 slarrr . . . 13323
90.86.1 Axiom unit tests . . . 13323
90.86.2 Axiom help page . . . 13324
90.86.3 fortran code . . . 13325
90.86.4 lisp code . . . 13327
90.87 slartg . . . 13329
90.87.1 Axiom unit tests . . . 13329
90.87.2 Axiom help page . . . 13329
90.87.3 fortran code . . . 13331
90.87.4 lisp code . . . 13333
90.88 slartgp . . . 13335
90.88.1 Axiom unit tests . . . 13335
90.88.2 Axiom help page . . . 13335
90.88.3 fortran code . . . 13337

90.88.4 lisp code . . . 13339
90.89 slaruv . . . 13341
90.89.1 Axiom unit tests . . . 13341
90.89.2 Axiom help page . . . 13341
90.89.3 fortran code . . . 13343
90.89.4 lisp code . . . 13350
90.90 slas2 . . . 13358
90.90.1 Axiom unit tests . . . 13358
90.90.2 Axiom help page . . . 13358
90.90.3 fortran code . . . 13360
90.90.4 lisp code . . . 13362
90.91 slascl . . . 13363
90.91.1 Axiom unit tests . . . 13363
90.91.2 Axiom help page . . . 13363
90.91.3 fortran code . . . 13365
90.91.4 lisp code . . . 13370
90.92 slasd0 . . . 13376
90.92.1 Axiom unit tests . . . 13376
90.92.2 Axiom help page . . . 13376
90.92.3 fortran code . . . 13378
90.92.4 lisp code . . . 13382
90.93 slasd1 . . . 13387
90.93.1 Axiom unit tests . . . 13387
90.93.2 Axiom help page . . . 13387
90.93.3 fortran code . . . 13390
90.93.4 lisp code . . . 13393
90.94 slasd2 . . . 13397
90.94.1 Axiom unit tests . . . 13397
90.94.2 Axiom help page . . . 13397
90.94.3 fortran code . . . 13401
90.94.4 lisp code . . . 13409
90.95 slasd3 . . . 13419
90.95.1 Axiom unit tests . . . 13419
90.95.2 Axiom help page . . . 13419
90.95.3 fortran code . . . 13422
90.95.4 lisp code . . . 13427
90.96 slasd4 . . . 13440
90.96.1 Axiom unit tests . . . 13440
90.96.2 Axiom help page . . . 13440
90.96.3 fortran code . . . 13442
90.96.4 lisp code . . . 13461
90.97 slasd5 . . . 13488
90.97.1 Axiom unit tests . . . 13488
90.97.2 Axiom help page . . . 13489
90.97.3 fortran code . . . 13490
90.97.4 lisp code . . . 13493

90.98 slasd6 . . . 13497
90.98.1 Axiom unit tests . . . 13497
90.98.2 Axiom help page . . . 13497
90.98.3 fortran code . . . 13502
90.98.4 lisp code . . . 13505
90.99 slasd7 . . . 13509
90.99.1 Axiom unit tests . . . 13509
90.99.2 Axiom help page . . . 13510
90.99.3 fortran code . . . 13513
90.99.4 lisp code . . . 13520
90.100 slasd8 . . . 13528
90.100.1 Axiom unit tests . . . 13528
90.100.2 Axiom help page . . . 13528
90.100.3 fortran code . . . 13530
90.100.4 lisp code . . . 13534
90.101 slasda . . . 13541
90.101.1 Axiom unit tests . . . 13541
90.101.2 Axiom help page . . . 13541
90.101.3 fortran code . . . 13545
90.101.4 lisp code . . . 13550
90.102 slasdq . . . 13563
90.102.1 Axiom unit tests . . . 13563
90.102.2 Axiom help page . . . 13564
90.102.3 fortran code . . . 13566
90.102.4 lisp code . . . 13571
90.103 slasdt . . . 13578
90.103.1 Axiom unit tests . . . 13578
90.103.2 Axiom help page . . . 13578
90.103.3 fortran code . . . 13579
90.103.4 lisp code . . . 13581
90.104 slaset . . . 13583
90.104.1 Axiom unit tests . . . 13583
90.104.2 Axiom help page . . . 13584
90.104.3 fortran code . . . 13585
90.104.4 lisp code . . . 13587
90.105 slasr . . . 13589
90.105.1 Axiom unit tests . . . 13589
90.105.2 Axiom help page . . . 13589
90.105.3 fortran code . . . 13592
90.105.4 lisp code . . . 13597
90.106 slassq . . . 13609
90.106.1 Axiom unit tests . . . 13609
90.106.2 Axiom help page . . . 13610
90.106.3 fortran code . . . 13611
90.106.4 lisp code . . . 13612
90.107 slasv2 . . . 13614

90.107.1 Axiom unit tests . . . 13614
90.107.2 Axiom help page . . . 13614
90.107.3 Fortran code . . . 13616
90.107.4 Lisp code . . . 13620
90.108 dstiee . . . 13622
90.108.1 Axiom unit tests . . . 13622
90.108.2 Axiom help page . . . 13623
90.108.3 Fortran code . . . 13623
90.108.4 Lisp code . . . 13623
90.109 xerbla_array . . . 13624
90.109.1 Axiom unit tests . . . 13624
90.109.2 Axiom help page . . . 13624
90.109.3 Fortran code . . . 13625
90.109.4 Lisp code . . . 13626
90.110 dbbcsd - CS Decomposition of Unitary Matrix . . . 13626
90.110.1 Axiom unit tests . . . 13626
90.110.2 Axiom help page . . . 13627
90.110.3 Fortran code . . . 13632
90.110.4 Lisp code . . . 13662
90.111 dbdsqr - compute singular values from an SVD . . . 13711
90.111.1 Axiom unit tests . . . 13711
90.111.2 Axiom help page . . . 13711
90.111.3 Fortran code . . . 13714
90.111.4 Lisp code . . . 13739
90.112 zgbbrd - reduce complex band matrix to real upper bidiagonal . . . 13761
90.112.1 Axiom unit tests . . . 13761
90.112.2 Axiom help page . . . 13762
90.112.3 Fortran code . . . 13764
90.112.4 Lisp code . . . 13780
90.113 dbdsdc . . . 13796
90.113.1 Axiom unit tests . . . 13796
90.113.2 Axiom help page . . . 13796
90.113.3 Fortran code . . . 13800
90.113.4 Lisp code . . . 13813
90.114 dbdsqr . . . 13826
90.114.1 Axiom unit tests . . . 13826
90.114.2 Axiom help page . . . 13826
90.114.3 Fortran code . . . 13829
90.114.4 Lisp code . . . 13855
90.115 ddisna . . . 13877
90.115.1 Axiom unit tests . . . 13877
90.115.2 Axiom help page . . . 13877
90.115.3 Fortran code . . . 13879
90.115.4 Lisp code . . . 13884
90.116 dgebak . . . 13888
90.116.1 Axiom unit tests . . . 13888

90.116.2 Axiom help page . . . 13888
90.116.3 fortran code . . . 13890
90.116.4 lisp code . . . 13895
90.117 dgebal . . . 13900
90.117.1 Axiom unit tests . . . 13900
90.117.2 Axiom help page . . . 13900
90.117.3 fortran code . . . 13902
90.117.4 lisp code . . . 13912
90.118 dgebd2 . . . 13918
90.118.1 Axiom unit tests . . . 13918
90.118.2 Axiom help page . . . 13918
90.118.3 fortran code . . . 13921
90.118.4 lisp code . . . 13926
90.119 dgebrd . . . 13930
90.119.1 Axiom unit tests . . . 13930
90.119.2 Axiom help page . . . 13931
90.119.3 fortran code . . . 13934
90.119.4 lisp code . . . 13940
90.120 dgeev . . . 13944
90.120.1 Axiom unit tests . . . 13944
90.120.2 Axiom help page . . . 13945
90.120.3 fortran code . . . 13947
90.120.4 lisp code . . . 13960
90.121 dgeevx . . . 13975
90.121.1 Axiom unit tests . . . 13975
90.121.2 Axiom help page . . . 13975
90.121.3 fortran code . . . 13979
90.121.4 lisp code . . . 13995
90.122 dgehd2 . . . 14012
90.122.1 Axiom unit tests . . . 14012
90.122.2 Axiom help page . . . 14012
90.122.3 fortran code . . . 14014
90.122.4 lisp code . . . 14017
90.123 dgehrd . . . 14019
90.123.1 Axiom unit tests . . . 14019
90.123.2 Axiom help page . . . 14019
90.123.3 fortran code . . . 14022
90.123.4 lisp code . . . 14029
90.124 dgelq2 . . . 14035
90.124.1 Axiom unit tests . . . 14035
90.124.2 Axiom help page . . . 14035
90.124.3 fortran code . . . 14036
90.124.4 lisp code . . . 14040
90.125 dgelqf . . . 14041
90.125.1 Axiom unit tests . . . 14041
90.125.2 Axiom help page . . . 14042

90.125.3 fortran code . . . 14043
90.125.4 lisp code . . . 14049
90.126 dgeqr2 . . . 14052
90.126.1 Axiom unit tests . . . 14052
90.126.2 Axiom help page . . . 14053
90.126.3 fortran code . . . 14054
90.126.4 lisp code . . . 14057
90.127 dgeqrf . . . 14059
90.127.1 Axiom unit tests . . . 14059
90.127.2 Axiom help page . . . 14059
90.127.3 fortran code . . . 14061
90.127.4 lisp code . . . 14066
90.128 dgesdd . . . 14070
90.128.1 Axiom unit tests . . . 14070
90.128.2 Axiom help page . . . 14070
90.128.3 fortran code . . . 14073
90.128.4 lisp code . . . 14122
90.129 dgesvd . . . 14198
90.129.1 Axiom unit tests . . . 14198
90.129.2 Axiom help page . . . 14199
90.129.3 fortran code . . . 14202
90.129.4 lisp code . . . 14334
90.130 dgesv . . . 14464
90.130.1 Axiom unit tests . . . 14464
90.130.2 Axiom help page . . . 14465
90.130.3 fortran code . . . 14466
90.130.4 lisp code . . . 14469
90.131 dgetf2 . . . 14470
90.131.1 Axiom unit tests . . . 14470
90.131.2 Axiom help page . . . 14470
90.131.3 fortran code . . . 14472
90.131.4 lisp code . . . 14476
90.132 dgetrf . . . 14478
90.132.1 Axiom unit tests . . . 14478
90.132.2 Axiom help page . . . 14478
90.132.3 fortran code . . . 14480
90.132.4 lisp code . . . 14485
90.133 dgetrs . . . 14488
90.133.1 Axiom unit tests . . . 14488
90.133.2 Axiom help page . . . 14488
90.133.3 fortran code . . . 14490
90.133.4 lisp code . . . 14494
90.134 dhseqr . . . 14497
90.134.1 Axiom unit tests . . . 14497
90.134.2 Axiom help page . . . 14497
90.134.3 fortran code . . . 14500

90.134.4 lisp code . . . 14511
90.135 disnan . . . 14518
90.135.1 Axiom unit tests . . . 14518
90.135.2 Axiom help page . . . 14518
90.135.3 fortran code . . . 14519
90.135.4 lisp code . . . 14520
90.136 dlabad . . . 14521
90.136.1 Axiom unit tests . . . 14521
90.136.2 Axiom help page . . . 14521
90.136.3 fortran code . . . 14522
90.136.4 lisp code . . . 14523
90.137 dlabrd . . . 14524
90.137.1 Axiom unit tests . . . 14524
90.137.2 Axiom help page . . . 14524
90.137.3 fortran code . . . 14527
90.137.4 lisp code . . . 14534
90.138 dlacon . . . 14548
90.138.1 Axiom unit tests . . . 14548
90.138.2 Axiom help page . . . 14548
90.138.3 fortran code . . . 14550
90.138.4 lisp code . . . 14556
90.139 dlacpy . . . 14560
90.139.1 Axiom unit tests . . . 14560
90.139.2 Axiom help page . . . 14560
90.139.3 fortran code . . . 14562
90.139.4 lisp code . . . 14564
90.140 dladiv . . . 14566
90.140.1 Axiom unit tests . . . 14566
90.140.2 Axiom help page . . . 14566
90.140.3 fortran code . . . 14567
90.140.4 lisp code . . . 14571
90.141 dlaed6 . . . 14572
90.141.1 Axiom unit tests . . . 14572
90.141.2 Axiom help page . . . 14573
90.141.3 fortran code . . . 14574
90.141.4 lisp code . . . 14585
90.142 dlaexc . . . 14591
90.142.1 Axiom unit tests . . . 14591
90.142.2 Axiom help page . . . 14591
90.142.3 fortran code . . . 14593
90.142.4 lisp code . . . 14605
90.143 dlahqr . . . 14616
90.143.1 Axiom unit tests . . . 14616
90.143.2 Axiom help page . . . 14617
90.143.3 fortran code . . . 14619
90.143.4 lisp code . . . 14634

90.144 dlahrd DEPRECATED see dlahr2 . . . 14648
90.144.1 Axiom unit tests . . . 14648
90.144.2 Axiom help page . . . 14649
90.144.3 fortran code . . . 14651
90.144.4 lisp code . . . 14653
90.145 dlaisnan . . . 14657
90.145.1 Axiom unit tests . . . 14657
90.145.2 Axiom help page . . . 14657
90.145.3 fortran code . . . 14659
90.145.4 lisp code . . . 14659
90.146 dlaln2 . . . 14660
90.146.1 Axiom unit tests . . . 14660
90.146.2 Axiom help page . . . 14660
90.146.3 fortran code . . . 14663
90.146.4 lisp code . . . 14671
90.147 dlamch . . . 14686
90.147.1 Axiom unit tests . . . 14686
90.147.2 Axiom help page . . . 14686
90.147.3 fortran code . . . 14687
90.147.4 lisp code . . . 14689
90.148 dlamc1 . . . 14693
90.148.1 Axiom unit tests . . . 14693
90.148.2 Axiom help page . . . 14693
90.148.3 fortran code . . . 14694
90.148.4 lisp code . . . 14698
90.149 dlamc2 . . . 14699
90.149.1 Axiom unit tests . . . 14699
90.149.2 Axiom help page . . . 14699
90.149.3 fortran code . . . 14701
90.149.4 lisp code . . . 14705
90.150 dlamc3 . . . 14709
90.150.1 Axiom unit tests . . . 14709
90.150.2 Axiom help page . . . 14710
90.150.3 fortran code . . . 14710
90.150.4 lisp code . . . 14711
90.151 dlamc4 . . . 14711
90.151.1 Axiom unit tests . . . 14711
90.151.2 Axiom help page . . . 14712
90.151.3 fortran code . . . 14713
90.151.4 lisp code . . . 14714
90.152 dlamc5 . . . 14715
90.152.1 Axiom unit tests . . . 14715
90.152.2 Axiom help page . . . 14716
90.152.3 fortran code . . . 14717
90.152.4 lisp code . . . 14720
90.153 dlamrg . . . 14721

90.153.1 Axiom unit tests . . . 14721
90.153.2 Axiom help page . . . 14721
90.153.3 fortran code . . . 14723
90.153.4 lisp code . . . 14724
90.154 dlange . . . 14725
90.154.1 Axiom unit tests . . . 14725
90.154.2 Axiom help page . . . 14726
90.154.3 fortran code . . . 14727
90.154.4 lisp code . . . 14729
90.155 dlanhs . . . 14733
90.155.1 Axiom unit tests . . . 14733
90.155.2 Axiom help page . . . 14733
90.155.3 fortran code . . . 14734
90.155.4 lisp code . . . 14737
90.156 dlanst . . . 14740
90.156.1 Axiom unit tests . . . 14740
90.156.2 Axiom help page . . . 14740
90.156.3 fortran code . . . 14742
90.156.4 lisp code . . . 14744
90.157 dlanv2 . . . 14747
90.157.1 Axiom unit tests . . . 14747
90.157.2 Axiom help page . . . 14747
90.157.3 fortran code . . . 14748
90.157.4 lisp code . . . 14752
90.158 dlapy2 . . . 14754
90.158.1 Axiom unit tests . . . 14754
90.158.2 Axiom help page . . . 14754
90.158.3 fortran code . . . 14755
90.158.4 lisp code . . . 14756
90.159 dlapy3 . . . 14757
90.159.1 Axiom unit tests . . . 14757
90.159.2 Axiom help page . . . 14757
90.159.3 fortran code . . . 14758
90.159.4 lisp code . . . 14759
90.160 dlaqtr . . . 14760
90.160.1 Axiom unit tests . . . 14760
90.160.2 Axiom help page . . . 14760
90.160.3 fortran code . . . 14762
90.160.4 lisp code . . . 14774
90.161 dlarfb . . . 14793
90.161.1 Axiom unit tests . . . 14793
90.161.2 Axiom help page . . . 14793
90.161.3 fortran code . . . 14796
90.161.4 lisp code . . . 14806
90.162 dlarfg . . . 14826
90.162.1 Axiom unit tests . . . 14826

90.162.2 Axiom help page . . . 14826
90.162.3 fortran code . . . 14828
90.162.4 lisp code . . . 14830
90.163 dlarf . . . 14831
90.163.1 Axiom unit tests . . . 14831
90.163.2 Axiom help page . . . 14832
90.163.3 fortran code . . . 14833
90.163.4 lisp code . . . 14835
90.164 dlarft . . . 14837
90.164.1 Axiom unit tests . . . 14837
90.164.2 Axiom help page . . . 14838
90.164.3 fortran code . . . 14840
90.164.4 lisp code . . . 14843
90.165 dlarfx . . . 14849
90.165.1 Axiom unit tests . . . 14849
90.165.2 Axiom help page . . . 14850
90.165.3 fortran code . . . 14851
90.165.4 lisp code . . . 14863
90.166 dlartg . . . 14890
90.166.1 Axiom unit tests . . . 14890
90.166.2 Axiom help page . . . 14890
90.166.3 fortran code . . . 14892
90.166.4 lisp code . . . 14894
90.167 dlas2 . . . 14896
90.167.1 Axiom unit tests . . . 14896
90.167.2 Axiom help page . . . 14896
90.167.3 fortran code . . . 14897
90.167.4 lisp code . . . 14899
90.168 dlascl . . . 14900
90.168.1 Axiom unit tests . . . 14900
90.168.2 Axiom help page . . . 14901
90.168.3 fortran code . . . 14902
90.168.4 lisp code . . . 14907
90.169 dlasd0 . . . 14913
90.169.1 Axiom unit tests . . . 14913
90.169.2 Axiom help page . . . 14913
90.169.3 fortran code . . . 14915
90.169.4 lisp code . . . 14918
90.170 dlasd1 . . . 14923
90.170.1 Axiom unit tests . . . 14923
90.170.2 Axiom help page . . . 14923
90.170.3 fortran code . . . 14926
90.170.4 lisp code . . . 14929
90.171 dlasd2 . . . 14932
90.171.1 Axiom unit tests . . . 14932
90.171.2 Axiom help page . . . 14933

90.171.3 fortran code . . . 14936
90.171.4 lisp code . . . 14944
90.172 dlasd3 . . . 14953
90.172.1 Axiom unit tests . . . 14953
90.172.2 Axiom help page . . . 14953
90.172.3 fortran code . . . 14956
90.172.4 lisp code . . . 14961
90.173 dlasd4 . . . 14973
90.173.1 Axiom unit tests . . . 14973
90.173.2 Axiom help page . . . 14973
90.173.3 fortran code . . . 14975
90.173.4 lisp code . . . 14992
90.174 dlasd5 . . . 15019
90.174.1 Axiom unit tests . . . 15019
90.174.2 Axiom help page . . . 15020
90.174.3 fortran code . . . 15021
90.174.4 lisp code . . . 15024
90.175 dlasd6 . . . 15027
90.175.1 Axiom unit tests . . . 15027
90.175.2 Axiom help page . . . 15028
90.175.3 fortran code . . . 15032
90.175.4 lisp code . . . 15035
90.176 dlasd7 . . . 15038
90.176.1 Axiom unit tests . . . 15038
90.176.2 Axiom help page . . . 15039
90.176.3 fortran code . . . 15042
90.176.4 lisp code . . . 15048
90.177 dlasd8 . . . 15055
90.177.1 Axiom unit tests . . . 15055
90.177.2 Axiom help page . . . 15056
90.177.3 fortran code . . . 15058
90.177.4 lisp code . . . 15062
90.178 dlasda . . . 15068
90.178.1 Axiom unit tests . . . 15068
90.178.2 Axiom help page . . . 15068
90.178.3 fortran code . . . 15072
90.178.4 lisp code . . . 15077
90.179 dlasdq . . . 15089
90.179.1 Axiom unit tests . . . 15089
90.179.2 Axiom help page . . . 15089
90.179.3 fortran code . . . 15092
90.179.4 lisp code . . . 15097
90.180 dlasdt . . . 15103
90.180.1 Axiom unit tests . . . 15103
90.180.2 Axiom help page . . . 15103
90.180.3 fortran code . . . 15104

90.180.4 Lisp code . . . 15106
90.181 dlaset . . . 15108
90.181.1 Axiom unit tests . . . 15108
90.181.2 Axiom help page . . . 15108
90.181.3 Fortran code . . . 15110
90.181.4 Lisp code . . . 15112
90.182 dlasq1 . . . 15113
90.182.1 Axiom unit tests . . . 15113
90.182.2 Axiom help page . . . 15114
90.182.3 Fortran code . . . 15115
90.182.4 Lisp code . . . 15117
90.183 dlasq2 . . . 15121
90.183.1 Axiom unit tests . . . 15121
90.183.2 Axiom help page . . . 15121
90.183.3 Fortran code . . . 15123
90.183.4 Lisp code . . . 15130
90.184 dlasq3 . . . 15150
90.184.1 Axiom unit tests . . . 15150
90.184.2 Axiom help page . . . 15150
90.184.3 Fortran code . . . 15152
90.184.4 Lisp code . . . 15157
90.185 dlasq4 . . . 15166
90.185.1 Axiom unit tests . . . 15166
90.185.2 Axiom help page . . . 15166
90.185.3 Fortran code . . . 15168
90.185.4 Lisp code . . . 15174
90.186 dlasq5 . . . 15182
90.186.1 Axiom unit tests . . . 15182
90.186.2 Axiom help page . . . 15182
90.186.3 Fortran code . . . 15184
90.186.4 Lisp code . . . 15187
90.187 dlasq6 . . . 15199
90.187.1 Axiom unit tests . . . 15199
90.187.2 Axiom help page . . . 15200
90.187.3 Fortran code . . . 15201
90.187.4 Lisp code . . . 15204
90.188 dlasr . . . 15210
90.188.1 Axiom unit tests . . . 15210
90.188.2 Axiom help page . . . 15210
90.188.3 Fortran code . . . 15213
90.188.4 Lisp code . . . 15218
90.189 dlasrt . . . 15229
90.189.1 Axiom unit tests . . . 15229
90.189.2 Axiom help page . . . 15230
90.189.3 Fortran code . . . 15231
90.189.4 Lisp code . . . 15235

90.190 dlassq . . . 15239
90.190.1 Axiom unit tests . . . 15239
90.190.2 Axiom help page . . . 15240
90.190.3 fortran code . . . 15241
90.190.4 lisp code . . . 15242
90.191 dlasv2 . . . 15243
90.191.1 Axiom unit tests . . . 15243
90.191.2 Axiom help page . . . 15244
90.191.3 fortran code . . . 15246
90.191.4 lisp code . . . 15249
90.192 dlaswp . . . 15251
90.192.1 Axiom unit tests . . . 15251
90.192.2 Axiom help page . . . 15252
90.192.3 fortran code . . . 15253
90.192.4 lisp code . . . 15255
90.193 dlasy2 . . . 15257
90.193.1 Axiom unit tests . . . 15257
90.193.2 Axiom help page . . . 15257
90.193.3 fortran code . . . 15259
90.193.4 lisp code . . . 15266
90.194 dorg2r . . . 15277
90.194.1 Axiom unit tests . . . 15277
90.194.2 Axiom help page . . . 15277
90.194.3 fortran code . . . 15279
90.194.4 lisp code . . . 15281
90.195 dorgbr . . . 15282
90.195.1 Axiom unit tests . . . 15282
90.195.2 Axiom help page . . . 15283
90.195.3 fortran code . . . 15285
90.195.4 lisp code . . . 15288
90.196 dorghr . . . 15293
90.196.1 Axiom unit tests . . . 15293
90.196.2 Axiom help page . . . 15294
90.196.3 fortran code . . . 15295
90.196.4 lisp code . . . 15298
90.197 dorgl2 . . . 15301
90.197.1 Axiom unit tests . . . 15301
90.197.2 Axiom help page . . . 15301
90.197.3 fortran code . . . 15302
90.197.4 lisp code . . . 15304
90.198 dorglq . . . 15306
90.198.1 Axiom unit tests . . . 15306
90.198.2 Axiom help page . . . 15307
90.198.3 fortran code . . . 15308
90.198.4 lisp code . . . 15312
90.199 dorgqr . . . 15316

90.199.1 Axiom unit tests . . . 15316
90.199.2 Axiom help page . . . 15316
90.199.3 Fortran code . . . 15318
90.199.4 Lisp code . . . 15321
90.200 dorm2r . . . 15326
90.200.1 Axiom unit tests . . . 15326
90.200.2 Axiom help page . . . 15326
90.200.3 Fortran code . . . 15328
90.200.4 Lisp code . . . 15331
90.201 dormbr . . . 15333
90.201.1 Axiom unit tests . . . 15333
90.201.2 Axiom help page . . . 15333
90.201.3 Fortran code . . . 15336
90.201.4 Lisp code . . . 15340
90.202 dorml2 . . . 15345
90.202.1 Axiom unit tests . . . 15345
90.202.2 Axiom help page . . . 15345
90.202.3 Fortran code . . . 15347
90.202.4 Lisp code . . . 15350
90.203 dormlq . . . 15352
90.203.1 Axiom unit tests . . . 15352
90.203.2 Axiom help page . . . 15353
90.203.3 Fortran code . . . 15355
90.203.4 Lisp code . . . 15359
90.204 dormqr . . . 15363
90.204.1 Axiom unit tests . . . 15363
90.204.2 Axiom help page . . . 15364
90.204.3 Fortran code . . . 15366
90.204.4 Lisp code . . . 15370
90.205 dtrevc . . . 15374
90.205.1 Axiom unit tests . . . 15374
90.205.2 Axiom help page . . . 15374
90.205.3 Fortran code . . . 15378
90.205.4 Lisp code . . . 15395
90.206 dtrexc . . . 15432
90.206.1 Axiom unit tests . . . 15432
90.206.2 Axiom help page . . . 15432
90.206.3 Fortran code . . . 15434
90.206.4 Lisp code . . . 15440
90.207 dtrsna . . . 15447
90.207.1 Axiom unit tests . . . 15447
90.207.2 Axiom help page . . . 15448
90.207.3 Fortran code . . . 15451
90.207.4 Lisp code . . . 15458
90.208 eeeck . . . 15471
90.208.1 Axiom unit tests . . . 15471

90.208.2 Axiom help page . . . 15471
90.208.3 fortran code . . . 15472
90.208.4 lisp code . . . 15475
90.209 ilaenv . . . 15476
90.209.1 Axiom unit tests . . . 15476
90.209.2 Axiom help page . . . 15476
90.209.3 fortran code . . . 15479
90.209.4 lisp code . . . 15488
90.210 ilazlc . . . 15495
90.210.1 Axiom unit tests . . . 15495
90.210.2 Axiom help page . . . 15495
90.210.3 fortran code . . . 15496
90.210.4 lisp code . . . 15497
90.211 ilazlr . . . 15498
90.211.1 Axiom unit tests . . . 15498
90.211.2 Axiom help page . . . 15499
90.211.3 fortran code . . . 15500
90.211.4 lisp code . . . 15501
90.212 zgebak . . . 15502
90.212.1 Axiom unit tests . . . 15502
90.212.2 Axiom help page . . . 15502
90.212.3 fortran code . . . 15505
90.212.4 lisp code . . . 15508
90.213 zgebal . . . 15512
90.213.1 Axiom unit tests . . . 15512
90.213.2 Axiom help page . . . 15512
90.213.3 fortran code . . . 15515
90.213.4 lisp code . . . 15520
90.214 zgeev . . . 15527
90.214.1 Axiom unit tests . . . 15527
90.214.2 Axiom help page . . . 15527
90.214.3 fortran code . . . 15530
90.214.4 lisp code . . . 15537
90.215 zgehd2 . . . 15549
90.215.1 Axiom unit tests . . . 15549
90.215.2 Axiom help page . . . 15549
90.215.3 fortran code . . . 15552
90.215.4 lisp code . . . 15553
90.216 zhseqr . . . 15555
90.216.1 Axiom unit tests . . . 15555
90.216.2 Axiom help page . . . 15556
90.216.3 fortran code . . . 15561
90.216.4 lisp code . . . 15565
90.217 zlacgv . . . 15571
90.217.1 Axiom unit tests . . . 15571
90.217.2 Axiom help page . . . 15571

90.217.3 Fortran code . . . 15573
90.217.4 Lisp code . . . 15574
90.218 slacpy . . . 15575
90.218.1 Axiom unit tests . . . 15575
90.218.2 Axiom help page . . . 15575
90.218.3 Fortran code . . . 15577
90.218.4 Lisp code . . . 15578
90.219 sladiv . . . 15580
90.219.1 Axiom unit tests . . . 15580
90.219.2 Axiom help page . . . 15580
90.219.3 Fortran code . . . 15581
90.219.4 Lisp code . . . 15582
90.220 slahqr . . . 15582
90.220.1 Axiom unit tests . . . 15582
90.220.2 Axiom help page . . . 15583
90.220.3 Fortran code . . . 15586
90.220.4 Lisp code . . . 15594
90.221 slahr2 . . . 15606
90.221.1 Axiom unit tests . . . 15606
90.221.2 Axiom help page . . . 15606
90.221.3 Fortran code . . . 15609
90.221.4 Lisp code . . . 15612
90.222 slange . . . 15619
90.222.1 Axiom unit tests . . . 15619
90.222.2 Axiom help page . . . 15619
90.222.3 Fortran code . . . 15621
90.222.4 Lisp code . . . 15623
90.223 slaqr0 . . . 15626
90.223.1 Axiom unit tests . . . 15626
90.223.2 Axiom help page . . . 15627
90.223.3 Fortran code . . . 15630
90.223.4 Lisp code . . . 15640
90.224 slaqr1 . . . 15653
90.224.1 Axiom unit tests . . . 15653
90.224.2 Axiom help page . . . 15653
90.224.3 Fortran code . . . 15655
90.224.4 Lisp code . . . 15656
90.225 slaqr2 . . . 15659
90.225.1 Axiom unit tests . . . 15659
90.225.2 Axiom help page . . . 15660
90.225.3 Fortran code . . . 15664
90.225.4 Lisp code . . . 15670
90.226 slaqr3 . . . 15680
90.226.1 Axiom unit tests . . . 15680
90.226.2 Axiom help page . . . 15680
90.226.3 Fortran code . . . 15685

90.226.4 lisp code . . . 15691
90.227 zlaqr4 . . . 15702
90.227.1 Axiom unit tests . . . 15702
90.227.2 Axiom help page . . . 15702
90.227.3 fortran code . . . 15706
90.227.4 lisp code . . . 15715
90.228 slaqr5 . . . 15727
90.228.1 Axiom unit tests . . . 15727
90.228.2 Axiom help page . . . 15728
90.228.3 fortran code . . . 15732
90.228.4 lisp code . . . 15745
90.229 zlarfb . . . 15779
90.229.1 Axiom unit tests . . . 15779
90.229.2 Axiom help page . . . 15779
90.229.3 fortran code . . . 15783
90.229.4 lisp code . . . 15794
90.230 zlarf . . . 15815
90.230.1 Axiom unit tests . . . 15815
90.230.2 Axiom help page . . . 15815
90.230.3 fortran code . . . 15817
90.230.4 lisp code . . . 15820
90.231 zlarfg . . . 15822
90.231.1 Axiom unit tests . . . 15822
90.231.2 Axiom help page . . . 15822
90.231.3 fortran code . . . 15824
90.231.4 lisp code . . . 15826
90.232 zlarft . . . 15828
90.232.1 Axiom unit tests . . . 15828
90.232.2 Axiom help page . . . 15829
90.232.3 fortran code . . . 15832
90.232.4 lisp code . . . 15835
90.233 slartg . . . 15842
90.233.1 Axiom unit tests . . . 15842
90.233.2 Axiom help page . . . 15843
90.233.3 fortran code . . . 15844
90.233.4 lisp code . . . 15848
90.234 slascl . . . 15850
90.234.1 Axiom unit tests . . . 15850
90.234.2 Axiom help page . . . 15851
90.234.3 fortran code . . . 15853
90.234.4 lisp code . . . 15858
90.235 zlaset . . . 15864
90.235.1 Axiom unit tests . . . 15864
90.235.2 Axiom help page . . . 15864
90.235.3 fortran code . . . 15866
90.235.4 lisp code . . . 15867

90.236 lassq . . . 15870
90.236.1 Axiom unit tests . . . 15870
90.236.2 Axiom help page . . . 15870
90.236.3 Fortran code . . . 15872
90.236.4 Lisp code . . . 15873
90.237 latrs . . . 15875
90.237.1 Axiom unit tests . . . 15875
90.237.2 Axiom help page . . . 15875
90.237.3 Fortran code . . . 15879
90.237.4 Lisp code . . . 15894
90.238 srot . . . 15910
90.238.1 Axiom unit tests . . . 15910
90.238.2 Axiom help page . . . 15910
90.238.3 Fortran code . . . 15912
90.238.4 Lisp code . . . 15914
90.239 trevc . . . 15915
90.239.1 Axiom unit tests . . . 15915
90.239.2 Axiom help page . . . 15915
90.239.3 Fortran code . . . 15919
90.239.4 Lisp code . . . 15925
90.240 trexc . . . 15934
90.240.1 Axiom unit tests . . . 15934
90.240.2 Axiom help page . . . 15935
90.240.3 Fortran code . . . 15937
90.240.4 Lisp code . . . 15939
90.241 ung2r . . . 15942
90.241.1 Axiom unit tests . . . 15942
90.241.2 Axiom help page . . . 15942
90.241.3 Fortran code . . . 15944
90.241.4 Lisp code . . . 15946
90.242 unghr . . . 15948
90.242.1 Axiom unit tests . . . 15948
90.242.2 Axiom help page . . . 15948
90.242.3 Fortran code . . . 15950
90.242.4 Lisp code . . . 15953
90.243 ungqr . . . 15955
90.243.1 Axiom unit tests . . . 15955
90.243.2 Axiom help page . . . 15956
90.243.3 Fortran code . . . 15958
90.243.4 Lisp code . . . 15961
90.244 unm2r . . . 15966
90.244.1 Axiom unit tests . . . 15966
90.244.2 Axiom help page . . . 15966
90.244.3 Fortran code . . . 15969
90.244.4 Lisp code . . . 15971
90.245 unmhr . . . 15974

90.245.1 Axiom unit tests . . . 15974
90.245.2 Axiom help page . . . 15974
90.245.3 fortran code . . . 15977
90.245.4 lisp code . . . 15980
90.246 unmqr . . . 15983
90.246.1 Axiom unit tests . . . 15983
90.246.2 Axiom help page . . . 15983
90.246.3 fortran code . . . 15986
90.246.4 lisp code . . . 15990

91 LAPACK tests **15995**

92 ISAAC Secure Random Number Generation **16011**
92.0.5 Axiom help page . . . 16012
92.0.6 defstruct isaacCtx . . . 16014
92.0.7 defun generateNextIsaacBlock . . . 16016
92.0.8 defun rand32 . . . 16017
92.0.9 defun randBits . . . 16017
92.0.10 defmacro incfWrap32 . . . 16017
92.0.11 defmacro mix . . . 16018
92.0.12 defun scramble . . . 16020
92.0.13 defun initKernelSeed . . . 16021
92.0.14 defun initCommonLispRandomSeed . . . 16022
92.0.15 defun initNullSeed . . . 16022

93 Chunk collections **16025**

Bibliography **16035**

Index **16047**

New Foreword

On October 1, 2001 Axiom was withdrawn from the market and ended life as a commercial product. On September 3, 2002 Axiom was released under the Modified BSD license, including this document. On August 27, 2003 Axiom was released as free and open source software available for download from the Free Software Foundation's website, Savannah.

Work on Axiom has had the generous support of the Center for Algorithms and Interactive Scientific Computation (CAISS) at City College of New York. Special thanks go to Dr. Gilbert Baumslag for his support of the long term goal.

The online version of this documentation is roughly 1000 pages. In order to make printed versions we've broken it up into three volumes. The first volume is tutorial in nature. The second volume is for programmers. The third volume is reference material. We've also added a fourth volume for developers. All of these changes represent an experiment in print-on-demand delivery of documentation. Time will tell whether the experiment succeeded.

Axiom has been in existence for over thirty years. It is estimated to contain about three hundred man-years of research and has, as of September 3, 2003, 143 people listed in the credits. All of these people have contributed directly or indirectly to making Axiom available. Axiom is being passed to the next generation. I'm looking forward to future milestones.

With that in mind I've introduced the theme of the "30 year horizon". We must invent the tools that support the Computational Mathematician working 30 years from now. How will research be done when every bit of mathematical knowledge is online and instantly available? What happens when we scale Axiom by a factor of 100, giving us 1.1 million domains? How can we integrate theory with code? How will we integrate theorems and proofs of the mathematics with space-time complexity proofs and running code? What visualization tools are needed? How do we support the conceptual structures and semantics of mathematics in effective ways? How do we support results from the sciences? How do we teach the next generation to be effective Computational Mathematicians?

The "30 year horizon" is much nearer than it appears.

Tim Daly
CAISS, City College of New York
November 10, 2003 ((iHy))

Chapter 1

Numerical Analysis

We can describe each number as x^* which has a machine-representable form which differs from the number x it is intended to represent. Quoting Householder [Hous81] we get:

$$x^* = \pm(x_1\beta^{-1} + x_2\beta^{-2} + \cdots + x_\lambda\beta^\lambda)\beta^\sigma$$

where β is the base, usually 2 or 10, λ is a positive integer, and σ is any integer, possibly zero. It may be that λ is fixed throughout the course of the computation, or it may vary, but in any case it is limited by practical considerations. Such a number will be called a representation. It may be that x^* is obtained by "rounding off" a number whose true value is x (for example, $x = 1/3$, $x^* = 0.33$), or that x^* is the result of measuring physically a quantity whose true value is x, or that x^* is the result of a computation intended to give an approximation to the quantity x.

Suppose one is interested in performing an operations ω upon a pair of numbers x and y. That is to say, $x\omega y$ may represent a product of x and y, a quotient of x by y, the yth power of x, ... In the numerical computation, however, one has only x^* and y^* upon which to operate, not x and y (or at least these are the quantitites upon which one does, in fact operate). Not only this, but often one does not even perform the strict operation ω, but rather a pseudo operation ω^*, which yields a rounded-off product, quotient, power, etc. Hence, instead of obtaining the desired result $x\omega y$, one obtains a result $x^*\omega^*y^*$.

The error in the result is therefore

$$x\omega y - x^*\omega^*y^* = (x\omega y - x^*\omega y^*) + (x^*\omega y^* - x^*\omega^*y^*)$$

Since x^* and y^* are numbers, the operation ω can be applied to them, and $x^*\omega y^*$ is well defined, except for special cases as when ω represents division and $y^* = 0$. But the expression in the first parenthesees on the right represents propagated error, and that in the second parentheses represents generated error, or round-off. Hence the total error in the result is the sum of the error propagated by the operation and that generated by the operation.

Householder notes that, given two operations ω and ϕ, it may be true that the operations are associative, e.g:

$$(x^*\omega y^*)\phi z^* = x^*\omega(y^*\phi z^*)$$

but if we expand these in terms of the above definitions of propogation and generation error we get two different expressions:

$$\begin{aligned}(x^*\omega y^*)\phi z^* - (x^*\omega^* y^*)\phi^* z^* &= [(x^*\omega y^*)\phi z^* - (x^*\omega^* y^*)\phi z^*] \\ &+ [(x^*\omega^* y^*)\phi z^* - (x^*\omega^* y^*)\phi^* z^*]\end{aligned}$$

$$\begin{aligned}x^*\omega(y^*\phi z^*) - x^*\omega^*(y^*\phi^* z^*) &= [x^*\omega(y^*\phi z^*) - x^*\omega(y^*\phi^* z^*)] \\ &+ [x^*\omega(y^*\phi^* z^*) - x^*\omega^*(y^*\phi^* z^*)]\end{aligned}$$

These are not always equal which implies that the strictly machine operations are not necessarily commutative.

Householder distinguishes a third class of error (besides propagation and generative) called residual errors. This occurs because some functions are approximated by infinite series. The finite computation of the series forces the truncation of the remaining terms causing these residual errors.

We will try to perform an analysis of each of the routines in this library for the given inputs to try to see the propagation errors and generation errors they introduce. Every effort will be made to minimize these errors. In particular, we will appeal to the machine generated code to see what approximations actually occur.

Chapter 2

The Quality of Computed Solutions by Sven Hammarling

This is quoted with permission from Hammarling[Hamm05]

2.1 Introduction

This report is concerned with the quality of the computed numerical solutions of mathematical problems. For example, suppose we wish to solve the system of linear equations $Ax = b$ using a numerical software package. The package will return a computed solution, say $\tilde{x}$, and we wish to judge whether or not $\tilde{x}$ is a reasonable solution to the equations. Sadly, all too often software packages return poor, or even incorrect, numerical results and give the user no means by which to judge the quality of the numerical results. In 1971, Leslie Fox commented [[Foxx71] 1971, p. 296]

> I have little doubt that about 80 per cent. of all the results printed from the computer are in error to a much greater extent than the user would believe, ...

More than thirty years on that paper is still very relevant and worth reading. Another very readable article is [Fors70].

The quality of computed solutions is concerned with assessing how good a computed solution is in some appropriate measure. Quality software should implement reliable algorithms and should, if possible, provide measures of solution quality.

In this report we give an introduction to ideas that are important in understanding and measuring the quality of computed solutions. In particular we review the ideas of condition, stability and error analysis, and their realisation in numerical software. We take as the principal example LAPACK [Ande99], a package for the solution of dense and banded linear algebra problems, but also draw on examples from the NAG Library [NAGa05] and

elsewhere . The aim is not to show how to perform an error analysis, but to indicate why an understanding of the ideas is important in judging the quality of numerical solutions, and to encourage the use of software that returns indicators of the quality of the results. We give simple examples to illustrate some of the ideas that are important when designing reliable numerical software.

Computing machines use floating point arithmetic for their computation, and so we start with an introduction to floating point numbers.

2.2 Floating Point Numbers and IEEE Arithmetic

Floating point numbers are a subset of the real numbers that can be conveniently represented in the finite word length of a computer, without unduly restricting the range of numbers represented. For example, the ANSI/IEEE standard for binary floating point arithmetic [IEEE85] uses 64 bits to represent double precision numbers in the approximate range $10^{\pm 308}$. A floating point number, x, can be represented in terms of four integers as

$$x = \pm m \times b^{e-t}$$

where b is the *base* or *radix*, t is the *precision*, e is the *exponent* with an *exponent range* of $[e_{min}, e_{max}]$ and m is the *mantissa* or *significand*, satisfying $0 \leq m \leq b^t - 1$. If $x \neq 0$ and $m \geq b^{t-1}$ then x is said to be *normalized*. An alternative, equivalent representation of x is

$$\begin{aligned} x &= \pm 0.d_1 d_2 \ldots d_t \times b^e \\ &= \pm \left(\tfrac{d_1}{b} + \tfrac{d_2}{b^2} + \cdots + \tfrac{d_t}{b^t} \right) \times b^e \end{aligned}$$

where each digit satisfies $0 \leq d_i \leq b - 1$. If $d_1 \neq 0$ then we see that x is normalized. If $d_1 = 0$ and $x \neq 0$ then x is said to be *denormalized*. Denormalized numbers between 0 and the smallest normalized number are called *subnormal*. Note that denormalized numbers do not have the full t digits of precision.

The following example, which is not intended to be realistic, illustrates the model.

2.2.1 Example 2.1 (Floating point numbers)

$$b = 2, \quad t = 2, \quad e_{\min} = -2, \quad e_{\max} = 2$$

All the normalized numbers have $d_1 = 1$ and either $d_2 = 0$ or $d_2 = 1$, that is m is one of the two binary integers $m = 10$ $(= 2)$ or $m = 11$ $(= 3)$. Denormalized numbers have $m = 01$ $(= 1)$. Thus the smallest positive normalized number is $2 \times 2^{e_{\min} - t} = \frac{1}{8}$ and the largest is $3 \times 2^{e_{\max} - t} = 3$. The value $1 \times 2^{e_{\min} - t} = \frac{1}{16}$ is the only positive subnormal number in this system. The complete set of non-negative normalized numbers is

$$0, \frac{1}{8}, \frac{3}{16}, \frac{1}{4}, \frac{3}{8}, \frac{1}{2}, \frac{3}{4}, 1, \frac{3}{2}, 2, 3$$

The set of non-negative floating point numbers is illustrated in Figure 1, where the subnormal number is indicated by a dashed line.

$b = 2,\ t = 2,\ e_{\min} = -2,\ e_{\max} = 2$

Figure 1: Floating Point Number Example

Note that floating point numbers are not equally spaced absolutely, but the relative spacing between numbers is approximately equal. The value

$$u = \frac{1}{2} \times b^{1-t} \tag{1}$$

is called the *unit roundoff*, or the *relative machine precision* and is the furthest distance relative to unity between a real number and the nearest floating point number. In Example 2.1, $u = \frac{1}{4} = 0.25$ and we can see, for example, that the furthest real number from 1.0 is 1.25 and the furthest real number from 2.0 is 2.5. u is fundamental to floating point error analysis.

The value $e_M = 2u$ is called *machine epsilon*.

The ANSI/IEEE standard mentioned above (usually referred to as IEEE arithmetic), which of course has $b = 2$, specifies:

- floating point number formats
- results of the basic floating point operations
- rounding modes
- signed zero, infinity ($\pm\infty$) and not-a-number (NaN)
- floating point exceptions and their handling and
- conversion between formats

Thankfully, nowadays almost all machines use IEEE arithmetic. There is also a generic ANSI/IEEE, base independent, standard [IEEE87]. The formats supported by the ANSI/IEEE binary standard are indicated in Table 1.

Format	Precision	Exponent	Approx Range	Approx Precision
Single	24 bits	8 bits	$10^{\pm 38}$	10^{-8}
Double	53 bits	11 bits	$10^{\pm 308}$	10^{-16}
Extended	$\geq$ 64 bits	$\geq$ 15 bits	$10^{\pm 4932}$	10^{-20}

The default rounding mode for IEEE arithmetic is round to nearest, in which a real number is represented by the nearest floating point number, with rules as to how to handle a tie [[Over01], Chapter 5].

Whilst the ANSI/IEEE standard has been an enormous help in standardizing floating point computation, it should be noted that moving a computation between machines that implement IEEE arithmetic does not guarantee that the computed results will be the same. Variations can occur due to such things as compiler optimization, the use of extended precision registers, and fused multiply-add.

Further discussion of floating point numbers and IEEE arithmetic can be found in [High02] and [Over01].

The value u can be obtained from the LAPACK function SLAMCH, for single precision arithmetic, or DLAMCH for double precision arithmetic by calling the function with the argument CMACH as e, and is also returned by the NAG Fortran Library routine X02AJF[1] It should be noted that on machines that truncate, rather than round, M is returned in place of u, but such machines are now rare. It should also be noted that e in S/DLAMCH represents eps, but this should not be confused with M. The Matlab built in variable eps returns M[Math05], as does the Fortran 95/Fortran 2003 numeric enquiry function epsilon [Metc96, Metc04].

2.3 Why Worry about Computed Solutions?

In this section we consider some simple examples of numerical computation that need care in order to obtain reasonable solutions. For clarity of exposition, most of the examples in this and the following sections are illustrated using decimal floating point (significant figure) arithmetic, with round to nearest.

The first example illustrates the problem of damaging subtraction, usually referred to as *cancellation*.

2.3.1 Example 3.1 (Cancellation)

Using four figure decimal arithmetic, suppose we wish to compute $s = 1.000 + 1.000 \times 10^4 - 1.000 \times 10^4$. *If we compute in the standard way from left to right we obtain*

$$\begin{aligned} s &= 1.000 + 1.000 \times 10^4 - 1.000 \times 10^4 \\ &= (1.000 + 1.000 \times 10^4) - 1.000 \times 10^4 \\ &= 1.000 \times 10^4 - 1.000 \times 10^4 \\ &= 0 \end{aligned}$$

instead of the correct result of 1.0. Although the cancellation (subtraction) was performed exactly, it lost all the information for the solution.

[1] In some ports it actually returns $u + b^{1=2t}$. See the X02 Chapter introduction[NAGb05].

As Example 3.1 illustrates, the cause of the poor result often happens before the cancellation, and the cancellation is just the final nail in the coffin. In Example 3.1, the damage was done in computing $s = 1.000 + 1.000 \times 10^4$, where we lost important information (1.000). It should be said that the subtraction of nearly equal numbers is not always damaging.

Most problems have alternative formulations which are theoretically equivalent, but may computationally yield quite different results. The following example illustrates this in the case of computing sample variances.

2.3.2 Example 3.2 (Sample variance [High02],Section 1.9)

The sample variance of a set of n values $x_1, x_2, \ldots, x_n$ is defined as

$$s_n^2 = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \overline{x})^2 \tag{2}$$

where $\overline{x}$ is the sample mean of the n values

$$\overline{x} = \frac{1}{n} \sum_{i=1}^{n} x_i$$

An alternative, theoretically equivalent, formula to compute the sample variance which requires only one pass through the data is given by

$$s_n^2 = \frac{1}{n-1} \left(\sum_{i=1}^{n} x_i^2 - \frac{1}{n} \left(\sum_{i=1}^{n} x_i \right)^2 \right) \tag{3}$$

If x^T = (10000 10001 10002) then using 8 figure arithmetic (2) gives $s^2 = 1.0$, the correct answer, but (3) gives $s^2 = 0.0$, with a relative error of 1.0.

(3) can clearly suffer from cancellation, as illustrated in the example. On the other hand, (2) always gives good results unless n is very large [[High02], Problem 1.10]. See also [Chan83] for further discussion of the problem of computing sample variances.

Sadly, it is not unknown for software packages and calculators to implement the algorithm of (3). For example in Excel 2002 from Microsoft Office XP (and in previous versions of Excel also), the function STDEV computes the standard deviation, s, of the data

$$x^T = (10000000010000000011000000002)$$

as $s = 0$. Considering the pervasive use of Excel and the importance of standard deviation and its use in applications, it is disappointing to realise that (3) has been used by these versions of Excel.[2] See [Coxx00] for further information, as well as [Knus98], [Mccu99] and [Mccu02]. The spreadsheet from OpenOffice.org version 1.0.2 produces the same result, but gives no information on the method used in its help system; on the other hand the Gnumeric

[2]The algorithm has at last been replaced in Excel from Office 2003, which now gives the correct answer.

spreadsheet (version 1.0.12) gives the correct result, although again the function description does not describe the method used.[3]

A result that is larger than the largest representable floating point number is said to *overflow*. For example, in double precision IEEE arithmetic for which the approximate range is $10^{\pm 308}$, if $x = 10^{200}$, then x^2 would overflow. Similarly, x^{-2} is said to *underflow* because it is smaller than the smallest non-zero representable floating point number.

As with the unit rounding error or machine epsilon discussed in Section 2, the overflow and underflow thresholds can be obtained from the LAPACK function S/DLAMCH by calling the function with the argument CMACH as o and u respectively; from the NAG Fortran Library routines X02ALF and X02AKF respectively; the Matlab built in variables `realmax` and `realmin`; and from the Fortran 95 numeric enquiry functions `huge` and `tiny`.

Care needs to be taken to avoid unnecessary overflow and damaging underflow. The following example illustrates this care in the case of computing the hypotenuse of the right angled triangle shown in Figure 2.

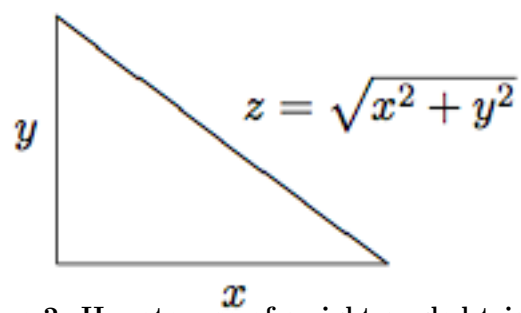

Figure 2: Hypotenuse of a right angled triangle

2.3.3 Example 3.3 (Hypotenuse)

in Figure 2, if x or y is very large there is a danger of overflow, even if z is representable. Assuming that x and y are non-negative, a safe method of computing z is

$$a = \max(x, y), \;\; b = \min(x, y)$$

$$z = \begin{cases} a\sqrt{1 + \left(\frac{b}{a}\right)^2}, & a > 0 \\ 0 & a = 0 \end{cases}$$

This also avoids z being computed as zero if x^2 and y^2 both underflow. We note that

[3]OpenOffice.org version 2.0 also now gives the correct result

[Stew98], p.139 and p144 actually recommends computing z as

$$z = \begin{cases} s\sqrt{\left(\frac{x}{s}\right)^2 + \left(\frac{y}{s}\right)^2}, \text{where } s = x + y, & x > 0 \\ 0 & s = 0 \end{cases}$$

because this is slightly more accurate on a hexadecimal machine. An interesting alternative for computing Pythagorean sums is given in [Mole83]; see also [Dubr83] and [High05], section 22.9.

We can see that (3) of Example 3.2 also has the potential for overflow and underflow and, as well as implementing this formula rather than a more stable version, Excel does not take the necessary care to avoid underflow and overflow. For example, for the values (1.0E200, 1.0E200), STDEV in Excel 2003 from Microsoft Office 2003 returns the mysterious symbol #NUM!, which signifies a numeric exception, in this case overflow, due to the fact that $(10.0^{200})^2$ overflows in IEEE double precision arithmetic. The correct standard deviation is of course 0. Similarily, for the values (0, 1.0E-200, 2.0E-200), STDEV returns the value 0 rather than the correct value of 1.0E-200. OpenOffice.org version 1.0.2 also returns zero for this data, and overflows on the previous data. Mimicking Excel is not necessarily a good thing![4]

The computation of the modulus of a complex number $x = x_r + ix_i$ requires almost the same computation as that in Example 3.3.

2.3.4 Example 3.4 (Modulus of a complex number)

$$|x| = \sqrt{x_r^2 + x_i^2}$$

$$a = \max(|x_r|, |x_i|),\ b = \min(|x_r|, |x_i|)$$

$$|x| = \begin{cases} a\sqrt{1 + \left(\frac{b}{a}\right)^2} & a > 0 \\ 0 & a = 0 \end{cases}$$

Again this also avoids $|x|$ being computed as zero if x_r^2 and x_i^2 both underflow.

Another example where care is needed in complex arithmetic is complex division

$$\frac{x}{y} = \frac{x_r + ix_i}{y_r + iy_i} = \frac{(x_r + ix_i)(y_r - iy_i)}{y_r^2 + y_i^2}$$

Again, some scaling is required to avoid overflow and underflow. See for example [Smit62], [Stew85] and [Prie04]. Algol 60 procedures for the complex operations of modulus, division and square root are given in [Mart68] and the NAG Library Chapter, A02, for complex arithmetic has routines based upon those Algol procedures, see for example [NAGb05]. A

[4] OpenOffice.org version 2.0 now produces the correct result for the data (1.0E200,1.0E200), but underflows for (0,1.0E-200,2.0E-200)

careful C function is given in the Priest reference above. Occasionally, some aspect of complex floating point arithmetic is incorrectly implemented, see for example [Blac97], Section 7.

Another example, similar to the previous examples, requiring care to avoid overflow and damaging underflow is that of real plane rotations where we need to compute $c = \cos\theta$ and $s = \sin\theta$ such that

$$c = \frac{x}{z},\ s = \frac{y}{z},\ \text{where } z = \sqrt{x^2 + y^2}$$

or alternatively

$$c = \frac{-x}{z},\ s = \frac{-y}{z}$$

Another convenient way to express the two choices is as

$$c = \frac{\pm 1}{\sqrt{1+t^2}},\ s = ct,\ \text{where } t \equiv \tan\theta = \frac{x}{y} \tag{4}$$

If G is the *plane rotation matrix*

$$G = \begin{pmatrix} c & s \\ -s & c \end{pmatrix}$$

then, with the choices of (4),

$$G \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} \pm z \\ 0 \end{pmatrix}$$

When used in this way for the introduction of zeros the rotation is generally termed a Givens plane rotation [Give54]; [Golu96]. Givens himself certainly took care in the computation of c and s. To see an extreme case of the detailed consideration necessary to implement a seemingly simple algorithm, but to be efficient, to preserve as much accuracy as possible throughout the range of floating point numbers, and to avoid overflow and damaging underflow see [Bind02], where the computation of the Givens plane rotation is fully discussed.

Sometimes computed solutions are in some sense reasonable, but may not be what the user was expecting. In the next example, the computed solution is close to the exact solution, but does not meet a constraint that the user might have expected the solution to meet.

2.3.5 Example 3.5 (Sample mean)[High98]

Using three figure floating point decimal arithmetic:

$$(5.01 + 5.03)/2 \Rightarrow 10.0/2 \Rightarrow 5.00$$

and we see that the computed value is outside the range of the data, although it is not inaccurate.

Whether or not such a result matters depends upon the application, but is an issue that needs to be considered when implementing numerical algorithms. For instance if

$$y = \cos x$$

then we probably expect the property $|y| \leq 1$ to be preserved computationally, so that a value $|y| > 1$ is never returned. For a monotonic function we may expect monotonicity to be preserved computationally.

In the next section we look at ideas that help our understanding of what constitutes a quality solution.

2.4 Condition, Stability and Error Analysis

2.4.1 Condition

Firstly we look at the condition of a problem. The condition of a problem is concerned with the sensitivity of the problem to perturbations in the data. A problem is ill-conditioned if small changes in the data cause relatively large changes in the solution. Otherwise a problem is well-conditioned. Note that condition is concerned with the sensitivity of the problem, and is independent of the method we use to solve the problem. We now give some examples to illustrate somewhat ill-conditioned problems.

2.4.1.1 Example 4.1 (Cubic equation)

Consider the cubic equation

$$x^3 - 21x^2 + 120x - 100 = 0$$

whose exact roots are $x_1 = 1$, $x_2 = x_3 = 10$. *If we perturb the coefficient of* x^3 *to give*

$$0.99x^3 - 21x^2 + 120x - 100 = 0$$

the roots become $x_1 \approx 1.000$,$x_2 \approx 11.17$, $x_3 \approx 9.041$, *so that the changes in the two roots* x_2 *and* x_3 *are significantly greater than the change in the coefficient. On the other hand, the roots of the perturbed cubic equation*

$$1.01x^3 - 21x^2 + 120x - 100 = 0$$

are $x_1 \approx 1.000$,$x_2, x_3 \approx 9.896 \pm 1.044i$,*and this time the double root has become a complex conjugate pair with a significant imaginary part.*

We can see that the roots x_2 *and* x_3 *are ill-conditionsed. Note that we cannot deduce anything about the condition of* x_1 *just from this data. The three cubic polynomials are plotted in Figure 3.*

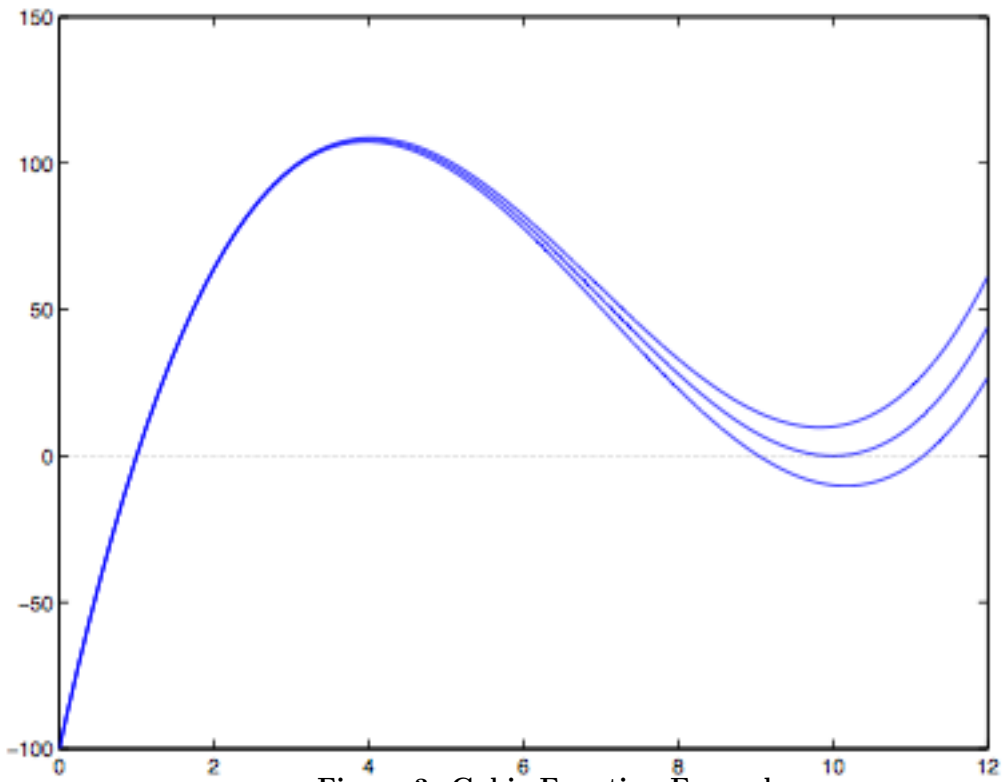

Figure 3: Cubic Equation Example

2.4.1.2 Example 4.2 (Eigenvalue problem)

The matrix

$$A = \begin{pmatrix} 10 & 100 & 0 & 0 \\ 0 & 10 & 100 & 0 \\ 0 & 0 & 10 & 100 \\ 0 & 0 & 0 & 10 \end{pmatrix}$$

has eigenvalues $\lambda_1 = \lambda_2 = \lambda_3 = \lambda_4 = 10$, *whereas the slightly perturbed matrix*

$$B = \begin{pmatrix} 10 & 100 & 0 & 0 \\ 0 & 10 & 100 & 0 \\ 0 & 0 & 10 & 100 \\ 10^{-6} & 0 & 0 & 10 \end{pmatrix}$$

has eigenvalues $\lambda_1 = 11, \lambda_2, \lambda_3 = 10 \pm i$, $\lambda_4 = 9$.

2.4.1.3 Example 4.3 (Integral)

$$I = \int_{-10}^{10} (ae^x - be^{-x})\, dx$$

When $a = b = 1$, $I = 0$, *but when* $a = 1$, $b+1.01$, $I \approx -220$. *The function* $f(x) = xe^x - be^{-x}$, *when* $a = b = 1$ *is plotted in Figure 4. Notice that the vertical scale has a scale factor of* 10^4, *so that a small change in function can make a large change in the area under the curve.*

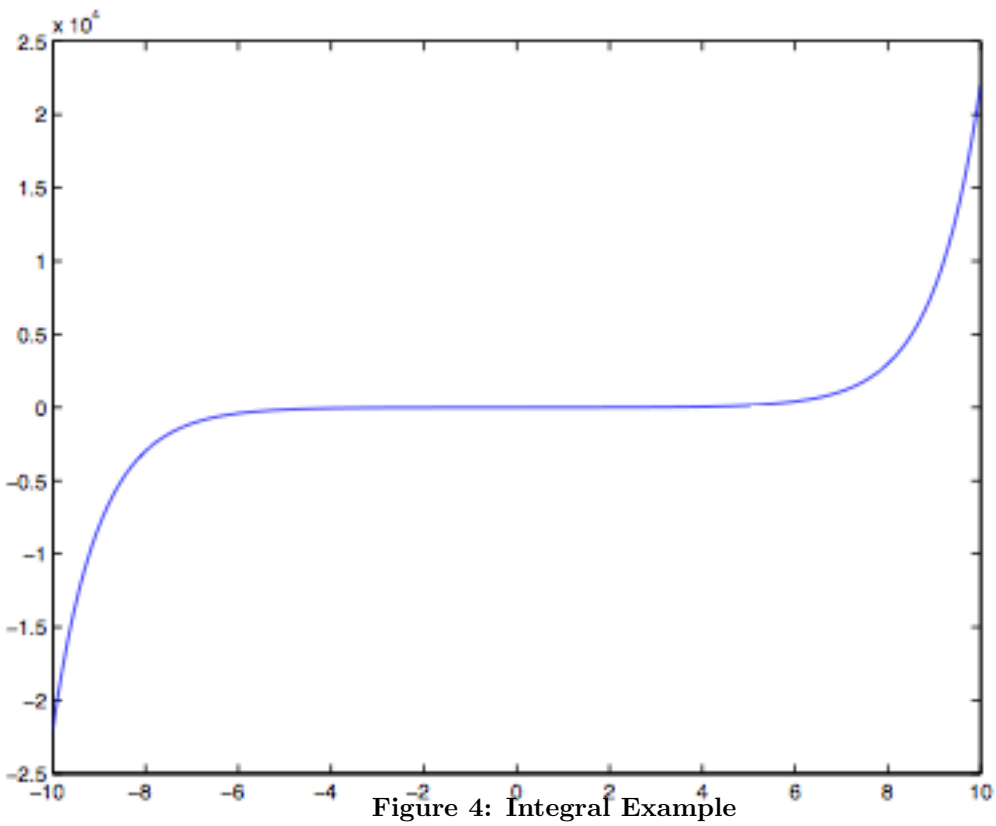

Figure 4: Integral Example

2.4.1.4 Example 4.4 (Linear Equations)

The equations $Ax = b$ *given by*

$$\begin{pmatrix} 99 & 98 \\ 100 & 99 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 197 \\ 199 \end{pmatrix} \tag{5}$$

have the solution $x_1 = x + 2 = 1$, *but the equations*

$$\begin{pmatrix} 98.99 & 98 \\ 100 & 99 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 197 \\ 199 \end{pmatrix}$$

have the solution $x_1 = 100$, $x_2 = -99$. The two straight lines represented by (5) are plotted in Figure 5, but to the granularity of the graph we cannot tell the two lines apart.

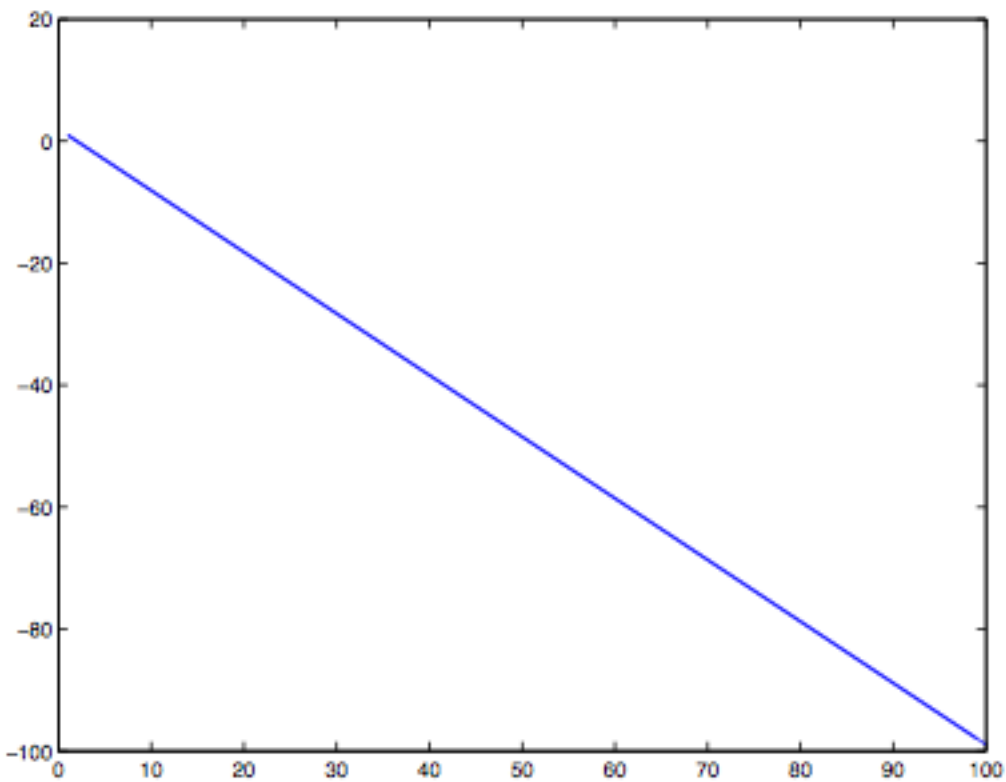

Figure 5: Linear Equations Example

To be able to decide whether or not a problem is ill-conditioned it is clearly desirable to have some measure of the condition of a problem. We show two simple cases where we can obtain such a measure, and quote the result for a third example. For the first case we derive the condition number for the evaluation of a function of one variable [[High02], Section 1.6].

Let $y = f(x)$ with f twice differentiable and $f(x) \neq 0$. Also let $\hat{y} = f(x + \epsilon)$. Then, using the mean value theorem

$$\begin{aligned} \hat{y} - y &= f(x+\epsilon) - f(x) \\ &= f'(x)\epsilon + \frac{f''(x+\theta\epsilon)}{2!}\epsilon^2, \ \theta \in (0,1) \end{aligned}$$

giving

$$\frac{\hat{y} - y}{y} = \left(\frac{x f'(x)}{f(x)}\right)\frac{\epsilon}{x} + O(\epsilon^2)$$

The quantity

$$\kappa(x) = \left| \frac{xf'(x)}{f(x)} \right|$$

is called the *condition number* of f since

$$\left| \frac{\hat{y} - y}{y} \right| \approx \kappa(x) \left| \frac{\epsilon}{x} \right|$$

Thus if $\kappa(x)$ is large the problem is ill-conditioned, that is small perturbations in x can induce large perturbations in the solution y.

2.4.1.5 Example 4.5

Let $y = f(x) = \cos x$. Then we see that

$$\kappa(x) = |x \tan x|$$

and, as we might expect, $\cos x$ is most sensitive close to asymptotes of $\tan x$, such as x close to $\pi/2$.[5] *If we take $x = 1.57$ and $\epsilon = -0.01$ then we find that*

$$\kappa(x) \left| \frac{\epsilon}{x} \right| \approx 12.5577$$

which is a very good estimate of $|(\hat{y} - y)/y| = 12.55739\ldots$.

For the second example we consider the condition number of a system of linear equations $Ax = b$. If we let $\hat{x}$ be the solution of the perturbed equations

$$(A + E)\hat{x} = b$$

then

$$A(\hat{x} - x) = -E\hat{x}, \text{so that } \hat{x} - x = -A^{-1}E\hat{x}$$

giving

$$\frac{\|\hat{x} - x\|}{\|\hat{x}\|} \leq \|A^{-1}\| \cdot \|E\| = (\|A\| \cdot \|A^{-1}\|)\frac{\|E\|}{\|A\|} \tag{6}$$

The quantity

$$\kappa(A) = \|A\| \cdot \|A^{-1}\|$$

is called the condition number of A with respect to the solution of the equations $Ax = b$, or the condition number of A with respect to matrix inversion. Since $I = AA^{-1}$, for any norm such that $\|I\| = 1$, we have that $1 \leq \kappa(A)$, with equality possible for the $1, 2$ and ∞ norms. If A is singular then $\kappa(A) = \infty$.

[5] The given condition number is not valid at $x = \pi/2$, since $\cos \pi/2 = 0$

2.4.1.6 Example 4.6 (Condition of matrix)

For the matrix of Example 4.4 we have that

$$A = \begin{pmatrix} 99 & 98 \\ 100 & 99 \end{pmatrix}, \ \|A\|_1 = 199$$

and

$$A^{-1} = \begin{pmatrix} 99 & -98 \\ -100 & 99 \end{pmatrix}, \ \|A^{-1}\|_1 = 199$$

so that

$$\kappa_1(A) = 199^2 \approx 4 \times 10^4$$

Thus we can see that if A is only accurate to about 4 figures, we cannot guarantee any accuracy in the solution.

The term condition number was first introduced by Turing in the context of systems of linear equations [Turi48]. Note that for an orthogonal or unitary matrix Q, $\kappa(Q) = 1$.

As a third illustration we quote results for the sensitivity of the root of a polynomial. Consider

$$f(x) = a_n x^n + a_{n-1}x^{n-1} + \ldots + a_1 x + a_0$$

and let α be a single root of $f(x)$ so that $f(\alpha) = 0$, but $f'(\alpha) \neq 0$. Let $p(x)$ be the perturbed polynomial

$$p(x) = f(x) = \epsilon g(x), \ g(x) = b_n x^n + b_{n-1}x^{n-1} + \ldots + b_1 x + b_0$$

with root $\hat{\alpha} = \alpha + \delta$, so that $p(\hat{\alpha}) = 0$. Then [[Wilk63],Section 7,Chapter 2] shows that

$$|\delta| \approx \left| \frac{\epsilon g(\alpha)}{f'(\alpha)} \right|$$

Wilkinson also shows that if α is a double root then

$$|\delta| \approx \left| \left(-\frac{2\epsilon g(\alpha)}{f''(\alpha)} \right)^{\frac{1}{2}} \right|$$

2.4.1.7 Example 4.7 (Condition of roots of cubic equation)

For the root $\alpha = x_1 = 1$ of the cubic equation of Example 4.1, with $g(x) = x^3$ and $\epsilon = -0.01$, we have

$$f'(x) = 3x^2 - 42x + 120$$

so that

$$|\delta| \approx \left| \frac{-0.01 \times 1^3}{81} \right| \approx 0.0001$$

and hence this root is very well-conditioned with respect to perturbations in the coefficients of x^3. On the other hand, for the double root $\alpha = 10$, we have

$$f''(x) = 6x - 42$$

so that

$$|\delta| \approx \left| \left(\frac{-2 \times -0.01 \times 10^3}{18} \right)^{\frac{1}{2}} \right| \approx 1.054$$

and this time the perturbation of ϵ produces a rather larger perturbation in the root. Because ϵ is not particularly small the estimate of δ is not particularly accurate, but we do get a good warning of the ill-conditioning.

Highman [[High02],Section 25.4] gives a result for the sensitivity of a root of a general nonlinear equation.

Problems can be ill-conditioned simply because they are poorly scaled, often as the result of a poor choice of measurement units. Some algorithms, or implementation of algorithms, are insensitive to scaling or attempt automatic scaling, but in other cases a good choice of scaling can be important to the success of an algorithm. It is also all too easy to turn a badly scaled problem into a genuinely ill-conditioned problem.

2.4.1.8 Example 4.8 (Badly scaled matrix)

If we let A be the matrix

$$A = \begin{pmatrix} 2 \times 10^9 & 10^9 \\ 10^{-9} & 2 \times 10^{-9} \end{pmatrix}$$

then $\kappa_2(A) \approx 1.67 \times 10^{18}$ and so A is ill-conditioned. However we can row scale A as

$$B = DA = \begin{pmatrix} 10^{-9} & 0 \\ 0 & 10^9 \end{pmatrix} \begin{pmatrix} 2 \times 10^9 & 10^9 \\ 10^{-9} & 2 \times 10^{-9} \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

for which $\kappa_2(B) = 3$, so that B is well-conditioned. On the other hand if we perform a plane rotation on A with $c = 0.8$, $s = 0.6$ we get

$$\begin{aligned} C = GA &= \begin{pmatrix} 0.8 & 0.6 \\ -0.6 & 0.8 \end{pmatrix} \begin{pmatrix} 2 \times 10^9 & 10^9 \\ 10^{-9} & 2 \times 10^{-9} \end{pmatrix} \\ &= 2 \begin{pmatrix} 8 \times 10^8 + 3 \times 10^{-10} & 4 \times 10^8 + 6 \times 10^{-10} \\ -6 \times 10^8 + 4 \times 10^{-10} & -3 \times 10^8 + 8 \times 10^{-10} \end{pmatrix} \end{aligned}$$

Since G is orthogonal, $\kappa_2(C) = \kappa_2(A) \approx 1.67 \times 10^{18}$, and so C is of course as ill-conditioned as A, but now scaling cannot recover the situation. To see that C is genuinely ill-conditioned, we note that

$$C \approx 2 \times 10^8 \begin{pmatrix} 8 & 4 \\ -6 & -3 \end{pmatrix}$$

which is singular. In double IEEE arithmetic, this would be the floating point represention of C.

Many of the LAPACK routines perform scaling, or have options to equilibrate the matrix in the case of linear equations [[Ande99], Sections 2.4.1 and 4.4.1], [[High02], Sections 7.3 and 9.8], or to balance in the case of eigenvalue problems [[Ande99], Sections 4.8.1.2 and 4.11.1.2].

2.4.2 Stability

The *stability* of a method for solving a problem is concerned with the sensitivity of the method to (rounding) errors in the solution process. A method that guarantees as accurate a solution as the data warrants is said to be stable, otherwise the method is unstable. To emphasise the point we note that, whereas condition is concerned with the sensitivity of the problem, stability is concerned with the sensitivity of the method of solution.

An example of an unstable method is that of (3) for computing sample variance. We now give two more simple illustrative examples.

2.4.2.1 Example 4.9 (Quadratic equation)

Consider the quadratic equation

$$1.6x^2 - 100.1x + 1.251 = 0$$

Four significant figure arithmetic when using the standard formula

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

gives

$$x_1 = 62.53,\ x_2 = 0.03125$$

If we use the relationship $x_1 x_2 = c/a$ to compute x_2 from x_1 we instead find that

$$x_2 = 0.01251$$

The correct solution is $x_1 = 62.55$,$x_2 = 0.0125$. We can see that in using the standard formula to compute the smaller root we have suffered from cancellation, since $\sqrt{b^2 - 4ac}$ is close to $(-b)$.

Even a simple problem such as computing the roots of a quadratic equation needs great care. A very nice discussion is to be found in [Fors69].

2.4.2.2 Example 4.10 (Recurrence relation)

Consider the computation of y_n defined by

$$y_n = (1/e) \int_0^1 x^n e^x \, dx \tag{7}$$

where n is a non-negative integer. We note that, since in the interval $[0, 1]$, $(1/e)e^x$ is bounded by unity, it is easy to show that

$$0 \le y_n \le 1/(n+1) \tag{8}$$

Integrating (7) by parts gives

$$y_n = 1 - n y_{n-1},\ y_0 = 1 - 1/e = 0.6321205882856\ldots \tag{9}$$

and we have a seemingly attractive method for computing y_n for a given value of n. The result of using this forward recurrence relation, with IEEE double precision arithmetic, to compute the values of y_i up to y_21 is shown in Table 2. Bearing in mind the bounds of (8), we see that later values are diverging seriously from the correct solution.

A simple analysis shows the reason for the instability. Since y 0 cannot be represented exactly, we cannot avoid starting with a slightly perturbed value, $\hat{y}_0$. So let

$$\hat{y}_0 = y_0 + \epsilon$$

y_0 0.6321	y_1 0.3679	y_2 0.2642	y_3 0.2073	y_4 0.1709	y_5 0.1455	y_6 0.1268	y_7 0.1124
y_8 0.1009	y_9 0.0916	y_{10} 0.0839	y_{11} 0.0774	y_{12} 0.0718	y_{13} 0.0669	y_{14} 0.0627	y_{15} 0.0590
y_{16} 0.0555	y_{17} 0.0572	y_{18} -0.0295	y_{19} 1.5596	y_{20} -30.1924	y_{21} 635.0403		

Table 2: Forward Recurrence for y_n

Then, even if the remaining computations are performed exactly we see that

$$\begin{array}{rcccl} \hat{y}_1 & = & 1 - \hat{y}_0 & = & y_1 - \epsilon \\ \hat{y}_2 & = & 1 - 2\hat{y}_1 & = & y_2 + 2\epsilon \\ \hat{y}_3 & = & 1 - 3\hat{y}_2 & = & y_3 - 6\epsilon \\ \hat{y}_4 & = & 1 - 4\hat{y}_3 & = & y_4 + 24\epsilon \end{array}$$

and a straightforward inductive proof shows that

$$\hat{y}_n = y_n + (-1)^n n!\epsilon$$

When $n = 21$,$n! \approx 5.1091 \times 10^{19}$. We see clearly that this forward recurrence is an unstable method of computing y_n, since the error grows rapidly as we move forward.

The next example illustrates a stable method of computing y_n.

2.4.2.3 Example 4.11 (Stable recurrence)

Rearranging (9) we obtain the backward recurrence

$$y_{n-1} = (1 - y_n)/n$$

Suppose that we have an approximation, $\hat{y}_{n+m}$, to y_{n+m} and we let

$$\hat{y}_{n+m} = y_{n+m} + \epsilon$$

Then, similarly to the result of Example 4.10, we find that

$$\hat{y}_n = y_n + \frac{(-1)^{m-n}\epsilon}{(n+m)(n+m-1)\dots(n+1)}$$

and this time the initial error decays rapidly, rather than grows rapidly as in Example 4.10. If we take an initial guess of $y_{21} = 0$, we see from (8) that

$$|\epsilon| \leq 1/21 \leq 0.05$$

Using this backward recurrence relation, with IEEE double precision arithmetic, gives the value

$$y_0 = 0.63212055882856$$

which is correct to all the figures shown. We see that this backward recurrence is stable.

It should also be said that the integral of (7) can be evaluated stably without difficulty using a good numerical integration (quadrature) formula, since the function $f(x) = (1/e)x^n e^x$ is non-negative and monotonic throughout the interval [0,1].

In the solution of ordinary and partial differential equations one form of instability can arise by replacing a differential equation by a difference equation. We first illustrate the problem by the solution of a simple nonlinear equation.

2.4.2.4 Example 4.12 (Parasitic solution)

The equation

$$e^{-x} = 99x \tag{10}$$

has a solution close to $x = 0.01$. By expanding e^{-x} as a power series we have that

$$e^{-x} = 1 - x + \frac{x^2}{2!} - \frac{x^3}{3!} + \dots \approx 1 - x + \frac{x^2}{2!}$$

and hence an approximate solution of (10) is a root of the quadratic equation

$$x^2 - 200x + 2 = 0$$

which has the two roots $x_1 \approx 0.0100005$, $x_2 \approx 199.99$. The second root clearly has nothing to do with the original equation and is called a **parasitic** *solution.*

In the above example we are unlikely to be fooled by the parasitic solution, since it so clearly does not come close to satisfying (10). But in the solution of ordinary or partial differential equations such bifurcurations, due to truncation error, may not be so obvious.

2.4.2.5 Example 4.13 (Instability for ODE)

For the initial value problem

$$y' = f(x, y),\ y = y_0 \text{ when } x = x_0 \tag{11}$$

the mid-point rule, or leap-frog method, for solving the differential equation is given by

$$y_{r+1} = y_{r-1} + 2hf_r \tag{12}$$

where $h = x_i - x_{i-1}$ *for all* i *and* $f_r = f(x_r, y_r)$*. This method has a truncation error of* $O(h^3)$ *[[Isaa94], Section 1.3, Chapter 8]*[6] *This method requires two starting values, so one starting value must be estimated by some other method. Consider the case where*

$$f(x, y) = \alpha y,\ y_0 = 1,\ x_0 = 0$$

so that the solution of (11) is $y = e^{\alpha x}$*. Figures 6 and 7 show the solution obtained by using (12) when* $h = 0.1$ *for the cases* $\alpha = 2.5$ *and* $\alpha = -2.5$ *respectively. In each case the value of* y_1 *is taken as the correct four figure value,* $y_1 = 1.284$ *when* $\alpha = 2.5$ *and* $y_1 = 0.7788$ *when* $\alpha = -2.5$*. We see that in the first case the numerical solution does a good job in following the exact solution, but in the second case oscillation sets in and the numerical solution diverges from the exact solution.*

The reason for this behaviour in the above example is that (12) has the solution

$$y_r = A\left(\alpha h + (1 + \alpha^2 h^2)^{\frac{1}{2}}\right)^r + B\left(\alpha h - (1 + \alpha^2 h^2)^{\frac{1}{2}}\right)^r \tag{13}$$

where A and B are constants that depend on the intial conditions. With the initial conditions $y_0 = 1$, $x_0 = 0$ and $y_1 = e^h$, $x_1 = h$ we find that $A = 1 + O(h^3)$, $B = O(h^3)$. We can see that the first term in (13) approximates the exact solution, but the second term is a parasitic solution. When $\alpha > 0$ the exact solution increases and the parasitic solution decays, and so is harmless, but when $\alpha < 0$ the exact solution decays and the parasitic solution grows as illustrated in Figure 7. An entertaining discussion, in the context of the Milne-Simpson method, of the above phenomenon is given in Acton [[Acto70], Chapter 5], a book full of good advice and insight. A more recent book by Action in the same vein is [Acto96].

[6] Here it is called the centered method. It is an example of a Nyström method.

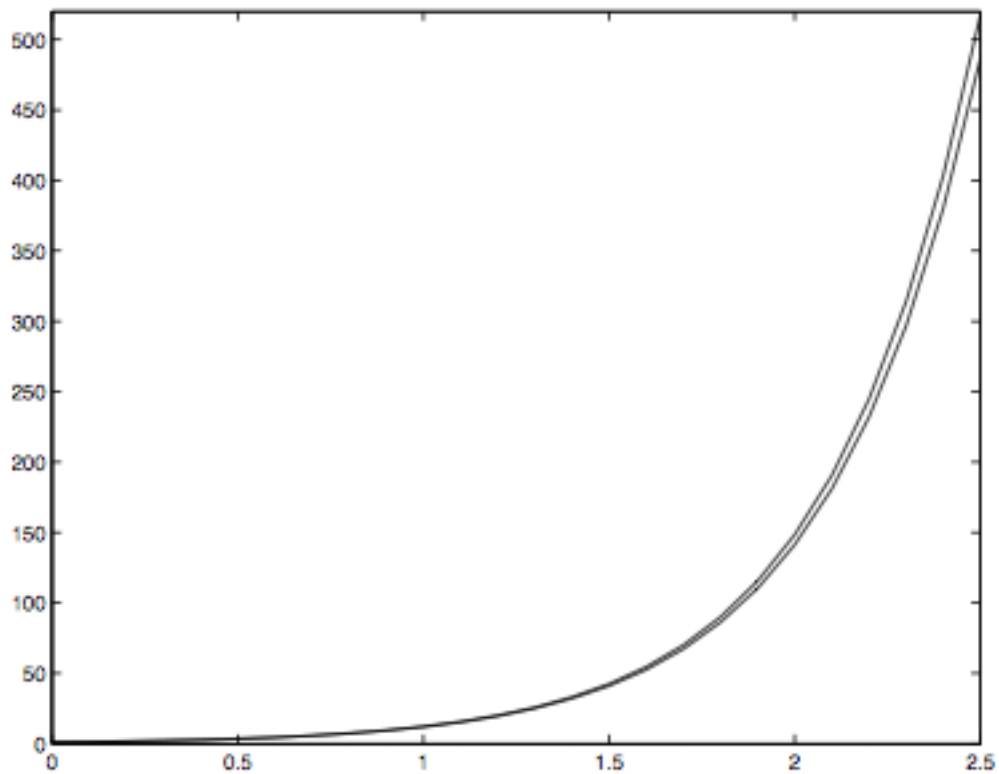

Figure 6: Stable ODE Example

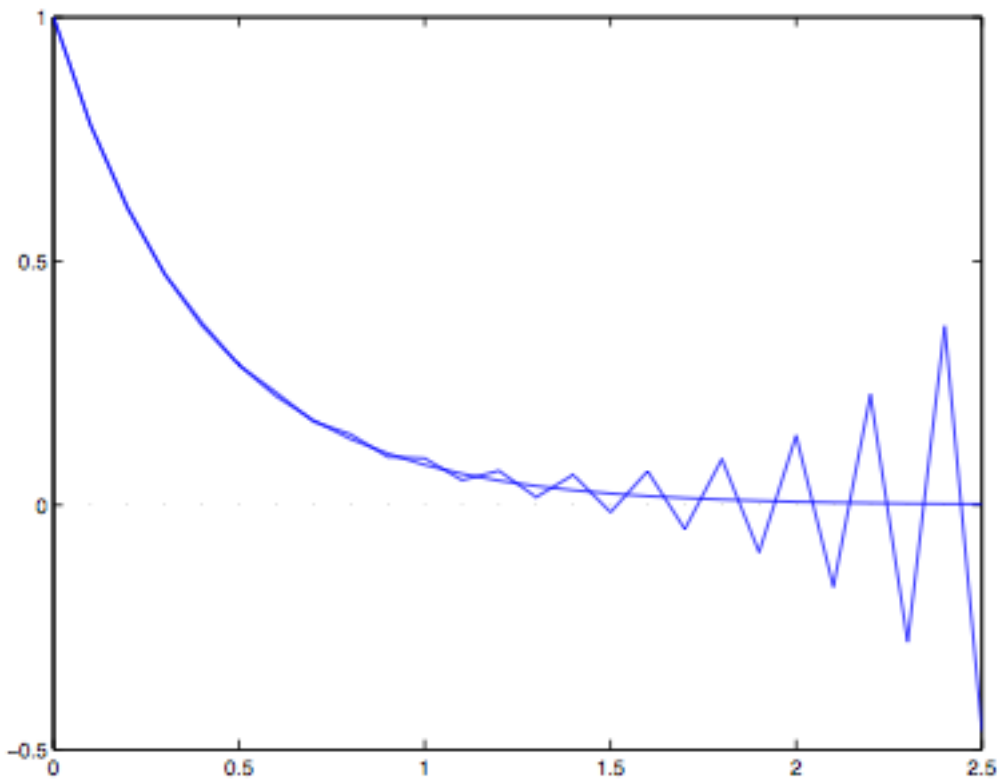

Figure 7: Unstable ODE Example

2.4.3 Error Analysis

Error analysis is concerned with analysing the cumulative effects of errors. Usually these errors will be rounding or truncation errors. For example, if the polynomial

$$p(x) = p_0 + p_1 x + p_2 x^2 + \cdots + p_n x^n$$

is evaluated at some point $x = \alpha$ using Horner's scheme (nested multiplication) as

$$p(\alpha) = p_0 + \alpha(p_1 + \cdots + \alpha(p_{n-2} + \alpha(p_{n-1} + \alpha p_n)) \ldots)$$

we might ask under what conditions, if any, on the coefficients $p_0, p_1, \ldots, p_n$ and α, the solution will, in some sense, be reasonable? To answer the question we need to perform an error analysis.

Error analysis is concerned with establishing whether or not an algorithm is stable for the problem in hand. A *forward error analysis* is concerned with how close the computed solution is to the exact solution. A *backward error analysis* is concerned with how well the computed

solution satisfies the problem to be solved. On first acquaintance, that a backward error analysis, as opposed to a forward error analysis, should be of interest often comes as a surprise. The next example illustrates the distinction between backward and forward errors.

2.4.3.1 Example 4.14 (Linear equations)

Let

$$A = \begin{pmatrix} 99 & 98 \\ 100 & 99 \end{pmatrix} \text{ and } b = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Then the exact solution of the equations $Ax = b$ is given by

$$x = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

Also let $\hat{x}$ be an approximate solution to the equations and define the **residual** *vector r as*

$$r = b - A\hat{x} \tag{14}$$

Of course, for the exact solution $r = 0$ and we might hope that for a solution close to x, r should be small. Consider the approximate solution

$$\hat{x} = \begin{pmatrix} 2.97 \\ -2.99 \end{pmatrix}, \text{ for which } \hat{x} - x = \begin{pmatrix} 1.97 \\ -1.99 \end{pmatrix}$$

and so $\hat{x}$ looks to be a rather poor solution. But for this solution we have that

$$r = \begin{pmatrix} -0.01 \\ 0.01 \end{pmatrix}$$

and we have almost solved the original problem. On the other hand the approximate solution

$$\hat{x} = \begin{pmatrix} 1.01 \\ -0.99 \end{pmatrix}, \text{ for which } \hat{x} - x = \begin{pmatrix} 0.01 \\ 0.01 \end{pmatrix}$$

gives

$$r = \begin{pmatrix} -1.97 \\ -1.97 \end{pmatrix}$$

and, although $\hat{x}$ is close to x, it does not solve a problem close to the original problem.

Once we have computed the solution to a system of linear equations $Ax = b$ we can, of course, readily compute the residual of (14). If we can find a matrix E such that

$$E\hat{x} = r \tag{15}$$

then

$$(A + E)\hat{x} = b$$

and we thus have a measure of the perturbation in A required to make $\hat{x}$ an exact solution. A particular E that satisfies (15) is given by

$$E = \frac{r\hat{x}^T}{\hat{x}^T\hat{x}}$$

From this equation we have that

$$\|E\|_2 \leq \frac{\|r\|_2\|\hat{x}\|_2}{\|\hat{x}\|_2^2} = \frac{\|r\|_2}{\|\hat{x}\|_2}$$

Thus, this particular E minimizes $\|E\|_2$. Since $x^T x = \|x\|_F^2$ it also minimizes E in the Frobenious norm. This gives us an *a posteriori* bound on the backward error.

2.4.3.2 Example 4.15 (Perturbation in linear equations)

Consider the equations $Ax = b$ of Example 4.14 and the 'computed' solution

$$\hat{x} = \begin{pmatrix} 2.97 \\ -2.99 \end{pmatrix} \text{ for which } r = \begin{pmatrix} -0.01 \\ 0.01 \end{pmatrix}$$

Then

$$r\hat{x}^T = \begin{pmatrix} -0.0297 & 0.0299 \\ 0.0297 & -0.0299 \end{pmatrix}, \ \hat{x}^T\hat{x} = 17.761$$

and

$$E \approx \begin{pmatrix} -0.00167 & 0.00168 \\ 0.00167 & -0.00168 \end{pmatrix}$$

Note that $\|E\|_F/\|A\|_F \approx 1.695 \times 10^{-5}$ and so the computed solution corresponds to a small relative perturbation in A.

From (6) we have that

$$\frac{\|\hat{x} - x\|}{\|\hat{x}\|} \leq \kappa(A)\frac{\|E\|}{\|A\|}$$

and so, if we know $\kappa(A)$, then an estimate of the backward error allows us to estimate the forward error.

As a general rule, we can say that approximately:

$$\text{forward error} \ \leq \ \text{condition error} \ \times \ \text{backward error}$$

Although the idea of backward error analysis had been introduced by others, it was James Hardy Wilkinson who really developed the theory and application, and gave us our understanding of error analysis and stability, particularly in the context of numerical linear algebra. See the classic books [Wilk63] and [Wilk65].

A wonderful modern book that continues the Wilkinson tradition is [High02]. The solid foundation for the numerical linear algebra of today relies heavily on the pioneering work of Wilkinson; see also [Wilk71][7]

[7] In Givens 1954 technical report quoted earlier [Give54], which was never published in full and must be one of the most oft quoted technical reports in numerical analysis, as well as the introduction of Givens plane rotations, it describes the use of Sturm sequences for computing eigenvalues of tridiagonal matrices, and contains probably the first explicit backward error analysis. Wilkinson, who so successfully developed and expounded the theory and analysis of rounding errors, regarded the *a priori* error analysis of Givens as "one of the landmarks in the history of the subject" [[Wilk65], Additional notes to Chapter 5]

Wilkinson recognised that error analysis could be tedious and often required great care, but was nevertheless essential to our understanding of the stability of algorithms.

> "The clear identification of the factors determining the stability of an algorithm soon led to the development of better algorithms. The proper understanding of inverse iteration for eigenvectors and the development of the QR algorithm by Francis are the crowning achievements of this line of research.
>
> For me, then, the primary purpose of the rounding error analysis was insight. [Wilk86], p. 197.

As a second example to illustrate forward and backward errors, we consider the quadratic equation of Example 4.9.

2.4.3.3 Example 4.16 (Errors in quadratic equation)

For the quadratic equation of Example 4.9 we saw that the standard formula gave the roots $x_1 = 62.53$, $x_2 = 0.03125$. *Since the correct solution is* $x_1 = 62.55$, $x_2 = 0.0125$ *the second root has a large forward error. If we form the quadratic* $q(x) = 1.6(xx_1)(xx_2)$, *rounding the answer to four significant figures, we get*

$$q(x) = 1.6x^2 - 100.1x + 3.127$$

and the constant term differs significantly from the original value of 1.251, so that there is also a large backward error. The standard method is neither forward nor backward stable. On the other hand, for the computed roots $x_1 = 62.53$, $x_2 = 0.01251$ *we get*

$$q(x) = 1.6x^2 - 100.1x + 1.252$$

so this time we have both forward and backward stability.

An example of a computation that is forward stable, but not backward stable is that of computing the coefficients of the polynomial

$$p(x) = (x - x_1)(x - x_2) \cdots (x - x_n), \ x_i > 0$$

In this case, since the x_i are all of the same sign, no cancellation occurs in computing the coefficients of $p(x)$ and the computed coefficients will be close to the exact coefficients; thus we have small forward errors. On the other hand, as Example 4.1 illustrates, the roots of polynomials can be sensitive to perturbations in the coefficients and so the roots of the computed polynomial could differ significantly from $x_1, x_2, \ldots, x_n$.

2.4.3.4 Example 4.17 (Ill-conditioned polynomial)

The polynomial whose roots are $x_i = i$, $i = 1, 2, \ldots, 20$, *is*

$$p(x) = x^{20} - 210x^{19} + \ldots + 20!$$

Suppose that the coefficient of x^{19} is computed as $-(210 + 2^{-23})$; then we find that x^{16}, $x^{17} \approx 16.73 \pm 2.813i$. Thus a small error in computing a coefficient produced a polynomial with significantly different roots from those of the exact polynomial. This polynomial is discussed in Wilkinson [[Wilk63], Chapter 2, Section 9] and [Wilk84]. See also Wilkinson [[Wilk85], Section 2].

2.5 Floating Point Error Analysis

Floating point error analysis is concerned with the analysis of errors in the presence of floating point arithmetic. It is based on the relative errors that result from each basic operation. We give just a brief introduction to floating point error analysis in order to illustrate the ideas.

Let x be a real number; then we use the notation $\mathrm{fl}(x)$ to represent the floating point value of x. The fundamental assumption is that

$$fl(x) = x(1+\epsilon), \ |\epsilon| \leq u \tag{16}$$

where u is the unit roundoff of (1). Of course,

$$\frac{fl(x) - x}{x} = \epsilon$$

A useful alternative is

$$fl(x) = \frac{x}{1+\delta}, \ |\delta| \leq u, \text{ so that } \frac{fl(x) - x}{fl(x)} = \delta \tag{17}$$

2.5.1 Example 5.1 (Floating point numbers)

Consider four figure decimal arithmetic with

$$u = \frac{1}{2} \times 10^{-3} = 5 \times 10^{-4}$$

If $x = \sqrt{2} = 1.414213\ldots$ then $fl(x) = 1.414$ and

$$|\epsilon| = \left| \frac{fl(x) - x}{x} \right| \approx 1.5 \times 10^{-4}$$

If $x = 1.000499\ldots$ then $fl(x) = 1.000$ and

$$|\epsilon| = \left| \frac{fl(x) - x}{x} \right| \approx 5 \times 10^{-4} = u$$

If $x = 1000.4999\ldots$ then fl(x)=1000 and again

$$|\epsilon| = \left| \frac{fl(x) - x}{x} \right| \approx 5 \times 10^{-4} = u$$

Bearing in mind (16), if x and y are floating point numbers, then the standard model of floating point arithmetic, introduced by Wilkinson [Wilk60], is given by

$$fl(x \otimes y) = (x \otimes y)(1+\epsilon),\ |\epsilon| \leq u$$

$$\text{where } \otimes \equiv +, -, \times, \div$$

It is assumed, of course, that $x \otimes y$ produces a value that is in the range of representable floating point numbers. Comparable to (17), a useful alternative is

$$fl(x \otimes y) = \frac{x \otimes y}{1+\delta},\ |\delta| \leq u$$

When we consider a sequence of floating point operations we frequently obtain products of error terms of the form

$$(1+\epsilon) = (1+\epsilon_1)(1+\epsilon_2)\ldots(1+\epsilon_r)$$

so that

$$(1-u)^r \leq 1+\epsilon \leq (1+u)^r$$

If we ignore second order terms then we have the reasonable assumption that[8]

$$|\epsilon| \leq ru \tag{19}$$

We now give three illustrative examples. In all three examples the x_i are assumed to be floating point numbers, that is, they are values that are already represented in the computer. This is, of course, a natural assumption to make when we are analysing the errors in a computation.

2.5.2 Example 5.2 (Product of values)

Let $x = x_0x_1\ldots x_n$ and $\tilde{x} = fl(x)$. Thus we have n products to form, each one introducing an error bounded by u. Hence from (18) we get

$$\tilde{x} = x_0x_1(1+\epsilon_1)x_2(1+\epsilon_2)\ldots x_n(1+\epsilon_n),\ |\epsilon_i| \leq u \tag{20}$$

and from (19) we see that

$$\tilde{x} = x(1+\epsilon),\ |\epsilon| \leq nu \tag{21}$$

where

$$1+\epsilon = (1+\epsilon_1)(1+\epsilon_2)\ldots(1+\epsilon_n)$$

We can see from (21) that this computation is forward stable, because the result is close to the exact result, and from (20) the computation is also backward stable, because the result is exact for a slightly perturbed problem; that is the result is exact for the data x_0,$x_1(1+\epsilon_1)$,$x_2(1+\epsilon_2)$,...,$x_n(1+\epsilon_n)$.

[8]Those who are uncomfortable with the approximation may prefer to replace the bound $|\epsilon| \leq ru$ with one of the form $|\epsilon| \leq \gamma_r$, where $\gamma_r = (ru)/(1-ru)$ and $ru < 1$ is assumed. See [High02], Lemma 3.1

2.5.3 Example 5.3 (Sum of values)

Let $s = x_1 + x_2 + \ldots + x_n$ *and* $\tilde{s} = fl(s)$. *By considering*

$$s_r = fl(s_{r-1} + x_r),\ s_1 = x_1$$

it is straightforward to show that

$$\begin{aligned} \tilde{s} &= x_1(1+\epsilon_1) + x_2(1+\epsilon_1) + x_3(1+\epsilon_2) + \ldots + x_n(1+\epsilon n - 1) \\ &= x + (x_1\epsilon_1 + x_2\epsilon_1 + x_3\epsilon_2 + \ldots + x_n\epsilon_{n-1}),\ |\epsilon_r| \leq (n-r+1)u \end{aligned}$$

Here we see that summation is backward stable, but is not necessarily forward stable. Example 3.1 gives a case where summation is not forward stable, but notice that the computed solution is the exact solution of the slightly perturbed problem

$$1.000 + 1.000 \times 10^4 - 1.0001 \times 10^4 = 0$$

which illustrates the backward stability.

Note that if the x_i all have the same sign, then summation is forward stable because

$$|\tilde{s} - s| \leq (|x_1| + |x_2| + \ldots + |x_n|)nu = |s|nu$$

so that

$$\frac{|\tilde{s} - s|}{|s|} \leq nu,\ s \neq 0$$

2.5.4 Example 5.4 (Difference of two squares)

Consider the computation

$$z = x^2 - y^2 \tag{22}$$

We can, of course, also express z *as*

$$z = (x+y)(x-y) \tag{23}$$

If we compute z *from (22) we find that*

$$\begin{aligned} \tilde{z} &= fl(z^2 - y^2) = x^2(1+\epsilon_1) = y^2(1 = \epsilon_2) \\ &= z + (x^2\epsilon_1 - y^2\epsilon_2),\ \epsilon_1, \epsilon_2 \leq 2u \end{aligned}$$

and so this is backward stable, but not forward stable. On the other hand, if we compute z *from (23)*

$$\begin{aligned} \hat{z} &= fl((x+y)(x-y)) = (x+y)(x-y)(1+\epsilon) \\ &= z(1+\epsilon),\ \epsilon \leq 3u \end{aligned}$$

and so this is both backward and forward stable. As an example, if we take

$$x = 543.2,\ y = 543.1, \quad \text{so that } z = 108.63$$

and use four significant figure arithmetic we find that

$$\tilde{z} = 100, \ but \hat{z} = 108.6$$

Clearly $\tilde{z}$ has suffered from cancallation, but $\hat{z}$ has not.

We now quote some results, without proof, of solving higher level linear algebra problems to illustrate the sort of results that are possible. Principally we consider the solution of the n linear equations

$$Ax = b \tag{24}$$

by Gaussian elimination and we assume that the reader is familiar with Gaussian elimination. The kth step of Gaussian elimination can be expressed as

$$A_k = M_k P_k A_{k-1} Q_k, \ A_0 = A \tag{25}$$

where P_k and Q_k are permutation matrices, one or both of which may be the unit matrix, chosen to implement whatever pivoting strategy is used and M_k is the multiplier matrix chosen to eliminate the elements below the diagonal of the kth column of A_{k1}. This results in the factorization

$$A = PLUQ$$

where P and Q are permutation matrices, L is a unit lower triangular matrix and U is upper triangular. To simplify analysis it is usual to assume that, with hindsight, A has already been permuted so that we can work with $A \Leftarrow P^T A Q^T$. In this case (25) becomes

$$A_k = M_k A_{k-1}, \ A_0 = A$$

and M_k and A_{k-1} have the form

$$M_k = \begin{pmatrix} I & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -m_k & I \end{pmatrix}, \ A_{k-1} = \begin{pmatrix} U_{k-1} & u_{k-1} & X_{k-1} \\ 0 & \alpha_{k-1} & b_{k-1}^T \\ 0 & a_{k-1} & \hat{A}_{k-1} \end{pmatrix}$$

m_k is chosen to eliminate a_{k-1}, so that

$$a_{k-1} - \alpha_{k-1} m_k = 0, \quad \text{giving } m_k = a_{k-1}/\alpha_{k-1}$$

$\hat{A}_{k-1}$ is updated as

$$\tilde{A}_k = \hat{A}_{k-1} - m_k b_{k-1}^T \equiv \begin{pmatrix} \alpha_k & b_k^T \\ a_k & \hat{A}_k \end{pmatrix}$$

and

$$A = LU, \quad \text{where } L = M_1^{-1} M_2^{-1} \ldots M_{n-1}^{-1}, \quad \text{and } U = A_{n-1}$$

Since

$$M_k^{-1} = \begin{pmatrix} I & 0 & 0 \\ 0 & 1 & 0 \\ 0 & m_k & I \end{pmatrix}$$

we have that

$$L = \begin{pmatrix} 1 & 0 & \cdots & 0 & 0 \\ m_{21} & 1 & \cdots & 0 & 0 \\ m_{31} & m_{32} & \cdots & 0 & 0 \\ \vdots & \vdots & & \vdots & \vdots \\ m_{n-1,1} & m_{n-1,2} & \cdots & 1 & 0 \\ m_{n1} & m_{n2} & \cdots & m_{n,n-1} & 1 \end{pmatrix}$$

It can be shown that the computed factors $\tilde{L}$ and $\tilde{U}$ satisfy

$$\tilde{L}\tilde{U} = A + F$$

where various bounds on F are possible; for example, for the 1, ∞ or F norms

$$\|F\| \leq 3ngu\|A\|, \;\; g = \frac{\max\|\tilde{A}_k\|}{\|A\|}$$

g is called the *growth factor*. Similarly it can be shown that the computed solution of (24), $\tilde{x}$, satisfies

$$(A + E)\hat{x} = b$$

where a typical bound is

$$\|E\| \leq 3n^2gu\|A\|$$

We can see that this bound is satisfactory unless g is large, so it is important to choose P or Q, or both, in order to control the size of g. This is essentially the classic result of Wilkinson[Wilk61] and Wilkinson [[Wilk63], Section 25], where the ∞-norm is used and the use of partial pivoting is assumed; see also Higham [[High02], Theorem 9.5].

The next example gives a simple demonstration of the need for pivoting.

2.5.5 Example 5.5 (The need for pivoting)

Consider the matrix

$$A = \begin{pmatrix} 0.001 & 12 \\ 10 & -10 \end{pmatrix}$$

and the use of four significant figure arithmetic. Since this is just a two by two matrix we have that $M_1^{-1} = L$ *and* $M_1 A = U$. *Denoting the computed matrix* X *by* $\tilde{X}$, *we find that*

$$L = \tilde{L} = \begin{pmatrix} 1 & 0 \\ 10000 & 1 \end{pmatrix}, \; U = \begin{pmatrix} 0.001 & 12 \\ 0 & -120010 \end{pmatrix} \text{ and } \tilde{U} = \begin{pmatrix} 0.001 & 12 \\ 0 & -120000 \end{pmatrix}$$

which gives

$$U = \tilde{U} = \begin{pmatrix} 0 & 0 \\ 0 & 10 \end{pmatrix}$$

and

$$F = \tilde{L}\tilde{U} = A - \begin{pmatrix} 0 & 0 \\ 0 & 10 \end{pmatrix} = U - \tilde{U}$$

Thus whilst $\|F\|$ is small relative to $\|U\|$, it corresponds to a large relative perturbation in $\|A\|$. On the other hand if we permute the two rows of A to give

$$\overline{A} = \begin{pmatrix} 10 & -10 \\ 0.001 & 12 \end{pmatrix}$$

we have that

$$L = \tilde{L} = \begin{pmatrix} 1 & 0 \\ 0.0001 & 1 \end{pmatrix}, \; U = \begin{pmatrix} 10 & -10 \\ 0 & 12.001 \end{pmatrix} \text{ and } \tilde{U} = \begin{pmatrix} 10 & -10 \\ 0 & 12.00 \end{pmatrix}$$

which gives

$$U - \tilde{U} = \begin{pmatrix} 0 & 0 \\ 0 & -0.001 \end{pmatrix}$$

and

$$F = \tilde{L}\tilde{U} - A = \begin{pmatrix} 0 & 0 \\ 0 & -0.001 \end{pmatrix} = U - \tilde{U}$$

This time $\|F\|$ is small relative to both $\|U\|$ and $\|A\|$.

If we put $m = \max|\tilde{m}_{ij}|$ then we can show that

$$g \leq (1+m)^{n-1}$$

Partial pivoting ensures that

$$m \leq 1 \text{ and hence } g \leq 2^{n-1}$$

Only very special examples get anywhere near this bound, one example due to Wilkinson being matrices of the form

$$A = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 & 1 \\ -1 & 1 & 0 & \cdots & 0 & 1 \\ -1 & -1 & 1 & \cdots & 0 & 1 \\ \vdots & \vdots & \vdots & & \vdots & \vdots \\ -1 & -1 & -1 & \cdots & 1 & 1 \\ -1 & -1 & -1 & \cdots & -1 & 1 \end{pmatrix} \text{ for which } U = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 & 1 \\ 0 & 1 & 0 & \cdots & 0 & 2 \\ 0 & 0 & 1 & \cdots & 0 & 4 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 1 & 2^{n-2} \\ 0 & 0 & 0 & \cdots & 0 & 2^{n-1} \end{pmatrix}$$

Despite such examples, in practice partial pivoting is the method of choice, but careful software should at least include an option to monitor the growth factor.

There are classes of matrices for which pivoting is not needed to control the growth of g [[High02], Table 9.1]. Perhaps the most important case is that of symmetric positive definite matrices for which it is known a priori that growth cannot occur, and so Gaussian elimination is stable when applied to a system of equations for which the matrix of coefficients is symmetric positive definite[9]

[9] The variant of Gaussian elimination that is usually used in this case is *Cholesky's method*

The choice of pivots is affected by scaling and equilibration, and a poor choice of scaling can lead to a poor choice of pivots. A full discussion on pivoting strategies, equilibration and scaling, as well as sage advice, can be found in [High02].

For methods that use orthogonal transformations we can usually obtain similar error bounds, but without the growth factor, since orthogonal transformations preserve the 2norm and Fnorm. For example, if we use Householder transformations to perform a QR factorization of A for the solution of the least squares problem $\min_x \|b - Ax\|_2$, where A is an m by n, $m \geq n$ matrix of rank n [Golu65], the computed solution $\tilde{x}$ satisfies

$$\min_x \|(b + f) - (A + E)\tilde{x}\|_2$$

where f and E satisfy bounds of the form

$$\|f\|_F \leq c_1 mnu\|b\|_F, \ \|E\|_F \leq c_2 mnu\|A\|_F$$

and c_1 and c_2 are small integer constants [[Laws95], page 90].

Similarly, for the solution of the eigenvalue problem $Ax = \lambda x$, where A is an n by n matrix, using Housholder transformations to reduce A to upper Hessenberg form, followed by the QR algorithm to further reduce the Hessenberg form to upper triangular Schur form, the computed solution satisifies

$$(A + E)\tilde{x} = \tilde{\lambda}\tilde{x}$$

where

$$\|E\|_F \leq p(n)u\|A\|_F$$

and $p(n)$ is a modestly growing function of n [Wilk65]; [Ande99].

We note that the bounds discussed so far are called *normwise* bounds, but in many cases they can be replaced by *componentwise* bounds which bound the absolute values of the individual elements, and so are rather more satisfactory. For instance, if A is a sparse matrix, we would probably prefer not to have to perturb the elements that are structurally zero. As a simple example, consider the triangular equations

$$Tx = b, \ T - n \text{ by } n \text{ triangular}$$

and let $\tilde{x}$ be the solution computed by forward or backward substitution, depending on whether T is lower or upper triangular respectively. Then it can readily be shown that $\tilde{x}$ satisifes

$$(T + E)\tilde{x} = b, \quad \text{with } |e_{ij}| \leq nu|t_{ij}|$$

which is a strong componentwise result showing backward stability [[High02], Theorem 8.5].

Associated with componentwise error bounds are componentwise condition numbers. Once again see [High02] for further details and references.

2.6 Posing the Mathematical Problem

In this short section we merely wish to raise awareness of the need to model a problem correctly, without offering any profound solution.

It can be all too easy to transform a well-conditioned problem into an ill-conditioned problem. For instance, in Example 4.10 we transformed the well-conditioned quadrature problem of finding

$$y_n = (1/e) \int_0^1 x^n e^x \, dx, \; n \geq 0$$

into the ill-conditioned problem of finding y_n from the forward recurrence relation

$$y_n = 1 - n y_{n-1}, \; y_0 = 1 - 1/e$$

As another example, we noted in Section 4.3 that polynomials can be very ill-conditioned. It follows that the eigenvalues of a matrix A should most certainly not be computed via the characteristic equation of A. For example, if A is a symmetric matrix with eigenvalues $\lambda i = i, \; i = 1, 2, \ldots, 20$, then the characteristic equation of A, $\det(A - \lambda A)$, is very ill-conditioned (see Example 4.17).

On the other hand, the eigenvalues of a symmetric matrix are always well-conditioned [[Wilk65], Section 31, Chapter 2].

The above two examples illustrate the dangers in transforming the mathematical problem. Sometimes it can be poor modelling of the physical problem that gives rise to an ill-conditioned mathematical problem, and so we need to think carefully about the whole modelling process.

We cannot blame software for giving us poor solutions if we provide the wrong problem. We can, of course, hope that the software might provide a measure for the condition of the problem, or some measure of the accuracy of the solution to give us warning of a poorly posed problem.

At the end of Section 4.1 we also mentioned the desirability of careful choice of measurement units, in order to help avoid the effects of poor scaling.

2.7 Error Bounds and Software

In this section we give examples of reliable software that return information about the quality of the solution. Firstly we look at the freely available software package LAPACK [Ande99], and then at an example of a commercial software library, the NAG Library [NAGa05]. The author of this report has to declare an interest in both of these software products; he is one of the authors of LAPACK and is currently a software developer employed by NAG Ltd. Naturally, the examples are chosen because of familiarity with the products and belief in them as quality products, but I have nevertheless tried not to introduce bias.

LAPACK stands for Linear Algebra PACKage and is a numerical software package for the solution of dense and banded linear algebra problems aimed at PCs, workstations and high-performance shared memory machines. One of the aims of LAPACK was to make the

software efficient on modern machines, whilst retaining portability, and to this end it makes extensive use of the Basic Linear Algebra Subprograms (BLAS), using block-partitioned algorithms based upon the Level 3 BLAS wherever possible. The BLAS specify the interface for a set of subprograms for common scalar and vector (Level 1), matrix-vector (Level 2) and matrix-matrix operations (Level 3). Their motivation and specification are given in [Laws79], [Dong88a] [Dong90] respectively. Information on block-partitioned algorithms and performance of LAPACK can be found in [[Ande99], Chapter 3]. See also [Golu96], particularly Section 1.3, and [Stew98], Chapter 2, which also has some nice discussion on computation.

LAPACK has routines for the solution of systems of linear equations, linear least squares problems, eigenvalue and singular value problems, including generalized problems, as well as routines for the underlying computational components such as matrix factorizations. In addition, a lot of effort was expended in providing condition and error estimates. Quoting from the first paragraph of Chapter 4 Accuracy and Stability of the LAPACK Users Guide:

> In addition to provide faster routines than previously available. LAPACK provides more comprehensive and better error bounds. Our goal is to provide error bounds for most quantities computed by LAPACK.

In many cases the routines return the bounds directly; in other cases the Users' Guide gives details of error bounds and provides code fragments to compute the bounds.

As an example, routine DGESVX[10] solves a system of linear equations $AX = B$, where B is a matrix of one or more right-hand sides, using Gaussian elimination with partial pivoting. Part of the interface is

```
SUBROUTINE DGESVX(..., RCOND, FERR, BERR, WORK, ..., INFO)
```

where the displayed arguments return the following information:

- `RCOND` - Estimate of reciprocal of condition number, $1/\kappa(A)$
- `FERR(`j`)` - Estimated forward error for X_j
- `BERR(`j`)` - Componentwise relative backward error for X_j (smallest relative change in any element of A and B_j that makes X_j an exact solution)
- `WORK(1)` - Reciprocal of pivot growth factor, $1/g$
- `INFO` - Returns a positive value if the computed triangular factor U is singular or nearly singular.

Thus DGESVX is returning all the information necessary to judge the quality of the computed solution.

The routine returns an estimate of $1/\kappa(A)$, rather than $\kappa(A)$ to avoid overflow when A is singular, or very ill-conditioned. The argument INFO is the LAPACK warning or error flag,

[10] In the LAPACK naming scheme the D stands for double precision, GE for general matrix, SV for solver and X for expert driver

and is present in all the LAPACK user callable routines. It returns zero on successful exit, a negative value if an input argument is incorrectly supplied, for example $n < 0$, and a positive value in the case of failure, or near failure as above. In the above example, INFO returns the value i if $u_{ii} = 0$, in which case no solution is computed since U is exactly singular, but returns the value $n+1$ if $1/\kappa(A) < u$, in which case A is non-singular to working precision. In the latter case a solution is returned, and so INFO $= n+1$ acts as a warning that the solution may have no correct digits. The routine also has the option to equilibrate the matrix A. See the documentation of the routine for further information, either in the Users Guide, or in the source code available from netlib at `http://www.netlib.org/lapack/index.html`.

As a second example from LAPACK, routine DGEEVX solves the eigenproblem $Ax = \lambda x$ for the eigenvalues and eigenvectors, $\lambda_i, x_i, i = 1, 2, \ldots, n$ of the n by n matrix A. Optionally, the matrix can be balanced and the left eigenvectors of A can also be computed. Part of the interface is

```
SUBROUTINE DGEEVX(..., ABNRM, RCONDE, RCONDV, ...)
```

where the displayed arguments return the following information:

- `ABNRM` - Norm of the balanced matrix
- `RCONDE(`i`)` - Reciprocal of the condition number of the ith eigenvalue, s_i
- `RCONDV(`i`)` - Reciprocal of the condition number for the ith eignevector, sep_i

Following a call to DGEEVX, approximate error bounds for the computed eigenvalues and eigen- vectors, say EERRBD(i) and VERRBD(i), such that

$$|\tilde{\lambda_i} - \lambda_i| \leq \texttt{EERRBD}(i)$$

$$\theta(\tilde{\nu}_i, \nu_i) \leq \texttt{VERRBD}(i)$$

where $\theta(\tilde{\nu}_i, \nu_i)$ is the angle between the computed and true eigenvector, may be returned by the following code fragment, taken from the Users' Guide:

```
      EPSMCH = DLAMCH('E')
      DO 10 I = 1, N
        EERRBD(I) = EPSMCH*ABNRM/RCONDE(I)
        VERRBD(I) = EPSMCH*ABNRM/RCONDV(I)
10    CONTINUE
```

These bounds are based upon Table 3, extracted from Table 4.5 of the LAPACK Users Guide, which gives approximate asymptotic error bounds for the nonsymmetric eigenproblem. These bounds

Simple eigenvalue	$\lvert\tilde{\lambda_i} - \lambda_i\rvert \lesssim \lVert E\rVert_2/s_i$
Eigenvector	$\theta(\tilde{\nu}_i, \nu_i) \lesssim \lVert E\rVert_F/\text{sep}_i$

assume that the eigenvalues are simple eigenvalues. In addition if the problem is ill-conditioned, these bounds may only hold for extremely small $\|E\|_2$ and so the Users Guide also provides a table of global error bounds which are not so restrictive on $\|E\|_2$. The tables in the Users Guide include bounds for clusters of eigenvalues and for invariant subspaces, and these bounds can be estimated using DGEESX in place of DGEEVX. For further details see The LAPACK Users Guide [[Ande99], Chapter 4] and for further information see [Golu96], Chapter 7 and [Stew90].

LAPACK is freely available via netlib[11], is included in the NAG Fortran 77 Library and is the basis of the dense linear algebra in the NAG Fortran 90 and C Libraries. Tuned versions of a number of LAPACK routines are included in the NAG Fortran SMP Library. The matrix computations of MATLAB have been based upon LAPACK since Version 6 [Math05]; [High05].

We now take an example from the NAG Fortran Library. Routine D01AJF is a general purpose integrator using an adaptive procedure, based on the QUADPACK routine QAGS [Pies83], which performs the integration

$$I = \int_a^b f(x)\ dx$$

where $[a, b]$ is a finite interval. Part of the interface to D01AJF is

```
SUBROUTINE D01AJF(..., EPSABS, EPSREL, RESULT, ABSERR, ...)
```

- `EPSABS` - The absolute accuracy required
- `EPSREL` - The relative accuracy required
- `RESULT` - The computed approximation to I
- `ABSERR` - An estimate of the absolute error

In normal circumstances `ABSERR` satisfies

$$|I - RESULT| \leq ABSERR \leq \max(EPSABS, EPSREL \times |I|)$$

See the NAG Library documentation [NAGb05] and [Pies83] for further details. QUADPACK is freely available from netlib,[12] and a Fortran 90 version of QAGS is available from the more recent quadature package, CUBPACK [Cool03], which is also available from netlib. Typically the error estimate for a quadrature routine is obtained at the expense of additional computation with a finer interval, or mesh, or the use of a higher order quadrature formula.

As a second example from the NAG Library we consider the solution of an ODE. Routine D02PCF integrates

$$y' = f(t, y), \quad \text{given } y(t_0) = y_0$$

[11] `http://www.netlib.org/lapack/index.html`
[12] `http://www.netlib.org/quadpack`

where y is the n element solution vector and t is the independent variable, using a Runge-Kutta method. Following the use of D02PCF, routine D02PZF may be used to compute global error estimates. Part of the interface to D02PZF is

```
SUBROUTINE D02PZF(RMSERR, ERRMAX, TERRMX, ...)
```

where the displayed arguments return the following information:

- `RMSERR(`i`)` - Approximate root mean square error for y_i
- `ERRMAX` - Maximum approximate true error
- `TERRMX` - First point at which maximum approximate true error occurred

The assessment of the error is determined at the expense of computing a more accurate solution using a higher order method to that used for the original solution.

The NAG D02P routines are based upon the RKSUITE software by [Bran92], which is also available from netlib.[13] See also [Sham92] and [Bran93]. A Fortran 90 version of RKSUITE is also available,[14] see [Bran97].

Many routines in the NAG Library attempt to return information about accuracy. The documentation of the routines includes a section labelled "Accuracy" which, when appropriate, gives further advice or information. For instance, the optimization routines generally quote the optimality conditions that need to be met for the routine to be successful. These routines are cautious, and sometimes return a warning, or error, when it is likely that an optimum point has been found, but not all the optimality conditions have been met. NAG and the authors of the routines feel that this is much the best approach for reliability even if users would sometimes prefer that we were more optimistic!

2.8 Other Approaches

What does one do if the software does not provide suitable estimates for the accuracy of the solution, or the sensitivity of the problem? One approach is to run the problem with perturbed data and compare solutions. Of course, the difficulty with this approach is to know how best to choose perturbations. If a small perturbation does significantly change the solution, then we can be sure that the problem is sensitive, but of course we cannot rely on the converse. If we can have trust that the software implements a stable method, then any sensitivity in the solution is due to the problem, but otherwise we cannot be sure whether it is the method or problem that is sensitive.

To help estimate such sensitivity there exists software that uses stochastic methods to give statistical estimates of backward error, or of sensitivity. One such example, PRECISE, is described in [Chai96], Chapter 8 and provides a module for statistical backward error analysis

[13] tt http://www.netlib.org/ode/rksuite/

[14] `http://www.netlib.org/ode/rksuite/` or `http://www.netlib.org/toms/771`

as well as a module for sensitivity analysis. Another example is CADNA[15]; see for example [Vign93].

Another approach to obtaining bounds on the solution is the use of interval arithmetic, in conjunction with interval analysis [Moor79] [Krei05]. Some problems can be successfully solved using interval arithmetic throughout, but for some problems the bounds obtained would be far too pessimistic; however interval arithmetic can often be applied as an posteriori tool to obtain realistic bounds. We note that there is a nice interval arithmetic toolbox for Matlab, INTLAB, by [Rump99] that is freely available[16]; see also [Harg02]. It should be noted that in general, the aim of interval arithmetic is to return forward error bounds on the solution.

2.8.1 Example 8.1 (Cancellation and interval arithmetic)

As a very simple example consider the computation of s in Example 3.1 using four figure interval arithmetic. Bearing in mind that interval arithmetic works with intervals that are guaranteed to contain the exact solution, we find that

$$\begin{aligned} s = [s1\ s2] &= [1.000\ 1.000] + [1.000 \times 10^4\ 1.000 \times 10^4] - [1.000 \times 10^4\ 1.000 \times 10^4] \\ &= [1.000 \times 10^4\ 1.001 \times 10^4] - [1.000 \times 10^4\ 1.000 \times 10^4] \\ &= [0\ 10] \end{aligned}$$

so whilst the result is somewhat pessimistic, it does give due warning of the cancellation.

Finally we comment that one should not be afraid to exert pressure on software developers to provide features that allow one to estimate the sensitivity of the problem and the accuracy of the solution.

2.9 Summary

We have tried to illustrate the niceties of numerical computation and the detail that needs to be considered when turning a numerical algorithm into reliable, robust numerical software. We have also tried to describe and illustrate the ideas that need to be understood to judge the quality of a numerical solution, especially condition, stability and error analysis, including the distinction between backward and forward errors.

We emphasise that one should most certainly be concerned about the quality of computed solutions, and use trustworthy quality software. We cannot just blithely assume that results returned by software packages are correct.

This is not always easy since scientists wish to concentrate on their science and should not really need to be able to analyse an algorithm to understand whether or not it is a stable method for solving their problem. Hence the emphasis in this report on the desirability of software providing proper measures of the quality of the solution.

[15] At the time of writing, a free academic version is available from `http://www-anp.lip6.fr/cadna/Accueil.php`

[16] `http://www.ti3.tu-harburg.de/english/index.html`

We conclude with a question:

> "You have been solving these damn problems better than I can pose them"
> Sir Edward Bullard, Director NPL, in a remark to Wilkinson in the mid 1950s,
> See [Wilk85],p. 11

Software developers should strive to provide solutions that are at least as good as the data deserves.

Chapter 3

Chapter Overview

Each routine in the Basic Linear Algebra Subroutine set (BLAS) [blas01] has a prefix where:

- C - Complex
- D - Double Precision
- S - Real
- Z - Complex*16

Routines in level 2 and level 3 of BLAS use the prefix for type:

- GE - general
- GB - general band
- SY - symmetric
- HE - hermitian
- TR - triangular
- SB - symmetric band
- HB - hermetian band
- TB - triangular band
- SP - Sum packed
- HP - hermitian packed
- TP - triangular packed

For level 2 and level 3 BLAS options the options argument is CHARACTER*1 and may be passed as character strings. They mean:

- TRANx
  - **N**o transpose
  - **T**ranspose
  - **C**onjugate transpose (X, X^T, X^H)
- UPLO
  - **U**pper triangular
  - **L**ower triangular
- DIAG
  - **N**on-unit triangular
  - **U**nit triangular
- SIDE
  - **L**eft - A or op(A) on the left
  - **R**ight - A or op(A) on the right

For real matrices, TRANSx=T and TRANSx=C have the same meaning. For Hermitian matrices, TRANSx=T is not allowed. For complex symmetric matrices, TRANSx=H is not allowed.

There were 38 BLAS Level 1 routines defined in [Laws79]. They are

- Dot product SDSDOT, DSDOT, DQ-IDOT DQ-ADOT C-UDOT C-CDOT DDOT SDOT
- Constant times a vector plus a vector CAXPY DAXPY SAXPY
- Set up Givens rotation DROTG SROTG
- Apply rotation DROT SROT
- Set up modified Givens rotation DROTMG SROTMG
- Apply modified rotation DROTM SROTM
- Copy x into y CCOPY DCOPY SCOPY
- Sway x and y CSWAP DSWAP SSWAP
- 2-norm (Euclidean length) SCNRM2 DNRM2 SNRM2
- Sum of absolute values SCASUM DASUM SASUM

- Constant times a vector CSSCAL CSCAL DSCAL SSCAL
- Index of maximum element ICAMAX IDAMAX ISAMAX

where

- I Integer
- S Single Precision
- D Double Precision
- C Single Precision Complex
- Q extended precision
- Z COMPLEX*16

Vector arguments are permitted to have a storage spacing between elements. This spacing is specified by an increment parameter. For example, suppose a vector x having components x_i, $i = 1, ldots, N$ is stored in a DOUBLE PRECISION array DX() with increment parameter INCX. If $INCX \geq 0$ then x_i is stored in $DX(1 + (i - 1) * INCX)$. If $INCX < 0$ then x_i is stored in $DX(1 + (N - 1) * |INCX|)$.

Chapter 4

Optimizations

This code was generated using Raymond Toy's Fortran to Lisp conversion program. Thus the code contains whatever optimizations were in the original Fortran.

That said, we quote from Chellappa, et al. [Chel06]:

> ... the task of achieving the highest performance with an implementation usually lies to a great extent with the programmer. For a given problem, he or she has to carefully consider different algorithms and possibly restructure them to adapt to the given platform's memory hierarchy and available parallelism. This is very difficult, time-consuming, and requires interdisciplinary knowledge about algorithms, software optimizations, and the hardware architecture. Further, the tuning process is platform-dependent: an implementation optimized for one computer will not necessarily be the fastest one on another, since performance depends on many microarchitectural features including but not restricted to the details of the memory hierarchy. Consequently, to achieve highest performance, tuning has to be repeated with the release of each new platform. Since the times of a free speedup (due to frequency scaling) are over, this retuning has become mandatory if any performance gains are desired. Needless to say, the problem is not merely an academic one, but one that affects the software industry as a whole.

The Axiom implementation is intended for prototyping work, not for production code. We maintain compatible API calls so prototype work can be easily ported to native optimized libraries if so desired.

An additional motivation is to research the use of Gustafson's [Gust16] unum ideas, which promises to reduce and/or eliminate overflow and underflow issues.

Chapter 5

Algebra Cover Code

5.1 package BLAS1 BlasLevelOne

5.1.1 Axiom Package unit tests

— BlasLevelOne.input —

```
)set break resume
)sys rm -f BlasLevelOne.output
)spool BlasLevelOne.output
)set message test on
)set message auto off
)clear all

--S 1 of 208
t1:Complex DoubleFloat := complex(1.0,0)
--R
--R
--R   (1)  1.
--R                                                    Type: Complex(DoubleFloat)
--E 1

--S 2 of 208
dcabs1(t1)
--R
--R
--R   (2)  1.
--R                                                          Type: DoubleFloat
--E 2

--S 3 of 208
```

```
t2:Complex DoubleFloat := complex(1.0,1.0)
--R
--R
--R   (3)  1. + %i
--R                                                    Type: Complex(DoubleFloat)
--E 3

--S 4 of 208
dcabs1(t2)
--R
--R
--R   (4)  2.
--R                                                             Type: DoubleFloat
--E 4

--S 5 of 208
t3:Complex DoubleFloat := complex(1.0,-1.0)
--R
--R
--R   (5)  1. - %i
--R                                                    Type: Complex(DoubleFloat)
--E 5

--S 6 of 208
dcabs1(t3)
--R
--R
--R   (6)  2.
--R                                                             Type: DoubleFloat
--E 6

--S 7 of 208
t4:Complex DoubleFloat := complex(-1.0,-1.0)
--R
--R
--R   (7)  - 1. - %i
--R                                                    Type: Complex(DoubleFloat)
--E 7

--S 8 of 208
dcabs1(t4)
--R
--R
--R   (8)  2.
--R                                                             Type: DoubleFloat
--E 8

--S 9 of 208
t5:Complex DoubleFloat := complex(-2.0,-2.0)
--R
```

```
--R
--R   (9)  - 2. - 2. %i
--R                                                    Type: Complex(DoubleFloat)
--E 9

--S 10 of 208
dcabs1(t5)
--R
--R
--R   (10)  4.
--R                                                          Type: DoubleFloat
--E 10

)clear all

--S 11 of 208
a:PRIMARR(DFLOAT):=[ [1.0,2.0,3.0,4,0,5,0,6,0] ]
--R
--R   (1)  [1.,2.,3.,4.,0.,5.,0.,6.,0.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 11

--S 12 of 208
dasum(3,a,-1) -- 0.0   neg incx
--R
--R   (2)  0.
--R                                                          Type: DoubleFloat
--E 12

--S 13 of 208
dasum(3,a,0) --  0.0   zero incx
--R
--R   (3)  0.
--R                                                          Type: DoubleFloat
--E 13

--S 14 of 208
dasum(-1,a,1) -- 0.0   neg elements
--R
--R   (4)  0.
--R                                                          Type: DoubleFloat
--E 14

--S 15 of 208
dasum(0,a,1) --  0.0   no elements
--R
--R   (5)  0.
--R                                                          Type: DoubleFloat
--E 15
```

```
--S 16 of 208
dasum(1,a,1) --  1.0   1.0
--R
--R   (6)  1.
--R                                                        Type: DoubleFloat
--E 16

--S 17 of 208
dasum(2,a,1) --  3.0   1.0+2.0
--R
--R   (7)  3.
--R                                                        Type: DoubleFloat
--E 17

--S 18 of 208
dasum(3,a,1) --  6.0   1.0+2.0+3.0
--R
--R   (8)  6.
--R                                                        Type: DoubleFloat
--E 18

--S 19 of 208
dasum(4,a,1) -- 10.0   1.0+2.0+3.0+4.0
--R
--R   (9)  10.
--R                                                        Type: DoubleFloat
--E 19

--S 20 of 208
dasum(5,a,1) -- 15.0   1.0+2.0+3.0+4.0+5.0
--R
--R   (10)  10.
--R                                                        Type: DoubleFloat
--E 20

--S 21 of 208
dasum(6,a,1) -- 21.0   1.0+2.0+3.0+4.0+5.0+6.0
--R
--R   (11)  15.
--R                                                        Type: DoubleFloat
--E 21

--S 22 of 208
dasum(7,a,1) -- 21.0   1.0+2.0+3.0+4.0+5.0+6.0
--R
--R   (12)  15.
--R                                                        Type: DoubleFloat
--E 22

--S 23 of 208
```

```
dasum(1,a,2) --  1.0   1.0
--R
--R   (13)  1.
--R                                                    Type: DoubleFloat
--E 23

--S 24 of 208
dasum(2,a,2) --  4.0   1.0+3.0
--R
--R   (14)  4.
--R                                                    Type: DoubleFloat
--E 24

--S 25 of 208
dasum(3,a,2) --  9.0   1.0+3.0+5.0
--R
--R   (15)  4.
--R                                                    Type: DoubleFloat
--E 25

--S 26 of 208
dasum(4,a,2) --  9.0   1.0+3.0+5.0
--R
--R   (16)  4.
--R                                                    Type: DoubleFloat
--E 26

--S 27 of 208
dasum(1,a,3) --  1.0   1.0
--R
--R   (17)  1.
--R                                                    Type: DoubleFloat
--E 27

--S 28 of 208
dasum(2,a,3) --  5.0   1.0+4.0
--R
--R   (18)  5.
--R                                                    Type: DoubleFloat
--E 28

--S 29 of 208
dasum(3,a,3) --  5.0   1.0+4.0
--R
--R   (19)  5.
--R                                                    Type: DoubleFloat
--E 29

--S 30 of 208
dasum(1,a,4) --  1.0   1.0
```

```
--R
--R   (20)  1.
--R                                                        Type: DoubleFloat
--E 30

--S 31 of 208
dasum(2,a,4) --  6.0   1.0+5.0
--R
--R   (21)  1.
--R                                                        Type: DoubleFloat
--E 31

--S 32 of 208
dasum(3,a,4) --  6.0   1.0+5.0
--R
--R   (22)  1.
--R                                                        Type: DoubleFloat
--E 32

--S 33 of 208
dasum(1,a,5) --  1.0   1.0
--R
--R   (23)  1.
--R                                                        Type: DoubleFloat
--E 33

--S 34 of 208
dasum(2,a,5) --  7.0   1.0+6.0
--R
--R   (24)  6.
--R                                                        Type: DoubleFloat
--E 34

--S 35 of 208
dasum(3,a,5) --  7.0   1.0+6.0
--R
--R   (25)  6.
--R                                                        Type: DoubleFloat
--E 35

--S 36 of 208
dasum(1,a,6) --  1.0   1.0
--R
--R   (26)  1.
--R                                                        Type: DoubleFloat
--E 36

--S 37 of 208
dasum(2,a,6) --  1.0   1.0
--R
```

```
--R   (27)  1.
--R                                                        Type: DoubleFloat
--E 37

--S 38 of 208
dasum(1,a,7) --  1.0   1.0
--R
--R   (28)  1.
--R                                                        Type: DoubleFloat
--E 38

)clear all

--S 39 of 208
a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (1)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 39

--S 40 of 208
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (2)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 40

--S 41 of 208
daxpy(3,2.0,a,1,b,1)
--R
--R
--R   (3)  [3.,6.,9.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 41

--S 42 of 208
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (4)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 42

--S 43 of 208
daxpy(7,2.0,a,1,b,1)
--R
--R
--R   (5)  [3.,6.,9.,12.,15.,18.,21.]
```

```
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 43

--S 44 of 208
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (6)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 44
```

———-

Note that Axiom properly handles array indexes that are out of bounds. The BLAS daxpy routine cannot check this condition.

— **BlasLevelOne.input** —

```
--S 45 of 208
daxpy(8,2.0,a,1,b,1)
--R
--R
--R   (7)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 45

--S 46 of 208
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (8)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 46

--S 47 of 208
daxpy(3,2.0,a,3,b,3)
--R
--R
--R   (9)  [3.,2.,3.,12.,5.,6.,21.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 47

--S 48 of 208
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (10)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 48
```

```
--S 49 of 208
daxpy(4,2.0,a,2,b,2)
--R
--R
--R   (11)  [3.,2.,9.,4.,15.,6.,21.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 49

--S 50 of 208
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (12)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 50

--S 51 of 208
daxpy(5,2.0,a,2,b,2)
--R
--R
--R   (13)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 51

--S 52 of 208
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (14)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 52

--S 53 of 208
daxpy(3,2.0,a,2,b,2)
--R
--R
--R   (15)  [3.,2.,9.,4.,15.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 53

--S 54 of 208
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (16)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 54

--S 55 of 208
daxpy(3,-2.0,a,2,b,2)
```

```
--R
--R
--R   (17)  [- 1.,2.,- 3.,4.,- 5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 55

--S 56 of 208
a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0] ]
--R
--R
--R   (18)  [1.,2.,3.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 56

--S 57 of 208
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0] ]
--R
--R
--R   (19)  [1.,2.,3.,4.,5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 57

--S 58 of 208
daxpy(3,-2.0,a,1,b,2)
--R
--R
--R   (20)  [- 1.,2.,- 1.,4.,- 1.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 58

--S 59 of 208
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (21)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 59

--S 60 of 208
daxpy(3,0.0,a,1,b,2)
--R
--R
--R   (22)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 60

)clear all

--S 61 of 208
a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
```

```
--R
--R
--R   (1)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 61

--S 62 of 208
b:PRIMARR(DFLOAT):=[ [ 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0] ]
--R
--R
--R   (2)  [0.,0.,0.,0.,0.,0.,0.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 62

--S 63 of 208
dcopy(3,a,1,b,1)
--R
--R
--R   (3)  [1.,2.,3.,0.,0.,0.,0.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 63

--S 64 of 208
b:PRIMARR(DFLOAT):=[ [ 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0] ]
--R
--R
--R   (4)  [0.,0.,0.,0.,0.,0.,0.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 64

--S 65 of 208
dcopy(7,a,1,b,1)
--R
--R
--R   (5)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 65

--S 66 of 208
b:PRIMARR(DFLOAT):=[ [ 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0] ]
--R
--R
--R   (6)  [0.,0.,0.,0.,0.,0.,0.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 66

--S 67 of 208
dcopy(8,a,1,b,1)
--R
--R
```

```
--R   (7)  [0.,0.,0.,0.,0.,0.,0.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 67

--S 68 of 208
b:PRIMARR(DFLOAT):=[ [ 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0] ]
--R
--R
--R   (8)  [0.,0.,0.,0.,0.,0.,0.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 68

--S 69 of 208
dcopy(3,a,3,b,3)
--R
--R
--R   (9)  [1.,0.,0.,4.,0.,0.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 69

--S 70 of 208
b:PRIMARR(DFLOAT):=[ [ 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0] ]
--R
--R
--R   (10)  [0.,0.,0.,0.,0.,0.,0.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 70

--S 71 of 208
dcopy(4,a,2,b,2)
--R
--R
--R   (11)  [1.,0.,3.,0.,5.,0.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 71

--S 72 of 208
b:PRIMARR(DFLOAT):=[ [ 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0] ]
--R
--R
--R   (12)  [0.,0.,0.,0.,0.,0.,0.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 72

--S 73 of 208
dcopy(5,a,2,b,2)
--R
--R
--R   (13)  [0.,0.,0.,0.,0.,0.,0.]
--R                                          Type: PrimitiveArray(DoubleFloat)
```

```
--E 73

--S 74 of 208
b:PRIMARR(DFLOAT):=[ [ 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0] ]
--R
--R
--R   (14)  [0.,0.,0.,0.,0.,0.,0.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 74

--S 75 of 208
dcopy(3,a,2,b,2)
--R
--R
--R   (15)  [1.,0.,3.,0.,5.,0.,0.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 75

--S 76 of 208
a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0] ]
--R
--R
--R   (16)  [1.,2.,3.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 76

--S 77 of 208
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0] ]
--R
--R
--R   (17)  [1.,2.,3.,4.,5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 77

--S 78 of 208
dcopy(3,a,1,b,1)
--R
--R
--R   (18)  [1.,2.,3.,4.,5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 78

--S 79 of 208
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0] ]
--R
--R
--R   (19)  [1.,2.,3.,4.,5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 79
```

```
--S 80 of 208
dcopy(3,a,1,b,2)
--R
--R
--R   (20)  [1.,2.,2.,4.,3.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 80

--S 81 of 208
a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0] ]
--R
--R
--R   (21)  [1.,2.,3.,4.,5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 81

--S 82 of 208
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0] ]
--R
--R
--R   (22)  [1.,2.,3.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 82

--S 83 of 208
dcopy(5,a,1,b,1)
--R
--R
--R   (23)  [1.,2.,3.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 83

)clear all

--S 84 of 208
a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0] ]
--R
--R   (1)  [1.,2.,3.,4.,5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 84

--S 85 of 208
b:PRIMARR(DFLOAT):=[ [ 5.0, 6.0, 7.0, 8.0, 9.0] ]
--R
--R   (2)  [5.,6.,7.,8.,9.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 85

--S 86 of 208
ddot(0,a,1,b,1)
```

```
--R
--R   (3)  0.
--R                                                    Type: DoubleFloat
--E 86

--S 87 of 208
ddot(3,a,1,b,1)
--R
--R   (4)  38.
--R                                                    Type: DoubleFloat
--E 87

--S 88 of 208
ddot(3,a,1,b,2)
--R
--R   (5)  46.
--R                                                    Type: DoubleFloat
--E 88

--S 89 of 208
ddot(3,a,2,b,1)
--R
--R   (6)  58.
--R                                                    Type: DoubleFloat
--E 89

--S 90 of 208
ddot(3,a,1,b,-2)
--R
--R   (7)  38.
--R                                                    Type: DoubleFloat
--E 90

--S 91 of 208
ddot(3,a,-2,b,1)
--R
--R   (8)  50.
--R                                                    Type: DoubleFloat
--E 91

--S 92 of 208
ddot(3,a,-2,b,-2)
--R
--R   (9)  71.
--R                                                    Type: DoubleFloat
--E 92

)clear all

--S 93 of 208
```

```
a:PRIMARR(DFLOAT):=[ [ 3.0, -4.0, 5.0, -7.0, 9.0] ]
--R
--R   (1)  [3.,- 4.,5.,- 7.,9.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 93

--S 94 of 208
dnrm2(3,a,1)
--R
--R   (2)  7.0710678118654755
--R                                                          Type: DoubleFloat
--E 94

--S 95 of 208
dnrm2(5,a,1)
--R
--R   (3)  13.416407864998739
--R                                                          Type: DoubleFloat
--E 95

--S 96 of 208
dnrm2(3,a,2)
--R
--R   (4)  10.72380529476361
--R                                                          Type: DoubleFloat
--E 96

)clear all
--S 97 of 208
a:MATRIX(DFLOAT):=[[6,5,0],[5,1,4],[0,4,3]]
--R
--R        +6.  5.  0.+
--R        |          |
--R   (1)  |5.  1.  4.|
--R        |          |
--R        +0.  4.  3.+
--R                                                Type: Matrix(DoubleFloat)
--E 97

--S 98 of 208
t1:=drotg(elt(a,1,1),elt(a,1,2),0.0,0.0)
--R
--R
--R   (2)
--R   [7.810249675906653,0.6401843996644799,0.768221279597376,0.6401843996644799]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 98

--S 99 of 208
g1:MATRIX(DFLOAT):=[[elt(t1,2), elt(t1,3),0.0],_
```

```
                    [-elt(t1,3),elt(t1,2),0.0],_
                    [0.0,       0.0,      1.0]]
--R
--R
--R        + 0.768221279597376    0.6401843996644799  0.+
--R        |                                          |
--R   (3)  |- 0.6401843996644799  0.768221279597376   0.|
--R        |                                          |
--R        +         0.                    0.          1.+
--R                                                     Type: Matrix(DoubleFloat)
--E 99

--S 100 of 208
t2:=g1*a
--R
--R
--R        +7.810249675906656    4.48129079765136    2.5607375986579197+
--R        |                                                          |
--R   (4)  |       0.          - 2.4327007187250236  3.072885118389504 |
--R        |                                                          |
--R        +       0.                   4.                   3.        +
--R                                                     Type: Matrix(DoubleFloat)
--E 100

--S 101 of 208
t3:=drotg(elt(t2,2,2),elt(a,3,2),0.0,0.0)
--R
--R
--R   (5)
--R   [4.681669871625428, - 1.9244742419770762, - 0.5196224393071984,
--R    0.8543959975142887]
--R                                            Type: PrimitiveArray(DoubleFloat)
--E 101

--S 102 of 208
g2:MATRIX(DFLOAT):=[[1.0, 0.0,      0.0],_
                    [0.0, elt(t3,2),elt(t3,3)],_
                    [0.0,-elt(t3,3),elt(t3,2)]]
--R
--R
--R        +1.           0.                    0.         +
--R        |                                              |
--R   (6)  |0.  - 0.5196224393071984   0.8543959975142887 |
--R        |                                              |
--R        +0.  - 0.8543959975142887  - 0.5196224393071984+
--R                                                     Type: Matrix(DoubleFloat)
--E 102

--S 103 of 208
g2*g1*a
```

```
--R
--R
--R        +   7.810249675906656     4.48129079765136   2.5607375986579197 +
--R        |                                                             |
--R   (7)  |- 4.440892098500626E-16  4.681669871625426  0.9664479316145229 |
--R        |                                                             |
--R        +- 4.440892098500626E-16         0.          - 4.184328063894808+
--R                                                   Type: Matrix(DoubleFloat)
--E 103

--S 104 of 208
q:=transpose(g1)*transpose(g2)
--R
--R
--R        +0.768221279597376   0.33265417936007147    0.5469709887444194 +
--R        |                                                             |
--R   (8)  |0.6401843996644799  - 0.3991850152320858  - 0.6563651864933033|
--R        |                                                             |
--R        +        0.           0.8543959975142887   - 0.5196224393071984+
--R                                                   Type: Matrix(DoubleFloat)
--E 104

)clear all

--S 105 of 208
dx:PRIMARR(DFLOAT):=[[6,0, 1.0, 4.0, -1.0, -1.0]]
--R
--R   (1)  [6.,0.,1.,4.,- 1.,- 1.]
--R                                              Type: PrimitiveArray(DoubleFloat)
--E 105

--S 106 of 208
dy:PRIMARR(DFLOAT):=[[5.0, 1.0, -4.0, 4.0, -4.0]]
--R
--R   (2)  [5.,1.,- 4.,4.,- 4.]
--R                                              Type: PrimitiveArray(DoubleFloat)
--E 106

--S 107 of 208
drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate by 45 degrees
--R
--R
--R   (3)
--R   [
--R     [7.778174591, 0.707106781, - 2.121320343, 5.656854248,
--R      - 3.5355339050000003, - 1.]
--R     ,
--R    [- 0.7071067809999993,0.707106781,- 3.5355339050000003,0.,- 2.121320343]]
--R                                         Type: List(PrimitiveArray(DoubleFloat))
--E 107
```

```
--S 108 of 208
[dx,dy] -- note that the input arguments, dx and dy were modified
--R
--R
--R   (4)
--R   [
--R     [7.778174591, 0.707106781, - 2.121320343, 5.656854248,
--R      - 3.5355339050000003, - 1.]
--R     ,
--R    [- 0.7071067809999993,0.707106781,- 3.5355339050000003,0.,- 2.121320343]]
--R                                       Type: List(PrimitiveArray(DoubleFloat))
--E 108

--S 109 of 208
drot(5,dx,1,dy,1,0.707106781,-0.707106781) -- rotate by -45 degrees
--R
--R
--R   (5)
--R   [
--R     [5.999999996834183, 5.551115123125783E-17, 0.9999999994723641,
--R      3.999999997889456, - 0.9999999994723647, - 1.]
--R     ,
--R
--R     [4.999999997361819, 0.9999999994723638, - 3.999999997889456,
--R      3.9999999978894554, - 3.9999999978894554]
--R     ]
--R                                       Type: List(PrimitiveArray(DoubleFloat))
--E 109

--S 110 of 208
[dx,dy] -- note that the input arguments, dx and dy were modified
--R
--R
--R   (6)
--R   [
--R     [5.999999996834183, 5.551115123125783E-17, 0.9999999994723641,
--R      3.999999997889456, - 0.9999999994723647, - 1.]
--R     ,
--R
--R     [4.999999997361819, 0.9999999994723638, - 3.999999997889456,
--R      3.9999999978894554, - 3.9999999978894554]
--R     ]
--R                                       Type: List(PrimitiveArray(DoubleFloat))
--E 110

--S 111 of 208
dx:PRIMARR(DFLOAT):=[[6,0, 1.0, 4.0, -1.0, -1.0]]
--R
--R   (7)  [6.,0.,1.,4.,- 1.,- 1.]
```

```
--R                                                Type: PrimitiveArray(DoubleFloat)
--E 111

--S 112 of 208
dy:PRIMARR(DFLOAT):=[[5.0, 1.0, -4.0, 4.0, -4.0]]
--R
--R   (8)  [5.,1.,- 4.,4.,- 4.]
--R                                                Type: PrimitiveArray(DoubleFloat)
--E 112

--S 113 of 208
drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate by 45 degrees
--R
--R
--R   (9)
--R   [
--R     [7.778174591, 0.707106781, - 2.121320343, 5.656854248,
--R      - 3.5355339050000003, - 1.]
--R     ,
--R    [- 0.7071067809999993,0.707106781,- 3.5355339050000003,0.,- 2.121320343]]
--R                                         Type: List(PrimitiveArray(DoubleFloat))
--E 113

--S 114 of 208
drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate to 90 degrees
--R
--R
--R   (10)
--R   [
--R     [4.99999999736182, 0.999999999472364, - 3.9999999978894563,
--R      3.999999997889456, - 3.9999999978894563, - 1.]
--R     ,
--R
--R     [- 5.999999996834183, 0., - 0.9999999994723645, - 3.999999997889456,
--R      0.9999999994723645]
--R     ]
--R                                         Type: List(PrimitiveArray(DoubleFloat))
--E 114

--S 115 of 208
drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate to 135 degrees
--R
--R
--R   (11)
--R   [
--R     [- 0.7071067806269045, 0.707106780626905, - 3.5355339031345254, 0.,
--R      - 2.1213203418807147, - 1.]
--R     ,
--R
--R     [- 7.778174586895954, - 0.707106780626905, 2.1213203418807147,
```

```
--R      - 5.65685424501524, 3.5355339031345254]
--R     ]
--R                                     Type: List(PrimitiveArray(DoubleFloat))
--E 115

--S 116 of 208
drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate to 180 degrees
--R
--R
--R   (12)
--R   [
--R     [- 5.999999993668366, 0., - 0.9999999989447283, - 3.999999995778912,
--R      0.9999999989447283, - 1.]
--R     ,
--R
--R     [- 4.999999994723639, - 0.999999998944728, 3.9999999957789116,
--R      - 3.999999995778912, 3.9999999957789116]
--R     ]
--R                                     Type: List(PrimitiveArray(DoubleFloat))
--E 116

--S 117 of 208
drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate to 225 degrees
--R
--R
--R   (13)
--R   [
--R     [- 7.778174582791909, - 0.70710678025381, 2.1213203407614296,
--R      - 5.65685424203048, 3.5355339012690505, - 1.]
--R     ,
--R
--R     [0.7071067802538096, - 0.70710678025381, 3.5355339012690505, 0.,
--R      2.1213203407614296]
--R     ]
--R                                     Type: List(PrimitiveArray(DoubleFloat))
--E 117

--S 118 of 208
drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate to 270 degrees
--R
--R
--R   (14)
--R   [
--R     [- 4.999999992085459, - 0.999999998417092, 3.999999993668368,
--R      - 3.999999993668368, 3.999999993668368, - 1.]
--R     ,
--R
--R     [5.999999990502551, 0., 0.9999999984170929, 3.999999993668368,
--R      - 0.9999999984170929]
--R     ]
```

```
--R                                       Type: List(PrimitiveArray(DoubleFloat))
--E 118

--S 119 of 208
drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate to 315 degrees
--R
--R
--R   (15)
--R   [
--R     [0.7071067798807156, - 0.707106779880715, 3.5355338994035757, 0.,
--R      2.1213203396421445, - 1.]
--R     ,
--R
--R     [7.778174578687864, 0.707106779880715, - 2.1213203396421445,
--R      5.65685423904572, - 3.5355338994035757]
--R     ]
--R                                       Type: List(PrimitiveArray(DoubleFloat))
--E 119

--S 120 of 208
drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate to 360 degrees
--R
--R
--R   (16)
--R   [
--R     [5.999999987336735, 0., 0.9999999978894567, 3.9999999915578237,
--R      - 0.9999999978894567, - 1.]
--R     ,
--R
--R     [4.999999989447278, 0.9999999978894559, - 3.9999999915578237,
--R      3.9999999915578237, - 3.9999999915578237]
--R     ]
--R                                       Type: List(PrimitiveArray(DoubleFloat))
--E 120

--S 121 of 208
[dx,dy] -- note that the input arguments, dx and dy were modified
--R
--R
--R   (17)
--R   [
--R     [5.999999987336735, 0., 0.9999999978894567, 3.9999999915578237,
--R      - 0.9999999978894567, - 1.]
--R     ,
--R
--R     [4.999999989447278, 0.9999999978894559, - 3.9999999915578237,
--R      3.9999999915578237, - 3.9999999915578237]
--R     ]
--R                                       Type: List(PrimitiveArray(DoubleFloat))
--E 121
```

```
)clear all
--S 122 of 208
dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]]
--R
--R   (1)  [1.,2.,3.,4.,5.,6.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 122

--S 123 of 208
dscal(6,2.0,dx,1)
--R
--R   (2)  [2.,4.,6.,8.,10.,12.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 123

--S 124 of 208
dx
--R
--R   (3)  [2.,4.,6.,8.,10.,12.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 124

--S 125 of 208
dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]]
--R
--R   (4)  [1.,2.,3.,4.,5.,6.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 125

--S 126 of 208
dscal(3,0.5,dx,1)
--R
--R   (5)  [0.5,1.,1.5,4.,5.,6.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 126

--S 127 of 208
dx
--R
--R   (6)  [0.5,1.,1.5,4.,5.,6.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 127

)clear all

--S 128 of 208
dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0,  5.0]]
--R
--R   (1)  [1.,2.,3.,4.,5.]
```

```
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 128

--S 129 of 208
dy:PRIMARR(DFLOAT):=[[9.0, 8.0, 7.0, 6.0, -5.0]]
--R
--R   (2)  [9.,8.,7.,6.,- 5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 129

--S 130 of 208
dswap(5,dx,1,dy,1)
--R
--R   (3)  [[9.,8.,7.,6.,- 5.],[1.,2.,3.,4.,5.]]
--R                                     Type: List(PrimitiveArray(DoubleFloat))
--E 130

--S 131 of 208
dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0,  5.0]]
--R
--R   (4)  [1.,2.,3.,4.,5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 131

--S 132 of 208
dy:PRIMARR(DFLOAT):=[[9.0, 8.0, 7.0, 6.0, -5.0]]
--R
--R   (5)  [9.,8.,7.,6.,- 5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 132

--S 133 of 208
dswap(3,dx,2,dy,2)
--R
--R   (6)  [[9.,2.,7.,4.,- 5.],[1.,8.,3.,6.,5.]]
--R                                     Type: List(PrimitiveArray(DoubleFloat))
--E 133

--S 134 of 208
dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0,  5.0]]
--R
--R   (7)  [1.,2.,3.,4.,5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 134

--S 135 of 208
dy:PRIMARR(DFLOAT):=[[9.0, 8.0, 7.0, 6.0, -5.0]]
--R
--R   (8)  [9.,8.,7.,6.,- 5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
```

```
--E 135

--S 136 of 208
dswap(5,dx,1,dy,-1)
--R
--R   (9)  [[9.,8.,7.,6.,- 5.],[1.,2.,3.,4.,5.]]
--R                                    Type: List(PrimitiveArray(DoubleFloat))
--E 136

)clear all

--S 137 of 208
d:PRIMARR(COMPLEX(DFLOAT)):=[[1.0+2.0*%i,-3.0+4.0*%i,5.0-6.0*%i]]
--R
--R   (1)  [1. + 2. %i,- 3. + 4. %i,5. - 6. %i]
--R                                 Type: PrimitiveArray(Complex(DoubleFloat))
--E 137

--S 138 of 208
dzasum(3,d,1) -- 21.0
--R
--R   (2)  21.
--R                                                          Type: DoubleFloat
--E 138

--S 139 of 208
dzasum(3,d,2) -- 14.0
--R
--R   (3)  14.
--R                                                          Type: DoubleFloat
--E 139

--S 140 of 208
dzasum(-3,d,1) -- 0.0
--R
--R   (4)  0.
--R                                                          Type: DoubleFloat
--E 140

)clear all

--S 141 of 208
a:PRIMARR(COMPLEX(DFLOAT)):=_
  [[3.+4.*%i, -4.+5.*%i, 5.+6.*%i, 7.-8.*%i, -9.-2.*%i]]
--R
--R   (1)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                 Type: PrimitiveArray(Complex(DoubleFloat))
--E 141

--S 142 of 208
```

```
dznrm2(5,a,1) -- should be 18.028
--R
--R   (2)  18.027756377319946
--R                                                        Type: DoubleFloat
--E 142

--S 143 of 208
dznrm2(3,a,2) -- should be 13.077
--R
--R   (3)  13.076696830622023
--R                                                        Type: DoubleFloat
--E 143

--S 144 of 208
dznrm2(3,a,1) -- should be 11.269
--R
--R   (4)  11.269427669584648
--R                                                        Type: DoubleFloat
--E 144

--S 145 of 208
dznrm2(3,a,-1) -- should be 0.0
--R
--R   (5)  0.
--R                                                        Type: DoubleFloat
--E 145

--S 146 of 208
dznrm2(-3,a,-1) -- should be 0.0
--R
--R   (6)  0.
--R                                                        Type: DoubleFloat
--E 146

--S 147 of 208
dznrm2(1,a,1) -- should be 5.0
--R
--R   (7)  5.
--R                                                        Type: DoubleFloat
--E 147

--S 148 of 208
dznrm2(1,a,2) -- should be 5.0
--R
--R   (8)  5.
--R                                                        Type: DoubleFloat
--E 148

)clear all
```

```
--S 149 of 208
a:PRIMARR(COMPLEX(FLOAT))
--R                                                                Type: Void
--E 149

--S 150 of 208
a:=[[3.+4.*%i,-4.+5.*%i,5.+6.*%i,7.-8.*%i,-9.-2.*%i]]
--R
--R   (2)  [3.0 + 4.0 %i,- 4.0 + 5.0 %i,5.0 + 6.0 %i,7.0 - 8.0 %i,- 9.0 - 2.0 %i]
--R                                        Type: PrimitiveArray(Complex(Float))
--E 150

--S 151 of 208
icamax(5,a,1)  -- should be 3
--R
--R   (3)  3
--R                                                     Type: PositiveInteger
--E 151

--S 152 of 208
icamax(0,a,1)  -- should be -1
--R
--R   (4)  - 1
--R                                                             Type: Integer
--E 152

--S 153 of 208
icamax(5,a,-1) -- should be -1
--R
--R   (5)  - 1
--R                                                             Type: Integer
--E 153

--S 154 of 208
icamax(3,a,1)  -- should be 2
--R
--R   (6)  2
--R                                                     Type: PositiveInteger
--E 155

--S 155 of 208
icamax(3,a,2)  -- should be 1
--R
--R   (7)  1
--R                                                     Type: PositiveInteger
--E 155

)clear all

--S 156 of 208
```

```
a:PRIMARR(DFLOAT):=[[3.0, 4.0, -3.0, 5.0, -1.0]]
--R
--R   (1)  [3.,4.,- 3.,5.,- 1.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 156

--S 157 of 208
idamax(5,a,1)  -- should be 3
--R
--R   (2)  3
--R                                                       Type: PositiveInteger
--E 157

--S 158 of 208
idamax(3,a,1)  -- should be 1
--R
--R   (3)  1
--R                                                       Type: PositiveInteger
--E 158

--S 159 of 208
idamax(0,a,1)  -- should be -1
--R
--R   (4)  - 1
--R                                                              Type: Integer
--E 159

--S 160 of 208
idamax(-5,a,1) -- should be -1
--R
--R   (5)  - 1
--R                                                              Type: Integer
--E 160

--S 161 of 208
idamax(5,a,-1) -- should be -1
--R
--R   (6)  - 1
--R                                                              Type: Integer
--E 161

--S 162 of 208
idamax(5,a,2)  -- should be 0
--R
--R   (7)  0
--R                                                   Type: NonNegativeInteger
--E 162

--S 163 of 208
idamax(1,a,0)  -- should be -1
```

```
--R
--R   (8)  - 1
--R                                                          Type: Integer
--E 163

--S 164 of 208
idamax(1,a,-1) -- should be -1
--R
--R   (9)  - 1
--R                                                          Type: Integer
--E 164

--S 165 of 208
a:PRIMARR(DFLOAT):=[[3.0, 4.0, -3.0, -5.0, -1.0]]
--R
--R   (10)  [3.,4.,- 3.,- 5.,- 1.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 165

--S 166 of 208
idamax(5,a,1)  -- should be 3
--R
--R   (11)  3
--R                                                     Type: PositiveInteger
--E 166

)clear all

--S 167 of 208
a:PRIMARR(FLOAT):=[[3.0, 4.0, -3.0, 5.0, -1.0]]
--R
--R
--R   (1)  [3.0,4.0,- 3.0,5.0,- 1.0]
--R                                                Type: PrimitiveArray(Float)
--E 167

--S 168 of 208
isamax(5,a,1)  -- should be 3
--R
--R   (2)  3
--R                                                     Type: PositiveInteger
--E 168

--S 169 of 208
isamax(3,a,1)  -- should be 1
--R
--R   (3)  1
--R                                                     Type: PositiveInteger
--E 169
```

```
--S 170 of 208
isamax(0,a,1)  -- should be -1
--R
--R   (4)  - 1
--R                                                          Type: Integer
--E 170

--S 171 of 208
isamax(-5,a,1) -- should be -1
--R
--R   (5)  - 1
--R                                                          Type: Integer
--E 171

--S 172 of 208
isamax(5,a,-1) -- should be -1
--R
--R   (6)  - 1
--R                                                          Type: Integer
--E 172

--S 173 of 208
isamax(5,a,2)  -- should be 0
--R
--R   (7)  0
--R                                                 Type: NonNegativeInteger
--E 173

--S 174 of 208
isamax(1,a,0)  -- should be -1
--R
--R   (8)  - 1
--R                                                          Type: Integer
--E 174

--S 175 of 208
isamax(1,a,-1) -- should be -1
--R
--R   (9)  - 1
--R                                                          Type: Integer
--E 175

--S 176 of 208
a:PRIMARR(FLOAT):=[[3.0, 4.0, -3.0, -5.0, -1.0]]
--R
--R
--R   (10)  [3.0,4.0,- 3.0,- 5.0,- 1.0]
--R                                            Type: PrimitiveArray(Float)
--E 176
```

```
--S 177 of 208
isamax(5,a,1)  -- should be 3
--R
--R   (11)  3
--R                                                    Type: PositiveInteger
--E 177

)clear all

--S 178 of 208
a:PRIMARR(COMPLEX(DFLOAT)):=[[3.+4.*%i,-4.+5.*%i,5.+6.*%i,7.-8.*%i,-9.-2.*%i]]
--R
--R   (1)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                  Type: PrimitiveArray(Complex(DoubleFloat))
--E 178

--S 179 of 208
izamax(5,a,1)  -- should be 3
--R
--R   (2)  3
--R                                                    Type: PositiveInteger
--E 179

--S 180 of 208
izamax(0,a,1)  -- should be -1
--R
--R   (3)  - 1
--R                                                            Type: Integer
--E 180

--S 181 of 208
izamax(5,a,-1) -- should be -1
--R
--R   (4)  - 1
--R                                                            Type: Integer
--E 181

--S 182 of 208
izamax(3,a,1)  -- should be 2
--R
--R   (5)  2
--R                                                    Type: PositiveInteger
--E 182

--S 183 of 208
izamax(3,a,2)  -- should be 1
--R
--R   (6)  1
--R                                                    Type: PositiveInteger
--E 183
```

```
)clear all

--S 184 of 208
a:PRIMARR(COMPLEX(DFLOAT)):=_
     [[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (1)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                    Type: PrimitiveArray(Complex(DoubleFloat))
--E 184

--S 185 of 208
b:PRIMARR(COMPLEX(DFLOAT)):=_
     [[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (2)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                    Type: PrimitiveArray(Complex(DoubleFloat))
--E 185

--S 186 of 208
zaxpy(3,2.0,a,1,b,1)
--R
--R   (3)  [9. + 12. %i,- 12. + 15. %i,15. + 18. %i,7. - 8. %i,- 9. - 2. %i]
--R                                    Type: PrimitiveArray(Complex(DoubleFloat))
--E 186

--S 187 of 208
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (4)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                    Type: PrimitiveArray(Complex(DoubleFloat))
--E 187

--S 188 of 208
zaxpy(5,2.0,a,1,b,1)
--R
--R   (5)  [9. + 12. %i,- 12. + 15. %i,15. + 18. %i,21. - 24. %i,- 27. - 6. %i]
--R                                    Type: PrimitiveArray(Complex(DoubleFloat))
--E 188

--S 189 of 208
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (6)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                    Type: PrimitiveArray(Complex(DoubleFloat))
--E 189

--S 190 of 208
zaxpy(3,2.0,a,3,b,3)
--R
```

```
--R   (7)  [9. + 12. %i,- 4. + 5. %i,5. + 6. %i,21. - 24. %i,- 9. - 2. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 190

--S 191 of 208
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (8)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 191

--S 192 of 208
zaxpy(4,2.0,a,2,b,2)
--R
--R   (9)  [9. + 12. %i,- 4. + 5. %i,15. + 18. %i,7. - 8. %i,- 27. - 6. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 192

--S 193 of 208
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (10)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 193

--S 194 of 208
zaxpy(3,2.0,a,2,b,2)
--R
--R   (11)  [9. + 12. %i,- 4. + 5. %i,15. + 18. %i,7. - 8. %i,- 27. - 6. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 194

--S 195 of 208
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (12)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 195

--S 196 of 208
zaxpy(3,-2.0,a,1,b,2)
--R
--R   (13)  [- 3. - 4. %i,- 4. + 5. %i,13. - 4. %i,7. - 8. %i,- 19. - 14. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 196

--S 197 of 208
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (14)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
```

```
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 197

--S 198 of 208
zaxpy(3,2.0,a,1,b,2)
--R
--R   (15)  [9. + 12. %i,- 4. + 5. %i,- 3. + 16. %i,7. - 8. %i,1. + 10. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 198

--S 199 of 208
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (16)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 199

--S 200 of 208
zaxpy(-3,2.0,a,1,b,2)
--R
--R   (17)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 200

--S 201 of 208
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (18)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 201

--S 202 of 208
zaxpy(3,2.0,a,-1,b,2)
--R
--R   (19)  [13. + 16. %i,- 4. + 5. %i,- 3. + 16. %i,7. - 8. %i,- 3. + 6. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 202

--S 203 of 208
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (20)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 203

--S 204 of 208
zaxpy(3,2.0,a,1,b,-2)
--R
--R   (21)  [13. + 16. %i,- 4. + 5. %i,- 3. + 16. %i,7. - 8. %i,- 3. + 6. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
```

```
--E 204

--S 205 of 208
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (22)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 205

--S 206 of 208
zaxpy(3,2.0,a,-1,b,-2)
--R
--R   (23)  [9. + 12. %i,- 4. + 5. %i,- 3. + 16. %i,7. - 8. %i,1. + 10. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 206

--S 207 of 208
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (24)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 207

--S 208 of 208
zaxpy(3,0.0,a,1,b,1)
--R
--R   (25)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 208

)spool
)lisp (bye)
```

———-

5.1.2 Axiom help page

— BlasLevelOne.help —

```
=====================================================================
BlasLevelOne examples
=====================================================================

The dcabs1 routine computes the sum of the absolute value of the
real and imaginary parts of a complex number.

t1:Complex DoubleFloat := complex(1.0,0)
   1.
```

```
dcabs1(t1)
   1.

t2:Complex DoubleFloat := complex(1.0,1.0)
   1. + %i

dcabs1(t2)
   2.

t3:Complex DoubleFloat := complex(1.0,-1.0)
   1. - %i

dcabs1(t3)
   2.

t4:Complex DoubleFloat := complex(-1.0,-1.0)
   - 1. - %i

dcabs1(t4)
   2.

t5:Complex DoubleFloat := complex(-2.0,-2.0)
   - 2. - 2. %i

dcabs1(t5)
   4.

See Also:
o )show BlasLevelOne
```

———-

5.1.3 BlasLevelOne (BLAS1 Spad code)

See: Chellappa, Franchetti and Puschel [Chel06]

— **package BLAS1 BlasLevelOne** —

```
)abbrev package BLAS1 BlasLevelOne
++ Author: Timothy Daly
++ Date Created: 2010
++ Date March 24, 2010
++ Reference: Chellappa, Franchetti and Puschel
++ How to Write Fast Numerical Code: A Small Introduction
++ Description:
++ This package provides an interface to the Blas library (level 1)
```

```
BlasLevelOne() : Exports == Implementation where

  SI   ==> SingleInteger
  INT  ==> Integer
  DF   ==> DoubleFloat
  SX   ==> PrimitiveArray(Float)
  DX   ==> PrimitiveArray(DoubleFloat)
  CDF  ==> Complex(DoubleFloat)
  LDX  ==> List(PrimitiveArray(DoubleFloat))
  PCDF ==> PrimitiveArray(Complex(DoubleFloat))
  PCF  ==> PrimitiveArray(Complex(Float))

  Exports == with

    dcabs1 : CDF -> DF
      ++ dcabs1(z) computes (+ (abs (realpart z)) (abs (imagpart z)))
      ++
      ++X t1:Complex DoubleFloat := complex(1.0,0)
      ++X dcabs1(t1)

    dasum : (SI, DX, SI) -> DF
      ++ dasum(n,array,incx) computes the sum of n elements in array
      ++ using a stride of incx
      ++
      ++X dx:PRIMARR(DFLOAT):=[[1.0,2.0,3.0,4.0,5.0,6.0]]
      ++X dasum(6,dx,1)
      ++X dasum(3,dx,2)

    daxpy : (SI, DF, DX, SI, DX, SI) -> DX
      ++ daxpy(n,da,x,incx,y,incy) computes a y = a*x + y
      ++ for each of the chosen elements of the vectors x and y
      ++ and a constant multiplier a
      ++ Note that the vector y is modified with the results.
      ++
      ++X x:PRIMARR(DFLOAT):=[[1.0,2.0,3.0,4.0,5.0,6.0]]
      ++X y:PRIMARR(DFLOAT):=[[1.0,2.0,3.0,4.0,5.0,6.0]]
      ++X daxpy(6,2.0,x,1,y,1)
      ++X y
      ++X m:PRIMARR(DFLOAT):=[[1.0,2.0,3.0]]
      ++X n:PRIMARR(DFLOAT):=[[1.0,2.0,3.0,4.0,5.0,6.0]]
      ++X daxpy(3,-2.0,m,1,n,2)
      ++X n

    dcopy : (SI, DX, SI, DX, SI) -> DX
      ++ dcopy(n,x,incx,y,incy) copies y from x
      ++ for each of the chosen elements of the vectors x and y
      ++ Note that the vector y is modified with the results.
      ++
      ++X x:PRIMARR(DFLOAT):=[[1.0,2.0,3.0,4.0,5.0,6.0]]
      ++X y:PRIMARR(DFLOAT):=[[0.0,0.0,0.0,0.0,0.0,0.0]]
```

```
  ++X dcopy(6,x,1,y,1)
  ++X y
  ++X m:PRIMARR(DFLOAT):=[[1.0,2.0,3.0]]
  ++X n:PRIMARR(DFLOAT):=[[0.0,0.0,0.0,0.0,0.0,0.0]]
  ++X dcopy(3,m,1,n,2)
  ++X n

ddot : (SI, DX, SI, DX, SI) -> DF
  ++ ddot(n,x,incx,y,incy) computes the vector dot product
  ++ of elements from the vector x and the vector y
  ++ If the indicies are negative the elements are taken
  ++ relative to the far end of the vector.
  ++
  ++X x:PRIMARR(DFLOAT):=[[1.0,2.0,3.0,4.0,5.0]]
  ++X y:PRIMARR(DFLOAT):=[[5.0,6.0,7.0,8.0,9.0]]
  ++X ddot(0,a,1,b,1) -- handle 0 elements ==> 0
  ++X ddot(3,a,1,b,1) -- (1,2,3) * (5,6,7) ==> 38.0
  ++X ddot(3,a,1,b,2) -- increment = 2 in b (1,2,3) * (5,7,9) ==> 46.0
  ++X ddot(3,a,2,b,1) -- increment = 2 in a (1,3,5) * (5,6,7) ==> 58.0
  ++X ddot(3,a,1,b,-2) -- increment = -2 in b (1,2,3) * (9,7,5) ==> 38.0
  ++X ddot(2,a,-2,b,1) -- increment = -2 in a (5,3,1) * (5,6,7) ==> 50.0
  ++X ddot(3,a,-2,b,-2) -- (5,3,1) * (9,7,5) ==> 71.0

dnrm2 : (SI, DX, SI) -> DF
  ++ dnrm2 takes the norm of the vector, ||x||
  ++
  ++X a:PRIMARR(DFLOAT):=[[3.0, -4.0, 5.0, -7.0, 9.0]]
  ++X dnrm2(3,a,1) -- 7.0710678118654755 = sqrt(3.0^2 + -4.0^2 + 5.0^2)
  ++X dnrm2(5,a,1) -- 13.416407864998739 = sqrt(180.0)
  ++X dnrm2(3,a,2) -- 10.72380529476361  = sqrt(115.0)

drotg : (DF, DF, DF, DF) -> DX
  ++ drotg computes a 2D plane Givens rotation spanned by two
  ++ coordinate axes.
  ++
  ++X a:MATRIX(DFLOAT):=[[6,5,0],[5,1,4],[0,4,3]]
  ++X drotg(elt(a,1,1),elt(a,1,2),0.0D0,0.0D0)

drot : (SI, DX, SI, DX, SI, DF, DF) -> LDX
  ++ drot computes a 2D plane Givens rotation spanned by two
  ++ coordinate axes. It modifies the arrays in place.
  ++ The call drot(n,dx,incx,dy,incy,c,s) has the dx array which
  ++ contains the y axis locations and dy which contains the
  ++ y axis locations. They are rotated in parallel where
  ++ c is the cosine of the angle and s is the sine of the angle and
  ++ \spad{c^2+s^2 = 1}
  ++
  ++X dx:PRIMARR(DFLOAT):=[[6,0, 1.0, 4.0, -1.0, -1.0]]
  ++X dy:PRIMARR(DFLOAT):=[[5.0, 1.0, -4.0, 4.0, -4.0]]
  ++X drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate by 45 degrees
```

```
  ++X dx -- dx has been modified
  ++X dy -- dy has been modified
  ++X drot(5,dx,1,dy,1,0.707106781,-0.707106781) -- rotate by -45 degrees
  ++X dx -- dx has been modified
  ++X dy -- dy has been modified

dscal : (SI, DF, DX, SI) -> DX
  ++ dscal scales each element of the vector by the scalar so
  ++ dscal(n,da,dx,incx) = da*dx for n elements, incremented by incx
  ++ Note that the dx array is modified in place.
  ++
  ++X dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]]
  ++X dscal(6,2.0,dx,1)
  ++X dx
  ++X dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]]
  ++X dscal(3,0.5,dx,1)
  ++X dx

dswap : (SI, DX, SI, DX, SI) -> LDX
  ++ dswap swaps elements from the first vector with the second
  ++ Note that the arrays are modified in place.
  ++
  ++X dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]]
  ++X dy:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]]
  ++X dswap(5,dx,1,dy,1)
  ++X dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]]
  ++X dy:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]]
  ++X dswap(3,dx,2,dy,2)
  ++X dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]]
  ++X dy:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]]
  ++X dswap(5,dx,1,dy,-1)

dzasum : (SI, PCDF, SI) -> DF
  ++ dzasum takes the sum over all of the array where each
  ++ element of the array sum is the sum of the absolute
  ++ value of the real part and the absolute value of the
  ++ imaginary part of each array element:
  ++   for i in array do sum = sum + (real(a(i)) + imag(a(i)))
  ++
  ++X d:PRIMARR(COMPLEX(DFLOAT)):=[[1.0+2.0*%i,-3.0+4.0*%i,5.0-6.0*%i]]
  ++X dzasum(3,d,1) -- 21.0
  ++X dzasum(3,d,2) -- 14.0
  ++X dzasum(-3,d,1) -- 0.0

dznrm2 : (SI, PCDF, SI) -> DF
  ++ dznrm2 returns the norm of a complex vector. It computes
  ++ sqrt(sum(v*conjugate(v)))
  ++
  ++X a:PRIMARR(COMPLEX(DFLOAT))
  ++X a:=[[3.+4.*%i,-4.+5.*%i,5.+6.*%i,7.-8.*%i,-9.-2.*%i]]
```

```
  ++X dznrm2(5,a,1)   -- should be 18.028
  ++X dznrm2(3,a,2)   -- should be 13.077
  ++X dznrm2(3,a,1)   -- should be 11.269
  ++X dznrm2(3,a,-1)  -- should be 0.0
  ++X dznrm2(-3,a,-1) -- should be 0.0
  ++X dznrm2(1,a,1)   -- should be 5.0
  ++X dznrm2(1,a,2)   -- should be 5.0

icamax : (INT, PCF, INT) -> INT
  ++ icamax computes the largest absolute value of the elements
  ++ of the array and returns the index of the first instance
  ++ of the maximum
  ++
  ++X a:PRIMARR(COMPLEX(FLOAT))
  ++X a:=[[3.+4.*%i,-4.+5.*%i,5.+6.*%i,7.-8.*%i,-9.-2.*%i]]
  ++X icamax(5,a,1)  -- should be 3
  ++X icamax(0,a,1)  -- should be -1
  ++X icamax(5,a,-1) -- should be -1
  ++X icamax(3,a,1)  -- should be 2
  ++X icamax(3,a,2)  -- should be 1

idamax : (INT, DX, INT) -> INT
  ++ idamax computes the largest absolute value of the elements
  ++ of the array and returns the index of the first instance
  ++ of the maximum.
  ++
  ++X a:PRIMARR(DFLOAT):=[[3.0, 4.0, -3.0, 5.0, -1.0]]
  ++X idamax(5,a,1)  -- should be 3
  ++X idamax(3,a,1)  -- should be 1
  ++X idamax(0,a,1)  -- should be -1
  ++X idamax(-5,a,1) -- should be -1
  ++X idamax(5,a,-1) -- should be -1
  ++X idamax(5,a,2)  -- should be 0
  ++X idamax(1,a,0)  -- should be -1
  ++X idamax(1,a,-1) -- should be -1
  ++X a:PRIMARR(DFLOAT):=[[3.0, 4.0, -3.0, -5.0, -1.0]]
  ++X idamax(5,a,1)  -- should be 3

isamax : (INT, SX, INT) -> INT
  ++ isamax computes the largest absolute value of the elements
  ++ of the array and returns the index of the first instance
  ++ of the maximum.
  ++
  ++X a:PRIMARR(FLOAT):=[[3.0, 4.0, -3.0, 5.0, -1.0]]
  ++X isamax(5,a,1)  -- should be 3
  ++X isamax(3,a,1)  -- should be 1
  ++X isamax(0,a,1)  -- should be -1
  ++X isamax(-5,a,1) -- should be -1
  ++X isamax(5,a,-1) -- should be -1
  ++X isamax(5,a,2)  -- should be 0
```

```
    ++X isamax(1,a,0)  -- should be -1
    ++X isamax(1,a,-1) -- should be -1
    ++X a:PRIMARR(FLOAT):=[[3.0, 4.0, -3.0, -5.0, -1.0]]
    ++X isamax(5,a,1)  -- should be 3

  izamax : (SI, PCDF, SI) -> INT
    ++ izamax computes the largest absolute value of the elements
    ++ of the array and returns the index of the first instance
    ++ of the maximum.
    ++
    ++X a:PRIMARR(COMPLEX(DFLOAT))
    ++X a:=[[3.+4.*%i,-4.+5.*%i,5.+6.*%i,7.-8.*%i,-9.-2.*%i]]
    ++X izamax(5,a,1)  -- should be 3
    ++X izamax(0,a,1)  -- should be -1
    ++X izamax(5,a,-1) -- should be -1
    ++X izamax(3,a,1)  -- should be 2
    ++X izamax(3,a,2)  -- should be 1

  zaxpy : (SI, CDF, PCDF, SI, PCDF, SI) -> PCDF
    ++ zaxpy(n,da,x,incx,y,incy) computes a y = a*x + y
    ++ for each of the chosen elements of the vectors x and y
    ++ and a constant multiplier a
    ++ Note that the vector y is modified with the results.
    ++
    ++X a:PRIMARR(COMPLEX(DFLOAT))
    ++X a:=[[3.+4.*%i, -4.+5.*%i, 5.+6.*%i, 7.-8.*%i, -9.-2.*%i]]
    ++X b:PRIMARR(COMPLEX(DFLOAT))
    ++X b:=[[3.+4.*%i, -4.+5.*%i, 5.+6.*%i, 7.-8.*%i, -9.-2.*%i]]
    ++X zaxpy(3,2.0,a,1,b,1)
    ++X b:=[[3.+4.*%i, -4.+5.*%i, 5.+6.*%i, 7.-8.*%i, -9.-2.*%i]]
    ++X zaxpy(5,2.0,a,1,b,1)
    ++X b:=[[3.+4.*%i, -4.+5.*%i, 5.+6.*%i, 7.-8.*%i, -9.-2.*%i]]
    ++X zaxpy(3,2.0,a,3,b,3)
    ++X b:=[[3.+4.*%i, -4.+5.*%i, 5.+6.*%i, 7.-8.*%i, -9.-2.*%i]]
    ++X zaxpy(4,2.0,a,2,b,2)

Implementation  == add

  dcabs1(z:CDF):DF ==
    DCABS1(COMPLEX(real(z),imag(z))$Lisp)$Lisp

  dasum(n:SI,dx:DX,incx:SI):DF ==
    DASUM(n,dx,incx)$Lisp

  daxpy(n:SI,da:DF,dx:DX,incx:SI,dy:DX,incy:SI):DX ==
    DAXPY(n,da,dx,incx,dy,incy)$Lisp

  dcopy(n:SI,dx:DX,incx:SI,dy:DX,incy:SI):DX ==
    DCOPY(n,dx,incx,dy,incy)$Lisp
```

```
ddot(n:SI,dx:DX,incx:SI,dy:DX,incy:SI):DF ==
  DDOT(n,dx,incx,dy,incy)$Lisp

dnrm2(n:SI,dx:DX,incx:SI):DF ==
  DNRM2(n,dx,incx)$Lisp

drotg(a:DF,b:DF,c:DF,s:DF):DX ==
  DROTG(a,b,c,s)$Lisp

drot(n:SI,dx:DX,incx:SI,dy:DX,incy:SI,c:DF,s:DF):LDX ==
  DROT(n,dx,incx,dy,incy,c,s)$Lisp

dscal(n:SI,da:DF,dx:DX,incx:SI):DX ==
  DSCAL(n,da,dx,incx)$Lisp

dswap(n:SI,dx:DX,incx:SI,dy:DX,incy:SI):LDX ==
  DSWAP(n,dx,incx,dy,incx)$Lisp

dzasum(n:SI,dz:PCDF,incx:SI):DF ==
  DZASUMSPAD(n,dz,incx)$Lisp

dznrm2(n:SI,dz:PCDF,incx:SI):DF ==
  DZNRM2SPAD(n,dz,incx)$Lisp

icamax(n:INT,dz:PCF,incx:INT):INT ==
  ICAMAXSPAD(n,dz,incx)$Lisp

idamax(n:INT,dz:DX,incx:INT):INT ==
  IDAMAX(n,dz,incx)$Lisp

isamax(n:INT,dz:SX,incx:INT):INT ==
  ISAMAXSPAD(n,dz,incx)$Lisp

izamax(n:SI,dz:PCDF,incx:SI):INT ==
  IZAMAXSPAD(n,dz,incx)$Lisp

zaxpy(n:SI,da:CDF,dx:PCDF,incx:SI,dy:PCDF,incy:SI):PCDF ==
  ZAXPYSPAD(n,da,dx,incx,dy,incy)$Lisp
```

———-

— BLAS1.dotabb —

```
"BLAS1" [color="#444488",href="bookvol10.5.pdf#nameddest=BLAS1"]
"FIELD" [color="#4488FF",href="bookvol10.2.pdf#nameddest=FIELD"]
"RADCAT" [color="#4488FF",href="bookvol10.2.pdf#nameddest=RADCAT"]
"BLAS1" -> "FIELD"
```

"BLAS1" -> "RADCAT"

5.2 lsame

The `lsame` function returns t if `ca` and
cb represent the same letter regardless of case.

This has been replaced everywhere with common lisp's char-equal function which compares characters ignoring case. The type (simple-array character (*)) has been replaced everywhere with character.

5.2.1 fortran code

— lsame.f —

```
      LOGICAL          FUNCTION LSAME( CA, CB )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     September 30, 1994
*
*     .. Scalar Arguments ..
      CHARACTER          CA, CB
*     ..
*
*  Purpose
*  =======
*
*  LSAME returns .TRUE. if CA is the same letter as CB regardless of
*  case.
*
*  Arguments
*  =========
*
*  CA      (input) CHARACTER*1
*  CB      (input) CHARACTER*1
*          CA and CB specify the single characters to be compared.
*
* =====================================================================
*
*     .. Intrinsic Functions ..
      INTRINSIC          ICHAR
```

```
*     ..
*     .. Local Scalars ..
      INTEGER            INTA, INTB, ZCODE
*     ..
*     .. Executable Statements ..
*
*     Test if the characters are equal
*
      LSAME = CA.EQ.CB
      IF( LSAME )
     $   RETURN
*
*     Now test for equivalence if both characters are alphabetic.
*
      ZCODE = ICHAR( 'Z' )
*
*     Use 'Z' rather than 'A' so that ASCII can be detected on Prime
*     machines, on which ICHAR returns a value with bit 8 set.
*     ICHAR('A') on Prime machines returns 193 which is the same as
*     ICHAR('A') on an EBCDIC machine.
*
      INTA = ICHAR( CA )
      INTB = ICHAR( CB )
*
      IF( ZCODE.EQ.90 .OR. ZCODE.EQ.122 ) THEN
*
*        ASCII is assumed - ZCODE is the ASCII code of either lower or
*        upper case 'Z'.
*
         IF( INTA.GE.97 .AND. INTA.LE.122 ) INTA = INTA - 32
         IF( INTB.GE.97 .AND. INTB.LE.122 ) INTB = INTB - 32
*
      ELSE IF( ZCODE.EQ.233 .OR. ZCODE.EQ.169 ) THEN
*
*        EBCDIC is assumed - ZCODE is the EBCDIC code of either lower or
*        upper case 'Z'.
*
         IF( INTA.GE.129 .AND. INTA.LE.137 .OR.
     $       INTA.GE.145 .AND. INTA.LE.153 .OR.
     $       INTA.GE.162 .AND. INTA.LE.169 ) INTA = INTA + 64
         IF( INTB.GE.129 .AND. INTB.LE.137 .OR.
     $       INTB.GE.145 .AND. INTB.LE.153 .OR.
     $       INTB.GE.162 .AND. INTB.LE.169 ) INTB = INTB + 64
*
      ELSE IF( ZCODE.EQ.218 .OR. ZCODE.EQ.250 ) THEN
*
*        ASCII is assumed, on Prime machines - ZCODE is the ASCII code
*        plus 128 of either lower or upper case 'Z'.
*
         IF( INTA.GE.225 .AND. INTA.LE.250 ) INTA = INTA - 32
```

```
         IF( INTB.GE.225 .AND. INTB.LE.250 ) INTB = INTB - 32
      END IF
      LSAME = INTA.EQ.INTB
*
*     RETURN
*
*     End of LSAME
*
      END
```

———-

5.2.2 lisp code

— BLAS lsame —

```
(defun lsame (ca cb)
  (declare (type (string *) cb ca))
  (f2cl-lib:with-multi-array-data
      ((ca character ca-%data% ca-%offset%)
       (cb character cb-%data% cb-%offset%))
    (prog ((inta 0) (intb 0) (zcode 0) (lsame nil))
      (declare (type f2cl-lib:logical lsame)
               (type (f2cl-lib:integer4) zcode intb inta))
      (setf lsame (coerce (f2cl-lib:fstring-= ca cb) 'f2cl-lib:logical))
      (if lsame
          (go end_label))
      (setf zcode (f2cl-lib:ichar "Z"))
      (setf inta (f2cl-lib:ichar ca))
      (setf intb (f2cl-lib:ichar cb))
      (cond
       ((or (= zcode 90) (= zcode 122))
        (if (and (>= inta 97) (<= inta 122))
            (setf inta (f2cl-lib:int-sub inta 32)))
        (if (and (>= intb 97) (<= intb 122))
            (setf intb (f2cl-lib:int-sub intb 32))))
       ((or (= zcode 233) (= zcode 169))
        (if (or (and (>= inta 129) (<= inta 137))
                (and (>= inta 145) (<= inta 153))
                (and (>= inta 162) (<= inta 169)))
            (setf inta (f2cl-lib:int-add inta 64)))
        (if (or (and (>= intb 129) (<= intb 137))
                (and (>= intb 145) (<= intb 153))
                (and (>= intb 162) (<= intb 169)))
            (setf intb (f2cl-lib:int-add intb 64))))
       ((or (= zcode 218) (= zcode 250))
        (if (and (>= inta 225) (<= inta 250))
```

```
            (setf inta (f2cl-lib:int-sub inta 32)))
        (if (and (>= intb 225) (<= intb 250))
            (setf intb (f2cl-lib:int-sub intb 32)))))
      (setf lsame (coerce (= inta intb) 'f2cl-lib:logical))
     end_label
      (return (values lsame nil nil)))))
```

5.3 xerbla

The `xerbla` routine is an error handler. It is called if an input parameter has an invalid value.

This function has been rewritten everywhere to use the common lisp error function.

5.3.1 fortran code

— xerbla.f —

```
*  ====================================================================
      SUBROUTINE XERBLA( SRNAME, INFO )
*
*  -- LAPACK auxiliary routine (preliminary version) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     February 29, 1992
*
*     .. Scalar Arguments ..
      CHARACTER*6        SRNAME
      INTEGER            INFO
*     ..
*
*  Purpose
*  =======
*
*  XERBLA  is an error handler for the LAPACK routines.
*  It is called by an LAPACK routine if an input parameter has an
*  invalid value.  A message is printed and execution stops.
*
*  Installers may consider modifying the STOP statement in order to
*  call system-specific exception-handling facilities.
*
*  Arguments
*  =========
```

```
*
*  SRNAME  (input) CHARACTER*6
*          The name of the routine which called XERBLA.
*
*  INFO    (input) INTEGER
*          The position of the invalid parameter in the parameter list
*          of the calling routine.
*
*
      WRITE( *, FMT = 9999 )SRNAME, INFO
*
*
* commented out by RLT
*      STOP
*
 9999 FORMAT( ' ** On entry to ', A6, ' parameter number ', I2, ' had ',
     $      'an illegal value' )
*
*     End of XERBLA
*
      END
```

———-

5.3.2 lisp code

— BLAS xerbla —

```
(defun xerbla (srname info)
  (declare (type (f2cl-lib:integer4) info)
           (type (string *) srname))
  (f2cl-lib:with-multi-array-data
      ((srname character srname-%data% srname-%offset%))
    (prog ()
      (declare)
      (f2cl-lib:fformat t
                        (" ** On entry to " ("~A") " parameter number " 1
                         (("~2D")) " had " "an illegal value" "~%")
                        (f2cl-lib:fref-string srname
                                              (1 (f2cl-lib:len_trim srname)))
                        info)
      (f2cl-lib::stop)
     end_label
      (return (values nil nil)))))
```

———-

Chapter 6

BLAS Level 1 Real Vector Operations

6.1 sasum - the sum of the absolute values

6.1.1 Axiom unit tests

— **sasum.input** —

```
)set break resume
)sys rm -f sasum.output
)spool sasum.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

6.1.2 Axiom help page

— **sasum.help** —

```
==========================================================================
sasum examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       real function sasum (N, SX, INCX)

Function/Subroutine Documentation
   real function sasum (integer N, real, dimension(*) SX, integer INCX)
       SASUM

Purpose:
  SASUM takes the sum of the absolute values.
  uses unrolled loops for increment equal to one.
```

6.1.3 fortran code

— sasum.f —

```
*  ====================================================================
      REAL FUNCTION SASUM(N,SX,INCX)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,N
*     ..
*     .. Array Arguments ..
      REAL SX(*)
*     ..
*
*  ====================================================================
*
*     .. Local Scalars ..
      REAL STEMP
      INTEGER I,M,MP1,NINCX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC ABS,MOD
*     ..
```

```
      SASUM = 0.0e0
      STEMP = 0.0e0
      IF (N.LE.0 .OR. INCX.LE.0) RETURN
      IF (INCX.EQ.1) THEN
*        code for increment equal to 1
*
*
*        clean-up loop
*
         M = MOD(N,6)
         IF (M.NE.0) THEN
            DO I = 1,M
               STEMP = STEMP + ABS(SX(I))
            END DO
            IF (N.LT.6) THEN
               SASUM = STEMP
               RETURN
            END IF
         END IF
         MP1 = M + 1
         DO I = MP1,N,6
            STEMP = STEMP + ABS(SX(I)) + ABS(SX(I+1)) +
     $              ABS(SX(I+2)) + ABS(SX(I+3)) +
     $              ABS(SX(I+4)) + ABS(SX(I+5))
         END DO
      ELSE
*
*        code for increment not equal to 1
*
         NINCX = N*INCX
         DO I = 1,NINCX,INCX
            STEMP = STEMP + ABS(SX(I))
         END DO
      END IF
      SASUM = STEMP
      RETURN
      END
```

———-

6.1.4 lisp code

— BLAS 1 sasum —

```
(defun sasum (n sx incx)
  (declare (type (array single-float (*)) sx)
           (type (f2cl-lib:integer4) incx n))
```

```
(f2cl-lib:with-multi-array-data
    ((sx single-float sx-%data% sx-%offset%))
  (prog ((i 0) (m 0) (mp1 0) (nincx 0) (stemp 0.0) (sasum 0.0))
    (declare (type (single-float) sasum stemp)
             (type (f2cl-lib:integer4) nincx mp1 m i))
    (setf sasum 0.0)
    (setf stemp 0.0)
    (if (or (<= n 0) (<= incx 0))
        (go end_label))
    (cond
     ((= incx 1) (setf m (mod n 6))
      (cond
       ((/= m 0)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf stemp
                    (+ stemp
                       (abs
                        (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%))))
           label100000))
        (cond ((< n 6) (setf sasum stemp) (go end_label)))))
      (setf mp1 (f2cl-lib:int-add m 1))
      (f2cl-lib:fdo (i mp1 (f2cl-lib:int-add i 6))
                    ((> i n) nil)
        (tagbody
          (setf stemp
                  (+ stemp
                     (abs (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%))
                     (abs
                      (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 1))
                                     ((1 *)) sx-%offset%))
                     (abs
                      (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 2))
                                     ((1 *)) sx-%offset%))
                     (abs
                      (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 3))
                                     ((1 *)) sx-%offset%))
                     (abs
                      (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 4))
                                     ((1 *)) sx-%offset%))
                     (abs
                      (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 5))
                                     ((1 *)) sx-%offset%))))
         label100001)))
     (t (setf nincx (f2cl-lib:int-mul n incx))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i incx))
                    ((> i nincx) nil)
        (tagbody
          (setf stemp
```

```
                  (+ stemp
                     (abs
                      (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%))))
          label100002))))
      (setf sasum stemp)
      (go end_label)
     end_label
      (return (values sasum nil nil nil)))))
```

———-

6.2 saxpy - constant times a vector plus a vector

6.2.1 Axiom unit tests

— **saxpy.input** —

```
)set break resume
)sys rm -f saxpy.output
)spool saxpy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

6.2.2 Axiom help page

— **saxpy.help** —

```
==========================================================================
saxpy examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
```

```
       subroutine saxpy (N, SA, SX, INCX, SY, INCY)
           SAXPY

Function/Subroutine Documentation
   subroutine saxpy (integer N, real SA, real, dimension(*) SX, integer INCX,
       real, dimension(*) SY, integer INCY)
       SAXPY

Purpose:
 SAXPY constant times a vector plus a vector.
 uses unrolled loops for increments equal to one.
```

———-

6.2.3 fortran code

— **saxpy.f** —

```
*  ======================================================================
      SUBROUTINE SAXPY(N,SA,SX,INCX,SY,INCY)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL SA
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      REAL SX(*),SY(*)
*     ..
*
*  ======================================================================
*
*     .. Local Scalars ..
      INTEGER I,IX,IY,M,MP1
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MOD
*     ..
      IF (N.LE.0) RETURN
      IF (SA.EQ.0.0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*        code for both increments equal to 1
```

```
*
*
*        clean-up loop
*
         M = MOD(N,4)
         IF (M.NE.0) THEN
            DO I = 1,M
               SY(I) = SY(I) + SA*SX(I)
            END DO
         END IF
         IF (N.LT.4) RETURN
         MP1 = M + 1
         DO I = MP1,N,4
            SY(I) = SY(I) + SA*SX(I)
            SY(I+1) = SY(I+1) + SA*SX(I+1)
            SY(I+2) = SY(I+2) + SA*SX(I+2)
            SY(I+3) = SY(I+3) + SA*SX(I+3)
         END DO
      ELSE
*
*        code for unequal increments or equal increments
*          not equal to 1
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
         IF (INCY.LT.0) IY = (-N+1)*INCY + 1
         DO I = 1,N
          SY(IY) = SY(IY) + SA*SX(IX)
          IX = IX + INCX
          IY = IY + INCY
         END DO
      END IF
      RETURN
      END
```

———-

6.2.4 lisp code

— BLAS 1 saxpy —

```
(defun saxpy (n sa sx incx sy incy)
  (declare (type (array single-float (*)) sy sx)
           (type (single-float) sa)
           (type (f2cl-lib:integer4) incy incx n))
  (f2cl-lib:with-multi-array-data
```

```
    ((sx single-float sx-%data% sx-%offset%)
     (sy single-float sy-%data% sy-%offset%))
  (prog ((i 0) (ix 0) (iy 0) (m 0) (mp1 0))
    (declare (type (f2cl-lib:integer4) mp1 m iy ix i))
    (if (<= n 0)
        (go end_label))
    (if (= sa 0.0)
        (go end_label))
    (cond
     ((and (= incx 1) (= incy 1)) (setf m (mod n 4))
      (cond
       ((/= m 0)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%)
                    (+ (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%)
                       (* sa
                          (f2cl-lib:fref sx-%data% (i) ((1 *))
                                         sx-%offset%))))
           label100000))))
      (if (< n 4)
          (go end_label))
      (setf mp1 (f2cl-lib:int-add m 1))
      (f2cl-lib:fdo (i mp1 (f2cl-lib:int-add i 4))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%)
                  (+ (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%)
                     (* sa
                        (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%))))
          (setf (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                               sy-%offset%)
                  (+
                   (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                  sy-%offset%)
                   (* sa
                      (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 1))
                                     ((1 *)) sx-%offset%))))
          (setf (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                               sy-%offset%)
                  (+
                   (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                                  sy-%offset%)
                   (* sa
                      (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 2))
                                     ((1 *)) sx-%offset%))))
          (setf (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                               sy-%offset%)
                  (+
```

```
                    (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                                   sy-%offset%)
                    (* sa
                       (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 3))
                                      ((1 *)) sx-%offset%))))
         label100001)))
      (t (setf ix 1) (setf iy 1)
       (if (< incx 0)
           (setf ix
                   (f2cl-lib:int-add
                    (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incx) 1)))
       (if (< incy 0)
           (setf iy
                   (f2cl-lib:int-add
                    (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incy) 1)))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref sy-%data% (iy) ((1 *)) sy-%offset%)
                   (+ (f2cl-lib:fref sy-%data% (iy) ((1 *)) sy-%offset%)
                      (* sa
                         (f2cl-lib:fref sx-%data% (ix) ((1 *)) sx-%offset%))))
           (setf ix (f2cl-lib:int-add ix incx))
           (setf iy (f2cl-lib:int-add iy incy))
         label100002))))
     (go end_label)
    end_label
     (return (values nil nil nil nil nil nil)))))
```

———-

6.3 scabs1 - abs(real(z))+abs(imag(z)) of a complex number

6.3.1 Axiom unit tests

— scabs1.input —

```
)set break resume
)sys rm -f scabs1.output
)spool scabs1.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

6.3.2 Axiom help page

— **scabs1.help** —

```
==========================================================================
scabs1 examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       real function scabs1 (Z)
           SCABS1

Function/Subroutine Documentation
   real function scabs1 (complex Z)
       SCABS1

Purpose:
 SCABS1 computes |Re(.)| + |Im(.)| of a complex number
```

———-

6.3.3 fortran code

— **scabs1.f** —

```
*  ==================================================================
      REAL FUNCTION SCABS1(Z)
*
*  -- Reference BLAS level1 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
```

```
      COMPLEX Z
*     ..
*
*  ===================================================================
*
*     .. Intrinsic Functions ..
      INTRINSIC ABS,AIMAG,REAL
*     ..
      SCABS1 = ABS(REAL(Z)) + ABS(AIMAG(Z))
      RETURN
      END
```

———-

6.3.4 lisp code

— BLAS 1 scabs1 —

```
(defun scabs1 (z)
  (declare (type (f2cl-lib:complex8) z))
  (prog ((scabs1 0.0))
    (declare (type (single-float) scabs1))
    (setf scabs1 (+ (abs (f2cl-lib:freal z)) (abs (f2cl-lib:aimag z))))
    (go end_label)
   end_label
    (return (values scabs1 nil))))
```

———-

6.4 scasum - abs(real(z))+abs(imag(z)) of a complex vector

6.4.1 Axiom unit tests

— scasum.input —

```
)set break resume
)sys rm -f scasum.output
)spool scasum.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

6.4.2 Axiom help page

— **scasum.help** —

```
==================================================================
scasum examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       real function scasum (N, CX, INCX)
           SCASUM

Function/Subroutine Documentation
   real function scasum (integer N, complex, dimension(*) CX, integer INCX)
       SCASUM

Purpose:
 SCASUM takes the sum of the (|Re(.)| + |Im(.)|)'s of a complex vector and
 returns a single precision result.
```

———-

6.4.3 fortran code

— **scasum.f** —

```
*  ==================================================================
      REAL FUNCTION SCASUM(N,CX,INCX)
*
*  -- Reference BLAS level1 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
```

```
*
*     .. Scalar Arguments ..
      INTEGER INCX,N
*     ..
*     .. Array Arguments ..
      COMPLEX CX(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      REAL STEMP
      INTEGER I,NINCX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC ABS,AIMAG,REAL
*     ..
      SCASUM = 0.0e0
      STEMP = 0.0e0
      IF (N.LE.0 .OR. INCX.LE.0) RETURN
      IF (INCX.EQ.1) THEN
*
*        code for increment equal to 1
*
         DO I = 1,N
            STEMP = STEMP + ABS(REAL(CX(I))) + ABS(AIMAG(CX(I)))
         END DO
      ELSE
*
*        code for increment not equal to 1
*
         NINCX = N*INCX
         DO I = 1,NINCX,INCX
            STEMP = STEMP + ABS(REAL(CX(I))) + ABS(AIMAG(CX(I)))
         END DO
      END IF
      SCASUM = STEMP
      RETURN
      END
```

———-

6.4.4 lisp code

— BLAS 1 scasum —

```
(defun scasum (n cx incx)
```

```
(declare (type (array f2cl-lib:complex8 (*)) cx)
         (type (f2cl-lib:integer4) incx n))
(f2cl-lib:with-multi-array-data
    ((cx f2cl-lib:complex8 cx-%data% cx-%offset%))
  (prog ((i 0) (nincx 0) (stemp 0.0) (scasum 0.0))
    (declare (type (single-float) scasum stemp)
             (type (f2cl-lib:integer4) nincx i))
    (setf scasum 0.0)
    (setf stemp 0.0)
    (if (or (<= n 0) (<= incx 0))
        (go end_label))
    (cond
     ((= incx 1)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf stemp
                  (+ stemp
                     (abs
                      (f2cl-lib:freal
                       (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%)))
                     (abs
                      (f2cl-lib:aimag
                       (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%)))))
         label100000)))
     (t (setf nincx (f2cl-lib:int-mul n incx))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i incx))
                    ((> i nincx) nil)
        (tagbody
          (setf stemp
                  (+ stemp
                     (abs
                      (f2cl-lib:freal
                       (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%)))
                     (abs
                      (f2cl-lib:aimag
                       (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%)))))
         label100001))))
    (setf scasum stemp)
    (go end_label)
   end_label
    (return (values scasum nil nil nil)))))
```

———-

6.5 scnrm2 - Euclidean norm of a vector

6.5.1 Axiom unit tests

— **scnrm2.input** —

```
)set break resume
)sys rm -f scnrm2.output
)spool scnrm2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

6.5.2 Axiom help page

— **scnrm2.help** —

```
========================================================================
scnrm2 examples
========================================================================

========================================================================
Man Page Details
========================================================================

SYNOPSIS
   Functions/Subroutines
       real function scnrm2 (N, X, INCX)
           SCNRM2

Function/Subroutine Documentation
   real function scnrm2 (integer N, complex, dimension(*) X, integer INCX)
       SCNRM2

Purpose:
 SCNRM2 returns the euclidean norm of a vector via the function name, so that
               SCNRM2 := sqrt( x**H*x )
```

———-

6.5.3 fortran code

— scnrm2.f —

```
*  ====================================================================
      REAL FUNCTION SCNRM2(N,X,INCX)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,N
*     ..
*     .. Array Arguments ..
      COMPLEX X(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL ONE,ZERO
      PARAMETER (ONE=1.0E+0,ZERO=0.0E+0)
*     ..
*     .. Local Scalars ..
      REAL NORM,SCALE,SSQ,TEMP
      INTEGER IX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC ABS,AIMAG,REAL,SQRT
*     ..
      IF (N.LT.1 .OR. INCX.LT.1) THEN
          NORM = ZERO
      ELSE
          SCALE = ZERO
          SSQ = ONE
*        The following loop is equivalent to this call to the LAPACK
*        auxiliary routine:
*        CALL CLASSQ( N, X, INCX, SCALE, SSQ )
*
          DO 10 IX = 1,1 + (N-1)*INCX,INCX
              IF (REAL(X(IX)).NE.ZERO) THEN
                  TEMP = ABS(REAL(X(IX)))
                  IF (SCALE.LT.TEMP) THEN
                      SSQ = ONE + SSQ* (SCALE/TEMP)**2
                      SCALE = TEMP
                  ELSE
                      SSQ = SSQ + (TEMP/SCALE)**2
```

```
                 END IF
             END IF
             IF (AIMAG(X(IX)).NE.ZERO) THEN
                 TEMP = ABS(AIMAG(X(IX)))
                 IF (SCALE.LT.TEMP) THEN
                     SSQ = ONE + SSQ* (SCALE/TEMP)**2
                     SCALE = TEMP
                 ELSE
                     SSQ = SSQ + (TEMP/SCALE)**2
                 END IF
             END IF
   10     CONTINUE
          NORM = SCALE*SQRT(SSQ)
      END IF
*
      SCNRM2 = NORM
      RETURN
*
*     End of SCNRM2.
*
      END
```

———-

6.5.4 lisp code

— BLAS 1 scnrm2 —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun scnrm2 (n x incx)
    (declare (type (array f2cl-lib:complex8 (*)) x)
             (type (f2cl-lib:integer4) incx n))
    (f2cl-lib:with-multi-array-data
        ((x f2cl-lib:complex8 x-%data% x-%offset%))
      (prog ((ix 0) (norm 0.0) (scale 0.0) (ssq 0.0) (temp 0.0) (scnrm2 0.0))
        (declare (type (f2cl-lib:integer4) ix)
                 (type (single-float) norm scale ssq temp scnrm2))
        (cond ((or (< n 1) (< incx 1)) (setf norm zero))
              (t (setf scale zero) (setf ssq one)
               (f2cl-lib:fdo (ix 1 (f2cl-lib:int-add ix incx))
                             ((> ix
                                 (f2cl-lib:int-add 1
                                                   (f2cl-lib:int-mul
                                                    (f2cl-lib:int-add n
```

```
                                                                    (f2cl-lib:int-sub
                                                                     1))
                                                incx)))
                          nil)
                (tagbody
                  (cond
                   ((/= (f2cl-lib:freal (f2cl-lib:fref x (ix) ((1 *)))) zero)
                    (setf temp
                            (abs
                             (f2cl-lib:freal
                              (f2cl-lib:fref x-%data% (ix) ((1 *))
                                             x-%offset%))))
                    (cond
                     ((< scale temp)
                      (setf ssq (+ one (* ssq (expt (/ scale temp) 2))))
                      (setf scale temp))
                     (t (setf ssq (+ ssq (expt (/ temp scale) 2)))))))
                  (cond
                   ((/= (f2cl-lib:aimag (f2cl-lib:fref x (ix) ((1 *)))) zero)
                    (setf temp
                            (abs
                             (f2cl-lib:aimag
                              (f2cl-lib:fref x-%data% (ix) ((1 *))
                                             x-%offset%))))
                    (cond
                     ((< scale temp)
                      (setf ssq (+ one (* ssq (expt (/ scale temp) 2))))
                      (setf scale temp))
                     (t (setf ssq (+ ssq (expt (/ temp scale) 2)))))))
                 label10))
               (setf norm (* scale (f2cl-lib:fsqrt ssq)))))
        (setf scnrm2 norm)
        (go end_label)
       end_label
        (return (values scnrm2 nil nil nil))))))
```

———-

6.6 scopy - copy a vector x to a vector y

6.6.1 Axiom unit tests

— **scopy.input** —

```
)set break resume
)sys rm -f scopy.output
```

```
)spool scopy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

6.6.2 Axiom help page

— **scopy.help** —

```
=====================================================================
scopy examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine scopy (N, SX, INCX, SY, INCY)
           SCOPY

Function/Subroutine Documentation
   subroutine scopy (integer N, real, dimension(*) SX, integer INCX, real,
       dimension(*) SY, integer INCY)
       SCOPY

Purpose:
 SCOPY copies a vector, x, to a vector, y.
 uses unrolled loops for increments equal to 1.
```

———-

6.6.3 fortran code

— **scopy.f** —

```
*  ====================================================================
      SUBROUTINE SCOPY(N,SX,INCX,SY,INCY)
```

```
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      REAL SX(*),SY(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER I,IX,IY,M,MP1
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MOD
*     ..
      IF (N.LE.0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*        code for both increments equal to 1
*
*
*        clean-up loop
*
         M = MOD(N,7)
         IF (M.NE.0) THEN
            DO I = 1,M
               SY(I) = SX(I)
            END DO
            IF (N.LT.7) RETURN
         END IF
         MP1 = M + 1
         DO I = MP1,N,7
            SY(I) = SX(I)
            SY(I+1) = SX(I+1)
            SY(I+2) = SX(I+2)
            SY(I+3) = SX(I+3)
            SY(I+4) = SX(I+4)
            SY(I+5) = SX(I+5)
            SY(I+6) = SX(I+6)
         END DO
      ELSE
*
*        code for unequal increments or equal increments
*          not equal to 1
```

```
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
         IF (INCY.LT.0) IY = (-N+1)*INCY + 1
         DO I = 1,N
            SY(IY) = SX(IX)
            IX = IX + INCX
            IY = IY + INCY
         END DO
      END IF
      RETURN
      END
```

———-

6.6.4 lisp code

— BLAS 1 scopy —

```
(defun scopy (n sx incx sy incy)
  (declare (type (array single-float (*)) sy sx)
           (type (f2cl-lib:integer4) incy incx n))
  (f2cl-lib:with-multi-array-data
      ((sx single-float sx-%data% sx-%offset%)
       (sy single-float sy-%data% sy-%offset%))
    (prog ((i 0) (ix 0) (iy 0) (m 0) (mp1 0))
      (declare (type (f2cl-lib:integer4) mp1 m iy ix i))
      (if (<= n 0)
          (go end_label))
      (cond
       ((and (= incx 1) (= incy 1)) (setf m (mod n 7))
        (cond
         ((/= m 0)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%)
                      (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%))
             label100000))
          (if (< n 7)
              (go end_label))))
        (setf mp1 (f2cl-lib:int-add m 1))
        (f2cl-lib:fdo (i mp1 (f2cl-lib:int-add i 7))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%)
```

```
                (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%))
        (setf (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                             sy-%offset%)
                (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                               sx-%offset%))
        (setf (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                             sy-%offset%)
                (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                               sx-%offset%))
        (setf (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                             sy-%offset%)
                (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                               sx-%offset%))
        (setf (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 4)) ((1 *))
                             sy-%offset%)
                (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 4)) ((1 *))
                               sx-%offset%))
        (setf (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 5)) ((1 *))
                             sy-%offset%)
                (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 5)) ((1 *))
                               sx-%offset%))
        (setf (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 6)) ((1 *))
                             sy-%offset%)
                (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 6)) ((1 *))
                               sx-%offset%))
       label100001)))
   (t (setf ix 1) (setf iy 1)
    (if (< incx 0)
        (setf ix
                (f2cl-lib:int-add
                 (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incx) 1)))
    (if (< incy 0)
        (setf iy
                (f2cl-lib:int-add
                 (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incy) 1)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref sy-%data% (iy) ((1 *)) sy-%offset%)
                (f2cl-lib:fref sx-%data% (ix) ((1 *)) sx-%offset%))
        (setf ix (f2cl-lib:int-add ix incx))
        (setf iy (f2cl-lib:int-add iy incy))
       label100002))))
  (go end_label)
 end_label
  (return (values nil nil nil nil nil)))))
```

———-

6.7 sdot - compute the dot product of two vectors

6.7.1 Axiom unit tests

— sdot.input —

```
)set break resume
)sys rm -f sdot.output
)spool sdot.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

6.7.2 Axiom help page

— sdot.help —

```
======================================================================
sdot examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       real function sdsdot (N, SB, SX, INCX, SY, INCY)
           SDSDOT

Function/Subroutine Documentation
   real function sdsdot (integer N, real SB, real, dimension(*) SX, integer
       INCX, real, dimension(*) SY, integer INCY)
       SDSDOT

Purpose:
 compute the dot product of two vectors
```

———-

6.7.3 fortran code

— sdot.f —

```
*  ====================================================================
      REAL FUNCTION SDOT(N,SX,INCX,SY,INCY)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      REAL SX(*),SY(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      REAL STEMP
      INTEGER I,IX,IY,M,MP1
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MOD
*     ..
      STEMP = 0.0e0
      SDOT = 0.0e0
      IF (N.LE.0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*        code for both increments equal to 1
*
*
*        clean-up loop
*
         M = MOD(N,5)
         IF (M.NE.0) THEN
            DO I = 1,M
               STEMP = STEMP + SX(I)*SY(I)
            END DO
            IF (N.LT.5) THEN
               SDOT=STEMP
            RETURN
            END IF
         END IF
         MP1 = M + 1
```

```
         DO I = MP1,N,5
          STEMP = STEMP + SX(I)*SY(I) + SX(I+1)*SY(I+1) +
     $            SX(I+2)*SY(I+2) + SX(I+3)*SY(I+3) + SX(I+4)*SY(I+4)
         END DO
      ELSE
*
*        code for unequal increments or equal increments
*          not equal to 1
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
         IF (INCY.LT.0) IY = (-N+1)*INCY + 1
         DO I = 1,N
            STEMP = STEMP + SX(IX)*SY(IY)
            IX = IX + INCX
            IY = IY + INCY
         END DO
      END IF
      SDOT = STEMP
      RETURN
      END
```

———-

6.7.4 lisp code

— BLAS 1 sdot —

```
(defun sdot (n sx incx sy incy)
  (declare (type (array single-float (*)) sy sx)
           (type (f2cl-lib:integer4) incy incx n))
  (f2cl-lib:with-multi-array-data
      ((sx single-float sx-%data% sx-%offset%)
       (sy single-float sy-%data% sy-%offset%))
    (prog ((i 0) (ix 0) (iy 0) (m 0) (mp1 0) (stemp 0.0) (sdot 0.0))
      (declare (type (single-float) sdot stemp)
               (type (f2cl-lib:integer4) mp1 m iy ix i))
      (setf stemp 0.0)
      (setf sdot 0.0)
      (if (<= n 0)
          (go end_label))
      (cond
       ((and (= incx 1) (= incy 1)) (setf m (mod n 5))
        (cond
         ((/= m 0)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                 ((> i m) nil)
      (tagbody
        (setf stemp
                (+ stemp
                   (* (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%)
                      (f2cl-lib:fref sy-%data% (i) ((1 *))
                                     sy-%offset%))))
       label100000))
    (cond ((< n 5) (setf sdot stemp) (go end_label)))))
  (setf mp1 (f2cl-lib:int-add m 1))
  (f2cl-lib:fdo (i mp1 (f2cl-lib:int-add i 5))
                ((> i n) nil)
    (tagbody
      (setf stemp
              (+ stemp
                 (* (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%)
                    (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%))
                 (*
                  (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) sx-%offset%)
                  (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) sy-%offset%))
                 (*
                  (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 2))
                                 ((1 *)) sx-%offset%)
                  (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 2))
                                 ((1 *)) sy-%offset%))
                 (*
                  (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 3))
                                 ((1 *)) sx-%offset%)
                  (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 3))
                                 ((1 *)) sy-%offset%))
                 (*
                  (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 4))
                                 ((1 *)) sx-%offset%)
                  (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 4))
                                 ((1 *)) sy-%offset%))))
     label100001)))
(t (setf ix 1) (setf iy 1)
 (if (< incx 0)
     (setf ix
             (f2cl-lib:int-add
              (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incx) 1)))
 (if (< incy 0)
     (setf iy
             (f2cl-lib:int-add
              (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incy) 1)))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
```

```
            (setf stemp
                    (+ stemp
                       (* (f2cl-lib:fref sx-%data% (ix) ((1 *)) sx-%offset%)
                          (f2cl-lib:fref sy-%data% (iy) ((1 *)) sy-%offset%))))
            (setf ix (f2cl-lib:int-add ix incx))
            (setf iy (f2cl-lib:int-add iy incy))
           label100002))))
      (setf sdot stemp)
      (go end_label)
     end_label
      (return (values sdot nil nil nil nil nil)))))
```

———-

6.8 sdsdot - compute the inner product of two vectors

6.8.1 Axiom unit tests

— **sdsdot.input** —

```
)set break resume
)sys rm -f sdsdot.output
)spool sdsdot.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

6.8.2 Axiom help page

— **sdsdot.help** —

```
==========================================================================
sdsdot examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       real function sdsdot (N, SB, SX, INCX, SY, INCY)
           SDSDOT

Function/Subroutine Documentation
   real function sdsdot (integer N, real SB, real, dimension(*) SX, integer
       INCX, real, dimension(*) SY, integer INCY)
       SDSDOT

Purpose:
 compute the inner product of two vectors
```

———-

6.8.3 fortran code

— sdsdot.f —

```
*  ====================================================================
      REAL FUNCTION SDSDOT(N,SB,SX,INCX,SY,INCY)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL SB
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      REAL SX(*),SY(*)
*     ..
*
*  PURPOSE
*  =======
*
*  Compute the inner product of two vectors with extended
*  precision accumulation.
*
*  Returns S.P. result with dot product accumulated in D.P.
*  SDSDOT = SB + sum for I = 0 to N-1 of SX(LX+I*INCX)*SY(LY+I*INCY),
*  where LX = 1 if INCX .GE. 0, else LX = 1+(1-N)*INCX, and LY is
*  defined in a similar way using INCY.
*
```

```
*  ARGUMENTS
*  =========
*
*  N      (input) INTEGER
*         number of elements in input vector(s)
*
*  SB     (input) REAL
*         single precision scalar to be added to inner product
*
*  SX     (input) REAL array, dimension (N)
*         single precision vector with N elements
*
*  INCX   (input) INTEGER
*         storage spacing between elements of SX
*
*  SY     (input) REAL array, dimension (N)
*         single precision vector with N elements
*
*  INCY   (input) INTEGER
*         storage spacing between elements of SY
*
*  SDSDOT (output) REAL
*         single precision dot product (SB if N .LE. 0)
*
*  Further Details
*  ===============
*
*  REFERENCES
*
*  C. L. Lawson, R. J. Hanson, D. R. Kincaid and F. T.
*  Krogh, Basic linear algebra subprograms for Fortran
*  usage, Algorithm No. 539, Transactions on Mathematical
*  Software 5, 3 (September 1979), pp. 308-323.
*
*  REVISION HISTORY  (YYMMDD)
*
*  791001  DATE WRITTEN
*  890531  Changed all specific intrinsics to generic.  (WRB)
*  890831  Modified array declarations.  (WRB)
*  890831  REVISION DATE from Version 3.2
*  891214  Prologue converted to Version 4.0 format.  (BAB)
*  920310  Corrected definition of LX in DESCRIPTION.  (WRB)
*  920501  Reformatted the REFERENCES section.  (WRB)
*  070118  Reformat to LAPACK coding style
*
*  ======================================================================
*
*     .. Local Scalars ..
      DOUBLE PRECISION DSDOT
      INTEGER I,KX,KY,NS
```

```
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DBLE
*     ..
      DSDOT = SB
      IF (N.LE.0) THEN
         SDSDOT = DSDOT
         RETURN
      END IF
      IF (INCX.EQ.INCY .AND. INCX.GT.0) THEN
*
*     Code for equal and positive increments.
*
         NS = N*INCX
         DO I = 1,NS,INCX
            DSDOT = DSDOT + DBLE(SX(I))*DBLE(SY(I))
         END DO
      ELSE
*
*     Code for unequal or nonpositive increments.
*
         KX = 1
         KY = 1
         IF (INCX.LT.0) KX = 1 + (1-N)*INCX
         IF (INCY.LT.0) KY = 1 + (1-N)*INCY
         DO I = 1,N
            DSDOT = DSDOT + DBLE(SX(KX))*DBLE(SY(KY))
            KX = KX + INCX
            KY = KY + INCY
         END DO
      END IF
      SDSDOT = DSDOT
      RETURN
      END
```

———-

6.8.4 lisp code

— BLAS 1 sdsdot —

```
(defun sdsdot (n sb sx incx sy incy)
  (declare (type (array single-float (*)) sy sx)
           (type (single-float) sb)
           (type (f2cl-lib:integer4) incy incx n))
  (f2cl-lib:with-multi-array-data
      ((sx single-float sx-%data% sx-%offset%)
```

```
    (sy single-float sy-%data% sy-%offset%))
  (prog ((i 0) (kx 0) (ky 0) (ns 0) (dsdot 0.0d0) (sdsdot 0.0))
    (declare (type (single-float) sdsdot)
             (type (double-float) dsdot)
             (type (f2cl-lib:integer4) ns ky kx i))
    (setf dsdot (coerce sb 'double-float))
    (cond
     ((<= n 0) (setf sdsdot (coerce dsdot 'single-float)) (go end_label)))
    (cond
     ((and (= incx incy) (> incx 0)) (setf ns (f2cl-lib:int-mul n incx))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i incx))
                    ((> i ns) nil)
        (tagbody
          (setf dsdot
                  (+ dsdot
                     (*
                      (f2cl-lib:dble
                       (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%))
                      (f2cl-lib:dble
                       (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%)))))
         label100000)))
     (t (setf kx 1) (setf ky 1)
      (if (< incx 0)
          (setf kx
                  (f2cl-lib:int-add 1
                                    (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n)
                                                      incx))))
      (if (< incy 0)
          (setf ky
                  (f2cl-lib:int-add 1
                                    (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n)
                                                      incy))))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf dsdot
                  (+ dsdot
                     (*
                      (f2cl-lib:dble
                       (f2cl-lib:fref sx-%data% (kx) ((1 *)) sx-%offset%))
                      (f2cl-lib:dble
                       (f2cl-lib:fref sy-%data% (ky) ((1 *)) sy-%offset%)))))
          (setf kx (f2cl-lib:int-add kx incx))
          (setf ky (f2cl-lib:int-add ky incy))
         label100001))))
    (setf sdsdot (coerce dsdot 'single-float))
    (go end_label)
   end_label
    (return (values sdsdot nil nil nil nil nil nil)))))
```

6.9 snrm2 - Euclidean norm of a vector $\sqrt{x'x}$

6.9.1 Axiom unit tests

— snrm2.input —

```
)set break resume
)sys rm -f snrm2.output
)spool snrm2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

6.9.2 Axiom help page

— snrm2.help —

```
======================================================================
snrm2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       real function snrm2 (N, X, INCX)
           SNRM2

Function/Subroutine Documentation
   real function snrm2 (integer N, real, dimension(*) X, integer INCX)
       SNRM2

Purpose:
 SNRM2 returns the euclidean norm of a vector via the function name, so that
```

```
SNRM2 := sqrt( x'*x ).
```

———-

6.9.3 fortran code

— **snrm2.f** —

```
*  ====================================================================
      REAL FUNCTION SNRM2(N,X,INCX)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,N
*     ..
*     .. Array Arguments ..
      REAL X(*)
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
      REAL ONE,ZERO
      PARAMETER (ONE=1.0E+0,ZERO=0.0E+0)
*     ..
*     .. Local Scalars ..
      REAL ABSXI,NORM,SCALE,SSQ
      INTEGER IX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC ABS,SQRT
*     ..
      IF (N.LT.1 .OR. INCX.LT.1) THEN
          NORM = ZERO
      ELSE IF (N.EQ.1) THEN
          NORM = ABS(X(1))
      ELSE
          SCALE = ZERO
          SSQ = ONE
*        The following loop is equivalent to this call to the LAPACK
*        auxiliary routine:
*        CALL SLASSQ( N, X, INCX, SCALE, SSQ )
*
```

```
         DO 10 IX = 1,1 + (N-1)*INCX,INCX
             IF (X(IX).NE.ZERO) THEN
                 ABSXI = ABS(X(IX))
                 IF (SCALE.LT.ABSXI) THEN
                     SSQ = ONE + SSQ* (SCALE/ABSXI)**2
                     SCALE = ABSXI
                 ELSE
                     SSQ = SSQ + (ABSXI/SCALE)**2
                 END IF
             END IF
   10    CONTINUE
         NORM = SCALE*SQRT(SSQ)
      END IF
*
      SNRM2 = NORM
      RETURN
*
*     End of SNRM2.
*
      END
```

———-

6.9.4 lisp code

— BLAS 1 snrm2 —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun snrm2 (n x incx)
    (declare (type (array single-float (*)) x)
             (type (f2cl-lib:integer4) incx n))
    (f2cl-lib:with-multi-array-data
        ((x single-float x-%data% x-%offset%))
      (prog ((ix 0) (absxi 0.0) (norm 0.0) (scale 0.0) (ssq 0.0) (snrm2 0.0))
        (declare (type (f2cl-lib:integer4) ix)
                 (type (single-float) absxi norm scale ssq snrm2))
        (cond ((or (< n 1) (< incx 1)) (setf norm zero))
              ((= n 1)
               (setf norm
                       (abs (f2cl-lib:fref x-%data% (1) ((1 *)) x-%offset%))))
              (t (setf scale zero) (setf ssq one)
               (f2cl-lib:fdo (ix 1 (f2cl-lib:int-add ix incx))
                             ((> ix
                                 (f2cl-lib:int-add 1
```

```
                                                (f2cl-lib:int-mul
                                                 (f2cl-lib:int-add n
                                                                   (f2cl-lib:int-sub
                                                                    1))
                                                 incx)))
                          nil)
                (tagbody
                  (cond
                   ((/= (f2cl-lib:fref x (ix) ((1 *))) zero)
                    (setf absxi
                            (abs
                             (f2cl-lib:fref x-%data% (ix) ((1 *))
                                            x-%offset%)))
                    (cond
                     ((< scale absxi)
                      (setf ssq (+ one (* ssq (expt (/ scale absxi) 2))))
                      (setf scale absxi))
                     (t (setf ssq (+ ssq (expt (/ absxi scale) 2)))))))
                 label10))
              (setf norm (* scale (f2cl-lib:fsqrt ssq)))))
        (setf snrm2 norm)
        (go end_label)
       end_label
        (return (values snrm2 nil nil nil))))))
```

———-

6.10 srot - applies a plane rotation

6.10.1 Axiom unit tests

— **srot.input** —

```
)set break resume
)sys rm -f srot.output
)spool srot.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

6.10.2 Axiom help page

— srot.help —

```
=====================================================================
srot examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine srot (N, SX, INCX, SY, INCY, C, S)
           SROT

Function/Subroutine Documentation
   subroutine srot (integer N, real, dimension(*) SX, integer INCX, real,
       dimension(*) SY, integer INCY, real C, real S)
       SROT

Purpose:
 applies a plane rotation.
```

———-

6.10.3 fortran code

— srot.f —

```
*  ====================================================================
      SUBROUTINE SROT(N,SX,INCX,SY,INCY,C,S)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL C,S
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      REAL SX(*),SY(*)
```

```
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      REAL STEMP
      INTEGER I,IX,IY
*     ..
      IF (N.LE.0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*       code for both increments equal to 1
*
         DO I = 1,N
            STEMP = C*SX(I) + S*SY(I)
            SY(I) = C*SY(I) - S*SX(I)
            SX(I) = STEMP
         END DO
      ELSE
*
*       code for unequal increments or equal increments not equal
*         to 1
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
         IF (INCY.LT.0) IY = (-N+1)*INCY + 1
         DO I = 1,N
            STEMP = C*SX(IX) + S*SY(IY)
            SY(IY) = C*SY(IY) - S*SX(IX)
            SX(IX) = STEMP
            IX = IX + INCX
            IY = IY + INCY
         END DO
      END IF
      RETURN
      END
```

———-

6.10.4 lisp code

— BLAS 1 srot —

```
(defun srot (n sx incx sy incy c s)
  (declare (type (single-float) s c)
           (type (array single-float (*)) sy sx)
```

```
          (type (f2cl-lib:integer4) incy incx n))
  (f2cl-lib:with-multi-array-data
      ((sx single-float sx-%data% sx-%offset%)
       (sy single-float sy-%data% sy-%offset%))
    (prog ((i 0) (ix 0) (iy 0) (stemp 0.0))
      (declare (type (single-float) stemp)
               (type (f2cl-lib:integer4) iy ix i))
      (if (<= n 0)
          (go end_label))
      (cond
       ((and (= incx 1) (= incy 1))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf stemp
                    (+ (* c (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%))
                       (* s
                          (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%))))
            (setf (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%)
                    (- (* c (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%))
                       (* s
                          (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%))))
            (setf (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%) stemp)
           label100000)))
       (t (setf ix 1) (setf iy 1)
        (if (< incx 0)
            (setf ix
                    (f2cl-lib:int-add
                     (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incx) 1)))
        (if (< incy 0)
            (setf iy
                    (f2cl-lib:int-add
                     (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incy) 1)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf stemp
                    (+ (* c (f2cl-lib:fref sx-%data% (ix) ((1 *)) sx-%offset%))
                       (* s
                          (f2cl-lib:fref sy-%data% (iy) ((1 *)) sy-%offset%))))
            (setf (f2cl-lib:fref sy-%data% (iy) ((1 *)) sy-%offset%)
                    (- (* c (f2cl-lib:fref sy-%data% (iy) ((1 *)) sy-%offset%))
                       (* s
                          (f2cl-lib:fref sx-%data% (ix) ((1 *)) sx-%offset%))))
            (setf (f2cl-lib:fref sx-%data% (ix) ((1 *)) sx-%offset%) stemp)
            (setf ix (f2cl-lib:int-add ix incx))
            (setf iy (f2cl-lib:int-add iy incy))
           label100001))))
      (go end_label)
     end_label
```

```
      (return (values nil nil nil nil nil nil nil)))))
```

———-

6.11 srotg - construct givens plane rotation

6.11.1 Axiom unit tests

— **srotg.input** —

```
)set break resume
)sys rm -f srotg.output
)spool srotg.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

6.11.2 Axiom help page

— **srotg.help** —

```
======================================================================
srotg examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine srotg (SA, SB, C, S)
           SROTG

Function/Subroutine Documentation
   subroutine srotg (real SA, real SB, real C, real S)
       SROTG
```

```
Purpose:
 SROTG construct givens plane rotation.
```

6.11.3 fortran code

— srotg.f —

```
*  ==================================================================
      SUBROUTINE SROTG(SA,SB,C,S)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL C,S,SA,SB
*     ..
*
*  ==================================================================
*
*     .. Local Scalars ..
      REAL R,ROE,SCALE,Z
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC ABS,SIGN,SQRT
*     ..
      ROE = SB
      IF (ABS(SA).GT.ABS(SB)) ROE = SA
      SCALE = ABS(SA) + ABS(SB)
      IF (SCALE.EQ.0.0) THEN
         C = 1.0
         S = 0.0
         R = 0.0
         Z = 0.0
      ELSE
         R = SCALE*SQRT((SA/SCALE)**2+ (SB/SCALE)**2)
         R = SIGN(1.0,ROE)*R
         C = SA/R
         S = SB/R
         Z = 1.0
         IF (ABS(SA).GT.ABS(SB)) Z = S
         IF (ABS(SB).GE.ABS(SA) .AND. C.NE.0.0) Z = 1.0/C
      END IF
      SA = R
```

```
      SB = Z
      RETURN
      END
```

———-

6.11.4 lisp code

— BLAS 1 srotg —

```
(defun srotg (sa sb c s)
  (declare (type (single-float) s c sb sa))
  (prog ((r 0.0) (roe 0.0) (scale 0.0) (z 0.0))
    (declare (type (single-float) z scale roe r))
    (setf roe sb)
    (if (> (abs sa) (abs sb))
        (setf roe sa))
    (setf scale (+ (abs sa) (abs sb)))
    (cond ((= scale 0.0) (setf c 1.0) (setf s 0.0) (setf r 0.0) (setf z 0.0))
          (t
           (setf r
                   (* scale
                      (f2cl-lib:fsqrt
                       (+ (expt (/ sa scale) 2) (expt (/ sb scale) 2)))))
           (setf r (* (f2cl-lib:sign 1.0 roe) r)) (setf c (/ sa r))
           (setf s (/ sb r)) (setf z 1.0)
           (if (> (abs sa) (abs sb))
               (setf z s))
           (if (and (>= (abs sb) (abs sa)) (/= c 0.0))
               (setf z (/ 1.0 c)))))
    (setf sa r)
    (setf sb z)
    (go end_label)
   end_label
    (return (values sa sb c s))))
```

———-

6.12 srotm - apply a modified givens transformation

6.12.1 Axiom unit tests

— srotm.input —

```
)set break resume
)sys rm -f srotm.output
)spool srotm.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

6.12.2 Axiom help page

— srotm.help —

```
======================================================================
srotm examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine srotm (N, SX, INCX, SY, INCY, SPARAM)
           SROTM

Function/Subroutine Documentation
   subroutine srotm (integer N, real, dimension(*) SX, integer INCX, real,
       dimension(*) SY, integer INCY, real, dimension(5) SPARAM)
       SROTM

Purpose:
 Apply the modified Givens transformation, H, to the 2 by N matrix
 (SX**T) , where **T indicates transpose. the elements of SX are in
 (SX**T)

 SX(LX+I*INCX), I = 0 TO N-1, where LX = 1 IF INCX .GE. 0, ELSE
 LX = (-INCX)*N, and similarly for sy using LY and INCY.
 with SPARAM(1)=SFLAG, H has one of the following forms..

 SFLAG=-1.E0     SFLAG=0.E0        SFLAG=1.E0     SFLAG=-2.E0

   (SH11  SH12)    (1.E0  SH12)    (SH11  1.E0)    (1.E0  0.E0)
 H=(          )    (          )    (          )    (          )
   (SH21  SH22),   (SH21  1.E0),   (-1.E0 SH22),   (0.E0  1.E0).
```

```
 See SROTMG for a description of data storage in SPARAM.

Parameters:
N
  N is INTEGER
  number of elements in input vector(s)

SX
  SX is REAL array, dimension N
  double precision vector with N elements

INCX
  INCX is INTEGER
  storage spacing between elements of SX

SY
  SY is REAL array, dimension N
  double precision vector with N elements

INCY
  INCY is INTEGER
  storage spacing between elements of SY

SPARAM
  SPARAM is REAL array, dimension 5
    SPARAM(1)=SFLAG
    SPARAM(2)=SH11
    SPARAM(3)=SH21
    SPARAM(4)=SH12
    SPARAM(5)=SH22
```

6.12.3 fortran code

— srotm.f —

```
*  ===================================================================
      SUBROUTINE SROTM(N,SX,INCX,SY,INCY,SPARAM)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,INCY,N
```

```
*     ..
*     .. Array Arguments ..
      REAL SPARAM(5),SX(*),SY(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      REAL SFLAG,SH11,SH12,SH21,SH22,TWO,W,Z,ZERO
      INTEGER I,KX,KY,NSTEPS
*     ..
*     .. Data statements ..
      DATA ZERO,TWO/0.E0,2.E0/
*     ..
*
      SFLAG = SPARAM(1)
      IF (N.LE.0 .OR. (SFLAG+TWO.EQ.ZERO)) RETURN
      IF (INCX.EQ.INCY.AND.INCX.GT.0) THEN
*
         NSTEPS = N*INCX
         IF (SFLAG.LT.ZERO) THEN
            SH11 = SPARAM(2)
            SH12 = SPARAM(4)
            SH21 = SPARAM(3)
            SH22 = SPARAM(5)
            DO I = 1,NSTEPS,INCX
               W = SX(I)
               Z = SY(I)
               SX(I) = W*SH11 + Z*SH12
               SY(I) = W*SH21 + Z*SH22
            END DO
         ELSE IF (SFLAG.EQ.ZERO) THEN
            SH12 = SPARAM(4)
            SH21 = SPARAM(3)
            DO I = 1,NSTEPS,INCX
               W = SX(I)
               Z = SY(I)
               SX(I) = W + Z*SH12
               SY(I) = W*SH21 + Z
            END DO
         ELSE
            SH11 = SPARAM(2)
            SH22 = SPARAM(5)
            DO I = 1,NSTEPS,INCX
               W = SX(I)
               Z = SY(I)
               SX(I) = W*SH11 + Z
               SY(I) = -W + SH22*Z
            END DO
         END IF
```

```
      ELSE
         KX = 1
         KY = 1
         IF (INCX.LT.0) KX = 1 + (1-N)*INCX
         IF (INCY.LT.0) KY = 1 + (1-N)*INCY
*
         IF (SFLAG.LT.ZERO) THEN
            SH11 = SPARAM(2)
            SH12 = SPARAM(4)
            SH21 = SPARAM(3)
            SH22 = SPARAM(5)
            DO I = 1,N
               W = SX(KX)
               Z = SY(KY)
               SX(KX) = W*SH11 + Z*SH12
               SY(KY) = W*SH21 + Z*SH22
               KX = KX + INCX
               KY = KY + INCY
            END DO
         ELSE IF (SFLAG.EQ.ZERO) THEN
            SH12 = SPARAM(4)
            SH21 = SPARAM(3)
            DO I = 1,N
               W = SX(KX)
               Z = SY(KY)
               SX(KX) = W + Z*SH12
               SY(KY) = W*SH21 + Z
               KX = KX + INCX
               KY = KY + INCY
            END DO
         ELSE
             SH11 = SPARAM(2)
             SH22 = SPARAM(5)
             DO I = 1,N
                W = SX(KX)
                Z = SY(KY)
                SX(KX) = W*SH11 + Z
                SY(KY) = -W + SH22*Z
                KX = KX + INCX
                KY = KY + INCY
            END DO
         END IF
      END IF
      RETURN
      END
```

———-

6.12.4 lisp code

— BLAS 1 srotm —

```
(let ((zero 0.0) (two 2.0))
  (declare (type (single-float) zero two))
  (defun srotm (n sx incx sy incy sparam)
    (declare (type (array single-float (*)) sparam)
             (type (array single-float (*)) sy sx)
             (type (f2cl-lib:integer4) incy incx n))
    (f2cl-lib:with-multi-array-data
        ((sx single-float sx-%data% sx-%offset%)
         (sy single-float sy-%data% sy-%offset%)
         (sparam single-float sparam-%data% sparam-%offset%))
      (prog ((i 0) (kx 0) (ky 0) (nsteps 0) (sflag 0.0) (sh11 0.0) (sh12 0.0)
             (sh21 0.0) (sh22 0.0) (w 0.0) (z 0.0))
        (declare (type (single-float) z w sh22 sh21 sh12 sh11 sflag)
                 (type (f2cl-lib:integer4) nsteps ky kx i))
        (setf sflag (f2cl-lib:fref sparam-%data% (1) ((1 5)) sparam-%offset%))
        (if (or (<= n 0) (= (+ sflag two) zero))
            (go end_label))
        (cond
         ((and (= incx incy) (> incx 0))
          (setf nsteps (f2cl-lib:int-mul n incx))
          (cond
           ((< sflag zero)
            (setf sh11
                    (f2cl-lib:fref sparam-%data% (2) ((1 5)) sparam-%offset%))
            (setf sh12
                    (f2cl-lib:fref sparam-%data% (4) ((1 5)) sparam-%offset%))
            (setf sh21
                    (f2cl-lib:fref sparam-%data% (3) ((1 5)) sparam-%offset%))
            (setf sh22
                    (f2cl-lib:fref sparam-%data% (5) ((1 5)) sparam-%offset%))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i incx))
                          ((> i nsteps) nil)
              (tagbody
                (setf w (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%))
                (setf z (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%))
                (setf (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%)
                        (+ (* w sh11) (* z sh12)))
                (setf (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%)
                        (+ (* w sh21) (* z sh22)))
               label100000)))
           ((= sflag zero)
            (setf sh12
                    (f2cl-lib:fref sparam-%data% (4) ((1 5)) sparam-%offset%))
            (setf sh21
                    (f2cl-lib:fref sparam-%data% (3) ((1 5)) sparam-%offset%))
```

```
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i incx))
                    ((> i nsteps) nil)
        (tagbody
          (setf w (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%))
          (setf z (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%))
          (setf (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%)
                  (+ w (* z sh12)))
          (setf (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%)
                  (+ (* w sh21) z))
       label100001)))
     (t
      (setf sh11
              (f2cl-lib:fref sparam-%data% (2) ((1 5)) sparam-%offset%))
      (setf sh22
              (f2cl-lib:fref sparam-%data% (5) ((1 5)) sparam-%offset%))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i incx))
                    ((> i nsteps) nil)
        (tagbody
          (setf w (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%))
          (setf z (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%))
          (setf (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%)
                  (+ (* w sh11) z))
          (setf (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%)
                  (- (* sh22 z) w))
       label100002)))))
   (t (setf kx 1) (setf ky 1)
    (if (< incx 0)
        (setf kx
                (f2cl-lib:int-add 1
                                  (f2cl-lib:int-mul
                                   (f2cl-lib:int-sub 1 n) incx))))
    (if (< incy 0)
        (setf ky
                (f2cl-lib:int-add 1
                                  (f2cl-lib:int-mul
                                   (f2cl-lib:int-sub 1 n) incy))))
    (cond
     ((< sflag zero)
      (setf sh11
              (f2cl-lib:fref sparam-%data% (2) ((1 5)) sparam-%offset%))
      (setf sh12
              (f2cl-lib:fref sparam-%data% (4) ((1 5)) sparam-%offset%))
      (setf sh21
              (f2cl-lib:fref sparam-%data% (3) ((1 5)) sparam-%offset%))
      (setf sh22
              (f2cl-lib:fref sparam-%data% (5) ((1 5)) sparam-%offset%))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf w (f2cl-lib:fref sx-%data% (kx) ((1 *)) sx-%offset%))
```

```
            (setf z (f2cl-lib:fref sy-%data% (ky) ((1 *)) sy-%offset%))
            (setf (f2cl-lib:fref sx-%data% (kx) ((1 *)) sx-%offset%)
                    (+ (* w sh11) (* z sh12)))
            (setf (f2cl-lib:fref sy-%data% (ky) ((1 *)) sy-%offset%)
                    (+ (* w sh21) (* z sh22)))
            (setf kx (f2cl-lib:int-add kx incx))
            (setf ky (f2cl-lib:int-add ky incy))
           label100003)))
        ((= sflag zero)
         (setf sh12
                 (f2cl-lib:fref sparam-%data% (4) ((1 5)) sparam-%offset%))
         (setf sh21
                 (f2cl-lib:fref sparam-%data% (3) ((1 5)) sparam-%offset%))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
            (setf w (f2cl-lib:fref sx-%data% (kx) ((1 *)) sx-%offset%))
            (setf z (f2cl-lib:fref sy-%data% (ky) ((1 *)) sy-%offset%))
            (setf (f2cl-lib:fref sx-%data% (kx) ((1 *)) sx-%offset%)
                    (+ w (* z sh12)))
            (setf (f2cl-lib:fref sy-%data% (ky) ((1 *)) sy-%offset%)
                    (+ (* w sh21) z))
            (setf kx (f2cl-lib:int-add kx incx))
            (setf ky (f2cl-lib:int-add ky incy))
           label100004)))
        (t
         (setf sh11
                 (f2cl-lib:fref sparam-%data% (2) ((1 5)) sparam-%offset%))
         (setf sh22
                 (f2cl-lib:fref sparam-%data% (5) ((1 5)) sparam-%offset%))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
            (setf w (f2cl-lib:fref sx-%data% (kx) ((1 *)) sx-%offset%))
            (setf z (f2cl-lib:fref sy-%data% (ky) ((1 *)) sy-%offset%))
            (setf (f2cl-lib:fref sx-%data% (kx) ((1 *)) sx-%offset%)
                    (+ (* w sh11) z))
            (setf (f2cl-lib:fref sy-%data% (ky) ((1 *)) sy-%offset%)
                    (- (* sh22 z) w))
            (setf kx (f2cl-lib:int-add kx incx))
            (setf ky (f2cl-lib:int-add ky incy))
           label100005))))))
     (go end_label)
    end_label
     (return (values nil nil nil nil nil nil))))))
```

———-

6.13 srotmg - apply a modified givens transformation

6.13.1 Axiom unit tests

— **srotmg.input** —

```
)set break resume
)sys rm -f srotmg.output
)spool srotmg.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

6.13.2 Axiom help page

— **srotmg.help** —

```
==================================================================
srotmg examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine srotmg (SD1, SD2, SX1, SY1, SPARAM)
           SROTMG

Function/Subroutine Documentation
   subroutine srotmg (real SD1, real SD2, real SX1, real SY1, real,
       dimension(5) SPARAM)
       SROTMG

Purpose:
 Construct the modified GIVENS transformation matrix H which zeros
 the second component of the 2-vector  (SQRT(SD1)*SX1,SQRT(SD2)*SY2)**T.
 with SPARAM(1)=SFLAG, H has one of the following forms..

  SFLAG=-1.E0     SFLAG=0.E0        SFLAG=1.E0     SFLAG=-2.E0
```

```
    (SH11  SH12)    (1.E0  SH12)    (SH11  1.E0)    (1.E0  0.E0)
  H=(          )    (          )    (          )    (          )
    (SH21  SH22),   (SH21  1.E0),   (-1.E0 SH22),   (0.E0  1.E0).
  locations 2-4 of SPARAM contain SH11,SH21,SH12, AND SH22
  respectively. (values of 1.E0, -1.E0, or 0.E0 implied by the
  value of SPARAM(1) are not stored in sparam.)

  The values of GAMSQ and RGAMSQ set in the data statement may be
  inexact.  This is ok as they are only used for testing the size
  of SD1 and SD2.  All actual scaling of data is done using gam.

Parameters:
 SD1
   SD1 is REAL

 SD2
   SD2 is REAL

 SX1
   SX1 is REAL

 SY1
   SY1 is REAL

 SPARAM
   SPARAM is REAL array, dimension 5
   SPARAM(1)=SFLAG
   SPARAM(2)=SH11
   SPARAM(3)=SH21
   SPARAM(4)=SH12
   SPARAM(5)=SH22
```

———-

6.13.3 fortran code

— srotmg.f —

```
*  =====================================================================
      SUBROUTINE SROTMG(SD1,SD2,SX1,SY1,SPARAM)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
```

```
*     .. Scalar Arguments ..
      REAL SD1,SD2,SX1,SY1
*     ..
*     .. Array Arguments ..
      REAL SPARAM(5)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      REAL GAM,GAMSQ,ONE,RGAMSQ,SFLAG,SH11,SH12,SH21,SH22,SP1,SP2,SQ1,
     $     SQ2,STEMP,SU,TWO,ZERO
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC ABS
*     ..
*     .. Data statements ..
*
      DATA ZERO,ONE,TWO/0.E0,1.E0,2.E0/
      DATA GAM,GAMSQ,RGAMSQ/4096.E0,1.67772E7,5.96046E-8/
*     ..

      IF (SD1.LT.ZERO) THEN
*        GO ZERO-H-D-AND-SX1..
         SFLAG = -ONE
         SH11 = ZERO
         SH12 = ZERO
         SH21 = ZERO
         SH22 = ZERO
*
         SD1 = ZERO
         SD2 = ZERO
         SX1 = ZERO
      ELSE
*        CASE-SD1-NONNEGATIVE
         SP2 = SD2*SY1
         IF (SP2.EQ.ZERO) THEN
            SFLAG = -TWO
            SPARAM(1) = SFLAG
            RETURN
         END IF
*        REGULAR-CASE..
         SP1 = SD1*SX1
         SQ2 = SP2*SY1
         SQ1 = SP1*SX1
*
         IF (ABS(SQ1).GT.ABS(SQ2)) THEN
            SH21 = -SY1/SX1
            SH12 = SP2/SP1
*
```

```
           SU = ONE - SH12*SH21
*
          IF (SU.GT.ZERO) THEN
            SFLAG = ZERO
            SD1 = SD1/SU
            SD2 = SD2/SU
            SX1 = SX1*SU
          END IF
        ELSE

           IF (SQ2.LT.ZERO) THEN
*             GO ZERO-H-D-AND-SX1..
              SFLAG = -ONE
              SH11 = ZERO
              SH12 = ZERO
              SH21 = ZERO
              SH22 = ZERO
*
              SD1 = ZERO
              SD2 = ZERO
              SX1 = ZERO
           ELSE
              SFLAG = ONE
              SH11 = SP1/SP2
              SH22 = SX1/SY1
              SU = ONE + SH11*SH22
              STEMP = SD2/SU
              SD2 = SD1/SU
              SD1 = STEMP
              SX1 = SY1*SU
           END IF
        END IF

*     PROCEDURE..SCALE-CHECK
        IF (SD1.NE.ZERO) THEN
           DO WHILE ((SD1.LE.RGAMSQ) .OR. (SD1.GE.GAMSQ))
              IF (SFLAG.EQ.ZERO) THEN
                 SH11 = ONE
                 SH22 = ONE
                 SFLAG = -ONE
              ELSE
                 SH21 = -ONE
                 SH12 = ONE
                 SFLAG = -ONE
              END IF
              IF (SD1.LE.RGAMSQ) THEN
                 SD1 = SD1*GAM**2
                 SX1 = SX1/GAM
                 SH11 = SH11/GAM
                 SH12 = SH12/GAM
```

```
         ELSE
            SD1 = SD1/GAM**2
            SX1 = SX1*GAM
            SH11 = SH11*GAM
            SH12 = SH12*GAM
         END IF
      ENDDO
   END IF

   IF (SD2.NE.ZERO) THEN
      DO WHILE ( (ABS(SD2).LE.RGAMSQ) .OR. (ABS(SD2).GE.GAMSQ) )
         IF (SFLAG.EQ.ZERO) THEN
            SH11 = ONE
            SH22 = ONE
            SFLAG = -ONE
         ELSE
            SH21 = -ONE
            SH12 = ONE
            SFLAG = -ONE
         END IF
         IF (ABS(SD2).LE.RGAMSQ) THEN
            SD2 = SD2*GAM**2
            SH21 = SH21/GAM
            SH22 = SH22/GAM
         ELSE
            SD2 = SD2/GAM**2
            SH21 = SH21*GAM
            SH22 = SH22*GAM
         END IF
      END DO
   END IF

END IF

IF (SFLAG.LT.ZERO) THEN
   SPARAM(2) = SH11
   SPARAM(3) = SH21
   SPARAM(4) = SH12
   SPARAM(5) = SH22
ELSE IF (SFLAG.EQ.ZERO) THEN
   SPARAM(3) = SH21
   SPARAM(4) = SH12
ELSE
   SPARAM(2) = SH11
   SPARAM(5) = SH22
END IF

SPARAM(1) = SFLAG
RETURN
END
```

———-

6.13.4 lisp code

— BLAS 1 srotmg —

```
(let ((zero 0.0)
      (one 1.0)
      (two 2.0)
      (gam 4096.0)
      (gamsq 1.67772e7)
      (rgamsq 5.96046e-8))
  (declare (type (single-float) zero one two gam gamsq rgamsq))
  (defun srotmg (sd1 sd2 sx1 sy1 sparam)
    (declare (type (array single-float (*)) sparam)
             (type (single-float) sy1 sx1 sd2 sd1))
    (f2cl-lib:with-multi-array-data
        ((sparam single-float sparam-%data% sparam-%offset%))
      (prog ((sflag 0.0) (sh11 0.0) (sh12 0.0) (sh21 0.0) (sh22 0.0) (sp1 0.0)
             (sp2 0.0) (sq1 0.0) (sq2 0.0) (stemp 0.0) (su 0.0))
        (declare
         (type (single-float) su stemp sq2 sq1 sp2 sp1 sh22 sh21 sh12 sh11
          sflag))
        (cond
         ((< sd1 zero) (setf sflag (- one)) (setf sh11 zero) (setf sh12 zero)
          (setf sh21 zero) (setf sh22 zero) (setf sd1 zero) (setf sd2 zero)
          (setf sx1 zero))
         (t (setf sp2 (* sd2 sy1))
          (cond
           ((= sp2 zero) (setf sflag (- two))
            (setf (f2cl-lib:fref sparam-%data% (1) ((1 5)) sparam-%offset%)
                    sflag)
            (go end_label)))
          (setf sp1 (* sd1 sx1)) (setf sq2 (* sp2 sy1)) (setf sq1 (* sp1 sx1))
          (cond
           ((> (abs sq1) (abs sq2)) (setf sh21 (/ (- sy1) sx1))
            (setf sh12 (/ sp2 sp1)) (setf su (- one (* sh12 sh21)))
            (cond
             ((> su zero) (setf sflag zero) (setf sd1 (/ sd1 su))
              (setf sd2 (/ sd2 su)) (setf sx1 (* sx1 su)))))
           (t
            (cond
             ((< sq2 zero) (setf sflag (- one)) (setf sh11 zero)
              (setf sh12 zero) (setf sh21 zero) (setf sh22 zero)
              (setf sd1 zero) (setf sd2 zero) (setf sx1 zero))
             (t (setf sflag one) (setf sh11 (/ sp1 sp2))
```

```
        (setf sh22 (/ sx1 sy1)) (setf su (+ one (* sh11 sh22)))
        (setf stemp (/ sd2 su)) (setf sd2 (/ sd1 su)) (setf sd1 stemp)
        (setf sx1 (* sy1 su))))))
    (cond
     ((/= sd1 zero)
      (tagbody
       label100000
        (if (not (or (<= sd1 rgamsq) (>= sd1 gamsq)))
            (go label100001))
        (cond
         ((= sflag zero) (setf sh11 one) (setf sh22 one)
          (setf sflag (- one)))
         (t (setf sh21 (- one)) (setf sh12 one) (setf sflag (- one))))
        (cond
         ((<= sd1 rgamsq) (setf sd1 (* sd1 (expt gam 2)))
          (setf sx1 (/ sx1 gam)) (setf sh11 (/ sh11 gam))
          (setf sh12 (/ sh12 gam)))
         (t (setf sd1 (/ sd1 (expt gam 2))) (setf sx1 (* sx1 gam))
          (setf sh11 (* sh11 gam)) (setf sh12 (* sh12 gam))))
        (go label100000)
       label100001)))
    (cond
     ((/= sd2 zero)
      (tagbody
       label100002
        (if (not (or (<= (abs sd2) rgamsq) (>= (abs sd2) gamsq)))
            (go label100003))
        (cond
         ((= sflag zero) (setf sh11 one) (setf sh22 one)
          (setf sflag (- one)))
         (t (setf sh21 (- one)) (setf sh12 one) (setf sflag (- one))))
        (cond
         ((<= (abs sd2) rgamsq) (setf sd2 (* sd2 (expt gam 2)))
          (setf sh21 (/ sh21 gam)) (setf sh22 (/ sh22 gam)))
         (t (setf sd2 (/ sd2 (expt gam 2))) (setf sh21 (* sh21 gam))
          (setf sh22 (* sh22 gam))))
        (go label100002)
       label100003)))))
  (cond
   ((< sflag zero)
    (setf (f2cl-lib:fref sparam-%data% (2) ((1 5)) sparam-%offset%) sh11)
    (setf (f2cl-lib:fref sparam-%data% (3) ((1 5)) sparam-%offset%) sh21)
    (setf (f2cl-lib:fref sparam-%data% (4) ((1 5)) sparam-%offset%) sh12)
    (setf (f2cl-lib:fref sparam-%data% (5) ((1 5)) sparam-%offset%)
            sh22))
   ((= sflag zero)
    (setf (f2cl-lib:fref sparam-%data% (3) ((1 5)) sparam-%offset%) sh21)
    (setf (f2cl-lib:fref sparam-%data% (4) ((1 5)) sparam-%offset%)
            sh12))
   (t
```

```
          (setf (f2cl-lib:fref sparam-%data% (2) ((1 5)) sparam-%offset%) sh11)
          (setf (f2cl-lib:fref sparam-%data% (5) ((1 5)) sparam-%offset%)
                  sh22)))
        (setf (f2cl-lib:fref sparam-%data% (1) ((1 5)) sparam-%offset%) sflag)
        (go end_label)
      end_label
        (return (values sd1 sd2 sx1 nil nil))))))
```

———-

6.14 sscal - scales a vector by a constant

6.14.1 Axiom unit tests

— **sscal.input** —

```
)set break resume
)sys rm -f sscal.output
)spool sscal.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

6.14.2 Axiom help page

— **sscal.help** —

```
==========================================================================
sscal examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sscal (N, SA, SX, INCX)
```

```
          SSCAL

Function/Subroutine Documentation
   subroutine sscal (integer N, real SA, real, dimension(*) SX, integer INCX)
       SSCAL

Purpose:
 scales a vector by a constant.
 uses unrolled loops for increment equal to 1.
```

———-

6.14.3 fortran code

— **sscal.f** —

```
*  ===================================================================
      SUBROUTINE SSCAL(N,SA,SX,INCX)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL SA
      INTEGER INCX,N
*     ..
*     .. Array Arguments ..
      REAL SX(*)
*     ..
*
*  ===================================================================
*
*     .. Local Scalars ..
      INTEGER I,M,MP1,NINCX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MOD
*     ..
      IF (N.LE.0 .OR. INCX.LE.0) RETURN
      IF (INCX.EQ.1) THEN
*
*        code for increment equal to 1
*
*
*        clean-up loop
```

```
*
         M = MOD(N,5)
         IF (M.NE.0) THEN
            DO I = 1,M
               SX(I) = SA*SX(I)
            END DO
            IF (N.LT.5) RETURN
         END IF
         MP1 = M + 1
         DO I = MP1,N,5
            SX(I) = SA*SX(I)
            SX(I+1) = SA*SX(I+1)
            SX(I+2) = SA*SX(I+2)
            SX(I+3) = SA*SX(I+3)
            SX(I+4) = SA*SX(I+4)
         END DO
      ELSE
*
*        code for increment not equal to 1
*
         NINCX = N*INCX
         DO I = 1,NINCX,INCX
            SX(I) = SA*SX(I)
         END DO
      END IF
      RETURN
      END
```

———-

6.14.4 lisp code

— BLAS 1 sscal —

```
(defun sscal (n sa sx incx)
  (declare (type (array single-float (*)) sx)
           (type (single-float) sa)
           (type (f2cl-lib:integer4) incx n))
  (f2cl-lib:with-multi-array-data
      ((sx single-float sx-%data% sx-%offset%))
    (prog ((i 0) (m 0) (mp1 0) (nincx 0))
      (declare (type (f2cl-lib:integer4) nincx mp1 m i))
      (if (or (<= n 0) (<= incx 0))
          (go end_label))
      (cond
       ((= incx 1) (setf m (mod n 5))
        (cond
```

```
    ((/= m 0)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i m) nil)
       (tagbody
         (setf (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%)
                 (* sa (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%)))
        label100000))
     (if (< n 5)
         (go end_label))))
   (setf mp1 (f2cl-lib:int-add m 1))
   (f2cl-lib:fdo (i mp1 (f2cl-lib:int-add i 5))
                 ((> i n) nil)
     (tagbody
       (setf (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%)
               (* sa (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%)))
       (setf (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                            sx-%offset%)
               (* sa
                  (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) sx-%offset%)))
       (setf (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                            sx-%offset%)
               (* sa
                  (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 2))
                                 ((1 *)) sx-%offset%)))
       (setf (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                            sx-%offset%)
               (* sa
                  (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 3))
                                 ((1 *)) sx-%offset%)))
       (setf (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 4)) ((1 *))
                            sx-%offset%)
               (* sa
                  (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 4))
                                 ((1 *)) sx-%offset%)))
      label100001)))
  (t (setf nincx (f2cl-lib:int-mul n incx))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i incx))
                 ((> i nincx) nil)
     (tagbody
       (setf (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%)
               (* sa (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%)))
      label100002))))
 (go end_label)
end_label
 (return (values nil nil nil nil)))))
```

———-

6.15 sswap - interchange two vectors

6.15.1 Axiom unit tests

— **sswap.input** —

```
)set break resume
)sys rm -f sswap.output
)spool sswap.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

6.15.2 Axiom help page

— **sswap.help** —

```
====================================================================
sswap examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sswap (N, SX, INCX, SY, INCY)
           SSWAP

Function/Subroutine Documentation
   subroutine sswap (integer N, real, dimension(*) SX, integer INCX, real,
       dimension(*) SY, integer INCY)
       SSWAP

Purpose:
 interchanges two vectors.
 uses unrolled loops for increments equal to 1.
```

———-

6.15.3 fortran code

— sswap.f —

```
*  =====================================================================
      SUBROUTINE SSWAP(N,SX,INCX,SY,INCY)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      REAL SX(*),SY(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      REAL STEMP
      INTEGER I,IX,IY,M,MP1
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MOD
*     ..
      IF (N.LE.0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*       code for both increments equal to 1
*
*
*       clean-up loop
*
         M = MOD(N,3)
         IF (M.NE.0) THEN
            DO I = 1,M
               STEMP = SX(I)
               SX(I) = SY(I)
               SY(I) = STEMP
            END DO
            IF (N.LT.3) RETURN
         END IF
         MP1 = M + 1
         DO I = MP1,N,3
            STEMP = SX(I)
            SX(I) = SY(I)
```

```
            SY(I) = STEMP
            STEMP = SX(I+1)
            SX(I+1) = SY(I+1)
            SY(I+1) = STEMP
            STEMP = SX(I+2)
            SX(I+2) = SY(I+2)
            SY(I+2) = STEMP
         END DO
      ELSE
*
*       code for unequal increments or equal increments not equal
*         to 1
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
         IF (INCY.LT.0) IY = (-N+1)*INCY + 1
         DO I = 1,N
            STEMP = SX(IX)
            SX(IX) = SY(IY)
            SY(IY) = STEMP
            IX = IX + INCX
            IY = IY + INCY
         END DO
      END IF
      RETURN
      END
```

———-

6.15.4 lisp code

— BLAS 1 sswap —

```
(defun sswap (n sx incx sy incy)
  (declare (type (array single-float (*)) sy sx)
           (type (f2cl-lib:integer4) incy incx n))
  (f2cl-lib:with-multi-array-data
      ((sx single-float sx-%data% sx-%offset%)
       (sy single-float sy-%data% sy-%offset%))
    (prog ((i 0) (ix 0) (iy 0) (m 0) (mp1 0) (stemp 0.0))
      (declare (type (single-float) stemp)
               (type (f2cl-lib:integer4) mp1 m iy ix i))
      (if (<= n 0)
          (go end_label))
      (cond
       ((and (= incx 1) (= incy 1)) (setf m (mod n 3))
```

```
(cond
 ((/= m 0)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf stemp (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%))
      (setf (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%)
              (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%))
      (setf (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%) stemp)
     label100000))
  (if (< n 3)
      (go end_label))))
(setf mp1 (f2cl-lib:int-add m 1))
(f2cl-lib:fdo (i mp1 (f2cl-lib:int-add i 3))
              ((> i n) nil)
  (tagbody
    (setf stemp (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%))
    (setf (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%)
            (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%))
    (setf (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%) stemp)
    (setf stemp
            (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                           sx-%offset%))
    (setf (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         sx-%offset%)
            (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                           sy-%offset%))
    (setf (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         sy-%offset%)
            stemp)
    (setf stemp
            (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                           sx-%offset%))
    (setf (f2cl-lib:fref sx-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                         sx-%offset%)
            (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                           sy-%offset%))
    (setf (f2cl-lib:fref sy-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                         sy-%offset%)
            stemp)
   label100001)))
(t (setf ix 1) (setf iy 1)
 (if (< incx 0)
     (setf ix
             (f2cl-lib:int-add
              (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incx) 1)))
 (if (< incy 0)
     (setf iy
             (f2cl-lib:int-add
              (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incy) 1)))
```

```
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf stemp (f2cl-lib:fref sx-%data% (ix) ((1 *)) sx-%offset%))
        (setf (f2cl-lib:fref sx-%data% (ix) ((1 *)) sx-%offset%)
                (f2cl-lib:fref sy-%data% (iy) ((1 *)) sy-%offset%))
        (setf (f2cl-lib:fref sy-%data% (iy) ((1 *)) sy-%offset%) stemp)
        (setf ix (f2cl-lib:int-add ix incx))
        (setf iy (f2cl-lib:int-add iy incy))
       label100002))))
  (go end_label)
 end_label
  (return (values nil nil nil nil nil)))))
```

———-

Chapter 7

BLAS Level 1 Double Vector Operations

7.1 dasum - sum of the absolute values

7.1.1 Axiom unit tests

— dasum.input —

```
)set break resume
)sys rm -f dasum.output
)spool dasum.output
)set message test on
)set message auto off
)clear all

--S 1 of 28
a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0] ]
--R
--R
--R   (1)  [1.,2.,3.,4.,5.,6.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 1

--S 2 of 28
dasum(3,a,-1) -- 0.0   neg incx
--R
--R
--R   (2)  0.
--R                                                          Type: DoubleFloat
--E 2
```

```
--S 3 of 28
dasum(3,a,0) --  0.0   zero incx
--R
--R
--R   (3)  0.
--R                                                        Type: DoubleFloat
--E 3

--S 4 of 28
dasum(-1,a,1) -- 0.0   neg elements
--R
--R
--R   (4)  0.
--R                                                        Type: DoubleFloat
--E 4

--S 5 of 28
dasum(0,a,1) --  0.0   no elements
--R
--R
--R   (5)  0.
--R                                                        Type: DoubleFloat
--E 5

--S 6 of 28
dasum(1,a,1) --  1.0   1.0
--R
--R
--R   (6)  1.
--R                                                        Type: DoubleFloat
--E 6

--S 7 of 28
dasum(2,a,1) --  3.0   1.0+2.0
--R
--R
--R   (7)  3.
--R                                                        Type: DoubleFloat
--E 7

--S 8 of 28
dasum(3,a,1) --  6.0   1.0+2.0+3.0
--R
--R
--R   (8)  6.
--R                                                        Type: DoubleFloat
--E 8

--S 9 of 28
```

```
dasum(4,a,1) -- 10.0   1.0+2.0+3.0+4.0
--R
--R
--R   (9)  10.
--R                                                        Type: DoubleFloat
--E 9

--S 10 of 28
dasum(5,a,1) -- 15.0   1.0+2.0+3.0+4.0+5.0
--R
--R
--R   (10)  15.
--R                                                        Type: DoubleFloat
--E 10

--S 11 of 28
dasum(6,a,1) -- 21.0   1.0+2.0+3.0+4.0+5.0+6.0
--R
--R
--R   (11)  21.
--R                                                        Type: DoubleFloat
--E 11

--S 12 of 28
dasum(7,a,1) -- 21.0   1.0+2.0+3.0+4.0+5.0+6.0
--R
--R
--R   (12)  21.
--R                                                        Type: DoubleFloat
--E 12

--S 13 of 28
dasum(1,a,2) --  1.0   1.0
--R
--R
--R   (13)  1.
--R                                                        Type: DoubleFloat
--E 13

--S 14 of 28
dasum(2,a,2) --  4.0   1.0+3.0
--R
--R
--R   (14)  4.
--R                                                        Type: DoubleFloat
--E 14

--S 15 of 28
dasum(3,a,2) --  9.0   1.0+3.0+5.0
--R
```

```
--R
--R   (15)  9.
--R                                                        Type: DoubleFloat
--E 15

--S 16 of 28
dasum(4,a,2) --  9.0   1.0+3.0+5.0
--R
--R
--R   (16)  9.
--R                                                        Type: DoubleFloat
--E 16

--S 17 of 28
dasum(1,a,3) --  1.0   1.0
--R
--R
--R   (17)  1.
--R                                                        Type: DoubleFloat
--E 17

--S 18 of 28
dasum(2,a,3) --  5.0   1.0+4.0
--R
--R
--R   (18)  5.
--R                                                        Type: DoubleFloat
--E 18

--S 19 of 28
dasum(3,a,3) --  5.0   1.0+4.0
--R
--R
--R   (19)  5.
--R                                                        Type: DoubleFloat
--E 19

--S 20 of 28
dasum(1,a,4) --  1.0   1.0
--R
--R
--R   (20)  1.
--R                                                        Type: DoubleFloat
--E 20

--S 21 of 28
dasum(2,a,4) --  6.0   1.0+5.0
--R
--R
--R   (21)  6.
```

```
--R                                                          Type: DoubleFloat
--E 21

--S 22 of 28
dasum(3,a,4) --  6.0   1.0+5.0
--R
--R
--R   (22)  6.
--R                                                          Type: DoubleFloat
--E 22

--S 23 of 28
dasum(1,a,5) --  1.0   1.0
--R
--R
--R   (23)  1.
--R                                                          Type: DoubleFloat
--E 23

--S 24 of 28
dasum(2,a,5) --  7.0   1.0+6.0
--R
--R
--R   (24)  7.
--R                                                          Type: DoubleFloat
--E 24

--S 25 of 28
dasum(3,a,5) --  7.0   1.0+6.0
--R
--R
--R   (25)  7.
--R                                                          Type: DoubleFloat
--E 25

--S 26 of 28
dasum(1,a,6) --  1.0   1.0
--R
--R
--R   (26)  1.
--R                                                          Type: DoubleFloat
--E 26

--S 27 of 28
dasum(2,a,6) --  1.0   1.0
--R
--R
--R   (27)  1.
--R                                                          Type: DoubleFloat
--E 27
```

```
--S 28 of 28
dasum(1,a,7) --  1.0   1.0
--R
--R
--R   (28)  1.
--R                                                        Type: DoubleFloat
--E 28

)spool
)lisp (bye)
```

7.1.2 Axiom help page

— dasum.help —

```
=====================================================================
dasum examples
=====================================================================

a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0] ]
    [1.,2.,3.,4.,5.,6.]

dasum(3,a,-1) -- 0.0   neg incx
    0.

dasum(3,a,0) --  0.0   zero incx
    0.

dasum(-1,a,1) -- 0.0   neg elements
    0.

dasum(0,a,1) --  0.0   no elements
    0.

dasum(1,a,1) --  1.0   1.0
    1.

dasum(2,a,1) --  3.0   1.0+2.0
    3.

dasum(3,a,1) --  6.0   1.0+2.0+3.0
    6.

dasum(4,a,1) -- 10.0   1.0+2.0+3.0+4.0
   10.
```

```
dasum(5,a,1) -- 15.0   1.0+2.0+3.0+4.0+5.0
   15.

dasum(6,a,1) -- 21.0   1.0+2.0+3.0+4.0+5.0+6.0
   21.

dasum(7,a,1) -- 21.0   1.0+2.0+3.0+4.0+5.0+6.0
   21.

dasum(1,a,2) --  1.0   1.0
    1.

dasum(2,a,2) --  4.0   1.0+3.0
    4.

dasum(3,a,2) --  9.0   1.0+3.0+5.0
    9.

dasum(4,a,2) --  9.0   1.0+3.0+5.0
    9.

dasum(1,a,3) --  1.0   1.0
    1.

dasum(2,a,3) --  5.0   1.0+4.0
    5.

dasum(3,a,3) --  5.0   1.0+4.0
    5.

dasum(1,a,4) --  1.0   1.0
    1.

dasum(2,a,4) --  6.0   1.0+5.0
    6.

dasum(3,a,4) --  6.0   1.0+5.0
    6.

dasum(1,a,5) --  1.0   1.0
    1.

dasum(2,a,5) --  7.0   1.0+6.0
    7.

dasum(3,a,5) --  7.0   1.0+6.0
    7.

dasum(1,a,6) --  1.0   1.0
```

```
    1.

dasum(2,a,6) --  1.0   1.0
    1.

dasum(1,a,7) --  1.0   1.0
    1.

======================================================================
Man Page Details
======================================================================

Computes doublefloat $asum \leftarrow ||re(x)||_1 + ||im(x)||_1$

Arguments are:
  * n - fixnum
  * dx -array doublefloat
  * incx - fixnum

Return values are:
  * nil
  * nil
  * nil

NAME
       DASUM  -  BLAS level one, sums the absolute values of the elements of a
       double precision vector

SYNOPSIS
       DOUBLE PRECISION FUNCTION DASUM ( n, x, incx )
           INTEGER                       n, incx
           DOUBLE PRECISION              x

AXIOM SIGNATURE:
       SI  ==> SingleInteger
       DF  ==> DoubleFloat
       DX  ==> PrimitiveArray(DoubleFloat)

       dasum: (SI, DX, SI) -> DF

DESCRIPTION
       This routine performs the following vector operation:

                           n
               DASUM <-- Sum abs(x(i))
                          i=1

ARGUMENTS
       n       INTEGER. (input)
               Number of vector elements to be summed.
```

```
                if n <= 0, the result will be 0.0
                if n > length(x) then the whole array is summed.

        x       DOUBLE PRECISION. (input)
                Array of dimension (n-1) * abs(incx)+ 1.
                Vector that contains elements to be summed.

        incx    INTEGER. (input)
                Increment between elements of x.
                If incx <= 0, the results will be 0.0

RESULT:
        DASUM   DOUBLE PRECISION. (output)
                Sum of the absolute values of the elements of the vector x.
                If n <= 0, DASUM is set to 0.0

NOTES:
        Axiom uses 0-based arrays. Fortran uses 1-based arrays.

        if the index of the array exceeds the length of x
        then no additional elements are added. Thus,
        if x = #(1.0 2.0 3.0 4.0 5.0 6.0)
        then
         (dasum 3 a -1) =  0.0 ; neg incx
         (dasum 3 a 0)  =  0.0 ; zero incx
         (dasum -1 a 1) =  0.0 ; neg elements
         (dasum 0 a 1)  =  0.0 ; no elements
         (dasum 1 a 1)  =  1.0 ; 1.0
         (dasum 2 a 1)  =  3.0 ; 1.0+2.0
         (dasum 3 a 1)  =  6.0 ; 1.0+2.0+3.0
         (dasum 4 a 1)  = 10.0 ; 1.0+2.0+3.0+4.0
         (dasum 5 a 1)  = 15.0 ; 1.0+2.0+3.0+4.0+5.0
         (dasum 6 a 1)  = 21.0 ; 1.0+2.0+3.0+4.0+5.0+6.0
         (dasum 7 a 1)  = 21.0 ; 1.0+2.0+3.0+4.0+5.0+6.0
         (dasum 1 a 2)  =  1.0 ; 1.0
         (dasum 2 a 2)  =  4.0 ; 1.0+3.0
         (dasum 3 a 2)  =  9.0 ; 1.0+3.0+5.0
         (dasum 4 a 2)  =  9.0 ; 1.0+3.0+5.0
         (dasum 1 a 3)  =  1.0 ; 1.0
         (dasum 2 a 3)  =  5.0 ; 1.0+4.0
         (dasum 3 a 3)  =  5.0 ; 1.0+4.0
         (dasum 1 a 4)  =  1.0 ; 1.0
         (dasum 2 a 4)  =  6.0 ; 1.0+5.0
         (dasum 3 a 4)  =  6.0 ; 1.0+5.0
         (dasum 1 a 5)  =  1.0 ; 1.0
         (dasum 2 a 5)  =  7.0 ; 1.0+6.0
         (dasum 3 a 5)  =  7.0 ; 1.0+6.0
         (dasum 1 a 6)  =  1.0 ; 1.0
         (dasum 2 a 6)  =  1.0 ; 1.0
         (dasum 1 a 7)  =  1.0 ; 1.0
```

———-

7.1.3 fortran code

— **dasum.f** —

```
*  ====================================================================
      DOUBLE PRECISION FUNCTION DASUM(N,DX,INCX)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION DX(*)
*     ..
*
*  ====================================================================
*
*     .. Local Scalars ..
      DOUBLE PRECISION DTEMP
      INTEGER I,M,MP1,NINCX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DABS,MOD
*     ..
      DASUM = 0.0d0
      DTEMP = 0.0d0
      IF (N.LE.0 .OR. INCX.LE.0) RETURN
      IF (INCX.EQ.1) THEN
*        code for increment equal to 1
*
*
*        clean-up loop
*
         M = MOD(N,6)
         IF (M.NE.0) THEN
            DO I = 1,M
               DTEMP = DTEMP + DABS(DX(I))
            END DO
            IF (N.LT.6) THEN
               DASUM = DTEMP
```

```
               RETURN
            END IF
         END IF
         MP1 = M + 1
         DO I = MP1,N,6
            DTEMP = DTEMP + DABS(DX(I)) + DABS(DX(I+1)) +
     $              DABS(DX(I+2)) + DABS(DX(I+3)) +
     $              DABS(DX(I+4)) + DABS(DX(I+5))
         END DO
      ELSE
*
*        code for increment not equal to 1
*
         NINCX = N*INCX
         DO I = 1,NINCX,INCX
            DTEMP = DTEMP + DABS(DX(I))
         END DO
      END IF
      DASUM = DTEMP
      RETURN
      END
```

———-

7.1.4 fortran unit test

— dasum example —

```
      program dasumEX
*     Tim Daly April 24, 2012
*     unit tests for BLAS dasum
      double precision a(6)
      double precision b
      a = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0, 5.0D0, 6.0D0 /)
      write(6,100)a(1),a(2),a(3)
 100  format("a(1)=",f6.3," a(2)=",f6.3," a(3)=",f6.3)
      write(6,200)a(4),a(5),a(6)
 200  format("a(4)=",f6.3," a(5)=",f6.3," a(6)=",f6.3)
      d=dasum(3,a,-1)
      write(6,300)d
 300  format("d=",f6.3," should be 0.0, negative index")
      d=dasum(3,a,0)
      write(6,301)d
 301  format("d=",f6.3," should be 0.0, zero increment")
      d=dasum(-11,a,1)
      write(6,302)d
 302  format("d=",f6.3," should be 0.0, negative elements")
```

```
      d=dasum(0,a,1)
      write(6,303)d
303   format("d=",f6.3," should be 0.0, no elements")
      d=dasum(1,a,1)
      write(6,304)d
304   format("d=",f6.3," should be 1.0")
      d=dasum(2,a,1)
      write(6,305)d
305   format("d=",f6.3," should be 3.0 = 1.0+2.0")
      d=dasum(3,a,1)
      write(6,306)d
306   format("d=",f6.3," should be 6.0 = 1.0+2.0+3.0")
      d=dasum(4,a,1)
      write(6,307)d
307   format("d=",f6.3," should be 10.0 = 1.0+2.0+3.0+4.0")
      d=dasum(5,a,1)
      write(6,308)d
308   format("d=",f6.3," should be 15.0 = 1.0+2.0+3.0+4.0+5.0")
      d=dasum(6,a,1)
      write(6,309)d
309   format("d=",f6.3," should be 21.0 = 1.0+2.0+3.0+4.0+5.0+6.0")
      d=dasum(7,a,1)
      write(6,310)d
310   format("d=",f6.3," should be 21.0 = 1.0+2.0+3.0+4.0+5.0+6.0")
      d=dasum(1,a,2)
      write(6,311)d
311   format("d=",f6.3," should be 1.0 = 1.0")
      d=dasum(2,a,2)
      write(6,312)d
312   format("d=",f6.3," should be 4.0 = 1.0+3.0")
      d=dasum(3,a,2)
      write(6,313)d
313   format("d=",f6.3," should be 9.0 = 1.0+3.0+5.0")
      d=dasum(4,a,2)
      write(6,314)d
314   format("d=",f6.3," should be 9.0 = 1.0+3.0+5.0")
      d=dasum(1,a,3)
      write(6,315)d
315   format("d=",f6.3," should be 1.0 = 1.0")
      d=dasum(2,a,3)
      write(6,316)d
316   format("d=",f6.3," should be 5.0 = 1.0+4.0")
      d=dasum(3,a,3)
      write(6,317)d
317   format("d=",f6.3," should be 5.0 = 1.0+4.0")
      d=dasum(1,a,4)
      write(6,318)d
318   format("d=",f6.3," should be 1.0 = 1.0")
      d=dasum(2,a,4)
      write(6,319)d
```

```
 319   format("d=",f6.3," should be 6.0 = 1.0+5.0")
       d=dasum(3,a,4)
       write(6,320)d
 320   format("d=",f6.3," should be 6.0 = 1.0+5.0")
       d=dasum(1,a,5)
       write(6,321)d
 321   format("d=",f6.3," should be 1.0 = 1.0")
       d=dasum(2,a,5)
       write(6,322)d
 322   format("d=",f6.3," should be 7.0 = 1.0+6.0")
       d=dasum(3,a,5)
       write(6,323)d
 323   format("d=",f6.3," should be 7.0 = 1.0+6.0")
       d=dasum(1,a,6)
       write(6,324)d
 324   format("d=",f6.3," should be 1.0 = 1.0")
       d=dasum(2,a,6)
       write(6,325)d
 325   format("d=",f6.3," should be 1.0 = 1.0")
       d=dasum(1,a,7)
       write(6,326)d
 326   format("d=",f6.3," should be 1.0 = 1.0")
       stop
       end
```

```
./tanglec bookvol10.5.pamphlet dasum.f >dasum.f
gcc -c dasum.f
./tanglec bookvol10.5.pamphlet "dasum example" >dasumEX.f
gcc -o dasumEX dasumEX.f -lgfortran dasum.o
./dasumEX

a(1)= 1.000 a(2)= 2.000 a(3)= 3.000
a(4)= 4.000 a(5)= 5.000 a(6)= 6.000
d= 0.000 should be 0.0, negative index
d= 0.000 should be 0.0, zero increment
d= 0.000 should be 0.0, negative elements
d= 0.000 should be 0.0, no elements
d= 1.000 should be 1.0
d= 3.000 should be 3.0 = 1.0+2.0
d= 6.000 should be 6.0 = 1.0+2.0+3.0
d=10.000 should be 10.0 = 1.0+2.0+3.0+4.0
d=15.000 should be 15.0 = 1.0+2.0+3.0+4.0+5.0
d=21.000 should be 21.0 = 1.0+2.0+3.0+4.0+5.0+6.0
d=21.000 should be 21.0 = 1.0+2.0+3.0+4.0+5.0+6.0
d= 1.000 should be 1.0 = 1.0
d= 4.000 should be 4.0 = 1.0+3.0
d= 9.000 should be 9.0 = 1.0+3.0+5.0
```

```
d= 9.000 should be 9.0 = 1.0+3.0+5.0
d= 1.000 should be 1.0 = 1.0
d= 5.000 should be 5.0 = 1.0+4.0
d= 5.000 should be 5.0 = 1.0+4.0
d= 1.000 should be 1.0 = 1.0
d= 6.000 should be 6.0 = 1.0+5.0
d= 6.000 should be 6.0 = 1.0+5.0
d= 1.000 should be 1.0 = 1.0
d= 7.000 should be 7.0 = 1.0+6.0
d= 7.000 should be 7.0 = 1.0+6.0
d= 1.000 should be 1.0 = 1.0
d= 1.000 should be 1.0 = 1.0
d= 1.000 should be 1.0 = 1.0
```

7.1.5 lisp code

— BLAS 1 dasum —

```
(declaim (ftype (function (fixnum (simple-array double-float (*)) fixnum)
                          double-float) dasum))
(defun dasum (n dx incx)
 ; Tim Daly April 24, 2012
 (declare (type (simple-array double-float (*)) dx) (type fixnum incx n))
 (let ((dasum 0.0) (maxlen (length dx)))
  (declare (type (double-float) dasum) (type fixnum maxlen))
  (when (> incx 0)
   (when (> n maxlen) (setq n maxlen))
   (unless (<= n 0)
     (do ((i 0 (1+ i)) (j 0 (+ j incx)))
         ((or (>= i n) (>= j maxlen)))
      (setq dasum
       (the double-float
        (+ (the double-float dasum)
         (the double-float (abs (the double-float (svref dx j))))))))))
    dasum))
```

———-

7.2 daxpy - constant time vector plus a vector

7.2.1 Axiom unit tests

— daxpy.input —

```
)set break resume
)sys rm -f daxpy.output
)spool daxpy.output
)set message test on
)set message auto off
)clear all

--S 1 of 22
a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (1)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 1

--S 2 of 22
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (2)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 2

--S 3 of 22
daxpy(3,2.0,a,1,b,1)
--R
--R
--R   (3)  [3.,6.,9.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 3

--S 4 of 22
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (4)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 4

--S 5 of 22
daxpy(7,2.0,a,1,b,1)
--R
--R
--R   (5)  [3.,6.,9.,12.,15.,18.,21.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 5

--S 6 of 22
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
```

```
--R
--R   (6)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 6
```

Note that Axiom properly handles array indexes that are out of bounds. The BLAS daxpy routine cannot check this condition.

— daxpy.input —

```
--S 7 of 22
daxpy(8,2.0,a,1,b,1)
--R
--R
--R   (7)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 7

--S 8 of 22
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (8)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 8

--S 9 of 22
daxpy(3,2.0,a,3,b,3)
--R
--R
--R   (9)  [3.,2.,3.,12.,5.,6.,21.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 9

--S 10 of 22
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (10)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 10

--S 11 of 22
daxpy(4,2.0,a,2,b,2)
--R
--R
--R   (11)  [3.,2.,9.,4.,15.,6.,21.]
--R                                          Type: PrimitiveArray(DoubleFloat)
```

```
--E 11

--S 12 of 22
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (12)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 12

--S 13 of 22
daxpy(5,2.0,a,2,b,2)
--R
--R
--R   (13)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 13

--S 14 of 22
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (14)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 14

--S 15 of 22
daxpy(3,2.0,a,2,b,2)
--R
--R
--R   (15)  [3.,2.,9.,4.,15.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 15

--S 16 of 22
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (16)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 16

--S 17 of 22
daxpy(3,-2.0,a,2,b,2)
--R
--R
--R   (17)  [- 1.,2.,- 3.,4.,- 5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 17
```

```
--S 18 of 22
a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0] ]
--R
--R
--R   (18)  [1.,2.,3.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 18

--S 19 of 22
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0] ]
--R
--R
--R   (19)  [1.,2.,3.,4.,5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 19

--S 20 of 22
daxpy(3,-2.0,a,1,b,2)
--R
--R
--R   (20)  [- 1.,2.,- 1.,4.,- 1.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 20

--S 21 of 22
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (21)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 21

--S 22 of 22
daxpy(3,0.0,a,1,b,2)
--R
--R
--R   (22)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 22

)spool
)lisp (bye)
```

7.2.2 Axiom help page

— daxpy.help —

```
=====================================================================
daxpy examples
=====================================================================

For each of the following examples we assume that we have preset
the variables to the following values. Note that the daxpy function
will modify the second array. Each example assumes we have reset
the variables to these values.

a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]

then we compute the sum of the first 3 elements of each vector
and we show the steps of the computation with trailing comments.
The comments show which elements are changed and what the computation is.

daxpy(3,2.0,a,1,b,1) ==> [3.,6.,9.,4.,5.,6.,7.]
  dy(0)[3.0] = dy(0)[1.0] + ( da[2.0] * dx(0)[1.0] )
  dy(1)[6.0] = dy(1)[2.0] + ( da[2.0] * dx(1)[2.0] )
  dy(2)[9.0] = dy(2)[3.0] + ( da[2.0] * dx(2)[3.0] )

or we compute the first 7 elements of each vector

daxpy(7,2.0,a,1,b,1) ==> [3.,6.,9.,12.,15.,18.,21.]
  dy(0) [3.0] = dy(0)[1.0] + ( da[2.0] * dx(0)[1.0] )
  dy(1) [6.0] = dy(1)[2.0] + ( da[2.0] * dx(1)[2.0] )
  dy(2) [9.0] = dy(2)[3.0] + ( da[2.0] * dx(2)[3.0] )
  dy(3)[12.0] = dy(3)[4.0] + ( da[2.0] * dx(3)[4.0] )
  dy(4)[15.0] = dy(4)[5.0] + ( da[2.0] * dx(4)[5.0] )
  dy(5)[18.0] = dy(5)[6.0] + ( da[2.0] * dx(5)[6.0] )
  dy(6)[21.0] = dy(6)[7.0] + ( da[2.0] * dx(6)[7.0] )

or we compute the first 8 elements, which fails due to the fact that
the vectors are not long enough.

daxpy(8,2.0,a,1,b,1) ==> [1.,2.,3.,4.,5.,6.,7.]

or we compute the 3 elements, taking every 3rd element, giving the
index of 0, 3, 6

daxpy(3,2.0,a,3,b,3) ==> [3.,2.,3.,12.,5.,6.,21.]
  dy(0) [3.0] = dy(0)[1.0] + ( da[2.0] * dx(0)[1.0] )
  dy(3)[12.0] = dy(3)[4.0] + ( da[2.0] * dx(3)[4.0] )
  dy(6)[21.0] = dy(6)[7.0] + ( da[2.0] * dx(6)[7.0] )

or we compute 4 elements, taking every 2nd element, giving the
index of 0, 2, 4, 6

daxpy(4,2.0,a,2,b,2) ==> [3.,2.,9.,4.,15.,6.,21.]
  dy(0) [3.0] = dy(0)[1.0] + ( da[2.0] * dx(0)[1.0] )
```

```
  dy(2) [9.0] = dy(2)[3.0] + ( da[2.0] * dx(2)[3.0] )
  dy(4)[15.0] = dy(4)[5.0] + ( da[2.0] * dx(4)[5.0] )
  dy(6)[21.0] = dy(6)[7.0] + ( da[2.0] * dx(6)[7.0] )

or we compute 5 elements, taking every 2nd element, which fails
due to vector length

daxpy(5,2.0,a,2,b,2) ==> [1.,2.,3.,4.,5.,6.,7.]

or we compute 3 elements, taking every 2nd value, giving the
index of 0, 2, 4

daxpy(3,2.0,a,2,b,2) ==> [3.,2.,9.,4.,15.,6.,7.]
  dy(0) [3.0] = dy(0)[1.0] + ( da[2.0] * dx(0)[1.0] )
  dy(2) [9.0] = dy(2)[3.0] + ( da[2.0] * dx(2)[3.0] )
  dy(4)[15.0] = dy(4)[5.0] + ( da[2.0] * dx(4)[5.0] )

or we compute 3 elements, taking every 2nd value, giving the
index of 0, 2, 4 but with a negative multiplier

daxpy(3,-2.0,a,2,b,2) ==> [- 1.,2.,- 3.,4.,- 5.,6.,7.]
  dy(0)[-1.0] = dy(0)[1.0] + ( da[-2.0] * dx(0)[1.0] )
  dy(2)[-3.0] = dy(2)[3.0] + ( da[-2.0] * dx(2)[3.0] )
  dy(4)[-5.0] = dy(4)[5.0] + ( da[-2.0] * dx(4)[5.0] )

or we change the lengths of the input vectors, making them unequal.
So for the next two examples we assume the arrays look like:

a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0] ]
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0] ]

We compute 3 elements, with a negative multiplier, and different increments
using an index for the 'a' array having values 0, 1, 2
using an index for the 'b' array having values 0, 2, 3

daxpy(3,-2.0,a,1,b,2) ==> [- 1.,2.,- 1.,4.,- 1.]
  dy(0)[-1.0] = dy(0)[1.0] + ( da[-2.0] * dx(0)[1.0] )
  dy(2)[-1.0] = dy(2)[3.0] + ( da[-2.0] * dx(1)[2.0] )
  dy(4)[-1.0] = dy(4)[5.0] + ( da[-2.0] * dx(2)[3.0] )

or we compute 3 elements with a multiplier of 0.0 which fails

daxpy(3,0.0,a,1,b,2) ==> [1.,2.,3.,4.,5.,6.,7.]

=====================================================================
Man Page Details
=====================================================================

NAME
       DAXPY - BLAS level one axpy subroutine
```

```
SYNOPSIS
       SUBROUTINE DAXPY    ( n, alpha, x, incx, y, incy )

           INTEGER           n, incx, incy

           DOUBLE PRECISION  alpha, x, y


DESCRIPTION
       DAXPY  adds  a  scalar multiple of a double precision vector to another
       double precision vector.

       DAXPY computes a constant alpha times a vector x plus a vector y.   The
       result overwrites the initial values of vector y.

       This routine performs the following vector operation:

            y  <--  alpha*x + y

       incx and incy specify the increment between two consecutive
       elements of respectively vector x and y.

ARGUMENTS
       n       INTEGER. (input)
               Number  of  elements in the vectors.  If n <= 0, these routines
               return without any computation.

       alpha   DOUBLE PRECISION. (input)
               If alpha = 0 this routine returns without any computation.

       x       DOUBLE PRECISION, (input)
               Array of dimension (n-1) * |incx| + 1.  Contains the vector  to
               be scaled before summation.

       incx    INTEGER. (input)
               Increment between elements of x.
               If incx = 0, the results will be unpredictable.

       y       DOUBLE PRECISION, (input and output)
               array of dimension (n-1) * |incy| + 1.
               Before calling the routine, y contains the vector to be summed.
               After the routine ends, y contains the result of the summation.

       incy    INTEGER. (input)
               Increment between elements of y.
               If incy = 0, the results will be unpredictable.

RETURN VALUES
       When  n <= 0, double precision alpha = 0., this routine returns immedi-
```

```
      ately with no change in its arguments.
```

———-

7.2.3 fortran code

— **daxpy.f** —

```
*  =====================================================================
      SUBROUTINE DAXPY(N,DA,DX,INCX,DY,INCY)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION DA
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION DX(*),DY(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER I,IX,IY,M,MP1
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MOD
*     ..
      IF (N.LE.0) RETURN
      IF (DA.EQ.0.0d0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*        code for both increments equal to 1
*
*
*        clean-up loop
*
         M = MOD(N,4)
         IF (M.NE.0) THEN
            DO I = 1,M
               DY(I) = DY(I) + DA*DX(I)
            END DO
         END IF
```

```
         IF (N.LT.4) RETURN
         MP1 = M + 1
         DO I = MP1,N,4
            DY(I) = DY(I) + DA*DX(I)
            DY(I+1) = DY(I+1) + DA*DX(I+1)
            DY(I+2) = DY(I+2) + DA*DX(I+2)
            DY(I+3) = DY(I+3) + DA*DX(I+3)
         END DO
      ELSE
*
*        code for unequal increments or equal increments
*          not equal to 1
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
         IF (INCY.LT.0) IY = (-N+1)*INCY + 1
         DO I = 1,N
          DY(IY) = DY(IY) + DA*DX(IX)
          IX = IX + INCX
          IY = IY + INCY
         END DO
      END IF
      RETURN
      END
```

Compile with

```
gcc -c daxpy.f
gcc -o daxpyEx daxpyEX.f -lgfortran daxpy.o
```

7.2.4 fortran unit test

— daxpy example —

```
       program daxpyEX
*      Tim Daly April 24, 2012
*      unit tests for BLAS daxpy (a*x+y)
       double precision a(7)
       double precision b(7)
       double precision c(7)
       a = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0, 5.0D0, 6.0D0, 7.0D0 /)
       b = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0, 5.0D0, 6.0D0, 7.0D0 /)
       write(6,100)a(1),a(2),a(3)
 100   format("a(1)=",f6.3," a(2)=",f6.3," a(3)=",f6.3)
```

```
      write(6,101)a(4),a(5),a(6)
101   format("a(4)=",f6.3," a(5)=",f6.3," a(6)=",f6.3)
      write(6,102)b(1),b(2),b(3)
102   format("b(1)=",f6.3," b(2)=",f6.3," b(3)=",f6.3)
      write(6,103)b(4),b(5),b(6)
103   format("b(4)=",f6.3," b(5)=",f6.3," b(6)=",f6.3)

      call daxpy(3,2.0d0,a,1,b,1)
      write(6,200)
200   format(/,"t200 is (/ 3.0, 6.0, 9.0, 4.0, 5.0, 6.0, 7.0 /)")
      write(6,201)b(1),b(2),b(3)
201   format("b(1)=",f6.3," b(2)=",f6.3," b(3)=",f6.3)
      write(6,202)b(4),b(5),b(6),b(7)
202   format("b(4)=",f6.3," b(5)=",f6.3," b(6)=",f6.3," b(7)=",f6.3)

      write(6,300)
300   format(/,"t300 is (/ 3.0, 6.0, 9.0, 12.0, 15.0, 18.0, 21.0 /)")
      b = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0, 5.0D0, 6.0D0, 7.0D0 /)
      call daxpy(7,2.0d0,a,1,b,1)
      write(6,201)b(1),b(2),b(3)
      write(6,202)b(4),b(5),b(6),b(7)

      write(6,302)
302   format(/,"t302 is (/ 3.0, 2.0, 3.0, 12.0, 5.0, 6.0, 21.0 /)")
      b = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0, 5.0D0, 6.0D0, 7.0D0 /)
      call daxpy(3,2.0d0,a,3,b,3)
      write(6,201)b(1),b(2),b(3)
      write(6,202)b(4),b(5),b(6),b(7)

      write(6,303)
303   format(/,"t303 is (/ 3.0, 2.0, 9.0, 4.0, 15.0, 6.0, 21.0 /)")
      b = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0, 5.0D0, 6.0D0, 7.0D0 /)
      call daxpy(4,2.0d0,a,2,b,2)
      write(6,201)b(1),b(2),b(3)
      write(6,202)b(4),b(5),b(6),b(7)

      write(6,305)
305   format(/,"t305 is (/  3.0, 2.0, 9.0, 4.0, 15.0, 6.0, 7.0 /)")
      b = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0, 5.0D0, 6.0D0, 7.0D0 /)
      call daxpy(3,2.0d0,a,2,b,2)
      write(6,201)b(1),b(2),b(3)
      write(6,202)b(4),b(5),b(6),b(7)

      write(6,306)
306   format(/,"t306 is (/ -1.0, 2.0, -1.0, 4.0, -1.0, 6.0, 7.0 /)")
      b = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0, 5.0D0, 6.0D0, 7.0D0 /)
      call daxpy(3,-2.0d0,a,1,b,2)
      write(6,201)b(1),b(2),b(3)
      write(6,202)b(4),b(5),b(6),b(7)
```

```
      write(6,307)
 307  format(/,"t307 is (/ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0 /)")
      b = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0, 5.0D0, 6.0D0, 7.0D0 /)
      call daxpy(3,0.0d0,a,1,b,2)
      write(6,201)b(1),b(2),b(3)
      write(6,202)b(4),b(5),b(6),b(7)
      stop
      end
```

———-

```
gcc -o daxpyEX daxpyEX.f -lgfortran daxpy.o && ./daxpyEX

a(1)= 1.000 a(2)= 2.000 a(3)= 3.000
a(4)= 4.000 a(5)= 5.000 a(6)= 6.000
b(1)= 1.000 b(2)= 2.000 b(3)= 3.000
b(4)= 4.000 b(5)= 5.000 b(6)= 6.000

t200 is (/ 3.0, 6.0, 9.0, 4.0, 5.0, 6.0, 7.0 /)
b(1)= 3.000 b(2)= 6.000 b(3)= 9.000
b(4)= 4.000 b(5)= 5.000 b(6)= 6.000 b(7)= 7.000

t300 is (/ 3.0, 6.0, 9.0, 12.0, 15.0, 18.0, 21.0 /)
b(1)= 3.000 b(2)= 6.000 b(3)= 9.000
b(4)=12.000 b(5)=15.000 b(6)=18.000 b(7)=21.000

t302 is (/ 3.0, 2.0, 3.0, 12.0, 5.0, 6.0, 21.0 /)
b(1)= 3.000 b(2)= 2.000 b(3)= 3.000
b(4)=12.000 b(5)= 5.000 b(6)= 6.000 b(7)=21.000

t303 is (/ 3.0, 2.0, 9.0, 4.0, 15.0, 6.0, 21.0 /)
b(1)= 3.000 b(2)= 2.000 b(3)= 9.000
b(4)= 4.000 b(5)=15.000 b(6)= 6.000 b(7)=21.000

t305 is (/  3.0, 2.0, 9.0, 4.0, 15.0, 6.0, 7.0 /)
b(1)= 3.000 b(2)= 2.000 b(3)= 9.000
b(4)= 4.000 b(5)=15.000 b(6)= 6.000 b(7)= 7.000

t306 is (/ -1.0, 2.0, -1.0, 4.0, -1.0, 6.0, 7.0 /)
b(1)=-1.000 b(2)= 2.000 b(3)=-1.000
b(4)= 4.000 b(5)=-1.000 b(6)= 6.000 b(7)= 7.000

t307 is (/ 2.0, 2.0, 7.0, 4.0, 11.0, 6.0, 7.0 /)
b(1)= 3.000 b(2)= 2.000 b(3)= 7.000
b(4)= 4.000 b(5)=11.000 b(6)= 6.000 b(7)= 7.000
```

7.2.5 lisp code

— BLAS 1 daxpy —

```
(declaim (ftype (function (fixnum double-float
                                  (simple-array double-float (*)) fixnum
                                  (simple-array double-float (*)) fixnum)
                 (simple-array double-float (*))) daxpy))
(defun daxpy (n da dx incx dy incy)
 ; Tim Daly April 24, 2012
 (declare (type (simple-array double-float) dx dy)
          (type double-float da) (type fixnum incy incx n))
  (let ((maxx (length dx)) (maxy (length dy)))
   (declare (type fixnum maxx maxy))
   (when (and (> n 0) (/= da 0.0)
              (> incx 0) (< (* (1- n) incx) maxx)
              (> incy 0) (< (* (1- n) incy) maxy))
    (if (and (= incx 1) (= incy 1))
     (dotimes (i n)
       (setf (the double-float (svref dy i))
        (+ (the double-float (svref dy i))
           (* (the double-float da)
              (the double-float (svref dx i))))))
     ; non-unit increments
     (let ((ix 0) (iy 0))
      (declare (type fixnum ix iy))
      (when (< incx 0) (setq ix (* (1+ (- n)) incx)))
      (when (< incy 0) (setq iy (* (1+ (- n)) incy)))
      (dotimes (i n)
        (setf (the double-float (svref dy iy))
         (+ (the double-float (svref dy iy))
            (* (the double-float da)
               (the double-float (svref dx ix)))))
        (setq ix (+ ix incx))
        (setq iy (+ iy incy)))))))
   dy)
```

———-

— BLAS 1 daxpy lisp test —

```
(setq a (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0 6.0d0 7.0d0))
(setq b (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0 6.0d0 7.0d0))
(daxpy 3 2.0d0 a 1 b 1)
(setq b (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0 6.0d0 7.0d0))
(daxpy 7 2.0d0 a 1 b 1)
(setq b (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0 6.0d0 7.0d0))
```

```
(daxpy 8 2.0d0 a 1 b 1)
(setq b (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0 6.0d0 7.0d0))
(daxpy 3 2.0d0 a 3 b 3)
(setq b (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0 6.0d0 7.0d0))
(daxpy 4 2.0d0 a 2 b 2)
(setq b (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0 6.0d0 7.0d0))
(daxpy 5 2.0d0 a 2 b 2)
(setq b (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0 6.0d0 7.0d0))
(daxpy 3 2.0d0 a 2 b 2)
(setq b (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0 6.0d0 7.0d0))
(daxpy 3 -2.0d0 a 2 b 2)
(setq a (vector 1.0d0 2.0d0 3.0d0))
(setq b (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0))
(daxpy 3 -2.0d0 a 1 b 2)
(setq b (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0 6.0d0 7.0d0))
(daxpy 3 0.0d0 a 1 b 2)
```

———-

7.3 dcabs1 - abs(real(z))+abs(imag(z))

7.3.1 Axiom unit tests

— dcabs1.input —

```
)set break resume
)sys rm -f dcabs1.output
)spool dcabs1.output
)set message test on
)set message auto off
)clear all

--S 1 of 10
t1:Complex DoubleFloat := complex(1.0,0)
--R
--R
--R   (1)  1.
--R                                                Type: Complex(DoubleFloat)
--E 1

--S 2 of 10
dcabs1(t1)
--R
--R
--R   (2)  1.
--R                                                          Type: DoubleFloat
```

```
--E 2

--S 3 of 10
t2:Complex DoubleFloat := complex(1.0,1.0)
--R
--R
--R   (3)  1. + %i
--R                                                    Type: Complex(DoubleFloat)
--E 3

--S 4 of 10
dcabs1(t2)
--R
--R
--R   (4)  2.
--R                                                            Type: DoubleFloat
--E 4

--S 5 of 10
t3:Complex DoubleFloat := complex(1.0,-1.0)
--R
--R
--R   (5)  1. - %i
--R                                                    Type: Complex(DoubleFloat)
--E 5

--S 6 of 10
dcabs1(t3)
--R
--R
--R   (6)  2.
--R                                                            Type: DoubleFloat
--E 6

--S 7 of 10
t4:Complex DoubleFloat := complex(-1.0,-1.0)
--R
--R
--R   (7)  - 1. - %i
--R                                                    Type: Complex(DoubleFloat)
--E 7

--S 8 of 10
dcabs1(t4)
--R
--R
--R   (8)  2.
--R                                                            Type: DoubleFloat
--E 8
```

```
--S 9 of 10
t5:Complex DoubleFloat := complex(-2.0,-2.0)
--R
--R
--R   (9)  - 2. - 2. %i
--R                                                    Type: Complex(DoubleFloat)
--E 9

--S 10 of 10
dcabs1(t5)
--R
--R
--R   (10)  4.
--R                                                          Type: DoubleFloat
--E 10

)spool
)lisp (bye)
```

———-

7.3.2 Axiom help page

— **dcabs1.help** —

```
====================================================================
dcabs1 examples
====================================================================

The dcabs1 routine computes the sum of the absolute value of the
real and imaginary parts of a complex number.

t1:Complex DoubleFloat := complex(1.0,0)
   1.

dcabs1(t1)
   1.

t2:Complex DoubleFloat := complex(1.0,1.0)
   1. + %i

dcabs1(t2)
   2.

t3:Complex DoubleFloat := complex(1.0,-1.0)
   1. - %i

dcabs1(t3)
```

```
   2.

t4:Complex DoubleFloat := complex(-1.0,-1.0)
   - 1. - %i

dcabs1(t4)
   2.

t5:Complex DoubleFloat := complex(-2.0,-2.0)
   - 2. - 2. %i

dcabs1(t5)
   4.

=====================================================================
Man Page Details
=====================================================================

The argument is:
\begin{itemize}
\item z - Complex DoubleFloat
\end{itemize}

The result is
\begin{itemize}
\item (+ (abs (realpart z)) (abs (imagpart z)))
\end{itemize}

See Also:
o )show BlasLevelOne
o )display operations dcabs1
o )help dcabs1
```

———-

Axiom represents the type Complex(DoubleFloat) as a pair whose car is the real part and whose cdr is the imaginary part. This fact is used in this implementation.

This should really be a macro.

7.3.3 fortran code

— **dcabs1.f** —

```
*  ====================================================================
      DOUBLE PRECISION FUNCTION DCABS1(Z)
*
```

```
*  -- Reference BLAS level1 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      COMPLEX*16 Z
*     ..
*     ..
*  =====================================================================
*
*     .. Intrinsic Functions ..
      INTRINSIC ABS,DBLE,DIMAG
*
      DCABS1 = ABS(DBLE(Z)) + ABS(DIMAG(Z))
      RETURN
      END
```

7.3.4 fortran unit test

```
tanglec bookvol10.5.pamphlet dasum.f >dasum.f
gcc -c dasum.f
tanglec bookvol10.5.pamphlet "dasum example" >dasumEX.f
gcc -o dcabs1EX dcabs1EX.f -lgfortran dcabs1.o
./dcabs1EX
```

— dcabs1 example —

```
      program dcabs1EX
*     Tim Daly April 23, 2012
*     unit tests for BLAS dcabs1
      double complex a,b,c,d
      a=COMPLEX(2.1,2.1)
      b=(3.1D2,4.1D3)
      c=a+b
      d=dcabs1(c)
      write(6,100)a
 100  format("        a=(",f10.3,",",f10.3,")")
      write(6,200)b
 200  format("        b=(",f10.3,",",f10.3,")")
      write(6,300)c
 300  format("      a+b=(",f10.3,",",f10.3,")")
      write(6,400)d
 400  format("dcabs1(c)=(",f10.3,",",f10.3,")")
```

```
      stop
      end
```

———-

```
./dcabs1EX

        a=(     2.100,     2.100)
        b=(   310.000,  4100.000)
      a+b=(   312.100,  4102.100)
dcabs1(c)=(  4414.200,     0.000)
```

7.3.5 lisp code

— BLAS 1 dcabs1 —

```
(declaim (ftype (function (cons) double-float) dcabs1))
(defun dcabs1 (z)
 ; Tim Daly April 23, 2012
 "Complex(DoubleFloat) z is a pair where (realpart . imaginarypart)
  so the spad caller needs to construct a lisp complex number.
  The result is a DoubleFloat (+ (abs (realpart z)) (abs (imagpart z)))"
 (the double-float
  (+ (the double-float (abs (the double-float (realpart z))))
     (the double-float (abs (the double-float (imagpart z)))))))
```

———-

— BLAS 1 dcabs1 test —

```
(dcabs1 #C(312.100 4102.100))
```

———-

7.4 dcopy - copy a vector x to a vector y

7.4.1 Axiom unit tests

— dcopy.input —

```
)set break resume
)sys rm -f dcopy.output
)spool dcopy.output
)set message test on
)set message auto off
)clear all

--S 1 of 23
a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]
--R
--R
--R   (1)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 1

--S 2 of 23
b:PRIMARR(DFLOAT):=[ [ 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0] ]
--R
--R
--R   (2)  [0.,0.,0.,0.,0.,0.,0.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 2

--S 3 of 23
dcopy(3,a,1,b,1)
--R
--R
--R   (3)  [1.,2.,3.,0.,0.,0.,0.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 3

--S 4 of 23
b:PRIMARR(DFLOAT):=[ [ 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0] ]
--R
--R
--R   (4)  [0.,0.,0.,0.,0.,0.,0.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 4

--S 5 of 23
dcopy(7,a,1,b,1)
--R
--R
--R   (5)  [1.,2.,3.,4.,5.,6.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 5

--S 6 of 23
b:PRIMARR(DFLOAT):=[ [ 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0] ]
--R
```

```
--R
--R   (6)  [0.,0.,0.,0.,0.,0.,0.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 6

--S 7 of 23
dcopy(8,a,1,b,1)
--R
--R
--R   (7)  [0.,0.,0.,0.,0.,0.,0.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 7

--S 8 of 23
b:PRIMARR(DFLOAT):=[ [ 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0] ]
--R
--R
--R   (8)  [0.,0.,0.,0.,0.,0.,0.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 8

--S 9 of 23
dcopy(3,a,3,b,3)
--R
--R
--R   (9)  [1.,0.,0.,4.,0.,0.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 9

--S 10 of 23
b:PRIMARR(DFLOAT):=[ [ 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0] ]
--R
--R
--R   (10)  [0.,0.,0.,0.,0.,0.,0.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 10

--S 11 of 23
dcopy(4,a,2,b,2)
--R
--R
--R   (11)  [1.,0.,3.,0.,5.,0.,7.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 11

--S 12 of 23
b:PRIMARR(DFLOAT):=[ [ 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0] ]
--R
--R
--R   (12)  [0.,0.,0.,0.,0.,0.,0.]
```

```
--R                                                Type: PrimitiveArray(DoubleFloat)
--E 12

--S 13 of 23
dcopy(5,a,2,b,2)
--R
--R
--R   (13)  [0.,0.,0.,0.,0.,0.,0.]
--R                                                Type: PrimitiveArray(DoubleFloat)
--E 13

--S 14 of 23
b:PRIMARR(DFLOAT):=[ [ 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0] ]
--R
--R
--R   (14)  [0.,0.,0.,0.,0.,0.,0.]
--R                                                Type: PrimitiveArray(DoubleFloat)
--E 14

--S 15 of 23
dcopy(3,a,2,b,2)
--R
--R
--R   (15)  [1.,0.,3.,0.,5.,0.,0.]
--R                                                Type: PrimitiveArray(DoubleFloat)
--E 15

--S 16 of 23
a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0] ]
--R
--R
--R   (16)  [1.,2.,3.]
--R                                                Type: PrimitiveArray(DoubleFloat)
--E 16

--S 17 of 23
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0] ]
--R
--R
--R   (17)  [1.,2.,3.,4.,5.]
--R                                                Type: PrimitiveArray(DoubleFloat)
--E 17

--S 18 of 23
dcopy(3,a,1,b,1)
--R
--R
--R   (18)  [1.,2.,3.,4.,5.]
--R                                                Type: PrimitiveArray(DoubleFloat)
--E 18
```

```
--S 19 of 23
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0] ]
--R
--R
--R   (19)  [1.,2.,3.,4.,5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 19

--S 20 of 23
dcopy(3,a,1,b,2)
--R
--R
--R   (20)  [1.,2.,2.,4.,3.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 20

--S 21 of 23
a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0] ]
--R
--R
--R   (21)  [1.,2.,3.,4.,5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 21

--S 22 of 23
b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0] ]
--R
--R
--R   (22)  [1.,2.,3.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 22

--S 23 of 23
dcopy(5,a,1,b,1)
--R
--R
--R   (23)  [1.,2.,3.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 23

)spool
)lisp (bye)
```

7.4.2 Axiom help page

— dcopy.help —

```
=====================================================================
dcopy examples
=====================================================================

Assume we have two arrays which we initialize to these values:
a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0] ]

b:PRIMARR(DFLOAT):=[ [ 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0] ]

Note that after each call to dcopy the b array is modified.
The changed value is shown. We reset it after each bcopy but we
do not show that here.

dcopy(3,a,1,b,1) ==> [1.,2.,3.,0.,0.,0.,0.]

dcopy(7,a,1,b,1) ==> [1.,2.,3.,4.,5.,6.,7.]

dcopy(8,a,1,b,1) ==> [0.,0.,0.,0.,0.,0.,0.]

dcopy(3,a,3,b,3) ==> [1.,0.,0.,4.,0.,0.,7.]

dcopy(4,a,2,b,2) ==> [1.,0.,3.,0.,5.,0.,7.]

dcopy(5,a,2,b,2) ==> [0.,0.,0.,0.,0.,0.,0.]

dcopy(3,a,2,b,2) ==> [1.,0.,3.,0.,5.,0.,0.]

The arrays can be of different lengths:

a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0] ]

b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0] ]

dcopy(3,a,1,b,1) ==>  [1.,2.,3.,4.,5.]

dcopy(3,a,1,b,2) ==>  [1.,2.,2.,4.,3.]

a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0] ]

b:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0] ]

dcopy(5,a,1,b,1) ==> [1.,2.,3.]

=====================================================================
Man Page Details
=====================================================================

NAME
```

```
       DCOPY  -  BLAS level one, copies a double precision vector into another
       double precision vector

SYNOPSIS
       SUBROUTINE DCOPY    ( n, x, incx, y, incy )

           INTEGER           n, incx, incy

           DOUBLE PRECISION  x, y


DESCRIPTION
       DCOPY copies a double precision vector into  another  double  precision
       vector.  DCOPY copies a vector x, whose length is n to a vector y. incx
       and incy specify the increment  between  two  consecutive  elements  of
       respectively vector x and y.

       This routine performs the following vector operation:

            y  <--  x

       where x and y are double precision vectors.

ARGUMENTS
       n       INTEGER. (input)
               Number of vector elements to be copied.
               If n <= 0, this routine returns without computation.

       x       DOUBLE PRECISION, (input)
               Vector from which to copy.

       incx    INTEGER. (input)
               Increment between elements of x.
               If incx = 0, the y vector is unchanged

       y       DOUBLE PRECISION, (output)
               array of dimension (n-1) * |incy| + 1, result vector.

       incy    INTEGER. (input)
               Increment between elements of y.
               If incy = 0, the y vector is unchanged


            ------
```

7.4.3 fortran code

— dcopy.f —

```
*  =====================================================================
      SUBROUTINE DCOPY(N,DX,INCX,DY,INCY)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION DX(*),DY(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER I,IX,IY,M,MP1
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MOD
*     ..
      IF (N.LE.0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*        code for both increments equal to 1
*
*
*        clean-up loop
*
         M = MOD(N,7)
         IF (M.NE.0) THEN
            DO I = 1,M
               DY(I) = DX(I)
            END DO
            IF (N.LT.7) RETURN
         END IF
         MP1 = M + 1
         DO I = MP1,N,7
            DY(I) = DX(I)
            DY(I+1) = DX(I+1)
            DY(I+2) = DX(I+2)
            DY(I+3) = DX(I+3)
            DY(I+4) = DX(I+4)
            DY(I+5) = DX(I+5)
            DY(I+6) = DX(I+6)
         END DO
      ELSE
*
```

```
*        code for unequal increments or equal increments
*          not equal to 1
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
         IF (INCY.LT.0) IY = (-N+1)*INCY + 1
         DO I = 1,N
            DY(IY) = DX(IX)
            IX = IX + INCX
            IY = IY + INCY
         END DO
      END IF
      RETURN
      END
```

Compile with

```
gcc -c dcopy.f
gcc -o dcopyEx dcopyEX.f -lgfortran dcopy.o
```

7.4.4 fortran unit test

— dcopy example —

```
       program dcopyEX
*      Tim Daly April 27, 2012
*      unit tests for BLAS dcopy
       double precision a(7)
       double precision b(7)
       a = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0, 5.0D0, 6.0D0, 7.0D0 /)
       b = (/ 0.0D0, 0.0D0, 0.0D0, 0.0D0, 0.0D0, 0.0D0, 0.0D0 /)
       write(6,100)a(1),a(2),a(3)
 100   format("a(1)=",f6.3," a(2)=",f6.3," a(3)=",f6.3)
       write(6,101)a(4),a(5),a(6)
 101   format("a(4)=",f6.3," a(5)=",f6.3," a(6)=",f6.3)
       write(6,102)b(1),b(2),b(3)
 102   format("b(1)=",f6.3," b(2)=",f6.3," b(3)=",f6.3)
       write(6,103)b(4),b(5),b(6)
 103   format("b(4)=",f6.3," b(5)=",f6.3," b(6)=",f6.3)

       call dcopy(3,a,1,b,1)
       write(6,200)
 200   format(/,"t200 is (/ 1.0 2.0 3.0 0.0 0.0 0.0 0.0 /)")
       write(6,201)b(1),b(2),b(3)
```

```
 201   format("b(1)=",f6.3," b(2)=",f6.3," b(3)=",f6.3)
       write(6,202)b(4),b(5),b(6),b(7)
 202   format("b(4)=",f6.3," b(5)=",f6.3," b(6)=",f6.3," b(7)=",f6.3)

       write(6,300)
 300   format(/,"t300 is (/ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0 /)")
       b = (/ 0.0D0, 0.0D0, 0.0D0, 0.0D0, 0.0D0, 0.0D0, 0.0D0 /)
       call dcopy(7,a,1,b,1)
       write(6,201)b(1),b(2),b(3)
       write(6,202)b(4),b(5),b(6),b(7)

       write(6,302)
 302   format(/,"t302 is (/ 1.0, 0.0, 0.0, 4.0, 0.0, 0.0, 7.0 /)")
       b = (/ 0.0D0, 0.0D0, 0.0D0, 0.0D0, 0.0D0, 0.0D0, 0.0D0 /)
       call dcopy(3,a,3,b,3)
       write(6,201)b(1),b(2),b(3)
       write(6,202)b(4),b(5),b(6),b(7)

       write(6,303)
 303   format(/,"t303 is (/ 1.0, 0.0, 3.0, 0.0, 5.0, 0.0, 7.0 /)")
       b = (/ 0.0D0, 0.0D0, 0.0D0, 0.0D0, 0.0D0, 0.0D0, 0.0D0 /)
       call dcopy(4,a,2,b,2)
       write(6,201)b(1),b(2),b(3)
       write(6,202)b(4),b(5),b(6),b(7)

       write(6,305)
 305   format(/,"t305 is (/ 1.0, 2.0, 3.0, 4.0, 5.0, 0.0, 0.0 /)")
       a = (/ 1.0D0, 2.0D0, 3.0D0, 0.0D0, 0.0D0, 0.0D0, 0.0D0 /)
       b = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0, 5.0D0, 0.0D0, 0.0D0 /)
       call dcopy(3,a,1,b,1)
       write(6,201)b(1),b(2),b(3)
       write(6,202)b(4),b(5),b(6),b(7)

       write(6,306)
 306   format(/,"t306 is (/ 1.0, 2.0, 2.0, 4.0, 3.0, 0.0, 0.0 /)")
       a = (/ 1.0D0, 2.0D0, 3.0D0, 0.0D0, 0.0D0, 0.0D0, 0.0D0 /)
       b = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0, 5.0D0, 0.0D0, 0.0D0 /)
       call dcopy(3,a,1,b,2)
       write(6,201)b(1),b(2),b(3)
       write(6,202)b(4),b(5),b(6),b(7)

       stop
       end
```

———-

```
gcc -o dcopyEX dcopyEX.f -lgfortran dcopy.o && ./dcopyEX

a(1)= 1.000 a(2)= 2.000 a(3)= 3.000
a(4)= 4.000 a(5)= 5.000 a(6)= 6.000
```

```
b(1)= 0.000 b(2)= 0.000 b(3)= 0.000
b(4)= 0.000 b(5)= 0.000 b(6)= 0.000

t200 is (/ 1.0 2.0 3.0 0.0 0.0 0.0 0.0 /)
b(1)= 1.000 b(2)= 2.000 b(3)= 3.000
b(4)= 0.000 b(5)= 0.000 b(6)= 0.000 b(7)= 0.000

t300 is (/ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0 /)
b(1)= 1.000 b(2)= 2.000 b(3)= 3.000
b(4)= 4.000 b(5)= 5.000 b(6)= 6.000 b(7)= 7.000

t302 is (/ 1.0, 0.0, 0.0, 4.0, 0.0, 0.0, 7.0 /)
b(1)= 1.000 b(2)= 0.000 b(3)= 0.000
b(4)= 4.000 b(5)= 0.000 b(6)= 0.000 b(7)= 7.000

t303 is (/ 1.0, 0.0, 3.0, 0.0, 5.0, 0.0, 7.0 /)
b(1)= 1.000 b(2)= 0.000 b(3)= 3.000
b(4)= 0.000 b(5)= 5.000 b(6)= 0.000 b(7)= 7.000

t305 is (/ 1.0, 2.0, 3.0, 4.0, 5.0, 0.0, 0.0 /)
b(1)= 1.000 b(2)= 2.000 b(3)= 3.000
b(4)= 4.000 b(5)= 5.000 b(6)= 0.000 b(7)= 0.000

t306 is (/ 1.0, 2.0, 2.0, 4.0, 3.0, 0.0, 0.0 /)
b(1)= 1.000 b(2)= 2.000 b(3)= 2.000
b(4)= 4.000 b(5)= 3.000 b(6)= 0.000 b(7)= 0.000
```

7.4.5 lisp code

— BLAS 1 dcopy —

```
(declaim (ftype (function (fixnum (simple-array double-float (*)) fixnum
                                  (simple-array double-float (*)) fixnum)
                 (simple-array double-float (*))) dcopy))
(defun dcopy (n dx incx dy incy)
 ; Tim Daly April 27, 2012
 (declare (type (simple-array double-float) dy dx)
          (type fixnum incy incx n))
  (let ((maxx (length dx)) (maxy (length dy)))
   (declare (type fixnum maxx maxy))
   (when (and (> n 0)
              (> incx 0) (< (* (1- n) incx) maxx)
              (> incy 0) (< (* (1- n) incy) maxy))
    (if (and (= incx 1) (= incy 1))
     ; unit increments
     (dotimes (i n)
       (setf (the double-float (svref dy i)) (the double-float (svref dx i))))
```

```
    ; non-unit increments
    (let ((ix 0) (iy 0))
     (declare (type fixnum ix iy))
     (when (< incx 0) (setq ix (* (1+ (- n)) incx)))
     (when (< incy 0) (setq ix (* (1+ (- n)) incy)))
     (dotimes (i n)
      (setf (the double-float (svref dy iy)) (the double-float (svref dx ix)))
      (setq ix (+ ix incx))
      (setq iy (+ iy incy)))))))
  dy)
```

———-

— BLAS 1 dcopy lisp test —

```
(setq a (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0 6.0d0 7.0d0))
(setq b (vector 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0))
(dcopy 3 a 1 b 1)
(setq b (vector 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0))
(dcopy 7 a 1 b 1)
(setq b (vector 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0))
(dcopy 8 a 1 b 1)
(setq b (vector 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0))
(dcopy 3 a 3 b 3)
(setq b (vector 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0))
(dcopy 4 a 2 b 2)
(setq b (vector 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0))
(dcopy 5 a 2 b 2)
(setq b (vector 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0 0.0d0))
(dcopy 3 a 2 b 2)
(setq a (vector 1.0d0 2.0d0 3.0d0))
(setq b (vector 0.0d0 0.0d0 3.0d0 4.0d0 5.0d0))
(dcopy 3 a 1 b 1)
b
(setq b (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0))
(dcopy 3 a 1 b 2)
(setq a (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0))
(setq b (vector 1.0d0 2.0d0 3.0d0))
(dcopy 5 a 1 b 1)
```

———-

7.5 ddot - dot product of two vectors

7.5.1 Axiom unit tests

— ddot.input —

```
)set break resume
)sys rm -f ddot.output
)spool ddot.output
)set message test on
)set message auto off
)clear all

--S 1 of 9
a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0] ]
--R
--R   (1)  [1.,2.,3.,4.,5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 1

--S 2 of 9
b:PRIMARR(DFLOAT):=[ [ 5.0, 6.0, 7.0, 8.0, 9.0] ]
--R
--R   (2)  [5.,6.,7.,8.,9.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 2

--S 3 of 9
ddot(0,a,1,b,1)
--R
--R   (3)  0.
--R                                                              Type: DoubleFloat
--E 3

--S 4 of 9
ddot(3,a,1,b,1)
--R
--R   (4)  38.
--R                                                              Type: DoubleFloat
--E 4

--S 5 of 9
ddot(3,a,1,b,2)
--R
--R   (5)  46.
--R                                                              Type: DoubleFloat
--E 5
```

```
--S 6 of 9
ddot(3,a,2,b,1)
--R
--R   (6)  58.
--R                                                    Type: DoubleFloat
--E 6

--S 7 of 9
ddot(3,a,1,b,-2)
--R
--R   (7)  38.
--R                                                    Type: DoubleFloat
--E 7

--S 8 of 9
ddot(3,a,-2,b,1)
--R
--R   (8)  50.
--R                                                    Type: DoubleFloat
--E 8

--S 9 of 9
ddot(3,a,-2,b,-2)
--R
--R   (9)  71.
--R                                                    Type: DoubleFloat
--E 9


)spool
)lisp (bye)
```

———-

7.5.2 Axiom help page

— ddot.help —

```
=====================================================================
ddot examples
=====================================================================

a:PRIMARR(DFLOAT):=[ [ 1.0, 2.0, 3.0, 4.0, 5.0] ]

  [1.,2.,3.,4.,5.]

b:PRIMARR(DFLOAT):=[ [ 5.0, 6.0, 7.0, 8.0, 9.0] ]
```

```
  [5.,6.,7.,8.,9.]

ddot(0,a,1,b,1)

  0.

ddot(3,a,1,b,1)

 38. ( = 1.0*5.0 + 2.0*6.0 + 3.0*7.0 )

ddot(3,a,1,b,2)

 46. ( = 1.0*5.0 + 2.0*7.0 + 3.0*9.0 )

ddot(3,a,2,b,1)

 58. ( = 1.0*5.0 + 3.0*6.0 + 5.0*7.0 )

ddot(3,a,1,b,-2)

 38. ( = 1.0*9.0 + 2.0*7.0 + 3.0*5.0 )

ddot(3,a,-2,b,1)

 50. ( = 5.0*5.0 + 3.0*6.0 + 1.0*7.0 )

ddot(3,a,-2,b,-2)

 71. ( = 5.0*9.0 + 3.0*7.0 + 1.0*5.0 )

====================================================================
Man Page Details
====================================================================

NAME
       DDOT  -  BLAS  level one, computes a dot product (inner product) of two
       double precision vectors

SYNOPSIS
       DOUBLE PRECISION FUNCTION DDOT ( n, x, incx, y, incy )

           INTEGER                      n, incx, incy

           DOUBLE PRECISION             x, y


DESCRIPTION
       DDOT computes a dot product of two double precision vectors  (1  double
       precision inner product).
         2
```

```
       This routine performs the following vector operation:

                                                  n
               DDOT  <--  (transpose of x) * y  =  Sum  x(i)*y(i)
                                                 i=1
       where x and y are double precision vectors.

       If n <= 0, DDOT is set to 0.

ARGUMENTS
       n       INTEGER. (input)
               Number of elements in each vector.

       x       DOUBLE PRECISION. (input)
               Array of dimension (n-1) * |incx| + 1.
               Array x contains the first vector operand.

       incx    INTEGER. (input)
               Increment between elements of x.
               If incx = 0, the results will be unpredictable.

       y       DOUBLE PRECISION, (input)
               Array of dimension (n-1) * |incy| + 1.
               Array y contains the second vector operand.

       incy    INTEGER. (input)
               Increment between elements of y.  If incy = 0, the results will
               be unpredictable.

RETURN VALUES
       DDOT    DOUBLE PRECISION. Result (dot product). (output)

NOTES
       When  working  backward  (incx < 0 or incy < 0), each routine starts at
       the end of the vector and moves backward, as follows:

            x(1-incx * (n-1)), x(1-incx * (n-2)), ..., x(1)

            y(1-incy * (n-1)), y(1-incy * (n-2)), ..., y(1)
```

———-

7.5.3 fortran code

— ddot.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION DDOT(N,DX,INCX,DY,INCY)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION DX(*),DY(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      DOUBLE PRECISION DTEMP
      INTEGER I,IX,IY,M,MP1
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MOD
*     ..
      DDOT = 0.0d0
      DTEMP = 0.0d0
      IF (N.LE.0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*        code for both increments equal to 1
*
*
*        clean-up loop
*
         M = MOD(N,5)
         IF (M.NE.0) THEN
            DO I = 1,M
               DTEMP = DTEMP + DX(I)*DY(I)
            END DO
            IF (N.LT.5) THEN
               DDOT=DTEMP
            RETURN
            END IF
         END IF
         MP1 = M + 1
         DO I = MP1,N,5
          DTEMP = DTEMP + DX(I)*DY(I) + DX(I+1)*DY(I+1) +
     $            DX(I+2)*DY(I+2) + DX(I+3)*DY(I+3) + DX(I+4)*DY(I+4)
         END DO
      ELSE
```

```
*
*        code for unequal increments or equal increments
*          not equal to 1
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
         IF (INCY.LT.0) IY = (-N+1)*INCY + 1
         DO I = 1,N
            DTEMP = DTEMP + DX(IX)*DY(IY)
            IX = IX + INCX
            IY = IY + INCY
         END DO
      END IF
      DDOT = DTEMP
      RETURN
      END
```

Compile with

```
gcc -c ddot.f
gcc -o ddotEx ddotEX.f -lgfortran ddot.o
```

7.5.4 fortran unit test

— ddot example —

```
       program ddotEX
*      Tim Daly April 27, 2012
*      unit tests for BLAS ddot
       double precision a(5)
       double precision b(5)
       double precision c
       a = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0, 5.0D0 /)
       b = (/ 5.0D0, 6.0D0, 7.0D0, 8.0D0, 9.0D0 /)
       write(6,100)a(1),a(2),a(3),a(4),a(5)
 100   format("a=(/",f6.3," ",f6.3," ",f6.3," ",f6.3," ",f6.3,"/)")
       write(6,101)b(1),b(2),b(3),b(4),b(5)
 101   format("b=(/",f6.3," ",f6.3," ",f6.3," ",f6.3," ",f6.3,"/)")

* handle 0 elements
       c=ddot(0,a,1,b,1)
       write(6,200)c
 200   format(/,"t200 is 0.0",/,"c=",f6.3)
```

```
* handle (1,2,3) * (5,6,7)
       c=ddot(3,a,1,b,1)
       write(6,201)c
 201   format(/,"t201 is 38.0",/,"c=",f6.3)

* handle increment = 2 in b (1,2,3) * (5,7,9)
       c=ddot(3,a,1,b,2)
       write(6,202)c
 202   format(/,"t202 is 46.0",/,"c=",f6.3)

* handle increment = 2 in a (1,3,5) * (5,6,7)
       c=ddot(3,a,2,b,1)
       write(6,203)c
 203   format(/,"t203 is 58.0",/,"c=",f6.3)

* handle increment = -2 in b (1,2,3) * (9,7,5)
       c=ddot(3,a,1,b,-2)
       write(6,204)c
 204   format(/,"t204 is 38.0",/,"c=",f6.3)

* handle increment = -2 in a (5,3,1) * (5,6,7)
       c=ddot(3,a,-2,b,1)
       write(6,205)c
 205   format(/,"t205 is 50.0",/,"c=",f6.3)

* handle increment = -2 in a and b(5,3,1) * (9,7,5)
       c=ddot(3,a,-2,b,-2)
       write(6,206)c
 206   format(/,"t206 is 71.0",/,"c=",f6.3)

       stop
       end
```

```
a=(/ 1.000  2.000  3.000  4.000  5.000/)
a=(/ 5.000  6.000  7.000  8.000  9.000/)

t200 is 0.0
c= 0.000

t201 is 28.0
c=38.000

t202 is 46.0
c=46.000

t203 is 58.0
c=58.000
```

```
t204 is 38.0
c=38.000

t205 is 50.0
c=50.000

t206 is 71.0
c=71.000
```

7.5.5 lisp code

— BLAS 1 ddot —

```
(declaim (ftype (function (fixnum (simple-array double-float (*)) fixnum
                                  (simple-array double-float (*)) fixnum)
                 double-float) ddot))
(defun ddot (n dx incx dy incy)
 ; Tim Daly April 27, 2012
 (declare (optimize (speed 3) (safety 0))
          (type (simple-array double-float (*)) dx dy)
          (type fixnum incy incx n))
 (let ((ix 0) (iy 0) (ddot 0.0d0))
 (declare (type (double-float) ddot) (type fixnum iy ix))
  (when (> n 0)
   (when (< incx 0)
     (setf ix (the fixnum (*
                  (the fixnum (- 1 (the fixnum n)))
                  (the fixnum incx)))))
   (when (< incy 0)
     (setf iy (the fixnum (*
                  (the fixnum (- 1 (the fixnum n)))
                  (the fixnum incy)))))
   (do ((i 0 (1+ i)) (x ix (+ x incx)) (y iy (+ y incy)))
       ((= i n))
     (setf ddot
        (the double-float (+ ddot
          (the double-float (* (aref dx x) (aref dy y))))))))
    ddot))
```

———-

— BLAS 1 dcopy lisp test —

```
(setq a (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0))
(setq b (vector 5.0d0 6.0d0 7.0d0 8.0d0 9.0d0))
(= 0.0d0 (ddot 0 a 1 b 1))
(= 38.0d0 (ddot 3 a 1 b 1))
```

```
(= 46.0d0 (ddot 3 a 1 b 2))
(= 58.0d0 (ddot 3 a 2 b 1))
(= 38.0d0 (ddot 3 a 1 b -2))
(= 50.0d0 (ddot 3 a -2 b 1))
(= 71.0d0 (ddot 3 a -2 b -2))
```

7.6 dnrm2 - Euclidean norm of a vector

7.6.1 Axiom unit tests

— dnrm2.input —

```
)set break resume
)sys rm -f dnrm2.output
)spool dnrm2.output
)set message test on
)set message auto off
)clear all

--S 1 of 4
a:PRIMARR(DFLOAT):=[ [ 3.0, -4.0, 5.0, -7.0, 9.0] ]
--R
--R   (1)  [3.,- 4.,5.,- 7.,9.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 1

--S 2 of 4
dnrm2(3,a,1)
--R
--R   (2)  7.0710678118654755
--R                                                          Type: DoubleFloat
--E 2

--S 3 of 4
dnrm2(5,a,1)
--R
--R   (3)  13.416407864998739
--R                                                          Type: DoubleFloat
--E 3

--S 4 of 4
dnrm2(3,a,2)
--R
--R   (4)  10.72380529476361
--R                                                          Type: DoubleFloat
```

```
--E 4

)spool
)lisp (bye)
```

———-

7.6.2 Axiom help page

— **dnrm2.help** —

```
=====================================================================
dnrm2 examples
=====================================================================

DNRM2 is a BLAS level one which computes the Euclidean norm of a vector.

Given the vector [ 3.0, -4.0, 5.0, -7.0, 9.0 ]

dnrm2(3,a,1)
   = sqrt(3.0^2 + -4.0^2 + 5.0^2)
   = sqrt(50.0)
   = 7.0710678118654755

dnrm2(5,a,1)
   = sqrt(3.0^2 + -4.0^2 + 5.0^2 + -7.0^2 + 9.0^2)
   = sqrt(180.0)
   = 13.416407864998739

dnrm2(3,a,2)
   = sqrt(3.0^2 + 5.0^2 + 9.0^2)
   = sqrt(115.0)
   = 10.72380529476361

=====================================================================
Man Page Details
=====================================================================

NAME
       DNRM2 - BLAS level one, computes the Euclidean norm of a vector

SYNOPSIS
       DOUBLE PRECISION FUNCTION DNRM2 ( n, x, incx )

           INTEGER                      n, incx

           DOUBLE PRECISION             x
```

```
DESCRIPTION
       DNRM2  computes the Euclidean (L2) norm of a double precision real vec-
       tor, as follows:

               DNRM2 <-- ||x||
                              2

       where x is a double precision real vector.

ARGUMENTS
       n       INTEGER. (input)
               Number of elements in the operand vector.

       x       DOUBLE PRECISION. (input)
               Array of dimension (n-1) * |incx| + 1.
               Array x contains the operand vector.

       incx    INTEGER. (input)
               Increment between elements of x.
               If incx = 0, the results will be unpredictable.

RETURN VALUES
       DNRM2   DOUBLE PRECISION. Result (Euclidean norm). (output)
               If n <= 0, DNRM2 is set to 0d0.

NOTES
       When working backward (incx < 0), each routine starts at the end of the
       vector and moves backward, as follows:

            x(1-incx * (n-1)), x(1-incx * (n-2)), ..., x(1)
```

7.6.3 fortran code

— dnrm2.f —

```
*  ====================================================================
      DOUBLE PRECISION FUNCTION DNRM2(N,X,INCX)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
```

```
      INTEGER INCX,N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION X(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION ONE,ZERO
      PARAMETER (ONE=1.0D+0,ZERO=0.0D+0)
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION ABSXI,NORM,SCALE,SSQ
      INTEGER IX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC ABS,SQRT
*     ..
      IF (N.LT.1 .OR. INCX.LT.1) THEN
          NORM = ZERO
      ELSE IF (N.EQ.1) THEN
          NORM = ABS(X(1))
      ELSE
          SCALE = ZERO
          SSQ = ONE
*        The following loop is equivalent to this call to the LAPACK
*        auxiliary routine:
*        CALL DLASSQ( N, X, INCX, SCALE, SSQ )
*
          DO 10 IX = 1,1 + (N-1)*INCX,INCX
              IF (X(IX).NE.ZERO) THEN
                  ABSXI = ABS(X(IX))
                  IF (SCALE.LT.ABSXI) THEN
                      SSQ = ONE + SSQ* (SCALE/ABSXI)**2
                      SCALE = ABSXI
                  ELSE
                      SSQ = SSQ + (ABSXI/SCALE)**2
                  END IF
              END IF
   10     CONTINUE
          NORM = SCALE*SQRT(SSQ)
      END IF
*
      DNRM2 = NORM
      RETURN
*
*     End of DNRM2.
*
      END
```

Compile with

```
gcc -c dnrm2.f
gcc -o dnrm2Ex dnrm2EX.f -lgfortran dnrm2.o
```

7.6.4 fortran unit test

— dnrm2 example —

```
      program dnrm2EX
*     Tim Daly April 28, 2012
*     unit tests for BLAS dnrm2
      double precision a(5)
      double precision c
      a = (/ 3.0D0, -4.0D0, 5.0D0, -7.0D0, 9.0D0/)
      write(6,100)a(1),a(2),a(3)
 100  format("a=(/",f6.3," ",f6.3," ",f6.3"/)")

      c=dnrm2(3,a,1)
      write(6,200)c
 200  format(/,"t200 is sqrt(50.0)=7.071",/,"c=",f6.3)

      c=dnrm2(5,a,1)
      write(6,201)c
 201  format(/,"t201 is sqrt(180.0)=13.416",/,"c=",f6.3)

      c=dnrm2(3,a,2)
      write(6,202)c
 202  format(/,"t202 is sqrt(115.0)=10.724",/,"c=",f9.3)

      stop
      end
```

```
gcc -o dnrm2EX dnrm2EX.f -lgfortran dnrm2.o && ./dnrm2EX
a=(/ 3.000 -4.000  5.000/)

t200 is sqrt(50.0)=7.071
c= 7.071

t201 is sqrt(180.0)=13.416
c=13.416
```

```
t202 is sqrt(115.0)=10.724
c=   10.724
```

7.6.5 lisp code

— BLAS 1 dnrm2 —

```
(declaim (ftype (function (fixnum (simple-array double-float (*)) fixnum)
                 double-float) dnrm2))
(defun dnrm2 (n x incx)
 ; Tim Daly April 28, 2012
 (declare (type (simple-array double-float (*)) x) (type fixnum incx n))
 (let ((absxi 0.0d0) (norm 0.0d0) (scale 0.0d0) (ssq 0.0d0)
       (limit 0) (t1 0.0D0))
  (declare (type fixnum limit) (type double-float absxi norm scale ssq t1))
  (cond
   ((or (< n 1) (< incx 1)) (setf norm 0.0d0))
   ((= n 1)                 (setf norm (abs (aref x 0))))
   (t
    (setf limit (min (* (- n 1) incx) (1- (length x))))
    (do ((ix 0 (+ ix incx)))
        ((> ix limit) nil)
      (unless (= 0.0d0 (the double-float (aref x ix)))
       (setf absxi (the double-float (abs (the double-float (aref x ix)))))
       (cond
         ((< scale absxi)
           (setf t1 (the double-float
                     (/ (the double-float scale) (the double-float absxi))))
           (setf ssq
            (the double-float
             (+ 1.0d0
              (the double-float
               (* (the double-float ssq)
                  (the double-float
                   (* (the double-float t1) (the double-float t1))))))))
           (setf scale absxi))
         (t
           (setf t1 (the double-float
                     (/ (the double-float absxi) (the double-float scale))))
           (setf ssq
            (the double-float
             (+ (the double-float ssq)
                (the double-float
                 (* (the double-float t1) (the double-float t1)))))))))
     (setf norm (the double-float (* scale (the double-float (sqrt ssq)))))))
    norm))
```

———-

— BLAS 1 dnrm2 lisp test —

```
(setq a (vector 3.0D0 -4.0D0 5.0D0 -7.0D0 9.0D0))
; sqrt(50.0)  = 7.0710678118654755
(dnrm2 3 a 1)
; sqrt(180.0) = 13.416407864998739
(dnrm2 5 a 1)
; sqrt(115.0) = 10.72380529476361
(dnrm2 3 a 2)
```

———-

7.7 drot - applies a plane rotation

7.7.1 Axiom unit tests

— drot.input —

```
)set break resume
)sys rm -f drot.output
)spool drot.output
)set message test on
)set message auto off
)clear all

--S 1 of 17
dx:PRIMARR(DFLOAT):=[[6,0, 1.0, 4.0, -1.0, -1.0]]
--R
--R   (1)  [6.,0.,1.,4.,- 1.,- 1.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 1

--S 2 of 17
dy:PRIMARR(DFLOAT):=[[5.0, 1.0, -4.0, 4.0, -4.0]]
--R
--R   (2)  [5.,1.,- 4.,4.,- 4.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 2

--S 3 of 17
drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate by 45 degrees
--R
--R   (3)
--R   [
```

```
--R     [7.778174591, 0.70710678100000002, - 2.1213203429999998,
--R      5.6568542480000001, - 3.5355339050000003, - 1.]
--R     ,
--R
--R     [- 0.70710678100000002, 0.70710678100000002, - 3.5355339050000003, 0.,
--R      - 2.1213203429999998]
--R     ]
--R                                     Type: List(PrimitiveArray(DoubleFloat))
--E 3

--S 4 of 17
[dx,dy] -- note that the input arguments, dx and dy were modified
--R
--R   (4)
--R   [
--R     [7.778174591, 0.70710678100000002, - 2.1213203429999998,
--R      5.6568542480000001, - 3.5355339050000003, - 1.]
--R     ,
--R
--R     [- 0.70710678100000002, 0.70710678100000002, - 3.5355339050000003, 0.,
--R      - 2.1213203429999998]
--R     ]
--R                                     Type: List(PrimitiveArray(DoubleFloat))
--E 4

--S 5 of 17
drot(5,dx,1,dy,1,0.707106781,-0.707106781) -- rotate by -45 degrees
--R
--R   (5)
--R   [
--R     [5.9999999968341839, 7.8496237287950521E-17, 0.99999999947236384,
--R      3.9999999978894558, - 0.99999999947236451, - 1.]
--R     ,
--R
--R     [4.9999999973618188, 0.99999999947236384, - 3.9999999978894558,
--R      3.9999999978894554, - 3.9999999978894554]
--R     ]
--R                                     Type: List(PrimitiveArray(DoubleFloat))
--E 5

--S 6 of 17
[dx,dy] -- note that the input arguments, dx and dy were modified
--R
--R   (6)
--R   [
--R     [5.9999999968341839, 7.8496237287950521E-17, 0.99999999947236384,
--R      3.9999999978894558, - 0.99999999947236451, - 1.]
--R     ,
--R
--R     [4.9999999973618188, 0.99999999947236384, - 3.9999999978894558,
```

```
--R      3.9999999978894554, - 3.9999999978894554]
--R     ]
--R                                        Type: List(PrimitiveArray(DoubleFloat))
--E 6

--S 7 of 17
dx:PRIMARR(DFLOAT):=[[6,0, 1.0, 4.0, -1.0, -1.0]]
--R
--R   (7)  [6.,0.,1.,4.,- 1.,- 1.]
--R                                              Type: PrimitiveArray(DoubleFloat)
--E 7

--S 8 of 17
dy:PRIMARR(DFLOAT):=[[5.0, 1.0, -4.0, 4.0, -4.0]]
--R
--R   (8)  [5.,1.,- 4.,4.,- 4.]
--R                                              Type: PrimitiveArray(DoubleFloat)
--E 8

--S 9 of 17
drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate by 45 degrees
--R
--R   (9)
--R   [
--R     [7.778174591, 0.70710678100000002, - 2.1213203429999998,
--R      5.6568542480000001, - 3.5355339050000003, - 1.]
--R     ,
--R
--R     [- 0.70710678100000002, 0.70710678100000002, - 3.5355339050000003, 0.,
--R      - 2.1213203429999998]
--R     ]
--R                                        Type: List(PrimitiveArray(DoubleFloat))
--E 9

--S 10 of 17
drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate to 90 degrees
--R
--R   (10)
--R   [
--R     [4.9999999973618197, 0.99999999947236395, - 3.9999999978894558,
--R      3.9999999978894558, - 3.9999999978894558, - 1.]
--R     ,
--R
--R     [- 5.9999999968341839, 0., - 0.99999999947236429, - 3.9999999978894558,
--R      0.99999999947236429]
--R     ]
--R                                        Type: List(PrimitiveArray(DoubleFloat))
--E 10

--S 11 of 17
```

```
drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate to 135 degrees
--R
--R   (11)
--R   [
--R     [- 0.70710678062690524, 0.70710678062690502, - 3.535533903134525, 0.,
--R      - 2.1213203418807147, - 1.]
--R     ,
--R
--R     [- 7.7781745868959549, - 0.70710678062690502, 2.1213203418807147,
--R      - 5.6568542450152401, 3.535533903134525]
--R     ]
--R                                      Type: List(PrimitiveArray(DoubleFloat))
--E 11

--S 12 of 17
drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate to 180 degrees
--R
--R   (12)
--R   [
--R     [- 5.9999999936683679, 0., - 0.99999999894472813, - 3.9999999957789121,
--R      0.99999999894472813, - 1.]
--R     ,
--R
--R     [- 4.9999999947236393, - 0.99999999894472802, 3.9999999957789116,
--R      - 3.9999999957789121, 3.9999999957789116]
--R     ]
--R                                      Type: List(PrimitiveArray(DoubleFloat))
--E 12

--S 13 of 17
drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate to 225 degrees
--R
--R   (13)
--R   [
--R     [- 7.7781745827919098, - 0.70710678025381002, 2.1213203407614296,
--R      - 5.6568542420304802, 3.5355339012690501, - 1.]
--R     ,
--R
--R     [0.70710678025381046, - 0.70710678025381002, 3.5355339012690501, 0.,
--R      2.1213203407614296]
--R     ]
--R                                      Type: List(PrimitiveArray(DoubleFloat))
--E 13

--S 14 of 17
drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate to 270 degrees
--R
--R   (14)
--R   [
--R     [- 4.999999992085459, - 0.99999999841709197, 3.9999999936683674,
```

```
--R      - 3.9999999936683679, 3.9999999936683674, - 1.]
--R     ,
--R
--R     [5.9999999905025518, 0., 0.99999999841709231, 3.9999999936683679,
--R      - 0.99999999841709231]
--R     ]
--R                                        Type: List(PrimitiveArray(DoubleFloat))
--E 14

--S 15 of 17
drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate to 315 degrees
--R
--R   (15)
--R   [
--R     [0.70710677988071569, - 0.70710677988071502, 3.5355338994035752, 0.,
--R      2.1213203396421445, - 1.]
--R     ,
--R
--R     [7.7781745786878647, 0.70710677988071502, - 2.1213203396421445,
--R      5.6568542390457202, - 3.5355338994035752]
--R     ]
--R                                        Type: List(PrimitiveArray(DoubleFloat))
--E 15

--S 16 of 17
drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate to 360 degrees
--R
--R   (16)
--R   [
--R     [5.9999999873367358, 0., 0.99999999788945637, 3.9999999915578237,
--R      - 0.99999999788945637, - 1.]
--R     ,
--R
--R     [4.9999999894472786, 0.99999999788945593, - 3.9999999915578233,
--R      3.9999999915578237, - 3.9999999915578233]
--R     ]
--R                                        Type: List(PrimitiveArray(DoubleFloat))
--E 16

--S 17 of 17
[dx,dy] -- note that the input arguments, dx and dy were modified
--R
--R   (17)
--R   [
--R     [5.9999999873367358, 0., 0.99999999788945637, 3.9999999915578237,
--R      - 0.99999999788945637, - 1.]
--R     ,
--R
--R     [4.9999999894472786, 0.99999999788945593, - 3.9999999915578233,
--R      3.9999999915578237, - 3.9999999915578233]
```

```
--R     ]
--R                                    Type: List(PrimitiveArray(DoubleFloat))
--E 17

)spool
)lisp (bye)
```

———-

7.7.2 Axiom help page

— **drot.help** —

```
=====================================================================
drot examples
=====================================================================

We set up two arrays of doublefloats, dx and dy

dx:PRIMARR(DFLOAT):=[[6,0, 1.0, 4.0, -1.0, -1.0]]

     [6.,0.,1.,4.,- 1.,- 1.]

dy:PRIMARR(DFLOAT):=[[5.0, 1.0, -4.0, 4.0, -4.0]]

     [5.,1.,- 4.,4.,- 4.]

We rotate them by 45 degrees where
   cos(45) = 0.707106781
   sin(45) = 0.707106781

drot(5,dx,1,dy,1,0.707106781,0.707106781) -- rotate by 45 degrees

   [ [7.778174591, 0.70710678100000002, - 2.1213203429999998,
      5.6568542480000001, - 3.5355339050000003, - 1.],

     [- 0.70710678100000002, 0.70710678100000002, - 3.5355339050000003, 0.,
      - 2.1213203429999998] ]

NOTE that the input arguments, dx and dy were modified, per BLAS spec.

[dx,dy]

   [ [7.778174591, 0.70710678100000002, - 2.1213203429999998,
      5.6568542480000001, - 3.5355339050000003, - 1.],

     [- 0.70710678100000002, 0.70710678100000002, - 3.5355339050000003, 0.,
      - 2.1213203429999998] ]
```

```
We rotate them by -45 degrees where
   cos(-45) =  0.707106781
   sin(-45) = -0.707106781

drot(5,dx,1,dy,1,0.707106781,-0.707106781) -- rotate by -45 degrees

   [ [5.9999999968341839, 7.8496237287950521E-17, 0.99999999947236384,
      3.9999999978894558, - 0.99999999947236451, - 1.],

     [4.9999999973618188, 0.99999999947236384, - 3.9999999978894558,
      3.9999999978894554, - 3.9999999978894554] ]


=====================================================================
Man Page Details
=====================================================================

NAME
       DROT - BLAS level one, plane rotation subroutines

SYNOPSIS
       SUBROUTINE DROT    ( n, x, incx, y, incy, c, s )

           INTEGER          n, incx, incy

           DOUBLE PRECISION x, y, c, s


DESCRIPTION
       DROT   applies  a  plane  rotation matrix to a real sequence of ordered
       pairs:

            (x  , y  ), for all i = 1, 2, ..., n.
              i    i

ARGUMENTS
       n       INTEGER. (input)
               Number of ordered pairs (planar points in DROT) to be  rotated.
               If n <= 0, this routine returns without computation.

       x       DOUBLE PRECISION, (input and output)
               Array  of dimension (n-1) * |incx| + 1.  On input, array x con-
               tains the x-coordinate of each planar point to be rotated.   On
               output,  array x contains the x-coordinate of each rotated pla-
               nar point.

       incx    INTEGER. (input)
               Increment between elements of x.  If incx = 0, the results will
               be unpredictable.
```

```
y       DOUBLE PRECISION, (input and output)
        array of dimension (n-1) * |incy| + 1.
        On  input,  array  y  contains  the y-coordinate of each planar
        point to be rotated.  On output, array y contains the y-coordi-
        nate of each rotated planar point.

incy    INTEGER. (input)
        Increment between elements of y.  If incy = 0, the results will
        be unpredictable.

c       DOUBLE PRECISION, Cosine of the angle of rotation.
        (input)

s       DOUBLE PRECISION, Sine of the angle of rotation. (input)

NOTES
       This routine applies the following plane rotation to each pair
       of elements (x , y):
                                i   i
            _      _       _      _     _      _
            | x(i) |  <--  | c   s |  .  | x(i) |
            | y(i) |       |-s   c |     | y(i) |
            -      -       -      -     -      -

            for i = 1,...,n
                                          2    2
       If coefficients c and s satisfy c  + s  = 1.0, the rotation matrix
       is orthogonal, and the transformation is called a Givens plane
       rotation. If c = 1 and s = 0, DROT returns without modifying any
       input parameters.

       To calculate the Givens coefficients c and s from a two-element
       vector to determine the angle of rotation, use SROTG(3S).

       When working backward (incx < 0 or incy < 0), each routine starts
       at the end of the vector and moves backward, as follows:

            x(1-incx * (n-1)), x(1-incx * (n-2)), ..., x(1)

            y(1-incy * (n-1)), y(1-incy * (n-2)), ..., y(1)


            ———————-
```

7.7.3 fortran code

— drot.f —

```
*  =====================================================================
      SUBROUTINE DROT(N,DX,INCX,DY,INCY,C,S)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION C,S
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION DX(*),DY(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      DOUBLE PRECISION DTEMP
      INTEGER I,IX,IY
*     ..
      IF (N.LE.0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*       code for both increments equal to 1
*
         DO I = 1,N
            DTEMP = C*DX(I) + S*DY(I)
            DY(I) = C*DY(I) - S*DX(I)
            DX(I) = DTEMP
         END DO
      ELSE
*
*       code for unequal increments or equal increments not equal
*         to 1
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
         IF (INCY.LT.0) IY = (-N+1)*INCY + 1
         DO I = 1,N
            DTEMP = C*DX(IX) + S*DY(IY)
            DY(IY) = C*DY(IY) - S*DX(IX)
            DX(IX) = DTEMP
            IX = IX + INCX
            IY = IY + INCY
         END DO
      END IF
      RETURN
```

```
      END
```

———-

7.7.4 fortran unit test

— drotg example —

```
      program drotEX
*     Tim Daly May 4, 2012
*     unit tests for BLAS drot
      double precision dx(5),dy(5)
      double precision c,s

      write(6,1)
 1    format(/,"rotate by 45, -45")
      dx= (/ 6.0d0, 1.0d0,  4.0d0, 1.0d0, -1.0d0 /)
      dy= (/ 5.0d0, 1.0d0, -4.0d0, 4.0d0, -4.0d0 /)
      write(6,10)dx(1),dy(1)
 10   format(/,"dx(1)=",f6.3," dy(1)=",f6.3)
      write(6,20)dx(2),dy(2)
 20   format("dx(2)=",f6.3," dy(2)=",f6.3)
      write(6,30)dx(3),dy(3)
 30   format("dx(3)=",f6.3," dy(2)=",f6.3)
      write(6,40)dx(4),dy(4)
 40   format("dx(4)=",f6.3," dy(4)=",f6.3)
      write(6,50)dx(5),dy(5)
 50   format("dx(5)=",f6.3," dy(5)=",f6.3)
      call drot(5,dx,1,dy,1,0.707106781d0,0.707106781d0)
      write(6,10)dx(1),dy(1)
      write(6,20)dx(2),dy(2)
      write(6,30)dx(3),dy(3)
      write(6,40)dx(4),dy(4)
      write(6,50)dx(5),dy(5)
      call drot(5,dx,1,dy,1,0.707106781d0,-0.707106781d0)
      write(6,10)dx(1),dy(1)
      write(6,20)dx(2),dy(2)
      write(6,30)dx(3),dy(3)
      write(6,40)dx(4),dy(4)
      write(6,50)dx(5),dy(5)

      write(6,2)
 2    format(/,"rotate by 45, -45, only some members")
      dx= (/ 6.0d0, 1.0d0,  4.0d0, 1.0d0, -1.0d0 /)
      dy= (/ 5.0d0, 1.0d0, -4.0d0, 4.0d0, -4.0d0 /)
      write(6,10)dx(1),dy(1)
      write(6,20)dx(2),dy(2)
```

```
      write(6,30)dx(3),dy(3)
      write(6,40)dx(4),dy(4)
      write(6,50)dx(5),dy(5)
      call drot(3,dx,2,dy,2,0.707106781d0,0.707106781d0)
      write(6,10)dx(1),dy(1)
      write(6,20)dx(2),dy(2)
      write(6,30)dx(3),dy(3)
      write(6,40)dx(4),dy(4)
      write(6,50)dx(5),dy(5)
      call drot(3,dx,2,dy,2,0.707106781d0,-0.707106781d0)
      write(6,10)dx(1),dy(1)
      write(6,20)dx(2),dy(2)
      write(6,30)dx(3),dy(3)
      write(6,40)dx(4),dy(4)
      write(6,50)dx(5),dy(5)

      write(6,3)
3     format(/,"rotate by 360")
      dx= (/ 6.0d0, 1.0d0,  4.0d0, 1.0d0, -1.0d0 /)
      dy= (/ 5.0d0, 1.0d0, -4.0d0, 4.0d0, -4.0d0 /)
      write(6,10)dx(1),dy(1)
      write(6,20)dx(2),dy(2)
      write(6,30)dx(3),dy(3)
      write(6,40)dx(4),dy(4)
      write(6,50)dx(5),dy(5)
      call drot(5,dx,1,dy,1,0.707106781d0,0.707106781d0)
      write(6,10)dx(1),dy(1)
      write(6,20)dx(2),dy(2)
      write(6,30)dx(3),dy(3)
      write(6,40)dx(4),dy(4)
      write(6,50)dx(5),dy(5)
      call drot(5,dx,1,dy,1,0.707106781d0,0.707106781d0)
      write(6,10)dx(1),dy(1)
      write(6,20)dx(2),dy(2)
      write(6,30)dx(3),dy(3)
      write(6,40)dx(4),dy(4)
      write(6,50)dx(5),dy(5)
      call drot(5,dx,1,dy,1,0.707106781d0,0.707106781d0)
      write(6,10)dx(1),dy(1)
      write(6,20)dx(2),dy(2)
      write(6,30)dx(3),dy(3)
      write(6,40)dx(4),dy(4)
      write(6,50)dx(5),dy(5)
      call drot(5,dx,1,dy,1,0.707106781d0,0.707106781d0)
      write(6,10)dx(1),dy(1)
      write(6,20)dx(2),dy(2)
      write(6,30)dx(3),dy(3)
      write(6,40)dx(4),dy(4)
      write(6,50)dx(5),dy(5)
      call drot(5,dx,1,dy,1,0.707106781d0,0.707106781d0)
```

```
      write(6,10)dx(1),dy(1)
      write(6,20)dx(2),dy(2)
      write(6,30)dx(3),dy(3)
      write(6,40)dx(4),dy(4)
      write(6,50)dx(5),dy(5)
      call drot(5,dx,1,dy,1,0.707106781d0,0.707106781d0)
      write(6,10)dx(1),dy(1)
      write(6,20)dx(2),dy(2)
      write(6,30)dx(3),dy(3)
      write(6,40)dx(4),dy(4)
      write(6,50)dx(5),dy(5)
      call drot(5,dx,1,dy,1,0.707106781d0,0.707106781d0)
      write(6,10)dx(1),dy(1)
      write(6,20)dx(2),dy(2)
      write(6,30)dx(3),dy(3)
      write(6,40)dx(4),dy(4)
      write(6,50)dx(5),dy(5)
      call drot(5,dx,1,dy,1,0.707106781d0,0.707106781d0)
      write(6,10)dx(1),dy(1)
      write(6,20)dx(2),dy(2)
      write(6,30)dx(3),dy(3)
      write(6,40)dx(4),dy(4)
      write(6,50)dx(5),dy(5)

      write(6,4)
 4    format(/,"rotate by 45, from end")
      dx= (/ 6.0d0, 1.0d0,  4.0d0, 1.0d0, -1.0d0 /)
      dy= (/ 5.0d0, 1.0d0, -4.0d0, 4.0d0, -4.0d0 /)
      write(6,10)dx(1),dy(1)
      write(6,20)dx(2),dy(2)
      write(6,30)dx(3),dy(3)
      write(6,40)dx(4),dy(4)
      write(6,50)dx(5),dy(5)
      call drot(2,dx,-1,dy,-1,0.707106781d0,0.707106781d0)
      write(6,10)dx(1),dy(1)
      write(6,20)dx(2),dy(2)
      write(6,30)dx(3),dy(3)
      write(6,40)dx(4),dy(4)
      write(6,50)dx(5),dy(5)
      stop
      end
```

———-

```
gcc -o drotEX drotEX.f -lgfortran drot.o && ./drotEX

rotate by 45, -45

dx(1)= 6.000 dy(1)= 5.000
dx(2)= 1.000 dy(2)= 1.000
```

```
dx(3)= 4.000 dy(2)=-4.000
dx(4)=-1.000 dy(4)= 4.000
dx(5)=-1.000 dy(5)=-4.000

dx(1)= 7.778 dy(1)=-0.707
dx(2)= 1.414 dy(2)= 0.000
dx(3)= 0.000 dy(2)=-5.657
dx(4)= 2.121 dy(4)= 3.536
dx(5)=-3.536 dy(5)=-2.121

dx(1)= 6.000 dy(1)= 5.000
dx(2)= 1.000 dy(2)= 1.000
dx(3)= 4.000 dy(2)=-4.000
dx(4)=-1.000 dy(4)= 4.000
dx(5)=-1.000 dy(5)=-4.000

rotate by 45, -45, only some members

dx(1)= 6.000 dy(1)= 5.000
dx(2)= 1.000 dy(2)= 1.000
dx(3)= 4.000 dy(2)=-4.000
dx(4)=-1.000 dy(4)= 4.000
dx(5)=-1.000 dy(5)=-4.000

dx(1)= 7.778 dy(1)=-0.707
dx(2)= 1.000 dy(2)= 1.000
dx(3)= 0.000 dy(2)=-5.657
dx(4)=-1.000 dy(4)= 4.000
dx(5)=-3.536 dy(5)=-2.121

dx(1)= 6.000 dy(1)= 5.000
dx(2)= 1.000 dy(2)= 1.000
dx(3)= 4.000 dy(2)=-4.000
dx(4)=-1.000 dy(4)= 4.000
dx(5)=-1.000 dy(5)=-4.000

rotate by 360

dx(1)= 6.000 dy(1)= 5.000
dx(2)= 1.000 dy(2)= 1.000
dx(3)= 4.000 dy(2)=-4.000
dx(4)=-1.000 dy(4)= 4.000
dx(5)=-1.000 dy(5)=-4.000

dx(1)= 7.778 dy(1)=-0.707
dx(2)= 1.414 dy(2)= 0.000
dx(3)= 0.000 dy(2)=-5.657
dx(4)= 2.121 dy(4)= 3.536
dx(5)=-3.536 dy(5)=-2.121
```

```
dx(1)= 5.000 dy(1)=-6.000
dx(2)= 1.000 dy(2)=-1.000
dx(3)=-4.000 dy(2)=-4.000
dx(4)= 4.000 dy(4)= 1.000
dx(5)=-4.000 dy(5)= 1.000

dx(1)=-0.707 dy(1)=-7.778
dx(2)= 0.000 dy(2)=-1.414
dx(3)=-5.657 dy(2)= 0.000
dx(4)= 3.536 dy(4)=-2.121
dx(5)=-2.121 dy(5)= 3.536

dx(1)=-6.000 dy(1)=-5.000
dx(2)=-1.000 dy(2)=-1.000
dx(3)=-4.000 dy(2)= 4.000
dx(4)= 1.000 dy(4)=-4.000
dx(5)= 1.000 dy(5)= 4.000

dx(1)=-7.778 dy(1)= 0.707
dx(2)=-1.414 dy(2)= 0.000
dx(3)= 0.000 dy(2)= 5.657
dx(4)=-2.121 dy(4)=-3.536
dx(5)= 3.536 dy(5)= 2.121

dx(1)=-5.000 dy(1)= 6.000
dx(2)=-1.000 dy(2)= 1.000
dx(3)= 4.000 dy(2)= 4.000
dx(4)=-4.000 dy(4)=-1.000
dx(5)= 4.000 dy(5)=-1.000

dx(1)= 0.707 dy(1)= 7.778
dx(2)= 0.000 dy(2)= 1.414
dx(3)= 5.657 dy(2)= 0.000
dx(4)=-3.536 dy(4)= 2.121
dx(5)= 2.121 dy(5)=-3.536

dx(1)= 6.000 dy(1)= 5.000
dx(2)= 1.000 dy(2)= 1.000
dx(3)= 4.000 dy(2)=-4.000
dx(4)=-1.000 dy(4)= 4.000
dx(5)=-1.000 dy(5)=-4.000

rotate by 45, from end

dx(1)= 6.000 dy(1)= 5.000
dx(2)= 1.000 dy(2)= 1.000
dx(3)= 4.000 dy(2)=-4.000
dx(4)=-1.000 dy(4)= 4.000
dx(5)=-1.000 dy(5)=-4.000
```

```
dx(1)= 7.778 dy(1)=-0.707
dx(2)= 1.414 dy(2)= 0.000
dx(3)= 4.000 dy(2)=-4.000
dx(4)=-1.000 dy(4)= 4.000
dx(5)=-1.000 dy(5)=-4.000
```

7.7.5 lisp code

— BLAS 1 drot —

```
(defun drot (n dx incx dy incy c s)
; Tim Daly May 4, 2012
 (declare (type (double-float) c s)
          (type (simple-array double-float (*)) dx dy)
          (type fixnum incy incx n))
 (let ((dtemp 0.0d0) (ix 0) (iy 0))
  (declare (type (double-float) dtemp) (type fixnum iy ix))
  (when (> n 0)
    (if (< incx 0)
     (setf ix (the fixnum (* (the fixnum (1+ (the fixnum (- n)))) incx))))
    (if (< incy 0)
     (setf iy (the fixnum (* (the fixnum (1+ (the fixnum (- n)))) incy))))
    (dotimes (i n)
     (setf dtemp (the double-float
      (+ (the double-float (* c (the double-float (svref dx ix))))
         (the double-float (* s (the double-float (svref dy iy)))))))
     (setf (the double-float (svref dy iy)) (the double-float
      (- (* c (the double-float (svref dy iy)))
         (* s (the double-float (svref dx ix))))))
     (setf (the double-float (svref dx ix)) (the double-float dtemp))
     (setf ix (the fixnum (+ ix incx)))
     (setf iy (the fixnum (+ iy incy)))))
  (list dx dy)))
```

———-

— BLAS 1 drot lisp test —

```
(load "drot.lisp")
; rotate by 45, -45
(setq dx (vector 6.0d0 1.0d0 4.0d0 -1.0d0 -1.0d0))
; #(6.0 1.0 4.0 -1.0 -1.0)

(setq dy (vector 5.0d0 1.0d0 -4.0d0 4.0d0 -4.0d0))
; #(5.0 1.0 -4.0 4.0 -4.0)
```

```
(drot 5 dx 1 dy 1 0.707106781d0 0.707106781d0)
; (#(7.778174591 1.414213562 0.0 2.1213203429999998 -3.5355339050000003)
;  #(-0.70710678099999935 0.0 -5.6568542480000001 3.5355339050000003
;   -2.1213203429999998))

(drot 5 dx 1 dy 1 0.707106781d0 -0.707106781d0)
; (#(5.9999999968341831 0.99999999947236395 3.9999999978894558
;    -0.99999999947236451 -0.99999999947236451)
;  #(4.9999999973618197 0.99999999947236395 -3.9999999978894558
;    3.9999999978894563 -3.9999999978894563))

; rotate by 45, -45, only some members

(setq dx (vector 6.0d0 1.0d0 4.0d0 -1.0d0 -1.0d0))
; #(6.0 1.0 4.0 -1.0 -1.0)

(setq dy (vector 5.0d0 1.0d0 -4.0d0 4.0d0 -4.0d0))
; #(5.0 1.0 -4.0 4.0 -4.0)

(drot 3 dx 2 dy 2 0.707106781d0 0.707106781d0)
; (#(7.778174591 1.0 0.0 -1.0 -3.5355339050000003)
;  #(-0.70710678099999935 1.0 -5.6568542480000001 4.0 -2.1213203429999998))

(drot 3 dx 2 dy 2 0.707106781d0 -0.707106781d0)
; (#(5.9999999968341831 1.0 3.9999999978894558 -1.0 -0.99999999947236451)
;  #(4.9999999973618197 1.0 -3.9999999978894558 4.0 -3.9999999978894563))

; rotate by 360

(setq dx (vector 6.0d0 1.0d0 4.0d0 -1.0d0 -1.0d0))
; #(6.0 1.0 4.0 -1.0 -1.0)

(setq dy (vector 5.0d0 1.0d0 -4.0d0 4.0d0 -4.0d0))
; #(5.0 1.0 -4.0 4.0 -4.0)

(drot 5 dx 1 dy 1 0.707106781d0 0.707106781d0)
; (#(7.778174591 1.414213562 0.0 2.1213203429999998 -3.5355339050000003)
;  #(-0.70710678099999935 0.0 -5.6568542480000001 3.5355339050000003
;    -2.1213203429999998))

(drot 5 dx 1 dy 1 0.707106781d0 0.707106781d0)
; (#(4.9999999973618197 0.99999999947236395 -3.9999999978894558
;    3.9999999978894563 -3.9999999978894563)
;  #(-5.9999999968341831 -0.99999999947236395 -3.9999999978894558
;    0.99999999947236451 0.99999999947236451))

(drot 5 dx 1 dy 1 0.707106781d0 0.707106781d0)
; (#(-0.70710678062690446 0.0 -5.6568542450152401 3.5355339031345254
;    -2.1213203418807147)
```

```
;  #(-7.778174586895954 -1.41421356125381 0.0 -2.1213203418807147
;    3.5355339031345254))

(drot 5 dx 1 dy 1 0.707106781d0 0.707106781d0)
; (#(-5.9999999936683661 -0.99999999894472802 -3.9999999957789121
;    0.99999999894472835 0.99999999894472835)
;  #(-4.9999999947236393 -0.99999999894472802 3.9999999957789121
;    -3.9999999957789116 3.9999999957789116))

(drot 5 dx 1 dy 1 0.707106781d0 0.707106781d0)
; (#(-7.7781745827919089 -1.41421356050762 0.0 -2.1213203407614296
;    3.5355339012690505)
;  #(0.70710678025380957 0.0 5.6568542420304802 -3.5355339012690505
;    2.1213203407614296))

(drot 5 dx 1 dy 1 0.707106781d0 0.707106781d0)
; (#(-4.999999992085459 -0.99999999841709197 3.9999999936683679
;    -3.9999999936683679 3.9999999936683679)
;  #(5.999999990502551 0.99999999841709197 3.9999999936683679
;   -0.99999999841709286 -0.99999999841709286))

(drot 5 dx 1 dy 1 0.707106781d0 0.707106781d0)
; (#(0.70710677988071557 0.0 5.6568542390457202 -3.5355338994035757
;    2.1213203396421445)
;  #(7.7781745786878638 1.41421355976143 0.0 2.1213203396421445
;    -3.5355338994035757))

(drot 5 dx 1 dy 1 0.707106781d0 0.707106781d0)
; (#(5.9999999873367349 0.99999999788945593 3.9999999915578237
;    -0.9999999978894567 -0.9999999978894567)
;  #(4.9999999894472777 0.99999999788945593 -3.9999999915578237
;    3.9999999915578237 -3.9999999915578237))

; rotate by 45, -45 from end

(setq dx (vector 6.0d0 1.0d0 4.0d0 -1.0d0 -1.0d0))
; #(6.0 1.0 4.0 -1.0 -1.0)

(setq dy (vector 5.0d0 1.0d0 -4.0d0 4.0d0 -4.0d0))
; #(5.0 1.0 -4.0 4.0 -4.0)

(drot 2 dx -1 dy -1 0.707106781d0 0.707106781d0)
; (#(7.778174591 1.414213562 4.0 -1.0 -1.0)
;  #(-0.70710678099999935 0.0 -4.0 4.0 -4.0))
```

7.8 drotg - constructs givens plane rotation

7.8.1 Axiom unit tests

— drotg.input —

```
)set break resume
)sys rm -f drotg.output
)spool drotg.output
)set message test on
)set message auto off
)clear all

--S 1 of 8
a:MATRIX(DFLOAT):=[[6,5,0],[5,1,4],[0,4,3]]
--R
--R        +6.  5.  0.+
--R        |          |
--R   (1)  |5.  1.  4.|
--R        |          |
--R        +0.  4.  3.+
--R                                                Type: Matrix(DoubleFloat)
--E 1

--S 2 of 8
t1:=drotg(elt(a,1,1),elt(a,1,2),0.0,0.0)
--R
--R   (2)
--R   [7.810249675906654, 0.64018439966447993, 0.76822127959737585,
--R    0.64018439966447993]
--R                                            Type: PrimitiveArray(DoubleFloat)
--E 2

--S 3 of 8
g1:MATRIX(DFLOAT):=[[elt(t1,2), elt(t1,3),0.0],_
                    [-elt(t1,3),elt(t1,2),0.0],_
                    [0.0,       0.0,      1.0]]
--R
--R
--R        + 0.76822127959737585   0.64018439966447993  0.+
--R        |                                            |
--R   (3)  |- 0.64018439966447993  0.76822127959737585  0.|
--R        |                                            |
--R        +         0.                    0.           1.+
--R                                                Type: Matrix(DoubleFloat)
--E 3

--S 4 of 8
```

```
t2:=g1*a
--R
--R        +   7.810249675906654       4.4812907976513596   2.5607375986579197+
--R        |                                                                  |
--R   (4)  |- 4.4408920985006262E-16  - 2.4327007187250236  3.0728851183895034|
--R        |                                                                  |
--R        +           0.                      4.                   3.        +
--R                                                   Type: Matrix(DoubleFloat)
--E 4

--S 5 of 8
t3:=drotg(elt(t2,2,2),elt(a,3,2),0.0,0.0)
--R
--R   (5)
--R   [4.6816698716254272, - 1.924474241977076, - 0.51962243930719854,
--R    0.85439599751428896]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 5

--S 6 of 8
g2:MATRIX(DFLOAT):=[[1.0, 0.0,      0.0],_
                    [0.0, elt(t3,2),elt(t3,3)],_
                    [0.0,-elt(t3,3),elt(t3,2)]]
--R
--R
--R        +1.           0.                     0.          +
--R        |                                                 |
--R   (6)  |0.  - 0.51962243930719854   0.85439599751428896 |
--R        |                                                 |
--R        +0.  - 0.85439599751428896  - 0.51962243930719854+
--R                                                   Type: Matrix(DoubleFloat)
--E 6

--S 7 of 8
g2*g1*a
--R
--R        +   7.810249675906654      4.4812907976513596   2.5607375986579197 +
--R        |                                                                  |
--R   (7)  | 2.2204460492503131E-16   4.6816698716254272  0.96644793161452336 |
--R        |                                                                  |
--R        +- 4.4408920985006262E-16          0.          - 4.1843280638948093+
--R                                                   Type: Matrix(DoubleFloat)
--E 7

--S 8 of 8
q:=transpose(g1)*transpose(g2)
--R
--R        +0.76822127959737585   0.33265417936007158    0.54697098874441952 +
--R        |                                                                  |
--R   (8)  |0.64018439966447993  - 0.39918501523208583  - 0.65636518649330344|
```

```
--R        |                                                                |
--R        +        0.            0.85439599751428896   - 0.51962243930719854+
--R                                                   Type: Matrix(DoubleFloat)
--E 8

)spool
)lisp (bye)
```

———-

7.8.2 Axiom help page

— drotg.help —

```
=====================================================================
drotg examples
=====================================================================

A Givens rotation is a rotation in the plane spanned by two
coordinate axes, named after Wallace Givens. [REF-Wiki3]

A Givens rotation is represented by a matrix of the form

               +-                          -+
               | 1  ...  0  ...  0  ...  0 |
               | .   .   .   .   .   .   . |
               | .    .   .    .   .    . |
               | .    .   .    .   .    . |
               | 0  ...  c  ... -s  ...  0 |
               | .   .   .   .   .   .   . |
G(i,j,theta) = | .    .   .    .   .    . |
               | .    .   .    .   .    . |
               | 0  ...  s  ...  c  ...  0 |
               | .   .   .   .   .   .   . |
               | .    .   .    .   .    . |
               | .    .   .    .   .    . |
               | 0  ...  0  ...  0  ...  1 |
               +-                          -+

where c = cos(theta) and s = sin(theta) appear at the intersections
of the ith, jth rows and columns. The non-zero elements of a Givens
matrix are given by:

g   = 1 for k != i,j
 kk
g   = c
 ii
g   = c              (sign of sine switches for j>i)
```

```
 jj
g   = -s
 ji
g   = s  for i > j
 ij
```

The product G(i,j,theta)*x represents the counterclockwise rotation of the vector in the (i,j) plane of theta radians.

The main use of Givens rotations in numerical linear algebra is to introduce zeros in vectors or matrices. This effect can be employed for computing the QR decomposition of a matrix. One advantage over the Householder tranformations is that they can easily be parallelized, and another is that often for very sparse matrices they have a lower operation count.

When a Givens rotation matrix, G(i,k,theta) multiplies another matrix A from the left, GA, only rows i and j of A are affected. Thus we restrict attention to the following problem. Given a and b, find c=cos(theta) and s=sin(theta) such that

```
+-     -+ +- -+   +- -+
| c  -s | | a |   | r |
|       | |   | = |   |
| s   c | | b |   | 0 |
+-     -+ +- -+   +- -+
```

The solution is

```
r = sqrt(a^2+b^2)
c = a/r
s = -b/r
```

However, the computation for r may overflow or underflow. An alternative formulation avoiding this problem [REF-GC96, p5.1.8] is implemented as the hypot function in many programming languages.

As Anderson [REF-And00] discovered in improving LAPACK, a previously overlooked numerical consideration is continuity. To achieve this we require r to be positive.

```
   if (b = 0) then { c = copysign(1,a); s=0; r=abs(a) }
   else if (a = 0) then { c = 0; s = copysign(1,b); r = abs(b) }
   else if (abs(b) > abs(a)) then
     t = a/b
     u = copysign(sqrt(1+t*t),b)
     s = 1/u
     c = s*t
     r = b*u
   else
```

```
     t = b/a
     u = copysign(sqrt(1+t*t),a)
     c = 1/u
     s = c*t
     r = a*u

copysign can be implemented as x*sgn(y) using the sign function.

=====================================================================
Man Page Details
=====================================================================

NAME
       DGROTG - BLAS level one rotation subroutines

SYNOPSIS
       SUBROUTINE DROTG    ( a, b, c, s )

           DOUBLE PRECISION  a, b, c, s


DESCRIPTION
       DROTG  computes  the  elements  of  a Givens plane rotation matrix such
       that:

             _      _     _   _    _   _
             | c  s |     | a |    | r |
             |-s  c | *   | b | =  | 0 |
             _      _     _   _    _   _

       where  r = +- sqrt ( a**2 + b**2 )  and c**2 + s**2 =1.

       The Givens plane rotation can be used to introduce zero elements into
       a matrix selectively.



ARGUMENTS
       a       (input and output) DOUBLE PRECISION

               First vector component.  On input, the first component  of  the
               vector to be rotated.  On output, a is overwritten by r, the
               first component of the vector in the rotated coordinate  system
               where:

               r = sign(sqrt(a**2 + b**2),a), if |a| > |b|

               r = sign(sqrt(a**2 + b**2),b), if |a| <= |b|

       b       (input and output) DOUBLE PRECISION
```

```
            Second vector component.

            On input, the second component of the vector to be rotated.  On
            output, b contains z, where:

                z=s    if  |a| >  |b|
                z=1/c  if  |a| <= |b|  and  c != 0  and r != 0
                z=1    if  |a| <= |b|  and  c  = 0  and r != 0
                z=0    if  r = 0

    c       (output) DOUBLE PRECISION
            Cosine of the angle of rotation:

                 c = a/r  if  r != 0
                 c = 1    if  r  = 0

    s       (output) DOUBLE PRECISION
            Sine of the angle of rotation:

                 s = b/r  if  r != 0
                 s = 0    if  r  = 0


NOTE
    The value of z, returned in b by DROTG, gives a compact  representation
    of  the rotation matrix, which can be used later to reconstruct c and s
    as in the following example:

         IF (B .EQ. 1. ) THEN
             C = 0.
             S = 1.
         ELSEIF( ABS( B) .LT. 1) THEN
             C = SQRT( 1. - B * B)
             S = B
         ELSE
             C = 1. / B
             S = SQRT( 1 - C * C)
         ENDIF
```

———-

Double precision. Computes plane rotation. Arguments are:

- da - double-float
- db - double-float
- c - double-float
- s - double-float

Returns multiple values where:

- 1 da - double-float
- 2 db - double-float
- 3 c - double-float
- 4 s - double-float

7.8.3 fortran code

— drotg.f —

```
*  =====================================================================
      SUBROUTINE DROTG(DA,DB,C,S)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION C,DA,DB,S
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      DOUBLE PRECISION R,ROE,SCALE,Z
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DABS,DSIGN,DSQRT
*     ..
      ROE = DB
      IF (DABS(DA).GT.DABS(DB)) ROE = DA
      SCALE = DABS(DA) + DABS(DB)
      IF (SCALE.EQ.0.0d0) THEN
         C = 1.0d0
         S = 0.0d0
         R = 0.0d0
         Z = 0.0d0
      ELSE
         R = SCALE*DSQRT((DA/SCALE)**2+ (DB/SCALE)**2)
         R = DSIGN(1.0d0,ROE)*R
         C = DA/R
         S = DB/R
         Z = 1.0d0
```

```
      IF (DABS(DA).GT.DABS(DB)) Z = S
      IF (DABS(DB).GE.DABS(DA) .AND. C.NE.0.0d0) Z = 1.0d0/C
    END IF
    DA = R
    DB = Z
    RETURN
    END
```

7.8.4 fortran unit test

— drotg example —

```
      program drotgEX
*     Tim Daly May 2, 2012
*     unit tests for BLAS drotg
      double precision a11,a12,a13,a21,a22,a23,a31,a32,a33
      double precision b11,b12,b13,b21,b22,b23,b31,b32,b33
      double precision c11,c12,c13,c21,c22,c23,c31,c32,c33
      double precision d11,d12,d13,d21,d22,d23,d31,d32,d33
      double precision e11,e12,e13,e21,e22,e23,e31,e32,e33
      double precision f11,f12,f13,f21,f22,f23,f31,f32,f33
      double precision a,b,c,s
      a11=6.0d0
      a12=5.0d0
      a13=0.0d0
      a21=5.0d0
      a22=1.0d0
      a23=4.0d0
      a31=0.0d0
      a32=4.0d0
      a33=3.0d0
      write(6,10)
      write(6,20)
      write(6,30)a11,a12,a13
      write(6,30)a21,a22,a23
      write(6,30)a31,a32,a33
      write(6,20)
 10   format("A=");
 20   format("  +-                      -+")
 30   format("  | ",f6.3," ",f6.3," ",f6.3,"  |")

      a=a11
      b=a21
      c=0.0d0
      s=0.0d0
```

```
      write(6,100)a,b,c,s
100   format(/,"a=",f6.3," b=",f6.3," c=",f6.3," s=",f6.3);

      call drotg(a,b,c,s)
      write(6,100)a,b,c,s

      b11=c
      b12=s
      b13=0.0d0
      b21=-s
      b22=c
      b23=0.0d0
      b31=0.0d0
      b32=0.0d0
      b33=1.0d0
      write(6,11)
11    format(/,"G1=")
      write(6,20)
      write(6,30)b11,b12,b13
      write(6,30)b21,b22,b23
      write(6,30)b31,b32,b33
      write(6,20)

      c11=b11*a11+b12*a21+b13*a31
      c21=b21*a11+b22*a21+b23*a31
      c31=b31*a11+b32*a21+b33*a31
      c12=b11*a12+b12*a22+b13*a32
      c22=b21*a12+b22*a22+b23*a32
      c32=b31*a12+b32*a22+b33*a32
      c13=b11*a13+b12*a23+b13*a33
      c23=b21*a13+b22*a23+b23*a33
      c33=b31*a13+b32*a23+b33*a33
      write(6,12)
12    format(/,"G1*A=")
      write(6,20)
      write(6,30)c11,c12,c13
      write(6,30)c21,c22,c23
      write(6,30)c31,c32,c33
      write(6,20)

      a=c22
      b=c32
      c=0.0d0
      s=0.0d0
      write(6,100)a,b,c,s

      call drotg(a,b,c,s)
      write(6,100)a,b,c,s

      d11=1.0d0
```

```
      d12=0.0d0
      d13=0.0d0
      d21=0.0d0
      d22=c
      d23=s
      d31=0.0d0
      d32=-s
      d33=c
      write(6,13)
13    format(/,"G2=")
      write(6,20)
      write(6,30)d11,d12,d13
      write(6,30)d21,d22,d23
      write(6,30)d31,d32,d33
      write(6,20)

      e11=d11*c11+d12*c21+d13*c31
      e21=d21*c11+d22*c21+d23*c31
      e31=d31*c11+d32*c21+d33*c31
      e12=d11*c12+d12*c22+d13*c32
      e22=d21*c12+d22*c22+d23*c32
      e32=d31*c12+d32*c22+d33*c32
      e13=d11*c13+d12*c23+d13*c33
      e23=d21*c13+d22*c23+d23*c33
      e33=d31*c13+d32*c23+d33*c33
      write(6,14)
14    format(/,"G2*G1*A=")
      write(6,20)
      write(6,30)e11,e12,e13
      write(6,30)e21,e22,e23
      write(6,30)e31,e32,e33
      write(6,20)

      f11=b31*d13 + b21*d12 + b11*d11
      f12=b31*d23 + b21*d22 + b11*d21
      f13=b31*d33 + b21*d32 + b11*d31
      f21=b32*d13 + b22*d12 + b12*d11
      f22=b32*d23 + b22*d22 + b12*d21
      f23=b32*d33 + b22*d32 + b12*d31
      f31=b33*d13 + b23*d12 + b13*d11
      f32=b33*d23 + b23*d22 + b13*d21
      f33=b33*d33 + b23*d32 + b13*d31
      write(6,15)
15    format(/,"Q=transpose(G1)*transpose(G2)")
      write(6,20)
      write(6,30)f11,f12,f13
      write(6,30)f21,f22,f23
      write(6,30)f31,f32,f33
      write(6,20)
```

```
      stop
      end
```

```
gcc -o drotgEX drotgEX.f -lgfortran drotg.o && ./drotgEX

A=
  +-                      -+
  |  6.000  5.000  0.000  |
  |  5.000  1.000  4.000  |
  |  0.000  4.000  3.000  |
  +-                      -+

a= 6.000 b= 5.000 c= 0.000 s= 0.000

a= 7.810 b= 0.640 c= 0.768 s= 0.640

G1=
  +-                      -+
  |  0.768  0.640  0.000  |
  | -0.640  0.768  0.000  |
  |  0.000  0.000  1.000  |
  +-                      -+

G1*A=
  +-                      -+
  |  7.810  4.481  2.561  |
  | -0.000 -2.433  3.073  |
  |  0.000  4.000  3.000  |
  +-                      -+

a=-2.433 b= 4.000 c= 0.000 s= 0.000

a= 4.682 b=-1.924 c=-0.520 s= 0.854

G2=
  +-                      -+
  |  1.000  0.000  0.000  |
  |  0.000 -0.520  0.854  |
  |  0.000 -0.854 -0.520  |
  +-                      -+

G2*G1*A=
  +-                      -+
  |  7.810  4.481  2.561  |
  |  0.000  4.682  0.966  |
  |  0.000  0.000 -4.184  |
  +-                      -+
```

```
Q=transpose(G1)*transpose(G2)
  +-                      -+
  |  0.768  0.333  0.547  |
  |  0.640 -0.399 -0.656  |
  |  0.000  0.854 -0.520  |
  +-                      -+
```

7.8.5 lisp code

— BLAS 1 drotg —

```
(defun drotg (da db c s)
 ; Tim Daly May 2, 2012
 (declare (type (double-float) s c db da))
 (let ((roe 0.0d0) (scale 0.0d0) (r 0.0d0) (z 0.0d0))
  (declare (type (double-float) z r scale roe))
   (setf roe db)
   (when (> (the double-float (abs da)) (the double-float (abs db)))
    (setf roe da))
   (setf scale (+ (the double-float (abs da)) (the double-float (abs db))))
   (if (/= scale 0.0d0)
    (progn
     (setf r
      (the double-float (*
        scale
        (the double-float (sqrt
         (the double-float (+
          (the double-float (*
           (the double-float (/ da scale))
           (the double-float (/ da scale))))
          (the double-float (*
           (the double-float (/ db scale))
           (the double-float (/ db scale)))))))))))
     (setf r (* (float-sign roe 1.0d0) r))
     (setf c (the double-float (/ da r)))
     (setf s (the double-float (/ db r)))
     (setf z 1.0d0)
     (when (> (the double-float (abs da)) (the double-float (abs db)))
       (setf z s))
     (if (and (>= (the double-float (abs db)) (the double-float (abs da)))
              (/= c 0.0))
       (setf z (the double-float (/ 1.0d0 c))))
     (make-array 4 :initial-contents (list r z c s)))
    (make-array 4 :initial-contents (list 0.0d0 0.0d0 1.0d0 0.0d0)))))
```

———-

— BLAS 1 drotg lisp test —

```
(load "drotg.lisp")
(defun m (m i j) (svref (svref m (1- i)) (1- j)))
(defun matprint (mat)
  (format t "+-                        -+~%")
  (format t "|  ~6,3f ~6,3f ~6,3f  |~%" (m mat 1 1) (m mat 1 2) (m mat 1 3))
  (format t "|  ~6,3f ~6,3f ~6,3f  |~%" (m mat 2 1) (m mat 2 2) (m mat 2 3))
  (format t "|  ~6,3f ~6,3f ~6,3f  |~%" (m mat 3 1) (m mat 3 2) (m mat 3 3))
  (format t "+-                        -+~%"))
(defun matmult (a b)
 (vector
  (vector (+ (* (m a 1 1) (m b 1 1))
             (* (m a 1 2) (m b 2 1))
             (* (m a 1 3) (m b 3 1)))
          (+ (* (m a 1 1) (m b 1 2))
             (* (m a 1 2) (m b 2 2))
             (* (m a 1 3) (m b 3 2)))
          (+ (* (m a 1 1) (m b 1 3))
             (* (m a 1 2) (m b 2 3))
             (* (m a 1 3) (m b 3 3))))
  (vector (+ (* (m a 2 1) (m b 1 1))
             (* (m a 2 2) (m b 2 1))
             (* (m a 2 3) (m b 3 1)))
          (+ (* (m a 2 1) (m b 1 2))
             (* (m a 2 2) (m b 2 2))
             (* (m a 2 3) (m b 3 2)))
          (+ (* (m a 2 1) (m b 1 3))
             (* (m a 2 2) (m b 2 3))
             (* (m a 2 3) (m b 3 3))))
  (vector (+ (* (m a 3 1) (m b 1 1))
             (* (m a 3 2) (m b 2 1))
             (* (m a 3 3) (m b 3 1)))
          (+ (* (m a 3 1) (m b 1 2))
             (* (m a 3 2) (m b 2 2))
             (* (m a 3 3) (m b 3 2)))
          (+ (* (m a 3 1) (m b 1 3))
             (* (m a 3 2) (m b 2 3))
             (* (m a 3 3) (m b 3 3))))))
(defun transpose (mat)
 (vector (vector (m mat 1 1) (m mat 1 2) (m mat 1 3))
         (vector (m mat 2 1) (m mat 2 2) (m mat 2 3))
         (vector (m mat 3 1) (m mat 3 2) (m mat 3 3))))
(setq a #(#(6.0d0 5.0d0 0.0d0)
          #(5.0d0 1.0d0 4.0d0)
          #(0.0d0 4.0d0 3.0d0)))
(multiple-value-setq (x y c s) (drotg (m a 1 1) (m a 2 1) 0.0d0 0.0d0))
(list x y c s)
(setq g1 (vector (vector c         s 0.0d0)
```

```
                (vector (- s)     c 0.0d0)
                (vector 0.0d0 0.0d0 1.0d0)))
(matprint (setq g1a (matmult g1 a)))
(multiple-value-setq (xx yy cc ss) (drotg (m g1a 2 2) (m g1a 3 2) 0.0d0 0.0d0))
(list xx yy cc ss)
(matprint (setq g2 (vector (vector 1.0d0 0.0d0 0.0d0) (vector 0.0d0 cc ss) (vector 0.0d0 (- ss)
(matprint (setq g2 (matmult g1 a)))
```

7.9 drotm - Apply the modified givens transformation to DX^T

7.9.1 Axiom unit tests

— drotm.input —

```
)set break resume
)sys rm -f drotm.output
)spool drotm.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

7.9.2 Axiom help page

— drotm.help —

```
=====================================================================
drotm examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
```

```
       subroutine drotm (N, DX, INCX, DY, INCY, DPARAM)
           DROTM

Function/Subroutine Documentation
   subroutine drotm (integer N, double precision, dimension(*) DX, integer
       INCX, double precision, dimension(*) DY, integer INCY, double
       precision, dimension(5) DPARAM)
       DROTM

Purpose:
   Apply the modified GIVENS transformation, H, to the 2 by N matrix
   (DX**T) , where **T indicates transpose. the elements of DX are in
   (DY**T)

  DX(LX+I*INCX), I = 0 TO N-1, where LX = 1 IF INCX .GE. 0, ELSE
  LX = (-INCX)*N, and similarly for SY using LY and INCY.
  with DPARAM(1)=DFLAG, H has one of the following forms..

  DFLAG=-1.D0     DFLAG=0.D0        DFLAG=1.D0     DFLAG=-2.D0

    (DH11  DH12)    (1.D0  DH12)    (DH11  1.D0)    (1.D0  0.D0)
  H=(          )    (          )    (          )    (          )
    (DH21  DH22),   (DH21  1.D0),   (-1.D0 DH22),   (0.D0  1.D0).
  see DROTMG for a description of data storage in DPARAM.

Parameters:
 N
   N is INTEGER
   number of elements in input vector(s)

 DX
   DX is DOUBLE PRECISION array, dimension N
   double precision vector with N elements

 INCX
   INCX is INTEGER
   storage spacing between elements of DX

 DY
   DY is DOUBLE PRECISION array, dimension N
   double precision vector with N elements

 INCY
   INCY is INTEGER
   storage spacing between elements of DY

 DPARAM
   DPARAM is DOUBLE PRECISION array, dimension 5
     DPARAM(1)=DFLAG
     DPARAM(2)=DH11
```

```
      DPARAM(3)=DH21
      DPARAM(4)=DH12
      DPARAM(5)=DH22
```

———-

7.9.3 fortran code

— **drotm.f** —

```
*  ====================================================================
      SUBROUTINE DROTM(N,DX,INCX,DY,INCY,DPARAM)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION DPARAM(5),DX(*),DY(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      DOUBLE PRECISION DFLAG,DH11,DH12,DH21,DH22,TWO,W,Z,ZERO
      INTEGER I,KX,KY,NSTEPS
*     ..
*     .. Data statements ..
      DATA ZERO,TWO/0.D0,2.D0/
*     ..
*
      DFLAG = DPARAM(1)
      IF (N.LE.0 .OR. (DFLAG+TWO.EQ.ZERO)) RETURN
      IF (INCX.EQ.INCY.AND.INCX.GT.0) THEN
*
         NSTEPS = N*INCX
         IF (DFLAG.LT.ZERO) THEN
            DH11 = DPARAM(2)
            DH12 = DPARAM(4)
            DH21 = DPARAM(3)
            DH22 = DPARAM(5)
            DO I = 1,NSTEPS,INCX
               W = DX(I)
```

```
            Z = DY(I)
            DX(I) = W*DH11 + Z*DH12
            DY(I) = W*DH21 + Z*DH22
         END DO
      ELSE IF (DFLAG.EQ.ZERO) THEN
         DH12 = DPARAM(4)
         DH21 = DPARAM(3)
         DO I = 1,NSTEPS,INCX
            W = DX(I)
            Z = DY(I)
            DX(I) = W + Z*DH12
            DY(I) = W*DH21 + Z
         END DO
      ELSE
         DH11 = DPARAM(2)
         DH22 = DPARAM(5)
         DO I = 1,NSTEPS,INCX
            W = DX(I)
            Z = DY(I)
            DX(I) = W*DH11 + Z
            DY(I) = -W + DH22*Z
         END DO
      END IF
   ELSE
      KX = 1
      KY = 1
      IF (INCX.LT.0) KX = 1 + (1-N)*INCX
      IF (INCY.LT.0) KY = 1 + (1-N)*INCY
*
      IF (DFLAG.LT.ZERO) THEN
         DH11 = DPARAM(2)
         DH12 = DPARAM(4)
         DH21 = DPARAM(3)
         DH22 = DPARAM(5)
         DO I = 1,N
            W = DX(KX)
            Z = DY(KY)
            DX(KX) = W*DH11 + Z*DH12
            DY(KY) = W*DH21 + Z*DH22
            KX = KX + INCX
            KY = KY + INCY
         END DO
      ELSE IF (DFLAG.EQ.ZERO) THEN
         DH12 = DPARAM(4)
         DH21 = DPARAM(3)
         DO I = 1,N
            W = DX(KX)
            Z = DY(KY)
            DX(KX) = W + Z*DH12
            DY(KY) = W*DH21 + Z
```

```
            KX = KX + INCX
            KY = KY + INCY
         END DO
      ELSE
          DH11 = DPARAM(2)
          DH22 = DPARAM(5)
          DO I = 1,N
             W = DX(KX)
             Z = DY(KY)
             DX(KX) = W*DH11 + Z
             DY(KY) = -W + DH22*Z
             KX = KX + INCX
             KY = KY + INCY
         END DO
      END IF
   END IF
   RETURN
   END
```

———-

7.9.4 lisp code

— BLAS 1 drotm —

```
(let ((zero 0.0d0) (two 2.0d0))
  (declare (type (double-float) zero two))
  (defun drotm (n dx incx dy incy dparam)
    (declare (type (array double-float (*)) dparam)
             (type (array double-float (*)) dy dx)
             (type (f2cl-lib:integer4) incy incx n))
    (f2cl-lib:with-multi-array-data
        ((dx double-float dx-%data% dx-%offset%)
         (dy double-float dy-%data% dy-%offset%)
         (dparam double-float dparam-%data% dparam-%offset%))
      (prog ((i 0) (kx 0) (ky 0) (nsteps 0) (dflag 0.0d0) (dh11 0.0d0)
             (dh12 0.0d0) (dh21 0.0d0) (dh22 0.0d0) (w 0.0d0) (z 0.0d0))
        (declare (type (double-float) z w dh22 dh21 dh12 dh11 dflag)
                 (type (f2cl-lib:integer4) nsteps ky kx i))
        (setf dflag (f2cl-lib:fref dparam-%data% (1) ((1 5)) dparam-%offset%))
        (if (or (<= n 0) (= (+ dflag two) zero))
            (go end_label))
        (cond
         ((and (= incx incy) (> incx 0))
          (setf nsteps (f2cl-lib:int-mul n incx))
          (cond
           ((< dflag zero)
```

```
    (setf dh11
            (f2cl-lib:fref dparam-%data% (2) ((1 5)) dparam-%offset%))
    (setf dh12
            (f2cl-lib:fref dparam-%data% (4) ((1 5)) dparam-%offset%))
    (setf dh21
            (f2cl-lib:fref dparam-%data% (3) ((1 5)) dparam-%offset%))
    (setf dh22
            (f2cl-lib:fref dparam-%data% (5) ((1 5)) dparam-%offset%))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i incx))
                  ((> i nsteps) nil)
      (tagbody
        (setf w (f2cl-lib:fref dx-%data% (i) ((1 *)) dx-%offset%))
        (setf z (f2cl-lib:fref dy-%data% (i) ((1 *)) dy-%offset%))
        (setf (f2cl-lib:fref dx-%data% (i) ((1 *)) dx-%offset%)
                (+ (* w dh11) (* z dh12)))
        (setf (f2cl-lib:fref dy-%data% (i) ((1 *)) dy-%offset%)
                (+ (* w dh21) (* z dh22)))
       label100000)))
   ((= dflag zero)
    (setf dh12
            (f2cl-lib:fref dparam-%data% (4) ((1 5)) dparam-%offset%))
    (setf dh21
            (f2cl-lib:fref dparam-%data% (3) ((1 5)) dparam-%offset%))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i incx))
                  ((> i nsteps) nil)
      (tagbody
        (setf w (f2cl-lib:fref dx-%data% (i) ((1 *)) dx-%offset%))
        (setf z (f2cl-lib:fref dy-%data% (i) ((1 *)) dy-%offset%))
        (setf (f2cl-lib:fref dx-%data% (i) ((1 *)) dx-%offset%)
                (+ w (* z dh12)))
        (setf (f2cl-lib:fref dy-%data% (i) ((1 *)) dy-%offset%)
                (+ (* w dh21) z))
       label100001)))
   (t
    (setf dh11
            (f2cl-lib:fref dparam-%data% (2) ((1 5)) dparam-%offset%))
    (setf dh22
            (f2cl-lib:fref dparam-%data% (5) ((1 5)) dparam-%offset%))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i incx))
                  ((> i nsteps) nil)
      (tagbody
        (setf w (f2cl-lib:fref dx-%data% (i) ((1 *)) dx-%offset%))
        (setf z (f2cl-lib:fref dy-%data% (i) ((1 *)) dy-%offset%))
        (setf (f2cl-lib:fref dx-%data% (i) ((1 *)) dx-%offset%)
                (+ (* w dh11) z))
        (setf (f2cl-lib:fref dy-%data% (i) ((1 *)) dy-%offset%)
                (- (* dh22 z) w))
       label100002)))))
 (t (setf kx 1) (setf ky 1)
  (if (< incx 0)
```

```
    (setf kx
            (f2cl-lib:int-add 1
                              (f2cl-lib:int-mul
                               (f2cl-lib:int-sub 1 n) incx))))
(if (< incy 0)
    (setf ky
            (f2cl-lib:int-add 1
                              (f2cl-lib:int-mul
                               (f2cl-lib:int-sub 1 n) incy))))
(cond
 ((< dflag zero)
  (setf dh11
          (f2cl-lib:fref dparam-%data% (2) ((1 5)) dparam-%offset%))
  (setf dh12
          (f2cl-lib:fref dparam-%data% (4) ((1 5)) dparam-%offset%))
  (setf dh21
          (f2cl-lib:fref dparam-%data% (3) ((1 5)) dparam-%offset%))
  (setf dh22
          (f2cl-lib:fref dparam-%data% (5) ((1 5)) dparam-%offset%))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf w (f2cl-lib:fref dx-%data% (kx) ((1 *)) dx-%offset%))
      (setf z (f2cl-lib:fref dy-%data% (ky) ((1 *)) dy-%offset%))
      (setf (f2cl-lib:fref dx-%data% (kx) ((1 *)) dx-%offset%)
              (+ (* w dh11) (* z dh12)))
      (setf (f2cl-lib:fref dy-%data% (ky) ((1 *)) dy-%offset%)
              (+ (* w dh21) (* z dh22)))
      (setf kx (f2cl-lib:int-add kx incx))
      (setf ky (f2cl-lib:int-add ky incy))
     label100003)))
 ((= dflag zero)
  (setf dh12
          (f2cl-lib:fref dparam-%data% (4) ((1 5)) dparam-%offset%))
  (setf dh21
          (f2cl-lib:fref dparam-%data% (3) ((1 5)) dparam-%offset%))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf w (f2cl-lib:fref dx-%data% (kx) ((1 *)) dx-%offset%))
      (setf z (f2cl-lib:fref dy-%data% (ky) ((1 *)) dy-%offset%))
      (setf (f2cl-lib:fref dx-%data% (kx) ((1 *)) dx-%offset%)
              (+ w (* z dh12)))
      (setf (f2cl-lib:fref dy-%data% (ky) ((1 *)) dy-%offset%)
              (+ (* w dh21) z))
      (setf kx (f2cl-lib:int-add kx incx))
      (setf ky (f2cl-lib:int-add ky incy))
     label100004)))
 (t
  (setf dh11
```

```
                (f2cl-lib:fref dparam-%data% (2) ((1 5)) dparam-%offset%))
        (setf dh22
                (f2cl-lib:fref dparam-%data% (5) ((1 5)) dparam-%offset%))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf w (f2cl-lib:fref dx-%data% (kx) ((1 *)) dx-%offset%))
            (setf z (f2cl-lib:fref dy-%data% (ky) ((1 *)) dy-%offset%))
            (setf (f2cl-lib:fref dx-%data% (kx) ((1 *)) dx-%offset%)
                    (+ (* w dh11) z))
            (setf (f2cl-lib:fref dy-%data% (ky) ((1 *)) dy-%offset%)
                    (- (* dh22 z) w))
            (setf kx (f2cl-lib:int-add kx incx))
            (setf ky (f2cl-lib:int-add ky incy))
           label100005))))))
  (go end_label)
 end_label
  (return (values nil nil nil nil nil nil))))))
```

———-

7.10 drotmg - Construct the modified givens transformation matrix

7.10.1 Axiom unit tests

— **drotmg.input** —

```
)set break resume
)sys rm -f drotmg.output
)spool drotmg.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

7.10.2 Axiom help page

— **drotmg.help** —

```
====================================================================
drotmg examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine drotmg (DD1, DD2, DX1, DY1, DPARAM)
           DROTMG

Function/Subroutine Documentation
   subroutine drotmg (double precision DD1, double precision DD2, double
       precision DX1, double precision DY1, double precision, dimension(5)
       DPARAM)
       DROTMG

Purpose:
 Construct the modified GIVENS transformation matrix H which zeros
 the second component of the 2-vector  (DSQRT(DD1)*DX1,DSQRT(DD2)*DY2)**T.
 with DPARAM(1)=DFLAG, H has one of the following forms..

  DFLAG=-1.D0     DFLAG=0.D0        DFLAG=1.D0     DFLAG=-2.D0

    (DH11  DH12)    (1.D0  DH12)    (DH11  1.D0)    (1.D0  0.D0)
  H=(          )    (          )    (          )    (          )
    (DH21  DH22),   (DH21  1.D0),   (-1.D0 DH22),   (0.D0  1.D0).
  locations 2-4 of DPARAM contain DH11, DH21, DH12, AND DH22
  respectively. (Values of 1.D0, -1.D0, or 0.D0 implied by the
  value of DPARAM(1) are not stored in dparam.)

  the values of GAMSQ and RGAMSQ set in the data statement may be
  inexact.  this is ok as they are only used for testing the size
  of DD1 and DD2.  All actual scaling of data is done using gam.

Parameters:
  DD1
    DD1 is DOUBLE PRECISION

  DD2
    DD2 is DOUBLE PRECISION

  DX1
    DX1 is DOUBLE PRECISION

  DY1
    DY1 is DOUBLE PRECISION
```

```
 DPARAM
   DPARAM is DOUBLE PRECISION array, dimension 5
     DPARAM(1)=DFLAG
     DPARAM(2)=DH11
     DPARAM(3)=DH21
     DPARAM(4)=DH12
     DPARAM(5)=DH22
```

———-

7.10.3 fortran code

— **drotmg.f** —

```
*  ====================================================================
      SUBROUTINE DROTMG(DD1,DD2,DX1,DY1,DPARAM)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION DD1,DD2,DX1,DY1
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION DPARAM(5)
*     ..
*
*  ====================================================================
*
*     .. Local Scalars ..
      DOUBLE PRECISION DFLAG,DH11,DH12,DH21,DH22,DP1,DP2,DQ1,DQ2,DTEMP,
     $                 DU,GAM,GAMSQ,ONE,RGAMSQ,TWO,ZERO
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DABS
*     ..
*     .. Data statements ..
*
      DATA ZERO,ONE,TWO/0.D0,1.D0,2.D0/
      DATA GAM,GAMSQ,RGAMSQ/4096.D0,16777216.D0,5.9604645D-8/
*     ..

      IF (DD1.LT.ZERO) THEN
*        GO ZERO-H-D-AND-DX1..
         DFLAG = -ONE
```

```
         DH11 = ZERO
         DH12 = ZERO
         DH21 = ZERO
         DH22 = ZERO
*
         DD1 = ZERO
         DD2 = ZERO
         DX1 = ZERO
      ELSE
*        CASE-DD1-NONNEGATIVE
         DP2 = DD2*DY1
         IF (DP2.EQ.ZERO) THEN
            DFLAG = -TWO
            DPARAM(1) = DFLAG
            RETURN
         END IF
*        REGULAR-CASE..
         DP1 = DD1*DX1
         DQ2 = DP2*DY1
         DQ1 = DP1*DX1
*
         IF (DABS(DQ1).GT.DABS(DQ2)) THEN
            DH21 = -DY1/DX1
            DH12 = DP2/DP1
*
            DU = ONE - DH12*DH21
*
           IF (DU.GT.ZERO) THEN
             DFLAG = ZERO
             DD1 = DD1/DU
             DD2 = DD2/DU
             DX1 = DX1*DU
           END IF
         ELSE

            IF (DQ2.LT.ZERO) THEN
*              GO ZERO-H-D-AND-DX1..
               DFLAG = -ONE
               DH11 = ZERO
               DH12 = ZERO
               DH21 = ZERO
               DH22 = ZERO
*
               DD1 = ZERO
               DD2 = ZERO
               DX1 = ZERO
            ELSE
               DFLAG = ONE
               DH11 = DP1/DP2
               DH22 = DX1/DY1
```

```
               DU = ONE + DH11*DH22
               DTEMP = DD2/DU
               DD2 = DD1/DU
               DD1 = DTEMP
               DX1 = DY1*DU
            END IF
         END IF

*     PROCEDURE..SCALE-CHECK
         IF (DD1.NE.ZERO) THEN
            DO WHILE ((DD1.LE.RGAMSQ) .OR. (DD1.GE.GAMSQ))
               IF (DFLAG.EQ.ZERO) THEN
                  DH11 = ONE
                  DH22 = ONE
                  DFLAG = -ONE
               ELSE
                  DH21 = -ONE
                  DH12 = ONE
                  DFLAG = -ONE
               END IF
               IF (DD1.LE.RGAMSQ) THEN
                  DD1 = DD1*GAM**2
                  DX1 = DX1/GAM
                  DH11 = DH11/GAM
                  DH12 = DH12/GAM
               ELSE
                  DD1 = DD1/GAM**2
                  DX1 = DX1*GAM
                  DH11 = DH11*GAM
                  DH12 = DH12*GAM
               END IF
            ENDDO
         END IF

         IF (DD2.NE.ZERO) THEN
            DO WHILE ( (DABS(DD2).LE.RGAMSQ) .OR. (DABS(DD2).GE.GAMSQ) )
               IF (DFLAG.EQ.ZERO) THEN
                  DH11 = ONE
                  DH22 = ONE
                  DFLAG = -ONE
               ELSE
                  DH21 = -ONE
                  DH12 = ONE
                  DFLAG = -ONE
               END IF
               IF (DABS(DD2).LE.RGAMSQ) THEN
                  DD2 = DD2*GAM**2
                  DH21 = DH21/GAM
                  DH22 = DH22/GAM
               ELSE
```

```
                DD2 = DD2/GAM**2
                DH21 = DH21*GAM
                DH22 = DH22*GAM
             END IF
          END DO
       END IF

    END IF

    IF (DFLAG.LT.ZERO) THEN
       DPARAM(2) = DH11
       DPARAM(3) = DH21
       DPARAM(4) = DH12
       DPARAM(5) = DH22
    ELSE IF (DFLAG.EQ.ZERO) THEN
       DPARAM(3) = DH21
       DPARAM(4) = DH12
    ELSE
       DPARAM(2) = DH11
       DPARAM(5) = DH22
    END IF

    DPARAM(1) = DFLAG
    RETURN
    END
```

———-

7.10.4 lisp code

— BLAS 1 drotmg —

```
(let ((zero 0.0d0)
      (one 1.0d0)
      (two 2.0d0)
      (gam 4096.0d0)
      (gamsq 1.6777216d7)
      (rgamsq 5.9604645d-8))
  (declare (type (double-float) zero one two gam gamsq rgamsq))
  (defun drotmg (dd1 dd2 dx1 dy1 dparam)
    (declare (type (array double-float (*)) dparam)
             (type (double-float) dy1 dx1 dd2 dd1))
    (f2cl-lib:with-multi-array-data
        ((dparam double-float dparam-%data% dparam-%offset%))
      (prog ((dflag 0.0d0) (dh11 0.0d0) (dh12 0.0d0) (dh21 0.0d0) (dh22 0.0d0)
             (dp1 0.0d0) (dp2 0.0d0) (dq1 0.0d0) (dq2 0.0d0) (dtemp 0.0d0)
             (du 0.0d0))
```

```
(declare
 (type (double-float) du dtemp dq2 dq1 dp2 dp1 dh22 dh21 dh12 dh11
  dflag))
(cond
 ((< dd1 zero) (setf dflag (- one)) (setf dh11 zero) (setf dh12 zero)
  (setf dh21 zero) (setf dh22 zero) (setf dd1 zero) (setf dd2 zero)
  (setf dx1 zero))
 (t (setf dp2 (* dd2 dy1))
  (cond
   ((= dp2 zero) (setf dflag (- two))
    (setf (f2cl-lib:fref dparam-%data% (1) ((1 5)) dparam-%offset%)
            dflag)
    (go end_label)))
  (setf dp1 (* dd1 dx1)) (setf dq2 (* dp2 dy1)) (setf dq1 (* dp1 dx1))
  (cond
   ((> (f2cl-lib:dabs dq1) (f2cl-lib:dabs dq2))
    (setf dh21 (/ (- dy1) dx1)) (setf dh12 (/ dp2 dp1))
    (setf du (- one (* dh12 dh21)))
    (cond
     ((> du zero) (setf dflag zero) (setf dd1 (/ dd1 du))
      (setf dd2 (/ dd2 du)) (setf dx1 (* dx1 du)))))
   (t
    (cond
     ((< dq2 zero) (setf dflag (- one)) (setf dh11 zero)
      (setf dh12 zero) (setf dh21 zero) (setf dh22 zero)
      (setf dd1 zero) (setf dd2 zero) (setf dx1 zero))
     (t (setf dflag one) (setf dh11 (/ dp1 dp2))
      (setf dh22 (/ dx1 dy1)) (setf du (+ one (* dh11 dh22)))
      (setf dtemp (/ dd2 du)) (setf dd2 (/ dd1 du)) (setf dd1 dtemp)
      (setf dx1 (* dy1 du))))))
  (cond
   ((/= dd1 zero)
    (tagbody
     label100000
      (if (not (or (<= dd1 rgamsq) (>= dd1 gamsq)))
          (go label100001))
      (cond
       ((= dflag zero) (setf dh11 one) (setf dh22 one)
        (setf dflag (- one)))
       (t (setf dh21 (- one)) (setf dh12 one) (setf dflag (- one))))
      (cond
       ((<= dd1 rgamsq) (setf dd1 (* dd1 (expt gam 2)))
        (setf dx1 (/ dx1 gam)) (setf dh11 (/ dh11 gam))
        (setf dh12 (/ dh12 gam)))
       (t (setf dd1 (/ dd1 (expt gam 2))) (setf dx1 (* dx1 gam))
        (setf dh11 (* dh11 gam)) (setf dh12 (* dh12 gam))))
      (go label100000)
     label100001)))
  (cond
   ((/= dd2 zero)
```

```
      (tagbody
       label100002
        (if (not
             (or (<= (f2cl-lib:dabs dd2) rgamsq)
                 (>= (f2cl-lib:dabs dd2) gamsq)))
            (go label100003))
        (cond
         ((= dflag zero) (setf dh11 one) (setf dh22 one)
          (setf dflag (- one)))
         (t (setf dh21 (- one)) (setf dh12 one) (setf dflag (- one))))
        (cond
         ((<= (f2cl-lib:dabs dd2) rgamsq) (setf dd2 (* dd2 (expt gam 2)))
          (setf dh21 (/ dh21 gam)) (setf dh22 (/ dh22 gam)))
         (t (setf dd2 (/ dd2 (expt gam 2))) (setf dh21 (* dh21 gam))
          (setf dh22 (* dh22 gam))))
        (go label100002)
       label100003)))))
  (cond
   ((< dflag zero)
    (setf (f2cl-lib:fref dparam-%data% (2) ((1 5)) dparam-%offset%) dh11)
    (setf (f2cl-lib:fref dparam-%data% (3) ((1 5)) dparam-%offset%) dh21)
    (setf (f2cl-lib:fref dparam-%data% (4) ((1 5)) dparam-%offset%) dh12)
    (setf (f2cl-lib:fref dparam-%data% (5) ((1 5)) dparam-%offset%)
            dh22))
   ((= dflag zero)
    (setf (f2cl-lib:fref dparam-%data% (3) ((1 5)) dparam-%offset%) dh21)
    (setf (f2cl-lib:fref dparam-%data% (4) ((1 5)) dparam-%offset%)
            dh12))
   (t
    (setf (f2cl-lib:fref dparam-%data% (2) ((1 5)) dparam-%offset%) dh11)
    (setf (f2cl-lib:fref dparam-%data% (5) ((1 5)) dparam-%offset%)
            dh22)))
  (setf (f2cl-lib:fref dparam-%data% (1) ((1 5)) dparam-%offset%) dflag)
  (go end_label)
 end_label
  (return (values dd1 dd2 dx1 nil nil))))))
```

———-

7.11 dscal - scale a vector by a constant

7.11.1 Axiom unit tests

— **dscal.input** —

```
)set break resume
```

```
)sys rm -f dscal.output
)spool dscal.output
)set message test on
)set message auto off
)clear all

--S 1 of 6
dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]]
--R
--R   (1)  [1.,2.,3.,4.,5.,6.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 1

--S 2 of 6
dscal(6,2.0,dx,1)
--R
--R   (2)  [2.,4.,6.,8.,10.,12.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 2

--S 3 of 6
dx
--R
--R   (3)  [2.,4.,6.,8.,10.,12.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 3

--S 4 of 6
dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]]
--R
--R   (4)  [1.,2.,3.,4.,5.,6.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 4

--S 5 of 6
dscal(3,0.5,dx,1)
--R
--R   (5)  [0.5,1.,1.5,4.,5.,6.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 5

--S 6 of 6
dx
--R
--R   (6)  [0.5,1.,1.5,4.,5.,6.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 6


)spool
```

```
)lisp (bye)
```

———-

7.11.2 Axiom help page

— dscal.help —

```
======================================================================
dscal examples
======================================================================

Given an initial array

dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]]

   [1.,2.,3.,4.,5.,6.]

scale each element by 2.0

dscal(6,2.0,dx,1)

   [2.,4.,6.,8.,10.,12.]

Note that this changes the initial array in place.

dx

   [2.,4.,6.,8.,10.,12.]

Given the initial array

dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]]

   [1.,2.,3.,4.,5.,6.]

scale each element by 0.5

dscal(6,0.5,dx,1)

    [0.5,1.,1.5,2.,2.5,3.]

Given the initial array

dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]]

   [1.,2.,3.,4.,5.,6.]
```

```
scale the first three elements by 0.5

dscal(3,0.5,dx,1)

   [0.5,1.,1.5,4.,5.,6.]

Given the initial array

dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]]

   [1.,2.,3.,4.,5.,6.]

dscal will do nothing with a negative count

dscal(-3,0.5,dx,1)

   [1.,2.,3.,4.,5.,6.]

Given the initial array

dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0, 5.0, 6.0]]

   [1.,2.,3.,4.,5.,6.]

dscal will do nothing with a negative increment

dscal(3,0.5,dx,-1)

   [1.,2.,3.,4.,5.,6.]


=====================================================================
Man Page Details
=====================================================================

NAME
       DSCAL - BLAS level one, scales a double precision vector

SYNOPSIS
       SUBROUTINE DSCAL    ( n, alpha, x, incx )

           INTEGER           n, incx

           DOUBLE PRECISION  alpha, x


DESCRIPTION
       DSCAL  scales a double precision vector with a double precision scalar.
       DSCAL scales the vector x of length n and increment incx  by  the  con-
       stant a.
```

```
       This routine performs the following vector operation:

                 x <-- alpha x

       where alpha is a double precisoin scalar, and x is a double precision
       vector.

ARGUMENTS
       n       INTEGER. (input)
               Number of elements in the vector.
               If n <= 0, this routine returns without computation.

       alpha   DOUBLE PRECISION scalar alpha. (input)

       x       DOUBLE PRECISION, (input and output)
               Array of dimension (n-1) * |incx| + 1. Vector to be scaled.

       incx    INTEGER. (input)
               Increment between elements of x.
               If incx = 0, the results will be unpredictable.
```

———-

7.11.3 fortran code

— dscal.f —

```
*  ====================================================================
      SUBROUTINE DSCAL(N,DA,DX,INCX)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION DA
      INTEGER INCX,N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION DX(*)
*     ..
*
*  ====================================================================
*
*     .. Local Scalars ..
```

```
      INTEGER I,M,MP1,NINCX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MOD
*     ..
      IF (N.LE.0 .OR. INCX.LE.0) RETURN
      IF (INCX.EQ.1) THEN
*
*        code for increment equal to 1
*
*
*        clean-up loop
*
         M = MOD(N,5)
         IF (M.NE.0) THEN
            DO I = 1,M
               DX(I) = DA*DX(I)
            END DO
            IF (N.LT.5) RETURN
         END IF
         MP1 = M + 1
         DO I = MP1,N,5
            DX(I) = DA*DX(I)
            DX(I+1) = DA*DX(I+1)
            DX(I+2) = DA*DX(I+2)
            DX(I+3) = DA*DX(I+3)
            DX(I+4) = DA*DX(I+4)
         END DO
      ELSE
*
*        code for increment not equal to 1
*
         NINCX = N*INCX
         DO I = 1,NINCX,INCX
            DX(I) = DA*DX(I)
         END DO
      END IF
      RETURN
      END
```

———-

7.11.4 fortran unit test

— drotg example —

```
 program dscalEX
```

```
*       Tim Daly May 6, 2012
*       unit tests for BLAS dscal
        double precision a(6)
        double precision b
        a = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0, 5.0D0, 6.0D0 /)

        write(6,10)
 10     format(/,"the initial array")
        write(6,100)a(1),a(2),a(3)
 100    format("a(1)=",f6.3," a(2)=",f6.3," a(3)=",f6.3)
        write(6,200)a(4),a(5),a(6)
 200    format("a(4)=",f6.3," a(5)=",f6.3," a(6)=",f6.3)

        write(6,11)
 11     format(/,"scale whole array by 2.0")
        call dscal(6,2.0d0,a,1)
        write(6,100)a(1),a(2),a(3)
        write(6,200)a(4),a(5),a(6)

        write(6,12)
 12     format(/,"scale whole array by 0.5")
        a = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0, 5.0D0, 6.0D0 /)
        call dscal(6,0.5d0,a,1)
        write(6,100)a(1),a(2),a(3)
        write(6,200)a(4),a(5),a(6)

        write(6,13)
 13     format(/,"scale 3 elements by 0.5")
        a = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0, 5.0D0, 6.0D0 /)
        call dscal(3,0.5d0,a,1)
        write(6,100)a(1),a(2),a(3)
        write(6,200)a(4),a(5),a(6)

        write(6,14)
 14     format(/,"do nothing. negative length")
        a = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0, 5.0D0, 6.0D0 /)
        call dscal(-3,0.5d0,a,1)
        write(6,100)a(1),a(2),a(3)
        write(6,200)a(4),a(5),a(6)

        write(6,15)
 15     format(/,"scale 3 elements by -0.5, from end (does nothing)")
        a = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0, 5.0D0, 6.0D0 /)
        call dscal(3,0.5d0,a,-1)
        write(6,100)a(1),a(2),a(3)
        write(6,200)a(4),a(5),a(6)

        stop
        end
```

———-

```
gcc -o dscalEX dscalEX.f -lgfortran dscal.o && ./dscalEX

the initial array
a(1)= 1.000 a(2)= 2.000 a(3)= 3.000
a(4)= 4.000 a(5)= 5.000 a(6)= 6.000

scale whole array by 2.0
a(1)= 2.000 a(2)= 4.000 a(3)= 6.000
a(4)= 8.000 a(5)=10.000 a(6)=12.000

scale whole array by 0.5
a(1)= 0.500 a(2)= 1.000 a(3)= 1.500
a(4)= 2.000 a(5)= 2.500 a(6)= 3.000

scale 3 elements by 0.5
a(1)= 0.500 a(2)= 1.000 a(3)= 1.500
a(4)= 4.000 a(5)= 5.000 a(6)= 6.000

do nothing. negative length
a(1)= 1.000 a(2)= 2.000 a(3)= 3.000
a(4)= 4.000 a(5)= 5.000 a(6)= 6.000

scale 3 elements by -0.5, from end (does nothing)
a(1)= 1.000 a(2)= 2.000 a(3)= 3.000
a(4)= 4.000 a(5)= 5.000 a(6)= 6.000
```

7.11.5 lisp code

— BLAS 1 dscal —

```
(defun dscal (n da dx incx)
; Tim Daly May 6, 2012
  (declare (type (simple-array double-float (*)) dx)
           (type (double-float) da)
           (type fixnum n incx))
 (let ((i 0) (limit (length dx)))
 (declare (type fixnum limit i))
  (when (and (> n 0) (> incx 0))
   (do ((i 0 (the fixnum (+ i incx))))
       ((or (>= i n) (>= i limit)))
    (setf (svref dx i)
     (the double-float (* da (the double-float (svref dx i)))))))
  dx))
```

———-

— BLAS 1 dscal lisp test —

```
(load "dscal.lisp")
(setq a (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0 6.0d0))
; #(1.0 2.0 3.0 4.0 5.0 6.0)
(dscal 6 2.0d0 a 1)
; #(2.0 4.0 6.0 8.0 10.0 12.0)

(setq a (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0 6.0d0))
; #(1.0 2.0 3.0 4.0 5.0 6.0)
(dscal 6 0.5d0 a 1)
; #(0.5 1.0 1.5 2.0 2.5 3.0)

(setq a (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0 6.0d0))
; #(1.0 2.0 3.0 4.0 5.0 6.0)
(dscal 3 0.5d0 a 1)
; #(0.5 1.0 1.5 4.0 5.0 6.0)

(setq a (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0 6.0d0))
; #(1.0 2.0 3.0 4.0 5.0 6.0)
(dscal -3 0.5d0 a 1)
; #(1.0 2.0 3.0 4.0 5.0 6.0)

(setq a (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0 6.0d0))
; #(1.0 2.0 3.0 4.0 5.0 6.0)
(dscal 3 0.5d0 a -1)
; #(1.0 2.0 3.0 4.0 5.0 6.0)
```

———-

7.12 dsdot - compute the inner product of two vectors

See Lawson, et al. [Laws79a]

7.12.1 Axiom unit tests

— dsdot.input —

```
)set break resume
)sys rm -f dsdot.output
)spool dsdot.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

7.12.2 Axiom help page

— **dsdot.help** —

```
====================================================================
dsdot examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function dsdot (N, SX, INCX, SY, INCY)
           DSDOT

Function/Subroutine Documentation
   double precision function dsdot (integer N, real, dimension(*) SX, integer
       INCX, real, dimension(*) SY, integer INCY)
       DSDOT

Purpose:
  Compute the inner product of two vectors with extended
  precision accumulation and result.

  Returns D.P. dot product accumulated in D.P., for S.P. SX and SY
  DSDOT = sum for I = 0 to N-1 of  SX(LX+I*INCX) * SY(LY+I*INCY),
  where LX = 1 if INCX .GE. 0, else LX = 1+(1-N)*INCX, and LY is
  defined in a similar way using INCY.

Parameters:
 N
   N is INTEGER
   number of elements in input vector(s)

 SX
   SX is REAL array, dimension(N)
   single precision vector with N elements

 INCX
   INCX is INTEGER
```

```
   storage spacing between elements of SX

 SY
   SY is REAL array, dimension(N)
   single precision vector with N elements

 INCY
   INCY is INTEGER
   storage spacing between elements of SY

Returns:

 DSDOT
   DSDOT is DOUBLE PRECISION
   DSDOT  double precision dot product (zero if N.LE.0)
```

7.12.3 fortran code

— dsdot.f —

```
*  ===================================================================
      DOUBLE PRECISION FUNCTION DSDOT(N,SX,INCX,SY,INCY)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      REAL SX(*),SY(*)
*     ..
*
*  ===================================================================
*
*     .. Local Scalars ..
      INTEGER I,KX,KY,NS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DBLE
*     ..
      DSDOT = 0.0D0
      IF (N.LE.0) RETURN
```

```
      IF (INCX.EQ.INCY .AND. INCX.GT.0) THEN
*
*     Code for equal, positive, non-unit increments.
*
         NS = N*INCX
         DO I = 1,NS,INCX
            DSDOT = DSDOT + DBLE(SX(I))*DBLE(SY(I))
         END DO
      ELSE
*
*     Code for unequal or nonpositive increments.
*
         KX = 1
         KY = 1
         IF (INCX.LT.0) KX = 1 + (1-N)*INCX
         IF (INCY.LT.0) KY = 1 + (1-N)*INCY
         DO I = 1,N
            DSDOT = DSDOT + DBLE(SX(KX))*DBLE(SY(KY))
            KX = KX + INCX
            KY = KY + INCY
         END DO
      END IF
      RETURN
      END
```

———-

7.12.4 lisp code

— BLAS 1 dsdot —

```
(defun dsdot (n sx incx sy incy)
  (declare (type (array single-float (*)) sy sx)
           (type (f2cl-lib:integer4) incy incx n))
  (f2cl-lib:with-multi-array-data
      ((sx single-float sx-%data% sx-%offset%)
       (sy single-float sy-%data% sy-%offset%))
    (prog ((i 0) (kx 0) (ky 0) (ns 0) (dsdot 0.0d0))
      (declare (type (double-float) dsdot)
               (type (f2cl-lib:integer4) ns ky kx i))
      (setf dsdot 0.0d0)
      (if (<= n 0)
          (go end_label))
      (cond
       ((and (= incx incy) (> incx 0)) (setf ns (f2cl-lib:int-mul n incx))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i incx))
                      ((> i ns) nil)
```

```
        (tagbody
          (setf dsdot
                  (+ dsdot
                     (*
                      (f2cl-lib:dble
                       (f2cl-lib:fref sx-%data% (i) ((1 *)) sx-%offset%))
                      (f2cl-lib:dble
                       (f2cl-lib:fref sy-%data% (i) ((1 *)) sy-%offset%)))))
         label100000)))
      (t (setf kx 1) (setf ky 1)
       (if (< incx 0)
           (setf kx
                   (f2cl-lib:int-add 1
                                     (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n)
                                                       incx))))
       (if (< incy 0)
           (setf ky
                   (f2cl-lib:int-add 1
                                     (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n)
                                                       incy))))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
        (tagbody
          (setf dsdot
                  (+ dsdot
                     (*
                      (f2cl-lib:dble
                       (f2cl-lib:fref sx-%data% (kx) ((1 *)) sx-%offset%))
                      (f2cl-lib:dble
                       (f2cl-lib:fref sy-%data% (ky) ((1 *)) sy-%offset%)))))
          (setf kx (f2cl-lib:int-add kx incx))
          (setf ky (f2cl-lib:int-add ky incy))
         label100001))))
     (go end_label)
    end_label
     (return (values dsdot nil nil nil nil nil)))))
```

———-

7.13 dswap - interchanges two vectors

7.13.1 Axiom unit tests

— dswap.input —

```
)set break resume
```

```
)sys rm -f dswap.output
)spool dswap.output
)set message test on
)set message auto off
)clear all

--S 1 of 9
dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0,  5.0]]
--R
--R   (1)  [1.,2.,3.,4.,5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 1

--S 2 of 9
dy:PRIMARR(DFLOAT):=[[9.0, 8.0, 7.0, 6.0, -5.0]]
--R
--R   (2)  [9.,8.,7.,6.,- 5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 2

--S 3 of 9
dswap(5,dx,1,dy,1)
--R
--R   (3)  [[9.,8.,7.,6.,- 5.],[1.,2.,3.,4.,5.]]
--R                                     Type: List(PrimitiveArray(DoubleFloat))
--E 3

--S 4 of 9
dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0,  5.0]]
--R
--R   (4)  [1.,2.,3.,4.,5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 4

--S 5 of 9
dy:PRIMARR(DFLOAT):=[[9.0, 8.0, 7.0, 6.0, -5.0]]
--R
--R   (5)  [9.,8.,7.,6.,- 5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 5

--S 6 of 9
dswap(3,dx,2,dy,2)
--R
--R   (6)  [[9.,2.,7.,4.,- 5.],[1.,8.,3.,6.,5.]]
--R                                     Type: List(PrimitiveArray(DoubleFloat))
--E 6

--S 7 of 9
dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0,  5.0]]
```

```
--R
--R   (7)  [1.,2.,3.,4.,5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 7

--S 8 of 9
dy:PRIMARR(DFLOAT):=[[9.0, 8.0, 7.0, 6.0, -5.0]]
--R
--R   (8)  [9.,8.,7.,6.,- 5.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 8

--S 9 of 9
dswap(5,dx,1,dy,-1)
--R
--R   (9)  [[9.,8.,7.,6.,- 5.],[1.,2.,3.,4.,5.]]
--R                                     Type: List(PrimitiveArray(DoubleFloat))
--E 9

)spool
)lisp (bye)
```

7.13.2 Axiom help page

— dswap.help —

```
=======================================================================
dswap examples
=======================================================================

dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0,  5.0]]

  [1.,2.,3.,4.,5.]

dy:PRIMARR(DFLOAT):=[[9.0, 8.0, 7.0, 6.0, -5.0]]

  [9.,8.,7.,6.,- 5.]

dswap(5,dx,1,dy,1)

  [[9.,8.,7.,6.,- 5.],[1.,2.,3.,4.,5.]]

dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0,  5.0]]

  [1.,2.,3.,4.,5.]
```

```
dy:PRIMARR(DFLOAT):=[[9.0, 8.0, 7.0, 6.0, -5.0]]

  [9.,8.,7.,6.,- 5.]

dswap(3,dx,2,dy,2)

  [[9.,2.,7.,4.,- 5.],[1.,8.,3.,6.,5.]]

dx:PRIMARR(DFLOAT):=[[1.0, 2.0, 3.0, 4.0,  5.0]]

  [1.,2.,3.,4.,5.]

dy:PRIMARR(DFLOAT):=[[9.0, 8.0, 7.0, 6.0, -5.0]]

  [9.,8.,7.,6.,- 5.]

dswap(5,dx,1,dy,-1)

 [[9.,8.,7.,6.,- 5.],[1.,2.,3.,4.,5.]]


=====================================================================
Man Page Details
=====================================================================

NAME
       DSWAP - BLAS level one, Swaps two double precision vectors

SYNOPSIS
       SUBROUTINE DSWAP    ( n, x, incx, y, incy )

           INTEGER           n, incx, incy

           DOUBLE PRECISION  x, y


DESCRIPTION
       DSWAP  swaps  two double precision vectors, it interchanges n values of
       vector x and vector y.  incx and incy specify the increment between two
       consecutive elements of respectively vector x and y.


       This routine performs the following vector operation:

                 x <-> y

            where x and y are double precision vectors.

ARGUMENTS
       n       INTEGER. (input)
```

```
               Number of vector elements to be swapped.
               If n <= 0, this routine returns without computation.

       x       DOUBLE PRECISION, (input and output)
               Array of dimension (n-1) * |incx| + 1.

       incx    INTEGER. (input)
               Increment between elements of x.
               If incx = 0, the results will be unpredictable.

       y       DOUBLE PRECISION, (input and output)
               array of dimension (n-1) * |incy| + 1. Vector to be swapped.

       incy    INTEGER. (input)
               Increment between elements of y.  If incy = 0, the results will
               be unpredictable.

NOTES
       When  working  backward  (incx < 0 or incy < 0), each routine starts at
       the end of the vector and moves backward, as follows:

            x(1-incx * (n-1)), x(1-incx * (n-2)), ..., x(1)

            y(1-incy * (n-1)), y(1-incy * (n-2)), ..., y(1)
```

———-

7.13.3 fortran code

— dswap.f —

```
*  ====================================================================
      SUBROUTINE DSWAP(N,DX,INCX,DY,INCY)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION DX(*),DY(*)
*     ..
*
*  ====================================================================
```

```
*
*     .. Local Scalars ..
      DOUBLE PRECISION DTEMP
      INTEGER I,IX,IY,M,MP1
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MOD
*     ..
      IF (N.LE.0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*       code for both increments equal to 1
*
*
*       clean-up loop
*
         M = MOD(N,3)
         IF (M.NE.0) THEN
            DO I = 1,M
               DTEMP = DX(I)
               DX(I) = DY(I)
               DY(I) = DTEMP
            END DO
            IF (N.LT.3) RETURN
         END IF
         MP1 = M + 1
         DO I = MP1,N,3
            DTEMP = DX(I)
            DX(I) = DY(I)
            DY(I) = DTEMP
            DTEMP = DX(I+1)
            DX(I+1) = DY(I+1)
            DY(I+1) = DTEMP
            DTEMP = DX(I+2)
            DX(I+2) = DY(I+2)
            DY(I+2) = DTEMP
         END DO
      ELSE
*
*       code for unequal increments or equal increments not equal
*         to 1
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
         IF (INCY.LT.0) IY = (-N+1)*INCY + 1
         DO I = 1,N
            DTEMP = DX(IX)
            DX(IX) = DY(IY)
            DY(IY) = DTEMP
```

```
            IX = IX + INCX
            IY = IY + INCY
         END DO
      END IF
      RETURN
      END
```

There appears to be a bit of confusion using negative increments. The fortran code does not work as I would have guessed. I don't understand what it was intended to do.

7.13.4 fortran unit test

— dswap example —

```
      program dswapEX
*     Tim Daly May 6, 2012
*     unit tests for BLAS dswap
      double precision a(5)
      double precision b(5)
      a = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0,  5.0D0/)
      b = (/ 9.0D0, 8.0D0, 7.0D0, 6.0D0, -5.0D0/)

      write(6,10)
 10   format(/,"before:")
      write(6,100)a(1),a(2),a(3),a(4),a(5)
 100  format("dx=(/",f6.3,", ",f6.3,", ",f6.3,", ",f6.3,", ",f6.3,"/)")
      write(6,101)b(1),b(2),b(3),b(4),b(5)
 101  format("dy=(/",f6.3,", ",f6.3,", ",f6.3,", ",f6.3,", ",f6.3,"/)")
      write(6,11)
 11   format(/,"swap all elements from dx to dy");
      call dswap(5,a,1,b,1)
      write(6,100)a(1),a(2),a(3),a(4),a(5)
      write(6,101)b(1),b(2),b(3),b(4),b(5)

      write(6,10)
      a = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0,  5.0D0/)
      b = (/ 9.0D0, 8.0D0, 7.0D0, 6.0D0, -5.0D0/)
      write(6,100)a(1),a(2),a(3),a(4),a(5)
      write(6,101)b(1),b(2),b(3),b(4),b(5)
      write(6,20)
 20   format(/,"swap every other element from dx to dy");
      a = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0,  5.0D0/)
      b = (/ 9.0D0, 8.0D0, 7.0D0, 6.0D0, -5.0D0/)
      call dswap(3,a,2,b,2)
      write(6,100)a(1),a(2),a(3),a(4),a(5)
```

```
      write(6,101)b(1),b(2),b(3),b(4),b(5)

      write(6,10)
      a = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0,  5.0D0/)
      b = (/ 9.0D0, 8.0D0, 7.0D0, 6.0D0, -5.0D0/)
      write(6,100)a(1),a(2),a(3),a(4),a(5)
      write(6,101)b(1),b(2),b(3),b(4),b(5)
      write(6,22)
 22   format(/,"swap and reverse elements dx to dy");
      a = (/ 1.0D0, 2.0D0, 3.0D0, 4.0D0,  5.0D0/)
      b = (/ 9.0D0, 8.0D0, 7.0D0, 6.0D0, -5.0D0/)
      call dswap(5,a,1,b,-1)
      write(6,100)a(1),a(2),a(3),a(4),a(5)
      write(6,101)b(1),b(2),b(3),b(4),b(5)

      stop
      end
```

———-

```
gcc -o dswapEX dswapEX.f -lgfortran dswap.o && ./dswapEX

before:
dx=(/ 1.000,  2.000,  3.000,  4.000,  5.000/)
dy=(/ 9.000,  8.000,  7.000,  6.000, -5.000/)

swap all elements from dx to dy
dx=(/ 9.000,  8.000,  7.000,  6.000, -5.000/)
dy=(/ 1.000,  2.000,  3.000,  4.000,  5.000/)

before:
dx=(/ 1.000,  2.000,  3.000,  4.000,  5.000/)
dy=(/ 9.000,  8.000,  7.000,  6.000, -5.000/)

swap every other element from dx to dy
dx=(/ 9.000,  2.000,  7.000,  4.000, -5.000/)
dy=(/ 1.000,  8.000,  3.000,  6.000,  5.000/)

before:
dx=(/ 1.000,  2.000,  3.000,  4.000,  5.000/)
dy=(/ 9.000,  8.000,  7.000,  6.000, -5.000/)

swap and reverse elements dx to dy
dx=(/-5.000,  6.000,  7.000,  8.000,  9.000/)
dy=(/ 5.000,  4.000,  3.000,  2.000,  1.000/)
```

7.13.5 lisp code

— BLAS 1 dswap —

```
(defun dswap (n dx incx dy incy)
; Tim Daly May 6, 2012
 (declare (type (simple-array double-float (*)) dy dx)
          (type fixnum incy incx n))
 (let ((ix 0) (iy 0) (limitx (length dx)) (limity (length dy))
       (dtemp 0.0d0))
 (declare (type (double-float) dtemp) (type fixnum ix iy limitx limity))
  (when (> n 0)
   (when (< incx 0)
    (setf ix (the fixnum (* (the fixnum (1+ (the fixnum (- n)))) incx))))
   (when (< incy 0)
    (setf iy (the fixnum (* (the fixnum (1+ (the fixnum (- n)))) incy))))
   (do ((i 0 (1+ i)))
       ((or (>= i n) (>= i limitx) (>= i limity)))
     (setf dtemp (svref dx ix))
     (setf (svref dx ix) (svref dy iy))
     (setf (svref dy iy) dtemp)
     (setf ix (the fixnum (+ ix incx)))
     (setf iy (the fixnum (+ iy incy)))))
  (list dx dy)))
```

———-

— BLAS 1 dswap lisp test —

```
(load "dswap.lisp")
(setq a (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0))
; #(1.0 2.0 3.0 4.0 5.0)
(setq b (vector 9.0d0 8.0d0 7.0d0 6.0d0 -5.0d0))
; #(9.0 8.0 7.0 6.0 -5.0)
(dswap 5 a 1 b 1)
; (#(9.0 8.0 7.0 6.0 -5.0) #(1.0 2.0 3.0 4.0 5.0))
(setq a (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0))
; #(1.0 2.0 3.0 4.0 5.0)
(setq b (vector 9.0d0 8.0d0 7.0d0 6.0d0 -5.0d0))
; #(9.0 8.0 7.0 6.0 -5.0)
(dswap 3 a 2 b 2)
; (#(9.0 2.0 7.0 4.0 -5.0) #(1.0 8.0 3.0 6.0 5.0))
(setq a (vector 1.0d0 2.0d0 3.0d0 4.0d0 5.0d0))
; #(1.0 2.0 3.0 4.0 5.0)
(setq b (vector 9.0d0 8.0d0 7.0d0 6.0d0 -5.0d0))
; #(9.0 8.0 7.0 6.0 -5.0)
(dswap 5 a 1 b -1)
; (#(-5.0 6.0 7.0 8.0 9.0) #(5.0 4.0 3.0 2.0 1.0))
```

———-

7.14 dtrsv - solve $Ax = b$ or $A^T x = b$

7.14.1 Axiom unit tests

— **dtrsv.input** —

```
)set break resume
)sys rm -f dtrsv.output
)spool dtrsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

7.14.2 Axiom help page

— **dtrsv.help** —

```
======================================================================
dtrsv examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dtrsv (UPLO, TRANS, DIAG, N, A, LDA, X, INCX)
           DTRSV

Function/Subroutine Documentation
   subroutine dtrsv (character UPLO, character TRANS, character DIAG, integer
       N, double precision, dimension(lda,*) A, integer LDA, double precision,
       dimension(*) X, integer INCX)
       DTRSV
```

```
Purpose:
 DTRSV  solves one of the systems of equations
               A*x = b,   or   A**T*x = b,
 where b and x are n element vectors and A is an n by n unit, or
 non-unit, upper or lower triangular matrix.

 No test for singularity or near-singularity is included in this
 routine. Such tests must be performed before calling this routine.

Parameters:
 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the matrix is an upper or
   lower triangular matrix as follows:

     UPLO = 'U' or 'u'   A is an upper triangular matrix.
     UPLO = 'L' or 'l'   A is a lower triangular matrix.

 TRANS
   TRANS is CHARACTER*1
   On entry, TRANS specifies the equations to be solved as follows:

     TRANS = 'N' or 'n'   A*x = b.
     TRANS = 'T' or 't'   A**T*x = b.
     TRANS = 'C' or 'c'   A**T*x = b.

 DIAG
   DIAG is CHARACTER*1
   On entry, DIAG specifies whether or not A is unit triangular as follows:

     DIAG = 'U' or 'u'   A is assumed to be unit triangular.
     DIAG = 'N' or 'n'   A is not assumed to be unit triangular.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
   N must be at least zero.

 A
   A is DOUBLE PRECISION array of DIMENSION ( LDA, n ).

   Before entry with  UPLO = 'U' or 'u', the leading n by n
   upper triangular part of the array A must contain the upper
   triangular matrix and the strictly lower triangular part of
   A is not referenced.

   Before entry with UPLO = 'L' or 'l', the leading n by n
   lower triangular part of the array A must contain the lower
   triangular matrix and the strictly upper triangular part of
```

```
  A is not referenced.

  Note that when  DIAG = 'U' or 'u', the diagonal elements of
  A are not referenced either, but are assumed to be unity.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, n ).

X
  X is DOUBLE PRECISION array of dimension at least
  ( 1 + ( n - 1 )*abs( INCX ) ).

  Before entry, the incremented array X must contain the n
  element right-hand side vector b. On exit, X is overwritten
  with the solution vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.
```

———-

7.14.3 fortran code

— dtrsv.f —

```
*  ====================================================================
      SUBROUTINE DTRSV(UPLO,TRANS,DIAG,N,A,LDA,X,INCX)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,LDA,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A(LDA,*),X(*)
*     ..
*
*  ====================================================================
*
```

```
*     .. Parameters ..
      DOUBLE PRECISION ZERO
      PARAMETER (ZERO=0.0D+0)
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP
      INTEGER I,INFO,IX,J,JX,KX
      LOGICAL NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 6
      ELSE IF (INCX.EQ.0) THEN
          INFO = 8
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('DTRSV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX  too small for descending loops.
```

```
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x := inv( A )*x.
*
          IF (LSAME(UPLO,'U')) THEN
              IF (INCX.EQ.1) THEN
                  DO 20 J = N,1,-1
                      IF (X(J).NE.ZERO) THEN
                          IF (NOUNIT) X(J) = X(J)/A(J,J)
                          TEMP = X(J)
                          DO 10 I = J - 1,1,-1
                              X(I) = X(I) - TEMP*A(I,J)
   10                     CONTINUE
                      END IF
   20             CONTINUE
              ELSE
                  JX = KX + (N-1)*INCX
                  DO 40 J = N,1,-1
                      IF (X(JX).NE.ZERO) THEN
                          IF (NOUNIT) X(JX) = X(JX)/A(J,J)
                          TEMP = X(JX)
                          IX = JX
                          DO 30 I = J - 1,1,-1
                              IX = IX - INCX
                              X(IX) = X(IX) - TEMP*A(I,J)
   30                     CONTINUE
                      END IF
                      JX = JX - INCX
   40             CONTINUE
              END IF
          ELSE
              IF (INCX.EQ.1) THEN
                  DO 60 J = 1,N
                      IF (X(J).NE.ZERO) THEN
                          IF (NOUNIT) X(J) = X(J)/A(J,J)
                          TEMP = X(J)
                          DO 50 I = J + 1,N
                              X(I) = X(I) - TEMP*A(I,J)
   50                     CONTINUE
                      END IF
```

```
   60             CONTINUE
              ELSE
                  JX = KX
                  DO 80 J = 1,N
                      IF (X(JX).NE.ZERO) THEN
                          IF (NOUNIT) X(JX) = X(JX)/A(J,J)
                          TEMP = X(JX)
                          IX = JX
                          DO 70 I = J + 1,N
                              IX = IX + INCX
                              X(IX) = X(IX) - TEMP*A(I,J)
   70                     CONTINUE
                      END IF
                      JX = JX + INCX
   80             CONTINUE
              END IF
          END IF
      ELSE
*
*        Form  x := inv( A**T )*x.
*
          IF (LSAME(UPLO,'U')) THEN
              IF (INCX.EQ.1) THEN
                  DO 100 J = 1,N
                      TEMP = X(J)
                      DO 90 I = 1,J - 1
                          TEMP = TEMP - A(I,J)*X(I)
   90                 CONTINUE
                      IF (NOUNIT) TEMP = TEMP/A(J,J)
                      X(J) = TEMP
  100             CONTINUE
              ELSE
                  JX = KX
                  DO 120 J = 1,N
                      TEMP = X(JX)
                      IX = KX
                      DO 110 I = 1,J - 1
                          TEMP = TEMP - A(I,J)*X(IX)
                          IX = IX + INCX
  110                 CONTINUE
                      IF (NOUNIT) TEMP = TEMP/A(J,J)
                      X(JX) = TEMP
                      JX = JX + INCX
  120             CONTINUE
              END IF
          ELSE
              IF (INCX.EQ.1) THEN
                  DO 140 J = N,1,-1
                      TEMP = X(J)
                      DO 130 I = N,J + 1,-1
```

```
                        TEMP = TEMP - A(I,J)*X(I)
  130                 CONTINUE
                      IF (NOUNIT) TEMP = TEMP/A(J,J)
                      X(J) = TEMP
  140             CONTINUE
              ELSE
                  KX = KX + (N-1)*INCX
                  JX = KX
                  DO 160 J = N,1,-1
                      TEMP = X(JX)
                      IX = KX
                      DO 150 I = N,J + 1,-1
                          TEMP = TEMP - A(I,J)*X(IX)
                          IX = IX - INCX
  150                 CONTINUE
                      IF (NOUNIT) TEMP = TEMP/A(J,J)
                      X(JX) = TEMP
                      JX = JX - INCX
  160             CONTINUE
              END IF
          END IF
      END IF
*
      RETURN
*
*     End of DTRSV .
*
      END
```

———-

7.14.4 lisp code

— BLAS 1 dtrsv —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun dtrsv (uplo trans diag n a lda x incx)
    (declare (type (array double-float (*)) x a)
             (type (f2cl-lib:integer4) incx lda n)
             (type (string *) diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a double-float a-%data% a-%offset%)
```

```
   (x double-float x-%data% x-%offset%))
(prog ((nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0) (kx 0)
       (temp 0.0d0))
  (declare (type f2cl-lib:logical nounit)
           (type (f2cl-lib:integer4) i info ix j jx kx)
           (type (double-float) temp))
  (setf info 0)
  (cond
   ((and
     (not
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame uplo "U")
        (declare (ignore var-1))
        (when var-0 (setf uplo var-0))
        ret-val))
     (not
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame uplo "L")
        (declare (ignore var-1))
        (when var-0 (setf uplo var-0))
        ret-val)))
    (setf info 1))
   ((and
     (not
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame trans "N")
        (declare (ignore var-1))
        (when var-0 (setf trans var-0))
        ret-val))
     (not
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame trans "T")
        (declare (ignore var-1))
        (when var-0 (setf trans var-0))
        ret-val))
     (not
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame trans "C")
        (declare (ignore var-1))
        (when var-0 (setf trans var-0))
        ret-val)))
    (setf info 2))
   ((and
     (not
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame diag "U")
        (declare (ignore var-1))
        (when var-0 (setf diag var-0))
        ret-val))
     (not
```

```
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame diag "N")
      (declare (ignore var-1))
      (when var-0 (setf diag var-0))
      ret-val)))
  (setf info 3))
 ((< n 0) (setf info 4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info 6))
 ((= incx 0) (setf info 8)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "DTRSV " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (= n 0)
    (go end_label))
(setf nounit
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame diag "N")
          (declare (ignore var-1))
          (when var-0 (setf diag var-0))
          ret-val))
(cond
 ((<= incx 0)
  (setf kx
          (f2cl-lib:int-sub 1
                            (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                              incx))))
 ((/= incx 1) (setf kx 1)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame trans "N")
    (declare (ignore var-1))
    (when var-0 (setf trans var-0))
    ret-val)
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)
    (cond
     ((= incx 1)
      (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                    ((> j 1) nil)
        (tagbody
          (cond
```

```
      ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
       (if nounit
           (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                   (/
                    (f2cl-lib:fref x-%data% (j) ((1 *))
                                   x-%offset%)
                    (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                   a-%offset%))))
       (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
       (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                      (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     ((> i 1) nil)
         (tagbody
           (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                   (-
                    (f2cl-lib:fref x-%data% (i) ((1 *))
                                   x-%offset%)
                    (* temp
                       (f2cl-lib:fref a-%data% (i j)
                                      ((1 lda) (1 *))
                                      a-%offset%))))
          label10))))
    label20)))
(t
 (setf jx
         (f2cl-lib:int-add kx
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-sub n 1) incx)))
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (cond
      ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
       (if nounit
           (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                   (/
                    (f2cl-lib:fref x-%data% (jx) ((1 *))
                                   x-%offset%)
                    (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                   a-%offset%))))
       (setf temp
               (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
       (setf ix jx)
       (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                      (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     ((> i 1) nil)
         (tagbody
           (setf ix (f2cl-lib:int-sub ix incx))
           (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                   (-
```

```
                        (f2cl-lib:fref x-%data% (ix) ((1 *))
                                       x-%offset%)
                        (* temp
                           (f2cl-lib:fref a-%data% (i j)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
            label30))))
       (setf jx (f2cl-lib:int-sub jx incx))
      label40)))))
(t
 (cond
  ((= incx 1)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
         (if nounit
             (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                     (/
                      (f2cl-lib:fref x-%data% (j) ((1 *))
                                     x-%offset%)
                      (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%))))
         (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
         (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                        (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                     (-
                      (f2cl-lib:fref x-%data% (i) ((1 *))
                                     x-%offset%)
                      (* temp
                         (f2cl-lib:fref a-%data% (i j)
                                        ((1 lda) (1 *))
                                        a-%offset%))))
            label50))))
      label60)))
  (t (setf jx kx)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
         (if nounit
             (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                     (/
                      (f2cl-lib:fref x-%data% (jx) ((1 *))
                                     x-%offset%)
```

```
                    (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                   a-%offset%))))
          (setf temp
                (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
          (setf ix jx)
          (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                           (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf ix (f2cl-lib:int-add ix incx))
              (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                    (-
                     (f2cl-lib:fref x-%data% (ix) ((1 *))
                                    x-%offset%)
                     (* temp
                        (f2cl-lib:fref a-%data% (i j)
                                       ((1 lda) (1 *))
                                       a-%offset%))))
              label70))))
        (setf jx (f2cl-lib:int-add jx incx))
       label80)))))))
(t
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame uplo "U")
     (declare (ignore var-1))
     (when var-0 (setf uplo var-0))
     ret-val)
   (cond
    ((= incx 1)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i
                           (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                        nil)
           (tagbody
             (setf temp
                     (- temp
                        (*
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)
                         (f2cl-lib:fref x-%data% (i) ((1 *))
                                        x-%offset%))))
             label90))
         (if nounit
             (setf temp
                     (/ temp
```

```
                        (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                       a-%offset%))))
          (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
         label100)))
      (t (setf jx kx)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
          (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
          (setf ix kx)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf temp
                      (- temp
                         (*
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref x-%data% (ix) ((1 *))
                                         x-%offset%))))
              (setf ix (f2cl-lib:int-add ix incx))
             label110))
          (if nounit
              (setf temp
                      (/ temp
                         (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                        a-%offset%))))
          (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
          (setf jx (f2cl-lib:int-add jx incx))
         label120)))))
    (t
     (cond
      ((= incx 1)
       (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     ((> j 1) nil)
         (tagbody
          (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
          (f2cl-lib:fdo (i n (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                        ((> i (f2cl-lib:int-add j 1)) nil)
            (tagbody
              (setf temp
                      (- temp
                         (*
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref x-%data% (i) ((1 *))
                                         x-%offset%))))
             label130))
```

```
            (if nounit
                (setf temp
                        (/ temp
                           (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                          a-%offset%))))
            (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
           label140)))
        (t
         (setf kx
                 (f2cl-lib:int-add kx
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-sub n 1) incx)))
         (setf jx kx)
         (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((> j 1) nil)
           (tagbody
             (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
             (setf ix kx)
             (f2cl-lib:fdo (i n (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                           ((> i (f2cl-lib:int-add j 1)) nil)
               (tagbody
                 (setf temp
                         (- temp
                            (*
                             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                            a-%offset%)
                             (f2cl-lib:fref x-%data% (ix) ((1 *))
                                            x-%offset%))))
                 (setf ix (f2cl-lib:int-sub ix incx))
                label150))
             (if nounit
                 (setf temp
                         (/ temp
                            (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                           a-%offset%))))
             (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
             (setf jx (f2cl-lib:int-sub jx incx))
            label160))))))))
  (go end_label)
end_label
  (return (values uplo trans diag nil nil nil nil nil))))))
```

———-

7.15 dzasum - sum of the dcabs1 of a complex vector

7.15.1 Axiom unit tests

— dzasum.input —

```
)set break resume
)sys rm -f dzasum.output
)spool dzasum.output
)set message test on
)set message auto off
)clear all

--S 1 of 4
d:PRIMARR(COMPLEX(DFLOAT)):=[[1.0+2.0*%i,-3.0+4.0*%i,5.0-6.0*%i]]
--R
--R   (1)  [1. + 2. %i,- 3. + 4. %i,5. - 6. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 1

--S 2 of 4
dzasum(3,d,1) -- 21.0
--R
--R   (2)  21.
--R                                                          Type: DoubleFloat
--E 2

--S 3 of 4
dzasum(3,d,2) -- 14.0
--R
--R   (3)  14.
--R                                                          Type: DoubleFloat
--E 3

--S 4 of 4
dzasum(-3,d,1) -- 0.0
--R
--R   (4)  0.
--R                                                          Type: DoubleFloat
--E 4

)spool
)lisp (bye)
```

———-

7.15.2 Axiom help page

— dzasum.help —

```
====================================================================
dzasum examples
====================================================================

d:PRIMARR(COMPLEX(DFLOAT)):=[[1.0+2.0*%i,-3.0+4.0*%i,5.0-6.0*%i]]

  [1. + 2. %i,- 3. + 4. %i,5. - 6. %i]

dzasum(3,d,1)

  21.  = (1+2) + (abs(-3)+4) + (5+abs(-6))

dzasum(3,d,2)

  14.  = (1+2) + (5+abs(-6))

dzasum(-3,d,1)

  0.   -- never execute for negative arguments

====================================================================
Man Page Details
====================================================================

NAME
       DZASUM - BLAS level one, sums the absolute values of the real and imag-
       inary parts of the elements of a double complex vector

SYNOPSIS
       DOUBLE PRECISION FUNCTION DZASUM ( n, x, incx )

           INTEGER                     n, incx

           DOUBLE COMPLEX              x


DESCRIPTION
       This routine performs the following vector operation:

                            n
               DZASUM <-- Sum abs( real(x(i)) ) + abs( aimag(x(i)) )
                           i=1

ARGUMENTS
       n       INTEGER. (input)
```

```
               Number of vector elements to be summed.

       x       DOUBLE COMPLEX. (input)
               Array of dimension (n-1) * abs(incx) + 1.
               Vector that contains elements to be summed.

       incx    INTEGER. (input)
               Increment between elements of x.
               If incx = 0, the results will be unpredictable.

RETURN VALUES
       DZASUM  DOUBLE PRECISION. (output)
               Sum of the absolute values of the real and imaginary  parts  of
               the elements of the vector x.
               If n <= 0, DZASUM is set to 0.

NOTES
       When working backward (incx < 0), each routine starts at the end of the
       vector and moves backward, as follows:

            x(1-incx * (n-1)), x(1-incx * (n-2)), ..., x(1)
```

———-

Computes (complex double-float) $asum \leftarrow ||re(x)||_1 + ||im(x)||_1$

7.15.3 fortran code

— dzasum.f —

```
*  ====================================================================
      DOUBLE PRECISION FUNCTION DZASUM(N,ZX,INCX)
*
*  -- Reference BLAS level1 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER INCX,N
*     ..
*     .. Array Arguments ..
      COMPLEX*16 ZX(*)
*     ..
*
*  ====================================================================
*
```

```
*     .. Local Scalars ..
      DOUBLE PRECISION STEMP
      INTEGER I,NINCX
*     ..
*     .. External Functions ..
      DOUBLE PRECISION DCABS1
      EXTERNAL DCABS1
*     ..
      DZASUM = 0.0d0
      STEMP = 0.0d0
      IF (N.LE.0 .OR. INCX.LE.0) RETURN
      IF (INCX.EQ.1) THEN
*
*        code for increment equal to 1
*
         DO I = 1,N
            STEMP = STEMP + DCABS1(ZX(I))
         END DO
      ELSE
*
*        code for increment not equal to 1
*
         NINCX = N*INCX
         DO I = 1,NINCX,INCX
            STEMP = STEMP + DCABS1(ZX(I))
         END DO
      END IF
      DZASUM = STEMP
      RETURN
      END
```

7.15.4 fortran unit test

— dzasum example —

```
       program dzasumEX
*      Tim Daly May 7, 2012
*      unit tests for BLAS dzasum
       double complex a(3)
       double precision b
       a = (/ (1.0D0,2.0D0), (-3.0D0,4.0D0), (5.0D0,-6.0D0)/)

       write(6,10)
 10    format(/,"before:")
       write(6,100)dble(a(1)),dimag(a(1))
```

```
 100   format("a(1)=C(",f6.3," ",f6.3,")")
       write(6,101)dble(a(2)),dimag(a(2))
 101   format("a(2)=C(",f6.3," ",f6.3,")")
       write(6,102)dble(a(3)),dimag(a(3))
 102   format("a(3)=C(",f6.3," ",f6.3,")")

       b=dzasum(3,a,1)
       write(6,11)
 11    format(/,"should be 21:")
       write(6,200)b
 200   format("b=",f6.3)

       b=dzasum(3,a,2)
       write(6,12)
 12    format(/,"should be 14:")
       write(6,200)b

       b=dzasum(-3,a,1)
       write(6,13)
 13    format(/,"should be 0:")
       write(6,200)b

       stop
       end
```

```
gcc -o dzasumEX dzasumEX.f -lgfortran dzasum.o dcabs1.o && ./dzasumEX

before:
a(1)=C( 1.000  2.000)
a(2)=C(-3.000  4.000)
a(3)=C( 5.000 -6.000)

should be 21:
b=21.000

should be 14:
b=14.000

should be 0:
b= 0.000
```

Spad represents complex numbers as a pair where the car is the real part and the cons is the imaginary part. BLAS wants a complex number in fortran format. So we have a design choice to make. Either we could write all of the BLAS code using Spad internal representation or we could follow the BLAS code standard. I've decided to follow the BLAS code standard so we need to create thunks to do the data translation.

7.15.5 lisp code

— BLAS 1 dzasum —

```
(defun dzasumSpad (n zx incx)
; Tim Daly May 7, 2012
 (let (result vec)
  (dotimes (i (length zx))
   (push (complex (car (svref zx i)) (cdr (svref zx i))) result))
  (setq vec (make-array (length result) :initial-contents (nreverse result)))
  (dzasum n vec incx)))

(defun dzasum (n zx incx)
; Tim Daly May 7, 2012
 (declare (type (simple-array (complex double-float) (*)) zx)
          (type fixnum incx n))
 (let ((ix 0) (stemp 0.0d0) (dzasum 0.0d0) (limit (length zx)))
      (declare (type (double-float) dzasum stemp)
               (type fixnum ix limit))
  (when (and (> n 0) (> incx 0))
   (do ((i 0 (1+ i)))
       ((or (>= i n) (>= ix limit)) nil)
    (setf dzasum (+ dzasum (the double-float (dcabs1 (svref zx ix)))))
    (setf ix (+ ix incx))))
  dzasum))
```

———-

— BLAS 1 dzasum lisp test —

```
(load "dzasum.lisp")
(load "dcabs1.lisp")
(setq a (vector #C(1.0d0 2.0d0) #C(-3.0d0 4.0d0) #C(5.0d0 -6.0d0)))
; #(#C(1.0 2.0) #C(-3.0 4.0) #C(5.0 -6.0))
(dzasum 3 a 1)
; 21.0
(dzasum 3 a 2)
; 14.0
(dzasum -3 a 1)
; 0.0
```

———-

7.16 dznrm2 - Euclidean norm of a vector = $\sqrt{x^H * x}$

7.16.1 Axiom unit tests

— dznrm2.input —

```
)set break resume
)sys rm -f dznrm2.output
)spool dznrm2.output
)set message test on
)set message auto off
)clear all

--S 1 of 8
a:PRIMARR(COMPLEX(DFLOAT)):=_
  [[3.+4.*%i, -4.+5.*%i, 5.+6.*%i, 7.-8.*%i, -9.-2.*%i]]
--R
--R   (1)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 1

--S 2 of 8
dznrm2(5,a,1) -- should be 18.028
--R
--R   (2)  18.027756377319946
--R                                                          Type: DoubleFloat
--E 2

--S 3 of 8
dznrm2(3,a,2) -- should be 13.077
--R
--R   (3)  13.076696830622021
--R                                                          Type: DoubleFloat
--E 3

--S 4 of 8
dznrm2(3,a,1) -- should be 11.269
--R
--R   (4)  11.269427669584644
--R                                                          Type: DoubleFloat
--E 4

--S 5 of 8
dznrm2(3,a,-1) -- should be 0.0
--R
--R   (5)  0.
--R                                                          Type: DoubleFloat
--E 5
```

```
--S 6 of 8
dznrm2(-3,a,-1) -- should be 0.0
--R
--R   (6)  0.
--R                                                        Type: DoubleFloat
--E 6

--S 7 of 8
dznrm2(1,a,1) -- should be 5.0
--R
--R   (7)  5.
--R                                                        Type: DoubleFloat
--E 7

--S 8 of 8
dznrm2(1,a,2) -- should be 5.0
--R
--R   (8)  5.
--R                                                        Type: DoubleFloat
--E 8

)spool
)lisp (bye)
```

———-

7.16.2 Axiom help page

— dznrm2.help —

```
=====================================================================
dznrm2 examples
=====================================================================

dznrm2 computes the complex norm of the input vector.
For each element it computes the sum
    a.i*conjugate(a.i)
for all i. Then it returns the square root of the final sum.

If the count or increment are negative, the result is 0.0


a:PRIMARR(COMPLEX(DFLOAT)):=_
  [[3.+4.*%i, -4.+5.*%i, 5.+6.*%i, 7.-8.*%i, -9.-2.*%i]]

  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
```

```
dznrm2(5,a,1)

  18.027756377319946

dznrm2(3,a,2)

  13.076696830622021

dznrm2(3,a,1)

  11.269427669584644

dznrm2(3,a,-1)

  0.

dznrm2(-3,a,-1)

  0.

dznrm2(1,a,1)

  5.

dznrm2(1,a,2)
  5.

====================================================================
Man Page Details
====================================================================

NAME
       DZNRM2 - BLAS level one, computes the Euclidean norm of a vector

SYNOPSIS
       DOUBLE PRECISION FUNCTION DZNRM2 ( n, x, incx )

           INTEGER                        n, incx

           DOUBLE COMPLEX                 x


DESCRIPTION
       DZNRM2  computes  the Euclidean (L2) norm of a double precision complex
       vector:

               DZNRM2 <-- ||x||
                               2

       where x is a double precision complex vector.
```

```
ARGUMENTS
       n       INTEGER (input)
               Number of elements in the operand vector.

       x       DOUBLE COMPLEX (input)
               Array of dimension (n-1) * |incx| + 1.
               Array x contains the operand vector.

       incx    INTEGER (input)
               Increment between elements of x.
               If incx = 0, the results will be unpredictable.

RETURN VALUES
       DZNRM2  DOUBLE PRECISION Result (Euclidean norm). (output)
               If n <= 0, DZNRM2 is set to 0d0.

NOTES
       When  working backward (incx < 0), DZNRM2 starts at the end of the vec-
       tor and moves backward, as follows:

               x(1-incx * (n-1)), x(1-incx * (n-2)), ..., x(1)
```

———-

7.16.3 fortran code

— dznrm2.f —

```
*  ====================================================================
      DOUBLE PRECISION FUNCTION DZNRM2(N,X,INCX)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,N
*     ..
*     .. Array Arguments ..
      COMPLEX*16 X(*)
*     ..
*
*  ====================================================================
*
```

```
*     .. Parameters ..
      DOUBLE PRECISION ONE,ZERO
      PARAMETER (ONE=1.0D+0,ZERO=0.0D+0)
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION NORM,SCALE,SSQ,TEMP
      INTEGER IX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC ABS,DBLE,DIMAG,SQRT
*     ..
      IF (N.LT.1 .OR. INCX.LT.1) THEN
          NORM = ZERO
      ELSE
          SCALE = ZERO
          SSQ = ONE
*        The following loop is equivalent to this call to the LAPACK
*        auxiliary routine:
*        CALL ZLASSQ( N, X, INCX, SCALE, SSQ )
*
          DO 10 IX = 1,1 + (N-1)*INCX,INCX
              IF (DBLE(X(IX)).NE.ZERO) THEN
                  TEMP = ABS(DBLE(X(IX)))
                  IF (SCALE.LT.TEMP) THEN
                      SSQ = ONE + SSQ* (SCALE/TEMP)**2
                      SCALE = TEMP
                  ELSE
                      SSQ = SSQ + (TEMP/SCALE)**2
                  END IF
              END IF
              IF (DIMAG(X(IX)).NE.ZERO) THEN
                  TEMP = ABS(DIMAG(X(IX)))
                  IF (SCALE.LT.TEMP) THEN
                      SSQ = ONE + SSQ* (SCALE/TEMP)**2
                      SCALE = TEMP
                  ELSE
                      SSQ = SSQ + (TEMP/SCALE)**2
                  END IF
              END IF
   10     CONTINUE
          NORM = SCALE*SQRT(SSQ)
      END IF
*
      DZNRM2 = NORM
      RETURN
*
*     End of DZNRM2.
*
      END
```

7.16.4 fortran unit test

— dznrm2 example —

```
      program dznrm2EX
*     Tim Daly May 8, 2012
*     unit tests for BLAS dznrm2
      complex*16 a(5)
      double precision c
      a(1) = ( 3.0D0, 4.0D0)
      a(2) = (-4.0D0, 5.0D0)
      a(3) = ( 5.0D0, 6.0D0)
      a(4) = ( 7.0D0,-8.0D0)
      a(5) = (-9.0D0,-2.0D0)
      write(6,100)a(1),a(2),a(3),a(4),a(5)
 100  format("a=(/ (",f6.3,f6.3,"),(",f6.3,f6.3"),(",f6.3,f6.3"),",/,
    C        "     (",f6.3,f6.3,"),(",f6.3,f6.3") /)")
      c=dznrm2(5,a,1)
      write(6,200)c
 200  format("should be 18.028",/,"c=",f6.3)
      c=dznrm2(3,a,2)
      write(6,201)c
 201  format("should be 13.077",/,"c=",f6.3)
      c=dznrm2(3,a,1)
      write(6,202)c
 202  format("should be 11.269",/,"c=",f6.3)
      c=dznrm2(3,a,-1)
      write(6,203)c
 203  format("should be 0.000",/,"c=",f6.3)
      c=dznrm2(-3,a,-1)
      write(6,204)c
 204  format("should be 0.000",/,"c=",f6.3)
      c=dznrm2(1,a,1)
      write(6,205)c
 205  format("should be 5.000",/,"c=",f6.3)
      c=dznrm2(1,a,2)
      write(6,206)c
 206  format("should be 5.000",/,"c=",f6.3)
      stop
      end
```

```
gcc -o dznrm2EX dznrm2EX.f -lgfortran dznrm2.o && ./dznrm2EX
a=(/ ( 3.000 4.000),(-4.000 5.000),( 5.000 6.000),
```

```
     ( 7.000-8.000),(-9.000-2.000) /)
should be 18.028
c=18.028
should be 13.077
c=13.077
should be 11.269
c=11.269
should be 0.000
c= 0.000
should be 0.000
c= 0.000
should be 5.000
c= 5.000
should be 5.000
c= 5.000
```

Spad represents complex numbers as a pair where the car is the real part and the cons is the imaginary part. BLAS wants a complex number in fortran format. So we have a design choice to make. Either we could write all of the BLAS code using Spad internal representation or we could follow the BLAS code standard. I've decided to follow the BLAS code standard so we need to create thunks to do the data translation.

NOTE: The Axiom internal representation of PRIMARR(COMPLEX(FLOAT)) is an array where each array element AE is

```
AE = (cons (cons (therealpart 0)) (cons (theimagpart 0)))
```

so we need (caar AE) and (cadr AE) to get the real and imag parts.

7.16.5 lisp code

— BLAS 1 dznrm2 —

```
(defun dznrm2Spad (n zx incx)
; Tim Daly May 8, 2012
 (let (result vec)
  (dotimes (i (length zx))
   (push (complex (car (svref zx i)) (cdr (svref zx i))) result))
  (setq vec (make-array (length result) :initial-contents (nreverse result)))
  (dznrm2 n vec incx)))

(defun dznrm2 (n x incx)
; Tim Daly May 8, 2012
 (declare (type (simple-array (complex double-float) (*)) x)
          (type fixnum incx n))
 (let ((scale 0.0d0) (ssq 0.0d0) (temp 0.0d0)
       (dznrm2 0.0d0) (limitx (min (* n incx) (length x))))
```

```
  (declare (type fixnum limitx) (type (double-float) scale ssq temp dznrm2))
  (when (and (>= n 1) (>= incx 1))
   (do ((ix 0 (the fixnum (+ ix incx))))
        ((>= ix limitx))
    (when (/= (the double-float (realpart
                 (the (complex double-float) (svref x ix))))
               0.0d0)
      (setf temp
       (the double-float (abs
        (the double-float (realpart
         (the (complex double-float) (svref x ix)))))))
      (cond
       ((< scale temp)
         (setf ssq (+ 1.0d0
                    (the double-float (*
                      ssq
                      (the double-float (*
                       (the double-float (/ scale temp))
                       (the double-float (/ scale temp))))))))
         (setf scale temp))
       (t
         (setf ssq (+ ssq (*
                           (the double-float (/ temp scale))
                           (the double-float (/ temp scale)))))))
      (when (/= (the double-float (imagpart
                 (the (complex double-float) (svref x ix)))) 0.0d0)
        (setf temp
         (the double-float (abs
          (the double-float (imagpart
           (the (complex double-float) (svref x ix)))))))
        (cond
         ((< scale temp)
           (setf ssq (+ 1.0d0 (* ssq (*
                                      (the double-float (/ scale temp))
                                      (the double-float (/ scale temp))))))
           (setf scale temp))
          (t
           (setf ssq (+ ssq (*
                             (the double-float (/ temp scale))
                             (the double-float (/ temp scale)))))))))
    (setf dznrm2 (the double-float (* scale (the double-float (sqrt ssq))))))
  dznrm2))
```

———-

— BLAS 1 dznrm2 lisp test —

```
(load "dznrm2.lisp")
```

```
(setq a (vector #C(3.0d0 4.0d0) #C(-4.0d0 5.0d0) #C(5.0d0 6.0d0)
                #C(7.0d0 -8.0d0) #C(-9.0d0 -2.0d0)))
; #(#C(3.0 4.0) #C(-4.0 5.0) #C(5.0 6.0) #C(7.0 -8.0) #C(-9.0 -2.0))
(dznrm2 5 a 1)
; 18.027756377319946
(dznrm2 3 a 2)
; 13.076696830622023
(dznrm2 3 a 1)
; 11.269427669584648
(dznrm2 3 a -1)
; 0.0
(dznrm2 -3 a -1)
; 0.0
(dznrm2 1 a 1)
; 5.0
(dznrm2 1 a 2)
; 5.0
```

———-

7.17 icamax - maximum index function

7.17.1 Axiom unit tests

— **icamax.input** —

```
)set break resume
)sys rm -f icamax.output
)spool icamax.output
)set message test on
)set message auto off
)clear all

--S 1 of 7
a:PRIMARR(COMPLEX(FLOAT))
--R                                                                Type: Void
--E 1

--S 2 of 7
a:=[[3.+4.*%i,-4.+5.*%i,5.+6.*%i,7.-8.*%i,-9.-2.*%i]]
--R
--R   (9)  [3.0 + 4.0 %i,- 4.0 + 5.0 %i,5.0 + 6.0 %i,7.0 - 8.0 %i,- 9.0 - 2.0 %i]
--R                                        Type: PrimitiveArray(Complex(Float))
--E 2

--S 3 of 7
icamax(5,a,1)  -- should be 3
```

```
--R
--R   (10)  3
--R                                                    Type: PositiveInteger
--E 3

--S 4 of 7
icamax(0,a,1)  -- should be -1
--R
--R   (11)  - 1
--R                                                            Type: Integer
--E 4

--S 5 of 7
icamax(5,a,-1) -- should be -1
--R
--R   (12)  - 1
--R                                                            Type: Integer
--E 5

--S 6 of 7
icamax(3,a,1)  -- should be 2
--R
--R   (13)  2
--R                                                    Type: PositiveInteger
--E 6

--S 7 of 7
icamax(3,a,2)  -- should be 1
--R
--R   (14)  1
--R                                                    Type: PositiveInteger
--E 7

)spool
)lisp (bye)
```

———-

7.17.2 Axiom help page

— icamax.help —

```
====================================================================
icamax examples
====================================================================

a:PRIMARR(COMPLEX(FLOAT))
```

```
a:=[[3.+4.*%i,-4.+5.*%i,5.+6.*%i,7.-8.*%i,-9.-2.*%i]]

  [3.0 + 4.0 %i,- 4.0 + 5.0 %i,5.0 + 6.0 %i,7.0 - 8.0 %i,- 9.0 - 2.0 %i]

Note that Axiom arrays are 0-based

icamax(5,a,1)  -- should be 3

  3

If the arguments are not valid, return an invalid array index

The count must be greater than 0.

icamax(0,a,1)  -- should be -1

  - 1

The increment must be positive

icamax(5,a,-1) -- should be -1

  - 1

icamax(3,a,1)  -- should be 2

  2

icamax(3,a,2)  -- should be 1

  1

======================================================================
Man Page Details
======================================================================

NAME
       ICAMAX - BLAS level one, maximum index function

SYNOPSIS
       INTEGER FUNCTION ICAMAX ( n, x, incx )

           INTEGER               n, incx

           COMPLEX               x


DESCRIPTION
       ICAMAX  searches a complex vector for the first occurrence of the maxi-
       mum absolute value.
```

```
       ICAMAX determines the first index i such that

       |Real(x )|+ |Imag(x ) | = MAX(|Real(x )| + | Imag(x )|): j = 1, ..., n
              i           i                 j             j
               where x  is an element of a complex vector.
                      j

ARGUMENTS
       n       INTEGER. (input)
               Number of elements to process in the vector to be searched.  If
               n <= 0, these routines return 0.

       x       COMPLEX. (input)
               Array of dimension (n-1) * |incx| + 1.
               Array x contains the vector to be searched.

       incx    INTEGER. (input)
               Increment between elements of x.

RETURN VALUES
       ICAMAX  INTEGER. (output)
               Return  the first index of the maximum absolute value of vector
               x.  The vector x has length n and increment incx.

NOTES
       When working backward (incx < 0), each routine starts at the end of the
       vector and moves backward, as follows:

            x(1-incx * (n-1)), x(1-incx * (n-2)), ..., x(1)

       The largest absolute value is:

            ABS (x(1+(index-1) * incx)) when incx > 0

            ABS (x(1+(n-index) * |incx|)) when incx < 0


            ———-
```

7.17.3 fortran code

— icamax.f —

```
*  ====================================================================
      INTEGER FUNCTION ICAMAX(N,CX,INCX)
*
*  -- Reference BLAS level1 routine (version 3.6.0) --
```

```
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER INCX,N
*     ..
*     .. Array Arguments ..
      COMPLEX CX(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      REAL SMAX
      INTEGER I,IX
*     ..
*     .. External Functions ..
      REAL SCABS1
      EXTERNAL SCABS1
*     ..
      ICAMAX = 0
      IF (N.LT.1 .OR. INCX.LE.0) RETURN
      ICAMAX = 1
      IF (N.EQ.1) RETURN
      IF (INCX.EQ.1) THEN
*
*        code for increment equal to 1
*
         SMAX = SCABS1(CX(1))
         DO I = 2,N
            IF (SCABS1(CX(I)).GT.SMAX) THEN
               ICAMAX = I
               SMAX = SCABS1(CX(I))
            END IF
         END DO
      ELSE
*
*        code for increment not equal to 1
*
         IX = 1
         SMAX = SCABS1(CX(1))
         IX = IX + INCX
         DO I = 2,N
            IF (SCABS1(CX(IX)).GT.SMAX) THEN
               ICAMAX = I
               SMAX = SCABS1(CX(IX))
            END IF
            IX = IX + INCX
         END DO
```

```
      END IF
      RETURN
      END
```

———-

7.17.4 fortran unit test

— icamax example —

```
      program icamaxEX
*     Tim Daly May 17, 2012
*     unit tests for BLAS icamax
      complex a(5)
      integer n
      a(1) = ( 1.0, 2.0)
      a(2) = (-2.0, 2.0)
      a(3) = ( 2.0,-1.0)
      a(4) = ( 2.0,-3.0)
      a(5) = (-1.0,-0.0)
      write(6,100)a(1),a(2),a(3),a(4),a(5)
 100  format("a=(/ (",f6.3,f6.3,"),(",f6.3,f6.3"),(",f6.3,f6.3"),",/,
    C        "     (",f6.3,f6.3,"),(",f6.3,f6.3") /)")
      n=icamax(5,a,1)
      write(6,200)n
 200  format("should be 4",/,"n=",i3)
      n=icamax(3,a,1)
      write(6,201)n
 201  format("should be 2",/,"n=",i3)
      n=icamax(0,a,1)
      write(6,202)n
 202  format("should be 0",/,"n=",i3)
      n=icamax(-5,a,1)
      write(6,203)n
 203  format("should be 0",/,"n=",i3)
      n=icamax(5,a,-1)
      write(6,204)n
 204  format("should be 0",/,"n=",i3)
      n=icamax(5,a,2)
      write(6,205)n
 205  format("should be 1",/,"n=",i3)
      n=icamax(1,a,0)
      write(6,206)n
 206  format("should be 0",/,"n=",i3)
      n=icamax(1,a,-1)
      write(6,207)n
 207  format("should be 0",/,"n=",i3)
```

```
      n=icamax(1,a,5)
      write(6,208)n
 208  format("should be 1",/,"n=",i3)
      a(1) = ( 1.0, 2.0)
      a(2) = (-2.0, 2.0)
      a(3) = ( 2.0,-1.0)
      a(4) = (-2.0,-3.0)
      a(5) = (-1.0,-0.0)
      write(6,100)a(1),a(2),a(3),a(4),a(5)
      n=icamax(5,a,1)
      write(6,209)n
 209  format("should be 4",/,"n=",i3)
      stop
      end
```

```
gcc -o icamaxEX icamaxEX.f -lgfortran icamax.o && ./icamaxEX
a=(/ ( 1.000 2.000),(-2.000 2.000),( 2.000-1.000),
     ( 2.000-3.000),(-1.000-0.000) /)
should be 4
n=  4
should be 2
n=  2
should be 0
n=  0
should be 0
n=  0
should be 0
n=  0
should be 1
n=  1
should be 0
n=  0
should be 0
n=  0
should be 1
n=  1
a=(/ ( 1.000 2.000),(-2.000 2.000),( 2.000-1.000),
     (-2.000-3.000),(-1.000-0.000) /)
should be 4
n=  4
```

Spad represents complex numbers as a pair where the car is the real part and the cons is the imaginary part. BLAS wants a complex number in fortran format. So we have a design choice to make. Either we could write all of the BLAS code using Spad internal representation or we could follow the BLAS code standard. I've decided to follow the BLAS code standard so we need to create thunks to do the data translation.

NOTE: The Axiom internal representation of PRIMARR(COMPLEX(FLOAT)) is an array where each array element AE is

```
AE = (cons (cons (therealpart 0)) (cons (theimagpart 0)))
```

so we need (caar AE) and (cadr AE) to get the real and imag parts.

NOTE: Array indexing in Axiom is 0-based but Fortran is 1-based so the results are displaced by 1.

7.17.5 lisp code

— BLAS 1 icamax —

```
(defun icamaxSpad (n zx incx)
; Tim Daly May 13, 2012
 (let (result vec)
  (dotimes (i (length zx))
   (push (complex
          (coerce (caar (svref zx i)) 'single-float)
          (coerce (cadr (svref zx i))'single-float)) result))
  (setq vec (make-array (length result) :initial-contents (nreverse result)
                        :element-type '(complex single-float)))
  (icamax n vec incx)))

(defun icamax (n cx incx)
 (declare (type (simple-array (complex single-float) (*)) cx)
          (type fixnum incx n))
 (labels (
  (cabs1 (zdum)
   (the single-float
    (+ (the single-float (abs (the single-float (realpart zdum))))
       (the single-float (abs (the single-float (imagpart zdum))))))))
 (declare (ftype (function (complex single-float) single-float) cabs1))
 (let ((ix 0) (smax 0.0f0) (icamax -1) (limit (length cx)))
  (declare (type (single-float) smax) (type fixnum icamax ix limit))
  (when (and (>= n 1) (> incx 0))
   (setq icamax 0)
   (setf smax (the single-float (cabs1 (svref cx 0))))
   (setf ix (the fixnum (+ ix incx)))
   (do ((i 1 (+ i 1)))
       ((or (>= ix limit) (>= i n)))
     (when (> (cabs1 (the (complex single-float) (svref cx ix)))
              smax)
      (setf icamax i)
      (setf smax (cabs1 (svref cx ix))))
     (setf ix (the fixnum (+ ix incx)))))
  icamax)))
```

———-

— BLAS 1 icamax lisp test —

```
(load "icamax.lisp")
(setq a
 (vector #C(1.0 2.0) #C(-2.0 2.0) #C(2.0 -1.0) #C(2.0 -3.0) #C(-1.0 -0.0)))
; #(#C(1.0 2.0) #C(-2.0 2.0) #C(2.0 -1.0) #C(2.0 -3.0) #C(-1.0 0.0))
(icamax 5 a 1)
; 3
(icamax 3 a 1)
; 1
(icamax 0 a 1)
; -1
(icamax -5 a 1)
; -1
(icamax 5 a -1)
; -1
(icamax 5 a 2)
; 0
(icamax 1 a 0)
; -1
(icamax 1 a -1)
; -1
(icamax 1 a 5)
; 0
(setq a
  (vector #C(1.0 2.0) #C(-2.0 2.0) #C(2.0 -1.0) #C(-2.0 -3.0) #C(-1.0 -0.0)))
; #(#C(1.0 2.0) #C(-2.0 2.0) #C(2.0 -1.0) #C(-2.0 -3.0) #C(-1.0 0.0))
(icamax 5 a 1)
; 3
```

———-

7.18 idamax - index of element having max. absolute value

7.18.1 Axiom unit tests

— idamax.input —

```
)set break resume
```

```
)sys rm -f idamax.output
)spool idamax.output
)set message test on
)set message auto off
)clear all

--S 1 of 11
a:PRIMARR(DFLOAT):=[[3.0, 4.0, -3.0, 5.0, -1.0]]
--R
--R   (1)  [3.,4.,- 3.,5.,- 1.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 1

--S 2 of 11
idamax(5,a,1)  -- should be 3
--R
--R   (2)  3
--R                                                     Type: PositiveInteger
--E 2

--S 3 of 11
idamax(3,a,1)  -- should be 1
--R
--R   (3)  1
--R                                                     Type: PositiveInteger
--E 3

--S 4 of 11
idamax(0,a,1)  -- should be -1
--R
--R   (4)  - 1
--R                                                            Type: Integer
--E 4

--S 5 of 11
idamax(-5,a,1) -- should be -1
--R
--R   (5)  - 1
--R                                                            Type: Integer
--E 5

--S 6 of 11
idamax(5,a,-1) -- should be -1
--R
--R   (6)  - 1
--R                                                            Type: Integer
--E 6

--S 7 of 11
idamax(5,a,2)  -- should be 0
```

```
--R
--R   (7)  0
--R                                                    Type: NonNegativeInteger
--E 7

--S 8 of 11
idamax(1,a,0)  -- should be -1
--R
--R   (8)  - 1
--R                                                          Type: Integer
--E 8

--S 9 of 11
idamax(1,a,-1) -- should be -1
--R
--R   (9)  - 1
--R                                                          Type: Integer
--E 9

--S 10 of 11
a:PRIMARR(DFLOAT):=[[3.0, 4.0, -3.0, -5.0, -1.0]]
--R
--R   (10)  [3.,4.,- 3.,- 5.,- 1.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 10

--S 11 of 11
idamax(5,a,1)  -- should be 3
--R
--R   (11)  3
--R                                                       Type: PositiveInteger
--E 11

)spool
)lisp (bye)
```

———-

7.18.2 Axiom help page

— idamax.help —

```
=====================================================================
idamax examples
=====================================================================

a:PRIMARR(DFLOAT):=[[3.0, 4.0, -3.0, 5.0, -1.0]]
```

```
  [3.,4.,- 3.,5.,- 1.]

Note that Axiom arrays are 0-based.

idamax(5,a,1)  -- should be 3

  3

idamax(3,a,1)  -- should be 1

  1

The count and increment must both be positive numbers

idamax(0,a,1)  -- should be -1

  - 1

idamax(-5,a,1) -- should be -1

  - 1

idamax(5,a,-1) -- should be -1

  - 1

idamax(5,a,2)  -- should be 0

  0

idamax(1,a,0)  -- should be -1

  - 1

idamax(1,a,-1) -- should be -1

  - 1

The comparison for maximum uses the absolute value

a:PRIMARR(DFLOAT):=[[3.0, 4.0, -3.0, -5.0, -1.0]]

  [3.,4.,- 3.,- 5.,- 1.]

idamax(5,a,1)  -- should be 3

  3

=====================================================================
Man Page Details
```

```
=====================================================================

NAME
       IDAMAX - BLAS level one, maximum index function

SYNOPSIS
       INTEGER FUNCTION IDAMAX ( n, x, incx )

           INTEGER               n, incx

           DOUBLE PRECISION      x


DESCRIPTION
       IDAMAX  searches  a double precision vector for the first occurrence of
       the the maximum absolute value. The vector x has length n and increment
       incx.

ARGUMENTS
       n       INTEGER. (input)
               Number of elements to process in the vector to be searched.  If
               n <= 0, these routines return 0.

       x       DOUBLE PRECISION. (input)
               Array of dimension (n-1) * |incx| + 1.
               Array x contains the vector to be searched.

       incx    INTEGER. (input)
               Increment between elements of x.

RETURN VALUES
       IDAMAX  INTEGER. (output)
               Return the first index of the maximum absolute value of  vector
               x.  The vector x has length n and increment incx.

NOTES
       When working backward (incx < 0), each routine starts at the end of the
       vector and moves backward, as follows:

            x(1-incx * (n-1)), x(1-incx * (n-2)), ..., x(1)

       The largest absolute value is:

            ABS (x(1+(index-1) * incx)) when incx > 0

            ABS (x(1+(n-index) * |incx|)) when incx < 0


            _______
```

7.18.3 fortran code

— idamax.f —

```
*  ====================================================================
      INTEGER FUNCTION IDAMAX(N,DX,INCX)
*
*  -- Reference BLAS level1 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER INCX,N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION DX(*)
*     ..
*
*  ====================================================================
*
*     .. Local Scalars ..
      DOUBLE PRECISION DMAX
      INTEGER I,IX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DABS
*     ..
      IDAMAX = 0
      IF (N.LT.1 .OR. INCX.LE.0) RETURN
      IDAMAX = 1
      IF (N.EQ.1) RETURN
      IF (INCX.EQ.1) THEN
*
*        code for increment equal to 1
*
         DMAX = DABS(DX(1))
         DO I = 2,N
            IF (DABS(DX(I)).GT.DMAX) THEN
               IDAMAX = I
               DMAX = DABS(DX(I))
            END IF
         END DO
      ELSE
*
*        code for increment not equal to 1
*
         IX = 1
         DMAX = DABS(DX(1))
```

```
         IX = IX + INCX
         DO I = 2,N
            IF (DABS(DX(IX)).GT.DMAX) THEN
               IDAMAX = I
               DMAX = DABS(DX(IX))
            END IF
            IX = IX + INCX
         END DO
      END IF
      RETURN
      END
      integer function idamax(n,dx,incx)
c
c     finds the index of element having max. absolute value.
c     jack dongarra, linpack, 3/11/78.
c     modified 3/93 to return if incx .le. 0.
c     modified 12/3/93, array(1) declarations changed to array(*)
c
      double precision dx(*),dmax
      integer i,incx,ix,n
c
      idamax = 0
      if( n.lt.1 .or. incx.le.0 ) return
      idamax = 1
      if(n.eq.1)return
      if(incx.eq.1)go to 20
c
c        code for increment not equal to 1
c
      ix = 1
      dmax = dabs(dx(1))
      ix = ix + incx
      do 10 i = 2,n
         if(dabs(dx(ix)).le.dmax) go to 5
         idamax = i
         dmax = dabs(dx(ix))
    5    ix = ix + incx
   10 continue
      return
c
c        code for increment equal to 1
c
   20 dmax = dabs(dx(1))
      do 30 i = 2,n
         if(dabs(dx(i)).le.dmax) go to 30
         idamax = i
         dmax = dabs(dx(i))
   30 continue
      return
      end
```

———-

7.18.4 fortran unit test

— idamaxEX example —

```
      program idamaxEX
*     Tim Daly May 16, 2012
*     unit tests for BLAS idamax
      double precision a(5)
      integer n
      a = (/ 3.0, 4.0, -3.0, 5.0, -1.0 /)
      write(6,100)a(1),a(2),a(3),a(4),a(5)
 100  format("a=(/ (/",f6.3," ",f6.3," ",f6.3," ",f6.3," ",f6.3," /)")
      n=idamax(5,a,1)
      write(6,200)n
 200  format("should be 4",/,"n=",i3)
      n=idamax(3,a,1)
      write(6,201)n
 201  format("should be 2",/,"n=",i3)
      n=idamax(0,a,1)
      write(6,202)n
 202  format("should be 0",/,"n=",i3)
      n=idamax(-5,a,1)
      write(6,203)n
 203  format("should be 0",/,"n=",i3)
      n=idamax(5,a,-1)
      write(6,204)n
 204  format("should be 0",/,"n=",i3)
      n=idamax(5,a,2)
      write(6,205)n
 205  format("should be 1",/,"n=",i3)
      n=idamax(1,a,0)
      write(6,206)n
 206  format("should be 0",/,"n=",i3)
      n=idamax(1,a,-1)
      write(6,207)n
 207  format("should be 0",/,"n=",i3)
      n=idamax(1,a,5)
      write(6,208)n
 208  format("should be 1",/,"n=",i3)
      a = (/ 3.0, 4.0, -3.0, -5.0, -1.0 /)
      write(6,100)a(1),a(2),a(3),a(4),a(5)
      n=idamax(5,a,1)
      write(6,209)n
 209  format("should be 4",/,"n=",i3)
```

```
      stop
      end
```

```
gcc -o idamaxEX idamaxEX.f -lgfortran idamax.o && ./idamaxEX
a=(/ (/ 3.000  4.000 -3.000  5.000 -1.000 /)
should be 4
n=  4
should be 2
n=  2
should be 0
n=  0
should be 0
n=  0
should be 0
n=  0
should be 1
n=  1
should be 0
n=  0
should be 0
n=  0
should be 1
n=  1
a=(/ (/ 3.000  4.000 -3.000 -5.000 -1.000 /)
should be 4
n=  4
```

7.18.5 lisp code

— BLAS 1 idamax —

```
(defun idamax (n dx incx)
 (declare (type (simple-array double-float (*)) dx)
          (type fixnum incx n))
 (let ((ix 0) (dmax 0.0) (idamax -1) (limit (length dx)))
  (declare (type (double-float) dmax) (type fixnum idamax ix limit))
  (when (and (>= n 1) (> incx 0))
   (setq idamax 0)
   (setf dmax (the double-float (abs (the double-float (svref dx 0)))))
   (setf ix (the fixnum (+ ix incx)))
   (do ((i 1 (+ i 1)))
       ((or (>= ix limit) (>= i n)))
    (when (> (the double-float (abs (the double-float (svref dx ix)))) dmax)
     (setf idamax i)
     (setf dmax (the double-float (abs (the double-float (svref dx ix))))))
```

```
    (setf ix (the fixnum (+ ix incx)))))
 idamax))
```

———-

— BLAS 1 idamax lisp test —

```
(load "idamax.lisp")
(setq a (vector 3.0d0 4.0d0 -3.0d0 5.0d0 -1.0d0))
; #(3.0 4.0 -3.0 5.0 -1.0)
(idamax 5 a 1)
; 3
(idamax 3 a 1)
; 1
(idamax 0 a 1)
; -1
(idamax -5 a 1)
; -1
(idamax 5 a -1)
; -1
(idamax 5 a 2)
; 0
(idamax 1 a 0)
; -1
(idamax 1 a -1)
; -1
(idamax 1 a 5)
; 0
(setq a (vector 3.0d0 4.0d0 -3.0d0 -5.0d0 -1.0d0))
; #(3.0 4.0 -3.0 -5.0 -1.0)
(idamax 5 a 1)
; 3
```

———-

7.19 isamax - index of element having max. absolute value

7.19.1 Axiom unit tests

— isamax.input —

```
)set break resume
)sys rm -f isamax.output
```

```
)spool isamax.output
)set message test on
)set message auto off
)clear all

--S 1 of 11
a:PRIMARR(FLOAT):=[[3.0, 4.0, -3.0, 5.0, -1.0]]
--R
--R   (1)  [3.,4.,- 3.,5.,- 1.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 1

--S 2 of 11
isamax(5,a,1)  -- should be 3
--R
--R   (2)  3
--R                                                     Type: PositiveInteger
--E 2

--S 3 of 11
isamax(3,a,1)  -- should be 1
--R
--R   (3)  1
--R                                                     Type: PositiveInteger
--E 3

--S 4 of 11
isamax(0,a,1)  -- should be -1
--R
--R   (4)  - 1
--R                                                             Type: Integer
--E 4

--S 5 of 11
isamax(-5,a,1) -- should be -1
--R
--R   (5)  - 1
--R                                                             Type: Integer
--E 5

--S 6 of 11
isamax(5,a,-1) -- should be -1
--R
--R   (6)  - 1
--R                                                             Type: Integer
--E 6

--S 7 of 11
isamax(5,a,2)  -- should be 0
--R
```

```
--R   (7)  0
--R                                                   Type: NonNegativeInteger
--E 7

--S 8 of 11
isamax(1,a,0)  -- should be -1
--R
--R   (8)  - 1
--R                                                            Type: Integer
--E 8

--S 9 of 11
isamax(1,a,-1) -- should be -1
--R
--R   (9)  - 1
--R                                                            Type: Integer
--E 9

--S 10 of 11
a:PRIMARR(FLOAT):=[[3.0, 4.0, -3.0, -5.0, -1.0]]
--R
--R   (10)  [3.,4.,- 3.,- 5.,- 1.]
--R                                          Type: PrimitiveArray(DoubleFloat)
--E 10

--S 11 of 11
isamax(5,a,1)  -- should be 3
--R
--R   (11)  3
--R                                                       Type: PositiveInteger
--E 11

)spool
)lisp (bye)
```

———-

7.19.2 Axiom help page

— isamax.help —

```
======================================================================
isamax examples
======================================================================

a:PRIMARR(FLOAT):=[[3.0, 4.0, -3.0, 5.0, -1.0]]

  [3.,4.,- 3.,5.,- 1.]
```

```
Note that Axiom arrays are 0-based.

isamax(5,a,1)  -- should be 3

  3

isamax(3,a,1)  -- should be 1

  1

The count and increment must both be positive numbers

isamax(0,a,1)  -- should be -1

  - 1

isamax(-5,a,1) -- should be -1

  - 1

isamax(5,a,-1) -- should be -1

  - 1

isamax(5,a,2)  -- should be 0

  0

isamax(1,a,0)  -- should be -1

  - 1

isamax(1,a,-1) -- should be -1

  - 1

The comparison for maximum uses the absolute value

a:PRIMARR(FLOAT):=[[3.0, 4.0, -3.0, -5.0, -1.0]]

  [3.,4.,- 3.,- 5.,- 1.]

isamax(5,a,1)  -- should be 3

  3

======================================================================
Man Page Details
======================================================================
```

```
NAME
       ISAMAX - BLAS level one, maximum index function

SYNOPSIS
       INTEGER FUNCTION ISAMAX ( n, x, incx )

           INTEGER               n, incx

           REAL                  x


DESCRIPTION
       ISAMAX searches a real vector for the first occurrence of the the maxi-
       mum absolute value. The vector x has length n and increment incx.

       ISAMAX returns the first index i such that
               |x | = MAX |x | : j = 1, ..., n
                 i          j
       where x  is an element of a real vector.
              j

ARGUMENTS
       n       INTEGER. (input)
               Number of elements to process in the vector to be searched.  If
               n <= 0, these routines return 0.

       x       REAL. (input)
               Array of dimension (n-1) * |incx| + 1.
               Array x contains the vector to be searched.

       incx    INTEGER. (input)
               Increment between elements of x.

RETURN VALUES
       ISAMAX  INTEGER. (output)
               Return  the first index of the maximum absolute value of vector
               x.  The vector x has length n and increment incx.

NOTES
       When working backward (incx < 0), each routine starts at the end of the
       vector and moves backward, as follows:

            x(1-incx * (n-1)), x(1-incx * (n-2)), ..., x(1)

       The largest absolute value is:

            ABS (x(1+(index-1) * incx)) when incx > 0

            ABS (x(1+(n-index) * |incx|)) when incx < 0
```

———-

7.19.3 fortran code

— **isamax.f** —

```
*  ====================================================================
      INTEGER FUNCTION ISAMAX(N,SX,INCX)
*
*  -- Reference BLAS level1 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER INCX,N
*     ..
*     .. Array Arguments ..
      REAL SX(*)
*     ..
*
*  ====================================================================
*
*     .. Local Scalars ..
      REAL SMAX
      INTEGER I,IX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC ABS
*     ..
      ISAMAX = 0
      IF (N.LT.1 .OR. INCX.LE.0) RETURN
      ISAMAX = 1
      IF (N.EQ.1) RETURN
      IF (INCX.EQ.1) THEN
*
*        code for increment equal to 1
*
         SMAX = ABS(SX(1))
         DO I = 2,N
            IF (ABS(SX(I)).GT.SMAX) THEN
               ISAMAX = I
               SMAX = ABS(SX(I))
            END IF
         END DO
      ELSE
```

```
*
*       code for increment not equal to 1
*
        IX = 1
        SMAX = ABS(SX(1))
        IX = IX + INCX
        DO I = 2,N
           IF (ABS(SX(IX)).GT.SMAX) THEN
              ISAMAX = I
              SMAX = ABS(SX(IX))
           END IF
           IX = IX + INCX
        END DO
     END IF
     RETURN
     END
```

7.19.4 fortran unit test

— isamaxEX example —

```
      program isamaxEX
*     Tim Daly May 19, 2012
*     unit tests for BLAS isamax
      real a(5)
      integer n
      a = (/ 3.0, 4.0, -3.0, 5.0, -1.0 /)
      write(6,100)a(1),a(2),a(3),a(4),a(5)
 100  format("a=(/ (/",f6.3," ",f6.3," ",f6.3," ",f6.3," ",f6.3," /)")
      n=isamax(5,a,1)
      write(6,200)n
 200  format("should be 4",/,"n=",i3)
      n=isamax(3,a,1)
      write(6,201)n
 201  format("should be 2",/,"n=",i3)
      n=isamax(0,a,1)
      write(6,202)n
 202  format("should be 0",/,"n=",i3)
      n=isamax(-5,a,1)
      write(6,203)n
 203  format("should be 0",/,"n=",i3)
      n=isamax(5,a,-1)
      write(6,204)n
 204  format("should be 0",/,"n=",i3)
      n=isamax(5,a,2)
```

```
      write(6,205)n
 205  format("should be 1",/,"n=",i3)
      n=isamax(1,a,0)
      write(6,206)n
 206  format("should be 0",/,"n=",i3)
      n=isamax(1,a,-1)
      write(6,207)n
 207  format("should be 0",/,"n=",i3)
      n=isamax(1,a,5)
      write(6,208)n
 208  format("should be 1",/,"n=",i3)
      a = (/ 3.0, 4.0, -3.0, -5.0, -1.0 /)
      write(6,100)a(1),a(2),a(3),a(4),a(5)
      n=isamax(5,a,1)
      write(6,209)n
 209  format("should be 4",/,"n=",i3)
      stop
      end
```

```
gcc -o isamaxEX isamaxEX.f -lgfortran isamax.o && ./isamaxEX
a=(/ (/ 3.000  4.000 -3.000  5.000 -1.000 /)
should be 4
n=  4
should be 2
n=  2
should be 0
n=  0
should be 0
n=  0
should be 0
n=  0
should be 1
n=  1
should be 0
n=  0
should be 0
n=  0
should be 1
n=  1
a=(/ (/ 3.000  4.000 -3.000 -5.000 -1.000 /)
should be 4
n=  4
```

Spad represents complex numbers as a pair where the car is the real part and the cons is the imaginary part. BLAS wants a complex number in fortran format. So we have a design choice to make. Either we could write all of the BLAS code using Spad internal representation or we could follow the BLAS code standard. I've decided to follow the BLAS code standard so we need to create thunks to do the data translation.

7.19.5 lisp code

— BLAS 1 isamax —

```
(defun isamaxSpad (n zx incx)
; Tim Daly May 19, 2012
 (let (result vec)
  (dotimes (i (length zx)) (push (float (car (svref zx i))) result))
  (setq vec (make-array (length result) :initial-contents (nreverse result)
                        :element-type 'single-float))
  (isamax n vec incx)))

(defun isamax (n dx incx)
; Tim Daly May 19, 2012
 (declare (type (simple-array single-float (*)) dx)
          (type fixnum incx n))
 (let ((ix 0) (dmax 0.0) (isamax -1) (limit (length dx)))
  (declare (type (single-float) dmax) (type fixnum isamax ix limit))
  (when (and (>= n 1) (> incx 0))
   (setq isamax 0)
   (setf dmax (the single-float (abs (the single-float (aref dx 0)))))
   (setf ix (the fixnum (+ ix incx)))
   (do ((i 1 (+ i 1)))
       ((or (>= ix limit) (>= i n)))
    (when (> (the single-float (abs (the single-float (aref dx ix)))) dmax)
     (setf isamax i)
     (setf dmax (the single-float (abs (the single-float (aref dx ix))))))
    (setf ix (the fixnum (+ ix incx)))))
 isamax))
```

———-

— BLAS 1 isamax lisp test —

```
(load "isamax.lisp")
(setq a (vector 3.0 4.0 -3.0 5.0 -1.0))
; #(3.0 4.0 -3.0 5.0 -1.0)
(isamax 5 a 1)
; 3
(isamax 3 a 1)
; 1
(isamax 0 a 1)
; -1
(isamax -5 a 1)
; -1
(isamax 5 a -1)
; -1
```

```
(isamax 5 a 2)
; 0
(isamax 1 a 0)
; -1
(isamax 1 a -1)
; -1
(isamax 1 a 5)
; 0
(setq a (vector 3.0 4.0 -3.0 -5.0 -1.0))
; #(3.0 4.0 -3.0 -5.0 -1.0)
(isamax 5 a 1)
; 3
```

———-

7.20 izamax

7.20.1 Axiom unit tests

— **izamax.input** —

```
)set break resume
)sys rm -f izamax.output
)spool izamax.output
)set message test on
)set message auto off
)clear all

--S 1 of 6
a:PRIMARR(COMPLEX(DFLOAT)):=[[3.+4.*%i,-4.+5.*%i,5.+6.*%i,7.-8.*%i,-9.-2.*%i]]
--R
--R   (1)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                   Type: PrimitiveArray(Complex(DoubleFloat))
--E 1

--S 2 of 6
izamax(5,a,1)  -- should be 3
--R
--R   (2)  3
--R                                                        Type: PositiveInteger
--E 2

--S 3 of 6
izamax(0,a,1)  -- should be -1
--R
--R   (3)  - 1
--R                                                                Type: Integer
```

```
--E 3

--S 4 of 6
izamax(5,a,-1) -- should be -1
--R
--R   (4)  - 1
--R                                                          Type: Integer
--E 4

--S 5 of 6
izamax(3,a,1)  -- should be 2
--R
--R   (5)  2
--R                                                   Type: PositiveInteger
--E 5

--S 6 of 6
izamax(3,a,2)  -- should be 1
--R
--R   (6)  1
--R                                                   Type: PositiveInteger
--E 6

)spool
)lisp (bye)
```

———-

7.20.2 Axiom help page

— izamax.help —

```
=====================================================================
izamax examples
=====================================================================

a:PRIMARR(COMPLEX(DFLOAT)):=[[3.+4.*%i,-4.+5.*%i,5.+6.*%i,7.-8.*%i,-9.-2.*%i]]

  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]

izamax(5,a,1)

  3

izamax(0,a,1)

  - 1
```

```
izamax(5,a,-1)

  - 1

izamax(3,a,1)

  2

izamax(3,a,2)

  1

=====================================================================
Man Page Details
=====================================================================

NAME
       IZAMAX - BLAS level one, maximum index function

SYNOPSIS
       INTEGER FUNCTION IZAMAX ( n, x, incx )

           INTEGER               n, incx

           DOUBLE COMPLEX        x


DESCRIPTION
       IZAMAX searches a double complex vector for the first occurrence of the
       maximum absolute value.

       IZAMAX determines the first index i such that

       |Real(x )|+ |Imag(x ) | = MAX(|Real(x )| + | Imag(x )|): j = 1, ..., n
              i           i                 j             j
               where x  is an element of a double complex vector.
                      j

ARGUMENTS
       n       INTEGER. (input)
               Number of elements to process in the vector to be searched.  If
               n <= 0, these routines return 0.

       x       DOUBLE COMPLEX. (input)
               Array of dimension (n-1) * |incx| + 1.
               Array x contains the vector to be searched.

       incx    INTEGER. (input)
               Increment between elements of x.
```

```
RETURN VALUES
       IZAMAX  INTEGER. (output)
               Return  the first index of the maximum absolute value of vector
               x.  The vector x has length n and increment incx.

NOTES
       When working backward (incx < 0), each routine starts at the end of the
       vector and moves backward, as follows:

            x(1-incx * (n-1)), x(1-incx * (n-2)), ..., x(1)

       The largest absolute value is:

            ABS (x(1+(index-1) * incx)) when incx > 0

            ABS (x(1+(n-index) * |incx|)) when incx < 0


            ———-
```

7.20.3 fortran code

— izamax.f —

```
*  ====================================================================
      INTEGER FUNCTION IZAMAX(N,ZX,INCX)
*
*  -- Reference BLAS level1 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER INCX,N
*     ..
*     .. Array Arguments ..
      COMPLEX*16 ZX(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      DOUBLE PRECISION DMAX
      INTEGER I,IX
*     ..
*     .. External Functions ..
      DOUBLE PRECISION DCABS1
      EXTERNAL DCABS1
```

```
*     ..
      IZAMAX = 0
      IF (N.LT.1 .OR. INCX.LE.0) RETURN
      IZAMAX = 1
      IF (N.EQ.1) RETURN
      IF (INCX.EQ.1) THEN
*
*        code for increment equal to 1
*
         DMAX = DCABS1(ZX(1))
         DO I = 2,N
            IF (DCABS1(ZX(I)).GT.DMAX) THEN
               IZAMAX = I
               DMAX = DCABS1(ZX(I))
            END IF
         END DO
      ELSE
*
*        code for increment not equal to 1
*
         IX = 1
         DMAX = DCABS1(ZX(1))
         IX = IX + INCX
         DO I = 2,N
            IF (DCABS1(ZX(IX)).GT.DMAX) THEN
               IZAMAX = I
               DMAX = DCABS1(ZX(IX))
            END IF
            IX = IX + INCX
         END DO
      END IF
      RETURN
      END
```

———-

7.20.4 fortran unit test

— izamaxEX example —

```
       program izamaxEX
*      Tim Daly May 20, 2012
*      unit tests for BLAS izamax
       double complex a(5)
       integer n
       a = (/ (1.0D0,2.0D0), (-3.0D0,4.0D0), (5.0D0,-6.0D0),
     C        (0.0D0,8.0D0), (0.0D0,-8.0D0) /)
```

```
      write(6,100)dble(a(1)),dimag(a(1))
 100  format("a(1)=C(",f6.3," ",f6.3,")")
      write(6,101)dble(a(2)),dimag(a(2))
 101  format("a(2)=C(",f6.3," ",f6.3,")")
      write(6,102)dble(a(3)),dimag(a(3))
 102  format("a(3)=C(",f6.3," ",f6.3,")")
      write(6,103)dble(a(4)),dimag(a(4))
 103  format("a(4)=C(",f6.3," ",f6.3,")")
      write(6,104)dble(a(5)),dimag(a(5))
 104  format("a(5)=C(",f6.3," ",f6.3,")")
      n=izamax(5,a,1)
      write(6,200)n
 200  format("should be 3",/,"n=",i3)
      n=izamax(3,a,1)
      write(6,201)n
 201  format("should be 3",/,"n=",i3)
      n=izamax(0,a,1)
      write(6,202)n
 202  format("should be 0",/,"n=",i3)
      n=izamax(-5,a,1)
      write(6,203)n
 203  format("should be 0",/,"n=",i3)
      n=izamax(5,a,-1)
      write(6,204)n
 204  format("should be 0",/,"n=",i3)
      n=izamax(5,a,2)
      write(6,205)n
 205  format("should be 2",/,"n=",i3)
      n=izamax(1,a,0)
      write(6,206)n
 206  format("should be 0",/,"n=",i3)
      n=izamax(1,a,-1)
      write(6,207)n
 207  format("should be 0",/,"n=",i3)
      n=izamax(1,a,5)
      write(6,208)n
 208  format("should be 1",/,"n=",i3)
      stop
      end
```

———-

```
gcc -o izamaxEX izamaxEX.f -lgfortran izamax.o dcabs1.o && ./izamaxEX
a(1)=C( 1.000  2.000)
a(2)=C(-3.000  4.000)
a(3)=C( 5.000 -6.000)
a(4)=C( 0.000  8.000)
a(5)=C( 0.000 -8.000)
should be 3
n=  3
```

```
should be 3
n=  3
should be 0
n=  0
should be 0
n=  0
should be 0
n=  0
should be 2
n=  2
should be 0
n=  0
should be 0
n=  0
should be 1
n=  1
```

Spad represents complex numbers as a pair where the car is the real part and the cons is the imaginary part. BLAS wants a complex number in fortran format. So we have a design choice to make. Either we could write all of the BLAS code using Spad internal representation or we could follow the BLAS code standard. I've decided to follow the BLAS code standard so we need to create thunks to do the data translation.

NOTE: The Axiom internal representation of PRIMARR(COMPLEX(DFLOAT)) is an array where each array element AE is

```
AE = (cons (therealpart 0)) (cons (theimagpart 0))
```

so we need (car AE) and (cdr AE) to get the real and imag parts.

NOTE: Array indexing in Axiom is 0-based but Fortran is 1-based so the results are displaced by 1.

7.20.5 lisp code

— BLAS 1 izamax —

```
(defun izamaxSpad (n zx incx)
; Tim Daly May 20, 2012
 (let (result vec)
  (dotimes (i (length zx))
   (push (complex (car (svref zx i)) (cdr (svref zx i))) result))
  (setq vec (make-array (length result) :initial-contents (nreverse result)))
  (izamax n vec incx)))

(defun izamax (n zx incx)
; Tim Daly May 20, 2012
 (declare (type (simple-array (complex double-float) (*)) zx)
```

```
         (type fixnum incx n))
 (let ((ix 0) (zmax 0.0d0) (izamax -1) (limit (length zx)))
  (declare (type (double-float) zmax) (type fixnum izamax ix limit))
  (when (and (>= n 1) (> incx 0))
   (setf izamax 0)
   (setf zmax (the double-float
                (dcabs1 (the (complex double-float) (svref zx 0)))))
   (setf ix (+ ix incx))
   (do ((i 1 (+ i 1)))
       ((or (>= ix limit) (>= i n)))
    (when (> (the double-float
                (dcabs1 (the (complex double-float) (svref zx ix))))
             zmax)
      (setf izamax i)
      (setf zmax (the double-float
                   (dcabs1 (the (complex double-float) (svref zx ix))))))
    (setf ix (the fixnum (+ ix incx)))))
 izamax))
```

———-

— BLAS 1 izamax lisp test —

```
(load "dcabs1.lisp")
(load "izamax.lisp")
(setq a (vector #C(1.0d0 2.0d0) #C(-3.0d0 4.0d0) #C(5.0d0 -6.0d0)
                #C(0.0d0 8.0d0) #C(0.0d0 -8.0d0)))
; #(#C(1.0 2.0) #C(-3.0 4.0) #C(5.0 -6.0) #C(0.0 8.0) #C(0.0 -8.0))
(izamax 5 a 1)
; 2
(izamax 3 a 1)
; 2
(izamax 0 a 1)
; -1
(izamax -5 a 1)
; -1
(izamax 5 a -1)
; -1
(izamax 5 a 2)
; 1
(izamax 1 a 0)
; -1
(izamax 1 a -1)
; -1
(izamax 1 a 5)
; 0
```

Chapter 8

BLAS Level 1 Complex Vector Operations

8.1 caxpy - constant times a vector plus a vector

8.1.1 Axiom unit tests

— **caxpy.input** —

```
)set break resume
)sys rm -f caxpy.output
)spool caxpy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

8.1.2 Axiom help page

— **caxpy.help** —

```
======================================================================
caxpy examples
======================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine caxpy (N, CA, CX, INCX, CY, INCY)
           CAXPY

Function/Subroutine Documentation
   subroutine caxpy (integer N, complex CA, complex, dimension(*) CX, integer
       INCX, complex, dimension(*) CY, integer INCY)
       CAXPY

 Purpose:
   CAXPY constant times a vector plus a vector.
```

8.1.3 fortran code

— caxpy.f —

```
*  ====================================================================
      SUBROUTINE CAXPY(N,CA,CX,INCX,CY,INCY)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX CA
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      COMPLEX CX(*),CY(*)
*     ..
*
*  ====================================================================
*
*     .. Local Scalars ..
      INTEGER I,IX,IY
*     ..
*     .. External Functions ..
      REAL SCABS1
```

```
      EXTERNAL SCABS1
*     ..
      IF (N.LE.0) RETURN
      IF (SCABS1(CA).EQ.0.0E+0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*        code for both increments equal to 1
*
         DO I = 1,N
            CY(I) = CY(I) + CA*CX(I)
         END DO
      ELSE
*
*        code for unequal increments or equal increments
*          not equal to 1
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
         IF (INCY.LT.0) IY = (-N+1)*INCY + 1
         DO I = 1,N
            CY(IY) = CY(IY) + CA*CX(IX)
            IX = IX + INCX
            IY = IY + INCY
         END DO
      END IF
*
      RETURN
      END
```

———-

8.1.4 lisp code

— BLAS 1 caxpy —

```
(defun caxpy (n ca cx incx cy incy)
  (declare (type (array f2cl-lib:complex8 (*)) cy cx)
           (type (f2cl-lib:complex8) ca)
           (type (f2cl-lib:integer4) incy incx n))
  (f2cl-lib:with-multi-array-data
      ((cx f2cl-lib:complex8 cx-%data% cx-%offset%)
       (cy f2cl-lib:complex8 cy-%data% cy-%offset%))
    (prog ((i 0) (ix 0) (iy 0))
      (declare (type (f2cl-lib:integer4) iy ix i))
      (if (<= n 0)
          (go end_label))
```

```
 (if (=
      (multiple-value-bind (ret-val var-0)
          (scabs1 ca)
        (declare (ignore))
        (when var-0 (setf ca var-0))
        ret-val)
      0.0)
     (go end_label))
 (cond
  ((and (= incx 1) (= incy 1))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf (f2cl-lib:fref cy-%data% (i) ((1 *)) cy-%offset%)
               (+ (f2cl-lib:fref cy-%data% (i) ((1 *)) cy-%offset%)
                  (* ca
                     (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%))))
      label100000)))
  (t (setf ix 1) (setf iy 1)
   (if (< incx 0)
       (setf ix
               (f2cl-lib:int-add
                (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incx) 1)))
   (if (< incy 0)
       (setf iy
               (f2cl-lib:int-add
                (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incy) 1)))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf (f2cl-lib:fref cy-%data% (iy) ((1 *)) cy-%offset%)
               (+ (f2cl-lib:fref cy-%data% (iy) ((1 *)) cy-%offset%)
                  (* ca
                     (f2cl-lib:fref cx-%data% (ix) ((1 *)) cx-%offset%))))
       (setf ix (f2cl-lib:int-add ix incx))
       (setf iy (f2cl-lib:int-add iy incy))
      label100001))))
 (go end_label)
end_label
 (return (values nil ca nil nil nil nil)))))
```

———-

8.2 ccopy - copies a vector x to a vector y

8.2.1 Axiom unit tests

— **ccopy.input** —

```
)set break resume
)sys rm -f ccopy.output
)spool ccopy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

8.2.2 Axiom help page

— **ccopy.help** —

```
======================================================================
ccopy examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ccopy (N, CX, INCX, CY, INCY)
           CCOPY

Function/Subroutine Documentation
   subroutine ccopy (integer N, complex, dimension(*) CX, integer INCX,
       complex, dimension(*) CY, integer INCY)
       CCOPY

Purpose:
  CCOPY copies a vector x to a vector y.
```

———-

8.2.3 fortran code

— ccopy.f —

```
*  ====================================================================
      SUBROUTINE CCOPY(N,CX,INCX,CY,INCY)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      COMPLEX CX(*),CY(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER I,IX,IY
*     ..
      IF (N.LE.0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*        code for both increments equal to 1
*
         DO I = 1,N
            CY(I) = CX(I)
         END DO
      ELSE
*
*        code for unequal increments or equal increments
*          not equal to 1
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
         IF (INCY.LT.0) IY = (-N+1)*INCY + 1
         DO I = 1,N
            CY(IY) = CX(IX)
            IX = IX + INCX
            IY = IY + INCY
         END DO
      END IF
      RETURN
      END
```

———-

8.2.4 lisp code

— BLAS 1 ccopy —

```
(defun ccopy (n cx incx cy incy)
  (declare (type (array f2cl-lib:complex8 (*)) cy cx)
           (type (f2cl-lib:integer4) incy incx n))
  (f2cl-lib:with-multi-array-data
      ((cx f2cl-lib:complex8 cx-%data% cx-%offset%)
       (cy f2cl-lib:complex8 cy-%data% cy-%offset%))
    (prog ((i 0) (ix 0) (iy 0))
      (declare (type (f2cl-lib:integer4) iy ix i))
      (if (<= n 0)
          (go end_label))
      (cond
       ((and (= incx 1) (= incy 1))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref cy-%data% (i) ((1 *)) cy-%offset%)
                    (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%))
           label100000)))
       (t (setf ix 1) (setf iy 1)
        (if (< incx 0)
            (setf ix
                    (f2cl-lib:int-add
                     (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incx) 1)))
        (if (< incy 0)
            (setf iy
                    (f2cl-lib:int-add
                     (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incy) 1)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref cy-%data% (iy) ((1 *)) cy-%offset%)
                    (f2cl-lib:fref cx-%data% (ix) ((1 *)) cx-%offset%))
            (setf ix (f2cl-lib:int-add ix incx))
            (setf iy (f2cl-lib:int-add iy incy))
           label100001))))
      (go end_label)
     end_label
      (return (values nil nil nil nil nil)))))
```

———-

8.3 cdotc - dot product of two complex vectors $X^H Y$

8.3.1 Axiom unit tests

— **cdotc.input** —

```
)set break resume
)sys rm -f cdotc.output
)spool cdotc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

8.3.2 Axiom help page

— **cdotc.help** —

```
======================================================================
cdotc examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       complex function cdotc (N, CX, INCX, CY, INCY)
           CDOTC

Function/Subroutine Documentation
   complex function cdotc (integer N, complex, dimension(*) CX, integer INCX,
       complex, dimension(*) CY, integer INCY)
       CDOTC

Purpose:
  CDOTC forms the dot product of two complex vectors CDOTC = X^H * Y
```

———-

8.3.3 fortran code

— cdotc.f —

```
*  ====================================================================
      COMPLEX FUNCTION CDOTC(N,CX,INCX,CY,INCY)
*
*  -- Reference BLAS level1 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      COMPLEX CX(*),CY(*)
*     ..
*
*  ====================================================================
*
*     .. Local Scalars ..
      COMPLEX CTEMP
      INTEGER I,IX,IY
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG
*     ..
      CTEMP = (0.0,0.0)
      CDOTC = (0.0,0.0)
      IF (N.LE.0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*        code for both increments equal to 1
*
         DO I = 1,N
            CTEMP = CTEMP + CONJG(CX(I))*CY(I)
         END DO
      ELSE
*
*        code for unequal increments or equal increments
*          not equal to 1
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
```

```
      IF (INCY.LT.0) IY = (-N+1)*INCY + 1
      DO I = 1,N
         CTEMP = CTEMP + CONJG(CX(IX))*CY(IY)
         IX = IX + INCX
         IY = IY + INCY
      END DO
   END IF
   CDOTC = CTEMP
   RETURN
   END
```

———-

8.3.4 lisp code

— BLAS 1 cdotc —

```
(defun cdotc (n cx incx cy incy)
  (declare (type (array f2cl-lib:complex8 (*)) cy cx)
           (type (f2cl-lib:integer4) incy incx n))
  (f2cl-lib:with-multi-array-data
      ((cx f2cl-lib:complex8 cx-%data% cx-%offset%)
       (cy f2cl-lib:complex8 cy-%data% cy-%offset%))
    (prog ((i 0) (ix 0) (iy 0) (ctemp #C(0.0 0.0)) (cdotc #C(0.0 0.0)))
      (declare (type (complex) cdotc)
               (type (f2cl-lib:complex8) ctemp)
               (type (f2cl-lib:integer4) iy ix i))
      (setf ctemp (f2cl-lib:cmplx 0.0 0.0))
      (setf cdotc (f2cl-lib:cmplx 0.0 0.0))
      (if (<= n 0)
          (go end_label))
      (cond
       ((and (= incx 1) (= incy 1))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf ctemp
                    (+ ctemp
                       (*
                        (f2cl-lib:conjg
                         (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%))
                        (f2cl-lib:fref cy-%data% (i) ((1 *)) cy-%offset%))))
           label100000)))
       (t (setf ix 1) (setf iy 1)
        (if (< incx 0)
            (setf ix
                    (f2cl-lib:int-add
```

```
                 (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incx) 1)))
      (if (< incy 0)
          (setf iy
                  (f2cl-lib:int-add
                   (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incy) 1)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf ctemp
                  (+ ctemp
                     (*
                      (f2cl-lib:conjg
                       (f2cl-lib:fref cx-%data% (ix) ((1 *)) cx-%offset%))
                      (f2cl-lib:fref cy-%data% (iy) ((1 *)) cy-%offset%))))
          (setf ix (f2cl-lib:int-add ix incx))
          (setf iy (f2cl-lib:int-add iy incy))
         label100001))))
    (setf cdotc ctemp)
    (go end_label)
   end_label
    (return (values cdotc nil nil nil nil nil)))))
```

———-

8.4 cdotu - dot product of two complex vectors X^TY

8.4.1 Axiom unit tests

— **cdotu.input** —

```
)set break resume
)sys rm -f cdotu.output
)spool cdotu.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

8.4.2 Axiom help page

— cdotu.help —

```
=====================================================================
cdotu examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       complex function cdotu (N, CX, INCX, CY, INCY)
           CDOTU

Function/Subroutine Documentation
   complex function cdotu (integer N, complex, dimension(*) CX, integer INCX,
       complex, dimension(*) CY, integer INCY)
       CDOTU

Purpose:
  CDOTU forms the dot product of two complex vectors CDOTU = X^T * Y
```

———-

8.4.3 fortran code

— cdotu.f —

```
*  ======================================================================
      COMPLEX FUNCTION CDOTU(N,CX,INCX,CY,INCY)
*
*  -- Reference BLAS level1 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      COMPLEX CX(*),CY(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
```

```
      COMPLEX CTEMP
      INTEGER I,IX,IY
*     ..
      CTEMP = (0.0,0.0)
      CDOTU = (0.0,0.0)
      IF (N.LE.0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*        code for both increments equal to 1
*
         DO I = 1,N
            CTEMP = CTEMP + CX(I)*CY(I)
         END DO
      ELSE
*
*        code for unequal increments or equal increments
*          not equal to 1
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
         IF (INCY.LT.0) IY = (-N+1)*INCY + 1
         DO I = 1,N
            CTEMP = CTEMP + CX(IX)*CY(IY)
            IX = IX + INCX
            IY = IY + INCY
         END DO
      END IF
      CDOTU = CTEMP
      RETURN
      END
```

———-

8.4.4 lisp code

— BLAS 1 cdotu —

```
(defun cdotu (n cx incx cy incy)
  (declare (type (array f2cl-lib:complex8 (*)) cy cx)
           (type (f2cl-lib:integer4) incy incx n))
  (f2cl-lib:with-multi-array-data
      ((cx f2cl-lib:complex8 cx-%data% cx-%offset%)
       (cy f2cl-lib:complex8 cy-%data% cy-%offset%))
    (prog ((i 0) (ix 0) (iy 0) (ctemp #C(0.0 0.0)) (cdotu #C(0.0 0.0)))
      (declare (type (complex) cdotu)
               (type (f2cl-lib:complex8) ctemp)
```

```
          (type (f2cl-lib:integer4) iy ix i))
  (setf ctemp (f2cl-lib:cmplx 0.0 0.0))
  (setf cdotu (f2cl-lib:cmplx 0.0 0.0))
  (if (<= n 0)
      (go end_label))
  (cond
   ((and (= incx 1) (= incy 1))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf ctemp
                (+ ctemp
                   (* (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%)
                      (f2cl-lib:fref cy-%data% (i) ((1 *)) cy-%offset%))))
       label100000)))
   (t (setf ix 1) (setf iy 1)
    (if (< incx 0)
        (setf ix
                (f2cl-lib:int-add
                 (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incx) 1)))
    (if (< incy 0)
        (setf iy
                (f2cl-lib:int-add
                 (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incy) 1)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf ctemp
                (+ ctemp
                   (* (f2cl-lib:fref cx-%data% (ix) ((1 *)) cx-%offset%)
                      (f2cl-lib:fref cy-%data% (iy) ((1 *)) cy-%offset%))))
        (setf ix (f2cl-lib:int-add ix incx))
        (setf iy (f2cl-lib:int-add iy incy))
       label100001))))
  (setf cdotu ctemp)
  (go end_label)
 end_label
  (return (values cdotu nil nil nil nil nil)))))
```

———-

8.5 crotg - determines a complex Givens rotation

8.5.1 Axiom unit tests

— **crotg.input** —

```
)set break resume
)sys rm -f crotg.output
)spool crotg.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

8.5.2 Axiom help page

— **crotg.help** —

```
==================================================================
crotg examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine crotg (CA, CB, C, S)
           CROTG

Function/Subroutine Documentation
   subroutine crotg (complex CA, complex CB, real C, complex S)
       CROTG

Purpose:
  CROTG determines a complex Givens rotation.
```

———-

8.5.3 fortran code

— **crotg.f** —

```
*  ==================================================================
      SUBROUTINE CROTG(CA,CB,C,S)
```

```
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX CA,CB,S
      REAL C
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      COMPLEX ALPHA
      REAL NORM,SCALE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CABS,CONJG,SQRT
*     ..
      IF (CABS(CA).EQ.0.) THEN
         C = 0.
         S = (1.,0.)
         CA = CB
      ELSE
         SCALE = CABS(CA) + CABS(CB)
         NORM = SCALE*SQRT((CABS(CA/SCALE))**2+ (CABS(CB/SCALE))**2)
         ALPHA = CA/CABS(CA)
         C = CABS(CA)/NORM
         S = ALPHA*CONJG(CB)/NORM
         CA = ALPHA*NORM
      END IF
      RETURN
      END
```

———-

8.5.4 lisp code

— BLAS 1 crotg —

```
(defun crotg (ca cb c s)
  (declare (type (single-float) c)
           (type (f2cl-lib:complex8) s cb ca))
  (prog ((norm 0.0) (scale 0.0) (alpha #C(0.0 0.0)))
    (declare (type (f2cl-lib:complex8) alpha)
             (type (single-float) scale norm))
```

```
    (cond
     ((= (f2cl-lib:cabs ca) 0.0) (setf c 0.0) (setf s (f2cl-lib:cmplx 1.0 0.0))
      (setf ca cb))
     (t
      (setf scale
              (coerce (realpart (+ (f2cl-lib:cabs ca) (f2cl-lib:cabs cb)))
                      'single-float))
      (setf norm
              (coerce
               (realpart
                (* scale
                   (f2cl-lib:fsqrt
                    (+ (expt (f2cl-lib:cabs (/ ca scale)) 2)
                       (expt (f2cl-lib:cabs (/ cb scale)) 2)))))
               'single-float))
      (setf alpha (/ ca (f2cl-lib:cabs ca)))
      (setf c (coerce (realpart (/ (f2cl-lib:cabs ca) norm)) 'single-float))
      (setf s (/ (* alpha (f2cl-lib:conjg cb)) norm))
      (setf ca (* alpha norm))))
    (go end_label)
   end_label
    (return (values ca nil c s))))
```

———-

8.6 cscal - scales a vector by a constant

8.6.1 Axiom unit tests

— **cscal.input** —

```
)set break resume
)sys rm -f cscal.output
)spool cscal.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

8.6.2 Axiom help page

— cscal.help —

```
==================================================================
cscal examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cscal (N, CA, CX, INCX)
           CSCAL

Function/Subroutine Documentation
   subroutine cscal (integer N, complex CA, complex, dimension(*) CX, integer
       INCX)
       CSCAL

Purpose:
  CSCAL scales a vector by a constant.
```

———-

8.6.3 fortran code

— cscal.f —

```
*  ==================================================================
      SUBROUTINE CSCAL(N,CA,CX,INCX)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX CA
      INTEGER INCX,N
*     ..
*     .. Array Arguments ..
      COMPLEX CX(*)
```

```
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER I,NINCX
*     ..
      IF (N.LE.0 .OR. INCX.LE.0) RETURN
      IF (INCX.EQ.1) THEN
*
*        code for increment equal to 1
*
         DO I = 1,N
            CX(I) = CA*CX(I)
         END DO
      ELSE
*
*        code for increment not equal to 1
*
         NINCX = N*INCX
         DO I = 1,NINCX,INCX
            CX(I) = CA*CX(I)
         END DO
      END IF
      RETURN
      END
```

———-

8.6.4 lisp code

— BLAS 1 cscal —

```
(defun cscal (n ca cx incx)
  (declare (type (array f2cl-lib:complex8 (*)) cx)
           (type (f2cl-lib:complex8) ca)
           (type (f2cl-lib:integer4) incx n))
  (f2cl-lib:with-multi-array-data
      ((cx f2cl-lib:complex8 cx-%data% cx-%offset%))
    (prog ((i 0) (nincx 0))
      (declare (type (f2cl-lib:integer4) nincx i))
      (if (or (<= n 0) (<= incx 0))
          (go end_label))
      (cond
       ((= incx 1)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
```

```
        (tagbody
          (setf (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%)
                  (* ca (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%)))
         label100000)))
      (t (setf nincx (f2cl-lib:int-mul n incx))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i incx))
                     ((> i nincx) nil)
        (tagbody
          (setf (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%)
                  (* ca (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%)))
         label100001))))
     (go end_label)
    end_label
     (return (values nil nil nil nil)))))
```

———-

8.7 csrot - appies a plane rotation

8.7.1 Axiom unit tests

— **csrot.input** —

```
)set break resume
)sys rm -f csrot.output
)spool csrot.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

8.7.2 Axiom help page

— **csrot.help** —

```
========================================================================
csrot examples
========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csrot (N, CX, INCX, CY, INCY, C, S)
           CSROT

Function/Subroutine Documentation
   subroutine csrot (integer N, complex, dimension( * ) CX, integer INCX,
       complex, dimension( * ) CY, integer INCY, real C, real S)
       CSROT

Purpose:
 CSROT applies a plane rotation, where the cos and sin (c and s) are real
 and the vectors cx and cy are complex.

Parameters:
 N
   N is INTEGER
   On entry, N specifies the order of the vectors cx and cy.
   N must be at least zero.

 CX
   CX is COMPLEX array, dimension at least ( 1 + ( N - 1 )*abs( INCX ) ).
   Before entry, the incremented array CX must contain the n
   element vector cx. On exit, CX is overwritten by the updated vector cx.

 INCX
   INCX is INTEGER
   On entry, INCX specifies the increment for the elements of
   CX. INCX must not be zero.

 CY
   CY is COMPLEX array, dimension at least ( 1 + ( N - 1 )*abs( INCY ) ).
   Before entry, the incremented array CY must contain the n
   element vector cy. On exit, CY is overwritten by the updated vector cy.

 INCY
   INCY is INTEGER
   On entry, INCY specifies the increment for the elements of
   CY. INCY must not be zero.

 C
   C is REAL
   On entry, C specifies the cosine, cos.

 S
   S is REAL
```

```
On entry, S specifies the sine, sin.
```

———-

8.7.3 fortran code

— **csrot.f** —

```
*  ===================================================================
      SUBROUTINE CSROT( N, CX, INCX, CY, INCY, C, S )
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER           INCX, INCY, N
      REAL              C, S
*     ..
*     .. Array Arguments ..
      COMPLEX           CX( * ), CY( * )
*     ..
*
*  ===================================================================
*
*     .. Local Scalars ..
      INTEGER           I, IX, IY
      COMPLEX           CTEMP
*     ..
*     .. Executable Statements ..
*
      IF( N.LE.0 )
     $   RETURN
      IF( INCX.EQ.1 .AND. INCY.EQ.1 ) THEN
*
*        code for both increments equal to 1
*
         DO I = 1, N
            CTEMP = C*CX( I ) + S*CY( I )
            CY( I ) = C*CY( I ) - S*CX( I )
            CX( I ) = CTEMP
         END DO
      ELSE
*
*        code for unequal increments or equal increments not equal
*          to 1
```

```
*
         IX = 1
         IY = 1
         IF( INCX.LT.0 )
     $      IX = ( -N+1 )*INCX + 1
         IF( INCY.LT.0 )
     $      IY = ( -N+1 )*INCY + 1
         DO I = 1, N
            CTEMP = C*CX( IX ) + S*CY( IY )
            CY( IY ) = C*CY( IY ) - S*CX( IX )
            CX( IX ) = CTEMP
            IX = IX + INCX
            IY = IY + INCY
         END DO
      END IF
      RETURN
      END
```

———-

8.7.4 lisp code

— BLAS 1 csrot —

```
(defun csrot (n cx incx cy incy c s)
  (declare (type (single-float) s c)
           (type (array f2cl-lib:complex8 (*)) cy cx)
           (type (f2cl-lib:integer4) incy incx n))
  (f2cl-lib:with-multi-array-data
      ((cx f2cl-lib:complex8 cx-%data% cx-%offset%)
       (cy f2cl-lib:complex8 cy-%data% cy-%offset%))
    (prog ((ctemp #C(0.0 0.0)) (i 0) (ix 0) (iy 0))
      (declare (type (f2cl-lib:integer4) iy ix i)
               (type (f2cl-lib:complex8) ctemp))
      (if (<= n 0)
          (go end_label))
      (cond
       ((and (= incx 1) (= incy 1))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf ctemp
                    (+ (* c (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%))
                       (* s
                          (f2cl-lib:fref cy-%data% (i) ((1 *)) cy-%offset%))))
            (setf (f2cl-lib:fref cy-%data% (i) ((1 *)) cy-%offset%)
                    (- (* c (f2cl-lib:fref cy-%data% (i) ((1 *)) cy-%offset%))
```

```
                    (* s
                       (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%))))
          (setf (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%) ctemp)
         label100000)))
      (t (setf ix 1) (setf iy 1)
       (if (< incx 0)
           (setf ix
                   (f2cl-lib:int-add
                    (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incx) 1)))
       (if (< incy 0)
           (setf iy
                   (f2cl-lib:int-add
                    (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incy) 1)))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf ctemp
                   (+ (* c (f2cl-lib:fref cx-%data% (ix) ((1 *)) cx-%offset%))
                      (* s
                         (f2cl-lib:fref cy-%data% (iy) ((1 *)) cy-%offset%))))
           (setf (f2cl-lib:fref cy-%data% (iy) ((1 *)) cy-%offset%)
                   (- (* c (f2cl-lib:fref cy-%data% (iy) ((1 *)) cy-%offset%))
                      (* s
                         (f2cl-lib:fref cx-%data% (ix) ((1 *)) cx-%offset%))))
           (setf (f2cl-lib:fref cx-%data% (ix) ((1 *)) cx-%offset%) ctemp)
           (setf ix (f2cl-lib:int-add ix incx))
           (setf iy (f2cl-lib:int-add iy incy))
          label100001))))
     (go end_label)
    end_label
     (return (values nil nil nil nil nil nil nil)))))
```

———-

8.8 csscal - scales a complex vector by a real constant

8.8.1 Axiom unit tests

— **csscal.input** —

```
)set break resume
)sys rm -f csscal.output
)spool csscal.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

8.8.2 Axiom help page

— **csscal.help** —

```
=====================================================================
csscal examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csscal (N, SA, CX, INCX)
           CSSCAL

Function/Subroutine Documentation
   subroutine csscal (integer N, real SA, complex, dimension(*) CX, integer
       INCX)
       CSSCAL

Purpose:
 CSSCAL scales a complex vector by a real constant.
```

———-

8.8.3 fortran code

— **csscal.f** —

```
*  ====================================================================
      SUBROUTINE CSSCAL(N,SA,CX,INCX)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
```

```
*
*     .. Scalar Arguments ..
      REAL SA
      INTEGER INCX,N
*     ..
*     .. Array Arguments ..
      COMPLEX CX(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER I,NINCX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC AIMAG,CMPLX,REAL
*     ..
      IF (N.LE.0 .OR. INCX.LE.0) RETURN
      IF (INCX.EQ.1) THEN
*
*        code for increment equal to 1
*
         DO I = 1,N
            CX(I) = CMPLX(SA*REAL(CX(I)),SA*AIMAG(CX(I)))
         END DO
      ELSE
*
*        code for increment not equal to 1
*
         NINCX = N*INCX
         DO I = 1,NINCX,INCX
            CX(I) = CMPLX(SA*REAL(CX(I)),SA*AIMAG(CX(I)))
         END DO
      END IF
      RETURN
      END
```

———-

8.8.4 lisp code

— BLAS 1 csscal —

```
(defun csscal (n sa cx incx)
  (declare (type (array f2cl-lib:complex8 (*)) cx)
           (type (single-float) sa)
           (type (f2cl-lib:integer4) incx n))
```

```
  (f2cl-lib:with-multi-array-data
      ((cx f2cl-lib:complex8 cx-%data% cx-%offset%))
    (prog ((i 0) (nincx 0))
      (declare (type (f2cl-lib:integer4) nincx i))
      (if (or (<= n 0) (<= incx 0))
          (go end_label))
      (cond
       ((= incx 1)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%)
                    (f2cl-lib:cmplx
                     (* sa
                        (f2cl-lib:freal
                         (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%)))
                     (* sa
                        (f2cl-lib:aimag
                         (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%)))))
           label100000)))
       (t (setf nincx (f2cl-lib:int-mul n incx))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i incx))
                      ((> i nincx) nil)
          (tagbody
            (setf (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%)
                    (f2cl-lib:cmplx
                     (* sa
                        (f2cl-lib:freal
                         (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%)))
                     (* sa
                        (f2cl-lib:aimag
                         (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%)))))
           label100001))))
      (go end_label)
     end_label
      (return (values nil nil nil nil)))))
```

———-

8.9 cswap - interchanges two vectors

8.9.1 Axiom unit tests

— **cswap.input** —

```
)set break resume
```

```
)sys rm -f cswap.output
)spool cswap.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

8.9.2 Axiom help page

— **cswap.help** —

```
==================================================================
cswap examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cswap (N, CX, INCX, CY, INCY)
           CSWAP

Function/Subroutine Documentation
   subroutine cswap (integer N, complex, dimension(*) CX, integer INCX,
       complex, dimension(*) CY, integer INCY)
       CSWAP

Purpose:
 CSWAP interchanges two vectors.
```

———-

8.9.3 fortran code

— **cswap.f** —

```
*  ====================================================================
      SUBROUTINE CSWAP(N,CX,INCX,CY,INCY)
```

```
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      COMPLEX CX(*),CY(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      COMPLEX CTEMP
      INTEGER I,IX,IY
*     ..
      IF (N.LE.0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*       code for both increments equal to 1
         DO I = 1,N
            CTEMP = CX(I)
            CX(I) = CY(I)
            CY(I) = CTEMP
         END DO
      ELSE
*
*       code for unequal increments or equal increments not equal
*         to 1
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
         IF (INCY.LT.0) IY = (-N+1)*INCY + 1
         DO I = 1,N
            CTEMP = CX(IX)
            CX(IX) = CY(IY)
            CY(IY) = CTEMP
            IX = IX + INCX
            IY = IY + INCY
         END DO
      END IF
      RETURN
      END
```

———-

8.9.4 lisp code

— BLAS 1 cswap —

```
(defun cswap (n cx incx cy incy)
  (declare (type (array f2cl-lib:complex8 (*)) cy cx)
           (type (f2cl-lib:integer4) incy incx n))
  (f2cl-lib:with-multi-array-data
      ((cx f2cl-lib:complex8 cx-%data% cx-%offset%)
       (cy f2cl-lib:complex8 cy-%data% cy-%offset%))
    (prog ((i 0) (ix 0) (iy 0) (ctemp #C(0.0 0.0)))
      (declare (type (f2cl-lib:complex8) ctemp)
               (type (f2cl-lib:integer4) iy ix i))
      (if (<= n 0)
          (go end_label))
      (cond
       ((and (= incx 1) (= incy 1))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf ctemp (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%))
            (setf (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%)
                    (f2cl-lib:fref cy-%data% (i) ((1 *)) cy-%offset%))
            (setf (f2cl-lib:fref cy-%data% (i) ((1 *)) cy-%offset%) ctemp)
           label100000)))
       (t (setf ix 1) (setf iy 1)
        (if (< incx 0)
            (setf ix
                    (f2cl-lib:int-add
                     (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incx) 1)))
        (if (< incy 0)
            (setf iy
                    (f2cl-lib:int-add
                     (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incy) 1)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf ctemp (f2cl-lib:fref cx-%data% (ix) ((1 *)) cx-%offset%))
            (setf (f2cl-lib:fref cx-%data% (ix) ((1 *)) cx-%offset%)
                    (f2cl-lib:fref cy-%data% (iy) ((1 *)) cy-%offset%))
            (setf (f2cl-lib:fref cy-%data% (iy) ((1 *)) cy-%offset%) ctemp)
            (setf ix (f2cl-lib:int-add ix incx))
            (setf iy (f2cl-lib:int-add iy incy))
           label100001))))
      (go end_label)
     end_label
      (return (values nil nil nil nil nil)))))
```

———-

Chapter 9

BLAS Level 1 Complex16 Vector Operations

9.1 zaxpy - constant times a vector plus a vector

9.1.1 Axiom unit tests

— zaxpy.input —

```
)set break resume
)sys rm -f zaxpy.output
)spool zaxpy.output
)set message test on
)set message auto off
)clear all

--S 1 of 25
a:PRIMARR(COMPLEX(DFLOAT)):=_
     [[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (1)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                      Type: PrimitiveArray Complex DoubleFloat
--E 1

--S 2 of 25
b:PRIMARR(COMPLEX(DFLOAT)):=_
     [[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (2)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                      Type: PrimitiveArray Complex DoubleFloat
--E 2
```

```
--S 3 of 25
zaxpy(3,2.0,a,1,b,1)
--R
--R   (3)  [9. + 12. %i,- 12. + 15. %i,15. + 18. %i,7. - 8. %i,- 9. - 2. %i]
--R                                     Type: PrimitiveArray Complex DoubleFloat
--E 3

--S 4 of 25
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (4)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                     Type: PrimitiveArray Complex DoubleFloat
--E 4

--S 5 of 25
zaxpy(5,2.0,a,1,b,1)
--R
--R   (5)  [9. + 12. %i,- 12. + 15. %i,15. + 18. %i,21. - 24. %i,- 27. - 6. %i]
--R                                     Type: PrimitiveArray Complex DoubleFloat
--E 5

--S 6 of 25
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (6)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                     Type: PrimitiveArray Complex DoubleFloat
--E 6

--S 7 of 25
zaxpy(3,2.0,a,3,b,3)
--R
--R   (7)  [9. + 12. %i,- 4. + 5. %i,5. + 6. %i,21. - 24. %i,- 9. - 2. %i]
--R                                     Type: PrimitiveArray Complex DoubleFloat
--E 7

--S 8 of 25
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (8)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                     Type: PrimitiveArray Complex DoubleFloat
--E 8

--S 9 of 25
zaxpy(4,2.0,a,2,b,2)
--R
--R   (9)  [9. + 12. %i,- 4. + 5. %i,15. + 18. %i,7. - 8. %i,- 27. - 6. %i]
--R                                     Type: PrimitiveArray Complex DoubleFloat
--E 9
```

```
--S 10 of 25
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (10)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                    Type: PrimitiveArray Complex DoubleFloat
--E 10

--S 11 of 25
zaxpy(3,2.0,a,2,b,2)
--R
--R   (11)  [9. + 12. %i,- 4. + 5. %i,15. + 18. %i,7. - 8. %i,- 27. - 6. %i]
--R                                    Type: PrimitiveArray Complex DoubleFloat
--E 11

--S 12 of 25
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (12)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                    Type: PrimitiveArray Complex DoubleFloat
--E 12

--S 13 of 25
zaxpy(3,-2.0,a,1,b,2)
--R
--R   (13)  [- 3. - 4. %i,- 4. + 5. %i,13. - 4. %i,7. - 8. %i,- 19. - 14. %i]
--R                                    Type: PrimitiveArray Complex DoubleFloat
--E 13

--S 14 of 25
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (14)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                    Type: PrimitiveArray Complex DoubleFloat
--E 14

--S 15 of 25
zaxpy(3,2.0,a,1,b,2)
--R
--R   (15)  [9. + 12. %i,- 4. + 5. %i,- 3. + 16. %i,7. - 8. %i,1. + 10. %i]
--R                                    Type: PrimitiveArray Complex DoubleFloat
--E 15

--S 16 of 25
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (16)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                    Type: PrimitiveArray Complex DoubleFloat
--E 16

--S 17 of 25
```

```
zaxpy(-3,2.0,a,1,b,2)
--R
--R   (17)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                     Type: PrimitiveArray Complex DoubleFloat
--E 17

--S 18 of 25
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (18)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                     Type: PrimitiveArray Complex DoubleFloat
--E 18

--S 19 of 25
zaxpy(3,2.0,a,-1,b,2)
--R
--R   (19)  [13. + 16. %i,- 4. + 5. %i,- 3. + 16. %i,7. - 8. %i,- 3. + 6. %i]
--R                                     Type: PrimitiveArray Complex DoubleFloat
--E 19

--S 20 of 25
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (20)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                     Type: PrimitiveArray Complex DoubleFloat
--E 20

--S 21 of 25
zaxpy(3,2.0,a,1,b,-2)
--R
--R   (21)  [13. + 16. %i,- 4. + 5. %i,- 3. + 16. %i,7. - 8. %i,- 3. + 6. %i]
--R                                     Type: PrimitiveArray Complex DoubleFloat
--E 21

--S 22 of 25
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
--R
--R   (22)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                     Type: PrimitiveArray Complex DoubleFloat
--E 22

--S 23 of 25
zaxpy(3,2.0,a,-1,b,-2)
--R
--R   (23)  [9. + 12. %i,- 4. + 5. %i,- 3. + 16. %i,7. - 8. %i,1. + 10. %i]
--R                                     Type: PrimitiveArray Complex DoubleFloat
--E 23

--S 24 of 25
b:=[[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
```

```
--R
--R   (24)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                    Type: PrimitiveArray Complex DoubleFloat
--E 24

--S 25 of 25
zaxpy(3,0.0,a,1,b,1)
--R
--R   (25)  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]
--R                                    Type: PrimitiveArray Complex DoubleFloat
--E 25

)spool
)lisp (bye)
```

———-

9.1.2 Axiom help page

— **zaxpy.help** —

```
=====================================================================
zaxpy examples
=====================================================================

a:PRIMARR(COMPLEX(DFLOAT)):=_
     [[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]
  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]

b:PRIMARR(COMPLEX(DFLOAT)):=_
     [[3.0+4.0*%i, -4.0+5.0*%i, 5.0+6.0*%i, 7.0-8.0*%i, -9.0-2.0*%i]]

  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]

zaxpy(3,2.0,a,1,b,1)

  [9. + 12. %i,- 12. + 15. %i,15. + 18. %i,7. - 8. %i,- 9. - 2. %i]

NOTE: We reset b to the above value before every execution

zaxpy(5,2.0,a,1,b,1)

  [9. + 12. %i,- 12. + 15. %i,15. + 18. %i,21. - 24. %i,- 27. - 6. %i]

zaxpy(3,2.0,a,3,b,3)

  [9. + 12. %i,- 4. + 5. %i,5. + 6. %i,21. - 24. %i,- 9. - 2. %i]
```

```
zaxpy(4,2.0,a,2,b,2)

  [9. + 12. %i,- 4. + 5. %i,15. + 18. %i,7. - 8. %i,- 27. - 6. %i]

zaxpy(3,2.0,a,2,b,2)

  [9. + 12. %i,- 4. + 5. %i,15. + 18. %i,7. - 8. %i,- 27. - 6. %i]

zaxpy(3,-2.0,a,1,b,2)

  [- 3. - 4. %i,- 4. + 5. %i,13. - 4. %i,7. - 8. %i,- 19. - 14. %i]

zaxpy(3,2.0,a,1,b,2)

  [9. + 12. %i,- 4. + 5. %i,- 3. + 16. %i,7. - 8. %i,1. + 10. %i]

zaxpy(-3,2.0,a,1,b,2)

  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]

zaxpy(3,2.0,a,-1,b,2)

  [13. + 16. %i,- 4. + 5. %i,- 3. + 16. %i,7. - 8. %i,- 3. + 6. %i]

zaxpy(3,2.0,a,1,b,-2)

  [13. + 16. %i,- 4. + 5. %i,- 3. + 16. %i,7. - 8. %i,- 3. + 6. %i]

zaxpy(3,2.0,a,-1,b,-2)

  [9. + 12. %i,- 4. + 5. %i,- 3. + 16. %i,7. - 8. %i,1. + 10. %i]

zaxpy(3,0.0,a,1,b,1)

  [3. + 4. %i,- 4. + 5. %i,5. + 6. %i,7. - 8. %i,- 9. - 2. %i]

=====================================================================
Man Page Details
=====================================================================

NAME
       ZAXPY - BLAS level one axpy subroutine

SYNOPSIS
       SUBROUTINE ZAXPY    ( n, alpha, x, incx, y, incy )

           INTEGER           n, incx, incy

           DOUBLE COMPLEX    alpha, x, y
```

```
DESCRIPTION
       ZAXPY adds a scalar multiple of a double complex vector to another dou-
       ble complex vector.

       ZAXPY computes a constant alpha times a vector x plus a vector y.   The
       result overwrites the initial values of vector y.

       This routine performs the following vector operation:

            y  <--  alpha*x + y

       incx and incy specify the increment between two consecutive
       elements of respectively vector x and y.

ARGUMENTS
       n       INTEGER. (input)
               Number  of  elements in the vectors.  If n <= 0, these routines
               return without any computation.

       alpha   DOUBLE COMPLEX. (input)
               If alpha = 0 this routine returns without any computation.

       x       DOUBLE COMPLEX. (input)
               Array of dimension (n-1) * |incx| + 1.  Contains the vector  to
               be scaled before summation.

       incx    INTEGER. (input)
               Increment between elements of x.
               If incx = 0, the results will be unpredictable.

       y       DOUBLE COMPLEX. (input and output)
               array of dimension (n-1) * |incy| + 1.
               Before calling the routine, y contains the vector to be summed.
               After the routine ends, y contains the result of the summation.

       incy    INTEGER. (input)
               Increment between elements of y.
               If incy = 0, the results will be unpredictable.

NOTES
       This  routine  is  Level  1  Basic  Linear Algebra Subprograms (Level 1
       BLAS).

       When working backward (incx < 0 or incy < 0), each  routine  starts  at
       the end of the vector and moves backward, as follows:

            x(1-incx * (n-1)), x(1-incx * (n-2)), ..., x(1)

            y(1-incy * (n-1)), y(1-incy * (n-2)), ..., y(1)
```

```
RETURN VALUES
       When  n  <=  0, double complex alpha = 0 = 0.+0.i, this routine returns
       immediately with no change in its arguments.
```

———-

Computes (complex double-float) $y \leftarrow \alpha x + y$

Arguments are:

- n - fixnum
- da - (complex double-float)
- dx - array (complex double-float)
- incx - fixnum
- dy - array (complex double-float)
- incy - fixnum

9.1.3 fortran code

— **zaxpy.f** —

```
*  ====================================================================
      SUBROUTINE ZAXPY(N,ZA,ZX,INCX,ZY,INCY)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX*16 ZA
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      COMPLEX*16 ZX(*),ZY(*)
*     ..
*
*  ====================================================================
*
*     .. Local Scalars ..
      INTEGER I,IX,IY
*     ..
```

```
*     .. External Functions ..
      DOUBLE PRECISION DCABS1
      EXTERNAL DCABS1
*     ..
      IF (N.LE.0) RETURN
      IF (DCABS1(ZA).EQ.0.0d0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*        code for both increments equal to 1
*
         DO I = 1,N
            ZY(I) = ZY(I) + ZA*ZX(I)
         END DO
      ELSE
*
*        code for unequal increments or equal increments
*          not equal to 1
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
         IF (INCY.LT.0) IY = (-N+1)*INCY + 1
         DO I = 1,N
            ZY(IY) = ZY(IY) + ZA*ZX(IX)
            IX = IX + INCX
            IY = IY + INCY
         END DO
      END IF
*
      RETURN
      END
```

———-

9.1.4 fortran unit test

— zaxpyEX example —

```
       program zaxpyEX
*      Tim Daly May 20, 2012
*      unit tests for BLAS zaxpy (a*x+y)
       double complex a(5)
       double complex b(5)
       double complex c(5)
       a(1) = ( 3.0D0, 4.0D0)
       a(2) = (-4.0D0, 5.0D0)
       a(3) = ( 5.0D0, 6.0D0)
```

```
      a(4) = ( 7.0D0,-8.0D0)
      a(5) = (-9.0D0,-2.0D0)
      b(1) = ( 3.0D0, 4.0D0)
      b(2) = (-4.0D0, 5.0D0)
      b(3) = ( 5.0D0, 6.0D0)
      b(4) = ( 7.0D0,-8.0D0)
      b(5) = (-9.0D0,-2.0D0)
      write(6,100)a(1),a(2),a(3),a(4),a(5)
100   format("a(1)=(",f7.2,",",f7.2,")",/
    C        "a(2)=(",f7.2,",",f7.2,")",/
    C        "a(3)=(",f7.2,",",f7.2,")",/
    C        "a(4)=(",f7.2,",",f7.2,")",/
    C        "a(5)=(",f7.2,",",f7.2,")")
      write(6,101)b(1),b(2),b(3),b(4),b(5)
101   format(/,"b(1)=(",f7.2,",",f7.2,")",/
    C        "b(2)=(",f7.2,",",f7.2,")",/
    C        "b(3)=(",f7.2,",",f7.2,")",/
    C        "b(4)=(",f7.2,",",f7.2,")",/
    C        "b(5)=(",f7.2,",",f7.2,")")

      call zaxpy(3,2.0d0,a,1,b,1)
      write(6,200)
200   format(/,"t200 is (9.00,12.00), (-12.00,15.00), (15.00,18.00) ",/,
    C          "        (7.00,-8.00), (-9.00,-2.00)")
      write(6,101)b(1),b(2),b(3),b(4),b(5)

      b(1) = ( 3.0D0, 4.0D0)
      b(2) = (-4.0D0, 5.0D0)
      b(3) = ( 5.0D0, 6.0D0)
      b(4) = ( 7.0D0,-8.0D0)
      b(5) = (-9.0D0,-2.0D0)
      call zaxpy(5,2.0d0,a,1,b,1)
      write(6,300)
300   format(/,"t300 is (9.00,12.00), (-12.00,15.00), (15.00,18.00) ",/,
    C          "        (21.00,-24.00), (-27.00,-6.00)")
      write(6,101)b(1),b(2),b(3),b(4),b(5)

      b(1) = ( 3.0D0, 4.0D0)
      b(2) = (-4.0D0, 5.0D0)
      b(3) = ( 5.0D0, 6.0D0)
      b(4) = ( 7.0D0,-8.0D0)
      b(5) = (-9.0D0,-2.0D0)
      call zaxpy(3,2.0d0,a,3,b,3)
      write(6,301)
301   format(/,"t301 is (9.00,12.00), (-4.00,5.00), (5.00,6.00) ",/,
    C          "        (21.00,-24.00), (-9.00,-2.00)")
      write(6,101)b(1),b(2),b(3),b(4),b(5)

      b(1) = ( 3.0D0, 4.0D0)
      b(2) = (-4.0D0, 5.0D0)
```

```
      b(3) = ( 5.0D0, 6.0D0)
      b(4) = ( 7.0D0,-8.0D0)
      b(5) = (-9.0D0,-2.0D0)
      call zaxpy(4,2.0d0,a,2,b,2)
      write(6,302)
302   format(/,"t302 is (9.00,12.00), (-4.00,5.00), (15.00,18.00) ",/,
    C          "        (7.00,-8.00), ( -27.00,  -6.00)")
      write(6,101)b(1),b(2),b(3),b(4),b(5)

      b(1) = ( 3.0D0, 4.0D0)
      b(2) = (-4.0D0, 5.0D0)
      b(3) = ( 5.0D0, 6.0D0)
      b(4) = ( 7.0D0,-8.0D0)
      b(5) = (-9.0D0,-2.0D0)
      call zaxpy(3,2.0d0,a,2,b,2)
      write(6,303)
303   format(/,"t303 is (9.00,12.00), (-4.00,5.00), (15.00,18.00) ",/,
    C          "        (7.00,-8.00), ( -27.00,  -6.00)")
      write(6,101)b(1),b(2),b(3),b(4),b(5)

      b(1) = ( 3.0D0, 4.0D0)
      b(2) = (-4.0D0, 5.0D0)
      b(3) = ( 5.0D0, 6.0D0)
      b(4) = ( 7.0D0,-8.0D0)
      b(5) = (-9.0D0,-2.0D0)
      call zaxpy(3,-2.0d0,a,1,b,2)
      write(6,304)
304   format(/,"t304 is (-3.00,-4.00), (-4.00,5.00), (13.00,-4.00) ",/,
    C          "        (7.00,-8.00), (-19.00,-14.00)")
      write(6,101)b(1),b(2),b(3),b(4),b(5)

      b(1) = ( 3.0D0, 4.0D0)
      b(2) = (-4.0D0, 5.0D0)
      b(3) = ( 5.0D0, 6.0D0)
      b(4) = ( 7.0D0,-8.0D0)
      b(5) = (-9.0D0,-2.0D0)
      call zaxpy(3,2.0d0,a,1,b,2)
      write(6,305)
305   format(/,"t305 is (9.00,12.00), (-4.00,5.00), (-3.00,16.00) ",/,
    C          "        (7.00,-8.00), (1.00,10.00)")
      write(6,101)b(1),b(2),b(3),b(4),b(5)

      b(1) = ( 3.0D0, 4.0D0)
      b(2) = (-4.0D0, 5.0D0)
      b(3) = ( 5.0D0, 6.0D0)
      b(4) = ( 7.0D0,-8.0D0)
      b(5) = (-9.0D0,-2.0D0)
      call zaxpy(-3,2.0d0,a,1,b,2)
      write(6,306)
306   format(/,"t306 is (3.00,4.00), (-4.00,5.00), (5.00,6.00) ",/,
```

```
C          "        (7.00,-8.00), (-9.00,-2.00)")
      write(6,101)b(1),b(2),b(3),b(4),b(5)

      b(1) = ( 3.0D0, 4.0D0)
      b(2) = (-4.0D0, 5.0D0)
      b(3) = ( 5.0D0, 6.0D0)
      b(4) = ( 7.0D0,-8.0D0)
      b(5) = (-9.0D0,-2.0D0)
      call zaxpy(3,2.0d0,a,-1,b,2)
      write(6,307)
307   format(/,"t307 is (13.00,16.00), (-4.00,5.00), (-3.00,16.00) ",/,
C          "        (7.00,-8.00), (-3.00,6.00)")
      write(6,101)b(1),b(2),b(3),b(4),b(5)

      b(1) = ( 3.0D0, 4.0D0)
      b(2) = (-4.0D0, 5.0D0)
      b(3) = ( 5.0D0, 6.0D0)
      b(4) = ( 7.0D0,-8.0D0)
      b(5) = (-9.0D0,-2.0D0)
      call zaxpy(3,2.0d0,a,1,b,-2)
      write(6,308)
308   format(/,"t308 is (13.00,16.00), (-4.00,5.00), (-3.00,16.00) ",/,
C          "        (7.00,-8.00), (-3.00,6.00)")
      write(6,101)b(1),b(2),b(3),b(4),b(5)

      b(1) = ( 3.0D0, 4.0D0)
      b(2) = (-4.0D0, 5.0D0)
      b(3) = ( 5.0D0, 6.0D0)
      b(4) = ( 7.0D0,-8.0D0)
      b(5) = (-9.0D0,-2.0D0)
      call zaxpy(3,2.0d0,a,-1,b,-2)
      write(6,309)
309   format(/,"t309 is (9.00,12.00), (-4.00,5.00), (-3.00,16.00) ",/,
C          "        (7.00,-8.00), (1.00,10.00)")
      write(6,101)b(1),b(2),b(3),b(4),b(5)

      b(1) = ( 3.0D0, 4.0D0)
      b(2) = (-4.0D0, 5.0D0)
      b(3) = ( 5.0D0, 6.0D0)
      b(4) = ( 7.0D0,-8.0D0)
      b(5) = (-9.0D0,-2.0D0)
      call zaxpy(3,0.0d0,a,1,b,1)
      write(6,310)
310   format(/,"t310 is (3.00,4.00), (-4.00,5.00), (5.00,6.00) ",/,
C          "        (7.00,-8.00), (-9.00,-2.00)")
      write(6,101)b(1),b(2),b(3),b(4),b(5)

      stop
      end
```

```
gcc -o zaxpyEX zaxpyEX.f -lgfortran zaxpy.o dcabs1.o && ./zaxpyEX
a(1)=(   3.00,   4.00)
a(2)=(  -4.00,   5.00)
a(3)=(   5.00,   6.00)
a(4)=(   7.00,  -8.00)
a(5)=(  -9.00,  -2.00)

b(1)=(   3.00,   4.00)
b(2)=(  -4.00,   5.00)
b(3)=(   5.00,   6.00)
b(4)=(   7.00,  -8.00)
b(5)=(  -9.00,  -2.00)

t200 is (9.00,12.00), (-12.00,15.00), (15.00,18.00)
        (7.00,-8.00), (-9.00,-2.00)

b(1)=(   9.00,  12.00)
b(2)=( -12.00,  15.00)
b(3)=(  15.00,  18.00)
b(4)=(   7.00,  -8.00)
b(5)=(  -9.00,  -2.00)

t300 is (9.00,12.00), (-12.00,15.00), (15.00,18.00)
        (21.00,-24.00), (-27.00,-6.00)

b(1)=(   9.00,  12.00)
b(2)=( -12.00,  15.00)
b(3)=(  15.00,  18.00)
b(4)=(  21.00, -24.00)
b(5)=( -27.00,  -6.00)

t301 is (9.00,12.00), (-4.00,5.00), (5.00,6.00)
        (21.00,-24.00), (-9.00,-2.00)

b(1)=(   9.00,  12.00)
b(2)=(  -4.00,   5.00)
b(3)=(   5.00,   6.00)
b(4)=(  21.00, -24.00)
b(5)=(  -9.00,  -2.00)

t302 is (9.00,12.00), (-4.00,5.00), (15.00,18.00)
        (7.00,-8.00), ( -27.00,  -6.00)

b(1)=(   9.00,  12.00)
b(2)=(  -4.00,   5.00)
b(3)=(  15.00,  18.00)
b(4)=(   7.00,  -8.00)
b(5)=( -27.00,  -6.00)
```

```
t303 is (9.00,12.00), (-4.00,5.00), (15.00,18.00)
        (7.00,-8.00), ( -27.00,  -6.00)

b(1)=(   9.00,  12.00)
b(2)=(  -4.00,   5.00)
b(3)=(  15.00,  18.00)
b(4)=(   7.00,  -8.00)
b(5)=( -27.00,  -6.00)

t304 is (-3.00,-4.00), (-4.00,5.00), (13.00,-4.00)
        (7.00,-8.00), (-19.00,-14.00)

b(1)=(  -3.00,  -4.00)
b(2)=(  -4.00,   5.00)
b(3)=(  13.00,  -4.00)
b(4)=(   7.00,  -8.00)
b(5)=( -19.00, -14.00)

t305 is (9.00,12.00), (-4.00,5.00), (-3.00,16.00)
        (7.00,-8.00), (1.00,10.00)

b(1)=(   9.00,  12.00)
b(2)=(  -4.00,   5.00)
b(3)=(  -3.00,  16.00)
b(4)=(   7.00,  -8.00)
b(5)=(   1.00,  10.00)

t306 is (3.00,4.00), (-4.00,5.00), (5.00,6.00)
        (7.00,-8.00), (-9.00,-2.00)

b(1)=(   3.00,   4.00)
b(2)=(  -4.00,   5.00)
b(3)=(   5.00,   6.00)
b(4)=(   7.00,  -8.00)
b(5)=(  -9.00,  -2.00)

t307 is (13.00,16.00), (-4.00,5.00), (-3.00,16.00)
        (7.00,-8.00), (-3.00,6.00)

b(1)=(  13.00,  16.00)
b(2)=(  -4.00,   5.00)
b(3)=(  -3.00,  16.00)
b(4)=(   7.00,  -8.00)
b(5)=(  -3.00,   6.00)

t308 is (13.00,16.00), (-4.00,5.00), (-3.00,16.00)
        (7.00,-8.00), (-3.00,6.00)

b(1)=(  13.00,  16.00)
```

```
b(2)=(  -4.00,   5.00)
b(3)=(  -3.00,  16.00)
b(4)=(   7.00,  -8.00)
b(5)=(  -3.00,   6.00)

t309 is (9.00,12.00), (-4.00,5.00), (-3.00,16.00)
        (7.00,-8.00), (1.00,10.00)

b(1)=(   9.00,  12.00)
b(2)=(  -4.00,   5.00)
b(3)=(  -3.00,  16.00)
b(4)=(   7.00,  -8.00)
b(5)=(   1.00,  10.00)

t310 is (3.00,4.00), (-4.00,5.00), (5.00,6.00)
        (7.00,-8.00), (-9.00,-2.00)

b(1)=(   3.00,   4.00)
b(2)=(  -4.00,   5.00)
b(3)=(   5.00,   6.00)
b(4)=(   7.00,  -8.00)
b(5)=(  -9.00,  -2.00)
```

Axiom delivers PRIMARR(COMPLEX(DFLOAT)) as a list of pairs. We create a thunk function zaxpySpad which creates an array of complex double-float objects required by zaxpy.

The zaxpy function is defined to modify the second array. So the thunk function has to setf this information back into the original array.

The zaxpySpad function could be more efficient if it only copied elements that were modified. This was not done because of the overhead of figuring out which elements this could be under all possible arguments.

9.1.5 lisp code

— BLAS 1 zaxpy —

```
(defun zaxpySpad (n za zx incx zy incy)
; Tim Daly May 23, 2012
 (let (result vecx vecy tx ty)
  (dotimes (i (length zx))
   (push (complex (car (svref zx i)) (cdr (svref zx i))) tx))
  (setq vecx (make-array (length tx) :initial-contents (nreverse tx)))
  (dotimes (i (length zy))
   (push (complex (car (svref zy i)) (cdr (svref zy i))) ty))
  (setq vecy (make-array (length ty) :initial-contents (nreverse ty)))
  (zaxpy n (complex (car za) (cdr za)) vecx incx vecy incy)
  (dotimes (i (length vecx))
```

```
   (setf (svref zy i)
     (cons (realpart (svref vecy i)) (imagpart (svref vecy i)))))
 zy))

(defun zaxpy (n za zx incx zy incy)
; Tim Daly May 23, 2012
 (declare (type (simple-array (complex double-float) (*)) zy zx)
          (type (complex double-float) za)
          (type fixnum incy incx n))
 (let ((ix 0) (iy 0) (limitx (length zx)) (limity (length zy)))
  (declare (type fixnum iy ix limitx limity))
  (when (and (> n 0) (/= (dcabs1 za) 0.0))
   (when (< incx 0)
    (setf ix (the fixnum (* (the fixnum (1+ (the fixnum (- n)))) incx))))
   (when (< incy 0)
    (setf iy (the fixnum (* (the fixnum (1+ (the fixnum (- n)))) incy))))
   (do ((i 0 (1+ i)))
       ((or (>= i n) (< ix 0) (< iy 0) (>= ix limitx) (>= iy limity)))
    (setf (the (complex double-float) (svref zy iy))
      (+ (the (complex double-float) (svref zy iy))
         (the (complex double-float)
           (* za (the (complex double-figure) (svref zx ix))))))
    (setf ix (+ ix incx))
    (setf iy (+ iy incy))))))
```

———-

— BLAS 1 izamax lisp test —

```
(load "zaxpy.lisp")
(load "dcabs1.lisp")
(setq zx (vector (cons 3.0d0 4.0d0) (cons -4.0d0 5.0d0) (cons 5.0d0 6.0d0)
                 (cons 7.0d0 -8.0d0) (cons -9.0d0 -2.0d0)))
; #((3.0 . 4.0) (-4.0 . 5.0) (5.0 . 6.0) (7.0 . -8.0) (-9.0 . -2.0))
(setq zy (vector (cons 3.0d0 4.0d0) (cons -4.0d0 5.0d0) (cons 5.0d0 6.0d0)
                 (cons 7.0d0 -8.0d0) (cons -9.0d0 -2.0d0)))
; #((3.0 . 4.0) (-4.0 . 5.0) (5.0 . 6.0) (7.0 . -8.0) (-9.0 . -2.0))
(zaxpyspad 3 2.0d0 zx 1 zy 1) ; t200
; #((9.0 . 12.0) (-12.0 . 15.0) (15.0 . 18.0) (7.0 . -8.0) (-9.0 . -2.0))
(setq zy (vector (cons 3.0d0 4.0d0) (cons -4.0d0 5.0d0) (cons 5.0d0 6.0d0)
                 (cons 7.0d0 -8.0d0) (cons -9.0d0 -2.0d0)))
; #((3.0 . 4.0) (-4.0 . 5.0) (5.0 . 6.0) (7.0 . -8.0) (-9.0 . -2.0))
(zaxpyspad 5 2.0d0 zx 1 zy 1) ; t300
; #((9.0 . 12.0) (-12.0 . 15.0) (15.0 . 18.0) (21.0 . -24.0) (-27.0 . -6.0))
(setq zy (vector (cons 3.0d0 4.0d0) (cons -4.0d0 5.0d0) (cons 5.0d0 6.0d0)
                 (cons 7.0d0 -8.0d0) (cons -9.0d0 -2.0d0)))
; #((3.0 . 4.0) (-4.0 . 5.0) (5.0 . 6.0) (7.0 . -8.0) (-9.0 . -2.0))
(zaxpyspad 3 2.0d0 zx 3 zy 3) ; t301
```

```
; #((9.0 . 12.0) (-4.0 . 5.0) (5.0 . 6.0) (21.0 . -24.0) (-9.0 . -2.0))
(setq zy (vector (cons 3.0d0 4.0d0) (cons -4.0d0 5.0d0) (cons 5.0d0 6.0d0)
                 (cons 7.0d0 -8.0d0) (cons -9.0d0 -2.0d0)))
; #((3.0 . 4.0) (-4.0 . 5.0) (5.0 . 6.0) (7.0 . -8.0) (-9.0 . -2.0))
(zaxpyspad 4 2.0d0 zx 2 zy 2) ; t302
; #((9.0 . 12.0) (-4.0 . 5.0) (15.0 . 18.0) (7.0 . -8.0) (-27.0 . -6.0))
(setq zy (vector (cons 3.0d0 4.0d0) (cons -4.0d0 5.0d0) (cons 5.0d0 6.0d0)
                 (cons 7.0d0 -8.0d0) (cons -9.0d0 -2.0d0)))
; #((3.0 . 4.0) (-4.0 . 5.0) (5.0 . 6.0) (7.0 . -8.0) (-9.0 . -2.0))
(zaxpyspad 3 2.0d0 zx 2 zy 2) ; t303
; #((9.0 . 12.0) (-4.0 . 5.0) (15.0 . 18.0) (7.0 . -8.0) (-27.0 . -6.0))
(setq zy (vector (cons 3.0d0 4.0d0) (cons -4.0d0 5.0d0) (cons 5.0d0 6.0d0)
                 (cons 7.0d0 -8.0d0) (cons -9.0d0 -2.0d0)))
; #((3.0 . 4.0) (-4.0 . 5.0) (5.0 . 6.0) (7.0 . -8.0) (-9.0 . -2.0))
(zaxpyspad 3 -2.0d0 zx 1 zy 2) ; t304
; #((-3.0 . -4.0) (-4.0 . 5.0) (13.0 . -4.0) (7.0 . -8.0) (-19.0 . -14.0))
(setq zy (vector (cons 3.0d0 4.0d0) (cons -4.0d0 5.0d0) (cons 5.0d0 6.0d0)
                 (cons 7.0d0 -8.0d0) (cons -9.0d0 -2.0d0)))
; #((3.0 . 4.0) (-4.0 . 5.0) (5.0 . 6.0) (7.0 . -8.0) (-9.0 . -2.0))
(zaxpyspad 3 2.0d0 zx 1 zy 2) ; t305
; #((9.0 . 12.0) (-4.0 . 5.0) (-3.0 . 16.0) (7.0 . -8.0) (1.0 . 10.0))
(setq zy (vector (cons 3.0d0 4.0d0) (cons -4.0d0 5.0d0) (cons 5.0d0 6.0d0)
                 (cons 7.0d0 -8.0d0) (cons -9.0d0 -2.0d0)))
; #((3.0 . 4.0) (-4.0 . 5.0) (5.0 . 6.0) (7.0 . -8.0) (-9.0 . -2.0))
(zaxpyspad -3 2.0d0 zx 1 zy 2) ; t306
; #((3.0 . 4.0) (-4.0 . 5.0) (5.0 . 6.0) (7.0 . -8.0) (-9.0 . -2.0))
(setq zy (vector (cons 3.0d0 4.0d0) (cons -4.0d0 5.0d0) (cons 5.0d0 6.0d0)
                 (cons 7.0d0 -8.0d0) (cons -9.0d0 -2.0d0)))
; #((3.0 . 4.0) (-4.0 . 5.0) (5.0 . 6.0) (7.0 . -8.0) (-9.0 . -2.0))
(zaxpyspad 3 2.0d0 zx -1 zy 2) ; t307
; #((13.0 . 16.0) (-4.0 . 5.0) (-3.0 . 16.0) (7.0 . -8.0) (-3.0 . 6.0))
(setq zy (vector (cons 3.0d0 4.0d0) (cons -4.0d0 5.0d0) (cons 5.0d0 6.0d0)
                 (cons 7.0d0 -8.0d0) (cons -9.0d0 -2.0d0)))
; #((3.0 . 4.0) (-4.0 . 5.0) (5.0 . 6.0) (7.0 . -8.0) (-9.0 . -2.0))
(zaxpyspad 3 2.0d0 zx 1 zy -2) ; t308
; #((13.0 . 16.0) (-4.0 . 5.0) (-3.0 . 16.0) (7.0 . -8.0) (-3.0 . 6.0))
(setq zy (vector (cons 3.0d0 4.0d0) (cons -4.0d0 5.0d0) (cons 5.0d0 6.0d0)
                 (cons 7.0d0 -8.0d0) (cons -9.0d0 -2.0d0)))
; #((3.0 . 4.0) (-4.0 . 5.0) (5.0 . 6.0) (7.0 . -8.0) (-9.0 . -2.0))
(zaxpyspad 3 2.0d0 zx -1 zy -2) ; t309
; #((9.0 . 12.0) (-4.0 . 5.0) (-3.0 . 16.0) (7.0 . -8.0) (1.0 . 10.0))
(setq zy (vector (cons 3.0d0 4.0d0) (cons -4.0d0 5.0d0) (cons 5.0d0 6.0d0)
                 (cons 7.0d0 -8.0d0) (cons -9.0d0 -2.0d0)))
; #((3.0 . 4.0) (-4.0 . 5.0) (5.0 . 6.0) (7.0 . -8.0) (-9.0 . -2.0))
(zaxpyspad 3 0.0d0 zx 1 zy 1) ; t310
; #((3.0 . 4.0) (-4.0 . 5.0) (5.0 . 6.0) (7.0 . -8.0) (-9.0 . -2.0))
```

9.2 zcopy - copies a vector x to a vector y

9.2.1 Axiom unit tests

— **zcopy.input** —

```
)set break resume
)sys rm -f zcopy.output
)spool zcopy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

9.2.2 Axiom help page

— **zcopy.help** —

```
====================================================================
zcopy examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       ZCOPY  -  BLAS  level  one, copies a double complex vector into another
       double complex vector

SYNOPSIS
       SUBROUTINE ZCOPY    ( n, x, incx, y, incy )

           INTEGER           n, incx, incy

           DOUBLE COMPLEX    x, y


DESCRIPTION
       ZCOPY copies a double complex vector into another double  complex  vec-
       tor.   ZCOPY  copies  a vector x, whose length is n to a vector y. incx
       and incy specify the increment  between  two  consecutive  elements  of
```

```
       respectively vector x and y.

       This routine performs the following vector operation:

            y  <--  x

       where x and y are double complex vectors.

ARGUMENTS
       n       INTEGER. (input)
               Number of vector elements to be copied.
               If n <= 0, this routine returns without computation.

       x       DOUBLE COMPLEX, (input)
               Vector from which to copy.

       incx    INTEGER. (input)
               Increment between elements of x.
               If incx = 0, the results will be unpredictable.

       y       DOUBLE COMPLEX, (output)
               array of dimension (n-1) * |incy| + 1, result vector.

       incy    INTEGER. (input)
               Increment between elements of y.  If incy = 0, the results will
               be unpredictable.

NOTES
       When  working  backward  (incx < 0 or incy < 0), each routine starts at
       the end of the vector and moves backward, as follows:

            x(1-incx * (n-1)), x(1-incx * (n-2)), ..., x(1)

            y(1-incy * (n-1)), y(1-incy * (n-2)), ..., y(1)
```

———-

9.2.3 fortran code

— **zcopy.f** —

```
*  ====================================================================
      SUBROUTINE ZCOPY(N,ZX,INCX,ZY,INCY)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
```

```
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      COMPLEX*16 ZX(*),ZY(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER I,IX,IY
*     ..
      IF (N.LE.0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*        code for both increments equal to 1
*
         DO I = 1,N
          ZY(I) = ZX(I)
         END DO
      ELSE
*
*        code for unequal increments or equal increments
*          not equal to 1
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
         IF (INCY.LT.0) IY = (-N+1)*INCY + 1
         DO I = 1,N
            ZY(IY) = ZX(IX)
            IX = IX + INCX
            IY = IY + INCY
         END DO
      END IF
      RETURN
      END
```

———-

9.2.4 lisp code

— BLAS 1 zcopy —

```
(defun zcopy (n zx incx zy incy)
```

```
  (declare (type (simple-array (complex double-float) (*)) zy zx)
           (type fixnum incy incx n))
  (f2cl-lib:with-multi-array-data
      ((zx (complex double-float) zx-%data% zx-%offset%)
       (zy (complex double-float) zy-%data% zy-%offset%))
    (prog ((i 0) (ix 0) (iy 0))
      (declare (type fixnum iy ix i))
      (if (<= n 0) (go end_label))
      (if (and (= incx 1) (= incy 1)) (go label20))
      (setf ix 1)
      (setf iy 1)
      (if (< incx 0)
          (setf ix
                  (f2cl-lib:int-add
                   (f2cl-lib:int-mul (the fixnum (1- n)) incx)
                   1)))
      (if (< incy 0)
          (setf iy
                  (f2cl-lib:int-add
                   (f2cl-lib:int-mul (the fixnum (1- n)) incy)
                   1)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref zy-%data% (iy) ((1 *)) zy-%offset%)
                  (f2cl-lib:fref zx-%data% (ix) ((1 *)) zx-%offset%))
          (setf ix (f2cl-lib:int-add ix incx))
          (setf iy (f2cl-lib:int-add iy incy))))
      (go end_label)
 label20
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref zy-%data% (i) ((1 *)) zy-%offset%)
                  (f2cl-lib:fref zx-%data% (i) ((1 *)) zx-%offset%))))
 end_label
      (return (values nil nil nil nil nil)))))
```

———-

9.3 zdotc - dot product of two complex vectors X^HY

9.3.1 Axiom unit tests

— zdotc.input —

```
)set break resume
)sys rm -f zdotc.output
)spool zdotc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

9.3.2 Axiom help page

— zdotc.help —

```
==================================================================
zdotc examples
==================================================================

==================================================================
Man Page Details
==================================================================

NAME
       ZDOTC  - BLAS level one, computes the hermitian dot product of
       vector x and vector y.

SYNOPSIS
       DOUBLE COMPLEX FUNCTION ZDOTC ( n, x, incx, y, incy )

           INTEGER                     n, incx, incy

           COMPLEX*16                  x, y


DESCRIPTION
       ZDOTC computes a dot product of the conjugate of a complex  vector
       and another complex vector (complex inner product).

       ZDOTC computes a dot product of the conjugate of a complex vector
       and another complex vector (complex inner product).

       This routine performs the following vector operation:

            ZDOTC  <--  (conjugate transpose of x) * y
                            n
                        =  Sum  (complex conjugate of x(i))*y(i)
```

```
                        i=1
                                                H
       where x and y are complex vectors, and x is the conjugate
       transpose of x.

       If n <= 0, ZDOTC is set to 0.

ARGUMENTS
       n       INTEGER. (input)
               Number of elements in each vector.

       x       COMPLEX*16. (input)
               Array of dimension (n-1) * |incx| + 1.
               Array x contains the first vector operand.

       incx    INTEGER. (input)
               Increment between elements of x.
               If incx = 0, the results will be unpredictable.

       y       COMPLEX*16. (input)
               Array of dimension (n-1) * |incy| + 1.
               Array y contains the second vector operand.

       incy    INTEGER. (input)
               Increment between elements of y.
               If incy = 0, the results will be unpredictable.

RETURN VALUES
       ZDOTC    DOUBLE COMPLEX. Result (dot product). (output)

NOTES
       When  working  backward  (incx < 0 or incy < 0), each routine
       starts at the end of the vector and moves backward, as follows:

            x(1-incx * (n-1)), x(1-incx * (n-2)), ..., x(1)

            y(1-incy * (n-1)), y(1-incy * (n-2)), ..., y(1)


            -------
```

9.3.3 fortran code

— zdotc.f —

```
*  ====================================================================
      COMPLEX*16 FUNCTION ZDOTC(N,ZX,INCX,ZY,INCY)
*
```

```
*  -- Reference BLAS level1 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      COMPLEX*16 ZX(*),ZY(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      COMPLEX*16 ZTEMP
      INTEGER I,IX,IY
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DCONJG
*     ..
      ZTEMP = (0.0d0,0.0d0)
      ZDOTC = (0.0d0,0.0d0)
      IF (N.LE.0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*        code for both increments equal to 1
*
         DO I = 1,N
            ZTEMP = ZTEMP + DCONJG(ZX(I))*ZY(I)
         END DO
      ELSE
*
*        code for unequal increments or equal increments
*          not equal to 1
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
         IF (INCY.LT.0) IY = (-N+1)*INCY + 1
         DO I = 1,N
            ZTEMP = ZTEMP + DCONJG(ZX(IX))*ZY(IY)
            IX = IX + INCX
            IY = IY + INCY
         END DO
      END IF
      ZDOTC = ZTEMP
      RETURN
      END
```

———-

9.3.4 lisp code

— BLAS 1 zdotc —

```
(defun zdotc (n zx incx zy incy)
  (declare (type (simple-array (complex double-float) (*)) zy zx)
           (type fixnum incy incx n))
  (f2cl-lib:with-multi-array-data
      ((zx (complex double-float) zx-%data% zx-%offset%)
       (zy (complex double-float) zy-%data% zy-%offset%))
    (prog ((i 0) (ix 0) (iy 0) (ztemp #C(0.0 0.0)) (zdotc #C(0.0 0.0)))
      (declare (type (complex double-float) zdotc ztemp)
               (type fixnum iy ix i))
      (setf ztemp (complex 0.0 0.0))
      (setf zdotc (complex 0.0 0.0))
      (if (<= n 0) (go end_label))
      (if (and (= incx 1) (= incy 1)) (go label20))
      (setf ix 1)
      (setf iy 1)
      (if (< incx 0)
          (setf ix
                  (f2cl-lib:int-add
                   (f2cl-lib:int-mul (the fixnum (1- n)) incx)
                   1)))
      (if (< incy 0)
          (setf iy
                  (f2cl-lib:int-add
                   (f2cl-lib:int-mul (the fixnum (1- n)) incy)
                   1)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf ztemp
                  (+ ztemp
                     (*
                      (f2cl-lib:dconjg
                       (f2cl-lib:fref zx-%data% (ix) ((1 *)) zx-%offset%))
                      (f2cl-lib:fref zy-%data% (iy) ((1 *)) zy-%offset%))))
          (setf ix (f2cl-lib:int-add ix incx))
          (setf iy (f2cl-lib:int-add iy incy))))
      (setf zdotc ztemp)
      (go end_label)
 label20
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
```

```
         (setf ztemp
                 (+ ztemp
                    (*
                     (f2cl-lib:dconjg
                      (f2cl-lib:fref zx-%data% (i) ((1 *)) zx-%offset%))
                     (f2cl-lib:fref zy-%data% (i) ((1 *)) zy-%offset%))))))
     (setf zdotc ztemp)
 end_label
     (return (values zdotc nil nil nil nil nil)))))
```

———-

9.4 zdotu - forms the dot product of two complex vectors X^TY

9.4.1 Axiom unit tests

— **zdotu.input** —

```
)set break resume
)sys rm -f zdotu.output
)spool zdotu.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

9.4.2 Axiom help page

— **zdotu.help** —

```
========================================================================
zdotu examples
========================================================================

========================================================================
Man Page Details
========================================================================
```

```
NAME
       ZDOTU - BLAS level one, computes computes a dot product (inner product)
       of two complex vectors

SYNOPSIS
       DOUBLE COMPLEX FUNCTION ZDOTU ( n, x, incx, y, incy )

           INTEGER                     n, incx, incy

           COMPLEX*16                  x, y


DESCRIPTION
       ZDOTU computes a dot product of two complex vectors.

       This routine performs the following vector operation:

                                                   n
               ZDOTU  <--  (transpose of x) * y  =  Sum  x(i)*y(i)
                                                   i=1
                                          T
       where x and y are real vectors, and x is the transpose of
       x.

       If n <= 0, ZDOTU is set to 0.

ARGUMENTS
       n       INTEGER. (input)
               Number of elements in each vector.

       x       COMPLEX*16. (input)
               Array of dimension (n-1) * |incx| + 1.
               Array x contains the first vector operand.

       incx    INTEGER. (input)
               Increment between elements of x.
               If incx = 0, the results will be unpredictable.

       y       COMPLEX*16. (input)
               Array of dimension (n-1) * |incy| + 1.
               Array y contains the second vector operand.

       incy    INTEGER. (input)
               Increment between elements of y.
               If incy = 0, the results will be unpredictable.

RETURN VALUES
       ZDOTU   DOUBLE COMPLEX. Result (dot product). (output)

NOTES
```

```
When  working  backward  (incx < 0 or incy < 0), each routine starts at
the end of the vector and moves backward, as follows:

     x(1-incx * (n-1)), x(1-incx * (n-2)), ..., x(1)

     y(1-incy * (n-1)), y(1-incy * (n-2)), ..., y(1)
```

———-

9.4.3 fortran code

— **zdotu.f** —

```
*  ====================================================================
      COMPLEX*16 FUNCTION ZDOTU(N,ZX,INCX,ZY,INCY)
*
*  -- Reference BLAS level1 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      COMPLEX*16 ZX(*),ZY(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      COMPLEX*16 ZTEMP
      INTEGER I,IX,IY
*     ..
      ZTEMP = (0.0d0,0.0d0)
      ZDOTU = (0.0d0,0.0d0)
      IF (N.LE.0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*        code for both increments equal to 1
*
         DO I = 1,N
            ZTEMP = ZTEMP + ZX(I)*ZY(I)
         END DO
      ELSE
*
*        code for unequal increments or equal increments
```

```
*          not equal to 1
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
         IF (INCY.LT.0) IY = (-N+1)*INCY + 1
         DO I = 1,N
            ZTEMP = ZTEMP + ZX(IX)*ZY(IY)
            IX = IX + INCX
            IY = IY + INCY
         END DO
      END IF
      ZDOTU = ZTEMP
      RETURN
      END
```

———-

9.4.4 lisp code

— BLAS 1 zdotu —

```
(defun zdotu (n zx incx zy incy)
  (declare (type (simple-array (complex double-float) (*)) zy zx)
           (type fixnum incy incx n))
  (f2cl-lib:with-multi-array-data
      ((zx (complex double-float) zx-%data% zx-%offset%)
       (zy (complex double-float) zy-%data% zy-%offset%))
    (prog ((i 0) (ix 0) (iy 0) (ztemp #C(0.0 0.0)) (zdotu #C(0.0 0.0)))
      (declare (type (complex double-float) zdotu ztemp)
               (type fixnum iy ix i))
      (setf ztemp (complex 0.0 0.0))
      (setf zdotu (complex 0.0 0.0))
      (if (<= n 0) (go end_label))
      (if (and (= incx 1) (= incy 1)) (go label20))
      (setf ix 1)
      (setf iy 1)
      (if (< incx 0)
          (setf ix
                  (f2cl-lib:int-add
                   (f2cl-lib:int-mul (the fixnum (1- n)) incx)
                   1)))
      (if (< incy 0)
          (setf iy
                  (f2cl-lib:int-add
                   (f2cl-lib:int-mul (the fixnum (1- n)) incy)
                   1)))
```

```
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf ztemp
                  (+ ztemp
                     (* (f2cl-lib:fref zx-%data% (ix) ((1 *)) zx-%offset%)
                        (f2cl-lib:fref zy-%data% (iy) ((1 *)) zy-%offset%))))
          (setf ix (f2cl-lib:int-add ix incx))
          (setf iy (f2cl-lib:int-add iy incy))))
      (setf zdotu ztemp)
      (go end_label)
 label20
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf ztemp
                  (+ ztemp
                     (* (f2cl-lib:fref zx-%data% (i) ((1 *)) zx-%offset%)
                        (f2cl-lib:fref zy-%data% (i) ((1 *)) zy-%offset%))))))
      (setf zdotu ztemp)
 end_label
      (return (values zdotu nil nil nil nil nil)))))
```

———-

9.5 zdrot - applies a plane rotation

9.5.1 Axiom unit tests

— **zdrot.input** —

```
)set break resume
)sys rm -f zdrot.output
)spool zdrot.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

9.5.2 Axiom help page

— zdrot.help —

```
==================================================================
zdrot examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zdrot (N, CX, INCX, CY, INCY, C, S)
           ZDROT

Function/Subroutine Documentation
   subroutine zdrot (integer N, complex*16, dimension( * ) CX, integer INCX,
       complex*16, dimension( * ) CY, integer INCY, double precision C, double
       precision S)
       ZDROT

Purpose:
 Applies a plane rotation, where the cos and sin (c and s) are real
 and the vectors cx and cy are complex.

Parameters:
 N
   N is INTEGER
   On entry, N specifies the order of the vectors cx and cy.
   N must be at least zero.

 CX
   CX is COMPLEX*16 array, dimension at least ( 1 + ( N - 1 )*abs( INCX ) ).
   Before entry, the incremented array CX must contain the n
   element vector cx. On exit, CX is overwritten by the updated vector cx.

 INCX
   INCX is INTEGER
     On entry, INCX specifies the increment for the elements of CX.
     INCX must not be zero.

 CY
   CY is COMPLEX*16 array, dimension at least ( 1 + ( N - 1 )*abs( INCY ) ).
   Before entry, the incremented array CY must contain the n
   element vector cy. On exit, CY is overwritten by the updated vector cy.

 INCY
```

```
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of CY.
  INCY must not be zero.

C
  C is DOUBLE PRECISION
  On entry, C specifies the cosine, cos.

S
  S is DOUBLE PRECISION
  On entry, S specifies the sine, sin.
```

———-

9.5.3 fortran code

— **zdrot.f** —

```
*  ====================================================================
      SUBROUTINE ZDROT( N, CX, INCX, CY, INCY, C, S )
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INCX, INCY, N
      DOUBLE PRECISION   C, S
*     ..
*     .. Array Arguments ..
      COMPLEX*16         CX( * ), CY( * )
*     ..
*
* ====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, IX, IY
      COMPLEX*16         CTEMP
*     ..
*     .. Executable Statements ..
*
      IF( N.LE.0 )
     $   RETURN
      IF( INCX.EQ.1 .AND. INCY.EQ.1 ) THEN
*
*        code for both increments equal to 1
```

```
*
         DO I = 1, N
            CTEMP = C*CX( I ) + S*CY( I )
            CY( I ) = C*CY( I ) - S*CX( I )
            CX( I ) = CTEMP
         END DO
      ELSE
*
*        code for unequal increments or equal increments not equal
*          to 1
*
         IX = 1
         IY = 1
         IF( INCX.LT.0 )
     $      IX = ( -N+1 )*INCX + 1
         IF( INCY.LT.0 )
     $      IY = ( -N+1 )*INCY + 1
         DO I = 1, N
            CTEMP = C*CX( IX ) + S*CY( IY )
            CY( IY ) = C*CY( IY ) - S*CX( IX )
            CX( IX ) = CTEMP
            IX = IX + INCX
            IY = IY + INCY
         END DO
      END IF
      RETURN
      END
```

———-

9.5.4 lisp code

— BLAS 1 zdrot —

```
(defun zdrot (n cx incx cy incy c s)
  (declare (type (double-float) s c)
           (type (array f2cl-lib:complex16 (*)) cy cx)
           (type (f2cl-lib:integer4) incy incx n))
  (f2cl-lib:with-multi-array-data
      ((cx f2cl-lib:complex16 cx-%data% cx-%offset%)
       (cy f2cl-lib:complex16 cy-%data% cy-%offset%))
    (prog ((ctemp #C(0.0d0 0.0d0)) (i 0) (ix 0) (iy 0))
      (declare (type (f2cl-lib:integer4) iy ix i)
               (type (f2cl-lib:complex16) ctemp))
      (if (<= n 0)
          (go end_label))
      (cond
```

```
  ((and (= incx 1) (= incy 1))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf ctemp
               (+ (* c (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%))
                  (* s
                     (f2cl-lib:fref cy-%data% (i) ((1 *)) cy-%offset%))))
       (setf (f2cl-lib:fref cy-%data% (i) ((1 *)) cy-%offset%)
               (- (* c (f2cl-lib:fref cy-%data% (i) ((1 *)) cy-%offset%))
                  (* s
                     (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%))))
       (setf (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%) ctemp)
      label100000)))
  (t (setf ix 1) (setf iy 1)
   (if (< incx 0)
       (setf ix
               (f2cl-lib:int-add
                (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incx) 1)))
   (if (< incy 0)
       (setf iy
               (f2cl-lib:int-add
                (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incy) 1)))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf ctemp
               (+ (* c (f2cl-lib:fref cx-%data% (ix) ((1 *)) cx-%offset%))
                  (* s
                     (f2cl-lib:fref cy-%data% (iy) ((1 *)) cy-%offset%))))
       (setf (f2cl-lib:fref cy-%data% (iy) ((1 *)) cy-%offset%)
               (- (* c (f2cl-lib:fref cy-%data% (iy) ((1 *)) cy-%offset%))
                  (* s
                     (f2cl-lib:fref cx-%data% (ix) ((1 *)) cx-%offset%))))
       (setf (f2cl-lib:fref cx-%data% (ix) ((1 *)) cx-%offset%) ctemp)
       (setf ix (f2cl-lib:int-add ix incx))
       (setf iy (f2cl-lib:int-add iy incy))
      label100001))))
 (go end_label)
end_label
 (return (values nil nil nil nil nil nil nil)))))
```

———-

9.6 zdscal - scales a vector by a constant

9.6.1 Axiom unit tests

— **zdscal.input** —

```
)set break resume
)sys rm -f zdscal.output
)spool zdscal.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

9.6.2 Axiom help page

— **zdscal.help** —

```
==================================================================
zdscal examples
==================================================================

==================================================================
Man Page Details
==================================================================

NAME
       ZDSCAL - BLAS level one, Scales a double complex vector

SYNOPSIS
       SUBROUTINE ZDSCAL    ( n, alpha, x, incx )

           INTEGER            n, incx

           DOUBLE COMPLEX     x

           DOUBLE PRECISION   alpha


DESCRIPTION
       ZDSCAL  scales  a double complex vector with a double precision scalar.
       ZDSCAL scales the vector x of length n and increment incx by  the  con-
```

```
       stant alpha.

       This routine performs the following vector operation:

                 x <-- alpha x

       where alpha is a double precision scalar, and x is a double complex
       vector.

ARGUMENTS
       n       INTEGER. (input)
               Number of elements in the vector.
               If n <= 0, this routine returns without computation.

       alpha   DOUBLE PRECISION. (input)
               Value used to scale vector

       x       DOUBLE COMPLEX. (input and output)
               Array  of dimension (n-1) * abs(incx) + 1. Vector to be scaled.

       incx    INTEGER. (input)
               Increment between elements of x.
               If incx = 0, the results will be unpredictable.

NOTES
       When  working  backward  (incx < 0 or incy < 0), each routine starts at
       the end of the vector and moves backward, as follows:

            x(1-incx * (n-1)), x(1-incx * (n-2)), ..., x(1)
```

9.6.3 fortran code

— zdscal.f —

```
*  ===================================================================
      SUBROUTINE ZDSCAL(N,DA,ZX,INCX)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION DA
      INTEGER INCX,N
```

```
*     ..
*     .. Array Arguments ..
      COMPLEX*16 ZX(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER I,NINCX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DCMPLX
*     ..
      IF (N.LE.0 .OR. INCX.LE.0) RETURN
      IF (INCX.EQ.1) THEN
*
*        code for increment equal to 1
*
         DO I = 1,N
            ZX(I) = DCMPLX(DA,0.0d0)*ZX(I)
         END DO
      ELSE
*
*        code for increment not equal to 1
*
         NINCX = N*INCX
         DO I = 1,NINCX,INCX
            ZX(I) = DCMPLX(DA,0.0d0)*ZX(I)
         END DO
      END IF
      RETURN
      END
```

———-

9.6.4 lisp code

— BLAS 1 zdscal —

```
(defun zdscal (n da zx incx)
  (declare (type (simple-array (complex double-float) (*)) zx)
           (type (double-float) da)
           (type fixnum incx n))
  (f2cl-lib:with-multi-array-data
      ((zx (complex double-float) zx-%data% zx-%offset%))
    (prog ((i 0) (ix 0))
      (declare (type fixnum ix i))
```

```
      (if (or (<= n 0) (<= incx 0)) (go end_label))
      (if (= incx 1) (go label20))
      (setf ix 1)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref zx-%data% (ix) ((1 *)) zx-%offset%)
                  (* (coerce (complex da 0.0) '(complex doublefloat))
                     (f2cl-lib:fref zx-%data% (ix) ((1 *)) zx-%offset%)))
          (setf ix (f2cl-lib:int-add ix incx))))
      (go end_label)
 label20
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref zx-%data% (i) ((1 *)) zx-%offset%)
                  (* (coerce (complex da 0.0) '(complex double-float))
                     (f2cl-lib:fref zx-%data% (i) ((1 *)) zx-%offset%)))))
 end_label
      (return (values nil nil nil nil)))))
```

———-

9.7 zrotg - determines a double complex Givens rotation

9.7.1 Axiom unit tests

— **zrotg.input** —

```
)set break resume
)sys rm -f zrotg.output
)spool zrotg.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

9.7.2 Axiom help page

— **zrotg.help** —

```
=====================================================================
zrotg examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

NAME
       ZROTG - Extensions to BLAS level one rotation subroutines


SYNOPSIS
           SUBROUTINE ZROTG ( a, b, c, s )

           DOUBLE COMPLEX
                        a, b, s

           DOUBLE PRECISION
                        c

DESCRIPTION
       ZROTG  computes  the  elements  of  a Givens plane rotation matrix such
       that:

            -            -    -   -    -   -
            |      c    s |    | a |    | r |
            |-congj(s)  c | *  | b | =  | 0 |
            -            -    -   -    -   -

       where r = (a / sqrt(conjg(a)*a)) * sqrt ( conjg(a)*a + conjg(b)*b ) ,
       and the notation conjg(z) represents the complex conjugate of z.

       The Givens plane rotation can be used to introduce zero elements into
       a matrix selectively.



ARGUMENTS
       a       (input and output) DOUBLE COMPLEX

               First vector component.

               On input, the first component of the vector to be rotated.   On
               output,  a is overwritten by the unique complex number r, whose
               size in the complex plane is the Euclidean norm of the  complex
               vector  (a,b),  and whose direction in the complex plane is the
               same as that of the original complex element a.

                   if |a| != 0
```

```
            r = a / |a| * sqrt( conjg(a)*a + conjg(b)*b )

            if |a| = 0
            r = b

b       (input) DOUBLE COMPLEX

        Second vector component.

        The second component of the vector to be rotated.



c       (output) DOUBLE PRECISION

        Cosine of the angle of rotation.

            if |a| != 0
            c = |a| /  sqrt( conjg(a)*a + conjg(b)*b )

            if |a| = 0
            c = 0

s       (output) DOUBLE COMPLEX
        Sine of the angle of rotation.

            if |a| != 0
            c=a/|a|*conjg(b)/sqrt(conjg(a)*a+conjg(b)*b)

            if |a| = 0
            s = ( 1.0 , 0.0 )
```

———-

(Complex Double-Float). Computes plane rotation. Arguments are:

- da - (complex double-float)
- db - (complex double-float)
- c - double-float
- s - (complex double-float)

Returns multiple values where:

- 1 da - ca
- 2 db - nil

- 3 c - c
- 4 s - s

9.7.3 fortran code

— **zrotg.f** —

```
*  ====================================================================
      SUBROUTINE ZROTG(CA,CB,C,S)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX*16 CA,CB,S
      DOUBLE PRECISION C
*     ..
*
*  ====================================================================
*
*     .. Local Scalars ..
      COMPLEX*16 ALPHA
      DOUBLE PRECISION NORM,SCALE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CDABS,DCMPLX,DCONJG,DSQRT
*     ..
      IF (CDABS(CA).EQ.0.0d0) THEN
         C = 0.0d0
         S = (1.0d0,0.0d0)
         CA = CB
      ELSE
         SCALE = CDABS(CA) + CDABS(CB)
         NORM = SCALE*DSQRT((CDABS(CA/DCMPLX(SCALE,0.0d0)))**2+
     $       (CDABS(CB/DCMPLX(SCALE,0.0d0)))**2)
         ALPHA = CA/CDABS(CA)
         C = CDABS(CA)/NORM
         S = ALPHA*DCONJG(CB)/NORM
         CA = ALPHA*NORM
      END IF
      RETURN
      END
```

———-

9.7.4 lisp code

— BLAS 1 zrotg —

```
(defun zrotg (ca cb c s)
  (declare (type (double-float) c) (type (complex double-float) s cb ca))
  (prog ((alpha #C(0.0 0.0)) (norm 0.0) (scale 0.0))
    (declare (type (double-float) scale norm)
             (type (complex double-float) alpha))
    (if (/= (f2cl-lib:cdabs ca) 0.0) (go label10))
    (setf c 0.0)
    (setf s (complex 1.0 0.0))
    (setf ca cb)
 (go label20)
 label10
    (setf scale
            (coerce (+ (f2cl-lib:cdabs ca) (f2cl-lib:cdabs cb)) 'double-float))
    (setf norm
            (* scale
               (f2cl-lib:dsqrt
                (+ (expt (f2cl-lib:cdabs (/ ca
                    (coerce (complex scale 0.0) '(complex double-float)))) 2)
                   (expt (f2cl-lib:cdabs (/ cb
                    (coerce (complex scale 0.0) '(complex double-float))))
                     2)))))
    (setf alpha (/ ca (f2cl-lib:cdabs ca)))
    (setf c (/ (f2cl-lib:cdabs ca) norm))
    (setf s (/ (* alpha (f2cl-lib:dconjg cb)) norm))
    (setf ca (* alpha norm))
 label20
    (return (values ca nil c s))))
```

———-

9.8 zscal - scales a vector by a constant

9.8.1 Axiom unit tests

— zscal.input —

```
)set break resume
)sys rm -f zscal.output
)spool zscal.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

9.8.2 Axiom help page

— **zscal.help** —

```
==================================================================
zscal examples
==================================================================

==================================================================
Man Page Details
==================================================================

NAME
       ZSCAL - BLAS level one, Scales a double complex vector

SYNOPSIS
       SUBROUTINE ZSCAL    ( n, alpha, x, incx )

           INTEGER            n, incx

           DOUBLE COMPLEX     x, alpha


DESCRIPTION
       ZSCAL  scales  a  double  complex  vector with a double complex scalar.
       ZSCAL scales the vector x of length n and increment incx  by  the  con-
       stant alpha.

       This routine performs the following vector operation:

                 x <-- alpha x

       where alpha is a double complex scalar, and x is a double complex
       vector.

ARGUMENTS
       n       INTEGER. (input)
               Number of elements in the vector.
               If n <= 0, this routine returns without computation.
```

```
       alpha   DOUBLE COMPLEX. (input)
               Value used to scale vector

       x       DOUBLE COMPLEX. (input and output)
               Array  of dimension (n-1) * abs(incx) + 1. Vector to be scaled.

       incx    INTEGER. (input)
               Increment between elements of x.
               If incx = 0, the results will be unpredictable.

NOTES
       When  working  backward  (incx < 0 or incy < 0), each routine starts at
       the end of the vector and moves backward, as follows:

            x(1-incx * (n-1)), x(1-incx * (n-2)), ..., x(1)
```

———-

9.8.3 fortran code

— zscal.f —

```
*  ===================================================================
      SUBROUTINE ZSCAL(N,ZA,ZX,INCX)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX*16 ZA
      INTEGER INCX,N
*     ..
*     .. Array Arguments ..
      COMPLEX*16 ZX(*)
*     ..
*
*  ===================================================================
*
*     .. Local Scalars ..
      INTEGER I,NINCX
*     ..
      IF (N.LE.0 .OR. INCX.LE.0) RETURN
      IF (INCX.EQ.1) THEN
*
*        code for increment equal to 1
```

```
*
         DO I = 1,N
            ZX(I) = ZA*ZX(I)
         END DO
      ELSE
*
*        code for increment not equal to 1
*
         NINCX = N*INCX
         DO I = 1,NINCX,INCX
            ZX(I) = ZA*ZX(I)
         END DO
      END IF
      RETURN
      END
```

———-

9.8.4 lisp code

— BLAS 1 zscal —

```
(defun zscal (n za zx incx)
  (declare (type (simple-array (complex double-float) (*)) zx)
           (type (complex double-float) za)
           (type fixnum incx n))
  (f2cl-lib:with-multi-array-data
      ((zx (complex double-float) zx-%data% zx-%offset%))
    (prog ((i 0) (ix 0))
      (declare (type fixnum ix i))
      (if (or (<= n 0) (<= incx 0)) (go end_label))
      (if (= incx 1) (go label20))
      (setf ix 1)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref zx-%data% (ix) ((1 *)) zx-%offset%)
                  (* za (f2cl-lib:fref zx-%data% (ix) ((1 *)) zx-%offset%)))
          (setf ix (f2cl-lib:int-add ix incx))))
      (go end_label)
 label20
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref zx-%data% (i) ((1 *)) zx-%offset%)
                  (* za (f2cl-lib:fref zx-%data% (i) ((1 *)) zx-%offset%)))))
 end_label
```

```
      (return (values nil nil nil nil)))))
```

———-

9.9 zswap - interchanges two vectors

9.9.1 Axiom unit tests

— **zswap.input** —

```
)set break resume
)sys rm -f zswap.output
)spool zswap.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

9.9.2 Axiom help page

— **zswap.help** —

```
==================================================================
zswap examples
==================================================================

==================================================================
Man Page Details
==================================================================

NAME
       ZSWAP - BLAS level one, Swaps two double complex vectors

SYNOPSIS
       SUBROUTINE ZSWAP    ( n, x, incx, y, incy )

           INTEGER           n, incx, incy

           DOUBLE COMPLEX    x, y
```

```
DESCRIPTION
       ZSWAP  swaps  two  double  complex vectors, it interchanges n values of
       vector x and vector y.  incx and incy specify the increment between two
       consecutive elements of respectively vector x and y.


       This routine performs the following vector operation:

                 x <-> y

            where x and y are double complex vectors.

ARGUMENTS
       n       INTEGER. (input)
               Number of vector elements to be swapped.
               If n <= 0, this routine returns without computation.

       x       DOUBLE COMPLEX, (input and output)
               Array of dimension (n-1) * |incx| + 1.

       incx    INTEGER. (input)
               Increment between elements of x.
               If incx = 0, the results will be unpredictable.

       y       DOUBLE COMPLEX, (input and output)
               array of dimension (n-1) * |incy| + 1. Vector to be swapped.

       incy    INTEGER. (input)
               Increment between elements of y.  If incy = 0, the results will
               be unpredictable.

NOTES
       When  working  backward  (incx < 0 or incy < 0), each routine starts at
       the end of the vector and moves backward, as follows:

            x(1-incx * (n-1)), x(1-incx * (n-2)), ..., x(1)

            y(1-incy * (n-1)), y(1-incy * (n-2)), ..., y(1)


            ———-
```

9.9.3 fortran code

— zswap.f —

```
*  =====================================================================
      SUBROUTINE ZSWAP(N,ZX,INCX,ZY,INCY)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,INCY,N
*     ..
*     .. Array Arguments ..
      COMPLEX*16 ZX(*),ZY(*)
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      COMPLEX*16 ZTEMP
      INTEGER I,IX,IY
*     ..
      IF (N.LE.0) RETURN
      IF (INCX.EQ.1 .AND. INCY.EQ.1) THEN
*
*       code for both increments equal to 1
         DO I = 1,N
            ZTEMP = ZX(I)
            ZX(I) = ZY(I)
            ZY(I) = ZTEMP
         END DO
      ELSE
*
*       code for unequal increments or equal increments not equal
*         to 1
*
         IX = 1
         IY = 1
         IF (INCX.LT.0) IX = (-N+1)*INCX + 1
         IF (INCY.LT.0) IY = (-N+1)*INCY + 1
         DO I = 1,N
            ZTEMP = ZX(IX)
            ZX(IX) = ZY(IY)
            ZY(IY) = ZTEMP
            IX = IX + INCX
            IY = IY + INCY
         END DO
      END IF
      RETURN
      END
```

———-

9.9.4 lisp code

— BLAS 1 zswap —

```
(defun zswap (n zx incx zy incy)
  (declare (type (simple-array (complex double-float) (*)) zy zx)
           (type fixnum incy incx n))
  (f2cl-lib:with-multi-array-data
      ((zx (complex double-float) zx-%data% zx-%offset%)
       (zy (complex double-float) zy-%data% zy-%offset%))
    (prog ((i 0) (ix 0) (iy 0) (ztemp #C(0.0 0.0)))
      (declare (type (complex double-float) ztemp)
               (type fixnum iy ix i))
      (if (<= n 0) (go end_label))
      (if (and (= incx 1) (= incy 1)) (go label20))
      (setf ix 1)
      (setf iy 1)
      (if (< incx 0)
          (setf ix
                  (f2cl-lib:int-add
                   (f2cl-lib:int-mul (the fixnum (1- n)) incx)
                   1)))
      (if (< incy 0)
          (setf iy
                  (f2cl-lib:int-add
                   (f2cl-lib:int-mul (the fixnum (1- n)) incy)
                   1)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf ztemp (f2cl-lib:fref zx-%data% (ix) ((1 *)) zx-%offset%))
          (setf (f2cl-lib:fref zx-%data% (ix) ((1 *)) zx-%offset%)
                  (f2cl-lib:fref zy-%data% (iy) ((1 *)) zy-%offset%))
          (setf (f2cl-lib:fref zy-%data% (iy) ((1 *)) zy-%offset%) ztemp)
          (setf ix (f2cl-lib:int-add ix incx))
          (setf iy (f2cl-lib:int-add iy incy))))
      (go end_label)
 label20
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf ztemp (f2cl-lib:fref zx-%data% (i) ((1 *)) zx-%offset%))
          (setf (f2cl-lib:fref zx-%data% (i) ((1 *)) zx-%offset%)
                  (f2cl-lib:fref zy-%data% (i) ((1 *)) zy-%offset%))
          (setf (f2cl-lib:fref zy-%data% (i) ((1 *)) zy-%offset%) ztemp)))
 end_label
```

```
(return (values nil nil nil nil nil)))))
```

———-

Chapter 10

BLAS Level 2 Real Matrix-Vector Operations

10.1 sgbmv - $\alpha Ax + \beta y$ or $\alpha A^T x + \beta y$

10.1.1 Axiom unit tests

— **sgbmv.input** —

```
)set break resume
)sys rm -f sgbmv.output
)spool sgbmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

10.1.2 Axiom help page

— **sgbmv.help** —

```
========================================================================
sgbmv examples
========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgbmv (TRANS, M, N, KL, KU, ALPHA, A, LDA, X, INCX, BETA, Y,
           INCY)
           SGBMV

Function/Subroutine Documentation
   subroutine sgbmv (character TRANS, integer M, integer N, integer KL,
       integer KU, real ALPHA, real, dimension(lda,*) A, integer LDA, real,
       dimension(*) X, integer INCX, real BETA, real, dimension(*) Y, integer
       INCY)
       SGBMV

Purpose:
 SGBMV  performs one of the matrix-vector operations

 y := alpha*A*x + beta*y,   or   y := alpha*A**T*x + beta*y,

 where alpha and beta are scalars, x and y are vectors and A is an
 m by n band matrix, with kl sub-diagonals and ku super-diagonals.

 Parameters:
  TRANS
    TRANS is CHARACTER*1
    On entry, TRANS specifies the operation to be performed as follows:

    TRANS = 'N' or 'n'   y := alpha*A*x + beta*y.

    TRANS = 'T' or 't'   y := alpha*A**T*x + beta*y.

    TRANS = 'C' or 'c'   y := alpha*A**T*x + beta*y.

 M
   M is INTEGER
   On entry, M specifies the number of rows of the matrix A.
   M must be at least zero.

 N
   N is INTEGER
   On entry, N specifies the number of columns of the matrix A.
   N must be at least zero.

 KL
   KL is INTEGER
   On entry, KL specifies the number of sub-diagonals of the
```

```
  matrix A. KL must satisfy  0 .le. KL.

KU
  KU is INTEGER
  On entry, KU specifies the number of super-diagonals of the
  matrix A. KU must satisfy  0 .le. KU.

ALPHA
  ALPHA is REAL
  On entry, ALPHA specifies the scalar alpha.

A
  A is REAL array of DIMENSION ( LDA, n ).
  Before entry, the leading ( kl + ku + 1 ) by n part of the
  array A must contain the matrix of coefficients, supplied
  column by column, with the leading diagonal of the matrix in
  row ( ku + 1 ) of the array, the first super-diagonal
  starting at position 2 in row ku, the first sub-diagonal
  starting at position 1 in row ( ku + 2 ), and so on.
  Elements in the array A that do not correspond to elements
  in the band matrix (such as the top left ku by ku triangle)
  are not referenced.

  The following program segment will transfer a band matrix
  from conventional full matrix storage to band storage:

       DO 20, J = 1, N
          K = KU + 1 - J
          DO 10, I = MAX( 1, J - KU ), MIN( M, J + KL )
             A( K + I, J ) = matrix( I, J )
    10    CONTINUE
    20 CONTINUE

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least ( kl + ku + 1 ).

X
  X is REAL array of DIMENSION at least
  ( 1 + ( n - 1 )*abs( INCX ) ) when TRANS = 'N' or 'n'
  and at least
  ( 1 + ( m - 1 )*abs( INCX ) ) otherwise.
  Before entry, the incremented array X must contain the vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.
```

```
BETA
  BETA is REAL
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.

Y
  Y is REAL array of DIMENSION at least
  ( 1 + ( m - 1 )*abs( INCY ) ) when TRANS = 'N' or 'n'
  and at least
  ( 1 + ( n - 1 )*abs( INCY ) ) otherwise.
  Before entry, the incremented array Y must contain the
  vector y. On exit, Y is overwritten by the updated vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of Y.
  INCY must not be zero.

The vector and matrix arguments are not referenced when N = 0, or M = 0
```

10.1.3 fortran code

— sgbmv.f —

```
*  ====================================================================
      SUBROUTINE SGBMV(TRANS,M,N,KL,KU,ALPHA,A,LDA,X,INCX,BETA,Y,INCY)
*
*  -- Reference BLAS level2 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      REAL ALPHA,BETA
      INTEGER INCX,INCY,KL,KU,LDA,M,N
      CHARACTER TRANS
*     ..
*     .. Array Arguments ..
      REAL A(LDA,*),X(*),Y(*)
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
      REAL ONE,ZERO
```

```
      PARAMETER (ONE=1.0E+0,ZERO=0.0E+0)
*     ..
*     .. Local Scalars ..
      REAL TEMP
      INTEGER I,INFO,IX,IY,J,JX,JY,K,KUP1,KX,KY,LENX,LENY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX,MIN
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +    .NOT.LSAME(TRANS,'C')) THEN
          INFO = 1
      ELSE IF (M.LT.0) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
          INFO = 3
      ELSE IF (KL.LT.0) THEN
          INFO = 4
      ELSE IF (KU.LT.0) THEN
          INFO = 5
      ELSE IF (LDA.LT. (KL+KU+1)) THEN
          INFO = 8
      ELSE IF (INCX.EQ.0) THEN
          INFO = 10
      ELSE IF (INCY.EQ.0) THEN
          INFO = 13
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('SGBMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR.
     +    ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     Set  LENX  and  LENY, the lengths of the vectors x and y, and set
*     up the start points in  X  and  Y.
```

```
*
      IF (LSAME(TRANS,'N')) THEN
          LENX = N
          LENY = M
      ELSE
          LENX = M
          LENY = N
      END IF
      IF (INCX.GT.0) THEN
          KX = 1
      ELSE
          KX = 1 - (LENX-1)*INCX
      END IF
      IF (INCY.GT.0) THEN
          KY = 1
      ELSE
          KY = 1 - (LENY-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the band part of A.
*
*     First form  y := beta*y.
*
      IF (BETA.NE.ONE) THEN
          IF (INCY.EQ.1) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 10 I = 1,LENY
                      Y(I) = ZERO
   10             CONTINUE
              ELSE
                  DO 20 I = 1,LENY
                      Y(I) = BETA*Y(I)
   20             CONTINUE
              END IF
          ELSE
              IY = KY
              IF (BETA.EQ.ZERO) THEN
                  DO 30 I = 1,LENY
                      Y(IY) = ZERO
                      IY = IY + INCY
   30             CONTINUE
              ELSE
                  DO 40 I = 1,LENY
                      Y(IY) = BETA*Y(IY)
                      IY = IY + INCY
   40             CONTINUE
              END IF
          END IF
      END IF
```

```
      IF (ALPHA.EQ.ZERO) RETURN
      KUP1 = KU + 1
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  y := alpha*A*x + y.
*
          JX = KX
          IF (INCY.EQ.1) THEN
              DO 60 J = 1,N
                  TEMP = ALPHA*X(JX)
                  K = KUP1 - J
                  DO 50 I = MAX(1,J-KU),MIN(M,J+KL)
                      Y(I) = Y(I) + TEMP*A(K+I,J)
   50             CONTINUE
                  JX = JX + INCX
   60         CONTINUE
          ELSE
              DO 80 J = 1,N
                  TEMP = ALPHA*X(JX)
                  IY = KY
                  K = KUP1 - J
                  DO 70 I = MAX(1,J-KU),MIN(M,J+KL)
                      Y(IY) = Y(IY) + TEMP*A(K+I,J)
                      IY = IY + INCY
   70             CONTINUE
                  JX = JX + INCX
                  IF (J.GT.KU) KY = KY + INCY
   80         CONTINUE
          END IF
      ELSE
*
*        Form  y := alpha*A**T*x + y.
*
          JY = KY
          IF (INCX.EQ.1) THEN
              DO 100 J = 1,N
                  TEMP = ZERO
                  K = KUP1 - J
                  DO 90 I = MAX(1,J-KU),MIN(M,J+KL)
                      TEMP = TEMP + A(K+I,J)*X(I)
   90             CONTINUE
                  Y(JY) = Y(JY) + ALPHA*TEMP
                  JY = JY + INCY
  100         CONTINUE
          ELSE
              DO 120 J = 1,N
                  TEMP = ZERO
                  IX = KX
                  K = KUP1 - J
                  DO 110 I = MAX(1,J-KU),MIN(M,J+KL)
```

```
                    TEMP = TEMP + A(K+I,J)*X(IX)
                    IX = IX + INCX
  110             CONTINUE
                  Y(JY) = Y(JY) + ALPHA*TEMP
                  JY = JY + INCY
                  IF (J.GT.KU) KX = KX + INCX
  120         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of SGBMV .
*
      END
```

———-

10.1.4 lisp code

— BLAS 2 sgbmv —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun sgbmv (trans m n kl ku alpha a lda x incx beta y incy)
    (declare (type (array single-float (*)) y x a)
             (type (single-float) beta alpha)
             (type (f2cl-lib:integer4) incy incx lda ku kl n m)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (a single-float a-%data% a-%offset%)
         (x single-float x-%data% x-%offset%)
         (y single-float y-%data% y-%offset%))
      (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (k 0) (kup1 0)
             (kx 0) (ky 0) (lenx 0) (leny 0) (temp 0.0))
        (declare
         (type (f2cl-lib:integer4) i info ix iy j jx jy k kup1 kx ky lenx leny)
         (type (single-float) temp))
        (setf info 0)
        (cond
         ((and (not (lsame trans "N")) (not (lsame trans "T"))
               (not (lsame trans "C")))
          (setf info 1))
         ((< m 0) (setf info 2)) ((< n 0) (setf info 3))
```

```
 ((< kl 0) (setf info 4)) ((< ku 0) (setf info 5))
 ((< lda (f2cl-lib:int-add kl ku 1)) (setf info 8))
 ((= incx 0) (setf info 10)) ((= incy 0) (setf info 13)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "SGBMV " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
    (go end_label))
(cond ((lsame trans "N") (setf lenx n) (setf leny m))
      (t (setf lenx m) (setf leny n)))
(cond ((> incx 0) (setf kx 1))
      (t
       (setf kx
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub lenx 1) incx)))))
(cond ((> incy 0) (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub leny 1) incy)))))
(cond
 ((/= beta one)
  (cond
   ((= incy 1)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i leny) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%) zero)
         label10)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i leny) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                  (* beta
                     (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)))
         label20)))))
   (t (setf iy ky)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i leny) nil)
```

```
          (tagbody
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) zero)
            (setf iy (f2cl-lib:int-add iy incy))
           label30)))
      (t
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i leny) nil)
          (tagbody
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (* beta
                       (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)))
            (setf iy (f2cl-lib:int-add iy incy))
           label40))))))))
(if (= alpha zero)
    (go end_label))
(setf kup1 (f2cl-lib:int-add ku 1))
(cond
 ((lsame trans "N") (setf jx kx)
  (cond
   ((= incy 1)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp
                (* alpha
                   (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
        (setf k (f2cl-lib:int-sub kup1 j))
        (f2cl-lib:fdo (i
                       (max (the f2cl-lib:integer4 1)
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add j
                                                   (f2cl-lib:int-sub
                                                    ku))))
                       (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the f2cl-lib:integer4 m)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add j kl))))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                       (* temp
                          (f2cl-lib:fref a-%data%
                                         ((f2cl-lib:int-add k i) j)
                                         ((1 lda) (1 *)) a-%offset%))))
           label50))
        (setf jx (f2cl-lib:int-add jx incx))
       label60)))
   (t
```

```
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf temp
                  (* alpha
                     (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
          (setf iy ky)
          (setf k (f2cl-lib:int-sub kup1 j))
          (f2cl-lib:fdo (i
                         (max (the f2cl-lib:integer4 1)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add j
                                                     (f2cl-lib:int-sub
                                                      ku))))
                         (f2cl-lib:int-add i 1))
                        ((> i
                            (min (the f2cl-lib:integer4 m)
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add j kl))))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (* temp
                            (f2cl-lib:fref a-%data%
                                           ((f2cl-lib:int-add k i) j)
                                           ((1 lda) (1 *)) a-%offset%))))
              (setf iy (f2cl-lib:int-add iy incy))
             label70))
          (setf jx (f2cl-lib:int-add jx incx))
          (if (> j ku)
              (setf ky (f2cl-lib:int-add ky incy)))
         label80)))))
    (t (setf jy ky)
     (cond
      ((= incx 1)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (setf temp zero)
           (setf k (f2cl-lib:int-sub kup1 j))
           (f2cl-lib:fdo (i
                          (max (the f2cl-lib:integer4 1)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add j
                                                      (f2cl-lib:int-sub
                                                       ku))))
                          (f2cl-lib:int-add i 1))
                         ((> i
                             (min (the f2cl-lib:integer4 m)
```

```
                                (the f2cl-lib:integer4
                                     (f2cl-lib:int-add j kl))))
                        nil)
          (tagbody
            (setf temp
                    (+ temp
                       (*
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add k i) j)
                                       ((1 lda) (1 *)) a-%offset%)
                        (f2cl-lib:fref x-%data% (i) ((1 *))
                                       x-%offset%))))
           label90))
        (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                   (* alpha temp)))
        (setf jy (f2cl-lib:int-add jy incy))
       label100)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp zero)
         (setf ix kx)
         (setf k (f2cl-lib:int-sub kup1 j))
         (f2cl-lib:fdo (i
                        (max (the f2cl-lib:integer4 1)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add j
                                                    (f2cl-lib:int-sub
                                                     ku))))
                        (f2cl-lib:int-add i 1))
                       ((> i
                           (min (the f2cl-lib:integer4 m)
                                (the f2cl-lib:integer4
                                     (f2cl-lib:int-add j kl))))
                        nil)
          (tagbody
            (setf temp
                    (+ temp
                       (*
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add k i) j)
                                       ((1 lda) (1 *)) a-%offset%)
                        (f2cl-lib:fref x-%data% (ix) ((1 *))
                                       x-%offset%))))
            (setf ix (f2cl-lib:int-add ix incx))
           label110))
        (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
```

```
                        (* alpha temp)))
                (setf jy (f2cl-lib:int-add jy incy))
                (if (> j ku)
                    (setf kx (f2cl-lib:int-add kx incx)))
               label120))))))
         (go end_label)
        end_label
         (return
          (values nil nil nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

10.2 sgemv - matrix-vector $\alpha Ax + \beta y$ or $\alpha A^T x + \beta y$

10.2.1 Axiom unit tests

— **sgemv.input** —

```
)set break resume
)sys rm -f sgemv.output
)spool sgemv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

10.2.2 Axiom help page

— **sgemv.help** —

```
======================================================================
sgemv examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
```

```
   Functions/Subroutines
       subroutine sgemv (TRANS, M, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)
           SGEMV

Function/Subroutine Documentation
   subroutine sgemv (character TRANS, integer M, integer N, real ALPHA, real,
       dimension(lda,*) A, integer LDA, real, dimension(*) X, integer INCX,
       real BETA, real, dimension(*) Y, integer INCY)
       SGEMV

Purpose:
  SGEMV  performs one of the matrix-vector operations

  y := alpha*A*x + beta*y,   or   y := alpha*A**T*x + beta*y,

  where alpha and beta are scalars, x and y are vectors and A is an
  m by n matrix.

Parameters:
  TRANS
    TRANS is CHARACTER*1
    On entry, TRANS specifies the operation to be performed as follows:

    TRANS = 'N' or 'n'   y := alpha*A*x + beta*y.

    TRANS = 'T' or 't'   y := alpha*A**T*x + beta*y.

    TRANS = 'C' or 'c'   y := alpha*A**T*x + beta*y.

 M
   M is INTEGER
   On entry, M specifies the number of rows of the matrix A.
   M must be at least zero.

 N
   N is INTEGER
   On entry, N specifies the number of columns of the matrix A.
   N must be at least zero.

 ALPHA
   ALPHA is REAL
   On entry, ALPHA specifies the scalar alpha.

 A
   A is REAL array of DIMENSION ( LDA, n ).
   Before entry, the leading m by n part of the array A must
   contain the matrix of coefficients.

 LDA
   LDA is INTEGER
```

```
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, m ).

X
  X is REAL array of DIMENSION at least
  ( 1 + ( n - 1 )*abs( INCX ) ) when TRANS = 'N' or 'n'
  and at least
  ( 1 + ( m - 1 )*abs( INCX ) ) otherwise.
  Before entry, the incremented array X must contain the vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.

BETA
  BETA is REAL
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.

Y
  Y is REAL array of DIMENSION at least
  ( 1 + ( m - 1 )*abs( INCY ) ) when TRANS = 'N' or 'n'
  and at least
  ( 1 + ( n - 1 )*abs( INCY ) ) otherwise.
  Before entry with BETA non-zero, the incremented array Y
  must contain the vector y. On exit, Y is overwritten by the
  updated vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of Y.
  INCY must not be zero.

  The vector and matrix arguments are not referenced when N = 0, or M = 0
```

———-

10.2.3 fortran code

— **sgemv.f** —

```
*  ====================================================================
      SUBROUTINE SGEMV(TRANS,M,N,ALPHA,A,LDA,X,INCX,BETA,Y,INCY)
*
*  -- Reference BLAS level2 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
```

```
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      REAL ALPHA,BETA
      INTEGER INCX,INCY,LDA,M,N
      CHARACTER TRANS
*     ..
*     .. Array Arguments ..
      REAL A(LDA,*),X(*),Y(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL ONE,ZERO
      PARAMETER (ONE=1.0E+0,ZERO=0.0E+0)
*     ..
*     .. Local Scalars ..
      REAL TEMP
      INTEGER I,INFO,IX,IY,J,JX,JY,KX,KY,LENX,LENY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +    .NOT.LSAME(TRANS,'C')) THEN
          INFO = 1
      ELSE IF (M.LT.0) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
          INFO = 3
      ELSE IF (LDA.LT.MAX(1,M)) THEN
          INFO = 6
      ELSE IF (INCX.EQ.0) THEN
          INFO = 8
      ELSE IF (INCY.EQ.0) THEN
          INFO = 11
      END IF
```

```
      IF (INFO.NE.0) THEN
          CALL XERBLA('SGEMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR.
     +    ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     Set  LENX  and  LENY, the lengths of the vectors x and y, and set
*     up the start points in  X  and  Y.
*
      IF (LSAME(TRANS,'N')) THEN
          LENX = N
          LENY = M
      ELSE
          LENX = M
          LENY = N
      END IF
      IF (INCX.GT.0) THEN
          KX = 1
      ELSE
          KX = 1 - (LENX-1)*INCX
      END IF
      IF (INCY.GT.0) THEN
          KY = 1
      ELSE
          KY = 1 - (LENY-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
*     First form  y := beta*y.
*
      IF (BETA.NE.ONE) THEN
          IF (INCY.EQ.1) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 10 I = 1,LENY
                      Y(I) = ZERO
   10             CONTINUE
              ELSE
                  DO 20 I = 1,LENY
                      Y(I) = BETA*Y(I)
   20             CONTINUE
              END IF
          ELSE
              IY = KY
              IF (BETA.EQ.ZERO) THEN
```

```
                DO 30 I = 1,LENY
                    Y(IY) = ZERO
                    IY = IY + INCY
   30           CONTINUE
            ELSE
                DO 40 I = 1,LENY
                    Y(IY) = BETA*Y(IY)
                    IY = IY + INCY
   40           CONTINUE
            END IF
        END IF
      END IF
      IF (ALPHA.EQ.ZERO) RETURN
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  y := alpha*A*x + y.
*
         JX = KX
         IF (INCY.EQ.1) THEN
            DO 60 J = 1,N
                TEMP = ALPHA*X(JX)
                DO 50 I = 1,M
                    Y(I) = Y(I) + TEMP*A(I,J)
   50           CONTINUE
                JX = JX + INCX
   60       CONTINUE
         ELSE
            DO 80 J = 1,N
                TEMP = ALPHA*X(JX)
                IY = KY
                DO 70 I = 1,M
                    Y(IY) = Y(IY) + TEMP*A(I,J)
                    IY = IY + INCY
   70           CONTINUE
                JX = JX + INCX
   80       CONTINUE
         END IF
      ELSE
*
*        Form  y := alpha*A**T*x + y.
*
         JY = KY
         IF (INCX.EQ.1) THEN
            DO 100 J = 1,N
                TEMP = ZERO
                DO 90 I = 1,M
                    TEMP = TEMP + A(I,J)*X(I)
   90           CONTINUE
                Y(JY) = Y(JY) + ALPHA*TEMP
                JY = JY + INCY
```

```
  100         CONTINUE
          ELSE
              DO 120 J = 1,N
                  TEMP = ZERO
                  IX = KX
                  DO 110 I = 1,M
                      TEMP = TEMP + A(I,J)*X(IX)
                      IX = IX + INCX
  110             CONTINUE
                  Y(JY) = Y(JY) + ALPHA*TEMP
                  JY = JY + INCY
  120         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of SGEMV .
*
      END
```

———-

10.2.4 lisp code

— BLAS 2 sgemv —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun sgemv (trans m n alpha a lda x incx beta y incy)
    (declare (type (array single-float (*)) y x a)
             (type (single-float) beta alpha)
             (type (f2cl-lib:integer4) incy incx lda n m)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (a single-float a-%data% a-%offset%)
         (x single-float x-%data% x-%offset%)
         (y single-float y-%data% y-%offset%))
      (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (kx 0) (ky 0)
             (lenx 0) (leny 0) (temp 0.0))
        (declare
         (type (f2cl-lib:integer4) i info ix iy j jx jy kx ky lenx leny)
         (type (single-float) temp))
        (setf info 0)
```

```
(cond
 ((and (not (lsame trans "N")) (not (lsame trans "T"))
       (not (lsame trans "C")))
  (setf info 1))
 ((< m 0) (setf info 2)) ((< n 0) (setf info 3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info 6))
 ((= incx 0) (setf info 8)) ((= incy 0) (setf info 11)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "SGEMV " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
    (go end_label))
(cond ((lsame trans "N") (setf lenx n) (setf leny m))
      (t (setf lenx m) (setf leny n)))
(cond ((> incx 0) (setf kx 1))
      (t
       (setf kx
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub lenx 1) incx)))))
(cond ((> incy 0) (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub leny 1) incy)))))
(cond
 ((/= beta one)
  (cond
   ((= incy 1)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i leny) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%) zero)
         label10)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i leny) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                  (* beta
                     (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)))
         label20)))))
```

```
    (t (setf iy ky)
     (cond
      ((= beta zero)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i leny) nil)
         (tagbody
           (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) zero)
           (setf iy (f2cl-lib:int-add iy incy))
          label30)))
      (t
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i leny) nil)
         (tagbody
           (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                   (* beta
                      (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)))
           (setf iy (f2cl-lib:int-add iy incy))
          label40))))))))
 (if (= alpha zero)
     (go end_label))
 (cond
  ((lsame trans "N") (setf jx kx)
   (cond
    ((= incy 1)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp
                 (* alpha
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                        (* temp
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%))))
            label50))
         (setf jx (f2cl-lib:int-add jx incx))
        label60)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp
                 (* alpha
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
         (setf iy ky)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                    ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (* temp
                            (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                           a-%offset%))))
              (setf iy (f2cl-lib:int-add iy incy))
             label70))
           (setf jx (f2cl-lib:int-add jx incx))
          label80)))))
      (t (setf jy ky)
       (cond
        ((= incx 1)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf temp zero)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i m) nil)
               (tagbody
                 (setf temp
                         (+ temp
                            (*
                             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                            a-%offset%)
                             (f2cl-lib:fref x-%data% (i) ((1 *))
                                            x-%offset%))))
                label90))
             (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                     (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                        (* alpha temp)))
             (setf jy (f2cl-lib:int-add jy incy))
            label100)))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf temp zero)
             (setf ix kx)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i m) nil)
               (tagbody
                 (setf temp
                         (+ temp
                            (*
                             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                            a-%offset%)
                             (f2cl-lib:fref x-%data% (ix) ((1 *))
                                            x-%offset%))))
```

```
                (setf ix (f2cl-lib:int-add ix incx))
               label110))
            (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                  (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                     (* alpha temp)))
            (setf jy (f2cl-lib:int-add jy incy))
           label120))))))
     (go end_label)
    end_label
     (return (values nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

10.3 sger - rank 1 $\alpha xy^T + A$

10.3.1 Axiom unit tests

— **sger.input** —

```
)set break resume
)sys rm -f sger.output
)spool sger.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

10.3.2 Axiom help page

— **sger.help** —

```
======================================================================
sger examples
======================================================================

======================================================================
Man Page Details
======================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine sger (M, N, ALPHA, X, INCX, Y, INCY, A, LDA)
           SGER

Function/Subroutine Documentation
   subroutine sger (integer M, integer N, real ALPHA, real, dimension(*) X,
       integer INCX, real, dimension(*) Y, integer INCY, real,
       dimension(lda,*) A, integer LDA)
       SGER

Purpose:

 SGER   performs the rank 1 operation A := alpha*x*y**T + A,
        where alpha is a scalar, x is an m element vector, y is an n element
        vector and A is an m by n matrix.

Parameters:

 M
   M is INTEGER
   On entry, M specifies the number of rows of the matrix A.
   M must be at least zero.

 N
   N is INTEGER
   On entry, N specifies the number of columns of the matrix A.
   N must be at least zero.

 ALPHA
   ALPHA is REAL
   On entry, ALPHA specifies the scalar alpha.

 X
   X is REAL array of dimension at least ( 1 + ( m - 1 )*abs( INCX ) ).
   Before entry, the incremented array X must contain the m
   element vector x.

 INCX
   INCX is INTEGER
   On entry, INCX specifies the increment for the elements of X.
   INCX must not be zero.

 Y
   Y is REAL array of dimension at least
   ( 1 + ( n - 1 )*abs( INCY ) ).
   Before entry, the incremented array Y must contain the n
   element vector y.

 INCY
```

```
   INCY is INTEGER
   On entry, INCY specifies the increment for the elements of Y.
   INCY must not be zero.

 A
   A is REAL array of DIMENSION ( LDA, n ).
   Before entry, the leading m by n part of the array A must
   contain the matrix of coefficients. On exit, A is
   overwritten by the updated matrix.

 LDA
   LDA is INTEGER
   On entry, LDA specifies the first dimension of A as declared
   in the calling (sub) program. LDA must be at least max( 1, m ).
```

10.3.3 fortran code

— sger.f —

```
*  =====================================================================
      SUBROUTINE SGER(M,N,ALPHA,X,INCX,Y,INCY,A,LDA)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL ALPHA
      INTEGER INCX,INCY,LDA,M,N
*     ..
*     .. Array Arguments ..
      REAL A(LDA,*),X(*),Y(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL ZERO
      PARAMETER (ZERO=0.0E+0)
*     ..
*     .. Local Scalars ..
      REAL TEMP
      INTEGER I,INFO,IX,J,JY,KX
*     ..
```

```
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (M.LT.0) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      ELSE IF (INCY.EQ.0) THEN
          INFO = 7
      ELSE IF (LDA.LT.MAX(1,M)) THEN
          INFO = 9
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('SGER  ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR. (ALPHA.EQ.ZERO)) RETURN
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
      IF (INCY.GT.0) THEN
          JY = 1
      ELSE
          JY = 1 - (N-1)*INCY
      END IF
      IF (INCX.EQ.1) THEN
          DO 20 J = 1,N
              IF (Y(JY).NE.ZERO) THEN
                  TEMP = ALPHA*Y(JY)
                  DO 10 I = 1,M
                      A(I,J) = A(I,J) + X(I)*TEMP
   10             CONTINUE
              END IF
              JY = JY + INCY
   20     CONTINUE
      ELSE
          IF (INCX.GT.0) THEN
```

```
            KX = 1
        ELSE
            KX = 1 - (M-1)*INCX
        END IF
        DO 40 J = 1,N
            IF (Y(JY).NE.ZERO) THEN
                TEMP = ALPHA*Y(JY)
                IX = KX
                DO 30 I = 1,M
                    A(I,J) = A(I,J) + X(IX)*TEMP
                    IX = IX + INCX
   30           CONTINUE
            END IF
            JY = JY + INCY
   40   CONTINUE
      END IF
*
      RETURN
*
*     End of SGER  .
*
      END
```

———-

10.3.4 lisp code

— BLAS 2 sger —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun sger (m n alpha x incx y incy a lda)
    (declare (type (array single-float (*)) a y x)
             (type (single-float) alpha)
             (type (f2cl-lib:integer4) lda incy incx n m))
    (f2cl-lib:with-multi-array-data
        ((x single-float x-%data% x-%offset%)
         (y single-float y-%data% y-%offset%)
         (a single-float a-%data% a-%offset%))
      (prog ((i 0) (info 0) (ix 0) (j 0) (jy 0) (kx 0) (temp 0.0))
        (declare (type (f2cl-lib:integer4) i info ix j jy kx)
                 (type (single-float) temp))
        (setf info 0)
        (cond ((< m 0) (setf info 1)) ((< n 0) (setf info 2))
              ((= incx 0) (setf info 5)) ((= incy 0) (setf info 7))
              ((< lda
```

```
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info 9)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "SGER  " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= m 0) (= n 0) (= alpha zero))
    (go end_label))
(cond ((> incy 0) (setf jy 1))
      (t
       (setf jy
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incy)))))
(cond
 ((= incx 1)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (cond
       ((/= (f2cl-lib:fref y (jy) ((1 *))) zero)
        (setf temp
                (* alpha
                   (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%)
                    (+
                     (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%)
                     (* (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                        temp)))
           label10))))
      (setf jy (f2cl-lib:int-add jy incy))
     label20)))
 (t
  (cond ((> incx 0) (setf kx 1))
        (t
         (setf kx
                 (f2cl-lib:int-sub 1
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-sub m 1) incx)))))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
```

```
            (cond
             ((/= (f2cl-lib:fref y (jy) ((1 *))) zero)
              (setf temp
                      (* alpha
                         (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)))
              (setf ix kx)
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i m) nil)
                (tagbody
                  (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%)
                          (+
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%)
                           (*
                            (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                            temp)))
                  (setf ix (f2cl-lib:int-add ix incx))
                 label30))))
            (setf jy (f2cl-lib:int-add jy incy))
           label40))))
      (go end_label)
     end_label
      (return (values nil nil nil nil nil nil nil nil nil))))))
```

———-

10.4 ssbmv - $\alpha Ax + \beta y$

10.4.1 Axiom unit tests

— **ssbmv.input** —

```
)set break resume
)sys rm -f ssbmv.output
)spool ssbmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

10.4.2 Axiom help page

— ssbmv.help —

```
==================================================================
ssbmv examples
==================================================================

==================================================================
Man Page Details
==================================================================

NAME
       ssbmv.f -

SYNOPSIS
   Functions/Subroutines
       subroutine ssbmv (UPLO, N, K, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)
           SSBMV

Function/Subroutine Documentation
   subroutine ssbmv (character UPLO, integer N, integer K, real ALPHA, real,
       dimension(lda,*) A, integer LDA, real, dimension(*) X, integer INCX,
       real BETA, real, dimension(*) Y, integer INCY)
       SSBMV

Purpose:
  SSBMV  performs the matrix-vector  operation

    y := alpha*A*x + beta*y,

 where alpha and beta are scalars, x and y are n element vectors and
 A is an n by n symmetric band matrix, with k super-diagonals.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the upper or lower
   triangular part of the band matrix A is being supplied as follows:

   UPLO = 'U' or 'u'   The upper triangular part of A is being supplied.

   UPLO = 'L' or 'l'   The lower triangular part of A is being supplied.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
   N must be at least zero.
```

```
K
  K is INTEGER
  On entry, K specifies the number of super-diagonals of the
  matrix A. K must satisfy  0 .le. K.

ALPHA
  ALPHA is REAL
  On entry, ALPHA specifies the scalar alpha.

A
  A is REAL array of DIMENSION ( LDA, n ).
  Before entry with UPLO = 'U' or 'u', the leading ( k + 1 )
  by n part of the array A must contain the upper triangular
  band part of the symmetric matrix, supplied column by
  column, with the leading diagonal of the matrix in row
  ( k + 1 ) of the array, the first super-diagonal starting at
  position 2 in row k, and so on. The top left k by k triangle
  of the array A is not referenced.

  The following program segment will transfer the upper
  triangular part of a symmetric band matrix from conventional
  full matrix storage to band storage:

       DO 20, J = 1, N
          M = K + 1 - J
          DO 10, I = MAX( 1, J - K ), J
             A( M + I, J ) = matrix( I, J )
    10    CONTINUE
    20 CONTINUE

  Before entry with UPLO = 'L' or 'l', the leading ( k + 1 )
  by n part of the array A must contain the lower triangular
  band part of the symmetric matrix, supplied column by
  column, with the leading diagonal of the matrix in row 1 of
  the array, the first sub-diagonal starting at position 1 in
  row 2, and so on. The bottom right k by k triangle of the
  array A is not referenced.

  The following program segment will transfer the lower
  triangular part of a symmetric band matrix from conventional
  full matrix storage to band storage:

       DO 20, J = 1, N
          M = 1 - J
          DO 10, I = J, MIN( N, J + K )
             A( M + I, J ) = matrix( I, J )
    10    CONTINUE
    20 CONTINUE
```

```
LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least ( k + 1 ).

X
  X is REAL array of DIMENSION at least ( 1 + ( n - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.

BETA
  BETA is REAL
  On entry, BETA specifies the scalar beta.

Y
  Y is REAL array of DIMENSION at least ( 1 + ( n - 1 )*abs( INCY ) ).
  Before entry, the incremented array Y must contain the
  vector y. On exit, Y is overwritten by the updated vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of Y.
  INCY must not be zero.

The vector and matrix arguments are not referenced when N = 0, or M = 0
```

10.4.3 fortran code

— ssbmv.f —

```
*  ===================================================================
      SUBROUTINE SSBMV(UPLO,N,K,ALPHA,A,LDA,X,INCX,BETA,Y,INCY)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL ALPHA,BETA
      INTEGER INCX,INCY,K,LDA,N
```

```
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      REAL A(LDA,*),X(*),Y(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL ONE,ZERO
      PARAMETER (ONE=1.0E+0,ZERO=0.0E+0)
*     ..
*     .. Local Scalars ..
      REAL TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,KPLUS1,KX,KY,L
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX,MIN
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (K.LT.0) THEN
          INFO = 3
      ELSE IF (LDA.LT. (K+1)) THEN
          INFO = 6
      ELSE IF (INCX.EQ.0) THEN
          INFO = 8
      ELSE IF (INCY.EQ.0) THEN
          INFO = 11
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('SSBMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
```

```
      IF ((N.EQ.0) .OR. ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     Set up the start points in  X  and  Y.
*
      IF (INCX.GT.0) THEN
          KX = 1
      ELSE
          KX = 1 - (N-1)*INCX
      END IF
      IF (INCY.GT.0) THEN
          KY = 1
      ELSE
          KY = 1 - (N-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of the array A
*     are accessed sequentially with one pass through A.
*
*     First form  y := beta*y.
*
      IF (BETA.NE.ONE) THEN
          IF (INCY.EQ.1) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 10 I = 1,N
                      Y(I) = ZERO
   10             CONTINUE
              ELSE
                  DO 20 I = 1,N
                      Y(I) = BETA*Y(I)
   20             CONTINUE
              END IF
          ELSE
              IY = KY
              IF (BETA.EQ.ZERO) THEN
                  DO 30 I = 1,N
                      Y(IY) = ZERO
                      IY = IY + INCY
   30             CONTINUE
              ELSE
                  DO 40 I = 1,N
                      Y(IY) = BETA*Y(IY)
                      IY = IY + INCY
   40             CONTINUE
              END IF
          END IF
      END IF
      IF (ALPHA.EQ.ZERO) RETURN
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  y  when upper triangle of A is stored.
```

```
*
         KPLUS1 = K + 1
         IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
             DO 60 J = 1,N
                 TEMP1 = ALPHA*X(J)
                 TEMP2 = ZERO
                 L = KPLUS1 - J
                 DO 50 I = MAX(1,J-K),J - 1
                     Y(I) = Y(I) + TEMP1*A(L+I,J)
                     TEMP2 = TEMP2 + A(L+I,J)*X(I)
   50            CONTINUE
                 Y(J) = Y(J) + TEMP1*A(KPLUS1,J) + ALPHA*TEMP2
   60        CONTINUE
         ELSE
             JX = KX
             JY = KY
             DO 80 J = 1,N
                 TEMP1 = ALPHA*X(JX)
                 TEMP2 = ZERO
                 IX = KX
                 IY = KY
                 L = KPLUS1 - J
                 DO 70 I = MAX(1,J-K),J - 1
                     Y(IY) = Y(IY) + TEMP1*A(L+I,J)
                     TEMP2 = TEMP2 + A(L+I,J)*X(IX)
                     IX = IX + INCX
                     IY = IY + INCY
   70            CONTINUE
                 Y(JY) = Y(JY) + TEMP1*A(KPLUS1,J) + ALPHA*TEMP2
                 JX = JX + INCX
                 JY = JY + INCY
                 IF (J.GT.K) THEN
                     KX = KX + INCX
                     KY = KY + INCY
                 END IF
   80        CONTINUE
         END IF
      ELSE
*
*        Form  y  when lower triangle of A is stored.
*
         IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
             DO 100 J = 1,N
                 TEMP1 = ALPHA*X(J)
                 TEMP2 = ZERO
                 Y(J) = Y(J) + TEMP1*A(1,J)
                 L = 1 - J
                 DO 90 I = J + 1,MIN(N,J+K)
                     Y(I) = Y(I) + TEMP1*A(L+I,J)
                     TEMP2 = TEMP2 + A(L+I,J)*X(I)
```

```
   90             CONTINUE
                  Y(J) = Y(J) + ALPHA*TEMP2
  100         CONTINUE
          ELSE
              JX = KX
              JY = KY
              DO 120 J = 1,N
                  TEMP1 = ALPHA*X(JX)
                  TEMP2 = ZERO
                  Y(JY) = Y(JY) + TEMP1*A(1,J)
                  L = 1 - J
                  IX = JX
                  IY = JY
                  DO 110 I = J + 1,MIN(N,J+K)
                      IX = IX + INCX
                      IY = IY + INCY
                      Y(IY) = Y(IY) + TEMP1*A(L+I,J)
                      TEMP2 = TEMP2 + A(L+I,J)*X(IX)
  110             CONTINUE
                  Y(JY) = Y(JY) + ALPHA*TEMP2
                  JX = JX + INCX
                  JY = JY + INCY
  120         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of SSBMV .
*
      END
```

———-

10.4.4 lisp code

— BLAS 2 ssbmv —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun ssbmv (uplo n k alpha a lda x incx beta y incy)
    (declare (type (array single-float (*)) y x a)
             (type (single-float) beta alpha)
             (type (f2cl-lib:integer4) incy incx lda k n)
             (type (string *) uplo))
```

```
(f2cl-lib:with-multi-array-data
    ((uplo character uplo-%data% uplo-%offset%)
     (a single-float a-%data% a-%offset%)
     (x single-float x-%data% x-%offset%)
     (y single-float y-%data% y-%offset%))
  (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (kplus1 0) (kx 0)
         (ky 0) (l 0) (temp1 0.0) (temp2 0.0))
    (declare (type (f2cl-lib:integer4) i info ix iy j jx jy kplus1 kx ky l)
             (type (single-float) temp1 temp2))
    (setf info 0)
    (cond
     ((and (not (lsame uplo "U")) (not (lsame uplo "L"))) (setf info 1))
     ((< n 0) (setf info 2)) ((< k 0) (setf info 3))
     ((< lda (f2cl-lib:int-add k 1)) (setf info 6))
     ((= incx 0) (setf info 8)) ((= incy 0) (setf info 11)))
    (cond
     ((/= info 0)
      (multiple-value-bind (var-0 var-1)
          (xerbla "SSBMV " info)
        (declare (ignore var-0))
        (when var-1 (setf info var-1)))
      (go end_label)))
    (if (or (= n 0) (and (= alpha zero) (= beta one)))
        (go end_label))
    (cond ((> incx 0) (setf kx 1))
          (t
           (setf kx
                   (f2cl-lib:int-sub 1
                                     (f2cl-lib:int-mul
                                      (f2cl-lib:int-sub n 1) incx)))))
    (cond ((> incy 0) (setf ky 1))
          (t
           (setf ky
                   (f2cl-lib:int-sub 1
                                     (f2cl-lib:int-mul
                                      (f2cl-lib:int-sub n 1) incy)))))
    (cond
     ((/= beta one)
      (cond
       ((= incy 1)
        (cond
         ((= beta zero)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%) zero)
             label10)))
         (t
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
```

```
            (tagbody
              (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                      (* beta
                         (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)))
             label20)))))
       (t (setf iy ky)
        (cond
         ((= beta zero)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) zero)
              (setf iy (f2cl-lib:int-add iy incy))
             label30)))
         (t
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (* beta
                         (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)))
              (setf iy (f2cl-lib:int-add iy incy))
             label40))))))))
    (if (= alpha zero)
        (go end_label))
    (cond
     ((lsame uplo "U") (setf kplus1 (f2cl-lib:int-add k 1))
      (cond
       ((and (= incx 1) (= incy 1))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf temp1
                    (* alpha
                       (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
            (setf temp2 zero)
            (setf l (f2cl-lib:int-sub kplus1 j))
            (f2cl-lib:fdo (i
                           (max (the f2cl-lib:integer4 1)
                                (the f2cl-lib:integer4
                                     (f2cl-lib:int-add j
                                                       (f2cl-lib:int-sub
                                                        k))))
                           (f2cl-lib:int-add i 1))
                          ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                        (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                           (* temp1
```

```
                          (f2cl-lib:fref a-%data%
                                         ((f2cl-lib:int-add l i) j)
                                         ((1 lda) (1 *)) a-%offset%))))
            (setf temp2
                    (+ temp2
                       (*
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add l i) j)
                                       ((1 lda) (1 *)) a-%offset%)
                        (f2cl-lib:fref x-%data% (i) ((1 *))
                                       x-%offset%))))
          label50))
        (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                   (* temp1
                      (f2cl-lib:fref a-%data% (kplus1 j)
                                     ((1 lda) (1 *)) a-%offset%))
                   (* alpha temp2)))
     label60)))
  (t (setf jx kx) (setf jy ky)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp1
               (* alpha
                  (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
       (setf temp2 zero)
       (setf ix kx)
       (setf iy ky)
       (setf l (f2cl-lib:int-sub kplus1 j))
       (f2cl-lib:fdo (i
                      (max (the f2cl-lib:integer4 1)
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add j
                                                  (f2cl-lib:int-sub
                                                   k))))
                      (f2cl-lib:int-add i 1))
                     ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                      nil)
         (tagbody
           (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                   (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (* temp1
                         (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-add l i) j)
                                        ((1 lda) (1 *)) a-%offset%))))
           (setf temp2
                   (+ temp2
                      (*
                       (f2cl-lib:fref a-%data%
```

```
                                        ((f2cl-lib:int-add l i) j)
                                        ((1 lda) (1 *)) a-%offset%)
                          (f2cl-lib:fref x-%data% (ix) ((1 *))
                                        x-%offset%))))
              (setf ix (f2cl-lib:int-add ix incx))
              (setf iy (f2cl-lib:int-add iy incy))
             label70))
          (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                  (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                     (* temp1
                        (f2cl-lib:fref a-%data% (kplus1 j)
                                       ((1 lda) (1 *)) a-%offset%))
                     (* alpha temp2)))
          (setf jx (f2cl-lib:int-add jx incx))
          (setf jy (f2cl-lib:int-add jy incy))
          (cond
           ((> j k) (setf kx (f2cl-lib:int-add kx incx))
            (setf ky (f2cl-lib:int-add ky incy))))
         label80)))))
(t
 (cond
  ((and (= incx 1) (= incy 1))
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp1
               (* alpha
                  (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
       (setf temp2 zero)
       (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
               (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                  (* temp1
                     (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *))
                                    a-%offset%))))
       (setf l (f2cl-lib:int-sub 1 j))
       (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                     ((> i
                         (min (the f2cl-lib:integer4 n)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add j k))))
                      nil)
         (tagbody
           (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                   (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                      (* temp1
                         (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-add l i) j)
                                        ((1 lda) (1 *)) a-%offset%))))
           (setf temp2
                   (+ temp2
```

```
                        (*
                         (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-add l i) j)
                                        ((1 lda) (1 *)) a-%offset%)
                         (f2cl-lib:fref x-%data% (i) ((1 *))
                                        x-%offset%))))
           label90))
        (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                   (* alpha temp2)))
      label100)))
    (t (setf jx kx) (setf jy ky)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp1
                 (* alpha
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
         (setf temp2 zero)
         (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                 (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                    (* temp1
                       (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *))
                                      a-%offset%))))
         (setf l (f2cl-lib:int-sub 1 j))
         (setf ix jx)
         (setf iy jy)
         (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                       ((> i
                           (min (the f2cl-lib:integer4 n)
                                (the f2cl-lib:integer4
                                     (f2cl-lib:int-add j k))))
                        nil)
           (tagbody
             (setf ix (f2cl-lib:int-add ix incx))
             (setf iy (f2cl-lib:int-add iy incy))
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (* temp1
                           (f2cl-lib:fref a-%data%
                                          ((f2cl-lib:int-add l i) j)
                                          ((1 lda) (1 *)) a-%offset%))))
             (setf temp2
                     (+ temp2
                        (*
                         (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-add l i) j)
                                        ((1 lda) (1 *)) a-%offset%)
                         (f2cl-lib:fref x-%data% (ix) ((1 *))
                                        x-%offset%))))
```

```
                 label110))
              (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                      (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                         (* alpha temp2)))
              (setf jx (f2cl-lib:int-add jx incx))
              (setf jy (f2cl-lib:int-add jy incy))
             label120))))))
       (go end_label)
      end_label
       (return (values nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

10.5 sspmv - matrix-vector $\alpha Ax + \beta y$

10.5.1 Axiom unit tests

— **sspmv.input** —

```
)set break resume
)sys rm -f sspmv.output
)spool sspmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

10.5.2 Axiom help page

— **sspmv.help** —

```
==========================================================================
sspmv examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine sspmv (UPLO, N, ALPHA, AP, X, INCX, BETA, Y, INCY)
           SSPMV

Function/Subroutine Documentation
   subroutine sspmv (character UPLO, integer N, real ALPHA, real, dimension(*)
       AP, real, dimension(*) X, integer INCX, real BETA, real, dimension(*)
       Y, integer INCY)
       SSPMV

Purpose:
  SSPMV  performs the matrix-vector operation

  y := alpha*A*x + beta*y,

  where alpha and beta are scalars, x and y are n element vectors and
  A is an n by n symmetric matrix, supplied in packed form.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the upper or lower
   triangular part of the matrix A is supplied in the packed
   array AP as follows:

   UPLO = 'U' or 'u'   The upper triangular part of A is supplied in AP.

   UPLO = 'L' or 'l'   The lower triangular part of A is supplied in AP.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
   N must be at least zero.

 ALPHA
   ALPHA is REAL
   On entry, ALPHA specifies the scalar alpha.

 AP
   AP is REAL array of DIMENSION at least ( ( n*( n + 1 ) )/2 ).

   Before entry with UPLO = 'U' or 'u', the array AP must
   contain the upper triangular part of the symmetric matrix
   packed sequentially, column by column, so that AP( 1 )
   contains a( 1, 1 ), AP( 2 ) and AP( 3 ) contain a( 1, 2 )
   and a( 2, 2 ) respectively, and so on.
   Before entry with UPLO = 'L' or 'l', the array AP must
   contain the lower triangular part of the symmetric matrix
```

```
  packed sequentially, column by column, so that AP( 1 )
  contains a( 1, 1 ), AP( 2 ) and AP( 3 ) contain a( 2, 1 )
  and a( 3, 1 ) respectively, and so on.

X
  X is REAL array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the n
  element vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.

BETA
  BETA is REAL
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.

Y
  Y is REAL array of dimension at least ( 1 + ( n - 1 )*abs( INCY ) ).
  Before entry, the incremented array Y must contain the n
  element vector y. On exit, Y is overwritten by the updated vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of Y.
  INCY must not be zero.

The vector and matrix arguments are not referenced when N = 0, or M = 0
```

10.5.3 fortran code

— sspmv.f —

```
*  ==================================================================
      SUBROUTINE SSPMV(UPLO,N,ALPHA,AP,X,INCX,BETA,Y,INCY)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL ALPHA,BETA
```

```
      INTEGER INCX,INCY,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      REAL AP(*),X(*),Y(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL ONE,ZERO
      PARAMETER (ONE=1.0E+0,ZERO=0.0E+0)
*     ..
*     .. Local Scalars ..
      REAL TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,K,KK,KX,KY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 6
      ELSE IF (INCY.EQ.0) THEN
          INFO = 9
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('SSPMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     Set up the start points in  X  and  Y.
*
      IF (INCX.GT.0) THEN
          KX = 1
```

```
      ELSE
          KX = 1 - (N-1)*INCX
      END IF
      IF (INCY.GT.0) THEN
          KY = 1
      ELSE
          KY = 1 - (N-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of the array AP
*     are accessed sequentially with one pass through AP.
*
*     First form  y := beta*y.
*
      IF (BETA.NE.ONE) THEN
          IF (INCY.EQ.1) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 10 I = 1,N
                      Y(I) = ZERO
   10             CONTINUE
              ELSE
                  DO 20 I = 1,N
                      Y(I) = BETA*Y(I)
   20             CONTINUE
              END IF
          ELSE
              IY = KY
              IF (BETA.EQ.ZERO) THEN
                  DO 30 I = 1,N
                      Y(IY) = ZERO
                      IY = IY + INCY
   30             CONTINUE
              ELSE
                  DO 40 I = 1,N
                      Y(IY) = BETA*Y(IY)
                      IY = IY + INCY
   40             CONTINUE
              END IF
          END IF
      END IF
      IF (ALPHA.EQ.ZERO) RETURN
      KK = 1
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  y  when AP contains the upper triangle.
*
          IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
              DO 60 J = 1,N
                  TEMP1 = ALPHA*X(J)
                  TEMP2 = ZERO
```

```
                K = KK
                DO 50 I = 1,J - 1
                    Y(I) = Y(I) + TEMP1*AP(K)
                    TEMP2 = TEMP2 + AP(K)*X(I)
                    K = K + 1
   50           CONTINUE
                Y(J) = Y(J) + TEMP1*AP(KK+J-1) + ALPHA*TEMP2
                KK = KK + J
   60       CONTINUE
        ELSE
            JX = KX
            JY = KY
            DO 80 J = 1,N
                TEMP1 = ALPHA*X(JX)
                TEMP2 = ZERO
                IX = KX
                IY = KY
                DO 70 K = KK,KK + J - 2
                    Y(IY) = Y(IY) + TEMP1*AP(K)
                    TEMP2 = TEMP2 + AP(K)*X(IX)
                    IX = IX + INCX
                    IY = IY + INCY
   70           CONTINUE
                Y(JY) = Y(JY) + TEMP1*AP(KK+J-1) + ALPHA*TEMP2
                JX = JX + INCX
                JY = JY + INCY
                KK = KK + J
   80       CONTINUE
        END IF
      ELSE
*
*       Form  y  when AP contains the lower triangle.
*
         IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
            DO 100 J = 1,N
                TEMP1 = ALPHA*X(J)
                TEMP2 = ZERO
                Y(J) = Y(J) + TEMP1*AP(KK)
                K = KK + 1
                DO 90 I = J + 1,N
                    Y(I) = Y(I) + TEMP1*AP(K)
                    TEMP2 = TEMP2 + AP(K)*X(I)
                    K = K + 1
   90           CONTINUE
                Y(J) = Y(J) + ALPHA*TEMP2
                KK = KK + (N-J+1)
  100       CONTINUE
        ELSE
            JX = KX
            JY = KY
```

```
            DO 120 J = 1,N
                TEMP1 = ALPHA*X(JX)
                TEMP2 = ZERO
                Y(JY) = Y(JY) + TEMP1*AP(KK)
                IX = JX
                IY = JY
                DO 110 K = KK + 1,KK + N - J
                    IX = IX + INCX
                    IY = IY + INCY
                    Y(IY) = Y(IY) + TEMP1*AP(K)
                    TEMP2 = TEMP2 + AP(K)*X(IX)
 110            CONTINUE
                Y(JY) = Y(JY) + ALPHA*TEMP2
                JX = JX + INCX
                JY = JY + INCY
                KK = KK + (N-J+1)
 120        CONTINUE
        END IF
    END IF
*
    RETURN
*
*   End of SSPMV .
*
    END
```

———-

10.5.4 lisp code

— BLAS 2 sspmv —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun sspmv (uplo n alpha ap x incx beta y incy)
    (declare (type (array single-float (*)) y x ap)
             (type (single-float) beta alpha)
             (type (f2cl-lib:integer4) incy incx n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (ap single-float ap-%data% ap-%offset%)
         (x single-float x-%data% x-%offset%)
         (y single-float y-%data% y-%offset%))
      (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (k 0) (kk 0)
```

```
     (kx 0) (ky 0) (temp1 0.0) (temp2 0.0))
(declare (type (f2cl-lib:integer4) i info ix iy j jx jy k kk kx ky)
         (type (single-float) temp1 temp2))
(setf info 0)
(cond
 ((and (not (lsame uplo "U")) (not (lsame uplo "L"))) (setf info 1))
 ((< n 0) (setf info 2)) ((= incx 0) (setf info 6))
 ((= incy 0) (setf info 9)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "SSPMV " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= n 0) (and (= alpha zero) (= beta one)))
    (go end_label))
(cond ((> incx 0) (setf kx 1))
      (t
       (setf kx
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incx)))))
(cond ((> incy 0) (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incy)))))
(cond
 ((/= beta one)
  (cond
   ((= incy 1)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%) zero)
         label10)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                  (* beta
                     (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)))
         label20)))))
   (t (setf iy ky)
    (cond
```

```
      ((= beta zero)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) zero)
           (setf iy (f2cl-lib:int-add iy incy))
          label30)))
      (t
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                   (* beta
                      (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)))
           (setf iy (f2cl-lib:int-add iy incy))
          label40))))))))
(if (= alpha zero)
    (go end_label))
(setf kk 1)
(cond
 ((lsame uplo "U")
  (cond
   ((and (= incx 1) (= incy 1))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp1
                (* alpha
                   (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
        (setf temp2 zero)
        (setf k kk)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                       (* temp1
                          (f2cl-lib:fref ap-%data% (k) ((1 *))
                                         ap-%offset%))))
            (setf temp2
                    (+ temp2
                       (*
                        (f2cl-lib:fref ap-%data% (k) ((1 *))
                                       ap-%offset%)
                        (f2cl-lib:fref x-%data% (i) ((1 *))
                                       x-%offset%))))
            (setf k (f2cl-lib:int-add k 1))
           label50))
        (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
```

```
                (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                   (* temp1
                      (f2cl-lib:fref ap-%data%
                                     ((f2cl-lib:int-sub
                                       (f2cl-lib:int-add kk j) 1))
                                     ((1 *)) ap-%offset%))
                   (* alpha temp2)))
        (setf kk (f2cl-lib:int-add kk j))
       label60)))
    (t (setf jx kx) (setf jy ky)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp1
                 (* alpha
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
         (setf temp2 zero)
         (setf ix kx)
         (setf iy ky)
         (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                       ((> k
                           (f2cl-lib:int-add kk j (f2cl-lib:int-sub 2)))
                        nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (* temp1
                           (f2cl-lib:fref ap-%data% (k) ((1 *))
                                          ap-%offset%))))
             (setf temp2
                     (+ temp2
                        (*
                         (f2cl-lib:fref ap-%data% (k) ((1 *))
                                        ap-%offset%)
                         (f2cl-lib:fref x-%data% (ix) ((1 *))
                                        x-%offset%))))
             (setf ix (f2cl-lib:int-add ix incx))
             (setf iy (f2cl-lib:int-add iy incy))
            label70))
         (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                 (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                    (* temp1
                       (f2cl-lib:fref ap-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add kk j) 1))
                                      ((1 *)) ap-%offset%))
                    (* alpha temp2)))
         (setf jx (f2cl-lib:int-add jx incx))
         (setf jy (f2cl-lib:int-add jy incy))
         (setf kk (f2cl-lib:int-add kk j))
```

```
      label80)))))
(t
 (cond
  ((and (= incx 1) (= incy 1))
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp1
               (* alpha
                  (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
       (setf temp2 zero)
       (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
               (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                  (* temp1
                     (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                    ap-%offset%))))
       (setf k (f2cl-lib:int-add kk 1))
       (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                   (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                      (* temp1
                         (f2cl-lib:fref ap-%data% (k) ((1 *))
                                        ap-%offset%))))
           (setf temp2
                   (+ temp2
                      (*
                       (f2cl-lib:fref ap-%data% (k) ((1 *))
                                      ap-%offset%)
                       (f2cl-lib:fref x-%data% (i) ((1 *))
                                      x-%offset%))))
           (setf k (f2cl-lib:int-add k 1))
          label90))
       (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
               (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                  (* alpha temp2)))
       (setf kk
               (f2cl-lib:int-add kk
                                 (f2cl-lib:int-add
                                  (f2cl-lib:int-sub n j) 1)))
      label100)))
  (t (setf jx kx) (setf jy ky)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp1
               (* alpha
                  (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
       (setf temp2 zero)
```

```
          (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                  (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                     (* temp1
                        (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                       ap-%offset%))))
          (setf ix jx)
          (setf iy jy)
          (f2cl-lib:fdo (k (f2cl-lib:int-add kk 1) (f2cl-lib:int-add k 1))
                        ((> k
                            (f2cl-lib:int-add kk n (f2cl-lib:int-sub j)))
                         nil)
            (tagbody
              (setf ix (f2cl-lib:int-add ix incx))
              (setf iy (f2cl-lib:int-add iy incy))
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (* temp1
                            (f2cl-lib:fref ap-%data% (k) ((1 *))
                                           ap-%offset%))))
              (setf temp2
                      (+ temp2
                         (*
                          (f2cl-lib:fref ap-%data% (k) ((1 *))
                                         ap-%offset%)
                          (f2cl-lib:fref x-%data% (ix) ((1 *))
                                         x-%offset%))))
             label110))
          (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                  (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                     (* alpha temp2)))
          (setf jx (f2cl-lib:int-add jx incx))
          (setf jy (f2cl-lib:int-add jy incy))
          (setf kk
                  (f2cl-lib:int-add kk
                                    (f2cl-lib:int-add
                                     (f2cl-lib:int-sub n j) 1)))
        label120))))))
 (go end_label)
end_label
 (return (values nil nil nil nil nil nil nil nil nil))))))
```

———-

10.6 sspr - rank 1 $\alpha x x^T + A$

10.6.1 Axiom unit tests

— sspr.input —

```
)set break resume
)sys rm -f sspr.output
)spool sspr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

10.6.2 Axiom help page

— sspr.help —

```
====================================================================
sspr examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sspr (UPLO, N, ALPHA, X, INCX, AP)
           SSPR

Function/Subroutine Documentation
   subroutine sspr (character UPLO, integer N, real ALPHA, real, dimension(*)
       X, integer INCX, real, dimension(*) AP)
       SSPR

Purpose:
  SSPR    performs the symmetric rank 1 operation

    A := alpha*x*x**T + A,

  where alpha is a real scalar, x is an n element vector and A is an
```

```
  n by n symmetric matrix, supplied in packed form.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the upper or lower
   triangular part of the matrix A is supplied in the packed
   array AP as follows:

   UPLO = 'U' or 'u'   The upper triangular part of A is supplied in AP.

   UPLO = 'L' or 'l'   The lower triangular part of A is supplied in AP.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
   N must be at least zero.

 ALPHA
   ALPHA is REAL
   On entry, ALPHA specifies the scalar alpha.

 X
   X is REAL array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
   Before entry, the incremented array X must contain the n
   element vector x.

 INCX
   INCX is INTEGER
   On entry, INCX specifies the increment for the elements of X.
   INCX must not be zero.

 AP
   AP is REAL array of DIMENSION at least ( ( n*( n + 1 ) )/2 ).

   Before entry with  UPLO = 'U' or 'u', the array AP must
   contain the upper triangular part of the symmetric matrix
   packed sequentially, column by column, so that AP( 1 )
   contains a( 1, 1 ), AP( 2 ) and AP( 3 ) contain a( 1, 2 )
   and a( 2, 2 ) respectively, and so on. On exit, the array
   AP is overwritten by the upper triangular part of the
   updated matrix.

   Before entry with UPLO = 'L' or 'l', the array AP must
   contain the lower triangular part of the symmetric matrix
   packed sequentially, column by column, so that AP( 1 )
   contains a( 1, 1 ), AP( 2 ) and AP( 3 ) contain a( 2, 1 )
   and a( 3, 1 ) respectively, and so on. On exit, the array
   AP is overwritten by the lower triangular part of the
```

updated matrix.

———-

10.6.3 fortran code

— **sspr.f** —

```
*  ===================================================================
      SUBROUTINE SSPR(UPLO,N,ALPHA,X,INCX,AP)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL ALPHA
      INTEGER INCX,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      REAL AP(*),X(*)
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      REAL ZERO
      PARAMETER (ZERO=0.0E+0)
*     ..
*     .. Local Scalars ..
      REAL TEMP
      INTEGER I,INFO,IX,J,JX,K,KK,KX
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
```

```
         INFO = 1
      ELSE IF (N.LT.0) THEN
         INFO = 2
      ELSE IF (INCX.EQ.0) THEN
         INFO = 5
      END IF
      IF (INFO.NE.0) THEN
         CALL XERBLA('SSPR  ',INFO)
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (ALPHA.EQ.ZERO)) RETURN
*
*     Set the start point in X if the increment is not unity.
*
      IF (INCX.LE.0) THEN
         KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
         KX = 1
      END IF
*
*     Start the operations. In this version the elements of the array AP
*     are accessed sequentially with one pass through AP.
*
      KK = 1
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  A  when upper triangle is stored in AP.
*
         IF (INCX.EQ.1) THEN
            DO 20 J = 1,N
               IF (X(J).NE.ZERO) THEN
                  TEMP = ALPHA*X(J)
                  K = KK
                  DO 10 I = 1,J
                     AP(K) = AP(K) + X(I)*TEMP
                     K = K + 1
   10             CONTINUE
               END IF
               KK = KK + J
   20       CONTINUE
         ELSE
            JX = KX
            DO 40 J = 1,N
               IF (X(JX).NE.ZERO) THEN
                  TEMP = ALPHA*X(JX)
                  IX = KX
                  DO 30 K = KK,KK + J - 1
```

```
                        AP(K) = AP(K) + X(IX)*TEMP
                        IX = IX + INCX
   30                CONTINUE
                END IF
                JX = JX + INCX
                KK = KK + J
   40         CONTINUE
         END IF
      ELSE
*
*        Form  A  when lower triangle is stored in AP.
*
         IF (INCX.EQ.1) THEN
             DO 60 J = 1,N
                 IF (X(J).NE.ZERO) THEN
                     TEMP = ALPHA*X(J)
                     K = KK
                     DO 50 I = J,N
                         AP(K) = AP(K) + X(I)*TEMP
                         K = K + 1
   50                CONTINUE
                END IF
                KK = KK + N - J + 1
   60         CONTINUE
         ELSE
             JX = KX
             DO 80 J = 1,N
                 IF (X(JX).NE.ZERO) THEN
                     TEMP = ALPHA*X(JX)
                     IX = JX
                     DO 70 K = KK,KK + N - J
                         AP(K) = AP(K) + X(IX)*TEMP
                         IX = IX + INCX
   70                CONTINUE
                END IF
                JX = JX + INCX
                KK = KK + N - J + 1
   80         CONTINUE
         END IF
      END IF
*
      RETURN
*
*     End of SSPR  .
*
      END
```

10.6.4 lisp code

— BLAS 2 sspr —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun sspr (uplo n alpha x incx ap)
    (declare (type (array single-float (*)) ap x)
             (type (single-float) alpha)
             (type (f2cl-lib:integer4) incx n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (x single-float x-%data% x-%offset%)
         (ap single-float ap-%data% ap-%offset%))
      (prog ((i 0) (info 0) (ix 0) (j 0) (jx 0) (k 0) (kk 0) (kx 0) (temp 0.0))
        (declare (type (f2cl-lib:integer4) i info ix j jx k kk kx)
                 (type (single-float) temp))
        (setf info 0)
        (cond
         ((and (not (lsame uplo "U")) (not (lsame uplo "L"))) (setf info 1))
         ((< n 0) (setf info 2)) ((= incx 0) (setf info 5)))
        (cond
         ((/= info 0)
          (multiple-value-bind (var-0 var-1)
              (xerbla "SSPR  " info)
            (declare (ignore var-0))
            (when var-1 (setf info var-1)))
          (go end_label)))
        (if (or (= n 0) (= alpha zero))
            (go end_label))
        (cond
         ((<= incx 0)
          (setf kx
                  (f2cl-lib:int-sub 1
                                    (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                      incx))))
         ((/= incx 1) (setf kx 1)))
        (setf kk 1)
        (cond
         ((lsame uplo "U")
          (cond
           ((= incx 1)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (cond
                 ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
```

```
            (setf temp
                    (* alpha
                       (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
            (setf k kk)
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i j) nil)
              (tagbody
                (setf (f2cl-lib:fref ap-%data% (k) ((1 *)) ap-%offset%)
                        (+
                         (f2cl-lib:fref ap-%data% (k) ((1 *))
                                        ap-%offset%)
                         (*
                          (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                          temp)))
                (setf k (f2cl-lib:int-add k 1))
               label10))))
          (setf kk (f2cl-lib:int-add kk j))
         label20)))
      (t (setf jx kx)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (cond
            ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
             (setf temp
                     (* alpha
                        (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
             (setf ix kx)
             (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                           ((> k
                               (f2cl-lib:int-add kk j
                                                 (f2cl-lib:int-sub 1)))
                            nil)
               (tagbody
                 (setf (f2cl-lib:fref ap-%data% (k) ((1 *)) ap-%offset%)
                         (+
                          (f2cl-lib:fref ap-%data% (k) ((1 *))
                                         ap-%offset%)
                          (*
                           (f2cl-lib:fref x-%data% (ix) ((1 *))
                                          x-%offset%)
                           temp)))
                 (setf ix (f2cl-lib:int-add ix incx))
                label30))))
            (setf jx (f2cl-lib:int-add jx incx))
            (setf kk (f2cl-lib:int-add kk j))
           label40)))))
    (t
     (cond
      ((= incx 1)
```

```
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (cond
     ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
      (setf temp
              (* alpha
                 (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
      (setf k kk)
      (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref ap-%data% (k) ((1 *)) ap-%offset%)
                  (+
                   (f2cl-lib:fref ap-%data% (k) ((1 *))
                                  ap-%offset%)
                   (*
                    (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                    temp)))
          (setf k (f2cl-lib:int-add k 1))
         label50))))
    (setf kk
            (f2cl-lib:int-add
             (f2cl-lib:int-sub (f2cl-lib:int-add kk n) j) 1))
   label60)))
(t (setf jx kx)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
      ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
       (setf temp
               (* alpha
                  (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
       (setf ix jx)
       (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                     ((> k
                         (f2cl-lib:int-add kk n
                                           (f2cl-lib:int-sub j)))
                      nil)
         (tagbody
           (setf (f2cl-lib:fref ap-%data% (k) ((1 *)) ap-%offset%)
                   (+
                    (f2cl-lib:fref ap-%data% (k) ((1 *))
                                   ap-%offset%)
                    (*
                     (f2cl-lib:fref x-%data% (ix) ((1 *))
                                    x-%offset%)
                     temp)))
           (setf ix (f2cl-lib:int-add ix incx))
```

```
                    label70))))
                (setf jx (f2cl-lib:int-add jx incx))
                (setf kk
                        (f2cl-lib:int-add
                         (f2cl-lib:int-sub (f2cl-lib:int-add kk n) j) 1))
              label80))))))
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil))))))
```

———-

10.7 sspr2 - rank 2 $\alpha xy^T + \alpha yx^T + A$

10.7.1 Axiom unit tests

— **sspr2.input** —

```
)set break resume
)sys rm -f sspr2.output
)spool sspr2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

10.7.2 Axiom help page

— **sspr2.help** —

```
=================================================================
sspr2 examples
=================================================================

=================================================================
Man Page Details
=================================================================

SYNOPSIS
```

```
   Functions/Subroutines
       subroutine sspr2 (UPLO, N, ALPHA, X, INCX, Y, INCY, AP)
           SSPR2

Function/Subroutine Documentation
   subroutine sspr2 (character UPLO, integer N, real ALPHA, real, dimension(*)
       X, integer INCX, real, dimension(*) Y, integer INCY, real, dimension(*)
       AP)
       SSPR2

Purpose:
 SSPR2  performs the symmetric rank 2 operation

    A := alpha*x*y**T + alpha*y*x**T + A,

 where alpha is a scalar, x and y are n element vectors and A is an
 n by n symmetric matrix, supplied in packed form.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the upper or lower
   triangular part of the matrix A is supplied in the packed
   array AP as follows:

   UPLO = 'U' or 'u'   The upper triangular part of A is supplied in AP.

   UPLO = 'L' or 'l'   The lower triangular part of A is supplied in AP.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
   N must be at least zero.

 ALPHA
   ALPHA is REAL
   On entry, ALPHA specifies the scalar alpha.

 X
   X is REAL array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
   Before entry, the incremented array X must contain the n
   element vector x.

 INCX
   INCX is INTEGER
   On entry, INCX specifies the increment for the elements of X.
   INCX must not be zero.

 Y
```

```
  Y is REAL array of dimension at least ( 1 + ( n - 1 )*abs( INCY ) ).
  Before entry, the incremented array Y must contain the n
  element vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of Y.
  INCY must not be zero.

AP
  AP is REAL array of DIMENSION at least ( ( n*( n + 1 ) )/2 ).

  Before entry with  UPLO = 'U' or 'u', the array AP must
  contain the upper triangular part of the symmetric matrix
  packed sequentially, column by column, so that AP( 1 )
  contains a( 1, 1 ), AP( 2 ) and AP( 3 ) contain a( 1, 2 )
  and a( 2, 2 ) respectively, and so on. On exit, the array
  AP is overwritten by the upper triangular part of the
  updated matrix.

  Before entry with UPLO = 'L' or 'l', the array AP must
  contain the lower triangular part of the symmetric matrix
  packed sequentially, column by column, so that AP( 1 )
  contains a( 1, 1 ), AP( 2 ) and AP( 3 ) contain a( 2, 1 )
  and a( 3, 1 ) respectively, and so on. On exit, the array
  AP is overwritten by the lower triangular part of the
  updated matrix.
```

10.7.3 fortran code

— sspr2.f —

```
*  =====================================================================
      SUBROUTINE SSPR2(UPLO,N,ALPHA,X,INCX,Y,INCY,AP)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL ALPHA
      INTEGER INCX,INCY,N
      CHARACTER UPLO
*     ..
```

```
*     .. Array Arguments ..
      REAL AP(*),X(*),Y(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL ZERO
      PARAMETER (ZERO=0.0E+0)
*     ..
*     .. Local Scalars ..
      REAL TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,K,KK,KX,KY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      ELSE IF (INCY.EQ.0) THEN
          INFO = 7
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('SSPR2 ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (ALPHA.EQ.ZERO)) RETURN
*
*     Set up the start points in X and Y if the increments are not both
*     unity.
*
      IF ((INCX.NE.1) .OR. (INCY.NE.1)) THEN
          IF (INCX.GT.0) THEN
              KX = 1
          ELSE
```

```
            KX = 1 - (N-1)*INCX
        END IF
        IF (INCY.GT.0) THEN
            KY = 1
        ELSE
            KY = 1 - (N-1)*INCY
        END IF
        JX = KX
        JY = KY
    END IF
*
*     Start the operations. In this version the elements of the array AP
*     are accessed sequentially with one pass through AP.
*
      KK = 1
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  A  when upper triangle is stored in AP.
*
        IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
            DO 20 J = 1,N
                IF ((X(J).NE.ZERO) .OR. (Y(J).NE.ZERO)) THEN
                    TEMP1 = ALPHA*Y(J)
                    TEMP2 = ALPHA*X(J)
                    K = KK
                    DO 10 I = 1,J
                        AP(K) = AP(K) + X(I)*TEMP1 + Y(I)*TEMP2
                        K = K + 1
   10               CONTINUE
                END IF
                KK = KK + J
   20       CONTINUE
        ELSE
            DO 40 J = 1,N
                IF ((X(JX).NE.ZERO) .OR. (Y(JY).NE.ZERO)) THEN
                    TEMP1 = ALPHA*Y(JY)
                    TEMP2 = ALPHA*X(JX)
                    IX = KX
                    IY = KY
                    DO 30 K = KK,KK + J - 1
                        AP(K) = AP(K) + X(IX)*TEMP1 + Y(IY)*TEMP2
                        IX = IX + INCX
                        IY = IY + INCY
   30               CONTINUE
                END IF
                JX = JX + INCX
                JY = JY + INCY
                KK = KK + J
   40       CONTINUE
        END IF
```

```
      ELSE
*
*        Form  A  when lower triangle is stored in AP.
*
          IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
              DO 60 J = 1,N
                  IF ((X(J).NE.ZERO) .OR. (Y(J).NE.ZERO)) THEN
                      TEMP1 = ALPHA*Y(J)
                      TEMP2 = ALPHA*X(J)
                      K = KK
                      DO 50 I = J,N
                          AP(K) = AP(K) + X(I)*TEMP1 + Y(I)*TEMP2
                          K = K + 1
   50                 CONTINUE
                  END IF
                  KK = KK + N - J + 1
   60         CONTINUE
          ELSE
              DO 80 J = 1,N
                  IF ((X(JX).NE.ZERO) .OR. (Y(JY).NE.ZERO)) THEN
                      TEMP1 = ALPHA*Y(JY)
                      TEMP2 = ALPHA*X(JX)
                      IX = JX
                      IY = JY
                      DO 70 K = KK,KK + N - J
                          AP(K) = AP(K) + X(IX)*TEMP1 + Y(IY)*TEMP2
                          IX = IX + INCX
                          IY = IY + INCY
   70                 CONTINUE
                  END IF
                  JX = JX + INCX
                  JY = JY + INCY
                  KK = KK + N - J + 1
   80         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of SSPR2 .
*
      END
```

———-

10.7.4 lisp code

— BLAS 2 sspr2 —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun sspr2 (uplo n alpha x incx y incy ap)
    (declare (type (array single-float (*)) ap y x)
             (type (single-float) alpha)
             (type (f2cl-lib:integer4) incy incx n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (x single-float x-%data% x-%offset%)
         (y single-float y-%data% y-%offset%)
         (ap single-float ap-%data% ap-%offset%))
      (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (k 0) (kk 0)
             (kx 0) (ky 0) (temp1 0.0) (temp2 0.0))
        (declare (type (f2cl-lib:integer4) i info ix iy j jx jy k kk kx ky)
                 (type (single-float) temp1 temp2))
        (setf info 0)
        (cond
         ((and (not (lsame uplo "U")) (not (lsame uplo "L"))) (setf info 1))
         ((< n 0) (setf info 2)) ((= incx 0) (setf info 5))
         ((= incy 0) (setf info 7)))
        (cond
         ((/= info 0)
          (multiple-value-bind (var-0 var-1)
              (xerbla "SSPR2 " info)
            (declare (ignore var-0))
            (when var-1 (setf info var-1)))
          (go end_label)))
        (if (or (= n 0) (= alpha zero))
            (go end_label))
        (cond
         ((or (/= incx 1) (/= incy 1))
          (cond ((> incx 0) (setf kx 1))
                (t
                 (setf kx
                         (f2cl-lib:int-sub 1
                                           (f2cl-lib:int-mul
                                            (f2cl-lib:int-sub n 1) incx)))))
          (cond ((> incy 0) (setf ky 1))
                (t
                 (setf ky
                         (f2cl-lib:int-sub 1
                                           (f2cl-lib:int-mul
                                            (f2cl-lib:int-sub n 1) incy)))))
          (setf jx kx) (setf jy ky)))
        (setf kk 1)
        (cond
```

```
((lsame uplo "U")
 (cond
  ((and (= incx 1) (= incy 1))
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((or (/= (f2cl-lib:fref x (j) ((1 *))) zero)
             (/= (f2cl-lib:fref y (j) ((1 *))) zero))
         (setf temp1
                 (* alpha
                    (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)))
         (setf temp2
                 (* alpha
                    (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
         (setf k kk)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i j) nil)
           (tagbody
             (setf (f2cl-lib:fref ap-%data% (k) ((1 *)) ap-%offset%)
                     (+
                      (f2cl-lib:fref ap-%data% (k) ((1 *))
                                     ap-%offset%)
                      (*
                       (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                       temp1)
                      (*
                       (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                       temp2)))
             (setf k (f2cl-lib:int-add k 1))
            label10))))
       (setf kk (f2cl-lib:int-add kk j))
      label20)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((or (/= (f2cl-lib:fref x (jx) ((1 *))) zero)
             (/= (f2cl-lib:fref y (jy) ((1 *))) zero))
         (setf temp1
                 (* alpha
                    (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)))
         (setf temp2
                 (* alpha
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
         (setf ix kx) (setf iy ky)
         (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                       ((> k
                           (f2cl-lib:int-add kk j
```

```
                                                (f2cl-lib:int-sub 1)))
                          nil)
                (tagbody
                  (setf (f2cl-lib:fref ap-%data% (k) ((1 *)) ap-%offset%)
                          (+
                           (f2cl-lib:fref ap-%data% (k) ((1 *))
                                          ap-%offset%)
                           (*
                            (f2cl-lib:fref x-%data% (ix) ((1 *))
                                           x-%offset%)
                            temp1)
                           (*
                            (f2cl-lib:fref y-%data% (iy) ((1 *))
                                           y-%offset%)
                            temp2)))
                  (setf ix (f2cl-lib:int-add ix incx))
                  (setf iy (f2cl-lib:int-add iy incy))
                 label30))))
        (setf jx (f2cl-lib:int-add jx incx))
        (setf jy (f2cl-lib:int-add jy incy))
        (setf kk (f2cl-lib:int-add kk j))
       label40)))))
(t
 (cond
  ((and (= incx 1) (= incy 1))
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((or (/= (f2cl-lib:fref x (j) ((1 *))) zero)
             (/= (f2cl-lib:fref y (j) ((1 *))) zero))
         (setf temp1
                 (* alpha
                    (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)))
         (setf temp2
                 (* alpha
                    (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
         (setf k kk)
         (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref ap-%data% (k) ((1 *)) ap-%offset%)
                     (+
                      (f2cl-lib:fref ap-%data% (k) ((1 *))
                                     ap-%offset%)
                      (*
                       (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                       temp1)
                      (*
                       (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
```

```
                          temp2)))
                 (setf k (f2cl-lib:int-add k 1))
                label50))))
          (setf kk
                  (f2cl-lib:int-add
                   (f2cl-lib:int-sub (f2cl-lib:int-add kk n) j) 1))
         label60)))
      (t
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (cond
            ((or (/= (f2cl-lib:fref x (jx) ((1 *))) zero)
                 (/= (f2cl-lib:fref y (jy) ((1 *))) zero))
             (setf temp1
                     (* alpha
                        (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)))
             (setf temp2
                     (* alpha
                        (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
             (setf ix jx) (setf iy jy)
             (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                           ((> k
                               (f2cl-lib:int-add kk n
                                                 (f2cl-lib:int-sub j)))
                            nil)
               (tagbody
                 (setf (f2cl-lib:fref ap-%data% (k) ((1 *)) ap-%offset%)
                         (+
                          (f2cl-lib:fref ap-%data% (k) ((1 *))
                                         ap-%offset%)
                          (*
                           (f2cl-lib:fref x-%data% (ix) ((1 *))
                                          x-%offset%)
                           temp1)
                          (*
                           (f2cl-lib:fref y-%data% (iy) ((1 *))
                                          y-%offset%)
                           temp2)))
                 (setf ix (f2cl-lib:int-add ix incx))
                 (setf iy (f2cl-lib:int-add iy incy))
                label70))))
          (setf jx (f2cl-lib:int-add jx incx))
          (setf jy (f2cl-lib:int-add jy incy))
          (setf kk
                  (f2cl-lib:int-add
                   (f2cl-lib:int-sub (f2cl-lib:int-add kk n) j) 1))
         label80))))))
 (go end_label)
end_label
```

```
        (return (values nil nil nil nil nil nil nil nil))))))
```

———-

10.8 ssymv - matrix-vector $\alpha Ax + \beta y$

10.8.1 Axiom unit tests

— **ssymv.input** —

```
)set break resume
)sys rm -f ssymv.output
)spool ssymv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

10.8.2 Axiom help page

— **ssymv.help** —

```
======================================================================
ssymv examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       ssymv.f -

SYNOPSIS
   Functions/Subroutines
       subroutine ssymv (UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)
           SSYMV

Function/Subroutine Documentation
```

```
   subroutine ssymv (character UPLO, integer N, real ALPHA, real,
       dimension(lda,*) A, integer LDA, real, dimension(*) X, integer INCX,
       real BETA, real, dimension(*) Y, integer INCY)
       SSYMV

Purpose:
   SSYMV  performs the matrix-vector  operation

       y := alpha*A*x + beta*y,

   where alpha and beta are scalars, x and y are n element vectors and
   A is an n by n symmetric matrix.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the upper or lower
   triangular part of the array A is to be referenced as follows:

   UPLO = 'U' or 'u'   Only the upper triangular part of A
   is to be referenced.

   UPLO = 'L' or 'l'   Only the lower triangular part of A
   is to be referenced.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
   N must be at least zero.

 ALPHA
   ALPHA is REAL
   On entry, ALPHA specifies the scalar alpha.

 A
   A is REAL array of DIMENSION ( LDA, n ).

   Before entry with  UPLO = 'U' or 'u', the leading n by n
   upper triangular part of the array A must contain the upper
   triangular part of the symmetric matrix and the strictly
   lower triangular part of A is not referenced.
   Before entry with UPLO = 'L' or 'l', the leading n by n
   lower triangular part of the array A must contain the lower
   triangular part of the symmetric matrix and the strictly
   upper triangular part of A is not referenced.

 LDA
   LDA is INTEGER
   On entry, LDA specifies the first dimension of A as declared
```

```
  in the calling (sub) program. LDA must be at least max( 1, n ).

 X
  X is REAL array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the n
  element vector x.

 INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.

 BETA
  BETA is REAL
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.

 Y
  Y is REAL array of dimension at least ( 1 + ( n - 1 )*abs( INCY ) ).
  Before entry, the incremented array Y must contain the n
  element vector y. On exit, Y is overwritten by the updated vector y.

 INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of Y.
  INCY must not be zero.

  The vector and matrix arguments are not referenced when N = 0, or M = 0
```

10.8.3 fortran code

— ssymv.f —

```
*  =====================================================================
      SUBROUTINE SSYMV(UPLO,N,ALPHA,A,LDA,X,INCX,BETA,Y,INCY)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL ALPHA,BETA
      INTEGER INCX,INCY,LDA,N
      CHARACTER UPLO
```

```
*     ..
*     .. Array Arguments ..
      REAL A(LDA,*),X(*),Y(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL ONE,ZERO
      PARAMETER (ONE=1.0E+0,ZERO=0.0E+0)
*     ..
*     .. Local Scalars ..
      REAL TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,KX,KY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 5
      ELSE IF (INCX.EQ.0) THEN
          INFO = 7
      ELSE IF (INCY.EQ.0) THEN
          INFO = 10
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('SSYMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     Set up the start points in  X  and  Y.
```

```
*
      IF (INCX.GT.0) THEN
          KX = 1
      ELSE
          KX = 1 - (N-1)*INCX
      END IF
      IF (INCY.GT.0) THEN
          KY = 1
      ELSE
          KY = 1 - (N-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the triangular part
*     of A.
*
*     First form  y := beta*y.
*
      IF (BETA.NE.ONE) THEN
          IF (INCY.EQ.1) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 10 I = 1,N
                      Y(I) = ZERO
   10             CONTINUE
              ELSE
                  DO 20 I = 1,N
                      Y(I) = BETA*Y(I)
   20             CONTINUE
              END IF
          ELSE
              IY = KY
              IF (BETA.EQ.ZERO) THEN
                  DO 30 I = 1,N
                      Y(IY) = ZERO
                      IY = IY + INCY
   30             CONTINUE
              ELSE
                  DO 40 I = 1,N
                      Y(IY) = BETA*Y(IY)
                      IY = IY + INCY
   40             CONTINUE
              END IF
          END IF
      END IF
      IF (ALPHA.EQ.ZERO) RETURN
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  y  when A is stored in upper triangle.
*
          IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
```

```
            DO 60 J = 1,N
                TEMP1 = ALPHA*X(J)
                TEMP2 = ZERO
                DO 50 I = 1,J - 1
                    Y(I) = Y(I) + TEMP1*A(I,J)
                    TEMP2 = TEMP2 + A(I,J)*X(I)
   50           CONTINUE
                Y(J) = Y(J) + TEMP1*A(J,J) + ALPHA*TEMP2
   60       CONTINUE
        ELSE
            JX = KX
            JY = KY
            DO 80 J = 1,N
                TEMP1 = ALPHA*X(JX)
                TEMP2 = ZERO
                IX = KX
                IY = KY
                DO 70 I = 1,J - 1
                    Y(IY) = Y(IY) + TEMP1*A(I,J)
                    TEMP2 = TEMP2 + A(I,J)*X(IX)
                    IX = IX + INCX
                    IY = IY + INCY
   70           CONTINUE
                Y(JY) = Y(JY) + TEMP1*A(J,J) + ALPHA*TEMP2
                JX = JX + INCX
                JY = JY + INCY
   80       CONTINUE
        END IF
     ELSE
*
*       Form  y  when A is stored in lower triangle.
*
         IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
            DO 100 J = 1,N
                TEMP1 = ALPHA*X(J)
                TEMP2 = ZERO
                Y(J) = Y(J) + TEMP1*A(J,J)
                DO 90 I = J + 1,N
                    Y(I) = Y(I) + TEMP1*A(I,J)
                    TEMP2 = TEMP2 + A(I,J)*X(I)
   90           CONTINUE
                Y(J) = Y(J) + ALPHA*TEMP2
  100       CONTINUE
        ELSE
            JX = KX
            JY = KY
            DO 120 J = 1,N
                TEMP1 = ALPHA*X(JX)
                TEMP2 = ZERO
                Y(JY) = Y(JY) + TEMP1*A(J,J)
```

```
                IX = JX
                IY = JY
                DO 110 I = J + 1,N
                    IX = IX + INCX
                    IY = IY + INCY
                    Y(IY) = Y(IY) + TEMP1*A(I,J)
                    TEMP2 = TEMP2 + A(I,J)*X(IX)
  110           CONTINUE
                Y(JY) = Y(JY) + ALPHA*TEMP2
                JX = JX + INCX
                JY = JY + INCY
  120         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of SSYMV .
*
      END
```

———-

10.8.4 lisp code

— BLAS 2 ssymv —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun ssymv (uplo n alpha a lda x incx beta y incy)
    (declare (type (array single-float (*)) y x a)
             (type (single-float) beta alpha)
             (type (f2cl-lib:integer4) incy incx lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (x single-float x-%data% x-%offset%)
         (y single-float y-%data% y-%offset%))
      (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (kx 0) (ky 0)
             (temp1 0.0) (temp2 0.0))
        (declare (type (f2cl-lib:integer4) i info ix iy j jx jy kx ky)
                 (type (single-float) temp1 temp2))
        (setf info 0)
        (cond
```

```
 ((and (not (lsame uplo "U")) (not (lsame uplo "L"))) (setf info 1))
 ((< n 0) (setf info 2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info 5))
 ((= incx 0) (setf info 7)) ((= incy 0) (setf info 10)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "SSYMV " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= n 0) (and (= alpha zero) (= beta one)))
    (go end_label))
(cond ((> incx 0) (setf kx 1))
      (t
       (setf kx
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incx)))))
(cond ((> incy 0) (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incy)))))
(cond
 ((/= beta one)
  (cond
   ((= incy 1)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%) zero)
         label10)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                  (* beta
                     (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)))
         label20)))))
   (t (setf iy ky)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
```

```
          (tagbody
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) zero)
            (setf iy (f2cl-lib:int-add iy incy))
           label30)))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (* beta
                       (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)))
            (setf iy (f2cl-lib:int-add iy incy))
           label40))))))))
  (if (= alpha zero)
      (go end_label))
  (cond
   ((lsame uplo "U")
    (cond
     ((and (= incx 1) (= incy 1))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
       (tagbody
         (setf temp1
                 (* alpha
                    (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
         (setf temp2 zero)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                        nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                        (* temp1
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%))))
             (setf temp2
                     (+ temp2
                        (*
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)
                         (f2cl-lib:fref x-%data% (i) ((1 *))
                                        x-%offset%))))
            label50))
         (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                 (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                    (* temp1
                       (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                      a-%offset%))
                    (* alpha temp2)))
        label60)))
```

```
  (t (setf jx kx) (setf jy ky)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp1
               (* alpha
                  (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
       (setf temp2 zero)
       (setf ix kx)
       (setf iy ky)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                      nil)
         (tagbody
           (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                   (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (* temp1
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%))))
           (setf temp2
                   (+ temp2
                      (*
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                       (f2cl-lib:fref x-%data% (ix) ((1 *))
                                      x-%offset%))))
           (setf ix (f2cl-lib:int-add ix incx))
           (setf iy (f2cl-lib:int-add iy incy))
          label70))
       (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
               (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                  (* temp1
                     (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                    a-%offset%))
                  (* alpha temp2)))
       (setf jx (f2cl-lib:int-add jx incx))
       (setf jy (f2cl-lib:int-add jy incy))
      label80)))))
(t
 (cond
  ((and (= incx 1) (= incy 1))
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp1
               (* alpha
                  (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
       (setf temp2 zero)
       (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
               (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
```

```
                  (* temp1
                     (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                    a-%offset%))))
      (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                  (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     (* temp1
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))))
          (setf temp2
                  (+ temp2
                     (*
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%)
                      (f2cl-lib:fref x-%data% (i) ((1 *))
                                     x-%offset%))))
         label90))
      (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
              (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                 (* alpha temp2)))
    label100)))
(t (setf jx kx) (setf jy ky)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf temp1
             (* alpha
                (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
     (setf temp2 zero)
     (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
             (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                (* temp1
                   (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                  a-%offset%))))
     (setf ix jx)
     (setf iy jy)
     (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf ix (f2cl-lib:int-add ix incx))
         (setf iy (f2cl-lib:int-add iy incy))
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (* temp1
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%))))
         (setf temp2
                 (+ temp2
```

```
                              (*
                               (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                              a-%offset%)
                               (f2cl-lib:fref x-%data% (ix) ((1 *))
                                              x-%offset%))))
                 label110))
              (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                      (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                         (* alpha temp2)))
              (setf jx (f2cl-lib:int-add jx incx))
              (setf jy (f2cl-lib:int-add jy incy))
             label120))))))
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil nil nil nil nil))))))
```

———-

10.9 ssyr - symmetric rank 1 $\alpha xx^T + A$

10.9.1 Axiom unit tests

— **ssyr.input** —

```
)set break resume
)sys rm -f ssyr.output
)spool ssyr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

10.9.2 Axiom help page

— **ssyr.help** —

```
==========================================================================
ssyr examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssyr (UPLO, N, ALPHA, X, INCX, A, LDA)
           SSYR

Function/Subroutine Documentation
   subroutine ssyr (character UPLO, integer N, real ALPHA, real, dimension(*)
       X, integer INCX, real, dimension(lda,*) A, integer LDA)
       SSYR

Purpose:
  SSYR   performs the symmetric rank 1 operation

    A := alpha*x*x**T + A,

   where alpha is a real scalar, x is an n element vector and A is an
   n by n symmetric matrix.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the upper or lower
   triangular part of the array A is to be referenced as follows:

   UPLO = 'U' or 'u'   Only the upper triangular part of A
   is to be referenced.

   UPLO = 'L' or 'l'   Only the lower triangular part of A
   is to be referenced.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
   N must be at least zero.

 ALPHA
   ALPHA is REAL
   On entry, ALPHA specifies the scalar alpha.

 X
   X is REAL array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
   Before entry, the incremented array X must contain the n
   element vector x.
```

```
INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.

A
A is REAL array of DIMENSION ( LDA, n ).

  Before entry with  UPLO = 'U' or 'u', the leading n by n
  upper triangular part of the array A must contain the upper
  triangular part of the symmetric matrix and the strictly
  lower triangular part of A is not referenced. On exit, the
  upper triangular part of the array A is overwritten by the
  upper triangular part of the updated matrix.

  Before entry with UPLO = 'L' or 'l', the leading n by n
  lower triangular part of the array A must contain the lower
  triangular part of the symmetric matrix and the strictly
  upper triangular part of A is not referenced. On exit, the
  lower triangular part of the array A is overwritten by the
  lower triangular part of the updated matrix.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, n ).
```

———-

10.9.3 fortran code

— **ssyr.f** —

```
*  ===================================================================
      SUBROUTINE SSYR(UPLO,N,ALPHA,X,INCX,A,LDA)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL ALPHA
      INTEGER INCX,LDA,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
```

```
      REAL A(LDA,*),X(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL ZERO
      PARAMETER (ZERO=0.0E+0)
*     ..
*     .. Local Scalars ..
      REAL TEMP
      INTEGER I,INFO,IX,J,JX,KX
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 7
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('SSYR  ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (ALPHA.EQ.ZERO)) RETURN
*
*     Set the start point in X if the increment is not unity.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
```

```
         KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the triangular part
*     of A.
*
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  A  when A is stored in upper triangle.
*
         IF (INCX.EQ.1) THEN
            DO 20 J = 1,N
               IF (X(J).NE.ZERO) THEN
                  TEMP = ALPHA*X(J)
                  DO 10 I = 1,J
                     A(I,J) = A(I,J) + X(I)*TEMP
   10             CONTINUE
               END IF
   20       CONTINUE
         ELSE
            JX = KX
            DO 40 J = 1,N
               IF (X(JX).NE.ZERO) THEN
                  TEMP = ALPHA*X(JX)
                  IX = KX
                  DO 30 I = 1,J
                     A(I,J) = A(I,J) + X(IX)*TEMP
                     IX = IX + INCX
   30             CONTINUE
               END IF
               JX = JX + INCX
   40       CONTINUE
         END IF
      ELSE
*
*        Form  A  when A is stored in lower triangle.
*
         IF (INCX.EQ.1) THEN
            DO 60 J = 1,N
               IF (X(J).NE.ZERO) THEN
                  TEMP = ALPHA*X(J)
                  DO 50 I = J,N
                     A(I,J) = A(I,J) + X(I)*TEMP
   50             CONTINUE
               END IF
   60       CONTINUE
         ELSE
            JX = KX
            DO 80 J = 1,N
```

```
                IF (X(JX).NE.ZERO) THEN
                    TEMP = ALPHA*X(JX)
                    IX = JX
                    DO 70 I = J,N
                        A(I,J) = A(I,J) + X(IX)*TEMP
                        IX = IX + INCX
   70               CONTINUE
                END IF
                JX = JX + INCX
   80         CONTINUE
         END IF
      END IF
*
      RETURN
*
*     End of SSYR  .
*
      END
```

———-

10.9.4 lisp code

— BLAS 2 ssyr —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun ssyr (uplo n alpha x incx a lda)
    (declare (type (array single-float (*)) a x)
             (type (single-float) alpha)
             (type (f2cl-lib:integer4) lda incx n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (x single-float x-%data% x-%offset%)
         (a single-float a-%data% a-%offset%))
      (prog ((i 0) (info 0) (ix 0) (j 0) (jx 0) (kx 0) (temp 0.0))
        (declare (type (f2cl-lib:integer4) i info ix j jx kx)
                 (type (single-float) temp))
        (setf info 0)
        (cond
         ((and (not (lsame uplo "U")) (not (lsame uplo "L"))) (setf info 1))
         ((< n 0) (setf info 2)) ((= incx 0) (setf info 5))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info 7)))
        (cond
```

```
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "SSYR  " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= n 0) (= alpha zero))
    (go end_label))
(cond
 ((<= incx 0)
  (setf kx
          (f2cl-lib:int-sub 1
                            (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                              incx))))
 ((/= incx 1) (setf kx 1)))
(cond
 ((lsame uplo "U")
  (cond
   ((= incx 1)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
         ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
          (setf temp
                  (* alpha
                     (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%)
                      (+
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                       (*
                        (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                        temp)))
             label10))))
       label20)))
   (t (setf jx kx)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
         ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
          (setf temp
                  (* alpha
                     (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
          (setf ix kx)
```

```
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i j) nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%)
                      (+
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                       (*
                        (f2cl-lib:fref x-%data% (ix) ((1 *))
                                       x-%offset%)
                        temp)))
                (setf ix (f2cl-lib:int-add ix incx))
               label30))))
          (setf jx (f2cl-lib:int-add jx incx))
         label40)))))
(t
 (cond
  ((= incx 1)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
         (setf temp
                 (* alpha
                    (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
         (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                   (+
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%)
                    (*
                     (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                     temp)))
            label50))))
      label60)))
  (t (setf jx kx)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
         (setf temp
                 (* alpha
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
         (setf ix jx)
```

```
                (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                          (+
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%)
                           (*
                            (f2cl-lib:fref x-%data% (ix) ((1 *))
                                           x-%offset%)
                            temp)))
                    (setf ix (f2cl-lib:int-add ix incx))
                   label70))))
              (setf jx (f2cl-lib:int-add jx incx))
             label80))))))
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil nil))))))
```

———-

10.10 ssyr2 - $\alpha xy^T + \alpha yx^T + A$

10.10.1 Axiom unit tests

— **ssyr2.input** —

```
)set break resume
)sys rm -f ssyr2.output
)spool ssyr2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

10.10.2 Axiom help page

— **ssyr2.help** —

```
====================================================================
ssyr2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssyr2 (UPLO, N, ALPHA, X, INCX, Y, INCY, A, LDA)
           SSYR2

Function/Subroutine Documentation
   subroutine ssyr2 (character UPLO, integer N, real ALPHA, real, dimension(*)
       X, integer INCX, real, dimension(*) Y, integer INCY, real,
       dimension(lda,*) A, integer LDA)
       SSYR2

Purpose:
   SSYR2  performs the symmetric rank 2 operation

      A := alpha*x*y**T + alpha*y*x**T + A,

   where alpha is a scalar, x and y are n element vectors and A is an n
   by n symmetric matrix.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the upper or lower
   triangular part of the array A is to be referenced as follows:

   UPLO = 'U' or 'u'   Only the upper triangular part of A
   is to be referenced.

   UPLO = 'L' or 'l'   Only the lower triangular part of A
   is to be referenced.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
   N must be at least zero.

 ALPHA
   ALPHA is REAL
   On entry, ALPHA specifies the scalar alpha.

 X
```

```
  X is REAL array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the n
  element vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.

Y
  Y is REAL array of dimension at least ( 1 + ( n - 1 )*abs( INCY ) ).
  Before entry, the incremented array Y must contain the n
  element vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of Y.
  INCY must not be zero.

A
  A is REAL array of DIMENSION ( LDA, n ).

  Before entry with  UPLO = 'U' or 'u', the leading n by n
  upper triangular part of the array A must contain the upper
  triangular part of the symmetric matrix and the strictly
  lower triangular part of A is not referenced. On exit, the
  upper triangular part of the array A is overwritten by the
  upper triangular part of the updated matrix.

  Before entry with UPLO = 'L' or 'l', the leading n by n
  lower triangular part of the array A must contain the lower
  triangular part of the symmetric matrix and the strictly
  upper triangular part of A is not referenced. On exit, the
  lower triangular part of the array A is overwritten by the
  lower triangular part of the updated matrix.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, n ).
```

———-

10.10.3 fortran code

— ssyr2.f —

```
*  =====================================================================
      SUBROUTINE SSYR2(UPLO,N,ALPHA,X,INCX,Y,INCY,A,LDA)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL ALPHA
      INTEGER INCX,INCY,LDA,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      REAL A(LDA,*),X(*),Y(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL ZERO
      PARAMETER (ZERO=0.0E+0)
*     ..
*     .. Local Scalars ..
      REAL TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,KX,KY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      ELSE IF (INCY.EQ.0) THEN
          INFO = 7
      ELSE IF (LDA.LT.MAX(1,N)) THEN
```

```
         INFO = 9
      END IF
      IF (INFO.NE.0) THEN
         CALL XERBLA('SSYR2 ',INFO)
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (ALPHA.EQ.ZERO)) RETURN
*
*     Set up the start points in X and Y if the increments are not both
*     unity.
*
      IF ((INCX.NE.1) .OR. (INCY.NE.1)) THEN
         IF (INCX.GT.0) THEN
            KX = 1
         ELSE
            KX = 1 - (N-1)*INCX
         END IF
         IF (INCY.GT.0) THEN
            KY = 1
         ELSE
            KY = 1 - (N-1)*INCY
         END IF
         JX = KX
         JY = KY
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the triangular part
*     of A.
*
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  A  when A is stored in the upper triangle.
*
         IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
            DO 20 J = 1,N
               IF ((X(J).NE.ZERO) .OR. (Y(J).NE.ZERO)) THEN
                  TEMP1 = ALPHA*Y(J)
                  TEMP2 = ALPHA*X(J)
                  DO 10 I = 1,J
                     A(I,J) = A(I,J) + X(I)*TEMP1 + Y(I)*TEMP2
   10             CONTINUE
               END IF
   20       CONTINUE
         ELSE
            DO 40 J = 1,N
               IF ((X(JX).NE.ZERO) .OR. (Y(JY).NE.ZERO)) THEN
```

```
                    TEMP1 = ALPHA*Y(JY)
                    TEMP2 = ALPHA*X(JX)
                    IX = KX
                    IY = KY
                    DO 30 I = 1,J
                        A(I,J) = A(I,J) + X(IX)*TEMP1 + Y(IY)*TEMP2
                        IX = IX + INCX
                        IY = IY + INCY
   30               CONTINUE
                END IF
                JX = JX + INCX
                JY = JY + INCY
   40         CONTINUE
        END IF
      ELSE
*
*       Form  A  when A is stored in the lower triangle.
*
        IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
            DO 60 J = 1,N
                IF ((X(J).NE.ZERO) .OR. (Y(J).NE.ZERO)) THEN
                    TEMP1 = ALPHA*Y(J)
                    TEMP2 = ALPHA*X(J)
                    DO 50 I = J,N
                        A(I,J) = A(I,J) + X(I)*TEMP1 + Y(I)*TEMP2
   50               CONTINUE
                END IF
   60       CONTINUE
        ELSE
            DO 80 J = 1,N
                IF ((X(JX).NE.ZERO) .OR. (Y(JY).NE.ZERO)) THEN
                    TEMP1 = ALPHA*Y(JY)
                    TEMP2 = ALPHA*X(JX)
                    IX = JX
                    IY = JY
                    DO 70 I = J,N
                        A(I,J) = A(I,J) + X(IX)*TEMP1 + Y(IY)*TEMP2
                        IX = IX + INCX
                        IY = IY + INCY
   70               CONTINUE
                END IF
                JX = JX + INCX
                JY = JY + INCY
   80       CONTINUE
        END IF
      END IF
*
      RETURN
*
*     End of SSYR2 .
```

```
*
      END
```

———-

10.10.4 lisp code

— BLAS 2 ssyr2 —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun ssyr2 (uplo n alpha x incx y incy a lda)
    (declare (type (array single-float (*)) a y x)
             (type (single-float) alpha)
             (type (f2cl-lib:integer4) lda incy incx n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (x single-float x-%data% x-%offset%)
         (y single-float y-%data% y-%offset%)
         (a single-float a-%data% a-%offset%))
      (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (kx 0) (ky 0)
             (temp1 0.0) (temp2 0.0))
        (declare (type (f2cl-lib:integer4) i info ix iy j jx jy kx ky)
                 (type (single-float) temp1 temp2))
        (setf info 0)
        (cond
         ((and (not (lsame uplo "U")) (not (lsame uplo "L"))) (setf info 1))
         ((< n 0) (setf info 2)) ((= incx 0) (setf info 5))
         ((= incy 0) (setf info 7))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info 9)))
        (cond
         ((/= info 0)
          (multiple-value-bind (var-0 var-1)
              (xerbla "SSYR2 " info)
            (declare (ignore var-0))
            (when var-1 (setf info var-1)))
          (go end_label)))
        (if (or (= n 0) (= alpha zero))
            (go end_label))
        (cond
         ((or (/= incx 1) (/= incy 1))
          (cond ((> incx 0) (setf kx 1))
                (t
                 (setf kx
```

```
                    (f2cl-lib:int-sub 1
                                      (f2cl-lib:int-mul
                                       (f2cl-lib:int-sub n 1) incx)))))
  (cond ((> incy 0) (setf ky 1))
        (t
         (setf ky
                 (f2cl-lib:int-sub 1
                                      (f2cl-lib:int-mul
                                       (f2cl-lib:int-sub n 1) incy)))))
  (setf jx kx) (setf jy ky)))
(cond
 ((lsame uplo "U")
  (cond
   ((and (= incx 1) (= incy 1))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
         ((or (/= (f2cl-lib:fref x (j) ((1 *))) zero)
              (/= (f2cl-lib:fref y (j) ((1 *))) zero))
          (setf temp1
                  (* alpha
                     (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)))
          (setf temp2
                  (* alpha
                     (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%)
                      (+
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                       (*
                        (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                        temp1)
                       (*
                        (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                        temp2)))
             label10))))
      label20)))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
         ((or (/= (f2cl-lib:fref x (jx) ((1 *))) zero)
              (/= (f2cl-lib:fref y (jy) ((1 *))) zero))
          (setf temp1
```

```
                  (* alpha
                     (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)))
          (setf temp2
                  (* alpha
                     (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
          (setf ix kx) (setf iy ky)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%)
                    (+
                     (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%)
                     (*
                      (f2cl-lib:fref x-%data% (ix) ((1 *))
                                     x-%offset%)
                      temp1)
                     (*
                      (f2cl-lib:fref y-%data% (iy) ((1 *))
                                     y-%offset%)
                      temp2)))
              (setf ix (f2cl-lib:int-add ix incx))
              (setf iy (f2cl-lib:int-add iy incy))
             label30))))
       (setf jx (f2cl-lib:int-add jx incx))
       (setf jy (f2cl-lib:int-add jy incy))
      label40)))))
(t
 (cond
  ((and (= incx 1) (= incy 1))
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((or (/= (f2cl-lib:fref x (j) ((1 *))) zero)
             (/= (f2cl-lib:fref y (j) ((1 *))) zero))
         (setf temp1
                 (* alpha
                    (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)))
         (setf temp2
                 (* alpha
                    (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
         (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                   (+
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
```

```
                                        a-%offset%)
                              (*
                               (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                               temp1)
                              (*
                               (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                               temp2)))
                    label50))))
               label60)))
          (t
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (cond
                ((or (/= (f2cl-lib:fref x (jx) ((1 *))) zero)
                     (/= (f2cl-lib:fref y (jy) ((1 *))) zero))
                 (setf temp1
                         (* alpha
                            (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)))
                 (setf temp2
                         (* alpha
                            (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
                 (setf ix jx) (setf iy jy)
                 (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                               ((> i n) nil)
                   (tagbody
                     (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%)
                           (+
                            (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                           a-%offset%)
                            (*
                             (f2cl-lib:fref x-%data% (ix) ((1 *))
                                            x-%offset%)
                             temp1)
                            (*
                             (f2cl-lib:fref y-%data% (iy) ((1 *))
                                            y-%offset%)
                             temp2)))
                     (setf ix (f2cl-lib:int-add ix incx))
                     (setf iy (f2cl-lib:int-add iy incy))
                    label70))))
                (setf jx (f2cl-lib:int-add jx incx))
                (setf jy (f2cl-lib:int-add jy incy))
               label80))))))
 (go end_label)
end_label
 (return (values nil nil nil nil nil nil nil nil nil))))))
```

———-

10.11 stbmv - banded matrix-vector $x = A*x$ or $x = A^T x$

10.11.1 Axiom unit tests

— **stbmv.input** —

```
)set break resume
)sys rm -f stbmv.output
)spool stbmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

10.11.2 Axiom help page

— **stbmv.help** —

```
======================================================================
stbmv examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine stbmv (UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)
           STBMV

Function/Subroutine Documentation
   subroutine stbmv (character UPLO, character TRANS, character DIAG, integer
       N, integer K, real, dimension(lda,*) A, integer LDA, real, dimension(*)
       X, integer INCX)
       STBMV

Purpose:
```

```
   STBMV  performs one of the matrix-vector operations

      x := A*x,   or   x := A**T*x,

   where x is an n element vector and  A is an n by n unit, or non-unit,
   upper or lower triangular band matrix, with ( k + 1 ) diagonals.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the matrix is an upper or
   lower triangular matrix as follows:

   UPLO = 'U' or 'u'   A is an upper triangular matrix.

   UPLO = 'L' or 'l'   A is a lower triangular matrix.

 TRANS
   TRANS is CHARACTER*1
   On entry, TRANS specifies the operation to be performed as follows:

   TRANS = 'N' or 'n'   x := A*x.

   TRANS = 'T' or 't'   x := A**T*x.

   TRANS = 'C' or 'c'   x := A**T*x.

 DIAG
   DIAG is CHARACTER*1
   On entry, DIAG specifies whether or not A is unit
   triangular as follows:

   DIAG = 'U' or 'u'   A is assumed to be unit triangular.

   DIAG = 'N' or 'n'   A is not assumed to be unit triangular.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
   N must be at least zero.

 K
   K is INTEGER
   On entry with UPLO = 'U' or 'u', K specifies the number of
   super-diagonals of the matrix A.
   On entry with UPLO = 'L' or 'l', K specifies the number of
   sub-diagonals of the matrix A.
   K must satisfy  0 .le. K.
```

```
A
  A is REAL array of DIMENSION ( LDA, n ).

  Before entry with UPLO = 'U' or 'u', the leading ( k + 1 )
  by n part of the array A must contain the upper triangular
  band part of the matrix of coefficients, supplied column by
  column, with the leading diagonal of the matrix in row
  ( k + 1 ) of the array, the first super-diagonal starting at
  position 2 in row k, and so on. The top left k by k triangle
  of the array A is not referenced.

  The following program segment will transfer an upper
  triangular band matrix from conventional full matrix storage
  to band storage:

       DO 20, J = 1, N
          M = K + 1 - J
          DO 10, I = MAX( 1, J - K ), J
             A( M + I, J ) = matrix( I, J )
    10    CONTINUE
    20 CONTINUE

  Before entry with UPLO = 'L' or 'l', the leading ( k + 1 )
  by n part of the array A must contain the lower triangular
  band part of the matrix of coefficients, supplied column by
  column, with the leading diagonal of the matrix in row 1 of
  the array, the first sub-diagonal starting at position 1 in
  row 2, and so on. The bottom right k by k triangle of the
  array A is not referenced.

  The following program segment will transfer a lower
  triangular band matrix from conventional full matrix storage
  to band storage:

       DO 20, J = 1, N
          M = 1 - J
          DO 10, I = J, MIN( N, J + K )
             A( M + I, J ) = matrix( I, J )
    10    CONTINUE
    20 CONTINUE

  Note that when DIAG = 'U' or 'u' the elements of the array A
  corresponding to the diagonal elements of the matrix are not
  referenced, but are assumed to be unity.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least ( k + 1 ).
```

```
 X
   X is REAL array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
   Before entry, the incremented array X must contain the n
   element vector x. On exit, X is overwritten with the tranformed vector x.

 INCX
   INCX is INTEGER
   On entry, INCX specifies the increment for the elements of X.
   INCX must not be zero.

   The vector and matrix arguments are not referenced when N = 0, or M = 0
```

———-

10.11.3 fortran code

— stbmv.f —

```
*  =====================================================================
      SUBROUTINE STBMV(UPLO,TRANS,DIAG,N,K,A,LDA,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,K,LDA,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      REAL A(LDA,*),X(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL ZERO
      PARAMETER (ZERO=0.0E+0)
*     ..
*     .. Local Scalars ..
      REAL TEMP
      INTEGER I,INFO,IX,J,JX,KPLUS1,KX,L
      LOGICAL NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
```

```
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX,MIN
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (K.LT.0) THEN
          INFO = 5
      ELSE IF (LDA.LT. (K+1)) THEN
          INFO = 7
      ELSE IF (INCX.EQ.0) THEN
          INFO = 9
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('STBMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX   too small for descending loops.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
```

```
*
      IF (LSAME(TRANS,'N')) THEN
*
*         Form  x := A*x.
*
          IF (LSAME(UPLO,'U')) THEN
              KPLUS1 = K + 1
              IF (INCX.EQ.1) THEN
                  DO 20 J = 1,N
                      IF (X(J).NE.ZERO) THEN
                          TEMP = X(J)
                          L = KPLUS1 - J
                          DO 10 I = MAX(1,J-K),J - 1
                              X(I) = X(I) + TEMP*A(L+I,J)
   10                     CONTINUE
                          IF (NOUNIT) X(J) = X(J)*A(KPLUS1,J)
                      END IF
   20             CONTINUE
              ELSE
                  JX = KX
                  DO 40 J = 1,N
                      IF (X(JX).NE.ZERO) THEN
                          TEMP = X(JX)
                          IX = KX
                          L = KPLUS1 - J
                          DO 30 I = MAX(1,J-K),J - 1
                              X(IX) = X(IX) + TEMP*A(L+I,J)
                              IX = IX + INCX
   30                     CONTINUE
                          IF (NOUNIT) X(JX) = X(JX)*A(KPLUS1,J)
                      END IF
                      JX = JX + INCX
                      IF (J.GT.K) KX = KX + INCX
   40             CONTINUE
              END IF
          ELSE
              IF (INCX.EQ.1) THEN
                  DO 60 J = N,1,-1
                      IF (X(J).NE.ZERO) THEN
                          TEMP = X(J)
                          L = 1 - J
                          DO 50 I = MIN(N,J+K),J + 1,-1
                              X(I) = X(I) + TEMP*A(L+I,J)
   50                     CONTINUE
                          IF (NOUNIT) X(J) = X(J)*A(1,J)
                      END IF
   60             CONTINUE
              ELSE
                  KX = KX + (N-1)*INCX
                  JX = KX
```

```
                DO 80 J = N,1,-1
                    IF (X(JX).NE.ZERO) THEN
                        TEMP = X(JX)
                        IX = KX
                        L = 1 - J
                        DO 70 I = MIN(N,J+K),J + 1,-1
                            X(IX) = X(IX) + TEMP*A(L+I,J)
                            IX = IX - INCX
   70                   CONTINUE
                        IF (NOUNIT) X(JX) = X(JX)*A(1,J)
                    END IF
                    JX = JX - INCX
                    IF ((N-J).GE.K) KX = KX - INCX
   80           CONTINUE
            END IF
        END IF
      ELSE
*
*       Form  x := A**T*x.
*
         IF (LSAME(UPLO,'U')) THEN
            KPLUS1 = K + 1
            IF (INCX.EQ.1) THEN
                DO 100 J = N,1,-1
                    TEMP = X(J)
                    L = KPLUS1 - J
                    IF (NOUNIT) TEMP = TEMP*A(KPLUS1,J)
                    DO 90 I = J - 1,MAX(1,J-K),-1
                        TEMP = TEMP + A(L+I,J)*X(I)
   90               CONTINUE
                    X(J) = TEMP
  100           CONTINUE
            ELSE
                KX = KX + (N-1)*INCX
                JX = KX
                DO 120 J = N,1,-1
                    TEMP = X(JX)
                    KX = KX - INCX
                    IX = KX
                    L = KPLUS1 - J
                    IF (NOUNIT) TEMP = TEMP*A(KPLUS1,J)
                    DO 110 I = J - 1,MAX(1,J-K),-1
                        TEMP = TEMP + A(L+I,J)*X(IX)
                        IX = IX - INCX
  110               CONTINUE
                    X(JX) = TEMP
                    JX = JX - INCX
  120           CONTINUE
            END IF
         ELSE
```

```
            IF (INCX.EQ.1) THEN
                DO 140 J = 1,N
                    TEMP = X(J)
                    L = 1 - J
                    IF (NOUNIT) TEMP = TEMP*A(1,J)
                    DO 130 I = J + 1,MIN(N,J+K)
                        TEMP = TEMP + A(L+I,J)*X(I)
 130                CONTINUE
                    X(J) = TEMP
 140            CONTINUE
            ELSE
                JX = KX
                DO 160 J = 1,N
                    TEMP = X(JX)
                    KX = KX + INCX
                    IX = KX
                    L = 1 - J
                    IF (NOUNIT) TEMP = TEMP*A(1,J)
                    DO 150 I = J + 1,MIN(N,J+K)
                        TEMP = TEMP + A(L+I,J)*X(IX)
                        IX = IX + INCX
 150                CONTINUE
                    X(JX) = TEMP
                    JX = JX + INCX
 160            CONTINUE
            END IF
        END IF
    END IF
*
    RETURN
*
*   End of STBMV .
*
    END
```

———-

10.11.4 lisp code

— BLAS 2 ssbmv —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun stbmv (uplo trans diag n k a lda x incx)
    (declare (type (array single-float (*)) x a)
             (type (f2cl-lib:integer4) incx lda k n)
```

```
          (type (string *) diag trans uplo))
   (f2cl-lib:with-multi-array-data
       ((uplo character uplo-%data% uplo-%offset%)
        (trans character trans-%data% trans-%offset%)
        (diag character diag-%data% diag-%offset%)
        (a single-float a-%data% a-%offset%)
        (x single-float x-%data% x-%offset%))
     (prog ((nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0) (kplus1 0) (kx 0)
            (l 0) (temp 0.0))
       (declare (type f2cl-lib:logical nounit)
                (type (f2cl-lib:integer4) i info ix j jx kplus1 kx l)
                (type (single-float) temp))
       (setf info 0)
       (cond
        ((and (not (lsame uplo "U")) (not (lsame uplo "L"))) (setf info 1))
        ((and (not (lsame trans "N")) (not (lsame trans "T"))
              (not (lsame trans "C")))
         (setf info 2))
        ((and (not (lsame diag "U")) (not (lsame diag "N"))) (setf info 3))
        ((< n 0) (setf info 4)) ((< k 0) (setf info 5))
        ((< lda (f2cl-lib:int-add k 1)) (setf info 7))
        ((= incx 0) (setf info 9)))
       (cond
        ((/= info 0)
         (multiple-value-bind (var-0 var-1)
             (xerbla "STBMV " info)
           (declare (ignore var-0))
           (when var-1 (setf info var-1)))
         (go end_label)))
       (if (= n 0)
           (go end_label))
       (setf nounit (lsame diag "N"))
       (cond
        ((<= incx 0)
         (setf kx
                 (f2cl-lib:int-sub 1
                                   (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                     incx))))
        ((/= incx 1) (setf kx 1)))
       (cond
        ((lsame trans "N")
         (cond
          ((lsame uplo "U") (setf kplus1 (f2cl-lib:int-add k 1))
           (cond
            ((= incx 1)
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (cond
                  ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
```

```
        (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
        (setf l (f2cl-lib:int-sub kplus1 j))
        (f2cl-lib:fdo (i
                       (max (the f2cl-lib:integer4 1)
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add j
                                                   (f2cl-lib:int-sub
                                                    k))))
                       (f2cl-lib:int-add i 1))
                      ((> i
                          (f2cl-lib:int-add j
                                            (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                    (+
                     (f2cl-lib:fref x-%data% (i) ((1 *))
                                    x-%offset%)
                     (* temp
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add l i) j)
                                       ((1 lda) (1 *))
                                       a-%offset%))))
           label10))
        (if nounit
            (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                    (*
                     (f2cl-lib:fref x-%data% (j) ((1 *))
                                    x-%offset%)
                     (f2cl-lib:fref a-%data% (kplus1 j)
                                    ((1 lda) (1 *))
                                    a-%offset%))))))
    label20)))
(t (setf jx kx)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
      ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
       (setf temp
               (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
       (setf ix kx) (setf l (f2cl-lib:int-sub kplus1 j))
       (f2cl-lib:fdo (i
                       (max (the f2cl-lib:integer4 1)
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add j
                                                   (f2cl-lib:int-sub
                                                    k))))
                       (f2cl-lib:int-add i 1))
                      ((> i
```

```
                                (f2cl-lib:int-add j
                                                  (f2cl-lib:int-sub 1)))
                            nil)
                (tagbody
                  (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                          (+
                           (f2cl-lib:fref x-%data% (ix) ((1 *))
                                          x-%offset%)
                           (* temp
                              (f2cl-lib:fref a-%data%
                                             ((f2cl-lib:int-add l i) j)
                                             ((1 lda) (1 *))
                                             a-%offset%))))
                  (setf ix (f2cl-lib:int-add ix incx))
                 label30))
              (if nounit
                  (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                          (*
                           (f2cl-lib:fref x-%data% (jx) ((1 *))
                                          x-%offset%)
                           (f2cl-lib:fref a-%data% (kplus1 j)
                                          ((1 lda) (1 *))
                                          a-%offset%))))))
          (setf jx (f2cl-lib:int-add jx incx))
          (if (> j k)
              (setf kx (f2cl-lib:int-add kx incx)))
         label40)))))
    (t
     (cond
      ((= incx 1)
       (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     ((> j 1) nil)
         (tagbody
           (cond
            ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
             (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
             (setf l (f2cl-lib:int-sub 1 j))
             (f2cl-lib:fdo (i
                            (min (the f2cl-lib:integer4 n)
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add j k)))
                            (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                           ((> i (f2cl-lib:int-add j 1)) nil)
               (tagbody
                 (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                         (+
                          (f2cl-lib:fref x-%data% (i) ((1 *))
                                         x-%offset%)
                          (* temp
                             (f2cl-lib:fref a-%data%
```

```
                                        ((f2cl-lib:int-add l i) j)
                                        ((1 lda) (1 *))
                                        a-%offset%))))
          label50))
       (if nounit
           (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                   (*
                    (f2cl-lib:fref x-%data% (j) ((1 *))
                                   x-%offset%)
                    (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *))
                                   a-%offset%))))))
    label60)))
(t
 (setf kx
         (f2cl-lib:int-add kx
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-sub n 1) incx)))
 (setf jx kx)
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (cond
      ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
       (setf temp
               (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
       (setf ix kx) (setf l (f2cl-lib:int-sub 1 j))
       (f2cl-lib:fdo (i
                      (min (the f2cl-lib:integer4 n)
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add j k)))
                      (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     ((> i (f2cl-lib:int-add j 1)) nil)
         (tagbody
           (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                   (+
                    (f2cl-lib:fref x-%data% (ix) ((1 *))
                                   x-%offset%)
                    (* temp
                       (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add l i) j)
                                       ((1 lda) (1 *))
                                       a-%offset%))))
           (setf ix (f2cl-lib:int-sub ix incx))
          label70))
       (if nounit
           (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                   (*
                    (f2cl-lib:fref x-%data% (jx) ((1 *))
                                   x-%offset%)
                    (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *))
```

```
                                        a-%offset%))))))
            (setf jx (f2cl-lib:int-sub jx incx))
            (if (>= (f2cl-lib:int-sub n j) k)
                (setf kx (f2cl-lib:int-sub kx incx)))
           label80)))))))
  (t
   (cond
    ((lsame uplo "U") (setf kplus1 (f2cl-lib:int-add k 1))
     (cond
      ((= incx 1)
       (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     ((> j 1) nil)
         (tagbody
           (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
           (setf l (f2cl-lib:int-sub kplus1 j))
           (if nounit
               (setf temp
                       (* temp
                          (f2cl-lib:fref a-%data% (kplus1 j)
                                         ((1 lda) (1 *)) a-%offset%))))
           (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                          (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                         ((> i
                             (max (the f2cl-lib:integer4 1)
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-add j
                                                         (f2cl-lib:int-sub
                                                          k)))))
                          nil)
             (tagbody
               (setf temp
                       (+ temp
                          (*
                           (f2cl-lib:fref a-%data%
                                          ((f2cl-lib:int-add l i) j)
                                          ((1 lda) (1 *)) a-%offset%)
                           (f2cl-lib:fref x-%data% (i) ((1 *))
                                          x-%offset%))))
              label90))
           (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
          label100)))
      (t
       (setf kx
               (f2cl-lib:int-add kx
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incx)))
       (setf jx kx)
       (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     ((> j 1) nil)
         (tagbody
```

```
          (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
          (setf kx (f2cl-lib:int-sub kx incx))
          (setf ix kx)
          (setf l (f2cl-lib:int-sub kplus1 j))
          (if nounit
              (setf temp
                      (* temp
                         (f2cl-lib:fref a-%data% (kplus1 j)
                                        ((1 lda) (1 *)) a-%offset%))))
          (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                         (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                        ((> i
                            (max (the f2cl-lib:integer4 1)
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add j
                                                        (f2cl-lib:int-sub
                                                         k)))))
                         nil)
            (tagbody
              (setf temp
                      (+ temp
                         (*
                          (f2cl-lib:fref a-%data%
                                         ((f2cl-lib:int-add l i) j)
                                         ((1 lda) (1 *)) a-%offset%)
                          (f2cl-lib:fref x-%data% (ix) ((1 *))
                                         x-%offset%))))
              (setf ix (f2cl-lib:int-sub ix incx))
             label110))
          (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
          (setf jx (f2cl-lib:int-sub jx incx))
         label120)))))
(t
 (cond
  ((= incx 1)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
       (setf l (f2cl-lib:int-sub 1 j))
       (if nounit
           (setf temp
                   (* temp
                      (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *))
                                     a-%offset%))))
       (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                      (f2cl-lib:int-add i 1))
                     ((> i
                         (min (the f2cl-lib:integer4 n)
                              (the f2cl-lib:integer4
```

```
                                        (f2cl-lib:int-add j k))))
                              nil)
                 (tagbody
                   (setf temp
                           (+ temp
                              (*
                               (f2cl-lib:fref a-%data%
                                              ((f2cl-lib:int-add l i) j)
                                              ((1 lda) (1 *)) a-%offset%)
                               (f2cl-lib:fref x-%data% (i) ((1 *))
                                              x-%offset%))))
                  label130))
                (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
               label140)))
            (t (setf jx kx)
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
                 (setf kx (f2cl-lib:int-add kx incx))
                 (setf ix kx)
                 (setf l (f2cl-lib:int-sub 1 j))
                 (if nounit
                     (setf temp
                             (* temp
                                (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *))
                                               a-%offset%))))
                 (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                                (f2cl-lib:int-add i 1))
                               ((> i
                                   (min (the f2cl-lib:integer4 n)
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-add j k))))
                                nil)
                   (tagbody
                     (setf temp
                             (+ temp
                                (*
                                 (f2cl-lib:fref a-%data%
                                                ((f2cl-lib:int-add l i) j)
                                                ((1 lda) (1 *)) a-%offset%)
                                 (f2cl-lib:fref x-%data% (ix) ((1 *))
                                                x-%offset%))))
                     (setf ix (f2cl-lib:int-add ix incx))
                    label150))
                 (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
                 (setf jx (f2cl-lib:int-add jx incx))
                label160))))))))
 (go end_label)
end_label
```

```
        (return (values nil nil nil nil nil nil nil nil nil))))))
```

———-

10.12 stbsv - banded solve system of equations $Ax = b$ or $A^T x = b$

10.12.1 Axiom unit tests

— **stbsv.input** —

```
)set break resume
)sys rm -f stbsv.output
)spool stbsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

10.12.2 Axiom help page

— **stbsv.help** —

```
======================================================================
stbsv examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       stbsv.f -

SYNOPSIS
   Functions/Subroutines
       subroutine stbsv (UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)
           STBSV
```

```
Function/Subroutine Documentation
   subroutine stbsv (character UPLO, character TRANS, character DIAG, integer
       N, integer K, real, dimension(lda,*) A, integer LDA, real, dimension(*)
       X, integer INCX)
       STBSV

Purpose:
   STBSV  solves one of the systems of equations

      A*x = b,   or   A**T*x = b,

   where b and x are n element vectors and A is an n by n unit, or
   non-unit, upper or lower triangular band matrix, with ( k + 1 ) diagonals.

   No test for singularity or near-singularity is included in this
   routine. Such tests must be performed before calling this routine.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the matrix is an upper or
   lower triangular matrix as follows:

   UPLO = 'U' or 'u'   A is an upper triangular matrix.

   UPLO = 'L' or 'l'   A is a lower triangular matrix.

 TRANS
   TRANS is CHARACTER*1
   On entry, TRANS specifies the equations to be solved as follows:

   TRANS = 'N' or 'n'   A*x = b.

   TRANS = 'T' or 't'   A**T*x = b.

   TRANS = 'C' or 'c'   A**T*x = b.

 DIAG
   DIAG is CHARACTER*1
   On entry, DIAG specifies whether or not A is unit
   triangular as follows:

   DIAG = 'U' or 'u'   A is assumed to be unit triangular.

   DIAG = 'N' or 'n'   A is not assumed to be unit triangular.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
```

```
  N must be at least zero.

K
  K is INTEGER
  On entry with UPLO = 'U' or 'u', K specifies the number of
  super-diagonals of the matrix A.
  On entry with UPLO = 'L' or 'l', K specifies the number of
  sub-diagonals of the matrix A.
  K must satisfy  0 .le. K.

A
  A is REAL array of DIMENSION ( LDA, n ).

  Before entry with UPLO = 'U' or 'u', the leading ( k + 1 )
  by n part of the array A must contain the upper triangular
  band part of the matrix of coefficients, supplied column by
  column, with the leading diagonal of the matrix in row
  ( k + 1 ) of the array, the first super-diagonal starting at
  position 2 in row k, and so on. The top left k by k triangle
  of the array A is not referenced.

  The following program segment will transfer an upper
  triangular band matrix from conventional full matrix storage
  to band storage:

       DO 20, J = 1, N
          M = K + 1 - J
          DO 10, I = MAX( 1, J - K ), J
             A( M + I, J ) = matrix( I, J )
    10    CONTINUE
    20 CONTINUE

  Before entry with UPLO = 'L' or 'l', the leading ( k + 1 )
  by n part of the array A must contain the lower triangular
  band part of the matrix of coefficients, supplied column by
  column, with the leading diagonal of the matrix in row 1 of
  the array, the first sub-diagonal starting at position 1 in
  row 2, and so on. The bottom right k by k triangle of the
  array A is not referenced.

  The following program segment will transfer a lower
  triangular band matrix from conventional full matrix storage
  to band storage:

       DO 20, J = 1, N
          M = 1 - J
          DO 10, I = J, MIN( N, J + K )
             A( M + I, J ) = matrix( I, J )
    10    CONTINUE
    20 CONTINUE
```

```
  Note that when DIAG = 'U' or 'u' the elements of the array A
  corresponding to the diagonal elements of the matrix are not
  referenced, but are assumed to be unity.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least ( k + 1 ).

X
  X is REAL array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).

  Before entry, the incremented array X must contain the n
  element right-hand side vector b. On exit, X is overwritten
  with the solution vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.
```

———-

10.12.3 fortran code

— stbsv.f —

```
*  ====================================================================
      SUBROUTINE STBSV(UPLO,TRANS,DIAG,N,K,A,LDA,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,K,LDA,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      REAL A(LDA,*),X(*)
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
```

```
      REAL ZERO
      PARAMETER (ZERO=0.0E+0)
*     ..
*     .. Local Scalars ..
      REAL TEMP
      INTEGER I,INFO,IX,J,JX,KPLUS1,KX,L
      LOGICAL NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX,MIN
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (K.LT.0) THEN
          INFO = 5
      ELSE IF (LDA.LT. (K+1)) THEN
          INFO = 7
      ELSE IF (INCX.EQ.0) THEN
          INFO = 9
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('STBSV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
```

```
*     will be  ( N - 1 )*INCX  too small for descending loops.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed by sequentially with one pass through A.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x := inv( A )*x.
*
          IF (LSAME(UPLO,'U')) THEN
              KPLUS1 = K + 1
              IF (INCX.EQ.1) THEN
                  DO 20 J = N,1,-1
                      IF (X(J).NE.ZERO) THEN
                          L = KPLUS1 - J
                          IF (NOUNIT) X(J) = X(J)/A(KPLUS1,J)
                          TEMP = X(J)
                          DO 10 I = J - 1,MAX(1,J-K),-1
                              X(I) = X(I) - TEMP*A(L+I,J)
   10                     CONTINUE
                      END IF
   20             CONTINUE
              ELSE
                  KX = KX + (N-1)*INCX
                  JX = KX
                  DO 40 J = N,1,-1
                      KX = KX - INCX
                      IF (X(JX).NE.ZERO) THEN
                          IX = KX
                          L = KPLUS1 - J
                          IF (NOUNIT) X(JX) = X(JX)/A(KPLUS1,J)
                          TEMP = X(JX)
                          DO 30 I = J - 1,MAX(1,J-K),-1
                              X(IX) = X(IX) - TEMP*A(L+I,J)
                              IX = IX - INCX
   30                     CONTINUE
                      END IF
                      JX = JX - INCX
   40             CONTINUE
              END IF
          ELSE
              IF (INCX.EQ.1) THEN
                  DO 60 J = 1,N
                      IF (X(J).NE.ZERO) THEN
```

```
                        L = 1 - J
                        IF (NOUNIT) X(J) = X(J)/A(1,J)
                        TEMP = X(J)
                        DO 50 I = J + 1,MIN(N,J+K)
                            X(I) = X(I) - TEMP*A(L+I,J)
   50                   CONTINUE
                    END IF
   60           CONTINUE
            ELSE
                JX = KX
                DO 80 J = 1,N
                    KX = KX + INCX
                    IF (X(JX).NE.ZERO) THEN
                        IX = KX
                        L = 1 - J
                        IF (NOUNIT) X(JX) = X(JX)/A(1,J)
                        TEMP = X(JX)
                        DO 70 I = J + 1,MIN(N,J+K)
                            X(IX) = X(IX) - TEMP*A(L+I,J)
                            IX = IX + INCX
   70                   CONTINUE
                    END IF
                    JX = JX + INCX
   80           CONTINUE
            END IF
        END IF
     ELSE
*
*       Form  x := inv( A**T)*x.
*
        IF (LSAME(UPLO,'U')) THEN
            KPLUS1 = K + 1
            IF (INCX.EQ.1) THEN
                DO 100 J = 1,N
                    TEMP = X(J)
                    L = KPLUS1 - J
                    DO 90 I = MAX(1,J-K),J - 1
                        TEMP = TEMP - A(L+I,J)*X(I)
   90               CONTINUE
                    IF (NOUNIT) TEMP = TEMP/A(KPLUS1,J)
                    X(J) = TEMP
  100           CONTINUE
            ELSE
                JX = KX
                DO 120 J = 1,N
                    TEMP = X(JX)
                    IX = KX
                    L = KPLUS1 - J
                    DO 110 I = MAX(1,J-K),J - 1
                        TEMP = TEMP - A(L+I,J)*X(IX)
```

```
                        IX = IX + INCX
 110                CONTINUE
                    IF (NOUNIT) TEMP = TEMP/A(KPLUS1,J)
                    X(JX) = TEMP
                    JX = JX + INCX
                    IF (J.GT.K) KX = KX + INCX
 120            CONTINUE
            END IF
        ELSE
            IF (INCX.EQ.1) THEN
                DO 140 J = N,1,-1
                    TEMP = X(J)
                    L = 1 - J
                    DO 130 I = MIN(N,J+K),J + 1,-1
                        TEMP = TEMP - A(L+I,J)*X(I)
 130                CONTINUE
                    IF (NOUNIT) TEMP = TEMP/A(1,J)
                    X(J) = TEMP
 140            CONTINUE
            ELSE
                KX = KX + (N-1)*INCX
                JX = KX
                DO 160 J = N,1,-1
                    TEMP = X(JX)
                    IX = KX
                    L = 1 - J
                    DO 150 I = MIN(N,J+K),J + 1,-1
                        TEMP = TEMP - A(L+I,J)*X(IX)
                        IX = IX - INCX
 150                CONTINUE
                    IF (NOUNIT) TEMP = TEMP/A(1,J)
                    X(JX) = TEMP
                    JX = JX - INCX
                    IF ((N-J).GE.K) KX = KX - INCX
 160            CONTINUE
            END IF
        END IF
    END IF
*
    RETURN
*
*   End of STBSV .
*
    END
```

———-

10.12.4 lisp code

— BLAS 2 stbsv —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun stbsv (uplo trans diag n k a lda x incx)
    (declare (type (array single-float (*)) x a)
             (type (f2cl-lib:integer4) incx lda k n)
             (type (string *) diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a single-float a-%data% a-%offset%)
         (x single-float x-%data% x-%offset%))
      (prog ((nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0) (kplus1 0) (kx 0)
             (l 0) (temp 0.0))
        (declare (type f2cl-lib:logical nounit)
                 (type (f2cl-lib:integer4) i info ix j jx kplus1 kx l)
                 (type (single-float) temp))
        (setf info 0)
        (cond
         ((and (not (lsame uplo "U")) (not (lsame uplo "L"))) (setf info 1))
         ((and (not (lsame trans "N")) (not (lsame trans "T"))
               (not (lsame trans "C")))
          (setf info 2))
         ((and (not (lsame diag "U")) (not (lsame diag "N"))) (setf info 3))
         ((< n 0) (setf info 4)) ((< k 0) (setf info 5))
         ((< lda (f2cl-lib:int-add k 1)) (setf info 7))
         ((= incx 0) (setf info 9)))
        (cond
         ((/= info 0)
          (multiple-value-bind (var-0 var-1)
              (xerbla "STBSV " info)
            (declare (ignore var-0))
            (when var-1 (setf info var-1)))
          (go end_label)))
        (if (= n 0)
            (go end_label))
        (setf nounit (lsame diag "N"))
        (cond
         ((<= incx 0)
          (setf kx
                  (f2cl-lib:int-sub 1
                                    (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                      incx))))
         ((/= incx 1) (setf kx 1)))
```

```
(cond
 ((lsame trans "N")
  (cond
   ((lsame uplo "U") (setf kplus1 (f2cl-lib:int-add k 1))
    (cond
     ((= incx 1)
      (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                    ((> j 1) nil)
        (tagbody
          (cond
           ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
            (setf l (f2cl-lib:int-sub kplus1 j))
            (if nounit
                (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                        (/
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%)
                         (f2cl-lib:fref a-%data% (kplus1 j)
                                        ((1 lda) (1 *)) a-%offset%))))
            (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
            (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                           (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                          ((> i
                              (max (the f2cl-lib:integer4 1)
                                   (the f2cl-lib:integer4
                                        (f2cl-lib:int-add j
                                                          (f2cl-lib:int-sub
                                                           k)))))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                        (-
                         (f2cl-lib:fref x-%data% (i) ((1 *))
                                        x-%offset%)
                         (* temp
                            (f2cl-lib:fref a-%data%
                                           ((f2cl-lib:int-add l i) j)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
               label10))))
        label20)))
     (t
      (setf kx
              (f2cl-lib:int-add kx
                                (f2cl-lib:int-mul
                                 (f2cl-lib:int-sub n 1) incx)))
      (setf jx kx)
      (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                    ((> j 1) nil)
        (tagbody
```

```
        (setf kx (f2cl-lib:int-sub kx incx))
        (cond
         ((/= (f2cl-lib:fref x (jx) ((1 *))) zero) (setf ix kx)
          (setf l (f2cl-lib:int-sub kplus1 j))
          (if nounit
              (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                      (/
                       (f2cl-lib:fref x-%data% (jx) ((1 *))
                                      x-%offset%)
                       (f2cl-lib:fref a-%data% (kplus1 j)
                                      ((1 lda) (1 *)) a-%offset%))))
          (setf temp
                  (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
          (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                         (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                        ((> i
                            (max (the f2cl-lib:integer4 1)
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add j
                                                        (f2cl-lib:int-sub
                                                         k)))))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                      (-
                       (f2cl-lib:fref x-%data% (ix) ((1 *))
                                      x-%offset%)
                       (* temp
                          (f2cl-lib:fref a-%data%
                                         ((f2cl-lib:int-add l i) j)
                                         ((1 lda) (1 *))
                                         a-%offset%))))
              (setf ix (f2cl-lib:int-sub ix incx))
             label30))))
        (setf jx (f2cl-lib:int-sub jx incx))
       label40)))))
(t
 (cond
  ((= incx 1)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
         (setf l (f2cl-lib:int-sub 1 j))
         (if nounit
             (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                     (/
                      (f2cl-lib:fref x-%data% (j) ((1 *))
                                     x-%offset%)
```

```
                    (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *))
                                   a-%offset%))))
        (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                       (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the f2cl-lib:integer4 n)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add j k))))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                  (-
                   (f2cl-lib:fref x-%data% (i) ((1 *))
                                  x-%offset%)
                   (* temp
                      (f2cl-lib:fref a-%data%
                                     ((f2cl-lib:int-add l i) j)
                                     ((1 lda) (1 *))
                                     a-%offset%))))
           label50))))
    label60)))
 (t (setf jx kx)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf kx (f2cl-lib:int-add kx incx))
      (cond
       ((/= (f2cl-lib:fref x (jx) ((1 *))) zero) (setf ix kx)
        (setf l (f2cl-lib:int-sub 1 j))
        (if nounit
            (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                  (/
                   (f2cl-lib:fref x-%data% (jx) ((1 *))
                                  x-%offset%)
                   (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *))
                                  a-%offset%))))
        (setf temp
                (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                       (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the f2cl-lib:integer4 n)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add j k))))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                  (-
                   (f2cl-lib:fref x-%data% (ix) ((1 *))
```

```
                                        x-%offset%)
                          (* temp
                             (f2cl-lib:fref a-%data%
                                            ((f2cl-lib:int-add l i) j)
                                            ((1 lda) (1 *))
                                            a-%offset%))))
                 (setf ix (f2cl-lib:int-add ix incx))
                label70))))
           (setf jx (f2cl-lib:int-add jx incx))
          label80)))))))
(t
 (cond
  ((lsame uplo "U") (setf kplus1 (f2cl-lib:int-add k 1))
   (cond
    ((= incx 1)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
         (setf l (f2cl-lib:int-sub kplus1 j))
         (f2cl-lib:fdo (i
                        (max (the f2cl-lib:integer4 1)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add j
                                                    (f2cl-lib:int-sub
                                                     k))))
                        (f2cl-lib:int-add i 1))
                       ((> i
                           (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                        nil)
           (tagbody
             (setf temp
                     (- temp
                        (*
                         (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-add l i) j)
                                        ((1 lda) (1 *)) a-%offset%)
                         (f2cl-lib:fref x-%data% (i) ((1 *))
                                        x-%offset%))))
            label90))
         (if nounit
             (setf temp
                     (/ temp
                        (f2cl-lib:fref a-%data% (kplus1 j)
                                       ((1 lda) (1 *)) a-%offset%))))
         (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
        label100)))
    (t (setf jx kx)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
```

```
        (tagbody
          (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
          (setf ix kx)
          (setf l (f2cl-lib:int-sub kplus1 j))
          (f2cl-lib:fdo (i
                         (max (the f2cl-lib:integer4 1)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add j
                                                     (f2cl-lib:int-sub
                                                      k))))
                         (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf temp
                      (- temp
                         (*
                          (f2cl-lib:fref a-%data%
                                         ((f2cl-lib:int-add l i) j)
                                         ((1 lda) (1 *)) a-%offset%)
                          (f2cl-lib:fref x-%data% (ix) ((1 *))
                                         x-%offset%))))
              (setf ix (f2cl-lib:int-add ix incx))
             label110))
          (if nounit
              (setf temp
                      (/ temp
                         (f2cl-lib:fref a-%data% (kplus1 j)
                                        ((1 lda) (1 *)) a-%offset%))))
          (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
          (setf jx (f2cl-lib:int-add jx incx))
          (if (> j k)
              (setf kx (f2cl-lib:int-add kx incx)))
         label120)))))
      (t
       (cond
        ((= incx 1)
         (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((> j 1) nil)
          (tagbody
            (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
            (setf l (f2cl-lib:int-sub 1 j))
            (f2cl-lib:fdo (i
                           (min (the f2cl-lib:integer4 n)
                                (the f2cl-lib:integer4
                                     (f2cl-lib:int-add j k)))
                           (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                          ((> i (f2cl-lib:int-add j 1)) nil)
              (tagbody
```

```
        (setf temp
                (- temp
                   (*
                    (f2cl-lib:fref a-%data%
                                   ((f2cl-lib:int-add l i) j)
                                   ((1 lda) (1 *)) a-%offset%)
                    (f2cl-lib:fref x-%data% (i) ((1 *))
                                   x-%offset%))))
       label130))
    (if nounit
        (setf temp
                (/ temp
                   (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *))
                                  a-%offset%))))
    (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
   label140)))
(t
 (setf kx
        (f2cl-lib:int-add kx
                          (f2cl-lib:int-mul
                           (f2cl-lib:int-sub n 1) incx)))
 (setf jx kx)
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
     (setf ix kx)
     (setf l (f2cl-lib:int-sub 1 j))
     (f2cl-lib:fdo (i
                    (min (the f2cl-lib:integer4 n)
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add j k)))
                    (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                   ((> i (f2cl-lib:int-add j 1)) nil)
       (tagbody
        (setf temp
                (- temp
                   (*
                    (f2cl-lib:fref a-%data%
                                   ((f2cl-lib:int-add l i) j)
                                   ((1 lda) (1 *)) a-%offset%)
                    (f2cl-lib:fref x-%data% (ix) ((1 *))
                                   x-%offset%))))
        (setf ix (f2cl-lib:int-sub ix incx))
       label150))
    (if nounit
        (setf temp
                (/ temp
                   (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *))
                                  a-%offset%))))
```

```
                (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
                (setf jx (f2cl-lib:int-sub jx incx))
                (if (>= (f2cl-lib:int-sub n j) k)
                    (setf kx (f2cl-lib:int-sub kx incx)))
               label160))))))))
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil nil nil nil))))))
```

———-

10.13 stpmv - matrix-vector $x = A * x$ or $x = A^T x$

10.13.1 Axiom unit tests

— **stpmv.input** —

```
)set break resume
)sys rm -f stpmv.output
)spool stpmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

10.13.2 Axiom help page

— **stpmv.help** —

```
======================================================================
stpmv examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
```

```
      subroutine stpmv (UPLO, TRANS, DIAG, N, AP, X, INCX)
          STPMV

Function/Subroutine Documentation
   subroutine stpmv (character UPLO, character TRANS, character DIAG, integer
       N, real, dimension(*) AP, real, dimension(*) X, integer INCX)
       STPMV

Purpose:
   STPMV  performs one of the matrix-vector operations

      x := A*x,   or   x := A**T*x,

   where x is an n element vector and  A is an n by n unit, or non-unit,
   upper or lower triangular matrix, supplied in packed form.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the matrix is an upper or
   lower triangular matrix as follows:

   UPLO = 'U' or 'u'   A is an upper triangular matrix.

   UPLO = 'L' or 'l'   A is a lower triangular matrix.

 TRANS
   TRANS is CHARACTER*1
   On entry, TRANS specifies the operation to be performed as follows:

   TRANS = 'N' or 'n'   x := A*x.

   TRANS = 'T' or 't'   x := A**T*x.

   TRANS = 'C' or 'c'   x := A**T*x.

 DIAG
   DIAG is CHARACTER*1
   On entry, DIAG specifies whether or not A is unit triangular as follows:

   DIAG = 'U' or 'u'   A is assumed to be unit triangular.

   DIAG = 'N' or 'n'   A is not assumed to be unit triangular.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
   N must be at least zero.
```

```
AP
  AP is REAL array of DIMENSION at least ( ( n*( n + 1 ) )/2 ).

  Before entry with  UPLO = 'U' or 'u', the array AP must
  contain the upper triangular matrix packed sequentially,
  column by column, so that AP( 1 ) contains a( 1, 1 ),
  AP( 2 ) and AP( 3 ) contain a( 1, 2 ) and a( 2, 2 )
  respectively, and so on.

  Before entry with UPLO = 'L' or 'l', the array AP must
  contain the lower triangular matrix packed sequentially,
  column by column, so that AP( 1 ) contains a( 1, 1 ),
  AP( 2 ) and AP( 3 ) contain a( 2, 1 ) and a( 3, 1 )
  respectively, and so on.

  Note that when  DIAG = 'U' or 'u', the diagonal elements of
  A are not referenced, but are assumed to be unity.

X
  X is REAL array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the n
  element vector x. On exit, X is overwritten with the
  tranformed vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.

  The vector and matrix arguments are not referenced when N = 0, or M = 0
```

10.13.3 fortran code

— stpmv.f —

```
*  ===================================================================
      SUBROUTINE STPMV(UPLO,TRANS,DIAG,N,AP,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,N
```

```
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      REAL AP(*),X(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL ZERO
      PARAMETER (ZERO=0.0E+0)
*     ..
*     .. Local Scalars ..
      REAL TEMP
      INTEGER I,INFO,IX,J,JX,K,KK,KX
      LOGICAL NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (INCX.EQ.0) THEN
          INFO = 7
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('STPMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOUNIT = LSAME(DIAG,'N')
```

```
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX  too small for descending loops.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of AP are
*     accessed sequentially with one pass through AP.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x:= A*x.
*
          IF (LSAME(UPLO,'U')) THEN
              KK = 1
              IF (INCX.EQ.1) THEN
                  DO 20 J = 1,N
                      IF (X(J).NE.ZERO) THEN
                          TEMP = X(J)
                          K = KK
                          DO 10 I = 1,J - 1
                              X(I) = X(I) + TEMP*AP(K)
                              K = K + 1
   10                     CONTINUE
                          IF (NOUNIT) X(J) = X(J)*AP(KK+J-1)
                      END IF
                      KK = KK + J
   20             CONTINUE
              ELSE
                  JX = KX
                  DO 40 J = 1,N
                      IF (X(JX).NE.ZERO) THEN
                          TEMP = X(JX)
                          IX = KX
                          DO 30 K = KK,KK + J - 2
                              X(IX) = X(IX) + TEMP*AP(K)
                              IX = IX + INCX
   30                     CONTINUE
                          IF (NOUNIT) X(JX) = X(JX)*AP(KK+J-1)
                      END IF
                      JX = JX + INCX
                      KK = KK + J
   40             CONTINUE
              END IF
          ELSE
              KK = (N* (N+1))/2
```

```
            IF (INCX.EQ.1) THEN
                DO 60 J = N,1,-1
                    IF (X(J).NE.ZERO) THEN
                        TEMP = X(J)
                        K = KK
                        DO 50 I = N,J + 1,-1
                            X(I) = X(I) + TEMP*AP(K)
                            K = K - 1
   50                   CONTINUE
                        IF (NOUNIT) X(J) = X(J)*AP(KK-N+J)
                    END IF
                    KK = KK - (N-J+1)
   60           CONTINUE
            ELSE
                KX = KX + (N-1)*INCX
                JX = KX
                DO 80 J = N,1,-1
                    IF (X(JX).NE.ZERO) THEN
                        TEMP = X(JX)
                        IX = KX
                        DO 70 K = KK,KK - (N- (J+1)),-1
                            X(IX) = X(IX) + TEMP*AP(K)
                            IX = IX - INCX
   70                   CONTINUE
                        IF (NOUNIT) X(JX) = X(JX)*AP(KK-N+J)
                    END IF
                    JX = JX - INCX
                    KK = KK - (N-J+1)
   80           CONTINUE
            END IF
        END IF
    ELSE
*
*       Form  x := A**T*x.
*
        IF (LSAME(UPLO,'U')) THEN
            KK = (N* (N+1))/2
            IF (INCX.EQ.1) THEN
                DO 100 J = N,1,-1
                    TEMP = X(J)
                    IF (NOUNIT) TEMP = TEMP*AP(KK)
                    K = KK - 1
                    DO 90 I = J - 1,1,-1
                        TEMP = TEMP + AP(K)*X(I)
                        K = K - 1
   90               CONTINUE
                    X(J) = TEMP
                    KK = KK - J
  100           CONTINUE
            ELSE
```

```
                JX = KX + (N-1)*INCX
                DO 120 J = N,1,-1
                    TEMP = X(JX)
                    IX = JX
                    IF (NOUNIT) TEMP = TEMP*AP(KK)
                    DO 110 K = KK - 1,KK - J + 1,-1
                        IX = IX - INCX
                        TEMP = TEMP + AP(K)*X(IX)
 110                CONTINUE
                    X(JX) = TEMP
                    JX = JX - INCX
                    KK = KK - J
 120            CONTINUE
            END IF
        ELSE
            KK = 1
            IF (INCX.EQ.1) THEN
                DO 140 J = 1,N
                    TEMP = X(J)
                    IF (NOUNIT) TEMP = TEMP*AP(KK)
                    K = KK + 1
                    DO 130 I = J + 1,N
                        TEMP = TEMP + AP(K)*X(I)
                        K = K + 1
 130                CONTINUE
                    X(J) = TEMP
                    KK = KK + (N-J+1)
 140            CONTINUE
            ELSE
                JX = KX
                DO 160 J = 1,N
                    TEMP = X(JX)
                    IX = JX
                    IF (NOUNIT) TEMP = TEMP*AP(KK)
                    DO 150 K = KK + 1,KK + N - J
                        IX = IX + INCX
                        TEMP = TEMP + AP(K)*X(IX)
 150                CONTINUE
                    X(JX) = TEMP
                    JX = JX + INCX
                    KK = KK + (N-J+1)
 160            CONTINUE
            END IF
        END IF
    END IF
*
    RETURN
*
*   End of STPMV .
*
```

```
      END
```

———-

10.13.4 lisp code

— BLAS 2 stpmv —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun stpmv (uplo trans diag n ap x incx)
    (declare (type (array single-float (*)) x ap)
             (type (f2cl-lib:integer4) incx n)
             (type (string *) diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (ap single-float ap-%data% ap-%offset%)
         (x single-float x-%data% x-%offset%))
      (prog ((nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0) (k 0) (kk 0)
             (kx 0) (temp 0.0))
        (declare (type f2cl-lib:logical nounit)
                 (type (f2cl-lib:integer4) i info ix j jx k kk kx)
                 (type (single-float) temp))
        (setf info 0)
        (cond
         ((and (not (lsame uplo "U")) (not (lsame uplo "L"))) (setf info 1))
         ((and (not (lsame trans "N")) (not (lsame trans "T"))
               (not (lsame trans "C")))
          (setf info 2))
         ((and (not (lsame diag "U")) (not (lsame diag "N"))) (setf info 3))
         ((< n 0) (setf info 4)) ((= incx 0) (setf info 7)))
        (cond
         ((/= info 0)
          (multiple-value-bind (var-0 var-1)
              (xerbla "STPMV " info)
            (declare (ignore var-0))
            (when var-1 (setf info var-1)))
          (go end_label)))
        (if (= n 0)
            (go end_label))
        (setf nounit (lsame diag "N"))
        (cond
         ((<= incx 0)
          (setf kx
```

```
          (f2cl-lib:int-sub 1
                            (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                              incx))))
 ((/= incx 1) (setf kx 1)))
(cond
 ((lsame trans "N")
  (cond
   ((lsame uplo "U") (setf kk 1)
    (cond
     ((= incx 1)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
           ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
            (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
            (setf k kk)
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i
                              (f2cl-lib:int-add j
                                                (f2cl-lib:int-sub 1)))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                        (+
                         (f2cl-lib:fref x-%data% (i) ((1 *))
                                        x-%offset%)
                         (* temp
                            (f2cl-lib:fref ap-%data% (k) ((1 *))
                                           ap-%offset%))))
                (setf k (f2cl-lib:int-add k 1))
               label10))
            (if nounit
                (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                        (*
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%)
                         (f2cl-lib:fref ap-%data%
                                        ((f2cl-lib:int-sub
                                          (f2cl-lib:int-add kk j) 1))
                                        ((1 *)) ap-%offset%))))))
          (setf kk (f2cl-lib:int-add kk j))
         label20)))
     (t (setf jx kx)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
           ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
            (setf temp
```

```
                (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
        (setf ix kx)
        (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                      ((> k
                          (f2cl-lib:int-add kk j
                                            (f2cl-lib:int-sub 2)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                  (+
                   (f2cl-lib:fref x-%data% (ix) ((1 *))
                                  x-%offset%)
                   (* temp
                      (f2cl-lib:fref ap-%data% (k) ((1 *))
                                     ap-%offset%))))
            (setf ix (f2cl-lib:int-add ix incx))
           label30))
        (if nounit
            (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                  (*
                   (f2cl-lib:fref x-%data% (jx) ((1 *))
                                  x-%offset%)
                   (f2cl-lib:fref ap-%data%
                                  ((f2cl-lib:int-sub
                                    (f2cl-lib:int-add kk j) 1))
                                  ((1 *)) ap-%offset%))))))
      (setf jx (f2cl-lib:int-add jx incx))
      (setf kk (f2cl-lib:int-add kk j))
     label40)))))
(t (setf kk (the f2cl-lib:integer4 (truncate (* n (+ n 1)) 2)))
 (cond
  ((= incx 1)
   (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                 ((> j 1) nil)
    (tagbody
      (cond
       ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
        (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
        (setf k kk)
        (f2cl-lib:fdo (i n
                       (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i (f2cl-lib:int-add j 1)) nil)
          (tagbody
            (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                  (+
                   (f2cl-lib:fref x-%data% (i) ((1 *))
                                  x-%offset%)
                   (* temp
                      (f2cl-lib:fref ap-%data% (k) ((1 *))
                                     ap-%offset%))))
```

```
                (setf k (f2cl-lib:int-sub k 1))
               label50))
            (if nounit
                (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                        (*
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%)
                         (f2cl-lib:fref ap-%data%
                                        ((f2cl-lib:int-add
                                          (f2cl-lib:int-sub kk n) j))
                                        ((1 *)) ap-%offset%))))))
          (setf kk
                  (f2cl-lib:int-sub kk
                                    (f2cl-lib:int-add
                                     (f2cl-lib:int-sub n j) 1)))
        label60)))
      (t
       (setf kx
               (f2cl-lib:int-add kx
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incx)))
       (setf jx kx)
       (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     ((> j 1) nil)
         (tagbody
           (cond
            ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
             (setf temp
                     (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
             (setf ix kx)
             (f2cl-lib:fdo (k kk
                            (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                           ((> k
                               (f2cl-lib:int-add kk
                                                 (f2cl-lib:int-sub
                                                  (f2cl-lib:int-add n
                                                                    (f2cl-lib:int-sub
                                                                     (f2cl-lib:int-add
                                                                      j
                                                                      1))))))
                           nil)
               (tagbody
                 (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                         (+
                          (f2cl-lib:fref x-%data% (ix) ((1 *))
                                         x-%offset%)
                          (* temp
                             (f2cl-lib:fref ap-%data% (k) ((1 *))
                                            ap-%offset%))))
                 (setf ix (f2cl-lib:int-sub ix incx))
```

```
                label70))
              (if nounit
                  (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                          (*
                           (f2cl-lib:fref x-%data% (jx) ((1 *))
                                          x-%offset%)
                           (f2cl-lib:fref ap-%data%
                                          ((f2cl-lib:int-add
                                            (f2cl-lib:int-sub kk n) j))
                                          ((1 *)) ap-%offset%))))))
            (setf jx (f2cl-lib:int-sub jx incx))
            (setf kk
                    (f2cl-lib:int-sub kk
                                      (f2cl-lib:int-add
                                       (f2cl-lib:int-sub n j) 1)))
          label80)))))))
(t
 (cond
  ((lsame uplo "U")
   (setf kk (the f2cl-lib:integer4 (truncate (* n (+ n 1)) 2)))
   (cond
    ((= incx 1)
     (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   ((> j 1) nil)
       (tagbody
         (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
         (if nounit
             (setf temp
                     (* temp
                        (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                       ap-%offset%))))
         (setf k (f2cl-lib:int-sub kk 1))
         (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                        (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                       ((> i 1) nil)
           (tagbody
             (setf temp
                     (+ temp
                        (*
                         (f2cl-lib:fref ap-%data% (k) ((1 *))
                                        ap-%offset%)
                         (f2cl-lib:fref x-%data% (i) ((1 *))
                                        x-%offset%))))
             (setf k (f2cl-lib:int-sub k 1))
            label90))
         (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
         (setf kk (f2cl-lib:int-sub kk j))
        label100)))
    (t
     (setf jx
```

```
            (f2cl-lib:int-add kx
                              (f2cl-lib:int-mul
                               (f2cl-lib:int-sub n 1) incx)))
    (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                  ((> j 1) nil)
      (tagbody
        (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
        (setf ix jx)
        (if nounit
            (setf temp
                    (* temp
                       (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                      ap-%offset%))))
        (f2cl-lib:fdo (k (f2cl-lib:int-add kk (f2cl-lib:int-sub 1))
                       (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                      ((> k
                          (f2cl-lib:int-add kk (f2cl-lib:int-sub j)
                                            1))
                       nil)
          (tagbody
            (setf ix (f2cl-lib:int-sub ix incx))
            (setf temp
                    (+ temp
                       (*
                        (f2cl-lib:fref ap-%data% (k) ((1 *))
                                       ap-%offset%)
                        (f2cl-lib:fref x-%data% (ix) ((1 *))
                                       x-%offset%))))
           label110))
        (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
        (setf jx (f2cl-lib:int-sub jx incx))
        (setf kk (f2cl-lib:int-sub kk j))
       label120)))))
(t (setf kk 1)
 (cond
  ((= incx 1)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
       (if nounit
           (setf temp
                   (* temp
                      (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                     ap-%offset%))))
       (setf k (f2cl-lib:int-add kk 1))
       (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                      (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
```

```
                (setf temp
                        (+ temp
                           (*
                            (f2cl-lib:fref ap-%data% (k) ((1 *))
                                           ap-%offset%)
                            (f2cl-lib:fref x-%data% (i) ((1 *))
                                           x-%offset%))))
                (setf k (f2cl-lib:int-add k 1))
               label130))
            (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
            (setf kk
                    (f2cl-lib:int-add kk
                                      (f2cl-lib:int-add
                                       (f2cl-lib:int-sub n j) 1)))
        label140)))
      (t (setf jx kx)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
            (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
            (setf ix jx)
            (if nounit
                (setf temp
                        (* temp
                           (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                          ap-%offset%))))
            (f2cl-lib:fdo (k (f2cl-lib:int-add kk 1)
                           (f2cl-lib:int-add k 1))
                          ((> k
                              (f2cl-lib:int-add kk n
                                                (f2cl-lib:int-sub j)))
                           nil)
              (tagbody
                (setf ix (f2cl-lib:int-add ix incx))
                (setf temp
                        (+ temp
                           (*
                            (f2cl-lib:fref ap-%data% (k) ((1 *))
                                           ap-%offset%)
                            (f2cl-lib:fref x-%data% (ix) ((1 *))
                                           x-%offset%))))
               label150))
            (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
            (setf jx (f2cl-lib:int-add jx incx))
            (setf kk
                    (f2cl-lib:int-add kk
                                      (f2cl-lib:int-add
                                       (f2cl-lib:int-sub n j) 1)))
        label160))))))))
(go end_label)
```

```
      end_label
       (return (values nil nil nil nil nil nil nil))))))
```

———-

10.14 stpsv - solve systems of equations $Ax = b$ or $A^T x = b$

10.14.1 Axiom unit tests

— **stpsv.input** —

```
)set break resume
)sys rm -f stpsv.output
)spool stpsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

10.14.2 Axiom help page

— **stpsv.help** —

```
====================================================================
stpsv examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine stpsv (UPLO, TRANS, DIAG, N, AP, X, INCX)
           STPSV

Function/Subroutine Documentation
   subroutine stpsv (character UPLO, character TRANS, character DIAG, integer
```

```
      N, real, dimension(*) AP, real, dimension(*) X, integer INCX)
      STPSV

Purpose:
   STPSV  solves one of the systems of equations

      A*x = b,   or   A**T*x = b,

   where b and x are n element vectors and A is an n by n unit, or
   non-unit, upper or lower triangular matrix, supplied in packed form.

   No test for singularity or near-singularity is included in this
   routine. Such tests must be performed before calling this routine.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the matrix is an upper or
   lower triangular matrix as follows:

   UPLO = 'U' or 'u'   A is an upper triangular matrix.

   UPLO = 'L' or 'l'   A is a lower triangular matrix.

 TRANS
   TRANS is CHARACTER*1
   On entry, TRANS specifies the equations to be solved as follows:

   TRANS = 'N' or 'n'   A*x = b.

   TRANS = 'T' or 't'   A**T*x = b.

   TRANS = 'C' or 'c'   A**T*x = b.

 DIAG
   DIAG is CHARACTER*1
   On entry, DIAG specifies whether or not A is unit
   triangular as follows:

   DIAG = 'U' or 'u'   A is assumed to be unit triangular.

   DIAG = 'N' or 'n'   A is not assumed to be unit triangular.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
   N must be at least zero.

 AP
```

```
  AP is REAL array of DIMENSION at least ( ( n*( n + 1 ) )/2 ).

  Before entry with  UPLO = 'U' or 'u', the array AP must
  contain the upper triangular matrix packed sequentially,
  column by column, so that AP( 1 ) contains a( 1, 1 ),
  AP( 2 ) and AP( 3 ) contain a( 1, 2 ) and a( 2, 2 )
  respectively, and so on.

  Before entry with UPLO = 'L' or 'l', the array AP must
  contain the lower triangular matrix packed sequentially,
  column by column, so that AP( 1 ) contains a( 1, 1 ),
  AP( 2 ) and AP( 3 ) contain a( 2, 1 ) and a( 3, 1 )
  respectively, and so on.

  Note that when  DIAG = 'U' or 'u', the diagonal elements of
  A are not referenced, but are assumed to be unity.

X
  X is REAL array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).

  Before entry, the incremented array X must contain the n
  element right-hand side vector b. On exit, X is overwritten
  with the solution vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.
```

10.14.3 fortran code

— stpsv.f —

```
*  ===================================================================
      SUBROUTINE STPSV(UPLO,TRANS,DIAG,N,AP,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
```

```
*     .. Array Arguments ..
      REAL AP(*),X(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL ZERO
      PARAMETER (ZERO=0.0E+0)
*     ..
*     .. Local Scalars ..
      REAL TEMP
      INTEGER I,INFO,IX,J,JX,K,KK,KX
      LOGICAL NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (INCX.EQ.0) THEN
          INFO = 7
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('STPSV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
```

```
*     will be  ( N - 1 )*INCX  too small for descending loops.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of AP are
*     accessed sequentially with one pass through AP.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x := inv( A )*x.
*
          IF (LSAME(UPLO,'U')) THEN
              KK = (N* (N+1))/2
              IF (INCX.EQ.1) THEN
                  DO 20 J = N,1,-1
                      IF (X(J).NE.ZERO) THEN
                          IF (NOUNIT) X(J) = X(J)/AP(KK)
                          TEMP = X(J)
                          K = KK - 1
                          DO 10 I = J - 1,1,-1
                              X(I) = X(I) - TEMP*AP(K)
                              K = K - 1
   10                     CONTINUE
                      END IF
                      KK = KK - J
   20             CONTINUE
              ELSE
                  JX = KX + (N-1)*INCX
                  DO 40 J = N,1,-1
                      IF (X(JX).NE.ZERO) THEN
                          IF (NOUNIT) X(JX) = X(JX)/AP(KK)
                          TEMP = X(JX)
                          IX = JX
                          DO 30 K = KK - 1,KK - J + 1,-1
                              IX = IX - INCX
                              X(IX) = X(IX) - TEMP*AP(K)
   30                     CONTINUE
                      END IF
                      JX = JX - INCX
                      KK = KK - J
   40             CONTINUE
              END IF
          ELSE
              KK = 1
              IF (INCX.EQ.1) THEN
                  DO 60 J = 1,N
```

```
                    IF (X(J).NE.ZERO) THEN
                        IF (NOUNIT) X(J) = X(J)/AP(KK)
                        TEMP = X(J)
                        K = KK + 1
                        DO 50 I = J + 1,N
                            X(I) = X(I) - TEMP*AP(K)
                            K = K + 1
   50                   CONTINUE
                    END IF
                    KK = KK + (N-J+1)
   60           CONTINUE
            ELSE
                JX = KX
                DO 80 J = 1,N
                    IF (X(JX).NE.ZERO) THEN
                        IF (NOUNIT) X(JX) = X(JX)/AP(KK)
                        TEMP = X(JX)
                        IX = JX
                        DO 70 K = KK + 1,KK + N - J
                            IX = IX + INCX
                            X(IX) = X(IX) - TEMP*AP(K)
   70                   CONTINUE
                    END IF
                    JX = JX + INCX
                    KK = KK + (N-J+1)
   80           CONTINUE
            END IF
        END IF
    ELSE
*
*       Form  x := inv( A**T )*x.
*
        IF (LSAME(UPLO,'U')) THEN
            KK = 1
            IF (INCX.EQ.1) THEN
                DO 100 J = 1,N
                    TEMP = X(J)
                    K = KK
                    DO 90 I = 1,J - 1
                        TEMP = TEMP - AP(K)*X(I)
                        K = K + 1
   90               CONTINUE
                    IF (NOUNIT) TEMP = TEMP/AP(KK+J-1)
                    X(J) = TEMP
                    KK = KK + J
  100           CONTINUE
            ELSE
                JX = KX
                DO 120 J = 1,N
                    TEMP = X(JX)
```

```
                    IX = KX
                    DO 110 K = KK,KK + J - 2
                        TEMP = TEMP - AP(K)*X(IX)
                        IX = IX + INCX
 110                CONTINUE
                    IF (NOUNIT) TEMP = TEMP/AP(KK+J-1)
                    X(JX) = TEMP
                    JX = JX + INCX
                    KK = KK + J
 120            CONTINUE
            END IF
        ELSE
            KK = (N* (N+1))/2
            IF (INCX.EQ.1) THEN
                DO 140 J = N,1,-1
                    TEMP = X(J)
                    K = KK
                    DO 130 I = N,J + 1,-1
                        TEMP = TEMP - AP(K)*X(I)
                        K = K - 1
 130                CONTINUE
                    IF (NOUNIT) TEMP = TEMP/AP(KK-N+J)
                    X(J) = TEMP
                    KK = KK - (N-J+1)
 140            CONTINUE
            ELSE
                KX = KX + (N-1)*INCX
                JX = KX
                DO 160 J = N,1,-1
                    TEMP = X(JX)
                    IX = KX
                    DO 150 K = KK,KK - (N- (J+1)),-1
                        TEMP = TEMP - AP(K)*X(IX)
                        IX = IX - INCX
 150                CONTINUE
                    IF (NOUNIT) TEMP = TEMP/AP(KK-N+J)
                    X(JX) = TEMP
                    JX = JX - INCX
                    KK = KK - (N-J+1)
 160            CONTINUE
            END IF
        END IF
    END IF
*
    RETURN
*
*   End of STPSV .
*
    END
```

———-

10.14.4 lisp code

— BLAS 2 stpsv —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun stpsv (uplo trans diag n ap x incx)
    (declare (type (array single-float (*)) x ap)
             (type (f2cl-lib:integer4) incx n)
             (type (string *) diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (ap single-float ap-%data% ap-%offset%)
         (x single-float x-%data% x-%offset%))
      (prog ((nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0) (k 0) (kk 0)
             (kx 0) (temp 0.0))
        (declare (type f2cl-lib:logical nounit)
                 (type (f2cl-lib:integer4) i info ix j jx k kk kx)
                 (type (single-float) temp))
        (setf info 0)
        (cond
         ((and (not (lsame uplo "U")) (not (lsame uplo "L"))) (setf info 1))
         ((and (not (lsame trans "N")) (not (lsame trans "T"))
               (not (lsame trans "C")))
          (setf info 2))
         ((and (not (lsame diag "U")) (not (lsame diag "N"))) (setf info 3))
         ((< n 0) (setf info 4)) ((= incx 0) (setf info 7)))
        (cond
         ((/= info 0)
          (multiple-value-bind (var-0 var-1)
              (xerbla "STPSV " info)
            (declare (ignore var-0))
            (when var-1 (setf info var-1)))
          (go end_label)))
        (if (= n 0)
            (go end_label))
        (setf nounit (lsame diag "N"))
        (cond
         ((<= incx 0)
          (setf kx
                  (f2cl-lib:int-sub 1
                                    (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                      incx))))
```

```
 ((/= incx 1) (setf kx 1)))
(cond
 ((lsame trans "N")
  (cond
   ((lsame uplo "U")
    (setf kk (the f2cl-lib:integer4 (truncate (* n (+ n 1)) 2)))
    (cond
     ((= incx 1)
      (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                    ((> j 1) nil)
        (tagbody
          (cond
           ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
            (if nounit
                (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                        (/
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%)
                         (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                        ap-%offset%))))
            (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
            (setf k (f2cl-lib:int-sub kk 1))
            (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                           (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                          ((> i 1) nil)
              (tagbody
                (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                        (-
                         (f2cl-lib:fref x-%data% (i) ((1 *))
                                        x-%offset%)
                         (* temp
                            (f2cl-lib:fref ap-%data% (k) ((1 *))
                                           ap-%offset%))))
                (setf k (f2cl-lib:int-sub k 1))
               label10))))
          (setf kk (f2cl-lib:int-sub kk j))
         label20)))
     (t
      (setf jx
              (f2cl-lib:int-add kx
                                (f2cl-lib:int-mul
                                 (f2cl-lib:int-sub n 1) incx)))
      (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                    ((> j 1) nil)
        (tagbody
          (cond
           ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
            (if nounit
                (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                        (/
```

```
                        (f2cl-lib:fref x-%data% (jx) ((1 *))
                                       x-%offset%)
                        (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                       ap-%offset%))))
          (setf temp
                  (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
          (setf ix jx)
          (f2cl-lib:fdo (k (f2cl-lib:int-add kk (f2cl-lib:int-sub 1))
                         (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                        ((> k
                            (f2cl-lib:int-add kk (f2cl-lib:int-sub j)
                                              1))
                         nil)
            (tagbody
              (setf ix (f2cl-lib:int-sub ix incx))
              (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                      (-
                       (f2cl-lib:fref x-%data% (ix) ((1 *))
                                      x-%offset%)
                       (* temp
                          (f2cl-lib:fref ap-%data% (k) ((1 *))
                                         ap-%offset%))))
             label30))))
        (setf jx (f2cl-lib:int-sub jx incx))
        (setf kk (f2cl-lib:int-sub kk j))
       label40)))))
(t (setf kk 1)
 (cond
  ((= incx 1)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
         (if nounit
             (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                     (/
                      (f2cl-lib:fref x-%data% (j) ((1 *))
                                     x-%offset%)
                      (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                     ap-%offset%))))
         (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
         (setf k (f2cl-lib:int-add kk 1))
         (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                        (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                     (-
                      (f2cl-lib:fref x-%data% (i) ((1 *))
```

```
                                        x-%offset%)
                          (* temp
                             (f2cl-lib:fref ap-%data% (k) ((1 *))
                                            ap-%offset%))))
                 (setf k (f2cl-lib:int-add k 1))
                label50))))
            (setf kk
                    (f2cl-lib:int-add kk
                                      (f2cl-lib:int-add
                                       (f2cl-lib:int-sub n j) 1)))
           label60)))
        (t (setf jx kx)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (cond
              ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
               (if nounit
                   (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                           (/
                            (f2cl-lib:fref x-%data% (jx) ((1 *))
                                           x-%offset%)
                            (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                           ap-%offset%))))
               (setf temp
                       (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
               (setf ix jx)
               (f2cl-lib:fdo (k (f2cl-lib:int-add kk 1)
                              (f2cl-lib:int-add k 1))
                             ((> k
                                 (f2cl-lib:int-add kk n
                                                   (f2cl-lib:int-sub j)))
                              nil)
                 (tagbody
                   (setf ix (f2cl-lib:int-add ix incx))
                   (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                           (-
                            (f2cl-lib:fref x-%data% (ix) ((1 *))
                                           x-%offset%)
                            (* temp
                               (f2cl-lib:fref ap-%data% (k) ((1 *))
                                              ap-%offset%))))
                 label70))))
            (setf jx (f2cl-lib:int-add jx incx))
            (setf kk
                    (f2cl-lib:int-add kk
                                      (f2cl-lib:int-add
                                       (f2cl-lib:int-sub n j) 1)))
           label80)))))))
(t
```

```
(cond
 ((lsame uplo "U") (setf kk 1)
  (cond
   ((= incx 1)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
        (setf k kk)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i
                          (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf temp
                    (- temp
                       (*
                        (f2cl-lib:fref ap-%data% (k) ((1 *))
                                       ap-%offset%)
                        (f2cl-lib:fref x-%data% (i) ((1 *))
                                       x-%offset%))))
            (setf k (f2cl-lib:int-add k 1))
           label90))
        (if nounit
            (setf temp
                    (/ temp
                       (f2cl-lib:fref ap-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add kk j) 1))
                                      ((1 *)) ap-%offset%))))
        (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
        (setf kk (f2cl-lib:int-add kk j))
       label100)))
   (t (setf jx kx)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
        (setf ix kx)
        (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                      ((> k
                          (f2cl-lib:int-add kk j
                                            (f2cl-lib:int-sub 2)))
                       nil)
          (tagbody
            (setf temp
                    (- temp
                       (*
                        (f2cl-lib:fref ap-%data% (k) ((1 *))
                                       ap-%offset%)
```

```
                        (f2cl-lib:fref x-%data% (ix) ((1 *))
                                       x-%offset%))))
            (setf ix (f2cl-lib:int-add ix incx))
           label110))
        (if nounit
            (setf temp
                    (/ temp
                       (f2cl-lib:fref ap-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add kk j) 1))
                                      ((1 *)) ap-%offset%))))
        (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
        (setf jx (f2cl-lib:int-add jx incx))
        (setf kk (f2cl-lib:int-add kk j))
       label120)))))
(t (setf kk (the f2cl-lib:integer4 (truncate (* n (+ n 1)) 2)))
 (cond
  ((= incx 1)
   (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                 ((> j 1) nil)
     (tagbody
       (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
       (setf k kk)
       (f2cl-lib:fdo (i n (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     ((> i (f2cl-lib:int-add j 1)) nil)
         (tagbody
           (setf temp
                   (- temp
                      (*
                       (f2cl-lib:fref ap-%data% (k) ((1 *))
                                      ap-%offset%)
                       (f2cl-lib:fref x-%data% (i) ((1 *))
                                      x-%offset%))))
           (setf k (f2cl-lib:int-sub k 1))
           label130))
       (if nounit
           (setf temp
                   (/ temp
                      (f2cl-lib:fref ap-%data%
                                     ((f2cl-lib:int-add
                                       (f2cl-lib:int-sub kk n) j))
                                     ((1 *)) ap-%offset%))))
       (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
       (setf kk
               (f2cl-lib:int-sub kk
                                 (f2cl-lib:int-add
                                  (f2cl-lib:int-sub n j) 1)))
       label140)))
  (t
   (setf kx
```

```
            (f2cl-lib:int-add kx
                              (f2cl-lib:int-mul
                               (f2cl-lib:int-sub n 1) incx)))
        (setf jx kx)
        (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                      ((> j 1) nil)
          (tagbody
            (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
            (setf ix kx)
            (f2cl-lib:fdo (k kk (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                          ((> k
                              (f2cl-lib:int-add kk
                                                (f2cl-lib:int-sub
                                                 (f2cl-lib:int-add n
                                                                   (f2cl-lib:int-sub
                                                                    (f2cl-lib:int-add
                                                                     j
                                                                     1))))))
                           nil)
              (tagbody
                (setf temp
                        (- temp
                           (*
                            (f2cl-lib:fref ap-%data% (k) ((1 *))
                                           ap-%offset%)
                            (f2cl-lib:fref x-%data% (ix) ((1 *))
                                           x-%offset%))))
                (setf ix (f2cl-lib:int-sub ix incx))
               label150))
            (if nounit
                (setf temp
                        (/ temp
                           (f2cl-lib:fref ap-%data%
                                          ((f2cl-lib:int-add
                                            (f2cl-lib:int-sub kk n) j))
                                          ((1 *)) ap-%offset%))))
            (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
            (setf jx (f2cl-lib:int-sub jx incx))
            (setf kk
                    (f2cl-lib:int-sub kk
                                      (f2cl-lib:int-add
                                       (f2cl-lib:int-sub n j) 1)))
         label160))))))))
 (go end_label)
end_label
 (return (values nil nil nil nil nil nil nil))))))
```

———-

10.15 strmv - matrix-vector $x = Ax$ or $x = A^T x$

10.15.1 Axiom unit tests

— strmv.input —

```
)set break resume
)sys rm -f strmv.output
)spool strmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

10.15.2 Axiom help page

— strmv.help —

```
======================================================================
strmv examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine strmv (UPLO, TRANS, DIAG, N, A, LDA, X, INCX)
           STRMV

Function/Subroutine Documentation
   subroutine strmv (character UPLO, character TRANS, character DIAG, integer
       N, real, dimension(lda,*) A, integer LDA, real, dimension(*) X, integer
       INCX)
       STRMV

Purpose:
   STRMV  performs one of the matrix-vector operations

      x := A*x,   or   x := A**T*x,
```

```
   where x is an n element vector and  A is an n by n unit, or non-unit,
   upper or lower triangular matrix.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the matrix is an upper or
   lower triangular matrix as follows:

   UPLO = 'U' or 'u'   A is an upper triangular matrix.

   UPLO = 'L' or 'l'   A is a lower triangular matrix.

 TRANS
   TRANS is CHARACTER*1
   On entry, TRANS specifies the operation to be performed as follows:

   TRANS = 'N' or 'n'   x := A*x.

   TRANS = 'T' or 't'   x := A**T*x.

   TRANS = 'C' or 'c'   x := A**T*x.

 DIAG
   DIAG is CHARACTER*1
   On entry, DIAG specifies whether or not A is unit
   triangular as follows:

   DIAG = 'U' or 'u'   A is assumed to be unit triangular.

   DIAG = 'N' or 'n'   A is not assumed to be unit triangular.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
   N must be at least zero.

 A
   A is REAL array of DIMENSION ( LDA, n ).

   Before entry with  UPLO = 'U' or 'u', the leading n by n
   upper triangular part of the array A must contain the upper
   triangular matrix and the strictly lower triangular part of
   A is not referenced.

   Before entry with UPLO = 'L' or 'l', the leading n by n
   lower triangular part of the array A must contain the lower
   triangular matrix and the strictly upper triangular part of
   A is not referenced.
```

```
  Note that when  DIAG = 'U' or 'u', the diagonal elements of
  A are not referenced either, but are assumed to be unity.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, n ).

X
  X is REAL array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the n
  element vector x. On exit, X is overwritten with the
  tranformed vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.

  The vector and matrix arguments are not referenced when N = 0, or M = 0
```

———-

10.15.3 fortran code

— strmv.f —

```
*  ====================================================================
      SUBROUTINE STRMV(UPLO,TRANS,DIAG,N,A,LDA,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,LDA,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      REAL A(LDA,*),X(*)
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
```

```
      REAL ZERO
      PARAMETER (ZERO=0.0E+0)
*     ..
*     .. Local Scalars ..
      REAL TEMP
      INTEGER I,INFO,IX,J,JX,KX
      LOGICAL NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 6
      ELSE IF (INCX.EQ.0) THEN
          INFO = 8
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('STRMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX  too small for descending loops.
*
```

```
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x := A*x.
*
          IF (LSAME(UPLO,'U')) THEN
              IF (INCX.EQ.1) THEN
                  DO 20 J = 1,N
                      IF (X(J).NE.ZERO) THEN
                          TEMP = X(J)
                          DO 10 I = 1,J - 1
                              X(I) = X(I) + TEMP*A(I,J)
   10                     CONTINUE
                          IF (NOUNIT) X(J) = X(J)*A(J,J)
                      END IF
   20             CONTINUE
              ELSE
                  JX = KX
                  DO 40 J = 1,N
                      IF (X(JX).NE.ZERO) THEN
                          TEMP = X(JX)
                          IX = KX
                          DO 30 I = 1,J - 1
                              X(IX) = X(IX) + TEMP*A(I,J)
                              IX = IX + INCX
   30                     CONTINUE
                          IF (NOUNIT) X(JX) = X(JX)*A(J,J)
                      END IF
                      JX = JX + INCX
   40             CONTINUE
              END IF
          ELSE
              IF (INCX.EQ.1) THEN
                  DO 60 J = N,1,-1
                      IF (X(J).NE.ZERO) THEN
                          TEMP = X(J)
                          DO 50 I = N,J + 1,-1
                              X(I) = X(I) + TEMP*A(I,J)
   50                     CONTINUE
                          IF (NOUNIT) X(J) = X(J)*A(J,J)
                      END IF
   60             CONTINUE
```

```
            ELSE
                KX = KX + (N-1)*INCX
                JX = KX
                DO 80 J = N,1,-1
                    IF (X(JX).NE.ZERO) THEN
                        TEMP = X(JX)
                        IX = KX
                        DO 70 I = N,J + 1,-1
                            X(IX) = X(IX) + TEMP*A(I,J)
                            IX = IX - INCX
   70                   CONTINUE
                        IF (NOUNIT) X(JX) = X(JX)*A(J,J)
                    END IF
                    JX = JX - INCX
   80           CONTINUE
            END IF
        END IF
     ELSE
*
*       Form  x := A**T*x.
*
        IF (LSAME(UPLO,'U')) THEN
            IF (INCX.EQ.1) THEN
                DO 100 J = N,1,-1
                    TEMP = X(J)
                    IF (NOUNIT) TEMP = TEMP*A(J,J)
                    DO 90 I = J - 1,1,-1
                        TEMP = TEMP + A(I,J)*X(I)
   90               CONTINUE
                    X(J) = TEMP
  100           CONTINUE
            ELSE
                JX = KX + (N-1)*INCX
                DO 120 J = N,1,-1
                    TEMP = X(JX)
                    IX = JX
                    IF (NOUNIT) TEMP = TEMP*A(J,J)
                    DO 110 I = J - 1,1,-1
                        IX = IX - INCX
                        TEMP = TEMP + A(I,J)*X(IX)
  110               CONTINUE
                    X(JX) = TEMP
                    JX = JX - INCX
  120           CONTINUE
            END IF
        ELSE
            IF (INCX.EQ.1) THEN
                DO 140 J = 1,N
                    TEMP = X(J)
                    IF (NOUNIT) TEMP = TEMP*A(J,J)
```

```
                    DO 130 I = J + 1,N
                        TEMP = TEMP + A(I,J)*X(I)
  130               CONTINUE
                    X(J) = TEMP
  140           CONTINUE
            ELSE
                JX = KX
                DO 160 J = 1,N
                    TEMP = X(JX)
                    IX = JX
                    IF (NOUNIT) TEMP = TEMP*A(J,J)
                    DO 150 I = J + 1,N
                        IX = IX + INCX
                        TEMP = TEMP + A(I,J)*X(IX)
  150               CONTINUE
                    X(JX) = TEMP
                    JX = JX + INCX
  160           CONTINUE
            END IF
        END IF
    END IF
*
    RETURN
*
*   End of STRMV .
*
    END
```

———-

10.15.4 lisp code

— BLAS 2 strmv —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun strmv (uplo trans diag n a lda x incx)
    (declare (type (array single-float (*)) x a)
             (type (f2cl-lib:integer4) incx lda n)
             (type (string *) diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a single-float a-%data% a-%offset%)
         (x single-float x-%data% x-%offset%))
```

```
(prog ((nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0) (kx 0) (temp 0.0))
  (declare (type f2cl-lib:logical nounit)
           (type (f2cl-lib:integer4) i info ix j jx kx)
           (type (single-float) temp))
  (setf info 0)
  (cond
   ((and (not (lsame uplo "U")) (not (lsame uplo "L"))) (setf info 1))
   ((and (not (lsame trans "N")) (not (lsame trans "T"))
         (not (lsame trans "C")))
    (setf info 2))
   ((and (not (lsame diag "U")) (not (lsame diag "N"))) (setf info 3))
   ((< n 0) (setf info 4))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info 6))
   ((= incx 0) (setf info 8)))
  (cond
   ((/= info 0)
    (multiple-value-bind (var-0 var-1)
        (xerbla "STRMV " info)
      (declare (ignore var-0))
      (when var-1 (setf info var-1)))
    (go end_label)))
  (if (= n 0)
      (go end_label))
  (setf nounit (lsame diag "N"))
  (cond
   ((<= incx 0)
    (setf kx
            (f2cl-lib:int-sub 1
                              (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                incx))))
   ((/= incx 1) (setf kx 1)))
  (cond
   ((lsame trans "N")
    (cond
     ((lsame uplo "U")
      (cond
       ((= incx 1)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (cond
             ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
              (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i
                                (f2cl-lib:int-add j
                                                  (f2cl-lib:int-sub 1)))
                             nil)
                (tagbody
```

```
                (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                        (+
                         (f2cl-lib:fref x-%data% (i) ((1 *))
                                        x-%offset%)
                         (* temp
                            (f2cl-lib:fref a-%data% (i j)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
              label10))
            (if nounit
                (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                        (*
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%)
                         (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                        a-%offset%))))))
        label20)))
    (t (setf jx kx)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (cond
          ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
           (setf temp
                   (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
           (setf ix kx)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i
                             (f2cl-lib:int-add j
                                               (f2cl-lib:int-sub 1)))
                          nil)
             (tagbody
               (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                       (+
                        (f2cl-lib:fref x-%data% (ix) ((1 *))
                                       x-%offset%)
                        (* temp
                           (f2cl-lib:fref a-%data% (i j)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
               (setf ix (f2cl-lib:int-add ix incx))
              label30))
           (if nounit
               (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                       (*
                        (f2cl-lib:fref x-%data% (jx) ((1 *))
                                       x-%offset%)
                        (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                       a-%offset%))))))
         (setf jx (f2cl-lib:int-add jx incx))
```

```
      label40)))))
  (t
   (cond
    ((= incx 1)
     (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   ((> j 1) nil)
       (tagbody
         (cond
          ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
           (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
           (f2cl-lib:fdo (i n
                          (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                         ((> i (f2cl-lib:int-add j 1)) nil)
             (tagbody
               (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                       (+
                        (f2cl-lib:fref x-%data% (i) ((1 *))
                                       x-%offset%)
                        (* temp
                           (f2cl-lib:fref a-%data% (i j)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
              label50))
           (if nounit
               (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                       (*
                        (f2cl-lib:fref x-%data% (j) ((1 *))
                                       x-%offset%)
                        (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                       a-%offset%))))))
      label60)))
    (t
     (setf kx
             (f2cl-lib:int-add kx
                               (f2cl-lib:int-mul
                                (f2cl-lib:int-sub n 1) incx)))
     (setf jx kx)
     (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   ((> j 1) nil)
       (tagbody
         (cond
          ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
           (setf temp
                   (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
           (setf ix kx)
           (f2cl-lib:fdo (i n
                          (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                         ((> i (f2cl-lib:int-add j 1)) nil)
             (tagbody
               (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
```

```
                        (+
                         (f2cl-lib:fref x-%data% (ix) ((1 *))
                                        x-%offset%)
                         (* temp
                            (f2cl-lib:fref a-%data% (i j)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
                 (setf ix (f2cl-lib:int-sub ix incx))
                label70))
            (if nounit
                (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                        (*
                         (f2cl-lib:fref x-%data% (jx) ((1 *))
                                        x-%offset%)
                         (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                        a-%offset%))))))
          (setf jx (f2cl-lib:int-sub jx incx))
         label80)))))))
(t
 (cond
  ((lsame uplo "U")
   (cond
    ((= incx 1)
     (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   ((> j 1) nil)
       (tagbody
         (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
         (if nounit
             (setf temp
                     (* temp
                        (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                       a-%offset%))))
         (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                        (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                       ((> i 1) nil)
           (tagbody
             (setf temp
                     (+ temp
                        (*
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)
                         (f2cl-lib:fref x-%data% (i) ((1 *))
                                        x-%offset%))))
            label90))
         (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
        label100)))
    (t
     (setf jx
             (f2cl-lib:int-add kx
                               (f2cl-lib:int-mul
```

```
                                (f2cl-lib:int-sub n 1) incx)))
    (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                  ((> j 1) nil)
      (tagbody
        (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
        (setf ix jx)
        (if nounit
            (setf temp
                    (* temp
                       (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                      a-%offset%))))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                       (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (setf ix (f2cl-lib:int-sub ix incx))
            (setf temp
                    (+ temp
                       (*
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%)
                        (f2cl-lib:fref x-%data% (ix) ((1 *))
                                       x-%offset%))))
           label110))
        (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
        (setf jx (f2cl-lib:int-sub jx incx))
       label120)))))
(t
 (cond
  ((= incx 1)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
       (if nounit
           (setf temp
                   (* temp
                      (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%))))
       (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                      (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf temp
                   (+ temp
                      (*
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                       (f2cl-lib:fref x-%data% (i) ((1 *))
                                      x-%offset%))))
```

```
                  label130))
                (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
               label140)))
            (t (setf jx kx)
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
                 (setf ix jx)
                 (if nounit
                     (setf temp
                             (* temp
                                (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                               a-%offset%))))
                 (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                                (f2cl-lib:int-add i 1))
                               ((> i n) nil)
                   (tagbody
                     (setf ix (f2cl-lib:int-add ix incx))
                     (setf temp
                             (+ temp
                                (*
                                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                                a-%offset%)
                                 (f2cl-lib:fref x-%data% (ix) ((1 *))
                                                x-%offset%))))
                  label150))
                 (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
                 (setf jx (f2cl-lib:int-add jx incx))
                label160))))))))
      (go end_label)
     end_label
      (return (values nil nil nil nil nil nil nil nil))))))
```

———-

10.16 strsv - solve systems of equations $Ax = b$ or $A^T x = b$

10.16.1 Axiom unit tests

— **strsv.input** —

```
)set break resume
)sys rm -f strsv.output
```

```
)spool strsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

10.16.2 Axiom help page

— strsv.help —

```
==================================================================
strsv examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine strsv (UPLO, TRANS, DIAG, N, A, LDA, X, INCX)
           STRSV

Function/Subroutine Documentation
   subroutine strsv (character UPLO, character TRANS, character DIAG, integer
       N, real, dimension(lda,*) A, integer LDA, real, dimension(*) X, integer
       INCX)
       STRSV

Purpose:
   STRSV  solves one of the systems of equations

     A*x = b,   or   A**T*x = b,

   where b and x are n element vectors and A is an n by n unit, or
   non-unit, upper or lower triangular matrix.

   No test for singularity or near-singularity is included in this
   routine. Such tests must be performed before calling this routine.

Parameters:

 UPLO
   UPLO is CHARACTER*1
```

```
  On entry, UPLO specifies whether the matrix is an upper or
  lower triangular matrix as follows:

  UPLO = 'U' or 'u'   A is an upper triangular matrix.

  UPLO = 'L' or 'l'   A is a lower triangular matrix.

TRANS
  TRANS is CHARACTER*1
  On entry, TRANS specifies the equations to be solved as follows:

  TRANS = 'N' or 'n'   A*x = b.

  TRANS = 'T' or 't'   A**T*x = b.

  TRANS = 'C' or 'c'   A**T*x = b.

DIAG
  DIAG is CHARACTER*1
  On entry, DIAG specifies whether or not A is unit
  triangular as follows:

  DIAG = 'U' or 'u'   A is assumed to be unit triangular.

  DIAG = 'N' or 'n'   A is not assumed to be unit triangular.

N
  N is INTEGER
  On entry, N specifies the order of the matrix A.
  N must be at least zero.

A
  A is REAL array of DIMENSION ( LDA, n ).

  Before entry with  UPLO = 'U' or 'u', the leading n by n
  upper triangular part of the array A must contain the upper
  triangular matrix and the strictly lower triangular part of
  A is not referenced.

  Before entry with UPLO = 'L' or 'l', the leading n by n
  lower triangular part of the array A must contain the lower
  triangular matrix and the strictly upper triangular part of
  A is not referenced.

  Note that when  DIAG = 'U' or 'u', the diagonal elements of
  A are not referenced either, but are assumed to be unity.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
```

```
  in the calling (sub) program. LDA must be at least max( 1, n ).

 X
  X is REAL array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the n
  element right-hand side vector b. On exit, X is overwritten
  with the solution vector x.

 INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.
```

———-

10.16.3 fortran code

— **strsv.f** —

```
*  =====================================================================
      SUBROUTINE STRSV(UPLO,TRANS,DIAG,N,A,LDA,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,LDA,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      REAL A(LDA,*),X(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL ZERO
      PARAMETER (ZERO=0.0E+0)
*     ..
*     .. Local Scalars ..
      REAL TEMP
      INTEGER I,INFO,IX,J,JX,KX
      LOGICAL NOUNIT
*     ..
*     .. External Functions ..
```

```
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 6
      ELSE IF (INCX.EQ.0) THEN
          INFO = 8
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('STRSV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX  too small for descending loops.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
```

```
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x := inv( A )*x.
*
         IF (LSAME(UPLO,'U')) THEN
             IF (INCX.EQ.1) THEN
                 DO 20 J = N,1,-1
                     IF (X(J).NE.ZERO) THEN
                         IF (NOUNIT) X(J) = X(J)/A(J,J)
                         TEMP = X(J)
                         DO 10 I = J - 1,1,-1
                             X(I) = X(I) - TEMP*A(I,J)
   10                    CONTINUE
                     END IF
   20            CONTINUE
             ELSE
                 JX = KX + (N-1)*INCX
                 DO 40 J = N,1,-1
                     IF (X(JX).NE.ZERO) THEN
                         IF (NOUNIT) X(JX) = X(JX)/A(J,J)
                         TEMP = X(JX)
                         IX = JX
                         DO 30 I = J - 1,1,-1
                             IX = IX - INCX
                             X(IX) = X(IX) - TEMP*A(I,J)
   30                    CONTINUE
                     END IF
                     JX = JX - INCX
   40            CONTINUE
             END IF
         ELSE
             IF (INCX.EQ.1) THEN
                 DO 60 J = 1,N
                     IF (X(J).NE.ZERO) THEN
                         IF (NOUNIT) X(J) = X(J)/A(J,J)
                         TEMP = X(J)
                         DO 50 I = J + 1,N
                             X(I) = X(I) - TEMP*A(I,J)
   50                    CONTINUE
                     END IF
   60            CONTINUE
             ELSE
                 JX = KX
                 DO 80 J = 1,N
                     IF (X(JX).NE.ZERO) THEN
                         IF (NOUNIT) X(JX) = X(JX)/A(J,J)
                         TEMP = X(JX)
                         IX = JX
                         DO 70 I = J + 1,N
                             IX = IX + INCX
```

```
                            X(IX) = X(IX) - TEMP*A(I,J)
   70                    CONTINUE
                     END IF
                     JX = JX + INCX
   80            CONTINUE
            END IF
         END IF
      ELSE
*
*        Form  x := inv( A**T )*x.
*
         IF (LSAME(UPLO,'U')) THEN
            IF (INCX.EQ.1) THEN
               DO 100 J = 1,N
                  TEMP = X(J)
                  DO 90 I = 1,J - 1
                     TEMP = TEMP - A(I,J)*X(I)
   90             CONTINUE
                  IF (NOUNIT) TEMP = TEMP/A(J,J)
                  X(J) = TEMP
  100            CONTINUE
            ELSE
               JX = KX
               DO 120 J = 1,N
                  TEMP = X(JX)
                  IX = KX
                  DO 110 I = 1,J - 1
                     TEMP = TEMP - A(I,J)*X(IX)
                     IX = IX + INCX
  110             CONTINUE
                  IF (NOUNIT) TEMP = TEMP/A(J,J)
                  X(JX) = TEMP
                  JX = JX + INCX
  120            CONTINUE
            END IF
         ELSE
            IF (INCX.EQ.1) THEN
               DO 140 J = N,1,-1
                  TEMP = X(J)
                  DO 130 I = N,J + 1,-1
                     TEMP = TEMP - A(I,J)*X(I)
  130             CONTINUE
                  IF (NOUNIT) TEMP = TEMP/A(J,J)
                  X(J) = TEMP
  140            CONTINUE
            ELSE
               KX = KX + (N-1)*INCX
               JX = KX
               DO 160 J = N,1,-1
                  TEMP = X(JX)
```

```
                    IX = KX
                    DO 150 I = N,J + 1,-1
                        TEMP = TEMP - A(I,J)*X(IX)
                        IX = IX - INCX
  150               CONTINUE
                    IF (NOUNIT) TEMP = TEMP/A(J,J)
                    X(JX) = TEMP
                    JX = JX - INCX
  160            CONTINUE
             END IF
         END IF
     END IF
*
     RETURN
*
*    End of STRSV .
*
     END
```

———-

10.16.4 lisp code

— BLAS 2 strsv —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun strsv (uplo trans diag n a lda x incx)
    (declare (type (array single-float (*)) x a)
             (type (f2cl-lib:integer4) incx lda n)
             (type (string *) diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a single-float a-%data% a-%offset%)
         (x single-float x-%data% x-%offset%))
      (prog ((nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0) (kx 0) (temp 0.0))
        (declare (type f2cl-lib:logical nounit)
                 (type (f2cl-lib:integer4) i info ix j jx kx)
                 (type (single-float) temp))
        (setf info 0)
        (cond
         ((and (not (lsame uplo "U")) (not (lsame uplo "L"))) (setf info 1))
         ((and (not (lsame trans "N")) (not (lsame trans "T"))
               (not (lsame trans "C")))
```

```
  (setf info 2))
 ((and (not (lsame diag "U")) (not (lsame diag "N"))) (setf info 3))
 ((< n 0) (setf info 4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info 6))
 ((= incx 0) (setf info 8)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "STRSV " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (= n 0)
    (go end_label))
(setf nounit (lsame diag "N"))
(cond
 ((<= incx 0)
  (setf kx
          (f2cl-lib:int-sub 1
                            (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                              incx))))
 ((/= incx 1) (setf kx 1)))
(cond
 ((lsame trans "N")
  (cond
   ((lsame uplo "U")
    (cond
     ((= incx 1)
      (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                    ((> j 1) nil)
        (tagbody
          (cond
           ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
            (if nounit
                (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                        (/
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%)
                         (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                        a-%offset%))))
            (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
            (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                           (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                          ((> i 1) nil)
              (tagbody
                (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                        (-
                         (f2cl-lib:fref x-%data% (i) ((1 *))
                                        x-%offset%)
```

```
                        (* temp
                           (f2cl-lib:fref a-%data% (i j)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
              label10))))
         label20)))
     (t
      (setf jx
              (f2cl-lib:int-add kx
                                (f2cl-lib:int-mul
                                 (f2cl-lib:int-sub n 1) incx)))
      (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                    ((> j 1) nil)
        (tagbody
          (cond
           ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
            (if nounit
                (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                        (/
                         (f2cl-lib:fref x-%data% (jx) ((1 *))
                                        x-%offset%)
                         (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                        a-%offset%))))
            (setf temp
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
            (setf ix jx)
            (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                           (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                          ((> i 1) nil)
              (tagbody
                (setf ix (f2cl-lib:int-sub ix incx))
                (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                        (-
                         (f2cl-lib:fref x-%data% (ix) ((1 *))
                                        x-%offset%)
                         (* temp
                            (f2cl-lib:fref a-%data% (i j)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
              label30))))
          (setf jx (f2cl-lib:int-sub jx incx))
         label40)))))
(t
 (cond
  ((= incx 1)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
```

```
            (if nounit
                (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                        (/
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%)
                         (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                        a-%offset%))))
            (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
            (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                           (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                        (-
                         (f2cl-lib:fref x-%data% (i) ((1 *))
                                        x-%offset%)
                         (* temp
                            (f2cl-lib:fref a-%data% (i j)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
               label50))))
        label60)))
    (t (setf jx kx)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (cond
          ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
           (if nounit
               (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                       (/
                        (f2cl-lib:fref x-%data% (jx) ((1 *))
                                       x-%offset%)
                        (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                       a-%offset%))))
           (setf temp
                   (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
           (setf ix jx)
           (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                          (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf ix (f2cl-lib:int-add ix incx))
               (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                       (-
                        (f2cl-lib:fref x-%data% (ix) ((1 *))
                                       x-%offset%)
                        (* temp
                           (f2cl-lib:fref a-%data% (i j)
                                          ((1 lda) (1 *))
```

```
                                                a-%offset%))))
                     label70))))
                 (setf jx (f2cl-lib:int-add jx incx))
                label80)))))))
        (t
         (cond
          ((lsame uplo "U")
           (cond
            ((= incx 1)
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                               ((> i
                                   (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                                nil)
                   (tagbody
                     (setf temp
                             (- temp
                                (*
                                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                                a-%offset%)
                                 (f2cl-lib:fref x-%data% (i) ((1 *))
                                                x-%offset%))))
                    label90))
                 (if nounit
                     (setf temp
                             (/ temp
                                (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                               a-%offset%))))
                 (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
                label100)))
            (t (setf jx kx)
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
                 (setf ix kx)
                 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                               ((> i
                                   (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                                nil)
                   (tagbody
                     (setf temp
                             (- temp
                                (*
                                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                                a-%offset%)
                                 (f2cl-lib:fref x-%data% (ix) ((1 *))
```

```
                                        x-%offset%))))
                (setf ix (f2cl-lib:int-add ix incx))
               label110))
            (if nounit
                (setf temp
                        (/ temp
                           (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                          a-%offset%))))
            (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
            (setf jx (f2cl-lib:int-add jx incx))
           label120)))))
      (t
       (cond
        ((= incx 1)
         (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((> j 1) nil)
           (tagbody
             (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
             (f2cl-lib:fdo (i n (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                           ((> i (f2cl-lib:int-add j 1)) nil)
               (tagbody
                 (setf temp
                         (- temp
                            (*
                             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                            a-%offset%)
                             (f2cl-lib:fref x-%data% (i) ((1 *))
                                            x-%offset%))))
               label130))
             (if nounit
                 (setf temp
                         (/ temp
                            (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                           a-%offset%))))
             (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
            label140)))
        (t
         (setf kx
                 (f2cl-lib:int-add kx
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-sub n 1) incx)))
         (setf jx kx)
         (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((> j 1) nil)
           (tagbody
             (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
             (setf ix kx)
             (f2cl-lib:fdo (i n (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                           ((> i (f2cl-lib:int-add j 1)) nil)
               (tagbody
```

```
                (setf temp
                        (- temp
                           (*
                            (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                           a-%offset%)
                            (f2cl-lib:fref x-%data% (ix) ((1 *))
                                           x-%offset%))))
                (setf ix (f2cl-lib:int-sub ix incx))
               label150))
            (if nounit
                (setf temp
                        (/ temp
                           (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                          a-%offset%))))
            (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
            (setf jx (f2cl-lib:int-sub jx incx))
           label160))))))))
 (go end_label)
end_label
 (return (values nil nil nil nil nil nil nil nil))))))
```

———-

Chapter 11

BLAS Level 2 Double Matrix-Vector Operations

11.1 dgbmv - band matrix $\alpha Ax + \beta y$ or $\alpha A^T x + \beta y$

11.1.1 Axiom unit tests

— **dgbmv.input** —

```
)set break resume
)sys rm -f dgbmv.output
)spool dgbmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

11.1.2 Axiom help page

— **dgbmv.help** —

```
==========================================================================
dgbmv examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

 NAME
      DGBMV - perform one of the matrix-vector operations   y :=
      alpha*A*x + beta*y, or y := alpha*A'*x + beta*y,

 SYNOPSIS
      SUBROUTINE DGBMV ( TRANS, M, N, KL, KU, ALPHA, A, LDA, X,
                         INCX, BETA, Y, INCY )

          DOUBLE       PRECISION ALPHA, BETA

          INTEGER      INCX, INCY, KL, KU, LDA, M, N

          CHARACTER*1  TRANS

          DOUBLE       PRECISION A( LDA, * ), X( * ), Y( * )

 PURPOSE
      DGBMV  performs one of the matrix-vector operations

      where alpha and beta are scalars, x and y are vectors and A
      is an m by n band matrix, with kl sub-diagonals and ku
      super-diagonals.

 PARAMETERS
      TRANS  - CHARACTER*1.
             On entry, TRANS specifies the operation to be per-
             formed as follows:

             TRANS = 'N' or 'n'   y := alpha*A*x + beta*y.

             TRANS = 'T' or 't'   y := alpha*A'*x + beta*y.

             TRANS = 'C' or 'c'   y := alpha*A'*x + beta*y.

             Unchanged on exit.

      M      - INTEGER.
             On entry, M specifies the number of rows of the
             matrix A.  M must be at least zero.  Unchanged on
             exit.

      N      - INTEGER.
             On entry, N specifies the number of columns of the
             matrix A.  N must be at least zero.  Unchanged on
             exit.
```

```
KL     - INTEGER.
         On entry, KL specifies the number of sub-diagonals of
         the matrix A. KL must satisfy  0 .le. KL.  Unchanged
         on exit.

KU     - INTEGER.
         On entry, KU specifies the number of super-diagonals
         of the matrix A. KU must satisfy  0 .le. KU.
         Unchanged on exit.

ALPHA  - DOUBLE PRECISION.
         On entry, ALPHA specifies the scalar alpha.
         Unchanged on exit.

A      - DOUBLE PRECISION array of DIMENSION ( LDA, n ).
         Before entry, the leading ( kl + ku + 1 ) by n part
         of the array A must contain the matrix of coeffi-
         cients, supplied column by column, with the leading
         diagonal of the matrix in row ( ku + 1 ) of the
         array, the first super-diagonal starting at position
         2 in row ku, the first sub-diagonal starting at posi-
         tion 1 in row ( ku + 2 ), and so on.  Elements in the
         array A that do not correspond to elements in the
         band matrix (such as the top left ku by ku triangle)
         are not referenced.  The following program segment
         will transfer a band matrix from conventional full
         matrix storage to band storage:

         DO 20, J = 1, N K = KU + 1 - J DO 10, I = MAX( 1, J -
         KU ), MIN( M, J + KL ) A( K + I, J ) = matrix( I, J )
         10    CONTINUE 20 CONTINUE

         Unchanged on exit.

LDA    - INTEGER.
         On entry, LDA specifies the first dimension of A as
         declared in the calling (sub) program. LDA must be at
         least ( kl + ku + 1 ).  Unchanged on exit.

X      - DOUBLE PRECISION array of DIMENSION at least
         ( 1 + ( n - 1 )*abs( INCX ) ) when TRANS = 'N' or 'n'
         and at least ( 1 + ( m - 1 )*abs( INCX ) ) otherwise.
         Before entry, the incremented array X must contain
         the vector x.  Unchanged on exit.

INCX   - INTEGER.
         On entry, INCX specifies the increment for the ele-
         ments of X. INCX must not be zero.  Unchanged on
         exit.
```

```
      BETA   - DOUBLE PRECISION.
             On entry, BETA specifies the scalar beta. When BETA
             is supplied as zero then Y need not be set on input.
             Unchanged on exit.

      Y      - DOUBLE PRECISION array of DIMENSION at least
             ( 1 + ( m - 1 )*abs( INCY ) ) when TRANS = 'N' or 'n'

             and at least ( 1 + ( n - 1 )*abs( INCY ) ) otherwise.
             Before entry, the incremented array Y must contain
             the vector y. On exit, Y is overwritten by the
             updated vector y.

      INCY   - INTEGER.
             On entry, INCY specifies the increment for the ele-
             ments of Y. INCY must not be zero.  Unchanged on
             exit.
```

———-

11.1.3 fortran code

— dgbmv.f —

```
*  ===================================================================
      SUBROUTINE DGBMV(TRANS,M,N,KL,KU,ALPHA,A,LDA,X,INCX,BETA,Y,INCY)
*
*  -- Reference BLAS level2 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION ALPHA,BETA
      INTEGER INCX,INCY,KL,KU,LDA,M,N
      CHARACTER TRANS
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A(LDA,*),X(*),Y(*)
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION ONE,ZERO
      PARAMETER (ONE=1.0D+0,ZERO=0.0D+0)
```

```
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP
      INTEGER I,INFO,IX,IY,J,JX,JY,K,KUP1,KX,KY,LENX,LENY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX,MIN
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +    .NOT.LSAME(TRANS,'C')) THEN
          INFO = 1
      ELSE IF (M.LT.0) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
          INFO = 3
      ELSE IF (KL.LT.0) THEN
          INFO = 4
      ELSE IF (KU.LT.0) THEN
          INFO = 5
      ELSE IF (LDA.LT. (KL+KU+1)) THEN
          INFO = 8
      ELSE IF (INCX.EQ.0) THEN
          INFO = 10
      ELSE IF (INCY.EQ.0) THEN
          INFO = 13
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('DGBMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR.
     +    ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     Set  LENX  and  LENY, the lengths of the vectors x and y, and set
*     up the start points in  X  and  Y.
*
```

```
      IF (LSAME(TRANS,'N')) THEN
          LENX = N
          LENY = M
      ELSE
          LENX = M
          LENY = N
      END IF
      IF (INCX.GT.0) THEN
          KX = 1
      ELSE
          KX = 1 - (LENX-1)*INCX
      END IF
      IF (INCY.GT.0) THEN
          KY = 1
      ELSE
          KY = 1 - (LENY-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the band part of A.
*
*     First form  y := beta*y.
*
      IF (BETA.NE.ONE) THEN
          IF (INCY.EQ.1) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 10 I = 1,LENY
                      Y(I) = ZERO
   10             CONTINUE
              ELSE
                  DO 20 I = 1,LENY
                      Y(I) = BETA*Y(I)
   20             CONTINUE
              END IF
          ELSE
              IY = KY
              IF (BETA.EQ.ZERO) THEN
                  DO 30 I = 1,LENY
                      Y(IY) = ZERO
                      IY = IY + INCY
   30             CONTINUE
              ELSE
                  DO 40 I = 1,LENY
                      Y(IY) = BETA*Y(IY)
                      IY = IY + INCY
   40             CONTINUE
              END IF
          END IF
      END IF
      IF (ALPHA.EQ.ZERO) RETURN
```

```
      KUP1 = KU + 1
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  y := alpha*A*x + y.
*
          JX = KX
          IF (INCY.EQ.1) THEN
              DO 60 J = 1,N
                  TEMP = ALPHA*X(JX)
                  K = KUP1 - J
                  DO 50 I = MAX(1,J-KU),MIN(M,J+KL)
                      Y(I) = Y(I) + TEMP*A(K+I,J)
   50             CONTINUE
                  JX = JX + INCX
   60         CONTINUE
          ELSE
              DO 80 J = 1,N
                  TEMP = ALPHA*X(JX)
                  IY = KY
                  K = KUP1 - J
                  DO 70 I = MAX(1,J-KU),MIN(M,J+KL)
                      Y(IY) = Y(IY) + TEMP*A(K+I,J)
                      IY = IY + INCY
   70             CONTINUE
                  JX = JX + INCX
                  IF (J.GT.KU) KY = KY + INCY
   80         CONTINUE
          END IF
      ELSE
*
*        Form  y := alpha*A**T*x + y.
*
          JY = KY
          IF (INCX.EQ.1) THEN
              DO 100 J = 1,N
                  TEMP = ZERO
                  K = KUP1 - J
                  DO 90 I = MAX(1,J-KU),MIN(M,J+KL)
                      TEMP = TEMP + A(K+I,J)*X(I)
   90             CONTINUE
                  Y(JY) = Y(JY) + ALPHA*TEMP
                  JY = JY + INCY
  100         CONTINUE
          ELSE
              DO 120 J = 1,N
                  TEMP = ZERO
                  IX = KX
                  K = KUP1 - J
                  DO 110 I = MAX(1,J-KU),MIN(M,J+KL)
                      TEMP = TEMP + A(K+I,J)*X(IX)
```

```
                    IX = IX + INCX
  110             CONTINUE
                  Y(JY) = Y(JY) + ALPHA*TEMP
                  JY = JY + INCY
                  IF (J.GT.KU) KX = KX + INCX
  120         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of DGBMV .
*
      END
```

———-

11.1.4 lisp code

— BLAS 2 dgbmv —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (double-float 1.0 1.0) one)
           (type (double-float 0.0 0.0) zero))
  (defun dgbmv (trans m n kl ku alpha a lda x incx beta y incy)
    (declare (type (simple-array double-float (*)) y x a)
             (type (double-float) beta alpha)
             (type fixnum incy incx lda ku kl n m)
             (type character trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (a double-float a-%data% a-%offset%)
         (x double-float x-%data% x-%offset%)
         (y double-float y-%data% y-%offset%))
      (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (k 0) (kup1 0)
             (kx 0) (ky 0) (lenx 0) (leny 0) (temp 0.0))
        (declare (type fixnum i info ix iy j jx jy k kup1 kx ky
                                          lenx leny)
                 (type (double-float) temp))
        (setf info 0)
        (cond
          ((and (not (char-equal trans #\N))
                (not (char-equal trans #\T))
                (not (char-equal trans #\C)))
           (setf info 1))
          ((< m 0)
           (setf info 2))
```

```
          ((< n 0)
           (setf info 3))
          ((< kl 0)
           (setf info 4))
          ((< ku 0)
           (setf info 5))
          ((< lda (f2cl-lib:int-add kl ku 1))
           (setf info 8))
          ((= incx 0)
           (setf info 10))
          ((= incy 0)
           (setf info 13)))
        (cond
          ((/= info 0)
            (error
              " ** On entry to ~a parameter number ~a had an illegal value~%"
              "DGBMV" info)
           (go end_label)))
        (if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
            (go end_label))
        (cond
          ((char-equal trans #\N)
           (setf lenx n)
           (setf leny m))
          (t
           (setf lenx m)
           (setf leny n)))
        (cond
          ((> incx 0)
           (setf kx 1))
          (t
           (setf kx
                   (the fixnum (- 1
                                  (f2cl-lib:int-mul
                                   (the fixnum (1- lenx))
                                   incx))))))
        (cond
          ((> incy 0)
           (setf ky 1))
          (t
           (setf ky
                   (the fixnum (- 1
                                  (f2cl-lib:int-mul
                                   (the fixnum (1- leny))
                                   incy)))
)))
        (cond
          ((/= beta one)
           (cond
             ((= incy 1)
```

```
      (cond
        ((= beta zero)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i leny) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     zero))))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i leny) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     (* beta
                        (f2cl-lib:fref y-%data%
                                       (i)
                                       ((1 *))
                                       y-%offset%))))))))
     (t
      (setf iy ky)
      (cond
        ((= beta zero)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i leny) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     zero)
             (setf iy (f2cl-lib:int-add iy incy)))))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i leny) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (* beta
                        (f2cl-lib:fref y-%data%
                                       (iy)
                                       ((1 *))
                                       y-%offset%)))
             (setf iy (f2cl-lib:int-add iy incy))))))))
(if (= alpha zero) (go end_label))
(setf kup1 (f2cl-lib:int-add ku 1))
(cond
  ((char-equal trans #\N)
   (setf jx kx)
   (cond
     ((= incy 1)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
            ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
```

```
            (setf temp
                    (* alpha
                       (f2cl-lib:fref x-%data%
                                      (jx)
                                      ((1 *))
                                      x-%offset%)))
            (setf k (f2cl-lib:int-sub kup1 j))
            (f2cl-lib:fdo (i
                           (max (the fixnum 1)
                                (the fixnum
                                     (f2cl-lib:int-add j
                                                       (f2cl-lib:int-sub
                                                        ku))))
                           (f2cl-lib:int-add i 1))
                          ((> i
                              (min (the fixnum m)
                                   (the fixnum
                                        (f2cl-lib:int-add j kl))))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data%
                                        (i)
                                        ((1 *))
                                        y-%offset%)
                         (* temp
                            (f2cl-lib:fref a-%data%
                                           ((f2cl-lib:int-add k i) j)
                                           ((1 lda) (1 *))
                                           a-%offset%))))))))
        (setf jx (f2cl-lib:int-add jx incx)))))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (cond
           ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
            (setf temp
                    (* alpha
                       (f2cl-lib:fref x-%data%
                                      (jx)
                                      ((1 *))
                                      x-%offset%)))
            (setf iy ky)
            (setf k (f2cl-lib:int-sub kup1 j))
            (f2cl-lib:fdo (i
                           (max (the fixnum 1)
                                (the fixnum
                                     (f2cl-lib:int-add j
```

```
                                                        (f2cl-lib:int-sub
                                                         ku))))
                              (f2cl-lib:int-add i 1))
                             ((> i
                                 (min (the fixnum m)
                                      (the fixnum
                                           (f2cl-lib:int-add j kl))))
                              nil)
                  (tagbody
                    (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                            (+
                             (f2cl-lib:fref y-%data%
                                            (iy)
                                            ((1 *))
                                            y-%offset%)
                             (* temp
                                (f2cl-lib:fref a-%data%
                                               ((f2cl-lib:int-add k i) j)
                                               ((1 lda) (1 *))
                                               a-%offset%))))
                    (setf iy (f2cl-lib:int-add iy incy))))))
            (setf jx (f2cl-lib:int-add jx incx))
            (if (> j ku) (setf ky (f2cl-lib:int-add ky incy))))))))
    (t
     (setf jy ky)
     (cond
       ((= incx 1)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf temp zero)
            (setf k (f2cl-lib:int-sub kup1 j))
            (f2cl-lib:fdo (i
                           (max (the fixnum 1)
                                (the fixnum
                                     (f2cl-lib:int-add j
                                                       (f2cl-lib:int-sub
                                                        ku))))
                           (f2cl-lib:int-add i 1))
                          ((> i
                              (min (the fixnum m)
                                   (the fixnum
                                        (f2cl-lib:int-add j kl))))
                           nil)
              (tagbody
                (setf temp
                        (+ temp
                           (*
                            (f2cl-lib:fref a-%data%
                                           ((f2cl-lib:int-add k i) j)
```

```
                                                ((1 lda) (1 *))
                                                a-%offset%)
                                   (f2cl-lib:fref x-%data%
                                                (i)
                                                ((1 *))
                                                x-%offset%))))))
                 (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                         (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                            (* alpha temp)))
                 (setf jy (f2cl-lib:int-add jy incy)))))
             (t
              (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                            ((> j n) nil)
                (tagbody
                  (setf temp zero)
                  (setf ix kx)
                  (setf k (f2cl-lib:int-sub kup1 j))
                  (f2cl-lib:fdo (i
                                 (max (the fixnum 1)
                                      (the fixnum
                                           (f2cl-lib:int-add j
                                                             (f2cl-lib:int-sub
                                                              ku))))
                                 (f2cl-lib:int-add i 1))
                                ((> i
                                    (min (the fixnum m)
                                         (the fixnum
                                              (f2cl-lib:int-add j kl))))
                                 nil)
                    (tagbody
                      (setf temp
                              (+ temp
                                 (*
                                  (f2cl-lib:fref a-%data%
                                                 ((f2cl-lib:int-add k i) j)
                                                 ((1 lda) (1 *))
                                                 a-%offset%)
                                  (f2cl-lib:fref x-%data%
                                                 (ix)
                                                 ((1 *))
                                                 x-%offset%))))
                      (setf ix (f2cl-lib:int-add ix incx))))
                  (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                          (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                             (* alpha temp)))
                  (setf jy (f2cl-lib:int-add jy incy))
                  (if (> j ku) (setf kx (f2cl-lib:int-add kx incx)))))))))
end_label
        (return
         (values nil nil nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

11.2 dgemv - $\alpha Ax + \beta y$ or $\alpha A^T x + \beta y$

11.2.1 Axiom unit tests

— **dgemv.input** —

```
)set break resume
)sys rm -f dgemv.output
)spool dgemv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

11.2.2 Axiom help page

— **dgemv.help** —

```
======================================================================
dgemv examples
======================================================================

======================================================================
Man Page Details
======================================================================

 NAME
      DGEMV - perform one of the matrix-vector operations   y :=
      alpha*A*x + beta*y, or y := alpha*A'*x + beta*y,

 SYNOPSIS
      SUBROUTINE DGEMV ( TRANS, M, N, ALPHA, A, LDA, X, INCX,
                         BETA, Y, INCY )

          DOUBLE       PRECISION ALPHA, BETA
```

```
     INTEGER      INCX, INCY, LDA, M, N

     CHARACTER*1  TRANS

     DOUBLE       PRECISION A( LDA, * ), X( * ), Y( * )

PURPOSE
     DGEMV  performs one of the matrix-vector operations

     where alpha and beta are scalars, x and y are vectors and A
     is an m by n matrix.

PARAMETERS
     TRANS  - CHARACTER*1.
            On entry, TRANS specifies the operation to be per-
            formed as follows:

            TRANS = 'N' or 'n'   y := alpha*A*x + beta*y.

            TRANS = 'T' or 't'   y := alpha*A'*x + beta*y.

            TRANS = 'C' or 'c'   y := alpha*A'*x + beta*y.

            Unchanged on exit.

     M      - INTEGER.
            On entry, M specifies the number of rows of the
            matrix A.  M must be at least zero.  Unchanged on
            exit.

     N      - INTEGER.
            On entry, N specifies the number of columns of the
            matrix A.  N must be at least zero.  Unchanged on
            exit.

     ALPHA  - DOUBLE PRECISION.
            On entry, ALPHA specifies the scalar alpha.
            Unchanged on exit.

     A      - DOUBLE PRECISION array of DIMENSION ( LDA, n ).
            Before entry, the leading m by n part of the array A

            must contain the matrix of coefficients.  Unchanged
            on exit.

     LDA    - INTEGER.
            On entry, LDA specifies the first dimension of A as
            declared in the calling (sub) program. LDA must be at
            least max( 1, m ).  Unchanged on exit.
```

```
X      - DOUBLE PRECISION array of DIMENSION at least
         ( 1 + ( n - 1 )*abs( INCX ) ) when TRANS = 'N' or 'n'
         and at least ( 1 + ( m - 1 )*abs( INCX ) ) otherwise.
         Before entry, the incremented array X must contain
         the vector x.  Unchanged on exit.

INCX   - INTEGER.
         On entry, INCX specifies the increment for the ele-
         ments of X. INCX must not be zero.  Unchanged on
         exit.

BETA   - DOUBLE PRECISION.
         On entry, BETA specifies the scalar beta. When BETA
         is supplied as zero then Y need not be set on input.
         Unchanged on exit.

Y      - DOUBLE PRECISION array of DIMENSION at least
         ( 1 + ( m - 1 )*abs( INCY ) ) when TRANS = 'N' or 'n'
         and at least ( 1 + ( n - 1 )*abs( INCY ) ) otherwise.
         Before entry with BETA non-zero, the incremented
         array Y must contain the vector y. On exit, Y is
         overwritten by the updated vector y.

INCY   - INTEGER.
         On entry, INCY specifies the increment for the ele-
         ments of Y. INCY must not be zero.  Unchanged on
         exit.
```

———-

11.2.3 fortran code

— dgemv.f —

```
*  =====================================================================
      SUBROUTINE DGEMV(TRANS,M,N,ALPHA,A,LDA,X,INCX,BETA,Y,INCY)
*
*  -- Reference BLAS level2 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION ALPHA,BETA
      INTEGER INCX,INCY,LDA,M,N
      CHARACTER TRANS
*     ..
```

```
*     .. Array Arguments ..
      DOUBLE PRECISION A(LDA,*),X(*),Y(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION ONE,ZERO
      PARAMETER (ONE=1.0D+0,ZERO=0.0D+0)
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP
      INTEGER I,INFO,IX,IY,J,JX,JY,KX,KY,LENX,LENY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +    .NOT.LSAME(TRANS,'C')) THEN
          INFO = 1
      ELSE IF (M.LT.0) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
          INFO = 3
      ELSE IF (LDA.LT.MAX(1,M)) THEN
          INFO = 6
      ELSE IF (INCX.EQ.0) THEN
          INFO = 8
      ELSE IF (INCY.EQ.0) THEN
          INFO = 11
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('DGEMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR.
```

```
     +    ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     Set  LENX  and  LENY, the lengths of the vectors x and y, and set
*     up the start points in  X  and  Y.
*
      IF (LSAME(TRANS,'N')) THEN
          LENX = N
          LENY = M
      ELSE
          LENX = M
          LENY = N
      END IF
      IF (INCX.GT.0) THEN
          KX = 1
      ELSE
          KX = 1 - (LENX-1)*INCX
      END IF
      IF (INCY.GT.0) THEN
          KY = 1
      ELSE
          KY = 1 - (LENY-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
*     First form  y := beta*y.
*
      IF (BETA.NE.ONE) THEN
          IF (INCY.EQ.1) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 10 I = 1,LENY
                      Y(I) = ZERO
   10             CONTINUE
              ELSE
                  DO 20 I = 1,LENY
                      Y(I) = BETA*Y(I)
   20             CONTINUE
              END IF
          ELSE
              IY = KY
              IF (BETA.EQ.ZERO) THEN
                  DO 30 I = 1,LENY
                      Y(IY) = ZERO
                      IY = IY + INCY
   30             CONTINUE
              ELSE
                  DO 40 I = 1,LENY
                      Y(IY) = BETA*Y(IY)
                      IY = IY + INCY
```

```
   40             CONTINUE
               END IF
           END IF
       END IF
       IF (ALPHA.EQ.ZERO) RETURN
       IF (LSAME(TRANS,'N')) THEN
*
*        Form  y := alpha*A*x + y.
*
           JX = KX
           IF (INCY.EQ.1) THEN
               DO 60 J = 1,N
                   TEMP = ALPHA*X(JX)
                   DO 50 I = 1,M
                       Y(I) = Y(I) + TEMP*A(I,J)
   50              CONTINUE
                   JX = JX + INCX
   60          CONTINUE
           ELSE
               DO 80 J = 1,N
                   TEMP = ALPHA*X(JX)
                   IY = KY
                   DO 70 I = 1,M
                       Y(IY) = Y(IY) + TEMP*A(I,J)
                       IY = IY + INCY
   70              CONTINUE
                   JX = JX + INCX
   80          CONTINUE
           END IF
       ELSE
*
*        Form  y := alpha*A**T*x + y.
*
           JY = KY
           IF (INCX.EQ.1) THEN
               DO 100 J = 1,N
                   TEMP = ZERO
                   DO 90 I = 1,M
                       TEMP = TEMP + A(I,J)*X(I)
   90              CONTINUE
                   Y(JY) = Y(JY) + ALPHA*TEMP
                   JY = JY + INCY
  100          CONTINUE
           ELSE
               DO 120 J = 1,N
                   TEMP = ZERO
                   IX = KX
                   DO 110 I = 1,M
                       TEMP = TEMP + A(I,J)*X(IX)
                       IX = IX + INCX
```

```
  110             CONTINUE
                  Y(JY) = Y(JY) + ALPHA*TEMP
                  JY = JY + INCY
  120         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of DGEMV .
*
      END
```

———-

11.2.4 lisp code

— BLAS 2 dgemv —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (double-float 1.0 1.0) one)
           (type (double-float 0.0 0.0) zero))
  (defun dgemv (trans m n alpha a lda x incx beta y incy)
    (declare (type (simple-array double-float (*)) y x a)
             (type (double-float) beta alpha)
             (type fixnum incy incx lda n m)
             (type character trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (a double-float a-%data% a-%offset%)
         (x double-float x-%data% x-%offset%)
         (y double-float y-%data% y-%offset%))
      (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (kx 0) (ky 0)
             (lenx 0) (leny 0) (temp 0.0))
        (declare (type fixnum i info ix iy j jx jy kx ky lenx
                                          leny)
                 (type (double-float) temp))
        (setf info 0)
        (cond
          ((and (not (char-equal trans #\N))
                (not (char-equal trans #\T))
                (not (char-equal trans #\C)))
           (setf info 1))
          ((< m 0)
           (setf info 2))
          ((< n 0)
           (setf info 3))
```

```
    ((< lda (max (the fixnum 1) (the fixnum m)))
     (setf info 6))
    ((= incx 0)
     (setf info 8))
    ((= incy 0)
     (setf info 11)))
  (cond
    ((/= info 0)
     (error
       " ** On entry to ~a parameter number ~a had an illegal value~%"
       "DGEMV" info)
     (go end_label)))
  (if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
      (go end_label))
  (cond
    ((char-equal trans #\N)
     (setf lenx n)
     (setf leny m))
    (t
     (setf lenx m)
     (setf leny n)))
  (cond
    ((> incx 0)
     (setf kx 1))
    (t
     (setf kx
             (f2cl-lib:int-sub 1
                               (f2cl-lib:int-mul
                                (f2cl-lib:int-sub lenx 1)
                                incx)))))
  (cond
    ((> incy 0)
     (setf ky 1))
    (t
     (setf ky
             (f2cl-lib:int-sub 1
                               (f2cl-lib:int-mul
                                (f2cl-lib:int-sub leny 1)
                                incy)))))
  (cond
    ((/= beta one)
     (cond
       ((= incy 1)
        (cond
          ((= beta zero)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i leny) nil)
             (tagbody
               (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                       zero))))
```

```
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i leny) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     (* beta
                        (f2cl-lib:fref y-%data%
                                       (i)
                                       ((1 *))
                                       y-%offset%))))))))
      (t
       (setf iy ky)
       (cond
         ((= beta zero)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i leny) nil)
            (tagbody
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      zero)
              (setf iy (f2cl-lib:int-add iy incy)))))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i leny) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (* beta
                        (f2cl-lib:fref y-%data%
                                       (iy)
                                       ((1 *))
                                       y-%offset%)))
             (setf iy (f2cl-lib:int-add iy incy))))))))))
(if (= alpha zero) (go end_label))
(cond
  ((char-equal trans #\N)
   (setf jx kx)
   (cond
     ((= incy 1)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
            ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
             (setf temp
                     (* alpha
                        (f2cl-lib:fref x-%data%
                                       (jx)
                                       ((1 *))
                                       x-%offset%)))
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i m) nil)
```

```
                (tagbody
                  (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                          (+
                           (f2cl-lib:fref y-%data%
                                          (i)
                                          ((1 *))
                                          y-%offset%)
                           (* temp
                              (f2cl-lib:fref a-%data%
                                             (i j)
                                             ((1 lda) (1 *))
                                             a-%offset%))))))))
            (setf jx (f2cl-lib:int-add jx incx)))))
      (t
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (cond
             ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
              (setf temp
                      (* alpha
                         (f2cl-lib:fref x-%data%
                                        (jx)
                                        ((1 *))
                                        x-%offset%)))
              (setf iy ky)
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i m) nil)
                (tagbody
                  (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                          (+
                           (f2cl-lib:fref y-%data%
                                          (iy)
                                          ((1 *))
                                          y-%offset%)
                           (* temp
                              (f2cl-lib:fref a-%data%
                                             (i j)
                                             ((1 lda) (1 *))
                                             a-%offset%))))
                  (setf iy (f2cl-lib:int-add iy incy))))))
            (setf jx (f2cl-lib:int-add jx incx)))))))
    (t
     (setf jy ky)
     (cond
       ((= incx 1)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf temp zero)
```

```
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i m) nil)
                  (tagbody
                    (setf temp
                            (+ temp
                               (*
                                (f2cl-lib:fref a-%data%
                                               (i j)
                                               ((1 lda) (1 *))
                                               a-%offset%)
                                (f2cl-lib:fref x-%data%
                                               (i)
                                               ((1 *))
                                               x-%offset%))))))
                (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                        (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                           (* alpha temp)))
                (setf jy (f2cl-lib:int-add jy incy)))))
            (t
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (setf temp zero)
                 (setf ix kx)
                 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                               ((> i m) nil)
                   (tagbody
                     (setf temp
                             (+ temp
                                (*
                                 (f2cl-lib:fref a-%data%
                                                (i j)
                                                ((1 lda) (1 *))
                                                a-%offset%)
                                 (f2cl-lib:fref x-%data%
                                                (ix)
                                                ((1 *))
                                                x-%offset%))))
                     (setf ix (f2cl-lib:int-add ix incx))))
                 (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                         (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                            (* alpha temp)))
                 (setf jy (f2cl-lib:int-add jy incy))))))))
end_label
       (return (values nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

11.3 dger - rank 1 $\alpha xy^T + a$

11.3.1 Axiom unit tests

— **dger.input** —

```
)set break resume
)sys rm -f dger.output
)spool dger.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

11.3.2 Axiom help page

— **dger.help** —

```
======================================================================
dger examples
======================================================================

======================================================================
Man Page Details
======================================================================

 NAME
      DGER - perform the rank 1 operation   A := alpha*x*y' + A,

 SYNOPSIS
      SUBROUTINE DGER ( M, N, ALPHA, X, INCX, Y, INCY, A, LDA )

          DOUBLE      PRECISION ALPHA

          INTEGER     INCX, INCY, LDA, M, N

          DOUBLE      PRECISION A( LDA, * ), X( * ), Y( * )

 PURPOSE
      DGER   performs the rank 1 operation

      where alpha is a scalar, x is an m element vector, y is an n
```

```
     element vector and A is an m by n matrix.

PARAMETERS
     M      - INTEGER.
            On entry, M specifies the number of rows of the
            matrix A.  M must be at least zero.  Unchanged on
            exit.

     N      - INTEGER.
            On entry, N specifies the number of columns of the
            matrix A.  N must be at least zero.  Unchanged on
            exit.

     ALPHA  - DOUBLE PRECISION.
            On entry, ALPHA specifies the scalar alpha.
            Unchanged on exit.

     X      - DOUBLE PRECISION array of dimension at least
            ( 1 + ( m - 1 )*abs( INCX ) ).  Before entry, the
            incremented array X must contain the m element vector
            x.  Unchanged on exit.

     INCX   - INTEGER.
            On entry, INCX specifies the increment for the ele-
            ments of X. INCX must not be zero.  Unchanged on
            exit.

     Y      - DOUBLE PRECISION array of dimension at least
            ( 1 + ( n - 1 )*abs( INCY ) ).  Before entry, the
            incremented array Y must contain the n element vector
            y.  Unchanged on exit.

     INCY   - INTEGER.
            On entry, INCY specifies the increment for the ele-
            ments of Y. INCY must not be zero.  Unchanged on

            exit.

     A      - DOUBLE PRECISION array of DIMENSION ( LDA, n ).
            Before entry, the leading m by n part of the array A
            must contain the matrix of coefficients. On exit, A
            is overwritten by the updated matrix.

     LDA    - INTEGER.
            On entry, LDA specifies the first dimension of A as
            declared in the calling (sub) program. LDA must be at
            least max( 1, m ).  Unchanged on exit.

            -------
```

11.3.3 fortran code

— dger.f —

```
*  =====================================================================
      SUBROUTINE DGER(M,N,ALPHA,X,INCX,Y,INCY,A,LDA)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION ALPHA
      INTEGER INCX,INCY,LDA,M,N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A(LDA,*),X(*),Y(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION ZERO
      PARAMETER (ZERO=0.0D+0)
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP
      INTEGER I,INFO,IX,J,JY,KX
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (M.LT.0) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      ELSE IF (INCY.EQ.0) THEN
          INFO = 7
      ELSE IF (LDA.LT.MAX(1,M)) THEN
```

```
         INFO = 9
      END IF
      IF (INFO.NE.0) THEN
         CALL XERBLA('DGER  ',INFO)
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR. (ALPHA.EQ.ZERO)) RETURN
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
      IF (INCY.GT.0) THEN
         JY = 1
      ELSE
         JY = 1 - (N-1)*INCY
      END IF
      IF (INCX.EQ.1) THEN
         DO 20 J = 1,N
            IF (Y(JY).NE.ZERO) THEN
               TEMP = ALPHA*Y(JY)
               DO 10 I = 1,M
                  A(I,J) = A(I,J) + X(I)*TEMP
   10          CONTINUE
            END IF
            JY = JY + INCY
   20    CONTINUE
      ELSE
         IF (INCX.GT.0) THEN
            KX = 1
         ELSE
            KX = 1 - (M-1)*INCX
         END IF
         DO 40 J = 1,N
            IF (Y(JY).NE.ZERO) THEN
               TEMP = ALPHA*Y(JY)
               IX = KX
               DO 30 I = 1,M
                  A(I,J) = A(I,J) + X(IX)*TEMP
                  IX = IX + INCX
   30          CONTINUE
            END IF
            JY = JY + INCY
   40    CONTINUE
      END IF
*
      RETURN
*
```

```
*     End of DGER  .
*
      END
```

———-

11.3.4 lisp code

— BLAS 2 dger —

```
(let* ((zero 0.0))
  (declare (type (double-float 0.0 0.0) zero))
  (defun dger (m n alpha x incx y incy a lda)
    (declare (type (simple-array double-float (*)) a y x)
             (type (double-float) alpha)
             (type fixnum lda incy incx n m))
    (f2cl-lib:with-multi-array-data
        ((x double-float x-%data% x-%offset%)
         (y double-float y-%data% y-%offset%)
         (a double-float a-%data% a-%offset%))
      (prog ((i 0) (info 0) (ix 0) (j 0) (jy 0) (kx 0) (temp 0.0))
        (declare (type fixnum i info ix j jy kx)
                 (type (double-float) temp))
        (setf info 0)
        (cond
          ((< m 0)
           (setf info 1))
          ((< n 0)
           (setf info 2))
          ((= incx 0)
           (setf info 5))
          ((= incy 0)
           (setf info 7))
          ((< lda (max (the fixnum 1) (the fixnum m)))
           (setf info 9)))
        (cond
          ((/= info 0)
           (error
            " ** On entry to ~a parameter number ~a had an illegal value~%"
            "DYER" info)
           (go end_label)))
        (if (or (= m 0) (= n 0) (= alpha zero)) (go end_label))
        (cond
          ((> incy 0)
           (setf jy 1))
          (t
           (setf jy
```

```
            (f2cl-lib:int-sub 1
                              (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                incy)))))
(cond
  ((= incx 1)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
         ((/= (f2cl-lib:fref y (jy) ((1 *))) zero)
          (setf temp
                  (* alpha
                     (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref a-%data%
                                   (i j)
                                   ((1 lda) (1 *))
                                   a-%offset%)
                    (+
                     (f2cl-lib:fref a-%data%
                                    (i j)
                                    ((1 lda) (1 *))
                                    a-%offset%)
                     (*
                      (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                      temp)))))))
       (setf jy (f2cl-lib:int-add jy incy)))))
  (t
   (cond
     ((> incx 0)
      (setf kx 1))
     (t
      (setf kx
              (f2cl-lib:int-sub 1
                                (f2cl-lib:int-mul
                                 (f2cl-lib:int-sub m 1)
                                 incx)))))
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
         ((/= (f2cl-lib:fref y (jy) ((1 *))) zero)
          (setf temp
                  (* alpha
                     (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)))
          (setf ix kx)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
```

```
                    (tagbody
                      (setf (f2cl-lib:fref a-%data%
                                           (i j)
                                           ((1 lda) (1 *))
                                           a-%offset%)
                            (+
                             (f2cl-lib:fref a-%data%
                                            (i j)
                                            ((1 lda) (1 *))
                                            a-%offset%)
                             (*
                              (f2cl-lib:fref x-%data%
                                             (ix)
                                             ((1 *))
                                             x-%offset%)
                              temp)))
                      (setf ix (f2cl-lib:int-add ix incx))))))
              (setf jy (f2cl-lib:int-add jy incy))))))
 end_label
       (return (values nil nil nil nil nil nil nil nil nil))))))
```

———-

11.4 dsbmv - banded $\alpha Ax + \beta y$

11.4.1 Axiom unit tests

— **dsbmv.input** —

```
)set break resume
)sys rm -f dsbmv.output
)spool dsbmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

11.4.2 Axiom help page

— **dsbmv.help** —

```
=====================================================================
dsbmv examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

 NAME
      DSBMV - perform the matrix-vector operation   y := alpha*A*x
      + beta*y,

 SYNOPSIS
      SUBROUTINE DSBMV ( UPLO, N, K, ALPHA, A, LDA, X, INCX, BETA,
                         Y, INCY )

          DOUBLE       PRECISION ALPHA, BETA

          INTEGER      INCX, INCY, K, LDA, N

          CHARACTER*1  UPLO

          DOUBLE       PRECISION A( LDA, * ), X( * ), Y( * )

 PURPOSE
      DSBMV  performs the matrix-vector  operation

      where alpha and beta are scalars, x and y are n element vec-
      tors and A is an n by n symmetric band matrix, with k
      super-diagonals.

 PARAMETERS
      UPLO   - CHARACTER*1.
             On entry, UPLO specifies whether the upper or lower
             triangular part of the band matrix A is being sup-
             plied as follows:

             UPLO = 'U' or 'u'   The upper triangular part of A is
             being supplied.

             UPLO = 'L' or 'l'   The lower triangular part of A is
             being supplied.

             Unchanged on exit.

      N      - INTEGER.
             On entry, N specifies the order of the matrix A.  N
             must be at least zero.  Unchanged on exit.

      K      - INTEGER.
```

```
        On entry, K specifies the number of super-diagonals
        of the matrix A. K must satisfy  0 .le. K.  Unchanged
        on exit.

ALPHA   - DOUBLE PRECISION.
        On entry, ALPHA specifies the scalar alpha.
        Unchanged on exit.

A       - DOUBLE PRECISION array of DIMENSION ( LDA, n ).

        Before entry with UPLO = 'U' or 'u', the leading ( k
        + 1 ) by n part of the array A must contain the upper
        triangular band part of the symmetric matrix, sup-
        plied column by column, with the leading diagonal of
        the matrix in row ( k + 1 ) of the array, the first
        super-diagonal starting at position 2 in row k, and
        so on. The top left k by k triangle of the array A is
        not referenced.  The following program segment will
        transfer the upper triangular part of a symmetric
        band matrix from conventional full matrix storage to
        band storage:

        DO 20, J = 1, N M = K + 1 - J DO 10, I = MAX( 1, J -
        K ), J A( M + I, J ) = matrix( I, J ) 10    CONTINUE
        20 CONTINUE

        Before entry with UPLO = 'L' or 'l', the leading ( k
        + 1 ) by n part of the array A must contain the lower
        triangular band part of the symmetric matrix, sup-
        plied column by column, with the leading diagonal of
        the matrix in row 1 of the array, the first sub-
        diagonal starting at position 1 in row 2, and so on.
        The bottom right k by k triangle of the array A is
        not referenced.  The following program segment will
        transfer the lower triangular part of a symmetric
        band matrix from conventional full matrix storage to
        band storage:

        DO 20, J = 1, N M = 1 - J DO 10, I = J, MIN( N, J + K
        ) A( M + I, J ) = matrix( I, J ) 10    CONTINUE 20
        CONTINUE

        Unchanged on exit.

LDA     - INTEGER.
        On entry, LDA specifies the first dimension of A as
        declared in the calling (sub) program. LDA must be at
        least ( k + 1 ).  Unchanged on exit.

X       - DOUBLE PRECISION array of DIMENSION at least
```

```
             ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
             incremented array X must contain the vector x.
             Unchanged on exit.

      INCX   - INTEGER.
             On entry, INCX specifies the increment for the ele-
             ments of X. INCX must not be zero.  Unchanged on
             exit.

      BETA   - DOUBLE PRECISION.
             On entry, BETA specifies the scalar beta.  Unchanged
             on exit.

      Y      - DOUBLE PRECISION array of DIMENSION at least
             ( 1 + ( n - 1 )*abs( INCY ) ).  Before entry, the
             incremented array Y must contain the vector y. On
             exit, Y is overwritten by the updated vector y.

      INCY   - INTEGER.
             On entry, INCY specifies the increment for the ele-
             ments of Y. INCY must not be zero.  Unchanged on
             exit.
```

———-

11.4.3 fortran code

— dsbmv.f —

```
*  =====================================================================
      SUBROUTINE DSBMV(UPLO,N,K,ALPHA,A,LDA,X,INCX,BETA,Y,INCY)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION ALPHA,BETA
      INTEGER INCX,INCY,K,LDA,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A(LDA,*),X(*),Y(*)
*     ..
*
*  =====================================================================
```

```
*
*     .. Parameters ..
      DOUBLE PRECISION ONE,ZERO
      PARAMETER (ONE=1.0D+0,ZERO=0.0D+0)
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,KPLUS1,KX,KY,L
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX,MIN
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (K.LT.0) THEN
          INFO = 3
      ELSE IF (LDA.LT. (K+1)) THEN
          INFO = 6
      ELSE IF (INCX.EQ.0) THEN
          INFO = 8
      ELSE IF (INCY.EQ.0) THEN
          INFO = 11
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('DSBMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     Set up the start points in  X  and  Y.
*
      IF (INCX.GT.0) THEN
          KX = 1
      ELSE
```

```
         KX = 1 - (N-1)*INCX
      END IF
      IF (INCY.GT.0) THEN
         KY = 1
      ELSE
         KY = 1 - (N-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of the array A
*     are accessed sequentially with one pass through A.
*
*     First form  y := beta*y.
*
      IF (BETA.NE.ONE) THEN
         IF (INCY.EQ.1) THEN
            IF (BETA.EQ.ZERO) THEN
               DO 10 I = 1,N
                  Y(I) = ZERO
   10          CONTINUE
            ELSE
               DO 20 I = 1,N
                  Y(I) = BETA*Y(I)
   20          CONTINUE
            END IF
         ELSE
            IY = KY
            IF (BETA.EQ.ZERO) THEN
               DO 30 I = 1,N
                  Y(IY) = ZERO
                  IY = IY + INCY
   30          CONTINUE
            ELSE
               DO 40 I = 1,N
                  Y(IY) = BETA*Y(IY)
                  IY = IY + INCY
   40          CONTINUE
            END IF
         END IF
      END IF
      IF (ALPHA.EQ.ZERO) RETURN
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  y  when upper triangle of A is stored.
*
         KPLUS1 = K + 1
         IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
            DO 60 J = 1,N
               TEMP1 = ALPHA*X(J)
               TEMP2 = ZERO
               L = KPLUS1 - J
```

```
                DO 50 I = MAX(1,J-K),J - 1
                    Y(I) = Y(I) + TEMP1*A(L+I,J)
                    TEMP2 = TEMP2 + A(L+I,J)*X(I)
   50           CONTINUE
                Y(J) = Y(J) + TEMP1*A(KPLUS1,J) + ALPHA*TEMP2
   60       CONTINUE
        ELSE
            JX = KX
            JY = KY
            DO 80 J = 1,N
                TEMP1 = ALPHA*X(JX)
                TEMP2 = ZERO
                IX = KX
                IY = KY
                L = KPLUS1 - J
                DO 70 I = MAX(1,J-K),J - 1
                    Y(IY) = Y(IY) + TEMP1*A(L+I,J)
                    TEMP2 = TEMP2 + A(L+I,J)*X(IX)
                    IX = IX + INCX
                    IY = IY + INCY
   70           CONTINUE
                Y(JY) = Y(JY) + TEMP1*A(KPLUS1,J) + ALPHA*TEMP2
                JX = JX + INCX
                JY = JY + INCY
                IF (J.GT.K) THEN
                    KX = KX + INCX
                    KY = KY + INCY
                END IF
   80       CONTINUE
        END IF
    ELSE
*
*       Form  y  when lower triangle of A is stored.
*
        IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
            DO 100 J = 1,N
                TEMP1 = ALPHA*X(J)
                TEMP2 = ZERO
                Y(J) = Y(J) + TEMP1*A(1,J)
                L = 1 - J
                DO 90 I = J + 1,MIN(N,J+K)
                    Y(I) = Y(I) + TEMP1*A(L+I,J)
                    TEMP2 = TEMP2 + A(L+I,J)*X(I)
   90           CONTINUE
                Y(J) = Y(J) + ALPHA*TEMP2
  100       CONTINUE
        ELSE
            JX = KX
            JY = KY
            DO 120 J = 1,N
```

```
                TEMP1 = ALPHA*X(JX)
                TEMP2 = ZERO
                Y(JY) = Y(JY) + TEMP1*A(1,J)
                L = 1 - J
                IX = JX
                IY = JY
                DO 110 I = J + 1,MIN(N,J+K)
                    IX = IX + INCX
                    IY = IY + INCY
                    Y(IY) = Y(IY) + TEMP1*A(L+I,J)
                    TEMP2 = TEMP2 + A(L+I,J)*X(IX)
  110           CONTINUE
                Y(JY) = Y(JY) + ALPHA*TEMP2
                JX = JX + INCX
                JY = JY + INCY
  120         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of DSBMV .
*
      END
```

———-

11.4.4 lisp code

— BLAS 2 dsbmv —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (double-float 1.0 1.0) one)
           (type (double-float 0.0 0.0) zero))
  (defun dsbmv (uplo n k alpha a lda x incx beta y incy)
    (declare (type (simple-array double-float (*)) y x a)
             (type (double-float) beta alpha)
             (type fixnum incy incx lda k n)
             (type character uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (x double-float x-%data% x-%offset%)
         (y double-float y-%data% y-%offset%))
      (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (kplus1 0) (kx 0)
             (ky 0) (l 0) (temp1 0.0) (temp2 0.0))
        (declare (type fixnum i info ix iy j jx jy kplus1 kx ky l)
```

```
          (type (double-float) temp1 temp2))
(setf info 0)
(cond
  ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
   (setf info 1))
  ((< n 0)
   (setf info 2))
  ((< k 0)
   (setf info 3))
  ((< lda (f2cl-lib:int-add k 1))
   (setf info 6))
  ((= incx 0)
   (setf info 8))
  ((= incy 0)
   (setf info 11)))
(cond
  ((/= info 0)
   (error
    " ** On entry to ~a parameter number ~a had an illegal value~%"
    "DSBMV" info)
   (go end_label)))
(if (or (= n 0) (and (= alpha zero) (= beta one))) (go end_label))
(cond
  ((> incx 0)
   (setf kx 1))
  (t
   (setf kx
           (f2cl-lib:int-sub 1
                             (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                               incx)))))
(cond
  ((> incy 0)
   (setf ky 1))
  (t
   (setf ky
           (f2cl-lib:int-sub 1
                             (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                               incy)))))
(cond
  ((/= beta one)
   (cond
     ((= incy 1)
      (cond
        ((= beta zero)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     zero))))
        (t
```

```
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                      (* beta
                         (f2cl-lib:fref y-%data%
                                        (i)
                                        ((1 *))
                                        y-%offset%))))))))
      (t
       (setf iy ky)
       (cond
         ((= beta zero)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      zero)
              (setf iy (f2cl-lib:int-add iy incy)))))
         (t
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (* beta
                         (f2cl-lib:fref y-%data%
                                        (iy)
                                        ((1 *))
                                        y-%offset%)))
              (setf iy (f2cl-lib:int-add iy incy))))))))))
(if (= alpha zero) (go end_label))
(cond
  ((char-equal uplo #\U)
   (setf kplus1 (f2cl-lib:int-add k 1))
   (cond
     ((and (= incx 1) (= incy 1))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf temp1
                  (* alpha
                     (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
          (setf temp2 zero)
          (setf l (f2cl-lib:int-sub kplus1 j))
          (f2cl-lib:fdo (i
                         (max (the fixnum 1)
                              (the fixnum
                                   (f2cl-lib:int-add j
                                                     (f2cl-lib:int-sub
                                                      k))))
```

```
                        (f2cl-lib:int-add i 1))
                       ((> i
                           (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                        nil)
          (tagbody
            (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                    (+
                     (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     (* temp1
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add l i) j)
                                       ((1 lda) (1 *))
                                       a-%offset%))))
            (setf temp2
                    (+ temp2
                       (*
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add l i) j)
                                       ((1 lda) (1 *))
                                       a-%offset%)
                        (f2cl-lib:fref x-%data%
                                       (i)
                                       ((1 *))
                                       x-%offset%))))))
      (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
              (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                 (* temp1
                    (f2cl-lib:fref a-%data%
                                   (kplus1 j)
                                   ((1 lda) (1 *))
                                   a-%offset%))
                 (* alpha temp2))))))
  (t
   (setf jx kx)
   (setf jy ky)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp1
               (* alpha
                  (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
       (setf temp2 zero)
       (setf ix kx)
       (setf iy ky)
       (setf l (f2cl-lib:int-sub kplus1 j))
       (f2cl-lib:fdo (i
                      (max (the fixnum 1)
                           (the fixnum
                                (f2cl-lib:int-add j
                                                  (f2cl-lib:int-sub
```

```
                                                  k))))
                        (f2cl-lib:int-add i 1))
                       ((> i
                           (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                        nil)
          (tagbody
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+
                     (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (* temp1
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add l i) j)
                                       ((1 lda) (1 *))
                                       a-%offset%))))
            (setf temp2
                    (+ temp2
                       (*
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add l i) j)
                                       ((1 lda) (1 *))
                                       a-%offset%)
                        (f2cl-lib:fref x-%data%
                                       (ix)
                                       ((1 *))
                                       x-%offset%))))
            (setf ix (f2cl-lib:int-add ix incx))
            (setf iy (f2cl-lib:int-add iy incy))))
        (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                   (* temp1
                      (f2cl-lib:fref a-%data%
                                     (kplus1 j)
                                     ((1 lda) (1 *))
                                     a-%offset%))
                   (* alpha temp2)))
        (setf jx (f2cl-lib:int-add jx incx))
        (setf jy (f2cl-lib:int-add jy incy))
        (cond
          ((> j k)
           (setf kx (f2cl-lib:int-add kx incx))
           (setf ky (f2cl-lib:int-add ky incy)))))))))
(t
 (cond
   ((and (= incx 1) (= incy 1))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp1
                (* alpha
                   (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
```

```
        (setf temp2 zero)
        (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                   (* temp1
                      (f2cl-lib:fref a-%data%
                                     (1 j)
                                     ((1 lda) (1 *))
                                     a-%offset%))))
        (setf l (f2cl-lib:int-sub 1 j))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                       (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the fixnum n)
                               (the fixnum
                                    (f2cl-lib:int-add j k))))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                    (+
                     (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     (* temp1
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add l i) j)
                                       ((1 lda) (1 *))
                                       a-%offset%))))
            (setf temp2
                    (+ temp2
                       (*
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add l i) j)
                                       ((1 lda) (1 *))
                                       a-%offset%)
                        (f2cl-lib:fref x-%data%
                                       (i)
                                       ((1 *))
                                       x-%offset%))))))
        (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                   (* alpha temp2))))))
    (t
     (setf jx kx)
     (setf jy ky)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp1
                 (* alpha
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
         (setf temp2 zero)
         (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
```

```
                    (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                       (* temp1
                          (f2cl-lib:fref a-%data%
                                         (1 j)
                                         ((1 lda) (1 *))
                                         a-%offset%))))
                (setf l (f2cl-lib:int-sub 1 j))
                (setf ix jx)
                (setf iy jy)
                (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                               (f2cl-lib:int-add i 1))
                              ((> i
                                  (min (the fixnum n)
                                       (the fixnum
                                            (f2cl-lib:int-add j k))))
                               nil)
                  (tagbody
                    (setf ix (f2cl-lib:int-add ix incx))
                    (setf iy (f2cl-lib:int-add iy incy))
                    (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                            (+
                             (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                             (* temp1
                                (f2cl-lib:fref a-%data%
                                               ((f2cl-lib:int-add l i) j)
                                               ((1 lda) (1 *))
                                               a-%offset%))))
                    (setf temp2
                            (+ temp2
                               (*
                                (f2cl-lib:fref a-%data%
                                               ((f2cl-lib:int-add l i) j)
                                               ((1 lda) (1 *))
                                               a-%offset%)
                                (f2cl-lib:fref x-%data%
                                               (ix)
                                               ((1 *))
                                               x-%offset%))))))
                (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                        (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                           (* alpha temp2)))
                (setf jx (f2cl-lib:int-add jx incx))
                (setf jy (f2cl-lib:int-add jy incy))))))))
end_label
       (return (values nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

11.5 dspmv - matrix-vector operation $\alpha Ax + \beta y$

11.5.1 Axiom unit tests

— dspmv.input —

```
)set break resume
)sys rm -f dspmv.output
)spool dspmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

11.5.2 Axiom help page

— dspmv.help —

```
======================================================================
dspmv examples
======================================================================

======================================================================
Man Page Details
======================================================================

 NAME
      DSPMV - perform the matrix-vector operation   y := alpha*A*x
      + beta*y,

 SYNOPSIS
      SUBROUTINE DSPMV ( UPLO, N, ALPHA, AP, X, INCX, BETA, Y,
                         INCY )

          DOUBLE       PRECISION ALPHA, BETA

          INTEGER      INCX, INCY, N

          CHARACTER*1  UPLO

          DOUBLE       PRECISION AP( * ), X( * ), Y( * )
```

```
PURPOSE
     DSPMV  performs the matrix-vector operation

     where alpha and beta are scalars, x and y are n element vec-
     tors and A is an n by n symmetric matrix, supplied in packed
     form.

PARAMETERS
     UPLO   - CHARACTER*1.
              On entry, UPLO specifies whether the upper or lower
              triangular part of the matrix A is supplied in the
              packed array AP as follows:

              UPLO = 'U' or 'u'   The upper triangular part of A is
              supplied in AP.

              UPLO = 'L' or 'l'   The lower triangular part of A is
              supplied in AP.

              Unchanged on exit.

     N      - INTEGER.
              On entry, N specifies the order of the matrix A.  N
              must be at least zero.  Unchanged on exit.

     ALPHA  - DOUBLE PRECISION.
              On entry, ALPHA specifies the scalar alpha.
              Unchanged on exit.

     AP     - DOUBLE PRECISION array of DIMENSION at least
              ( ( n*( n + 1 ) )/2 ).  Before entry with UPLO = 'U'
              or 'u', the array AP must contain the upper triangu-
              lar part of the symmetric matrix packed sequentially,
              column by column, so that AP( 1 ) contains a( 1, 1 ),
              AP( 2 ) and AP( 3 ) contain a( 1, 2 ) and a( 2, 2 )

              respectively, and so on.  Before entry with UPLO =
              'L' or 'l', the array AP must contain the lower tri-
              angular part of the symmetric matrix packed sequen-
              tially, column by column, so that AP( 1 ) contains a(
              1, 1 ), AP( 2 ) and AP( 3 ) contain a( 2, 1 ) and a(
              3, 1 ) respectively, and so on.  Unchanged on exit.

     X      - DOUBLE PRECISION array of dimension at least
              ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
              incremented array X must contain the n element vector
              x.  Unchanged on exit.

     INCX   - INTEGER.
              On entry, INCX specifies the increment for the ele-
```

```
             ments of X. INCX must not be zero.  Unchanged on
             exit.

      BETA   - DOUBLE PRECISION.
             On entry, BETA specifies the scalar beta. When BETA
             is supplied as zero then Y need not be set on input.
             Unchanged on exit.

      Y      - DOUBLE PRECISION array of dimension at least
             ( 1 + ( n - 1 )*abs( INCY ) ).  Before entry, the
             incremented array Y must contain the n element vector
             y. On exit, Y is overwritten by the updated vector y.

      INCY   - INTEGER.
             On entry, INCY specifies the increment for the ele-
             ments of Y. INCY must not be zero.  Unchanged on
             exit.
```

———-

11.5.3 fortran code

— dspmv.f —

```
*  =====================================================================
      SUBROUTINE DSPMV(UPLO,N,ALPHA,AP,X,INCX,BETA,Y,INCY)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION ALPHA,BETA
      INTEGER INCX,INCY,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION AP(*),X(*),Y(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION ONE,ZERO
      PARAMETER (ONE=1.0D+0,ZERO=0.0D+0)
*     ..
```

```
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,K,KK,KX,KY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 6
      ELSE IF (INCY.EQ.0) THEN
          INFO = 9
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('DSPMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     Set up the start points in  X  and  Y.
*
      IF (INCX.GT.0) THEN
          KX = 1
      ELSE
          KX = 1 - (N-1)*INCX
      END IF
      IF (INCY.GT.0) THEN
          KY = 1
      ELSE
          KY = 1 - (N-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of the array AP
*     are accessed sequentially with one pass through AP.
*
*     First form  y := beta*y.
```

```
*
      IF (BETA.NE.ONE) THEN
          IF (INCY.EQ.1) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 10 I = 1,N
                      Y(I) = ZERO
   10             CONTINUE
              ELSE
                  DO 20 I = 1,N
                      Y(I) = BETA*Y(I)
   20             CONTINUE
              END IF
          ELSE
              IY = KY
              IF (BETA.EQ.ZERO) THEN
                  DO 30 I = 1,N
                      Y(IY) = ZERO
                      IY = IY + INCY
   30             CONTINUE
              ELSE
                  DO 40 I = 1,N
                      Y(IY) = BETA*Y(IY)
                      IY = IY + INCY
   40             CONTINUE
              END IF
          END IF
      END IF
      IF (ALPHA.EQ.ZERO) RETURN
      KK = 1
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  y  when AP contains the upper triangle.
*
          IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
              DO 60 J = 1,N
                  TEMP1 = ALPHA*X(J)
                  TEMP2 = ZERO
                  K = KK
                  DO 50 I = 1,J - 1
                      Y(I) = Y(I) + TEMP1*AP(K)
                      TEMP2 = TEMP2 + AP(K)*X(I)
                      K = K + 1
   50             CONTINUE
                  Y(J) = Y(J) + TEMP1*AP(KK+J-1) + ALPHA*TEMP2
                  KK = KK + J
   60         CONTINUE
          ELSE
              JX = KX
              JY = KY
              DO 80 J = 1,N
```

```
                TEMP1 = ALPHA*X(JX)
                TEMP2 = ZERO
                IX = KX
                IY = KY
                DO 70 K = KK,KK + J - 2
                    Y(IY) = Y(IY) + TEMP1*AP(K)
                    TEMP2 = TEMP2 + AP(K)*X(IX)
                    IX = IX + INCX
                    IY = IY + INCY
   70           CONTINUE
                Y(JY) = Y(JY) + TEMP1*AP(KK+J-1) + ALPHA*TEMP2
                JX = JX + INCX
                JY = JY + INCY
                KK = KK + J
   80       CONTINUE
        END IF
      ELSE
*
*       Form  y  when AP contains the lower triangle.
*
         IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
             DO 100 J = 1,N
                 TEMP1 = ALPHA*X(J)
                 TEMP2 = ZERO
                 Y(J) = Y(J) + TEMP1*AP(KK)
                 K = KK + 1
                 DO 90 I = J + 1,N
                     Y(I) = Y(I) + TEMP1*AP(K)
                     TEMP2 = TEMP2 + AP(K)*X(I)
                     K = K + 1
   90            CONTINUE
                 Y(J) = Y(J) + ALPHA*TEMP2
                 KK = KK + (N-J+1)
  100         CONTINUE
         ELSE
             JX = KX
             JY = KY
             DO 120 J = 1,N
                 TEMP1 = ALPHA*X(JX)
                 TEMP2 = ZERO
                 Y(JY) = Y(JY) + TEMP1*AP(KK)
                 IX = JX
                 IY = JY
                 DO 110 K = KK + 1,KK + N - J
                     IX = IX + INCX
                     IY = IY + INCY
                     Y(IY) = Y(IY) + TEMP1*AP(K)
                     TEMP2 = TEMP2 + AP(K)*X(IX)
  110            CONTINUE
                 Y(JY) = Y(JY) + ALPHA*TEMP2
```

```
                 JX = JX + INCX
                 JY = JY + INCY
                 KK = KK + (N-J+1)
  120         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of DSPMV .
*
      END
```

———-

11.5.4 lisp code

— BLAS 2 dspmv —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (double-float 1.0 1.0) one)
           (type (double-float 0.0 0.0) zero))
  (defun dspmv (uplo n alpha ap x incx beta y incy)
    (declare (type (simple-array double-float (*)) y x ap)
             (type (double-float) beta alpha)
             (type fixnum incy incx n)
             (type character uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (ap double-float ap-%data% ap-%offset%)
         (x double-float x-%data% x-%offset%)
         (y double-float y-%data% y-%offset%))
      (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (k 0) (kk 0)
             (kx 0) (ky 0) (temp1 0.0) (temp2 0.0))
        (declare (type fixnum i info ix iy j jx jy k kk kx ky)
                 (type (double-float) temp1 temp2))
        (setf info 0)
        (cond
          ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
           (setf info 1))
          ((< n 0)
           (setf info 2))
          ((= incx 0)
           (setf info 6))
          ((= incy 0)
           (setf info 9)))
        (cond
```

```
  ((/= info 0)
   (error
    " ** On entry to ~a parameter number ~a had an illegal value~%"
    "DSPMV" info)
   (go end_label)))
(if (or (= n 0) (and (= alpha zero) (= beta one))) (go end_label))
(cond
  ((> incx 0)
   (setf kx 1))
  (t
   (setf kx
           (f2cl-lib:int-sub 1
                             (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                               incx)))))
(cond
  ((> incy 0)
   (setf ky 1))
  (t
   (setf ky
           (f2cl-lib:int-sub 1
                             (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                               incy)))))
(cond
  ((/= beta one)
   (cond
     ((= incy 1)
      (cond
        ((= beta zero)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     zero))))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     (* beta
                        (f2cl-lib:fref y-%data%
                                       (i)
                                       ((1 *))
                                       y-%offset%))))))))
     (t
      (setf iy ky)
      (cond
        ((= beta zero)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
```

```
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        zero)
                (setf iy (f2cl-lib:int-add iy incy)))))
            (t
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i n) nil)
               (tagbody
                 (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (* beta
                            (f2cl-lib:fref y-%data%
                                           (iy)
                                           ((1 *))
                                           y-%offset%)))
                 (setf iy (f2cl-lib:int-add iy incy))))))))))
  (if (= alpha zero) (go end_label))
  (setf kk 1)
  (cond
    ((char-equal uplo #\U)
     (cond
       ((and (= incx 1) (= incy 1))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf temp1
                    (* alpha
                       (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
            (setf temp2 zero)
            (setf k kk)
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i
                              (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                         (* temp1
                            (f2cl-lib:fref ap-%data%
                                           (k)
                                           ((1 *))
                                           ap-%offset%))))
                (setf temp2
                        (+ temp2
                           (*
                            (f2cl-lib:fref ap-%data%
                                           (k)
                                           ((1 *))
                                           ap-%offset%)
                            (f2cl-lib:fref x-%data%
                                           (i)
```

```
                                        ((1 *))
                                        x-%offset%))))
            (setf k (f2cl-lib:int-add k 1))))
        (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                   (* temp1
                      (f2cl-lib:fref ap-%data%
                                     ((f2cl-lib:int-sub
                                       (f2cl-lib:int-add kk j)
                                       1))
                                     ((1 *))
                                     ap-%offset%))
                   (* alpha temp2)))
        (setf kk (f2cl-lib:int-add kk j)))))
    (t
     (setf jx kx)
     (setf jy ky)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp1
                 (* alpha
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
         (setf temp2 zero)
         (setf ix kx)
         (setf iy ky)
         (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                       ((> k
                           (f2cl-lib:int-add kk
                                             j
                                             (f2cl-lib:int-sub 2)))
                        nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (+
                      (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (* temp1
                         (f2cl-lib:fref ap-%data%
                                        (k)
                                        ((1 *))
                                        ap-%offset%))))
             (setf temp2
                     (+ temp2
                        (*
                         (f2cl-lib:fref ap-%data%
                                        (k)
                                        ((1 *))
                                        ap-%offset%)
                         (f2cl-lib:fref x-%data%
                                        (ix)
```

```
                                        ((1 *))
                                        x-%offset%))))
                (setf ix (f2cl-lib:int-add ix incx))
                (setf iy (f2cl-lib:int-add iy incy))))
            (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                       (* temp1
                          (f2cl-lib:fref ap-%data%
                                         ((f2cl-lib:int-sub
                                           (f2cl-lib:int-add kk j)
                                           1))
                                         ((1 *))
                                         ap-%offset%))
                       (* alpha temp2)))
            (setf jx (f2cl-lib:int-add jx incx))
            (setf jy (f2cl-lib:int-add jy incy))
            (setf kk (f2cl-lib:int-add kk j)))))))
    (t
     (cond
       ((and (= incx 1) (= incy 1))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf temp1
                    (* alpha
                       (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
            (setf temp2 zero)
            (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                       (* temp1
                          (f2cl-lib:fref ap-%data%
                                         (kk)
                                         ((1 *))
                                         ap-%offset%))))
            (setf k (f2cl-lib:int-add kk 1))
            (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                           (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                         (* temp1
                            (f2cl-lib:fref ap-%data%
                                           (k)
                                           ((1 *))
                                           ap-%offset%))))
                (setf temp2
                        (+ temp2
                           (*
```

```
                        (f2cl-lib:fref ap-%data%
                                       (k)
                                       ((1 *))
                                       ap-%offset%)
                        (f2cl-lib:fref x-%data%
                                       (i)
                                       ((1 *))
                                       x-%offset%))))
            (setf k (f2cl-lib:int-add k 1))))
        (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                   (* alpha temp2)))
        (setf kk
                (f2cl-lib:int-add kk
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-sub n j)
                                   1))))))
  (t
   (setf jx kx)
   (setf jy ky)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp1
               (* alpha
                  (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
       (setf temp2 zero)
       (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
               (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                  (* temp1
                     (f2cl-lib:fref ap-%data%
                                    (kk)
                                    ((1 *))
                                    ap-%offset%))))
       (setf ix jx)
       (setf iy jy)
       (f2cl-lib:fdo (k (f2cl-lib:int-add kk 1)
                        (f2cl-lib:int-add k 1))
                     ((> k
                         (f2cl-lib:int-add kk
                                           n
                                           (f2cl-lib:int-sub j)))
                      nil)
         (tagbody
           (setf ix (f2cl-lib:int-add ix incx))
           (setf iy (f2cl-lib:int-add iy incy))
           (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                   (+
                    (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (* temp1
```

```
                                (f2cl-lib:fref ap-%data%
                                               (k)
                                               ((1 *))
                                               ap-%offset%))))
                        (setf temp2
                                (+ temp2
                                   (*
                                (f2cl-lib:fref ap-%data%
                                               (k)
                                               ((1 *))
                                               ap-%offset%)
                                (f2cl-lib:fref x-%data%
                                               (ix)
                                               ((1 *))
                                               x-%offset%))))))
                    (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                            (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                               (* alpha temp2)))
                    (setf jx (f2cl-lib:int-add jx incx))
                    (setf jy (f2cl-lib:int-add jy incy))
                    (setf kk
                            (f2cl-lib:int-add kk
                                              (f2cl-lib:int-add
                                               (f2cl-lib:int-sub n j)
                                               1)))))))))
 end_label
        (return (values nil nil nil nil nil nil nil nil nil))))))
```

———-

11.6 dspr - symmetric rank 1 $\alpha xx^T + A$

11.6.1 Axiom unit tests

— dspr.input —

```
)set break resume
)sys rm -f dspr.output
)spool dspr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

11.6.2 Axiom help page

— dspr.help —

```
==========================================================================
dspr examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

 NAME
      DSPR - perform the symmetric rank 1 operation   A :=
      alpha*x*x' + A,

 SYNOPSIS
      SUBROUTINE DSPR ( UPLO, N, ALPHA, X, INCX, AP )

          DOUBLE      PRECISION ALPHA

          INTEGER     INCX, N

          CHARACTER*1 UPLO

          DOUBLE      PRECISION AP( * ), X( * )

 PURPOSE
      DSPR    performs the symmetric rank 1 operation

      where alpha is a real scalar, x is an n element vector and A
      is an n by n symmetric matrix, supplied in packed form.

 PARAMETERS
      UPLO   - CHARACTER*1.
             On entry, UPLO specifies whether the upper or lower
             triangular part of the matrix A is supplied in the
             packed array AP as follows:

             UPLO = 'U' or 'u'   The upper triangular part of A is
             supplied in AP.

             UPLO = 'L' or 'l'   The lower triangular part of A is
             supplied in AP.

             Unchanged on exit.
```

```
N      - INTEGER.
         On entry, N specifies the order of the matrix A.  N
         must be at least zero.  Unchanged on exit.

ALPHA  - DOUBLE PRECISION.
         On entry, ALPHA specifies the scalar alpha.
         Unchanged on exit.

X      - DOUBLE PRECISION array of dimension at least
         ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
         incremented array X must contain the n element vector
         x.  Unchanged on exit.

INCX   - INTEGER.
         On entry, INCX specifies the increment for the ele-
         ments of X. INCX must not be zero.  Unchanged on

         exit.

AP     - DOUBLE PRECISION array of DIMENSION at least
         ( ( n*( n + 1 ) )/2 ).  Before entry with  UPLO = 'U'
         or 'u', the array AP must contain the upper triangu-
         lar part of the symmetric matrix packed sequentially,
         column by column, so that AP( 1 ) contains a( 1, 1 ),
         AP( 2 ) and AP( 3 ) contain a( 1, 2 ) and a( 2, 2 )
         respectively, and so on. On exit, the array AP is
         overwritten by the upper triangular part of the
         updated matrix.  Before entry with UPLO = 'L' or 'l',
         the array AP must contain the lower triangular part
         of the symmetric matrix packed sequentially, column
         by column, so that AP( 1 ) contains a( 1, 1 ), AP( 2
         ) and AP( 3 ) contain a( 2, 1 ) and a( 3, 1 ) respec-
         tively, and so on. On exit, the array AP is overwrit-
         ten by the lower triangular part of the updated
         matrix.
```

———-

11.6.3 fortran code

— dspr.f —

```
*  ====================================================================
      SUBROUTINE DSPR(UPLO,N,ALPHA,X,INCX,AP)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
```

```
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION ALPHA
      INTEGER INCX,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION AP(*),X(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION ZERO
      PARAMETER (ZERO=0.0D+0)
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP
      INTEGER I,INFO,IX,J,JX,K,KK,KX
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('DSPR  ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (ALPHA.EQ.ZERO)) RETURN
*
```

```
*     Set the start point in X if the increment is not unity.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of the array AP
*     are accessed sequentially with one pass through AP.
*
      KK = 1
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  A  when upper triangle is stored in AP.
*
          IF (INCX.EQ.1) THEN
              DO 20 J = 1,N
                  IF (X(J).NE.ZERO) THEN
                      TEMP = ALPHA*X(J)
                      K = KK
                      DO 10 I = 1,J
                          AP(K) = AP(K) + X(I)*TEMP
                          K = K + 1
   10                 CONTINUE
                  END IF
                  KK = KK + J
   20         CONTINUE
          ELSE
              JX = KX
              DO 40 J = 1,N
                  IF (X(JX).NE.ZERO) THEN
                      TEMP = ALPHA*X(JX)
                      IX = KX
                      DO 30 K = KK,KK + J - 1
                          AP(K) = AP(K) + X(IX)*TEMP
                          IX = IX + INCX
   30                 CONTINUE
                  END IF
                  JX = JX + INCX
                  KK = KK + J
   40         CONTINUE
          END IF
      ELSE
*
*        Form  A  when lower triangle is stored in AP.
*
          IF (INCX.EQ.1) THEN
              DO 60 J = 1,N
                  IF (X(J).NE.ZERO) THEN
```

```
                    TEMP = ALPHA*X(J)
                    K = KK
                    DO 50 I = J,N
                        AP(K) = AP(K) + X(I)*TEMP
                        K = K + 1
   50               CONTINUE
                END IF
                KK = KK + N - J + 1
   60         CONTINUE
        ELSE
            JX = KX
            DO 80 J = 1,N
                IF (X(JX).NE.ZERO) THEN
                    TEMP = ALPHA*X(JX)
                    IX = JX
                    DO 70 K = KK,KK + N - J
                        AP(K) = AP(K) + X(IX)*TEMP
                        IX = IX + INCX
   70               CONTINUE
                END IF
                JX = JX + INCX
                KK = KK + N - J + 1
   80         CONTINUE
        END IF
      END IF
*
      RETURN
*
*     End of DSPR  .
*
      END
```

———-

11.6.4 lisp code

— BLAS 2 dspr —

```
(let* ((zero 0.0))
  (declare (type (double-float 0.0 0.0) zero))
  (defun dspr (uplo n alpha x incx ap)
    (declare (type (simple-array double-float (*)) ap x)
             (type (double-float) alpha)
             (type fixnum incx n)
             (type character uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
```

```
   (x double-float x-%data% x-%offset%)
   (ap double-float ap-%data% ap-%offset%))
(prog ((i 0) (info 0) (ix 0) (j 0) (jx 0) (k 0) (kk 0) (kx 0) (temp 0.0))
  (declare (type fixnum i info ix j jx k kk kx)
           (type (double-float) temp))
  (setf info 0)
  (cond
    ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
     (setf info 1))
    ((< n 0)
     (setf info 2))
    ((= incx 0)
     (setf info 5)))
  (cond
    ((/= info 0)
     (error
      " ** On entry to ~a parameter number ~a had an illegal value~%"
       "DSPR" info)
     (go end_label)))
  (if (or (= n 0) (= alpha zero)) (go end_label))
  (cond
    ((<= incx 0)
     (setf kx
             (f2cl-lib:int-sub 1
                               (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                 incx))))
    ((/= incx 1)
     (setf kx 1)))
  (setf kk 1)
  (cond
    ((char-equal uplo #\U)
     (cond
       ((= incx 1)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (cond
              ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
               (setf temp
                       (* alpha
                          (f2cl-lib:fref x-%data%
                                         (j)
                                         ((1 *))
                                         x-%offset%)))
               (setf k kk)
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i j) nil)
                 (tagbody
                   (setf (f2cl-lib:fref ap-%data%
                                        (k)
```

```
                                    ((1 *))
                                    ap-%offset%)
                          (+
                           (f2cl-lib:fref ap-%data%
                                          (k)
                                          ((1 *))
                                          ap-%offset%)
                           (*
                            (f2cl-lib:fref x-%data%
                                           (i)
                                           ((1 *))
                                           x-%offset%)
                            temp)))
                 (setf k (f2cl-lib:int-add k 1))))))
         (setf kk (f2cl-lib:int-add kk j)))))
    (t
     (setf jx kx)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (cond
           ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
            (setf temp
                    (* alpha
                       (f2cl-lib:fref x-%data%
                                      (jx)
                                      ((1 *))
                                      x-%offset%)))
            (setf ix kx)
            (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                          ((> k
                              (f2cl-lib:int-add kk
                                                j
                                                (f2cl-lib:int-sub 1)))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref ap-%data%
                                     (k)
                                     ((1 *))
                                     ap-%offset%)
                      (+
                       (f2cl-lib:fref ap-%data%
                                      (k)
                                      ((1 *))
                                      ap-%offset%)
                       (*
                        (f2cl-lib:fref x-%data%
                                       (ix)
                                       ((1 *))
                                       x-%offset%)
```

```
                          temp)))
                 (setf ix (f2cl-lib:int-add ix incx))))))
          (setf jx (f2cl-lib:int-add jx incx))
          (setf kk (f2cl-lib:int-add kk j)))))))
(t
 (cond
   ((= incx 1)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
          ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
           (setf temp
                   (* alpha
                      (f2cl-lib:fref x-%data%
                                     (j)
                                     ((1 *))
                                     x-%offset%)))
           (setf k kk)
           (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref ap-%data%
                                    (k)
                                    ((1 *))
                                    ap-%offset%)
                     (+
                      (f2cl-lib:fref ap-%data%
                                     (k)
                                     ((1 *))
                                     ap-%offset%)
                      (*
                       (f2cl-lib:fref x-%data%
                                      (i)
                                      ((1 *))
                                      x-%offset%)
                       temp)))
               (setf k (f2cl-lib:int-add k 1))))))
          (setf kk
                  (f2cl-lib:int-add
                   (f2cl-lib:int-sub (f2cl-lib:int-add kk n) j)
                   1)))))
   (t
    (setf jx kx)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
          ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
           (setf temp
```

```
                        (* alpha
                           (f2cl-lib:fref x-%data%
                                          (jx)
                                          ((1 *))
                                          x-%offset%)))
                    (setf ix jx)
                    (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                                  ((> k
                                      (f2cl-lib:int-add kk
                                                        n
                                                        (f2cl-lib:int-sub j)))
                                   nil)
                      (tagbody
                        (setf (f2cl-lib:fref ap-%data%
                                             (k)
                                             ((1 *))
                                             ap-%offset%)
                              (+
                               (f2cl-lib:fref ap-%data%
                                              (k)
                                              ((1 *))
                                              ap-%offset%)
                               (*
                                (f2cl-lib:fref x-%data%
                                               (ix)
                                               ((1 *))
                                               x-%offset%)
                                temp)))
                        (setf ix (f2cl-lib:int-add ix incx))))))
                  (setf jx (f2cl-lib:int-add jx incx))
                  (setf kk
                          (f2cl-lib:int-add
                           (f2cl-lib:int-sub (f2cl-lib:int-add kk n) j)
                           1))))))))
 end_label
        (return (values nil nil nil nil nil nil))))))
```

———-

11.7 dspr2 - symmetric rank 2 $\alpha xy^T + \alpha yx^T + A$

11.7.1 Axiom unit tests

— **dspr2.input** —

```
)set break resume
```

```
)sys rm -f dspr2.output
)spool dspr2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

11.7.2 Axiom help page

— dspr2.help —

```
======================================================================
dspr2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

 NAME
      DSPR2 - perform the symmetric rank 2 operation   A :=
      alpha*x*y' + alpha*y*x' + A,

 SYNOPSIS
      SUBROUTINE DSPR2 ( UPLO, N, ALPHA, X, INCX, Y, INCY, AP )

          DOUBLE       PRECISION ALPHA

          INTEGER      INCX, INCY, N

          CHARACTER*1  UPLO

          DOUBLE       PRECISION AP( * ), X( * ), Y( * )

 PURPOSE
      DSPR2  performs the symmetric rank 2 operation

      where alpha is a scalar, x and y are n element vectors and A
      is an n by n symmetric matrix, supplied in packed form.

 PARAMETERS
      UPLO   - CHARACTER*1.
               On entry, UPLO specifies whether the upper or lower
               triangular part of the matrix A is supplied in the
```

```
        packed array AP as follows:

        UPLO = 'U' or 'u'   The upper triangular part of A is
        supplied in AP.

        UPLO = 'L' or 'l'   The lower triangular part of A is
        supplied in AP.

        Unchanged on exit.

N       - INTEGER.
        On entry, N specifies the order of the matrix A.  N
        must be at least zero.  Unchanged on exit.

ALPHA   - DOUBLE PRECISION.
        On entry, ALPHA specifies the scalar alpha.
        Unchanged on exit.

X       - DOUBLE PRECISION array of dimension at least
        ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
        incremented array X must contain the n element vector
        x.  Unchanged on exit.

INCX    - INTEGER.
        On entry, INCX specifies the increment for the ele-
        ments of X. INCX must not be zero.  Unchanged on

        exit.

Y       - DOUBLE PRECISION array of dimension at least
        ( 1 + ( n - 1 )*abs( INCY ) ).  Before entry, the
        incremented array Y must contain the n element vector
        y.  Unchanged on exit.

INCY    - INTEGER.
        On entry, INCY specifies the increment for the ele-
        ments of Y. INCY must not be zero.  Unchanged on
        exit.

AP      - DOUBLE PRECISION array of DIMENSION at least
        ( ( n*( n + 1 ) )/2 ).  Before entry with  UPLO = 'U'
        or 'u', the array AP must contain the upper triangu-
        lar part of the symmetric matrix packed sequentially,
        column by column, so that AP( 1 ) contains a( 1, 1 ),
        AP( 2 ) and AP( 3 ) contain a( 1, 2 ) and a( 2, 2 )
        respectively, and so on. On exit, the array AP is
        overwritten by the upper triangular part of the
        updated matrix.  Before entry with UPLO = 'L' or 'l',
        the array AP must contain the lower triangular part
        of the symmetric matrix packed sequentially, column
```

```
by column, so that AP( 1 ) contains a( 1, 1 ), AP( 2
) and AP( 3 ) contain a( 2, 1 ) and a( 3, 1 ) respec-
tively, and so on. On exit, the array AP is overwrit-
ten by the lower triangular part of the updated
matrix.
```

———-

11.7.3 fortran code

— dspr2.f —

```
*  ===================================================================
      SUBROUTINE DSPR2(UPLO,N,ALPHA,X,INCX,Y,INCY,AP)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION ALPHA
      INTEGER INCX,INCY,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION AP(*),X(*),Y(*)
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION ZERO
      PARAMETER (ZERO=0.0D+0)
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,K,KK,KX,KY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*
```

```
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      ELSE IF (INCY.EQ.0) THEN
          INFO = 7
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('DSPR2 ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (ALPHA.EQ.ZERO)) RETURN
*
*     Set up the start points in X and Y if the increments are not both
*     unity.
*
      IF ((INCX.NE.1) .OR. (INCY.NE.1)) THEN
          IF (INCX.GT.0) THEN
              KX = 1
          ELSE
              KX = 1 - (N-1)*INCX
          END IF
          IF (INCY.GT.0) THEN
              KY = 1
          ELSE
              KY = 1 - (N-1)*INCY
          END IF
          JX = KX
          JY = KY
      END IF
*
*     Start the operations. In this version the elements of the array AP
*     are accessed sequentially with one pass through AP.
*
      KK = 1
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  A  when upper triangle is stored in AP.
*
          IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
              DO 20 J = 1,N
                  IF ((X(J).NE.ZERO) .OR. (Y(J).NE.ZERO)) THEN
```

```
                    TEMP1 = ALPHA*Y(J)
                    TEMP2 = ALPHA*X(J)
                    K = KK
                    DO 10 I = 1,J
                        AP(K) = AP(K) + X(I)*TEMP1 + Y(I)*TEMP2
                        K = K + 1
   10               CONTINUE
                END IF
                KK = KK + J
   20       CONTINUE
        ELSE
            DO 40 J = 1,N
                IF ((X(JX).NE.ZERO) .OR. (Y(JY).NE.ZERO)) THEN
                    TEMP1 = ALPHA*Y(JY)
                    TEMP2 = ALPHA*X(JX)
                    IX = KX
                    IY = KY
                    DO 30 K = KK,KK + J - 1
                        AP(K) = AP(K) + X(IX)*TEMP1 + Y(IY)*TEMP2
                        IX = IX + INCX
                        IY = IY + INCY
   30               CONTINUE
                END IF
                JX = JX + INCX
                JY = JY + INCY
                KK = KK + J
   40       CONTINUE
        END IF
     ELSE
*
*       Form  A  when lower triangle is stored in AP.
*
         IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
            DO 60 J = 1,N
                IF ((X(J).NE.ZERO) .OR. (Y(J).NE.ZERO)) THEN
                    TEMP1 = ALPHA*Y(J)
                    TEMP2 = ALPHA*X(J)
                    K = KK
                    DO 50 I = J,N
                        AP(K) = AP(K) + X(I)*TEMP1 + Y(I)*TEMP2
                        K = K + 1
   50               CONTINUE
                END IF
                KK = KK + N - J + 1
   60       CONTINUE
        ELSE
            DO 80 J = 1,N
                IF ((X(JX).NE.ZERO) .OR. (Y(JY).NE.ZERO)) THEN
                    TEMP1 = ALPHA*Y(JY)
                    TEMP2 = ALPHA*X(JX)
```

```
                    IX = JX
                    IY = JY
                    DO 70 K = KK,KK + N - J
                        AP(K) = AP(K) + X(IX)*TEMP1 + Y(IY)*TEMP2
                        IX = IX + INCX
                        IY = IY + INCY
   70               CONTINUE
                END IF
                JX = JX + INCX
                JY = JY + INCY
                KK = KK + N - J + 1
   80         CONTINUE
         END IF
      END IF
*
      RETURN
*
*     End of DSPR2 .
*
      END
```

———-

11.7.4 lisp code

— BLAS 2 dspr2 —

```
(let* ((zero 0.0))
  (declare (type (double-float 0.0 0.0) zero))
  (defun dspr2 (uplo n alpha x incx y incy ap)
    (declare (type (simple-array double-float (*)) ap y x)
             (type (double-float) alpha)
             (type fixnum incy incx n)
             (type character uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (x double-float x-%data% x-%offset%)
         (y double-float y-%data% y-%offset%)
         (ap double-float ap-%data% ap-%offset%))
      (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (k 0) (kk 0)
             (kx 0) (ky 0) (temp1 0.0) (temp2 0.0))
        (declare (type fixnum i info ix iy j jx jy k kk kx ky)
                 (type (double-float) temp1 temp2))
        (setf info 0)
        (cond
          ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
           (setf info 1))
```

```
    ((< n 0)
     (setf info 2))
    ((= incx 0)
     (setf info 5))
    ((= incy 0)
     (setf info 7)))
  (cond
    ((/= info 0)
     (error
      " ** On entry to ~a parameter number ~a had an illegal value~%"
      "DSPR2" info)
     (go end_label)))
  (if (or (= n 0) (= alpha zero)) (go end_label))
  (cond
    ((or (/= incx 1) (/= incy 1))
     (cond
       ((> incx 0)
        (setf kx 1))
       (t
        (setf kx
                (f2cl-lib:int-sub 1
                                  (f2cl-lib:int-mul
                                   (f2cl-lib:int-sub n 1)
                                   incx)))))
     (cond
       ((> incy 0)
        (setf ky 1))
       (t
        (setf ky
                (f2cl-lib:int-sub 1
                                  (f2cl-lib:int-mul
                                   (f2cl-lib:int-sub n 1)
                                   incy)))))
     (setf jx kx)
     (setf jy ky)))
  (setf kk 1)
  (cond
    ((char-equal uplo #\U)
     (cond
       ((and (= incx 1) (= incy 1))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (cond
              ((or (/= (f2cl-lib:fref x (j) ((1 *))) zero)
                   (/= (f2cl-lib:fref y (j) ((1 *))) zero))
               (setf temp1
                       (* alpha
                          (f2cl-lib:fref y-%data%
                                         (j)
```

```
                                ((1 *))
                                y-%offset%)))
          (setf temp2
                  (* alpha
                     (f2cl-lib:fref x-%data%
                                    (j)
                                    ((1 *))
                                    x-%offset%)))
          (setf k kk)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref ap-%data%
                                   (k)
                                   ((1 *))
                                   ap-%offset%)
                    (+
                     (f2cl-lib:fref ap-%data%
                                    (k)
                                    ((1 *))
                                    ap-%offset%)
                     (*
                      (f2cl-lib:fref x-%data%
                                     (i)
                                     ((1 *))
                                     x-%offset%)
                      temp1)
                     (*
                      (f2cl-lib:fref y-%data%
                                     (i)
                                     ((1 *))
                                     y-%offset%)
                      temp2)))
              (setf k (f2cl-lib:int-add k 1))))))
      (setf kk (f2cl-lib:int-add kk j)))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((or (/= (f2cl-lib:fref x (jx) ((1 *))) zero)
            (/= (f2cl-lib:fref y (jy) ((1 *))) zero))
        (setf temp1
                (* alpha
                   (f2cl-lib:fref y-%data%
                                  (jy)
                                  ((1 *))
                                  y-%offset%)))
        (setf temp2
                (* alpha
```

```
                        (f2cl-lib:fref x-%data%
                                       (jx)
                                       ((1 *))
                                       x-%offset%)))
                (setf ix kx)
                (setf iy ky)
                (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                              ((> k
                                  (f2cl-lib:int-add kk
                                                    j
                                                    (f2cl-lib:int-sub 1)))
                               nil)
                  (tagbody
                    (setf (f2cl-lib:fref ap-%data%
                                         (k)
                                         ((1 *))
                                         ap-%offset%)
                          (+
                           (f2cl-lib:fref ap-%data%
                                          (k)
                                          ((1 *))
                                          ap-%offset%)
                           (*
                            (f2cl-lib:fref x-%data%
                                           (ix)
                                           ((1 *))
                                           x-%offset%)
                            temp1)
                           (*
                            (f2cl-lib:fref y-%data%
                                           (iy)
                                           ((1 *))
                                           y-%offset%)
                            temp2)))
                    (setf ix (f2cl-lib:int-add ix incx))
                    (setf iy (f2cl-lib:int-add iy incy))))))
            (setf jx (f2cl-lib:int-add jx incx))
            (setf jy (f2cl-lib:int-add jy incy))
            (setf kk (f2cl-lib:int-add kk j)))))))
        (t
         (cond
           ((and (= incx 1) (= incy 1))
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (cond
                  ((or (/= (f2cl-lib:fref x (j) ((1 *))) zero)
                       (/= (f2cl-lib:fref y (j) ((1 *))) zero))
                   (setf temp1
                         (* alpha
```

```
                    (f2cl-lib:fref y-%data%
                                   (j)
                                   ((1 *))
                                   y-%offset%)))
          (setf temp2
                  (* alpha
                     (f2cl-lib:fref x-%data%
                                    (j)
                                    ((1 *))
                                    x-%offset%)))
          (setf k kk)
          (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref ap-%data%
                                   (k)
                                   ((1 *))
                                   ap-%offset%)
                    (+
                     (f2cl-lib:fref ap-%data%
                                    (k)
                                    ((1 *))
                                    ap-%offset%)
                     (*
                      (f2cl-lib:fref x-%data%
                                     (i)
                                     ((1 *))
                                     x-%offset%)
                      temp1)
                     (*
                      (f2cl-lib:fref y-%data%
                                     (i)
                                     ((1 *))
                                     y-%offset%)
                      temp2)))
              (setf k (f2cl-lib:int-add k 1))))))
      (setf kk
              (f2cl-lib:int-add
               (f2cl-lib:int-sub (f2cl-lib:int-add kk n) j)
               1)))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((or (/= (f2cl-lib:fref x (jx) ((1 *))) zero)
            (/= (f2cl-lib:fref y (jy) ((1 *))) zero))
        (setf temp1
                (* alpha
                   (f2cl-lib:fref y-%data%
```

```
                                        (jy)
                                        ((1 *))
                                        y-%offset%)))
                    (setf temp2
                            (* alpha
                               (f2cl-lib:fref x-%data%
                                        (jx)
                                        ((1 *))
                                        x-%offset%)))
                    (setf ix jx)
                    (setf iy jy)
                    (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                                  ((> k
                                      (f2cl-lib:int-add kk
                                                        n
                                                        (f2cl-lib:int-sub j)))
                                   nil)
                      (tagbody
                        (setf (f2cl-lib:fref ap-%data%
                                             (k)
                                             ((1 *))
                                             ap-%offset%)
                              (+
                               (f2cl-lib:fref ap-%data%
                                              (k)
                                              ((1 *))
                                              ap-%offset%)
                               (*
                                (f2cl-lib:fref x-%data%
                                               (ix)
                                               ((1 *))
                                               x-%offset%)
                                temp1)
                               (*
                                (f2cl-lib:fref y-%data%
                                               (iy)
                                               ((1 *))
                                               y-%offset%)
                                temp2)))
                        (setf ix (f2cl-lib:int-add ix incx))
                        (setf iy (f2cl-lib:int-add iy incy))))))
                  (setf jx (f2cl-lib:int-add jx incx))
                  (setf jy (f2cl-lib:int-add jy incy))
                  (setf kk
                          (f2cl-lib:int-add
                           (f2cl-lib:int-sub (f2cl-lib:int-add kk n) j)
                           1))))))))
end_label
       (return (values nil nil nil nil nil nil nil nil))))))
```

11.8 dsymv - matrix-vector operation $\alpha Ax + \beta y$

11.8.1 Axiom unit tests

— dsymv.input —

```
)set break resume
)sys rm -f dsymv.output
)spool dsymv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

11.8.2 Axiom help page

— dsymv.help —

```
====================================================================
dsymv examples
====================================================================

====================================================================
Man Page Details
====================================================================

 NAME
      DSYMV - perform the matrix-vector operation   y := alpha*A*x
      + beta*y,

 SYNOPSIS
      SUBROUTINE DSYMV ( UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y,
                         INCY )

          DOUBLE       PRECISION ALPHA, BETA

          INTEGER      INCX, INCY, LDA, N
```

```
      CHARACTER*1  UPLO

      DOUBLE       PRECISION A( LDA, * ), X( * ), Y( * )

PURPOSE
     DSYMV  performs the matrix-vector  operation

     where alpha and beta are scalars, x and y are n element vec-
     tors and A is an n by n symmetric matrix.

PARAMETERS
     UPLO   - CHARACTER*1.
              On entry, UPLO specifies whether the upper or lower
              triangular part of the array A is to be referenced as
              follows:

              UPLO = 'U' or 'u'   Only the upper triangular part of
              A is to be referenced.

              UPLO = 'L' or 'l'   Only the lower triangular part of
              A is to be referenced.

              Unchanged on exit.

     N      - INTEGER.
              On entry, N specifies the order of the matrix A.  N
              must be at least zero.  Unchanged on exit.

     ALPHA  - DOUBLE PRECISION.
              On entry, ALPHA specifies the scalar alpha.
              Unchanged on exit.

     A      - DOUBLE PRECISION array of DIMENSION ( LDA, n ).
              Before entry with  UPLO = 'U' or 'u', the leading n
              by n upper triangular part of the array A must con-
              tain the upper triangular part of the symmetric
              matrix and the strictly lower triangular part of A is
              not referenced.  Before entry with UPLO = 'L' or 'l',
              the leading n by n lower triangular part of the array

              A must contain the lower triangular part of the sym-
              metric matrix and the strictly upper triangular part
              of A is not referenced.  Unchanged on exit.

     LDA    - INTEGER.
              On entry, LDA specifies the first dimension of A as
              declared in the calling (sub) program. LDA must be at
              least max( 1, n ).  Unchanged on exit.

     X      - DOUBLE PRECISION array of dimension at least
```

```
             ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
             incremented array X must contain the n element vector
             x.  Unchanged on exit.

      INCX   - INTEGER.
             On entry, INCX specifies the increment for the ele-
             ments of X. INCX must not be zero.  Unchanged on
             exit.

      BETA   - DOUBLE PRECISION.
             On entry, BETA specifies the scalar beta. When BETA
             is supplied as zero then Y need not be set on input.
             Unchanged on exit.

      Y      - DOUBLE PRECISION array of dimension at least
             ( 1 + ( n - 1 )*abs( INCY ) ).  Before entry, the
             incremented array Y must contain the n element vector
             y. On exit, Y is overwritten by the updated vector y.

      INCY   - INTEGER.
             On entry, INCY specifies the increment for the ele-
             ments of Y. INCY must not be zero.  Unchanged on
             exit.
```

———-

11.8.3 fortran code

— dsymv.f —

```
*  =====================================================================
      SUBROUTINE DSYMV(UPLO,N,ALPHA,A,LDA,X,INCX,BETA,Y,INCY)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION ALPHA,BETA
      INTEGER INCX,INCY,LDA,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A(LDA,*),X(*),Y(*)
*     ..
*
```

```
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION ONE,ZERO
      PARAMETER (ONE=1.0D+0,ZERO=0.0D+0)
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,KX,KY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 5
      ELSE IF (INCX.EQ.0) THEN
          INFO = 7
      ELSE IF (INCY.EQ.0) THEN
          INFO = 10
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('DSYMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     Set up the start points in  X  and  Y.
*
      IF (INCX.GT.0) THEN
          KX = 1
      ELSE
          KX = 1 - (N-1)*INCX
```

```
      END IF
      IF (INCY.GT.0) THEN
          KY = 1
      ELSE
          KY = 1 - (N-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the triangular part
*     of A.
*
*     First form  y := beta*y.
*
      IF (BETA.NE.ONE) THEN
          IF (INCY.EQ.1) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 10 I = 1,N
                      Y(I) = ZERO
   10             CONTINUE
              ELSE
                  DO 20 I = 1,N
                      Y(I) = BETA*Y(I)
   20             CONTINUE
              END IF
          ELSE
              IY = KY
              IF (BETA.EQ.ZERO) THEN
                  DO 30 I = 1,N
                      Y(IY) = ZERO
                      IY = IY + INCY
   30             CONTINUE
              ELSE
                  DO 40 I = 1,N
                      Y(IY) = BETA*Y(IY)
                      IY = IY + INCY
   40             CONTINUE
              END IF
          END IF
      END IF
      IF (ALPHA.EQ.ZERO) RETURN
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  y  when A is stored in upper triangle.
*
          IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
              DO 60 J = 1,N
                  TEMP1 = ALPHA*X(J)
                  TEMP2 = ZERO
                  DO 50 I = 1,J - 1
                      Y(I) = Y(I) + TEMP1*A(I,J)
```

```
                    TEMP2 = TEMP2 + A(I,J)*X(I)
   50            CONTINUE
                 Y(J) = Y(J) + TEMP1*A(J,J) + ALPHA*TEMP2
   60         CONTINUE
          ELSE
              JX = KX
              JY = KY
              DO 80 J = 1,N
                  TEMP1 = ALPHA*X(JX)
                  TEMP2 = ZERO
                  IX = KX
                  IY = KY
                  DO 70 I = 1,J - 1
                      Y(IY) = Y(IY) + TEMP1*A(I,J)
                      TEMP2 = TEMP2 + A(I,J)*X(IX)
                      IX = IX + INCX
                      IY = IY + INCY
   70             CONTINUE
                  Y(JY) = Y(JY) + TEMP1*A(J,J) + ALPHA*TEMP2
                  JX = JX + INCX
                  JY = JY + INCY
   80         CONTINUE
          END IF
      ELSE
*
*        Form  y  when A is stored in lower triangle.
*
          IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
              DO 100 J = 1,N
                  TEMP1 = ALPHA*X(J)
                  TEMP2 = ZERO
                  Y(J) = Y(J) + TEMP1*A(J,J)
                  DO 90 I = J + 1,N
                      Y(I) = Y(I) + TEMP1*A(I,J)
                      TEMP2 = TEMP2 + A(I,J)*X(I)
   90             CONTINUE
                  Y(J) = Y(J) + ALPHA*TEMP2
  100         CONTINUE
          ELSE
              JX = KX
              JY = KY
              DO 120 J = 1,N
                  TEMP1 = ALPHA*X(JX)
                  TEMP2 = ZERO
                  Y(JY) = Y(JY) + TEMP1*A(J,J)
                  IX = JX
                  IY = JY
                  DO 110 I = J + 1,N
                      IX = IX + INCX
                      IY = IY + INCY
```

```
                    Y(IY) = Y(IY) + TEMP1*A(I,J)
                    TEMP2 = TEMP2 + A(I,J)*X(IX)
  110            CONTINUE
                 Y(JY) = Y(JY) + ALPHA*TEMP2
                 JX = JX + INCX
                 JY = JY + INCY
  120         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of DSYMV .
*
      END
```

———-

11.8.4 lisp code

— BLAS 2 dsymv —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (double-float 1.0 1.0) one)
           (type (double-float 0.0 0.0) zero))
  (defun dsymv (uplo n alpha a lda x incx beta y incy)
    (declare (type (simple-array double-float (*)) y x a)
             (type (double-float) beta alpha)
             (type fixnum incy incx lda n)
             (type character uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (x double-float x-%data% x-%offset%)
         (y double-float y-%data% y-%offset%))
      (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (kx 0) (ky 0)
             (temp1 0.0) (temp2 0.0))
        (declare (type fixnum i info ix iy j jx jy kx ky)
                 (type (double-float) temp1 temp2))
        (setf info 0)
        (cond
          ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
           (setf info 1))
          ((< n 0)
           (setf info 2))
          ((< lda (max (the fixnum 1) (the fixnum n)))
           (setf info 5))
```

```
    ((= incx 0)
     (setf info 7))
    ((= incy 0)
     (setf info 10)))
  (cond
    ((/= info 0)
     (error
      " ** On entry to ~a parameter number ~a had an illegal value~%"
      "DSYMV" info)
     (go end_label)))
  (if (or (= n 0) (and (= alpha zero) (= beta one))) (go end_label))
  (cond
    ((> incx 0)
     (setf kx 1))
    (t
     (setf kx
             (f2cl-lib:int-sub 1
                               (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                 incx)))))
  (cond
    ((> incy 0)
     (setf ky 1))
    (t
     (setf ky
             (f2cl-lib:int-sub 1
                               (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                 incy)))))
  (cond
    ((/= beta one)
     (cond
       ((= incy 1)
        (cond
          ((= beta zero)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                       zero))))
          (t
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                       (* beta
                          (f2cl-lib:fref y-%data%
                                         (i)
                                         ((1 *))
                                         y-%offset%))))))))
       (t
        (setf iy ky)
```

```
      (cond
        ((= beta zero)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     zero)
             (setf iy (f2cl-lib:int-add iy incy)))))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (* beta
                        (f2cl-lib:fref y-%data%
                                       (iy)
                                       ((1 *))
                                       y-%offset%)))
             (setf iy (f2cl-lib:int-add iy incy))))))))))
(if (= alpha zero) (go end_label))
(cond
  ((char-equal uplo #\U)
   (cond
     ((and (= incx 1) (= incy 1))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf temp1
                  (* alpha
                     (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
          (setf temp2 zero)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                      (+
                       (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                       (* temp1
                          (f2cl-lib:fref a-%data%
                                         (i j)
                                         ((1 lda) (1 *))
                                         a-%offset%))))
              (setf temp2
                      (+ temp2
                         (*
                          (f2cl-lib:fref a-%data%
                                         (i j)
                                         ((1 lda) (1 *))
```

```
                                        a-%offset%)
                          (f2cl-lib:fref x-%data%
                                        (i)
                                        ((1 *))
                                        x-%offset%))))))
        (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                   (* temp1
                      (f2cl-lib:fref a-%data%
                                     (j j)
                                     ((1 lda) (1 *))
                                     a-%offset%))
                   (* alpha temp2))))))
    (t
     (setf jx kx)
     (setf jy ky)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp1
                 (* alpha
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
         (setf temp2 zero)
         (setf ix kx)
         (setf iy ky)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i
                           (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                        nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (+
                      (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (* temp1
                         (f2cl-lib:fref a-%data%
                                        (i j)
                                        ((1 lda) (1 *))
                                        a-%offset%))))
             (setf temp2
                     (+ temp2
                        (*
                         (f2cl-lib:fref a-%data%
                                        (i j)
                                        ((1 lda) (1 *))
                                        a-%offset%)
                         (f2cl-lib:fref x-%data%
                                        (ix)
                                        ((1 *))
                                        x-%offset%))))
             (setf ix (f2cl-lib:int-add ix incx))
```

```
            (setf iy (f2cl-lib:int-add iy incy))))
        (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                   (* temp1
                      (f2cl-lib:fref a-%data%
                                     (j j)
                                     ((1 lda) (1 *))
                                     a-%offset%))
                   (* alpha temp2)))
        (setf jx (f2cl-lib:int-add jx incx))
        (setf jy (f2cl-lib:int-add jy incy)))))))
(t
 (cond
   ((and (= incx 1) (= incy 1))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp1
                (* alpha
                   (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
        (setf temp2 zero)
        (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                   (* temp1
                      (f2cl-lib:fref a-%data%
                                     (j j)
                                     ((1 lda) (1 *))
                                     a-%offset%))))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                       (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                    (+
                     (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     (* temp1
                        (f2cl-lib:fref a-%data%
                                       (i j)
                                       ((1 lda) (1 *))
                                       a-%offset%))))
            (setf temp2
                    (+ temp2
                       (*
                        (f2cl-lib:fref a-%data%
                                       (i j)
                                       ((1 lda) (1 *))
                                       a-%offset%)
                        (f2cl-lib:fref x-%data%
                                       (i)
                                       ((1 *))
```

```
                                    x-%offset%))))))
      (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
              (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                 (* alpha temp2))))))
(t
 (setf jx kx)
 (setf jy ky)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf temp1
             (* alpha
                (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
     (setf temp2 zero)
     (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
             (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                (* temp1
                   (f2cl-lib:fref a-%data%
                                  (j j)
                                  ((1 lda) (1 *))
                                  a-%offset%))))
     (setf ix jx)
     (setf iy jy)
     (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                    (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf ix (f2cl-lib:int-add ix incx))
         (setf iy (f2cl-lib:int-add iy incy))
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (+
                  (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (* temp1
                     (f2cl-lib:fref a-%data%
                                    (i j)
                                    ((1 lda) (1 *))
                                    a-%offset%))))
         (setf temp2
                 (+ temp2
                    (*
                     (f2cl-lib:fref a-%data%
                                    (i j)
                                    ((1 lda) (1 *))
                                    a-%offset%)
                     (f2cl-lib:fref x-%data%
                                    (ix)
                                    ((1 *))
                                    x-%offset%))))))
     (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
             (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
```

```
                              (* alpha temp2)))
                    (setf jx (f2cl-lib:int-add jx incx))
                    (setf jy (f2cl-lib:int-add jy incy))))))))
 end_label
        (return (values nil nil nil nil nil nil nil nil nil nil))))))
```

———-

11.9 dsyr - rank 1 $\alpha x x^T + A$

11.9.1 Axiom unit tests

— **dsyr.input** —

```
)set break resume
)sys rm -f dsyr.output
)spool dsyr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

11.9.2 Axiom help page

— **dsyr.help** —

```
==================================================================
dsyr examples
==================================================================

==================================================================
Man Page Details
==================================================================

 NAME
      DSYR - perform the symmetric rank 1 operation   A :=
      alpha*x*x' + A,

 SYNOPSIS
```

```
     SUBROUTINE DSYR ( UPLO, N, ALPHA, X, INCX, A, LDA )

         DOUBLE      PRECISION ALPHA

         INTEGER     INCX, LDA, N

         CHARACTER*1 UPLO

         DOUBLE      PRECISION A( LDA, * ), X( * )

PURPOSE
     DSYR   performs the symmetric rank 1 operation

     where alpha is a real scalar, x is an n element vector and A
     is an n by n symmetric matrix.

PARAMETERS
     UPLO   - CHARACTER*1.
            On entry, UPLO specifies whether the upper or lower
            triangular part of the array A is to be referenced as
            follows:

            UPLO = 'U' or 'u'   Only the upper triangular part of
            A is to be referenced.

            UPLO = 'L' or 'l'   Only the lower triangular part of
            A is to be referenced.

            Unchanged on exit.

     N      - INTEGER.
            On entry, N specifies the order of the matrix A.  N
            must be at least zero.  Unchanged on exit.

     ALPHA  - DOUBLE PRECISION.
            On entry, ALPHA specifies the scalar alpha.
            Unchanged on exit.

     X      - DOUBLE PRECISION array of dimension at least
            ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
            incremented array X must contain the n element vector
            x.  Unchanged on exit.

     INCX   - INTEGER.
            On entry, INCX specifies the increment for the ele-
            ments of X. INCX must not be zero.  Unchanged on

            exit.

     A      - DOUBLE PRECISION array of DIMENSION ( LDA, n ).
```

```
            Before entry with  UPLO = 'U' or 'u', the leading n
            by n upper triangular part of the array A must con-
            tain the upper triangular part of the symmetric
            matrix and the strictly lower triangular part of A is
            not referenced. On exit, the upper triangular part of
            the array A is overwritten by the upper triangular
            part of the updated matrix.  Before entry with UPLO =
            'L' or 'l', the leading n by n lower triangular part
            of the array A must contain the lower triangular part
            of the symmetric matrix and the strictly upper tri-
            angular part of A is not referenced. On exit, the
            lower triangular part of the array A is overwritten
            by the lower triangular part of the updated matrix.

      LDA    - INTEGER.
             On entry, LDA specifies the first dimension of A as
             declared in the calling (sub) program. LDA must be at
             least max( 1, n ).  Unchanged on exit.
```

———-

11.9.3 fortran code

— dsyr.f —

```
*  ==================================================================
      SUBROUTINE DSYR(UPLO,N,ALPHA,X,INCX,A,LDA)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION ALPHA
      INTEGER INCX,LDA,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A(LDA,*),X(*)
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION ZERO
      PARAMETER (ZERO=0.0D+0)
```

```
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP
      INTEGER I,INFO,IX,J,JX,KX
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 7
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('DSYR  ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (ALPHA.EQ.ZERO)) RETURN
*
*     Set the start point in X if the increment is not unity.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the triangular part
*     of A.
*
      IF (LSAME(UPLO,'U')) THEN
```

```
*
*        Form  A  when A is stored in upper triangle.
*
         IF (INCX.EQ.1) THEN
             DO 20 J = 1,N
                 IF (X(J).NE.ZERO) THEN
                     TEMP = ALPHA*X(J)
                     DO 10 I = 1,J
                         A(I,J) = A(I,J) + X(I)*TEMP
   10                CONTINUE
                 END IF
   20        CONTINUE
         ELSE
             JX = KX
             DO 40 J = 1,N
                 IF (X(JX).NE.ZERO) THEN
                     TEMP = ALPHA*X(JX)
                     IX = KX
                     DO 30 I = 1,J
                         A(I,J) = A(I,J) + X(IX)*TEMP
                         IX = IX + INCX
   30                CONTINUE
                 END IF
                 JX = JX + INCX
   40        CONTINUE
         END IF
      ELSE
*
*        Form  A  when A is stored in lower triangle.
*
         IF (INCX.EQ.1) THEN
             DO 60 J = 1,N
                 IF (X(J).NE.ZERO) THEN
                     TEMP = ALPHA*X(J)
                     DO 50 I = J,N
                         A(I,J) = A(I,J) + X(I)*TEMP
   50                CONTINUE
                 END IF
   60        CONTINUE
         ELSE
             JX = KX
             DO 80 J = 1,N
                 IF (X(JX).NE.ZERO) THEN
                     TEMP = ALPHA*X(JX)
                     IX = JX
                     DO 70 I = J,N
                         A(I,J) = A(I,J) + X(IX)*TEMP
                         IX = IX + INCX
   70                CONTINUE
                 END IF
```

```
                JX = JX + INCX
   80         CONTINUE
         END IF
      END IF
*
      RETURN
*
*     End of DSYR  .
*
      END
```

———-

11.9.4 lisp code

— BLAS 2 dsyr —

```
(let* ((zero 0.0))
  (declare (type (double-float 0.0 0.0) zero))
  (defun dsyr (uplo n alpha x incx a lda)
    (declare (type (simple-array double-float (*)) a x)
             (type (double-float) alpha)
             (type fixnum lda incx n)
             (type character uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (x double-float x-%data% x-%offset%)
         (a double-float a-%data% a-%offset%))
      (prog ((i 0) (info 0) (ix 0) (j 0) (jx 0) (kx 0) (temp 0.0))
        (declare (type fixnum i info ix j jx kx)
                 (type (double-float) temp))
        (setf info 0)
        (cond
          ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
           (setf info 1))
          ((< n 0)
           (setf info 2))
          ((= incx 0)
           (setf info 5))
          ((< lda (max (the fixnum 1) (the fixnum n)))
           (setf info 7)))
        (cond
          ((/= info 0)
           (error
            " ** On entry to ~a parameter number ~a had an illegal value~%"
            "DSYR" info)
           (go end_label)))
```

```
(if (or (= n 0) (= alpha zero)) (go end_label))
(cond
  ((<= incx 0)
   (setf kx
           (f2cl-lib:int-sub 1
                              (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                incx))))
  ((/= incx 1)
   (setf kx 1)))
(cond
  ((char-equal uplo #\U)
   (cond
     ((= incx 1)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
            ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
             (setf temp
                     (* alpha
                        (f2cl-lib:fref x-%data%
                                       (j)
                                       ((1 *))
                                       x-%offset%)))
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i j) nil)
               (tagbody
                 (setf (f2cl-lib:fref a-%data%
                                      (i j)
                                      ((1 lda) (1 *))
                                      a-%offset%)
                        (+
                         (f2cl-lib:fref a-%data%
                                        (i j)
                                        ((1 lda) (1 *))
                                        a-%offset%)
                         (*
                          (f2cl-lib:fref x-%data%
                                         (i)
                                         ((1 *))
                                         x-%offset%)
                          temp)))))))))
     (t
      (setf jx kx)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
            ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
             (setf temp
```

```
                    (* alpha
                       (f2cl-lib:fref x-%data%
                                      (jx)
                                      ((1 *))
                                      x-%offset%)))
            (setf ix kx)
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i j) nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data%
                                     (i j)
                                     ((1 lda) (1 *))
                                     a-%offset%)
                      (+
                       (f2cl-lib:fref a-%data%
                                      (i j)
                                      ((1 lda) (1 *))
                                      a-%offset%)
                       (*
                        (f2cl-lib:fref x-%data%
                                       (ix)
                                       ((1 *))
                                       x-%offset%)
                        temp)))
                (setf ix (f2cl-lib:int-add ix incx))))))
        (setf jx (f2cl-lib:int-add jx incx)))))))
(t
 (cond
   ((= incx 1)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
          ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
           (setf temp
                   (* alpha
                      (f2cl-lib:fref x-%data%
                                     (j)
                                     ((1 *))
                                     x-%offset%)))
           (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref a-%data%
                                    (i j)
                                    ((1 lda) (1 *))
                                    a-%offset%)
                     (+
                      (f2cl-lib:fref a-%data%
                                     (i j)
```

```
                                                ((1 lda) (1 *))
                                                a-%offset%)
                                (*
                                 (f2cl-lib:fref x-%data%
                                                 (i)
                                                 ((1 *))
                                                 x-%offset%)
                                 temp)))))))))))
            (t
             (setf jx kx)
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (cond
                   ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
                    (setf temp
                            (* alpha
                               (f2cl-lib:fref x-%data%
                                              (jx)
                                              ((1 *))
                                              x-%offset%)))
                    (setf ix jx)
                    (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                                  ((> i n) nil)
                      (tagbody
                        (setf (f2cl-lib:fref a-%data%
                                             (i j)
                                             ((1 lda) (1 *))
                                             a-%offset%)
                              (+
                               (f2cl-lib:fref a-%data%
                                              (i j)
                                              ((1 lda) (1 *))
                                              a-%offset%)
                               (*
                                (f2cl-lib:fref x-%data%
                                               (ix)
                                               ((1 *))
                                               x-%offset%)
                                temp)))
                        (setf ix (f2cl-lib:int-add ix incx))))))
                 (setf jx (f2cl-lib:int-add jx incx))))))))
end_label
       (return (values nil nil nil nil nil nil nil))))))
```

———-

11.10 dsyr2 - symmetric rank 2 $\alpha xy^T + \alpha yx^T + A$

11.10.1 Axiom unit tests

— dsyr2.input —

```
)set break resume
)sys rm -f dsyr2.output
)spool dsyr2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

11.10.2 Axiom help page

— dsyr2.help —

```
======================================================================
dsyr2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

 NAME
      DSYR2 - perform the symmetric rank 2 operation   A :=
      alpha*x*y' + alpha*y*x' + A,

 SYNOPSIS
      SUBROUTINE DSYR2 ( UPLO, N, ALPHA, X, INCX, Y, INCY, A, LDA
                       )

          DOUBLE       PRECISION ALPHA

          INTEGER      INCX, INCY, LDA, N

          CHARACTER*1  UPLO

          DOUBLE       PRECISION A( LDA, * ), X( * ), Y( * )
```

```
PURPOSE
     DSYR2  performs the symmetric rank 2 operation

     where alpha is a scalar, x and y are n element vectors and A
     is an n by n symmetric matrix.

PARAMETERS
     UPLO   - CHARACTER*1.
            On entry, UPLO specifies whether the upper or lower
            triangular part of the array A is to be referenced as
            follows:

            UPLO = 'U' or 'u'   Only the upper triangular part of
            A is to be referenced.

            UPLO = 'L' or 'l'   Only the lower triangular part of
            A is to be referenced.

            Unchanged on exit.

     N      - INTEGER.
            On entry, N specifies the order of the matrix A.  N
            must be at least zero.  Unchanged on exit.

     ALPHA  - DOUBLE PRECISION.
            On entry, ALPHA specifies the scalar alpha.
            Unchanged on exit.

     X      - DOUBLE PRECISION array of dimension at least
            ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
            incremented array X must contain the n element vector
            x.  Unchanged on exit.

     INCX   - INTEGER.
            On entry, INCX specifies the increment for the

            elements of X. INCX must not be zero.  Unchanged on
            exit.

     Y      - DOUBLE PRECISION array of dimension at least
            ( 1 + ( n - 1 )*abs( INCY ) ).  Before entry, the
            incremented array Y must contain the n element vector
            y.  Unchanged on exit.

     INCY   - INTEGER.
            On entry, INCY specifies the increment for the ele-
            ments of Y. INCY must not be zero.  Unchanged on
            exit.

     A      - DOUBLE PRECISION array of DIMENSION ( LDA, n ).
```

```
            Before entry with  UPLO = 'U' or 'u', the leading n
            by n upper triangular part of the array A must con-
            tain the upper triangular part of the symmetric
            matrix and the strictly lower triangular part of A is
            not referenced. On exit, the upper triangular part of
            the array A is overwritten by the upper triangular
            part of the updated matrix.  Before entry with UPLO =
            'L' or 'l', the leading n by n lower triangular part
            of the array A must contain the lower triangular part
            of the symmetric matrix and the strictly upper tri-
            angular part of A is not referenced. On exit, the
            lower triangular part of the array A is overwritten
            by the lower triangular part of the updated matrix.

      LDA    - INTEGER.
            On entry, LDA specifies the first dimension of A as
            declared in the calling (sub) program. LDA must be at
            least max( 1, n ).  Unchanged on exit.
```

———-

11.10.3 fortran code

— dsyr2.f —

```
*  =====================================================================
      SUBROUTINE DSYR2(UPLO,N,ALPHA,X,INCX,Y,INCY,A,LDA)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION ALPHA
      INTEGER INCX,INCY,LDA,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A(LDA,*),X(*),Y(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION ZERO
      PARAMETER (ZERO=0.0D+0)
```

```
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,KX,KY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      ELSE IF (INCY.EQ.0) THEN
          INFO = 7
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 9
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('DSYR2 ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (ALPHA.EQ.ZERO)) RETURN
*
*     Set up the start points in X and Y if the increments are not both
*     unity.
*
      IF ((INCX.NE.1) .OR. (INCY.NE.1)) THEN
          IF (INCX.GT.0) THEN
              KX = 1
          ELSE
              KX = 1 - (N-1)*INCX
          END IF
          IF (INCY.GT.0) THEN
              KY = 1
```

```
         ELSE
             KY = 1 - (N-1)*INCY
         END IF
         JX = KX
         JY = KY
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the triangular part
*     of A.
*
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  A  when A is stored in the upper triangle.
*
         IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
             DO 20 J = 1,N
                 IF ((X(J).NE.ZERO) .OR. (Y(J).NE.ZERO)) THEN
                     TEMP1 = ALPHA*Y(J)
                     TEMP2 = ALPHA*X(J)
                     DO 10 I = 1,J
                         A(I,J) = A(I,J) + X(I)*TEMP1 + Y(I)*TEMP2
   10                CONTINUE
                 END IF
   20        CONTINUE
         ELSE
             DO 40 J = 1,N
                 IF ((X(JX).NE.ZERO) .OR. (Y(JY).NE.ZERO)) THEN
                     TEMP1 = ALPHA*Y(JY)
                     TEMP2 = ALPHA*X(JX)
                     IX = KX
                     IY = KY
                     DO 30 I = 1,J
                         A(I,J) = A(I,J) + X(IX)*TEMP1 + Y(IY)*TEMP2
                         IX = IX + INCX
                         IY = IY + INCY
   30                CONTINUE
                 END IF
                 JX = JX + INCX
                 JY = JY + INCY
   40        CONTINUE
         END IF
      ELSE
*
*        Form  A  when A is stored in the lower triangle.
*
         IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
             DO 60 J = 1,N
                 IF ((X(J).NE.ZERO) .OR. (Y(J).NE.ZERO)) THEN
                     TEMP1 = ALPHA*Y(J)
```

```
                    TEMP2 = ALPHA*X(J)
                    DO 50 I = J,N
                        A(I,J) = A(I,J) + X(I)*TEMP1 + Y(I)*TEMP2
   50               CONTINUE
                END IF
   60         CONTINUE
        ELSE
            DO 80 J = 1,N
                IF ((X(JX).NE.ZERO) .OR. (Y(JY).NE.ZERO)) THEN
                    TEMP1 = ALPHA*Y(JY)
                    TEMP2 = ALPHA*X(JX)
                    IX = JX
                    IY = JY
                    DO 70 I = J,N
                        A(I,J) = A(I,J) + X(IX)*TEMP1 + Y(IY)*TEMP2
                        IX = IX + INCX
                        IY = IY + INCY
   70               CONTINUE
                END IF
                JX = JX + INCX
                JY = JY + INCY
   80         CONTINUE
        END IF
      END IF
*
      RETURN
*
*     End of DSYR2 .
*
      END
```

———-

11.10.4 lisp code

— BLAS 2 dsyr2 —

```
(let* ((zero 0.0))
  (declare (type (double-float 0.0 0.0) zero))
  (defun dsyr2 (uplo n alpha x incx y incy a lda)
    (declare (type (simple-array double-float (*)) a y x)
             (type (double-float) alpha)
             (type fixnum lda incy incx n)
             (type character uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (x double-float x-%data% x-%offset%)
```

```
    (y double-float y-%data% y-%offset%)
    (a double-float a-%data% a-%offset%))
  (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (kx 0) (ky 0)
         (temp1 0.0) (temp2 0.0))
    (declare (type fixnum i info ix iy j jx jy kx ky)
             (type (double-float) temp1 temp2))
    (setf info 0)
    (cond
      ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
       (setf info 1))
      ((< n 0)
       (setf info 2))
      ((= incx 0)
       (setf info 5))
      ((= incy 0)
       (setf info 7))
      ((< lda (max (the fixnum 1) (the fixnum n)))
       (setf info 9)))
    (cond
      ((/= info 0)
       (error
        " ** On entry to ~a parameter number ~a had an illegal value~%"
        "DSYR2" info)
       (go end_label)))
    (if (or (= n 0) (= alpha zero)) (go end_label))
    (cond
      ((or (/= incx 1) (/= incy 1))
       (cond
         ((> incx 0)
          (setf kx 1))
         (t
          (setf kx
                  (f2cl-lib:int-sub 1
                                    (f2cl-lib:int-mul
                                     (f2cl-lib:int-sub n 1)
                                     incx)))))
       (cond
         ((> incy 0)
          (setf ky 1))
         (t
          (setf ky
                  (f2cl-lib:int-sub 1
                                    (f2cl-lib:int-mul
                                     (f2cl-lib:int-sub n 1)
                                     incy)))))
       (setf jx kx)
       (setf jy ky)))
    (cond
      ((char-equal uplo #\U)
       (cond
```

```
((and (= incx 1) (= incy 1))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((or (/= (f2cl-lib:fref x (j) ((1 *))) zero)
            (/= (f2cl-lib:fref y (j) ((1 *))) zero))
        (setf temp1
                (* alpha
                   (f2cl-lib:fref y-%data%
                                  (j)
                                  ((1 *))
                                  y-%offset%)))
        (setf temp2
                (* alpha
                   (f2cl-lib:fref x-%data%
                                  (j)
                                  ((1 *))
                                  x-%offset%)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i j) nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data%
                                 (i j)
                                 ((1 lda) (1 *))
                                 a-%offset%)
                  (+
                   (f2cl-lib:fref a-%data%
                                  (i j)
                                  ((1 lda) (1 *))
                                  a-%offset%)
                   (*
                    (f2cl-lib:fref x-%data%
                                   (i)
                                   ((1 *))
                                   x-%offset%)
                    temp1)
                   (*
                    (f2cl-lib:fref y-%data%
                                   (i)
                                   ((1 *))
                                   y-%offset%)
                    temp2))))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((or (/= (f2cl-lib:fref x (jx) ((1 *))) zero)
            (/= (f2cl-lib:fref y (jy) ((1 *))) zero))
```

```
                (setf temp1
                        (* alpha
                           (f2cl-lib:fref y-%data%
                                          (jy)
                                          ((1 *))
                                          y-%offset%)))
                (setf temp2
                        (* alpha
                           (f2cl-lib:fref x-%data%
                                          (jx)
                                          ((1 *))
                                          x-%offset%)))
                (setf ix kx)
                (setf iy ky)
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i j) nil)
                  (tagbody
                    (setf (f2cl-lib:fref a-%data%
                                         (i j)
                                         ((1 lda) (1 *))
                                         a-%offset%)
                          (+
                           (f2cl-lib:fref a-%data%
                                          (i j)
                                          ((1 lda) (1 *))
                                          a-%offset%)
                           (*
                            (f2cl-lib:fref x-%data%
                                           (ix)
                                           ((1 *))
                                           x-%offset%)
                            temp1)
                           (*
                            (f2cl-lib:fref y-%data%
                                           (iy)
                                           ((1 *))
                                           y-%offset%)
                            temp2)))
                    (setf ix (f2cl-lib:int-add ix incx))
                    (setf iy (f2cl-lib:int-add iy incy))))))
            (setf jx (f2cl-lib:int-add jx incx))
            (setf jy (f2cl-lib:int-add jy incy)))))))
    (t
     (cond
       ((and (= incx 1) (= incy 1))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (cond
              ((or (/= (f2cl-lib:fref x (j) ((1 *))) zero)
```

```
                (/= (f2cl-lib:fref y (j) ((1 *))) zero))
            (setf temp1
                    (* alpha
                       (f2cl-lib:fref y-%data%
                                      (j)
                                      ((1 *))
                                      y-%offset%)))
            (setf temp2
                    (* alpha
                       (f2cl-lib:fref x-%data%
                                      (j)
                                      ((1 *))
                                      x-%offset%)))
            (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data%
                                     (i j)
                                     ((1 lda) (1 *))
                                     a-%offset%)
                      (+
                       (f2cl-lib:fref a-%data%
                                      (i j)
                                      ((1 lda) (1 *))
                                      a-%offset%)
                       (*
                        (f2cl-lib:fref x-%data%
                                       (i)
                                       ((1 *))
                                       x-%offset%)
                        temp1)
                       (*
                        (f2cl-lib:fref y-%data%
                                       (i)
                                       ((1 *))
                                       y-%offset%)
                        temp2)))))))))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (cond
           ((or (/= (f2cl-lib:fref x (jx) ((1 *))) zero)
                (/= (f2cl-lib:fref y (jy) ((1 *))) zero))
            (setf temp1
                    (* alpha
                       (f2cl-lib:fref y-%data%
                                      (jy)
                                      ((1 *))
                                      y-%offset%)))
```

```
                    (setf temp2
                            (* alpha
                               (f2cl-lib:fref x-%data%
                                              (jx)
                                              ((1 *))
                                              x-%offset%)))
                    (setf ix jx)
                    (setf iy jy)
                    (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                                  ((> i n) nil)
                      (tagbody
                        (setf (f2cl-lib:fref a-%data%
                                             (i j)
                                             ((1 lda) (1 *))
                                             a-%offset%)
                              (+
                               (f2cl-lib:fref a-%data%
                                              (i j)
                                              ((1 lda) (1 *))
                                              a-%offset%)
                               (*
                                (f2cl-lib:fref x-%data%
                                               (ix)
                                               ((1 *))
                                               x-%offset%)
                                temp1)
                               (*
                                (f2cl-lib:fref y-%data%
                                               (iy)
                                               ((1 *))
                                               y-%offset%)
                                temp2)))
                        (setf ix (f2cl-lib:int-add ix incx))
                        (setf iy (f2cl-lib:int-add iy incy))))))
                  (setf jx (f2cl-lib:int-add jx incx))
                  (setf jy (f2cl-lib:int-add jy incy))))))))
end_label
       (return (values nil nil nil nil nil nil nil nil nil))))))
```

———-

11.11 dtbmv - triangular banded matrix $x = Ax$ or $x = A^T x$

11.11.1 Axiom unit tests

— dtbmv.input —

```
)set break resume
)sys rm -f dtbmv.output
)spool dtbmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

11.11.2 Axiom help page

— dtbmv.help —

```
======================================================================
dtbmv examples
======================================================================

======================================================================
Man Page Details
======================================================================

 NAME
      DTBMV - perform one of the matrix-vector operations   x :=
      A*x, or x := A'*x,

 SYNOPSIS
      SUBROUTINE DTBMV ( UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX
                       )

          INTEGER      INCX, K, LDA, N

          CHARACTER*1  DIAG, TRANS, UPLO

          DOUBLE       PRECISION A( LDA, * ), X( * )
```

```
PURPOSE
     DTBMV  performs one of the matrix-vector operations

     where x is an n element vector and  A is an n by n unit, or
     non-unit, upper or lower triangular band matrix, with ( k +
     1 ) diagonals.

PARAMETERS
     UPLO   - CHARACTER*1.
            On entry, UPLO specifies whether the matrix is an
            upper or lower triangular matrix as follows:

            UPLO = 'U' or 'u'   A is an upper triangular matrix.

            UPLO = 'L' or 'l'   A is a lower triangular matrix.

            Unchanged on exit.

     TRANS  - CHARACTER*1.
            On entry, TRANS specifies the operation to be per-
            formed as follows:

            TRANS = 'N' or 'n'   x := A*x.

            TRANS = 'T' or 't'   x := A'*x.

            TRANS = 'C' or 'c'   x := A'*x.

            Unchanged on exit.

     DIAG   - CHARACTER*1.
            On entry, DIAG specifies whether or not A is unit
            triangular as follows:

            DIAG = 'U' or 'u'   A is assumed to be unit triangu-
            lar.

            DIAG = 'N' or 'n'   A is not assumed to be unit tri-
            angular.

            Unchanged on exit.

     N      - INTEGER.
            On entry, N specifies the order of the matrix A.  N
            must be at least zero.  Unchanged on exit.

     K      - INTEGER.
            On entry with UPLO = 'U' or 'u', K specifies the
            number of super-diagonals of the matrix A.  On entry
            with UPLO = 'L' or 'l', K specifies the number of
```

```
        sub-diagonals of the matrix A.  K must satisfy  0
        .le. K.  Unchanged on exit.

A       - DOUBLE PRECISION array of DIMENSION ( LDA, n ).
        Before entry with UPLO = 'U' or 'u', the leading ( k
        + 1 ) by n part of the array A must contain the upper
        triangular band part of the matrix of coefficients,
        supplied column by column, with the leading diagonal
        of the matrix in row ( k + 1 ) of the array, the
        first super-diagonal starting at position 2 in row k,
        and so on. The top left k by k triangle of the array
        A is not referenced.  The following program segment
        will transfer an upper triangular band matrix from
        conventional full matrix storage to band storage:

        DO 20, J = 1, N M = K + 1 - J DO 10, I = MAX( 1, J -
        K ), J A( M + I, J ) = matrix( I, J ) 10    CONTINUE
        20 CONTINUE

        Before entry with UPLO = 'L' or 'l', the leading ( k
        + 1 ) by n part of the array A must contain the lower
        triangular band part of the matrix of coefficients,
        supplied column by column, with the leading diagonal
        of the matrix in row 1 of the array, the first sub-
        diagonal starting at position 1 in row 2, and so on.
        The bottom right k by k triangle of the array A is
        not referenced.  The following program segment will
        transfer a lower triangular band matrix from conven-
        tional full matrix storage to band storage:

        DO 20, J = 1, N M = 1 - J DO 10, I = J, MIN( N, J + K
        ) A( M + I, J ) = matrix( I, J ) 10    CONTINUE 20
        CONTINUE

        Note that when DIAG = 'U' or 'u' the elements of the
        array A corresponding to the diagonal elements of the
        matrix are not referenced, but are assumed to be
        unity.  Unchanged on exit.

LDA     - INTEGER.
        On entry, LDA specifies the first dimension of A as
        declared in the calling (sub) program. LDA must be at
        least ( k + 1 ).  Unchanged on exit.

X       - DOUBLE PRECISION array of dimension at least
        ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
        incremented array X must contain the n element vector
        x. On exit, X is overwritten with the transformed
        vector x.
```

```
      INCX   - INTEGER.
             On entry, INCX specifies the increment for the ele-
             ments of X. INCX must not be zero.  Unchanged on
             exit.
```

———-

11.11.3 fortran code

— **dtbmv.f** —

```
*  ==================================================================
      SUBROUTINE DTBMV(UPLO,TRANS,DIAG,N,K,A,LDA,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,K,LDA,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A(LDA,*),X(*)
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION ZERO
      PARAMETER (ZERO=0.0D+0)
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP
      INTEGER I,INFO,IX,J,JX,KPLUS1,KX,L
      LOGICAL NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX,MIN
```

```
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (K.LT.0) THEN
          INFO = 5
      ELSE IF (LDA.LT. (K+1)) THEN
          INFO = 7
      ELSE IF (INCX.EQ.0) THEN
          INFO = 9
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('DTBMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX   too small for descending loops.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
      IF (LSAME(TRANS,'N')) THEN
*
*         Form  x := A*x.
*
          IF (LSAME(UPLO,'U')) THEN
              KPLUS1 = K + 1
```

```
            IF (INCX.EQ.1) THEN
                DO 20 J = 1,N
                    IF (X(J).NE.ZERO) THEN
                        TEMP = X(J)
                        L = KPLUS1 - J
                        DO 10 I = MAX(1,J-K),J - 1
                            X(I) = X(I) + TEMP*A(L+I,J)
 10                     CONTINUE
                        IF (NOUNIT) X(J) = X(J)*A(KPLUS1,J)
                    END IF
 20             CONTINUE
            ELSE
                JX = KX
                DO 40 J = 1,N
                    IF (X(JX).NE.ZERO) THEN
                        TEMP = X(JX)
                        IX = KX
                        L = KPLUS1 - J
                        DO 30 I = MAX(1,J-K),J - 1
                            X(IX) = X(IX) + TEMP*A(L+I,J)
                            IX = IX + INCX
 30                     CONTINUE
                        IF (NOUNIT) X(JX) = X(JX)*A(KPLUS1,J)
                    END IF
                    JX = JX + INCX
                    IF (J.GT.K) KX = KX + INCX
 40             CONTINUE
            END IF
        ELSE
            IF (INCX.EQ.1) THEN
                DO 60 J = N,1,-1
                    IF (X(J).NE.ZERO) THEN
                        TEMP = X(J)
                        L = 1 - J
                        DO 50 I = MIN(N,J+K),J + 1,-1
                            X(I) = X(I) + TEMP*A(L+I,J)
 50                     CONTINUE
                        IF (NOUNIT) X(J) = X(J)*A(1,J)
                    END IF
 60             CONTINUE
            ELSE
                KX = KX + (N-1)*INCX
                JX = KX
                DO 80 J = N,1,-1
                    IF (X(JX).NE.ZERO) THEN
                        TEMP = X(JX)
                        IX = KX
                        L = 1 - J
                        DO 70 I = MIN(N,J+K),J + 1,-1
                            X(IX) = X(IX) + TEMP*A(L+I,J)
```

```
                            IX = IX - INCX
   70                    CONTINUE
                         IF (NOUNIT) X(JX) = X(JX)*A(1,J)
                     END IF
                     JX = JX - INCX
                     IF ((N-J).GE.K) KX = KX - INCX
   80            CONTINUE
             END IF
         END IF
      ELSE
*
*        Form  x := A**T*x.
*
         IF (LSAME(UPLO,'U')) THEN
             KPLUS1 = K + 1
             IF (INCX.EQ.1) THEN
                 DO 100 J = N,1,-1
                     TEMP = X(J)
                     L = KPLUS1 - J
                     IF (NOUNIT) TEMP = TEMP*A(KPLUS1,J)
                     DO 90 I = J - 1,MAX(1,J-K),-1
                         TEMP = TEMP + A(L+I,J)*X(I)
   90                CONTINUE
                     X(J) = TEMP
  100            CONTINUE
             ELSE
                 KX = KX + (N-1)*INCX
                 JX = KX
                 DO 120 J = N,1,-1
                     TEMP = X(JX)
                     KX = KX - INCX
                     IX = KX
                     L = KPLUS1 - J
                     IF (NOUNIT) TEMP = TEMP*A(KPLUS1,J)
                     DO 110 I = J - 1,MAX(1,J-K),-1
                         TEMP = TEMP + A(L+I,J)*X(IX)
                         IX = IX - INCX
  110                CONTINUE
                     X(JX) = TEMP
                     JX = JX - INCX
  120            CONTINUE
             END IF
         ELSE
             IF (INCX.EQ.1) THEN
                 DO 140 J = 1,N
                     TEMP = X(J)
                     L = 1 - J
                     IF (NOUNIT) TEMP = TEMP*A(1,J)
                     DO 130 I = J + 1,MIN(N,J+K)
                         TEMP = TEMP + A(L+I,J)*X(I)
```

```
  130                 CONTINUE
                      X(J) = TEMP
  140             CONTINUE
              ELSE
                  JX = KX
                  DO 160 J = 1,N
                      TEMP = X(JX)
                      KX = KX + INCX
                      IX = KX
                      L = 1 - J
                      IF (NOUNIT) TEMP = TEMP*A(1,J)
                      DO 150 I = J + 1,MIN(N,J+K)
                          TEMP = TEMP + A(L+I,J)*X(IX)
                          IX = IX + INCX
  150                 CONTINUE
                      X(JX) = TEMP
                      JX = JX + INCX
  160             CONTINUE
              END IF
          END IF
      END IF
*
      RETURN
*
*     End of DTBMV .
*
      END
```

———-

11.11.4 lisp code

— BLAS 2 dtbmv —

```
(let* ((zero 0.0))
  (declare (type (double-float 0.0 0.0) zero))
  (defun dtbmv (uplo trans diag n k a lda x incx)
    (declare (type (simple-array double-float (*)) x a)
             (type fixnum incx lda k n)
             (type character diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a double-float a-%data% a-%offset%)
         (x double-float x-%data% x-%offset%))
      (prog ((nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0) (kplus1 0) (kx 0)
```

```
     (l 0) (temp 0.0))
(declare (type (member t nil) nounit)
         (type fixnum i info ix j jx kplus1 kx l)
         (type (double-float) temp))
(setf info 0)
(cond
  ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
   (setf info 1))
  ((and (not (char-equal trans #\N))
        (not (char-equal trans #\T))
        (not (char-equal trans #\C)))
   (setf info 2))
  ((and (not (char-equal diag #\U)) (not (char-equal diag #\N)))
   (setf info 3))
  ((< n 0)
   (setf info 4))
  ((< k 0)
   (setf info 5))
  ((< lda (f2cl-lib:int-add k 1))
   (setf info 7))
  ((= incx 0)
   (setf info 9)))
(cond
  ((/= info 0)
   (error
    " ** On entry to ~a parameter number ~a had an illegal value~%"
    "DTBMV" info)
   (go end_label)))
(if (= n 0) (go end_label))
(setf nounit (char-equal diag #\N))
(cond
  ((<= incx 0)
   (setf kx
           (f2cl-lib:int-sub 1
                             (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                               incx))))
  ((/= incx 1)
   (setf kx 1)))
(cond
  ((char-equal trans #\N)
   (cond
     ((char-equal uplo #\U)
      (setf kplus1 (f2cl-lib:int-add k 1))
      (cond
        ((= incx 1)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (cond
               ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
```

```
                (setf temp
                        (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                (setf l (f2cl-lib:int-sub kplus1 j))
                (f2cl-lib:fdo (i
                               (max (the fixnum 1)
                                    (the fixnum
                                         (f2cl-lib:int-add j
                                                           (f2cl-lib:int-sub
                                                            k))))
                               (f2cl-lib:int-add i 1))
                              ((> i
                                  (f2cl-lib:int-add j
                                                    (f2cl-lib:int-sub
                                                     1)))
                               nil)
                  (tagbody
                    (setf (f2cl-lib:fref x-%data%
                                         (i)
                                         ((1 *))
                                         x-%offset%)
                          (+
                           (f2cl-lib:fref x-%data%
                                          (i)
                                          ((1 *))
                                          x-%offset%)
                           (* temp
                              (f2cl-lib:fref a-%data%
                                             ((f2cl-lib:int-add l i)
                                              j)
                                             ((1 lda) (1 *))
                                             a-%offset%))))))
                (if nounit
                    (setf (f2cl-lib:fref x-%data%
                                         (j)
                                         ((1 *))
                                         x-%offset%)
                          (*
                           (f2cl-lib:fref x-%data%
                                          (j)
                                          ((1 *))
                                          x-%offset%)
                           (f2cl-lib:fref a-%data%
                                          (kplus1 j)
                                          ((1 lda) (1 *))
                                          a-%offset%)))))))))
        (t
         (setf jx kx)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
```

```
(cond
  ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
   (setf temp
           (f2cl-lib:fref x-%data%
                          (jx)
                          ((1 *))
                          x-%offset%))
   (setf ix kx)
   (setf l (f2cl-lib:int-sub kplus1 j))
   (f2cl-lib:fdo (i
                  (max (the fixnum 1)
                       (the fixnum
                            (f2cl-lib:int-add j
                                              (f2cl-lib:int-sub
                                               k))))
                  (f2cl-lib:int-add i 1))
                 ((> i
                     (f2cl-lib:int-add j
                                       (f2cl-lib:int-sub
                                        1)))
                  nil)
     (tagbody
       (setf (f2cl-lib:fref x-%data%
                            (ix)
                            ((1 *))
                            x-%offset%)
             (+
              (f2cl-lib:fref x-%data%
                             (ix)
                             ((1 *))
                             x-%offset%)
              (* temp
                 (f2cl-lib:fref a-%data%
                                ((f2cl-lib:int-add l i)
                                 j)
                                ((1 lda) (1 *))
                                a-%offset%))))
       (setf ix (f2cl-lib:int-add ix incx))))
   (if nounit
       (setf (f2cl-lib:fref x-%data%
                            (jx)
                            ((1 *))
                            x-%offset%)
             (*
              (f2cl-lib:fref x-%data%
                             (jx)
                             ((1 *))
                             x-%offset%)
              (f2cl-lib:fref a-%data%
                             (kplus1 j)
```

```
                                        ((1 lda) (1 *))
                                        a-%offset%)))))))
            (setf jx (f2cl-lib:int-add jx incx))
            (if (> j k) (setf kx (f2cl-lib:int-add kx incx))))))))
    (t
     (cond
       ((= incx 1)
        (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                      ((> j 1) nil)
          (tagbody
            (cond
              ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
               (setf temp
                       (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
               (setf l (f2cl-lib:int-sub 1 j))
               (f2cl-lib:fdo (i
                              (min (the fixnum n)
                                   (the fixnum
                                        (f2cl-lib:int-add j k)))
                              (f2cl-lib:int-add i
                                                (f2cl-lib:int-sub 1)))
                             ((> i (f2cl-lib:int-add j 1)) nil)
                 (tagbody
                   (setf (f2cl-lib:fref x-%data%
                                        (i)
                                        ((1 *))
                                        x-%offset%)
                         (+
                          (f2cl-lib:fref x-%data%
                                         (i)
                                         ((1 *))
                                         x-%offset%)
                          (* temp
                             (f2cl-lib:fref a-%data%
                                            ((f2cl-lib:int-add l i)
                                             j)
                                            ((1 lda) (1 *))
                                            a-%offset%))))))
               (if nounit
                   (setf (f2cl-lib:fref x-%data%
                                        (j)
                                        ((1 *))
                                        x-%offset%)
                         (*
                          (f2cl-lib:fref x-%data%
                                         (j)
                                         ((1 *))
                                         x-%offset%)
                          (f2cl-lib:fref a-%data%
                                         (1 j)
```

```
                                        ((1 lda) (1 *))
                                        a-%offset%)))))))))
      (t
       (setf kx
               (f2cl-lib:int-add kx
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1)
                                  incx)))
       (setf jx kx)
       (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     ((> j 1) nil)
         (tagbody
           (cond
             ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
              (setf temp
                      (f2cl-lib:fref x-%data%
                                     (jx)
                                     ((1 *))
                                     x-%offset%))
              (setf ix kx)
              (setf l (f2cl-lib:int-sub 1 j))
              (f2cl-lib:fdo (i
                             (min (the fixnum n)
                                  (the fixnum
                                       (f2cl-lib:int-add j k)))
                             (f2cl-lib:int-add i
                                               (f2cl-lib:int-sub 1)))
                            ((> i (f2cl-lib:int-add j 1)) nil)
                (tagbody
                  (setf (f2cl-lib:fref x-%data%
                                       (ix)
                                       ((1 *))
                                       x-%offset%)
                        (+
                         (f2cl-lib:fref x-%data%
                                        (ix)
                                        ((1 *))
                                        x-%offset%)
                         (* temp
                            (f2cl-lib:fref a-%data%
                                           ((f2cl-lib:int-add l i)
                                            j)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
                  (setf ix (f2cl-lib:int-sub ix incx))))
              (if nounit
                  (setf (f2cl-lib:fref x-%data%
                                       (jx)
                                       ((1 *))
                                       x-%offset%)
```

```
                              (*
                               (f2cl-lib:fref x-%data%
                                              (jx)
                                              ((1 *))
                                              x-%offset%)
                               (f2cl-lib:fref a-%data%
                                              (1 j)
                                              ((1 lda) (1 *))
                                              a-%offset%)))))))
                (setf jx (f2cl-lib:int-sub jx incx))
                (if (>= (f2cl-lib:int-sub n j) k)
                    (setf kx (f2cl-lib:int-sub kx incx)))))))))
            (t
             (cond
               ((char-equal uplo #\U)
                (setf kplus1 (f2cl-lib:int-add k 1))
                (cond
                  ((= incx 1)
                   (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                                 ((> j 1) nil)
                     (tagbody
                       (setf temp
                               (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                       (setf l (f2cl-lib:int-sub kplus1 j))
                       (if nounit
                           (setf temp
                                   (* temp
                                      (f2cl-lib:fref a-%data%
                                                     (kplus1 j)
                                                     ((1 lda) (1 *))
                                                     a-%offset%))))
                       (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                                      (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                                     ((> i
                                         (max (the fixnum 1)
                                              (the fixnum
                                                   (f2cl-lib:int-add j
                                                                     (f2cl-lib:int-sub
                                                                      k)))))
                                      nil)
                         (tagbody
                           (setf temp
                                   (+ temp
                                      (*
                                       (f2cl-lib:fref a-%data%
                                                      ((f2cl-lib:int-add l i) j)
                                                      ((1 lda) (1 *))
                                                      a-%offset%)
                                       (f2cl-lib:fref x-%data%
                                                      (i)
```

```
                                        ((1 *))
                                        x-%offset%))))))
            (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                    temp))))
        (t
         (setf kx
                 (f2cl-lib:int-add kx
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-sub n 1)
                                    incx)))
         (setf jx kx)
         (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((> j 1) nil)
           (tagbody
             (setf temp
                     (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
             (setf kx (f2cl-lib:int-sub kx incx))
             (setf ix kx)
             (setf l (f2cl-lib:int-sub kplus1 j))
             (if nounit
                 (setf temp
                         (* temp
                            (f2cl-lib:fref a-%data%
                                           (kplus1 j)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
             (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                            (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                           ((> i
                               (max (the fixnum 1)
                                    (the fixnum
                                         (f2cl-lib:int-add j
                                                           (f2cl-lib:int-sub
                                                            k)))))
                            nil)
               (tagbody
                 (setf temp
                         (+ temp
                            (*
                             (f2cl-lib:fref a-%data%
                                            ((f2cl-lib:int-add l i) j)
                                            ((1 lda) (1 *))
                                            a-%offset%)
                             (f2cl-lib:fref x-%data%
                                            (ix)
                                            ((1 *))
                                            x-%offset%))))
                 (setf ix (f2cl-lib:int-sub ix incx))))
             (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                     temp)
```

```
        (setf jx (f2cl-lib:int-sub jx incx)))))))
(t
 (cond
   ((= incx 1)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp
                (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
        (setf l (f2cl-lib:int-sub 1 j))
        (if nounit
            (setf temp
                    (* temp
                       (f2cl-lib:fref a-%data%
                                      (1 j)
                                      ((1 lda) (1 *))
                                      a-%offset%))))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                       (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the fixnum n)
                               (the fixnum
                                    (f2cl-lib:int-add j k))))
                       nil)
          (tagbody
            (setf temp
                    (+ temp
                       (*
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add l i) j)
                                       ((1 lda) (1 *))
                                       a-%offset%)
                        (f2cl-lib:fref x-%data%
                                       (i)
                                       ((1 *))
                                       x-%offset%))))))
        (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                temp))))
   (t
    (setf jx kx)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp
                (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
        (setf kx (f2cl-lib:int-add kx incx))
        (setf ix kx)
        (setf l (f2cl-lib:int-sub 1 j))
        (if nounit
            (setf temp
```

```
                                (* temp
                                   (f2cl-lib:fref a-%data%
                                                  (1 j)
                                                  ((1 lda) (1 *))
                                                  a-%offset%))))
                     (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                                    (f2cl-lib:int-add i 1))
                                   ((> i
                                       (min (the fixnum n)
                                            (the fixnum
                                                 (f2cl-lib:int-add j k))))
                                    nil)
                       (tagbody
                         (setf temp
                                 (+ temp
                                    (*
                                     (f2cl-lib:fref a-%data%
                                                    ((f2cl-lib:int-add l i) j)
                                                    ((1 lda) (1 *))
                                                    a-%offset%)
                                     (f2cl-lib:fref x-%data%
                                                    (ix)
                                                    ((1 *))
                                                    x-%offset%))))
                         (setf ix (f2cl-lib:int-add ix incx))))
                     (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                             temp)
                     (setf jx (f2cl-lib:int-add jx incx)))))))))
 end_label
        (return (values nil nil nil nil nil nil nil nil nil))))))
```

———-

11.12 dtbsv - triangular banded matrix $Ax = b$ or $A^T x = b$

11.12.1 Axiom unit tests

— dtbsv.input —

```
)set break resume
)sys rm -f dtbsv.output
)spool dtbsv.output
)set message test on
)set message auto off
```

```
)clear all

)spool
)lisp (bye)
```

———-

11.12.2 Axiom help page

— **dtbsv.help** —

```
======================================================================
dtbsv examples
======================================================================

======================================================================
Man Page Details
======================================================================

 NAME
      DTBSV - solve one of the systems of equations   A*x = b, or
      A'*x = b,

 SYNOPSIS
      SUBROUTINE DTBSV ( UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX
                       )

          INTEGER      INCX, K, LDA, N

          CHARACTER*1  DIAG, TRANS, UPLO

          DOUBLE       PRECISION A( LDA, * ), X( * )

 PURPOSE
      DTBSV  solves one of the systems of equations

      where b and x are n element vectors and A is an n by n unit,
      or non-unit, upper or lower triangular band matrix, with ( k
      + 1 ) diagonals.

      No test for singularity or near-singularity is included in
      this routine. Such tests must be performed before calling
      this routine.

 PARAMETERS
      UPLO   - CHARACTER*1.
               On entry, UPLO specifies whether the matrix is an
               upper or lower triangular matrix as follows:
```

```
         UPLO = 'U' or 'u'   A is an upper triangular matrix.

         UPLO = 'L' or 'l'   A is a lower triangular matrix.

         Unchanged on exit.

TRANS  - CHARACTER*1.
         On entry, TRANS specifies the equations to be solved
         as follows:

         TRANS = 'N' or 'n'   A*x = b.

         TRANS = 'T' or 't'   A'*x = b.

         TRANS = 'C' or 'c'   A'*x = b.

         Unchanged on exit.

DIAG   - CHARACTER*1.
         On entry, DIAG specifies whether or not A is unit
         triangular as follows:

         DIAG = 'U' or 'u'   A is assumed to be unit triangu-
         lar.

         DIAG = 'N' or 'n'   A is not assumed to be unit tri-
         angular.

         Unchanged on exit.

N      - INTEGER.
         On entry, N specifies the order of the matrix A.  N
         must be at least zero.  Unchanged on exit.

K      - INTEGER.
         On entry with UPLO = 'U' or 'u', K specifies the
         number of super-diagonals of the matrix A.  On entry
         with UPLO = 'L' or 'l', K specifies the number of
         sub-diagonals of the matrix A.  K must satisfy  0
         .le. K.  Unchanged on exit.

A      - DOUBLE PRECISION array of DIMENSION ( LDA, n ).
         Before entry with UPLO = 'U' or 'u', the leading ( k
         + 1 ) by n part of the array A must contain the upper
         triangular band part of the matrix of coefficients,
         supplied column by column, with the leading diagonal
         of the matrix in row ( k + 1 ) of the array, the
         first super-diagonal starting at position 2 in row k,
         and so on. The top left k by k triangle of the array
```

```
A is not referenced.  The following program segment
will transfer an upper triangular band matrix from
conventional full matrix storage to band storage:

DO 20, J = 1, N M = K + 1 - J DO 10, I = MAX( 1, J -
K ), J A( M + I, J ) = matrix( I, J ) 10    CONTINUE
20 CONTINUE

Before entry with UPLO = 'L' or 'l', the leading ( k
+ 1 ) by n part of the array A must contain the lower
triangular band part of the matrix of coefficients,
supplied column by column, with the leading diagonal
of the matrix in row 1 of the array, the first sub-
diagonal starting at position 1 in row 2, and so on.
The bottom right k by k triangle of the array A is
not referenced.  The following program segment will
transfer a lower triangular band matrix from conven-
tional full matrix storage to band storage:

DO 20, J = 1, N M = 1 - J DO 10, I = J, MIN( N, J + K
) A( M + I, J ) = matrix( I, J ) 10    CONTINUE 20
CONTINUE

Note that when DIAG = 'U' or 'u' the elements of the
array A corresponding to the diagonal elements of the

matrix are not referenced, but are assumed to be
unity.  Unchanged on exit.

LDA     - INTEGER.
        On entry, LDA specifies the first dimension of A as
        declared in the calling (sub) program. LDA must be at
        least ( k + 1 ).  Unchanged on exit.

X       - DOUBLE PRECISION array of dimension at least
        ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
        incremented array X must contain the n element
        right-hand side vector b. On exit, X is overwritten
        with the solution vector x.

INCX    - INTEGER.
        On entry, INCX specifies the increment for the ele-
        ments of X. INCX must not be zero.  Unchanged on
        exit.
```

11.12.3 fortran code

— dtbsv.f —

```
*  =====================================================================
      SUBROUTINE DTBSV(UPLO,TRANS,DIAG,N,K,A,LDA,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,K,LDA,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A(LDA,*),X(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION ZERO
      PARAMETER (ZERO=0.0D+0)
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP
      INTEGER I,INFO,IX,J,JX,KPLUS1,KX,L
      LOGICAL NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX,MIN
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
```

```
         INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
         INFO = 3
      ELSE IF (N.LT.0) THEN
         INFO = 4
      ELSE IF (K.LT.0) THEN
         INFO = 5
      ELSE IF (LDA.LT. (K+1)) THEN
         INFO = 7
      ELSE IF (INCX.EQ.0) THEN
         INFO = 9
      END IF
      IF (INFO.NE.0) THEN
         CALL XERBLA('DTBSV ',INFO)
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX  too small for descending loops.
*
      IF (INCX.LE.0) THEN
         KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
         KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed by sequentially with one pass through A.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x := inv( A )*x.
*
          IF (LSAME(UPLO,'U')) THEN
              KPLUS1 = K + 1
              IF (INCX.EQ.1) THEN
                  DO 20 J = N,1,-1
                      IF (X(J).NE.ZERO) THEN
                          L = KPLUS1 - J
                          IF (NOUNIT) X(J) = X(J)/A(KPLUS1,J)
                          TEMP = X(J)
                          DO 10 I = J - 1,MAX(1,J-K),-1
                              X(I) = X(I) - TEMP*A(L+I,J)
   10                     CONTINUE
```

```
                END IF
20          CONTINUE
        ELSE
            KX = KX + (N-1)*INCX
            JX = KX
            DO 40 J = N,1,-1
                KX = KX - INCX
                IF (X(JX).NE.ZERO) THEN
                    IX = KX
                    L = KPLUS1 - J
                    IF (NOUNIT) X(JX) = X(JX)/A(KPLUS1,J)
                    TEMP = X(JX)
                    DO 30 I = J - 1,MAX(1,J-K),-1
                        X(IX) = X(IX) - TEMP*A(L+I,J)
                        IX = IX - INCX
30                  CONTINUE
                END IF
                JX = JX - INCX
40          CONTINUE
        END IF
    ELSE
        IF (INCX.EQ.1) THEN
            DO 60 J = 1,N
                IF (X(J).NE.ZERO) THEN
                    L = 1 - J
                    IF (NOUNIT) X(J) = X(J)/A(1,J)
                    TEMP = X(J)
                    DO 50 I = J + 1,MIN(N,J+K)
                        X(I) = X(I) - TEMP*A(L+I,J)
50                  CONTINUE
                END IF
60          CONTINUE
        ELSE
            JX = KX
            DO 80 J = 1,N
                KX = KX + INCX
                IF (X(JX).NE.ZERO) THEN
                    IX = KX
                    L = 1 - J
                    IF (NOUNIT) X(JX) = X(JX)/A(1,J)
                    TEMP = X(JX)
                    DO 70 I = J + 1,MIN(N,J+K)
                        X(IX) = X(IX) - TEMP*A(L+I,J)
                        IX = IX + INCX
70                  CONTINUE
                END IF
                JX = JX + INCX
80          CONTINUE
        END IF
    END IF
```

```
      ELSE
*
*        Form  x := inv( A**T)*x.
*
          IF (LSAME(UPLO,'U')) THEN
              KPLUS1 = K + 1
              IF (INCX.EQ.1) THEN
                  DO 100 J = 1,N
                      TEMP = X(J)
                      L = KPLUS1 - J
                      DO 90 I = MAX(1,J-K),J - 1
                          TEMP = TEMP - A(L+I,J)*X(I)
   90                 CONTINUE
                      IF (NOUNIT) TEMP = TEMP/A(KPLUS1,J)
                      X(J) = TEMP
  100             CONTINUE
              ELSE
                  JX = KX
                  DO 120 J = 1,N
                      TEMP = X(JX)
                      IX = KX
                      L = KPLUS1 - J
                      DO 110 I = MAX(1,J-K),J - 1
                          TEMP = TEMP - A(L+I,J)*X(IX)
                          IX = IX + INCX
  110                 CONTINUE
                      IF (NOUNIT) TEMP = TEMP/A(KPLUS1,J)
                      X(JX) = TEMP
                      JX = JX + INCX
                      IF (J.GT.K) KX = KX + INCX
  120             CONTINUE
              END IF
          ELSE
              IF (INCX.EQ.1) THEN
                  DO 140 J = N,1,-1
                      TEMP = X(J)
                      L = 1 - J
                      DO 130 I = MIN(N,J+K),J + 1,-1
                          TEMP = TEMP - A(L+I,J)*X(I)
  130                 CONTINUE
                      IF (NOUNIT) TEMP = TEMP/A(1,J)
                      X(J) = TEMP
  140             CONTINUE
              ELSE
                  KX = KX + (N-1)*INCX
                  JX = KX
                  DO 160 J = N,1,-1
                      TEMP = X(JX)
                      IX = KX
                      L = 1 - J
```

```
                    DO 150 I = MIN(N,J+K),J + 1,-1
                        TEMP = TEMP - A(L+I,J)*X(IX)
                        IX = IX - INCX
  150               CONTINUE
                    IF (NOUNIT) TEMP = TEMP/A(1,J)
                    X(JX) = TEMP
                    JX = JX - INCX
                    IF ((N-J).GE.K) KX = KX - INCX
  160            CONTINUE
            END IF
         END IF
      END IF
*
      RETURN
*
*     End of DTBSV .
*
      END
```

———-

11.12.4 lisp code

— BLAS 2 dtbsv —

```
(let* ((zero 0.0))
  (declare (type (double-float 0.0 0.0) zero))
  (defun dtbsv (uplo trans diag n k a lda x incx)
    (declare (type (simple-array double-float (*)) x a)
             (type fixnum incx lda k n)
             (type character diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a double-float a-%data% a-%offset%)
         (x double-float x-%data% x-%offset%))
      (prog ((nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0) (kplus1 0) (kx 0)
             (l 0) (temp 0.0))
        (declare (type (member t nil) nounit)
                 (type fixnum i info ix j jx kplus1 kx l)
                 (type (double-float) temp))
        (setf info 0)
        (cond
          ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
           (setf info 1))
          ((and (not (char-equal trans #\N))
```

```
        (not (char-equal trans #\T))
        (not (char-equal trans #\C)))
   (setf info 2))
  ((and (not (char-equal diag #\U)) (not (char-equal diag #\N)))
   (setf info 3))
  ((< n 0)
   (setf info 4))
  ((< k 0)
   (setf info 5))
  ((< lda (f2cl-lib:int-add k 1))
   (setf info 7))
  ((= incx 0)
   (setf info 9)))
(cond
  ((/= info 0)
   (error
    " ** On entry to ~a parameter number ~a had an illegal value~%"
    "DTBSV" info)
   (go end_label)))
(if (= n 0) (go end_label))
(setf nounit (char-equal diag #\N))
(cond
  ((<= incx 0)
   (setf kx
           (f2cl-lib:int-sub 1
                             (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                               incx))))
  ((/= incx 1)
   (setf kx 1)))
(cond
  ((char-equal trans #\N)
   (cond
     ((char-equal uplo #\U)
      (setf kplus1 (f2cl-lib:int-add k 1))
      (cond
        ((= incx 1)
         (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((> j 1) nil)
           (tagbody
             (cond
               ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
                (setf l (f2cl-lib:int-sub kplus1 j))
                (if nounit
                    (setf (f2cl-lib:fref x-%data%
                                         (j)
                                         ((1 *))
                                         x-%offset%)
                          (/
                           (f2cl-lib:fref x-%data%
                                          (j)
```

```
                                        ((1 *))
                                        x-%offset%)
                              (f2cl-lib:fref a-%data%
                                        (kplus1 j)
                                        ((1 lda) (1 *))
                                        a-%offset%))))
                (setf temp
                      (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                (f2cl-lib:fdo (i
                               (f2cl-lib:int-add j
                                                 (f2cl-lib:int-sub 1))
                               (f2cl-lib:int-add i
                                                 (f2cl-lib:int-sub 1)))
                              ((> i
                                  (max (the fixnum 1)
                                       (the fixnum
                                        (f2cl-lib:int-add j
                                           (f2cl-lib:int-sub
                                                          k)))))
                               nil)
                 (tagbody
                   (setf (f2cl-lib:fref x-%data%
                                        (i)
                                        ((1 *))
                                        x-%offset%)
                         (-
                          (f2cl-lib:fref x-%data%
                                        (i)
                                        ((1 *))
                                        x-%offset%)
                          (* temp
                             (f2cl-lib:fref a-%data%
                                           ((f2cl-lib:int-add l i)
                                            j)
                                           ((1 lda) (1 *))
                                           a-%offset%)))))))))))
        (t
         (setf kx
                 (f2cl-lib:int-add kx
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-sub n 1)
                                    incx)))
         (setf jx kx)
         (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((> j 1) nil)
           (tagbody
             (setf kx (f2cl-lib:int-sub kx incx))
             (cond
               ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
                (setf ix kx)
```

```
    (setf l (f2cl-lib:int-sub kplus1 j))
    (if nounit
        (setf (f2cl-lib:fref x-%data%
                             (jx)
                             ((1 *))
                             x-%offset%)
              (/
               (f2cl-lib:fref x-%data%
                              (jx)
                              ((1 *))
                              x-%offset%)
               (f2cl-lib:fref a-%data%
                              (kplus1 j)
                              ((1 lda) (1 *))
                              a-%offset%))))
    (setf temp
            (f2cl-lib:fref x-%data%
                           (jx)
                           ((1 *))
                           x-%offset%))
    (f2cl-lib:fdo (i
                   (f2cl-lib:int-add j
                                     (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add i
                                     (f2cl-lib:int-sub 1)))
                  ((> i
                      (max (the fixnum 1)
                           (the fixnum
                                (f2cl-lib:int-add j
                                 (f2cl-lib:int-sub
                                                   k)))))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref x-%data%
                             (ix)
                             ((1 *))
                             x-%offset%)
              (-
               (f2cl-lib:fref x-%data%
                              (ix)
                              ((1 *))
                              x-%offset%)
               (* temp
                  (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add l i)
                                  j)
                                 ((1 lda) (1 *))
                                 a-%offset%))))
        (setf ix (f2cl-lib:int-sub ix incx))))))
(setf jx (f2cl-lib:int-sub jx incx)))))))
```

```
(t
 (cond
   ((= incx 1)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
          ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
           (setf l (f2cl-lib:int-sub 1 j))
           (if nounit
               (setf (f2cl-lib:fref x-%data%
                                    (j)
                                    ((1 *))
                                    x-%offset%)
                     (/
                      (f2cl-lib:fref x-%data%
                                     (j)
                                     ((1 *))
                                     x-%offset%)
                      (f2cl-lib:fref a-%data%
                                     (1 j)
                                     ((1 lda) (1 *))
                                     a-%offset%))))
           (setf temp
                   (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
           (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                            (f2cl-lib:int-add i 1))
                         ((> i
                             (min (the fixnum n)
                                  (the fixnum
                                       (f2cl-lib:int-add j k))))
                          nil)
             (tagbody
               (setf (f2cl-lib:fref x-%data%
                                    (i)
                                    ((1 *))
                                    x-%offset%)
                     (-
                      (f2cl-lib:fref x-%data%
                                     (i)
                                     ((1 *))
                                     x-%offset%)
                      (* temp
                         (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-add l i)
                                         j)
                                        ((1 lda) (1 *))
                                        a-%offset%))))))))))
   (t
    (setf jx kx)
```

```
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf kx (f2cl-lib:int-add kx incx))
    (cond
      ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
       (setf ix kx)
       (setf l (f2cl-lib:int-sub 1 j))
       (if nounit
           (setf (f2cl-lib:fref x-%data%
                                (jx)
                                ((1 *))
                                x-%offset%)
                 (/
                  (f2cl-lib:fref x-%data%
                                 (jx)
                                 ((1 *))
                                 x-%offset%)
                  (f2cl-lib:fref a-%data%
                                 (1 j)
                                 ((1 lda) (1 *))
                                 a-%offset%))))
       (setf temp
               (f2cl-lib:fref x-%data%
                              (jx)
                              ((1 *))
                              x-%offset%))
       (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                        (f2cl-lib:int-add i 1))
                     ((> i
                         (min (the fixnum n)
                              (the fixnum
                                   (f2cl-lib:int-add j k))))
                      nil)
         (tagbody
           (setf (f2cl-lib:fref x-%data%
                                (ix)
                                ((1 *))
                                x-%offset%)
                 (-
                  (f2cl-lib:fref x-%data%
                                 (ix)
                                 ((1 *))
                                 x-%offset%)
                  (* temp
                     (f2cl-lib:fref a-%data%
                                    ((f2cl-lib:int-add l i)
                                     j)
                                    ((1 lda) (1 *))
                                    a-%offset%))))
```

```
                  (setf ix (f2cl-lib:int-add ix incx))))))
            (setf jx (f2cl-lib:int-add jx incx)))))))))
(t
 (cond
   ((char-equal uplo #\U)
    (setf kplus1 (f2cl-lib:int-add k 1))
    (cond
      ((= incx 1)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (setf temp
                   (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
           (setf l (f2cl-lib:int-sub kplus1 j))
           (f2cl-lib:fdo (i
                          (max (the fixnum 1)
                               (the fixnum
                                    (f2cl-lib:int-add j
                                                      (f2cl-lib:int-sub
                                                       k))))
                          (f2cl-lib:int-add i 1))
                         ((> i
                             (f2cl-lib:int-add j
                                               (f2cl-lib:int-sub 1)))
                          nil)
             (tagbody
               (setf temp
                       (- temp
                          (*
                           (f2cl-lib:fref a-%data%
                                          ((f2cl-lib:int-add l i) j)
                                          ((1 lda) (1 *))
                                          a-%offset%)
                           (f2cl-lib:fref x-%data%
                                          (i)
                                          ((1 *))
                                          x-%offset%))))))
           (if nounit
               (setf temp
                       (/ temp
                          (f2cl-lib:fref a-%data%
                                         (kplus1 j)
                                         ((1 lda) (1 *))
                                         a-%offset%))))
           (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                   temp))))
      (t
       (setf jx kx)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
```

```
        (tagbody
          (setf temp
                  (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
          (setf ix kx)
          (setf l (f2cl-lib:int-sub kplus1 j))
          (f2cl-lib:fdo (i
                         (max (the fixnum 1)
                              (the fixnum
                                   (f2cl-lib:int-add j
                                                     (f2cl-lib:int-sub
                                                      k))))
                         (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add j
                                              (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf temp
                      (- temp
                         (*
                          (f2cl-lib:fref a-%data%
                                         ((f2cl-lib:int-add l i) j)
                                         ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref x-%data%
                                         (ix)
                                         ((1 *))
                                         x-%offset%))))
              (setf ix (f2cl-lib:int-add ix incx))))
          (if nounit
              (setf temp
                      (/ temp
                         (f2cl-lib:fref a-%data%
                                        (kplus1 j)
                                        ((1 lda) (1 *))
                                        a-%offset%))))
          (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                  temp)
          (setf jx (f2cl-lib:int-add jx incx))
          (if (> j k) (setf kx (f2cl-lib:int-add kx incx))))))))
      (t
       (cond
         ((= incx 1)
          (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                        ((> j 1) nil)
            (tagbody
              (setf temp
                      (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
              (setf l (f2cl-lib:int-sub 1 j))
              (f2cl-lib:fdo (i
```

```
                        (min (the fixnum n)
                             (the fixnum
                                  (f2cl-lib:int-add j k)))
                        (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                       ((> i (f2cl-lib:int-add j 1)) nil)
          (tagbody
            (setf temp
                    (- temp
                       (*
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add l i) j)
                                       ((1 lda) (1 *))
                                       a-%offset%)
                        (f2cl-lib:fref x-%data%
                                       (i)
                                       ((1 *))
                                       x-%offset%))))))
      (if nounit
          (setf temp
                  (/ temp
                     (f2cl-lib:fref a-%data%
                                    (1 j)
                                    ((1 lda) (1 *))
                                    a-%offset%))))
      (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
              temp))))
(t
 (setf kx
         (f2cl-lib:int-add kx
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-sub n 1)
                            incx)))
 (setf jx kx)
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (setf temp
             (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
     (setf ix kx)
     (setf l (f2cl-lib:int-sub 1 j))
     (f2cl-lib:fdo (i
                    (min (the fixnum n)
                         (the fixnum
                              (f2cl-lib:int-add j k)))
                    (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                   ((> i (f2cl-lib:int-add j 1)) nil)
       (tagbody
         (setf temp
                 (- temp
                    (*
```

```
                                    (f2cl-lib:fref a-%data%
                                                   ((f2cl-lib:int-add l i) j)
                                                   ((1 lda) (1 *))
                                                   a-%offset%)
                                    (f2cl-lib:fref x-%data%
                                                   (ix)
                                                   ((1 *))
                                                   x-%offset%))))
                        (setf ix (f2cl-lib:int-sub ix incx))))
                    (if nounit
                        (setf temp
                                (/ temp
                                   (f2cl-lib:fref a-%data%
                                                  (1 j)
                                                  ((1 lda) (1 *))
                                                  a-%offset%))))
                    (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                            temp)
                    (setf jx (f2cl-lib:int-sub jx incx))
                    (if (>= (f2cl-lib:int-sub n j) k)
                        (setf kx (f2cl-lib:int-sub kx incx))))))))))
 end_label
        (return (values nil nil nil nil nil nil nil nil nil))))))
```

———-

11.13 dtpmv - matrix-vector $x = Ax$ or $x = A^T x$

11.13.1 Axiom unit tests

— dtpmv.input —

```
)set break resume
)sys rm -f dtpmv.output
)spool dtpmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

11.13.2 Axiom help page

— dtpmv.help —

```
==================================================================
dtpmv examples
==================================================================

==================================================================
Man Page Details
==================================================================

 NAME
      DTPMV - perform one of the matrix-vector operations   x :=
      A*x, or x := A'*x,

 SYNOPSIS
      SUBROUTINE DTPMV ( UPLO, TRANS, DIAG, N, AP, X, INCX )

          INTEGER      INCX, N

          CHARACTER*1  DIAG, TRANS, UPLO

          DOUBLE       PRECISION AP( * ), X( * )

 PURPOSE
      DTPMV  performs one of the matrix-vector operations

      where x is an n element vector and  A is an n by n unit, or
      non-unit, upper or lower triangular matrix, supplied in
      packed form.

 PARAMETERS
      UPLO   - CHARACTER*1.
             On entry, UPLO specifies whether the matrix is an
             upper or lower triangular matrix as follows:

             UPLO = 'U' or 'u'   A is an upper triangular matrix.

             UPLO = 'L' or 'l'   A is a lower triangular matrix.

             Unchanged on exit.

      TRANS  - CHARACTER*1.
             On entry, TRANS specifies the operation to be per-
             formed as follows:

             TRANS = 'N' or 'n'   x := A*x.
```

```
        TRANS = 'T' or 't'   x := A'*x.

        TRANS = 'C' or 'c'   x := A'*x.

        Unchanged on exit.

DIAG    - CHARACTER*1.
        On entry, DIAG specifies whether or not A is unit
        triangular as follows:

        DIAG = 'U' or 'u'   A is assumed to be unit triangu-
        lar.

        DIAG = 'N' or 'n'   A is not assumed to be unit

        triangular.

        Unchanged on exit.

N       - INTEGER.
        On entry, N specifies the order of the matrix A.  N
        must be at least zero.  Unchanged on exit.

AP      - DOUBLE PRECISION array of DIMENSION at least
        ( ( n*( n + 1 ) )/2 ).  Before entry with  UPLO = 'U'
        or 'u', the array AP must contain the upper triangu-
        lar matrix packed sequentially, column by column, so
        that AP( 1 ) contains a( 1, 1 ), AP( 2 ) and AP( 3 )
        contain a( 1, 2 ) and a( 2, 2 ) respectively, and so
        on.  Before entry with UPLO = 'L' or 'l', the array
        AP must contain the lower triangular matrix packed
        sequentially, column by column, so that AP( 1 ) con-
        tains a( 1, 1 ), AP( 2 ) and AP( 3 ) contain a( 2, 1
        ) and a( 3, 1 ) respectively, and so on.  Note that
        when  DIAG = 'U' or 'u', the diagonal elements of A
        are not referenced, but are assumed to be unity.
        Unchanged on exit.

X       - DOUBLE PRECISION array of dimension at least
        ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
        incremented array X must contain the n element vector
        x. On exit, X is overwritten with the transformed
        vector x.

INCX    - INTEGER.
        On entry, INCX specifies the increment for the ele-
        ments of X. INCX must not be zero.  Unchanged on
        exit.
```

———-

11.13.3 fortran code

— dtpmv.f —

```
*  ===================================================================
      SUBROUTINE DTPMV(UPLO,TRANS,DIAG,N,AP,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION AP(*),X(*)
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION ZERO
      PARAMETER (ZERO=0.0D+0)
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP
      INTEGER I,INFO,IX,J,JX,K,KK,KX
      LOGICAL NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
```

```
         INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
         INFO = 3
      ELSE IF (N.LT.0) THEN
         INFO = 4
      ELSE IF (INCX.EQ.0) THEN
         INFO = 7
      END IF
      IF (INFO.NE.0) THEN
         CALL XERBLA('DTPMV ',INFO)
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX  too small for descending loops.
*
      IF (INCX.LE.0) THEN
         KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
         KX = 1
      END IF
*
*     Start the operations. In this version the elements of AP are
*     accessed sequentially with one pass through AP.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x:= A*x.
*
          IF (LSAME(UPLO,'U')) THEN
              KK = 1
              IF (INCX.EQ.1) THEN
                  DO 20 J = 1,N
                      IF (X(J).NE.ZERO) THEN
                          TEMP = X(J)
                          K = KK
                          DO 10 I = 1,J - 1
                              X(I) = X(I) + TEMP*AP(K)
                              K = K + 1
   10                     CONTINUE
                          IF (NOUNIT) X(J) = X(J)*AP(KK+J-1)
                      END IF
                      KK = KK + J
   20             CONTINUE
```

```
            ELSE
                JX = KX
                DO 40 J = 1,N
                    IF (X(JX).NE.ZERO) THEN
                        TEMP = X(JX)
                        IX = KX
                        DO 30 K = KK,KK + J - 2
                            X(IX) = X(IX) + TEMP*AP(K)
                            IX = IX + INCX
30                      CONTINUE
                        IF (NOUNIT) X(JX) = X(JX)*AP(KK+J-1)
                    END IF
                    JX = JX + INCX
                    KK = KK + J
40              CONTINUE
            END IF
        ELSE
            KK = (N* (N+1))/2
            IF (INCX.EQ.1) THEN
                DO 60 J = N,1,-1
                    IF (X(J).NE.ZERO) THEN
                        TEMP = X(J)
                        K = KK
                        DO 50 I = N,J + 1,-1
                            X(I) = X(I) + TEMP*AP(K)
                            K = K - 1
50                      CONTINUE
                        IF (NOUNIT) X(J) = X(J)*AP(KK-N+J)
                    END IF
                    KK = KK - (N-J+1)
60              CONTINUE
            ELSE
                KX = KX + (N-1)*INCX
                JX = KX
                DO 80 J = N,1,-1
                    IF (X(JX).NE.ZERO) THEN
                        TEMP = X(JX)
                        IX = KX
                        DO 70 K = KK,KK - (N- (J+1)),-1
                            X(IX) = X(IX) + TEMP*AP(K)
                            IX = IX - INCX
70                      CONTINUE
                        IF (NOUNIT) X(JX) = X(JX)*AP(KK-N+J)
                    END IF
                    JX = JX - INCX
                    KK = KK - (N-J+1)
80              CONTINUE
            END IF
        END IF
    ELSE
```

```
*
*        Form  x := A**T*x.
*
          IF (LSAME(UPLO,'U')) THEN
              KK = (N* (N+1))/2
              IF (INCX.EQ.1) THEN
                  DO 100 J = N,1,-1
                      TEMP = X(J)
                      IF (NOUNIT) TEMP = TEMP*AP(KK)
                      K = KK - 1
                      DO 90 I = J - 1,1,-1
                          TEMP = TEMP + AP(K)*X(I)
                          K = K - 1
   90                 CONTINUE
                      X(J) = TEMP
                      KK = KK - J
  100             CONTINUE
              ELSE
                  JX = KX + (N-1)*INCX
                  DO 120 J = N,1,-1
                      TEMP = X(JX)
                      IX = JX
                      IF (NOUNIT) TEMP = TEMP*AP(KK)
                      DO 110 K = KK - 1,KK - J + 1,-1
                          IX = IX - INCX
                          TEMP = TEMP + AP(K)*X(IX)
  110                 CONTINUE
                      X(JX) = TEMP
                      JX = JX - INCX
                      KK = KK - J
  120             CONTINUE
              END IF
          ELSE
              KK = 1
              IF (INCX.EQ.1) THEN
                  DO 140 J = 1,N
                      TEMP = X(J)
                      IF (NOUNIT) TEMP = TEMP*AP(KK)
                      K = KK + 1
                      DO 130 I = J + 1,N
                          TEMP = TEMP + AP(K)*X(I)
                          K = K + 1
  130                 CONTINUE
                      X(J) = TEMP
                      KK = KK + (N-J+1)
  140             CONTINUE
              ELSE
                  JX = KX
                  DO 160 J = 1,N
                      TEMP = X(JX)
```

```
                    IX = JX
                    IF (NOUNIT) TEMP = TEMP*AP(KK)
                    DO 150 K = KK + 1,KK + N - J
                        IX = IX + INCX
                        TEMP = TEMP + AP(K)*X(IX)
  150               CONTINUE
                    X(JX) = TEMP
                    JX = JX + INCX
                    KK = KK + (N-J+1)
  160            CONTINUE
             END IF
         END IF
     END IF
*
     RETURN
*
*    End of DTPMV .
*
     END
```

———-

11.13.4 lisp code

— BLAS 2 dtpmv —

```
(let* ((zero 0.0))
  (declare (type (double-float 0.0 0.0) zero))
  (defun dtpmv (uplo trans diag n ap x incx)
    (declare (type (simple-array double-float (*)) x ap)
             (type fixnum incx n)
             (type character diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (ap double-float ap-%data% ap-%offset%)
         (x double-float x-%data% x-%offset%))
      (prog ((nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0) (k 0) (kk 0)
             (kx 0) (temp 0.0))
        (declare (type (member t nil) nounit)
                 (type fixnum i info ix j jx k kk kx)
                 (type (double-float) temp))
        (setf info 0)
        (cond
          ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
           (setf info 1))
```

```
  ((and (not (char-equal trans #\N))
        (not (char-equal trans #\T))
        (not (char-equal trans #\C)))
   (setf info 2))
  ((and (not (char-equal diag #\U)) (not (char-equal diag #\N)))
   (setf info 3))
  ((< n 0)
   (setf info 4))
  ((= incx 0)
   (setf info 7)))
(cond
  ((/= info 0)
   (error
    " ** On entry to ~a parameter number ~a had an illegal value~%"
    "DTPMV" info)
   (go end_label)))
(if (= n 0) (go end_label))
(setf nounit (char-equal diag #\N))
(cond
  ((<= incx 0)
   (setf kx
           (f2cl-lib:int-sub 1
                             (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                               incx))))
  ((/= incx 1)
   (setf kx 1)))
(cond
  ((char-equal trans #\N)
   (cond
     ((char-equal uplo #\U)
      (setf kk 1)
      (cond
        ((= incx 1)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (cond
               ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
                (setf temp
                        (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                (setf k kk)
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i
                                  (f2cl-lib:int-add j
                                                    (f2cl-lib:int-sub
                                                     1)))
                               nil)
                  (tagbody
                    (setf (f2cl-lib:fref x-%data%
                                         (i)
```

```
                                        ((1 *))
                                        x-%offset%)
                                (+
                                 (f2cl-lib:fref x-%data%
                                                (i)
                                                ((1 *))
                                                x-%offset%)
                                 (* temp
                                    (f2cl-lib:fref ap-%data%
                                                   (k)
                                                   ((1 *))
                                                   ap-%offset%))))
                            (setf k (f2cl-lib:int-add k 1))))
                        (if nounit
                            (setf (f2cl-lib:fref x-%data%
                                                 (j)
                                                 ((1 *))
                                                 x-%offset%)
                                  (*
                                   (f2cl-lib:fref x-%data%
                                                  (j)
                                                  ((1 *))
                                                  x-%offset%)
                                   (f2cl-lib:fref ap-%data%
                                                  ((f2cl-lib:int-sub
                                                    (f2cl-lib:int-add kk j)
                                                    1))
                                                  ((1 *))
                                                  ap-%offset%))))))
                    (setf kk (f2cl-lib:int-add kk j)))))
                (t
                 (setf jx kx)
                 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                               ((> j n) nil)
                   (tagbody
                     (cond
                       ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
                        (setf temp
                                (f2cl-lib:fref x-%data%
                                               (jx)
                                               ((1 *))
                                               x-%offset%))
                        (setf ix kx)
                        (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                                      ((> k
                                          (f2cl-lib:int-add kk
                                                            j
                                                            (f2cl-lib:int-sub
                                                             2)))
                                       nil)
```

```
                (tagbody
                  (setf (f2cl-lib:fref x-%data%
                                       (ix)
                                       ((1 *))
                                       x-%offset%)
                        (+
                         (f2cl-lib:fref x-%data%
                                        (ix)
                                        ((1 *))
                                        x-%offset%)
                         (* temp
                            (f2cl-lib:fref ap-%data%
                                           (k)
                                           ((1 *))
                                           ap-%offset%))))
                  (setf ix (f2cl-lib:int-add ix incx))))
              (if nounit
                  (setf (f2cl-lib:fref x-%data%
                                       (jx)
                                       ((1 *))
                                       x-%offset%)
                        (*
                         (f2cl-lib:fref x-%data%
                                        (jx)
                                        ((1 *))
                                        x-%offset%)
                         (f2cl-lib:fref ap-%data%
                                        ((f2cl-lib:int-sub
                                          (f2cl-lib:int-add kk j)
                                          1))
                                        ((1 *))
                                        ap-%offset%))))))
          (setf jx (f2cl-lib:int-add jx incx))
          (setf kk (f2cl-lib:int-add kk j)))))))
(t
 (setf kk (the fixnum (truncate (* n (+ n 1)) 2)))
 (cond
   ((= incx 1)
    (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                  ((> j 1) nil)
      (tagbody
        (cond
          ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
           (setf temp
                   (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
           (setf k kk)
           (f2cl-lib:fdo (i n
                          (f2cl-lib:int-add i
                                            (f2cl-lib:int-sub 1)))
                         ((> i (f2cl-lib:int-add j 1)) nil)
```

```
                (tagbody
                  (setf (f2cl-lib:fref x-%data%
                                       (i)
                                       ((1 *))
                                       x-%offset%)
                        (+
                         (f2cl-lib:fref x-%data%
                                        (i)
                                        ((1 *))
                                        x-%offset%)
                         (* temp
                            (f2cl-lib:fref ap-%data%
                                           (k)
                                           ((1 *))
                                           ap-%offset%))))
                  (setf k (f2cl-lib:int-sub k 1))))
              (if nounit
                  (setf (f2cl-lib:fref x-%data%
                                       (j)
                                       ((1 *))
                                       x-%offset%)
                        (*
                         (f2cl-lib:fref x-%data%
                                        (j)
                                        ((1 *))
                                        x-%offset%)
                         (f2cl-lib:fref ap-%data%
                                        ((f2cl-lib:int-add
                                          (f2cl-lib:int-sub kk n)
                                          j))
                                        ((1 *))
                                        ap-%offset%))))))
          (setf kk
                  (f2cl-lib:int-sub kk
                                    (f2cl-lib:int-add
                                     (f2cl-lib:int-sub n j)
                                     1))))))
      (t
       (setf kx
               (f2cl-lib:int-add kx
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1)
                                  incx)))
       (setf jx kx)
       (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     ((> j 1) nil)
         (tagbody
           (cond
             ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
              (setf temp
```

```
        (f2cl-lib:fref x-%data%
                       (jx)
                       ((1 *))
                       x-%offset%))
(setf ix kx)
(f2cl-lib:fdo (k kk
               (f2cl-lib:int-add k
                                 (f2cl-lib:int-sub 1)))
              ((> k
                  (f2cl-lib:int-add kk
                                    (f2cl-lib:int-sub
                                     (f2cl-lib:int-add
                                      n
                                      (f2cl-lib:int-sub
                                       (f2cl-lib:int-add
                                        j
                                        1))))))
               nil)
  (tagbody
    (setf (f2cl-lib:fref x-%data%
                         (ix)
                         ((1 *))
                         x-%offset%)
          (+
           (f2cl-lib:fref x-%data%
                          (ix)
                          ((1 *))
                          x-%offset%)
           (* temp
              (f2cl-lib:fref ap-%data%
                             (k)
                             ((1 *))
                             ap-%offset%))))
    (setf ix (f2cl-lib:int-sub ix incx))))
(if nounit
    (setf (f2cl-lib:fref x-%data%
                         (jx)
                         ((1 *))
                         x-%offset%)
          (*
           (f2cl-lib:fref x-%data%
                          (jx)
                          ((1 *))
                          x-%offset%)
           (f2cl-lib:fref ap-%data%
                          ((f2cl-lib:int-add
                            (f2cl-lib:int-sub kk n)
                            j))
                          ((1 *))
                          ap-%offset%))))))
```

```
            (setf jx (f2cl-lib:int-sub jx incx))
            (setf kk
                    (f2cl-lib:int-sub kk
                                      (f2cl-lib:int-add
                                       (f2cl-lib:int-sub n j)
                                       1)))))))))))
(t
 (cond
   ((char-equal uplo #\U)
    (setf kk (the fixnum (truncate (* n (+ n 1)) 2)))
    (cond
      ((= incx 1)
       (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     ((> j 1) nil)
         (tagbody
           (setf temp
                   (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
           (if nounit
               (setf temp
                       (* temp
                          (f2cl-lib:fref ap-%data%
                                         (kk)
                                         ((1 *))
                                         ap-%offset%))))
           (setf k (f2cl-lib:int-sub kk 1))
           (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                          (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                         ((> i 1) nil)
             (tagbody
               (setf temp
                       (+ temp
                          (*
                           (f2cl-lib:fref ap-%data%
                                          (k)
                                          ((1 *))
                                          ap-%offset%)
                           (f2cl-lib:fref x-%data%
                                          (i)
                                          ((1 *))
                                          x-%offset%))))
               (setf k (f2cl-lib:int-sub k 1))))
           (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                   temp)
           (setf kk (f2cl-lib:int-sub kk j)))))
      (t
       (setf jx
               (f2cl-lib:int-add kx
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1)
                                  incx)))
```

```
        (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                      ((> j 1) nil)
          (tagbody
            (setf temp
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
            (setf ix jx)
            (if nounit
                (setf temp
                        (* temp
                           (f2cl-lib:fref ap-%data%
                                          (kk)
                                          ((1 *))
                                          ap-%offset%))))
            (f2cl-lib:fdo (k
                           (f2cl-lib:int-add kk (f2cl-lib:int-sub 1))
                           (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                          ((> k
                              (f2cl-lib:int-add kk
                                                (f2cl-lib:int-sub j)
                                                1))
                           nil)
              (tagbody
                (setf ix (f2cl-lib:int-sub ix incx))
                (setf temp
                        (+ temp
                           (*
                            (f2cl-lib:fref ap-%data%
                                           (k)
                                           ((1 *))
                                           ap-%offset%)
                            (f2cl-lib:fref x-%data%
                                           (ix)
                                           ((1 *))
                                           x-%offset%))))))
            (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                    temp)
            (setf jx (f2cl-lib:int-sub jx incx))
            (setf kk (f2cl-lib:int-sub kk j)))))))
    (t
     (setf kk 1)
     (cond
       ((= incx 1)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf temp
                    (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
            (if nounit
                (setf temp
                        (* temp
```

```
                        (f2cl-lib:fref ap-%data%
                                       (kk)
                                       ((1 *))
                                       ap-%offset%))))
            (setf k (f2cl-lib:int-add kk 1))
            (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                           (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf temp
                        (+ temp
                           (*
                            (f2cl-lib:fref ap-%data%
                                           (k)
                                           ((1 *))
                                           ap-%offset%)
                            (f2cl-lib:fref x-%data%
                                           (i)
                                           ((1 *))
                                           x-%offset%))))
                (setf k (f2cl-lib:int-add k 1))))
            (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                  temp)
            (setf kk
                  (f2cl-lib:int-add kk
                                    (f2cl-lib:int-add
                                     (f2cl-lib:int-sub n j)
                                     1))))))
        (t
         (setf jx kx)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf temp
                   (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
             (setf ix jx)
             (if nounit
                 (setf temp
                         (* temp
                            (f2cl-lib:fref ap-%data%
                                           (kk)
                                           ((1 *))
                                           ap-%offset%))))
             (f2cl-lib:fdo (k (f2cl-lib:int-add kk 1)
                            (f2cl-lib:int-add k 1))
                           ((> k
                               (f2cl-lib:int-add kk
                                                 n
                                                 (f2cl-lib:int-sub j)))
                            nil)
```

```
                      (tagbody
                        (setf ix (f2cl-lib:int-add ix incx))
                        (setf temp
                                (+ temp
                                   (*
                                    (f2cl-lib:fref ap-%data%
                                                   (k)
                                                   ((1 *))
                                                   ap-%offset%)
                                    (f2cl-lib:fref x-%data%
                                                   (ix)
                                                   ((1 *))
                                                   x-%offset%))))))
                    (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                            temp)
                    (setf jx (f2cl-lib:int-add jx incx))
                    (setf kk
                            (f2cl-lib:int-add kk
                                              (f2cl-lib:int-add
                                               (f2cl-lib:int-sub n j)
                                               1)))))))))))
 end_label
        (return (values nil nil nil nil nil nil nil))))))
```

———-

11.14 dtpsv - solve systems of equations $Ax = b$ or $A^T x = b$

11.14.1 Axiom unit tests

— **dtpsv.input** —

```
)set break resume
)sys rm -f dtpsv.output
)spool dtpsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

11.14.2 Axiom help page

— dtpsv.help —

```
==================================================================
dtpsv examples
==================================================================

==================================================================
Man Page Details
==================================================================

 NAME
      DTPSV - solve one of the systems of equations   A*x = b, or
      A'*x = b,

 SYNOPSIS
      SUBROUTINE DTPSV ( UPLO, TRANS, DIAG, N, AP, X, INCX )

          INTEGER      INCX, N

          CHARACTER*1  DIAG, TRANS, UPLO

          DOUBLE       PRECISION AP( * ), X( * )

 PURPOSE
      DTPSV  solves one of the systems of equations

      where b and x are n element vectors and A is an n by n unit,
      or non-unit, upper or lower triangular matrix, supplied in
      packed form.

      No test for singularity or near-singularity is included in
      this routine. Such tests must be performed before calling
      this routine.

 PARAMETERS
      UPLO   - CHARACTER*1.
             On entry, UPLO specifies whether the matrix is an
             upper or lower triangular matrix as follows:

             UPLO = 'U' or 'u'   A is an upper triangular matrix.

             UPLO = 'L' or 'l'   A is a lower triangular matrix.

             Unchanged on exit.

      TRANS  - CHARACTER*1.
             On entry, TRANS specifies the equations to be solved
```

```
        as follows:

        TRANS = 'N' or 'n'   A*x = b.

        TRANS = 'T' or 't'   A'*x = b.

        TRANS = 'C' or 'c'   A'*x = b.

        Unchanged on exit.

DIAG    - CHARACTER*1.
        On entry, DIAG specifies whether or not A is unit
        triangular as follows:

        DIAG = 'U' or 'u'   A is assumed to be unit triangu-
        lar.

        DIAG = 'N' or 'n'   A is not assumed to be unit tri-
        angular.

        Unchanged on exit.

N       - INTEGER.
        On entry, N specifies the order of the matrix A.  N
        must be at least zero.  Unchanged on exit.

AP      - DOUBLE PRECISION array of DIMENSION at least
        ( ( n*( n + 1 ) )/2 ).  Before entry with  UPLO = 'U'
        or 'u', the array AP must contain the upper triangu-
        lar matrix packed sequentially, column by column, so
        that AP( 1 ) contains a( 1, 1 ), AP( 2 ) and AP( 3 )
        contain a( 1, 2 ) and a( 2, 2 ) respectively, and so
        on.  Before entry with UPLO = 'L' or 'l', the array
        AP must contain the lower triangular matrix packed
        sequentially, column by column, so that AP( 1 ) con-
        tains a( 1, 1 ), AP( 2 ) and AP( 3 ) contain a( 2, 1
        ) and a( 3, 1 ) respectively, and so on.  Note that
        when  DIAG = 'U' or 'u', the diagonal elements of A
        are not referenced, but are assumed to be unity.
        Unchanged on exit.

X       - DOUBLE PRECISION array of dimension at least
        ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
        incremented array X must contain the n element
        right-hand side vector b. On exit, X is overwritten
        with the solution vector x.

INCX    - INTEGER.
        On entry, INCX specifies the increment for the ele-
        ments of X. INCX must not be zero.  Unchanged on
```

exit.

———-

11.14.3 fortran code

— **dtpsv.f** —

```
*  ==================================================================
      SUBROUTINE DTPSV(UPLO,TRANS,DIAG,N,AP,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION AP(*),X(*)
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION ZERO
      PARAMETER (ZERO=0.0D+0)
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP
      INTEGER I,INFO,IX,J,JX,K,KK,KX
      LOGICAL NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
```

```
         INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
         INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
         INFO = 3
      ELSE IF (N.LT.0) THEN
         INFO = 4
      ELSE IF (INCX.EQ.0) THEN
         INFO = 7
      END IF
      IF (INFO.NE.0) THEN
         CALL XERBLA('DTPSV ',INFO)
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX  too small for descending loops.
*
      IF (INCX.LE.0) THEN
         KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
         KX = 1
      END IF
*
*     Start the operations. In this version the elements of AP are
*     accessed sequentially with one pass through AP.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x := inv( A )*x.
*
          IF (LSAME(UPLO,'U')) THEN
              KK = (N* (N+1))/2
              IF (INCX.EQ.1) THEN
                  DO 20 J = N,1,-1
                      IF (X(J).NE.ZERO) THEN
                          IF (NOUNIT) X(J) = X(J)/AP(KK)
                          TEMP = X(J)
                          K = KK - 1
                          DO 10 I = J - 1,1,-1
                              X(I) = X(I) - TEMP*AP(K)
                              K = K - 1
   10                     CONTINUE
```

```
                END IF
                KK = KK - J
20          CONTINUE
        ELSE
            JX = KX + (N-1)*INCX
            DO 40 J = N,1,-1
                IF (X(JX).NE.ZERO) THEN
                    IF (NOUNIT) X(JX) = X(JX)/AP(KK)
                    TEMP = X(JX)
                    IX = JX
                    DO 30 K = KK - 1,KK - J + 1,-1
                        IX = IX - INCX
                        X(IX) = X(IX) - TEMP*AP(K)
30                  CONTINUE
                END IF
                JX = JX - INCX
                KK = KK - J
40          CONTINUE
        END IF
    ELSE
        KK = 1
        IF (INCX.EQ.1) THEN
            DO 60 J = 1,N
                IF (X(J).NE.ZERO) THEN
                    IF (NOUNIT) X(J) = X(J)/AP(KK)
                    TEMP = X(J)
                    K = KK + 1
                    DO 50 I = J + 1,N
                        X(I) = X(I) - TEMP*AP(K)
                        K = K + 1
50                  CONTINUE
                END IF
                KK = KK + (N-J+1)
60          CONTINUE
        ELSE
            JX = KX
            DO 80 J = 1,N
                IF (X(JX).NE.ZERO) THEN
                    IF (NOUNIT) X(JX) = X(JX)/AP(KK)
                    TEMP = X(JX)
                    IX = JX
                    DO 70 K = KK + 1,KK + N - J
                        IX = IX + INCX
                        X(IX) = X(IX) - TEMP*AP(K)
70                  CONTINUE
                END IF
                JX = JX + INCX
                KK = KK + (N-J+1)
80          CONTINUE
        END IF
```

```
         END IF
      ELSE
*
*        Form  x := inv( A**T )*x.
*
         IF (LSAME(UPLO,'U')) THEN
             KK = 1
             IF (INCX.EQ.1) THEN
                 DO 100 J = 1,N
                     TEMP = X(J)
                     K = KK
                     DO 90 I = 1,J - 1
                         TEMP = TEMP - AP(K)*X(I)
                         K = K + 1
   90                CONTINUE
                     IF (NOUNIT) TEMP = TEMP/AP(KK+J-1)
                     X(J) = TEMP
                     KK = KK + J
  100            CONTINUE
             ELSE
                 JX = KX
                 DO 120 J = 1,N
                     TEMP = X(JX)
                     IX = KX
                     DO 110 K = KK,KK + J - 2
                         TEMP = TEMP - AP(K)*X(IX)
                         IX = IX + INCX
  110                CONTINUE
                     IF (NOUNIT) TEMP = TEMP/AP(KK+J-1)
                     X(JX) = TEMP
                     JX = JX + INCX
                     KK = KK + J
  120            CONTINUE
             END IF
         ELSE
             KK = (N* (N+1))/2
             IF (INCX.EQ.1) THEN
                 DO 140 J = N,1,-1
                     TEMP = X(J)
                     K = KK
                     DO 130 I = N,J + 1,-1
                         TEMP = TEMP - AP(K)*X(I)
                         K = K - 1
  130                CONTINUE
                     IF (NOUNIT) TEMP = TEMP/AP(KK-N+J)
                     X(J) = TEMP
                     KK = KK - (N-J+1)
  140            CONTINUE
             ELSE
                 KX = KX + (N-1)*INCX
```

```
                JX = KX
                DO 160 J = N,1,-1
                    TEMP = X(JX)
                    IX = KX
                    DO 150 K = KK,KK - (N- (J+1)),-1
                        TEMP = TEMP - AP(K)*X(IX)
                        IX = IX - INCX
 150                CONTINUE
                    IF (NOUNIT) TEMP = TEMP/AP(KK-N+J)
                    X(JX) = TEMP
                    JX = JX - INCX
                    KK = KK - (N-J+1)
 160            CONTINUE
            END IF
        END IF
    END IF
*
    RETURN
*
*   End of DTPSV .
*
    END
```

———-

11.14.4 lisp code

— BLAS 2 dtpsv —

```
(let* ((zero 0.0))
  (declare (type (double-float 0.0 0.0) zero))
  (defun dtpsv (uplo trans diag n ap x incx)
    (declare (type (simple-array double-float (*)) x ap)
             (type fixnum incx n)
             (type character diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (ap double-float ap-%data% ap-%offset%)
         (x double-float x-%data% x-%offset%))
      (prog ((nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0) (k 0) (kk 0)
             (kx 0) (temp 0.0))
        (declare (type (member t nil) nounit)
                 (type fixnum i info ix j jx k kk kx)
                 (type (double-float) temp))
        (setf info 0)
```

```
(cond
  ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
   (setf info 1))
  ((and (not (char-equal trans #\N))
        (not (char-equal trans #\T))
        (not (char-equal trans #\C)))
   (setf info 2))
  ((and (not (char-equal diag #\U)) (not (char-equal diag #\N)))
   (setf info 3))
  ((< n 0)
   (setf info 4))
  ((= incx 0)
   (setf info 7)))
(cond
  ((/= info 0)
   (error
    " ** On entry to ~a parameter number ~a had an illegal value~%"
    "DTPSV" info)
   (go end_label)))
(if (= n 0) (go end_label))
(setf nounit (char-equal diag #\N))
(cond
  ((<= incx 0)
   (setf kx
           (f2cl-lib:int-sub 1
                             (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                               incx))))
  ((/= incx 1)
   (setf kx 1)))
(cond
  ((char-equal trans #\N)
   (cond
     ((char-equal uplo #\U)
      (setf kk (the fixnum (truncate (* n (+ n 1)) 2)))
      (cond
        ((= incx 1)
         (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((> j 1) nil)
           (tagbody
             (cond
               ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
                (if nounit
                    (setf (f2cl-lib:fref x-%data%
                                         (j)
                                         ((1 *))
                                         x-%offset%)
                          (/
                           (f2cl-lib:fref x-%data%
                                          (j)
                                          ((1 *))
```

```
                                        x-%offset%)
                          (f2cl-lib:fref ap-%data%
                                         (kk)
                                         ((1 *))
                                         ap-%offset%))))
                 (setf temp
                         (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                 (setf k (f2cl-lib:int-sub kk 1))
                 (f2cl-lib:fdo (i
                                (f2cl-lib:int-add j
                                                  (f2cl-lib:int-sub 1))
                                (f2cl-lib:int-add i
                                                  (f2cl-lib:int-sub 1)))
                               ((> i 1) nil)
                   (tagbody
                     (setf (f2cl-lib:fref x-%data%
                                          (i)
                                          ((1 *))
                                          x-%offset%)
                           (-
                            (f2cl-lib:fref x-%data%
                                           (i)
                                           ((1 *))
                                           x-%offset%)
                            (* temp
                               (f2cl-lib:fref ap-%data%
                                              (k)
                                              ((1 *))
                                              ap-%offset%))))
                     (setf k (f2cl-lib:int-sub k 1))))))
             (setf kk (f2cl-lib:int-sub kk j)))))
        (t
         (setf jx
                 (f2cl-lib:int-add kx
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-sub n 1)
                                    incx)))
         (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((> j 1) nil)
           (tagbody
             (cond
               ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
                (if nounit
                    (setf (f2cl-lib:fref x-%data%
                                         (jx)
                                         ((1 *))
                                         x-%offset%)
                          (/
                           (f2cl-lib:fref x-%data%
                                          (jx)
```

```
                                        ((1 *))
                                        x-%offset%)
                              (f2cl-lib:fref ap-%data%
                                        (kk)
                                        ((1 *))
                                        ap-%offset%))))
                (setf temp
                        (f2cl-lib:fref x-%data%
                                      (jx)
                                      ((1 *))
                                      x-%offset%))
                (setf ix jx)
                (f2cl-lib:fdo (k
                               (f2cl-lib:int-add kk
                                                 (f2cl-lib:int-sub 1))
                               (f2cl-lib:int-add k
                                                 (f2cl-lib:int-sub 1)))
                              ((> k
                                  (f2cl-lib:int-add kk
                                                    (f2cl-lib:int-sub
                                                     j)
                                                    1))
                               nil)
                  (tagbody
                    (setf ix (f2cl-lib:int-sub ix incx))
                    (setf (f2cl-lib:fref x-%data%
                                         (ix)
                                         ((1 *))
                                         x-%offset%)
                          (-
                           (f2cl-lib:fref x-%data%
                                          (ix)
                                          ((1 *))
                                          x-%offset%)
                           (* temp
                              (f2cl-lib:fref ap-%data%
                                             (k)
                                             ((1 *))
                                             ap-%offset%))))))))
            (setf jx (f2cl-lib:int-sub jx incx))
            (setf kk (f2cl-lib:int-sub kk j)))))))
    (t
     (setf kk 1)
     (cond
       ((= incx 1)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (cond
              ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
```

```
            (if nounit
                (setf (f2cl-lib:fref x-%data%
                                     (j)
                                     ((1 *))
                                     x-%offset%)
                      (/
                       (f2cl-lib:fref x-%data%
                                      (j)
                                      ((1 *))
                                      x-%offset%)
                       (f2cl-lib:fref ap-%data%
                                      (kk)
                                      ((1 *))
                                      ap-%offset%))))
            (setf temp
                    (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
            (setf k (f2cl-lib:int-add kk 1))
            (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                           (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref x-%data%
                                     (i)
                                     ((1 *))
                                     x-%offset%)
                      (-
                       (f2cl-lib:fref x-%data%
                                      (i)
                                      ((1 *))
                                      x-%offset%)
                       (* temp
                          (f2cl-lib:fref ap-%data%
                                         (k)
                                         ((1 *))
                                         ap-%offset%))))
                (setf k (f2cl-lib:int-add k 1))))))
        (setf kk
                (f2cl-lib:int-add kk
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-sub n j)
                                   1))))))
    (t
     (setf jx kx)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (cond
           ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
            (if nounit
                (setf (f2cl-lib:fref x-%data%
```

```
                                        (jx)
                                        ((1 *))
                                        x-%offset%)
                                (/
                                 (f2cl-lib:fref x-%data%
                                                (jx)
                                                ((1 *))
                                                x-%offset%)
                                 (f2cl-lib:fref ap-%data%
                                                (kk)
                                                ((1 *))
                                                ap-%offset%))))
                    (setf temp
                            (f2cl-lib:fref x-%data%
                                           (jx)
                                           ((1 *))
                                           x-%offset%))
                    (setf ix jx)
                    (f2cl-lib:fdo (k (f2cl-lib:int-add kk 1)
                                   (f2cl-lib:int-add k 1))
                                  ((> k
                                      (f2cl-lib:int-add kk
                                                        n
                                                        (f2cl-lib:int-sub
                                                         j)))
                                   nil)
                      (tagbody
                        (setf ix (f2cl-lib:int-add ix incx))
                        (setf (f2cl-lib:fref x-%data%
                                             (ix)
                                             ((1 *))
                                             x-%offset%)
                                (-
                                 (f2cl-lib:fref x-%data%
                                                (ix)
                                                ((1 *))
                                                x-%offset%)
                                 (* temp
                                    (f2cl-lib:fref ap-%data%
                                                   (k)
                                                   ((1 *))
                                                   ap-%offset%))))))))
                (setf jx (f2cl-lib:int-add jx incx))
                (setf kk
                        (f2cl-lib:int-add kk
                                          (f2cl-lib:int-add
                                           (f2cl-lib:int-sub n j)
                                           1))))))))))
            (t
             (cond
```

```
((char-equal uplo #\U)
 (setf kk 1)
 (cond
   ((= incx 1)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp
                (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
        (setf k kk)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i
                          (f2cl-lib:int-add j
                                            (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf temp
                    (- temp
                       (*
                        (f2cl-lib:fref ap-%data%
                                       (k)
                                       ((1 *))
                                       ap-%offset%)
                        (f2cl-lib:fref x-%data%
                                       (i)
                                       ((1 *))
                                       x-%offset%))))
            (setf k (f2cl-lib:int-add k 1))))
        (if nounit
            (setf temp
                    (/ temp
                       (f2cl-lib:fref ap-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add kk j)
                                        1))
                                      ((1 *))
                                      ap-%offset%))))
        (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                temp)
        (setf kk (f2cl-lib:int-add kk j)))))
   (t
    (setf jx kx)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp
                (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
        (setf ix kx)
        (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                      ((> k
```

```
                                (f2cl-lib:int-add kk
                                                  j
                                                  (f2cl-lib:int-sub 2)))
                            nil)
                (tagbody
                  (setf temp
                          (- temp
                             (*
                              (f2cl-lib:fref ap-%data%
                                             (k)
                                             ((1 *))
                                             ap-%offset%)
                              (f2cl-lib:fref x-%data%
                                             (ix)
                                             ((1 *))
                                             x-%offset%))))
                  (setf ix (f2cl-lib:int-add ix incx))))
              (if nounit
                  (setf temp
                          (/ temp
                             (f2cl-lib:fref ap-%data%
                                            ((f2cl-lib:int-sub
                                              (f2cl-lib:int-add kk j)
                                              1))
                                            ((1 *))
                                            ap-%offset%))))
              (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                      temp)
              (setf jx (f2cl-lib:int-add jx incx))
              (setf kk (f2cl-lib:int-add kk j)))))))
      (t
       (setf kk (the fixnum (truncate (* n (+ n 1)) 2)))
       (cond
         ((= incx 1)
          (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                        ((> j 1) nil)
            (tagbody
              (setf temp
                      (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
              (setf k kk)
              (f2cl-lib:fdo (i n
                              (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                            ((> i (f2cl-lib:int-add j 1)) nil)
                (tagbody
                  (setf temp
                          (- temp
                             (*
                              (f2cl-lib:fref ap-%data%
                                             (k)
                                             ((1 *))
```

```
                                        ap-%offset%)
                        (f2cl-lib:fref x-%data%
                                        (i)
                                        ((1 *))
                                        x-%offset%))))
            (setf k (f2cl-lib:int-sub k 1))))
        (if nounit
            (setf temp
                    (/ temp
                       (f2cl-lib:fref ap-%data%
                                      ((f2cl-lib:int-add
                                        (f2cl-lib:int-sub kk n)
                                        j))
                                      ((1 *))
                                      ap-%offset%))))
        (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                temp)
        (setf kk
                (f2cl-lib:int-sub kk
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-sub n j)
                                   1))))))
    (t
     (setf kx
             (f2cl-lib:int-add kx
                               (f2cl-lib:int-mul
                                (f2cl-lib:int-sub n 1)
                                incx)))
     (setf jx kx)
     (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   ((> j 1) nil)
       (tagbody
         (setf temp
                 (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
         (setf ix kx)
         (f2cl-lib:fdo (k kk
                        (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                       ((> k
                           (f2cl-lib:int-add kk
                             (f2cl-lib:int-sub
                              (f2cl-lib:int-add n
                               (f2cl-lib:int-sub
                                (f2cl-lib:int-add
                                  j
                                  1))))))
                        nil)
           (tagbody
             (setf temp
                     (- temp
                        (*
```

```
                                    (f2cl-lib:fref ap-%data%
                                                   (k)
                                                   ((1 *))
                                                   ap-%offset%)
                                    (f2cl-lib:fref x-%data%
                                                   (ix)
                                                   ((1 *))
                                                   x-%offset%))))
                        (setf ix (f2cl-lib:int-sub ix incx))))
                    (if nounit
                        (setf temp
                                (/ temp
                                   (f2cl-lib:fref ap-%data%
                                                  ((f2cl-lib:int-add
                                                    (f2cl-lib:int-sub kk n)
                                                    j))
                                                  ((1 *))
                                                  ap-%offset%))))
                    (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                            temp)
                    (setf jx (f2cl-lib:int-sub jx incx))
                    (setf kk
                            (f2cl-lib:int-sub kk
                                              (f2cl-lib:int-add
                                               (f2cl-lib:int-sub n j)
                                               1)))))))))))
 end_label
        (return (values nil nil nil nil nil nil nil))))))
```

———-

11.15 dtrmv - matrix-vector $x = Ax$ or $x = A^T x$

11.15.1 Axiom unit tests

— dtrmv.input —

```
)set break resume
)sys rm -f dtrmv.output
)spool dtrmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

11.15.2 Axiom help page

— dtrmv.help —

```
==========================================================================
dtrmv examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

 NAME
      DTRMV - perform one of the matrix-vector operations   x :=
      A*x, or x := A'*x,

 SYNOPSIS
      SUBROUTINE DTRMV ( UPLO, TRANS, DIAG, N, A, LDA, X, INCX )

          INTEGER      INCX, LDA, N

          CHARACTER*1  DIAG, TRANS, UPLO

          DOUBLE       PRECISION A( LDA, * ), X( * )

 PURPOSE
      DTRMV  performs one of the matrix-vector operations

      where x is an n element vector and  A is an n by n unit, or
      non-unit, upper or lower triangular matrix.

 PARAMETERS
      UPLO   - CHARACTER*1.
             On entry, UPLO specifies whether the matrix is an
             upper or lower triangular matrix as follows:

             UPLO = 'U' or 'u'   A is an upper triangular matrix.

             UPLO = 'L' or 'l'   A is a lower triangular matrix.

             Unchanged on exit.

      TRANS  - CHARACTER*1.
             On entry, TRANS specifies the operation to be per-
             formed as follows:
```

```
        TRANS = 'N' or 'n'   x := A*x.

        TRANS = 'T' or 't'   x := A'*x.

        TRANS = 'C' or 'c'   x := A'*x.

        Unchanged on exit.

DIAG    - CHARACTER*1.
        On entry, DIAG specifies whether or not A is unit
        triangular as follows:

        DIAG = 'U' or 'u'   A is assumed to be unit triangu-
        lar.

        DIAG = 'N' or 'n'   A is not assumed to be unit tri-
        angular.

        Unchanged on exit.

N       - INTEGER.
        On entry, N specifies the order of the matrix A.  N
        must be at least zero.  Unchanged on exit.

A       - DOUBLE PRECISION array of DIMENSION ( LDA, n ).
        Before entry with  UPLO = 'U' or 'u', the leading n
        by n upper triangular part of the array A must con-
        tain the upper triangular matrix and the strictly
        lower triangular part of A is not referenced.  Before
        entry with UPLO = 'L' or 'l', the leading n by n
        lower triangular part of the array A must contain the
        lower triangular matrix and the strictly upper tri-
        angular part of A is not referenced.  Note that when
        DIAG = 'U' or 'u', the diagonal elements of A are not
        referenced either, but are assumed to be unity.
        Unchanged on exit.

LDA     - INTEGER.
        On entry, LDA specifies the first dimension of A as
        declared in the calling (sub) program. LDA must be at
        least max( 1, n ).  Unchanged on exit.

X       - DOUBLE PRECISION array of dimension at least
        ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
        incremented array X must contain the n element vector
        x. On exit, X is overwritten with the transformed
        vector x.

INCX    - INTEGER.
        On entry, INCX specifies the increment for the ele-
```

```
ments of X. INCX must not be zero.  Unchanged on
exit.
```

———-

11.15.3 fortran code

— dtrmv.f —

```
*  ==================================================================
      SUBROUTINE DTRMV(UPLO,TRANS,DIAG,N,A,LDA,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,LDA,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A(LDA,*),X(*)
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION ZERO
      PARAMETER (ZERO=0.0D+0)
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP
      INTEGER I,INFO,IX,J,JX,KX
      LOGICAL NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*
```

```
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 6
      ELSE IF (INCX.EQ.0) THEN
          INFO = 8
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('DTRMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX  too small for descending loops.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x := A*x.
*
          IF (LSAME(UPLO,'U')) THEN
              IF (INCX.EQ.1) THEN
                  DO 20 J = 1,N
                      IF (X(J).NE.ZERO) THEN
                          TEMP = X(J)
                          DO 10 I = 1,J - 1
```

```
                            X(I) = X(I) + TEMP*A(I,J)
   10                    CONTINUE
                         IF (NOUNIT) X(J) = X(J)*A(J,J)
                      END IF
   20             CONTINUE
               ELSE
                  JX = KX
                  DO 40 J = 1,N
                      IF (X(JX).NE.ZERO) THEN
                         TEMP = X(JX)
                         IX = KX
                         DO 30 I = 1,J - 1
                            X(IX) = X(IX) + TEMP*A(I,J)
                            IX = IX + INCX
   30                    CONTINUE
                         IF (NOUNIT) X(JX) = X(JX)*A(J,J)
                      END IF
                      JX = JX + INCX
   40             CONTINUE
               END IF
            ELSE
               IF (INCX.EQ.1) THEN
                  DO 60 J = N,1,-1
                      IF (X(J).NE.ZERO) THEN
                         TEMP = X(J)
                         DO 50 I = N,J + 1,-1
                            X(I) = X(I) + TEMP*A(I,J)
   50                    CONTINUE
                         IF (NOUNIT) X(J) = X(J)*A(J,J)
                      END IF
   60             CONTINUE
               ELSE
                  KX = KX + (N-1)*INCX
                  JX = KX
                  DO 80 J = N,1,-1
                      IF (X(JX).NE.ZERO) THEN
                         TEMP = X(JX)
                         IX = KX
                         DO 70 I = N,J + 1,-1
                            X(IX) = X(IX) + TEMP*A(I,J)
                            IX = IX - INCX
   70                    CONTINUE
                         IF (NOUNIT) X(JX) = X(JX)*A(J,J)
                      END IF
                      JX = JX - INCX
   80             CONTINUE
               END IF
            END IF
         ELSE
*
```

```
*        Form  x := A**T*x.
*
          IF (LSAME(UPLO,'U')) THEN
              IF (INCX.EQ.1) THEN
                  DO 100 J = N,1,-1
                      TEMP = X(J)
                      IF (NOUNIT) TEMP = TEMP*A(J,J)
                      DO 90 I = J - 1,1,-1
                          TEMP = TEMP + A(I,J)*X(I)
   90                 CONTINUE
                      X(J) = TEMP
  100             CONTINUE
              ELSE
                  JX = KX + (N-1)*INCX
                  DO 120 J = N,1,-1
                      TEMP = X(JX)
                      IX = JX
                      IF (NOUNIT) TEMP = TEMP*A(J,J)
                      DO 110 I = J - 1,1,-1
                          IX = IX - INCX
                          TEMP = TEMP + A(I,J)*X(IX)
  110                 CONTINUE
                      X(JX) = TEMP
                      JX = JX - INCX
  120             CONTINUE
              END IF
          ELSE
              IF (INCX.EQ.1) THEN
                  DO 140 J = 1,N
                      TEMP = X(J)
                      IF (NOUNIT) TEMP = TEMP*A(J,J)
                      DO 130 I = J + 1,N
                          TEMP = TEMP + A(I,J)*X(I)
  130                 CONTINUE
                      X(J) = TEMP
  140             CONTINUE
              ELSE
                  JX = KX
                  DO 160 J = 1,N
                      TEMP = X(JX)
                      IX = JX
                      IF (NOUNIT) TEMP = TEMP*A(J,J)
                      DO 150 I = J + 1,N
                          IX = IX + INCX
                          TEMP = TEMP + A(I,J)*X(IX)
  150                 CONTINUE
                      X(JX) = TEMP
                      JX = JX + INCX
  160             CONTINUE
              END IF
```

```
         END IF
      END IF
*
      RETURN
*
*     End of DTRMV .
*
      END
```

———-

11.15.4 lisp code

— BLAS 2 dtrmv —

```
(let* ((zero 0.0))
  (declare (type (double-float 0.0 0.0) zero))
  (defun dtrmv (uplo trans diag n a lda x incx)
    (declare (type (simple-array double-float (*)) x a)
             (type fixnum incx lda n)
             (type character diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a double-float a-%data% a-%offset%)
         (x double-float x-%data% x-%offset%))
      (prog ((nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0) (kx 0) (temp 0.0))
        (declare (type (member t nil) nounit)
                 (type fixnum i info ix j jx kx)
                 (type (double-float) temp))
        (setf info 0)
        (cond
          ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
           (setf info 1))
          ((and (not (char-equal trans #\N))
                (not (char-equal trans #\T))
                (not (char-equal trans #\C)))
           (setf info 2))
          ((and (not (char-equal diag #\U)) (not (char-equal diag #\N)))
           (setf info 3))
          ((< n 0)
           (setf info 4))
          ((< lda (max (the fixnum 1) (the fixnum n)))
           (setf info 6))
          ((= incx 0)
           (setf info 8)))
```

```
(cond
  ((/= info 0)
   (error
    " ** On entry to ~a parameter number ~a had an illegal value~%"
    "DTRMV" info)
   (go end_label)))
(if (= n 0) (go end_label))
(setf nounit (char-equal diag #\N))
(cond
  ((<= incx 0)
   (setf kx
           (f2cl-lib:int-sub 1
                             (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                               incx))))
  ((/= incx 1)
   (setf kx 1)))
(cond
  ((char-equal trans #\N)
   (cond
     ((char-equal uplo #\U)
      (cond
        ((= incx 1)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (cond
               ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
                (setf temp
                        (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i
                                  (f2cl-lib:int-add j
                                                    (f2cl-lib:int-sub
                                                     1)))
                               nil)
                  (tagbody
                    (setf (f2cl-lib:fref x-%data%
                                         (i)
                                         ((1 *))
                                         x-%offset%)
                          (+
                           (f2cl-lib:fref x-%data%
                                          (i)
                                          ((1 *))
                                          x-%offset%)
                           (* temp
                              (f2cl-lib:fref a-%data%
                                             (i j)
                                             ((1 lda) (1 *))
                                             a-%offset%))))))
```

```
            (if nounit
                (setf (f2cl-lib:fref x-%data%
                                     (j)
                                     ((1 *))
                                     x-%offset%)
                      (*
                       (f2cl-lib:fref x-%data%
                                      (j)
                                      ((1 *))
                                      x-%offset%)
                       (f2cl-lib:fref a-%data%
                                      (j j)
                                      ((1 lda) (1 *))
                                      a-%offset%)))))))))
        (t
         (setf jx kx)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (cond
               ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
                (setf temp
                        (f2cl-lib:fref x-%data%
                                       (jx)
                                       ((1 *))
                                       x-%offset%))
                (setf ix kx)
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i
                                  (f2cl-lib:int-add j
                                                    (f2cl-lib:int-sub
                                                     1)))
                               nil)
                  (tagbody
                    (setf (f2cl-lib:fref x-%data%
                                         (ix)
                                         ((1 *))
                                         x-%offset%)
                          (+
                           (f2cl-lib:fref x-%data%
                                          (ix)
                                          ((1 *))
                                          x-%offset%)
                           (* temp
                              (f2cl-lib:fref a-%data%
                                             (i j)
                                             ((1 lda) (1 *))
                                             a-%offset%))))
                    (setf ix (f2cl-lib:int-add ix incx))))
                (if nounit
```

```
                    (setf (f2cl-lib:fref x-%data%
                                         (jx)
                                         ((1 *))
                                         x-%offset%)
                          (*
                           (f2cl-lib:fref x-%data%
                                          (jx)
                                          ((1 *))
                                          x-%offset%)
                           (f2cl-lib:fref a-%data%
                                          (j j)
                                          ((1 lda) (1 *))
                                          a-%offset%))))))
            (setf jx (f2cl-lib:int-add jx incx)))))))
        (t
         (cond
           ((= incx 1)
            (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                          ((> j 1) nil)
              (tagbody
                (cond
                  ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
                   (setf temp
                           (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                   (f2cl-lib:fdo (i n
                                  (f2cl-lib:int-add i
                                                    (f2cl-lib:int-sub 1)))
                                 ((> i (f2cl-lib:int-add j 1)) nil)
                     (tagbody
                       (setf (f2cl-lib:fref x-%data%
                                            (i)
                                            ((1 *))
                                            x-%offset%)
                             (+
                              (f2cl-lib:fref x-%data%
                                             (i)
                                             ((1 *))
                                             x-%offset%)
                              (* temp
                                 (f2cl-lib:fref a-%data%
                                                (i j)
                                                ((1 lda) (1 *))
                                                a-%offset%))))))
                   (if nounit
                       (setf (f2cl-lib:fref x-%data%
                                            (j)
                                            ((1 *))
                                            x-%offset%)
                             (*
                              (f2cl-lib:fref x-%data%
```

```
                                        (j)
                                        ((1 *))
                                        x-%offset%)
                        (f2cl-lib:fref a-%data%
                                        (j j)
                                        ((1 lda) (1 *))
                                        a-%offset%)))))))))
(t
 (setf kx
         (f2cl-lib:int-add kx
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-sub n 1)
                            incx)))
 (setf jx kx)
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (cond
       ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
        (setf temp
                (f2cl-lib:fref x-%data%
                               (jx)
                               ((1 *))
                               x-%offset%))
        (setf ix kx)
        (f2cl-lib:fdo (i n
                       (f2cl-lib:int-add i
                                         (f2cl-lib:int-sub 1)))
                      ((> i (f2cl-lib:int-add j 1)) nil)
          (tagbody
            (setf (f2cl-lib:fref x-%data%
                                 (ix)
                                 ((1 *))
                                 x-%offset%)
                  (+
                   (f2cl-lib:fref x-%data%
                                  (ix)
                                  ((1 *))
                                  x-%offset%)
                   (* temp
                      (f2cl-lib:fref a-%data%
                                     (i j)
                                     ((1 lda) (1 *))
                                     a-%offset%))))
            (setf ix (f2cl-lib:int-sub ix incx))))
        (if nounit
            (setf (f2cl-lib:fref x-%data%
                                 (jx)
                                 ((1 *))
                                 x-%offset%)
```

```
                                (*
                                 (f2cl-lib:fref x-%data%
                                                (jx)
                                                ((1 *))
                                                x-%offset%)
                                 (f2cl-lib:fref a-%data%
                                                (j j)
                                                ((1 lda) (1 *))
                                                a-%offset%)))))))
                (setf jx (f2cl-lib:int-sub jx incx)))))))))
        (t
         (cond
           ((char-equal uplo #\U)
            (cond
              ((= incx 1)
               (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                             ((> j 1) nil)
                 (tagbody
                   (setf temp
                           (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                   (if nounit
                       (setf temp
                               (* temp
                                  (f2cl-lib:fref a-%data%
                                                 (j j)
                                                 ((1 lda) (1 *))
                                                 a-%offset%))))
                   (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                                  (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                                 ((> i 1) nil)
                     (tagbody
                       (setf temp
                               (+ temp
                                  (*
                                   (f2cl-lib:fref a-%data%
                                                  (i j)
                                                  ((1 lda) (1 *))
                                                  a-%offset%)
                                   (f2cl-lib:fref x-%data%
                                                  (i)
                                                  ((1 *))
                                                  x-%offset%))))))
                   (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                           temp))))
              (t
               (setf jx
                       (f2cl-lib:int-add kx
                                         (f2cl-lib:int-mul
                                          (f2cl-lib:int-sub n 1)
                                          incx)))
```

```
      (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                    ((> j 1) nil)
        (tagbody
          (setf temp
                  (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
          (setf ix jx)
          (if nounit
              (setf temp
                      (* temp
                         (f2cl-lib:fref a-%data%
                                        (j j)
                                        ((1 lda) (1 *))
                                        a-%offset%))))
          (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                         (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                        ((> i 1) nil)
            (tagbody
              (setf ix (f2cl-lib:int-sub ix incx))
              (setf temp
                      (+ temp
                         (*
                          (f2cl-lib:fref a-%data%
                                         (i j)
                                         ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref x-%data%
                                         (ix)
                                         ((1 *))
                                         x-%offset%))))))
          (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                  temp)
          (setf jx (f2cl-lib:int-sub jx incx)))))))
(t
 (cond
   ((= incx 1)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp
                (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
        (if nounit
            (setf temp
                    (* temp
                       (f2cl-lib:fref a-%data%
                                      (j j)
                                      ((1 lda) (1 *))
                                      a-%offset%))))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                       (f2cl-lib:int-add i 1))
                      ((> i n) nil)
```

```
                    (tagbody
                      (setf temp
                              (+ temp
                                 (*
                                  (f2cl-lib:fref a-%data%
                                                 (i j)
                                                 ((1 lda) (1 *))
                                                 a-%offset%)
                                  (f2cl-lib:fref x-%data%
                                                 (i)
                                                 ((1 *))
                                                 x-%offset%))))))
                  (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                          temp))))
              (t
               (setf jx kx)
               (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                             ((> j n) nil)
                 (tagbody
                   (setf temp
                           (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
                   (setf ix jx)
                   (if nounit
                       (setf temp
                               (* temp
                                  (f2cl-lib:fref a-%data%
                                                 (j j)
                                                 ((1 lda) (1 *))
                                                 a-%offset%))))
                   (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                                    (f2cl-lib:int-add i 1))
                                 ((> i n) nil)
                     (tagbody
                       (setf ix (f2cl-lib:int-add ix incx))
                       (setf temp
                               (+ temp
                                  (*
                                   (f2cl-lib:fref a-%data%
                                                  (i j)
                                                  ((1 lda) (1 *))
                                                  a-%offset%)
                                   (f2cl-lib:fref x-%data%
                                                  (ix)
                                                  ((1 *))
                                                  x-%offset%))))))
                   (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                           temp)
                   (setf jx (f2cl-lib:int-add jx incx)))))))))
end_label
       (return (values nil nil nil nil nil nil nil nil))))))
```

———-

11.16 dtrsv - solve system of equations $Ax = b$ or $A' = b$

11.16.1 Axiom unit tests

— **dtrsv.input** —

```
)set break resume
)sys rm -f dtrsv.output
)spool dtrsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

11.16.2 Axiom help page

— **dtrsv.help** —

```
======================================================================
dtrsv examples
======================================================================

======================================================================
Man Page Details
======================================================================

 NAME
      DTRSV - solve one of the systems of equations   A*x = b, or
      A'*x = b,

 SYNOPSIS
      SUBROUTINE DTRSV ( UPLO, TRANS, DIAG, N, A, LDA, X, INCX )

          INTEGER      INCX, LDA, N

          CHARACTER*1  DIAG, TRANS, UPLO
```

```
      DOUBLE       PRECISION A( LDA, * ), X( * )

PURPOSE
     DTRSV  solves one of the systems of equations

     where b and x are n element vectors and A is an n by n unit,
     or non-unit, upper or lower triangular matrix.

     No test for singularity or near-singularity is included in
     this routine. Such tests must be performed before calling
     this routine.

PARAMETERS
     UPLO   - CHARACTER*1.
              On entry, UPLO specifies whether the matrix is an
              upper or lower triangular matrix as follows:

              UPLO = 'U' or 'u'   A is an upper triangular matrix.

              UPLO = 'L' or 'l'   A is a lower triangular matrix.

              Unchanged on exit.

     TRANS  - CHARACTER*1.
              On entry, TRANS specifies the equations to be solved
              as follows:

              TRANS = 'N' or 'n'   A*x = b.

              TRANS = 'T' or 't'   A'*x = b.

              TRANS = 'C' or 'c'   A'*x = b.

              Unchanged on exit.

     DIAG   - CHARACTER*1.
              On entry, DIAG specifies whether or not A is unit
              triangular as follows:

              DIAG = 'U' or 'u'   A is assumed to be unit

              triangular.

              DIAG = 'N' or 'n'   A is not assumed to be unit tri-
              angular.

              Unchanged on exit.

     N      - INTEGER.
```

```
             On entry, N specifies the order of the matrix A.  N
             must be at least zero.  Unchanged on exit.

      A      - DOUBLE PRECISION array of DIMENSION ( LDA, n ).
             Before entry with  UPLO = 'U' or 'u', the leading n
             by n upper triangular part of the array A must con-
             tain the upper triangular matrix and the strictly
             lower triangular part of A is not referenced.  Before
             entry with UPLO = 'L' or 'l', the leading n by n
             lower triangular part of the array A must contain the
             lower triangular matrix and the strictly upper tri-
             angular part of A is not referenced.  Note that when
             DIAG = 'U' or 'u', the diagonal elements of A are not
             referenced either, but are assumed to be unity.
             Unchanged on exit.

      LDA    - INTEGER.
             On entry, LDA specifies the first dimension of A as
             declared in the calling (sub) program. LDA must be at
             least max( 1, n ).  Unchanged on exit.

      X      - DOUBLE PRECISION array of dimension at least
             ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
             incremented array X must contain the n element
             right-hand side vector b. On exit, X is overwritten
             with the solution vector x.

      INCX   - INTEGER.
             On entry, INCX specifies the increment for the ele-
             ments of X. INCX must not be zero.  Unchanged on
             exit.
```

———-

11.16.3 fortran code

— dtrsv.f —

```
*  ====================================================================
      SUBROUTINE DTRSV(UPLO,TRANS,DIAG,N,A,LDA,X,INCX)
*
*  -- Reference BLAS level1 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
```

```
      INTEGER INCX,LDA,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A(LDA,*),X(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION ZERO
      PARAMETER (ZERO=0.0D+0)
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP
      INTEGER I,INFO,IX,J,JX,KX
      LOGICAL NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 6
      ELSE IF (INCX.EQ.0) THEN
          INFO = 8
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('DTRSV ',INFO)
          RETURN
      END IF
```

```
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX  too small for descending loops.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x := inv( A )*x.
*
          IF (LSAME(UPLO,'U')) THEN
              IF (INCX.EQ.1) THEN
                  DO 20 J = N,1,-1
                      IF (X(J).NE.ZERO) THEN
                          IF (NOUNIT) X(J) = X(J)/A(J,J)
                          TEMP = X(J)
                          DO 10 I = J - 1,1,-1
                              X(I) = X(I) - TEMP*A(I,J)
   10                     CONTINUE
                      END IF
   20             CONTINUE
              ELSE
                  JX = KX + (N-1)*INCX
                  DO 40 J = N,1,-1
                      IF (X(JX).NE.ZERO) THEN
                          IF (NOUNIT) X(JX) = X(JX)/A(J,J)
                          TEMP = X(JX)
                          IX = JX
                          DO 30 I = J - 1,1,-1
                              IX = IX - INCX
                              X(IX) = X(IX) - TEMP*A(I,J)
   30                     CONTINUE
                      END IF
                      JX = JX - INCX
   40             CONTINUE
              END IF
          ELSE
```

```
            IF (INCX.EQ.1) THEN
                DO 60 J = 1,N
                    IF (X(J).NE.ZERO) THEN
                        IF (NOUNIT) X(J) = X(J)/A(J,J)
                        TEMP = X(J)
                        DO 50 I = J + 1,N
                            X(I) = X(I) - TEMP*A(I,J)
   50                   CONTINUE
                    END IF
   60           CONTINUE
            ELSE
                JX = KX
                DO 80 J = 1,N
                    IF (X(JX).NE.ZERO) THEN
                        IF (NOUNIT) X(JX) = X(JX)/A(J,J)
                        TEMP = X(JX)
                        IX = JX
                        DO 70 I = J + 1,N
                            IX = IX + INCX
                            X(IX) = X(IX) - TEMP*A(I,J)
   70                   CONTINUE
                    END IF
                    JX = JX + INCX
   80           CONTINUE
            END IF
        END IF
    ELSE
*
*       Form  x := inv( A**T )*x.
*
        IF (LSAME(UPLO,'U')) THEN
            IF (INCX.EQ.1) THEN
                DO 100 J = 1,N
                    TEMP = X(J)
                    DO 90 I = 1,J - 1
                        TEMP = TEMP - A(I,J)*X(I)
   90               CONTINUE
                    IF (NOUNIT) TEMP = TEMP/A(J,J)
                    X(J) = TEMP
  100           CONTINUE
            ELSE
                JX = KX
                DO 120 J = 1,N
                    TEMP = X(JX)
                    IX = KX
                    DO 110 I = 1,J - 1
                        TEMP = TEMP - A(I,J)*X(IX)
                        IX = IX + INCX
  110               CONTINUE
                    IF (NOUNIT) TEMP = TEMP/A(J,J)
```

```
                    X(JX) = TEMP
                    JX = JX + INCX
 120            CONTINUE
            END IF
        ELSE
            IF (INCX.EQ.1) THEN
                DO 140 J = N,1,-1
                    TEMP = X(J)
                    DO 130 I = N,J + 1,-1
                        TEMP = TEMP - A(I,J)*X(I)
 130                CONTINUE
                    IF (NOUNIT) TEMP = TEMP/A(J,J)
                    X(J) = TEMP
 140            CONTINUE
            ELSE
                KX = KX + (N-1)*INCX
                JX = KX
                DO 160 J = N,1,-1
                    TEMP = X(JX)
                    IX = KX
                    DO 150 I = N,J + 1,-1
                        TEMP = TEMP - A(I,J)*X(IX)
                        IX = IX - INCX
 150                CONTINUE
                    IF (NOUNIT) TEMP = TEMP/A(J,J)
                    X(JX) = TEMP
                    JX = JX - INCX
 160            CONTINUE
            END IF
        END IF
    END IF
*
    RETURN
*
*   End of DTRSV .
*
    END
```

———-

11.16.4 lisp code

— BLAS 2 dtrsv —

```
(let* ((zero 0.0))
  (declare (type (double-float 0.0 0.0) zero))
  (defun dtrsv (uplo trans diag n a lda x incx)
```

```
(declare (type (simple-array double-float (*)) x a)
         (type fixnum incx lda n)
         (type character diag trans uplo))
(f2cl-lib:with-multi-array-data
    ((uplo character uplo-%data% uplo-%offset%)
     (trans character trans-%data% trans-%offset%)
     (diag character diag-%data% diag-%offset%)
     (a double-float a-%data% a-%offset%)
     (x double-float x-%data% x-%offset%))
  (prog ((nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0) (kx 0) (temp 0.0))
    (declare (type (member t nil) nounit)
             (type fixnum i info ix j jx kx)
             (type (double-float) temp))
    (setf info 0)
    (cond
      ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
       (setf info 1))
      ((and (not (char-equal trans #\N))
            (not (char-equal trans #\T))
            (not (char-equal trans #\C)))
       (setf info 2))
      ((and (not (char-equal diag #\U)) (not (char-equal diag #\N)))
       (setf info 3))
      ((< n 0)
       (setf info 4))
      ((< lda (max (the fixnum 1) (the fixnum n)))
       (setf info 6))
      ((= incx 0)
       (setf info 8)))
    (cond
      ((/= info 0)
       (error
        " ** On entry to ~a parameter number ~a had an illegal value~%"
        "DTRSV" info)
       (go end_label)))
    (if (= n 0) (go end_label))
    (setf nounit (char-equal diag #\N))
    (cond
      ((<= incx 0)
       (setf kx
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                   incx))))
      ((/= incx 1)
       (setf kx 1)))
    (cond
      ((char-equal trans #\N)
       (cond
         ((char-equal uplo #\U)
          (cond
```

```
((= incx 1)
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (cond
       ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
        (if nounit
            (setf (f2cl-lib:fref x-%data%
                                 (j)
                                 ((1 *))
                                 x-%offset%)
                  (/
                   (f2cl-lib:fref x-%data%
                                  (j)
                                  ((1 *))
                                  x-%offset%)
                   (f2cl-lib:fref a-%data%
                                  (j j)
                                  ((1 lda) (1 *))
                                  a-%offset%))))
        (setf temp
                (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
        (f2cl-lib:fdo (i
                       (f2cl-lib:int-add j
                                         (f2cl-lib:int-sub 1))
                       (f2cl-lib:int-add i
                                         (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (setf (f2cl-lib:fref x-%data%
                                 (i)
                                 ((1 *))
                                 x-%offset%)
                  (-
                   (f2cl-lib:fref x-%data%
                                  (i)
                                  ((1 *))
                                  x-%offset%)
                   (* temp
                      (f2cl-lib:fref a-%data%
                                     (i j)
                                     ((1 lda) (1 *))
                                     a-%offset%))))))))))
(t
 (setf jx
         (f2cl-lib:int-add kx
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-sub n 1)
                            incx)))
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
```

```
                  ((> j 1) nil)
          (tagbody
            (cond
              ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
               (if nounit
                   (setf (f2cl-lib:fref x-%data%
                                        (jx)
                                        ((1 *))
                                        x-%offset%)
                         (/
                          (f2cl-lib:fref x-%data%
                                         (jx)
                                         ((1 *))
                                         x-%offset%)
                          (f2cl-lib:fref a-%data%
                                         (j j)
                                         ((1 lda) (1 *))
                                         a-%offset%))))
               (setf temp
                       (f2cl-lib:fref x-%data%
                                      (jx)
                                      ((1 *))
                                      x-%offset%))
               (setf ix jx)
               (f2cl-lib:fdo (i
                              (f2cl-lib:int-add j
                                                (f2cl-lib:int-sub 1))
                              (f2cl-lib:int-add i
                                                (f2cl-lib:int-sub 1)))
                             ((> i 1) nil)
                 (tagbody
                   (setf ix (f2cl-lib:int-sub ix incx))
                   (setf (f2cl-lib:fref x-%data%
                                        (ix)
                                        ((1 *))
                                        x-%offset%)
                         (-
                          (f2cl-lib:fref x-%data%
                                         (ix)
                                         ((1 *))
                                         x-%offset%)
                          (* temp
                             (f2cl-lib:fref a-%data%
                                            (i j)
                                            ((1 lda) (1 *))
                                            a-%offset%))))))))
            (setf jx (f2cl-lib:int-sub jx incx)))))))
(t
 (cond
   ((= incx 1)
```

```
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (cond
      ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
       (if nounit
           (setf (f2cl-lib:fref x-%data%
                                (j)
                                ((1 *))
                                x-%offset%)
                 (/
                  (f2cl-lib:fref x-%data%
                                 (j)
                                 ((1 *))
                                 x-%offset%)
                  (f2cl-lib:fref a-%data%
                                 (j j)
                                 ((1 lda) (1 *))
                                 a-%offset%))))
       (setf temp
               (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
       (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                        (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref x-%data%
                                (i)
                                ((1 *))
                                x-%offset%)
                 (-
                  (f2cl-lib:fref x-%data%
                                 (i)
                                 ((1 *))
                                 x-%offset%)
                  (* temp
                     (f2cl-lib:fref a-%data%
                                    (i j)
                                    ((1 lda) (1 *))
                                    a-%offset%))))))))))
(t
 (setf jx kx)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
        (if nounit
            (setf (f2cl-lib:fref x-%data%
                                 (jx)
                                 ((1 *))
```

```
                                        x-%offset%)
                                (/
                                 (f2cl-lib:fref x-%data%
                                                (jx)
                                                ((1 *))
                                                x-%offset%)
                                 (f2cl-lib:fref a-%data%
                                                (j j)
                                                ((1 lda) (1 *))
                                                a-%offset%))))
                (setf temp
                        (f2cl-lib:fref x-%data%
                                       (jx)
                                       ((1 *))
                                       x-%offset%))
                (setf ix jx)
                (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                               (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf ix (f2cl-lib:int-add ix incx))
                    (setf (f2cl-lib:fref x-%data%
                                         (ix)
                                         ((1 *))
                                         x-%offset%)
                                (-
                                 (f2cl-lib:fref x-%data%
                                                (ix)
                                                ((1 *))
                                                x-%offset%)
                                 (* temp
                                    (f2cl-lib:fref a-%data%
                                                   (i j)
                                                   ((1 lda) (1 *))
                                                   a-%offset%))))))))
            (setf jx (f2cl-lib:int-add jx incx)))))))))
(t
 (cond
   ((char-equal uplo #\U)
    (cond
      ((= incx 1)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (setf temp
                   (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i
                             (f2cl-lib:int-add j
                                               (f2cl-lib:int-sub 1)))
```

```
                    nil)
            (tagbody
              (setf temp
                      (- temp
                         (*
                          (f2cl-lib:fref a-%data%
                                         (i j)
                                         ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref x-%data%
                                         (i)
                                         ((1 *))
                                         x-%offset%))))))
            (if nounit
                (setf temp
                        (/ temp
                           (f2cl-lib:fref a-%data%
                                          (j j)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
            (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                    temp))))
        (t
         (setf jx kx)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf temp
                     (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
             (setf ix kx)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i
                               (f2cl-lib:int-add j
                                                 (f2cl-lib:int-sub 1)))
                            nil)
               (tagbody
                 (setf temp
                         (- temp
                            (*
                             (f2cl-lib:fref a-%data%
                                            (i j)
                                            ((1 lda) (1 *))
                                            a-%offset%)
                             (f2cl-lib:fref x-%data%
                                            (ix)
                                            ((1 *))
                                            x-%offset%))))
                 (setf ix (f2cl-lib:int-add ix incx))))
             (if nounit
                 (setf temp
```

```
                    (/ temp
                       (f2cl-lib:fref a-%data%
                                      (j j)
                                      ((1 lda) (1 *))
                                      a-%offset%))))
            (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                  temp)
            (setf jx (f2cl-lib:int-add jx incx)))))))
    (t
     (cond
       ((= incx 1)
        (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                      ((> j 1) nil)
          (tagbody
            (setf temp
                  (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
            (f2cl-lib:fdo (i n
                           (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                          ((> i (f2cl-lib:int-add j 1)) nil)
              (tagbody
                (setf temp
                        (- temp
                           (*
                            (f2cl-lib:fref a-%data%
                                           (i j)
                                           ((1 lda) (1 *))
                                           a-%offset%)
                            (f2cl-lib:fref x-%data%
                                           (i)
                                           ((1 *))
                                           x-%offset%))))))
            (if nounit
                (setf temp
                        (/ temp
                           (f2cl-lib:fref a-%data%
                                          (j j)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
            (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                  temp))))
       (t
        (setf kx
                (f2cl-lib:int-add kx
                                  (f2cl-lib:int-mul
                                   (f2cl-lib:int-sub n 1)
                                   incx)))
        (setf jx kx)
        (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                      ((> j 1) nil)
          (tagbody
```

```
                    (setf temp
                            (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
                    (setf ix kx)
                    (f2cl-lib:fdo (i n
                                   (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                                  ((> i (f2cl-lib:int-add j 1)) nil)
                      (tagbody
                        (setf temp
                                (- temp
                                   (*
                                    (f2cl-lib:fref a-%data%
                                                   (i j)
                                                   ((1 lda) (1 *))
                                                   a-%offset%)
                                    (f2cl-lib:fref x-%data%
                                                   (ix)
                                                   ((1 *))
                                                   x-%offset%))))
                        (setf ix (f2cl-lib:int-sub ix incx))))
                    (if nounit
                        (setf temp
                                (/ temp
                                   (f2cl-lib:fref a-%data%
                                                  (j j)
                                                  ((1 lda) (1 *))
                                                  a-%offset%))))
                    (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                            temp)
                    (setf jx (f2cl-lib:int-sub jx incx)))))))))
end_label
       (return (values nil nil nil nil nil nil nil nil))))))
```

———-

Chapter 12

BLAS Level 2 Complex Matrix-Vector Operations

12.1 cgbmv - band mat-vec $\alpha Ax + \beta y$ or $\alpha A^T + \beta y$ or $\alpha A^H x + \beta y$

12.1.1 Axiom unit tests

— **cgbmv.input** —

```
)set break resume
)sys rm -f cgbmv.output
)spool cgbmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

12.1.2 Axiom help page

— **cgbmv.help** —

```
======================================================================
cgbmv examples
```

```
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgbmv (TRANS, M, N, KL, KU, ALPHA, A, LDA, X, INCX, BETA, Y,
           INCY)
           CGBMV

Function/Subroutine Documentation
   subroutine cgbmv (character TRANS, integer M, integer N, integer KL,
       integer KU, complex ALPHA, complex, dimension(lda,*) A, integer LDA,
       complex, dimension(*) X, integer INCX, complex BETA, complex,
       dimension(*) Y, integer INCY)
       CGBMV

Purpose:
   CGBMV  performs one of the matrix-vector operations

     y := alpha*A*x + beta*y,   or   y := alpha*A**T*x + beta*y,   or

   y := alpha*A**H*x + beta*y,

   where alpha and beta are scalars, x and y are vectors and A is an
   m by n band matrix, with kl sub-diagonals and ku super-diagonals.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   On entry, TRANS specifies the operation to be performed as follows:

   TRANS = 'N' or 'n'   y := alpha*A*x + beta*y.

   TRANS = 'T' or 't'   y := alpha*A**T*x + beta*y.

   TRANS = 'C' or 'c'   y := alpha*A**H*x + beta*y.

 M
   M is INTEGER
   On entry, M specifies the number of rows of the matrix A.
   M must be at least zero.

 N
   N is INTEGER
   On entry, N specifies the number of columns of the matrix A.
   N must be at least zero.
```

```
KL
  KL is INTEGER
  On entry, KL specifies the number of sub-diagonals of the
  matrix A. KL must satisfy  0 .le. KL.

KU
  KU is INTEGER
  On entry, KU specifies the number of super-diagonals of the
  matrix A. KU must satisfy  0 .le. KU.

ALPHA
  ALPHA is COMPLEX
  On entry, ALPHA specifies the scalar alpha.

A
  A is COMPLEX array of DIMENSION ( LDA, n ).

  Before entry, the leading ( kl + ku + 1 ) by n part of th
  e
  array A must contain the matrix of coefficients, supplied
  column by column, with the leading diagonal of the matrix in
  row ( ku + 1 ) of the array, the first super-diagonal
  starting at position 2 in row ku, the first sub-diagonal
  starting at position 1 in row ( ku + 2 ), and so on.
  Elements in the array A that do not correspond to elements
  in the band matrix (such as the top left ku by ku triangle)
  are not referenced.

  The following program segment will transfer a band matrix
  from conventional full matrix storage to band storage:

       DO 20, J = 1, N
          K = KU + 1 - J
          DO 10, I = MAX( 1, J - KU ), MIN( M, J + KL )
             A( K + I, J ) = matrix( I, J )
    10    CONTINUE
    20 CONTINUE

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least ( kl + ku + 1 ).

X
  X is COMPLEX array of DIMENSION at least
  ( 1 + ( n - 1 )*abs( INCX ) ) when TRANS = 'N' or 'n'
  and at least
  ( 1 + ( m - 1 )*abs( INCX ) ) otherwise.
  Before entry, the incremented array X must contain the vector x.
```

```
 INCX
   INCX is INTEGER
   On entry, INCX specifies the increment for the elements of X.
   INCX must not be zero.

 BETA
   BETA is COMPLEX
   On entry, BETA specifies the scalar beta. When BETA is
   supplied as zero then Y need not be set on input.

 Y
   Y is COMPLEX array of DIMENSION at least
   ( 1 + ( m - 1 )*abs( INCY ) ) when TRANS = 'N' or 'n'
   and at least
   ( 1 + ( n - 1 )*abs( INCY ) ) otherwise.
   Before entry, the incremented array Y must contain the
   vector y. On exit, Y is overwritten by the updated vector y.

 INCY
   INCY is INTEGER
   On entry, INCY specifies the increment for the elements of Y.
   INCY must not be zero.

   The vector and matrix arguments are not referenced when N = 0, or M = 0
```

———-

12.1.3 fortran code

— **cgbmv.f** —

```
*  ===================================================================
      SUBROUTINE CGBMV(TRANS,M,N,KL,KU,ALPHA,A,LDA,X,INCX,BETA,Y,INCY)
*
*  -- Reference BLAS level2 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      COMPLEX ALPHA,BETA
      INTEGER INCX,INCY,KL,KU,LDA,M,N
      CHARACTER TRANS
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),X(*),Y(*)
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX ONE
      PARAMETER (ONE= (1.0E+0,0.0E+0))
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP
      INTEGER I,INFO,IX,IY,J,JX,JY,K,KUP1,KX,KY,LENX,LENY
      LOGICAL NOCONJ
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG,MAX,MIN
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +    .NOT.LSAME(TRANS,'C')) THEN
          INFO = 1
      ELSE IF (M.LT.0) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
          INFO = 3
      ELSE IF (KL.LT.0) THEN
          INFO = 4
      ELSE IF (KU.LT.0) THEN
          INFO = 5
      ELSE IF (LDA.LT. (KL+KU+1)) THEN
          INFO = 8
      ELSE IF (INCX.EQ.0) THEN
          INFO = 10
      ELSE IF (INCY.EQ.0) THEN
          INFO = 13
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CGBMV ',INFO)
          RETURN
```

```
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR.
     +    ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
      NOCONJ = LSAME(TRANS,'T')
*
*     Set  LENX  and  LENY, the lengths of the vectors x and y, and set
*     up the start points in  X  and  Y.
*
      IF (LSAME(TRANS,'N')) THEN
          LENX = N
          LENY = M
      ELSE
          LENX = M
          LENY = N
      END IF
      IF (INCX.GT.0) THEN
          KX = 1
      ELSE
          KX = 1 - (LENX-1)*INCX
      END IF
      IF (INCY.GT.0) THEN
          KY = 1
      ELSE
          KY = 1 - (LENY-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the band part of A.
*
*     First form  y := beta*y.
*
      IF (BETA.NE.ONE) THEN
          IF (INCY.EQ.1) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 10 I = 1,LENY
                      Y(I) = ZERO
   10             CONTINUE
              ELSE
                  DO 20 I = 1,LENY
                      Y(I) = BETA*Y(I)
   20             CONTINUE
              END IF
          ELSE
              IY = KY
              IF (BETA.EQ.ZERO) THEN
                  DO 30 I = 1,LENY
```

```
                    Y(IY) = ZERO
                    IY = IY + INCY
   30            CONTINUE
            ELSE
                DO 40 I = 1,LENY
                    Y(IY) = BETA*Y(IY)
                    IY = IY + INCY
   40            CONTINUE
            END IF
        END IF
      END IF
      IF (ALPHA.EQ.ZERO) RETURN
      KUP1 = KU + 1
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  y := alpha*A*x + y.
*
         JX = KX
         IF (INCY.EQ.1) THEN
            DO 60 J = 1,N
                TEMP = ALPHA*X(JX)
                K = KUP1 - J
                DO 50 I = MAX(1,J-KU),MIN(M,J+KL)
                    Y(I) = Y(I) + TEMP*A(K+I,J)
   50           CONTINUE
                JX = JX + INCX
   60       CONTINUE
         ELSE
            DO 80 J = 1,N
                TEMP = ALPHA*X(JX)
                IY = KY
                K = KUP1 - J
                DO 70 I = MAX(1,J-KU),MIN(M,J+KL)
                    Y(IY) = Y(IY) + TEMP*A(K+I,J)
                    IY = IY + INCY
   70           CONTINUE
                JX = JX + INCX
                IF (J.GT.KU) KY = KY + INCY
   80       CONTINUE
         END IF
      ELSE
*
*        Form  y := alpha*A**T*x + y  or  y := alpha*A**H*x + y.
*
         JY = KY
         IF (INCX.EQ.1) THEN
            DO 110 J = 1,N
                TEMP = ZERO
                K = KUP1 - J
                IF (NOCONJ) THEN
```

```
                    DO 90 I = MAX(1,J-KU),MIN(M,J+KL)
                        TEMP = TEMP + A(K+I,J)*X(I)
   90               CONTINUE
                ELSE
                    DO 100 I = MAX(1,J-KU),MIN(M,J+KL)
                        TEMP = TEMP + CONJG(A(K+I,J))*X(I)
  100               CONTINUE
                END IF
                Y(JY) = Y(JY) + ALPHA*TEMP
                JY = JY + INCY
  110         CONTINUE
        ELSE
            DO 140 J = 1,N
                TEMP = ZERO
                IX = KX
                K = KUP1 - J
                IF (NOCONJ) THEN
                    DO 120 I = MAX(1,J-KU),MIN(M,J+KL)
                        TEMP = TEMP + A(K+I,J)*X(IX)
                        IX = IX + INCX
  120               CONTINUE
                ELSE
                    DO 130 I = MAX(1,J-KU),MIN(M,J+KL)
                        TEMP = TEMP + CONJG(A(K+I,J))*X(IX)
                        IX = IX + INCX
  130               CONTINUE
                END IF
                Y(JY) = Y(JY) + ALPHA*TEMP
                JY = JY + INCY
                IF (J.GT.KU) KX = KX + INCX
  140         CONTINUE
        END IF
    END IF
*
    RETURN
*
*   End of CGBMV .
*
    END
```

———-

12.1.4 lisp code

— BLAS 2 cgbmv —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)) (zero (f2cl-lib:cmplx 0.0 0.0)))
```

```
(declare (type (f2cl-lib:complex8) one)
         (type (f2cl-lib:complex8) zero)
         (ignorable one zero))
(defun cgbmv (trans m n kl ku alpha a lda x incx beta y incy)
  (declare (type (array f2cl-lib:complex8 (*)) y x a)
           (type (f2cl-lib:complex8) beta alpha)
           (type (f2cl-lib:integer4) incy incx lda ku kl n m)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (x f2cl-lib:complex8 x-%data% x-%offset%)
       (y f2cl-lib:complex8 y-%data% y-%offset%))
    (prog ((noconj nil) (i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0)
           (k 0) (kup1 0) (kx 0) (ky 0) (lenx 0) (leny 0) (temp #C(0.0 0.0)))
      (declare (type f2cl-lib:logical noconj)
               (type (f2cl-lib:integer4) i info ix iy j jx jy k kup1 kx ky
                lenx leny)
               (type (f2cl-lib:complex8) temp))
      (setf info 0)
      (cond
       ((and
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame trans "N")
            (declare (ignore var-1))
            (when var-0 (setf trans var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame trans "T")
            (declare (ignore var-1))
            (when var-0 (setf trans var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame trans "C")
            (declare (ignore var-1))
            (when var-0 (setf trans var-0))
            ret-val)))
        (setf info 1))
       ((< m 0) (setf info 2)) ((< n 0) (setf info 3))
       ((< kl 0) (setf info 4)) ((< ku 0) (setf info 5))
       ((< lda (f2cl-lib:int-add kl ku 1)) (setf info 8))
       ((= incx 0) (setf info 10)) ((= incy 0) (setf info 13)))
      (cond
       ((/= info 0)
        (multiple-value-bind (var-0 var-1)
            (xerbla "CGBMV " info)
          (declare (ignore var-0))
```

```
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
    (go end_label))
(setf noconj
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame trans "N")
    (declare (ignore var-1))
    (when var-0 (setf trans var-0))
    ret-val)
  (setf lenx n) (setf leny m))
 (t (setf lenx m) (setf leny n)))
(cond ((> incx 0) (setf kx 1))
      (t
       (setf kx
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub lenx 1) incx)))))
(cond ((> incy 0) (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub leny 1) incy)))))
(cond
 ((/= beta one)
  (cond
   ((= incy 1)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i leny) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%) zero)
         label10)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i leny) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                  (* beta
                     (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)))
         label20)))))
   (t (setf iy ky)
```

```
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i leny) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) zero)
          (setf iy (f2cl-lib:int-add iy incy))
         label30)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i leny) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (* beta
                     (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)))
          (setf iy (f2cl-lib:int-add iy incy))
         label40))))))))
(if (= alpha zero)
    (go end_label))
(setf kup1 (f2cl-lib:int-add ku 1))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame trans "N")
    (declare (ignore var-1))
    (when var-0 (setf trans var-0))
    ret-val)
  (setf jx kx)
  (cond
   ((= incy 1)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp
                (* alpha
                   (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
        (setf k (f2cl-lib:int-sub kup1 j))
        (f2cl-lib:fdo (i
                       (max (the f2cl-lib:integer4 1)
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add j
                                                   (f2cl-lib:int-sub
                                                    ku))))
                       (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the f2cl-lib:integer4 m)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add j kl))))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
```

```
                    (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                       (* temp
                          (f2cl-lib:fref a-%data%
                                         ((f2cl-lib:int-add k i) j)
                                         ((1 lda) (1 *)) a-%offset%))))
           label50))
        (setf jx (f2cl-lib:int-add jx incx))
       label60)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp
               (* alpha
                  (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
       (setf iy ky)
       (setf k (f2cl-lib:int-sub kup1 j))
       (f2cl-lib:fdo (i
                      (max (the f2cl-lib:integer4 1)
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add j
                                                  (f2cl-lib:int-sub
                                                   ku))))
                      (f2cl-lib:int-add i 1))
                     ((> i
                         (min (the f2cl-lib:integer4 m)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add j kl))))
                      nil)
         (tagbody
           (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                   (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (* temp
                         (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-add k i) j)
                                        ((1 lda) (1 *)) a-%offset%))))
           (setf iy (f2cl-lib:int-add iy incy))
          label70))
       (setf jx (f2cl-lib:int-add jx incx))
       (if (> j ku)
           (setf ky (f2cl-lib:int-add ky incy)))
      label80)))))
(t (setf jy ky)
 (cond
  ((= incx 1)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp zero)
       (setf k (f2cl-lib:int-sub kup1 j))
```

```
(cond
 (noconj
  (f2cl-lib:fdo (i
                 (max (the f2cl-lib:integer4 1)
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add j
                                             (f2cl-lib:int-sub
                                              ku))))
                 (f2cl-lib:int-add i 1))
                ((> i
                    (min (the f2cl-lib:integer4 m)
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add j kl))))
                 nil)
    (tagbody
      (setf temp
              (+ temp
                 (*
                  (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add k i) j)
                                 ((1 lda) (1 *)) a-%offset%)
                  (f2cl-lib:fref x-%data% (i) ((1 *))
                                 x-%offset%))))
     label90)))
 (t
  (f2cl-lib:fdo (i
                 (max (the f2cl-lib:integer4 1)
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add j
                                             (f2cl-lib:int-sub
                                              ku))))
                 (f2cl-lib:int-add i 1))
                ((> i
                    (min (the f2cl-lib:integer4 m)
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add j kl))))
                 nil)
    (tagbody
      (setf temp
              (+ temp
                 (*
                  (f2cl-lib:conjg
                   (f2cl-lib:fref a-%data%
                                  ((f2cl-lib:int-add k i) j)
                                  ((1 lda) (1 *)) a-%offset%))
                  (f2cl-lib:fref x-%data% (i) ((1 *))
                                 x-%offset%))))
     label100))))
(setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
        (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
```

```
                 (* alpha temp)))
        (setf jy (f2cl-lib:int-add jy incy))
       label110)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp zero)
         (setf ix kx)
         (setf k (f2cl-lib:int-sub kup1 j))
         (cond
          (noconj
           (f2cl-lib:fdo (i
                          (max (the f2cl-lib:integer4 1)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add j
                                                      (f2cl-lib:int-sub
                                                       ku))))
                          (f2cl-lib:int-add i 1))
                         ((> i
                             (min (the f2cl-lib:integer4 m)
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-add j kl))))
                          nil)
             (tagbody
               (setf temp
                       (+ temp
                          (*
                           (f2cl-lib:fref a-%data%
                                          ((f2cl-lib:int-add k i) j)
                                          ((1 lda) (1 *)) a-%offset%)
                           (f2cl-lib:fref x-%data% (ix) ((1 *))
                                          x-%offset%))))
               (setf ix (f2cl-lib:int-add ix incx))
              label120)))
          (t
           (f2cl-lib:fdo (i
                          (max (the f2cl-lib:integer4 1)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add j
                                                      (f2cl-lib:int-sub
                                                       ku))))
                          (f2cl-lib:int-add i 1))
                         ((> i
                             (min (the f2cl-lib:integer4 m)
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-add j kl))))
                          nil)
             (tagbody
               (setf temp
```

```
                              (+ temp
                                 (*
                                  (f2cl-lib:conjg
                                   (f2cl-lib:fref a-%data%
                                                  ((f2cl-lib:int-add k i) j)
                                                  ((1 lda) (1 *)) a-%offset%))
                                  (f2cl-lib:fref x-%data% (ix) ((1 *))
                                                 x-%offset%))))
                        (setf ix (f2cl-lib:int-add ix incx))
                       label130))))
                (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                        (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                           (* alpha temp)))
                (setf jy (f2cl-lib:int-add jy incy))
                (if (> j ku)
                    (setf kx (f2cl-lib:int-add kx incx)))
               label140))))))
         (go end_label)
        end_label
         (return
          (values trans nil nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

12.2 cgemv - matrix-vector $\alpha Ax + \beta y$ or $\alpha A^T + \beta y$ or $\alpha A^H x + \beta y$

12.2.1 Axiom unit tests

— **cgemv.input** —

```
)set break resume
)sys rm -f cgemv.output
)spool cgemv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

12.2.2 Axiom help page

— cgemv.help —

```
=====================================================================
cgemv examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgemv (TRANS, M, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)
           CGEMV

Function/Subroutine Documentation
   subroutine cgemv (character TRANS, integer M, integer N, complex ALPHA,
       complex, dimension(lda,*) A, integer LDA, complex, dimension(*) X,
       integer INCX, complex BETA, complex, dimension(*) Y, integer INCY)
       CGEMV

Purpose:
   CGEMV performs one of the matrix-vector operations

   y := alpha*A*x + beta*y,   or   y := alpha*A**T*x + beta*y,   or

   y := alpha*A**H*x + beta*y,

   where alpha and beta are scalars, x and y are vectors and A is an
   m by n matrix.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   On entry, TRANS specifies the operation to be performed as follows:

   TRANS = 'N' or 'n'   y := alpha*A*x + beta*y.

   TRANS = 'T' or 't'   y := alpha*A**T*x + beta*y.

   TRANS = 'C' or 'c'   y := alpha*A**H*x + beta*y.

 M
   M is INTEGER
   On entry, M specifies the number of rows of the matrix A.
   M must be at least zero.
```

```
N

  N is INTEGER
  On entry, N specifies the number of columns of the matrix A.
  N must be at least zero.

ALPHA
  ALPHA is COMPLEX
  On entry, ALPHA specifies the scalar alpha.

A
  A is COMPLEX array of DIMENSION ( LDA, n ).
  Before entry, the leading m by n part of the array A must
  contain the matrix of coefficients.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, m ).

X
  X is COMPLEX array of DIMENSION at least
  ( 1 + ( n - 1 )*abs( INCX ) ) when TRANS = 'N' or 'n'
  and at least
  ( 1 + ( m - 1 )*abs( INCX ) ) otherwise.
  Before entry, the incremented array X must contain the vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.

BETA
  BETA is COMPLEX
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.

Y
  Y is COMPLEX array of DIMENSION at least
  ( 1 + ( m - 1 )*abs( INCY ) ) when TRANS = 'N' or 'n'
  and at least
  ( 1 + ( n - 1 )*abs( INCY ) ) otherwise.
  Before entry with BETA non-zero, the incremented array Y
  must contain the vector y. On exit, Y is overwritten by the
  updated vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of Y.
```

```
INCY must not be zero.

The vector and matrix arguments are not referenced when N = 0, or M = 0
```

12.2.3 fortran code

— cgemv.f —

```
*  ===================================================================
      SUBROUTINE CGEMV(TRANS,M,N,ALPHA,A,LDA,X,INCX,BETA,Y,INCY)
*
*  -- Reference BLAS level2 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      COMPLEX ALPHA,BETA
      INTEGER INCX,INCY,LDA,M,N
      CHARACTER TRANS
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),X(*),Y(*)
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      COMPLEX ONE
      PARAMETER (ONE= (1.0E+0,0.0E+0))
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP
      INTEGER I,INFO,IX,IY,J,JX,JY,KX,KY,LENX,LENY
      LOGICAL NOCONJ
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
```

```
*     .. Intrinsic Functions ..
      INTRINSIC CONJG,MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +    .NOT.LSAME(TRANS,'C')) THEN
          INFO = 1
      ELSE IF (M.LT.0) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
          INFO = 3
      ELSE IF (LDA.LT.MAX(1,M)) THEN
          INFO = 6
      ELSE IF (INCX.EQ.0) THEN
          INFO = 8
      ELSE IF (INCY.EQ.0) THEN
          INFO = 11
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CGEMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR.
     +    ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
      NOCONJ = LSAME(TRANS,'T')
*
*     Set  LENX  and  LENY, the lengths of the vectors x and y, and set
*     up the start points in  X  and  Y.
*
      IF (LSAME(TRANS,'N')) THEN
          LENX = N
          LENY = M
      ELSE
          LENX = M
          LENY = N
      END IF
      IF (INCX.GT.0) THEN
          KX = 1
      ELSE
          KX = 1 - (LENX-1)*INCX
      END IF
      IF (INCY.GT.0) THEN
          KY = 1
```

```
      ELSE
          KY = 1 - (LENY-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
*     First form  y := beta*y.
*
      IF (BETA.NE.ONE) THEN
          IF (INCY.EQ.1) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 10 I = 1,LENY
                      Y(I) = ZERO
   10             CONTINUE
              ELSE
                  DO 20 I = 1,LENY
                      Y(I) = BETA*Y(I)
   20             CONTINUE
              END IF
          ELSE
              IY = KY
              IF (BETA.EQ.ZERO) THEN
                  DO 30 I = 1,LENY
                      Y(IY) = ZERO
                      IY = IY + INCY
   30             CONTINUE
              ELSE
                  DO 40 I = 1,LENY
                      Y(IY) = BETA*Y(IY)
                      IY = IY + INCY
   40             CONTINUE
              END IF
          END IF
      END IF
      IF (ALPHA.EQ.ZERO) RETURN
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  y := alpha*A*x + y.
*
          JX = KX
          IF (INCY.EQ.1) THEN
              DO 60 J = 1,N
                  TEMP = ALPHA*X(JX)
                  DO 50 I = 1,M
                      Y(I) = Y(I) + TEMP*A(I,J)
   50             CONTINUE
                  JX = JX + INCX
   60         CONTINUE
          ELSE
```

```
            DO 80 J = 1,N
                TEMP = ALPHA*X(JX)
                IY = KY
                DO 70 I = 1,M
                    Y(IY) = Y(IY) + TEMP*A(I,J)
                    IY = IY + INCY
   70           CONTINUE
                JX = JX + INCX
   80       CONTINUE
        END IF
      ELSE
*
*       Form  y := alpha*A**T*x + y  or  y := alpha*A**H*x + y.
*
         JY = KY
         IF (INCX.EQ.1) THEN
            DO 110 J = 1,N
                TEMP = ZERO
                IF (NOCONJ) THEN
                    DO 90 I = 1,M
                        TEMP = TEMP + A(I,J)*X(I)
   90               CONTINUE
                ELSE
                    DO 100 I = 1,M
                        TEMP = TEMP + CONJG(A(I,J))*X(I)
  100               CONTINUE
                END IF
                Y(JY) = Y(JY) + ALPHA*TEMP
                JY = JY + INCY
  110       CONTINUE
         ELSE
            DO 140 J = 1,N
                TEMP = ZERO
                IX = KX
                IF (NOCONJ) THEN
                    DO 120 I = 1,M
                        TEMP = TEMP + A(I,J)*X(IX)
                        IX = IX + INCX
  120               CONTINUE
                ELSE
                    DO 130 I = 1,M
                        TEMP = TEMP + CONJG(A(I,J))*X(IX)
                        IX = IX + INCX
  130               CONTINUE
                END IF
                Y(JY) = Y(JY) + ALPHA*TEMP
                JY = JY + INCY
  140       CONTINUE
         END IF
      END IF
```

```
*
      RETURN
*
*     End of CGEMV .
*
      END
```

———-

12.2.4 lisp code

— BLAS 2 cgemv —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)) (zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (ignorable one zero))
  (defun cgemv (trans m n alpha a lda x incx beta y incy)
    (declare (type (array f2cl-lib:complex8 (*)) y x a)
             (type (f2cl-lib:complex8) beta alpha)
             (type (f2cl-lib:integer4) incy incx lda n m)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%)
         (y f2cl-lib:complex8 y-%data% y-%offset%))
      (prog ((noconj nil) (i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0)
             (kx 0) (ky 0) (lenx 0) (leny 0) (temp #C(0.0 0.0)))
        (declare (type f2cl-lib:logical noconj)
                 (type (f2cl-lib:integer4) i info ix iy j jx jy kx ky lenx
                  leny)
                 (type (f2cl-lib:complex8) temp))
        (setf info 0)
        (cond
         ((and
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "N")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "T")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
```

```
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame trans "C")
      (declare (ignore var-1))
      (when var-0 (setf trans var-0))
      ret-val)))
  (setf info 1))
 ((< m 0) (setf info 2)) ((< n 0) (setf info 3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info 6))
 ((= incx 0) (setf info 8)) ((= incy 0) (setf info 11)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "CGEMV " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
    (go end_label))
(setf noconj
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame trans "N")
    (declare (ignore var-1))
    (when var-0 (setf trans var-0))
    ret-val)
  (setf lenx n) (setf leny m))
 (t (setf lenx m) (setf leny n)))
(cond ((> incx 0) (setf kx 1))
      (t
       (setf kx
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub lenx 1) incx)))))
(cond ((> incy 0) (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub leny 1) incy)))))
(cond
 ((/= beta one)
  (cond
```

```
   ((= incy 1)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i leny) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%) zero)
         label10)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i leny) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                  (* beta
                     (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)))
         label20)))))
   (t (setf iy ky)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i leny) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) zero)
          (setf iy (f2cl-lib:int-add iy incy))
         label30)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i leny) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (* beta
                     (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)))
          (setf iy (f2cl-lib:int-add iy incy))
         label40))))))))
(if (= alpha zero)
    (go end_label))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame trans "N")
    (declare (ignore var-1))
    (when var-0 (setf trans var-0))
    ret-val)
  (setf jx kx)
  (cond
   ((= incy 1)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp
                (* alpha
```

```
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                      (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                         (* temp
                            (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                           a-%offset%))))
             label50))
          (setf jx (f2cl-lib:int-add jx incx))
         label60)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp
                 (* alpha
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
         (setf iy ky)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (* temp
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%))))
             (setf iy (f2cl-lib:int-add iy incy))
            label70))
         (setf jx (f2cl-lib:int-add jx incx))
        label80)))))
  (t (setf jy ky)
   (cond
    ((= incx 1)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp zero)
         (cond
          (noconj
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf temp
                       (+ temp
                          (*
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%)
                           (f2cl-lib:fref x-%data% (i) ((1 *))
```

```
                                        x-%offset%))))
            label90)))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf temp
                     (+ temp
                        (*
                         (f2cl-lib:conjg
                          (f2cl-lib:fref a-%data% (i j)
                                         ((1 lda) (1 *)) a-%offset%))
                         (f2cl-lib:fref x-%data% (i) ((1 *))
                                        x-%offset%))))
            label100))))
       (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
               (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                  (* alpha temp)))
       (setf jy (f2cl-lib:int-add jy incy))
      label110)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp zero)
       (setf ix kx)
       (cond
        (noconj
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf temp
                     (+ temp
                        (*
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)
                         (f2cl-lib:fref x-%data% (ix) ((1 *))
                                        x-%offset%))))
             (setf ix (f2cl-lib:int-add ix incx))
            label120)))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf temp
                     (+ temp
                        (*
                         (f2cl-lib:conjg
                          (f2cl-lib:fref a-%data% (i j)
                                         ((1 lda) (1 *)) a-%offset%))
```

```
                                (f2cl-lib:fref x-%data% (ix) ((1 *))
                                               x-%offset%))))
                      (setf ix (f2cl-lib:int-add ix incx))
                     label130))))
                (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                        (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                           (* alpha temp)))
                (setf jy (f2cl-lib:int-add jy incy))
               label140))))))
        (go end_label)
       end_label
        (return (values trans nil nil nil nil nil nil nil nil nil nil))))))
```

———-

12.3 cgerc - rank 1 $\alpha xy^H + A$

12.3.1 Axiom unit tests

— **cgerc.input** —

```
)set break resume
)sys rm -f cgerc.output
)spool cgerc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

12.3.2 Axiom help page

— **cgerc.help** —

```
==========================================================================
cgerc examples
==========================================================================

==========================================================================
Man Page Details
```

```
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgerc (M, N, ALPHA, X, INCX, Y, INCY, A, LDA)
           CGERC

Function/Subroutine Documentation
   subroutine cgerc (integer M, integer N, complex ALPHA, complex,
       dimension(*) X, integer INCX, complex, dimension(*) Y, integer INCY,
       complex, dimension(lda,*) A, integer LDA)
       CGERC

Purpose:
   CGERC  performs the rank 1 operation

   A := alpha*x*y**H + A,

   where alpha is a scalar, x is an m element vector, y is an n element
   vector and A is an m by n matrix.

Parameters:

 M
   M is INTEGER
   On entry, M specifies the number of rows of the matrix A.
   M must be at least zero.

 N
   N is INTEGER
   On entry, N specifies the number of columns of the matrix A.
   N must be at least zero.

 ALPHA
   ALPHA is COMPLEX
   On entry, ALPHA specifies the scalar alpha.

 X
   X is COMPLEX array of dimension at least ( 1 + ( m - 1 )*abs( INCX ) ).
   Before entry, the incremented array X must contain the m
   element vector x.

 INCX
   INCX is INTEGER
   On entry, INCX specifies the increment for the elements of X.
   INCX must not be zero.

 Y
   Y is COMPLEX array of dimension at least ( 1 + ( n - 1 )*abs( INCY ) ).
   Before entry, the incremented array Y must contain the n
```

```
  element vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of Y.
  INCY must not be zero.

A
  A is COMPLEX array of DIMENSION ( LDA, n ).
  Before entry, the leading m by n part of the array A must
  contain the matrix of coefficients. On exit, A is
  overwritten by the updated matrix.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, m ).
```

12.3.3 fortran code

— cgerc.f —

```
*  ====================================================================
      SUBROUTINE CGERC(M,N,ALPHA,X,INCX,Y,INCY,A,LDA)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX ALPHA
      INTEGER INCX,INCY,LDA,M,N
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),X(*),Y(*)
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*     .. Local Scalars ..
```

```
      COMPLEX TEMP
      INTEGER I,INFO,IX,J,JY,KX
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG,MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (M.LT.0) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      ELSE IF (INCY.EQ.0) THEN
          INFO = 7
      ELSE IF (LDA.LT.MAX(1,M)) THEN
          INFO = 9
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CGERC ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR. (ALPHA.EQ.ZERO)) RETURN
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
      IF (INCY.GT.0) THEN
          JY = 1
      ELSE
          JY = 1 - (N-1)*INCY
      END IF
      IF (INCX.EQ.1) THEN
          DO 20 J = 1,N
              IF (Y(JY).NE.ZERO) THEN
                  TEMP = ALPHA*CONJG(Y(JY))
                  DO 10 I = 1,M
                      A(I,J) = A(I,J) + X(I)*TEMP
   10             CONTINUE
              END IF
              JY = JY + INCY
```

```
   20     CONTINUE
      ELSE
          IF (INCX.GT.0) THEN
              KX = 1
          ELSE
              KX = 1 - (M-1)*INCX
          END IF
          DO 40 J = 1,N
              IF (Y(JY).NE.ZERO) THEN
                  TEMP = ALPHA*CONJG(Y(JY))
                  IX = KX
                  DO 30 I = 1,M
                      A(I,J) = A(I,J) + X(IX)*TEMP
                      IX = IX + INCX
   30             CONTINUE
              END IF
              JY = JY + INCY
   40     CONTINUE
      END IF
*
      RETURN
*
*     End of CGERC .
*
      END
```

———-

12.3.4 lisp code

— BLAS 2 cgerc —

```
(let* ((zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) zero)
           (ignorable zero))
  (defun cgerc (m n alpha x incx y incy a lda)
    (declare (type (array f2cl-lib:complex8 (*)) a y x)
             (type (f2cl-lib:complex8) alpha)
             (type (f2cl-lib:integer4) lda incy incx n m))
    (f2cl-lib:with-multi-array-data
        ((x f2cl-lib:complex8 x-%data% x-%offset%)
         (y f2cl-lib:complex8 y-%data% y-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%))
      (prog ((i 0) (info 0) (ix 0) (j 0) (jy 0) (kx 0) (temp #C(0.0 0.0)))
        (declare (type (f2cl-lib:integer4) i info ix j jy kx)
                 (type (f2cl-lib:complex8) temp))
        (setf info 0)
```

```
(cond ((< m 0) (setf info 1)) ((< n 0) (setf info 2))
      ((= incx 0) (setf info 5)) ((= incy 0) (setf info 7))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info 9)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "CGERC " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= m 0) (= n 0) (= alpha zero))
    (go end_label))
(cond ((> incy 0) (setf jy 1))
      (t
       (setf jy
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incy)))))
(cond
 ((= incx 1)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (cond
       ((/= (f2cl-lib:fref y (jy) ((1 *))) zero)
        (setf temp
                (* alpha
                   (f2cl-lib:conjg
                    (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%))))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%)
                  (+
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                   (* (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                      temp)))
           label10))))
      (setf jy (f2cl-lib:int-add jy incy))
     label20)))
 (t
  (cond ((> incx 0) (setf kx 1))
        (t
         (setf kx
                 (f2cl-lib:int-sub 1
                                   (f2cl-lib:int-mul
```

```
                                        (f2cl-lib:int-sub m 1) incx)))))
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (cond
               ((/= (f2cl-lib:fref y (jy) ((1 *))) zero)
                (setf temp
                        (* alpha
                           (f2cl-lib:conjg
                            (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%))))
                (setf ix kx)
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i m) nil)
                  (tagbody
                    (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                            (+
                             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                            a-%offset%)
                             (*
                              (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                              temp)))
                    (setf ix (f2cl-lib:int-add ix incx))
                   label30))))
              (setf jy (f2cl-lib:int-add jy incy))
             label40))))
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil nil nil nil))))))
```

———-

12.4 cgeru - rank 1 $\alpha xy^T + A$

12.4.1 Axiom unit tests

— cgeru.input —

```
)set break resume
)sys rm -f cgeru.output
)spool cgeru.output
)set message test on
)set message auto off
)clear all

)spool
```

```
)lisp (bye)
```

———-

12.4.2 Axiom help page

— **cgeru.help** —

```
==========================================================================
cgeru examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgeru (M, N, ALPHA, X, INCX, Y, INCY, A, LDA)
           CGERU

Function/Subroutine Documentation
   subroutine cgeru (integer M, integer N, complex ALPHA, complex,
       dimension(*) X, integer INCX, complex, dimension(*) Y, integer INCY,
       complex, dimension(lda,*) A, integer LDA)
       CGERU

Purpose:
   CGERU  performs the rank 1 operation

      A := alpha*x*y**T + A,

   where alpha is a scalar, x is an m element vector, y is an n element
   vector and A is an m by n matrix.

Parameters:

 M
   M is INTEGER
   On entry, M specifies the number of rows of the matrix A.
   M must be at least zero.

 N
   N is INTEGER
   On entry, N specifies the number of columns of the matrix A.
   N must be at least zero.

 ALPHA
```

```
  ALPHA is COMPLEX
  On entry, ALPHA specifies the scalar alpha.

X
  X is COMPLEX array of dimension at least ( 1 + ( m - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the m
  element vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.

Y
  Y is COMPLEX array of dimension at least ( 1 + ( n - 1 )*abs( INCY ) ).
  Before entry, the incremented array Y must contain the n
  element vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of Y.
  INCY must not be zero.

A
  A is COMPLEX array of DIMENSION ( LDA, n ).
  Before entry, the leading m by n part of the array A must
  contain the matrix of coefficients. On exit, A is
  overwritten by the updated matrix.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, m ).
```

———-

12.4.3 fortran code

— cgeru.f —

```
*  ==================================================================
      SUBROUTINE CGERU(M,N,ALPHA,X,INCX,Y,INCY,A,LDA)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
```

```
*
*     .. Scalar Arguments ..
      COMPLEX ALPHA
      INTEGER INCX,INCY,LDA,M,N
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),X(*),Y(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP
      INTEGER I,INFO,IX,J,JY,KX
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (M.LT.0) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      ELSE IF (INCY.EQ.0) THEN
          INFO = 7
      ELSE IF (LDA.LT.MAX(1,M)) THEN
          INFO = 9
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CGERU ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR. (ALPHA.EQ.ZERO)) RETURN
*
*     Start the operations. In this version the elements of A are
```

```
*     accessed sequentially with one pass through A.
*
      IF (INCY.GT.0) THEN
          JY = 1
      ELSE
          JY = 1 - (N-1)*INCY
      END IF
      IF (INCX.EQ.1) THEN
          DO 20 J = 1,N
              IF (Y(JY).NE.ZERO) THEN
                  TEMP = ALPHA*Y(JY)
                  DO 10 I = 1,M
                      A(I,J) = A(I,J) + X(I)*TEMP
   10             CONTINUE
              END IF
              JY = JY + INCY
   20     CONTINUE
      ELSE
          IF (INCX.GT.0) THEN
              KX = 1
          ELSE
              KX = 1 - (M-1)*INCX
          END IF
          DO 40 J = 1,N
              IF (Y(JY).NE.ZERO) THEN
                  TEMP = ALPHA*Y(JY)
                  IX = KX
                  DO 30 I = 1,M
                      A(I,J) = A(I,J) + X(IX)*TEMP
                      IX = IX + INCX
   30             CONTINUE
              END IF
              JY = JY + INCY
   40     CONTINUE
      END IF
*
      RETURN
*
*     End of CGERU .
*
      END
```

———-

12.4.4 lisp code

— BLAS 2 cgeru —

```
(let* ((zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) zero)
           (ignorable zero))
  (defun cgeru (m n alpha x incx y incy a lda)
    (declare (type (array f2cl-lib:complex8 (*)) a y x)
             (type (f2cl-lib:complex8) alpha)
             (type (f2cl-lib:integer4) lda incy incx n m))
    (f2cl-lib:with-multi-array-data
        ((x f2cl-lib:complex8 x-%data% x-%offset%)
         (y f2cl-lib:complex8 y-%data% y-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%))
      (prog ((i 0) (info 0) (ix 0) (j 0) (jy 0) (kx 0) (temp #C(0.0 0.0)))
        (declare (type (f2cl-lib:integer4) i info ix j jy kx)
                 (type (f2cl-lib:complex8) temp))
        (setf info 0)
        (cond ((< m 0) (setf info 1)) ((< n 0) (setf info 2))
              ((= incx 0) (setf info 5)) ((= incy 0) (setf info 7))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info 9)))
        (cond
         ((/= info 0)
          (multiple-value-bind (var-0 var-1)
              (xerbla "CGERU " info)
            (declare (ignore var-0))
            (when var-1 (setf info var-1)))
          (go end_label)))
        (if (or (= m 0) (= n 0) (= alpha zero))
            (go end_label))
        (cond ((> incy 0) (setf jy 1))
              (t
               (setf jy
                       (f2cl-lib:int-sub 1
                                         (f2cl-lib:int-mul
                                          (f2cl-lib:int-sub n 1) incy)))))
        (cond
         ((= incx 1)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (cond
               ((/= (f2cl-lib:fref y (jy) ((1 *))) zero)
                (setf temp
                        (* alpha
                           (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)))
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i m) nil)
                  (tagbody
                    (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
```

```
                        (+
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)
                         (* (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                            temp)))
              label10))))
          (setf jy (f2cl-lib:int-add jy incy))
         label20)))
     (t
      (cond ((> incx 0) (setf kx 1))
            (t
             (setf kx
                     (f2cl-lib:int-sub 1
                                       (f2cl-lib:int-mul
                                        (f2cl-lib:int-sub m 1) incx)))))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
           ((/= (f2cl-lib:fref y (jy) ((1 *))) zero)
            (setf temp
                    (* alpha
                       (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)))
            (setf ix kx)
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%)
                      (+
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                       (*
                        (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                        temp)))
                (setf ix (f2cl-lib:int-add ix incx))
              label30))))
          (setf jy (f2cl-lib:int-add jy incy))
         label40))))
    (go end_label)
   end_label
    (return (values nil nil nil nil nil nil nil nil nil))))))
```

———-

12.5 chbmv - hermitian band matrix-vector $\alpha Ax + \beta y$

12.5.1 Axiom unit tests

— chbmv.input —

```
)set break resume
)sys rm -f chbmv.output
)spool chbmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

12.5.2 Axiom help page

— chbmv.help —

```
======================================================================
chbmv examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine chbmv (UPLO, N, K, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)
           CHBMV

Function/Subroutine Documentation
   subroutine chbmv (character UPLO, integer N, integer K, complex ALPHA,
       complex, dimension(lda,*) A, integer LDA, complex, dimension(*) X,
       integer INCX, complex BETA, complex, dimension(*) Y, integer INCY)
       CHBMV

Purpose:
   CHBMV  performs the matrix-vector  operation

      y := alpha*A*x + beta*y,
```

```
  where alpha and beta are scalars, x and y are n element vectors and
  A is an n by n hermitian band matrix, with k super-diagonals.

Parameters:

 UPLO
  UPLO is CHARACTER*1
  On entry, UPLO specifies whether the upper or lower
  triangular part of the band matrix A is being supplied as follows:

  UPLO = 'U' or 'u'   The upper triangular part of A is being supplied.

  UPLO = 'L' or 'l'   The lower triangular part of A is being supplied.

 N
  N is INTEGER
  On entry, N specifies the order of the matrix A.
  N must be at least zero.

 K
  K is INTEGER
  On entry, K specifies the number of super-diagonals of the
  matrix A. K must satisfy  0 .le. K.

 ALPHA
  ALPHA is COMPLEX
  On entry, ALPHA specifies the scalar alpha.

 A
  A is COMPLEX array of DIMENSION ( LDA, n ).

  Before entry with UPLO = 'U' or 'u', the leading ( k + 1 )
  by n part of the array A must contain the upper triangular
  band part of the hermitian matrix, supplied column by
  column, with the leading diagonal of the matrix in row
  ( k + 1 ) of the array, the first super-diagonal starting at
  position 2 in row k, and so on. The top left k by k triangle
  of the array A is not referenced.

  The following program segment will transfer the upper
  triangular part of a hermitian band matrix from conventional
  full matrix storage to band storage:

       DO 20, J = 1, N
          M = K + 1 - J
          DO 10, I = MAX( 1, J - K ), J
             A( M + I, J ) = matrix( I, J )
   10    CONTINUE
   20 CONTINUE
```

```
  Before entry with UPLO = 'L' or 'l', the leading ( k + 1 )
  by n part of the array A must contain the lower triangular
  band part of the hermitian matrix, supplied column by
  column, with the leading diagonal of the matrix in row 1 of
  the array, the first sub-diagonal starting at position 1 in
  row 2, and so on. The bottom right k by k triangle of the
  array A is not referenced.

  The following program segment will transfer the lower
  triangular part of a hermitian band matrix from conventional
  full matrix storage to band storage:

       DO 20, J = 1, N
          M = 1 - J
          DO 10, I = J, MIN( N, J + K )
             A( M + I, J ) = matrix( I, J )
    10    CONTINUE
    20 CONTINUE

  Note that the imaginary parts of the diagonal elements need
  not be set and are assumed to be zero.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least ( k + 1 ).

X
  X is COMPLEX array of DIMENSION at least ( 1 + ( n - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.

BETA
  BETA is COMPLEX
  On entry, BETA specifies the scalar beta.

Y
  Y is COMPLEX array of DIMENSION at least ( 1 + ( n - 1 )*abs( INCY ) ).
  Before entry, the incremented array Y must contain the
  vector y. On exit, Y is overwritten by the updated vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of Y.
  INCY must not be zero.
```

```
The vector and matrix arguments are not referenced when N = 0, or M = 0
```

———-

12.5.3 fortran code

— **chbmv.f** —

```
*  ===================================================================
      SUBROUTINE CHBMV(UPLO,N,K,ALPHA,A,LDA,X,INCX,BETA,Y,INCY)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX ALPHA,BETA
      INTEGER INCX,INCY,K,LDA,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),X(*),Y(*)
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      COMPLEX ONE
      PARAMETER (ONE= (1.0E+0,0.0E+0))
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,KPLUS1,KX,KY,L
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG,MAX,MIN,REAL
*     ..
```

```
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (K.LT.0) THEN
          INFO = 3
      ELSE IF (LDA.LT. (K+1)) THEN
          INFO = 6
      ELSE IF (INCX.EQ.0) THEN
          INFO = 8
      ELSE IF (INCY.EQ.0) THEN
          INFO = 11
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CHBMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     Set up the start points in  X  and  Y.
*
      IF (INCX.GT.0) THEN
          KX = 1
      ELSE
          KX = 1 - (N-1)*INCX
      END IF
      IF (INCY.GT.0) THEN
          KY = 1
      ELSE
          KY = 1 - (N-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of the array A
*     are accessed sequentially with one pass through A.
*
*     First form  y := beta*y.
*
      IF (BETA.NE.ONE) THEN
          IF (INCY.EQ.1) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 10 I = 1,N
                      Y(I) = ZERO
   10             CONTINUE
```

```
            ELSE
                DO 20 I = 1,N
                    Y(I) = BETA*Y(I)
   20           CONTINUE
            END IF
        ELSE
            IY = KY
            IF (BETA.EQ.ZERO) THEN
                DO 30 I = 1,N
                    Y(IY) = ZERO
                    IY = IY + INCY
   30           CONTINUE
            ELSE
                DO 40 I = 1,N
                    Y(IY) = BETA*Y(IY)
                    IY = IY + INCY
   40           CONTINUE
            END IF
        END IF
      END IF
      IF (ALPHA.EQ.ZERO) RETURN
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  y  when upper triangle of A is stored.
*
         KPLUS1 = K + 1
         IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
            DO 60 J = 1,N
                TEMP1 = ALPHA*X(J)
                TEMP2 = ZERO
                L = KPLUS1 - J
                DO 50 I = MAX(1,J-K),J - 1
                    Y(I) = Y(I) + TEMP1*A(L+I,J)
                    TEMP2 = TEMP2 + CONJG(A(L+I,J))*X(I)
   50           CONTINUE
                Y(J) = Y(J) + TEMP1*REAL(A(KPLUS1,J)) + ALPHA*TEMP2
   60       CONTINUE
         ELSE
            JX = KX
            JY = KY
            DO 80 J = 1,N
                TEMP1 = ALPHA*X(JX)
                TEMP2 = ZERO
                IX = KX
                IY = KY
                L = KPLUS1 - J
                DO 70 I = MAX(1,J-K),J - 1
                    Y(IY) = Y(IY) + TEMP1*A(L+I,J)
                    TEMP2 = TEMP2 + CONJG(A(L+I,J))*X(IX)
                    IX = IX + INCX
```

```
                    IY = IY + INCY
   70            CONTINUE
                 Y(JY) = Y(JY) + TEMP1*REAL(A(KPLUS1,J)) + ALPHA*TEMP2
                 JX = JX + INCX
                 JY = JY + INCY
                 IF (J.GT.K) THEN
                    KX = KX + INCX
                    KY = KY + INCY
                 END IF
   80         CONTINUE
         END IF
      ELSE
*
*        Form  y  when lower triangle of A is stored.
*
          IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
             DO 100 J = 1,N
                 TEMP1 = ALPHA*X(J)
                 TEMP2 = ZERO
                 Y(J) = Y(J) + TEMP1*REAL(A(1,J))
                 L = 1 - J
                 DO 90 I = J + 1,MIN(N,J+K)
                    Y(I) = Y(I) + TEMP1*A(L+I,J)
                    TEMP2 = TEMP2 + CONJG(A(L+I,J))*X(I)
   90            CONTINUE
                 Y(J) = Y(J) + ALPHA*TEMP2
  100         CONTINUE
          ELSE
             JX = KX
             JY = KY
             DO 120 J = 1,N
                 TEMP1 = ALPHA*X(JX)
                 TEMP2 = ZERO
                 Y(JY) = Y(JY) + TEMP1*REAL(A(1,J))
                 L = 1 - J
                 IX = JX
                 IY = JY
                 DO 110 I = J + 1,MIN(N,J+K)
                    IX = IX + INCX
                    IY = IY + INCY
                    Y(IY) = Y(IY) + TEMP1*A(L+I,J)
                    TEMP2 = TEMP2 + CONJG(A(L+I,J))*X(IX)
  110            CONTINUE
                 Y(JY) = Y(JY) + ALPHA*TEMP2
                 JX = JX + INCX
                 JY = JY + INCY
  120         CONTINUE
          END IF
      END IF
*
```

```
      RETURN
*
*     End of CHBMV .
*
      END
```

———-

12.5.4 lisp code

— BLAS 2 chbmv —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)) (zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (ignorable one zero))
  (defun chbmv (uplo n k alpha a lda x incx beta y incy)
    (declare (type (array f2cl-lib:complex8 (*)) y x a)
             (type (f2cl-lib:complex8) beta alpha)
             (type (f2cl-lib:integer4) incy incx lda k n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%)
         (y f2cl-lib:complex8 y-%data% y-%offset%))
      (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (kplus1 0) (kx 0)
             (ky 0) (l 0) (temp1 #C(0.0 0.0)) (temp2 #C(0.0 0.0)))
        (declare (type (f2cl-lib:integer4) i info ix iy j jx jy kplus1 kx ky l)
                 (type (f2cl-lib:complex8) temp1 temp2))
        (setf info 0)
        (cond
         ((and
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
          (setf info 1))
         ((< n 0) (setf info 2)) ((< k 0) (setf info 3))
```

```
 ((< lda (f2cl-lib:int-add k 1)) (setf info 6))
 ((= incx 0) (setf info 8)) ((= incy 0) (setf info 11)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "CHBMV " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= n 0) (and (= alpha zero) (= beta one)))
    (go end_label))
(cond ((> incx 0) (setf kx 1))
      (t
       (setf kx
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incx)))))
(cond ((> incy 0) (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incy)))))
(cond
 ((/= beta one)
  (cond
   ((= incy 1)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%) zero)
         label10)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                  (* beta
                     (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)))
         label20)))))
   (t (setf iy ky)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) zero)
          (setf iy (f2cl-lib:int-add iy incy))
```

```
          label30)))
      (t
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                   (* beta
                      (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)))
           (setf iy (f2cl-lib:int-add iy incy))
          label40))))))))
(if (= alpha zero)
    (go end_label))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame uplo "U")
    (declare (ignore var-1))
    (when var-0 (setf uplo var-0))
    ret-val)
  (setf kplus1 (f2cl-lib:int-add k 1))
  (cond
   ((and (= incx 1) (= incy 1))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp1
                (* alpha
                   (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
        (setf temp2 zero)
        (setf l (f2cl-lib:int-sub kplus1 j))
        (f2cl-lib:fdo (i
                       (max (the f2cl-lib:integer4 1)
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add j
                                                   (f2cl-lib:int-sub
                                                    k))))
                       (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                       (* temp1
                          (f2cl-lib:fref a-%data%
                                         ((f2cl-lib:int-add l i) j)
                                         ((1 lda) (1 *)) a-%offset%))))
            (setf temp2
                    (+ temp2
                       (*
                        (f2cl-lib:conjg
                         (f2cl-lib:fref a-%data%
```

```
                                        ((f2cl-lib:int-add l i) j)
                                        ((1 lda) (1 *)) a-%offset%))
                        (f2cl-lib:fref x-%data% (i) ((1 *))
                                       x-%offset%))))
           label50))
       (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
               (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                  (* temp1
                     (f2cl-lib:freal
                      (f2cl-lib:fref a-%data% (kplus1 j)
                                     ((1 lda) (1 *)) a-%offset%)))
                  (* alpha temp2)))
      label60)))
  (t (setf jx kx) (setf jy ky)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp1
               (* alpha
                  (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
       (setf temp2 zero)
       (setf ix kx)
       (setf iy ky)
       (setf l (f2cl-lib:int-sub kplus1 j))
       (f2cl-lib:fdo (i
                      (max (the f2cl-lib:integer4 1)
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add j
                                                  (f2cl-lib:int-sub
                                                   k))))
                      (f2cl-lib:int-add i 1))
                     ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                      nil)
         (tagbody
           (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                   (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (* temp1
                         (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-add l i) j)
                                        ((1 lda) (1 *)) a-%offset%))))
           (setf temp2
                   (+ temp2
                      (*
                       (f2cl-lib:conjg
                        (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-add l i) j)
                                        ((1 lda) (1 *)) a-%offset%))
                       (f2cl-lib:fref x-%data% (ix) ((1 *))
                                      x-%offset%))))
           (setf ix (f2cl-lib:int-add ix incx))
```

```
            (setf iy (f2cl-lib:int-add iy incy))
           label70))
        (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                   (* temp1
                      (f2cl-lib:freal
                       (f2cl-lib:fref a-%data% (kplus1 j)
                                      ((1 lda) (1 *)) a-%offset%)))
                   (* alpha temp2)))
        (setf jx (f2cl-lib:int-add jx incx))
        (setf jy (f2cl-lib:int-add jy incy))
        (cond
         ((> j k) (setf kx (f2cl-lib:int-add kx incx))
          (setf ky (f2cl-lib:int-add ky incy))))
       label80)))))
(t
 (cond
  ((and (= incx 1) (= incy 1))
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp1
               (* alpha
                  (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
       (setf temp2 zero)
       (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
               (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                  (* temp1
                     (f2cl-lib:freal
                      (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *))
                                     a-%offset%)))))
       (setf l (f2cl-lib:int-sub 1 j))
       (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                     ((> i
                         (min (the f2cl-lib:integer4 n)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add j k))))
                      nil)
         (tagbody
           (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                   (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                      (* temp1
                         (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-add l i) j)
                                        ((1 lda) (1 *)) a-%offset%))))
           (setf temp2
                   (+ temp2
                      (*
                       (f2cl-lib:conjg
                        (f2cl-lib:fref a-%data%
```

```
                                        ((f2cl-lib:int-add l i) j)
                                        ((1 lda) (1 *)) a-%offset%))
                        (f2cl-lib:fref x-%data% (i) ((1 *))
                                       x-%offset%))))
          label90))
      (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
              (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                 (* alpha temp2)))
     label100)))
  (t (setf jx kx) (setf jy ky)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp1
               (* alpha
                  (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
       (setf temp2 zero)
       (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
               (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                  (* temp1
                     (f2cl-lib:freal
                      (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *))
                                     a-%offset%)))))
       (setf l (f2cl-lib:int-sub 1 j))
       (setf ix jx)
       (setf iy jy)
       (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                     ((> i
                         (min (the f2cl-lib:integer4 n)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add j k))))
                      nil)
         (tagbody
           (setf ix (f2cl-lib:int-add ix incx))
           (setf iy (f2cl-lib:int-add iy incy))
           (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                   (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (* temp1
                         (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-add l i) j)
                                        ((1 lda) (1 *)) a-%offset%))))
           (setf temp2
                   (+ temp2
                      (*
                       (f2cl-lib:conjg
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add l i) j)
                                       ((1 lda) (1 *)) a-%offset%))
                       (f2cl-lib:fref x-%data% (ix) ((1 *))
                                      x-%offset%))))
```

```
                 label110))
               (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                       (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                          (* alpha temp2)))
               (setf jx (f2cl-lib:int-add jx incx))
               (setf jy (f2cl-lib:int-add jy incy))
              label120))))))
        (go end_label)
       end_label
        (return (values uplo nil nil nil nil nil nil nil nil nil nil))))))
```

———-

12.6 chemv - hermitian matrix-vector $\alpha Ax + \beta y$

12.6.1 Axiom unit tests

— **chemv.input** —

```
)set break resume
)sys rm -f chemv.output
)spool chemv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

12.6.2 Axiom help page

— **chemv.help** —

```
========================================================================
chemv examples
========================================================================

========================================================================
Man Page Details
========================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine chemv (UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)
           CHEMV

Function/Subroutine Documentation
   subroutine chemv (character UPLO, integer N, complex ALPHA, complex,
       dimension(lda,*) A, integer LDA, complex, dimension(*) X, integer INCX,
       complex BETA, complex, dimension(*) Y, integer INCY)
       CHEMV

Purpose:
   CHEMV  performs the matrix-vector  operation

      y := alpha*A*x + beta*y,

   where alpha and beta are scalars, x and y are n element vectors and
   A is an n by n hermitian matrix.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the upper or lower
   triangular part of the array A is to be referenced as follows:

   UPLO = 'U' or 'u'   Only the upper triangular part of A
   is to be referenced.

   UPLO = 'L' or 'l'   Only the lower triangular part of A
   is to be referenced.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
   N must be at least zero.

 ALPHA
   ALPHA is COMPLEX
   On entry, ALPHA specifies the scalar alpha.

 A
   A is COMPLEX array of DIMENSION ( LDA, n ).
   Before entry with  UPLO = 'U' or 'u', the leading n by n
   upper triangular part of the array A must contain the upper
   triangular part of the hermitian matrix and the strictly
   lower triangular part of A is not referenced.

   Before entry with UPLO = 'L' or 'l', the leading n by n
   lower triangular part of the array A must contain the lower
```

```
  triangular part of the hermitian matrix and the strictly
  upper triangular part of A is not referenced.

  Note that the imaginary parts of the diagonal elements need
  not be set and are assumed to be zero.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, n ).

X
  X is COMPLEX array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the n
  element vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.

BETA
  BETA is COMPLEX
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.

Y
  Y is COMPLEX array of dimension at least ( 1 + ( n - 1 )*abs( INCY ) ).
  Before entry, the incremented array Y must contain the n
  element vector y. On exit, Y is overwritten by the updated vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of Y.
  INCY must not be zero.

  The vector and matrix arguments are not referenced when N = 0, or M = 0
```

———-

12.6.3 fortran code

— chemv.f —

```
*  ======================================================================
      SUBROUTINE CHEMV(UPLO,N,ALPHA,A,LDA,X,INCX,BETA,Y,INCY)
*
```

```
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX ALPHA,BETA
      INTEGER INCX,INCY,LDA,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),X(*),Y(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX ONE
      PARAMETER (ONE= (1.0E+0,0.0E+0))
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,KX,KY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG,MAX,REAL
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 5
      ELSE IF (INCX.EQ.0) THEN
          INFO = 7
      ELSE IF (INCY.EQ.0) THEN
          INFO = 10
```

```
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CHEMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     Set up the start points in  X  and  Y.
*
      IF (INCX.GT.0) THEN
          KX = 1
      ELSE
          KX = 1 - (N-1)*INCX
      END IF
      IF (INCY.GT.0) THEN
          KY = 1
      ELSE
          KY = 1 - (N-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the triangular part
*     of A.
*
*     First form  y := beta*y.
*
      IF (BETA.NE.ONE) THEN
          IF (INCY.EQ.1) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 10 I = 1,N
                      Y(I) = ZERO
   10             CONTINUE
              ELSE
                  DO 20 I = 1,N
                      Y(I) = BETA*Y(I)
   20             CONTINUE
              END IF
          ELSE
              IY = KY
              IF (BETA.EQ.ZERO) THEN
                  DO 30 I = 1,N
                      Y(IY) = ZERO
                      IY = IY + INCY
   30             CONTINUE
              ELSE
                  DO 40 I = 1,N
                      Y(IY) = BETA*Y(IY)
```

```
                    IY = IY + INCY
   40            CONTINUE
            END IF
         END IF
      END IF
      IF (ALPHA.EQ.ZERO) RETURN
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  y  when A is stored in upper triangle.
*
         IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
            DO 60 J = 1,N
               TEMP1 = ALPHA*X(J)
               TEMP2 = ZERO
               DO 50 I = 1,J - 1
                  Y(I) = Y(I) + TEMP1*A(I,J)
                  TEMP2 = TEMP2 + CONJG(A(I,J))*X(I)
   50          CONTINUE
               Y(J) = Y(J) + TEMP1*REAL(A(J,J)) + ALPHA*TEMP2
   60       CONTINUE
         ELSE
            JX = KX
            JY = KY
            DO 80 J = 1,N
               TEMP1 = ALPHA*X(JX)
               TEMP2 = ZERO
               IX = KX
               IY = KY
               DO 70 I = 1,J - 1
                  Y(IY) = Y(IY) + TEMP1*A(I,J)
                  TEMP2 = TEMP2 + CONJG(A(I,J))*X(IX)
                  IX = IX + INCX
                  IY = IY + INCY
   70          CONTINUE
               Y(JY) = Y(JY) + TEMP1*REAL(A(J,J)) + ALPHA*TEMP2
               JX = JX + INCX
               JY = JY + INCY
   80       CONTINUE
         END IF
      ELSE
*
*        Form  y  when A is stored in lower triangle.
*
         IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
            DO 100 J = 1,N
               TEMP1 = ALPHA*X(J)
               TEMP2 = ZERO
               Y(J) = Y(J) + TEMP1*REAL(A(J,J))
               DO 90 I = J + 1,N
                  Y(I) = Y(I) + TEMP1*A(I,J)
```

```
                    TEMP2 = TEMP2 + CONJG(A(I,J))*X(I)
   90             CONTINUE
                  Y(J) = Y(J) + ALPHA*TEMP2
  100         CONTINUE
          ELSE
              JX = KX
              JY = KY
              DO 120 J = 1,N
                  TEMP1 = ALPHA*X(JX)
                  TEMP2 = ZERO
                  Y(JY) = Y(JY) + TEMP1*REAL(A(J,J))
                  IX = JX
                  IY = JY
                  DO 110 I = J + 1,N
                      IX = IX + INCX
                      IY = IY + INCY
                      Y(IY) = Y(IY) + TEMP1*A(I,J)
                      TEMP2 = TEMP2 + CONJG(A(I,J))*X(IX)
  110             CONTINUE
                  Y(JY) = Y(JY) + ALPHA*TEMP2
                  JX = JX + INCX
                  JY = JY + INCY
  120         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of CHEMV .
*
      END
```

———-

12.6.4 lisp code

— BLAS 2 chemv —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)) (zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (ignorable one zero))
  (defun chemv (uplo n alpha a lda x incx beta y incy)
    (declare (type (array f2cl-lib:complex8 (*)) y x a)
             (type (f2cl-lib:complex8) beta alpha)
             (type (f2cl-lib:integer4) incy incx lda n)
             (type (string *) uplo))
```

```
(f2cl-lib:with-multi-array-data
    ((uplo character uplo-%data% uplo-%offset%)
     (a f2cl-lib:complex8 a-%data% a-%offset%)
     (x f2cl-lib:complex8 x-%data% x-%offset%)
     (y f2cl-lib:complex8 y-%data% y-%offset%))
  (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (kx 0) (ky 0)
         (temp1 #C(0.0 0.0)) (temp2 #C(0.0 0.0)))
    (declare (type (f2cl-lib:integer4) i info ix iy j jx jy kx ky)
             (type (f2cl-lib:complex8) temp1 temp2))
    (setf info 0)
    (cond
     ((and
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
      (setf info 1))
     ((< n 0) (setf info 2))
     ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info 5))
     ((= incx 0) (setf info 7)) ((= incy 0) (setf info 10)))
    (cond
     ((/= info 0)
      (multiple-value-bind (var-0 var-1)
          (xerbla "CHEMV " info)
        (declare (ignore var-0))
        (when var-1 (setf info var-1)))
      (go end_label)))
    (if (or (= n 0) (and (= alpha zero) (= beta one)))
        (go end_label))
    (cond ((> incx 0) (setf kx 1))
          (t
           (setf kx
                   (f2cl-lib:int-sub 1
                                     (f2cl-lib:int-mul
                                      (f2cl-lib:int-sub n 1) incx)))))
    (cond ((> incy 0) (setf ky 1))
          (t
           (setf ky
                   (f2cl-lib:int-sub 1
                                     (f2cl-lib:int-mul
                                      (f2cl-lib:int-sub n 1) incy)))))
```

```
(cond
 ((/= beta one)
  (cond
   ((= incy 1)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%) zero)
         label10)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                  (* beta
                     (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)))
         label20)))))
   (t (setf iy ky)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) zero)
          (setf iy (f2cl-lib:int-add iy incy))
         label30)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (* beta
                     (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)))
          (setf iy (f2cl-lib:int-add iy incy))
         label40))))))))
(if (= alpha zero)
    (go end_label))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame uplo "U")
    (declare (ignore var-1))
    (when var-0 (setf uplo var-0))
    ret-val)
  (cond
   ((and (= incx 1) (= incy 1))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
```

```
        (setf temp1
                (* alpha
                   (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
        (setf temp2 zero)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                       (* temp1
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%))))
            (setf temp2
                    (+ temp2
                       (*
                        (f2cl-lib:conjg
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%))
                        (f2cl-lib:fref x-%data% (i) ((1 *))
                                       x-%offset%))))
           label50))
        (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                   (* temp1
                      (f2cl-lib:freal
                       (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                      a-%offset%)))
                   (* alpha temp2)))
      label60)))
    (t (setf jx kx) (setf jy ky)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
        (setf temp1
                (* alpha
                   (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
        (setf temp2 zero)
        (setf ix kx)
        (setf iy ky)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (* temp1
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%))))
            (setf temp2
```

```
                    (+ temp2
                       (*
                        (f2cl-lib:conjg
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%))
                        (f2cl-lib:fref x-%data% (ix) ((1 *))
                                       x-%offset%))))
            (setf ix (f2cl-lib:int-add ix incx))
            (setf iy (f2cl-lib:int-add iy incy))
          label70))
        (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                   (* temp1
                      (f2cl-lib:freal
                       (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                      a-%offset%)))
                   (* alpha temp2)))
        (setf jx (f2cl-lib:int-add jx incx))
        (setf jy (f2cl-lib:int-add jy incy))
      label80)))))
(t
 (cond
  ((and (= incx 1) (= incy 1))
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp1
               (* alpha
                  (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
       (setf temp2 zero)
       (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
               (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                  (* temp1
                     (f2cl-lib:freal
                      (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%)))))
       (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                   (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                      (* temp1
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%))))
           (setf temp2
                   (+ temp2
                      (*
                       (f2cl-lib:conjg
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))
```

```
                          (f2cl-lib:fref x-%data% (i) ((1 *))
                                         x-%offset%))))
            label90))
          (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                  (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                     (* alpha temp2)))
         label100)))
    (t (setf jx kx) (setf jy ky)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp1
                 (* alpha
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
         (setf temp2 zero)
         (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                 (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                    (* temp1
                       (f2cl-lib:freal
                        (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                       a-%offset%)))))
         (setf ix jx)
         (setf iy jy)
         (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf ix (f2cl-lib:int-add ix incx))
             (setf iy (f2cl-lib:int-add iy incy))
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (* temp1
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%))))
             (setf temp2
                     (+ temp2
                        (*
                         (f2cl-lib:conjg
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%))
                         (f2cl-lib:fref x-%data% (ix) ((1 *))
                                        x-%offset%))))
            label110))
         (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                 (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                    (* alpha temp2)))
         (setf jx (f2cl-lib:int-add jx incx))
         (setf jy (f2cl-lib:int-add jy incy))
        label120))))))
 (go end_label)
end_label
```

```
        (return (values uplo nil nil nil nil nil nil nil nil nil))))))
```

———-

12.7 cher - hermitian rank 1 $\alpha xx^H + A$

12.7.1 Axiom unit tests

— **cher.input** —

```
)set break resume
)sys rm -f cher.output
)spool cher.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

12.7.2 Axiom help page

— **cher.help** —

```
======================================================================
cher examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cher (UPLO, N, ALPHA, X, INCX, A, LDA)
           CHER

Function/Subroutine Documentation
   subroutine cher (character UPLO, integer N, real ALPHA, complex,
       dimension(*) X, integer INCX, complex, dimension(lda,*) A, integer LDA)
       CHER
```

```
Purpose:
   CHER  performs the hermitian rank 1 operation

      A := alpha*x*x**H + A,

   where alpha is a real scalar, x is an n element vector and A is an
   n by n hermitian matrix.

Parameters:

   UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the upper or lower
   triangular part of the array A is to be referenced as follows:

   UPLO = 'U' or 'u'   Only the upper triangular part of A
   is to be referenced.

   UPLO = 'L' or 'l'   Only the lower triangular part of A
   is to be referenced.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
   N must be at least zero.

 ALPHA
   ALPHA is REAL
   On entry, ALPHA specifies the scalar alpha.

 X
   X is COMPLEX array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
   Before entry, the incremented array X must contain the n
   element vector x.

 INCX
   INCX is INTEGER
   On entry, INCX specifies the increment for the elements of X.
   INCX must not be zero.

 A
   A is COMPLEX array of DIMENSION ( LDA, n ).

   Before entry with  UPLO = 'U' or 'u', the leading n by n
   upper triangular part of the array A must contain the upper
   triangular part of the hermitian matrix and the strictly
   lower triangular part of A is not referenced. On exit, the
   upper triangular part of the array A is overwritten by the
   upper triangular part of the updated matrix.
```

```
  Before entry with UPLO = 'L' or 'l', the leading n by n
  lower triangular part of the array A must contain the lower
  triangular part of the hermitian matrix and the strictly
  upper triangular part of A is not referenced. On exit, the
  lower triangular part of the array A is overwritten by the
  lower triangular part of the updated matrix.

  Note that the imaginary parts of the diagonal elements need
  not be set, they are assumed to be zero, and on exit they
  are set to zero.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, n ).
```

12.7.3 fortran code

— cher.f —

```
*  ==================================================================
      SUBROUTINE CHER(UPLO,N,ALPHA,X,INCX,A,LDA)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL ALPHA
      INTEGER INCX,LDA,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),X(*)
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*     .. Local Scalars ..
```

```
      COMPLEX TEMP
      INTEGER I,INFO,IX,J,JX,KX
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG,MAX,REAL
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 7
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CHER  ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (ALPHA.EQ.REAL(ZERO))) RETURN
*
*     Set the start point in X if the increment is not unity.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the triangular part
*     of A.
*
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  A  when A is stored in upper triangle.
```

```
*
         IF (INCX.EQ.1) THEN
             DO 20 J = 1,N
                 IF (X(J).NE.ZERO) THEN
                     TEMP = ALPHA*CONJG(X(J))
                     DO 10 I = 1,J - 1
                         A(I,J) = A(I,J) + X(I)*TEMP
   10                CONTINUE
                     A(J,J) = REAL(A(J,J)) + REAL(X(J)*TEMP)
                 ELSE
                     A(J,J) = REAL(A(J,J))
                 END IF
   20        CONTINUE
         ELSE
             JX = KX
             DO 40 J = 1,N
                 IF (X(JX).NE.ZERO) THEN
                     TEMP = ALPHA*CONJG(X(JX))
                     IX = KX
                     DO 30 I = 1,J - 1
                         A(I,J) = A(I,J) + X(IX)*TEMP
                         IX = IX + INCX
   30                CONTINUE
                     A(J,J) = REAL(A(J,J)) + REAL(X(JX)*TEMP)
                 ELSE
                     A(J,J) = REAL(A(J,J))
                 END IF
                 JX = JX + INCX
   40        CONTINUE
         END IF
      ELSE
*
*        Form  A  when A is stored in lower triangle.
*
         IF (INCX.EQ.1) THEN
             DO 60 J = 1,N
                 IF (X(J).NE.ZERO) THEN
                     TEMP = ALPHA*CONJG(X(J))
                     A(J,J) = REAL(A(J,J)) + REAL(TEMP*X(J))
                     DO 50 I = J + 1,N
                         A(I,J) = A(I,J) + X(I)*TEMP
   50                CONTINUE
                 ELSE
                     A(J,J) = REAL(A(J,J))
                 END IF
   60        CONTINUE
         ELSE
             JX = KX
             DO 80 J = 1,N
                 IF (X(JX).NE.ZERO) THEN
```

```
                    TEMP = ALPHA*CONJG(X(JX))
                    A(J,J) = REAL(A(J,J)) + REAL(TEMP*X(JX))
                    IX = JX
                    DO 70 I = J + 1,N
                        IX = IX + INCX
                        A(I,J) = A(I,J) + X(IX)*TEMP
   70               CONTINUE
                ELSE
                    A(J,J) = REAL(A(J,J))
                END IF
                JX = JX + INCX
   80         CONTINUE
           END IF
      END IF
*
      RETURN
*
*     End of CHER  .
*
      END
```

———-

12.7.4 lisp code

— BLAS 2 cher —

```
(let* ((zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) zero)
           (ignorable zero))
  (defun cher (uplo n alpha x incx a lda)
    (declare (type (array f2cl-lib:complex8 (*)) a x)
             (type (single-float) alpha)
             (type (f2cl-lib:integer4) lda incx n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%))
      (prog ((i 0) (info 0) (ix 0) (j 0) (jx 0) (kx 0) (temp #C(0.0 0.0)))
        (declare (type (f2cl-lib:integer4) i info ix j jx kx)
                 (type (f2cl-lib:complex8) temp))
        (setf info 0)
        (cond
         ((and
           (not
            (multiple-value-bind (ret-val var-0 var-1)
```

```
          (lsame uplo "U")
        (declare (ignore var-1))
        (when var-0 (setf uplo var-0))
        ret-val))
     (not
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame uplo "L")
        (declare (ignore var-1))
        (when var-0 (setf uplo var-0))
        ret-val)))
    (setf info 1))
   ((< n 0) (setf info 2)) ((= incx 0) (setf info 5))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info 7)))
  (cond
   ((/= info 0)
    (multiple-value-bind (var-0 var-1)
        (xerbla "CHER  " info)
      (declare (ignore var-0))
      (when var-1 (setf info var-1)))
    (go end_label)))
  (if (or (= n 0) (= alpha (f2cl-lib:freal zero)))
      (go end_label))
  (cond
   ((<= incx 0)
    (setf kx
            (f2cl-lib:int-sub 1
                              (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                incx))))
   ((/= incx 1) (setf kx 1)))
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)
    (cond
     ((= incx 1)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
           ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
            (setf temp
                    (* alpha
                       (f2cl-lib:conjg
                        (f2cl-lib:fref x-%data% (j) ((1 *))
                                       x-%offset%))))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i
```

```
                                (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                            nil)
                (tagbody
                  (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%)
                        (+
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)
                         (*
                          (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                          temp)))
                 label10))
               (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                    a-%offset%)
                     (coerce
                      (+
                       (f2cl-lib:freal
                        (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                       a-%offset%))
                       (f2cl-lib:freal
                        (* (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                           temp)))
                      'f2cl-lib:complex8)))
              (t
               (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                    a-%offset%)
                     (coerce
                      (f2cl-lib:freal
                       (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                      a-%offset%))
                      'f2cl-lib:complex8))))
            label20)))
        (t (setf jx kx)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (cond
              ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
               (setf temp
                     (* alpha
                        (f2cl-lib:conjg
                         (f2cl-lib:fref x-%data% (jx) ((1 *))
                                        x-%offset%))))
               (setf ix kx)
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i
                                 (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                              nil)
                 (tagbody
                   (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
```

```
                                        a-%offset%)
                                 (+
                                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                                 a-%offset%)
                                  (*
                                   (f2cl-lib:fref x-%data% (ix) ((1 *))
                                                  x-%offset%)
                                   temp)))
                          (setf ix (f2cl-lib:int-add ix incx))
                        label30))
                    (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                         a-%offset%)
                          (coerce
                           (+
                            (f2cl-lib:freal
                             (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                            a-%offset%))
                            (f2cl-lib:freal
                             (*
                              (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                              temp)))
                           'f2cl-lib:complex8)))
                  (t
                   (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                        a-%offset%)
                         (coerce
                          (f2cl-lib:freal
                           (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                          a-%offset%))
                          'f2cl-lib:complex8))))
                 (setf jx (f2cl-lib:int-add jx incx))
                label40)))))
        (t
         (cond
          ((= incx 1)
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (cond
                ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
                 (setf temp
                         (* alpha
                            (f2cl-lib:conjg
                             (f2cl-lib:fref x-%data% (j) ((1 *))
                                            x-%offset%))))
                 (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                      a-%offset%)
                       (coerce
                        (+
                         (f2cl-lib:freal
```

```
                    (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                   a-%offset%))
                  (f2cl-lib:freal
                   (* temp
                      (f2cl-lib:fref x-%data% (j) ((1 *))
                                     x-%offset%))))
                 'f2cl-lib:complex8))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                       (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%)
                  (+
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                   (*
                    (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                    temp)))
           label50)))
       (t
        (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                             a-%offset%)
              (coerce
               (f2cl-lib:freal
                (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                               a-%offset%))
               'f2cl-lib:complex8))))
    label60)))
(t (setf jx kx)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
      ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
       (setf temp
               (* alpha
                  (f2cl-lib:conjg
                   (f2cl-lib:fref x-%data% (jx) ((1 *))
                                  x-%offset%))))
       (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                            a-%offset%)
               (coerce
                (+
                 (f2cl-lib:freal
                  (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                 a-%offset%))
                 (f2cl-lib:freal
                  (* temp
                     (f2cl-lib:fref x-%data% (jx) ((1 *))
```

```
                                        x-%offset%))))
                    'f2cl-lib:complex8))
                (setf ix jx)
                (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                               (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf ix (f2cl-lib:int-add ix incx))
                    (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                          (+
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%)
                           (*
                            (f2cl-lib:fref x-%data% (ix) ((1 *))
                                           x-%offset%)
                            temp)))
                   label70)))
               (t
                (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%)
                      (coerce
                       (f2cl-lib:freal
                        (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                       a-%offset%))
                       'f2cl-lib:complex8))))
             (setf jx (f2cl-lib:int-add jx incx))
            label80))))))
  (go end_label)
 end_label
  (return (values uplo nil nil nil nil nil nil))))))
```

———-

12.8 cher2 - hermitian rank 2 $\alpha xy^H + conjg(\alpha)yx^H + A$

12.8.1 Axiom unit tests

— cher2.input —

```
)set break resume
)sys rm -f cher2.output
)spool cher2.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

12.8.2 Axiom help page

— **cher2.help** —

```
==========================================================================
cher2 examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cher2 (UPLO, N, ALPHA, X, INCX, Y, INCY, A, LDA)
           CHER2

Function/Subroutine Documentation
   subroutine cher2 (character UPLO, integer N, complex ALPHA, complex,
       dimension(*) X, integer INCX, complex, dimension(*) Y, integer INCY,
       complex, dimension(lda,*) A, integer LDA)
       CHER2

Purpose:
   CHER2  performs the hermitian rank 2 operation

      A := alpha*x*y**H + conjg( alpha )*y*x**H + A,

   where alpha is a scalar, x and y are n element vectors and A is an n
   by n hermitian matrix.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the upper or lower
   triangular part of the array A is to be referenced as follows:

   UPLO = 'U' or 'u'   Only the upper triangular part of A
   is to be referenced.

   UPLO = 'L' or 'l'   Only the lower triangular part of A
```

```
  is to be referenced.

N
  N is INTEGER
  On entry, N specifies the order of the matrix A.
  N must be at least zero.

ALPHA
  ALPHA is COMPLEX
  On entry, ALPHA specifies the scalar alpha.

X
  X is COMPLEX array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the n
  element vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.

Y
  Y is COMPLEX array of dimension at least ( 1 + ( n - 1 )*abs( INCY ) ).
  Before entry, the incremented array Y must contain the n
  element vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of Y.
  INCY must not be zero.

A
  A is COMPLEX array of DIMENSION ( LDA, n ).

  Before entry with  UPLO = 'U' or 'u', the leading n by n
  upper triangular part of the array A must contain the upper
  triangular part of the hermitian matrix and the strictly
  lower triangular part of A is not referenced. On exit, the
  upper triangular part of the array A is overwritten by the
  upper triangular part of the updated matrix.

  Before entry with UPLO = 'L' or 'l', the leading n by n
  lower triangular part of the array A must contain the lower
  triangular part of the hermitian matrix and the strictly
  upper triangular part of A is not referenced. On exit, the
  lower triangular part of the array A is overwritten by the
  lower triangular part of the updated matrix.

  Note that the imaginary parts of the diagonal elements need
  not be set, they are assumed to be zero, and on exit they
```

```
  are set to zero.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, n ).
```

12.8.3 fortran code

— cher2.f —

```
*  =====================================================================
      SUBROUTINE CHER2(UPLO,N,ALPHA,X,INCX,Y,INCY,A,LDA)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX ALPHA
      INTEGER INCX,INCY,LDA,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),X(*),Y(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,KX,KY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
```

```
*     .. Intrinsic Functions ..
      INTRINSIC CONJG,MAX,REAL
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      ELSE IF (INCY.EQ.0) THEN
          INFO = 7
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 9
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CHER2 ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (ALPHA.EQ.ZERO)) RETURN
*
*     Set up the start points in X and Y if the increments are not both
*     unity.
*
      IF ((INCX.NE.1) .OR. (INCY.NE.1)) THEN
          IF (INCX.GT.0) THEN
              KX = 1
          ELSE
              KX = 1 - (N-1)*INCX
          END IF
          IF (INCY.GT.0) THEN
              KY = 1
          ELSE
              KY = 1 - (N-1)*INCY
          END IF
          JX = KX
          JY = KY
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the triangular part
*     of A.
*
      IF (LSAME(UPLO,'U')) THEN
```

```
*
*        Form  A  when A is stored in the upper triangle.
*
         IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
             DO 20 J = 1,N
                 IF ((X(J).NE.ZERO) .OR. (Y(J).NE.ZERO)) THEN
                     TEMP1 = ALPHA*CONJG(Y(J))
                     TEMP2 = CONJG(ALPHA*X(J))
                     DO 10 I = 1,J - 1
                         A(I,J) = A(I,J) + X(I)*TEMP1 + Y(I)*TEMP2
   10                CONTINUE
                     A(J,J) = REAL(A(J,J)) +
     +                       REAL(X(J)*TEMP1+Y(J)*TEMP2)
                 ELSE
                     A(J,J) = REAL(A(J,J))
                 END IF
   20        CONTINUE
         ELSE
             DO 40 J = 1,N
                 IF ((X(JX).NE.ZERO) .OR. (Y(JY).NE.ZERO)) THEN
                     TEMP1 = ALPHA*CONJG(Y(JY))
                     TEMP2 = CONJG(ALPHA*X(JX))
                     IX = KX
                     IY = KY
                     DO 30 I = 1,J - 1
                         A(I,J) = A(I,J) + X(IX)*TEMP1 + Y(IY)*TEMP2
                         IX = IX + INCX
                         IY = IY + INCY
   30                CONTINUE
                     A(J,J) = REAL(A(J,J)) +
     +                       REAL(X(JX)*TEMP1+Y(JY)*TEMP2)
                 ELSE
                     A(J,J) = REAL(A(J,J))
                 END IF
                 JX = JX + INCX
                 JY = JY + INCY
   40        CONTINUE
         END IF
     ELSE
*
*        Form  A  when A is stored in the lower triangle.
*
         IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
             DO 60 J = 1,N
                 IF ((X(J).NE.ZERO) .OR. (Y(J).NE.ZERO)) THEN
                     TEMP1 = ALPHA*CONJG(Y(J))
                     TEMP2 = CONJG(ALPHA*X(J))
                     A(J,J) = REAL(A(J,J)) +
     +                       REAL(X(J)*TEMP1+Y(J)*TEMP2)
                     DO 50 I = J + 1,N
```

```
                        A(I,J) = A(I,J) + X(I)*TEMP1 + Y(I)*TEMP2
   50                CONTINUE
                 ELSE
                     A(J,J) = REAL(A(J,J))
                 END IF
   60         CONTINUE
          ELSE
              DO 80 J = 1,N
                  IF ((X(JX).NE.ZERO) .OR. (Y(JY).NE.ZERO)) THEN
                      TEMP1 = ALPHA*CONJG(Y(JY))
                      TEMP2 = CONJG(ALPHA*X(JX))
                      A(J,J) = REAL(A(J,J)) +
     +                         REAL(X(JX)*TEMP1+Y(JY)*TEMP2)
                      IX = JX
                      IY = JY
                      DO 70 I = J + 1,N
                          IX = IX + INCX
                          IY = IY + INCY
                          A(I,J) = A(I,J) + X(IX)*TEMP1 + Y(IY)*TEMP2
   70                 CONTINUE
                 ELSE
                     A(J,J) = REAL(A(J,J))
                 END IF
                 JX = JX + INCX
                 JY = JY + INCY
   80         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of CHER2 .
*
      END
```

———-

12.8.4 lisp code

— BLAS 2 cher2 —

```
(let* ((zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) zero)
           (ignorable zero))
  (defun cher2 (uplo n alpha x incx y incy a lda)
    (declare (type (array f2cl-lib:complex8 (*)) a y x)
             (type (f2cl-lib:complex8) alpha)
```

```
          (type (f2cl-lib:integer4) lda incy incx n)
          (type (string *) uplo))
(f2cl-lib:with-multi-array-data
    ((uplo character uplo-%data% uplo-%offset%)
     (x f2cl-lib:complex8 x-%data% x-%offset%)
     (y f2cl-lib:complex8 y-%data% y-%offset%)
     (a f2cl-lib:complex8 a-%data% a-%offset%))
  (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (kx 0) (ky 0)
         (temp1 #C(0.0 0.0)) (temp2 #C(0.0 0.0)))
    (declare (type (f2cl-lib:integer4) i info ix iy j jx jy kx ky)
             (type (f2cl-lib:complex8) temp1 temp2))
    (setf info 0)
    (cond
     ((and
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
      (setf info 1))
     ((< n 0) (setf info 2)) ((= incx 0) (setf info 5))
     ((= incy 0) (setf info 7))
     ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info 9)))
    (cond
     ((/= info 0)
      (multiple-value-bind (var-0 var-1)
          (xerbla "CHER2 " info)
        (declare (ignore var-0))
        (when var-1 (setf info var-1)))
      (go end_label)))
    (if (or (= n 0) (= alpha zero))
        (go end_label))
    (cond
     ((or (/= incx 1) (/= incy 1))
      (cond ((> incx 0) (setf kx 1))
            (t
             (setf kx
                     (f2cl-lib:int-sub 1
                                       (f2cl-lib:int-mul
                                        (f2cl-lib:int-sub n 1) incx)))))
      (cond ((> incy 0) (setf ky 1))
            (t
```

```
          (setf ky
                 (f2cl-lib:int-sub 1
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-sub n 1) incy)))))
  (setf jx kx) (setf jy ky)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame uplo "U")
    (declare (ignore var-1))
    (when var-0 (setf uplo var-0))
    ret-val)
  (cond
   ((and (= incx 1) (= incy 1))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
         ((or (/= (f2cl-lib:fref x (j) ((1 *))) zero)
              (/= (f2cl-lib:fref y (j) ((1 *))) zero))
          (setf temp1
                  (* alpha
                     (f2cl-lib:conjg
                      (f2cl-lib:fref y-%data% (j) ((1 *))
                                     y-%offset%))))
          (setf temp2
                  (f2cl-lib:conjg
                   (* alpha
                      (f2cl-lib:fref x-%data% (j) ((1 *))
                                     x-%offset%))))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%)
                      (+
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                       (*
                        (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                        temp1)
                       (*
                        (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                        temp2)))
             label10))
          (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                               a-%offset%)
                  (coerce
                   (+
```

```
                    (f2cl-lib:freal
                     (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                    a-%offset%))
                    (f2cl-lib:freal
                     (+
                      (*
                       (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                       temp1)
                      (*
                       (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                       temp2))))
                   'f2cl-lib:complex8)))
        (t
         (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                              a-%offset%)
                 (coerce
                  (f2cl-lib:freal
                   (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                  a-%offset%))
                  'f2cl-lib:complex8))))
    label20)))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
      ((or (/= (f2cl-lib:fref x (jx) ((1 *))) zero)
           (/= (f2cl-lib:fref y (jy) ((1 *))) zero))
       (setf temp1
               (* alpha
                  (f2cl-lib:conjg
                   (f2cl-lib:fref y-%data% (jy) ((1 *))
                                  y-%offset%))))
       (setf temp2
               (f2cl-lib:conjg
                (* alpha
                   (f2cl-lib:fref x-%data% (jx) ((1 *))
                                  x-%offset%))))
       (setf ix kx) (setf iy ky)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i
                         (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                      nil)
         (tagbody
           (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                   (+
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%)
                    (*
```

```
                        (f2cl-lib:fref x-%data% (ix) ((1 *))
                                       x-%offset%)
                        temp1)
                       (*
                        (f2cl-lib:fref y-%data% (iy) ((1 *))
                                       y-%offset%)
                        temp2)))
                (setf ix (f2cl-lib:int-add ix incx))
                (setf iy (f2cl-lib:int-add iy incy))
               label30))
            (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                 a-%offset%)
                  (coerce
                   (+
                    (f2cl-lib:freal
                     (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                    a-%offset%))
                    (f2cl-lib:freal
                     (+
                      (*
                       (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                       temp1)
                      (*
                       (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                       temp2))))
                   'f2cl-lib:complex8)))
          (t
           (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                a-%offset%)
                 (coerce
                  (f2cl-lib:freal
                   (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                  a-%offset%))
                  'f2cl-lib:complex8))))
         (setf jx (f2cl-lib:int-add jx incx))
         (setf jy (f2cl-lib:int-add jy incy))
        label40)))))
(t
 (cond
  ((and (= incx 1) (= incy 1))
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((or (/= (f2cl-lib:fref x (j) ((1 *))) zero)
             (/= (f2cl-lib:fref y (j) ((1 *))) zero))
         (setf temp1
                 (* alpha
                    (f2cl-lib:conjg
                     (f2cl-lib:fref y-%data% (j) ((1 *))
```

```
                                y-%offset%))))
        (setf temp2
                (f2cl-lib:conjg
                 (* alpha
                    (f2cl-lib:fref x-%data% (j) ((1 *))
                                   x-%offset%))))
        (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                             a-%offset%)
                (coerce
                 (+
                  (f2cl-lib:freal
                   (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                  a-%offset%))
                  (f2cl-lib:freal
                   (+
                    (*
                     (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                     temp1)
                    (*
                     (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                     temp2))))
                 'f2cl-lib:complex8))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                       (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%)
                    (+
                     (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%)
                     (*
                      (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                      temp1)
                     (*
                      (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                      temp2)))
            label50)))
       (t
        (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                             a-%offset%)
                (coerce
                 (f2cl-lib:freal
                  (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                 a-%offset%))
                 'f2cl-lib:complex8))))
    label60)))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
```

```
(tagbody
  (cond
   ((or (/= (f2cl-lib:fref x (jx) ((1 *))) zero)
        (/= (f2cl-lib:fref y (jy) ((1 *))) zero))
    (setf temp1
            (* alpha
               (f2cl-lib:conjg
                (f2cl-lib:fref y-%data% (jy) ((1 *))
                               y-%offset%))))
    (setf temp2
            (f2cl-lib:conjg
             (* alpha
                (f2cl-lib:fref x-%data% (jx) ((1 *))
                               x-%offset%))))
    (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                         a-%offset%)
            (coerce
             (+
              (f2cl-lib:freal
               (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                              a-%offset%))
              (f2cl-lib:freal
               (+
                (*
                 (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                 temp1)
                (*
                 (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                 temp2))))
             'f2cl-lib:complex8))
    (setf ix jx) (setf iy jy)
    (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                   (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf ix (f2cl-lib:int-add ix incx))
        (setf iy (f2cl-lib:int-add iy incy))
        (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                             a-%offset%)
                (+
                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                 (*
                  (f2cl-lib:fref x-%data% (ix) ((1 *))
                                 x-%offset%)
                  temp1)
                 (*
                  (f2cl-lib:fref y-%data% (iy) ((1 *))
                                 y-%offset%)
                  temp2)))
```

```
                    label70)))
                 (t
                  (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                       a-%offset%)
                        (coerce
                         (f2cl-lib:freal
                          (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                         a-%offset%))
                         'f2cl-lib:complex8))))
                 (setf jx (f2cl-lib:int-add jx incx))
                 (setf jy (f2cl-lib:int-add jy incy))
                label80))))))
        (go end_label)
       end_label
        (return (values uplo nil nil nil nil nil nil nil nil))))))
```

———-

12.9 chpmv - hermitian matrix-vector $\alpha Ax + \beta y$

12.9.1 Axiom unit tests

— **chpmv.input** —

```
)set break resume
)sys rm -f chpmv.output
)spool chpmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

12.9.2 Axiom help page

— **chpmv.help** —

```
========================================================================
chpmv examples
========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine chpmv (UPLO, N, ALPHA, AP, X, INCX, BETA, Y, INCY)
           CHPMV

Function/Subroutine Documentation
   subroutine chpmv (character UPLO, integer N, complex ALPHA, complex,
       dimension(*) AP, complex, dimension(*) X, integer INCX, complex BETA,
       complex, dimension(*) Y, integer INCY)
       CHPMV

Purpose:
   CHPMV  performs the matrix-vector operation

     y := alpha*A*x + beta*y,

   where alpha and beta are scalars, x and y are n element vectors and
   A is an n by n hermitian matrix, supplied in packed form.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the upper or lower
   triangular part of the matrix A is supplied in the packed
   array AP as follows:

   UPLO = 'U' or 'u'   The upper triangular part of A is supplied in AP.

   UPLO = 'L' or 'l'   The lower triangular part of A is supplied in AP.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
   N must be at least zero.

 ALPHA
   ALPHA is COMPLEX
   On entry, ALPHA specifies the scalar alpha.

 AP
   AP is COMPLEX array of DIMENSION at least ( ( n*( n + 1 ) )/2 ).

   Before entry with UPLO = 'U' or 'u', the array AP must
   contain the upper triangular part of the hermitian matrix
```

```
  packed sequentially, column by column, so that AP( 1 )
  contains a( 1, 1 ), AP( 2 ) and AP( 3 ) contain a( 1, 2 )
  and a( 2, 2 ) respectively, and so on.

  Before entry with UPLO = 'L' or 'l', the array AP must
  contain the lower triangular part of the hermitian matrix
  packed sequentially, column by column, so that AP( 1 )
  contains a( 1, 1 ), AP( 2 ) and AP( 3 ) contain a( 2, 1 )
  and a( 3, 1 ) respectively, and so on.

  Note that the imaginary parts of the diagonal elements need
  not be set and are assumed to be zero.

X
  X is COMPLEX array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the n
  element vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.

BETA
  BETA is COMPLEX
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.

Y
  Y is COMPLEX array of dimension at least ( 1 + ( n - 1 )*abs( INCY ) ).
  Before entry, the incremented array Y must contain the n
  element vector y. On exit, Y is overwritten by the updated vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of Y.
  INCY must not be zero.

  The vector and matrix arguments are not referenced when N = 0, or M = 0
```

———-

12.9.3 fortran code

— chpmv.f —

```
*  ====================================================================
```

```
      SUBROUTINE CHPMV(UPLO,N,ALPHA,AP,X,INCX,BETA,Y,INCY)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX ALPHA,BETA
      INTEGER INCX,INCY,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX AP(*),X(*),Y(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX ONE
      PARAMETER (ONE= (1.0E+0,0.0E+0))
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,K,KK,KX,KY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG,REAL
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 6
      ELSE IF (INCY.EQ.0) THEN
          INFO = 9
```

```
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CHPMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     Set up the start points in  X  and  Y.
*
      IF (INCX.GT.0) THEN
          KX = 1
      ELSE
          KX = 1 - (N-1)*INCX
      END IF
      IF (INCY.GT.0) THEN
          KY = 1
      ELSE
          KY = 1 - (N-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of the array AP
*     are accessed sequentially with one pass through AP.
*
*     First form  y := beta*y.
*
      IF (BETA.NE.ONE) THEN
          IF (INCY.EQ.1) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 10 I = 1,N
                      Y(I) = ZERO
   10             CONTINUE
              ELSE
                  DO 20 I = 1,N
                      Y(I) = BETA*Y(I)
   20             CONTINUE
              END IF
          ELSE
              IY = KY
              IF (BETA.EQ.ZERO) THEN
                  DO 30 I = 1,N
                      Y(IY) = ZERO
                      IY = IY + INCY
   30             CONTINUE
              ELSE
                  DO 40 I = 1,N
                      Y(IY) = BETA*Y(IY)
                      IY = IY + INCY
```

```
   40             CONTINUE
                END IF
            END IF
        END IF
        IF (ALPHA.EQ.ZERO) RETURN
        KK = 1
        IF (LSAME(UPLO,'U')) THEN
*
*          Form  y  when AP contains the upper triangle.
*
            IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
                DO 60 J = 1,N
                    TEMP1 = ALPHA*X(J)
                    TEMP2 = ZERO
                    K = KK
                    DO 50 I = 1,J - 1
                        Y(I) = Y(I) + TEMP1*AP(K)
                        TEMP2 = TEMP2 + CONJG(AP(K))*X(I)
                        K = K + 1
   50               CONTINUE
                    Y(J) = Y(J) + TEMP1*REAL(AP(KK+J-1)) + ALPHA*TEMP2
                    KK = KK + J
   60           CONTINUE
            ELSE
                JX = KX
                JY = KY
                DO 80 J = 1,N
                    TEMP1 = ALPHA*X(JX)
                    TEMP2 = ZERO
                    IX = KX
                    IY = KY
                    DO 70 K = KK,KK + J - 2
                        Y(IY) = Y(IY) + TEMP1*AP(K)
                        TEMP2 = TEMP2 + CONJG(AP(K))*X(IX)
                        IX = IX + INCX
                        IY = IY + INCY
   70               CONTINUE
                    Y(JY) = Y(JY) + TEMP1*REAL(AP(KK+J-1)) + ALPHA*TEMP2
                    JX = JX + INCX
                    JY = JY + INCY
                    KK = KK + J
   80           CONTINUE
            END IF
        ELSE
*
*          Form  y  when AP contains the lower triangle.
*
            IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
                DO 100 J = 1,N
                    TEMP1 = ALPHA*X(J)
```

```
                TEMP2 = ZERO
                Y(J) = Y(J) + TEMP1*REAL(AP(KK))
                K = KK + 1
                DO 90 I = J + 1,N
                    Y(I) = Y(I) + TEMP1*AP(K)
                    TEMP2 = TEMP2 + CONJG(AP(K))*X(I)
                    K = K + 1
   90           CONTINUE
                Y(J) = Y(J) + ALPHA*TEMP2
                KK = KK + (N-J+1)
  100       CONTINUE
        ELSE
            JX = KX
            JY = KY
            DO 120 J = 1,N
                TEMP1 = ALPHA*X(JX)
                TEMP2 = ZERO
                Y(JY) = Y(JY) + TEMP1*REAL(AP(KK))
                IX = JX
                IY = JY
                DO 110 K = KK + 1,KK + N - J
                    IX = IX + INCX
                    IY = IY + INCY
                    Y(IY) = Y(IY) + TEMP1*AP(K)
                    TEMP2 = TEMP2 + CONJG(AP(K))*X(IX)
  110           CONTINUE
                Y(JY) = Y(JY) + ALPHA*TEMP2
                JX = JX + INCX
                JY = JY + INCY
                KK = KK + (N-J+1)
  120       CONTINUE
        END IF
    END IF
*
    RETURN
*
*   End of CHPMV .
*
    END
```

———-

12.9.4 lisp code

— BLAS 2 chpmv —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)) (zero (f2cl-lib:cmplx 0.0 0.0)))
```

```
(declare (type (f2cl-lib:complex8) one)
         (type (f2cl-lib:complex8) zero)
         (ignorable one zero))
(defun chpmv (uplo n alpha ap x incx beta y incy)
  (declare (type (array f2cl-lib:complex8 (*)) y x ap)
           (type (f2cl-lib:complex8) beta alpha)
           (type (f2cl-lib:integer4) incy incx n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (ap f2cl-lib:complex8 ap-%data% ap-%offset%)
       (x f2cl-lib:complex8 x-%data% x-%offset%)
       (y f2cl-lib:complex8 y-%data% y-%offset%))
    (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (k 0) (kk 0)
           (kx 0) (ky 0) (temp1 #C(0.0 0.0)) (temp2 #C(0.0 0.0)))
      (declare (type (f2cl-lib:integer4) i info ix iy j jx jy k kk kx ky)
               (type (f2cl-lib:complex8) temp1 temp2))
      (setf info 0)
      (cond
       ((and
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
        (setf info 1))
       ((< n 0) (setf info 2)) ((= incx 0) (setf info 6))
       ((= incy 0) (setf info 9)))
      (cond
       ((/= info 0)
        (multiple-value-bind (var-0 var-1)
            (xerbla "CHPMV " info)
          (declare (ignore var-0))
          (when var-1 (setf info var-1)))
        (go end_label)))
      (if (or (= n 0) (and (= alpha zero) (= beta one)))
          (go end_label))
      (cond ((> incx 0) (setf kx 1))
            (t
             (setf kx
                     (f2cl-lib:int-sub 1
                                       (f2cl-lib:int-mul
                                        (f2cl-lib:int-sub n 1) incx)))))
```

```
(cond ((> incy 0) (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incy)))))
(cond
 ((/= beta one)
  (cond
   ((= incy 1)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%) zero)
         label10)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                  (* beta
                     (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)))
         label20)))))
   (t (setf iy ky)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) zero)
          (setf iy (f2cl-lib:int-add iy incy))
         label30)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (* beta
                     (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)))
          (setf iy (f2cl-lib:int-add iy incy))
         label40))))))))
(if (= alpha zero)
    (go end_label))
(setf kk 1)
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame uplo "U")
    (declare (ignore var-1))
```

```
  (when var-0 (setf uplo var-0))
  ret-val)
(cond
 ((and (= incx 1) (= incy 1))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf temp1
              (* alpha
                 (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
      (setf temp2 zero)
      (setf k kk)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                  (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     (* temp1
                        (f2cl-lib:fref ap-%data% (k) ((1 *))
                                       ap-%offset%))))
          (setf temp2
                  (+ temp2
                     (*
                      (f2cl-lib:conjg
                       (f2cl-lib:fref ap-%data% (k) ((1 *))
                                      ap-%offset%))
                      (f2cl-lib:fref x-%data% (i) ((1 *))
                                     x-%offset%))))
          (setf k (f2cl-lib:int-add k 1))
         label50))
      (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
              (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                 (* temp1
                    (f2cl-lib:freal
                     (f2cl-lib:fref ap-%data%
                                    ((f2cl-lib:int-sub
                                      (f2cl-lib:int-add kk j) 1))
                                    ((1 *)) ap-%offset%)))
                 (* alpha temp2)))
      (setf kk (f2cl-lib:int-add kk j))
     label60)))
 (t (setf jx kx) (setf jy ky)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf temp1
              (* alpha
                 (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
      (setf temp2 zero)
```

```
        (setf ix kx)
        (setf iy ky)
        (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                      ((> k
                          (f2cl-lib:int-add kk j (f2cl-lib:int-sub 2)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (* temp1
                          (f2cl-lib:fref ap-%data% (k) ((1 *))
                                         ap-%offset%))))
            (setf temp2
                    (+ temp2
                       (*
                        (f2cl-lib:conjg
                         (f2cl-lib:fref ap-%data% (k) ((1 *))
                                        ap-%offset%))
                        (f2cl-lib:fref x-%data% (ix) ((1 *))
                                       x-%offset%))))
            (setf ix (f2cl-lib:int-add ix incx))
            (setf iy (f2cl-lib:int-add iy incy))
           label70))
        (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                   (* temp1
                      (f2cl-lib:freal
                       (f2cl-lib:fref ap-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add kk j) 1))
                                      ((1 *)) ap-%offset%)))
                   (* alpha temp2)))
        (setf jx (f2cl-lib:int-add jx incx))
        (setf jy (f2cl-lib:int-add jy incy))
        (setf kk (f2cl-lib:int-add kk j))
       label80)))))
(t
 (cond
  ((and (= incx 1) (= incy 1))
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp1
               (* alpha
                  (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
       (setf temp2 zero)
       (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
               (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                  (* temp1
                     (f2cl-lib:freal
```

```
                    (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                   ap-%offset%)))))
        (setf k (f2cl-lib:int-add kk 1))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                       (* temp1
                          (f2cl-lib:fref ap-%data% (k) ((1 *))
                                         ap-%offset%))))
            (setf temp2
                    (+ temp2
                       (*
                        (f2cl-lib:conjg
                         (f2cl-lib:fref ap-%data% (k) ((1 *))
                                        ap-%offset%))
                        (f2cl-lib:fref x-%data% (i) ((1 *))
                                       x-%offset%))))
            (setf k (f2cl-lib:int-add k 1))
           label90))
        (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                   (* alpha temp2)))
        (setf kk
                (f2cl-lib:int-add kk
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-sub n j) 1)))
     label100)))
  (t (setf jx kx) (setf jy ky)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp1
               (* alpha
                  (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
       (setf temp2 zero)
       (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
               (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                  (* temp1
                     (f2cl-lib:freal
                      (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                     ap-%offset%)))))
       (setf ix jx)
       (setf iy jy)
       (f2cl-lib:fdo (k (f2cl-lib:int-add kk 1) (f2cl-lib:int-add k 1))
                     ((> k
                         (f2cl-lib:int-add kk n (f2cl-lib:int-sub j)))
                      nil)
         (tagbody
```

```
                (setf ix (f2cl-lib:int-add ix incx))
                (setf iy (f2cl-lib:int-add iy incy))
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                           (* temp1
                              (f2cl-lib:fref ap-%data% (k) ((1 *))
                                             ap-%offset%))))
                (setf temp2
                        (+ temp2
                           (*
                            (f2cl-lib:conjg
                             (f2cl-lib:fref ap-%data% (k) ((1 *))
                                            ap-%offset%))
                            (f2cl-lib:fref x-%data% (ix) ((1 *))
                                           x-%offset%))))
              label110))
            (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                       (* alpha temp2)))
            (setf jx (f2cl-lib:int-add jx incx))
            (setf jy (f2cl-lib:int-add jy incy))
            (setf kk
                    (f2cl-lib:int-add kk
                                      (f2cl-lib:int-add
                                       (f2cl-lib:int-sub n j) 1)))
          label120))))))
  (go end_label)
 end_label
  (return (values uplo nil nil nil nil nil nil nil nil))))))
```

———-

12.10 chpr - hermitian $\alpha x x^H + A$

12.10.1 Axiom unit tests

— chpr.input —

```
)set break resume
)sys rm -f chpr.output
)spool chpr.output
)set message test on
)set message auto off
)clear all

)spool
```

```
)lisp (bye)
```

———-

12.10.2 Axiom help page

— **chpr.help** —

```
=====================================================================
chpr examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine chpr (UPLO, N, ALPHA, X, INCX, AP)
           CHPR

Function/Subroutine Documentation
   subroutine chpr (character UPLO, integer N, real ALPHA, complex,
       dimension(*) X, integer INCX, complex, dimension(*) AP)
       CHPR

Purpose:
   CHPR    performs the hermitian rank 1 operation

      A := alpha*x*x**H + A,

   where alpha is a real scalar, x is an n element vector and A is an
   n by n hermitian matrix, supplied in packed form.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the upper or lower
   triangular part of the matrix A is supplied in the packed
   array AP as follows:

   UPLO = 'U' or 'u'   The upper triangular part of A is supplied in AP.

   UPLO = 'L' or 'l'   The lower triangular part of A is supplied in AP.

 N
   N is INTEGER
```

```
  On entry, N specifies the order of the matrix A.
  N must be at least zero.

ALPHA
  ALPHA is REAL
  On entry, ALPHA specifies the scalar alpha.

X
  X is COMPLEX array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the n
  element vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.

  AP
  AP is COMPLEX array of DIMENSION at least ( ( n*( n + 1 ) )/2 ).

  Before entry with  UPLO = 'U' or 'u', the array AP must
  contain the upper triangular part of the hermitian matrix
  packed sequentially, column by column, so that AP( 1 )
  contains a( 1, 1 ), AP( 2 ) and AP( 3 ) contain a( 1, 2 )
  and a( 2, 2 ) respectively, and so on. On exit, the array
  AP is overwritten by the upper triangular part of the
  updated matrix.

  Before entry with UPLO = 'L' or 'l', the array AP must
  contain the lower triangular part of the hermitian matrix
  packed sequentially, column by column, so that AP( 1 )
  contains a( 1, 1 ), AP( 2 ) and AP( 3 ) contain a( 2, 1 )
  and a( 3, 1 ) respectively, and so on. On exit, the array
  AP is overwritten by the lower triangular part of the
  updated matrix.

  Note that the imaginary parts of the diagonal elements need
  not be set, they are assumed to be zero, and on exit they
  are set to zero.
```

———-

12.10.3 fortran code

— chpr.f —

```
*  ====================================================================
```

```
      SUBROUTINE CHPR(UPLO,N,ALPHA,X,INCX,AP)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL ALPHA
      INTEGER INCX,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX AP(*),X(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP
      INTEGER I,INFO,IX,J,JX,K,KK,KX
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG,REAL
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CHPR  ',INFO)
          RETURN
```

```
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (ALPHA.EQ.REAL(ZERO))) RETURN
*
*     Set the start point in X if the increment is not unity.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of the array AP
*     are accessed sequentially with one pass through AP.
*
      KK = 1
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  A  when upper triangle is stored in AP.
*
          IF (INCX.EQ.1) THEN
              DO 20 J = 1,N
                  IF (X(J).NE.ZERO) THEN
                      TEMP = ALPHA*CONJG(X(J))
                      K = KK
                      DO 10 I = 1,J - 1
                          AP(K) = AP(K) + X(I)*TEMP
                          K = K + 1
   10                 CONTINUE
                      AP(KK+J-1) = REAL(AP(KK+J-1)) + REAL(X(J)*TEMP)
                  ELSE
                      AP(KK+J-1) = REAL(AP(KK+J-1))
                  END IF
                  KK = KK + J
   20         CONTINUE
          ELSE
              JX = KX
              DO 40 J = 1,N
                  IF (X(JX).NE.ZERO) THEN
                      TEMP = ALPHA*CONJG(X(JX))
                      IX = KX
                      DO 30 K = KK,KK + J - 2
                          AP(K) = AP(K) + X(IX)*TEMP
                          IX = IX + INCX
   30                 CONTINUE
                      AP(KK+J-1) = REAL(AP(KK+J-1)) + REAL(X(JX)*TEMP)
                  ELSE
                      AP(KK+J-1) = REAL(AP(KK+J-1))
```

```
                END IF
                JX = JX + INCX
                KK = KK + J
   40         CONTINUE
         END IF
      ELSE
*
*        Form  A  when lower triangle is stored in AP.
*
         IF (INCX.EQ.1) THEN
             DO 60 J = 1,N
                 IF (X(J).NE.ZERO) THEN
                     TEMP = ALPHA*CONJG(X(J))
                     AP(KK) = REAL(AP(KK)) + REAL(TEMP*X(J))
                     K = KK + 1
                     DO 50 I = J + 1,N
                         AP(K) = AP(K) + X(I)*TEMP
                         K = K + 1
   50                CONTINUE
                 ELSE
                     AP(KK) = REAL(AP(KK))
                 END IF
                 KK = KK + N - J + 1
   60         CONTINUE
         ELSE
             JX = KX
             DO 80 J = 1,N
                 IF (X(JX).NE.ZERO) THEN
                     TEMP = ALPHA*CONJG(X(JX))
                     AP(KK) = REAL(AP(KK)) + REAL(TEMP*X(JX))
                     IX = JX
                     DO 70 K = KK + 1,KK + N - J
                         IX = IX + INCX
                         AP(K) = AP(K) + X(IX)*TEMP
   70                CONTINUE
                 ELSE
                     AP(KK) = REAL(AP(KK))
                 END IF
                 JX = JX + INCX
                 KK = KK + N - J + 1
   80         CONTINUE
         END IF
      END IF
*
      RETURN
*
*     End of CHPR  .
*
      END
```

———-

12.10.4 lisp code

— BLAS 2 chpr —

```
(let* ((zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) zero)
           (ignorable zero))
  (defun chpr (uplo n alpha x incx ap)
    (declare (type (array f2cl-lib:complex8 (*)) ap x)
             (type (single-float) alpha)
             (type (f2cl-lib:integer4) incx n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%)
         (ap f2cl-lib:complex8 ap-%data% ap-%offset%))
      (prog ((i 0) (info 0) (ix 0) (j 0) (jx 0) (k 0) (kk 0) (kx 0)
             (temp #C(0.0 0.0)))
        (declare (type (f2cl-lib:integer4) i info ix j jx k kk kx)
                 (type (f2cl-lib:complex8) temp))
        (setf info 0)
        (cond
         ((and
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
          (setf info 1))
         ((< n 0) (setf info 2)) ((= incx 0) (setf info 5)))
        (cond
         ((/= info 0)
          (multiple-value-bind (var-0 var-1)
              (xerbla "CHPR  " info)
            (declare (ignore var-0))
            (when var-1 (setf info var-1)))
          (go end_label)))
        (if (or (= n 0) (= alpha (f2cl-lib:freal zero)))
            (go end_label))
```

```
(cond
 ((<= incx 0)
  (setf kx
          (f2cl-lib:int-sub 1
                            (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                              incx))))
 ((/= incx 1) (setf kx 1)))
(setf kk 1)
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame uplo "U")
    (declare (ignore var-1))
    (when var-0 (setf uplo var-0))
    ret-val)
  (cond
   ((= incx 1)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
         ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
          (setf temp
                  (* alpha
                     (f2cl-lib:conjg
                      (f2cl-lib:fref x-%data% (j) ((1 *))
                                     x-%offset%))))
          (setf k kk)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref ap-%data% (k) ((1 *)) ap-%offset%)
                      (+
                       (f2cl-lib:fref ap-%data% (k) ((1 *))
                                      ap-%offset%)
                       (*
                        (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                        temp)))
              (setf k (f2cl-lib:int-add k 1))
             label10))
          (setf (f2cl-lib:fref ap-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add kk j) 1))
                               ((1 *)) ap-%offset%)
                  (coerce
                   (+
                    (f2cl-lib:freal
                     (f2cl-lib:fref ap-%data%
                                    ((f2cl-lib:int-sub
```

```
                                        (f2cl-lib:int-add kk j) 1))
                                      ((1 *)) ap-%offset%))
                      (f2cl-lib:freal
                       (* (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                          temp)))
                     'f2cl-lib:complex8)))
          (t
           (setf (f2cl-lib:fref ap-%data%
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-add kk j) 1))
                                ((1 *)) ap-%offset%)
                 (coerce
                  (f2cl-lib:freal
                   (f2cl-lib:fref ap-%data%
                                  ((f2cl-lib:int-sub
                                    (f2cl-lib:int-add kk j) 1))
                                  ((1 *)) ap-%offset%))
                  'f2cl-lib:complex8))))
         (setf kk (f2cl-lib:int-add kk j))
        label20)))
    (t (setf jx kx)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (cond
          ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
           (setf temp
                   (* alpha
                      (f2cl-lib:conjg
                       (f2cl-lib:fref x-%data% (jx) ((1 *))
                                      x-%offset%))))
           (setf ix kx)
           (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                         ((> k
                             (f2cl-lib:int-add kk j
                                               (f2cl-lib:int-sub 2)))
                          nil)
             (tagbody
               (setf (f2cl-lib:fref ap-%data% (k) ((1 *)) ap-%offset%)
                       (+
                        (f2cl-lib:fref ap-%data% (k) ((1 *))
                                       ap-%offset%)
                        (*
                         (f2cl-lib:fref x-%data% (ix) ((1 *))
                                        x-%offset%)
                         temp)))
               (setf ix (f2cl-lib:int-add ix incx))
              label30))
           (setf (f2cl-lib:fref ap-%data%
                                ((f2cl-lib:int-sub
```

```
                                  (f2cl-lib:int-add kk j) 1))
                                ((1 *)) ap-%offset%)
                      (coerce
                       (+
                        (f2cl-lib:freal
                         (f2cl-lib:fref ap-%data%
                                        ((f2cl-lib:int-sub
                                          (f2cl-lib:int-add kk j) 1))
                                        ((1 *)) ap-%offset%))
                        (f2cl-lib:freal
                         (*
                          (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                          temp)))
                       'f2cl-lib:complex8)))
            (t
             (setf (f2cl-lib:fref ap-%data%
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-add kk j) 1))
                                ((1 *)) ap-%offset%)
                      (coerce
                       (f2cl-lib:freal
                        (f2cl-lib:fref ap-%data%
                                       ((f2cl-lib:int-sub
                                         (f2cl-lib:int-add kk j) 1))
                                       ((1 *)) ap-%offset%))
                       'f2cl-lib:complex8))))
           (setf jx (f2cl-lib:int-add jx incx))
           (setf kk (f2cl-lib:int-add kk j))
          label40)))))
    (t
     (cond
      ((= incx 1)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (cond
            ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
             (setf temp
                     (* alpha
                        (f2cl-lib:conjg
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%))))
             (setf (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%)
                     (coerce
                      (+
                       (f2cl-lib:freal
                        (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                       ap-%offset%))
                       (f2cl-lib:freal
                        (* temp
```

```
                        (f2cl-lib:fref x-%data% (j) ((1 *))
                                       x-%offset%))))
                 'f2cl-lib:complex8))
          (setf k (f2cl-lib:int-add kk 1))
          (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                         (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref ap-%data% (k) ((1 *)) ap-%offset%)
                      (+
                       (f2cl-lib:fref ap-%data% (k) ((1 *))
                                      ap-%offset%)
                       (*
                        (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                        temp)))
              (setf k (f2cl-lib:int-add k 1))
             label50)))
         (t
          (setf (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%)
                  (coerce
                   (f2cl-lib:freal
                    (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%))
                   'f2cl-lib:complex8))))
        (setf kk
                (f2cl-lib:int-add
                 (f2cl-lib:int-sub (f2cl-lib:int-add kk n) j) 1))
       label60)))
    (t (setf jx kx)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (cond
          ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
           (setf temp
                   (* alpha
                      (f2cl-lib:conjg
                       (f2cl-lib:fref x-%data% (jx) ((1 *))
                                      x-%offset%))))
           (setf (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%)
                   (coerce
                    (+
                     (f2cl-lib:freal
                      (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                     ap-%offset%))
                     (f2cl-lib:freal
                      (* temp
                         (f2cl-lib:fref x-%data% (jx) ((1 *))
                                        x-%offset%))))
                   'f2cl-lib:complex8))
           (setf ix jx)
```

```
                (f2cl-lib:fdo (k (f2cl-lib:int-add kk 1)
                               (f2cl-lib:int-add k 1))
                              ((> k
                                  (f2cl-lib:int-add kk n
                                                    (f2cl-lib:int-sub j)))
                               nil)
                  (tagbody
                    (setf ix (f2cl-lib:int-add ix incx))
                    (setf (f2cl-lib:fref ap-%data% (k) ((1 *)) ap-%offset%)
                            (+
                             (f2cl-lib:fref ap-%data% (k) ((1 *))
                                            ap-%offset%)
                             (*
                              (f2cl-lib:fref x-%data% (ix) ((1 *))
                                             x-%offset%)
                              temp)))
                     label70)))
               (t
                (setf (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%)
                        (coerce
                         (f2cl-lib:freal
                          (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%))
                         'f2cl-lib:complex8))))
             (setf jx (f2cl-lib:int-add jx incx))
             (setf kk
                     (f2cl-lib:int-add
                      (f2cl-lib:int-sub (f2cl-lib:int-add kk n) j) 1))
            label80))))))
      (go end_label)
     end_label
      (return (values uplo nil nil nil nil nil))))))
```

———-

12.11 chpr2 - hermitian rank 2 $\alpha xy^H + conjg(\alpha)yx^H + A$

12.11.1 Axiom unit tests

— **chpr2.input** —

```
)set break resume
)sys rm -f chpr2.output
)spool chpr2.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

12.11.2 Axiom help page

— **chpr2.help** —

```
==================================================================
chpr2 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine chpr2 (UPLO, N, ALPHA, X, INCX, Y, INCY, AP)
           CHPR2

Function/Subroutine Documentation
   subroutine chpr2 (character UPLO, integer N, complex ALPHA, complex,
       dimension(*) X, integer INCX, complex, dimension(*) Y, integer INCY,
       complex, dimension(*) AP)
       CHPR2

Purpose:
   CHPR2  performs the hermitian rank 2 operation

   A := alpha*x*y**H + conjg( alpha )*y*x**H + A,

   where alpha is a scalar, x and y are n element vectors and A is an
   n by n hermitian matrix, supplied in packed form.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the upper or lower
   triangular part of the matrix A is supplied in the packed
   array AP as follows:

   UPLO = 'U' or 'u'   The upper triangular part of A is supplied in AP.

   UPLO = 'L' or 'l'   The lower triangular part of A is supplied in AP.
```

```
N
  N is INTEGER
  On entry, N specifies the order of the matrix A.
  N must be at least zero.

ALPHA
  ALPHA is COMPLEX
  On entry, ALPHA specifies the scalar alpha.

X
  X is COMPLEX array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the n
  element vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.

Y
  Y is COMPLEX array of dimension at least ( 1 + ( n - 1 )*abs( INCY ) ).
  Before entry, the incremented array Y must contain the n
  element vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of Y.
  INCY must not be zero.

AP
  AP is COMPLEX array of DIMENSION at least ( ( n*( n + 1 ) )/2 ).

  Before entry with  UPLO = 'U' or 'u', the array AP must
  contain the upper triangular part of the hermitian matrix
  packed sequentially, column by column, so that AP( 1 )
  contains a( 1, 1 ), AP( 2 ) and AP( 3 ) contain a( 1, 2 )
  and a( 2, 2 ) respectively, and so on. On exit, the array
  AP is overwritten by the upper triangular part of the updated matrix.

  Before entry with UPLO = 'L' or 'l', the array AP must
  contain the lower triangular part of the hermitian matrix
  packed sequentially, column by column, so that AP( 1 )
  contains a( 1, 1 ), AP( 2 ) and AP( 3 ) contain a( 2, 1 )
  and a( 3, 1 ) respectively, and so on. On exit, the array
  AP is overwritten by the lower triangular part of the updated matrix.

  Note that the imaginary parts of the diagonal elements need
  not be set, they are assumed to be zero, and on exit they
  are set to zero.
```

———-

12.11.3 fortran code

— **chpr2.f** —

```
*  ==================================================================
      SUBROUTINE CHPR2(UPLO,N,ALPHA,X,INCX,Y,INCY,AP)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX ALPHA
      INTEGER INCX,INCY,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX AP(*),X(*),Y(*)
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,K,KK,KX,KY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG,REAL
*     ..
*
*     Test the input parameters.
*
```

```
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      ELSE IF (INCY.EQ.0) THEN
          INFO = 7
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CHPR2 ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (ALPHA.EQ.ZERO)) RETURN
*
*     Set up the start points in X and Y if the increments are not both
*     unity.
*
      IF ((INCX.NE.1) .OR. (INCY.NE.1)) THEN
          IF (INCX.GT.0) THEN
              KX = 1
          ELSE
              KX = 1 - (N-1)*INCX
          END IF
          IF (INCY.GT.0) THEN
              KY = 1
          ELSE
              KY = 1 - (N-1)*INCY
          END IF
          JX = KX
          JY = KY
      END IF
*
*     Start the operations. In this version the elements of the array AP
*     are accessed sequentially with one pass through AP.
*
      KK = 1
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  A  when upper triangle is stored in AP.
*
          IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
              DO 20 J = 1,N
                  IF ((X(J).NE.ZERO) .OR. (Y(J).NE.ZERO)) THEN
                      TEMP1 = ALPHA*CONJG(Y(J))
                      TEMP2 = CONJG(ALPHA*X(J))
```

```
                    K = KK
                    DO 10 I = 1,J - 1
                        AP(K) = AP(K) + X(I)*TEMP1 + Y(I)*TEMP2
                        K = K + 1
   10               CONTINUE
                    AP(KK+J-1) = REAL(AP(KK+J-1)) +
     +                          REAL(X(J)*TEMP1+Y(J)*TEMP2)
                ELSE
                    AP(KK+J-1) = REAL(AP(KK+J-1))
                END IF
                KK = KK + J
   20         CONTINUE
           ELSE
               DO 40 J = 1,N
                IF ((X(JX).NE.ZERO) .OR. (Y(JY).NE.ZERO)) THEN
                    TEMP1 = ALPHA*CONJG(Y(JY))
                    TEMP2 = CONJG(ALPHA*X(JX))
                    IX = KX
                    IY = KY
                    DO 30 K = KK,KK + J - 2
                        AP(K) = AP(K) + X(IX)*TEMP1 + Y(IY)*TEMP2
                        IX = IX + INCX
                        IY = IY + INCY
   30               CONTINUE
                    AP(KK+J-1) = REAL(AP(KK+J-1)) +
     +                          REAL(X(JX)*TEMP1+Y(JY)*TEMP2)
                ELSE
                    AP(KK+J-1) = REAL(AP(KK+J-1))
                END IF
                JX = JX + INCX
                JY = JY + INCY
                KK = KK + J
   40         CONTINUE
           END IF
       ELSE
*
*        Form  A  when lower triangle is stored in AP.
*
           IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
               DO 60 J = 1,N
                IF ((X(J).NE.ZERO) .OR. (Y(J).NE.ZERO)) THEN
                    TEMP1 = ALPHA*CONJG(Y(J))
                    TEMP2 = CONJG(ALPHA*X(J))
                    AP(KK) = REAL(AP(KK)) +
     +                       REAL(X(J)*TEMP1+Y(J)*TEMP2)
                    K = KK + 1
                    DO 50 I = J + 1,N
                        AP(K) = AP(K) + X(I)*TEMP1 + Y(I)*TEMP2
                        K = K + 1
   50               CONTINUE
```

```
                ELSE
                    AP(KK) = REAL(AP(KK))
                END IF
                KK = KK + N - J + 1
   60         CONTINUE
          ELSE
              DO 80 J = 1,N
                  IF ((X(JX).NE.ZERO) .OR. (Y(JY).NE.ZERO)) THEN
                      TEMP1 = ALPHA*CONJG(Y(JY))
                      TEMP2 = CONJG(ALPHA*X(JX))
                      AP(KK) = REAL(AP(KK)) +
     +                         REAL(X(JX)*TEMP1+Y(JY)*TEMP2)
                      IX = JX
                      IY = JY
                      DO 70 K = KK + 1,KK + N - J
                          IX = IX + INCX
                          IY = IY + INCY
                          AP(K) = AP(K) + X(IX)*TEMP1 + Y(IY)*TEMP2
   70                 CONTINUE
                  ELSE
                      AP(KK) = REAL(AP(KK))
                  END IF
                  JX = JX + INCX
                  JY = JY + INCY
                  KK = KK + N - J + 1
   80         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of CHPR2 .
*
      END
```

———-

12.11.4 lisp code

— BLAS 2 chpr2 —

```
(let* ((zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) zero)
           (ignorable zero))
  (defun chpr2 (uplo n alpha x incx y incy ap)
    (declare (type (array f2cl-lib:complex8 (*)) ap y x)
             (type (f2cl-lib:complex8) alpha)
```

```
          (type (f2cl-lib:integer4) incy incx n)
          (type (string *) uplo))
(f2cl-lib:with-multi-array-data
    ((uplo character uplo-%data% uplo-%offset%)
     (x f2cl-lib:complex8 x-%data% x-%offset%)
     (y f2cl-lib:complex8 y-%data% y-%offset%)
     (ap f2cl-lib:complex8 ap-%data% ap-%offset%))
  (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (k 0) (kk 0)
         (kx 0) (ky 0) (temp1 #C(0.0 0.0)) (temp2 #C(0.0 0.0)))
    (declare (type (f2cl-lib:integer4) i info ix iy j jx jy k kk kx ky)
             (type (f2cl-lib:complex8) temp1 temp2))
    (setf info 0)
    (cond
     ((and
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
      (setf info 1))
     ((< n 0) (setf info 2)) ((= incx 0) (setf info 5))
     ((= incy 0) (setf info 7)))
    (cond
     ((/= info 0)
      (multiple-value-bind (var-0 var-1)
          (xerbla "CHPR2 " info)
        (declare (ignore var-0))
        (when var-1 (setf info var-1)))
      (go end_label)))
    (if (or (= n 0) (= alpha zero))
        (go end_label))
    (cond
     ((or (/= incx 1) (/= incy 1))
      (cond ((> incx 0) (setf kx 1))
            (t
             (setf kx
                     (f2cl-lib:int-sub 1
                                       (f2cl-lib:int-mul
                                        (f2cl-lib:int-sub n 1) incx)))))
      (cond ((> incy 0) (setf ky 1))
            (t
             (setf ky
                     (f2cl-lib:int-sub 1
```

```
                                    (f2cl-lib:int-mul
                                     (f2cl-lib:int-sub n 1) incy)))))
  (setf jx kx) (setf jy ky)))
(setf kk 1)
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame uplo "U")
    (declare (ignore var-1))
    (when var-0 (setf uplo var-0))
    ret-val)
  (cond
   ((and (= incx 1) (= incy 1))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
         ((or (/= (f2cl-lib:fref x (j) ((1 *))) zero)
              (/= (f2cl-lib:fref y (j) ((1 *))) zero))
          (setf temp1
                  (* alpha
                     (f2cl-lib:conjg
                      (f2cl-lib:fref y-%data% (j) ((1 *))
                                     y-%offset%))))
          (setf temp2
                  (f2cl-lib:conjg
                   (* alpha
                      (f2cl-lib:fref x-%data% (j) ((1 *))
                                     x-%offset%))))
          (setf k kk)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref ap-%data% (k) ((1 *)) ap-%offset%)
                      (+
                       (f2cl-lib:fref ap-%data% (k) ((1 *))
                                      ap-%offset%)
                       (*
                        (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                        temp1)
                       (*
                        (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                        temp2)))
              (setf k (f2cl-lib:int-add k 1))
             label10))
          (setf (f2cl-lib:fref ap-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add kk j) 1))
                               ((1 *)) ap-%offset%)
```

```
                 (coerce
                  (+
                   (f2cl-lib:freal
                    (f2cl-lib:fref ap-%data%
                                   ((f2cl-lib:int-sub
                                     (f2cl-lib:int-add kk j) 1))
                                   ((1 *)) ap-%offset%))
                   (f2cl-lib:freal
                    (+
                     (*
                      (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                      temp1)
                     (*
                      (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                      temp2))))
                  'f2cl-lib:complex8)))
        (t
         (setf (f2cl-lib:fref ap-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add kk j) 1))
                              ((1 *)) ap-%offset%)
                 (coerce
                  (f2cl-lib:freal
                   (f2cl-lib:fref ap-%data%
                                  ((f2cl-lib:int-sub
                                    (f2cl-lib:int-add kk j) 1))
                                  ((1 *)) ap-%offset%))
                  'f2cl-lib:complex8))))
       (setf kk (f2cl-lib:int-add kk j))
      label20)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((or (/= (f2cl-lib:fref x (jx) ((1 *))) zero)
             (/= (f2cl-lib:fref y (jy) ((1 *))) zero))
         (setf temp1
                 (* alpha
                    (f2cl-lib:conjg
                     (f2cl-lib:fref y-%data% (jy) ((1 *))
                                    y-%offset%))))
         (setf temp2
                 (f2cl-lib:conjg
                  (* alpha
                     (f2cl-lib:fref x-%data% (jx) ((1 *))
                                    x-%offset%))))
         (setf ix kx) (setf iy ky)
         (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                       ((> k
```

```
                    (f2cl-lib:int-add kk j
                                      (f2cl-lib:int-sub 2)))
                nil)
   (tagbody
     (setf (f2cl-lib:fref ap-%data% (k) ((1 *)) ap-%offset%)
             (+
              (f2cl-lib:fref ap-%data% (k) ((1 *))
                             ap-%offset%)
              (*
               (f2cl-lib:fref x-%data% (ix) ((1 *))
                              x-%offset%)
               temp1)
              (*
               (f2cl-lib:fref y-%data% (iy) ((1 *))
                              y-%offset%)
               temp2)))
     (setf ix (f2cl-lib:int-add ix incx))
     (setf iy (f2cl-lib:int-add iy incy))
    label30))
 (setf (f2cl-lib:fref ap-%data%
                      ((f2cl-lib:int-sub
                        (f2cl-lib:int-add kk j) 1))
                      ((1 *)) ap-%offset%)
         (coerce
          (+
           (f2cl-lib:freal
            (f2cl-lib:fref ap-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add kk j) 1))
                           ((1 *)) ap-%offset%))
           (f2cl-lib:freal
            (+
             (*
              (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
              temp1)
             (*
              (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
              temp2))))
          'f2cl-lib:complex8)))
(t
 (setf (f2cl-lib:fref ap-%data%
                      ((f2cl-lib:int-sub
                        (f2cl-lib:int-add kk j) 1))
                      ((1 *)) ap-%offset%)
         (coerce
          (f2cl-lib:freal
           (f2cl-lib:fref ap-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add kk j) 1))
                          ((1 *)) ap-%offset%))
```

```
                    'f2cl-lib:complex8))))
        (setf jx (f2cl-lib:int-add jx incx))
        (setf jy (f2cl-lib:int-add jy incy))
        (setf kk (f2cl-lib:int-add kk j))
       label40)))))
(t
 (cond
  ((and (= incx 1) (= incy 1))
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((or (/= (f2cl-lib:fref x (j) ((1 *))) zero)
             (/= (f2cl-lib:fref y (j) ((1 *))) zero))
         (setf temp1
                 (* alpha
                    (f2cl-lib:conjg
                     (f2cl-lib:fref y-%data% (j) ((1 *))
                                    y-%offset%))))
         (setf temp2
                 (f2cl-lib:conjg
                  (* alpha
                     (f2cl-lib:fref x-%data% (j) ((1 *))
                                    x-%offset%))))
         (setf (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%)
                 (coerce
                  (+
                   (f2cl-lib:freal
                    (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                   ap-%offset%))
                   (f2cl-lib:freal
                    (+
                     (*
                      (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                      temp1)
                     (*
                      (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                      temp2))))
                  'f2cl-lib:complex8))
         (setf k (f2cl-lib:int-add kk 1))
         (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                        (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref ap-%data% (k) ((1 *)) ap-%offset%)
                     (+
                      (f2cl-lib:fref ap-%data% (k) ((1 *))
                                     ap-%offset%)
                      (*
                       (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
```

```
                        temp1)
                       (*
                        (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                        temp2)))
            (setf k (f2cl-lib:int-add k 1))
           label50)))
       (t
        (setf (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%)
                (coerce
                 (f2cl-lib:freal
                  (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%))
                 'f2cl-lib:complex8))))
      (setf kk
              (f2cl-lib:int-add
               (f2cl-lib:int-sub (f2cl-lib:int-add kk n) j) 1))
    label60)))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
      ((or (/= (f2cl-lib:fref x (jx) ((1 *))) zero)
           (/= (f2cl-lib:fref y (jy) ((1 *))) zero))
       (setf temp1
               (* alpha
                  (f2cl-lib:conjg
                   (f2cl-lib:fref y-%data% (jy) ((1 *))
                                  y-%offset%))))
       (setf temp2
               (f2cl-lib:conjg
                (* alpha
                   (f2cl-lib:fref x-%data% (jx) ((1 *))
                                  x-%offset%))))
       (setf (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%)
               (coerce
                (+
                 (f2cl-lib:freal
                  (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                 ap-%offset%))
                 (f2cl-lib:freal
                  (+
                   (*
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                    temp1)
                   (*
                    (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                    temp2))))
                'f2cl-lib:complex8))
       (setf ix jx) (setf iy jy)
       (f2cl-lib:fdo (k (f2cl-lib:int-add kk 1)
```

```
                              (f2cl-lib:int-add k 1))
                             ((> k
                                 (f2cl-lib:int-add kk n
                                                   (f2cl-lib:int-sub j)))
                              nil)
                   (tagbody
                     (setf ix (f2cl-lib:int-add ix incx))
                     (setf iy (f2cl-lib:int-add iy incy))
                     (setf (f2cl-lib:fref ap-%data% (k) ((1 *)) ap-%offset%)
                             (+
                              (f2cl-lib:fref ap-%data% (k) ((1 *))
                                             ap-%offset%)
                              (*
                               (f2cl-lib:fref x-%data% (ix) ((1 *))
                                              x-%offset%)
                               temp1)
                              (*
                               (f2cl-lib:fref y-%data% (iy) ((1 *))
                                              y-%offset%)
                               temp2)))
                       label70)))
                 (t
                  (setf (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%)
                          (coerce
                           (f2cl-lib:freal
                            (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%))
                           'f2cl-lib:complex8))))
                 (setf jx (f2cl-lib:int-add jx incx))
                 (setf jy (f2cl-lib:int-add jy incy))
                 (setf kk
                         (f2cl-lib:int-add
                          (f2cl-lib:int-sub (f2cl-lib:int-add kk n) j) 1))
               label80))))))
        (go end_label)
       end_label
        (return (values uplo nil nil nil nil nil nil nil))))))
```

———-

12.12 ctbmv - banded matrix-vector Ax or A^Tx or A^Hx

12.12.1 Axiom unit tests

— ctbmv.input —

```
)set break resume
```

```
)sys rm -f ctbmv.output
)spool ctbmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

12.12.2 Axiom help page

— ctbmv.help —

```
======================================================================
ctbmv examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ctbmv (UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)
           CTBMV

Function/Subroutine Documentation
   subroutine ctbmv (character UPLO, character TRANS, character DIAG, integer
       N, integer K, complex, dimension(lda,*) A, integer LDA, complex,
       dimension(*) X, integer INCX)
       CTBMV

Purpose:
   CTBMV  performs one of the matrix-vector operations

   x := A*x,   or   x := A**T*x,   or   x := A**H*x,

   where x is an n element vector and  A is an n by n unit, or non-unit,
   upper or lower triangular band matrix, with ( k + 1 ) diagonals.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the matrix is an upper or
   lower triangular matrix as follows:
```

```
  UPLO = 'U' or 'u'   A is an upper triangular matrix.

  UPLO = 'L' or 'l'   A is a lower triangular matrix.

TRANS
  TRANS is CHARACTER*1
  On entry, TRANS specifies the operation to be performed as follows:

  TRANS = 'N' or 'n'   x := A*x.

  TRANS = 'T' or 't'   x := A**T*x.

  TRANS = 'C' or 'c'   x := A**H*x.

DIAG
  DIAG is CHARACTER*1
  On entry, DIAG specifies whether or not A is unit triangular as follows:

  DIAG = 'U' or 'u'   A is assumed to be unit triangular.

  DIAG = 'N' or 'n'   A is not assumed to be unit triangular.

N
  N is INTEGER
  On entry, N specifies the order of the matrix A.
  N must be at least zero.

K
  K is INTEGER
  On entry with UPLO = 'U' or 'u', K specifies the number of
  super-diagonals of the matrix A.
  On entry with UPLO = 'L' or 'l', K specifies the number of
  sub-diagonals of the matrix A.
  K must satisfy  0 .le. K.

A
  A is COMPLEX array of DIMENSION ( LDA, n ).

  Before entry with UPLO = 'U' or 'u', the leading ( k + 1 )
  by n part of the array A must contain the upper triangular
  band part of the matrix of coefficients, supplied column by
  column, with the leading diagonal of the matrix in row
  ( k + 1 ) of the array, the first super-diagonal starting at
  position 2 in row k, and so on. The top left k by k triangle
  of the array A is not referenced.

  The following program segment will transfer an upper
  triangular band matrix from conventional full matrix storage
  to band storage:
```

```
      DO 20, J = 1, N
         M = K + 1 - J
         DO 10, I = MAX( 1, J - K ), J
            A( M + I, J ) = matrix( I, J )
   10    CONTINUE
   20 CONTINUE
```

Before entry with UPLO = 'L' or 'l', the leading (k + 1) by n part of the array A must contain the lower triangular band part of the matrix of coefficients, supplied column by column, with the leading diagonal of the matrix in row 1 of the array, the first sub-diagonal starting at position 1 in row 2, and so on. The bottom right k by k triangle of the array A is not referenced.

The following program segment will transfer a lower triangular band matrix from conventional full matrix storage to band storage:

```
      DO 20, J = 1, N
         M = 1 - J
         DO 10, I = J, MIN( N, J + K )
            A( M + I, J ) = matrix( I, J )
   10    CONTINUE
   20 CONTINUE
```

Note that when DIAG = 'U' or 'u' the elements of the array A corresponding to the diagonal elements of the matrix are not referenced, but are assumed to be unity.

LDA

LDA is INTEGER
On entry, LDA specifies the first dimension of A as declared in the calling (sub) program. LDA must be at least (k + 1).

X

X is COMPLEX array of dimension at least (1 + (n - 1)*abs(INCX)).
Before entry, the incremented array X must contain the n element vector x. On exit, X is overwritten with the tranformed vector x.

INCX

INCX is INTEGER
On entry, INCX specifies the increment for the elements of X. INCX must not be zero.

The vector and matrix arguments are not referenced when N = 0, or M = 0

———-

12.12.3 fortran code

— ctbmv.f —

```
*  ===================================================================
      SUBROUTINE CTBMV(UPLO,TRANS,DIAG,N,K,A,LDA,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,K,LDA,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),X(*)
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP
      INTEGER I,INFO,IX,J,JX,KPLUS1,KX,L
      LOGICAL NOCONJ,NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG,MAX,MIN
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
```

```
         INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
         INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
         INFO = 3
      ELSE IF (N.LT.0) THEN
         INFO = 4
      ELSE IF (K.LT.0) THEN
         INFO = 5
      ELSE IF (LDA.LT. (K+1)) THEN
         INFO = 7
      ELSE IF (INCX.EQ.0) THEN
         INFO = 9
      END IF
      IF (INFO.NE.0) THEN
         CALL XERBLA('CTBMV ',INFO)
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOCONJ = LSAME(TRANS,'T')
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX   too small for descending loops.
*
      IF (INCX.LE.0) THEN
         KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
         KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x := A*x.
*
         IF (LSAME(UPLO,'U')) THEN
            KPLUS1 = K + 1
            IF (INCX.EQ.1) THEN
               DO 20 J = 1,N
                  IF (X(J).NE.ZERO) THEN
                     TEMP = X(J)
                     L = KPLUS1 - J
```

```
                    DO 10 I = MAX(1,J-K),J - 1
                        X(I) = X(I) + TEMP*A(L+I,J)
10                  CONTINUE
                    IF (NOUNIT) X(J) = X(J)*A(KPLUS1,J)
                END IF
20          CONTINUE
        ELSE
            JX = KX
            DO 40 J = 1,N
                IF (X(JX).NE.ZERO) THEN
                    TEMP = X(JX)
                    IX = KX
                    L = KPLUS1 - J
                    DO 30 I = MAX(1,J-K),J - 1
                        X(IX) = X(IX) + TEMP*A(L+I,J)
                        IX = IX + INCX
30                  CONTINUE
                    IF (NOUNIT) X(JX) = X(JX)*A(KPLUS1,J)
                END IF
                JX = JX + INCX
                IF (J.GT.K) KX = KX + INCX
40          CONTINUE
        END IF
    ELSE
        IF (INCX.EQ.1) THEN
            DO 60 J = N,1,-1
                IF (X(J).NE.ZERO) THEN
                    TEMP = X(J)
                    L = 1 - J
                    DO 50 I = MIN(N,J+K),J + 1,-1
                        X(I) = X(I) + TEMP*A(L+I,J)
50                  CONTINUE
                    IF (NOUNIT) X(J) = X(J)*A(1,J)
                END IF
60          CONTINUE
        ELSE
            KX = KX + (N-1)*INCX
            JX = KX
            DO 80 J = N,1,-1
                IF (X(JX).NE.ZERO) THEN
                    TEMP = X(JX)
                    IX = KX
                    L = 1 - J
                    DO 70 I = MIN(N,J+K),J + 1,-1
                        X(IX) = X(IX) + TEMP*A(L+I,J)
                        IX = IX - INCX
70                  CONTINUE
                    IF (NOUNIT) X(JX) = X(JX)*A(1,J)
                END IF
                JX = JX - INCX
```

```
                    IF ((N-J).GE.K) KX = KX - INCX
   80            CONTINUE
            END IF
         END IF
      ELSE
*
*        Form  x := A**T*x  or  x := A**H*x.
*
         IF (LSAME(UPLO,'U')) THEN
            KPLUS1 = K + 1
            IF (INCX.EQ.1) THEN
               DO 110 J = N,1,-1
                  TEMP = X(J)
                  L = KPLUS1 - J
                  IF (NOCONJ) THEN
                     IF (NOUNIT) TEMP = TEMP*A(KPLUS1,J)
                     DO 90 I = J - 1,MAX(1,J-K),-1
                        TEMP = TEMP + A(L+I,J)*X(I)
   90                CONTINUE
                  ELSE
                     IF (NOUNIT) TEMP = TEMP*CONJG(A(KPLUS1,J))
                     DO 100 I = J - 1,MAX(1,J-K),-1
                        TEMP = TEMP + CONJG(A(L+I,J))*X(I)
  100                CONTINUE
                  END IF
                  X(J) = TEMP
  110            CONTINUE
            ELSE
               KX = KX + (N-1)*INCX
               JX = KX
               DO 140 J = N,1,-1
                  TEMP = X(JX)
                  KX = KX - INCX
                  IX = KX
                  L = KPLUS1 - J
                  IF (NOCONJ) THEN
                     IF (NOUNIT) TEMP = TEMP*A(KPLUS1,J)
                     DO 120 I = J - 1,MAX(1,J-K),-1
                        TEMP = TEMP + A(L+I,J)*X(IX)
                        IX = IX - INCX
  120                CONTINUE
                  ELSE
                     IF (NOUNIT) TEMP = TEMP*CONJG(A(KPLUS1,J))
                     DO 130 I = J - 1,MAX(1,J-K),-1
                        TEMP = TEMP + CONJG(A(L+I,J))*X(IX)
                        IX = IX - INCX
  130                CONTINUE
                  END IF
                  X(JX) = TEMP
                  JX = JX - INCX
```

```
 140            CONTINUE
            END IF
        ELSE
            IF (INCX.EQ.1) THEN
                DO 170 J = 1,N
                    TEMP = X(J)
                    L = 1 - J
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*A(1,J)
                        DO 150 I = J + 1,MIN(N,J+K)
                            TEMP = TEMP + A(L+I,J)*X(I)
 150                    CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*CONJG(A(1,J))
                        DO 160 I = J + 1,MIN(N,J+K)
                            TEMP = TEMP + CONJG(A(L+I,J))*X(I)
 160                    CONTINUE
                    END IF
                    X(J) = TEMP
 170            CONTINUE
            ELSE
                JX = KX
                DO 200 J = 1,N
                    TEMP = X(JX)
                    KX = KX + INCX
                    IX = KX
                    L = 1 - J
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*A(1,J)
                        DO 180 I = J + 1,MIN(N,J+K)
                            TEMP = TEMP + A(L+I,J)*X(IX)
                            IX = IX + INCX
 180                    CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*CONJG(A(1,J))
                        DO 190 I = J + 1,MIN(N,J+K)
                            TEMP = TEMP + CONJG(A(L+I,J))*X(IX)
                            IX = IX + INCX
 190                    CONTINUE
                    END IF
                    X(JX) = TEMP
                    JX = JX + INCX
 200            CONTINUE
            END IF
        END IF
    END IF
*
    RETURN
*
*   End of CTBMV .
```

```
*
      END
```

———-

12.12.4 lisp code

— BLAS 2 ctbmv —

```
(let* ((zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) zero)
           (ignorable zero))
  (defun ctbmv (uplo trans diag n k a lda x incx)
    (declare (type (array f2cl-lib:complex8 (*)) x a)
             (type (f2cl-lib:integer4) incx lda k n)
             (type (string *) diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%))
      (prog ((noconj nil) (nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0)
             (kplus1 0) (kx 0) (l 0) (temp #C(0.0 0.0)))
        (declare (type f2cl-lib:logical noconj nounit)
                 (type (f2cl-lib:integer4) i info ix j jx kplus1 kx l)
                 (type (f2cl-lib:complex8) temp))
        (setf info 0)
        (cond
         ((and
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
          (setf info 1))
         ((and
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "N")
```

```
      (declare (ignore var-1))
      (when var-0 (setf trans var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame trans "T")
      (declare (ignore var-1))
      (when var-0 (setf trans var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame trans "C")
      (declare (ignore var-1))
      (when var-0 (setf trans var-0))
      ret-val)))
  (setf info 2))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame diag "U")
      (declare (ignore var-1))
      (when var-0 (setf diag var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame diag "N")
      (declare (ignore var-1))
      (when var-0 (setf diag var-0))
      ret-val)))
  (setf info 3))
 ((< n 0) (setf info 4)) ((< k 0) (setf info 5))
 ((< lda (f2cl-lib:int-add k 1)) (setf info 7))
 ((= incx 0) (setf info 9)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "CTBMV " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (= n 0)
    (go end_label))
(setf noconj
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(setf nounit
        (multiple-value-bind (ret-val var-0 var-1)
```

```
            (lsame diag "N")
          (declare (ignore var-1))
          (when var-0 (setf diag var-0))
          ret-val))
(cond
 ((<= incx 0)
  (setf kx
          (f2cl-lib:int-sub 1
                            (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                              incx))))
 ((/= incx 1) (setf kx 1)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame trans "N")
    (declare (ignore var-1))
    (when var-0 (setf trans var-0))
    ret-val)
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)
    (setf kplus1 (f2cl-lib:int-add k 1))
    (cond
     ((= incx 1)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
           ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
            (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
            (setf l (f2cl-lib:int-sub kplus1 j))
            (f2cl-lib:fdo (i
                           (max (the f2cl-lib:integer4 1)
                                (the f2cl-lib:integer4
                                     (f2cl-lib:int-add j
                                                       (f2cl-lib:int-sub
                                                        k))))
                           (f2cl-lib:int-add i 1))
                          ((> i
                              (f2cl-lib:int-add j
                                                (f2cl-lib:int-sub 1)))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                      (+
                       (f2cl-lib:fref x-%data% (i) ((1 *))
                                      x-%offset%)
                       (* temp
```

```
                              (f2cl-lib:fref a-%data%
                                             ((f2cl-lib:int-add l i) j)
                                             ((1 lda) (1 *))
                                             a-%offset%))))
                 label10))
              (if nounit
                  (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                          (*
                           (f2cl-lib:fref x-%data% (j) ((1 *))
                                          x-%offset%)
                           (f2cl-lib:fref a-%data% (kplus1 j)
                                          ((1 lda) (1 *))
                                          a-%offset%))))))
          label20)))
      (t (setf jx kx)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (cond
            ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
             (setf temp
                     (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
             (setf ix kx) (setf l (f2cl-lib:int-sub kplus1 j))
             (f2cl-lib:fdo (i
                            (max (the f2cl-lib:integer4 1)
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add j
                                                        (f2cl-lib:int-sub
                                                         k))))
                            (f2cl-lib:int-add i 1))
                           ((> i
                               (f2cl-lib:int-add j
                                                 (f2cl-lib:int-sub 1)))
                            nil)
               (tagbody
                 (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                         (+
                          (f2cl-lib:fref x-%data% (ix) ((1 *))
                                         x-%offset%)
                          (* temp
                             (f2cl-lib:fref a-%data%
                                            ((f2cl-lib:int-add l i) j)
                                            ((1 lda) (1 *))
                                            a-%offset%))))
                 (setf ix (f2cl-lib:int-add ix incx))
                 label30))
             (if nounit
                 (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                         (*
                          (f2cl-lib:fref x-%data% (jx) ((1 *))
```

```
                                        x-%offset%)
                          (f2cl-lib:fref a-%data% (kplus1 j)
                                         ((1 lda) (1 *))
                                         a-%offset%))))))
          (setf jx (f2cl-lib:int-add jx incx))
          (if (> j k)
              (setf kx (f2cl-lib:int-add kx incx)))
        label40)))))
    (t
     (cond
      ((= incx 1)
       (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     ((> j 1) nil)
         (tagbody
           (cond
            ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
             (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
             (setf l (f2cl-lib:int-sub 1 j))
             (f2cl-lib:fdo (i
                            (min (the f2cl-lib:integer4 n)
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add j k)))
                            (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                           ((> i (f2cl-lib:int-add j 1)) nil)
               (tagbody
                 (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                         (+
                          (f2cl-lib:fref x-%data% (i) ((1 *))
                                         x-%offset%)
                          (* temp
                             (f2cl-lib:fref a-%data%
                                            ((f2cl-lib:int-add l i) j)
                                            ((1 lda) (1 *))
                                            a-%offset%))))
                 label50))
             (if nounit
                 (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                         (*
                          (f2cl-lib:fref x-%data% (j) ((1 *))
                                         x-%offset%)
                          (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *))
                                         a-%offset%))))))
        label60)))
      (t
       (setf kx
               (f2cl-lib:int-add kx
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incx)))
       (setf jx kx)
       (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
```

```
                    ((> j 1) nil)
          (tagbody
            (cond
             ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
              (setf temp
                      (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
              (setf ix kx) (setf l (f2cl-lib:int-sub 1 j))
              (f2cl-lib:fdo (i
                             (min (the f2cl-lib:integer4 n)
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-add j k)))
                             (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                            ((> i (f2cl-lib:int-add j 1)) nil)
                (tagbody
                  (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                          (+
                           (f2cl-lib:fref x-%data% (ix) ((1 *))
                                          x-%offset%)
                           (* temp
                              (f2cl-lib:fref a-%data%
                                             ((f2cl-lib:int-add l i) j)
                                             ((1 lda) (1 *))
                                             a-%offset%))))
                  (setf ix (f2cl-lib:int-sub ix incx))
                 label70))
              (if nounit
                  (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                          (*
                           (f2cl-lib:fref x-%data% (jx) ((1 *))
                                          x-%offset%)
                           (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *))
                                          a-%offset%))))))
            (setf jx (f2cl-lib:int-sub jx incx))
            (if (>= (f2cl-lib:int-sub n j) k)
                (setf kx (f2cl-lib:int-sub kx incx)))
           label80)))))))
(t
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame uplo "U")
     (declare (ignore var-1))
     (when var-0 (setf uplo var-0))
     ret-val)
   (setf kplus1 (f2cl-lib:int-add k 1))
   (cond
    ((= incx 1)
     (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   ((> j 1) nil)
       (tagbody
         (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
```

```
(setf l (f2cl-lib:int-sub kplus1 j))
(cond
 (noconj
  (if nounit
      (setf temp
              (* temp
                 (f2cl-lib:fref a-%data% (kplus1 j)
                                ((1 lda) (1 *))
                                a-%offset%))))
  (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                 (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                ((> i
                    (max (the f2cl-lib:integer4 1)
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add j
                                                (f2cl-lib:int-sub
                                                 k)))))
                 nil)
    (tagbody
      (setf temp
              (+ temp
                 (*
                  (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add l i) j)
                                 ((1 lda) (1 *)) a-%offset%)
                  (f2cl-lib:fref x-%data% (i) ((1 *))
                                 x-%offset%))))
     label90)))
 (t
  (if nounit
      (setf temp
              (* temp
                 (f2cl-lib:conjg
                  (f2cl-lib:fref a-%data% (kplus1 j)
                                 ((1 lda) (1 *))
                                 a-%offset%)))))
  (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                 (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                ((> i
                    (max (the f2cl-lib:integer4 1)
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add j
                                                (f2cl-lib:int-sub
                                                 k)))))
                 nil)
    (tagbody
      (setf temp
              (+ temp
                 (*
                  (f2cl-lib:conjg
```

```
                            (f2cl-lib:fref a-%data%
                                           ((f2cl-lib:int-add l i) j)
                                           ((1 lda) (1 *))
                                           a-%offset%))
                          (f2cl-lib:fref x-%data% (i) ((1 *))
                                         x-%offset%))))
          label100))))
     (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
    label110)))
(t
 (setf kx
         (f2cl-lib:int-add kx
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-sub n 1) incx)))
 (setf jx kx)
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
     (setf kx (f2cl-lib:int-sub kx incx))
     (setf ix kx)
     (setf l (f2cl-lib:int-sub kplus1 j))
     (cond
      (noconj
       (if nounit
           (setf temp
                   (* temp
                      (f2cl-lib:fref a-%data% (kplus1 j)
                                     ((1 lda) (1 *))
                                     a-%offset%))))
       (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                      (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     ((> i
                         (max (the f2cl-lib:integer4 1)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add j
                                                     (f2cl-lib:int-sub
                                                      k)))))
                      nil)
           (tagbody
             (setf temp
                     (+ temp
                        (*
                         (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-add l i) j)
                                        ((1 lda) (1 *)) a-%offset%)
                         (f2cl-lib:fref x-%data% (ix) ((1 *))
                                        x-%offset%))))
             (setf ix (f2cl-lib:int-sub ix incx))
            label120)))
```

```
            (t
             (if nounit
                 (setf temp
                         (* temp
                            (f2cl-lib:conjg
                             (f2cl-lib:fref a-%data% (kplus1 j)
                                            ((1 lda) (1 *))
                                            a-%offset%)))))
             (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                            (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                           ((> i
                               (max (the f2cl-lib:integer4 1)
                                    (the f2cl-lib:integer4
                                         (f2cl-lib:int-add j
                                                           (f2cl-lib:int-sub
                                                            k)))))
                            nil)
               (tagbody
                 (setf temp
                         (+ temp
                            (*
                             (f2cl-lib:conjg
                              (f2cl-lib:fref a-%data%
                                             ((f2cl-lib:int-add l i) j)
                                             ((1 lda) (1 *))
                                             a-%offset%))
                             (f2cl-lib:fref x-%data% (ix) ((1 *))
                                            x-%offset%))))
                 (setf ix (f2cl-lib:int-sub ix incx))
                label130))))
           (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
           (setf jx (f2cl-lib:int-sub jx incx))
          label140)))))
      (t
       (cond
        ((= incx 1)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
             (setf l (f2cl-lib:int-sub 1 j))
             (cond
              (noconj
               (if nounit
                   (setf temp
                           (* temp
                              (f2cl-lib:fref a-%data% (1 j)
                                             ((1 lda) (1 *))
                                             a-%offset%))))
               (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
```

```
                          (f2cl-lib:int-add i 1))
                         ((> i
                             (min (the f2cl-lib:integer4 n)
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-add j k))))
                          nil)
                 (tagbody
                   (setf temp
                           (+ temp
                              (*
                               (f2cl-lib:fref a-%data%
                                              ((f2cl-lib:int-add l i) j)
                                              ((1 lda) (1 *)) a-%offset%)
                               (f2cl-lib:fref x-%data% (i) ((1 *))
                                              x-%offset%))))
                  label150)))
               (t
                (if nounit
                    (setf temp
                            (* temp
                               (f2cl-lib:conjg
                                (f2cl-lib:fref a-%data% (1 j)
                                               ((1 lda) (1 *))
                                               a-%offset%)))))
                (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                                (f2cl-lib:int-add i 1))
                               ((> i
                                   (min (the f2cl-lib:integer4 n)
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-add j k))))
                                nil)
                 (tagbody
                   (setf temp
                           (+ temp
                              (*
                               (f2cl-lib:conjg
                                (f2cl-lib:fref a-%data%
                                               ((f2cl-lib:int-add l i) j)
                                               ((1 lda) (1 *))
                                               a-%offset%))
                               (f2cl-lib:fref x-%data% (i) ((1 *))
                                              x-%offset%))))
                  label160))))
             (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
            label170)))
        (t (setf jx kx)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
```

```
(setf kx (f2cl-lib:int-add kx incx))
(setf ix kx)
(setf l (f2cl-lib:int-sub 1 j))
(cond
 (noconj
  (if nounit
      (setf temp
              (* temp
                 (f2cl-lib:fref a-%data% (1 j)
                                ((1 lda) (1 *))
                                a-%offset%))))
  (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                 (f2cl-lib:int-add i 1))
                ((> i
                    (min (the f2cl-lib:integer4 n)
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add j k))))
                 nil)
    (tagbody
      (setf temp
              (+ temp
                 (*
                  (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add l i) j)
                                 ((1 lda) (1 *)) a-%offset%)
                  (f2cl-lib:fref x-%data% (ix) ((1 *))
                                 x-%offset%))))
      (setf ix (f2cl-lib:int-add ix incx))
     label180)))
 (t
  (if nounit
      (setf temp
              (* temp
                 (f2cl-lib:conjg
                  (f2cl-lib:fref a-%data% (1 j)
                                 ((1 lda) (1 *))
                                 a-%offset%)))))
  (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                 (f2cl-lib:int-add i 1))
                ((> i
                    (min (the f2cl-lib:integer4 n)
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add j k))))
                 nil)
    (tagbody
      (setf temp
              (+ temp
                 (*
                  (f2cl-lib:conjg
                   (f2cl-lib:fref a-%data%
```

```
                                                  ((f2cl-lib:int-add l i) j)
                                                  ((1 lda) (1 *))
                                                  a-%offset%))
                                        (f2cl-lib:fref x-%data% (ix) ((1 *))
                                                       x-%offset%))))
                          (setf ix (f2cl-lib:int-add ix incx))
                         label190))))
                    (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
                    (setf jx (f2cl-lib:int-add jx incx))
                   label200))))))))
        (go end_label)
       end_label
        (return (values uplo trans diag nil nil nil nil nil nil))))))
```

———-

12.13 ctbsv - banded solved systems of equations $Ax = b$ or $A^T x = b$ or $A^H x = b$

12.13.1 Axiom unit tests

— **ctbsv.input** —

```
)set break resume
)sys rm -f ctbsv.output
)spool ctbsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

12.13.2 Axiom help page

— **ctbsv.help** —

```
==================================================================
ctpsv examples
==================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ctbsv (UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX)
           CTBSV

Function/Subroutine Documentation
   subroutine ctbsv (character UPLO, character TRANS, character DIAG, integer
       N, integer K, complex, dimension(lda,*) A, integer LDA, complex,
       dimension(*) X, integer INCX)
       CTBSV

Purpose:
   CTBSV  solves one of the systems of equations

   A*x = b,   or   A**T*x = b,   or   A**H*x = b,

   where b and x are n element vectors and A is an n by n unit, or
   non-unit, upper or lower triangular band matrix, with ( k + 1 )
   diagonals.

   No test for singularity or near-singularity is included in this
   routine. Such tests must be performed before calling this routine.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the matrix is an upper or
   lower triangular matrix as follows:

   UPLO = 'U' or 'u'   A is an upper triangular matrix.

   UPLO = 'L' or 'l'   A is a lower triangular matrix.

 TRANS
   TRANS is CHARACTER*1
   On entry, TRANS specifies the equations to be solved as follows:

   TRANS = 'N' or 'n'   A*x = b.

   TRANS = 'T' or 't'   A**T*x = b.

   TRANS = 'C' or 'c'   A**H*x = b.

 DIAG
   DIAG is CHARACTER*1
```

```
  On entry, DIAG specifies whether or not A is unit
  triangular as follows:

  DIAG = 'U' or 'u'   A is assumed to be unit triangular.

  DIAG = 'N' or 'n'   A is not assumed to be unit triangular.

N
  N is INTEGER
  On entry, N specifies the order of the matrix A.
  N must be at least zero.

K
  K is INTEGER
  On entry with UPLO = 'U' or 'u', K specifies the number of
  super-diagonals of the matrix A.
  On entry with UPLO = 'L' or 'l', K specifies the number of
  sub-diagonals of the matrix A.
  K must satisfy  0 .le. K.

A
  A is COMPLEX array of DIMENSION ( LDA, n ).

  Before entry with UPLO = 'U' or 'u', the leading ( k + 1 )
  by n part of the array A must contain the upper triangular
  band part of the matrix of coefficients, supplied column by
  column, with the leading diagonal of the matrix in row
  ( k + 1 ) of the array, the first super-diagonal starting at
  position 2 in row k, and so on. The top left k by k triangle
  of the array A is not referenced.

  The following program segment will transfer an upper
  triangular band matrix from conventional full matrix storage
  to band storage:

       DO 20, J = 1, N
          M = K + 1 - J
          DO 10, I = MAX( 1, J - K ), J
             A( M + I, J ) = matrix( I, J )
    10    CONTINUE
    20 CONTINUE

  Before entry with UPLO = 'L' or 'l', the leading ( k + 1 )
  by n part of the array A must contain the lower triangular
  band part of the matrix of coefficients, supplied column by
  column, with the leading diagonal of the matrix in row 1 of
  the array, the first sub-diagonal starting at position 1 in
  row 2, and so on. The bottom right k by k triangle of the
  array A is not referenced.
```

```
  The following program segment will transfer a lower
  triangular band matrix from conventional full matrix storage
  to band storage:

       DO 20, J = 1, N
          M = 1 - J
          DO 10, I = J, MIN( N, J + K )
             A( M + I, J ) = matrix( I, J )
    10    CONTINUE
    20 CONTINUE

  Note that when DIAG = 'U' or 'u' the elements of the array A
  corresponding to the diagonal elements of the matrix are not
  referenced, but are assumed to be unity.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least ( k + 1 ).

X
  X is COMPLEX array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the n
  element right-hand side vector b. On exit, X is overwritten
  with the solution vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.
```

———-

12.13.3 fortran code

— **ctbsv.f** —

```
*  ===================================================================
      SUBROUTINE CTBSV(UPLO,TRANS,DIAG,N,K,A,LDA,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,K,LDA,N
```

```
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),X(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP
      INTEGER I,INFO,IX,J,JX,KPLUS1,KX,L
      LOGICAL NOCONJ,NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG,MAX,MIN
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (K.LT.0) THEN
          INFO = 5
      ELSE IF (LDA.LT. (K+1)) THEN
          INFO = 7
      ELSE IF (INCX.EQ.0) THEN
          INFO = 9
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CTBSV ',INFO)
          RETURN
```

```
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOCONJ = LSAME(TRANS,'T')
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX  too small for descending loops.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed by sequentially with one pass through A.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x := inv( A )*x.
*
          IF (LSAME(UPLO,'U')) THEN
              KPLUS1 = K + 1
              IF (INCX.EQ.1) THEN
                  DO 20 J = N,1,-1
                      IF (X(J).NE.ZERO) THEN
                          L = KPLUS1 - J
                          IF (NOUNIT) X(J) = X(J)/A(KPLUS1,J)
                          TEMP = X(J)
                          DO 10 I = J - 1,MAX(1,J-K),-1
                              X(I) = X(I) - TEMP*A(L+I,J)
   10                     CONTINUE
                      END IF
   20             CONTINUE
              ELSE
                  KX = KX + (N-1)*INCX
                  JX = KX
                  DO 40 J = N,1,-1
                      KX = KX - INCX
                      IF (X(JX).NE.ZERO) THEN
                          IX = KX
                          L = KPLUS1 - J
                          IF (NOUNIT) X(JX) = X(JX)/A(KPLUS1,J)
                          TEMP = X(JX)
                          DO 30 I = J - 1,MAX(1,J-K),-1
                              X(IX) = X(IX) - TEMP*A(L+I,J)
```

```
                            IX = IX - INCX
   30                    CONTINUE
                     END IF
                     JX = JX - INCX
   40            CONTINUE
             END IF
         ELSE
             IF (INCX.EQ.1) THEN
                 DO 60 J = 1,N
                     IF (X(J).NE.ZERO) THEN
                         L = 1 - J
                         IF (NOUNIT) X(J) = X(J)/A(1,J)
                         TEMP = X(J)
                         DO 50 I = J + 1,MIN(N,J+K)
                             X(I) = X(I) - TEMP*A(L+I,J)
   50                    CONTINUE
                     END IF
   60            CONTINUE
             ELSE
                 JX = KX
                 DO 80 J = 1,N
                     KX = KX + INCX
                     IF (X(JX).NE.ZERO) THEN
                         IX = KX
                         L = 1 - J
                         IF (NOUNIT) X(JX) = X(JX)/A(1,J)
                         TEMP = X(JX)
                         DO 70 I = J + 1,MIN(N,J+K)
                             X(IX) = X(IX) - TEMP*A(L+I,J)
                             IX = IX + INCX
   70                    CONTINUE
                     END IF
                     JX = JX + INCX
   80            CONTINUE
             END IF
         END IF
      ELSE
*
*        Form  x := inv( A**T )*x  or  x := inv( A**H )*x.
*
         IF (LSAME(UPLO,'U')) THEN
             KPLUS1 = K + 1
             IF (INCX.EQ.1) THEN
                 DO 110 J = 1,N
                     TEMP = X(J)
                     L = KPLUS1 - J
                     IF (NOCONJ) THEN
                         DO 90 I = MAX(1,J-K),J - 1
                             TEMP = TEMP - A(L+I,J)*X(I)
   90                    CONTINUE
```

```
                    IF (NOUNIT) TEMP = TEMP/A(KPLUS1,J)
                ELSE
                    DO 100 I = MAX(1,J-K),J - 1
                        TEMP = TEMP - CONJG(A(L+I,J))*X(I)
100                 CONTINUE
                    IF (NOUNIT) TEMP = TEMP/CONJG(A(KPLUS1,J))
                END IF
                X(J) = TEMP
110         CONTINUE
            ELSE
                JX = KX
                DO 140 J = 1,N
                    TEMP = X(JX)
                    IX = KX
                    L = KPLUS1 - J
                    IF (NOCONJ) THEN
                        DO 120 I = MAX(1,J-K),J - 1
                            TEMP = TEMP - A(L+I,J)*X(IX)
                            IX = IX + INCX
120                     CONTINUE
                        IF (NOUNIT) TEMP = TEMP/A(KPLUS1,J)
                    ELSE
                        DO 130 I = MAX(1,J-K),J - 1
                            TEMP = TEMP - CONJG(A(L+I,J))*X(IX)
                            IX = IX + INCX
130                     CONTINUE
                        IF (NOUNIT) TEMP = TEMP/CONJG(A(KPLUS1,J))
                    END IF
                    X(JX) = TEMP
                    JX = JX + INCX
                    IF (J.GT.K) KX = KX + INCX
140         CONTINUE
            END IF
        ELSE
            IF (INCX.EQ.1) THEN
                DO 170 J = N,1,-1
                    TEMP = X(J)
                    L = 1 - J
                    IF (NOCONJ) THEN
                        DO 150 I = MIN(N,J+K),J + 1,-1
                            TEMP = TEMP - A(L+I,J)*X(I)
150                     CONTINUE
                        IF (NOUNIT) TEMP = TEMP/A(1,J)
                    ELSE
                        DO 160 I = MIN(N,J+K),J + 1,-1
                            TEMP = TEMP - CONJG(A(L+I,J))*X(I)
160                     CONTINUE
                        IF (NOUNIT) TEMP = TEMP/CONJG(A(1,J))
                    END IF
                    X(J) = TEMP
```

```
 170            CONTINUE
            ELSE
                KX = KX + (N-1)*INCX
                JX = KX
                DO 200 J = N,1,-1
                    TEMP = X(JX)
                    IX = KX
                    L = 1 - J
                    IF (NOCONJ) THEN
                        DO 180 I = MIN(N,J+K),J + 1,-1
                            TEMP = TEMP - A(L+I,J)*X(IX)
                            IX = IX - INCX
 180                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/A(1,J)
                    ELSE
                        DO 190 I = MIN(N,J+K),J + 1,-1
                            TEMP = TEMP - CONJG(A(L+I,J))*X(IX)
                            IX = IX - INCX
 190                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/CONJG(A(1,J))
                    END IF
                    X(JX) = TEMP
                    JX = JX - INCX
                    IF ((N-J).GE.K) KX = KX - INCX
 200            CONTINUE
            END IF
        END IF
    END IF
*
    RETURN
*
*   End of CTBSV .
*
    END
```

———-

12.13.4 lisp code

— BLAS 2 ctbsv —

```
(let* ((zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) zero)
           (ignorable zero))
  (defun ctbsv (uplo trans diag n k a lda x incx)
    (declare (type (array f2cl-lib:complex8 (*)) x a)
             (type (f2cl-lib:integer4) incx lda k n)
```

```
          (type (string *) diag trans uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (trans character trans-%data% trans-%offset%)
       (diag character diag-%data% diag-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (x f2cl-lib:complex8 x-%data% x-%offset%))
    (prog ((noconj nil) (nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0)
           (kplus1 0) (kx 0) (l 0) (temp #C(0.0 0.0)))
      (declare (type f2cl-lib:logical noconj nounit)
               (type (f2cl-lib:integer4) i info ix j jx kplus1 kx l)
               (type (f2cl-lib:complex8) temp))
      (setf info 0)
      (cond
       ((and
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
        (setf info 1))
       ((and
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame trans "N")
            (declare (ignore var-1))
            (when var-0 (setf trans var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame trans "T")
            (declare (ignore var-1))
            (when var-0 (setf trans var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame trans "C")
            (declare (ignore var-1))
            (when var-0 (setf trans var-0))
            ret-val)))
        (setf info 2))
       ((and
         (not
```

```
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame diag "U")
      (declare (ignore var-1))
      (when var-0 (setf diag var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame diag "N")
      (declare (ignore var-1))
      (when var-0 (setf diag var-0))
      ret-val)))
  (setf info 3))
 ((< n 0) (setf info 4)) ((< k 0) (setf info 5))
 ((< lda (f2cl-lib:int-add k 1)) (setf info 7))
 ((= incx 0) (setf info 9)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "CTBSV " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (= n 0)
    (go end_label))
(setf noconj
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(setf nounit
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame diag "N")
          (declare (ignore var-1))
          (when var-0 (setf diag var-0))
          ret-val))
(cond
 ((<= incx 0)
  (setf kx
          (f2cl-lib:int-sub 1
                            (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                              incx))))
 ((/= incx 1) (setf kx 1)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame trans "N")
    (declare (ignore var-1))
    (when var-0 (setf trans var-0))
    ret-val)
  (cond
```

```
((multiple-value-bind (ret-val var-0 var-1)
     (lsame uplo "U")
   (declare (ignore var-1))
   (when var-0 (setf uplo var-0))
   ret-val)
 (setf kplus1 (f2cl-lib:int-add k 1))
 (cond
  ((= incx 1)
   (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                 ((> j 1) nil)
     (tagbody
       (cond
        ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
         (setf l (f2cl-lib:int-sub kplus1 j))
         (if nounit
             (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                     (/
                      (f2cl-lib:fref x-%data% (j) ((1 *))
                                     x-%offset%)
                      (f2cl-lib:fref a-%data% (kplus1 j)
                                     ((1 lda) (1 *)) a-%offset%))))
         (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
         (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                        (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                       ((> i
                           (max (the f2cl-lib:integer4 1)
                                (the f2cl-lib:integer4
                                     (f2cl-lib:int-add j
                                                       (f2cl-lib:int-sub
                                                        k)))))
                        nil)
           (tagbody
             (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                     (-
                      (f2cl-lib:fref x-%data% (i) ((1 *))
                                     x-%offset%)
                      (* temp
                         (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-add l i) j)
                                        ((1 lda) (1 *))
                                        a-%offset%))))
            label10))))
     label20)))
  (t
   (setf kx
           (f2cl-lib:int-add kx
                             (f2cl-lib:int-mul
                              (f2cl-lib:int-sub n 1) incx)))
   (setf jx kx)
   (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
```

```
                  ((> j 1) nil)
        (tagbody
          (setf kx (f2cl-lib:int-sub kx incx))
          (cond
           ((/= (f2cl-lib:fref x (jx) ((1 *))) zero) (setf ix kx)
            (setf l (f2cl-lib:int-sub kplus1 j))
            (if nounit
                (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                        (/
                         (f2cl-lib:fref x-%data% (jx) ((1 *))
                                        x-%offset%)
                         (f2cl-lib:fref a-%data% (kplus1 j)
                                        ((1 lda) (1 *)) a-%offset%))))
            (setf temp
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
            (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                           (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                          ((> i
                              (max (the f2cl-lib:integer4 1)
                                   (the f2cl-lib:integer4
                                        (f2cl-lib:int-add j
                                                          (f2cl-lib:int-sub
                                                           k)))))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                        (-
                         (f2cl-lib:fref x-%data% (ix) ((1 *))
                                        x-%offset%)
                         (* temp
                            (f2cl-lib:fref a-%data%
                                           ((f2cl-lib:int-add l i) j)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
                (setf ix (f2cl-lib:int-sub ix incx))
               label30))))
          (setf jx (f2cl-lib:int-sub jx incx))
         label40)))))
    (t
     (cond
      ((= incx 1)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
        (tagbody
          (cond
           ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
            (setf l (f2cl-lib:int-sub 1 j))
            (if nounit
                (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                        (/
```

```
                    (f2cl-lib:fref x-%data% (j) ((1 *))
                                   x-%offset%)
                    (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *))
                                   a-%offset%))))
        (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                       (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the f2cl-lib:integer4 n)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add j k))))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                  (-
                   (f2cl-lib:fref x-%data% (i) ((1 *))
                                  x-%offset%)
                   (* temp
                      (f2cl-lib:fref a-%data%
                                     ((f2cl-lib:int-add l i) j)
                                     ((1 lda) (1 *))
                                     a-%offset%))))
           label50))))
    label60)))
(t (setf jx kx)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf kx (f2cl-lib:int-add kx incx))
     (cond
      ((/= (f2cl-lib:fref x (jx) ((1 *))) zero) (setf ix kx)
       (setf l (f2cl-lib:int-sub 1 j))
       (if nounit
           (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                 (/
                  (f2cl-lib:fref x-%data% (jx) ((1 *))
                                 x-%offset%)
                  (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *))
                                 a-%offset%))))
       (setf temp
               (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
       (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                       (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the f2cl-lib:integer4 n)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add j k))))
                       nil)
         (tagbody
           (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
```

```
                        (-
                         (f2cl-lib:fref x-%data% (ix) ((1 *))
                                        x-%offset%)
                         (* temp
                            (f2cl-lib:fref a-%data%
                                           ((f2cl-lib:int-add l i) j)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
                (setf ix (f2cl-lib:int-add ix incx))
               label70))))
          (setf jx (f2cl-lib:int-add jx incx))
         label80)))))))
(t
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame uplo "U")
     (declare (ignore var-1))
     (when var-0 (setf uplo var-0))
     ret-val)
   (setf kplus1 (f2cl-lib:int-add k 1))
   (cond
    ((= incx 1)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
         (setf l (f2cl-lib:int-sub kplus1 j))
         (cond
          (noconj
           (f2cl-lib:fdo (i
                          (max (the f2cl-lib:integer4 1)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add j
                                                      (f2cl-lib:int-sub
                                                       k))))
                          (f2cl-lib:int-add i 1))
                         ((> i
                             (f2cl-lib:int-add j
                                               (f2cl-lib:int-sub 1)))
                          nil)
             (tagbody
               (setf temp
                       (- temp
                          (*
                           (f2cl-lib:fref a-%data%
                                          ((f2cl-lib:int-add l i) j)
                                          ((1 lda) (1 *)) a-%offset%)
                           (f2cl-lib:fref x-%data% (i) ((1 *))
                                          x-%offset%))))
              label90))
```

```
          (if nounit
              (setf temp
                      (/ temp
                         (f2cl-lib:fref a-%data% (kplus1 j)
                                        ((1 lda) (1 *))
                                        a-%offset%)))))
         (t
          (f2cl-lib:fdo (i
                         (max (the f2cl-lib:integer4 1)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add j
                                                     (f2cl-lib:int-sub
                                                      k))))
                         (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add j
                                              (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf temp
                      (- temp
                         (*
                          (f2cl-lib:conjg
                           (f2cl-lib:fref a-%data%
                                          ((f2cl-lib:int-add l i) j)
                                          ((1 lda) (1 *))
                                          a-%offset%))
                          (f2cl-lib:fref x-%data% (i) ((1 *))
                                         x-%offset%))))
             label100))
          (if nounit
              (setf temp
                      (/ temp
                         (f2cl-lib:conjg
                          (f2cl-lib:fref a-%data% (kplus1 j)
                                         ((1 lda) (1 *))
                                         a-%offset%)))))))
       (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
      label110)))
    (t (setf jx kx)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
         (setf ix kx)
         (setf l (f2cl-lib:int-sub kplus1 j))
         (cond
          (noconj
           (f2cl-lib:fdo (i
                          (max (the f2cl-lib:integer4 1)
```

```
                                (the f2cl-lib:integer4
                                     (f2cl-lib:int-add j
                                                       (f2cl-lib:int-sub
                                                        k))))
                         (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add j
                                              (f2cl-lib:int-sub 1)))
                         nil)
                 (tagbody
                   (setf temp
                           (- temp
                              (*
                               (f2cl-lib:fref a-%data%
                                              ((f2cl-lib:int-add l i) j)
                                              ((1 lda) (1 *)) a-%offset%)
                               (f2cl-lib:fref x-%data% (ix) ((1 *))
                                              x-%offset%))))
                   (setf ix (f2cl-lib:int-add ix incx))
                  label120))
                (if nounit
                   (setf temp
                           (/ temp
                              (f2cl-lib:fref a-%data% (kplus1 j)
                                             ((1 lda) (1 *))
                                             a-%offset%)))))
               (t
                (f2cl-lib:fdo (i
                               (max (the f2cl-lib:integer4 1)
                                    (the f2cl-lib:integer4
                                         (f2cl-lib:int-add j
                                                           (f2cl-lib:int-sub
                                                            k))))
                               (f2cl-lib:int-add i 1))
                              ((> i
                                  (f2cl-lib:int-add j
                                                    (f2cl-lib:int-sub 1)))
                               nil)
                 (tagbody
                   (setf temp
                           (- temp
                              (*
                               (f2cl-lib:conjg
                                (f2cl-lib:fref a-%data%
                                               ((f2cl-lib:int-add l i) j)
                                               ((1 lda) (1 *))
                                               a-%offset%))
                               (f2cl-lib:fref x-%data% (ix) ((1 *))
                                              x-%offset%))))
                   (setf ix (f2cl-lib:int-add ix incx))
```

```
              label130))
            (if nounit
                (setf temp
                        (/ temp
                           (f2cl-lib:conjg
                            (f2cl-lib:fref a-%data% (kplus1 j)
                                           ((1 lda) (1 *))
                                           a-%offset%)))))))
          (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
          (setf jx (f2cl-lib:int-add jx incx))
          (if (> j k)
              (setf kx (f2cl-lib:int-add kx incx)))
        label140)))))
  (t
   (cond
    ((= incx 1)
     (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   ((> j 1) nil)
       (tagbody
         (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
         (setf l (f2cl-lib:int-sub 1 j))
         (cond
          (noconj
           (f2cl-lib:fdo (i
                          (min (the f2cl-lib:integer4 n)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add j k)))
                          (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                         ((> i (f2cl-lib:int-add j 1)) nil)
             (tagbody
               (setf temp
                       (- temp
                          (*
                           (f2cl-lib:fref a-%data%
                                          ((f2cl-lib:int-add l i) j)
                                          ((1 lda) (1 *)) a-%offset%)
                           (f2cl-lib:fref x-%data% (i) ((1 *))
                                          x-%offset%))))
               label150))
            (if nounit
                (setf temp
                        (/ temp
                           (f2cl-lib:fref a-%data% (1 j)
                                          ((1 lda) (1 *))
                                          a-%offset%)))))
          (t
           (f2cl-lib:fdo (i
                          (min (the f2cl-lib:integer4 n)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add j k)))
```

```
                          (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                         ((> i (f2cl-lib:int-add j 1)) nil)
            (tagbody
              (setf temp
                      (- temp
                         (*
                          (f2cl-lib:conjg
                           (f2cl-lib:fref a-%data%
                                          ((f2cl-lib:int-add l i) j)
                                          ((1 lda) (1 *))
                                          a-%offset%))
                          (f2cl-lib:fref x-%data% (i) ((1 *))
                                         x-%offset%))))
             label160))
          (if nounit
              (setf temp
                      (/ temp
                         (f2cl-lib:conjg
                          (f2cl-lib:fref a-%data% (1 j)
                                         ((1 lda) (1 *))
                                         a-%offset%)))))))
      (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
     label170)))
(t
 (setf kx
         (f2cl-lib:int-add kx
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-sub n 1) incx)))
 (setf jx kx)
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
     (setf ix kx)
     (setf l (f2cl-lib:int-sub 1 j))
     (cond
      (noconj
       (f2cl-lib:fdo (i
                      (min (the f2cl-lib:integer4 n)
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add j k)))
                      (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     ((> i (f2cl-lib:int-add j 1)) nil)
         (tagbody
           (setf temp
                   (- temp
                      (*
                       (f2cl-lib:fref a-%data%
                                      ((f2cl-lib:int-add l i) j)
                                      ((1 lda) (1 *)) a-%offset%)
```

```
                                (f2cl-lib:fref x-%data% (ix) ((1 *))
                                               x-%offset%))))
                    (setf ix (f2cl-lib:int-sub ix incx))
                   label180))
                 (if nounit
                     (setf temp
                             (/ temp
                                (f2cl-lib:fref a-%data% (1 j)
                                               ((1 lda) (1 *))
                                               a-%offset%)))))
                (t
                 (f2cl-lib:fdo (i
                                (min (the f2cl-lib:integer4 n)
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add j k)))
                                (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                               ((> i (f2cl-lib:int-add j 1)) nil)
                   (tagbody
                     (setf temp
                             (- temp
                                (*
                                 (f2cl-lib:conjg
                                  (f2cl-lib:fref a-%data%
                                                 ((f2cl-lib:int-add l i) j)
                                                 ((1 lda) (1 *))
                                                 a-%offset%))
                                 (f2cl-lib:fref x-%data% (ix) ((1 *))
                                                x-%offset%))))
                     (setf ix (f2cl-lib:int-sub ix incx))
                    label190))
                 (if nounit
                     (setf temp
                             (/ temp
                                (f2cl-lib:conjg
                                 (f2cl-lib:fref a-%data% (1 j)
                                                ((1 lda) (1 *))
                                                a-%offset%)))))))
                (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
                (setf jx (f2cl-lib:int-sub jx incx))
                (if (>= (f2cl-lib:int-sub n j) k)
                    (setf kx (f2cl-lib:int-sub kx incx)))
               label200))))))))
 (go end_label)
end_label
 (return (values uplo trans diag nil nil nil nil nil nil))))))
```

———-

12.14 ctpmv - packed matrix-vector Ax or $A^T x$ or $A^H x$

12.14.1 Axiom unit tests

— ctpmv.input —

```
)set break resume
)sys rm -f ctpmv.output
)spool ctpmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

12.14.2 Axiom help page

— ctpmv.help —

```
======================================================================
ctpmv examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ctpmv (UPLO, TRANS, DIAG, N, AP, X, INCX)
           CTPMV

Function/Subroutine Documentation
   subroutine ctpmv (character UPLO, character TRANS, character DIAG, integer
       N, complex, dimension(*) AP, complex, dimension(*) X, integer INCX)
       CTPMV

Purpose:
   CTPMV  performs one of the matrix-vector operations

   x := A*x,   or   x := A**T*x,   or   x := A**H*x,

   where x is an n element vector and  A is an n by n unit, or non-unit,
```

```
  upper or lower triangular matrix, supplied in packed form.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the matrix is an upper or
   lower triangular matrix as follows:

   UPLO = 'U' or 'u'   A is an upper triangular matrix.

   UPLO = 'L' or 'l'   A is a lower triangular matrix.

 TRANS
   TRANS is CHARACTER*1
   On entry, TRANS specifies the operation to be performed as follows:

   TRANS = 'N' or 'n'   x := A*x.

   TRANS = 'T' or 't'   x := A**T*x.

   TRANS = 'C' or 'c'   x := A**H*x.

 DIAG
   DIAG is CHARACTER*1
   On entry, DIAG specifies whether or not A is unit
   triangular as follows:

   DIAG = 'U' or 'u'   A is assumed to be unit triangular.

   DIAG = 'N' or 'n'   A is not assumed to be unit triangular.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
   N must be at least zero.

 AP
   AP is COMPLEX array of DIMENSION at least ( ( n*( n + 1 ) )/2 ).

   Before entry with  UPLO = 'U' or 'u', the array AP must
   contain the upper triangular matrix packed sequentially,
   column by column, so that AP( 1 ) contains a( 1, 1 ),
   AP( 2 ) and AP( 3 ) contain a( 1, 2 ) and a( 2, 2 )
   respectively, and so on.

   Before entry with UPLO = 'L' or 'l', the array AP must
   contain the lower triangular matrix packed sequentially,
   column by column, so that AP( 1 ) contains a( 1, 1 ),
   AP( 2 ) and AP( 3 ) contain a( 2, 1 ) and a( 3, 1 )
```

```
  respectively, and so on.

  Note that when  DIAG = 'U' or 'u', the diagonal elements of
  A are not referenced, but are assumed to be unity.

X
  X is COMPLEX array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the n
  element vector x. On exit, X is overwritten with the
  tranformed vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.

  The vector and matrix arguments are not referenced when N = 0, or M = 0
```

12.14.3 fortran code

— ctpmv.f —

```
*  ====================================================================
      SUBROUTINE CTPMV(UPLO,TRANS,DIAG,N,AP,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX AP(*),X(*)
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*     .. Local Scalars ..
```

```
      COMPLEX TEMP
      INTEGER I,INFO,IX,J,JX,K,KK,KX
      LOGICAL NOCONJ,NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (INCX.EQ.0) THEN
          INFO = 7
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CTPMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOCONJ = LSAME(TRANS,'T')
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX  too small for descending loops.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
```

```
*
*     Start the operations. In this version the elements of AP are
*     accessed sequentially with one pass through AP.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x:= A*x.
*
          IF (LSAME(UPLO,'U')) THEN
              KK = 1
              IF (INCX.EQ.1) THEN
                  DO 20 J = 1,N
                      IF (X(J).NE.ZERO) THEN
                          TEMP = X(J)
                          K = KK
                          DO 10 I = 1,J - 1
                              X(I) = X(I) + TEMP*AP(K)
                              K = K + 1
   10                     CONTINUE
                          IF (NOUNIT) X(J) = X(J)*AP(KK+J-1)
                      END IF
                      KK = KK + J
   20             CONTINUE
              ELSE
                  JX = KX
                  DO 40 J = 1,N
                      IF (X(JX).NE.ZERO) THEN
                          TEMP = X(JX)
                          IX = KX
                          DO 30 K = KK,KK + J - 2
                              X(IX) = X(IX) + TEMP*AP(K)
                              IX = IX + INCX
   30                     CONTINUE
                          IF (NOUNIT) X(JX) = X(JX)*AP(KK+J-1)
                      END IF
                      JX = JX + INCX
                      KK = KK + J
   40             CONTINUE
              END IF
          ELSE
              KK = (N* (N+1))/2
              IF (INCX.EQ.1) THEN
                  DO 60 J = N,1,-1
                      IF (X(J).NE.ZERO) THEN
                          TEMP = X(J)
                          K = KK
                          DO 50 I = N,J + 1,-1
                              X(I) = X(I) + TEMP*AP(K)
                              K = K - 1
   50                     CONTINUE
```

```
                        IF (NOUNIT) X(J) = X(J)*AP(KK-N+J)
                    END IF
                    KK = KK - (N-J+1)
   60           CONTINUE
            ELSE
                KX = KX + (N-1)*INCX
                JX = KX
                DO 80 J = N,1,-1
                    IF (X(JX).NE.ZERO) THEN
                        TEMP = X(JX)
                        IX = KX
                        DO 70 K = KK,KK - (N- (J+1)),-1
                            X(IX) = X(IX) + TEMP*AP(K)
                            IX = IX - INCX
   70                   CONTINUE
                        IF (NOUNIT) X(JX) = X(JX)*AP(KK-N+J)
                    END IF
                    JX = JX - INCX
                    KK = KK - (N-J+1)
   80           CONTINUE
            END IF
        END IF
      ELSE
*
*       Form  x := A**T*x  or  x := A**H*x.
*
         IF (LSAME(UPLO,'U')) THEN
            KK = (N* (N+1))/2
            IF (INCX.EQ.1) THEN
                DO 110 J = N,1,-1
                    TEMP = X(J)
                    K = KK - 1
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*AP(KK)
                        DO 90 I = J - 1,1,-1
                            TEMP = TEMP + AP(K)*X(I)
                            K = K - 1
   90                   CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*CONJG(AP(KK))
                        DO 100 I = J - 1,1,-1
                            TEMP = TEMP + CONJG(AP(K))*X(I)
                            K = K - 1
  100                   CONTINUE
                    END IF
                    X(J) = TEMP
                    KK = KK - J
  110           CONTINUE
            ELSE
                JX = KX + (N-1)*INCX
```

```
                DO 140 J = N,1,-1
                    TEMP = X(JX)
                    IX = JX
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*AP(KK)
                        DO 120 K = KK - 1,KK - J + 1,-1
                            IX = IX - INCX
                            TEMP = TEMP + AP(K)*X(IX)
 120                    CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*CONJG(AP(KK))
                        DO 130 K = KK - 1,KK - J + 1,-1
                            IX = IX - INCX
                            TEMP = TEMP + CONJG(AP(K))*X(IX)
 130                    CONTINUE
                    END IF
                    X(JX) = TEMP
                    JX = JX - INCX
                    KK = KK - J
 140            CONTINUE
            END IF
        ELSE
            KK = 1
            IF (INCX.EQ.1) THEN
                DO 170 J = 1,N
                    TEMP = X(J)
                    K = KK + 1
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*AP(KK)
                        DO 150 I = J + 1,N
                            TEMP = TEMP + AP(K)*X(I)
                            K = K + 1
 150                    CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*CONJG(AP(KK))
                        DO 160 I = J + 1,N
                            TEMP = TEMP + CONJG(AP(K))*X(I)
                            K = K + 1
 160                    CONTINUE
                    END IF
                    X(J) = TEMP
                    KK = KK + (N-J+1)
 170            CONTINUE
            ELSE
                JX = KX
                DO 200 J = 1,N
                    TEMP = X(JX)
                    IX = JX
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*AP(KK)
```

```
                        DO 180 K = KK + 1,KK + N - J
                            IX = IX + INCX
                            TEMP = TEMP + AP(K)*X(IX)
  180                   CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*CONJG(AP(KK))
                        DO 190 K = KK + 1,KK + N - J
                            IX = IX + INCX
                            TEMP = TEMP + CONJG(AP(K))*X(IX)
  190                   CONTINUE
                    END IF
                    X(JX) = TEMP
                    JX = JX + INCX
                    KK = KK + (N-J+1)
  200             CONTINUE
            END IF
        END IF
    END IF
*
    RETURN
*
*   End of CTPMV .
*
    END
```

———-

12.14.4 lisp code

— BLAS 2 ctpmv —

```
(let* ((zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) zero)
           (ignorable zero))
  (defun ctpmv (uplo trans diag n ap x incx)
    (declare (type (array f2cl-lib:complex8 (*)) x ap)
             (type (f2cl-lib:integer4) incx n)
             (type (string *) diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (ap f2cl-lib:complex8 ap-%data% ap-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%))
      (prog ((noconj nil) (nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0) (k 0)
             (kk 0) (kx 0) (temp #C(0.0 0.0)))
        (declare (type f2cl-lib:logical noconj nounit)
```

```
        (type (f2cl-lib:integer4) i info ix j jx k kk kx)
        (type (f2cl-lib:complex8) temp))
(setf info 0)
(cond
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)))
  (setf info 1))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame trans "N")
      (declare (ignore var-1))
      (when var-0 (setf trans var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame trans "T")
      (declare (ignore var-1))
      (when var-0 (setf trans var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame trans "C")
      (declare (ignore var-1))
      (when var-0 (setf trans var-0))
      ret-val)))
  (setf info 2))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame diag "U")
      (declare (ignore var-1))
      (when var-0 (setf diag var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame diag "N")
      (declare (ignore var-1))
      (when var-0 (setf diag var-0))
```

```
      ret-val)))
  (setf info 3))
 ((< n 0) (setf info 4)) ((= incx 0) (setf info 7)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "CTPMV " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (= n 0)
    (go end_label))
(setf noconj
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(setf nounit
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame diag "N")
          (declare (ignore var-1))
          (when var-0 (setf diag var-0))
          ret-val))
(cond
 ((<= incx 0)
  (setf kx
          (f2cl-lib:int-sub 1
                            (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                              incx))))
 ((/= incx 1) (setf kx 1)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame trans "N")
    (declare (ignore var-1))
    (when var-0 (setf trans var-0))
    ret-val)
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)
    (setf kk 1)
    (cond
     ((= incx 1)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
```

```
          ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
           (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
           (setf k kk)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i
                             (f2cl-lib:int-add j
                                               (f2cl-lib:int-sub 1)))
                          nil)
             (tagbody
               (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                       (+
                        (f2cl-lib:fref x-%data% (i) ((1 *))
                                       x-%offset%)
                        (* temp
                           (f2cl-lib:fref ap-%data% (k) ((1 *))
                                          ap-%offset%))))
               (setf k (f2cl-lib:int-add k 1))
              label10))
           (if nounit
               (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                       (*
                        (f2cl-lib:fref x-%data% (j) ((1 *))
                                       x-%offset%)
                        (f2cl-lib:fref ap-%data%
                                       ((f2cl-lib:int-sub
                                         (f2cl-lib:int-add kk j) 1))
                                       ((1 *)) ap-%offset%))))))
       (setf kk (f2cl-lib:int-add kk j))
      label20)))
  (t (setf jx kx)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
         (setf temp
                 (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
         (setf ix kx)
         (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                       ((> k
                           (f2cl-lib:int-add kk j
                                             (f2cl-lib:int-sub 2)))
                        nil)
           (tagbody
             (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                     (+
                      (f2cl-lib:fref x-%data% (ix) ((1 *))
                                     x-%offset%)
                      (* temp
                         (f2cl-lib:fref ap-%data% (k) ((1 *))
```

```
                                        ap-%offset%))))
                 (setf ix (f2cl-lib:int-add ix incx))
               label30))
             (if nounit
                 (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                         (*
                          (f2cl-lib:fref x-%data% (jx) ((1 *))
                                         x-%offset%)
                          (f2cl-lib:fref ap-%data%
                                         ((f2cl-lib:int-sub
                                           (f2cl-lib:int-add kk j) 1))
                                         ((1 *)) ap-%offset%))))))
           (setf jx (f2cl-lib:int-add jx incx))
           (setf kk (f2cl-lib:int-add kk j))
          label40)))))
      (t (setf kk (the f2cl-lib:integer4 (truncate (* n (+ n 1)) 2)))
       (cond
        ((= incx 1)
         (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((> j 1) nil)
           (tagbody
             (cond
              ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
               (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
               (setf k kk)
               (f2cl-lib:fdo (i n
                              (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                             ((> i (f2cl-lib:int-add j 1)) nil)
                 (tagbody
                   (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                           (+
                            (f2cl-lib:fref x-%data% (i) ((1 *))
                                           x-%offset%)
                            (* temp
                               (f2cl-lib:fref ap-%data% (k) ((1 *))
                                              ap-%offset%))))
                   (setf k (f2cl-lib:int-sub k 1))
                 label50))
               (if nounit
                   (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                           (*
                            (f2cl-lib:fref x-%data% (j) ((1 *))
                                           x-%offset%)
                            (f2cl-lib:fref ap-%data%
                                           ((f2cl-lib:int-add
                                             (f2cl-lib:int-sub kk n) j))
                                           ((1 *)) ap-%offset%))))))
               (setf kk
                       (f2cl-lib:int-sub kk
                                         (f2cl-lib:int-add
```

```
                                    (f2cl-lib:int-sub n j) 1)))
      label60)))
   (t
    (setf kx
            (f2cl-lib:int-add kx
                              (f2cl-lib:int-mul
                               (f2cl-lib:int-sub n 1) incx)))
    (setf jx kx)
    (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                  ((> j 1) nil)
      (tagbody
        (cond
         ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
          (setf temp
                  (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
          (setf ix kx)
          (f2cl-lib:fdo (k kk
                         (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                        ((> k
                            (f2cl-lib:int-add kk
                                              (f2cl-lib:int-sub
                                               (f2cl-lib:int-add n
                                                                 (f2cl-lib:int-sub
                                                                  (f2cl-lib:int-add
                                                                   j
                                                                   1))))))
                        nil)
            (tagbody
              (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                    (+
                     (f2cl-lib:fref x-%data% (ix) ((1 *))
                                    x-%offset%)
                     (* temp
                        (f2cl-lib:fref ap-%data% (k) ((1 *))
                                       ap-%offset%))))
              (setf ix (f2cl-lib:int-sub ix incx))
             label70))
          (if nounit
              (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                    (*
                     (f2cl-lib:fref x-%data% (jx) ((1 *))
                                    x-%offset%)
                     (f2cl-lib:fref ap-%data%
                                    ((f2cl-lib:int-add
                                      (f2cl-lib:int-sub kk n) j))
                                    ((1 *)) ap-%offset%))))))
        (setf jx (f2cl-lib:int-sub jx incx))
        (setf kk
                (f2cl-lib:int-sub kk
                                  (f2cl-lib:int-add
```

```
                                        (f2cl-lib:int-sub n j) 1)))
          label80)))))))
  (t
   (cond
    ((multiple-value-bind (ret-val var-0 var-1)
         (lsame uplo "U")
       (declare (ignore var-1))
       (when var-0 (setf uplo var-0))
       ret-val)
     (setf kk (the f2cl-lib:integer4 (truncate (* n (+ n 1)) 2)))
     (cond
      ((= incx 1)
       (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     ((> j 1) nil)
         (tagbody
           (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
           (setf k (f2cl-lib:int-sub kk 1))
           (cond
            (noconj
             (if nounit
                 (setf temp
                         (* temp
                            (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                           ap-%offset%))))
             (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                            (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                           ((> i 1) nil)
               (tagbody
                 (setf temp
                         (+ temp
                            (*
                             (f2cl-lib:fref ap-%data% (k) ((1 *))
                                            ap-%offset%)
                             (f2cl-lib:fref x-%data% (i) ((1 *))
                                            x-%offset%))))
                 (setf k (f2cl-lib:int-sub k 1))
                label90)))
            (t
             (if nounit
                 (setf temp
                         (* temp
                            (f2cl-lib:conjg
                             (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                            ap-%offset%)))))
             (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                            (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                           ((> i 1) nil)
               (tagbody
                 (setf temp
                         (+ temp
```

```
                        (*
                         (f2cl-lib:conjg
                          (f2cl-lib:fref ap-%data% (k) ((1 *))
                                         ap-%offset%))
                         (f2cl-lib:fref x-%data% (i) ((1 *))
                                        x-%offset%))))
                (setf k (f2cl-lib:int-sub k 1))
               label100))))
          (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
          (setf kk (f2cl-lib:int-sub kk j))
         label110)))
     (t
      (setf jx
              (f2cl-lib:int-add kx
                                (f2cl-lib:int-mul
                                 (f2cl-lib:int-sub n 1) incx)))
      (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                    ((> j 1) nil)
        (tagbody
          (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
          (setf ix jx)
          (cond
           (noconj
            (if nounit
                (setf temp
                        (* temp
                           (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                          ap-%offset%))))
            (f2cl-lib:fdo (k (f2cl-lib:int-add kk (f2cl-lib:int-sub 1))
                           (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                          ((> k
                              (f2cl-lib:int-add kk (f2cl-lib:int-sub j)
                                                1))
                           nil)
              (tagbody
                (setf ix (f2cl-lib:int-sub ix incx))
                (setf temp
                        (+ temp
                           (*
                            (f2cl-lib:fref ap-%data% (k) ((1 *))
                                           ap-%offset%)
                            (f2cl-lib:fref x-%data% (ix) ((1 *))
                                           x-%offset%))))
               label120)))
           (t
            (if nounit
                (setf temp
                        (* temp
                           (f2cl-lib:conjg
                            (f2cl-lib:fref ap-%data% (kk) ((1 *))
```

```
                                        ap-%offset%)))))
              (f2cl-lib:fdo (k (f2cl-lib:int-add kk (f2cl-lib:int-sub 1))
                             (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                            ((> k
                                (f2cl-lib:int-add kk (f2cl-lib:int-sub j)
                                                  1))
                             nil)
                (tagbody
                  (setf ix (f2cl-lib:int-sub ix incx))
                  (setf temp
                          (+ temp
                             (*
                              (f2cl-lib:conjg
                               (f2cl-lib:fref ap-%data% (k) ((1 *))
                                              ap-%offset%))
                              (f2cl-lib:fref x-%data% (ix) ((1 *))
                                             x-%offset%))))
                 label130))))
            (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
            (setf jx (f2cl-lib:int-sub jx incx))
            (setf kk (f2cl-lib:int-sub kk j))
           label140)))))
      (t (setf kk 1)
       (cond
        ((= incx 1)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
             (setf k (f2cl-lib:int-add kk 1))
             (cond
              (noconj
               (if nounit
                   (setf temp
                           (* temp
                              (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                             ap-%offset%))))
               (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                              (f2cl-lib:int-add i 1))
                             ((> i n) nil)
                 (tagbody
                   (setf temp
                           (+ temp
                              (*
                               (f2cl-lib:fref ap-%data% (k) ((1 *))
                                              ap-%offset%)
                               (f2cl-lib:fref x-%data% (i) ((1 *))
                                              x-%offset%))))
                   (setf k (f2cl-lib:int-add k 1))
                  label150)))
```

```
          (t
           (if nounit
               (setf temp
                       (* temp
                          (f2cl-lib:conjg
                           (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                          ap-%offset%)))))
           (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                          (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf temp
                       (+ temp
                          (*
                           (f2cl-lib:conjg
                            (f2cl-lib:fref ap-%data% (k) ((1 *))
                                           ap-%offset%))
                           (f2cl-lib:fref x-%data% (i) ((1 *))
                                          x-%offset%))))
               (setf k (f2cl-lib:int-add k 1))
              label160))))
        (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
        (setf kk
                (f2cl-lib:int-add kk
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-sub n j) 1)))
       label170)))
    (t (setf jx kx)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
         (setf ix jx)
         (cond
          (noconj
           (if nounit
               (setf temp
                       (* temp
                          (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                         ap-%offset%))))
           (f2cl-lib:fdo (k (f2cl-lib:int-add kk 1)
                            (f2cl-lib:int-add k 1))
                          ((> k
                              (f2cl-lib:int-add kk n
                                                (f2cl-lib:int-sub j)))
                            nil)
             (tagbody
               (setf ix (f2cl-lib:int-add ix incx))
               (setf temp
                       (+ temp
```

```
                              (*
                               (f2cl-lib:fref ap-%data% (k) ((1 *))
                                              ap-%offset%)
                               (f2cl-lib:fref x-%data% (ix) ((1 *))
                                              x-%offset%))))
                 label180)))
              (t
               (if nounit
                   (setf temp
                           (* temp
                              (f2cl-lib:conjg
                               (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                              ap-%offset%)))))
               (f2cl-lib:fdo (k (f2cl-lib:int-add kk 1)
                              (f2cl-lib:int-add k 1))
                             ((> k
                                 (f2cl-lib:int-add kk n
                                                   (f2cl-lib:int-sub j)))
                              nil)
                 (tagbody
                   (setf ix (f2cl-lib:int-add ix incx))
                   (setf temp
                           (+ temp
                              (*
                               (f2cl-lib:conjg
                                (f2cl-lib:fref ap-%data% (k) ((1 *))
                                               ap-%offset%))
                               (f2cl-lib:fref x-%data% (ix) ((1 *))
                                              x-%offset%))))
                 label190))))
            (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
            (setf jx (f2cl-lib:int-add jx incx))
            (setf kk
                    (f2cl-lib:int-add kk
                                      (f2cl-lib:int-add
                                       (f2cl-lib:int-sub n j) 1)))
          label200))))))))
 (go end_label)
end_label
 (return (values uplo trans diag nil nil nil nil))))))
```

———-

12.15 ctpsv - solve system of equations $Ax = b$ or $A^T x = b$ or $A^H x = b$

12.15.1 Axiom unit tests

— **ctpsv.input** —

```
)set break resume
)sys rm -f ctpsv.output
)spool ctpsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

12.15.2 Axiom help page

— **ctpsv.help** —

```
==========================================================================
ctpsv examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ctpsv (UPLO, TRANS, DIAG, N, AP, X, INCX)
           CTPSV

Function/Subroutine Documentation
   subroutine ctpsv (character UPLO, character TRANS, character DIAG, integer
       N, complex, dimension(*) AP, complex, dimension(*) X, integer INCX)
       CTPSV

 Purpose:
   CTPSV  solves one of the systems of equations

   A*x = b,   or   A**T*x = b,   or   A**H*x = b,
```

```
   where b and x are n element vectors and A is an n by n unit, or
   non-unit, upper or lower triangular matrix, supplied in packed form.

   No test for singularity or near-singularity is included in this
   routine. Such tests must be performed before calling this routine.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the matrix is an upper or
   lower triangular matrix as follows:

   UPLO = 'U' or 'u'   A is an upper triangular matrix.

   UPLO = 'L' or 'l'   A is a lower triangular matrix.

 TRANS

   TRANS is CHARACTER*1
   On entry, TRANS specifies the equations to be solved as follows:

   TRANS = 'N' or 'n'   A*x = b.

   TRANS = 'T' or 't'   A**T*x = b.

   TRANS = 'C' or 'c'   A**H*x = b.

 DIAG

   DIAG is CHARACTER*1
   On entry, DIAG specifies whether or not A is unit
   triangular as follows:

   DIAG = 'U' or 'u'   A is assumed to be unit triangular.

   DIAG = 'N' or 'n'   A is not assumed to be unit triangular.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
   N must be at least zero.

 AP
   AP is COMPLEX array of DIMENSION at least ( ( n*( n + 1 ) )/2 ).

   Before entry with  UPLO = 'U' or 'u', the array AP must
   contain the upper triangular matrix packed sequentially,
   column by column, so that AP( 1 ) contains a( 1, 1 ),
```

```
  AP( 2 ) and AP( 3 ) contain a( 1, 2 ) and a( 2, 2 )
  respectively, and so on.

  Before entry with UPLO = 'L' or 'l', the array AP must
  contain the lower triangular matrix packed sequentially,
  column by column, so that AP( 1 ) contains a( 1, 1 ),
  AP( 2 ) and AP( 3 ) contain a( 2, 1 ) and a( 3, 1 )
  respectively, and so on.

  Note that when  DIAG = 'U' or 'u', the diagonal elements of
  A are not referenced, but are assumed to be unity.

X
  X is COMPLEX array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the n
  element right-hand side vector b. On exit, X is overwritten
  with the solution vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.
```

———-

12.15.3 fortran code

— ctpsv.f —

```
*  ===================================================================
      SUBROUTINE CTPSV(UPLO,TRANS,DIAG,N,AP,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX AP(*),X(*)
*     ..
*
*  ===================================================================
*
```

```
*     .. Parameters ..
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP
      INTEGER I,INFO,IX,J,JX,K,KK,KX
      LOGICAL NOCONJ,NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (INCX.EQ.0) THEN
          INFO = 7
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CTPSV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOCONJ = LSAME(TRANS,'T')
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX  too small for descending loops.
*
```

```
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of AP are
*     accessed sequentially with one pass through AP.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x := inv( A )*x.
*
          IF (LSAME(UPLO,'U')) THEN
              KK = (N* (N+1))/2
              IF (INCX.EQ.1) THEN
                  DO 20 J = N,1,-1
                      IF (X(J).NE.ZERO) THEN
                          IF (NOUNIT) X(J) = X(J)/AP(KK)
                          TEMP = X(J)
                          K = KK - 1
                          DO 10 I = J - 1,1,-1
                              X(I) = X(I) - TEMP*AP(K)
                              K = K - 1
   10                     CONTINUE
                      END IF
                      KK = KK - J
   20             CONTINUE
              ELSE
                  JX = KX + (N-1)*INCX
                  DO 40 J = N,1,-1
                      IF (X(JX).NE.ZERO) THEN
                          IF (NOUNIT) X(JX) = X(JX)/AP(KK)
                          TEMP = X(JX)
                          IX = JX
                          DO 30 K = KK - 1,KK - J + 1,-1
                              IX = IX - INCX
                              X(IX) = X(IX) - TEMP*AP(K)
   30                     CONTINUE
                      END IF
                      JX = JX - INCX
                      KK = KK - J
   40             CONTINUE
              END IF
          ELSE
              KK = 1
              IF (INCX.EQ.1) THEN
                  DO 60 J = 1,N
                      IF (X(J).NE.ZERO) THEN
                          IF (NOUNIT) X(J) = X(J)/AP(KK)
```

```
                        TEMP = X(J)
                        K = KK + 1
                        DO 50 I = J + 1,N
                            X(I) = X(I) - TEMP*AP(K)
                            K = K + 1
   50                   CONTINUE
                    END IF
                    KK = KK + (N-J+1)
   60           CONTINUE
            ELSE
                JX = KX
                DO 80 J = 1,N
                    IF (X(JX).NE.ZERO) THEN
                        IF (NOUNIT) X(JX) = X(JX)/AP(KK)
                        TEMP = X(JX)
                        IX = JX
                        DO 70 K = KK + 1,KK + N - J
                            IX = IX + INCX
                            X(IX) = X(IX) - TEMP*AP(K)
   70                   CONTINUE
                    END IF
                    JX = JX + INCX
                    KK = KK + (N-J+1)
   80           CONTINUE
            END IF
        END IF
     ELSE
*
*       Form  x := inv( A**T )*x  or  x := inv( A**H )*x.
*
        IF (LSAME(UPLO,'U')) THEN
            KK = 1
            IF (INCX.EQ.1) THEN
                DO 110 J = 1,N
                    TEMP = X(J)
                    K = KK
                    IF (NOCONJ) THEN
                        DO 90 I = 1,J - 1
                            TEMP = TEMP - AP(K)*X(I)
                            K = K + 1
   90                   CONTINUE
                        IF (NOUNIT) TEMP = TEMP/AP(KK+J-1)
                    ELSE
                        DO 100 I = 1,J - 1
                            TEMP = TEMP - CONJG(AP(K))*X(I)
                            K = K + 1
  100                   CONTINUE
                        IF (NOUNIT) TEMP = TEMP/CONJG(AP(KK+J-1))
                    END IF
                    X(J) = TEMP
```

```
                KK = KK + J
110         CONTINUE
        ELSE
            JX = KX
            DO 140 J = 1,N
                TEMP = X(JX)
                IX = KX
                IF (NOCONJ) THEN
                    DO 120 K = KK,KK + J - 2
                        TEMP = TEMP - AP(K)*X(IX)
                        IX = IX + INCX
120                 CONTINUE
                    IF (NOUNIT) TEMP = TEMP/AP(KK+J-1)
                ELSE
                    DO 130 K = KK,KK + J - 2
                        TEMP = TEMP - CONJG(AP(K))*X(IX)
                        IX = IX + INCX
130                 CONTINUE
                    IF (NOUNIT) TEMP = TEMP/CONJG(AP(KK+J-1))
                END IF
                X(JX) = TEMP
                JX = JX + INCX
                KK = KK + J
140         CONTINUE
        END IF
    ELSE
        KK = (N* (N+1))/2
        IF (INCX.EQ.1) THEN
            DO 170 J = N,1,-1
                TEMP = X(J)
                K = KK
                IF (NOCONJ) THEN
                    DO 150 I = N,J + 1,-1
                        TEMP = TEMP - AP(K)*X(I)
                        K = K - 1
150                 CONTINUE
                    IF (NOUNIT) TEMP = TEMP/AP(KK-N+J)
                ELSE
                    DO 160 I = N,J + 1,-1
                        TEMP = TEMP - CONJG(AP(K))*X(I)
                        K = K - 1
160                 CONTINUE
                    IF (NOUNIT) TEMP = TEMP/CONJG(AP(KK-N+J))
                END IF
                X(J) = TEMP
                KK = KK - (N-J+1)
170         CONTINUE
        ELSE
            KX = KX + (N-1)*INCX
            JX = KX
```

```
                 DO 200 J = N,1,-1
                     TEMP = X(JX)
                     IX = KX
                     IF (NOCONJ) THEN
                         DO 180 K = KK,KK - (N- (J+1)),-1
                             TEMP = TEMP - AP(K)*X(IX)
                             IX = IX - INCX
  180                    CONTINUE
                         IF (NOUNIT) TEMP = TEMP/AP(KK-N+J)
                     ELSE
                         DO 190 K = KK,KK - (N- (J+1)),-1
                             TEMP = TEMP - CONJG(AP(K))*X(IX)
                             IX = IX - INCX
  190                    CONTINUE
                         IF (NOUNIT) TEMP = TEMP/CONJG(AP(KK-N+J))
                     END IF
                     X(JX) = TEMP
                     JX = JX - INCX
                     KK = KK - (N-J+1)
  200            CONTINUE
             END IF
         END IF
     END IF
*
     RETURN
*
*    End of CTPSV .
*
     END
```

———-

12.15.4 lisp code

— BLAS 2 ctpsv —

```
(let* ((zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) zero)
           (ignorable zero))
  (defun ctpsv (uplo trans diag n ap x incx)
    (declare (type (array f2cl-lib:complex8 (*)) x ap)
             (type (f2cl-lib:integer4) incx n)
             (type (string *) diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
```

```
    (ap f2cl-lib:complex8 ap-%data% ap-%offset%)
    (x f2cl-lib:complex8 x-%data% x-%offset%))
 (prog ((noconj nil) (nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0) (k 0)
        (kk 0) (kx 0) (temp #C(0.0 0.0)))
   (declare (type f2cl-lib:logical noconj nounit)
            (type (f2cl-lib:integer4) i info ix j jx k kk kx)
            (type (f2cl-lib:complex8) temp))
   (setf info 0)
   (cond
    ((and
      (not
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame uplo "U")
         (declare (ignore var-1))
         (when var-0 (setf uplo var-0))
         ret-val))
      (not
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame uplo "L")
         (declare (ignore var-1))
         (when var-0 (setf uplo var-0))
         ret-val)))
     (setf info 1))
    ((and
      (not
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame trans "N")
         (declare (ignore var-1))
         (when var-0 (setf trans var-0))
         ret-val))
      (not
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame trans "T")
         (declare (ignore var-1))
         (when var-0 (setf trans var-0))
         ret-val))
      (not
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame trans "C")
         (declare (ignore var-1))
         (when var-0 (setf trans var-0))
         ret-val)))
     (setf info 2))
    ((and
      (not
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame diag "U")
         (declare (ignore var-1))
         (when var-0 (setf diag var-0))
         ret-val))
```

```
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame diag "N")
      (declare (ignore var-1))
      (when var-0 (setf diag var-0))
      ret-val)))
  (setf info 3))
 ((< n 0) (setf info 4)) ((= incx 0) (setf info 7)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "CTPSV " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (= n 0)
    (go end_label))
(setf noconj
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(setf nounit
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame diag "N")
          (declare (ignore var-1))
          (when var-0 (setf diag var-0))
          ret-val))
(cond
 ((<= incx 0)
  (setf kx
          (f2cl-lib:int-sub 1
                            (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                              incx))))
 ((/= incx 1) (setf kx 1)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame trans "N")
    (declare (ignore var-1))
    (when var-0 (setf trans var-0))
    ret-val)
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)
    (setf kk (the f2cl-lib:integer4 (truncate (* n (+ n 1)) 2)))
    (cond
```

```
((= incx 1)
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (cond
      ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
       (if nounit
           (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                   (/
                    (f2cl-lib:fref x-%data% (j) ((1 *))
                                   x-%offset%)
                    (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                   ap-%offset%))))
       (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
       (setf k (f2cl-lib:int-sub kk 1))
       (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                        (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
         (tagbody
           (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                   (-
                    (f2cl-lib:fref x-%data% (i) ((1 *))
                                   x-%offset%)
                    (* temp
                       (f2cl-lib:fref ap-%data% (k) ((1 *))
                                      ap-%offset%))))
           (setf k (f2cl-lib:int-sub k 1))
          label10))))
     (setf kk (f2cl-lib:int-sub kk j))
    label20)))
(t
 (setf jx
         (f2cl-lib:int-add kx
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-sub n 1) incx)))
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (cond
      ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
       (if nounit
           (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                   (/
                    (f2cl-lib:fref x-%data% (jx) ((1 *))
                                   x-%offset%)
                    (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                   ap-%offset%))))
       (setf temp
               (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
       (setf ix jx)
```

```
            (f2cl-lib:fdo (k (f2cl-lib:int-add kk (f2cl-lib:int-sub 1))
                           (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                          ((> k
                              (f2cl-lib:int-add kk (f2cl-lib:int-sub j)
                                                1))
                           nil)
              (tagbody
                (setf ix (f2cl-lib:int-sub ix incx))
                (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                        (-
                         (f2cl-lib:fref x-%data% (ix) ((1 *))
                                        x-%offset%)
                         (* temp
                            (f2cl-lib:fref ap-%data% (k) ((1 *))
                                           ap-%offset%))))
               label30))))
          (setf jx (f2cl-lib:int-sub jx incx))
          (setf kk (f2cl-lib:int-sub kk j))
         label40)))))
  (t (setf kk 1)
   (cond
    ((= incx 1)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (cond
          ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
           (if nounit
               (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                       (/
                        (f2cl-lib:fref x-%data% (j) ((1 *))
                                       x-%offset%)
                        (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                       ap-%offset%))))
           (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
           (setf k (f2cl-lib:int-add kk 1))
           (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                            (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                       (-
                        (f2cl-lib:fref x-%data% (i) ((1 *))
                                       x-%offset%)
                        (* temp
                           (f2cl-lib:fref ap-%data% (k) ((1 *))
                                          ap-%offset%))))
               (setf k (f2cl-lib:int-add k 1))
              label50))))
          (setf kk
```

```
                  (f2cl-lib:int-add kk
                                    (f2cl-lib:int-add
                                     (f2cl-lib:int-sub n j) 1)))
         label60)))
     (t (setf jx kx)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
           ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
            (if nounit
                (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                        (/
                         (f2cl-lib:fref x-%data% (jx) ((1 *))
                                        x-%offset%)
                         (f2cl-lib:fref ap-%data% (kk) ((1 *))
                                        ap-%offset%))))
            (setf temp
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
            (setf ix jx)
            (f2cl-lib:fdo (k (f2cl-lib:int-add kk 1)
                           (f2cl-lib:int-add k 1))
                          ((> k
                              (f2cl-lib:int-add kk n
                                                (f2cl-lib:int-sub j)))
                           nil)
              (tagbody
                (setf ix (f2cl-lib:int-add ix incx))
                (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                        (-
                         (f2cl-lib:fref x-%data% (ix) ((1 *))
                                        x-%offset%)
                         (* temp
                            (f2cl-lib:fref ap-%data% (k) ((1 *))
                                           ap-%offset%))))
               label70))))
          (setf jx (f2cl-lib:int-add jx incx))
          (setf kk
                  (f2cl-lib:int-add kk
                                    (f2cl-lib:int-add
                                     (f2cl-lib:int-sub n j) 1)))
         label80)))))))
(t
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame uplo "U")
     (declare (ignore var-1))
     (when var-0 (setf uplo var-0))
     ret-val)
   (setf kk 1)
```

```
(cond
 ((= incx 1)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
      (setf k kk)
      (cond
       (noconj
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i
                          (f2cl-lib:int-add j
                                            (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf temp
                    (- temp
                       (*
                        (f2cl-lib:fref ap-%data% (k) ((1 *))
                                       ap-%offset%)
                        (f2cl-lib:fref x-%data% (i) ((1 *))
                                       x-%offset%))))
            (setf k (f2cl-lib:int-add k 1))
           label90))
        (if nounit
            (setf temp
                    (/ temp
                       (f2cl-lib:fref ap-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add kk j) 1))
                                      ((1 *)) ap-%offset%)))))
       (t
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i
                          (f2cl-lib:int-add j
                                            (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf temp
                    (- temp
                       (*
                        (f2cl-lib:conjg
                         (f2cl-lib:fref ap-%data% (k) ((1 *))
                                        ap-%offset%))
                        (f2cl-lib:fref x-%data% (i) ((1 *))
                                       x-%offset%))))
            (setf k (f2cl-lib:int-add k 1))
           label100))
        (if nounit
            (setf temp
```

```
                        (/ temp
                           (f2cl-lib:conjg
                            (f2cl-lib:fref ap-%data%
                                           ((f2cl-lib:int-sub
                                             (f2cl-lib:int-add kk j)
                                             1))
                                           ((1 *)) ap-%offset%)))))))
      (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
      (setf kk (f2cl-lib:int-add kk j))
     label110)))
 (t (setf jx kx)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
      (setf ix kx)
      (cond
       (noconj
        (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                      ((> k
                          (f2cl-lib:int-add kk j
                                            (f2cl-lib:int-sub 2)))
                       nil)
          (tagbody
            (setf temp
                    (- temp
                       (*
                        (f2cl-lib:fref ap-%data% (k) ((1 *))
                                       ap-%offset%)
                        (f2cl-lib:fref x-%data% (ix) ((1 *))
                                       x-%offset%))))
            (setf ix (f2cl-lib:int-add ix incx))
           label120))
        (if nounit
            (setf temp
                    (/ temp
                       (f2cl-lib:fref ap-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add kk j) 1))
                                      ((1 *)) ap-%offset%)))))
       (t
        (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                      ((> k
                          (f2cl-lib:int-add kk j
                                            (f2cl-lib:int-sub 2)))
                       nil)
          (tagbody
            (setf temp
                    (- temp
                       (*
```

```
                              (f2cl-lib:conjg
                               (f2cl-lib:fref ap-%data% (k) ((1 *))
                                              ap-%offset%))
                              (f2cl-lib:fref x-%data% (ix) ((1 *))
                                             x-%offset%))))
                 (setf ix (f2cl-lib:int-add ix incx))
                label130))
            (if nounit
                (setf temp
                        (/ temp
                           (f2cl-lib:conjg
                            (f2cl-lib:fref ap-%data%
                                           ((f2cl-lib:int-sub
                                             (f2cl-lib:int-add kk j)
                                             1))
                                           ((1 *)) ap-%offset%)))))))
           (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
           (setf jx (f2cl-lib:int-add jx incx))
           (setf kk (f2cl-lib:int-add kk j))
          label140)))))
  (t (setf kk (the f2cl-lib:integer4 (truncate (* n (+ n 1)) 2)))
   (cond
    ((= incx 1)
     (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   ((> j 1) nil)
       (tagbody
         (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
         (setf k kk)
         (cond
          (noconj
           (f2cl-lib:fdo (i n
                          (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                         ((> i (f2cl-lib:int-add j 1)) nil)
             (tagbody
               (setf temp
                       (- temp
                          (*
                           (f2cl-lib:fref ap-%data% (k) ((1 *))
                                          ap-%offset%)
                           (f2cl-lib:fref x-%data% (i) ((1 *))
                                          x-%offset%))))
               (setf k (f2cl-lib:int-sub k 1))
              label150))
            (if nounit
                (setf temp
                        (/ temp
                           (f2cl-lib:fref ap-%data%
                                          ((f2cl-lib:int-add
                                            (f2cl-lib:int-sub kk n) j))
                                          ((1 *)) ap-%offset%)))))
```

```
          (t
           (f2cl-lib:fdo (i n
                          (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                         ((> i (f2cl-lib:int-add j 1)) nil)
             (tagbody
               (setf temp
                       (- temp
                          (*
                           (f2cl-lib:conjg
                            (f2cl-lib:fref ap-%data% (k) ((1 *))
                                           ap-%offset%))
                           (f2cl-lib:fref x-%data% (i) ((1 *))
                                          x-%offset%))))
               (setf k (f2cl-lib:int-sub k 1))
              label160))
           (if nounit
               (setf temp
                       (/ temp
                          (f2cl-lib:conjg
                           (f2cl-lib:fref ap-%data%
                                          ((f2cl-lib:int-add
                                            (f2cl-lib:int-sub kk n)
                                            j))
                                          ((1 *)) ap-%offset%)))))))
        (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
        (setf kk
                (f2cl-lib:int-sub kk
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-sub n j) 1)))
       label170)))
     (t
      (setf kx
              (f2cl-lib:int-add kx
                                (f2cl-lib:int-mul
                                 (f2cl-lib:int-sub n 1) incx)))
      (setf jx kx)
      (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                    ((> j 1) nil)
        (tagbody
          (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
          (setf ix kx)
          (cond
           (noconj
            (f2cl-lib:fdo (k kk
                           (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                          ((> k
                              (f2cl-lib:int-add kk
                                                (f2cl-lib:int-sub
                                                 (f2cl-lib:int-add n
                                                                   (f2cl-lib:int-sub
```

```
                                                                    (f2cl-lib:int-add
                                                                     j
                                                                     1))))))
                  nil)
      (tagbody
        (setf temp
                (- temp
                   (*
                    (f2cl-lib:fref ap-%data% (k) ((1 *))
                                   ap-%offset%)
                    (f2cl-lib:fref x-%data% (ix) ((1 *))
                                   x-%offset%))))
        (setf ix (f2cl-lib:int-sub ix incx))
       label180))
    (if nounit
        (setf temp
                (/ temp
                   (f2cl-lib:fref ap-%data%
                                  ((f2cl-lib:int-add
                                    (f2cl-lib:int-sub kk n) j))
                                  ((1 *)) ap-%offset%)))))
   (t
    (f2cl-lib:fdo (k kk
                   (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                  ((> k
                      (f2cl-lib:int-add kk
                                        (f2cl-lib:int-sub
                                         (f2cl-lib:int-add n
                                                           (f2cl-lib:int-sub
                                                            (f2cl-lib:int-add
                                                             j
                                                             1))))))
                   nil)
      (tagbody
        (setf temp
                (- temp
                   (*
                    (f2cl-lib:conjg
                     (f2cl-lib:fref ap-%data% (k) ((1 *))
                                    ap-%offset%))
                    (f2cl-lib:fref x-%data% (ix) ((1 *))
                                   x-%offset%))))
        (setf ix (f2cl-lib:int-sub ix incx))
       label190))
    (if nounit
        (setf temp
                (/ temp
                   (f2cl-lib:conjg
                    (f2cl-lib:fref ap-%data%
                                   ((f2cl-lib:int-add
```

```
                                                  (f2cl-lib:int-sub kk n)
                                                  j))
                                               ((1 *)) ap-%offset%)))))))
                (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
                (setf jx (f2cl-lib:int-sub jx incx))
                (setf kk
                        (f2cl-lib:int-sub kk
                                          (f2cl-lib:int-add
                                           (f2cl-lib:int-sub n j) 1)))
              label200))))))))
        (go end_label)
       end_label
        (return (values uplo trans diag nil nil nil nil))))))
```

———-

12.16 ctrmv - matrix-vector Ax or $A^T x$ or $A^H x$

12.16.1 Axiom unit tests

— **ctrmv.input** —

```
)set break resume
)sys rm -f ctrmv.output
)spool ctrmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

12.16.2 Axiom help page

— **ctrmv.help** —

```
======================================================================
ctrmv examples
======================================================================

======================================================================
```

```
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ctrmv (UPLO, TRANS, DIAG, N, A, LDA, X, INCX)
           CTRMV

Function/Subroutine Documentation
   subroutine ctrmv (character UPLO, character TRANS, character DIAG, integer
       N, complex, dimension(lda,*) A, integer LDA, complex, dimension(*) X,
       integer INCX)
       CTRMV

Purpose:
   CTRMV  performs one of the matrix-vector operations

   x := A*x,   or   x := A**T*x,   or   x := A**H*x,

   where x is an n element vector and  A is an n by n unit, or non-unit,
   upper or lower triangular matrix.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the matrix is an upper or
   lower triangular matrix as follows:

   UPLO = 'U' or 'u'   A is an upper triangular matrix.

   UPLO = 'L' or 'l'   A is a lower triangular matrix.

 TRANS

   TRANS is CHARACTER*1
   On entry, TRANS specifies the operation to be performed as follows:

   TRANS = 'N' or 'n'   x := A*x.

   TRANS = 'T' or 't'   x := A**T*x.

   TRANS = 'C' or 'c'   x := A**H*x.

 DIAG

   DIAG is CHARACTER*1
   On entry, DIAG specifies whether or not A is unit
   triangular as follows:
```

```
  DIAG = 'U' or 'u'   A is assumed to be unit triangular.

  DIAG = 'N' or 'n'   A is not assumed to be unit triangular.

N
  N is INTEGER
  On entry, N specifies the order of the matrix A.
  N must be at least zero.

A
  A is COMPLEX array of DIMENSION ( LDA, n ).
  Before entry with  UPLO = 'U' or 'u', the leading n by n
  upper triangular part of the array A must contain the upper
  triangular matrix and the strictly lower triangular part of
  A is not referenced.

  Before entry with UPLO = 'L' or 'l', the leading n by n
  lower triangular part of the array A must contain the lower
  triangular matrix and the strictly upper triangular part of
  A is not referenced.

  Note that when  DIAG = 'U' or 'u', the diagonal elements of
  A are not referenced either, but are assumed to be unity.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, n ).

X
  X is COMPLEX array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the n
  element vector x. On exit, X is overwritten with the
  tranformed vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.

  The vector and matrix arguments are not referenced when N = 0, or M = 0
```

12.16.3 fortran code

— ctrmv.f —

```
*  =====================================================================
      SUBROUTINE CTRMV(UPLO,TRANS,DIAG,N,A,LDA,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,LDA,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),X(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP
      INTEGER I,INFO,IX,J,JX,KX
      LOGICAL NOCONJ,NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG,MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
```

```
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 6
      ELSE IF (INCX.EQ.0) THEN
          INFO = 8
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CTRMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOCONJ = LSAME(TRANS,'T')
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX  too small for descending loops.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x := A*x.
*
          IF (LSAME(UPLO,'U')) THEN
              IF (INCX.EQ.1) THEN
                  DO 20 J = 1,N
                      IF (X(J).NE.ZERO) THEN
                          TEMP = X(J)
                          DO 10 I = 1,J - 1
                              X(I) = X(I) + TEMP*A(I,J)
   10                     CONTINUE
                          IF (NOUNIT) X(J) = X(J)*A(J,J)
                      END IF
   20             CONTINUE
              ELSE
                  JX = KX
                  DO 40 J = 1,N
                      IF (X(JX).NE.ZERO) THEN
                          TEMP = X(JX)
                          IX = KX
```

```
                        DO 30 I = 1,J - 1
                            X(IX) = X(IX) + TEMP*A(I,J)
                            IX = IX + INCX
   30                   CONTINUE
                        IF (NOUNIT) X(JX) = X(JX)*A(J,J)
                    END IF
                    JX = JX + INCX
   40            CONTINUE
            END IF
        ELSE
            IF (INCX.EQ.1) THEN
                DO 60 J = N,1,-1
                    IF (X(J).NE.ZERO) THEN
                        TEMP = X(J)
                        DO 50 I = N,J + 1,-1
                            X(I) = X(I) + TEMP*A(I,J)
   50                   CONTINUE
                        IF (NOUNIT) X(J) = X(J)*A(J,J)
                    END IF
   60            CONTINUE
            ELSE
                KX = KX + (N-1)*INCX
                JX = KX
                DO 80 J = N,1,-1
                    IF (X(JX).NE.ZERO) THEN
                        TEMP = X(JX)
                        IX = KX
                        DO 70 I = N,J + 1,-1
                            X(IX) = X(IX) + TEMP*A(I,J)
                            IX = IX - INCX
   70                   CONTINUE
                        IF (NOUNIT) X(JX) = X(JX)*A(J,J)
                    END IF
                    JX = JX - INCX
   80            CONTINUE
            END IF
        END IF
    ELSE
*
*       Form  x := A**T*x  or  x := A**H*x.
*
        IF (LSAME(UPLO,'U')) THEN
            IF (INCX.EQ.1) THEN
                DO 110 J = N,1,-1
                    TEMP = X(J)
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*A(J,J)
                        DO 90 I = J - 1,1,-1
                            TEMP = TEMP + A(I,J)*X(I)
   90                   CONTINUE
```

```
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*CONJG(A(J,J))
                        DO 100 I = J - 1,1,-1
                            TEMP = TEMP + CONJG(A(I,J))*X(I)
 100                    CONTINUE
                    END IF
                    X(J) = TEMP
 110            CONTINUE
            ELSE
                JX = KX + (N-1)*INCX
                DO 140 J = N,1,-1
                    TEMP = X(JX)
                    IX = JX
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*A(J,J)
                        DO 120 I = J - 1,1,-1
                            IX = IX - INCX
                            TEMP = TEMP + A(I,J)*X(IX)
 120                    CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*CONJG(A(J,J))
                        DO 130 I = J - 1,1,-1
                            IX = IX - INCX
                            TEMP = TEMP + CONJG(A(I,J))*X(IX)
 130                    CONTINUE
                    END IF
                    X(JX) = TEMP
                    JX = JX - INCX
 140            CONTINUE
            END IF
        ELSE
            IF (INCX.EQ.1) THEN
                DO 170 J = 1,N
                    TEMP = X(J)
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*A(J,J)
                        DO 150 I = J + 1,N
                            TEMP = TEMP + A(I,J)*X(I)
 150                    CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*CONJG(A(J,J))
                        DO 160 I = J + 1,N
                            TEMP = TEMP + CONJG(A(I,J))*X(I)
 160                    CONTINUE
                    END IF
                    X(J) = TEMP
 170            CONTINUE
            ELSE
                JX = KX
                DO 200 J = 1,N
```

```
                    TEMP = X(JX)
                    IX = JX
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*A(J,J)
                        DO 180 I = J + 1,N
                            IX = IX + INCX
                            TEMP = TEMP + A(I,J)*X(IX)
  180                   CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*CONJG(A(J,J))
                        DO 190 I = J + 1,N
                            IX = IX + INCX
                            TEMP = TEMP + CONJG(A(I,J))*X(IX)
  190                   CONTINUE
                    END IF
                    X(JX) = TEMP
                    JX = JX + INCX
  200            CONTINUE
             END IF
         END IF
     END IF
*
      RETURN
*
*     End of CTRMV .
*
      END
```

———-

12.16.4 lisp code

— BLAS 2 ctrmv —

```
(let* ((zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) zero)
           (ignorable zero))
  (defun ctrmv (uplo trans diag n a lda x incx)
    (declare (type (array f2cl-lib:complex8 (*)) x a)
             (type (f2cl-lib:integer4) incx lda n)
             (type (string *) diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%))
```

```
(prog ((noconj nil) (nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0)
       (kx 0) (temp #C(0.0 0.0)))
  (declare (type f2cl-lib:logical noconj nounit)
           (type (f2cl-lib:integer4) i info ix j jx kx)
           (type (f2cl-lib:complex8) temp))
  (setf info 0)
  (cond
   ((and
     (not
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame uplo "U")
        (declare (ignore var-1))
        (when var-0 (setf uplo var-0))
        ret-val))
     (not
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame uplo "L")
        (declare (ignore var-1))
        (when var-0 (setf uplo var-0))
        ret-val)))
    (setf info 1))
   ((and
     (not
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame trans "N")
        (declare (ignore var-1))
        (when var-0 (setf trans var-0))
        ret-val))
     (not
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame trans "T")
        (declare (ignore var-1))
        (when var-0 (setf trans var-0))
        ret-val))
     (not
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame trans "C")
        (declare (ignore var-1))
        (when var-0 (setf trans var-0))
        ret-val)))
    (setf info 2))
   ((and
     (not
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame diag "U")
        (declare (ignore var-1))
        (when var-0 (setf diag var-0))
        ret-val))
     (not
      (multiple-value-bind (ret-val var-0 var-1)
```

```
        (lsame diag "N")
      (declare (ignore var-1))
      (when var-0 (setf diag var-0))
      ret-val)))
  (setf info 3))
 ((< n 0) (setf info 4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info 6))
 ((= incx 0) (setf info 8)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "CTRMV " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (= n 0)
    (go end_label))
(setf noconj
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(setf nounit
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame diag "N")
          (declare (ignore var-1))
          (when var-0 (setf diag var-0))
          ret-val))
(cond
 ((<= incx 0)
  (setf kx
          (f2cl-lib:int-sub 1
                            (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                              incx))))
 ((/= incx 1) (setf kx 1)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame trans "N")
    (declare (ignore var-1))
    (when var-0 (setf trans var-0))
    ret-val)
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)
    (cond
```

```
((= incx 1)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
      ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
       (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i
                         (f2cl-lib:int-add j
                                           (f2cl-lib:int-sub 1)))
                      nil)
         (tagbody
           (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                 (+
                  (f2cl-lib:fref x-%data% (i) ((1 *))
                                 x-%offset%)
                  (* temp
                     (f2cl-lib:fref a-%data% (i j)
                                    ((1 lda) (1 *))
                                    a-%offset%))))
          label10))
       (if nounit
           (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                 (*
                  (f2cl-lib:fref x-%data% (j) ((1 *))
                                 x-%offset%)
                  (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                 a-%offset%))))))
    label20)))
(t (setf jx kx)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
      ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
       (setf temp
               (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
       (setf ix kx)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i
                         (f2cl-lib:int-add j
                                           (f2cl-lib:int-sub 1)))
                      nil)
         (tagbody
           (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                 (+
                  (f2cl-lib:fref x-%data% (ix) ((1 *))
                                 x-%offset%)
                  (* temp
```

```
                            (f2cl-lib:fref a-%data% (i j)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
                 (setf ix (f2cl-lib:int-add ix incx))
                label30))
            (if nounit
                (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                        (*
                         (f2cl-lib:fref x-%data% (jx) ((1 *))
                                        x-%offset%)
                         (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                        a-%offset%))))))
          (setf jx (f2cl-lib:int-add jx incx))
         label40)))))
  (t
   (cond
    ((= incx 1)
     (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   ((> j 1) nil)
       (tagbody
         (cond
          ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
           (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
           (f2cl-lib:fdo (i n
                          (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                         ((> i (f2cl-lib:int-add j 1)) nil)
             (tagbody
               (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                       (+
                        (f2cl-lib:fref x-%data% (i) ((1 *))
                                       x-%offset%)
                        (* temp
                           (f2cl-lib:fref a-%data% (i j)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
               label50))
            (if nounit
                (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                        (*
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%)
                         (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                        a-%offset%))))))
         label60)))
    (t
     (setf kx
             (f2cl-lib:int-add kx
                               (f2cl-lib:int-mul
                                (f2cl-lib:int-sub n 1) incx)))
     (setf jx kx)
```

```
      (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                    ((> j 1) nil)
        (tagbody
          (cond
           ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
            (setf temp
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
            (setf ix kx)
            (f2cl-lib:fdo (i n
                           (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                          ((> i (f2cl-lib:int-add j 1)) nil)
              (tagbody
                (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                        (+
                         (f2cl-lib:fref x-%data% (ix) ((1 *))
                                        x-%offset%)
                         (* temp
                            (f2cl-lib:fref a-%data% (i j)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
                (setf ix (f2cl-lib:int-sub ix incx))
               label70))
            (if nounit
                (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                        (*
                         (f2cl-lib:fref x-%data% (jx) ((1 *))
                                        x-%offset%)
                         (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                        a-%offset%))))))
          (setf jx (f2cl-lib:int-sub jx incx))
         label80)))))))
(t
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame uplo "U")
     (declare (ignore var-1))
     (when var-0 (setf uplo var-0))
     ret-val)
   (cond
    ((= incx 1)
     (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   ((> j 1) nil)
       (tagbody
         (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
         (cond
          (noconj
           (if nounit
               (setf temp
                       (* temp
                          (f2cl-lib:fref a-%data% (j j)
```

```
                                    ((1 lda) (1 *))
                                    a-%offset%))))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                       (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (setf temp
                    (+ temp
                       (*
                        (f2cl-lib:fref a-%data% (i j)
                                       ((1 lda) (1 *)) a-%offset%)
                        (f2cl-lib:fref x-%data% (i) ((1 *))
                                       x-%offset%))))
           label90)))
       (t
        (if nounit
            (setf temp
                    (* temp
                       (f2cl-lib:conjg
                        (f2cl-lib:fref a-%data% (j j)
                                       ((1 lda) (1 *))
                                       a-%offset%)))))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                       (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (setf temp
                    (+ temp
                       (*
                        (f2cl-lib:conjg
                         (f2cl-lib:fref a-%data% (i j)
                                        ((1 lda) (1 *))
                                        a-%offset%))
                        (f2cl-lib:fref x-%data% (i) ((1 *))
                                       x-%offset%))))
           label100))))
      (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
     label110)))
  (t
   (setf jx
           (f2cl-lib:int-add kx
                             (f2cl-lib:int-mul
                              (f2cl-lib:int-sub n 1) incx)))
   (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                 ((> j 1) nil)
     (tagbody
       (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
       (setf ix jx)
       (cond
        (noconj
```

```
          (if nounit
              (setf temp
                      (* temp
                         (f2cl-lib:fref a-%data% (j j)
                                        ((1 lda) (1 *))
                                        a-%offset%))))
          (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                         (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                        ((> i 1) nil)
            (tagbody
              (setf ix (f2cl-lib:int-sub ix incx))
              (setf temp
                      (+ temp
                         (*
                          (f2cl-lib:fref a-%data% (i j)
                                         ((1 lda) (1 *)) a-%offset%)
                          (f2cl-lib:fref x-%data% (ix) ((1 *))
                                         x-%offset%))))
             label120)))
         (t
          (if nounit
              (setf temp
                      (* temp
                         (f2cl-lib:conjg
                          (f2cl-lib:fref a-%data% (j j)
                                         ((1 lda) (1 *))
                                         a-%offset%)))))
          (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                         (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                        ((> i 1) nil)
            (tagbody
              (setf ix (f2cl-lib:int-sub ix incx))
              (setf temp
                      (+ temp
                         (*
                          (f2cl-lib:conjg
                           (f2cl-lib:fref a-%data% (i j)
                                          ((1 lda) (1 *))
                                          a-%offset%))
                          (f2cl-lib:fref x-%data% (ix) ((1 *))
                                         x-%offset%))))
             label130))))
        (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
        (setf jx (f2cl-lib:int-sub jx incx))
       label140)))))
(t
 (cond
  ((= incx 1)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
```

```
(tagbody
  (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
  (cond
   (noconj
    (if nounit
        (setf temp
                (* temp
                   (f2cl-lib:fref a-%data% (j j)
                                  ((1 lda) (1 *))
                                  a-%offset%))))
    (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                   (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf temp
                (+ temp
                   (*
                    (f2cl-lib:fref a-%data% (i j)
                                   ((1 lda) (1 *)) a-%offset%)
                    (f2cl-lib:fref x-%data% (i) ((1 *))
                                   x-%offset%))))
       label150)))
   (t
    (if nounit
        (setf temp
                (* temp
                   (f2cl-lib:conjg
                    (f2cl-lib:fref a-%data% (j j)
                                   ((1 lda) (1 *))
                                   a-%offset%)))))
    (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                   (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf temp
                (+ temp
                   (*
                    (f2cl-lib:conjg
                     (f2cl-lib:fref a-%data% (i j)
                                    ((1 lda) (1 *))
                                    a-%offset%))
                    (f2cl-lib:fref x-%data% (i) ((1 *))
                                   x-%offset%))))
       label160))))
  (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
 label170)))
(t (setf jx kx)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
  (tagbody
```

```
            (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
            (setf ix jx)
            (cond
             (noconj
              (if nounit
                  (setf temp
                          (* temp
                             (f2cl-lib:fref a-%data% (j j)
                                            ((1 lda) (1 *))
                                            a-%offset%))))
              (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                             (f2cl-lib:int-add i 1))
                            ((> i n) nil)
                (tagbody
                  (setf ix (f2cl-lib:int-add ix incx))
                  (setf temp
                          (+ temp
                             (*
                              (f2cl-lib:fref a-%data% (i j)
                                             ((1 lda) (1 *)) a-%offset%)
                              (f2cl-lib:fref x-%data% (ix) ((1 *))
                                             x-%offset%))))
                 label180)))
             (t
              (if nounit
                  (setf temp
                          (* temp
                             (f2cl-lib:conjg
                              (f2cl-lib:fref a-%data% (j j)
                                             ((1 lda) (1 *))
                                             a-%offset%)))))
              (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                             (f2cl-lib:int-add i 1))
                            ((> i n) nil)
                (tagbody
                  (setf ix (f2cl-lib:int-add ix incx))
                  (setf temp
                          (+ temp
                             (*
                              (f2cl-lib:conjg
                               (f2cl-lib:fref a-%data% (i j)
                                              ((1 lda) (1 *))
                                              a-%offset%))
                              (f2cl-lib:fref x-%data% (ix) ((1 *))
                                             x-%offset%))))
                 label190))))
            (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
            (setf jx (f2cl-lib:int-add jx incx))
           label200))))))))
(go end_label)
```

```
      end_label
       (return (values uplo trans diag nil nil nil nil nil))))))
```

———-

12.17 ctrsv - solve systems of equations $Ax = b$ or $A^T x = b$ or $A^H x = b$

12.17.1 Axiom unit tests

— **ctrsv.input** —

```
)set break resume
)sys rm -f ctrsv.output
)spool ctrsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

12.17.2 Axiom help page

— **ctrsv.help** —

```
======================================================================
ctrsv examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ctrsv (UPLO, TRANS, DIAG, N, A, LDA, X, INCX)
           CTRSV

Function/Subroutine Documentation
   subroutine ctrsv (character UPLO, character TRANS, character DIAG, integer
```

```
      N, complex, dimension(lda,*) A, integer LDA, complex, dimension(*) X,
      integer INCX)
      CTRSV

Purpose:
   CTRSV  solves one of the systems of equations

   A*x = b,   or   A**T*x = b,   or   A**H*x = b,

   where b and x are n element vectors and A is an n by n unit, or
   non-unit, upper or lower triangular matrix.

   No test for singularity or near-singularity is included in this
   routine. Such tests must be performed before calling this routine.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the matrix is an upper or
   lower triangular matrix as follows:

   UPLO = 'U' or 'u'   A is an upper triangular matrix.

   UPLO = 'L' or 'l'   A is a lower triangular matrix.

 TRANS
   TRANS is CHARACTER*1
   On entry, TRANS specifies the equations to be solved as follows:

   TRANS = 'N' or 'n'   A*x = b.

   TRANS = 'T' or 't'   A**T*x = b.

   TRANS = 'C' or 'c'   A**H*x = b.

 DIAG
   DIAG is CHARACTER*1
   On entry, DIAG specifies whether or not A is unit
   triangular as follows:

 DIAG = 'U' or 'u'   A is assumed to be unit triangular.

   DIAG = 'N' or 'n'   A is not assumed to be unit triangular.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
   N must be at least zero.
```

```
A
                    A is COMPLEX array of DIMENSION ( LDA, n ).

  Before entry with  UPLO = 'U' or 'u', the leading n by n
  upper triangular part of the array A must contain the upper
  triangular matrix and the strictly lower triangular part of
  A is not referenced.

  Before entry with UPLO = 'L' or 'l', the leading n by n
  lower triangular part of the array A must contain the lower
  triangular matrix and the strictly upper triangular part of
  A is not referenced.

  Note that when  DIAG = 'U' or 'u', the diagonal elements of
  A are not referenced either, but are assumed to be unity.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, n ).

X
  X is COMPLEX array of dimension at least ( 1 + ( n - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the n
  element right-hand side vector b. On exit, X is overwritten
  with the solution vector x.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of X.
  INCX must not be zero.
```

———-

12.17.3 fortran code

— **ctrsv.f** —

```
*  ===================================================================
      SUBROUTINE CTRSV(UPLO,TRANS,DIAG,N,A,LDA,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
```

```
      INTEGER INCX,LDA,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),X(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP
      INTEGER I,INFO,IX,J,JX,KX
      LOGICAL NOCONJ,NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG,MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 6
      ELSE IF (INCX.EQ.0) THEN
          INFO = 8
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CTRSV ',INFO)
          RETURN
      END IF
```

```
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOCONJ = LSAME(TRANS,'T')
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX  too small for descending loops.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x := inv( A )*x.
*
          IF (LSAME(UPLO,'U')) THEN
              IF (INCX.EQ.1) THEN
                  DO 20 J = N,1,-1
                      IF (X(J).NE.ZERO) THEN
                          IF (NOUNIT) X(J) = X(J)/A(J,J)
                          TEMP = X(J)
                          DO 10 I = J - 1,1,-1
                              X(I) = X(I) - TEMP*A(I,J)
   10                     CONTINUE
                      END IF
   20             CONTINUE
              ELSE
                  JX = KX + (N-1)*INCX
                  DO 40 J = N,1,-1
                      IF (X(JX).NE.ZERO) THEN
                          IF (NOUNIT) X(JX) = X(JX)/A(J,J)
                          TEMP = X(JX)
                          IX = JX
                          DO 30 I = J - 1,1,-1
                              IX = IX - INCX
                              X(IX) = X(IX) - TEMP*A(I,J)
   30                     CONTINUE
                      END IF
                      JX = JX - INCX
   40             CONTINUE
              END IF
```

```
         ELSE
             IF (INCX.EQ.1) THEN
                 DO 60 J = 1,N
                     IF (X(J).NE.ZERO) THEN
                         IF (NOUNIT) X(J) = X(J)/A(J,J)
                         TEMP = X(J)
                         DO 50 I = J + 1,N
                             X(I) = X(I) - TEMP*A(I,J)
   50                    CONTINUE
                     END IF
   60            CONTINUE
             ELSE
                 JX = KX
                 DO 80 J = 1,N
                     IF (X(JX).NE.ZERO) THEN
                         IF (NOUNIT) X(JX) = X(JX)/A(J,J)
                         TEMP = X(JX)
                         IX = JX
                         DO 70 I = J + 1,N
                             IX = IX + INCX
                             X(IX) = X(IX) - TEMP*A(I,J)
   70                    CONTINUE
                     END IF
                     JX = JX + INCX
   80            CONTINUE
             END IF
         END IF
     ELSE
*
*        Form  x := inv( A**T )*x  or  x := inv( A**H )*x.
*
         IF (LSAME(UPLO,'U')) THEN
             IF (INCX.EQ.1) THEN
                 DO 110 J = 1,N
                     TEMP = X(J)
                     IF (NOCONJ) THEN
                         DO 90 I = 1,J - 1
                             TEMP = TEMP - A(I,J)*X(I)
   90                    CONTINUE
                         IF (NOUNIT) TEMP = TEMP/A(J,J)
                     ELSE
                         DO 100 I = 1,J - 1
                             TEMP = TEMP - CONJG(A(I,J))*X(I)
  100                    CONTINUE
                         IF (NOUNIT) TEMP = TEMP/CONJG(A(J,J))
                     END IF
                     X(J) = TEMP
  110            CONTINUE
             ELSE
                 JX = KX
```

```
                DO 140 J = 1,N
                    IX = KX
                    TEMP = X(JX)
                    IF (NOCONJ) THEN
                        DO 120 I = 1,J - 1
                            TEMP = TEMP - A(I,J)*X(IX)
                            IX = IX + INCX
 120                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/A(J,J)
                    ELSE
                        DO 130 I = 1,J - 1
                            TEMP = TEMP - CONJG(A(I,J))*X(IX)
                            IX = IX + INCX
 130                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/CONJG(A(J,J))
                    END IF
                    X(JX) = TEMP
                    JX = JX + INCX
 140            CONTINUE
            END IF
        ELSE
            IF (INCX.EQ.1) THEN
                DO 170 J = N,1,-1
                    TEMP = X(J)
                    IF (NOCONJ) THEN
                        DO 150 I = N,J + 1,-1
                            TEMP = TEMP - A(I,J)*X(I)
 150                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/A(J,J)
                    ELSE
                        DO 160 I = N,J + 1,-1
                            TEMP = TEMP - CONJG(A(I,J))*X(I)
 160                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/CONJG(A(J,J))
                    END IF
                    X(J) = TEMP
 170            CONTINUE
            ELSE
                KX = KX + (N-1)*INCX
                JX = KX
                DO 200 J = N,1,-1
                    IX = KX
                    TEMP = X(JX)
                    IF (NOCONJ) THEN
                        DO 180 I = N,J + 1,-1
                            TEMP = TEMP - A(I,J)*X(IX)
                            IX = IX - INCX
 180                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/A(J,J)
                    ELSE
```

```
                        DO 190 I = N,J + 1,-1
                            TEMP = TEMP - CONJG(A(I,J))*X(IX)
                            IX = IX - INCX
  190                   CONTINUE
                        IF (NOUNIT) TEMP = TEMP/CONJG(A(J,J))
                    END IF
                    X(JX) = TEMP
                    JX = JX - INCX
  200             CONTINUE
            END IF
        END IF
    END IF
*
    RETURN
*
*   End of CTRSV .
*
    END
```

———-

12.17.4 lisp code

— BLAS 2 ctrsv —

```
(let* ((zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) zero)
           (ignorable zero))
  (defun ctrsv (uplo trans diag n a lda x incx)
    (declare (type (array f2cl-lib:complex8 (*)) x a)
             (type (f2cl-lib:integer4) incx lda n)
             (type (string *) diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%))
      (prog ((noconj nil) (nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0)
             (kx 0) (temp #C(0.0 0.0)))
        (declare (type f2cl-lib:logical noconj nounit)
                 (type (f2cl-lib:integer4) i info ix j jx kx)
                 (type (f2cl-lib:complex8) temp))
        (setf info 0)
        (cond
         ((and
           (not
```

```
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)))
  (setf info 1))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame trans "N")
      (declare (ignore var-1))
      (when var-0 (setf trans var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame trans "T")
      (declare (ignore var-1))
      (when var-0 (setf trans var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame trans "C")
      (declare (ignore var-1))
      (when var-0 (setf trans var-0))
      ret-val)))
  (setf info 2))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame diag "U")
      (declare (ignore var-1))
      (when var-0 (setf diag var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame diag "N")
      (declare (ignore var-1))
      (when var-0 (setf diag var-0))
      ret-val)))
  (setf info 3))
 ((< n 0) (setf info 4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info 6))
 ((= incx 0) (setf info 8)))
```

```
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "CTRSV " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (= n 0)
    (go end_label))
(setf noconj
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(setf nounit
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame diag "N")
          (declare (ignore var-1))
          (when var-0 (setf diag var-0))
          ret-val))
(cond
 ((<= incx 0)
  (setf kx
          (f2cl-lib:int-sub 1
                            (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                              incx))))
 ((/= incx 1) (setf kx 1)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame trans "N")
    (declare (ignore var-1))
    (when var-0 (setf trans var-0))
    ret-val)
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)
    (cond
     ((= incx 1)
      (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                    ((> j 1) nil)
        (tagbody
          (cond
           ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
            (if nounit
                (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                        (/
```

```
                        (f2cl-lib:fref x-%data% (j) ((1 *))
                                       x-%offset%)
                        (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                       a-%offset%))))
        (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                       (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                  (-
                   (f2cl-lib:fref x-%data% (i) ((1 *))
                                  x-%offset%)
                   (* temp
                      (f2cl-lib:fref a-%data% (i j)
                                     ((1 lda) (1 *))
                                     a-%offset%))))
           label10))))
    label20)))
(t
 (setf jx
         (f2cl-lib:int-add kx
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-sub n 1) incx)))
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (cond
      ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
       (if nounit
           (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                 (/
                  (f2cl-lib:fref x-%data% (jx) ((1 *))
                                 x-%offset%)
                  (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                 a-%offset%))))
       (setf temp
               (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
       (setf ix jx)
       (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                      (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     ((> i 1) nil)
         (tagbody
           (setf ix (f2cl-lib:int-sub ix incx))
           (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                 (-
                  (f2cl-lib:fref x-%data% (ix) ((1 *))
                                 x-%offset%)
                  (* temp
                     (f2cl-lib:fref a-%data% (i j)
```

```
                                        ((1 lda) (1 *))
                                        a-%offset%))))
            label30))))
        (setf jx (f2cl-lib:int-sub jx incx))
      label40)))))
(t
 (cond
  ((= incx 1)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
         (if nounit
             (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                     (/
                      (f2cl-lib:fref x-%data% (j) ((1 *))
                                     x-%offset%)
                      (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%))))
         (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
         (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                        (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                     (-
                      (f2cl-lib:fref x-%data% (i) ((1 *))
                                     x-%offset%)
                      (* temp
                         (f2cl-lib:fref a-%data% (i j)
                                        ((1 lda) (1 *))
                                        a-%offset%))))
            label50))))
      label60)))
  (t (setf jx kx)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
         (if nounit
             (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                     (/
                      (f2cl-lib:fref x-%data% (jx) ((1 *))
                                     x-%offset%)
                      (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%))))
         (setf temp
                 (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
```

```
                (setf ix jx)
                (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                                 (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf ix (f2cl-lib:int-add ix incx))
                    (setf (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)
                            (-
                             (f2cl-lib:fref x-%data% (ix) ((1 *))
                                            x-%offset%)
                             (* temp
                                (f2cl-lib:fref a-%data% (i j)
                                               ((1 lda) (1 *))
                                               a-%offset%))))
                     label70))))
                 (setf jx (f2cl-lib:int-add jx incx))
                label80)))))))
        (t
         (cond
          ((multiple-value-bind (ret-val var-0 var-1)
               (lsame uplo "U")
             (declare (ignore var-1))
             (when var-0 (setf uplo var-0))
             ret-val)
           (cond
            ((= incx 1)
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                 (cond
                  (noconj
                   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                 ((> i
                                      (f2cl-lib:int-add j
                                                        (f2cl-lib:int-sub 1)))
                                  nil)
                     (tagbody
                       (setf temp
                               (- temp
                                  (*
                                   (f2cl-lib:fref a-%data% (i j)
                                                  ((1 lda) (1 *)) a-%offset%)
                                   (f2cl-lib:fref x-%data% (i) ((1 *))
                                                  x-%offset%))))
                      label90))
                   (if nounit
                       (setf temp
                               (/ temp
                                  (f2cl-lib:fref a-%data% (j j)
```

```
                                        ((1 lda) (1 *))
                                        a-%offset%)))))
            (t
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i
                               (f2cl-lib:int-add j
                                                 (f2cl-lib:int-sub 1)))
                            nil)
               (tagbody
                 (setf temp
                         (- temp
                            (*
                             (f2cl-lib:conjg
                              (f2cl-lib:fref a-%data% (i j)
                                             ((1 lda) (1 *))
                                             a-%offset%))
                             (f2cl-lib:fref x-%data% (i) ((1 *))
                                            x-%offset%))))
                label100))
             (if nounit
                 (setf temp
                         (/ temp
                            (f2cl-lib:conjg
                             (f2cl-lib:fref a-%data% (j j)
                                            ((1 lda) (1 *))
                                            a-%offset%)))))))
          (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
         label110)))
      (t (setf jx kx)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (setf ix kx)
           (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
           (cond
            (noconj
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i
                               (f2cl-lib:int-add j
                                                 (f2cl-lib:int-sub 1)))
                            nil)
               (tagbody
                 (setf temp
                         (- temp
                            (*
                             (f2cl-lib:fref a-%data% (i j)
                                            ((1 lda) (1 *)) a-%offset%)
                             (f2cl-lib:fref x-%data% (ix) ((1 *))
                                            x-%offset%))))
                 (setf ix (f2cl-lib:int-add ix incx))
```

```
                 label120))
              (if nounit
                  (setf temp
                          (/ temp
                             (f2cl-lib:fref a-%data% (j j)
                                            ((1 lda) (1 *))
                                            a-%offset%)))))
             (t
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i
                                (f2cl-lib:int-add j
                                                  (f2cl-lib:int-sub 1)))
                             nil)
                (tagbody
                  (setf temp
                          (- temp
                             (*
                              (f2cl-lib:conjg
                               (f2cl-lib:fref a-%data% (i j)
                                              ((1 lda) (1 *))
                                              a-%offset%))
                              (f2cl-lib:fref x-%data% (ix) ((1 *))
                                             x-%offset%))))
                  (setf ix (f2cl-lib:int-add ix incx))
                 label130))
              (if nounit
                  (setf temp
                          (/ temp
                             (f2cl-lib:conjg
                              (f2cl-lib:fref a-%data% (j j)
                                             ((1 lda) (1 *))
                                             a-%offset%)))))))
              (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
              (setf jx (f2cl-lib:int-add jx incx))
             label140)))))
        (t
         (cond
          ((= incx 1)
           (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                         ((> j 1) nil)
             (tagbody
               (setf temp (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
               (cond
                (noconj
                 (f2cl-lib:fdo (i n
                                (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                               ((> i (f2cl-lib:int-add j 1)) nil)
                   (tagbody
                     (setf temp
                             (- temp
```

```
                          (*
                           (f2cl-lib:fref a-%data% (i j)
                                          ((1 lda) (1 *)) a-%offset%)
                           (f2cl-lib:fref x-%data% (i) ((1 *))
                                          x-%offset%))))
            label150))
        (if nounit
            (setf temp
                    (/ temp
                       (f2cl-lib:fref a-%data% (j j)
                                      ((1 lda) (1 *))
                                      a-%offset%)))))
       (t
        (f2cl-lib:fdo (i n
                       (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i (f2cl-lib:int-add j 1)) nil)
          (tagbody
            (setf temp
                    (- temp
                       (*
                        (f2cl-lib:conjg
                         (f2cl-lib:fref a-%data% (i j)
                                        ((1 lda) (1 *))
                                        a-%offset%))
                        (f2cl-lib:fref x-%data% (i) ((1 *))
                                       x-%offset%))))
            label160))
        (if nounit
            (setf temp
                    (/ temp
                       (f2cl-lib:conjg
                        (f2cl-lib:fref a-%data% (j j)
                                       ((1 lda) (1 *))
                                       a-%offset%)))))))
      (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) temp)
     label170)))
(t
 (setf kx
         (f2cl-lib:int-add kx
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-sub n 1) incx)))
 (setf jx kx)
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (setf ix kx)
     (setf temp (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
     (cond
      (noconj
       (f2cl-lib:fdo (i n
```

```
                              (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                             ((> i (f2cl-lib:int-add j 1)) nil)
                 (tagbody
                   (setf temp
                           (- temp
                              (*
                               (f2cl-lib:fref a-%data% (i j)
                                              ((1 lda) (1 *)) a-%offset%)
                               (f2cl-lib:fref x-%data% (ix) ((1 *))
                                              x-%offset%))))
                   (setf ix (f2cl-lib:int-sub ix incx))
                  label180))
                (if nounit
                    (setf temp
                            (/ temp
                               (f2cl-lib:fref a-%data% (j j)
                                             ((1 lda) (1 *))
                                             a-%offset%)))))
               (t
                (f2cl-lib:fdo (i n
                              (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                             ((> i (f2cl-lib:int-add j 1)) nil)
                 (tagbody
                   (setf temp
                           (- temp
                              (*
                               (f2cl-lib:conjg
                                (f2cl-lib:fref a-%data% (i j)
                                               ((1 lda) (1 *))
                                               a-%offset%))
                               (f2cl-lib:fref x-%data% (ix) ((1 *))
                                              x-%offset%))))
                   (setf ix (f2cl-lib:int-sub ix incx))
                  label190))
                (if nounit
                    (setf temp
                            (/ temp
                               (f2cl-lib:conjg
                                (f2cl-lib:fref a-%data% (j j)
                                              ((1 lda) (1 *))
                                              a-%offset%)))))))
                (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%) temp)
                (setf jx (f2cl-lib:int-sub jx incx))
               label200))))))))
  (go end_label)
 end_label
  (return (values uplo trans diag nil nil nil nil nil))))))
```

———-

Chapter 13

BLAS Level 2 Complex16 Matrix-Vector Operations

13.1 zgbmv - banded $\alpha Ax+\beta y$ or $\alpha A^T x+\beta y$ or $\alpha A^H x+\beta y$

13.1.1 Axiom unit tests

— **zgbmv.input** —

```
)set break resume
)sys rm -f zgbmv.output
)spool zgbmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

13.1.2 Axiom help page

— **zgbmv.help** —

```
======================================================================
zgbmv examples
======================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

 NAME
      ZGBMV - perform one of the matrix-vector operations   y :=
      alpha*A*x + beta*y, or y := alpha*A'*x + beta*y, or   y :=
      alpha*conjg( A' )*x + beta*y,

 SYNOPSIS
      SUBROUTINE ZGBMV ( TRANS, M, N, KL, KU, ALPHA, A, LDA, X,
                         INCX, BETA, Y, INCY )

          COMPLEX*16   ALPHA, BETA

          INTEGER      INCX, INCY, KL, KU, LDA, M, N

          CHARACTER*1  TRANS

          COMPLEX*16   A( LDA, * ), X( * ), Y( * )

 PURPOSE
      ZGBMV  performs one of the matrix-vector operations

      where alpha and beta are scalars, x and y are vectors and A
      is an m by n band matrix, with kl sub-diagonals and ku
      super-diagonals.

 PARAMETERS
      TRANS  - CHARACTER*1.
             On entry, TRANS specifies the operation to be per-
             formed as follows:

             TRANS = 'N' or 'n'   y := alpha*A*x + beta*y.

             TRANS = 'T' or 't'   y := alpha*A'*x + beta*y.

             TRANS = 'C' or 'c'   y := alpha*conjg( A' )*x +
             beta*y.

             Unchanged on exit.

      M      - INTEGER.
             On entry, M specifies the number of rows of the
             matrix A.  M must be at least zero.  Unchanged on
             exit.

      N      - INTEGER.
             On entry, N specifies the number of columns of the
```

```
        matrix A.  N must be at least zero.  Unchanged on
        exit.

KL      - INTEGER.
        On entry, KL specifies the number of sub-diagonals of
        the matrix A. KL must satisfy  0 .le. KL.  Unchanged

        on exit.

KU      - INTEGER.
        On entry, KU specifies the number of super-diagonals
        of the matrix A. KU must satisfy  0 .le. KU.
        Unchanged on exit.

ALPHA   - COMPLEX*16      .
        On entry, ALPHA specifies the scalar alpha.
        Unchanged on exit.

A       - COMPLEX*16       array of DIMENSION ( LDA, n ).
        Before entry, the leading ( kl + ku + 1 ) by n part
        of the array A must contain the matrix of coeffi-
        cients, supplied column by column, with the leading
        diagonal of the matrix in row ( ku + 1 ) of the
        array, the first super-diagonal starting at position
        2 in row ku, the first sub-diagonal starting at posi-
        tion 1 in row ( ku + 2 ), and so on.  Elements in the
        array A that do not correspond to elements in the
        band matrix (such as the top left ku by ku triangle)
        are not referenced.  The following program segment
        will transfer a band matrix from conventional full
        matrix storage to band storage:

        DO 20, J = 1, N K = KU + 1 - J DO 10, I = MAX( 1, J -
        KU ), MIN( M, J + KL ) A( K + I, J ) = matrix( I, J )
        10    CONTINUE 20 CONTINUE

        Unchanged on exit.

LDA     - INTEGER.
        On entry, LDA specifies the first dimension of A as
        declared in the calling (sub) program. LDA must be at
        least ( kl + ku + 1 ).  Unchanged on exit.

X       - COMPLEX*16       array of DIMENSION at least
        ( 1 + ( n - 1 )*abs( INCX ) ) when TRANS = 'N' or 'n'
        and at least ( 1 + ( m - 1 )*abs( INCX ) ) otherwise.
        Before entry, the incremented array X must contain
        the vector x.  Unchanged on exit.

INCX    - INTEGER.
```

```
             On entry, INCX specifies the increment for the ele-
             ments of X. INCX must not be zero.  Unchanged on
             exit.

      BETA   - COMPLEX*16      .
             On entry, BETA specifies the scalar beta. When BETA
             is supplied as zero then Y need not be set on input.
             Unchanged on exit.

      Y      - COMPLEX*16       array of DIMENSION at least
             ( 1 + ( m - 1 )*abs( INCY ) ) when TRANS = 'N' or 'n'
             and at least ( 1 + ( n - 1 )*abs( INCY ) ) otherwise.
             Before entry, the incremented array Y must contain
             the vector y. On exit, Y is overwritten by the
             updated vector y.

      INCY   - INTEGER.
             On entry, INCY specifies the increment for the ele-
             ments of Y. INCY must not be zero.  Unchanged on
             exit.
```

———-

13.1.3 fortran code

— zgbmv.f —

```
*  ====================================================================
      SUBROUTINE ZGBMV(TRANS,M,N,KL,KU,ALPHA,A,LDA,X,INCX,BETA,Y,INCY)
*
*  -- Reference BLAS level2 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      COMPLEX*16 ALPHA,BETA
      INTEGER INCX,INCY,KL,KU,LDA,M,N
      CHARACTER TRANS
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),X(*),Y(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
```

```
      COMPLEX*16 ONE
      PARAMETER (ONE= (1.0D+0,0.0D+0))
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP
      INTEGER I,INFO,IX,IY,J,JX,JY,K,KUP1,KX,KY,LENX,LENY
      LOGICAL NOCONJ
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DCONJG,MAX,MIN
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +    .NOT.LSAME(TRANS,'C')) THEN
          INFO = 1
      ELSE IF (M.LT.0) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
          INFO = 3
      ELSE IF (KL.LT.0) THEN
          INFO = 4
      ELSE IF (KU.LT.0) THEN
          INFO = 5
      ELSE IF (LDA.LT. (KL+KU+1)) THEN
          INFO = 8
      ELSE IF (INCX.EQ.0) THEN
          INFO = 10
      ELSE IF (INCY.EQ.0) THEN
          INFO = 13
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZGBMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR.
```

```
     +    ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
      NOCONJ = LSAME(TRANS,'T')
*
*     Set  LENX  and  LENY, the lengths of the vectors x and y, and set
*     up the start points in  X  and  Y.
*
      IF (LSAME(TRANS,'N')) THEN
          LENX = N
          LENY = M
      ELSE
          LENX = M
          LENY = N
      END IF
      IF (INCX.GT.0) THEN
          KX = 1
      ELSE
          KX = 1 - (LENX-1)*INCX
      END IF
      IF (INCY.GT.0) THEN
          KY = 1
      ELSE
          KY = 1 - (LENY-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the band part of A.
*
*     First form  y := beta*y.
*
      IF (BETA.NE.ONE) THEN
          IF (INCY.EQ.1) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 10 I = 1,LENY
                      Y(I) = ZERO
   10             CONTINUE
              ELSE
                  DO 20 I = 1,LENY
                      Y(I) = BETA*Y(I)
   20             CONTINUE
              END IF
          ELSE
              IY = KY
              IF (BETA.EQ.ZERO) THEN
                  DO 30 I = 1,LENY
                      Y(IY) = ZERO
                      IY = IY + INCY
   30             CONTINUE
              ELSE
                  DO 40 I = 1,LENY
```

```
                    Y(IY) = BETA*Y(IY)
                    IY = IY + INCY
   40            CONTINUE
            END IF
         END IF
      END IF
      IF (ALPHA.EQ.ZERO) RETURN
      KUP1 = KU + 1
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  y := alpha*A*x + y.
*
         JX = KX
         IF (INCY.EQ.1) THEN
            DO 60 J = 1,N
               TEMP = ALPHA*X(JX)
               K = KUP1 - J
               DO 50 I = MAX(1,J-KU),MIN(M,J+KL)
                  Y(I) = Y(I) + TEMP*A(K+I,J)
   50          CONTINUE
               JX = JX + INCX
   60       CONTINUE
         ELSE
            DO 80 J = 1,N
               TEMP = ALPHA*X(JX)
               IY = KY
               K = KUP1 - J
               DO 70 I = MAX(1,J-KU),MIN(M,J+KL)
                  Y(IY) = Y(IY) + TEMP*A(K+I,J)
                  IY = IY + INCY
   70          CONTINUE
               JX = JX + INCX
               IF (J.GT.KU) KY = KY + INCY
   80       CONTINUE
         END IF
      ELSE
*
*        Form  y := alpha*A**T*x + y  or  y := alpha*A**H*x + y.
*
         JY = KY
         IF (INCX.EQ.1) THEN
            DO 110 J = 1,N
               TEMP = ZERO
               K = KUP1 - J
               IF (NOCONJ) THEN
                  DO 90 I = MAX(1,J-KU),MIN(M,J+KL)
                     TEMP = TEMP + A(K+I,J)*X(I)
   90             CONTINUE
               ELSE
                  DO 100 I = MAX(1,J-KU),MIN(M,J+KL)
```

```
                        TEMP = TEMP + DCONJG(A(K+I,J))*X(I)
  100                 CONTINUE
                  END IF
                  Y(JY) = Y(JY) + ALPHA*TEMP
                  JY = JY + INCY
  110         CONTINUE
          ELSE
              DO 140 J = 1,N
                  TEMP = ZERO
                  IX = KX
                  K = KUP1 - J
                  IF (NOCONJ) THEN
                      DO 120 I = MAX(1,J-KU),MIN(M,J+KL)
                          TEMP = TEMP + A(K+I,J)*X(IX)
                          IX = IX + INCX
  120                 CONTINUE
                  ELSE
                      DO 130 I = MAX(1,J-KU),MIN(M,J+KL)
                          TEMP = TEMP + DCONJG(A(K+I,J))*X(IX)
                          IX = IX + INCX
  130                 CONTINUE
                  END IF
                  Y(JY) = Y(JY) + ALPHA*TEMP
                  JY = JY + INCY
                  IF (J.GT.KU) KX = KX + INCX
  140         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of ZGBMV .
*
      END
```

———-

13.1.4 lisp code

— BLAS 2 zgbmv —

```
(let* ((one (complex 1.0 0.0)) (zero (complex 0.0 0.0)))
 (declare (type (complex double-float) one) (type (complex double-float) zero))
  (defun zgbmv (trans m n kl ku alpha a lda x incx beta y incy)
    (declare (type (simple-array (complex double-float) (*)) y x a)
             (type (complex double-float) beta alpha)
             (type fixnum incy incx lda ku kl n m)
```

```
          (type character trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (a (complex double-float) a-%data% a-%offset%)
       (x (complex double-float) x-%data% x-%offset%)
       (y (complex double-float) y-%data% y-%offset%))
    (prog ((noconj nil) (i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0)
           (k 0) (kup1 0) (kx 0) (ky 0) (lenx 0) (leny 0) (temp #C(0.0 0.0)))
      (declare (type (member t nil) noconj)
               (type fixnum i info ix iy j jx jy k kup1 kx ky
                                          lenx leny)
               (type (complex double-float) temp))
      (setf info 0)
      (cond
        ((and (not (char-equal trans #\N))
              (not (char-equal trans #\T))
              (not (char-equal trans #\C)))
         (setf info 1))
        ((< m 0)
         (setf info 2))
        ((< n 0)
         (setf info 3))
        ((< kl 0)
         (setf info 4))
        ((< ku 0)
         (setf info 5))
        ((< lda (f2cl-lib:int-add kl ku 1))
         (setf info 8))
        ((= incx 0)
         (setf info 10))
        ((= incy 0)
         (setf info 13)))
      (cond
        ((/= info 0)
         (error
          " ** On entry to ~a parameter number ~a had an illegal value~%"
          "ZGBMV" info)
         (go end_label)))
      (if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
          (go end_label))
      (setf noconj (char-equal trans #\T))
      (cond
        ((char-equal trans #\N)
         (setf lenx n)
         (setf leny m))
        (t
         (setf lenx m)
         (setf leny n)))
      (cond
        ((> incx 0)
```

```
    (setf kx 1))
   (t
    (setf kx
            (f2cl-lib:int-sub 1
                              (f2cl-lib:int-mul
                               (f2cl-lib:int-sub lenx 1)
                               incx)))))
(cond
  ((> incy 0)
   (setf ky 1))
  (t
   (setf ky
            (f2cl-lib:int-sub 1
                              (f2cl-lib:int-mul
                               (f2cl-lib:int-sub leny 1)
                               incy)))))
(cond
  ((/= beta one)
   (cond
     ((= incy 1)
      (cond
        ((= beta zero)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i leny) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     zero))))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i leny) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     (* beta
                        (f2cl-lib:fref y-%data%
                                       (i)
                                       ((1 *))
                                       y-%offset%))))))))
     (t
      (setf iy ky)
      (cond
        ((= beta zero)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i leny) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     zero)
             (setf iy (f2cl-lib:int-add iy incy)))))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i leny) nil)
```

```
            (tagbody
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (* beta
                         (f2cl-lib:fref y-%data%
                                        (iy)
                                        ((1 *))
                                        y-%offset%)))
              (setf iy (f2cl-lib:int-add iy incy))))))))))
(if (= alpha zero) (go end_label))
(setf kup1 (f2cl-lib:int-add ku 1))
(cond
  ((char-equal trans #\N)
   (setf jx kx)
   (cond
     ((= incy 1)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
            ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
             (setf temp
                     (* alpha
                        (f2cl-lib:fref x-%data%
                                       (jx)
                                       ((1 *))
                                       x-%offset%)))
             (setf k (f2cl-lib:int-sub kup1 j))
             (f2cl-lib:fdo (i
                            (max (the fixnum 1)
                                 (the fixnum
                                      (f2cl-lib:int-add j
                                                        (f2cl-lib:int-sub
                                                         ku))))
                            (f2cl-lib:int-add i 1))
                           ((> i
                               (min (the fixnum m)
                                    (the fixnum
                                         (f2cl-lib:int-add j kl))))
                            nil)
               (tagbody
                 (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                         (+
                          (f2cl-lib:fref y-%data%
                                         (i)
                                         ((1 *))
                                         y-%offset%)
                          (* temp
                             (f2cl-lib:fref a-%data%
                                            ((f2cl-lib:int-add k i) j)
                                            ((1 lda) (1 *))
```

```
                                          a-%offset%)))))))))
            (setf jx (f2cl-lib:int-add jx incx)))))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (cond
               ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
                (setf temp
                        (* alpha
                           (f2cl-lib:fref x-%data%
                                          (jx)
                                          ((1 *))
                                          x-%offset%)))
                (setf iy ky)
                (setf k (f2cl-lib:int-sub kup1 j))
                (f2cl-lib:fdo (i
                               (max (the fixnum 1)
                                    (the fixnum
                                         (f2cl-lib:int-add j
                                                           (f2cl-lib:int-sub
                                                            ku))))
                               (f2cl-lib:int-add i 1))
                              ((> i
                                  (min (the fixnum m)
                                       (the fixnum
                                            (f2cl-lib:int-add j kl))))
                               nil)
                  (tagbody
                    (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                            (+
                             (f2cl-lib:fref y-%data%
                                            (iy)
                                            ((1 *))
                                            y-%offset%)
                             (* temp
                                (f2cl-lib:fref a-%data%
                                               ((f2cl-lib:int-add k i) j)
                                               ((1 lda) (1 *))
                                               a-%offset%))))
                    (setf iy (f2cl-lib:int-add iy incy))))))
            (setf jx (f2cl-lib:int-add jx incx))
            (if (> j ku) (setf ky (f2cl-lib:int-add ky incy)))))))
    (t
     (setf jy ky)
     (cond
       ((= incx 1)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
```

```
(setf temp zero)
(setf k (f2cl-lib:int-sub kup1 j))
(cond
  (noconj
   (f2cl-lib:fdo (i
                  (max (the fixnum 1)
                       (the fixnum
                            (f2cl-lib:int-add j
                                              (f2cl-lib:int-sub
                                               ku))))
                  (f2cl-lib:int-add i 1))
                 ((> i
                     (min (the fixnum m)
                          (the fixnum
                               (f2cl-lib:int-add j kl))))
                  nil)
     (tagbody
       (setf temp
               (+ temp
                  (*
                   (f2cl-lib:fref a-%data%
                                  ((f2cl-lib:int-add k i) j)
                                  ((1 lda) (1 *))
                                  a-%offset%)
                   (f2cl-lib:fref x-%data%
                                  (i)
                                  ((1 *))
                                  x-%offset%)))))))
  (t
   (f2cl-lib:fdo (i
                  (max (the fixnum 1)
                       (the fixnum
                            (f2cl-lib:int-add j
                                              (f2cl-lib:int-sub
                                               ku))))
                  (f2cl-lib:int-add i 1))
                 ((> i
                     (min (the fixnum m)
                          (the fixnum
                               (f2cl-lib:int-add j kl))))
                  nil)
     (tagbody
       (setf temp
               (+ temp
                  (*
                   (f2cl-lib:dconjg
                    (f2cl-lib:fref a-%data%
                                   ((f2cl-lib:int-add k i) j)
                                   ((1 lda) (1 *))
                                   a-%offset%))
```

```
                              (f2cl-lib:fref x-%data%
                                             (i)
                                             ((1 *))
                                             x-%offset%))))))))
      (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
              (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                 (* alpha temp)))
      (setf jy (f2cl-lib:int-add jy incy)))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf temp zero)
     (setf ix kx)
     (setf k (f2cl-lib:int-sub kup1 j))
     (cond
       (noconj
        (f2cl-lib:fdo (i
                       (max (the fixnum 1)
                            (the fixnum
                                 (f2cl-lib:int-add j
                                                   (f2cl-lib:int-sub
                                                    ku))))
                       (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the fixnum m)
                               (the fixnum
                                    (f2cl-lib:int-add j kl))))
                       nil)
          (tagbody
            (setf temp
                    (+ temp
                       (*
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add k i) j)
                                       ((1 lda) (1 *))
                                       a-%offset%)
                        (f2cl-lib:fref x-%data%
                                       (ix)
                                       ((1 *))
                                       x-%offset%))))
            (setf ix (f2cl-lib:int-add ix incx)))))
       (t
        (f2cl-lib:fdo (i
                       (max (the fixnum 1)
                            (the fixnum
                                 (f2cl-lib:int-add j
                                                   (f2cl-lib:int-sub
                                                    ku))))
                       (f2cl-lib:int-add i 1))
```

```
                                  ((> i
                                      (min (the fixnum m)
                                           (the fixnum
                                                (f2cl-lib:int-add j kl))))
                                   nil)
                         (tagbody
                           (setf temp
                                   (+ temp
                                      (*
                                       (f2cl-lib:dconjg
                                        (f2cl-lib:fref a-%data%
                                                       ((f2cl-lib:int-add k i) j)
                                                       ((1 lda) (1 *))
                                                       a-%offset%))
                                       (f2cl-lib:fref x-%data%
                                                      (ix)
                                                      ((1 *))
                                                      x-%offset%))))
                           (setf ix (f2cl-lib:int-add ix incx))))))
                   (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                           (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                              (* alpha temp)))
                   (setf jy (f2cl-lib:int-add jy incy))
                   (if (> j ku) (setf kx (f2cl-lib:int-add kx incx)))))))))
 end_label
        (return
         (values nil nil nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

13.2 zgemv - matrix-vector $\alpha Ax + \beta y$ or $\alpha A^T x + \beta y$ or $\alpha A^H x + \beta y$

13.2.1 Axiom unit tests

— **zgemv.input** —

```
)set break resume
)sys rm -f zgemv.output
)spool zgemv.output
)set message test on
)set message auto off
)clear all

)spool
```

```
)lisp (bye)
```

———-

13.2.2 Axiom help page

— **zgemv.help** —

```
==========================================================================
zgemv examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

 NAME
      ZGEMV - perform one of the matrix-vector operations   y :=
      alpha*A*x + beta*y, or y := alpha*A'*x + beta*y, or   y :=
      alpha*conjg( A' )*x + beta*y,

 SYNOPSIS
      SUBROUTINE ZGEMV ( TRANS, M, N, ALPHA, A, LDA, X, INCX,
                         BETA, Y, INCY )

          COMPLEX*16   ALPHA, BETA

          INTEGER      INCX, INCY, LDA, M, N

          CHARACTER*1  TRANS

          COMPLEX*16   A( LDA, * ), X( * ), Y( * )

 PURPOSE
      ZGEMV  performs one of the matrix-vector operations

      where alpha and beta are scalars, x and y are vectors and A
      is an m by n matrix.

 PARAMETERS
      TRANS  - CHARACTER*1.
             On entry, TRANS specifies the operation to be per-
             formed as follows:

             TRANS = 'N' or 'n'   y := alpha*A*x + beta*y.

             TRANS = 'T' or 't'   y := alpha*A'*x + beta*y.
```

```
        TRANS = 'C' or 'c'   y := alpha*conjg( A' )*x +
        beta*y.

        Unchanged on exit.

M       - INTEGER.
        On entry, M specifies the number of rows of the
        matrix A.  M must be at least zero.  Unchanged on
        exit.

N       - INTEGER.
        On entry, N specifies the number of columns of the
        matrix A.  N must be at least zero.  Unchanged on
        exit.

ALPHA   - COMPLEX*16      .
        On entry, ALPHA specifies the scalar alpha.
        Unchanged on exit.

A       - COMPLEX*16       array of DIMENSION ( LDA, n ).
        Before entry, the leading m by n part of the array A
        must contain the matrix of coefficients.  Unchanged
        on exit.

LDA     - INTEGER.
        On entry, LDA specifies the first dimension of A as
        declared in the calling (sub) program. LDA must be at
        least max( 1, m ).  Unchanged on exit.

X       - COMPLEX*16       array of DIMENSION at least
        ( 1 + ( n - 1 )*abs( INCX ) ) when TRANS = 'N' or 'n'
        and at least ( 1 + ( m - 1 )*abs( INCX ) ) otherwise.
        Before entry, the incremented array X must contain
        the vector x.  Unchanged on exit.

INCX    - INTEGER.
        On entry, INCX specifies the increment for the ele-
        ments of X. INCX must not be zero.  Unchanged on
        exit.

BETA    - COMPLEX*16      .
        On entry, BETA specifies the scalar beta. When BETA
        is supplied as zero then Y need not be set on input.
        Unchanged on exit.

Y       - COMPLEX*16       array of DIMENSION at least
        ( 1 + ( m - 1 )*abs( INCY ) ) when TRANS = 'N' or 'n'
        and at least ( 1 + ( n - 1 )*abs( INCY ) ) otherwise.
        Before entry with BETA non-zero, the incremented
        array Y must contain the vector y. On exit, Y is
```

```
             overwritten by the updated vector y.

      INCY   - INTEGER.
             On entry, INCY specifies the increment for the ele-
             ments of Y. INCY must not be zero.  Unchanged on
             exit.


             ———-
```

13.2.3 fortran code

— zgemv.f —

```
*  ===================================================================
      SUBROUTINE ZGEMV(TRANS,M,N,ALPHA,A,LDA,X,INCX,BETA,Y,INCY)
*
*  -- Reference BLAS level2 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      COMPLEX*16 ALPHA,BETA
      INTEGER INCX,INCY,LDA,M,N
      CHARACTER TRANS
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),X(*),Y(*)
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      COMPLEX*16 ONE
      PARAMETER (ONE= (1.0D+0,0.0D+0))
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP
      INTEGER I,INFO,IX,IY,J,JX,JY,KX,KY,LENX,LENY
      LOGICAL NOCONJ
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
```

```
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DCONJG,MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +    .NOT.LSAME(TRANS,'C')) THEN
          INFO = 1
      ELSE IF (M.LT.0) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
          INFO = 3
      ELSE IF (LDA.LT.MAX(1,M)) THEN
          INFO = 6
      ELSE IF (INCX.EQ.0) THEN
          INFO = 8
      ELSE IF (INCY.EQ.0) THEN
          INFO = 11
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZGEMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR.
     +    ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
      NOCONJ = LSAME(TRANS,'T')
*
*     Set  LENX  and  LENY, the lengths of the vectors x and y, and set
*     up the start points in  X  and  Y.
*
      IF (LSAME(TRANS,'N')) THEN
          LENX = N
          LENY = M
      ELSE
          LENX = M
          LENY = N
      END IF
      IF (INCX.GT.0) THEN
          KX = 1
      ELSE
          KX = 1 - (LENX-1)*INCX
```

```
      END IF
      IF (INCY.GT.0) THEN
          KY = 1
      ELSE
          KY = 1 - (LENY-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
*     First form  y := beta*y.
*
      IF (BETA.NE.ONE) THEN
          IF (INCY.EQ.1) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 10 I = 1,LENY
                      Y(I) = ZERO
   10             CONTINUE
              ELSE
                  DO 20 I = 1,LENY
                      Y(I) = BETA*Y(I)
   20             CONTINUE
              END IF
          ELSE
              IY = KY
              IF (BETA.EQ.ZERO) THEN
                  DO 30 I = 1,LENY
                      Y(IY) = ZERO
                      IY = IY + INCY
   30             CONTINUE
              ELSE
                  DO 40 I = 1,LENY
                      Y(IY) = BETA*Y(IY)
                      IY = IY + INCY
   40             CONTINUE
              END IF
          END IF
      END IF
      IF (ALPHA.EQ.ZERO) RETURN
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  y := alpha*A*x + y.
*
          JX = KX
          IF (INCY.EQ.1) THEN
              DO 60 J = 1,N
                  TEMP = ALPHA*X(JX)
                  DO 50 I = 1,M
                      Y(I) = Y(I) + TEMP*A(I,J)
   50             CONTINUE
```

```
                JX = JX + INCX
   60         CONTINUE
          ELSE
              DO 80 J = 1,N
                  TEMP = ALPHA*X(JX)
                  IY = KY
                  DO 70 I = 1,M
                      Y(IY) = Y(IY) + TEMP*A(I,J)
                      IY = IY + INCY
   70             CONTINUE
                  JX = JX + INCX
   80         CONTINUE
          END IF
      ELSE
*
*        Form  y := alpha*A**T*x + y  or  y := alpha*A**H*x + y.
*
          JY = KY
          IF (INCX.EQ.1) THEN
              DO 110 J = 1,N
                  TEMP = ZERO
                  IF (NOCONJ) THEN
                      DO 90 I = 1,M
                          TEMP = TEMP + A(I,J)*X(I)
   90                 CONTINUE
                  ELSE
                      DO 100 I = 1,M
                          TEMP = TEMP + DCONJG(A(I,J))*X(I)
  100                 CONTINUE
                  END IF
                  Y(JY) = Y(JY) + ALPHA*TEMP
                  JY = JY + INCY
  110         CONTINUE
          ELSE
              DO 140 J = 1,N
                  TEMP = ZERO
                  IX = KX
                  IF (NOCONJ) THEN
                      DO 120 I = 1,M
                          TEMP = TEMP + A(I,J)*X(IX)
                          IX = IX + INCX
  120                 CONTINUE
                  ELSE
                      DO 130 I = 1,M
                          TEMP = TEMP + DCONJG(A(I,J))*X(IX)
                          IX = IX + INCX
  130                 CONTINUE
                  END IF
                  Y(JY) = Y(JY) + ALPHA*TEMP
                  JY = JY + INCY
```

```
  140         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of ZGEMV .
*
      END
```

———-

13.2.4 lisp code

— BLAS 2 zgemv —

```
(let* ((one (complex 1.0 0.0)) (zero (complex 0.0 0.0)))
 (declare (type (complex double-float) one) (type (complex double-float) zero))
  (defun zgemv (trans m n alpha a lda x incx beta y incy)
    (declare (type (simple-array (complex double-float) (*)) y x a)
             (type (complex double-float) beta alpha)
             (type fixnum incy incx lda n m)
             (type character trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (a (complex double-float) a-%data% a-%offset%)
         (x (complex double-float) x-%data% x-%offset%)
         (y (complex double-float) y-%data% y-%offset%))
      (prog ((noconj nil) (i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0)
             (kx 0) (ky 0) (lenx 0) (leny 0) (temp #C(0.0 0.0)))
        (declare (type (member t nil) noconj)
                 (type fixnum i info ix iy j jx jy kx ky lenx
                                          leny)
                 (type (complex double-float) temp))
        (setf info 0)
        (cond
          ((and (not (char-equal trans #\N))
                (not (char-equal trans #\T))
                (not (char-equal trans #\C)))
           (setf info 1))
          ((< m 0)
           (setf info 2))
          ((< n 0)
           (setf info 3))
          ((< lda (max (the fixnum 1) (the fixnum m)))
           (setf info 6))
          ((= incx 0)
```

```
    (setf info 8))
   ((= incy 0)
    (setf info 11)))
(cond
  ((/= info 0)
   (error
    " ** On entry to ~a parameter number ~a had an illegal value~%"
    "ZGEMV" info)
   (go end_label)))
(if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
    (go end_label))
(setf noconj (char-equal trans #\T))
(cond
  ((char-equal trans #\N)
   (setf lenx n)
   (setf leny m))
  (t
   (setf lenx m)
   (setf leny n)))
(cond
  ((> incx 0)
   (setf kx 1))
  (t
   (setf kx
           (f2cl-lib:int-sub 1
                             (f2cl-lib:int-mul
                              (f2cl-lib:int-sub lenx 1)
                              incx)))))
(cond
  ((> incy 0)
   (setf ky 1))
  (t
   (setf ky
           (f2cl-lib:int-sub 1
                             (f2cl-lib:int-mul
                              (f2cl-lib:int-sub leny 1)
                              incy)))))
(cond
  ((/= beta one)
   (cond
     ((= incy 1)
      (cond
        ((= beta zero)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i leny) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     zero))))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                        ((> i leny) nil)
                (tagbody
                  (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                          (* beta
                             (f2cl-lib:fref y-%data%
                                            (i)
                                            ((1 *))
                                            y-%offset%))))))))
        (t
         (setf iy ky)
         (cond
           ((= beta zero)
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i leny) nil)
              (tagbody
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        zero)
                (setf iy (f2cl-lib:int-add iy incy)))))
           (t
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i leny) nil)
              (tagbody
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (* beta
                           (f2cl-lib:fref y-%data%
                                          (iy)
                                          ((1 *))
                                          y-%offset%)))
                (setf iy (f2cl-lib:int-add iy incy))))))))))
(if (= alpha zero) (go end_label))
(cond
  ((char-equal trans #\N)
   (setf jx kx)
   (cond
     ((= incy 1)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
            ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
             (setf temp
                     (* alpha
                        (f2cl-lib:fref x-%data%
                                       (jx)
                                       ((1 *))
                                       x-%offset%)))
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i m) nil)
               (tagbody
                 (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
```

```
                          (+
                           (f2cl-lib:fref y-%data%
                                          (i)
                                          ((1 *))
                                          y-%offset%)
                           (* temp
                              (f2cl-lib:fref a-%data%
                                             (i j)
                                             ((1 lda) (1 *))
                                             a-%offset%)))))))
          (setf jx (f2cl-lib:int-add jx incx)))))
     (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
            ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
             (setf temp
                     (* alpha
                        (f2cl-lib:fref x-%data%
                                       (jx)
                                       ((1 *))
                                       x-%offset%)))
             (setf iy ky)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i m) nil)
               (tagbody
                 (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (+
                          (f2cl-lib:fref y-%data%
                                         (iy)
                                         ((1 *))
                                         y-%offset%)
                          (* temp
                             (f2cl-lib:fref a-%data%
                                            (i j)
                                            ((1 lda) (1 *))
                                            a-%offset%))))
                 (setf iy (f2cl-lib:int-add iy incy))))))
          (setf jx (f2cl-lib:int-add jx incx)))))))
(t
 (setf jy ky)
 (cond
   ((= incx 1)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp zero)
        (cond
          (noconj
```

```
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i m) nil)
                  (tagbody
                    (setf temp
                            (+ temp
                               (*
                                (f2cl-lib:fref a-%data%
                                               (i j)
                                               ((1 lda) (1 *))
                                               a-%offset%)
                                (f2cl-lib:fref x-%data%
                                               (i)
                                               ((1 *))
                                               x-%offset%)))))))
               (t
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i m) nil)
                  (tagbody
                    (setf temp
                            (+ temp
                               (*
                                (f2cl-lib:dconjg
                                 (f2cl-lib:fref a-%data%
                                                (i j)
                                                ((1 lda) (1 *))
                                                a-%offset%))
                                (f2cl-lib:fref x-%data%
                                               (i)
                                               ((1 *))
                                               x-%offset%))))))))
             (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                     (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                        (* alpha temp)))
             (setf jy (f2cl-lib:int-add jy incy)))))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf temp zero)
             (setf ix kx)
             (cond
               (noconj
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i m) nil)
                  (tagbody
                    (setf temp
                            (+ temp
                               (*
                                (f2cl-lib:fref a-%data%
                                               (i j)
```

```
                                                  ((1 lda) (1 *))
                                                  a-%offset%)
                                        (f2cl-lib:fref x-%data%
                                                  (ix)
                                                  ((1 *))
                                                  x-%offset%))))
                          (setf ix (f2cl-lib:int-add ix incx)))))
                      (t
                       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                     ((> i m) nil)
                         (tagbody
                           (setf temp
                                   (+ temp
                                      (*
                                       (f2cl-lib:dconjg
                                        (f2cl-lib:fref a-%data%
                                                       (i j)
                                                       ((1 lda) (1 *))
                                                       a-%offset%))
                                       (f2cl-lib:fref x-%data%
                                                  (ix)
                                                  ((1 *))
                                                  x-%offset%))))
                           (setf ix (f2cl-lib:int-add ix incx))))))
                    (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                            (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                               (* alpha temp)))
                    (setf jy (f2cl-lib:int-add jy incy))))))))
 end_label
        (return (values nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

13.3 zgerc - rank 1 $\alpha xy^H + A$

13.3.1 Axiom unit tests

— zgerc.input —

```
)set break resume
)sys rm -f zgerc.output
)spool zgerc.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

13.3.2 Axiom help page

— **zgerc.help** —

```
==========================================================================
zgerc examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

 NAME
      ZGERC - perform the rank 1 operation
              A := alpha*x*conjg( y' ) + A,

 SYNOPSIS
      SUBROUTINE ZGERC ( M, N, ALPHA, X, INCX, Y, INCY, A, LDA )

          COMPLEX*16   ALPHA

          INTEGER      INCX, INCY, LDA, M, N

          COMPLEX*16   A( LDA, * ), X( * ), Y( * )

 PURPOSE
      ZGERC  performs the rank 1 operation

      where alpha is a scalar, x is an m element vector, y is an n
      element vector and A is an m by n matrix.

 PARAMETERS
      M      - INTEGER.
             On entry, M specifies the number of rows of the
             matrix A.  M must be at least zero.  Unchanged on
             exit.

      N      - INTEGER.
             On entry, N specifies the number of columns of the
             matrix A.  N must be at least zero.  Unchanged on
             exit.

      ALPHA  - COMPLEX*16      .
```

```
          On entry, ALPHA specifies the scalar alpha.
          Unchanged on exit.

     X    - COMPLEX*16       array of dimension at least
          ( 1 + ( m - 1 )*abs( INCX ) ).  Before entry, the
          incremented array X must contain the m element vector
          x.  Unchanged on exit.

     INCX - INTEGER.
          On entry, INCX specifies the increment for the ele-
          ments of X. INCX must not be zero.  Unchanged on
          exit.

     Y    - COMPLEX*16       array of dimension at least
          ( 1 + ( n - 1 )*abs( INCY ) ).  Before entry, the
          incremented array Y must contain the n element vector
          y.  Unchanged on exit.

     INCY - INTEGER.
          On entry, INCY specifies the increment for the

          elements of Y. INCY must not be zero.  Unchanged on
          exit.

     A    - COMPLEX*16       array of DIMENSION ( LDA, n ).
          Before entry, the leading m by n part of the array A
          must contain the matrix of coefficients. On exit, A
          is overwritten by the updated matrix.

     LDA  - INTEGER.
          On entry, LDA specifies the first dimension of A as
          declared in the calling (sub) program. LDA must be at
          least max( 1, m ).  Unchanged on exit.
```

———-

13.3.3 fortran code

— zgerc.f —

```
*  ====================================================================
      SUBROUTINE ZGERC(M,N,ALPHA,X,INCX,Y,INCY,A,LDA)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
```

```
*
*     .. Scalar Arguments ..
      COMPLEX*16 ALPHA
      INTEGER INCX,INCY,LDA,M,N
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),X(*),Y(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP
      INTEGER I,INFO,IX,J,JY,KX
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DCONJG,MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (M.LT.0) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      ELSE IF (INCY.EQ.0) THEN
          INFO = 7
      ELSE IF (LDA.LT.MAX(1,M)) THEN
          INFO = 9
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZGERC ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR. (ALPHA.EQ.ZERO)) RETURN
*
*     Start the operations. In this version the elements of A are
```

```
*     accessed sequentially with one pass through A.
*
      IF (INCY.GT.0) THEN
          JY = 1
      ELSE
          JY = 1 - (N-1)*INCY
      END IF
      IF (INCX.EQ.1) THEN
          DO 20 J = 1,N
              IF (Y(JY).NE.ZERO) THEN
                  TEMP = ALPHA*DCONJG(Y(JY))
                  DO 10 I = 1,M
                      A(I,J) = A(I,J) + X(I)*TEMP
   10             CONTINUE
              END IF
              JY = JY + INCY
   20     CONTINUE
      ELSE
          IF (INCX.GT.0) THEN
              KX = 1
          ELSE
              KX = 1 - (M-1)*INCX
          END IF
          DO 40 J = 1,N
              IF (Y(JY).NE.ZERO) THEN
                  TEMP = ALPHA*DCONJG(Y(JY))
                  IX = KX
                  DO 30 I = 1,M
                      A(I,J) = A(I,J) + X(IX)*TEMP
                      IX = IX + INCX
   30             CONTINUE
              END IF
              JY = JY + INCY
   40     CONTINUE
      END IF
*
      RETURN
*
*     End of ZGERC .
*
      END
```

———-

13.3.4 lisp code

— BLAS 2 zgerc —

```
(let* ((zero (complex 0.0 0.0)))
  (declare (type (complex double-float) zero))
  (defun zgerc (m n alpha x incx y incy a lda)
    (declare (type (simple-array (complex double-float) (*)) a y x)
             (type (complex double-float) alpha)
             (type fixnum lda incy incx n m))
    (f2cl-lib:with-multi-array-data
        ((x (complex double-float) x-%data% x-%offset%)
         (y (complex double-float) y-%data% y-%offset%)
         (a (complex double-float) a-%data% a-%offset%))
      (prog ((i 0) (info 0) (ix 0) (j 0) (jy 0) (kx 0) (temp #C(0.0 0.0)))
        (declare (type fixnum i info ix j jy kx)
                 (type (complex double-float) temp))
        (setf info 0)
        (cond
          ((< m 0)
           (setf info 1))
          ((< n 0)
           (setf info 2))
          ((= incx 0)
           (setf info 5))
          ((= incy 0)
           (setf info 7))
          ((< lda (max (the fixnum 1) (the fixnum m)))
           (setf info 9)))
        (cond
          ((/= info 0)
           (error
            " ** On entry to ~a parameter number ~a had an illegal value~%"
            "ZGERC" info)
           (go end_label)))
        (if (or (= m 0) (= n 0) (= alpha zero)) (go end_label))
        (cond
          ((> incy 0)
           (setf jy 1))
          (t
           (setf jy
                   (f2cl-lib:int-sub 1
                                     (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                       incy)))))
        (cond
          ((= incx 1)
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (cond
                 ((/= (f2cl-lib:fref y (jy) ((1 *))) zero)
                  (setf temp
                          (* alpha
                             (f2cl-lib:dconjg
```

```
                        (f2cl-lib:fref y-%data%
                                       (jy)
                                       ((1 *))
                                       y-%offset%))))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref a-%data%
                                   (i j)
                                   ((1 lda) (1 *))
                                   a-%offset%)
                        (+
                         (f2cl-lib:fref a-%data%
                                        (i j)
                                        ((1 lda) (1 *))
                                        a-%offset%)
                         (*
                          (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                          temp)))))))
      (setf jy (f2cl-lib:int-add jy incy)))))
(t
 (cond
   ((> incx 0)
    (setf kx 1))
   (t
    (setf kx
            (f2cl-lib:int-sub 1
                              (f2cl-lib:int-mul
                               (f2cl-lib:int-sub m 1)
                               incx)))))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((/= (f2cl-lib:fref y (jy) ((1 *))) zero)
        (setf temp
                (* alpha
                   (f2cl-lib:dconjg
                    (f2cl-lib:fref y-%data%
                                   (jy)
                                   ((1 *))
                                   y-%offset%))))
        (setf ix kx)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data%
                                 (i j)
                                 ((1 lda) (1 *))
                                 a-%offset%)
```

```
                                (+
                                 (f2cl-lib:fref a-%data%
                                                (i j)
                                                ((1 lda) (1 *))
                                                a-%offset%)
                                 (*
                                  (f2cl-lib:fref x-%data%
                                                 (ix)
                                                 ((1 *))
                                                 x-%offset%)
                                  temp)))
                        (setf ix (f2cl-lib:int-add ix incx))))))
                (setf jy (f2cl-lib:int-add jy incy))))))
 end_label
        (return (values nil nil nil nil nil nil nil nil nil))))))
```

———-

13.4 zgeru - rank 1 $\alpha xy^T + A$

13.4.1 Axiom unit tests

— **zgeru.input** —

```
)set break resume
)sys rm -f zgeru.output
)spool zgeru.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

13.4.2 Axiom help page

— **zgeru.help** —

```
========================================================================
zgeru examples
========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

 NAME
      ZGERU - perform the rank 1 operation   A := alpha*x*y' + A,

 SYNOPSIS
      SUBROUTINE ZGERU ( M, N, ALPHA, X, INCX, Y, INCY, A, LDA )

          COMPLEX*16   ALPHA

          INTEGER      INCX, INCY, LDA, M, N

          COMPLEX*16   A( LDA, * ), X( * ), Y( * )

 PURPOSE
      ZGERU  performs the rank 1 operation

      where alpha is a scalar, x is an m element vector, y is an n
      element vector and A is an m by n matrix.

 PARAMETERS
      M      - INTEGER.
               On entry, M specifies the number of rows of the
               matrix A.  M must be at least zero.  Unchanged on
               exit.

      N      - INTEGER.
               On entry, N specifies the number of columns of the
               matrix A.  N must be at least zero.  Unchanged on
               exit.

      ALPHA  - COMPLEX*16      .
               On entry, ALPHA specifies the scalar alpha.
               Unchanged on exit.

      X      - COMPLEX*16       array of dimension at least
               ( 1 + ( m - 1 )*abs( INCX ) ).  Before entry, the
               incremented array X must contain the m element vector
               x.  Unchanged on exit.

      INCX   - INTEGER.
               On entry, INCX specifies the increment for the ele-
               ments of X. INCX must not be zero.  Unchanged on
               exit.

      Y      - COMPLEX*16       array of dimension at least
               ( 1 + ( n - 1 )*abs( INCY ) ).  Before entry, the
```

```
            incremented array Y must contain the n element vector
            y.  Unchanged on exit.

      INCY   - INTEGER.
            On entry, INCY specifies the increment for the ele-
            ments of Y. INCY must not be zero.  Unchanged on

            exit.

      A      - COMPLEX*16       array of DIMENSION ( LDA, n ).
            Before entry, the leading m by n part of the array A
            must contain the matrix of coefficients. On exit, A
            is overwritten by the updated matrix.

      LDA    - INTEGER.
            On entry, LDA specifies the first dimension of A as
            declared in the calling (sub) program. LDA must be at
            least max( 1, m ).  Unchanged on exit.
```

13.4.3 fortran code

— zgeru.f —

```
*  ==================================================================
      SUBROUTINE ZGERU(M,N,ALPHA,X,INCX,Y,INCY,A,LDA)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX*16 ALPHA
      INTEGER INCX,INCY,LDA,M,N
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),X(*),Y(*)
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
```

```
*     .. Local Scalars ..
      COMPLEX*16 TEMP
      INTEGER I,INFO,IX,J,JY,KX
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (M.LT.0) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      ELSE IF (INCY.EQ.0) THEN
          INFO = 7
      ELSE IF (LDA.LT.MAX(1,M)) THEN
          INFO = 9
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZGERU ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR. (ALPHA.EQ.ZERO)) RETURN
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
      IF (INCY.GT.0) THEN
          JY = 1
      ELSE
          JY = 1 - (N-1)*INCY
      END IF
      IF (INCX.EQ.1) THEN
          DO 20 J = 1,N
              IF (Y(JY).NE.ZERO) THEN
                  TEMP = ALPHA*Y(JY)
                  DO 10 I = 1,M
                      A(I,J) = A(I,J) + X(I)*TEMP
   10             CONTINUE
              END IF
```

```
            JY = JY + INCY
   20     CONTINUE
      ELSE
          IF (INCX.GT.0) THEN
              KX = 1
          ELSE
              KX = 1 - (M-1)*INCX
          END IF
          DO 40 J = 1,N
              IF (Y(JY).NE.ZERO) THEN
                  TEMP = ALPHA*Y(JY)
                  IX = KX
                  DO 30 I = 1,M
                      A(I,J) = A(I,J) + X(IX)*TEMP
                      IX = IX + INCX
   30             CONTINUE
              END IF
              JY = JY + INCY
   40     CONTINUE
      END IF
*
      RETURN
*
*     End of ZGERU .
*
      END
```

———-

13.4.4 lisp code

— BLAS 2 zgeru —

```
(let* ((zero (complex 0.0 0.0)))
  (declare (type (complex double-float) zero))
  (defun zgeru (m n alpha x incx y incy a lda)
    (declare (type (simple-array (complex double-float) (*)) a y x)
             (type (complex double-float) alpha)
             (type fixnum lda incy incx n m))
    (f2cl-lib:with-multi-array-data
        ((x (complex double-float) x-%data% x-%offset%)
         (y (complex double-float) y-%data% y-%offset%)
         (a (complex double-float) a-%data% a-%offset%))
      (prog ((i 0) (info 0) (ix 0) (j 0) (jy 0) (kx 0) (temp #C(0.0 0.0)))
        (declare (type fixnum i info ix j jy kx)
                 (type (complex double-float) temp))
        (setf info 0)
```

```
(cond
  ((< m 0)
   (setf info 1))
  ((< n 0)
   (setf info 2))
  ((= incx 0)
   (setf info 5))
  ((= incy 0)
   (setf info 7))
  ((< lda (max (the fixnum 1) (the fixnum m)))
   (setf info 9)))
(cond
  ((/= info 0)
   (error
    " ** On entry to ~a parameter number ~a had an illegal value~%"
    "ZGERU" info)
   (go end_label)))
(if (or (= m 0) (= n 0) (= alpha zero)) (go end_label))
(cond
  ((> incy 0)
   (setf jy 1))
  (t
   (setf jy
           (f2cl-lib:int-sub 1
                             (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                               incy)))))
(cond
  ((= incx 1)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
         ((/= (f2cl-lib:fref y (jy) ((1 *))) zero)
          (setf temp
                  (* alpha
                     (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref a-%data%
                                   (i j)
                                   ((1 lda) (1 *))
                                   a-%offset%)
                    (+
                     (f2cl-lib:fref a-%data%
                                    (i j)
                                    ((1 lda) (1 *))
                                    a-%offset%)
                     (*
                      (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
```

```
                              temp)))))))
                 (setf jy (f2cl-lib:int-add jy incy)))))
            (t
             (cond
               ((> incx 0)
                (setf kx 1))
               (t
                (setf kx
                        (f2cl-lib:int-sub 1
                                          (f2cl-lib:int-mul
                                           (f2cl-lib:int-sub m 1)
                                           incx)))))
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (cond
                   ((/= (f2cl-lib:fref y (jy) ((1 *))) zero)
                    (setf temp
                            (* alpha
                               (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)))
                    (setf ix kx)
                    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                  ((> i m) nil)
                      (tagbody
                        (setf (f2cl-lib:fref a-%data%
                                             (i j)
                                             ((1 lda) (1 *))
                                             a-%offset%)
                              (+
                               (f2cl-lib:fref a-%data%
                                              (i j)
                                              ((1 lda) (1 *))
                                              a-%offset%)
                               (*
                                (f2cl-lib:fref x-%data%
                                               (ix)
                                               ((1 *))
                                               x-%offset%)
                                temp)))
                        (setf ix (f2cl-lib:int-add ix incx))))))
                 (setf jy (f2cl-lib:int-add jy incy))))))
end_label
       (return (values nil nil nil nil nil nil nil nil nil))))))
```

———-

13.5 zhbmv - hermitian band matrix-vector $\alpha Ax + \beta y$

13.5.1 Axiom unit tests

— **zhbmv.input** —

```
)set break resume
)sys rm -f zhbmv.output
)spool zhbmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

13.5.2 Axiom help page

— **zhbmv.help** —

```
======================================================================
zhbmv examples
======================================================================

======================================================================
Man Page Details
======================================================================

 NAME
      ZHBMV - perform the matrix-vector operation
              y := alpha*A*x + beta*y,

 SYNOPSIS
      SUBROUTINE ZHBMV ( UPLO, N, K, ALPHA, A, LDA, X, INCX, BETA,
                         Y, INCY )

          COMPLEX*16   ALPHA, BETA

          INTEGER      INCX, INCY, K, LDA, N

          CHARACTER*1  UPLO

          COMPLEX*16   A( LDA, * ), X( * ), Y( * )
```

```
PURPOSE
     ZHBMV  performs the matrix-vector  operation

     where alpha and beta are scalars, x and y are n element vec-
     tors and A is an n by n hermitian band matrix, with k
     super-diagonals.

PARAMETERS
     UPLO   - CHARACTER*1.
              On entry, UPLO specifies whether the upper or lower
              triangular part of the band matrix A is being sup-
              plied as follows:

              UPLO = 'U' or 'u'   The upper triangular part of A is
              being supplied.

              UPLO = 'L' or 'l'   The lower triangular part of A is
              being supplied.

              Unchanged on exit.

     N      - INTEGER.
              On entry, N specifies the order of the matrix A.  N
              must be at least zero.  Unchanged on exit.

     K      - INTEGER.
              On entry, K specifies the number of super-diagonals
              of the matrix A. K must satisfy  0 .le. K.  Unchanged
              on exit.

     ALPHA  - COMPLEX*16      .
              On entry, ALPHA specifies the scalar alpha.
              Unchanged on exit.

     A      - COMPLEX*16       array of DIMENSION ( LDA, n ).

              Before entry with UPLO = 'U' or 'u', the leading ( k
              + 1 ) by n part of the array A must contain the upper
              triangular band part of the hermitian matrix, sup-
              plied column by column, with the leading diagonal of
              the matrix in row ( k + 1 ) of the array, the first
              super-diagonal starting at position 2 in row k, and
              so on. The top left k by k triangle of the array A is
              not referenced.  The following program segment will
              transfer the upper triangular part of a hermitian
              band matrix from conventional full matrix storage to
              band storage:

              DO 20, J = 1, N M = K + 1 - J DO 10, I = MAX( 1, J -
              K ), J A( M + I, J ) = matrix( I, J ) 10    CONTINUE
```

```
        20 CONTINUE

        Before entry with UPLO = 'L' or 'l', the leading ( k
        + 1 ) by n part of the array A must contain the lower
        triangular band part of the hermitian matrix, sup-
        plied column by column, with the leading diagonal of
        the matrix in row 1 of the array, the first sub-
        diagonal starting at position 1 in row 2, and so on.
        The bottom right k by k triangle of the array A is
        not referenced.  The following program segment will
        transfer the lower triangular part of a hermitian
        band matrix from conventional full matrix storage to
        band storage:

        DO 20, J = 1, N M = 1 - J DO 10, I = J, MIN( N, J + K
        ) A( M + I, J ) = matrix( I, J ) 10    CONTINUE 20
        CONTINUE

        Note that the imaginary parts of the diagonal ele-
        ments need not be set and are assumed to be zero.
        Unchanged on exit.

LDA     - INTEGER.
        On entry, LDA specifies the first dimension of A as
        declared in the calling (sub) program. LDA must be at
        least ( k + 1 ).  Unchanged on exit.

X       - COMPLEX*16       array of DIMENSION at least
        ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
        incremented array X must contain the vector x.
        Unchanged on exit.

INCX    - INTEGER.
        On entry, INCX specifies the increment for the ele-
        ments of X. INCX must not be zero.  Unchanged on
        exit.

BETA    - COMPLEX*16      .

        On entry, BETA specifies the scalar beta.  Unchanged
        on exit.

Y       - COMPLEX*16       array of DIMENSION at least
        ( 1 + ( n - 1 )*abs( INCY ) ).  Before entry, the
        incremented array Y must contain the vector y. On
        exit, Y is overwritten by the updated vector y.

INCY    - INTEGER.
        On entry, INCY specifies the increment for the ele-
        ments of Y. INCY must not be zero.  Unchanged on
```

```
exit.
```

———-

13.5.3 fortran code

— **zhbmv.f** —

```
*  ===================================================================
      SUBROUTINE ZHBMV(UPLO,N,K,ALPHA,A,LDA,X,INCX,BETA,Y,INCY)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX*16 ALPHA,BETA
      INTEGER INCX,INCY,K,LDA,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),X(*),Y(*)
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      COMPLEX*16 ONE
      PARAMETER (ONE= (1.0D+0,0.0D+0))
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,KPLUS1,KX,KY,L
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DBLE,DCONJG,MAX,MIN
*     ..
```

```
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (K.LT.0) THEN
          INFO = 3
      ELSE IF (LDA.LT. (K+1)) THEN
          INFO = 6
      ELSE IF (INCX.EQ.0) THEN
          INFO = 8
      ELSE IF (INCY.EQ.0) THEN
          INFO = 11
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZHBMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     Set up the start points in  X  and  Y.
*
      IF (INCX.GT.0) THEN
          KX = 1
      ELSE
          KX = 1 - (N-1)*INCX
      END IF
      IF (INCY.GT.0) THEN
          KY = 1
      ELSE
          KY = 1 - (N-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of the array A
*     are accessed sequentially with one pass through A.
*
*     First form  y := beta*y.
*
      IF (BETA.NE.ONE) THEN
          IF (INCY.EQ.1) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 10 I = 1,N
                      Y(I) = ZERO
   10             CONTINUE
```

```
            ELSE
                DO 20 I = 1,N
                    Y(I) = BETA*Y(I)
   20           CONTINUE
            END IF
        ELSE
            IY = KY
            IF (BETA.EQ.ZERO) THEN
                DO 30 I = 1,N
                    Y(IY) = ZERO
                    IY = IY + INCY
   30           CONTINUE
            ELSE
                DO 40 I = 1,N
                    Y(IY) = BETA*Y(IY)
                    IY = IY + INCY
   40           CONTINUE
            END IF
        END IF
      END IF
      IF (ALPHA.EQ.ZERO) RETURN
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  y  when upper triangle of A is stored.
*
        KPLUS1 = K + 1
        IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
            DO 60 J = 1,N
                TEMP1 = ALPHA*X(J)
                TEMP2 = ZERO
                L = KPLUS1 - J
                DO 50 I = MAX(1,J-K),J - 1
                    Y(I) = Y(I) + TEMP1*A(L+I,J)
                    TEMP2 = TEMP2 + DCONJG(A(L+I,J))*X(I)
   50           CONTINUE
                Y(J) = Y(J) + TEMP1*DBLE(A(KPLUS1,J)) + ALPHA*TEMP2
   60       CONTINUE
        ELSE
            JX = KX
            JY = KY
            DO 80 J = 1,N
                TEMP1 = ALPHA*X(JX)
                TEMP2 = ZERO
                IX = KX
                IY = KY
                L = KPLUS1 - J
                DO 70 I = MAX(1,J-K),J - 1
                    Y(IY) = Y(IY) + TEMP1*A(L+I,J)
                    TEMP2 = TEMP2 + DCONJG(A(L+I,J))*X(IX)
                    IX = IX + INCX
```

```
                    IY = IY + INCY
   70            CONTINUE
                 Y(JY) = Y(JY) + TEMP1*DBLE(A(KPLUS1,J)) + ALPHA*TEMP2
                 JX = JX + INCX
                 JY = JY + INCY
                 IF (J.GT.K) THEN
                     KX = KX + INCX
                     KY = KY + INCY
                 END IF
   80         CONTINUE
         END IF
      ELSE
*
*        Form  y  when lower triangle of A is stored.
*
          IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
              DO 100 J = 1,N
                  TEMP1 = ALPHA*X(J)
                  TEMP2 = ZERO
                  Y(J) = Y(J) + TEMP1*DBLE(A(1,J))
                  L = 1 - J
                  DO 90 I = J + 1,MIN(N,J+K)
                      Y(I) = Y(I) + TEMP1*A(L+I,J)
                      TEMP2 = TEMP2 + DCONJG(A(L+I,J))*X(I)
   90             CONTINUE
                  Y(J) = Y(J) + ALPHA*TEMP2
  100         CONTINUE
          ELSE
              JX = KX
              JY = KY
              DO 120 J = 1,N
                  TEMP1 = ALPHA*X(JX)
                  TEMP2 = ZERO
                  Y(JY) = Y(JY) + TEMP1*DBLE(A(1,J))
                  L = 1 - J
                  IX = JX
                  IY = JY
                  DO 110 I = J + 1,MIN(N,J+K)
                      IX = IX + INCX
                      IY = IY + INCY
                      Y(IY) = Y(IY) + TEMP1*A(L+I,J)
                      TEMP2 = TEMP2 + DCONJG(A(L+I,J))*X(IX)
  110             CONTINUE
                  Y(JY) = Y(JY) + ALPHA*TEMP2
                  JX = JX + INCX
                  JY = JY + INCY
  120         CONTINUE
          END IF
      END IF
*
```

```
      RETURN
*
*     End of ZHBMV .
*
      END
```

———-

13.5.4 lisp code

— BLAS 2 zhbmv —

```
(let* ((one (complex 1.0 0.0)) (zero (complex 0.0 0.0)))
 (declare (type (complex double-float) one) (type (complex double-float) zero))
  (defun zhbmv (uplo n k alpha a lda x incx beta y incy)
    (declare (type (simple-array (complex double-float) (*)) y x a)
             (type (complex double-float) beta alpha)
             (type fixnum incy incx lda k n)
             (type character uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a (complex double-float) a-%data% a-%offset%)
         (x (complex double-float) x-%data% x-%offset%)
         (y (complex double-float) y-%data% y-%offset%))
      (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (kplus1 0) (kx 0)
             (ky 0) (l 0) (temp1 #C(0.0 0.0)) (temp2 #C(0.0 0.0)))
        (declare (type fixnum i info ix iy j jx jy kplus1 kx ky l)
                 (type (complex double-float) temp1 temp2))
        (setf info 0)
        (cond
          ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
           (setf info 1))
          ((< n 0)
           (setf info 2))
          ((< k 0)
           (setf info 3))
          ((< lda (f2cl-lib:int-add k 1))
           (setf info 6))
          ((= incx 0)
           (setf info 8))
          ((= incy 0)
           (setf info 11)))
        (cond
          ((/= info 0)
           (error
            " ** On entry to ~a parameter number ~a had an illegal value~%"
            "ZHBMV" info)
```

```
   (go end_label)))
(if (or (= n 0) (and (= alpha zero) (= beta one))) (go end_label))
(cond
  ((> incx 0)
   (setf kx 1))
  (t
   (setf kx
           (f2cl-lib:int-sub 1
                             (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                               incx)))))
(cond
  ((> incy 0)
   (setf ky 1))
  (t
   (setf ky
           (f2cl-lib:int-sub 1
                             (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                               incy)))))
(cond
  ((/= beta one)
   (cond
     ((= incy 1)
      (cond
        ((= beta zero)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     zero))))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     (* beta
                        (f2cl-lib:fref y-%data%
                                       (i)
                                       ((1 *))
                                       y-%offset%))))))))
     (t
      (setf iy ky)
      (cond
        ((= beta zero)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     zero)
             (setf iy (f2cl-lib:int-add iy incy)))))
        (t
```

```
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (* beta
                           (f2cl-lib:fref y-%data%
                                          (iy)
                                          ((1 *))
                                          y-%offset%)))
                (setf iy (f2cl-lib:int-add iy incy))))))))))
(if (= alpha zero) (go end_label))
(cond
  ((char-equal uplo #\U)
   (setf kplus1 (f2cl-lib:int-add k 1))
   (cond
     ((and (= incx 1) (= incy 1))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf temp1
                  (* alpha
                     (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
          (setf temp2 zero)
          (setf l (f2cl-lib:int-sub kplus1 j))
          (f2cl-lib:fdo (i
                         (max (the fixnum 1)
                              (the fixnum
                                   (f2cl-lib:int-add j
                                                     (f2cl-lib:int-sub
                                                      k))))
                         (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                      (+
                       (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                       (* temp1
                          (f2cl-lib:fref a-%data%
                                         ((f2cl-lib:int-add l i) j)
                                         ((1 lda) (1 *))
                                         a-%offset%))))
              (setf temp2
                      (+ temp2
                         (*
                          (f2cl-lib:dconjg
                           (f2cl-lib:fref a-%data%
                                          ((f2cl-lib:int-add l i) j)
                                          ((1 lda) (1 *))
```

```
                                        a-%offset%))
                          (f2cl-lib:fref x-%data%
                                         (i)
                                         ((1 *))
                                         x-%offset%))))))
        (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                   (* temp1
                      (coerce (realpart
                       (f2cl-lib:fref a-%data%
                                      (kplus1 j)
                                      ((1 lda) (1 *))
                                      a-%offset%)) 'double-float))
                   (* alpha temp2))))))
    (t
     (setf jx kx)
     (setf jy ky)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp1
                 (* alpha
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
         (setf temp2 zero)
         (setf ix kx)
         (setf iy ky)
         (setf l (f2cl-lib:int-sub kplus1 j))
         (f2cl-lib:fdo (i
                        (max (the fixnum 1)
                             (the fixnum
                                  (f2cl-lib:int-add j
                                                    (f2cl-lib:int-sub
                                                     k))))
                        (f2cl-lib:int-add i 1))
                       ((> i
                           (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                        nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (+
                      (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (* temp1
                         (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-add l i) j)
                                        ((1 lda) (1 *))
                                        a-%offset%))))
             (setf temp2
                     (+ temp2
                        (*
                         (f2cl-lib:dconjg
```

```
                                (f2cl-lib:fref a-%data%
                                               ((f2cl-lib:int-add l i) j)
                                               ((1 lda) (1 *))
                                               a-%offset%))
                              (f2cl-lib:fref x-%data%
                                             (ix)
                                             ((1 *))
                                             x-%offset%))))
                  (setf ix (f2cl-lib:int-add ix incx))
                  (setf iy (f2cl-lib:int-add iy incy))))
            (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                       (* temp1
                          (coerce (realpart
                           (f2cl-lib:fref a-%data%
                                          (kplus1 j)
                                          ((1 lda) (1 *))
                                          a-%offset%)) 'double-float))
                       (* alpha temp2)))
            (setf jx (f2cl-lib:int-add jx incx))
            (setf jy (f2cl-lib:int-add jy incy))
            (cond
              ((> j k)
               (setf kx (f2cl-lib:int-add kx incx))
               (setf ky (f2cl-lib:int-add ky incy)))))))))
(t
 (cond
   ((and (= incx 1) (= incy 1))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp1
                (* alpha
                   (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
        (setf temp2 zero)
        (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                   (* temp1
                      (coerce (realpart
                       (f2cl-lib:fref a-%data%
                                      (1 j)
                                      ((1 lda) (1 *))
                                      a-%offset%)) 'double-float))))
        (setf l (f2cl-lib:int-sub 1 j))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                       (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the fixnum n)
                               (the fixnum
                                    (f2cl-lib:int-add j k))))
```

```
                    nil)
          (tagbody
            (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                    (+
                     (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     (* temp1
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add l i) j)
                                       ((1 lda) (1 *))
                                       a-%offset%))))
            (setf temp2
                    (+ temp2
                       (*
                        (f2cl-lib:dconjg
                         (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-add l i) j)
                                        ((1 lda) (1 *))
                                        a-%offset%))
                        (f2cl-lib:fref x-%data%
                                       (i)
                                       ((1 *))
                                       x-%offset%))))))
        (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                   (* alpha temp2))))))
  (t
   (setf jx kx)
   (setf jy ky)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp1
               (* alpha
                  (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
       (setf temp2 zero)
       (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
               (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                  (* temp1
                     (coerce (realpart
                      (f2cl-lib:fref a-%data%
                                     (1 j)
                                     ((1 lda) (1 *))
                                     a-%offset%)) 'double-float))))
       (setf l (f2cl-lib:int-sub 1 j))
       (setf ix jx)
       (setf iy jy)
       (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                      (f2cl-lib:int-add i 1))
                     ((> i
                         (min (the fixnum n)
```

```
                                        (the fixnum
                                             (f2cl-lib:int-add j k))))
                                   nil)
                        (tagbody
                          (setf ix (f2cl-lib:int-add ix incx))
                          (setf iy (f2cl-lib:int-add iy incy))
                          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                                  (+
                                   (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                                   (* temp1
                                      (f2cl-lib:fref a-%data%
                                                     ((f2cl-lib:int-add l i) j)
                                                     ((1 lda) (1 *))
                                                     a-%offset%))))
                          (setf temp2
                                  (+ temp2
                                     (*
                                      (f2cl-lib:dconjg
                                       (f2cl-lib:fref a-%data%
                                                      ((f2cl-lib:int-add l i) j)
                                                      ((1 lda) (1 *))
                                                      a-%offset%))
                                      (f2cl-lib:fref x-%data%
                                                     (ix)
                                                     ((1 *))
                                                     x-%offset%))))))
                    (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                            (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                               (* alpha temp2)))
                    (setf jx (f2cl-lib:int-add jx incx))
                    (setf jy (f2cl-lib:int-add jy incy))))))))
 end_label
        (return (values nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

13.6 zhemv - hermitian matrix-vector $\alpha Ax + \beta y$

13.6.1 Axiom unit tests

— zhemv.input —

```
)set break resume
)sys rm -f zhemv.output
)spool zhemv.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

13.6.2 Axiom help page

— **zhemv.help** —

```
=====================================================================
zhemv examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

 NAME
      ZHEMV - perform the matrix-vector operation
              y := alpha*A*x + beta*y,

 SYNOPSIS
      SUBROUTINE ZHEMV ( UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y,
                         INCY )

          COMPLEX*16   ALPHA, BETA

          INTEGER      INCX, INCY, LDA, N

          CHARACTER*1  UPLO

          COMPLEX*16   A( LDA, * ), X( * ), Y( * )

 PURPOSE
      ZHEMV  performs the matrix-vector  operation

      where alpha and beta are scalars, x and y are n element vec-
      tors and A is an n by n hermitian matrix.

 PARAMETERS
      UPLO   - CHARACTER*1.
             On entry, UPLO specifies whether the upper or lower
             triangular part of the array A is to be referenced as
             follows:
```

```
         UPLO = 'U' or 'u'   Only the upper triangular part of
         A is to be referenced.

         UPLO = 'L' or 'l'   Only the lower triangular part of
         A is to be referenced.

         Unchanged on exit.

N        - INTEGER.
         On entry, N specifies the order of the matrix A.  N
         must be at least zero.  Unchanged on exit.

ALPHA    - COMPLEX*16      .
         On entry, ALPHA specifies the scalar alpha.
         Unchanged on exit.

A        - COMPLEX*16       array of DIMENSION ( LDA, n ).
         Before entry with  UPLO = 'U' or 'u', the leading n
         by n upper triangular part of the array A must con-
         tain the upper triangular part of the hermitian
         matrix and the strictly lower triangular part of A is
         not referenced.  Before entry with UPLO = 'L' or 'l',
         the leading n by n lower triangular part of the array

         A must contain the lower triangular part of the her-
         mitian matrix and the strictly upper triangular part
         of A is not referenced.  Note that the imaginary
         parts of the diagonal elements need not be set and
         are assumed to be zero.  Unchanged on exit.

LDA      - INTEGER.
         On entry, LDA specifies the first dimension of A as
         declared in the calling (sub) program. LDA must be at
         least max( 1, n ).  Unchanged on exit.

X        - COMPLEX*16       array of dimension at least
         ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
         incremented array X must contain the n element vector
         x.  Unchanged on exit.

INCX     - INTEGER.
         On entry, INCX specifies the increment for the ele-
         ments of X. INCX must not be zero.  Unchanged on
         exit.

BETA     - COMPLEX*16      .
         On entry, BETA specifies the scalar beta. When BETA
         is supplied as zero then Y need not be set on input.
         Unchanged on exit.
```

```
Y      - COMPLEX*16       array of dimension at least
       ( 1 + ( n - 1 )*abs( INCY ) ).  Before entry, the
       incremented array Y must contain the n element vector
       y. On exit, Y is overwritten by the updated vector y.

INCY   - INTEGER.
       On entry, INCY specifies the increment for the ele-
       ments of Y. INCY must not be zero.  Unchanged on
       exit.
```

———-

13.6.3 fortran code

— zhemv.f —

```
*  ====================================================================
      SUBROUTINE ZHEMV(UPLO,N,ALPHA,A,LDA,X,INCX,BETA,Y,INCY)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX*16 ALPHA,BETA
      INTEGER INCX,INCY,LDA,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),X(*),Y(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16 ONE
      PARAMETER (ONE= (1.0D+0,0.0D+0))
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,KX,KY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
```

```
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DBLE,DCONJG,MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 5
      ELSE IF (INCX.EQ.0) THEN
          INFO = 7
      ELSE IF (INCY.EQ.0) THEN
          INFO = 10
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZHEMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     Set up the start points in  X  and  Y.
*
      IF (INCX.GT.0) THEN
          KX = 1
      ELSE
          KX = 1 - (N-1)*INCX
      END IF
      IF (INCY.GT.0) THEN
          KY = 1
      ELSE
          KY = 1 - (N-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the triangular part
*     of A.
*
*     First form  y := beta*y.
```

```
*
      IF (BETA.NE.ONE) THEN
          IF (INCY.EQ.1) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 10 I = 1,N
                      Y(I) = ZERO
   10             CONTINUE
              ELSE
                  DO 20 I = 1,N
                      Y(I) = BETA*Y(I)
   20             CONTINUE
              END IF
          ELSE
              IY = KY
              IF (BETA.EQ.ZERO) THEN
                  DO 30 I = 1,N
                      Y(IY) = ZERO
                      IY = IY + INCY
   30             CONTINUE
              ELSE
                  DO 40 I = 1,N
                      Y(IY) = BETA*Y(IY)
                      IY = IY + INCY
   40             CONTINUE
              END IF
          END IF
      END IF
      IF (ALPHA.EQ.ZERO) RETURN
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  y  when A is stored in upper triangle.
*
          IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
              DO 60 J = 1,N
                  TEMP1 = ALPHA*X(J)
                  TEMP2 = ZERO
                  DO 50 I = 1,J - 1
                      Y(I) = Y(I) + TEMP1*A(I,J)
                      TEMP2 = TEMP2 + DCONJG(A(I,J))*X(I)
   50             CONTINUE
                  Y(J) = Y(J) + TEMP1*DBLE(A(J,J)) + ALPHA*TEMP2
   60         CONTINUE
          ELSE
              JX = KX
              JY = KY
              DO 80 J = 1,N
                  TEMP1 = ALPHA*X(JX)
                  TEMP2 = ZERO
                  IX = KX
                  IY = KY
```

```
                DO 70 I = 1,J - 1
                    Y(IY) = Y(IY) + TEMP1*A(I,J)
                    TEMP2 = TEMP2 + DCONJG(A(I,J))*X(IX)
                    IX = IX + INCX
                    IY = IY + INCY
   70           CONTINUE
                Y(JY) = Y(JY) + TEMP1*DBLE(A(J,J)) + ALPHA*TEMP2
                JX = JX + INCX
                JY = JY + INCY
   80       CONTINUE
        END IF
      ELSE
*
*       Form  y  when A is stored in lower triangle.
*
         IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
             DO 100 J = 1,N
                 TEMP1 = ALPHA*X(J)
                 TEMP2 = ZERO
                 Y(J) = Y(J) + TEMP1*DBLE(A(J,J))
                 DO 90 I = J + 1,N
                     Y(I) = Y(I) + TEMP1*A(I,J)
                     TEMP2 = TEMP2 + DCONJG(A(I,J))*X(I)
   90            CONTINUE
                 Y(J) = Y(J) + ALPHA*TEMP2
  100         CONTINUE
         ELSE
             JX = KX
             JY = KY
             DO 120 J = 1,N
                 TEMP1 = ALPHA*X(JX)
                 TEMP2 = ZERO
                 Y(JY) = Y(JY) + TEMP1*DBLE(A(J,J))
                 IX = JX
                 IY = JY
                 DO 110 I = J + 1,N
                     IX = IX + INCX
                     IY = IY + INCY
                     Y(IY) = Y(IY) + TEMP1*A(I,J)
                     TEMP2 = TEMP2 + DCONJG(A(I,J))*X(IX)
  110            CONTINUE
                 Y(JY) = Y(JY) + ALPHA*TEMP2
                 JX = JX + INCX
                 JY = JY + INCY
  120         CONTINUE
         END IF
      END IF
*
      RETURN
*
```

```
*     End of ZHEMV .
*
      END
```

———-

13.6.4 lisp code

— BLAS 2 zhemv —

```
(let* ((one (complex 1.0 0.0)) (zero (complex 0.0 0.0)))
 (declare (type (complex double-float) one) (type (complex double-float) zero))
  (defun zhemv (uplo n alpha a lda x incx beta y incy)
    (declare (type (simple-array (complex double-float) (*)) y x a)
             (type (complex double-float) beta alpha)
             (type fixnum incy incx lda n)
             (type character uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a (complex double-float) a-%data% a-%offset%)
         (x (complex double-float) x-%data% x-%offset%)
         (y (complex double-float) y-%data% y-%offset%))
      (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (kx 0) (ky 0)
             (temp1 #C(0.0 0.0)) (temp2 #C(0.0 0.0)))
        (declare (type fixnum i info ix iy j jx jy kx ky)
                 (type (complex double-float) temp1 temp2))
        (setf info 0)
        (cond
          ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
           (setf info 1))
          ((< n 0)
           (setf info 2))
          ((< lda (max (the fixnum 1) (the fixnum n)))
           (setf info 5))
          ((= incx 0)
           (setf info 7))
          ((= incy 0)
           (setf info 10)))
        (cond
          ((/= info 0)
           (error
            " ** On entry to ~a parameter number ~a had an illegal value~%"
            "ZHEMV" info)
           (go end_label)))
        (if (or (= n 0) (and (= alpha zero) (= beta one))) (go end_label))
        (cond
          ((> incx 0)
```

```
    (setf kx 1))
   (t
    (setf kx
            (f2cl-lib:int-sub 1
                              (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                incx)))))
(cond
  ((> incy 0)
   (setf ky 1))
  (t
   (setf ky
            (f2cl-lib:int-sub 1
                              (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                incy)))))
(cond
  ((/= beta one)
   (cond
     ((= incy 1)
      (cond
        ((= beta zero)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     zero))))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     (* beta
                        (f2cl-lib:fref y-%data%
                                       (i)
                                       ((1 *))
                                       y-%offset%))))))))
     (t
      (setf iy ky)
      (cond
        ((= beta zero)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     zero)
             (setf iy (f2cl-lib:int-add iy incy)))))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
```

```
                        (* beta
                           (f2cl-lib:fref y-%data%
                                          (iy)
                                          ((1 *))
                                          y-%offset%)))
                (setf iy (f2cl-lib:int-add iy incy)))))))))
(if (= alpha zero) (go end_label))
(cond
  ((char-equal uplo #\U)
   (cond
     ((and (= incx 1) (= incy 1))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf temp1
                  (* alpha
                     (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
          (setf temp2 zero)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                      (+
                       (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                       (* temp1
                          (f2cl-lib:fref a-%data%
                                         (i j)
                                         ((1 lda) (1 *))
                                         a-%offset%))))
              (setf temp2
                      (+ temp2
                         (*
                          (f2cl-lib:dconjg
                           (f2cl-lib:fref a-%data%
                                          (i j)
                                          ((1 lda) (1 *))
                                          a-%offset%))
                          (f2cl-lib:fref x-%data%
                                         (i)
                                         ((1 *))
                                         x-%offset%))))))
          (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                  (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                     (* temp1
                        (coerce (realpart
                         (f2cl-lib:fref a-%data%
                                        (j j)
                                        ((1 lda) (1 *))
```

```
                                        a-%offset%)) 'double-float))
                    (* alpha temp2))))))
          (t
           (setf jx kx)
           (setf jy ky)
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (setf temp1
                       (* alpha
                          (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
               (setf temp2 zero)
               (setf ix kx)
               (setf iy ky)
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i
                                 (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                              nil)
                 (tagbody
                   (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                           (+
                            (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                            (* temp1
                               (f2cl-lib:fref a-%data%
                                              (i j)
                                              ((1 lda) (1 *))
                                              a-%offset%))))
                   (setf temp2
                           (+ temp2
                              (*
                               (f2cl-lib:dconjg
                                (f2cl-lib:fref a-%data%
                                               (i j)
                                               ((1 lda) (1 *))
                                               a-%offset%))
                               (f2cl-lib:fref x-%data%
                                              (ix)
                                              ((1 *))
                                              x-%offset%))))
                   (setf ix (f2cl-lib:int-add ix incx))
                   (setf iy (f2cl-lib:int-add iy incy))))
               (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                       (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                          (* temp1
                             (coerce (realpart
                              (f2cl-lib:fref a-%data%
                                             (j j)
                                             ((1 lda) (1 *))
                                             a-%offset%)) 'double-float))
                          (* alpha temp2)))
```

```
            (setf jx (f2cl-lib:int-add jx incx))
            (setf jy (f2cl-lib:int-add jy incy)))))))
    (t
     (cond
       ((and (= incx 1) (= incy 1))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf temp1
                    (* alpha
                       (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
            (setf temp2 zero)
            (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                       (* temp1
                          (coerce (realpart
                           (f2cl-lib:fref a-%data%
                                          (j j)
                                          ((1 lda) (1 *))
                                          a-%offset%)) 'double-float))))
            (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                           (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                         (* temp1
                            (f2cl-lib:fref a-%data%
                                           (i j)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
                (setf temp2
                        (+ temp2
                           (*
                            (f2cl-lib:dconjg
                             (f2cl-lib:fref a-%data%
                                            (i j)
                                            ((1 lda) (1 *))
                                            a-%offset%))
                            (f2cl-lib:fref x-%data%
                                           (i)
                                           ((1 *))
                                           x-%offset%))))))
            (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                       (* alpha temp2))))))
       (t
        (setf jx kx)
        (setf jy ky)
```

```
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf temp1
                        (* alpha
                           (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
                (setf temp2 zero)
                (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                        (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                           (* temp1
                              (coerce (realpart
                               (f2cl-lib:fref a-%data%
                                              (j j)
                                              ((1 lda) (1 *))
                                              a-%offset%)) 'double-float))))
                (setf ix jx)
                (setf iy jy)
                (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                               (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf ix (f2cl-lib:int-add ix incx))
                    (setf iy (f2cl-lib:int-add iy incy))
                    (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                            (+
                             (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                             (* temp1
                                (f2cl-lib:fref a-%data%
                                               (i j)
                                               ((1 lda) (1 *))
                                               a-%offset%))))
                    (setf temp2
                            (+ temp2
                               (*
                                (f2cl-lib:dconjg
                                 (f2cl-lib:fref a-%data%
                                                (i j)
                                                ((1 lda) (1 *))
                                                a-%offset%))
                                (f2cl-lib:fref x-%data%
                                               (ix)
                                               ((1 *))
                                               x-%offset%))))))
                (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                        (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                           (* alpha temp2)))
                (setf jx (f2cl-lib:int-add jx incx))
                (setf jy (f2cl-lib:int-add jy incy))))))))
end_label
       (return (values nil nil nil nil nil nil nil nil nil nil))))))
```

———-

13.7 zher - hermitian rank 1 $\alpha xx^H + A$

13.7.1 Axiom unit tests

— **zher.input** —

```
)set break resume
)sys rm -f zher.output
)spool zher.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

13.7.2 Axiom help page

— **zher.help** —

```
======================================================================
zher examples
======================================================================

======================================================================
Man Page Details
======================================================================

 NAME
      ZHER - perform the hermitian rank 1 operation
             A := alpha*x*conjg( x' ) + A,

 SYNOPSIS
      SUBROUTINE ZHER ( UPLO, N, ALPHA, X, INCX, A, LDA )

          DOUBLE      PRECISION ALPHA

          INTEGER     INCX, LDA, N
```

```
         CHARACTER*1 UPLO

         COMPLEX*16  A( LDA, * ), X( * )

PURPOSE
     ZHER   performs the hermitian rank 1 operation

     where alpha is a real scalar, x is an n element vector and A
     is an n by n hermitian matrix.

PARAMETERS
     UPLO   - CHARACTER*1.
              On entry, UPLO specifies whether the upper or lower
              triangular part of the array A is to be referenced as
              follows:

              UPLO = 'U' or 'u'   Only the upper triangular part of
              A is to be referenced.

              UPLO = 'L' or 'l'   Only the lower triangular part of
              A is to be referenced.

              Unchanged on exit.

     N      - INTEGER.
              On entry, N specifies the order of the matrix A.  N
              must be at least zero.  Unchanged on exit.

     ALPHA  - DOUBLE PRECISION.
              On entry, ALPHA specifies the scalar alpha.
              Unchanged on exit.

     X      - COMPLEX*16       array of dimension at least
              ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
              incremented array X must contain the n element vector
              x.  Unchanged on exit.

     INCX   - INTEGER.
              On entry, INCX specifies the increment for the ele-
              ments of X. INCX must not be zero.  Unchanged on

              exit.

     A      - COMPLEX*16       array of DIMENSION ( LDA, n ).
              Before entry with  UPLO = 'U' or 'u', the leading n
              by n upper triangular part of the array A must con-
              tain the upper triangular part of the hermitian
              matrix and the strictly lower triangular part of A is
              not referenced. On exit, the upper triangular part of
```

```
             the array A is overwritten by the upper triangular
             part of the updated matrix.  Before entry with UPLO =
             'L' or 'l', the leading n by n lower triangular part
             of the array A must contain the lower triangular part
             of the hermitian matrix and the strictly upper tri-
             angular part of A is not referenced. On exit, the
             lower triangular part of the array A is overwritten
             by the lower triangular part of the updated matrix.
             Note that the imaginary parts of the diagonal ele-
             ments need not be set, they are assumed to be zero,
             and on exit they are set to zero.

      LDA    - INTEGER.
             On entry, LDA specifies the first dimension of A as
             declared in the calling (sub) program. LDA must be at
             least max( 1, n ).  Unchanged on exit.
```

13.7.3 fortran code

— zher.f —

```
*  ==================================================================
      SUBROUTINE ZHER(UPLO,N,ALPHA,X,INCX,A,LDA)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION ALPHA
      INTEGER INCX,LDA,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),X(*)
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*     .. Local Scalars ..
```

```
      COMPLEX*16 TEMP
      INTEGER I,INFO,IX,J,JX,KX
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DBLE,DCONJG,MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 7
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZHER  ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (ALPHA.EQ.DBLE(ZERO))) RETURN
*
*     Set the start point in X if the increment is not unity.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the triangular part
*     of A.
*
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  A  when A is stored in upper triangle.
```

```
*
         IF (INCX.EQ.1) THEN
             DO 20 J = 1,N
                 IF (X(J).NE.ZERO) THEN
                     TEMP = ALPHA*DCONJG(X(J))
                     DO 10 I = 1,J - 1
                         A(I,J) = A(I,J) + X(I)*TEMP
   10                CONTINUE
                     A(J,J) = DBLE(A(J,J)) + DBLE(X(J)*TEMP)
                 ELSE
                     A(J,J) = DBLE(A(J,J))
                 END IF
   20        CONTINUE
         ELSE
             JX = KX
             DO 40 J = 1,N
                 IF (X(JX).NE.ZERO) THEN
                     TEMP = ALPHA*DCONJG(X(JX))
                     IX = KX
                     DO 30 I = 1,J - 1
                         A(I,J) = A(I,J) + X(IX)*TEMP
                         IX = IX + INCX
   30                CONTINUE
                     A(J,J) = DBLE(A(J,J)) + DBLE(X(JX)*TEMP)
                 ELSE
                     A(J,J) = DBLE(A(J,J))
                 END IF
                 JX = JX + INCX
   40        CONTINUE
         END IF
      ELSE
*
*        Form  A  when A is stored in lower triangle.
*
         IF (INCX.EQ.1) THEN
             DO 60 J = 1,N
                 IF (X(J).NE.ZERO) THEN
                     TEMP = ALPHA*DCONJG(X(J))
                     A(J,J) = DBLE(A(J,J)) + DBLE(TEMP*X(J))
                     DO 50 I = J + 1,N
                         A(I,J) = A(I,J) + X(I)*TEMP
   50                CONTINUE
                 ELSE
                     A(J,J) = DBLE(A(J,J))
                 END IF
   60        CONTINUE
         ELSE
             JX = KX
             DO 80 J = 1,N
                 IF (X(JX).NE.ZERO) THEN
```

```
                    TEMP = ALPHA*DCONJG(X(JX))
                    A(J,J) = DBLE(A(J,J)) + DBLE(TEMP*X(JX))
                    IX = JX
                    DO 70 I = J + 1,N
                        IX = IX + INCX
                        A(I,J) = A(I,J) + X(IX)*TEMP
   70               CONTINUE
                ELSE
                    A(J,J) = DBLE(A(J,J))
                END IF
                JX = JX + INCX
   80         CONTINUE
           END IF
      END IF
*
      RETURN
*
*     End of ZHER  .
*
      END
```

———-

13.7.4 lisp code

— BLAS 2 zher —

```
(let* ((zero (complex 0.0 0.0)))
  (declare (type (complex double-float) zero))
  (defun zher (uplo n alpha x incx a lda)
    (declare (type (simple-array (complex double-float) (*)) a x)
             (type (double-float) alpha)
             (type fixnum lda incx n)
             (type character uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (x (complex double-float) x-%data% x-%offset%)
         (a (complex double-float) a-%data% a-%offset%))
      (prog ((i 0) (info 0) (ix 0) (j 0) (jx 0) (kx 0) (temp #C(0.0 0.0)))
        (declare (type fixnum i info ix j jx kx)
                 (type (complex double-float) temp))
        (setf info 0)
        (cond
          ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
           (setf info 1))
          ((< n 0)
           (setf info 2))
```

```
    ((= incx 0)
     (setf info 5))
    ((< lda (max (the fixnum 1) (the fixnum n)))
     (setf info 7)))
  (cond
    ((/= info 0)
     (error
      " ** On entry to ~a parameter number ~a had an illegal value~%"
      "ZHER" info)
     (go end_label)))
  (if (or (= n 0) (= alpha (coerce (realpart zero) 'double-float)))
     (go end_label))
  (cond
    ((<= incx 0)
     (setf kx
             (f2cl-lib:int-sub 1
                               (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                 incx))))
    ((/= incx 1)
     (setf kx 1)))
  (cond
    ((char-equal uplo #\U)
     (cond
       ((= incx 1)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (cond
              ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
               (setf temp
                       (coerce
                        (* alpha
                           (f2cl-lib:dconjg
                            (f2cl-lib:fref x-%data%
                                           (j)
                                           ((1 *))
                                           x-%offset%)))
                        '(complex double-float)))
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i
                                 (f2cl-lib:int-add j
                                                   (f2cl-lib:int-sub 1)))
                              nil)
                 (tagbody
                   (setf (f2cl-lib:fref a-%data%
                                        (i j)
                                        ((1 lda) (1 *))
                                        a-%offset%)
                         (+
                          (f2cl-lib:fref a-%data%
```

```
                                        (i j)
                                        ((1 lda) (1 *))
                                        a-%offset%)
                                (*
                                 (f2cl-lib:fref x-%data%
                                                (i)
                                                ((1 *))
                                                x-%offset%)
                                 temp)))))
                 (setf (f2cl-lib:fref a-%data%
                                      (j j)
                                      ((1 lda) (1 *))
                                      a-%offset%)
                       (coerce
                        (+
                         (coerce (realpart
                          (f2cl-lib:fref a-%data%
                                         (j j)
                                         ((1 lda) (1 *))
                                         a-%offset%)) 'double-float)
                         (coerce (realpart
                          (*
                           (f2cl-lib:fref x-%data%
                                          (j)
                                          ((1 *))
                                          x-%offset%)
                           temp)) 'double-float))
                        '(complex double-float))))
                (t
                 (setf (f2cl-lib:fref a-%data%
                                      (j j)
                                      ((1 lda) (1 *))
                                      a-%offset%)
                       (coerce
                        (coerce (realpart
                         (f2cl-lib:fref a-%data%
                                        (j j)
                                        ((1 lda) (1 *))
                                        a-%offset%)) 'double-float)
                        '(complex double-float))))))))
        (t
         (setf jx kx)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (cond
               ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
                (setf temp
                       (coerce
                        (* alpha
```

```
                 (f2cl-lib:dconjg
                  (f2cl-lib:fref x-%data%
                                 (jx)
                                 ((1 *))
                                 x-%offset%)))
            '(complex double-float)))
 (setf ix kx)
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i
                   (f2cl-lib:int-add j
                                     (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (setf (f2cl-lib:fref a-%data%
                          (i j)
                          ((1 lda) (1 *))
                          a-%offset%)
                 (+
                  (f2cl-lib:fref a-%data%
                                 (i j)
                                 ((1 lda) (1 *))
                                 a-%offset%)
                  (*
                   (f2cl-lib:fref x-%data%
                                  (ix)
                                  ((1 *))
                                  x-%offset%)
                   temp)))
     (setf ix (f2cl-lib:int-add ix incx))))
 (setf (f2cl-lib:fref a-%data%
                      (j j)
                      ((1 lda) (1 *))
                      a-%offset%)
           (coerce
            (+
             (coerce (realpart
              (f2cl-lib:fref a-%data%
                             (j j)
                             ((1 lda) (1 *))
                             a-%offset%)) 'double-float)
             (coerce (realpart
              (*
               (f2cl-lib:fref x-%data%
                              (jx)
                              ((1 *))
                              x-%offset%)
               temp)) 'double-float))
            '(complex double-float))))
(t
 (setf (f2cl-lib:fref a-%data%
```

```
                                    (j j)
                                    ((1 lda) (1 *))
                                    a-%offset%)
                          (coerce
                           (coerce (realpart
                            (f2cl-lib:fref a-%data%
                                           (j j)
                                           ((1 lda) (1 *))
                                           a-%offset%)) 'double-float)
                           '(complex double-float)))))
                 (setf jx (f2cl-lib:int-add jx incx)))))))
        (t
         (cond
           ((= incx 1)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (cond
                  ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
                   (setf temp
                           (coerce
                            (* alpha
                               (f2cl-lib:dconjg
                                (f2cl-lib:fref x-%data%
                                               (j)
                                               ((1 *))
                                               x-%offset%)))
                            '(complex double-float)))
                   (setf (f2cl-lib:fref a-%data%
                                        (j j)
                                        ((1 lda) (1 *))
                                        a-%offset%)
                           (coerce
                            (+
                             (coerce (realpart
                              (f2cl-lib:fref a-%data%
                                             (j j)
                                             ((1 lda) (1 *))
                                             a-%offset%)) 'double-float)
                             (coerce (realpart
                              (* temp
                                 (f2cl-lib:fref x-%data%
                                                (j)
                                                ((1 *))
                                                x-%offset%))) 'double-float))
                            '(complex double-float)))
                   (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                                  (f2cl-lib:int-add i 1))
                                 ((> i n) nil)
                     (tagbody
```

```
                (setf (f2cl-lib:fref a-%data%
                                     (i j)
                                     ((1 lda) (1 *))
                                     a-%offset%)
                      (+
                       (f2cl-lib:fref a-%data%
                                      (i j)
                                      ((1 lda) (1 *))
                                      a-%offset%)
                       (*
                        (f2cl-lib:fref x-%data%
                                       (i)
                                       ((1 *))
                                       x-%offset%)
                        temp))))))
        (t
         (setf (f2cl-lib:fref a-%data%
                              (j j)
                              ((1 lda) (1 *))
                              a-%offset%)
                (coerce
                 (coerce (realpart
                  (f2cl-lib:fref a-%data%
                                 (j j)
                                 ((1 lda) (1 *))
                                 a-%offset%)) 'double-float)
                 '(complex double-float))))))))
(t
 (setf jx kx)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
        (setf temp
                (coerce
                 (* alpha
                    (f2cl-lib:dconjg
                     (f2cl-lib:fref x-%data%
                                    (jx)
                                    ((1 *))
                                    x-%offset%)))
                 '(complex double-float)))
        (setf (f2cl-lib:fref a-%data%
                             (j j)
                             ((1 lda) (1 *))
                             a-%offset%)
                (coerce
                 (+
                  (coerce (realpart
```

```
                                (f2cl-lib:fref a-%data%
                                               (j j)
                                               ((1 lda) (1 *))
                                               a-%offset%)) 'double-float)
                              (coerce (realpart
                                (* temp
                                   (f2cl-lib:fref x-%data%
                                                  (jx)
                                                  ((1 *))
                                                  x-%offset%))) 'double-float))
                            '(complex double-float)))
                      (setf ix jx)
                      (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                                     (f2cl-lib:int-add i 1))
                                    ((> i n) nil)
                        (tagbody
                          (setf ix (f2cl-lib:int-add ix incx))
                          (setf (f2cl-lib:fref a-%data%
                                               (i j)
                                               ((1 lda) (1 *))
                                               a-%offset%)
                                  (+
                                   (f2cl-lib:fref a-%data%
                                                  (i j)
                                                  ((1 lda) (1 *))
                                                  a-%offset%)
                                   (*
                                    (f2cl-lib:fref x-%data%
                                                   (ix)
                                                   ((1 *))
                                                   x-%offset%)
                                    temp))))))
                     (t
                      (setf (f2cl-lib:fref a-%data%
                                           (j j)
                                           ((1 lda) (1 *))
                                           a-%offset%)
                              (coerce
                               (coerce (realpart
                                 (f2cl-lib:fref a-%data%
                                                (j j)
                                                ((1 lda) (1 *))
                                                a-%offset%)) 'double-float)
                            '(complex double-float)))))
                  (setf jx (f2cl-lib:int-add jx incx))))))))
end_label
       (return (values nil nil nil nil nil nil nil))))))
```

———-

13.8 zher2 - hermitian rank 2 $\alpha xy^H + conjg(\alpha)yx^H + A$

13.8.1 Axiom unit tests

— **zher2.input** —

```
)set break resume
)sys rm -f zher2.output
)spool zher2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

13.8.2 Axiom help page

— **zher2.help** —

```
======================================================================
zher2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

 NAME
      ZHER2 - perform the hermitian rank 2 operation
      A := alpha*x*conjg( y' ) + conjg( alpha )*y*conjg( x' ) + A,

 SYNOPSIS
      SUBROUTINE ZHER2 ( UPLO, N, ALPHA, X, INCX, Y, INCY, A, LDA
                       )

          COMPLEX*16   ALPHA

          INTEGER      INCX, INCY, LDA, N

          CHARACTER*1  UPLO

          COMPLEX*16   A( LDA, * ), X( * ), Y( * )
```

```
PURPOSE
     ZHER2  performs the hermitian rank 2 operation

     where alpha is a scalar, x and y are n element vectors and A
     is an n by n hermitian matrix.

PARAMETERS
     UPLO   - CHARACTER*1.
              On entry, UPLO specifies whether the upper or lower
              triangular part of the array A is to be referenced as
              follows:

              UPLO = 'U' or 'u'   Only the upper triangular part of
              A is to be referenced.

              UPLO = 'L' or 'l'   Only the lower triangular part of
              A is to be referenced.

              Unchanged on exit.

     N      - INTEGER.
              On entry, N specifies the order of the matrix A.  N
              must be at least zero.  Unchanged on exit.

     ALPHA  - COMPLEX*16      .
              On entry, ALPHA specifies the scalar alpha.
              Unchanged on exit.

     X      - COMPLEX*16       array of dimension at least
              ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
              incremented array X must contain the n element vector
              x.  Unchanged on exit.

     INCX   - INTEGER.
              On entry, INCX specifies the increment for the

              elements of X. INCX must not be zero.  Unchanged on
              exit.

     Y      - COMPLEX*16       array of dimension at least
              ( 1 + ( n - 1 )*abs( INCY ) ).  Before entry, the
              incremented array Y must contain the n element vector
              y.  Unchanged on exit.

     INCY   - INTEGER.
              On entry, INCY specifies the increment for the ele-
              ments of Y. INCY must not be zero.  Unchanged on
              exit.

     A      - COMPLEX*16       array of DIMENSION ( LDA, n ).
```

```
            Before entry with  UPLO = 'U' or 'u', the leading n
            by n upper triangular part of the array A must con-
            tain the upper triangular part of the hermitian
            matrix and the strictly lower triangular part of A is
            not referenced. On exit, the upper triangular part of
            the array A is overwritten by the upper triangular
            part of the updated matrix.  Before entry with UPLO =
            'L' or 'l', the leading n by n lower triangular part
            of the array A must contain the lower triangular part
            of the hermitian matrix and the strictly upper tri-
            angular part of A is not referenced. On exit, the
            lower triangular part of the array A is overwritten
            by the lower triangular part of the updated matrix.
            Note that the imaginary parts of the diagonal ele-
            ments need not be set, they are assumed to be zero,
            and on exit they are set to zero.

      LDA    - INTEGER.
            On entry, LDA specifies the first dimension of A as
            declared in the calling (sub) program. LDA must be at
            least max( 1, n ).  Unchanged on exit.
```

———-

13.8.3 fortran code

— zher2.f —

```
*  ==================================================================
      SUBROUTINE ZHER2(UPLO,N,ALPHA,X,INCX,Y,INCY,A,LDA)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX*16 ALPHA
      INTEGER INCX,INCY,LDA,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),X(*),Y(*)
*     ..
*
*  ==================================================================
*
```

```
*     .. Parameters ..
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,KX,KY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DBLE,DCONJG,MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      ELSE IF (INCY.EQ.0) THEN
          INFO = 7
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 9
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZHER2 ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (ALPHA.EQ.ZERO)) RETURN
*
*     Set up the start points in X and Y if the increments are not both
*     unity.
*
      IF ((INCX.NE.1) .OR. (INCY.NE.1)) THEN
          IF (INCX.GT.0) THEN
              KX = 1
          ELSE
              KX = 1 - (N-1)*INCX
```

```
         END IF
         IF (INCY.GT.0) THEN
             KY = 1
         ELSE
             KY = 1 - (N-1)*INCY
         END IF
         JX = KX
         JY = KY
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the triangular part
*     of A.
*
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  A  when A is stored in the upper triangle.
*
         IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
             DO 20 J = 1,N
                 IF ((X(J).NE.ZERO) .OR. (Y(J).NE.ZERO)) THEN
                     TEMP1 = ALPHA*DCONJG(Y(J))
                     TEMP2 = DCONJG(ALPHA*X(J))
                     DO 10 I = 1,J - 1
                         A(I,J) = A(I,J) + X(I)*TEMP1 + Y(I)*TEMP2
   10                CONTINUE
                     A(J,J) = DBLE(A(J,J)) +
     +                       DBLE(X(J)*TEMP1+Y(J)*TEMP2)
                 ELSE
                     A(J,J) = DBLE(A(J,J))
                 END IF
   20        CONTINUE
         ELSE
             DO 40 J = 1,N
                 IF ((X(JX).NE.ZERO) .OR. (Y(JY).NE.ZERO)) THEN
                     TEMP1 = ALPHA*DCONJG(Y(JY))
                     TEMP2 = DCONJG(ALPHA*X(JX))
                     IX = KX
                     IY = KY
                     DO 30 I = 1,J - 1
                         A(I,J) = A(I,J) + X(IX)*TEMP1 + Y(IY)*TEMP2
                         IX = IX + INCX
                         IY = IY + INCY
   30                CONTINUE
                     A(J,J) = DBLE(A(J,J)) +
     +                       DBLE(X(JX)*TEMP1+Y(JY)*TEMP2)
                 ELSE
                     A(J,J) = DBLE(A(J,J))
                 END IF
                 JX = JX + INCX
```

```
                JY = JY + INCY
   40         CONTINUE
          END IF
      ELSE
*
*        Form  A  when A is stored in the lower triangle.
*
          IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
              DO 60 J = 1,N
                  IF ((X(J).NE.ZERO) .OR. (Y(J).NE.ZERO)) THEN
                      TEMP1 = ALPHA*DCONJG(Y(J))
                      TEMP2 = DCONJG(ALPHA*X(J))
                      A(J,J) = DBLE(A(J,J)) +
     +                         DBLE(X(J)*TEMP1+Y(J)*TEMP2)
                      DO 50 I = J + 1,N
                          A(I,J) = A(I,J) + X(I)*TEMP1 + Y(I)*TEMP2
   50                 CONTINUE
                  ELSE
                      A(J,J) = DBLE(A(J,J))
                  END IF
   60         CONTINUE
          ELSE
              DO 80 J = 1,N
                  IF ((X(JX).NE.ZERO) .OR. (Y(JY).NE.ZERO)) THEN
                      TEMP1 = ALPHA*DCONJG(Y(JY))
                      TEMP2 = DCONJG(ALPHA*X(JX))
                      A(J,J) = DBLE(A(J,J)) +
     +                         DBLE(X(JX)*TEMP1+Y(JY)*TEMP2)
                      IX = JX
                      IY = JY
                      DO 70 I = J + 1,N
                          IX = IX + INCX
                          IY = IY + INCY
                          A(I,J) = A(I,J) + X(IX)*TEMP1 + Y(IY)*TEMP2
   70                 CONTINUE
                  ELSE
                      A(J,J) = DBLE(A(J,J))
                  END IF
                  JX = JX + INCX
                  JY = JY + INCY
   80         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of ZHER2 .
*
      END
```

———-

13.8.4 lisp code

— BLAS 2 zher2 —

```
(let* ((zero (complex 0.0 0.0)))
  (declare (type (complex double-float) zero))
  (defun zher2 (uplo n alpha x incx y incy a lda)
    (declare (type (simple-array (complex double-float) (*)) a y x)
             (type (complex double-float) alpha)
             (type fixnum lda incy incx n)
             (type character uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (x (complex double-float) x-%data% x-%offset%)
         (y (complex double-float) y-%data% y-%offset%)
         (a (complex double-float) a-%data% a-%offset%))
      (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (kx 0) (ky 0)
             (temp1 #C(0.0 0.0)) (temp2 #C(0.0 0.0)))
        (declare (type fixnum i info ix iy j jx jy kx ky)
                 (type (complex double-float) temp1 temp2))
        (setf info 0)
        (cond
          ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
           (setf info 1))
          ((< n 0)
           (setf info 2))
          ((= incx 0)
           (setf info 5))
          ((= incy 0)
           (setf info 7))
          ((< lda (max (the fixnum 1) (the fixnum n)))
           (setf info 9)))
        (cond
          ((/= info 0)
           (error
            " ** On entry to ~a parameter number ~a had an illegal value~%"
            "ZHER2" info)
           (go end_label)))
        (if (or (= n 0) (= alpha zero)) (go end_label))
        (cond
          ((or (/= incx 1) (/= incy 1))
           (cond
             ((> incx 0)
              (setf kx 1))
             (t
              (setf kx
```

```
                (f2cl-lib:int-sub 1
                                  (f2cl-lib:int-mul
                                   (f2cl-lib:int-sub n 1)
                                   incx)))))
    (cond
      ((> incy 0)
       (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                  (f2cl-lib:int-mul
                                   (f2cl-lib:int-sub n 1)
                                   incy)))))
    (setf jx kx)
    (setf jy ky)))
  (cond
    ((char-equal uplo #\U)
     (cond
       ((and (= incx 1) (= incy 1))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (cond
              ((or (/= (f2cl-lib:fref x (j) ((1 *))) zero)
                   (/= (f2cl-lib:fref y (j) ((1 *))) zero))
               (setf temp1
                       (* alpha
                          (f2cl-lib:dconjg
                           (f2cl-lib:fref y-%data%
                                          (j)
                                          ((1 *))
                                          y-%offset%))))
               (setf temp2
                       (coerce
                        (f2cl-lib:dconjg
                         (* alpha
                            (f2cl-lib:fref x-%data%
                                           (j)
                                           ((1 *))
                                           x-%offset%)))
                        '(complex double-float)))
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i
                                 (f2cl-lib:int-add j
                                                   (f2cl-lib:int-sub 1)))
                              nil)
                 (tagbody
                   (setf (f2cl-lib:fref a-%data%
                                        (i j)
                                        ((1 lda) (1 *))
```

```
                                a-%offset%)
                      (+
                       (f2cl-lib:fref a-%data%
                                      (i j)
                                      ((1 lda) (1 *))
                                      a-%offset%)
                       (*
                        (f2cl-lib:fref x-%data%
                                       (i)
                                       ((1 *))
                                       x-%offset%)
                        temp1)
                       (*
                        (f2cl-lib:fref y-%data%
                                       (i)
                                       ((1 *))
                                       y-%offset%)
                        temp2)))))
          (setf (f2cl-lib:fref a-%data%
                               (j j)
                               ((1 lda) (1 *))
                               a-%offset%)
                (coerce
                 (+
                  (coerce (realpart
                   (f2cl-lib:fref a-%data%
                                  (j j)
                                  ((1 lda) (1 *))
                                  a-%offset%)) 'double-float)
                  (coerce (realpart
                   (+
                    (*
                     (f2cl-lib:fref x-%data%
                                    (j)
                                    ((1 *))
                                    x-%offset%)
                     temp1)
                    (*
                     (f2cl-lib:fref y-%data%
                                    (j)
                                    ((1 *))
                                    y-%offset%)
                     temp2))) 'double-float))
                 '(complex double-float))))
        (t
         (setf (f2cl-lib:fref a-%data%
                              (j j)
                              ((1 lda) (1 *))
                              a-%offset%)
               (coerce
```

```
                    (coerce (realpart
                     (f2cl-lib:fref a-%data%
                                    (j j)
                                    ((1 lda) (1 *))
                                    a-%offset%)) 'double-float)
                    '(complex double-float))))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((or (/= (f2cl-lib:fref x (jx) ((1 *))) zero)
            (/= (f2cl-lib:fref y (jy) ((1 *))) zero))
        (setf temp1
                (* alpha
                   (f2cl-lib:dconjg
                    (f2cl-lib:fref y-%data%
                                   (jy)
                                   ((1 *))
                                   y-%offset%))))
        (setf temp2
                (coerce
                 (f2cl-lib:dconjg
                  (* alpha
                     (f2cl-lib:fref x-%data%
                                    (jx)
                                    ((1 *))
                                    x-%offset%)))
                 '(complex double-float)))
        (setf ix kx)
        (setf iy ky)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i
                          (f2cl-lib:int-add j
                                            (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data%
                                 (i j)
                                 ((1 lda) (1 *))
                                 a-%offset%)
                  (+
                   (f2cl-lib:fref a-%data%
                                  (i j)
                                  ((1 lda) (1 *))
                                  a-%offset%)
                   (*
                    (f2cl-lib:fref x-%data%
                                   (ix)
                                   ((1 *))
```

```
                                    x-%offset%)
                     temp1)
                    (*
                     (f2cl-lib:fref y-%data%
                                    (iy)
                                    ((1 *))
                                    y-%offset%)
                     temp2)))
            (setf ix (f2cl-lib:int-add ix incx))
            (setf iy (f2cl-lib:int-add iy incy))))
      (setf (f2cl-lib:fref a-%data%
                           (j j)
                           ((1 lda) (1 *))
                           a-%offset%)
              (coerce
               (+
                (coerce (realpart
                 (f2cl-lib:fref a-%data%
                                (j j)
                                ((1 lda) (1 *))
                                a-%offset%)) 'double-float)
                (coerce (realpart
                 (+
                  (*
                   (f2cl-lib:fref x-%data%
                                  (jx)
                                  ((1 *))
                                  x-%offset%)
                   temp1)
                  (*
                   (f2cl-lib:fref y-%data%
                                  (jy)
                                  ((1 *))
                                  y-%offset%)
                   temp2))) 'double-float))
               '(complex double-float))))
     (t
      (setf (f2cl-lib:fref a-%data%
                           (j j)
                           ((1 lda) (1 *))
                           a-%offset%)
              (coerce
               (coerce (realpart
                (f2cl-lib:fref a-%data%
                               (j j)
                               ((1 lda) (1 *))
                               a-%offset%)) 'double-float)
               '(complex double-float)))))
    (setf jx (f2cl-lib:int-add jx incx))
    (setf jy (f2cl-lib:int-add jy incy)))))))
```

```
(t
 (cond
   ((and (= incx 1) (= incy 1))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
          ((or (/= (f2cl-lib:fref x (j) ((1 *))) zero)
               (/= (f2cl-lib:fref y (j) ((1 *))) zero))
           (setf temp1
                   (* alpha
                      (f2cl-lib:dconjg
                       (f2cl-lib:fref y-%data%
                                      (j)
                                      ((1 *))
                                      y-%offset%))))
           (setf temp2
                   (coerce
                    (f2cl-lib:dconjg
                     (* alpha
                        (f2cl-lib:fref x-%data%
                                       (j)
                                       ((1 *))
                                       x-%offset%)))
                    '(complex double-float)))
           (setf (f2cl-lib:fref a-%data%
                                (j j)
                                ((1 lda) (1 *))
                                a-%offset%)
                   (coerce
                    (+
                     (coerce (realpart
                      (f2cl-lib:fref a-%data%
                                     (j j)
                                     ((1 lda) (1 *))
                                     a-%offset%)) 'double-float)
                     (coerce (realpart
                      (+
                       (*
                        (f2cl-lib:fref x-%data%
                                       (j)
                                       ((1 *))
                                       x-%offset%)
                        temp1)
                       (*
                        (f2cl-lib:fref y-%data%
                                       (j)
                                       ((1 *))
                                       y-%offset%)
                        temp2))) 'double-float))
```

```
                 '(complex double-float)))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                       (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data%
                                 (i j)
                                 ((1 lda) (1 *))
                                 a-%offset%)
                  (+
                   (f2cl-lib:fref a-%data%
                                  (i j)
                                  ((1 lda) (1 *))
                                  a-%offset%)
                   (*
                    (f2cl-lib:fref x-%data%
                                   (i)
                                   ((1 *))
                                   x-%offset%)
                    temp1)
                   (*
                    (f2cl-lib:fref y-%data%
                                   (i)
                                   ((1 *))
                                   y-%offset%)
                    temp2))))))
       (t
        (setf (f2cl-lib:fref a-%data%
                             (j j)
                             ((1 lda) (1 *))
                             a-%offset%)
              (coerce
               (coerce (realpart
                (f2cl-lib:fref a-%data%
                               (j j)
                               ((1 lda) (1 *))
                               a-%offset%)) 'double-float)
               '(complex double-float))))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((or (/= (f2cl-lib:fref x (jx) ((1 *))) zero)
            (/= (f2cl-lib:fref y (jy) ((1 *))) zero))
        (setf temp1
              (* alpha
                 (f2cl-lib:dconjg
                  (f2cl-lib:fref y-%data%
                                 (jy)
```

```
                            ((1 *))
                            y-%offset%))))
(setf temp2
        (coerce
         (f2cl-lib:dconjg
          (* alpha
             (f2cl-lib:fref x-%data%
                            (jx)
                            ((1 *))
                            x-%offset%)))
         '(complex double-float)))
(setf (f2cl-lib:fref a-%data%
                     (j j)
                     ((1 lda) (1 *))
                     a-%offset%)
        (coerce
         (+
          (coerce (realpart
           (f2cl-lib:fref a-%data%
                          (j j)
                          ((1 lda) (1 *))
                          a-%offset%)) 'double-float)
          (coerce (realpart
           (+
            (*
             (f2cl-lib:fref x-%data%
                            (jx)
                            ((1 *))
                            x-%offset%)
             temp1)
            (*
             (f2cl-lib:fref y-%data%
                            (jy)
                            ((1 *))
                            y-%offset%)
             temp2))) 'double-float))
         '(complex double-float)))
(setf ix jx)
(setf iy jy)
(f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
               (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf ix (f2cl-lib:int-add ix incx))
    (setf iy (f2cl-lib:int-add iy incy))
    (setf (f2cl-lib:fref a-%data%
                         (i j)
                         ((1 lda) (1 *))
                         a-%offset%)
            (+
```

```
                                (f2cl-lib:fref a-%data%
                                               (i j)
                                               ((1 lda) (1 *))
                                               a-%offset%)
                                (*
                                 (f2cl-lib:fref x-%data%
                                                (ix)
                                                ((1 *))
                                                x-%offset%)
                                 temp1)
                                (*
                                 (f2cl-lib:fref y-%data%
                                                (iy)
                                                ((1 *))
                                                y-%offset%)
                                 temp2))))))
                     (t
                      (setf (f2cl-lib:fref a-%data%
                                           (j j)
                                           ((1 lda) (1 *))
                                           a-%offset%)
                            (coerce
                             (coerce (realpart
                              (f2cl-lib:fref a-%data%
                                             (j j)
                                             ((1 lda) (1 *))
                                             a-%offset%)) 'double-float)
                             '(complex double-float)))))
                  (setf jx (f2cl-lib:int-add jx incx))
                  (setf jy (f2cl-lib:int-add jy incy))))))))
 end_label
        (return (values nil nil nil nil nil nil nil nil nil))))))
```

———-

13.9 zhpmv - hermitian matrix-vector $\alpha Ax + \beta y$

13.9.1 Axiom unit tests

— zhpmv.input —

```
)set break resume
)sys rm -f zhpmv.output
)spool zhpmv.output
)set message test on
)set message auto off
```

```
)clear all

)spool
)lisp (bye)
```

———-

13.9.2 Axiom help page

— zhpmv.help —

```
==================================================================
zhpmv examples
==================================================================

==================================================================
Man Page Details
==================================================================

 NAME
      ZHPMV - perform the matrix-vector operation
              y := alpha*A*x + beta*y,

 SYNOPSIS
      SUBROUTINE ZHPMV ( UPLO, N, ALPHA, AP, X, INCX, BETA, Y,
                         INCY )

          COMPLEX*16   ALPHA, BETA

          INTEGER      INCX, INCY, N

          CHARACTER*1  UPLO

          COMPLEX*16   AP( * ), X( * ), Y( * )

 PURPOSE
      ZHPMV  performs the matrix-vector operation

      where alpha and beta are scalars, x and y are n element vec-
      tors and A is an n by n hermitian matrix, supplied in packed
      form.

 PARAMETERS
      UPLO   - CHARACTER*1.
             On entry, UPLO specifies whether the upper or lower
             triangular part of the matrix A is supplied in the
             packed array AP as follows:
```

```
        UPLO = 'U' or 'u'   The upper triangular part of A is
        supplied in AP.

        UPLO = 'L' or 'l'   The lower triangular part of A is
        supplied in AP.

        Unchanged on exit.

N       - INTEGER.
        On entry, N specifies the order of the matrix A.  N
        must be at least zero.  Unchanged on exit.

ALPHA   - COMPLEX*16      .
        On entry, ALPHA specifies the scalar alpha.
        Unchanged on exit.

AP      - COMPLEX*16       array of DIMENSION at least
        ( ( n*( n + 1 ) )/2 ).  Before entry with UPLO = 'U'
        or 'u', the array AP must contain the upper triangu-
        lar part of the hermitian matrix packed sequentially,
        column by column, so that AP( 1 ) contains a( 1, 1 ),
        AP( 2 ) and AP( 3 ) contain a( 1, 2 ) and a( 2, 2 )

        respectively, and so on.  Before entry with UPLO =
        'L' or 'l', the array AP must contain the lower tri-
        angular part of the hermitian matrix packed sequen-
        tially, column by column, so that AP( 1 ) contains a(
        1, 1 ), AP( 2 ) and AP( 3 ) contain a( 2, 1 ) and a(
        3, 1 ) respectively, and so on.  Note that the ima-
        ginary parts of the diagonal elements need not be set
        and are assumed to be zero.  Unchanged on exit.

X       - COMPLEX*16       array of dimension at least
        ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
        incremented array X must contain the n element vector
        x.  Unchanged on exit.

INCX    - INTEGER.
        On entry, INCX specifies the increment for the ele-
        ments of X. INCX must not be zero.  Unchanged on
        exit.

BETA    - COMPLEX*16      .
        On entry, BETA specifies the scalar beta. When BETA
        is supplied as zero then Y need not be set on input.
        Unchanged on exit.

Y       - COMPLEX*16       array of dimension at least
        ( 1 + ( n - 1 )*abs( INCY ) ).  Before entry, the
        incremented array Y must contain the n element vector
```

```
            y. On exit, Y is overwritten by the updated vector y.

      INCY   - INTEGER.
             On entry, INCY specifies the increment for the ele-
             ments of Y. INCY must not be zero.  Unchanged on
             exit.
```

13.9.3 fortran code

— zhpmv.f —

```
*  ==================================================================
      SUBROUTINE ZHPMV(UPLO,N,ALPHA,AP,X,INCX,BETA,Y,INCY)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX*16 ALPHA,BETA
      INTEGER INCX,INCY,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16 AP(*),X(*),Y(*)
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
      COMPLEX*16 ONE
      PARAMETER (ONE= (1.0D+0,0.0D+0))
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,K,KK,KX,KY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
```

```
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DBLE,DCONJG
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 6
      ELSE IF (INCY.EQ.0) THEN
          INFO = 9
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZHPMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     Set up the start points in  X  and  Y.
*
      IF (INCX.GT.0) THEN
          KX = 1
      ELSE
          KX = 1 - (N-1)*INCX
      END IF
      IF (INCY.GT.0) THEN
          KY = 1
      ELSE
          KY = 1 - (N-1)*INCY
      END IF
*
*     Start the operations. In this version the elements of the array AP
*     are accessed sequentially with one pass through AP.
*
*     First form  y := beta*y.
*
      IF (BETA.NE.ONE) THEN
          IF (INCY.EQ.1) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 10 I = 1,N
                      Y(I) = ZERO
```

```
   10             CONTINUE
              ELSE
                  DO 20 I = 1,N
                      Y(I) = BETA*Y(I)
   20             CONTINUE
              END IF
          ELSE
              IY = KY
              IF (BETA.EQ.ZERO) THEN
                  DO 30 I = 1,N
                      Y(IY) = ZERO
                      IY = IY + INCY
   30             CONTINUE
              ELSE
                  DO 40 I = 1,N
                      Y(IY) = BETA*Y(IY)
                      IY = IY + INCY
   40             CONTINUE
              END IF
          END IF
      END IF
      IF (ALPHA.EQ.ZERO) RETURN
      KK = 1
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  y  when AP contains the upper triangle.
*
          IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
              DO 60 J = 1,N
                  TEMP1 = ALPHA*X(J)
                  TEMP2 = ZERO
                  K = KK
                  DO 50 I = 1,J - 1
                      Y(I) = Y(I) + TEMP1*AP(K)
                      TEMP2 = TEMP2 + DCONJG(AP(K))*X(I)
                      K = K + 1
   50             CONTINUE
                  Y(J) = Y(J) + TEMP1*DBLE(AP(KK+J-1)) + ALPHA*TEMP2
                  KK = KK + J
   60         CONTINUE
          ELSE
              JX = KX
              JY = KY
              DO 80 J = 1,N
                  TEMP1 = ALPHA*X(JX)
                  TEMP2 = ZERO
                  IX = KX
                  IY = KY
                  DO 70 K = KK,KK + J - 2
                      Y(IY) = Y(IY) + TEMP1*AP(K)
```

```
                    TEMP2 = TEMP2 + DCONJG(AP(K))*X(IX)
                    IX = IX + INCX
                    IY = IY + INCY
   70            CONTINUE
                 Y(JY) = Y(JY) + TEMP1*DBLE(AP(KK+J-1)) + ALPHA*TEMP2
                 JX = JX + INCX
                 JY = JY + INCY
                 KK = KK + J
   80         CONTINUE
          END IF
      ELSE
*
*        Form  y  when AP contains the lower triangle.
*
          IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
              DO 100 J = 1,N
                  TEMP1 = ALPHA*X(J)
                  TEMP2 = ZERO
                  Y(J) = Y(J) + TEMP1*DBLE(AP(KK))
                  K = KK + 1
                  DO 90 I = J + 1,N
                      Y(I) = Y(I) + TEMP1*AP(K)
                      TEMP2 = TEMP2 + DCONJG(AP(K))*X(I)
                      K = K + 1
   90             CONTINUE
                  Y(J) = Y(J) + ALPHA*TEMP2
                  KK = KK + (N-J+1)
  100         CONTINUE
          ELSE
              JX = KX
              JY = KY
              DO 120 J = 1,N
                  TEMP1 = ALPHA*X(JX)
                  TEMP2 = ZERO
                  Y(JY) = Y(JY) + TEMP1*DBLE(AP(KK))
                  IX = JX
                  IY = JY
                  DO 110 K = KK + 1,KK + N - J
                      IX = IX + INCX
                      IY = IY + INCY
                      Y(IY) = Y(IY) + TEMP1*AP(K)
                      TEMP2 = TEMP2 + DCONJG(AP(K))*X(IX)
  110             CONTINUE
                  Y(JY) = Y(JY) + ALPHA*TEMP2
                  JX = JX + INCX
                  JY = JY + INCY
                  KK = KK + (N-J+1)
  120         CONTINUE
          END IF
      END IF
```

```
*
      RETURN
*
*     End of ZHPMV .
*
      END
```

———-

13.9.4 lisp code

— BLAS 2 zhpmv —

```
(let* ((one (complex 1.0 0.0)) (zero (complex 0.0 0.0)))
 (declare (type (complex double-float) one) (type (complex double-float) zero))
  (defun zhpmv (uplo n alpha ap x incx beta y incy)
    (declare (type (simple-array (complex double-float) (*)) y x ap)
             (type (complex double-float) beta alpha)
             (type fixnum incy incx n)
             (type character uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (ap (complex double-float) ap-%data% ap-%offset%)
         (x (complex double-float) x-%data% x-%offset%)
         (y (complex double-float) y-%data% y-%offset%))
      (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (k 0) (kk 0)
             (kx 0) (ky 0) (temp1 #C(0.0 0.0)) (temp2 #C(0.0 0.0)))
        (declare (type fixnum i info ix iy j jx jy k kk kx ky)
                 (type (complex double-float) temp1 temp2))
        (setf info 0)
        (cond
          ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
           (setf info 1))
          ((< n 0)
           (setf info 2))
          ((= incx 0)
           (setf info 6))
          ((= incy 0)
           (setf info 9)))
        (cond
          ((/= info 0)
           (error
            " ** On entry to ~a parameter number ~a had an illegal value~%"
            "ZHPMV" info)
           (go end_label)))
        (if (or (= n 0) (and (= alpha zero) (= beta one))) (go end_label))
        (cond
```

```
    ((> incx 0)
     (setf kx 1))
    (t
     (setf kx
             (f2cl-lib:int-sub 1
                               (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                 incx)))))
  (cond
    ((> incy 0)
     (setf ky 1))
    (t
     (setf ky
             (f2cl-lib:int-sub 1
                               (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                 incy)))))
  (cond
    ((/= beta one)
     (cond
       ((= incy 1)
        (cond
          ((= beta zero)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                       zero))))
          (t
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                       (* beta
                          (f2cl-lib:fref y-%data%
                                         (i)
                                         ((1 *))
                                         y-%offset%))))))))
       (t
        (setf iy ky)
        (cond
          ((= beta zero)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       zero)
               (setf iy (f2cl-lib:int-add iy incy)))))
          (t
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
```

```
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (* beta
                           (f2cl-lib:fref y-%data%
                                          (iy)
                                          ((1 *))
                                          y-%offset%)))
                (setf iy (f2cl-lib:int-add iy incy)))))))))
(if (= alpha zero) (go end_label))
(setf kk 1)
(cond
  ((char-equal uplo #\U)
   (cond
     ((and (= incx 1) (= incy 1))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf temp1
                  (* alpha
                     (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
          (setf temp2 zero)
          (setf k kk)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                      (+
                       (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                       (* temp1
                          (f2cl-lib:fref ap-%data%
                                         (k)
                                         ((1 *))
                                         ap-%offset%))))
              (setf temp2
                      (+ temp2
                         (*
                          (f2cl-lib:dconjg
                           (f2cl-lib:fref ap-%data%
                                          (k)
                                          ((1 *))
                                          ap-%offset%))
                          (f2cl-lib:fref x-%data%
                                         (i)
                                         ((1 *))
                                         x-%offset%))))
              (setf k (f2cl-lib:int-add k 1))))
          (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                  (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                     (* temp1
```

```
                    (coerce (realpart
                     (f2cl-lib:fref ap-%data%
                                    ((f2cl-lib:int-sub
                                      (f2cl-lib:int-add kk j)
                                      1))
                                    ((1 *))
                                    ap-%offset%)) 'double-float))
                (* alpha temp2)))
     (setf kk (f2cl-lib:int-add kk j)))))
(t
 (setf jx kx)
 (setf jy ky)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf temp1
             (* alpha
                (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
     (setf temp2 zero)
     (setf ix kx)
     (setf iy ky)
     (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                   ((> k
                       (f2cl-lib:int-add kk
                                         j
                                         (f2cl-lib:int-sub 2)))
                    nil)
       (tagbody
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (+
                  (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (* temp1
                     (f2cl-lib:fref ap-%data%
                                    (k)
                                    ((1 *))
                                    ap-%offset%))))
         (setf temp2
                 (+ temp2
                    (*
                     (f2cl-lib:dconjg
                      (f2cl-lib:fref ap-%data%
                                     (k)
                                     ((1 *))
                                     ap-%offset%))
                     (f2cl-lib:fref x-%data%
                                    (ix)
                                    ((1 *))
                                    x-%offset%))))
         (setf ix (f2cl-lib:int-add ix incx))
         (setf iy (f2cl-lib:int-add iy incy))))
```

```
            (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                       (* temp1
                          (coerce (realpart
                           (f2cl-lib:fref ap-%data%
                                          ((f2cl-lib:int-sub
                                            (f2cl-lib:int-add kk j)
                                            1))
                                          ((1 *))
                                          ap-%offset%)) 'double-float))
                       (* alpha temp2)))
            (setf jx (f2cl-lib:int-add jx incx))
            (setf jy (f2cl-lib:int-add jy incy))
            (setf kk (f2cl-lib:int-add kk j)))))))
    (t
     (cond
       ((and (= incx 1) (= incy 1))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf temp1
                    (* alpha
                       (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
            (setf temp2 zero)
            (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                       (* temp1
                          (coerce (realpart
                           (f2cl-lib:fref ap-%data%
                                          (kk)
                                          ((1 *))
                                          ap-%offset%)) 'double-float))))
            (setf k (f2cl-lib:int-add kk 1))
            (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                           (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                         (* temp1
                            (f2cl-lib:fref ap-%data%
                                           (k)
                                           ((1 *))
                                           ap-%offset%))))
                (setf temp2
                        (+ temp2
                           (*
                            (f2cl-lib:dconjg
                             (f2cl-lib:fref ap-%data%
```

```
                                        (k)
                                        ((1 *))
                                        ap-%offset%))
                          (f2cl-lib:fref x-%data%
                                        (i)
                                        ((1 *))
                                        x-%offset%))))
              (setf k (f2cl-lib:int-add k 1))))
          (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                  (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                     (* alpha temp2)))
          (setf kk
                  (f2cl-lib:int-add kk
                                    (f2cl-lib:int-add
                                     (f2cl-lib:int-sub n j)
                                     1))))))
    (t
     (setf jx kx)
     (setf jy ky)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp1
                  (* alpha
                     (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
         (setf temp2 zero)
         (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                  (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                     (* temp1
                        (coerce (realpart
                         (f2cl-lib:fref ap-%data%
                                        (kk)
                                        ((1 *))
                                        ap-%offset%)) 'double-float))))
         (setf ix jx)
         (setf iy jy)
         (f2cl-lib:fdo (k (f2cl-lib:int-add kk 1)
                         (f2cl-lib:int-add k 1))
                        ((> k
                            (f2cl-lib:int-add kk
                                              n
                                              (f2cl-lib:int-sub j)))
                         nil)
           (tagbody
             (setf ix (f2cl-lib:int-add ix incx))
             (setf iy (f2cl-lib:int-add iy incy))
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (+
                      (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (* temp1
```

```
                                (f2cl-lib:fref ap-%data%
                                               (k)
                                               ((1 *))
                                               ap-%offset%))))
                    (setf temp2
                            (+ temp2
                               (*
                                (f2cl-lib:dconjg
                                 (f2cl-lib:fref ap-%data%
                                                (k)
                                                ((1 *))
                                                ap-%offset%))
                                (f2cl-lib:fref x-%data%
                                               (ix)
                                               ((1 *))
                                               x-%offset%))))))
                (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                        (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                           (* alpha temp2)))
                (setf jx (f2cl-lib:int-add jx incx))
                (setf jy (f2cl-lib:int-add jy incy))
                (setf kk
                        (f2cl-lib:int-add kk
                                          (f2cl-lib:int-add
                                           (f2cl-lib:int-sub n j)
                                           1)))))))))
end_label
       (return (values nil nil nil nil nil nil nil nil nil))))))
```

———-

13.10 zhpr - hermitian rank 1 $\alpha x x^H + A$

13.10.1 Axiom unit tests

— zhpr.input —

```
)set break resume
)sys rm -f zhpr.output
)spool zhpr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

13.10.2 Axiom help page

— zhpr.help —

```
=====================================================================
zhpr examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

 NAME
      ZHPR - perform the hermitian rank 1 operation
             A := alpha*x*conjg( x' ) + A,

 SYNOPSIS
      SUBROUTINE ZHPR ( UPLO, N, ALPHA, X, INCX, AP )

          DOUBLE      PRECISION ALPHA

          INTEGER     INCX, N

          CHARACTER*1 UPLO

          COMPLEX*16  AP( * ), X( * )

 PURPOSE
      ZHPR    performs the hermitian rank 1 operation

      where alpha is a real scalar, x is an n element vector and A
      is an n by n hermitian matrix, supplied in packed form.

 PARAMETERS
      UPLO   - CHARACTER*1.
             On entry, UPLO specifies whether the upper or lower
             triangular part of the matrix A is supplied in the
             packed array AP as follows:

             UPLO = 'U' or 'u'   The upper triangular part of A is
             supplied in AP.

             UPLO = 'L' or 'l'   The lower triangular part of A is
             supplied in AP.

             Unchanged on exit.
```

```
N      - INTEGER.
       On entry, N specifies the order of the matrix A.  N
       must be at least zero.  Unchanged on exit.

ALPHA  - DOUBLE PRECISION.
       On entry, ALPHA specifies the scalar alpha.
       Unchanged on exit.

X      - COMPLEX*16       array of dimension at least
       ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
       incremented array X must contain the n element vector
       x.  Unchanged on exit.

INCX   - INTEGER.
       On entry, INCX specifies the increment for the ele-
       ments of X. INCX must not be zero.  Unchanged on

       exit.

AP     - COMPLEX*16       array of DIMENSION at least
       ( ( n*( n + 1 ) )/2 ).  Before entry with  UPLO = 'U'
       or 'u', the array AP must contain the upper triangu-
       lar part of the hermitian matrix packed sequentially,
       column by column, so that AP( 1 ) contains a( 1, 1 ),
       AP( 2 ) and AP( 3 ) contain a( 1, 2 ) and a( 2, 2 )
       respectively, and so on. On exit, the array AP is
       overwritten by the upper triangular part of the
       updated matrix.  Before entry with UPLO = 'L' or 'l',
       the array AP must contain the lower triangular part
       of the hermitian matrix packed sequentially, column
       by column, so that AP( 1 ) contains a( 1, 1 ), AP( 2
       ) and AP( 3 ) contain a( 2, 1 ) and a( 3, 1 ) respec-
       tively, and so on. On exit, the array AP is overwrit-
       ten by the lower triangular part of the updated
       matrix.  Note that the imaginary parts of the diago-
       nal elements need not be set, they are assumed to be
       zero, and on exit they are set to zero.
```

———-

13.10.3 fortran code

— zhpr.f —

```
*  ======================================================================
      SUBROUTINE ZHPR(UPLO,N,ALPHA,X,INCX,AP)
```

```
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION ALPHA
      INTEGER INCX,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16 AP(*),X(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP
      INTEGER I,INFO,IX,J,JX,K,KK,KX
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DBLE,DCONJG
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZHPR  ',INFO)
          RETURN
      END IF
```

```
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (ALPHA.EQ.DBLE(ZERO))) RETURN
*
*     Set the start point in X if the increment is not unity.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of the array AP
*     are accessed sequentially with one pass through AP.
*
      KK = 1
      IF (LSAME(UPLO,'U')) THEN
*
*        Form  A  when upper triangle is stored in AP.
*
          IF (INCX.EQ.1) THEN
              DO 20 J = 1,N
                  IF (X(J).NE.ZERO) THEN
                      TEMP = ALPHA*DCONJG(X(J))
                      K = KK
                      DO 10 I = 1,J - 1
                          AP(K) = AP(K) + X(I)*TEMP
                          K = K + 1
   10                 CONTINUE
                      AP(KK+J-1) = DBLE(AP(KK+J-1)) + DBLE(X(J)*TEMP)
                  ELSE
                      AP(KK+J-1) = DBLE(AP(KK+J-1))
                  END IF
                  KK = KK + J
   20         CONTINUE
          ELSE
              JX = KX
              DO 40 J = 1,N
                  IF (X(JX).NE.ZERO) THEN
                      TEMP = ALPHA*DCONJG(X(JX))
                      IX = KX
                      DO 30 K = KK,KK + J - 2
                          AP(K) = AP(K) + X(IX)*TEMP
                          IX = IX + INCX
   30                 CONTINUE
                      AP(KK+J-1) = DBLE(AP(KK+J-1)) + DBLE(X(JX)*TEMP)
                  ELSE
                      AP(KK+J-1) = DBLE(AP(KK+J-1))
                  END IF
```

```
                JX = JX + INCX
                KK = KK + J
   40         CONTINUE
          END IF
      ELSE
*
*        Form  A  when lower triangle is stored in AP.
*
          IF (INCX.EQ.1) THEN
              DO 60 J = 1,N
                  IF (X(J).NE.ZERO) THEN
                      TEMP = ALPHA*DCONJG(X(J))
                      AP(KK) = DBLE(AP(KK)) + DBLE(TEMP*X(J))
                      K = KK + 1
                      DO 50 I = J + 1,N
                          AP(K) = AP(K) + X(I)*TEMP
                          K = K + 1
   50                 CONTINUE
                  ELSE
                      AP(KK) = DBLE(AP(KK))
                  END IF
                  KK = KK + N - J + 1
   60         CONTINUE
          ELSE
              JX = KX
              DO 80 J = 1,N
                  IF (X(JX).NE.ZERO) THEN
                      TEMP = ALPHA*DCONJG(X(JX))
                      AP(KK) = DBLE(AP(KK)) + DBLE(TEMP*X(JX))
                      IX = JX
                      DO 70 K = KK + 1,KK + N - J
                          IX = IX + INCX
                          AP(K) = AP(K) + X(IX)*TEMP
   70                 CONTINUE
                  ELSE
                      AP(KK) = DBLE(AP(KK))
                  END IF
                  JX = JX + INCX
                  KK = KK + N - J + 1
   80         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of ZHPR  .
*
      END
```

———-

13.10.4 lisp code

— BLAS 2 zhpr —

```
(let* ((zero (complex 0.0 0.0)))
  (declare (type (complex double-float) zero))
  (defun zhpr (uplo n alpha x incx ap)
    (declare (type (simple-array (complex double-float) (*)) ap x)
             (type (double-float) alpha)
             (type fixnum incx n)
             (type character uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (x (complex double-float) x-%data% x-%offset%)
         (ap (complex double-float) ap-%data% ap-%offset%))
      (prog ((i 0) (info 0) (ix 0) (j 0) (jx 0) (k 0) (kk 0) (kx 0)
             (temp #C(0.0 0.0)))
        (declare (type fixnum i info ix j jx k kk kx)
                 (type (complex double-float) temp))
        (setf info 0)
        (cond
          ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
           (setf info 1))
          ((< n 0)
           (setf info 2))
          ((= incx 0)
           (setf info 5)))
        (cond
          ((/= info 0)
           (error
            " ** On entry to ~a parameter number ~a had an illegal value~%"
            "ZHPR" info)
           (go end_label)))
        (if (or (= n 0) (= alpha (coerce (realpart zero) 'double-float)))
           (go end_label))
        (cond
          ((<= incx 0)
           (setf kx
                   (f2cl-lib:int-sub 1
                                     (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                       incx))))
          ((/= incx 1)
           (setf kx 1)))
        (setf kk 1)
        (cond
          ((char-equal uplo #\U)
```

```
(cond
  ((= incx 1)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
         ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
          (setf temp
                  (coerce
                   (* alpha
                      (f2cl-lib:dconjg
                       (f2cl-lib:fref x-%data%
                                      (j)
                                      ((1 *))
                                      x-%offset%)))
                   '(complex double-float)))
          (setf k kk)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add j
                                              (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref ap-%data%
                                   (k)
                                   ((1 *))
                                   ap-%offset%)
                    (+
                     (f2cl-lib:fref ap-%data%
                                    (k)
                                    ((1 *))
                                    ap-%offset%)
                     (*
                      (f2cl-lib:fref x-%data%
                                     (i)
                                     ((1 *))
                                     x-%offset%)
                      temp)))
              (setf k (f2cl-lib:int-add k 1))))
          (setf (f2cl-lib:fref ap-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add kk j)
                                 1))
                               ((1 *))
                               ap-%offset%)
                (coerce
                 (+
                  (coerce (realpart
                   (f2cl-lib:fref ap-%data%
                                  ((f2cl-lib:int-sub
```

```
                                        (f2cl-lib:int-add kk j)
                                        1))
                                      ((1 *))
                                      ap-%offset%)) 'double-float)
                      (coerce (realpart
                       (*
                        (f2cl-lib:fref x-%data%
                                       (j)
                                       ((1 *))
                                       x-%offset%)
                        temp)) 'double-float))
                     '(complex double-float))))
            (t
             (setf (f2cl-lib:fref ap-%data%
                                  ((f2cl-lib:int-sub
                                    (f2cl-lib:int-add kk j)
                                    1))
                                  ((1 *))
                                  ap-%offset%)
                   (coerce
                    (coerce (realpart
                     (f2cl-lib:fref ap-%data%
                                    ((f2cl-lib:int-sub
                                      (f2cl-lib:int-add kk j)
                                      1))
                                    ((1 *))
                                    ap-%offset%)) 'double-float)
                    '(complex double-float)))))
         (setf kk (f2cl-lib:int-add kk j)))))
    (t
     (setf jx kx)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (cond
           ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
            (setf temp
                  (coerce
                   (* alpha
                      (f2cl-lib:dconjg
                       (f2cl-lib:fref x-%data%
                                      (jx)
                                      ((1 *))
                                      x-%offset%)))
                   '(complex double-float)))
            (setf ix kx)
            (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                          ((> k
                              (f2cl-lib:int-add kk
                                                j
```

```
                                        (f2cl-lib:int-sub 2)))
                 nil)
      (tagbody
        (setf (f2cl-lib:fref ap-%data%
                             (k)
                             ((1 *))
                             ap-%offset%)
              (+
               (f2cl-lib:fref ap-%data%
                              (k)
                              ((1 *))
                              ap-%offset%)
               (*
                (f2cl-lib:fref x-%data%
                               (ix)
                               ((1 *))
                               x-%offset%)
                temp)))
        (setf ix (f2cl-lib:int-add ix incx))))
    (setf (f2cl-lib:fref ap-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add kk j)
                           1))
                         ((1 *))
                         ap-%offset%)
          (coerce
           (+
            (coerce (realpart
             (f2cl-lib:fref ap-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add kk j)
                              1))
                            ((1 *))
                            ap-%offset%)) 'double-float)
            (coerce (realpart
             (*
              (f2cl-lib:fref x-%data%
                             (jx)
                             ((1 *))
                             x-%offset%)
              temp)) 'double-float))
           '(complex double-float))))
  (t
   (setf (f2cl-lib:fref ap-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add kk j)
                          1))
                        ((1 *))
                        ap-%offset%)
         (coerce
```

```
                        (coerce (realpart
                         (f2cl-lib:fref ap-%data%
                                        ((f2cl-lib:int-sub
                                          (f2cl-lib:int-add kk j)
                                          1))
                                        ((1 *))
                                        ap-%offset%)) 'double-float)
                        '(complex double-float)))))
                (setf jx (f2cl-lib:int-add jx incx))
                (setf kk (f2cl-lib:int-add kk j)))))))
        (t
         (cond
           ((= incx 1)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (cond
                  ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
                   (setf temp
                         (coerce
                          (* alpha
                             (f2cl-lib:dconjg
                              (f2cl-lib:fref x-%data%
                                             (j)
                                             ((1 *))
                                             x-%offset%)))
                          '(complex double-float)))
                   (setf (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%)
                         (coerce
                          (+
                           (coerce (realpart
                            (f2cl-lib:fref ap-%data%
                                           (kk)
                                           ((1 *))
                                           ap-%offset%)) 'double-float)
                           (coerce (realpart
                            (* temp
                               (f2cl-lib:fref x-%data%
                                              (j)
                                              ((1 *))
                                              x-%offset%))) 'double-float))
                          '(complex double-float)))
                   (setf k (f2cl-lib:int-add kk 1))
                   (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                                  (f2cl-lib:int-add i 1))
                                 ((> i n) nil)
                     (tagbody
                       (setf (f2cl-lib:fref ap-%data%
                                            (k)
                                            ((1 *))
```

```
                                    ap-%offset%)
                          (+
                           (f2cl-lib:fref ap-%data%
                                          (k)
                                          ((1 *))
                                          ap-%offset%)
                           (*
                            (f2cl-lib:fref x-%data%
                                           (i)
                                           ((1 *))
                                           x-%offset%)
                            temp)))
                     (setf k (f2cl-lib:int-add k 1)))))
              (t
               (setf (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%)
                       (coerce
                        (coerce (realpart
                         (f2cl-lib:fref ap-%data%
                                        (kk)
                                        ((1 *))
                                        ap-%offset%)) 'double-float)
                        '(complex double-float)))))
            (setf kk
                    (f2cl-lib:int-add
                     (f2cl-lib:int-sub (f2cl-lib:int-add kk n) j)
                     1)))))
       (t
        (setf jx kx)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (cond
              ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
               (setf temp
                       (coerce
                        (* alpha
                           (f2cl-lib:dconjg
                            (f2cl-lib:fref x-%data%
                                           (jx)
                                           ((1 *))
                                           x-%offset%)))
                        '(complex double-float)))
               (setf (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%)
                       (coerce
                        (+
                         (coerce (realpart
                          (f2cl-lib:fref ap-%data%
                                         (kk)
                                         ((1 *))
                                         ap-%offset%)) 'double-float)
```

```
                                (coerce (realpart
                                 (* temp
                                    (f2cl-lib:fref x-%data%
                                                   (jx)
                                                   ((1 *))
                                                   x-%offset%))) 'double-float))
                              '(complex double-float)))
                      (setf ix jx)
                      (f2cl-lib:fdo (k (f2cl-lib:int-add kk 1)
                                     (f2cl-lib:int-add k 1))
                                    ((> k
                                        (f2cl-lib:int-add kk
                                                          n
                                                          (f2cl-lib:int-sub j)))
                                     nil)
                        (tagbody
                          (setf ix (f2cl-lib:int-add ix incx))
                          (setf (f2cl-lib:fref ap-%data%
                                               (k)
                                               ((1 *))
                                               ap-%offset%)
                                (+
                                 (f2cl-lib:fref ap-%data%
                                                (k)
                                                ((1 *))
                                                ap-%offset%)
                                 (*
                                  (f2cl-lib:fref x-%data%
                                                 (ix)
                                                 ((1 *))
                                                 x-%offset%)
                                  temp))))))
                     (t
                      (setf (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%)
                              (coerce
                               (coerce (realpart
                                 (f2cl-lib:fref ap-%data%
                                                (kk)
                                                ((1 *))
                                                ap-%offset%)) 'double-float)
                              '(complex double-float)))))
                    (setf jx (f2cl-lib:int-add jx incx))
                    (setf kk
                            (f2cl-lib:int-add
                             (f2cl-lib:int-sub (f2cl-lib:int-add kk n) j)
                             1))))))))
end_label
       (return (values nil nil nil nil nil nil))))))
```

———-

13.11 zhpr2 - hermitian rank 2 $\alpha xy^H + conjg(\alpha)yx^H + A$

13.11.1 Axiom unit tests

— **zhpr2.input** —

```
)set break resume
)sys rm -f zhpr2.output
)spool zhpr2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

13.11.2 Axiom help page

— **zhpr2.help** —

```
======================================================================
zhpr2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

 NAME
      ZHPR2 - perform the hermitian rank 2 operation
      A := alpha*x*conjg( y' ) + conjg( alpha )*y*conjg( x' ) + A,

 SYNOPSIS
      SUBROUTINE ZHPR2 ( UPLO, N, ALPHA, X, INCX, Y, INCY, AP )

          COMPLEX*16   ALPHA

          INTEGER      INCX, INCY, N

          CHARACTER*1  UPLO
```

```
         COMPLEX*16   AP( * ), X( * ), Y( * )

PURPOSE
     ZHPR2  performs the hermitian rank 2 operation

     where alpha is a scalar, x and y are n element vectors and A
     is an n by n hermitian matrix, supplied in packed form.

PARAMETERS
     UPLO   - CHARACTER*1.
            On entry, UPLO specifies whether the upper or lower
            triangular part of the matrix A is supplied in the
            packed array AP as follows:

            UPLO = 'U' or 'u'   The upper triangular part of A is
            supplied in AP.

            UPLO = 'L' or 'l'   The lower triangular part of A is
            supplied in AP.

            Unchanged on exit.

     N      - INTEGER.
            On entry, N specifies the order of the matrix A.  N
            must be at least zero.  Unchanged on exit.

     ALPHA  - COMPLEX*16      .
            On entry, ALPHA specifies the scalar alpha.
            Unchanged on exit.

     X      - COMPLEX*16       array of dimension at least
            ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
            incremented array X must contain the n element vector
            x.  Unchanged on exit.

     INCX   - INTEGER.
            On entry, INCX specifies the increment for the ele-
            ments of X. INCX must not be zero.  Unchanged on

            exit.

     Y      - COMPLEX*16       array of dimension at least
            ( 1 + ( n - 1 )*abs( INCY ) ).  Before entry, the
            incremented array Y must contain the n element vector
            y.  Unchanged on exit.

     INCY   - INTEGER.
            On entry, INCY specifies the increment for the ele-
            ments of Y. INCY must not be zero.  Unchanged on
```

```
            exit.

      AP     - COMPLEX*16       array of DIMENSION at least
             ( ( n*( n + 1 ) )/2 ).  Before entry with  UPLO = 'U'
             or 'u', the array AP must contain the upper triangu-
             lar part of the hermitian matrix packed sequentially,
             column by column, so that AP( 1 ) contains a( 1, 1 ),
             AP( 2 ) and AP( 3 ) contain a( 1, 2 ) and a( 2, 2 )
             respectively, and so on. On exit, the array AP is
             overwritten by the upper triangular part of the
             updated matrix.  Before entry with UPLO = 'L' or 'l',
             the array AP must contain the lower triangular part
             of the hermitian matrix packed sequentially, column
             by column, so that AP( 1 ) contains a( 1, 1 ), AP( 2
             ) and AP( 3 ) contain a( 2, 1 ) and a( 3, 1 ) respec-
             tively, and so on. On exit, the array AP is overwrit-
             ten by the lower triangular part of the updated
             matrix.  Note that the imaginary parts of the diago-
             nal elements need not be set, they are assumed to be
             zero, and on exit they are set to zero.
```

———-

13.11.3 fortran code

— zhpr2.f —

```
*  ===================================================================
      SUBROUTINE ZHPR2(UPLO,N,ALPHA,X,INCX,Y,INCY,AP)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX*16 ALPHA
      INTEGER INCX,INCY,N
      CHARACTER UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16 AP(*),X(*),Y(*)
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
```

```
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP1,TEMP2
      INTEGER I,INFO,IX,IY,J,JX,JY,K,KK,KX,KY
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DBLE,DCONJG
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (N.LT.0) THEN
          INFO = 2
      ELSE IF (INCX.EQ.0) THEN
          INFO = 5
      ELSE IF (INCY.EQ.0) THEN
          INFO = 7
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZHPR2 ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (ALPHA.EQ.ZERO)) RETURN
*
*     Set up the start points in X and Y if the increments are not both
*     unity.
*
      IF ((INCX.NE.1) .OR. (INCY.NE.1)) THEN
          IF (INCX.GT.0) THEN
              KX = 1
          ELSE
              KX = 1 - (N-1)*INCX
          END IF
          IF (INCY.GT.0) THEN
              KY = 1
```

```
         ELSE
             KY = 1 - (N-1)*INCY
         END IF
         JX = KX
         JY = KY
     END IF
*
*    Start the operations. In this version the elements of the array AP
*    are accessed sequentially with one pass through AP.
*
     KK = 1
     IF (LSAME(UPLO,'U')) THEN
*
*       Form  A  when upper triangle is stored in AP.
*
         IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
             DO 20 J = 1,N
                 IF ((X(J).NE.ZERO) .OR. (Y(J).NE.ZERO)) THEN
                     TEMP1 = ALPHA*DCONJG(Y(J))
                     TEMP2 = DCONJG(ALPHA*X(J))
                     K = KK
                     DO 10 I = 1,J - 1
                         AP(K) = AP(K) + X(I)*TEMP1 + Y(I)*TEMP2
                         K = K + 1
  10                 CONTINUE
                     AP(KK+J-1) = DBLE(AP(KK+J-1)) +
    +                             DBLE(X(J)*TEMP1+Y(J)*TEMP2)
                 ELSE
                     AP(KK+J-1) = DBLE(AP(KK+J-1))
                 END IF
                 KK = KK + J
  20         CONTINUE
         ELSE
             DO 40 J = 1,N
                 IF ((X(JX).NE.ZERO) .OR. (Y(JY).NE.ZERO)) THEN
                     TEMP1 = ALPHA*DCONJG(Y(JY))
                     TEMP2 = DCONJG(ALPHA*X(JX))
                     IX = KX
                     IY = KY
                     DO 30 K = KK,KK + J - 2
                         AP(K) = AP(K) + X(IX)*TEMP1 + Y(IY)*TEMP2
                         IX = IX + INCX
                         IY = IY + INCY
  30                 CONTINUE
                     AP(KK+J-1) = DBLE(AP(KK+J-1)) +
    +                             DBLE(X(JX)*TEMP1+Y(JY)*TEMP2)
                 ELSE
                     AP(KK+J-1) = DBLE(AP(KK+J-1))
                 END IF
                 JX = JX + INCX
```

```
                JY = JY + INCY
                KK = KK + J
   40         CONTINUE
          END IF
      ELSE
*
*        Form  A  when lower triangle is stored in AP.
*
          IF ((INCX.EQ.1) .AND. (INCY.EQ.1)) THEN
              DO 60 J = 1,N
                  IF ((X(J).NE.ZERO) .OR. (Y(J).NE.ZERO)) THEN
                      TEMP1 = ALPHA*DCONJG(Y(J))
                      TEMP2 = DCONJG(ALPHA*X(J))
                      AP(KK) = DBLE(AP(KK)) +
     +                         DBLE(X(J)*TEMP1+Y(J)*TEMP2)
                      K = KK + 1
                      DO 50 I = J + 1,N
                          AP(K) = AP(K) + X(I)*TEMP1 + Y(I)*TEMP2
                          K = K + 1
   50                 CONTINUE
                  ELSE
                      AP(KK) = DBLE(AP(KK))
                  END IF
                  KK = KK + N - J + 1
   60         CONTINUE
          ELSE
              DO 80 J = 1,N
                  IF ((X(JX).NE.ZERO) .OR. (Y(JY).NE.ZERO)) THEN
                      TEMP1 = ALPHA*DCONJG(Y(JY))
                      TEMP2 = DCONJG(ALPHA*X(JX))
                      AP(KK) = DBLE(AP(KK)) +
     +                         DBLE(X(JX)*TEMP1+Y(JY)*TEMP2)
                      IX = JX
                      IY = JY
                      DO 70 K = KK + 1,KK + N - J
                          IX = IX + INCX
                          IY = IY + INCY
                          AP(K) = AP(K) + X(IX)*TEMP1 + Y(IY)*TEMP2
   70                 CONTINUE
                  ELSE
                      AP(KK) = DBLE(AP(KK))
                  END IF
                  JX = JX + INCX
                  JY = JY + INCY
                  KK = KK + N - J + 1
   80         CONTINUE
          END IF
      END IF
*
      RETURN
```

```
*
*     End of ZHPR2 .
*
      END
```

———-

13.11.4 lisp code

— BLAS 2 zhpr2 —

```
(let* ((zero (complex 0.0 0.0)))
  (declare (type (complex double-float) zero))
  (defun zhpr2 (uplo n alpha x incx y incy ap)
    (declare (type (simple-array (complex double-float) (*)) ap y x)
             (type (complex double-float) alpha)
             (type fixnum incy incx n)
             (type character uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (x (complex double-float) x-%data% x-%offset%)
         (y (complex double-float) y-%data% y-%offset%)
         (ap (complex double-float) ap-%data% ap-%offset%))
      (prog ((i 0) (info 0) (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (k 0) (kk 0)
             (kx 0) (ky 0) (temp1 #C(0.0 0.0)) (temp2 #C(0.0 0.0)))
        (declare (type fixnum i info ix iy j jx jy k kk kx ky)
                 (type (complex double-float) temp1 temp2))
        (setf info 0)
        (cond
          ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
           (setf info 1))
          ((< n 0)
           (setf info 2))
          ((= incx 0)
           (setf info 5))
          ((= incy 0)
           (setf info 7)))
        (cond
          ((/= info 0)
           (error
            " ** On entry to ~a parameter number ~a had an illegal value~%"
            "ZHPR2" info)
           (go end_label)))
        (if (or (= n 0) (= alpha zero)) (go end_label))
        (cond
          ((or (/= incx 1) (/= incy 1))
           (cond
```

```
      ((> incx 0)
       (setf kx 1))
      (t
       (setf kx
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1)
                                  incx)))))
    (cond
      ((> incy 0)
       (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1)
                                  incy)))))
    (setf jx kx)
    (setf jy ky)))
  (setf kk 1)
  (cond
    ((char-equal uplo #\U)
     (cond
       ((and (= incx 1) (= incy 1))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (cond
              ((or (/= (f2cl-lib:fref x (j) ((1 *))) zero)
                   (/= (f2cl-lib:fref y (j) ((1 *))) zero))
               (setf temp1
                       (* alpha
                          (f2cl-lib:dconjg
                           (f2cl-lib:fref y-%data%
                                          (j)
                                          ((1 *))
                                          y-%offset%))))
               (setf temp2
                       (coerce
                        (f2cl-lib:dconjg
                         (* alpha
                            (f2cl-lib:fref x-%data%
                                           (j)
                                           ((1 *))
                                           x-%offset%)))
                        '(complex double-float)))
               (setf k kk)
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i
                                 (f2cl-lib:int-add j
```

```
                                        (f2cl-lib:int-sub 1)))
                 nil)
        (tagbody
          (setf (f2cl-lib:fref ap-%data%
                               (k)
                               ((1 *))
                               ap-%offset%)
                (+
                 (f2cl-lib:fref ap-%data%
                                (k)
                                ((1 *))
                                ap-%offset%)
                 (*
                  (f2cl-lib:fref x-%data%
                                 (i)
                                 ((1 *))
                                 x-%offset%)
                  temp1)
                 (*
                  (f2cl-lib:fref y-%data%
                                 (i)
                                 ((1 *))
                                 y-%offset%)
                  temp2)))
          (setf k (f2cl-lib:int-add k 1))))
      (setf (f2cl-lib:fref ap-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add kk j)
                             1))
                           ((1 *))
                           ap-%offset%)
            (coerce
             (+
              (coerce (realpart
               (f2cl-lib:fref ap-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add kk j)
                                1))
                              ((1 *))
                              ap-%offset%)) 'double-float)
              (coerce (realpart
               (+
                (*
                 (f2cl-lib:fref x-%data%
                                (j)
                                ((1 *))
                                x-%offset%)
                 temp1)
                (*
                 (f2cl-lib:fref y-%data%
```

```
                                        (j)
                                        ((1 *))
                                        y-%offset%)
                          temp2))) 'double-float))
                     '(complex double-float))))
            (t
             (setf (f2cl-lib:fref ap-%data%
                                  ((f2cl-lib:int-sub
                                    (f2cl-lib:int-add kk j)
                                    1))
                                  ((1 *))
                                  ap-%offset%)
                   (coerce
                    (coerce (realpart
                     (f2cl-lib:fref ap-%data%
                                    ((f2cl-lib:int-sub
                                      (f2cl-lib:int-add kk j)
                                      1))
                                    ((1 *))
                                    ap-%offset%)) 'double-float)
                    '(complex double-float)))))
        (setf kk (f2cl-lib:int-add kk j)))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((or (/= (f2cl-lib:fref x (jx) ((1 *))) zero)
            (/= (f2cl-lib:fref y (jy) ((1 *))) zero))
        (setf temp1
                (* alpha
                   (f2cl-lib:dconjg
                    (f2cl-lib:fref y-%data%
                                   (jy)
                                   ((1 *))
                                   y-%offset%))))
        (setf temp2
                (coerce
                 (f2cl-lib:dconjg
                  (* alpha
                     (f2cl-lib:fref x-%data%
                                    (jx)
                                    ((1 *))
                                    x-%offset%)))
                 '(complex double-float)))
        (setf ix kx)
        (setf iy ky)
        (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                      ((> k
                          (f2cl-lib:int-add kk
```

```
                                        j
                                        (f2cl-lib:int-sub 2)))
                 nil)
      (tagbody
        (setf (f2cl-lib:fref ap-%data%
                             (k)
                             ((1 *))
                             ap-%offset%)
              (+
               (f2cl-lib:fref ap-%data%
                              (k)
                              ((1 *))
                              ap-%offset%)
               (*
                (f2cl-lib:fref x-%data%
                               (ix)
                               ((1 *))
                               x-%offset%)
                temp1)
               (*
                (f2cl-lib:fref y-%data%
                               (iy)
                               ((1 *))
                               y-%offset%)
                temp2)))
        (setf ix (f2cl-lib:int-add ix incx))
        (setf iy (f2cl-lib:int-add iy incy))))
    (setf (f2cl-lib:fref ap-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add kk j)
                           1))
                         ((1 *))
                         ap-%offset%)
          (coerce
           (+
            (coerce (realpart
             (f2cl-lib:fref ap-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add kk j)
                              1))
                            ((1 *))
                            ap-%offset%)) 'double-float)
            (coerce (realpart
             (+
              (*
               (f2cl-lib:fref x-%data%
                              (jx)
                              ((1 *))
                              x-%offset%)
               temp1)
```

```
                              (*
                               (f2cl-lib:fref y-%data%
                                              (jy)
                                              ((1 *))
                                              y-%offset%)
                               temp2))) 'double-float))
                         '(complex double-float))))
                 (t
                  (setf (f2cl-lib:fref ap-%data%
                                       ((f2cl-lib:int-sub
                                         (f2cl-lib:int-add kk j)
                                         1))
                                       ((1 *))
                                       ap-%offset%)
                        (coerce
                         (coerce (realpart
                          (f2cl-lib:fref ap-%data%
                                         ((f2cl-lib:int-sub
                                           (f2cl-lib:int-add kk j)
                                           1))
                                         ((1 *))
                                         ap-%offset%)) 'double-float)
                         '(complex double-float)))))
                (setf jx (f2cl-lib:int-add jx incx))
                (setf jy (f2cl-lib:int-add jy incy))
                (setf kk (f2cl-lib:int-add kk j)))))))
        (t
         (cond
           ((and (= incx 1) (= incy 1))
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (cond
                  ((or (/= (f2cl-lib:fref x (j) ((1 *))) zero)
                       (/= (f2cl-lib:fref y (j) ((1 *))) zero))
                   (setf temp1
                         (* alpha
                            (f2cl-lib:dconjg
                             (f2cl-lib:fref y-%data%
                                            (j)
                                            ((1 *))
                                            y-%offset%))))
                   (setf temp2
                         (coerce
                          (f2cl-lib:dconjg
                           (* alpha
                              (f2cl-lib:fref x-%data%
                                             (j)
                                             ((1 *))
                                             x-%offset%)))
```

```
          '(complex double-float)))
(setf (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%)
        (coerce
         (+
          (coerce (realpart
           (f2cl-lib:fref ap-%data%
                          (kk)
                          ((1 *))
                          ap-%offset%)) 'double-float)
          (coerce (realpart
           (+
            (*
             (f2cl-lib:fref x-%data%
                            (j)
                            ((1 *))
                            x-%offset%)
             temp1)
            (*
             (f2cl-lib:fref y-%data%
                            (j)
                            ((1 *))
                            y-%offset%)
             temp2))) 'double-float))
          '(complex double-float)))
(setf k (f2cl-lib:int-add kk 1))
(f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
               (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref ap-%data%
                         (k)
                         ((1 *))
                         ap-%offset%)
            (+
             (f2cl-lib:fref ap-%data%
                            (k)
                            ((1 *))
                            ap-%offset%)
             (*
              (f2cl-lib:fref x-%data%
                             (i)
                             ((1 *))
                             x-%offset%)
              temp1)
             (*
              (f2cl-lib:fref y-%data%
                             (i)
                             ((1 *))
                             y-%offset%)
              temp2)))
```

```
                (setf k (f2cl-lib:int-add k 1)))))
            (t
             (setf (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%)
                     (coerce
                      (coerce (realpart
                       (f2cl-lib:fref ap-%data%
                                      (kk)
                                      ((1 *))
                                      ap-%offset%)) 'double-float)
                      '(complex double-float)))))
          (setf kk
                  (f2cl-lib:int-add
                   (f2cl-lib:int-sub (f2cl-lib:int-add kk n) j)
                   1)))))
      (t
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (cond
             ((or (/= (f2cl-lib:fref x (jx) ((1 *))) zero)
                  (/= (f2cl-lib:fref y (jy) ((1 *))) zero))
              (setf temp1
                      (* alpha
                         (f2cl-lib:dconjg
                          (f2cl-lib:fref y-%data%
                                         (jy)
                                         ((1 *))
                                         y-%offset%))))
              (setf temp2
                      (coerce
                       (f2cl-lib:dconjg
                        (* alpha
                           (f2cl-lib:fref x-%data%
                                          (jx)
                                          ((1 *))
                                          x-%offset%)))
                       '(complex double-float)))
              (setf (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%)
                      (coerce
                       (+
                        (coerce (realpart
                         (f2cl-lib:fref ap-%data%
                                        (kk)
                                        ((1 *))
                                        ap-%offset%)) 'double-float)
                        (coerce (realpart
                         (+
                          (*
                           (f2cl-lib:fref x-%data%
                                          (jx)
```

```
                                ((1 *))
                                x-%offset%)
                temp1)
               (*
                (f2cl-lib:fref y-%data%
                               (jy)
                               ((1 *))
                               y-%offset%)
                temp2))) 'double-float))
            '(complex double-float)))
 (setf ix jx)
 (setf iy jy)
 (f2cl-lib:fdo (k (f2cl-lib:int-add kk 1)
                 (f2cl-lib:int-add k 1))
                ((> k
                    (f2cl-lib:int-add kk
                                      n
                                      (f2cl-lib:int-sub j)))
                 nil)
   (tagbody
     (setf ix (f2cl-lib:int-add ix incx))
     (setf iy (f2cl-lib:int-add iy incy))
     (setf (f2cl-lib:fref ap-%data%
                          (k)
                          ((1 *))
                          ap-%offset%)
              (+
               (f2cl-lib:fref ap-%data%
                              (k)
                              ((1 *))
                              ap-%offset%)
               (*
                (f2cl-lib:fref x-%data%
                               (ix)
                               ((1 *))
                               x-%offset%)
                temp1)
               (*
                (f2cl-lib:fref y-%data%
                               (iy)
                               ((1 *))
                               y-%offset%)
                temp2))))))
(t
 (setf (f2cl-lib:fref ap-%data% (kk) ((1 *)) ap-%offset%)
         (coerce
          (coerce (realpart
           (f2cl-lib:fref ap-%data%
                          (kk)
                          ((1 *))
```

```
                                          ap-%offset%)) 'double-float)
                            '(complex double-float)))))
                 (setf jx (f2cl-lib:int-add jx incx))
                 (setf jy (f2cl-lib:int-add jy incy))
                 (setf kk
                         (f2cl-lib:int-add
                          (f2cl-lib:int-sub (f2cl-lib:int-add kk n) j)
                          1)))))))
 end_label
        (return (values nil nil nil nil nil nil nil nil))))))
```

———-

13.12 ztbmv - band matrix-vector Ax or $A^T x$ or $A^H x$

13.12.1 Axiom unit tests

— **ztbmv.input** —

```
)set break resume
)sys rm -f ztbmv.output
)spool ztbmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

13.12.2 Axiom help page

— **ztbmv.help** —

```
======================================================================
ztbmv examples
======================================================================

======================================================================
Man Page Details
======================================================================
```

```
NAME
     ZTBMV - perform one of the matrix-vector operations
     x := A*x, or x := A'*x, or x := conjg( A' )*x,

SYNOPSIS
     SUBROUTINE ZTBMV ( UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX
                      )

         INTEGER      INCX, K, LDA, N

         CHARACTER*1  DIAG, TRANS, UPLO

         COMPLEX*16   A( LDA, * ), X( * )

PURPOSE
     ZTBMV  performs one of the matrix-vector operations

     where x is an n element vector and  A is an n by n unit, or
     non-unit, upper or lower triangular band matrix, with ( k +
     1 ) diagonals.

PARAMETERS
     UPLO   - CHARACTER*1.
            On entry, UPLO specifies whether the matrix is an
            upper or lower triangular matrix as follows:

            UPLO = 'U' or 'u'   A is an upper triangular matrix.

            UPLO = 'L' or 'l'   A is a lower triangular matrix.

            Unchanged on exit.

     TRANS  - CHARACTER*1.
            On entry, TRANS specifies the operation to be per-
            formed as follows:

            TRANS = 'N' or 'n'   x := A*x.

            TRANS = 'T' or 't'   x := A'*x.

            TRANS = 'C' or 'c'   x := conjg( A' )*x.

            Unchanged on exit.

     DIAG   - CHARACTER*1.
            On entry, DIAG specifies whether or not A is unit
            triangular as follows:

            DIAG = 'U' or 'u'   A is assumed to be unit triangu-
            lar.
```

```
         DIAG = 'N' or 'n'   A is not assumed to be unit tri-
         angular.

         Unchanged on exit.

N        - INTEGER.
         On entry, N specifies the order of the matrix A.  N
         must be at least zero.  Unchanged on exit.

K        - INTEGER.
         On entry with UPLO = 'U' or 'u', K specifies the
         number of super-diagonals of the matrix A.  On entry
         with UPLO = 'L' or 'l', K specifies the number of
         sub-diagonals of the matrix A.  K must satisfy  0
         .le. K.  Unchanged on exit.

A        - COMPLEX*16       array of DIMENSION ( LDA, n ).
         Before entry with UPLO = 'U' or 'u', the leading ( k
         + 1 ) by n part of the array A must contain the upper
         triangular band part of the matrix of coefficients,
         supplied column by column, with the leading diagonal
         of the matrix in row ( k + 1 ) of the array, the
         first super-diagonal starting at position 2 in row k,
         and so on. The top left k by k triangle of the array
         A is not referenced.  The following program segment
         will transfer an upper triangular band matrix from
         conventional full matrix storage to band storage:

         DO 20, J = 1, N M = K + 1 - J DO 10, I = MAX( 1, J -
         K ), J A( M + I, J ) = matrix( I, J ) 10    CONTINUE
         20 CONTINUE

         Before entry with UPLO = 'L' or 'l', the leading ( k
         + 1 ) by n part of the array A must contain the lower
         triangular band part of the matrix of coefficients,
         supplied column by column, with the leading diagonal
         of the matrix in row 1 of the array, the first sub-
         diagonal starting at position 1 in row 2, and so on.
         The bottom right k by k triangle of the array A is
         not referenced.  The following program segment will
         transfer a lower triangular band matrix from conven-
         tional full matrix storage to band storage:

         DO 20, J = 1, N M = 1 - J DO 10, I = J, MIN( N, J + K
         ) A( M + I, J ) = matrix( I, J ) 10    CONTINUE 20
         CONTINUE

         Note that when DIAG = 'U' or 'u' the elements of the
         array A corresponding to the diagonal elements of the
```

```
            matrix are not referenced, but are assumed to be
            unity.  Unchanged on exit.

      LDA    - INTEGER.
             On entry, LDA specifies the first dimension of A as
             declared in the calling (sub) program. LDA must be at
             least ( k + 1 ).  Unchanged on exit.

      X      - COMPLEX*16       array of dimension at least
             ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
             incremented array X must contain the n element vector
             x. On exit, X is overwritten with the transformed
             vector x.

      INCX   - INTEGER.
             On entry, INCX specifies the increment for the ele-
             ments of X. INCX must not be zero.  Unchanged on
             exit.
```

13.12.3 fortran code

— ztbmv.f —

```
*  ==================================================================
      SUBROUTINE ZTBMV(UPLO,TRANS,DIAG,N,K,A,LDA,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,K,LDA,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),X(*)
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
```

```
*     .. Local Scalars ..
      COMPLEX*16 TEMP
      INTEGER I,INFO,IX,J,JX,KPLUS1,KX,L
      LOGICAL NOCONJ,NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DCONJG,MAX,MIN
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (K.LT.0) THEN
          INFO = 5
      ELSE IF (LDA.LT. (K+1)) THEN
          INFO = 7
      ELSE IF (INCX.EQ.0) THEN
          INFO = 9
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZTBMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOCONJ = LSAME(TRANS,'T')
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX   too small for descending loops.
*
```

```
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
      IF (LSAME(TRANS,'N')) THEN
*
*         Form  x := A*x.
*
          IF (LSAME(UPLO,'U')) THEN
              KPLUS1 = K + 1
              IF (INCX.EQ.1) THEN
                  DO 20 J = 1,N
                      IF (X(J).NE.ZERO) THEN
                          TEMP = X(J)
                          L = KPLUS1 - J
                          DO 10 I = MAX(1,J-K),J - 1
                              X(I) = X(I) + TEMP*A(L+I,J)
   10                     CONTINUE
                          IF (NOUNIT) X(J) = X(J)*A(KPLUS1,J)
                      END IF
   20             CONTINUE
              ELSE
                  JX = KX
                  DO 40 J = 1,N
                      IF (X(JX).NE.ZERO) THEN
                          TEMP = X(JX)
                          IX = KX
                          L = KPLUS1 - J
                          DO 30 I = MAX(1,J-K),J - 1
                              X(IX) = X(IX) + TEMP*A(L+I,J)
                              IX = IX + INCX
   30                     CONTINUE
                          IF (NOUNIT) X(JX) = X(JX)*A(KPLUS1,J)
                      END IF
                      JX = JX + INCX
                      IF (J.GT.K) KX = KX + INCX
   40             CONTINUE
              END IF
          ELSE
              IF (INCX.EQ.1) THEN
                  DO 60 J = N,1,-1
                      IF (X(J).NE.ZERO) THEN
                          TEMP = X(J)
                          L = 1 - J
                          DO 50 I = MIN(N,J+K),J + 1,-1
```

```
                            X(I) = X(I) + TEMP*A(L+I,J)
   50                    CONTINUE
                         IF (NOUNIT) X(J) = X(J)*A(1,J)
                     END IF
   60             CONTINUE
              ELSE
                 KX = KX + (N-1)*INCX
                 JX = KX
                 DO 80 J = N,1,-1
                     IF (X(JX).NE.ZERO) THEN
                         TEMP = X(JX)
                         IX = KX
                         L = 1 - J
                         DO 70 I = MIN(N,J+K),J + 1,-1
                             X(IX) = X(IX) + TEMP*A(L+I,J)
                             IX = IX - INCX
   70                    CONTINUE
                         IF (NOUNIT) X(JX) = X(JX)*A(1,J)
                     END IF
                     JX = JX - INCX
                     IF ((N-J).GE.K) KX = KX - INCX
   80             CONTINUE
              END IF
          END IF
      ELSE
*
*        Form  x := A**T*x  or  x := A**H*x.
*
          IF (LSAME(UPLO,'U')) THEN
              KPLUS1 = K + 1
              IF (INCX.EQ.1) THEN
                  DO 110 J = N,1,-1
                      TEMP = X(J)
                      L = KPLUS1 - J
                      IF (NOCONJ) THEN
                          IF (NOUNIT) TEMP = TEMP*A(KPLUS1,J)
                          DO 90 I = J - 1,MAX(1,J-K),-1
                              TEMP = TEMP + A(L+I,J)*X(I)
   90                     CONTINUE
                      ELSE
                          IF (NOUNIT) TEMP = TEMP*DCONJG(A(KPLUS1,J))
                          DO 100 I = J - 1,MAX(1,J-K),-1
                              TEMP = TEMP + DCONJG(A(L+I,J))*X(I)
  100                     CONTINUE
                      END IF
                      X(J) = TEMP
  110             CONTINUE
              ELSE
                  KX = KX + (N-1)*INCX
                  JX = KX
```

```
                DO 140 J = N,1,-1
                    TEMP = X(JX)
                    KX = KX - INCX
                    IX = KX
                    L = KPLUS1 - J
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*A(KPLUS1,J)
                        DO 120 I = J - 1,MAX(1,J-K),-1
                            TEMP = TEMP + A(L+I,J)*X(IX)
                            IX = IX - INCX
 120                    CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*DCONJG(A(KPLUS1,J))
                        DO 130 I = J - 1,MAX(1,J-K),-1
                            TEMP = TEMP + DCONJG(A(L+I,J))*X(IX)
                            IX = IX - INCX
 130                    CONTINUE
                    END IF
                    X(JX) = TEMP
                    JX = JX - INCX
 140            CONTINUE
            END IF
        ELSE
            IF (INCX.EQ.1) THEN
                DO 170 J = 1,N
                    TEMP = X(J)
                    L = 1 - J
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*A(1,J)
                        DO 150 I = J + 1,MIN(N,J+K)
                            TEMP = TEMP + A(L+I,J)*X(I)
 150                    CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*DCONJG(A(1,J))
                        DO 160 I = J + 1,MIN(N,J+K)
                            TEMP = TEMP + DCONJG(A(L+I,J))*X(I)
 160                    CONTINUE
                    END IF
                    X(J) = TEMP
 170            CONTINUE
            ELSE
                JX = KX
                DO 200 J = 1,N
                    TEMP = X(JX)
                    KX = KX + INCX
                    IX = KX
                    L = 1 - J
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*A(1,J)
                        DO 180 I = J + 1,MIN(N,J+K)
```

```
                            TEMP = TEMP + A(L+I,J)*X(IX)
                            IX = IX + INCX
  180                   CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*DCONJG(A(1,J))
                        DO 190 I = J + 1,MIN(N,J+K)
                            TEMP = TEMP + DCONJG(A(L+I,J))*X(IX)
                            IX = IX + INCX
  190                   CONTINUE
                    END IF
                    X(JX) = TEMP
                    JX = JX + INCX
  200             CONTINUE
                END IF
            END IF
        END IF
*
      RETURN
*
*     End of ZTBMV .
*
      END
```

———-

13.12.4 lisp code

— BLAS 2 ztbmv —

```
(let* ((zero (complex 0.0 0.0)))
  (declare (type (complex double-float) zero))
  (defun ztbmv (uplo trans diag n k a lda x incx)
    (declare (type (simple-array (complex double-float) (*)) x a)
             (type fixnum incx lda k n)
             (type character diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a (complex double-float) a-%data% a-%offset%)
         (x (complex double-float) x-%data% x-%offset%))
      (prog ((noconj nil) (nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0)
             (kplus1 0) (kx 0) (l 0) (temp #C(0.0 0.0)))
        (declare (type (member t nil) noconj nounit)
                 (type fixnum i info ix j jx kplus1 kx l)
                 (type (complex double-float) temp))
        (setf info 0)
```

```
(cond
  ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
   (setf info 1))
  ((and (not (char-equal trans #\N))
        (not (char-equal trans #\T))
        (not (char-equal trans #\C)))
   (setf info 2))
  ((and (not (char-equal diag #\U)) (not (char-equal diag #\N)))
   (setf info 3))
  ((< n 0)
   (setf info 4))
  ((< k 0)
   (setf info 5))
  ((< lda (f2cl-lib:int-add k 1))
   (setf info 7))
  ((= incx 0)
   (setf info 9)))
(cond
  ((/= info 0)
   (error
    " ** On entry to ~a parameter number ~a had an illegal value~%"
    "ZTBMV" info)
   (go end_label)))
(if (= n 0) (go end_label))
(setf noconj (char-equal trans #\T))
(setf nounit (char-equal diag #\N))
(cond
  ((<= incx 0)
   (setf kx
           (f2cl-lib:int-sub 1
                             (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                               incx))))
  ((/= incx 1)
   (setf kx 1)))
(cond
  ((char-equal trans #\N)
   (cond
     ((char-equal uplo #\U)
      (setf kplus1 (f2cl-lib:int-add k 1))
      (cond
        ((= incx 1)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (cond
               ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
                (setf temp
                        (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                (setf l (f2cl-lib:int-sub kplus1 j))
                (f2cl-lib:fdo (i
```

```
                                (max (the fixnum 1)
                                     (the fixnum
                                          (f2cl-lib:int-add j
                                                            (f2cl-lib:int-sub
                                                             k))))
                                (f2cl-lib:int-add i 1))
                               ((> i
                                   (f2cl-lib:int-add j
                                                     (f2cl-lib:int-sub
                                                      1)))
                                nil)
                      (tagbody
                        (setf (f2cl-lib:fref x-%data%
                                             (i)
                                             ((1 *))
                                             x-%offset%)
                              (+
                               (f2cl-lib:fref x-%data%
                                              (i)
                                              ((1 *))
                                              x-%offset%)
                               (* temp
                                  (f2cl-lib:fref a-%data%
                                                 ((f2cl-lib:int-add l i)
                                                  j)
                                                 ((1 lda) (1 *))
                                                 a-%offset%))))))
                    (if nounit
                        (setf (f2cl-lib:fref x-%data%
                                             (j)
                                             ((1 *))
                                             x-%offset%)
                              (*
                               (f2cl-lib:fref x-%data%
                                              (j)
                                              ((1 *))
                                              x-%offset%)
                               (f2cl-lib:fref a-%data%
                                              (kplus1 j)
                                              ((1 lda) (1 *))
                                              a-%offset%))))))))
            (t
             (setf jx kx)
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (cond
                   ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
                    (setf temp
                            (f2cl-lib:fref x-%data%
```

```
                          (jx)
                          ((1 *))
                          x-%offset%))
    (setf ix kx)
    (setf l (f2cl-lib:int-sub kplus1 j))
    (f2cl-lib:fdo (i
                   (max (the fixnum 1)
                        (the fixnum
                             (f2cl-lib:int-add j
                                               (f2cl-lib:int-sub
                                                k))))
                   (f2cl-lib:int-add i 1))
                  ((> i
                      (f2cl-lib:int-add j
                                        (f2cl-lib:int-sub
                                         1)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref x-%data%
                             (ix)
                             ((1 *))
                             x-%offset%)
              (+
               (f2cl-lib:fref x-%data%
                              (ix)
                              ((1 *))
                              x-%offset%)
               (* temp
                  (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add l i)
                                  j)
                                 ((1 lda) (1 *))
                                 a-%offset%))))
        (setf ix (f2cl-lib:int-add ix incx))))
    (if nounit
        (setf (f2cl-lib:fref x-%data%
                             (jx)
                             ((1 *))
                             x-%offset%)
              (*
               (f2cl-lib:fref x-%data%
                              (jx)
                              ((1 *))
                              x-%offset%)
               (f2cl-lib:fref a-%data%
                              (kplus1 j)
                              ((1 lda) (1 *))
                              a-%offset%))))))
(setf jx (f2cl-lib:int-add jx incx))
(if (> j k) (setf kx (f2cl-lib:int-add kx incx))))))))
```

```
(t
 (cond
   ((= incx 1)
    (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                  ((> j 1) nil)
      (tagbody
        (cond
          ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
           (setf temp
                   (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
           (setf l (f2cl-lib:int-sub 1 j))
           (f2cl-lib:fdo (i
                          (min (the fixnum n)
                               (the fixnum
                                    (f2cl-lib:int-add j k)))
                          (f2cl-lib:int-add i
                                            (f2cl-lib:int-sub 1)))
                         ((> i (f2cl-lib:int-add j 1)) nil)
             (tagbody
               (setf (f2cl-lib:fref x-%data%
                                    (i)
                                    ((1 *))
                                    x-%offset%)
                     (+
                      (f2cl-lib:fref x-%data%
                                     (i)
                                     ((1 *))
                                     x-%offset%)
                      (* temp
                         (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-add l i)
                                         j)
                                        ((1 lda) (1 *))
                                        a-%offset%))))))
           (if nounit
               (setf (f2cl-lib:fref x-%data%
                                    (j)
                                    ((1 *))
                                    x-%offset%)
                     (*
                      (f2cl-lib:fref x-%data%
                                     (j)
                                     ((1 *))
                                     x-%offset%)
                      (f2cl-lib:fref a-%data%
                                     (1 j)
                                     ((1 lda) (1 *))
                                     a-%offset%)))))))))
   (t
    (setf kx
```

```
        (f2cl-lib:int-add kx
                          (f2cl-lib:int-mul
                           (f2cl-lib:int-sub n 1)
                           incx)))
(setf jx kx)
(f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j 1) nil)
  (tagbody
    (cond
      ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
       (setf temp
               (f2cl-lib:fref x-%data%
                              (jx)
                              ((1 *))
                              x-%offset%))
       (setf ix kx)
       (setf l (f2cl-lib:int-sub 1 j))
       (f2cl-lib:fdo (i
                      (min (the fixnum n)
                           (the fixnum
                                (f2cl-lib:int-add j k)))
                      (f2cl-lib:int-add i
                                        (f2cl-lib:int-sub 1)))
                     ((> i (f2cl-lib:int-add j 1)) nil)
         (tagbody
           (setf (f2cl-lib:fref x-%data%
                                (ix)
                                ((1 *))
                                x-%offset%)
                 (+
                  (f2cl-lib:fref x-%data%
                                 (ix)
                                 ((1 *))
                                 x-%offset%)
                  (* temp
                     (f2cl-lib:fref a-%data%
                                    ((f2cl-lib:int-add l i)
                                     j)
                                    ((1 lda) (1 *))
                                    a-%offset%))))
           (setf ix (f2cl-lib:int-sub ix incx))))
       (if nounit
           (setf (f2cl-lib:fref x-%data%
                                (jx)
                                ((1 *))
                                x-%offset%)
                 (*
                  (f2cl-lib:fref x-%data%
                                 (jx)
                                 ((1 *))
```

```
                                                x-%offset%)
                                  (f2cl-lib:fref a-%data%
                                                 (1 j)
                                                 ((1 lda) (1 *))
                                                 a-%offset%))))))
                 (setf jx (f2cl-lib:int-sub jx incx))
                 (if (>= (f2cl-lib:int-sub n j) k)
                     (setf kx (f2cl-lib:int-sub kx incx)))))))))
        (t
         (cond
           ((char-equal uplo #\U)
            (setf kplus1 (f2cl-lib:int-add k 1))
            (cond
              ((= incx 1)
               (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                             ((> j 1) nil)
                 (tagbody
                   (setf temp
                           (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                   (setf l (f2cl-lib:int-sub kplus1 j))
                   (cond
                     (noconj
                      (if nounit
                          (setf temp
                                  (* temp
                                     (f2cl-lib:fref a-%data%
                                                    (kplus1 j)
                                                    ((1 lda) (1 *))
                                                    a-%offset%))))
                      (f2cl-lib:fdo (i
                                     (f2cl-lib:int-add j
                                                       (f2cl-lib:int-sub 1))
                                     (f2cl-lib:int-add i
                                                       (f2cl-lib:int-sub 1)))
                                    ((> i
                                        (max (the fixnum 1)
                                             (the fixnum
                                                  (f2cl-lib:int-add j
                                                      (f2cl-lib:int-sub
                                                                     k)))))
                                     nil)
                        (tagbody
                          (setf temp
                                  (+ temp
                                     (*
                                      (f2cl-lib:fref a-%data%
                                                     ((f2cl-lib:int-add l i)
                                                      j)
                                                     ((1 lda) (1 *))
                                                     a-%offset%)
```

```
                              (f2cl-lib:fref x-%data%
                                             (i)
                                             ((1 *))
                                             x-%offset%)))))))
                 (t
                  (if nounit
                      (setf temp
                              (* temp
                                 (f2cl-lib:dconjg
                                  (f2cl-lib:fref a-%data%
                                                 (kplus1 j)
                                                 ((1 lda) (1 *))
                                                 a-%offset%)))))
                  (f2cl-lib:fdo (i
                                 (f2cl-lib:int-add j
                                                   (f2cl-lib:int-sub 1))
                                 (f2cl-lib:int-add i
                                                   (f2cl-lib:int-sub 1)))
                                ((> i
                                    (max (the fixnum 1)
                                         (the fixnum
                                              (f2cl-lib:int-add j
                                                  (f2cl-lib:int-sub
                                                                 k)))))
                                 nil)
                    (tagbody
                      (setf temp
                              (+ temp
                                 (*
                                  (f2cl-lib:dconjg
                                   (f2cl-lib:fref a-%data%
                                                  ((f2cl-lib:int-add l i)
                                                   j)
                                                  ((1 lda) (1 *))
                                                  a-%offset%))
                                  (f2cl-lib:fref x-%data%
                                                 (i)
                                                 ((1 *))
                                                 x-%offset%))))))))
              (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                     temp))))
        (t
         (setf kx
                 (f2cl-lib:int-add kx
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-sub n 1)
                                    incx)))
         (setf jx kx)
         (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((> j 1) nil)
```

```
(tagbody
  (setf temp
          (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
  (setf kx (f2cl-lib:int-sub kx incx))
  (setf ix kx)
  (setf l (f2cl-lib:int-sub kplus1 j))
  (cond
    (noconj
     (if nounit
         (setf temp
                 (* temp
                    (f2cl-lib:fref a-%data%
                                   (kplus1 j)
                                   ((1 lda) (1 *))
                                   a-%offset%))))
     (f2cl-lib:fdo (i
                    (f2cl-lib:int-add j
                                      (f2cl-lib:int-sub 1))
                    (f2cl-lib:int-add i
                                      (f2cl-lib:int-sub 1)))
                   ((> i
                       (max (the fixnum 1)
                            (the fixnum
                                 (f2cl-lib:int-add j
                                        (f2cl-lib:int-sub
                                                    k)))))
                    nil)
       (tagbody
         (setf temp
                 (+ temp
                    (*
                     (f2cl-lib:fref a-%data%
                                    ((f2cl-lib:int-add l i)
                                     j)
                                    ((1 lda) (1 *))
                                    a-%offset%)
                     (f2cl-lib:fref x-%data%
                                    (ix)
                                    ((1 *))
                                    x-%offset%))))
         (setf ix (f2cl-lib:int-sub ix incx)))))
    (t
     (if nounit
         (setf temp
                 (* temp
                    (f2cl-lib:dconjg
                     (f2cl-lib:fref a-%data%
                                    (kplus1 j)
                                    ((1 lda) (1 *))
                                    a-%offset%)))))
```

```
                (f2cl-lib:fdo (i
                               (f2cl-lib:int-add j
                                                 (f2cl-lib:int-sub 1))
                               (f2cl-lib:int-add i
                                                 (f2cl-lib:int-sub 1)))
                              ((> i
                                  (max (the fixnum 1)
                                       (the fixnum
                                            (f2cl-lib:int-add j
                                                 (f2cl-lib:int-sub
                                                               k)))))
                               nil)
                  (tagbody
                    (setf temp
                            (+ temp
                               (*
                                (f2cl-lib:dconjg
                                 (f2cl-lib:fref a-%data%
                                                ((f2cl-lib:int-add l i)
                                                 j)
                                                ((1 lda) (1 *))
                                                a-%offset%))
                                (f2cl-lib:fref x-%data%
                                               (ix)
                                               ((1 *))
                                               x-%offset%))))
                    (setf ix (f2cl-lib:int-sub ix incx))))))
            (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                    temp)
            (setf jx (f2cl-lib:int-sub jx incx)))))))
    (t
     (cond
       ((= incx 1)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf temp
                    (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
            (setf l (f2cl-lib:int-sub 1 j))
            (cond
              (noconj
               (if nounit
                   (setf temp
                           (* temp
                              (f2cl-lib:fref a-%data%
                                             (1 j)
                                             ((1 lda) (1 *))
                                             a-%offset%))))
               (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                                (f2cl-lib:int-add i 1))
```

```
                    ((> i
                        (min (the fixnum n)
                             (the fixnum
                                  (f2cl-lib:int-add j k))))
                     nil)
        (tagbody
          (setf temp
                  (+ temp
                     (*
                      (f2cl-lib:fref a-%data%
                                     ((f2cl-lib:int-add l i)
                                      j)
                                     ((1 lda) (1 *))
                                     a-%offset%)
                      (f2cl-lib:fref x-%data%
                                     (i)
                                     ((1 *))
                                     x-%offset%)))))))
    (t
     (if nounit
         (setf temp
                 (* temp
                    (f2cl-lib:dconjg
                     (f2cl-lib:fref a-%data%
                                    (1 j)
                                    ((1 lda) (1 *))
                                    a-%offset%)))))
     (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                    (f2cl-lib:int-add i 1))
                   ((> i
                       (min (the fixnum n)
                            (the fixnum
                                 (f2cl-lib:int-add j k))))
                    nil)
        (tagbody
          (setf temp
                  (+ temp
                     (*
                      (f2cl-lib:dconjg
                       (f2cl-lib:fref a-%data%
                                      ((f2cl-lib:int-add l i)
                                       j)
                                      ((1 lda) (1 *))
                                      a-%offset%))
                      (f2cl-lib:fref x-%data%
                                     (i)
                                     ((1 *))
                                     x-%offset%))))))))
(setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
        temp))))
```

```
(t
 (setf jx kx)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf temp
             (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
     (setf kx (f2cl-lib:int-add kx incx))
     (setf ix kx)
     (setf l (f2cl-lib:int-sub 1 j))
     (cond
       (noconj
        (if nounit
            (setf temp
                    (* temp
                       (f2cl-lib:fref a-%data%
                                      (1 j)
                                      ((1 lda) (1 *))
                                      a-%offset%))))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                       (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the fixnum n)
                               (the fixnum
                                    (f2cl-lib:int-add j k))))
                       nil)
          (tagbody
            (setf temp
                    (+ temp
                       (*
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add l i)
                                        j)
                                       ((1 lda) (1 *))
                                       a-%offset%)
                        (f2cl-lib:fref x-%data%
                                       (ix)
                                       ((1 *))
                                       x-%offset%))))
            (setf ix (f2cl-lib:int-add ix incx)))))
       (t
        (if nounit
            (setf temp
                    (* temp
                       (f2cl-lib:dconjg
                        (f2cl-lib:fref a-%data%
                                       (1 j)
                                       ((1 lda) (1 *))
                                       a-%offset%)))))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
```

```
                                        (f2cl-lib:int-add i 1))
                                       ((> i
                                           (min (the fixnum n)
                                                (the fixnum
                                                     (f2cl-lib:int-add j k))))
                                        nil)
                              (tagbody
                                (setf temp
                                        (+ temp
                                           (*
                                            (f2cl-lib:dconjg
                                             (f2cl-lib:fref a-%data%
                                                            ((f2cl-lib:int-add l i)
                                                             j)
                                                            ((1 lda) (1 *))
                                                            a-%offset%))
                                            (f2cl-lib:fref x-%data%
                                                           (ix)
                                                           ((1 *))
                                                           x-%offset%))))
                                (setf ix (f2cl-lib:int-add ix incx))))))
                        (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                                temp)
                        (setf jx (f2cl-lib:int-add jx incx)))))))))
 end_label
        (return (values nil nil nil nil nil nil nil nil nil))))))
```

———-

13.13 ztbsv - solve equations $Ax = b$ or $A^T x = b$ or $A^H x = b$

13.13.1 Axiom unit tests

— **ztbsv.input** —

```
)set break resume
)sys rm -f ztbsv.output
)spool ztbsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

13.13.2 Axiom help page

— ztbsv.help —

```
==========================================================================
ztbsv examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

 NAME
      ZTBSV - solve one of the systems of equations
      A*x = b, or A'*x = b, or conjg( A' )*x = b,

 SYNOPSIS
      SUBROUTINE ZTBSV ( UPLO, TRANS, DIAG, N, K, A, LDA, X, INCX
                       )

          INTEGER      INCX, K, LDA, N

          CHARACTER*1  DIAG, TRANS, UPLO

          COMPLEX*16   A( LDA, * ), X( * )

 PURPOSE
      ZTBSV  solves one of the systems of equations

      where b and x are n element vectors and A is an n by n unit,
      or non-unit, upper or lower triangular band matrix, with ( k
      + 1 ) diagonals.

      No test for singularity or near-singularity is included in
      this routine. Such tests must be performed before calling
      this routine.

 PARAMETERS
      UPLO   - CHARACTER*1.
               On entry, UPLO specifies whether the matrix is an
               upper or lower triangular matrix as follows:

               UPLO = 'U' or 'u'   A is an upper triangular matrix.

               UPLO = 'L' or 'l'   A is a lower triangular matrix.
```

```
       Unchanged on exit.

TRANS  - CHARACTER*1.
       On entry, TRANS specifies the equations to be solved
       as follows:

       TRANS = 'N' or 'n'   A*x = b.

       TRANS = 'T' or 't'   A'*x = b.

       TRANS = 'C' or 'c'   conjg( A' )*x = b.

       Unchanged on exit.

DIAG   - CHARACTER*1.
       On entry, DIAG specifies whether or not A is unit
       triangular as follows:

       DIAG = 'U' or 'u'   A is assumed to be unit triangu-
       lar.

       DIAG = 'N' or 'n'   A is not assumed to be unit tri-
       angular.

       Unchanged on exit.

N      - INTEGER.
       On entry, N specifies the order of the matrix A.  N
       must be at least zero.  Unchanged on exit.

K      - INTEGER.
       On entry with UPLO = 'U' or 'u', K specifies the
       number of super-diagonals of the matrix A.  On entry
       with UPLO = 'L' or 'l', K specifies the number of
       sub-diagonals of the matrix A.  K must satisfy  0
       .le. K.  Unchanged on exit.

A      - COMPLEX*16       array of DIMENSION ( LDA, n ).
       Before entry with UPLO = 'U' or 'u', the leading ( k
       + 1 ) by n part of the array A must contain the upper
       triangular band part of the matrix of coefficients,
       supplied column by column, with the leading diagonal
       of the matrix in row ( k + 1 ) of the array, the
       first super-diagonal starting at position 2 in row k,
       and so on. The top left k by k triangle of the array
       A is not referenced.  The following program segment
       will transfer an upper triangular band matrix from
       conventional full matrix storage to band storage:

       DO 20, J = 1, N M = K + 1 - J DO 10, I = MAX( 1, J -
```

```
K ), J A( M + I, J ) = matrix( I, J ) 10    CONTINUE
20 CONTINUE

Before entry with UPLO = 'L' or 'l', the leading ( k
+ 1 ) by n part of the array A must contain the lower
triangular band part of the matrix of coefficients,
supplied column by column, with the leading diagonal
of the matrix in row 1 of the array, the first sub-
diagonal starting at position 1 in row 2, and so on.
The bottom right k by k triangle of the array A is
not referenced.  The following program segment will
transfer a lower triangular band matrix from conven-
tional full matrix storage to band storage:

DO 20, J = 1, N M = 1 - J DO 10, I = J, MIN( N, J + K
) A( M + I, J ) = matrix( I, J ) 10    CONTINUE 20
CONTINUE

Note that when DIAG = 'U' or 'u' the elements of the
array A corresponding to the diagonal elements of the

matrix are not referenced, but are assumed to be
unity.  Unchanged on exit.

LDA    - INTEGER.
       On entry, LDA specifies the first dimension of A as
       declared in the calling (sub) program. LDA must be at
       least ( k + 1 ).  Unchanged on exit.

X      - COMPLEX*16       array of dimension at least
       ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
       incremented array X must contain the n element
       right-hand side vector b. On exit, X is overwritten
       with the solution vector x.

INCX   - INTEGER.
       On entry, INCX specifies the increment for the ele-
       ments of X. INCX must not be zero.  Unchanged on
       exit.
```

———-

13.13.3 fortran code

— ztbsv.f —

```
*  ====================================================================
```

```
      SUBROUTINE ZTBSV(UPLO,TRANS,DIAG,N,K,A,LDA,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,K,LDA,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),X(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP
      INTEGER I,INFO,IX,J,JX,KPLUS1,KX,L
      LOGICAL NOCONJ,NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DCONJG,MAX,MIN
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (K.LT.0) THEN
```

```
         INFO = 5
      ELSE IF (LDA.LT. (K+1)) THEN
         INFO = 7
      ELSE IF (INCX.EQ.0) THEN
         INFO = 9
      END IF
      IF (INFO.NE.0) THEN
         CALL XERBLA('ZTBSV ',INFO)
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOCONJ = LSAME(TRANS,'T')
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX  too small for descending loops.
*
      IF (INCX.LE.0) THEN
         KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
         KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed by sequentially with one pass through A.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x := inv( A )*x.
*
          IF (LSAME(UPLO,'U')) THEN
              KPLUS1 = K + 1
              IF (INCX.EQ.1) THEN
                  DO 20 J = N,1,-1
                      IF (X(J).NE.ZERO) THEN
                          L = KPLUS1 - J
                          IF (NOUNIT) X(J) = X(J)/A(KPLUS1,J)
                          TEMP = X(J)
                          DO 10 I = J - 1,MAX(1,J-K),-1
                              X(I) = X(I) - TEMP*A(L+I,J)
   10                     CONTINUE
                      END IF
   20             CONTINUE
              ELSE
                  KX = KX + (N-1)*INCX
                  JX = KX
```

```
                DO 40 J = N,1,-1
                    KX = KX - INCX
                    IF (X(JX).NE.ZERO) THEN
                        IX = KX
                        L = KPLUS1 - J
                        IF (NOUNIT) X(JX) = X(JX)/A(KPLUS1,J)
                        TEMP = X(JX)
                        DO 30 I = J - 1,MAX(1,J-K),-1
                            X(IX) = X(IX) - TEMP*A(L+I,J)
                            IX = IX - INCX
   30                   CONTINUE
                    END IF
                    JX = JX - INCX
   40           CONTINUE
            END IF
        ELSE
            IF (INCX.EQ.1) THEN
                DO 60 J = 1,N
                    IF (X(J).NE.ZERO) THEN
                        L = 1 - J
                        IF (NOUNIT) X(J) = X(J)/A(1,J)
                        TEMP = X(J)
                        DO 50 I = J + 1,MIN(N,J+K)
                            X(I) = X(I) - TEMP*A(L+I,J)
   50                   CONTINUE
                    END IF
   60           CONTINUE
            ELSE
                JX = KX
                DO 80 J = 1,N
                    KX = KX + INCX
                    IF (X(JX).NE.ZERO) THEN
                        IX = KX
                        L = 1 - J
                        IF (NOUNIT) X(JX) = X(JX)/A(1,J)
                        TEMP = X(JX)
                        DO 70 I = J + 1,MIN(N,J+K)
                            X(IX) = X(IX) - TEMP*A(L+I,J)
                            IX = IX + INCX
   70                   CONTINUE
                    END IF
                    JX = JX + INCX
   80           CONTINUE
            END IF
        END IF
    ELSE
*
*       Form  x := inv( A**T )*x  or  x := inv( A**H )*x.
*
        IF (LSAME(UPLO,'U')) THEN
```

```
            KPLUS1 = K + 1
            IF (INCX.EQ.1) THEN
                DO 110 J = 1,N
                    TEMP = X(J)
                    L = KPLUS1 - J
                    IF (NOCONJ) THEN
                        DO 90 I = MAX(1,J-K),J - 1
                            TEMP = TEMP - A(L+I,J)*X(I)
 90                     CONTINUE
                        IF (NOUNIT) TEMP = TEMP/A(KPLUS1,J)
                    ELSE
                        DO 100 I = MAX(1,J-K),J - 1
                            TEMP = TEMP - DCONJG(A(L+I,J))*X(I)
 100                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/DCONJG(A(KPLUS1,J))
                    END IF
                    X(J) = TEMP
 110            CONTINUE
            ELSE
                JX = KX
                DO 140 J = 1,N
                    TEMP = X(JX)
                    IX = KX
                    L = KPLUS1 - J
                    IF (NOCONJ) THEN
                        DO 120 I = MAX(1,J-K),J - 1
                            TEMP = TEMP - A(L+I,J)*X(IX)
                            IX = IX + INCX
 120                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/A(KPLUS1,J)
                    ELSE
                        DO 130 I = MAX(1,J-K),J - 1
                            TEMP = TEMP - DCONJG(A(L+I,J))*X(IX)
                            IX = IX + INCX
 130                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/DCONJG(A(KPLUS1,J))
                    END IF
                    X(JX) = TEMP
                    JX = JX + INCX
                    IF (J.GT.K) KX = KX + INCX
 140            CONTINUE
            END IF
        ELSE
            IF (INCX.EQ.1) THEN
                DO 170 J = N,1,-1
                    TEMP = X(J)
                    L = 1 - J
                    IF (NOCONJ) THEN
                        DO 150 I = MIN(N,J+K),J + 1,-1
                            TEMP = TEMP - A(L+I,J)*X(I)
```

```
 150                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/A(1,J)
                    ELSE
                        DO 160 I = MIN(N,J+K),J + 1,-1
                            TEMP = TEMP - DCONJG(A(L+I,J))*X(I)
 160                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/DCONJG(A(1,J))
                    END IF
                    X(J) = TEMP
 170            CONTINUE
            ELSE
                KX = KX + (N-1)*INCX
                JX = KX
                DO 200 J = N,1,-1
                    TEMP = X(JX)
                    IX = KX
                    L = 1 - J
                    IF (NOCONJ) THEN
                        DO 180 I = MIN(N,J+K),J + 1,-1
                            TEMP = TEMP - A(L+I,J)*X(IX)
                            IX = IX - INCX
 180                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/A(1,J)
                    ELSE
                        DO 190 I = MIN(N,J+K),J + 1,-1
                            TEMP = TEMP - DCONJG(A(L+I,J))*X(IX)
                            IX = IX - INCX
 190                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/DCONJG(A(1,J))
                    END IF
                    X(JX) = TEMP
                    JX = JX - INCX
                    IF ((N-J).GE.K) KX = KX - INCX
 200            CONTINUE
            END IF
        END IF
    END IF
*
    RETURN
*
*   End of ZTBSV .
*
    END
```

———-

13.13.4 lisp code

— BLAS 2 ztbsv —

```
(let* ((zero (complex 0.0 0.0)))
  (declare (type (complex double-float) zero))
  (defun ztbsv (uplo trans diag n k a lda x incx)
    (declare (type (simple-array (complex double-float) (*)) x a)
             (type fixnum incx lda k n)
             (type character diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a (complex double-float) a-%data% a-%offset%)
         (x (complex double-float) x-%data% x-%offset%))
      (prog ((noconj nil) (nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0)
             (kplus1 0) (kx 0) (l 0) (temp #C(0.0 0.0)))
        (declare (type (member t nil) noconj nounit)
                 (type fixnum i info ix j jx kplus1 kx l)
                 (type (complex double-float) temp))
        (setf info 0)
        (cond
          ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
           (setf info 1))
          ((and (not (char-equal trans #\N))
                (not (char-equal trans #\T))
                (not (char-equal trans #\C)))
           (setf info 2))
          ((and (not (char-equal diag #\U)) (not (char-equal diag #\N)))
           (setf info 3))
          ((< n 0)
           (setf info 4))
          ((< k 0)
           (setf info 5))
          ((< lda (f2cl-lib:int-add k 1))
           (setf info 7))
          ((= incx 0)
           (setf info 9)))
        (cond
          ((/= info 0)
           (error
            " ** On entry to ~a parameter number ~a had an illegal value~%"
            "ZTBSV" info)
           (go end_label)))
        (if (= n 0) (go end_label))
        (setf noconj (char-equal trans #\T))
        (setf nounit (char-equal diag #\N))
        (cond
```

```
    ((<= incx 0)
     (setf kx
             (f2cl-lib:int-sub 1
                               (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                 incx))))
    ((/= incx 1)
     (setf kx 1)))
  (cond
    ((char-equal trans #\N)
     (cond
       ((char-equal uplo #\U)
        (setf kplus1 (f2cl-lib:int-add k 1))
        (cond
          ((= incx 1)
           (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                         ((> j 1) nil)
             (tagbody
               (cond
                 ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
                  (setf l (f2cl-lib:int-sub kplus1 j))
                  (if nounit
                      (setf (f2cl-lib:fref x-%data%
                                           (j)
                                           ((1 *))
                                           x-%offset%)
                            (/
                             (f2cl-lib:fref x-%data%
                                            (j)
                                            ((1 *))
                                            x-%offset%)
                             (f2cl-lib:fref a-%data%
                                            (kplus1 j)
                                            ((1 lda) (1 *))
                                            a-%offset%))))
                  (setf temp
                          (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                  (f2cl-lib:fdo (i
                                 (f2cl-lib:int-add j
                                                   (f2cl-lib:int-sub 1))
                                 (f2cl-lib:int-add i
                                                   (f2cl-lib:int-sub 1)))
                                ((> i
                                    (max (the fixnum 1)
                                         (the fixnum
                                              (f2cl-lib:int-add j
                                                 (f2cl-lib:int-sub
                                                                 k)))))
                                 nil)
                    (tagbody
                      (setf (f2cl-lib:fref x-%data%
```

```
                                    (i)
                                    ((1 *))
                                    x-%offset%)
                         (-
                          (f2cl-lib:fref x-%data%
                                         (i)
                                         ((1 *))
                                         x-%offset%)
                          (* temp
                             (f2cl-lib:fref a-%data%
                                            ((f2cl-lib:int-add l i)
                                             j)
                                            ((1 lda) (1 *))
                                            a-%offset%)))))))))))
(t
 (setf kx
         (f2cl-lib:int-add kx
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-sub n 1)
                            incx)))
 (setf jx kx)
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (setf kx (f2cl-lib:int-sub kx incx))
     (cond
       ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
        (setf ix kx)
        (setf l (f2cl-lib:int-sub kplus1 j))
        (if nounit
            (setf (f2cl-lib:fref x-%data%
                                 (jx)
                                 ((1 *))
                                 x-%offset%)
                  (/
                   (f2cl-lib:fref x-%data%
                                  (jx)
                                  ((1 *))
                                  x-%offset%)
                   (f2cl-lib:fref a-%data%
                                  (kplus1 j)
                                  ((1 lda) (1 *))
                                  a-%offset%))))
        (setf temp
                (f2cl-lib:fref x-%data%
                               (jx)
                               ((1 *))
                               x-%offset%))
        (f2cl-lib:fdo (i
                       (f2cl-lib:int-add j
```

```
                                                    (f2cl-lib:int-sub 1))
                                  (f2cl-lib:int-add i
                                                    (f2cl-lib:int-sub 1)))
                                 ((> i
                                     (max (the fixnum 1)
                                          (the fixnum
                                               (f2cl-lib:int-add j
                                                   (f2cl-lib:int-sub
                                                                 k)))))
                                  nil)
                        (tagbody
                          (setf (f2cl-lib:fref x-%data%
                                               (ix)
                                               ((1 *))
                                               x-%offset%)
                                (-
                                 (f2cl-lib:fref x-%data%
                                                (ix)
                                                ((1 *))
                                                x-%offset%)
                                 (* temp
                                    (f2cl-lib:fref a-%data%
                                                   ((f2cl-lib:int-add l i)
                                                    j)
                                                   ((1 lda) (1 *))
                                                   a-%offset%))))
                          (setf ix (f2cl-lib:int-sub ix incx))))))
                 (setf jx (f2cl-lib:int-sub jx incx)))))))
        (t
         (cond
           ((= incx 1)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (cond
                  ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
                   (setf l (f2cl-lib:int-sub 1 j))
                   (if nounit
                       (setf (f2cl-lib:fref x-%data%
                                            (j)
                                            ((1 *))
                                            x-%offset%)
                             (/
                              (f2cl-lib:fref x-%data%
                                             (j)
                                             ((1 *))
                                             x-%offset%)
                              (f2cl-lib:fref a-%data%
                                             (1 j)
                                             ((1 lda) (1 *))
```

```
                                    a-%offset%))))
            (setf temp
                    (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
            (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                           (f2cl-lib:int-add i 1))
                          ((> i
                              (min (the fixnum n)
                                   (the fixnum
                                        (f2cl-lib:int-add j k))))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref x-%data%
                                     (i)
                                     ((1 *))
                                     x-%offset%)
                      (-
                       (f2cl-lib:fref x-%data%
                                      (i)
                                      ((1 *))
                                      x-%offset%)
                       (* temp
                          (f2cl-lib:fref a-%data%
                                         ((f2cl-lib:int-add l i)
                                          j)
                                         ((1 lda) (1 *))
                                         a-%offset%))))))))))
(t
 (setf jx kx)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf kx (f2cl-lib:int-add kx incx))
     (cond
       ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
        (setf ix kx)
        (setf l (f2cl-lib:int-sub 1 j))
        (if nounit
            (setf (f2cl-lib:fref x-%data%
                                 (jx)
                                 ((1 *))
                                 x-%offset%)
                  (/
                   (f2cl-lib:fref x-%data%
                                  (jx)
                                  ((1 *))
                                  x-%offset%)
                   (f2cl-lib:fref a-%data%
                                  (1 j)
                                  ((1 lda) (1 *))
                                  a-%offset%))))
```

```
                    (setf temp
                            (f2cl-lib:fref x-%data%
                                           (jx)
                                           ((1 *))
                                           x-%offset%))
                    (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                                   (f2cl-lib:int-add i 1))
                                  ((> i
                                      (min (the fixnum n)
                                           (the fixnum
                                                (f2cl-lib:int-add j k))))
                                   nil)
                      (tagbody
                        (setf (f2cl-lib:fref x-%data%
                                             (ix)
                                             ((1 *))
                                             x-%offset%)
                              (-
                               (f2cl-lib:fref x-%data%
                                              (ix)
                                              ((1 *))
                                              x-%offset%)
                               (* temp
                                  (f2cl-lib:fref a-%data%
                                                 ((f2cl-lib:int-add l i)
                                                  j)
                                                 ((1 lda) (1 *))
                                                 a-%offset%))))
                        (setf ix (f2cl-lib:int-add ix incx))))))
                  (setf jx (f2cl-lib:int-add jx incx)))))))))
            (t
             (cond
               ((char-equal uplo #\U)
                (setf kplus1 (f2cl-lib:int-add k 1))
                (cond
                  ((= incx 1)
                   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                                 ((> j n) nil)
                     (tagbody
                       (setf temp
                               (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                       (setf l (f2cl-lib:int-sub kplus1 j))
                       (cond
                         (noconj
                          (f2cl-lib:fdo (i
                                         (max (the fixnum 1)
                                              (the fixnum
                                                   (f2cl-lib:int-add j
                                                                     (f2cl-lib:int-sub
                                                                      k))))
```

```
                    (f2cl-lib:int-add i 1))
                   ((> i
                       (f2cl-lib:int-add j
                                         (f2cl-lib:int-sub
                                          1)))
                    nil)
      (tagbody
        (setf temp
                (- temp
                   (*
                    (f2cl-lib:fref a-%data%
                                   ((f2cl-lib:int-add l i)
                                    j)
                                   ((1 lda) (1 *))
                                   a-%offset%)
                    (f2cl-lib:fref x-%data%
                                   (i)
                                   ((1 *))
                                   x-%offset%))))))
    (if nounit
        (setf temp
                (/ temp
                   (f2cl-lib:fref a-%data%
                                  (kplus1 j)
                                  ((1 lda) (1 *))
                                  a-%offset%)))))
   (t
    (f2cl-lib:fdo (i
                   (max (the fixnum 1)
                        (the fixnum
                             (f2cl-lib:int-add j
                                               (f2cl-lib:int-sub
                                                k))))
                   (f2cl-lib:int-add i 1))
                  ((> i
                      (f2cl-lib:int-add j
                                        (f2cl-lib:int-sub
                                         1)))
                   nil)
      (tagbody
        (setf temp
                (- temp
                   (*
                    (f2cl-lib:dconjg
                     (f2cl-lib:fref a-%data%
                                    ((f2cl-lib:int-add l i)
                                     j)
                                    ((1 lda) (1 *))
                                    a-%offset%))
                    (f2cl-lib:fref x-%data%
```

```
                                          (i)
                                          ((1 *))
                                          x-%offset%))))))
                (if nounit
                    (setf temp
                            (/ temp
                               (f2cl-lib:dconjg
                                (f2cl-lib:fref a-%data%
                                               (kplus1 j)
                                               ((1 lda) (1 *))
                                               a-%offset%)))))))
            (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                    temp))))
        (t
         (setf jx kx)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf temp
                     (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
             (setf ix kx)
             (setf l (f2cl-lib:int-sub kplus1 j))
             (cond
               (noconj
                (f2cl-lib:fdo (i
                               (max (the fixnum 1)
                                    (the fixnum
                                         (f2cl-lib:int-add j
                                                           (f2cl-lib:int-sub
                                                            k))))
                               (f2cl-lib:int-add i 1))
                              ((> i
                                  (f2cl-lib:int-add j
                                                    (f2cl-lib:int-sub
                                                     1)))
                               nil)
                 (tagbody
                   (setf temp
                           (- temp
                              (*
                               (f2cl-lib:fref a-%data%
                                              ((f2cl-lib:int-add l i)
                                               j)
                                              ((1 lda) (1 *))
                                              a-%offset%)
                               (f2cl-lib:fref x-%data%
                                              (ix)
                                              ((1 *))
                                              x-%offset%))))
                   (setf ix (f2cl-lib:int-add ix incx))))
```

```
                 (if nounit
                     (setf temp
                             (/ temp
                                (f2cl-lib:fref a-%data%
                                               (kplus1 j)
                                               ((1 lda) (1 *))
                                               a-%offset%)))))
                (t
                 (f2cl-lib:fdo (i
                                (max (the fixnum 1)
                                     (the fixnum
                                          (f2cl-lib:int-add j
                                                            (f2cl-lib:int-sub
                                                             k))))
                                (f2cl-lib:int-add i 1))
                               ((> i
                                   (f2cl-lib:int-add j
                                                     (f2cl-lib:int-sub
                                                      1)))
                                nil)
                   (tagbody
                     (setf temp
                             (- temp
                                (*
                                 (f2cl-lib:dconjg
                                  (f2cl-lib:fref a-%data%
                                                 ((f2cl-lib:int-add l i)
                                                  j)
                                                 ((1 lda) (1 *))
                                                 a-%offset%))
                                 (f2cl-lib:fref x-%data%
                                                (ix)
                                                ((1 *))
                                                x-%offset%))))
                     (setf ix (f2cl-lib:int-add ix incx))))
                 (if nounit
                     (setf temp
                             (/ temp
                                (f2cl-lib:dconjg
                                 (f2cl-lib:fref a-%data%
                                                (kplus1 j)
                                                ((1 lda) (1 *))
                                                a-%offset%)))))))
            (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                    temp)
            (setf jx (f2cl-lib:int-add jx incx))
            (if (> j k) (setf kx (f2cl-lib:int-add kx incx))))))))
    (t
     (cond
       ((= incx 1)
```

```
(f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j 1) nil)
  (tagbody
    (setf temp
            (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
    (setf l (f2cl-lib:int-sub 1 j))
    (cond
      (noconj
       (f2cl-lib:fdo (i
                      (min (the fixnum n)
                           (the fixnum
                                (f2cl-lib:int-add j k)))
                      (f2cl-lib:int-add i
                                        (f2cl-lib:int-sub 1)))
                     ((> i (f2cl-lib:int-add j 1)) nil)
         (tagbody
           (setf temp
                   (- temp
                      (*
                       (f2cl-lib:fref a-%data%
                                      ((f2cl-lib:int-add l i)
                                       j)
                                      ((1 lda) (1 *))
                                      a-%offset%)
                       (f2cl-lib:fref x-%data%
                                      (i)
                                      ((1 *))
                                      x-%offset%))))))
       (if nounit
           (setf temp
                   (/ temp
                      (f2cl-lib:fref a-%data%
                                     (1 j)
                                     ((1 lda) (1 *))
                                     a-%offset%)))))
      (t
       (f2cl-lib:fdo (i
                      (min (the fixnum n)
                           (the fixnum
                                (f2cl-lib:int-add j k)))
                      (f2cl-lib:int-add i
                                        (f2cl-lib:int-sub 1)))
                     ((> i (f2cl-lib:int-add j 1)) nil)
         (tagbody
           (setf temp
                   (- temp
                      (*
                       (f2cl-lib:dconjg
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add l i)
```

```
                                          j)
                                         ((1 lda) (1 *))
                                         a-%offset%))
                            (f2cl-lib:fref x-%data%
                                          (i)
                                          ((1 *))
                                          x-%offset%))))))
          (if nounit
              (setf temp
                      (/ temp
                         (f2cl-lib:dconjg
                          (f2cl-lib:fref a-%data%
                                          (1 j)
                                          ((1 lda) (1 *))
                                          a-%offset%)))))))
      (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
              temp))))
(t
 (setf kx
         (f2cl-lib:int-add kx
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-sub n 1)
                            incx)))
 (setf jx kx)
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (setf temp
             (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
     (setf ix kx)
     (setf l (f2cl-lib:int-sub 1 j))
     (cond
       (noconj
        (f2cl-lib:fdo (i
                       (min (the fixnum n)
                            (the fixnum
                                 (f2cl-lib:int-add j k)))
                       (f2cl-lib:int-add i
                                         (f2cl-lib:int-sub 1)))
                      ((> i (f2cl-lib:int-add j 1)) nil)
          (tagbody
            (setf temp
                    (- temp
                       (*
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add l i)
                                        j)
                                       ((1 lda) (1 *))
                                       a-%offset%)
                        (f2cl-lib:fref x-%data%
```

```
                                                (ix)
                                                ((1 *))
                                                x-%offset%))))
                                    (setf ix (f2cl-lib:int-sub ix incx))))
                                 (if nounit
                                     (setf temp
                                             (/ temp
                                                (f2cl-lib:fref a-%data%
                                                               (1 j)
                                                               ((1 lda) (1 *))
                                                               a-%offset%)))))
                                (t
                                 (f2cl-lib:fdo (i
                                                (min (the fixnum n)
                                                     (the fixnum
                                                          (f2cl-lib:int-add j k)))
                                                (f2cl-lib:int-add i
                                                                  (f2cl-lib:int-sub 1)))
                                               ((> i (f2cl-lib:int-add j 1)) nil)
                                   (tagbody
                                     (setf temp
                                             (- temp
                                                (*
                                                 (f2cl-lib:dconjg
                                                  (f2cl-lib:fref a-%data%
                                                                 ((f2cl-lib:int-add l i)
                                                                  j)
                                                                 ((1 lda) (1 *))
                                                                 a-%offset%))
                                                 (f2cl-lib:fref x-%data%
                                                                (ix)
                                                                ((1 *))
                                                                x-%offset%))))
                                     (setf ix (f2cl-lib:int-sub ix incx))))
                                 (if nounit
                                     (setf temp
                                             (/ temp
                                                (f2cl-lib:dconjg
                                                 (f2cl-lib:fref a-%data%
                                                                (1 j)
                                                                ((1 lda) (1 *))
                                                                a-%offset%)))))))
                            (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                                    temp)
                            (setf jx (f2cl-lib:int-sub jx incx))
                            (if (>= (f2cl-lib:int-sub n j) k)
                                (setf kx (f2cl-lib:int-sub kx incx))))))))))
end_label
       (return (values nil nil nil nil nil nil nil nil nil))))))
```

———-

13.14 ztpmv - matrix-vector Ax or $A^T x$ or $A^H x$

13.14.1 Axiom unit tests

— **ztpmv.input** —

```
)set break resume
)sys rm -f ztpmv.output
)spool ztpmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

13.14.2 Axiom help page

— **ztpmv.help** —

```
====================================================================
ztpmv examples
====================================================================

====================================================================
Man Page Details
====================================================================

 NAME
      ZTPMV - perform one of the matrix-vector operations
      x := A*x, or x := A'*x, or x := conjg( A' )*x,

 SYNOPSIS
      SUBROUTINE ZTPMV ( UPLO, TRANS, DIAG, N, AP, X, INCX )

          INTEGER      INCX, N

          CHARACTER*1  DIAG, TRANS, UPLO

          COMPLEX*16   AP( * ), X( * )
```

```
PURPOSE
     ZTPMV  performs one of the matrix-vector operations

     where x is an n element vector and  A is an n by n unit, or
     non-unit, upper or lower triangular matrix, supplied in
     packed form.

PARAMETERS
     UPLO   - CHARACTER*1.
            On entry, UPLO specifies whether the matrix is an
            upper or lower triangular matrix as follows:

            UPLO = 'U' or 'u'   A is an upper triangular matrix.

            UPLO = 'L' or 'l'   A is a lower triangular matrix.

            Unchanged on exit.

     TRANS  - CHARACTER*1.
            On entry, TRANS specifies the operation to be per-
            formed as follows:

            TRANS = 'N' or 'n'   x := A*x.

            TRANS = 'T' or 't'   x := A'*x.

            TRANS = 'C' or 'c'   x := conjg( A' )*x.

            Unchanged on exit.

     DIAG   - CHARACTER*1.
            On entry, DIAG specifies whether or not A is unit
            triangular as follows:

            DIAG = 'U' or 'u'   A is assumed to be unit triangu-
            lar.

            DIAG = 'N' or 'n'   A is not assumed to be unit

            triangular.

            Unchanged on exit.

     N      - INTEGER.
            On entry, N specifies the order of the matrix A.  N
            must be at least zero.  Unchanged on exit.

     AP     - COMPLEX*16       array of DIMENSION at least
            ( ( n*( n + 1 ) )/2 ).  Before entry with  UPLO = 'U'
```

```
            or 'u', the array AP must contain the upper triangu-
            lar matrix packed sequentially, column by column, so
            that AP( 1 ) contains a( 1, 1 ), AP( 2 ) and AP( 3 )
            contain a( 1, 2 ) and a( 2, 2 ) respectively, and so
            on.  Before entry with UPLO = 'L' or 'l', the array
            AP must contain the lower triangular matrix packed
            sequentially, column by column, so that AP( 1 ) con-
            tains a( 1, 1 ), AP( 2 ) and AP( 3 ) contain a( 2, 1
            ) and a( 3, 1 ) respectively, and so on.  Note that
            when  DIAG = 'U' or 'u', the diagonal elements of A
            are not referenced, but are assumed to be unity.
            Unchanged on exit.

      X     - COMPLEX*16       array of dimension at least
            ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
            incremented array X must contain the n element vector
            x. On exit, X is overwritten with the transformed
            vector x.

      INCX  - INTEGER.
            On entry, INCX specifies the increment for the ele-
            ments of X. INCX must not be zero.  Unchanged on
            exit.
```

———-

13.14.3 fortran code

— ztpmv.f —

```
*  ==================================================================
      SUBROUTINE ZTPMV(UPLO,TRANS,DIAG,N,AP,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16 AP(*),X(*)
*     ..
*
*  ==================================================================
```

```
*
*     .. Parameters ..
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP
      INTEGER I,INFO,IX,J,JX,K,KK,KX
      LOGICAL NOCONJ,NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DCONJG
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (INCX.EQ.0) THEN
          INFO = 7
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZTPMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOCONJ = LSAME(TRANS,'T')
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX  too small for descending loops.
```

```
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of AP are
*     accessed sequentially with one pass through AP.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x:= A*x.
*
          IF (LSAME(UPLO,'U')) THEN
              KK = 1
              IF (INCX.EQ.1) THEN
                  DO 20 J = 1,N
                      IF (X(J).NE.ZERO) THEN
                          TEMP = X(J)
                          K = KK
                          DO 10 I = 1,J - 1
                              X(I) = X(I) + TEMP*AP(K)
                              K = K + 1
   10                     CONTINUE
                          IF (NOUNIT) X(J) = X(J)*AP(KK+J-1)
                      END IF
                      KK = KK + J
   20             CONTINUE
              ELSE
                  JX = KX
                  DO 40 J = 1,N
                      IF (X(JX).NE.ZERO) THEN
                          TEMP = X(JX)
                          IX = KX
                          DO 30 K = KK,KK + J - 2
                              X(IX) = X(IX) + TEMP*AP(K)
                              IX = IX + INCX
   30                     CONTINUE
                          IF (NOUNIT) X(JX) = X(JX)*AP(KK+J-1)
                      END IF
                      JX = JX + INCX
                      KK = KK + J
   40             CONTINUE
              END IF
          ELSE
              KK = (N* (N+1))/2
              IF (INCX.EQ.1) THEN
                  DO 60 J = N,1,-1
                      IF (X(J).NE.ZERO) THEN
```

```
                        TEMP = X(J)
                        K = KK
                        DO 50 I = N,J + 1,-1
                            X(I) = X(I) + TEMP*AP(K)
                            K = K - 1
   50                   CONTINUE
                        IF (NOUNIT) X(J) = X(J)*AP(KK-N+J)
                    END IF
                    KK = KK - (N-J+1)
   60            CONTINUE
            ELSE
                KX = KX + (N-1)*INCX
                JX = KX
                DO 80 J = N,1,-1
                    IF (X(JX).NE.ZERO) THEN
                        TEMP = X(JX)
                        IX = KX
                        DO 70 K = KK,KK - (N- (J+1)),-1
                            X(IX) = X(IX) + TEMP*AP(K)
                            IX = IX - INCX
   70                   CONTINUE
                        IF (NOUNIT) X(JX) = X(JX)*AP(KK-N+J)
                    END IF
                    JX = JX - INCX
                    KK = KK - (N-J+1)
   80            CONTINUE
            END IF
        END IF
     ELSE
*
*       Form  x := A**T*x  or  x := A**H*x.
*
        IF (LSAME(UPLO,'U')) THEN
            KK = (N* (N+1))/2
            IF (INCX.EQ.1) THEN
                DO 110 J = N,1,-1
                    TEMP = X(J)
                    K = KK - 1
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*AP(KK)
                        DO 90 I = J - 1,1,-1
                            TEMP = TEMP + AP(K)*X(I)
                            K = K - 1
   90                   CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*DCONJG(AP(KK))
                        DO 100 I = J - 1,1,-1
                            TEMP = TEMP + DCONJG(AP(K))*X(I)
                            K = K - 1
  100                   CONTINUE
```

```
                    END IF
                    X(J) = TEMP
                    KK = KK - J
 110            CONTINUE
            ELSE
                JX = KX + (N-1)*INCX
                DO 140 J = N,1,-1
                    TEMP = X(JX)
                    IX = JX
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*AP(KK)
                        DO 120 K = KK - 1,KK - J + 1,-1
                            IX = IX - INCX
                            TEMP = TEMP + AP(K)*X(IX)
 120                    CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*DCONJG(AP(KK))
                        DO 130 K = KK - 1,KK - J + 1,-1
                            IX = IX - INCX
                            TEMP = TEMP + DCONJG(AP(K))*X(IX)
 130                    CONTINUE
                    END IF
                    X(JX) = TEMP
                    JX = JX - INCX
                    KK = KK - J
 140            CONTINUE
            END IF
        ELSE
            KK = 1
            IF (INCX.EQ.1) THEN
                DO 170 J = 1,N
                    TEMP = X(J)
                    K = KK + 1
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*AP(KK)
                        DO 150 I = J + 1,N
                            TEMP = TEMP + AP(K)*X(I)
                            K = K + 1
 150                    CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*DCONJG(AP(KK))
                        DO 160 I = J + 1,N
                            TEMP = TEMP + DCONJG(AP(K))*X(I)
                            K = K + 1
 160                    CONTINUE
                    END IF
                    X(J) = TEMP
                    KK = KK + (N-J+1)
 170            CONTINUE
            ELSE
```

```
                JX = KX
                DO 200 J = 1,N
                    TEMP = X(JX)
                    IX = JX
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*AP(KK)
                        DO 180 K = KK + 1,KK + N - J
                            IX = IX + INCX
                            TEMP = TEMP + AP(K)*X(IX)
  180                   CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*DCONJG(AP(KK))
                        DO 190 K = KK + 1,KK + N - J
                            IX = IX + INCX
                            TEMP = TEMP + DCONJG(AP(K))*X(IX)
  190                   CONTINUE
                    END IF
                    X(JX) = TEMP
                    JX = JX + INCX
                    KK = KK + (N-J+1)
  200           CONTINUE
            END IF
        END IF
    END IF
*
    RETURN
*
*   End of ZTPMV .
*
    END
```

———-

13.14.4 lisp code

— BLAS 2 ztpmv —

```
(let* ((zero (complex 0.0 0.0)))
  (declare (type (complex double-float) zero))
  (defun ztpmv (uplo trans diag n ap x incx)
    (declare (type (simple-array (complex double-float) (*)) x ap)
             (type fixnum incx n)
             (type character diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
```

```
    (ap (complex double-float) ap-%data% ap-%offset%)
    (x (complex double-float) x-%data% x-%offset%))
  (prog ((noconj nil) (nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0) (k 0)
         (kk 0) (kx 0) (temp #C(0.0 0.0)))
    (declare (type (member t nil) noconj nounit)
             (type fixnum i info ix j jx k kk kx)
             (type (complex double-float) temp))
    (setf info 0)
    (cond
      ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
       (setf info 1))
      ((and (not (char-equal trans #\N))
            (not (char-equal trans #\T))
            (not (char-equal trans #\C)))
       (setf info 2))
      ((and (not (char-equal diag #\U)) (not (char-equal diag #\N)))
       (setf info 3))
      ((< n 0)
       (setf info 4))
      ((= incx 0)
       (setf info 7)))
    (cond
      ((/= info 0)
       (error
        " ** On entry to ~a parameter number ~a had an illegal value~%"
        "ZTPMV" info)
       (go end_label)))
    (if (= n 0) (go end_label))
    (setf noconj (char-equal trans #\T))
    (setf nounit (char-equal diag #\N))
    (cond
      ((<= incx 0)
       (setf kx
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                   incx))))
      ((/= incx 1)
       (setf kx 1)))
    (cond
      ((char-equal trans #\N)
       (cond
         ((char-equal uplo #\U)
          (setf kk 1)
          (cond
            ((= incx 1)
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (cond
                   ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
```

```
            (setf temp
                    (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
            (setf k kk)
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i
                              (f2cl-lib:int-add j
                                                (f2cl-lib:int-sub
                                                 1)))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref x-%data%
                                     (i)
                                     ((1 *))
                                     x-%offset%)
                      (+
                       (f2cl-lib:fref x-%data%
                                      (i)
                                      ((1 *))
                                      x-%offset%)
                       (* temp
                          (f2cl-lib:fref ap-%data%
                                         (k)
                                         ((1 *))
                                         ap-%offset%))))
                (setf k (f2cl-lib:int-add k 1))))
            (if nounit
                (setf (f2cl-lib:fref x-%data%
                                     (j)
                                     ((1 *))
                                     x-%offset%)
                      (*
                       (f2cl-lib:fref x-%data%
                                      (j)
                                      ((1 *))
                                      x-%offset%)
                       (f2cl-lib:fref ap-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add kk j)
                                        1))
                                      ((1 *))
                                      ap-%offset%))))))
        (setf kk (f2cl-lib:int-add kk j)))))
    (t
     (setf jx kx)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (cond
           ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
            (setf temp
```

```
                    (f2cl-lib:fref x-%data%
                                   (jx)
                                   ((1 *))
                                   x-%offset%))
            (setf ix kx)
            (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                          ((> k
                              (f2cl-lib:int-add kk
                                                j
                                                (f2cl-lib:int-sub
                                                 2)))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref x-%data%
                                     (ix)
                                     ((1 *))
                                     x-%offset%)
                      (+
                       (f2cl-lib:fref x-%data%
                                      (ix)
                                      ((1 *))
                                      x-%offset%)
                       (* temp
                          (f2cl-lib:fref ap-%data%
                                         (k)
                                         ((1 *))
                                         ap-%offset%))))
                (setf ix (f2cl-lib:int-add ix incx))))
            (if nounit
                (setf (f2cl-lib:fref x-%data%
                                     (jx)
                                     ((1 *))
                                     x-%offset%)
                      (*
                       (f2cl-lib:fref x-%data%
                                      (jx)
                                      ((1 *))
                                      x-%offset%)
                       (f2cl-lib:fref ap-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add kk j)
                                        1))
                                      ((1 *))
                                      ap-%offset%))))))
        (setf jx (f2cl-lib:int-add jx incx))
        (setf kk (f2cl-lib:int-add kk j)))))))
(t
 (setf kk (the fixnum (truncate (* n (+ n 1)) 2)))
 (cond
   ((= incx 1)
```

```
(f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j 1) nil)
  (tagbody
    (cond
      ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
       (setf temp
               (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
       (setf k kk)
       (f2cl-lib:fdo (i n
                      (f2cl-lib:int-add i
                                        (f2cl-lib:int-sub 1)))
                     ((> i (f2cl-lib:int-add j 1)) nil)
         (tagbody
           (setf (f2cl-lib:fref x-%data%
                                (i)
                                ((1 *))
                                x-%offset%)
                 (+
                  (f2cl-lib:fref x-%data%
                                 (i)
                                 ((1 *))
                                 x-%offset%)
                  (* temp
                     (f2cl-lib:fref ap-%data%
                                    (k)
                                    ((1 *))
                                    ap-%offset%))))
           (setf k (f2cl-lib:int-sub k 1))))
       (if nounit
           (setf (f2cl-lib:fref x-%data%
                                (j)
                                ((1 *))
                                x-%offset%)
                 (*
                  (f2cl-lib:fref x-%data%
                                 (j)
                                 ((1 *))
                                 x-%offset%)
                  (f2cl-lib:fref ap-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-sub kk n)
                                   j))
                                 ((1 *))
                                 ap-%offset%))))))
    (setf kk
            (f2cl-lib:int-sub kk
                              (f2cl-lib:int-add
                               (f2cl-lib:int-sub n j)
                               1))))))
(t
```

```
(setf kx
        (f2cl-lib:int-add kx
                          (f2cl-lib:int-mul
                           (f2cl-lib:int-sub n 1)
                           incx)))
(setf jx kx)
(f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j 1) nil)
  (tagbody
    (cond
      ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
       (setf temp
               (f2cl-lib:fref x-%data%
                              (jx)
                              ((1 *))
                              x-%offset%))
       (setf ix kx)
       (f2cl-lib:fdo (k kk
                      (f2cl-lib:int-add k
                                        (f2cl-lib:int-sub 1)))
                     ((> k
                         (f2cl-lib:int-add kk
                                           (f2cl-lib:int-sub
                                            (f2cl-lib:int-add
                                             n
                                             (f2cl-lib:int-sub
                                              (f2cl-lib:int-add
                                               j
                                               1))))))
                      nil)
         (tagbody
           (setf (f2cl-lib:fref x-%data%
                                (ix)
                                ((1 *))
                                x-%offset%)
                 (+
                  (f2cl-lib:fref x-%data%
                                 (ix)
                                 ((1 *))
                                 x-%offset%)
                  (* temp
                     (f2cl-lib:fref ap-%data%
                                    (k)
                                    ((1 *))
                                    ap-%offset%))))
           (setf ix (f2cl-lib:int-sub ix incx))))
       (if nounit
           (setf (f2cl-lib:fref x-%data%
                                (jx)
                                ((1 *))
```

```
                                        x-%offset%)
                                (*
                                 (f2cl-lib:fref x-%data%
                                                (jx)
                                                ((1 *))
                                                x-%offset%)
                                 (f2cl-lib:fref ap-%data%
                                                ((f2cl-lib:int-add
                                                  (f2cl-lib:int-sub kk n)
                                                  j))
                                                ((1 *))
                                                ap-%offset%))))))
                  (setf jx (f2cl-lib:int-sub jx incx))
                  (setf kk
                          (f2cl-lib:int-sub kk
                                            (f2cl-lib:int-add
                                             (f2cl-lib:int-sub n j)
                                             1)))))))))
        (t
         (cond
           ((char-equal uplo #\U)
            (setf kk (the fixnum (truncate (* n (+ n 1)) 2)))
            (cond
              ((= incx 1)
               (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                             ((> j 1) nil)
                 (tagbody
                   (setf temp
                           (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                   (setf k (f2cl-lib:int-sub kk 1))
                   (cond
                     (noconj
                      (if nounit
                          (setf temp
                                  (* temp
                                     (f2cl-lib:fref ap-%data%
                                                    (kk)
                                                    ((1 *))
                                                    ap-%offset%))))
                      (f2cl-lib:fdo (i
                                     (f2cl-lib:int-add j
                                                       (f2cl-lib:int-sub 1))
                                     (f2cl-lib:int-add i
                                                       (f2cl-lib:int-sub 1)))
                                    ((> i 1) nil)
                        (tagbody
                          (setf temp
                                  (+ temp
                                     (*
                                      (f2cl-lib:fref ap-%data%
```

```
                                        (k)
                                        ((1 *))
                                        ap-%offset%)
                            (f2cl-lib:fref x-%data%
                                        (i)
                                        ((1 *))
                                        x-%offset%))))
                (setf k (f2cl-lib:int-sub k 1)))))
            (t
             (if nounit
                 (setf temp
                         (* temp
                            (f2cl-lib:dconjg
                             (f2cl-lib:fref ap-%data%
                                        (kk)
                                        ((1 *))
                                        ap-%offset%)))))
             (f2cl-lib:fdo (i
                            (f2cl-lib:int-add j
                                              (f2cl-lib:int-sub 1))
                            (f2cl-lib:int-add i
                                              (f2cl-lib:int-sub 1)))
                           ((> i 1) nil)
               (tagbody
                 (setf temp
                         (+ temp
                            (*
                             (f2cl-lib:dconjg
                              (f2cl-lib:fref ap-%data%
                                         (k)
                                         ((1 *))
                                         ap-%offset%))
                             (f2cl-lib:fref x-%data%
                                        (i)
                                        ((1 *))
                                        x-%offset%))))
                 (setf k (f2cl-lib:int-sub k 1))))))
         (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                temp)
         (setf kk (f2cl-lib:int-sub kk j)))))
    (t
     (setf jx
             (f2cl-lib:int-add kx
                               (f2cl-lib:int-mul
                                (f2cl-lib:int-sub n 1)
                                incx)))
     (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   ((> j 1) nil)
       (tagbody
         (setf temp
```

```
        (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
(setf ix jx)
(cond
  (noconj
   (if nounit
       (setf temp
               (* temp
                  (f2cl-lib:fref ap-%data%
                                 (kk)
                                 ((1 *))
                                 ap-%offset%))))
   (f2cl-lib:fdo (k
                  (f2cl-lib:int-add kk
                                    (f2cl-lib:int-sub 1))
                  (f2cl-lib:int-add k
                                    (f2cl-lib:int-sub 1)))
                 ((> k
                     (f2cl-lib:int-add kk
                                       (f2cl-lib:int-sub
                                        j)
                                       1))
                  nil)
     (tagbody
       (setf ix (f2cl-lib:int-sub ix incx))
       (setf temp
               (+ temp
                  (*
                   (f2cl-lib:fref ap-%data%
                                  (k)
                                  ((1 *))
                                  ap-%offset%)
                   (f2cl-lib:fref x-%data%
                                  (ix)
                                  ((1 *))
                                  x-%offset%)))))))
  (t
   (if nounit
       (setf temp
               (* temp
                  (f2cl-lib:dconjg
                   (f2cl-lib:fref ap-%data%
                                  (kk)
                                  ((1 *))
                                  ap-%offset%)))))
   (f2cl-lib:fdo (k
                  (f2cl-lib:int-add kk
                                    (f2cl-lib:int-sub 1))
                  (f2cl-lib:int-add k
                                    (f2cl-lib:int-sub 1)))
                 ((> k
```

```
                                  (f2cl-lib:int-add kk
                                                    (f2cl-lib:int-sub
                                                     j)
                                                    1))
                              nil)
                    (tagbody
                      (setf ix (f2cl-lib:int-sub ix incx))
                      (setf temp
                              (+ temp
                                 (*
                                  (f2cl-lib:dconjg
                                   (f2cl-lib:fref ap-%data%
                                                  (k)
                                                  ((1 *))
                                                  ap-%offset%))
                                  (f2cl-lib:fref x-%data%
                                                 (ix)
                                                 ((1 *))
                                                 x-%offset%))))))))
              (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                      temp)
              (setf jx (f2cl-lib:int-sub jx incx))
              (setf kk (f2cl-lib:int-sub kk j)))))))
      (t
       (setf kk 1)
       (cond
         ((= incx 1)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf temp
                      (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
              (setf k (f2cl-lib:int-add kk 1))
              (cond
                (noconj
                 (if nounit
                     (setf temp
                             (* temp
                                (f2cl-lib:fref ap-%data%
                                               (kk)
                                               ((1 *))
                                               ap-%offset%))))
                 (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                                  (f2cl-lib:int-add i 1))
                               ((> i n) nil)
                   (tagbody
                     (setf temp
                             (+ temp
                                (*
                                 (f2cl-lib:fref ap-%data%
```

```
                                        (k)
                                        ((1 *))
                                        ap-%offset%)
                              (f2cl-lib:fref x-%data%
                                        (i)
                                        ((1 *))
                                        x-%offset%))))
                 (setf k (f2cl-lib:int-add k 1)))))
            (t
             (if nounit
                 (setf temp
                         (* temp
                            (f2cl-lib:dconjg
                             (f2cl-lib:fref ap-%data%
                                        (kk)
                                        ((1 *))
                                        ap-%offset%)))))
             (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                            (f2cl-lib:int-add i 1))
                           ((> i n) nil)
               (tagbody
                 (setf temp
                         (+ temp
                            (*
                             (f2cl-lib:dconjg
                              (f2cl-lib:fref ap-%data%
                                         (k)
                                         ((1 *))
                                         ap-%offset%))
                             (f2cl-lib:fref x-%data%
                                        (i)
                                        ((1 *))
                                        x-%offset%))))
                 (setf k (f2cl-lib:int-add k 1))))))
          (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                 temp)
          (setf kk
                 (f2cl-lib:int-add kk
                                   (f2cl-lib:int-add
                                    (f2cl-lib:int-sub n j)
                                    1))))))
    (t
     (setf jx kx)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp
                 (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
         (setf ix jx)
         (cond
```

```
(noconj
 (if nounit
     (setf temp
             (* temp
                (f2cl-lib:fref ap-%data%
                               (kk)
                               ((1 *))
                               ap-%offset%))))
 (f2cl-lib:fdo (k (f2cl-lib:int-add kk 1)
                (f2cl-lib:int-add k 1))
               ((> k
                   (f2cl-lib:int-add kk
                                     n
                                     (f2cl-lib:int-sub
                                      j)))
                nil)
   (tagbody
     (setf ix (f2cl-lib:int-add ix incx))
     (setf temp
             (+ temp
                (*
                 (f2cl-lib:fref ap-%data%
                                (k)
                                ((1 *))
                                ap-%offset%)
                 (f2cl-lib:fref x-%data%
                                (ix)
                                ((1 *))
                                x-%offset%)))))))
(t
 (if nounit
     (setf temp
             (* temp
                (f2cl-lib:dconjg
                 (f2cl-lib:fref ap-%data%
                                (kk)
                                ((1 *))
                                ap-%offset%)))))
 (f2cl-lib:fdo (k (f2cl-lib:int-add kk 1)
                (f2cl-lib:int-add k 1))
               ((> k
                   (f2cl-lib:int-add kk
                                     n
                                     (f2cl-lib:int-sub
                                      j)))
                nil)
   (tagbody
     (setf ix (f2cl-lib:int-add ix incx))
     (setf temp
             (+ temp
```

```
                                        (*
                                         (f2cl-lib:dconjg
                                          (f2cl-lib:fref ap-%data%
                                                         (k)
                                                         ((1 *))
                                                         ap-%offset%))
                                         (f2cl-lib:fref x-%data%
                                                        (ix)
                                                        ((1 *))
                                                        x-%offset%))))))))
                    (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                            temp)
                    (setf jx (f2cl-lib:int-add jx incx))
                    (setf kk
                            (f2cl-lib:int-add kk
                                              (f2cl-lib:int-add
                                               (f2cl-lib:int-sub n j)
                                               1))))))))))
 end_label
        (return (values nil nil nil nil nil nil nil))))))
```

———-

13.15 ztpsv - solves systems of equations $Ax = b$ or $A^T x = b$ or $A^H x = b$

13.15.1 Axiom unit tests

— **ztpsv.input** —

```
)set break resume
)sys rm -f ztpsv.output
)spool ztpsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

13.15.2 Axiom help page

— ztpsv.help —

```
=====================================================================
ztpsv examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

 NAME
      ZTPSV - solve one of the systems of equations
      A*x = b, or A'*x = b, or conjg( A' )*x = b,

 SYNOPSIS
      SUBROUTINE ZTPSV ( UPLO, TRANS, DIAG, N, AP, X, INCX )

          INTEGER      INCX, N

          CHARACTER*1  DIAG, TRANS, UPLO

          COMPLEX*16   AP( * ), X( * )

 PURPOSE
      ZTPSV  solves one of the systems of equations

      where b and x are n element vectors and A is an n by n unit,
      or non-unit, upper or lower triangular matrix, supplied in
      packed form.

      No test for singularity or near-singularity is included in
      this routine. Such tests must be performed before calling
      this routine.

 PARAMETERS
      UPLO   - CHARACTER*1.
               On entry, UPLO specifies whether the matrix is an
               upper or lower triangular matrix as follows:

               UPLO = 'U' or 'u'   A is an upper triangular matrix.

               UPLO = 'L' or 'l'   A is a lower triangular matrix.

               Unchanged on exit.

      TRANS  - CHARACTER*1.
               On entry, TRANS specifies the equations to be solved
               as follows:

               TRANS = 'N' or 'n'   A*x = b.
```

```
         TRANS = 'T' or 't'   A'*x = b.

         TRANS = 'C' or 'c'   conjg( A' )*x = b.

         Unchanged on exit.

DIAG   - CHARACTER*1.
         On entry, DIAG specifies whether or not A is unit
         triangular as follows:

         DIAG = 'U' or 'u'   A is assumed to be unit triangu-
         lar.

         DIAG = 'N' or 'n'   A is not assumed to be unit tri-
         angular.

         Unchanged on exit.

N      - INTEGER.
         On entry, N specifies the order of the matrix A.  N
         must be at least zero.  Unchanged on exit.

AP     - COMPLEX*16       array of DIMENSION at least
         ( ( n*( n + 1 ) )/2 ).  Before entry with  UPLO = 'U'
         or 'u', the array AP must contain the upper triangu-
         lar matrix packed sequentially, column by column, so
         that AP( 1 ) contains a( 1, 1 ), AP( 2 ) and AP( 3 )
         contain a( 1, 2 ) and a( 2, 2 ) respectively, and so
         on.  Before entry with UPLO = 'L' or 'l', the array
         AP must contain the lower triangular matrix packed
         sequentially, column by column, so that AP( 1 ) con-
         tains a( 1, 1 ), AP( 2 ) and AP( 3 ) contain a( 2, 1
         ) and a( 3, 1 ) respectively, and so on.  Note that
         when  DIAG = 'U' or 'u', the diagonal elements of A
         are not referenced, but are assumed to be unity.
         Unchanged on exit.

X      - COMPLEX*16       array of dimension at least
         ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
         incremented array X must contain the n element
         right-hand side vector b. On exit, X is overwritten
         with the solution vector x.

INCX   - INTEGER.
         On entry, INCX specifies the increment for the ele-
         ments of X. INCX must not be zero.  Unchanged on
         exit.
```

———-

13.15.3 fortran code

— ztpsv.f —

```
*  ====================================================================
      SUBROUTINE ZTPSV(UPLO,TRANS,DIAG,N,AP,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16 AP(*),X(*)
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP
      INTEGER I,INFO,IX,J,JX,K,KK,KX
      LOGICAL NOCONJ,NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DCONJG
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
```

```
         INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
         INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
         INFO = 3
      ELSE IF (N.LT.0) THEN
         INFO = 4
      ELSE IF (INCX.EQ.0) THEN
         INFO = 7
      END IF
      IF (INFO.NE.0) THEN
         CALL XERBLA('ZTPSV ',INFO)
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOCONJ = LSAME(TRANS,'T')
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX  too small for descending loops.
*
      IF (INCX.LE.0) THEN
         KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
         KX = 1
      END IF
*
*     Start the operations. In this version the elements of AP are
*     accessed sequentially with one pass through AP.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x := inv( A )*x.
*
          IF (LSAME(UPLO,'U')) THEN
              KK = (N* (N+1))/2
              IF (INCX.EQ.1) THEN
                  DO 20 J = N,1,-1
                      IF (X(J).NE.ZERO) THEN
                          IF (NOUNIT) X(J) = X(J)/AP(KK)
                          TEMP = X(J)
                          K = KK - 1
                          DO 10 I = J - 1,1,-1
                              X(I) = X(I) - TEMP*AP(K)
                              K = K - 1
```

```
10                  CONTINUE
                END IF
                KK = KK - J
20          CONTINUE
        ELSE
            JX = KX + (N-1)*INCX
            DO 40 J = N,1,-1
                IF (X(JX).NE.ZERO) THEN
                    IF (NOUNIT) X(JX) = X(JX)/AP(KK)
                    TEMP = X(JX)
                    IX = JX
                    DO 30 K = KK - 1,KK - J + 1,-1
                        IX = IX - INCX
                        X(IX) = X(IX) - TEMP*AP(K)
30                  CONTINUE
                END IF
                JX = JX - INCX
                KK = KK - J
40          CONTINUE
        END IF
    ELSE
        KK = 1
        IF (INCX.EQ.1) THEN
            DO 60 J = 1,N
                IF (X(J).NE.ZERO) THEN
                    IF (NOUNIT) X(J) = X(J)/AP(KK)
                    TEMP = X(J)
                    K = KK + 1
                    DO 50 I = J + 1,N
                        X(I) = X(I) - TEMP*AP(K)
                        K = K + 1
50                  CONTINUE
                END IF
                KK = KK + (N-J+1)
60          CONTINUE
        ELSE
            JX = KX
            DO 80 J = 1,N
                IF (X(JX).NE.ZERO) THEN
                    IF (NOUNIT) X(JX) = X(JX)/AP(KK)
                    TEMP = X(JX)
                    IX = JX
                    DO 70 K = KK + 1,KK + N - J
                        IX = IX + INCX
                        X(IX) = X(IX) - TEMP*AP(K)
70                  CONTINUE
                END IF
                JX = JX + INCX
                KK = KK + (N-J+1)
80          CONTINUE
```

```
            END IF
        END IF
    ELSE
*
*       Form  x := inv( A**T )*x  or  x := inv( A**H )*x.
*
        IF (LSAME(UPLO,'U')) THEN
            KK = 1
            IF (INCX.EQ.1) THEN
                DO 110 J = 1,N
                    TEMP = X(J)
                    K = KK
                    IF (NOCONJ) THEN
                        DO 90 I = 1,J - 1
                            TEMP = TEMP - AP(K)*X(I)
                            K = K + 1
 90                     CONTINUE
                        IF (NOUNIT) TEMP = TEMP/AP(KK+J-1)
                    ELSE
                        DO 100 I = 1,J - 1
                            TEMP = TEMP - DCONJG(AP(K))*X(I)
                            K = K + 1
100                     CONTINUE
                        IF (NOUNIT) TEMP = TEMP/DCONJG(AP(KK+J-1))
                    END IF
                    X(J) = TEMP
                    KK = KK + J
110             CONTINUE
            ELSE
                JX = KX
                DO 140 J = 1,N
                    TEMP = X(JX)
                    IX = KX
                    IF (NOCONJ) THEN
                        DO 120 K = KK,KK + J - 2
                            TEMP = TEMP - AP(K)*X(IX)
                            IX = IX + INCX
120                     CONTINUE
                        IF (NOUNIT) TEMP = TEMP/AP(KK+J-1)
                    ELSE
                        DO 130 K = KK,KK + J - 2
                            TEMP = TEMP - DCONJG(AP(K))*X(IX)
                            IX = IX + INCX
130                     CONTINUE
                        IF (NOUNIT) TEMP = TEMP/DCONJG(AP(KK+J-1))
                    END IF
                    X(JX) = TEMP
                    JX = JX + INCX
                    KK = KK + J
140             CONTINUE
```

```
            END IF
        ELSE
            KK = (N* (N+1))/2
            IF (INCX.EQ.1) THEN
                DO 170 J = N,1,-1
                    TEMP = X(J)
                    K = KK
                    IF (NOCONJ) THEN
                        DO 150 I = N,J + 1,-1
                            TEMP = TEMP - AP(K)*X(I)
                            K = K - 1
 150                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/AP(KK-N+J)
                    ELSE
                        DO 160 I = N,J + 1,-1
                            TEMP = TEMP - DCONJG(AP(K))*X(I)
                            K = K - 1
 160                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/DCONJG(AP(KK-N+J))
                    END IF
                    X(J) = TEMP
                    KK = KK - (N-J+1)
 170            CONTINUE
            ELSE
                KX = KX + (N-1)*INCX
                JX = KX
                DO 200 J = N,1,-1
                    TEMP = X(JX)
                    IX = KX
                    IF (NOCONJ) THEN
                        DO 180 K = KK,KK - (N- (J+1)),-1
                            TEMP = TEMP - AP(K)*X(IX)
                            IX = IX - INCX
 180                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/AP(KK-N+J)
                    ELSE
                        DO 190 K = KK,KK - (N- (J+1)),-1
                            TEMP = TEMP - DCONJG(AP(K))*X(IX)
                            IX = IX - INCX
 190                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/DCONJG(AP(KK-N+J))
                    END IF
                    X(JX) = TEMP
                    JX = JX - INCX
                    KK = KK - (N-J+1)
 200            CONTINUE
            END IF
        END IF
    END IF
*
```

```
      RETURN
*
*     End of ZTPSV .
*
      END
```

———-

13.15.4 lisp code

— BLAS 2 ztpsv —

```
(let* ((zero (complex 0.0 0.0)))
  (declare (type (complex double-float) zero))
  (defun ztpsv (uplo trans diag n ap x incx)
    (declare (type (simple-array (complex double-float) (*)) x ap)
             (type fixnum incx n)
             (type character diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (ap (complex double-float) ap-%data% ap-%offset%)
         (x (complex double-float) x-%data% x-%offset%))
      (prog ((noconj nil) (nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0) (k 0)
             (kk 0) (kx 0) (temp #C(0.0 0.0)))
        (declare (type (member t nil) noconj nounit)
                 (type fixnum i info ix j jx k kk kx)
                 (type (complex double-float) temp))
        (setf info 0)
        (cond
          ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
           (setf info 1))
          ((and (not (char-equal trans #\N))
                (not (char-equal trans #\T))
                (not (char-equal trans #\C)))
           (setf info 2))
          ((and (not (char-equal diag #\U)) (not (char-equal diag #\N)))
           (setf info 3))
          ((< n 0)
           (setf info 4))
          ((= incx 0)
           (setf info 7)))
        (cond
          ((/= info 0)
           (error
            " ** On entry to ~a parameter number ~a had an illegal value~%"
```

```
    "ZTPSV" info)
   (go end_label)))
(if (= n 0) (go end_label))
(setf noconj (char-equal trans #\T))
(setf nounit (char-equal diag #\N))
(cond
  ((<= incx 0)
   (setf kx
           (f2cl-lib:int-sub 1
                             (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                               incx))))
  ((/= incx 1)
   (setf kx 1)))
(cond
  ((char-equal trans #\N)
   (cond
     ((char-equal uplo #\U)
      (setf kk (the fixnum (truncate (* n (+ n 1)) 2)))
      (cond
        ((= incx 1)
         (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((> j 1) nil)
           (tagbody
             (cond
               ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
                (if nounit
                    (setf (f2cl-lib:fref x-%data%
                                         (j)
                                         ((1 *))
                                         x-%offset%)
                          (/
                           (f2cl-lib:fref x-%data%
                                          (j)
                                          ((1 *))
                                          x-%offset%)
                           (f2cl-lib:fref ap-%data%
                                          (kk)
                                          ((1 *))
                                          ap-%offset%))))
                (setf temp
                        (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                (setf k (f2cl-lib:int-sub kk 1))
                (f2cl-lib:fdo (i
                               (f2cl-lib:int-add j
                                                 (f2cl-lib:int-sub 1))
                               (f2cl-lib:int-add i
                                                 (f2cl-lib:int-sub 1)))
                              ((> i 1) nil)
                  (tagbody
                    (setf (f2cl-lib:fref x-%data%
```

```
                                        (i)
                                        ((1 *))
                                        x-%offset%)
                                (-
                                 (f2cl-lib:fref x-%data%
                                                (i)
                                                ((1 *))
                                                x-%offset%)
                                 (* temp
                                    (f2cl-lib:fref ap-%data%
                                                   (k)
                                                   ((1 *))
                                                   ap-%offset%))))
                        (setf k (f2cl-lib:int-sub k 1))))))
                    (setf kk (f2cl-lib:int-sub kk j)))))
                (t
                 (setf jx
                         (f2cl-lib:int-add kx
                                           (f2cl-lib:int-mul
                                            (f2cl-lib:int-sub n 1)
                                            incx)))
                 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                               ((> j 1) nil)
                   (tagbody
                     (cond
                       ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
                        (if nounit
                            (setf (f2cl-lib:fref x-%data%
                                                 (jx)
                                                 ((1 *))
                                                 x-%offset%)
                                  (/
                                   (f2cl-lib:fref x-%data%
                                                  (jx)
                                                  ((1 *))
                                                  x-%offset%)
                                   (f2cl-lib:fref ap-%data%
                                                  (kk)
                                                  ((1 *))
                                                  ap-%offset%))))
                        (setf temp
                                (f2cl-lib:fref x-%data%
                                               (jx)
                                               ((1 *))
                                               x-%offset%))
                        (setf ix jx)
                        (f2cl-lib:fdo (k
                                       (f2cl-lib:int-add kk
                                                         (f2cl-lib:int-sub 1))
                                       (f2cl-lib:int-add k
```

```
                                                    (f2cl-lib:int-sub 1)))
                                  ((> k
                                      (f2cl-lib:int-add kk
                                                        (f2cl-lib:int-sub
                                                         j)
                                                        1))
                                   nil)
                        (tagbody
                          (setf ix (f2cl-lib:int-sub ix incx))
                          (setf (f2cl-lib:fref x-%data%
                                               (ix)
                                               ((1 *))
                                               x-%offset%)
                                (-
                                 (f2cl-lib:fref x-%data%
                                                (ix)
                                                ((1 *))
                                                x-%offset%)
                                 (* temp
                                    (f2cl-lib:fref ap-%data%
                                                   (k)
                                                   ((1 *))
                                                   ap-%offset%)))))))
                  (setf jx (f2cl-lib:int-sub jx incx))
                  (setf kk (f2cl-lib:int-sub kk j)))))))
        (t
         (setf kk 1)
         (cond
           ((= incx 1)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (cond
                  ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
                   (if nounit
                       (setf (f2cl-lib:fref x-%data%
                                            (j)
                                            ((1 *))
                                            x-%offset%)
                             (/
                              (f2cl-lib:fref x-%data%
                                             (j)
                                             ((1 *))
                                             x-%offset%)
                              (f2cl-lib:fref ap-%data%
                                             (kk)
                                             ((1 *))
                                             ap-%offset%))))
                   (setf temp
                           (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
```

```
                (setf k (f2cl-lib:int-add kk 1))
                (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                               (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf (f2cl-lib:fref x-%data%
                                         (i)
                                         ((1 *))
                                         x-%offset%)
                          (-
                           (f2cl-lib:fref x-%data%
                                          (i)
                                          ((1 *))
                                          x-%offset%)
                           (* temp
                              (f2cl-lib:fref ap-%data%
                                             (k)
                                             ((1 *))
                                             ap-%offset%))))
                    (setf k (f2cl-lib:int-add k 1))))))
            (setf kk
                    (f2cl-lib:int-add kk
                                      (f2cl-lib:int-add
                                       (f2cl-lib:int-sub n j)
                                       1))))))
        (t
         (setf jx kx)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (cond
               ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
                (if nounit
                    (setf (f2cl-lib:fref x-%data%
                                         (jx)
                                         ((1 *))
                                         x-%offset%)
                          (/
                           (f2cl-lib:fref x-%data%
                                          (jx)
                                          ((1 *))
                                          x-%offset%)
                           (f2cl-lib:fref ap-%data%
                                          (kk)
                                          ((1 *))
                                          ap-%offset%))))
                (setf temp
                        (f2cl-lib:fref x-%data%
                                       (jx)
                                       ((1 *))
```

```
                                    x-%offset%))
                (setf ix jx)
                (f2cl-lib:fdo (k (f2cl-lib:int-add kk 1)
                                 (f2cl-lib:int-add k 1))
                               ((> k
                                   (f2cl-lib:int-add kk
                                                     n
                                                     (f2cl-lib:int-sub
                                                      j)))
                                 nil)
                  (tagbody
                    (setf ix (f2cl-lib:int-add ix incx))
                    (setf (f2cl-lib:fref x-%data%
                                         (ix)
                                         ((1 *))
                                         x-%offset%)
                          (-
                           (f2cl-lib:fref x-%data%
                                          (ix)
                                          ((1 *))
                                          x-%offset%)
                           (* temp
                              (f2cl-lib:fref ap-%data%
                                             (k)
                                             ((1 *))
                                             ap-%offset%))))))))
            (setf jx (f2cl-lib:int-add jx incx))
            (setf kk
                    (f2cl-lib:int-add kk
                                      (f2cl-lib:int-add
                                       (f2cl-lib:int-sub n j)
                                       1))))))))))
(t
 (cond
   ((char-equal uplo #\U)
    (setf kk 1)
    (cond
      ((= incx 1)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (setf temp
                   (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
           (setf k kk)
           (cond
             (noconj
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i
                                (f2cl-lib:int-add j
                                                  (f2cl-lib:int-sub
```

```
                                        1)))
                  nil)
      (tagbody
        (setf temp
                (- temp
                   (*
                    (f2cl-lib:fref ap-%data%
                                   (k)
                                   ((1 *))
                                   ap-%offset%)
                    (f2cl-lib:fref x-%data%
                                   (i)
                                   ((1 *))
                                   x-%offset%))))
        (setf k (f2cl-lib:int-add k 1))))
    (if nounit
        (setf temp
                (/ temp
                   (f2cl-lib:fref ap-%data%
                                  ((f2cl-lib:int-sub
                                    (f2cl-lib:int-add kk j)
                                    1))
                                  ((1 *))
                                  ap-%offset%)))))
   (t
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i
                      (f2cl-lib:int-add j
                                        (f2cl-lib:int-sub
                                         1)))
                  nil)
      (tagbody
        (setf temp
                (- temp
                   (*
                    (f2cl-lib:dconjg
                     (f2cl-lib:fref ap-%data%
                                    (k)
                                    ((1 *))
                                    ap-%offset%))
                    (f2cl-lib:fref x-%data%
                                   (i)
                                   ((1 *))
                                   x-%offset%))))
        (setf k (f2cl-lib:int-add k 1))))
    (if nounit
        (setf temp
                (/ temp
                   (f2cl-lib:dconjg
                    (f2cl-lib:fref ap-%data%
```

```
                                        ((f2cl-lib:int-sub
                                          (f2cl-lib:int-add kk
                                                            j)
                                          1))
                                        ((1 *))
                                        ap-%offset%)))))))
      (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
              temp)
      (setf kk (f2cl-lib:int-add kk j)))))
(t
 (setf jx kx)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf temp
             (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
     (setf ix kx)
     (cond
       (noconj
        (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
                      ((> k
                          (f2cl-lib:int-add kk
                                            j
                                            (f2cl-lib:int-sub
                                             2)))
                       nil)
          (tagbody
            (setf temp
                    (- temp
                       (*
                        (f2cl-lib:fref ap-%data%
                                       (k)
                                       ((1 *))
                                       ap-%offset%)
                        (f2cl-lib:fref x-%data%
                                       (ix)
                                       ((1 *))
                                       x-%offset%))))
            (setf ix (f2cl-lib:int-add ix incx))))
        (if nounit
            (setf temp
                    (/ temp
                       (f2cl-lib:fref ap-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add kk j)
                                        1))
                                      ((1 *))
                                      ap-%offset%)))))
       (t
        (f2cl-lib:fdo (k kk (f2cl-lib:int-add k 1))
```

```
                                ((> k
                                    (f2cl-lib:int-add kk
                                                      j
                                                      (f2cl-lib:int-sub
                                                       2)))
                                 nil)
                        (tagbody
                          (setf temp
                                  (- temp
                                     (*
                                      (f2cl-lib:dconjg
                                       (f2cl-lib:fref ap-%data%
                                                      (k)
                                                      ((1 *))
                                                      ap-%offset%))
                                      (f2cl-lib:fref x-%data%
                                                     (ix)
                                                     ((1 *))
                                                     x-%offset%))))
                          (setf ix (f2cl-lib:int-add ix incx))))
                      (if nounit
                          (setf temp
                                  (/ temp
                                     (f2cl-lib:dconjg
                                      (f2cl-lib:fref ap-%data%
                                                     ((f2cl-lib:int-sub
                                                       (f2cl-lib:int-add kk
                                                                         j)
                                                       1))
                                                     ((1 *))
                                                     ap-%offset%)))))))
                  (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                          temp)
                  (setf jx (f2cl-lib:int-add jx incx))
                  (setf kk (f2cl-lib:int-add kk j)))))))
          (t
           (setf kk (the fixnum (truncate (* n (+ n 1)) 2)))
           (cond
             ((= incx 1)
              (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                            ((> j 1) nil)
                (tagbody
                  (setf temp
                          (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                  (setf k kk)
                  (cond
                    (noconj
                     (f2cl-lib:fdo (i n
                                    (f2cl-lib:int-add i
                                                      (f2cl-lib:int-sub 1)))
```

```
                ((> i (f2cl-lib:int-add j 1)) nil)
    (tagbody
      (setf temp
              (- temp
                 (*
                  (f2cl-lib:fref ap-%data%
                                 (k)
                                 ((1 *))
                                 ap-%offset%)
                  (f2cl-lib:fref x-%data%
                                 (i)
                                 ((1 *))
                                 x-%offset%))))
      (setf k (f2cl-lib:int-sub k 1))))
  (if nounit
      (setf temp
              (/ temp
                 (f2cl-lib:fref ap-%data%
                                ((f2cl-lib:int-add
                                  (f2cl-lib:int-sub kk n)
                                  j))
                                ((1 *))
                                ap-%offset%)))))
 (t
  (f2cl-lib:fdo (i n
                 (f2cl-lib:int-add i
                                   (f2cl-lib:int-sub 1)))
                ((> i (f2cl-lib:int-add j 1)) nil)
    (tagbody
      (setf temp
              (- temp
                 (*
                  (f2cl-lib:dconjg
                   (f2cl-lib:fref ap-%data%
                                  (k)
                                  ((1 *))
                                  ap-%offset%))
                  (f2cl-lib:fref x-%data%
                                 (i)
                                 ((1 *))
                                 x-%offset%))))
      (setf k (f2cl-lib:int-sub k 1))))
  (if nounit
      (setf temp
              (/ temp
                 (f2cl-lib:dconjg
                  (f2cl-lib:fref ap-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-sub kk
                                                     n)
```

```
                                                j))
                                              ((1 *))
                                              ap-%offset%)))))))
                (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                        temp)
                (setf kk
                        (f2cl-lib:int-sub kk
                                        (f2cl-lib:int-add
                                         (f2cl-lib:int-sub n j)
                                         1))))))
            (t
             (setf kx
                     (f2cl-lib:int-add kx
                                     (f2cl-lib:int-mul
                                      (f2cl-lib:int-sub n 1)
                                      incx)))
             (setf jx kx)
             (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                           ((> j 1) nil)
               (tagbody
                 (setf temp
                         (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
                 (setf ix kx)
                 (cond
                   (noconj
                    (f2cl-lib:fdo (k kk
                                   (f2cl-lib:int-add k
                                                     (f2cl-lib:int-sub 1)))
                                  ((> k
                                      (f2cl-lib:int-add kk
                                                        (f2cl-lib:int-sub
                                                         (f2cl-lib:int-add
                                                          n
                                                          (f2cl-lib:int-sub
                                                           (f2cl-lib:int-add
                                                            j
                                                            1))))))
                                   nil)
                     (tagbody
                       (setf temp
                               (- temp
                                  (*
                                   (f2cl-lib:fref ap-%data%
                                                  (k)
                                                  ((1 *))
                                                  ap-%offset%)
                                   (f2cl-lib:fref x-%data%
                                                  (ix)
                                                  ((1 *))
                                                  x-%offset%))))
```

```
            (setf ix (f2cl-lib:int-sub ix incx))))
        (if nounit
            (setf temp
                    (/ temp
                       (f2cl-lib:fref ap-%data%
                                      ((f2cl-lib:int-add
                                        (f2cl-lib:int-sub kk n)
                                        j))
                                      ((1 *))
                                      ap-%offset%)))))
       (t
        (f2cl-lib:fdo (k kk
                       (f2cl-lib:int-add k
                                         (f2cl-lib:int-sub 1)))
                      ((> k
                          (f2cl-lib:int-add kk
                                            (f2cl-lib:int-sub
                                             (f2cl-lib:int-add
                                              n
                                              (f2cl-lib:int-sub
                                               (f2cl-lib:int-add
                                                j
                                                 1))))))
                       nil)
          (tagbody
            (setf temp
                    (- temp
                       (*
                        (f2cl-lib:dconjg
                         (f2cl-lib:fref ap-%data%
                                        (k)
                                        ((1 *))
                                        ap-%offset%))
                        (f2cl-lib:fref x-%data%
                                       (ix)
                                       ((1 *))
                                       x-%offset%))))
            (setf ix (f2cl-lib:int-sub ix incx))))
        (if nounit
            (setf temp
                    (/ temp
                       (f2cl-lib:dconjg
                        (f2cl-lib:fref ap-%data%
                                       ((f2cl-lib:int-add
                                         (f2cl-lib:int-sub kk
                                                           n)
                                         j))
                                       ((1 *))
                                       ap-%offset%)))))))
    (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
```

```
                        temp)
                (setf jx (f2cl-lib:int-sub jx incx))
                (setf kk
                        (f2cl-lib:int-sub kk
                                          (f2cl-lib:int-add
                                           (f2cl-lib:int-sub n j)
                                           1))))))))))
 end_label
        (return (values nil nil nil nil nil nil nil))))))
```

———-

13.16 ztrmv - triangular matrix-vector Ax or $A^T x$ or $A^H x$

13.16.1 Axiom unit tests

— **ztrmv.input** —

```
)set break resume
)sys rm -f ztrmv.output
)spool ztrmv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

13.16.2 Axiom help page

— **ztrmv.help** —

```
======================================================================
ztrmv examples
======================================================================

======================================================================
Man Page Details
======================================================================

 NAME
```

```
     ZTRMV - perform one of the matrix-vector operations
     x := A*x, or x := A'*x, or x := conjg( A' )*x,

SYNOPSIS
     SUBROUTINE ZTRMV ( UPLO, TRANS, DIAG, N, A, LDA, X, INCX )

         INTEGER      INCX, LDA, N

         CHARACTER*1  DIAG, TRANS, UPLO

         COMPLEX*16   A( LDA, * ), X( * )

PURPOSE
     ZTRMV  performs one of the matrix-vector operations

     where x is an n element vector and  A is an n by n unit, or
     non-unit, upper or lower triangular matrix.

PARAMETERS
     UPLO   - CHARACTER*1.
            On entry, UPLO specifies whether the matrix is an
            upper or lower triangular matrix as follows:

            UPLO = 'U' or 'u'   A is an upper triangular matrix.

            UPLO = 'L' or 'l'   A is a lower triangular matrix.

            Unchanged on exit.

     TRANS  - CHARACTER*1.
            On entry, TRANS specifies the operation to be per-
            formed as follows:

            TRANS = 'N' or 'n'   x := A*x.

            TRANS = 'T' or 't'   x := A'*x.

            TRANS = 'C' or 'c'   x := conjg( A' )*x.

            Unchanged on exit.

     DIAG   - CHARACTER*1.
            On entry, DIAG specifies whether or not A is unit
            triangular as follows:

            DIAG = 'U' or 'u'   A is assumed to be unit triangu-
            lar.

            DIAG = 'N' or 'n'   A is not assumed to be unit tri-
            angular.
```

```
            Unchanged on exit.

      N     - INTEGER.
            On entry, N specifies the order of the matrix A.  N
            must be at least zero.  Unchanged on exit.

      A     - COMPLEX*16       array of DIMENSION ( LDA, n ).
            Before entry with  UPLO = 'U' or 'u', the leading n
            by n upper triangular part of the array A must con-
            tain the upper triangular matrix and the strictly
            lower triangular part of A is not referenced.  Before
            entry with UPLO = 'L' or 'l', the leading n by n
            lower triangular part of the array A must contain the
            lower triangular matrix and the strictly upper tri-
            angular part of A is not referenced.  Note that when
            DIAG = 'U' or 'u', the diagonal elements of A are not
            referenced either, but are assumed to be unity.
            Unchanged on exit.

      LDA   - INTEGER.
            On entry, LDA specifies the first dimension of A as
            declared in the calling (sub) program. LDA must be at
            least max( 1, n ).  Unchanged on exit.

      X     - COMPLEX*16       array of dimension at least
            ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
            incremented array X must contain the n element vector
            x. On exit, X is overwritten with the transformed
            vector x.

      INCX  - INTEGER.
            On entry, INCX specifies the increment for the ele-
            ments of X. INCX must not be zero.  Unchanged on
            exit.
```

———-

13.16.3 fortran code

— ztrmv.f —

```
*  ====================================================================
      SUBROUTINE ZTRMV(UPLO,TRANS,DIAG,N,A,LDA,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
```

```
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,LDA,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),X(*)
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP
      INTEGER I,INFO,IX,J,JX,KX
      LOGICAL NOCONJ,NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DCONJG,MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 6
      ELSE IF (INCX.EQ.0) THEN
          INFO = 8
      END IF
```

```
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZTRMV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
*
      NOCONJ = LSAME(TRANS,'T')
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX  too small for descending loops.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x := A*x.
*
          IF (LSAME(UPLO,'U')) THEN
              IF (INCX.EQ.1) THEN
                  DO 20 J = 1,N
                      IF (X(J).NE.ZERO) THEN
                          TEMP = X(J)
                          DO 10 I = 1,J - 1
                              X(I) = X(I) + TEMP*A(I,J)
   10                     CONTINUE
                          IF (NOUNIT) X(J) = X(J)*A(J,J)
                      END IF
   20             CONTINUE
              ELSE
                  JX = KX
                  DO 40 J = 1,N
                      IF (X(JX).NE.ZERO) THEN
                          TEMP = X(JX)
                          IX = KX
                          DO 30 I = 1,J - 1
                              X(IX) = X(IX) + TEMP*A(I,J)
                              IX = IX + INCX
   30                     CONTINUE
                          IF (NOUNIT) X(JX) = X(JX)*A(J,J)
```

```
                    END IF
                    JX = JX + INCX
   40            CONTINUE
            END IF
         ELSE
            IF (INCX.EQ.1) THEN
               DO 60 J = N,1,-1
                    IF (X(J).NE.ZERO) THEN
                        TEMP = X(J)
                        DO 50 I = N,J + 1,-1
                            X(I) = X(I) + TEMP*A(I,J)
   50                   CONTINUE
                        IF (NOUNIT) X(J) = X(J)*A(J,J)
                    END IF
   60            CONTINUE
            ELSE
               KX = KX + (N-1)*INCX
               JX = KX
               DO 80 J = N,1,-1
                    IF (X(JX).NE.ZERO) THEN
                        TEMP = X(JX)
                        IX = KX
                        DO 70 I = N,J + 1,-1
                            X(IX) = X(IX) + TEMP*A(I,J)
                            IX = IX - INCX
   70                   CONTINUE
                        IF (NOUNIT) X(JX) = X(JX)*A(J,J)
                    END IF
                    JX = JX - INCX
   80            CONTINUE
            END IF
         END IF
      ELSE
*
*        Form  x := A**T*x  or  x := A**H*x.
*
         IF (LSAME(UPLO,'U')) THEN
            IF (INCX.EQ.1) THEN
               DO 110 J = N,1,-1
                    TEMP = X(J)
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*A(J,J)
                        DO 90 I = J - 1,1,-1
                            TEMP = TEMP + A(I,J)*X(I)
   90                   CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*DCONJG(A(J,J))
                        DO 100 I = J - 1,1,-1
                            TEMP = TEMP + DCONJG(A(I,J))*X(I)
  100                   CONTINUE
```

```
                    END IF
                    X(J) = TEMP
 110            CONTINUE
            ELSE
                JX = KX + (N-1)*INCX
                DO 140 J = N,1,-1
                    TEMP = X(JX)
                    IX = JX
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*A(J,J)
                        DO 120 I = J - 1,1,-1
                            IX = IX - INCX
                            TEMP = TEMP + A(I,J)*X(IX)
 120                    CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*DCONJG(A(J,J))
                        DO 130 I = J - 1,1,-1
                            IX = IX - INCX
                            TEMP = TEMP + DCONJG(A(I,J))*X(IX)
 130                    CONTINUE
                    END IF
                    X(JX) = TEMP
                    JX = JX - INCX
 140            CONTINUE
            END IF
        ELSE
            IF (INCX.EQ.1) THEN
                DO 170 J = 1,N
                    TEMP = X(J)
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*A(J,J)
                        DO 150 I = J + 1,N
                            TEMP = TEMP + A(I,J)*X(I)
 150                    CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*DCONJG(A(J,J))
                        DO 160 I = J + 1,N
                            TEMP = TEMP + DCONJG(A(I,J))*X(I)
 160                    CONTINUE
                    END IF
                    X(J) = TEMP
 170            CONTINUE
            ELSE
                JX = KX
                DO 200 J = 1,N
                    TEMP = X(JX)
                    IX = JX
                    IF (NOCONJ) THEN
                        IF (NOUNIT) TEMP = TEMP*A(J,J)
                        DO 180 I = J + 1,N
```

```
                            IX = IX + INCX
                            TEMP = TEMP + A(I,J)*X(IX)
  180                   CONTINUE
                    ELSE
                        IF (NOUNIT) TEMP = TEMP*DCONJG(A(J,J))
                        DO 190 I = J + 1,N
                            IX = IX + INCX
                            TEMP = TEMP + DCONJG(A(I,J))*X(IX)
  190                   CONTINUE
                    END IF
                    X(JX) = TEMP
                    JX = JX + INCX
  200             CONTINUE
            END IF
        END IF
    END IF
*
    RETURN
*
*   End of ZTRMV .
*
    END
```

———-

13.16.4 lisp code

— BLAS 2 ztrmv —

```
(let* ((zero (complex 0.0 0.0)))
  (declare (type (complex double-float) zero))
  (defun ztrmv (uplo trans diag n a lda x incx)
    (declare (type (simple-array (complex double-float) (*)) x a)
             (type fixnum incx lda n)
             (type character diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a (complex double-float) a-%data% a-%offset%)
         (x (complex double-float) x-%data% x-%offset%))
      (prog ((noconj nil) (nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0)
             (kx 0) (temp #C(0.0 0.0)))
        (declare (type (member t nil) noconj nounit)
                 (type fixnum i info ix j jx kx)
                 (type (complex double-float) temp))
        (setf info 0)
```

```
(cond
  ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
   (setf info 1))
  ((and (not (char-equal trans #\N))
        (not (char-equal trans #\T))
        (not (char-equal trans #\C)))
   (setf info 2))
  ((and (not (char-equal diag #\U)) (not (char-equal diag #\N)))
   (setf info 3))
  ((< n 0)
   (setf info 4))
  ((< lda (max (the fixnum 1) (the fixnum n)))
   (setf info 6))
  ((= incx 0)
   (setf info 8)))
(cond
  ((/= info 0)
   (error
    " ** On entry to ~a parameter number ~a had an illegal value~%"
    "ZTRMV" info)
   (go end_label)))
(if (= n 0) (go end_label))
(setf noconj (char-equal trans #\T))
(setf nounit (char-equal diag #\N))
(cond
  ((<= incx 0)
   (setf kx
           (f2cl-lib:int-sub 1
                             (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                               incx))))
  ((/= incx 1)
   (setf kx 1)))
(cond
  ((char-equal trans #\N)
   (cond
     ((char-equal uplo #\U)
      (cond
        ((= incx 1)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (cond
               ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
                (setf temp
                        (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i
                                  (f2cl-lib:int-add j
                                                    (f2cl-lib:int-sub
                                                     1)))
```

```
                    nil)
            (tagbody
              (setf (f2cl-lib:fref x-%data%
                                   (i)
                                   ((1 *))
                                   x-%offset%)
                    (+
                     (f2cl-lib:fref x-%data%
                                    (i)
                                    ((1 *))
                                    x-%offset%)
                     (* temp
                        (f2cl-lib:fref a-%data%
                                       (i j)
                                       ((1 lda) (1 *))
                                       a-%offset%))))))
          (if nounit
              (setf (f2cl-lib:fref x-%data%
                                   (j)
                                   ((1 *))
                                   x-%offset%)
                    (*
                     (f2cl-lib:fref x-%data%
                                    (j)
                                    ((1 *))
                                    x-%offset%)
                     (f2cl-lib:fref a-%data%
                                    (j j)
                                    ((1 lda) (1 *))
                                    a-%offset%)))))))))
(t
 (setf jx kx)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
        (setf temp
                (f2cl-lib:fref x-%data%
                               (jx)
                               ((1 *))
                               x-%offset%))
        (setf ix kx)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i
                          (f2cl-lib:int-add j
                                            (f2cl-lib:int-sub
                                             1)))
                       nil)
          (tagbody
```

```
                    (setf (f2cl-lib:fref x-%data%
                                         (ix)
                                         ((1 *))
                                         x-%offset%)
                          (+
                           (f2cl-lib:fref x-%data%
                                          (ix)
                                          ((1 *))
                                          x-%offset%)
                           (* temp
                              (f2cl-lib:fref a-%data%
                                             (i j)
                                             ((1 lda) (1 *))
                                             a-%offset%))))
                    (setf ix (f2cl-lib:int-add ix incx))))
                (if nounit
                    (setf (f2cl-lib:fref x-%data%
                                         (jx)
                                         ((1 *))
                                         x-%offset%)
                          (*
                           (f2cl-lib:fref x-%data%
                                          (jx)
                                          ((1 *))
                                          x-%offset%)
                           (f2cl-lib:fref a-%data%
                                          (j j)
                                          ((1 lda) (1 *))
                                          a-%offset%))))))
            (setf jx (f2cl-lib:int-add jx incx)))))))
(t
 (cond
   ((= incx 1)
    (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                  ((> j 1) nil)
      (tagbody
        (cond
          ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
           (setf temp
                 (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
           (f2cl-lib:fdo (i n
                          (f2cl-lib:int-add i
                                            (f2cl-lib:int-sub 1)))
                         ((> i (f2cl-lib:int-add j 1)) nil)
             (tagbody
               (setf (f2cl-lib:fref x-%data%
                                    (i)
                                    ((1 *))
                                    x-%offset%)
                     (+
```

```
                        (f2cl-lib:fref x-%data%
                                       (i)
                                       ((1 *))
                                       x-%offset%)
                        (* temp
                           (f2cl-lib:fref a-%data%
                                          (i j)
                                          ((1 lda) (1 *))
                                          a-%offset%))))))
        (if nounit
            (setf (f2cl-lib:fref x-%data%
                                 (j)
                                 ((1 *))
                                 x-%offset%)
                  (*
                   (f2cl-lib:fref x-%data%
                                  (j)
                                  ((1 *))
                                  x-%offset%)
                   (f2cl-lib:fref a-%data%
                                  (j j)
                                  ((1 lda) (1 *))
                                  a-%offset%)))))))))
(t
 (setf kx
         (f2cl-lib:int-add kx
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-sub n 1)
                            incx)))
 (setf jx kx)
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (cond
       ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
        (setf temp
                (f2cl-lib:fref x-%data%
                               (jx)
                               ((1 *))
                               x-%offset%))
        (setf ix kx)
        (f2cl-lib:fdo (i n
                       (f2cl-lib:int-add i
                                         (f2cl-lib:int-sub 1)))
                      ((> i (f2cl-lib:int-add j 1)) nil)
          (tagbody
            (setf (f2cl-lib:fref x-%data%
                                 (ix)
                                 ((1 *))
                                 x-%offset%)
```

```
                                (+
                                 (f2cl-lib:fref x-%data%
                                                (ix)
                                                ((1 *))
                                                x-%offset%)
                                 (* temp
                                    (f2cl-lib:fref a-%data%
                                                   (i j)
                                                   ((1 lda) (1 *))
                                                   a-%offset%))))
                            (setf ix (f2cl-lib:int-sub ix incx))))
                        (if nounit
                            (setf (f2cl-lib:fref x-%data%
                                                 (jx)
                                                 ((1 *))
                                                 x-%offset%)
                                  (*
                                   (f2cl-lib:fref x-%data%
                                                  (jx)
                                                  ((1 *))
                                                  x-%offset%)
                                   (f2cl-lib:fref a-%data%
                                                  (j j)
                                                  ((1 lda) (1 *))
                                                  a-%offset%))))))
                    (setf jx (f2cl-lib:int-sub jx incx)))))))))
            (t
             (cond
               ((char-equal uplo #\U)
                (cond
                  ((= incx 1)
                   (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                                 ((> j 1) nil)
                     (tagbody
                       (setf temp
                               (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                       (cond
                         (noconj
                          (if nounit
                              (setf temp
                                      (* temp
                                         (f2cl-lib:fref a-%data%
                                                        (j j)
                                                        ((1 lda) (1 *))
                                                        a-%offset%))))
                          (f2cl-lib:fdo (i
                                         (f2cl-lib:int-add j
                                                           (f2cl-lib:int-sub 1))
                                         (f2cl-lib:int-add i
                                                           (f2cl-lib:int-sub 1)))
```

```
                        ((> i 1) nil)
            (tagbody
              (setf temp
                      (+ temp
                         (*
                          (f2cl-lib:fref a-%data%
                                         (i j)
                                         ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref x-%data%
                                         (i)
                                         ((1 *))
                                         x-%offset%)))))))
        (t
         (if nounit
             (setf temp
                     (* temp
                        (f2cl-lib:dconjg
                         (f2cl-lib:fref a-%data%
                                        (j j)
                                        ((1 lda) (1 *))
                                        a-%offset%)))))
         (f2cl-lib:fdo (i
                        (f2cl-lib:int-add j
                                          (f2cl-lib:int-sub 1))
                        (f2cl-lib:int-add i
                                          (f2cl-lib:int-sub 1)))
                       ((> i 1) nil)
            (tagbody
              (setf temp
                      (+ temp
                         (*
                          (f2cl-lib:dconjg
                           (f2cl-lib:fref a-%data%
                                          (i j)
                                          ((1 lda) (1 *))
                                          a-%offset%))
                          (f2cl-lib:fref x-%data%
                                         (i)
                                         ((1 *))
                                         x-%offset%))))))))
     (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
             temp))))
(t
 (setf jx
         (f2cl-lib:int-add kx
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-sub n 1)
                            incx)))
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
```

```
            ((> j 1) nil)
(tagbody
  (setf temp
          (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
  (setf ix jx)
  (cond
    (noconj
     (if nounit
         (setf temp
                 (* temp
                    (f2cl-lib:fref a-%data%
                                   (j j)
                                   ((1 lda) (1 *))
                                   a-%offset%))))
     (f2cl-lib:fdo (i
                    (f2cl-lib:int-add j
                                      (f2cl-lib:int-sub 1))
                    (f2cl-lib:int-add i
                                      (f2cl-lib:int-sub 1)))
                   ((> i 1) nil)
       (tagbody
         (setf ix (f2cl-lib:int-sub ix incx))
         (setf temp
                 (+ temp
                    (*
                     (f2cl-lib:fref a-%data%
                                    (i j)
                                    ((1 lda) (1 *))
                                    a-%offset%)
                     (f2cl-lib:fref x-%data%
                                    (ix)
                                    ((1 *))
                                    x-%offset%)))))))
    (t
     (if nounit
         (setf temp
                 (* temp
                    (f2cl-lib:dconjg
                     (f2cl-lib:fref a-%data%
                                    (j j)
                                    ((1 lda) (1 *))
                                    a-%offset%)))))
     (f2cl-lib:fdo (i
                    (f2cl-lib:int-add j
                                      (f2cl-lib:int-sub 1))
                    (f2cl-lib:int-add i
                                      (f2cl-lib:int-sub 1)))
                   ((> i 1) nil)
       (tagbody
         (setf ix (f2cl-lib:int-sub ix incx))
```

```
                    (setf temp
                            (+ temp
                               (*
                                (f2cl-lib:dconjg
                                 (f2cl-lib:fref a-%data%
                                                (i j)
                                                ((1 lda) (1 *))
                                                a-%offset%))
                                (f2cl-lib:fref x-%data%
                                               (ix)
                                               ((1 *))
                                               x-%offset%))))))))
            (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                    temp)
            (setf jx (f2cl-lib:int-sub jx incx)))))))
    (t
     (cond
       ((= incx 1)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf temp
                    (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
            (cond
              (noconj
               (if nounit
                   (setf temp
                           (* temp
                              (f2cl-lib:fref a-%data%
                                             (j j)
                                             ((1 lda) (1 *))
                                             a-%offset%))))
               (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                                (f2cl-lib:int-add i 1))
                             ((> i n) nil)
                 (tagbody
                   (setf temp
                           (+ temp
                              (*
                               (f2cl-lib:fref a-%data%
                                              (i j)
                                              ((1 lda) (1 *))
                                              a-%offset%)
                               (f2cl-lib:fref x-%data%
                                              (i)
                                              ((1 *))
                                              x-%offset%)))))))
              (t
               (if nounit
                   (setf temp
```

```
                        (* temp
                           (f2cl-lib:dconjg
                            (f2cl-lib:fref a-%data%
                                           (j j)
                                           ((1 lda) (1 *))
                                           a-%offset%)))))
                (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                               (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf temp
                            (+ temp
                               (*
                                (f2cl-lib:dconjg
                                 (f2cl-lib:fref a-%data%
                                                (i j)
                                                ((1 lda) (1 *))
                                                a-%offset%))
                                (f2cl-lib:fref x-%data%
                                               (i)
                                               ((1 *))
                                               x-%offset%))))))))
             (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                    temp))))
        (t
         (setf jx kx)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf temp
                     (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
             (setf ix jx)
             (cond
               (noconj
                (if nounit
                    (setf temp
                            (* temp
                               (f2cl-lib:fref a-%data%
                                              (j j)
                                              ((1 lda) (1 *))
                                              a-%offset%))))
                (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                               (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf ix (f2cl-lib:int-add ix incx))
                    (setf temp
                            (+ temp
                               (*
                                (f2cl-lib:fref a-%data%
```

```
                                                (i j)
                                                ((1 lda) (1 *))
                                                a-%offset%)
                                        (f2cl-lib:fref x-%data%
                                                (ix)
                                                ((1 *))
                                                x-%offset%)))))))
                        (t
                         (if nounit
                             (setf temp
                                     (* temp
                                        (f2cl-lib:dconjg
                                         (f2cl-lib:fref a-%data%
                                                (j j)
                                                ((1 lda) (1 *))
                                                a-%offset%)))))
                         (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                                        (f2cl-lib:int-add i 1))
                                       ((> i n) nil)
                           (tagbody
                             (setf ix (f2cl-lib:int-add ix incx))
                             (setf temp
                                     (+ temp
                                        (*
                                         (f2cl-lib:dconjg
                                          (f2cl-lib:fref a-%data%
                                                 (i j)
                                                 ((1 lda) (1 *))
                                                 a-%offset%))
                                         (f2cl-lib:fref x-%data%
                                                (ix)
                                                ((1 *))
                                                x-%offset%))))))))
                    (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                            temp)
                    (setf jx (f2cl-lib:int-add jx incx))))))))))
end_label
       (return (values nil nil nil nil nil nil nil nil))))))
```

13.17 ztrsv - triangular solve system of equations $Ax = b$ or $A^T x = b$ or $A^H x = b$

13.17.1 Axiom unit tests

— ztrsv.input —

```
)set break resume
)sys rm -f ztrsv.output
)spool ztrsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

13.17.2 Axiom help page

— ztrsv.help —

```
======================================================================
ztrsv examples
======================================================================

======================================================================
Man Page Details
======================================================================

 NAME
      ZTRSV - solve one of the systems of equations
      A*x = b, or A'*x = b, or conjg( A' )*x = b,

 SYNOPSIS
      SUBROUTINE ZTRSV ( UPLO, TRANS, DIAG, N, A, LDA, X, INCX )

          INTEGER      INCX, LDA, N

          CHARACTER*1  DIAG, TRANS, UPLO

          COMPLEX*16   A( LDA, * ), X( * )

 PURPOSE
```

```
     ZTRSV  solves one of the systems of equations

     where b and x are n element vectors and A is an n by n unit,
     or non-unit, upper or lower triangular matrix.

     No test for singularity or near-singularity is included in
     this routine. Such tests must be performed before calling
     this routine.

PARAMETERS
     UPLO   - CHARACTER*1.
              On entry, UPLO specifies whether the matrix is an
              upper or lower triangular matrix as follows:

              UPLO = 'U' or 'u'   A is an upper triangular matrix.

              UPLO = 'L' or 'l'   A is a lower triangular matrix.

              Unchanged on exit.

     TRANS  - CHARACTER*1.
              On entry, TRANS specifies the equations to be solved
              as follows:

              TRANS = 'N' or 'n'   A*x = b.

              TRANS = 'T' or 't'   A'*x = b.

              TRANS = 'C' or 'c'   conjg( A' )*x = b.

              Unchanged on exit.

     DIAG   - CHARACTER*1.
              On entry, DIAG specifies whether or not A is unit
              triangular as follows:

              DIAG = 'U' or 'u'   A is assumed to be unit

              triangular.

              DIAG = 'N' or 'n'   A is not assumed to be unit tri-
              angular.

              Unchanged on exit.

     N      - INTEGER.
              On entry, N specifies the order of the matrix A.  N
              must be at least zero.  Unchanged on exit.

     A      - COMPLEX*16       array of DIMENSION ( LDA, n ).
```

```
            Before entry with  UPLO = 'U' or 'u', the leading n
            by n upper triangular part of the array A must con-
            tain the upper triangular matrix and the strictly
            lower triangular part of A is not referenced.  Before
            entry with UPLO = 'L' or 'l', the leading n by n
            lower triangular part of the array A must contain the
            lower triangular matrix and the strictly upper tri-
            angular part of A is not referenced.  Note that when
            DIAG = 'U' or 'u', the diagonal elements of A are not
            referenced either, but are assumed to be unity.
            Unchanged on exit.

      LDA    - INTEGER.
            On entry, LDA specifies the first dimension of A as
            declared in the calling (sub) program. LDA must be at
            least max( 1, n ).  Unchanged on exit.

      X      - COMPLEX*16       array of dimension at least
            ( 1 + ( n - 1 )*abs( INCX ) ).  Before entry, the
            incremented array X must contain the n element
            right-hand side vector b. On exit, X is overwritten
            with the solution vector x.

      INCX   - INTEGER.
            On entry, INCX specifies the increment for the ele-
            ments of X. INCX must not be zero.  Unchanged on
            exit.
```

13.17.3 fortran code

— ztrsv.f —

```
*  ==================================================================
      SUBROUTINE ZTRSV(UPLO,TRANS,DIAG,N,A,LDA,X,INCX)
*
*  -- Reference BLAS level2 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER INCX,LDA,N
      CHARACTER DIAG,TRANS,UPLO
*     ..
*     .. Array Arguments ..
```

```
      COMPLEX*16 A(LDA,*),X(*)
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP
      INTEGER I,INFO,IX,J,JX,KX
      LOGICAL NOCONJ,NOUNIT
*     ..
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DCONJG,MAX
*     ..
*
*     Test the input parameters.
*
      INFO = 0
      IF (.NOT.LSAME(UPLO,'U') .AND. .NOT.LSAME(UPLO,'L')) THEN
          INFO = 1
      ELSE IF (.NOT.LSAME(TRANS,'N') .AND. .NOT.LSAME(TRANS,'T') .AND.
     +         .NOT.LSAME(TRANS,'C')) THEN
          INFO = 2
      ELSE IF (.NOT.LSAME(DIAG,'U') .AND. .NOT.LSAME(DIAG,'N')) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,N)) THEN
          INFO = 6
      ELSE IF (INCX.EQ.0) THEN
          INFO = 8
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZTRSV ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (N.EQ.0) RETURN
```

```
*
      NOCONJ = LSAME(TRANS,'T')
      NOUNIT = LSAME(DIAG,'N')
*
*     Set up the start point in X if the increment is not unity. This
*     will be  ( N - 1 )*INCX  too small for descending loops.
*
      IF (INCX.LE.0) THEN
          KX = 1 - (N-1)*INCX
      ELSE IF (INCX.NE.1) THEN
          KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through A.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  x := inv( A )*x.
*
          IF (LSAME(UPLO,'U')) THEN
              IF (INCX.EQ.1) THEN
                  DO 20 J = N,1,-1
                      IF (X(J).NE.ZERO) THEN
                          IF (NOUNIT) X(J) = X(J)/A(J,J)
                          TEMP = X(J)
                          DO 10 I = J - 1,1,-1
                              X(I) = X(I) - TEMP*A(I,J)
   10                     CONTINUE
                      END IF
   20             CONTINUE
              ELSE
                  JX = KX + (N-1)*INCX
                  DO 40 J = N,1,-1
                      IF (X(JX).NE.ZERO) THEN
                          IF (NOUNIT) X(JX) = X(JX)/A(J,J)
                          TEMP = X(JX)
                          IX = JX
                          DO 30 I = J - 1,1,-1
                              IX = IX - INCX
                              X(IX) = X(IX) - TEMP*A(I,J)
   30                     CONTINUE
                      END IF
                      JX = JX - INCX
   40             CONTINUE
              END IF
          ELSE
              IF (INCX.EQ.1) THEN
                  DO 60 J = 1,N
                      IF (X(J).NE.ZERO) THEN
```

```
                        IF (NOUNIT) X(J) = X(J)/A(J,J)
                        TEMP = X(J)
                        DO 50 I = J + 1,N
                            X(I) = X(I) - TEMP*A(I,J)
   50                   CONTINUE
                    END IF
   60            CONTINUE
            ELSE
                JX = KX
                DO 80 J = 1,N
                    IF (X(JX).NE.ZERO) THEN
                        IF (NOUNIT) X(JX) = X(JX)/A(J,J)
                        TEMP = X(JX)
                        IX = JX
                        DO 70 I = J + 1,N
                            IX = IX + INCX
                            X(IX) = X(IX) - TEMP*A(I,J)
   70                   CONTINUE
                    END IF
                    JX = JX + INCX
   80            CONTINUE
            END IF
        END IF
     ELSE
*
*       Form  x := inv( A**T )*x  or  x := inv( A**H )*x.
*
        IF (LSAME(UPLO,'U')) THEN
            IF (INCX.EQ.1) THEN
                DO 110 J = 1,N
                    TEMP = X(J)
                    IF (NOCONJ) THEN
                        DO 90 I = 1,J - 1
                            TEMP = TEMP - A(I,J)*X(I)
   90                   CONTINUE
                        IF (NOUNIT) TEMP = TEMP/A(J,J)
                    ELSE
                        DO 100 I = 1,J - 1
                            TEMP = TEMP - DCONJG(A(I,J))*X(I)
  100                   CONTINUE
                        IF (NOUNIT) TEMP = TEMP/DCONJG(A(J,J))
                    END IF
                    X(J) = TEMP
  110           CONTINUE
            ELSE
                JX = KX
                DO 140 J = 1,N
                    IX = KX
                    TEMP = X(JX)
                    IF (NOCONJ) THEN
```

```
                    DO 120 I = 1,J - 1
                        TEMP = TEMP - A(I,J)*X(IX)
                        IX = IX + INCX
 120                CONTINUE
                    IF (NOUNIT) TEMP = TEMP/A(J,J)
                ELSE
                    DO 130 I = 1,J - 1
                        TEMP = TEMP - DCONJG(A(I,J))*X(IX)
                        IX = IX + INCX
 130                CONTINUE
                    IF (NOUNIT) TEMP = TEMP/DCONJG(A(J,J))
                END IF
                X(JX) = TEMP
                JX = JX + INCX
 140        CONTINUE
            END IF
        ELSE
            IF (INCX.EQ.1) THEN
                DO 170 J = N,1,-1
                    TEMP = X(J)
                    IF (NOCONJ) THEN
                        DO 150 I = N,J + 1,-1
                            TEMP = TEMP - A(I,J)*X(I)
 150                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/A(J,J)
                    ELSE
                        DO 160 I = N,J + 1,-1
                            TEMP = TEMP - DCONJG(A(I,J))*X(I)
 160                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/DCONJG(A(J,J))
                    END IF
                    X(J) = TEMP
 170            CONTINUE
            ELSE
                KX = KX + (N-1)*INCX
                JX = KX
                DO 200 J = N,1,-1
                    IX = KX
                    TEMP = X(JX)
                    IF (NOCONJ) THEN
                        DO 180 I = N,J + 1,-1
                            TEMP = TEMP - A(I,J)*X(IX)
                            IX = IX - INCX
 180                    CONTINUE
                        IF (NOUNIT) TEMP = TEMP/A(J,J)
                    ELSE
                        DO 190 I = N,J + 1,-1
                            TEMP = TEMP - DCONJG(A(I,J))*X(IX)
                            IX = IX - INCX
 190                    CONTINUE
```

```
                        IF (NOUNIT) TEMP = TEMP/DCONJG(A(J,J))
                    END IF
                    X(JX) = TEMP
                    JX = JX - INCX
  200             CONTINUE
              END IF
          END IF
      END IF
*
      RETURN
*
*     End of ZTRSV .
*
      END
```

———-

13.17.4 lisp code

— BLAS 2 ztrsv —

```
(let* ((zero (complex 0.0 0.0)))
  (declare (type (complex double-float) zero))
  (defun ztrsv (uplo trans diag n a lda x incx)
    (declare (type (simple-array (complex double-float) (*)) x a)
             (type fixnum incx lda n)
             (type character diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a (complex double-float) a-%data% a-%offset%)
         (x (complex double-float) x-%data% x-%offset%))
      (prog ((noconj nil) (nounit nil) (i 0) (info 0) (ix 0) (j 0) (jx 0)
             (kx 0) (temp #C(0.0 0.0)))
        (declare (type (member t nil) noconj nounit)
                 (type fixnum i info ix j jx kx)
                 (type (complex double-float) temp))
        (setf info 0)
        (cond
          ((and (not (char-equal uplo #\U)) (not (char-equal uplo #\L)))
           (setf info 1))
          ((and (not (char-equal trans #\N))
                (not (char-equal trans #\T))
                (not (char-equal trans #\C)))
           (setf info 2))
          ((and (not (char-equal diag #\U)) (not (char-equal diag #\N)))
```

```
    (setf info 3))
   ((< n 0)
    (setf info 4))
   ((< lda (max (the fixnum 1) (the fixnum n)))
    (setf info 6))
   ((= incx 0)
    (setf info 8)))
(cond
  ((/= info 0)
   (error
    " ** On entry to ~a parameter number ~a had an illegal value~%"
    "ZTRSV" info)
   (go end_label)))
(if (= n 0) (go end_label))
(setf noconj (char-equal trans #\T))
(setf nounit (char-equal diag #\N))
(cond
  ((<= incx 0)
   (setf kx
           (f2cl-lib:int-sub 1
                             (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                               incx))))
  ((/= incx 1)
   (setf kx 1)))
(cond
  ((char-equal trans #\N)
   (cond
     ((char-equal uplo #\U)
      (cond
        ((= incx 1)
         (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((> j 1) nil)
           (tagbody
             (cond
               ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
                (if nounit
                    (setf (f2cl-lib:fref x-%data%
                                         (j)
                                         ((1 *))
                                         x-%offset%)
                          (/
                           (f2cl-lib:fref x-%data%
                                          (j)
                                          ((1 *))
                                          x-%offset%)
                           (f2cl-lib:fref a-%data%
                                          (j j)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
                (setf temp
```

```
                (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
        (f2cl-lib:fdo (i
                       (f2cl-lib:int-add j
                                         (f2cl-lib:int-sub 1))
                       (f2cl-lib:int-add i
                                         (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (setf (f2cl-lib:fref x-%data%
                                 (i)
                                 ((1 *))
                                 x-%offset%)
                  (-
                   (f2cl-lib:fref x-%data%
                                  (i)
                                  ((1 *))
                                  x-%offset%)
                   (* temp
                      (f2cl-lib:fref a-%data%
                                     (i j)
                                     ((1 lda) (1 *))
                                     a-%offset%))))))))))
(t
 (setf jx
         (f2cl-lib:int-add kx
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-sub n 1)
                            incx)))
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (cond
       ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
        (if nounit
            (setf (f2cl-lib:fref x-%data%
                                 (jx)
                                 ((1 *))
                                 x-%offset%)
                  (/
                   (f2cl-lib:fref x-%data%
                                  (jx)
                                  ((1 *))
                                  x-%offset%)
                   (f2cl-lib:fref a-%data%
                                  (j j)
                                  ((1 lda) (1 *))
                                  a-%offset%))))
        (setf temp
                (f2cl-lib:fref x-%data%
                               (jx)
```

```
                                        ((1 *))
                                        x-%offset%))
                        (setf ix jx)
                        (f2cl-lib:fdo (i
                                       (f2cl-lib:int-add j
                                                         (f2cl-lib:int-sub 1))
                                       (f2cl-lib:int-add i
                                                         (f2cl-lib:int-sub 1)))
                                      ((> i 1) nil)
                          (tagbody
                            (setf ix (f2cl-lib:int-sub ix incx))
                            (setf (f2cl-lib:fref x-%data%
                                                 (ix)
                                                 ((1 *))
                                                 x-%offset%)
                                  (-
                                   (f2cl-lib:fref x-%data%
                                                  (ix)
                                                  ((1 *))
                                                  x-%offset%)
                                   (* temp
                                      (f2cl-lib:fref a-%data%
                                                     (i j)
                                                     ((1 lda) (1 *))
                                                     a-%offset%)))))))
                  (setf jx (f2cl-lib:int-sub jx incx)))))))
        (t
         (cond
           ((= incx 1)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (cond
                  ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
                   (if nounit
                       (setf (f2cl-lib:fref x-%data%
                                            (j)
                                            ((1 *))
                                            x-%offset%)
                             (/
                              (f2cl-lib:fref x-%data%
                                             (j)
                                             ((1 *))
                                             x-%offset%)
                              (f2cl-lib:fref a-%data%
                                             (j j)
                                             ((1 lda) (1 *))
                                             a-%offset%))))
                   (setf temp
                           (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
```

```
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                       (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref x-%data%
                                 (i)
                                 ((1 *))
                                 x-%offset%)
                  (-
                   (f2cl-lib:fref x-%data%
                                  (i)
                                  ((1 *))
                                  x-%offset%)
                   (* temp
                      (f2cl-lib:fref a-%data%
                                     (i j)
                                     ((1 lda) (1 *))
                                     a-%offset%))))))))))
(t
 (setf jx kx)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
        (if nounit
            (setf (f2cl-lib:fref x-%data%
                                 (jx)
                                 ((1 *))
                                 x-%offset%)
                  (/
                   (f2cl-lib:fref x-%data%
                                  (jx)
                                  ((1 *))
                                  x-%offset%)
                   (f2cl-lib:fref a-%data%
                                  (j j)
                                  ((1 lda) (1 *))
                                  a-%offset%))))
        (setf temp
                (f2cl-lib:fref x-%data%
                               (jx)
                               ((1 *))
                               x-%offset%))
        (setf ix jx)
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                       (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf ix (f2cl-lib:int-add ix incx))
```

```
                    (setf (f2cl-lib:fref x-%data%
                                         (ix)
                                         ((1 *))
                                         x-%offset%)
                          (-
                           (f2cl-lib:fref x-%data%
                                          (ix)
                                          ((1 *))
                                          x-%offset%)
                           (* temp
                              (f2cl-lib:fref a-%data%
                                             (i j)
                                             ((1 lda) (1 *))
                                             a-%offset%))))))))
            (setf jx (f2cl-lib:int-add jx incx)))))))))
(t
 (cond
   ((char-equal uplo #\U)
    (cond
      ((= incx 1)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (setf temp
                   (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
           (cond
             (noconj
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i
                                (f2cl-lib:int-add j
                                                  (f2cl-lib:int-sub
                                                   1)))
                             nil)
                (tagbody
                  (setf temp
                          (- temp
                             (*
                              (f2cl-lib:fref a-%data%
                                             (i j)
                                             ((1 lda) (1 *))
                                             a-%offset%)
                              (f2cl-lib:fref x-%data%
                                             (i)
                                             ((1 *))
                                             x-%offset%))))))
              (if nounit
                  (setf temp
                          (/ temp
                             (f2cl-lib:fref a-%data%
                                            (j j)
```

```
                                        ((1 lda) (1 *))
                                        a-%offset%)))))
          (t
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i
                             (f2cl-lib:int-add j
                                               (f2cl-lib:int-sub
                                                1)))
                          nil)
             (tagbody
               (setf temp
                       (- temp
                          (*
                           (f2cl-lib:dconjg
                            (f2cl-lib:fref a-%data%
                                           (i j)
                                           ((1 lda) (1 *))
                                           a-%offset%))
                           (f2cl-lib:fref x-%data%
                                          (i)
                                          ((1 *))
                                          x-%offset%))))))
           (if nounit
               (setf temp
                       (/ temp
                          (f2cl-lib:dconjg
                           (f2cl-lib:fref a-%data%
                                          (j j)
                                          ((1 lda) (1 *))
                                          a-%offset%)))))))
      (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
              temp))))
(t
 (setf jx kx)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf ix kx)
     (setf temp
             (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
     (cond
       (noconj
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i
                          (f2cl-lib:int-add j
                                            (f2cl-lib:int-sub
                                             1)))
                       nil)
          (tagbody
            (setf temp
```

```
                (- temp
                   (*
                    (f2cl-lib:fref a-%data%
                                   (i j)
                                   ((1 lda) (1 *))
                                   a-%offset%)
                    (f2cl-lib:fref x-%data%
                                   (ix)
                                   ((1 *))
                                   x-%offset%))))
        (setf ix (f2cl-lib:int-add ix incx))))
    (if nounit
        (setf temp
                (/ temp
                   (f2cl-lib:fref a-%data%
                                  (j j)
                                  ((1 lda) (1 *))
                                  a-%offset%)))))
   (t
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i
                      (f2cl-lib:int-add j
                                        (f2cl-lib:int-sub
                                         1)))
                  nil)
      (tagbody
        (setf temp
                (- temp
                   (*
                    (f2cl-lib:dconjg
                     (f2cl-lib:fref a-%data%
                                    (i j)
                                    ((1 lda) (1 *))
                                    a-%offset%))
                    (f2cl-lib:fref x-%data%
                                   (ix)
                                   ((1 *))
                                   x-%offset%))))
        (setf ix (f2cl-lib:int-add ix incx))))
    (if nounit
        (setf temp
                (/ temp
                   (f2cl-lib:dconjg
                    (f2cl-lib:fref a-%data%
                                   (j j)
                                   ((1 lda) (1 *))
                                   a-%offset%)))))))
(setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
        temp)
(setf jx (f2cl-lib:int-add jx incx)))))))
```

```
(t
 (cond
   ((= incx 1)
    (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                  ((> j 1) nil)
      (tagbody
        (setf temp
                (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
        (cond
          (noconj
           (f2cl-lib:fdo (i n
                          (f2cl-lib:int-add i
                                            (f2cl-lib:int-sub 1)))
                         ((> i (f2cl-lib:int-add j 1)) nil)
             (tagbody
               (setf temp
                       (- temp
                          (*
                           (f2cl-lib:fref a-%data%
                                          (i j)
                                          ((1 lda) (1 *))
                                          a-%offset%)
                           (f2cl-lib:fref x-%data%
                                          (i)
                                          ((1 *))
                                          x-%offset%))))))
           (if nounit
               (setf temp
                       (/ temp
                          (f2cl-lib:fref a-%data%
                                         (j j)
                                         ((1 lda) (1 *))
                                         a-%offset%)))))
          (t
           (f2cl-lib:fdo (i n
                          (f2cl-lib:int-add i
                                            (f2cl-lib:int-sub 1)))
                         ((> i (f2cl-lib:int-add j 1)) nil)
             (tagbody
               (setf temp
                       (- temp
                          (*
                           (f2cl-lib:dconjg
                            (f2cl-lib:fref a-%data%
                                           (i j)
                                           ((1 lda) (1 *))
                                           a-%offset%))
                           (f2cl-lib:fref x-%data%
                                          (i)
                                          ((1 *))
```

```
                                          x-%offset%))))))
            (if nounit
                (setf temp
                        (/ temp
                           (f2cl-lib:dconjg
                            (f2cl-lib:fref a-%data%
                                           (j j)
                                           ((1 lda) (1 *))
                                           a-%offset%)))))))
        (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
              temp))))
    (t
     (setf kx
             (f2cl-lib:int-add kx
                               (f2cl-lib:int-mul
                                (f2cl-lib:int-sub n 1)
                                incx)))
     (setf jx kx)
     (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   ((> j 1) nil)
       (tagbody
         (setf ix kx)
         (setf temp
                 (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%))
         (cond
           (noconj
            (f2cl-lib:fdo (i n
                           (f2cl-lib:int-add i
                                             (f2cl-lib:int-sub 1)))
                          ((> i (f2cl-lib:int-add j 1)) nil)
              (tagbody
                (setf temp
                        (- temp
                           (*
                            (f2cl-lib:fref a-%data%
                                           (i j)
                                           ((1 lda) (1 *))
                                           a-%offset%)
                            (f2cl-lib:fref x-%data%
                                           (ix)
                                           ((1 *))
                                           x-%offset%))))
                (setf ix (f2cl-lib:int-sub ix incx))))
            (if nounit
                (setf temp
                        (/ temp
                           (f2cl-lib:fref a-%data%
                                          (j j)
                                          ((1 lda) (1 *))
                                          a-%offset%)))))
```

```
                        (t
                         (f2cl-lib:fdo (i n
                                        (f2cl-lib:int-add i
                                                          (f2cl-lib:int-sub 1)))
                                       ((> i (f2cl-lib:int-add j 1)) nil)
                           (tagbody
                             (setf temp
                                     (- temp
                                        (*
                                         (f2cl-lib:dconjg
                                          (f2cl-lib:fref a-%data%
                                                         (i j)
                                                         ((1 lda) (1 *))
                                                         a-%offset%))
                                         (f2cl-lib:fref x-%data%
                                                        (ix)
                                                        ((1 *))
                                                        x-%offset%))))
                             (setf ix (f2cl-lib:int-sub ix incx))))
                         (if nounit
                             (setf temp
                                     (/ temp
                                        (f2cl-lib:dconjg
                                         (f2cl-lib:fref a-%data%
                                                        (j j)
                                                        ((1 lda) (1 *))
                                                        a-%offset%)))))))
                      (setf (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)
                              temp)
                      (setf jx (f2cl-lib:int-sub jx incx))))))))))
end_label
       (return (values nil nil nil nil nil nil nil nil))))))
```

———-

Chapter 14

BLAS Level 3 Real Matrix-Matrix Operations

14.1 sgemm - $\alpha op(A)op(B)+\beta C$, $op(X) = X$ or $op(X) = X^T$

14.1.1 Axiom unit tests

— **sgemm.input** —

```
)set break resume
)sys rm -f sgemm.output
)spool sgemm.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

14.1.2 Axiom help page

— **sgemm.help** —

```
=====================================================================
sgemm examples
=====================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgemm (TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA,
           C, LDC)
           SGEMM

Function/Subroutine Documentation
   subroutine sgemm (character TRANSA, character TRANSB, integer M, integer N,
       integer K, real ALPHA, real, dimension(lda,*) A, integer LDA, real,
       dimension(ldb,*) B, integer LDB, real BETA, real, dimension(ldc,*) C,
       integer LDC)
       SGEMM

Purpose:
   SGEMM  performs one of the matrix-matrix operations

   C := alpha*op( A )*op( B ) + beta*C,

   where  op( X ) is one of

   op( X ) = X   or   op( X ) = X**T,

   alpha and beta are scalars, and A, B and C are matrices, with op( A )
   an m by k matrix,  op( B )  a  k by n matrix and  C an m by n matrix.

Parameters:

 TRANSA
   TRANSA is CHARACTER*1
   On entry, TRANSA specifies the form of op( A ) to be used in
   the matrix multiplication as follows:

   TRANSA = 'N' or 'n',  op( A ) = A.

   TRANSA = 'T' or 't',  op( A ) = A**T.

   TRANSA = 'C' or 'c',  op( A ) = A**T.

 TRANSB
   TRANSB is CHARACTER*1
   On entry, TRANSB specifies the form of op( B ) to be used in
   the matrix multiplication as follows:

   TRANSB = 'N' or 'n',  op( B ) = B.
```

```
  TRANSB = 'T' or 't',  op( B ) = B**T.

  TRANSB = 'C' or 'c',  op( B ) = B**T.

M
  M is INTEGER
  On entry,  M  specifies  the number  of rows  of the  matrix
  op( A )  and of the  matrix  C.  M  must  be at least  zero.

N
  N is INTEGER
  On entry,  N  specifies the number  of columns of the matrix
  op( B ) and the number of columns of the matrix C. N must be
  at least zero.

K
  K is INTEGER
  On entry,  K  specifies  the number of columns of the matrix
  op( A ) and the number of rows of the matrix op( B ). K must
  be at least  zero.

ALPHA
  ALPHA is REAL
  On entry, ALPHA specifies the scalar alpha.

A
  A is REAL array of DIMENSION ( LDA, ka ), where ka is
  k  when  TRANSA = 'N' or 'n',  and is  m  otherwise.
  Before entry with  TRANSA = 'N' or 'n',  the leading  m by k
  part of the array  A  must contain the matrix  A,  otherwise
  the leading  k by m  part of the array  A  must contain  the
  matrix A.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. When  TRANSA = 'N' or 'n' then
  LDA must be at least  max( 1, m ), otherwise  LDA must be at
  least  max( 1, k ).

B
  B is REAL array of DIMENSION ( LDB, kb ), where kb is
  n  when  TRANSB = 'N' or 'n',  and is  k  otherwise.
  Before entry with  TRANSB = 'N' or 'n',  the leading  k by n
  part of the array  B  must contain the matrix  B,  otherwise
  the leading  n by k  part of the array  B  must contain  the
  matrix B.

LDB
  LDB is INTEGER
```

```
  On entry, LDB specifies the first dimension of B as declared
  in the calling (sub) program. When  TRANSB = 'N' or 'n' then
  LDB must be at least  max( 1, k ), otherwise  LDB must be at
  least  max( 1, n ).

BETA
  BETA is REAL
  On entry,  BETA  specifies the scalar  beta.  When  BETA  is
  supplied as zero then C need not be set on input.

C
  C is REAL array of DIMENSION ( LDC, n ).
  Before entry, the leading  m by n  part of the array  C must
  contain the matrix  C,  except when  beta  is zero, in which
  case C need not be set on entry.
  On exit, the array  C  is overwritten by the  m by n  matrix
  ( alpha*op( A )*op( B ) + beta*C ).

LDC
  LDC is INTEGER
  On entry, LDC specifies the first dimension of C as declared
  in  the  calling  (sub)  program.  LDC  must  be  at  least max( 1, m ).
```

———-

14.1.3 fortran code

— sgemm.f —

```
*  ===================================================================
      SUBROUTINE SGEMM(TRANSA,TRANSB,M,N,K,ALPHA,A,LDA,B,LDB,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      REAL ALPHA,BETA
      INTEGER K,LDA,LDB,LDC,M,N
      CHARACTER TRANSA,TRANSB
*     ..
*     .. Array Arguments ..
      REAL A(LDA,*),B(LDB,*),C(LDC,*)
*     ..
*
*  ===================================================================
```

```
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*     .. Local Scalars ..
      REAL TEMP
      INTEGER I,INFO,J,L,NCOLA,NROWA,NROWB
      LOGICAL NOTA,NOTB
*     ..
*     .. Parameters ..
      REAL ONE,ZERO
      PARAMETER (ONE=1.0E+0,ZERO=0.0E+0)
*     ..
*
*     Set  NOTA  and  NOTB  as  true if  A  and  B  respectively are not
*     transposed and set  NROWA, NCOLA and  NROWB  as the number of rows
*     and  columns of  A  and the  number of  rows  of  B  respectively.
*
      NOTA = LSAME(TRANSA,'N')
      NOTB = LSAME(TRANSB,'N')
      IF (NOTA) THEN
          NROWA = M
          NCOLA = K
      ELSE
          NROWA = K
          NCOLA = M
      END IF
      IF (NOTB) THEN
          NROWB = K
      ELSE
          NROWB = N
      END IF
*
*     Test the input parameters.
*
      INFO = 0
      IF ((.NOT.NOTA) .AND. (.NOT.LSAME(TRANSA,'C')) .AND.
     +    (.NOT.LSAME(TRANSA,'T'))) THEN
          INFO = 1
      ELSE IF ((.NOT.NOTB) .AND. (.NOT.LSAME(TRANSB,'C')) .AND.
     +         (.NOT.LSAME(TRANSB,'T'))) THEN
          INFO = 2
      ELSE IF (M.LT.0) THEN
```

```
         INFO = 3
      ELSE IF (N.LT.0) THEN
         INFO = 4
      ELSE IF (K.LT.0) THEN
         INFO = 5
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
         INFO = 8
      ELSE IF (LDB.LT.MAX(1,NROWB)) THEN
         INFO = 10
      ELSE IF (LDC.LT.MAX(1,M)) THEN
         INFO = 13
      END IF
      IF (INFO.NE.0) THEN
         CALL XERBLA('SGEMM ',INFO)
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR.
     +    (((ALPHA.EQ.ZERO).OR. (K.EQ.0)).AND. (BETA.EQ.ONE))) RETURN
*
*     And if  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
         IF (BETA.EQ.ZERO) THEN
            DO 20 J = 1,N
               DO 10 I = 1,M
                  C(I,J) = ZERO
   10          CONTINUE
   20       CONTINUE
         ELSE
            DO 40 J = 1,N
               DO 30 I = 1,M
                  C(I,J) = BETA*C(I,J)
   30          CONTINUE
   40       CONTINUE
         END IF
         RETURN
      END IF
*
*     Start the operations.
*
      IF (NOTB) THEN
         IF (NOTA) THEN
*
*          Form  C := alpha*A*B + beta*C.
*
            DO 90 J = 1,N
               IF (BETA.EQ.ZERO) THEN
```

```
                    DO 50 I = 1,M
                        C(I,J) = ZERO
   50               CONTINUE
                ELSE IF (BETA.NE.ONE) THEN
                    DO 60 I = 1,M
                        C(I,J) = BETA*C(I,J)
   60               CONTINUE
                END IF
                DO 80 L = 1,K
                    TEMP = ALPHA*B(L,J)
                    DO 70 I = 1,M
                        C(I,J) = C(I,J) + TEMP*A(I,L)
   70               CONTINUE
   80           CONTINUE
   90       CONTINUE
        ELSE
*
*         Form  C := alpha*A**T*B + beta*C
*
            DO 120 J = 1,N
                DO 110 I = 1,M
                    TEMP = ZERO
                    DO 100 L = 1,K
                        TEMP = TEMP + A(L,I)*B(L,J)
  100               CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP
                    ELSE
                        C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                    END IF
  110           CONTINUE
  120       CONTINUE
        END IF
    ELSE
        IF (NOTA) THEN
*
*         Form  C := alpha*A*B**T + beta*C
*
            DO 170 J = 1,N
                IF (BETA.EQ.ZERO) THEN
                    DO 130 I = 1,M
                        C(I,J) = ZERO
  130               CONTINUE
                ELSE IF (BETA.NE.ONE) THEN
                    DO 140 I = 1,M
                        C(I,J) = BETA*C(I,J)
  140               CONTINUE
                END IF
                DO 160 L = 1,K
                    TEMP = ALPHA*B(J,L)
```

```
                    DO 150 I = 1,M
                        C(I,J) = C(I,J) + TEMP*A(I,L)
  150               CONTINUE
  160           CONTINUE
  170       CONTINUE
        ELSE
*
*         Form  C := alpha*A**T*B**T + beta*C
*
            DO 200 J = 1,N
                DO 190 I = 1,M
                    TEMP = ZERO
                    DO 180 L = 1,K
                        TEMP = TEMP + A(L,I)*B(J,L)
  180               CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP
                    ELSE
                        C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                    END IF
  190           CONTINUE
  200       CONTINUE
        END IF
    END IF
*
    RETURN
*
*   End of SGEMM .
*
    END
```

———-

14.1.4 lisp code

— BLAS 3 sgemm —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun sgemm (transa transb m n k alpha a lda b ldb$ beta c ldc)
    (declare (type (array single-float (*)) c b a)
             (type (single-float) beta alpha)
             (type (f2cl-lib:integer4) ldc ldb$ lda k n m)
             (type (string *) transb transa))
    (f2cl-lib:with-multi-array-data
```

```
  ((transa character transa-%data% transa-%offset%)
   (transb character transb-%data% transb-%offset%)
   (a single-float a-%data% a-%offset%)
   (b single-float b-%data% b-%offset%)
   (c single-float c-%data% c-%offset%))
(prog ((nota nil) (notb nil) (i 0) (info 0) (j 0) (l 0) (ncola 0)
       (nrowa 0) (nrowb 0) (temp 0.0))
  (declare (type f2cl-lib:logical nota notb)
           (type (f2cl-lib:integer4) i info j l ncola nrowa nrowb)
           (type (single-float) temp))
  (setf nota (lsame transa "N"))
  (setf notb (lsame transb "N"))
  (cond (nota (setf nrowa m) (setf ncola k))
        (t (setf nrowa k) (setf ncola m)))
  (cond (notb (setf nrowb k)) (t (setf nrowb n)))
  (setf info 0)
  (cond
   ((and (not nota) (not (lsame transa "C")) (not (lsame transa "T")))
    (setf info 1))
   ((and (not notb) (not (lsame transb "C")) (not (lsame transb "T")))
    (setf info 2))
   ((< m 0) (setf info 3)) ((< n 0) (setf info 4))
   ((< k 0) (setf info 5))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nrowa)))
    (setf info 8))
   ((< ldb$
       (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nrowb)))
    (setf info 10))
   ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
    (setf info 13)))
  (cond
   ((/= info 0)
    (multiple-value-bind (var-0 var-1)
        (xerbla "SGEMM " info)
      (declare (ignore var-0))
      (when var-1 (setf info var-1)))
    (go end_label)))
  (if (or (= m 0) (= n 0) (and (or (= alpha zero) (= k 0)) (= beta one)))
      (go end_label))
  (cond
   ((= alpha zero)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
```

```
                                    c-%offset%)
                        zero)
                 label10))
             label20)))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i m) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)
                         (* beta
                            (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                           c-%offset%)))
                 label30))
             label40))))
      (go end_label)))
    (cond
     (notb
      (cond
       (nota
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (cond
             ((= beta zero)
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i m) nil)
                (tagbody
                  (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)
                        zero)
                 label50)))
             ((/= beta one)
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i m) nil)
                (tagbody
                  (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)
                        (* beta
                           (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                          c-%offset%)))
                 label60))))
            (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                          ((> l k) nil)
              (tagbody
                (setf temp
                        (* alpha
```

```
                        (f2cl-lib:fref b-%data% (l j) ((1 ldb$) (1 *))
                                       b-%offset%)))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                      (+
                       (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)
                       (* temp
                          (f2cl-lib:fref a-%data% (i l)
                                         ((1 lda) (1 *))
                                         a-%offset%))))
                label70))
          label80))
      label90)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf temp zero)
           (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                         ((> l k) nil)
             (tagbody
               (setf temp
                     (+ temp
                        (*
                         (f2cl-lib:fref a-%data% (l i)
                                        ((1 lda) (1 *)) a-%offset%)
                         (f2cl-lib:fref b-%data% (l j)
                                        ((1 ldb$) (1 *))
                                        b-%offset%))))
               label100))
           (cond
            ((= beta zero)
             (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                  c-%offset%)
                   (* alpha temp)))
            (t
             (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                  c-%offset%)
                   (+ (* alpha temp)
                      (* beta
                         (f2cl-lib:fref c-%data% (i j)
                                        ((1 ldc) (1 *))
                                        c-%offset%))))))
```

```
            label110))
        label120)))))
  (t
   (cond
    (nota
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (cond
          ((= beta zero)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                     zero)
              label130)))
          ((/= beta one)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                     (* beta
                        (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)))
              label140))))
         (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                       ((> l k) nil)
           (tagbody
             (setf temp
                   (* alpha
                      (f2cl-lib:fref b-%data% (j l) ((1 ldb$) (1 *))
                                     b-%offset%)))
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i m) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)
                       (+
                        (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)
                        (* temp
                           (f2cl-lib:fref a-%data% (i l)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
              label150))
            label160))
        label170)))
    (t
```

```
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf temp zero)
              (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                            ((> l k) nil)
                (tagbody
                  (setf temp
                          (+ temp
                             (*
                              (f2cl-lib:fref a-%data% (l i)
                                             ((1 lda) (1 *)) a-%offset%)
                              (f2cl-lib:fref b-%data% (j l)
                                             ((1 ldb$) (1 *))
                                             b-%offset%))))
                   label180))
              (cond
               ((= beta zero)
                (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                        (* alpha temp)))
               (t
                (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                        (+ (* alpha temp)
                           (* beta
                              (f2cl-lib:fref c-%data% (i j)
                                             ((1 ldc) (1 *))
                                             c-%offset%))))))
            label190))
          label200))))))
 (go end_label)
end_label
 (return
  (values nil nil nil nil nil nil nil nil nil nil nil nil nil)))))
```

———-

14.2 ssymm - matrix-matrix $\alpha AB + \beta C$ or $\alpha BA + \beta C$

14.2.1 Axiom unit tests

— **ssymm.input** —

```
)set break resume
)sys rm -f ssymm.output
)spool ssymm.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

14.2.2 Axiom help page

— ssymm.help —

```
==================================================================
ssymm examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssymm (SIDE, UPLO, M, N, ALPHA, A, LDA, B, LDB, BETA, C,
           LDC)
           SSYMM

Function/Subroutine Documentation
   subroutine ssymm (character SIDE, character UPLO, integer M, integer N,
       real ALPHA, real, dimension(lda,*) A, integer LDA, real,
       dimension(ldb,*) B, integer LDB, real BETA, real, dimension(ldc,*) C,
       integer LDC)
       SSYMM

Purpose:
   SSYMM  performs one of the matrix-matrix operations

     C := alpha*A*B + beta*C,

   or

     C := alpha*B*A + beta*C,

   where alpha and beta are scalars,  A is a symmetric matrix and  B and
   C are  m by n matrices.
```

```
Parameters:

 SIDE
   SIDE is CHARACTER*1
   On entry,  SIDE  specifies whether  the  symmetric matrix  A
   appears on the  left or right  in the  operation as follows:

   SIDE = 'L' or 'l'   C := alpha*A*B + beta*C,

   SIDE = 'R' or 'r'   C := alpha*B*A + beta*C,

 UPLO
   UPLO is CHARACTER*1
   On  entry,  UPLO  specifies  whether  the  upper  or  lower
   triangular  part  of  the  symmetric  matrix  A  is  to  be
   referenced as follows:

   UPLO = 'U' or 'u'   Only the upper triangular part of the
   symmetric matrix is to be referenced.

   UPLO = 'L' or 'l'   Only the lower triangular part of the
   symmetric matrix is to be referenced.

 M
   M is INTEGER
   On entry,  M  specifies the number of rows of the matrix  C.
   M  must be at least zero.

 N
   N is INTEGER
   On entry, N specifies the number of columns of the matrix C.
   N  must be at least zero.

 ALPHA
   ALPHA is REAL
   On entry, ALPHA specifies the scalar alpha.

 A
   A is REAL array of DIMENSION ( LDA, ka ), where ka is
   m  when  SIDE = 'L' or 'l'  and is  n otherwise.

   Before entry  with  SIDE = 'L' or 'l',  the  m by m  part of
   the array  A  must contain the  symmetric matrix,  such that
   when  UPLO = 'U' or 'u', the leading m by m upper triangular
   part of the array  A  must contain the upper triangular part
   of the  symmetric matrix and the  strictly  lower triangular
   part of  A  is not referenced,  and when  UPLO = 'L' or 'l',
   the leading  m by m  lower triangular part  of the  array  A
   must  contain  the  lower triangular part  of the  symmetric
```

```
  matrix and the  strictly upper triangular part of  A  is not
  referenced.

  Before entry  with  SIDE = 'R' or 'r',  the  n by n  part of
  the array  A  must contain the  symmetric matrix,  such that
  when  UPLO = 'U' or 'u', the leading n by n upper triangular
  part of the array  A  must contain the upper triangular part
  of the  symmetric matrix and the  strictly  lower triangular
  part of  A  is not referenced,  and when  UPLO = 'L' or 'l',
  the leading  n by n  lower triangular part  of the  array  A
  must  contain  the  lower triangular part  of the  symmetric
  matrix and the  strictly upper triangular part of  A  is not
  referenced.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program.  When  SIDE = 'L' or 'l'  then
  LDA must be at least  max( 1, m ), otherwise  LDA must be at
  least  max( 1, n ).

B
  B is REAL array of DIMENSION ( LDB, n ).
  Before entry, the leading  m by n part of the array  B  must
  contain the matrix B.

LDB
  LDB is INTEGER
  On entry, LDB specifies the first dimension of B as declared
  in  the  calling  (sub)  program.  LDB  must  be  at  least
  max( 1, m ).

BETA
  BETA is REAL
  On entry,  BETA  specifies the scalar  beta.  When  BETA  is
  supplied as zero then C need not be set on input.

C
  C is REAL array of DIMENSION ( LDC, n ).
  Before entry, the leading  m by n  part of the array  C must
  contain the matrix  C,  except when  beta  is zero, in which
  case C need not be set on entry.
  On exit, the array  C  is overwritten by the  m by n updated
  matrix.

LDC
  LDC is INTEGER
  On entry, LDC specifies the first dimension of C as declared
  in  the  calling  (sub)  program.  LDC  must  be  at  least
  max( 1, m ).
```

———-

14.2.3 fortran code

— **ssymm.f** —

```
*  ====================================================================
      SUBROUTINE SSYMM(SIDE,UPLO,M,N,ALPHA,A,LDA,B,LDB,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL ALPHA,BETA
      INTEGER LDA,LDB,LDC,M,N
      CHARACTER SIDE,UPLO
*     ..
*     .. Array Arguments ..
      REAL A(LDA,*),B(LDB,*),C(LDC,*)
*     ..
*
*  ====================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*     .. Local Scalars ..
      REAL TEMP1,TEMP2
      INTEGER I,INFO,J,K,NROWA
      LOGICAL UPPER
*     ..
*     .. Parameters ..
      REAL ONE,ZERO
      PARAMETER (ONE=1.0E+0,ZERO=0.0E+0)
*     ..
*
*     Set NROWA as the number of rows of A.
```

```
*
      IF (LSAME(SIDE,'L')) THEN
          NROWA = M
      ELSE
          NROWA = N
      END IF
      UPPER = LSAME(UPLO,'U')
*
*     Test the input parameters.
*
      INFO = 0
      IF ((.NOT.LSAME(SIDE,'L')) .AND. (.NOT.LSAME(SIDE,'R'))) THEN
          INFO = 1
      ELSE IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 2
      ELSE IF (M.LT.0) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 7
      ELSE IF (LDB.LT.MAX(1,M)) THEN
          INFO = 9
      ELSE IF (LDC.LT.MAX(1,M)) THEN
          INFO = 12
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('SSYMM ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR.
     +    ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
          IF (BETA.EQ.ZERO) THEN
              DO 20 J = 1,N
                  DO 10 I = 1,M
                      C(I,J) = ZERO
   10             CONTINUE
   20         CONTINUE
          ELSE
              DO 40 J = 1,N
                  DO 30 I = 1,M
                      C(I,J) = BETA*C(I,J)
   30             CONTINUE
```

```
   40         CONTINUE
          END IF
          RETURN
      END IF
*
*     Start the operations.
*
      IF (LSAME(SIDE,'L')) THEN
*
*        Form  C := alpha*A*B + beta*C.
*
          IF (UPPER) THEN
              DO 70 J = 1,N
                  DO 60 I = 1,M
                      TEMP1 = ALPHA*B(I,J)
                      TEMP2 = ZERO
                      DO 50 K = 1,I - 1
                          C(K,J) = C(K,J) + TEMP1*A(K,I)
                          TEMP2 = TEMP2 + B(K,J)*A(K,I)
   50                 CONTINUE
                      IF (BETA.EQ.ZERO) THEN
                          C(I,J) = TEMP1*A(I,I) + ALPHA*TEMP2
                      ELSE
                          C(I,J) = BETA*C(I,J) + TEMP1*A(I,I) +
     +                             ALPHA*TEMP2
                      END IF
   60             CONTINUE
   70         CONTINUE
          ELSE
              DO 100 J = 1,N
                  DO 90 I = M,1,-1
                      TEMP1 = ALPHA*B(I,J)
                      TEMP2 = ZERO
                      DO 80 K = I + 1,M
                          C(K,J) = C(K,J) + TEMP1*A(K,I)
                          TEMP2 = TEMP2 + B(K,J)*A(K,I)
   80                 CONTINUE
                      IF (BETA.EQ.ZERO) THEN
                          C(I,J) = TEMP1*A(I,I) + ALPHA*TEMP2
                      ELSE
                          C(I,J) = BETA*C(I,J) + TEMP1*A(I,I) +
     +                             ALPHA*TEMP2
                      END IF
   90             CONTINUE
  100         CONTINUE
          END IF
      ELSE
*
*        Form  C := alpha*B*A + beta*C.
*
```

```
         DO 170 J = 1,N
             TEMP1 = ALPHA*A(J,J)
             IF (BETA.EQ.ZERO) THEN
                 DO 110 I = 1,M
                     C(I,J) = TEMP1*B(I,J)
 110             CONTINUE
             ELSE
                 DO 120 I = 1,M
                     C(I,J) = BETA*C(I,J) + TEMP1*B(I,J)
 120             CONTINUE
             END IF
             DO 140 K = 1,J - 1
                 IF (UPPER) THEN
                     TEMP1 = ALPHA*A(K,J)
                 ELSE
                     TEMP1 = ALPHA*A(J,K)
                 END IF
                 DO 130 I = 1,M
                     C(I,J) = C(I,J) + TEMP1*B(I,K)
 130             CONTINUE
 140         CONTINUE
             DO 160 K = J + 1,N
                 IF (UPPER) THEN
                     TEMP1 = ALPHA*A(J,K)
                 ELSE
                     TEMP1 = ALPHA*A(K,J)
                 END IF
                 DO 150 I = 1,M
                     C(I,J) = C(I,J) + TEMP1*B(I,K)
 150             CONTINUE
 160         CONTINUE
 170     CONTINUE
     END IF
*
     RETURN
*
*    End of SSYMM .
*
     END
```

———-

14.2.4 lisp code

— BLAS 3 ssymm —

```
(let* ((one 1.0) (zero 0.0))
```

```
(declare (type (single-float 1.0 1.0) one)
         (type (single-float 0.0 0.0) zero)
         (ignorable one zero))
(defun ssymm (side uplo m n alpha a lda b ldb$ beta c ldc)
  (declare (type (array single-float (*)) c b a)
           (type (single-float) beta alpha)
           (type (f2cl-lib:integer4) ldc ldb$ lda n m)
           (type (string *) uplo side))
  (f2cl-lib:with-multi-array-data
      ((side character side-%data% side-%offset%)
       (uplo character uplo-%data% uplo-%offset%)
       (a single-float a-%data% a-%offset%)
       (b single-float b-%data% b-%offset%)
       (c single-float c-%data% c-%offset%))
    (prog ((upper nil) (i 0) (info 0) (j 0) (k 0) (nrowa 0) (temp1 0.0)
           (temp2 0.0))
      (declare (type f2cl-lib:logical upper)
               (type (f2cl-lib:integer4) i info j k nrowa)
               (type (single-float) temp1 temp2))
      (cond ((lsame side "L") (setf nrowa m)) (t (setf nrowa n)))
      (setf upper (lsame uplo "U"))
      (setf info 0)
      (cond
       ((and (not (lsame side "L")) (not (lsame side "R"))) (setf info 1))
       ((and (not upper) (not (lsame uplo "L"))) (setf info 2))
       ((< m 0) (setf info 3)) ((< n 0) (setf info 4))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nrowa)))
        (setf info 7))
       ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
        (setf info 9))
       ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
        (setf info 12)))
      (cond
       ((/= info 0)
        (multiple-value-bind (var-0 var-1)
            (xerbla "SSYMM " info)
          (declare (ignore var-0))
          (when var-1 (setf info var-1)))
        (go end_label)))
      (if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
          (go end_label))
      (cond
       ((= alpha zero)
        (cond
         ((= beta zero)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i m) nil)
```

```
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                    zero)
             label10))
         label20)))
     (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                      (* beta
                         (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                        c-%offset%)))
             label30))
         label40))))
   (go end_label)))
 (cond
  ((lsame side "L")
   (cond
    (upper
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf temp1
                     (* alpha
                        (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)))
             (setf temp2 zero)
             (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                           ((> k
                               (f2cl-lib:int-add i
                                                 (f2cl-lib:int-sub 1)))
                            nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data% (k j) ((1 ldc) (1 *))
                                       c-%offset%)
                         (+
                          (f2cl-lib:fref c-%data% (k j) ((1 ldc) (1 *))
                                         c-%offset%)
                          (* temp1
                             (f2cl-lib:fref a-%data% (k i)
                                            ((1 lda) (1 *))
```

```
                                        a-%offset%))))
                (setf temp2
                        (+ temp2
                           (*
                            (f2cl-lib:fref b-%data% (k j)
                                           ((1 ldb$) (1 *)) b-%offset%)
                            (f2cl-lib:fref a-%data% (k i)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
               label50))
             (cond
              ((= beta zero)
               (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                     (+
                      (* temp1
                         (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                        a-%offset%))
                      (* alpha temp2))))
              (t
               (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                     (+
                      (* beta
                         (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                        c-%offset%))
                      (* temp1
                         (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                        a-%offset%))
                      (* alpha temp2)))))
            label60))
        label70)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                       ((> i 1) nil)
           (tagbody
             (setf temp1
                     (* alpha
                        (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)))
             (setf temp2 zero)
             (f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
                            (f2cl-lib:int-add k 1))
                           ((> k m) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data% (k j) ((1 ldc) (1 *))
                                      c-%offset%)
```

```
                        (+
                         (f2cl-lib:fref c-%data% (k j) ((1 ldc) (1 *))
                                        c-%offset%)
                         (* temp1
                            (f2cl-lib:fref a-%data% (k i)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
                 (setf temp2
                        (+ temp2
                           (*
                            (f2cl-lib:fref b-%data% (k j)
                                           ((1 ldb$) (1 *)) b-%offset%)
                            (f2cl-lib:fref a-%data% (k i)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
                  label80))
               (cond
                ((= beta zero)
                 (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)
                       (+
                        (* temp1
                           (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                          a-%offset%))
                        (* alpha temp2))))
                (t
                 (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)
                       (+
                        (* beta
                           (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                          c-%offset%))
                        (* temp1
                           (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                          a-%offset%))
                        (* alpha temp2)))))
            label90))
        label100)))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf temp1
             (* alpha
                (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                               a-%offset%)))
     (cond
      ((= beta zero)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
```

```
      (tagbody
        (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                             c-%offset%)
                (* temp1
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)))
       label110)))
    (t
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i m) nil)
       (tagbody
         (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                              c-%offset%)
                 (+
                  (* beta
                     (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%))
                  (* temp1
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%))))
       label120))))
   (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                 ((> k (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                  nil)
     (tagbody
       (cond
        (upper
         (setf temp1
                 (* alpha
                    (f2cl-lib:fref a-%data% (k j) ((1 lda) (1 *))
                                   a-%offset%))))
        (t
         (setf temp1
                 (* alpha
                    (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                   a-%offset%)))))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                c-%offset%)
                   (+
                    (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                   c-%offset%)
                    (* temp1
                       (f2cl-lib:fref b-%data% (i k)
                                      ((1 ldb$) (1 *))
                                      b-%offset%))))
         label130))
       label140))
```

```
            (f2cl-lib:fdo (k (f2cl-lib:int-add j 1) (f2cl-lib:int-add k 1))
                          ((> k n) nil)
              (tagbody
                (cond
                 (upper
                  (setf temp1
                          (* alpha
                             (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                            a-%offset%))))
                 (t
                  (setf temp1
                          (* alpha
                             (f2cl-lib:fref a-%data% (k j) ((1 lda) (1 *))
                                            a-%offset%)))))
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i m) nil)
                  (tagbody
                    (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                         c-%offset%)
                          (+
                           (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                          c-%offset%)
                           (* temp1
                              (f2cl-lib:fref b-%data% (i k)
                                             ((1 ldb$) (1 *))
                                             b-%offset%))))
                   label150))
              label160))
          label170))))
     (go end_label)
    end_label
     (return (values nil nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

14.3 ssyr2k - rank2k $\alpha AB^T + \alpha BA^T + \beta C$ or $\alpha A^T B + \alpha B^T A + \beta C$

14.3.1 Axiom unit tests

— **ssyr2k.input** —

```
)set break resume
)sys rm -f ssyr2k.output
)spool ssyr2k.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

14.3.2 Axiom help page

— ssyr2k.help —

```
=====================================================================
ssyr2k examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssyr2k (UPLO, TRANS, N, K, ALPHA, A, LDA, B, LDB, BETA, C,
           LDC)
           SSYR2K

Function/Subroutine Documentation
   subroutine ssyr2k (character UPLO, character TRANS, integer N, integer K,
       real ALPHA, real, dimension(lda,*) A, integer LDA, real,
       dimension(ldb,*) B, integer LDB, real BETA, real, dimension(ldc,*) C,
       integer LDC)
       SSYR2K

Purpose:
   SSYR2K  performs one of the symmetric rank 2k operations

     C := alpha*A*B**T + alpha*B*A**T + beta*C,

   or

     C := alpha*A**T*B + alpha*B**T*A + beta*C,

   where  alpha and beta  are scalars, C is an  n by n  symmetric matrix
   and  A and B  are  n by k  matrices  in the  first  case  and  k by n
   matrices in the second case.

Parameters:
```

```
UPLO
  UPLO is CHARACTER*1
  On  entry,   UPLO  specifies  whether  the  upper  or  lower
  triangular  part  of the  array  C  is to be  referenced  as follows:

  UPLO = 'U' or 'u'   Only the  upper triangular part of  C
  is to be referenced.

  UPLO = 'L' or 'l'   Only the  lower triangular part of  C
  is to be referenced.

TRANS
  TRANS is CHARACTER*1
  On entry,  TRANS  specifies the operation to be performed as follows:

  TRANS = 'N' or 'n'   C := alpha*A*B**T + alpha*B*A**T + beta*C.

  TRANS = 'T' or 't'   C := alpha*A**T*B + alpha*B**T*A + beta*C.

  TRANS = 'C' or 'c'   C := alpha*A**T*B + alpha*B**T*A + beta*C.

N
  N is INTEGER
  On entry,  N specifies the order of the matrix C.  N must be
  at least zero.

K
  K is INTEGER
  On entry with  TRANS = 'N' or 'n',  K  specifies  the number
  of  columns  of the  matrices  A and B,  and on  entry  with
  TRANS = 'T' or 't' or 'C' or 'c',  K  specifies  the  number
  of rows of the matrices  A and B.  K must be at least  zero.

ALPHA
  ALPHA is REAL
  On entry, ALPHA specifies the scalar alpha.

A
  A is REAL array of DIMENSION ( LDA, ka ), where ka is
  k  when  TRANS = 'N' or 'n',  and is  n  otherwise.
  Before entry with  TRANS = 'N' or 'n',  the  leading  n by k
  part of the array  A  must contain the matrix  A,  otherwise
  the leading  k by n  part of the array  A  must contain  the
  matrix A.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in  the  calling  (sub)  program.   When  TRANS = 'N' or 'n'
```

```
  then  LDA must be at least  max( 1, n ), otherwise  LDA must
  be at least  max( 1, k ).

B
  B is REAL array of DIMENSION ( LDB, kb ), where kb is
  k  when  TRANS = 'N' or 'n',  and is  n  otherwise.
  Before entry with  TRANS = 'N' or 'n',  the  leading  n by k
  part of the array  B  must contain the matrix  B,  otherwise
  the leading  k by n  part of the array  B  must contain  the
  matrix B.

LDB
  LDB is INTEGER
  On entry, LDB specifies the first dimension of B as declared
  in  the  calling  (sub)  program.   When  TRANS = 'N' or 'n'
  then  LDB must be at least  max( 1, n ), otherwise  LDB must
  be at least  max( 1, k ).

BETA
  BETA is REAL
  On entry, BETA specifies the scalar beta.

C
  C is REAL array of DIMENSION ( LDC, n ).

  Before entry  with  UPLO = 'U' or 'u',  the leading  n by n
  upper triangular part of the array C must contain the upper
  triangular part  of the  symmetric matrix  and the strictly
  lower triangular part of C is not referenced.  On exit, the
  upper triangular part of the array  C is overwritten by the
  upper triangular part of the updated matrix.

  Before entry  with  UPLO = 'L' or 'l',  the leading  n by n
  lower triangular part of the array C must contain the lower
  triangular part  of the  symmetric matrix  and the strictly
  upper triangular part of C is not referenced.  On exit, the
  lower triangular part of the array  C is overwritten by the
  lower triangular part of the updated matrix.

LDC
  LDC is INTEGER
  On entry, LDC specifies the first dimension of C as declared
  in  the  calling  (sub)  program.  LDC  must  be  at  least
  max( 1, n ).
```

14.3.3 fortran code

— ssyr2k.f —

```
*  ===================================================================
      SUBROUTINE SSYR2K(UPLO,TRANS,N,K,ALPHA,A,LDA,B,LDB,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL ALPHA,BETA
      INTEGER K,LDA,LDB,LDC,N
      CHARACTER TRANS,UPLO
*     ..
*     .. Array Arguments ..
      REAL A(LDA,*),B(LDB,*),C(LDC,*)
*     ..
*
*  ===================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*     .. Local Scalars ..
      REAL TEMP1,TEMP2
      INTEGER I,INFO,J,L,NROWA
      LOGICAL UPPER
*     ..
*     .. Parameters ..
      REAL ONE,ZERO
      PARAMETER (ONE=1.0E+0,ZERO=0.0E+0)
*     ..
*
*     Test the input parameters.
*
      IF (LSAME(TRANS,'N')) THEN
          NROWA = N
      ELSE
          NROWA = K
```

```
      END IF
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 1
      ELSE IF ((.NOT.LSAME(TRANS,'N')) .AND.
     +         (.NOT.LSAME(TRANS,'T')) .AND.
     +         (.NOT.LSAME(TRANS,'C'))) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
          INFO = 3
      ELSE IF (K.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 7
      ELSE IF (LDB.LT.MAX(1,NROWA)) THEN
          INFO = 9
      ELSE IF (LDC.LT.MAX(1,N)) THEN
          INFO = 12
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('SSYR2K',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (((ALPHA.EQ.ZERO).OR.
     +    (K.EQ.0)).AND. (BETA.EQ.ONE))) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
          IF (UPPER) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 20 J = 1,N
                      DO 10 I = 1,J
                          C(I,J) = ZERO
   10                 CONTINUE
   20             CONTINUE
              ELSE
                  DO 40 J = 1,N
                      DO 30 I = 1,J
                          C(I,J) = BETA*C(I,J)
   30                 CONTINUE
   40             CONTINUE
              END IF
          ELSE
              IF (BETA.EQ.ZERO) THEN
```

```
                DO 60 J = 1,N
                    DO 50 I = J,N
                        C(I,J) = ZERO
   50               CONTINUE
   60           CONTINUE
            ELSE
                DO 80 J = 1,N
                    DO 70 I = J,N
                        C(I,J) = BETA*C(I,J)
   70               CONTINUE
   80           CONTINUE
            END IF
        END IF
        RETURN
    END IF
*
*   Start the operations.
*
    IF (LSAME(TRANS,'N')) THEN
*
*       Form  C := alpha*A*B**T + alpha*B*A**T + C.
*
        IF (UPPER) THEN
            DO 130 J = 1,N
                IF (BETA.EQ.ZERO) THEN
                    DO 90 I = 1,J
                        C(I,J) = ZERO
   90               CONTINUE
                ELSE IF (BETA.NE.ONE) THEN
                    DO 100 I = 1,J
                        C(I,J) = BETA*C(I,J)
  100               CONTINUE
                END IF
                DO 120 L = 1,K
                    IF ((A(J,L).NE.ZERO) .OR. (B(J,L).NE.ZERO)) THEN
                        TEMP1 = ALPHA*B(J,L)
                        TEMP2 = ALPHA*A(J,L)
                        DO 110 I = 1,J
                            C(I,J) = C(I,J) + A(I,L)*TEMP1 +
     +                               B(I,L)*TEMP2
  110                   CONTINUE
                    END IF
  120           CONTINUE
  130       CONTINUE
        ELSE
            DO 180 J = 1,N
                IF (BETA.EQ.ZERO) THEN
                    DO 140 I = J,N
                        C(I,J) = ZERO
  140               CONTINUE
```

```
                ELSE IF (BETA.NE.ONE) THEN
                    DO 150 I = J,N
                        C(I,J) = BETA*C(I,J)
 150                CONTINUE
                END IF
                DO 170 L = 1,K
                    IF ((A(J,L).NE.ZERO) .OR. (B(J,L).NE.ZERO)) THEN
                        TEMP1 = ALPHA*B(J,L)
                        TEMP2 = ALPHA*A(J,L)
                        DO 160 I = J,N
                            C(I,J) = C(I,J) + A(I,L)*TEMP1 +
     +                               B(I,L)*TEMP2
 160                    CONTINUE
                    END IF
 170            CONTINUE
 180        CONTINUE
        END IF
    ELSE
*
*       Form  C := alpha*A**T*B + alpha*B**T*A + C.
*
        IF (UPPER) THEN
            DO 210 J = 1,N
                DO 200 I = 1,J
                    TEMP1 = ZERO
                    TEMP2 = ZERO
                    DO 190 L = 1,K
                        TEMP1 = TEMP1 + A(L,I)*B(L,J)
                        TEMP2 = TEMP2 + B(L,I)*A(L,J)
 190                CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP1 + ALPHA*TEMP2
                    ELSE
                        C(I,J) = BETA*C(I,J) + ALPHA*TEMP1 +
     +                           ALPHA*TEMP2
                    END IF
 200            CONTINUE
 210        CONTINUE
        ELSE
            DO 240 J = 1,N
                DO 230 I = J,N
                    TEMP1 = ZERO
                    TEMP2 = ZERO
                    DO 220 L = 1,K
                        TEMP1 = TEMP1 + A(L,I)*B(L,J)
                        TEMP2 = TEMP2 + B(L,I)*A(L,J)
 220                CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP1 + ALPHA*TEMP2
                    ELSE
```

```
                        C(I,J) = BETA*C(I,J) + ALPHA*TEMP1 +
     +                           ALPHA*TEMP2
                    END IF
  230            CONTINUE
  240         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of SSYR2K.
*
      END
```

———-

14.3.4 lisp code

— BLAS 3 ssyr2k —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun ssyr2k (uplo trans n k alpha a lda b ldb$ beta c ldc)
    (declare (type (array single-float (*)) c b a)
             (type (single-float) beta alpha)
             (type (f2cl-lib:integer4) ldc ldb$ lda k n)
             (type (string *) trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%)
         (c single-float c-%data% c-%offset%))
      (prog ((upper nil) (i 0) (info 0) (j 0) (l 0) (nrowa 0) (temp1 0.0)
             (temp2 0.0))
        (declare (type f2cl-lib:logical upper)
                 (type (f2cl-lib:integer4) i info j l nrowa)
                 (type (single-float) temp1 temp2))
        (cond ((lsame trans "N") (setf nrowa n)) (t (setf nrowa k)))
        (setf upper (lsame uplo "U"))
        (setf info 0)
        (cond ((and (not upper) (not (lsame uplo "L"))) (setf info 1))
              ((and (not (lsame trans "N")) (not (lsame trans "T"))
                    (not (lsame trans "C")))
               (setf info 2))
```

```
        ((< n 0) (setf info 3)) ((< k 0) (setf info 4))
        ((< lda
            (max (the f2cl-lib:integer4 1)
                 (the f2cl-lib:integer4 nrowa)))
         (setf info 7))
        ((< ldb$
            (max (the f2cl-lib:integer4 1)
                 (the f2cl-lib:integer4 nrowa)))
         (setf info 9))
        ((< ldc
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info 12)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "SSYR2K" info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= n 0) (and (or (= alpha zero) (= k 0)) (= beta one)))
    (go end_label))
(cond
 ((= alpha zero)
  (cond
   (upper
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                    zero)
             label10))
         label20)))
     (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                    (* beta
                       (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)))
```

```
                label30))
            label40)))))
      (t
       (cond
        ((= beta zero)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                           ((> i n) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)
                       zero)
                label50))
            label60)))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                           ((> i n) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)
                       (* beta
                          (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                         c-%offset%)))
                label70))
            label80))))))
     (go end_label)))
   (cond
    ((lsame trans "N")
     (cond
      (upper
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (cond
            ((= beta zero)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i j) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)
                       zero)
                label90)))
            ((/= beta one)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i j) nil)
```

```
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                      (* beta
                         (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                        c-%offset%)))
             label100))))
        (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                      ((> l k) nil)
          (tagbody
            (cond
             ((or (/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
                  (/= (f2cl-lib:fref b (j l) ((1 ldb$) (1 *))) zero))
              (setf temp1
                      (* alpha
                         (f2cl-lib:fref b-%data% (j l) ((1 ldb$) (1 *))
                                        b-%offset%)))
              (setf temp2
                      (* alpha
                         (f2cl-lib:fref a-%data% (j l) ((1 lda) (1 *))
                                        a-%offset%)))
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i j) nil)
                (tagbody
                  (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)
                        (+
                         (f2cl-lib:fref c-%data% (i j)
                                        ((1 ldc) (1 *)) c-%offset%)
                         (*
                          (f2cl-lib:fref a-%data% (i l)
                                         ((1 lda) (1 *)) a-%offset%)
                          temp1)
                         (*
                          (f2cl-lib:fref b-%data% (i l)
                                         ((1 ldb$) (1 *)) b-%offset%)
                          temp2)))
                 label110))))
          label120))
      label130)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((= beta zero)
         (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
```

```
                                  c-%offset%)
                        zero)
                 label140)))
            ((/= beta one)
             (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                           ((> i n) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                  c-%offset%)
                        (* beta
                           (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                          c-%offset%)))
                 label150))))
           (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                        ((> l k) nil)
             (tagbody
               (cond
                ((or (/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
                     (/= (f2cl-lib:fref b (j l) ((1 ldb$) (1 *))) zero))
                 (setf temp1
                        (* alpha
                           (f2cl-lib:fref b-%data% (j l) ((1 ldb$) (1 *))
                                          b-%offset%)))
                 (setf temp2
                        (* alpha
                           (f2cl-lib:fref a-%data% (j l) ((1 lda) (1 *))
                                          a-%offset%)))
                 (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                               ((> i n) nil)
                   (tagbody
                     (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                          c-%offset%)
                           (+
                            (f2cl-lib:fref c-%data% (i j)
                                           ((1 ldc) (1 *)) c-%offset%)
                            (*
                             (f2cl-lib:fref a-%data% (i l)
                                            ((1 lda) (1 *)) a-%offset%)
                             temp1)
                            (*
                             (f2cl-lib:fref b-%data% (i l)
                                            ((1 ldb$) (1 *)) b-%offset%)
                             temp2)))
                 label160))))
          label170))
      label180)))))
(t
 (cond
  (upper
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i j) nil)
          (tagbody
            (setf temp1 zero)
            (setf temp2 zero)
            (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                          ((> l k) nil)
              (tagbody
                (setf temp1
                        (+ temp1
                           (*
                            (f2cl-lib:fref a-%data% (l i)
                                           ((1 lda) (1 *)) a-%offset%)
                            (f2cl-lib:fref b-%data% (l j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%))))
                (setf temp2
                        (+ temp2
                           (*
                            (f2cl-lib:fref b-%data% (l i)
                                           ((1 ldb$) (1 *)) b-%offset%)
                            (f2cl-lib:fref a-%data% (l j)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
                 label190))
            (cond
             ((= beta zero)
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                   c-%offset%)
                      (+ (* alpha temp1) (* alpha temp2))))
             (t
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                   c-%offset%)
                      (+
                       (* beta
                          (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                         c-%offset%))
                       (* alpha temp1) (* alpha temp2)))))
           label200))
       label210)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf temp1 zero)
```

```
                (setf temp2 zero)
                (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                              ((> l k) nil)
                  (tagbody
                    (setf temp1
                            (+ temp1
                               (*
                                (f2cl-lib:fref a-%data% (l i)
                                               ((1 lda) (1 *)) a-%offset%)
                                (f2cl-lib:fref b-%data% (l j)
                                               ((1 ldb$) (1 *))
                                               b-%offset%))))
                    (setf temp2
                            (+ temp2
                               (*
                                (f2cl-lib:fref b-%data% (l i)
                                               ((1 ldb$) (1 *)) b-%offset%)
                                (f2cl-lib:fref a-%data% (l j)
                                               ((1 lda) (1 *))
                                               a-%offset%))))
                   label220))
                (cond
                 ((= beta zero)
                  (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)
                          (+ (* alpha temp1) (* alpha temp2))))
                 (t
                  (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)
                          (+
                           (* beta
                              (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                             c-%offset%))
                           (* alpha temp1) (* alpha temp2)))))
            label230))
        label240))))))
 (go end_label)
end_label
 (return (values nil nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

14.4 ssyrk - symmetric rank k $\alpha AA^T + \beta C$ or $\alpha A^T A + \beta C$

14.4.1 Axiom unit tests

— ssyrk.input —

```
)set break resume
)sys rm -f ssyrk.output
)spool ssyrk.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

14.4.2 Axiom help page

— ssyrk.help —

```
======================================================================
ssyrk examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssyrk (UPLO, TRANS, N, K, ALPHA, A, LDA, BETA, C, LDC)
           SSYRK

Function/Subroutine Documentation
   subroutine ssyrk (character UPLO, character TRANS, integer N, integer K,
       real ALPHA, real, dimension(lda,*) A, integer LDA, real BETA, real,
       dimension(ldc,*) C, integer LDC)
       SSYRK

Purpose:
   SSYRK  performs one of the symmetric rank k operations

     C := alpha*A*A**T + beta*C,

   or

     C := alpha*A**T*A + beta*C,

   where  alpha and beta  are scalars, C is an  n by n  symmetric matrix
   and  A  is an  n by k  matrix in the first case and a  k by n  matrix
```

```
   in the second case.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On  entry,  UPLO  specifies  whether  the  upper  or  lower
   triangular  part  of the  array  C  is to be  referenced  as
   follows:

   UPLO = 'U' or 'u'   Only the  upper triangular part of  C
   is to be referenced.

   UPLO = 'L' or 'l'   Only the  lower triangular part of  C
   is to be referenced.

 TRANS
   TRANS is CHARACTER*1
   On entry,  TRANS  specifies the operation to be performed as
   follows:

   TRANS = 'N' or 'n'   C := alpha*A*A**T + beta*C.

   TRANS = 'T' or 't'   C := alpha*A**T*A + beta*C.

   TRANS = 'C' or 'c'   C := alpha*A**T*A + beta*C.

 N
   N is INTEGER
   On entry,  N specifies the order of the matrix C.  N must be
   at least zero.

 K
   K is INTEGER
   On entry with  TRANS = 'N' or 'n',  K  specifies  the number
   of  columns  of  the  matrix  A,  and  on  entry  with
   TRANS = 'T' or 't' or 'C' or 'c',  K  specifies  the  number
   of rows of the matrix  A.  K must be at least zero.

 ALPHA
   ALPHA is REAL
   On entry, ALPHA specifies the scalar alpha.

 A
   A is REAL array of DIMENSION ( LDA, ka ), where ka is
   k  when  TRANS = 'N' or 'n',  and is  n  otherwise.
   Before entry with  TRANS = 'N' or 'n',  the  leading  n by k
   part of the array  A  must contain the matrix  A,  otherwise
   the leading  k by n  part of the array  A  must contain  the
   matrix A.
```

```
LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in  the  calling  (sub)  program.   When  TRANS = 'N' or 'n'
  then  LDA must be at least  max( 1, n ), otherwise  LDA must
  be at least  max( 1, k ).

BETA
  BETA is REAL
  On entry, BETA specifies the scalar beta.

C
  C is REAL array of DIMENSION ( LDC, n ).

  Before entry  with  UPLO = 'U' or 'u',  the leading  n by n
  upper triangular part of the array C must contain the upper
  triangular part  of the  symmetric matrix  and the strictly
  lower triangular part of C is not referenced.  On exit, the
  upper triangular part of the array  C is overwritten by the
  upper triangular part of the updated matrix.

  Before entry  with  UPLO = 'L' or 'l',  the leading  n by n
  lower triangular part of the array C must contain the lower
  triangular part  of the  symmetric matrix  and the strictly
  upper triangular part of C is not referenced.  On exit, the
  lower triangular part of the array  C is overwritten by the
  lower triangular part of the updated matrix.

LDC
  LDC is INTEGER
  On entry, LDC specifies the first dimension of C as declared
  in  the  calling  (sub)  program.   LDC  must  be  at  least
  max( 1, n ).
```

———-

14.4.3 fortran code

— **ssyrk.f** —

```
*  ====================================================================
      SUBROUTINE SSYRK(UPLO,TRANS,N,K,ALPHA,A,LDA,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
```

```
*     November 2011
*
*     .. Scalar Arguments ..
      REAL ALPHA,BETA
      INTEGER K,LDA,LDC,N
      CHARACTER TRANS,UPLO
*     ..
*     .. Array Arguments ..
      REAL A(LDA,*),C(LDC,*)
*     ..
*
*  =====================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*     .. Local Scalars ..
      REAL TEMP
      INTEGER I,INFO,J,L,NROWA
      LOGICAL UPPER
*     ..
*     .. Parameters ..
      REAL ONE,ZERO
      PARAMETER (ONE=1.0E+0,ZERO=0.0E+0)
*     ..
*
*     Test the input parameters.
*
      IF (LSAME(TRANS,'N')) THEN
          NROWA = N
      ELSE
          NROWA = K
      END IF
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 1
      ELSE IF ((.NOT.LSAME(TRANS,'N')) .AND.
     +         (.NOT.LSAME(TRANS,'T')) .AND.
     +         (.NOT.LSAME(TRANS,'C'))) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
```

```
         INFO = 3
      ELSE IF (K.LT.0) THEN
         INFO = 4
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
         INFO = 7
      ELSE IF (LDC.LT.MAX(1,N)) THEN
         INFO = 10
      END IF
      IF (INFO.NE.0) THEN
         CALL XERBLA('SSYRK ',INFO)
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (((ALPHA.EQ.ZERO).OR.
     +    (K.EQ.0)).AND. (BETA.EQ.ONE))) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
         IF (UPPER) THEN
            IF (BETA.EQ.ZERO) THEN
               DO 20 J = 1,N
                  DO 10 I = 1,J
                     C(I,J) = ZERO
   10             CONTINUE
   20          CONTINUE
            ELSE
               DO 40 J = 1,N
                  DO 30 I = 1,J
                     C(I,J) = BETA*C(I,J)
   30             CONTINUE
   40          CONTINUE
            END IF
         ELSE
            IF (BETA.EQ.ZERO) THEN
               DO 60 J = 1,N
                  DO 50 I = J,N
                     C(I,J) = ZERO
   50             CONTINUE
   60          CONTINUE
            ELSE
               DO 80 J = 1,N
                  DO 70 I = J,N
                     C(I,J) = BETA*C(I,J)
   70             CONTINUE
   80          CONTINUE
            END IF
         END IF
```

```
         RETURN
      END IF
*
*     Start the operations.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  C := alpha*A*A**T + beta*C.
*
         IF (UPPER) THEN
            DO 130 J = 1,N
               IF (BETA.EQ.ZERO) THEN
                  DO 90 I = 1,J
                     C(I,J) = ZERO
   90             CONTINUE
               ELSE IF (BETA.NE.ONE) THEN
                  DO 100 I = 1,J
                     C(I,J) = BETA*C(I,J)
  100             CONTINUE
               END IF
               DO 120 L = 1,K
                  IF (A(J,L).NE.ZERO) THEN
                     TEMP = ALPHA*A(J,L)
                     DO 110 I = 1,J
                        C(I,J) = C(I,J) + TEMP*A(I,L)
  110                CONTINUE
                  END IF
  120          CONTINUE
  130       CONTINUE
         ELSE
            DO 180 J = 1,N
               IF (BETA.EQ.ZERO) THEN
                  DO 140 I = J,N
                     C(I,J) = ZERO
  140             CONTINUE
               ELSE IF (BETA.NE.ONE) THEN
                  DO 150 I = J,N
                     C(I,J) = BETA*C(I,J)
  150             CONTINUE
               END IF
               DO 170 L = 1,K
                  IF (A(J,L).NE.ZERO) THEN
                     TEMP = ALPHA*A(J,L)
                     DO 160 I = J,N
                        C(I,J) = C(I,J) + TEMP*A(I,L)
  160                CONTINUE
                  END IF
  170          CONTINUE
  180       CONTINUE
         END IF
```

```
      ELSE
*
*        Form  C := alpha*A**T*A + beta*C.
*
          IF (UPPER) THEN
              DO 210 J = 1,N
                  DO 200 I = 1,J
                      TEMP = ZERO
                      DO 190 L = 1,K
                          TEMP = TEMP + A(L,I)*A(L,J)
  190                 CONTINUE
                      IF (BETA.EQ.ZERO) THEN
                          C(I,J) = ALPHA*TEMP
                      ELSE
                          C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                      END IF
  200             CONTINUE
  210         CONTINUE
          ELSE
              DO 240 J = 1,N
                  DO 230 I = J,N
                      TEMP = ZERO
                      DO 220 L = 1,K
                          TEMP = TEMP + A(L,I)*A(L,J)
  220                 CONTINUE
                      IF (BETA.EQ.ZERO) THEN
                          C(I,J) = ALPHA*TEMP
                      ELSE
                          C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                      END IF
  230             CONTINUE
  240         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of SSYRK .
*
      END
```

———-

14.4.4 lisp code

— BLAS 3 ssyrk —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun ssyrk (uplo trans n k alpha a lda beta c ldc)
    (declare (type (array single-float (*)) c a)
             (type (single-float) beta alpha)
             (type (f2cl-lib:integer4) ldc lda k n)
             (type (string *) trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (a single-float a-%data% a-%offset%)
         (c single-float c-%data% c-%offset%))
      (prog ((upper nil) (i 0) (info 0) (j 0) (l 0) (nrowa 0) (temp 0.0))
        (declare (type f2cl-lib:logical upper)
                 (type (f2cl-lib:integer4) i info j l nrowa)
                 (type (single-float) temp))
        (cond ((lsame trans "N") (setf nrowa n)) (t (setf nrowa k)))
        (setf upper (lsame uplo "U"))
        (setf info 0)
        (cond ((and (not upper) (not (lsame uplo "L"))) (setf info 1))
              ((and (not (lsame trans "N")) (not (lsame trans "T"))
                    (not (lsame trans "C")))
               (setf info 2))
              ((< n 0) (setf info 3)) ((< k 0) (setf info 4))
              ((< lda
                  (max (the f2cl-lib:integer4 1)
                       (the f2cl-lib:integer4 nrowa)))
               (setf info 7))
              ((< ldc
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info 10)))
        (cond
         ((/= info 0)
          (multiple-value-bind (var-0 var-1)
              (xerbla "SSYRK " info)
            (declare (ignore var-0))
            (when var-1 (setf info var-1)))
          (go end_label)))
        (if (or (= n 0) (and (or (= alpha zero) (= k 0)) (= beta one)))
            (go end_label))
        (cond
         ((= alpha zero)
          (cond
           (upper
            (cond
             ((= beta zero)
              (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                            ((> j n) nil)
```

```
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                   c-%offset%)
                    zero)
            label10))
         label20)))
      (t
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                   c-%offset%)
                    (* beta
                       (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)))
            label30))
         label40)))))
    (t
     (cond
      ((= beta zero)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                   c-%offset%)
                    zero)
            label50))
         label60)))
      (t
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                   c-%offset%)
                    (* beta
                       (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)))
            label70))
```

```
          label80))))))
  (go end_label)))
(cond
 ((lsame trans "N")
  (cond
   (upper
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
         ((= beta zero)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                    zero)
           label90)))
         ((/= beta one)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                    (* beta
                       (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)))
           label100))))
        (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                      ((> l k) nil)
          (tagbody
            (cond
             ((/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
              (setf temp
                    (* alpha
                       (f2cl-lib:fref a-%data% (j l) ((1 lda) (1 *))
                                      a-%offset%)))
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i j) nil)
                (tagbody
                  (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                        c-%offset%)
                        (+
                         (f2cl-lib:fref c-%data% (i j)
                                        ((1 ldc) (1 *)) c-%offset%)
                         (* temp
                            (f2cl-lib:fref a-%data% (i l)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
                 label110))))
```

```
            label120))
        label130)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (cond
          ((= beta zero)
           (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                     zero)
              label140)))
          ((/= beta one)
           (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                     (* beta
                        (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)))
              label150))))
       (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                     ((> l k) nil)
         (tagbody
           (cond
            ((/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
             (setf temp
                     (* alpha
                        (f2cl-lib:fref a-%data% (j l) ((1 lda) (1 *))
                                       a-%offset%)))
             (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                           ((> i n) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)
                       (+
                        (f2cl-lib:fref c-%data% (i j)
                                       ((1 ldc) (1 *)) c-%offset%)
                        (* temp
                           (f2cl-lib:fref a-%data% (i l)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
                label160))))
          label170))
      label180)))))
(t
```

```
(cond
 (upper
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i j) nil)
        (tagbody
          (setf temp zero)
          (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                        ((> l k) nil)
            (tagbody
              (setf temp
                      (+ temp
                         (*
                          (f2cl-lib:fref a-%data% (l i)
                                         ((1 lda) (1 *)) a-%offset%)
                          (f2cl-lib:fref a-%data% (l j)
                                         ((1 lda) (1 *))
                                         a-%offset%))))
             label190))
          (cond
           ((= beta zero)
            (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                 c-%offset%)
                    (* alpha temp)))
           (t
            (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                 c-%offset%)
                    (+ (* alpha temp)
                       (* beta
                          (f2cl-lib:fref c-%data% (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%))))))
         label200))
     label210)))
 (t
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf temp zero)
          (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                        ((> l k) nil)
            (tagbody
              (setf temp
                      (+ temp
                         (*
```

```
                                  (f2cl-lib:fref a-%data% (l i)
                                                 ((1 lda) (1 *)) a-%offset%)
                                  (f2cl-lib:fref a-%data% (l j)
                                                 ((1 lda) (1 *))
                                                 a-%offset%))))
                     label220))
                   (cond
                    ((= beta zero)
                     (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                          c-%offset%)
                             (* alpha temp)))
                    (t
                     (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                          c-%offset%)
                             (+ (* alpha temp)
                                (* beta
                                   (f2cl-lib:fref c-%data% (i j)
                                                  ((1 ldc) (1 *))
                                                  c-%offset%))))))
                 label230))
             label240))))))
      (go end_label)
     end_label
      (return (values nil nil nil nil nil nil nil nil nil nil))))))
```

———-

14.5 strmm - $\alpha op(A)B$ or $\alpha Bop(A)$, $op(A) = A$ or $op(A) = A^T$

14.5.1 Axiom unit tests

— **strmm.input** —

```
)set break resume
)sys rm -f strmm.output
)spool strmm.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

14.5.2 Axiom help page

— strmm.help —

```
====================================================================
strmm examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine strmm (SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B,
           LDB)
           STRMM

Function/Subroutine Documentation
   subroutine strmm (character SIDE, character UPLO, character TRANSA,
       character DIAG, integer M, integer N, real ALPHA, real,
       dimension(lda,*) A, integer LDA, real, dimension(ldb,*) B, integer LDB)
       STRMM

Purpose:
   STRMM  performs one of the matrix-matrix operations

     B := alpha*op( A )*B,   or   B := alpha*B*op( A ),

   where  alpha  is a scalar,  B  is an m by n matrix,  A  is a unit, or
   non-unit,  upper or lower triangular matrix  and  op( A )  is one  of

     op( A ) = A   or   op( A ) = A**T.

Parameters:

 SIDE
   SIDE is CHARACTER*1
   On entry,  SIDE specifies whether  op( A ) multiplies B from
   the left or right as follows:

   SIDE = 'L' or 'l'   B := alpha*op( A )*B.

   SIDE = 'R' or 'r'   B := alpha*B*op( A ).

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the matrix A is an upper or
   lower triangular matrix as follows:
```

```
  UPLO = 'U' or 'u'   A is an upper triangular matrix.

  UPLO = 'L' or 'l'   A is a lower triangular matrix.

TRANSA
  TRANSA is CHARACTER*1
  On entry, TRANSA specifies the form of op( A ) to be used in
  the matrix multiplication as follows:

  TRANSA = 'N' or 'n'   op( A ) = A.

  TRANSA = 'T' or 't'   op( A ) = A**T.

  TRANSA = 'C' or 'c'   op( A ) = A**T.

DIAG
  DIAG is CHARACTER*1
  On entry, DIAG specifies whether or not A is unit triangular
  as follows:

  DIAG = 'U' or 'u'   A is assumed to be unit triangular.

  DIAG = 'N' or 'n'   A is not assumed to be unit triangular.

M
  M is INTEGER
  On entry, M specifies the number of rows of B. M must be at
  least zero.

N
  N is INTEGER
  On entry, N specifies the number of columns of B.  N must be
  at least zero.

ALPHA
  ALPHA is REAL
  On entry,  ALPHA specifies the scalar  alpha. When  alpha is
  zero then  A is not referenced and  B need not be set before entry.

A
  A is REAL array of DIMENSION ( LDA, k ), where k is m
  when  SIDE = 'L' or 'l'  and is  n  when  SIDE = 'R' or 'r'.
  Before entry  with  UPLO = 'U' or 'u',  the  leading  k by k
  upper triangular part of the array  A must contain the upper
  triangular matrix  and the strictly lower triangular part of
  A is not referenced.

  Before entry  with  UPLO = 'L' or 'l',  the  leading  k by k
  lower triangular part of the array  A must contain the lower
```

```
  triangular matrix  and the strictly upper triangular part of
  A is not referenced.

  Note that when  DIAG = 'U' or 'u',  the diagonal elements of
  A  are not referenced either,  but are assumed to be  unity.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program.  When  SIDE = 'L' or 'l'  then
  LDA  must be at least  max( 1, m ),  when  SIDE = 'R' or 'r'
  then LDA must be at least max( 1, n ).

B
  B is REAL array of DIMENSION ( LDB, n ).
  Before entry,  the leading  m by n part of the array  B must
  contain the matrix  B,  and  on exit  is overwritten  by the
  transformed matrix.

LDB
  LDB is INTEGER
  On entry, LDB specifies the first dimension of B as declared
  in  the  calling  (sub)  program.  LDB  must  be  at  least
  max( 1, m ).
```

———-

14.5.3 fortran code

— strmm.f —

```
*  ===================================================================
      SUBROUTINE STRMM(SIDE,UPLO,TRANSA,DIAG,M,N,ALPHA,A,LDA,B,LDB)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL ALPHA
      INTEGER LDA,LDB,M,N
      CHARACTER DIAG,SIDE,TRANSA,UPLO
*     ..
*     .. Array Arguments ..
      REAL A(LDA,*),B(LDB,*)
*     ..
```

```
*
*  =====================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*     .. Local Scalars ..
      REAL TEMP
      INTEGER I,INFO,J,K,NROWA
      LOGICAL LSIDE,NOUNIT,UPPER
*     ..
*     .. Parameters ..
      REAL ONE,ZERO
      PARAMETER (ONE=1.0E+0,ZERO=0.0E+0)
*     ..
*
*     Test the input parameters.
*
      LSIDE = LSAME(SIDE,'L')
      IF (LSIDE) THEN
          NROWA = M
      ELSE
          NROWA = N
      END IF
      NOUNIT = LSAME(DIAG,'N')
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.LSIDE) .AND. (.NOT.LSAME(SIDE,'R'))) THEN
          INFO = 1
      ELSE IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 2
      ELSE IF ((.NOT.LSAME(TRANSA,'N')) .AND.
     +         (.NOT.LSAME(TRANSA,'T')) .AND.
     +         (.NOT.LSAME(TRANSA,'C'))) THEN
          INFO = 3
      ELSE IF ((.NOT.LSAME(DIAG,'U')) .AND. (.NOT.LSAME(DIAG,'N'))) THEN
          INFO = 4
      ELSE IF (M.LT.0) THEN
          INFO = 5
      ELSE IF (N.LT.0) THEN
          INFO = 6
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
```

```
         INFO = 9
      ELSE IF (LDB.LT.MAX(1,M)) THEN
         INFO = 11
      END IF
      IF (INFO.NE.0) THEN
         CALL XERBLA('STRMM ',INFO)
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF (M.EQ.0 .OR. N.EQ.0) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
         DO 20 J = 1,N
            DO 10 I = 1,M
               B(I,J) = ZERO
   10       CONTINUE
   20    CONTINUE
         RETURN
      END IF
*
*     Start the operations.
*
      IF (LSIDE) THEN
         IF (LSAME(TRANSA,'N')) THEN
*
*          Form  B := alpha*A*B.
*
            IF (UPPER) THEN
               DO 50 J = 1,N
                  DO 40 K = 1,M
                     IF (B(K,J).NE.ZERO) THEN
                        TEMP = ALPHA*B(K,J)
                        DO 30 I = 1,K - 1
                           B(I,J) = B(I,J) + TEMP*A(I,K)
   30                   CONTINUE
                        IF (NOUNIT) TEMP = TEMP*A(K,K)
                        B(K,J) = TEMP
                     END IF
   40             CONTINUE
   50          CONTINUE
            ELSE
               DO 80 J = 1,N
                  DO 70 K = M,1,-1
                     IF (B(K,J).NE.ZERO) THEN
                        TEMP = ALPHA*B(K,J)
                        B(K,J) = TEMP
```

```
                            IF (NOUNIT) B(K,J) = B(K,J)*A(K,K)
                            DO 60 I = K + 1,M
                                B(I,J) = B(I,J) + TEMP*A(I,K)
   60                       CONTINUE
                        END IF
   70                CONTINUE
   80            CONTINUE
             END IF
         ELSE
*
*          Form  B := alpha*A**T*B.
*
             IF (UPPER) THEN
                 DO 110 J = 1,N
                     DO 100 I = M,1,-1
                         TEMP = B(I,J)
                         IF (NOUNIT) TEMP = TEMP*A(I,I)
                         DO 90 K = 1,I - 1
                             TEMP = TEMP + A(K,I)*B(K,J)
   90                    CONTINUE
                         B(I,J) = ALPHA*TEMP
  100                CONTINUE
  110            CONTINUE
             ELSE
                 DO 140 J = 1,N
                     DO 130 I = 1,M
                         TEMP = B(I,J)
                         IF (NOUNIT) TEMP = TEMP*A(I,I)
                         DO 120 K = I + 1,M
                             TEMP = TEMP + A(K,I)*B(K,J)
  120                    CONTINUE
                         B(I,J) = ALPHA*TEMP
  130                CONTINUE
  140            CONTINUE
             END IF
         END IF
      ELSE
         IF (LSAME(TRANSA,'N')) THEN
*
*          Form  B := alpha*B*A.
*
             IF (UPPER) THEN
                 DO 180 J = N,1,-1
                     TEMP = ALPHA
                     IF (NOUNIT) TEMP = TEMP*A(J,J)
                     DO 150 I = 1,M
                         B(I,J) = TEMP*B(I,J)
  150                CONTINUE
                     DO 170 K = 1,J - 1
                         IF (A(K,J).NE.ZERO) THEN
```

```
                            TEMP = ALPHA*A(K,J)
                            DO 160 I = 1,M
                                B(I,J) = B(I,J) + TEMP*B(I,K)
 160                        CONTINUE
                        END IF
 170                CONTINUE
 180            CONTINUE
            ELSE
                DO 220 J = 1,N
                    TEMP = ALPHA
                    IF (NOUNIT) TEMP = TEMP*A(J,J)
                    DO 190 I = 1,M
                        B(I,J) = TEMP*B(I,J)
 190                CONTINUE
                    DO 210 K = J + 1,N
                        IF (A(K,J).NE.ZERO) THEN
                            TEMP = ALPHA*A(K,J)
                            DO 200 I = 1,M
                                B(I,J) = B(I,J) + TEMP*B(I,K)
 200                        CONTINUE
                        END IF
 210                CONTINUE
 220            CONTINUE
            END IF
        ELSE
*
*           Form  B := alpha*B*A**T.
*
            IF (UPPER) THEN
                DO 260 K = 1,N
                    DO 240 J = 1,K - 1
                        IF (A(J,K).NE.ZERO) THEN
                            TEMP = ALPHA*A(J,K)
                            DO 230 I = 1,M
                                B(I,J) = B(I,J) + TEMP*B(I,K)
 230                        CONTINUE
                        END IF
 240                CONTINUE
                    TEMP = ALPHA
                    IF (NOUNIT) TEMP = TEMP*A(K,K)
                    IF (TEMP.NE.ONE) THEN
                        DO 250 I = 1,M
                            B(I,K) = TEMP*B(I,K)
 250                    CONTINUE
                    END IF
 260            CONTINUE
            ELSE
                DO 300 K = N,1,-1
                    DO 280 J = K + 1,N
                        IF (A(J,K).NE.ZERO) THEN
```

```
                              TEMP = ALPHA*A(J,K)
                              DO 270 I = 1,M
                                  B(I,J) = B(I,J) + TEMP*B(I,K)
  270                         CONTINUE
                          END IF
  280                 CONTINUE
                      TEMP = ALPHA
                      IF (NOUNIT) TEMP = TEMP*A(K,K)
                      IF (TEMP.NE.ONE) THEN
                          DO 290 I = 1,M
                              B(I,K) = TEMP*B(I,K)
  290                     CONTINUE
                      END IF
  300             CONTINUE
              END IF
          END IF
      END IF
*
      RETURN
*
*     End of STRMM .
*
      END
```

———-

14.5.4 lisp code

— BLAS 3 strmm —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun strmm (side uplo transa diag m n alpha a lda b ldb$)
    (declare (type (array single-float (*)) b a)
             (type (single-float) alpha)
             (type (f2cl-lib:integer4) ldb$ lda n m)
             (type (string *) diag transa uplo side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (transa character transa-%data% transa-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%))
      (prog ((lside nil) (nounit nil) (upper nil) (i 0) (info 0) (j 0) (k 0)
```

```
     (nrowa 0) (temp 0.0))
(declare (type f2cl-lib:logical lside nounit upper)
         (type (f2cl-lib:integer4) i info j k nrowa)
         (type (single-float) temp))
(setf lside (lsame side "L"))
(cond (lside (setf nrowa m)) (t (setf nrowa n)))
(setf nounit (lsame diag "N"))
(setf upper (lsame uplo "U"))
(setf info 0)
(cond ((and (not lside) (not (lsame side "R"))) (setf info 1))
      ((and (not upper) (not (lsame uplo "L"))) (setf info 2))
      ((and (not (lsame transa "N")) (not (lsame transa "T"))
            (not (lsame transa "C")))
       (setf info 3))
      ((and (not (lsame diag "U")) (not (lsame diag "N")))
       (setf info 4))
      ((< m 0) (setf info 5)) ((< n 0) (setf info 6))
      ((< lda
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4 nrowa)))
       (setf info 9))
      ((< ldb$
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info 11)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "STRMM " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= m 0) (= n 0))
    (go end_label))
(cond
 ((= alpha zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                  zero)
         label10))
     label20))
  (go end_label)))
(cond
 (lside
  (cond
```

```
((lsame transa "N")
 (cond
  (upper
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                     ((> k m) nil)
         (tagbody
           (cond
            ((/= (f2cl-lib:fref b (k j) ((1 ldb$) (1 *))) zero)
             (setf temp
                     (* alpha
                        (f2cl-lib:fref b-%data% (k j)
                                       ((1 ldb$) (1 *))
                                       b-%offset%)))
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i
                               (f2cl-lib:int-add k
                                                 (f2cl-lib:int-sub
                                                  1)))
                            nil)
               (tagbody
                 (setf (f2cl-lib:fref b-%data% (i j)
                                      ((1 ldb$) (1 *)) b-%offset%)
                         (+
                          (f2cl-lib:fref b-%data% (i j)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)
                          (* temp
                             (f2cl-lib:fref a-%data% (i k)
                                            ((1 lda) (1 *))
                                            a-%offset%))))
                 label30))
             (if nounit
                 (setf temp
                         (* temp
                            (f2cl-lib:fref a-%data% (k k)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
             (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                  b-%offset%)
                     temp)))
           label40))
       label50)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (f2cl-lib:fdo (k m (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
```

```
                    ((> k 1) nil)
            (tagbody
              (cond
               ((/= (f2cl-lib:fref b (k j) ((1 ldb$) (1 *))) zero)
                (setf temp
                        (* alpha
                           (f2cl-lib:fref b-%data% (k j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)))
                (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                     b-%offset%)
                        temp)
                (if nounit
                    (setf (f2cl-lib:fref b-%data% (k j)
                                         ((1 ldb$) (1 *)) b-%offset%)
                          (*
                           (f2cl-lib:fref b-%data% (k j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)
                           (f2cl-lib:fref a-%data% (k k)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
                (f2cl-lib:fdo (i (f2cl-lib:int-add k 1)
                                 (f2cl-lib:int-add i 1))
                               ((> i m) nil)
                  (tagbody
                    (setf (f2cl-lib:fref b-%data% (i j)
                                         ((1 ldb$) (1 *)) b-%offset%)
                          (+
                           (f2cl-lib:fref b-%data% (i j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)
                           (* temp
                              (f2cl-lib:fref a-%data% (i k)
                                             ((1 lda) (1 *))
                                             a-%offset%))))
                  label60))))
          label70))
      label80)))))
(t
 (cond
  (upper
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     ((> i 1) nil)
         (tagbody
           (setf temp
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
```

```
                                b-%offset%))
            (if nounit
                (setf temp
                        (* temp
                           (f2cl-lib:fref a-%data% (i i)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
            (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                          ((> k
                              (f2cl-lib:int-add i
                                                (f2cl-lib:int-sub
                                                 1)))
                           nil)
              (tagbody
                (setf temp
                        (+ temp
                           (*
                            (f2cl-lib:fref a-%data% (k i)
                                           ((1 lda) (1 *))
                                           a-%offset%)
                            (f2cl-lib:fref b-%data% (k j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%))))
               label90))
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                    (* alpha temp))
          label100))
      label110)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf temp
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%))
           (if nounit
               (setf temp
                       (* temp
                          (f2cl-lib:fref a-%data% (i i)
                                         ((1 lda) (1 *))
                                         a-%offset%))))
           (f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
                            (f2cl-lib:int-add k 1))
                         ((> k m) nil)
             (tagbody
               (setf temp
```

```
                          (+ temp
                             (*
                              (f2cl-lib:fref a-%data% (k i)
                                             ((1 lda) (1 *))
                                             a-%offset%)
                              (f2cl-lib:fref b-%data% (k j)
                                             ((1 ldb$) (1 *))
                                             b-%offset%))))
                   label120))
                (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                     b-%offset%)
                      (* alpha temp))
            label130))
        label140)))))))
(t
 (cond
  ((lsame transa "N")
   (cond
    (upper
     (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   ((> j 1) nil)
       (tagbody
         (setf temp alpha)
         (if nounit
             (setf temp
                     (* temp
                        (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                       a-%offset%))))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)
                     (* temp
                        (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)))
            label150))
         (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                       ((> k
                           (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                        nil)
           (tagbody
             (cond
              ((/= (f2cl-lib:fref a (k j) ((1 lda) (1 *))) zero)
               (setf temp
                       (* alpha
                          (f2cl-lib:fref a-%data% (k j)
                                         ((1 lda) (1 *)) a-%offset%)))
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i m) nil)
```

```
                (tagbody
                  (setf (f2cl-lib:fref b-%data% (i j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                          (+
                           (f2cl-lib:fref b-%data% (i j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)
                           (* temp
                              (f2cl-lib:fref b-%data% (i k)
                                             ((1 ldb$) (1 *))
                                             b-%offset%))))
                 label160))))
          label170))
      label180)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp alpha)
       (if nounit
           (setf temp
                   (* temp
                      (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%))))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                   (* temp
                      (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                     b-%offset%)))
          label190))
       (f2cl-lib:fdo (k (f2cl-lib:int-add j 1)
                        (f2cl-lib:int-add k 1))
                     ((> k n) nil)
         (tagbody
           (cond
            ((/= (f2cl-lib:fref a (k j) ((1 lda) (1 *))) zero)
             (setf temp
                     (* alpha
                        (f2cl-lib:fref a-%data% (k j)
                                       ((1 lda) (1 *)) a-%offset%)))
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i m) nil)
               (tagbody
                 (setf (f2cl-lib:fref b-%data% (i j)
                                      ((1 ldb$) (1 *)) b-%offset%)
                         (+
                          (f2cl-lib:fref b-%data% (i j)
```

```
                                          ((1 ldb$) (1 *))
                                          b-%offset%)
                                 (* temp
                                    (f2cl-lib:fref b-%data% (i k)
                                                   ((1 ldb$) (1 *))
                                                   b-%offset%))))
                        label200))))
                  label210))
            label220)))))
      (t
       (cond
        (upper
         (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                       ((> k n) nil)
           (tagbody
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j
                               (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                            nil)
               (tagbody
                 (cond
                  ((/= (f2cl-lib:fref a (j k) ((1 lda) (1 *))) zero)
                   (setf temp
                           (* alpha
                              (f2cl-lib:fref a-%data% (j k)
                                             ((1 lda) (1 *)) a-%offset%)))
                   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                 ((> i m) nil)
                     (tagbody
                       (setf (f2cl-lib:fref b-%data% (i j)
                                            ((1 ldb$) (1 *)) b-%offset%)
                               (+
                                (f2cl-lib:fref b-%data% (i j)
                                               ((1 ldb$) (1 *))
                                               b-%offset%)
                                (* temp
                                   (f2cl-lib:fref b-%data% (i k)
                                                  ((1 ldb$) (1 *))
                                                  b-%offset%))))
                        label230))))
                  label240))
             (setf temp alpha)
             (if nounit
                 (setf temp
                         (* temp
                            (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                           a-%offset%))))
             (cond
              ((/= temp one)
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref b-%data% (i k) ((1 ldb$) (1 *))
                                   b-%offset%)
                    (* temp
                       (f2cl-lib:fref b-%data% (i k)
                                      ((1 ldb$) (1 *))
                                      b-%offset%)))
             label250))))
    label260)))
(t
 (f2cl-lib:fdo (k n (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
               ((> k 1) nil)
   (tagbody
     (f2cl-lib:fdo (j (f2cl-lib:int-add k 1)
                    (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (cond
          ((/= (f2cl-lib:fref a (j k) ((1 lda) (1 *))) zero)
           (setf temp
                   (* alpha
                      (f2cl-lib:fref a-%data% (j k)
                                     ((1 lda) (1 *)) a-%offset%)))
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf (f2cl-lib:fref b-%data% (i j)
                                    ((1 ldb$) (1 *)) b-%offset%)
                     (+
                      (f2cl-lib:fref b-%data% (i j)
                                     ((1 ldb$) (1 *))
                                     b-%offset%)
                      (* temp
                         (f2cl-lib:fref b-%data% (i k)
                                        ((1 ldb$) (1 *))
                                        b-%offset%))))
              label270))))
        label280))
     (setf temp alpha)
     (if nounit
         (setf temp
                 (* temp
                    (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                   a-%offset%))))
     (cond
      ((/= temp one)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
```

```
                    (setf (f2cl-lib:fref b-%data% (i k) ((1 ldb$) (1 *))
                                         b-%offset%)
                            (* temp
                               (f2cl-lib:fref b-%data% (i k)
                                              ((1 ldb$) (1 *))
                                              b-%offset%)))
                  label290))))
            label300))))))))
     (go end_label)
    end_label
     (return (values nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

14.6 strsm - $op(A)X = \alpha B$ or $Xop(A) = \alpha B$, $op(A) = A$ or $op(A) = A^T$

14.6.1 Axiom unit tests

— **strsm.input** —

```
)set break resume
)sys rm -f strsm.output
)spool strsm.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

14.6.2 Axiom help page

— **strsm.help** —

```
======================================================================
strsm examples
======================================================================

======================================================================
Man Page Details
```

```
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine strsm (SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B,
           LDB)
           STRSM

Function/Subroutine Documentation
   subroutine strsm (character SIDE, character UPLO, character TRANSA,
       character DIAG, integer M, integer N, real ALPHA, real,
       dimension(lda,*) A, integer LDA, real, dimension(ldb,*) B, integer LDB)
       STRSM

Purpose:
   STRSM  solves one of the matrix equations

     op( A )*X = alpha*B,   or   X*op( A ) = alpha*B,

   where alpha is a scalar, X and B are m by n matrices, A is a unit, or
   non-unit,  upper or lower triangular matrix  and  op( A )  is one  of

     op( A ) = A   or   op( A ) = A**T.

   The matrix X is overwritten on B.

Parameters:

 SIDE
   SIDE is CHARACTER*1
   On entry, SIDE specifies whether op( A ) appears on the left
   or right of X as follows:

   SIDE = 'L' or 'l'   op( A )*X = alpha*B.

   SIDE = 'R' or 'r'   X*op( A ) = alpha*B.

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the matrix A is an upper or
   lower triangular matrix as follows:

   UPLO = 'U' or 'u'   A is an upper triangular matrix.

   UPLO = 'L' or 'l'   A is a lower triangular matrix.

 TRANSA
   TRANSA is CHARACTER*1
   On entry, TRANSA specifies the form of op( A ) to be used in
   the matrix multiplication as follows:
```

```
  TRANSA = 'N' or 'n'   op( A ) = A.

  TRANSA = 'T' or 't'   op( A ) = A**T.

  TRANSA = 'C' or 'c'   op( A ) = A**T.

DIAG
  DIAG is CHARACTER*1
  On entry, DIAG specifies whether or not A is unit triangular
  as follows:

  DIAG = 'U' or 'u'   A is assumed to be unit triangular.

  DIAG = 'N' or 'n'   A is not assumed to be unit triangular.

M
  M is INTEGER
  On entry, M specifies the number of rows of B. M must be at
  least zero.

N
  N is INTEGER
  On entry, N specifies the number of columns of B.  N must be
  at least zero.

ALPHA
  ALPHA is REAL
  On entry,  ALPHA specifies the scalar  alpha. When  alpha is
  zero then  A is not referenced and  B need not be set before entry.

A
  A is REAL array of DIMENSION ( LDA, k ),
  where k is m when SIDE = 'L' or 'l'
  and k is n when SIDE = 'R' or 'r'.

  Before entry  with  UPLO = 'U' or 'u',  the  leading  k by k
  upper triangular part of the array  A must contain the upper
  triangular matrix  and the strictly lower triangular part of
  A is not referenced.

  Before entry  with  UPLO = 'L' or 'l',  the  leading  k by k
  lower triangular part of the array  A must contain the lower
  triangular matrix  and the strictly upper triangular part of
  A is not referenced.

  Note that when  DIAG = 'U' or 'u',  the diagonal elements of
  A  are not referenced either,  but are assumed to be  unity.

LDA
```

```
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program.  When  SIDE = 'L' or 'l'  then
  LDA  must be at least  max( 1, m ),  when  SIDE = 'R' or 'r'
  then LDA must be at least max( 1, n ).

B
  B is REAL array of DIMENSION ( LDB, n ).
  Before entry,  the leading  m by n part of the array  B must
  contain  the  right-hand  side  matrix  B,  and  on exit  is
  overwritten by the solution matrix  X.

LDB
  LDB is INTEGER
  On entry, LDB specifies the first dimension of B as declared
  in  the  calling  (sub)  program.  LDB  must  be  at  least
  max( 1, m ).
```

———-

14.6.3 fortran code

— **strsm.f** —

```
*  ===================================================================
      SUBROUTINE STRSM(SIDE,UPLO,TRANSA,DIAG,M,N,ALPHA,A,LDA,B,LDB)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL ALPHA
      INTEGER LDA,LDB,M,N
      CHARACTER DIAG,SIDE,TRANSA,UPLO
*     ..
*     .. Array Arguments ..
      REAL A(LDA,*),B(LDB,*)
*     ..
*
*  ===================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
```

```
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*     .. Local Scalars ..
      REAL TEMP
      INTEGER I,INFO,J,K,NROWA
      LOGICAL LSIDE,NOUNIT,UPPER
*     ..
*     .. Parameters ..
      REAL ONE,ZERO
      PARAMETER (ONE=1.0E+0,ZERO=0.0E+0)
*     ..
*
*     Test the input parameters.
*
      LSIDE = LSAME(SIDE,'L')
      IF (LSIDE) THEN
          NROWA = M
      ELSE
          NROWA = N
      END IF
      NOUNIT = LSAME(DIAG,'N')
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.LSIDE) .AND. (.NOT.LSAME(SIDE,'R'))) THEN
          INFO = 1
      ELSE IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 2
      ELSE IF ((.NOT.LSAME(TRANSA,'N')) .AND.
     +         (.NOT.LSAME(TRANSA,'T')) .AND.
     +         (.NOT.LSAME(TRANSA,'C'))) THEN
          INFO = 3
      ELSE IF ((.NOT.LSAME(DIAG,'U')) .AND. (.NOT.LSAME(DIAG,'N'))) THEN
          INFO = 4
      ELSE IF (M.LT.0) THEN
          INFO = 5
      ELSE IF (N.LT.0) THEN
          INFO = 6
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 9
      ELSE IF (LDB.LT.MAX(1,M)) THEN
          INFO = 11
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('STRSM ',INFO)
          RETURN
```

```
      END IF
*
*     Quick return if possible.
*
      IF (M.EQ.0 .OR. N.EQ.0) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
          DO 20 J = 1,N
              DO 10 I = 1,M
                  B(I,J) = ZERO
   10         CONTINUE
   20     CONTINUE
          RETURN
      END IF
*
*     Start the operations.
*
      IF (LSIDE) THEN
          IF (LSAME(TRANSA,'N')) THEN
*
*           Form  B := alpha*inv( A )*B.
*
              IF (UPPER) THEN
                  DO 60 J = 1,N
                      IF (ALPHA.NE.ONE) THEN
                          DO 30 I = 1,M
                              B(I,J) = ALPHA*B(I,J)
   30                     CONTINUE
                      END IF
                      DO 50 K = M,1,-1
                          IF (B(K,J).NE.ZERO) THEN
                              IF (NOUNIT) B(K,J) = B(K,J)/A(K,K)
                              DO 40 I = 1,K - 1
                                  B(I,J) = B(I,J) - B(K,J)*A(I,K)
   40                         CONTINUE
                          END IF
   50                 CONTINUE
   60             CONTINUE
              ELSE
                  DO 100 J = 1,N
                      IF (ALPHA.NE.ONE) THEN
                          DO 70 I = 1,M
                              B(I,J) = ALPHA*B(I,J)
   70                     CONTINUE
                      END IF
                      DO 90 K = 1,M
                          IF (B(K,J).NE.ZERO) THEN
                              IF (NOUNIT) B(K,J) = B(K,J)/A(K,K)
```

```
                            DO 80 I = K + 1,M
                                B(I,J) = B(I,J) - B(K,J)*A(I,K)
   80                       CONTINUE
                        END IF
   90                CONTINUE
  100            CONTINUE
             END IF
         ELSE
*
*          Form  B := alpha*inv( A**T )*B.
*
             IF (UPPER) THEN
                 DO 130 J = 1,N
                     DO 120 I = 1,M
                         TEMP = ALPHA*B(I,J)
                         DO 110 K = 1,I - 1
                             TEMP = TEMP - A(K,I)*B(K,J)
  110                    CONTINUE
                         IF (NOUNIT) TEMP = TEMP/A(I,I)
                         B(I,J) = TEMP
  120                CONTINUE
  130            CONTINUE
             ELSE
                 DO 160 J = 1,N
                     DO 150 I = M,1,-1
                         TEMP = ALPHA*B(I,J)
                         DO 140 K = I + 1,M
                             TEMP = TEMP - A(K,I)*B(K,J)
  140                    CONTINUE
                         IF (NOUNIT) TEMP = TEMP/A(I,I)
                         B(I,J) = TEMP
  150                CONTINUE
  160            CONTINUE
             END IF
         END IF
     ELSE
         IF (LSAME(TRANSA,'N')) THEN
*
*          Form  B := alpha*B*inv( A ).
*
             IF (UPPER) THEN
                 DO 210 J = 1,N
                     IF (ALPHA.NE.ONE) THEN
                         DO 170 I = 1,M
                             B(I,J) = ALPHA*B(I,J)
  170                    CONTINUE
                     END IF
                     DO 190 K = 1,J - 1
                         IF (A(K,J).NE.ZERO) THEN
                             DO 180 I = 1,M
```

```
                                B(I,J) = B(I,J) - A(K,J)*B(I,K)
 180                        CONTINUE
                        END IF
 190                CONTINUE
                    IF (NOUNIT) THEN
                        TEMP = ONE/A(J,J)
                        DO 200 I = 1,M
                            B(I,J) = TEMP*B(I,J)
 200                    CONTINUE
                    END IF
 210            CONTINUE
            ELSE
                DO 260 J = N,1,-1
                    IF (ALPHA.NE.ONE) THEN
                        DO 220 I = 1,M
                            B(I,J) = ALPHA*B(I,J)
 220                    CONTINUE
                    END IF
                    DO 240 K = J + 1,N
                        IF (A(K,J).NE.ZERO) THEN
                            DO 230 I = 1,M
                                B(I,J) = B(I,J) - A(K,J)*B(I,K)
 230                        CONTINUE
                        END IF
 240                CONTINUE
                    IF (NOUNIT) THEN
                        TEMP = ONE/A(J,J)
                        DO 250 I = 1,M
                            B(I,J) = TEMP*B(I,J)
 250                    CONTINUE
                    END IF
 260            CONTINUE
            END IF
        ELSE
*
*           Form  B := alpha*B*inv( A**T ).
*
            IF (UPPER) THEN
                DO 310 K = N,1,-1
                    IF (NOUNIT) THEN
                        TEMP = ONE/A(K,K)
                        DO 270 I = 1,M
                            B(I,K) = TEMP*B(I,K)
 270                    CONTINUE
                    END IF
                    DO 290 J = 1,K - 1
                        IF (A(J,K).NE.ZERO) THEN
                            TEMP = A(J,K)
                            DO 280 I = 1,M
                                B(I,J) = B(I,J) - TEMP*B(I,K)
```

```
 280                        CONTINUE
                        END IF
 290                CONTINUE
                    IF (ALPHA.NE.ONE) THEN
                        DO 300 I = 1,M
                            B(I,K) = ALPHA*B(I,K)
 300                    CONTINUE
                    END IF
 310            CONTINUE
            ELSE
                DO 360 K = 1,N
                    IF (NOUNIT) THEN
                        TEMP = ONE/A(K,K)
                        DO 320 I = 1,M
                            B(I,K) = TEMP*B(I,K)
 320                    CONTINUE
                    END IF
                    DO 340 J = K + 1,N
                        IF (A(J,K).NE.ZERO) THEN
                            TEMP = A(J,K)
                            DO 330 I = 1,M
                                B(I,J) = B(I,J) - TEMP*B(I,K)
 330                        CONTINUE
                        END IF
 340                CONTINUE
                    IF (ALPHA.NE.ONE) THEN
                        DO 350 I = 1,M
                            B(I,K) = ALPHA*B(I,K)
 350                    CONTINUE
                    END IF
 360            CONTINUE
            END IF
        END IF
    END IF
*
    RETURN
*
*   End of STRSM .
*
    END
```

———-

14.6.4 lisp code

— BLAS 3 strsm —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun strsm (side uplo transa diag m n alpha a lda b ldb$)
    (declare (type (array single-float (*)) b a)
             (type (single-float) alpha)
             (type (f2cl-lib:integer4) ldb$ lda n m)
             (type (string *) diag transa uplo side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (transa character transa-%data% transa-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%))
      (prog ((lside nil) (nounit nil) (upper nil) (i 0) (info 0) (j 0) (k 0)
             (nrowa 0) (temp 0.0))
        (declare (type f2cl-lib:logical lside nounit upper)
                 (type (f2cl-lib:integer4) i info j k nrowa)
                 (type (single-float) temp))
        (setf lside (lsame side "L"))
        (cond (lside (setf nrowa m)) (t (setf nrowa n)))
        (setf nounit (lsame diag "N"))
        (setf upper (lsame uplo "U"))
        (setf info 0)
        (cond ((and (not lside) (not (lsame side "R"))) (setf info 1))
              ((and (not upper) (not (lsame uplo "L"))) (setf info 2))
              ((and (not (lsame transa "N")) (not (lsame transa "T"))
                    (not (lsame transa "C")))
               (setf info 3))
              ((and (not (lsame diag "U")) (not (lsame diag "N")))
               (setf info 4))
              ((< m 0) (setf info 5)) ((< n 0) (setf info 6))
              ((< lda
                  (max (the f2cl-lib:integer4 1)
                       (the f2cl-lib:integer4 nrowa)))
               (setf info 9))
              ((< ldb$
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info 11)))
        (cond
         ((/= info 0)
          (multiple-value-bind (var-0 var-1)
              (xerbla "STRSM " info)
            (declare (ignore var-0))
            (when var-1 (setf info var-1)))
          (go end_label)))
        (if (or (= m 0) (= n 0))
            (go end_label))
```

```
(cond
 ((= alpha zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                  zero)
         label10))
     label20))
  (go end_label)))
(cond
 (lside
  (cond
   ((lsame transa "N")
    (cond
     (upper
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
           ((/= alpha one)
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                     b-%offset%)
                        (* alpha
                           (f2cl-lib:fref b-%data% (i j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)))
               label30))))
          (f2cl-lib:fdo (k m (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                        ((> k 1) nil)
            (tagbody
              (cond
               ((/= (f2cl-lib:fref b (k j) ((1 ldb$) (1 *))) zero)
                (if nounit
                    (setf (f2cl-lib:fref b-%data% (k j)
                                         ((1 ldb$) (1 *)) b-%offset%)
                            (/
                             (f2cl-lib:fref b-%data% (k j)
                                            ((1 ldb$) (1 *))
                                            b-%offset%)
                             (f2cl-lib:fref a-%data% (k k)
                                            ((1 lda) (1 *))
                                            a-%offset%))))
```

```
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i
                                  (f2cl-lib:int-add k
                                                    (f2cl-lib:int-sub
                                                     1)))
                               nil)
                  (tagbody
                    (setf (f2cl-lib:fref b-%data% (i j)
                                         ((1 ldb$) (1 *)) b-%offset%)
                          (-
                           (f2cl-lib:fref b-%data% (i j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)
                           (*
                            (f2cl-lib:fref b-%data% (k j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%)
                            (f2cl-lib:fref a-%data% (i k)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
                   label40))))
          label50))
     label60)))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
      ((/= alpha one)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                 (* alpha
                    (f2cl-lib:fref b-%data% (i j)
                                   ((1 ldb$) (1 *))
                                   b-%offset%)))
          label70))))
     (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                   ((> k m) nil)
       (tagbody
         (cond
          ((/= (f2cl-lib:fref b (k j) ((1 ldb$) (1 *))) zero)
           (if nounit
               (setf (f2cl-lib:fref b-%data% (k j)
                                    ((1 ldb$) (1 *)) b-%offset%)
                     (/
                      (f2cl-lib:fref b-%data% (k j)
                                     ((1 ldb$) (1 *))
```

```
                                          b-%offset%)
                              (f2cl-lib:fref a-%data% (k k)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
                 (f2cl-lib:fdo (i (f2cl-lib:int-add k 1)
                                (f2cl-lib:int-add i 1))
                               ((> i m) nil)
                   (tagbody
                     (setf (f2cl-lib:fref b-%data% (i j)
                                          ((1 ldb$) (1 *)) b-%offset%)
                           (-
                            (f2cl-lib:fref b-%data% (i j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)
                            (*
                             (f2cl-lib:fref b-%data% (k j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%)
                             (f2cl-lib:fref a-%data% (i k)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
                    label80))))
             label90))
         label100)))))
(t
 (cond
  (upper
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf temp
                   (* alpha
                      (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                     b-%offset%)))
           (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                         ((> k
                             (f2cl-lib:int-add i
                                               (f2cl-lib:int-sub
                                                1)))
                          nil)
             (tagbody
               (setf temp
                       (- temp
                          (*
                           (f2cl-lib:fref a-%data% (k i)
                                          ((1 lda) (1 *))
                                          a-%offset%)
```

```
                                (f2cl-lib:fref b-%data% (k j)
                                               ((1 ldb$) (1 *))
                                               b-%offset%))))
                 label110))
              (if nounit
                  (setf temp
                          (/ temp
                             (f2cl-lib:fref a-%data% (i i)
                                            ((1 lda) (1 *))
                                            a-%offset%))))
              (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)
                      temp)
            label120))
        label130)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                       ((> i 1) nil)
           (tagbody
             (setf temp
                     (* alpha
                        (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)))
             (f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
                              (f2cl-lib:int-add k 1))
                            ((> k m) nil)
               (tagbody
                 (setf temp
                         (- temp
                            (*
                             (f2cl-lib:fref a-%data% (k i)
                                            ((1 lda) (1 *))
                                            a-%offset%)
                             (f2cl-lib:fref b-%data% (k j)
                                            ((1 ldb$) (1 *))
                                            b-%offset%))))
                 label140))
              (if nounit
                  (setf temp
                          (/ temp
                             (f2cl-lib:fref a-%data% (i i)
                                            ((1 lda) (1 *))
                                            a-%offset%))))
              (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)
                      temp)
            label150))
```

```
        label160)))))))
(t
 (cond
  ((lsame transa "N")
   (cond
    (upper
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (cond
          ((/= alpha one)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                     b-%offset%)
                       (* alpha
                          (f2cl-lib:fref b-%data% (i j)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)))
              label170))))
         (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                       ((> k
                           (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                        nil)
           (tagbody
             (cond
              ((/= (f2cl-lib:fref a (k j) ((1 lda) (1 *))) zero)
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i m) nil)
                 (tagbody
                   (setf (f2cl-lib:fref b-%data% (i j)
                                         ((1 ldb$) (1 *)) b-%offset%)
                           (-
                            (f2cl-lib:fref b-%data% (i j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%)
                            (*
                             (f2cl-lib:fref a-%data% (k j)
                                            ((1 lda) (1 *))
                                            a-%offset%)
                             (f2cl-lib:fref b-%data% (i k)
                                            ((1 ldb$) (1 *))
                                            b-%offset%))))
                  label180))))
             label190))
         (cond
          (nounit
           (setf temp
                   (/ one
```

```
                    (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                   a-%offset%)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (* temp
                     (f2cl-lib:fref b-%data% (i j)
                                    ((1 ldb$) (1 *))
                                    b-%offset%)))
           label200))))
    label210)))
(t
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (cond
      ((/= alpha one)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                 (* alpha
                    (f2cl-lib:fref b-%data% (i j)
                                   ((1 ldb$) (1 *))
                                   b-%offset%)))
           label220))))
     (f2cl-lib:fdo (k (f2cl-lib:int-add j 1)
                    (f2cl-lib:int-add k 1))
                   ((> k n) nil)
       (tagbody
         (cond
          ((/= (f2cl-lib:fref a (k j) ((1 lda) (1 *))) zero)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf (f2cl-lib:fref b-%data% (i j)
                                    ((1 ldb$) (1 *)) b-%offset%)
                     (-
                      (f2cl-lib:fref b-%data% (i j)
                                     ((1 ldb$) (1 *))
                                     b-%offset%)
                      (*
                       (f2cl-lib:fref a-%data% (k j)
                                      ((1 lda) (1 *))
                                      a-%offset%)
                       (f2cl-lib:fref b-%data% (i k)
                                      ((1 ldb$) (1 *))
```

```
                                                b-%offset%))))
                      label230))))
                 label240))
            (cond
             (nounit
              (setf temp
                      (/ one
                         (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                        a-%offset%)))
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i m) nil)
                (tagbody
                  (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)
                          (* temp
                             (f2cl-lib:fref b-%data% (i j)
                                            ((1 ldb$) (1 *))
                                            b-%offset%)))
                 label250))))
           label260)))))
(t
 (cond
  (upper
   (f2cl-lib:fdo (k n (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                 ((> k 1) nil)
     (tagbody
       (cond
        (nounit
         (setf temp
                 (/ one
                    (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                   a-%offset%)))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf (f2cl-lib:fref b-%data% (i k) ((1 ldb$) (1 *))
                                  b-%offset%)
                     (* temp
                        (f2cl-lib:fref b-%data% (i k)
                                       ((1 ldb$) (1 *))
                                       b-%offset%)))
            label270))))
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j
                         (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                      nil)
         (tagbody
           (cond
            ((/= (f2cl-lib:fref a (j k) ((1 lda) (1 *))) zero)
             (setf temp
```

```
                    (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                   a-%offset%))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf (f2cl-lib:fref b-%data% (i j)
                                     ((1 ldb$) (1 *)) b-%offset%)
                      (-
                       (f2cl-lib:fref b-%data% (i j)
                                      ((1 ldb$) (1 *))
                                      b-%offset%)
                       (* temp
                          (f2cl-lib:fref b-%data% (i k)
                                         ((1 ldb$) (1 *))
                                         b-%offset%))))
               label280))))
         label290))
      (cond
       ((/= alpha one)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i k) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (* alpha
                     (f2cl-lib:fref b-%data% (i k)
                                    ((1 ldb$) (1 *))
                                    b-%offset%)))
           label300))))
    label310)))
(t
 (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
               ((> k n) nil)
   (tagbody
     (cond
      (nounit
       (setf temp
               (/ one
                  (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                 a-%offset%)))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf (f2cl-lib:fref b-%data% (i k) ((1 ldb$) (1 *))
                                b-%offset%)
                 (* temp
                    (f2cl-lib:fref b-%data% (i k)
                                   ((1 ldb$) (1 *))
                                   b-%offset%)))
           label320))))
```

```
            (f2cl-lib:fdo (j (f2cl-lib:int-add k 1)
                           (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (cond
                 ((/= (f2cl-lib:fref a (j k) ((1 lda) (1 *))) zero)
                  (setf temp
                          (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                         a-%offset%))
                  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                ((> i m) nil)
                    (tagbody
                      (setf (f2cl-lib:fref b-%data% (i j)
                                           ((1 ldb$) (1 *)) b-%offset%)
                            (-
                             (f2cl-lib:fref b-%data% (i j)
                                            ((1 ldb$) (1 *))
                                            b-%offset%)
                             (* temp
                                (f2cl-lib:fref b-%data% (i k)
                                               ((1 ldb$) (1 *))
                                               b-%offset%))))
                      label330))))
                label340))
            (cond
             ((/= alpha one)
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i m) nil)
                (tagbody
                  (setf (f2cl-lib:fref b-%data% (i k) ((1 ldb$) (1 *))
                                       b-%offset%)
                          (* alpha
                             (f2cl-lib:fref b-%data% (i k)
                                            ((1 ldb$) (1 *))
                                            b-%offset%)))
                  label350))))
          label360))))))))
 (go end_label)
end_label
 (return (values nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

Chapter 15

BLAS Level 3 Double Matrix-Matrix Operations

15.1 dgemm - $\alpha op(A)op(B)+\beta C$, $op(X) = X$ or $op(X) = X^T$

15.1.1 Axiom unit tests

— **dgemm.input** —

```
)set break resume
)sys rm -f dgemm.output
)spool dgemm.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

15.1.2 Axiom help page

— **dgemm.help** —

```
==========================================================================
dgemm examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

 NAME
      DGEMM - perform one of the matrix-matrix operations   C :=
      alpha*op( A )*op( B ) + beta*C,

 SYNOPSIS
      SUBROUTINE DGEMM ( TRANSA, TRANSB, M, N, K, ALPHA, A, LDA,
                       B, LDB, BETA, C, LDC )

          CHARACTER*1  TRANSA, TRANSB

          INTEGER      M, N, K, LDA, LDB, LDC

          DOUBLE       PRECISION ALPHA, BETA

          DOUBLE       PRECISION A( LDA, * ), B( LDB, * ), C( LDC,
                       * )

 PURPOSE
      DGEMM  performs one of the matrix-matrix operations

      where  op( X ) is one of

         op( X ) = X   or   op( X ) = X',

      alpha and beta are scalars, and A, B and C are matrices,
      with op( A ) an m by k matrix,  op( B )  a  k by n matrix
      and  C an m by n matrix.

 PARAMETERS
      TRANSA - CHARACTER*1.  On entry, TRANSA specifies the form
      of op( A ) to be used in the matrix multiplication as fol-
      lows:

      TRANSA = 'N' or 'n',  op( A ) = A.

      TRANSA = 'T' or 't',  op( A ) = A'.

      TRANSA = 'C' or 'c',  op( A ) = A'.

      Unchanged on exit.

      TRANSB - CHARACTER*1.  On entry, TRANSB specifies the form
      of op( B ) to be used in the matrix multiplication as fol-
      lows:
```

```
     TRANSB = 'N' or 'n',  op( B ) = B.

     TRANSB = 'T' or 't',  op( B ) = B'.

     TRANSB = 'C' or 'c',  op( B ) = B'.

     Unchanged on exit.

     M      - INTEGER.
              On entry,  M  specifies  the number  of rows  of the
              matrix op( A )  and of the  matrix  C.  M  must  be
              at least  zero.  Unchanged on exit.

     N      - INTEGER.
              On entry,  N  specifies the number  of columns of the
              matrix op( B ) and the number of columns of the
              matrix C. N must be at least zero.  Unchanged on
              exit.

     K      - INTEGER.
              On entry,  K  specifies  the number of columns of the
              matrix op( A ) and the number of rows of the matrix
              op( B ). K must be at least  zero.  Unchanged on
              exit.

     ALPHA  - DOUBLE PRECISION.
              On entry, ALPHA specifies the scalar alpha.
              Unchanged on exit.

ka is
     A      -
               DOUBLE PRECISION array of DIMENSION ( LDA, ka ), where
              k  when  TRANSA = 'N' or 'n',  and is  m  otherwise.
              Before entry with  TRANSA = 'N' or 'n',  the leading
              m by k part of the array  A  must contain the matrix
              A,  otherwise the leading  k by m  part of the array
              A  must contain  the matrix A.  Unchanged on exit.

     LDA    - INTEGER.
              On entry, LDA specifies the first dimension of A as
              declared in the calling (sub) program. When  TRANSA =
              'N' or 'n' then LDA must be at least  max( 1, m ),
              otherwise  LDA must be at least  max( 1, k ).
              Unchanged on exit.

kb is
     B      -
               DOUBLE PRECISION array of DIMENSION ( LDB, kb ), where
              n  when  TRANSB = 'N' or 'n',  and is  k  otherwise.
              Before entry with  TRANSB = 'N' or 'n',  the leading
```

```
            k by n part of the array  B  must contain the matrix
            B,  otherwise the leading  n by k  part of the array
            B  must contain  the matrix B.  Unchanged on exit.

      LDB    - INTEGER.
             On entry, LDB specifies the first dimension of B as
             declared in the calling (sub) program. When  TRANSB =
             'N' or 'n' then LDB must be at least  max( 1, k ),

             otherwise  LDB must be at least  max( 1, n ).
             Unchanged on exit.

      BETA   - DOUBLE PRECISION.
             On entry,  BETA  specifies the scalar  beta.  When
             BETA  is supplied as zero then C need not be set on
             input.  Unchanged on exit.

      C      - DOUBLE PRECISION array of DIMENSION ( LDC, n ).
             Before entry, the leading  m by n  part of the array
             C must contain the matrix  C,  except when  beta  is
             zero, in which case C need not be set on entry.  On
             exit, the array  C  is overwritten by the  m by n
             matrix ( alpha*op( A )*op( B ) + beta*C ).

      LDC    - INTEGER.
             On entry, LDC specifies the first dimension of C as
             declared in  the  calling  (sub)  program.   LDC
             must  be  at  least max( 1, m ).  Unchanged on exit.


            ———-
```

15.1.3 fortran code

— dgemm.f —

```
*  ==================================================================
      SUBROUTINE DGEMM(TRANSA,TRANSB,M,N,K,ALPHA,A,LDA,B,LDB,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION ALPHA,BETA
      INTEGER K,LDA,LDB,LDC,M,N
      CHARACTER TRANSA,TRANSB
```

```
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A(LDA,*),B(LDB,*),C(LDC,*)
*     ..
*
*  =====================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP
      INTEGER I,INFO,J,L,NCOLA,NROWA,NROWB
      LOGICAL NOTA,NOTB
*     ..
*     .. Parameters ..
      DOUBLE PRECISION ONE,ZERO
      PARAMETER (ONE=1.0D+0,ZERO=0.0D+0)
*     ..
*
*     Set  NOTA  and  NOTB  as  true if  A  and  B  respectively are not
*     transposed and set  NROWA, NCOLA and  NROWB  as the number of rows
*     and  columns of  A  and the  number of  rows  of  B  respectively.
*
      NOTA = LSAME(TRANSA,'N')
      NOTB = LSAME(TRANSB,'N')
      IF (NOTA) THEN
          NROWA = M
          NCOLA = K
      ELSE
          NROWA = K
          NCOLA = M
      END IF
      IF (NOTB) THEN
          NROWB = K
      ELSE
          NROWB = N
      END IF
*
*     Test the input parameters.
*
      INFO = 0
      IF ((.NOT.NOTA) .AND. (.NOT.LSAME(TRANSA,'C')) .AND.
```

```
     +    (.NOT.LSAME(TRANSA,'T'))) THEN
          INFO = 1
      ELSE IF ((.NOT.NOTB) .AND. (.NOT.LSAME(TRANSB,'C')) .AND.
     +         (.NOT.LSAME(TRANSB,'T'))) THEN
          INFO = 2
      ELSE IF (M.LT.0) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (K.LT.0) THEN
          INFO = 5
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 8
      ELSE IF (LDB.LT.MAX(1,NROWB)) THEN
          INFO = 10
      ELSE IF (LDC.LT.MAX(1,M)) THEN
          INFO = 13
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('DGEMM ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR.
     +    (((ALPHA.EQ.ZERO).OR. (K.EQ.0)).AND. (BETA.EQ.ONE))) RETURN
*
*     And if  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
          IF (BETA.EQ.ZERO) THEN
              DO 20 J = 1,N
                  DO 10 I = 1,M
                      C(I,J) = ZERO
   10             CONTINUE
   20         CONTINUE
          ELSE
              DO 40 J = 1,N
                  DO 30 I = 1,M
                      C(I,J) = BETA*C(I,J)
   30             CONTINUE
   40         CONTINUE
          END IF
          RETURN
      END IF
*
*     Start the operations.
*
      IF (NOTB) THEN
```

```
         IF (NOTA) THEN
*
*          Form  C := alpha*A*B + beta*C.
*
             DO 90 J = 1,N
                 IF (BETA.EQ.ZERO) THEN
                     DO 50 I = 1,M
                         C(I,J) = ZERO
   50                CONTINUE
                 ELSE IF (BETA.NE.ONE) THEN
                     DO 60 I = 1,M
                         C(I,J) = BETA*C(I,J)
   60                CONTINUE
                 END IF
                 DO 80 L = 1,K
                     TEMP = ALPHA*B(L,J)
                     DO 70 I = 1,M
                         C(I,J) = C(I,J) + TEMP*A(I,L)
   70                CONTINUE
   80            CONTINUE
   90        CONTINUE
         ELSE
*
*          Form  C := alpha*A**T*B + beta*C
*
             DO 120 J = 1,N
                 DO 110 I = 1,M
                     TEMP = ZERO
                     DO 100 L = 1,K
                         TEMP = TEMP + A(L,I)*B(L,J)
  100                CONTINUE
                     IF (BETA.EQ.ZERO) THEN
                         C(I,J) = ALPHA*TEMP
                     ELSE
                         C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                     END IF
  110            CONTINUE
  120        CONTINUE
         END IF
      ELSE
         IF (NOTA) THEN
*
*          Form  C := alpha*A*B**T + beta*C
*
             DO 170 J = 1,N
                 IF (BETA.EQ.ZERO) THEN
                     DO 130 I = 1,M
                         C(I,J) = ZERO
  130                CONTINUE
                 ELSE IF (BETA.NE.ONE) THEN
```

```
                    DO 140 I = 1,M
                        C(I,J) = BETA*C(I,J)
  140               CONTINUE
                END IF
                DO 160 L = 1,K
                    TEMP = ALPHA*B(J,L)
                    DO 150 I = 1,M
                        C(I,J) = C(I,J) + TEMP*A(I,L)
  150               CONTINUE
  160           CONTINUE
  170       CONTINUE
        ELSE
*
*         Form  C := alpha*A**T*B**T + beta*C
*
            DO 200 J = 1,N
                DO 190 I = 1,M
                    TEMP = ZERO
                    DO 180 L = 1,K
                        TEMP = TEMP + A(L,I)*B(J,L)
  180               CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP
                    ELSE
                        C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                    END IF
  190           CONTINUE
  200       CONTINUE
        END IF
    END IF
*
    RETURN
*
*   End of DGEMM .
*
    END
```

———-

15.1.4 lisp code

— BLAS 3 dgemm —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (double-float 1.0 1.0) one)
           (type (double-float 0.0 0.0) zero))
  (defun dgemm (transa transb m n k alpha a lda b ldb$ beta c ldc)
```

```
(declare (type (simple-array double-float (*)) c b a)
         (type (double-float) beta alpha)
         (type fixnum ldc ldb$ lda k n m)
         (type character transb transa))
(f2cl-lib:with-multi-array-data
    ((transa character transa-%data% transa-%offset%)
     (transb character transb-%data% transb-%offset%)
     (a double-float a-%data% a-%offset%)
     (b double-float b-%data% b-%offset%)
     (c double-float c-%data% c-%offset%))
  (prog ((temp 0.0) (i 0) (info 0) (j 0) (l 0) (ncola 0) (nrowa 0)
         (nrowb 0) (nota nil) (notb nil))
    (declare (type (double-float) temp)
             (type fixnum i info j l ncola nrowa nrowb)
             (type (member t nil) nota notb))
    (setf nota (char-equal transa #\N))
    (setf notb (char-equal transb #\N))
    (cond
      (nota
       (setf nrowa m)
       (setf ncola k))
      (t
       (setf nrowa k)
       (setf ncola m)))
    (cond
      (notb
       (setf nrowb k))
      (t
       (setf nrowb n)))
    (setf info 0)
    (cond
      ((and (not nota) (not (char-equal transa #\C))
            (not (char-equal transa #\T)))
       (setf info 1))
      ((and (not notb) (not (char-equal transb #\C))
            (not (char-equal transb #\T)))
       (setf info 2))
      ((< m 0)
       (setf info 3))
      ((< n 0)
       (setf info 4))
      ((< k 0)
       (setf info 5))
      ((< lda (max (the fixnum 1) (the fixnum nrowa)))
       (setf info 8))
      ((< ldb$
          (max (the fixnum 1) (the fixnum nrowb)))
       (setf info 10))
      ((< ldc (max (the fixnum 1) (the fixnum m)))
       (setf info 13)))
```

```
(cond
  ((/= info 0)
   (error
    " ** On entry to ~a parameter number ~a had an illegal value~%"
    "DGEMM" info)
   (go end_label)))
(if (or (= m 0) (= n 0) (and (or (= alpha zero) (= k 0)) (= beta one)))
    (go end_label))
(cond
  ((= alpha zero)
   (cond
     ((= beta zero)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data%
                                   (i j)
                                   ((1 ldc) (1 *))
                                   c-%offset%)
                    zero))))))
     (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data%
                                   (i j)
                                   ((1 ldc) (1 *))
                                   c-%offset%)
                    (* beta
                       (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)))))))))
   (go end_label)))
(cond
  (notb
   (cond
     (nota
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
            ((= beta zero)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                    ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data%
                                   (i j)
                                   ((1 ldc) (1 *))
                                   c-%offset%)
                    zero))))
        ((/= beta one)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf (f2cl-lib:fref c-%data%
                                  (i j)
                                  ((1 ldc) (1 *))
                                  c-%offset%)
                   (* beta
                      (f2cl-lib:fref c-%data%
                                     (i j)
                                     ((1 ldc) (1 *))
                                     c-%offset%)))))))
    (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                  ((> l k) nil)
      (tagbody
        (cond
          ((/= (f2cl-lib:fref b (l j) ((1 ldb$) (1 *))) zero)
           (setf temp
                   (* alpha
                      (f2cl-lib:fref b-%data%
                                     (l j)
                                     ((1 ldb$) (1 *))
                                     b-%offset%)))
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                     (+
                      (f2cl-lib:fref c-%data%
                                     (i j)
                                     ((1 ldc) (1 *))
                                     c-%offset%)
                      (* temp
                         (f2cl-lib:fref a-%data%
                                        (i l)
                                        ((1 lda) (1 *))
                                        a-%offset%))))))))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf temp zero)
            (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                          ((> l k) nil)
              (tagbody
                (setf temp
                        (+ temp
                           (*
                            (f2cl-lib:fref a-%data%
                                           (l i)
                                           ((1 lda) (1 *))
                                           a-%offset%)
                            (f2cl-lib:fref b-%data%
                                           (l j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%))))))
            (cond
              ((= beta zero)
               (setf (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                     (* alpha temp)))
              (t
               (setf (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                     (+ (* alpha temp)
                        (* beta
                           (f2cl-lib:fref c-%data%
                                          (i j)
                                          ((1 ldc) (1 *))
                                          c-%offset%))))))))))
(t
 (cond
   (nota
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
          ((= beta zero)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf (f2cl-lib:fref c-%data%
```

```
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                        zero))))
          ((/= beta one)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                       (* beta
                          (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)))))))
      (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                    ((> l k) nil)
        (tagbody
          (cond
            ((/= (f2cl-lib:fref b (j l) ((1 ldb$) (1 *))) zero)
             (setf temp
                     (* alpha
                        (f2cl-lib:fref b-%data%
                                       (j l)
                                       ((1 ldb$) (1 *))
                                       b-%offset%)))
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i m) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                         (+
                          (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)
                          (* temp
                             (f2cl-lib:fref a-%data%
                                            (i l)
                                            ((1 lda) (1 *))
                                            a-%offset%))))))))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                    ((> i m) nil)
            (tagbody
              (setf temp zero)
              (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                            ((> l k) nil)
                (tagbody
                  (setf temp
                          (+ temp
                             (*
                              (f2cl-lib:fref a-%data%
                                             (l i)
                                             ((1 lda) (1 *))
                                             a-%offset%)
                              (f2cl-lib:fref b-%data%
                                             (j l)
                                             ((1 ldb$) (1 *))
                                             b-%offset%))))))
              (cond
                ((= beta zero)
                 (setf (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                        (* alpha temp)))
                (t
                 (setf (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                        (+ (* alpha temp)
                           (* beta
                              (f2cl-lib:fref c-%data%
                                             (i j)
                                             ((1 ldc) (1 *))
                                             c-%offset%))))))))))))))
end_label
       (return
        (values nil nil nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

15.2 dsymm - $\alpha AB + \beta C$ or $\alpha BA + \beta C$

15.2.1 Axiom unit tests

— **dsymm.input** —

```
)set break resume
)sys rm -f dsymm.output
)spool dsymm.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

15.2.2 Axiom help page

— dsymm.help —

```
======================================================================
dsymm examples
======================================================================

======================================================================
Man Page Details
======================================================================

 NAME
      DSYMM - perform one of the matrix-matrix operations   C :=
      alpha*A*B + beta*C,

 SYNOPSIS
      SUBROUTINE DSYMM ( SIDE, UPLO, M, N, ALPHA, A, LDA, B, LDB,
                         BETA, C, LDC )

          CHARACTER*1  SIDE, UPLO

          INTEGER      M, N, LDA, LDB, LDC

          DOUBLE       PRECISION ALPHA, BETA

          DOUBLE       PRECISION A( LDA, * ), B( LDB, * ), C( LDC,
                       * )

 PURPOSE
      DSYMM  performs one of the matrix-matrix operations

      or

         C := alpha*B*A + beta*C,
```

```
     where alpha and beta are scalars,  A is a symmetric matrix
     and  B and C are  m by n matrices.

PARAMETERS
     SIDE   - CHARACTER*1.
            On entry,  SIDE  specifies whether  the  symmetric
            matrix  A appears on the  left or right  in the
            operation as follows:

            SIDE = 'L' or 'l'   C := alpha*A*B + beta*C,

            SIDE = 'R' or 'r'   C := alpha*B*A + beta*C,

            Unchanged on exit.

     UPLO   - CHARACTER*1.
            On  entry,   UPLO  specifies  whether  the  upper  or
            lower triangular  part  of  the  symmetric  matrix
            A  is  to  be referenced as follows:

            UPLO = 'U' or 'u'   Only the upper triangular part of
            the symmetric matrix is to be referenced.

            UPLO = 'L' or 'l'   Only the lower triangular part of
            the symmetric matrix is to be referenced.

            Unchanged on exit.

     M      - INTEGER.
            On entry,  M  specifies the number of rows of the
            matrix  C.  M  must be at least zero.  Unchanged on
            exit.

     N      - INTEGER.
            On entry, N specifies the number of columns of the
            matrix C.  N  must be at least zero.  Unchanged on
            exit.

     ALPHA  - DOUBLE PRECISION.
            On entry, ALPHA specifies the scalar alpha.
            Unchanged on exit.

ka is
     A      -
             DOUBLE PRECISION array of DIMENSION ( LDA, ka ), where
            m  when  SIDE = 'L' or 'l'  and is  n otherwise.
            Before entry  with  SIDE = 'L' or 'l',  the  m by m
            part of the array  A  must contain the  symmetric
            matrix,  such that when  UPLO = 'U' or 'u', the lead-
            ing m by m upper triangular part of the array  A
```

```
        must contain the upper triangular part of the  sym-
        metric matrix and the  strictly  lower triangular
        part of  A  is not referenced,  and when  UPLO = 'L'
        or 'l', the leading  m by m  lower triangular part
        of the  array  A must  contain  the  lower triangular
        part  of the  symmetric matrix and the  strictly
        upper triangular part of  A  is not referenced.
        Before entry  with  SIDE = 'R' or 'r',  the  n by n
        part of the array  A  must contain the  symmetric
        matrix,  such that when  UPLO = 'U' or 'u', the lead-
        ing n by n upper triangular part of the array  A
        must contain the upper triangular part of the  sym-
        metric matrix and the  strictly  lower triangular
        part of  A  is not referenced,  and when  UPLO = 'L'
        or 'l', the leading  n by n  lower triangular part
        of the  array  A must  contain  the  lower triangular
        part  of the  symmetric matrix and the  strictly
        upper triangular part of  A  is not referenced.
        Unchanged on exit.

LDA     - INTEGER.
        On entry, LDA specifies the first dimension of A as
        declared in the calling (sub) program.  When  SIDE =
        'L' or 'l'  then LDA must be at least  max( 1, m ),
        otherwise  LDA must be at least  max( 1, n ).
        Unchanged on exit.

B       - DOUBLE PRECISION array of DIMENSION ( LDB, n ).
        Before entry, the leading  m by n part of the array
        B  must contain the matrix B.  Unchanged on exit.

LDB     - INTEGER.
        On entry, LDB specifies the first dimension of B as
        declared in  the  calling  (sub)  program.   LDB
        must  be  at  least max( 1, m ).  Unchanged on exit.

BETA    - DOUBLE PRECISION.
        On entry,  BETA  specifies the scalar  beta.  When
        BETA  is supplied as zero then C need not be set on
        input.  Unchanged on exit.

C       - DOUBLE PRECISION array of DIMENSION ( LDC, n ).
        Before entry, the leading  m by n  part of the array
        C must contain the matrix  C,  except when  beta  is
        zero, in which case C need not be set on entry.  On
        exit, the array  C  is overwritten by the  m by n
        updated matrix.

LDC     - INTEGER.
        On entry, LDC specifies the first dimension of C as
```

```
declared in  the  calling  (sub)  program.   LDC
must  be  at  least max( 1, m ).  Unchanged on exit.

Level 3 Blas routine.
```

———-

15.2.3 fortran code

— dsymm.f —

```
*  ===================================================================
      SUBROUTINE DSYMM(SIDE,UPLO,M,N,ALPHA,A,LDA,B,LDB,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION ALPHA,BETA
      INTEGER LDA,LDB,LDC,M,N
      CHARACTER SIDE,UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A(LDA,*),B(LDB,*),C(LDC,*)
*     ..
*
*  ====================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP1,TEMP2
      INTEGER I,INFO,J,K,NROWA
      LOGICAL UPPER
*     ..
*     .. Parameters ..
      DOUBLE PRECISION ONE,ZERO
```

```
      PARAMETER (ONE=1.0D+0,ZERO=0.0D+0)
*     ..
*
*     Set NROWA as the number of rows of A.
*
      IF (LSAME(SIDE,'L')) THEN
          NROWA = M
      ELSE
          NROWA = N
      END IF
      UPPER = LSAME(UPLO,'U')
*
*     Test the input parameters.
*
      INFO = 0
      IF ((.NOT.LSAME(SIDE,'L')) .AND. (.NOT.LSAME(SIDE,'R'))) THEN
          INFO = 1
      ELSE IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 2
      ELSE IF (M.LT.0) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 7
      ELSE IF (LDB.LT.MAX(1,M)) THEN
          INFO = 9
      ELSE IF (LDC.LT.MAX(1,M)) THEN
          INFO = 12
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('DSYMM ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR.
     +    ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
          IF (BETA.EQ.ZERO) THEN
              DO 20 J = 1,N
                  DO 10 I = 1,M
                      C(I,J) = ZERO
   10             CONTINUE
   20         CONTINUE
          ELSE
```

```
            DO 40 J = 1,N
                DO 30 I = 1,M
                    C(I,J) = BETA*C(I,J)
   30           CONTINUE
   40       CONTINUE
        END IF
        RETURN
    END IF
*
*   Start the operations.
*
    IF (LSAME(SIDE,'L')) THEN
*
*      Form  C := alpha*A*B + beta*C.
*
        IF (UPPER) THEN
            DO 70 J = 1,N
                DO 60 I = 1,M
                    TEMP1 = ALPHA*B(I,J)
                    TEMP2 = ZERO
                    DO 50 K = 1,I - 1
                        C(K,J) = C(K,J) + TEMP1*A(K,I)
                        TEMP2 = TEMP2 + B(K,J)*A(K,I)
   50               CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = TEMP1*A(I,I) + ALPHA*TEMP2
                    ELSE
                        C(I,J) = BETA*C(I,J) + TEMP1*A(I,I) +
     +                           ALPHA*TEMP2
                    END IF
   60           CONTINUE
   70       CONTINUE
        ELSE
            DO 100 J = 1,N
                DO 90 I = M,1,-1
                    TEMP1 = ALPHA*B(I,J)
                    TEMP2 = ZERO
                    DO 80 K = I + 1,M
                        C(K,J) = C(K,J) + TEMP1*A(K,I)
                        TEMP2 = TEMP2 + B(K,J)*A(K,I)
   80               CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = TEMP1*A(I,I) + ALPHA*TEMP2
                    ELSE
                        C(I,J) = BETA*C(I,J) + TEMP1*A(I,I) +
     +                           ALPHA*TEMP2
                    END IF
   90           CONTINUE
  100       CONTINUE
        END IF
```

```
      ELSE
*
*        Form  C := alpha*B*A + beta*C.
*
          DO 170 J = 1,N
              TEMP1 = ALPHA*A(J,J)
              IF (BETA.EQ.ZERO) THEN
                  DO 110 I = 1,M
                      C(I,J) = TEMP1*B(I,J)
  110             CONTINUE
              ELSE
                  DO 120 I = 1,M
                      C(I,J) = BETA*C(I,J) + TEMP1*B(I,J)
  120             CONTINUE
              END IF
              DO 140 K = 1,J - 1
                  IF (UPPER) THEN
                      TEMP1 = ALPHA*A(K,J)
                  ELSE
                      TEMP1 = ALPHA*A(J,K)
                  END IF
                  DO 130 I = 1,M
                      C(I,J) = C(I,J) + TEMP1*B(I,K)
  130             CONTINUE
  140         CONTINUE
              DO 160 K = J + 1,N
                  IF (UPPER) THEN
                      TEMP1 = ALPHA*A(J,K)
                  ELSE
                      TEMP1 = ALPHA*A(K,J)
                  END IF
                  DO 150 I = 1,M
                      C(I,J) = C(I,J) + TEMP1*B(I,K)
  150             CONTINUE
  160         CONTINUE
  170     CONTINUE
      END IF
*
      RETURN
*
*     End of DSYMM .
*
      END
```

———-

15.2.4 lisp code

— BLAS 3 dsymm —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (double-float 1.0 1.0) one)
           (type (double-float 0.0 0.0) zero))
  (defun dsymm (side uplo m n alpha a lda b ldb$ beta c ldc)
    (declare (type (simple-array double-float (*)) c b a)
             (type (double-float) beta alpha)
             (type fixnum ldc ldb$ lda n m)
             (type character uplo side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%)
         (c double-float c-%data% c-%offset%))
      (prog ((temp1 0.0) (temp2 0.0) (i 0) (info 0) (j 0) (k 0) (nrowa 0)
             (upper nil))
        (declare (type (double-float) temp1 temp2)
                 (type fixnum i info j k nrowa)
                 (type (member t nil) upper))
        (cond
          ((char-equal side #\L)
           (setf nrowa m))
          (t
           (setf nrowa n)))
        (setf upper (char-equal uplo #\U))
        (setf info 0)
        (cond
          ((and (not (char-equal side #\L)) (not (char-equal side #\R)))
           (setf info 1))
          ((and (not upper) (not (char-equal uplo #\L)))
           (setf info 2))
          ((< m 0)
           (setf info 3))
          ((< n 0)
           (setf info 4))
          ((< lda (max (the fixnum 1) (the fixnum nrowa)))
           (setf info 7))
          ((< ldb$ (max (the fixnum 1) (the fixnum m)))
           (setf info 9))
          ((< ldc (max (the fixnum 1) (the fixnum m)))
           (setf info 12)))
        (cond
          ((/= info 0)
           (error
            " ** On entry to ~a parameter number ~a had an illegal value~%"
```

```
    "DSYMM" info)
   (go end_label)))
(if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
    (go end_label))
(cond
  ((= alpha zero)
   (cond
     ((= beta zero)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data%
                                   (i j)
                                   ((1 ldc) (1 *))
                                   c-%offset%)
                    zero))))))
     (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data%
                                   (i j)
                                   ((1 ldc) (1 *))
                                   c-%offset%)
                    (* beta
                       (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)))))))))
   (go end_label)))
(cond
  ((char-equal side #\L)
   (cond
     (upper
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf temp1
                    (* alpha
                       (f2cl-lib:fref b-%data%
                                      (i j)
```

```
                          ((1 ldb$) (1 *))
                          b-%offset%)))
(setf temp2 zero)
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k
                  (f2cl-lib:int-add i
                                    (f2cl-lib:int-sub
                                     1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref c-%data%
                         (k j)
                         ((1 ldc) (1 *))
                         c-%offset%)
          (+
           (f2cl-lib:fref c-%data%
                          (k j)
                          ((1 ldc) (1 *))
                          c-%offset%)
           (* temp1
              (f2cl-lib:fref a-%data%
                             (k i)
                             ((1 lda) (1 *))
                             a-%offset%))))
    (setf temp2
          (+ temp2
             (*
              (f2cl-lib:fref b-%data%
                             (k j)
                             ((1 ldb$) (1 *))
                             b-%offset%)
              (f2cl-lib:fref a-%data%
                             (k i)
                             ((1 lda) (1 *))
                             a-%offset%))))))
(cond
  ((= beta zero)
   (setf (f2cl-lib:fref c-%data%
                        (i j)
                        ((1 ldc) (1 *))
                        c-%offset%)
         (+
          (* temp1
             (f2cl-lib:fref a-%data%
                            (i i)
                            ((1 lda) (1 *))
                            a-%offset%))
          (* alpha temp2))))
  (t
   (setf (f2cl-lib:fref c-%data%
```

```
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                         (+
                          (* beta
                             (f2cl-lib:fref c-%data%
                                            (i j)
                                            ((1 ldc) (1 *))
                                            c-%offset%))
                          (* temp1
                             (f2cl-lib:fref a-%data%
                                            (i i)
                                            ((1 lda) (1 *))
                                            a-%offset%))
                          (* alpha temp2)))))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                   ((> i 1) nil)
       (tagbody
         (setf temp1
                 (* alpha
                    (f2cl-lib:fref b-%data%
                                   (i j)
                                   ((1 ldb$) (1 *))
                                   b-%offset%)))
         (setf temp2 zero)
         (f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
                          (f2cl-lib:int-add k 1))
                        ((> k m) nil)
           (tagbody
             (setf (f2cl-lib:fref c-%data%
                                  (k j)
                                  ((1 ldc) (1 *))
                                  c-%offset%)
                     (+
                      (f2cl-lib:fref c-%data%
                                     (k j)
                                     ((1 ldc) (1 *))
                                     c-%offset%)
                      (* temp1
                         (f2cl-lib:fref a-%data%
                                        (k i)
                                        ((1 lda) (1 *))
                                        a-%offset%))))
             (setf temp2
                     (+ temp2
                        (*
```

```
                                (f2cl-lib:fref b-%data%
                                               (k j)
                                               ((1 ldb$) (1 *))
                                               b-%offset%)
                                (f2cl-lib:fref a-%data%
                                               (k i)
                                               ((1 lda) (1 *))
                                               a-%offset%))))))
                (cond
                  ((= beta zero)
                   (setf (f2cl-lib:fref c-%data%
                                        (i j)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                           (+
                            (* temp1
                               (f2cl-lib:fref a-%data%
                                              (i i)
                                              ((1 lda) (1 *))
                                              a-%offset%))
                            (* alpha temp2))))
                  (t
                   (setf (f2cl-lib:fref c-%data%
                                        (i j)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                           (+
                            (* beta
                               (f2cl-lib:fref c-%data%
                                              (i j)
                                              ((1 ldc) (1 *))
                                              c-%offset%))
                            (* temp1
                               (f2cl-lib:fref a-%data%
                                              (i i)
                                              ((1 lda) (1 *))
                                              a-%offset%))
                            (* alpha temp2))))))))))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf temp1
                     (* alpha
                        (f2cl-lib:fref a-%data%
                                       (j j)
                                       ((1 lda) (1 *))
                                       a-%offset%)))
             (cond
               ((= beta zero)
```

```
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data%
                             (i j)
                             ((1 ldc) (1 *))
                             c-%offset%)
                (* temp1
                   (f2cl-lib:fref b-%data%
                                  (i j)
                                  ((1 ldb$) (1 *))
                                  b-%offset%))))))
  (t
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data%
                             (i j)
                             ((1 ldc) (1 *))
                             c-%offset%)
                (+
                 (* beta
                    (f2cl-lib:fref c-%data%
                                   (i j)
                                   ((1 ldc) (1 *))
                                   c-%offset%))
                 (* temp1
                    (f2cl-lib:fref b-%data%
                                   (i j)
                                   ((1 ldb$) (1 *))
                                   b-%offset%))))))))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (cond
      (upper
       (setf temp1
               (* alpha
                  (f2cl-lib:fref a-%data%
                                 (k j)
                                 ((1 lda) (1 *))
                                 a-%offset%))))
      (t
       (setf temp1
               (* alpha
                  (f2cl-lib:fref a-%data%
                                 (j k)
                                 ((1 lda) (1 *))
                                 a-%offset%)))))
```

```
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref c-%data%
                               (i j)
                               ((1 ldc) (1 *))
                               c-%offset%)
                  (+
                   (f2cl-lib:fref c-%data%
                                  (i j)
                                  ((1 ldc) (1 *))
                                  c-%offset%)
                   (* temp1
                      (f2cl-lib:fref b-%data%
                                     (i k)
                                     ((1 ldb$) (1 *))
                                     b-%offset%))))))))
(f2cl-lib:fdo (k (f2cl-lib:int-add j 1) (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (cond
      (upper
       (setf temp1
               (* alpha
                  (f2cl-lib:fref a-%data%
                                 (j k)
                                 ((1 lda) (1 *))
                                 a-%offset%))))
      (t
       (setf temp1
               (* alpha
                  (f2cl-lib:fref a-%data%
                                 (k j)
                                 ((1 lda) (1 *))
                                 a-%offset%)))))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref c-%data%
                               (i j)
                               ((1 ldc) (1 *))
                               c-%offset%)
                  (+
                   (f2cl-lib:fref c-%data%
                                  (i j)
                                  ((1 ldc) (1 *))
                                  c-%offset%)
                   (* temp1
                      (f2cl-lib:fref b-%data%
                                     (i k)
```

```
                                                  ((1 ldb$) (1 *))
                                                  b-%offset%)))))))))))))
 end_label
        (return (values nil nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

15.3 dsyr2k - $\alpha AB^T + \alpha BA^T + \beta C$ or $\alpha A^T B + \alpha B^T A + \beta C$

15.3.1 Axiom unit tests

— **dsyr2k.input** —

```
)set break resume
)sys rm -f dsyr2k.output
)spool dsyr2k.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

15.3.2 Axiom help page

— **dsyr2k.help** —

```
==========================================================================
dsyr2k examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

 NAME
      DSYR2K - perform one of the symmetric rank 2k operations   C
      := alpha*A*B' + alpha*B*A' + beta*C,

 SYNOPSIS
      SUBROUTINE DSYR2K( UPLO, TRANS, N, K, ALPHA, A, LDA, B, LDB,
```

```
                    BETA, C, LDC )

      CHARACTER*1   UPLO, TRANS

      INTEGER       N, K, LDA, LDB, LDC

      DOUBLE        PRECISION ALPHA, BETA

      DOUBLE        PRECISION A( LDA, * ), B( LDB, * ), C(
                    LDC, * )

PURPOSE
     DSYR2K  performs one of the symmetric rank 2k operations

     or

        C := alpha*A'*B + alpha*B'*A + beta*C,

     where  alpha and beta  are scalars, C is an  n by n  sym-
     metric matrix and  A and B  are  n by k  matrices  in the
     first  case  and  k by n matrices in the second case.

PARAMETERS
     UPLO   - CHARACTER*1.
            On  entry,  UPLO  specifies  whether  the  upper  or
            lower triangular  part  of the  array  C  is to be
            referenced  as follows:

            UPLO = 'U' or 'u'   Only the  upper triangular part
            of  C is to be referenced.

            UPLO = 'L' or 'l'   Only the  lower triangular part
            of  C is to be referenced.

            Unchanged on exit.

     TRANS  - CHARACTER*1.
            On entry,  TRANS  specifies the operation to be per-
            formed as follows:

            TRANS = 'N' or 'n'   C := alpha*A*B' + alpha*B*A' +
            beta*C.

            TRANS = 'T' or 't'   C := alpha*A'*B + alpha*B'*A +
            beta*C.

            TRANS = 'C' or 'c'   C := alpha*A'*B + alpha*B'*A +
            beta*C.

            Unchanged on exit.
```

```
     N      - INTEGER.
            On entry,  N specifies the order of the matrix C.  N
            must be at least zero.  Unchanged on exit.

     K      - INTEGER.
            On entry with  TRANS = 'N' or 'n',  K  specifies  the
            number of  columns  of the  matrices  A and B,  and
            on  entry  with TRANS = 'T' or 't' or 'C' or 'c',  K
            specifies  the  number of rows of the matrices  A and
            B.  K must be at least  zero.  Unchanged on exit.

     ALPHA  - DOUBLE PRECISION.
            On entry, ALPHA specifies the scalar alpha.
            Unchanged on exit.

ka is
     A      -
             DOUBLE PRECISION array of DIMENSION ( LDA, ka ), where
            k  when  TRANS = 'N' or 'n',  and is  n  otherwise.
            Before entry with  TRANS = 'N' or 'n',  the  leading
            n by k part of the array  A  must contain the matrix
            A,  otherwise the leading  k by n  part of the array
            A  must contain  the matrix A.  Unchanged on exit.

     LDA    - INTEGER.
            On entry, LDA specifies the first dimension of A as
            declared in  the  calling  (sub)  program.   When
            TRANS = 'N' or 'n' then  LDA must be at least  max(
            1, n ), otherwise  LDA must be at least  max( 1, k ).
            Unchanged on exit.

kb is
     B      -
             DOUBLE PRECISION array of DIMENSION ( LDB, kb ), where
            k  when  TRANS = 'N' or 'n',  and is  n  otherwise.
            Before entry with  TRANS = 'N' or 'n',  the  leading
            n by k part of the array  B  must contain the matrix
            B,  otherwise the leading  k by n  part of the array
            B  must contain  the matrix B.  Unchanged on exit.

     LDB    - INTEGER.
            On entry, LDB specifies the first dimension of B as
            declared in  the  calling  (sub)  program.   When
            TRANS = 'N' or 'n' then  LDB must be at least  max(
            1, n ), otherwise  LDB must be at least  max( 1, k ).
            Unchanged on exit.

     BETA   - DOUBLE PRECISION.
            On entry, BETA specifies the scalar beta.  Unchanged
```

```
            on exit.

      C     - DOUBLE PRECISION array of DIMENSION ( LDC, n ).
            Before entry  with  UPLO = 'U' or 'u',  the leading
            n by n upper triangular part of the array C must con-
            tain the upper triangular part  of the  symmetric
            matrix  and the strictly lower triangular part of C
            is not referenced.  On exit, the upper triangular
            part of the array  C is overwritten by the upper tri-
            angular part of the updated matrix.  Before entry
            with  UPLO = 'L' or 'l',  the leading  n by n lower
            triangular part of the array C must contain the lower
            triangular part  of the  symmetric matrix  and the
            strictly upper triangular part of C is not refer-
            enced.  On exit, the lower triangular part of the
            array  C is overwritten by the lower triangular part
            of the updated matrix.

      LDC   - INTEGER.
            On entry, LDC specifies the first dimension of C as
            declared in  the  calling  (sub)  program.   LDC
            must  be  at  least max( 1, n ).  Unchanged on exit.
```

———-

15.3.3 fortran code

— dsyr2k.f —

```
*  ===================================================================
      SUBROUTINE DSYR2K(UPLO,TRANS,N,K,ALPHA,A,LDA,B,LDB,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION ALPHA,BETA
      INTEGER K,LDA,LDB,LDC,N
      CHARACTER TRANS,UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A(LDA,*),B(LDB,*),C(LDC,*)
*     ..
*
*  ===================================================================
```

```
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP1,TEMP2
      INTEGER I,INFO,J,L,NROWA
      LOGICAL UPPER
*     ..
*     .. Parameters ..
      DOUBLE PRECISION ONE,ZERO
      PARAMETER (ONE=1.0D+0,ZERO=0.0D+0)
*     ..
*
*     Test the input parameters.
*
      IF (LSAME(TRANS,'N')) THEN
          NROWA = N
      ELSE
          NROWA = K
      END IF
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 1
      ELSE IF ((.NOT.LSAME(TRANS,'N')) .AND.
     +         (.NOT.LSAME(TRANS,'T')) .AND.
     +         (.NOT.LSAME(TRANS,'C'))) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
          INFO = 3
      ELSE IF (K.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 7
      ELSE IF (LDB.LT.MAX(1,NROWA)) THEN
          INFO = 9
      ELSE IF (LDC.LT.MAX(1,N)) THEN
          INFO = 12
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('DSYR2K',INFO)
```

```
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (((ALPHA.EQ.ZERO).OR.
     +    (K.EQ.0)).AND. (BETA.EQ.ONE))) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
         IF (UPPER) THEN
            IF (BETA.EQ.ZERO) THEN
               DO 20 J = 1,N
                  DO 10 I = 1,J
                     C(I,J) = ZERO
   10             CONTINUE
   20          CONTINUE
            ELSE
               DO 40 J = 1,N
                  DO 30 I = 1,J
                     C(I,J) = BETA*C(I,J)
   30             CONTINUE
   40          CONTINUE
            END IF
         ELSE
            IF (BETA.EQ.ZERO) THEN
               DO 60 J = 1,N
                  DO 50 I = J,N
                     C(I,J) = ZERO
   50             CONTINUE
   60          CONTINUE
            ELSE
               DO 80 J = 1,N
                  DO 70 I = J,N
                     C(I,J) = BETA*C(I,J)
   70             CONTINUE
   80          CONTINUE
            END IF
         END IF
         RETURN
      END IF
*
*     Start the operations.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  C := alpha*A*B**T + alpha*B*A**T + C.
*
         IF (UPPER) THEN
```

```
            DO 130 J = 1,N
                IF (BETA.EQ.ZERO) THEN
                    DO 90 I = 1,J
                        C(I,J) = ZERO
 90                 CONTINUE
                ELSE IF (BETA.NE.ONE) THEN
                    DO 100 I = 1,J
                        C(I,J) = BETA*C(I,J)
100                 CONTINUE
                END IF
                DO 120 L = 1,K
                    IF ((A(J,L).NE.ZERO) .OR. (B(J,L).NE.ZERO)) THEN
                        TEMP1 = ALPHA*B(J,L)
                        TEMP2 = ALPHA*A(J,L)
                        DO 110 I = 1,J
                            C(I,J) = C(I,J) + A(I,L)*TEMP1 +
     +                              B(I,L)*TEMP2
110                     CONTINUE
                    END IF
120             CONTINUE
130         CONTINUE
        ELSE
            DO 180 J = 1,N
                IF (BETA.EQ.ZERO) THEN
                    DO 140 I = J,N
                        C(I,J) = ZERO
140                 CONTINUE
                ELSE IF (BETA.NE.ONE) THEN
                    DO 150 I = J,N
                        C(I,J) = BETA*C(I,J)
150                 CONTINUE
                END IF
                DO 170 L = 1,K
                    IF ((A(J,L).NE.ZERO) .OR. (B(J,L).NE.ZERO)) THEN
                        TEMP1 = ALPHA*B(J,L)
                        TEMP2 = ALPHA*A(J,L)
                        DO 160 I = J,N
                            C(I,J) = C(I,J) + A(I,L)*TEMP1 +
     +                              B(I,L)*TEMP2
160                     CONTINUE
                    END IF
170             CONTINUE
180         CONTINUE
        END IF
    ELSE
*
*       Form  C := alpha*A**T*B + alpha*B**T*A + C.
*
        IF (UPPER) THEN
            DO 210 J = 1,N
```

```
                DO 200 I = 1,J
                    TEMP1 = ZERO
                    TEMP2 = ZERO
                    DO 190 L = 1,K
                        TEMP1 = TEMP1 + A(L,I)*B(L,J)
                        TEMP2 = TEMP2 + B(L,I)*A(L,J)
 190                CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP1 + ALPHA*TEMP2
                    ELSE
                        C(I,J) = BETA*C(I,J) + ALPHA*TEMP1 +
   +                             ALPHA*TEMP2
                    END IF
 200            CONTINUE
 210        CONTINUE
        ELSE
            DO 240 J = 1,N
                DO 230 I = J,N
                    TEMP1 = ZERO
                    TEMP2 = ZERO
                    DO 220 L = 1,K
                        TEMP1 = TEMP1 + A(L,I)*B(L,J)
                        TEMP2 = TEMP2 + B(L,I)*A(L,J)
 220                CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP1 + ALPHA*TEMP2
                    ELSE
                        C(I,J) = BETA*C(I,J) + ALPHA*TEMP1 +
   +                             ALPHA*TEMP2
                    END IF
 230            CONTINUE
 240        CONTINUE
        END IF
    END IF
*
    RETURN
*
*   End of DSYR2K.
*
    END
```

———-

15.3.4 lisp code

— BLAS 3 dsyr2k —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (double-float 1.0 1.0) one)
           (type (double-float 0.0 0.0) zero))
  (defun dsyr2k (uplo trans n k alpha a lda b ldb$ beta c ldc)
    (declare (type (simple-array double-float (*)) c b a)
             (type (double-float) beta alpha)
             (type fixnum ldc ldb$ lda k n)
             (type character trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%)
         (c double-float c-%data% c-%offset%))
      (prog ((temp1 0.0) (temp2 0.0) (i 0) (info 0) (j 0) (l 0) (nrowa 0)
             (upper nil))
        (declare (type (double-float) temp1 temp2)
                 (type fixnum i info j l nrowa)
                 (type (member t nil) upper))
        (cond
          ((char-equal trans #\N)
           (setf nrowa n))
          (t
           (setf nrowa k)))
        (setf upper (char-equal uplo #\U))
        (setf info 0)
        (cond
          ((and (not upper) (not (char-equal uplo #\L)))
           (setf info 1))
          ((and (not (char-equal trans #\N))
                (not (char-equal trans #\T))
                (not (char-equal trans #\C)))
           (setf info 2))
          ((< n 0)
           (setf info 3))
          ((< k 0)
           (setf info 4))
          ((< lda (max (the fixnum 1) (the fixnum nrowa)))
           (setf info 7))
          ((< ldb$
              (max (the fixnum 1) (the fixnum nrowa)))
           (setf info 9))
          ((< ldc (max (the fixnum 1) (the fixnum n)))
           (setf info 12)))
        (cond
          ((/= info 0)
           (error
            " ** On entry to ~a parameter number ~a had an illegal value~%"
            "DSYR2K" info)
           (go end_label)))
```

```
(if (or (= n 0) (and (or (= alpha zero) (= k 0)) (= beta one)))
    (go end_label))
(cond
  ((= alpha zero)
   (cond
     (upper
      (cond
        ((= beta zero)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i j) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                       zero))))))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i j) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                       (* beta
                          (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%))))))))))
     (t
      (cond
        ((= beta zero)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                           ((> i n) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                       zero))))))
        (t
```

```
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)
                          (* beta
                             (f2cl-lib:fref c-%data%
                                            (i j)
                                            ((1 ldc) (1 *))
                                            c-%offset%))))))))))
   (go end_label)))
(cond
  ((char-equal trans #\N)
   (cond
     (upper
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
            ((= beta zero)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i j) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                       zero))))
            ((/= beta one)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i j) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                       (* beta
                          (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%))))))
          (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                        ((> l k) nil)
            (tagbody
              (cond
```

```
            ((or (/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
                 (/= (f2cl-lib:fref b (j l) ((1 ldb$) (1 *))) zero))
             (setf temp1
                     (* alpha
                        (f2cl-lib:fref b-%data%
                                       (j l)
                                       ((1 ldb$) (1 *))
                                       b-%offset%)))
             (setf temp2
                     (* alpha
                        (f2cl-lib:fref a-%data%
                                       (j l)
                                       ((1 lda) (1 *))
                                       a-%offset%)))
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i j) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                       (+
                        (f2cl-lib:fref c-%data%
                                       (i j)
                                       ((1 ldc) (1 *))
                                       c-%offset%)
                        (*
                         (f2cl-lib:fref a-%data%
                                        (i l)
                                        ((1 lda) (1 *))
                                        a-%offset%)
                         temp1)
                        (*
                         (f2cl-lib:fref b-%data%
                                        (i l)
                                        ((1 ldb$) (1 *))
                                        b-%offset%)
                         temp2))))))))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((= beta zero)
        (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data%
                                 (i j)
                                 ((1 ldc) (1 *))
```

```
                                c-%offset%)
                  zero))))
      ((/= beta one)
       (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref c-%data%
                                (i j)
                                ((1 ldc) (1 *))
                                c-%offset%)
                 (* beta
                    (f2cl-lib:fref c-%data%
                                   (i j)
                                   ((1 ldc) (1 *))
                                   c-%offset%)))))))
    (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                  ((> l k) nil)
      (tagbody
        (cond
          ((or (/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
               (/= (f2cl-lib:fref b (j l) ((1 ldb$) (1 *))) zero))
           (setf temp1
                 (* alpha
                    (f2cl-lib:fref b-%data%
                                   (j l)
                                   ((1 ldb$) (1 *))
                                   b-%offset%)))
           (setf temp2
                 (* alpha
                    (f2cl-lib:fref a-%data%
                                   (j l)
                                   ((1 lda) (1 *))
                                   a-%offset%)))
           (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                     (+
                      (f2cl-lib:fref c-%data%
                                     (i j)
                                     ((1 ldc) (1 *))
                                     c-%offset%)
                      (*
                       (f2cl-lib:fref a-%data%
                                      (i l)
                                      ((1 lda) (1 *))
                                      a-%offset%)
```

```
                              temp1)
                             (*
                              (f2cl-lib:fref b-%data%
                                             (i l)
                                             ((1 ldb$) (1 *))
                                             b-%offset%)
                              temp2)))))))))))))))
(t
 (cond
   (upper
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i j) nil)
          (tagbody
            (setf temp1 zero)
            (setf temp2 zero)
            (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                          ((> l k) nil)
              (tagbody
                (setf temp1
                        (+ temp1
                           (*
                            (f2cl-lib:fref a-%data%
                                           (l i)
                                           ((1 lda) (1 *))
                                           a-%offset%)
                            (f2cl-lib:fref b-%data%
                                           (l j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%))))
                (setf temp2
                        (+ temp2
                           (*
                            (f2cl-lib:fref b-%data%
                                           (l i)
                                           ((1 ldb$) (1 *))
                                           b-%offset%)
                            (f2cl-lib:fref a-%data%
                                           (l j)
                                           ((1 lda) (1 *))
                                           a-%offset%))))))
            (cond
              ((= beta zero)
               (setf (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                     (+ (* alpha temp1) (* alpha temp2))))
```

```
            (t
             (setf (f2cl-lib:fref c-%data%
                                  (i j)
                                  ((1 ldc) (1 *))
                                  c-%offset%)
                   (+
                    (* beta
                       (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%))
                    (* alpha temp1)
                    (* alpha temp2)))))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf temp1 zero)
         (setf temp2 zero)
         (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                       ((> l k) nil)
           (tagbody
             (setf temp1
                     (+ temp1
                        (*
                         (f2cl-lib:fref a-%data%
                                        (l i)
                                        ((1 lda) (1 *))
                                        a-%offset%)
                         (f2cl-lib:fref b-%data%
                                        (l j)
                                        ((1 ldb$) (1 *))
                                        b-%offset%))))
             (setf temp2
                     (+ temp2
                        (*
                         (f2cl-lib:fref b-%data%
                                        (l i)
                                        ((1 ldb$) (1 *))
                                        b-%offset%)
                         (f2cl-lib:fref a-%data%
                                        (l j)
                                        ((1 lda) (1 *))
                                        a-%offset%))))))
         (cond
           ((= beta zero)
            (setf (f2cl-lib:fref c-%data%
```

```
                                        (i j)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                                (+ (* alpha temp1) (* alpha temp2))))
                        (t
                         (setf (f2cl-lib:fref c-%data%
                                        (i j)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                                (+
                                 (* beta
                                    (f2cl-lib:fref c-%data%
                                                   (i j)
                                                   ((1 ldc) (1 *))
                                                   c-%offset%))
                                 (* alpha temp1)
                                 (* alpha temp2)))))))))))))
 end_label
        (return (values nil nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

15.4 dsyrk - symmetric rank k $\alpha AA^T + \beta C$ or $\alpha A^T A + \beta C$

15.4.1 Axiom unit tests

— **dsyrk.input** —

```
)set break resume
)sys rm -f dsyrk.output
)spool dsyrk.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

15.4.2 Axiom help page

— **dsyrk.help** —

```
=====================================================================
dsyrk examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

 NAME
      DSYRK - perform one of the symmetric rank k operations   C
      := alpha*A*A' + beta*C,

 SYNOPSIS
      SUBROUTINE DSYRK ( UPLO, TRANS, N, K, ALPHA, A, LDA, BETA,
                         C, LDC )

          CHARACTER*1  UPLO, TRANS

          INTEGER      N, K, LDA, LDC

          DOUBLE       PRECISION ALPHA, BETA

          DOUBLE       PRECISION A( LDA, * ), C( LDC, * )

 PURPOSE
      DSYRK  performs one of the symmetric rank k operations

      or

         C := alpha*A'*A + beta*C,

      where  alpha and beta  are scalars, C is an  n by n  sym-
      metric matrix and  A  is an  n by k  matrix in the first
      case and a  k by n  matrix in the second case.

 PARAMETERS
      UPLO   - CHARACTER*1.
             On  entry,  UPLO  specifies  whether  the  upper  or
             lower triangular  part  of the  array  C  is to be
             referenced  as follows:

             UPLO = 'U' or 'u'   Only the  upper triangular part
             of  C is to be referenced.

             UPLO = 'L' or 'l'   Only the  lower triangular part
             of  C is to be referenced.

             Unchanged on exit.

      TRANS  - CHARACTER*1.
```

```
            On entry,  TRANS  specifies the operation to be per-
            formed as follows:

            TRANS = 'N' or 'n'   C := alpha*A*A' + beta*C.

            TRANS = 'T' or 't'   C := alpha*A'*A + beta*C.

            TRANS = 'C' or 'c'   C := alpha*A'*A + beta*C.

            Unchanged on exit.

     N      - INTEGER.
            On entry,  N specifies the order of the matrix C.  N
            must be at least zero.  Unchanged on exit.

     K      - INTEGER.
            On entry with  TRANS = 'N' or 'n',  K  specifies  the
            number of  columns   of  the   matrix   A,   and  on
            entry   with TRANS = 'T' or 't' or 'C' or 'c',  K
            specifies  the  number of rows of the matrix  A.  K
            must be at least zero.  Unchanged on exit.

     ALPHA  - DOUBLE PRECISION.
            On entry, ALPHA specifies the scalar alpha.
            Unchanged on exit.

ka is
     A      -
             DOUBLE PRECISION array of DIMENSION ( LDA, ka ), where
            k  when  TRANS = 'N' or 'n',  and is  n  otherwise.
            Before entry with  TRANS = 'N' or 'n',  the  leading
            n by k part of the array  A  must contain the matrix
            A,  otherwise the leading  k by n  part of the array
            A  must contain  the matrix A.  Unchanged on exit.

     LDA    - INTEGER.
            On entry, LDA specifies the first dimension of A as
            declared in  the  calling  (sub)  program.   When
            TRANS = 'N' or 'n' then  LDA must be at least  max(
            1, n ), otherwise  LDA must be at least  max( 1, k ).
            Unchanged on exit.

     BETA   - DOUBLE PRECISION.
            On entry, BETA specifies the scalar beta.  Unchanged
            on exit.

     C      - DOUBLE PRECISION array of DIMENSION ( LDC, n ).
            Before entry  with  UPLO = 'U' or 'u',  the leading
            n by n upper triangular part of the array C must con-
            tain the upper triangular part  of the  symmetric
```

```
            matrix  and the strictly lower triangular part of C
            is not referenced.  On exit, the upper triangular
            part of the array  C is overwritten by the upper tri-
            angular part of the updated matrix.  Before entry
            with  UPLO = 'L' or 'l',  the leading  n by n lower
            triangular part of the array C must contain the lower
            triangular part  of the  symmetric matrix  and the
            strictly upper triangular part of C is not refer-
            enced.  On exit, the lower triangular part of the
            array  C is overwritten by the lower triangular part
            of the updated matrix.

      LDC    - INTEGER.
             On entry, LDC specifies the first dimension of C as
             declared in  the  calling  (sub)  program.   LDC
             must  be  at  least max( 1, n ).  Unchanged on exit.
```

———-

15.4.3 fortran code

— dsyrk.f —

```
*  ==================================================================
      SUBROUTINE DSYRK(UPLO,TRANS,N,K,ALPHA,A,LDA,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION ALPHA,BETA
      INTEGER K,LDA,LDC,N
      CHARACTER TRANS,UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A(LDA,*),C(LDC,*)
*     ..
*
*  ==================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
```

```
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP
      INTEGER I,INFO,J,L,NROWA
      LOGICAL UPPER
*     ..
*     .. Parameters ..
      DOUBLE PRECISION ONE,ZERO
      PARAMETER (ONE=1.0D+0,ZERO=0.0D+0)
*     ..
*
*     Test the input parameters.
*
      IF (LSAME(TRANS,'N')) THEN
          NROWA = N
      ELSE
          NROWA = K
      END IF
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 1
      ELSE IF ((.NOT.LSAME(TRANS,'N')) .AND.
     +         (.NOT.LSAME(TRANS,'T')) .AND.
     +         (.NOT.LSAME(TRANS,'C'))) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
          INFO = 3
      ELSE IF (K.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 7
      ELSE IF (LDC.LT.MAX(1,N)) THEN
          INFO = 10
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('DSYRK ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (((ALPHA.EQ.ZERO).OR.
     +    (K.EQ.0)).AND. (BETA.EQ.ONE))) RETURN
*
```

```
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
          IF (UPPER) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 20 J = 1,N
                      DO 10 I = 1,J
                          C(I,J) = ZERO
   10                 CONTINUE
   20             CONTINUE
              ELSE
                  DO 40 J = 1,N
                      DO 30 I = 1,J
                          C(I,J) = BETA*C(I,J)
   30                 CONTINUE
   40             CONTINUE
              END IF
          ELSE
              IF (BETA.EQ.ZERO) THEN
                  DO 60 J = 1,N
                      DO 50 I = J,N
                          C(I,J) = ZERO
   50                 CONTINUE
   60             CONTINUE
              ELSE
                  DO 80 J = 1,N
                      DO 70 I = J,N
                          C(I,J) = BETA*C(I,J)
   70                 CONTINUE
   80             CONTINUE
              END IF
          END IF
          RETURN
      END IF
*
*     Start the operations.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  C := alpha*A*A**T + beta*C.
*
          IF (UPPER) THEN
              DO 130 J = 1,N
                  IF (BETA.EQ.ZERO) THEN
                      DO 90 I = 1,J
                          C(I,J) = ZERO
   90                 CONTINUE
                  ELSE IF (BETA.NE.ONE) THEN
                      DO 100 I = 1,J
                          C(I,J) = BETA*C(I,J)
```

```
 100                CONTINUE
                END IF
                DO 120 L = 1,K
                    IF (A(J,L).NE.ZERO) THEN
                        TEMP = ALPHA*A(J,L)
                        DO 110 I = 1,J
                            C(I,J) = C(I,J) + TEMP*A(I,L)
 110                    CONTINUE
                    END IF
 120            CONTINUE
 130        CONTINUE
        ELSE
            DO 180 J = 1,N
                IF (BETA.EQ.ZERO) THEN
                    DO 140 I = J,N
                        C(I,J) = ZERO
 140                CONTINUE
                ELSE IF (BETA.NE.ONE) THEN
                    DO 150 I = J,N
                        C(I,J) = BETA*C(I,J)
 150                CONTINUE
                END IF
                DO 170 L = 1,K
                    IF (A(J,L).NE.ZERO) THEN
                        TEMP = ALPHA*A(J,L)
                        DO 160 I = J,N
                            C(I,J) = C(I,J) + TEMP*A(I,L)
 160                    CONTINUE
                    END IF
 170            CONTINUE
 180        CONTINUE
        END IF
    ELSE
*
*       Form  C := alpha*A**T*A + beta*C.
*
        IF (UPPER) THEN
            DO 210 J = 1,N
                DO 200 I = 1,J
                    TEMP = ZERO
                    DO 190 L = 1,K
                        TEMP = TEMP + A(L,I)*A(L,J)
 190                CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP
                    ELSE
                        C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                    END IF
 200            CONTINUE
 210        CONTINUE
```

```
         ELSE
             DO 240 J = 1,N
                 DO 230 I = J,N
                     TEMP = ZERO
                     DO 220 L = 1,K
                         TEMP = TEMP + A(L,I)*A(L,J)
  220                CONTINUE
                     IF (BETA.EQ.ZERO) THEN
                         C(I,J) = ALPHA*TEMP
                     ELSE
                         C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                     END IF
  230            CONTINUE
  240         CONTINUE
         END IF
      END IF
*
      RETURN
*
*     End of DSYRK .
*
      END
```

———-

15.4.4 lisp code

— BLAS 3 dsyrk —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (double-float 1.0 1.0) one)
           (type (double-float 0.0 0.0) zero))
  (defun dsyrk (uplo trans n k alpha a lda beta c ldc)
    (declare (type (simple-array double-float (*)) c a)
             (type (double-float) beta alpha)
             (type fixnum ldc lda k n)
             (type character trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (a double-float a-%data% a-%offset%)
         (c double-float c-%data% c-%offset%))
      (prog ((temp 0.0) (i 0) (info 0) (j 0) (l 0) (nrowa 0) (upper nil))
        (declare (type (double-float) temp)
                 (type fixnum i info j l nrowa)
                 (type (member t nil) upper))
        (cond
```

```
  ((char-equal trans #\N)
   (setf nrowa n))
  (t
   (setf nrowa k)))
(setf upper (char-equal uplo #\U))
(setf info 0)
(cond
  ((and (not upper) (not (char-equal uplo #\L)))
   (setf info 1))
  ((and (not (char-equal trans #\N))
        (not (char-equal trans #\T))
        (not (char-equal trans #\C)))
   (setf info 2))
  ((< n 0)
   (setf info 3))
  ((< k 0)
   (setf info 4))
  ((< lda (max (the fixnum 1) (the fixnum nrowa)))
   (setf info 7))
  ((< ldc (max (the fixnum 1) (the fixnum n)))
   (setf info 10)))
(cond
  ((/= info 0)
   (error
    " ** On entry to ~a parameter number ~a had an illegal value~%"
    "DSYRK" info)
   (go end_label)))
(if (or (= n 0) (and (or (= alpha zero) (= k 0)) (= beta one)))
    (go end_label))
(cond
  ((= alpha zero)
   (cond
     (upper
      (cond
        ((= beta zero)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i j) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                       zero))))))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
```

```
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i j) nil)
                  (tagbody
                    (setf (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)
                          (* beta
                             (f2cl-lib:fref c-%data%
                                            (i j)
                                            ((1 ldc) (1 *))
                                            c-%offset%))))))))))
        (t
         (cond
           ((= beta zero)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)
                          zero))))))
           (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)
                          (* beta
                             (f2cl-lib:fref c-%data%
                                            (i j)
                                            ((1 ldc) (1 *))
                                            c-%offset%))))))))))
    (go end_label)))
  (cond
    ((char-equal trans #\N)
     (cond
       (upper
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
```

```
(cond
  ((= beta zero)
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i j) nil)
     (tagbody
       (setf (f2cl-lib:fref c-%data%
                            (i j)
                            ((1 ldc) (1 *))
                            c-%offset%)
             zero))))
  ((/= beta one)
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i j) nil)
     (tagbody
       (setf (f2cl-lib:fref c-%data%
                            (i j)
                            ((1 ldc) (1 *))
                            c-%offset%)
             (* beta
                (f2cl-lib:fref c-%data%
                               (i j)
                               ((1 ldc) (1 *))
                               c-%offset%)))))))
(f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
              ((> l k) nil)
  (tagbody
    (cond
      ((/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
       (setf temp
             (* alpha
                (f2cl-lib:fref a-%data%
                               (j l)
                               ((1 lda) (1 *))
                               a-%offset%)))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i j) nil)
         (tagbody
           (setf (f2cl-lib:fref c-%data%
                                (i j)
                                ((1 ldc) (1 *))
                                c-%offset%)
                 (+
                  (f2cl-lib:fref c-%data%
                                 (i j)
                                 ((1 ldc) (1 *))
                                 c-%offset%)
                  (* temp
                     (f2cl-lib:fref a-%data%
                                    (i l)
                                    ((1 lda) (1 *))
```

```
                                                a-%offset%))))))))))))))
            (t
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (cond
                   ((= beta zero)
                    (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                                  ((> i n) nil)
                      (tagbody
                        (setf (f2cl-lib:fref c-%data%
                                             (i j)
                                             ((1 ldc) (1 *))
                                             c-%offset%)
                              zero))))
                   ((/= beta one)
                    (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                                  ((> i n) nil)
                      (tagbody
                        (setf (f2cl-lib:fref c-%data%
                                             (i j)
                                             ((1 ldc) (1 *))
                                             c-%offset%)
                              (* beta
                                 (f2cl-lib:fref c-%data%
                                                (i j)
                                                ((1 ldc) (1 *))
                                                c-%offset%)))))))
                 (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                               ((> l k) nil)
                   (tagbody
                     (cond
                       ((/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
                        (setf temp
                              (* alpha
                                 (f2cl-lib:fref a-%data%
                                                (j l)
                                                ((1 lda) (1 *))
                                                a-%offset%)))
                        (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                                      ((> i n) nil)
                          (tagbody
                            (setf (f2cl-lib:fref c-%data%
                                                 (i j)
                                                 ((1 ldc) (1 *))
                                                 c-%offset%)
                                  (+
                                   (f2cl-lib:fref c-%data%
                                                  (i j)
                                                  ((1 ldc) (1 *))
```

```
                                          c-%offset%)
                            (* temp
                               (f2cl-lib:fref a-%data%
                                         (i l)
                                         ((1 lda) (1 *))
                                         a-%offset%)))))))))))))))
(t
 (cond
   (upper
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i j) nil)
          (tagbody
            (setf temp zero)
            (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                          ((> l k) nil)
              (tagbody
                (setf temp
                        (+ temp
                           (*
                            (f2cl-lib:fref a-%data%
                                           (l i)
                                           ((1 lda) (1 *))
                                           a-%offset%)
                            (f2cl-lib:fref a-%data%
                                           (l j)
                                           ((1 lda) (1 *))
                                           a-%offset%))))))
            (cond
              ((= beta zero)
               (setf (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                       (* alpha temp)))
              (t
               (setf (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                       (+ (* alpha temp)
                          (* beta
                             (f2cl-lib:fref c-%data%
                                            (i j)
                                            ((1 ldc) (1 *))
                                            c-%offset%))))))))))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                    ((> j n) nil)
            (tagbody
              (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                            ((> i n) nil)
                (tagbody
                  (setf temp zero)
                  (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                                ((> l k) nil)
                    (tagbody
                      (setf temp
                              (+ temp
                                 (*
                                  (f2cl-lib:fref a-%data%
                                                 (l i)
                                                 ((1 lda) (1 *))
                                                 a-%offset%)
                                  (f2cl-lib:fref a-%data%
                                                 (l j)
                                                 ((1 lda) (1 *))
                                                 a-%offset%))))))
                  (cond
                    ((= beta zero)
                     (setf (f2cl-lib:fref c-%data%
                                          (i j)
                                          ((1 ldc) (1 *))
                                          c-%offset%)
                             (* alpha temp)))
                    (t
                     (setf (f2cl-lib:fref c-%data%
                                          (i j)
                                          ((1 ldc) (1 *))
                                          c-%offset%)
                             (+ (* alpha temp)
                                (* beta
                                   (f2cl-lib:fref c-%data%
                                                  (i j)
                                                  ((1 ldc) (1 *))
                                                  c-%offset%))))))))))))))
end_label
       (return (values nil nil nil nil nil nil nil nil nil nil))))))
```

———-

15.5 dtrmm - $\alpha op(A)B$ or $\alpha Bop(A)$, $op(A) = A$ or $op(X) = A^T$

15.5.1 Axiom unit tests

— **dtrmm.input** —

```
)set break resume
)sys rm -f dtrmm.output
)spool dtrmm.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

15.5.2 Axiom help page

— **dtrmm.help** —

```
====================================================================
dtrmm examples
====================================================================

====================================================================
Man Page Details
====================================================================

 NAME
      DTRMM - perform one of the matrix-matrix operations   B :=
      alpha*op( A )*B, or B := alpha*B*op( A ),

 SYNOPSIS
      SUBROUTINE DTRMM ( SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A,
                         LDA, B, LDB )

          CHARACTER*1  SIDE, UPLO, TRANSA, DIAG

          INTEGER      M, N, LDA, LDB

          DOUBLE       PRECISION ALPHA
```

```
         DOUBLE       PRECISION A( LDA, * ), B( LDB, * )

PURPOSE
     DTRMM  performs one of the matrix-matrix operations

     where  alpha  is a scalar,  B  is an m by n matrix,  A  is a
     unit, or non-unit,  upper or lower triangular matrix  and
     op( A )  is one  of

        op( A ) = A   or   op( A ) = A'.

PARAMETERS
     SIDE   - CHARACTER*1.
            On entry,  SIDE specifies whether  op( A ) multiplies
            B from the left or right as follows:

            SIDE = 'L' or 'l'   B := alpha*op( A )*B.

            SIDE = 'R' or 'r'   B := alpha*B*op( A ).

            Unchanged on exit.

     UPLO   - CHARACTER*1.
            On entry, UPLO specifies whether the matrix A is an
            upper or lower triangular matrix as follows:

            UPLO = 'U' or 'u'   A is an upper triangular matrix.

            UPLO = 'L' or 'l'   A is a lower triangular matrix.

            Unchanged on exit.

            TRANSA - CHARACTER*1.  On entry, TRANSA specifies the
            form of op( A ) to be used in the matrix multiplica-
            tion as follows:

            TRANSA = 'N' or 'n'   op( A ) = A.

            TRANSA = 'T' or 't'   op( A ) = A'.

            TRANSA = 'C' or 'c'   op( A ) = A'.

            Unchanged on exit.

     DIAG   - CHARACTER*1.
            On entry, DIAG specifies whether or not A is unit
            triangular as follows:

            DIAG = 'U' or 'u'   A is assumed to be unit triangu-
            lar.
```

```
            DIAG = 'N' or 'n'   A is not assumed to be unit tri-
            angular.

            Unchanged on exit.

      M     - INTEGER.
            On entry, M specifies the number of rows of B. M must
            be at least zero.  Unchanged on exit.

      N     - INTEGER.
            On entry, N specifies the number of columns of B.  N
            must be at least zero.  Unchanged on exit.

      ALPHA - DOUBLE PRECISION.
            On entry,  ALPHA specifies the scalar  alpha. When
            alpha is zero then  A is not referenced and  B need
            not be set before entry.  Unchanged on exit.

 is m
      A     -
             DOUBLE PRECISION array of DIMENSION ( LDA, k ), where k
            when  SIDE = 'L' or 'l'  and is  n  when  SIDE = 'R'
            or 'r'.  Before entry  with  UPLO = 'U' or 'u',  the
            leading  k by k upper triangular part of the array  A
            must contain the upper triangular matrix  and the
            strictly lower triangular part of A is not refer-
            enced.  Before entry  with  UPLO = 'L' or 'l',  the
            leading  k by k lower triangular part of the array  A
            must contain the lower triangular matrix  and the
            strictly upper triangular part of A is not refer-
            enced.  Note that when  DIAG = 'U' or 'u',  the diag-
            onal elements of A  are not referenced either,  but
            are assumed to be  unity.  Unchanged on exit.

      LDA   - INTEGER.
            On entry, LDA specifies the first dimension of A as
            declared in the calling (sub) program.  When  SIDE =
            'L' or 'l'  then LDA  must be at least  max( 1, m ),
            when  SIDE = 'R' or 'r' then LDA must be at least

            max( 1, n ).  Unchanged on exit.

      B     - DOUBLE PRECISION array of DIMENSION ( LDB, n ).
            Before entry,  the leading  m by n part of the array
            B must contain the matrix  B,  and  on exit  is
            overwritten  by the transformed matrix.

      LDB   - INTEGER.
            On entry, LDB specifies the first dimension of B as
```

```
declared in  the  calling  (sub)  program.   LDB
must  be  at  least max( 1, m ).  Unchanged on exit.
```

———-

15.5.3 fortran code

— dtrmm.f —

```
*  ===================================================================
      SUBROUTINE DTRMM(SIDE,UPLO,TRANSA,DIAG,M,N,ALPHA,A,LDA,B,LDB)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION ALPHA
      INTEGER LDA,LDB,M,N
      CHARACTER DIAG,SIDE,TRANSA,UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A(LDA,*),B(LDB,*)
*     ..
*
*  ===================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP
      INTEGER I,INFO,J,K,NROWA
      LOGICAL LSIDE,NOUNIT,UPPER
*     ..
*     .. Parameters ..
      DOUBLE PRECISION ONE,ZERO
      PARAMETER (ONE=1.0D+0,ZERO=0.0D+0)
*     ..
```

```
*
*     Test the input parameters.
*
      LSIDE = LSAME(SIDE,'L')
      IF (LSIDE) THEN
          NROWA = M
      ELSE
          NROWA = N
      END IF
      NOUNIT = LSAME(DIAG,'N')
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.LSIDE) .AND. (.NOT.LSAME(SIDE,'R'))) THEN
          INFO = 1
      ELSE IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 2
      ELSE IF ((.NOT.LSAME(TRANSA,'N')) .AND.
     +         (.NOT.LSAME(TRANSA,'T')) .AND.
     +         (.NOT.LSAME(TRANSA,'C'))) THEN
          INFO = 3
      ELSE IF ((.NOT.LSAME(DIAG,'U')) .AND. (.NOT.LSAME(DIAG,'N'))) THEN
          INFO = 4
      ELSE IF (M.LT.0) THEN
          INFO = 5
      ELSE IF (N.LT.0) THEN
          INFO = 6
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 9
      ELSE IF (LDB.LT.MAX(1,M)) THEN
          INFO = 11
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('DTRMM ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (M.EQ.0 .OR. N.EQ.0) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
          DO 20 J = 1,N
              DO 10 I = 1,M
                  B(I,J) = ZERO
   10         CONTINUE
   20     CONTINUE
          RETURN
```

```
      END IF
*
*     Start the operations.
*
      IF (LSIDE) THEN
          IF (LSAME(TRANSA,'N')) THEN
*
*           Form  B := alpha*A*B.
*
              IF (UPPER) THEN
                  DO 50 J = 1,N
                      DO 40 K = 1,M
                          IF (B(K,J).NE.ZERO) THEN
                              TEMP = ALPHA*B(K,J)
                              DO 30 I = 1,K - 1
                                  B(I,J) = B(I,J) + TEMP*A(I,K)
   30                         CONTINUE
                              IF (NOUNIT) TEMP = TEMP*A(K,K)
                              B(K,J) = TEMP
                          END IF
   40                 CONTINUE
   50             CONTINUE
              ELSE
                  DO 80 J = 1,N
                      DO 70 K = M,1,-1
                          IF (B(K,J).NE.ZERO) THEN
                              TEMP = ALPHA*B(K,J)
                              B(K,J) = TEMP
                              IF (NOUNIT) B(K,J) = B(K,J)*A(K,K)
                              DO 60 I = K + 1,M
                                  B(I,J) = B(I,J) + TEMP*A(I,K)
   60                         CONTINUE
                          END IF
   70                 CONTINUE
   80             CONTINUE
              END IF
          ELSE
*
*           Form  B := alpha*A**T*B.
*
              IF (UPPER) THEN
                  DO 110 J = 1,N
                      DO 100 I = M,1,-1
                          TEMP = B(I,J)
                          IF (NOUNIT) TEMP = TEMP*A(I,I)
                          DO 90 K = 1,I - 1
                              TEMP = TEMP + A(K,I)*B(K,J)
   90                     CONTINUE
                          B(I,J) = ALPHA*TEMP
  100                 CONTINUE
```

```
 110            CONTINUE
            ELSE
                DO 140 J = 1,N
                    DO 130 I = 1,M
                        TEMP = B(I,J)
                        IF (NOUNIT) TEMP = TEMP*A(I,I)
                        DO 120 K = I + 1,M
                            TEMP = TEMP + A(K,I)*B(K,J)
 120                    CONTINUE
                        B(I,J) = ALPHA*TEMP
 130                CONTINUE
 140            CONTINUE
            END IF
        END IF
    ELSE
        IF (LSAME(TRANSA,'N')) THEN
*
*         Form  B := alpha*B*A.
*
            IF (UPPER) THEN
                DO 180 J = N,1,-1
                    TEMP = ALPHA
                    IF (NOUNIT) TEMP = TEMP*A(J,J)
                    DO 150 I = 1,M
                        B(I,J) = TEMP*B(I,J)
 150                CONTINUE
                    DO 170 K = 1,J - 1
                        IF (A(K,J).NE.ZERO) THEN
                            TEMP = ALPHA*A(K,J)
                            DO 160 I = 1,M
                                B(I,J) = B(I,J) + TEMP*B(I,K)
 160                        CONTINUE
                        END IF
 170                CONTINUE
 180            CONTINUE
            ELSE
                DO 220 J = 1,N
                    TEMP = ALPHA
                    IF (NOUNIT) TEMP = TEMP*A(J,J)
                    DO 190 I = 1,M
                        B(I,J) = TEMP*B(I,J)
 190                CONTINUE
                    DO 210 K = J + 1,N
                        IF (A(K,J).NE.ZERO) THEN
                            TEMP = ALPHA*A(K,J)
                            DO 200 I = 1,M
                                B(I,J) = B(I,J) + TEMP*B(I,K)
 200                        CONTINUE
                        END IF
 210                CONTINUE
```

```
  220             CONTINUE
              END IF
          ELSE
*
*           Form  B := alpha*B*A**T.
*
              IF (UPPER) THEN
                  DO 260 K = 1,N
                      DO 240 J = 1,K - 1
                          IF (A(J,K).NE.ZERO) THEN
                              TEMP = ALPHA*A(J,K)
                              DO 230 I = 1,M
                                  B(I,J) = B(I,J) + TEMP*B(I,K)
  230                         CONTINUE
                          END IF
  240                 CONTINUE
                      TEMP = ALPHA
                      IF (NOUNIT) TEMP = TEMP*A(K,K)
                      IF (TEMP.NE.ONE) THEN
                          DO 250 I = 1,M
                              B(I,K) = TEMP*B(I,K)
  250                     CONTINUE
                      END IF
  260             CONTINUE
              ELSE
                  DO 300 K = N,1,-1
                      DO 280 J = K + 1,N
                          IF (A(J,K).NE.ZERO) THEN
                              TEMP = ALPHA*A(J,K)
                              DO 270 I = 1,M
                                  B(I,J) = B(I,J) + TEMP*B(I,K)
  270                         CONTINUE
                          END IF
  280                 CONTINUE
                      TEMP = ALPHA
                      IF (NOUNIT) TEMP = TEMP*A(K,K)
                      IF (TEMP.NE.ONE) THEN
                          DO 290 I = 1,M
                              B(I,K) = TEMP*B(I,K)
  290                     CONTINUE
                      END IF
  300             CONTINUE
              END IF
          END IF
      END IF
*
      RETURN
*
*     End of DTRMM .
*
```

```
      END
```

———-

15.5.4 lisp code

— BLAS 3 dtrmm —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (double-float 1.0 1.0) one)
           (type (double-float 0.0 0.0) zero))
  (defun dtrmm (side uplo transa diag m n alpha a lda b ldb$)
    (declare (type (simple-array double-float (*)) b a)
             (type (double-float) alpha)
             (type fixnum ldb$ lda n m)
             (type character diag transa uplo side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (transa character transa-%data% transa-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%))
      (prog ((temp 0.0) (i 0) (info 0) (j 0) (k 0) (nrowa 0) (lside nil)
             (nounit nil) (upper nil))
        (declare (type (double-float) temp)
                 (type fixnum i info j k nrowa)
                 (type (member t nil) lside nounit upper))
        (setf lside (char-equal side #\L))
        (cond
          (lside
           (setf nrowa m))
          (t
           (setf nrowa n)))
        (setf nounit (char-equal diag #\N))
        (setf upper (char-equal uplo #\U))
        (setf info 0)
        (cond
          ((and (not lside) (not (char-equal side #\R)))
           (setf info 1))
          ((and (not upper) (not (char-equal uplo #\L)))
           (setf info 2))
          ((and (not (char-equal transa #\N))
                (not (char-equal transa #\T))
                (not (char-equal transa #\C)))
           (setf info 3))
          ((and (not (char-equal diag #\U)) (not (char-equal diag #\N)))
```

```
    (setf info 4))
   ((< m 0)
    (setf info 5))
   ((< n 0)
    (setf info 6))
   ((< lda (max (the fixnum 1) (the fixnum nrowa)))
    (setf info 9))
   ((< ldb$ (max (the fixnum 1) (the fixnum m)))
    (setf info 11)))
 (cond
   ((/= info 0)
    (error
     " ** On entry to ~a parameter number ~a had an illegal value~%"
     "DTRMM" info)
    (go end_label)))
 (if (= n 0) (go end_label))
 (cond
   ((= alpha zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data%
                                 (i j)
                                 ((1 ldb$) (1 *))
                                 b-%offset%)
                  zero)))))
    (go end_label)))
 (cond
   (lside
    (cond
      ((char-equal transa #\N)
       (cond
         (upper
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                            ((> k m) nil)
                (tagbody
                  (cond
                    ((/= (f2cl-lib:fref b (k j) ((1 ldb$) (1 *))) zero)
                     (setf temp
                             (* alpha
                                (f2cl-lib:fref b-%data%
                                               (k j)
                                               ((1 ldb$) (1 *))
                                               b-%offset%)))
```

```
                    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                  ((> i
                                      (f2cl-lib:int-add k
                                                        (f2cl-lib:int-sub
                                                         1)))
                                   nil)
                      (tagbody
                        (setf (f2cl-lib:fref b-%data%
                                             (i j)
                                             ((1 ldb$) (1 *))
                                             b-%offset%)
                              (+
                               (f2cl-lib:fref b-%data%
                                              (i j)
                                              ((1 ldb$) (1 *))
                                              b-%offset%)
                               (* temp
                                  (f2cl-lib:fref a-%data%
                                                 (i k)
                                                 ((1 lda) (1 *))
                                                 a-%offset%))))))
                    (if nounit
                        (setf temp
                              (* temp
                                 (f2cl-lib:fref a-%data%
                                                (k k)
                                                ((1 lda) (1 *))
                                                a-%offset%))))
                    (setf (f2cl-lib:fref b-%data%
                                         (k j)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)
                          temp))))))))
            (t
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (f2cl-lib:fdo (k m
                                (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                               ((> k 1) nil)
                   (tagbody
                     (cond
                       ((/= (f2cl-lib:fref b (k j) ((1 ldb$) (1 *))) zero)
                        (setf temp
                              (* alpha
                                 (f2cl-lib:fref b-%data%
                                                (k j)
                                                ((1 ldb$) (1 *))
                                                b-%offset%)))
                        (setf (f2cl-lib:fref b-%data%
```

```
                                        (k j)
                                        ((1 ldb$) (1 *))
                                        b-%offset%)
                              temp)
                      (if nounit
                          (setf (f2cl-lib:fref b-%data%
                                               (k j)
                                               ((1 ldb$) (1 *))
                                               b-%offset%)
                                (*
                                 (f2cl-lib:fref b-%data%
                                                (k j)
                                                ((1 ldb$) (1 *))
                                                b-%offset%)
                                 (f2cl-lib:fref a-%data%
                                                (k k)
                                                ((1 lda) (1 *))
                                                a-%offset%))))
                      (f2cl-lib:fdo (i (f2cl-lib:int-add k 1)
                                     (f2cl-lib:int-add i 1))
                                    ((> i m) nil)
                        (tagbody
                          (setf (f2cl-lib:fref b-%data%
                                               (i j)
                                               ((1 ldb$) (1 *))
                                               b-%offset%)
                                (+
                                 (f2cl-lib:fref b-%data%
                                                (i j)
                                                ((1 ldb$) (1 *))
                                                b-%offset%)
                                 (* temp
                                    (f2cl-lib:fref a-%data%
                                                   (i k)
                                                   ((1 lda) (1 *))
                                                   a-%offset%))))))))))))))))
                (t
                 (cond
                   (upper
                    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                                  ((> j n) nil)
                      (tagbody
                        (f2cl-lib:fdo (i m
                                       (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                                      ((> i 1) nil)
                          (tagbody
                            (setf temp
                                  (f2cl-lib:fref b-%data%
                                                 (i j)
                                                 ((1 ldb$) (1 *))
```

```
                                b-%offset%))
            (if nounit
                (setf temp
                        (* temp
                           (f2cl-lib:fref a-%data%
                                          (i i)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
            (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                          ((> k
                              (f2cl-lib:int-add i
                                                (f2cl-lib:int-sub
                                                 1)))
                           nil)
              (tagbody
                (setf temp
                        (+ temp
                           (*
                            (f2cl-lib:fref a-%data%
                                           (k i)
                                           ((1 lda) (1 *))
                                           a-%offset%)
                            (f2cl-lib:fref b-%data%
                                           (k j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%))))))
            (setf (f2cl-lib:fref b-%data%
                                 (i j)
                                 ((1 ldb$) (1 *))
                                 b-%offset%)
                  (* alpha temp)))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i m) nil)
       (tagbody
         (setf temp
                 (f2cl-lib:fref b-%data%
                                (i j)
                                ((1 ldb$) (1 *))
                                b-%offset%))
         (if nounit
             (setf temp
                     (* temp
                        (f2cl-lib:fref a-%data%
                                       (i i)
                                       ((1 lda) (1 *))
                                       a-%offset%))))
```

```
                    (f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
                                   (f2cl-lib:int-add k 1))
                                  ((> k m) nil)
                      (tagbody
                        (setf temp
                                (+ temp
                                   (*
                                    (f2cl-lib:fref a-%data%
                                                   (k i)
                                                   ((1 lda) (1 *))
                                                   a-%offset%)
                                    (f2cl-lib:fref b-%data%
                                                   (k j)
                                                   ((1 ldb$) (1 *))
                                                   b-%offset%))))))
                    (setf (f2cl-lib:fref b-%data%
                                         (i j)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)
                          (* alpha temp)))))))))))
(t
 (cond
   ((char-equal transa #\N)
    (cond
      (upper
       (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     ((> j 1) nil)
         (tagbody
           (setf temp alpha)
           (if nounit
               (setf temp
                       (* temp
                          (f2cl-lib:fref a-%data%
                                         (j j)
                                         ((1 lda) (1 *))
                                         a-%offset%))))
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf (f2cl-lib:fref b-%data%
                                    (i j)
                                    ((1 ldb$) (1 *))
                                    b-%offset%)
                       (* temp
                          (f2cl-lib:fref b-%data%
                                         (i j)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)))))
           (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                         ((> k
```

```
                                (f2cl-lib:int-add j
                                                  (f2cl-lib:int-sub 1)))
                            nil)
                  (tagbody
                    (cond
                      ((/= (f2cl-lib:fref a (k j) ((1 lda) (1 *))) zero)
                       (setf temp
                               (* alpha
                                  (f2cl-lib:fref a-%data%
                                                 (k j)
                                                 ((1 lda) (1 *))
                                                 a-%offset%)))
                       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                     ((> i m) nil)
                         (tagbody
                           (setf (f2cl-lib:fref b-%data%
                                                (i j)
                                                ((1 ldb$) (1 *))
                                                b-%offset%)
                                 (+
                                  (f2cl-lib:fref b-%data%
                                                 (i j)
                                                 ((1 ldb$) (1 *))
                                                 b-%offset%)
                                  (* temp
                                     (f2cl-lib:fref b-%data%
                                                    (i k)
                                                    ((1 ldb$) (1 *))
                                                    b-%offset%))))))))))
            (t
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (setf temp alpha)
                 (if nounit
                     (setf temp
                             (* temp
                                (f2cl-lib:fref a-%data%
                                               (j j)
                                               ((1 lda) (1 *))
                                               a-%offset%))))
                 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                               ((> i m) nil)
                   (tagbody
                     (setf (f2cl-lib:fref b-%data%
                                          (i j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)
                             (* temp
                                (f2cl-lib:fref b-%data%
```

```
                                        (i j)
                                        ((1 ldb$) (1 *))
                                        b-%offset%)))))
            (f2cl-lib:fdo (k (f2cl-lib:int-add j 1)
                           (f2cl-lib:int-add k 1))
                          ((> k n) nil)
              (tagbody
                (cond
                  ((/= (f2cl-lib:fref a (k j) ((1 lda) (1 *))) zero)
                   (setf temp
                           (* alpha
                              (f2cl-lib:fref a-%data%
                                             (k j)
                                             ((1 lda) (1 *))
                                             a-%offset%)))
                   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                 ((> i m) nil)
                     (tagbody
                       (setf (f2cl-lib:fref b-%data%
                                            (i j)
                                            ((1 ldb$) (1 *))
                                            b-%offset%)
                             (+
                              (f2cl-lib:fref b-%data%
                                             (i j)
                                             ((1 ldb$) (1 *))
                                             b-%offset%)
                              (* temp
                                 (f2cl-lib:fref b-%data%
                                            (i k)
                                            ((1 ldb$) (1 *))
                                            b-%offset%))))))))))))
(t
 (cond
   (upper
    (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                  ((> k n) nil)
      (tagbody
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j
                          (f2cl-lib:int-add k
                                            (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (cond
              ((/= (f2cl-lib:fref a (j k) ((1 lda) (1 *))) zero)
               (setf temp
                       (* alpha
                          (f2cl-lib:fref a-%data%
                                         (j k)
```

```
                                        ((1 lda) (1 *))
                                        a-%offset%)))
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i m) nil)
                  (tagbody
                    (setf (f2cl-lib:fref b-%data%
                                         (i j)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)
                          (+
                           (f2cl-lib:fref b-%data%
                                          (i j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)
                           (* temp
                              (f2cl-lib:fref b-%data%
                                             (i k)
                                             ((1 ldb$) (1 *))
                                             b-%offset%))))))))))
     (setf temp alpha)
     (if nounit
         (setf temp
                 (* temp
                    (f2cl-lib:fref a-%data%
                                   (k k)
                                   ((1 lda) (1 *))
                                   a-%offset%))))
     (cond
       ((/= temp one)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data%
                                 (i k)
                                 ((1 ldb$) (1 *))
                                 b-%offset%)
                  (* temp
                     (f2cl-lib:fref b-%data%
                                    (i k)
                                    ((1 ldb$) (1 *))
                                    b-%offset%))))))))))
(t
 (f2cl-lib:fdo (k n (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
               ((> k 1) nil)
   (tagbody
     (f2cl-lib:fdo (j (f2cl-lib:int-add k 1)
                    (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (cond
```

```
                    ((/= (f2cl-lib:fref a (j k) ((1 lda) (1 *))) zero)
                     (setf temp
                             (* alpha
                                (f2cl-lib:fref a-%data%
                                               (j k)
                                               ((1 lda) (1 *))
                                               a-%offset%)))
                     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                   ((> i m) nil)
                       (tagbody
                         (setf (f2cl-lib:fref b-%data%
                                              (i j)
                                              ((1 ldb$) (1 *))
                                              b-%offset%)
                               (+
                                (f2cl-lib:fref b-%data%
                                               (i j)
                                               ((1 ldb$) (1 *))
                                               b-%offset%)
                                (* temp
                                   (f2cl-lib:fref b-%data%
                                               (i k)
                                               ((1 ldb$) (1 *))
                                               b-%offset%)))))))))
                (setf temp alpha)
                (if nounit
                    (setf temp
                            (* temp
                               (f2cl-lib:fref a-%data%
                                              (k k)
                                              ((1 lda) (1 *))
                                              a-%offset%))))
                (cond
                  ((/= temp one)
                   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                 ((> i m) nil)
                     (tagbody
                       (setf (f2cl-lib:fref b-%data%
                                            (i k)
                                            ((1 ldb$) (1 *))
                                            b-%offset%)
                             (* temp
                                (f2cl-lib:fref b-%data%
                                            (i k)
                                            ((1 ldb$) (1 *))
                                            b-%offset%))))))))))))))
end_label
       (return (values nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

15.6 dtrsm - $op(A)X = \alpha B$ or $Xop(A) = \alpha B$, $op(A) = A$ or $op(A) = A^T$

15.6.1 Axiom unit tests

— **dtrsm.input** —

```
)set break resume
)sys rm -f dtrsm.output
)spool dtrsm.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

15.6.2 Axiom help page

— **dtrsm.help** —

```
======================================================================
dtrsm examples
======================================================================

======================================================================
Man Page Details
======================================================================

 NAME
      DTRSM - solve one of the matrix equations   op( A )*X =
      alpha*B, or X*op( A ) = alpha*B,

 SYNOPSIS
      SUBROUTINE DTRSM ( SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A,
                         LDA, B, LDB )

          CHARACTER*1  SIDE, UPLO, TRANSA, DIAG

          INTEGER      M, N, LDA, LDB
```

```
          DOUBLE       PRECISION ALPHA

          DOUBLE       PRECISION A( LDA, * ), B( LDB, * )

PURPOSE
     DTRSM  solves one of the matrix equations

     where alpha is a scalar, X and B are m by n matrices, A is a
     unit, or non-unit,  upper or lower triangular matrix  and
     op( A )  is one  of

        op( A ) = A   or   op( A ) = A'.

     The matrix X is overwritten on B.

PARAMETERS
     SIDE   - CHARACTER*1.
            On entry, SIDE specifies whether op( A ) appears on
            the left or right of X as follows:

            SIDE = 'L' or 'l'   op( A )*X = alpha*B.

            SIDE = 'R' or 'r'   X*op( A ) = alpha*B.

            Unchanged on exit.

     UPLO   - CHARACTER*1.
            On entry, UPLO specifies whether the matrix A is an
            upper or lower triangular matrix as follows:

            UPLO = 'U' or 'u'   A is an upper triangular matrix.

            UPLO = 'L' or 'l'   A is a lower triangular matrix.

            Unchanged on exit.

            TRANSA - CHARACTER*1.  On entry, TRANSA specifies the
            form of op( A ) to be used in the matrix multiplica-
            tion as follows:

            TRANSA = 'N' or 'n'   op( A ) = A.

            TRANSA = 'T' or 't'   op( A ) = A'.

            TRANSA = 'C' or 'c'   op( A ) = A'.

            Unchanged on exit.

     DIAG   - CHARACTER*1.
```

```
          On entry, DIAG specifies whether or not A is unit
          triangular as follows:

          DIAG = 'U' or 'u'   A is assumed to be unit triangu-
          lar.

          DIAG = 'N' or 'n'   A is not assumed to be unit tri-
          angular.

          Unchanged on exit.

     M      - INTEGER.
          On entry, M specifies the number of rows of B. M must
          be at least zero.  Unchanged on exit.

     N      - INTEGER.
          On entry, N specifies the number of columns of B.  N
          must be at least zero.  Unchanged on exit.

     ALPHA  - DOUBLE PRECISION.
          On entry,  ALPHA specifies the scalar  alpha. When
          alpha is zero then  A is not referenced and  B need
          not be set before entry.  Unchanged on exit.

is m
     A      -
           DOUBLE PRECISION array of DIMENSION ( LDA, k ), where k
          when  SIDE = 'L' or 'l'  and is  n  when  SIDE = 'R'
          or 'r'.  Before entry  with  UPLO = 'U' or 'u',  the
          leading  k by k upper triangular part of the array  A
          must contain the upper triangular matrix  and the
          strictly lower triangular part of A is not refer-
          enced.  Before entry  with  UPLO = 'L' or 'l',  the
          leading  k by k lower triangular part of the array  A
          must contain the lower triangular matrix  and the
          strictly upper triangular part of A is not refer-
          enced.  Note that when  DIAG = 'U' or 'u',  the diag-
          onal elements of A  are not referenced either,  but
          are assumed to be  unity.  Unchanged on exit.

     LDA    - INTEGER.
          On entry, LDA specifies the first dimension of A as
          declared in the calling (sub) program.  When  SIDE =

          'L' or 'l'  then LDA  must be at least  max( 1, m ),
          when  SIDE = 'R' or 'r' then LDA must be at least
          max( 1, n ).  Unchanged on exit.

     B      - DOUBLE PRECISION array of DIMENSION ( LDB, n ).
          Before entry,  the leading  m by n part of the array
```

```
            B must contain  the  right-hand  side  matrix  B,
            and  on exit  is overwritten by the solution matrix
            X.

      LDB    - INTEGER.
             On entry, LDB specifies the first dimension of B as
             declared in  the  calling  (sub)  program.   LDB
             must  be  at  least max( 1, m ).  Unchanged on exit.
```

———-

15.6.3 fortran code

— dtrsm.f —

```
*  ===================================================================
      SUBROUTINE DTRSM(SIDE,UPLO,TRANSA,DIAG,M,N,ALPHA,A,LDA,B,LDB)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION ALPHA
      INTEGER LDA,LDB,M,N
      CHARACTER DIAG,SIDE,TRANSA,UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A(LDA,*),B(LDB,*)
*     ..
*
*  ===================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION TEMP
      INTEGER I,INFO,J,K,NROWA
```

```
      LOGICAL LSIDE,NOUNIT,UPPER
*     ..
*     .. Parameters ..
      DOUBLE PRECISION ONE,ZERO
      PARAMETER (ONE=1.0D+0,ZERO=0.0D+0)
*     ..
*
*     Test the input parameters.
*
      LSIDE = LSAME(SIDE,'L')
      IF (LSIDE) THEN
          NROWA = M
      ELSE
          NROWA = N
      END IF
      NOUNIT = LSAME(DIAG,'N')
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.LSIDE) .AND. (.NOT.LSAME(SIDE,'R'))) THEN
          INFO = 1
      ELSE IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 2
      ELSE IF ((.NOT.LSAME(TRANSA,'N')) .AND.
     +         (.NOT.LSAME(TRANSA,'T')) .AND.
     +         (.NOT.LSAME(TRANSA,'C'))) THEN
          INFO = 3
      ELSE IF ((.NOT.LSAME(DIAG,'U')) .AND. (.NOT.LSAME(DIAG,'N'))) THEN
          INFO = 4
      ELSE IF (M.LT.0) THEN
          INFO = 5
      ELSE IF (N.LT.0) THEN
          INFO = 6
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 9
      ELSE IF (LDB.LT.MAX(1,M)) THEN
          INFO = 11
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('DTRSM ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (M.EQ.0 .OR. N.EQ.0) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
```

```
         DO 20 J = 1,N
             DO 10 I = 1,M
                 B(I,J) = ZERO
   10        CONTINUE
   20    CONTINUE
         RETURN
      END IF
*
*     Start the operations.
*
      IF (LSIDE) THEN
         IF (LSAME(TRANSA,'N')) THEN
*
*          Form  B := alpha*inv( A )*B.
*
             IF (UPPER) THEN
                 DO 60 J = 1,N
                     IF (ALPHA.NE.ONE) THEN
                         DO 30 I = 1,M
                             B(I,J) = ALPHA*B(I,J)
   30                    CONTINUE
                     END IF
                     DO 50 K = M,1,-1
                         IF (B(K,J).NE.ZERO) THEN
                             IF (NOUNIT) B(K,J) = B(K,J)/A(K,K)
                             DO 40 I = 1,K - 1
                                 B(I,J) = B(I,J) - B(K,J)*A(I,K)
   40                        CONTINUE
                         END IF
   50                CONTINUE
   60            CONTINUE
             ELSE
                 DO 100 J = 1,N
                     IF (ALPHA.NE.ONE) THEN
                         DO 70 I = 1,M
                             B(I,J) = ALPHA*B(I,J)
   70                    CONTINUE
                     END IF
                     DO 90 K = 1,M
                         IF (B(K,J).NE.ZERO) THEN
                             IF (NOUNIT) B(K,J) = B(K,J)/A(K,K)
                             DO 80 I = K + 1,M
                                 B(I,J) = B(I,J) - B(K,J)*A(I,K)
   80                        CONTINUE
                         END IF
   90                CONTINUE
  100            CONTINUE
             END IF
         ELSE
*
```

```
*          Form  B := alpha*inv( A**T )*B.
*
             IF (UPPER) THEN
                 DO 130 J = 1,N
                     DO 120 I = 1,M
                         TEMP = ALPHA*B(I,J)
                         DO 110 K = 1,I - 1
                             TEMP = TEMP - A(K,I)*B(K,J)
  110                    CONTINUE
                         IF (NOUNIT) TEMP = TEMP/A(I,I)
                         B(I,J) = TEMP
  120                 CONTINUE
  130             CONTINUE
             ELSE
                 DO 160 J = 1,N
                     DO 150 I = M,1,-1
                         TEMP = ALPHA*B(I,J)
                         DO 140 K = I + 1,M
                             TEMP = TEMP - A(K,I)*B(K,J)
  140                    CONTINUE
                         IF (NOUNIT) TEMP = TEMP/A(I,I)
                         B(I,J) = TEMP
  150                 CONTINUE
  160             CONTINUE
             END IF
         END IF
     ELSE
         IF (LSAME(TRANSA,'N')) THEN
*
*          Form  B := alpha*B*inv( A ).
*
             IF (UPPER) THEN
                 DO 210 J = 1,N
                     IF (ALPHA.NE.ONE) THEN
                         DO 170 I = 1,M
                             B(I,J) = ALPHA*B(I,J)
  170                    CONTINUE
                     END IF
                     DO 190 K = 1,J - 1
                         IF (A(K,J).NE.ZERO) THEN
                             DO 180 I = 1,M
                                 B(I,J) = B(I,J) - A(K,J)*B(I,K)
  180                        CONTINUE
                         END IF
  190                 CONTINUE
                     IF (NOUNIT) THEN
                         TEMP = ONE/A(J,J)
                         DO 200 I = 1,M
                             B(I,J) = TEMP*B(I,J)
  200                    CONTINUE
```

```
                    END IF
  210            CONTINUE
             ELSE
                 DO 260 J = N,1,-1
                     IF (ALPHA.NE.ONE) THEN
                         DO 220 I = 1,M
                             B(I,J) = ALPHA*B(I,J)
  220                    CONTINUE
                     END IF
                     DO 240 K = J + 1,N
                         IF (A(K,J).NE.ZERO) THEN
                             DO 230 I = 1,M
                                 B(I,J) = B(I,J) - A(K,J)*B(I,K)
  230                        CONTINUE
                         END IF
  240                CONTINUE
                     IF (NOUNIT) THEN
                         TEMP = ONE/A(J,J)
                         DO 250 I = 1,M
                             B(I,J) = TEMP*B(I,J)
  250                    CONTINUE
                     END IF
  260            CONTINUE
             END IF
         ELSE
*
*          Form  B := alpha*B*inv( A**T ).
*
             IF (UPPER) THEN
                 DO 310 K = N,1,-1
                     IF (NOUNIT) THEN
                         TEMP = ONE/A(K,K)
                         DO 270 I = 1,M
                             B(I,K) = TEMP*B(I,K)
  270                    CONTINUE
                     END IF
                     DO 290 J = 1,K - 1
                         IF (A(J,K).NE.ZERO) THEN
                             TEMP = A(J,K)
                             DO 280 I = 1,M
                                 B(I,J) = B(I,J) - TEMP*B(I,K)
  280                        CONTINUE
                         END IF
  290                CONTINUE
                     IF (ALPHA.NE.ONE) THEN
                         DO 300 I = 1,M
                             B(I,K) = ALPHA*B(I,K)
  300                    CONTINUE
                     END IF
  310            CONTINUE
```

```
            ELSE
                DO 360 K = 1,N
                    IF (NOUNIT) THEN
                        TEMP = ONE/A(K,K)
                        DO 320 I = 1,M
                            B(I,K) = TEMP*B(I,K)
  320                   CONTINUE
                    END IF
                    DO 340 J = K + 1,N
                        IF (A(J,K).NE.ZERO) THEN
                            TEMP = A(J,K)
                            DO 330 I = 1,M
                                B(I,J) = B(I,J) - TEMP*B(I,K)
  330                       CONTINUE
                        END IF
  340               CONTINUE
                    IF (ALPHA.NE.ONE) THEN
                        DO 350 I = 1,M
                            B(I,K) = ALPHA*B(I,K)
  350                   CONTINUE
                    END IF
  360           CONTINUE
            END IF
        END IF
    END IF
*
    RETURN
*
*   End of DTRSM .
*
    END
```

———-

15.6.4 lisp code

— BLAS 3 dtrsm —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (double-float 1.0 1.0) one)
           (type (double-float 0.0 0.0) zero))
  (defun dtrsm (side uplo transa diag m n alpha a lda b ldb$)
    (declare (type (simple-array double-float (*)) b a)
             (type (double-float) alpha)
             (type fixnum ldb$ lda n m)
             (type character diag transa uplo side))
    (f2cl-lib:with-multi-array-data
```

```
  ((side character side-%data% side-%offset%)
   (uplo character uplo-%data% uplo-%offset%)
   (transa character transa-%data% transa-%offset%)
   (diag character diag-%data% diag-%offset%)
   (a double-float a-%data% a-%offset%)
   (b double-float b-%data% b-%offset%))
(prog ((temp 0.0) (i 0) (info 0) (j 0) (k 0) (nrowa 0) (lside nil)
       (nounit nil) (upper nil))
  (declare (type (double-float) temp)
           (type fixnum i info j k nrowa)
           (type (member t nil) lside nounit upper))
  (setf lside (char-equal side #\L))
  (cond
    (lside
     (setf nrowa m))
    (t
     (setf nrowa n)))
  (setf nounit (char-equal diag #\N))
  (setf upper (char-equal uplo #\U))
  (setf info 0)
  (cond
    ((and (not lside) (not (char-equal side #\R)))
     (setf info 1))
    ((and (not upper) (not (char-equal uplo #\L)))
     (setf info 2))
    ((and (not (char-equal transa #\N))
          (not (char-equal transa #\T))
          (not (char-equal transa #\C)))
     (setf info 3))
    ((and (not (char-equal diag #\U)) (not (char-equal diag #\N)))
     (setf info 4))
    ((< m 0)
     (setf info 5))
    ((< n 0)
     (setf info 6))
    ((< lda (max (the fixnum 1) (the fixnum nrowa)))
     (setf info 9))
    ((< ldb$ (max (the fixnum 1) (the fixnum m)))
     (setf info 11)))
  (cond
    ((/= info 0)
     (error
      " ** On entry to ~a parameter number ~a had an illegal value~%"
      "DTRSM" info)
     (go end_label)))
  (if (= n 0) (go end_label))
  (cond
    ((= alpha zero)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
```

```
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data%
                                 (i j)
                                 ((1 ldb$) (1 *))
                                 b-%offset%)
                  zero)))))
   (go end_label)))
(cond
  (lside
   (cond
     ((char-equal transa #\N)
      (cond
        (upper
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (cond
               ((/= alpha one)
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i m) nil)
                  (tagbody
                    (setf (f2cl-lib:fref b-%data%
                                         (i j)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)
                          (* alpha
                             (f2cl-lib:fref b-%data%
                                            (i j)
                                            ((1 ldb$) (1 *))
                                            b-%offset%)))))))
               (f2cl-lib:fdo (k m
                              (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                             ((> k 1) nil)
                 (tagbody
                   (cond
                     ((/= (f2cl-lib:fref b (k j) ((1 ldb$) (1 *))) zero)
                      (if nounit
                          (setf (f2cl-lib:fref b-%data%
                                               (k j)
                                               ((1 ldb$) (1 *))
                                               b-%offset%)
                                (/
                                 (f2cl-lib:fref b-%data%
                                                (k j)
                                                ((1 ldb$) (1 *))
                                                b-%offset%)
                                 (f2cl-lib:fref a-%data%
```

```
                                        (k k)
                                        ((1 lda) (1 *))
                                        a-%offset%))))
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i
                                  (f2cl-lib:int-add k
                                                    (f2cl-lib:int-sub
                                                     1)))
                               nil)
                  (tagbody
                    (setf (f2cl-lib:fref b-%data%
                                         (i j)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)
                          (-
                           (f2cl-lib:fref b-%data%
                                          (i j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)
                           (*
                            (f2cl-lib:fref b-%data%
                                           (k j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%)
                            (f2cl-lib:fref a-%data%
                                      (i k)
                                      ((1 lda) (1 *))
                                      a-%offset%))))))))))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((/= alpha one)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data%
                                 (i j)
                                 ((1 ldb$) (1 *))
                                 b-%offset%)
                  (* alpha
                     (f2cl-lib:fref b-%data%
                                    (i j)
                                    ((1 ldb$) (1 *))
                                    b-%offset%)))))))
     (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                   ((> k m) nil)
       (tagbody
         (cond
```

```
                ((/= (f2cl-lib:fref b (k j) ((1 ldb$) (1 *))) zero)
                 (if nounit
                     (setf (f2cl-lib:fref b-%data%
                                          (k j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)
                           (/
                            (f2cl-lib:fref b-%data%
                                           (k j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%)
                            (f2cl-lib:fref a-%data%
                                           (k k)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
                 (f2cl-lib:fdo (i (f2cl-lib:int-add k 1)
                                (f2cl-lib:int-add i 1))
                               ((> i m) nil)
                   (tagbody
                     (setf (f2cl-lib:fref b-%data%
                                          (i j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)
                           (-
                            (f2cl-lib:fref b-%data%
                                           (i j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%)
                            (*
                             (f2cl-lib:fref b-%data%
                                            (k j)
                                            ((1 ldb$) (1 *))
                                            b-%offset%)
                             (f2cl-lib:fref a-%data%
                                            (i k)
                                            ((1 lda) (1 *))
                                            a-%offset%))))))))))))))))
(t
 (cond
   (upper
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf temp
                    (* alpha
                       (f2cl-lib:fref b-%data%
                                      (i j)
```

```
                                  ((1 ldb$) (1 *))
                                  b-%offset%)))
          (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                        ((> k
                            (f2cl-lib:int-add i
                                              (f2cl-lib:int-sub
                                               1)))
                         nil)
            (tagbody
              (setf temp
                      (- temp
                         (*
                          (f2cl-lib:fref a-%data%
                                         (k i)
                                         ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref b-%data%
                                         (k j)
                                         ((1 ldb$) (1 *))
                                         b-%offset%))))))
          (if nounit
              (setf temp
                      (/ temp
                         (f2cl-lib:fref a-%data%
                                        (i i)
                                        ((1 lda) (1 *))
                                        a-%offset%))))
          (setf (f2cl-lib:fref b-%data%
                               (i j)
                               ((1 ldb$) (1 *))
                               b-%offset%)
                temp))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i m
                    (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                   ((> i 1) nil)
       (tagbody
         (setf temp
                 (* alpha
                    (f2cl-lib:fref b-%data%
                                   (i j)
                                   ((1 ldb$) (1 *))
                                   b-%offset%)))
         (f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
                          (f2cl-lib:int-add k 1))
                       ((> k m) nil)
           (tagbody
```

```
                    (setf temp
                            (- temp
                               (*
                                (f2cl-lib:fref a-%data%
                                               (k i)
                                               ((1 lda) (1 *))
                                               a-%offset%)
                                (f2cl-lib:fref b-%data%
                                               (k j)
                                               ((1 ldb$) (1 *))
                                               b-%offset%))))))
                (if nounit
                    (setf temp
                            (/ temp
                               (f2cl-lib:fref a-%data%
                                              (i i)
                                              ((1 lda) (1 *))
                                              a-%offset%))))
                (setf (f2cl-lib:fref b-%data%
                                     (i j)
                                     ((1 ldb$) (1 *))
                                     b-%offset%)
                      temp)))))))))
(t
 (cond
   ((char-equal transa #\N)
    (cond
      (upper
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (cond
             ((/= alpha one)
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i m) nil)
                (tagbody
                  (setf (f2cl-lib:fref b-%data%
                                       (i j)
                                       ((1 ldb$) (1 *))
                                       b-%offset%)
                        (* alpha
                           (f2cl-lib:fref b-%data%
                                          (i j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)))))))
           (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                         ((> k
                             (f2cl-lib:int-add j
                                               (f2cl-lib:int-sub 1)))
                          nil)
```

```
            (tagbody
              (cond
                ((/= (f2cl-lib:fref a (k j) ((1 lda) (1 *))) zero)
                 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                               ((> i m) nil)
                   (tagbody
                     (setf (f2cl-lib:fref b-%data%
                                          (i j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)
                           (-
                            (f2cl-lib:fref b-%data%
                                           (i j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%)
                            (*
                             (f2cl-lib:fref a-%data%
                                            (k j)
                                            ((1 lda) (1 *))
                                            a-%offset%)
                             (f2cl-lib:fref b-%data%
                                            (i k)
                                            ((1 ldb$) (1 *))
                                            b-%offset%))))))))))
        (cond
          (nounit
           (setf temp
                 (/ one
                    (f2cl-lib:fref a-%data%
                                   (j j)
                                   ((1 lda) (1 *))
                                   a-%offset%)))
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf (f2cl-lib:fref b-%data%
                                    (i j)
                                    ((1 ldb$) (1 *))
                                    b-%offset%)
                     (* temp
                        (f2cl-lib:fref b-%data%
                                       (i j)
                                       ((1 ldb$) (1 *))
                                       b-%offset%))))))))))
    (t
     (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   ((> j 1) nil)
       (tagbody
         (cond
           ((/= alpha one)
```

```
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i m) nil)
     (tagbody
       (setf (f2cl-lib:fref b-%data%
                            (i j)
                            ((1 ldb$) (1 *))
                            b-%offset%)
               (* alpha
                  (f2cl-lib:fref b-%data%
                                 (i j)
                                 ((1 ldb$) (1 *))
                                 b-%offset%)))))))
(f2cl-lib:fdo (k (f2cl-lib:int-add j 1)
               (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (cond
      ((/= (f2cl-lib:fref a (k j) ((1 lda) (1 *))) zero)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf (f2cl-lib:fref b-%data%
                                (i j)
                                ((1 ldb$) (1 *))
                                b-%offset%)
                 (-
                  (f2cl-lib:fref b-%data%
                                 (i j)
                                 ((1 ldb$) (1 *))
                                 b-%offset%)
                  (*
                   (f2cl-lib:fref a-%data%
                                  (k j)
                                  ((1 lda) (1 *))
                                  a-%offset%)
                   (f2cl-lib:fref b-%data%
                                  (i k)
                                  ((1 ldb$) (1 *))
                                  b-%offset%))))))))))
(cond
  (nounit
   (setf temp
           (/ one
              (f2cl-lib:fref a-%data%
                             (j j)
                             ((1 lda) (1 *))
                             a-%offset%)))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i m) nil)
     (tagbody
```

```
                    (setf (f2cl-lib:fref b-%data%
                                         (i j)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)
                          (* temp
                             (f2cl-lib:fref b-%data%
                                            (i j)
                                            ((1 ldb$) (1 *))
                                            b-%offset%)))))))))))
(t
 (cond
   (upper
    (f2cl-lib:fdo (k n (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                  ((> k 1) nil)
      (tagbody
        (cond
          (nounit
           (setf temp
                   (/ one
                      (f2cl-lib:fref a-%data%
                                     (k k)
                                     ((1 lda) (1 *))
                                     a-%offset%)))
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf (f2cl-lib:fref b-%data%
                                    (i k)
                                    ((1 ldb$) (1 *))
                                    b-%offset%)
                     (* temp
                        (f2cl-lib:fref b-%data%
                                       (i k)
                                       ((1 ldb$) (1 *))
                                       b-%offset%)))))))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j
                          (f2cl-lib:int-add k
                                            (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (cond
              ((/= (f2cl-lib:fref a (j k) ((1 lda) (1 *))) zero)
               (setf temp
                       (f2cl-lib:fref a-%data%
                                      (j k)
                                      ((1 lda) (1 *))
                                      a-%offset%))
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i m) nil)
```

```
                    (tagbody
                      (setf (f2cl-lib:fref b-%data%
                                           (i j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%)
                            (-
                             (f2cl-lib:fref b-%data%
                                            (i j)
                                            ((1 ldb$) (1 *))
                                            b-%offset%)
                             (* temp
                                (f2cl-lib:fref b-%data%
                                               (i k)
                                               ((1 ldb$) (1 *))
                                               b-%offset%))))))))))
        (cond
          ((/= alpha one)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf (f2cl-lib:fref b-%data%
                                    (i k)
                                    ((1 ldb$) (1 *))
                                    b-%offset%)
                     (* alpha
                        (f2cl-lib:fref b-%data%
                                       (i k)
                                       ((1 ldb$) (1 *))
                                       b-%offset%))))))))))
(t
 (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
               ((> k n) nil)
   (tagbody
     (cond
       (nounit
        (setf temp
                (/ one
                   (f2cl-lib:fref a-%data%
                                  (k k)
                                  ((1 lda) (1 *))
                                  a-%offset%)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data%
                                 (i k)
                                 ((1 ldb$) (1 *))
                                 b-%offset%)
                  (* temp
                     (f2cl-lib:fref b-%data%
```

```
                                        (i k)
                                        ((1 ldb$) (1 *))
                                        b-%offset%)))))))
                (f2cl-lib:fdo (j (f2cl-lib:int-add k 1)
                               (f2cl-lib:int-add j 1))
                              ((> j n) nil)
                  (tagbody
                    (cond
                      ((/= (f2cl-lib:fref a (j k) ((1 lda) (1 *))) zero)
                       (setf temp
                               (f2cl-lib:fref a-%data%
                                              (j k)
                                              ((1 lda) (1 *))
                                              a-%offset%))
                       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                     ((> i m) nil)
                         (tagbody
                           (setf (f2cl-lib:fref b-%data%
                                                (i j)
                                                ((1 ldb$) (1 *))
                                                b-%offset%)
                                 (-
                                  (f2cl-lib:fref b-%data%
                                                 (i j)
                                                 ((1 ldb$) (1 *))
                                                 b-%offset%)
                                  (* temp
                                     (f2cl-lib:fref b-%data%
                                                    (i k)
                                                    ((1 ldb$) (1 *))
                                                    b-%offset%))))))))))
                (cond
                  ((/= alpha one)
                   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                 ((> i m) nil)
                     (tagbody
                       (setf (f2cl-lib:fref b-%data%
                                            (i k)
                                            ((1 ldb$) (1 *))
                                            b-%offset%)
                             (* alpha
                                (f2cl-lib:fref b-%data%
                                               (i k)
                                               ((1 ldb$) (1 *))
                                               b-%offset%))))))))))))))))
end_label
       (return (values nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

Chapter 16

BLAS Level 3 Complex Matrix-Matrix Operations

16.1 cgemm - $\alpha op(A)op(B)+\beta C$, $op(X) = X$ or $op(X) = X^T$ or $op(X) = X^H$

16.1.1 Axiom unit tests

— **cgemm.input** —

```
)set break resume
)sys rm -f cgemm.output
)spool cgemm.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

16.1.2 Axiom help page

— **cgemm.help** —

```
======================================================================
cgemm examples
```

```
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgemm (TRANSA, TRANSB, M, N, K, ALPHA, A, LDA, B, LDB, BETA,
           C, LDC)
           CGEMM

Function/Subroutine Documentation
   subroutine cgemm (character TRANSA, character TRANSB, integer M, integer N,
       integer K, complex ALPHA, complex, dimension(lda,*) A, integer LDA,
       complex, dimension(ldb,*) B, integer LDB, complex BETA, complex,
       dimension(ldc,*) C, integer LDC)
       CGEMM

Purpose:
   CGEMM  performs one of the matrix-matrix operations

   C := alpha*op( A )*op( B ) + beta*C,

   where  op( X ) is one of

   op( X ) = X   or   op( X ) = X**T   or   op( X ) = X**H,

   alpha and beta are scalars, and A, B and C are matrices, with op( A )
   an m by k matrix,  op( B )  a  k by n matrix and  C an m by n matrix.

Parameters:

 TRANSA
   TRANSA is CHARACTER*1
   On entry, TRANSA specifies the form of op( A ) to be used in
   the matrix multiplication as follows:

   TRANSA = 'N' or 'n',  op( A ) = A.

   TRANSA = 'T' or 't',  op( A ) = A**T.

   TRANSA = 'C' or 'c',  op( A ) = A**H.

 TRANSB
   TRANSB is CHARACTER*1
   On entry, TRANSB specifies the form of op( B ) to be used in
   the matrix multiplication as follows:

   TRANSB = 'N' or 'n',  op( B ) = B.
```

```
  TRANSB = 'T' or 't',  op( B ) = B**T.

  TRANSB = 'C' or 'c',  op( B ) = B**H.

M
  M is INTEGER
  On entry,  M  specifies  the number  of rows  of the  matrix
  op( A )  and of the  matrix  C.  M  must  be at least  zero.

N
  N is INTEGER
  On entry,  N  specifies the number  of columns of the matrix
  op( B ) and the number of columns of the matrix C. N must be
  at least zero.

K
  K is INTEGER
  On entry,  K  specifies  the number of columns of the matrix
  op( A ) and the number of rows of the matrix op( B ). K must
  be at least  zero.

ALPHA
  ALPHA is COMPLEX
  On entry, ALPHA specifies the scalar alpha.

A
  A is COMPLEX array of DIMENSION ( LDA, ka ), where ka is
  k  when  TRANSA = 'N' or 'n',  and is  m  otherwise.
  Before entry with  TRANSA = 'N' or 'n',  the leading  m by k
  part of the array  A  must contain the matrix  A,  otherwise
  the leading  k by m  part of the array  A  must contain  the
  matrix A.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. When  TRANSA = 'N' or 'n' then
  LDA must be at least  max( 1, m ), otherwise  LDA must be at
  least  max( 1, k ).

B
  B is COMPLEX array of DIMENSION ( LDB, kb ), where kb is
  n  when  TRANSB = 'N' or 'n',  and is  k  otherwise.
  Before entry with  TRANSB = 'N' or 'n',  the leading  k by n
  part of the array  B  must contain the matrix  B,  otherwise
  the leading  n by k  part of the array  B  must contain  the
  matrix B.

LDB
```

```
  LDB is INTEGER
  On entry, LDB specifies the first dimension of B as declared
  in the calling (sub) program. When  TRANSB = 'N' or 'n' then
  LDB must be at least  max( 1, k ), otherwise  LDB must be at
  least  max( 1, n ).

BETA
  BETA is COMPLEX
  On entry,  BETA  specifies the scalar  beta.  When  BETA  is
  supplied as zero then C need not be set on input.

C
  C is COMPLEX array of DIMENSION ( LDC, n ).
  Before entry, the leading  m by n  part of the array  C must
  contain the matrix  C,  except when  beta  is zero, in which
  case C need not be set on entry.
  On exit, the array  C  is overwritten by the  m by n  matrix
  ( alpha*op( A )*op( B ) + beta*C ).

LDC
  LDC is INTEGER
  On entry, LDC specifies the first dimension of C as declared
  in  the  calling  (sub)  program.  LDC  must  be  at  least
  max( 1, m ).
```

———-

16.1.3 fortran code

— cgemm.f —

```
*  ===================================================================
      SUBROUTINE CGEMM(TRANSA,TRANSB,M,N,K,ALPHA,A,LDA,B,LDB,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      COMPLEX ALPHA,BETA
      INTEGER K,LDA,LDB,LDC,M,N
      CHARACTER TRANSA,TRANSB
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),B(LDB,*),C(LDC,*)
*     ..
```

```
*
*  ====================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG,MAX
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP
      INTEGER I,INFO,J,L,NCOLA,NROWA,NROWB
      LOGICAL CONJA,CONJB,NOTA,NOTB
*     ..
*     .. Parameters ..
      COMPLEX ONE
      PARAMETER (ONE= (1.0E+0,0.0E+0))
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*
*     Set  NOTA  and  NOTB  as  true if  A  and  B  respectively are not
*     conjugated or transposed, set  CONJA and CONJB  as true if  A  and
*     B  respectively are to be  transposed but  not conjugated  and set
*     NROWA, NCOLA and  NROWB  as the number of rows and  columns  of  A
*     and the number of rows of  B  respectively.
*
      NOTA = LSAME(TRANSA,'N')
      NOTB = LSAME(TRANSB,'N')
      CONJA = LSAME(TRANSA,'C')
      CONJB = LSAME(TRANSB,'C')
      IF (NOTA) THEN
          NROWA = M
          NCOLA = K
      ELSE
          NROWA = K
          NCOLA = M
      END IF
      IF (NOTB) THEN
          NROWB = K
      ELSE
          NROWB = N
      END IF
*
*     Test the input parameters.
*
```

```
      INFO = 0
      IF ((.NOT.NOTA) .AND. (.NOT.CONJA) .AND.
     +    (.NOT.LSAME(TRANSA,'T'))) THEN
          INFO = 1
      ELSE IF ((.NOT.NOTB) .AND. (.NOT.CONJB) .AND.
     +         (.NOT.LSAME(TRANSB,'T'))) THEN
          INFO = 2
      ELSE IF (M.LT.0) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (K.LT.0) THEN
          INFO = 5
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 8
      ELSE IF (LDB.LT.MAX(1,NROWB)) THEN
          INFO = 10
      ELSE IF (LDC.LT.MAX(1,M)) THEN
          INFO = 13
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CGEMM ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR.
     +    (((ALPHA.EQ.ZERO).OR. (K.EQ.0)).AND. (BETA.EQ.ONE))) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
          IF (BETA.EQ.ZERO) THEN
              DO 20 J = 1,N
                  DO 10 I = 1,M
                      C(I,J) = ZERO
   10             CONTINUE
   20         CONTINUE
          ELSE
              DO 40 J = 1,N
                  DO 30 I = 1,M
                      C(I,J) = BETA*C(I,J)
   30             CONTINUE
   40         CONTINUE
          END IF
          RETURN
      END IF
*
*     Start the operations.
```

```
*
      IF (NOTB) THEN
          IF (NOTA) THEN
*
*           Form  C := alpha*A*B + beta*C.
*
              DO 90 J = 1,N
                  IF (BETA.EQ.ZERO) THEN
                      DO 50 I = 1,M
                          C(I,J) = ZERO
   50                 CONTINUE
                  ELSE IF (BETA.NE.ONE) THEN
                      DO 60 I = 1,M
                          C(I,J) = BETA*C(I,J)
   60                 CONTINUE
                  END IF
                  DO 80 L = 1,K
                      TEMP = ALPHA*B(L,J)
                      DO 70 I = 1,M
                          C(I,J) = C(I,J) + TEMP*A(I,L)
   70                 CONTINUE
   80             CONTINUE
   90         CONTINUE
          ELSE IF (CONJA) THEN
*
*           Form  C := alpha*A**H*B + beta*C.
*
              DO 120 J = 1,N
                  DO 110 I = 1,M
                      TEMP = ZERO
                      DO 100 L = 1,K
                          TEMP = TEMP + CONJG(A(L,I))*B(L,J)
  100                 CONTINUE
                      IF (BETA.EQ.ZERO) THEN
                          C(I,J) = ALPHA*TEMP
                      ELSE
                          C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                      END IF
  110             CONTINUE
  120         CONTINUE
          ELSE
*
*           Form  C := alpha*A**T*B + beta*C
*
              DO 150 J = 1,N
                  DO 140 I = 1,M
                      TEMP = ZERO
                      DO 130 L = 1,K
                          TEMP = TEMP + A(L,I)*B(L,J)
  130                 CONTINUE
```

```
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP
                    ELSE
                        C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                    END IF
 140            CONTINUE
 150        CONTINUE
        END IF
    ELSE IF (NOTA) THEN
        IF (CONJB) THEN
*
*         Form  C := alpha*A*B**H + beta*C.
*
            DO 200 J = 1,N
                IF (BETA.EQ.ZERO) THEN
                    DO 160 I = 1,M
                        C(I,J) = ZERO
 160                CONTINUE
                ELSE IF (BETA.NE.ONE) THEN
                    DO 170 I = 1,M
                        C(I,J) = BETA*C(I,J)
 170                CONTINUE
                END IF
                DO 190 L = 1,K
                    TEMP = ALPHA*CONJG(B(J,L))
                    DO 180 I = 1,M
                        C(I,J) = C(I,J) + TEMP*A(I,L)
 180                CONTINUE
 190            CONTINUE
 200        CONTINUE
        ELSE
*
*         Form  C := alpha*A*B**T + beta*C
*
            DO 250 J = 1,N
                IF (BETA.EQ.ZERO) THEN
                    DO 210 I = 1,M
                        C(I,J) = ZERO
 210                CONTINUE
                ELSE IF (BETA.NE.ONE) THEN
                    DO 220 I = 1,M
                        C(I,J) = BETA*C(I,J)
 220                CONTINUE
                END IF
                DO 240 L = 1,K
                    TEMP = ALPHA*B(J,L)
                    DO 230 I = 1,M
                        C(I,J) = C(I,J) + TEMP*A(I,L)
 230                CONTINUE
 240            CONTINUE
```

```
  250         CONTINUE
          END IF
      ELSE IF (CONJA) THEN
          IF (CONJB) THEN
*
*           Form  C := alpha*A**H*B**H + beta*C.
*
              DO 280 J = 1,N
                  DO 270 I = 1,M
                      TEMP = ZERO
                      DO 260 L = 1,K
                          TEMP = TEMP + CONJG(A(L,I))*CONJG(B(J,L))
  260                 CONTINUE
                      IF (BETA.EQ.ZERO) THEN
                          C(I,J) = ALPHA*TEMP
                      ELSE
                          C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                      END IF
  270             CONTINUE
  280         CONTINUE
          ELSE
*
*           Form  C := alpha*A**H*B**T + beta*C
*
              DO 310 J = 1,N
                  DO 300 I = 1,M
                      TEMP = ZERO
                      DO 290 L = 1,K
                          TEMP = TEMP + CONJG(A(L,I))*B(J,L)
  290                 CONTINUE
                      IF (BETA.EQ.ZERO) THEN
                          C(I,J) = ALPHA*TEMP
                      ELSE
                          C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                      END IF
  300             CONTINUE
  310         CONTINUE
          END IF
      ELSE
          IF (CONJB) THEN
*
*           Form  C := alpha*A**T*B**H + beta*C
*
              DO 340 J = 1,N
                  DO 330 I = 1,M
                      TEMP = ZERO
                      DO 320 L = 1,K
                          TEMP = TEMP + A(L,I)*CONJG(B(J,L))
  320                 CONTINUE
                      IF (BETA.EQ.ZERO) THEN
```

```
                        C(I,J) = ALPHA*TEMP
                    ELSE
                        C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                    END IF
  330            CONTINUE
  340        CONTINUE
         ELSE
*
*          Form  C := alpha*A**T*B**T + beta*C
*
             DO 370 J = 1,N
                 DO 360 I = 1,M
                     TEMP = ZERO
                     DO 350 L = 1,K
                         TEMP = TEMP + A(L,I)*B(J,L)
  350                CONTINUE
                     IF (BETA.EQ.ZERO) THEN
                         C(I,J) = ALPHA*TEMP
                     ELSE
                         C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                     END IF
  360            CONTINUE
  370        CONTINUE
         END IF
     END IF
*
     RETURN
*
*    End of CGEMM .
*
     END
```

———-

16.1.4 lisp code

— BLAS 3 cgemm —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)) (zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (ignorable one zero))
  (defun cgemm (transa transb m n k alpha a lda b ldb$ beta c ldc)
    (declare (type (array f2cl-lib:complex8 (*)) c b a)
             (type (f2cl-lib:complex8) beta alpha)
             (type (f2cl-lib:integer4) ldc ldb$ lda k n m)
             (type (string *) transb transa))
```

```
(f2cl-lib:with-multi-array-data
    ((transa character transa-%data% transa-%offset%)
     (transb character transb-%data% transb-%offset%)
     (a f2cl-lib:complex8 a-%data% a-%offset%)
     (b f2cl-lib:complex8 b-%data% b-%offset%)
     (c f2cl-lib:complex8 c-%data% c-%offset%))
  (prog ((conja nil) (conjb nil) (nota nil) (notb nil) (i 0) (info 0) (j 0)
         (l 0) (ncola 0) (nrowa 0) (nrowb 0) (temp #C(0.0 0.0)))
    (declare (type f2cl-lib:logical conja conjb nota notb)
             (type (f2cl-lib:integer4) i info j l ncola nrowa nrowb)
             (type (f2cl-lib:complex8) temp))
    (setf nota
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame transa "N")
              (declare (ignore var-1))
              (when var-0 (setf transa var-0))
              ret-val))
    (setf notb
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame transb "N")
              (declare (ignore var-1))
              (when var-0 (setf transb var-0))
              ret-val))
    (setf conja
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame transa "C")
              (declare (ignore var-1))
              (when var-0 (setf transa var-0))
              ret-val))
    (setf conjb
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame transb "C")
              (declare (ignore var-1))
              (when var-0 (setf transb var-0))
              ret-val))
    (cond (nota (setf nrowa m) (setf ncola k))
          (t (setf nrowa k) (setf ncola m)))
    (cond (notb (setf nrowb k)) (t (setf nrowb n)))
    (setf info 0)
    (cond
     ((and (not nota) (not conja)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame transa "T")
              (declare (ignore var-1))
              (when var-0 (setf transa var-0))
              ret-val)))
      (setf info 1))
     ((and (not notb) (not conjb)
           (not
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame transb "T")
          (declare (ignore var-1))
          (when var-0 (setf transb var-0))
          ret-val)))
  (setf info 2))
 ((< m 0) (setf info 3)) ((< n 0) (setf info 4))
 ((< k 0) (setf info 5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nrowa)))
  (setf info 8))
 ((< ldb$
     (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nrowb)))
  (setf info 10))
 ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info 13)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "CGEMM " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= m 0) (= n 0) (and (or (= alpha zero) (= k 0)) (= beta one)))
    (go end_label))
(cond
 ((= alpha zero)
  (cond
   ((= beta zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                 c-%offset%)
                  zero)
           label10))
       label20)))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                 c-%offset%)
                  (* beta
                     (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
```

```
                                        c-%offset%)))
            label30))
         label40))))
    (go end_label)))
  (cond
   (notb
    (cond
     (nota
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
           ((= beta zero)
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                      zero)
               label50)))
           ((/= beta one)
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                      (* beta
                         (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                        c-%offset%)))
               label60))))
          (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                        ((> l k) nil)
            (tagbody
              (setf temp
                      (* alpha
                         (f2cl-lib:fref b-%data% (l j) ((1 ldb$) (1 *))
                                        b-%offset%)))
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i m) nil)
                (tagbody
                  (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)
                        (+
                         (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                        c-%offset%)
                         (* temp
                            (f2cl-lib:fref a-%data% (i l)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
                 label70))
```

```
        label80))
    label90)))
(conja
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i m) nil)
       (tagbody
         (setf temp zero)
         (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                       ((> l k) nil)
           (tagbody
             (setf temp
                     (+ temp
                        (*
                         (f2cl-lib:conjg
                          (f2cl-lib:fref a-%data% (l i)
                                         ((1 lda) (1 *))
                                         a-%offset%))
                         (f2cl-lib:fref b-%data% (l j)
                                        ((1 ldb$) (1 *))
                                        b-%offset%))))
            label100))
         (cond
          ((= beta zero)
           (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* alpha temp)))
          (t
           (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                c-%offset%)
                 (+ (* alpha temp)
                    (* beta
                       (f2cl-lib:fref c-%data% (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%))))))
        label110))
    label120)))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i m) nil)
       (tagbody
         (setf temp zero)
         (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                       ((> l k) nil)
           (tagbody
```

```
                (setf temp
                        (+ temp
                           (*
                            (f2cl-lib:fref a-%data% (l i)
                                           ((1 lda) (1 *)) a-%offset%)
                            (f2cl-lib:fref b-%data% (l j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%))))
               label130))
             (cond
              ((= beta zero)
               (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                       (* alpha temp)))
              (t
               (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                       (+ (* alpha temp)
                          (* beta
                            (f2cl-lib:fref c-%data% (i j)
                                           ((1 ldc) (1 *))
                                           c-%offset%))))))
           label140))
       label150)))))
(nota
 (cond
  (conjb
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((= beta zero)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                  c-%offset%)
                     zero)
            label160)))
        ((/= beta one)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                  c-%offset%)
                     (* beta
                        (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)))
            label170))))
       (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
```

```
                    ((> l k) nil)
          (tagbody
            (setf temp
                    (* alpha
                       (f2cl-lib:conjg
                        (f2cl-lib:fref b-%data% (j l) ((1 ldb$) (1 *))
                                       b-%offset%))))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                      (+
                       (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)
                       (* temp
                          (f2cl-lib:fref a-%data% (i l)
                                         ((1 lda) (1 *))
                                         a-%offset%))))
                label180))
           label190))
       label200)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((= beta zero)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                  c-%offset%)
                   zero)
           label210)))
        ((/= beta one)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                  c-%offset%)
                   (* beta
                      (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)))
           label220))))
       (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                     ((> l k) nil)
         (tagbody
           (setf temp
                   (* alpha
```

```
                        (f2cl-lib:fref b-%data% (j l) ((1 ldb$) (1 *))
                                       b-%offset%)))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                      (+
                       (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)
                       (* temp
                          (f2cl-lib:fref a-%data% (i l)
                                         ((1 lda) (1 *))
                                         a-%offset%))))
                label230))
          label240))
      label250)))))
(conja
 (cond
  (conjb
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf temp zero)
           (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                         ((> l k) nil)
             (tagbody
               (setf temp
                       (+ temp
                          (*
                           (f2cl-lib:conjg
                            (f2cl-lib:fref a-%data% (l i)
                                           ((1 lda) (1 *))
                                           a-%offset%))
                           (f2cl-lib:conjg
                            (f2cl-lib:fref b-%data% (j l)
                                           ((1 ldb$) (1 *))
                                           b-%offset%)))))
               label260))
           (cond
            ((= beta zero)
             (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                  c-%offset%)
                   (* alpha temp)))
            (t
             (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                  c-%offset%)
```

```
                        (+ (* alpha temp)
                           (* beta
                              (f2cl-lib:fref c-%data% (i j)
                                             ((1 ldc) (1 *))
                                             c-%offset%))))))
            label270))
        label280)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf temp zero)
             (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                           ((> l k) nil)
               (tagbody
                 (setf temp
                         (+ temp
                            (*
                             (f2cl-lib:conjg
                              (f2cl-lib:fref a-%data% (l i)
                                             ((1 lda) (1 *))
                                             a-%offset%))
                             (f2cl-lib:fref b-%data% (j l)
                                             ((1 ldb$) (1 *))
                                             b-%offset%))))
                 label290))
             (cond
              ((= beta zero)
               (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                       (* alpha temp)))
              (t
               (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                       (+ (* alpha temp)
                          (* beta
                             (f2cl-lib:fref c-%data% (i j)
                                             ((1 ldc) (1 *))
                                             c-%offset%))))))
            label300))
        label310)))))
(t
 (cond
  (conjb
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
```

```
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf temp zero)
              (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                            ((> l k) nil)
                (tagbody
                  (setf temp
                          (+ temp
                             (*
                              (f2cl-lib:fref a-%data% (l i)
                                             ((1 lda) (1 *)) a-%offset%)
                              (f2cl-lib:conjg
                               (f2cl-lib:fref b-%data% (j l)
                                              ((1 ldb$) (1 *))
                                              b-%offset%)))))
                 label320))
              (cond
               ((= beta zero)
                (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                        (* alpha temp)))
               (t
                (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                        (+ (* alpha temp)
                           (* beta
                              (f2cl-lib:fref c-%data% (i j)
                                             ((1 ldc) (1 *))
                                             c-%offset%))))))
             label330))
         label340)))
      (t
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf temp zero)
               (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                             ((> l k) nil)
                 (tagbody
                   (setf temp
                           (+ temp
                              (*
                               (f2cl-lib:fref a-%data% (l i)
                                              ((1 lda) (1 *)) a-%offset%)
                               (f2cl-lib:fref b-%data% (j l)
                                              ((1 ldb$) (1 *))
```

```
                                                b-%offset%))))
                      label350))
                    (cond
                     ((= beta zero)
                      (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                           c-%offset%)
                              (* alpha temp)))
                     (t
                      (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                           c-%offset%)
                              (+ (* alpha temp)
                                 (* beta
                                    (f2cl-lib:fref c-%data% (i j)
                                                   ((1 ldc) (1 *))
                                                   c-%offset%))))))
                 label360))
            label370))))))
       (go end_label)
      end_label
       (return
        (values transa transb nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

16.2 chemm - hermitian $\alpha AB + \beta C$ or $\alpha BA + \beta C$

16.2.1 Axiom unit tests

— **chemm.input** —

```
)set break resume
)sys rm -f chemm.output
)spool chemm.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

16.2.2 Axiom help page

— chemm.help —

```
=====================================================================
chemm examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine chemm (SIDE, UPLO, M, N, ALPHA, A, LDA, B, LDB, BETA, C,
           LDC)
           CHEMM

Function/Subroutine Documentation
   subroutine chemm (character SIDE, character UPLO, integer M, integer N,
       complex ALPHA, complex, dimension(lda,*) A, integer LDA, complex,
       dimension(ldb,*) B, integer LDB, complex BETA, complex,
       dimension(ldc,*) C, integer LDC)
       CHEMM

Purpose:
   CHEMM  performs one of the matrix-matrix operations

   C := alpha*A*B + beta*C,

   or

   C := alpha*B*A + beta*C,

   where alpha and beta are scalars, A is an hermitian matrix and  B and
   C are m by n matrices.

Parameters:

 SIDE
   SIDE is CHARACTER*1
   On entry,  SIDE  specifies whether  the  hermitian matrix  A
   appears on the  left or right  in the  operation as follows:

   SIDE = 'L' or 'l'   C := alpha*A*B + beta*C,

   SIDE = 'R' or 'r'   C := alpha*B*A + beta*C,

 UPLO
   UPLO is CHARACTER*1
   On  entry,   UPLO  specifies  whether  the  upper  or  lower
   triangular  part  of  the  hermitian  matrix   A  is  to  be
```

```
  referenced as follows:

  UPLO = 'U' or 'u'   Only the upper triangular part of the
  hermitian matrix is to be referenced.

  UPLO = 'L' or 'l'   Only the lower triangular part of the
  hermitian matrix is to be referenced.

M
  M is INTEGER
  On entry,  M  specifies the number of rows of the matrix  C.
  M  must be at least zero.

N
  N is INTEGER
  On entry, N specifies the number of columns of the matrix C.
  N  must be at least zero.

ALPHA
  ALPHA is COMPLEX
  On entry, ALPHA specifies the scalar alpha.

A
  A is COMPLEX array of DIMENSION ( LDA, ka ), where ka is
  m  when  SIDE = 'L' or 'l'  and is n  otherwise.
  Before entry  with  SIDE = 'L' or 'l',  the  m by m  part of
  the array  A  must contain the  hermitian matrix,  such that
  when  UPLO = 'U' or 'u', the leading m by m upper triangular
  part of the array  A  must contain the upper triangular part
  of the  hermitian matrix and the  strictly  lower triangular
  part of  A  is not referenced,  and when  UPLO = 'L' or 'l',
  the leading  m by m  lower triangular part  of the  array  A
  must  contain  the  lower triangular part  of the  hermitian
  matrix and the  strictly upper triangular part of  A  is not
  referenced.

  Before entry  with  SIDE = 'R' or 'r',  the  n by n  part of
  the array  A  must contain the  hermitian matrix,  such that
  when  UPLO = 'U' or 'u', the leading n by n upper triangular
  part of the array  A  must contain the upper triangular part
  of the  hermitian matrix and the  strictly  lower triangular
  part of  A  is not referenced,  and when  UPLO = 'L' or 'l',
  the leading  n by n  lower triangular part  of the  array  A
  must  contain  the  lower triangular part  of the  hermitian
  matrix and the  strictly upper triangular part of  A  is not
  referenced.

  Note that the imaginary parts  of the diagonal elements need
  not be set, they are assumed to be zero.
```

```
LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the  calling (sub) program. When  SIDE = 'L' or 'l'  then
  LDA must be at least  max( 1, m ), otherwise  LDA must be at
  least max( 1, n ).

B
  B is COMPLEX array of DIMENSION ( LDB, n ).
  Before entry, the leading  m by n part of the array  B  must
  contain the matrix B.

LDB
  LDB is INTEGER
  On entry, LDB specifies the first dimension of B as declared
  in  the  calling  (sub)  program.  LDB  must  be  at  least
  max( 1, m ).

BETA
  BETA is COMPLEX
  On entry,  BETA  specifies the scalar  beta.  When  BETA  is
  supplied as zero then C need not be set on input.

C
  C is COMPLEX array of DIMENSION ( LDC, n ).
  Before entry, the leading  m by n  part of the array  C must
  contain the matrix  C,  except when  beta  is zero, in which
  case C need not be set on entry.
  On exit, the array  C  is overwritten by the  m by n updated
  matrix.

LDC
  LDC is INTEGER
  On entry, LDC specifies the first dimension of C as declared
  in  the  calling  (sub)  program.  LDC  must  be  at  least
  max( 1, m ).
```

———-

16.2.3 fortran code

— chemm.f —

```
*  ====================================================================
      SUBROUTINE CHEMM(SIDE,UPLO,M,N,ALPHA,A,LDA,B,LDB,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
```

```
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX ALPHA,BETA
      INTEGER LDA,LDB,LDC,M,N
      CHARACTER SIDE,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),B(LDB,*),C(LDC,*)
*     ..
*
*  =====================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG,MAX,REAL
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP1,TEMP2
      INTEGER I,INFO,J,K,NROWA
      LOGICAL UPPER
*     ..
*     .. Parameters ..
      COMPLEX ONE
      PARAMETER (ONE= (1.0E+0,0.0E+0))
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*
*     Set NROWA as the number of rows of A.
*
      IF (LSAME(SIDE,'L')) THEN
          NROWA = M
      ELSE
          NROWA = N
      END IF
      UPPER = LSAME(UPLO,'U')
*
*     Test the input parameters.
*
      INFO = 0
      IF ((.NOT.LSAME(SIDE,'L')) .AND. (.NOT.LSAME(SIDE,'R'))) THEN
```

```
         INFO = 1
      ELSE IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
         INFO = 2
      ELSE IF (M.LT.0) THEN
         INFO = 3
      ELSE IF (N.LT.0) THEN
         INFO = 4
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
         INFO = 7
      ELSE IF (LDB.LT.MAX(1,M)) THEN
         INFO = 9
      ELSE IF (LDC.LT.MAX(1,M)) THEN
         INFO = 12
      END IF
      IF (INFO.NE.0) THEN
         CALL XERBLA('CHEMM ',INFO)
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR.
     +    ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
         IF (BETA.EQ.ZERO) THEN
            DO 20 J = 1,N
               DO 10 I = 1,M
                  C(I,J) = ZERO
   10          CONTINUE
   20       CONTINUE
         ELSE
            DO 40 J = 1,N
               DO 30 I = 1,M
                  C(I,J) = BETA*C(I,J)
   30          CONTINUE
   40       CONTINUE
         END IF
         RETURN
      END IF
*
*     Start the operations.
*
      IF (LSAME(SIDE,'L')) THEN
*
*        Form  C := alpha*A*B + beta*C.
*
         IF (UPPER) THEN
```

```
            DO 70 J = 1,N
                DO 60 I = 1,M
                    TEMP1 = ALPHA*B(I,J)
                    TEMP2 = ZERO
                    DO 50 K = 1,I - 1
                        C(K,J) = C(K,J) + TEMP1*A(K,I)
                        TEMP2 = TEMP2 + B(K,J)*CONJG(A(K,I))
   50               CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = TEMP1*REAL(A(I,I)) + ALPHA*TEMP2
                    ELSE
                        C(I,J) = BETA*C(I,J) + TEMP1*REAL(A(I,I)) +
     +                           ALPHA*TEMP2
                    END IF
   60           CONTINUE
   70       CONTINUE
        ELSE
            DO 100 J = 1,N
                DO 90 I = M,1,-1
                    TEMP1 = ALPHA*B(I,J)
                    TEMP2 = ZERO
                    DO 80 K = I + 1,M
                        C(K,J) = C(K,J) + TEMP1*A(K,I)
                        TEMP2 = TEMP2 + B(K,J)*CONJG(A(K,I))
   80               CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = TEMP1*REAL(A(I,I)) + ALPHA*TEMP2
                    ELSE
                        C(I,J) = BETA*C(I,J) + TEMP1*REAL(A(I,I)) +
     +                           ALPHA*TEMP2
                    END IF
   90           CONTINUE
  100       CONTINUE
        END IF
    ELSE
*
*       Form  C := alpha*B*A + beta*C.
*
        DO 170 J = 1,N
            TEMP1 = ALPHA*REAL(A(J,J))
            IF (BETA.EQ.ZERO) THEN
                DO 110 I = 1,M
                    C(I,J) = TEMP1*B(I,J)
  110           CONTINUE
            ELSE
                DO 120 I = 1,M
                    C(I,J) = BETA*C(I,J) + TEMP1*B(I,J)
  120           CONTINUE
            END IF
            DO 140 K = 1,J - 1
```

```
                IF (UPPER) THEN
                    TEMP1 = ALPHA*A(K,J)
                ELSE
                    TEMP1 = ALPHA*CONJG(A(J,K))
                END IF
                DO 130 I = 1,M
                    C(I,J) = C(I,J) + TEMP1*B(I,K)
 130            CONTINUE
 140        CONTINUE
            DO 160 K = J + 1,N
                IF (UPPER) THEN
                    TEMP1 = ALPHA*CONJG(A(J,K))
                ELSE
                    TEMP1 = ALPHA*A(K,J)
                END IF
                DO 150 I = 1,M
                    C(I,J) = C(I,J) + TEMP1*B(I,K)
 150            CONTINUE
 160        CONTINUE
 170    CONTINUE
    END IF
*
    RETURN
*
*   End of CHEMM .
*
    END
```

———-

16.2.4 lisp code

— BLAS 3 chemm —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)) (zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (ignorable one zero))
  (defun chemm (side uplo m n alpha a lda b ldb$ beta c ldc)
    (declare (type (array f2cl-lib:complex8 (*)) c b a)
             (type (f2cl-lib:complex8) beta alpha)
             (type (f2cl-lib:integer4) ldc ldb$ lda n m)
             (type (string *) uplo side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
```

```
    (b f2cl-lib:complex8 b-%data% b-%offset%)
    (c f2cl-lib:complex8 c-%data% c-%offset%))
  (prog ((upper nil) (i 0) (info 0) (j 0) (k 0) (nrowa 0)
         (temp1 #C(0.0 0.0)) (temp2 #C(0.0 0.0)))
    (declare (type f2cl-lib:logical upper)
             (type (f2cl-lib:integer4) i info j k nrowa)
             (type (f2cl-lib:complex8) temp1 temp2))
    (cond
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame side "L")
        (declare (ignore var-1))
        (when var-0 (setf side var-0))
        ret-val)
      (setf nrowa m))
     (t (setf nrowa n)))
    (setf upper
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
    (setf info 0)
    (cond
     ((and
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "L")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "R")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val)))
      (setf info 1))
     ((and (not upper)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
      (setf info 2))
     ((< m 0) (setf info 3)) ((< n 0) (setf info 4))
     ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nrowa)))
      (setf info 7))
     ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
      (setf info 9))
```

```
 ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info 12)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "CHEMM " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
    (go end_label))
(cond
 ((= alpha zero)
  (cond
   ((= beta zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                 c-%offset%)
                  zero)
           label10))
       label20)))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                 c-%offset%)
                  (* beta
                     (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)))
           label30))
       label40))))
  (go end_label)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame side "L")
    (declare (ignore var-1))
    (when var-0 (setf side var-0))
    ret-val)
  (cond
   (upper
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
            ((> j n) nil)
(tagbody
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf temp1
              (* alpha
                 (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)))
      (setf temp2 zero)
      (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                    ((> k
                        (f2cl-lib:int-add i
                                          (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref c-%data% (k j) ((1 ldc) (1 *))
                               c-%offset%)
                  (+
                   (f2cl-lib:fref c-%data% (k j) ((1 ldc) (1 *))
                                  c-%offset%)
                   (* temp1
                      (f2cl-lib:fref a-%data% (k i)
                                     ((1 lda) (1 *))
                                     a-%offset%))))
          (setf temp2
                  (+ temp2
                     (*
                      (f2cl-lib:fref b-%data% (k j)
                                     ((1 ldb$) (1 *)) b-%offset%)
                      (f2cl-lib:conjg
                       (f2cl-lib:fref a-%data% (k i)
                                      ((1 lda) (1 *))
                                      a-%offset%)))))
         label50))
      (cond
       ((= beta zero)
        (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                             c-%offset%)
                (+
                 (* temp1
                    (f2cl-lib:freal
                     (f2cl-lib:fref a-%data% (i i)
                                    ((1 lda) (1 *)) a-%offset%)))
                 (* alpha temp2))))
       (t
        (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                             c-%offset%)
                (+
                 (* beta
```

```
                          (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                         c-%offset%))
                       (* temp1
                          (f2cl-lib:freal
                           (f2cl-lib:fref a-%data% (i i)
                                          ((1 lda) (1 *)) a-%offset%)))
                       (* alpha temp2)))))
            label60))
        label70)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                       ((> i 1) nil)
           (tagbody
             (setf temp1
                     (* alpha
                        (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)))
             (setf temp2 zero)
             (f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
                            (f2cl-lib:int-add k 1))
                           ((> k m) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data% (k j) ((1 ldc) (1 *))
                                      c-%offset%)
                         (+
                          (f2cl-lib:fref c-%data% (k j) ((1 ldc) (1 *))
                                         c-%offset%)
                          (* temp1
                             (f2cl-lib:fref a-%data% (k i)
                                            ((1 lda) (1 *))
                                            a-%offset%))))
                 (setf temp2
                         (+ temp2
                            (*
                             (f2cl-lib:fref b-%data% (k j)
                                            ((1 ldb$) (1 *)) b-%offset%)
                             (f2cl-lib:conjg
                              (f2cl-lib:fref a-%data% (k i)
                                             ((1 lda) (1 *))
                                             a-%offset%)))))
                 label80))
             (cond
              ((= beta zero)
               (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                       (+
                        (* temp1
```

```
                          (f2cl-lib:freal
                           (f2cl-lib:fref a-%data% (i i)
                                          ((1 lda) (1 *)) a-%offset%)))
                       (* alpha temp2))))
             (t
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                   c-%offset%)
                     (+
                      (* beta
                         (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                        c-%offset%))
                      (* temp1
                         (f2cl-lib:freal
                          (f2cl-lib:fref a-%data% (i i)
                                         ((1 lda) (1 *)) a-%offset%)))
                      (* alpha temp2)))))
          label90))
      label100)))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf temp1
             (* alpha
                (f2cl-lib:freal
                 (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                a-%offset%))))
     (cond
      ((= beta zero)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* temp1
                    (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)))
          label110)))
      (t
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                c-%offset%)
                 (+
                  (* beta
                     (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%))
                  (* temp1
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
```

```
                                  b-%offset%))))
     label120))))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (cond
     (upper
      (setf temp1
              (* alpha
                 (f2cl-lib:fref a-%data% (k j) ((1 lda) (1 *))
                                a-%offset%))))
     (t
      (setf temp1
              (* alpha
                 (f2cl-lib:conjg
                  (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                 a-%offset%))))))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                             c-%offset%)
                (+
                 (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* temp1
                    (f2cl-lib:fref b-%data% (i k)
                                   ((1 ldb$) (1 *))
                                   b-%offset%))))
       label130))
   label140))
(f2cl-lib:fdo (k (f2cl-lib:int-add j 1) (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (cond
     (upper
      (setf temp1
              (* alpha
                 (f2cl-lib:conjg
                  (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                 a-%offset%)))))
     (t
      (setf temp1
              (* alpha
                 (f2cl-lib:fref a-%data% (k j) ((1 lda) (1 *))
                                a-%offset%)))))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
```

```
                    (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                         c-%offset%)
                            (+
                             (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                            c-%offset%)
                             (* temp1
                                (f2cl-lib:fref b-%data% (i k)
                                               ((1 ldb$) (1 *))
                                               b-%offset%))))
                  label150))
              label160))
          label170))))
     (go end_label)
    end_label
     (return (values side uplo nil nil nil nil nil nil nil nil nil nil))))))
```

———-

16.3 cher2k - hermitian rank 2 $\alpha AB^H + conjg(\alpha)BA^H + \beta C$ or $\alpha A^H B + conjg(\alpha)B^H A + \beta C$

16.3.1 Axiom unit tests

— **cher2k.input** —

```
)set break resume
)sys rm -f cher2k.output
)spool cher2k.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

16.3.2 Axiom help page

— **cher2k.help** —

```
=====================================================================
cher2k examples
```

```
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cher2k (UPLO, TRANS, N, K, ALPHA, A, LDA, B, LDB, BETA, C,
           LDC)
           CHER2K

Function/Subroutine Documentation
   subroutine cher2k (character UPLO, character TRANS, integer N, integer K,
       complex ALPHA, complex, dimension(lda,*) A, integer LDA, complex,
       dimension(ldb,*) B, integer LDB, real BETA, complex, dimension(ldc,*)
       C, integer LDC)
       CHER2K

Purpose:
   CHER2K  performs one of the hermitian rank 2k operations

   C := alpha*A*B**H + conjg( alpha )*B*A**H + beta*C,

   or

   C := alpha*A**H*B + conjg( alpha )*B**H*A + beta*C,

   where  alpha and beta  are scalars with  beta  real,  C is an  n by n
   hermitian matrix and  A and B  are  n by k matrices in the first case
   and  k by n  matrices in the second case.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On  entry,   UPLO  specifies  whether  the  upper  or  lower
   triangular  part  of the  array  C  is to be  referenced  as follows:

   UPLO = 'U' or 'u'   Only the  upper triangular part of  C
   is to be referenced.

   UPLO = 'L' or 'l'   Only the  lower triangular part of  C
   is to be referenced.

 TRANS

   TRANS is CHARACTER*1
   On entry,  TRANS  specifies the operation to be performed as follows:
```

```
  TRANS = 'N' or 'n'
    C := alpha*A*B**H + conjg( alpha )*B*A**H + beta*C.

  TRANS = 'C' or 'c'
    C := alpha*A**H*B + conjg( alpha )*B**H*A + beta*C.

N
  N is INTEGER
  On entry,  N specifies the order of the matrix C.  N must be
  at least zero.

K
  K is INTEGER
  On entry with  TRANS = 'N' or 'n',  K  specifies  the number
  of  columns  of the  matrices  A and B,  and on  entry  with
  TRANS = 'C' or 'c',  K  specifies  the number of rows of the
  matrices  A and B.  K must be at least zero.

ALPHA
  ALPHA is COMPLEX
  On entry, ALPHA specifies the scalar alpha.

A
  A is COMPLEX array of DIMENSION ( LDA, ka ), where ka is
  k  when  TRANS = 'N' or 'n',  and is  n  otherwise.
  Before entry with  TRANS = 'N' or 'n',  the  leading  n by k
  part of the array  A  must contain the matrix  A,  otherwise
  the leading  k by n  part of the array  A  must contain  the
  matrix A.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in  the  calling  (sub)  program.  When  TRANS = 'N' or 'n'
  then  LDA must be at least  max( 1, n ), otherwise  LDA must
  be at least  max( 1, k ).

B
  B is COMPLEX array of DIMENSION ( LDB, kb ), where kb is
  k  when  TRANS = 'N' or 'n',  and is  n  otherwise.
  Before entry with  TRANS = 'N' or 'n',  the  leading  n by k
  part of the array  B  must contain the matrix  B,  otherwise
  the leading  k by n  part of the array  B  must contain  the
  matrix B.

LDB
  LDB is INTEGER
  On entry, LDB specifies the first dimension of B as declared
  in  the  calling  (sub)  program.  When  TRANS = 'N' or 'n'
  then  LDB must be at least  max( 1, n ), otherwise  LDB must
```

```
  be at least  max( 1, k ).

BETA
  BETA is REAL
  On entry, BETA specifies the scalar beta.

C
  C is COMPLEX array of DIMENSION ( LDC, n ).

  Before entry  with  UPLO = 'U' or 'u',  the leading  n by n
  upper triangular part of the array C must contain the upper
  triangular part  of the  hermitian matrix  and the strictly
  lower triangular part of C is not referenced.  On exit, the
  upper triangular part of the array  C is overwritten by the
  upper triangular part of the updated matrix.

  Before entry  with  UPLO = 'L' or 'l',  the leading  n by n
  lower triangular part of the array C must contain the lower
  triangular part  of the  hermitian matrix  and the strictly
  upper triangular part of C is not referenced.  On exit, the
  lower triangular part of the array  C is overwritten by the
  lower triangular part of the updated matrix.

  Note that the imaginary parts of the diagonal elements need
  not be set,  they are assumed to be zero,  and on exit they
  are set to zero.

LDC
  LDC is INTEGER
  On entry, LDC specifies the first dimension of C as declared
  in  the  calling  (sub)  program.  LDC  must  be  at  least
  max( 1, n ).
```

———-

16.3.3 fortran code

— cher2k.f —

```
*  ====================================================================
      SUBROUTINE CHER2K(UPLO,TRANS,N,K,ALPHA,A,LDA,B,LDB,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
```

```
*     .. Scalar Arguments ..
      COMPLEX ALPHA
      REAL BETA
      INTEGER K,LDA,LDB,LDC,N
      CHARACTER TRANS,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),B(LDB,*),C(LDC,*)
*     ..
*
*  =====================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG,MAX,REAL
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP1,TEMP2
      INTEGER I,INFO,J,L,NROWA
      LOGICAL UPPER
*     ..
*     .. Parameters ..
      REAL ONE
      PARAMETER (ONE=1.0E+0)
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*
*     Test the input parameters.
*
      IF (LSAME(TRANS,'N')) THEN
          NROWA = N
      ELSE
          NROWA = K
      END IF
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 1
      ELSE IF ((.NOT.LSAME(TRANS,'N')) .AND.
     +         (.NOT.LSAME(TRANS,'C'))) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
```

```
         INFO = 3
      ELSE IF (K.LT.0) THEN
         INFO = 4
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
         INFO = 7
      ELSE IF (LDB.LT.MAX(1,NROWA)) THEN
         INFO = 9
      ELSE IF (LDC.LT.MAX(1,N)) THEN
         INFO = 12
      END IF
      IF (INFO.NE.0) THEN
         CALL XERBLA('CHER2K',INFO)
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (((ALPHA.EQ.ZERO).OR.
     +    (K.EQ.0)).AND. (BETA.EQ.ONE))) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
         IF (UPPER) THEN
            IF (BETA.EQ.REAL(ZERO)) THEN
               DO 20 J = 1,N
                  DO 10 I = 1,J
                     C(I,J) = ZERO
   10             CONTINUE
   20          CONTINUE
            ELSE
               DO 40 J = 1,N
                  DO 30 I = 1,J - 1
                     C(I,J) = BETA*C(I,J)
   30             CONTINUE
                  C(J,J) = BETA*REAL(C(J,J))
   40          CONTINUE
            END IF
         ELSE
            IF (BETA.EQ.REAL(ZERO)) THEN
               DO 60 J = 1,N
                  DO 50 I = J,N
                     C(I,J) = ZERO
   50             CONTINUE
   60          CONTINUE
            ELSE
               DO 80 J = 1,N
                  C(J,J) = BETA*REAL(C(J,J))
                  DO 70 I = J + 1,N
                     C(I,J) = BETA*C(I,J)
```

```
   70                CONTINUE
   80            CONTINUE
             END IF
         END IF
         RETURN
      END IF
*
*     Start the operations.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  C := alpha*A*B**H + conjg( alpha )*B*A**H +
*                   C.
*
         IF (UPPER) THEN
             DO 130 J = 1,N
                 IF (BETA.EQ.REAL(ZERO)) THEN
                     DO 90 I = 1,J
                         C(I,J) = ZERO
   90                CONTINUE
                 ELSE IF (BETA.NE.ONE) THEN
                     DO 100 I = 1,J - 1
                         C(I,J) = BETA*C(I,J)
  100                CONTINUE
                     C(J,J) = BETA*REAL(C(J,J))
                 ELSE
                     C(J,J) = REAL(C(J,J))
                 END IF
                 DO 120 L = 1,K
                     IF ((A(J,L).NE.ZERO) .OR. (B(J,L).NE.ZERO)) THEN
                         TEMP1 = ALPHA*CONJG(B(J,L))
                         TEMP2 = CONJG(ALPHA*A(J,L))
                         DO 110 I = 1,J - 1
                             C(I,J) = C(I,J) + A(I,L)*TEMP1 +
     +                                B(I,L)*TEMP2
  110                    CONTINUE
                         C(J,J) = REAL(C(J,J)) +
     +                           REAL(A(J,L)*TEMP1+B(J,L)*TEMP2)
                     END IF
  120            CONTINUE
  130        CONTINUE
         ELSE
             DO 180 J = 1,N
                 IF (BETA.EQ.REAL(ZERO)) THEN
                     DO 140 I = J,N
                         C(I,J) = ZERO
  140                CONTINUE
                 ELSE IF (BETA.NE.ONE) THEN
                     DO 150 I = J + 1,N
                         C(I,J) = BETA*C(I,J)
```

```
 150                CONTINUE
                    C(J,J) = BETA*REAL(C(J,J))
                ELSE
                    C(J,J) = REAL(C(J,J))
                END IF
                DO 170 L = 1,K
                    IF ((A(J,L).NE.ZERO) .OR. (B(J,L).NE.ZERO)) THEN
                        TEMP1 = ALPHA*CONJG(B(J,L))
                        TEMP2 = CONJG(ALPHA*A(J,L))
                        DO 160 I = J + 1,N
                            C(I,J) = C(I,J) + A(I,L)*TEMP1 +
     +                               B(I,L)*TEMP2
 160                    CONTINUE
                        C(J,J) = REAL(C(J,J)) +
     +                          REAL(A(J,L)*TEMP1+B(J,L)*TEMP2)
                    END IF
 170            CONTINUE
 180        CONTINUE
        END IF
    ELSE
*
*       Form  C := alpha*A**H*B + conjg( alpha )*B**H*A +
*                  C.
*
         IF (UPPER) THEN
             DO 210 J = 1,N
                 DO 200 I = 1,J
                     TEMP1 = ZERO
                     TEMP2 = ZERO
                     DO 190 L = 1,K
                         TEMP1 = TEMP1 + CONJG(A(L,I))*B(L,J)
                         TEMP2 = TEMP2 + CONJG(B(L,I))*A(L,J)
 190                 CONTINUE
                     IF (I.EQ.J) THEN
                         IF (BETA.EQ.REAL(ZERO)) THEN
                             C(J,J) = REAL(ALPHA*TEMP1+
     +                               CONJG(ALPHA)*TEMP2)
                         ELSE
                             C(J,J) = BETA*REAL(C(J,J)) +
     +                               REAL(ALPHA*TEMP1+
     +                               CONJG(ALPHA)*TEMP2)
                         END IF
                     ELSE
                         IF (BETA.EQ.REAL(ZERO)) THEN
                             C(I,J) = ALPHA*TEMP1 + CONJG(ALPHA)*TEMP2
                         ELSE
                             C(I,J) = BETA*C(I,J) + ALPHA*TEMP1 +
     +                               CONJG(ALPHA)*TEMP2
                         END IF
                     END IF
```

```
 200            CONTINUE
 210        CONTINUE
        ELSE
            DO 240 J = 1,N
                DO 230 I = J,N
                    TEMP1 = ZERO
                    TEMP2 = ZERO
                    DO 220 L = 1,K
                        TEMP1 = TEMP1 + CONJG(A(L,I))*B(L,J)
                        TEMP2 = TEMP2 + CONJG(B(L,I))*A(L,J)
 220                CONTINUE
                    IF (I.EQ.J) THEN
                        IF (BETA.EQ.REAL(ZERO)) THEN
                            C(J,J) = REAL(ALPHA*TEMP1+
     +                               CONJG(ALPHA)*TEMP2)
                        ELSE
                            C(J,J) = BETA*REAL(C(J,J)) +
     +                               REAL(ALPHA*TEMP1+
     +                               CONJG(ALPHA)*TEMP2)
                        END IF
                    ELSE
                        IF (BETA.EQ.REAL(ZERO)) THEN
                            C(I,J) = ALPHA*TEMP1 + CONJG(ALPHA)*TEMP2
                        ELSE
                            C(I,J) = BETA*C(I,J) + ALPHA*TEMP1 +
     +                               CONJG(ALPHA)*TEMP2
                        END IF
                    END IF
 230            CONTINUE
 240        CONTINUE
        END IF
    END IF
*
    RETURN
*
*   End of CHER2K.
*
    END
```

———-

16.3.4 lisp code

— BLAS 3 cher2k —

```
(let* ((one 1.0) (zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (single-float 1.0 1.0) one)
```

```
          (type (f2cl-lib:complex8) zero)
          (ignorable one zero))
 (defun cher2k (uplo trans n k alpha a lda b ldb$ beta c ldc)
   (declare (type (single-float) beta)
            (type (array f2cl-lib:complex8 (*)) c b a)
            (type (f2cl-lib:complex8) alpha)
            (type (f2cl-lib:integer4) ldc ldb$ lda k n)
            (type (string *) trans uplo))
   (f2cl-lib:with-multi-array-data
       ((uplo character uplo-%data% uplo-%offset%)
        (trans character trans-%data% trans-%offset%)
        (a f2cl-lib:complex8 a-%data% a-%offset%)
        (b f2cl-lib:complex8 b-%data% b-%offset%)
        (c f2cl-lib:complex8 c-%data% c-%offset%))
     (prog ((upper nil) (i 0) (info 0) (j 0) (l 0) (nrowa 0)
            (temp1 #C(0.0 0.0)) (temp2 #C(0.0 0.0)))
       (declare (type f2cl-lib:logical upper)
                (type (f2cl-lib:integer4) i info j l nrowa)
                (type (f2cl-lib:complex8) temp1 temp2))
       (cond
        ((multiple-value-bind (ret-val var-0 var-1)
             (lsame trans "N")
           (declare (ignore var-1))
           (when var-0 (setf trans var-0))
           ret-val)
         (setf nrowa n))
        (t (setf nrowa k)))
       (setf upper
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame uplo "U")
                 (declare (ignore var-1))
                 (when var-0 (setf uplo var-0))
                 ret-val))
       (setf info 0)
       (cond
        ((and (not upper)
              (not
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame uplo "L")
                 (declare (ignore var-1))
                 (when var-0 (setf uplo var-0))
                 ret-val)))
         (setf info 1))
        ((and
          (not
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame trans "N")
             (declare (ignore var-1))
             (when var-0 (setf trans var-0))
             ret-val))
```

```
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame trans "C")
      (declare (ignore var-1))
      (when var-0 (setf trans var-0))
      ret-val)))
  (setf info 2))
 ((< n 0) (setf info 3)) ((< k 0) (setf info 4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nrowa)))
  (setf info 7))
 ((< ldb$
     (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nrowa)))
  (setf info 9))
 ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info 12)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "CHER2K" info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= n 0) (and (or (= alpha zero) (= k 0)) (= beta one)))
    (go end_label))
(cond
 ((= alpha zero)
  (cond
   (upper
    (cond
     ((= beta (f2cl-lib:freal zero))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                   c-%offset%)
                    zero)
             label10))
         label20)))
     (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
```

```
                (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)
                        (* beta
                           (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                          c-%offset%)))
            label30))
          (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                c-%offset%)
                (coerce
                 (* beta
                    (f2cl-lib:freal
                     (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                    c-%offset%)))
                 'f2cl-lib:complex8))
       label40)))))
  (t
   (cond
    ((= beta (f2cl-lib:freal zero))
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                   c-%offset%)
                     zero)
            label50))
       label60)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                               c-%offset%)
                (coerce
                 (* beta
                    (f2cl-lib:freal
                     (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                    c-%offset%)))
                 'f2cl-lib:complex8))
         (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                        (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                   c-%offset%)
                     (* beta
                        (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)))
```

```
            label70))
        label80))))))
  (go end_label)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame trans "N")
    (declare (ignore var-1))
    (when var-0 (setf trans var-0))
    ret-val)
  (cond
   (upper
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
         ((= beta (f2cl-lib:freal zero))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                    zero)
            label90)))
         ((/= beta one)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                    (* beta
                       (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)))
            label100))
          (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                c-%offset%)
                (coerce
                 (* beta
                    (f2cl-lib:freal
                     (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                    c-%offset%)))
                 'f2cl-lib:complex8)))
         (t
          (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                c-%offset%)
                (coerce
                 (f2cl-lib:freal
                  (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                 c-%offset%))
```

```
            'f2cl-lib:complex8))))
(f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
              ((> l k) nil)
  (tagbody
    (cond
     ((or (/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
          (/= (f2cl-lib:fref b (j l) ((1 ldb$) (1 *))) zero))
      (setf temp1
              (* alpha
                 (f2cl-lib:conjg
                  (f2cl-lib:fref b-%data% (j l)
                                 ((1 ldb$) (1 *))
                                 b-%offset%))))
      (setf temp2
              (f2cl-lib:conjg
               (* alpha
                  (f2cl-lib:fref a-%data% (j l) ((1 lda) (1 *))
                                 a-%offset%))))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (f2cl-lib:int-add j
                                          (f2cl-lib:int-sub
                                           1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                               c-%offset%)
                  (+
                   (f2cl-lib:fref c-%data% (i j)
                                  ((1 ldc) (1 *)) c-%offset%)
                   (*
                    (f2cl-lib:fref a-%data% (i l)
                                   ((1 lda) (1 *)) a-%offset%)
                    temp1)
                   (*
                    (f2cl-lib:fref b-%data% (i l)
                                   ((1 ldb$) (1 *)) b-%offset%)
                    temp2)))
          label110))
      (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                           c-%offset%)
              (coerce
               (+
                (f2cl-lib:freal
                 (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                c-%offset%))
                (f2cl-lib:freal
                 (+
                  (*
                   (f2cl-lib:fref a-%data% (j l)
```

```
                                        ((1 lda) (1 *)) a-%offset%)
                         temp1)
                        (*
                         (f2cl-lib:fref b-%data% (j l)
                                        ((1 ldb$) (1 *)) b-%offset%)
                         temp2))))
                     'f2cl-lib:complex8))))
        label120))
    label130)))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
      ((= beta (f2cl-lib:freal zero))
       (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                c-%offset%)
                 zero)
          label140)))
      ((/= beta one)
       (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                        (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* beta
                    (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                   c-%offset%)))
          label150))
       (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                            c-%offset%)
             (coerce
              (* beta
                 (f2cl-lib:freal
                  (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                 c-%offset%)))
              'f2cl-lib:complex8)))
      (t
       (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                            c-%offset%)
             (coerce
              (f2cl-lib:freal
               (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                              c-%offset%))
              'f2cl-lib:complex8))))
     (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
```

```
            ((> l k) nil)
(tagbody
  (cond
   ((or (/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
        (/= (f2cl-lib:fref b (j l) ((1 ldb$) (1 *))) zero))
    (setf temp1
            (* alpha
               (f2cl-lib:conjg
                (f2cl-lib:fref b-%data% (j l)
                               ((1 ldb$) (1 *))
                               b-%offset%))))
    (setf temp2
            (f2cl-lib:conjg
             (* alpha
                (f2cl-lib:fref a-%data% (j l) ((1 lda) (1 *))
                               a-%offset%))))
    (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                   (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                             c-%offset%)
                (+
                 (f2cl-lib:fref c-%data% (i j)
                                ((1 ldc) (1 *)) c-%offset%)
                 (*
                  (f2cl-lib:fref a-%data% (i l)
                                 ((1 lda) (1 *)) a-%offset%)
                  temp1)
                 (*
                  (f2cl-lib:fref b-%data% (i l)
                                 ((1 ldb$) (1 *)) b-%offset%)
                  temp2)))
       label160))
    (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                         c-%offset%)
            (coerce
             (+
              (f2cl-lib:freal
               (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                              c-%offset%))
              (f2cl-lib:freal
               (+
                (*
                 (f2cl-lib:fref a-%data% (j l)
                                ((1 lda) (1 *)) a-%offset%)
                 temp1)
                (*
                 (f2cl-lib:fref b-%data% (j l)
                                ((1 ldb$) (1 *)) b-%offset%)
```

```
                          temp2))))
                       'f2cl-lib:complex8))))
           label170))
      label180)))))
(t
 (cond
  (upper
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i j) nil)
         (tagbody
           (setf temp1 zero)
           (setf temp2 zero)
           (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                         ((> l k) nil)
             (tagbody
               (setf temp1
                       (+ temp1
                          (*
                           (f2cl-lib:conjg
                            (f2cl-lib:fref a-%data% (l i)
                                           ((1 lda) (1 *))
                                           a-%offset%))
                           (f2cl-lib:fref b-%data% (l j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%))))
               (setf temp2
                       (+ temp2
                          (*
                           (f2cl-lib:conjg
                            (f2cl-lib:fref b-%data% (l i)
                                           ((1 ldb$) (1 *))
                                           b-%offset%))
                           (f2cl-lib:fref a-%data% (l j)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
               label190))
           (cond
            ((= i j)
             (cond
              ((= beta (f2cl-lib:freal zero))
               (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                     c-%offset%)
                       (coerce
                        (f2cl-lib:freal
                         (+ (* alpha temp1)
                            (* (f2cl-lib:conjg alpha) temp2)))
                        'f2cl-lib:complex8)))
```

```
                (t
                 (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                       c-%offset%)
                         (coerce
                          (+
                           (* beta
                              (f2cl-lib:freal
                               (f2cl-lib:fref c-%data% (j j)
                                              ((1 ldc) (1 *))
                                              c-%offset%)))
                           (f2cl-lib:freal
                            (+ (* alpha temp1)
                               (* (f2cl-lib:conjg alpha) temp2))))
                          'f2cl-lib:complex8)))))
            (t
             (cond
              ((= beta (f2cl-lib:freal zero))
               (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                       (+ (* alpha temp1)
                          (* (f2cl-lib:conjg alpha) temp2))))
              (t
               (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                       (+
                        (* beta
                           (f2cl-lib:fref c-%data% (i j)
                                          ((1 ldc) (1 *)) c-%offset%))
                        (* alpha temp1)
                        (* (f2cl-lib:conjg alpha) temp2)))))))
        label200))
    label210)))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf temp1 zero)
         (setf temp2 zero)
         (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                       ((> l k) nil)
           (tagbody
             (setf temp1
                     (+ temp1
                        (*
                         (f2cl-lib:conjg
                          (f2cl-lib:fref a-%data% (l i)
                                         ((1 lda) (1 *))
```

```
                                    a-%offset%))
                    (f2cl-lib:fref b-%data% (l j)
                                   ((1 ldb$) (1 *))
                                   b-%offset%))))
     (setf temp2
             (+ temp2
                (*
                 (f2cl-lib:conjg
                  (f2cl-lib:fref b-%data% (l i)
                                  ((1 ldb$) (1 *))
                                  b-%offset%))
                 (f2cl-lib:fref a-%data% (l j)
                                ((1 lda) (1 *))
                                a-%offset%))))
   label220))
(cond
 ((= i j)
  (cond
   ((= beta (f2cl-lib:freal zero))
    (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                         c-%offset%)
            (coerce
             (f2cl-lib:freal
              (+ (* alpha temp1)
                 (* (f2cl-lib:conjg alpha) temp2)))
             'f2cl-lib:complex8)))
   (t
    (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                         c-%offset%)
            (coerce
             (+
              (* beta
                 (f2cl-lib:freal
                  (f2cl-lib:fref c-%data% (j j)
                                 ((1 ldc) (1 *))
                                 c-%offset%)))
              (f2cl-lib:freal
               (+ (* alpha temp1)
                  (* (f2cl-lib:conjg alpha) temp2))))
             'f2cl-lib:complex8)))))
 (t
  (cond
   ((= beta (f2cl-lib:freal zero))
    (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                         c-%offset%)
            (+ (* alpha temp1)
               (* (f2cl-lib:conjg alpha) temp2))))
   (t
    (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                         c-%offset%)
```

```
                                (+
                                 (* beta
                                    (f2cl-lib:fref c-%data% (i j)
                                                   ((1 ldc) (1 *)) c-%offset%))
                                 (* alpha temp1)
                                 (* (f2cl-lib:conj alpha) temp2)))))))
                 label230))
            label240))))))
        (go end_label)
       end_label
        (return (values uplo trans nil nil nil nil nil nil nil nil nil nil))))))
```

———-

16.4 cherk - hermitian rank k $\alpha AA^H + \beta C$ or $\alpha A^H A + \beta C$

16.4.1 Axiom unit tests

— **cherk.input** —

```
)set break resume
)sys rm -f cherk.output
)spool cherk.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

16.4.2 Axiom help page

— **cherk.help** —

```
======================================================================
cherk examples
======================================================================

======================================================================
Man Page Details
======================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine cherk (UPLO, TRANS, N, K, ALPHA, A, LDA, BETA, C, LDC)
           CHERK

Function/Subroutine Documentation
   subroutine cherk (character UPLO, character TRANS, integer N, integer K,
       real ALPHA, complex, dimension(lda,*) A, integer LDA, real BETA,
       complex, dimension(ldc,*) C, integer LDC)
       CHERK

Purpose:
   CHERK  performs one of the hermitian rank k operations

   C := alpha*A*A**H + beta*C,

   or

   C := alpha*A**H*A + beta*C,

   where  alpha and beta  are  real scalars,  C is an  n by n  hermitian
   matrix and  A  is an  n by k  matrix in the  first case and a  k by n
   matrix in the second case.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On  entry,   UPLO  specifies  whether  the  upper  or  lower
   triangular part of the array C is to be referenced as follows:

   UPLO = 'U' or 'u'   Only the  upper triangular part of  C
   is to be referenced.

   UPLO = 'L' or 'l'   Only the  lower triangular part of  C
   is to be referenced.

 TRANS
   TRANS is CHARACTER*1
   On entry, TRANS specifies the operation to be performed as follows:

   TRANS = 'N' or 'n'   C := alpha*A*A**H + beta*C.

   TRANS = 'C' or 'c'   C := alpha*A**H*A + beta*C.

 N
   N is INTEGER
   On entry,  N specifies the order of the matrix C.  N must be
   at least zero.
```

```
K
  K is INTEGER
  On entry with  TRANS = 'N' or 'n',  K  specifies  the number
  of  columns   of  the   matrix   A,   and  on   entry   with
  TRANS = 'C' or 'c',  K  specifies  the number of rows of the
  matrix A.  K must be at least zero.

ALPHA
  ALPHA is REAL
  On entry, ALPHA specifies the scalar alpha.

A
  A is COMPLEX array of DIMENSION ( LDA, ka ), where ka is
  k  when  TRANS = 'N' or 'n',  and is  n  otherwise.
  Before entry with  TRANS = 'N' or 'n',  the  leading  n by k
  part of the array  A  must contain the matrix  A,  otherwise
  the leading  k by n  part of the array  A  must contain  the
  matrix A.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in  the  calling  (sub)  program.   When  TRANS = 'N' or 'n'
  then  LDA must be at least  max( 1, n ), otherwise  LDA must
  be at least  max( 1, k ).

BETA
  BETA is REAL
  On entry, BETA specifies the scalar beta.

C
  C is COMPLEX array of DIMENSION ( LDC, n ).

  Before entry  with  UPLO = 'U' or 'u',  the leading  n by n
  upper triangular part of the array C must contain the upper
  triangular part  of the  hermitian matrix  and the strictly
  lower triangular part of C is not referenced.  On exit, the
  upper triangular part of the array  C is overwritten by the
  upper triangular part of the updated matrix.

  Before entry  with  UPLO = 'L' or 'l',  the leading  n by n
  lower triangular part of the array C must contain the lower
  triangular part  of the  hermitian matrix  and the strictly
  upper triangular part of C is not referenced.  On exit, the
  lower triangular part of the array  C is overwritten by the
  lower triangular part of the updated matrix.

  Note that the imaginary parts of the diagonal elements need
  not be set,  they are assumed to be zero,  and on exit they
```

```
  are set to zero.

LDC
  LDC is INTEGER
  On entry, LDC specifies the first dimension of C as declared
  in  the  calling  (sub)  program.  LDC  must  be  at  least
  max( 1, n ).
```

16.4.3 fortran code

— cherk.f —

```
*  ===================================================================
      SUBROUTINE CHERK(UPLO,TRANS,N,K,ALPHA,A,LDA,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL ALPHA,BETA
      INTEGER K,LDA,LDC,N
      CHARACTER TRANS,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),C(LDC,*)
*     ..
*
*  ===================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CMPLX,CONJG,MAX,REAL
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP
      REAL RTEMP
      INTEGER I,INFO,J,L,NROWA
```

```
      LOGICAL UPPER
*     ..
*     .. Parameters ..
      REAL ONE,ZERO
      PARAMETER (ONE=1.0E+0,ZERO=0.0E+0)
*     ..
*
*     Test the input parameters.
*
      IF (LSAME(TRANS,'N')) THEN
          NROWA = N
      ELSE
          NROWA = K
      END IF
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 1
      ELSE IF ((.NOT.LSAME(TRANS,'N')) .AND.
     +         (.NOT.LSAME(TRANS,'C'))) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
          INFO = 3
      ELSE IF (K.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 7
      ELSE IF (LDC.LT.MAX(1,N)) THEN
          INFO = 10
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CHERK ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (((ALPHA.EQ.ZERO).OR.
     +    (K.EQ.0)).AND. (BETA.EQ.ONE))) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
          IF (UPPER) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 20 J = 1,N
                      DO 10 I = 1,J
                          C(I,J) = ZERO
   10                 CONTINUE
```

```
   20            CONTINUE
              ELSE
                 DO 40 J = 1,N
                    DO 30 I = 1,J - 1
                       C(I,J) = BETA*C(I,J)
   30               CONTINUE
                    C(J,J) = BETA*REAL(C(J,J))
   40            CONTINUE
              END IF
           ELSE
              IF (BETA.EQ.ZERO) THEN
                 DO 60 J = 1,N
                    DO 50 I = J,N
                       C(I,J) = ZERO
   50               CONTINUE
   60            CONTINUE
              ELSE
                 DO 80 J = 1,N
                    C(J,J) = BETA*REAL(C(J,J))
                    DO 70 I = J + 1,N
                       C(I,J) = BETA*C(I,J)
   70               CONTINUE
   80            CONTINUE
              END IF
           END IF
           RETURN
        END IF
*
*       Start the operations.
*
        IF (LSAME(TRANS,'N')) THEN
*
*          Form  C := alpha*A*A**H + beta*C.
*
           IF (UPPER) THEN
              DO 130 J = 1,N
                 IF (BETA.EQ.ZERO) THEN
                    DO 90 I = 1,J
                       C(I,J) = ZERO
   90               CONTINUE
                 ELSE IF (BETA.NE.ONE) THEN
                    DO 100 I = 1,J - 1
                       C(I,J) = BETA*C(I,J)
  100               CONTINUE
                    C(J,J) = BETA*REAL(C(J,J))
                 ELSE
                    C(J,J) = REAL(C(J,J))
                 END IF
                 DO 120 L = 1,K
                    IF (A(J,L).NE.CMPLX(ZERO)) THEN
```

```
                        TEMP = ALPHA*CONJG(A(J,L))
                        DO 110 I = 1,J - 1
                            C(I,J) = C(I,J) + TEMP*A(I,L)
  110                   CONTINUE
                        C(J,J) = REAL(C(J,J)) + REAL(TEMP*A(I,L))
                    END IF
  120           CONTINUE
  130       CONTINUE
        ELSE
            DO 180 J = 1,N
                IF (BETA.EQ.ZERO) THEN
                    DO 140 I = J,N
                        C(I,J) = ZERO
  140               CONTINUE
                ELSE IF (BETA.NE.ONE) THEN
                    C(J,J) = BETA*REAL(C(J,J))
                    DO 150 I = J + 1,N
                        C(I,J) = BETA*C(I,J)
  150               CONTINUE
                ELSE
                    C(J,J) = REAL(C(J,J))
                END IF
                DO 170 L = 1,K
                    IF (A(J,L).NE.CMPLX(ZERO)) THEN
                        TEMP = ALPHA*CONJG(A(J,L))
                        C(J,J) = REAL(C(J,J)) + REAL(TEMP*A(J,L))
                        DO 160 I = J + 1,N
                            C(I,J) = C(I,J) + TEMP*A(I,L)
  160                   CONTINUE
                    END IF
  170           CONTINUE
  180       CONTINUE
        END IF
    ELSE
*
*       Form  C := alpha*A**H*A + beta*C.
*
        IF (UPPER) THEN
            DO 220 J = 1,N
                DO 200 I = 1,J - 1
                    TEMP = ZERO
                    DO 190 L = 1,K
                        TEMP = TEMP + CONJG(A(L,I))*A(L,J)
  190               CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP
                    ELSE
                        C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                    END IF
  200           CONTINUE
```

```
                RTEMP = ZERO
                DO 210 L = 1,K
                    RTEMP = RTEMP + CONJG(A(L,J))*A(L,J)
 210            CONTINUE
                IF (BETA.EQ.ZERO) THEN
                    C(J,J) = ALPHA*RTEMP
                ELSE
                    C(J,J) = ALPHA*RTEMP + BETA*REAL(C(J,J))
                END IF
 220        CONTINUE
        ELSE
            DO 260 J = 1,N
                RTEMP = ZERO
                DO 230 L = 1,K
                    RTEMP = RTEMP + CONJG(A(L,J))*A(L,J)
 230            CONTINUE
                IF (BETA.EQ.ZERO) THEN
                    C(J,J) = ALPHA*RTEMP
                ELSE
                    C(J,J) = ALPHA*RTEMP + BETA*REAL(C(J,J))
                END IF
                DO 250 I = J + 1,N
                    TEMP = ZERO
                    DO 240 L = 1,K
                        TEMP = TEMP + CONJG(A(L,I))*A(L,J)
 240                CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP
                    ELSE
                        C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                    END IF
 250            CONTINUE
 260        CONTINUE
        END IF
    END IF
*
    RETURN
*
*   End of CHERK .
*
    END
```

———-

16.4.4 lisp code

— BLAS 3 cherk —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun cherk (uplo trans n k alpha a lda beta c ldc)
    (declare (type (array f2cl-lib:complex8 (*)) c a)
             (type (single-float) beta alpha)
             (type (f2cl-lib:integer4) ldc lda k n)
             (type (string *) trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (c f2cl-lib:complex8 c-%data% c-%offset%))
      (prog ((upper nil) (i 0) (info 0) (j 0) (l 0) (nrowa 0) (rtemp 0.0)
             (temp #C(0.0 0.0)))
        (declare (type f2cl-lib:logical upper)
                 (type (f2cl-lib:integer4) i info j l nrowa)
                 (type (single-float) rtemp)
                 (type (f2cl-lib:complex8) temp))
        (cond
         ((multiple-value-bind (ret-val var-0 var-1)
              (lsame trans "N")
            (declare (ignore var-1))
            (when var-0 (setf trans var-0))
            ret-val)
          (setf nrowa n))
         (t (setf nrowa k)))
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (setf info 0)
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info 1))
         ((and
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "N")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
```

```
      ret-val))
    (not
     (multiple-value-bind (ret-val var-0 var-1)
         (lsame trans "C")
       (declare (ignore var-1))
       (when var-0 (setf trans var-0))
       ret-val)))
  (setf info 2))
 ((< n 0) (setf info 3)) ((< k 0) (setf info 4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nrowa)))
  (setf info 7))
 ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info 10)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "CHERK " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= n 0) (and (or (= alpha zero) (= k 0)) (= beta one)))
    (go end_label))
(cond
 ((= alpha zero)
  (cond
   (upper
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                      (coerce zero 'f2cl-lib:complex8))
             label10))
         label20)))
     (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
```

```
                    (* beta
                       (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)))
          label30))
        (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                             c-%offset%)
              (coerce
               (* beta
                  (f2cl-lib:freal
                   (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                  c-%offset%)))
               'f2cl-lib:complex8))
      label40)))))
(t
 (cond
  ((= beta zero)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                c-%offset%)
                 (coerce zero 'f2cl-lib:complex8))
          label50))
      label60)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                            c-%offset%)
             (coerce
              (* beta
                 (f2cl-lib:freal
                  (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                 c-%offset%)))
              'f2cl-lib:complex8))
       (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                      (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* beta
                    (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                   c-%offset%)))
          label70))
      label80))))))
```

```
  (go end_label)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame trans "N")
    (declare (ignore var-1))
    (when var-0 (setf trans var-0))
    ret-val)
  (cond
   (upper
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
         ((= beta zero)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                    (coerce zero 'f2cl-lib:complex8))
             label90)))
         ((/= beta one)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                    (* beta
                       (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)))
             label100))
          (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                               c-%offset%)
                (coerce
                 (* beta
                    (f2cl-lib:freal
                     (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                    c-%offset%)))
                 'f2cl-lib:complex8)))
         (t
          (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                               c-%offset%)
                (coerce
                 (f2cl-lib:freal
                  (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                 c-%offset%))
                 'f2cl-lib:complex8))))
        (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
```

```
                    ((> l k) nil)
            (tagbody
              (cond
               ((/= (f2cl-lib:fref a (j l) ((1 lda) (1 *)))
                    (f2cl-lib:cmplx zero))
                (setf temp
                        (* alpha
                           (f2cl-lib:conjg
                            (f2cl-lib:fref a-%data% (j l) ((1 lda) (1 *))
                                           a-%offset%))))
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i
                                  (f2cl-lib:int-add j
                                                    (f2cl-lib:int-sub
                                                     1)))
                               nil)
                  (tagbody
                    (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                         c-%offset%)
                          (+
                           (f2cl-lib:fref c-%data% (i j)
                                          ((1 ldc) (1 *)) c-%offset%)
                           (* temp
                              (f2cl-lib:fref a-%data% (i l)
                                             ((1 lda) (1 *))
                                             a-%offset%))))
                   label110))
                (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                     c-%offset%)
                        (coerce
                         (+
                          (f2cl-lib:freal
                           (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                          c-%offset%))
                          (f2cl-lib:freal
                           (* temp
                              (f2cl-lib:fref a-%data% (i l)
                                             ((1 lda) (1 *))
                                             a-%offset%))))
                         'f2cl-lib:complex8))))
             label120))
         label130)))
      (t
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (cond
            ((= beta zero)
             (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                           ((> i n) nil)
```

```
      (tagbody
        (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                             c-%offset%)
                (coerce zero 'f2cl-lib:complex8))
       label140)))
    ((/= beta one)
     (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                          c-%offset%)
             (coerce
              (* beta
                 (f2cl-lib:freal
                  (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                 c-%offset%)))
              'f2cl-lib:complex8))
     (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                    (f2cl-lib:int-add i 1))
                   ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                             c-%offset%)
                (* beta
                   (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                  c-%offset%)))
       label150)))
    (t
     (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                          c-%offset%)
             (coerce
              (f2cl-lib:freal
               (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                              c-%offset%))
              'f2cl-lib:complex8))))
   (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                 ((> l k) nil)
     (tagbody
       (cond
        ((/= (f2cl-lib:fref a (j l) ((1 lda) (1 *)))
             (f2cl-lib:cmplx zero))
         (setf temp
                 (* alpha
                    (f2cl-lib:conjg
                     (f2cl-lib:fref a-%data% (j l) ((1 lda) (1 *))
                                    a-%offset%))))
         (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                              c-%offset%)
                 (coerce
                  (+
                   (f2cl-lib:freal
                    (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                   c-%offset%))
```

```
                        (f2cl-lib:freal
                         (* temp
                            (f2cl-lib:fref a-%data% (j l)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
                       'f2cl-lib:complex8))
              (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                             (f2cl-lib:int-add i 1))
                            ((> i n) nil)
                (tagbody
                  (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)
                        (+
                         (f2cl-lib:fref c-%data% (i j)
                                        ((1 ldc) (1 *)) c-%offset%)
                         (* temp
                            (f2cl-lib:fref a-%data% (i l)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
                 label160))))
          label170))
      label180)))))
(t
 (cond
  (upper
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                      nil)
         (tagbody
           (setf temp (coerce zero 'f2cl-lib:complex8))
           (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                         ((> l k) nil)
             (tagbody
               (setf temp
                       (+ temp
                          (*
                           (f2cl-lib:conjg
                            (f2cl-lib:fref a-%data% (l i)
                                           ((1 lda) (1 *))
                                           a-%offset%))
                           (f2cl-lib:fref a-%data% (l j)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
               label190))
           (cond
            ((= beta zero)
             (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
```

```
                                c-%offset%)
                      (* alpha temp)))
             (t
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                c-%offset%)
                      (+ (* alpha temp)
                         (* beta
                            (f2cl-lib:fref c-%data% (i j)
                                           ((1 ldc) (1 *))
                                           c-%offset%))))))
           label200))
        (setf rtemp zero)
        (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                      ((> l k) nil)
          (tagbody
            (setf rtemp
                    (coerce
                     (realpart
                      (+ rtemp
                         (*
                          (f2cl-lib:conjg
                           (f2cl-lib:fref a-%data% (l j)
                                          ((1 lda) (1 *)) a-%offset%))
                          (f2cl-lib:fref a-%data% (l j) ((1 lda) (1 *))
                                         a-%offset%))))
                     'single-float))
           label210))
        (cond
         ((= beta zero)
          (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                               c-%offset%)
                (coerce (* alpha rtemp) 'f2cl-lib:complex8)))
         (t
          (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                               c-%offset%)
                (coerce
                 (+ (* alpha rtemp)
                    (* beta
                       (f2cl-lib:freal
                        (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                                       c-%offset%))))
                 'f2cl-lib:complex8))))
      label220)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf rtemp zero)
       (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                     ((> l k) nil)
```

```
  (tagbody
    (setf rtemp
            (coerce
             (realpart
              (+ rtemp
                 (*
                  (f2cl-lib:conjg
                   (f2cl-lib:fref a-%data% (l j)
                                  ((1 lda) (1 *)) a-%offset%))
                  (f2cl-lib:fref a-%data% (l j) ((1 lda) (1 *))
                                 a-%offset%))))
             'single-float))
   label230))
(cond
 ((= beta zero)
  (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                       c-%offset%)
        (coerce (* alpha rtemp) 'f2cl-lib:complex8)))
 (t
  (setf (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                       c-%offset%)
        (coerce
         (+ (* alpha rtemp)
            (* beta
               (f2cl-lib:freal
                (f2cl-lib:fref c-%data% (j j) ((1 ldc) (1 *))
                               c-%offset%))))
         'f2cl-lib:complex8))))
(f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf temp (coerce zero 'f2cl-lib:complex8))
    (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                  ((> l k) nil)
      (tagbody
        (setf temp
                (+ temp
                   (*
                    (f2cl-lib:conjg
                     (f2cl-lib:fref a-%data% (l i)
                                    ((1 lda) (1 *))
                                    a-%offset%))
                    (f2cl-lib:fref a-%data% (l j)
                                   ((1 lda) (1 *))
                                   a-%offset%))))
       label240))
    (cond
     ((= beta zero)
      (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                           c-%offset%)
```

```
                          (* alpha temp)))
                    (t
                     (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                          c-%offset%)
                          (+ (* alpha temp)
                             (* beta
                                (f2cl-lib:fref c-%data% (i j)
                                               ((1 ldc) (1 *))
                                               c-%offset%))))))
                label250))
            label260))))))
  (go end_label)
 end_label
  (return (values uplo trans nil nil nil nil nil nil nil nil))))))
```

———-

16.5 csymm - symmetric $\alpha AB + \beta C$ or $\alpha BA + \beta C$

16.5.1 Axiom unit tests

— **csymm.input** —

```
)set break resume
)sys rm -f csymm.output
)spool csymm.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

16.5.2 Axiom help page

— **csymm.help** —

```
======================================================================
csymm examples
======================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csymm (SIDE, UPLO, M, N, ALPHA, A, LDA, B, LDB, BETA, C,
           LDC)
           CSYMM

Function/Subroutine Documentation
   subroutine csymm (character SIDE, character UPLO, integer M, integer N,
       complex ALPHA, complex, dimension(lda,*) A, integer LDA, complex,
       dimension(ldb,*) B, integer LDB, complex BETA, complex,
       dimension(ldc,*) C, integer LDC)
       CSYMM

Purpose:
   CSYMM  performs one of the matrix-matrix operations

   C := alpha*A*B + beta*C,

   or

   C := alpha*B*A + beta*C,

   where  alpha and beta are scalars, A is a symmetric matrix and  B and
   C are m by n matrices.

Parameters:

 SIDE
   SIDE is CHARACTER*1
   On entry,  SIDE  specifies whether  the  symmetric matrix  A
   appears on the  left or right  in the  operation as follows:

   SIDE = 'L' or 'l'   C := alpha*A*B + beta*C,

   SIDE = 'R' or 'r'   C := alpha*B*A + beta*C,

 UPLO

   UPLO is CHARACTER*1
   On  entry,   UPLO  specifies  whether  the  upper  or  lower
   triangular  part  of  the  symmetric  matrix   A  is  to  be
   referenced as follows:

   UPLO = 'U' or 'u'   Only the upper triangular part of the
   symmetric matrix is to be referenced.
```

```
  UPLO = 'L' or 'l'   Only the lower triangular part of the
  symmetric matrix is to be referenced.

M
  M is INTEGER
  On entry,  M  specifies the number of rows of the matrix  C.
  M  must be at least zero.

N
  N is INTEGER
  On entry, N specifies the number of columns of the matrix C.
  N  must be at least zero.

ALPHA
  ALPHA is COMPLEX
  On entry, ALPHA specifies the scalar alpha.

A
  A is COMPLEX array of DIMENSION ( LDA, ka ), where ka is
  m  when  SIDE = 'L' or 'l'  and is n  otherwise.

  Before entry  with  SIDE = 'L' or 'l',  the  m by m  part of
  the array  A  must contain the  symmetric matrix,  such that
  when  UPLO = 'U' or 'u', the leading m by m upper triangular
  part of the array  A  must contain the upper triangular part
  of the  symmetric matrix and the  strictly  lower triangular
  part of  A  is not referenced,  and when  UPLO = 'L' or 'l',
  the leading  m by m  lower triangular part  of the  array  A
  must  contain  the  lower triangular part  of the  symmetric
  matrix and the  strictly upper triangular part of  A  is not
  referenced.

  Before entry  with  SIDE = 'R' or 'r',  the  n by n  part of
  the array  A  must contain the  symmetric matrix,  such that
  when  UPLO = 'U' or 'u', the leading n by n upper triangular
  part of the array  A  must contain the upper triangular part
  of the  symmetric matrix and the  strictly  lower triangular
  part of  A  is not referenced,  and when  UPLO = 'L' or 'l',
  the leading  n by n  lower triangular part  of the  array  A
  must  contain  the  lower triangular part  of the  symmetric
  matrix and the  strictly upper triangular part of  A  is not
  referenced.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the  calling (sub) program. When  SIDE = 'L' or 'l'  then
  LDA must be at least  max( 1, m ), otherwise  LDA must be at
  least max( 1, n ).
```

```
B
  B is COMPLEX array of DIMENSION ( LDB, n ).
  Before entry, the leading  m by n part of the array  B  must
  contain the matrix B.

LDB
  LDB is INTEGER
  On entry, LDB specifies the first dimension of B as declared
  in  the  calling  (sub)  program.  LDB  must  be  at  least
  max( 1, m ).

BETA
  BETA is COMPLEX
  On entry,  BETA  specifies the scalar  beta.  When  BETA  is
  supplied as zero then C need not be set on input.

C
  C is COMPLEX array of DIMENSION ( LDC, n ).

  Before entry, the leading  m by n  part of the array  C must
  contain the matrix  C,  except when  beta  is zero, in which
  case C need not be set on entry.
  On exit, the array C is overwritten by the m by n updated matrix.

LDC
  LDC is INTEGER
  On entry, LDC specifies the first dimension of C as declared
  in  the  calling  (sub)  program.  LDC  must  be  at  least
  max( 1, m ).
```

———-

16.5.3 fortran code

— csymm.f —

```
*  ===================================================================
      SUBROUTINE CSYMM(SIDE,UPLO,M,N,ALPHA,A,LDA,B,LDB,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX ALPHA,BETA
      INTEGER LDA,LDB,LDC,M,N
```

```
      CHARACTER SIDE,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),B(LDB,*),C(LDC,*)
*     ..
*
*  =====================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP1,TEMP2
      INTEGER I,INFO,J,K,NROWA
      LOGICAL UPPER
*     ..
*     .. Parameters ..
      COMPLEX ONE
      PARAMETER (ONE= (1.0E+0,0.0E+0))
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*
*     Set NROWA as the number of rows of A.
*
      IF (LSAME(SIDE,'L')) THEN
          NROWA = M
      ELSE
          NROWA = N
      END IF
      UPPER = LSAME(UPLO,'U')
*
*     Test the input parameters.
*
      INFO = 0
      IF ((.NOT.LSAME(SIDE,'L')) .AND. (.NOT.LSAME(SIDE,'R'))) THEN
          INFO = 1
      ELSE IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 2
      ELSE IF (M.LT.0) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
```

```
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 7
      ELSE IF (LDB.LT.MAX(1,M)) THEN
          INFO = 9
      ELSE IF (LDC.LT.MAX(1,M)) THEN
          INFO = 12
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CSYMM ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR.
     +    ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
          IF (BETA.EQ.ZERO) THEN
              DO 20 J = 1,N
                  DO 10 I = 1,M
                      C(I,J) = ZERO
   10             CONTINUE
   20         CONTINUE
          ELSE
              DO 40 J = 1,N
                  DO 30 I = 1,M
                      C(I,J) = BETA*C(I,J)
   30             CONTINUE
   40         CONTINUE
          END IF
          RETURN
      END IF
*
*     Start the operations.
*
      IF (LSAME(SIDE,'L')) THEN
*
*        Form  C := alpha*A*B + beta*C.
*
          IF (UPPER) THEN
              DO 70 J = 1,N
                  DO 60 I = 1,M
                      TEMP1 = ALPHA*B(I,J)
                      TEMP2 = ZERO
                      DO 50 K = 1,I - 1
                          C(K,J) = C(K,J) + TEMP1*A(K,I)
                          TEMP2 = TEMP2 + B(K,J)*A(K,I)
```

```
   50                 CONTINUE
                      IF (BETA.EQ.ZERO) THEN
                          C(I,J) = TEMP1*A(I,I) + ALPHA*TEMP2
                      ELSE
                          C(I,J) = BETA*C(I,J) + TEMP1*A(I,I) +
     +                            ALPHA*TEMP2
                      END IF
   60             CONTINUE
   70         CONTINUE
          ELSE
              DO 100 J = 1,N
                  DO 90 I = M,1,-1
                      TEMP1 = ALPHA*B(I,J)
                      TEMP2 = ZERO
                      DO 80 K = I + 1,M
                          C(K,J) = C(K,J) + TEMP1*A(K,I)
                          TEMP2 = TEMP2 + B(K,J)*A(K,I)
   80                 CONTINUE
                      IF (BETA.EQ.ZERO) THEN
                          C(I,J) = TEMP1*A(I,I) + ALPHA*TEMP2
                      ELSE
                          C(I,J) = BETA*C(I,J) + TEMP1*A(I,I) +
     +                            ALPHA*TEMP2
                      END IF
   90             CONTINUE
  100         CONTINUE
          END IF
      ELSE
*
*        Form  C := alpha*B*A + beta*C.
*
          DO 170 J = 1,N
              TEMP1 = ALPHA*A(J,J)
              IF (BETA.EQ.ZERO) THEN
                  DO 110 I = 1,M
                      C(I,J) = TEMP1*B(I,J)
  110             CONTINUE
              ELSE
                  DO 120 I = 1,M
                      C(I,J) = BETA*C(I,J) + TEMP1*B(I,J)
  120             CONTINUE
              END IF
              DO 140 K = 1,J - 1
                  IF (UPPER) THEN
                      TEMP1 = ALPHA*A(K,J)
                  ELSE
                      TEMP1 = ALPHA*A(J,K)
                  END IF
                  DO 130 I = 1,M
                      C(I,J) = C(I,J) + TEMP1*B(I,K)
```

```
  130             CONTINUE
  140         CONTINUE
              DO 160 K = J + 1,N
                  IF (UPPER) THEN
                      TEMP1 = ALPHA*A(J,K)
                  ELSE
                      TEMP1 = ALPHA*A(K,J)
                  END IF
                  DO 150 I = 1,M
                      C(I,J) = C(I,J) + TEMP1*B(I,K)
  150             CONTINUE
  160         CONTINUE
  170     CONTINUE
      END IF
*
      RETURN
*
*     End of CSYMM .
*
      END
```

———-

16.5.4 lisp code

— BLAS 3 csymm —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)) (zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (ignorable one zero))
  (defun csymm (side uplo m n alpha a lda b ldb$ beta c ldc)
    (declare (type (array f2cl-lib:complex8 (*)) c b a)
             (type (f2cl-lib:complex8) beta alpha)
             (type (f2cl-lib:integer4) ldc ldb$ lda n m)
             (type (string *) uplo side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (c f2cl-lib:complex8 c-%data% c-%offset%))
      (prog ((upper nil) (i 0) (info 0) (j 0) (k 0) (nrowa 0)
             (temp1 #C(0.0 0.0)) (temp2 #C(0.0 0.0)))
        (declare (type f2cl-lib:logical upper)
                 (type (f2cl-lib:integer4) i info j k nrowa)
                 (type (f2cl-lib:complex8) temp1 temp2))
```

```
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame side "L")
    (declare (ignore var-1))
    (when var-0 (setf side var-0))
    ret-val)
  (setf nrowa m))
 (t (setf nrowa n)))
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(setf info 0)
(cond
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame side "L")
      (declare (ignore var-1))
      (when var-0 (setf side var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame side "R")
      (declare (ignore var-1))
      (when var-0 (setf side var-0))
      ret-val)))
  (setf info 1))
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info 2))
 ((< m 0) (setf info 3)) ((< n 0) (setf info 4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nrowa)))
  (setf info 7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info 9))
 ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info 12)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "CSYMM " info)
    (declare (ignore var-0))
```

```
      (when var-1 (setf info var-1)))
    (go end_label)))
  (if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
      (go end_label))
  (cond
   ((= alpha zero)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                   c-%offset%)
                    zero)
            label10))
        label20)))
     (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                   c-%offset%)
                    (* beta
                       (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)))
            label30))
        label40))))
    (go end_label)))
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame side "L")
      (declare (ignore var-1))
      (when var-0 (setf side var-0))
      ret-val)
    (cond
     (upper
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf temp1
                    (* alpha
```

```
                    (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)))
          (setf temp2 zero)
          (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                        ((> k
                            (f2cl-lib:int-add i
                                              (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (k j) ((1 ldc) (1 *))
                                   c-%offset%)
                    (+
                     (f2cl-lib:fref c-%data% (k j) ((1 ldc) (1 *))
                                    c-%offset%)
                     (* temp1
                        (f2cl-lib:fref a-%data% (k i)
                                       ((1 lda) (1 *))
                                       a-%offset%))))
              (setf temp2
                    (+ temp2
                       (*
                        (f2cl-lib:fref b-%data% (k j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                        (f2cl-lib:fref a-%data% (k i)
                                       ((1 lda) (1 *))
                                       a-%offset%))))
             label50))
          (cond
           ((= beta zero)
            (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                 c-%offset%)
                  (+
                   (* temp1
                      (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                     a-%offset%))
                   (* alpha temp2))))
           (t
            (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                 c-%offset%)
                  (+
                   (* beta
                      (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%))
                   (* temp1
                      (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                     a-%offset%))
                   (* alpha temp2)))))
        label60))
    label70)))
(t
```

```
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
      (tagbody
        (setf temp1
                (* alpha
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)))
        (setf temp2 zero)
        (f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
                         (f2cl-lib:int-add k 1))
                       ((> k m) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data% (k j) ((1 ldc) (1 *))
                                 c-%offset%)
                    (+
                     (f2cl-lib:fref c-%data% (k j) ((1 ldc) (1 *))
                                    c-%offset%)
                     (* temp1
                        (f2cl-lib:fref a-%data% (k i)
                                       ((1 lda) (1 *))
                                       a-%offset%))))
            (setf temp2
                    (+ temp2
                       (*
                        (f2cl-lib:fref b-%data% (k j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                        (f2cl-lib:fref a-%data% (k i)
                                       ((1 lda) (1 *))
                                       a-%offset%))))
            label80))
        (cond
         ((= beta zero)
          (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                               c-%offset%)
                  (+
                   (* temp1
                      (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                     a-%offset%))
                   (* alpha temp2))))
         (t
          (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                               c-%offset%)
                  (+
                   (* beta
                      (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%))
                   (* temp1
```

```
                          (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                         a-%offset%))
                       (* alpha temp2)))))
           label90))
      label100)))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf temp1
             (* alpha
                (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                               a-%offset%)))
     (cond
      ((= beta zero)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* temp1
                    (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)))
           label110)))
      (t
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                c-%offset%)
                 (+
                  (* beta
                     (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%))
                  (* temp1
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%))))
           label120))))
     (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                   ((> k (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                    nil)
       (tagbody
         (cond
          (upper
           (setf temp1
                 (* alpha
                    (f2cl-lib:fref a-%data% (k j) ((1 lda) (1 *))
                                   a-%offset%))))
          (t
           (setf temp1
```

```
                    (* alpha
                       (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                      a-%offset%)))))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                      (+
                       (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)
                       (* temp1
                          (f2cl-lib:fref b-%data% (i k)
                                         ((1 ldb$) (1 *))
                                         b-%offset%))))
               label130))
           label140))
        (f2cl-lib:fdo (k (f2cl-lib:int-add j 1) (f2cl-lib:int-add k 1))
                      ((> k n) nil)
          (tagbody
            (cond
             (upper
              (setf temp1
                    (* alpha
                       (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                      a-%offset%))))
             (t
              (setf temp1
                    (* alpha
                       (f2cl-lib:fref a-%data% (k j) ((1 lda) (1 *))
                                      a-%offset%)))))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                      (+
                       (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)
                       (* temp1
                          (f2cl-lib:fref b-%data% (i k)
                                         ((1 ldb$) (1 *))
                                         b-%offset%))))
               label150))
           label160))
      label170))))
  (go end_label)
 end_label
  (return (values side uplo nil nil nil nil nil nil nil nil nil nil))))))
```

———-

16.6 csyr2k - symmetric rank 2k $\alpha AB^T + \alpha BA^T + \beta C$ or $\alpha A^T B + \alpha B^T A + \beta C$

16.6.1 Axiom unit tests

— **csyr2k.input** —

```
)set break resume
)sys rm -f csyr2k.output
)spool csyr2k.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

16.6.2 Axiom help page

— **csyr2k.help** —

```
======================================================================
csyr2k examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csyr2k (UPLO, TRANS, N, K, ALPHA, A, LDA, B, LDB, BETA, C,
           LDC)
           CSYR2K

Function/Subroutine Documentation
   subroutine csyr2k (character UPLO, character TRANS, integer N, integer K,
       complex ALPHA, complex, dimension(lda,*) A, integer LDA, complex,
       dimension(ldb,*) B, integer LDB, complex BETA, complex,
       dimension(ldc,*) C, integer LDC)
```

```
      CSYR2K

Purpose:
   CSYR2K  performs one of the symmetric rank 2k operations

   C := alpha*A*B**T + alpha*B*A**T + beta*C,

   or

   C := alpha*A**T*B + alpha*B**T*A + beta*C,

   where  alpha and beta  are scalars,  C is an  n by n symmetric matrix
   and  A and B  are  n by k  matrices  in the  first  case  and  k by n
   matrices in the second case.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On  entry,   UPLO  specifies  whether  the  upper  or  lower
   triangular part of the array C is to be referenced as follows:

   UPLO = 'U' or 'u'   Only the  upper triangular part of  C
   is to be referenced.

   UPLO = 'L' or 'l'   Only the  lower triangular part of  C
   is to be referenced.

 TRANS
   TRANS is CHARACTER*1
   On entry, TRANS specifies the operation to be performed as follows:

   TRANS = 'N' or 'n'    C := alpha*A*B**T + alpha*B*A**T + beta*C.

   TRANS = 'T' or 't'    C := alpha*A**T*B + alpha*B**T*A + beta*C.

 N
   N is INTEGER
   On entry,  N specifies the order of the matrix C.  N must be
   at least zero.

 K
   K is INTEGER
   On entry with  TRANS = 'N' or 'n',  K  specifies  the number
   of  columns  of the  matrices  A and B,  and on  entry  with
   TRANS = 'T' or 't',  K  specifies  the number of rows of the
   matrices  A and B.  K must be at least zero.

 ALPHA
   ALPHA is COMPLEX
```

```
  On entry, ALPHA specifies the scalar alpha.

A
  A is COMPLEX array of DIMENSION ( LDA, ka ), where ka is
  k  when  TRANS = 'N' or 'n',  and is  n  otherwise.

  Before entry with  TRANS = 'N' or 'n',  the  leading  n by k
  part of the array  A  must contain the matrix  A,  otherwise
  the leading  k by n  part of the array  A  must contain  the
  matrix A.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in  the  calling  (sub)  program.   When  TRANS = 'N' or 'n'
  then  LDA must be at least  max( 1, n ), otherwise  LDA must
  be at least  max( 1, k ).

B
  B is COMPLEX array of DIMENSION ( LDB, kb ), where kb is
  k  when  TRANS = 'N' or 'n',  and is  n  otherwise.
  Before entry with  TRANS = 'N' or 'n',  the  leading  n by k
  part of the array  B  must contain the matrix  B,  otherwise
  the leading  k by n  part of the array  B  must contain  the
  matrix B.

LDB
  LDB is INTEGER
  On entry, LDB specifies the first dimension of B as declared
  in  the  calling  (sub)  program.   When  TRANS = 'N' or 'n'
  then  LDB must be at least  max( 1, n ), otherwise  LDB must
  be at least  max( 1, k ).

BETA
  BETA is COMPLEX
  On entry, BETA specifies the scalar beta.

C
  C is COMPLEX array of DIMENSION ( LDC, n ).

  Before entry  with  UPLO = 'U' or 'u',  the leading  n by n
  upper triangular part of the array C must contain the upper
  triangular part  of the  symmetric matrix  and the strictly
  lower triangular part of C is not referenced.  On exit, the
  upper triangular part of the array  C is overwritten by the
  upper triangular part of the updated matrix.

  Before entry  with  UPLO = 'L' or 'l',  the leading  n by n
  lower triangular part of the array C must contain the lower
  triangular part  of the  symmetric matrix  and the strictly
```

```
  upper triangular part of C is not referenced.  On exit, the
  lower triangular part of the array  C is overwritten by the
  lower triangular part of the updated matrix.

LDC
  LDC is INTEGER
  On entry, LDC specifies the first dimension of C as declared
  in  the  calling  (sub)  program.  LDC  must  be  at  least
  max( 1, n ).
```

———-

16.6.3 fortran code

— **csyr2k.f** —

```
*  ===================================================================
      SUBROUTINE CSYR2K(UPLO,TRANS,N,K,ALPHA,A,LDA,B,LDB,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX ALPHA,BETA
      INTEGER K,LDA,LDB,LDC,N
      CHARACTER TRANS,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),B(LDB,*),C(LDC,*)
*     ..
*
*  ===================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP1,TEMP2
```

```
      INTEGER I,INFO,J,L,NROWA
      LOGICAL UPPER
*     ..
*     .. Parameters ..
      COMPLEX ONE
      PARAMETER (ONE= (1.0E+0,0.0E+0))
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*
*     Test the input parameters.
*
      IF (LSAME(TRANS,'N')) THEN
          NROWA = N
      ELSE
          NROWA = K
      END IF
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 1
      ELSE IF ((.NOT.LSAME(TRANS,'N')) .AND.
     +         (.NOT.LSAME(TRANS,'T'))) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
          INFO = 3
      ELSE IF (K.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 7
      ELSE IF (LDB.LT.MAX(1,NROWA)) THEN
          INFO = 9
      ELSE IF (LDC.LT.MAX(1,N)) THEN
          INFO = 12
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CSYR2K',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (((ALPHA.EQ.ZERO).OR.
     +    (K.EQ.0)).AND. (BETA.EQ.ONE))) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
          IF (UPPER) THEN
```

```
            IF (BETA.EQ.ZERO) THEN
                DO 20 J = 1,N
                    DO 10 I = 1,J
                        C(I,J) = ZERO
   10               CONTINUE
   20           CONTINUE
            ELSE
                DO 40 J = 1,N
                    DO 30 I = 1,J
                        C(I,J) = BETA*C(I,J)
   30               CONTINUE
   40           CONTINUE
            END IF
        ELSE
            IF (BETA.EQ.ZERO) THEN
                DO 60 J = 1,N
                    DO 50 I = J,N
                        C(I,J) = ZERO
   50               CONTINUE
   60           CONTINUE
            ELSE
                DO 80 J = 1,N
                    DO 70 I = J,N
                        C(I,J) = BETA*C(I,J)
   70               CONTINUE
   80           CONTINUE
            END IF
        END IF
        RETURN
    END IF
*
*     Start the operations.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  C := alpha*A*B**T + alpha*B*A**T + C.
*
         IF (UPPER) THEN
            DO 130 J = 1,N
                IF (BETA.EQ.ZERO) THEN
                    DO 90 I = 1,J
                        C(I,J) = ZERO
   90               CONTINUE
                ELSE IF (BETA.NE.ONE) THEN
                    DO 100 I = 1,J
                        C(I,J) = BETA*C(I,J)
  100               CONTINUE
                END IF
                DO 120 L = 1,K
                    IF ((A(J,L).NE.ZERO) .OR. (B(J,L).NE.ZERO)) THEN
```

```
                        TEMP1 = ALPHA*B(J,L)
                        TEMP2 = ALPHA*A(J,L)
                        DO 110 I = 1,J
                            C(I,J) = C(I,J) + A(I,L)*TEMP1 +
     +                              B(I,L)*TEMP2
  110                   CONTINUE
                    END IF
  120             CONTINUE
  130         CONTINUE
          ELSE
              DO 180 J = 1,N
                  IF (BETA.EQ.ZERO) THEN
                      DO 140 I = J,N
                          C(I,J) = ZERO
  140                 CONTINUE
                  ELSE IF (BETA.NE.ONE) THEN
                      DO 150 I = J,N
                          C(I,J) = BETA*C(I,J)
  150                 CONTINUE
                  END IF
                  DO 170 L = 1,K
                      IF ((A(J,L).NE.ZERO) .OR. (B(J,L).NE.ZERO)) THEN
                          TEMP1 = ALPHA*B(J,L)
                          TEMP2 = ALPHA*A(J,L)
                          DO 160 I = J,N
                              C(I,J) = C(I,J) + A(I,L)*TEMP1 +
     +                              B(I,L)*TEMP2
  160                     CONTINUE
                      END IF
  170             CONTINUE
  180         CONTINUE
          END IF
      ELSE
*
*        Form  C := alpha*A**T*B + alpha*B**T*A + C.
*
          IF (UPPER) THEN
              DO 210 J = 1,N
                  DO 200 I = 1,J
                      TEMP1 = ZERO
                      TEMP2 = ZERO
                      DO 190 L = 1,K
                          TEMP1 = TEMP1 + A(L,I)*B(L,J)
                          TEMP2 = TEMP2 + B(L,I)*A(L,J)
  190                 CONTINUE
                      IF (BETA.EQ.ZERO) THEN
                          C(I,J) = ALPHA*TEMP1 + ALPHA*TEMP2
                      ELSE
                          C(I,J) = BETA*C(I,J) + ALPHA*TEMP1 +
     +                             ALPHA*TEMP2
```

```
                    END IF
  200             CONTINUE
  210         CONTINUE
          ELSE
              DO 240 J = 1,N
                  DO 230 I = J,N
                      TEMP1 = ZERO
                      TEMP2 = ZERO
                      DO 220 L = 1,K
                          TEMP1 = TEMP1 + A(L,I)*B(L,J)
                          TEMP2 = TEMP2 + B(L,I)*A(L,J)
  220                 CONTINUE
                      IF (BETA.EQ.ZERO) THEN
                          C(I,J) = ALPHA*TEMP1 + ALPHA*TEMP2
                      ELSE
                          C(I,J) = BETA*C(I,J) + ALPHA*TEMP1 +
     +                             ALPHA*TEMP2
                      END IF
  230             CONTINUE
  240         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of CSYR2K.
*
      END
```

———-

16.6.4 lisp code

— BLAS 3 csyr2k —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)) (zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (ignorable one zero))
  (defun csyr2k (uplo trans n k alpha a lda b ldb$ beta c ldc)
    (declare (type (array f2cl-lib:complex8 (*)) c b a)
             (type (f2cl-lib:complex8) beta alpha)
             (type (f2cl-lib:integer4) ldc ldb$ lda k n)
             (type (string *) trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
```

```
    (a f2cl-lib:complex8 a-%data% a-%offset%)
    (b f2cl-lib:complex8 b-%data% b-%offset%)
    (c f2cl-lib:complex8 c-%data% c-%offset%))
  (prog ((upper nil) (i 0) (info 0) (j 0) (l 0) (nrowa 0)
         (temp1 #C(0.0 0.0)) (temp2 #C(0.0 0.0)))
    (declare (type f2cl-lib:logical upper)
             (type (f2cl-lib:integer4) i info j l nrowa)
             (type (f2cl-lib:complex8) temp1 temp2))
    (cond
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame trans "N")
        (declare (ignore var-1))
        (when var-0 (setf trans var-0))
        ret-val)
      (setf nrowa n))
     (t (setf nrowa k)))
    (setf upper
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
    (setf info 0)
    (cond
     ((and (not upper)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
      (setf info 1))
     ((and
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
      (setf info 2))
     ((< n 0) (setf info 3)) ((< k 0) (setf info 4))
     ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nrowa)))
      (setf info 7))
     ((< ldb$
```

```
      (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nrowa)))
   (setf info 9))
  ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
   (setf info 12)))
 (cond
  ((/= info 0)
   (multiple-value-bind (var-0 var-1)
       (xerbla "CSYR2K" info)
     (declare (ignore var-0))
     (when var-1 (setf info var-1)))
   (go end_label)))
 (if (or (= n 0) (and (or (= alpha zero) (= k 0)) (= beta one)))
     (go end_label))
 (cond
  ((= alpha zero)
   (cond
    (upper
     (cond
      ((= beta zero)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i j) nil)
             (tagbody
               (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                       zero)
             label10))
         label20)))
      (t
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i j) nil)
             (tagbody
               (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                       (* beta
                          (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                         c-%offset%)))
             label30))
         label40)))))
    (t
     (cond
      ((= beta zero)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
```

```
          (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                    zero)
             label50))
         label60)))
     (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                    (* beta
                       (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)))
             label70))
         label80))))))
  (go end_label)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame trans "N")
    (declare (ignore var-1))
    (when var-0 (setf trans var-0))
    ret-val)
  (cond
   (upper
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
         ((= beta zero)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                    zero)
             label90)))
         ((/= beta one)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                    (* beta
```

```
                        (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)))
            label100))))
      (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                    ((> l k) nil)
        (tagbody
          (cond
           ((or (/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
                (/= (f2cl-lib:fref b (j l) ((1 ldb$) (1 *))) zero))
            (setf temp1
                    (* alpha
                       (f2cl-lib:fref b-%data% (j l) ((1 ldb$) (1 *))
                                      b-%offset%)))
            (setf temp2
                    (* alpha
                       (f2cl-lib:fref a-%data% (j l) ((1 lda) (1 *))
                                      a-%offset%)))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i j) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                      (+
                       (f2cl-lib:fref c-%data% (i j)
                                      ((1 ldc) (1 *)) c-%offset%)
                       (*
                        (f2cl-lib:fref a-%data% (i l)
                                       ((1 lda) (1 *)) a-%offset%)
                        temp1)
                       (*
                        (f2cl-lib:fref b-%data% (i l)
                                       ((1 ldb$) (1 *)) b-%offset%)
                        temp2)))
              label110))))
        label120))
    label130)))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
      ((= beta zero)
       (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                c-%offset%)
                 zero)
         label140)))
      ((/= beta one)
```

```
          (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                      (* beta
                         (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                        c-%offset%)))
             label150))))
        (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                      ((> l k) nil)
          (tagbody
            (cond
             ((or (/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
                  (/= (f2cl-lib:fref b (j l) ((1 ldb$) (1 *))) zero))
              (setf temp1
                      (* alpha
                         (f2cl-lib:fref b-%data% (j l) ((1 ldb$) (1 *))
                                        b-%offset%)))
              (setf temp2
                      (* alpha
                         (f2cl-lib:fref a-%data% (j l) ((1 lda) (1 *))
                                        a-%offset%)))
              (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                            ((> i n) nil)
                (tagbody
                  (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)
                          (+
                           (f2cl-lib:fref c-%data% (i j)
                                          ((1 ldc) (1 *)) c-%offset%)
                           (*
                            (f2cl-lib:fref a-%data% (i l)
                                           ((1 lda) (1 *)) a-%offset%)
                            temp1)
                           (*
                            (f2cl-lib:fref b-%data% (i l)
                                           ((1 ldb$) (1 *)) b-%offset%)
                            temp2)))
                 label160))))
           label170))
      label180)))))
(t
 (cond
  (upper
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i j) nil)
```

```
          (tagbody
            (setf temp1 zero)
            (setf temp2 zero)
            (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                          ((> l k) nil)
              (tagbody
                (setf temp1
                        (+ temp1
                           (*
                            (f2cl-lib:fref a-%data% (l i)
                                           ((1 lda) (1 *)) a-%offset%)
                            (f2cl-lib:fref b-%data% (l j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%))))
                (setf temp2
                        (+ temp2
                           (*
                            (f2cl-lib:fref b-%data% (l i)
                                           ((1 ldb$) (1 *)) b-%offset%)
                            (f2cl-lib:fref a-%data% (l j)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
               label190))
            (cond
             ((= beta zero)
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                   c-%offset%)
                      (+ (* alpha temp1) (* alpha temp2))))
             (t
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                   c-%offset%)
                      (+
                       (* beta
                          (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                         c-%offset%))
                       (* alpha temp1) (* alpha temp2)))))
           label200))
        label210)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf temp1 zero)
             (setf temp2 zero)
             (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                           ((> l k) nil)
               (tagbody
```

```
                    (setf temp1
                            (+ temp1
                               (*
                                (f2cl-lib:fref a-%data% (l i)
                                               ((1 lda) (1 *)) a-%offset%)
                                (f2cl-lib:fref b-%data% (l j)
                                               ((1 ldb$) (1 *))
                                               b-%offset%))))
                    (setf temp2
                            (+ temp2
                               (*
                                (f2cl-lib:fref b-%data% (l i)
                                               ((1 ldb$) (1 *)) b-%offset%)
                                (f2cl-lib:fref a-%data% (l j)
                                               ((1 lda) (1 *))
                                               a-%offset%))))
                  label220))
                (cond
                 ((= beta zero)
                  (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)
                          (+ (* alpha temp1) (* alpha temp2))))
                 (t
                  (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)
                          (+
                           (* beta
                              (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                             c-%offset%))
                           (* alpha temp1) (* alpha temp2)))))
              label230))
            label240))))))
  (go end_label)
 end_label
  (return (values uplo trans nil nil nil nil nil nil nil nil nil nil))))))
```

———-

16.7 csyrk - symmetric $\alpha AA^T + \beta C$ or $\alpha A^T A + \beta C$

16.7.1 Axiom unit tests

— **csyrk.input** —

```
)set break resume
)sys rm -f csyrk.output
```

```
)spool csyrk.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

16.7.2 Axiom help page

— **csyrk.help** —

```
=====================================================================
csyrk examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csyrk (UPLO, TRANS, N, K, ALPHA, A, LDA, BETA, C, LDC)
           CSYRK

Function/Subroutine Documentation
   subroutine csyrk (character UPLO, character TRANS, integer N, integer K,
       complex ALPHA, complex, dimension(lda,*) A, integer LDA, complex BETA,
       complex, dimension(ldc,*) C, integer LDC)
       CSYRK

Purpose:
   CSYRK  performs one of the symmetric rank k operations

   C := alpha*A*A**T + beta*C,

   or

   C := alpha*A**T*A + beta*C,

   where  alpha and beta  are scalars,  C is an  n by n symmetric matrix
   and  A  is an  n by k  matrix in the first case and a  k by n  matrix
   in the second case.

Parameters:
```

```
UPLO
  UPLO is CHARACTER*1
  On  entry,  UPLO  specifies  whether  the  upper  or  lower
  triangular part of the array C is to be referenced as follows:

  UPLO = 'U' or 'u'   Only the  upper triangular part of  C
  is to be referenced.

  UPLO = 'L' or 'l'   Only the  lower triangular part of  C
  is to be referenced.

TRANS
  TRANS is CHARACTER*1
  On entry,  TRANS  specifies the operation to be performed as follows:

  TRANS = 'N' or 'n'   C := alpha*A*A**T + beta*C.

  TRANS = 'T' or 't'   C := alpha*A**T*A + beta*C.

N
  N is INTEGER
  On entry,  N specifies the order of the matrix C.  N must be
  at least zero.

K
  K is INTEGER
  On entry with  TRANS = 'N' or 'n',  K  specifies  the number
  of  columns  of  the  matrix  A,  and  on  entry  with
  TRANS = 'T' or 't',  K  specifies  the number of rows of the
  matrix A.  K must be at least zero.

ALPHA
  ALPHA is COMPLEX
  On entry, ALPHA specifies the scalar alpha.

A
  A is COMPLEX array of DIMENSION ( LDA, ka ), where ka is
  k  when  TRANS = 'N' or 'n',  and is  n  otherwise.

  Before entry with  TRANS = 'N' or 'n',  the  leading  n by k
  part of the array  A  must contain the matrix  A,  otherwise
  the leading  k by n  part of the array  A  must contain  the
  matrix A.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in  the  calling  (sub)  program.  When  TRANS = 'N' or 'n'
  then  LDA must be at least  max( 1, n ), otherwise  LDA must
  be at least  max( 1, k ).
```

```
BETA
  BETA is COMPLEX
  On entry, BETA specifies the scalar beta.

C
  C is COMPLEX array of DIMENSION ( LDC, n ).

  Before entry  with  UPLO = 'U' or 'u',  the leading  n by n
  upper triangular part of the array C must contain the upper
  triangular part  of the  symmetric matrix  and the strictly
  lower triangular part of C is not referenced.  On exit, the
  upper triangular part of the array  C is overwritten by the
  upper triangular part of the updated matrix.

  Before entry  with  UPLO = 'L' or 'l',  the leading  n by n
  lower triangular part of the array C must contain the lower
  triangular part  of the  symmetric matrix  and the strictly
  upper triangular part of C is not referenced.  On exit, the
  lower triangular part of the array  C is overwritten by the
  lower triangular part of the updated matrix.

LDC
  LDC is INTEGER
  On entry, LDC specifies the first dimension of C as declared
  in  the  calling  (sub)  program.  LDC  must  be  at  least
  max( 1, n ).
```

———-

16.7.3 fortran code

— **csyrk.f** —

```
*  ===================================================================
      SUBROUTINE CSYRK(UPLO,TRANS,N,K,ALPHA,A,LDA,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX ALPHA,BETA
      INTEGER K,LDA,LDC,N
      CHARACTER TRANS,UPLO
*     ..
```

```
*     .. Array Arguments ..
      COMPLEX A(LDA,*),C(LDC,*)
*     ..
*
*  =====================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP
      INTEGER I,INFO,J,L,NROWA
      LOGICAL UPPER
*     ..
*     .. Parameters ..
      COMPLEX ONE
      PARAMETER (ONE= (1.0E+0,0.0E+0))
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*
*     Test the input parameters.
*
      IF (LSAME(TRANS,'N')) THEN
          NROWA = N
      ELSE
          NROWA = K
      END IF
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 1
      ELSE IF ((.NOT.LSAME(TRANS,'N')) .AND.
     +         (.NOT.LSAME(TRANS,'T'))) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
          INFO = 3
      ELSE IF (K.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 7
      ELSE IF (LDC.LT.MAX(1,N)) THEN
```

```
          INFO = 10
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CSYRK ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (((ALPHA.EQ.ZERO).OR.
     +    (K.EQ.0)).AND. (BETA.EQ.ONE))) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
          IF (UPPER) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 20 J = 1,N
                      DO 10 I = 1,J
                          C(I,J) = ZERO
   10                 CONTINUE
   20             CONTINUE
              ELSE
                  DO 40 J = 1,N
                      DO 30 I = 1,J
                          C(I,J) = BETA*C(I,J)
   30                 CONTINUE
   40             CONTINUE
              END IF
          ELSE
              IF (BETA.EQ.ZERO) THEN
                  DO 60 J = 1,N
                      DO 50 I = J,N
                          C(I,J) = ZERO
   50                 CONTINUE
   60             CONTINUE
              ELSE
                  DO 80 J = 1,N
                      DO 70 I = J,N
                          C(I,J) = BETA*C(I,J)
   70                 CONTINUE
   80             CONTINUE
              END IF
          END IF
          RETURN
      END IF
*
*     Start the operations.
*
      IF (LSAME(TRANS,'N')) THEN
```

```
*
*        Form  C := alpha*A*A**T + beta*C.
*
         IF (UPPER) THEN
             DO 130 J = 1,N
                 IF (BETA.EQ.ZERO) THEN
                     DO 90 I = 1,J
                         C(I,J) = ZERO
   90                CONTINUE
                 ELSE IF (BETA.NE.ONE) THEN
                     DO 100 I = 1,J
                         C(I,J) = BETA*C(I,J)
  100                CONTINUE
                 END IF
                 DO 120 L = 1,K
                     IF (A(J,L).NE.ZERO) THEN
                         TEMP = ALPHA*A(J,L)
                         DO 110 I = 1,J
                             C(I,J) = C(I,J) + TEMP*A(I,L)
  110                    CONTINUE
                     END IF
  120            CONTINUE
  130         CONTINUE
         ELSE
             DO 180 J = 1,N
                 IF (BETA.EQ.ZERO) THEN
                     DO 140 I = J,N
                         C(I,J) = ZERO
  140                CONTINUE
                 ELSE IF (BETA.NE.ONE) THEN
                     DO 150 I = J,N
                         C(I,J) = BETA*C(I,J)
  150                CONTINUE
                 END IF
                 DO 170 L = 1,K
                     IF (A(J,L).NE.ZERO) THEN
                         TEMP = ALPHA*A(J,L)
                         DO 160 I = J,N
                             C(I,J) = C(I,J) + TEMP*A(I,L)
  160                    CONTINUE
                     END IF
  170            CONTINUE
  180         CONTINUE
         END IF
     ELSE
*
*        Form  C := alpha*A**T*A + beta*C.
*
         IF (UPPER) THEN
             DO 210 J = 1,N
```

```
                DO 200 I = 1,J
                    TEMP = ZERO
                    DO 190 L = 1,K
                        TEMP = TEMP + A(L,I)*A(L,J)
 190                CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP
                    ELSE
                        C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                    END IF
 200            CONTINUE
 210        CONTINUE
        ELSE
            DO 240 J = 1,N
                DO 230 I = J,N
                    TEMP = ZERO
                    DO 220 L = 1,K
                        TEMP = TEMP + A(L,I)*A(L,J)
 220                CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP
                    ELSE
                        C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                    END IF
 230            CONTINUE
 240        CONTINUE
        END IF
    END IF
*
    RETURN
*
*   End of CSYRK .
*
    END
```

———-

16.7.4 lisp code

— BLAS 3 csyrk —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)) (zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (ignorable one zero))
  (defun csyrk (uplo trans n k alpha a lda beta c ldc)
    (declare (type (array f2cl-lib:complex8 (*)) c a)
```

```
          (type (f2cl-lib:complex8) beta alpha)
          (type (f2cl-lib:integer4) ldc lda k n)
          (type (string *) trans uplo))
(f2cl-lib:with-multi-array-data
    ((uplo character uplo-%data% uplo-%offset%)
     (trans character trans-%data% trans-%offset%)
     (a f2cl-lib:complex8 a-%data% a-%offset%)
     (c f2cl-lib:complex8 c-%data% c-%offset%))
  (prog ((upper nil) (i 0) (info 0) (j 0) (l 0) (nrowa 0)
         (temp #C(0.0 0.0)))
    (declare (type f2cl-lib:logical upper)
             (type (f2cl-lib:integer4) i info j l nrowa)
             (type (f2cl-lib:complex8) temp))
    (cond
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame trans "N")
        (declare (ignore var-1))
        (when var-0 (setf trans var-0))
        ret-val)
      (setf nrowa n))
     (t (setf nrowa k)))
    (setf upper
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
    (setf info 0)
    (cond
     ((and (not upper)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
      (setf info 1))
     ((and
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
```

```
  (setf info 2))
 ((< n 0) (setf info 3)) ((< k 0) (setf info 4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nrowa)))
  (setf info 7))
 ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info 10)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "CSYRK " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= n 0) (and (or (= alpha zero) (= k 0)) (= beta one)))
    (go end_label))
(cond
 ((= alpha zero)
  (cond
   (upper
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                    zero)
             label10))
         label20)))
     (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                    (* beta
                       (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)))
             label30))
         label40)))))
   (t
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                    ((> j n) nil)
          (tagbody
            (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)
                      zero)
              label50))
           label60)))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
          (tagbody
            (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                      c-%offset%)
                      (* beta
                         (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                        c-%offset%)))
              label70))
           label80))))))
  (go end_label)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame trans "N")
    (declare (ignore var-1))
    (when var-0 (setf trans var-0))
    ret-val)
  (cond
   (upper
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
         ((= beta zero)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                    zero)
            label90)))
         ((/= beta one)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
```

```
                                    c-%offset%)
                          (* beta
                             (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                            c-%offset%)))
               label100))))
         (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                       ((> l k) nil)
           (tagbody
             (cond
              ((/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
               (setf temp
                       (* alpha
                          (f2cl-lib:fref a-%data% (j l) ((1 lda) (1 *))
                                         a-%offset%)))
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i j) nil)
                 (tagbody
                   (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                        c-%offset%)
                         (+
                          (f2cl-lib:fref c-%data% (i j)
                                         ((1 ldc) (1 *)) c-%offset%)
                          (* temp
                             (f2cl-lib:fref a-%data% (i l)
                                            ((1 lda) (1 *))
                                            a-%offset%))))
                 label110))))
           label120))
       label130)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (cond
          ((= beta zero)
           (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                     zero)
               label140)))
          ((/= beta one)
           (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                     (* beta
                        (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
```

```
                                        c-%offset%)))
             label150))))
        (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                      ((> l k) nil)
          (tagbody
            (cond
             ((/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
              (setf temp
                      (* alpha
                         (f2cl-lib:fref a-%data% (j l) ((1 lda) (1 *))
                                        a-%offset%)))
              (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                            ((> i n) nil)
                (tagbody
                  (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)
                        (+
                         (f2cl-lib:fref c-%data% (i j)
                                        ((1 ldc) (1 *)) c-%offset%)
                         (* temp
                            (f2cl-lib:fref a-%data% (i l)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
                label160))))
          label170))
      label180)))))
(t
 (cond
  (upper
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i j) nil)
         (tagbody
           (setf temp zero)
           (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                         ((> l k) nil)
             (tagbody
               (setf temp
                       (+ temp
                          (*
                           (f2cl-lib:fref a-%data% (l i)
                                          ((1 lda) (1 *)) a-%offset%)
                           (f2cl-lib:fref a-%data% (l j)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
               label190))
           (cond
            ((= beta zero)
```

```
                 (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)
                         (* alpha temp)))
                (t
                 (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                       c-%offset%)
                         (+ (* alpha temp)
                            (* beta
                               (f2cl-lib:fref c-%data% (i j)
                                              ((1 ldc) (1 *))
                                              c-%offset%))))))
            label200))
        label210)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf temp zero)
             (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                           ((> l k) nil)
               (tagbody
                 (setf temp
                         (+ temp
                            (*
                             (f2cl-lib:fref a-%data% (l i)
                                            ((1 lda) (1 *)) a-%offset%)
                             (f2cl-lib:fref a-%data% (l j)
                                            ((1 lda) (1 *))
                                            a-%offset%))))
                 label220))
             (cond
              ((= beta zero)
               (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                       (* alpha temp)))
              (t
               (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                     c-%offset%)
                       (+ (* alpha temp)
                          (* beta
                             (f2cl-lib:fref c-%data% (i j)
                                            ((1 ldc) (1 *))
                                            c-%offset%))))))
            label230))
        label240))))))
 (go end_label)
end_label
```

```
(return (values uplo trans nil nil nil nil nil nil nil nil))))))
```

———-

16.8 ctrmm - $\alpha op(A)B$ or $\alpha Bop(A)$, $op(A) = A$ or $op(A) = A^T$ or $op(A) = A^H$

16.8.1 Axiom unit tests

— **ctrmm.input** —

```
)set break resume
)sys rm -f ctrmm.output
)spool ctrmm.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

16.8.2 Axiom help page

— **ctrmm.help** —

```
======================================================================
ctrmm examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ctrmm (SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B,
           LDB)
           CTRMM

Function/Subroutine Documentation
   subroutine ctrmm (character SIDE, character UPLO, character TRANSA,
```

```
      character DIAG, integer M, integer N, complex ALPHA, complex,
      dimension(lda,*) A, integer LDA, complex, dimension(ldb,*) B, integer
      LDB)
      CTRMM

Purpose:
   CTRMM  performs one of the matrix-matrix operations

   B := alpha*op( A )*B,   or   B := alpha*B*op( A )

   where  alpha  is a scalar,  B  is an m by n matrix,  A  is a unit, or
   non-unit,  upper or lower triangular matrix  and  op( A )  is one  of

   op( A ) = A   or   op( A ) = A**T   or   op( A ) = A**H.

Parameters:

 SIDE
   SIDE is CHARACTER*1
   On entry,  SIDE specifies whether  op( A ) multiplies B from
   the left or right as follows:

   SIDE = 'L' or 'l'   B := alpha*op( A )*B.

   SIDE = 'R' or 'r'   B := alpha*B*op( A ).

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the matrix A is an upper or
   lower triangular matrix as follows:

   UPLO = 'U' or 'u'   A is an upper triangular matrix.

   UPLO = 'L' or 'l'   A is a lower triangular matrix.

 TRANSA
   TRANSA is CHARACTER*1
   On entry, TRANSA specifies the form of op( A ) to be used in
   the matrix multiplication as follows:

   TRANSA = 'N' or 'n'   op( A ) = A.

   TRANSA = 'T' or 't'   op( A ) = A**T.

   TRANSA = 'C' or 'c'   op( A ) = A**H.

 DIAG
   DIAG is CHARACTER*1
   On entry, DIAG specifies whether or not A is unit triangular
   as follows:
```

```
  DIAG = 'U' or 'u'   A is assumed to be unit triangular.

  DIAG = 'N' or 'n'   A is not assumed to be unit triangular.

M
  M is INTEGER
  On entry, M specifies the number of rows of B. M must be at
  least zero.

N
  N is INTEGER
  On entry, N specifies the number of columns of B.  N must be
  at least zero.

ALPHA
  ALPHA is COMPLEX
  On entry,  ALPHA specifies the scalar  alpha. When  alpha is
  zero then  A is not referenced and  B need not be set before entry.

A
  A is COMPLEX array of DIMENSION ( LDA, k ), where k is m
  when  SIDE = 'L' or 'l'  and is  n  when  SIDE = 'R' or 'r'.

  Before entry  with  UPLO = 'U' or 'u',  the  leading  k by k
  upper triangular part of the array  A must contain the upper
  triangular matrix  and the strictly lower triangular part of
  A is not referenced.

  Before entry  with  UPLO = 'L' or 'l',  the  leading  k by k
  lower triangular part of the array  A must contain the lower
  triangular matrix  and the strictly upper triangular part of
  A is not referenced.

  Note that when  DIAG = 'U' or 'u',  the diagonal elements of
  A  are not referenced either,  but are assumed to be  unity.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program.  When  SIDE = 'L' or 'l'  then
  LDA  must be at least  max( 1, m ),  when  SIDE = 'R' or 'r'
  then LDA must be at least max( 1, n ).

B
  B is COMPLEX array of DIMENSION ( LDB, n ).
  Before entry,  the leading  m by n part of the array  B must
  contain the matrix  B,  and  on exit  is overwritten  by the
  transformed matrix.
```

```
 LDB
   LDB is INTEGER
   On entry, LDB specifies the first dimension of B as declared
   in  the  calling  (sub)  program.   LDB  must  be  at  least
   max( 1, m ).
```

———-

16.8.3 fortran code

— **ctrmm.f** —

```
*  =====================================================================
      SUBROUTINE CTRMM(SIDE,UPLO,TRANSA,DIAG,M,N,ALPHA,A,LDA,B,LDB)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX ALPHA
      INTEGER LDA,LDB,M,N
      CHARACTER DIAG,SIDE,TRANSA,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),B(LDB,*)
*     ..
*
*  =====================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG,MAX
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP
      INTEGER I,INFO,J,K,NROWA
      LOGICAL LSIDE,NOCONJ,NOUNIT,UPPER
*     ..
*     .. Parameters ..
```

```
      COMPLEX ONE
      PARAMETER (ONE= (1.0E+0,0.0E+0))
      COMPLEX ZERO
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*
*     Test the input parameters.
*
      LSIDE = LSAME(SIDE,'L')
      IF (LSIDE) THEN
          NROWA = M
      ELSE
          NROWA = N
      END IF
      NOCONJ = LSAME(TRANSA,'T')
      NOUNIT = LSAME(DIAG,'N')
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.LSIDE) .AND. (.NOT.LSAME(SIDE,'R'))) THEN
          INFO = 1
      ELSE IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 2
      ELSE IF ((.NOT.LSAME(TRANSA,'N')) .AND.
     +         (.NOT.LSAME(TRANSA,'T')) .AND.
     +         (.NOT.LSAME(TRANSA,'C'))) THEN
          INFO = 3
      ELSE IF ((.NOT.LSAME(DIAG,'U')) .AND. (.NOT.LSAME(DIAG,'N'))) THEN
          INFO = 4
      ELSE IF (M.LT.0) THEN
          INFO = 5
      ELSE IF (N.LT.0) THEN
          INFO = 6
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 9
      ELSE IF (LDB.LT.MAX(1,M)) THEN
          INFO = 11
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CTRMM ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (M.EQ.0 .OR. N.EQ.0) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
```

```
         DO 20 J = 1,N
             DO 10 I = 1,M
                 B(I,J) = ZERO
   10        CONTINUE
   20    CONTINUE
         RETURN
      END IF
*
*     Start the operations.
*
      IF (LSIDE) THEN
         IF (LSAME(TRANSA,'N')) THEN
*
*          Form  B := alpha*A*B.
*
             IF (UPPER) THEN
                 DO 50 J = 1,N
                     DO 40 K = 1,M
                         IF (B(K,J).NE.ZERO) THEN
                             TEMP = ALPHA*B(K,J)
                             DO 30 I = 1,K - 1
                                 B(I,J) = B(I,J) + TEMP*A(I,K)
   30                        CONTINUE
                             IF (NOUNIT) TEMP = TEMP*A(K,K)
                             B(K,J) = TEMP
                         END IF
   40                CONTINUE
   50            CONTINUE
             ELSE
                 DO 80 J = 1,N
                     DO 70 K = M,1,-1
                         IF (B(K,J).NE.ZERO) THEN
                             TEMP = ALPHA*B(K,J)
                             B(K,J) = TEMP
                             IF (NOUNIT) B(K,J) = B(K,J)*A(K,K)
                             DO 60 I = K + 1,M
                                 B(I,J) = B(I,J) + TEMP*A(I,K)
   60                        CONTINUE
                         END IF
   70                CONTINUE
   80            CONTINUE
             END IF
         ELSE
*
*          Form  B := alpha*A**T*B   or   B := alpha*A**H*B.
*
             IF (UPPER) THEN
                 DO 120 J = 1,N
                     DO 110 I = M,1,-1
                         TEMP = B(I,J)
```

```
                        IF (NOCONJ) THEN
                            IF (NOUNIT) TEMP = TEMP*A(I,I)
                            DO 90 K = 1,I - 1
                                TEMP = TEMP + A(K,I)*B(K,J)
   90                       CONTINUE
                        ELSE
                            IF (NOUNIT) TEMP = TEMP*CONJG(A(I,I))
                            DO 100 K = 1,I - 1
                                TEMP = TEMP + CONJG(A(K,I))*B(K,J)
  100                       CONTINUE
                        END IF
                        B(I,J) = ALPHA*TEMP
  110                 CONTINUE
  120             CONTINUE
            ELSE
                DO 160 J = 1,N
                    DO 150 I = 1,M
                        TEMP = B(I,J)
                        IF (NOCONJ) THEN
                            IF (NOUNIT) TEMP = TEMP*A(I,I)
                            DO 130 K = I + 1,M
                                TEMP = TEMP + A(K,I)*B(K,J)
  130                       CONTINUE
                        ELSE
                            IF (NOUNIT) TEMP = TEMP*CONJG(A(I,I))
                            DO 140 K = I + 1,M
                                TEMP = TEMP + CONJG(A(K,I))*B(K,J)
  140                       CONTINUE
                        END IF
                        B(I,J) = ALPHA*TEMP
  150                 CONTINUE
  160             CONTINUE
            END IF
        END IF
    ELSE
        IF (LSAME(TRANSA,'N')) THEN
*
*         Form  B := alpha*B*A.
*
            IF (UPPER) THEN
                DO 200 J = N,1,-1
                    TEMP = ALPHA
                    IF (NOUNIT) TEMP = TEMP*A(J,J)
                    DO 170 I = 1,M
                        B(I,J) = TEMP*B(I,J)
  170                 CONTINUE
                    DO 190 K = 1,J - 1
                        IF (A(K,J).NE.ZERO) THEN
                            TEMP = ALPHA*A(K,J)
                            DO 180 I = 1,M
```

```
                                B(I,J) = B(I,J) + TEMP*B(I,K)
 180                        CONTINUE
                        END IF
 190                CONTINUE
 200            CONTINUE
            ELSE
                DO 240 J = 1,N
                    TEMP = ALPHA
                    IF (NOUNIT) TEMP = TEMP*A(J,J)
                    DO 210 I = 1,M
                        B(I,J) = TEMP*B(I,J)
 210                CONTINUE
                    DO 230 K = J + 1,N
                        IF (A(K,J).NE.ZERO) THEN
                            TEMP = ALPHA*A(K,J)
                            DO 220 I = 1,M
                                B(I,J) = B(I,J) + TEMP*B(I,K)
 220                        CONTINUE
                        END IF
 230                CONTINUE
 240            CONTINUE
            END IF
        ELSE
*
*           Form  B := alpha*B*A**T   or   B := alpha*B*A**H.
*
            IF (UPPER) THEN
                DO 280 K = 1,N
                    DO 260 J = 1,K - 1
                        IF (A(J,K).NE.ZERO) THEN
                            IF (NOCONJ) THEN
                                TEMP = ALPHA*A(J,K)
                            ELSE
                                TEMP = ALPHA*CONJG(A(J,K))
                            END IF
                            DO 250 I = 1,M
                                B(I,J) = B(I,J) + TEMP*B(I,K)
 250                        CONTINUE
                        END IF
 260                CONTINUE
                    TEMP = ALPHA
                    IF (NOUNIT) THEN
                        IF (NOCONJ) THEN
                            TEMP = TEMP*A(K,K)
                        ELSE
                            TEMP = TEMP*CONJG(A(K,K))
                        END IF
                    END IF
                    IF (TEMP.NE.ONE) THEN
                        DO 270 I = 1,M
```

```
                            B(I,K) = TEMP*B(I,K)
 270                    CONTINUE
                    END IF
 280            CONTINUE
            ELSE
                DO 320 K = N,1,-1
                    DO 300 J = K + 1,N
                        IF (A(J,K).NE.ZERO) THEN
                            IF (NOCONJ) THEN
                                TEMP = ALPHA*A(J,K)
                            ELSE
                                TEMP = ALPHA*CONJG(A(J,K))
                            END IF
                            DO 290 I = 1,M
                                B(I,J) = B(I,J) + TEMP*B(I,K)
 290                        CONTINUE
                        END IF
 300                CONTINUE
                    TEMP = ALPHA
                    IF (NOUNIT) THEN
                        IF (NOCONJ) THEN
                            TEMP = TEMP*A(K,K)
                        ELSE
                            TEMP = TEMP*CONJG(A(K,K))
                        END IF
                    END IF
                    IF (TEMP.NE.ONE) THEN
                        DO 310 I = 1,M
                            B(I,K) = TEMP*B(I,K)
 310                    CONTINUE
                    END IF
 320            CONTINUE
            END IF
        END IF
    END IF
*
    RETURN
*
*   End of CTRMM .
*
    END
```

———-

16.8.4 lisp code

— BLAS 3 ctrmm —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)) (zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (ignorable one zero))
  (defun ctrmm (side uplo transa diag m n alpha a lda b ldb$)
    (declare (type (array f2cl-lib:complex8 (*)) b a)
             (type (f2cl-lib:complex8) alpha)
             (type (f2cl-lib:integer4) ldb$ lda n m)
             (type (string *) diag transa uplo side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (transa character transa-%data% transa-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%))
      (prog ((lside nil) (noconj nil) (nounit nil) (upper nil) (i 0) (info 0)
             (j 0) (k 0) (nrowa 0) (temp #C(0.0 0.0)))
        (declare (type f2cl-lib:logical lside noconj nounit upper)
                 (type (f2cl-lib:integer4) i info j k nrowa)
                 (type (f2cl-lib:complex8) temp))
        (setf lside
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame side "L")
                  (declare (ignore var-1))
                  (when var-0 (setf side var-0))
                  ret-val))
        (cond (lside (setf nrowa m)) (t (setf nrowa n)))
        (setf noconj
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame transa "T")
                  (declare (ignore var-1))
                  (when var-0 (setf transa var-0))
                  ret-val))
        (setf nounit
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame diag "N")
                  (declare (ignore var-1))
                  (when var-0 (setf diag var-0))
                  ret-val))
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (setf info 0)
        (cond
         ((and (not lside)
               (not
```

```
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame side "R")
         (declare (ignore var-1))
         (when var-0 (setf side var-0))
         ret-val)))
 (setf info 1))
((and (not upper)
      (not
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame uplo "L")
         (declare (ignore var-1))
         (when var-0 (setf uplo var-0))
         ret-val)))
 (setf info 2))
((and
  (not
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame transa "N")
     (declare (ignore var-1))
     (when var-0 (setf transa var-0))
     ret-val))
  (not
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame transa "T")
     (declare (ignore var-1))
     (when var-0 (setf transa var-0))
     ret-val))
  (not
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame transa "C")
     (declare (ignore var-1))
     (when var-0 (setf transa var-0))
     ret-val)))
 (setf info 3))
((and
  (not
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame diag "U")
     (declare (ignore var-1))
     (when var-0 (setf diag var-0))
     ret-val))
  (not
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame diag "N")
     (declare (ignore var-1))
     (when var-0 (setf diag var-0))
     ret-val)))
 (setf info 4))
((< m 0) (setf info 5)) ((< n 0) (setf info 6))
((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nrowa)))
```

```
  (setf info 9))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info 11)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "CTRMM " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= m 0) (= n 0))
    (go end_label))
(cond
 ((= alpha zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                  zero)
         label10))
     label20))
  (go end_label)))
(cond
 (lside
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame transa "N")
      (declare (ignore var-1))
      (when var-0 (setf transa var-0))
      ret-val)
    (cond
     (upper
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                        ((> k m) nil)
            (tagbody
              (cond
               ((/= (f2cl-lib:fref b (k j) ((1 ldb$) (1 *))) zero)
                (setf temp
                        (* alpha
                           (f2cl-lib:fref b-%data% (k j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)))
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                                ((> i
                                    (f2cl-lib:int-add k
                                                      (f2cl-lib:int-sub
                                                       1)))
                                 nil)
                        (tagbody
                          (setf (f2cl-lib:fref b-%data% (i j)
                                               ((1 ldb$) (1 *)) b-%offset%)
                                (+
                                 (f2cl-lib:fref b-%data% (i j)
                                                ((1 ldb$) (1 *))
                                                b-%offset%)
                                 (* temp
                                    (f2cl-lib:fref a-%data% (i k)
                                                   ((1 lda) (1 *))
                                                   a-%offset%))))
                           label30))
                      (if nounit
                          (setf temp
                                (* temp
                                   (f2cl-lib:fref a-%data% (k k)
                                                  ((1 lda) (1 *))
                                                  a-%offset%))))
                      (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                           b-%offset%)
                            temp)))
                 label40))
             label50)))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (f2cl-lib:fdo (k m (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                           ((> k 1) nil)
               (tagbody
                 (cond
                  ((/= (f2cl-lib:fref b (k j) ((1 ldb$) (1 *))) zero)
                   (setf temp
                         (* alpha
                            (f2cl-lib:fref b-%data% (k j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%)))
                   (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                        b-%offset%)
                         temp)
                   (if nounit
                       (setf (f2cl-lib:fref b-%data% (k j)
                                            ((1 ldb$) (1 *)) b-%offset%)
                             (*
                              (f2cl-lib:fref b-%data% (k j)
```

```
                                          ((1 ldb$) (1 *))
                                          b-%offset%)
                              (f2cl-lib:fref a-%data% (k k)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
                 (f2cl-lib:fdo (i (f2cl-lib:int-add k 1)
                                (f2cl-lib:int-add i 1))
                               ((> i m) nil)
                   (tagbody
                     (setf (f2cl-lib:fref b-%data% (i j)
                                     ((1 ldb$) (1 *)) b-%offset%)
                             (+
                              (f2cl-lib:fref b-%data% (i j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)
                              (* temp
                                 (f2cl-lib:fref a-%data% (i k)
                                             ((1 lda) (1 *))
                                             a-%offset%))))
                    label60))))
            label70))
       label80)))))
(t
 (cond
  (upper
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     ((> i 1) nil)
         (tagbody
           (setf temp
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%))
           (cond
            (noconj
             (if nounit
                 (setf temp
                         (* temp
                            (f2cl-lib:fref a-%data% (i i)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
             (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                           ((> k
                               (f2cl-lib:int-add i
                                                 (f2cl-lib:int-sub
                                                  1)))
                            nil)
               (tagbody
                 (setf temp
```

```
                              (+ temp
                                 (*
                                  (f2cl-lib:fref a-%data% (k i)
                                                 ((1 lda) (1 *))
                                                 a-%offset%)
                                  (f2cl-lib:fref b-%data% (k j)
                                                 ((1 ldb$) (1 *))
                                                 b-%offset%))))
                         label90)))
                     (t
                      (if nounit
                          (setf temp
                                (* temp
                                   (f2cl-lib:conjg
                                    (f2cl-lib:fref a-%data% (i i)
                                                   ((1 lda) (1 *))
                                                   a-%offset%)))))
                      (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                                    ((> k
                                        (f2cl-lib:int-add i
                                                          (f2cl-lib:int-sub
                                                           1)))
                                    nil)
                        (tagbody
                          (setf temp
                                (+ temp
                                   (*
                                    (f2cl-lib:conjg
                                     (f2cl-lib:fref a-%data% (k i)
                                                    ((1 lda) (1 *))
                                                    a-%offset%))
                                    (f2cl-lib:fref b-%data% (k j)
                                                   ((1 ldb$) (1 *))
                                                   b-%offset%))))
                         label100))))
                  (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)
                        (* alpha temp))
                label110))
            label120)))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i m) nil)
               (tagbody
                 (setf temp
                        (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%))
```

```
      (cond
       (noconj
        (if nounit
            (setf temp
                    (* temp
                       (f2cl-lib:fref a-%data% (i i)
                                      ((1 lda) (1 *))
                                      a-%offset%))))
        (f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
                       (f2cl-lib:int-add k 1))
                      ((> k m) nil)
          (tagbody
            (setf temp
                    (+ temp
                       (*
                        (f2cl-lib:fref a-%data% (k i)
                                       ((1 lda) (1 *))
                                       a-%offset%)
                        (f2cl-lib:fref b-%data% (k j)
                                       ((1 ldb$) (1 *))
                                       b-%offset%))))
             label130)))
       (t
        (if nounit
            (setf temp
                    (* temp
                       (f2cl-lib:conjg
                        (f2cl-lib:fref a-%data% (i i)
                                       ((1 lda) (1 *))
                                       a-%offset%)))))
        (f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
                       (f2cl-lib:int-add k 1))
                      ((> k m) nil)
          (tagbody
            (setf temp
                    (+ temp
                       (*
                        (f2cl-lib:conjg
                         (f2cl-lib:fref a-%data% (k i)
                                        ((1 lda) (1 *))
                                        a-%offset%))
                        (f2cl-lib:fref b-%data% (k j)
                                       ((1 ldb$) (1 *))
                                       b-%offset%))))
             label140))))
      (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                           b-%offset%)
            (* alpha temp))
     label150))
 label160)))))))
```

```
(t
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame transa "N")
     (declare (ignore var-1))
     (when var-0 (setf transa var-0))
     ret-val)
   (cond
    (upper
     (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   ((> j 1) nil)
       (tagbody
         (setf temp alpha)
         (if nounit
             (setf temp
                     (* temp
                        (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                       a-%offset%))))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)
                     (* temp
                        (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)))
            label170))
         (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                       ((> k
                           (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                        nil)
           (tagbody
             (cond
              ((/= (f2cl-lib:fref a (k j) ((1 lda) (1 *))) zero)
               (setf temp
                       (* alpha
                          (f2cl-lib:fref a-%data% (k j)
                                         ((1 lda) (1 *)) a-%offset%)))
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i m) nil)
                 (tagbody
                   (setf (f2cl-lib:fref b-%data% (i j)
                                        ((1 ldb$) (1 *)) b-%offset%)
                           (+
                            (f2cl-lib:fref b-%data% (i j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%)
                            (* temp
                               (f2cl-lib:fref b-%data% (i k)
                                              ((1 ldb$) (1 *))
```

```
                                                b-%offset%))))
                    label180))))
            label190))
        label200)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp alpha)
         (if nounit
             (setf temp
                     (* temp
                        (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                       a-%offset%))))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)
                     (* temp
                        (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)))
            label210))
         (f2cl-lib:fdo (k (f2cl-lib:int-add j 1)
                          (f2cl-lib:int-add k 1))
                       ((> k n) nil)
           (tagbody
             (cond
              ((/= (f2cl-lib:fref a (k j) ((1 lda) (1 *))) zero)
               (setf temp
                       (* alpha
                          (f2cl-lib:fref a-%data% (k j)
                                         ((1 lda) (1 *)) a-%offset%)))
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i m) nil)
                 (tagbody
                   (setf (f2cl-lib:fref b-%data% (i j)
                                        ((1 ldb$) (1 *)) b-%offset%)
                           (+
                            (f2cl-lib:fref b-%data% (i j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%)
                            (* temp
                               (f2cl-lib:fref b-%data% (i k)
                                              ((1 ldb$) (1 *))
                                              b-%offset%))))
                    label220))))
            label230))
        label240)))))
(t
```

```
(cond
 (upper
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j
                        (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (cond
           ((/= (f2cl-lib:fref a (j k) ((1 lda) (1 *))) zero)
            (cond
             (noconj
              (setf temp
                      (* alpha
                         (f2cl-lib:fref a-%data% (j k)
                                        ((1 lda) (1 *))
                                        a-%offset%))))
             (t
              (setf temp
                      (* alpha
                         (f2cl-lib:conjg
                          (f2cl-lib:fref a-%data% (j k)
                                         ((1 lda) (1 *))
                                         a-%offset%))))))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf (f2cl-lib:fref b-%data% (i j)
                                     ((1 ldb$) (1 *)) b-%offset%)
                      (+
                       (f2cl-lib:fref b-%data% (i j)
                                      ((1 ldb$) (1 *))
                                      b-%offset%)
                       (* temp
                          (f2cl-lib:fref b-%data% (i k)
                                         ((1 ldb$) (1 *))
                                         b-%offset%))))
               label250))))
          label260))
      (setf temp alpha)
      (cond
       (nounit
        (cond
         (noconj
          (setf temp
                  (* temp
                     (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                    a-%offset%))))
```

```
            (t
             (setf temp
                     (* temp
                        (f2cl-lib:conjg
                         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                        a-%offset%))))))))
        (cond
         ((/= temp one)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref b-%data% (i k) ((1 ldb$) (1 *))
                                    b-%offset%)
                      (* temp
                         (f2cl-lib:fref b-%data% (i k)
                                        ((1 ldb$) (1 *))
                                        b-%offset%)))
             label270))))
       label280)))
   (t
    (f2cl-lib:fdo (k n (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                  ((> k 1) nil)
      (tagbody
        (f2cl-lib:fdo (j (f2cl-lib:int-add k 1)
                       (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (cond
             ((/= (f2cl-lib:fref a (j k) ((1 lda) (1 *))) zero)
              (cond
               (noconj
                (setf temp
                        (* alpha
                           (f2cl-lib:fref a-%data% (j k)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
               (t
                (setf temp
                        (* alpha
                           (f2cl-lib:conjg
                            (f2cl-lib:fref a-%data% (j k)
                                           ((1 lda) (1 *))
                                           a-%offset%))))))
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i m) nil)
                (tagbody
                  (setf (f2cl-lib:fref b-%data% (i j)
                                        ((1 ldb$) (1 *)) b-%offset%)
                          (+
                           (f2cl-lib:fref b-%data% (i j)
```

```
                                                ((1 ldb$) (1 *))
                                                b-%offset%)
                                     (* temp
                                        (f2cl-lib:fref b-%data% (i k)
                                                       ((1 ldb$) (1 *))
                                                       b-%offset%))))
                          label290))))
                   label300))
                (setf temp alpha)
                (cond
                 (nounit
                  (cond
                   (noconj
                    (setf temp
                            (* temp
                               (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                              a-%offset%))))
                   (t
                    (setf temp
                            (* temp
                               (f2cl-lib:conjg
                                (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                               a-%offset%))))))))
                (cond
                 ((/= temp one)
                  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                ((> i m) nil)
                    (tagbody
                      (setf (f2cl-lib:fref b-%data% (i k) ((1 ldb$) (1 *))
                                           b-%offset%)
                              (* temp
                                 (f2cl-lib:fref b-%data% (i k)
                                                ((1 ldb$) (1 *))
                                                b-%offset%)))
                     label310))))
               label320))))))))
 (go end_label)
end_label
 (return (values side uplo transa diag nil nil nil nil nil nil nil))))))
```

———-

16.9 ctrsm - triangular $op(A)X = \alpha B$ or $Xop(A) = \alpha B$, $op(A) = A$ or $op(A) = A^T$ or $op(A) = A^H$

16.9.1 Axiom unit tests

— ctrsm.input —

```
)set break resume
)sys rm -f ctrsm.output
)spool ctrsm.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

16.9.2 Axiom help page

— ctrsm.help —

```
======================================================================
ctrsm examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ctrsm (SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A, LDA, B,
           LDB)
           CTRSM

Function/Subroutine Documentation
   subroutine ctrsm (character SIDE, character UPLO, character TRANSA,
       character DIAG, integer M, integer N, complex ALPHA, complex,
       dimension(lda,*) A, integer LDA, complex, dimension(ldb,*) B, integer
       LDB)
       CTRSM

Purpose:
```

```
  CTRSM  solves one of the matrix equations

  op( A )*X = alpha*B,   or   X*op( A ) = alpha*B,

  where alpha is a scalar, X and B are m by n matrices, A is a unit, or
  non-unit,  upper or lower triangular matrix  and  op( A )  is one  of

  op( A ) = A   or   op( A ) = A**T   or   op( A ) = A**H.

  The matrix X is overwritten on B.

Parameters:

 SIDE
  SIDE is CHARACTER*1
  On entry, SIDE specifies whether op( A ) appears on the left
  or right of X as follows:

  SIDE = 'L' or 'l'   op( A )*X = alpha*B.

  SIDE = 'R' or 'r'   X*op( A ) = alpha*B.

 UPLO
  UPLO is CHARACTER*1
  On entry, UPLO specifies whether the matrix A is an upper or
  lower triangular matrix as follows:

  UPLO = 'U' or 'u'   A is an upper triangular matrix.

  UPLO = 'L' or 'l'   A is a lower triangular matrix.

 TRANSA
  TRANSA is CHARACTER*1
  On entry, TRANSA specifies the form of op( A ) to be used in
  the matrix multiplication as follows:

  TRANSA = 'N' or 'n'   op( A ) = A.

  TRANSA = 'T' or 't'   op( A ) = A**T.

  TRANSA = 'C' or 'c'   op( A ) = A**H.

 DIAG
  DIAG is CHARACTER*1
  On entry, DIAG specifies whether or not A is unit triangular
  as follows:

  DIAG = 'U' or 'u'   A is assumed to be unit triangular.

  DIAG = 'N' or 'n'   A is not assumed to be unit triangular.
```

```
M
  M is INTEGER
  On entry, M specifies the number of rows of B. M must be at
  least zero.

N
  N is INTEGER
  On entry, N specifies the number of columns of B.  N must be
  at least zero.

ALPHA
  ALPHA is COMPLEX
  On entry,  ALPHA specifies the scalar  alpha. When  alpha is
  zero then  A is not referenced and  B need not be set before entry.

A
  A is COMPLEX array of DIMENSION ( LDA, k ),
  where k is m when SIDE = 'L' or 'l'
  and k is n when SIDE = 'R' or 'r'.

  Before entry  with  UPLO = 'U' or 'u',  the  leading  k by k
  upper triangular part of the array  A must contain the upper
  triangular matrix  and the strictly lower triangular part of
  A is not referenced.

  Before entry  with  UPLO = 'L' or 'l',  the  leading  k by k
  lower triangular part of the array  A must contain the lower
  triangular matrix  and the strictly upper triangular part of
  A is not referenced.

  Note that when  DIAG = 'U' or 'u',  the diagonal elements of
  A  are not referenced either,  but are assumed to be  unity.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program.  When  SIDE = 'L' or 'l'  then
  LDA  must be at least  max( 1, m ),  when  SIDE = 'R' or 'r'
  then LDA must be at least max( 1, n ).

B
  B is COMPLEX array of DIMENSION ( LDB, n ).
  Before entry,  the leading  m by n part of the array  B must
  contain  the  right-hand  side  matrix  B,  and  on exit  is
  overwritten by the solution matrix  X.

LDB
  LDB is INTEGER
  On entry, LDB specifies the first dimension of B as declared
```

```
in the calling (sub) program. LDB must be at least
max( 1, m ).
```

———-

16.9.3 fortran code

— **ctrsm.f** —

```
*  ===================================================================
      SUBROUTINE CTRSM(SIDE,UPLO,TRANSA,DIAG,M,N,ALPHA,A,LDA,B,LDB)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX ALPHA
      INTEGER LDA,LDB,M,N
      CHARACTER DIAG,SIDE,TRANSA,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX A(LDA,*),B(LDB,*)
*     ..
*
*  ===================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC CONJG,MAX
*     ..
*     .. Local Scalars ..
      COMPLEX TEMP
      INTEGER I,INFO,J,K,NROWA
      LOGICAL LSIDE,NOCONJ,NOUNIT,UPPER
*     ..
*     .. Parameters ..
      COMPLEX ONE
      PARAMETER (ONE= (1.0E+0,0.0E+0))
      COMPLEX ZERO
```

```
      PARAMETER (ZERO= (0.0E+0,0.0E+0))
*     ..
*
*     Test the input parameters.
*
      LSIDE = LSAME(SIDE,'L')
      IF (LSIDE) THEN
          NROWA = M
      ELSE
          NROWA = N
      END IF
      NOCONJ = LSAME(TRANSA,'T')
      NOUNIT = LSAME(DIAG,'N')
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.LSIDE) .AND. (.NOT.LSAME(SIDE,'R'))) THEN
          INFO = 1
      ELSE IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 2
      ELSE IF ((.NOT.LSAME(TRANSA,'N')) .AND.
     +         (.NOT.LSAME(TRANSA,'T')) .AND.
     +         (.NOT.LSAME(TRANSA,'C'))) THEN
          INFO = 3
      ELSE IF ((.NOT.LSAME(DIAG,'U')) .AND. (.NOT.LSAME(DIAG,'N'))) THEN
          INFO = 4
      ELSE IF (M.LT.0) THEN
          INFO = 5
      ELSE IF (N.LT.0) THEN
          INFO = 6
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 9
      ELSE IF (LDB.LT.MAX(1,M)) THEN
          INFO = 11
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('CTRSM ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF (M.EQ.0 .OR. N.EQ.0) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
          DO 20 J = 1,N
              DO 10 I = 1,M
                  B(I,J) = ZERO
```

```
   10         CONTINUE
   20     CONTINUE
          RETURN
      END IF
*
*     Start the operations.
*
      IF (LSIDE) THEN
          IF (LSAME(TRANSA,'N')) THEN
*
*           Form  B := alpha*inv( A )*B.
*
              IF (UPPER) THEN
                  DO 60 J = 1,N
                      IF (ALPHA.NE.ONE) THEN
                          DO 30 I = 1,M
                              B(I,J) = ALPHA*B(I,J)
   30                     CONTINUE
                      END IF
                      DO 50 K = M,1,-1
                          IF (B(K,J).NE.ZERO) THEN
                              IF (NOUNIT) B(K,J) = B(K,J)/A(K,K)
                              DO 40 I = 1,K - 1
                                  B(I,J) = B(I,J) - B(K,J)*A(I,K)
   40                         CONTINUE
                          END IF
   50                 CONTINUE
   60             CONTINUE
              ELSE
                  DO 100 J = 1,N
                      IF (ALPHA.NE.ONE) THEN
                          DO 70 I = 1,M
                              B(I,J) = ALPHA*B(I,J)
   70                     CONTINUE
                      END IF
                      DO 90 K = 1,M
                          IF (B(K,J).NE.ZERO) THEN
                              IF (NOUNIT) B(K,J) = B(K,J)/A(K,K)
                              DO 80 I = K + 1,M
                                  B(I,J) = B(I,J) - B(K,J)*A(I,K)
   80                         CONTINUE
                          END IF
   90                 CONTINUE
  100             CONTINUE
              END IF
          ELSE
*
*           Form  B := alpha*inv( A**T )*B
*           or    B := alpha*inv( A**H )*B.
*
```

```
            IF (UPPER) THEN
                DO 140 J = 1,N
                    DO 130 I = 1,M
                        TEMP = ALPHA*B(I,J)
                        IF (NOCONJ) THEN
                            DO 110 K = 1,I - 1
                                TEMP = TEMP - A(K,I)*B(K,J)
 110                        CONTINUE
                            IF (NOUNIT) TEMP = TEMP/A(I,I)
                        ELSE
                            DO 120 K = 1,I - 1
                                TEMP = TEMP - CONJG(A(K,I))*B(K,J)
 120                        CONTINUE
                            IF (NOUNIT) TEMP = TEMP/CONJG(A(I,I))
                        END IF
                        B(I,J) = TEMP
 130                CONTINUE
 140            CONTINUE
            ELSE
                DO 180 J = 1,N
                    DO 170 I = M,1,-1
                        TEMP = ALPHA*B(I,J)
                        IF (NOCONJ) THEN
                            DO 150 K = I + 1,M
                                TEMP = TEMP - A(K,I)*B(K,J)
 150                        CONTINUE
                            IF (NOUNIT) TEMP = TEMP/A(I,I)
                        ELSE
                            DO 160 K = I + 1,M
                                TEMP = TEMP - CONJG(A(K,I))*B(K,J)
 160                        CONTINUE
                            IF (NOUNIT) TEMP = TEMP/CONJG(A(I,I))
                        END IF
                        B(I,J) = TEMP
 170                CONTINUE
 180            CONTINUE
            END IF
        END IF
    ELSE
        IF (LSAME(TRANSA,'N')) THEN
*
*          Form  B := alpha*B*inv( A ).
*
            IF (UPPER) THEN
                DO 230 J = 1,N
                    IF (ALPHA.NE.ONE) THEN
                        DO 190 I = 1,M
                            B(I,J) = ALPHA*B(I,J)
 190                    CONTINUE
                    END IF
```

```
                    DO 210 K = 1,J - 1
                        IF (A(K,J).NE.ZERO) THEN
                            DO 200 I = 1,M
                                B(I,J) = B(I,J) - A(K,J)*B(I,K)
 200                        CONTINUE
                        END IF
 210                CONTINUE
                    IF (NOUNIT) THEN
                        TEMP = ONE/A(J,J)
                        DO 220 I = 1,M
                            B(I,J) = TEMP*B(I,J)
 220                    CONTINUE
                    END IF
 230            CONTINUE
            ELSE
                DO 280 J = N,1,-1
                    IF (ALPHA.NE.ONE) THEN
                        DO 240 I = 1,M
                            B(I,J) = ALPHA*B(I,J)
 240                    CONTINUE
                    END IF
                    DO 260 K = J + 1,N
                        IF (A(K,J).NE.ZERO) THEN
                            DO 250 I = 1,M
                                B(I,J) = B(I,J) - A(K,J)*B(I,K)
 250                        CONTINUE
                        END IF
 260                CONTINUE
                    IF (NOUNIT) THEN
                        TEMP = ONE/A(J,J)
                        DO 270 I = 1,M
                            B(I,J) = TEMP*B(I,J)
 270                    CONTINUE
                    END IF
 280            CONTINUE
            END IF
        ELSE
*
*         Form  B := alpha*B*inv( A**T )
*         or    B := alpha*B*inv( A**H ).
*
            IF (UPPER) THEN
                DO 330 K = N,1,-1
                    IF (NOUNIT) THEN
                        IF (NOCONJ) THEN
                            TEMP = ONE/A(K,K)
                        ELSE
                            TEMP = ONE/CONJG(A(K,K))
                        END IF
                        DO 290 I = 1,M
```

```
                        B(I,K) = TEMP*B(I,K)
290                 CONTINUE
                END IF
                DO 310 J = 1,K - 1
                    IF (A(J,K).NE.ZERO) THEN
                        IF (NOCONJ) THEN
                            TEMP = A(J,K)
                        ELSE
                            TEMP = CONJG(A(J,K))
                        END IF
                        DO 300 I = 1,M
                            B(I,J) = B(I,J) - TEMP*B(I,K)
300                     CONTINUE
                    END IF
310             CONTINUE
                IF (ALPHA.NE.ONE) THEN
                    DO 320 I = 1,M
                        B(I,K) = ALPHA*B(I,K)
320                 CONTINUE
                END IF
330         CONTINUE
        ELSE
            DO 380 K = 1,N
                IF (NOUNIT) THEN
                    IF (NOCONJ) THEN
                        TEMP = ONE/A(K,K)
                    ELSE
                        TEMP = ONE/CONJG(A(K,K))
                    END IF
                    DO 340 I = 1,M
                        B(I,K) = TEMP*B(I,K)
340                 CONTINUE
                END IF
                DO 360 J = K + 1,N
                    IF (A(J,K).NE.ZERO) THEN
                        IF (NOCONJ) THEN
                            TEMP = A(J,K)
                        ELSE
                            TEMP = CONJG(A(J,K))
                        END IF
                        DO 350 I = 1,M
                            B(I,J) = B(I,J) - TEMP*B(I,K)
350                     CONTINUE
                    END IF
360             CONTINUE
                IF (ALPHA.NE.ONE) THEN
                    DO 370 I = 1,M
                        B(I,K) = ALPHA*B(I,K)
370                 CONTINUE
                END IF
```

```
  380             CONTINUE
              END IF
          END IF
      END IF
*
      RETURN
*
*     End of CTRSM .
*
      END
```

———-

16.9.4 lisp code

— BLAS 3 ctrsm —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)) (zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (ignorable one zero))
  (defun ctrsm (side uplo transa diag m n alpha a lda b ldb$)
    (declare (type (array f2cl-lib:complex8 (*)) b a)
             (type (f2cl-lib:complex8) alpha)
             (type (f2cl-lib:integer4) ldb$ lda n m)
             (type (string *) diag transa uplo side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (transa character transa-%data% transa-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%))
      (prog ((lside nil) (noconj nil) (nounit nil) (upper nil) (i 0) (info 0)
             (j 0) (k 0) (nrowa 0) (temp #C(0.0 0.0)))
        (declare (type f2cl-lib:logical lside noconj nounit upper)
                 (type (f2cl-lib:integer4) i info j k nrowa)
                 (type (f2cl-lib:complex8) temp))
        (setf lside
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame side "L")
                  (declare (ignore var-1))
                  (when var-0 (setf side var-0))
                  ret-val))
        (cond (lside (setf nrowa m)) (t (setf nrowa n)))
        (setf noconj
                (multiple-value-bind (ret-val var-0 var-1)
```

```
            (lsame transa "T")
          (declare (ignore var-1))
          (when var-0 (setf transa var-0))
          ret-val))
(setf nounit
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame diag "N")
          (declare (ignore var-1))
          (when var-0 (setf diag var-0))
          ret-val))
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(setf info 0)
(cond
 ((and (not lside)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "R")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val)))
  (setf info 1))
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info 2))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame transa "N")
      (declare (ignore var-1))
      (when var-0 (setf transa var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame transa "T")
      (declare (ignore var-1))
      (when var-0 (setf transa var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame transa "C")
```

```
      (declare (ignore var-1))
      (when var-0 (setf transa var-0))
      ret-val)))
  (setf info 3))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame diag "U")
      (declare (ignore var-1))
      (when var-0 (setf diag var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame diag "N")
      (declare (ignore var-1))
      (when var-0 (setf diag var-0))
      ret-val)))
  (setf info 4))
 ((< m 0) (setf info 5)) ((< n 0) (setf info 6))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nrowa)))
  (setf info 9))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info 11)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "CTRSM " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= m 0) (= n 0))
    (go end_label))
(cond
 ((= alpha zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                zero)
         label10))
     label20))
  (go end_label)))
(cond
 (lside
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
```

```
      (lsame transa "N")
    (declare (ignore var-1))
    (when var-0 (setf transa var-0))
    ret-val)
  (cond
   (upper
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
         ((/= alpha one)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)
                    (* alpha
                       (f2cl-lib:fref b-%data% (i j)
                                      ((1 ldb$) (1 *))
                                      b-%offset%)))
             label30))))
        (f2cl-lib:fdo (k m (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                      ((> k 1) nil)
          (tagbody
            (cond
             ((/= (f2cl-lib:fref b (k j) ((1 ldb$) (1 *))) zero)
              (if nounit
                  (setf (f2cl-lib:fref b-%data% (k j)
                                        ((1 ldb$) (1 *)) b-%offset%)
                        (/
                         (f2cl-lib:fref b-%data% (k j)
                                        ((1 ldb$) (1 *))
                                        b-%offset%)
                         (f2cl-lib:fref a-%data% (k k)
                                        ((1 lda) (1 *))
                                        a-%offset%))))
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i
                                (f2cl-lib:int-add k
                                                  (f2cl-lib:int-sub
                                                   1)))
                             nil)
                (tagbody
                  (setf (f2cl-lib:fref b-%data% (i j)
                                        ((1 ldb$) (1 *)) b-%offset%)
                        (-
                         (f2cl-lib:fref b-%data% (i j)
                                        ((1 ldb$) (1 *))
                                        b-%offset%)
                         (*
```

```
                                (f2cl-lib:fref b-%data% (k j)
                                               ((1 ldb$) (1 *))
                                               b-%offset%)
                                (f2cl-lib:fref a-%data% (i k)
                                               ((1 lda) (1 *))
                                               a-%offset%))))
                 label40))))
           label50))
       label60)))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
         ((/= alpha one)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)
                    (* alpha
                       (f2cl-lib:fref b-%data% (i j)
                                      ((1 ldb$) (1 *))
                                      b-%offset%)))
             label70))))
        (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                      ((> k m) nil)
          (tagbody
            (cond
             ((/= (f2cl-lib:fref b (k j) ((1 ldb$) (1 *))) zero)
              (if nounit
                  (setf (f2cl-lib:fref b-%data% (k j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                        (/
                         (f2cl-lib:fref b-%data% (k j)
                                        ((1 ldb$) (1 *))
                                        b-%offset%)
                         (f2cl-lib:fref a-%data% (k k)
                                        ((1 lda) (1 *))
                                        a-%offset%))))
              (f2cl-lib:fdo (i (f2cl-lib:int-add k 1)
                               (f2cl-lib:int-add i 1))
                            ((> i m) nil)
                (tagbody
                  (setf (f2cl-lib:fref b-%data% (i j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                        (-
                         (f2cl-lib:fref b-%data% (i j)
                                        ((1 ldb$) (1 *))
                                        b-%offset%)
```

```
                                (*
                                 (f2cl-lib:fref b-%data% (k j)
                                                ((1 ldb$) (1 *))
                                                b-%offset%)
                                 (f2cl-lib:fref a-%data% (i k)
                                                ((1 lda) (1 *))
                                                a-%offset%))))
                     label80))))
               label90))
          label100)))))
     (t
      (cond
       (upper
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf temp
                        (* alpha
                           (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                          b-%offset%)))
                (cond
                 (noconj
                  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                                ((> k
                                    (f2cl-lib:int-add i
                                                      (f2cl-lib:int-sub
                                                       1)))
                                 nil)
                    (tagbody
                      (setf temp
                              (- temp
                                 (*
                                  (f2cl-lib:fref a-%data% (k i)
                                                 ((1 lda) (1 *))
                                                 a-%offset%)
                                  (f2cl-lib:fref b-%data% (k j)
                                                 ((1 ldb$) (1 *))
                                                 b-%offset%))))
                    label110))
                  (if nounit
                      (setf temp
                              (/ temp
                                 (f2cl-lib:fref a-%data% (i i)
                                                ((1 lda) (1 *))
                                                a-%offset%)))))
                 (t
                  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
```

```
                                ((> k
                                    (f2cl-lib:int-add i
                                                      (f2cl-lib:int-sub
                                                       1)))
                                 nil)
                        (tagbody
                          (setf temp
                                  (- temp
                                     (*
                                      (f2cl-lib:conjg
                                       (f2cl-lib:fref a-%data% (k i)
                                                      ((1 lda) (1 *))
                                                      a-%offset%))
                                      (f2cl-lib:fref b-%data% (k j)
                                                     ((1 ldb$) (1 *))
                                                     b-%offset%))))
                         label120))
                      (if nounit
                          (setf temp
                                  (/ temp
                                     (f2cl-lib:conjg
                                      (f2cl-lib:fref a-%data% (i i)
                                                     ((1 lda) (1 *))
                                                     a-%offset%)))))))
                  (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)
                        temp)
                label130))
            label140)))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                           ((> i 1) nil)
               (tagbody
                 (setf temp
                         (* alpha
                            (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                           b-%offset%)))
                 (cond
                  (noconj
                   (f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
                                    (f2cl-lib:int-add k 1))
                                 ((> k m) nil)
                     (tagbody
                       (setf temp
                               (- temp
                                  (*
                                   (f2cl-lib:fref a-%data% (k i)
```

```
                                                ((1 lda) (1 *))
                                                a-%offset%)
                                     (f2cl-lib:fref b-%data% (k j)
                                                ((1 ldb$) (1 *))
                                                b-%offset%))))
                        label150))
                     (if nounit
                         (setf temp
                                 (/ temp
                                    (f2cl-lib:fref a-%data% (i i)
                                                   ((1 lda) (1 *))
                                                   a-%offset%)))))
                    (t
                     (f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
                                    (f2cl-lib:int-add k 1))
                                   ((> k m) nil)
                       (tagbody
                         (setf temp
                                 (- temp
                                    (*
                                     (f2cl-lib:conjg
                                      (f2cl-lib:fref a-%data% (k i)
                                                     ((1 lda) (1 *))
                                                     a-%offset%))
                                     (f2cl-lib:fref b-%data% (k j)
                                                ((1 ldb$) (1 *))
                                                b-%offset%))))
                        label160))
                     (if nounit
                         (setf temp
                                 (/ temp
                                    (f2cl-lib:conjg
                                     (f2cl-lib:fref a-%data% (i i)
                                                ((1 lda) (1 *))
                                                a-%offset%)))))))
                  (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)
                          temp)
                label170))
            label180)))))))
    (t
     (cond
      ((multiple-value-bind (ret-val var-0 var-1)
           (lsame transa "N")
         (declare (ignore var-1))
         (when var-0 (setf transa var-0))
         ret-val)
       (cond
        (upper
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
            ((> j n) nil)
(tagbody
  (cond
   ((/= alpha one)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)
                (* alpha
                   (f2cl-lib:fref b-%data% (i j)
                                  ((1 ldb$) (1 *))
                                  b-%offset%)))
       label190))))
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k
                    (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (cond
       ((/= (f2cl-lib:fref a (k j) ((1 lda) (1 *))) zero)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                  (-
                   (f2cl-lib:fref b-%data% (i j)
                                  ((1 ldb$) (1 *))
                                  b-%offset%)
                   (*
                    (f2cl-lib:fref a-%data% (k j)
                                   ((1 lda) (1 *))
                                   a-%offset%)
                    (f2cl-lib:fref b-%data% (i k)
                                   ((1 ldb$) (1 *))
                                   b-%offset%))))
           label200))))
     label210))
  (cond
   (nounit
    (setf temp
            (/ one
               (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                              a-%offset%)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)
```

```
                    (* temp
                       (f2cl-lib:fref b-%data% (i j)
                                      ((1 ldb$) (1 *))
                                      b-%offset%)))
          label220))))
    label230)))
(t
 (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (cond
      ((/= alpha one)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                    (* alpha
                       (f2cl-lib:fref b-%data% (i j)
                                      ((1 ldb$) (1 *))
                                      b-%offset%)))
          label240))))
     (f2cl-lib:fdo (k (f2cl-lib:int-add j 1)
                    (f2cl-lib:int-add k 1))
                   ((> k n) nil)
       (tagbody
         (cond
          ((/= (f2cl-lib:fref a (k j) ((1 lda) (1 *))) zero)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf (f2cl-lib:fref b-%data% (i j)
                                    ((1 ldb$) (1 *)) b-%offset%)
                       (-
                        (f2cl-lib:fref b-%data% (i j)
                                       ((1 ldb$) (1 *))
                                       b-%offset%)
                        (*
                         (f2cl-lib:fref a-%data% (k j)
                                        ((1 lda) (1 *))
                                        a-%offset%)
                         (f2cl-lib:fref b-%data% (i k)
                                        ((1 ldb$) (1 *))
                                        b-%offset%))))
              label250))))
        label260))
     (cond
      (nounit
       (setf temp
               (/ one
```

```
                    (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                   a-%offset%)))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)
                    (* temp
                       (f2cl-lib:fref b-%data% (i j)
                                      ((1 ldb$) (1 *))
                                      b-%offset%)))
            label270))))
      label280)))))
(t
 (cond
  (upper
   (f2cl-lib:fdo (k n (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                 ((> k 1) nil)
     (tagbody
       (cond
        (nounit
         (cond
          (noconj
           (setf temp
                 (/ one
                    (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                   a-%offset%))))
          (t
           (setf temp
                 (/ one
                    (f2cl-lib:conjg
                     (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                    a-%offset%))))))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf (f2cl-lib:fref b-%data% (i k) ((1 ldb$) (1 *))
                                  b-%offset%)
                   (* temp
                      (f2cl-lib:fref b-%data% (i k)
                                     ((1 ldb$) (1 *))
                                     b-%offset%)))
           label290))))
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j
                         (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                      nil)
         (tagbody
           (cond
            ((/= (f2cl-lib:fref a (j k) ((1 lda) (1 *))) zero)
```

```
                (cond
                 (noconj
                  (setf temp
                          (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                         a-%offset%)))
                 (t
                  (setf temp
                          (f2cl-lib:conjg
                           (f2cl-lib:fref a-%data% (j k)
                                          ((1 lda) (1 *))
                                          a-%offset%)))))
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i m) nil)
                  (tagbody
                    (setf (f2cl-lib:fref b-%data% (i j)
                                         ((1 ldb$) (1 *)) b-%offset%)
                          (-
                           (f2cl-lib:fref b-%data% (i j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)
                           (* temp
                              (f2cl-lib:fref b-%data% (i k)
                                             ((1 ldb$) (1 *))
                                             b-%offset%))))
                  label300))))
          label310))
      (cond
       ((/= alpha one)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i k) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (* alpha
                     (f2cl-lib:fref b-%data% (i k)
                                    ((1 ldb$) (1 *))
                                    b-%offset%)))
           label320))))
    label330)))
(t
 (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
               ((> k n) nil)
   (tagbody
     (cond
      (nounit
       (cond
        (noconj
         (setf temp
                 (/ one
                    (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
```

```
                                a-%offset%))))
    (t
     (setf temp
             (/ one
                (f2cl-lib:conjg
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%))))))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf (f2cl-lib:fref b-%data% (i k) ((1 ldb$) (1 *))
                           b-%offset%)
                (* temp
                   (f2cl-lib:fref b-%data% (i k)
                                  ((1 ldb$) (1 *))
                                  b-%offset%)))
     label340))))
(f2cl-lib:fdo (j (f2cl-lib:int-add k 1)
               (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (cond
     ((/= (f2cl-lib:fref a (j k) ((1 lda) (1 *))) zero)
      (cond
       (noconj
        (setf temp
                (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                               a-%offset%)))
       (t
        (setf temp
                (f2cl-lib:conjg
                 (f2cl-lib:fref a-%data% (j k)
                                ((1 lda) (1 *))
                                a-%offset%)))))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j)
                               ((1 ldb$) (1 *)) b-%offset%)
                    (-
                     (f2cl-lib:fref b-%data% (i j)
                                    ((1 ldb$) (1 *))
                                    b-%offset%)
                     (* temp
                        (f2cl-lib:fref b-%data% (i k)
                                       ((1 ldb$) (1 *))
                                       b-%offset%))))
         label350))))
   label360))
(cond
```

```
            ((/= alpha one)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i m) nil)
               (tagbody
                 (setf (f2cl-lib:fref b-%data% (i k) ((1 ldb$) (1 *))
                                      b-%offset%)
                       (* alpha
                          (f2cl-lib:fref b-%data% (i k)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)))
                label370))))
          label380))))))))
 (go end_label)
end_label
 (return (values side uplo transa diag nil nil nil nil nil nil nil))))))
```

———-

Chapter 17

BLAS Level 3 Complex16 Matrix-Matrix Operations

17.1 zgemm - $\alpha op(A)op(B)+\beta C$, $op(X) = X$ or $op(X) = X^T$ or $op(X) = X^H$

17.1.1 Axiom unit tests

— **zgemm.input** —

```
)set break resume
)sys rm -f zgemm.output
)spool zgemm.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

17.1.2 Axiom help page

— **zgemm.help** —

```
======================================================================
zgemm examples
```

```
=====================================================================

=====================================================================
Man Page Details
=====================================================================

 NAME
      ZGEMM - perform one of the matrix-matrix operations   C :=
      alpha*op( A )*op( B ) + beta*C,

 SYNOPSIS
      SUBROUTINE ZGEMM ( TRANSA, TRANSB, M, N, K, ALPHA, A, LDA,
                       B, LDB, BETA, C, LDC )

          CHARACTER*1  TRANSA, TRANSB

          INTEGER      M, N, K, LDA, LDB, LDC

          COMPLEX*16   ALPHA, BETA

          COMPLEX*16   A( LDA, * ), B( LDB, * ), C( LDC, * )

 PURPOSE
      ZGEMM  performs one of the matrix-matrix operations

      where  op( X ) is one of

         op( X ) = X   or   op( X ) = X'   or   op( X ) = conjg(
      X' ),

      alpha and beta are scalars, and A, B and C are matrices,
      with op( A ) an m by k matrix,  op( B )  a  k by n matrix
      and  C an m by n matrix.

 PARAMETERS
      TRANSA - CHARACTER*1.  On entry, TRANSA specifies the form
      of op( A ) to be used in the matrix multiplication as fol-
      lows:

      TRANSA = 'N' or 'n',  op( A ) = A.

      TRANSA = 'T' or 't',  op( A ) = A'.

      TRANSA = 'C' or 'c',  op( A ) = conjg( A' ).

      Unchanged on exit.

      TRANSB - CHARACTER*1.  On entry, TRANSB specifies the form
      of op( B ) to be used in the matrix multiplication as fol-
      lows:
```

```
     TRANSB = 'N' or 'n',  op( B ) = B.

     TRANSB = 'T' or 't',  op( B ) = B'.

     TRANSB = 'C' or 'c',  op( B ) = conjg( B' ).

     Unchanged on exit.

     M      - INTEGER.
              On entry,  M  specifies  the number  of rows  of the
              matrix op( A )  and of the  matrix  C.  M  must  be
              at least  zero.  Unchanged on exit.

     N      - INTEGER.
              On entry,  N  specifies the number  of columns of the
              matrix op( B ) and the number of columns of the
              matrix C. N must be at least zero.  Unchanged on
              exit.

     K      - INTEGER.
              On entry,  K  specifies  the number of columns of the
              matrix op( A ) and the number of rows of the matrix
              op( B ). K must be at least  zero.  Unchanged on
              exit.

     ALPHA  - COMPLEX*16      .
              On entry, ALPHA specifies the scalar alpha.
              Unchanged on exit.

ka is
     A      -
               COMPLEX*16       array of DIMENSION ( LDA, ka ), where
              k  when  TRANSA = 'N' or 'n',  and is  m  otherwise.
              Before entry with  TRANSA = 'N' or 'n',  the leading
              m by k part of the array  A  must contain the matrix
              A,  otherwise the leading  k by m  part of the array
              A  must contain  the matrix A.  Unchanged on exit.

     LDA    - INTEGER.
              On entry, LDA specifies the first dimension of A as
              declared in the calling (sub) program. When  TRANSA =
              'N' or 'n' then LDA must be at least  max( 1, m ),
              otherwise  LDA must be at least  max( 1, k ).
              Unchanged on exit.

kb is
     B      -
               COMPLEX*16       array of DIMENSION ( LDB, kb ), where
              n  when  TRANSB = 'N' or 'n',  and is  k  otherwise.
```

```
            Before entry with  TRANSB = 'N' or 'n',  the leading
            k by n part of the array  B  must contain the matrix
            B,  otherwise the leading  n by k  part of the array
            B  must contain  the matrix B.  Unchanged on exit.

      LDB    - INTEGER.
            On entry, LDB specifies the first dimension of B as
            declared in the calling (sub) program. When  TRANSB =
            'N' or 'n' then LDB must be at least  max( 1, k ),

            otherwise  LDB must be at least  max( 1, n ).
            Unchanged on exit.

      BETA   - COMPLEX*16      .
            On entry,  BETA  specifies the scalar  beta.  When
            BETA  is supplied as zero then C need not be set on
            input.  Unchanged on exit.

      C      - COMPLEX*16       array of DIMENSION ( LDC, n ).
            Before entry, the leading  m by n  part of the array
            C must contain the matrix  C,  except when  beta  is
            zero, in which case C need not be set on entry.  On
            exit, the array  C  is overwritten by the  m by n
            matrix ( alpha*op( A )*op( B ) + beta*C ).

      LDC    - INTEGER.
            On entry, LDC specifies the first dimension of C as
            declared in  the  calling  (sub)  program.   LDC
            must  be  at  least max( 1, m ).  Unchanged on exit.
```

———-

17.1.3 fortran code

— **zgemm.f** —

```
*  ===================================================================
      SUBROUTINE ZGEMM(TRANSA,TRANSB,M,N,K,ALPHA,A,LDA,B,LDB,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.6.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      COMPLEX*16 ALPHA,BETA
      INTEGER K,LDA,LDB,LDC,M,N
```

```
      CHARACTER TRANSA,TRANSB
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),B(LDB,*),C(LDC,*)
*     ..
*
*  =====================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DCONJG,MAX
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP
      INTEGER I,INFO,J,L,NCOLA,NROWA,NROWB
      LOGICAL CONJA,CONJB,NOTA,NOTB
*     ..
*     .. Parameters ..
      COMPLEX*16 ONE
      PARAMETER (ONE= (1.0D+0,0.0D+0))
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*
*     Set  NOTA  and  NOTB  as  true if  A  and  B  respectively are not
*     conjugated or transposed, set  CONJA and CONJB  as true if  A  and
*     B  respectively are to be  transposed but  not conjugated  and set
*     NROWA, NCOLA and  NROWB  as the number of rows and  columns  of  A
*     and the number of rows of  B  respectively.
*
      NOTA = LSAME(TRANSA,'N')
      NOTB = LSAME(TRANSB,'N')
      CONJA = LSAME(TRANSA,'C')
      CONJB = LSAME(TRANSB,'C')
      IF (NOTA) THEN
          NROWA = M
          NCOLA = K
      ELSE
          NROWA = K
          NCOLA = M
      END IF
      IF (NOTB) THEN
          NROWB = K
      ELSE
```

```
         NROWB = N
      END IF
*
*     Test the input parameters.
*
      INFO = 0
      IF ((.NOT.NOTA) .AND. (.NOT.CONJA) .AND.
     +    (.NOT.LSAME(TRANSA,'T'))) THEN
         INFO = 1
      ELSE IF ((.NOT.NOTB) .AND. (.NOT.CONJB) .AND.
     +         (.NOT.LSAME(TRANSB,'T'))) THEN
         INFO = 2
      ELSE IF (M.LT.0) THEN
         INFO = 3
      ELSE IF (N.LT.0) THEN
         INFO = 4
      ELSE IF (K.LT.0) THEN
         INFO = 5
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
         INFO = 8
      ELSE IF (LDB.LT.MAX(1,NROWB)) THEN
         INFO = 10
      ELSE IF (LDC.LT.MAX(1,M)) THEN
         INFO = 13
      END IF
      IF (INFO.NE.0) THEN
         CALL XERBLA('ZGEMM ',INFO)
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR.
     +    (((ALPHA.EQ.ZERO).OR. (K.EQ.0)).AND. (BETA.EQ.ONE))) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
         IF (BETA.EQ.ZERO) THEN
            DO 20 J = 1,N
               DO 10 I = 1,M
                  C(I,J) = ZERO
   10          CONTINUE
   20       CONTINUE
         ELSE
            DO 40 J = 1,N
               DO 30 I = 1,M
                  C(I,J) = BETA*C(I,J)
   30          CONTINUE
   40       CONTINUE
```

```
         END IF
         RETURN
      END IF
*
*     Start the operations.
*
      IF (NOTB) THEN
         IF (NOTA) THEN
*
*          Form  C := alpha*A*B + beta*C.
*
            DO 90 J = 1,N
               IF (BETA.EQ.ZERO) THEN
                  DO 50 I = 1,M
                     C(I,J) = ZERO
   50             CONTINUE
               ELSE IF (BETA.NE.ONE) THEN
                  DO 60 I = 1,M
                     C(I,J) = BETA*C(I,J)
   60             CONTINUE
               END IF
               DO 80 L = 1,K
                  TEMP = ALPHA*B(L,J)
                  DO 70 I = 1,M
                     C(I,J) = C(I,J) + TEMP*A(I,L)
   70             CONTINUE
   80          CONTINUE
   90       CONTINUE
         ELSE IF (CONJA) THEN
*
*          Form  C := alpha*A**H*B + beta*C.
*
            DO 120 J = 1,N
               DO 110 I = 1,M
                  TEMP = ZERO
                  DO 100 L = 1,K
                     TEMP = TEMP + DCONJG(A(L,I))*B(L,J)
  100             CONTINUE
                  IF (BETA.EQ.ZERO) THEN
                     C(I,J) = ALPHA*TEMP
                  ELSE
                     C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                  END IF
  110          CONTINUE
  120       CONTINUE
         ELSE
*
*          Form  C := alpha*A**T*B + beta*C
*
            DO 150 J = 1,N
```

```
                DO 140 I = 1,M
                    TEMP = ZERO
                    DO 130 L = 1,K
                        TEMP = TEMP + A(L,I)*B(L,J)
  130               CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP
                    ELSE
                        C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                    END IF
  140           CONTINUE
  150       CONTINUE
        END IF
    ELSE IF (NOTA) THEN
        IF (CONJB) THEN
*
*         Form  C := alpha*A*B**H + beta*C.
*
            DO 200 J = 1,N
                IF (BETA.EQ.ZERO) THEN
                    DO 160 I = 1,M
                        C(I,J) = ZERO
  160               CONTINUE
                ELSE IF (BETA.NE.ONE) THEN
                    DO 170 I = 1,M
                        C(I,J) = BETA*C(I,J)
  170               CONTINUE
                END IF
                DO 190 L = 1,K
                    TEMP = ALPHA*DCONJG(B(J,L))
                    DO 180 I = 1,M
                        C(I,J) = C(I,J) + TEMP*A(I,L)
  180               CONTINUE
  190           CONTINUE
  200       CONTINUE
        ELSE
*
*         Form  C := alpha*A*B**T + beta*C
*
            DO 250 J = 1,N
                IF (BETA.EQ.ZERO) THEN
                    DO 210 I = 1,M
                        C(I,J) = ZERO
  210               CONTINUE
                ELSE IF (BETA.NE.ONE) THEN
                    DO 220 I = 1,M
                        C(I,J) = BETA*C(I,J)
  220               CONTINUE
                END IF
                DO 240 L = 1,K
```

```
                    TEMP = ALPHA*B(J,L)
                    DO 230 I = 1,M
                        C(I,J) = C(I,J) + TEMP*A(I,L)
  230               CONTINUE
  240           CONTINUE
  250       CONTINUE
        END IF
    ELSE IF (CONJA) THEN
        IF (CONJB) THEN
*
*         Form  C := alpha*A**H*B**H + beta*C.
*
            DO 280 J = 1,N
                DO 270 I = 1,M
                    TEMP = ZERO
                    DO 260 L = 1,K
                        TEMP = TEMP + DCONJG(A(L,I))*DCONJG(B(J,L))
  260               CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP
                    ELSE
                        C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                    END IF
  270           CONTINUE
  280       CONTINUE
        ELSE
*
*         Form  C := alpha*A**H*B**T + beta*C
*
            DO 310 J = 1,N
                DO 300 I = 1,M
                    TEMP = ZERO
                    DO 290 L = 1,K
                        TEMP = TEMP + DCONJG(A(L,I))*B(J,L)
  290               CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP
                    ELSE
                        C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                    END IF
  300           CONTINUE
  310       CONTINUE
        END IF
    ELSE
        IF (CONJB) THEN
*
*         Form  C := alpha*A**T*B**H + beta*C
*
            DO 340 J = 1,N
                DO 330 I = 1,M
```

```
                    TEMP = ZERO
                    DO 320 L = 1,K
                        TEMP = TEMP + A(L,I)*DCONJG(B(J,L))
  320               CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP
                    ELSE
                        C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                    END IF
  330            CONTINUE
  340        CONTINUE
         ELSE
*
*          Form  C := alpha*A**T*B**T + beta*C
*
             DO 370 J = 1,N
                 DO 360 I = 1,M
                    TEMP = ZERO
                    DO 350 L = 1,K
                        TEMP = TEMP + A(L,I)*B(J,L)
  350               CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP
                    ELSE
                        C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                    END IF
  360            CONTINUE
  370        CONTINUE
         END IF
      END IF
*
      RETURN
*
*     End of ZGEMM .
*
      END
```

———-

17.1.4 lisp code

— BLAS 3 zgemm —

```
(let* ((one (complex 1.0 0.0)) (zero (complex 0.0 0.0)))
 (declare (type (complex double-float) one) (type (complex double-float) zero))
  (defun zgemm (transa transb m n k alpha a lda b ldb$ beta c ldc)
    (declare (type (simple-array (complex double-float) (*)) c b a)
```

```
            (type (complex double-float) beta alpha)
            (type fixnum ldc ldb$ lda k n m)
            (type character transb transa))
    (f2cl-lib:with-multi-array-data
        ((transa character transa-%data% transa-%offset%)
         (transb character transb-%data% transb-%offset%)
         (a (complex double-float) a-%data% a-%offset%)
         (b (complex double-float) b-%data% b-%offset%)
         (c (complex double-float) c-%data% c-%offset%))
      (prog ((temp #C(0.0 0.0)) (i 0) (info 0) (j 0) (l 0) (ncola 0) (nrowa 0)
             (nrowb 0) (conja nil) (conjb nil) (nota nil) (notb nil))
        (declare (type (complex double-float) temp)
                 (type fixnum i info j l ncola nrowa nrowb)
                 (type (member t nil) conja conjb nota notb))
        (setf nota (char-equal transa #\N))
        (setf notb (char-equal transb #\N))
        (setf conja (char-equal transa #\C))
        (setf conjb (char-equal transb #\C))
        (cond
          (nota
           (setf nrowa m)
           (setf ncola k))
          (t
           (setf nrowa k)
           (setf ncola m)))
        (cond
          (notb
           (setf nrowb k))
          (t
           (setf nrowb n)))
        (setf info 0)
        (cond
          ((and (not nota) (not conja) (not (char-equal transa #\T)))
           (setf info 1))
          ((and (not notb) (not conjb) (not (char-equal transb #\T)))
           (setf info 2))
          ((< m 0)
           (setf info 3))
          ((< n 0)
           (setf info 4))
          ((< k 0)
           (setf info 5))
          ((< lda (max (the fixnum 1) (the fixnum nrowa)))
           (setf info 8))
          ((< ldb$
              (max (the fixnum 1) (the fixnum nrowb)))
           (setf info 10))
          ((< ldc (max (the fixnum 1) (the fixnum m)))
           (setf info 13)))
        (cond
```

```
  ((/= info 0)
   (error
    " ** On entry to ~a parameter number ~a had an illegal value~%"
    "ZGEMM" info)
   (go end_label)))
(if (or (= m 0) (= n 0) (and (or (= alpha zero) (= k 0)) (= beta one)))
    (go end_label))
(cond
  ((= alpha zero)
   (cond
     ((= beta zero)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data%
                                   (i j)
                                   ((1 ldc) (1 *))
                                   c-%offset%)
                    zero))))))
     (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data%
                                   (i j)
                                   ((1 ldc) (1 *))
                                   c-%offset%)
                    (* beta
                       (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)))))))))
   (go end_label)))
(cond
  (notb
   (cond
     (nota
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
            ((= beta zero)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i m) nil)
```

```
                (tagbody
                  (setf (f2cl-lib:fref c-%data%
                                       (i j)
                                       ((1 ldc) (1 *))
                                       c-%offset%)
                        zero))))
            ((/= beta one)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i m) nil)
                (tagbody
                  (setf (f2cl-lib:fref c-%data%
                                       (i j)
                                       ((1 ldc) (1 *))
                                       c-%offset%)
                        (* beta
                           (f2cl-lib:fref c-%data%
                                          (i j)
                                          ((1 ldc) (1 *))
                                          c-%offset%)))))))
        (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                      ((> l k) nil)
          (tagbody
            (cond
              ((/= (f2cl-lib:fref b (l j) ((1 ldb$) (1 *))) zero)
               (setf temp
                       (* alpha
                          (f2cl-lib:fref b-%data%
                                         (l j)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)))
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i m) nil)
                 (tagbody
                   (setf (f2cl-lib:fref c-%data%
                                        (i j)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                         (+
                          (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)
                          (* temp
                             (f2cl-lib:fref a-%data%
                                            (i l)
                                            ((1 lda) (1 *))
                                            a-%offset%))))))))))
    (conja
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
```

```
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf temp zero)
            (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                          ((> l k) nil)
              (tagbody
                (setf temp
                        (+ temp
                           (*
                            (f2cl-lib:dconjg
                             (f2cl-lib:fref a-%data%
                                            (l i)
                                            ((1 lda) (1 *))
                                            a-%offset%))
                            (f2cl-lib:fref b-%data%
                                           (l j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%))))))
            (cond
              ((= beta zero)
               (setf (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                     (* alpha temp)))
              (t
               (setf (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                     (+ (* alpha temp)
                        (* beta
                           (f2cl-lib:fref c-%data%
                                          (i j)
                                          ((1 ldc) (1 *))
                                          c-%offset%))))))))))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf temp zero)
             (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                           ((> l k) nil)
               (tagbody
                 (setf temp
```

```
                                (+ temp
                                   (*
                                    (f2cl-lib:fref a-%data%
                                                   (l i)
                                                   ((1 lda) (1 *))
                                                   a-%offset%)
                                    (f2cl-lib:fref b-%data%
                                                   (l j)
                                                   ((1 ldb$) (1 *))
                                                   b-%offset%))))))
                  (cond
                    ((= beta zero)
                     (setf (f2cl-lib:fref c-%data%
                                          (i j)
                                          ((1 ldc) (1 *))
                                          c-%offset%)
                           (* alpha temp)))
                    (t
                     (setf (f2cl-lib:fref c-%data%
                                          (i j)
                                          ((1 ldc) (1 *))
                                          c-%offset%)
                           (+ (* alpha temp)
                              (* beta
                                 (f2cl-lib:fref c-%data%
                                                (i j)
                                                ((1 ldc) (1 *))
                                                c-%offset%))))))))))))))
(nota
 (cond
   (conjb
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
          ((= beta zero)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                     zero))))
          ((/= beta one)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf (f2cl-lib:fref c-%data%
                                    (i j)
```

```
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                              (* beta
                                 (f2cl-lib:fref c-%data%
                                                (i j)
                                                ((1 ldc) (1 *))
                                                c-%offset%)))))))
          (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                        ((> l k) nil)
            (tagbody
              (cond
                ((/= (f2cl-lib:fref b (j l) ((1 ldb$) (1 *))) zero)
                 (setf temp
                         (* alpha
                            (f2cl-lib:dconjg
                             (f2cl-lib:fref b-%data%
                                            (j l)
                                            ((1 ldb$) (1 *))
                                            b-%offset%))))
                 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                               ((> i m) nil)
                   (tagbody
                     (setf (f2cl-lib:fref c-%data%
                                          (i j)
                                          ((1 ldc) (1 *))
                                          c-%offset%)
                           (+
                            (f2cl-lib:fref c-%data%
                                           (i j)
                                           ((1 ldc) (1 *))
                                           c-%offset%)
                            (* temp
                               (f2cl-lib:fref a-%data%
                                              (i l)
                                              ((1 lda) (1 *))
                                              a-%offset%))))))))))))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (cond
           ((= beta zero)
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data%
                                     (i j)
                                     ((1 ldc) (1 *))
                                     c-%offset%)
                      zero))))
```

```
                ((/= beta one)
                 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                               ((> i m) nil)
                   (tagbody
                     (setf (f2cl-lib:fref c-%data%
                                          (i j)
                                          ((1 ldc) (1 *))
                                          c-%offset%)
                           (* beta
                              (f2cl-lib:fref c-%data%
                                             (i j)
                                             ((1 ldc) (1 *))
                                             c-%offset%)))))))
            (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                          ((> l k) nil)
              (tagbody
                (cond
                  ((/= (f2cl-lib:fref b (j l) ((1 ldb$) (1 *))) zero)
                   (setf temp
                           (* alpha
                              (f2cl-lib:fref b-%data%
                                             (j l)
                                             ((1 ldb$) (1 *))
                                             b-%offset%)))
                   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                 ((> i m) nil)
                     (tagbody
                       (setf (f2cl-lib:fref c-%data%
                                            (i j)
                                            ((1 ldc) (1 *))
                                            c-%offset%)
                             (+
                              (f2cl-lib:fref c-%data%
                                             (i j)
                                             ((1 ldc) (1 *))
                                             c-%offset%)
                              (* temp
                                 (f2cl-lib:fref a-%data%
                                                (i l)
                                                ((1 lda) (1 *))
                                                a-%offset%))))))))))))))
(conja
 (cond
   (conjb
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
```

```
            (setf temp zero)
            (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                          ((> l k) nil)
              (tagbody
                (setf temp
                        (+ temp
                           (*
                            (f2cl-lib:dconjg
                             (f2cl-lib:fref a-%data%
                                            (l i)
                                            ((1 lda) (1 *))
                                            a-%offset%))
                            (f2cl-lib:dconjg
                             (f2cl-lib:fref b-%data%
                                            (j l)
                                            ((1 ldb$) (1 *))
                                            b-%offset%)))))))
            (cond
              ((= beta zero)
               (setf (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                     (* alpha temp)))
              (t
               (setf (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                     (+ (* alpha temp)
                        (* beta
                           (f2cl-lib:fref c-%data%
                                          (i j)
                                          ((1 ldc) (1 *))
                                          c-%offset%))))))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i m) nil)
       (tagbody
         (setf temp zero)
         (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                       ((> l k) nil)
           (tagbody
             (setf temp
                     (+ temp
                        (*
                         (f2cl-lib:dconjg
```

```
                                    (f2cl-lib:fref a-%data%
                                                   (l i)
                                                   ((1 lda) (1 *))
                                                   a-%offset%))
                                  (f2cl-lib:fref b-%data%
                                                 (j l)
                                                 ((1 ldb$) (1 *))
                                                 b-%offset%))))))
                 (cond
                   ((= beta zero)
                    (setf (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)
                          (* alpha temp)))
                   (t
                    (setf (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)
                          (+ (* alpha temp)
                             (* beta
                                (f2cl-lib:fref c-%data%
                                               (i j)
                                               ((1 ldc) (1 *))
                                               c-%offset%))))))))))))
(t
 (cond
   (conjb
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf temp zero)
            (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                          ((> l k) nil)
              (tagbody
                (setf temp
                        (+ temp
                           (*
                            (f2cl-lib:fref a-%data%
                                           (l i)
                                           ((1 lda) (1 *))
                                           a-%offset%)
                            (f2cl-lib:dconjg
                             (f2cl-lib:fref b-%data%
                                            (j l)
                                            ((1 ldb$) (1 *))
```

```
                                                b-%offset%)))))))
                  (cond
                    ((= beta zero)
                     (setf (f2cl-lib:fref c-%data%
                                          (i j)
                                          ((1 ldc) (1 *))
                                          c-%offset%)
                             (* alpha temp)))
                    (t
                     (setf (f2cl-lib:fref c-%data%
                                          (i j)
                                          ((1 ldc) (1 *))
                                          c-%offset%)
                             (+ (* alpha temp)
                                (* beta
                                   (f2cl-lib:fref c-%data%
                                                  (i j)
                                                  ((1 ldc) (1 *))
                                                  c-%offset%))))))))))
          (t
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i m) nil)
                 (tagbody
                   (setf temp zero)
                   (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                                 ((> l k) nil)
                     (tagbody
                       (setf temp
                               (+ temp
                                  (*
                                   (f2cl-lib:fref a-%data%
                                                  (l i)
                                                  ((1 lda) (1 *))
                                                  a-%offset%)
                                   (f2cl-lib:fref b-%data%
                                                  (j l)
                                                  ((1 ldb$) (1 *))
                                                  b-%offset%))))))
                   (cond
                     ((= beta zero)
                      (setf (f2cl-lib:fref c-%data%
                                           (i j)
                                           ((1 ldc) (1 *))
                                           c-%offset%)
                              (* alpha temp)))
                     (t
                      (setf (f2cl-lib:fref c-%data%
```

```
                                                (i j)
                                                ((1 ldc) (1 *))
                                                c-%offset%)
                                        (+ (* alpha temp)
                                           (* beta
                                              (f2cl-lib:fref c-%data%
                                                             (i j)
                                                             ((1 ldc) (1 *))
                                                             c-%offset%)))))))))))))))
 end_label
        (return
         (values nil nil nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

17.2 zhemm - hermitian $\alpha AB + \beta C$ or $\alpha BA + \beta C$

17.2.1 Axiom unit tests

— **zhemm.input** —

```
)set break resume
)sys rm -f zhemm.output
)spool zhemm.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

17.2.2 Axiom help page

— **zhemm.help** —

```
======================================================================
zhemm examples
======================================================================

======================================================================
Man Page Details
```

```
=====================================================================

 NAME
      ZHEMM - perform one of the matrix-matrix operations
              C := alpha*A*B + beta*C,

 SYNOPSIS
      SUBROUTINE ZHEMM ( SIDE, UPLO, M, N, ALPHA, A, LDA, B, LDB,
                         BETA, C, LDC )

          CHARACTER*1  SIDE, UPLO

          INTEGER      M, N, LDA, LDB, LDC

          COMPLEX*16   ALPHA, BETA

          COMPLEX*16   A( LDA, * ), B( LDB, * ), C( LDC, * )

 PURPOSE
      ZHEMM  performs one of the matrix-matrix operations

      or

         C := alpha*B*A + beta*C,

      where alpha and beta are scalars, A is an hermitian matrix
      and  B and C are m by n matrices.

 PARAMETERS
      SIDE   - CHARACTER*1.
             On entry,  SIDE  specifies whether  the  hermitian
             matrix  A appears on the  left or right  in the
             operation as follows:

             SIDE = 'L' or 'l'   C := alpha*A*B + beta*C,

             SIDE = 'R' or 'r'   C := alpha*B*A + beta*C,

             Unchanged on exit.

      UPLO   - CHARACTER*1.
             On  entry,   UPLO  specifies  whether  the  upper  or
             lower triangular  part  of  the  hermitian  matrix
             A  is  to  be referenced as follows:

             UPLO = 'U' or 'u'   Only the upper triangular part of
             the hermitian matrix is to be referenced.

             UPLO = 'L' or 'l'   Only the lower triangular part of
             the hermitian matrix is to be referenced.
```

```
        Unchanged on exit.

M       - INTEGER.
        On entry,  M  specifies the number of rows of the
        matrix  C.  M  must be at least zero.  Unchanged on
        exit.

N       - INTEGER.
        On entry, N specifies the number of columns of the
        matrix C.  N  must be at least zero.  Unchanged on
        exit.

ALPHA   - COMPLEX*16      .
        On entry, ALPHA specifies the scalar alpha.
        Unchanged on exit.

A       -
         COMPLEX*16       array of DIMENSION ( LDA, ka ), where
        ka is m  when  SIDE = 'L' or 'l'  and is n  otherwise.
        Before entry  with  SIDE = 'L' or 'l',  the  m by m
        part of the array  A  must contain the  hermitian
        matrix,  such that when  UPLO = 'U' or 'u', the lead-
        ing m by m upper triangular part of the array  A
        must contain the upper triangular part of the  hermi-
        tian matrix and the  strictly  lower triangular part
        of  A  is not referenced,  and when  UPLO = 'L' or
        'l', the leading  m by m  lower triangular part  of
        the  array  A must  contain  the  lower triangular
        part  of the  hermitian matrix and the  strictly
        upper triangular part of  A  is not referenced.
        Before entry  with  SIDE = 'R' or 'r',  the  n by n
        part of the array  A  must contain the  hermitian
        matrix,  such that when  UPLO = 'U' or 'u', the lead-
        ing n by n upper triangular part of the array  A
        must contain the upper triangular part of the  hermi-
        tian matrix and the  strictly  lower triangular part
        of  A  is not referenced,  and when  UPLO = 'L' or
        'l', the leading  n by n  lower triangular part  of
        the  array  A must  contain  the  lower triangular
        part  of the  hermitian matrix and the  strictly
        upper triangular part of  A  is not referenced.  Note
        that the imaginary parts  of the diagonal elements
        need not be set, they are assumed to be zero.
        Unchanged on exit.

LDA     - INTEGER.
        On entry, LDA specifies the first dimension of A as
        declared in the  calling (sub) program. When  SIDE =
        'L' or 'l'  then LDA must be at least  max( 1, m ),
```

```
            otherwise  LDA must be at least max( 1, n ).
            Unchanged on exit.

      B     - COMPLEX*16       array of DIMENSION ( LDB, n ).

            Before entry, the leading  m by n part of the array
            B  must contain the matrix B.  Unchanged on exit.

      LDB   - INTEGER.
            On entry, LDB specifies the first dimension of B as
            declared in  the  calling  (sub)  program.   LDB
            must  be  at  least max( 1, m ).  Unchanged on exit.

      BETA  - COMPLEX*16      .
            On entry,  BETA  specifies the scalar  beta.  When
            BETA  is supplied as zero then C need not be set on
            input.  Unchanged on exit.

      C     - COMPLEX*16       array of DIMENSION ( LDC, n ).
            Before entry, the leading  m by n  part of the array
            C must contain the matrix  C,  except when  beta  is
            zero, in which case C need not be set on entry.  On
            exit, the array  C  is overwritten by the  m by n
            updated matrix.

      LDC   - INTEGER.
            On entry, LDC specifies the first dimension of C as
            declared in  the  calling  (sub)  program.   LDC
            must  be  at  least max( 1, m ).  Unchanged on exit.
```

17.2.3 fortran code

— zhemm.f —

```
*  ===================================================================
      SUBROUTINE ZHEMM(SIDE,UPLO,M,N,ALPHA,A,LDA,B,LDB,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX*16 ALPHA,BETA
      INTEGER LDA,LDB,LDC,M,N
```

```
      CHARACTER SIDE,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),B(LDB,*),C(LDC,*)
*     ..
*
*  =====================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DBLE,DCONJG,MAX
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP1,TEMP2
      INTEGER I,INFO,J,K,NROWA
      LOGICAL UPPER
*     ..
*     .. Parameters ..
      COMPLEX*16 ONE
      PARAMETER (ONE= (1.0D+0,0.0D+0))
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*
*     Set NROWA as the number of rows of A.
*
      IF (LSAME(SIDE,'L')) THEN
          NROWA = M
      ELSE
          NROWA = N
      END IF
      UPPER = LSAME(UPLO,'U')
*
*     Test the input parameters.
*
      INFO = 0
      IF ((.NOT.LSAME(SIDE,'L')) .AND. (.NOT.LSAME(SIDE,'R'))) THEN
          INFO = 1
      ELSE IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 2
      ELSE IF (M.LT.0) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
```

```
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 7
      ELSE IF (LDB.LT.MAX(1,M)) THEN
          INFO = 9
      ELSE IF (LDC.LT.MAX(1,M)) THEN
          INFO = 12
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZHEMM ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR.
     +    ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
          IF (BETA.EQ.ZERO) THEN
              DO 20 J = 1,N
                  DO 10 I = 1,M
                      C(I,J) = ZERO
   10             CONTINUE
   20         CONTINUE
          ELSE
              DO 40 J = 1,N
                  DO 30 I = 1,M
                      C(I,J) = BETA*C(I,J)
   30             CONTINUE
   40         CONTINUE
          END IF
          RETURN
      END IF
*
*     Start the operations.
*
      IF (LSAME(SIDE,'L')) THEN
*
*        Form  C := alpha*A*B + beta*C.
*
          IF (UPPER) THEN
              DO 70 J = 1,N
                  DO 60 I = 1,M
                      TEMP1 = ALPHA*B(I,J)
                      TEMP2 = ZERO
                      DO 50 K = 1,I - 1
                          C(K,J) = C(K,J) + TEMP1*A(K,I)
                          TEMP2 = TEMP2 + B(K,J)*DCONJG(A(K,I))
```

```
   50                 CONTINUE
                      IF (BETA.EQ.ZERO) THEN
                          C(I,J) = TEMP1*DBLE(A(I,I)) + ALPHA*TEMP2
                      ELSE
                          C(I,J) = BETA*C(I,J) + TEMP1*DBLE(A(I,I)) +
     +                             ALPHA*TEMP2
                      END IF
   60             CONTINUE
   70         CONTINUE
          ELSE
              DO 100 J = 1,N
                  DO 90 I = M,1,-1
                      TEMP1 = ALPHA*B(I,J)
                      TEMP2 = ZERO
                      DO 80 K = I + 1,M
                          C(K,J) = C(K,J) + TEMP1*A(K,I)
                          TEMP2 = TEMP2 + B(K,J)*DCONJG(A(K,I))
   80                 CONTINUE
                      IF (BETA.EQ.ZERO) THEN
                          C(I,J) = TEMP1*DBLE(A(I,I)) + ALPHA*TEMP2
                      ELSE
                          C(I,J) = BETA*C(I,J) + TEMP1*DBLE(A(I,I)) +
     +                             ALPHA*TEMP2
                      END IF
   90             CONTINUE
  100         CONTINUE
          END IF
      ELSE
*
*        Form  C := alpha*B*A + beta*C.
*
          DO 170 J = 1,N
              TEMP1 = ALPHA*DBLE(A(J,J))
              IF (BETA.EQ.ZERO) THEN
                  DO 110 I = 1,M
                      C(I,J) = TEMP1*B(I,J)
  110             CONTINUE
              ELSE
                  DO 120 I = 1,M
                      C(I,J) = BETA*C(I,J) + TEMP1*B(I,J)
  120             CONTINUE
              END IF
              DO 140 K = 1,J - 1
                  IF (UPPER) THEN
                      TEMP1 = ALPHA*A(K,J)
                  ELSE
                      TEMP1 = ALPHA*DCONJG(A(J,K))
                  END IF
                  DO 130 I = 1,M
                      C(I,J) = C(I,J) + TEMP1*B(I,K)
```

```
  130             CONTINUE
  140         CONTINUE
              DO 160 K = J + 1,N
                  IF (UPPER) THEN
                      TEMP1 = ALPHA*DCONJG(A(J,K))
                  ELSE
                      TEMP1 = ALPHA*A(K,J)
                  END IF
                  DO 150 I = 1,M
                      C(I,J) = C(I,J) + TEMP1*B(I,K)
  150             CONTINUE
  160         CONTINUE
  170     CONTINUE
      END IF
*
      RETURN
*
*     End of ZHEMM .
*
      END
```

———-

17.2.4 lisp code

— BLAS 3 zhemm —

```
(let* ((one (complex 1.0 0.0)) (zero (complex 0.0 0.0)))
 (declare (type (complex double-float) one) (type (complex double-float) zero))
  (defun zhemm (side uplo m n alpha a lda b ldb$ beta c ldc)
    (declare (type (simple-array (complex double-float) (*)) c b a)
             (type (complex double-float) beta alpha)
             (type fixnum ldc ldb$ lda n m)
             (type character uplo side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a (complex double-float) a-%data% a-%offset%)
         (b (complex double-float) b-%data% b-%offset%)
         (c (complex double-float) c-%data% c-%offset%))
      (prog ((temp1 #C(0.0 0.0)) (temp2 #C(0.0 0.0)) (i 0) (info 0) (j 0) (k 0)
             (nrowa 0) (upper nil))
        (declare (type (complex double-float) temp1 temp2)
                 (type fixnum i info j k nrowa)
                 (type (member t nil) upper))
        (cond
          ((char-equal side #\L)
```

```
    (setf nrowa m))
   (t
    (setf nrowa n)))
(setf upper (char-equal uplo #\U))
(setf info 0)
(cond
  ((and (not (char-equal side #\L)) (not (char-equal side #\R)))
   (setf info 1))
  ((and (not upper) (not (char-equal uplo #\L)))
   (setf info 2))
  ((< m 0)
   (setf info 3))
  ((< n 0)
   (setf info 4))
  ((< lda (max (the fixnum 1) (the fixnum nrowa)))
   (setf info 7))
  ((< ldb$ (max (the fixnum 1) (the fixnum m)))
   (setf info 9))
  ((< ldc (max (the fixnum 1) (the fixnum m)))
   (setf info 12)))
(cond
  ((/= info 0)
   (error
    " ** On entry to ~a parameter number ~a had an illegal value~%"
    "ZHEMM" info)
   (go end_label)))
(if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
    (go end_label))
(cond
  ((= alpha zero)
   (cond
     ((= beta zero)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data%
                                   (i j)
                                   ((1 ldc) (1 *))
                                   c-%offset%)
                    zero))))))
     (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
```

```
                (setf (f2cl-lib:fref c-%data%
                                     (i j)
                                     ((1 ldc) (1 *))
                                     c-%offset%)
                        (* beta
                           (f2cl-lib:fref c-%data%
                                          (i j)
                                          ((1 ldc) (1 *))
                                          c-%offset%)))))))))
   (go end_label)))
(cond
  ((char-equal side #\L)
   (cond
     (upper
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf temp1
                      (* alpha
                         (f2cl-lib:fref b-%data%
                                        (i j)
                                        ((1 ldb$) (1 *))
                                        b-%offset%)))
              (setf temp2 zero)
              (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                            ((> k
                                (f2cl-lib:int-add i
                                                  (f2cl-lib:int-sub
                                                   1)))
                             nil)
                (tagbody
                  (setf (f2cl-lib:fref c-%data%
                                       (k j)
                                       ((1 ldc) (1 *))
                                       c-%offset%)
                        (+
                         (f2cl-lib:fref c-%data%
                                        (k j)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                         (* temp1
                            (f2cl-lib:fref a-%data%
                                           (k i)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
                  (setf temp2
                          (+ temp2
```

```
                                (*
                                 (f2cl-lib:fref b-%data%
                                                (k j)
                                                ((1 ldb$) (1 *))
                                                b-%offset%)
                                 (f2cl-lib:dconjg
                                  (f2cl-lib:fref a-%data%
                                                 (k i)
                                                 ((1 lda) (1 *))
                                                 a-%offset%)))))))
            (cond
              ((= beta zero)
               (setf
                (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                               c-%offset%)
                (+
                 (* temp1
                  (coerce (realpart
                   (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                  a-%offset%))
                     'double-float))
                 (* alpha temp2))))
              (t
               (setf (f2cl-lib:fref c-%data%
                               (i j)
                               ((1 ldc) (1 *))
                               c-%offset%)
                       (+
                        (* beta
                           (f2cl-lib:fref c-%data%
                                          (i j)
                                          ((1 ldc) (1 *))
                                          c-%offset%))
                        (* temp1
                           (coerce (realpart
                            (f2cl-lib:fref a-%data%
                                      (i i)
                                      ((1 lda) (1 *))
                                      a-%offset%)) 'double-float))

                        (* alpha temp2)))))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                   ((> i 1) nil)
       (tagbody
         (setf temp1
                 (* alpha
```

```
            (f2cl-lib:fref b-%data%
                           (i j)
                           ((1 ldb$) (1 *))
                           b-%offset%)))
(setf temp2 zero)
(f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
               (f2cl-lib:int-add k 1))
              ((> k m) nil)
  (tagbody
    (setf (f2cl-lib:fref c-%data%
                         (k j)
                         ((1 ldc) (1 *))
                         c-%offset%)
            (+
             (f2cl-lib:fref c-%data%
                            (k j)
                            ((1 ldc) (1 *))
                            c-%offset%)
             (* temp1
                (f2cl-lib:fref a-%data%
                               (k i)
                               ((1 lda) (1 *))
                               a-%offset%))))
    (setf temp2
            (+ temp2
               (*
                (f2cl-lib:fref b-%data%
                               (k j)
                               ((1 ldb$) (1 *))
                               b-%offset%)
                (f2cl-lib:dconjg
                 (f2cl-lib:fref a-%data%
                                (k i)
                                ((1 lda) (1 *))
                                a-%offset%)))))))
(cond
  ((= beta zero)
   (setf (f2cl-lib:fref c-%data%
                        (i j)
                        ((1 ldc) (1 *))
                        c-%offset%)
            (+
             (* temp1
               (coerce (realpart
                (f2cl-lib:fref a-%data%
                          (i i)
                          ((1 lda) (1 *))
                          a-%offset%)) 'double-float))
             (* alpha temp2))))
  (t
```

```
                    (setf (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)
                          (+
                           (* beta
                              (f2cl-lib:fref c-%data%
                                             (i j)
                                             ((1 ldc) (1 *))
                                             c-%offset%))
                           (* temp1
                              (coerce (realpart
                               (f2cl-lib:fref a-%data%
                                              (i i)
                                              ((1 lda) (1 *))
                                              a-%offset%)) 'double-float))
                           (* alpha temp2))))))))))))
          (t
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (setf temp1
                       (* alpha
                          (coerce (realpart
                           (f2cl-lib:fref a-%data%
                                          (j j)
                                          ((1 lda) (1 *))
                                          a-%offset%)) 'double-float)))
               (cond
                 ((= beta zero)
                  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                ((> i m) nil)
                    (tagbody
                      (setf (f2cl-lib:fref c-%data%
                                           (i j)
                                           ((1 ldc) (1 *))
                                           c-%offset%)
                            (* temp1
                               (f2cl-lib:fref b-%data%
                                              (i j)
                                              ((1 ldb$) (1 *))
                                              b-%offset%))))))
                 (t
                  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                ((> i m) nil)
                    (tagbody
                      (setf (f2cl-lib:fref c-%data%
                                           (i j)
                                           ((1 ldc) (1 *))
                                           c-%offset%)
```

```
                    (+
                     (* beta
                        (f2cl-lib:fref c-%data%
                                       (i j)
                                       ((1 ldc) (1 *))
                                       c-%offset%))
                     (* temp1
                        (f2cl-lib:fref b-%data%
                                       (i j)
                                       ((1 ldb$) (1 *))
                                       b-%offset%))))))))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (cond
      (upper
       (setf temp1
               (* alpha
                  (f2cl-lib:fref a-%data%
                                 (k j)
                                 ((1 lda) (1 *))
                                 a-%offset%))))
      (t
       (setf temp1
               (* alpha
                  (f2cl-lib:dconjg
                   (f2cl-lib:fref a-%data%
                                  (j k)
                                  ((1 lda) (1 *))
                                  a-%offset%))))))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data%
                             (i j)
                             ((1 ldc) (1 *))
                             c-%offset%)
                (+
                 (f2cl-lib:fref c-%data%
                                (i j)
                                ((1 ldc) (1 *))
                                c-%offset%)
                 (* temp1
                    (f2cl-lib:fref b-%data%
                                   (i k)
                                   ((1 ldb$) (1 *))
                                   b-%offset%))))))))
(f2cl-lib:fdo (k (f2cl-lib:int-add j 1) (f2cl-lib:int-add k 1))
              ((> k n) nil)
```

```
                (tagbody
                  (cond
                    (upper
                     (setf temp1
                             (* alpha
                                (f2cl-lib:dconjg
                                 (f2cl-lib:fref a-%data%
                                                (j k)
                                                ((1 lda) (1 *))
                                                a-%offset%)))))
                    (t
                     (setf temp1
                             (* alpha
                                (f2cl-lib:fref a-%data%
                                               (k j)
                                               ((1 lda) (1 *))
                                               a-%offset%)))))
                  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                ((> i m) nil)
                    (tagbody
                      (setf (f2cl-lib:fref c-%data%
                                           (i j)
                                           ((1 ldc) (1 *))
                                           c-%offset%)
                            (+
                             (f2cl-lib:fref c-%data%
                                            (i j)
                                            ((1 ldc) (1 *))
                                            c-%offset%)
                             (* temp1
                                (f2cl-lib:fref b-%data%
                                               (i k)
                                               ((1 ldb$) (1 *))
                                               b-%offset%)))))))))))
end_label
       (return (values nil nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

17.3 zher2k - hermitian rank 2k $\alpha AB^H + conjg(\alpha)BA^H + \beta C$ or $\alpha A^H B + conjg(\alpha)B^H A + \beta C$

17.3.1 Axiom unit tests

— **zher2k.input** —

```
)set break resume
)sys rm -f zher2k.output
)spool zher2k.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

17.3.2 Axiom help page

— zher2k.help —

```
==========================================================================
zher2k examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

 NAME
      ZHER2K - perform one of the hermitian rank 2k operations   C
      := alpha*A*conjg( B' ) + conjg( alpha )*B*conjg( A' ) +
      beta*C,

 SYNOPSIS
      SUBROUTINE ZHER2K( UPLO, TRANS, N, K, ALPHA, A, LDA, B, LDB,
                         BETA, C, LDC )

          CHARACTER*1    UPLO, TRANS

          INTEGER        N, K, LDA, LDB, LDC

          DOUBLE         PRECISION BETA

          COMPLEX*16     ALPHA

          COMPLEX*16     A( LDA, * ), B( LDB, * ), C( LDC, * )

 PURPOSE
      ZHER2K  performs one of the hermitian rank 2k operations

      or
```

```
   C := alpha*conjg( A' )*B + conjg( alpha )*conjg( B' )*A +
beta*C,

where  alpha and beta  are scalars with  beta  real,  C is
an  n by n hermitian matrix and  A and B  are  n by k
matrices in the first case and  k by n  matrices in the
second case.

PARAMETERS
     UPLO   - CHARACTER*1.
            On  entry,   UPLO  specifies  whether  the  upper  or
            lower triangular  part  of the  array  C  is to be
            referenced  as follows:

            UPLO = 'U' or 'u'   Only the  upper triangular part
            of  C is to be referenced.

            UPLO = 'L' or 'l'   Only the  lower triangular part
            of  C is to be referenced.

            Unchanged on exit.

     TRANS  - CHARACTER*1.
            On entry,  TRANS  specifies the operation to be per-
            formed as follows:

            TRANS = 'N' or 'n'    C := alpha*A*conjg( B' )

            + conjg( alpha )*B*conjg( A' ) + beta*C.

            TRANS = 'C' or 'c'    C := alpha*conjg( A' )*B
            + conjg( alpha )*conjg( B' )*A + beta*C.

            Unchanged on exit.

     N      - INTEGER.
            On entry,  N specifies the order of the matrix C.  N
            must be at least zero.  Unchanged on exit.

     K      - INTEGER.
            On entry with  TRANS = 'N' or 'n',  K  specifies  the
            number of  columns  of the  matrices  A and B,  and
            on  entry  with TRANS = 'C' or 'c',  K  specifies
            the number of rows of the matrices  A and B.  K must
            be at least zero.  Unchanged on exit.

     ALPHA  - COMPLEX*16      .
            On entry, ALPHA specifies the scalar alpha.
            Unchanged on exit.
```

```
ka is
     A      -
             COMPLEX*16       array of DIMENSION ( LDA, ka ), where
            k  when  TRANS = 'N' or 'n',  and is  n  otherwise.
            Before entry with  TRANS = 'N' or 'n',  the  leading
            n by k part of the array  A  must contain the matrix
            A,  otherwise the leading  k by n  part of the array
            A  must contain  the matrix A.  Unchanged on exit.

     LDA    - INTEGER.
            On entry, LDA specifies the first dimension of A as
            declared in  the  calling  (sub)  program.   When
            TRANS = 'N' or 'n' then  LDA must be at least  max(
            1, n ), otherwise  LDA must be at least  max( 1, k ).
            Unchanged on exit.

kb is
     B      -
             COMPLEX*16       array of DIMENSION ( LDB, kb ), where
            k  when  TRANS = 'N' or 'n',  and is  n  otherwise.
            Before entry with  TRANS = 'N' or 'n',  the  leading
            n by k part of the array  B  must contain the matrix
            B,  otherwise the leading  k by n  part of the array
            B  must contain  the matrix B.  Unchanged on exit.

     LDB    - INTEGER.
            On entry, LDB specifies the first dimension of B as
            declared in  the  calling  (sub)  program.   When
            TRANS = 'N' or 'n' then  LDB must be at least  max(
            1, n ), otherwise  LDB must be at least  max( 1, k ).

            Unchanged on exit.

     BETA   - DOUBLE PRECISION.
            On entry, BETA specifies the scalar beta.  Unchanged
            on exit.

     C      - COMPLEX*16       array of DIMENSION ( LDC, n ).
            Before entry  with  UPLO = 'U' or 'u',  the leading
            n by n upper triangular part of the array C must con-
            tain the upper triangular part  of the  hermitian
            matrix  and the strictly lower triangular part of C
            is not referenced.  On exit, the upper triangular
            part of the array  C is overwritten by the upper tri-
            angular part of the updated matrix.  Before entry
            with  UPLO = 'L' or 'l',  the leading  n by n lower
            triangular part of the array C must contain the lower
            triangular part  of the  hermitian matrix  and the
            strictly upper triangular part of C is not refer-
            enced.  On exit, the lower triangular part of the
```

```
            array  C is overwritten by the lower triangular part
            of the updated matrix.  Note that the imaginary parts
            of the diagonal elements need not be set,  they are
            assumed to be zero,  and on exit they are set to
            zero.

      LDC    - INTEGER.
             On entry, LDC specifies the first dimension of C as
             declared in  the  calling  (sub)  program.   LDC
             must  be  at  least max( 1, n ).  Unchanged on exit.
```

———-

17.3.3 fortran code

— zher2k.f —

```
*  ===================================================================
      SUBROUTINE ZHER2K(UPLO,TRANS,N,K,ALPHA,A,LDA,B,LDB,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX*16 ALPHA
      DOUBLE PRECISION BETA
      INTEGER K,LDA,LDB,LDC,N
      CHARACTER TRANS,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),B(LDB,*),C(LDC,*)
*     ..
*
*  ====================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DBLE,DCONJG,MAX
*     ..
```

```
*     .. Local Scalars ..
      COMPLEX*16 TEMP1,TEMP2
      INTEGER I,INFO,J,L,NROWA
      LOGICAL UPPER
*     ..
*     .. Parameters ..
      DOUBLE PRECISION ONE
      PARAMETER (ONE=1.0D+0)
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*
*     Test the input parameters.
*
      IF (LSAME(TRANS,'N')) THEN
          NROWA = N
      ELSE
          NROWA = K
      END IF
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 1
      ELSE IF ((.NOT.LSAME(TRANS,'N')) .AND.
     +         (.NOT.LSAME(TRANS,'C'))) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
          INFO = 3
      ELSE IF (K.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 7
      ELSE IF (LDB.LT.MAX(1,NROWA)) THEN
          INFO = 9
      ELSE IF (LDC.LT.MAX(1,N)) THEN
          INFO = 12
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZHER2K',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (((ALPHA.EQ.ZERO).OR.
     +    (K.EQ.0)).AND. (BETA.EQ.ONE))) RETURN
*
*     And when  alpha.eq.zero.
*
```

```
      IF (ALPHA.EQ.ZERO) THEN
          IF (UPPER) THEN
              IF (BETA.EQ.DBLE(ZERO)) THEN
                  DO 20 J = 1,N
                      DO 10 I = 1,J
                          C(I,J) = ZERO
   10                 CONTINUE
   20             CONTINUE
              ELSE
                  DO 40 J = 1,N
                      DO 30 I = 1,J - 1
                          C(I,J) = BETA*C(I,J)
   30                 CONTINUE
                      C(J,J) = BETA*DBLE(C(J,J))
   40             CONTINUE
              END IF
          ELSE
              IF (BETA.EQ.DBLE(ZERO)) THEN
                  DO 60 J = 1,N
                      DO 50 I = J,N
                          C(I,J) = ZERO
   50                 CONTINUE
   60             CONTINUE
              ELSE
                  DO 80 J = 1,N
                      C(J,J) = BETA*DBLE(C(J,J))
                      DO 70 I = J + 1,N
                          C(I,J) = BETA*C(I,J)
   70                 CONTINUE
   80             CONTINUE
              END IF
          END IF
          RETURN
      END IF
*
*     Start the operations.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  C := alpha*A*B**H + conjg( alpha )*B*A**H +
*                   C.
*
          IF (UPPER) THEN
              DO 130 J = 1,N
                  IF (BETA.EQ.DBLE(ZERO)) THEN
                      DO 90 I = 1,J
                          C(I,J) = ZERO
   90                 CONTINUE
                  ELSE IF (BETA.NE.ONE) THEN
                      DO 100 I = 1,J - 1
```

```
                        C(I,J) = BETA*C(I,J)
 100                CONTINUE
                    C(J,J) = BETA*DBLE(C(J,J))
                ELSE
                    C(J,J) = DBLE(C(J,J))
                END IF
                DO 120 L = 1,K
                    IF ((A(J,L).NE.ZERO) .OR. (B(J,L).NE.ZERO)) THEN
                        TEMP1 = ALPHA*DCONJG(B(J,L))
                        TEMP2 = DCONJG(ALPHA*A(J,L))
                        DO 110 I = 1,J - 1
                            C(I,J) = C(I,J) + A(I,L)*TEMP1 +
    +                               B(I,L)*TEMP2
 110                    CONTINUE
                        C(J,J) = DBLE(C(J,J)) +
    +                           DBLE(A(J,L)*TEMP1+B(J,L)*TEMP2)
                    END IF
 120            CONTINUE
 130        CONTINUE
        ELSE
            DO 180 J = 1,N
                IF (BETA.EQ.DBLE(ZERO)) THEN
                    DO 140 I = J,N
                        C(I,J) = ZERO
 140                CONTINUE
                ELSE IF (BETA.NE.ONE) THEN
                    DO 150 I = J + 1,N
                        C(I,J) = BETA*C(I,J)
 150                CONTINUE
                    C(J,J) = BETA*DBLE(C(J,J))
                ELSE
                    C(J,J) = DBLE(C(J,J))
                END IF
                DO 170 L = 1,K
                    IF ((A(J,L).NE.ZERO) .OR. (B(J,L).NE.ZERO)) THEN
                        TEMP1 = ALPHA*DCONJG(B(J,L))
                        TEMP2 = DCONJG(ALPHA*A(J,L))
                        DO 160 I = J + 1,N
                            C(I,J) = C(I,J) + A(I,L)*TEMP1 +
    +                               B(I,L)*TEMP2
 160                    CONTINUE
                        C(J,J) = DBLE(C(J,J)) +
    +                           DBLE(A(J,L)*TEMP1+B(J,L)*TEMP2)
                    END IF
 170            CONTINUE
 180        CONTINUE
        END IF
    ELSE
*
*       Form  C := alpha*A**H*B + conjg( alpha )*B**H*A +
```

```
*                  C.
*
          IF (UPPER) THEN
              DO 210 J = 1,N
                  DO 200 I = 1,J
                      TEMP1 = ZERO
                      TEMP2 = ZERO
                      DO 190 L = 1,K
                          TEMP1 = TEMP1 + DCONJG(A(L,I))*B(L,J)
                          TEMP2 = TEMP2 + DCONJG(B(L,I))*A(L,J)
  190                 CONTINUE
                      IF (I.EQ.J) THEN
                          IF (BETA.EQ.DBLE(ZERO)) THEN
                              C(J,J) = DBLE(ALPHA*TEMP1+
     +                                 DCONJG(ALPHA)*TEMP2)
                          ELSE
                              C(J,J) = BETA*DBLE(C(J,J)) +
     +                                 DBLE(ALPHA*TEMP1+
     +                                 DCONJG(ALPHA)*TEMP2)
                          END IF
                      ELSE
                          IF (BETA.EQ.DBLE(ZERO)) THEN
                              C(I,J) = ALPHA*TEMP1 + DCONJG(ALPHA)*TEMP2
                          ELSE
                              C(I,J) = BETA*C(I,J) + ALPHA*TEMP1 +
     +                                 DCONJG(ALPHA)*TEMP2
                          END IF
                      END IF
  200             CONTINUE
  210         CONTINUE
          ELSE
              DO 240 J = 1,N
                  DO 230 I = J,N
                      TEMP1 = ZERO
                      TEMP2 = ZERO
                      DO 220 L = 1,K
                          TEMP1 = TEMP1 + DCONJG(A(L,I))*B(L,J)
                          TEMP2 = TEMP2 + DCONJG(B(L,I))*A(L,J)
  220                 CONTINUE
                      IF (I.EQ.J) THEN
                          IF (BETA.EQ.DBLE(ZERO)) THEN
                              C(J,J) = DBLE(ALPHA*TEMP1+
     +                                 DCONJG(ALPHA)*TEMP2)
                          ELSE
                              C(J,J) = BETA*DBLE(C(J,J)) +
     +                                 DBLE(ALPHA*TEMP1+
     +                                 DCONJG(ALPHA)*TEMP2)
                          END IF
                      ELSE
                          IF (BETA.EQ.DBLE(ZERO)) THEN
```

```
                              C(I,J) = ALPHA*TEMP1 + DCONJG(ALPHA)*TEMP2
                           ELSE
                              C(I,J) = BETA*C(I,J) + ALPHA*TEMP1 +
     +                                 DCONJG(ALPHA)*TEMP2
                           END IF
                        END IF
  230                CONTINUE
  240            CONTINUE
            END IF
        END IF
*
      RETURN
*
*     End of ZHER2K.
*
      END
```

———-

17.3.4 lisp code

— BLAS 3 zher2k —

```
(let* ((one 1.0) (zero (complex 0.0 0.0)))
 (declare (type (double-float 1.0 1.0) one) (type (complex double-float) zero))
  (defun zher2k (uplo trans n k alpha a lda b ldb$ beta c ldc)
    (declare (type (double-float) beta)
             (type (simple-array (complex double-float) (*)) c b a)
             (type (complex double-float) alpha)
             (type fixnum ldc ldb$ lda k n)
             (type character trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (a (complex double-float) a-%data% a-%offset%)
         (b (complex double-float) b-%data% b-%offset%)
         (c (complex double-float) c-%data% c-%offset%))
      (prog ((temp1 #C(0.0 0.0)) (temp2 #C(0.0 0.0)) (i 0) (info 0) (j 0) (l 0)
             (nrowa 0) (upper nil))
        (declare (type (complex double-float) temp1 temp2)
                 (type fixnum i info j l nrowa)
                 (type (member t nil) upper))
        (cond
          ((char-equal trans #\N)
           (setf nrowa n))
          (t
           (setf nrowa k)))
```

```
(setf upper (char-equal uplo #\U))
(setf info 0)
(cond
  ((and (not upper) (not (char-equal uplo #\L)))
   (setf info 1))
  ((and (not (char-equal trans #\N)) (not (char-equal trans #\C)))
   (setf info 2))
  ((< n 0)
   (setf info 3))
  ((< k 0)
   (setf info 4))
  ((< lda (max (the fixnum 1) (the fixnum nrowa)))
   (setf info 7))
  ((< ldb$
      (max (the fixnum 1) (the fixnum nrowa)))
   (setf info 9))
  ((< ldc (max (the fixnum 1) (the fixnum n)))
   (setf info 12)))
(cond
  ((/= info 0)
   (error
    " ** On entry to ~a parameter number ~a had an illegal value~%"
    "ZHER2K" info)
   (go end_label)))
(if (or (= n 0) (and (or (= alpha zero) (= k 0)) (= beta one)))
    (go end_label))
(cond
  ((= alpha zero)
   (cond
     (upper
      (cond
        ((= beta (coerce (realpart zero) 'double-float))
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i j) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                       zero))))))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i
                               (f2cl-lib:int-add j
```

```
                                                    (f2cl-lib:int-sub 1)))
                              nil)
                      (tagbody
                        (setf (f2cl-lib:fref c-%data%
                                             (i j)
                                             ((1 ldc) (1 *))
                                             c-%offset%)
                              (* beta
                                 (f2cl-lib:fref c-%data%
                                                (i j)
                                                ((1 ldc) (1 *))
                                                c-%offset%)))))
                (setf (f2cl-lib:fref c-%data%
                                     (j j)
                                     ((1 ldc) (1 *))
                                     c-%offset%)
                      (coerce
                       (* beta
                          (coerce (realpart
                           (f2cl-lib:fref c-%data%
                                          (j j)
                                          ((1 ldc) (1 *))
                                          c-%offset%)) 'double-float))
                       '(complex double-float))))))))
        (t
         (cond
           ((= beta (coerce (realpart zero) 'double-float))
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)
                          zero))))))
           (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data%
                                     (j j)
                                     ((1 ldc) (1 *))
                                     c-%offset%)
                      (coerce
                       (* beta
                          (coerce (realpart
                           (f2cl-lib:fref c-%data%
```

```
                                          (j j)
                                          ((1 ldc) (1 *))
                                          c-%offset%)) 'double-float))
                        '(complex double-float)))
                (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                               (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)
                          (* beta
                             (f2cl-lib:fref c-%data%
                                            (i j)
                                            ((1 ldc) (1 *))
                                            c-%offset%))))))))))
   (go end_label)))
(cond
  ((char-equal trans #\N)
   (cond
     (upper
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
            ((= beta (coerce (realpart zero) 'double-float))
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i j) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                       zero))))
            ((/= beta one)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i
                               (f2cl-lib:int-add j
                                                 (f2cl-lib:int-sub 1)))
                            nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                       (* beta
                          (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
```

```
                                    c-%offset%)))))
      (setf (f2cl-lib:fref c-%data%
                           (j j)
                           ((1 ldc) (1 *))
                           c-%offset%)
            (coerce
             (* beta
                (coerce (realpart
                 (f2cl-lib:fref c-%data%
                                (j j)
                                ((1 ldc) (1 *))
                                c-%offset%)) 'double-float))
             '(complex double-float))))
    (t
     (setf (f2cl-lib:fref c-%data%
                          (j j)
                          ((1 ldc) (1 *))
                          c-%offset%)
            (coerce
             (coerce (realpart
              (f2cl-lib:fref c-%data%
                             (j j)
                             ((1 ldc) (1 *))
                             c-%offset%)) 'double-float)
             '(complex double-float)))))
  (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                ((> l k) nil)
    (tagbody
      (cond
        ((or (/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
             (/= (f2cl-lib:fref b (j l) ((1 ldb$) (1 *))) zero))
         (setf temp1
                 (* alpha
                    (f2cl-lib:dconjg
                     (f2cl-lib:fref b-%data%
                                    (j l)
                                    ((1 ldb$) (1 *))
                                    b-%offset%))))
         (setf temp2
                 (coerce
                  (f2cl-lib:dconjg
                   (* alpha
                      (f2cl-lib:fref a-%data%
                                     (j l)
                                     ((1 lda) (1 *))
                                     a-%offset%)))
                  '(complex double-float)))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i
                           (f2cl-lib:int-add j
```

```
                                                (f2cl-lib:int-sub
                                                 1)))
                          nil)
                (tagbody
                  (setf (f2cl-lib:fref c-%data%
                                       (i j)
                                       ((1 ldc) (1 *))
                                       c-%offset%)
                        (+
                         (f2cl-lib:fref c-%data%
                                        (i j)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                         (*
                          (f2cl-lib:fref a-%data%
                                         (i l)
                                         ((1 lda) (1 *))
                                         a-%offset%)
                          temp1)
                         (*
                          (f2cl-lib:fref b-%data%
                                         (i l)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)
                          temp2)))))
            (setf (f2cl-lib:fref c-%data%
                                 (j j)
                                 ((1 ldc) (1 *))
                                 c-%offset%)
                  (coerce
                   (+
                    (coerce (realpart
                     (f2cl-lib:fref c-%data%
                                    (j j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)) 'double-float)
                    (coerce (realpart
                     (+
                      (*
                       (f2cl-lib:fref a-%data%
                                      (j l)
                                      ((1 lda) (1 *))
                                      a-%offset%)
                       temp1)
                      (*
                       (f2cl-lib:fref b-%data%
                                      (j l)
                                      ((1 ldb$) (1 *))
                                      b-%offset%)
                       temp2))) 'double-float))
```

```
                        '(complex double-float)))))))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((= beta (coerce (realpart zero) 'double-float))
        (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data%
                                 (i j)
                                 ((1 ldc) (1 *))
                                 c-%offset%)
                  zero))))
       ((/= beta one)
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                       (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data%
                                 (i j)
                                 ((1 ldc) (1 *))
                                 c-%offset%)
                  (* beta
                     (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)))))
        (setf (f2cl-lib:fref c-%data%
                             (j j)
                             ((1 ldc) (1 *))
                             c-%offset%)
              (coerce
               (* beta
                  (coerce (realpart
                   (f2cl-lib:fref c-%data%
                                  (j j)
                                  ((1 ldc) (1 *))
                                  c-%offset%))' double-float))
               '(complex double-float))))
       (t
        (setf (f2cl-lib:fref c-%data%
                             (j j)
                             ((1 ldc) (1 *))
                             c-%offset%)
              (coerce
               (coerce (realpart
                (f2cl-lib:fref c-%data%
                               (j j)
```

```
                                ((1 ldc) (1 *))
                                c-%offset%)) 'double-float)
                '(complex double-float)))))
(f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
              ((> l k) nil)
  (tagbody
    (cond
     ((or (/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
          (/= (f2cl-lib:fref b (j l) ((1 ldb$) (1 *))) zero))
       (setf temp1
               (* alpha
                  (f2cl-lib:dconjg
                   (f2cl-lib:fref b-%data%
                                  (j l)
                                  ((1 ldb$) (1 *))
                                  b-%offset%))))
       (setf temp2
               (coerce
                (f2cl-lib:dconjg
                 (* alpha
                    (f2cl-lib:fref a-%data%
                                   (j l)
                                   ((1 lda) (1 *))
                                   a-%offset%)))
                '(complex double-float)))
       (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                      (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref c-%data%
                                (i j)
                                ((1 ldc) (1 *))
                                c-%offset%)
                 (+
                  (f2cl-lib:fref c-%data%
                                 (i j)
                                 ((1 ldc) (1 *))
                                 c-%offset%)
                  (*
                   (f2cl-lib:fref a-%data%
                                  (i l)
                                  ((1 lda) (1 *))
                                  a-%offset%)
                   temp1)
                  (*
                   (f2cl-lib:fref b-%data%
                                  (i l)
                                  ((1 ldb$) (1 *))
                                  b-%offset%)
                   temp2)))))
```

```
                    (setf (f2cl-lib:fref c-%data%
                                         (j j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)
                          (coerce
                           (+
                            (coerce (realpart
                             (f2cl-lib:fref c-%data%
                                            (j j)
                                            ((1 ldc) (1 *))
                                            c-%offset%)) 'double-float)
                            (coerce (realpart
                             (+
                              (*
                               (f2cl-lib:fref a-%data%
                                              (j l)
                                              ((1 lda) (1 *))
                                              a-%offset%)
                               temp1)
                              (*
                               (f2cl-lib:fref b-%data%
                                              (j l)
                                              ((1 ldb$) (1 *))
                                              b-%offset%)
                               temp2))) 'double-float))
                           '(complex double-float)))))))))))
(t
 (cond
   (upper
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i j) nil)
          (tagbody
            (setf temp1 zero)
            (setf temp2 zero)
            (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                          ((> l k) nil)
              (tagbody
                (setf temp1
                        (+ temp1
                           (*
                            (f2cl-lib:dconjg
                             (f2cl-lib:fref a-%data%
                                            (l i)
                                            ((1 lda) (1 *))
                                            a-%offset%))
                            (f2cl-lib:fref b-%data%
                                           (l j)
```

```
                                    ((1 ldb$) (1 *))
                                    b-%offset%))))
        (setf temp2
                (+ temp2
                   (*
                    (f2cl-lib:dconjg
                     (f2cl-lib:fref b-%data%
                                    (l i)
                                    ((1 ldb$) (1 *))
                                    b-%offset%))
                    (f2cl-lib:fref a-%data%
                                   (l j)
                                   ((1 lda) (1 *))
                                   a-%offset%))))))
(cond
  ((= i j)
   (cond
     ((= beta (coerce (realpart zero) 'double-float))
      (setf (f2cl-lib:fref c-%data%
                           (j j)
                           ((1 ldc) (1 *))
                           c-%offset%)
              (coerce
               (coerce (realpart
                (+ (* alpha temp1)
                   (* (f2cl-lib:dconjg alpha) temp2)))
                   'double-float)
               '(complex double-float))))
     (t
      (setf (f2cl-lib:fref c-%data%
                           (j j)
                           ((1 ldc) (1 *))
                           c-%offset%)
              (coerce
               (+
                (* beta
                   (coerce (realpart
                    (f2cl-lib:fref c-%data%
                                   (j j)
                                   ((1 ldc) (1 *))
                                   c-%offset%))
                       'double-float))
                (coerce (realpart
                 (+ (* alpha temp1)
                    (* (f2cl-lib:dconjg alpha) temp2)))
                 'double-float))
               '(complex double-float))))))
  (t
   (cond
     ((= beta (coerce (realpart zero) 'double-float))
```

```
                    (setf (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)
                            (+ (* alpha temp1)
                               (* (f2cl-lib:dconjg alpha) temp2))))
                  (t
                    (setf (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)
                            (+
                             (* beta
                                (f2cl-lib:fref c-%data%
                                               (i j)
                                               ((1 ldc) (1 *))
                                               c-%offset%))
                             (* alpha temp1)
                             (* (f2cl-lib:dconjg alpha) temp2))))))))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf temp1 zero)
         (setf temp2 zero)
         (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                       ((> l k) nil)
           (tagbody
             (setf temp1
                     (+ temp1
                        (*
                         (f2cl-lib:dconjg
                          (f2cl-lib:fref a-%data%
                                         (l i)
                                         ((1 lda) (1 *))
                                         a-%offset%))
                         (f2cl-lib:fref b-%data%
                                        (l j)
                                        ((1 ldb$) (1 *))
                                        b-%offset%))))
             (setf temp2
                     (+ temp2
                        (*
                         (f2cl-lib:dconjg
                          (f2cl-lib:fref b-%data%
                                         (l i)
                                         ((1 ldb$) (1 *))
```

```
                                    b-%offset%))
                    (f2cl-lib:fref a-%data%
                                   (l j)
                                   ((1 lda) (1 *))
                                   a-%offset%))))))
(cond
  ((= i j)
   (cond
     ((= beta (coerce (realpart zero) 'double-float))
      (setf (f2cl-lib:fref c-%data%
                           (j j)
                           ((1 ldc) (1 *))
                           c-%offset%)
              (coerce
               (coerce (realpart
                (+ (* alpha temp1)
                   (* (f2cl-lib:dconjg alpha) temp2)))
                'double-float)
               '(complex double-float))))
     (t
      (setf (f2cl-lib:fref c-%data%
                           (j j)
                           ((1 ldc) (1 *))
                           c-%offset%)
              (coerce
               (+
                (* beta
                   (coerce (realpart
                    (f2cl-lib:fref c-%data%
                                   (j j)
                                   ((1 ldc) (1 *))
                                   c-%offset%))
                    'double-float))
                (coerce (realpart
                 (+ (* alpha temp1)
                    (* (f2cl-lib:dconjg alpha) temp2)))
                  'double-float))
               '(complex double-float))))))
  (t
   (cond
     ((= beta (coerce (realpart zero) 'double-float))
      (setf (f2cl-lib:fref c-%data%
                           (i j)
                           ((1 ldc) (1 *))
                           c-%offset%)
              (+ (* alpha temp1)
                 (* (f2cl-lib:dconjg alpha) temp2))))
     (t
      (setf (f2cl-lib:fref c-%data%
                           (i j)
```

```
                                                ((1 ldc) (1 *))
                                                c-%offset%)
                                        (+
                                         (* beta
                                            (f2cl-lib:fref c-%data%
                                                           (i j)
                                                           ((1 ldc) (1 *))
                                                           c-%offset%))
                                         (* alpha temp1)
                                         (* (f2cl-lib:dconjg alpha) temp2)
                                           ))))))))))))))
 end_label
        (return (values nil nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

17.4 zherk - hermitian rank k $\alpha AA^H + \beta C$ or $\alpha A^H A + \beta C$

17.4.1 Axiom unit tests

— **zherk.input** —

```
)set break resume
)sys rm -f zherk.output
)spool zherk.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

17.4.2 Axiom help page

— **zherk.help** —

```
======================================================================
zherk examples
======================================================================

======================================================================
```

```
Man Page Details
=====================================================================

 NAME
      ZHERK - perform one of the hermitian rank k operations   C
      := alpha*A*conjg( A' ) + beta*C,

 SYNOPSIS
      SUBROUTINE ZHERK ( UPLO, TRANS, N, K, ALPHA, A, LDA, BETA,
                         C, LDC )

          CHARACTER*1  UPLO, TRANS

          INTEGER      N, K, LDA, LDC

          DOUBLE       PRECISION ALPHA, BETA

          COMPLEX*16   A( LDA, * ), C( LDC, * )

 PURPOSE
      ZHERK  performs one of the hermitian rank k operations

      or

         C := alpha*conjg( A' )*A + beta*C,

      where  alpha and beta  are  real scalars,  C is an  n by n
      hermitian matrix and  A  is an  n by k  matrix in the  first
      case and a  k by n matrix in the second case.

 PARAMETERS
      UPLO   - CHARACTER*1.
             On  entry,   UPLO  specifies  whether  the  upper  or
             lower triangular  part  of the  array  C  is to be
             referenced  as follows:

             UPLO = 'U' or 'u'   Only the  upper triangular part
             of  C is to be referenced.

             UPLO = 'L' or 'l'   Only the  lower triangular part
             of  C is to be referenced.

             Unchanged on exit.

      TRANS  - CHARACTER*1.
             On entry,  TRANS  specifies the operation to be per-
             formed as follows:

             TRANS = 'N' or 'n'   C := alpha*A*conjg( A' ) +
             beta*C.
```

```
            TRANS = 'C' or 'c'   C := alpha*conjg( A' )*A +
            beta*C.

            Unchanged on exit.

      N     - INTEGER.
            On entry,  N specifies the order of the matrix C.  N
            must be at least zero.  Unchanged on exit.

      K     - INTEGER.
            On entry with  TRANS = 'N' or 'n',  K  specifies  the
            number of  columns   of  the   matrix   A,   and  on
            entry   with TRANS = 'C' or 'c',  K  specifies  the
            number of rows of the matrix A.  K must be at least
            zero.  Unchanged on exit.

      ALPHA - DOUBLE PRECISION.
            On entry, ALPHA specifies the scalar alpha.
            Unchanged on exit.

ka is
      A     -
             COMPLEX*16       array of DIMENSION ( LDA, ka ), where
            k  when  TRANS = 'N' or 'n',  and is  n  otherwise.
            Before entry with  TRANS = 'N' or 'n',  the  leading
            n by k part of the array  A  must contain the matrix
            A,  otherwise the leading  k by n  part of the array
            A  must contain  the matrix A.  Unchanged on exit.

      LDA   - INTEGER.
            On entry, LDA specifies the first dimension of A as
            declared in  the  calling  (sub)  program.   When
            TRANS = 'N' or 'n' then  LDA must be at least  max(
            1, n ), otherwise  LDA must be at least  max( 1, k ).
            Unchanged on exit.

      BETA  - DOUBLE PRECISION.
            On entry, BETA specifies the scalar beta.  Unchanged
            on exit.

      C     - COMPLEX*16       array of DIMENSION ( LDC, n ).
            Before entry  with  UPLO = 'U' or 'u',  the leading
            n by n upper triangular part of the array C must con-
            tain the upper triangular part  of the  hermitian
            matrix  and the strictly lower triangular part of C
            is not referenced.  On exit, the upper triangular
            part of the array  C is overwritten by the upper tri-
            angular part of the updated matrix.  Before entry
            with  UPLO = 'L' or 'l',  the leading  n by n lower
```

```
            triangular part of the array C must contain the lower
            triangular part  of the  hermitian matrix  and the
            strictly upper triangular part of C is not refer-
            enced.  On exit, the lower triangular part of the
            array  C is overwritten by the lower triangular part
            of the updated matrix.  Note that the imaginary parts

            of the diagonal elements need not be set,  they are
            assumed to be zero,  and on exit they are set to
            zero.

      LDC    - INTEGER.
             On entry, LDC specifies the first dimension of C as
             declared in  the  calling  (sub)  program.   LDC
             must  be  at  least max( 1, n ).  Unchanged on exit.
```

17.4.3 fortran code

— zherk.f —

```
*  ===================================================================
      SUBROUTINE ZHERK(UPLO,TRANS,N,K,ALPHA,A,LDA,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION ALPHA,BETA
      INTEGER K,LDA,LDC,N
      CHARACTER TRANS,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),C(LDC,*)
*     ..
*
*  ===================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
```

```
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DBLE,DCMPLX,DCONJG,MAX
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP
      DOUBLE PRECISION RTEMP
      INTEGER I,INFO,J,L,NROWA
      LOGICAL UPPER
*     ..
*     .. Parameters ..
      DOUBLE PRECISION ONE,ZERO
      PARAMETER (ONE=1.0D+0,ZERO=0.0D+0)
*     ..
*
*     Test the input parameters.
*
      IF (LSAME(TRANS,'N')) THEN
          NROWA = N
      ELSE
          NROWA = K
      END IF
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 1
      ELSE IF ((.NOT.LSAME(TRANS,'N')) .AND.
     +         (.NOT.LSAME(TRANS,'C'))) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
          INFO = 3
      ELSE IF (K.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 7
      ELSE IF (LDC.LT.MAX(1,N)) THEN
          INFO = 10
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZHERK ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (((ALPHA.EQ.ZERO).OR.
     +    (K.EQ.0)).AND. (BETA.EQ.ONE))) RETURN
*
*     And when  alpha.eq.zero.
```

```
*
      IF (ALPHA.EQ.ZERO) THEN
          IF (UPPER) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 20 J = 1,N
                      DO 10 I = 1,J
                          C(I,J) = ZERO
   10                 CONTINUE
   20             CONTINUE
              ELSE
                  DO 40 J = 1,N
                      DO 30 I = 1,J - 1
                          C(I,J) = BETA*C(I,J)
   30                 CONTINUE
                      C(J,J) = BETA*DBLE(C(J,J))
   40             CONTINUE
              END IF
          ELSE
              IF (BETA.EQ.ZERO) THEN
                  DO 60 J = 1,N
                      DO 50 I = J,N
                          C(I,J) = ZERO
   50                 CONTINUE
   60             CONTINUE
              ELSE
                  DO 80 J = 1,N
                      C(J,J) = BETA*DBLE(C(J,J))
                      DO 70 I = J + 1,N
                          C(I,J) = BETA*C(I,J)
   70                 CONTINUE
   80             CONTINUE
              END IF
          END IF
          RETURN
      END IF
*
*     Start the operations.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  C := alpha*A*A**H + beta*C.
*
          IF (UPPER) THEN
              DO 130 J = 1,N
                  IF (BETA.EQ.ZERO) THEN
                      DO 90 I = 1,J
                          C(I,J) = ZERO
   90                 CONTINUE
                  ELSE IF (BETA.NE.ONE) THEN
                      DO 100 I = 1,J - 1
```

```
                        C(I,J) = BETA*C(I,J)
 100                CONTINUE
                    C(J,J) = BETA*DBLE(C(J,J))
                ELSE
                    C(J,J) = DBLE(C(J,J))
                END IF
                DO 120 L = 1,K
                    IF (A(J,L).NE.DCMPLX(ZERO)) THEN
                        TEMP = ALPHA*DCONJG(A(J,L))
                        DO 110 I = 1,J - 1
                            C(I,J) = C(I,J) + TEMP*A(I,L)
 110                    CONTINUE
                        C(J,J) = DBLE(C(J,J)) + DBLE(TEMP*A(I,L))
                    END IF
 120            CONTINUE
 130        CONTINUE
        ELSE
            DO 180 J = 1,N
                IF (BETA.EQ.ZERO) THEN
                    DO 140 I = J,N
                        C(I,J) = ZERO
 140                CONTINUE
                ELSE IF (BETA.NE.ONE) THEN
                    C(J,J) = BETA*DBLE(C(J,J))
                    DO 150 I = J + 1,N
                        C(I,J) = BETA*C(I,J)
 150                CONTINUE
                ELSE
                    C(J,J) = DBLE(C(J,J))
                END IF
                DO 170 L = 1,K
                    IF (A(J,L).NE.DCMPLX(ZERO)) THEN
                        TEMP = ALPHA*DCONJG(A(J,L))
                        C(J,J) = DBLE(C(J,J)) + DBLE(TEMP*A(J,L))
                        DO 160 I = J + 1,N
                            C(I,J) = C(I,J) + TEMP*A(I,L)
 160                    CONTINUE
                    END IF
 170            CONTINUE
 180        CONTINUE
        END IF
    ELSE
*
*       Form  C := alpha*A**H*A + beta*C.
*
        IF (UPPER) THEN
            DO 220 J = 1,N
                DO 200 I = 1,J - 1
                    TEMP = ZERO
                    DO 190 L = 1,K
```

```
                        TEMP = TEMP + DCONJG(A(L,I))*A(L,J)
 190                CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP
                    ELSE
                        C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                    END IF
 200            CONTINUE
                RTEMP = ZERO
                DO 210 L = 1,K
                    RTEMP = RTEMP + DCONJG(A(L,J))*A(L,J)
 210            CONTINUE
                IF (BETA.EQ.ZERO) THEN
                    C(J,J) = ALPHA*RTEMP
                ELSE
                    C(J,J) = ALPHA*RTEMP + BETA*DBLE(C(J,J))
                END IF
 220        CONTINUE
        ELSE
            DO 260 J = 1,N
                RTEMP = ZERO
                DO 230 L = 1,K
                    RTEMP = RTEMP + DCONJG(A(L,J))*A(L,J)
 230            CONTINUE
                IF (BETA.EQ.ZERO) THEN
                    C(J,J) = ALPHA*RTEMP
                ELSE
                    C(J,J) = ALPHA*RTEMP + BETA*DBLE(C(J,J))
                END IF
                DO 250 I = J + 1,N
                    TEMP = ZERO
                    DO 240 L = 1,K
                        TEMP = TEMP + DCONJG(A(L,I))*A(L,J)
 240                CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP
                    ELSE
                        C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                    END IF
 250            CONTINUE
 260        CONTINUE
        END IF
    END IF
*
    RETURN
*
*   End of ZHERK .
*
    END
```

———-

17.4.4 lisp code

— BLAS 3 zherk —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (double-float 1.0 1.0) one)
           (type (double-float 0.0 0.0) zero))
  (defun zherk (uplo trans n k alpha a lda beta c ldc)
    (declare (type (simple-array (complex double-float) (*)) c a)
             (type (double-float) beta alpha)
             (type fixnum ldc lda k n)
             (type character trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (a (complex double-float) a-%data% a-%offset%)
         (c (complex double-float) c-%data% c-%offset%))
      (prog ((temp #C(0.0 0.0)) (rtemp 0.0) (i 0) (info 0) (j 0) (l 0)
             (nrowa 0) (upper nil))
        (declare (type (complex double-float) temp)
                 (type (double-float) rtemp)
                 (type fixnum i info j l nrowa)
                 (type (member t nil) upper))
        (cond
          ((char-equal trans #\N)
           (setf nrowa n))
          (t
           (setf nrowa k)))
        (setf upper (char-equal uplo #\U))
        (setf info 0)
        (cond
          ((and (not upper) (not (char-equal uplo #\L)))
           (setf info 1))
          ((and (not (char-equal trans #\N)) (not (char-equal trans #\C)))
           (setf info 2))
          ((< n 0)
           (setf info 3))
          ((< k 0)
           (setf info 4))
          ((< lda (max (the fixnum 1) (the fixnum nrowa)))
           (setf info 7))
          ((< ldc (max (the fixnum 1) (the fixnum n)))
           (setf info 10)))
        (cond
          ((/= info 0)
           (error
```

```
      " ** On entry to ~a parameter number ~a had an illegal value~%"
      "ZHERK" info)
     (go end_label)))
  (if (or (= n 0) (and (or (= alpha zero) (= k 0)) (= beta one)))
      (go end_label))
  (cond
    ((= alpha zero)
     (cond
       (upper
        (cond
          ((= beta zero)
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i j) nil)
                 (tagbody
                   (setf (f2cl-lib:fref c-%data%
                                        (i j)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                         (coerce zero '(complex double-float))))))))
          (t
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i
                                 (f2cl-lib:int-add j
                                                   (f2cl-lib:int-sub 1)))
                              nil)
                 (tagbody
                   (setf (f2cl-lib:fref c-%data%
                                        (i j)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                         (* beta
                            (f2cl-lib:fref c-%data%
                                           (i j)
                                           ((1 ldc) (1 *))
                                           c-%offset%)))))
               (setf (f2cl-lib:fref c-%data%
                                    (j j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                     (coerce
                      (* beta
                         (coerce (realpart
                          (f2cl-lib:fref c-%data%
                                         (j j)
```

```
                                          ((1 ldc) (1 *))
                                          c-%offset%)) 'double-float))
                        '(complex double-float))))))))
        (t
         (cond
           ((= beta zero)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)
                          (coerce zero '(complex double-float))))))))
           (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data%
                                     (j j)
                                     ((1 ldc) (1 *))
                                     c-%offset%)
                      (coerce
                       (* beta
                          (coerce (realpart
                           (f2cl-lib:fref c-%data%
                                          (j j)
                                          ((1 ldc) (1 *))
                                          c-%offset%)) 'double-float))
                       '(complex double-float)))
                (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                                 (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)
                          (* beta
                             (f2cl-lib:fref c-%data%
                                            (i j)
                                            ((1 ldc) (1 *))
                                            c-%offset%))))))))))
    (go end_label)))
  (cond
    ((char-equal trans #\N)
     (cond
```

```
(upper
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((= beta zero)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i j) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data%
                                 (i j)
                                 ((1 ldc) (1 *))
                                 c-%offset%)
                  (coerce zero '(complex double-float))))))
       ((/= beta one)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i
                          (f2cl-lib:int-add j
                                            (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data%
                                 (i j)
                                 ((1 ldc) (1 *))
                                 c-%offset%)
                  (* beta
                     (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)))))
        (setf (f2cl-lib:fref c-%data%
                             (j j)
                             ((1 ldc) (1 *))
                             c-%offset%)
              (coerce
               (* beta
                  (coerce (realpart
                   (f2cl-lib:fref c-%data%
                                  (j j)
                                  ((1 ldc) (1 *))
                                  c-%offset%)) 'double-float))
               '(complex double-float))))
       (t
        (setf (f2cl-lib:fref c-%data%
                             (j j)
                             ((1 ldc) (1 *))
                             c-%offset%)
              (coerce
               (coerce (realpart
                (f2cl-lib:fref c-%data%
```

```
                                (j j)
                                ((1 ldc) (1 *))
                                c-%offset%)) 'double-float)
                 '(complex double-float)))))
        (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                      ((> l k) nil)
          (tagbody
            (cond
              ((/= (f2cl-lib:fref a (j l) ((1 lda) (1 *)))
                   (coerce (complex zero) '(complex double-float)))
               (setf temp
                       (coerce
                        (* alpha
                           (f2cl-lib:dconjg
                            (f2cl-lib:fref a-%data%
                                           (j l)
                                           ((1 lda) (1 *))
                                           a-%offset%)))
                        '(complex double-float)))
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i
                                 (f2cl-lib:int-add j
                                                   (f2cl-lib:int-sub
                                                    1)))
                              nil)
                 (tagbody
                   (setf (f2cl-lib:fref c-%data%
                                        (i j)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                         (+
                          (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)
                          (* temp
                             (f2cl-lib:fref a-%data%
                                            (i l)
                                            ((1 lda) (1 *))
                                            a-%offset%))))))
               (setf (f2cl-lib:fref c-%data%
                                    (j j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                       (coerce
                        (+
                         (coerce (realpart
                          (f2cl-lib:fref c-%data%
                                         (j j)
                                         ((1 ldc) (1 *))
```

```
                                        c-%offset%)) 'double-float)
                        (coerce (realpart
                         (* temp
                            (f2cl-lib:fref a-%data%
                                           (i l)
                                           ((1 lda) (1 *))
                                           a-%offset%)))
                          'double-float))
                       '(complex double-float)))))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((= beta zero)
        (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data%
                                 (i j)
                                 ((1 ldc) (1 *))
                                 c-%offset%)
                  (coerce zero '(complex double-float))))))
       ((/= beta one)
        (setf (f2cl-lib:fref c-%data%
                             (j j)
                             ((1 ldc) (1 *))
                             c-%offset%)
              (coerce
               (* beta
                  (coerce (realpart
                   (f2cl-lib:fref c-%data%
                                  (j j)
                                  ((1 ldc) (1 *))
                                  c-%offset%)) 'double-float))
               '(complex double-float)))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                         (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data%
                                 (i j)
                                 ((1 ldc) (1 *))
                                 c-%offset%)
                  (* beta
                     (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%))))))
       (t
```

```
    (setf (f2cl-lib:fref c-%data%
                         (j j)
                         ((1 ldc) (1 *))
                         c-%offset%)
          (coerce
           (coerce (realpart
            (f2cl-lib:fref c-%data%
                           (j j)
                           ((1 ldc) (1 *))
                           c-%offset%)) 'double-float)
           '(complex double-float)))))
(f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
              ((> l k) nil)
  (tagbody
    (cond
      ((/= (f2cl-lib:fref a (j l) ((1 lda) (1 *)))
           (coerce (complex zero) '(complex double-float)))
       (setf temp
               (coerce
                (* alpha
                   (f2cl-lib:dconjg
                    (f2cl-lib:fref a-%data%
                                   (j l)
                                   ((1 lda) (1 *))
                                   a-%offset%)))
                '(complex double-float)))
       (setf (f2cl-lib:fref c-%data%
                            (j j)
                            ((1 ldc) (1 *))
                            c-%offset%)
               (coerce
                (+
                 (coerce (realpart
                  (f2cl-lib:fref c-%data%
                                 (j j)
                                 ((1 ldc) (1 *))
                                 c-%offset%)) 'double-float)
                 (coerce (realpart
                  (* temp
                     (f2cl-lib:fref a-%data%
                                    (j l)
                                    ((1 lda) (1 *))
                                    a-%offset%)))
                     'double-float))
                '(complex double-float)))
       (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                      (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref c-%data%
```

```
                                          (i j)
                                          ((1 ldc) (1 *))
                                          c-%offset%)
                                 (+
                                  (f2cl-lib:fref c-%data%
                                                 (i j)
                                                 ((1 ldc) (1 *))
                                                 c-%offset%)
                                  (* temp
                                     (f2cl-lib:fref a-%data%
                                                    (i l)
                                                    ((1 lda) (1 *))
                                                    a-%offset%)))))))))))))))
            (t
             (cond
               (upper
                (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                              ((> j n) nil)
                  (tagbody
                    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                  ((> i
                                      (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                                   nil)
                      (tagbody
                        (setf temp (coerce zero '(complex double-float)))
                        (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                                      ((> l k) nil)
                          (tagbody
                            (setf temp
                                  (+ temp
                                     (*
                                      (f2cl-lib:dconjg
                                       (f2cl-lib:fref a-%data%
                                                      (l i)
                                                      ((1 lda) (1 *))
                                                      a-%offset%))
                                      (f2cl-lib:fref a-%data%
                                                     (l j)
                                                     ((1 lda) (1 *))
                                                     a-%offset%))))))
                        (cond
                          ((= beta zero)
                           (setf (f2cl-lib:fref c-%data%
                                                (i j)
                                                ((1 ldc) (1 *))
                                                c-%offset%)
                                 (* alpha temp)))
                          (t
                           (setf (f2cl-lib:fref c-%data%
                                                (i j)
```

```
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                         (+ (* alpha temp)
                            (* beta
                               (f2cl-lib:fref c-%data%
                                              (i j)
                                              ((1 ldc) (1 *))
                                              c-%offset%)))))))
        (setf rtemp zero)
        (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                      ((> l k) nil)
          (tagbody
            (setf rtemp
                    (coerce
                     (realpart
                      (+ rtemp
                         (*
                          (f2cl-lib:dconjg
                           (f2cl-lib:fref a-%data%
                                          (l j)
                                          ((1 lda) (1 *))
                                          a-%offset%))
                          (f2cl-lib:fref a-%data%
                                         (l j)
                                         ((1 lda) (1 *))
                                         a-%offset%))))
                      'double-float))))
        (cond
          ((= beta zero)
           (setf (f2cl-lib:fref c-%data%
                                (j j)
                                ((1 ldc) (1 *))
                                c-%offset%)
                 (coerce (* alpha rtemp) '(complex double-float))))
          (t
           (setf (f2cl-lib:fref c-%data%
                                (j j)
                                ((1 ldc) (1 *))
                                c-%offset%)
                 (coerce
                  (+ (* alpha rtemp)
                     (* beta
                        (coerce (realpart
                         (f2cl-lib:fref c-%data%
                                        (j j)
                                        ((1 ldc) (1 *))
                                        c-%offset%))
                          'double-float)))
                  '(complex double-float))))))))
(t
```

```
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf rtemp zero)
    (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                  ((> l k) nil)
      (tagbody
        (setf rtemp
                (coerce
                 (realpart
                  (+ rtemp
                     (*
                      (f2cl-lib:dconjg
                       (f2cl-lib:fref a-%data%
                                      (l j)
                                      ((1 lda) (1 *))
                                      a-%offset%))
                      (f2cl-lib:fref a-%data%
                                     (l j)
                                     ((1 lda) (1 *))
                                     a-%offset%))))
                  'double-float))))
    (cond
      ((= beta zero)
       (setf (f2cl-lib:fref c-%data%
                            (j j)
                            ((1 ldc) (1 *))
                            c-%offset%)
             (coerce (* alpha rtemp) '(complex double-float))))
      (t
       (setf (f2cl-lib:fref c-%data%
                            (j j)
                            ((1 ldc) (1 *))
                            c-%offset%)
             (coerce
              (+ (* alpha rtemp)
                 (* beta
                    (coerce (realpart
                     (f2cl-lib:fref c-%data%
                                    (j j)
                                    ((1 ldc) (1 *))
                                    c-%offset%))
                      'double-float)))
              '(complex double-float)))))
    (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                     (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf temp (coerce zero '(complex double-float)))
        (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
```

```
                                ((> l k) nil)
                        (tagbody
                          (setf temp
                                  (+ temp
                                     (*
                                      (f2cl-lib:dconjg
                                       (f2cl-lib:fref a-%data%
                                                      (l i)
                                                      ((1 lda) (1 *))
                                                      a-%offset%))
                                      (f2cl-lib:fref a-%data%
                                                     (l j)
                                                     ((1 lda) (1 *))
                                                     a-%offset%))))))
                      (cond
                        ((= beta zero)
                         (setf (f2cl-lib:fref c-%data%
                                              (i j)
                                              ((1 ldc) (1 *))
                                              c-%offset%)
                               (* alpha temp)))
                        (t
                         (setf (f2cl-lib:fref c-%data%
                                              (i j)
                                              ((1 ldc) (1 *))
                                              c-%offset%)
                               (+ (* alpha temp)
                                  (* beta
                                     (f2cl-lib:fref c-%data%
                                                    (i j)
                                                    ((1 ldc) (1 *))
                                                    c-%offset%))))))))))))))
 end_label
        (return (values nil nil nil nil nil nil nil nil nil nil))))))
```

———-

17.5 zsymm - symmetric $\alpha AB + \beta C$ or $\alpha BA + \beta C$

17.5.1 Axiom unit tests

— **zsymm.input** —

```
)set break resume
)sys rm -f zsymm.output
)spool zsymm.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

17.5.2 Axiom help page

— **zsymm.help** —

```
==================================================================
zsymm examples
==================================================================

==================================================================
Man Page Details
==================================================================

 NAME
      ZSYMM - perform one of the matrix-matrix operations   C :=
      alpha*A*B + beta*C,

 SYNOPSIS
      SUBROUTINE ZSYMM ( SIDE, UPLO, M, N, ALPHA, A, LDA, B, LDB,
                         BETA, C, LDC )

          CHARACTER*1  SIDE, UPLO

          INTEGER      M, N, LDA, LDB, LDC

          COMPLEX*16   ALPHA, BETA

          COMPLEX*16   A( LDA, * ), B( LDB, * ), C( LDC, * )

 PURPOSE
      ZSYMM  performs one of the matrix-matrix operations

      or

         C := alpha*B*A + beta*C,

      where  alpha and beta are scalars, A is a symmetric matrix
      and  B and C are m by n matrices.

 PARAMETERS
```

```
     SIDE   - CHARACTER*1.
            On entry,  SIDE  specifies whether  the  symmetric
            matrix  A appears on the  left or right  in the
            operation as follows:

            SIDE = 'L' or 'l'   C := alpha*A*B + beta*C,

            SIDE = 'R' or 'r'   C := alpha*B*A + beta*C,

            Unchanged on exit.

     UPLO   - CHARACTER*1.
            On  entry,   UPLO  specifies  whether  the  upper  or
            lower triangular  part  of  the  symmetric  matrix
            A  is  to  be referenced as follows:

            UPLO = 'U' or 'u'   Only the upper triangular part of
            the symmetric matrix is to be referenced.

            UPLO = 'L' or 'l'   Only the lower triangular part of
            the symmetric matrix is to be referenced.

            Unchanged on exit.

     M      - INTEGER.
            On entry,  M  specifies the number of rows of the
            matrix  C.  M  must be at least zero.  Unchanged on
            exit.

     N      - INTEGER.
            On entry, N specifies the number of columns of the
            matrix C.  N  must be at least zero.  Unchanged on
            exit.

     ALPHA  - COMPLEX*16      .
            On entry, ALPHA specifies the scalar alpha.
            Unchanged on exit.

 ka is
     A      -
             COMPLEX*16       array of DIMENSION ( LDA, ka ), where
            m  when  SIDE = 'L' or 'l'  and is n  otherwise.
            Before entry  with  SIDE = 'L' or 'l',  the  m by m
            part of the array  A  must contain the  symmetric
            matrix,  such that when  UPLO = 'U' or 'u', the lead-
            ing m by m upper triangular part of the array  A
            must contain the upper triangular part of the  sym-
            metric matrix and the  strictly  lower triangular
            part of  A  is not referenced,  and when  UPLO = 'L'
            or 'l', the leading  m by m  lower triangular part
```

```
        of the  array  A must  contain  the  lower triangular
        part  of the  symmetric matrix and the  strictly
        upper triangular part of  A  is not referenced.
        Before entry  with  SIDE = 'R' or 'r',  the  n by n
        part of the array  A  must contain the  symmetric
        matrix,  such that when  UPLO = 'U' or 'u', the lead-
        ing n by n upper triangular part of the array  A
        must contain the upper triangular part of the  sym-
        metric matrix and the  strictly  lower triangular
        part of  A  is not referenced,  and when  UPLO = 'L'
        or 'l', the leading  n by n  lower triangular part
        of the  array  A must  contain  the  lower triangular
        part  of the  symmetric matrix and the  strictly
        upper triangular part of  A  is not referenced.
        Unchanged on exit.

LDA     - INTEGER.
        On entry, LDA specifies the first dimension of A as
        declared in the  calling (sub) program. When  SIDE =
        'L' or 'l'  then LDA must be at least  max( 1, m ),
        otherwise  LDA must be at least max( 1, n ).
        Unchanged on exit.

B       - COMPLEX*16       array of DIMENSION ( LDB, n ).
        Before entry, the leading  m by n part of the array
        B  must contain the matrix B.  Unchanged on exit.

LDB     - INTEGER.
        On entry, LDB specifies the first dimension of B as
        declared in  the  calling  (sub)  program.   LDB
        must  be  at  least max( 1, m ).  Unchanged on exit.

BETA    - COMPLEX*16      .
        On entry,  BETA  specifies the scalar  beta.  When
        BETA  is supplied as zero then C need not be set on
        input.  Unchanged on exit.

C       - COMPLEX*16       array of DIMENSION ( LDC, n ).
        Before entry, the leading  m by n  part of the array
        C must contain the matrix  C,  except when  beta  is
        zero, in which case C need not be set on entry.  On
        exit, the array  C  is overwritten by the  m by n
        updated matrix.

LDC     - INTEGER.
        On entry, LDC specifies the first dimension of C as
        declared in  the  calling  (sub)  program.   LDC
        must  be  at  least max( 1, m ).  Unchanged on exit.
```

———-

17.5.3 fortran code

— zsymm.f —

```
*  ====================================================================
      SUBROUTINE ZSYMM(SIDE,UPLO,M,N,ALPHA,A,LDA,B,LDB,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX*16 ALPHA,BETA
      INTEGER LDA,LDB,LDC,M,N
      CHARACTER SIDE,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),B(LDB,*),C(LDC,*)
*     ..
*
*  ====================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP1,TEMP2
      INTEGER I,INFO,J,K,NROWA
      LOGICAL UPPER
*     ..
*     .. Parameters ..
      COMPLEX*16 ONE
      PARAMETER (ONE= (1.0D+0,0.0D+0))
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*
*     Set NROWA as the number of rows of A.
```

```
*
      IF (LSAME(SIDE,'L')) THEN
          NROWA = M
      ELSE
          NROWA = N
      END IF
      UPPER = LSAME(UPLO,'U')
*
*     Test the input parameters.
*
      INFO = 0
      IF ((.NOT.LSAME(SIDE,'L')) .AND. (.NOT.LSAME(SIDE,'R'))) THEN
          INFO = 1
      ELSE IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 2
      ELSE IF (M.LT.0) THEN
          INFO = 3
      ELSE IF (N.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 7
      ELSE IF (LDB.LT.MAX(1,M)) THEN
          INFO = 9
      ELSE IF (LDC.LT.MAX(1,M)) THEN
          INFO = 12
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZSYMM ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((M.EQ.0) .OR. (N.EQ.0) .OR.
     +    ((ALPHA.EQ.ZERO).AND. (BETA.EQ.ONE))) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
          IF (BETA.EQ.ZERO) THEN
              DO 20 J = 1,N
                  DO 10 I = 1,M
                      C(I,J) = ZERO
   10             CONTINUE
   20         CONTINUE
          ELSE
              DO 40 J = 1,N
                  DO 30 I = 1,M
                      C(I,J) = BETA*C(I,J)
   30             CONTINUE
```

```
   40         CONTINUE
         END IF
         RETURN
      END IF
*
*     Start the operations.
*
      IF (LSAME(SIDE,'L')) THEN
*
*        Form  C := alpha*A*B + beta*C.
*
          IF (UPPER) THEN
              DO 70 J = 1,N
                  DO 60 I = 1,M
                      TEMP1 = ALPHA*B(I,J)
                      TEMP2 = ZERO
                      DO 50 K = 1,I - 1
                          C(K,J) = C(K,J) + TEMP1*A(K,I)
                          TEMP2 = TEMP2 + B(K,J)*A(K,I)
   50                 CONTINUE
                      IF (BETA.EQ.ZERO) THEN
                          C(I,J) = TEMP1*A(I,I) + ALPHA*TEMP2
                      ELSE
                          C(I,J) = BETA*C(I,J) + TEMP1*A(I,I) +
     +                             ALPHA*TEMP2
                      END IF
   60             CONTINUE
   70         CONTINUE
          ELSE
              DO 100 J = 1,N
                  DO 90 I = M,1,-1
                      TEMP1 = ALPHA*B(I,J)
                      TEMP2 = ZERO
                      DO 80 K = I + 1,M
                          C(K,J) = C(K,J) + TEMP1*A(K,I)
                          TEMP2 = TEMP2 + B(K,J)*A(K,I)
   80                 CONTINUE
                      IF (BETA.EQ.ZERO) THEN
                          C(I,J) = TEMP1*A(I,I) + ALPHA*TEMP2
                      ELSE
                          C(I,J) = BETA*C(I,J) + TEMP1*A(I,I) +
     +                             ALPHA*TEMP2
                      END IF
   90             CONTINUE
  100         CONTINUE
          END IF
      ELSE
*
*        Form  C := alpha*B*A + beta*C.
*
```

```
         DO 170 J = 1,N
             TEMP1 = ALPHA*A(J,J)
             IF (BETA.EQ.ZERO) THEN
                 DO 110 I = 1,M
                     C(I,J) = TEMP1*B(I,J)
 110             CONTINUE
             ELSE
                 DO 120 I = 1,M
                     C(I,J) = BETA*C(I,J) + TEMP1*B(I,J)
 120             CONTINUE
             END IF
             DO 140 K = 1,J - 1
                 IF (UPPER) THEN
                     TEMP1 = ALPHA*A(K,J)
                 ELSE
                     TEMP1 = ALPHA*A(J,K)
                 END IF
                 DO 130 I = 1,M
                     C(I,J) = C(I,J) + TEMP1*B(I,K)
 130             CONTINUE
 140         CONTINUE
             DO 160 K = J + 1,N
                 IF (UPPER) THEN
                     TEMP1 = ALPHA*A(J,K)
                 ELSE
                     TEMP1 = ALPHA*A(K,J)
                 END IF
                 DO 150 I = 1,M
                     C(I,J) = C(I,J) + TEMP1*B(I,K)
 150             CONTINUE
 160         CONTINUE
 170     CONTINUE
     END IF
*
     RETURN
*
*    End of ZSYMM .
*
     END
```

———-

17.5.4 lisp code

— BLAS 3 zsymm —

```
(let* ((one (complex 1.0 0.0)) (zero (complex 0.0 0.0)))
```

```
(declare (type (complex double-float) one) (type (complex double-float) zero))
 (defun zsymm (side uplo m n alpha a lda b ldb$ beta c ldc)
   (declare (type (simple-array (complex double-float) (*)) c b a)
            (type (complex double-float) beta alpha)
            (type fixnum ldc ldb$ lda n m)
            (type character uplo side))
   (f2cl-lib:with-multi-array-data
       ((side character side-%data% side-%offset%)
        (uplo character uplo-%data% uplo-%offset%)
        (a (complex double-float) a-%data% a-%offset%)
        (b (complex double-float) b-%data% b-%offset%)
        (c (complex double-float) c-%data% c-%offset%))
     (prog ((temp1 #C(0.0 0.0)) (temp2 #C(0.0 0.0)) (i 0) (info 0) (j 0) (k 0)
            (nrowa 0) (upper nil))
       (declare (type (complex double-float) temp1 temp2)
                (type fixnum i info j k nrowa)
                (type (member t nil) upper))
       (cond
         ((char-equal side #\L)
          (setf nrowa m))
         (t
          (setf nrowa n)))
       (setf upper (char-equal uplo #\U))
       (setf info 0)
       (cond
         ((and (not (char-equal side #\L)) (not (char-equal side #\R)))
          (setf info 1))
         ((and (not upper) (not (char-equal uplo #\L)))
          (setf info 2))
         ((< m 0)
          (setf info 3))
         ((< n 0)
          (setf info 4))
         ((< lda (max (the fixnum 1) (the fixnum nrowa)))
          (setf info 7))
         ((< ldb$ (max (the fixnum 1) (the fixnum m)))
          (setf info 9))
         ((< ldc (max (the fixnum 1) (the fixnum m)))
          (setf info 12)))
       (cond
         ((/= info 0)
          (error
           " ** On entry to ~a parameter number ~a had an illegal value~%"
           "ZSYMM" info)
          (go end_label)))
       (if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
           (go end_label))
       (cond
         ((= alpha zero)
          (cond
```

```
      ((= beta zero)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                     zero))))))
      (t
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
                     (* beta
                        (f2cl-lib:fref c-%data%
                                       (i j)
                                       ((1 ldc) (1 *))
                                       c-%offset%)))))))))
   (go end_label)))
(cond
  ((char-equal side #\L)
   (cond
     (upper
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf temp1
                      (* alpha
                         (f2cl-lib:fref b-%data%
                                        (i j)
                                        ((1 ldb$) (1 *))
                                        b-%offset%)))
              (setf temp2 zero)
              (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                            ((> k
                                (f2cl-lib:int-add i
                                                  (f2cl-lib:int-sub
```

```
                                        1)))
                  nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data%
                             (k j)
                             ((1 ldc) (1 *))
                             c-%offset%)
              (+
               (f2cl-lib:fref c-%data%
                              (k j)
                              ((1 ldc) (1 *))
                              c-%offset%)
               (* temp1
                  (f2cl-lib:fref a-%data%
                                 (k i)
                                 ((1 lda) (1 *))
                                 a-%offset%))))
        (setf temp2
              (+ temp2
                 (*
                  (f2cl-lib:fref b-%data%
                                 (k j)
                                 ((1 ldb$) (1 *))
                                 b-%offset%)
                  (f2cl-lib:fref a-%data%
                                 (k i)
                                 ((1 lda) (1 *))
                                 a-%offset%))))))
    (cond
      ((= beta zero)
       (setf (f2cl-lib:fref c-%data%
                            (i j)
                            ((1 ldc) (1 *))
                            c-%offset%)
             (+
              (* temp1
                 (f2cl-lib:fref a-%data%
                                (i i)
                                ((1 lda) (1 *))
                                a-%offset%))
              (* alpha temp2))))
      (t
       (setf (f2cl-lib:fref c-%data%
                            (i j)
                            ((1 ldc) (1 *))
                            c-%offset%)
             (+
              (* beta
                 (f2cl-lib:fref c-%data%
                                (i j)
```

```
                                        ((1 ldc) (1 *))
                                        c-%offset%))
                        (* temp1
                           (f2cl-lib:fref a-%data%
                                        (i i)
                                        ((1 lda) (1 *))
                                        a-%offset%))
                        (* alpha temp2)))))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                   ((> i 1) nil)
       (tagbody
         (setf temp1
                 (* alpha
                    (f2cl-lib:fref b-%data%
                                   (i j)
                                   ((1 ldb$) (1 *))
                                   b-%offset%)))
         (setf temp2 zero)
         (f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
                          (f2cl-lib:int-add k 1))
                        ((> k m) nil)
           (tagbody
             (setf (f2cl-lib:fref c-%data%
                                  (k j)
                                  ((1 ldc) (1 *))
                                  c-%offset%)
                     (+
                      (f2cl-lib:fref c-%data%
                                     (k j)
                                     ((1 ldc) (1 *))
                                     c-%offset%)
                      (* temp1
                         (f2cl-lib:fref a-%data%
                                        (k i)
                                        ((1 lda) (1 *))
                                        a-%offset%))))
             (setf temp2
                     (+ temp2
                        (*
                         (f2cl-lib:fref b-%data%
                                        (k j)
                                        ((1 ldb$) (1 *))
                                        b-%offset%)
                         (f2cl-lib:fref a-%data%
                                        (k i)
                                        ((1 lda) (1 *))
```

```
                                                a-%offset%))))))
                    (cond
                      ((= beta zero)
                       (setf (f2cl-lib:fref c-%data%
                                            (i j)
                                            ((1 ldc) (1 *))
                                            c-%offset%)
                             (+
                              (* temp1
                                 (f2cl-lib:fref a-%data%
                                                (i i)
                                                ((1 lda) (1 *))
                                                a-%offset%))
                              (* alpha temp2))))
                      (t
                       (setf (f2cl-lib:fref c-%data%
                                            (i j)
                                            ((1 ldc) (1 *))
                                            c-%offset%)
                             (+
                              (* beta
                                 (f2cl-lib:fref c-%data%
                                                (i j)
                                                ((1 ldc) (1 *))
                                                c-%offset%))
                              (* temp1
                                 (f2cl-lib:fref a-%data%
                                                (i i)
                                                ((1 lda) (1 *))
                                                a-%offset%))
                              (* alpha temp2))))))))))
            (t
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (setf temp1
                         (* alpha
                            (f2cl-lib:fref a-%data%
                                           (j j)
                                           ((1 lda) (1 *))
                                           a-%offset%)))
                 (cond
                   ((= beta zero)
                    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                  ((> i m) nil)
                      (tagbody
                        (setf (f2cl-lib:fref c-%data%
                                             (i j)
                                             ((1 ldc) (1 *))
                                             c-%offset%)
```

```
                    (* temp1
                       (f2cl-lib:fref b-%data%
                                      (i j)
                                      ((1 ldb$) (1 *))
                                      b-%offset%))))))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i m) nil)
     (tagbody
       (setf (f2cl-lib:fref c-%data%
                            (i j)
                            ((1 ldc) (1 *))
                            c-%offset%)
                    (+
                     (* beta
                        (f2cl-lib:fref c-%data%
                                       (i j)
                                       ((1 ldc) (1 *))
                                       c-%offset%))
                     (* temp1
                        (f2cl-lib:fref b-%data%
                                       (i j)
                                       ((1 ldb$) (1 *))
                                       b-%offset%))))))))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (cond
      (upper
       (setf temp1
                    (* alpha
                       (f2cl-lib:fref a-%data%
                                      (k j)
                                      ((1 lda) (1 *))
                                      a-%offset%))))
      (t
       (setf temp1
                    (* alpha
                       (f2cl-lib:fref a-%data%
                                      (j k)
                                      ((1 lda) (1 *))
                                      a-%offset%)))))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data%
                             (i j)
                             ((1 ldc) (1 *))
                             c-%offset%)
```

```
                              (+
                               (f2cl-lib:fref c-%data%
                                              (i j)
                                              ((1 ldc) (1 *))
                                              c-%offset%)
                               (* temp1
                                  (f2cl-lib:fref b-%data%
                                                 (i k)
                                                 ((1 ldb$) (1 *))
                                                 b-%offset%))))))))
                (f2cl-lib:fdo (k (f2cl-lib:int-add j 1) (f2cl-lib:int-add k 1))
                              ((> k n) nil)
                  (tagbody
                    (cond
                      (upper
                       (setf temp1
                               (* alpha
                                  (f2cl-lib:fref a-%data%
                                                 (j k)
                                                 ((1 lda) (1 *))
                                                 a-%offset%))))
                      (t
                       (setf temp1
                               (* alpha
                                  (f2cl-lib:fref a-%data%
                                                 (k j)
                                                 ((1 lda) (1 *))
                                                 a-%offset%)))))
                    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                  ((> i m) nil)
                      (tagbody
                        (setf (f2cl-lib:fref c-%data%
                                             (i j)
                                             ((1 ldc) (1 *))
                                             c-%offset%)
                              (+
                               (f2cl-lib:fref c-%data%
                                              (i j)
                                              ((1 ldc) (1 *))
                                              c-%offset%)
                               (* temp1
                                  (f2cl-lib:fref b-%data%
                                                 (i k)
                                                 ((1 ldb$) (1 *))
                                                 b-%offset%))))))))))))
end_label
       (return (values nil nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

17.6 zsyr2k - symmetric rank 2k $\alpha AB^T + \alpha BA^T + \beta C$ or $\alpha A^TB + \alpha B^TA + \beta C$

17.6.1 Axiom unit tests

— **zsyr2k.input** —

```
)set break resume
)sys rm -f zsyr2k.output
)spool zsyr2k.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

17.6.2 Axiom help page

— **zsyr2k.help** —

```
======================================================================
zsyr2k examples
======================================================================

======================================================================
Man Page Details
======================================================================

 NAME
      ZSYR2K - perform one of the symmetric rank 2k operations   C
      := alpha*A*B' + alpha*B*A' + beta*C,

 SYNOPSIS
      SUBROUTINE ZSYR2K( UPLO, TRANS, N, K, ALPHA, A, LDA, B, LDB,
                         BETA, C, LDC )

          CHARACTER*1    UPLO, TRANS

          INTEGER        N, K, LDA, LDB, LDC

          COMPLEX*16     ALPHA, BETA
```

```
         COMPLEX*16     A( LDA, * ), B( LDB, * ), C( LDC, * )

PURPOSE
     ZSYR2K  performs one of the symmetric rank 2k operations

     or

        C := alpha*A'*B + alpha*B'*A + beta*C,

     where  alpha and beta  are scalars,  C is an  n by n sym-
     metric matrix and  A and B  are  n by k  matrices  in the
     first  case  and  k by n matrices in the second case.

PARAMETERS
     UPLO   - CHARACTER*1.
            On  entry,  UPLO  specifies  whether  the  upper  or
            lower triangular  part  of the  array  C  is to be
            referenced  as follows:

            UPLO = 'U' or 'u'   Only the  upper triangular part
            of  C is to be referenced.

            UPLO = 'L' or 'l'   Only the  lower triangular part
            of  C is to be referenced.

            Unchanged on exit.

     TRANS  - CHARACTER*1.
            On entry,  TRANS  specifies the operation to be per-
            formed as follows:

            TRANS = 'N' or 'n'    C := alpha*A*B' + alpha*B*A' +
            beta*C.

            TRANS = 'T' or 't'    C := alpha*A'*B + alpha*B'*A +
            beta*C.

            Unchanged on exit.

     N      - INTEGER.
            On entry,  N specifies the order of the matrix C.  N
            must be at least zero.  Unchanged on exit.

     K      - INTEGER.
            On entry with  TRANS = 'N' or 'n',  K  specifies  the
            number of  columns  of the  matrices  A and B,  and
            on  entry  with TRANS = 'T' or 't',  K  specifies
            the number of rows of the matrices  A and B.  K must
            be at least zero.  Unchanged on exit.
```

```
     ALPHA  - COMPLEX*16      .
            On entry, ALPHA specifies the scalar alpha.
            Unchanged on exit.

ka is
     A      -
             COMPLEX*16       array of DIMENSION ( LDA, ka ), where
            k  when  TRANS = 'N' or 'n',  and is  n  otherwise.
            Before entry with  TRANS = 'N' or 'n',  the  leading
            n by k part of the array  A  must contain the matrix
            A,  otherwise the leading  k by n  part of the array
            A  must contain  the matrix A.  Unchanged on exit.

     LDA    - INTEGER.
            On entry, LDA specifies the first dimension of A as
            declared in  the  calling  (sub)  program.   When
            TRANS = 'N' or 'n' then  LDA must be at least  max(
            1, n ), otherwise  LDA must be at least  max( 1, k ).
            Unchanged on exit.

kb is
     B      -
             COMPLEX*16       array of DIMENSION ( LDB, kb ), where
            k  when  TRANS = 'N' or 'n',  and is  n  otherwise.
            Before entry with  TRANS = 'N' or 'n',  the  leading
            n by k part of the array  B  must contain the matrix
            B,  otherwise the leading  k by n  part of the array
            B  must contain  the matrix B.  Unchanged on exit.

     LDB    - INTEGER.
            On entry, LDB specifies the first dimension of B as
            declared in  the  calling  (sub)  program.   When
            TRANS = 'N' or 'n' then  LDB must be at least  max(
            1, n ), otherwise  LDB must be at least  max( 1, k ).
            Unchanged on exit.

     BETA   - COMPLEX*16      .
            On entry, BETA specifies the scalar beta.  Unchanged
            on exit.

     C      - COMPLEX*16       array of DIMENSION ( LDC, n ).
            Before entry  with  UPLO = 'U' or 'u',  the leading
            n by n upper triangular part of the array C must con-
            tain the upper triangular part  of the  symmetric
            matrix  and the strictly lower triangular part of C
            is not referenced.  On exit, the upper triangular
            part of the array  C is overwritten by the upper tri-
            angular part of the updated matrix.  Before entry
            with  UPLO = 'L' or 'l',  the leading  n by n lower
            triangular part of the array C must contain the lower
```

```
            triangular part  of the  symmetric matrix  and the
            strictly upper triangular part of C is not refer-
            enced.  On exit, the lower triangular part of the
            array  C is overwritten by the lower triangular part
            of the updated matrix.

      LDC    - INTEGER.
             On entry, LDC specifies the first dimension of C as
             declared in  the  calling  (sub)  program.   LDC
             must  be  at  least max( 1, n ).  Unchanged on exit.
```

———-

17.6.3 fortran code

— zsyr2k.f —

```
*  =====================================================================
      SUBROUTINE ZSYR2K(UPLO,TRANS,N,K,ALPHA,A,LDA,B,LDB,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX*16 ALPHA,BETA
      INTEGER K,LDA,LDB,LDC,N
      CHARACTER TRANS,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),B(LDB,*),C(LDC,*)
*     ..
*
*  =====================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*     .. Local Scalars ..
```

```
      COMPLEX*16 TEMP1,TEMP2
      INTEGER I,INFO,J,L,NROWA
      LOGICAL UPPER
*     ..
*     .. Parameters ..
      COMPLEX*16 ONE
      PARAMETER (ONE= (1.0D+0,0.0D+0))
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*
*     Test the input parameters.
*
      IF (LSAME(TRANS,'N')) THEN
          NROWA = N
      ELSE
          NROWA = K
      END IF
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 1
      ELSE IF ((.NOT.LSAME(TRANS,'N')) .AND.
     +         (.NOT.LSAME(TRANS,'T'))) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
          INFO = 3
      ELSE IF (K.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 7
      ELSE IF (LDB.LT.MAX(1,NROWA)) THEN
          INFO = 9
      ELSE IF (LDC.LT.MAX(1,N)) THEN
          INFO = 12
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZSYR2K',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (((ALPHA.EQ.ZERO).OR.
     +    (K.EQ.0)).AND. (BETA.EQ.ONE))) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
```

```
         IF (UPPER) THEN
             IF (BETA.EQ.ZERO) THEN
                 DO 20 J = 1,N
                     DO 10 I = 1,J
                         C(I,J) = ZERO
   10                CONTINUE
   20            CONTINUE
             ELSE
                 DO 40 J = 1,N
                     DO 30 I = 1,J
                         C(I,J) = BETA*C(I,J)
   30                CONTINUE
   40            CONTINUE
             END IF
         ELSE
             IF (BETA.EQ.ZERO) THEN
                 DO 60 J = 1,N
                     DO 50 I = J,N
                         C(I,J) = ZERO
   50                CONTINUE
   60            CONTINUE
             ELSE
                 DO 80 J = 1,N
                     DO 70 I = J,N
                         C(I,J) = BETA*C(I,J)
   70                CONTINUE
   80            CONTINUE
             END IF
         END IF
         RETURN
      END IF
*
*     Start the operations.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  C := alpha*A*B**T + alpha*B*A**T + C.
*
         IF (UPPER) THEN
             DO 130 J = 1,N
                 IF (BETA.EQ.ZERO) THEN
                     DO 90 I = 1,J
                         C(I,J) = ZERO
   90                CONTINUE
                 ELSE IF (BETA.NE.ONE) THEN
                     DO 100 I = 1,J
                         C(I,J) = BETA*C(I,J)
  100                CONTINUE
                 END IF
                 DO 120 L = 1,K
```

```
                    IF ((A(J,L).NE.ZERO) .OR. (B(J,L).NE.ZERO)) THEN
                        TEMP1 = ALPHA*B(J,L)
                        TEMP2 = ALPHA*A(J,L)
                        DO 110 I = 1,J
                            C(I,J) = C(I,J) + A(I,L)*TEMP1 +
     +                              B(I,L)*TEMP2
  110                   CONTINUE
                    END IF
  120           CONTINUE
  130       CONTINUE
        ELSE
            DO 180 J = 1,N
                IF (BETA.EQ.ZERO) THEN
                    DO 140 I = J,N
                        C(I,J) = ZERO
  140               CONTINUE
                ELSE IF (BETA.NE.ONE) THEN
                    DO 150 I = J,N
                        C(I,J) = BETA*C(I,J)
  150               CONTINUE
                END IF
                DO 170 L = 1,K
                    IF ((A(J,L).NE.ZERO) .OR. (B(J,L).NE.ZERO)) THEN
                        TEMP1 = ALPHA*B(J,L)
                        TEMP2 = ALPHA*A(J,L)
                        DO 160 I = J,N
                            C(I,J) = C(I,J) + A(I,L)*TEMP1 +
     +                              B(I,L)*TEMP2
  160                   CONTINUE
                    END IF
  170           CONTINUE
  180       CONTINUE
        END IF
    ELSE
*
*       Form  C := alpha*A**T*B + alpha*B**T*A + C.
*
         IF (UPPER) THEN
             DO 210 J = 1,N
                 DO 200 I = 1,J
                     TEMP1 = ZERO
                     TEMP2 = ZERO
                     DO 190 L = 1,K
                         TEMP1 = TEMP1 + A(L,I)*B(L,J)
                         TEMP2 = TEMP2 + B(L,I)*A(L,J)
  190                CONTINUE
                     IF (BETA.EQ.ZERO) THEN
                         C(I,J) = ALPHA*TEMP1 + ALPHA*TEMP2
                     ELSE
                         C(I,J) = BETA*C(I,J) + ALPHA*TEMP1 +
```

```
     +                           ALPHA*TEMP2
                        END IF
  200             CONTINUE
  210         CONTINUE
          ELSE
              DO 240 J = 1,N
                  DO 230 I = J,N
                      TEMP1 = ZERO
                      TEMP2 = ZERO
                      DO 220 L = 1,K
                          TEMP1 = TEMP1 + A(L,I)*B(L,J)
                          TEMP2 = TEMP2 + B(L,I)*A(L,J)
  220                 CONTINUE
                      IF (BETA.EQ.ZERO) THEN
                          C(I,J) = ALPHA*TEMP1 + ALPHA*TEMP2
                      ELSE
                          C(I,J) = BETA*C(I,J) + ALPHA*TEMP1 +
     +                           ALPHA*TEMP2
                      END IF
  230             CONTINUE
  240         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of ZSYR2K.
*
      END
```

———-

17.6.4 lisp code

— BLAS 3 zsyr2k —

```
(let* ((one (complex 1.0 0.0)) (zero (complex 0.0 0.0)))
 (declare (type (complex double-float) one) (type (complex double-float) zero))
  (defun zsyr2k (uplo trans n k alpha a lda b ldb$ beta c ldc)
    (declare (type (simple-array (complex double-float) (*)) c b a)
             (type (complex double-float) beta alpha)
             (type fixnum ldc ldb$ lda k n)
             (type character trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (a (complex double-float) a-%data% a-%offset%)
```

```
    (b (complex double-float) b-%data% b-%offset%)
    (c (complex double-float) c-%data% c-%offset%))
 (prog ((temp1 #C(0.0 0.0)) (temp2 #C(0.0 0.0)) (i 0) (info 0) (j 0) (l 0)
        (nrowa 0) (upper nil))
   (declare (type (complex double-float) temp1 temp2)
            (type fixnum i info j l nrowa)
            (type (member t nil) upper))
   (cond
     ((char-equal trans #\N)
      (setf nrowa n))
     (t
      (setf nrowa k)))
   (setf upper (char-equal uplo #\U))
   (setf info 0)
   (cond
     ((and (not upper) (not (char-equal uplo #\L)))
      (setf info 1))
     ((and (not (char-equal trans #\N)) (not (char-equal trans #\T)))
      (setf info 2))
     ((< n 0)
      (setf info 3))
     ((< k 0)
      (setf info 4))
     ((< lda (max (the fixnum 1) (the fixnum nrowa)))
      (setf info 7))
     ((< ldb$
         (max (the fixnum 1) (the fixnum nrowa)))
      (setf info 9))
     ((< ldc (max (the fixnum 1) (the fixnum n)))
      (setf info 12)))
   (cond
     ((/= info 0)
      (error
       " ** On entry to ~a parameter number ~a had an illegal value~%"
       "ZSYR2K" info)
      (go end_label)))
   (if (or (= n 0) (and (or (= alpha zero) (= k 0)) (= beta one)))
       (go end_label))
   (cond
     ((= alpha zero)
      (cond
        (upper
         (cond
           ((= beta zero)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i j) nil)
                  (tagbody
```

```
                (setf (f2cl-lib:fref c-%data%
                                     (i j)
                                     ((1 ldc) (1 *))
                                     c-%offset%)
                      zero))))))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i j) nil)
           (tagbody
             (setf (f2cl-lib:fref c-%data%
                                  (i j)
                                  ((1 ldc) (1 *))
                                  c-%offset%)
                   (* beta
                      (f2cl-lib:fref c-%data%
                                     (i j)
                                     ((1 ldc) (1 *))
                                     c-%offset%))))))))))
(t
 (cond
   ((= beta zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data%
                                 (i j)
                                 ((1 ldc) (1 *))
                                 c-%offset%)
                  zero))))))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data%
                                 (i j)
                                 ((1 ldc) (1 *))
                                 c-%offset%)
                  (* beta
                     (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
```

```
                                                c-%offset%))))))))))))
        (go end_label)))
      (cond
        ((char-equal trans #\N)
         (cond
           (upper
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (cond
                  ((= beta zero)
                   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                 ((> i j) nil)
                     (tagbody
                       (setf (f2cl-lib:fref c-%data%
                                            (i j)
                                            ((1 ldc) (1 *))
                                            c-%offset%)
                             zero))))
                  ((/= beta one)
                   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                 ((> i j) nil)
                     (tagbody
                       (setf (f2cl-lib:fref c-%data%
                                            (i j)
                                            ((1 ldc) (1 *))
                                            c-%offset%)
                             (* beta
                                (f2cl-lib:fref c-%data%
                                               (i j)
                                               ((1 ldc) (1 *))
                                               c-%offset%)))))))
                (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                              ((> l k) nil)
                  (tagbody
                    (cond
                      ((or (/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
                           (/= (f2cl-lib:fref b (j l) ((1 ldb$) (1 *))) zero))
                       (setf temp1
                             (* alpha
                                (f2cl-lib:fref b-%data%
                                               (j l)
                                               ((1 ldb$) (1 *))
                                               b-%offset%)))
                       (setf temp2
                             (* alpha
                                (f2cl-lib:fref a-%data%
                                               (j l)
                                               ((1 lda) (1 *))
                                               a-%offset%)))
```

```
                    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                  ((> i j) nil)
                      (tagbody
                        (setf (f2cl-lib:fref c-%data%
                                             (i j)
                                             ((1 ldc) (1 *))
                                             c-%offset%)
                              (+
                               (f2cl-lib:fref c-%data%
                                              (i j)
                                              ((1 ldc) (1 *))
                                              c-%offset%)
                               (*
                                (f2cl-lib:fref a-%data%
                                               (i l)
                                               ((1 lda) (1 *))
                                               a-%offset%)
                                temp1)
                               (*
                                (f2cl-lib:fref b-%data%
                                               (i l)
                                               ((1 ldb$) (1 *))
                                               b-%offset%)
                                temp2)))))))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((= beta zero)
        (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data%
                                 (i j)
                                 ((1 ldc) (1 *))
                                 c-%offset%)
                  zero))))
       ((/= beta one)
        (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data%
                                 (i j)
                                 ((1 ldc) (1 *))
                                 c-%offset%)
                  (* beta
                     (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
```

```
                                        c-%offset%)))))))
              (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                            ((> l k) nil)
                (tagbody
                  (cond
                    ((or (/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
                         (/= (f2cl-lib:fref b (j l) ((1 ldb$) (1 *))) zero))
                     (setf temp1
                             (* alpha
                                (f2cl-lib:fref b-%data%
                                               (j l)
                                               ((1 ldb$) (1 *))
                                               b-%offset%)))
                     (setf temp2
                             (* alpha
                                (f2cl-lib:fref a-%data%
                                               (j l)
                                               ((1 lda) (1 *))
                                               a-%offset%)))
                     (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                                   ((> i n) nil)
                       (tagbody
                         (setf (f2cl-lib:fref c-%data%
                                              (i j)
                                              ((1 ldc) (1 *))
                                              c-%offset%)
                               (+
                                (f2cl-lib:fref c-%data%
                                               (i j)
                                               ((1 ldc) (1 *))
                                               c-%offset%)
                                (*
                                 (f2cl-lib:fref a-%data%
                                                (i l)
                                                ((1 lda) (1 *))
                                                a-%offset%)
                                 temp1)
                                (*
                                 (f2cl-lib:fref b-%data%
                                                (i l)
                                                ((1 ldb$) (1 *))
                                                b-%offset%)
                                 temp2))))))))))))))
(t
 (cond
   (upper
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                    ((> i j) nil)
            (tagbody
              (setf temp1 zero)
              (setf temp2 zero)
              (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                            ((> l k) nil)
                (tagbody
                  (setf temp1
                          (+ temp1
                             (*
                              (f2cl-lib:fref a-%data%
                                             (l i)
                                             ((1 lda) (1 *))
                                             a-%offset%)
                              (f2cl-lib:fref b-%data%
                                             (l j)
                                             ((1 ldb$) (1 *))
                                             b-%offset%))))
                  (setf temp2
                          (+ temp2
                             (*
                              (f2cl-lib:fref b-%data%
                                             (l i)
                                             ((1 ldb$) (1 *))
                                             b-%offset%)
                              (f2cl-lib:fref a-%data%
                                             (l j)
                                             ((1 lda) (1 *))
                                             a-%offset%))))))
              (cond
                ((= beta zero)
                 (setf (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                        (+ (* alpha temp1) (* alpha temp2))))
                (t
                 (setf (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                        (+
                         (* beta
                            (f2cl-lib:fref c-%data%
                                           (i j)
                                           ((1 ldc) (1 *))
                                           c-%offset%))
                         (* alpha temp1)
                         (* alpha temp2))))))))))
            (t
```

```
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf temp1 zero)
        (setf temp2 zero)
        (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                      ((> l k) nil)
          (tagbody
            (setf temp1
                    (+ temp1
                       (*
                        (f2cl-lib:fref a-%data%
                                       (l i)
                                       ((1 lda) (1 *))
                                       a-%offset%)
                        (f2cl-lib:fref b-%data%
                                       (l j)
                                       ((1 ldb$) (1 *))
                                       b-%offset%))))
            (setf temp2
                    (+ temp2
                       (*
                        (f2cl-lib:fref b-%data%
                                       (l i)
                                       ((1 ldb$) (1 *))
                                       b-%offset%)
                        (f2cl-lib:fref a-%data%
                                       (l j)
                                       ((1 lda) (1 *))
                                       a-%offset%))))))
        (cond
          ((= beta zero)
           (setf (f2cl-lib:fref c-%data%
                                (i j)
                                ((1 ldc) (1 *))
                                c-%offset%)
                 (+ (* alpha temp1) (* alpha temp2))))
          (t
           (setf (f2cl-lib:fref c-%data%
                                (i j)
                                ((1 ldc) (1 *))
                                c-%offset%)
                 (+
                  (* beta
                     (f2cl-lib:fref c-%data%
                                    (i j)
                                    ((1 ldc) (1 *))
```

```
                                                    c-%offset%))
                                (* alpha temp1)
                                (* alpha temp2)))))))))))))
 end_label
        (return (values nil nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

17.7 zsyrk - symmetric rank k $\alpha AA^T + \beta C$ or $\alpha A^T A + \beta C$

17.7.1 Axiom unit tests

— **zsyrk.input** —

```
)set break resume
)sys rm -f zsyrk.output
)spool zsyrk.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

17.7.2 Axiom help page

— **zsyrk.help** —

```
=====================================================================
zsyrk examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

 NAME
      ZSYRK - perform one of the symmetric rank k operations   C
      := alpha*A*A' + beta*C,

 SYNOPSIS
```

```
     SUBROUTINE ZSYRK ( UPLO, TRANS, N, K, ALPHA, A, LDA, BETA,
                        C, LDC )

         CHARACTER*1  UPLO, TRANS

         INTEGER      N, K, LDA, LDC

         COMPLEX*16   ALPHA, BETA

         COMPLEX*16   A( LDA, * ), C( LDC, * )

PURPOSE
     ZSYRK  performs one of the symmetric rank k operations

     or

        C := alpha*A'*A + beta*C,

     where  alpha and beta  are scalars,  C is an  n by n sym-
     metric matrix and  A  is an  n by k  matrix in the first
     case and a  k by n  matrix in the second case.

PARAMETERS
     UPLO   - CHARACTER*1.
            On  entry,  UPLO  specifies  whether  the  upper  or
            lower triangular  part  of the  array  C  is to be
            referenced  as follows:

            UPLO = 'U' or 'u'   Only the  upper triangular part
            of  C is to be referenced.

            UPLO = 'L' or 'l'   Only the  lower triangular part
            of  C is to be referenced.

            Unchanged on exit.

     TRANS  - CHARACTER*1.
            On entry,  TRANS  specifies the operation to be per-
            formed as follows:

            TRANS = 'N' or 'n'   C := alpha*A*A' + beta*C.

            TRANS = 'T' or 't'   C := alpha*A'*A + beta*C.

            Unchanged on exit.

     N      - INTEGER.
            On entry,  N specifies the order of the matrix C.  N
            must be at least zero.  Unchanged on exit.
```

```
     K      - INTEGER.
            On entry with  TRANS = 'N' or 'n',  K  specifies  the
            number of  columns   of  the   matrix   A,   and  on
            entry   with TRANS = 'T' or 't',  K  specifies  the
            number of rows of the matrix A.  K must be at least
            zero.  Unchanged on exit.

     ALPHA  - COMPLEX*16      .
            On entry, ALPHA specifies the scalar alpha.
            Unchanged on exit.

 ka is
     A      -
             COMPLEX*16       array of DIMENSION ( LDA, ka ), where
            k  when  TRANS = 'N' or 'n',  and is  n  otherwise.
            Before entry with  TRANS = 'N' or 'n',  the  leading
            n by k part of the array  A  must contain the matrix
            A,  otherwise the leading  k by n  part of the array
            A  must contain  the matrix A.  Unchanged on exit.

     LDA    - INTEGER.
            On entry, LDA specifies the first dimension of A as
            declared in  the  calling  (sub)  program.   When
            TRANS = 'N' or 'n' then  LDA must be at least  max(
            1, n ), otherwise  LDA must be at least  max( 1, k ).
            Unchanged on exit.

     BETA   - COMPLEX*16      .
            On entry, BETA specifies the scalar beta.  Unchanged
            on exit.

     C      - COMPLEX*16       array of DIMENSION ( LDC, n ).
            Before entry  with  UPLO = 'U' or 'u',  the leading
            n by n upper triangular part of the array C must con-
            tain the upper triangular part  of the  symmetric
            matrix  and the strictly lower triangular part of C
            is not referenced.  On exit, the upper triangular
            part of the array  C is overwritten by the upper tri-
            angular part of the updated matrix.  Before entry
            with  UPLO = 'L' or 'l',  the leading  n by n lower
            triangular part of the array C must contain the lower
            triangular part  of the  symmetric matrix  and the
            strictly upper triangular part of C is not refer-
            enced.  On exit, the lower triangular part of the
            array  C is overwritten by the lower triangular part
            of the updated matrix.

     LDC    - INTEGER.

            On entry, LDC specifies the first dimension of C as
```

```
declared in  the  calling  (sub)  program.   LDC
must  be  at  least max( 1, n ).  Unchanged on exit.
```

———-

17.7.3 fortran code

— **zsyrk.f** —

```
*  ===================================================================
      SUBROUTINE ZSYRK(UPLO,TRANS,N,K,ALPHA,A,LDA,BETA,C,LDC)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX*16 ALPHA,BETA
      INTEGER K,LDA,LDC,N
      CHARACTER TRANS,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),C(LDC,*)
*     ..
*
*  ===================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MAX
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP
      INTEGER I,INFO,J,L,NROWA
      LOGICAL UPPER
*     ..
*     .. Parameters ..
      COMPLEX*16 ONE
      PARAMETER (ONE= (1.0D+0,0.0D+0))
      COMPLEX*16 ZERO
```

```
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*
*     Test the input parameters.
*
      IF (LSAME(TRANS,'N')) THEN
          NROWA = N
      ELSE
          NROWA = K
      END IF
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 1
      ELSE IF ((.NOT.LSAME(TRANS,'N')) .AND.
     +         (.NOT.LSAME(TRANS,'T'))) THEN
          INFO = 2
      ELSE IF (N.LT.0) THEN
          INFO = 3
      ELSE IF (K.LT.0) THEN
          INFO = 4
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
          INFO = 7
      ELSE IF (LDC.LT.MAX(1,N)) THEN
          INFO = 10
      END IF
      IF (INFO.NE.0) THEN
          CALL XERBLA('ZSYRK ',INFO)
          RETURN
      END IF
*
*     Quick return if possible.
*
      IF ((N.EQ.0) .OR. (((ALPHA.EQ.ZERO).OR.
     +    (K.EQ.0)).AND. (BETA.EQ.ONE))) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
          IF (UPPER) THEN
              IF (BETA.EQ.ZERO) THEN
                  DO 20 J = 1,N
                      DO 10 I = 1,J
                          C(I,J) = ZERO
   10                 CONTINUE
   20             CONTINUE
              ELSE
                  DO 40 J = 1,N
                      DO 30 I = 1,J
```

```
                        C(I,J) = BETA*C(I,J)
   30                CONTINUE
   40            CONTINUE
            END IF
         ELSE
            IF (BETA.EQ.ZERO) THEN
               DO 60 J = 1,N
                  DO 50 I = J,N
                     C(I,J) = ZERO
   50             CONTINUE
   60            CONTINUE
            ELSE
               DO 80 J = 1,N
                  DO 70 I = J,N
                     C(I,J) = BETA*C(I,J)
   70             CONTINUE
   80            CONTINUE
            END IF
         END IF
         RETURN
      END IF
*
*     Start the operations.
*
      IF (LSAME(TRANS,'N')) THEN
*
*        Form  C := alpha*A*A**T + beta*C.
*
         IF (UPPER) THEN
            DO 130 J = 1,N
               IF (BETA.EQ.ZERO) THEN
                  DO 90 I = 1,J
                     C(I,J) = ZERO
   90             CONTINUE
               ELSE IF (BETA.NE.ONE) THEN
                  DO 100 I = 1,J
                     C(I,J) = BETA*C(I,J)
  100             CONTINUE
               END IF
               DO 120 L = 1,K
                  IF (A(J,L).NE.ZERO) THEN
                     TEMP = ALPHA*A(J,L)
                     DO 110 I = 1,J
                        C(I,J) = C(I,J) + TEMP*A(I,L)
  110                CONTINUE
                  END IF
  120            CONTINUE
  130         CONTINUE
         ELSE
            DO 180 J = 1,N
```

```
                IF (BETA.EQ.ZERO) THEN
                    DO 140 I = J,N
                        C(I,J) = ZERO
  140               CONTINUE
                ELSE IF (BETA.NE.ONE) THEN
                    DO 150 I = J,N
                        C(I,J) = BETA*C(I,J)
  150               CONTINUE
                END IF
                DO 170 L = 1,K
                    IF (A(J,L).NE.ZERO) THEN
                        TEMP = ALPHA*A(J,L)
                        DO 160 I = J,N
                            C(I,J) = C(I,J) + TEMP*A(I,L)
  160                   CONTINUE
                    END IF
  170           CONTINUE
  180       CONTINUE
        END IF
    ELSE
*
*       Form  C := alpha*A**T*A + beta*C.
*
        IF (UPPER) THEN
            DO 210 J = 1,N
                DO 200 I = 1,J
                    TEMP = ZERO
                    DO 190 L = 1,K
                        TEMP = TEMP + A(L,I)*A(L,J)
  190               CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP
                    ELSE
                        C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                    END IF
  200           CONTINUE
  210       CONTINUE
        ELSE
            DO 240 J = 1,N
                DO 230 I = J,N
                    TEMP = ZERO
                    DO 220 L = 1,K
                        TEMP = TEMP + A(L,I)*A(L,J)
  220               CONTINUE
                    IF (BETA.EQ.ZERO) THEN
                        C(I,J) = ALPHA*TEMP
                    ELSE
                        C(I,J) = ALPHA*TEMP + BETA*C(I,J)
                    END IF
  230           CONTINUE
```

```
  240         CONTINUE
          END IF
      END IF
*
      RETURN
*
*     End of ZSYRK .
*
      END
```

———-

17.7.4 lisp code

— BLAS 3 zsyrk —

```
(let* ((one (complex 1.0 0.0)) (zero (complex 0.0 0.0)))
 (declare (type (complex double-float) one) (type (complex double-float) zero))
  (defun zsyrk (uplo trans n k alpha a lda beta c ldc)
    (declare (type (simple-array (complex double-float) (*)) c a)
             (type (complex double-float) beta alpha)
             (type fixnum ldc lda k n)
             (type character trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (a (complex double-float) a-%data% a-%offset%)
         (c (complex double-float) c-%data% c-%offset%))
      (prog ((temp #C(0.0 0.0)) (i 0) (info 0) (j 0) (l 0) (nrowa 0)
             (upper nil))
        (declare (type (complex double-float) temp)
                 (type fixnum i info j l nrowa)
                 (type (member t nil) upper))
        (cond
          ((char-equal trans #\N)
           (setf nrowa n))
          (t
           (setf nrowa k)))
        (setf upper (char-equal uplo #\U))
        (setf info 0)
        (cond
          ((and (not upper) (not (char-equal uplo #\L)))
           (setf info 1))
          ((and (not (char-equal trans #\N)) (not (char-equal trans #\T)))
           (setf info 2))
          ((< n 0)
           (setf info 3))
```

```
  ((< k 0)
   (setf info 4))
  ((< lda (max (the fixnum 1) (the fixnum nrowa)))
   (setf info 7))
  ((< ldc (max (the fixnum 1) (the fixnum n)))
   (setf info 10)))
(cond
  ((/= info 0)
   (error
    " ** On entry to ~a parameter number ~a had an illegal value~%"
    "ZSYRK" info)
   (go end_label)))
(if (or (= n 0) (and (or (= alpha zero) (= k 0)) (= beta one)))
    (go end_label))
(cond
  ((= alpha zero)
   (cond
     (upper
      (cond
        ((= beta zero)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i j) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                       zero))))))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i j) nil)
               (tagbody
                 (setf (f2cl-lib:fref c-%data%
                                      (i j)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                       (* beta
                          (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%))))))))))
     (t
      (cond
        ((= beta zero)
```

```
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)
                          zero))))))
          (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf (f2cl-lib:fref c-%data%
                                         (i j)
                                         ((1 ldc) (1 *))
                                         c-%offset%)
                          (* beta
                             (f2cl-lib:fref c-%data%
                                            (i j)
                                            ((1 ldc) (1 *))
                                            c-%offset%))))))))))
    (go end_label)))
  (cond
    ((char-equal trans #\N)
     (cond
       (upper
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (cond
              ((= beta zero)
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i j) nil)
                 (tagbody
                   (setf (f2cl-lib:fref c-%data%
                                        (i j)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                         zero))))
              ((/= beta one)
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i j) nil)
                 (tagbody
                   (setf (f2cl-lib:fref c-%data%
```

```
                                        (i j)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                              (* beta
                                 (f2cl-lib:fref c-%data%
                                                (i j)
                                                ((1 ldc) (1 *))
                                                c-%offset%)))))))
          (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                        ((> l k) nil)
            (tagbody
              (cond
                ((/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
                 (setf temp
                         (* alpha
                            (f2cl-lib:fref a-%data%
                                           (j l)
                                           ((1 lda) (1 *))
                                           a-%offset%)))
                 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                               ((> i j) nil)
                   (tagbody
                     (setf (f2cl-lib:fref c-%data%
                                          (i j)
                                          ((1 ldc) (1 *))
                                          c-%offset%)
                           (+
                            (f2cl-lib:fref c-%data%
                                           (i j)
                                           ((1 ldc) (1 *))
                                           c-%offset%)
                            (* temp
                               (f2cl-lib:fref a-%data%
                                              (i l)
                                              ((1 lda) (1 *))
                                              a-%offset%))))))))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((= beta zero)
        (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data%
                                 (i j)
                                 ((1 ldc) (1 *))
                                 c-%offset%)
                  zero))))
```

```
                ((/= beta one)
                 (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                               ((> i n) nil)
                   (tagbody
                     (setf (f2cl-lib:fref c-%data%
                                          (i j)
                                          ((1 ldc) (1 *))
                                          c-%offset%)
                           (* beta
                              (f2cl-lib:fref c-%data%
                                             (i j)
                                             ((1 ldc) (1 *))
                                             c-%offset%)))))))
            (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                          ((> l k) nil)
              (tagbody
                (cond
                  ((/= (f2cl-lib:fref a (j l) ((1 lda) (1 *))) zero)
                   (setf temp
                           (* alpha
                              (f2cl-lib:fref a-%data%
                                             (j l)
                                             ((1 lda) (1 *))
                                             a-%offset%)))
                   (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                                 ((> i n) nil)
                     (tagbody
                       (setf (f2cl-lib:fref c-%data%
                                            (i j)
                                            ((1 ldc) (1 *))
                                            c-%offset%)
                             (+
                              (f2cl-lib:fref c-%data%
                                             (i j)
                                             ((1 ldc) (1 *))
                                             c-%offset%)
                              (* temp
                                 (f2cl-lib:fref a-%data%
                                                (i l)
                                                ((1 lda) (1 *))
                                                a-%offset%))))))))))))
        (t
         (cond
           (upper
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i j) nil)
                  (tagbody
```

```
                (setf temp zero)
                (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                              ((> l k) nil)
                  (tagbody
                    (setf temp
                            (+ temp
                               (*
                                (f2cl-lib:fref a-%data%
                                               (l i)
                                               ((1 lda) (1 *))
                                               a-%offset%)
                                (f2cl-lib:fref a-%data%
                                               (l j)
                                               ((1 lda) (1 *))
                                               a-%offset%))))))
                (cond
                  ((= beta zero)
                   (setf (f2cl-lib:fref c-%data%
                                        (i j)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                         (* alpha temp)))
                  (t
                   (setf (f2cl-lib:fref c-%data%
                                        (i j)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                         (+ (* alpha temp)
                            (* beta
                               (f2cl-lib:fref c-%data%
                                              (i j)
                                              ((1 ldc) (1 *))
                                              c-%offset%))))))))))
            (t
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                               ((> i n) nil)
                   (tagbody
                     (setf temp zero)
                     (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                                   ((> l k) nil)
                       (tagbody
                         (setf temp
                                 (+ temp
                                    (*
                                     (f2cl-lib:fref a-%data%
                                                    (l i)
                                                    ((1 lda) (1 *))
```

```
                                                    a-%offset%)
                                        (f2cl-lib:fref a-%data%
                                                    (l j)
                                                    ((1 lda) (1 *))
                                                    a-%offset%))))))
                        (cond
                          ((= beta zero)
                           (setf (f2cl-lib:fref c-%data%
                                                (i j)
                                                ((1 ldc) (1 *))
                                                c-%offset%)
                                 (* alpha temp)))
                          (t
                           (setf (f2cl-lib:fref c-%data%
                                                (i j)
                                                ((1 ldc) (1 *))
                                                c-%offset%)
                                 (+ (* alpha temp)
                                    (* beta
                                       (f2cl-lib:fref c-%data%
                                                      (i j)
                                                      ((1 ldc) (1 *))
                                                      c-%offset%))))))))))))))
 end_label
        (return (values nil nil nil nil nil nil nil nil nil nil))))))
```

———-

17.8 ztrmm - triangular $\alpha op(A)B$ or $\alpha Bop(A)$, $op(A) = A$ or $op(A) = A^T$ or $op(A) = A^H$

17.8.1 Axiom unit tests

— **ztrmm.input** —

```
)set break resume
)sys rm -f ztrmm.output
)spool ztrmm.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

17.8.2 Axiom help page

— ztrmm.help —

```
=====================================================================
ztrmm examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

 NAME
      ZTRMM - perform one of the matrix-matrix operations   B :=
      alpha*op( A )*B, or B := alpha*B*op( A )  where alpha is a
      scalar, B is an m by n matrix, A is a unit, or non-unit,
      upper or lower triangular matrix and op( A ) is one of   op(
      A ) = A or op( A ) = A' or op( A ) = conjg( A' )

 SYNOPSIS
      SUBROUTINE ZTRMM ( SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A,
                         LDA, B, LDB )

          CHARACTER*1  SIDE, UPLO, TRANSA, DIAG

          INTEGER      M, N, LDA, LDB

          COMPLEX*16   ALPHA

          COMPLEX*16   A( LDA, * ), B( LDB, * )

 PURPOSE
      ZTRMM  performs one of the matrix-matrix operations

 PARAMETERS
      SIDE   - CHARACTER*1.
             On entry,  SIDE specifies whether  op( A ) multiplies
             B from the left or right as follows:

             SIDE = 'L' or 'l'   B := alpha*op( A )*B.

             SIDE = 'R' or 'r'   B := alpha*B*op( A ).

             Unchanged on exit.

      UPLO   - CHARACTER*1.
```

```
        On entry, UPLO specifies whether the matrix A is an
        upper or lower triangular matrix as follows:

        UPLO = 'U' or 'u'   A is an upper triangular matrix.

        UPLO = 'L' or 'l'   A is a lower triangular matrix.

        Unchanged on exit.

        TRANSA - CHARACTER*1.  On entry, TRANSA specifies the
        form of op( A ) to be used in the matrix multiplica-
        tion as follows:

        TRANSA = 'N' or 'n'   op( A ) = A.

        TRANSA = 'T' or 't'   op( A ) = A'.

        TRANSA = 'C' or 'c'   op( A ) = conjg( A' ).

        Unchanged on exit.

DIAG    - CHARACTER*1.
        On entry, DIAG specifies whether or not A is unit
        triangular as follows:

        DIAG = 'U' or 'u'   A is assumed to be unit triangu-
        lar.

        DIAG = 'N' or 'n'   A is not assumed to be unit tri-
        angular.

        Unchanged on exit.

M       - INTEGER.
        On entry, M specifies the number of rows of B. M must
        be at least zero.  Unchanged on exit.

N       - INTEGER.
        On entry, N specifies the number of columns of B.  N
        must be at least zero.  Unchanged on exit.

ALPHA   - COMPLEX*16      .
        On entry,  ALPHA specifies the scalar  alpha. When
        alpha is zero then  A is not referenced and  B need
        not be set before entry.  Unchanged on exit.

A       -
         COMPLEX*16       array of DIMENSION ( LDA, k ), where k
        is m when  SIDE = 'L' or 'l'  and is  n  when  SIDE = 'R'
        or 'r'.  Before entry  with  UPLO = 'U' or 'u',  the
```

```
            leading  k by k upper triangular part of the array  A
            must contain the upper triangular matrix  and the
            strictly lower triangular part of A is not refer-
            enced.  Before entry  with  UPLO = 'L' or 'l',  the
            leading  k by k lower triangular part of the array  A
            must contain the lower triangular matrix  and the
            strictly upper triangular part of A is not refer-
            enced.  Note that when  DIAG = 'U' or 'u',  the diag-
            onal elements of A  are not referenced either,  but
            are assumed to be  unity.  Unchanged on exit.

      LDA   - INTEGER.
            On entry, LDA specifies the first dimension of A as
            declared in the calling (sub) program.  When  SIDE =
            'L' or 'l'  then LDA  must be at least  max( 1, m ),
            when  SIDE = 'R' or 'r' then LDA must be at least
            max( 1, n ).  Unchanged on exit.

      B     - COMPLEX*16       array of DIMENSION ( LDB, n ).
            Before entry,  the leading  m by n part of the array
            B must contain the matrix  B,  and  on exit  is
            overwritten  by the transformed matrix.

      LDB   - INTEGER.
            On entry, LDB specifies the first dimension of B as
            declared in  the  calling  (sub)  program.   LDB
            must  be  at  least max( 1, m ).  Unchanged on exit.


            ———-
```

17.8.3 fortran code

— ztrmm.f —

```
*  ===================================================================
      SUBROUTINE ZTRMM(SIDE,UPLO,TRANSA,DIAG,M,N,ALPHA,A,LDA,B,LDB)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX*16 ALPHA
      INTEGER LDA,LDB,M,N
      CHARACTER DIAG,SIDE,TRANSA,UPLO
*     ..
```

```
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),B(LDB,*)
*     ..
*
*  =====================================================================
*
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DCONJG,MAX
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP
      INTEGER I,INFO,J,K,NROWA
      LOGICAL LSIDE,NOCONJ,NOUNIT,UPPER
*     ..
*     .. Parameters ..
      COMPLEX*16 ONE
      PARAMETER (ONE= (1.0D+0,0.0D+0))
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*
*     Test the input parameters.
*
      LSIDE = LSAME(SIDE,'L')
      IF (LSIDE) THEN
          NROWA = M
      ELSE
          NROWA = N
      END IF
      NOCONJ = LSAME(TRANSA,'T')
      NOUNIT = LSAME(DIAG,'N')
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.LSIDE) .AND. (.NOT.LSAME(SIDE,'R'))) THEN
          INFO = 1
      ELSE IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 2
      ELSE IF ((.NOT.LSAME(TRANSA,'N')) .AND.
     +         (.NOT.LSAME(TRANSA,'T')) .AND.
     +         (.NOT.LSAME(TRANSA,'C'))) THEN
          INFO = 3
      ELSE IF ((.NOT.LSAME(DIAG,'U')) .AND. (.NOT.LSAME(DIAG,'N'))) THEN
```

```
         INFO = 4
      ELSE IF (M.LT.0) THEN
         INFO = 5
      ELSE IF (N.LT.0) THEN
         INFO = 6
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
         INFO = 9
      ELSE IF (LDB.LT.MAX(1,M)) THEN
         INFO = 11
      END IF
      IF (INFO.NE.0) THEN
         CALL XERBLA('ZTRMM ',INFO)
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF (M.EQ.0 .OR. N.EQ.0) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
         DO 20 J = 1,N
            DO 10 I = 1,M
               B(I,J) = ZERO
   10       CONTINUE
   20    CONTINUE
         RETURN
      END IF
*
*     Start the operations.
*
      IF (LSIDE) THEN
         IF (LSAME(TRANSA,'N')) THEN
*
*          Form  B := alpha*A*B.
*
            IF (UPPER) THEN
               DO 50 J = 1,N
                  DO 40 K = 1,M
                     IF (B(K,J).NE.ZERO) THEN
                        TEMP = ALPHA*B(K,J)
                        DO 30 I = 1,K - 1
                           B(I,J) = B(I,J) + TEMP*A(I,K)
   30                   CONTINUE
                        IF (NOUNIT) TEMP = TEMP*A(K,K)
                        B(K,J) = TEMP
                     END IF
   40             CONTINUE
   50          CONTINUE
```

```
            ELSE
                DO 80 J = 1,N
                    DO 70 K = M,1,-1
                        IF (B(K,J).NE.ZERO) THEN
                            TEMP = ALPHA*B(K,J)
                            B(K,J) = TEMP
                            IF (NOUNIT) B(K,J) = B(K,J)*A(K,K)
                            DO 60 I = K + 1,M
                                B(I,J) = B(I,J) + TEMP*A(I,K)
   60                       CONTINUE
                        END IF
   70               CONTINUE
   80           CONTINUE
            END IF
        ELSE
*
*         Form  B := alpha*A**T*B   or   B := alpha*A**H*B.
*
            IF (UPPER) THEN
                DO 120 J = 1,N
                    DO 110 I = M,1,-1
                        TEMP = B(I,J)
                        IF (NOCONJ) THEN
                            IF (NOUNIT) TEMP = TEMP*A(I,I)
                            DO 90 K = 1,I - 1
                                TEMP = TEMP + A(K,I)*B(K,J)
   90                       CONTINUE
                        ELSE
                            IF (NOUNIT) TEMP = TEMP*DCONJG(A(I,I))
                            DO 100 K = 1,I - 1
                                TEMP = TEMP + DCONJG(A(K,I))*B(K,J)
  100                       CONTINUE
                        END IF
                        B(I,J) = ALPHA*TEMP
  110               CONTINUE
  120           CONTINUE
            ELSE
                DO 160 J = 1,N
                    DO 150 I = 1,M
                        TEMP = B(I,J)
                        IF (NOCONJ) THEN
                            IF (NOUNIT) TEMP = TEMP*A(I,I)
                            DO 130 K = I + 1,M
                                TEMP = TEMP + A(K,I)*B(K,J)
  130                       CONTINUE
                        ELSE
                            IF (NOUNIT) TEMP = TEMP*DCONJG(A(I,I))
                            DO 140 K = I + 1,M
                                TEMP = TEMP + DCONJG(A(K,I))*B(K,J)
  140                       CONTINUE
```

```
                        END IF
                        B(I,J) = ALPHA*TEMP
 150                CONTINUE
 160            CONTINUE
            END IF
        END IF
    ELSE
        IF (LSAME(TRANSA,'N')) THEN
*
*         Form  B := alpha*B*A.
*
            IF (UPPER) THEN
                DO 200 J = N,1,-1
                    TEMP = ALPHA
                    IF (NOUNIT) TEMP = TEMP*A(J,J)
                    DO 170 I = 1,M
                        B(I,J) = TEMP*B(I,J)
 170                CONTINUE
                    DO 190 K = 1,J - 1
                        IF (A(K,J).NE.ZERO) THEN
                            TEMP = ALPHA*A(K,J)
                            DO 180 I = 1,M
                                B(I,J) = B(I,J) + TEMP*B(I,K)
 180                        CONTINUE
                        END IF
 190                CONTINUE
 200            CONTINUE
            ELSE
                DO 240 J = 1,N
                    TEMP = ALPHA
                    IF (NOUNIT) TEMP = TEMP*A(J,J)
                    DO 210 I = 1,M
                        B(I,J) = TEMP*B(I,J)
 210                CONTINUE
                    DO 230 K = J + 1,N
                        IF (A(K,J).NE.ZERO) THEN
                            TEMP = ALPHA*A(K,J)
                            DO 220 I = 1,M
                                B(I,J) = B(I,J) + TEMP*B(I,K)
 220                        CONTINUE
                        END IF
 230                CONTINUE
 240            CONTINUE
            END IF
        ELSE
*
*         Form  B := alpha*B*A**T   or   B := alpha*B*A**H.
*
            IF (UPPER) THEN
                DO 280 K = 1,N
```

```
                    DO 260 J = 1,K - 1
                        IF (A(J,K).NE.ZERO) THEN
                            IF (NOCONJ) THEN
                                TEMP = ALPHA*A(J,K)
                            ELSE
                                TEMP = ALPHA*DCONJG(A(J,K))
                            END IF
                            DO 250 I = 1,M
                                B(I,J) = B(I,J) + TEMP*B(I,K)
 250                        CONTINUE
                        END IF
 260                CONTINUE
                    TEMP = ALPHA
                    IF (NOUNIT) THEN
                        IF (NOCONJ) THEN
                            TEMP = TEMP*A(K,K)
                        ELSE
                            TEMP = TEMP*DCONJG(A(K,K))
                        END IF
                    END IF
                    IF (TEMP.NE.ONE) THEN
                        DO 270 I = 1,M
                            B(I,K) = TEMP*B(I,K)
 270                    CONTINUE
                    END IF
 280            CONTINUE
            ELSE
                DO 320 K = N,1,-1
                    DO 300 J = K + 1,N
                        IF (A(J,K).NE.ZERO) THEN
                            IF (NOCONJ) THEN
                                TEMP = ALPHA*A(J,K)
                            ELSE
                                TEMP = ALPHA*DCONJG(A(J,K))
                            END IF
                            DO 290 I = 1,M
                                B(I,J) = B(I,J) + TEMP*B(I,K)
 290                        CONTINUE
                        END IF
 300                CONTINUE
                    TEMP = ALPHA
                    IF (NOUNIT) THEN
                        IF (NOCONJ) THEN
                            TEMP = TEMP*A(K,K)
                        ELSE
                            TEMP = TEMP*DCONJG(A(K,K))
                        END IF
                    END IF
                    IF (TEMP.NE.ONE) THEN
                        DO 310 I = 1,M
```

```
                              B(I,K) = TEMP*B(I,K)
  310                     CONTINUE
                        END IF
  320             CONTINUE
                END IF
            END IF
        END IF
*
      RETURN
*
*     End of ZTRMM .
*
      END
```

———-

17.8.4 lisp code

— BLAS 3 ztrmm —

```
(let* ((one (complex 1.0 0.0)) (zero (complex 0.0 0.0)))
 (declare (type (complex double-float) one) (type (complex double-float) zero))
  (defun ztrmm (side uplo transa diag m n alpha a lda b ldb$)
    (declare (type (simple-array (complex double-float) (*)) b a)
             (type (complex double-float) alpha)
             (type fixnum ldb$ lda n m)
             (type character diag transa uplo side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (transa character transa-%data% transa-%offset%)
         (diag character diag-%data% diag-%offset%)
         (a (complex double-float) a-%data% a-%offset%)
         (b (complex double-float) b-%data% b-%offset%))
      (prog ((temp #C(0.0 0.0)) (i 0) (info 0) (j 0) (k 0) (nrowa 0)
             (lside nil) (noconj nil) (nounit nil) (upper nil))
        (declare (type (complex double-float) temp)
                 (type fixnum i info j k nrowa)
                 (type (member t nil) lside noconj nounit upper))
        (setf lside (char-equal side #\L))
        (cond
          (lside
           (setf nrowa m))
          (t
           (setf nrowa n)))
        (setf noconj (char-equal transa #\T))
        (setf nounit (char-equal diag #\N))
```

```
(setf upper (char-equal uplo #\U))
(setf info 0)
(cond
  ((and (not lside) (not (char-equal side #\R)))
   (setf info 1))
  ((and (not upper) (not (char-equal uplo #\L)))
   (setf info 2))
  ((and (not (char-equal transa #\N))
        (not (char-equal transa #\T))
        (not (char-equal transa #\C)))
   (setf info 3))
  ((and (not (char-equal diag #\U)) (not (char-equal diag #\N)))
   (setf info 4))
  ((< m 0)
   (setf info 5))
  ((< n 0)
   (setf info 6))
  ((< lda (max (the fixnum 1) (the fixnum nrowa)))
   (setf info 9))
  ((< ldb$ (max (the fixnum 1) (the fixnum m)))
   (setf info 11)))
(cond
  ((/= info 0)
   (error
    " ** On entry to ~a parameter number ~a had an illegal value~%"
    "ZTRMM" info)
   (go end_label)))
(if (= n 0) (go end_label))
(cond
  ((= alpha zero)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf (f2cl-lib:fref b-%data%
                                (i j)
                                ((1 ldb$) (1 *))
                                b-%offset%)
                 zero)))))
   (go end_label)))
(cond
  (lside
   (cond
     ((char-equal transa #\N)
      (cond
        (upper
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
```

```
                    (tagbody
                      (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                                    ((> k m) nil)
                        (tagbody
                          (cond
                            ((/= (f2cl-lib:fref b (k j) ((1 ldb$) (1 *))) zero)
                             (setf temp
                                     (* alpha
                                        (f2cl-lib:fref b-%data%
                                                       (k j)
                                                       ((1 ldb$) (1 *))
                                                       b-%offset%)))
                             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                           ((> i
                                               (f2cl-lib:int-add k
                                                                 (f2cl-lib:int-sub
                                                                  1)))
                                            nil)
                               (tagbody
                                 (setf (f2cl-lib:fref b-%data%
                                                      (i j)
                                                      ((1 ldb$) (1 *))
                                                      b-%offset%)
                                       (+
                                        (f2cl-lib:fref b-%data%
                                                       (i j)
                                                       ((1 ldb$) (1 *))
                                                       b-%offset%)
                                        (* temp
                                           (f2cl-lib:fref a-%data%
                                                          (i k)
                                                          ((1 lda) (1 *))
                                                          a-%offset%))))))
                             (if nounit
                                 (setf temp
                                       (* temp
                                          (f2cl-lib:fref a-%data%
                                                         (k k)
                                                         ((1 lda) (1 *))
                                                         a-%offset%))))
                             (setf (f2cl-lib:fref b-%data%
                                                  (k j)
                                                  ((1 ldb$) (1 *))
                                                  b-%offset%)
                                   temp))))))))
                (t
                 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                               ((> j n) nil)
                   (tagbody
                     (f2cl-lib:fdo (k m
```

```
                    (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                   ((> k 1) nil)
          (tagbody
            (cond
              ((/= (f2cl-lib:fref b (k j) ((1 ldb$) (1 *))) zero)
               (setf temp
                       (* alpha
                          (f2cl-lib:fref b-%data%
                                         (k j)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)))
               (setf (f2cl-lib:fref b-%data%
                                    (k j)
                                    ((1 ldb$) (1 *))
                                    b-%offset%)
                       temp)
               (if nounit
                   (setf (f2cl-lib:fref b-%data%
                                        (k j)
                                        ((1 ldb$) (1 *))
                                        b-%offset%)
                         (*
                          (f2cl-lib:fref b-%data%
                                         (k j)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)
                          (f2cl-lib:fref a-%data%
                                         (k k)
                                         ((1 lda) (1 *))
                                         a-%offset%))))
               (f2cl-lib:fdo (i (f2cl-lib:int-add k 1)
                              (f2cl-lib:int-add i 1))
                             ((> i m) nil)
                 (tagbody
                   (setf (f2cl-lib:fref b-%data%
                                        (i j)
                                        ((1 ldb$) (1 *))
                                        b-%offset%)
                         (+
                          (f2cl-lib:fref b-%data%
                                         (i j)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)
                          (* temp
                             (f2cl-lib:fref a-%data%
                                            (i k)
                                            ((1 lda) (1 *))
                                            a-%offset%))))))))))))))
    (t
     (cond
```

```
(upper
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i m
                    (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                   ((> i 1) nil)
       (tagbody
         (setf temp
                 (f2cl-lib:fref b-%data%
                                (i j)
                                ((1 ldb$) (1 *))
                                b-%offset%))
         (cond
           (noconj
            (if nounit
                (setf temp
                        (* temp
                           (f2cl-lib:fref a-%data%
                                          (i i)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
            (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                          ((> k
                              (f2cl-lib:int-add i
                                                (f2cl-lib:int-sub
                                                 1)))
                           nil)
              (tagbody
                (setf temp
                        (+ temp
                           (*
                            (f2cl-lib:fref a-%data%
                                           (k i)
                                           ((1 lda) (1 *))
                                           a-%offset%)
                            (f2cl-lib:fref b-%data%
                                           (k j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%)))))))
           (t
            (if nounit
                (setf temp
                        (* temp
                           (f2cl-lib:dconjg
                            (f2cl-lib:fref a-%data%
                                           (i i)
                                           ((1 lda) (1 *))
                                           a-%offset%)))))
            (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
```

```
                                ((> k
                                    (f2cl-lib:int-add i
                                                      (f2cl-lib:int-sub
                                                       1)))
                                 nil)
                        (tagbody
                          (setf temp
                                  (+ temp
                                     (*
                                      (f2cl-lib:dconjg
                                       (f2cl-lib:fref a-%data%
                                                      (k i)
                                                      ((1 lda) (1 *))
                                                      a-%offset%))
                                      (f2cl-lib:fref b-%data%
                                                     (k j)
                                                     ((1 ldb$) (1 *))
                                                     b-%offset%))))))))
                  (setf (f2cl-lib:fref b-%data%
                                       (i j)
                                       ((1 ldb$) (1 *))
                                       b-%offset%)
                        (* alpha temp)))))))
          (t
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i m) nil)
                 (tagbody
                   (setf temp
                           (f2cl-lib:fref b-%data%
                                          (i j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%))
                   (cond
                     (noconj
                      (if nounit
                          (setf temp
                                  (* temp
                                     (f2cl-lib:fref a-%data%
                                                    (i i)
                                                    ((1 lda) (1 *))
                                                    a-%offset%))))
                      (f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
                                     (f2cl-lib:int-add k 1))
                                    ((> k m) nil)
                        (tagbody
                          (setf temp
                                  (+ temp
```

```
                                        (*
                                         (f2cl-lib:fref a-%data%
                                                        (k i)
                                                        ((1 lda) (1 *))
                                                        a-%offset%)
                                         (f2cl-lib:fref b-%data%
                                                        (k j)
                                                        ((1 ldb$) (1 *))
                                                        b-%offset%)))))))
                            (t
                             (if nounit
                                 (setf temp
                                         (* temp
                                            (f2cl-lib:dconjg
                                             (f2cl-lib:fref a-%data%
                                                            (i i)
                                                            ((1 lda) (1 *))
                                                            a-%offset%)))))
                             (f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
                                            (f2cl-lib:int-add k 1))
                                           ((> k m) nil)
                               (tagbody
                                 (setf temp
                                         (+ temp
                                            (*
                                             (f2cl-lib:dconjg
                                              (f2cl-lib:fref a-%data%
                                                             (k i)
                                                             ((1 lda) (1 *))
                                                             a-%offset%))
                                             (f2cl-lib:fref b-%data%
                                                            (k j)
                                                            ((1 ldb$) (1 *))
                                                            b-%offset%))))))))
                         (setf (f2cl-lib:fref b-%data%
                                              (i j)
                                              ((1 ldb$) (1 *))
                                              b-%offset%)
                               (* alpha temp))))))))))
                (t
                 (cond
                   ((char-equal transa #\N)
                    (cond
                      (upper
                       (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                                     ((> j 1) nil)
                         (tagbody
                           (setf temp alpha)
                           (if nounit
                               (setf temp
```

```
                    (* temp
                       (f2cl-lib:fref a-%data%
                                      (j j)
                                      ((1 lda) (1 *))
                                      a-%offset%))))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref b-%data%
                                   (i j)
                                   ((1 ldb$) (1 *))
                                   b-%offset%)
                    (* temp
                       (f2cl-lib:fref b-%data%
                                      (i j)
                                      ((1 ldb$) (1 *))
                                      b-%offset%)))))
          (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                        ((> k
                            (f2cl-lib:int-add j
                                              (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (cond
                ((/= (f2cl-lib:fref a (k j) ((1 lda) (1 *))) zero)
                 (setf temp
                         (* alpha
                            (f2cl-lib:fref a-%data%
                                           (k j)
                                           ((1 lda) (1 *))
                                           a-%offset%)))
                 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                               ((> i m) nil)
                   (tagbody
                     (setf (f2cl-lib:fref b-%data%
                                          (i j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)
                           (+
                            (f2cl-lib:fref b-%data%
                                           (i j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%)
                            (* temp
                               (f2cl-lib:fref b-%data%
                                              (i k)
                                              ((1 ldb$) (1 *))
                                              b-%offset%))))))))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
            ((> j n) nil)
(tagbody
  (setf temp alpha)
  (if nounit
      (setf temp
              (* temp
                 (f2cl-lib:fref a-%data%
                                (j j)
                                ((1 lda) (1 *))
                                a-%offset%))))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf (f2cl-lib:fref b-%data%
                           (i j)
                           ((1 ldb$) (1 *))
                           b-%offset%)
              (* temp
                 (f2cl-lib:fref b-%data%
                                (i j)
                                ((1 ldb$) (1 *))
                                b-%offset%)))))
  (f2cl-lib:fdo (k (f2cl-lib:int-add j 1)
                 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (cond
        ((/= (f2cl-lib:fref a (k j) ((1 lda) (1 *))) zero)
         (setf temp
                 (* alpha
                    (f2cl-lib:fref a-%data%
                                   (k j)
                                   ((1 lda) (1 *))
                                   a-%offset%)))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf (f2cl-lib:fref b-%data%
                                  (i j)
                                  ((1 ldb$) (1 *))
                                  b-%offset%)
                     (+
                      (f2cl-lib:fref b-%data%
                                     (i j)
                                     ((1 ldb$) (1 *))
                                     b-%offset%)
                      (* temp
                         (f2cl-lib:fref b-%data%
                                        (i k)
                                        ((1 ldb$) (1 *))
```

```
                                                  b-%offset%)))))))))))))))
            (t
             (cond
               (upper
                (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                              ((> k n) nil)
                  (tagbody
                    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                                  ((> j
                                      (f2cl-lib:int-add k
                                                        (f2cl-lib:int-sub 1)))
                                   nil)
                      (tagbody
                        (cond
                          ((/= (f2cl-lib:fref a (j k) ((1 lda) (1 *))) zero)
                           (cond
                             (noconj
                              (setf temp
                                      (* alpha
                                         (f2cl-lib:fref a-%data%
                                                        (j k)
                                                        ((1 lda) (1 *))
                                                        a-%offset%))))
                             (t
                              (setf temp
                                      (* alpha
                                         (f2cl-lib:dconjg
                                          (f2cl-lib:fref a-%data%
                                                         (j k)
                                                         ((1 lda) (1 *))
                                                         a-%offset%))))))
                           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                         ((> i m) nil)
                             (tagbody
                               (setf (f2cl-lib:fref b-%data%
                                                    (i j)
                                                    ((1 ldb$) (1 *))
                                                    b-%offset%)
                                     (+
                                      (f2cl-lib:fref b-%data%
                                                     (i j)
                                                     ((1 ldb$) (1 *))
                                                     b-%offset%)
                                      (* temp
                                         (f2cl-lib:fref b-%data%
                                                        (i k)
                                                        ((1 ldb$) (1 *))
                                                        b-%offset%))))))))))
                    (setf temp alpha)
                    (cond
```

```
                (nounit
                 (cond
                   (noconj
                    (setf temp
                            (* temp
                               (f2cl-lib:fref a-%data%
                                              (k k)
                                              ((1 lda) (1 *))
                                              a-%offset%))))
                   (t
                    (setf temp
                            (* temp
                               (f2cl-lib:dconjg
                                (f2cl-lib:fref a-%data%
                                               (k k)
                                               ((1 lda) (1 *))
                                               a-%offset%))))))))
            (cond
              ((/= temp one)
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i m) nil)
                 (tagbody
                   (setf (f2cl-lib:fref b-%data%
                                        (i k)
                                        ((1 ldb$) (1 *))
                                        b-%offset%)
                         (* temp
                            (f2cl-lib:fref b-%data%
                                           (i k)
                                           ((1 ldb$) (1 *))
                                           b-%offset%))))))))
          (t
           (f2cl-lib:fdo (k n (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                         ((> k 1) nil)
             (tagbody
               (f2cl-lib:fdo (j (f2cl-lib:int-add k 1)
                                (f2cl-lib:int-add j 1))
                             ((> j n) nil)
                 (tagbody
                   (cond
                     ((/= (f2cl-lib:fref a (j k) ((1 lda) (1 *))) zero)
                      (cond
                        (noconj
                         (setf temp
                                 (* alpha
                                    (f2cl-lib:fref a-%data%
                                                   (j k)
                                                   ((1 lda) (1 *))
                                                   a-%offset%))))
                        (t
```

```
            (setf temp
                    (* alpha
                       (f2cl-lib:dconjg
                        (f2cl-lib:fref a-%data%
                                       (j k)
                                       ((1 lda) (1 *))
                                       a-%offset%))))))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data%
                                 (i j)
                                 ((1 ldb$) (1 *))
                                 b-%offset%)
                  (+
                   (f2cl-lib:fref b-%data%
                                  (i j)
                                  ((1 ldb$) (1 *))
                                  b-%offset%)
                   (* temp
                      (f2cl-lib:fref b-%data%
                                     (i k)
                                     ((1 ldb$) (1 *))
                                     b-%offset%))))))))))
(setf temp alpha)
(cond
  (nounit
   (cond
     (noconj
      (setf temp
              (* temp
                 (f2cl-lib:fref a-%data%
                                (k k)
                                ((1 lda) (1 *))
                                a-%offset%))))
     (t
      (setf temp
              (* temp
                 (f2cl-lib:dconjg
                  (f2cl-lib:fref a-%data%
                                 (k k)
                                 ((1 lda) (1 *))
                                 a-%offset%))))))))
(cond
  ((/= temp one)
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i m) nil)
     (tagbody
       (setf (f2cl-lib:fref b-%data%
                            (i k)
```

```
                                                ((1 ldb$) (1 *))
                                                b-%offset%)
                                        (* temp
                                           (f2cl-lib:fref b-%data%
                                                          (i k)
                                                          ((1 ldb$) (1 *))
                                                          b-%offset%)))))))))))))))
 end_label
        (return (values nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

17.9 ztrsm - triangular $op(A)X = \alpha B$ $Xop(A) = \alpha B$, $op(A) = A$ or $op(A) = A^T$ or $op(A) = A^H$

17.9.1 Axiom unit tests

— **ztrsm.input** —

```
)set break resume
)sys rm -f ztrsm.output
)spool ztrsm.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

17.9.2 Axiom help page

— **ztrsm.help** —

```
==================================================================
ztrsm examples
==================================================================

==================================================================
Man Page Details
==================================================================
```

```
NAME
     ZTRSM - solve one of the matrix equations   op( A )*X =
     alpha*B, or X*op( A ) = alpha*B,

SYNOPSIS
     SUBROUTINE ZTRSM ( SIDE, UPLO, TRANSA, DIAG, M, N, ALPHA, A,
                        LDA, B, LDB )

         CHARACTER*1  SIDE, UPLO, TRANSA, DIAG

         INTEGER      M, N, LDA, LDB

         COMPLEX*16   ALPHA

         COMPLEX*16   A( LDA, * ), B( LDB, * )

PURPOSE
     ZTRSM  solves one of the matrix equations

     where alpha is a scalar, X and B are m by n matrices, A is a
     unit, or non-unit,  upper or lower triangular matrix  and
     op( A )  is one  of

        op( A ) = A   or   op( A ) = A'   or   op( A ) = conjg(
     A' ).

     The matrix X is overwritten on B.

PARAMETERS
     SIDE   - CHARACTER*1.
            On entry, SIDE specifies whether op( A ) appears on
            the left or right of X as follows:

            SIDE = 'L' or 'l'   op( A )*X = alpha*B.

            SIDE = 'R' or 'r'   X*op( A ) = alpha*B.

            Unchanged on exit.

     UPLO   - CHARACTER*1.
            On entry, UPLO specifies whether the matrix A is an
            upper or lower triangular matrix as follows:

            UPLO = 'U' or 'u'   A is an upper triangular matrix.

            UPLO = 'L' or 'l'   A is a lower triangular matrix.

            Unchanged on exit.

            TRANSA - CHARACTER*1.  On entry, TRANSA specifies the
```

```
            form of op( A ) to be used in the matrix

            multiplication as follows:

            TRANSA = 'N' or 'n'   op( A ) = A.

            TRANSA = 'T' or 't'   op( A ) = A'.

            TRANSA = 'C' or 'c'   op( A ) = conjg( A' ).

            Unchanged on exit.

      DIAG   - CHARACTER*1.
            On entry, DIAG specifies whether or not A is unit
            triangular as follows:

            DIAG = 'U' or 'u'   A is assumed to be unit triangu-
            lar.

            DIAG = 'N' or 'n'   A is not assumed to be unit tri-
            angular.

            Unchanged on exit.

      M      - INTEGER.
            On entry, M specifies the number of rows of B. M must
            be at least zero.  Unchanged on exit.

      N      - INTEGER.
            On entry, N specifies the number of columns of B.  N
            must be at least zero.  Unchanged on exit.

      ALPHA  - COMPLEX*16      .
            On entry,  ALPHA specifies the scalar  alpha. When
            alpha is zero then  A is not referenced and  B need
            not be set before entry.  Unchanged on exit.

is m
      A      -
             COMPLEX*16       array of DIMENSION ( LDA, k ), where k
            when  SIDE = 'L' or 'l'  and is  n  when  SIDE = 'R'
            or 'r'.  Before entry  with  UPLO = 'U' or 'u',  the
            leading  k by k upper triangular part of the array  A
            must contain the upper triangular matrix  and the
            strictly lower triangular part of A is not refer-
            enced.  Before entry  with  UPLO = 'L' or 'l',  the
            leading  k by k lower triangular part of the array  A
            must contain the lower triangular matrix  and the
            strictly upper triangular part of A is not refer-
            enced.  Note that when  DIAG = 'U' or 'u',  the diag-
```

```
            onal elements of A  are not referenced either,  but
            are assumed to be  unity.  Unchanged on exit.

      LDA    - INTEGER.

            On entry, LDA specifies the first dimension of A as
            declared in the calling (sub) program.  When  SIDE =
            'L' or 'l'  then LDA  must be at least  max( 1, m ),
            when  SIDE = 'R' or 'r' then LDA must be at least
            max( 1, n ).  Unchanged on exit.

      B      - COMPLEX*16       array of DIMENSION ( LDB, n ).
            Before entry,  the leading  m by n part of the array
            B must contain  the  right-hand  side  matrix  B,
            and  on exit  is overwritten by the solution matrix
            X.

      LDB    - INTEGER.
            On entry, LDB specifies the first dimension of B as
            declared in  the  calling  (sub)  program.   LDB
            must  be  at  least max( 1, m ).  Unchanged on exit.
```

———-

17.9.3 fortran code

— ztrsm.f —

```
*  ====================================================================
      SUBROUTINE ZTRSM(SIDE,UPLO,TRANSA,DIAG,M,N,ALPHA,A,LDA,B,LDB)
*
*  -- Reference BLAS level3 routine (version 3.4.0) --
*  -- Reference BLAS is a software package provided by Univ. of Tennessee,
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX*16 ALPHA
      INTEGER LDA,LDB,M,N
      CHARACTER DIAG,SIDE,TRANSA,UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A(LDA,*),B(LDB,*)
*     ..
*
*  ====================================================================
*
```

```
*     .. External Functions ..
      LOGICAL LSAME
      EXTERNAL LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DCONJG,MAX
*     ..
*     .. Local Scalars ..
      COMPLEX*16 TEMP
      INTEGER I,INFO,J,K,NROWA
      LOGICAL LSIDE,NOCONJ,NOUNIT,UPPER
*     ..
*     .. Parameters ..
      COMPLEX*16 ONE
      PARAMETER (ONE= (1.0D+0,0.0D+0))
      COMPLEX*16 ZERO
      PARAMETER (ZERO= (0.0D+0,0.0D+0))
*     ..
*
*     Test the input parameters.
*
      LSIDE = LSAME(SIDE,'L')
      IF (LSIDE) THEN
          NROWA = M
      ELSE
          NROWA = N
      END IF
      NOCONJ = LSAME(TRANSA,'T')
      NOUNIT = LSAME(DIAG,'N')
      UPPER = LSAME(UPLO,'U')
*
      INFO = 0
      IF ((.NOT.LSIDE) .AND. (.NOT.LSAME(SIDE,'R'))) THEN
          INFO = 1
      ELSE IF ((.NOT.UPPER) .AND. (.NOT.LSAME(UPLO,'L'))) THEN
          INFO = 2
      ELSE IF ((.NOT.LSAME(TRANSA,'N')) .AND.
     +         (.NOT.LSAME(TRANSA,'T')) .AND.
     +         (.NOT.LSAME(TRANSA,'C'))) THEN
          INFO = 3
      ELSE IF ((.NOT.LSAME(DIAG,'U')) .AND. (.NOT.LSAME(DIAG,'N'))) THEN
          INFO = 4
      ELSE IF (M.LT.0) THEN
          INFO = 5
      ELSE IF (N.LT.0) THEN
          INFO = 6
      ELSE IF (LDA.LT.MAX(1,NROWA)) THEN
```

```
         INFO = 9
      ELSE IF (LDB.LT.MAX(1,M)) THEN
         INFO = 11
      END IF
      IF (INFO.NE.0) THEN
         CALL XERBLA('ZTRSM ',INFO)
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF (M.EQ.0 .OR. N.EQ.0) RETURN
*
*     And when  alpha.eq.zero.
*
      IF (ALPHA.EQ.ZERO) THEN
         DO 20 J = 1,N
            DO 10 I = 1,M
               B(I,J) = ZERO
   10       CONTINUE
   20    CONTINUE
         RETURN
      END IF
*
*     Start the operations.
*
      IF (LSIDE) THEN
         IF (LSAME(TRANSA,'N')) THEN
*
*          Form  B := alpha*inv( A )*B.
*
            IF (UPPER) THEN
               DO 60 J = 1,N
                  IF (ALPHA.NE.ONE) THEN
                     DO 30 I = 1,M
                        B(I,J) = ALPHA*B(I,J)
   30                CONTINUE
                  END IF
                  DO 50 K = M,1,-1
                     IF (B(K,J).NE.ZERO) THEN
                        IF (NOUNIT) B(K,J) = B(K,J)/A(K,K)
                        DO 40 I = 1,K - 1
                           B(I,J) = B(I,J) - B(K,J)*A(I,K)
   40                   CONTINUE
                     END IF
   50             CONTINUE
   60          CONTINUE
            ELSE
               DO 100 J = 1,N
                  IF (ALPHA.NE.ONE) THEN
```

```
                        DO 70 I = 1,M
                            B(I,J) = ALPHA*B(I,J)
   70                   CONTINUE
                    END IF
                    DO 90 K = 1,M
                        IF (B(K,J).NE.ZERO) THEN
                            IF (NOUNIT) B(K,J) = B(K,J)/A(K,K)
                            DO 80 I = K + 1,M
                                B(I,J) = B(I,J) - B(K,J)*A(I,K)
   80                       CONTINUE
                        END IF
   90               CONTINUE
  100           CONTINUE
            END IF
        ELSE
*
*         Form  B := alpha*inv( A**T )*B
*         or    B := alpha*inv( A**H )*B.
*
            IF (UPPER) THEN
                DO 140 J = 1,N
                    DO 130 I = 1,M
                        TEMP = ALPHA*B(I,J)
                        IF (NOCONJ) THEN
                            DO 110 K = 1,I - 1
                                TEMP = TEMP - A(K,I)*B(K,J)
  110                       CONTINUE
                            IF (NOUNIT) TEMP = TEMP/A(I,I)
                        ELSE
                            DO 120 K = 1,I - 1
                                TEMP = TEMP - DCONJG(A(K,I))*B(K,J)
  120                       CONTINUE
                            IF (NOUNIT) TEMP = TEMP/DCONJG(A(I,I))
                        END IF
                        B(I,J) = TEMP
  130               CONTINUE
  140           CONTINUE
            ELSE
                DO 180 J = 1,N
                    DO 170 I = M,1,-1
                        TEMP = ALPHA*B(I,J)
                        IF (NOCONJ) THEN
                            DO 150 K = I + 1,M
                                TEMP = TEMP - A(K,I)*B(K,J)
  150                       CONTINUE
                            IF (NOUNIT) TEMP = TEMP/A(I,I)
                        ELSE
                            DO 160 K = I + 1,M
                                TEMP = TEMP - DCONJG(A(K,I))*B(K,J)
  160                       CONTINUE
```

```
                            IF (NOUNIT) TEMP = TEMP/DCONJG(A(I,I))
                        END IF
                        B(I,J) = TEMP
  170                 CONTINUE
  180             CONTINUE
              END IF
          END IF
      ELSE
          IF (LSAME(TRANSA,'N')) THEN
*
*           Form  B := alpha*B*inv( A ).
*
              IF (UPPER) THEN
                  DO 230 J = 1,N
                      IF (ALPHA.NE.ONE) THEN
                          DO 190 I = 1,M
                              B(I,J) = ALPHA*B(I,J)
  190                     CONTINUE
                      END IF
                      DO 210 K = 1,J - 1
                          IF (A(K,J).NE.ZERO) THEN
                              DO 200 I = 1,M
                                  B(I,J) = B(I,J) - A(K,J)*B(I,K)
  200                         CONTINUE
                          END IF
  210                 CONTINUE
                      IF (NOUNIT) THEN
                          TEMP = ONE/A(J,J)
                          DO 220 I = 1,M
                              B(I,J) = TEMP*B(I,J)
  220                     CONTINUE
                      END IF
  230             CONTINUE
              ELSE
                  DO 280 J = N,1,-1
                      IF (ALPHA.NE.ONE) THEN
                          DO 240 I = 1,M
                              B(I,J) = ALPHA*B(I,J)
  240                     CONTINUE
                      END IF
                      DO 260 K = J + 1,N
                          IF (A(K,J).NE.ZERO) THEN
                              DO 250 I = 1,M
                                  B(I,J) = B(I,J) - A(K,J)*B(I,K)
  250                         CONTINUE
                          END IF
  260                 CONTINUE
                      IF (NOUNIT) THEN
                          TEMP = ONE/A(J,J)
                          DO 270 I = 1,M
```

```
                            B(I,J) = TEMP*B(I,J)
  270                   CONTINUE
                    END IF
  280            CONTINUE
             END IF
         ELSE
*
*          Form  B := alpha*B*inv( A**T )
*          or    B := alpha*B*inv( A**H ).
*
             IF (UPPER) THEN
                 DO 330 K = N,1,-1
                     IF (NOUNIT) THEN
                         IF (NOCONJ) THEN
                             TEMP = ONE/A(K,K)
                         ELSE
                             TEMP = ONE/DCONJG(A(K,K))
                         END IF
                         DO 290 I = 1,M
                             B(I,K) = TEMP*B(I,K)
  290                    CONTINUE
                     END IF
                     DO 310 J = 1,K - 1
                         IF (A(J,K).NE.ZERO) THEN
                             IF (NOCONJ) THEN
                                 TEMP = A(J,K)
                             ELSE
                                 TEMP = DCONJG(A(J,K))
                             END IF
                             DO 300 I = 1,M
                                 B(I,J) = B(I,J) - TEMP*B(I,K)
  300                        CONTINUE
                         END IF
  310                CONTINUE
                     IF (ALPHA.NE.ONE) THEN
                         DO 320 I = 1,M
                             B(I,K) = ALPHA*B(I,K)
  320                    CONTINUE
                     END IF
  330            CONTINUE
             ELSE
                 DO 380 K = 1,N
                     IF (NOUNIT) THEN
                         IF (NOCONJ) THEN
                             TEMP = ONE/A(K,K)
                         ELSE
                             TEMP = ONE/DCONJG(A(K,K))
                         END IF
                         DO 340 I = 1,M
                             B(I,K) = TEMP*B(I,K)
```

```
 340                    CONTINUE
                    END IF
                    DO 360 J = K + 1,N
                        IF (A(J,K).NE.ZERO) THEN
                            IF (NOCONJ) THEN
                                TEMP = A(J,K)
                            ELSE
                                TEMP = DCONJG(A(J,K))
                            END IF
                            DO 350 I = 1,M
                                B(I,J) = B(I,J) - TEMP*B(I,K)
 350                        CONTINUE
                        END IF
 360                CONTINUE
                    IF (ALPHA.NE.ONE) THEN
                        DO 370 I = 1,M
                            B(I,K) = ALPHA*B(I,K)
 370                    CONTINUE
                    END IF
 380            CONTINUE
            END IF
        END IF
    END IF
*
    RETURN
*
*   End of ZTRSM .
*
    END
```

———-

17.9.4 lisp code

— BLAS 3 ztrsm —

```
(let* ((one (complex 1.0 0.0)) (zero (complex 0.0 0.0)))
 (declare (type (complex double-float) one) (type (complex double-float) zero))
  (defun ztrsm (side uplo transa diag m n alpha a lda b ldb$)
    (declare (type (simple-array (complex double-float) (*)) b a)
             (type (complex double-float) alpha)
             (type fixnum ldb$ lda n m)
             (type character diag transa uplo side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (transa character transa-%data% transa-%offset%)
```

```
   (diag character diag-%data% diag-%offset%)
   (a (complex double-float) a-%data% a-%offset%)
   (b (complex double-float) b-%data% b-%offset%))
(prog ((temp #C(0.0 0.0)) (i 0) (info 0) (j 0) (k 0) (nrowa 0)
       (lside nil) (noconj nil) (nounit nil) (upper nil))
  (declare (type (complex double-float) temp)
           (type fixnum i info j k nrowa)
           (type (member t nil) lside noconj nounit upper))
  (setf lside (char-equal side #\L))
  (cond
    (lside
     (setf nrowa m))
    (t
     (setf nrowa n)))
  (setf noconj (char-equal transa #\T))
  (setf nounit (char-equal diag #\N))
  (setf upper (char-equal uplo #\U))
  (setf info 0)
  (cond
    ((and (not lside) (not (char-equal side #\R)))
     (setf info 1))
    ((and (not upper) (not (char-equal uplo #\L)))
     (setf info 2))
    ((and (not (char-equal transa #\N))
          (not (char-equal transa #\T))
          (not (char-equal transa #\C)))
     (setf info 3))
    ((and (not (char-equal diag #\U)) (not (char-equal diag #\N)))
     (setf info 4))
    ((< m 0)
     (setf info 5))
    ((< n 0)
     (setf info 6))
    ((< lda (max (the fixnum 1) (the fixnum nrowa)))
     (setf info 9))
    ((< ldb$ (max (the fixnum 1) (the fixnum m)))
     (setf info 11)))
  (cond
    ((/= info 0)
     (error
      " ** On entry to ~a parameter number ~a had an illegal value~%"
      "ZTRSM" info)
     (go end_label)))
  (if (= n 0) (go end_label))
  (cond
    ((= alpha zero)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                    ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref b-%data%
                                   (i j)
                                   ((1 ldb$) (1 *))
                                   b-%offset%)
                    zero)))))
   (go end_label)))
(cond
  (lside
   (cond
     ((char-equal transa #\N)
      (cond
        (upper
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (cond
               ((/= alpha one)
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i m) nil)
                  (tagbody
                    (setf (f2cl-lib:fref b-%data%
                                         (i j)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)
                          (* alpha
                             (f2cl-lib:fref b-%data%
                                            (i j)
                                            ((1 ldb$) (1 *))
                                            b-%offset%)))))))
             (f2cl-lib:fdo (k m
                            (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                           ((> k 1) nil)
               (tagbody
                 (cond
                   ((/= (f2cl-lib:fref b (k j) ((1 ldb$) (1 *))) zero)
                    (if nounit
                        (setf (f2cl-lib:fref b-%data%
                                             (k j)
                                             ((1 ldb$) (1 *))
                                             b-%offset%)
                              (/
                               (f2cl-lib:fref b-%data%
                                              (k j)
                                              ((1 ldb$) (1 *))
                                              b-%offset%)
                               (f2cl-lib:fref a-%data%
                                              (k k)
                                              ((1 lda) (1 *))
```

```
                                        a-%offset%))))
                  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                ((> i
                                    (f2cl-lib:int-add k
                                                      (f2cl-lib:int-sub
                                                       1)))
                                 nil)
                    (tagbody
                      (setf (f2cl-lib:fref b-%data%
                                           (i j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%)
                            (-
                             (f2cl-lib:fref b-%data%
                                            (i j)
                                            ((1 ldb$) (1 *))
                                            b-%offset%)
                             (*
                              (f2cl-lib:fref b-%data%
                                             (k j)
                                             ((1 ldb$) (1 *))
                                             b-%offset%)
                              (f2cl-lib:fref a-%data%
                                             (i k)
                                             ((1 lda) (1 *))
                                             a-%offset%))))))))))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
       ((/= alpha one)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data%
                                 (i j)
                                 ((1 ldb$) (1 *))
                                 b-%offset%)
                  (* alpha
                     (f2cl-lib:fref b-%data%
                                    (i j)
                                    ((1 ldb$) (1 *))
                                    b-%offset%)))))))
     (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                   ((> k m) nil)
       (tagbody
         (cond
           ((/= (f2cl-lib:fref b (k j) ((1 ldb$) (1 *))) zero)
            (if nounit
```

```
                    (setf (f2cl-lib:fref b-%data%
                                         (k j)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)
                          (/
                           (f2cl-lib:fref b-%data%
                                          (k j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)
                           (f2cl-lib:fref a-%data%
                                          (k k)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
                (f2cl-lib:fdo (i (f2cl-lib:int-add k 1)
                               (f2cl-lib:int-add i 1))
                              ((> i m) nil)
                  (tagbody
                    (setf (f2cl-lib:fref b-%data%
                                         (i j)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)
                          (-
                           (f2cl-lib:fref b-%data%
                                          (i j)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)
                           (*
                            (f2cl-lib:fref b-%data%
                                           (k j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%)
                            (f2cl-lib:fref a-%data%
                                           (i k)
                                           ((1 lda) (1 *))
                                           a-%offset%))))))))))))))))
(t
 (cond
   (upper
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf temp
                    (* alpha
                       (f2cl-lib:fref b-%data%
                                      (i j)
                                      ((1 ldb$) (1 *))
                                      b-%offset%)))
```

```
(cond
  (noconj
   (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                 ((> k
                     (f2cl-lib:int-add i
                                       (f2cl-lib:int-sub
                                        1)))
                  nil)
     (tagbody
       (setf temp
               (- temp
                  (*
                   (f2cl-lib:fref a-%data%
                                  (k i)
                                  ((1 lda) (1 *))
                                  a-%offset%)
                   (f2cl-lib:fref b-%data%
                                  (k j)
                                  ((1 ldb$) (1 *))
                                  b-%offset%))))))
   (if nounit
       (setf temp
               (/ temp
                  (f2cl-lib:fref a-%data%
                                 (i i)
                                 ((1 lda) (1 *))
                                 a-%offset%)))))
  (t
   (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                 ((> k
                     (f2cl-lib:int-add i
                                       (f2cl-lib:int-sub
                                        1)))
                  nil)
     (tagbody
       (setf temp
               (- temp
                  (*
                   (f2cl-lib:dconjg
                    (f2cl-lib:fref a-%data%
                                   (k i)
                                   ((1 lda) (1 *))
                                   a-%offset%))
                   (f2cl-lib:fref b-%data%
                                  (k j)
                                  ((1 ldb$) (1 *))
                                  b-%offset%))))))
   (if nounit
       (setf temp
               (/ temp
```

```
                                (f2cl-lib:dconjg
                                 (f2cl-lib:fref a-%data%
                                                (i i)
                                                ((1 lda) (1 *))
                                                a-%offset%)))))))
                (setf (f2cl-lib:fref b-%data%
                                     (i j)
                                     ((1 ldb$) (1 *))
                                     b-%offset%)
                      temp))))))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (f2cl-lib:fdo (i m
                            (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                           ((> i 1) nil)
               (tagbody
                 (setf temp
                         (* alpha
                            (f2cl-lib:fref b-%data%
                                           (i j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%)))
                 (cond
                   (noconj
                    (f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
                                     (f2cl-lib:int-add k 1))
                                  ((> k m) nil)
                      (tagbody
                        (setf temp
                                (- temp
                                   (*
                                    (f2cl-lib:fref a-%data%
                                                   (k i)
                                                   ((1 lda) (1 *))
                                                   a-%offset%)
                                    (f2cl-lib:fref b-%data%
                                                   (k j)
                                                   ((1 ldb$) (1 *))
                                                   b-%offset%))))))
                    (if nounit
                        (setf temp
                                (/ temp
                                   (f2cl-lib:fref a-%data%
                                                  (i i)
                                                  ((1 lda) (1 *))
                                                  a-%offset%)))))
                   (t
                    (f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
```

```
                                        (f2cl-lib:int-add k 1))
                                       ((> k m) nil)
                              (tagbody
                                (setf temp
                                        (- temp
                                           (*
                                            (f2cl-lib:dconjg
                                             (f2cl-lib:fref a-%data%
                                                            (k i)
                                                            ((1 lda) (1 *))
                                                            a-%offset%))
                                            (f2cl-lib:fref b-%data%
                                                           (k j)
                                                           ((1 ldb$) (1 *))
                                                           b-%offset%))))))
                            (if nounit
                                (setf temp
                                        (/ temp
                                           (f2cl-lib:dconjg
                                            (f2cl-lib:fref a-%data%
                                                           (i i)
                                                           ((1 lda) (1 *))
                                                           a-%offset%)))))))
                        (setf (f2cl-lib:fref b-%data%
                                             (i j)
                                             ((1 ldb$) (1 *))
                                             b-%offset%)
                              temp)))))))))
                (t
                 (cond
                   ((char-equal transa #\N)
                    (cond
                      (upper
                       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                                     ((> j n) nil)
                         (tagbody
                           (cond
                             ((/= alpha one)
                              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                            ((> i m) nil)
                               (tagbody
                                 (setf (f2cl-lib:fref b-%data%
                                                      (i j)
                                                      ((1 ldb$) (1 *))
                                                      b-%offset%)
                                       (* alpha
                                          (f2cl-lib:fref b-%data%
                                                         (i j)
                                                         ((1 ldb$) (1 *))
                                                         b-%offset%)))))))
```

```
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k
                  (f2cl-lib:int-add j
                                    (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (cond
      ((/= (f2cl-lib:fref a (k j) ((1 lda) (1 *))) zero)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf (f2cl-lib:fref b-%data%
                                (i j)
                                ((1 ldb$) (1 *))
                                b-%offset%)
                 (-
                  (f2cl-lib:fref b-%data%
                                 (i j)
                                 ((1 ldb$) (1 *))
                                 b-%offset%)
                  (*
                   (f2cl-lib:fref a-%data%
                                  (k j)
                                  ((1 lda) (1 *))
                                  a-%offset%)
                   (f2cl-lib:fref b-%data%
                                  (i k)
                                  ((1 ldb$) (1 *))
                                  b-%offset%))))))))))
(cond
  (nounit
   (setf temp
           (/ one
              (f2cl-lib:fref a-%data%
                             (j j)
                             ((1 lda) (1 *))
                             a-%offset%)))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i m) nil)
     (tagbody
       (setf (f2cl-lib:fref b-%data%
                            (i j)
                            ((1 ldb$) (1 *))
                            b-%offset%)
             (* temp
                (f2cl-lib:fref b-%data%
                               (i j)
                               ((1 ldb$) (1 *))
                               b-%offset%))))))))))
(t
```

```
(f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j 1) nil)
  (tagbody
    (cond
      ((/= alpha one)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf (f2cl-lib:fref b-%data%
                                (i j)
                                ((1 ldb$) (1 *))
                                b-%offset%)
                 (* alpha
                    (f2cl-lib:fref b-%data%
                                   (i j)
                                   ((1 ldb$) (1 *))
                                   b-%offset%)))))))
    (f2cl-lib:fdo (k (f2cl-lib:int-add j 1)
                   (f2cl-lib:int-add k 1))
                  ((> k n) nil)
      (tagbody
        (cond
          ((/= (f2cl-lib:fref a (k j) ((1 lda) (1 *))) zero)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf (f2cl-lib:fref b-%data%
                                    (i j)
                                    ((1 ldb$) (1 *))
                                    b-%offset%)
                     (-
                      (f2cl-lib:fref b-%data%
                                     (i j)
                                     ((1 ldb$) (1 *))
                                     b-%offset%)
                      (*
                       (f2cl-lib:fref a-%data%
                                      (k j)
                                      ((1 lda) (1 *))
                                      a-%offset%)
                       (f2cl-lib:fref b-%data%
                                      (i k)
                                      ((1 ldb$) (1 *))
                                      b-%offset%))))))))))
    (cond
      (nounit
       (setf temp
               (/ one
                  (f2cl-lib:fref a-%data%
                                 (j j)
```

```
                                        ((1 lda) (1 *))
                                        a-%offset%)))
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i m) nil)
                  (tagbody
                    (setf (f2cl-lib:fref b-%data%
                                         (i j)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)
                          (* temp
                             (f2cl-lib:fref b-%data%
                                            (i j)
                                            ((1 ldb$) (1 *))
                                            b-%offset%))))))))))))
(t
 (cond
   (upper
    (f2cl-lib:fdo (k n (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                  ((> k 1) nil)
      (tagbody
        (cond
          (nounit
           (cond
             (noconj
              (setf temp
                      (/ one
                         (f2cl-lib:fref a-%data%
                                        (k k)
                                        ((1 lda) (1 *))
                                        a-%offset%))))
             (t
              (setf temp
                      (/ one
                         (f2cl-lib:dconjg
                          (f2cl-lib:fref a-%data%
                                         (k k)
                                         ((1 lda) (1 *))
                                         a-%offset%))))))
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i m) nil)
                  (tagbody
                    (setf (f2cl-lib:fref b-%data%
                                         (i k)
                                         ((1 ldb$) (1 *))
                                         b-%offset%)
                          (* temp
                             (f2cl-lib:fref b-%data%
                                            (i k)
                                            ((1 ldb$) (1 *))
                                            b-%offset%)))))))
```

```
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j
                  (f2cl-lib:int-add k
                                    (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (cond
      ((/= (f2cl-lib:fref a (j k) ((1 lda) (1 *))) zero)
       (cond
         (noconj
          (setf temp
                  (f2cl-lib:fref a-%data%
                                 (j k)
                                 ((1 lda) (1 *))
                                 a-%offset%)))
         (t
          (setf temp
                  (coerce
                   (f2cl-lib:dconjg
                    (f2cl-lib:fref a-%data%
                                   (j k)
                                   ((1 lda) (1 *))
                                   a-%offset%))
                   '(complex double-float)))))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf (f2cl-lib:fref b-%data%
                                (i j)
                                ((1 ldb$) (1 *))
                                b-%offset%)
                   (-
                    (f2cl-lib:fref b-%data%
                                   (i j)
                                   ((1 ldb$) (1 *))
                                   b-%offset%)
                    (* temp
                       (f2cl-lib:fref b-%data%
                                      (i k)
                                      ((1 ldb$) (1 *))
                                      b-%offset%)))))))))
(cond
  ((/= alpha one)
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i m) nil)
     (tagbody
       (setf (f2cl-lib:fref b-%data%
                            (i k)
                            ((1 ldb$) (1 *))
                            b-%offset%)
```

```
                        (* alpha
                           (f2cl-lib:fref b-%data%
                                          (i k)
                                          ((1 ldb$) (1 *))
                                          b-%offset%)))))))))))
(t
 (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
               ((> k n) nil)
   (tagbody
     (cond
       (nounit
        (cond
          (noconj
           (setf temp
                   (/ one
                      (f2cl-lib:fref a-%data%
                                     (k k)
                                     ((1 lda) (1 *))
                                     a-%offset%))))
          (t
           (setf temp
                   (/ one
                      (f2cl-lib:dconjg
                       (f2cl-lib:fref a-%data%
                                      (k k)
                                      ((1 lda) (1 *))
                                      a-%offset%))))))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data%
                                 (i k)
                                 ((1 ldb$) (1 *))
                                 b-%offset%)
                  (* temp
                     (f2cl-lib:fref b-%data%
                                    (i k)
                                    ((1 ldb$) (1 *))
                                    b-%offset%)))))))
     (f2cl-lib:fdo (j (f2cl-lib:int-add k 1)
                    (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (cond
           ((/= (f2cl-lib:fref a (j k) ((1 lda) (1 *))) zero)
            (cond
              (noconj
               (setf temp
                       (f2cl-lib:fref a-%data%
                                      (j k)
```

```
                                                    ((1 lda) (1 *))
                                                    a-%offset%)))
                                  (t
                                   (setf temp
                                           (coerce
                                            (f2cl-lib:dconjg
                                             (f2cl-lib:fref a-%data%
                                                            (j k)
                                                            ((1 lda) (1 *))
                                                            a-%offset%))
                                            '(complex double-float)))))
                                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                              ((> i m) nil)
                                  (tagbody
                                    (setf (f2cl-lib:fref b-%data%
                                                         (i j)
                                                         ((1 ldb$) (1 *))
                                                         b-%offset%)
                                            (-
                                             (f2cl-lib:fref b-%data%
                                                            (i j)
                                                            ((1 ldb$) (1 *))
                                                            b-%offset%)
                                             (* temp
                                                (f2cl-lib:fref b-%data%
                                                               (i k)
                                                               ((1 ldb$) (1 *))
                                                               b-%offset%))))))))))
                        (cond
                          ((/= alpha one)
                           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                         ((> i m) nil)
                             (tagbody
                               (setf (f2cl-lib:fref b-%data%
                                                    (i k)
                                                    ((1 ldb$) (1 *))
                                                    b-%offset%)
                                       (* alpha
                                          (f2cl-lib:fref b-%data%
                                                         (i k)
                                                         ((1 ldb$) (1 *))
                                                         b-%offset%))))))))))))))))
end_label
       (return (values nil nil nil nil nil nil nil nil nil nil nil))))))
```

———-

Chapter 18

LAPACK Concepts

18.1 Tridiagonal matrices [High86]

Tridiagonal matrices

$$A = \begin{bmatrix} a_1 & c_1 & & & 0 \\ b_2 & a_2 & c_2 & & \\ & b_3 & a_3 & \ddots & \\ & & \ddots & & c_{n-1} \\ 0 & & & b_n & a_n \end{bmatrix} \in \mathbb{R}^{n \times n}$$

arise in many areas of numerical analysis, such as spline analysis, difference methods for boundary value problems, and the numerical solution of linear second order recurrence relations. Since the nonzero elements of A occur only within a band of width three, the cost of solving a tridiagonal system $Ax = b$ using Gaussian elimination with partial pivoting is $O(n)$ flops, as opposed to the $O(n^3)$ flops required required when A is a full matrix.

18.2 Condition Number [High86]

Let A be a square nonsingular matrix and $||\cdot||$ a matrix norm. When computing solutions of linear systems $Ax = b$ one would usually like to know the condition number of A,

$$\text{cond } (A) = ||A|| \; ||A^{-1}||$$

, or at least an estimate, since this quantity measures the sensitivity of the true solution to perturbations in the data A and b, and features in various bounds for the norm of the error in the computed solution. For the I_∞ matrix norm, given for $A = (a_{ij})$ by

$$||A||_\infty = max_i \Sigma_j |a_{ij}|$$

or the l_1 matrix norm, $||A||_1 = ||A^T||_\infty ||A||$ is readily obtained whereas computation of $||A^{-1}||$ is more difficult.

18.3 Euclidean Norm [Ahre15]

The Euclidean norm of a vector $\overrightarrow{x}$ (sometimes referred to as $||\overrightarrow{x}||$) is defined as

$$||\overrightarrow{x}|| = \sqrt{\sum_{j=0}^{n-1} x_j^2}, \qquad x_j \in \mathbb{F}$$

Since the summands in the summation are all non-negative, there is no cancellation, so the Euclidean norm operation is usually of high accuracy. It can be implemented simply using a dot product operation on two copies of the same input vector, or by using the sum operation on the squares of the input vector. However, both of these methods are prone to unnecessary overflow and underflow. On one hand, if some input values were too big ($\geq \sqrt{2^{e_{max}+1}}$), the partial sum would overflow. The computed result would be `Inf` when the real Euclidean norm would be much smaller than the overflow threshold. On the other hand, if all input values were too small ($< \sqrt{2^{e_{min}}}$), their squared value would be smaller than the smallest representable floating point number and underflow would occur, causing the returned result to be 0. Scaling techniques can be used to handle the underflow/overflow issues. For example, the function **xnrm2** from the BLAS library scales input data by the intermediate maximum absolute value to avoid both overflow and underflow.

Chapter 19

LAPACK Overview

From the LAPACK User's Guide [LAPA99]:

We use the standard notation for a system of simultaneous linear equations:

$$AX = b$$

where A is the **coefficient matrix**, b is the **right hand side**, and x is the **solution**. A is assumed to be a square matrix of order n, but some of the individual routines allow A to be rectangular. If there are several right hand sides, we write

$$AX = B$$

where the columns of B are the individual right hand sides, and the columns of X are the corresponding solutions. The basic task is to compute X, given A and B.

If A is upper or lower triangular, this can be solved by a straighforward process of backward or forward substitution. Otherwise, the solution is obtained after first factorizing A as a product of triangular matrices (and possibly also a diagonal matrix or permutation matrix).

The form of the factorization depends on the properties of the matrix A. LAPACK provides routines for the following types of matrices, based on the stated factorizations:

- **general** matrices (LU factorization with partial pivoting):

  $$A = PLU$$

  where P is the permutation matrix, L is lower triangular with unit diagonal elements (lower trapezoidal if $m > n$), and U is upper triangular (upper trapezoidal if $m < n$).

- **general band** matrices including **tridiagonal** matrices (LU factorization with partial pivoting): If A is m-by-n with kl subdiagonals and ku superdiagonals, the factorization is

  $$A = LU$$

  where L is a product of permutations and unit lower triangular matrices with kl subdiagonals, and U is upper triangular with $kl + ku$ superdiagonals.

- **symmetric and Hermitian positive definite** matrices including **band** matrices (Cholesky factorization):

$$A = U^T U \text{ or } A = LL^T \text{ in the symmetric case}$$

$$A = U^H U \text{ or } A = LL^H \text{ in the Hermitian case}$$

  where U is the upper triangular matrix and L is the lower triangular.

- **symmetric and Hermitian positive definite tridiagonal** matrices (LDL^T factorization):

$$A = UDU^T \text{ or } A = LDL^T \text{ in the symmetric case}$$

$$A = UDU^H \text{ or } A = LDL^H \text{ in the Hermitian case}$$

  where U is a unit upper bidiagonal matrix, L is unit lower bidiagonal, and D is diagonal.

- **symmetric and Hermitian indefinite** matrices (symmetric indefinite factorization):

$$A = UDU^T \text{ or } A = LDL^T \text{ in the symmetric case}$$

$$A = UDU^H \text{ or } A = LDL^H \text{ in the Hermitian case}$$

  where U (or L) is a product of permutation and unit upper (lower) triangular matrices, and D is symmetric and block diagonal with diagonal blocks of order 1 or 2.

The factorization for a general tridiagonal matrix is like that for a general band matrix with $kl = 1$ and $ku = 1$. The factorization for a symmetric positive definite band matrix with k superdiagonals (or subdiagonals) has the same form as for a symmetric positive definite matrix, but the factor U (or L) is a band matrix with k superdiagonals (subdiagonals). Band matrices use a compact band storage scheme describe in the LAPACK User's Guide [LAPA99] section 5.3.3. LAPACK routines are also provided for symmetric matrices (whether positive definite or indefinite) using **packed** storage, as described in the LAPACK User's Guide section 5.3.2.

While the primary use of a matrix factorization is to solve a system of equations, other related tasks are provided as well. Whereever possible, LAPACK provides routines to perform each of these tasks for each type of matrix and storage scheme. The following list relates the tasks to the last 3 characters of the name of the corresponding computational routine:

- **xyyTRF** factorize (obviously not needed for triangular matrices)
- **xyyTRS** use the factorization (or the matrix A itself if it is triangular) to solve by forward or backward substitution.
- **xyyCON** estimate the reciprocal of the condition number

$$\kappa(A) = ||A|| \cdot ||A^{-1}||$$

  Highman's modification and Hagar's method is used to estimate $|A^{-1}|$ except for symmetric positive definite tridiagonal matrices for which it is computed directly with comparable efficiency

- **xyyRFS** compute bounds on the error in the computed solution (returned by the xyyTRS routine), and refine the solution to reduce the backward error.
- **xyyTRI** use the factorization (or the matrix A itself if it is triangular) to compute A^{-1} (not provided for band matrices, because the inverse does not in general perserve bandedness)
- **xyyEQU** compute scaling factors to equilibrate A (not provided for tridiagonal, symmetric indefinite, or triangular matrices). These routines do not actually scale the matrices: auxiliary routines xLAQyy may be used for that purpose – see the code of the driver routines xyySVX for sample usage

Note that some of the above routines depend on the output of others:

- **xyyTRF** may work on any equilibrated matrix produced by xyyEQU and xLAQyy, if yy is one of {GE, GB, PO, PP, PB}.
- **xyyTRS** requires the factorization returned by xyyTRF.
- **xyyCON** requires the norm of the original matrix A, and the factorization returned by xyyTRF
- **xyyRFS** requires the original matrices A and B, the factorization returned by xyyTRF, and the solution X returned by xyyTRS
- **xyyTRI** requires the factorization returned by xyyTRF

The RFS ("refine solution") routines perform iterative refinement and compute backward and forward error bounds for the solution. Iterative refinement is done in the same precision as the input data. In particular, the residual is *not* computed with extra precision, as has been traditionally done. The benefit of this procedure is discussed in the LAPACK User's Guide section 4.4

19.1 General Matrix

operation	**Single precision**		**Double precision**	
	real	**complex**	**real**	**complex**
factorize	sgetrf	cgetrf	dgetrf	zgetrf
solve using factorization	sgetrs	cgetrs	dgetrs	zgetrs
estimate condition number	sgecon	cgecon	dgecon	zgecon
error bounds for solution	sgerfs	cgerfs	dgerfs	zgerfs
invert using factorization	sgetri	cgetri	dgetri	zgetri
equilibrate	sgeequ	cgeequ	dgeequ	zgeequ

19.2 General Band Matrix

operation	Single precision		Double precision	
	real	complex	real	complex
factorize	sgbtrf	cgbtrf	dgbtrf	zgbtrf
solve using factorization	sgbtrs	cgbtrs	dgbtrs	zgbtrs
estimate condition number	sgbcon	cgbcon	dgbcon	zgbcon
error bounds for solution	sgbrfs	cgbrfs	dgbrfs	zgbrfs
equilibrate	sgbequ	cgbequ	dgbequ	zgbequ

19.3 General Tridiagonal Matrix

operation	Single precision		Double precision	
	real	complex	real	complex
factorize	sgttrf	cgttrf	dgttrf	zgttrf
solve using factorization	sgttrs	cgttrs	dgttrs	zgttrs
estimate condition number	sgtcon	cgtcon	dgtcon	zgtcon
error bounds for solution	sgtrfs	cgtrfs	dgtrfs	zgtrfs

19.4 Symmetric/Hermitian Positive Definite

operation	Single precision		Double precision	
	real	complex	real	complex
factorize	spotrf	cpotrf	dpotrf	zpotrf
solve using factorization	spotrs	cpotrs	dpotrs	zpotrs
estimate condition number	spocon	cpocon	dpocon	zpocon
error bounds for solution	sporfs	cporfs	dporfs	zporfs
invert using factorization	spotri	cpotri	dpotri	zpotri
equilibrate	spoequ	cpoequ	dpoequ	zpoequ

19.5 Symmetric/Hermitian Positive Definite (packed storage)

operation	Single precision		Double precision	
	real	complex	real	complex
factorize	spptrf	cpptrf	dpptrf	zpptrf
solve using factorization	spptrs	cpptrs	dpptrs	zpptrs
estimate condition number	sppcon	cppcon	dppcon	zppcon
error bounds for solution	spprfs	cpprfs	dpprfs	zpprfs
invert using factorization	spptri	cpptri	dpptri	zpptri
equilibrate	sppequ	cppequ	dppequ	zppequ

19.6 Symmetric/Hermitian Positive Definite Band

operation	Single precision		Double precision	
	real	complex	real	complex
factorize	spbtrf	cpbtrf	dpbtrf	zpbtrf
solve using factorization	spbtrs	cpbtrs	dpbtrs	zpbtrs
estimate condition number	spbcon	cpbcon	dpbcon	zpbcon
error bounds for solution	spbrfs	cpbrfs	dpbrfs	zpbrfs
equilibrate	spbequ	cpbequ	dpbequ	zpbequ

19.7 Symmetric/Hermitian Positive Definite Tridiagonal

operation	Single precision		Double precision	
	real	complex	real	complex
factorize	spttrf	cpttrf	dpttrf	zpttrf
solve using factorization	spttrs	cpttrs	dpttrs	zpttrs
estimate condition number	sptcon	cptcon	dptcon	zptcon
error bounds for solution	sptrfs	cptrfs	dptrfs	zptrfs

19.8 Symmetric/Hermitian Indefinite Matrix

operation	Single precision		Double precision	
	real	complex	real	complex
factorize	ssytrf	chetrf	dsytrf	zhetrf
solve using factorization	ssytrs	chetrs	dsytrs	zhetrs
estimate condition number	ssycon	checon	dsycon	zhecon
error bounds for solution	ssyrfs	cherfs	dsyrfs	zherfs
invert using factorization	ssytri	chetri	dsytri	zhetri

19.9 Complex Symmetric Matrix

operation	Single precision		Double precision	
	real	complex	real	complex
factorize		csytrf		zsytrf
solve using factorization		csytrs		zsytrs
estimate condition number		csycon		zsycon
error bounds for solution		csyrfs		zsyrfs
invert using factorization		csytri		zsytri

19.10 Symmetric/Hermitian Indefinite Matrix (packed storage)

operation	**Single precision**		**Double precision**	
	real	**complex**	**real**	**complex**
factorize	ssptrf	chptrf	dsptrf	zhptrf
solve using factorization	ssptrs	chptrs	dsptrs	zhptrs
estimate condition number	sspcon	chpcon	dspcon	zhpcon
error bounds for solution	ssprfs	chprfs	dsprfs	zhprfs
invert using factorization	ssptri	chptri	dsptri	zhptri

19.11 Complex Symmetric (packed storage) Matrix

operation	**Single precision**		**Double precision**	
	real	**complex**	**real**	**complex**
factorize		csptrf		zsptrf
solve using factorization		csptrs		zsptrs
estimate condition number		cspcon		zspcon
error bounds for solution		csprfs		zsprfs
invert using factorization		csptri		zsptri

19.12 Triangular Matrix

operation	**Single precision**		**Double precision**	
	real	**complex**	**real**	**complex**
solve	strtrs	ctrtrs	dtrtrs	ztrtrs
estimate condition number	strcon	ctrcon	dtrcon	ztrcon
error bounds for solution	strrfs	ctrrfs	dtrrfs	ztrrfs
invert	strtri	ctrtri	dtrtri	ztrtri

19.13 Triangular Matrix (packed storage)

operation	**Single precision**		**Double precision**	
	real	**complex**	**real**	**complex**
solve	stptrs	ctptrs	dtptrs	ztptrs
estimate condition number	stpcon	ctpcon	dtpcon	ztpcon
error bounds for solution	stprfs	ctprfs	dtprfs	ztprfs
invert	stptri	ctptri	dtptri	ztptri

19.14 Triangular Band Matrix

operation	**Single precision**		**Double precision**	
	real	**complex**	**real**	**complex**
solve	stbtrs	ctbtrs	dtbtrs	ztbtrs
estimate condition number	stbcon	ctbcon	dtbcon	ztbcon
error bounds for solution	stbrfs	ctbrfs	dtbrfs	ztbrfs

Chapter 20

LAPACK - General Matrices, Linear Solve, Double

20.1 dgels

20.1.1 Axiom unit tests

— **dgels.input** —

```
)set break resume
)sys rm -f dgels.output
)spool dgels.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

20.1.2 Axiom help page

— **dgels.help** —

```
==========================================================================
dgels examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgels (TRANS, M, N, NRHS, A, LDA, B, LDB, WORK, LWORK, INFO)
            DGELS solves overdetermined or underdetermined systems for GE
           matrices

Function/Subroutine Documentation
   subroutine dgels (character TRANS, integer M, integer N, integer NRHS,
       double precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldb, * ) B, integer LDB, double precision, dimension( * )
       WORK, integer LWORK, integer INFO)
        DGELS solves overdetermined or underdetermined systems for GE matrices

Purpose:
   DGELS solves overdetermined or underdetermined real linear systems
   involving an M-by-N matrix A, or its transpose, using a QR or LQ
   factorization of A.  It is assumed that A has full rank.

   The following options are provided:

   1. If TRANS = 'N' and m >= n:  find the least squares solution of
      an overdetermined system, i.e., solve the least squares problem
      minimize || B - A*X ||.

   2. If TRANS = 'N' and m < n:  find the minimum norm solution of
      an underdetermined system A * X = B.

   3. If TRANS = 'T' and m >= n:  find the minimum norm solution of
      an undetermined system A**T * X = B.

   4. If TRANS = 'T' and m < n:  find the least squares solution of
      an overdetermined system, i.e., solve the least squares problem
      minimize || B - A**T * X ||.

   Several right hand side vectors b and solution vectors x can be
   handled in a single call; they are stored as the columns of the
   M-by-NRHS right hand side matrix B and the N-by-NRHS solution
   matrix X.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   = 'N': the linear system involves A;
   = 'T': the linear system involves A**T.
```

```
M
  M is INTEGER
  The number of rows of the matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of
  columns of the matrices B and X. NRHS >=0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit,
  if M >= N, A is overwritten by details of its QR
             factorization as returned by DGEQRF;
  if M <  N, A is overwritten by details of its LQ
             factorization as returned by DGELQF.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the matrix B of right hand side vectors, stored
  columnwise; B is M-by-NRHS if TRANS = 'N', or N-by-NRHS
  if TRANS = 'T'.
  On exit, if INFO = 0, B is overwritten by the solution
  vectors, stored columnwise:
  if TRANS = 'N' and m >= n, rows 1 to n of B contain the least
  squares solution vectors; the residual sum of squares for the
  solution in each column is given by the sum of squares of
  elements N+1 to M in that column;
  if TRANS = 'N' and m < n, rows 1 to N of B contain the
  minimum norm solution vectors;
  if TRANS = 'T' and m >= n, rows 1 to M of B contain the
  minimum norm solution vectors;
  if TRANS = 'T' and m < n, rows 1 to M of B contain the
  least squares solution vectors; the residual sum of squares
  for the solution in each column is given by the sum of
  squares of elements M+1 to N in that column.

LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= MAX(1,M,N).
```

```
 WORK
   WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.
   LWORK >= max( 1, MN + max( MN, NRHS ) ).
   For optimal performance,
   LWORK >= max( 1, MN + max( MN, NRHS )*NB ).
   where MN = min(M,N) and NB is the optimum block size.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO =  i, the i-th diagonal element of the
         triangular factor of A is zero, so that A does not have
         full rank; the least squares solution could not be computed.
```

———-

20.1.3 fortran code

— **dgels.f** —

```
*  =====================================================================
      SUBROUTINE DGELS( TRANS, M, N, NRHS, A, LDA, B, LDB, WORK, LWORK,
     $                  INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDA, LDB, LWORK, M, N, NRHS
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * ), WORK( * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, TPSD
      INTEGER            BROW, I, IASCL, IBSCL, J, MN, NB, SCLLEN, WSIZE
      DOUBLE PRECISION   ANRM, BIGNUM, BNRM, SMLNUM
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   RWORK( 1 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           LSAME, ILAENV, DLABAD, DLAMCH, DLANGE
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGELQF, DGEQRF, DLASCL, DLASET, DORMLQ, DORMQR,
     $                   DTRTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      MN = MIN( M, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.( LSAME( TRANS, 'N' ) .OR. LSAME( TRANS, 'T' ) ) ) THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -6
      ELSE IF( LDB.LT.MAX( 1, M, N ) ) THEN
         INFO = -8
      ELSE IF( LWORK.LT.MAX( 1, MN+MAX( MN, NRHS ) ) .AND. .NOT.LQUERY )
     $          THEN
```

```
         INFO = -10
      END IF
*
*     Figure out optimal block size
*
      IF( INFO.EQ.0 .OR. INFO.EQ.-10 ) THEN
*
         TPSD = .TRUE.
         IF( LSAME( TRANS, 'N' ) )
     $      TPSD = .FALSE.
*
         IF( M.GE.N ) THEN
            NB = ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
            IF( TPSD ) THEN
               NB = MAX( NB, ILAENV( 1, 'DORMQR', 'LN', M, NRHS, N,
     $              -1 ) )
            ELSE
               NB = MAX( NB, ILAENV( 1, 'DORMQR', 'LT', M, NRHS, N,
     $              -1 ) )
            END IF
         ELSE
            NB = ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
            IF( TPSD ) THEN
               NB = MAX( NB, ILAENV( 1, 'DORMLQ', 'LT', N, NRHS, M,
     $              -1 ) )
            ELSE
               NB = MAX( NB, ILAENV( 1, 'DORMLQ', 'LN', N, NRHS, M,
     $              -1 ) )
            END IF
         END IF
*
         WSIZE = MAX( 1, MN+MAX( MN, NRHS )*NB )
         WORK( 1 ) = DBLE( WSIZE )
*
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGELS ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( MIN( M, N, NRHS ).EQ.0 ) THEN
         CALL DLASET( 'Full', MAX( M, N ), NRHS, ZERO, ZERO, B, LDB )
         RETURN
      END IF
*
```

```
*     Get machine parameters
*
      SMLNUM = DLAMCH( 'S' ) / DLAMCH( 'P' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
*
*     Scale A, B if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = DLANGE( 'M', M, N, A, LDA, RWORK )
      IASCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL DLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
         IASCL = 1
      ELSE IF( ANRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL DLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
         IASCL = 2
      ELSE IF( ANRM.EQ.ZERO ) THEN
*
*        Matrix all zero. Return zero solution.
*
         CALL DLASET( 'F', MAX( M, N ), NRHS, ZERO, ZERO, B, LDB )
         GO TO 50
      END IF
*
      BROW = M
      IF( TPSD )
     $   BROW = N
      BNRM = DLANGE( 'M', BROW, NRHS, B, LDB, RWORK )
      IBSCL = 0
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL DLASCL( 'G', 0, 0, BNRM, SMLNUM, BROW, NRHS, B, LDB,
     $                INFO )
         IBSCL = 1
      ELSE IF( BNRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL DLASCL( 'G', 0, 0, BNRM, BIGNUM, BROW, NRHS, B, LDB,
     $                INFO )
         IBSCL = 2
      END IF
```

```
*
      IF( M.GE.N ) THEN
*
*        compute QR factorization of A
*
         CALL DGEQRF( M, N, A, LDA, WORK( 1 ), WORK( MN+1 ), LWORK-MN,
     $                INFO )
*
*        workspace at least N, optimally N*NB
*
         IF( .NOT.TPSD ) THEN
*
*           Least-Squares Problem min || A * X - B ||
*
*           B(1:M,1:NRHS) := Q**T * B(1:M,1:NRHS)
*
            CALL DORMQR( 'Left', 'Transpose', M, NRHS, N, A, LDA,
     $                   WORK( 1 ), B, LDB, WORK( MN+1 ), LWORK-MN,
     $                   INFO )
*
*           workspace at least NRHS, optimally NRHS*NB
*
*           B(1:N,1:NRHS) := inv(R) * B(1:N,1:NRHS)
*
            CALL DTRTRS( 'Upper', 'No transpose', 'Non-unit', N, NRHS,
     $                   A, LDA, B, LDB, INFO )
*
            IF( INFO.GT.0 ) THEN
               RETURN
            END IF
*
            SCLLEN = N
*
         ELSE
*
*           Overdetermined system of equations A**T * X = B
*
*           B(1:N,1:NRHS) := inv(R**T) * B(1:N,1:NRHS)
*
            CALL DTRTRS( 'Upper', 'Transpose', 'Non-unit', N, NRHS,
     $                   A, LDA, B, LDB, INFO )
*
            IF( INFO.GT.0 ) THEN
               RETURN
            END IF
*
*           B(N+1:M,1:NRHS) = ZERO
*
            DO 20 J = 1, NRHS
               DO 10 I = N + 1, M
```

```
                B( I, J ) = ZERO
   10         CONTINUE
   20      CONTINUE
*
*          B(1:M,1:NRHS) := Q(1:N,:) * B(1:N,1:NRHS)
*
           CALL DORMQR( 'Left', 'No transpose', M, NRHS, N, A, LDA,
     $                  WORK( 1 ), B, LDB, WORK( MN+1 ), LWORK-MN,
     $                  INFO )
*
*          workspace at least NRHS, optimally NRHS*NB
*
           SCLLEN = M
*
        END IF
*
      ELSE
*
*       Compute LQ factorization of A
*
        CALL DGELQF( M, N, A, LDA, WORK( 1 ), WORK( MN+1 ), LWORK-MN,
     $               INFO )
*
*       workspace at least M, optimally M*NB.
*
        IF( .NOT.TPSD ) THEN
*
*          underdetermined system of equations A * X = B
*
           B(1:M,1:NRHS) := inv(L) * B(1:M,1:NRHS)
*
           CALL DTRTRS( 'Lower', 'No transpose', 'Non-unit', M, NRHS,
     $                  A, LDA, B, LDB, INFO )
*
           IF( INFO.GT.0 ) THEN
              RETURN
           END IF
*
           B(M+1:N,1:NRHS) = 0
*
           DO 40 J = 1, NRHS
              DO 30 I = M + 1, N
                 B( I, J ) = ZERO
   30         CONTINUE
   40      CONTINUE
*
*          B(1:N,1:NRHS) := Q(1:N,:)**T * B(1:M,1:NRHS)
*
           CALL DORMLQ( 'Left', 'Transpose', N, NRHS, M, A, LDA,
     $                  WORK( 1 ), B, LDB, WORK( MN+1 ), LWORK-MN,
```

```
     $                   INFO )
*
*          workspace at least NRHS, optimally NRHS*NB
*
           SCLLEN = N
*
        ELSE
*
*          overdetermined system min || A**T * X - B ||
*
*          B(1:N,1:NRHS) := Q * B(1:N,1:NRHS)
*
           CALL DORMLQ( 'Left', 'No transpose', N, NRHS, M, A, LDA,
     $                   WORK( 1 ), B, LDB, WORK( MN+1 ), LWORK-MN,
     $                   INFO )
*
*          workspace at least NRHS, optimally NRHS*NB
*
*          B(1:M,1:NRHS) := inv(L**T) * B(1:M,1:NRHS)
*
           CALL DTRTRS( 'Lower', 'Transpose', 'Non-unit', M, NRHS,
     $                   A, LDA, B, LDB, INFO )
*
           IF( INFO.GT.0 ) THEN
              RETURN
           END IF
*
           SCLLEN = M
*
        END IF
*
      END IF
*
*     Undo scaling
*
      IF( IASCL.EQ.1 ) THEN
         CALL DLASCL( 'G', 0, 0, ANRM, SMLNUM, SCLLEN, NRHS, B, LDB,
     $                INFO )
      ELSE IF( IASCL.EQ.2 ) THEN
         CALL DLASCL( 'G', 0, 0, ANRM, BIGNUM, SCLLEN, NRHS, B, LDB,
     $                INFO )
      END IF
      IF( IBSCL.EQ.1 ) THEN
         CALL DLASCL( 'G', 0, 0, SMLNUM, BNRM, SCLLEN, NRHS, B, LDB,
     $                INFO )
      ELSE IF( IBSCL.EQ.2 ) THEN
         CALL DLASCL( 'G', 0, 0, BIGNUM, BNRM, SCLLEN, NRHS, B, LDB,
     $                INFO )
      END IF
*
```

```
   50 CONTINUE
      WORK( 1 ) = DBLE( WSIZE )
*
      RETURN
*
*     End of DGELS
*
      END
```

20.1.4 lisp code

— **dgels.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dgels (trans m n nrhs a lda b ldb$ work lwork info)
    (declare (type (array double-float (*)) work b a)
             (type (f2cl-lib:integer4) info lwork ldb$ lda nrhs n m)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((rwork (make-array 1 :element-type 'double-float)) (anrm 0.0d0)
             (bignum 0.0d0) (bnrm 0.0d0) (smlnum 0.0d0) (brow 0) (i 0)
             (iascl 0) (ibscl 0) (j 0) (mn 0) (nb 0) (scllen 0) (wsize 0)
             (lquery nil) (tpsd nil))
        (declare (type (array double-float (1)) rwork)
                 (type (double-float) anrm bignum bnrm smlnum)
                 (type (f2cl-lib:integer4) brow i iascl ibscl j mn nb scllen
                  wsize)
                 (type f2cl-lib:logical lquery tpsd))
        (setf info 0)
        (setf mn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond
         ((not
           (or
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "N")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val)
```

```
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame trans "T")
      (declare (ignore var-1))
      (when var-0 (setf trans var-0))
      ret-val)))
  (setf info -1))
 ((< m 0) (setf info -2)) ((< n 0) (setf info -3))
 ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -6))
 ((< ldb$
     (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)
          (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and
   (< lwork
      (max (the f2cl-lib:integer4 1)
           (the f2cl-lib:integer4
                (f2cl-lib:int-add mn
                                  (max (the f2cl-lib:integer4 mn)
                                       (the f2cl-lib:integer4
                                            nrhs))))))
   (not lquery))
  (setf info -10)))
(cond
 ((or (= info 0) (= info (f2cl-lib:int-sub 10)))
  (setf tpsd f2cl-lib:%true%)
  (if (multiple-value-bind (ret-val var-0 var-1)
          (lsame trans "N")
        (declare (ignore var-1))
        (when var-0 (setf trans var-0))
        ret-val)
      (setf tpsd f2cl-lib:%false%))
  (cond
   ((>= m n)
    (setf nb
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 1 "DGEQRF" " " m n -1 -1)
              (declare (ignore var-0 var-1 var-2 var-5 var-6))
              (when var-3 (setf m var-3))
              (when var-4 (setf n var-4))
              ret-val))
    (cond
     (tpsd
      (setf nb
              (max (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4
```

```
                                var-5 var-6)
                               (ilaenv 1 "DORMQR" "LN" m nrhs n -1)
                             (declare (ignore var-0 var-1 var-2 var-6))
                             (when var-3 (setf m var-3))
                             (when var-4 (setf nrhs var-4))
                             (when var-5 (setf n var-5))
                             ret-val)))))
          (t
           (setf nb
                   (max (the f2cl-lib:integer4 nb)
                        (the f2cl-lib:integer4
                             (multiple-value-bind
                                 (ret-val var-0 var-1 var-2 var-3 var-4
                                  var-5 var-6)
                                 (ilaenv 1 "DORMQR" "LT" m nrhs n -1)
                               (declare (ignore var-0 var-1 var-2 var-6))
                               (when var-3 (setf m var-3))
                               (when var-4 (setf nrhs var-4))
                               (when var-5 (setf n var-5))
                               ret-val)))))))
        (t
         (setf nb
                 (multiple-value-bind
                     (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                     (ilaenv 1 "DGELQF" " " m n -1 -1)
                   (declare (ignore var-0 var-1 var-2 var-5 var-6))
                   (when var-3 (setf m var-3))
                   (when var-4 (setf n var-4))
                   ret-val))
         (cond
          (tpsd
           (setf nb
                   (max (the f2cl-lib:integer4 nb)
                        (the f2cl-lib:integer4
                             (multiple-value-bind
                                 (ret-val var-0 var-1 var-2 var-3 var-4
                                  var-5 var-6)
                                 (ilaenv 1 "DORMLQ" "LT" n nrhs m -1)
                               (declare (ignore var-0 var-1 var-2 var-6))
                               (when var-3 (setf n var-3))
                               (when var-4 (setf nrhs var-4))
                               (when var-5 (setf m var-5))
                               ret-val)))))
          (t
           (setf nb
                   (max (the f2cl-lib:integer4 nb)
                        (the f2cl-lib:integer4
                             (multiple-value-bind
                                 (ret-val var-0 var-1 var-2 var-3 var-4
                                  var-5 var-6)
```

```
                              (ilaenv 1 "DORMLQ" "LN" n nrhs m -1)
                            (declare (ignore var-0 var-1 var-2 var-6))
                            (when var-3 (setf n var-3))
                            (when var-4 (setf nrhs var-4))
                            (when var-5 (setf m var-5))
                            ret-val))))))))
  (setf wsize
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add mn
                                      (f2cl-lib:int-mul
                                       (max (the f2cl-lib:integer4 mn)
                                            (the f2cl-lib:integer4
                                                 nrhs))
                                       nb)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:dble wsize))))
(cond
 ((/= info 0) (xerbla "DGELS " (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond
 ((=
   (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)
        (the f2cl-lib:integer4 nrhs))
   0)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "Full"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       zero zero b ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldb$ var-6)))
  (go end_label)))
(setf smlnum (/ (dlamch "S") (dlamch "P")))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "M" m n a lda rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
```

```
          ret-val))
(setf iascl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 anrm smlnum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 1))
 ((> anrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 anrm bignum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 2))
 ((= anrm zero)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "F"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       zero zero b ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldb$ var-6)))
  (go label50)))
(setf brow m)
(if tpsd
    (setf brow n))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "M" brow nrhs b ldb$ rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf brow var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
```

```
(setf ibscl 0)
(cond
 ((and (> bnrm zero) (< bnrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 bnrm smlnum brow nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf brow var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 1))
 ((> bnrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 bnrm bignum brow nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf brow var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 2)))
(cond
 ((>= m n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float ((+ mn 1))
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-sub lwork mn) info)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf info var-7))
  (cond
   ((not tpsd)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (dormqr "Left" "Transpose" m nrhs n a lda
         (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                               work-%offset%)
         b ldb$
```

```
      (f2cl-lib:array-slice work-%data% double-float ((+ mn 1))
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-sub lwork mn) info)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf n var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldb$ var-9))
    (when var-12 (setf info var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dtrtrs "Upper" "No transpose" "Non-unit" n nrhs a lda b ldb$
       info)
    (declare (ignore var-0 var-1 var-2 var-5 var-7))
    (when var-3 (setf n var-3))
    (when var-4 (setf nrhs var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (cond ((> info 0) (go end_label))) (setf scllen n))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dtrtrs "Upper" "Transpose" "Non-unit" n nrhs a lda b ldb$
       info)
    (declare (ignore var-0 var-1 var-2 var-5 var-7))
    (when var-3 (setf n var-3))
    (when var-4 (setf nrhs var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (cond ((> info 0) (go end_label)))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (f2cl-lib:fdo (i (f2cl-lib:int-add n 1) (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                zero)
        label10))
    label20))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dormqr "Left" "No transpose" m nrhs n a lda
       (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                             work-%offset%)
```

```
        b ldb$
        (f2cl-lib:array-slice work-%data% double-float ((+ mn 1))
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-sub lwork mn) info)
     (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
     (when var-2 (setf m var-2))
     (when var-3 (setf nrhs var-3))
     (when var-4 (setf n var-4))
     (when var-6 (setf lda var-6))
     (when var-9 (setf ldb$ var-9))
     (when var-12 (setf info var-12)))
   (setf scllen m))))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (dgelqf m n a lda
      (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float ((+ mn 1))
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-sub lwork mn) info)
   (declare (ignore var-2 var-4 var-5 var-6))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf info var-7))
 (cond
  ((not tpsd)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (dtrtrs "Lower" "No transpose" "Non-unit" m nrhs a lda b ldb$
        info)
     (declare (ignore var-0 var-1 var-2 var-5 var-7))
     (when var-3 (setf m var-3))
     (when var-4 (setf nrhs var-4))
     (when var-6 (setf lda var-6))
     (when var-8 (setf ldb$ var-8))
     (when var-9 (setf info var-9)))
   (cond ((> info 0) (go end_label)))
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j nrhs) nil)
     (tagbody
       (f2cl-lib:fdo (i (f2cl-lib:int-add m 1) (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                 zero)
          label30))
      label40))
```

```
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (dormlq "Left" "Transpose" n nrhs m a lda
         (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                               work-%offset%)
         b ldb$
         (f2cl-lib:array-slice work-%data% double-float ((+ mn 1))
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-sub lwork mn) info)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf n var-2))
      (when var-3 (setf nrhs var-3))
      (when var-4 (setf m var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldb$ var-9))
      (when var-12 (setf info var-12)))
    (setf scllen n))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (dormlq "Left" "No transpose" n nrhs m a lda
         (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                               work-%offset%)
         b ldb$
         (f2cl-lib:array-slice work-%data% double-float ((+ mn 1))
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-sub lwork mn) info)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf n var-2))
      (when var-3 (setf nrhs var-3))
      (when var-4 (setf m var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldb$ var-9))
      (when var-12 (setf info var-12)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dtrtrs "Lower" "Transpose" "Non-unit" m nrhs a lda b ldb$
         info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7))
      (when var-3 (setf m var-3))
      (when var-4 (setf nrhs var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))
    (cond ((> info 0) (go end_label))) (setf scllen m)))))
(cond
 ((= iascl 1)
  (multiple-value-bind
```

```
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 anrm smlnum scllen nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf smlnum var-4))
      (when var-5 (setf scllen var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9))))
  ((= iascl 2)
   (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 anrm bignum scllen nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf bignum var-4))
      (when var-5 (setf scllen var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))))
 (cond
  ((= ibscl 1)
   (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 smlnum bnrm scllen nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf smlnum var-3))
      (when var-4 (setf bnrm var-4))
      (when var-5 (setf scllen var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9))))
  ((= ibscl 2)
   (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 bignum bnrm scllen nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bignum var-3))
      (when var-4 (setf bnrm var-4))
      (when var-5 (setf scllen var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))))
label50
 (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (f2cl-lib:dble wsize))
 (go end_label)
end_label
 (return (values trans m n nrhs nil lda nil ldb$ nil nil info))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgels fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$ nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::dgeqrf
                                              fortran-to-lisp::dgelqf))))
```

———-

20.2 dgelsd

20.2.1 Axiom unit tests

— dgelsd.input —

```
)set break resume
)sys rm -f dgelsd.output
)spool dgelsd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

20.2.2 Axiom help page

— dgelsd.help —

```
==================================================================
dgelsd examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgelsd (M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK, WORK,
           LWORK, IWORK, INFO)
            DGELSD computes the minimum-norm solution to a linear least
           squares problem for GE matrices

Function/Subroutine Documentation
   subroutine dgelsd (integer M, integer N, integer NRHS, double precision,
       dimension( lda, * ) A, integer LDA, double precision, dimension( ldb, *
       ) B, integer LDB, double precision, dimension( * ) S, double precision
       RCOND, integer RANK, double precision, dimension( * ) WORK, integer
       LWORK, integer, dimension( * ) IWORK, integer INFO)
        DGELSD computes the minimum-norm solution to a linear least squares
       problem for GE matrices

Purpose:
   DGELSD computes the minimum-norm solution to a real linear least
   squares problem:
      minimize 2-norm(| b - A*x |)
   using the singular value decomposition (SVD) of A. A is an M-by-N
   matrix which may be rank-deficient.

   Several right hand side vectors b and solution vectors x can be
   handled in a single call; they are stored as the columns of the
   M-by-NRHS right hand side matrix B and the N-by-NRHS solution matrix X.

   The problem is solved in three steps:
   (1) Reduce the coefficient matrix A to bidiagonal form with
       Householder transformations, reducing the original problem
       into a "bidiagonal least squares problem" (BLS)
   (2) Solve the BLS using a divide and conquer approach.
   (3) Apply back all the Householder tranformations to solve
       the original least squares problem.
```

```
  The effective rank of A is determined by treating as zero those
  singular values which are less than RCOND times the largest singular
  value.

  The divide and conquer algorithm makes very mild assumptions about
  floating point arithmetic. It will work on machines with a guard
  digit in add/subtract, or on those binary machines without guard
  digits which subtract like the Cray X-MP, Cray Y-MP, Cray C-90, or
  Cray-2. It could conceivably fail on hexadecimal or decimal machines
  without guard digits, but we know of none.

Parameters:

 M
   M is INTEGER
   The number of rows of A. M >= 0.

 N
   N is INTEGER
   The number of columns of A. N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X. NRHS >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, A has been destroyed.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 B
   B is DOUBLE PRECISION array, dimension (LDB,NRHS)
   On entry, the M-by-NRHS right hand side matrix B.
   On exit, B is overwritten by the N-by-NRHS solution
   matrix X.  If m >= n and RANK = n, the residual
   sum-of-squares for the solution in the i-th column is given
   by the sum of squares of elements n+1:m in that column.

 LDB
   LDB is INTEGER
   The leading dimension of the array B. LDB >= max(1,max(M,N)).

 S
   S is DOUBLE PRECISION array, dimension (min(M,N))
```

```
  The singular values of A in decreasing order.
  The condition number of A in the 2-norm = S(1)/S(min(m,n)).

RCOND
  RCOND is DOUBLE PRECISION
  RCOND is used to determine the effective rank of A.
  Singular values S(i) <= RCOND*S(1) are treated as zero.
  If RCOND < 0, machine precision is used instead.

RANK
  RANK is INTEGER
  The effective rank of A, i.e., the number of singular values
  which are greater than RCOND*S(1).

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK must be at least 1.
  The exact minimum amount of workspace needed depends on M,
  N and NRHS. As long as LWORK is at least
  12*N + 2*N*SMLSIZ + 8*N*NLVL + N*NRHS + (SMLSIZ+1)**2,
  if M is greater than or equal to N or
  12*M + 2*M*SMLSIZ + 8*M*NLVL + M*NRHS + (SMLSIZ+1)**2,
  if M is less than N, the code will execute correctly.
  SMLSIZ is returned by ILAENV and is equal to the maximum
  size of the subproblems at the bottom of the computation
  tree (usually about 25), and
  NLVL = MAX( 0, INT( LOG_2( MIN( M,N )/(SMLSIZ+1) ) ) + 1 )
  For good performance, LWORK should generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (MAX(1,LIWORK))
  LIWORK >= max(1, 3 * MINMN * NLVL + 11 * MINMN),
  where MINMN = MIN( M,N ).
  On exit, if INFO = 0, IWORK(1) returns the minimum LIWORK.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  the algorithm for computing the SVD failed to converge;
        if INFO = i, i off-diagonal elements of an intermediate
```

```
bidiagonal form did not converge to zero.
```

———-

20.2.3 fortran code

— **dgelsd.f** —

```
*  =====================================================================
      SUBROUTINE DGELSD( M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK,
     $                   WORK, LWORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDB, LWORK, M, N, NRHS, RANK
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * ), S( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE, TWO
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0, TWO = 2.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            IASCL, IBSCL, IE, IL, ITAU, ITAUP, ITAUQ,
     $                   LDWORK, LIWORK, MAXMN, MAXWRK, MINMN, MINWRK,
     $                   MM, MNTHR, NLVL, NWORK, SMLSIZ, WLALSD
      DOUBLE PRECISION   ANRM, BIGNUM, BNRM, EPS, SFMIN, SMLNUM
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEBRD, DGELQF, DGEQRF, DLABAD, DLACPY, DLALSD,
     $                   DLASCL, DLASET, DORMBR, DORMLQ, DORMQR, XERBLA
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           ILAENV, DLAMCH, DLANGE
```

```
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, INT, LOG, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      MINMN = MIN( M, N )
      MAXMN = MAX( M, N )
      MNTHR = ILAENV( 6, 'DGELSD', ' ', M, N, NRHS, -1 )
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, MAXMN ) ) THEN
         INFO = -7
      END IF
*
      SMLSIZ = ILAENV( 9, 'DGELSD', ' ', 0, 0, 0, 0 )
*
*     Compute workspace.
*     (Note: Comments in the code beginning "Workspace:" describe the
*     minimal amount of workspace needed at that point in the code,
*     as well as the preferred amount for good performance.
*     NB refers to the optimal block size for the immediately
*     following subroutine, as returned by ILAENV.)
*
      MINWRK = 1
      LIWORK = 1
      MINMN = MAX( 1, MINMN )
      NLVL = MAX( INT( LOG( DBLE( MINMN ) / DBLE( SMLSIZ+1 ) ) /
     $       LOG( TWO ) ) + 1, 0 )
*
      IF( INFO.EQ.0 ) THEN
         MAXWRK = 0
         LIWORK = 3*MINMN*NLVL + 11*MINMN
         MM = M
         IF( M.GE.N .AND. M.GE.MNTHR ) THEN
*
*           Path 1a - overdetermined, with many more rows than columns.
*
            MM = N
            MAXWRK = MAX( MAXWRK, N+N*ILAENV( 1, 'DGEQRF', ' ', M, N,
```

```
     $               -1, -1 ) )
            MAXWRK = MAX( MAXWRK, N+NRHS*
     $               ILAENV( 1, 'DORMQR', 'LT', M, NRHS, N, -1 ) )
         END IF
         IF( M.GE.N ) THEN
*
*           Path 1 - overdetermined or exactly determined.
*
            MAXWRK = MAX( MAXWRK, 3*N+( MM+N )*
     $               ILAENV( 1, 'DGEBRD', ' ', MM, N, -1, -1 ) )
            MAXWRK = MAX( MAXWRK, 3*N+NRHS*
     $               ILAENV( 1, 'DORMBR', 'QLT', MM, NRHS, N, -1 ) )
            MAXWRK = MAX( MAXWRK, 3*N+( N-1 )*
     $               ILAENV( 1, 'DORMBR', 'PLN', N, NRHS, N, -1 ) )
            WLALSD = 9*N+2*N*SMLSIZ+8*N*NLVL+N*NRHS+(SMLSIZ+1)**2
            MAXWRK = MAX( MAXWRK, 3*N+WLALSD )
            MINWRK = MAX( 3*N+MM, 3*N+NRHS, 3*N+WLALSD )
         END IF
         IF( N.GT.M ) THEN
            WLALSD = 9*M+2*M*SMLSIZ+8*M*NLVL+M*NRHS+(SMLSIZ+1)**2
            IF( N.GE.MNTHR ) THEN
*
*              Path 2a - underdetermined, with many more columns
*              than rows.
*
               MAXWRK = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
               MAXWRK = MAX( MAXWRK, M*M+4*M+2*M*
     $                  ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
               MAXWRK = MAX( MAXWRK, M*M+4*M+NRHS*
     $                  ILAENV( 1, 'DORMBR', 'QLT', M, NRHS, M, -1 ) )
               MAXWRK = MAX( MAXWRK, M*M+4*M+( M-1 )*
     $                  ILAENV( 1, 'DORMBR', 'PLN', M, NRHS, M, -1 ) )
               IF( NRHS.GT.1 ) THEN
                  MAXWRK = MAX( MAXWRK, M*M+M+M*NRHS )
               ELSE
                  MAXWRK = MAX( MAXWRK, M*M+2*M )
               END IF
               MAXWRK = MAX( MAXWRK, M+NRHS*
     $                  ILAENV( 1, 'DORMLQ', 'LT', N, NRHS, M, -1 ) )
               MAXWRK = MAX( MAXWRK, M*M+4*M+WLALSD )
!     XXX: Ensure the Path 2a case below is triggered.  The workspace
!     calculation should use queries for all routines eventually.
               MAXWRK = MAX( MAXWRK,
     $              4*M+M*M+MAX( M, 2*M-4, NRHS, N-3*M ) )
            ELSE
*
*              Path 2 - remaining underdetermined cases.
*
               MAXWRK = 3*M + ( N+M )*ILAENV( 1, 'DGEBRD', ' ', M, N,
     $                  -1, -1 )
```

```
            MAXWRK = MAX( MAXWRK, 3*M+NRHS*
     $                ILAENV( 1, 'DORMBR', 'QLT', M, NRHS, N, -1 ) )
            MAXWRK = MAX( MAXWRK, 3*M+M*
     $                ILAENV( 1, 'DORMBR', 'PLN', N, NRHS, M, -1 ) )
            MAXWRK = MAX( MAXWRK, 3*M+WLALSD )
         END IF
         MINWRK = MAX( 3*M+NRHS, 3*M+M, 3*M+WLALSD )
      END IF
      MINWRK = MIN( MINWRK, MAXWRK )
      WORK( 1 ) = MAXWRK
      IWORK( 1 ) = LIWORK

      IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
         INFO = -12
      END IF
   END IF
*
   IF( INFO.NE.0 ) THEN
      CALL XERBLA( 'DGELSD', -INFO )
      RETURN
   ELSE IF( LQUERY ) THEN
      GO TO 10
   END IF
*
*  Quick return if possible.
*
   IF( M.EQ.0 .OR. N.EQ.0 ) THEN
      RANK = 0
      RETURN
   END IF
*
*  Get machine parameters.
*
   EPS = DLAMCH( 'P' )
   SFMIN = DLAMCH( 'S' )
   SMLNUM = SFMIN / EPS
   BIGNUM = ONE / SMLNUM
   CALL DLABAD( SMLNUM, BIGNUM )
*
*  Scale A if max entry outside range [SMLNUM,BIGNUM].
*
   ANRM = DLANGE( 'M', M, N, A, LDA, WORK )
   IASCL = 0
   IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
*
*     Scale matrix norm up to SMLNUM.
*
      CALL DLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
      IASCL = 1
   ELSE IF( ANRM.GT.BIGNUM ) THEN
```

```
*
*        Scale matrix norm down to BIGNUM.
*
         CALL DLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
         IASCL = 2
      ELSE IF( ANRM.EQ.ZERO ) THEN
*
*        Matrix all zero. Return zero solution.
*
         CALL DLASET( 'F', MAX( M, N ), NRHS, ZERO, ZERO, B, LDB )
         CALL DLASET( 'F', MINMN, 1, ZERO, ZERO, S, 1 )
         RANK = 0
         GO TO 10
      END IF
*
*     Scale B if max entry outside range [SMLNUM,BIGNUM].
*
      BNRM = DLANGE( 'M', M, NRHS, B, LDB, WORK )
      IBSCL = 0
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM.
*
         CALL DLASCL( 'G', 0, 0, BNRM, SMLNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 1
      ELSE IF( BNRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM.
*
         CALL DLASCL( 'G', 0, 0, BNRM, BIGNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 2
      END IF
*
*     If M < N make sure certain entries of B are zero.
*
      IF( M.LT.N )
     $   CALL DLASET( 'F', N-M, NRHS, ZERO, ZERO, B( M+1, 1 ), LDB )
*
*     Overdetermined case.
*
      IF( M.GE.N ) THEN
*
*        Path 1 - overdetermined or exactly determined.
*
         MM = M
         IF( M.GE.MNTHR ) THEN
*
*           Path 1a - overdetermined, with many more rows than columns.
*
            MM = N
```

```
            ITAU = 1
            NWORK = ITAU + N
*
*           Compute A=Q*R.
*           (Workspace: need 2*N, prefer N+N*NB)
*
            CALL DGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                   LWORK-NWORK+1, INFO )
*
*           Multiply B by transpose(Q).
*           (Workspace: need N+NRHS, prefer N+NRHS*NB)
*
            CALL DORMQR( 'L', 'T', M, NRHS, N, A, LDA, WORK( ITAU ), B,
     $                   LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
*           Zero out below R.
*
            IF( N.GT.1 ) THEN
               CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ), LDA )
            END IF
         END IF
*
         IE = 1
         ITAUQ = IE + N
         ITAUP = ITAUQ + N
         NWORK = ITAUP + N
*
*        Bidiagonalize R in A.
*        (Workspace: need 3*N+MM, prefer 3*N+(MM+N)*NB)
*
         CALL DGEBRD( MM, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors of R.
*        (Workspace: need 3*N+NRHS, prefer 3*N+NRHS*NB)
*
         CALL DORMBR( 'Q', 'L', 'T', MM, NRHS, N, A, LDA, WORK( ITAUQ ),
     $                B, LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
*        Solve the bidiagonal least squares problem.
*
         CALL DLALSD( 'U', SMLSIZ, N, NRHS, S, WORK( IE ), B, LDB,
     $                RCOND, RANK, WORK( NWORK ), IWORK, INFO )
         IF( INFO.NE.0 ) THEN
            GO TO 10
         END IF
*
*        Multiply B by right bidiagonalizing vectors of R.
*
```

```
         CALL DORMBR( 'P', 'L', 'N', N, NRHS, N, A, LDA, WORK( ITAUP ),
     $                B, LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
      ELSE IF( N.GE.MNTHR .AND. LWORK.GE.4*M+M*M+
     $         MAX( M, 2*M-4, NRHS, N-3*M, WLALSD ) ) THEN
*
*        Path 2a - underdetermined, with many more columns than rows
*        and sufficient workspace for an efficient algorithm.
*
         LDWORK = M
         IF( LWORK.GE.MAX( 4*M+M*LDA+MAX( M, 2*M-4, NRHS, N-3*M ),
     $       M*LDA+M+M*NRHS, 4*M+M*LDA+WLALSD ) )LDWORK = LDA
         ITAU = 1
         NWORK = M + 1
*
*        Compute A=L*Q.
*        (Workspace: need 2*M, prefer M+M*NB)
*
         CALL DGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                LWORK-NWORK+1, INFO )
         IL = NWORK
*
*        Copy L to WORK(IL), zeroing out above its diagonal.
*
         CALL DLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWORK )
         CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, WORK( IL+LDWORK ),
     $                LDWORK )
         IE = IL + LDWORK*M
         ITAUQ = IE + M
         ITAUP = ITAUQ + M
         NWORK = ITAUP + M
*
*        Bidiagonalize L in WORK(IL).
*        (Workspace: need M*M+5*M, prefer M*M+4*M+2*M*NB)
*
         CALL DGEBRD( M, M, WORK( IL ), LDWORK, S, WORK( IE ),
     $                WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                LWORK-NWORK+1, INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors of L.
*        (Workspace: need M*M+4*M+NRHS, prefer M*M+4*M+NRHS*NB)
*
         CALL DORMBR( 'Q', 'L', 'T', M, NRHS, M, WORK( IL ), LDWORK,
     $                WORK( ITAUQ ), B, LDB, WORK( NWORK ),
     $                LWORK-NWORK+1, INFO )
*
*        Solve the bidiagonal least squares problem.
*
         CALL DLALSD( 'U', SMLSIZ, M, NRHS, S, WORK( IE ), B, LDB,
     $                RCOND, RANK, WORK( NWORK ), IWORK, INFO )
```

```
         IF( INFO.NE.0 ) THEN
            GO TO 10
         END IF
*
*        Multiply B by right bidiagonalizing vectors of L.
*
         CALL DORMBR( 'P', 'L', 'N', M, NRHS, M, WORK( IL ), LDWORK,
     $                WORK( ITAUP ), B, LDB, WORK( NWORK ),
     $                LWORK-NWORK+1, INFO )
*
*        Zero out below first M rows of B.
*
         CALL DLASET( 'F', N-M, NRHS, ZERO, ZERO, B( M+1, 1 ), LDB )
         NWORK = ITAU + M
*
*        Multiply transpose(Q) by B.
*        (Workspace: need M+NRHS, prefer M+NRHS*NB)
*
         CALL DORMLQ( 'L', 'T', N, NRHS, M, A, LDA, WORK( ITAU ), B,
     $                LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
      ELSE
*
*        Path 2 - remaining underdetermined cases.
*
         IE = 1
         ITAUQ = IE + M
         ITAUP = ITAUQ + M
         NWORK = ITAUP + M
*
*        Bidiagonalize A.
*        (Workspace: need 3*M+N, prefer 3*M+(M+N)*NB)
*
         CALL DGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors.
*        (Workspace: need 3*M+NRHS, prefer 3*M+NRHS*NB)
*
         CALL DORMBR( 'Q', 'L', 'T', M, NRHS, N, A, LDA, WORK( ITAUQ ),
     $                B, LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
*        Solve the bidiagonal least squares problem.
*
         CALL DLALSD( 'L', SMLSIZ, M, NRHS, S, WORK( IE ), B, LDB,
     $                RCOND, RANK, WORK( NWORK ), IWORK, INFO )
         IF( INFO.NE.0 ) THEN
            GO TO 10
         END IF
```

```
*
*        Multiply B by right bidiagonalizing vectors of A.
*
         CALL DORMBR( 'P', 'L', 'N', N, NRHS, M, A, LDA, WORK( ITAUP ),
     $                B, LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
      END IF
*
*     Undo scaling.
*
      IF( IASCL.EQ.1 ) THEN
         CALL DLASCL( 'G', 0, 0, ANRM, SMLNUM, N, NRHS, B, LDB, INFO )
         CALL DLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
     $                INFO )
      ELSE IF( IASCL.EQ.2 ) THEN
         CALL DLASCL( 'G', 0, 0, ANRM, BIGNUM, N, NRHS, B, LDB, INFO )
         CALL DLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                INFO )
      END IF
      IF( IBSCL.EQ.1 ) THEN
         CALL DLASCL( 'G', 0, 0, SMLNUM, BNRM, N, NRHS, B, LDB, INFO )
      ELSE IF( IBSCL.EQ.2 ) THEN
         CALL DLASCL( 'G', 0, 0, BIGNUM, BNRM, N, NRHS, B, LDB, INFO )
      END IF
*
   10 CONTINUE
      WORK( 1 ) = MAXWRK
      IWORK( 1 ) = LIWORK
      RETURN
*
*     End of DGELSD
*
      END
```

———-

20.2.4 lisp code

— dgelsd.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0) (two 2.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (ignorable zero one two))
  (defun dgelsd (m n nrhs a lda b ldb$ s rcond rank work lwork iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (double-float) rcond)
```

```
          (type (array double-float (*)) work s b a)
          (type (f2cl-lib:integer4) info lwork rank ldb$ lda nrhs n m))
(f2cl-lib:with-multi-array-data
    ((a double-float a-%data% a-%offset%)
     (b double-float b-%data% b-%offset%)
     (s double-float s-%data% s-%offset%)
     (work double-float work-%data% work-%offset%)
     (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((anrm 0.0d0) (bignum 0.0d0) (bnrm 0.0d0) (eps 0.0d0) (sfmin 0.0d0)
         (smlnum 0.0d0) (iascl 0) (ibscl 0) (ie 0) (il 0) (itau 0)
         (itaup 0) (itauq 0) (ldwork 0) (liwork 0) (maxmn 0) (maxwrk 0)
         (minmn 0) (minwrk 0) (mm 0) (mnthr 0) (nlvl 0) (nwork 0)
         (smlsiz 0) (wlalsd 0) (lquery nil))
    (declare (type (double-float) anrm bignum bnrm eps sfmin smlnum)
             (type (f2cl-lib:integer4) iascl ibscl ie il itau itaup itauq
              ldwork liwork maxmn maxwrk minmn minwrk mm mnthr nlvl nwork
              smlsiz wlalsd)
             (type f2cl-lib:logical lquery))
    (setf info 0)
    (setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
    (setf maxmn (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
    (setf mnthr
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 6 "DGELSD" " " m n nrhs -1)
              (declare (ignore var-0 var-1 var-2 var-6))
              (when var-3 (setf m var-3))
              (when var-4 (setf n var-4))
              (when var-5 (setf nrhs var-5))
              ret-val))
    (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
    (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
          ((< nrhs 0) (setf info -3))
          ((< lda
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
           (setf info -5))
          ((< ldb$
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 maxmn)))
           (setf info -7)))
    (setf smlsiz (ilaenv 9 "DGELSD" " " 0 0 0 0))
    (setf minwrk 1)
    (setf liwork 1)
    (setf minmn
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 minmn)))
    (setf nlvl
            (max
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add
                   (f2cl-lib:int
```

```
                    (/
                     (f2cl-lib:flog
                      (/ (f2cl-lib:dble minmn)
                         (f2cl-lib:dble (f2cl-lib:int-add smlsiz 1))))
                     (f2cl-lib:flog two)))
                   1))
            (the f2cl-lib:integer4 0)))
    (cond
     ((= info 0) (setf maxwrk 0)
      (setf liwork
              (f2cl-lib:int-add (f2cl-lib:int-mul 3 minmn nlvl)
                                (f2cl-lib:int-mul 11 minmn)))
      (setf mm m)
      (cond
       ((and (>= m n) (>= m mnthr)) (setf mm n)
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add n
                                            (f2cl-lib:int-mul n
                                                              (multiple-value-bind
                                                                  (ret-val
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-3
                                                                   var-4
                                                                   var-5
                                                                   var-6)
                                                                  (ilaenv 1
                                                                   "DGEQRF"
                                                                   " " m n
                                                                   -1 -1)
                                                                (declare
                                                                 (ignore
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-5
                                                                  var-6))
                                                                (when var-3
                                                                  (setf m
                                                                          var-3))
                                                                (when var-4
                                                                  (setf n
                                                                          var-4))
                                                                ret-val))))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
```

```
                    (f2cl-lib:int-add n
                                      (f2cl-lib:int-mul nrhs
                                                        (multiple-value-bind
                                                            (ret-val
                                                             var-0
                                                             var-1
                                                             var-2
                                                             var-3
                                                             var-4
                                                             var-5
                                                             var-6)
                                                            (ilaenv 1
                                                             "DORMQR"
                                                             "LT" m
                                                             nrhs n
                                                             -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-6))
                                                          (when var-3
                                                            (setf m
                                                                  var-3))
                                                          (when var-4
                                                            (setf nrhs
                                                                  var-4))
                                                          (when var-5
                                                            (setf n
                                                                  var-5))
                                                          ret-val))))))))
(cond
 ((>= m n)
  (setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul
                                     (f2cl-lib:int-add mm n)
                                     (multiple-value-bind
                                         (ret-val var-0 var-1 var-2
                                          var-3 var-4 var-5 var-6)
                                         (ilaenv 1 "DGEBRD" " " mm
                                          n -1 -1)
                                       (declare
                                        (ignore var-0 var-1 var-2
                                         var-5 var-6))
                                       (when var-3 (setf mm var-3))
                                       (when var-4 (setf n var-4))
```

```
                                        ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul nrhs
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv 1
                                                           "DORMBR"
                                                           "QLT" mm
                                                           nrhs n
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (when var-3
                                                          (setf mm
                                                                  var-3))
                                                        (when var-4
                                                          (setf nrhs
                                                                  var-4))
                                                        (when var-5
                                                          (setf n
                                                                  var-5))
                                                        ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul
                                     (f2cl-lib:int-sub n 1)
                                     (multiple-value-bind
                                         (ret-val var-0 var-1 var-2
                                          var-3 var-4 var-5 var-6)
                                         (ilaenv 1 "DORMBR" "PLN" n
                                          nrhs n -1)
                                       (declare
                                        (ignore var-0 var-1 var-2
                                         var-6))
```

```
                                          (when var-3 (setf n var-3))
                                          (when var-4
                                            (setf nrhs var-4))
                                          (when var-5 (setf n var-5))
                                          ret-val))))))
  (setf wlalsd
          (f2cl-lib:int-add (f2cl-lib:int-mul 9 n)
                            (f2cl-lib:int-mul 2 n smlsiz)
                            (f2cl-lib:int-mul 8 n nlvl)
                            (f2cl-lib:int-mul n nrhs)
                            (expt (f2cl-lib:int-add smlsiz 1) 2)))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                      wlalsd))))
  (setf minwrk
          (max
           (the f2cl-lib:integer4
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) mm))
           (the f2cl-lib:integer4
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) nrhs))
           (the f2cl-lib:integer4
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) wlalsd))))))
(cond
 ((> n m)
  (setf wlalsd
          (f2cl-lib:int-add (f2cl-lib:int-mul 9 m)
                            (f2cl-lib:int-mul 2 m smlsiz)
                            (f2cl-lib:int-mul 8 m nlvl)
                            (f2cl-lib:int-mul m nrhs)
                            (expt (f2cl-lib:int-add smlsiz 1) 2)))
  (cond
   ((>= n mnthr)
    (setf maxwrk
            (f2cl-lib:int-add m
                              (f2cl-lib:int-mul m
                                                (multiple-value-bind
                                                    (ret-val var-0
                                                     var-1 var-2
                                                     var-3 var-4
                                                     var-5 var-6)
                                                    (ilaenv 1
                                                     "DGELQF" " " m n
                                                     -1 -1)
                                                  (declare
                                                   (ignore var-0
                                                    var-1 var-2 var-5
                                                    var-6))
                                                  (when var-3
```

```
                                                  (setf m var-3))
                                                (when var-4
                                                  (setf n var-4))
                                                ret-val))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                    (f2cl-lib:int-mul 4 m)
                                    (f2cl-lib:int-mul 2 m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "DGEBRD"
                                                           " " m
                                                           m -1
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-5
                                                          var-6))
                                                        (when
                                                            var-3
                                                          (setf m
                                                                var-3))
                                                        (when
                                                            var-4
                                                          (setf m
                                                                var-4))
                                                        ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                    (f2cl-lib:int-mul 4 m)
                                    (f2cl-lib:int-mul nrhs
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
```

```
                                                                    var-1
                                                                    var-2
                                                                    var-3
                                                                    var-4
                                                                    var-5
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "DORMBR"
                                                                    "QLT"
                                                                    m nrhs
                                                                    m -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf nrhs
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf m
                                                                            var-5))
                                                                  ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                    (f2cl-lib:int-mul 4 m)
                                    (f2cl-lib:int-mul
                                     (f2cl-lib:int-sub m 1)
                                     (multiple-value-bind
                                         (ret-val var-0 var-1
                                          var-2 var-3 var-4 var-5
                                          var-6)
                                         (ilaenv 1 "DORMBR" "PLN"
                                          m nrhs m -1)
                                       (declare
                                        (ignore var-0 var-1 var-2
                                         var-6))
                                       (when var-3
                                         (setf m var-3))
                                       (when var-4
```

```
                                          (setf nrhs var-4))
                                         (when var-5
                                          (setf m var-5))
                                         ret-val))))))
(cond
 ((> nrhs 1)
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul m m) m
                                      (f2cl-lib:int-mul m
                                                        nrhs))))))
 (t
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                      (f2cl-lib:int-mul 2
                                                        m)))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add m
                                    (f2cl-lib:int-mul nrhs
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "DORMLQ"
                                                           "LT" n
                                                           nrhs m
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (when
                                                            var-3
                                                          (setf n
                                                                  var-3))
                                                        (when
```

```
                                                                    var-4
                                                                  (setf nrhs
                                                                          var-4))
                                                                (when
                                                                    var-5
                                                                  (setf m
                                                                          var-5))
                                                                ret-val))))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                            (f2cl-lib:int-mul 4 m)
                                            wlalsd))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 4 m)
                                            (f2cl-lib:int-mul m m)
                                            (max
                                             (the f2cl-lib:integer4 m)
                                             (the f2cl-lib:integer4
                                                  (f2cl-lib:int-sub
                                                   (f2cl-lib:int-mul 2 m)
                                                   4))
                                             (the f2cl-lib:integer4 nrhs)
                                             (the f2cl-lib:integer4
                                                  (f2cl-lib:int-sub n
                                                                    (f2cl-lib:int-mul
                                                                     3
                                                                     m)))))))))
       (t
        (setf maxwrk
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                  (f2cl-lib:int-mul
                                   (f2cl-lib:int-add n m)
                                   (multiple-value-bind
                                       (ret-val var-0 var-1 var-2 var-3
                                        var-4 var-5 var-6)
                                       (ilaenv 1 "DGEBRD" " " m n -1 -1)
                                     (declare
                                      (ignore var-0 var-1 var-2 var-5
                                       var-6))
                                     (when var-3 (setf m var-3))
                                     (when var-4 (setf n var-4))
                                     ret-val))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
```

```
                                        (f2cl-lib:int-mul nrhs
                                                          (multiple-value-bind
                                                              (ret-val
                                                               var-0
                                                               var-1
                                                               var-2
                                                               var-3
                                                               var-4
                                                               var-5
                                                               var-6)
                                                              (ilaenv
                                                               1
                                                               "DORMBR"
                                                               "QLT"
                                                               m nrhs
                                                               n -1)
                                                            (declare
                                                             (ignore
                                                              var-0
                                                              var-1
                                                              var-2
                                                              var-6))
                                                            (when
                                                                var-3
                                                              (setf m
                                                                      var-3))
                                                            (when
                                                                var-4
                                                              (setf nrhs
                                                                      var-4))
                                                            (when
                                                                var-5
                                                              (setf n
                                                                      var-5))
                                                            ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
```

```
                                                                    1
                                                                    "DORMBR"
                                                                    "PLN"
                                                                    n nrhs
                                                                    m -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf n
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf nrhs
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf m
                                                                            var-5))
                                                                  ret-val))))))
      (setf maxwrk
              (max (the f2cl-lib:integer4 maxwrk)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                          wlalsd))))))
    (setf minwrk
            (max
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) nrhs))
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) m))
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) wlalsd))))))
  (setf minwrk
          (min (the f2cl-lib:integer4 minwrk)
               (the f2cl-lib:integer4 maxwrk)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 maxwrk) 'double-float))
  (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) liwork)
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -12)))))
(cond
 ((/= info 0) (xerbla "DGELSD" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go label10)))
(cond ((or (= m 0) (= n 0)) (setf rank 0) (go end_label)))
(setf eps (dlamch "P"))
(setf sfmin (dlamch "S"))
```

```
(setf smlnum (/ sfmin eps))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "M" m n a lda work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iascl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 anrm smlnum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 1))
 ((> anrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 anrm bignum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 2))
 ((= anrm zero)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "F"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       zero zero b ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf zero var-3))
```

```
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldb$ var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "F" minmn 1 zero zero s 1)
    (declare (ignore var-0 var-2 var-5 var-6))
    (when var-1 (setf minmn var-1))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4)))
  (setf rank 0) (go label10)))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "M" m nrhs b ldb$ work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ibscl 0)
(cond
 ((and (> bnrm zero) (< bnrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 bnrm smlnum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 1))
 ((> bnrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 bnrm bignum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 2)))
(if (< m n)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "F" (f2cl-lib:int-sub n m) nrhs zero zero
         (f2cl-lib:array-slice b-%data% double-float ((+ m 1) 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
```

```
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf nrhs var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldb$ var-6))))
(cond
 ((>= m n) (setf mm m)
  (cond
   ((>= m mnthr) (setf mm n) (setf itau 1)
    (setf nwork (f2cl-lib:int-add itau n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (dgeqrf m n a lda
         (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                               work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (nwork) ((1 *))
                               work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
      (declare (ignore var-2 var-4 var-5 var-6))
      (setf m var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf info var-7))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (dormqr "L" "T" m nrhs n a lda
         (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                               work-%offset%)
         b ldb$
         (f2cl-lib:array-slice work-%data% double-float (nwork) ((1 *))
                               work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf nrhs var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldb$ var-9))
      (when var-12 (setf info var-12)))
    (cond
     ((> n 1)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
           zero zero
           (f2cl-lib:array-slice a-%data% double-float (2 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-5))
        (when var-3 (setf zero var-3))
```

```
      (when var-4 (setf zero var-4))
      (when var-6 (setf lda var-6)))))))
(setf ie 1) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf nwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgebrd mm n a lda s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (nwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf mm var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "Q" "L" "T" mm nrhs n a lda
     (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                           work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% double-float (nwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf mm var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dlalsd "U" smlsiz n nrhs s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     b ldb$ rcond rank
     (f2cl-lib:array-slice work-%data% double-float (nwork) ((1 *))
                           work-%offset%)
```

```
        iwork info)
      (declare (ignore var-0 var-4 var-5 var-6 var-10 var-11))
      (when var-1 (setf smlsiz var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf nrhs var-3))
      (when var-7 (setf ldb$ var-7))
      (when var-8 (setf rcond var-8))
      (when var-9 (setf rank var-9))
      (when var-12 (setf info var-12)))
    (cond ((/= info 0) (go label10)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (dormbr "P" "L" "N" n nrhs n a lda
         (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                               work-%offset%)
         b ldb$
         (f2cl-lib:array-slice work-%data% double-float (nwork) ((1 *))
                               work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
      (when var-3 (setf n var-3))
      (when var-4 (setf nrhs var-4))
      (when var-5 (setf n var-5))
      (when var-7 (setf lda var-7))
      (when var-10 (setf ldb$ var-10))
      (when var-13 (setf info var-13))))
  ((and (>= n mnthr)
        (>= lwork
            (f2cl-lib:int-add (f2cl-lib:int-mul 4 m)
                              (f2cl-lib:int-mul m m)
                              (max (the f2cl-lib:integer4 m)
                                   (the f2cl-lib:integer4
                                        (f2cl-lib:int-add
                                         (f2cl-lib:int-mul 2 m)
                                         (f2cl-lib:int-sub 4)))
                                   (the f2cl-lib:integer4 nrhs)
                                   (the f2cl-lib:integer4
                                        (f2cl-lib:int-add n
                                                          (f2cl-lib:int-mul
                                                           -1 3 m)))
                                   (the f2cl-lib:integer4 wlalsd)))))
    (setf ldwork m)
    (if (>= lwork
            (max
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 4 m)
                                    (f2cl-lib:int-mul m lda)
                                    (max (the f2cl-lib:integer4 m)
```

```
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-sub
                                              (f2cl-lib:int-mul 2 m) 4))
                                        (the f2cl-lib:integer4 nrhs)
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-sub n
                                                               (f2cl-lib:int-mul
                                                                3
                                                                m))))))
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul m lda) m
                                 (f2cl-lib:int-mul m nrhs)))
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 4 m)
                                 (f2cl-lib:int-mul m lda) wlalsd))))
    (setf ldwork lda))
(setf itau 1) (setf nwork (f2cl-lib:int-add m 1))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dgelqf m n a lda
     (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (nwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-7))
(setf il nwork)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                           work-%offset%)
     ldwork)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwork var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1) zero
     zero
     (f2cl-lib:array-slice work-%data% double-float ((+ il ldwork))
                           ((1 *)) work-%offset%)
     ldwork)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
```

```
  (when var-6 (setf ldwork var-6)))
(setf ie (f2cl-lib:int-add il (f2cl-lib:int-mul ldwork m)))
(setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf nwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgebrd m m
     (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                           work-%offset%)
     ldwork s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (nwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwork var-3)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "Q" "L" "T" m nrhs m
     (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                           work-%offset%)
     ldwork
     (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                           work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% double-float (nwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwork var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
```

```
    (dlalsd "U" smlsiz m nrhs s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     b ldb$ rcond rank
     (f2cl-lib:array-slice work-%data% double-float (nwork) ((1 *))
                           work-%offset%)
     iwork info)
  (declare (ignore var-0 var-4 var-5 var-6 var-10 var-11))
  (when var-1 (setf smlsiz var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf nrhs var-3))
  (when var-7 (setf ldb$ var-7))
  (when var-8 (setf rcond var-8))
  (when var-9 (setf rank var-9))
  (when var-12 (setf info var-12)))
(cond ((/= info 0) (go label10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "P" "L" "N" m nrhs m
     (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                           work-%offset%)
     ldwork
     (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                           work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% double-float (nwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwork var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "F" (f2cl-lib:int-sub n m) nrhs zero zero
     (f2cl-lib:array-slice b-%data% double-float ((+ m 1) 1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$)
  (declare (ignore var-0 var-1 var-5))
  (when var-2 (setf nrhs var-2))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldb$ var-6)))
(setf nwork (f2cl-lib:int-add itau m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
        var-10 var-11 var-12)
       (dormlq "L" "T" n nrhs m a lda
        (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                              work-%offset%)
        b ldb$
        (f2cl-lib:array-slice work-%data% double-float (nwork) ((1 *))
                              work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
     (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
     (when var-2 (setf n var-2))
     (when var-3 (setf nrhs var-3))
     (when var-4 (setf m var-4))
     (when var-6 (setf lda var-6))
     (when var-9 (setf ldb$ var-9))
     (when var-12 (setf info var-12))))
  (t (setf ie 1) (setf itauq (f2cl-lib:int-add ie m))
   (setf itaup (f2cl-lib:int-add itauq m))
   (setf nwork (f2cl-lib:int-add itaup m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (dgebrd m n a lda s
        (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (nwork) ((1 *))
                              work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
     (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf info var-10))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13)
       (dormbr "Q" "L" "T" m nrhs n a lda
        (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                              work-%offset%)
        b ldb$
        (f2cl-lib:array-slice work-%data% double-float (nwork) ((1 *))
                              work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
     (declare
      (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
     (when var-3 (setf m var-3))
     (when var-4 (setf nrhs var-4))
```

```
    (when var-5 (setf n var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldb$ var-10))
    (when var-13 (setf info var-13)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dlalsd "L" smlsiz m nrhs s
       (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                             work-%offset%)
       b ldb$ rcond rank
       (f2cl-lib:array-slice work-%data% double-float (nwork) ((1 *))
                             work-%offset%)
       iwork info)
    (declare (ignore var-0 var-4 var-5 var-6 var-10 var-11))
    (when var-1 (setf smlsiz var-1))
    (when var-2 (setf m var-2))
    (when var-3 (setf nrhs var-3))
    (when var-7 (setf ldb$ var-7))
    (when var-8 (setf rcond var-8))
    (when var-9 (setf rank var-9))
    (when var-12 (setf info var-12)))
  (cond ((/= info 0) (go label10)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dormbr "P" "L" "N" n nrhs m a lda
       (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                             work-%offset%)
       b ldb$
       (f2cl-lib:array-slice work-%data% double-float (nwork) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf n var-3))
    (when var-4 (setf nrhs var-4))
    (when var-5 (setf m var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldb$ var-10))
    (when var-13 (setf info var-13)))))
(cond
 ((= iascl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 anrm smlnum n nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf n var-5))
```

```
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 smlnum anrm minmn 1 s minmn info)
      (declare (ignore var-0 var-1 var-2 var-6 var-7))
      (when var-3 (setf smlnum var-3))
      (when var-4 (setf anrm var-4))
      (when var-5 (setf minmn var-5))
      (when var-8 (setf minmn var-8))
      (when var-9 (setf info var-9))))
   ((= iascl 2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 anrm bignum n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf bignum var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 bignum anrm minmn 1 s minmn info)
      (declare (ignore var-0 var-1 var-2 var-6 var-7))
      (when var-3 (setf bignum var-3))
      (when var-4 (setf anrm var-4))
      (when var-5 (setf minmn var-5))
      (when var-8 (setf minmn var-8))
      (when var-9 (setf info var-9)))))
  (cond
   ((= ibscl 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 smlnum bnrm n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf smlnum var-3))
      (when var-4 (setf bnrm var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9))))
   ((= ibscl 2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 bignum bnrm n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bignum var-3))
```

```
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf n var-5))
            (when var-6 (setf nrhs var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf info var-9)))))
       label10
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 maxwrk) 'double-float))
        (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) liwork)
        (go end_label)
       end_label
        (return
         (values m n nrhs nil lda nil ldb$ nil rcond rank nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgelsd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (double-float)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$ nil
                                              fortran-to-lisp::rcond
                                              fortran-to-lisp::rank nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::dgeqrf
                                              fortran-to-lisp::dgelqf
                                              fortran-to-lisp::dgebrd))))
```

———-

20.3 dgelss

20.3.1 Axiom unit tests

— dgelss.input —

```
)set break resume
)sys rm -f dgelss.output
)spool dgelss.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

20.3.2 Axiom help page

— dgelss.help —

```
====================================================================
dgelss examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgelss (M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK, WORK,
           LWORK, INFO)
            DGELSS solves overdetermined or underdetermined systems for GE
           matrices

Function/Subroutine Documentation
   subroutine dgelss (integer M, integer N, integer NRHS, double precision,
       dimension( lda, * ) A, integer LDA, double precision, dimension( ldb, *
       ) B, integer LDB, double precision, dimension( * ) S, double precision
       RCOND, integer RANK, double precision, dimension( * ) WORK, integer
```

```
      LWORK, integer INFO)
       DGELSS solves overdetermined or underdetermined systems for GE
      matrices

Purpose:
   DGELSS computes the minimum norm solution to a real linear least
   squares problem:

   Minimize 2-norm(| b - A*x |).

   using the singular value decomposition (SVD) of A. A is an M-by-N
   matrix which may be rank-deficient.

   Several right hand side vectors b and solution vectors x can be
   handled in a single call; they are stored as the columns of the
   M-by-NRHS right hand side matrix B and the N-by-NRHS solution matrix X.

   The effective rank of A is determined by treating as zero those
   singular values which are less than RCOND times the largest singular value.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A. M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A. N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X. NRHS >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, the first min(m,n) rows of A are overwritten with
   its right singular vectors, stored rowwise.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 B
   B is DOUBLE PRECISION array, dimension (LDB,NRHS)
   On entry, the M-by-NRHS right hand side matrix B.
   On exit, B is overwritten by the N-by-NRHS solution
   matrix X.  If m >= n and RANK = n, the residual
```

```
  sum-of-squares for the solution in the i-th column is given
  by the sum of squares of elements n+1:m in that column.

LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= max(1,max(M,N)).

S
  S is DOUBLE PRECISION array, dimension (min(M,N))
  The singular values of A in decreasing order.
  The condition number of A in the 2-norm = S(1)/S(min(m,n)).

RCOND
  RCOND is DOUBLE PRECISION
  RCOND is used to determine the effective rank of A.
  Singular values S(i) <= RCOND*S(1) are treated as zero.
  If RCOND < 0, machine precision is used instead.

RANK
  RANK is INTEGER
  The effective rank of A, i.e., the number of singular values
  which are greater than RCOND*S(1).

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK >= 1, and also:
  LWORK >= 3*min(M,N) + max( 2*min(M,N), max(M,N), NRHS )
  For good performance, LWORK should generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  the algorithm for computing the SVD failed to converge;
        if INFO = i, i off-diagonal elements of an intermediate
        bidiagonal form did not converge to zero.
```

———-

20.3.3 fortran code

— dgelss.f —

```
*  =====================================================================
      SUBROUTINE DGELSS( M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK,
     $                   WORK, LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDB, LWORK, M, N, NRHS, RANK
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * ), S( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            BDSPAC, BL, CHUNK, I, IASCL, IBSCL, IE, IL,
     $                   ITAU, ITAUP, ITAUQ, IWORK, LDWORK, MAXMN,
     $                   MAXWRK, MINMN, MINWRK, MM, MNTHR
      INTEGER            LWORK_DGEQRF, LWORK_DORMQR, LWORK_DGEBRD,
     $                   LWORK_DORMBR, LWORK_DORGBR, LWORK_DORMLQ,
     $                   LWORK_DGELQF
      DOUBLE PRECISION   ANRM, BIGNUM, BNRM, EPS, SFMIN, SMLNUM, THR
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DBDSQR, DCOPY, DGEBRD, DGELQF, DGEMM, DGEMV,
     $                   DGEQRF, DLABAD, DLACPY, DLASCL, DLASET, DORGBR,
     $                   DORMBR, DORMLQ, DORMQR, DRSCL, XERBLA
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           ILAENV, DLAMCH, DLANGE
```

```
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      MINMN = MIN( M, N )
      MAXMN = MAX( M, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, MAXMN ) ) THEN
         INFO = -7
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = 1
         MAXWRK = 1
         IF( MINMN.GT.0 ) THEN
            MM = M
            MNTHR = ILAENV( 6, 'DGELSS', ' ', M, N, NRHS, -1 )
            IF( M.GE.N .AND. M.GE.MNTHR ) THEN
*
*              Path 1a - overdetermined, with many more rows than
*                        columns
*
*              Compute space needed for DGEQRF
               CALL DGEQRF( M, N, A, LDA, DUM(1), DUM(1), -1, INFO )
               LWORK_DGEQRF=DUM(1)
*              Compute space needed for DORMQR
               CALL DORMQR( 'L', 'T', M, NRHS, N, A, LDA, DUM(1), B,
     $                    LDB, DUM(1), -1, INFO )
               LWORK_DORMQR=DUM(1)
               MM = N
```

```
            MAXWRK = MAX( MAXWRK, N + LWORK_DGEQRF )
            MAXWRK = MAX( MAXWRK, N + LWORK_DORMQR )
         END IF
         IF( M.GE.N ) THEN
*
*           Path 1 - overdetermined or exactly determined
*
*           Compute workspace needed for DBDSQR
*
            BDSPAC = MAX( 1, 5*N )
*           Compute space needed for DGEBRD
            CALL DGEBRD( MM, N, A, LDA, S, DUM(1), DUM(1),
     $                   DUM(1), DUM(1), -1, INFO )
            LWORK_DGEBRD=DUM(1)
*           Compute space needed for DORMBR
            CALL DORMBR( 'Q', 'L', 'T', MM, NRHS, N, A, LDA, DUM(1),
     $             B, LDB, DUM(1), -1, INFO )
            LWORK_DORMBR=DUM(1)
*           Compute space needed for DORGBR
            CALL DORGBR( 'P', N, N, N, A, LDA, DUM(1),
     $               DUM(1), -1, INFO )
            LWORK_DORGBR=DUM(1)
*           Compute total workspace needed
            MAXWRK = MAX( MAXWRK, 3*N + LWORK_DGEBRD )
            MAXWRK = MAX( MAXWRK, 3*N + LWORK_DORMBR )
            MAXWRK = MAX( MAXWRK, 3*N + LWORK_DORGBR )
            MAXWRK = MAX( MAXWRK, BDSPAC )
            MAXWRK = MAX( MAXWRK, N*NRHS )
            MINWRK = MAX( 3*N + MM, 3*N + NRHS, BDSPAC )
            MAXWRK = MAX( MINWRK, MAXWRK )
         END IF
         IF( N.GT.M ) THEN
*
*           Compute workspace needed for DBDSQR
*
            BDSPAC = MAX( 1, 5*M )
            MINWRK = MAX( 3*M+NRHS, 3*M+N, BDSPAC )
            IF( N.GE.MNTHR ) THEN
*
*              Path 2a - underdetermined, with many more columns
*              than rows
*
*              Compute space needed for DGELQF
               CALL DGELQF( M, N, A, LDA, DUM(1), DUM(1),
     $             -1, INFO )
               LWORK_DGELQF=DUM(1)
*              Compute space needed for DGEBRD
               CALL DGEBRD( M, M, A, LDA, S, DUM(1), DUM(1),
     $                   DUM(1), DUM(1), -1, INFO )
               LWORK_DGEBRD=DUM(1)
```

```
*                 Compute space needed for DORMBR
                  CALL DORMBR( 'Q', 'L', 'T', M, NRHS, N, A, LDA,
     $                DUM(1), B, LDB, DUM(1), -1, INFO )
                  LWORK_DORMBR=DUM(1)
*                 Compute space needed for DORGBR
                  CALL DORGBR( 'P', M, M, M, A, LDA, DUM(1),
     $                   DUM(1), -1, INFO )
                  LWORK_DORGBR=DUM(1)
*                 Compute space needed for DORMLQ
                  CALL DORMLQ( 'L', 'T', N, NRHS, M, A, LDA, DUM(1),
     $                 B, LDB, DUM(1), -1, INFO )
                  LWORK_DORMLQ=DUM(1)
*                 Compute total workspace needed
                  MAXWRK = M + LWORK_DGELQF
                  MAXWRK = MAX( MAXWRK, M*M + 4*M + LWORK_DGEBRD )
                  MAXWRK = MAX( MAXWRK, M*M + 4*M + LWORK_DORMBR )
                  MAXWRK = MAX( MAXWRK, M*M + 4*M + LWORK_DORGBR )
                  MAXWRK = MAX( MAXWRK, M*M + M + BDSPAC )
                  IF( NRHS.GT.1 ) THEN
                     MAXWRK = MAX( MAXWRK, M*M + M + M*NRHS )
                  ELSE
                     MAXWRK = MAX( MAXWRK, M*M + 2*M )
                  END IF
                  MAXWRK = MAX( MAXWRK, M + LWORK_DORMLQ )
               ELSE
*
*                 Path 2 - underdetermined
*
*                 Compute space needed for DGEBRD
                  CALL DGEBRD( M, N, A, LDA, S, DUM(1), DUM(1),
     $                    DUM(1), DUM(1), -1, INFO )
                  LWORK_DGEBRD=DUM(1)
*                 Compute space needed for DORMBR
                  CALL DORMBR( 'Q', 'L', 'T', M, NRHS, M, A, LDA,
     $                DUM(1), B, LDB, DUM(1), -1, INFO )
                  LWORK_DORMBR=DUM(1)
*                 Compute space needed for DORGBR
                  CALL DORGBR( 'P', M, N, M, A, LDA, DUM(1),
     $                   DUM(1), -1, INFO )
                  LWORK_DORGBR=DUM(1)
                  MAXWRK = 3*M + LWORK_DGEBRD
                  MAXWRK = MAX( MAXWRK, 3*M + LWORK_DORMBR )
                  MAXWRK = MAX( MAXWRK, 3*M + LWORK_DORGBR )
                  MAXWRK = MAX( MAXWRK, BDSPAC )
                  MAXWRK = MAX( MAXWRK, N*NRHS )
               END IF
            END IF
            MAXWRK = MAX( MINWRK, MAXWRK )
         END IF
         WORK( 1 ) = MAXWRK
```

```
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY )
     $      INFO = -12
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGELSS', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         RANK = 0
         RETURN
      END IF
*
*     Get machine parameters
*
      EPS = DLAMCH( 'P' )
      SFMIN = DLAMCH( 'S' )
      SMLNUM = SFMIN / EPS
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = DLANGE( 'M', M, N, A, LDA, WORK )
      IASCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL DLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
         IASCL = 1
      ELSE IF( ANRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL DLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
         IASCL = 2
      ELSE IF( ANRM.EQ.ZERO ) THEN
*
*        Matrix all zero. Return zero solution.
*
         CALL DLASET( 'F', MAX( M, N ), NRHS, ZERO, ZERO, B, LDB )
         CALL DLASET( 'F', MINMN, 1, ZERO, ZERO, S, MINMN )
         RANK = 0
```

```
         GO TO 70
      END IF
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = DLANGE( 'M', M, NRHS, B, LDB, WORK )
      IBSCL = 0
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL DLASCL( 'G', 0, 0, BNRM, SMLNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 1
      ELSE IF( BNRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL DLASCL( 'G', 0, 0, BNRM, BIGNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 2
      END IF
*
*     Overdetermined case
*
      IF( M.GE.N ) THEN
*
*        Path 1 - overdetermined or exactly determined
*
         MM = M
         IF( M.GE.MNTHR ) THEN
*
*           Path 1a - overdetermined, with many more rows than columns
*
            MM = N
            ITAU = 1
            IWORK = ITAU + N
*
*           Compute A=Q*R
*           (Workspace: need 2*N, prefer N+N*NB)
*
            CALL DGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
     $                   LWORK-IWORK+1, INFO )
*
*           Multiply B by transpose(Q)
*           (Workspace: need N+NRHS, prefer N+NRHS*NB)
*
            CALL DORMQR( 'L', 'T', M, NRHS, N, A, LDA, WORK( ITAU ), B,
     $                   LDB, WORK( IWORK ), LWORK-IWORK+1, INFO )
*
*           Zero out below R
*
```

```
            IF( N.GT.1 )
     $         CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ), LDA )
         END IF
*
         IE = 1
         ITAUQ = IE + N
         ITAUP = ITAUQ + N
         IWORK = ITAUP + N
*
*        Bidiagonalize R in A
*        (Workspace: need 3*N+MM, prefer 3*N+(MM+N)*NB)
*
         CALL DGEBRD( MM, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors of R
*        (Workspace: need 3*N+NRHS, prefer 3*N+NRHS*NB)
*
         CALL DORMBR( 'Q', 'L', 'T', MM, NRHS, N, A, LDA, WORK( ITAUQ ),
     $                B, LDB, WORK( IWORK ), LWORK-IWORK+1, INFO )
*
*        Generate right bidiagonalizing vectors of R in A
*        (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
         CALL DORGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                WORK( IWORK ), LWORK-IWORK+1, INFO )
         IWORK = IE + N
*
*        Perform bidiagonal QR iteration
*          multiply B by transpose of left singular vectors
*          compute right singular vectors in A
*        (Workspace: need BDSPAC)
*
         CALL DBDSQR( 'U', N, N, 0, NRHS, S, WORK( IE ), A, LDA, DUM,
     $                1, B, LDB, WORK( IWORK ), INFO )
         IF( INFO.NE.0 )
     $      GO TO 70
*
*        Multiply B by reciprocals of singular values
*
         THR = MAX( RCOND*S( 1 ), SFMIN )
         IF( RCOND.LT.ZERO )
     $      THR = MAX( EPS*S( 1 ), SFMIN )
         RANK = 0
         DO 10 I = 1, N
            IF( S( I ).GT.THR ) THEN
               CALL DRSCL( NRHS, S( I ), B( I, 1 ), LDB )
               RANK = RANK + 1
            ELSE
```

```
            CALL DLASET( 'F', 1, NRHS, ZERO, ZERO, B( I, 1 ), LDB )
         END IF
   10    CONTINUE
*
*        Multiply B by right singular vectors
*        (Workspace: need N, prefer N*NRHS)
*
         IF( LWORK.GE.LDB*NRHS .AND. NRHS.GT.1 ) THEN
            CALL DGEMM( 'T', 'N', N, NRHS, N, ONE, A, LDA, B, LDB, ZERO,
     $                  WORK, LDB )
            CALL DLACPY( 'G', N, NRHS, WORK, LDB, B, LDB )
         ELSE IF( NRHS.GT.1 ) THEN
            CHUNK = LWORK / N
            DO 20 I = 1, NRHS, CHUNK
               BL = MIN( NRHS-I+1, CHUNK )
               CALL DGEMM( 'T', 'N', N, BL, N, ONE, A, LDA, B( 1, I ),
     $                     LDB, ZERO, WORK, N )
               CALL DLACPY( 'G', N, BL, WORK, N, B( 1, I ), LDB )
   20       CONTINUE
         ELSE
            CALL DGEMV( 'T', N, N, ONE, A, LDA, B, 1, ZERO, WORK, 1 )
            CALL DCOPY( N, WORK, 1, B, 1 )
         END IF
*
      ELSE IF( N.GE.MNTHR .AND. LWORK.GE.4*M+M*M+
     $         MAX( M, 2*M-4, NRHS, N-3*M ) ) THEN
*
*        Path 2a - underdetermined, with many more columns than rows
*        and sufficient workspace for an efficient algorithm
*
         LDWORK = M
         IF( LWORK.GE.MAX( 4*M+M*LDA+MAX( M, 2*M-4, NRHS, N-3*M ),
     $       M*LDA+M+M*NRHS ) )LDWORK = LDA
         ITAU = 1
         IWORK = M + 1
*
*        Compute A=L*Q
*        (Workspace: need 2*M, prefer M+M*NB)
*
         CALL DGELQF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
     $                LWORK-IWORK+1, INFO )
         IL = IWORK
*
*        Copy L to WORK(IL), zeroing out above it
*
         CALL DLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWORK )
         CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, WORK( IL+LDWORK ),
     $                LDWORK )
         IE = IL + LDWORK*M
         ITAUQ = IE + M
```

```
         ITAUP = ITAUQ + M
         IWORK = ITAUP + M
*
*        Bidiagonalize L in WORK(IL)
*        (Workspace: need M*M+5*M, prefer M*M+4*M+2*M*NB)
*
         CALL DGEBRD( M, M, WORK( IL ), LDWORK, S, WORK( IE ),
     $                WORK( ITAUQ ), WORK( ITAUP ), WORK( IWORK ),
     $                LWORK-IWORK+1, INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors of L
*        (Workspace: need M*M+4*M+NRHS, prefer M*M+4*M+NRHS*NB)
*
         CALL DORMBR( 'Q', 'L', 'T', M, NRHS, M, WORK( IL ), LDWORK,
     $                WORK( ITAUQ ), B, LDB, WORK( IWORK ),
     $                LWORK-IWORK+1, INFO )
*
*        Generate right bidiagonalizing vectors of R in WORK(IL)
*        (Workspace: need M*M+5*M-1, prefer M*M+4*M+(M-1)*NB)
*
         CALL DORGBR( 'P', M, M, M, WORK( IL ), LDWORK, WORK( ITAUP ),
     $                WORK( IWORK ), LWORK-IWORK+1, INFO )
         IWORK = IE + M
*
*        Perform bidiagonal QR iteration,
*           computing right singular vectors of L in WORK(IL) and
*           multiplying B by transpose of left singular vectors
*        (Workspace: need M*M+M+BDSPAC)
*
         CALL DBDSQR( 'U', M, M, 0, NRHS, S, WORK( IE ), WORK( IL ),
     $                LDWORK, A, LDA, B, LDB, WORK( IWORK ), INFO )
         IF( INFO.NE.0 )
     $      GO TO 70
*
*        Multiply B by reciprocals of singular values
*
         THR = MAX( RCOND*S( 1 ), SFMIN )
         IF( RCOND.LT.ZERO )
     $      THR = MAX( EPS*S( 1 ), SFMIN )
         RANK = 0
         DO 30 I = 1, M
            IF( S( I ).GT.THR ) THEN
               CALL DRSCL( NRHS, S( I ), B( I, 1 ), LDB )
               RANK = RANK + 1
            ELSE
               CALL DLASET( 'F', 1, NRHS, ZERO, ZERO, B( I, 1 ), LDB )
            END IF
   30    CONTINUE
         IWORK = IE
*
```

```
*        Multiply B by right singular vectors of L in WORK(IL)
*        (Workspace: need M*M+2*M, prefer M*M+M+M*NRHS)
*
         IF( LWORK.GE.LDB*NRHS+IWORK-1 .AND. NRHS.GT.1 ) THEN
            CALL DGEMM( 'T', 'N', M, NRHS, M, ONE, WORK( IL ), LDWORK,
     $                  B, LDB, ZERO, WORK( IWORK ), LDB )
            CALL DLACPY( 'G', M, NRHS, WORK( IWORK ), LDB, B, LDB )
         ELSE IF( NRHS.GT.1 ) THEN
            CHUNK = ( LWORK-IWORK+1 ) / M
            DO 40 I = 1, NRHS, CHUNK
               BL = MIN( NRHS-I+1, CHUNK )
               CALL DGEMM( 'T', 'N', M, BL, M, ONE, WORK( IL ), LDWORK,
     $                     B( 1, I ), LDB, ZERO, WORK( IWORK ), M )
               CALL DLACPY( 'G', M, BL, WORK( IWORK ), M, B( 1, I ),
     $                      LDB )
   40       CONTINUE
         ELSE
            CALL DGEMV( 'T', M, M, ONE, WORK( IL ), LDWORK, B( 1, 1 ),
     $                  1, ZERO, WORK( IWORK ), 1 )
            CALL DCOPY( M, WORK( IWORK ), 1, B( 1, 1 ), 1 )
         END IF
*
*        Zero out below first M rows of B
*
         CALL DLASET( 'F', N-M, NRHS, ZERO, ZERO, B( M+1, 1 ), LDB )
         IWORK = ITAU + M
*
*        Multiply transpose(Q) by B
*        (Workspace: need M+NRHS, prefer M+NRHS*NB)
*
         CALL DORMLQ( 'L', 'T', N, NRHS, M, A, LDA, WORK( ITAU ), B,
     $               LDB, WORK( IWORK ), LWORK-IWORK+1, INFO )
*
      ELSE
*
*        Path 2 - remaining underdetermined cases
*
         IE = 1
         ITAUQ = IE + M
         ITAUP = ITAUQ + M
         IWORK = ITAUP + M
*
*        Bidiagonalize A
*        (Workspace: need 3*M+N, prefer 3*M+(M+N)*NB)
*
         CALL DGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $               WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $               INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors
```

```
*        (Workspace: need 3*M+NRHS, prefer 3*M+NRHS*NB)
*
         CALL DORMBR( 'Q', 'L', 'T', M, NRHS, N, A, LDA, WORK( ITAUQ ),
     $                B, LDB, WORK( IWORK ), LWORK-IWORK+1, INFO )
*
*        Generate right bidiagonalizing vectors in A
*        (Workspace: need 4*M, prefer 3*M+M*NB)
*
         CALL DORGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                WORK( IWORK ), LWORK-IWORK+1, INFO )
         IWORK = IE + M
*
*        Perform bidiagonal QR iteration,
*           computing right singular vectors of A in A and
*           multiplying B by transpose of left singular vectors
*        (Workspace: need BDSPAC)
*
         CALL DBDSQR( 'L', M, N, 0, NRHS, S, WORK( IE ), A, LDA, DUM,
     $                1, B, LDB, WORK( IWORK ), INFO )
         IF( INFO.NE.0 )
     $      GO TO 70
*
*        Multiply B by reciprocals of singular values
*
         THR = MAX( RCOND*S( 1 ), SFMIN )
         IF( RCOND.LT.ZERO )
     $      THR = MAX( EPS*S( 1 ), SFMIN )
         RANK = 0
         DO 50 I = 1, M
            IF( S( I ).GT.THR ) THEN
               CALL DRSCL( NRHS, S( I ), B( I, 1 ), LDB )
               RANK = RANK + 1
            ELSE
               CALL DLASET( 'F', 1, NRHS, ZERO, ZERO, B( I, 1 ), LDB )
            END IF
   50    CONTINUE
*
*        Multiply B by right singular vectors of A
*        (Workspace: need N, prefer N*NRHS)
*
         IF( LWORK.GE.LDB*NRHS .AND. NRHS.GT.1 ) THEN
            CALL DGEMM( 'T', 'N', N, NRHS, M, ONE, A, LDA, B, LDB, ZERO,
     $                  WORK, LDB )
            CALL DLACPY( 'F', N, NRHS, WORK, LDB, B, LDB )
         ELSE IF( NRHS.GT.1 ) THEN
            CHUNK = LWORK / N
            DO 60 I = 1, NRHS, CHUNK
               BL = MIN( NRHS-I+1, CHUNK )
               CALL DGEMM( 'T', 'N', N, BL, M, ONE, A, LDA, B( 1, I ),
     $                     LDB, ZERO, WORK, N )
```

```
            CALL DLACPY( 'F', N, BL, WORK, N, B( 1, I ), LDB )
   60       CONTINUE
         ELSE
            CALL DGEMV( 'T', M, N, ONE, A, LDA, B, 1, ZERO, WORK, 1 )
            CALL DCOPY( N, WORK, 1, B, 1 )
         END IF
      END IF
*
*     Undo scaling
*
      IF( IASCL.EQ.1 ) THEN
         CALL DLASCL( 'G', 0, 0, ANRM, SMLNUM, N, NRHS, B, LDB, INFO )
         CALL DLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
     $                INFO )
      ELSE IF( IASCL.EQ.2 ) THEN
         CALL DLASCL( 'G', 0, 0, ANRM, BIGNUM, N, NRHS, B, LDB, INFO )
         CALL DLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                INFO )
      END IF
      IF( IBSCL.EQ.1 ) THEN
         CALL DLASCL( 'G', 0, 0, SMLNUM, BNRM, N, NRHS, B, LDB, INFO )
      ELSE IF( IBSCL.EQ.2 ) THEN
         CALL DLASCL( 'G', 0, 0, BIGNUM, BNRM, N, NRHS, B, LDB, INFO )
      END IF
*
   70 CONTINUE
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of DGELSS
*
      END
```

———-

20.3.4 lisp code

— dgelss.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dgelss (m n nrhs a lda b ldb$ s rcond rank work lwork info)
    (declare (type (double-float) rcond)
             (type (array double-float (*)) work s b a)
             (type (f2cl-lib:integer4) info lwork rank ldb$ lda nrhs n m))
    (f2cl-lib:with-multi-array-data
```

```
  ((a double-float a-%data% a-%offset%)
   (b double-float b-%data% b-%offset%)
   (s double-float s-%data% s-%offset%)
   (work double-float work-%data% work-%offset%))
(prog ((dum (make-array 1 :element-type 'double-float)) (anrm 0.0d0)
       (bignum 0.0d0) (bnrm 0.0d0) (eps 0.0d0) (sfmin 0.0d0)
       (smlnum 0.0d0) (thr 0.0d0) (lwork_dgeqrf 0) (lwork_dormqr 0)
       (lwork_dgebrd 0) (lwork_dormbr 0) (lwork_dorgbr 0)
       (lwork_dormlq 0) (lwork_dgelqf 0) (bdspac 0) (bl 0) (chunk 0)
       (i 0) (iascl 0) (ibscl 0) (ie 0) (il 0) (itau 0) (itaup 0)
       (itauq 0) (iwork 0) (ldwork 0) (maxmn 0) (maxwrk 0) (minmn 0)
       (minwrk 0) (mm 0) (mnthr 0) (lquery nil))
  (declare (type (array double-float (1)) dum)
           (type (double-float) anrm bignum bnrm eps sfmin smlnum thr)
           (type (f2cl-lib:integer4) lwork_dgeqrf lwork_dormqr
            lwork_dgebrd lwork_dormbr lwork_dorgbr lwork_dormlq
            lwork_dgelqf bdspac bl chunk i iascl ibscl ie il itau itaup
            itauq iwork ldwork maxmn maxwrk minmn minwrk mm mnthr)
           (type f2cl-lib:logical lquery))
  (setf info 0)
  (setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
  (setf maxmn (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
  (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
  (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
        ((< nrhs 0) (setf info -3))
        ((< lda
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -5))
        ((< ldb$
            (max (the f2cl-lib:integer4 1)
                 (the f2cl-lib:integer4 maxmn)))
         (setf info -7)))
  (cond
   ((= info 0) (setf minwrk 1) (setf maxwrk 1)
    (cond
     ((> minmn 0) (setf mm m)
      (setf mnthr
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 6 "DGELSS" " " m n nrhs -1)
                (declare (ignore var-0 var-1 var-2 var-6))
                (when var-3 (setf m var-3))
                (when var-4 (setf n var-4))
                (when var-5 (setf nrhs var-5))
                ret-val))
      (cond
       ((and (>= m n) (>= m mnthr))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (dgeqrf m n a lda
```

```
          (f2cl-lib:array-slice dum double-float (1) ((1 1)))
          (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1 info)
        (declare (ignore var-2 var-4 var-5 var-6))
        (setf m var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf info var-7))
      (setf lwork_dgeqrf
              (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (dormqr "L" "T" m nrhs n a lda
           (f2cl-lib:array-slice dum double-float (1) ((1 1))) b ldb$
           (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1 info)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf nrhs var-3))
        (when var-4 (setf n var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldb$ var-9))
        (when var-12 (setf info var-12)))
      (setf lwork_dormqr
              (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
      (setf mm n)
      (setf maxwrk
              (max (the f2cl-lib:integer4 maxwrk)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add n lwork_dgeqrf))))
      (setf maxwrk
              (max (the f2cl-lib:integer4 maxwrk)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add n lwork_dormqr))))))
    (cond
     ((>= m n)
      (setf bdspac
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 (f2cl-lib:int-mul 5 n))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (dgebrd mm n a lda s
           (f2cl-lib:array-slice dum double-float (1) ((1 1)))
           (f2cl-lib:array-slice dum double-float (1) ((1 1)))
           (f2cl-lib:array-slice dum double-float (1) ((1 1)))
           (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1 info)
        (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
        (setf mm var-0)
        (setf n var-1)
        (setf lda var-3)
```

```
  (setf info var-10))
(setf lwork_dgebrd
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "Q" "L" "T" mm nrhs n a lda
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) b ldb$
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1 info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf mm var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(setf lwork_dormbr
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dorgbr "P" n n n a lda
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1 info)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf info var-9)))
(setf lwork_dorgbr
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    lwork_dgebrd))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    lwork_dormbr))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    lwork_dorgbr))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4 bdspac)))
```

```
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul n nrhs))))
  (setf minwrk
          (max
           (the f2cl-lib:integer4
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) mm))
           (the f2cl-lib:integer4
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) nrhs))
           (the f2cl-lib:integer4 bdspac)))
  (setf maxwrk
          (max (the f2cl-lib:integer4 minwrk)
               (the f2cl-lib:integer4 maxwrk)))))
(cond
 ((> n m)
  (setf bdspac
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul 5 m))))
  (setf minwrk
          (max
           (the f2cl-lib:integer4
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) nrhs))
           (the f2cl-lib:integer4
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
           (the f2cl-lib:integer4 bdspac)))
  (cond
   ((>= n mnthr)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (dgelqf m n a lda
         (f2cl-lib:array-slice dum double-float (1) ((1 1)))
         (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1
         info)
      (declare (ignore var-2 var-4 var-5 var-6))
      (setf m var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf info var-7))
    (setf lwork_dgelqf
            (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10)
        (dgebrd m m a lda s
         (f2cl-lib:array-slice dum double-float (1) ((1 1)))
         (f2cl-lib:array-slice dum double-float (1) ((1 1)))
         (f2cl-lib:array-slice dum double-float (1) ((1 1)))
         (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1
         info)
      (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
```

```
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf info var-10))
(setf lwork_dgebrd
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (dormbr "Q" "L" "T" m nrhs n a lda
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) b ldb$
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1
     info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(setf lwork_dormbr
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "P" m m m a lda
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1
     info)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf info var-9)))
(setf lwork_dorgbr
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (dormlq "L" "T" n nrhs m a lda
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) b ldb$
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1
     info)
  (declare
   (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf n var-2))
  (when var-3 (setf nrhs var-3))
  (when var-4 (setf m var-4))
```

```
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldb$ var-9))
    (when var-12 (setf info var-12)))
  (setf lwork_dormlq
          (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
  (setf maxwrk (f2cl-lib:int-add m lwork_dgelqf))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                      (f2cl-lib:int-mul 4 m)
                                      lwork_dgebrd))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                      (f2cl-lib:int-mul 4 m)
                                      lwork_dormbr))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                      (f2cl-lib:int-mul 4 m)
                                      lwork_dorgbr))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul m m) m
                                      bdspac))))
  (cond
   ((> nrhs 1)
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul m m) m
                                        (f2cl-lib:int-mul m
                                                          nrhs))))))
   (t
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                        (f2cl-lib:int-mul 2
                                                          m)))))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add m lwork_dormlq)))))
 (t
  (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd m n a lda s
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1
     info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-10))
(setf lwork_dgebrd
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (dormbr "Q" "L" "T" m nrhs m a lda
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) b ldb$
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1
     info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(setf lwork_dormbr
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "P" m n m a lda
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1
     info)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf info var-9)))
(setf lwork_dorgbr
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(setf maxwrk
        (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) lwork_dgebrd))
(setf maxwrk
```

```
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                              lwork_dormbr))))
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                              lwork_dorgbr))))
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4 bdspac)))
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-mul n nrhs))))))))
      (setf maxwrk
              (max (the f2cl-lib:integer4 minwrk)
                   (the f2cl-lib:integer4 maxwrk)))))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce (the f2cl-lib:integer4 maxwrk) 'double-float))
    (if (and (< lwork minwrk) (not lquery))
        (setf info -12))))
  (cond
   ((/= info 0) (xerbla "DGELSS" (f2cl-lib:int-sub info)) (go end_label))
   (lquery (go end_label)))
  (cond ((or (= m 0) (= n 0)) (setf rank 0) (go end_label)))
  (setf eps (dlamch "P"))
  (setf sfmin (dlamch "S"))
  (setf smlnum (/ sfmin eps))
  (setf bignum (/ one smlnum))
  (multiple-value-bind (var-0 var-1)
      (dlabad smlnum bignum)
    (declare (ignore))
    (when var-0 (setf smlnum var-0))
    (when var-1 (setf bignum var-1)))
  (setf anrm
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
              (dlange "M" m n a lda work)
            (declare (ignore var-0 var-3 var-5))
            (when var-1 (setf m var-1))
            (when var-2 (setf n var-2))
            (when var-4 (setf lda var-4))
            ret-val))
  (setf iascl 0)
  (cond
   ((and (> anrm zero) (< anrm smlnum))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
```

```
      (dlascl "G" 0 0 anrm smlnum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 1))
 ((> anrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 anrm bignum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 2))
 ((= anrm zero)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "F"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       zero zero b ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldb$ var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "F" minmn 1 zero zero s minmn)
    (declare (ignore var-0 var-2 var-5))
    (when var-1 (setf minmn var-1))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf minmn var-6)))
  (setf rank 0) (go label70)))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "M" m nrhs b ldb$ work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ibscl 0)
(cond
```

```
  ((and (> bnrm zero) (< bnrm smlnum))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (dlascl "G" 0 0 bnrm smlnum m nrhs b ldb$ info)
     (declare (ignore var-0 var-1 var-2 var-7))
     (when var-3 (setf bnrm var-3))
     (when var-4 (setf smlnum var-4))
     (when var-5 (setf m var-5))
     (when var-6 (setf nrhs var-6))
     (when var-8 (setf ldb$ var-8))
     (when var-9 (setf info var-9)))
   (setf ibscl 1))
  ((> bnrm bignum)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (dlascl "G" 0 0 bnrm bignum m nrhs b ldb$ info)
     (declare (ignore var-0 var-1 var-2 var-7))
     (when var-3 (setf bnrm var-3))
     (when var-4 (setf bignum var-4))
     (when var-5 (setf m var-5))
     (when var-6 (setf nrhs var-6))
     (when var-8 (setf ldb$ var-8))
     (when var-9 (setf info var-9)))
   (setf ibscl 2)))
 (cond
  ((>= m n) (setf mm m)
   (cond
    ((>= m mnthr) (setf mm n) (setf itau 1)
     (setf iwork (f2cl-lib:int-add itau n))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (dgeqrf m n a lda
          (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                                work-%offset%)
          (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                                work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
       (declare (ignore var-2 var-4 var-5 var-6))
       (setf m var-0)
       (setf n var-1)
       (setf lda var-3)
       (setf info var-7))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12)
         (dormqr "L" "T" m nrhs n a lda
          (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                                work-%offset%)
          b ldb$
          (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
```

```
                                work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf nrhs var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldb$ var-9))
      (when var-12 (setf info var-12)))
    (if (> n 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
             zero zero
             (f2cl-lib:array-slice a-%data% double-float (2 1)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-1 var-2 var-5))
          (when var-3 (setf zero var-3))
          (when var-4 (setf zero var-4))
          (when var-6 (setf lda var-6))))))
  (setf ie 1) (setf itauq (f2cl-lib:int-add ie n))
  (setf itaup (f2cl-lib:int-add itauq n))
  (setf iwork (f2cl-lib:int-add itaup n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dgebrd mm n a lda s
       (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
    (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
    (setf mm var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf info var-10))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dormbr "Q" "L" "T" mm nrhs n a lda
       (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                             work-%offset%)
       b ldb$
       (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
```

```
                            work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf mm var-3))
    (when var-4 (setf nrhs var-4))
    (when var-5 (setf n var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldb$ var-10))
    (when var-13 (setf info var-13)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dorgbr "P" n n n a lda
       (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                            work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                            work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf info var-9)))
  (setf iwork (f2cl-lib:int-add ie n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (dbdsqr "U" n n 0 nrhs s
       (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                            work-%offset%)
       a lda dum 1 b ldb$
       (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                            work-%offset%)
       info)
    (declare
     (ignore var-0 var-3 var-5 var-6 var-7 var-9 var-10 var-11 var-13))
    (setf n var-1)
    (setf n var-2)
    (setf nrhs var-4)
    (setf lda var-8)
    (setf ldb$ var-12)
    (setf info var-14))
  (if (/= info 0)
      (go label70))
  (setf thr
          (max
           (* rcond (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
           sfmin))
  (if (< rcond zero)
```

```
    (setf thr
            (max
             (* eps (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
             sfmin)))
(setf rank 0)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref s (i) ((1 *))) thr)
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (drscl nrhs
           (f2cl-lib:array-slice s-%data% double-float (i) ((1 *))
                                 s-%offset%)
           (f2cl-lib:array-slice b-%data% double-float (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-2))
        (when var-0 (setf nrhs var-0))
        (when var-3 (setf ldb$ var-3)))
      (setf rank (f2cl-lib:int-add rank 1)))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlaset "F" 1 nrhs zero zero
           (f2cl-lib:array-slice b-%data% double-float (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-1 var-5))
        (when var-2 (setf nrhs var-2))
        (when var-3 (setf zero var-3))
        (when var-4 (setf zero var-4))
        (when var-6 (setf ldb$ var-6)))))
   label10))
(cond
 ((and (>= lwork (f2cl-lib:int-mul ldb$ nrhs)) (> nrhs 1))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dgemm "T" "N" n nrhs n one a lda b ldb$ zero work ldb$)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldb$ var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldb$ var-12)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
      (dlacpy "G" n nrhs work ldb$ b ldb$)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf nrhs var-2))
    (when var-4 (setf ldb$ var-4))
    (when var-6 (setf ldb$ var-6))))
  ((> nrhs 1) (setf chunk (the f2cl-lib:integer4 (truncate lwork n)))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
                 ((> i nrhs) nil)
    (tagbody
      (setf bl
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub nrhs i) 1))
               (the f2cl-lib:integer4 chunk)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (dgemm "T" "N" n bl n one a lda
           (f2cl-lib:array-slice b-%data% double-float (1 i)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$ zero work n)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf n var-2))
        (when var-3 (setf bl var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf one var-5))
        (when var-7 (setf lda var-7))
        (when var-9 (setf ldb$ var-9))
        (when var-10 (setf zero var-10))
        (when var-12 (setf n var-12)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlacpy "G" n bl work n
           (f2cl-lib:array-slice b-%data% double-float (1 i)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf bl var-2))
        (when var-4 (setf n var-4))
        (when var-6 (setf ldb$ var-6)))
     label20)))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (dgemv "T" n n one a lda b 1 zero work 1)
     (declare (ignore var-0 var-4 var-6 var-7 var-9 var-10))
     (when var-1 (setf n var-1))
```

```
      (when var-2 (setf n var-2))
      (when var-3 (setf one var-3))
      (when var-5 (setf lda var-5))
      (when var-8 (setf zero var-8)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy n work 1 b 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0))))))
((and (>= n mnthr)
      (>= lwork
          (f2cl-lib:int-add (f2cl-lib:int-mul 4 m)
                            (f2cl-lib:int-mul m m)
                            (max (the f2cl-lib:integer4 m)
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add
                                       (f2cl-lib:int-mul 2 m)
                                       (f2cl-lib:int-sub 4)))
                                 (the f2cl-lib:integer4 nrhs)
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add n
                                                        (f2cl-lib:int-mul
                                                         -1 3 m)))))))
 (setf ldwork m)
 (if (>= lwork
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 4 m)
                                 (f2cl-lib:int-mul m lda)
                                 (max (the f2cl-lib:integer4 m)
                                      (the f2cl-lib:integer4
                                           (f2cl-lib:int-sub
                                            (f2cl-lib:int-mul 2 m) 4))
                                      (the f2cl-lib:integer4 nrhs)
                                      (the f2cl-lib:integer4
                                           (f2cl-lib:int-sub n
                                                             (f2cl-lib:int-mul
                                                              3
                                                              m))))))
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul m lda) m
                                 (f2cl-lib:int-mul m nrhs)))))
     (setf ldwork lda))
 (setf itau 1) (setf iwork (f2cl-lib:int-add m 1))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (dgelqf m n a lda
      (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                            work-%offset%)
```

```
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-7))
(setf il iwork)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                           work-%offset%)
     ldwork)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwork var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1) zero
     zero
     (f2cl-lib:array-slice work-%data% double-float ((+ il ldwork))
                           ((1 *)) work-%offset%)
     ldwork)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwork var-6)))
(setf ie (f2cl-lib:int-add il (f2cl-lib:int-mul ldwork m)))
(setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgebrd m m
     (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                           work-%offset%)
     ldwork s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
```

```
  (setf ldwork var-3)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "Q" "L" "T" m nrhs m
     (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                           work-%offset%)
     ldwork
     (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                           work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwork var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dorgbr "P" m m m
     (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                           work-%offset%)
     ldwork
     (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwork var-5))
  (when var-9 (setf info var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" m m 0 nrhs s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                           work-%offset%)
     ldwork a lda b ldb$
```

```
      (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                            work-%offset%)
      info)
   (declare
    (ignore var-0 var-3 var-5 var-6 var-7 var-9 var-11 var-13))
   (setf m var-1)
   (setf m var-2)
   (setf nrhs var-4)
   (setf ldwork var-8)
   (setf lda var-10)
   (setf ldb$ var-12)
   (setf info var-14))
 (if (/= info 0)
     (go label70))
 (setf thr
         (max
          (* rcond (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
          sfmin))
 (if (< rcond zero)
     (setf thr
             (max
              (* eps (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
              sfmin)))
 (setf rank 0)
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i m) nil)
   (tagbody
     (cond
      ((> (f2cl-lib:fref s (i) ((1 *))) thr)
       (multiple-value-bind (var-0 var-1 var-2 var-3)
           (drscl nrhs
            (f2cl-lib:array-slice s-%data% double-float (i) ((1 *))
                                  s-%offset%)
            (f2cl-lib:array-slice b-%data% double-float (i 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
         (declare (ignore var-1 var-2))
         (when var-0 (setf nrhs var-0))
         (when var-3 (setf ldb$ var-3)))
       (setf rank (f2cl-lib:int-add rank 1)))
      (t
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
           (dlaset "F" 1 nrhs zero zero
            (f2cl-lib:array-slice b-%data% double-float (i 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
         (declare (ignore var-0 var-1 var-5))
         (when var-2 (setf nrhs var-2))
         (when var-3 (setf zero var-3))
```

```
        (when var-4 (setf zero var-4))
        (when var-6 (setf ldb$ var-6)))))
   label30))
(setf iwork ie)
(cond
 ((and
   (>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul ldb$ nrhs) iwork
                         (f2cl-lib:int-sub 1)))
   (> nrhs 1))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dgemm "T" "N" m nrhs m one
       (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                             work-%offset%)
       ldwork b ldb$ zero
       (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                             work-%offset%)
       ldb$)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldwork var-7))
    (when var-9 (setf ldb$ var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldb$ var-12)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "G" m nrhs
       (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                             work-%offset%)
       ldb$ b ldb$)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf nrhs var-2))
    (when var-4 (setf ldb$ var-4))
    (when var-6 (setf ldb$ var-6))))
 ((> nrhs 1)
  (setf chunk
          (the f2cl-lib:integer4 (truncate (+ (- lwork iwork) 1) m)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
                ((> i nrhs) nil)
    (tagbody
      (setf bl
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub nrhs i) 1))
               (the f2cl-lib:integer4 chunk)))
```

```
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12)
            (dgemm "T" "N" m bl m one
             (f2cl-lib:array-slice work-%data% double-float (il)
                                   ((1 *)) work-%offset%)
             ldwork
             (f2cl-lib:array-slice b-%data% double-float (1 i)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$ zero
             (f2cl-lib:array-slice work-%data% double-float (iwork)
                                   ((1 *)) work-%offset%)
             m)
          (declare (ignore var-0 var-1 var-6 var-8 var-11))
          (when var-2 (setf m var-2))
          (when var-3 (setf bl var-3))
          (when var-4 (setf m var-4))
          (when var-5 (setf one var-5))
          (when var-7 (setf ldwork var-7))
          (when var-9 (setf ldb$ var-9))
          (when var-10 (setf zero var-10))
          (when var-12 (setf m var-12)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dlacpy "G" m bl
             (f2cl-lib:array-slice work-%data% double-float (iwork)
                                   ((1 *)) work-%offset%)
             m
             (f2cl-lib:array-slice b-%data% double-float (1 i)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf bl var-2))
          (when var-4 (setf m var-4))
          (when var-6 (setf ldb$ var-6)))
       label40)))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (dgemv "T" m m one
          (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                                work-%offset%)
          ldwork
          (f2cl-lib:array-slice b-%data% double-float (1 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          1 zero
          (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                                work-%offset%)
```

```
        1)
      (declare (ignore var-0 var-4 var-6 var-7 var-9 var-10))
      (when var-1 (setf m var-1))
      (when var-2 (setf m var-2))
      (when var-3 (setf one var-3))
      (when var-5 (setf ldwork var-5))
      (when var-8 (setf zero var-8)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy m
         (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                               work-%offset%)
         1
         (f2cl-lib:array-slice b-%data% double-float (1 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf m var-0)))))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "F" (f2cl-lib:int-sub n m) nrhs zero zero
       (f2cl-lib:array-slice b-%data% double-float ((+ m 1) 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldb$ var-6)))
  (setf iwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dormlq "L" "T" n nrhs m a lda
       (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                             work-%offset%)
       b ldb$
       (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf m var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldb$ var-9))
    (when var-12 (setf info var-12))))
 (t (setf ie 1) (setf itauq (f2cl-lib:int-add ie m))
  (setf itaup (f2cl-lib:int-add itauq m))
  (setf iwork (f2cl-lib:int-add itaup m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
     var-10)
    (dgebrd m n a lda s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "Q" "L" "T" m nrhs n a lda
     (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                           work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dorgbr "P" m n m a lda
     (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf info var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "L" m n 0 nrhs s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     a lda dum 1 b ldb$
     (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                           work-%offset%)
     info)
  (declare
   (ignore var-0 var-3 var-5 var-6 var-7 var-9 var-10 var-11 var-13))
  (setf m var-1)
  (setf n var-2)
  (setf nrhs var-4)
  (setf lda var-8)
  (setf ldb$ var-12)
  (setf info var-14))
(if (/= info 0)
    (go label70))
(setf thr
        (max
         (* rcond (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
         sfmin))
(if (< rcond zero)
    (setf thr
            (max
             (* eps (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
             sfmin)))
(setf rank 0)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref s (i) ((1 *))) thr)
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (drscl nrhs
           (f2cl-lib:array-slice s-%data% double-float (i) ((1 *))
                                 s-%offset%)
           (f2cl-lib:array-slice b-%data% double-float (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-2))
        (when var-0 (setf nrhs var-0))
        (when var-3 (setf ldb$ var-3)))
      (setf rank (f2cl-lib:int-add rank 1)))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlaset "F" 1 nrhs zero zero
           (f2cl-lib:array-slice b-%data% double-float (i 1)
```

```
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
          (declare (ignore var-0 var-1 var-5))
          (when var-2 (setf nrhs var-2))
          (when var-3 (setf zero var-3))
          (when var-4 (setf zero var-4))
          (when var-6 (setf ldb$ var-6)))))
    label50))
  (cond
   ((and (>= lwork (f2cl-lib:int-mul ldb$ nrhs)) (> nrhs 1))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (dgemm "T" "N" n nrhs m one a lda b ldb$ zero work ldb$)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf n var-2))
      (when var-3 (setf nrhs var-3))
      (when var-4 (setf m var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf lda var-7))
      (when var-9 (setf ldb$ var-9))
      (when var-10 (setf zero var-10))
      (when var-12 (setf ldb$ var-12)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "F" n nrhs work ldb$ b ldb$)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf nrhs var-2))
      (when var-4 (setf ldb$ var-4))
      (when var-6 (setf ldb$ var-6))))
   ((> nrhs 1) (setf chunk (the f2cl-lib:integer4 (truncate lwork n)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
                  ((> i nrhs) nil)
      (tagbody
        (setf bl
                (min
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-sub nrhs i) 1))
                 (the f2cl-lib:integer4 chunk)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12)
            (dgemm "T" "N" n bl m one a lda
             (f2cl-lib:array-slice b-%data% double-float (1 i)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$ zero work n)
          (declare (ignore var-0 var-1 var-6 var-8 var-11))
          (when var-2 (setf n var-2))
          (when var-3 (setf bl var-3))
          (when var-4 (setf m var-4))
```

```
          (when var-5 (setf one var-5))
          (when var-7 (setf lda var-7))
          (when var-9 (setf ldb$ var-9))
          (when var-10 (setf zero var-10))
          (when var-12 (setf n var-12)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dlacpy "F" n bl work n
             (f2cl-lib:array-slice b-%data% double-float (1 i)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf bl var-2))
          (when var-4 (setf n var-4))
          (when var-6 (setf ldb$ var-6)))
       label60)))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (dgemv "T" m n one a lda b 1 zero work 1)
      (declare (ignore var-0 var-4 var-6 var-7 var-9 var-10))
      (when var-1 (setf m var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf one var-3))
      (when var-5 (setf lda var-5))
      (when var-8 (setf zero var-8)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy n work 1 b 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))))))
(cond
 ((= iascl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 anrm smlnum n nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf n var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 smlnum anrm minmn 1 s minmn info)
    (declare (ignore var-0 var-1 var-2 var-6 var-7))
    (when var-3 (setf smlnum var-3))
    (when var-4 (setf anrm var-4))
```

```
      (when var-5 (setf minmn var-5))
      (when var-8 (setf minmn var-8))
      (when var-9 (setf info var-9))))
    ((= iascl 2)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (dlascl "G" 0 0 anrm bignum n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf bignum var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (dlascl "G" 0 0 bignum anrm minmn 1 s minmn info)
      (declare (ignore var-0 var-1 var-2 var-6 var-7))
      (when var-3 (setf bignum var-3))
      (when var-4 (setf anrm var-4))
      (when var-5 (setf minmn var-5))
      (when var-8 (setf minmn var-8))
      (when var-9 (setf info var-9)))))
  (cond
   ((= ibscl 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 smlnum bnrm n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf smlnum var-3))
      (when var-4 (setf bnrm var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9))))
   ((= ibscl 2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 bignum bnrm n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bignum var-3))
      (when var-4 (setf bnrm var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))))
 label70
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce (the f2cl-lib:integer4 maxwrk) 'double-float))
  (go end_label)
```

```
      end_label
       (return
        (values m n nrhs nil lda nil ldb$ nil nil rank nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgelss
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (double-float)
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(fortran-to-lisp::m
                                             fortran-to-lisp::n
                                             fortran-to-lisp::nrhs nil
                                             fortran-to-lisp::lda nil
                                             fortran-to-lisp::ldb$ nil nil
                                             fortran-to-lisp::rank nil nil
                                             fortran-to-lisp::info)
                                           :calls
                                           '(fortran-to-lisp::dgeqrf
                                             fortran-to-lisp::dbdsqr
                                             fortran-to-lisp::dgelqf
                                             fortran-to-lisp::dgebrd))))
```

20.4 dgelsx - DEPRECATED. Use dgelsy

20.5 dgelsy

20.5.1 Axiom unit tests

— dgelsy.input —

```
)set break resume
)sys rm -f dgelsy.output
)spool dgelsy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

20.5.2 Axiom help page

— dgelsy.help —

```
======================================================================
dgelsy examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgelsy (M, N, NRHS, A, LDA, B, LDB, JPVT, RCOND, RANK, WORK,
           LWORK, INFO)
            DGELSY solves overdetermined or underdetermined systems for GE
           matrices

Function/Subroutine Documentation
   subroutine dgelsy (integer M, integer N, integer NRHS, double precision,
       dimension( lda, * ) A, integer LDA, double precision, dimension( ldb, *
       ) B, integer LDB, integer, dimension( * ) JPVT, double precision RCOND,
       integer RANK, double precision, dimension( * ) WORK, integer LWORK,
       integer INFO)
```

```
       DGELSY solves overdetermined or underdetermined systems for GE
      matrices

Purpose:
   DGELSY computes the minimum-norm solution to a real linear least
   squares problem:
   minimize || A * X - B ||
   using a complete orthogonal factorization of A.  A is an M-by-N
   matrix which may be rank-deficient.

   Several right hand side vectors b and solution vectors x can be
   handled in a single call; they are stored as the columns of the
   M-by-NRHS right hand side matrix B and the N-by-NRHS solution
   matrix X.

   The routine first computes a QR factorization with column pivoting:
               A * P = Q * [ R11 R12 ]
                           [  0  R22 ]
   with R11 defined as the largest leading submatrix whose estimated
   condition number is less than 1/RCOND.  The order of R11, RANK,
   is the effective rank of A.

   Then, R22 is considered to be negligible, and R12 is annihilated
   by orthogonal transformations from the right, arriving at the
   complete orthogonal factorization:
             A * P = Q * [ T11 0 ] * Z
                         [  0  0 ]
   The minimum-norm solution is then
             X = P * Z**T [ inv(T11)*Q1**T*B ]
                          [        0         ]
   where Q1 consists of the first RANK columns of Q.

   This routine is basically identical to the original xGELSX except
   three differences:
   o The call to the subroutine xGEQPF has been substituted by the
      the call to the subroutine xGEQP3. This subroutine is a Blas-3
      version of the QR factorization with column pivoting.
   o Matrix B (the right hand side) is updated with Blas-3.
   o The permutation of matrix B (the right hand side) is faster and
      more simple.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.
```

```
NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of
  columns of matrices B and X. NRHS >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit, A has been overwritten by details of its
  complete orthogonal factorization.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the M-by-NRHS right hand side matrix B.
  On exit, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= max(1,M,N).

JPVT
  JPVT is INTEGER array, dimension (N)
  On entry, if JPVT(i) .ne. 0, the i-th column of A is permuted
  to the front of AP, otherwise column i is a free column.
  On exit, if JPVT(i) = k, then the i-th column of AP
  was the k-th column of A.

RCOND
  RCOND is DOUBLE PRECISION
  RCOND is used to determine the effective rank of A, which
  is defined as the order of the largest leading triangular
  submatrix R11 in the QR factorization with pivoting of A,
  whose estimated condition number < 1/RCOND.

RANK
  RANK is INTEGER
  The effective rank of A, i.e., the order of the submatrix
  R11.  This is the same as the order of the submatrix T11
  in the complete orthogonal factorization of A.

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
```

```
  LWORK is INTEGER
  The dimension of the array WORK.
  The unblocked strategy requires that:
     LWORK >= MAX( MN+3*N+1, 2*MN+NRHS ),
  where MN = min( M, N ).
  The block algorithm requires that:
  LWORK >= MAX( MN+2*N+NB*(N+1), 2*MN+NB*NRHS ),
  where NB is an upper bound on the blocksize returned
  by ILAENV for the routines DGEQP3, DTZRZF, STZRQF, DORMQR,
  and DORMRZ.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: If INFO = -i, the i-th argument had an illegal value.
```

20.5.3 fortran code

— dgelsy.f —

```
*  ===================================================================
      SUBROUTINE DGELSY( M, N, NRHS, A, LDA, B, LDB, JPVT, RCOND, RANK,
     $                   WORK, LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDB, LWORK, M, N, NRHS, RANK
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            JPVT( * )
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
*  ===================================================================
*
```

```
*     .. Parameters ..
      INTEGER            IMAX, IMIN
      PARAMETER          ( IMAX = 1, IMIN = 2 )
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IASCL, IBSCL, ISMAX, ISMIN, J, LWKMIN,
     $                   LWKOPT, MN, NB, NB1, NB2, NB3, NB4
      DOUBLE PRECISION   ANRM, BIGNUM, BNRM, C1, C2, S1, S2, SMAX,
     $                   SMAXPR, SMIN, SMINPR, SMLNUM, WSIZE
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           ILAENV, DLAMCH, DLANGE
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DGEQP3, DLABAD, DLAIC1, DLASCL, DLASET,
     $                   DORMQR, DORMRZ, DTRSM, DTZRZF, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      MN = MIN( M, N )
      ISMIN = MN + 1
      ISMAX = 2*MN + 1
*
*     Test the input arguments.
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, M, N ) ) THEN
         INFO = -7
      END IF
*
*     Figure out optimal block size
*
      IF( INFO.EQ.0 ) THEN
```

```
         IF( MN.EQ.0 .OR. NRHS.EQ.0 ) THEN
            LWKMIN = 1
            LWKOPT = 1
         ELSE
            NB1 = ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
            NB2 = ILAENV( 1, 'DGERQF', ' ', M, N, -1, -1 )
            NB3 = ILAENV( 1, 'DORMQR', ' ', M, N, NRHS, -1 )
            NB4 = ILAENV( 1, 'DORMRQ', ' ', M, N, NRHS, -1 )
            NB = MAX( NB1, NB2, NB3, NB4 )
            LWKMIN = MN + MAX( 2*MN, N + 1, MN + NRHS )
            LWKOPT = MAX( LWKMIN,
     $                    MN + 2*N + NB*( N + 1 ), 2*MN + NB*NRHS )
         END IF
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.LWKMIN .AND. .NOT.LQUERY ) THEN
            INFO = -12
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGELSY', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( MN.EQ.0 .OR. NRHS.EQ.0 ) THEN
         RANK = 0
         RETURN
      END IF
*
*     Get machine parameters
*
      SMLNUM = DLAMCH( 'S' ) / DLAMCH( 'P' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
*
*     Scale A, B if max entries outside range [SMLNUM,BIGNUM]
*
      ANRM = DLANGE( 'M', M, N, A, LDA, WORK )
      IASCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL DLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
         IASCL = 1
```

```
      ELSE IF( ANRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL DLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
         IASCL = 2
      ELSE IF( ANRM.EQ.ZERO ) THEN
*
*        Matrix all zero. Return zero solution.
*
         CALL DLASET( 'F', MAX( M, N ), NRHS, ZERO, ZERO, B, LDB )
         RANK = 0
         GO TO 70
      END IF
*
      BNRM = DLANGE( 'M', M, NRHS, B, LDB, WORK )
      IBSCL = 0
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL DLASCL( 'G', 0, 0, BNRM, SMLNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 1
      ELSE IF( BNRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL DLASCL( 'G', 0, 0, BNRM, BIGNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 2
      END IF
*
*     Compute QR factorization with column pivoting of A:
*        A * P = Q * R
*
      CALL DGEQP3( M, N, A, LDA, JPVT, WORK( 1 ), WORK( MN+1 ),
     $             LWORK-MN, INFO )
      WSIZE = MN + WORK( MN+1 )
*
*     workspace: MN+2*N+NB*(N+1).
*     Details of Householder rotations stored in WORK(1:MN).
*
*     Determine RANK using incremental condition estimation
*
      WORK( ISMIN ) = ONE
      WORK( ISMAX ) = ONE
      SMAX = ABS( A( 1, 1 ) )
      SMIN = SMAX
      IF( ABS( A( 1, 1 ) ).EQ.ZERO ) THEN
         RANK = 0
         CALL DLASET( 'F', MAX( M, N ), NRHS, ZERO, ZERO, B, LDB )
```

```
         GO TO 70
      ELSE
         RANK = 1
      END IF
*
   10 CONTINUE
      IF( RANK.LT.MN ) THEN
         I = RANK + 1
         CALL DLAIC1( IMIN, RANK, WORK( ISMIN ), SMIN, A( 1, I ),
     $                A( I, I ), SMINPR, S1, C1 )
         CALL DLAIC1( IMAX, RANK, WORK( ISMAX ), SMAX, A( 1, I ),
     $                A( I, I ), SMAXPR, S2, C2 )
*
         IF( SMAXPR*RCOND.LE.SMINPR ) THEN
            DO 20 I = 1, RANK
               WORK( ISMIN+I-1 ) = S1*WORK( ISMIN+I-1 )
               WORK( ISMAX+I-1 ) = S2*WORK( ISMAX+I-1 )
   20       CONTINUE
            WORK( ISMIN+RANK ) = C1
            WORK( ISMAX+RANK ) = C2
            SMIN = SMINPR
            SMAX = SMAXPR
            RANK = RANK + 1
            GO TO 10
         END IF
      END IF
*
*     workspace: 3*MN.
*
*     Logically partition R = [ R11 R12 ]
*                             [  0  R22 ]
*     where R11 = R(1:RANK,1:RANK)
*
*     [R11,R12] = [ T11, 0 ] * Y
*
      IF( RANK.LT.N )
     $   CALL DTZRZF( RANK, N, A, LDA, WORK( MN+1 ), WORK( 2*MN+1 ),
     $                LWORK-2*MN, INFO )
*
*     workspace: 2*MN.
*     Details of Householder rotations stored in WORK(MN+1:2*MN)
*
*     B(1:M,1:NRHS) := Q**T * B(1:M,1:NRHS)
*
      CALL DORMQR( 'Left', 'Transpose', M, NRHS, MN, A, LDA, WORK( 1 ),
     $             B, LDB, WORK( 2*MN+1 ), LWORK-2*MN, INFO )
      WSIZE = MAX( WSIZE, 2*MN+WORK( 2*MN+1 ) )
*
*     workspace: 2*MN+NB*NRHS.
*
```

```
*     B(1:RANK,1:NRHS) := inv(T11) * B(1:RANK,1:NRHS)
*
      CALL DTRSM( 'Left', 'Upper', 'No transpose', 'Non-unit', RANK,
     $            NRHS, ONE, A, LDA, B, LDB )
*
      DO 40 J = 1, NRHS
         DO 30 I = RANK + 1, N
            B( I, J ) = ZERO
   30    CONTINUE
   40 CONTINUE
*
*     B(1:N,1:NRHS) := Y**T * B(1:N,1:NRHS)
*
      IF( RANK.LT.N ) THEN
         CALL DORMRZ( 'Left', 'Transpose', N, NRHS, RANK, N-RANK, A,
     $                LDA, WORK( MN+1 ), B, LDB, WORK( 2*MN+1 ),
     $                LWORK-2*MN, INFO )
      END IF
*
*     workspace: 2*MN+NRHS.
*
*     B(1:N,1:NRHS) := P * B(1:N,1:NRHS)
*
      DO 60 J = 1, NRHS
         DO 50 I = 1, N
            WORK( JPVT( I ) ) = B( I, J )
   50    CONTINUE
         CALL DCOPY( N, WORK( 1 ), 1, B( 1, J ), 1 )
   60 CONTINUE
*
*     workspace: N.
*
*     Undo scaling
*
      IF( IASCL.EQ.1 ) THEN
         CALL DLASCL( 'G', 0, 0, ANRM, SMLNUM, N, NRHS, B, LDB, INFO )
         CALL DLASCL( 'U', 0, 0, SMLNUM, ANRM, RANK, RANK, A, LDA,
     $                INFO )
      ELSE IF( IASCL.EQ.2 ) THEN
         CALL DLASCL( 'G', 0, 0, ANRM, BIGNUM, N, NRHS, B, LDB, INFO )
         CALL DLASCL( 'U', 0, 0, BIGNUM, ANRM, RANK, RANK, A, LDA,
     $                INFO )
      END IF
      IF( IBSCL.EQ.1 ) THEN
         CALL DLASCL( 'G', 0, 0, SMLNUM, BNRM, N, NRHS, B, LDB, INFO )
      ELSE IF( IBSCL.EQ.2 ) THEN
         CALL DLASCL( 'G', 0, 0, BIGNUM, BNRM, N, NRHS, B, LDB, INFO )
      END IF
*
   70 CONTINUE
```

```
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of DGELSY
*
      END
```

20.5.4 lisp code

— dgelsy.lisp —

```
(let* ((imax 1) (imin 2) (zero 0.0d0) (one 1.0d0))
  (declare (type (f2cl-lib:integer4 1 1) imax)
           (type (f2cl-lib:integer4 2 2) imin)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable imax imin zero one))
  (defun dgelsy (m n nrhs a lda b ldb$ jpvt rcond rank work lwork info)
    (declare (type (double-float) rcond)
             (type (array f2cl-lib:integer4 (*)) jpvt)
             (type (array double-float (*)) work b a)
             (type (f2cl-lib:integer4) info lwork rank ldb$ lda nrhs n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%)
         (work double-float work-%data% work-%offset%)
         (jpvt f2cl-lib:integer4 jpvt-%data% jpvt-%offset%))
      (prog ((anrm 0.0d0) (bignum 0.0d0) (bnrm 0.0d0) (c1 0.0d0) (c2 0.0d0)
             (s1 0.0d0) (s2 0.0d0) (smax 0.0d0) (smaxpr 0.0d0) (smin 0.0d0)
             (sminpr 0.0d0) (smlnum 0.0d0) (wsize 0.0d0) (i 0) (iascl 0)
             (ibscl 0) (ismax 0) (ismin 0) (j 0) (lwkmin 0) (lwkopt 0) (mn 0)
             (nb 0) (nb1 0) (nb2 0) (nb3 0) (nb4 0) (lquery nil))
        (declare
         (type (double-float) anrm bignum bnrm c1 c2 s1 s2 smax smaxpr smin
          sminpr smlnum wsize)
         (type (f2cl-lib:integer4) i iascl ibscl ismax ismin j lwkmin lwkopt mn
          nb nb1 nb2 nb3 nb4)
         (type f2cl-lib:logical lquery))
        (setf mn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (setf ismin (f2cl-lib:int-add mn 1))
        (setf ismax (f2cl-lib:int-add (f2cl-lib:int-mul 2 mn) 1))
        (setf info 0)
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< nrhs 0) (setf info -3))
```

```
        ((< lda
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -5))
        ((< ldb$
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)
                 (the f2cl-lib:integer4 n)))
         (setf info -7)))
(cond
 ((= info 0)
  (cond ((or (= mn 0) (= nrhs 0)) (setf lwkmin 1) (setf lwkopt 1))
        (t
         (setf nb1
                 (multiple-value-bind
                     (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                      var-6)
                     (ilaenv 1 "DGEQRF" " " m n -1 -1)
                   (declare (ignore var-0 var-1 var-2 var-5 var-6))
                   (when var-3 (setf m var-3))
                   (when var-4 (setf n var-4))
                   ret-val))
         (setf nb2
                 (multiple-value-bind
                     (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                      var-6)
                     (ilaenv 1 "DGERQF" " " m n -1 -1)
                   (declare (ignore var-0 var-1 var-2 var-5 var-6))
                   (when var-3 (setf m var-3))
                   (when var-4 (setf n var-4))
                   ret-val))
         (setf nb3
                 (multiple-value-bind
                     (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                      var-6)
                     (ilaenv 1 "DORMQR" " " m n nrhs -1)
                   (declare (ignore var-0 var-1 var-2 var-6))
                   (when var-3 (setf m var-3))
                   (when var-4 (setf n var-4))
                   (when var-5 (setf nrhs var-5))
                   ret-val))
         (setf nb4
                 (multiple-value-bind
                     (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                      var-6)
                     (ilaenv 1 "DORMRQ" " " m n nrhs -1)
                   (declare (ignore var-0 var-1 var-2 var-6))
                   (when var-3 (setf m var-3))
                   (when var-4 (setf n var-4))
                   (when var-5 (setf nrhs var-5))
                   ret-val))
         (setf nb
```

```
                 (max (the f2cl-lib:integer4 nb1)
                      (the f2cl-lib:integer4 nb2)
                      (the f2cl-lib:integer4 nb3)
                      (the f2cl-lib:integer4 nb4)))
         (setf lwkmin
                 (f2cl-lib:int-add mn
                                   (max
                                    (the f2cl-lib:integer4
                                         (f2cl-lib:int-mul 2 mn))
                                    (the f2cl-lib:integer4
                                         (f2cl-lib:int-add n 1))
                                    (the f2cl-lib:integer4
                                         (f2cl-lib:int-add mn nrhs)))))
         (setf lwkopt
                 (max (the f2cl-lib:integer4 lwkmin)
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add mn (f2cl-lib:int-mul 2 n)
                                             (f2cl-lib:int-mul nb
                                                               (f2cl-lib:int-add
                                                                n 1))))
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add (f2cl-lib:int-mul 2 mn)
                                             (f2cl-lib:int-mul nb
                                                               nrhs)))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
  (cond ((and (< lwork lwkmin) (not lquery)) (setf info -12)))))
(cond
 ((/= info 0) (xerbla "DGELSY" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond ((or (= mn 0) (= nrhs 0)) (setf rank 0) (go end_label)))
(setf smlnum (/ (dlamch "S") (dlamch "P")))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "M" m n a lda work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iascl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum))
```

```
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 anrm smlnum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 1))
 ((> anrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 anrm bignum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 2))
 ((= anrm zero)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "F"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       zero zero b ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldb$ var-6)))
  (setf rank 0) (go label70)))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "M" m nrhs b ldb$ work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ibscl 0)
(cond
 ((and (> bnrm zero) (< bnrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 bnrm smlnum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
```

```
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 1))
 ((> bnrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 bnrm bignum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 2)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dgeqp3 m n a lda jpvt
     (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float ((+ mn 1)) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-sub lwork mn) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-8))
(setf wsize
        (+ mn
           (f2cl-lib:fref work-%data% ((f2cl-lib:int-add mn 1)) ((1 *))
                          work-%offset%)))
(setf (f2cl-lib:fref work-%data% (ismin) ((1 *)) work-%offset%) one)
(setf (f2cl-lib:fref work-%data% (ismax) ((1 *)) work-%offset%) one)
(setf smax
        (abs
         (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)))
(setf smin smax)
(cond
 ((= (abs (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))) zero) (setf rank 0)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "F"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       zero zero b ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
```

```
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldb$ var-6)))
    (go label70))
   (t (setf rank 1)))
 label10
  (cond
   ((< rank mn) (setf i (f2cl-lib:int-add rank 1))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dlaic1 imin rank
         (f2cl-lib:array-slice work-%data% double-float (ismin) ((1 *))
                               work-%offset%)
         smin
         (f2cl-lib:array-slice a-%data% double-float (1 i)
                               ((1 lda) (1 *)) a-%offset%)
         (f2cl-lib:array-slice a-%data% double-float (i i)
                               ((1 lda) (1 *)) a-%offset%)
         sminpr s1 c1)
      (declare (ignore var-2 var-4 var-5))
      (when var-0 (setf imin var-0))
      (when var-1 (setf rank var-1))
      (when var-3 (setf smin var-3))
      (when var-6 (setf sminpr var-6))
      (when var-7 (setf s1 var-7))
      (when var-8 (setf c1 var-8)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dlaic1 imax rank
         (f2cl-lib:array-slice work-%data% double-float (ismax) ((1 *))
                               work-%offset%)
         smax
         (f2cl-lib:array-slice a-%data% double-float (1 i)
                               ((1 lda) (1 *)) a-%offset%)
         (f2cl-lib:array-slice a-%data% double-float (i i)
                               ((1 lda) (1 *)) a-%offset%)
         smaxpr s2 c2)
      (declare (ignore var-2 var-4 var-5))
      (when var-0 (setf imax var-0))
      (when var-1 (setf rank var-1))
      (when var-3 (setf smax var-3))
      (when var-6 (setf smaxpr var-6))
      (when var-7 (setf s2 var-7))
      (when var-8 (setf c2 var-8)))
    (cond
     ((<= (* smaxpr rcond) sminpr)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i rank) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data%
```

```
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-add ismin i) 1))
                                ((1 *)) work-%offset%)
                   (* s1
                      (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-sub
                                       (f2cl-lib:int-add ismin i) 1))
                                     ((1 *)) work-%offset%)))
            (setf (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-add ismax i) 1))
                                ((1 *)) work-%offset%)
                   (* s2
                      (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-sub
                                       (f2cl-lib:int-add ismax i) 1))
                                     ((1 *)) work-%offset%)))
          label20))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ismin rank))
                             ((1 *)) work-%offset%)
              c1)
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ismax rank))
                             ((1 *)) work-%offset%)
              c2)
        (setf smin sminpr) (setf smax smaxpr)
        (setf rank (f2cl-lib:int-add rank 1)) (go label10)))))
    (if (< rank n)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (dtzrzf rank n a lda
             (f2cl-lib:array-slice work-%data% double-float ((+ mn 1))
                                   ((1 *)) work-%offset%)
             (f2cl-lib:array-slice work-%data% double-float
                                   ((+ (f2cl-lib:int-mul 2 mn) 1)) ((1 *))
                                   work-%offset%)
             (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 mn)) info)
          (declare (ignore var-2 var-4 var-5 var-6))
          (when var-0 (setf rank var-0))
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-7 (setf info var-7))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
         var-11 var-12)
        (dormqr "Left" "Transpose" m nrhs mn a lda
         (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                               work-%offset%)
         b ldb$
         (f2cl-lib:array-slice work-%data% double-float
                               ((+ (f2cl-lib:int-mul 2 mn) 1)) ((1 *))
```

```
                            work-%offset%)
      (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 mn)) info)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf nrhs var-3))
  (when var-4 (setf mn var-4))
  (when var-6 (setf lda var-6))
  (when var-9 (setf ldb$ var-9))
  (when var-12 (setf info var-12)))
(setf wsize
        (max wsize
             (+ (f2cl-lib:int-mul 2 mn)
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add
                                 (f2cl-lib:int-mul 2 mn) 1))
                               ((1 *)) work-%offset%))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrsm "Left" "Upper" "No transpose" "Non-unit" rank nrhs one a lda
     b ldb$)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf rank var-4))
  (when var-5 (setf nrhs var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf ldb$ var-10)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (f2cl-lib:fdo (i (f2cl-lib:int-add rank 1) (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)
              zero)
       label30))
   label40))
(cond
 ((< rank n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dormrz "Left" "Transpose" n nrhs rank (f2cl-lib:int-sub n rank)
       a lda
       (f2cl-lib:array-slice work-%data% double-float ((+ mn 1))
                             ((1 *)) work-%offset%)
       b ldb$
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ (f2cl-lib:int-mul 2 mn) 1)) ((1 *))
```

```
                              work-%offset%)
        (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 mn)) info)
    (declare
     (ignore var-0 var-1 var-5 var-6 var-8 var-9 var-11 var-12))
    (when var-2 (setf n var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf rank var-4))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldb$ var-10))
    (when var-13 (setf info var-13)))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data%
                              ((f2cl-lib:fref jpvt (i) ((1 *)))) ((1 *))
                              work-%offset%)
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%))
       label50))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy n
         (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                               work-%offset%)
         1
         (f2cl-lib:array-slice b-%data% double-float (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
   label60))
(cond
 ((= iascl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 anrm smlnum n nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf n var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "U" 0 0 smlnum anrm rank rank a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf smlnum var-3))
```

```
      (when var-4 (setf anrm var-4))
      (when var-5 (setf rank var-5))
      (when var-6 (setf rank var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf info var-9))))
  ((= iascl 2)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (dlascl "G" 0 0 anrm bignum n nrhs b ldb$ info)
     (declare (ignore var-0 var-1 var-2 var-7))
     (when var-3 (setf anrm var-3))
     (when var-4 (setf bignum var-4))
     (when var-5 (setf n var-5))
     (when var-6 (setf nrhs var-6))
     (when var-8 (setf ldb$ var-8))
     (when var-9 (setf info var-9)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (dlascl "U" 0 0 bignum anrm rank rank a lda info)
     (declare (ignore var-0 var-1 var-2 var-7))
     (when var-3 (setf bignum var-3))
     (when var-4 (setf anrm var-4))
     (when var-5 (setf rank var-5))
     (when var-6 (setf rank var-6))
     (when var-8 (setf lda var-8))
     (when var-9 (setf info var-9)))))
 (cond
  ((= ibscl 1)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (dlascl "G" 0 0 smlnum bnrm n nrhs b ldb$ info)
     (declare (ignore var-0 var-1 var-2 var-7))
     (when var-3 (setf smlnum var-3))
     (when var-4 (setf bnrm var-4))
     (when var-5 (setf n var-5))
     (when var-6 (setf nrhs var-6))
     (when var-8 (setf ldb$ var-8))
     (when var-9 (setf info var-9))))
  ((= ibscl 2)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (dlascl "G" 0 0 bignum bnrm n nrhs b ldb$ info)
     (declare (ignore var-0 var-1 var-2 var-7))
     (when var-3 (setf bignum var-3))
     (when var-4 (setf bnrm var-4))
     (when var-5 (setf n var-5))
     (when var-6 (setf nrhs var-6))
     (when var-8 (setf ldb$ var-8))
     (when var-9 (setf info var-9)))))
label70
```

```
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
        (go end_label)
       end_label
        (return
         (values m n nrhs nil lda nil ldb$ nil nil rank nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgelsy
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (double-float)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$ nil nil
                                              fortran-to-lisp::rank nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::dgeqp3))))
```

———-

20.6 dgesv

20.6.1 Axiom unit tests

— dgesv.input —

```
)set break resume
)sys rm -f dgesv.output
)spool dgesv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

20.6.2 Axiom help page

— **dgesv.help** —

```
=====================================================================
dgesv examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgesv (N, NRHS, A, LDA, IPIV, B, LDB, INFO)
            DGESV computes the solution to system of linear equations A * X =
           B for GE matrices

Function/Subroutine Documentation
   subroutine dgesv (integer N, integer NRHS, double precision, dimension(
       lda, * ) A, integer LDA, integer, dimension( * ) IPIV, double
       precision, dimension( ldb, * ) B, integer LDB, integer INFO)
        DGESV computes the solution to system of linear equations A * X = B
       for GE matrices

Purpose:
   DGESV computes the solution to a real system of linear equations
               A * X = B,
   where A is an N-by-N matrix and X and B are N-by-NRHS matrices.

   The LU decomposition with partial pivoting and row interchanges is
   used to factor A as
               A = P * L * U,
   where P is a permutation matrix, L is unit lower triangular, and U is
   upper triangular.  The factored form of A is then used to solve the
   system of equations A * X = B.
```

```
Parameters:

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the N-by-N coefficient matrix A.
   On exit, the factors L and U from the factorization
   A = P*L*U; the unit diagonal elements of L are not stored.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   The pivot indices that define the permutation matrix P;
   row i of the matrix was interchanged with row IPIV(i).

 B
   B is DOUBLE PRECISION array, dimension (LDB,NRHS)
   On entry, the N-by-NRHS matrix of right hand side matrix B.
   On exit, if INFO = 0, the N-by-NRHS solution matrix X.

 LDB
   LDB is INTEGER
   The leading dimension of the array B.  LDB >= max(1,N).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, U(i,i) is exactly zero.  The factorization
         has been completed, but the factor U is exactly
         singular, so the solution could not be computed.
```

20.6.3 fortran code

— dgesv.f —

```
*  =====================================================================
      SUBROUTINE DGESV( N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. External Subroutines ..
      EXTERNAL           DGETRF, DGETRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGESV ', -INFO )
         RETURN
      END IF
*
*     Compute the LU factorization of A.
*
```

```
      CALL DGETRF( N, N, A, LDA, IPIV, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
         CALL DGETRS( 'No transpose', N, NRHS, A, LDA, IPIV, B, LDB,
     $                INFO )
      END IF
      RETURN
*
*     End of DGESV
*
      END
```

———-

20.6.4 lisp code

— **dgesv.lisp** —

———-

20.7 dgesvx

20.7.1 Axiom unit tests

— **dgesvx.input** —

```
)set break resume
)sys rm -f dgesvx.output
)spool dgesvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

20.7.2 Axiom help page

— dgesvx.help —

```
==================================================================
dgesvx examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgesvx (FACT, TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV, EQUED,
           R, C, B, LDB, X, LDX, RCOND, FERR, BERR, WORK, IWORK, INFO)
            DGESVX computes the solution to system of linear equations A * X =
           B for GE matrices

Function/Subroutine Documentation
   subroutine dgesvx (character FACT, character TRANS, integer N, integer
       NRHS, double precision, dimension( lda, * ) A, integer LDA, double
       precision, dimension( ldaf, * ) AF, integer LDAF, integer, dimension( *
       ) IPIV, character EQUED, double precision, dimension( * ) R, double
       precision, dimension( * ) C, double precision, dimension( ldb, * ) B,
       integer LDB, double precision, dimension( ldx, * ) X, integer LDX,
       double precision RCOND, double precision, dimension( * ) FERR, double
       precision, dimension( * ) BERR, double precision, dimension( * ) WORK,
       integer, dimension( * ) IWORK, integer INFO)
        DGESVX computes the solution to system of linear equations A * X = B
       for GE matrices

Purpose:
   DGESVX uses the LU factorization to compute the solution to a real
   system of linear equations
               A * X = B,
   where A is an N-by-N matrix and X and B are N-by-NRHS matrices.

   Error bounds on the solution and a condition estimate are also
   provided.

Description:
   The following steps are performed:

   1. If FACT = 'E', real scaling factors are computed to equilibrate
      the system:
      TRANS = 'N':  diag(R)*A*diag(C)     *inv(diag(C))*X = diag(R)*B
      TRANS = 'T': (diag(R)*A*diag(C))**T *inv(diag(R))*X = diag(C)*B
      TRANS = 'C': (diag(R)*A*diag(C))**H *inv(diag(R))*X = diag(C)*B
```

```
      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(R)*A*diag(C) and B by diag(R)*B (if TRANS='N')
      or diag(C)*B (if TRANS = 'T' or 'C').

   2. If FACT = 'N' or 'E', the LU decomposition is used to factor the
      matrix A (after equilibration if FACT = 'E') as
                 A = P * L * U,
      where P is a permutation matrix, L is a unit lower triangular
      matrix, and U is upper triangular.

   3. If some U(i,i)=0, so that U is exactly singular, then the routine
      returns with INFO = i. Otherwise, the factored form of A is used
      to estimate the condition number of the matrix A.  If the
      reciprocal of the condition number is less than machine precision,
      INFO = N+1 is returned as a warning, but the routine still goes on
      to solve for X and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(C) (if TRANS = 'N') or diag(R) (if TRANS = 'T' or 'C') so
      that it solves the original system before equilibration.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AF and IPIV contain the factored form of A.
           If EQUED is not 'N', the matrix A has been
           equilibrated with scaling factors given by R and C.
           A, AF, and IPIV are not modified.
   = 'N':  The matrix A will be copied to AF and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
           copied to AF and factored.

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Transpose)

 N
```

```
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the N-by-N matrix A.  If FACT = 'F' and EQUED is
  not 'N', then A must have been equilibrated by the scaling
  factors in R and/or C.  A is not modified if FACT = 'F' or
  'N', or if FACT = 'E' and EQUED = 'N' on exit.

  On exit, if EQUED .ne. 'N', A is scaled as follows:
  EQUED = 'R':  A := diag(R) * A
  EQUED = 'C':  A := A * diag(C)
  EQUED = 'B':  A := diag(R) * A * diag(C).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is DOUBLE PRECISION array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the factors L and U from the factorization
  A = P*L*U as computed by DGETRF.  If EQUED .ne. 'N', then
  AF is the factored form of the equilibrated matrix A.

  If FACT = 'N', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the equilibrated matrix A (see the description of A for
  the form of the equilibrated matrix).

LDAF

  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains the pivot indices from the factorization A = P*L*U
  as computed by DGETRF; row i of the matrix was interchanged
```

```
  with row IPIV(i).

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the equilibrated matrix A.

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
  = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
  = 'B':  Both row and column equilibration, i.e., A has been
          replaced by diag(R) * A * diag(C).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

R
  R is DOUBLE PRECISION array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
  EQUED = 'R' or 'B', each element of R must be positive.

C
  C is DOUBLE PRECISION array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if TRANS = 'N' and EQUED = 'R' or 'B', B is overwritten by diag(R)*B;
  if TRANS = 'T' or 'C' and EQUED = 'C' or 'B', B is
  overwritten by diag(C)*B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).
```

```
X
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X
  to the original system of equations.  Note that A and B are
  modified on exit if EQUED .ne. 'N', and the solution to the
  equilibrated system is inv(diag(C))*X if TRANS = 'N' and
  EQUED = 'C' or 'B', or inv(diag(R))*X if TRANS = 'T' or 'C'
  and EQUED = 'R' or 'B'.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  The estimate of the reciprocal condition number of the matrix
  A after equilibration (if done).  If RCOND is less than the
  machine precision (in particular, if RCOND = 0), the matrix
  is singular to working precision.  This condition is
  indicated by a return code of INFO > 0.

FERR
  FERR is DOUBLE PRECISION array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is DOUBLE PRECISION array, dimension (4*N)
  On exit, WORK(1) contains the reciprocal pivot growth
  factor norm(A)/norm(U). The "max absolute element" norm is
  used. If WORK(1) is much less than 1, then the stability
  of the LU factorization of the (equilibrated) matrix A
  could be poor. This also means that the solution X, condition
  estimator RCOND, and forward error bound FERR could be
  unreliable. If factorization fails with 0<INFO<=N, then
  WORK(1) contains the reciprocal pivot growth factor for the
  leading INFO columns of A.
```

```
 IWORK
   IWORK is INTEGER array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, and i is
     <= N:  U(i,i) is exactly zero.  The factorization has
            been completed, but the factor U is exactly
            singular, so the solution and error bounds
            could not be computed. RCOND = 0 is returned.
     = N+1: U is nonsingular, but RCOND is less than machine
            precision, meaning that the matrix is singular
            to working precision.  Nevertheless, the
            solution and error bounds are computed because
            there are a number of situations where the
            computed solution can be more accurate than the
            value of RCOND would suggest.
```

20.7.3 fortran code

— dgesvx.f —

```
*  =====================================================================
      SUBROUTINE DGESVX( FACT, TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $                   EQUED, R, C, B, LDB, X, LDX, RCOND, FERR, BERR,
     $                   WORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, TRANS
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   BERR( * ), C( * ), FERR( * ), R( * ),
     $                   WORK( * ), X( LDX, * )
*     ..
```

```
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            COLEQU, EQUIL, NOFACT, NOTRAN, ROWEQU
      CHARACTER          NORM
      INTEGER            I, INFEQU, J
      DOUBLE PRECISION   AMAX, ANORM, BIGNUM, COLCND, RCMAX, RCMIN,
     $                   ROWCND, RPVGRW, SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, DLANGE, DLANTR
      EXTERNAL           LSAME, DLAMCH, DLANGE, DLANTR
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGECON, DGEEQU, DGERFS, DGETRF, DGETRS, DLACPY,
     $                   DLAQGE, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      NOTRAN = LSAME( TRANS, 'N' )
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         ROWEQU = .FALSE.
         COLEQU = .FALSE.
      ELSE
         ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
         COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         SMLNUM = DLAMCH( 'Safe minimum' )
         BIGNUM = ONE / SMLNUM
      END IF
*
*     Test the input parameters.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.LSAME( FACT, 'F' ) )
     $     THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $         LSAME( TRANS, 'C' ) ) THEN
```

```
      INFO = -2
   ELSE IF( N.LT.0 ) THEN
      INFO = -3
   ELSE IF( NRHS.LT.0 ) THEN
      INFO = -4
   ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
      INFO = -6
   ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
      INFO = -8
   ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
  $         ( ROWEQU .OR. COLEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
      INFO = -10
   ELSE
      IF( ROWEQU ) THEN
         RCMIN = BIGNUM
         RCMAX = ZERO
         DO 10 J = 1, N
            RCMIN = MIN( RCMIN, R( J ) )
            RCMAX = MAX( RCMAX, R( J ) )
10       CONTINUE
         IF( RCMIN.LE.ZERO ) THEN
            INFO = -11
         ELSE IF( N.GT.0 ) THEN
            ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
         ELSE
            ROWCND = ONE
         END IF
      END IF
      IF( COLEQU .AND. INFO.EQ.0 ) THEN
         RCMIN = BIGNUM
         RCMAX = ZERO
         DO 20 J = 1, N
            RCMIN = MIN( RCMIN, C( J ) )
            RCMAX = MAX( RCMAX, C( J ) )
20       CONTINUE
         IF( RCMIN.LE.ZERO ) THEN
            INFO = -12
         ELSE IF( N.GT.0 ) THEN
            COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
         ELSE
            COLCND = ONE
         END IF
      END IF
      IF( INFO.EQ.0 ) THEN
         IF( LDB.LT.MAX( 1, N ) ) THEN
            INFO = -14
         ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
            INFO = -16
         END IF
      END IF
```

```
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGESVX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*        Compute row and column scalings to equilibrate the matrix A.
*
         CALL DGEEQU( N, N, A, LDA, R, C, ROWCND, COLCND, AMAX, INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*           Equilibrate the matrix.
*
            CALL DLAQGE( N, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $                   EQUED )
            ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
            COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         END IF
      END IF
*
*     Scale the right hand side.
*
      IF( NOTRAN ) THEN
         IF( ROWEQU ) THEN
            DO 40 J = 1, NRHS
               DO 30 I = 1, N
                  B( I, J ) = R( I )*B( I, J )
   30          CONTINUE
   40       CONTINUE
         END IF
      ELSE IF( COLEQU ) THEN
         DO 60 J = 1, NRHS
            DO 50 I = 1, N
               B( I, J ) = C( I )*B( I, J )
   50       CONTINUE
   60    CONTINUE
      END IF
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the LU factorization of A.
*
         CALL DLACPY( 'Full', N, N, A, LDA, AF, LDAF )
         CALL DGETRF( N, N, AF, LDAF, IPIV, INFO )
*
*        Return if INFO is non-zero.
*
```

```
         IF( INFO.GT.0 ) THEN
*
*           Compute the reciprocal pivot growth factor of the
*           leading rank-deficient INFO columns of A.
*
            RPVGRW = DLANTR( 'M', 'U', 'N', INFO, INFO, AF, LDAF,
     $               WORK )
            IF( RPVGRW.EQ.ZERO ) THEN
               RPVGRW = ONE
            ELSE
               RPVGRW = DLANGE( 'M', N, INFO, A, LDA, WORK ) / RPVGRW
            END IF
            WORK( 1 ) = RPVGRW
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A and the
*     reciprocal pivot growth factor RPVGRW.
*
      IF( NOTRAN ) THEN
         NORM = '1'
      ELSE
         NORM = 'I'
      END IF
      ANORM = DLANGE( NORM, N, N, A, LDA, WORK )
      RPVGRW = DLANTR( 'M', 'U', 'N', N, N, AF, LDAF, WORK )
      IF( RPVGRW.EQ.ZERO ) THEN
         RPVGRW = ONE
      ELSE
         RPVGRW = DLANGE( 'M', N, N, A, LDA, WORK ) / RPVGRW
      END IF
*
*     Compute the reciprocal of the condition number of A.
*
      CALL DGECON( NORM, N, AF, LDAF, ANORM, RCOND, WORK, IWORK, INFO )
*
*     Compute the solution matrix X.
*
      CALL DLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL DGETRS( TRANS, N, NRHS, AF, LDAF, IPIV, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL DGERFS( TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB, X,
     $             LDX, FERR, BERR, WORK, IWORK, INFO )
*
*     Transform the solution matrix X to a solution of the original
```

```
*     system.
*
      IF( NOTRAN ) THEN
         IF( COLEQU ) THEN
            DO 80 J = 1, NRHS
               DO 70 I = 1, N
                  X( I, J ) = C( I )*X( I, J )
   70          CONTINUE
   80       CONTINUE
            DO 90 J = 1, NRHS
               FERR( J ) = FERR( J ) / COLCND
   90       CONTINUE
         END IF
      ELSE IF( ROWEQU ) THEN
         DO 110 J = 1, NRHS
            DO 100 I = 1, N
               X( I, J ) = R( I )*X( I, J )
  100       CONTINUE
  110    CONTINUE
         DO 120 J = 1, NRHS
            FERR( J ) = FERR( J ) / ROWCND
  120    CONTINUE
      END IF
*
      WORK( 1 ) = RPVGRW
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.DLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
      RETURN
*
*     End of DGESVX
*
      END
```

———-

20.7.4 lisp code

— **dgesvx.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dgesvx
         (fact trans n nrhs a lda af ldaf ipiv equed r c b ldb$ x ldx rcond
```

```
      ferr berr work iwork info)
(declare (type (double-float) rcond)
         (type (array f2cl-lib:integer4 (*)) iwork ipiv)
         (type (array double-float (*)) work berr ferr x b c r af a)
         (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
         (type (string *) equed trans fact))
(f2cl-lib:with-multi-array-data
    ((fact character fact-%data% fact-%offset%)
     (trans character trans-%data% trans-%offset%)
     (equed character equed-%data% equed-%offset%)
     (a double-float a-%data% a-%offset%)
     (af double-float af-%data% af-%offset%)
     (r double-float r-%data% r-%offset%)
     (c double-float c-%data% c-%offset%)
     (b double-float b-%data% b-%offset%)
     (x double-float x-%data% x-%offset%)
     (ferr double-float ferr-%data% ferr-%offset%)
     (berr double-float berr-%data% berr-%offset%)
     (work double-float work-%data% work-%offset%)
     (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
     (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((amax 0.0d0) (anorm 0.0d0) (bignum 0.0d0) (colcnd 0.0d0)
         (rcmax 0.0d0) (rcmin 0.0d0) (rowcnd 0.0d0) (rpvgrw 0.0d0)
         (smlnum 0.0d0) (i 0) (infequ 0) (j 0)
         (norm
          (make-array '(1) :element-type 'character :initial-element #\ ))
         (colequ nil) (equil nil) (nofact nil) (notran nil) (rowequ nil))
    (declare
     (type (double-float) amax anorm bignum colcnd rcmax rcmin rowcnd
      rpvgrw smlnum)
     (type (f2cl-lib:integer4) i infequ j)
     (type (string 1) norm)
     (type f2cl-lib:logical colequ equil nofact notran rowequ))
    (setf info 0)
    (setf nofact
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "N")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val))
    (setf equil
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "E")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val))
    (setf notran
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "N")
              (declare (ignore var-1))
```

```
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
  (setf rowequ f2cl-lib:%false%) (setf colequ f2cl-lib:%false%))
 (t
  (setf rowequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "R")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)))
  (setf colequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "C")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)))
  (setf smlnum (dlamch "Safe minimum")) (setf bignum (/ one smlnum))))
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
```

```
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
 (setf info -2))
((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
 (setf info -6))
((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
 (setf info -8))
((and
  (multiple-value-bind (ret-val var-0 var-1)
      (lsame fact "F")
    (declare (ignore var-1))
    (when var-0 (setf fact var-0))
    ret-val)
  (not
   (or rowequ colequ
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame equed "N")
         (declare (ignore var-1))
         (when var-0 (setf equed var-0))
         ret-val))))
 (setf info -10))
(t
 (cond
  (rowequ (setf rcmin bignum) (setf rcmax zero)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf rcmin
               (min rcmin
                    (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
       (setf rcmax
               (max rcmax
                    (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
      label10))
   (cond ((<= rcmin zero) (setf info -11))
         ((> n 0)
          (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum))))
         (t (setf rowcnd one)))))
 (cond
  ((and colequ (= info 0)) (setf rcmin bignum) (setf rcmax zero)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf rcmin
               (min rcmin
                    (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
       (setf rcmax
               (max rcmax
```

```
                        (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
        label20))
      (cond ((<= rcmin zero) (setf info -12))
            ((> n 0)
             (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum))))
            (t (setf colcnd one)))))
    (cond
     ((= info 0)
      (cond
       ((< ldb$
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -14))
       ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -16)))))))
  (cond
   ((/= info 0) (xerbla "DGESVX" (f2cl-lib:int-sub info))
    (go end_label)))
  (cond
   (equil
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dgeequ n n a lda r c rowcnd colcnd amax infequ)
      (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5))
      (setf rowcnd var-6)
      (setf colcnd var-7)
      (setf amax var-8)
      (setf infequ var-9))
    (cond
     ((= infequ 0)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (dlaqge n n a lda r c rowcnd colcnd amax equed)
        (declare (ignore var-2 var-4 var-5))
        (when var-0 (setf n var-0))
        (when var-1 (setf n var-1))
        (when var-3 (setf lda var-3))
        (when var-6 (setf rowcnd var-6))
        (when var-7 (setf colcnd var-7))
        (when var-8 (setf amax var-8))
        (when var-9 (setf equed var-9)))
      (setf rowequ
              (or
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame equed "R")
                 (declare (ignore var-1))
                 (when var-0 (setf equed var-0))
                 ret-val)
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame equed "B")
                 (declare (ignore var-1))
```

```
                (when var-0 (setf equed var-0))
                ret-val)))
    (setf colequ
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "C")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "B")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)))))))
(cond
 (notran
  (cond
   (rowequ
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)))
           label30))
       label40)))))
 (colequ
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)
                (* (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)))
         label50))
     label60))))
(cond
 ((or nofact equil)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "Full" n n a lda af ldaf)
    (declare (ignore var-0 var-3 var-5))
```

```
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldaf var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (dgetrf n n af ldaf ipiv info)
    (declare (ignore var-2 var-4))
    (setf n var-0)
    (setf n var-1)
    (setf ldaf var-3)
    (setf info var-5))
  (cond
   ((> info 0)
    (setf rpvgrw
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6
                 var-7)
                (dlantr "M" "U" "N" info info af ldaf work)
              (declare (ignore var-0 var-1 var-2 var-5 var-7))
              (when var-3 (setf info var-3))
              (when var-4 (setf info var-4))
              (when var-6 (setf ldaf var-6))
              ret-val))
    (cond ((= rpvgrw zero) (setf rpvgrw one))
          (t
           (setf rpvgrw
                   (/
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                        (dlange "M" n info a lda work)
                      (declare (ignore var-0 var-3 var-5))
                      (when var-1 (setf n var-1))
                      (when var-2 (setf info var-2))
                      (when var-4 (setf lda var-4))
                      ret-val)
                    rpvgrw))))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) rpvgrw)
    (setf rcond zero) (go end_label)))))
(cond (notran (f2cl-lib:f2cl-set-string norm "1" (string 1)))
      (t (f2cl-lib:f2cl-set-string norm "I" (string 1))))
(setf anorm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange norm n n a lda work)
          (declare (ignore var-3 var-5))
          (when var-0 (setf norm var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
```

```
(setf rpvgrw
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (dlantr "M" "U" "N" n n af ldaf work)
          (declare (ignore var-0 var-1 var-2 var-5 var-7))
          (when var-3 (setf n var-3))
          (when var-4 (setf n var-4))
          (when var-6 (setf ldaf var-6))
          ret-val))
(cond ((= rpvgrw zero) (setf rpvgrw one))
      (t
       (setf rpvgrw
               (/
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                    (dlange "M" n n a lda work)
                  (declare (ignore var-0 var-3 var-5))
                  (when var-1 (setf n var-1))
                  (when var-2 (setf n var-2))
                  (when var-4 (setf lda var-4))
                  ret-val)
                rpvgrw))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dgecon norm n af ldaf anorm rcond work iwork info)
  (declare (ignore var-2 var-4 var-6 var-7))
  (setf norm var-0)
  (setf n var-1)
  (setf ldaf var-3)
  (setf rcond var-5)
  (setf info var-8))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "Full" n nrhs b ldb$ x ldx)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf ldx var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dgetrs trans n nrhs af ldaf ipiv x ldx info)
  (declare (ignore var-3 var-5 var-6))
  (setf trans var-0)
  (setf n var-1)
  (setf nrhs var-2)
  (setf ldaf var-4)
  (setf ldx var-7)
  (setf info var-8))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
```

```
      var-11 var-12 var-13 var-14 var-15 var-16)
     (dgerfs trans n nrhs a lda af ldaf ipiv b ldb$ x ldx ferr berr work
      iwork info)
   (declare
    (ignore var-2 var-3 var-5 var-7 var-8 var-9 var-10 var-11 var-12
     var-13 var-14 var-15))
   (setf trans var-0)
   (setf n var-1)
   (setf lda var-4)
   (setf ldaf var-6)
   (setf info var-16))
 (cond
  (notran
   (cond
    (colequ
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j nrhs) nil)
       (tagbody
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                   x-%offset%)
                     (* (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)
                        (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                       x-%offset%)))
           label70))
       label80))
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j nrhs) nil)
       (tagbody
         (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                 (/
                  (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                  colcnd))
       label90)))))
  (rowequ
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j nrhs) nil)
     (tagbody
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                 x-%offset%)
                   (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                      (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                     x-%offset%)))
         label100))
     label110))
```

```
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j nrhs) nil)
            (tagbody
              (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                      (/ (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                         rowcnd))
             label120))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) rpvgrw)
        (if (< rcond (dlamch "Epsilon"))
            (setf info (f2cl-lib:int-add n 1)))
        (go end_label)
       end_label
        (return
         (values fact trans n nrhs nil lda nil ldaf nil equed nil nil nil ldb$
                 nil ldx rcond nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgesvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (string) (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::fact
                                              fortran-to-lisp::trans
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
```

```
  fortran-to-lisp::ldaf nil
  fortran-to-lisp::equed nil nil
  nil fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond nil nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::dgetrs
  fortran-to-lisp::dgetrf
  fortran-to-lisp::dgerfs
  fortran-to-lisp::dgecon
  fortran-to-lisp::dgeequ))))
```

———-

20.8 dgesvxx

20.8.1 Axiom unit tests

— **dgesvxx.input** —

```
)set break resume
)sys rm -f dgesvxx.output
)spool dgesvxx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

20.8.2 Axiom help page

— **dgesvxx.help** —

```
==================================================================
dgesvxx examples
==================================================================

==================================================================
```

```
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgesvxx (FACT, TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV,
           EQUED, R, C, B, LDB, X, LDX, RCOND, RPVGRW, BERR, N_ERR_BNDS,
           ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, IWORK, INFO)
            DGESVXX computes the solution to system of linear equations A * X
           = B for GE matrices

Function/Subroutine Documentation
   subroutine dgesvxx (character FACT, character TRANS, integer N, integer
       NRHS, double precision, dimension( lda, * ) A, integer LDA, double
       precision, dimension( ldaf, * ) AF, integer LDAF, integer, dimension( *
       ) IPIV, character EQUED, double precision, dimension( * ) R, double
       precision, dimension( * ) C, double precision, dimension( ldb, * ) B,
       integer LDB, double precision, dimension( ldx , * ) X, integer LDX,
       double precision RCOND, double precision RPVGRW, double precision,
       dimension( * ) BERR, integer N_ERR_BNDS, double precision, dimension(
       nrhs, * ) ERR_BNDS_NORM, double precision, dimension( nrhs, * )
       ERR_BNDS_COMP, integer NPARAMS, double precision, dimension( * )
       PARAMS, double precision, dimension( * ) WORK, integer, dimension( * )
       IWORK, integer INFO)
        DGESVXX computes the solution to system of linear equations A * X = B
       for GE matrices

Purpose:
   DGESVXX uses the LU factorization to compute the solution to a
   double precision system of linear equations  A * X = B,  where A is an
   N-by-N matrix and X and B are N-by-NRHS matrices.

   If requested, both normwise and maximum componentwise error bounds
   are returned. DGESVXX will return a solution with a tiny
   guaranteed error (O(eps) where eps is the working machine
   precision) unless the matrix is very ill-conditioned, in which
   case a warning is returned. Relevant condition numbers also are
   calculated and returned.

   DGESVXX accepts user-provided factorizations and equilibration
   factors; see the definitions of the FACT and EQUED options.
   Solving with refinement and using a factorization from a previous
   DGESVXX call will also produce a solution with either O(eps)
   errors or warnings, but we cannot make that claim for general
   user-provided factorizations and equilibration factors if they
   differ from what DGESVXX would itself produce.

Description:
   The following steps are performed:
```

```
   1. If FACT = 'E', double precision scaling factors are computed to
      equilibrate the system:
      TRANS = 'N':  diag(R)*A*diag(C)     *inv(diag(C))*X = diag(R)*B
      TRANS = 'T': (diag(R)*A*diag(C))**T *inv(diag(R))*X = diag(C)*B
      TRANS = 'C': (diag(R)*A*diag(C))**H *inv(diag(R))*X = diag(C)*B

   Whether or not the system will be equilibrated depends on the
   scaling of the matrix A, but if equilibration is used, A is
   overwritten by diag(R)*A*diag(C) and B by diag(R)*B (if TRANS='N')
   or diag(C)*B (if TRANS = 'T' or 'C').

   2. If FACT = 'N' or 'E', the LU decomposition is used to factor
      the matrix A (after equilibration if FACT = 'E') as
                 A = P * L * U,
      where P is a permutation matrix, L is a unit lower triangular
      matrix, and U is upper triangular.

   3. If some U(i,i)=0, so that U is exactly singular, then the
      routine returns with INFO = i. Otherwise, the factored form of A
      is used to estimate the condition number of the matrix A (see
      argument RCOND). If the reciprocal of the condition number is less
      than machine precision, the routine still goes on to solve for X
      and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. By default (unless PARAMS(LA_LINRX_ITREF_I) is set to zero),
      the routine will use iterative refinement to try to get a small
      error and error bounds.  Refinement calculates the residual to at
      least twice the working precision.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(C) (if TRANS = 'N') or diag(R) (if TRANS = 'T' or 'C') so
      that it solves the original system before equilibration.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AF and IPIV contain the factored form of A.
           If EQUED is not 'N', the matrix A has been
```

```
          equilibrated with scaling factors given by R and C.
          A, AF, and IPIV are not modified.
  = 'N':  The matrix A will be copied to AF and factored.
  = 'E':  The matrix A will be equilibrated if necessary, then
          copied to AF and factored.

TRANS
  TRANS is CHARACTER*1
  Specifies the form of the system of equations:
  = 'N':  A * X = B     (No transpose)
  = 'T':  A**T * X = B  (Transpose)
  = 'C':  A**H * X = B  (Conjugate Transpose = Transpose)

N
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the N-by-N matrix A.  If FACT = 'F' and EQUED is
  not 'N', then A must have been equilibrated by the scaling
  factors in R and/or C.  A is not modified if FACT = 'F' or
  'N', or if FACT = 'E' and EQUED = 'N' on exit.

  On exit, if EQUED .ne. 'N', A is scaled as follows:
  EQUED = 'R':  A := diag(R) * A
  EQUED = 'C':  A := A * diag(C)
  EQUED = 'B':  A := diag(R) * A * diag(C).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is DOUBLE PRECISION array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the factors L and U from the factorization
  A = P*L*U as computed by DGETRF.  If EQUED .ne. 'N', then
  AF is the factored form of the equilibrated matrix A.

  If FACT = 'N', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then AF is an output argument and on exit
```

```
  returns the factors L and U from the factorization A = P*L*U
  of the equilibrated matrix A (see the description of A for
  the form of the equilibrated matrix).

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains the pivot indices from the factorization A = P*L*U
  as computed by DGETRF; row i of the matrix was interchanged
  with row IPIV(i).

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the equilibrated matrix A.

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
  = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
  = 'B':  Both row and column equilibration, i.e., A has been
          replaced by diag(R) * A * diag(C).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

R
  R is DOUBLE PRECISION array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
  EQUED = 'R' or 'B', each element of R must be positive.
  If R is output, each element of R is a power of the radix.
  If R is input, each element of R should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

C
```

```
  C is DOUBLE PRECISION array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.
  If C is output, each element of C is a power of the radix.
  If C is input, each element of C should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if TRANS = 'N' and EQUED = 'R' or 'B', B is overwritten by diag(R)*B;
  if TRANS = 'T' or 'C' and EQUED = 'C' or 'B', B is overwritten by diag(C)*B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  If INFO = 0, the N-by-NRHS solution matrix X to the original
  system of equations.  Note that A and B are modified on exit
  if EQUED .ne. 'N', and the solution to the equilibrated system is
  inv(diag(C))*X if TRANS = 'N' and EQUED = 'C' or 'B', or
  inv(diag(R))*X if TRANS = 'T' or 'C' and EQUED = 'R' or 'B'.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

RPVGRW
  RPVGRW is DOUBLE PRECISION
  Reciprocal pivot growth.  On exit, this contains the reciprocal
```

```
  pivot growth factor norm(A)/norm(U). The "max absolute element"
  norm is used.  If this is much less than 1, then the stability of
  the LU factorization of the (equilibrated) matrix A could be poor.
  This also means that the solution X, estimated condition numbers,
  and error bounds could be unreliable. If factorization fails with
  0<INFO<=N, then this contains the reciprocal pivot growth factor
  for the leading INFO columns of A.  In DGESVX, this quantity is
  returned in WORK(1).

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
                      max_j (abs(XTRUE(j,i) - X(j,i)))
                     ------------------------------
                            max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
          reciprocal condition number is less than the threshold
          sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
          almost certainly within a factor of 10 of the true error
          so long as the next entry is greater than the threshold
          sqrt(n) * dlamch('Epsilon'). This error bound should only
```

```
          be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
           reciprocal condition number.  Compared with the threshold
           sqrt(n) * dlamch('Epsilon') to determine if the error
           estimate is "guaranteed". These reciprocal condition
           numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
           appropriately scaled matrix Z.
           Let Z = S*A, where S scales each row by a power of the
           radix so all absolute row sums of Z are approximately 1.

  See Lapack Working Note 165 for further details and extra cautions.

ERR_BNDS_COMP
  ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                             abs(XTRUE(j,i) - X(j,i))
                      max_j ----------------------
                                   abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
           reciprocal condition number is less than the threshold
           sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
           almost certainly within a factor of 10 of the true error
           so long as the next entry is greater than the threshold
           sqrt(n) * dlamch('Epsilon'). This error bound should only
           be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
           reciprocal condition number.  Compared with the threshold
           sqrt(n) * dlamch('Epsilon') to determine if the error
```

```
          estimate is "guaranteed". These reciprocal condition
          numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
          appropriately scaled matrix Z.
          Let Z = S*(A*diag(x)), where x is the solution for the
          current right-hand side and S scales each row of
          A*diag(x) by a power of the radix so all absolute row
          sums of Z are approximately 1.

  See Lapack Working Note 165 for further details and extra cautions.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is DOUBLE PRECISION array, dimension (NPARAMS)
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0D+0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the extra-precise refinement algorithm.
          (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is DOUBLE PRECISION array, dimension (4*N)

IWORK
  IWORK is INTEGER array, dimension (N)
```

```
 INFO
   INFO is INTEGER
   = 0:  Successful exit. The solution to every right-hand side is guaranteed.
   < 0:  If INFO = -i, the i-th argument had an illegal value
   > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
       has been completed, but the factor U is exactly singular, so
       the solution and error bounds could not be computed. RCOND = 0
       is returned.
   = N+J: The solution corresponding to the Jth right-hand side is
       not guaranteed. The solutions corresponding to other right-
       hand sides K with K > J may not be guaranteed as well, but
       only the first such right-hand side is reported. If a small
       componentwise error is not requested (PARAMS(3) = 0.0) then
       the Jth right-hand side is the first with a normwise error
       bound that is not guaranteed (the smallest J such
       that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
       the Jth right-hand side is the first with either a normwise or
       componentwise error bound that is not guaranteed (the smallest
       J such that either ERR_BNDS_NORM(J,1) = 0.0 or
       ERR_BNDS_COMP(J,1) = 0.0). See the definition of
       ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
       about all of the right-hand sides check ERR_BNDS_NORM or
       ERR_BNDS_COMP.
```

———-

20.8.3 fortran code

— **dgesvxx.f** —

```
*  ==================================================================
      SUBROUTINE DGESVXX( FACT, TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $                    EQUED, R, C, B, LDB, X, LDX, RCOND, RPVGRW,
     $                    BERR, N_ERR_BNDS, ERR_BNDS_NORM,
     $                    ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, IWORK,
     $                    INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, TRANS
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      DOUBLE PRECISION   RCOND, RPVGRW
```

```
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX , * ),WORK( * )
      DOUBLE PRECISION   R( * ), C( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
*     ..
*     .. Local Scalars ..
      LOGICAL            COLEQU, EQUIL, NOFACT, NOTRAN, ROWEQU
      INTEGER            INFEQU, J
      DOUBLE PRECISION   AMAX, BIGNUM, COLCND, RCMAX, RCMIN, ROWCND,
     $                   SMLNUM
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, DLAMCH, DLA_GERPVGRW
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, DLA_GERPVGRW
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEEQUB, DGETRF, DGETRS, DLACPY, DLAQGE,
     $                   XERBLA, DLASCL2, DGERFSX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      NOTRAN = LSAME( TRANS, 'N' )
      SMLNUM = DLAMCH( 'Safe minimum' )
      BIGNUM = ONE / SMLNUM
```

```
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         ROWEQU = .FALSE.
         COLEQU = .FALSE.
      ELSE
         ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
         COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
      END IF
*
*     Default is failure.  If an input parameter is wrong or
*     factorization fails, make everything look horrible.  Only the
*     pivot growth is set here, the rest is initialized in DGERFSX.
*
      RPVGRW = ZERO
*
*     Test the input parameters.  PARAMS is not tested until DGERFSX.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.
     $     LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $        LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $        ( ROWEQU .OR. COLEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -10
      ELSE
         IF( ROWEQU ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 10 J = 1, N
               RCMIN = MIN( RCMIN, R( J ) )
               RCMAX = MAX( RCMAX, R( J ) )
 10         CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -11
            ELSE IF( N.GT.0 ) THEN
               ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               ROWCND = ONE
            END IF
         END IF
```

```
         IF( COLEQU .AND. INFO.EQ.0 ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 20 J = 1, N
               RCMIN = MIN( RCMIN, C( J ) )
               RCMAX = MAX( RCMAX, C( J ) )
 20         CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -12
            ELSE IF( N.GT.0 ) THEN
               COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               COLCND = ONE
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -14
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -16
            END IF
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGESVXX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*     Compute row and column scalings to equilibrate the matrix A.
*
         CALL DGEEQUB( N, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $        INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*     Equilibrate the matrix.
*
            CALL DLAQGE( N, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $           EQUED )
            ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
            COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         END IF
*
*     If the scaling factors are not applied, set them to 1.0.
*
         IF ( .NOT.ROWEQU ) THEN
            DO J = 1, N
               R( J ) = 1.0D+0
```

```
            END DO
         END IF
         IF ( .NOT.COLEQU ) THEN
            DO J = 1, N
               C( J ) = 1.0D+0
            END DO
         END IF
      END IF
*
*     Scale the right-hand side.
*
      IF( NOTRAN ) THEN
         IF( ROWEQU ) CALL DLASCL2( N, NRHS, R, B, LDB )
      ELSE
         IF( COLEQU ) CALL DLASCL2( N, NRHS, C, B, LDB )
      END IF
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the LU factorization of A.
*
         CALL DLACPY( 'Full', N, N, A, LDA, AF, LDAF )
         CALL DGETRF( N, N, AF, LDAF, IPIV, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 ) THEN
*
*           Pivot in column INFO is exactly 0
*           Compute the reciprocal pivot growth factor of the
*           leading rank-deficient INFO columns of A.
*
            RPVGRW = DLA_GERPVGRW( N, INFO, A, LDA, AF, LDAF )
            RETURN
         END IF
      END IF
*
*     Compute the reciprocal pivot growth factor RPVGRW.
*
      RPVGRW = DLA_GERPVGRW( N, N, A, LDA, AF, LDAF )
*
*     Compute the solution matrix X.
*
      CALL DLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL DGETRS( TRANS, N, NRHS, AF, LDAF, IPIV, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL DGERFSX( TRANS, EQUED, N, NRHS, A, LDA, AF, LDAF,
```

```
     $     IPIV, R, C, B, LDB, X, LDX, RCOND, BERR,
     $     N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $     WORK, IWORK, INFO )
*
*     Scale solutions.
*
      IF ( COLEQU .AND. NOTRAN ) THEN
         CALL DLASCL2 ( N, NRHS, C, X, LDX )
      ELSE IF ( ROWEQU .AND. .NOT.NOTRAN ) THEN
         CALL DLASCL2 ( N, NRHS, R, X, LDX )
      END IF
*
      RETURN
*
*     End of DGESVXX

      END
```

———-

20.8.4 lisp code

— dgesvxx.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dgesvxx
         (fact trans n nrhs a lda af ldaf ipiv equed r c b ldb$ x ldx rcond
          rpvgrw berr n_err_bnds err_bnds_norm err_bnds_comp nparams params
          work iwork info)
    (declare (type (double-float) rpvgrw rcond)
             (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array double-float (*)) work params err_bnds_comp
              err_bnds_norm berr x b c r af a)
             (type (f2cl-lib:integer4) info nparams n_err_bnds ldx ldb$ ldaf
              lda nrhs n)
             (type (string *) equed trans fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (trans character trans-%data% trans-%offset%)
         (equed character equed-%data% equed-%offset%)
         (a double-float a-%data% a-%offset%)
         (af double-float af-%data% af-%offset%)
         (r double-float r-%data% r-%offset%)
         (c double-float c-%data% c-%offset%)
         (b double-float b-%data% b-%offset%)
```

```
   (x double-float x-%data% x-%offset%)
   (berr double-float berr-%data% berr-%offset%)
   (err_bnds_norm double-float err_bnds_norm-%data%
    err_bnds_norm-%offset%)
   (err_bnds_comp double-float err_bnds_comp-%data%
    err_bnds_comp-%offset%)
   (params double-float params-%data% params-%offset%)
   (work double-float work-%data% work-%offset%)
   (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
   (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
(prog ((amax 0.0d0) (bignum 0.0d0) (colcnd 0.0d0) (rcmax 0.0d0)
       (rcmin 0.0d0) (rowcnd 0.0d0) (smlnum 0.0d0) (infequ 0) (j 0)
       (colequ nil) (equil nil) (nofact nil) (notran nil) (rowequ nil))
  (declare
   (type (double-float) amax bignum colcnd rcmax rcmin rowcnd smlnum)
   (type (f2cl-lib:integer4) infequ j)
   (type f2cl-lib:logical colequ equil nofact notran rowequ))
  (setf info 0)
  (setf nofact
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame fact "N")
            (declare (ignore var-1))
            (when var-0 (setf fact var-0))
            ret-val))
  (setf equil
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame fact "E")
            (declare (ignore var-1))
            (when var-0 (setf fact var-0))
            ret-val))
  (setf notran
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame trans "N")
            (declare (ignore var-1))
            (when var-0 (setf trans var-0))
            ret-val))
  (setf smlnum (dlamch "Safe minimum"))
  (setf bignum (/ one smlnum))
  (cond
   ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
    (setf rowequ f2cl-lib:%false%) (setf colequ f2cl-lib:%false%))
   (t
    (setf rowequ
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "R")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
```

```
                (lsame equed "B")
              (declare (ignore var-1))
              (when var-0 (setf equed var-0))
              ret-val)))
  (setf colequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "C")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)))))
(setf rpvgrw zero)
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame fact "F")
     (declare (ignore var-1))
     (when var-0 (setf fact var-0))
```

```
    ret-val)
   (not
    (or rowequ colequ
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame equed "N")
          (declare (ignore var-1))
          (when var-0 (setf equed var-0))
          ret-val))))
  (setf info -10))
 (t
  (cond
   (rowequ (setf rcmin bignum) (setf rcmax zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf rcmin
                (min rcmin
                     (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
        (setf rcmax
                (max rcmax
                     (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
       label10))
    (cond ((<= rcmin zero) (setf info -11))
          ((> n 0)
           (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum))))
          (t (setf rowcnd one)))))
  (cond
   ((and colequ (= info 0)) (setf rcmin bignum) (setf rcmax zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf rcmin
                (min rcmin
                     (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
        (setf rcmax
                (max rcmax
                     (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
       label20))
    (cond ((<= rcmin zero) (setf info -12))
          ((> n 0)
           (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum))))
          (t (setf colcnd one)))))
  (cond
   ((= info 0)
    (cond
     ((< ldb$
         (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -14))
     ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -16)))))))
```

```
(cond
 ((/= info 0) (xerbla "DGESVXX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (equil
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dgeequb n n a lda r c rowcnd colcnd amax infequ)
    (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5))
    (setf rowcnd var-6)
    (setf colcnd var-7)
    (setf amax var-8)
    (setf infequ var-9))
  (cond
   ((= infequ 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlaqge n n a lda r c rowcnd colcnd amax equed)
      (declare (ignore var-2 var-4 var-5))
      (when var-0 (setf n var-0))
      (when var-1 (setf n var-1))
      (when var-3 (setf lda var-3))
      (when var-6 (setf rowcnd var-6))
      (when var-7 (setf colcnd var-7))
      (when var-8 (setf amax var-8))
      (when var-9 (setf equed var-9)))
    (setf rowequ
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "R")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "B")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)))
    (setf colequ
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "C")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "B")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)))))
```

```
  (cond
   ((not rowequ)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%) 1.0d0)
       label100000))))
  (cond
   ((not colequ)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) 1.0d0)
       label100001))))))
(cond
 (notran
  (if rowequ
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dlascl2 n nrhs r b ldb$)
        (declare (ignore var-2 var-3))
        (when var-0 (setf n var-0))
        (when var-1 (setf nrhs var-1))
        (when var-4 (setf ldb$ var-4)))))
 (t
  (if colequ
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dlascl2 n nrhs c b ldb$)
        (declare (ignore var-2 var-3))
        (when var-0 (setf n var-0))
        (when var-1 (setf nrhs var-1))
        (when var-4 (setf ldb$ var-4))))))
(cond
 ((or nofact equil)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "Full" n n a lda af ldaf)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldaf var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (dgetrf n n af ldaf ipiv info)
    (declare (ignore var-2 var-4))
    (setf n var-0)
    (setf n var-1)
    (setf ldaf var-3)
    (setf info var-5))
  (cond
   ((> info 0)
    (setf rpvgrw
```

```
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                (dla_gerpvgrw n info a lda af ldaf)
              (declare (ignore var-2 var-4))
              (when var-0 (setf n var-0))
              (when var-1 (setf info var-1))
              (when var-3 (setf lda var-3))
              (when var-5 (setf ldaf var-5))
              ret-val))
    (go end_label)))))
(setf rpvgrw
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dla_gerpvgrw n n a lda af ldaf)
          (declare (ignore var-2 var-4))
          (when var-0 (setf n var-0))
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-5 (setf ldaf var-5))
          ret-val))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "Full" n nrhs b ldb$ x ldx)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf ldx var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dgetrs trans n nrhs af ldaf ipiv x ldx info)
  (declare (ignore var-3 var-5 var-6))
  (setf trans var-0)
  (setf n var-1)
  (setf nrhs var-2)
  (setf ldaf var-4)
  (setf ldx var-7)
  (setf info var-8))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19
     var-20 var-21 var-22 var-23 var-24)
    (dgerfsx trans equed n nrhs a lda af ldaf ipiv r c b ldb$ x ldx
     rcond berr n_err_bnds err_bnds_norm err_bnds_comp nparams params
     work iwork info)
  (declare
   (ignore var-4 var-6 var-8 var-9 var-10 var-11 var-13 var-16 var-18
    var-19 var-21 var-22 var-23))
  (when var-0 (setf trans var-0))
  (when var-1 (setf equed var-1))
  (when var-2 (setf n var-2))
```

```
          (when var-3 (setf nrhs var-3))
          (when var-5 (setf lda var-5))
          (when var-7 (setf ldaf var-7))
          (when var-12 (setf ldb$ var-12))
          (when var-14 (setf ldx var-14))
          (when var-15 (setf rcond var-15))
          (when var-17 (setf n_err_bnds var-17))
          (when var-20 (setf nparams var-20))
          (when var-24 (setf info var-24)))
        (cond
         ((and colequ notran)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dlascl2 n nrhs c x ldx)
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf nrhs var-1))
            (when var-4 (setf ldx var-4))))
         ((and rowequ (not notran))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dlascl2 n nrhs r x ldx)
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf nrhs var-1))
            (when var-4 (setf ldx var-4)))))
        (go end_label)
       end_label
        (return
         (values fact trans n nrhs nil lda nil ldaf nil equed nil nil nil ldb$
                 nil ldx rcond rpvgrw nil n_err_bnds nil nil nparams nil nil
                 nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgesvxx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (string) (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
```

```
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (double-float) (double-float)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::fact
  fortran-to-lisp::trans
  fortran-to-lisp::n
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldaf nil
  fortran-to-lisp::equed nil nil
  nil fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond
  fortran-to-lisp::rpvgrw nil
  fortran-to-lisp::n_err_bnds nil
  nil fortran-to-lisp::nparams nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::dgetrs
  fortran-to-lisp::dgetrf
  fortran-to-lisp::dgeequb))))
```

———-

20.9 dsgesv

20.9.1 Axiom unit tests

— dsgesv.input —

```
)set break resume
)sys rm -f dsgesv.output
)spool dsgesv.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

20.9.2 Axiom help page

— dsgesv.help —

```
=====================================================================
dsgesv examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsgesv (N, NRHS, A, LDA, IPIV, B, LDB, X, LDX, WORK, SWORK,
           ITER, INFO)
            DSGESV computes the solution to system of linear equations A * X =
           B for GE matrices (mixed precision with iterative refinement)

Function/Subroutine Documentation
   subroutine dsgesv (integer N, integer NRHS, double precision, dimension(
       lda, * ) A, integer LDA, integer, dimension( * ) IPIV, double
       precision, dimension( ldb, * ) B, integer LDB, double precision,
       dimension( ldx, * ) X, integer LDX, double precision, dimension( n, * )
       WORK, real, dimension( * ) SWORK, integer ITER, integer INFO)
        DSGESV computes the solution to system of linear equations A * X = B
       for GE matrices (mixed precision with iterative refinement)

 Purpose:
   DSGESV computes the solution to a real system of linear equations
               A * X = B,
   where A is an N-by-N matrix and X and B are N-by-NRHS matrices.

   DSGESV first attempts to factorize the matrix in SINGLE PRECISION
   and use this factorization within an iterative refinement procedure
   to produce a solution with DOUBLE PRECISION normwise backward error
   quality (see below). If the approach fails the method switches to a
   DOUBLE PRECISION factorization and solve.

   The iterative refinement is not going to be a winning strategy if
```

```
   the ratio SINGLE PRECISION performance over DOUBLE PRECISION
   performance is too small. A reasonable strategy should take the
   number of right-hand sides and the size of the matrix into account.
   This might be done with a call to ILAENV in the future. Up to now, we
   always try iterative refinement.

   The iterative refinement process is stopped if
                ITER > ITERMAX
   or for all the RHS we have:
                RNRM < SQRT(N)*XNRM*ANRM*EPS*BWDMAX
   where
   o ITER is the number of the current iteration in the iterative
     refinement process
   o RNRM is the infinity-norm of the residual
   o XNRM is the infinity-norm of the solution
   o ANRM is the infinity-operator-norm of the matrix A
   o EPS is the machine epsilon returned by DLAMCH('Epsilon')
   The value ITERMAX and BWDMAX are fixed to 30 and 1.0D+00 respectively.

Parameters:

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is DOUBLE PRECISION array,
   dimension (LDA,N)
   On entry, the N-by-N coefficient matrix A.
   On exit, if iterative refinement has been successfully used
   (INFO.EQ.0 and ITER.GE.0, see description below), then A is
   unchanged, if double precision factorization has been used
   (INFO.EQ.0 and ITER.LT.0, see description below), then the
   array A contains the factors L and U from the factorization
   A = P*L*U; the unit diagonal elements of L are not stored.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   The pivot indices that define the permutation matrix P;
   row i of the matrix was interchanged with row IPIV(i).
```

```
  Corresponds either to the single precision factorization
  (if INFO.EQ.0 and ITER.GE.0) or the double precision
  factorization (if INFO.EQ.0 and ITER.LT.0).

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  The N-by-NRHS right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  If INFO = 0, the N-by-NRHS solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

WORK
  WORK is DOUBLE PRECISION array, dimension (N,NRHS)
  This array is used to hold the residual vectors.

SWORK
  SWORK is REAL array, dimension (N*(N+NRHS))
  This array is used to use the single precision matrix and the
  right-hand sides or solutions in single precision.

ITER
  ITER is INTEGER
  < 0: iterative refinement has failed, double precision
       factorization has been performed
  -1 : the routine fell back to full precision for
       implementation- or machine-specific reasons
  -2 : narrowing the precision induced an overflow,
       the routine fell back to full precision
  -3 : failure of SGETRF
  -31: stop the iterative refinement after the 30th iterations
   > 0: iterative refinement has been sucessfully used.
        Returns the number of iterations

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, U(i,i) computed in DOUBLE PRECISION is
        exactly zero.  The factorization has been completed,
        but the factor U is exactly singular, so the solution
        could not be computed.
```

———-

20.9.3 fortran code

— **dsgesv.f** —

```
*  ==================================================================
      SUBROUTINE DSGESV( N, NRHS, A, LDA, IPIV, B, LDB, X, LDX, WORK,
     $                   SWORK, ITER, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, ITER, LDA, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               SWORK( * )
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * ), WORK( N, * ),
     $                   X( LDX, * )
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
      LOGICAL            DOITREF
      PARAMETER          ( DOITREF = .TRUE. )
*
      INTEGER            ITERMAX
      PARAMETER          ( ITERMAX = 30 )
*
      DOUBLE PRECISION   BWDMAX
      PARAMETER          ( BWDMAX = 1.0E+00 )
*
      DOUBLE PRECISION   NEGONE, ONE
      PARAMETER          ( NEGONE = -1.0D+0, ONE = 1.0D+0 )
*
*     .. Local Scalars ..
      INTEGER            I, IITER, PTSA, PTSX
      DOUBLE PRECISION   ANRM, CTE, EPS, RNRM, XNRM
*
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DGEMM, DLACPY, DLAG2S, SLAG2D, SGETRF,
```

```
     $                   SGETRS, XERBLA
*     ..
*     .. External Functions ..
      INTEGER            IDAMAX
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           IDAMAX, DLAMCH, DLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      ITER = 0
*
*     Test the input parameters.
*
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSGESV', -INFO )
         RETURN
      END IF
*
*     Quick return if (N.EQ.0).
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Skip single precision iterative refinement if a priori slower
*     than double precision factorization.
*
      IF( .NOT.DOITREF ) THEN
         ITER = -1
         GO TO 40
      END IF
*
*     Compute some constants.
*
      ANRM = DLANGE( 'I', N, N, A, LDA, WORK )
      EPS = DLAMCH( 'Epsilon' )
```

```
      CTE = ANRM*EPS*SQRT( DBLE( N ) )*BWDMAX
*
*     Set the indices PTSA, PTSX for referencing SA and SX in SWORK.
*
      PTSA = 1
      PTSX = PTSA + N*N
*
*     Convert B from double precision to single precision and store the
*     result in SX.
*
      CALL DLAG2S( N, NRHS, B, LDB, SWORK( PTSX ), N, INFO )
*
      IF( INFO.NE.0 ) THEN
         ITER = -2
         GO TO 40
      END IF
*
*     Convert A from double precision to single precision and store the
*     result in SA.
*
      CALL DLAG2S( N, N, A, LDA, SWORK( PTSA ), N, INFO )
*
      IF( INFO.NE.0 ) THEN
         ITER = -2
         GO TO 40
      END IF
*
*     Compute the LU factorization of SA.
*
      CALL SGETRF( N, N, SWORK( PTSA ), N, IPIV, INFO )
*
      IF( INFO.NE.0 ) THEN
         ITER = -3
         GO TO 40
      END IF
*
*     Solve the system SA*SX = SB.
*
      CALL SGETRS( 'No transpose', N, NRHS, SWORK( PTSA ), N, IPIV,
     $             SWORK( PTSX ), N, INFO )
*
*     Convert SX back to double precision
*
      CALL SLAG2D( N, NRHS, SWORK( PTSX ), N, X, LDX, INFO )
*
*     Compute R = B - AX (R is WORK).
*
      CALL DLACPY( 'All', N, NRHS, B, LDB, WORK, N )
*
      CALL DGEMM( 'No Transpose', 'No Transpose', N, NRHS, N, NEGONE, A,
```

```
     $            LDA, X, LDX, ONE, WORK, N )
*
*     Check whether the NRHS normwise backward errors satisfy the
*     stopping criterion. If yes, set ITER=0 and return.
*
      DO I = 1, NRHS
         XNRM = ABS( X( IDAMAX( N, X( 1, I ), 1 ), I ) )
         RNRM = ABS( WORK( IDAMAX( N, WORK( 1, I ), 1 ), I ) )
         IF( RNRM.GT.XNRM*CTE )
     $      GO TO 10
      END DO
*
*     If we are here, the NRHS normwise backward errors satisfy the
*     stopping criterion. We are good to exit.
*
      ITER = 0
      RETURN
*
   10 CONTINUE
*
      DO 30 IITER = 1, ITERMAX
*
*        Convert R (in WORK) from double precision to single precision
*        and store the result in SX.
*
         CALL DLAG2S( N, NRHS, WORK, N, SWORK( PTSX ), N, INFO )
*
         IF( INFO.NE.0 ) THEN
            ITER = -2
            GO TO 40
         END IF
*
*        Solve the system SA*SX = SR.
*
         CALL SGETRS( 'No transpose', N, NRHS, SWORK( PTSA ), N, IPIV,
     $                SWORK( PTSX ), N, INFO )
*
*        Convert SX back to double precision and update the current
*        iterate.
*
         CALL SLAG2D( N, NRHS, SWORK( PTSX ), N, WORK, N, INFO )
*
         DO I = 1, NRHS
            CALL DAXPY( N, ONE, WORK( 1, I ), 1, X( 1, I ), 1 )
         END DO
*
*        Compute R = B - AX (R is WORK).
*
         CALL DLACPY( 'All', N, NRHS, B, LDB, WORK, N )
*
```

```
         CALL DGEMM( 'No Transpose', 'No Transpose', N, NRHS, N, NEGONE,
     $               A, LDA, X, LDX, ONE, WORK, N )
*
*        Check whether the NRHS normwise backward errors satisfy the
*        stopping criterion. If yes, set ITER=IITER>0 and return.
*
         DO I = 1, NRHS
            XNRM = ABS( X( IDAMAX( N, X( 1, I ), 1 ), I ) )
            RNRM = ABS( WORK( IDAMAX( N, WORK( 1, I ), 1 ), I ) )
            IF( RNRM.GT.XNRM*CTE )
     $         GO TO 20
         END DO
*
*        If we are here, the NRHS normwise backward errors satisfy the
*        stopping criterion, we are good to exit.
*
         ITER = IITER
*
         RETURN
*
   20    CONTINUE
*
   30 CONTINUE
*
*     If we are at this place of the code, this is because we have
*     performed ITER=ITERMAX iterations and never satisified the
*     stopping criterion, set up the ITER flag accordingly and follow up
*     on double precision routine.
*
      ITER = -ITERMAX - 1
*
   40 CONTINUE
*
*     Single-precision iterative refinement failed to converge to a
*     satisfactory solution, so we resort to double precision.
*
      CALL DGETRF( N, N, A, LDA, IPIV, INFO )
*
      IF( INFO.NE.0 )
     $   RETURN
*
      CALL DLACPY( 'All', N, NRHS, B, LDB, X, LDX )
      CALL DGETRS( 'No transpose', N, NRHS, A, LDA, IPIV, X, LDX,
     $             INFO )
*
      RETURN
*
*     End of DSGESV.
*
      END
```

———-

20.9.4 lisp code

— dsgesv.lisp —

```
(let* ((doitref f2cl-lib:%true%)
       (itermax 30)
       (bwdmax (coerce 1.0 'double-float))
       (negone (- 1.0d0))
       (one 1.0d0))
  (declare (type f2cl-lib:logical doitref)
           (type (f2cl-lib:integer4 30 30) itermax)
           (type (double-float) bwdmax)
           (type (double-float) negone)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable doitref itermax bwdmax negone one))
  (defun dsgesv (n nrhs a lda ipiv b ldb$ x ldx work swork iter info)
    (declare (type (array single-float (*)) swork)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array double-float (*)) work x b a)
             (type (f2cl-lib:integer4) info iter ldx ldb$ lda nrhs n))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%)
         (x double-float x-%data% x-%offset%)
         (work double-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (swork single-float swork-%data% swork-%offset%))
      (prog ((anrm 0.0d0) (cte 0.0d0) (eps 0.0d0) (rnrm 0.0d0) (xnrm 0.0d0)
             (i 0) (iiter 0) (ptsa 0) (ptsx 0))
        (declare (type (double-float) anrm cte eps rnrm xnrm)
                 (type (f2cl-lib:integer4) i iiter ptsa ptsx))
        (setf info 0)
        (setf iter 0)
        (cond ((< n 0) (setf info -1)) ((< nrhs 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -4))
              ((< ldb$
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -7))
              ((< ldx
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -9)))
        (cond
         ((/= info 0) (xerbla "DSGESV" (f2cl-lib:int-sub info))
          (go end_label)))
```

```
(if (= n 0)
    (go end_label))
(cond ((not doitref) (setf iter -1) (go label40)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "I" n n a lda work)
          (declare (ignore var-0 var-2 var-3 var-4 var-5))
          (setf n var-1)
          ret-val))
(setf eps (dlamch "Epsilon"))
(setf cte (* anrm eps (f2cl-lib:fsqrt (f2cl-lib:dble n)) bwdmax))
(setf ptsa 1)
(setf ptsx (f2cl-lib:int-add ptsa (f2cl-lib:int-mul n n)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlag2s n nrhs b ldb$
     (f2cl-lib:array-slice swork-%data% single-float (ptsx) ((1 *))
                           swork-%offset%)
     n info)
  (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5))
  (setf info var-6))
(cond ((/= info 0) (setf iter -2) (go label40)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlag2s n n a lda
     (f2cl-lib:array-slice swork-%data% single-float (ptsa) ((1 *))
                           swork-%offset%)
     n info)
  (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5))
  (setf info var-6))
(cond ((/= info 0) (setf iter -2) (go label40)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (sgetrf n n
     (f2cl-lib:array-slice swork-%data% single-float (ptsa) ((1 *))
                           swork-%offset%)
     n ipiv info)
  (declare (ignore var-2 var-4))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf n var-3))
  (when var-5 (setf info var-5)))
(cond ((/= info 0) (setf iter -3) (go label40)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sgetrs "No transpose" n nrhs
     (f2cl-lib:array-slice swork-%data% single-float (ptsa) ((1 *))
                           swork-%offset%)
     n ipiv
     (f2cl-lib:array-slice swork-%data% single-float (ptsx) ((1 *))
                           swork-%offset%)
     n info)
```

```
  (declare (ignore var-0 var-3 var-5 var-6))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf n var-4))
  (when var-7 (setf n var-7))
  (when var-8 (setf info var-8)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slag2d n nrhs
     (f2cl-lib:array-slice swork-%data% single-float (ptsx) ((1 *))
                           swork-%offset%)
     n x ldx info)
  (declare (ignore var-2 var-4))
  (when var-0 (setf n var-0))
  (when var-1 (setf nrhs var-1))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldx var-5))
  (when var-6 (setf info var-6)))
(dlacpy "All" n nrhs b ldb$ work n)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (dgemm "No Transpose" "No Transpose" n nrhs n negone a lda x ldx
     one work n)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf n var-2))
  (when var-3 (setf nrhs var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf negone var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldx var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf n var-12)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nrhs) nil)
  (tagbody
    (setf xnrm
            (abs
             (f2cl-lib:fref x-%data%
                            ((multiple-value-bind
                                 (ret-val var-0 var-1 var-2)
                                 (idamax n
                                  (f2cl-lib:array-slice x double-float
                                                        (1 i)
                                                        ((1 ldx)
                                                         (1 *)))
                                  1)
                                (declare (ignore var-1 var-2))
                                (when var-0 (setf n var-0))
                                ret-val)
                             i)
```

```
                              ((1 ldx) (1 *)) x-%offset%)))
      (setf rnrm
              (abs
               (f2cl-lib:fref work-%data%
                              ((multiple-value-bind
                                   (ret-val var-0 var-1 var-2)
                                   (idamax n
                                    (f2cl-lib:array-slice work
                                                          double-float
                                                          (1 i)
                                                          ((1 n) (1 *)))
                                    1)
                                  (declare (ignore var-1 var-2))
                                  (when var-0 (setf n var-0))
                                  ret-val)
                               i)
                              ((1 n) (1 *)) work-%offset%)))
      (if (> rnrm (* xnrm cte))
          (go label10))
    label100000))
  (setf iter 0)
  (go end_label)
 label10
  (f2cl-lib:fdo (iiter 1 (f2cl-lib:int-add iiter 1))
                ((> iiter itermax) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlag2s n nrhs work n
           (f2cl-lib:array-slice swork-%data% single-float (ptsx) ((1 *))
                                 swork-%offset%)
           n info)
        (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5))
        (setf info var-6))
      (cond ((/= info 0) (setf iter -2) (go label40)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (sgetrs "No transpose" n nrhs
           (f2cl-lib:array-slice swork-%data% single-float (ptsa) ((1 *))
                                 swork-%offset%)
           n ipiv
           (f2cl-lib:array-slice swork-%data% single-float (ptsx) ((1 *))
                                 swork-%offset%)
           n info)
        (declare (ignore var-0 var-3 var-5 var-6))
        (when var-1 (setf n var-1))
        (when var-2 (setf nrhs var-2))
        (when var-4 (setf n var-4))
        (when var-7 (setf n var-7))
        (when var-8 (setf info var-8)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
    (slag2d n nrhs
     (f2cl-lib:array-slice swork-%data% single-float (ptsx) ((1 *))
                           swork-%offset%)
     n work n info)
  (declare (ignore var-2 var-4))
  (when var-0 (setf n var-0))
  (when var-1 (setf nrhs var-1))
  (when var-3 (setf n var-3))
  (when var-5 (setf n var-5))
  (when var-6 (setf info var-6)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nrhs) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (daxpy n one
         (f2cl-lib:array-slice work-%data% double-float (1 i)
                               ((1 n) (1 *)) work-%offset%)
         1
         (f2cl-lib:array-slice x-%data% double-float (1 i)
                               ((1 ldx) (1 *)) x-%offset%)
         1)
      (declare (ignore var-2 var-3 var-4 var-5))
      (when var-0 (setf n var-0))
      (when var-1 (setf one var-1)))
   label100001))
(dlacpy "All" n nrhs b ldb$ work n)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dgemm "No Transpose" "No Transpose" n nrhs n negone a lda x
     ldx one work n)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf n var-2))
  (when var-3 (setf nrhs var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf negone var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldx var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf n var-12)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nrhs) nil)
  (tagbody
    (setf xnrm
            (abs
             (f2cl-lib:fref x-%data%
                            ((multiple-value-bind
                                 (ret-val var-0 var-1 var-2)
                                 (idamax n
                                  (f2cl-lib:array-slice x
```

```
                                                                  double-float
                                                                  (1 i)
                                                                  ((1 ldx)
                                                                   (1 *)))
                                          1)
                                       (declare (ignore var-1 var-2))
                                       (when var-0 (setf n var-0))
                                       ret-val)
                                    i)
                                   ((1 ldx) (1 *)) x-%offset%)))
            (setf rnrm
                    (abs
                     (f2cl-lib:fref work-%data%
                                   ((multiple-value-bind
                                        (ret-val var-0 var-1 var-2)
                                        (idamax n
                                         (f2cl-lib:array-slice work
                                                                  double-float
                                                                  (1 i)
                                                                  ((1 n)
                                                                   (1 *)))
                                          1)
                                       (declare (ignore var-1 var-2))
                                       (when var-0 (setf n var-0))
                                       ret-val)
                                    i)
                                   ((1 n) (1 *)) work-%offset%)))
            (if (> rnrm (* xnrm cte))
                (go label20))
           label100002))
        (setf iter iiter)
        (go end_label)
       label20
       label30))
    (setf iter (f2cl-lib:int-sub -1 itermax))
   label40
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (dgetrf n n a lda ipiv info)
      (declare (ignore var-2 var-4))
      (setf n var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf info var-5))
    (if (/= info 0)
        (go end_label))
    (dlacpy "All" n nrhs b ldb$ x ldx)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dgetrs "No transpose" n nrhs a lda ipiv x ldx info)
      (declare (ignore var-0 var-3 var-5 var-6))
```

```
          (setf n var-1)
          (setf nrhs var-2)
          (setf lda var-4)
          (setf ldx var-7)
          (setf info var-8))
        (go end_label)
       end_label
        (return
         (values n nrhs nil lda nil nil nil nil ldx nil nil iter info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsgesv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil nil nil
                                              nil fortran-to-lisp::ldx nil nil
                                              fortran-to-lisp::iter
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::dgetrs
                                              fortran-to-lisp::dgetrf
                                              fortran-to-lisp::dlacpy
                                              fortran-to-lisp::dlag2s
                                              fortran-to-lisp::dlange))))
```

Chapter 21

LAPACK - General Matrices, Linear Solve, Real

21.1 sgels

21.1.1 Axiom unit tests

— **sgels.input** —

```
)set break resume
)sys rm -f sgels.output
)spool sgels.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

21.1.2 Axiom help page

— **sgels.help** —

```
==========================================================================
sgels examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgels (TRANS, M, N, NRHS, A, LDA, B, LDB, WORK, LWORK, INFO)
            SGELS solves overdetermined or underdetermined systems for GE
           matrices

Function/Subroutine Documentation
   subroutine sgels (character TRANS, integer M, integer N, integer NRHS,
       real, dimension( lda, * ) A, integer LDA, real, dimension( ldb, * ) B,
       integer LDB, real, dimension( * ) WORK, integer LWORK, integer INFO)
        SGELS solves overdetermined or underdetermined systems for GE matrices

Purpose:
   SGELS solves overdetermined or underdetermined real linear systems
   involving an M-by-N matrix A, or its transpose, using a QR or LQ
   factorization of A.  It is assumed that A has full rank.

   The following options are provided:

   1. If TRANS = 'N' and m >= n:  find the least squares solution of
      an overdetermined system, i.e., solve the least squares problem
            minimize || B - A*X ||.

   2. If TRANS = 'N' and m < n:  find the minimum norm solution of
      an underdetermined system A * X = B.

   3. If TRANS = 'T' and m >= n:  find the minimum norm solution of
      an undetermined system A**T * X = B.

   4. If TRANS = 'T' and m < n:  find the least squares solution of
      an overdetermined system, i.e., solve the least squares problem
           minimize || B - A**T * X ||.

   Several right hand side vectors b and solution vectors x can be
   handled in a single call; they are stored as the columns of the
   M-by-NRHS right hand side matrix B and the N-by-NRHS solution matrix X.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   = 'N': the linear system involves A;
   = 'T': the linear system involves A**T.

 M
```

```
  M is INTEGER
  The number of rows of the matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of
  columns of the matrices B and X. NRHS >=0.

A
  A is REAL array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit,
  if M >= N, A is overwritten by details of its QR
     factorization as returned by SGEQRF;
  if M <  N, A is overwritten by details of its LQ
     factorization as returned by SGELQF.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

B
  B is REAL array, dimension (LDB,NRHS)
  On entry, the matrix B of right hand side vectors, stored
  columnwise; B is M-by-NRHS if TRANS = 'N', or N-by-NRHS
  if TRANS = 'T'.
  On exit, if INFO = 0, B is overwritten by the solution
  vectors, stored columnwise:
  if TRANS = 'N' and m >= n, rows 1 to n of B contain the least
  squares solution vectors; the residual sum of squares for the
  solution in each column is given by the sum of squares of
  elements N+1 to M in that column;
  if TRANS = 'N' and m < n, rows 1 to N of B contain the
  minimum norm solution vectors;
  if TRANS = 'T' and m >= n, rows 1 to M of B contain the
  minimum norm solution vectors;
  if TRANS = 'T' and m < n, rows 1 to M of B contain the
  least squares solution vectors; the residual sum of squares
  for the solution in each column is given by the sum of
  squares of elements M+1 to N in that column.

LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= MAX(1,M,N).

WORK
```

```
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  LWORK >= max( 1, MN + max( MN, NRHS ) ).
  For optimal performance,
  LWORK >= max( 1, MN + max( MN, NRHS )*NB ).
  where MN = min(M,N) and NB is the optimum block size.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO =  i, the i-th diagonal element of the
        triangular factor of A is zero, so that A does not have
        full rank; the least squares solution could not be computed.
```

———-

21.1.3 fortran code

— **sgels.f** —

```
*  =====================================================================
      SUBROUTINE SGELS( TRANS, M, N, NRHS, A, LDA, B, LDB, WORK, LWORK,
     $                  INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDA, LDB, LWORK, M, N, NRHS
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
```

```
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, TPSD
      INTEGER            BROW, I, IASCL, IBSCL, J, MN, NB, SCLLEN, WSIZE
      REAL               ANRM, BIGNUM, BNRM, SMLNUM
*     ..
*     .. Local Arrays ..
      REAL               RWORK( 1 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               SLAMCH, SLANGE
      EXTERNAL           LSAME, ILAENV, SLAMCH, SLANGE
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGELQF, SGEQRF, SLABAD, SLASCL, SLASET, SORMLQ,
     $                   SORMQR, STRTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, REAL
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      MN = MIN( M, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.( LSAME( TRANS, 'N' ) .OR. LSAME( TRANS, 'T' ) ) ) THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -6
      ELSE IF( LDB.LT.MAX( 1, M, N ) ) THEN
         INFO = -8
      ELSE IF( LWORK.LT.MAX( 1, MN + MAX( MN, NRHS ) ) .AND.
     $   .NOT.LQUERY ) THEN
         INFO = -10
      END IF
```

```
*
*     Figure out optimal block size
*
      IF( INFO.EQ.0 .OR. INFO.EQ.-10 ) THEN
*
         TPSD = .TRUE.
         IF( LSAME( TRANS, 'N' ) )
     $      TPSD = .FALSE.
*
         IF( M.GE.N ) THEN
            NB = ILAENV( 1, 'SGEQRF', ' ', M, N, -1, -1 )
            IF( TPSD ) THEN
               NB = MAX( NB, ILAENV( 1, 'SORMQR', 'LN', M, NRHS, N,
     $              -1 ) )
            ELSE
               NB = MAX( NB, ILAENV( 1, 'SORMQR', 'LT', M, NRHS, N,
     $              -1 ) )
            END IF
         ELSE
            NB = ILAENV( 1, 'SGELQF', ' ', M, N, -1, -1 )
            IF( TPSD ) THEN
               NB = MAX( NB, ILAENV( 1, 'SORMLQ', 'LT', N, NRHS, M,
     $              -1 ) )
            ELSE
               NB = MAX( NB, ILAENV( 1, 'SORMLQ', 'LN', N, NRHS, M,
     $              -1 ) )
            END IF
         END IF
*
         WSIZE = MAX( 1, MN + MAX( MN, NRHS )*NB )
         WORK( 1 ) = REAL( WSIZE )
*
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGELS ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( MIN( M, N, NRHS ).EQ.0 ) THEN
         CALL SLASET( 'Full', MAX( M, N ), NRHS, ZERO, ZERO, B, LDB )
         RETURN
      END IF
*
*     Get machine parameters
*
```

```
      SMLNUM = SLAMCH( 'S' ) / SLAMCH( 'P' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
*
*     Scale A, B if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = SLANGE( 'M', M, N, A, LDA, RWORK )
      IASCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL SLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
         IASCL = 1
      ELSE IF( ANRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL SLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
         IASCL = 2
      ELSE IF( ANRM.EQ.ZERO ) THEN
*
*        Matrix all zero. Return zero solution.
*
         CALL SLASET( 'F', MAX( M, N ), NRHS, ZERO, ZERO, B, LDB )
         GO TO 50
      END IF
*
      BROW = M
      IF( TPSD )
     $   BROW = N
      BNRM = SLANGE( 'M', BROW, NRHS, B, LDB, RWORK )
      IBSCL = 0
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL SLASCL( 'G', 0, 0, BNRM, SMLNUM, BROW, NRHS, B, LDB,
     $                INFO )
         IBSCL = 1
      ELSE IF( BNRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL SLASCL( 'G', 0, 0, BNRM, BIGNUM, BROW, NRHS, B, LDB,
     $                INFO )
         IBSCL = 2
      END IF
*
      IF( M.GE.N ) THEN
```

```
*
*        compute QR factorization of A
*
         CALL SGEQRF( M, N, A, LDA, WORK( 1 ), WORK( MN+1 ), LWORK-MN,
     $                INFO )
*
*        workspace at least N, optimally N*NB
*
         IF( .NOT.TPSD ) THEN
*
*           Least-Squares Problem min || A * X - B ||
*
*           B(1:M,1:NRHS) := Q**T * B(1:M,1:NRHS)
*
            CALL SORMQR( 'Left', 'Transpose', M, NRHS, N, A, LDA,
     $                   WORK( 1 ), B, LDB, WORK( MN+1 ), LWORK-MN,
     $                   INFO )
*
*           workspace at least NRHS, optimally NRHS*NB
*
*           B(1:N,1:NRHS) := inv(R) * B(1:N,1:NRHS)
*
            CALL STRTRS( 'Upper', 'No transpose', 'Non-unit', N, NRHS,
     $                   A, LDA, B, LDB, INFO )
*
            IF( INFO.GT.0 ) THEN
               RETURN
            END IF
*
            SCLLEN = N
*
         ELSE
*
*           Overdetermined system of equations A**T * X = B
*
*           B(1:N,1:NRHS) := inv(R**T) * B(1:N,1:NRHS)
*
            CALL STRTRS( 'Upper', 'Transpose', 'Non-unit', N, NRHS,
     $                   A, LDA, B, LDB, INFO )
*
            IF( INFO.GT.0 ) THEN
               RETURN
            END IF
*
*           B(N+1:M,1:NRHS) = ZERO
*
            DO 20 J = 1, NRHS
               DO 10 I = N + 1, M
                  B( I, J ) = ZERO
   10          CONTINUE
```

```
   20       CONTINUE
*
*           B(1:M,1:NRHS) := Q(1:N,:) * B(1:N,1:NRHS)
*
            CALL SORMQR( 'Left', 'No transpose', M, NRHS, N, A, LDA,
     $                   WORK( 1 ), B, LDB, WORK( MN+1 ), LWORK-MN,
     $                   INFO )
*
*           workspace at least NRHS, optimally NRHS*NB
*
            SCLLEN = M
*
         END IF
*
      ELSE
*
*        Compute LQ factorization of A
*
         CALL SGELQF( M, N, A, LDA, WORK( 1 ), WORK( MN+1 ), LWORK-MN,
     $                INFO )
*
*        workspace at least M, optimally M*NB.
*
         IF( .NOT.TPSD ) THEN
*
*           underdetermined system of equations A * X = B
*
            B(1:M,1:NRHS) := inv(L) * B(1:M,1:NRHS)
*
            CALL STRTRS( 'Lower', 'No transpose', 'Non-unit', M, NRHS,
     $                   A, LDA, B, LDB, INFO )
*
            IF( INFO.GT.0 ) THEN
               RETURN
            END IF
*
            B(M+1:N,1:NRHS) = 0
*
            DO 40 J = 1, NRHS
               DO 30 I = M + 1, N
                  B( I, J ) = ZERO
   30          CONTINUE
   40       CONTINUE
*
*           B(1:N,1:NRHS) := Q(1:N,:)**T * B(1:M,1:NRHS)
*
            CALL SORMLQ( 'Left', 'Transpose', N, NRHS, M, A, LDA,
     $                   WORK( 1 ), B, LDB, WORK( MN+1 ), LWORK-MN,
     $                   INFO )
*
```

```
*          workspace at least NRHS, optimally NRHS*NB
*
           SCLLEN = N
*
        ELSE
*
*          overdetermined system min || A**T * X - B ||
*
*          B(1:N,1:NRHS) := Q * B(1:N,1:NRHS)
*
           CALL SORMLQ( 'Left', 'No transpose', N, NRHS, M, A, LDA,
     $                  WORK( 1 ), B, LDB, WORK( MN+1 ), LWORK-MN,
     $                  INFO )
*
*          workspace at least NRHS, optimally NRHS*NB
*
*          B(1:M,1:NRHS) := inv(L**T) * B(1:M,1:NRHS)
*
           CALL STRTRS( 'Lower', 'Transpose', 'Non-unit', M, NRHS,
     $                  A, LDA, B, LDB, INFO )
*
           IF( INFO.GT.0 ) THEN
              RETURN
           END IF
*
           SCLLEN = M
*
        END IF
*
     END IF
*
*    Undo scaling
*
     IF( IASCL.EQ.1 ) THEN
        CALL SLASCL( 'G', 0, 0, ANRM, SMLNUM, SCLLEN, NRHS, B, LDB,
     $               INFO )
     ELSE IF( IASCL.EQ.2 ) THEN
        CALL SLASCL( 'G', 0, 0, ANRM, BIGNUM, SCLLEN, NRHS, B, LDB,
     $               INFO )
     END IF
     IF( IBSCL.EQ.1 ) THEN
        CALL SLASCL( 'G', 0, 0, SMLNUM, BNRM, SCLLEN, NRHS, B, LDB,
     $               INFO )
     ELSE IF( IBSCL.EQ.2 ) THEN
        CALL SLASCL( 'G', 0, 0, BIGNUM, BNRM, SCLLEN, NRHS, B, LDB,
     $               INFO )
     END IF
*
   50 CONTINUE
      WORK( 1 ) = REAL( WSIZE )
```

```
*
      RETURN
*
*     End of SGELS
*
      END
```

———-

21.1.4 lisp code

— **sgels.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sgels (trans m n nrhs a lda b ldb$ work lwork info)
    (declare (type (array single-float (*)) work b a)
             (type (f2cl-lib:integer4) info lwork ldb$ lda nrhs n m)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((rwork (make-array 1 :element-type 'single-float)) (anrm 0.0)
             (bignum 0.0) (bnrm 0.0) (smlnum 0.0) (brow 0) (i 0) (iascl 0)
             (ibscl 0) (j 0) (mn 0) (nb 0) (scllen 0) (wsize 0) (lquery nil)
             (tpsd nil))
        (declare (type (array single-float (1)) rwork)
                 (type (single-float) anrm bignum bnrm smlnum)
                 (type (f2cl-lib:integer4) brow i iascl ibscl j mn nb scllen
                  wsize)
                 (type f2cl-lib:logical lquery tpsd))
        (setf info 0)
        (setf mn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond
         ((not
           (or
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "N")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val)
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "T")
```

```
      (declare (ignore var-1))
      (when var-0 (setf trans var-0))
      ret-val)))
  (setf info -1))
 ((< m 0) (setf info -2)) ((< n 0) (setf info -3))
 ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -6))
 ((< ldb$
     (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)
          (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and
   (< lwork
      (max (the f2cl-lib:integer4 1)
           (the f2cl-lib:integer4
                (f2cl-lib:int-add mn
                                  (max (the f2cl-lib:integer4 mn)
                                       (the f2cl-lib:integer4
                                            nrhs))))))
   (not lquery))
  (setf info -10)))
(cond
 ((or (= info 0) (= info (f2cl-lib:int-sub 10)))
  (setf tpsd f2cl-lib:%true%)
  (if (multiple-value-bind (ret-val var-0 var-1)
          (lsame trans "N")
        (declare (ignore var-1))
        (when var-0 (setf trans var-0))
        ret-val)
      (setf tpsd f2cl-lib:%false%))
  (cond
   ((>= m n)
    (setf nb
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 1 "SGEQRF" " " m n -1 -1)
              (declare (ignore var-0 var-1 var-2 var-5 var-6))
              (setf m var-3)
              (setf n var-4)
              ret-val))
    (cond
     (tpsd
      (setf nb
              (max (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4
                             var-5 var-6)
                            (ilaenv 1 "SORMQR" "LN" m nrhs n -1)
```

```
                          (declare (ignore var-0 var-1 var-2 var-6))
                          (setf m var-3)
                          (setf nrhs var-4)
                          (setf n var-5)
                          ret-val)))))
            (t
             (setf nb
                     (max (the f2cl-lib:integer4 nb)
                          (the f2cl-lib:integer4
                               (multiple-value-bind
                                   (ret-val var-0 var-1 var-2 var-3 var-4
                                    var-5 var-6)
                                   (ilaenv 1 "SORMQR" "LT" m nrhs n -1)
                                 (declare (ignore var-0 var-1 var-2 var-6))
                                 (setf m var-3)
                                 (setf nrhs var-4)
                                 (setf n var-5)
                                 ret-val)))))))
          (t
           (setf nb
                   (multiple-value-bind
                       (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                       (ilaenv 1 "SGELQF" " " m n -1 -1)
                     (declare (ignore var-0 var-1 var-2 var-5 var-6))
                     (setf m var-3)
                     (setf n var-4)
                     ret-val))
           (cond
            (tpsd
             (setf nb
                     (max (the f2cl-lib:integer4 nb)
                          (the f2cl-lib:integer4
                               (multiple-value-bind
                                   (ret-val var-0 var-1 var-2 var-3 var-4
                                    var-5 var-6)
                                   (ilaenv 1 "SORMLQ" "LT" n nrhs m -1)
                                 (declare (ignore var-0 var-1 var-2 var-6))
                                 (setf n var-3)
                                 (setf nrhs var-4)
                                 (setf m var-5)
                                 ret-val)))))
            (t
             (setf nb
                     (max (the f2cl-lib:integer4 nb)
                          (the f2cl-lib:integer4
                               (multiple-value-bind
                                   (ret-val var-0 var-1 var-2 var-3 var-4
                                    var-5 var-6)
                                   (ilaenv 1 "SORMLQ" "LN" n nrhs m -1)
                                 (declare (ignore var-0 var-1 var-2 var-6))
```

```
                          (setf n var-3)
                          (setf nrhs var-4)
                          (setf m var-5)
                          ret-val))))))))
  (setf wsize
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add mn
                                      (f2cl-lib:int-mul
                                       (max (the f2cl-lib:integer4 mn)
                                            (the f2cl-lib:integer4
                                                 nrhs))
                                       nb)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:freal wsize))))
(cond
 ((/= info 0) (xerbla "SGELS " (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond
 ((=
   (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)
        (the f2cl-lib:integer4 nrhs))
   0)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "Full"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       zero zero b ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldb$ var-6)))
  (go end_label)))
(setf smlnum (/ (slamch "S") (slamch "P")))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (slabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "M" m n a lda rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iascl 0)
```

```
(cond
 ((and (> anrm zero) (< anrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 anrm smlnum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 1))
 ((> anrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 anrm bignum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 2))
 ((= anrm zero)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "F"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       zero zero b ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldb$ var-6)))
  (go label50)))
(setf brow m)
(if tpsd
    (setf brow n))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "M" brow nrhs b ldb$ rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf brow var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ibscl 0)
(cond
```

```
 ((and (> bnrm zero) (< bnrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 bnrm smlnum brow nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf brow var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 1))
 ((> bnrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 bnrm bignum brow nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf brow var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 2)))
(cond
 ((>= m n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (sgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float ((+ mn 1))
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-sub lwork mn) info)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf info var-7))
  (cond
   ((not tpsd)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (sormqr "Left" "Transpose" m nrhs n a lda
         (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                               work-%offset%)
         b ldb$
         (f2cl-lib:array-slice work-%data% single-float ((+ mn 1))
                               ((1 *)) work-%offset%)
```

```
      (f2cl-lib:int-sub lwork mn) info)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf n var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldb$ var-9))
    (when var-12 (setf info var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (strtrs "Upper" "No transpose" "Non-unit" n nrhs a lda b ldb$
       info)
    (declare (ignore var-0 var-1 var-2 var-5 var-7))
    (when var-3 (setf n var-3))
    (when var-4 (setf nrhs var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (cond ((> info 0) (go end_label))) (setf scllen n))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (strtrs "Upper" "Transpose" "Non-unit" n nrhs a lda b ldb$
       info)
    (declare (ignore var-0 var-1 var-2 var-5 var-7))
    (when var-3 (setf n var-3))
    (when var-4 (setf nrhs var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (cond ((> info 0) (go end_label)))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (f2cl-lib:fdo (i (f2cl-lib:int-add n 1) (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                  zero)
        label10))
    label20))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sormqr "Left" "No transpose" m nrhs n a lda
       (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                             work-%offset%)
       b ldb$
       (f2cl-lib:array-slice work-%data% single-float ((+ mn 1))
```

```
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-sub lwork mn) info)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf nrhs var-3))
        (when var-4 (setf n var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldb$ var-9))
        (when var-12 (setf info var-12)))
      (setf scllen m))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (sgelqf m n a lda
         (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                               work-%offset%)
         (f2cl-lib:array-slice work-%data% single-float ((+ mn 1))
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-sub lwork mn) info)
      (declare (ignore var-2 var-4 var-5 var-6))
      (setf m var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf info var-7))
    (cond
     ((not tpsd)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (strtrs "Lower" "No transpose" "Non-unit" m nrhs a lda b ldb$
           info)
        (declare (ignore var-0 var-1 var-2 var-5 var-7))
        (when var-3 (setf m var-3))
        (when var-4 (setf nrhs var-4))
        (when var-6 (setf lda var-6))
        (when var-8 (setf ldb$ var-8))
        (when var-9 (setf info var-9)))
      (cond ((> info 0) (go end_label)))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nrhs) nil)
        (tagbody
          (f2cl-lib:fdo (i (f2cl-lib:int-add m 1) (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)
                    zero)
             label30))
         label40))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
          var-10 var-11 var-12)
         (sormlq "Left" "Transpose" n nrhs m a lda
          (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                                work-%offset%)
          b ldb$
          (f2cl-lib:array-slice work-%data% single-float ((+ mn 1))
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-sub lwork mn) info)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf n var-2))
        (when var-3 (setf nrhs var-3))
        (when var-4 (setf m var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldb$ var-9))
        (when var-12 (setf info var-12)))
      (setf scllen n))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (sormlq "Left" "No transpose" n nrhs m a lda
           (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                                 work-%offset%)
           b ldb$
           (f2cl-lib:array-slice work-%data% single-float ((+ mn 1))
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-sub lwork mn) info)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf n var-2))
        (when var-3 (setf nrhs var-3))
        (when var-4 (setf m var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldb$ var-9))
        (when var-12 (setf info var-12)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (strtrs "Lower" "Transpose" "Non-unit" m nrhs a lda b ldb$
           info)
        (declare (ignore var-0 var-1 var-2 var-5 var-7))
        (when var-3 (setf m var-3))
        (when var-4 (setf nrhs var-4))
        (when var-6 (setf lda var-6))
        (when var-8 (setf ldb$ var-8))
        (when var-9 (setf info var-9)))
      (cond ((> info 0) (go end_label))) (setf scllen m)))))
  (cond
   ((= iascl 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl "G" 0 0 anrm smlnum scllen nrhs b ldb$ info)
```

```
            (declare (ignore var-0 var-1 var-2 var-7))
            (when var-3 (setf anrm var-3))
            (when var-4 (setf smlnum var-4))
            (when var-5 (setf scllen var-5))
            (when var-6 (setf nrhs var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf info var-9))))
          ((= iascl 2)
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
               (slascl "G" 0 0 anrm bignum scllen nrhs b ldb$ info)
            (declare (ignore var-0 var-1 var-2 var-7))
            (when var-3 (setf anrm var-3))
            (when var-4 (setf bignum var-4))
            (when var-5 (setf scllen var-5))
            (when var-6 (setf nrhs var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf info var-9)))))
         (cond
          ((= ibscl 1)
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
               (slascl "G" 0 0 smlnum bnrm scllen nrhs b ldb$ info)
            (declare (ignore var-0 var-1 var-2 var-7))
            (when var-3 (setf smlnum var-3))
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf scllen var-5))
            (when var-6 (setf nrhs var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf info var-9))))
          ((= ibscl 2)
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
               (slascl "G" 0 0 bignum bnrm scllen nrhs b ldb$ info)
            (declare (ignore var-0 var-1 var-2 var-7))
            (when var-3 (setf bignum var-3))
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf scllen var-5))
            (when var-6 (setf nrhs var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf info var-9)))))
        label50
         (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                 (f2cl-lib:freal wsize))
         (go end_label)
        end_label
         (return (values trans m n nrhs nil lda nil ldb$ nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
```

```
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgels fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$ nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::sgeqrf
                                              fortran-to-lisp::sgelqf
                                              fortran-to-lisp::ilaenv))))
```

———-

21.2 sgelsd

21.2.1 Axiom unit tests

— **sgelsd.input** —

```
)set break resume
)sys rm -f sgelsd.output
)spool sgelsd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

21.2.2 Axiom help page

— sgelsd.help —

```
=====================================================================
sgelsd examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgelsd (M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK, WORK,
           LWORK, IWORK, INFO)
            SGELSD computes the minimum-norm solution to a linear least
           squares problem for GE matrices

Function/Subroutine Documentation
   subroutine sgelsd (integer M, integer N, integer NRHS, real, dimension(
       lda, * ) A, integer LDA, real, dimension( ldb, * ) B, integer LDB,
       real, dimension( * ) S, real RCOND, integer RANK, real, dimension( * )
       WORK, integer LWORK, integer, dimension( * ) IWORK, integer INFO)
        SGELSD computes the minimum-norm solution to a linear least squares
       problem for GE matrices

Purpose:
   SGELSD computes the minimum-norm solution to a real linear least
   squares problem:
                minimize 2-norm(| b - A*x |)
   using the singular value decomposition (SVD) of A. A is an M-by-N
   matrix which may be rank-deficient.

   Several right hand side vectors b and solution vectors x can be
   handled in a single call; they are stored as the columns of the
   M-by-NRHS right hand side matrix B and the N-by-NRHS solution
   matrix X.

   The problem is solved in three steps:
   (1) Reduce the coefficient matrix A to bidiagonal form with
       Householder transformations, reducing the original problem
       into a "bidiagonal least squares problem" (BLS)
   (2) Solve the BLS using a divide and conquer approach.
   (3) Apply back all the Householder tranformations to solve
       the original least squares problem.
```

```
  The effective rank of A is determined by treating as zero those
  singular values which are less than RCOND times the largest singular
  value.

  The divide and conquer algorithm makes very mild assumptions about
  floating point arithmetic. It will work on machines with a guard
  digit in add/subtract, or on those binary machines without guard
  digits which subtract like the Cray X-MP, Cray Y-MP, Cray C-90, or
  Cray-2. It could conceivably fail on hexadecimal or decimal machines
  without guard digits, but we know of none.

Parameters:

 M
   M is INTEGER
   The number of rows of A. M >= 0.

 N
   N is INTEGER
   The number of columns of A. N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X. NRHS >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, A has been destroyed.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 B
   B is REAL array, dimension (LDB,NRHS)
   On entry, the M-by-NRHS right hand side matrix B.
   On exit, B is overwritten by the N-by-NRHS solution
   matrix X.  If m >= n and RANK = n, the residual
   sum-of-squares for the solution in the i-th column is given
   by the sum of squares of elements n+1:m in that column.

 LDB
   LDB is INTEGER
   The leading dimension of the array B. LDB >= max(1,max(M,N)).

 S
   S is REAL array, dimension (min(M,N))
```

```
  The singular values of A in decreasing order.
  The condition number of A in the 2-norm = S(1)/S(min(m,n)).

RCOND
  RCOND is REAL
  RCOND is used to determine the effective rank of A.
  Singular values S(i) <= RCOND*S(1) are treated as zero.
  If RCOND < 0, machine precision is used instead.

RANK
  RANK is INTEGER
  The effective rank of A, i.e., the number of singular values
  which are greater than RCOND*S(1).

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK must be at least 1.
  The exact minimum amount of workspace needed depends on M,
  N and NRHS. As long as LWORK is at least
  12*N + 2*N*SMLSIZ + 8*N*NLVL + N*NRHS + (SMLSIZ+1)**2,
  if M is greater than or equal to N or
  12*M + 2*M*SMLSIZ + 8*M*NLVL + M*NRHS + (SMLSIZ+1)**2,
  if M is less than N, the code will execute correctly.
  SMLSIZ is returned by ILAENV and is equal to the maximum
  size of the subproblems at the bottom of the computation
  tree (usually about 25), and
  NLVL = MAX( 0, INT( LOG_2( MIN( M,N )/(SMLSIZ+1) ) ) + 1 )
  For good performance, LWORK should generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the array WORK and the
  minimum size of the array IWORK, and returns these values as
  the first entries of the WORK and IWORK arrays, and no error
  message related to LWORK is issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (MAX(1,LIWORK))
  LIWORK >= max(1, 3*MINMN*NLVL + 11*MINMN),
  where MINMN = MIN( M,N ).
  On exit, if INFO = 0, IWORK(1) returns the minimum LIWORK.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  the algorithm for computing the SVD failed to converge;
```

```
if INFO = i, i off-diagonal elements of an intermediate
bidiagonal form did not converge to zero.
```

———-

21.2.3 fortran code

— **sgelsd.f** —

```
*  =====================================================================
      SUBROUTINE SGELSD( M, N, NRHS, A, LDA, B, LDB, S, RCOND,
     $                   RANK, WORK, LWORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDB, LWORK, M, N, NRHS, RANK
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      REAL               A( LDA, * ), B( LDB, * ), S( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE, TWO
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0, TWO = 2.0E0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            IASCL, IBSCL, IE, IL, ITAU, ITAUP, ITAUQ,
     $                   LDWORK, LIWORK, MAXMN, MAXWRK, MINMN, MINWRK,
     $                   MM, MNTHR, NLVL, NWORK, SMLSIZ, WLALSD
      REAL               ANRM, BIGNUM, BNRM, EPS, SFMIN, SMLNUM
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEBRD, SGELQF, SGEQRF, SLABAD, SLACPY, SLALSD,
     $                   SLASCL, SLASET, SORMBR, SORMLQ, SORMQR, XERBLA
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      REAL               SLAMCH, SLANGE
```

```
      EXTERNAL           SLAMCH, SLANGE, ILAENV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          INT, LOG, MAX, MIN, REAL
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      MINMN = MIN( M, N )
      MAXMN = MAX( M, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, MAXMN ) ) THEN
         INFO = -7
      END IF
*
*     Compute workspace.
*     (Note: Comments in the code beginning "Workspace:" describe the
*     minimal amount of workspace needed at that point in the code,
*     as well as the preferred amount for good performance.
*     NB refers to the optimal block size for the immediately
*     following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = 1
         MAXWRK = 1
         LIWORK = 1
         IF( MINMN.GT.0 ) THEN
            SMLSIZ = ILAENV( 9, 'SGELSD', ' ', 0, 0, 0, 0 )
            MNTHR = ILAENV( 6, 'SGELSD', ' ', M, N, NRHS, -1 )
            NLVL = MAX( INT( LOG( REAL( MINMN ) / REAL( SMLSIZ + 1 ) ) /
     $                  LOG( TWO ) ) + 1, 0 )
            LIWORK = 3*MINMN*NLVL + 11*MINMN
            MM = M
            IF( M.GE.N .AND. M.GE.MNTHR ) THEN
*
*              Path 1a - overdetermined, with many more rows than
*                        columns.
*
               MM = N
               MAXWRK = MAX( MAXWRK, N + N*ILAENV( 1, 'SGEQRF', ' ', M,
```

```
     $                       N, -1, -1 ) )
               MAXWRK = MAX( MAXWRK, N + NRHS*ILAENV( 1, 'SORMQR', 'LT',
     $                       M, NRHS, N, -1 ) )
            END IF
            IF( M.GE.N ) THEN
*
*              Path 1 - overdetermined or exactly determined.
*
               MAXWRK = MAX( MAXWRK, 3*N + ( MM + N )*ILAENV( 1,
     $                       'SGEBRD', ' ', MM, N, -1, -1 ) )
               MAXWRK = MAX( MAXWRK, 3*N + NRHS*ILAENV( 1, 'SORMBR',
     $                       'QLT', MM, NRHS, N, -1 ) )
               MAXWRK = MAX( MAXWRK, 3*N + ( N - 1 )*ILAENV( 1,
     $                       'SORMBR', 'PLN', N, NRHS, N, -1 ) )
               WLALSD = 9*N + 2*N*SMLSIZ + 8*N*NLVL + N*NRHS +
     $                  ( SMLSIZ + 1 )**2
               MAXWRK = MAX( MAXWRK, 3*N + WLALSD )
               MINWRK = MAX( 3*N + MM, 3*N + NRHS, 3*N + WLALSD )
            END IF
            IF( N.GT.M ) THEN
               WLALSD = 9*M + 2*M*SMLSIZ + 8*M*NLVL + M*NRHS +
     $                  ( SMLSIZ + 1 )**2
               IF( N.GE.MNTHR ) THEN
*
*                 Path 2a - underdetermined, with many more columns
*                           than rows.
*
                  MAXWRK = M + M*ILAENV( 1, 'SGELQF', ' ', M, N, -1,
     $                                   -1 )
                  MAXWRK = MAX( MAXWRK, M*M + 4*M + 2*M*ILAENV( 1,
     $                          'SGEBRD', ' ', M, M, -1, -1 ) )
                  MAXWRK = MAX( MAXWRK, M*M + 4*M + NRHS*ILAENV( 1,
     $                          'SORMBR', 'QLT', M, NRHS, M, -1 ) )
                  MAXWRK = MAX( MAXWRK, M*M + 4*M + ( M - 1 )*ILAENV( 1,
     $                          'SORMBR', 'PLN', M, NRHS, M, -1 ) )
                  IF( NRHS.GT.1 ) THEN
                     MAXWRK = MAX( MAXWRK, M*M + M + M*NRHS )
                  ELSE
                     MAXWRK = MAX( MAXWRK, M*M + 2*M )
                  END IF
                  MAXWRK = MAX( MAXWRK, M + NRHS*ILAENV( 1, 'SORMLQ',
     $                          'LT', N, NRHS, M, -1 ) )
                  MAXWRK = MAX( MAXWRK, M*M + 4*M + WLALSD )
!     XXX: Ensure the Path 2a case below is triggered.  The workspace
!     calculation should use queries for all routines eventually.
                  MAXWRK = MAX( MAXWRK,
     $                 4*M+M*M+MAX( M, 2*M-4, NRHS, N-3*M ) )
               ELSE
*
*                 Path 2 - remaining underdetermined cases.
```

```
*
                 MAXWRK = 3*M + ( N + M )*ILAENV( 1, 'SGEBRD', ' ', M,
     $                   N, -1, -1 )
                 MAXWRK = MAX( MAXWRK, 3*M + NRHS*ILAENV( 1, 'SORMBR',
     $                        'QLT', M, NRHS, N, -1 ) )
                 MAXWRK = MAX( MAXWRK, 3*M + M*ILAENV( 1, 'SORMBR',
     $                        'PLN', N, NRHS, M, -1 ) )
                 MAXWRK = MAX( MAXWRK, 3*M + WLALSD )
              END IF
              MINWRK = MAX( 3*M + NRHS, 3*M + M, 3*M + WLALSD )
           END IF
        END IF
        MINWRK = MIN( MINWRK, MAXWRK )
        WORK( 1 ) = MAXWRK
        IWORK( 1 ) = LIWORK
*
        IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
           INFO = -12
        END IF
     END IF
*
     IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'SGELSD', -INFO )
        RETURN
     ELSE IF( LQUERY ) THEN
        RETURN
     END IF
*
*    Quick return if possible.
*
     IF( M.EQ.0 .OR. N.EQ.0 ) THEN
        RANK = 0
        RETURN
     END IF
*
*    Get machine parameters.
*
     EPS = SLAMCH( 'P' )
     SFMIN = SLAMCH( 'S' )
     SMLNUM = SFMIN / EPS
     BIGNUM = ONE / SMLNUM
     CALL SLABAD( SMLNUM, BIGNUM )
*
*    Scale A if max entry outside range [SMLNUM,BIGNUM].
*
     ANRM = SLANGE( 'M', M, N, A, LDA, WORK )
     IASCL = 0
     IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
*
*       Scale matrix norm up to SMLNUM.
```

```
*
         CALL SLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
         IASCL = 1
      ELSE IF( ANRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM.
*
         CALL SLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
         IASCL = 2
      ELSE IF( ANRM.EQ.ZERO ) THEN
*
*        Matrix all zero. Return zero solution.
*
         CALL SLASET( 'F', MAX( M, N ), NRHS, ZERO, ZERO, B, LDB )
         CALL SLASET( 'F', MINMN, 1, ZERO, ZERO, S, 1 )
         RANK = 0
         GO TO 10
      END IF
*
*     Scale B if max entry outside range [SMLNUM,BIGNUM].
*
      BNRM = SLANGE( 'M', M, NRHS, B, LDB, WORK )
      IBSCL = 0
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM.
*
         CALL SLASCL( 'G', 0, 0, BNRM, SMLNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 1
      ELSE IF( BNRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM.
*
         CALL SLASCL( 'G', 0, 0, BNRM, BIGNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 2
      END IF
*
*     If M < N make sure certain entries of B are zero.
*
      IF( M.LT.N )
     $   CALL SLASET( 'F', N-M, NRHS, ZERO, ZERO, B( M+1, 1 ), LDB )
*
*     Overdetermined case.
*
      IF( M.GE.N ) THEN
*
*        Path 1 - overdetermined or exactly determined.
*
         MM = M
         IF( M.GE.MNTHR ) THEN
```

```
*
*          Path 1a - overdetermined, with many more rows than columns.
*
           MM = N
           ITAU = 1
           NWORK = ITAU + N
*
*          Compute A=Q*R.
*          (Workspace: need 2*N, prefer N+N*NB)
*
           CALL SGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                  LWORK-NWORK+1, INFO )
*
*          Multiply B by transpose(Q).
*          (Workspace: need N+NRHS, prefer N+NRHS*NB)
*
           CALL SORMQR( 'L', 'T', M, NRHS, N, A, LDA, WORK( ITAU ), B,
     $                  LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
*          Zero out below R.
*
           IF( N.GT.1 ) THEN
              CALL SLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ), LDA )
           END IF
        END IF
*
        IE = 1
        ITAUQ = IE + N
        ITAUP = ITAUQ + N
        NWORK = ITAUP + N
*
*       Bidiagonalize R in A.
*       (Workspace: need 3*N+MM, prefer 3*N+(MM+N)*NB)
*
        CALL SGEBRD( MM, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $               WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $               INFO )
*
*       Multiply B by transpose of left bidiagonalizing vectors of R.
*       (Workspace: need 3*N+NRHS, prefer 3*N+NRHS*NB)
*
        CALL SORMBR( 'Q', 'L', 'T', MM, NRHS, N, A, LDA, WORK( ITAUQ ),
     $               B, LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
*       Solve the bidiagonal least squares problem.
*
        CALL SLALSD( 'U', SMLSIZ, N, NRHS, S, WORK( IE ), B, LDB,
     $               RCOND, RANK, WORK( NWORK ), IWORK, INFO )
        IF( INFO.NE.0 ) THEN
           GO TO 10
```

```
         END IF
*
*        Multiply B by right bidiagonalizing vectors of R.
*
         CALL SORMBR( 'P', 'L', 'N', N, NRHS, N, A, LDA, WORK( ITAUP ),
     $                B, LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
      ELSE IF( N.GE.MNTHR .AND. LWORK.GE.4*M+M*M+
     $         MAX( M, 2*M-4, NRHS, N-3*M, WLALSD ) ) THEN
*
*        Path 2a - underdetermined, with many more columns than rows
*        and sufficient workspace for an efficient algorithm.
*
         LDWORK = M
         IF( LWORK.GE.MAX( 4*M+M*LDA+MAX( M, 2*M-4, NRHS, N-3*M ),
     $       M*LDA+M+M*NRHS, 4*M+M*LDA+WLALSD ) )LDWORK = LDA
         ITAU = 1
         NWORK = M + 1
*
*        Compute A=L*Q.
*        (Workspace: need 2*M, prefer M+M*NB)
*
         CALL SGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                LWORK-NWORK+1, INFO )
         IL = NWORK
*
*        Copy L to WORK(IL), zeroing out above its diagonal.
*
         CALL SLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWORK )
         CALL SLASET( 'U', M-1, M-1, ZERO, ZERO, WORK( IL+LDWORK ),
     $                LDWORK )
         IE = IL + LDWORK*M
         ITAUQ = IE + M
         ITAUP = ITAUQ + M
         NWORK = ITAUP + M
*
*        Bidiagonalize L in WORK(IL).
*        (Workspace: need M*M+5*M, prefer M*M+4*M+2*M*NB)
*
         CALL SGEBRD( M, M, WORK( IL ), LDWORK, S, WORK( IE ),
     $                WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                LWORK-NWORK+1, INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors of L.
*        (Workspace: need M*M+4*M+NRHS, prefer M*M+4*M+NRHS*NB)
*
         CALL SORMBR( 'Q', 'L', 'T', M, NRHS, M, WORK( IL ), LDWORK,
     $                WORK( ITAUQ ), B, LDB, WORK( NWORK ),
     $                LWORK-NWORK+1, INFO )
*
```

```
*        Solve the bidiagonal least squares problem.
*
         CALL SLALSD( 'U', SMLSIZ, M, NRHS, S, WORK( IE ), B, LDB,
     $               RCOND, RANK, WORK( NWORK ), IWORK, INFO )
         IF( INFO.NE.0 ) THEN
            GO TO 10
         END IF
*
*        Multiply B by right bidiagonalizing vectors of L.
*
         CALL SORMBR( 'P', 'L', 'N', M, NRHS, M, WORK( IL ), LDWORK,
     $               WORK( ITAUP ), B, LDB, WORK( NWORK ),
     $               LWORK-NWORK+1, INFO )
*
*        Zero out below first M rows of B.
*
         CALL SLASET( 'F', N-M, NRHS, ZERO, ZERO, B( M+1, 1 ), LDB )
         NWORK = ITAU + M
*
*        Multiply transpose(Q) by B.
*        (Workspace: need M+NRHS, prefer M+NRHS*NB)
*
         CALL SORMLQ( 'L', 'T', N, NRHS, M, A, LDA, WORK( ITAU ), B,
     $               LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
      ELSE
*
*        Path 2 - remaining underdetermined cases.
*
         IE = 1
         ITAUQ = IE + M
         ITAUP = ITAUQ + M
         NWORK = ITAUP + M
*
*        Bidiagonalize A.
*        (Workspace: need 3*M+N, prefer 3*M+(M+N)*NB)
*
         CALL SGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $               WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $               INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors.
*        (Workspace: need 3*M+NRHS, prefer 3*M+NRHS*NB)
*
         CALL SORMBR( 'Q', 'L', 'T', M, NRHS, N, A, LDA, WORK( ITAUQ ),
     $               B, LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
*        Solve the bidiagonal least squares problem.
*
         CALL SLALSD( 'L', SMLSIZ, M, NRHS, S, WORK( IE ), B, LDB,
```

```
     $                RCOND, RANK, WORK( NWORK ), IWORK, INFO )
         IF( INFO.NE.0 ) THEN
            GO TO 10
         END IF
*
*        Multiply B by right bidiagonalizing vectors of A.
*
         CALL SORMBR( 'P', 'L', 'N', N, NRHS, M, A, LDA, WORK( ITAUP ),
     $                B, LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
      END IF
*
*     Undo scaling.
*
      IF( IASCL.EQ.1 ) THEN
         CALL SLASCL( 'G', 0, 0, ANRM, SMLNUM, N, NRHS, B, LDB, INFO )
         CALL SLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
     $                INFO )
      ELSE IF( IASCL.EQ.2 ) THEN
         CALL SLASCL( 'G', 0, 0, ANRM, BIGNUM, N, NRHS, B, LDB, INFO )
         CALL SLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                INFO )
      END IF
      IF( IBSCL.EQ.1 ) THEN
         CALL SLASCL( 'G', 0, 0, SMLNUM, BNRM, N, NRHS, B, LDB, INFO )
      ELSE IF( IBSCL.EQ.2 ) THEN
         CALL SLASCL( 'G', 0, 0, BIGNUM, BNRM, N, NRHS, B, LDB, INFO )
      END IF
*
   10 CONTINUE
      WORK( 1 ) = MAXWRK
      IWORK( 1 ) = LIWORK
      RETURN
*
*     End of SGELSD
*
      END
```

21.2.4 lisp code

— sgelsd.lisp —

```
(let* ((zero 0.0) (one 1.0) (two 2.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
```

```
          (ignorable zero one two))
  (defun sgelsd (m n nrhs a lda b ldb$ s rcond rank work lwork iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (single-float) rcond)
             (type (array single-float (*)) work s b a)
             (type (f2cl-lib:integer4) info lwork rank ldb$ lda nrhs n m))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%)
         (s single-float s-%data% s-%offset%)
         (work single-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((anrm 0.0) (bignum 0.0) (bnrm 0.0) (eps 0.0) (sfmin 0.0)
             (smlnum 0.0) (iascl 0) (ibscl 0) (ie 0) (il 0) (itau 0) (itaup 0)
             (itauq 0) (ldwork 0) (liwork 0) (maxmn 0) (maxwrk 0) (minmn 0)
             (minwrk 0) (mm 0) (mnthr 0) (nlvl 0) (nwork 0) (smlsiz 0)
             (wlalsd 0) (lquery nil))
        (declare (type (single-float) anrm bignum bnrm eps sfmin smlnum)
                 (type (f2cl-lib:integer4) iascl ibscl ie il itau itaup itauq
                  ldwork liwork maxmn maxwrk minmn minwrk mm mnthr nlvl nwork
                  smlsiz wlalsd)
                 (type f2cl-lib:logical lquery))
        (setf info 0)
        (setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (setf maxmn (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< nrhs 0) (setf info -3))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -5))
              ((< ldb$
                  (max (the f2cl-lib:integer4 1)
                       (the f2cl-lib:integer4 maxmn)))
               (setf info -7)))
        (cond
         ((= info 0) (setf minwrk 1) (setf maxwrk 1) (setf liwork 1)
          (cond
           ((> minmn 0) (setf smlsiz (ilaenv 9 "SGELSD" " " 0 0 0 0))
            (setf mnthr
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                        (ilaenv 6 "SGELSD" " " m n nrhs -1)
                      (declare (ignore var-0 var-1 var-2 var-6))
                      (setf m var-3)
                      (setf n var-4)
                      (setf nrhs var-5)
                      ret-val))
            (setf nlvl
                    (max
```

```
          (the f2cl-lib:integer4
               (f2cl-lib:int-add
                (f2cl-lib:int
                 (/
                  (f2cl-lib:flog
                   (/ (f2cl-lib:freal minmn)
                      (f2cl-lib:freal (f2cl-lib:int-add smlsiz 1))))
                  (f2cl-lib:flog two)))
                1))
          (the f2cl-lib:integer4 0)))
(setf liwork
        (f2cl-lib:int-add (f2cl-lib:int-mul 3 minmn nlvl)
                          (f2cl-lib:int-mul 11 minmn)))
(setf mm m)
(cond
 ((and (>= m n) (>= m mnthr)) (setf mm n)
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add n
                                      (f2cl-lib:int-mul n
                                                        (multiple-value-bind
                                                            (ret-val
                                                             var-0
                                                             var-1
                                                             var-2
                                                             var-3
                                                             var-4
                                                             var-5
                                                             var-6)
                                                            (ilaenv
                                                             1
                                                             "SGEQRF"
                                                             " " m
                                                             n -1
                                                             -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-5
                                                            var-6))
                                                          (setf m
                                                                  var-3)
                                                          (setf n
                                                                  var-4)
                                                          ret-val))))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
```

```
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add n
                                           (f2cl-lib:int-mul nrhs
                                                             (multiple-value-bind
                                                                 (ret-val
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-3
                                                                  var-4
                                                                  var-5
                                                                  var-6)
                                                                 (ilaenv
                                                                  1
                                                                  "SORMQR"
                                                                  "LT" m
                                                                  nrhs n
                                                                  -1)
                                                               (declare
                                                                (ignore
                                                                 var-0
                                                                 var-1
                                                                 var-2
                                                                 var-6))
                                                               (setf m
                                                                       var-3)
                                                               (setf nrhs
                                                                       var-4)
                                                               (setf n
                                                                       var-5)
                                                               ret-val))))))))
(cond
 ((>= m n)
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                      (f2cl-lib:int-mul
                                       (f2cl-lib:int-add mm n)
                                       (multiple-value-bind
                                           (ret-val var-0 var-1
                                            var-2 var-3 var-4 var-5
                                            var-6)
                                           (ilaenv 1 "SGEBRD" " "
                                            mm n -1 -1)
                                         (declare
                                          (ignore var-0 var-1 var-2
                                           var-5 var-6))
                                         (setf mm var-3)
                                         (setf n var-4)
```

```
                                        ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul nrhs
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "SORMBR"
                                                           "QLT"
                                                           mm
                                                           nrhs n
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (setf mm
                                                                var-3)
                                                        (setf nrhs
                                                                var-4)
                                                        (setf n
                                                                var-5)
                                                        ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul
                                     (f2cl-lib:int-sub n 1)
                                     (multiple-value-bind
                                         (ret-val var-0 var-1
                                          var-2 var-3 var-4 var-5
                                          var-6)
                                         (ilaenv 1 "SORMBR" "PLN"
                                          n nrhs n -1)
                                       (declare
                                        (ignore var-0 var-1 var-2
                                         var-6))
```

```
                                        (setf n var-3)
                                        (setf nrhs var-4)
                                        (setf n var-5)
                                        ret-val))))))
  (setf wlalsd
          (f2cl-lib:int-add (f2cl-lib:int-mul 9 n)
                            (f2cl-lib:int-mul 2 n smlsiz)
                            (f2cl-lib:int-mul 8 n nlvl)
                            (f2cl-lib:int-mul n nrhs)
                            (expt (f2cl-lib:int-add smlsiz 1) 2)))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                      wlalsd))))
  (setf minwrk
          (max
           (the f2cl-lib:integer4
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) mm))
           (the f2cl-lib:integer4
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) nrhs))
           (the f2cl-lib:integer4
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                  wlalsd))))))
(cond
 ((> n m)
  (setf wlalsd
          (f2cl-lib:int-add (f2cl-lib:int-mul 9 m)
                            (f2cl-lib:int-mul 2 m smlsiz)
                            (f2cl-lib:int-mul 8 m nlvl)
                            (f2cl-lib:int-mul m nrhs)
                            (expt (f2cl-lib:int-add smlsiz 1) 2)))
  (cond
   ((>= n mnthr)
    (setf maxwrk
            (f2cl-lib:int-add m
                              (f2cl-lib:int-mul m
                                                (multiple-value-bind
                                                    (ret-val var-0
                                                     var-1 var-2
                                                     var-3 var-4
                                                     var-5 var-6)
                                                    (ilaenv 1
                                                     "SGELQF" " " m
                                                     n -1 -1)
                                                  (declare
                                                   (ignore var-0
                                                    var-1 var-2
                                                    var-5 var-6))
                                                  (setf m var-3)
```

```
                                                    (setf n var-4)
                                                    ret-val))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                    (f2cl-lib:int-mul 4 m)
                                    (f2cl-lib:int-mul 2 m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "SGEBRD"
                                                           " "
                                                           m m
                                                           -1
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-5
                                                          var-6))
                                                        (setf m
                                                                var-3)
                                                        (setf m
                                                                var-4)
                                                        ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                    (f2cl-lib:int-mul 4 m)
                                    (f2cl-lib:int-mul nrhs
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
```

```
                                                                var-6)
                                                               (ilaenv
                                                                1
                                                                "SORMBR"
                                                                "QLT"
                                                                m
                                                                nrhs
                                                                m
                                                                -1)
                                                              (declare
                                                               (ignore
                                                                var-0
                                                                var-1
                                                                var-2
                                                                var-6))
                                                              (setf m
                                                                      var-3)
                                                              (setf nrhs
                                                                      var-4)
                                                              (setf m
                                                                      var-5)
                                                              ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                    (f2cl-lib:int-mul 4 m)
                                    (f2cl-lib:int-mul
                                     (f2cl-lib:int-sub m 1)
                                     (multiple-value-bind
                                         (ret-val var-0 var-1
                                          var-2 var-3 var-4
                                          var-5 var-6)
                                         (ilaenv 1 "SORMBR"
                                          "PLN" m nrhs m -1)
                                       (declare
                                        (ignore var-0 var-1
                                         var-2 var-6))
                                       (setf m var-3)
                                       (setf nrhs var-4)
                                       (setf m var-5)
                                       ret-val))))))
(cond
 ((> nrhs 1)
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul m m) m
                                      (f2cl-lib:int-mul m
                                                        nrhs))))))
```

```
 (t
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                      (f2cl-lib:int-mul 2
                                                        m)))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add m
                                    (f2cl-lib:int-mul nrhs
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "SORMLQ"
                                                           "LT"
                                                           n
                                                           nrhs
                                                           m
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (setf n
                                                                var-3)
                                                        (setf nrhs
                                                                var-4)
                                                        (setf m
                                                                var-5)
                                                        ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                    (f2cl-lib:int-mul 4 m)
                                    wlalsd))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
```

```
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul 4 m)
                                       (f2cl-lib:int-mul m m)
                                       (max
                                        (the f2cl-lib:integer4 m)
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-sub
                                              (f2cl-lib:int-mul 2
                                                                m)
                                              4))
                                        (the f2cl-lib:integer4
                                             nrhs)
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-sub n
                                                               (f2cl-lib:int-mul
                                                                3
                                                                m)))))))))
        (t
         (setf maxwrk
                 (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-add n m)
                                    (multiple-value-bind
                                        (ret-val var-0 var-1 var-2 var-3
                                         var-4 var-5 var-6)
                                        (ilaenv 1 "SGEBRD" " " m n -1
                                         -1)
                                      (declare
                                       (ignore var-0 var-1 var-2 var-5
                                        var-6))
                                      (setf m var-3)
                                      (setf n var-4)
                                      ret-val))))
         (setf maxwrk
                 (max (the f2cl-lib:integer4 maxwrk)
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                             (f2cl-lib:int-mul nrhs
                                                               (multiple-value-bind
                                                                   (ret-val
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-3
                                                                    var-4
                                                                    var-5
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "SORMBR"
```

```
                                                                    "QLT"
                                                                    m
                                                                    nrhs
                                                                    n
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (setf m
                                                                          var-3)
                                                                  (setf nrhs
                                                                          var-4)
                                                                  (setf n
                                                                          var-5)
                                                                  ret-val))))))
                  (setf maxwrk
                          (max (the f2cl-lib:integer4 maxwrk)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                                      (f2cl-lib:int-mul m
                                                                        (multiple-value-bind
                                                                            (ret-val
                                                                             var-0
                                                                             var-1
                                                                             var-2
                                                                             var-3
                                                                             var-4
                                                                             var-5
                                                                             var-6)
                                                                            (ilaenv
                                                                             1
                                                                             "SORMBR"
                                                                             "PLN"
                                                                             n
                                                                             nrhs
                                                                             m
                                                                             -1)
                                                                          (declare
                                                                           (ignore
                                                                            var-0
                                                                            var-1
                                                                            var-2
                                                                            var-6))
                                                                          (setf n
                                                                                  var-3)
                                                                          (setf nrhs
                                                                                  var-4)
```

```
                                                                    (setf m
                                                                            var-5)
                                                                    ret-val))))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                            wlalsd))))))
      (setf minwrk
              (max
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) nrhs))
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) m))
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                      wlalsd))))))))
  (setf minwrk
          (min (the f2cl-lib:integer4 minwrk)
               (the f2cl-lib:integer4 maxwrk)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 maxwrk) 'single-float))
  (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) liwork)
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -12)))))
(cond
 ((/= info 0) (xerbla "SGELSD" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond ((or (= m 0) (= n 0)) (setf rank 0) (go end_label)))
(setf eps (slamch "P"))
(setf sfmin (slamch "S"))
(setf smlnum (/ sfmin eps))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (slabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "M" m n a lda work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iascl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum))
  (multiple-value-bind
```

```
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl "G" 0 0 anrm smlnum m n a lda info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf smlnum var-4))
      (when var-5 (setf m var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf info var-9)))
    (setf iascl 1))
   ((> anrm bignum)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl "G" 0 0 anrm bignum m n a lda info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf bignum var-4))
      (when var-5 (setf m var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf info var-9)))
    (setf iascl 2))
   ((= anrm zero)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "F"
         (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
         zero zero b ldb$)
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf nrhs var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldb$ var-6)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "F" minmn 1 zero zero s 1)
      (declare (ignore var-0 var-2 var-5 var-6))
      (when var-1 (setf minmn var-1))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4)))
    (setf rank 0) (go label10)))
  (setf bnrm
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
              (slange "M" m nrhs b ldb$ work)
            (declare (ignore var-0 var-3 var-5))
            (when var-1 (setf m var-1))
            (when var-2 (setf nrhs var-2))
            (when var-4 (setf ldb$ var-4))
            ret-val))
  (setf ibscl 0)
  (cond
```

```
 ((and (> bnrm zero) (< bnrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 bnrm smlnum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 1))
 ((> bnrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 bnrm bignum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 2)))
(if (< m n)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "F" (f2cl-lib:int-sub n m) nrhs zero zero
         (f2cl-lib:array-slice b-%data% single-float ((+ m 1) 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf nrhs var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldb$ var-6))))
(cond
 ((>= m n) (setf mm m)
  (cond
   ((>= m mnthr) (setf mm n) (setf itau 1)
    (setf nwork (f2cl-lib:int-add itau n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (sgeqrf m n a lda
         (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                               work-%offset%)
         (f2cl-lib:array-slice work-%data% single-float (nwork) ((1 *))
                               work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
      (declare (ignore var-2 var-4 var-5 var-6))
      (setf m var-0)
```

```
      (setf n var-1)
      (setf lda var-3)
      (setf info var-7))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (sormqr "L" "T" m nrhs n a lda
         (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                               work-%offset%)
         b ldb$
         (f2cl-lib:array-slice work-%data% single-float (nwork) ((1 *))
                               work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf nrhs var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldb$ var-9))
      (when var-12 (setf info var-12)))
    (cond
     ((> n 1)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
           zero zero
           (f2cl-lib:array-slice a-%data% single-float (2 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-5))
        (when var-3 (setf zero var-3))
        (when var-4 (setf zero var-4))
        (when var-6 (setf lda var-6)))))))
  (setf ie 1) (setf itauq (f2cl-lib:int-add ie n))
  (setf itaup (f2cl-lib:int-add itauq n))
  (setf nwork (f2cl-lib:int-add itaup n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (sgebrd mm n a lda s
       (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (nwork) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
    (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
    (setf mm var-0)
```

```
  (setf n var-1)
  (setf lda var-3)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "Q" "L" "T" mm nrhs n a lda
     (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
                           work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% single-float (nwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf mm var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (slalsd "U" smlsiz n nrhs s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     b ldb$ rcond rank
     (f2cl-lib:array-slice work-%data% single-float (nwork) ((1 *))
                           work-%offset%)
     iwork info)
  (declare (ignore var-0 var-4 var-5 var-6 var-10 var-11))
  (when var-1 (setf smlsiz var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf nrhs var-3))
  (when var-7 (setf ldb$ var-7))
  (when var-8 (setf rcond var-8))
  (when var-9 (setf rank var-9))
  (when var-12 (setf info var-12)))
(cond ((/= info 0) (go label10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "P" "L" "N" n nrhs n a lda
     (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                           work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% single-float (nwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
```

```
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
      (when var-3 (setf n var-3))
      (when var-4 (setf nrhs var-4))
      (when var-5 (setf n var-5))
      (when var-7 (setf lda var-7))
      (when var-10 (setf ldb$ var-10))
      (when var-13 (setf info var-13))))
    ((and (>= n mnthr)
          (>= lwork
              (f2cl-lib:int-add (f2cl-lib:int-mul 4 m)
                                (f2cl-lib:int-mul m m)
                                (max (the f2cl-lib:integer4 m)
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           (f2cl-lib:int-mul 2 m)
                                           (f2cl-lib:int-sub 4)))
                                     (the f2cl-lib:integer4 nrhs)
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add n
                                                            (f2cl-lib:int-mul
                                                             -1 3 m)))
                                     (the f2cl-lib:integer4 wlalsd)))))
     (setf ldwork m)
     (if (>= lwork
             (max
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 4 m)
                                     (f2cl-lib:int-mul m lda)
                                     (max (the f2cl-lib:integer4 m)
                                          (the f2cl-lib:integer4
                                               (f2cl-lib:int-sub
                                                (f2cl-lib:int-mul 2 m) 4))
                                          (the f2cl-lib:integer4 nrhs)
                                          (the f2cl-lib:integer4
                                               (f2cl-lib:int-sub n
                                                                 (f2cl-lib:int-mul
                                                                  3
                                                                  m))))))
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul m lda) m
                                     (f2cl-lib:int-mul m nrhs)))
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 4 m)
                                     (f2cl-lib:int-mul m lda) wlalsd))))
         (setf ldwork lda))
     (setf itau 1) (setf nwork (f2cl-lib:int-add m 1))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (sgelqf m n a lda
```

```
     (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (nwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-7))
(setf il nwork)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                           work-%offset%)
     ldwork)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwork var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1) zero
     zero
     (f2cl-lib:array-slice work-%data% single-float ((+ il ldwork))
                           ((1 *)) work-%offset%)
     ldwork)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwork var-6)))
(setf ie (f2cl-lib:int-add il (f2cl-lib:int-mul ldwork m)))
(setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf nwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (sgebrd m m
     (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                           work-%offset%)
     ldwork s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (nwork) ((1 *))
                           work-%offset%)
```

```
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwork var-3)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "Q" "L" "T" m nrhs m
     (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                           work-%offset%)
     ldwork
     (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
                           work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% single-float (nwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwork var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (slalsd "U" smlsiz m nrhs s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     b ldb$ rcond rank
     (f2cl-lib:array-slice work-%data% single-float (nwork) ((1 *))
                           work-%offset%)
     iwork info)
  (declare (ignore var-0 var-4 var-5 var-6 var-10 var-11))
  (when var-1 (setf smlsiz var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf nrhs var-3))
  (when var-7 (setf ldb$ var-7))
  (when var-8 (setf rcond var-8))
  (when var-9 (setf rank var-9))
  (when var-12 (setf info var-12)))
(cond ((/= info 0) (go label10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "P" "L" "N" m nrhs m
```

```
      (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                            work-%offset%)
      ldwork
      (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                            work-%offset%)
      b ldb$
      (f2cl-lib:array-slice work-%data% single-float (nwork) ((1 *))
                            work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf nrhs var-4))
    (when var-5 (setf m var-5))
    (when var-7 (setf ldwork var-7))
    (when var-10 (setf ldb$ var-10))
    (when var-13 (setf info var-13)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "F" (f2cl-lib:int-sub n m) nrhs zero zero
       (f2cl-lib:array-slice b-%data% single-float ((+ m 1) 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldb$ var-6)))
  (setf nwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sormlq "L" "T" n nrhs m a lda
       (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                             work-%offset%)
       b ldb$
       (f2cl-lib:array-slice work-%data% single-float (nwork) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf m var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldb$ var-9))
    (when var-12 (setf info var-12))))
 (t (setf ie 1) (setf itauq (f2cl-lib:int-add ie m))
  (setf itaup (f2cl-lib:int-add itauq m))
  (setf nwork (f2cl-lib:int-add itaup m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
     var-10)
    (sgebrd m n a lda s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (nwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "Q" "L" "T" m nrhs n a lda
     (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
                           work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% single-float (nwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (slalsd "L" smlsiz m nrhs s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     b ldb$ rcond rank
     (f2cl-lib:array-slice work-%data% single-float (nwork) ((1 *))
                           work-%offset%)
     iwork info)
  (declare (ignore var-0 var-4 var-5 var-6 var-10 var-11))
  (when var-1 (setf smlsiz var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf nrhs var-3))
  (when var-7 (setf ldb$ var-7))
  (when var-8 (setf rcond var-8))
```

```
      (when var-9 (setf rank var-9))
      (when var-12 (setf info var-12)))
    (cond ((/= info 0) (go label10)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (sormbr "P" "L" "N" n nrhs m a lda
         (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                               work-%offset%)
         b ldb$
         (f2cl-lib:array-slice work-%data% single-float (nwork) ((1 *))
                               work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
      (when var-3 (setf n var-3))
      (when var-4 (setf nrhs var-4))
      (when var-5 (setf m var-5))
      (when var-7 (setf lda var-7))
      (when var-10 (setf ldb$ var-10))
      (when var-13 (setf info var-13)))))
  (cond
   ((= iascl 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl "G" 0 0 anrm smlnum n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf smlnum var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl "G" 0 0 smlnum anrm minmn 1 s minmn info)
      (declare (ignore var-0 var-1 var-2 var-6 var-7))
      (when var-3 (setf smlnum var-3))
      (when var-4 (setf anrm var-4))
      (when var-5 (setf minmn var-5))
      (when var-8 (setf minmn var-8))
      (when var-9 (setf info var-9))))
   ((= iascl 2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl "G" 0 0 anrm bignum n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf bignum var-4))
      (when var-5 (setf n var-5))
```

```
            (when var-6 (setf nrhs var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf info var-9)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (slascl "G" 0 0 bignum anrm minmn 1 s minmn info)
            (declare (ignore var-0 var-1 var-2 var-6 var-7))
            (when var-3 (setf bignum var-3))
            (when var-4 (setf anrm var-4))
            (when var-5 (setf minmn var-5))
            (when var-8 (setf minmn var-8))
            (when var-9 (setf info var-9)))))
        (cond
         ((= ibscl 1)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (slascl "G" 0 0 smlnum bnrm n nrhs b ldb$ info)
            (declare (ignore var-0 var-1 var-2 var-7))
            (when var-3 (setf smlnum var-3))
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf n var-5))
            (when var-6 (setf nrhs var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf info var-9))))
         ((= ibscl 2)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (slascl "G" 0 0 bignum bnrm n nrhs b ldb$ info)
            (declare (ignore var-0 var-1 var-2 var-7))
            (when var-3 (setf bignum var-3))
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf n var-5))
            (when var-6 (setf nrhs var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf info var-9)))))
       label10
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 maxwrk) 'single-float))
        (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) liwork)
        (go end_label)
       end_label
        (return
         (values m n nrhs nil lda nil ldb$ nil rcond rank nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgelsd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
```

```
'((fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (single-float)
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::m
  fortran-to-lisp::n
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$ nil
  fortran-to-lisp::rcond
  fortran-to-lisp::rank nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::sgeqrf
  fortran-to-lisp::sgelqf
  fortran-to-lisp::sgebrd
  fortran-to-lisp::ilaenv))))
```

———-

21.3 sgelss

21.3.1 Axiom unit tests

— sgelss.input —

```
)set break resume
)sys rm -f sgelss.output
)spool sgelss.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

21.3.2 Axiom help page

— **sgelss.help** —

```
=====================================================================
sgelss examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgelss (M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK, WORK,
           LWORK, INFO)
            SGELSS solves overdetermined or underdetermined systems for GE
           matrices

Function/Subroutine Documentation
   subroutine sgelss (integer M, integer N, integer NRHS, real, dimension(
       lda, * ) A, integer LDA, real, dimension( ldb, * ) B, integer LDB,
       real, dimension( * ) S, real RCOND, integer RANK, real, dimension( * )
       WORK, integer LWORK, integer INFO)
        SGELSS solves overdetermined or underdetermined systems for GE
       matrices

Purpose:
   SGELSS computes the minimum norm solution to a real linear least
   squares problem:

   Minimize 2-norm(| b - A*x |).

   using the singular value decomposition (SVD) of A. A is an M-by-N
   matrix which may be rank-deficient.

   Several right hand side vectors b and solution vectors x can be
   handled in a single call; they are stored as the columns of the
   M-by-NRHS right hand side matrix B and the N-by-NRHS solution matrix X.

   The effective rank of A is determined by treating as zero those
   singular values which are less than RCOND times the largest singular
   value.
```

```
Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A. M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A. N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X. NRHS >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, the first min(m,n) rows of A are overwritten with
   its right singular vectors, stored rowwise.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 B
   B is REAL array, dimension (LDB,NRHS)
   On entry, the M-by-NRHS right hand side matrix B.
   On exit, B is overwritten by the N-by-NRHS solution
   matrix X.  If m >= n and RANK = n, the residual
   sum-of-squares for the solution in the i-th column is given
   by the sum of squares of elements n+1:m in that column.

 LDB
   LDB is INTEGER
   The leading dimension of the array B. LDB >= max(1,max(M,N)).

 S
   S is REAL array, dimension (min(M,N))
   The singular values of A in decreasing order.
   The condition number of A in the 2-norm = S(1)/S(min(m,n)).

 RCOND
   RCOND is REAL
   RCOND is used to determine the effective rank of A.
   Singular values S(i) <= RCOND*S(1) are treated as zero.
   If RCOND < 0, machine precision is used instead.

 RANK
```

```
  RANK is INTEGER
  The effective rank of A, i.e., the number of singular values
  which are greater than RCOND*S(1).

 WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK >= 1, and also:
  LWORK >= 3*min(M,N) + max( 2*min(M,N), max(M,N), NRHS )
  For good performance, LWORK should generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

 INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  the algorithm for computing the SVD failed to converge;
        if INFO = i, i off-diagonal elements of an intermediate
        bidiagonal form did not converge to zero.
```

———-

21.3.3 fortran code

— sgelss.f —

```
*  =====================================================================
      SUBROUTINE SGELSS( M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK,
     $                   WORK, LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDB, LWORK, M, N, NRHS, RANK
      REAL               RCOND
*     ..
*     .. Array Arguments ..
```

```
      REAL               A( LDA, * ), B( LDB, * ), S( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            BDSPAC, BL, CHUNK, I, IASCL, IBSCL, IE, IL,
     $                   ITAU, ITAUP, ITAUQ, IWORK, LDWORK, MAXMN,
     $                   MAXWRK, MINMN, MINWRK, MM, MNTHR
      INTEGER            LWORK_SGEQRF, LWORK_SORMQR, LWORK_SGEBRD,
     $                   LWORK_SORMBR, LWORK_SORGBR, LWORK_SORMLQ
      REAL               ANRM, BIGNUM, BNRM, EPS, SFMIN, SMLNUM, THR
*     ..
*     .. Local Arrays ..
      REAL               DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SBDSQR, SCOPY, SGEBRD, SGELQF, SGEMM, SGEMV,
     $                   SGEQRF, SLABAD, SLACPY, SLASCL, SLASET, SORGBR,
     $                   SORMBR, SORMLQ, SORMQR, SRSCL, XERBLA
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      REAL               SLAMCH, SLANGE
      EXTERNAL           ILAENV, SLAMCH, SLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      MINMN = MIN( M, N )
      MAXMN = MAX( M, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
```

```
      ELSE IF( LDB.LT.MAX( 1, MAXMN ) ) THEN
         INFO = -7
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = 1
         MAXWRK = 1
         IF( MINMN.GT.0 ) THEN
            MM = M
            MNTHR = ILAENV( 6, 'SGELSS', ' ', M, N, NRHS, -1 )
            IF( M.GE.N .AND. M.GE.MNTHR ) THEN
*
*              Path 1a - overdetermined, with many more rows than
*                        columns
*
*              Compute space needed for SGEQRF
               CALL SGEQRF( M, N, A, LDA, DUM(1), DUM(1), -1, INFO )
               LWORK_SGEQRF=DUM(1)
*              Compute space needed for SORMQR
               CALL SORMQR( 'L', 'T', M, NRHS, N, A, LDA, DUM(1), B,
     $                    LDB, DUM(1), -1, INFO )
               LWORK_SORMQR=DUM(1)
               MM = N
               MAXWRK = MAX( MAXWRK, N + LWORK_SGEQRF )
               MAXWRK = MAX( MAXWRK, N + LWORK_SORMQR )
            END IF
            IF( M.GE.N ) THEN
*
*              Path 1 - overdetermined or exactly determined
*
*              Compute workspace needed for SBDSQR
*
               BDSPAC = MAX( 1, 5*N )
*              Compute space needed for SGEBRD
               CALL SGEBRD( MM, N, A, LDA, S, DUM(1), DUM(1),
     $                      DUM(1), DUM(1), -1, INFO )
               LWORK_SGEBRD=DUM(1)
*              Compute space needed for SORMBR
               CALL SORMBR( 'Q', 'L', 'T', MM, NRHS, N, A, LDA, DUM(1),
     $                B, LDB, DUM(1), -1, INFO )
               LWORK_SORMBR=DUM(1)
*              Compute space needed for SORGBR
               CALL SORGBR( 'P', N, N, N, A, LDA, DUM(1),
```

```
     $                  DUM(1), -1, INFO )
            LWORK_SORGBR=DUM(1)
*           Compute total workspace needed
            MAXWRK = MAX( MAXWRK, 3*N + LWORK_SGEBRD )
            MAXWRK = MAX( MAXWRK, 3*N + LWORK_SORMBR )
            MAXWRK = MAX( MAXWRK, 3*N + LWORK_SORGBR )
            MAXWRK = MAX( MAXWRK, BDSPAC )
            MAXWRK = MAX( MAXWRK, N*NRHS )
            MINWRK = MAX( 3*N + MM, 3*N + NRHS, BDSPAC )
            MAXWRK = MAX( MINWRK, MAXWRK )
         END IF
         IF( N.GT.M ) THEN
*
*           Compute workspace needed for SBDSQR
*
            BDSPAC = MAX( 1, 5*M )
            MINWRK = MAX( 3*M+NRHS, 3*M+N, BDSPAC )
            IF( N.GE.MNTHR ) THEN
*
*              Path 2a - underdetermined, with many more columns
*              than rows
*
*              Compute space needed for SGEBRD
               CALL SGEBRD( M, M, A, LDA, S, DUM(1), DUM(1),
     $                    DUM(1), DUM(1), -1, INFO )
               LWORK_SGEBRD=DUM(1)
*              Compute space needed for SORMBR
               CALL SORMBR( 'Q', 'L', 'T', M, NRHS, N, A, LDA,
     $                 DUM(1), B, LDB, DUM(1), -1, INFO )
               LWORK_SORMBR=DUM(1)
*              Compute space needed for SORGBR
               CALL SORGBR( 'P', M, M, M, A, LDA, DUM(1),
     $                   DUM(1), -1, INFO )
               LWORK_SORGBR=DUM(1)
*              Compute space needed for SORMLQ
               CALL SORMLQ( 'L', 'T', N, NRHS, M, A, LDA, DUM(1),
     $                  B, LDB, DUM(1), -1, INFO )
               LWORK_SORMLQ=DUM(1)
*              Compute total workspace needed
               MAXWRK = M + M*ILAENV( 1, 'SGELQF', ' ', M, N, -1,
     $                              -1 )
               MAXWRK = MAX( MAXWRK, M*M + 4*M + LWORK_SGEBRD )
               MAXWRK = MAX( MAXWRK, M*M + 4*M + LWORK_SORMBR )
               MAXWRK = MAX( MAXWRK, M*M + 4*M + LWORK_SORGBR )
               MAXWRK = MAX( MAXWRK, M*M + M + BDSPAC )
               IF( NRHS.GT.1 ) THEN
                  MAXWRK = MAX( MAXWRK, M*M + M + M*NRHS )
               ELSE
                  MAXWRK = MAX( MAXWRK, M*M + 2*M )
               END IF
```

```
                 MAXWRK = MAX( MAXWRK, M + LWORK_SORMLQ )
              ELSE
*
*                Path 2 - underdetermined
*
*                Compute space needed for SGEBRD
                 CALL SGEBRD( M, N, A, LDA, S, DUM(1), DUM(1),
     $                     DUM(1), DUM(1), -1, INFO )
                 LWORK_SGEBRD=DUM(1)
*                Compute space needed for SORMBR
                 CALL SORMBR( 'Q', 'L', 'T', M, NRHS, M, A, LDA,
     $               DUM(1), B, LDB, DUM(1), -1, INFO )
                 LWORK_SORMBR=DUM(1)
*                Compute space needed for SORGBR
                 CALL SORGBR( 'P', M, N, M, A, LDA, DUM(1),
     $                  DUM(1), -1, INFO )
                 LWORK_SORGBR=DUM(1)
                 MAXWRK = 3*M + LWORK_SGEBRD
                 MAXWRK = MAX( MAXWRK, 3*M + LWORK_SORMBR )
                 MAXWRK = MAX( MAXWRK, 3*M + LWORK_SORGBR )
                 MAXWRK = MAX( MAXWRK, BDSPAC )
                 MAXWRK = MAX( MAXWRK, N*NRHS )
              END IF
           END IF
           MAXWRK = MAX( MINWRK, MAXWRK )
        END IF
        WORK( 1 ) = MAXWRK
*
        IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY )
     $      INFO = -12
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGELSS', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         RANK = 0
         RETURN
      END IF
*
*     Get machine parameters
*
      EPS = SLAMCH( 'P' )
      SFMIN = SLAMCH( 'S' )
```

```
      SMLNUM = SFMIN / EPS
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = SLANGE( 'M', M, N, A, LDA, WORK )
      IASCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL SLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
         IASCL = 1
      ELSE IF( ANRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL SLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
         IASCL = 2
      ELSE IF( ANRM.EQ.ZERO ) THEN
*
*        Matrix all zero. Return zero solution.
*
         CALL SLASET( 'F', MAX( M, N ), NRHS, ZERO, ZERO, B, LDB )
         CALL SLASET( 'F', MINMN, 1, ZERO, ZERO, S, 1 )
         RANK = 0
         GO TO 70
      END IF
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = SLANGE( 'M', M, NRHS, B, LDB, WORK )
      IBSCL = 0
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL SLASCL( 'G', 0, 0, BNRM, SMLNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 1
      ELSE IF( BNRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL SLASCL( 'G', 0, 0, BNRM, BIGNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 2
      END IF
*
*     Overdetermined case
*
```

```
      IF( M.GE.N ) THEN
*
*        Path 1 - overdetermined or exactly determined
*
         MM = M
         IF( M.GE.MNTHR ) THEN
*
*           Path 1a - overdetermined, with many more rows than columns
*
            MM = N
            ITAU = 1
            IWORK = ITAU + N
*
*           Compute A=Q*R
*           (Workspace: need 2*N, prefer N+N*NB)
*
            CALL SGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
     $                   LWORK-IWORK+1, INFO )
*
*           Multiply B by transpose(Q)
*           (Workspace: need N+NRHS, prefer N+NRHS*NB)
*
            CALL SORMQR( 'L', 'T', M, NRHS, N, A, LDA, WORK( ITAU ), B,
     $                   LDB, WORK( IWORK ), LWORK-IWORK+1, INFO )
*
*           Zero out below R
*
            IF( N.GT.1 )
     $         CALL SLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ), LDA )
         END IF
*
         IE = 1
         ITAUQ = IE + N
         ITAUP = ITAUQ + N
         IWORK = ITAUP + N
*
*        Bidiagonalize R in A
*        (Workspace: need 3*N+MM, prefer 3*N+(MM+N)*NB)
*
         CALL SGEBRD( MM, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors of R
*        (Workspace: need 3*N+NRHS, prefer 3*N+NRHS*NB)
*
         CALL SORMBR( 'Q', 'L', 'T', MM, NRHS, N, A, LDA, WORK( ITAUQ ),
     $                B, LDB, WORK( IWORK ), LWORK-IWORK+1, INFO )
*
*        Generate right bidiagonalizing vectors of R in A
```

```
*        (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
         CALL SORGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                WORK( IWORK ), LWORK-IWORK+1, INFO )
         IWORK = IE + N
*
*        Perform bidiagonal QR iteration
*          multiply B by transpose of left singular vectors
*          compute right singular vectors in A
*        (Workspace: need BDSPAC)
*
         CALL SBDSQR( 'U', N, N, 0, NRHS, S, WORK( IE ), A, LDA, DUM,
     $                1, B, LDB, WORK( IWORK ), INFO )
         IF( INFO.NE.0 )
     $      GO TO 70
*
*        Multiply B by reciprocals of singular values
*
         THR = MAX( RCOND*S( 1 ), SFMIN )
         IF( RCOND.LT.ZERO )
     $      THR = MAX( EPS*S( 1 ), SFMIN )
         RANK = 0
         DO 10 I = 1, N
            IF( S( I ).GT.THR ) THEN
               CALL SRSCL( NRHS, S( I ), B( I, 1 ), LDB )
               RANK = RANK + 1
            ELSE
               CALL SLASET( 'F', 1, NRHS, ZERO, ZERO, B( I, 1 ), LDB )
            END IF
   10    CONTINUE
*
*        Multiply B by right singular vectors
*        (Workspace: need N, prefer N*NRHS)
*
         IF( LWORK.GE.LDB*NRHS .AND. NRHS.GT.1 ) THEN
            CALL SGEMM( 'T', 'N', N, NRHS, N, ONE, A, LDA, B, LDB, ZERO,
     $                  WORK, LDB )
            CALL SLACPY( 'G', N, NRHS, WORK, LDB, B, LDB )
         ELSE IF( NRHS.GT.1 ) THEN
            CHUNK = LWORK / N
            DO 20 I = 1, NRHS, CHUNK
               BL = MIN( NRHS-I+1, CHUNK )
               CALL SGEMM( 'T', 'N', N, BL, N, ONE, A, LDA, B( 1, I ),
     $                     LDB, ZERO, WORK, N )
               CALL SLACPY( 'G', N, BL, WORK, N, B( 1, I ), LDB )
   20       CONTINUE
         ELSE
            CALL SGEMV( 'T', N, N, ONE, A, LDA, B, 1, ZERO, WORK, 1 )
            CALL SCOPY( N, WORK, 1, B, 1 )
         END IF
```

```
*
      ELSE IF( N.GE.MNTHR .AND. LWORK.GE.4*M+M*M+
     $         MAX( M, 2*M-4, NRHS, N-3*M ) ) THEN
*
*        Path 2a - underdetermined, with many more columns than rows
*        and sufficient workspace for an efficient algorithm
*
         LDWORK = M
         IF( LWORK.GE.MAX( 4*M+M*LDA+MAX( M, 2*M-4, NRHS, N-3*M ),
     $       M*LDA+M+M*NRHS ) )LDWORK = LDA
         ITAU = 1
         IWORK = M + 1
*
*        Compute A=L*Q
*        (Workspace: need 2*M, prefer M+M*NB)
*
         CALL SGELQF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
     $               LWORK-IWORK+1, INFO )
         IL = IWORK
*
*        Copy L to WORK(IL), zeroing out above it
*
         CALL SLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWORK )
         CALL SLASET( 'U', M-1, M-1, ZERO, ZERO, WORK( IL+LDWORK ),
     $               LDWORK )
         IE = IL + LDWORK*M
         ITAUQ = IE + M
         ITAUP = ITAUQ + M
         IWORK = ITAUP + M
*
*        Bidiagonalize L in WORK(IL)
*        (Workspace: need M*M+5*M, prefer M*M+4*M+2*M*NB)
*
         CALL SGEBRD( M, M, WORK( IL ), LDWORK, S, WORK( IE ),
     $               WORK( ITAUQ ), WORK( ITAUP ), WORK( IWORK ),
     $               LWORK-IWORK+1, INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors of L
*        (Workspace: need M*M+4*M+NRHS, prefer M*M+4*M+NRHS*NB)
*
         CALL SORMBR( 'Q', 'L', 'T', M, NRHS, M, WORK( IL ), LDWORK,
     $               WORK( ITAUQ ), B, LDB, WORK( IWORK ),
     $               LWORK-IWORK+1, INFO )
*
*        Generate right bidiagonalizing vectors of R in WORK(IL)
*        (Workspace: need M*M+5*M-1, prefer M*M+4*M+(M-1)*NB)
*
         CALL SORGBR( 'P', M, M, M, WORK( IL ), LDWORK, WORK( ITAUP ),
     $               WORK( IWORK ), LWORK-IWORK+1, INFO )
         IWORK = IE + M
```

```
*
*        Perform bidiagonal QR iteration,
*           computing right singular vectors of L in WORK(IL) and
*           multiplying B by transpose of left singular vectors
*        (Workspace: need M*M+M+BDSPAC)
*
         CALL SBDSQR( 'U', M, M, 0, NRHS, S, WORK( IE ), WORK( IL ),
     $               LDWORK, A, LDA, B, LDB, WORK( IWORK ), INFO )
         IF( INFO.NE.0 )
     $      GO TO 70
*
*        Multiply B by reciprocals of singular values
*
         THR = MAX( RCOND*S( 1 ), SFMIN )
         IF( RCOND.LT.ZERO )
     $      THR = MAX( EPS*S( 1 ), SFMIN )
         RANK = 0
         DO 30 I = 1, M
            IF( S( I ).GT.THR ) THEN
               CALL SRSCL( NRHS, S( I ), B( I, 1 ), LDB )
               RANK = RANK + 1
            ELSE
               CALL SLASET( 'F', 1, NRHS, ZERO, ZERO, B( I, 1 ), LDB )
            END IF
   30    CONTINUE
         IWORK = IE
*
*        Multiply B by right singular vectors of L in WORK(IL)
*        (Workspace: need M*M+2*M, prefer M*M+M+M*NRHS)
*
         IF( LWORK.GE.LDB*NRHS+IWORK-1 .AND. NRHS.GT.1 ) THEN
            CALL SGEMM( 'T', 'N', M, NRHS, M, ONE, WORK( IL ), LDWORK,
     $                  B, LDB, ZERO, WORK( IWORK ), LDB )
            CALL SLACPY( 'G', M, NRHS, WORK( IWORK ), LDB, B, LDB )
         ELSE IF( NRHS.GT.1 ) THEN
            CHUNK = ( LWORK-IWORK+1 ) / M
            DO 40 I = 1, NRHS, CHUNK
               BL = MIN( NRHS-I+1, CHUNK )
               CALL SGEMM( 'T', 'N', M, BL, M, ONE, WORK( IL ), LDWORK,
     $                     B( 1, I ), LDB, ZERO, WORK( IWORK ), M )
               CALL SLACPY( 'G', M, BL, WORK( IWORK ), M, B( 1, I ),
     $                      LDB )
   40       CONTINUE
         ELSE
            CALL SGEMV( 'T', M, M, ONE, WORK( IL ), LDWORK, B( 1, 1 ),
     $                  1, ZERO, WORK( IWORK ), 1 )
            CALL SCOPY( M, WORK( IWORK ), 1, B( 1, 1 ), 1 )
         END IF
*
*        Zero out below first M rows of B
```

```
*
         CALL SLASET( 'F', N-M, NRHS, ZERO, ZERO, B( M+1, 1 ), LDB )
         IWORK = ITAU + M
*
*        Multiply transpose(Q) by B
*        (Workspace: need M+NRHS, prefer M+NRHS*NB)
*
         CALL SORMLQ( 'L', 'T', N, NRHS, M, A, LDA, WORK( ITAU ), B,
     $                LDB, WORK( IWORK ), LWORK-IWORK+1, INFO )
*
      ELSE
*
*        Path 2 - remaining underdetermined cases
*
         IE = 1
         ITAUQ = IE + M
         ITAUP = ITAUQ + M
         IWORK = ITAUP + M
*
*        Bidiagonalize A
*        (Workspace: need 3*M+N, prefer 3*M+(M+N)*NB)
*
         CALL SGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors
*        (Workspace: need 3*M+NRHS, prefer 3*M+NRHS*NB)
*
         CALL SORMBR( 'Q', 'L', 'T', M, NRHS, N, A, LDA, WORK( ITAUQ ),
     $                B, LDB, WORK( IWORK ), LWORK-IWORK+1, INFO )
*
*        Generate right bidiagonalizing vectors in A
*        (Workspace: need 4*M, prefer 3*M+M*NB)
*
         CALL SORGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                WORK( IWORK ), LWORK-IWORK+1, INFO )
         IWORK = IE + M
*
*        Perform bidiagonal QR iteration,
*           computing right singular vectors of A in A and
*           multiplying B by transpose of left singular vectors
*        (Workspace: need BDSPAC)
*
         CALL SBDSQR( 'L', M, N, 0, NRHS, S, WORK( IE ), A, LDA, DUM,
     $                1, B, LDB, WORK( IWORK ), INFO )
         IF( INFO.NE.0 )
     $      GO TO 70
*
*        Multiply B by reciprocals of singular values
```

```
*
         THR = MAX( RCOND*S( 1 ), SFMIN )
         IF( RCOND.LT.ZERO )
     $      THR = MAX( EPS*S( 1 ), SFMIN )
         RANK = 0
         DO 50 I = 1, M
            IF( S( I ).GT.THR ) THEN
               CALL SRSCL( NRHS, S( I ), B( I, 1 ), LDB )
               RANK = RANK + 1
            ELSE
               CALL SLASET( 'F', 1, NRHS, ZERO, ZERO, B( I, 1 ), LDB )
            END IF
   50    CONTINUE
*
*        Multiply B by right singular vectors of A
*        (Workspace: need N, prefer N*NRHS)
*
         IF( LWORK.GE.LDB*NRHS .AND. NRHS.GT.1 ) THEN
            CALL SGEMM( 'T', 'N', N, NRHS, M, ONE, A, LDA, B, LDB, ZERO,
     $                  WORK, LDB )
            CALL SLACPY( 'F', N, NRHS, WORK, LDB, B, LDB )
         ELSE IF( NRHS.GT.1 ) THEN
            CHUNK = LWORK / N
            DO 60 I = 1, NRHS, CHUNK
               BL = MIN( NRHS-I+1, CHUNK )
               CALL SGEMM( 'T', 'N', N, BL, M, ONE, A, LDA, B( 1, I ),
     $                     LDB, ZERO, WORK, N )
               CALL SLACPY( 'F', N, BL, WORK, N, B( 1, I ), LDB )
   60       CONTINUE
         ELSE
            CALL SGEMV( 'T', M, N, ONE, A, LDA, B, 1, ZERO, WORK, 1 )
            CALL SCOPY( N, WORK, 1, B, 1 )
         END IF
      END IF
*
*     Undo scaling
*
      IF( IASCL.EQ.1 ) THEN
         CALL SLASCL( 'G', 0, 0, ANRM, SMLNUM, N, NRHS, B, LDB, INFO )
         CALL SLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
     $                INFO )
      ELSE IF( IASCL.EQ.2 ) THEN
         CALL SLASCL( 'G', 0, 0, ANRM, BIGNUM, N, NRHS, B, LDB, INFO )
         CALL SLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                INFO )
      END IF
      IF( IBSCL.EQ.1 ) THEN
         CALL SLASCL( 'G', 0, 0, SMLNUM, BNRM, N, NRHS, B, LDB, INFO )
      ELSE IF( IBSCL.EQ.2 ) THEN
         CALL SLASCL( 'G', 0, 0, BIGNUM, BNRM, N, NRHS, B, LDB, INFO )
```

```
      END IF
*
   70 CONTINUE
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of SGELSS
*
      END
```

———-

21.3.4 lisp code

— **sgelss.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sgelss (m n nrhs a lda b ldb$ s rcond rank work lwork info)
    (declare (type (single-float) rcond)
             (type (array single-float (*)) work s b a)
             (type (f2cl-lib:integer4) info lwork rank ldb$ lda nrhs n m))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%)
         (s single-float s-%data% s-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((dum (make-array 1 :element-type 'single-float)) (anrm 0.0)
             (bignum 0.0) (bnrm 0.0) (eps 0.0) (sfmin 0.0) (smlnum 0.0)
             (thr 0.0) (lwork_sgeqrf 0) (lwork_sormqr 0) (lwork_sgebrd 0)
             (lwork_sormbr 0) (lwork_sorgbr 0) (lwork_sormlq 0) (bdspac 0)
             (bl 0) (chunk 0) (i 0) (iascl 0) (ibscl 0) (ie 0) (il 0) (itau 0)
             (itaup 0) (itauq 0) (iwork 0) (ldwork 0) (maxmn 0) (maxwrk 0)
             (minmn 0) (minwrk 0) (mm 0) (mnthr 0) (lquery nil))
        (declare (type (array single-float (1)) dum)
                 (type (single-float) anrm bignum bnrm eps sfmin smlnum thr)
                 (type (f2cl-lib:integer4) lwork_sgeqrf lwork_sormqr
                  lwork_sgebrd lwork_sormbr lwork_sorgbr lwork_sormlq bdspac bl
                  chunk i iascl ibscl ie il itau itaup itauq iwork ldwork maxmn
                  maxwrk minmn minwrk mm mnthr)
                 (type f2cl-lib:logical lquery))
        (setf info 0)
        (setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (setf maxmn (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
```

```
      ((< nrhs 0) (setf info -3))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -5))
      ((< ldb$
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4 maxmn)))
       (setf info -7)))
(cond
 ((= info 0) (setf minwrk 1) (setf maxwrk 1)
  (cond
   ((> minmn 0) (setf mm m)
    (setf mnthr
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 6 "SGELSS" " " m n nrhs -1)
              (declare (ignore var-0 var-1 var-2 var-6))
              (setf m var-3)
              (setf n var-4)
              (setf nrhs var-5)
              ret-val))
    (cond
     ((and (>= m n) (>= m mnthr))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (sgeqrf m n a lda
           (f2cl-lib:array-slice dum single-float (1) ((1 1)))
           (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1 info)
        (declare (ignore var-2 var-4 var-5 var-6))
        (setf m var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf info var-7))
      (setf lwork_sgeqrf
              (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (sormqr "L" "T" m nrhs n a lda
           (f2cl-lib:array-slice dum single-float (1) ((1 1))) b ldb$
           (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1 info)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf nrhs var-3))
        (when var-4 (setf n var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldb$ var-9))
        (when var-12 (setf info var-12)))
      (setf lwork_sormqr
              (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
```

```
    (setf mm n)
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add n lwork_sgeqrf))))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add n lwork_sormqr))))))
  (cond
   ((>= m n)
    (setf bdspac
            (max (the f2cl-lib:integer4 1)
                 (the f2cl-lib:integer4 (f2cl-lib:int-mul 5 n))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (sgebrd mm n a lda s
         (f2cl-lib:array-slice dum single-float (1) ((1 1)))
         (f2cl-lib:array-slice dum single-float (1) ((1 1)))
         (f2cl-lib:array-slice dum single-float (1) ((1 1)))
         (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1 info)
      (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
      (setf mm var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf info var-10))
    (setf lwork_sgebrd
            (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (sormbr "Q" "L" "T" mm nrhs n a lda
         (f2cl-lib:array-slice dum single-float (1) ((1 1))) b ldb$
         (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1 info)
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
      (when var-3 (setf mm var-3))
      (when var-4 (setf nrhs var-4))
      (when var-5 (setf n var-5))
      (when var-7 (setf lda var-7))
      (when var-10 (setf ldb$ var-10))
      (when var-13 (setf info var-13)))
    (setf lwork_sormbr
            (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (sorgbr "P" n n n a lda
         (f2cl-lib:array-slice dum single-float (1) ((1 1)))
         (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1 info)
```

```
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf info var-9)))
  (setf lwork_sorgbr
          (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                      lwork_sgebrd))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                      lwork_sormbr))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                      lwork_sorgbr))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4 bdspac)))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul n nrhs))))
  (setf minwrk
          (max
           (the f2cl-lib:integer4
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) mm))
           (the f2cl-lib:integer4
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) nrhs))
           (the f2cl-lib:integer4 bdspac)))
  (setf maxwrk
          (max (the f2cl-lib:integer4 minwrk)
               (the f2cl-lib:integer4 maxwrk)))))
(cond
 ((> n m)
  (setf bdspac
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul 5 m))))
  (setf minwrk
          (max
           (the f2cl-lib:integer4
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) nrhs))
           (the f2cl-lib:integer4
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
```

```
        (the f2cl-lib:integer4 bdspac)))
(cond
 ((>= n mnthr)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (sgebrd m m a lda s
       (f2cl-lib:array-slice dum single-float (1) ((1 1)))
       (f2cl-lib:array-slice dum single-float (1) ((1 1)))
       (f2cl-lib:array-slice dum single-float (1) ((1 1)))
       (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1
       info)
    (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
    (setf m var-0)
    (setf m var-1)
    (setf lda var-3)
    (setf info var-10))
  (setf lwork_sgebrd
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13)
      (sormbr "Q" "L" "T" m nrhs n a lda
       (f2cl-lib:array-slice dum single-float (1) ((1 1))) b ldb$
       (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1
       info)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf nrhs var-4))
    (when var-5 (setf n var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldb$ var-10))
    (when var-13 (setf info var-13)))
  (setf lwork_sormbr
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (sorgbr "P" m m m a lda
       (f2cl-lib:array-slice dum single-float (1) ((1 1)))
       (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1
       info)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf info var-9)))
  (setf lwork_sorgbr
```

```
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (sormlq "L" "T" n nrhs m a lda
     (f2cl-lib:array-slice dum single-float (1) ((1 1))) b ldb$
     (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1
     info)
  (declare
   (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf n var-2))
  (when var-3 (setf nrhs var-3))
  (when var-4 (setf m var-4))
  (when var-6 (setf lda var-6))
  (when var-9 (setf ldb$ var-9))
  (when var-12 (setf info var-12)))
(setf lwork_sormlq
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(setf maxwrk
        (f2cl-lib:int-add m
                          (f2cl-lib:int-mul m
                                            (multiple-value-bind
                                                (ret-val var-0
                                                 var-1 var-2
                                                 var-3 var-4
                                                 var-5 var-6)
                                                (ilaenv 1
                                                 "SGELQF" " " m
                                                 n -1 -1)
                                              (declare
                                               (ignore var-0
                                                var-1 var-2
                                                var-5 var-6))
                                              (setf m var-3)
                                              (setf n var-4)
                                              ret-val))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                    (f2cl-lib:int-mul 4 m)
                                    lwork_sgebrd))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                    (f2cl-lib:int-mul 4 m)
                                    lwork_sormbr))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
```

```
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                       (f2cl-lib:int-mul 4 m)
                                       lwork_sorgbr))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul m m) m
                                      bdspac))))
  (cond
   ((> nrhs 1)
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul m m) m
                                        (f2cl-lib:int-mul m
                                                          nrhs))))))
   (t
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                        (f2cl-lib:int-mul 2
                                                          m)))))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add m lwork_sormlq)))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (sgebrd m n a lda s
       (f2cl-lib:array-slice dum single-float (1) ((1 1)))
       (f2cl-lib:array-slice dum single-float (1) ((1 1)))
       (f2cl-lib:array-slice dum single-float (1) ((1 1)))
       (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1
       info)
    (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf info var-10))
  (setf lwork_sgebrd
          (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13)
      (sormbr "Q" "L" "T" m nrhs m a lda
       (f2cl-lib:array-slice dum single-float (1) ((1 1))) b ldb$
```

```
            (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1
            info)
          (declare
           (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
          (when var-3 (setf m var-3))
          (when var-4 (setf nrhs var-4))
          (when var-5 (setf m var-5))
          (when var-7 (setf lda var-7))
          (when var-10 (setf ldb$ var-10))
          (when var-13 (setf info var-13)))
        (setf lwork_sormbr
                (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9)
            (sorgbr "P" m n m a lda
             (f2cl-lib:array-slice dum single-float (1) ((1 1)))
             (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1
             info)
          (declare (ignore var-0 var-4 var-6 var-7 var-8))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-3 (setf m var-3))
          (when var-5 (setf lda var-5))
          (when var-9 (setf info var-9)))
        (setf lwork_sorgbr
                (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
        (setf maxwrk
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) lwork_sgebrd))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                            lwork_sormbr))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                            lwork_sorgbr))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4 bdspac)))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-mul n nrhs))))))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 minwrk)
               (the f2cl-lib:integer4 maxwrk)))))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
```

```
          (coerce (the f2cl-lib:integer4 maxwrk) 'single-float))
  (if (and (< lwork minwrk) (not lquery))
      (setf info -12))))
(cond
 ((/= info 0) (xerbla "SGELSS" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond ((or (= m 0) (= n 0)) (setf rank 0) (go end_label)))
(setf eps (slamch "P"))
(setf sfmin (slamch "S"))
(setf smlnum (/ sfmin eps))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (slabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "M" m n a lda work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iascl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 anrm smlnum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 1))
 ((> anrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 anrm bignum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
```

```
  (setf iascl 2))
 ((= anrm zero)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "F"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       zero zero b ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldb$ var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "F" minmn 1 zero zero s 1)
    (declare (ignore var-0 var-2 var-5 var-6))
    (when var-1 (setf minmn var-1))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4)))
  (setf rank 0) (go label70)))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "M" m nrhs b ldb$ work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ibscl 0)
(cond
 ((and (> bnrm zero) (< bnrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 bnrm smlnum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 1))
 ((> bnrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 bnrm bignum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
```

```
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))
    (setf ibscl 2)))
  (cond
   ((>= m n) (setf mm m)
    (cond
     ((>= m mnthr) (setf mm n) (setf itau 1)
      (setf iwork (f2cl-lib:int-add itau n))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (sgeqrf m n a lda
           (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                                 work-%offset%)
           (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                                 work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
        (declare (ignore var-2 var-4 var-5 var-6))
        (setf m var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf info var-7))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (sormqr "L" "T" m nrhs n a lda
           (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                                 work-%offset%)
           b ldb$
           (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                                 work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf nrhs var-3))
        (when var-4 (setf n var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldb$ var-9))
        (when var-12 (setf info var-12)))
      (if (> n 1)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (slaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
               zero zero
               (f2cl-lib:array-slice a-%data% single-float (2 1)
                                     ((1 lda) (1 *)) a-%offset%)
               lda)
            (declare (ignore var-0 var-1 var-2 var-5))
            (when var-3 (setf zero var-3))
            (when var-4 (setf zero var-4))
            (when var-6 (setf lda var-6))))))
```

```
(setf ie 1) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (sgebrd mm n a lda s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf mm var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "Q" "L" "T" mm nrhs n a lda
     (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
                           work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf mm var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (sorgbr "P" n n n a lda
     (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
```

```
  (when var-3 (setf n var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf info var-9)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" n n 0 nrhs s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     a lda dum 1 b ldb$
     (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                           work-%offset%)
     info)
  (declare
   (ignore var-0 var-3 var-5 var-6 var-7 var-9 var-10 var-11 var-13))
  (setf n var-1)
  (setf n var-2)
  (setf nrhs var-4)
  (setf lda var-8)
  (setf ldb$ var-12)
  (setf info var-14))
(if (/= info 0)
    (go label70))
(setf thr
        (max
         (* rcond (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
         sfmin))
(if (< rcond zero)
    (setf thr
            (max
             (* eps (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
             sfmin)))
(setf rank 0)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref s (i) ((1 *))) thr)
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (srscl nrhs
           (f2cl-lib:array-slice s-%data% single-float (i) ((1 *))
                                 s-%offset%)
           (f2cl-lib:array-slice b-%data% single-float (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-2))
        (when var-0 (setf nrhs var-0))
        (when var-3 (setf ldb$ var-3)))
      (setf rank (f2cl-lib:int-add rank 1)))
```

```
      (t
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
           (slaset "F" 1 nrhs zero zero
            (f2cl-lib:array-slice b-%data% single-float (i 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
         (declare (ignore var-0 var-1 var-5))
         (when var-2 (setf nrhs var-2))
         (when var-3 (setf zero var-3))
         (when var-4 (setf zero var-4))
         (when var-6 (setf ldb$ var-6)))))
   label10))
(cond
 ((and (>= lwork (f2cl-lib:int-mul ldb$ nrhs)) (> nrhs 1))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sgemm "T" "N" n nrhs n one a lda b ldb$ zero work ldb$)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldb$ var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldb$ var-12)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "G" n nrhs work ldb$ b ldb$)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf nrhs var-2))
    (when var-4 (setf ldb$ var-4))
    (when var-6 (setf ldb$ var-6))))
 ((> nrhs 1) (setf chunk (the f2cl-lib:integer4 (truncate lwork n)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
                ((> i nrhs) nil)
    (tagbody
      (setf bl
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub nrhs i) 1))
               (the f2cl-lib:integer4 chunk)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (sgemm "T" "N" n bl n one a lda
           (f2cl-lib:array-slice b-%data% single-float (1 i)
                                 ((1 ldb$) (1 *)) b-%offset%)
```

```
              ldb$ zero work n)
            (declare (ignore var-0 var-1 var-6 var-8 var-11))
            (when var-2 (setf n var-2))
            (when var-3 (setf bl var-3))
            (when var-4 (setf n var-4))
            (when var-5 (setf one var-5))
            (when var-7 (setf lda var-7))
            (when var-9 (setf ldb$ var-9))
            (when var-10 (setf zero var-10))
            (when var-12 (setf n var-12)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (slacpy "G" n bl work n
               (f2cl-lib:array-slice b-%data% single-float (1 i)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-0 var-3 var-5))
            (when var-1 (setf n var-1))
            (when var-2 (setf bl var-2))
            (when var-4 (setf n var-4))
            (when var-6 (setf ldb$ var-6)))
         label20)))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (sgemv "T" n n one a lda b 1 zero work 1)
       (declare (ignore var-0 var-4 var-6 var-7 var-9 var-10))
       (when var-1 (setf n var-1))
       (when var-2 (setf n var-2))
       (when var-3 (setf one var-3))
       (when var-5 (setf lda var-5))
       (when var-8 (setf zero var-8)))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (scopy n work 1 b 1)
       (declare (ignore var-1 var-2 var-3 var-4))
       (when var-0 (setf n var-0))))))
  ((and (>= n mnthr)
        (>= lwork
            (f2cl-lib:int-add (f2cl-lib:int-mul 4 m)
                              (f2cl-lib:int-mul m m)
                              (max (the f2cl-lib:integer4 m)
                                   (the f2cl-lib:integer4
                                        (f2cl-lib:int-add
                                         (f2cl-lib:int-mul 2 m)
                                         (f2cl-lib:int-sub 4)))
                                   (the f2cl-lib:integer4 nrhs)
                                   (the f2cl-lib:integer4
                                        (f2cl-lib:int-add n
                                                          (f2cl-lib:int-mul
```

```
                                                          -1 3 m)))))))
(setf ldwork m)
(if (>= lwork
        (max
         (the f2cl-lib:integer4
              (f2cl-lib:int-add (f2cl-lib:int-mul 4 m)
                                (f2cl-lib:int-mul m lda)
                                (max (the f2cl-lib:integer4 m)
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-sub
                                           (f2cl-lib:int-mul 2 m) 4))
                                     (the f2cl-lib:integer4 nrhs)
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-sub n
                                                            (f2cl-lib:int-mul
                                                             3
                                                             m))))))
         (the f2cl-lib:integer4
              (f2cl-lib:int-add (f2cl-lib:int-mul m lda) m
                                (f2cl-lib:int-mul m nrhs)))))
    (setf ldwork lda))
(setf itau 1) (setf iwork (f2cl-lib:int-add m 1))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (sgelqf m n a lda
     (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-7))
(setf il iwork)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                           work-%offset%)
     ldwork)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwork var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1) zero
     zero
     (f2cl-lib:array-slice work-%data% single-float ((+ il ldwork))
```

```
                          ((1 *)) work-%offset%)
     ldwork)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwork var-6)))
(setf ie (f2cl-lib:int-add il (f2cl-lib:int-mul ldwork m)))
(setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (sgebrd m m
     (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                           work-%offset%)
     ldwork s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwork var-3)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "Q" "L" "T" m nrhs m
     (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                           work-%offset%)
     ldwork
     (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
                           work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwork var-7))
```

```
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (sorgbr "P" m m m
     (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                           work-%offset%)
     ldwork
     (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwork var-5))
  (when var-9 (setf info var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" m m 0 nrhs s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                           work-%offset%)
     ldwork a lda b ldb$
     (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                           work-%offset%)
     info)
  (declare
   (ignore var-0 var-3 var-5 var-6 var-7 var-9 var-11 var-13))
  (setf m var-1)
  (setf m var-2)
  (setf nrhs var-4)
  (setf ldwork var-8)
  (setf lda var-10)
  (setf ldb$ var-12)
  (setf info var-14))
(if (/= info 0)
    (go label70))
(setf thr
        (max
         (* rcond (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
         sfmin))
(if (< rcond zero)
    (setf thr
            (max
```

```
              (* eps (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
              sfmin)))
(setf rank 0)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref s (i) ((1 *))) thr)
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (srscl nrhs
           (f2cl-lib:array-slice s-%data% single-float (i) ((1 *))
                                 s-%offset%)
           (f2cl-lib:array-slice b-%data% single-float (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-2))
        (when var-0 (setf nrhs var-0))
        (when var-3 (setf ldb$ var-3)))
      (setf rank (f2cl-lib:int-add rank 1)))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slaset "F" 1 nrhs zero zero
           (f2cl-lib:array-slice b-%data% single-float (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-1 var-5))
        (when var-2 (setf nrhs var-2))
        (when var-3 (setf zero var-3))
        (when var-4 (setf zero var-4))
        (when var-6 (setf ldb$ var-6)))))
   label30))
(setf iwork ie)
(cond
 ((and
   (>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul ldb$ nrhs) iwork
                         (f2cl-lib:int-sub 1)))
   (> nrhs 1))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sgemm "T" "N" m nrhs m one
       (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                             work-%offset%)
       ldwork b ldb$ zero
       (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                             work-%offset%)
       ldb$)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
```

```
    (when var-2 (setf m var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldwork var-7))
    (when var-9 (setf ldb$ var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldb$ var-12)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "G" m nrhs
       (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                             work-%offset%)
       ldb$ b ldb$)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf nrhs var-2))
    (when var-4 (setf ldb$ var-4))
    (when var-6 (setf ldb$ var-6))))
 ((> nrhs 1)
  (setf chunk
          (the f2cl-lib:integer4 (truncate (+ (- lwork iwork) 1) m)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
                ((> i nrhs) nil)
    (tagbody
      (setf bl
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub nrhs i) 1))
               (the f2cl-lib:integer4 chunk)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (sgemm "T" "N" m bl m one
           (f2cl-lib:array-slice work-%data% single-float (il)
                                 ((1 *)) work-%offset%)
           ldwork
           (f2cl-lib:array-slice b-%data% single-float (1 i)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$ zero
           (f2cl-lib:array-slice work-%data% single-float (iwork)
                                 ((1 *)) work-%offset%)
           m)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf bl var-3))
        (when var-4 (setf m var-4))
        (when var-5 (setf one var-5))
        (when var-7 (setf ldwork var-7))
        (when var-9 (setf ldb$ var-9))
        (when var-10 (setf zero var-10))
```

```
          (when var-12 (setf m var-12)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (slacpy "G" m bl
             (f2cl-lib:array-slice work-%data% single-float (iwork)
                                   ((1 *)) work-%offset%)
             m
             (f2cl-lib:array-slice b-%data% single-float (1 i)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf bl var-2))
          (when var-4 (setf m var-4))
          (when var-6 (setf ldb$ var-6)))
      label40)))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (sgemv "T" m m one
        (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                              work-%offset%)
        ldwork
        (f2cl-lib:array-slice b-%data% single-float (1 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        1 zero
        (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                              work-%offset%)
        1)
     (declare (ignore var-0 var-4 var-6 var-7 var-9 var-10))
     (when var-1 (setf m var-1))
     (when var-2 (setf m var-2))
     (when var-3 (setf one var-3))
     (when var-5 (setf ldwork var-5))
     (when var-8 (setf zero var-8)))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
       (scopy m
        (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                              work-%offset%)
        1
        (f2cl-lib:array-slice b-%data% single-float (1 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        1)
     (declare (ignore var-1 var-2 var-3 var-4))
     (when var-0 (setf m var-0)))))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slaset "F" (f2cl-lib:int-sub n m) nrhs zero zero
      (f2cl-lib:array-slice b-%data% single-float ((+ m 1) 1)
                            ((1 ldb$) (1 *)) b-%offset%)
```

```
      ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldb$ var-6)))
  (setf iwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sormlq "L" "T" n nrhs m a lda
       (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                             work-%offset%)
       b ldb$
       (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf m var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldb$ var-9))
    (when var-12 (setf info var-12))))
 (t (setf ie 1) (setf itauq (f2cl-lib:int-add ie m))
  (setf itaup (f2cl-lib:int-add itauq m))
  (setf iwork (f2cl-lib:int-add itaup m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (sgebrd m n a lda s
       (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
    (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf info var-10))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (sormbr "Q" "L" "T" m nrhs n a lda
       (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
```

```
                              work-%offset%)
        b ldb$
        (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                              work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf nrhs var-4))
    (when var-5 (setf n var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldb$ var-10))
    (when var-13 (setf info var-13)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (sorgbr "P" m n m a lda
       (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                              work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                              work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf info var-9)))
  (setf iwork (f2cl-lib:int-add ie m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (sbdsqr "L" m n 0 nrhs s
       (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                              work-%offset%)
       a lda dum 1 b ldb$
       (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                              work-%offset%)
       info)
    (declare
     (ignore var-0 var-3 var-5 var-6 var-7 var-9 var-10 var-11 var-13))
    (setf m var-1)
    (setf n var-2)
    (setf nrhs var-4)
    (setf lda var-8)
    (setf ldb$ var-12)
    (setf info var-14))
  (if (/= info 0)
      (go label70))
  (setf thr
          (max
```

```
         (* rcond (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
         sfmin))
(if (< rcond zero)
    (setf thr
            (max
             (* eps (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
             sfmin)))
(setf rank 0)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref s (i) ((1 *))) thr)
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (srscl nrhs
           (f2cl-lib:array-slice s-%data% single-float (i) ((1 *))
                                 s-%offset%)
           (f2cl-lib:array-slice b-%data% single-float (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-2))
        (when var-0 (setf nrhs var-0))
        (when var-3 (setf ldb$ var-3)))
      (setf rank (f2cl-lib:int-add rank 1)))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slaset "F" 1 nrhs zero zero
           (f2cl-lib:array-slice b-%data% single-float (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-1 var-5))
        (when var-2 (setf nrhs var-2))
        (when var-3 (setf zero var-3))
        (when var-4 (setf zero var-4))
        (when var-6 (setf ldb$ var-6)))))
   label50))
(cond
 ((and (>= lwork (f2cl-lib:int-mul ldb$ nrhs)) (> nrhs 1))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sgemm "T" "N" n nrhs m one a lda b ldb$ zero work ldb$)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldb$ var-9))
```

```
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldb$ var-12)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "F" n nrhs work ldb$ b ldb$)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf nrhs var-2))
    (when var-4 (setf ldb$ var-4))
    (when var-6 (setf ldb$ var-6))))
 ((> nrhs 1) (setf chunk (the f2cl-lib:integer4 (truncate lwork n)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
                ((> i nrhs) nil)
    (tagbody
      (setf bl
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub nrhs i) 1))
               (the f2cl-lib:integer4 chunk)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (sgemm "T" "N" n bl m one a lda
           (f2cl-lib:array-slice b-%data% single-float (1 i)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$ zero work n)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf n var-2))
        (when var-3 (setf bl var-3))
        (when var-4 (setf m var-4))
        (when var-5 (setf one var-5))
        (when var-7 (setf lda var-7))
        (when var-9 (setf ldb$ var-9))
        (when var-10 (setf zero var-10))
        (when var-12 (setf n var-12)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slacpy "F" n bl work n
           (f2cl-lib:array-slice b-%data% single-float (1 i)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf bl var-2))
        (when var-4 (setf n var-4))
        (when var-6 (setf ldb$ var-6)))
    label60)))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
```

```
        (sgemv "T" m n one a lda b 1 zero work 1)
      (declare (ignore var-0 var-4 var-6 var-7 var-9 var-10))
      (when var-1 (setf m var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf one var-3))
      (when var-5 (setf lda var-5))
      (when var-8 (setf zero var-8)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy n work 1 b 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))))))
(cond
 ((= iascl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 anrm smlnum n nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf n var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 smlnum anrm minmn 1 s minmn info)
    (declare (ignore var-0 var-1 var-2 var-6 var-7))
    (when var-3 (setf smlnum var-3))
    (when var-4 (setf anrm var-4))
    (when var-5 (setf minmn var-5))
    (when var-8 (setf minmn var-8))
    (when var-9 (setf info var-9))))
 ((= iascl 2)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 anrm bignum n nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf n var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 bignum anrm minmn 1 s minmn info)
    (declare (ignore var-0 var-1 var-2 var-6 var-7))
    (when var-3 (setf bignum var-3))
    (when var-4 (setf anrm var-4))
    (when var-5 (setf minmn var-5))
```

```
            (when var-8 (setf minmn var-8))
            (when var-9 (setf info var-9)))))
        (cond
         ((= ibscl 1)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (slascl "G" 0 0 smlnum bnrm n nrhs b ldb$ info)
            (declare (ignore var-0 var-1 var-2 var-7))
            (when var-3 (setf smlnum var-3))
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf n var-5))
            (when var-6 (setf nrhs var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf info var-9))))
         ((= ibscl 2)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (slascl "G" 0 0 bignum bnrm n nrhs b ldb$ info)
            (declare (ignore var-0 var-1 var-2 var-7))
            (when var-3 (setf bignum var-3))
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf n var-5))
            (when var-6 (setf nrhs var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf info var-9)))))
       label70
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 maxwrk) 'single-float))
        (go end_label)
       end_label
        (return
         (values m n nrhs nil lda nil ldb$ nil nil rank nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgelss
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (single-float)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
```

```
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::m
  fortran-to-lisp::n
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$ nil nil
  fortran-to-lisp::rank nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::sgeqrf
  fortran-to-lisp::sgelqf
  fortran-to-lisp::sbdsqr
  fortran-to-lisp::sgebrd
  fortran-to-lisp::ilaenv))))
```

———-

21.4 sgelsx - DEPRECATED use sgelsy

21.5 sgelsy

21.5.1 Axiom unit tests

— **sgelsy.input** —

```
)set break resume
)sys rm -f sgelsy.output
)spool sgelsy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

21.5.2 Axiom help page

— **sgelsy.help** —

```
====================================================================
sgelsy examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgelsy (M, N, NRHS, A, LDA, B, LDB, JPVT, RCOND, RANK, WORK,
           LWORK, INFO)
            SGELSY solves overdetermined or underdetermined systems for GE
           matrices

Function/Subroutine Documentation
   subroutine sgelsy (integer M, integer N, integer NRHS, real, dimension(
       lda, * ) A, integer LDA, real, dimension( ldb, * ) B, integer LDB,
       integer, dimension( * ) JPVT, real RCOND, integer RANK, real,
       dimension( * ) WORK, integer LWORK, integer INFO)
        SGELSY solves overdetermined or underdetermined systems for GE
       matrices

Purpose:
   SGELSY computes the minimum-norm solution to a real linear least
   squares problem:
                minimize || A * X - B ||
   using a complete orthogonal factorization of A.  A is an M-by-N
   matrix which may be rank-deficient.

   Several right hand side vectors b and solution vectors x can be
   handled in a single call; they are stored as the columns of the
   M-by-NRHS right hand side matrix B and the N-by-NRHS solution
   matrix X.

   The routine first computes a QR factorization with column pivoting:
                A * P = Q * [ R11 R12 ]
                            [  0  R22 ]
   with R11 defined as the largest leading submatrix whose estimated
   condition number is less than 1/RCOND.  The order of R11, RANK,
   is the effective rank of A.

   Then, R22 is considered to be negligible, and R12 is annihilated
   by orthogonal transformations from the right, arriving at the
   complete orthogonal factorization:
               A * P = Q * [ T11 0 ] * Z
                           [  0  0 ]
   The minimum-norm solution is then
               X = P * Z**T [ inv(T11)*Q1**T*B ]
                            [        0         ]
```

```
  where Q1 consists of the first RANK columns of Q.

  This routine is basically identical to the original xGELSX except
  three differences:
  o The call to the subroutine xGEQPF has been substituted by the
    the call to the subroutine xGEQP3. This subroutine is a Blas-3
    version of the QR factorization with column pivoting.
  o Matrix B (the right hand side) is updated with Blas-3.
  o The permutation of matrix B (the right hand side) is faster and
    more simple.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of
   columns of matrices B and X. NRHS >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, A has been overwritten by details of its
   complete orthogonal factorization.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 B
   B is REAL array, dimension (LDB,NRHS)
   On entry, the M-by-NRHS right hand side matrix B.
   On exit, the N-by-NRHS solution matrix X.

 LDB
   LDB is INTEGER
   The leading dimension of the array B. LDB >= max(1,M,N).

 JPVT
   JPVT is INTEGER array, dimension (N)
   On entry, if JPVT(i) .ne. 0, the i-th column of A is permuted
   to the front of AP, otherwise column i is a free column.
   On exit, if JPVT(i) = k, then the i-th column of AP
```

```
  was the k-th column of A.

RCOND
  RCOND is REAL
  RCOND is used to determine the effective rank of A, which
  is defined as the order of the largest leading triangular
  submatrix R11 in the QR factorization with pivoting of A,
  whose estimated condition number < 1/RCOND.

RANK
  RANK is INTEGER
  The effective rank of A, i.e., the order of the submatrix
  R11.  This is the same as the order of the submatrix T11
  in the complete orthogonal factorization of A.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  The unblocked strategy requires that:
  LWORK >= MAX( MN+3*N+1, 2*MN+NRHS ),
  where MN = min( M, N ).
  The block algorithm requires that:
  LWORK >= MAX( MN+2*N+NB*(N+1), 2*MN+NB*NRHS ),
  where NB is an upper bound on the blocksize returned
  by ILAENV for the routines SGEQP3, STZRZF, STZRQF, SORMQR,
  and SORMRZ.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: If INFO = -i, the i-th argument had an illegal value.
```

———-

21.5.3 fortran code

— sgelsy.f —

```
*  =====================================================================
      SUBROUTINE SGELSY( M, N, NRHS, A, LDA, B, LDB, JPVT, RCOND, RANK,
     $                   WORK, LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDB, LWORK, M, N, NRHS, RANK
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            JPVT( * )
      REAL               A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            IMAX, IMIN
      PARAMETER          ( IMAX = 1, IMIN = 2 )
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IASCL, IBSCL, ISMAX, ISMIN, J, LWKMIN,
     $                   LWKOPT, MN, NB, NB1, NB2, NB3, NB4
      REAL               ANRM, BIGNUM, BNRM, C1, C2, S1, S2, SMAX,
     $                   SMAXPR, SMIN, SMINPR, SMLNUM, WSIZE
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      REAL               SLAMCH, SLANGE
      EXTERNAL           ILAENV, SLAMCH, SLANGE
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SGEQP3, SLABAD, SLAIC1, SLASCL, SLASET,
     $                   SORMQR, SORMRZ, STRSM, STZRZF, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      MN = MIN( M, N )
      ISMIN = MN + 1
      ISMAX = 2*MN + 1
```

```
*
*     Test the input arguments.
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, M, N ) ) THEN
         INFO = -7
      END IF
*
*     Figure out optimal block size
*
      IF( INFO.EQ.0 ) THEN
         IF( MN.EQ.0 .OR. NRHS.EQ.0 ) THEN
            LWKMIN = 1
            LWKOPT = 1
         ELSE
            NB1 = ILAENV( 1, 'SGEQRF', ' ', M, N, -1, -1 )
            NB2 = ILAENV( 1, 'SGERQF', ' ', M, N, -1, -1 )
            NB3 = ILAENV( 1, 'SORMQR', ' ', M, N, NRHS, -1 )
            NB4 = ILAENV( 1, 'SORMRQ', ' ', M, N, NRHS, -1 )
            NB = MAX( NB1, NB2, NB3, NB4 )
            LWKMIN = MN + MAX( 2*MN, N + 1, MN + NRHS )
            LWKOPT = MAX( LWKMIN,
     $                    MN + 2*N + NB*( N + 1 ), 2*MN + NB*NRHS )
         END IF
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.LWKMIN .AND. .NOT.LQUERY ) THEN
            INFO = -12
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGELSY', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( MN.EQ.0 .OR. NRHS.EQ.0 ) THEN
```

```
         RANK = 0
         RETURN
      END IF
*
*     Get machine parameters
*
      SMLNUM = SLAMCH( 'S' ) / SLAMCH( 'P' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
*
*     Scale A, B if max entries outside range [SMLNUM,BIGNUM]
*
      ANRM = SLANGE( 'M', M, N, A, LDA, WORK )
      IASCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL SLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
         IASCL = 1
      ELSE IF( ANRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL SLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
         IASCL = 2
      ELSE IF( ANRM.EQ.ZERO ) THEN
*
*        Matrix all zero. Return zero solution.
*
         CALL SLASET( 'F', MAX( M, N ), NRHS, ZERO, ZERO, B, LDB )
         RANK = 0
         GO TO 70
      END IF
*
      BNRM = SLANGE( 'M', M, NRHS, B, LDB, WORK )
      IBSCL = 0
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL SLASCL( 'G', 0, 0, BNRM, SMLNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 1
      ELSE IF( BNRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL SLASCL( 'G', 0, 0, BNRM, BIGNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 2
      END IF
```

```
*
*     Compute QR factorization with column pivoting of A:
*        A * P = Q * R
*
      CALL SGEQP3( M, N, A, LDA, JPVT, WORK( 1 ), WORK( MN+1 ),
     $             LWORK-MN, INFO )
      WSIZE = MN + WORK( MN+1 )
*
*     workspace: MN+2*N+NB*(N+1).
*     Details of Householder rotations stored in WORK(1:MN).
*
*     Determine RANK using incremental condition estimation
*
      WORK( ISMIN ) = ONE
      WORK( ISMAX ) = ONE
      SMAX = ABS( A( 1, 1 ) )
      SMIN = SMAX
      IF( ABS( A( 1, 1 ) ).EQ.ZERO ) THEN
         RANK = 0
         CALL SLASET( 'F', MAX( M, N ), NRHS, ZERO, ZERO, B, LDB )
         GO TO 70
      ELSE
         RANK = 1
      END IF
*
   10 CONTINUE
      IF( RANK.LT.MN ) THEN
         I = RANK + 1
         CALL SLAIC1( IMIN, RANK, WORK( ISMIN ), SMIN, A( 1, I ),
     $                A( I, I ), SMINPR, S1, C1 )
         CALL SLAIC1( IMAX, RANK, WORK( ISMAX ), SMAX, A( 1, I ),
     $                A( I, I ), SMAXPR, S2, C2 )
*
         IF( SMAXPR*RCOND.LE.SMINPR ) THEN
            DO 20 I = 1, RANK
               WORK( ISMIN+I-1 ) = S1*WORK( ISMIN+I-1 )
               WORK( ISMAX+I-1 ) = S2*WORK( ISMAX+I-1 )
   20       CONTINUE
            WORK( ISMIN+RANK ) = C1
            WORK( ISMAX+RANK ) = C2
            SMIN = SMINPR
            SMAX = SMAXPR
            RANK = RANK + 1
            GO TO 10
         END IF
      END IF
*
*     workspace: 3*MN.
*
*     Logically partition R = [ R11 R12 ]
```

```
*                              [  0  R22 ]
*     where R11 = R(1:RANK,1:RANK)
*
*     [R11,R12] = [ T11, 0 ] * Y
*
      IF( RANK.LT.N )
     $   CALL STZRZF( RANK, N, A, LDA, WORK( MN+1 ), WORK( 2*MN+1 ),
     $                LWORK-2*MN, INFO )
*
*     workspace: 2*MN.
*     Details of Householder rotations stored in WORK(MN+1:2*MN)
*
*     B(1:M,1:NRHS) := Q**T * B(1:M,1:NRHS)
*
      CALL SORMQR( 'Left', 'Transpose', M, NRHS, MN, A, LDA, WORK( 1 ),
     $             B, LDB, WORK( 2*MN+1 ), LWORK-2*MN, INFO )
      WSIZE = MAX( WSIZE, 2*MN+WORK( 2*MN+1 ) )
*
*     workspace: 2*MN+NB*NRHS.
*
*     B(1:RANK,1:NRHS) := inv(T11) * B(1:RANK,1:NRHS)
*
      CALL STRSM( 'Left', 'Upper', 'No transpose', 'Non-unit', RANK,
     $            NRHS, ONE, A, LDA, B, LDB )
*
      DO 40 J = 1, NRHS
         DO 30 I = RANK + 1, N
            B( I, J ) = ZERO
   30    CONTINUE
   40 CONTINUE
*
*     B(1:N,1:NRHS) := Y**T * B(1:N,1:NRHS)
*
      IF( RANK.LT.N ) THEN
         CALL SORMRZ( 'Left', 'Transpose', N, NRHS, RANK, N-RANK, A,
     $                LDA, WORK( MN+1 ), B, LDB, WORK( 2*MN+1 ),
     $                LWORK-2*MN, INFO )
      END IF
*
*     workspace: 2*MN+NRHS.
*
*     B(1:N,1:NRHS) := P * B(1:N,1:NRHS)
*
      DO 60 J = 1, NRHS
         DO 50 I = 1, N
            WORK( JPVT( I ) ) = B( I, J )
   50    CONTINUE
         CALL SCOPY( N, WORK( 1 ), 1, B( 1, J ), 1 )
   60 CONTINUE
*
```

```
*     workspace: N.
*
*     Undo scaling
*
      IF( IASCL.EQ.1 ) THEN
         CALL SLASCL( 'G', 0, 0, ANRM, SMLNUM, N, NRHS, B, LDB, INFO )
         CALL SLASCL( 'U', 0, 0, SMLNUM, ANRM, RANK, RANK, A, LDA,
     $                INFO )
      ELSE IF( IASCL.EQ.2 ) THEN
         CALL SLASCL( 'G', 0, 0, ANRM, BIGNUM, N, NRHS, B, LDB, INFO )
         CALL SLASCL( 'U', 0, 0, BIGNUM, ANRM, RANK, RANK, A, LDA,
     $                INFO )
      END IF
      IF( IBSCL.EQ.1 ) THEN
         CALL SLASCL( 'G', 0, 0, SMLNUM, BNRM, N, NRHS, B, LDB, INFO )
      ELSE IF( IBSCL.EQ.2 ) THEN
         CALL SLASCL( 'G', 0, 0, BIGNUM, BNRM, N, NRHS, B, LDB, INFO )
      END IF
*
   70 CONTINUE
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of SGELSY
*
      END
```

———-

21.5.4 lisp code

— sgelsy.lisp —

```
(let* ((imax 1) (imin 2) (zero 0.0) (one 1.0))
  (declare (type (f2cl-lib:integer4 1 1) imax)
           (type (f2cl-lib:integer4 2 2) imin)
           (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable imax imin zero one))
  (defun sgelsy (m n nrhs a lda b ldb$ jpvt rcond rank work lwork info)
    (declare (type (single-float) rcond)
             (type (array f2cl-lib:integer4 (*)) jpvt)
             (type (array single-float (*)) work b a)
             (type (f2cl-lib:integer4) info lwork rank ldb$ lda nrhs n m))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%)
```

```
    (work single-float work-%data% work-%offset%)
    (jpvt f2cl-lib:integer4 jpvt-%data% jpvt-%offset%))
  (prog ((anrm 0.0) (bignum 0.0) (bnrm 0.0) (c1 0.0) (c2 0.0) (s1 0.0)
         (s2 0.0) (smax 0.0) (smaxpr 0.0) (smin 0.0) (sminpr 0.0)
         (smlnum 0.0) (wsize 0.0) (i 0) (iascl 0) (ibscl 0) (ismax 0)
         (ismin 0) (j 0) (lwkmin 0) (lwkopt 0) (mn 0) (nb 0) (nb1 0)
         (nb2 0) (nb3 0) (nb4 0) (lquery nil))
    (declare
     (type (single-float) anrm bignum bnrm c1 c2 s1 s2 smax smaxpr smin
      sminpr smlnum wsize)
     (type (f2cl-lib:integer4) i iascl ibscl ismax ismin j lwkmin lwkopt mn
      nb nb1 nb2 nb3 nb4)
     (type f2cl-lib:logical lquery))
    (setf mn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
    (setf ismin (f2cl-lib:int-add mn 1))
    (setf ismax (f2cl-lib:int-add (f2cl-lib:int-mul 2 mn) 1))
    (setf info 0)
    (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
    (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
          ((< nrhs 0) (setf info -3))
          ((< lda
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
           (setf info -5))
          ((< ldb$
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)
                   (the f2cl-lib:integer4 n)))
           (setf info -7)))
    (cond
     ((= info 0)
      (cond ((or (= mn 0) (= nrhs 0)) (setf lwkmin 1) (setf lwkopt 1))
            (t
             (setf nb1
                     (multiple-value-bind
                         (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                          var-6)
                         (ilaenv 1 "SGEQRF" " " m n -1 -1)
                       (declare (ignore var-0 var-1 var-2 var-5 var-6))
                       (setf m var-3)
                       (setf n var-4)
                       ret-val))
             (setf nb2
                     (multiple-value-bind
                         (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                          var-6)
                         (ilaenv 1 "SGERQF" " " m n -1 -1)
                       (declare (ignore var-0 var-1 var-2 var-5 var-6))
                       (setf m var-3)
                       (setf n var-4)
                       ret-val))
             (setf nb3
```

```
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                     var-6)
                    (ilaenv 1 "SORMQR" " " m n nrhs -1)
                  (declare (ignore var-0 var-1 var-2 var-6))
                  (setf m var-3)
                  (setf n var-4)
                  (setf nrhs var-5)
                  ret-val))
        (setf nb4
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                     var-6)
                    (ilaenv 1 "SORMRQ" " " m n nrhs -1)
                  (declare (ignore var-0 var-1 var-2 var-6))
                  (setf m var-3)
                  (setf n var-4)
                  (setf nrhs var-5)
                  ret-val))
        (setf nb
                (max (the f2cl-lib:integer4 nb1)
                     (the f2cl-lib:integer4 nb2)
                     (the f2cl-lib:integer4 nb3)
                     (the f2cl-lib:integer4 nb4)))
        (setf lwkmin
                (f2cl-lib:int-add mn
                                  (max
                                   (the f2cl-lib:integer4
                                        (f2cl-lib:int-mul 2 mn))
                                   (the f2cl-lib:integer4
                                        (f2cl-lib:int-add n 1))
                                   (the f2cl-lib:integer4
                                        (f2cl-lib:int-add mn nrhs)))))
        (setf lwkopt
                (max (the f2cl-lib:integer4 lwkmin)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add mn (f2cl-lib:int-mul 2 n)
                                            (f2cl-lib:int-mul nb
                                                              (f2cl-lib:int-add
                                                               n 1))))
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 2 mn)
                                            (f2cl-lib:int-mul nb
                                                              nrhs)))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
  (cond ((and (< lwork lwkmin) (not lquery)) (setf info -12)))))
(cond
 ((/= info 0) (xerbla "SGELSY" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
```

```
(cond ((or (= mn 0) (= nrhs 0)) (setf rank 0) (go end_label)))
(setf smlnum (/ (slamch "S") (slamch "P")))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (slabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "M" m n a lda work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iascl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 anrm smlnum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 1))
 ((> anrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 anrm bignum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 2))
 ((= anrm zero)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "F"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       zero zero b ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
```

```
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldb$ var-6)))
  (setf rank 0) (go label70)))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "M" m nrhs b ldb$ work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ibscl 0)
(cond
 ((and (> bnrm zero) (< bnrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 bnrm smlnum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 1))
 ((> bnrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 bnrm bignum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 2)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sgeqp3 m n a lda jpvt
     (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float ((+ mn 1)) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-sub lwork mn) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7))
  (setf m var-0)
  (setf n var-1)
```

```
    (setf lda var-3)
    (setf info var-8))
  (setf wsize
          (+ mn
             (f2cl-lib:fref work-%data% ((f2cl-lib:int-add mn 1)) ((1 *))
                            work-%offset%)))
  (setf (f2cl-lib:fref work-%data% (ismin) ((1 *)) work-%offset%) one)
  (setf (f2cl-lib:fref work-%data% (ismax) ((1 *)) work-%offset%) one)
  (setf smax
          (abs
           (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)))
  (setf smin smax)
  (cond
   ((= (abs (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))) zero) (setf rank 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "F"
         (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
         zero zero b ldb$)
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf nrhs var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldb$ var-6)))
    (go label70))
   (t (setf rank 1)))
 label10
  (cond
   ((< rank mn) (setf i (f2cl-lib:int-add rank 1))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (slaic1 imin rank
         (f2cl-lib:array-slice work-%data% single-float (ismin) ((1 *))
                               work-%offset%)
         smin
         (f2cl-lib:array-slice a-%data% single-float (1 i)
                               ((1 lda) (1 *)) a-%offset%)
         (f2cl-lib:array-slice a-%data% single-float (i i)
                               ((1 lda) (1 *)) a-%offset%)
         sminpr s1 c1)
      (declare (ignore var-2 var-4 var-5))
      (when var-0 (setf imin var-0))
      (when var-1 (setf rank var-1))
      (when var-3 (setf smin var-3))
      (when var-6 (setf sminpr var-6))
      (when var-7 (setf s1 var-7))
      (when var-8 (setf c1 var-8)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (slaic1 imax rank
         (f2cl-lib:array-slice work-%data% single-float (ismax) ((1 *))
```

```
                                work-%offset%)
            smax
            (f2cl-lib:array-slice a-%data% single-float (1 i)
                                ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:array-slice a-%data% single-float (i i)
                                ((1 lda) (1 *)) a-%offset%)
            smaxpr s2 c2)
        (declare (ignore var-2 var-4 var-5))
        (when var-0 (setf imax var-0))
        (when var-1 (setf rank var-1))
        (when var-3 (setf smax var-3))
        (when var-6 (setf smaxpr var-6))
        (when var-7 (setf s2 var-7))
        (when var-8 (setf c2 var-8)))
      (cond
       ((<= (* smaxpr rcond) sminpr)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i rank) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-add ismin i) 1))
                                ((1 *)) work-%offset%)
                  (* s1
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-sub
                                      (f2cl-lib:int-add ismin i) 1))
                                    ((1 *)) work-%offset%)))
            (setf (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-add ismax i) 1))
                                ((1 *)) work-%offset%)
                  (* s2
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-sub
                                      (f2cl-lib:int-add ismax i) 1))
                                    ((1 *)) work-%offset%)))
            label20))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ismin rank))
                             ((1 *)) work-%offset%)
              c1)
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ismax rank))
                             ((1 *)) work-%offset%)
              c2)
        (setf smin sminpr) (setf smax smaxpr)
        (setf rank (f2cl-lib:int-add rank 1)) (go label10)))))
    (if (< rank n)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (stzrzf rank n a lda
```

```
          (f2cl-lib:array-slice work-%data% single-float ((+ mn 1))
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% single-float
                                ((+ (f2cl-lib:int-mul 2 mn) 1)) ((1 *))
                                work-%offset%)
          (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 mn)) info)
      (declare (ignore var-2 var-4 var-5 var-6))
      (when var-0 (setf rank var-0))
      (when var-1 (setf n var-1))
      (when var-3 (setf lda var-3))
      (when var-7 (setf info var-7))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (sormqr "Left" "Transpose" m nrhs mn a lda
     (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                           work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul 2 mn) 1)) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 mn)) info)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf nrhs var-3))
  (when var-4 (setf mn var-4))
  (when var-6 (setf lda var-6))
  (when var-9 (setf ldb$ var-9))
  (when var-12 (setf info var-12)))
(setf wsize
        (max wsize
             (+ (f2cl-lib:int-mul 2 mn)
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add
                                 (f2cl-lib:int-mul 2 mn) 1))
                               ((1 *)) work-%offset%))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (strsm "Left" "Upper" "No transpose" "Non-unit" rank nrhs one a lda
     b ldb$)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf rank var-4))
  (when var-5 (setf nrhs var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf ldb$ var-10)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
```

```
      (f2cl-lib:fdo (i (f2cl-lib:int-add rank 1) (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                zero)
        label30))
    label40))
  (cond
   ((< rank n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (sormrz "Left" "Transpose" n nrhs rank (f2cl-lib:int-sub n rank)
         a lda
         (f2cl-lib:array-slice work-%data% single-float ((+ mn 1))
                               ((1 *)) work-%offset%)
         b ldb$
         (f2cl-lib:array-slice work-%data% single-float
                               ((+ (f2cl-lib:int-mul 2 mn) 1)) ((1 *))
                               work-%offset%)
         (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 mn)) info)
      (declare
       (ignore var-0 var-1 var-5 var-6 var-8 var-9 var-11 var-12))
      (when var-2 (setf n var-2))
      (when var-3 (setf nrhs var-3))
      (when var-4 (setf rank var-4))
      (when var-7 (setf lda var-7))
      (when var-10 (setf ldb$ var-10))
      (when var-13 (setf info var-13)))))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data%
                               ((f2cl-lib:fref jpvt (i) ((1 *)))) ((1 *))
                               work-%offset%)
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%))
        label50))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (scopy n
           (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                                 work-%offset%)
           1
           (f2cl-lib:array-slice b-%data% single-float (1 j)
                                 ((1 ldb$) (1 *)) b-%offset%)
           1)
```

```
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
   label60))
(cond
 ((= iascl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 anrm smlnum n nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf n var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "U" 0 0 smlnum anrm rank rank a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf smlnum var-3))
    (when var-4 (setf anrm var-4))
    (when var-5 (setf rank var-5))
    (when var-6 (setf rank var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9))))
 ((= iascl 2)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 anrm bignum n nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf n var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "U" 0 0 bignum anrm rank rank a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bignum var-3))
    (when var-4 (setf anrm var-4))
    (when var-5 (setf rank var-5))
    (when var-6 (setf rank var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))))
(cond
 ((= ibscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
```

```
              (slascl "G" 0 0 smlnum bnrm n nrhs b ldb$ info)
            (declare (ignore var-0 var-1 var-2 var-7))
            (when var-3 (setf smlnum var-3))
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf n var-5))
            (when var-6 (setf nrhs var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf info var-9))))
          ((= ibscl 2)
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
               (slascl "G" 0 0 bignum bnrm n nrhs b ldb$ info)
            (declare (ignore var-0 var-1 var-2 var-7))
            (when var-3 (setf bignum var-3))
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf n var-5))
            (when var-6 (setf nrhs var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf info var-9)))))
        label70
         (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                 (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
         (go end_label)
        end_label
         (return
          (values m n nrhs nil lda nil ldb$ nil nil rank nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgelsy
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (single-float)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n
```

```
                                                  fortran-to-lisp::nrhs nil
                                                  fortran-to-lisp::lda nil
                                                  fortran-to-lisp::ldb$ nil nil
                                                  fortran-to-lisp::rank nil nil
                                                  fortran-to-lisp::info)
                                                :calls
                                                '(fortran-to-lisp::sgeqp3
                                                  fortran-to-lisp::ilaenv))))
```

———-

21.6 sgesv

21.6.1 Axiom unit tests

— **sgesv.input** —

```
)set break resume
)sys rm -f sgesv.output
)spool sgesv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

21.6.2 Axiom help page

— **sgesv.help** —

```
====================================================================
sgesv examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
```

```
   Functions/Subroutines
       subroutine sgesv (N, NRHS, A, LDA, IPIV, B, LDB, INFO)
            SGESV computes the solution to system of linear equations A * X =
           B for GE matrices (simple driver)

Function/Subroutine Documentation
   subroutine sgesv (integer N, integer NRHS, real, dimension( lda, * ) A,
       integer LDA, integer, dimension( * ) IPIV, real, dimension( ldb, * ) B,
       integer LDB, integer INFO)
        SGESV computes the solution to system of linear equations A * X = B
       for GE matrices (simple driver)

Purpose:
   SGESV computes the solution to a real system of linear equations
               A * X = B,
   where A is an N-by-N matrix and X and B are N-by-NRHS matrices.

   The LU decomposition with partial pivoting and row interchanges is
   used to factor A as
               A = P * L * U,
   where P is a permutation matrix, L is unit lower triangular, and U is
   upper triangular.  The factored form of A is then used to solve the
   system of equations A * X = B.

Parameters:

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the N-by-N coefficient matrix A.
   On exit, the factors L and U from the factorization
   A = P*L*U; the unit diagonal elements of L are not stored.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   The pivot indices that define the permutation matrix P;
   row i of the matrix was interchanged with row IPIV(i).
```

```
B
  B is REAL array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS matrix of right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, U(i,i) is exactly zero.  The factorization
        has been completed, but the factor U is exactly
        singular, so the solution could not be computed.
```

21.6.3 fortran code

— sgesv.f —

```
*  =====================================================================
      SUBROUTINE SGESV( N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. External Subroutines ..
      EXTERNAL           SGETRF, SGETRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
```

```
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGESV ', -INFO )
         RETURN
      END IF
*
*     Compute the LU factorization of A.
*
      CALL SGETRF( N, N, A, LDA, IPIV, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
         CALL SGETRS( 'No transpose', N, NRHS, A, LDA, IPIV, B, LDB,
     $                INFO )
      END IF
      RETURN
*
*     End of SGESV
*
      END
```

———-

21.6.4 lisp code

— **sgesv.lisp** —

```
(defun sgesv (n nrhs a lda ipiv b ldb$ info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array single-float (*)) b a)
           (type (f2cl-lib:integer4) info ldb$ lda nrhs n))
  (f2cl-lib:with-multi-array-data
      ((a single-float a-%data% a-%offset%)
```

```
       (b single-float b-%data% b-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ()
      (declare)
      (setf info 0)
      (cond ((< n 0) (setf info -1)) ((< nrhs 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -4))
            ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -7)))
      (cond
       ((/= info 0) (xerbla "SGESV " (f2cl-lib:int-sub info)) (go end_label)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (sgetrf n n a lda ipiv info)
        (declare (ignore var-2 var-4))
        (setf n var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf info var-5))
      (cond
       ((= info 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (sgetrs "No transpose" n nrhs a lda ipiv b ldb$ info)
          (declare (ignore var-0 var-3 var-5 var-6))
          (setf n var-1)
          (setf nrhs var-2)
          (setf lda var-4)
          (setf ldb$ var-7)
          (setf info var-8))))
      (go end_label)
     end_label
      (return (values n nrhs nil lda nil nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgesv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n
```

```
                                        fortran-to-lisp::nrhs nil
                                        fortran-to-lisp::lda nil nil
                                        fortran-to-lisp::ldb$
                                        fortran-to-lisp::info)
                                      :calls
                                      '(fortran-to-lisp::sgetrs
                                        fortran-to-lisp::sgetrf))))
```

———-

21.7 sgesvx

21.7.1 Axiom unit tests

— **sgesvx.input** —

```
)set break resume
)sys rm -f sgesvx.output
)spool sgesvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

21.7.2 Axiom help page

— **sgesvx.help** —

```
=================================================================
sgesvx examples
=================================================================

=================================================================
Man Page Details
=================================================================

SYNOPSIS
   Functions/Subroutines
```

```
      subroutine sgesvx (FACT, TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV, EQUED,
          R, C, B, LDB, X, LDX, RCOND, FERR, BERR, WORK, IWORK, INFO)
           SGESVX computes the solution to system of linear equations A * X =
          B for GE matrices

Function/Subroutine Documentation
   subroutine sgesvx (character FACT, character TRANS, integer N, integer
       NRHS, real, dimension( lda, * ) A, integer LDA, real, dimension( ldaf,
       * ) AF, integer LDAF, integer, dimension( * ) IPIV, character EQUED,
       real, dimension( * ) R, real, dimension( * ) C, real, dimension( ldb, *
       ) B, integer LDB, real, dimension( ldx, * ) X, integer LDX, real RCOND,
       real, dimension( * ) FERR, real, dimension( * ) BERR, real, dimension(
       * ) WORK, integer, dimension( * ) IWORK, integer INFO)
        SGESVX computes the solution to system of linear equations A * X = B
       for GE matrices

Purpose:
   SGESVX uses the LU factorization to compute the solution to a real
   system of linear equations
               A * X = B,
   where A is an N-by-N matrix and X and B are N-by-NRHS matrices.

   Error bounds on the solution and a condition estimate are also provided.

Description:
   The following steps are performed:

   1. If FACT = 'E', real scaling factors are computed to equilibrate
      the system:
      TRANS = 'N':  diag(R)*A*diag(C)     *inv(diag(C))*X = diag(R)*B
      TRANS = 'T': (diag(R)*A*diag(C))**T *inv(diag(R))*X = diag(C)*B
      TRANS = 'C': (diag(R)*A*diag(C))**H *inv(diag(R))*X = diag(C)*B
      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(R)*A*diag(C) and B by diag(R)*B (if TRANS='N')
      or diag(C)*B (if TRANS = 'T' or 'C').

   2. If FACT = 'N' or 'E', the LU decomposition is used to factor the
      matrix A (after equilibration if FACT = 'E') as
                  A = P * L * U,
      where P is a permutation matrix, L is a unit lower triangular
      matrix, and U is upper triangular.

   3. If some U(i,i)=0, so that U is exactly singular, then the routine
      returns with INFO = i. Otherwise, the factored form of A is used
      to estimate the condition number of the matrix A.  If the
      reciprocal of the condition number is less than machine precision,
      INFO = N+1 is returned as a warning, but the routine still goes on
      to solve for X and compute error bounds as described below.
```

```
   4. The system of equations is solved for X using the factored form of A.

   5. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(C) (if TRANS = 'N') or diag(R) (if TRANS = 'T' or 'C') so
      that it solves the original system before equilibration.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AF and IPIV contain the factored form of A.
           If EQUED is not 'N', the matrix A has been
           equilibrated with scaling factors given by R and C.
           A, AF, and IPIV are not modified.
   = 'N':  The matrix A will be copied to AF and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
           copied to AF and factored.

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Transpose)

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the N-by-N matrix A.  If FACT = 'F' and EQUED is
   not 'N', then A must have been equilibrated by the scaling
   factors in R and/or C.  A is not modified if FACT = 'F' or
   'N', or if FACT = 'E' and EQUED = 'N' on exit.

   On exit, if EQUED .ne. 'N', A is scaled as follows:
   EQUED = 'R':  A := diag(R) * A
```

```
  EQUED = 'C':  A := A * diag(C)
  EQUED = 'B':  A := diag(R) * A * diag(C).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is REAL array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the factors L and U from the factorization
  A = P*L*U as computed by SGETRF.  If EQUED .ne. 'N', then
  AF is the factored form of the equilibrated matrix A.

  If FACT = 'N', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the equilibrated matrix A (see the description of A for
  the form of the equilibrated matrix).

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains the pivot indices from the factorization A = P*L*U
  as computed by SGETRF; row i of the matrix was interchanged
  with row IPIV(i).

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the equilibrated matrix A.

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
  = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
  = 'B':  Both row and column equilibration, i.e., A has been
          replaced by diag(R) * A * diag(C).
```

```
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

R
  R is REAL array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
  EQUED = 'R' or 'B', each element of R must be positive.

C
  C is REAL array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.

B
  B is REAL array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if TRANS = 'N' and EQUED = 'R' or 'B', B is overwritten by
  diag(R)*B;
  if TRANS = 'T' or 'C' and EQUED = 'C' or 'B', B is
  overwritten by diag(C)*B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is REAL array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X
  to the original system of equations.  Note that A and B are
  modified on exit if EQUED .ne. 'N', and the solution to the
  equilibrated system is inv(diag(C))*X if TRANS = 'N' and
  EQUED = 'C' or 'B', or inv(diag(R))*X if TRANS = 'T' or 'C'
  and EQUED = 'R' or 'B'.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  The estimate of the reciprocal condition number of the matrix
  A after equilibration (if done).  If RCOND is less than the
```

```
  machine precision (in particular, if RCOND = 0), the matrix
  is singular to working precision.  This condition is
  indicated by a return code of INFO > 0.

FERR
  FERR is REAL array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is REAL array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is REAL array, dimension (4*N)
  On exit, WORK(1) contains the reciprocal pivot growth
  factor norm(A)/norm(U). The "max absolute element" norm is
  used. If WORK(1) is much less than 1, then the stability
  of the LU factorization of the (equilibrated) matrix A
  could be poor. This also means that the solution X, condition
  estimator RCOND, and forward error bound FERR could be
  unreliable. If factorization fails with 0<INFO<=N, then
  WORK(1) contains the reciprocal pivot growth factor for the
  leading INFO columns of A.

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, and i is
    <= N:  U(i,i) is exactly zero.  The factorization has
           been completed, but the factor U is exactly
           singular, so the solution and error bounds
           could not be computed. RCOND = 0 is returned.
    = N+1: U is nonsingular, but RCOND is less than machine
           precision, meaning that the matrix is singular
           to working precision.  Nevertheless, the
           solution and error bounds are computed because
           there are a number of situations where the
```

```
computed solution can be more accurate than the
value of RCOND would suggest.
```

———-

21.7.3 fortran code

— **sgesvx.f** —

```
*  =====================================================================
      SUBROUTINE SGESVX( FACT, TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $                   EQUED, R, C, B, LDB, X, LDX, RCOND, FERR, BERR,
     $                   WORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, TRANS
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      REAL               A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   BERR( * ), C( * ), FERR( * ), R( * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            COLEQU, EQUIL, NOFACT, NOTRAN, ROWEQU
      CHARACTER          NORM
      INTEGER            I, INFEQU, J
      REAL               AMAX, ANORM, BIGNUM, COLCND, RCMAX, RCMIN,
     $                   ROWCND, RPVGRW, SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH, SLANGE, SLANTR
```

```
      EXTERNAL           LSAME, SLAMCH, SLANGE, SLANTR
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGECON, SGEEQU, SGERFS, SGETRF, SGETRS, SLACPY,
     $                   SLAQGE, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      NOTRAN = LSAME( TRANS, 'N' )
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         ROWEQU = .FALSE.
         COLEQU = .FALSE.
      ELSE
         ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
         COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         SMLNUM = SLAMCH( 'Safe minimum' )
         BIGNUM = ONE / SMLNUM
      END IF
*
*     Test the input parameters.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.LSAME( FACT, 'F' ) )
     $     THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $         LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $         ( ROWEQU .OR. COLEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -10
      ELSE
         IF( ROWEQU ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 10 J = 1, N
```

```
            RCMIN = MIN( RCMIN, R( J ) )
            RCMAX = MAX( RCMAX, R( J ) )
   10       CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -11
            ELSE IF( N.GT.0 ) THEN
               ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               ROWCND = ONE
            END IF
         END IF
         IF( COLEQU .AND. INFO.EQ.0 ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 20 J = 1, N
               RCMIN = MIN( RCMIN, C( J ) )
               RCMAX = MAX( RCMAX, C( J ) )
   20       CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -12
            ELSE IF( N.GT.0 ) THEN
               COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               COLCND = ONE
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -14
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -16
            END IF
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGESVX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*        Compute row and column scalings to equilibrate the matrix A.
*
         CALL SGEEQU( N, N, A, LDA, R, C, ROWCND, COLCND, AMAX, INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*           Equilibrate the matrix.
*
            CALL SLAQGE( N, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
```

```
     $                   EQUED )
            ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
            COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         END IF
      END IF
*
*     Scale the right hand side.
*
      IF( NOTRAN ) THEN
         IF( ROWEQU ) THEN
            DO 40 J = 1, NRHS
               DO 30 I = 1, N
                  B( I, J ) = R( I )*B( I, J )
   30          CONTINUE
   40       CONTINUE
         END IF
      ELSE IF( COLEQU ) THEN
         DO 60 J = 1, NRHS
            DO 50 I = 1, N
               B( I, J ) = C( I )*B( I, J )
   50       CONTINUE
   60    CONTINUE
      END IF
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the LU factorization of A.
*
         CALL SLACPY( 'Full', N, N, A, LDA, AF, LDAF )
         CALL SGETRF( N, N, AF, LDAF, IPIV, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 ) THEN
*
*           Compute the reciprocal pivot growth factor of the
*           leading rank-deficient INFO columns of A.
*
            RPVGRW = SLANTR( 'M', 'U', 'N', INFO, INFO, AF, LDAF,
     $               WORK )
            IF( RPVGRW.EQ.ZERO ) THEN
               RPVGRW = ONE
            ELSE
               RPVGRW = SLANGE( 'M', N, INFO, A, LDA, WORK ) / RPVGRW
            END IF
            WORK( 1 ) = RPVGRW
            RCOND = ZERO
            RETURN
         END IF
      END IF
```

```
*
*     Compute the norm of the matrix A and the
*     reciprocal pivot growth factor RPVGRW.
*
      IF( NOTRAN ) THEN
         NORM = '1'
      ELSE
         NORM = 'I'
      END IF
      ANORM = SLANGE( NORM, N, N, A, LDA, WORK )
      RPVGRW = SLANTR( 'M', 'U', 'N', N, N, AF, LDAF, WORK )
      IF( RPVGRW.EQ.ZERO ) THEN
         RPVGRW = ONE
      ELSE
         RPVGRW = SLANGE( 'M', N, N, A, LDA, WORK ) / RPVGRW
      END IF
*
*     Compute the reciprocal of the condition number of A.
*
      CALL SGECON( NORM, N, AF, LDAF, ANORM, RCOND, WORK, IWORK, INFO )
*
*     Compute the solution matrix X.
*
      CALL SLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL SGETRS( TRANS, N, NRHS, AF, LDAF, IPIV, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL SGERFS( TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB, X,
     $             LDX, FERR, BERR, WORK, IWORK, INFO )
*
*     Transform the solution matrix X to a solution of the original
*     system.
*
      IF( NOTRAN ) THEN
         IF( COLEQU ) THEN
            DO 80 J = 1, NRHS
               DO 70 I = 1, N
                  X( I, J ) = C( I )*X( I, J )
   70          CONTINUE
   80       CONTINUE
            DO 90 J = 1, NRHS
               FERR( J ) = FERR( J ) / COLCND
   90       CONTINUE
         END IF
      ELSE IF( ROWEQU ) THEN
         DO 110 J = 1, NRHS
            DO 100 I = 1, N
               X( I, J ) = R( I )*X( I, J )
```

```
  100       CONTINUE
  110    CONTINUE
         DO 120 J = 1, NRHS
            FERR( J ) = FERR( J ) / ROWCND
  120    CONTINUE
      END IF
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.SLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      WORK( 1 ) = RPVGRW
      RETURN
*
*     End of SGESVX
*
      END
```

21.7.4 lisp code

— sgesvx.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sgesvx
         (fact trans n nrhs a lda af ldaf ipiv equed r c b ldb$ x ldx rcond
          ferr berr work iwork info)
    (declare (type (single-float) rcond)
             (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array single-float (*)) work berr ferr x b c r af a)
             (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
             (type (string *) equed trans fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (trans character trans-%data% trans-%offset%)
         (equed character equed-%data% equed-%offset%)
         (a single-float a-%data% a-%offset%)
         (af single-float af-%data% af-%offset%)
         (r single-float r-%data% r-%offset%)
         (c single-float c-%data% c-%offset%)
         (b single-float b-%data% b-%offset%)
         (x single-float x-%data% x-%offset%)
         (ferr single-float ferr-%data% ferr-%offset%)
```

```
   (berr single-float berr-%data% berr-%offset%)
   (work single-float work-%data% work-%offset%)
   (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
   (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
(prog ((amax 0.0) (anorm 0.0) (bignum 0.0) (colcnd 0.0) (rcmax 0.0)
       (rcmin 0.0) (rowcnd 0.0) (rpvgrw 0.0) (smlnum 0.0) (i 0)
       (infequ 0) (j 0)
       (norm
        (make-array '(1) :element-type 'character :initial-element #\ ))
       (colequ nil) (equil nil) (nofact nil) (notran nil) (rowequ nil))
  (declare
   (type (single-float) amax anorm bignum colcnd rcmax rcmin rowcnd
    rpvgrw smlnum)
   (type (f2cl-lib:integer4) i infequ j)
   (type (string 1) norm)
   (type f2cl-lib:logical colequ equil nofact notran rowequ))
  (setf info 0)
  (setf nofact
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame fact "N")
            (declare (ignore var-1))
            (when var-0 (setf fact var-0))
            ret-val))
  (setf equil
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame fact "E")
            (declare (ignore var-1))
            (when var-0 (setf fact var-0))
            ret-val))
  (setf notran
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame trans "N")
            (declare (ignore var-1))
            (when var-0 (setf trans var-0))
            ret-val))
  (cond
   ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
    (setf rowequ f2cl-lib:%false%) (setf colequ f2cl-lib:%false%))
   (t
    (setf rowequ
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "R")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "B")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
```

```
            ret-val)))
  (setf colequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "C")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)))
  (setf smlnum (slamch "Safe minimum")) (setf bignum (/ one smlnum))))
 (cond
  ((and (not nofact) (not equil)
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame fact "F")
           (declare (ignore var-1))
           (when var-0 (setf fact var-0))
           ret-val)))
  (setf info -1))
  ((and (not notran)
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame trans "T")
           (declare (ignore var-1))
           (when var-0 (setf trans var-0))
           ret-val))
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame trans "C")
           (declare (ignore var-1))
           (when var-0 (setf trans var-0))
           ret-val)))
  (setf info -2))
  ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
  ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
  ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
  ((and
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame fact "F")
      (declare (ignore var-1))
      (when var-0 (setf fact var-0))
      ret-val)
    (not
     (or rowequ colequ
```

```
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame equed "N")
            (declare (ignore var-1))
            (when var-0 (setf equed var-0))
            ret-val))))
  (setf info -10))
 (t
  (cond
   (rowequ (setf rcmin bignum) (setf rcmax zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf rcmin
                (min rcmin
                     (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
        (setf rcmax
                (max rcmax
                     (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
       label10))
    (cond ((<= rcmin zero) (setf info -11))
          ((> n 0)
           (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum))))
          (t (setf rowcnd one)))))
  (cond
   ((and colequ (= info 0)) (setf rcmin bignum) (setf rcmax zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf rcmin
                (min rcmin
                     (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
        (setf rcmax
                (max rcmax
                     (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
       label20))
    (cond ((<= rcmin zero) (setf info -12))
          ((> n 0)
           (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum))))
          (t (setf colcnd one)))))
  (cond
   ((= info 0)
    (cond
     ((< ldb$
         (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -14))
     ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -16)))))))
(cond
 ((/= info 0) (xerbla "SGESVX" (f2cl-lib:int-sub info))
  (go end_label)))
```

```
(cond
 (equil
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (sgeequ n n a lda r c rowcnd colcnd amax infequ)
    (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5))
    (setf rowcnd var-6)
    (setf colcnd var-7)
    (setf amax var-8)
    (setf infequ var-9))
  (cond
   ((= infequ 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slaqge n n a lda r c rowcnd colcnd amax equed)
      (declare (ignore var-2 var-4 var-5))
      (when var-0 (setf n var-0))
      (when var-1 (setf n var-1))
      (when var-3 (setf lda var-3))
      (when var-6 (setf rowcnd var-6))
      (when var-7 (setf colcnd var-7))
      (when var-8 (setf amax var-8))
      (when var-9 (setf equed var-9)))
    (setf rowequ
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "R")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "B")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)))
    (setf colequ
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "C")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "B")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)))))))
(cond
 (notran
  (cond
```

```
    (rowequ
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j nrhs) nil)
       (tagbody
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)
                     (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                        (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)))
           label30))
       label40)))))
  (colequ
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j nrhs) nil)
     (tagbody
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                   (* (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)
                      (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                     b-%offset%)))
         label50))
     label60))))
 (cond
  ((or nofact equil)
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slacpy "Full" n n a lda af ldaf)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf n var-1))
     (when var-2 (setf n var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf ldaf var-6)))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
       (sgetrf n n af ldaf ipiv info)
     (declare (ignore var-2 var-4))
     (setf n var-0)
     (setf n var-1)
     (setf ldaf var-3)
     (setf info var-5))
   (cond
    ((> info 0)
     (setf rpvgrw
             (multiple-value-bind
                 (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6
                  var-7)
```

```
                (slantr "M" "U" "N" info info af ldaf work)
              (declare (ignore var-0 var-1 var-2 var-5 var-7))
              (when var-3 (setf info var-3))
              (when var-4 (setf info var-4))
              (when var-6 (setf ldaf var-6))
              ret-val))
    (cond ((= rpvgrw zero) (setf rpvgrw one))
          (t
           (setf rpvgrw
                   (/
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                        (slange "M" n info a lda work)
                      (declare (ignore var-0 var-3 var-5))
                      (when var-1 (setf n var-1))
                      (when var-2 (setf info var-2))
                      (when var-4 (setf lda var-4))
                      ret-val)
                    rpvgrw))))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) rpvgrw)
    (setf rcond zero) (go end_label)))))
(cond (notran (f2cl-lib:f2cl-set-string norm "1" (string 1)))
      (t (f2cl-lib:f2cl-set-string norm "I" (string 1))))
(setf anorm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange norm n n a lda work)
          (declare (ignore var-3 var-5))
          (when var-0 (setf norm var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf rpvgrw
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (slantr "M" "U" "N" n n af ldaf work)
          (declare (ignore var-0 var-1 var-2 var-5 var-7))
          (when var-3 (setf n var-3))
          (when var-4 (setf n var-4))
          (when var-6 (setf ldaf var-6))
          ret-val))
(cond ((= rpvgrw zero) (setf rpvgrw one))
      (t
       (setf rpvgrw
               (/
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                    (slange "M" n n a lda work)
                  (declare (ignore var-0 var-3 var-5))
```

```
                    (when var-1 (setf n var-1))
                    (when var-2 (setf n var-2))
                    (when var-4 (setf lda var-4))
                    ret-val)
                  rpvgrw))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sgecon norm n af ldaf anorm rcond work iwork info)
  (declare (ignore var-2 var-4 var-6 var-7))
  (setf norm var-0)
  (setf n var-1)
  (setf ldaf var-3)
  (setf rcond var-5)
  (setf info var-8))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "Full" n nrhs b ldb$ x ldx)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf ldx var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sgetrs trans n nrhs af ldaf ipiv x ldx info)
  (declare (ignore var-3 var-5 var-6))
  (setf trans var-0)
  (setf n var-1)
  (setf nrhs var-2)
  (setf ldaf var-4)
  (setf ldx var-7)
  (setf info var-8))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16)
    (sgerfs trans n nrhs a lda af ldaf ipiv b ldb$ x ldx ferr berr work
     iwork info)
  (declare
   (ignore var-2 var-3 var-5 var-7 var-8 var-9 var-10 var-11 var-12
    var-13 var-14 var-15))
  (setf trans var-0)
  (setf n var-1)
  (setf lda var-4)
  (setf ldaf var-6)
  (setf info var-16))
(cond
 (notran
  (cond
   (colequ
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
```

```
              (tagbody
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                         x-%offset%)
                          (* (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)
                             (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                            x-%offset%)))
                   label70))
               label80))
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j nrhs) nil)
              (tagbody
                (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                      (/
                       (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                       colcnd))
               label90)))))
          (rowequ
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j nrhs) nil)
             (tagbody
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i n) nil)
                 (tagbody
                   (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                        x-%offset%)
                         (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                            (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                           x-%offset%)))
                  label100))
              label110))
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j nrhs) nil)
             (tagbody
               (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                       (/ (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                          rowcnd))
              label120))))
         (if (< rcond (slamch "Epsilon"))
             (setf info (f2cl-lib:int-add n 1)))
         (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) rpvgrw)
         (go end_label)
        end_label
         (return
          (values fact trans n nrhs nil lda nil ldaf nil equed nil nil nil ldb$
                  nil ldx rcond nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
```

```
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgesvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (string) (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::fact
                                              fortran-to-lisp::trans
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldaf nil
                                              fortran-to-lisp::equed nil nil
                                              nil fortran-to-lisp::ldb$ nil
                                              fortran-to-lisp::ldx
                                              fortran-to-lisp::rcond nil nil
                                              nil nil fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::sgetrs
                                              fortran-to-lisp::sgetrf
                                              fortran-to-lisp::sgerfs
                                              fortran-to-lisp::sgecon
                                              fortran-to-lisp::sgeequ))))
```

21.8 sgesvxx

21.8.1 Axiom unit tests

— sgesvxx.input —

```
)set break resume
)sys rm -f sgesvxx.output
)spool sgesvxx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

21.8.2 Axiom help page

— sgesvxx.help —

```
====================================================================
sgesvxx examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgesvxx (FACT, TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV,
           EQUED, R, C, B, LDB, X, LDX, RCOND, RPVGRW, BERR, N_ERR_BNDS,
           ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, IWORK, INFO)
            SGESVXX computes the solution to system of linear equations A * X
           = B for GE matrices

Function/Subroutine Documentation
   subroutine sgesvxx (character FACT, character TRANS, integer N, integer
       NRHS, real, dimension( lda, * ) A, integer LDA, real, dimension( ldaf,
       * ) AF, integer LDAF, integer, dimension( * ) IPIV, character EQUED,
       real, dimension( * ) R, real, dimension( * ) C, real, dimension( ldb, *
       ) B, integer LDB, real, dimension( ldx , * ) X, integer LDX, real
       RCOND, real RPVGRW, real, dimension( * ) BERR, integer N_ERR_BNDS,
       real, dimension( nrhs, * ) ERR_BNDS_NORM, real, dimension( nrhs, * )
```

```
      ERR_BNDS_COMP, integer NPARAMS, real, dimension( * ) PARAMS, real,
      dimension( * ) WORK, integer, dimension( * ) IWORK, integer INFO)
       SGESVXX computes the solution to system of linear equations A * X = B
      for GE matrices

Purpose:
   SGESVXX uses the LU factorization to compute the solution to a
   real system of linear equations  A * X = B,  where A is an
   N-by-N matrix and X and B are N-by-NRHS matrices.

   If requested, both normwise and maximum componentwise error bounds
   are returned. SGESVXX will return a solution with a tiny
   guaranteed error (O(eps) where eps is the working machine
   precision) unless the matrix is very ill-conditioned, in which
   case a warning is returned. Relevant condition numbers also are
   calculated and returned.

   SGESVXX accepts user-provided factorizations and equilibration
   factors; see the definitions of the FACT and EQUED options.
   Solving with refinement and using a factorization from a previous
   SGESVXX call will also produce a solution with either O(eps)
   errors or warnings, but we cannot make that claim for general
   user-provided factorizations and equilibration factors if they
   differ from what SGESVXX would itself produce.

Description:
   The following steps are performed:

   1. If FACT = 'E', real scaling factors are computed to equilibrate
      the system:

      TRANS = 'N':  diag(R)*A*diag(C)     *inv(diag(C))*X = diag(R)*B
      TRANS = 'T': (diag(R)*A*diag(C))**T *inv(diag(R))*X = diag(C)*B
      TRANS = 'C': (diag(R)*A*diag(C))**H *inv(diag(R))*X = diag(C)*B

      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(R)*A*diag(C) and B by diag(R)*B (if TRANS='N')
      or diag(C)*B (if TRANS = 'T' or 'C').

   2. If FACT = 'N' or 'E', the LU decomposition is used to factor
      the matrix A (after equilibration if FACT = 'E') as

                 A = P * L * U,

      where P is a permutation matrix, L is a unit lower triangular
      matrix, and U is upper triangular.

   3. If some U(i,i)=0, so that U is exactly singular, then the
      routine returns with INFO = i. Otherwise, the factored form of A
```

```
      is used to estimate the condition number of the matrix A (see
      argument RCOND). If the reciprocal of the condition number is less
      than machine precision, the routine still goes on to solve for X
      and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. By default (unless PARAMS(LA_LINRX_ITREF_I) is set to zero),
      the routine will use iterative refinement to try to get a small
      error and error bounds.  Refinement calculates the residual to at
      least twice the working precision.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(C) (if TRANS = 'N') or diag(R) (if TRANS = 'T' or 'C') so
      that it solves the original system before equilibration.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AF and IPIV contain the factored form of A.
           If EQUED is not 'N', the matrix A has been
           equilibrated with scaling factors given by R and C.
           A, AF, and IPIV are not modified.
   = 'N':  The matrix A will be copied to AF and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
           copied to AF and factored.

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate Transpose = Transpose)

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
```

```
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is REAL array, dimension (LDA,N)
  On entry, the N-by-N matrix A.  If FACT = 'F' and EQUED is
  not 'N', then A must have been equilibrated by the scaling
  factors in R and/or C.  A is not modified if FACT = 'F' or
  'N', or if FACT = 'E' and EQUED = 'N' on exit.

  On exit, if EQUED .ne. 'N', A is scaled as follows:
  EQUED = 'R':  A := diag(R) * A
  EQUED = 'C':  A := A * diag(C)
  EQUED = 'B':  A := diag(R) * A * diag(C).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is REAL array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the factors L and U from the factorization
  A = P*L*U as computed by SGETRF.  If EQUED .ne. 'N', then
  AF is the factored form of the equilibrated matrix A.

  If FACT = 'N', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the equilibrated matrix A (see the description of A for
  the form of the equilibrated matrix).

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains the pivot indices from the factorization A = P*L*U
  as computed by SGETRF; row i of the matrix was interchanged
  with row IPIV(i).

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the original matrix A.
```

```
  If FACT = 'E', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the equilibrated matrix A.

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
  = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
  = 'B':  Both row and column equilibration, i.e., A has been
          replaced by diag(R) * A * diag(C).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

R
  R is REAL array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
  EQUED = 'R' or 'B', each element of R must be positive.
  If R is output, each element of R is a power of the radix.
  If R is input, each element of R should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

C
  C is REAL array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.
  If C is output, each element of C is a power of the radix.
  If C is input, each element of C should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is REAL array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
```

```
  if TRANS = 'N' and EQUED = 'R' or 'B', B is overwritten by diag(R)*B;
  if TRANS = 'T' or 'C' and EQUED = 'C' or 'B', B is overwritten by diag(C)*B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is REAL array, dimension (LDX,NRHS)
  If INFO = 0, the N-by-NRHS solution matrix X to the original
  system of equations.  Note that A and B are modified on exit
  if EQUED .ne. 'N', and the solution to the equilibrated system is
  inv(diag(C))*X if TRANS = 'N' and EQUED = 'C' or 'B', or
  inv(diag(R))*X if TRANS = 'T' or 'C' and EQUED = 'R' or 'B'.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

RPVGRW
  RPVGRW is REAL
  Reciprocal pivot growth.  On exit, this contains the reciprocal
  pivot growth factor norm(A)/norm(U). The "max absolute element"
  norm is used.  If this is much less than 1, then the stability of
  the LU factorization of the (equilibrated) matrix A could be poor.
  This also means that the solution X, estimated condition numbers,
  and error bounds could be unreliable. If factorization fails with
  0<INFO<=N, then this contains the reciprocal pivot growth factor
  for the leading INFO columns of A.  In SGESVX, this quantity is
  returned in WORK(1).

BERR
  BERR is REAL array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
```

```
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
                      max_j (abs(XTRUE(j,i) - X(j,i)))
                     ------------------------------
                           max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
          reciprocal condition number is less than the threshold
          sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
          almost certainly within a factor of 10 of the true error
          so long as the next entry is greater than the threshold
          sqrt(n) * slamch('Epsilon'). This error bound should only
          be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
          reciprocal condition number.  Compared with the threshold
          sqrt(n) * slamch('Epsilon') to determine if the error
          estimate is "guaranteed". These reciprocal condition
          numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
          appropriately scaled matrix Z.
          Let Z = S*A, where S scales each row by a power of the
          radix so all absolute row sums of Z are approximately 1.

  See Lapack Working Note 165 for further details and extra cautions.

ERR_BNDS_COMP

  ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:
```

```
  Componentwise relative error in the ith solution vector:
                             abs(XTRUE(j,i) - X(j,i))
                      max_j ----------------------
                                   abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
           reciprocal condition number is less than the threshold
           sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
          almost certainly within a factor of 10 of the true error
          so long as the next entry is greater than the threshold
          sqrt(n) * slamch('Epsilon'). This error bound should only
          be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
           reciprocal condition number.  Compared with the threshold
           sqrt(n) * slamch('Epsilon') to determine if the error
           estimate is "guaranteed". These reciprocal condition
           numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
           appropriately scaled matrix Z.
           Let Z = S*(A*diag(x)), where x is the solution for the
           current right-hand side and S scales each row of
           A*diag(x) by a power of the radix so all absolute row
           sums of Z are approximately 1.

  See Lapack Working Note 165 for further details and extra cautions.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is REAL array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
```

```
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
          (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is REAL array, dimension (4*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is
        guaranteed.
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
      has been completed, but the factor U is exactly singular, so
      the solution and error bounds could not be computed. RCOND = 0
      is returned.
    = N+J: The solution corresponding to the Jth right-hand side is
           not guaranteed. The solutions corresponding to other right-
           hand sides K with K > J may not be guaranteed as well, but
           only the first such right-hand side is reported. If a small
           componentwise error is not requested (PARAMS(3) = 0.0) then
           the Jth right-hand side is the first with a normwise error
           bound that is not guaranteed (the smallest J such
```

```
that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
the Jth right-hand side is the first with either a normwise or
componentwise error bound that is not guaranteed (the smallest
J such that either ERR_BNDS_NORM(J,1) = 0.0 or
ERR_BNDS_COMP(J,1) = 0.0). See the definition of
ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
about all of the right-hand sides check ERR_BNDS_NORM or
ERR_BNDS_COMP.
```

———-

21.8.3 fortran code

— sgesvxx.f —

```
*  ====================================================================
      SUBROUTINE SGESVXX( FACT, TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $                    EQUED, R, C, B, LDB, X, LDX, RCOND, RPVGRW,
     $                    BERR, N_ERR_BNDS, ERR_BNDS_NORM,
     $                    ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, IWORK,
     $                    INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, TRANS
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      REAL               RCOND, RPVGRW
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      REAL               A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX , * ),WORK( * )
      REAL               R( * ), C( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
```

```
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
*     ..
*     .. Local Scalars ..
      LOGICAL            COLEQU, EQUIL, NOFACT, NOTRAN, ROWEQU
      INTEGER            INFEQU, J
      REAL               AMAX, BIGNUM, COLCND, RCMAX, RCMIN, ROWCND,
     $                   SMLNUM
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, SLAMCH, SLA_GERPVGRW
      LOGICAL            LSAME
      REAL               SLAMCH, SLA_GERPVGRW
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEEQUB, SGETRF, SGETRS, SLACPY, SLAQGE,
     $                   XERBLA, SLASCL2, SGERFSX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      NOTRAN = LSAME( TRANS, 'N' )
      SMLNUM = SLAMCH( 'Safe minimum' )
      BIGNUM = ONE / SMLNUM
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         ROWEQU = .FALSE.
         COLEQU = .FALSE.
      ELSE
         ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
         COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
      END IF
*
*     Default is failure.  If an input parameter is wrong or
*     factorization fails, make everything look horrible.  Only the
*     pivot growth is set here, the rest is initialized in SGERFSX.
*
      RPVGRW = ZERO
*
```

```
*     Test the input parameters.  PARAMS is not tested until SGERFSX.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.
     $     LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $        LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $        ( ROWEQU .OR. COLEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -10
      ELSE
         IF( ROWEQU ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 10 J = 1, N
               RCMIN = MIN( RCMIN, R( J ) )
               RCMAX = MAX( RCMAX, R( J ) )
 10         CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -11
            ELSE IF( N.GT.0 ) THEN
               ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               ROWCND = ONE
            END IF
         END IF
         IF( COLEQU .AND. INFO.EQ.0 ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 20 J = 1, N
               RCMIN = MIN( RCMIN, C( J ) )
               RCMAX = MAX( RCMAX, C( J ) )
 20         CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -12
            ELSE IF( N.GT.0 ) THEN
               COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               COLCND = ONE
            END IF
         END IF
```

```
         IF( INFO.EQ.0 ) THEN
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -14
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -16
            END IF
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGESVXX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*     Compute row and column scalings to equilibrate the matrix A.
*
         CALL SGEEQUB( N, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $        INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*     Equilibrate the matrix.
*
            CALL SLAQGE( N, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $           EQUED )
            ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
            COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         END IF
*
*     If the scaling factors are not applied, set them to 1.0.
*
         IF ( .NOT.ROWEQU ) THEN
            DO J = 1, N
               R( J ) = 1.0
            END DO
         END IF
         IF ( .NOT.COLEQU ) THEN
            DO J = 1, N
               C( J ) = 1.0
            END DO
         END IF
      END IF
*
*     Scale the right-hand side.
*
      IF( NOTRAN ) THEN
         IF( ROWEQU ) CALL SLASCL2( N, NRHS, R, B, LDB )
      ELSE
         IF( COLEQU ) CALL SLASCL2( N, NRHS, C, B, LDB )
```

```
      END IF
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the LU factorization of A.
*
         CALL SLACPY( 'Full', N, N, A, LDA, AF, LDAF )
         CALL SGETRF( N, N, AF, LDAF, IPIV, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 ) THEN
*
*           Pivot in column INFO is exactly 0
*           Compute the reciprocal pivot growth factor of the
*           leading rank-deficient INFO columns of A.
*
            RPVGRW = SLA_GERPVGRW( N, INFO, A, LDA, AF, LDAF )
            RETURN
         END IF
      END IF
*
*     Compute the reciprocal pivot growth factor RPVGRW.
*
      RPVGRW = SLA_GERPVGRW( N, N, A, LDA, AF, LDAF )
*
*     Compute the solution matrix X.
*
      CALL SLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL SGETRS( TRANS, N, NRHS, AF, LDAF, IPIV, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL SGERFSX( TRANS, EQUED, N, NRHS, A, LDA, AF, LDAF,
     $     IPIV, R, C, B, LDB, X, LDX, RCOND, BERR,
     $     N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $     WORK, IWORK, INFO )
*
*     Scale solutions.
*
      IF ( COLEQU .AND. NOTRAN ) THEN
         CALL SLASCL2 ( N, NRHS, C, X, LDX )
      ELSE IF ( ROWEQU .AND. .NOT.NOTRAN ) THEN
         CALL SLASCL2 ( N, NRHS, R, X, LDX )
      END IF
*
      RETURN
*
*     End of SGESVXX
```

```
      END
```

———-

21.8.4 lisp code

— **sgesvxx.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sgesvxx
         (fact trans n nrhs a lda af ldaf ipiv equed r c b ldb$ x ldx rcond
          rpvgrw berr n_err_bnds err_bnds_norm err_bnds_comp nparams params
          work iwork info)
    (declare (type (single-float) rpvgrw rcond)
             (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array single-float (*)) work params err_bnds_comp
              err_bnds_norm berr x b c r af a)
             (type (f2cl-lib:integer4) info nparams n_err_bnds ldx ldb$ ldaf
              lda nrhs n)
             (type (string *) equed trans fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (trans character trans-%data% trans-%offset%)
         (equed character equed-%data% equed-%offset%)
         (a single-float a-%data% a-%offset%)
         (af single-float af-%data% af-%offset%)
         (r single-float r-%data% r-%offset%)
         (c single-float c-%data% c-%offset%)
         (b single-float b-%data% b-%offset%)
         (x single-float x-%data% x-%offset%)
         (berr single-float berr-%data% berr-%offset%)
         (err_bnds_norm single-float err_bnds_norm-%data%
          err_bnds_norm-%offset%)
         (err_bnds_comp single-float err_bnds_comp-%data%
          err_bnds_comp-%offset%)
         (params single-float params-%data% params-%offset%)
         (work single-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((amax 0.0) (bignum 0.0) (colcnd 0.0) (rcmax 0.0) (rcmin 0.0)
             (rowcnd 0.0) (smlnum 0.0) (infequ 0) (j 0) (colequ nil)
             (equil nil) (nofact nil) (notran nil) (rowequ nil))
        (declare
         (type (single-float) amax bignum colcnd rcmax rcmin rowcnd smlnum)
```

```
 (type (f2cl-lib:integer4) infequ j)
 (type f2cl-lib:logical colequ equil nofact notran rowequ))
(setf info 0)
(setf nofact
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "N")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val))
(setf equil
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "E")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val))
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(setf smlnum (slamch "Safe minimum"))
(setf bignum (/ one smlnum))
(cond
 ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
  (setf rowequ f2cl-lib:%false%) (setf colequ f2cl-lib:%false%))
 (t
  (setf rowequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "R")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)))
  (setf colequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "C")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
```

```
            ret-val)))))
(setf rpvgrw zero)
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame fact "F")
     (declare (ignore var-1))
     (when var-0 (setf fact var-0))
     ret-val)
   (not
    (or rowequ colequ
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame equed "N")
          (declare (ignore var-1))
          (when var-0 (setf equed var-0))
          ret-val))))
  (setf info -10))
 (t
  (cond
   (rowequ (setf rcmin bignum) (setf rcmax zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
```

```
          (setf rcmin
                  (min rcmin
                       (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
          (setf rcmax
                  (max rcmax
                       (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
        label10))
      (cond ((<= rcmin zero) (setf info -11))
            ((> n 0)
             (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum))))
            (t (setf rowcnd one)))))
    (cond
     ((and colequ (= info 0)) (setf rcmin bignum) (setf rcmax zero)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf rcmin
                  (min rcmin
                       (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
          (setf rcmax
                  (max rcmax
                       (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
        label20))
      (cond ((<= rcmin zero) (setf info -12))
            ((> n 0)
             (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum))))
            (t (setf colcnd one)))))
    (cond
     ((= info 0)
      (cond
       ((< ldb$
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -14))
       ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -16)))))))
  (cond
   ((/= info 0) (xerbla "SGESVXX" (f2cl-lib:int-sub info))
    (go end_label)))
  (cond
   (equil
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (sgeequb n n a lda r c rowcnd colcnd amax infequ)
      (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5))
      (setf rowcnd var-6)
      (setf colcnd var-7)
      (setf amax var-8)
      (setf infequ var-9))
    (cond
     ((= infequ 0)
```

```
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slaqge n n a lda r c rowcnd colcnd amax equed)
      (declare (ignore var-2 var-4 var-5))
      (when var-0 (setf n var-0))
      (when var-1 (setf n var-1))
      (when var-3 (setf lda var-3))
      (when var-6 (setf rowcnd var-6))
      (when var-7 (setf colcnd var-7))
      (when var-8 (setf amax var-8))
      (when var-9 (setf equed var-9)))
    (setf rowequ
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "R")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "B")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)))
    (setf colequ
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "C")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "B")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)))))
  (cond
   ((not rowequ)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%) 1.0)
      label100000))))
  (cond
   ((not colequ)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) 1.0)
      label100001))))))
(cond
```

```
 (notran
  (if rowequ
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (slascl2 n nrhs r b ldb$)
        (declare (ignore var-2 var-3))
        (when var-0 (setf n var-0))
        (when var-1 (setf nrhs var-1))
        (when var-4 (setf ldb$ var-4)))))
 (t
  (if colequ
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (slascl2 n nrhs c b ldb$)
        (declare (ignore var-2 var-3))
        (when var-0 (setf n var-0))
        (when var-1 (setf nrhs var-1))
        (when var-4 (setf ldb$ var-4))))))
(cond
 ((or nofact equil)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "Full" n n a lda af ldaf)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldaf var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (sgetrf n n af ldaf ipiv info)
    (declare (ignore var-2 var-4))
    (setf n var-0)
    (setf n var-1)
    (setf ldaf var-3)
    (setf info var-5))
  (cond
   ((> info 0)
    (setf rpvgrw
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                (sla_gerpvgrw n info a lda af ldaf)
              (declare (ignore var-2 var-4))
              (when var-0 (setf n var-0))
              (when var-1 (setf info var-1))
              (when var-3 (setf lda var-3))
              (when var-5 (setf ldaf var-5))
              ret-val))
    (go end_label)))))
(setf rpvgrw
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (sla_gerpvgrw n n a lda af ldaf)
          (declare (ignore var-2 var-4))
```

```
          (when var-0 (setf n var-0))
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-5 (setf ldaf var-5))
          ret-val))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "Full" n nrhs b ldb$ x ldx)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf ldx var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sgetrs trans n nrhs af ldaf ipiv x ldx info)
  (declare (ignore var-3 var-5 var-6))
  (setf trans var-0)
  (setf n var-1)
  (setf nrhs var-2)
  (setf ldaf var-4)
  (setf ldx var-7)
  (setf info var-8))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19
     var-20 var-21 var-22 var-23 var-24)
    (sgerfsx trans equed n nrhs a lda af ldaf ipiv r c b ldb$ x ldx
     rcond berr n_err_bnds err_bnds_norm err_bnds_comp nparams params
     work iwork info)
  (declare
   (ignore var-4 var-6 var-8 var-9 var-10 var-11 var-13 var-16 var-18
    var-19 var-21 var-22 var-23))
  (when var-0 (setf trans var-0))
  (when var-1 (setf equed var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf nrhs var-3))
  (when var-5 (setf lda var-5))
  (when var-7 (setf ldaf var-7))
  (when var-12 (setf ldb$ var-12))
  (when var-14 (setf ldx var-14))
  (when var-15 (setf rcond var-15))
  (when var-17 (setf n_err_bnds var-17))
  (when var-20 (setf nparams var-20))
  (when var-24 (setf info var-24)))
(cond
 ((and colequ notran)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slascl2 n nrhs c x ldx)
    (declare (ignore var-2 var-3))
    (when var-0 (setf n var-0))
```

```
            (when var-1 (setf nrhs var-1))
            (when var-4 (setf ldx var-4))))
          ((and rowequ (not notran))
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
               (slascl2 n nrhs r x ldx)
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf nrhs var-1))
            (when var-4 (setf ldx var-4)))))
         (go end_label)
        end_label
         (return
          (values fact trans n nrhs nil lda nil ldaf nil equed nil nil nil ldb$
                  nil ldx rcond rpvgrw nil n_err_bnds nil nil nparams nil nil
                  nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgesvxx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (string) (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::fact
```

```
  fortran-to-lisp::trans
  fortran-to-lisp::n
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldaf nil
  fortran-to-lisp::equed nil nil
  nil fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond
  fortran-to-lisp::rpvgrw nil
  fortran-to-lisp::n_err_bnds nil
  nil fortran-to-lisp::nparams nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::sgetrs
  fortran-to-lisp::sgetrf
  fortran-to-lisp::sgeequb))))
```

Chapter 22

LAPACK - General Matrices, Linear Solve, Complex

22.1 cgels

22.1.1 Axiom unit tests

— **cgels.input** —

```
)set break resume
)sys rm -f cgels.output
)spool cgels.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

22.1.2 Axiom help page

— **cgels.help** —

```
==========================================================================
cgels examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgels (TRANS, M, N, NRHS, A, LDA, B, LDB, WORK, LWORK, INFO)
            CGELS solves overdetermined or underdetermined systems for GE
           matrices

Function/Subroutine Documentation
   subroutine cgels (character TRANS, integer M, integer N, integer NRHS,
       complex, dimension( lda, * ) A, integer LDA, complex, dimension( ldb, *
       ) B, integer LDB, complex, dimension( * ) WORK, integer LWORK, integer
       INFO)
        CGELS solves overdetermined or underdetermined systems for GE matrices

Purpose:
   CGELS solves overdetermined or underdetermined complex linear systems
   involving an M-by-N matrix A, or its conjugate-transpose, using a QR
   or LQ factorization of A.  It is assumed that A has full rank.

   The following options are provided:

   1. If TRANS = 'N' and m >= n:  find the least squares solution of
      an overdetermined system, i.e., solve the least squares problem
      minimize || B - A*X ||.

   2. If TRANS = 'N' and m < n:  find the minimum norm solution of
      an underdetermined system A * X = B.

   3. If TRANS = 'C' and m >= n:  find the minimum norm solution of
      an undetermined system A**H * X = B.

   4. If TRANS = 'C' and m < n:  find the least squares solution of
      an overdetermined system, i.e., solve the least squares problem
      minimize || B - A**H * X ||.

   Several right hand side vectors b and solution vectors x can be
   handled in a single call; they are stored as the columns of the
   M-by-NRHS right hand side matrix B and the N-by-NRHS solution matrix X.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   = 'N': the linear system involves A;
   = 'C': the linear system involves A**H.
```

```
M
  M is INTEGER
  The number of rows of the matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of
  columns of the matrices B and X. NRHS >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  if M >= N, A is overwritten by details of its QR
    factorization as returned by CGEQRF;
  if M <  N, A is overwritten by details of its LQ
     factorization as returned by CGELQF.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the matrix B of right hand side vectors, stored
  columnwise; B is M-by-NRHS if TRANS = 'N', or N-by-NRHS
  if TRANS = 'C'.
  On exit, if INFO = 0, B is overwritten by the solution
  vectors, stored columnwise:
  if TRANS = 'N' and m >= n, rows 1 to n of B contain the least
  squares solution vectors; the residual sum of squares for the
  solution in each column is given by the sum of squares of the
  modulus of elements N+1 to M in that column;
  if TRANS = 'N' and m < n, rows 1 to N of B contain the
  minimum norm solution vectors;
  if TRANS = 'C' and m >= n, rows 1 to M of B contain the
  minimum norm solution vectors;
  if TRANS = 'C' and m < n, rows 1 to M of B contain the
  least squares solution vectors; the residual sum of squares
  for the solution in each column is given by the sum of
  squares of the modulus of elements M+1 to N in that column.

LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= MAX(1,M,N).

WORK
```

```
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  LWORK >= max( 1, MN + max( MN, NRHS ) ).
  For optimal performance,
  LWORK >= max( 1, MN + max( MN, NRHS )*NB ).
  where MN = min(M,N) and NB is the optimum block size.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO =  i, the i-th diagonal element of the
     triangular factor of A is zero, so that A does not have
     full rank; the least squares solution could not be computed.
```

———-

22.1.3 fortran code

— **cgels.f** —

```
*  =====================================================================
      SUBROUTINE CGELS( TRANS, M, N, NRHS, A, LDA, B, LDB, WORK, LWORK,
     $                  INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDA, LDB, LWORK, M, N, NRHS
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
```

```
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      COMPLEX            CZERO
      PARAMETER          ( CZERO = ( 0.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, TPSD
      INTEGER            BROW, I, IASCL, IBSCL, J, MN, NB, SCLLEN, WSIZE
      REAL               ANRM, BIGNUM, BNRM, SMLNUM
*     ..
*     .. Local Arrays ..
      REAL               RWORK( 1 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               CLANGE, SLAMCH
      EXTERNAL           LSAME, ILAENV, CLANGE, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGELQF, CGEQRF, CLASCL, CLASET, CTRTRS, CUNMLQ,
     $                   CUNMQR, SLABAD, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, REAL
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      MN = MIN( M, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.( LSAME( TRANS, 'N' ) .OR. LSAME( TRANS, 'C' ) ) ) THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -6
      ELSE IF( LDB.LT.MAX( 1, M, N ) ) THEN
         INFO = -8
      ELSE IF( LWORK.LT.MAX( 1, MN+MAX( MN, NRHS ) ) .AND.
     $   .NOT.LQUERY ) THEN
```

```
         INFO = -10
      END IF
*
*     Figure out optimal block size
*
      IF( INFO.EQ.0 .OR. INFO.EQ.-10 ) THEN
*
         TPSD = .TRUE.
         IF( LSAME( TRANS, 'N' ) )
     $      TPSD = .FALSE.
*
         IF( M.GE.N ) THEN
            NB = ILAENV( 1, 'CGEQRF', ' ', M, N, -1, -1 )
            IF( TPSD ) THEN
               NB = MAX( NB, ILAENV( 1, 'CUNMQR', 'LN', M, NRHS, N,
     $              -1 ) )
            ELSE
               NB = MAX( NB, ILAENV( 1, 'CUNMQR', 'LC', M, NRHS, N,
     $              -1 ) )
            END IF
         ELSE
            NB = ILAENV( 1, 'CGELQF', ' ', M, N, -1, -1 )
            IF( TPSD ) THEN
               NB = MAX( NB, ILAENV( 1, 'CUNMLQ', 'LC', N, NRHS, M,
     $              -1 ) )
            ELSE
               NB = MAX( NB, ILAENV( 1, 'CUNMLQ', 'LN', N, NRHS, M,
     $              -1 ) )
            END IF
         END IF
*
         WSIZE = MAX( 1, MN + MAX( MN, NRHS )*NB )
         WORK( 1 ) = REAL( WSIZE )
*
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGELS ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( MIN( M, N, NRHS ).EQ.0 ) THEN
         CALL CLASET( 'Full', MAX( M, N ), NRHS, CZERO, CZERO, B, LDB )
         RETURN
      END IF
*
```

```
*     Get machine parameters
*
      SMLNUM = SLAMCH( 'S' ) / SLAMCH( 'P' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
*
*     Scale A, B if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = CLANGE( 'M', M, N, A, LDA, RWORK )
      IASCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL CLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
         IASCL = 1
      ELSE IF( ANRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL CLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
         IASCL = 2
      ELSE IF( ANRM.EQ.ZERO ) THEN
*
*        Matrix all zero. Return zero solution.
*
         CALL CLASET( 'F', MAX( M, N ), NRHS, CZERO, CZERO, B, LDB )
         GO TO 50
      END IF
*
      BROW = M
      IF( TPSD )
     $   BROW = N
      BNRM = CLANGE( 'M', BROW, NRHS, B, LDB, RWORK )
      IBSCL = 0
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL CLASCL( 'G', 0, 0, BNRM, SMLNUM, BROW, NRHS, B, LDB,
     $                INFO )
         IBSCL = 1
      ELSE IF( BNRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL CLASCL( 'G', 0, 0, BNRM, BIGNUM, BROW, NRHS, B, LDB,
     $                INFO )
         IBSCL = 2
      END IF
```

```
*
      IF( M.GE.N ) THEN
*
*        compute QR factorization of A
*
         CALL CGEQRF( M, N, A, LDA, WORK( 1 ), WORK( MN+1 ), LWORK-MN,
     $                INFO )
*
*        workspace at least N, optimally N*NB
*
         IF( .NOT.TPSD ) THEN
*
*           Least-Squares Problem min || A * X - B ||
*
*           B(1:M,1:NRHS) := Q**H * B(1:M,1:NRHS)
*
            CALL CUNMQR( 'Left', 'Conjugate transpose', M, NRHS, N, A,
     $                   LDA, WORK( 1 ), B, LDB, WORK( MN+1 ), LWORK-MN,
     $                   INFO )
*
*           workspace at least NRHS, optimally NRHS*NB
*
*           B(1:N,1:NRHS) := inv(R) * B(1:N,1:NRHS)
*
            CALL CTRTRS( 'Upper', 'No transpose', 'Non-unit', N, NRHS,
     $                   A, LDA, B, LDB, INFO )
*
            IF( INFO.GT.0 ) THEN
               RETURN
            END IF
*
            SCLLEN = N
*
         ELSE
*
*           Overdetermined system of equations A**H * X = B
*
*           B(1:N,1:NRHS) := inv(R**H) * B(1:N,1:NRHS)
*
            CALL CTRTRS( 'Upper', 'Conjugate transpose','Non-unit',
     $                   N, NRHS, A, LDA, B, LDB, INFO )
*
            IF( INFO.GT.0 ) THEN
               RETURN
            END IF
*
*           B(N+1:M,1:NRHS) = ZERO
*
            DO 20 J = 1, NRHS
               DO 10 I = N + 1, M
```

```
                B( I, J ) = CZERO
   10         CONTINUE
   20      CONTINUE
*
*          B(1:M,1:NRHS) := Q(1:N,:) * B(1:N,1:NRHS)
*
           CALL CUNMQR( 'Left', 'No transpose', M, NRHS, N, A, LDA,
     $                  WORK( 1 ), B, LDB, WORK( MN+1 ), LWORK-MN,
     $                  INFO )
*
*          workspace at least NRHS, optimally NRHS*NB
*
           SCLLEN = M
*
        END IF
*
      ELSE
*
*       Compute LQ factorization of A
*
        CALL CGELQF( M, N, A, LDA, WORK( 1 ), WORK( MN+1 ), LWORK-MN,
     $               INFO )
*
*       workspace at least M, optimally M*NB.
*
        IF( .NOT.TPSD ) THEN
*
*          underdetermined system of equations A * X = B
*
           B(1:M,1:NRHS) := inv(L) * B(1:M,1:NRHS)
*
           CALL CTRTRS( 'Lower', 'No transpose', 'Non-unit', M, NRHS,
     $                  A, LDA, B, LDB, INFO )
*
           IF( INFO.GT.0 ) THEN
              RETURN
           END IF
*
           B(M+1:N,1:NRHS) = 0
*
           DO 40 J = 1, NRHS
              DO 30 I = M + 1, N
                 B( I, J ) = CZERO
   30         CONTINUE
   40      CONTINUE
*
*          B(1:N,1:NRHS) := Q(1:N,:)**H * B(1:M,1:NRHS)
*
           CALL CUNMLQ( 'Left', 'Conjugate transpose', N, NRHS, M, A,
     $                  LDA, WORK( 1 ), B, LDB, WORK( MN+1 ), LWORK-MN,
```

```
     $                   INFO )
*
*          workspace at least NRHS, optimally NRHS*NB
*
           SCLLEN = N
*
        ELSE
*
*          overdetermined system min || A**H * X - B ||
*
*          B(1:N,1:NRHS) := Q * B(1:N,1:NRHS)
*
           CALL CUNMLQ( 'Left', 'No transpose', N, NRHS, M, A, LDA,
     $                  WORK( 1 ), B, LDB, WORK( MN+1 ), LWORK-MN,
     $                  INFO )
*
*          workspace at least NRHS, optimally NRHS*NB
*
*          B(1:M,1:NRHS) := inv(L**H) * B(1:M,1:NRHS)
*
           CALL CTRTRS( 'Lower', 'Conjugate transpose', 'Non-unit',
     $                  M, NRHS, A, LDA, B, LDB, INFO )
*
           IF( INFO.GT.0 ) THEN
              RETURN
           END IF
*
           SCLLEN = M
*
        END IF
*
      END IF
*
*     Undo scaling
*
      IF( IASCL.EQ.1 ) THEN
         CALL CLASCL( 'G', 0, 0, ANRM, SMLNUM, SCLLEN, NRHS, B, LDB,
     $                INFO )
      ELSE IF( IASCL.EQ.2 ) THEN
         CALL CLASCL( 'G', 0, 0, ANRM, BIGNUM, SCLLEN, NRHS, B, LDB,
     $                INFO )
      END IF
      IF( IBSCL.EQ.1 ) THEN
         CALL CLASCL( 'G', 0, 0, SMLNUM, BNRM, SCLLEN, NRHS, B, LDB,
     $                INFO )
      ELSE IF( IBSCL.EQ.2 ) THEN
         CALL CLASCL( 'G', 0, 0, BIGNUM, BNRM, SCLLEN, NRHS, B, LDB,
     $                INFO )
      END IF
*
```

```
   50 CONTINUE
      WORK( 1 ) = REAL( WSIZE )
*
      RETURN
*
*     End of CGELS
*
      END
```

———-

22.1.4 lisp code

— **cgels.lisp** —

```
(let* ((zero 0.0) (one 1.0) (czero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (f2cl-lib:complex8) czero)
           (ignorable zero one czero))
  (defun cgels (trans m n nrhs a lda b ldb$ work lwork info)
    (declare (type (array f2cl-lib:complex8 (*)) work b a)
             (type (f2cl-lib:integer4) info lwork ldb$ lda nrhs n m)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%))
      (prog ((rwork (make-array 1 :element-type 'single-float)) (anrm 0.0)
             (bignum 0.0) (bnrm 0.0) (smlnum 0.0) (brow 0) (i 0) (iascl 0)
             (ibscl 0) (j 0) (mn 0) (nb 0) (scllen 0) (wsize 0) (lquery nil)
             (tpsd nil))
        (declare (type (array single-float (1)) rwork)
                 (type (single-float) anrm bignum bnrm smlnum)
                 (type (f2cl-lib:integer4) brow i iascl ibscl j mn nb scllen
                  wsize)
                 (type f2cl-lib:logical lquery tpsd))
        (setf info 0)
        (setf mn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond
         ((not
           (or
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "N")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
```

```
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame trans "C")
      (declare (ignore var-1))
      (when var-0 (setf trans var-0))
      ret-val)))
  (setf info -1))
 ((< m 0) (setf info -2)) ((< n 0) (setf info -3))
 ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -6))
 ((< ldb$
     (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)
          (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and
   (< lwork
      (max (the f2cl-lib:integer4 1)
           (the f2cl-lib:integer4
                (f2cl-lib:int-add mn
                                  (max (the f2cl-lib:integer4 mn)
                                       (the f2cl-lib:integer4
                                            nrhs))))))
   (not lquery))
  (setf info -10)))
(cond
 ((or (= info 0) (= info (f2cl-lib:int-sub 10)))
  (setf tpsd f2cl-lib:%true%)
  (if (multiple-value-bind (ret-val var-0 var-1)
          (lsame trans "N")
        (declare (ignore var-1))
        (when var-0 (setf trans var-0))
        ret-val)
      (setf tpsd f2cl-lib:%false%))
  (cond
   ((>= m n)
    (setf nb
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 1 "CGEQRF" " " m n -1 -1)
              (declare (ignore var-0 var-1 var-2 var-5 var-6))
              (when var-3 (setf m var-3))
              (when var-4 (setf n var-4))
              ret-val))
    (cond
     (tpsd
      (setf nb
              (max (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
```

```
                              (ret-val var-0 var-1 var-2 var-3 var-4
                               var-5 var-6)
                              (ilaenv 1 "CUNMQR" "LN" m nrhs n -1)
                            (declare (ignore var-0 var-1 var-2 var-6))
                            (when var-3 (setf m var-3))
                            (when var-4 (setf nrhs var-4))
                            (when var-5 (setf n var-5))
                            ret-val)))))
          (t
           (setf nb
                   (max (the f2cl-lib:integer4 nb)
                        (the f2cl-lib:integer4
                             (multiple-value-bind
                                 (ret-val var-0 var-1 var-2 var-3 var-4
                                  var-5 var-6)
                                 (ilaenv 1 "CUNMQR" "LC" m nrhs n -1)
                               (declare (ignore var-0 var-1 var-2 var-6))
                               (when var-3 (setf m var-3))
                               (when var-4 (setf nrhs var-4))
                               (when var-5 (setf n var-5))
                               ret-val)))))))
        (t
         (setf nb
                 (multiple-value-bind
                     (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                     (ilaenv 1 "CGELQF" " " m n -1 -1)
                   (declare (ignore var-0 var-1 var-2 var-5 var-6))
                   (when var-3 (setf m var-3))
                   (when var-4 (setf n var-4))
                   ret-val))
         (cond
          (tpsd
           (setf nb
                   (max (the f2cl-lib:integer4 nb)
                        (the f2cl-lib:integer4
                             (multiple-value-bind
                                 (ret-val var-0 var-1 var-2 var-3 var-4
                                  var-5 var-6)
                                 (ilaenv 1 "CUNMLQ" "LC" n nrhs m -1)
                               (declare (ignore var-0 var-1 var-2 var-6))
                               (when var-3 (setf n var-3))
                               (when var-4 (setf nrhs var-4))
                               (when var-5 (setf m var-5))
                               ret-val)))))
          (t
           (setf nb
                   (max (the f2cl-lib:integer4 nb)
                        (the f2cl-lib:integer4
                             (multiple-value-bind
                                 (ret-val var-0 var-1 var-2 var-3 var-4
```

```
                                var-5 var-6)
                               (ilaenv 1 "CUNMLQ" "LN" n nrhs m -1)
                             (declare (ignore var-0 var-1 var-2 var-6))
                             (when var-3 (setf n var-3))
                             (when var-4 (setf nrhs var-4))
                             (when var-5 (setf m var-5))
                             ret-val))))))))
  (setf wsize
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add mn
                                      (f2cl-lib:int-mul
                                       (max (the f2cl-lib:integer4 mn)
                                            (the f2cl-lib:integer4
                                                 nrhs))
                                       nb)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (f2cl-lib:freal wsize) 'f2cl-lib:complex8))))
(cond
 ((/= info 0) (xerbla "CGELS " (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond
 ((=
   (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)
        (the f2cl-lib:integer4 nrhs))
   0)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "Full"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       czero czero b ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldb$ var-6)))
  (go end_label)))
(setf smlnum (/ (slamch "S") (slamch "P")))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (slabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "M" m n a lda rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
```

```
          (when var-4 (setf lda var-4))
          ret-val))
(setf iascl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 anrm smlnum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 1))
 ((> anrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 anrm bignum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 2))
 ((= anrm zero)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "F"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       czero czero b ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldb$ var-6)))
  (go label50)))
(setf brow m)
(if tpsd
    (setf brow n))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "M" brow nrhs b ldb$ rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf brow var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldb$ var-4))
```

```
          ret-val))
(setf ibscl 0)
(cond
 ((and (> bnrm zero) (< bnrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 bnrm smlnum brow nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf brow var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 1))
 ((> bnrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 bnrm bignum brow nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf brow var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 2)))
(cond
 ((>= m n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (cgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (1) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ mn 1))
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-sub lwork mn) info)
    (declare (ignore var-2 var-4 var-5 var-6))
    (when var-0 (setf m var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf lda var-3))
    (when var-7 (setf info var-7)))
  (cond
   ((not tpsd)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (cunmqr "Left" "Conjugate transpose" m nrhs n a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (1)
                               ((1 *)) work-%offset%)
```

```
      b ldb$
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ mn 1))
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-sub lwork mn) info)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf n var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldb$ var-9))
    (when var-12 (setf info var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (ctrtrs "Upper" "No transpose" "Non-unit" n nrhs a lda b ldb$
       info)
    (declare (ignore var-0 var-1 var-2 var-5 var-7))
    (when var-3 (setf n var-3))
    (when var-4 (setf nrhs var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (cond ((> info 0) (go end_label))) (setf scllen n))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (ctrtrs "Upper" "Conjugate transpose" "Non-unit" n nrhs a lda b
       ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-5 var-7))
    (when var-3 (setf n var-3))
    (when var-4 (setf nrhs var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (cond ((> info 0) (go end_label)))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (f2cl-lib:fdo (i (f2cl-lib:int-add n 1) (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                czero)
        label10))
    label20))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (cunmqr "Left" "No transpose" m nrhs n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (1)
```

```
                                ((1 *)) work-%offset%)
            b ldb$
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ mn 1))
                                ((1 *)) work-%offset%)
            (f2cl-lib:int-sub lwork mn) info)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf nrhs var-3))
        (when var-4 (setf n var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldb$ var-9))
        (when var-12 (setf info var-12)))
      (setf scllen m))))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (cgelqf m n a lda
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (1) ((1 *))
                               work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ mn 1))
                               ((1 *)) work-%offset%)
          (f2cl-lib:int-sub lwork mn) info)
      (declare (ignore var-2 var-4 var-5 var-6))
      (setf m var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf info var-7))
     (cond
      ((not tpsd)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (ctrtrs "Lower" "No transpose" "Non-unit" m nrhs a lda b ldb$
           info)
        (declare (ignore var-0 var-1 var-2 var-5 var-7))
        (when var-3 (setf m var-3))
        (when var-4 (setf nrhs var-4))
        (when var-6 (setf lda var-6))
        (when var-8 (setf ldb$ var-8))
        (when var-9 (setf info var-9)))
      (cond ((> info 0) (go end_label)))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nrhs) nil)
        (tagbody
          (f2cl-lib:fdo (i (f2cl-lib:int-add m 1) (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)
                    czero)
             label30))
```

```
          label40))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (cunmlq "Left" "Conjugate transpose" n nrhs m a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (1)
                                 ((1 *)) work-%offset%)
           b ldb$
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ mn 1))
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-sub lwork mn) info)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf n var-2))
        (when var-3 (setf nrhs var-3))
        (when var-4 (setf m var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldb$ var-9))
        (when var-12 (setf info var-12)))
      (setf scllen n))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (cunmlq "Left" "No transpose" n nrhs m a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (1)
                                 ((1 *)) work-%offset%)
           b ldb$
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ mn 1))
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-sub lwork mn) info)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf n var-2))
        (when var-3 (setf nrhs var-3))
        (when var-4 (setf m var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldb$ var-9))
        (when var-12 (setf info var-12)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (ctrtrs "Lower" "Conjugate transpose" "Non-unit" m nrhs a lda b
           ldb$ info)
        (declare (ignore var-0 var-1 var-2 var-5 var-7))
        (when var-3 (setf m var-3))
        (when var-4 (setf nrhs var-4))
        (when var-6 (setf lda var-6))
        (when var-8 (setf ldb$ var-8))
        (when var-9 (setf info var-9)))
      (cond ((> info 0) (go end_label))) (setf scllen m)))))
  (cond
   ((= iascl 1)
```

```
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (clascl "G" 0 0 anrm smlnum scllen nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf smlnum var-4))
      (when var-5 (setf scllen var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9))))
   ((= iascl 2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (clascl "G" 0 0 anrm bignum scllen nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf bignum var-4))
      (when var-5 (setf scllen var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))))
  (cond
   ((= ibscl 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (clascl "G" 0 0 smlnum bnrm scllen nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf smlnum var-3))
      (when var-4 (setf bnrm var-4))
      (when var-5 (setf scllen var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9))))
   ((= ibscl 2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (clascl "G" 0 0 bignum bnrm scllen nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bignum var-3))
      (when var-4 (setf bnrm var-4))
      (when var-5 (setf scllen var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))))
label50
 (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce (f2cl-lib:freal wsize) 'f2cl-lib:complex8))
 (go end_label)
end_label
 (return (values trans m n nrhs nil lda nil ldb$ nil nil info))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgels fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$ nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::cgelqf))))
```

———-

22.2 cgelsd

22.2.1 Axiom unit tests

— cgelsd.input —

```
)set break resume
)sys rm -f cgelsd.output
)spool cgelsd.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

22.2.2 Axiom help page

— **cgelsd.help** —

```
==================================================================
cgelsd examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgelsd (M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK, WORK,
           LWORK, RWORK, IWORK, INFO)
            CGELSD computes the minimum-norm solution to a linear least
           squares problem for GE matrices

Function/Subroutine Documentation
   subroutine cgelsd (integer M, integer N, integer NRHS, complex, dimension(
       lda, * ) A, integer LDA, complex, dimension( ldb, * ) B, integer LDB,
       real, dimension( * ) S, real RCOND, integer RANK, complex, dimension( *
       ) WORK, integer LWORK, real, dimension( * ) RWORK, integer, dimension(
       * ) IWORK, integer INFO)
        CGELSD computes the minimum-norm solution to a linear least squares
       problem for GE matrices

Purpose:

   CGELSD computes the minimum-norm solution to a real linear least
   squares problem:
      minimize 2-norm(| b - A*x |)
   using the singular value decomposition (SVD) of A. A is an M-by-N
   matrix which may be rank-deficient.

   Several right hand side vectors b and solution vectors x can be
   handled in a single call; they are stored as the columns of the
   M-by-NRHS right hand side matrix B and the N-by-NRHS solution matrix X.

   The problem is solved in three steps:
   (1) Reduce the coefficient matrix A to bidiagonal form with
       Householder tranformations, reducing the original problem
```

```
      into a "bidiagonal least squares problem" (BLS)
  (2) Solve the BLS using a divide and conquer approach.
  (3) Apply back all the Householder tranformations to solve
      the original least squares problem.

  The effective rank of A is determined by treating as zero those
  singular values which are less than RCOND times the largest singular
  value.

  The divide and conquer algorithm makes very mild assumptions about
  floating point arithmetic. It will work on machines with a guard
  digit in add/subtract, or on those binary machines without guard
  digits which subtract like the Cray X-MP, Cray Y-MP, Cray C-90, or
  Cray-2. It could conceivably fail on hexadecimal or decimal machines
  without guard digits, but we know of none.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A. M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A. N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X. NRHS >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, A has been destroyed.

 LDA
   LDA is INTEGER
   The leading dimension of the array A. LDA >= max(1,M).

 B
   B is COMPLEX array, dimension (LDB,NRHS)
   On entry, the M-by-NRHS right hand side matrix B.
   On exit, B is overwritten by the N-by-NRHS solution matrix X.
   If m >= n and RANK = n, the residual sum-of-squares for
   the solution in the i-th column is given by the sum of
   squares of the modulus of elements n+1:m in that column.

 LDB
   LDB is INTEGER
```

```
  The leading dimension of the array B.  LDB >= max(1,M,N).

S
  S is REAL array, dimension (min(M,N))
  The singular values of A in decreasing order.
  The condition number of A in the 2-norm = S(1)/S(min(m,n)).

RCOND
  RCOND is REAL
  RCOND is used to determine the effective rank of A.
  Singular values S(i) <= RCOND*S(1) are treated as zero.
  If RCOND < 0, machine precision is used instead.

RANK
  RANK is INTEGER
  The effective rank of A, i.e., the number of singular values
  which are greater than RCOND*S(1).

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK must be at least 1.
  The exact minimum amount of workspace needed depends on M,
  N and NRHS. As long as LWORK is at least
     2 * N + N * NRHS
  if M is greater than or equal to N or
     2 * M + M * NRHS
  if M is less than N, the code will execute correctly.
  For good performance, LWORK should generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the array WORK and the
  minimum sizes of the arrays RWORK and IWORK, and returns
  these values as the first entries of the WORK, RWORK and
  IWORK arrays, and no error message related to LWORK is issued
  by XERBLA.

RWORK
  RWORK is REAL array, dimension (MAX(1,LRWORK))
  LRWORK >=
     10*N + 2*N*SMLSIZ + 8*N*NLVL + 3*SMLSIZ*NRHS +
     MAX( (SMLSIZ+1)**2, N*(1+NRHS) + 2*NRHS )
  if M is greater than or equal to N or
     10*M + 2*M*SMLSIZ + 8*M*NLVL + 3*SMLSIZ*NRHS +
     MAX( (SMLSIZ+1)**2, N*(1+NRHS) + 2*NRHS )
  if M is less than N, the code will execute correctly.
  SMLSIZ is returned by ILAENV and is equal to the maximum
```

```
  size of the subproblems at the bottom of the computation
  tree (usually about 25), and
     NLVL = MAX( 0, INT( LOG_2( MIN( M,N )/(SMLSIZ+1) ) ) + 1 )
  On exit, if INFO = 0, RWORK(1) returns the minimum LRWORK.

 IWORK
  IWORK is INTEGER array, dimension (MAX(1,LIWORK))
  LIWORK >= max(1, 3*MINMN*NLVL + 11*MINMN),
  where MINMN = MIN( M,N ).
  On exit, if INFO = 0, IWORK(1) returns the minimum LIWORK.

 INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value.
  > 0:  the algorithm for computing the SVD failed to converge;
        if INFO = i, i off-diagonal elements of an intermediate
        bidiagonal form did not converge to zero.
```

22.2.3 fortran code

— cgelsd.f —

```
*  =====================================================================
      SUBROUTINE CGELSD( M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK,
     $                   WORK, LWORK, RWORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDB, LWORK, M, N, NRHS, RANK
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      REAL               RWORK( * ), S( * )
      COMPLEX            A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
```

```
      REAL               ZERO, ONE, TWO
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0, TWO = 2.0E+0 )
      COMPLEX            CZERO
      PARAMETER          ( CZERO = ( 0.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            IASCL, IBSCL, IE, IL, ITAU, ITAUP, ITAUQ,
     $                   LDWORK, LIWORK, LRWORK, MAXMN, MAXWRK, MINMN,
     $                   MINWRK, MM, MNTHR, NLVL, NRWORK, NWORK, SMLSIZ
      REAL               ANRM, BIGNUM, BNRM, EPS, SFMIN, SMLNUM
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEBRD, CGELQF, CGEQRF, CLACPY,
     $                   CLALSD, CLASCL, CLASET, CUNMBR,
     $                   CUNMLQ, CUNMQR, SLABAD, SLASCL,
     $                   SLASET, XERBLA
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      REAL               CLANGE, SLAMCH
      EXTERNAL           CLANGE, SLAMCH, ILAENV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          INT, LOG, MAX, MIN, REAL
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      MINMN = MIN( M, N )
      MAXMN = MAX( M, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, MAXMN ) ) THEN
         INFO = -7
      END IF
*
*     Compute workspace.
*     (Note: Comments in the code beginning "Workspace:" describe the
*     minimal amount of workspace needed at that point in the code,
*     as well as the preferred amount for good performance.
```

```
*     NB refers to the optimal block size for the immediately
*     following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = 1
         MAXWRK = 1
         LIWORK = 1
         LRWORK = 1
         IF( MINMN.GT.0 ) THEN
            SMLSIZ = ILAENV( 9, 'CGELSD', ' ', 0, 0, 0, 0 )
            MNTHR = ILAENV( 6, 'CGELSD', ' ', M, N, NRHS, -1 )
            NLVL = MAX( INT( LOG( REAL( MINMN ) / REAL( SMLSIZ + 1 ) ) /
     $                  LOG( TWO ) ) + 1, 0 )
            LIWORK = 3*MINMN*NLVL + 11*MINMN
            MM = M
            IF( M.GE.N .AND. M.GE.MNTHR ) THEN
*
*              Path 1a - overdetermined, with many more rows than
*                        columns.
*
               MM = N
               MAXWRK = MAX( MAXWRK, N*ILAENV( 1, 'CGEQRF', ' ', M, N,
     $                       -1, -1 ) )
               MAXWRK = MAX( MAXWRK, NRHS*ILAENV( 1, 'CUNMQR', 'LC', M,
     $                       NRHS, N, -1 ) )
            END IF
            IF( M.GE.N ) THEN
*
*              Path 1 - overdetermined or exactly determined.
*
               LRWORK = 10*N + 2*N*SMLSIZ + 8*N*NLVL + 3*SMLSIZ*NRHS +
     $                  MAX( (SMLSIZ+1)**2, N*(1+NRHS) + 2*NRHS )
               MAXWRK = MAX( MAXWRK, 2*N + ( MM + N )*ILAENV( 1,
     $                       'CGEBRD', ' ', MM, N, -1, -1 ) )
               MAXWRK = MAX( MAXWRK, 2*N + NRHS*ILAENV( 1, 'CUNMBR',
     $                       'QLC', MM, NRHS, N, -1 ) )
               MAXWRK = MAX( MAXWRK, 2*N + ( N - 1 )*ILAENV( 1,
     $                       'CUNMBR', 'PLN', N, NRHS, N, -1 ) )
               MAXWRK = MAX( MAXWRK, 2*N + N*NRHS )
               MINWRK = MAX( 2*N + MM, 2*N + N*NRHS )
            END IF
            IF( N.GT.M ) THEN
               LRWORK = 10*M + 2*M*SMLSIZ + 8*M*NLVL + 3*SMLSIZ*NRHS +
     $                  MAX( (SMLSIZ+1)**2, N*(1+NRHS) + 2*NRHS )
               IF( N.GE.MNTHR ) THEN
*
*                 Path 2a - underdetermined, with many more columns
*                           than rows.
*
                  MAXWRK = M + M*ILAENV( 1, 'CGELQF', ' ', M, N, -1,
```

```
     $                     -1 )
                   MAXWRK = MAX( MAXWRK, M*M + 4*M + 2*M*ILAENV( 1,
     $                           'CGEBRD', ' ', M, M, -1, -1 ) )
                   MAXWRK = MAX( MAXWRK, M*M + 4*M + NRHS*ILAENV( 1,
     $                           'CUNMBR', 'QLC', M, NRHS, M, -1 ) )
                   MAXWRK = MAX( MAXWRK, M*M + 4*M + ( M - 1 )*ILAENV( 1,
     $                           'CUNMLQ', 'LC', N, NRHS, M, -1 ) )
                   IF( NRHS.GT.1 ) THEN
                      MAXWRK = MAX( MAXWRK, M*M + M + M*NRHS )
                   ELSE
                      MAXWRK = MAX( MAXWRK, M*M + 2*M )
                   END IF
                   MAXWRK = MAX( MAXWRK, M*M + 4*M + M*NRHS )
!     XXX: Ensure the Path 2a case below is triggered.  The workspace
!     calculation should use queries for all routines eventually.
                   MAXWRK = MAX( MAXWRK,
     $                 4*M+M*M+MAX( M, 2*M-4, NRHS, N-3*M ) )
               ELSE
*
*                  Path 2 - underdetermined.
*
                   MAXWRK = 2*M + ( N + M )*ILAENV( 1, 'CGEBRD', ' ', M,
     $                     N, -1, -1 )
                   MAXWRK = MAX( MAXWRK, 2*M + NRHS*ILAENV( 1, 'CUNMBR',
     $                           'QLC', M, NRHS, M, -1 ) )
                   MAXWRK = MAX( MAXWRK, 2*M + M*ILAENV( 1, 'CUNMBR',
     $                           'PLN', N, NRHS, M, -1 ) )
                   MAXWRK = MAX( MAXWRK, 2*M + M*NRHS )
               END IF
               MINWRK = MAX( 2*M + N, 2*M + M*NRHS )
            END IF
         END IF
         MINWRK = MIN( MINWRK, MAXWRK )
         WORK( 1 ) = MAXWRK
         IWORK( 1 ) = LIWORK
         RWORK( 1 ) = LRWORK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -12
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGELSD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible.
```

```
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         RANK = 0
         RETURN
      END IF
*
*     Get machine parameters.
*
      EPS = SLAMCH( 'P' )
      SFMIN = SLAMCH( 'S' )
      SMLNUM = SFMIN / EPS
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
*
*     Scale A if max entry outside range [SMLNUM,BIGNUM].
*
      ANRM = CLANGE( 'M', M, N, A, LDA, RWORK )
      IASCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL CLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
         IASCL = 1
      ELSE IF( ANRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM.
*
         CALL CLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
         IASCL = 2
      ELSE IF( ANRM.EQ.ZERO ) THEN
*
*        Matrix all zero. Return zero solution.
*
         CALL CLASET( 'F', MAX( M, N ), NRHS, CZERO, CZERO, B, LDB )
         CALL SLASET( 'F', MINMN, 1, ZERO, ZERO, S, 1 )
         RANK = 0
         GO TO 10
      END IF
*
*     Scale B if max entry outside range [SMLNUM,BIGNUM].
*
      BNRM = CLANGE( 'M', M, NRHS, B, LDB, RWORK )
      IBSCL = 0
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM.
*
         CALL CLASCL( 'G', 0, 0, BNRM, SMLNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 1
```

```
      ELSE IF( BNRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM.
*
         CALL CLASCL( 'G', 0, 0, BNRM, BIGNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 2
      END IF
*
*     If M < N make sure B(M+1:N,:) = 0
*
      IF( M.LT.N )
     $   CALL CLASET( 'F', N-M, NRHS, CZERO, CZERO, B( M+1, 1 ), LDB )
*
*     Overdetermined case.
*
      IF( M.GE.N ) THEN
*
*        Path 1 - overdetermined or exactly determined.
*
         MM = M
         IF( M.GE.MNTHR ) THEN
*
*           Path 1a - overdetermined, with many more rows than columns
*
            MM = N
            ITAU = 1
            NWORK = ITAU + N
*
*           Compute A=Q*R.
*           (RWorkspace: need N)
*           (CWorkspace: need N, prefer N*NB)
*
            CALL CGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                   LWORK-NWORK+1, INFO )
*
*           Multiply B by transpose(Q).
*           (RWorkspace: need N)
*           (CWorkspace: need NRHS, prefer NRHS*NB)
*
            CALL CUNMQR( 'L', 'C', M, NRHS, N, A, LDA, WORK( ITAU ), B,
     $                   LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
*           Zero out below R.
*
            IF( N.GT.1 ) THEN
               CALL CLASET( 'L', N-1, N-1, CZERO, CZERO, A( 2, 1 ),
     $                      LDA )
            END IF
         END IF
*
```

```
         ITAUQ = 1
         ITAUP = ITAUQ + N
         NWORK = ITAUP + N
         IE = 1
         NRWORK = IE + N
*
*        Bidiagonalize R in A.
*        (RWorkspace: need N)
*        (CWorkspace: need 2*N+MM, prefer 2*N+(MM+N)*NB)
*
         CALL CGEBRD( MM, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors of R.
*        (CWorkspace: need 2*N+NRHS, prefer 2*N+NRHS*NB)
*
         CALL CUNMBR( 'Q', 'L', 'C', MM, NRHS, N, A, LDA, WORK( ITAUQ ),
     $                B, LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
*        Solve the bidiagonal least squares problem.
*
         CALL CLALSD( 'U', SMLSIZ, N, NRHS, S, RWORK( IE ), B, LDB,
     $                RCOND, RANK, WORK( NWORK ), RWORK( NRWORK ),
     $                IWORK, INFO )
         IF( INFO.NE.0 ) THEN
            GO TO 10
         END IF
*
*        Multiply B by right bidiagonalizing vectors of R.
*
         CALL CUNMBR( 'P', 'L', 'N', N, NRHS, N, A, LDA, WORK( ITAUP ),
     $                B, LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
      ELSE IF( N.GE.MNTHR .AND. LWORK.GE.4*M+M*M+
     $         MAX( M, 2*M-4, NRHS, N-3*M ) ) THEN
*
*        Path 2a - underdetermined, with many more columns than rows
*        and sufficient workspace for an efficient algorithm.
*
         LDWORK = M
         IF( LWORK.GE.MAX( 4*M+M*LDA+MAX( M, 2*M-4, NRHS, N-3*M ),
     $       M*LDA+M+M*NRHS ) )LDWORK = LDA
         ITAU = 1
         NWORK = M + 1
*
*        Compute A=L*Q.
*        (CWorkspace: need 2*M, prefer M+M*NB)
*
         CALL CGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
```

```
     $                LWORK-NWORK+1, INFO )
         IL = NWORK
*
*        Copy L to WORK(IL), zeroing out above its diagonal.
*
         CALL CLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWORK )
         CALL CLASET( 'U', M-1, M-1, CZERO, CZERO, WORK( IL+LDWORK ),
     $                LDWORK )
         ITAUQ = IL + LDWORK*M
         ITAUP = ITAUQ + M
         NWORK = ITAUP + M
         IE = 1
         NRWORK = IE + M
*
*        Bidiagonalize L in WORK(IL).
*        (RWorkspace: need M)
*        (CWorkspace: need M*M+4*M, prefer M*M+4*M+2*M*NB)
*
         CALL CGEBRD( M, M, WORK( IL ), LDWORK, S, RWORK( IE ),
     $                WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                LWORK-NWORK+1, INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors of L.
*        (CWorkspace: need M*M+4*M+NRHS, prefer M*M+4*M+NRHS*NB)
*
         CALL CUNMBR( 'Q', 'L', 'C', M, NRHS, M, WORK( IL ), LDWORK,
     $                WORK( ITAUQ ), B, LDB, WORK( NWORK ),
     $                LWORK-NWORK+1, INFO )
*
*        Solve the bidiagonal least squares problem.
*
         CALL CLALSD( 'U', SMLSIZ, M, NRHS, S, RWORK( IE ), B, LDB,
     $                RCOND, RANK, WORK( NWORK ), RWORK( NRWORK ),
     $                IWORK, INFO )
         IF( INFO.NE.0 ) THEN
            GO TO 10
         END IF
*
*        Multiply B by right bidiagonalizing vectors of L.
*
         CALL CUNMBR( 'P', 'L', 'N', M, NRHS, M, WORK( IL ), LDWORK,
     $                WORK( ITAUP ), B, LDB, WORK( NWORK ),
     $                LWORK-NWORK+1, INFO )
*
*        Zero out below first M rows of B.
*
         CALL CLASET( 'F', N-M, NRHS, CZERO, CZERO, B( M+1, 1 ), LDB )
         NWORK = ITAU + M
*
*        Multiply transpose(Q) by B.
```

```
*        (CWorkspace: need NRHS, prefer NRHS*NB)
*
         CALL CUNMLQ( 'L', 'C', N, NRHS, M, A, LDA, WORK( ITAU ), B,
     $                LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
      ELSE
*
*        Path 2 - remaining underdetermined cases.
*
         ITAUQ = 1
         ITAUP = ITAUQ + M
         NWORK = ITAUP + M
         IE = 1
         NRWORK = IE + M
*
*        Bidiagonalize A.
*        (RWorkspace: need M)
*        (CWorkspace: need 2*M+N, prefer 2*M+(M+N)*NB)
*
         CALL CGEBRD( M, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors.
*        (CWorkspace: need 2*M+NRHS, prefer 2*M+NRHS*NB)
*
         CALL CUNMBR( 'Q', 'L', 'C', M, NRHS, N, A, LDA, WORK( ITAUQ ),
     $                B, LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
*        Solve the bidiagonal least squares problem.
*
         CALL CLALSD( 'L', SMLSIZ, M, NRHS, S, RWORK( IE ), B, LDB,
     $                RCOND, RANK, WORK( NWORK ), RWORK( NRWORK ),
     $                IWORK, INFO )
         IF( INFO.NE.0 ) THEN
            GO TO 10
         END IF
*
*        Multiply B by right bidiagonalizing vectors of A.
*
         CALL CUNMBR( 'P', 'L', 'N', N, NRHS, M, A, LDA, WORK( ITAUP ),
     $                B, LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
      END IF
*
*     Undo scaling.
*
      IF( IASCL.EQ.1 ) THEN
         CALL CLASCL( 'G', 0, 0, ANRM, SMLNUM, N, NRHS, B, LDB, INFO )
         CALL SLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
```

```
     $                INFO )
      ELSE IF( IASCL.EQ.2 ) THEN
         CALL CLASCL( 'G', 0, 0, ANRM, BIGNUM, N, NRHS, B, LDB, INFO )
         CALL SLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                INFO )
      END IF
      IF( IBSCL.EQ.1 ) THEN
         CALL CLASCL( 'G', 0, 0, SMLNUM, BNRM, N, NRHS, B, LDB, INFO )
      ELSE IF( IBSCL.EQ.2 ) THEN
         CALL CLASCL( 'G', 0, 0, BIGNUM, BNRM, N, NRHS, B, LDB, INFO )
      END IF
*
   10 CONTINUE
      WORK( 1 ) = MAXWRK
      IWORK( 1 ) = LIWORK
      RWORK( 1 ) = LRWORK
      RETURN
*
*     End of CGELSD
*
      END
```

———-

22.2.4 lisp code

— cgelsd.lisp —

```
(let* ((zero 0.0) (one 1.0) (two 2.0) (czero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (f2cl-lib:complex8) czero)
           (ignorable zero one two czero))
  (defun cgelsd
         (m n nrhs a lda b ldb$ s rcond rank work lwork rwork iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (single-float) rcond)
             (type (array single-float (*)) rwork s)
             (type (array f2cl-lib:complex8 (*)) work b a)
             (type (f2cl-lib:integer4) info lwork rank ldb$ lda nrhs n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (s single-float s-%data% s-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
```

```
(prog ((anrm 0.0) (bignum 0.0) (bnrm 0.0) (eps 0.0) (sfmin 0.0)
       (smlnum 0.0) (iascl 0) (ibscl 0) (ie 0) (il 0) (itau 0) (itaup 0)
       (itauq 0) (ldwork 0) (liwork 0) (lrwork 0) (maxmn 0) (maxwrk 0)
       (minmn 0) (minwrk 0) (mm 0) (mnthr 0) (nlvl 0) (nrwork 0)
       (nwork 0) (smlsiz 0) (lquery nil))
  (declare (type (single-float) anrm bignum bnrm eps sfmin smlnum)
           (type (f2cl-lib:integer4) iascl ibscl ie il itau itaup itauq
            ldwork liwork lrwork maxmn maxwrk minmn minwrk mm mnthr nlvl
            nrwork nwork smlsiz)
           (type f2cl-lib:logical lquery))
  (setf info 0)
  (setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
  (setf maxmn (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
  (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
  (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
        ((< nrhs 0) (setf info -3))
        ((< lda
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -5))
        ((< ldb$
            (max (the f2cl-lib:integer4 1)
                 (the f2cl-lib:integer4 maxmn)))
         (setf info -7)))
  (cond
   ((= info 0) (setf minwrk 1) (setf maxwrk 1) (setf liwork 1)
    (setf lrwork 1)
    (cond
     ((> minmn 0) (setf smlsiz (ilaenv 9 "CGELSD" " " 0 0 0 0))
      (setf mnthr
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 6 "CGELSD" " " m n nrhs -1)
                (declare (ignore var-0 var-1 var-2 var-6))
                (when var-3 (setf m var-3))
                (when var-4 (setf n var-4))
                (when var-5 (setf nrhs var-5))
                ret-val))
      (setf nlvl
              (max
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add
                     (f2cl-lib:int
                      (/
                       (f2cl-lib:flog
                        (/ (f2cl-lib:freal minmn)
                           (f2cl-lib:freal (f2cl-lib:int-add smlsiz 1))))
                       (f2cl-lib:flog two)))
                     1))
               (the f2cl-lib:integer4 0)))
      (setf liwork
```

```
        (f2cl-lib:int-add (f2cl-lib:int-mul 3 minmn nlvl)
                          (f2cl-lib:int-mul 11 minmn)))
(setf mm m)
(cond
 ((and (>= m n) (>= m mnthr)) (setf mm n)
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-mul n
                                      (multiple-value-bind
                                          (ret-val var-0 var-1
                                           var-2 var-3 var-4 var-5
                                           var-6)
                                          (ilaenv 1 "CGEQRF" " " m
                                           n -1 -1)
                                        (declare
                                         (ignore var-0 var-1 var-2
                                          var-5 var-6))
                                        (when var-3 (setf m var-3))
                                        (when var-4 (setf n var-4))
                                        ret-val)))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-mul nrhs
                                      (multiple-value-bind
                                          (ret-val var-0 var-1
                                           var-2 var-3 var-4 var-5
                                           var-6)
                                          (ilaenv 1 "CUNMQR" "LC" m
                                           nrhs n -1)
                                        (declare
                                         (ignore var-0 var-1 var-2
                                          var-6))
                                        (when var-3 (setf m var-3))
                                        (when var-4
                                          (setf nrhs var-4))
                                        (when var-5 (setf n var-5))
                                        ret-val)))))))
(cond
 ((>= m n)
  (setf lrwork
          (f2cl-lib:int-add (f2cl-lib:int-mul 10 n)
                            (f2cl-lib:int-mul 2 n smlsiz)
                            (f2cl-lib:int-mul 8 n nlvl)
                            (f2cl-lib:int-mul 3 smlsiz nrhs)
                            (max
                             (the f2cl-lib:integer4
                                  (expt (f2cl-lib:int-add smlsiz 1)
                                        2))
```

```
                                (the f2cl-lib:integer4
                                     (f2cl-lib:int-add
                                      (f2cl-lib:int-mul n
                                                        (f2cl-lib:int-add
                                                         1 nrhs))
                                      (f2cl-lib:int-mul 2 nrhs))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul
                                     (f2cl-lib:int-add mm n)
                                     (multiple-value-bind
                                         (ret-val var-0 var-1
                                          var-2 var-3 var-4 var-5
                                          var-6)
                                         (ilaenv 1 "CGEBRD" " "
                                          mm n -1 -1)
                                       (declare
                                        (ignore var-0 var-1 var-2
                                        var-5 var-6))
                                       (when var-3
                                         (setf mm var-3))
                                       (when var-4
                                         (setf n var-4))
                                       ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul nrhs
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "CUNMBR"
                                                           "QLC"
                                                           mm
                                                           nrhs n
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
```

```
                                                                var-1
                                                                var-2
                                                                var-6))
                                                              (when
                                                                  var-3
                                                                (setf mm
                                                                        var-3))
                                                              (when
                                                                  var-4
                                                                (setf nrhs
                                                                        var-4))
                                                              (when
                                                                  var-5
                                                                (setf n
                                                                        var-5))
                                                              ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul
                                     (f2cl-lib:int-sub n 1)
                                     (multiple-value-bind
                                         (ret-val var-0 var-1
                                          var-2 var-3 var-4 var-5
                                          var-6)
                                         (ilaenv 1 "CUNMBR" "PLN"
                                          n nrhs n -1)
                                       (declare
                                        (ignore var-0 var-1 var-2
                                         var-6))
                                       (when var-3
                                         (setf n var-3))
                                       (when var-4
                                         (setf nrhs var-4))
                                       (when var-5
                                         (setf n var-5))
                                       ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul n nrhs)))))
(setf minwrk
        (max
         (the f2cl-lib:integer4
              (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) mm))
         (the f2cl-lib:integer4
              (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                (f2cl-lib:int-mul n nrhs)))))))
```

```
(cond
 ((> n m)
  (setf lrwork
          (f2cl-lib:int-add (f2cl-lib:int-mul 10 m)
                            (f2cl-lib:int-mul 2 m smlsiz)
                            (f2cl-lib:int-mul 8 m nlvl)
                            (f2cl-lib:int-mul 3 smlsiz nrhs)
                            (max
                             (the f2cl-lib:integer4
                                  (expt (f2cl-lib:int-add smlsiz 1)
                                        2))
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-mul n
                                                     (f2cl-lib:int-add
                                                      1 nrhs))
                                   (f2cl-lib:int-mul 2 nrhs))))))
  (cond
   ((>= n mnthr)
    (setf maxwrk
            (f2cl-lib:int-add m
                              (f2cl-lib:int-mul m
                                                (multiple-value-bind
                                                    (ret-val var-0
                                                     var-1 var-2
                                                     var-3 var-4
                                                     var-5 var-6)
                                                    (ilaenv 1
                                                     "CGELQF" " " m
                                                     n -1 -1)
                                                  (declare
                                                   (ignore var-0
                                                    var-1 var-2
                                                    var-5 var-6))
                                                  (when var-3
                                                    (setf m var-3))
                                                  (when var-4
                                                    (setf n var-4))
                                                  ret-val))))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                        (f2cl-lib:int-mul 4 m)
                                        (f2cl-lib:int-mul 2 m
                                                          (multiple-value-bind
                                                              (ret-val
                                                               var-0
                                                               var-1
                                                               var-2
```

```
                                                                    var-3
                                                                    var-4
                                                                    var-5
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "CGEBRD"
                                                                    " "
                                                                    m m
                                                                    -1
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-5
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf m
                                                                            var-4))
                                                                  ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                    (f2cl-lib:int-mul 4 m)
                                    (f2cl-lib:int-mul nrhs
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "CUNMBR"
                                                           "QLC"
                                                           m
                                                           nrhs
                                                           m
                                                           -1)
```

```
                                                                (declare
                                                                 (ignore
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-6))
                                                                (when
                                                                    var-3
                                                                  (setf m
                                                                          var-3))
                                                                (when
                                                                    var-4
                                                                  (setf nrhs
                                                                          var-4))
                                                                (when
                                                                    var-5
                                                                  (setf m
                                                                          var-5))
                                                                ret-val))))))
                (setf maxwrk
                        (max (the f2cl-lib:integer4 maxwrk)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                                    (f2cl-lib:int-mul 4 m)
                                                    (f2cl-lib:int-mul
                                                     (f2cl-lib:int-sub m 1)
                                                     (multiple-value-bind
                                                         (ret-val var-0 var-1
                                                          var-2 var-3 var-4
                                                          var-5 var-6)
                                                         (ilaenv 1 "CUNMLQ"
                                                          "LC" n nrhs m -1)
                                                       (declare
                                                        (ignore var-0 var-1
                                                        var-2 var-6))
                                                       (when var-3
                                                         (setf n var-3))
                                                       (when var-4
                                                         (setf nrhs var-4))
                                                       (when var-5
                                                         (setf m var-5))
                                                       ret-val))))))
                (cond
                 ((> nrhs 1)
                  (setf maxwrk
                          (max (the f2cl-lib:integer4 maxwrk)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add (f2cl-lib:int-mul m m) m
                                                      (f2cl-lib:int-mul m
                                                                        nrhs))))))
```

```
 (t
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                      (f2cl-lib:int-mul 2
                                                        m)))))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                     (f2cl-lib:int-mul 4 m)
                                     (f2cl-lib:int-mul m
                                                       nrhs)))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 4 m)
                                     (f2cl-lib:int-mul m m)
                                     (max
                                      (the f2cl-lib:integer4 m)
                                      (the f2cl-lib:integer4
                                           (f2cl-lib:int-sub
                                            (f2cl-lib:int-mul 2
                                                              m)
                                            4))
                                      (the f2cl-lib:integer4
                                           nrhs)
                                      (the f2cl-lib:integer4
                                           (f2cl-lib:int-sub n
                                                             (f2cl-lib:int-mul
                                                              3
                                                              m)))))))))
(t
 (setf maxwrk
         (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-add n m)
                            (multiple-value-bind
                                (ret-val var-0 var-1 var-2 var-3
                                 var-4 var-5 var-6)
                                (ilaenv 1 "CGEBRD" " " m n -1
                                 -1)
                              (declare
                               (ignore var-0 var-1 var-2 var-5
                                var-6))
                              (when var-3 (setf m var-3))
                              (when var-4 (setf n var-4))
                              ret-val))))
 (setf maxwrk
```

```
                    (max (the f2cl-lib:integer4 maxwrk)
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                (f2cl-lib:int-mul nrhs
                                                                  (multiple-value-bind
                                                                      (ret-val
                                                                       var-0
                                                                       var-1
                                                                       var-2
                                                                       var-3
                                                                       var-4
                                                                       var-5
                                                                       var-6)
                                                                      (ilaenv
                                                                       1
                                                                       "CUNMBR"
                                                                       "QLC"
                                                                       m
                                                                       nrhs
                                                                       m
                                                                       -1)
                                                                    (declare
                                                                     (ignore
                                                                      var-0
                                                                      var-1
                                                                      var-2
                                                                      var-6))
                                                                    (when
                                                                        var-3
                                                                      (setf m
                                                                              var-3))
                                                                    (when
                                                                        var-4
                                                                      (setf nrhs
                                                                              var-4))
                                                                    (when
                                                                        var-5
                                                                      (setf m
                                                                              var-5))
                                                                    ret-val))))))
            (setf maxwrk
                    (max (the f2cl-lib:integer4 maxwrk)
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                (f2cl-lib:int-mul m
                                                                  (multiple-value-bind
                                                                      (ret-val
                                                                       var-0
                                                                       var-1
                                                                       var-2
```

```
                                                                var-3
                                                                var-4
                                                                var-5
                                                                var-6)
                                                               (ilaenv
                                                                1
                                                                "CUNMBR"
                                                                "PLN"
                                                                n
                                                                nrhs
                                                                m
                                                                -1)
                                                             (declare
                                                              (ignore
                                                               var-0
                                                               var-1
                                                               var-2
                                                               var-6))
                                                             (when
                                                                 var-3
                                                               (setf n
                                                                       var-3))
                                                             (when
                                                                 var-4
                                                               (setf nrhs
                                                                       var-4))
                                                             (when
                                                                 var-5
                                                               (setf m
                                                                       var-5))
                                                             ret-val))))))
      (setf maxwrk
              (max (the f2cl-lib:integer4 maxwrk)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                          (f2cl-lib:int-mul m
                                                            nrhs)))))))
    (setf minwrk
            (max
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n))
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    (f2cl-lib:int-mul m nrhs)))))))))
(setf minwrk
        (min (the f2cl-lib:integer4 minwrk)
             (the f2cl-lib:integer4 maxwrk)))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce maxwrk 'f2cl-lib:complex8))
(setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) liwork)
```

```
  (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
          (coerce (the f2cl-lib:integer4 lrwork) 'single-float))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -12)))))
(cond
 ((/= info 0) (xerbla "CGELSD" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond ((or (= m 0) (= n 0)) (setf rank 0) (go end_label)))
(setf eps (slamch "P"))
(setf sfmin (slamch "S"))
(setf smlnum (/ sfmin eps))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (slabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "M" m n a lda rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iascl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 anrm smlnum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 1))
 ((> anrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 anrm bignum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
```

```
  (setf iascl 2))
 ((= anrm zero)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "F"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       czero czero b ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldb$ var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "F" minmn 1 zero zero s 1)
    (declare (ignore var-0 var-2 var-5 var-6))
    (when var-1 (setf minmn var-1))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4)))
  (setf rank 0) (go label10)))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "M" m nrhs b ldb$ rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ibscl 0)
(cond
 ((and (> bnrm zero) (< bnrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 bnrm smlnum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 1))
 ((> bnrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 bnrm bignum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
```

```
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 2)))
(if (< m n)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "F" (f2cl-lib:int-sub n m) nrhs czero czero
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 ((+ m 1) 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf nrhs var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldb$ var-6))))
(cond
 ((>= m n) (setf mm m)
  (cond
   ((>= m mnthr) (setf mm n) (setf itau 1)
    (setf nwork (f2cl-lib:int-add itau n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (cgeqrf m n a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
      (declare (ignore var-2 var-4 var-5 var-6))
      (when var-0 (setf m var-0))
      (when var-1 (setf n var-1))
      (when var-3 (setf lda var-3))
      (when var-7 (setf info var-7)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (cunmqr "L" "C" m nrhs n a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                               ((1 *)) work-%offset%)
         b ldb$
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf nrhs var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldb$ var-9))
      (when var-12 (setf info var-12)))
    (cond
```

```
   ((> n 1)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
         czero czero
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (2 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf lda var-6)))))))
(setf itauq 1) (setf itaup (f2cl-lib:int-add itauq n))
(setf nwork (f2cl-lib:int-add itaup n)) (setf ie 1)
(setf nrwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgebrd mm n a lda s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf mm var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "Q" "L" "C" mm nrhs n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf mm var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (clalsd "U" smlsiz n nrhs s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     b ldb$ rcond rank
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork) ((1 *))
                           rwork-%offset%)
     iwork info)
  (declare (ignore var-0 var-4 var-5 var-6 var-10 var-11 var-12))
  (when var-1 (setf smlsiz var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf nrhs var-3))
  (when var-7 (setf ldb$ var-7))
  (when var-8 (setf rcond var-8))
  (when var-9 (setf rank var-9))
  (when var-13 (setf info var-13)))
(cond ((/= info 0) (go label10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "P" "L" "N" n nrhs n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13))))
((and (>= n mnthr)
      (>= lwork
          (f2cl-lib:int-add (f2cl-lib:int-mul 4 m)
                            (f2cl-lib:int-mul m m)
                            (max (the f2cl-lib:integer4 m)
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add
                                       (f2cl-lib:int-mul 2 m)
                                       (f2cl-lib:int-sub 4)))
                                 (the f2cl-lib:integer4 nrhs)
                                 (the f2cl-lib:integer4
```

```
                                        (f2cl-lib:int-add n
                                                          (f2cl-lib:int-mul
                                                           -1 3 m)))))))
(setf ldwork m)
(if (>= lwork
        (max
         (the f2cl-lib:integer4
              (f2cl-lib:int-add (f2cl-lib:int-mul 4 m)
                                (f2cl-lib:int-mul m lda)
                                (max (the f2cl-lib:integer4 m)
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-sub
                                           (f2cl-lib:int-mul 2 m) 4))
                                     (the f2cl-lib:integer4 nrhs)
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-sub n
                                                            (f2cl-lib:int-mul
                                                             3
                                                             m))))))
         (the f2cl-lib:integer4
              (f2cl-lib:int-add (f2cl-lib:int-mul m lda) m
                                (f2cl-lib:int-mul m nrhs)))))
    (setf ldwork lda))
(setf itau 1) (setf nwork (f2cl-lib:int-add m 1))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgelqf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-7))
(setf il nwork)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il) ((1 *))
                           work-%offset%)
     ldwork)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwork var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1) czero
```

```
      czero
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                            ((+ il ldwork)) ((1 *)) work-%offset%)
      ldwork)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwork var-6)))
(setf itauq (f2cl-lib:int-add il (f2cl-lib:int-mul ldwork m)))
(setf itaup (f2cl-lib:int-add itauq m))
(setf nwork (f2cl-lib:int-add itaup m)) (setf ie 1)
(setf nrwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il) ((1 *))
                           work-%offset%)
     ldwork s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwork var-3)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "Q" "L" "C" m nrhs m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il) ((1 *))
                           work-%offset%)
     ldwork
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
```

```
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwork var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (clalsd "U" smlsiz m nrhs s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     b ldb$ rcond rank
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork) ((1 *))
                           rwork-%offset%)
     iwork info)
  (declare (ignore var-0 var-4 var-5 var-6 var-10 var-11 var-12))
  (when var-1 (setf smlsiz var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf nrhs var-3))
  (when var-7 (setf ldb$ var-7))
  (when var-8 (setf rcond var-8))
  (when var-9 (setf rank var-9))
  (when var-13 (setf info var-13)))
(cond ((/= info 0) (go label10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "P" "L" "N" m nrhs m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il) ((1 *))
                           work-%offset%)
     ldwork
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwork var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "F" (f2cl-lib:int-sub n m) nrhs czero czero
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 ((+ m 1) 1)
                           ((1 ldb$) (1 *)) b-%offset%)
```

```
        ldb$)
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf nrhs var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldb$ var-6)))
    (setf nwork (f2cl-lib:int-add itau m))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (cunmlq "L" "C" n nrhs m a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                               ((1 *)) work-%offset%)
         b ldb$
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf n var-2))
      (when var-3 (setf nrhs var-3))
      (when var-4 (setf m var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldb$ var-9))
      (when var-12 (setf info var-12))))
  (t (setf itauq 1) (setf itaup (f2cl-lib:int-add itauq m))
    (setf nwork (f2cl-lib:int-add itaup m)) (setf ie 1)
    (setf nrwork (f2cl-lib:int-add ie m))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (cgebrd m n a lda s
         (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                               rwork-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
      (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
      (setf m var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf info var-10))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (cunmbr "Q" "L" "C" m nrhs n a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
```

```
                              ((1 *)) work-%offset%)
      b ldb$
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                              ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (clalsd "L" smlsiz m nrhs s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                              rwork-%offset%)
     b ldb$ rcond rank
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                              ((1 *)) work-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork) ((1 *))
                              rwork-%offset%)
     iwork info)
  (declare (ignore var-0 var-4 var-5 var-6 var-10 var-11 var-12))
  (when var-1 (setf smlsiz var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf nrhs var-3))
  (when var-7 (setf ldb$ var-7))
  (when var-8 (setf rcond var-8))
  (when var-9 (setf rank var-9))
  (when var-13 (setf info var-13)))
(cond ((/= info 0) (go label10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "P" "L" "N" n nrhs m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                              ((1 *)) work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                              ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
```

```
      (when var-10 (setf ldb$ var-10))
      (when var-13 (setf info var-13)))))
    (cond
     ((= iascl 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (clascl "G" 0 0 anrm smlnum n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf smlnum var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (slascl "G" 0 0 smlnum anrm minmn 1 s minmn info)
      (declare (ignore var-0 var-1 var-2 var-6 var-7))
      (when var-3 (setf smlnum var-3))
      (when var-4 (setf anrm var-4))
      (when var-5 (setf minmn var-5))
      (when var-8 (setf minmn var-8))
      (when var-9 (setf info var-9))))
     ((= iascl 2)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (clascl "G" 0 0 anrm bignum n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf bignum var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (slascl "G" 0 0 bignum anrm minmn 1 s minmn info)
      (declare (ignore var-0 var-1 var-2 var-6 var-7))
      (when var-3 (setf bignum var-3))
      (when var-4 (setf anrm var-4))
      (when var-5 (setf minmn var-5))
      (when var-8 (setf minmn var-8))
      (when var-9 (setf info var-9)))))
    (cond
     ((= ibscl 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (clascl "G" 0 0 smlnum bnrm n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf smlnum var-3))
```

```
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf n var-5))
            (when var-6 (setf nrhs var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf info var-9))))
          ((= ibscl 2)
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
               (clascl "G" 0 0 bignum bnrm n nrhs b ldb$ info)
            (declare (ignore var-0 var-1 var-2 var-7))
            (when var-3 (setf bignum var-3))
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf n var-5))
            (when var-6 (setf nrhs var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf info var-9)))))
        label10
         (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                 (coerce maxwrk 'f2cl-lib:complex8))
         (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) liwork)
         (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
                 (coerce (the f2cl-lib:integer4 lrwork) 'single-float))
         (go end_label)
        end_label
         (return
          (values m n nrhs nil lda nil ldb$ nil rcond rank nil nil nil nil
                  info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgelsd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (single-float)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
```

```
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::m
  fortran-to-lisp::n
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$ nil
  fortran-to-lisp::rcond
  fortran-to-lisp::rank nil nil nil
  nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::cgelqf
  fortran-to-lisp::cgebrd))))
```

———-

22.3 cgelss

22.3.1 Axiom unit tests

— **cgelss.input** —

```
)set break resume
)sys rm -f cgelss.output
)spool cgelss.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

22.3.2 Axiom help page

— **cgelss.help** —

```
=====================================================================
cgelss examples
```

```
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgelss (M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK, WORK,
           LWORK, RWORK, INFO)
            CGELSS solves overdetermined or underdetermined systems for GE
           matrices

Function/Subroutine Documentation
   subroutine cgelss (integer M, integer N, integer NRHS, complex, dimension(
       lda, * ) A, integer LDA, complex, dimension( ldb, * ) B, integer LDB,
       real, dimension( * ) S, real RCOND, integer RANK, complex, dimension( *
       ) WORK, integer LWORK, real, dimension( * ) RWORK, integer INFO)
        CGELSS solves overdetermined or underdetermined systems for GE
       matrices

Purpose:

   CGELSS computes the minimum norm solution to a complex linear
   least squares problem:

   Minimize 2-norm(| b - A*x |).

   using the singular value decomposition (SVD) of A. A is an M-by-N
   matrix which may be rank-deficient.

   Several right hand side vectors b and solution vectors x can be
   handled in a single call; they are stored as the columns of the
   M-by-NRHS right hand side matrix B and the N-by-NRHS solution matrix X.

   The effective rank of A is determined by treating as zero those
   singular values which are less than RCOND times the largest singular value.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A. M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A. N >= 0.

 NRHS
   NRHS is INTEGER
```

```
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X. NRHS >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit, the first min(m,n) rows of A are overwritten with
  its right singular vectors, stored rowwise.

LDA
  LDA is INTEGER
  The leading dimension of the array A. LDA >= max(1,M).

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the M-by-NRHS right hand side matrix B.
  On exit, B is overwritten by the N-by-NRHS solution matrix X.
  If m >= n and RANK = n, the residual sum-of-squares for
  the solution in the i-th column is given by the sum of
  squares of the modulus of elements n+1:m in that column.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,M,N).

S
  S is REAL array, dimension (min(M,N))
  The singular values of A in decreasing order.
  The condition number of A in the 2-norm = S(1)/S(min(m,n)).

RCOND
  RCOND is REAL
  RCOND is used to determine the effective rank of A.
  Singular values S(i) <= RCOND*S(1) are treated as zero.
  If RCOND < 0, machine precision is used instead.

RANK
  RANK is INTEGER
  The effective rank of A, i.e., the number of singular values
  which are greater than RCOND*S(1).

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK >= 1, and also:
  LWORK >=  2*min(M,N) + max(M,N,NRHS)
  For good performance, LWORK should generally be larger.
```

```
  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is REAL array, dimension (5*min(M,N))

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  the algorithm for computing the SVD failed to converge;
        if INFO = i, i off-diagonal elements of an intermediate
        bidiagonal form did not converge to zero.
```

22.3.3 fortran code

— cgelss.f —

```
*  =====================================================================
      SUBROUTINE CGELSS( M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK,
     $                   WORK, LWORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDB, LWORK, M, N, NRHS, RANK
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      REAL               RWORK( * ), S( * )
      COMPLEX            A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      COMPLEX            CZERO, CONE
```

```
      PARAMETER          ( CZERO = ( 0.0E+0, 0.0E+0 ),
     $                   CONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            BL, CHUNK, I, IASCL, IBSCL, IE, IL, IRWORK,
     $                   ITAU, ITAUP, ITAUQ, IWORK, LDWORK, MAXMN,
     $                   MAXWRK, MINMN, MINWRK, MM, MNTHR
      INTEGER            LWORK_CGEQRF, LWORK_CUNMQR, LWORK_CGEBRD,
     $                   LWORK_CUNMBR, LWORK_CUNGBR, LWORK_CUNMLQ,
     $                   LWORK_CGELQF
      REAL               ANRM, BIGNUM, BNRM, EPS, SFMIN, SMLNUM, THR
*     ..
*     .. Local Arrays ..
      COMPLEX            DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           CBDSQR, CCOPY, CGEBRD, CGELQF, CGEMM, CGEMV,
     $                   CGEQRF, CLACPY, CLASCL, CLASET, CSRSCL, CUNGBR,
     $                   CUNMBR, CUNMLQ, CUNMQR, SLABAD, SLASCL, SLASET,
     $                   XERBLA
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      REAL               CLANGE, SLAMCH
      EXTERNAL           ILAENV, CLANGE, SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      MINMN = MIN( M, N )
      MAXMN = MAX( M, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, MAXMN ) ) THEN
         INFO = -7
      END IF
*
```

```
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       CWorkspace refers to complex workspace, and RWorkspace refers
*       to real workspace. NB refers to the optimal block size for the
*       immediately following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = 1
         MAXWRK = 1
         IF( MINMN.GT.0 ) THEN
            MM = M
            MNTHR = ILAENV( 6, 'CGELSS', ' ', M, N, NRHS, -1 )
            IF( M.GE.N .AND. M.GE.MNTHR ) THEN
*
*              Path 1a - overdetermined, with many more rows than
*                        columns
*
*              Compute space needed for CGEQRF
               CALL CGEQRF( M, N, A, LDA, DUM(1), DUM(1), -1, INFO )
               LWORK_CGEQRF=DUM(1)
*              Compute space needed for CUNMQR
               CALL CUNMQR( 'L', 'C', M, NRHS, N, A, LDA, DUM(1), B,
     $                    LDB, DUM(1), -1, INFO )
               LWORK_CUNMQR=DUM(1)
               MM = N
               MAXWRK = MAX( MAXWRK, N + N*ILAENV( 1, 'CGEQRF', ' ', M,
     $                       N, -1, -1 ) )
               MAXWRK = MAX( MAXWRK, N + NRHS*ILAENV( 1, 'CUNMQR', 'LC',
     $                       M, NRHS, N, -1 ) )
            END IF
            IF( M.GE.N ) THEN
*
*              Path 1 - overdetermined or exactly determined
*
*              Compute space needed for CGEBRD
               CALL CGEBRD( MM, N, A, LDA, S, DUM(1), DUM(1),
     $                      DUM(1), DUM(1), -1, INFO )
               LWORK_CGEBRD=DUM(1)
*              Compute space needed for CUNMBR
               CALL CUNMBR( 'Q', 'L', 'C', MM, NRHS, N, A, LDA, DUM(1),
     $                B, LDB, DUM(1), -1, INFO )
               LWORK_CUNMBR=DUM(1)
*              Compute space needed for CUNGBR
               CALL CUNGBR( 'P', N, N, N, A, LDA, DUM(1),
     $                    DUM(1), -1, INFO )
               LWORK_CUNGBR=DUM(1)
*              Compute total workspace needed
               MAXWRK = MAX( MAXWRK, 2*N + LWORK_CGEBRD )
```

```
            MAXWRK = MAX( MAXWRK, 2*N + LWORK_CUNMBR )
            MAXWRK = MAX( MAXWRK, 2*N + LWORK_CUNGBR )
            MAXWRK = MAX( MAXWRK, N*NRHS )
            MINWRK = 2*N + MAX( NRHS, M )
         END IF
         IF( N.GT.M ) THEN
            MINWRK = 2*M + MAX( NRHS, N )
            IF( N.GE.MNTHR ) THEN
*
*              Path 2a - underdetermined, with many more columns
*              than rows
*
*              Compute space needed for CGELQF
               CALL CGELQF( M, N, A, LDA, DUM(1), DUM(1),
     $                -1, INFO )
               LWORK_CGELQF=DUM(1)
*              Compute space needed for CGEBRD
               CALL CGEBRD( M, M, A, LDA, S, DUM(1), DUM(1),
     $                     DUM(1), DUM(1), -1, INFO )
               LWORK_CGEBRD=DUM(1)
*              Compute space needed for CUNMBR
               CALL CUNMBR( 'Q', 'L', 'C', M, NRHS, N, A, LDA,
     $                DUM(1), B, LDB, DUM(1), -1, INFO )
               LWORK_CUNMBR=DUM(1)
*              Compute space needed for CUNGBR
               CALL CUNGBR( 'P', M, M, M, A, LDA, DUM(1),
     $                    DUM(1), -1, INFO )
               LWORK_CUNGBR=DUM(1)
*              Compute space needed for CUNMLQ
               CALL CUNMLQ( 'L', 'C', N, NRHS, M, A, LDA, DUM(1),
     $                 B, LDB, DUM(1), -1, INFO )
               LWORK_CUNMLQ=DUM(1)
*              Compute total workspace needed
               MAXWRK = M + LWORK_CGELQF
               MAXWRK = MAX( MAXWRK, 3*M + M*M + LWORK_CGEBRD )
               MAXWRK = MAX( MAXWRK, 3*M + M*M + LWORK_CUNMBR )
               MAXWRK = MAX( MAXWRK, 3*M + M*M + LWORK_CUNGBR )
               IF( NRHS.GT.1 ) THEN
                  MAXWRK = MAX( MAXWRK, M*M + M + M*NRHS )
               ELSE
                  MAXWRK = MAX( MAXWRK, M*M + 2*M )
               END IF
               MAXWRK = MAX( MAXWRK, M + LWORK_CUNMLQ )
            ELSE
*
*              Path 2 - underdetermined
*
*              Compute space needed for CGEBRD
               CALL CGEBRD( M, N, A, LDA, S, DUM(1), DUM(1),
     $                     DUM(1), DUM(1), -1, INFO )
```

```
                 LWORK_CGEBRD=DUM(1)
*                Compute space needed for CUNMBR
                 CALL CUNMBR( 'Q', 'L', 'C', M, NRHS, M, A, LDA,
     $               DUM(1), B, LDB, DUM(1), -1, INFO )
                 LWORK_CUNMBR=DUM(1)
*                Compute space needed for CUNGBR
                 CALL CUNGBR( 'P', M, N, M, A, LDA, DUM(1),
     $                  DUM(1), -1, INFO )
                 LWORK_CUNGBR=DUM(1)
                 MAXWRK = 2*M + LWORK_CGEBRD
                 MAXWRK = MAX( MAXWRK, 2*M + LWORK_CUNMBR )
                 MAXWRK = MAX( MAXWRK, 2*M + LWORK_CUNGBR )
                 MAXWRK = MAX( MAXWRK, N*NRHS )
              END IF
           END IF
           MAXWRK = MAX( MINWRK, MAXWRK )
        END IF
        WORK( 1 ) = MAXWRK
*
        IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY )
     $      INFO = -12
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGELSS', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         RANK = 0
         RETURN
      END IF
*
*     Get machine parameters
*
      EPS = SLAMCH( 'P' )
      SFMIN = SLAMCH( 'S' )
      SMLNUM = SFMIN / EPS
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = CLANGE( 'M', M, N, A, LDA, RWORK )
      IASCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
```

```
*
*        Scale matrix norm up to SMLNUM
*
         CALL CLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
         IASCL = 1
      ELSE IF( ANRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL CLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
         IASCL = 2
      ELSE IF( ANRM.EQ.ZERO ) THEN
*
*        Matrix all zero. Return zero solution.
*
         CALL CLASET( 'F', MAX( M, N ), NRHS, CZERO, CZERO, B, LDB )
         CALL SLASET( 'F', MINMN, 1, ZERO, ZERO, S, MINMN )
         RANK = 0
         GO TO 70
      END IF
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = CLANGE( 'M', M, NRHS, B, LDB, RWORK )
      IBSCL = 0
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL CLASCL( 'G', 0, 0, BNRM, SMLNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 1
      ELSE IF( BNRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL CLASCL( 'G', 0, 0, BNRM, BIGNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 2
      END IF
*
*     Overdetermined case
*
      IF( M.GE.N ) THEN
*
*        Path 1 - overdetermined or exactly determined
*
         MM = M
         IF( M.GE.MNTHR ) THEN
*
*           Path 1a - overdetermined, with many more rows than columns
*
```

```
            MM = N
            ITAU = 1
            IWORK = ITAU + N
*
*           Compute A=Q*R
*           (CWorkspace: need 2*N, prefer N+N*NB)
*           (RWorkspace: none)
*
            CALL CGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
     $                   LWORK-IWORK+1, INFO )
*
*           Multiply B by transpose(Q)
*           (CWorkspace: need N+NRHS, prefer N+NRHS*NB)
*           (RWorkspace: none)
*
            CALL CUNMQR( 'L', 'C', M, NRHS, N, A, LDA, WORK( ITAU ), B,
     $                   LDB, WORK( IWORK ), LWORK-IWORK+1, INFO )
*
*           Zero out below R
*
            IF( N.GT.1 )
     $         CALL CLASET( 'L', N-1, N-1, CZERO, CZERO, A( 2, 1 ),
     $                      LDA )
         END IF
*
         IE = 1
         ITAUQ = 1
         ITAUP = ITAUQ + N
         IWORK = ITAUP + N
*
*        Bidiagonalize R in A
*        (CWorkspace: need 2*N+MM, prefer 2*N+(MM+N)*NB)
*        (RWorkspace: need N)
*
         CALL CGEBRD( MM, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors of R
*        (CWorkspace: need 2*N+NRHS, prefer 2*N+NRHS*NB)
*        (RWorkspace: none)
*
         CALL CUNMBR( 'Q', 'L', 'C', MM, NRHS, N, A, LDA, WORK( ITAUQ ),
     $                B, LDB, WORK( IWORK ), LWORK-IWORK+1, INFO )
*
*        Generate right bidiagonalizing vectors of R in A
*        (CWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*        (RWorkspace: none)
*
         CALL CUNGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
```

```
     $                WORK( IWORK ), LWORK-IWORK+1, INFO )
         IRWORK = IE + N
*
*        Perform bidiagonal QR iteration
*          multiply B by transpose of left singular vectors
*          compute right singular vectors in A
*        (CWorkspace: none)
*        (RWorkspace: need BDSPAC)
*
         CALL CBDSQR( 'U', N, N, 0, NRHS, S, RWORK( IE ), A, LDA, DUM,
     $                1, B, LDB, RWORK( IRWORK ), INFO )
         IF( INFO.NE.0 )
     $      GO TO 70
*
*        Multiply B by reciprocals of singular values
*
         THR = MAX( RCOND*S( 1 ), SFMIN )
         IF( RCOND.LT.ZERO )
     $      THR = MAX( EPS*S( 1 ), SFMIN )
         RANK = 0
         DO 10 I = 1, N
            IF( S( I ).GT.THR ) THEN
               CALL CSRSCL( NRHS, S( I ), B( I, 1 ), LDB )
               RANK = RANK + 1
            ELSE
               CALL CLASET( 'F', 1, NRHS, CZERO, CZERO, B( I, 1 ), LDB )
            END IF
   10    CONTINUE
*
*        Multiply B by right singular vectors
*        (CWorkspace: need N, prefer N*NRHS)
*        (RWorkspace: none)
*
         IF( LWORK.GE.LDB*NRHS .AND. NRHS.GT.1 ) THEN
            CALL CGEMM( 'C', 'N', N, NRHS, N, CONE, A, LDA, B, LDB,
     $                  CZERO, WORK, LDB )
            CALL CLACPY( 'G', N, NRHS, WORK, LDB, B, LDB )
         ELSE IF( NRHS.GT.1 ) THEN
            CHUNK = LWORK / N
            DO 20 I = 1, NRHS, CHUNK
               BL = MIN( NRHS-I+1, CHUNK )
               CALL CGEMM( 'C', 'N', N, BL, N, CONE, A, LDA, B( 1, I ),
     $                     LDB, CZERO, WORK, N )
               CALL CLACPY( 'G', N, BL, WORK, N, B( 1, I ), LDB )
   20       CONTINUE
         ELSE
            CALL CGEMV( 'C', N, N, CONE, A, LDA, B, 1, CZERO, WORK, 1 )
            CALL CCOPY( N, WORK, 1, B, 1 )
         END IF
*
```

```
      ELSE IF( N.GE.MNTHR .AND. LWORK.GE.3*M+M*M+MAX( M, NRHS, N-2*M ) )
     $          THEN
*
*        Underdetermined case, M much less than N
*
*        Path 2a - underdetermined, with many more columns than rows
*        and sufficient workspace for an efficient algorithm
*
         LDWORK = M
         IF( LWORK.GE.3*M+M*LDA+MAX( M, NRHS, N-2*M ) )
     $      LDWORK = LDA
         ITAU = 1
         IWORK = M + 1
*
*        Compute A=L*Q
*        (CWorkspace: need 2*M, prefer M+M*NB)
*        (RWorkspace: none)
*
         CALL CGELQF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
     $                LWORK-IWORK+1, INFO )
         IL = IWORK
*
*        Copy L to WORK(IL), zeroing out above it
*
         CALL CLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWORK )
         CALL CLASET( 'U', M-1, M-1, CZERO, CZERO, WORK( IL+LDWORK ),
     $                LDWORK )
         IE = 1
         ITAUQ = IL + LDWORK*M
         ITAUP = ITAUQ + M
         IWORK = ITAUP + M
*
*        Bidiagonalize L in WORK(IL)
*        (CWorkspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*        (RWorkspace: need M)
*
         CALL CGEBRD( M, M, WORK( IL ), LDWORK, S, RWORK( IE ),
     $                WORK( ITAUQ ), WORK( ITAUP ), WORK( IWORK ),
     $                LWORK-IWORK+1, INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors of L
*        (CWorkspace: need M*M+3*M+NRHS, prefer M*M+3*M+NRHS*NB)
*        (RWorkspace: none)
*
         CALL CUNMBR( 'Q', 'L', 'C', M, NRHS, M, WORK( IL ), LDWORK,
     $                WORK( ITAUQ ), B, LDB, WORK( IWORK ),
     $                LWORK-IWORK+1, INFO )
*
*        Generate right bidiagonalizing vectors of R in WORK(IL)
*        (CWorkspace: need M*M+4*M-1, prefer M*M+3*M+(M-1)*NB)
```

```
*        (RWorkspace: none)
*
         CALL CUNGBR( 'P', M, M, M, WORK( IL ), LDWORK, WORK( ITAUP ),
     $                WORK( IWORK ), LWORK-IWORK+1, INFO )
         IRWORK = IE + M
*
*        Perform bidiagonal QR iteration, computing right singular
*        vectors of L in WORK(IL) and multiplying B by transpose of
*        left singular vectors
*        (CWorkspace: need M*M)
*        (RWorkspace: need BDSPAC)
*
         CALL CBDSQR( 'U', M, M, 0, NRHS, S, RWORK( IE ), WORK( IL ),
     $                LDWORK, A, LDA, B, LDB, RWORK( IRWORK ), INFO )
         IF( INFO.NE.0 )
     $      GO TO 70
*
*        Multiply B by reciprocals of singular values
*
         THR = MAX( RCOND*S( 1 ), SFMIN )
         IF( RCOND.LT.ZERO )
     $      THR = MAX( EPS*S( 1 ), SFMIN )
         RANK = 0
         DO 30 I = 1, M
            IF( S( I ).GT.THR ) THEN
               CALL CSRSCL( NRHS, S( I ), B( I, 1 ), LDB )
               RANK = RANK + 1
            ELSE
               CALL CLASET( 'F', 1, NRHS, CZERO, CZERO, B( I, 1 ), LDB )
            END IF
   30    CONTINUE
         IWORK = IL + M*LDWORK
*
*        Multiply B by right singular vectors of L in WORK(IL)
*        (CWorkspace: need M*M+2*M, prefer M*M+M+M*NRHS)
*        (RWorkspace: none)
*
         IF( LWORK.GE.LDB*NRHS+IWORK-1 .AND. NRHS.GT.1 ) THEN
            CALL CGEMM( 'C', 'N', M, NRHS, M, CONE, WORK( IL ), LDWORK,
     $                  B, LDB, CZERO, WORK( IWORK ), LDB )
            CALL CLACPY( 'G', M, NRHS, WORK( IWORK ), LDB, B, LDB )
         ELSE IF( NRHS.GT.1 ) THEN
            CHUNK = ( LWORK-IWORK+1 ) / M
            DO 40 I = 1, NRHS, CHUNK
               BL = MIN( NRHS-I+1, CHUNK )
               CALL CGEMM( 'C', 'N', M, BL, M, CONE, WORK( IL ), LDWORK,
     $                     B( 1, I ), LDB, CZERO, WORK( IWORK ), M )
               CALL CLACPY( 'G', M, BL, WORK( IWORK ), M, B( 1, I ),
     $                      LDB )
   40       CONTINUE
```

```
         ELSE
            CALL CGEMV( 'C', M, M, CONE, WORK( IL ), LDWORK, B( 1, 1 ),
     $                  1, CZERO, WORK( IWORK ), 1 )
            CALL CCOPY( M, WORK( IWORK ), 1, B( 1, 1 ), 1 )
         END IF
*
*        Zero out below first M rows of B
*
         CALL CLASET( 'F', N-M, NRHS, CZERO, CZERO, B( M+1, 1 ), LDB )
         IWORK = ITAU + M
*
*        Multiply transpose(Q) by B
*        (CWorkspace: need M+NRHS, prefer M+NHRS*NB)
*        (RWorkspace: none)
*
         CALL CUNMLQ( 'L', 'C', N, NRHS, M, A, LDA, WORK( ITAU ), B,
     $                LDB, WORK( IWORK ), LWORK-IWORK+1, INFO )
*
      ELSE
*
*        Path 2 - remaining underdetermined cases
*
         IE = 1
         ITAUQ = 1
         ITAUP = ITAUQ + M
         IWORK = ITAUP + M
*
*        Bidiagonalize A
*        (CWorkspace: need 3*M, prefer 2*M+(M+N)*NB)
*        (RWorkspace: need N)
*
         CALL CGEBRD( M, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors
*        (CWorkspace: need 2*M+NRHS, prefer 2*M+NRHS*NB)
*        (RWorkspace: none)
*
         CALL CUNMBR( 'Q', 'L', 'C', M, NRHS, N, A, LDA, WORK( ITAUQ ),
     $                B, LDB, WORK( IWORK ), LWORK-IWORK+1, INFO )
*
*        Generate right bidiagonalizing vectors in A
*        (CWorkspace: need 3*M, prefer 2*M+M*NB)
*        (RWorkspace: none)
*
         CALL CUNGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                WORK( IWORK ), LWORK-IWORK+1, INFO )
         IRWORK = IE + M
*
```

```
*        Perform bidiagonal QR iteration,
*           computing right singular vectors of A in A and
*           multiplying B by transpose of left singular vectors
*        (CWorkspace: none)
*        (RWorkspace: need BDSPAC)
*
         CALL CBDSQR( 'L', M, N, 0, NRHS, S, RWORK( IE ), A, LDA, DUM,
     $                1, B, LDB, RWORK( IRWORK ), INFO )
         IF( INFO.NE.0 )
     $      GO TO 70
*
*        Multiply B by reciprocals of singular values
*
         THR = MAX( RCOND*S( 1 ), SFMIN )
         IF( RCOND.LT.ZERO )
     $      THR = MAX( EPS*S( 1 ), SFMIN )
         RANK = 0
         DO 50 I = 1, M
            IF( S( I ).GT.THR ) THEN
               CALL CSRSCL( NRHS, S( I ), B( I, 1 ), LDB )
               RANK = RANK + 1
            ELSE
               CALL CLASET( 'F', 1, NRHS, CZERO, CZERO, B( I, 1 ), LDB )
            END IF
   50    CONTINUE
*
*        Multiply B by right singular vectors of A
*        (CWorkspace: need N, prefer N*NRHS)
*        (RWorkspace: none)
*
         IF( LWORK.GE.LDB*NRHS .AND. NRHS.GT.1 ) THEN
            CALL CGEMM( 'C', 'N', N, NRHS, M, CONE, A, LDA, B, LDB,
     $                  CZERO, WORK, LDB )
            CALL CLACPY( 'G', N, NRHS, WORK, LDB, B, LDB )
         ELSE IF( NRHS.GT.1 ) THEN
            CHUNK = LWORK / N
            DO 60 I = 1, NRHS, CHUNK
               BL = MIN( NRHS-I+1, CHUNK )
               CALL CGEMM( 'C', 'N', N, BL, M, CONE, A, LDA, B( 1, I ),
     $                     LDB, CZERO, WORK, N )
               CALL CLACPY( 'F', N, BL, WORK, N, B( 1, I ), LDB )
   60       CONTINUE
         ELSE
            CALL CGEMV( 'C', M, N, CONE, A, LDA, B, 1, CZERO, WORK, 1 )
            CALL CCOPY( N, WORK, 1, B, 1 )
         END IF
      END IF
*
*     Undo scaling
*
```

```
      IF( IASCL.EQ.1 ) THEN
         CALL CLASCL( 'G', 0, 0, ANRM, SMLNUM, N, NRHS, B, LDB, INFO )
         CALL SLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
     $                INFO )
      ELSE IF( IASCL.EQ.2 ) THEN
         CALL CLASCL( 'G', 0, 0, ANRM, BIGNUM, N, NRHS, B, LDB, INFO )
         CALL SLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                INFO )
      END IF
      IF( IBSCL.EQ.1 ) THEN
         CALL CLASCL( 'G', 0, 0, SMLNUM, BNRM, N, NRHS, B, LDB, INFO )
      ELSE IF( IBSCL.EQ.2 ) THEN
         CALL CLASCL( 'G', 0, 0, BIGNUM, BNRM, N, NRHS, B, LDB, INFO )
      END IF
   70 CONTINUE
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of CGELSS
*
      END
```

———-

22.3.4 lisp code

— **cgelss.lisp** —

```
(let* ((zero 0.0)
       (one 1.0)
       (czero (f2cl-lib:cmplx 0.0 0.0))
       (cone (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (f2cl-lib:complex8) czero)
           (type (f2cl-lib:complex8) cone)
           (ignorable zero one czero cone))
  (defun cgelss (m n nrhs a lda b ldb$ s rcond rank work lwork rwork info)
    (declare (type (single-float) rcond)
             (type (array single-float (*)) rwork s)
             (type (array f2cl-lib:complex8 (*)) work b a)
             (type (f2cl-lib:integer4) info lwork rank ldb$ lda nrhs n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (s single-float s-%data% s-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%))
```

```
(prog ((dum (make-array 1 :element-type 'f2cl-lib:complex8)) (anrm 0.0)
       (bignum 0.0) (bnrm 0.0) (eps 0.0) (sfmin 0.0) (smlnum 0.0)
       (thr 0.0) (lwork_cgeqrf 0) (lwork_cunmqr 0) (lwork_cgebrd 0)
       (lwork_cunmbr 0) (lwork_cungbr 0) (lwork_cunmlq 0)
       (lwork_cgelqf 0) (bl 0) (chunk 0) (i 0) (iascl 0) (ibscl 0) (ie 0)
       (il 0) (irwork 0) (itau 0) (itaup 0) (itauq 0) (iwork 0)
       (ldwork 0) (maxmn 0) (maxwrk 0) (minmn 0) (minwrk 0) (mm 0)
       (mnthr 0) (lquery nil))
  (declare (type (array f2cl-lib:complex8 (1)) dum)
           (type (single-float) anrm bignum bnrm eps sfmin smlnum thr)
           (type (f2cl-lib:integer4) lwork_cgeqrf lwork_cunmqr
            lwork_cgebrd lwork_cunmbr lwork_cungbr lwork_cunmlq
            lwork_cgelqf bl chunk i iascl ibscl ie il irwork itau itaup
            itauq iwork ldwork maxmn maxwrk minmn minwrk mm mnthr)
           (type f2cl-lib:logical lquery))
  (setf info 0)
  (setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
  (setf maxmn (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
  (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
  (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
        ((< nrhs 0) (setf info -3))
        ((< lda
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -5))
        ((< ldb$
            (max (the f2cl-lib:integer4 1)
                 (the f2cl-lib:integer4 maxmn)))
         (setf info -7)))
  (cond
   ((= info 0) (setf minwrk 1) (setf maxwrk 1)
    (cond
     ((> minmn 0) (setf mm m)
      (setf mnthr
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 6 "CGELSS" " " m n nrhs -1)
                (declare (ignore var-0 var-1 var-2 var-6))
                (when var-3 (setf m var-3))
                (when var-4 (setf n var-4))
                (when var-5 (setf nrhs var-5))
                ret-val))
      (cond
       ((and (>= m n) (>= m mnthr))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (cgeqrf m n a lda
             (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
             (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1))) -1
             info)
          (declare (ignore var-2 var-4 var-5 var-6))
```

```
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-7 (setf info var-7)))
(setf lwork_cgeqrf
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (cunmqr "L" "C" m nrhs n a lda
     (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1))) b
     ldb$
     (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1))) -1
     info)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf nrhs var-3))
  (when var-4 (setf n var-4))
  (when var-6 (setf lda var-6))
  (when var-9 (setf ldb$ var-9))
  (when var-12 (setf info var-12)))
(setf lwork_cunmqr
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(setf mm n)
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add n
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "CGEQRF"
                                                           " " m
                                                           n -1
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-5
```

```
                                                                  var-6))
                                                                (when
                                                                    var-3
                                                                  (setf m
                                                                          var-3))
                                                                (when
                                                                    var-4
                                                                  (setf n
                                                                          var-4))
                                                                ret-val))))))
                (setf maxwrk
                        (max (the f2cl-lib:integer4 maxwrk)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add n
                                                    (f2cl-lib:int-mul nrhs
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "CUNMQR"
                                                                           "LC" m
                                                                           nrhs n
                                                                           -1)
                                                                        (declare
                                                                         (ignore
                                                                          var-0
                                                                          var-1
                                                                          var-2
                                                                          var-6))
                                                                        (when
                                                                            var-3
                                                                          (setf m
                                                                                  var-3))
                                                                        (when
                                                                            var-4
                                                                          (setf nrhs
                                                                                  var-4))
                                                                        (when
                                                                            var-5
                                                                          (setf n
                                                                                  var-5))
                                                                        ret-val))))))))
                (cond
```

```
((>= m n)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (cgebrd mm n a lda s
      (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
      (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
      (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
      (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1))) -1
      info)
   (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
   (setf mm var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf info var-10))
 (setf lwork_cgebrd
         (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13)
     (cunmbr "Q" "L" "C" mm nrhs n a lda
      (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1))) b
      ldb$
      (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1))) -1
      info)
   (declare
    (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
   (when var-3 (setf mm var-3))
   (when var-4 (setf nrhs var-4))
   (when var-5 (setf n var-5))
   (when var-7 (setf lda var-7))
   (when var-10 (setf ldb$ var-10))
   (when var-13 (setf info var-13)))
 (setf lwork_cunmbr
         (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
     (cungbr "P" n n n a lda
      (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
      (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1))) -1
      info)
   (declare (ignore var-0 var-4 var-6 var-7 var-8))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-3 (setf n var-3))
   (when var-5 (setf lda var-5))
   (when var-9 (setf info var-9)))
 (setf lwork_cungbr
         (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
 (setf maxwrk
```

```
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        lwork_cgebrd))))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        lwork_cunmbr))))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        lwork_cungbr))))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4 (f2cl-lib:int-mul n nrhs))))
    (setf minwrk
            (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                              (max (the f2cl-lib:integer4 nrhs)
                                   (the f2cl-lib:integer4 m))))))
  (cond
   ((> n m)
    (setf minwrk
            (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                              (max (the f2cl-lib:integer4 nrhs)
                                   (the f2cl-lib:integer4 n))))
    (cond
     ((>= n mnthr)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (cgelqf m n a lda
           (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
           (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
           -1 info)
        (declare (ignore var-2 var-4 var-5 var-6))
        (setf m var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf info var-7))
      (setf lwork_cgelqf
              (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (cgebrd m m a lda s
           (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
           (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
           (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
           (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
```

```
     -1 info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf info var-10))
(setf lwork_cgebrd
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (cunmbr "Q" "L" "C" m nrhs n a lda
     (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1))) b
     ldb$
     (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
     -1 info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(setf lwork_cunmbr
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "P" m m m a lda
     (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
     (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
     -1 info)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf info var-9)))
(setf lwork_cungbr
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (cunmlq "L" "C" n nrhs m a lda
     (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1))) b
     ldb$
     (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
     -1 info)
  (declare
```

```
    (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
   (when var-2 (setf n var-2))
   (when var-3 (setf nrhs var-3))
   (when var-4 (setf m var-4))
   (when var-6 (setf lda var-6))
   (when var-9 (setf ldb$ var-9))
   (when var-12 (setf info var-12)))
  (setf lwork_cunmlq
         (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
  (setf maxwrk (f2cl-lib:int-add m lwork_cgelqf))
  (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     (f2cl-lib:int-mul m m)
                                     lwork_cgebrd))))
  (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     (f2cl-lib:int-mul m m)
                                     lwork_cunmbr))))
  (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     (f2cl-lib:int-mul m m)
                                     lwork_cungbr))))
  (cond
   ((> nrhs 1)
    (setf maxwrk
           (max (the f2cl-lib:integer4 maxwrk)
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul m m) m
                                       (f2cl-lib:int-mul m
                                                         nrhs))))))
   (t
    (setf maxwrk
           (max (the f2cl-lib:integer4 maxwrk)
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                       (f2cl-lib:int-mul 2
                                                         m)))))))
  (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add m lwork_cunmlq)))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
     var-9 var-10)
    (cgebrd m n a lda s
     (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
     (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
     (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
     (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
     -1 info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-10))
(setf lwork_cgebrd
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (cunmbr "Q" "L" "C" m nrhs m a lda
     (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1))) b
     ldb$
     (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
     -1 info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(setf lwork_cunmbr
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "P" m n m a lda
     (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
     (f2cl-lib:array-slice dum f2cl-lib:complex8 (1) ((1 1)))
     -1 info)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf info var-9)))
(setf lwork_cungbr
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(setf maxwrk
        (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) lwork_cgebrd))
(setf maxwrk
```

```
                 (max (the f2cl-lib:integer4 maxwrk)
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                             lwork_cunmbr))))
        (setf maxwrk
                 (max (the f2cl-lib:integer4 maxwrk)
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                             lwork_cungbr))))
        (setf maxwrk
                 (max (the f2cl-lib:integer4 maxwrk)
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-mul n nrhs))))))))
    (setf maxwrk
            (max (the f2cl-lib:integer4 minwrk)
                 (the f2cl-lib:integer4 maxwrk)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce maxwrk 'f2cl-lib:complex8))
  (if (and (< lwork minwrk) (not lquery))
      (setf info -12))))
(cond
 ((/= info 0) (xerbla "CGELSS" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond ((or (= m 0) (= n 0)) (setf rank 0) (go end_label)))
(setf eps (slamch "P"))
(setf sfmin (slamch "S"))
(setf smlnum (/ sfmin eps))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (slabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "M" m n a lda rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iascl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 anrm smlnum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
```

```
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 1))
 ((> anrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 anrm bignum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 2))
 ((= anrm zero)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "F"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       czero czero b ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldb$ var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "F" minmn 1 zero zero s minmn)
    (declare (ignore var-0 var-2 var-5))
    (when var-1 (setf minmn var-1))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf minmn var-6)))
  (setf rank 0) (go label70)))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "M" m nrhs b ldb$ rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ibscl 0)
(cond
 ((and (> bnrm zero) (< bnrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
```

```
        (clascl "G" 0 0 bnrm smlnum m nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bnrm var-3))
      (when var-4 (setf smlnum var-4))
      (when var-5 (setf m var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))
    (setf ibscl 1))
   ((> bnrm bignum)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (clascl "G" 0 0 bnrm bignum m nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bnrm var-3))
      (when var-4 (setf bignum var-4))
      (when var-5 (setf m var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))
    (setf ibscl 2)))
  (cond
   ((>= m n) (setf mm m)
    (cond
     ((>= m mnthr) (setf mm n) (setf itau 1)
      (setf iwork (f2cl-lib:int-add itau n))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (cgeqrf m n a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
        (declare (ignore var-2 var-4 var-5 var-6))
        (when var-0 (setf m var-0))
        (when var-1 (setf n var-1))
        (when var-3 (setf lda var-3))
        (when var-7 (setf info var-7)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (cunmqr "L" "C" m nrhs n a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                                 ((1 *)) work-%offset%)
           b ldb$
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
```

```
    (when var-2 (setf m var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf n var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldb$ var-9))
    (when var-12 (setf info var-12)))
  (if (> n 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (claset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
           czero czero
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (2 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-5))
        (when var-3 (setf czero var-3))
        (when var-4 (setf czero var-4))
        (when var-6 (setf lda var-6))))))
(setf ie 1) (setf itauq 1) (setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgebrd mm n a lda s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf mm var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "Q" "L" "C" mm nrhs n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
```

```
  (when var-3 (setf mm var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (cungbr "P" n n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf info var-9)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" n n 0 nrhs s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     a lda dum 1 b ldb$
     (f2cl-lib:array-slice rwork-%data% single-float (irwork) ((1 *))
                           rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-3 var-5 var-6 var-7 var-9 var-10 var-11 var-13))
  (setf n var-1)
  (setf n var-2)
  (setf nrhs var-4)
  (setf lda var-8)
  (setf ldb$ var-12)
  (setf info var-14))
(if (/= info 0)
    (go label70))
(setf thr
        (max
         (* rcond (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
         sfmin))
(if (< rcond zero)
    (setf thr
            (max
             (* eps (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
             sfmin)))
```

```
(setf rank 0)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref s (i) ((1 *))) thr)
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (csrscl nrhs
           (f2cl-lib:array-slice s-%data% single-float (i) ((1 *))
                                 s-%offset%)
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-2))
        (when var-0 (setf nrhs var-0))
        (when var-3 (setf ldb$ var-3)))
      (setf rank (f2cl-lib:int-add rank 1)))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (claset "F" 1 nrhs czero czero
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-1 var-5))
        (when var-2 (setf nrhs var-2))
        (when var-3 (setf czero var-3))
        (when var-4 (setf czero var-4))
        (when var-6 (setf ldb$ var-6)))))
   label10))
(cond
 ((and (>= lwork (f2cl-lib:int-mul ldb$ nrhs)) (> nrhs 1))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (cgemm "C" "N" n nrhs n cone a lda b ldb$ czero work ldb$)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldb$ var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf ldb$ var-12)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "G" n nrhs work ldb$ b ldb$)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf nrhs var-2))
```

```
    (when var-4 (setf ldb$ var-4))
    (when var-6 (setf ldb$ var-6))))
  ((> nrhs 1) (setf chunk (the f2cl-lib:integer4 (truncate lwork n)))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
                 ((> i nrhs) nil)
    (tagbody
      (setf bl
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub nrhs i) 1))
               (the f2cl-lib:integer4 chunk)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (cgemm "C" "N" n bl n cone a lda
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 i)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$ czero work n)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf n var-2))
        (when var-3 (setf bl var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf cone var-5))
        (when var-7 (setf lda var-7))
        (when var-9 (setf ldb$ var-9))
        (when var-10 (setf czero var-10))
        (when var-12 (setf n var-12)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (clacpy "G" n bl work n
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 i)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf bl var-2))
        (when var-4 (setf n var-4))
        (when var-6 (setf ldb$ var-6)))
     label20)))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (cgemv "C" n n cone a lda b 1 czero work 1)
    (declare (ignore var-0 var-4 var-6 var-7 var-9 var-10))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf cone var-3))
    (when var-5 (setf lda var-5))
    (when var-8 (setf czero var-8)))
```

```
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (ccopy n work 1 b 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0))))))
  ((and (>= n mnthr)
        (>= lwork
            (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                              (f2cl-lib:int-mul m m)
                              (max (the f2cl-lib:integer4 m)
                                   (the f2cl-lib:integer4 nrhs)
                                   (the f2cl-lib:integer4
                                        (f2cl-lib:int-add n
                                                          (f2cl-lib:int-mul
                                                           -1 2 m)))))))
   (setf ldwork m)
   (if (>= lwork
           (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                             (f2cl-lib:int-mul m lda)
                             (max (the f2cl-lib:integer4 m)
                                  (the f2cl-lib:integer4 nrhs)
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-sub n
                                                         (f2cl-lib:int-mul
                                                          2 m))))))
       (setf ldwork lda))
   (setf itau 1) (setf iwork (f2cl-lib:int-add m 1))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (cgelqf m n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
     (declare (ignore var-2 var-4 var-5 var-6))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf info var-7))
   (setf il iwork)
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (clacpy "L" m m a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il) ((1 *))
                              work-%offset%)
        ldwork)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf m var-1))
     (when var-2 (setf m var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf ldwork var-6)))
```

```
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1) czero
     czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           ((+ il ldwork)) ((1 *)) work-%offset%)
     ldwork)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwork var-6)))
(setf ie 1)
(setf itauq (f2cl-lib:int-add il (f2cl-lib:int-mul ldwork m)))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il) ((1 *))
                           work-%offset%)
     ldwork s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwork var-3)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "Q" "L" "C" m nrhs m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il) ((1 *))
                           work-%offset%)
     ldwork
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
```

```
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwork var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (cungbr "P" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il) ((1 *))
                           work-%offset%)
     ldwork
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwork var-5))
  (when var-9 (setf info var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" m m 0 nrhs s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il) ((1 *))
                           work-%offset%)
     ldwork a lda b ldb$
     (f2cl-lib:array-slice rwork-%data% single-float (irwork) ((1 *))
                           rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-3 var-5 var-6 var-7 var-9 var-11 var-13))
  (setf m var-1)
  (setf m var-2)
  (setf nrhs var-4)
  (setf ldwork var-8)
  (setf lda var-10)
  (setf ldb$ var-12)
  (setf info var-14))
(if (/= info 0)
    (go label70))
(setf thr
        (max
         (* rcond (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
```

```
         sfmin))
(if (< rcond zero)
    (setf thr
            (max
             (* eps (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
             sfmin)))
(setf rank 0)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref s (i) ((1 *))) thr)
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (csrscl nrhs
           (f2cl-lib:array-slice s-%data% single-float (i) ((1 *))
                                 s-%offset%)
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-2))
        (when var-0 (setf nrhs var-0))
        (when var-3 (setf ldb$ var-3)))
      (setf rank (f2cl-lib:int-add rank 1)))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (claset "F" 1 nrhs czero czero
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-1 var-5))
        (when var-2 (setf nrhs var-2))
        (when var-3 (setf czero var-3))
        (when var-4 (setf czero var-4))
        (when var-6 (setf ldb$ var-6)))))
   label30))
(setf iwork (f2cl-lib:int-add il (f2cl-lib:int-mul m ldwork)))
(cond
 ((and
   (>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul ldb$ nrhs) iwork
                         (f2cl-lib:int-sub 1)))
   (> nrhs 1))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (cgemm "C" "N" m nrhs m cone
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il)
                             ((1 *)) work-%offset%)
       ldwork b ldb$ czero
```

```
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                            ((1 *)) work-%offset%)
      ldb$)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf ldwork var-7))
    (when var-9 (setf ldb$ var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf ldb$ var-12)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "G" m nrhs
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                            ((1 *)) work-%offset%)
      ldb$ b ldb$)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf nrhs var-2))
    (when var-4 (setf ldb$ var-4))
    (when var-6 (setf ldb$ var-6))))
 ((> nrhs 1)
  (setf chunk
          (the f2cl-lib:integer4 (truncate (+ (- lwork iwork) 1) m)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
                ((> i nrhs) nil)
    (tagbody
      (setf bl
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub nrhs i) 1))
               (the f2cl-lib:integer4 chunk)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (cgemm "C" "N" m bl m cone
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il)
                                 ((1 *)) work-%offset%)
           ldwork
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 i)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$ czero
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                 (iwork) ((1 *)) work-%offset%)
           m)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf bl var-3))
        (when var-4 (setf m var-4))
```

```
        (when var-5 (setf cone var-5))
        (when var-7 (setf ldwork var-7))
        (when var-9 (setf ldb$ var-9))
        (when var-10 (setf czero var-10))
        (when var-12 (setf m var-12)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (clacpy "G" m bl
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                 (iwork) ((1 *)) work-%offset%)
           m
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 i)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf bl var-2))
        (when var-4 (setf m var-4))
        (when var-6 (setf ldb$ var-6)))
    label40)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (cgemv "C" m m cone
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il)
                            ((1 *)) work-%offset%)
      ldwork
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      1 czero
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                            ((1 *)) work-%offset%)
      1)
   (declare (ignore var-0 var-4 var-6 var-7 var-9 var-10))
   (when var-1 (setf m var-1))
   (when var-2 (setf m var-2))
   (when var-3 (setf cone var-3))
   (when var-5 (setf ldwork var-5))
   (when var-8 (setf czero var-8)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (ccopy m
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                            ((1 *)) work-%offset%)
      1
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))))
```

```
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (claset "F" (f2cl-lib:int-sub n m) nrhs czero czero
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 ((+ m 1) 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
   (declare (ignore var-0 var-1 var-5))
   (when var-2 (setf nrhs var-2))
   (when var-3 (setf czero var-3))
   (when var-4 (setf czero var-4))
   (when var-6 (setf ldb$ var-6)))
 (setf iwork (f2cl-lib:int-add itau m))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12)
     (cunmlq "L" "C" n nrhs m a lda
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                            ((1 *)) work-%offset%)
      b ldb$
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
   (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
   (when var-2 (setf n var-2))
   (when var-3 (setf nrhs var-3))
   (when var-4 (setf m var-4))
   (when var-6 (setf lda var-6))
   (when var-9 (setf ldb$ var-9))
   (when var-12 (setf info var-12))))
(t (setf ie 1) (setf itauq 1) (setf itaup (f2cl-lib:int-add itauq m))
 (setf iwork (f2cl-lib:int-add itaup m))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (cgebrd m n a lda s
      (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                            rwork-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
   (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf info var-10))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
     var-10 var-11 var-12 var-13)
    (cunmbr "Q" "L" "C" m nrhs n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (cungbr "P" m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf info var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "L" m n 0 nrhs s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     a lda dum 1 b ldb$
     (f2cl-lib:array-slice rwork-%data% single-float (irwork) ((1 *))
                           rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-3 var-5 var-6 var-7 var-9 var-10 var-11 var-13))
  (setf m var-1)
  (setf n var-2)
  (setf nrhs var-4)
  (setf lda var-8)
  (setf ldb$ var-12)
  (setf info var-14))
(if (/= info 0)
```

```
    (go label70))
(setf thr
        (max
         (* rcond (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
         sfmin))
(if (< rcond zero)
    (setf thr
            (max
             (* eps (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
             sfmin)))
(setf rank 0)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref s (i) ((1 *))) thr)
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (csrscl nrhs
           (f2cl-lib:array-slice s-%data% single-float (i) ((1 *))
                                 s-%offset%)
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-2))
        (when var-0 (setf nrhs var-0))
        (when var-3 (setf ldb$ var-3)))
      (setf rank (f2cl-lib:int-add rank 1)))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (claset "F" 1 nrhs czero czero
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-1 var-5))
        (when var-2 (setf nrhs var-2))
        (when var-3 (setf czero var-3))
        (when var-4 (setf czero var-4))
        (when var-6 (setf ldb$ var-6)))))
   label50))
(cond
 ((and (>= lwork (f2cl-lib:int-mul ldb$ nrhs)) (> nrhs 1))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (cgemm "C" "N" n nrhs m cone a lda b ldb$ czero work ldb$)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf m var-4))
```

```
    (when var-5 (setf cone var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldb$ var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf ldb$ var-12)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "G" n nrhs work ldb$ b ldb$)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf nrhs var-2))
    (when var-4 (setf ldb$ var-4))
    (when var-6 (setf ldb$ var-6))))
 ((> nrhs 1) (setf chunk (the f2cl-lib:integer4 (truncate lwork n)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
                ((> i nrhs) nil)
    (tagbody
      (setf bl
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub nrhs i) 1))
               (the f2cl-lib:integer4 chunk)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (cgemm "C" "N" n bl m cone a lda
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 i)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$ czero work n)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf n var-2))
        (when var-3 (setf bl var-3))
        (when var-4 (setf m var-4))
        (when var-5 (setf cone var-5))
        (when var-7 (setf lda var-7))
        (when var-9 (setf ldb$ var-9))
        (when var-10 (setf czero var-10))
        (when var-12 (setf n var-12)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (clacpy "F" n bl work n
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 i)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf bl var-2))
        (when var-4 (setf n var-4))
        (when var-6 (setf ldb$ var-6)))
    label60)))
 (t
```

```
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (cgemv "C" m n cone a lda b 1 czero work 1)
        (declare (ignore var-0 var-4 var-6 var-7 var-9 var-10))
        (when var-1 (setf m var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf cone var-3))
        (when var-5 (setf lda var-5))
        (when var-8 (setf czero var-8)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (ccopy n work 1 b 1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf n var-0)))))))
(cond
 ((= iascl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 anrm smlnum n nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf n var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 smlnum anrm minmn 1 s minmn info)
    (declare (ignore var-0 var-1 var-2 var-6 var-7))
    (when var-3 (setf smlnum var-3))
    (when var-4 (setf anrm var-4))
    (when var-5 (setf minmn var-5))
    (when var-8 (setf minmn var-8))
    (when var-9 (setf info var-9))))
 ((= iascl 2)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 anrm bignum n nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf n var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 bignum anrm minmn 1 s minmn info)
    (declare (ignore var-0 var-1 var-2 var-6 var-7))
```

```
            (when var-3 (setf bignum var-3))
            (when var-4 (setf anrm var-4))
            (when var-5 (setf minmn var-5))
            (when var-8 (setf minmn var-8))
            (when var-9 (setf info var-9)))))
        (cond
         ((= ibscl 1)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (clascl "G" 0 0 smlnum bnrm n nrhs b ldb$ info)
            (declare (ignore var-0 var-1 var-2 var-7))
            (when var-3 (setf smlnum var-3))
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf n var-5))
            (when var-6 (setf nrhs var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf info var-9))))
         ((= ibscl 2)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (clascl "G" 0 0 bignum bnrm n nrhs b ldb$ info)
            (declare (ignore var-0 var-1 var-2 var-7))
            (when var-3 (setf bignum var-3))
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf n var-5))
            (when var-6 (setf nrhs var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf info var-9)))))
       label70
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce maxwrk 'f2cl-lib:complex8))
        (go end_label)
       end_label
        (return
         (values m n nrhs nil lda nil ldb$ nil nil rank nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgelss
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
```

```
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (single-float)
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::m
  fortran-to-lisp::n
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$ nil nil
  fortran-to-lisp::rank nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::cbdsqr
  fortran-to-lisp::cgelqf
  fortran-to-lisp::cgebrd))))
```

———-

22.4 cgelsx - DEPRECATED use cgelsy

22.5 cgelsy

22.5.1 Axiom unit tests

— cgelsy.input —

```
)set break resume
)sys rm -f cgelsy.output
)spool cgelsy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

22.5.2 Axiom help page

— cgelsy.help —

```
=====================================================================
cgelsy examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgelsy (M, N, NRHS, A, LDA, B, LDB, JPVT, RCOND, RANK, WORK,
           LWORK, RWORK, INFO)
            CGELSY solves overdetermined or underdetermined systems for GE
           matrices

Function/Subroutine Documentation
   subroutine cgelsy (integer M, integer N, integer NRHS, complex, dimension(
       lda, * ) A, integer LDA, complex, dimension( ldb, * ) B, integer LDB,
       integer, dimension( * ) JPVT, real RCOND, integer RANK, complex,
       dimension( * ) WORK, integer LWORK, real, dimension( * ) RWORK, integer
       INFO)
        CGELSY solves overdetermined or underdetermined systems for GE
       matrices

Purpose:

   CGELSY computes the minimum-norm solution to a complex linear least
   squares problem:
      minimize || A * X - B ||
   using a complete orthogonal factorization of A.  A is an M-by-N
   matrix which may be rank-deficient.

   Several right hand side vectors b and solution vectors x can be
   handled in a single call; they are stored as the columns of the
   M-by-NRHS right hand side matrix B and the N-by-NRHS solution matrix X.

   The routine first computes a QR factorization with column pivoting:
      A * P = Q * [ R11 R12 ]
                  [  0  R22 ]
   with R11 defined as the largest leading submatrix whose estimated
   condition number is less than 1/RCOND.  The order of R11, RANK,
   is the effective rank of A.

   Then, R22 is considered to be negligible, and R12 is annihilated
   by unitary transformations from the right, arriving at the
```

```
   complete orthogonal factorization:
        A * P = Q * [ T11 0 ] * Z
                    [  0  0 ]
   The minimum-norm solution is then
        X = P * Z**H [ inv(T11)*Q1**H*B ]
                     [        0         ]
   where Q1 consists of the first RANK columns of Q.

   This routine is basically identical to the original xGELSX except
   three differences:
     o The permutation of matrix B (the right hand side) is faster and
       more simple.
     o The call to the subroutine xGEQPF has been substituted by the
       the call to the subroutine xGEQP3. This subroutine is a Blas-3
       version of the QR factorization with column pivoting.
     o Matrix B (the right hand side) is updated with Blas-3.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of
   columns of matrices B and X. NRHS >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, A has been overwritten by details of its
   complete orthogonal factorization.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 B
   B is COMPLEX array, dimension (LDB,NRHS)
   On entry, the M-by-NRHS right hand side matrix B.
   On exit, the N-by-NRHS solution matrix X.

 LDB
   LDB is INTEGER
   The leading dimension of the array B. LDB >= max(1,M,N).
```

```
JPVT
  JPVT is INTEGER array, dimension (N)
  On entry, if JPVT(i) .ne. 0, the i-th column of A is permuted
  to the front of AP, otherwise column i is a free column.
  On exit, if JPVT(i) = k, then the i-th column of A*P
  was the k-th column of A.

RCOND
  RCOND is REAL
  RCOND is used to determine the effective rank of A, which
  is defined as the order of the largest leading triangular
  submatrix R11 in the QR factorization with pivoting of A,
  whose estimated condition number < 1/RCOND.

RANK
  RANK is INTEGER
  The effective rank of A, i.e., the order of the submatrix
  R11.  This is the same as the order of the submatrix T11
  in the complete orthogonal factorization of A.

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  The unblocked strategy requires that:
    LWORK >= MN + MAX( 2*MN, N+1, MN+NRHS )
  where MN = min(M,N).
  The block algorithm requires that:
    LWORK >= MN + MAX( 2*MN, NB*(N+1), MN+MN*NB, MN+NB*NRHS )
  where NB is an upper bound on the blocksize returned
  by ILAENV for the routines CGEQP3, CTZRZF, CTZRQF, CUNMQR,
  and CUNMRZ.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is REAL array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
```

———-

22.5.3 fortran code

— **cgelsy.f** —

```
*  =====================================================================
      SUBROUTINE CGELSY( M, N, NRHS, A, LDA, B, LDB, JPVT, RCOND, RANK,
     $                   WORK, LWORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDB, LWORK, M, N, NRHS, RANK
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            JPVT( * )
      REAL               RWORK( * )
      COMPLEX            A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            IMAX, IMIN
      PARAMETER          ( IMAX = 1, IMIN = 2 )
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      COMPLEX            CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0E+0, 0.0E+0 ),
     $                   CONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IASCL, IBSCL, ISMAX, ISMIN, J, LWKOPT, MN,
     $                   NB, NB1, NB2, NB3, NB4
      REAL               ANRM, BIGNUM, BNRM, SMAX, SMAXPR, SMIN, SMINPR,
     $                   SMLNUM, WSIZE
      COMPLEX            C1, C2, S1, S2
*     ..
*     .. External Subroutines ..
      EXTERNAL           CCOPY, CGEQP3, CLAIC1, CLASCL, CLASET, CTRSM,
     $                   CTZRZF, CUNMQR, CUNMRZ, SLABAD, XERBLA
*     ..
```

```
*     .. External Functions ..
      INTEGER            ILAENV
      REAL               CLANGE, SLAMCH
      EXTERNAL           CLANGE, ILAENV, SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, REAL, CMPLX
*     ..
*     .. Executable Statements ..
*
      MN = MIN( M, N )
      ISMIN = MN + 1
      ISMAX = 2*MN + 1
*
*     Test the input arguments.
*
      INFO = 0
      NB1 = ILAENV( 1, 'CGEQRF', ' ', M, N, -1, -1 )
      NB2 = ILAENV( 1, 'CGERQF', ' ', M, N, -1, -1 )
      NB3 = ILAENV( 1, 'CUNMQR', ' ', M, N, NRHS, -1 )
      NB4 = ILAENV( 1, 'CUNMRQ', ' ', M, N, NRHS, -1 )
      NB = MAX( NB1, NB2, NB3, NB4 )
      LWKOPT = MAX( 1, MN+2*N+NB*(N+1), 2*MN+NB*NRHS )
      WORK( 1 ) = CMPLX( LWKOPT )
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, M, N ) ) THEN
         INFO = -7
      ELSE IF( LWORK.LT.( MN+MAX( 2*MN, N+1, MN+NRHS ) ) .AND.
     $   .NOT.LQUERY ) THEN
         INFO = -12
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGELSY', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( MIN( M, N, NRHS ).EQ.0 ) THEN
```

```
         RANK = 0
         RETURN
      END IF
*
*     Get machine parameters
*
      SMLNUM = SLAMCH( 'S' ) / SLAMCH( 'P' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
*
*     Scale A, B if max entries outside range [SMLNUM,BIGNUM]
*
      ANRM = CLANGE( 'M', M, N, A, LDA, RWORK )
      IASCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL CLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
         IASCL = 1
      ELSE IF( ANRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL CLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
         IASCL = 2
      ELSE IF( ANRM.EQ.ZERO ) THEN
*
*        Matrix all zero. Return zero solution.
*
         CALL CLASET( 'F', MAX( M, N ), NRHS, CZERO, CZERO, B, LDB )
         RANK = 0
         GO TO 70
      END IF
*
      BNRM = CLANGE( 'M', M, NRHS, B, LDB, RWORK )
      IBSCL = 0
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL CLASCL( 'G', 0, 0, BNRM, SMLNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 1
      ELSE IF( BNRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL CLASCL( 'G', 0, 0, BNRM, BIGNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 2
      END IF
```

```
*
*     Compute QR factorization with column pivoting of A:
*        A * P = Q * R
*
      CALL CGEQP3( M, N, A, LDA, JPVT, WORK( 1 ), WORK( MN+1 ),
     $             LWORK-MN, RWORK, INFO )
      WSIZE = MN + REAL( WORK( MN+1 ) )
*
*     complex workspace: MN+NB*(N+1). real workspace 2*N.
*     Details of Householder rotations stored in WORK(1:MN).
*
*     Determine RANK using incremental condition estimation
*
      WORK( ISMIN ) = CONE
      WORK( ISMAX ) = CONE
      SMAX = ABS( A( 1, 1 ) )
      SMIN = SMAX
      IF( ABS( A( 1, 1 ) ).EQ.ZERO ) THEN
         RANK = 0
         CALL CLASET( 'F', MAX( M, N ), NRHS, CZERO, CZERO, B, LDB )
         GO TO 70
      ELSE
         RANK = 1
      END IF
*
   10 CONTINUE
      IF( RANK.LT.MN ) THEN
         I = RANK + 1
         CALL CLAIC1( IMIN, RANK, WORK( ISMIN ), SMIN, A( 1, I ),
     $                A( I, I ), SMINPR, S1, C1 )
         CALL CLAIC1( IMAX, RANK, WORK( ISMAX ), SMAX, A( 1, I ),
     $                A( I, I ), SMAXPR, S2, C2 )
*
         IF( SMAXPR*RCOND.LE.SMINPR ) THEN
            DO 20 I = 1, RANK
               WORK( ISMIN+I-1 ) = S1*WORK( ISMIN+I-1 )
               WORK( ISMAX+I-1 ) = S2*WORK( ISMAX+I-1 )
   20       CONTINUE
            WORK( ISMIN+RANK ) = C1
            WORK( ISMAX+RANK ) = C2
            SMIN = SMINPR
            SMAX = SMAXPR
            RANK = RANK + 1
            GO TO 10
         END IF
      END IF
*
*     complex workspace: 3*MN.
*
*     Logically partition R = [ R11 R12 ]
```

```
*                            [  0  R22 ]
*     where R11 = R(1:RANK,1:RANK)
*
*     [R11,R12] = [ T11, 0 ] * Y
*
      IF( RANK.LT.N )
     $   CALL CTZRZF( RANK, N, A, LDA, WORK( MN+1 ), WORK( 2*MN+1 ),
     $                LWORK-2*MN, INFO )
*
*     complex workspace: 2*MN.
*     Details of Householder rotations stored in WORK(MN+1:2*MN)
*
*     B(1:M,1:NRHS) := Q**H * B(1:M,1:NRHS)
*
      CALL CUNMQR( 'Left', 'Conjugate transpose', M, NRHS, MN, A, LDA,
     $             WORK( 1 ), B, LDB, WORK( 2*MN+1 ), LWORK-2*MN, INFO )
      WSIZE = MAX( WSIZE, 2*MN+REAL( WORK( 2*MN+1 ) ) )
*
*     complex workspace: 2*MN+NB*NRHS.
*
*     B(1:RANK,1:NRHS) := inv(T11) * B(1:RANK,1:NRHS)
*
      CALL CTRSM( 'Left', 'Upper', 'No transpose', 'Non-unit', RANK,
     $            NRHS, CONE, A, LDA, B, LDB )
*
      DO 40 J = 1, NRHS
         DO 30 I = RANK + 1, N
            B( I, J ) = CZERO
   30    CONTINUE
   40 CONTINUE
*
*     B(1:N,1:NRHS) := Y**H * B(1:N,1:NRHS)
*
      IF( RANK.LT.N ) THEN
         CALL CUNMRZ( 'Left', 'Conjugate transpose', N, NRHS, RANK,
     $                N-RANK, A, LDA, WORK( MN+1 ), B, LDB,
     $                WORK( 2*MN+1 ), LWORK-2*MN, INFO )
      END IF
*
*     complex workspace: 2*MN+NRHS.
*
*     B(1:N,1:NRHS) := P * B(1:N,1:NRHS)
*
      DO 60 J = 1, NRHS
         DO 50 I = 1, N
            WORK( JPVT( I ) ) = B( I, J )
   50    CONTINUE
         CALL CCOPY( N, WORK( 1 ), 1, B( 1, J ), 1 )
   60 CONTINUE
*
```

```
*     complex workspace: N.
*
*     Undo scaling
*
      IF( IASCL.EQ.1 ) THEN
         CALL CLASCL( 'G', 0, 0, ANRM, SMLNUM, N, NRHS, B, LDB, INFO )
         CALL CLASCL( 'U', 0, 0, SMLNUM, ANRM, RANK, RANK, A, LDA,
     $                INFO )
      ELSE IF( IASCL.EQ.2 ) THEN
         CALL CLASCL( 'G', 0, 0, ANRM, BIGNUM, N, NRHS, B, LDB, INFO )
         CALL CLASCL( 'U', 0, 0, BIGNUM, ANRM, RANK, RANK, A, LDA,
     $                INFO )
      END IF
      IF( IBSCL.EQ.1 ) THEN
         CALL CLASCL( 'G', 0, 0, SMLNUM, BNRM, N, NRHS, B, LDB, INFO )
      ELSE IF( IBSCL.EQ.2 ) THEN
         CALL CLASCL( 'G', 0, 0, BIGNUM, BNRM, N, NRHS, B, LDB, INFO )
      END IF
*
   70 CONTINUE
      WORK( 1 ) = CMPLX( LWKOPT )
*
      RETURN
*
*     End of CGELSY
*
      END
```

———-

22.5.4 lisp code

— cgelsy.lisp —

```
(let* ((imax 1)
       (imin 2)
       (zero 0.0)
       (one 1.0)
       (czero (f2cl-lib:cmplx 0.0 0.0))
       (cone (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:integer4 1 1) imax)
           (type (f2cl-lib:integer4 2 2) imin)
           (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (f2cl-lib:complex8) czero)
           (type (f2cl-lib:complex8) cone)
           (ignorable imax imin zero one czero cone))
  (defun cgelsy (m n nrhs a lda b ldb$ jpvt rcond rank work lwork rwork info)
```

```
(declare (type (array single-float (*)) rwork)
         (type (single-float) rcond)
         (type (array f2cl-lib:integer4 (*)) jpvt)
         (type (array f2cl-lib:complex8 (*)) work b a)
         (type (f2cl-lib:integer4) info lwork rank ldb$ lda nrhs n m))
(f2cl-lib:with-multi-array-data
    ((a f2cl-lib:complex8 a-%data% a-%offset%)
     (b f2cl-lib:complex8 b-%data% b-%offset%)
     (work f2cl-lib:complex8 work-%data% work-%offset%)
     (jpvt f2cl-lib:integer4 jpvt-%data% jpvt-%offset%)
     (rwork single-float rwork-%data% rwork-%offset%))
  (prog ((c1 #C(0.0 0.0)) (c2 #C(0.0 0.0)) (s1 #C(0.0 0.0))
         (s2 #C(0.0 0.0)) (anrm 0.0) (bignum 0.0) (bnrm 0.0) (smax 0.0)
         (smaxpr 0.0) (smin 0.0) (sminpr 0.0) (smlnum 0.0) (wsize 0.0)
         (i 0) (iascl 0) (ibscl 0) (ismax 0) (ismin 0) (j 0) (lwkopt 0)
         (mn 0) (nb 0) (nb1 0) (nb2 0) (nb3 0) (nb4 0) (lquery nil))
    (declare (type (f2cl-lib:complex8) c1 c2 s1 s2)
             (type (single-float) anrm bignum bnrm smax smaxpr smin sminpr
              smlnum wsize)
             (type (f2cl-lib:integer4) i iascl ibscl ismax ismin j lwkopt
              mn nb nb1 nb2 nb3 nb4)
             (type f2cl-lib:logical lquery))
    (setf mn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
    (setf ismin (f2cl-lib:int-add mn 1))
    (setf ismax (f2cl-lib:int-add (f2cl-lib:int-mul 2 mn) 1))
    (setf info 0)
    (setf nb1
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 1 "CGEQRF" " " m n -1 -1)
              (declare (ignore var-0 var-1 var-2 var-5 var-6))
              (when var-3 (setf m var-3))
              (when var-4 (setf n var-4))
              ret-val))
    (setf nb2
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 1 "CGERQF" " " m n -1 -1)
              (declare (ignore var-0 var-1 var-2 var-5 var-6))
              (when var-3 (setf m var-3))
              (when var-4 (setf n var-4))
              ret-val))
    (setf nb3
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 1 "CUNMQR" " " m n nrhs -1)
              (declare (ignore var-0 var-1 var-2 var-6))
              (when var-3 (setf m var-3))
              (when var-4 (setf n var-4))
              (when var-5 (setf nrhs var-5))
```

```
          ret-val))
(setf nb4
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 1 "CUNMRQ" " " m n nrhs -1)
          (declare (ignore var-0 var-1 var-2 var-6))
          (when var-3 (setf m var-3))
          (when var-4 (setf n var-4))
          (when var-5 (setf nrhs var-5))
          ret-val))
(setf nb
        (max (the f2cl-lib:integer4 nb1) (the f2cl-lib:integer4 nb2)
             (the f2cl-lib:integer4 nb3) (the f2cl-lib:integer4 nb4)))
(setf lwkopt
        (max (the f2cl-lib:integer4 1)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add mn (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul nb
                                                      (f2cl-lib:int-add
                                                       n 1))))
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 mn)
                                    (f2cl-lib:int-mul nb nrhs)))))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (f2cl-lib:cmplx lwkopt))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
      ((< nrhs 0) (setf info -3))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -5))
      ((< ldb$
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)
               (the f2cl-lib:integer4 n)))
       (setf info -7))
      ((and
        (< lwork
           (f2cl-lib:int-add mn
                             (max
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-mul 2 mn))
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add n 1))
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add mn nrhs)))))
        (not lquery))
       (setf info -12)))
(cond
 ((/= info 0) (xerbla "CGELSY" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
```

```
(cond
 ((=
   (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)
        (the f2cl-lib:integer4 nrhs))
   0)
  (setf rank 0) (go end_label)))
(setf smlnum (/ (slamch "S") (slamch "P")))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (slabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "M" m n a lda rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iascl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 anrm smlnum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 1))
 ((> anrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 anrm bignum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 2))
 ((= anrm zero)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
      (claset "F"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       czero czero b ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldb$ var-6)))
  (setf rank 0) (go label70)))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "M" m nrhs b ldb$ rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ibscl 0)
(cond
 ((and (> bnrm zero) (< bnrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 bnrm smlnum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 1))
 ((> bnrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 bnrm bignum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 2)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (cgeqp3 m n a lda jpvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (1) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ mn 1))
```

```
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-sub lwork mn) rwork info)
    (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8))
    (when var-0 (setf m var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf lda var-3))
    (when var-9 (setf info var-9)))
  (setf wsize
          (+ mn
             (f2cl-lib:freal
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add mn 1))
                              ((1 *)) work-%offset%))))
  (setf (f2cl-lib:fref work-%data% (ismin) ((1 *)) work-%offset%) cone)
  (setf (f2cl-lib:fref work-%data% (ismax) ((1 *)) work-%offset%) cone)
  (setf smax
          (abs
           (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)))
  (setf smin smax)
  (cond
   ((= (abs (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))) zero) (setf rank 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "F"
         (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
         czero czero b ldb$)
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf nrhs var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldb$ var-6)))
    (go label70))
   (t (setf rank 1)))
 label10
  (cond
   ((< rank mn) (setf i (f2cl-lib:int-add rank 1))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (claic1 imin rank
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ismin)
                               ((1 *)) work-%offset%)
         smin
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 i)
                               ((1 lda) (1 *)) a-%offset%)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                               ((1 lda) (1 *)) a-%offset%)
         sminpr s1 c1)
      (declare (ignore var-2 var-4 var-5))
      (when var-0 (setf imin var-0))
      (when var-1 (setf rank var-1))
      (when var-3 (setf smin var-3))
      (when var-6 (setf sminpr var-6))
```

```
  (when var-7 (setf s1 var-7))
  (when var-8 (setf c1 var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (claic1 imax rank
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ismax)
                           ((1 *)) work-%offset%)
     smax
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 i)
                           ((1 lda) (1 *)) a-%offset%)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                           ((1 lda) (1 *)) a-%offset%)
     smaxpr s2 c2)
  (declare (ignore var-2 var-4 var-5))
  (when var-0 (setf imax var-0))
  (when var-1 (setf rank var-1))
  (when var-3 (setf smax var-3))
  (when var-6 (setf smaxpr var-6))
  (when var-7 (setf s2 var-7))
  (when var-8 (setf c2 var-8)))
(cond
 ((<= (* smaxpr rcond) sminpr)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i rank) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add ismin i) 1))
                           ((1 *)) work-%offset%)
              (* s1
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-add ismin i) 1))
                                ((1 *)) work-%offset%)))
      (setf (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add ismax i) 1))
                           ((1 *)) work-%offset%)
              (* s2
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-add ismax i) 1))
                                ((1 *)) work-%offset%)))
     label20))
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ismin rank))
                       ((1 *)) work-%offset%)
        c1)
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ismax rank))
                       ((1 *)) work-%offset%)
        c2)
```

```
    (setf smin sminpr) (setf smax smaxpr)
    (setf rank (f2cl-lib:int-add rank 1)) (go label10)))))
(if (< rank n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (ctzrzf rank n a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ mn 1))
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                               ((+ (f2cl-lib:int-mul 2 mn) 1)) ((1 *))
                               work-%offset%)
         (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 mn)) info)
      (declare (ignore var-2 var-4 var-5 var-6))
      (when var-0 (setf rank var-0))
      (when var-1 (setf n var-1))
      (when var-3 (setf lda var-3))
      (when var-7 (setf info var-7))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (cunmqr "Left" "Conjugate transpose" m nrhs mn a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (1) ((1 *))
                           work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           ((+ (f2cl-lib:int-mul 2 mn) 1)) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 mn)) info)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf nrhs var-3))
  (when var-4 (setf mn var-4))
  (when var-6 (setf lda var-6))
  (when var-9 (setf ldb$ var-9))
  (when var-12 (setf info var-12)))
(setf wsize
        (max wsize
             (+ (f2cl-lib:int-mul 2 mn)
                (f2cl-lib:freal
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add
                                  (f2cl-lib:int-mul 2 mn) 1))
                                ((1 *)) work-%offset%)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ctrsm "Left" "Upper" "No transpose" "Non-unit" rank nrhs cone a
     lda b ldb$)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf rank var-4))
```

```
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf cone var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10)))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (f2cl-lib:fdo (i (f2cl-lib:int-add rank 1) (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                  czero)
         label30))
     label40))
  (cond
   ((< rank n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (cunmrz "Left" "Conjugate transpose" n nrhs rank
         (f2cl-lib:int-sub n rank) a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ mn 1))
                               ((1 *)) work-%offset%)
         b ldb$
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                               ((+ (f2cl-lib:int-mul 2 mn) 1)) ((1 *))
                               work-%offset%)
         (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 mn)) info)
      (declare
       (ignore var-0 var-1 var-5 var-6 var-8 var-9 var-11 var-12))
      (when var-2 (setf n var-2))
      (when var-3 (setf nrhs var-3))
      (when var-4 (setf rank var-4))
      (when var-7 (setf lda var-7))
      (when var-10 (setf ldb$ var-10))
      (when var-13 (setf info var-13)))))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data%
                               ((f2cl-lib:fref jpvt (i) ((1 *)))) ((1 *))
                               work-%offset%)
                  (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%))
         label50))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
          (ccopy n
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (1)
                                 ((1 *)) work-%offset%)
           1
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 j)
                                 ((1 ldb$) (1 *)) b-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf n var-0)))
    label60))
  (cond
   ((= iascl 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (clascl "G" 0 0 anrm smlnum n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf smlnum var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (clascl "U" 0 0 smlnum anrm rank rank a lda info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf smlnum var-3))
      (when var-4 (setf anrm var-4))
      (when var-5 (setf rank var-5))
      (when var-6 (setf rank var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf info var-9))))
   ((= iascl 2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (clascl "G" 0 0 anrm bignum n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf bignum var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (clascl "U" 0 0 bignum anrm rank rank a lda info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bignum var-3))
      (when var-4 (setf anrm var-4))
      (when var-5 (setf rank var-5))
```

```
            (when var-6 (setf rank var-6))
            (when var-8 (setf lda var-8))
            (when var-9 (setf info var-9)))))
        (cond
         ((= ibscl 1)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (clascl "G" 0 0 smlnum bnrm n nrhs b ldb$ info)
            (declare (ignore var-0 var-1 var-2 var-7))
            (when var-3 (setf smlnum var-3))
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf n var-5))
            (when var-6 (setf nrhs var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf info var-9))))
         ((= ibscl 2)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (clascl "G" 0 0 bignum bnrm n nrhs b ldb$ info)
            (declare (ignore var-0 var-1 var-2 var-7))
            (when var-3 (setf bignum var-3))
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf n var-5))
            (when var-6 (setf nrhs var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf info var-9)))))
       label70
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (f2cl-lib:cmplx lwkopt))
        (go end_label)
       end_label
        (return
         (values m n nrhs nil lda nil ldb$ nil nil rank nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgelsy
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
```

```
                                                  (*))
                                                 (single-float)
                                                 (fortran-to-lisp::integer4)
                                                 (array fortran-to-lisp::complex8
                                                  (*))
                                                 (fortran-to-lisp::integer4)
                                                 (array single-float (*))
                                                 (fortran-to-lisp::integer4))
                                               :return-values
                                               '(fortran-to-lisp::m
                                                 fortran-to-lisp::n
                                                 fortran-to-lisp::nrhs nil
                                                 fortran-to-lisp::lda nil
                                                 fortran-to-lisp::ldb$ nil nil
                                                 fortran-to-lisp::rank nil nil nil
                                                 fortran-to-lisp::info)
                                               :calls 'nil)))
```

———-

22.6 cgesv

22.6.1 Axiom unit tests

— **cgesv.input** —

```
)set break resume
)sys rm -f cgesv.output
)spool cgesv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

22.6.2 Axiom help page

— **cgesv.help** —

```
==================================================================
```

```
cgesv examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgesv (N, NRHS, A, LDA, IPIV, B, LDB, INFO)
            CGESV computes the solution to system of linear equations A * X =
           B for GE matrices (simple driver)

Function/Subroutine Documentation
   subroutine cgesv (integer N, integer NRHS, complex, dimension( lda, * ) A,
       integer LDA, integer, dimension( * ) IPIV, complex, dimension( ldb, * )
       B, integer LDB, integer INFO)
        CGESV computes the solution to system of linear equations A * X = B
       for GE matrices (simple driver)

Purpose:

   CGESV computes the solution to a complex system of linear equations
      A * X = B,
   where A is an N-by-N matrix and X and B are N-by-NRHS matrices.

   The LU decomposition with partial pivoting and row interchanges is
   used to factor A as
      A = P * L * U,
   where P is a permutation matrix, L is unit lower triangular, and U is
   upper triangular.  The factored form of A is then used to solve the
   system of equations A * X = B.

Parameters:

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the N-by-N coefficient matrix A.
   On exit, the factors L and U from the factorization
   A = P*L*U; the unit diagonal elements of L are not stored.
```

```
LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices that define the permutation matrix P;
  row i of the matrix was interchanged with row IPIV(i).

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS matrix of right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, U(i,i) is exactly zero.  The factorization
        has been completed, but the factor U is exactly
        singular, so the solution could not be computed.
```

———-

22.6.3 fortran code

— **cgesv.f** —

```
*  =====================================================================
      SUBROUTINE CGESV( N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), B( LDB, * )
*     ..
```

```
*
*  =====================================================================
*
*     .. External Subroutines ..
      EXTERNAL           CGETRF, CGETRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGESV ', -INFO )
         RETURN
      END IF
*
*     Compute the LU factorization of A.
*
      CALL CGETRF( N, N, A, LDA, IPIV, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
         CALL CGETRS( 'No transpose', N, NRHS, A, LDA, IPIV, B, LDB,
     $                INFO )
      END IF
      RETURN
*
*     End of CGESV
*
      END
```

———-

22.6.4 lisp code

— cgesv.lisp —

```
(defun cgesv (n nrhs a lda ipiv b ldb$ info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex8 (*)) b a)
           (type (f2cl-lib:integer4) info ldb$ lda nrhs n))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex8 a-%data% a-%offset%)
       (b f2cl-lib:complex8 b-%data% b-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ()
      (declare)
      (setf info 0)
      (cond ((< n 0) (setf info -1)) ((< nrhs 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -4))
            ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -7)))
      (cond
       ((/= info 0) (xerbla "CGESV " (f2cl-lib:int-sub info)) (go end_label)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (cgetrf n n a lda ipiv info)
        (declare (ignore var-2 var-4))
        (when var-0 (setf n var-0))
        (when var-1 (setf n var-1))
        (when var-3 (setf lda var-3))
        (when var-5 (setf info var-5)))
      (cond
       ((= info 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (cgetrs "No transpose" n nrhs a lda ipiv b ldb$ info)
          (declare (ignore var-0 var-3 var-5 var-6))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf lda var-4))
          (when var-7 (setf ldb$ var-7))
          (when var-8 (setf info var-8)))))
      (go end_label)
     end_label
      (return (values n nrhs nil lda nil nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgesv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
```

```
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::n
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::lda nil nil
  fortran-to-lisp::ldb$
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

22.7 cgesvx

22.7.1 Axiom unit tests

— **cgesvx.input** —

```
)set break resume
)sys rm -f cgesvx.output
)spool cgesvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

22.7.2 Axiom help page

— **cgesvx.help** —

```
======================================================================
cgesvx examples
```

```
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgesvx (FACT, TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV, EQUED,
           R, C, B, LDB, X, LDX, RCOND, FERR, BERR, WORK, RWORK, INFO)
            CGESVX computes the solution to system of linear equations A * X =
           B for GE matrices

Function/Subroutine Documentation
   subroutine cgesvx (character FACT, character TRANS, integer N, integer
       NRHS, complex, dimension( lda, * ) A, integer LDA, complex, dimension(
       ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV, character
       EQUED, real, dimension( * ) R, real, dimension( * ) C, complex,
       dimension( ldb, * ) B, integer LDB, complex, dimension( ldx, * ) X,
       integer LDX, real RCOND, real, dimension( * ) FERR, real, dimension( *
       ) BERR, complex, dimension( * ) WORK, real, dimension( * ) RWORK,
       integer INFO)
        CGESVX computes the solution to system of linear equations A * X = B
       for GE matrices

Purpose:

   CGESVX uses the LU factorization to compute the solution to a complex
   system of linear equations
       A * X = B,
   where A is an N-by-N matrix and X and B are N-by-NRHS matrices.

   Error bounds on the solution and a condition estimate are also provided.

Description:
   The following steps are performed:

   1. If FACT = 'E', real scaling factors are computed to equilibrate
      the system:
       TRANS = 'N':  diag(R)*A*diag(C)     *inv(diag(C))*X = diag(R)*B
       TRANS = 'T': (diag(R)*A*diag(C))**T *inv(diag(R))*X = diag(C)*B
       TRANS = 'C': (diag(R)*A*diag(C))**H *inv(diag(R))*X = diag(C)*B
     Whether or not the system will be equilibrated depends on the
     scaling of the matrix A, but if equilibration is used, A is
     overwritten by diag(R)*A*diag(C) and B by diag(R)*B (if TRANS='N')
     or diag(C)*B (if TRANS = 'T' or 'C').

   2. If FACT = 'N' or 'E', the LU decomposition is used to factor the
      matrix A (after equilibration if FACT = 'E') as
        A = P * L * U,
```

```
      where P is a permutation matrix, L is a unit lower triangular
      matrix, and U is upper triangular.

   3. If some U(i,i)=0, so that U is exactly singular, then the routine
      returns with INFO = i. Otherwise, the factored form of A is used
      to estimate the condition number of the matrix A.  If the
      reciprocal of the condition number is less than machine precision,
      INFO = N+1 is returned as a warning, but the routine still goes on
      to solve for X and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates
      for it.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(C) (if TRANS = 'N') or diag(R) (if TRANS = 'T' or 'C') so
      that it solves the original system before equilibration.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AF and IPIV contain the factored form of A.
           If EQUED is not 'N', the matrix A has been
           equilibrated with scaling factors given by R and C.
           A, AF, and IPIV are not modified.
   = 'N':  The matrix A will be copied to AF and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
           copied to AF and factored.

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose)

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
```

```
  of the matrices B and X.  NRHS >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the N-by-N matrix A.  If FACT = 'F' and EQUED is
  not 'N', then A must have been equilibrated by the scaling
  factors in R and/or C.  A is not modified if FACT = 'F' or
  'N', or if FACT = 'E' and EQUED = 'N' on exit.

  On exit, if EQUED .ne. 'N', A is scaled as follows:
  EQUED = 'R':  A := diag(R) * A
  EQUED = 'C':  A := A * diag(C)
  EQUED = 'B':  A := diag(R) * A * diag(C).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the factors L and U from the factorization
  A = P*L*U as computed by CGETRF.  If EQUED .ne. 'N', then
  AF is the factored form of the equilibrated matrix A.

  If FACT = 'N', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the equilibrated matrix A (see the description of A for
  the form of the equilibrated matrix).

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains the pivot indices from the factorization A = P*L*U
  as computed by CGETRF; row i of the matrix was interchanged
  with row IPIV(i).

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then IPIV is an output argument and on exit
```

```
  contains the pivot indices from the factorization A = P*L*U
  of the equilibrated matrix A.

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
  = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
  = 'B':  Both row and column equilibration, i.e., A has been
          replaced by diag(R) * A * diag(C).
          EQUED is an input argument if FACT = 'F'; otherwise, it is an
          output argument.

R
  R is REAL array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
  EQUED = 'R' or 'B', each element of R must be positive.

C
  C is REAL array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if TRANS = 'N' and EQUED = 'R' or 'B', B is overwritten by
  diag(R)*B;
  if TRANS = 'T' or 'C' and EQUED = 'C' or 'B', B is
  overwritten by diag(C)*B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X
  to the original system of equations.  Note that A and B are
  modified on exit if EQUED .ne. 'N', and the solution to the
  equilibrated system is inv(diag(C))*X if TRANS = 'N' and
```

```
  EQUED = 'C' or 'B', or inv(diag(R))*X if TRANS = 'T' or 'C'
  and EQUED = 'R' or 'B'.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  The estimate of the reciprocal condition number of the matrix
  A after equilibration (if done).  If RCOND is less than the
  machine precision (in particular, if RCOND = 0), the matrix
  is singular to working precision.  This condition is
  indicated by a return code of INFO > 0.

FERR
  FERR is REAL array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is REAL array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is COMPLEX array, dimension (2*N)

RWORK
  RWORK is REAL array, dimension (2*N)
  On exit, RWORK(1) contains the reciprocal pivot growth
  factor norm(A)/norm(U). The "max absolute element" norm is
  used. If RWORK(1) is much less than 1, then the stability
  of the LU factorization of the (equilibrated) matrix A
  could be poor. This also means that the solution X, condition
  estimator RCOND, and forward error bound FERR could be
  unreliable. If factorization fails with 0<INFO<=N, then
  RWORK(1) contains the reciprocal pivot growth factor for the
  leading INFO columns of A.

INFO
  INFO is INTEGER
  = 0:  successful exit
```

```
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, and i is
      <= N:  U(i,i) is exactly zero.  The factorization has
             been completed, but the factor U is exactly
             singular, so the solution and error bounds
             could not be computed. RCOND = 0 is returned.
      = N+1: U is nonsingular, but RCOND is less than machine
             precision, meaning that the matrix is singular
             to working precision.  Nevertheless, the
             solution and error bounds are computed because
             there are a number of situations where the
             computed solution can be more accurate than the
             value of RCOND would suggest.
```

———-

22.7.3 fortran code

— **cgesvx.f** —

```
*  =====================================================================
      SUBROUTINE CGESVX( FACT, TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $                   EQUED, R, C, B, LDB, X, LDX, RCOND, FERR, BERR,
     $                   WORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, TRANS
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               BERR( * ), C( * ), FERR( * ), R( * ),
     $                   RWORK( * )
      COMPLEX            A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
```

```
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            COLEQU, EQUIL, NOFACT, NOTRAN, ROWEQU
      CHARACTER          NORM
      INTEGER            I, INFEQU, J
      REAL               AMAX, ANORM, BIGNUM, COLCND, RCMAX, RCMIN,
     $                   ROWCND, RPVGRW, SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               CLANGE, CLANTR, SLAMCH
      EXTERNAL           LSAME, CLANGE, CLANTR, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGECON, CGEEQU, CGERFS, CGETRF, CGETRS, CLACPY,
     $                   CLAQGE, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      NOTRAN = LSAME( TRANS, 'N' )
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         ROWEQU = .FALSE.
         COLEQU = .FALSE.
      ELSE
         ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
         COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         SMLNUM = SLAMCH( 'Safe minimum' )
         BIGNUM = ONE / SMLNUM
      END IF
*
*     Test the input parameters.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.LSAME( FACT, 'F' ) )
     $     THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $         LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
```

```
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $         ( ROWEQU .OR. COLEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -10
      ELSE
         IF( ROWEQU ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 10 J = 1, N
               RCMIN = MIN( RCMIN, R( J ) )
               RCMAX = MAX( RCMAX, R( J ) )
   10       CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -11
            ELSE IF( N.GT.0 ) THEN
               ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               ROWCND = ONE
            END IF
         END IF
         IF( COLEQU .AND. INFO.EQ.0 ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 20 J = 1, N
               RCMIN = MIN( RCMIN, C( J ) )
               RCMAX = MAX( RCMAX, C( J ) )
   20       CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -12
            ELSE IF( N.GT.0 ) THEN
               COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               COLCND = ONE
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -14
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -16
            END IF
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGESVX', -INFO )
         RETURN
```

```
      END IF
*
      IF( EQUIL ) THEN
*
*        Compute row and column scalings to equilibrate the matrix A.
*
         CALL CGEEQU( N, N, A, LDA, R, C, ROWCND, COLCND, AMAX, INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*           Equilibrate the matrix.
*
            CALL CLAQGE( N, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $                   EQUED )
            ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
            COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         END IF
      END IF
*
*     Scale the right hand side.
*
      IF( NOTRAN ) THEN
         IF( ROWEQU ) THEN
            DO 40 J = 1, NRHS
               DO 30 I = 1, N
                  B( I, J ) = R( I )*B( I, J )
   30          CONTINUE
   40       CONTINUE
         END IF
      ELSE IF( COLEQU ) THEN
         DO 60 J = 1, NRHS
            DO 50 I = 1, N
               B( I, J ) = C( I )*B( I, J )
   50       CONTINUE
   60    CONTINUE
      END IF
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the LU factorization of A.
*
         CALL CLACPY( 'Full', N, N, A, LDA, AF, LDAF )
         CALL CGETRF( N, N, AF, LDAF, IPIV, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 ) THEN
*
*           Compute the reciprocal pivot growth factor of the
*           leading rank-deficient INFO columns of A.
*
```

```
            RPVGRW = CLANTR( 'M', 'U', 'N', INFO, INFO, AF, LDAF,
     $               RWORK )
            IF( RPVGRW.EQ.ZERO ) THEN
               RPVGRW = ONE
            ELSE
               RPVGRW = CLANGE( 'M', N, INFO, A, LDA, RWORK ) /
     $                  RPVGRW
            END IF
            RWORK( 1 ) = RPVGRW
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A and the
*     reciprocal pivot growth factor RPVGRW.
*
      IF( NOTRAN ) THEN
         NORM = '1'
      ELSE
         NORM = 'I'
      END IF
      ANORM = CLANGE( NORM, N, N, A, LDA, RWORK )
      RPVGRW = CLANTR( 'M', 'U', 'N', N, N, AF, LDAF, RWORK )
      IF( RPVGRW.EQ.ZERO ) THEN
         RPVGRW = ONE
      ELSE
         RPVGRW = CLANGE( 'M', N, N, A, LDA, RWORK ) / RPVGRW
      END IF
*
*     Compute the reciprocal of the condition number of A.
*
      CALL CGECON( NORM, N, AF, LDAF, ANORM, RCOND, WORK, RWORK, INFO )
*
*     Compute the solution matrix X.
*
      CALL CLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL CGETRS( TRANS, N, NRHS, AF, LDAF, IPIV, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL CGERFS( TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB, X,
     $             LDX, FERR, BERR, WORK, RWORK, INFO )
*
*     Transform the solution matrix X to a solution of the original
*     system.
*
      IF( NOTRAN ) THEN
         IF( COLEQU ) THEN
```

```
            DO 80 J = 1, NRHS
               DO 70 I = 1, N
                  X( I, J ) = C( I )*X( I, J )
   70          CONTINUE
   80       CONTINUE
            DO 90 J = 1, NRHS
               FERR( J ) = FERR( J ) / COLCND
   90       CONTINUE
         END IF
      ELSE IF( ROWEQU ) THEN
         DO 110 J = 1, NRHS
            DO 100 I = 1, N
               X( I, J ) = R( I )*X( I, J )
  100       CONTINUE
  110    CONTINUE
         DO 120 J = 1, NRHS
            FERR( J ) = FERR( J ) / ROWCND
  120    CONTINUE
      END IF
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.SLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      RWORK( 1 ) = RPVGRW
      RETURN
*
*     End of CGESVX
*
      END
```

22.7.4 lisp code

— **cgesvx.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun cgesvx
         (fact trans n nrhs a lda af ldaf ipiv equed r c b ldb$ x ldx rcond
          ferr berr work rwork info)
    (declare (type (single-float) rcond)
             (type (array single-float (*)) rwork berr ferr c r)
             (type (array f2cl-lib:integer4 (*)) ipiv)
```

```
          (type (array f2cl-lib:complex8 (*)) work x b af a)
          (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
          (type (string *) equed trans fact))
  (f2cl-lib:with-multi-array-data
      ((fact character fact-%data% fact-%offset%)
       (trans character trans-%data% trans-%offset%)
       (equed character equed-%data% equed-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (af f2cl-lib:complex8 af-%data% af-%offset%)
       (b f2cl-lib:complex8 b-%data% b-%offset%)
       (x f2cl-lib:complex8 x-%data% x-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (r single-float r-%data% r-%offset%)
       (c single-float c-%data% c-%offset%)
       (ferr single-float ferr-%data% ferr-%offset%)
       (berr single-float berr-%data% berr-%offset%)
       (rwork single-float rwork-%data% rwork-%offset%))
    (prog ((amax 0.0) (anorm 0.0) (bignum 0.0) (colcnd 0.0) (rcmax 0.0)
          (rcmin 0.0) (rowcnd 0.0) (rpvgrw 0.0) (smlnum 0.0) (i 0)
          (infequ 0) (j 0)
          (norm
           (make-array '(1) :element-type 'character :initial-element #\ ))
          (colequ nil) (equil nil) (nofact nil) (notran nil) (rowequ nil))
      (declare
       (type (single-float) amax anorm bignum colcnd rcmax rcmin rowcnd
        rpvgrw smlnum)
       (type (f2cl-lib:integer4) i infequ j)
       (type (string 1) norm)
       (type f2cl-lib:logical colequ equil nofact notran rowequ))
      (setf info 0)
      (setf nofact
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame fact "N")
                (declare (ignore var-1))
                (when var-0 (setf fact var-0))
                ret-val))
      (setf equil
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame fact "E")
                (declare (ignore var-1))
                (when var-0 (setf fact var-0))
                ret-val))
      (setf notran
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "N")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val))
      (cond
```

```
((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
 (setf rowequ f2cl-lib:%false%) (setf colequ f2cl-lib:%false%))
(t
 (setf rowequ
         (or
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame equed "R")
            (declare (ignore var-1))
            (when var-0 (setf equed var-0))
            ret-val)
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame equed "B")
            (declare (ignore var-1))
            (when var-0 (setf equed var-0))
            ret-val)))
 (setf colequ
         (or
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame equed "C")
            (declare (ignore var-1))
            (when var-0 (setf equed var-0))
            ret-val)
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame equed "B")
            (declare (ignore var-1))
            (when var-0 (setf equed var-0))
            ret-val)))
 (setf smlnum (slamch "Safe minimum")) (setf bignum (/ one smlnum))))
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
 (setf info -1))
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
```

```
 (setf info -2))
((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
 (setf info -6))
((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
 (setf info -8))
((and
  (multiple-value-bind (ret-val var-0 var-1)
      (lsame fact "F")
    (declare (ignore var-1))
    (when var-0 (setf fact var-0))
    ret-val)
  (not
   (or rowequ colequ
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame equed "N")
         (declare (ignore var-1))
         (when var-0 (setf equed var-0))
         ret-val))))
 (setf info -10))
(t
 (cond
  (rowequ (setf rcmin bignum) (setf rcmax zero)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf rcmin
               (min rcmin
                    (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
       (setf rcmax
               (max rcmax
                    (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
      label10))
   (cond ((<= rcmin zero) (setf info -11))
         ((> n 0)
          (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum))))
         (t (setf rowcnd one)))))
 (cond
  ((and colequ (= info 0)) (setf rcmin bignum) (setf rcmax zero)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf rcmin
               (min rcmin
                    (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
       (setf rcmax
               (max rcmax
                    (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
      label20))
   (cond ((<= rcmin zero) (setf info -12))
```

```
          ((> n 0)
           (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum))))
          (t (setf colcnd one)))))
  (cond
   ((= info 0)
    (cond
     ((< ldb$
         (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -14))
     ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -16)))))))
(cond
 ((/= info 0) (xerbla "CGESVX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (equil
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (cgeequ n n a lda r c rowcnd colcnd amax infequ)
    (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5))
    (setf rowcnd var-6)
    (setf colcnd var-7)
    (setf amax var-8)
    (setf infequ var-9))
  (cond
   ((= infequ 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (claqge n n a lda r c rowcnd colcnd amax equed)
      (declare (ignore var-2 var-4 var-5))
      (when var-0 (setf n var-0))
      (when var-1 (setf n var-1))
      (when var-3 (setf lda var-3))
      (when var-6 (setf rowcnd var-6))
      (when var-7 (setf colcnd var-7))
      (when var-8 (setf amax var-8))
      (when var-9 (setf equed var-9)))
    (setf rowequ
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "R")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "B")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)))
    (setf colequ
```

```
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "C")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "B")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)))))))
(cond
 (notran
  (cond
   (rowequ
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)))
          label30))
      label40)))))
 (colequ
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)
                (* (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)))
        label50))
    label60))))
(cond
 ((or nofact equil)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "Full" n n a lda af ldaf)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
```

```
    (when var-6 (setf ldaf var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (cgetrf n n af ldaf ipiv info)
    (declare (ignore var-2 var-4))
    (when var-0 (setf n var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf ldaf var-3))
    (when var-5 (setf info var-5)))
  (cond
   ((> info 0)
    (setf rpvgrw
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6
                 var-7)
                (clantr "M" "U" "N" info info af ldaf rwork)
              (declare (ignore var-0 var-1 var-2 var-5 var-7))
              (when var-3 (setf info var-3))
              (when var-4 (setf info var-4))
              (when var-6 (setf ldaf var-6))
              ret-val))
    (cond ((= rpvgrw zero) (setf rpvgrw one))
          (t
           (setf rpvgrw
                   (/
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                        (clange "M" n info a lda rwork)
                      (declare (ignore var-0 var-3 var-5))
                      (when var-1 (setf n var-1))
                      (when var-2 (setf info var-2))
                      (when var-4 (setf lda var-4))
                      ret-val)
                    rpvgrw))))
    (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
            rpvgrw)
    (setf rcond zero) (go end_label)))))
(cond (notran (f2cl-lib:f2cl-set-string norm "1" (string 1)))
      (t (f2cl-lib:f2cl-set-string norm "I" (string 1))))
(setf anorm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange norm n n a lda rwork)
          (declare (ignore var-3 var-5))
          (when var-0 (setf norm var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf rpvgrw
        (multiple-value-bind
```

```
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (clantr "M" "U" "N" n n af ldaf rwork)
          (declare (ignore var-0 var-1 var-2 var-5 var-7))
          (when var-3 (setf n var-3))
          (when var-4 (setf n var-4))
          (when var-6 (setf ldaf var-6))
          ret-val))
(cond ((= rpvgrw zero) (setf rpvgrw one))
      (t
       (setf rpvgrw
               (/
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                    (clange "M" n n a lda rwork)
                  (declare (ignore var-0 var-3 var-5))
                  (when var-1 (setf n var-1))
                  (when var-2 (setf n var-2))
                  (when var-4 (setf lda var-4))
                  ret-val)
                rpvgrw))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cgecon norm n af ldaf anorm rcond work rwork info)
  (declare (ignore var-2 var-4 var-6 var-7))
  (setf norm var-0)
  (setf n var-1)
  (setf ldaf var-3)
  (setf rcond var-5)
  (setf info var-8))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "Full" n nrhs b ldb$ x ldx)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf ldx var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cgetrs trans n nrhs af ldaf ipiv x ldx info)
  (declare (ignore var-3 var-5 var-6))
  (when var-0 (setf trans var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldaf var-4))
  (when var-7 (setf ldx var-7))
  (when var-8 (setf info var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16)
    (cgerfs trans n nrhs a lda af ldaf ipiv b ldb$ x ldx ferr berr work
```

```
      rwork info)
    (declare
     (ignore var-2 var-3 var-5 var-7 var-8 var-9 var-10 var-11 var-12
      var-13 var-14 var-15))
    (setf trans var-0)
    (setf n var-1)
    (setf lda var-4)
    (setf ldaf var-6)
    (setf info var-16))
  (cond
   (notran
    (cond
     (colequ
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nrhs) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                    x-%offset%)
                    (* (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)
                       (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                      x-%offset%)))
            label70))
        label80))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nrhs) nil)
        (tagbody
          (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                (/
                 (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                 colcnd))
        label90)))))
    (rowequ
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j nrhs) nil)
       (tagbody
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                   x-%offset%)
                   (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                      (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                     x-%offset%)))
           label100))
       label110))
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j nrhs) nil)
```

```
            (tagbody
              (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                      (/ (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                         rowcnd))
             label120))))
        (if (< rcond (slamch "Epsilon"))
            (setf info (f2cl-lib:int-add n 1)))
        (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%) rpvgrw)
        (go end_label)
       end_label
        (return
         (values fact trans n nrhs nil lda nil ldaf nil equed nil nil nil ldb$
                 nil ldx rcond nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgesvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (string) (array single-float (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::fact
                                              fortran-to-lisp::trans
                                              fortran-to-lisp::n
```

```
                                        fortran-to-lisp::nrhs nil
                                        fortran-to-lisp::lda nil
                                        fortran-to-lisp::ldaf nil
                                        fortran-to-lisp::equed nil nil
                                        nil fortran-to-lisp::ldb$ nil
                                        fortran-to-lisp::ldx
                                        fortran-to-lisp::rcond nil nil
                                        nil nil fortran-to-lisp::info)
                                      :calls
                                      '(fortran-to-lisp::cgerfs
                                        fortran-to-lisp::cgecon
                                        fortran-to-lisp::cgeequ))))
```

———-

22.8 cgesvxx

22.8.1 Axiom unit tests

— **cgesvxx.input** —

```
)set break resume
)sys rm -f cgesvxx.output
)spool cgesvxx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

22.8.2 Axiom help page [Demm05]

— **cgesvxx.help** —

```
==========================================================================
cgesvxx examples
==========================================================================

==========================================================================
```

```
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgesvxx (FACT, TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV,
           EQUED, R, C, B, LDB, X, LDX, RCOND, RPVGRW, BERR, N_ERR_BNDS,
           ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, RWORK, INFO)
            CGESVXX computes the solution to system of linear equations A * X
           = B for GE matrices

Function/Subroutine Documentation
   subroutine cgesvxx (character FACT, character TRANS, integer N, integer
       NRHS, complex, dimension( lda, * ) A, integer LDA, complex, dimension(
       ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV, character
       EQUED, real, dimension( * ) R, real, dimension( * ) C, complex,
       dimension( ldb, * ) B, integer LDB, complex, dimension( ldx , * ) X,
       integer LDX, real RCOND, real RPVGRW, real, dimension( * ) BERR,
       integer N_ERR_BNDS, real, dimension( nrhs, * ) ERR_BNDS_NORM, real,
       dimension( nrhs, * ) ERR_BNDS_COMP, integer NPARAMS, real, dimension( *
       ) PARAMS, complex, dimension( * ) WORK, real, dimension( * ) RWORK,
       integer INFO)
        CGESVXX computes the solution to system of linear equations A * X = B
       for GE matrices

Purpose:
   CGESVXX uses the LU factorization to compute the solution to a
   complex system of linear equations  A * X = B,  where A is an
   N-by-N matrix and X and B are N-by-NRHS matrices.

   If requested, both normwise and maximum componentwise error bounds
   are returned. CGESVXX will return a solution with a tiny
   guaranteed error (O(eps) where eps is the working machine
   precision) unless the matrix is very ill-conditioned, in which
   case a warning is returned. Relevant condition numbers also are
   calculated and returned.

   CGESVXX accepts user-provided factorizations and equilibration
   factors; see the definitions of the FACT and EQUED options.
   Solving with refinement and using a factorization from a previous
   CGESVXX call will also produce a solution with either O(eps)
   errors or warnings, but we cannot make that claim for general
   user-provided factorizations and equilibration factors if they
   differ from what CGESVXX would itself produce.

Description:
   The following steps are performed:

   1. If FACT = 'E', real scaling factors are computed to equilibrate
      the system:
```

```
      TRANS = 'N':  diag(R)*A*diag(C)     *inv(diag(C))*X = diag(R)*B
      TRANS = 'T': (diag(R)*A*diag(C))**T *inv(diag(R))*X = diag(C)*B
      TRANS = 'C': (diag(R)*A*diag(C))**H *inv(diag(R))*X = diag(C)*B

      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(R)*A*diag(C) and B by diag(R)*B (if TRANS='N')
      or diag(C)*B (if TRANS = 'T' or 'C').

   2. If FACT = 'N' or 'E', the LU decomposition is used to factor
      the matrix A (after equilibration if FACT = 'E') as
         A = P * L * U,
      where P is a permutation matrix, L is a unit lower triangular
      matrix, and U is upper triangular.

   3. If some U(i,i)=0, so that U is exactly singular, then the
      routine returns with INFO = i. Otherwise, the factored form of A
      is used to estimate the condition number of the matrix A (see
      argument RCOND). If the reciprocal of the condition number is less
      than machine precision, the routine still goes on to solve for X
      and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. By default (unless PARAMS(LA_LINRX_ITREF_I) is set to zero),
      the routine will use iterative refinement to try to get a small
      error and error bounds.  Refinement calculates the residual to at
      least twice the working precision.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(C) (if TRANS = 'N') or diag(R) (if TRANS = 'T' or 'C') so
      that it solves the original system before equilibration.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AF and IPIV contain the factored form of A.
           If EQUED is not 'N', the matrix A has been
           equilibrated with scaling factors given by R and C.
```

```
          A, AF, and IPIV are not modified.
  = 'N':  The matrix A will be copied to AF and factored.
  = 'E':  The matrix A will be equilibrated if necessary, then
          copied to AF and factored.

TRANS
  TRANS is CHARACTER*1
  Specifies the form of the system of equations:
  = 'N':  A * X = B     (No transpose)
  = 'T':  A**T * X = B  (Transpose)
  = 'C':  A**H * X = B  (Conjugate Transpose)

 N
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

 A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the N-by-N matrix A.  If FACT = 'F' and EQUED is
  not 'N', then A must have been equilibrated by the scaling
  factors in R and/or C.  A is not modified if FACT = 'F' or
  'N', or if FACT = 'E' and EQUED = 'N' on exit.

  On exit, if EQUED .ne. 'N', A is scaled as follows:
  EQUED = 'R':  A := diag(R) * A
  EQUED = 'C':  A := A * diag(C)
  EQUED = 'B':  A := diag(R) * A * diag(C).

 LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

 AF
  AF is COMPLEX array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the factors L and U from the factorization
  A = P*L*U as computed by CGETRF.  If EQUED .ne. 'N', then
  AF is the factored form of the equilibrated matrix A.

  If FACT = 'N', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
```

```
  of the equilibrated matrix A (see the description of A for
  the form of the equilibrated matrix).

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains the pivot indices from the factorization A = P*L*U
  as computed by CGETRF; row i of the matrix was interchanged
  with row IPIV(i).

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the equilibrated matrix A.

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
  = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
  = 'B':  Both row and column equilibration, i.e., A has been
          replaced by diag(R) * A * diag(C).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

R
  R is REAL array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
  EQUED = 'R' or 'B', each element of R must be positive.
  If R is output, each element of R is a power of the radix.
  If R is input, each element of R should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

C
  C is REAL array, dimension (N)
```

```
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.
  If C is output, each element of C is a power of the radix.
  If C is input, each element of C should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if TRANS = 'N' and EQUED = 'R' or 'B', B is overwritten by diag(R)*B;
  if TRANS = 'T' or 'C' and EQUED = 'C' or 'B', B is
     overwritten by diag(C)*B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  If INFO = 0, the N-by-NRHS solution matrix X to the original
  system of equations.  Note that A and B are modified on exit
  if EQUED .ne. 'N', and the solution to the equilibrated system is
  inv(diag(C))*X if TRANS = 'N' and EQUED = 'C' or 'B', or
  inv(diag(R))*X if TRANS = 'T' or 'C' and EQUED = 'R' or 'B'.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

RPVGRW
  RPVGRW is REAL
  Reciprocal pivot growth.  On exit, this contains the reciprocal
```

```
  pivot growth factor norm(A)/norm(U). The "max absolute element"
  norm is used.  If this is much less than 1, then the stability of
  the LU factorization of the (equilibrated) matrix A could be poor.
  This also means that the solution X, estimated condition numbers,
  and error bounds could be unreliable. If factorization fails with
  0<INFO<=N, then this contains the reciprocal pivot growth factor
  for the leading INFO columns of A.  In CGESVX, this quantity is
  returned in WORK(1).

BERR
  BERR is REAL array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
    max_j (abs(XTRUE(j,i) - X(j,i)))
    ------------------------------
         max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
    reciprocal condition number is less than the threshold
    sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
    almost certainly within a factor of 10 of the true error
    so long as the next entry is greater than the threshold
    sqrt(n) * slamch('Epsilon'). This error bound should only
```

```
    be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
    reciprocal condition number.  Compared with the threshold
    sqrt(n) * slamch('Epsilon') to determine if the error
    estimate is "guaranteed". These reciprocal condition
    numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
    appropriately scaled matrix Z.
    Let Z = S*A, where S scales each row by a power of the
    radix so all absolute row sums of Z are approximately 1.

ERR_BNDS_COMP
  ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
           abs(XTRUE(j,i) - X(j,i))
     max_j ----------------------
                 abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then

  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
```

```
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is REAL array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.

  Default: 1.0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
          (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.

  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
  factorizations or factorizations other than LU. If
  the factorization uses a technique other than
  Gaussian elimination, the guarantees in
  err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.

  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is COMPLEX array, dimension (2*N)

RWORK
```

```
  RWORK is REAL array, dimension (2*N)

 INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is guaranteed.
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
    has been completed, but the factor U is exactly singular, so
    the solution and error bounds could not be computed. RCOND = 0
    is returned.
  = N+J: The solution corresponding to the Jth right-hand side is
    not guaranteed. The solutions corresponding to other right-
    hand sides K with K > J may not be guaranteed as well, but
    only the first such right-hand side is reported. If a small
    componentwise error is not requested (PARAMS(3) = 0.0) then
    the Jth right-hand side is the first with a normwise error
    bound that is not guaranteed (the smallest J such
    that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
    the Jth right-hand side is the first with either a normwise or
    componentwise error bound that is not guaranteed (the smallest
    J such that either ERR_BNDS_NORM(J,1) = 0.0 or
    ERR_BNDS_COMP(J,1) = 0.0). See the definition of
    ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
    about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

———-

22.8.3 fortran code

— cgesvxx.f —

```
*  ==================================================================
      SUBROUTINE CGESVXX( FACT, TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $                    EQUED, R, C, B, LDB, X, LDX, RCOND, RPVGRW,
     $                    BERR, N_ERR_BNDS, ERR_BNDS_NORM,
     $                    ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, RWORK,
     $                    INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, TRANS
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
```

```
      REAL               RCOND, RPVGRW
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX , * ),WORK( * )
      REAL               R( * ), C( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * ), RWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
*     ..
*     .. Local Scalars ..
      LOGICAL            COLEQU, EQUIL, NOFACT, NOTRAN, ROWEQU
      INTEGER            INFEQU, J
      REAL               AMAX, BIGNUM, COLCND, RCMAX, RCMIN,
     $                   ROWCND, SMLNUM
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, SLAMCH, CLA_GERPVGRW
      LOGICAL            LSAME
      REAL               SLAMCH, CLA_GERPVGRW
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEEQUB, CGETRF, CGETRS, CLACPY, CLAQGE,
     $                   XERBLA, CLASCL2, CGERFSX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      NOTRAN = LSAME( TRANS, 'N' )
      SMLNUM = SLAMCH( 'Safe minimum' )
```

```
      BIGNUM = ONE / SMLNUM
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         ROWEQU = .FALSE.
         COLEQU = .FALSE.
      ELSE
         ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
         COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
      END IF
*
*     Default is failure.  If an input parameter is wrong or
*     factorization fails, make everything look horrible.  Only the
*     pivot growth is set here, the rest is initialized in CGERFSX.
*
      RPVGRW = ZERO
*
*     Test the input parameters.  PARAMS is not tested until CGERFSX.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.
     $     LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $        LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $        ( ROWEQU .OR. COLEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -10
      ELSE
         IF( ROWEQU ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 10 J = 1, N
               RCMIN = MIN( RCMIN, R( J ) )
               RCMAX = MAX( RCMAX, R( J ) )
 10         CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -11
            ELSE IF( N.GT.0 ) THEN
               ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               ROWCND = ONE
            END IF
```

```
         END IF
         IF( COLEQU .AND. INFO.EQ.0 ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 20 J = 1, N
               RCMIN = MIN( RCMIN, C( J ) )
               RCMAX = MAX( RCMAX, C( J ) )
 20         CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -12
            ELSE IF( N.GT.0 ) THEN
               COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               COLCND = ONE
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -14
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -16
            END IF
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGESVXX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*     Compute row and column scalings to equilibrate the matrix A.
*
         CALL CGEEQUB( N, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $        INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*     Equilibrate the matrix.
*
            CALL CLAQGE( N, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $           EQUED )
            ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
            COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         END IF
*
*     If the scaling factors are not applied, set them to 1.0.
*
         IF ( .NOT.ROWEQU ) THEN
            DO J = 1, N
```

```
            R( J ) = 1.0
         END DO
      END IF
      IF ( .NOT.COLEQU ) THEN
         DO J = 1, N
            C( J ) = 1.0
         END DO
      END IF
   END IF
*
*     Scale the right-hand side.
*
   IF( NOTRAN ) THEN
      IF( ROWEQU ) CALL CLASCL2( N, NRHS, R, B, LDB )
   ELSE
      IF( COLEQU ) CALL CLASCL2( N, NRHS, C, B, LDB )
   END IF
*
   IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the LU factorization of A.
*
      CALL CLACPY( 'Full', N, N, A, LDA, AF, LDAF )
      CALL CGETRF( N, N, AF, LDAF, IPIV, INFO )
*
*        Return if INFO is non-zero.
*
      IF( INFO.GT.0 ) THEN
*
*          Pivot in column INFO is exactly 0
*          Compute the reciprocal pivot growth factor of the
*          leading rank-deficient INFO columns of A.
*
         RPVGRW = CLA_GERPVGRW( N, INFO, A, LDA, AF, LDAF )
         RETURN
      END IF
   END IF
*
*     Compute the reciprocal pivot growth factor RPVGRW.
*
   RPVGRW = CLA_GERPVGRW( N, N, A, LDA, AF, LDAF )
*
*     Compute the solution matrix X.
*
   CALL CLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
   CALL CGETRS( TRANS, N, NRHS, AF, LDAF, IPIV, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
```

```
      CALL CGERFSX( TRANS, EQUED, N, NRHS, A, LDA, AF, LDAF,
     $     IPIV, R, C, B, LDB, X, LDX, RCOND, BERR,
     $     N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $     WORK, RWORK, INFO )
*
*     Scale solutions.
*
      IF ( COLEQU .AND. NOTRAN ) THEN
         CALL CLASCL2 ( N, NRHS, C, X, LDX )
      ELSE IF ( ROWEQU .AND. .NOT.NOTRAN ) THEN
         CALL CLASCL2 ( N, NRHS, R, X, LDX )
      END IF
*
      RETURN
*
*     End of CGESVXX
*
      END
```

22.8.4 lisp code

— cgesvxx.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun cgesvxx
         (fact trans n nrhs a lda af ldaf ipiv equed r c b ldb$ x ldx rcond
          rpvgrw berr n_err_bnds err_bnds_norm err_bnds_comp nparams params
          work rwork info)
    (declare (type (single-float) rpvgrw rcond)
             (type (array single-float (*)) rwork params err_bnds_comp
              err_bnds_norm berr c r)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) work x b af a)
             (type (f2cl-lib:integer4) info nparams n_err_bnds ldx ldb$ ldaf
              lda nrhs n)
             (type (string *) equed trans fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (trans character trans-%data% trans-%offset%)
         (equed character equed-%data% equed-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (af f2cl-lib:complex8 af-%data% af-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
```

```
    (x f2cl-lib:complex8 x-%data% x-%offset%)
    (work f2cl-lib:complex8 work-%data% work-%offset%)
    (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
    (r single-float r-%data% r-%offset%)
    (c single-float c-%data% c-%offset%)
    (berr single-float berr-%data% berr-%offset%)
    (err_bnds_norm single-float err_bnds_norm-%data%
     err_bnds_norm-%offset%)
    (err_bnds_comp single-float err_bnds_comp-%data%
     err_bnds_comp-%offset%)
    (params single-float params-%data% params-%offset%)
    (rwork single-float rwork-%data% rwork-%offset%))
  (prog ((amax 0.0) (bignum 0.0) (colcnd 0.0) (rcmax 0.0) (rcmin 0.0)
         (rowcnd 0.0) (smlnum 0.0) (infequ 0) (j 0) (colequ nil)
         (equil nil) (nofact nil) (notran nil) (rowequ nil))
    (declare
     (type (single-float) amax bignum colcnd rcmax rcmin rowcnd smlnum)
     (type (f2cl-lib:integer4) infequ j)
     (type f2cl-lib:logical colequ equil nofact notran rowequ))
    (setf info 0)
    (setf nofact
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "N")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val))
    (setf equil
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "E")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val))
    (setf notran
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "N")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val))
    (setf smlnum (slamch "Safe minimum"))
    (setf bignum (/ one smlnum))
    (cond
     ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
      (setf rowequ f2cl-lib:%false%) (setf colequ f2cl-lib:%false%))
     (t
      (setf rowequ
              (or
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame equed "R")
                 (declare (ignore var-1))
                 (when var-0 (setf equed var-0))
```

```
                ret-val)
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame equed "B")
                (declare (ignore var-1))
                (when var-0 (setf equed var-0))
                ret-val)))
  (setf colequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "C")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)))))
(setf rpvgrw zero)
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame fact "F")
```

```
      (declare (ignore var-1))
      (when var-0 (setf fact var-0))
      ret-val)
    (not
     (or rowequ colequ
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame equed "N")
           (declare (ignore var-1))
           (when var-0 (setf equed var-0))
           ret-val))))
  (setf info -10))
 (t
  (cond
   (rowequ (setf rcmin bignum) (setf rcmax zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf rcmin
                (min rcmin
                     (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
        (setf rcmax
                (max rcmax
                     (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
       label10))
    (cond ((<= rcmin zero) (setf info -11))
          ((> n 0)
           (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum))))
          (t (setf rowcnd one)))))
  (cond
   ((and colequ (= info 0)) (setf rcmin bignum) (setf rcmax zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf rcmin
                (min rcmin
                     (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
        (setf rcmax
                (max rcmax
                     (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
       label20))
    (cond ((<= rcmin zero) (setf info -12))
          ((> n 0)
           (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum))))
          (t (setf colcnd one)))))
  (cond
   ((= info 0)
    (cond
     ((< ldb$
         (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -14))
```

```
      ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -16)))))))
(cond
 ((/= info 0) (xerbla "CGESVXX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (equil
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (cgeequb n n a lda r c rowcnd colcnd amax infequ)
    (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5))
    (setf rowcnd var-6)
    (setf colcnd var-7)
    (setf amax var-8)
    (setf infequ var-9))
  (cond
   ((= infequ 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (claqge n n a lda r c rowcnd colcnd amax equed)
      (declare (ignore var-2 var-4 var-5))
      (when var-0 (setf n var-0))
      (when var-1 (setf n var-1))
      (when var-3 (setf lda var-3))
      (when var-6 (setf rowcnd var-6))
      (when var-7 (setf colcnd var-7))
      (when var-8 (setf amax var-8))
      (when var-9 (setf equed var-9)))
    (setf rowequ
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "R")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "B")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)))
    (setf colequ
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "C")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "B")
               (declare (ignore var-1))
```

```
                (when var-0 (setf equed var-0))
                ret-val)))))
  (cond
   ((not rowequ)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%) 1.0)
       label100000))))
  (cond
   ((not colequ)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) 1.0)
       label100001))))))
(cond
 (notran
  (if rowequ
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (clascl2 n nrhs r b ldb$)
        (declare (ignore var-2 var-3))
        (when var-0 (setf n var-0))
        (when var-1 (setf nrhs var-1))
        (when var-4 (setf ldb$ var-4)))))
 (t
  (if colequ
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (clascl2 n nrhs c b ldb$)
        (declare (ignore var-2 var-3))
        (when var-0 (setf n var-0))
        (when var-1 (setf nrhs var-1))
        (when var-4 (setf ldb$ var-4))))))
(cond
 ((or nofact equil)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "Full" n n a lda af ldaf)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldaf var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (cgetrf n n af ldaf ipiv info)
    (declare (ignore var-2 var-4))
    (when var-0 (setf n var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf ldaf var-3))
    (when var-5 (setf info var-5)))
  (cond
```

```
    ((> info 0)
     (setf rpvgrw
             (multiple-value-bind
                 (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                 (cla_gerpvgrw n info a lda af ldaf)
               (declare (ignore var-2 var-4))
               (when var-0 (setf n var-0))
               (when var-1 (setf info var-1))
               (when var-3 (setf lda var-3))
               (when var-5 (setf ldaf var-5))
               ret-val))
     (go end_label)))))
(setf rpvgrw
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (cla_gerpvgrw n n a lda af ldaf)
          (declare (ignore var-2 var-4))
          (when var-0 (setf n var-0))
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-5 (setf ldaf var-5))
          ret-val))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "Full" n nrhs b ldb$ x ldx)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf ldx var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cgetrs trans n nrhs af ldaf ipiv x ldx info)
  (declare (ignore var-3 var-5 var-6))
  (when var-0 (setf trans var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldaf var-4))
  (when var-7 (setf ldx var-7))
  (when var-8 (setf info var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19
     var-20 var-21 var-22 var-23 var-24)
    (cgerfsx trans equed n nrhs a lda af ldaf ipiv r c b ldb$ x ldx
     rcond berr n_err_bnds err_bnds_norm err_bnds_comp nparams params
     work rwork info)
  (declare
   (ignore var-4 var-6 var-8 var-9 var-10 var-11 var-13 var-16 var-18
    var-19 var-21 var-22 var-23))
  (when var-0 (setf trans var-0))
```

```
          (when var-1 (setf equed var-1))
          (when var-2 (setf n var-2))
          (when var-3 (setf nrhs var-3))
          (when var-5 (setf lda var-5))
          (when var-7 (setf ldaf var-7))
          (when var-12 (setf ldb$ var-12))
          (when var-14 (setf ldx var-14))
          (when var-15 (setf rcond var-15))
          (when var-17 (setf n_err_bnds var-17))
          (when var-20 (setf nparams var-20))
          (when var-24 (setf info var-24)))
        (cond
         ((and colequ notran)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (clascl2 n nrhs c x ldx)
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf nrhs var-1))
            (when var-4 (setf ldx var-4))))
         ((and rowequ (not notran))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (clascl2 n nrhs r x ldx)
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf nrhs var-1))
            (when var-4 (setf ldx var-4)))))
        (go end_label)
       end_label
        (return
         (values fact trans n nrhs nil lda nil ldaf nil equed nil nil nil ldb$
                 nil ldx rcond rpvgrw nil n_err_bnds nil nil nparams nil nil
                 nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgesvxx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
```

```
  (string) (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (single-float) (single-float)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array fortran-to-lisp::complex8
   (*))
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::fact
  fortran-to-lisp::trans
  fortran-to-lisp::n
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldaf nil
  fortran-to-lisp::equed nil nil
  nil fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond
  fortran-to-lisp::rpvgrw nil
  fortran-to-lisp::n_err_bnds nil
  nil fortran-to-lisp::nparams nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::cgeequb))))
```

Chapter 23

LAPACK - General Matrices, Linear Solve, Complex16

23.1 zcgesv

23.1.1 Axiom unit tests

— zcgesv.input —

```
)set break resume
)sys rm -f zcgesv.output
)spool zcgesv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

23.1.2 Axiom help page

— zcgesv.help —

```
==========================================================================
zcgesv examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zcgesv (N, NRHS, A, LDA, IPIV, B, LDB, X, LDX, WORK, SWORK,
           RWORK, ITER, INFO)
            ZCGESV computes the solution to system of linear equations A * X =
           B for GE matrices (mixed precision with iterative refinement)

Function/Subroutine Documentation
   subroutine zcgesv (integer N, integer NRHS, complex*16, dimension( lda, * )
       A, integer LDA, integer, dimension( * ) IPIV, complex*16, dimension(
       ldb, * ) B, integer LDB, complex*16, dimension( ldx, * ) X, integer
       LDX, complex*16, dimension( n, * ) WORK, complex, dimension( * ) SWORK,
       double precision, dimension( * ) RWORK, integer ITER, integer INFO)
        ZCGESV computes the solution to system of linear equations A * X = B
       for GE matrices (mixed precision with iterative refinement)

Purpose:
   ZCGESV computes the solution to a complex system of linear equations
              A * X = B,
   where A is an N-by-N matrix and X and B are N-by-NRHS matrices.

   ZCGESV first attempts to factorize the matrix in COMPLEX and use this
   factorization within an iterative refinement procedure to produce a
   solution with COMPLEX*16 normwise backward error quality (see below).
   If the approach fails the method switches to a COMPLEX*16
   factorization and solve.

   The iterative refinement is not going to be a winning strategy if
   the ratio COMPLEX performance over COMPLEX*16 performance is too
   small. A reasonable strategy should take the number of right-hand
   sides and the size of the matrix into account. This might be done
   with a call to ILAENV in the future. Up to now, we always try
   iterative refinement.

   The iterative refinement process is stopped if
         ITER > ITERMAX
   or for all the RHS we have:
         RNRM < SQRT(N)*XNRM*ANRM*EPS*BWDMAX
   where
   o ITER is the number of the current iteration in the iterative
   refinement process
   o RNRM is the infinity-norm of the residual
   o XNRM is the infinity-norm of the solution
   o ANRM is the infinity-operator-norm of the matrix A
   o EPS is the machine epsilon returned by DLAMCH('Epsilon')
```

```
   The value ITERMAX and BWDMAX are fixed to 30 and 1.0D+00 respectively.

Parameters:

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is COMPLEX*16 array,
   dimension (LDA,N)
   On entry, the N-by-N coefficient matrix A.
   On exit, if iterative refinement has been successfully used
   (INFO.EQ.0 and ITER.GE.0, see description below), then A is
   unchanged, if double precision factorization has been used
   (INFO.EQ.0 and ITER.LT.0, see description below), then the
   array A contains the factors L and U from the factorization
   A = P*L*U; the unit diagonal elements of L are not stored.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   The pivot indices that define the permutation matrix P;
   row i of the matrix was interchanged with row IPIV(i).
   Corresponds either to the single precision factorization
   (if INFO.EQ.0 and ITER.GE.0) or the double precision
   factorization (if INFO.EQ.0 and ITER.LT.0).

 B
   B is COMPLEX*16 array, dimension (LDB,NRHS)
   The N-by-NRHS right hand side matrix B.

 LDB
   LDB is INTEGER
   The leading dimension of the array B.  LDB >= max(1,N).

 X
   X is COMPLEX*16 array, dimension (LDX,NRHS)
   If INFO = 0, the N-by-NRHS solution matrix X.

 LDX
   LDX is INTEGER
```

```
  The leading dimension of the array X.  LDX >= max(1,N).

WORK
  WORK is COMPLEX*16 array, dimension (N*NRHS)
  This array is used to hold the residual vectors.

SWORK
  SWORK is COMPLEX array, dimension (N*(N+NRHS))
  This array is used to use the single precision matrix and the
  right-hand sides or solutions in single precision.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (N)

ITER
  ITER is INTEGER
  < 0: iterative refinement has failed, COMPLEX*16
       factorization has been performed
  -1 : the routine fell back to full precision for
       implementation- or machine-specific reasons
  -2 : narrowing the precision induced an overflow,
       the routine fell back to full precision
  -3 : failure of CGETRF
  -31: stop the iterative refinement after the 30th
       iterations
  > 0: iterative refinement has been sucessfully used.
       Returns the number of iterations

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, U(i,i) computed in COMPLEX*16 is exactly
        zero.  The factorization has been completed, but the
        factor U is exactly singular, so the solution
        could not be computed.
```

———-

23.1.3 fortran code

— **zcgesv.f** —

```
*  =====================================================================
      SUBROUTINE ZCGESV( N, NRHS, A, LDA, IPIV, B, LDB, X, LDX, WORK,
     $                   SWORK, RWORK, ITER, INFO )
*
```

```
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, ITER, LDA, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   RWORK( * )
      COMPLEX            SWORK( * )
      COMPLEX*16         A( LDA, * ), B( LDB, * ), WORK( N, * ),
     $                   X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      LOGICAL            DOITREF
      PARAMETER          ( DOITREF = .TRUE. )
*
      INTEGER            ITERMAX
      PARAMETER          ( ITERMAX = 30 )
*
      DOUBLE PRECISION   BWDMAX
      PARAMETER          ( BWDMAX = 1.0E+00 )
*
      COMPLEX*16         NEGONE, ONE
      PARAMETER          ( NEGONE = ( -1.0D+00, 0.0D+00 ),
     $                   ONE = ( 1.0D+00, 0.0D+00 ) )
*
*     .. Local Scalars ..
      INTEGER            I, IITER, PTSA, PTSX
      DOUBLE PRECISION   ANRM, CTE, EPS, RNRM, XNRM
      COMPLEX*16         ZDUM
*
*     .. External Subroutines ..
      EXTERNAL           CGETRS, CGETRF, CLAG2Z, XERBLA, ZAXPY, ZGEMM,
     $                   ZLACPY, ZLAG2C
*     ..
*     .. External Functions ..
      INTEGER            IZAMAX
      DOUBLE PRECISION   DLAMCH, ZLANGE
      EXTERNAL           IZAMAX, DLAMCH, ZLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, MAX, SQRT
*     ..
*     .. Statement Functions ..
```

```
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      ITER = 0
*
*     Test the input parameters.
*
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZCGESV', -INFO )
         RETURN
      END IF
*
*     Quick return if (N.EQ.0).
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Skip single precision iterative refinement if a priori slower
*     than double precision factorization.
*
      IF( .NOT.DOITREF ) THEN
         ITER = -1
         GO TO 40
      END IF
*
*     Compute some constants.
*
      ANRM = ZLANGE( 'I', N, N, A, LDA, RWORK )
      EPS = DLAMCH( 'Epsilon' )
      CTE = ANRM*EPS*SQRT( DBLE( N ) )*BWDMAX
*
*     Set the indices PTSA, PTSX for referencing SA and SX in SWORK.
*
      PTSA = 1
```

```
      PTSX = PTSA + N*N
*
*     Convert B from double precision to single precision and store the
*     result in SX.
*
      CALL ZLAG2C( N, NRHS, B, LDB, SWORK( PTSX ), N, INFO )
*
      IF( INFO.NE.0 ) THEN
         ITER = -2
         GO TO 40
      END IF
*
*     Convert A from double precision to single precision and store the
*     result in SA.
*
      CALL ZLAG2C( N, N, A, LDA, SWORK( PTSA ), N, INFO )
*
      IF( INFO.NE.0 ) THEN
         ITER = -2
         GO TO 40
      END IF
*
*     Compute the LU factorization of SA.
*
      CALL CGETRF( N, N, SWORK( PTSA ), N, IPIV, INFO )
*
      IF( INFO.NE.0 ) THEN
         ITER = -3
         GO TO 40
      END IF
*
*     Solve the system SA*SX = SB.
*
      CALL CGETRS( 'No transpose', N, NRHS, SWORK( PTSA ), N, IPIV,
     $             SWORK( PTSX ), N, INFO )
*
*     Convert SX back to double precision
*
      CALL CLAG2Z( N, NRHS, SWORK( PTSX ), N, X, LDX, INFO )
*
*     Compute R = B - AX (R is WORK).
*
      CALL ZLACPY( 'All', N, NRHS, B, LDB, WORK, N )
*
      CALL ZGEMM( 'No Transpose', 'No Transpose', N, NRHS, N, NEGONE, A,
     $            LDA, X, LDX, ONE, WORK, N )
*
*     Check whether the NRHS normwise backward errors satisfy the
*     stopping criterion. If yes, set ITER=0 and return.
*
```

```
      DO I = 1, NRHS
         XNRM = CABS1( X( IZAMAX( N, X( 1, I ), 1 ), I ) )
         RNRM = CABS1( WORK( IZAMAX( N, WORK( 1, I ), 1 ), I ) )
         IF( RNRM.GT.XNRM*CTE )
     $      GO TO 10
      END DO
*
*     If we are here, the NRHS normwise backward errors satisfy the
*     stopping criterion. We are good to exit.
*
      ITER = 0
      RETURN
*
   10 CONTINUE
*
      DO 30 IITER = 1, ITERMAX
*
*        Convert R (in WORK) from double precision to single precision
*        and store the result in SX.
*
         CALL ZLAG2C( N, NRHS, WORK, N, SWORK( PTSX ), N, INFO )
*
         IF( INFO.NE.0 ) THEN
            ITER = -2
            GO TO 40
         END IF
*
*        Solve the system SA*SX = SR.
*
         CALL CGETRS( 'No transpose', N, NRHS, SWORK( PTSA ), N, IPIV,
     $                SWORK( PTSX ), N, INFO )
*
*        Convert SX back to double precision and update the current
*        iterate.
*
         CALL CLAG2Z( N, NRHS, SWORK( PTSX ), N, WORK, N, INFO )
*
         DO I = 1, NRHS
            CALL ZAXPY( N, ONE, WORK( 1, I ), 1, X( 1, I ), 1 )
         END DO
*
*        Compute R = B - AX (R is WORK).
*
         CALL ZLACPY( 'All', N, NRHS, B, LDB, WORK, N )
*
         CALL ZGEMM( 'No Transpose', 'No Transpose', N, NRHS, N, NEGONE,
     $               A, LDA, X, LDX, ONE, WORK, N )
*
*        Check whether the NRHS normwise backward errors satisfy the
*        stopping criterion. If yes, set ITER=IITER>0 and return.
```

```
*
         DO I = 1, NRHS
            XNRM = CABS1( X( IZAMAX( N, X( 1, I ), 1 ), I ) )
            RNRM = CABS1( WORK( IZAMAX( N, WORK( 1, I ), 1 ), I ) )
            IF( RNRM.GT.XNRM*CTE )
     $         GO TO 20
         END DO
*
*        If we are here, the NRHS normwise backward errors satisfy the
*        stopping criterion, we are good to exit.
*
         ITER = IITER
*
         RETURN
*
   20    CONTINUE
*
   30 CONTINUE
*
*     If we are at this place of the code, this is because we have
*     performed ITER=ITERMAX iterations and never satisified the stopping
*     criterion, set up the ITER flag accordingly and follow up on double
*     precision routine.
*
      ITER = -ITERMAX - 1
*
   40 CONTINUE
*
*     Single-precision iterative refinement failed to converge to a
*     satisfactory solution, so we resort to double precision.
*
      CALL ZGETRF( N, N, A, LDA, IPIV, INFO )
*
      IF( INFO.NE.0 )
     $   RETURN
*
      CALL ZLACPY( 'All', N, NRHS, B, LDB, X, LDX )
      CALL ZGETRS( 'No transpose', N, NRHS, A, LDA, IPIV, X, LDX,
     $             INFO )
*
      RETURN
*
*     End of ZCGESV.
*
      END
```

———-

23.1.4 lisp code

— zcgesv.lisp —

```
(let* ((doitref f2cl-lib:%true%)
       (itermax 30)
       (bwdmax (coerce 1.0 'double-float))
       (negone (f2cl-lib:cmplx (- 1.0d0) 0.0d0))
       (one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type f2cl-lib:logical doitref)
           (type (f2cl-lib:integer4 30 30) itermax)
           (type (double-float) bwdmax)
           (type (f2cl-lib:complex16) negone)
           (type (f2cl-lib:complex16) one)
           (ignorable doitref itermax bwdmax negone one))
  (defun zcgesv (n nrhs a lda ipiv b ldb$ x ldx work swork rwork iter info)
    (declare (type (array double-float (*)) rwork)
             (type (array f2cl-lib:complex8 (*)) swork)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) work x b a)
             (type (f2cl-lib:integer4) info iter ldx ldb$ lda nrhs n))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (x f2cl-lib:complex16 x-%data% x-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (swork f2cl-lib:complex8 swork-%data% swork-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
          cabs1))
        (prog ((zdum #C(0.0d0 0.0d0)) (anrm 0.0d0) (cte 0.0d0) (eps 0.0d0)
               (rnrm 0.0d0) (xnrm 0.0d0) (i 0) (iiter 0) (ptsa 0) (ptsx 0))
          (declare (type (f2cl-lib:complex16) zdum)
                   (type (double-float) anrm cte eps rnrm xnrm)
                   (type (f2cl-lib:integer4) i iiter ptsa ptsx))
          (setf info 0)
          (setf iter 0)
          (cond ((< n 0) (setf info -1)) ((< nrhs 0) (setf info -2))
                ((< lda
                    (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
                 (setf info -4))
                ((< ldb$
                    (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
                 (setf info -7))
                ((< ldx
```

```
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -9)))
(cond
 ((/= info 0) (xerbla "ZCGESV" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(cond ((not doitref) (setf iter -1) (go label40)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "I" n n a lda rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf eps (dlamch "Epsilon"))
(setf cte (* anrm eps (f2cl-lib:fsqrt (f2cl-lib:dble n)) bwdmax))
(setf ptsa 1)
(setf ptsx (f2cl-lib:int-add ptsa (f2cl-lib:int-mul n n)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlag2c n nrhs b ldb$
     (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsx)
                           ((1 *)) swork-%offset%)
     n info)
  (declare (ignore var-2 var-4))
  (when var-0 (setf n var-0))
  (when var-1 (setf nrhs var-1))
  (when var-3 (setf ldb$ var-3))
  (when var-5 (setf n var-5))
  (when var-6 (setf info var-6)))
(cond ((/= info 0) (setf iter -2) (go label40)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlag2c n n a lda
     (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsa)
                           ((1 *)) swork-%offset%)
     n info)
  (declare (ignore var-2 var-4))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-5 (setf n var-5))
  (when var-6 (setf info var-6)))
(cond ((/= info 0) (setf iter -2) (go label40)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (cgetrf n n
     (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsa)
                           ((1 *)) swork-%offset%)
     n ipiv info)
```

```
  (declare (ignore var-2 var-4))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf n var-3))
  (when var-5 (setf info var-5)))
(cond ((/= info 0) (setf iter -3) (go label40)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cgetrs "No transpose" n nrhs
     (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsa)
                           ((1 *)) swork-%offset%)
     n ipiv
     (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsx)
                           ((1 *)) swork-%offset%)
     n info)
  (declare (ignore var-0 var-3 var-5 var-6))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf n var-4))
  (when var-7 (setf n var-7))
  (when var-8 (setf info var-8)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clag2z n nrhs
     (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsx)
                           ((1 *)) swork-%offset%)
     n x ldx info)
  (declare (ignore var-2 var-4))
  (when var-0 (setf n var-0))
  (when var-1 (setf nrhs var-1))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldx var-5))
  (when var-6 (setf info var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "All" n nrhs b ldb$ work n)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf n var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (zgemm "No Transpose" "No Transpose" n nrhs n negone a lda x ldx
     one work n)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf n var-2))
  (when var-3 (setf nrhs var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf negone var-5))
  (when var-7 (setf lda var-7))
```

```
      (when var-9 (setf ldx var-9))
      (when var-10 (setf one var-10))
      (when var-12 (setf n var-12)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i nrhs) nil)
      (tagbody
        (setf xnrm
                (cabs1
                 (f2cl-lib:fref x-%data%
                                ((multiple-value-bind
                                     (ret-val var-0 var-1 var-2)
                                     (izamax n
                                      (f2cl-lib:array-slice x
                                                            f2cl-lib:complex16
                                                            (1 i)
                                                            ((1 ldx)
                                                             (1 *)))
                                      1)
                                   (declare (ignore var-1 var-2))
                                   (when var-0 (setf n var-0))
                                   ret-val)
                                 i)
                                ((1 ldx) (1 *)) x-%offset%)))
        (setf rnrm
                (cabs1
                 (f2cl-lib:fref work-%data%
                                ((multiple-value-bind
                                     (ret-val var-0 var-1 var-2)
                                     (izamax n
                                      (f2cl-lib:array-slice work
                                                            f2cl-lib:complex16
                                                            (1 i)
                                                            ((1 n)
                                                             (1 *)))
                                      1)
                                   (declare (ignore var-1 var-2))
                                   (when var-0 (setf n var-0))
                                   ret-val)
                                 i)
                                ((1 n) (1 *)) work-%offset%)))
        (if (> rnrm (* xnrm cte))
            (go label10))
       label100000))
    (setf iter 0)
    (go end_label)
   label10
    (f2cl-lib:fdo (iiter 1 (f2cl-lib:int-add iiter 1))
                  ((> iiter itermax) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
    (zlag2c n nrhs work n
     (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsx)
                           ((1 *)) swork-%offset%)
     n info)
  (declare (ignore var-2 var-4))
  (when var-0 (setf n var-0))
  (when var-1 (setf nrhs var-1))
  (when var-3 (setf n var-3))
  (when var-5 (setf n var-5))
  (when var-6 (setf info var-6)))
(cond ((/= info 0) (setf iter -2) (go label40)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cgetrs "No transpose" n nrhs
     (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsa)
                           ((1 *)) swork-%offset%)
     n ipiv
     (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsx)
                           ((1 *)) swork-%offset%)
     n info)
  (declare (ignore var-0 var-3 var-5 var-6))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf n var-4))
  (when var-7 (setf n var-7))
  (when var-8 (setf info var-8)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clag2z n nrhs
     (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsx)
                           ((1 *)) swork-%offset%)
     n work n info)
  (declare (ignore var-2 var-4))
  (when var-0 (setf n var-0))
  (when var-1 (setf nrhs var-1))
  (when var-3 (setf n var-3))
  (when var-5 (setf n var-5))
  (when var-6 (setf info var-6)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nrhs) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (zaxpy n one
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (1 i) ((1 n) (1 *)) work-%offset%)
         1
         (f2cl-lib:array-slice x-%data% f2cl-lib:complex16 (1 i)
                               ((1 ldx) (1 *)) x-%offset%)
         1)
      (declare (ignore var-2 var-3 var-4 var-5))
      (when var-0 (setf n var-0))
```

```
      (when var-1 (setf one var-1)))
   label100001))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "All" n nrhs b ldb$ work n)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf n var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (zgemm "No Transpose" "No Transpose" n nrhs n negone a lda x
     ldx one work n)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf n var-2))
  (when var-3 (setf nrhs var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf negone var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldx var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf n var-12)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nrhs) nil)
  (tagbody
    (setf xnrm
            (cabs1
             (f2cl-lib:fref x-%data%
                            ((multiple-value-bind
                                 (ret-val var-0 var-1 var-2)
                                 (izamax n
                                  (f2cl-lib:array-slice x
                                                        f2cl-lib:complex16
                                                        (1 i)
                                                        ((1 ldx)
                                                         (1 *)))
                                  1)
                                (declare (ignore var-1 var-2))
                                (when var-0 (setf n var-0))
                                ret-val)
                             i)
                            ((1 ldx) (1 *)) x-%offset%)))
    (setf rnrm
            (cabs1
             (f2cl-lib:fref work-%data%
                            ((multiple-value-bind
                                 (ret-val var-0 var-1 var-2)
                                 (izamax n
                                  (f2cl-lib:array-slice work
```

```
                                                                    f2cl-lib:complex16
                                                                    (1 i)
                                                                    ((1 n)
                                                                     (1 *)))
                                                 1)
                                              (declare (ignore var-1 var-2))
                                              (when var-0 (setf n var-0))
                                              ret-val)
                                            i)
                                           ((1 n) (1 *)) work-%offset%)))
                    (if (> rnrm (* xnrm cte))
                        (go label20))
                   label100002))
              (setf iter iiter)
              (go end_label)
            label20
            label30))
         (setf iter (f2cl-lib:int-sub -1 itermax))
        label40
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
             (zgetrf n n a lda ipiv info)
           (declare (ignore var-2 var-4))
           (when var-0 (setf n var-0))
           (when var-1 (setf n var-1))
           (when var-3 (setf lda var-3))
           (when var-5 (setf info var-5)))
         (if (/= info 0)
             (go end_label))
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
             (zlacpy "All" n nrhs b ldb$ x ldx)
           (declare (ignore var-0 var-3 var-5))
           (when var-1 (setf n var-1))
           (when var-2 (setf nrhs var-2))
           (when var-4 (setf ldb$ var-4))
           (when var-6 (setf ldx var-6)))
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
             (zgetrs "No transpose" n nrhs a lda ipiv x ldx info)
           (declare (ignore var-0 var-3 var-5 var-6))
           (when var-1 (setf n var-1))
           (when var-2 (setf nrhs var-2))
           (when var-4 (setf lda var-4))
           (when var-7 (setf ldx var-7))
           (when var-8 (setf info var-8)))
         (go end_label)
        end_label
         (return
          (values n nrhs nil lda nil nil ldb$ nil ldx nil nil nil iter
                  info)))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zcgesv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::ldb$ nil
                                              fortran-to-lisp::ldx nil nil nil
                                              fortran-to-lisp::iter
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

23.2 zgels

23.2.1 Axiom unit tests

— zgels.input —

```
)set break resume
```

```
)sys rm -f zgels.output
)spool zgels.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

23.2.2 Axiom help page

— **zgels.help** —

```
=====================================================================
zgels examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgels (TRANS, M, N, NRHS, A, LDA, B, LDB, WORK, LWORK, INFO)
            ZGELS solves overdetermined or underdetermined systems for GE
           matrices

Function/Subroutine Documentation
   subroutine zgels (character TRANS, integer M, integer N, integer NRHS,
       complex*16, dimension( lda, * ) A, integer LDA, complex*16, dimension(
       ldb, * ) B, integer LDB, complex*16, dimension( * ) WORK, integer
       LWORK, integer INFO)
        ZGELS solves overdetermined or underdetermined systems for GE matrices

Purpose:
   1. If TRANS = 'N' and m >= n:  find the least squares solution of
      an overdetermined system, i.e., solve the least squares problem
          minimize || B - A*X ||.

   2. If TRANS = 'N' and m < n:  find the minimum norm solution of
      an underdetermined system A * X = B.

   3. If TRANS = 'C' and m >= n:  find the minimum norm solution of
      an undetermined system A**H * X = B.

   4. If TRANS = 'C' and m < n:  find the least squares solution of
```

```
      an overdetermined system, i.e., solve the least squares problem
           minimize || B - A**H * X ||.

   Several right hand side vectors b and solution vectors x can be
   handled in a single call; they are stored as the columns of the
   M-by-NRHS right hand side matrix B and the N-by-NRHS solution matrix X.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   = 'N': the linear system involves A;
   = 'C': the linear system involves A**H.

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of
   columns of the matrices B and X. NRHS >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   if M >= N, A is overwritten by details of its QR
              factorization as returned by ZGEQRF;
   if M <  N, A is overwritten by details of its LQ
              factorization as returned by ZGELQF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 B
   B is COMPLEX*16 array, dimension (LDB,NRHS)
   On entry, the matrix B of right hand side vectors, stored
   columnwise; B is M-by-NRHS if TRANS = 'N', or N-by-NRHS
   if TRANS = 'C'.
   On exit, if INFO = 0, B is overwritten by the solution
   vectors, stored columnwise:
   if TRANS = 'N' and m >= n, rows 1 to n of B contain the least
   squares solution vectors; the residual sum of squares for the
   solution in each column is given by the sum of squares of the
   modulus of elements N+1 to M in that column;
```

```
  if TRANS = 'N' and m < n, rows 1 to N of B contain the
  minimum norm solution vectors;
  if TRANS = 'C' and m >= n, rows 1 to M of B contain the
  minimum norm solution vectors;
  if TRANS = 'C' and m < n, rows 1 to M of B contain the
  least squares solution vectors; the residual sum of squares
  for the solution in each column is given by the sum of
  squares of the modulus of elements M+1 to N in that column.

LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= MAX(1,M,N).

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  LWORK >= max( 1, MN + max( MN, NRHS ) ).
  For optimal performance,
  LWORK >= max( 1, MN + max( MN, NRHS )*NB ).
  where MN = min(M,N) and NB is the optimum block size.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO =  i, the i-th diagonal element of the
        triangular factor of A is zero, so that A does not have
        full rank; the least squares solution could not be computed.
```

———-

23.2.3 fortran code

— **zgels.f** —

```
*  =====================================================================
      SUBROUTINE ZGELS( TRANS, M, N, NRHS, A, LDA, B, LDB, WORK, LWORK,
     $                  INFO )
```

```
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDA, LDB, LWORK, M, N, NRHS
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      COMPLEX*16         CZERO
      PARAMETER          ( CZERO = ( 0.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, TPSD
      INTEGER            BROW, I, IASCL, IBSCL, J, MN, NB, SCLLEN, WSIZE
      DOUBLE PRECISION   ANRM, BIGNUM, BNRM, SMLNUM
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   RWORK( 1 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, ZLANGE
      EXTERNAL           LSAME, ILAENV, DLAMCH, ZLANGE
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, XERBLA, ZGELQF, ZGEQRF, ZLASCL, ZLASET,
     $                   ZTRTRS, ZUNMLQ, ZUNMQR
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      MN = MIN( M, N )
      LQUERY = ( LWORK.EQ.-1 )
```

```
      IF( .NOT.( LSAME( TRANS, 'N' ) .OR. LSAME( TRANS, 'C' ) ) ) THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -6
      ELSE IF( LDB.LT.MAX( 1, M, N ) ) THEN
         INFO = -8
      ELSE IF( LWORK.LT.MAX( 1, MN+MAX( MN, NRHS ) ) .AND. .NOT.LQUERY )
     $          THEN
         INFO = -10
      END IF
*
*     Figure out optimal block size
*
      IF( INFO.EQ.0 .OR. INFO.EQ.-10 ) THEN
*
         TPSD = .TRUE.
         IF( LSAME( TRANS, 'N' ) )
     $      TPSD = .FALSE.
*
         IF( M.GE.N ) THEN
            NB = ILAENV( 1, 'ZGEQRF', ' ', M, N, -1, -1 )
            IF( TPSD ) THEN
               NB = MAX( NB, ILAENV( 1, 'ZUNMQR', 'LN', M, NRHS, N,
     $              -1 ) )
            ELSE
               NB = MAX( NB, ILAENV( 1, 'ZUNMQR', 'LC', M, NRHS, N,
     $              -1 ) )
            END IF
         ELSE
            NB = ILAENV( 1, 'ZGELQF', ' ', M, N, -1, -1 )
            IF( TPSD ) THEN
               NB = MAX( NB, ILAENV( 1, 'ZUNMLQ', 'LC', N, NRHS, M,
     $              -1 ) )
            ELSE
               NB = MAX( NB, ILAENV( 1, 'ZUNMLQ', 'LN', N, NRHS, M,
     $              -1 ) )
            END IF
         END IF
*
         WSIZE = MAX( 1, MN+MAX( MN, NRHS )*NB )
         WORK( 1 ) = DBLE( WSIZE )
*
      END IF
*
```

```
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGELS ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( MIN( M, N, NRHS ).EQ.0 ) THEN
         CALL ZLASET( 'Full', MAX( M, N ), NRHS, CZERO, CZERO, B, LDB )
         RETURN
      END IF
*
*     Get machine parameters
*
      SMLNUM = DLAMCH( 'S' ) / DLAMCH( 'P' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
*
*     Scale A, B if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = ZLANGE( 'M', M, N, A, LDA, RWORK )
      IASCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL ZLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
         IASCL = 1
      ELSE IF( ANRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL ZLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
         IASCL = 2
      ELSE IF( ANRM.EQ.ZERO ) THEN
*
*        Matrix all zero. Return zero solution.
*
         CALL ZLASET( 'F', MAX( M, N ), NRHS, CZERO, CZERO, B, LDB )
         GO TO 50
      END IF
*
      BROW = M
      IF( TPSD )
     $   BROW = N
      BNRM = ZLANGE( 'M', BROW, NRHS, B, LDB, RWORK )
      IBSCL = 0
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
```

```
*
*        Scale matrix norm up to SMLNUM
*
         CALL ZLASCL( 'G', 0, 0, BNRM, SMLNUM, BROW, NRHS, B, LDB,
     $                INFO )
         IBSCL = 1
      ELSE IF( BNRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL ZLASCL( 'G', 0, 0, BNRM, BIGNUM, BROW, NRHS, B, LDB,
     $                INFO )
         IBSCL = 2
      END IF
*
      IF( M.GE.N ) THEN
*
*        compute QR factorization of A
*
         CALL ZGEQRF( M, N, A, LDA, WORK( 1 ), WORK( MN+1 ), LWORK-MN,
     $                INFO )
*
*        workspace at least N, optimally N*NB
*
         IF( .NOT.TPSD ) THEN
*
*           Least-Squares Problem min || A * X - B ||
*
*           B(1:M,1:NRHS) := Q**H * B(1:M,1:NRHS)
*
            CALL ZUNMQR( 'Left', 'Conjugate transpose', M, NRHS, N, A,
     $                   LDA, WORK( 1 ), B, LDB, WORK( MN+1 ), LWORK-MN,
     $                   INFO )
*
*           workspace at least NRHS, optimally NRHS*NB
*
*           B(1:N,1:NRHS) := inv(R) * B(1:N,1:NRHS)
*
            CALL ZTRTRS( 'Upper', 'No transpose', 'Non-unit', N, NRHS,
     $                   A, LDA, B, LDB, INFO )
*
            IF( INFO.GT.0 ) THEN
               RETURN
            END IF
*
            SCLLEN = N
*
         ELSE
*
*           Overdetermined system of equations A**H * X = B
```

```
*
*          B(1:N,1:NRHS) := inv(R**H) * B(1:N,1:NRHS)
*
           CALL ZTRTRS( 'Upper', 'Conjugate transpose','Non-unit',
     $                  N, NRHS, A, LDA, B, LDB, INFO )
*
           IF( INFO.GT.0 ) THEN
              RETURN
           END IF
*
*          B(N+1:M,1:NRHS) = ZERO
*
           DO 20 J = 1, NRHS
              DO 10 I = N + 1, M
                 B( I, J ) = CZERO
   10         CONTINUE
   20      CONTINUE
*
*          B(1:M,1:NRHS) := Q(1:N,:) * B(1:N,1:NRHS)
*
           CALL ZUNMQR( 'Left', 'No transpose', M, NRHS, N, A, LDA,
     $                  WORK( 1 ), B, LDB, WORK( MN+1 ), LWORK-MN,
     $                  INFO )
*
*          workspace at least NRHS, optimally NRHS*NB
*
           SCLLEN = M
*
        END IF
*
      ELSE
*
*       Compute LQ factorization of A
*
        CALL ZGELQF( M, N, A, LDA, WORK( 1 ), WORK( MN+1 ), LWORK-MN,
     $               INFO )
*
*       workspace at least M, optimally M*NB.
*
        IF( .NOT.TPSD ) THEN
*
*          underdetermined system of equations A * X = B
*
           B(1:M,1:NRHS) := inv(L) * B(1:M,1:NRHS)
*
           CALL ZTRTRS( 'Lower', 'No transpose', 'Non-unit', M, NRHS,
     $                  A, LDA, B, LDB, INFO )
*
           IF( INFO.GT.0 ) THEN
              RETURN
```

```
            END IF
*
*           B(M+1:N,1:NRHS) = 0
*
            DO 40 J = 1, NRHS
               DO 30 I = M + 1, N
                  B( I, J ) = CZERO
   30          CONTINUE
   40       CONTINUE
*
*           B(1:N,1:NRHS) := Q(1:N,:)**H * B(1:M,1:NRHS)
*
            CALL ZUNMLQ( 'Left', 'Conjugate transpose', N, NRHS, M, A,
     $                   LDA, WORK( 1 ), B, LDB, WORK( MN+1 ), LWORK-MN,
     $                   INFO )
*
*           workspace at least NRHS, optimally NRHS*NB
*
            SCLLEN = N
*
         ELSE
*
*           overdetermined system min || A**H * X - B ||
*
*           B(1:N,1:NRHS) := Q * B(1:N,1:NRHS)
*
            CALL ZUNMLQ( 'Left', 'No transpose', N, NRHS, M, A, LDA,
     $                   WORK( 1 ), B, LDB, WORK( MN+1 ), LWORK-MN,
     $                   INFO )
*
*           workspace at least NRHS, optimally NRHS*NB
*
*           B(1:M,1:NRHS) := inv(L**H) * B(1:M,1:NRHS)
*
            CALL ZTRTRS( 'Lower', 'Conjugate transpose', 'Non-unit',
     $                   M, NRHS, A, LDA, B, LDB, INFO )
*
            IF( INFO.GT.0 ) THEN
               RETURN
            END IF
*
            SCLLEN = M
*
         END IF
*
      END IF
*
*     Undo scaling
*
      IF( IASCL.EQ.1 ) THEN
```

```
         CALL ZLASCL( 'G', 0, 0, ANRM, SMLNUM, SCLLEN, NRHS, B, LDB,
     $                INFO )
      ELSE IF( IASCL.EQ.2 ) THEN
         CALL ZLASCL( 'G', 0, 0, ANRM, BIGNUM, SCLLEN, NRHS, B, LDB,
     $                INFO )
      END IF
      IF( IBSCL.EQ.1 ) THEN
         CALL ZLASCL( 'G', 0, 0, SMLNUM, BNRM, SCLLEN, NRHS, B, LDB,
     $                INFO )
      ELSE IF( IBSCL.EQ.2 ) THEN
         CALL ZLASCL( 'G', 0, 0, BIGNUM, BNRM, SCLLEN, NRHS, B, LDB,
     $                INFO )
      END IF
*
   50 CONTINUE
      WORK( 1 ) = DBLE( WSIZE )
*
      RETURN
*
*     End of ZGELS
*
      END
```

———-

23.2.4 lisp code

— zgels.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0) (czero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (f2cl-lib:complex16) czero)
           (ignorable zero one czero))
  (defun zgels (trans m n nrhs a lda b ldb$ work lwork info)
    (declare (type (array f2cl-lib:complex16 (*)) work b a)
             (type (f2cl-lib:integer4) info lwork ldb$ lda nrhs n m)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%))
      (prog ((rwork (make-array 1 :element-type 'double-float)) (anrm 0.0d0)
             (bignum 0.0d0) (bnrm 0.0d0) (smlnum 0.0d0) (brow 0) (i 0)
             (iascl 0) (ibscl 0) (j 0) (mn 0) (nb 0) (scllen 0) (wsize 0)
             (lquery nil) (tpsd nil))
        (declare (type (array double-float (1)) rwork)
```

```
          (type (double-float) anrm bignum bnrm smlnum)
          (type (f2cl-lib:integer4) brow i iascl ibscl j mn nb scllen
           wsize)
          (type f2cl-lib:logical lquery tpsd))
(setf info 0)
(setf mn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond
 ((not
   (or
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame trans "N")
      (declare (ignore var-1))
      (when var-0 (setf trans var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame trans "C")
      (declare (ignore var-1))
      (when var-0 (setf trans var-0))
      ret-val)))
  (setf info -1))
 ((< m 0) (setf info -2)) ((< n 0) (setf info -3))
 ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -6))
 ((< ldb$
     (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)
          (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and
   (< lwork
      (max (the f2cl-lib:integer4 1)
           (the f2cl-lib:integer4
                (f2cl-lib:int-add mn
                                  (max (the f2cl-lib:integer4 mn)
                                       (the f2cl-lib:integer4
                                            nrhs))))))
   (not lquery))
  (setf info -10)))
(cond
 ((or (= info 0) (= info (f2cl-lib:int-sub 10)))
  (setf tpsd f2cl-lib:%true%)
  (if (multiple-value-bind (ret-val var-0 var-1)
          (lsame trans "N")
        (declare (ignore var-1))
        (when var-0 (setf trans var-0))
        ret-val)
      (setf tpsd f2cl-lib:%false%))
  (cond
   ((>= m n)
```

```
(setf nb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 1 "ZGEQRF" " " m n -1 -1)
          (declare (ignore var-0 var-1 var-2 var-5 var-6))
          (setf m var-3)
          (setf n var-4)
          ret-val))
(cond
 (tpsd
  (setf nb
          (max (the f2cl-lib:integer4 nb)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4
                         var-5 var-6)
                        (ilaenv 1 "ZUNMQR" "LN" m nrhs n -1)
                      (declare (ignore var-0 var-1 var-2 var-6))
                      (setf m var-3)
                      (setf nrhs var-4)
                      (setf n var-5)
                      ret-val)))))
 (t
  (setf nb
          (max (the f2cl-lib:integer4 nb)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4
                         var-5 var-6)
                        (ilaenv 1 "ZUNMQR" "LC" m nrhs n -1)
                      (declare (ignore var-0 var-1 var-2 var-6))
                      (setf m var-3)
                      (setf nrhs var-4)
                      (setf n var-5)
                      ret-val)))))))
(t
 (setf nb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 1 "ZGELQF" " " m n -1 -1)
          (declare (ignore var-0 var-1 var-2 var-5 var-6))
          (setf m var-3)
          (setf n var-4)
          ret-val))
(cond
 (tpsd
  (setf nb
          (max (the f2cl-lib:integer4 nb)
               (the f2cl-lib:integer4
                    (multiple-value-bind
```

```
                                (ret-val var-0 var-1 var-2 var-3 var-4
                                 var-5 var-6)
                                (ilaenv 1 "ZUNMLQ" "LC" n nrhs m -1)
                              (declare (ignore var-0 var-1 var-2 var-6))
                              (setf n var-3)
                              (setf nrhs var-4)
                              (setf m var-5)
                              ret-val)))))
        (t
         (setf nb
                 (max (the f2cl-lib:integer4 nb)
                      (the f2cl-lib:integer4
                           (multiple-value-bind
                               (ret-val var-0 var-1 var-2 var-3 var-4
                                var-5 var-6)
                               (ilaenv 1 "ZUNMLQ" "LN" n nrhs m -1)
                             (declare (ignore var-0 var-1 var-2 var-6))
                             (setf n var-3)
                             (setf nrhs var-4)
                             (setf m var-5)
                             ret-val))))))))
    (setf wsize
            (max (the f2cl-lib:integer4 1)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add mn
                                        (f2cl-lib:int-mul
                                         (max (the f2cl-lib:integer4 mn)
                                              (the f2cl-lib:integer4
                                                   nrhs))
                                         nb)))))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce (f2cl-lib:dble wsize) 'f2cl-lib:complex16))))
  (cond
   ((/= info 0) (xerbla "ZGELS " (f2cl-lib:int-sub info)) (go end_label))
   (lquery (go end_label)))
  (cond
   ((=
     (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)
          (the f2cl-lib:integer4 nrhs))
     0)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlaset "Full"
         (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
         czero czero b ldb$)
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf nrhs var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldb$ var-6)))
    (go end_label)))
```

```
(setf smlnum (/ (dlamch "S") (dlamch "P")))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "M" m n a lda rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iascl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 anrm smlnum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 1))
 ((> anrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 anrm bignum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 2))
 ((= anrm zero)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "F"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       czero czero b ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf czero var-3))
```

```
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldb$ var-6)))
  (go label50)))
(setf brow m)
(if tpsd
    (setf brow n))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "M" brow nrhs b ldb$ rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf brow var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ibscl 0)
(cond
 ((and (> bnrm zero) (< bnrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 bnrm smlnum brow nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf brow var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 1))
 ((> bnrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 bnrm bignum brow nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf brow var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 2)))
(cond
 ((>= m n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (1) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 ((+ mn 1))
                             ((1 *)) work-%offset%)
```

```
      (f2cl-lib:int-sub lwork mn) info)
    (declare (ignore var-2 var-4 var-5 var-6))
    (when var-0 (setf m var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf lda var-3))
    (when var-7 (setf info var-7)))
  (cond
   ((not tpsd)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (zunmqr "Left" "Conjugate transpose" m nrhs n a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (1)
                               ((1 *)) work-%offset%)
         b ldb$
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               ((+ mn 1)) ((1 *)) work-%offset%)
         (f2cl-lib:int-sub lwork mn) info)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf nrhs var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldb$ var-9))
      (when var-12 (setf info var-12)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (ztrtrs "Upper" "No transpose" "Non-unit" n nrhs a lda b ldb$
         info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7))
      (when var-3 (setf n var-3))
      (when var-4 (setf nrhs var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))
    (cond ((> info 0) (go end_label))) (setf scllen n))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (ztrtrs "Upper" "Conjugate transpose" "Non-unit" n nrhs a lda b
         ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7))
      (when var-3 (setf n var-3))
      (when var-4 (setf nrhs var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))
    (cond ((> info 0) (go end_label)))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
```

```
          (tagbody
            (f2cl-lib:fdo (i (f2cl-lib:int-add n 1) (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                     b-%offset%)
                      czero)
               label10))
           label20))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11 var-12)
            (zunmqr "Left" "No transpose" m nrhs n a lda
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (1)
                                   ((1 *)) work-%offset%)
             b ldb$
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                   ((+ mn 1)) ((1 *)) work-%offset%)
             (f2cl-lib:int-sub lwork mn) info)
          (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
          (when var-2 (setf m var-2))
          (when var-3 (setf nrhs var-3))
          (when var-4 (setf n var-4))
          (when var-6 (setf lda var-6))
          (when var-9 (setf ldb$ var-9))
          (when var-12 (setf info var-12)))
        (setf scllen m))))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (zgelqf m n a lda
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (1) ((1 *))
                                work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 ((+ mn 1))
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-sub lwork mn) info)
        (declare (ignore var-2 var-4 var-5 var-6))
        (setf m var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf info var-7))
     (cond
      ((not tpsd)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
           (ztrtrs "Lower" "No transpose" "Non-unit" m nrhs a lda b ldb$
            info)
         (declare (ignore var-0 var-1 var-2 var-5 var-7))
         (when var-3 (setf m var-3))
         (when var-4 (setf nrhs var-4))
```

```
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (cond ((> info 0) (go end_label)))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (f2cl-lib:fdo (i (f2cl-lib:int-add m 1) (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                czero)
        label30))
     label40))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (zunmlq "Left" "Conjugate transpose" n nrhs m a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (1)
                             ((1 *)) work-%offset%)
       b ldb$
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             ((+ mn 1)) ((1 *)) work-%offset%)
       (f2cl-lib:int-sub lwork mn) info)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf m var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldb$ var-9))
    (when var-12 (setf info var-12)))
  (setf scllen n))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (zunmlq "Left" "No transpose" n nrhs m a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (1)
                             ((1 *)) work-%offset%)
       b ldb$
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             ((+ mn 1)) ((1 *)) work-%offset%)
       (f2cl-lib:int-sub lwork mn) info)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf m var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldb$ var-9))
```

```
      (when var-12 (setf info var-12)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (ztrtrs "Lower" "Conjugate transpose" "Non-unit" m nrhs a lda b
         ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7))
      (when var-3 (setf m var-3))
      (when var-4 (setf nrhs var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))
    (cond ((> info 0) (go end_label))) (setf scllen m)))))
(cond
 ((= iascl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 anrm smlnum scllen nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf scllen var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9))))
 ((= iascl 2)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 anrm bignum scllen nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf scllen var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))))
(cond
 ((= ibscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 smlnum bnrm scllen nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf smlnum var-3))
    (when var-4 (setf bnrm var-4))
    (when var-5 (setf scllen var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9))))
 ((= ibscl 2)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
```

```
              (zlascl "G" 0 0 bignum bnrm scllen nrhs b ldb$ info)
            (declare (ignore var-0 var-1 var-2 var-7))
            (when var-3 (setf bignum var-3))
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf scllen var-5))
            (when var-6 (setf nrhs var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf info var-9)))))
       label50
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (f2cl-lib:dble wsize) 'f2cl-lib:complex16))
        (go end_label)
       end_label
        (return (values trans m n nrhs nil lda nil ldb$ nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgels fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$ nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::zgelqf
                                              fortran-to-lisp::xerbla
                                              fortran-to-lisp::ilaenv))))
```

———-

23.3 zgelsd

23.3.1 Axiom unit tests

— zgelsd.input —

```
)set break resume
)sys rm -f zgelsd.output
)spool zgelsd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

23.3.2 Axiom help page

— zgelsd.help —

```
======================================================================
zgelsd examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgelsd (M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK, WORK,
           LWORK, RWORK, IWORK, INFO)
            ZGELSD computes the minimum-norm solution to a linear least
           squares problem for GE matrices

Function/Subroutine Documentation
   subroutine zgelsd (integer M, integer N, integer NRHS, complex*16,
       dimension( lda, * ) A, integer LDA, complex*16, dimension( ldb, * ) B,
       integer LDB, double precision, dimension( * ) S, double precision
       RCOND, integer RANK, complex*16, dimension( * ) WORK, integer LWORK,
       double precision, dimension( * ) RWORK, integer, dimension( * ) IWORK,
       integer INFO)
        ZGELSD computes the minimum-norm solution to a linear least squares
       problem for GE matrices
```

```
Purpose:
   ZGELSD computes the minimum-norm solution to a real linear least
   squares problem:
          minimize 2-norm(| b - A*x |)
   using the singular value decomposition (SVD) of A. A is an M-by-N
   matrix which may be rank-deficient.

   Several right hand side vectors b and solution vectors x can be
   handled in a single call; they are stored as the columns of the
   M-by-NRHS right hand side matrix B and the N-by-NRHS solution matrix X.

   The problem is solved in three steps:
   (1) Reduce the coefficient matrix A to bidiagonal form with
       Householder tranformations, reducing the original problem
       into a "bidiagonal least squares problem" (BLS)
   (2) Solve the BLS using a divide and conquer approach.
   (3) Apply back all the Householder tranformations to solve
       the original least squares problem.

   The effective rank of A is determined by treating as zero those
   singular values which are less than RCOND times the largest singular
   value.

   The divide and conquer algorithm makes very mild assumptions about
   floating point arithmetic. It will work on machines with a guard
   digit in add/subtract, or on those binary machines without guard
   digits which subtract like the Cray X-MP, Cray Y-MP, Cray C-90, or
   Cray-2. It could conceivably fail on hexadecimal or decimal machines
   without guard digits, but we know of none.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A. M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A. N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X. NRHS >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, A has been destroyed.
```

```
LDA
  LDA is INTEGER
  The leading dimension of the array A. LDA >= max(1,M).

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  On entry, the M-by-NRHS right hand side matrix B.
  On exit, B is overwritten by the N-by-NRHS solution matrix X.
  If m >= n and RANK = n, the residual sum-of-squares for
  the solution in the i-th column is given by the sum of
  squares of the modulus of elements n+1:m in that column.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,M,N).

S
  S is DOUBLE PRECISION array, dimension (min(M,N))
  The singular values of A in decreasing order.
  The condition number of A in the 2-norm = S(1)/S(min(m,n)).

RCOND
  RCOND is DOUBLE PRECISION
  RCOND is used to determine the effective rank of A.
  Singular values S(i) <= RCOND*S(1) are treated as zero.
  If RCOND < 0, machine precision is used instead.

RANK
  RANK is INTEGER
  The effective rank of A, i.e., the number of singular values
  which are greater than RCOND*S(1).

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK must be at least 1.
  The exact minimum amount of workspace needed depends on M,
  N and NRHS. As long as LWORK is at least
        2*N + N*NRHS
  if M is greater than or equal to N or
        2*M + M*NRHS
  if M is less than N, the code will execute correctly.
  For good performance, LWORK should generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the array WORK and the
```

```
  minimum sizes of the arrays RWORK and IWORK, and returns
  these values as the first entries of the WORK, RWORK and
  IWORK arrays, and no error message related to LWORK is issued by XERBLA.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (MAX(1,LRWORK))
  LRWORK >=
    10*N + 2*N*SMLSIZ + 8*N*NLVL + 3*SMLSIZ*NRHS +
    MAX( (SMLSIZ+1)**2, N*(1+NRHS) + 2*NRHS )
  if M is greater than or equal to N or
    10*M + 2*M*SMLSIZ + 8*M*NLVL + 3*SMLSIZ*NRHS +
    MAX( (SMLSIZ+1)**2, N*(1+NRHS) + 2*NRHS )
  if M is less than N, the code will execute correctly.
  SMLSIZ is returned by ILAENV and is equal to the maximum
  size of the subproblems at the bottom of the computation
  tree (usually about 25), and
    NLVL = MAX( 0, INT( LOG_2( MIN( M,N )/(SMLSIZ+1) ) ) + 1 )
  On exit, if INFO = 0, RWORK(1) returns the minimum LRWORK.

IWORK
  IWORK is INTEGER array, dimension (MAX(1,LIWORK))
  LIWORK >= max(1, 3*MINMN*NLVL + 11*MINMN),
  where MINMN = MIN( M,N ).
  On exit, if INFO = 0, IWORK(1) returns the minimum LIWORK.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value.
  > 0:  the algorithm for computing the SVD failed to converge;
        if INFO = i, i off-diagonal elements of an intermediate
        bidiagonal form did not converge to zero.
```

———-

23.3.3 fortran code

— zgelsd.f —

```
*  =====================================================================
      SUBROUTINE ZGELSD( M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK,
     $                   WORK, LWORK, RWORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
```

```
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDB, LWORK, M, N, NRHS, RANK
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   RWORK( * ), S( * )
      COMPLEX*16         A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE, TWO
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0, TWO = 2.0D+0 )
      COMPLEX*16         CZERO
      PARAMETER          ( CZERO = ( 0.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            IASCL, IBSCL, IE, IL, ITAU, ITAUP, ITAUQ,
     $                   LDWORK, LIWORK, LRWORK, MAXMN, MAXWRK, MINMN,
     $                   MINWRK, MM, MNTHR, NLVL, NRWORK, NWORK, SMLSIZ
      DOUBLE PRECISION   ANRM, BIGNUM, BNRM, EPS, SFMIN, SMLNUM
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, DLASCL, DLASET, XERBLA, ZGEBRD, ZGELQF,
     $                   ZGEQRF, ZLACPY, ZLALSD, ZLASCL, ZLASET, ZUNMBR,
     $                   ZUNMLQ, ZUNMQR
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, ZLANGE
      EXTERNAL           ILAENV, DLAMCH, ZLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          INT, LOG, MAX, MIN, DBLE
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      MINMN = MIN( M, N )
      MAXMN = MAX( M, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
```

```
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, MAXMN ) ) THEN
         INFO = -7
      END IF
*
*     Compute workspace.
*     (Note: Comments in the code beginning "Workspace:" describe the
*     minimal amount of workspace needed at that point in the code,
*     as well as the preferred amount for good performance.
*     NB refers to the optimal block size for the immediately
*     following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = 1
         MAXWRK = 1
         LIWORK = 1
         LRWORK = 1
         IF( MINMN.GT.0 ) THEN
            SMLSIZ = ILAENV( 9, 'ZGELSD', ' ', 0, 0, 0, 0 )
            MNTHR = ILAENV( 6, 'ZGELSD', ' ', M, N, NRHS, -1 )
            NLVL = MAX( INT( LOG( DBLE( MINMN ) / DBLE( SMLSIZ + 1 ) ) /
     $                  LOG( TWO ) ) + 1, 0 )
            LIWORK = 3*MINMN*NLVL + 11*MINMN
            MM = M
            IF( M.GE.N .AND. M.GE.MNTHR ) THEN
*
*              Path 1a - overdetermined, with many more rows than
*                        columns.
*
               MM = N
               MAXWRK = MAX( MAXWRK, N*ILAENV( 1, 'ZGEQRF', ' ', M, N,
     $                       -1, -1 ) )
               MAXWRK = MAX( MAXWRK, NRHS*ILAENV( 1, 'ZUNMQR', 'LC', M,
     $                       NRHS, N, -1 ) )
            END IF
            IF( M.GE.N ) THEN
*
*              Path 1 - overdetermined or exactly determined.
*
               LRWORK = 10*N + 2*N*SMLSIZ + 8*N*NLVL + 3*SMLSIZ*NRHS +
     $                  MAX( (SMLSIZ+1)**2, N*(1+NRHS) + 2*NRHS )
               MAXWRK = MAX( MAXWRK, 2*N + ( MM + N )*ILAENV( 1,
     $                       'ZGEBRD', ' ', MM, N, -1, -1 ) )
               MAXWRK = MAX( MAXWRK, 2*N + NRHS*ILAENV( 1, 'ZUNMBR',
     $                       'QLC', MM, NRHS, N, -1 ) )
               MAXWRK = MAX( MAXWRK, 2*N + ( N - 1 )*ILAENV( 1,
```

```
     $                       'ZUNMBR', 'PLN', N, NRHS, N, -1 ) )
               MAXWRK = MAX( MAXWRK, 2*N + N*NRHS )
               MINWRK = MAX( 2*N + MM, 2*N + N*NRHS )
            END IF
            IF( N.GT.M ) THEN
               LRWORK = 10*M + 2*M*SMLSIZ + 8*M*NLVL + 3*SMLSIZ*NRHS +
     $                  MAX( (SMLSIZ+1)**2, N*(1+NRHS) + 2*NRHS )
               IF( N.GE.MNTHR ) THEN
*
*                 Path 2a - underdetermined, with many more columns
*                           than rows.
*
                  MAXWRK = M + M*ILAENV( 1, 'ZGELQF', ' ', M, N, -1,
     $                     -1 )
                  MAXWRK = MAX( MAXWRK, M*M + 4*M + 2*M*ILAENV( 1,
     $                          'ZGEBRD', ' ', M, M, -1, -1 ) )
                  MAXWRK = MAX( MAXWRK, M*M + 4*M + NRHS*ILAENV( 1,
     $                          'ZUNMBR', 'QLC', M, NRHS, M, -1 ) )
                  MAXWRK = MAX( MAXWRK, M*M + 4*M + ( M - 1 )*ILAENV( 1,
     $                          'ZUNMLQ', 'LC', N, NRHS, M, -1 ) )
                  IF( NRHS.GT.1 ) THEN
                     MAXWRK = MAX( MAXWRK, M*M + M + M*NRHS )
                  ELSE
                     MAXWRK = MAX( MAXWRK, M*M + 2*M )
                  END IF
                  MAXWRK = MAX( MAXWRK, M*M + 4*M + M*NRHS )
!     XXX: Ensure the Path 2a case below is triggered.  The workspace
!     calculation should use queries for all routines eventually.
                  MAXWRK = MAX( MAXWRK,
     $                 4*M+M*M+MAX( M, 2*M-4, NRHS, N-3*M ) )
               ELSE
*
*                 Path 2 - underdetermined.
*
                  MAXWRK = 2*M + ( N + M )*ILAENV( 1, 'ZGEBRD', ' ', M,
     $                     N, -1, -1 )
                  MAXWRK = MAX( MAXWRK, 2*M + NRHS*ILAENV( 1, 'ZUNMBR',
     $                          'QLC', M, NRHS, M, -1 ) )
                  MAXWRK = MAX( MAXWRK, 2*M + M*ILAENV( 1, 'ZUNMBR',
     $                          'PLN', N, NRHS, M, -1 ) )
                  MAXWRK = MAX( MAXWRK, 2*M + M*NRHS )
               END IF
               MINWRK = MAX( 2*M + N, 2*M + M*NRHS )
            END IF
         END IF
         MINWRK = MIN( MINWRK, MAXWRK )
         WORK( 1 ) = MAXWRK
         IWORK( 1 ) = LIWORK
         RWORK( 1 ) = LRWORK
*
```

```
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -12
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGELSD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         RANK = 0
         RETURN
      END IF
*
*     Get machine parameters.
*
      EPS = DLAMCH( 'P' )
      SFMIN = DLAMCH( 'S' )
      SMLNUM = SFMIN / EPS
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
*
*     Scale A if max entry outside range [SMLNUM,BIGNUM].
*
      ANRM = ZLANGE( 'M', M, N, A, LDA, RWORK )
      IASCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL ZLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
         IASCL = 1
      ELSE IF( ANRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM.
*
         CALL ZLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
         IASCL = 2
      ELSE IF( ANRM.EQ.ZERO ) THEN
*
*        Matrix all zero. Return zero solution.
*
         CALL ZLASET( 'F', MAX( M, N ), NRHS, CZERO, CZERO, B, LDB )
         CALL DLASET( 'F', MINMN, 1, ZERO, ZERO, S, 1 )
         RANK = 0
```

```
         GO TO 10
      END IF
*
*     Scale B if max entry outside range [SMLNUM,BIGNUM].
*
      BNRM = ZLANGE( 'M', M, NRHS, B, LDB, RWORK )
      IBSCL = 0
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM.
*
         CALL ZLASCL( 'G', 0, 0, BNRM, SMLNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 1
      ELSE IF( BNRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM.
*
         CALL ZLASCL( 'G', 0, 0, BNRM, BIGNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 2
      END IF
*
*     If M < N make sure B(M+1:N,:) = 0
*
      IF( M.LT.N )
     $   CALL ZLASET( 'F', N-M, NRHS, CZERO, CZERO, B( M+1, 1 ), LDB )
*
*     Overdetermined case.
*
      IF( M.GE.N ) THEN
*
*        Path 1 - overdetermined or exactly determined.
*
         MM = M
         IF( M.GE.MNTHR ) THEN
*
*           Path 1a - overdetermined, with many more rows than columns
*
            MM = N
            ITAU = 1
            NWORK = ITAU + N
*
*           Compute A=Q*R.
*           (RWorkspace: need N)
*           (CWorkspace: need N, prefer N*NB)
*
            CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                   LWORK-NWORK+1, INFO )
*
*           Multiply B by transpose(Q).
*           (RWorkspace: need N)
```

```
*          (CWorkspace: need NRHS, prefer NRHS*NB)
*
           CALL ZUNMQR( 'L', 'C', M, NRHS, N, A, LDA, WORK( ITAU ), B,
     $                  LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
*          Zero out below R.
*
           IF( N.GT.1 ) THEN
              CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO, A( 2, 1 ),
     $                     LDA )
           END IF
        END IF
*
        ITAUQ = 1
        ITAUP = ITAUQ + N
        NWORK = ITAUP + N
        IE = 1
        NRWORK = IE + N
*
*       Bidiagonalize R in A.
*       (RWorkspace: need N)
*       (CWorkspace: need 2*N+MM, prefer 2*N+(MM+N)*NB)
*
        CALL ZGEBRD( MM, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $               WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $               INFO )
*
*       Multiply B by transpose of left bidiagonalizing vectors of R.
*       (CWorkspace: need 2*N+NRHS, prefer 2*N+NRHS*NB)
*
        CALL ZUNMBR( 'Q', 'L', 'C', MM, NRHS, N, A, LDA, WORK( ITAUQ ),
     $               B, LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
*       Solve the bidiagonal least squares problem.
*
        CALL ZLALSD( 'U', SMLSIZ, N, NRHS, S, RWORK( IE ), B, LDB,
     $               RCOND, RANK, WORK( NWORK ), RWORK( NRWORK ),
     $               IWORK, INFO )
        IF( INFO.NE.0 ) THEN
           GO TO 10
        END IF
*
*       Multiply B by right bidiagonalizing vectors of R.
*
        CALL ZUNMBR( 'P', 'L', 'N', N, NRHS, N, A, LDA, WORK( ITAUP ),
     $               B, LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
     ELSE IF( N.GE.MNTHR .AND. LWORK.GE.4*M+M*M+
     $         MAX( M, 2*M-4, NRHS, N-3*M ) ) THEN
*
```

```
*        Path 2a - underdetermined, with many more columns than rows
*        and sufficient workspace for an efficient algorithm.
*
         LDWORK = M
         IF( LWORK.GE.MAX( 4*M+M*LDA+MAX( M, 2*M-4, NRHS, N-3*M ),
     $       M*LDA+M+M*NRHS ) )LDWORK = LDA
         ITAU = 1
         NWORK = M + 1
*
*        Compute A=L*Q.
*        (CWorkspace: need 2*M, prefer M+M*NB)
*
         CALL ZGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                LWORK-NWORK+1, INFO )
         IL = NWORK
*
*        Copy L to WORK(IL), zeroing out above its diagonal.
*
         CALL ZLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWORK )
         CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO, WORK( IL+LDWORK ),
     $                LDWORK )
         ITAUQ = IL + LDWORK*M
         ITAUP = ITAUQ + M
         NWORK = ITAUP + M
         IE = 1
         NRWORK = IE + M
*
*        Bidiagonalize L in WORK(IL).
*        (RWorkspace: need M)
*        (CWorkspace: need M*M+4*M, prefer M*M+4*M+2*M*NB)
*
         CALL ZGEBRD( M, M, WORK( IL ), LDWORK, S, RWORK( IE ),
     $                WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                LWORK-NWORK+1, INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors of L.
*        (CWorkspace: need M*M+4*M+NRHS, prefer M*M+4*M+NRHS*NB)
*
         CALL ZUNMBR( 'Q', 'L', 'C', M, NRHS, M, WORK( IL ), LDWORK,
     $                WORK( ITAUQ ), B, LDB, WORK( NWORK ),
     $                LWORK-NWORK+1, INFO )
*
*        Solve the bidiagonal least squares problem.
*
         CALL ZLALSD( 'U', SMLSIZ, M, NRHS, S, RWORK( IE ), B, LDB,
     $                RCOND, RANK, WORK( NWORK ), RWORK( NRWORK ),
     $                IWORK, INFO )
         IF( INFO.NE.0 ) THEN
            GO TO 10
         END IF
```

```
*
*        Multiply B by right bidiagonalizing vectors of L.
*
         CALL ZUNMBR( 'P', 'L', 'N', M, NRHS, M, WORK( IL ), LDWORK,
     $                WORK( ITAUP ), B, LDB, WORK( NWORK ),
     $                LWORK-NWORK+1, INFO )
*
*        Zero out below first M rows of B.
*
         CALL ZLASET( 'F', N-M, NRHS, CZERO, CZERO, B( M+1, 1 ), LDB )
         NWORK = ITAU + M
*
*        Multiply transpose(Q) by B.
*        (CWorkspace: need NRHS, prefer NRHS*NB)
*
         CALL ZUNMLQ( 'L', 'C', N, NRHS, M, A, LDA, WORK( ITAU ), B,
     $                LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
      ELSE
*
*        Path 2 - remaining underdetermined cases.
*
         ITAUQ = 1
         ITAUP = ITAUQ + M
         NWORK = ITAUP + M
         IE = 1
         NRWORK = IE + M
*
*        Bidiagonalize A.
*        (RWorkspace: need M)
*        (CWorkspace: need 2*M+N, prefer 2*M+(M+N)*NB)
*
         CALL ZGEBRD( M, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors.
*        (CWorkspace: need 2*M+NRHS, prefer 2*M+NRHS*NB)
*
         CALL ZUNMBR( 'Q', 'L', 'C', M, NRHS, N, A, LDA, WORK( ITAUQ ),
     $                B, LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
*        Solve the bidiagonal least squares problem.
*
         CALL ZLALSD( 'L', SMLSIZ, M, NRHS, S, RWORK( IE ), B, LDB,
     $                RCOND, RANK, WORK( NWORK ), RWORK( NRWORK ),
     $                IWORK, INFO )
         IF( INFO.NE.0 ) THEN
            GO TO 10
         END IF
```

```
*
*        Multiply B by right bidiagonalizing vectors of A.
*
         CALL ZUNMBR( 'P', 'L', 'N', N, NRHS, M, A, LDA, WORK( ITAUP ),
     $                B, LDB, WORK( NWORK ), LWORK-NWORK+1, INFO )
*
      END IF
*
*     Undo scaling.
*
      IF( IASCL.EQ.1 ) THEN
         CALL ZLASCL( 'G', 0, 0, ANRM, SMLNUM, N, NRHS, B, LDB, INFO )
         CALL DLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
     $                INFO )
      ELSE IF( IASCL.EQ.2 ) THEN
         CALL ZLASCL( 'G', 0, 0, ANRM, BIGNUM, N, NRHS, B, LDB, INFO )
         CALL DLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                INFO )
      END IF
      IF( IBSCL.EQ.1 ) THEN
         CALL ZLASCL( 'G', 0, 0, SMLNUM, BNRM, N, NRHS, B, LDB, INFO )
      ELSE IF( IBSCL.EQ.2 ) THEN
         CALL ZLASCL( 'G', 0, 0, BIGNUM, BNRM, N, NRHS, B, LDB, INFO )
      END IF
*
   10 CONTINUE
      WORK( 1 ) = MAXWRK
      IWORK( 1 ) = LIWORK
      RWORK( 1 ) = LRWORK
      RETURN
*
*     End of ZGELSD
*
      END
```

———-

23.3.4 lisp code

— zgelsd.lisp —

```
(let* ((zero 0.0d0)
       (one 1.0d0)
       (two 2.0d0)
       (czero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
```

```
          (type (f2cl-lib:complex16) czero)
          (ignorable zero one two czero))
(defun zgelsd
       (m n nrhs a lda b ldb$ s rcond rank work lwork rwork iwork info)
  (declare (type (array f2cl-lib:integer4 (*)) iwork)
           (type (double-float) rcond)
           (type (array double-float (*)) rwork s)
           (type (array f2cl-lib:complex16 (*)) work b a)
           (type (f2cl-lib:integer4) info lwork rank ldb$ lda nrhs n m))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex16 a-%data% a-%offset%)
       (b f2cl-lib:complex16 b-%data% b-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%)
       (s double-float s-%data% s-%offset%)
       (rwork double-float rwork-%data% rwork-%offset%)
       (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
    (prog ((anrm 0.0d0) (bignum 0.0d0) (bnrm 0.0d0) (eps 0.0d0) (sfmin 0.0d0)
           (smlnum 0.0d0) (iascl 0) (ibscl 0) (ie 0) (il 0) (itau 0)
           (itaup 0) (itauq 0) (ldwork 0) (liwork 0) (lrwork 0) (maxmn 0)
           (maxwrk 0) (minmn 0) (minwrk 0) (mm 0) (mnthr 0) (nlvl 0)
           (nrwork 0) (nwork 0) (smlsiz 0) (lquery nil))
      (declare (type (double-float) anrm bignum bnrm eps sfmin smlnum)
               (type (f2cl-lib:integer4) iascl ibscl ie il itau itaup itauq
                ldwork liwork lrwork maxmn maxwrk minmn minwrk mm mnthr nlvl
                nrwork nwork smlsiz)
               (type f2cl-lib:logical lquery))
      (setf info 0)
      (setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
      (setf maxmn (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< nrhs 0) (setf info -3))
            ((< lda
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -5))
            ((< ldb$
                (max (the f2cl-lib:integer4 1)
                     (the f2cl-lib:integer4 maxmn)))
             (setf info -7)))
      (cond
       ((= info 0) (setf minwrk 1) (setf maxwrk 1) (setf liwork 1)
        (setf lrwork 1)
        (cond
         ((> minmn 0) (setf smlsiz (ilaenv 9 "ZGELSD" " " 0 0 0 0))
          (setf mnthr
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                      (ilaenv 6 "ZGELSD" " " m n nrhs -1)
                    (declare (ignore var-0 var-1 var-2 var-6))
                    (setf m var-3)
```

```
          (setf n var-4)
          (setf nrhs var-5)
          ret-val))
(setf nlvl
        (max
         (the f2cl-lib:integer4
              (f2cl-lib:int-add
               (f2cl-lib:int
                (/
                 (f2cl-lib:flog
                  (/ (f2cl-lib:dble minmn)
                     (f2cl-lib:dble (f2cl-lib:int-add smlsiz 1))))
                 (f2cl-lib:flog two)))
               1))
         (the f2cl-lib:integer4 0)))
(setf liwork
        (f2cl-lib:int-add (f2cl-lib:int-mul 3 minmn nlvl)
                          (f2cl-lib:int-mul 11 minmn)))
(setf mm m)
(cond
 ((and (>= m n) (>= m mnthr)) (setf mm n)
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-mul n
                                      (multiple-value-bind
                                          (ret-val var-0 var-1
                                           var-2 var-3 var-4 var-5
                                           var-6)
                                          (ilaenv 1 "ZGEQRF" " " m
                                           n -1 -1)
                                        (declare
                                         (ignore var-0 var-1 var-2
                                          var-5 var-6))
                                        (setf m var-3)
                                        (setf n var-4)
                                        ret-val)))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-mul nrhs
                                      (multiple-value-bind
                                          (ret-val var-0 var-1
                                           var-2 var-3 var-4 var-5
                                           var-6)
                                          (ilaenv 1 "ZUNMQR" "LC" m
                                           nrhs n -1)
                                        (declare
                                         (ignore var-0 var-1 var-2
                                          var-6))
```

```
                                        (setf m var-3)
                                        (setf nrhs var-4)
                                        (setf n var-5)
                                        ret-val)))))))
(cond
 ((>= m n)
  (setf lrwork
          (f2cl-lib:int-add (f2cl-lib:int-mul 10 n)
                            (f2cl-lib:int-mul 2 n smlsiz)
                            (f2cl-lib:int-mul 8 n nlvl)
                            (f2cl-lib:int-mul 3 smlsiz nrhs)
                            (max
                             (the f2cl-lib:integer4
                                  (expt (f2cl-lib:int-add smlsiz 1)
                                        2))
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-mul n
                                                     (f2cl-lib:int-add
                                                      1 nrhs))
                                   (f2cl-lib:int-mul 2 nrhs))))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                      (f2cl-lib:int-mul
                                       (f2cl-lib:int-add mm n)
                                       (multiple-value-bind
                                           (ret-val var-0 var-1
                                            var-2 var-3 var-4 var-5
                                            var-6)
                                           (ilaenv 1 "ZGEBRD" " "
                                            mm n -1 -1)
                                         (declare
                                          (ignore var-0 var-1 var-2
                                           var-5 var-6))
                                         (setf mm var-3)
                                         (setf n var-4)
                                         ret-val))))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                      (f2cl-lib:int-mul nrhs
                                                        (multiple-value-bind
                                                            (ret-val
                                                             var-0
                                                             var-1
                                                             var-2
                                                             var-3
```

```
                                                                var-4
                                                                var-5
                                                                var-6)
                                                               (ilaenv
                                                                1
                                                                "ZUNMBR"
                                                                "QLC"
                                                                mm
                                                                nrhs n
                                                                -1)
                                                             (declare
                                                              (ignore
                                                               var-0
                                                               var-1
                                                               var-2
                                                               var-6))
                                                             (setf mm
                                                                     var-3)
                                                             (setf nrhs
                                                                     var-4)
                                                             (setf n
                                                                     var-5)
                                                             ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul
                                     (f2cl-lib:int-sub n 1)
                                     (multiple-value-bind
                                         (ret-val var-0 var-1
                                          var-2 var-3 var-4 var-5
                                          var-6)
                                         (ilaenv 1 "ZUNMBR" "PLN"
                                          n nrhs n -1)
                                       (declare
                                        (ignore var-0 var-1 var-2
                                         var-6))
                                       (setf n var-3)
                                       (setf nrhs var-4)
                                       (setf n var-5)
                                       ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul n nrhs)))))
(setf minwrk
        (max
         (the f2cl-lib:integer4
```

```
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) mm))
                (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                      (f2cl-lib:int-mul n nrhs)))))))
(cond
 ((> n m)
  (setf lrwork
          (f2cl-lib:int-add (f2cl-lib:int-mul 10 m)
                            (f2cl-lib:int-mul 2 m smlsiz)
                            (f2cl-lib:int-mul 8 m nlvl)
                            (f2cl-lib:int-mul 3 smlsiz nrhs)
                            (max
                             (the f2cl-lib:integer4
                                  (expt (f2cl-lib:int-add smlsiz 1)
                                        2))
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-mul n
                                                     (f2cl-lib:int-add
                                                      1 nrhs))
                                   (f2cl-lib:int-mul 2 nrhs))))))
  (cond
   ((>= n mnthr)
    (setf maxwrk
            (f2cl-lib:int-add m
                              (f2cl-lib:int-mul m
                                                (multiple-value-bind
                                                    (ret-val var-0
                                                     var-1 var-2
                                                     var-3 var-4
                                                     var-5 var-6)
                                                    (ilaenv 1
                                                     "ZGELQF" " " m
                                                     n -1 -1)
                                                  (declare
                                                   (ignore var-0
                                                    var-1 var-2
                                                    var-5 var-6))
                                                  (setf m var-3)
                                                  (setf n var-4)
                                                  ret-val))))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                        (f2cl-lib:int-mul 4 m)
                                        (f2cl-lib:int-mul 2 m
                                                          (multiple-value-bind
                                                              (ret-val
                                                               var-0
```

```
                                                                    var-1
                                                                    var-2
                                                                    var-3
                                                                    var-4
                                                                    var-5
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "ZGEBRD"
                                                                    " "
                                                                    m m
                                                                    -1
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-5
                                                                    var-6))
                                                                  (setf m
                                                                          var-3)
                                                                  (setf m
                                                                          var-4)
                                                                  ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                    (f2cl-lib:int-mul 4 m)
                                    (f2cl-lib:int-mul nrhs
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "ZUNMBR"
                                                           "QLC"
                                                           m
                                                           nrhs
                                                           m
                                                           -1)
                                                          (declare
                                                           (ignore
```

```
                                                                var-0
                                                                var-1
                                                                var-2
                                                                var-6))
                                                              (setf m
                                                                      var-3)
                                                              (setf nrhs
                                                                      var-4)
                                                              (setf m
                                                                      var-5)
                                                              ret-val))))))
                (setf maxwrk
                        (max (the f2cl-lib:integer4 maxwrk)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                                    (f2cl-lib:int-mul 4 m)
                                                    (f2cl-lib:int-mul
                                                     (f2cl-lib:int-sub m 1)
                                                     (multiple-value-bind
                                                         (ret-val var-0 var-1
                                                          var-2 var-3 var-4
                                                          var-5 var-6)
                                                         (ilaenv 1 "ZUNMLQ"
                                                          "LC" n nrhs m -1)
                                                       (declare
                                                        (ignore var-0 var-1
                                                         var-2 var-6))
                                                       (setf n var-3)
                                                       (setf nrhs var-4)
                                                       (setf m var-5)
                                                       ret-val))))))
                (cond
                 ((> nrhs 1)
                  (setf maxwrk
                          (max (the f2cl-lib:integer4 maxwrk)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add (f2cl-lib:int-mul m m) m
                                                      (f2cl-lib:int-mul m
                                                                        nrhs))))))
                 (t
                  (setf maxwrk
                          (max (the f2cl-lib:integer4 maxwrk)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                                      (f2cl-lib:int-mul 2
                                                                        m)))))))
                (setf maxwrk
                        (max (the f2cl-lib:integer4 maxwrk)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul m m)
```

```
                                        (f2cl-lib:int-mul 4 m)
                                        (f2cl-lib:int-mul m
                                                          nrhs)))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 4 m)
                                     (f2cl-lib:int-mul m m)
                                     (max
                                      (the f2cl-lib:integer4 m)
                                      (the f2cl-lib:integer4
                                           (f2cl-lib:int-sub
                                            (f2cl-lib:int-mul 2
                                                              m)
                                            4))
                                      (the f2cl-lib:integer4
                                           nrhs)
                                      (the f2cl-lib:integer4
                                           (f2cl-lib:int-sub n
                                                             (f2cl-lib:int-mul
                                                              3
                                                              m)))))))))
(t
 (setf maxwrk
         (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-add n m)
                            (multiple-value-bind
                                (ret-val var-0 var-1 var-2 var-3
                                 var-4 var-5 var-6)
                                (ilaenv 1 "ZGEBRD" " " m n -1
                                 -1)
                              (declare
                               (ignore var-0 var-1 var-2 var-5
                               var-6))
                              (setf m var-3)
                              (setf n var-4)
                              ret-val))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                     (f2cl-lib:int-mul nrhs
                                                       (multiple-value-bind
                                                           (ret-val
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-3
                                                            var-4
```

```
                                                                    var-5
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "ZUNMBR"
                                                                    "QLC"
                                                                    m
                                                                    nrhs
                                                                    m
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (setf m
                                                                          var-3)
                                                                  (setf nrhs
                                                                          var-4)
                                                                  (setf m
                                                                          var-5)
                                                                  ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "ZUNMBR"
                                                           "PLN"
                                                           n
                                                           nrhs
                                                           m
                                                           -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
```

```
                                                                    var-6))
                                                                  (setf n
                                                                          var-3)
                                                                  (setf nrhs
                                                                          var-4)
                                                                  (setf m
                                                                          var-5)
                                                                  ret-val))))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                            (f2cl-lib:int-mul m
                                                              nrhs)))))))
      (setf minwrk
              (max
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n))
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                      (f2cl-lib:int-mul m nrhs)))))))))
  (setf minwrk
          (min (the f2cl-lib:integer4 minwrk)
               (the f2cl-lib:integer4 maxwrk)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce maxwrk 'f2cl-lib:complex16))
  (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) liwork)
  (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
          (coerce (the f2cl-lib:integer4 lrwork) 'double-float))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -12)))))
(cond
 ((/= info 0) (xerbla "ZGELSD" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond ((or (= m 0) (= n 0)) (setf rank 0) (go end_label)))
(setf eps (dlamch "P"))
(setf sfmin (dlamch "S"))
(setf smlnum (/ sfmin eps))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "M" m n a lda rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
```

```
          (when var-4 (setf lda var-4))
          ret-val))
(setf iascl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 anrm smlnum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 1))
 ((> anrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 anrm bignum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 2))
 ((= anrm zero)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "F"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       czero czero b ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldb$ var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "F" minmn 1 zero zero s 1)
    (declare (ignore var-0 var-2 var-5 var-6))
    (when var-1 (setf minmn var-1))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4)))
  (setf rank 0) (go label10)))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "M" m nrhs b ldb$ rwork)
          (declare (ignore var-0 var-3 var-5))
```

```
          (when var-1 (setf m var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ibscl 0)
(cond
 ((and (> bnrm zero) (< bnrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 bnrm smlnum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 1))
 ((> bnrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 bnrm bignum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 2)))
(if (< m n)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlaset "F" (f2cl-lib:int-sub n m) nrhs czero czero
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 ((+ m 1) 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf nrhs var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldb$ var-6))))
(cond
 ((>= m n) (setf mm m)
  (cond
   ((>= m mnthr) (setf mm n) (setf itau 1)
    (setf nwork (f2cl-lib:int-add itau n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (zgeqrf m n a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
```

```
                              ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                              ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
      (declare (ignore var-2 var-4 var-5 var-6))
      (when var-0 (setf m var-0))
      (when var-1 (setf n var-1))
      (when var-3 (setf lda var-3))
      (when var-7 (setf info var-7)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (zunmqr "L" "C" m nrhs n a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                              ((1 *)) work-%offset%)
         b ldb$
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                              ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf nrhs var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldb$ var-9))
      (when var-12 (setf info var-12)))
    (cond
     ((> n 1)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
           czero czero
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (2 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-5))
        (when var-3 (setf czero var-3))
        (when var-4 (setf czero var-4))
        (when var-6 (setf lda var-6)))))))
  (setf itauq 1) (setf itaup (f2cl-lib:int-add itauq n))
  (setf nwork (f2cl-lib:int-add itaup n)) (setf ie 1)
  (setf nrwork (f2cl-lib:int-add ie n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (zgebrd mm n a lda s
       (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                             rwork-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
```

```
                              ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                              ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf mm var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "Q" "L" "C" mm nrhs n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                              ((1 *)) work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                              ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf mm var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zlalsd "U" smlsiz n nrhs s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                              rwork-%offset%)
     b ldb$ rcond rank
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                              ((1 *)) work-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork) ((1 *))
                              rwork-%offset%)
     iwork info)
  (declare (ignore var-0 var-4 var-5 var-6 var-10 var-11 var-12))
  (when var-1 (setf smlsiz var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf nrhs var-3))
  (when var-7 (setf ldb$ var-7))
  (when var-8 (setf rcond var-8))
  (when var-9 (setf rank var-9))
  (when var-13 (setf info var-13)))
(cond ((/= info 0) (go label10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
        var-10 var-11 var-12 var-13)
       (zunmbr "P" "L" "N" n nrhs n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                              ((1 *)) work-%offset%)
        b ldb$
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
      (when var-3 (setf n var-3))
      (when var-4 (setf nrhs var-4))
      (when var-5 (setf n var-5))
      (when var-7 (setf lda var-7))
      (when var-10 (setf ldb$ var-10))
      (when var-13 (setf info var-13))))
    ((and (>= n mnthr)
          (>= lwork
              (f2cl-lib:int-add (f2cl-lib:int-mul 4 m)
                                (f2cl-lib:int-mul m m)
                                (max (the f2cl-lib:integer4 m)
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           (f2cl-lib:int-mul 2 m)
                                           (f2cl-lib:int-sub 4)))
                                     (the f2cl-lib:integer4 nrhs)
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add n
                                                            (f2cl-lib:int-mul
                                                             -1 3 m)))))))
     (setf ldwork m)
     (if (>= lwork
             (max
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 4 m)
                                     (f2cl-lib:int-mul m lda)
                                     (max (the f2cl-lib:integer4 m)
                                          (the f2cl-lib:integer4
                                               (f2cl-lib:int-sub
                                                (f2cl-lib:int-mul 2 m) 4))
                                          (the f2cl-lib:integer4 nrhs)
                                          (the f2cl-lib:integer4
                                               (f2cl-lib:int-sub n
                                                                 (f2cl-lib:int-mul
                                                                  3
                                                                  m))))))
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul m lda) m
                                     (f2cl-lib:int-mul m nrhs)))))
         (setf ldwork lda))
```

```
(setf itau 1) (setf nwork (f2cl-lib:int-add m 1))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgelqf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-7))
(setf il nwork)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                           ((1 *)) work-%offset%)
     ldwork)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwork var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1) czero
     czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ il ldwork)) ((1 *)) work-%offset%)
     ldwork)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwork var-6)))
(setf itauq (f2cl-lib:int-add il (f2cl-lib:int-mul ldwork m)))
(setf itaup (f2cl-lib:int-add itauq m))
(setf nwork (f2cl-lib:int-add itaup m)) (setf ie 1)
(setf nrwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                           ((1 *)) work-%offset%)
     ldwork s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
```

```
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwork var-3)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "Q" "L" "C" m nrhs m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                           ((1 *)) work-%offset%)
     ldwork
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwork var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zlalsd "U" smlsiz m nrhs s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     b ldb$ rcond rank
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork) ((1 *))
                           rwork-%offset%)
     iwork info)
  (declare (ignore var-0 var-4 var-5 var-6 var-10 var-11 var-12))
  (when var-1 (setf smlsiz var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf nrhs var-3))
  (when var-7 (setf ldb$ var-7))
  (when var-8 (setf rcond var-8))
  (when var-9 (setf rank var-9))
```

```
  (when var-13 (setf info var-13)))
(cond ((/= info 0) (go label10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "P" "L" "N" m nrhs m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                           ((1 *)) work-%offset%)
     ldwork
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwork var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "F" (f2cl-lib:int-sub n m) nrhs czero czero
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 ((+ m 1) 1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$)
  (declare (ignore var-0 var-1 var-5))
  (when var-2 (setf nrhs var-2))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldb$ var-6)))
(setf nwork (f2cl-lib:int-add itau m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (zunmlq "L" "C" n nrhs m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                           ((1 *)) work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf n var-2))
  (when var-3 (setf nrhs var-3))
  (when var-4 (setf m var-4))
  (when var-6 (setf lda var-6))
  (when var-9 (setf ldb$ var-9))
```

```
    (when var-12 (setf info var-12))))
  (t (setf itauq 1) (setf itaup (f2cl-lib:int-add itauq m))
   (setf nwork (f2cl-lib:int-add itaup m)) (setf ie 1)
   (setf nrwork (f2cl-lib:int-add ie m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (zgebrd m n a lda s
        (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                              rwork-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
    (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf info var-10))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13)
       (zunmbr "Q" "L" "C" m nrhs n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                              ((1 *)) work-%offset%)
        b ldb$
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf nrhs var-4))
    (when var-5 (setf n var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldb$ var-10))
    (when var-13 (setf info var-13)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13)
       (zlalsd "L" smlsiz m nrhs s
        (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                              rwork-%offset%)
        b ldb$ rcond rank
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice rwork-%data% double-float (nrwork) ((1 *))
```

```
                                rwork-%offset%)
          iwork info)
      (declare (ignore var-0 var-4 var-5 var-6 var-10 var-11 var-12))
      (when var-1 (setf smlsiz var-1))
      (when var-2 (setf m var-2))
      (when var-3 (setf nrhs var-3))
      (when var-7 (setf ldb$ var-7))
      (when var-8 (setf rcond var-8))
      (when var-9 (setf rank var-9))
      (when var-13 (setf info var-13)))
    (cond ((/= info 0) (go label10)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (zunmbr "P" "L" "N" n nrhs m a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                                ((1 *)) work-%offset%)
         b ldb$
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                                ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) info)
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
      (when var-3 (setf n var-3))
      (when var-4 (setf nrhs var-4))
      (when var-5 (setf m var-5))
      (when var-7 (setf lda var-7))
      (when var-10 (setf ldb$ var-10))
      (when var-13 (setf info var-13)))))
  (cond
   ((= iascl 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (zlascl "G" 0 0 anrm smlnum n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf smlnum var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 smlnum anrm minmn 1 s minmn info)
      (declare (ignore var-0 var-1 var-2 var-6 var-7))
      (when var-3 (setf smlnum var-3))
      (when var-4 (setf anrm var-4))
      (when var-5 (setf minmn var-5))
      (when var-8 (setf minmn var-8))
      (when var-9 (setf info var-9))))
```

```
    ((= iascl 2)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (zlascl "G" 0 0 anrm bignum n nrhs b ldb$ info)
       (declare (ignore var-0 var-1 var-2 var-7))
       (when var-3 (setf anrm var-3))
       (when var-4 (setf bignum var-4))
       (when var-5 (setf n var-5))
       (when var-6 (setf nrhs var-6))
       (when var-8 (setf ldb$ var-8))
       (when var-9 (setf info var-9)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (dlascl "G" 0 0 bignum anrm minmn 1 s minmn info)
       (declare (ignore var-0 var-1 var-2 var-6 var-7))
       (when var-3 (setf bignum var-3))
       (when var-4 (setf anrm var-4))
       (when var-5 (setf minmn var-5))
       (when var-8 (setf minmn var-8))
       (when var-9 (setf info var-9)))))
  (cond
   ((= ibscl 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (zlascl "G" 0 0 smlnum bnrm n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf smlnum var-3))
      (when var-4 (setf bnrm var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9))))
   ((= ibscl 2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (zlascl "G" 0 0 bignum bnrm n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bignum var-3))
      (when var-4 (setf bnrm var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))))
 label10
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce maxwrk 'f2cl-lib:complex16))
  (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) liwork)
  (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
          (coerce (the f2cl-lib:integer4 lrwork) 'double-float))
  (go end_label)
```

```
      end_label
       (return
        (values m n nrhs nil lda nil ldb$ nil rcond rank nil nil nil nil
                info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgelsd
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (double-float)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (array fortran-to-lisp::integer4
                                              (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(fortran-to-lisp::m
                                             fortran-to-lisp::n
                                             fortran-to-lisp::nrhs nil
                                             fortran-to-lisp::lda nil
                                             fortran-to-lisp::ldb$ nil
                                             fortran-to-lisp::rcond
                                             fortran-to-lisp::rank nil nil nil
                                             nil fortran-to-lisp::info)
                                           :calls
                                           '(fortran-to-lisp::zgelqf
                                             fortran-to-lisp::zgebrd
                                             fortran-to-lisp::xerbla
                                             fortran-to-lisp::ilaenv))))
```

23.4 zgelss

23.4.1 Axiom unit tests

— **zgelss.input** —

```
)set break resume
)sys rm -f zgelss.output
)spool zgelss.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

23.4.2 Axiom help page

— **zgelss.help** —

```
======================================================================
zgelss examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgelss (M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK, WORK,
           LWORK, RWORK, INFO)
            ZGELSS solves overdetermined or underdetermined systems for GE
           matrices

Function/Subroutine Documentation
   subroutine zgelss (integer M, integer N, integer NRHS, complex*16,
       dimension( lda, * ) A, integer LDA, complex*16, dimension( ldb, * ) B,
       integer LDB, double precision, dimension( * ) S, double precision
       RCOND, integer RANK, complex*16, dimension( * ) WORK, integer LWORK,
       double precision, dimension( * ) RWORK, integer INFO)
        ZGELSS solves overdetermined or underdetermined systems for GE
       matrices
```

```
Purpose:
   ZGELSS computes the minimum norm solution to a complex linear
   least squares problem:

   Minimize 2-norm(| b - A*x |).

   using the singular value decomposition (SVD) of A. A is an M-by-N
   matrix which may be rank-deficient.

   Several right hand side vectors b and solution vectors x can be
   handled in a single call; they are stored as the columns of the
   M-by-NRHS right hand side matrix B and the N-by-NRHS solution matrix X.

   The effective rank of A is determined by treating as zero those
   singular values which are less than RCOND times the largest singular value.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A. M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A. N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X. NRHS >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, the first min(m,n) rows of A are overwritten with
   its right singular vectors, stored rowwise.

 LDA
   LDA is INTEGER
   The leading dimension of the array A. LDA >= max(1,M).

 B
   B is COMPLEX*16 array, dimension (LDB,NRHS)
   On entry, the M-by-NRHS right hand side matrix B.
   On exit, B is overwritten by the N-by-NRHS solution matrix X.
   If m >= n and RANK = n, the residual sum-of-squares for
   the solution in the i-th column is given by the sum of
   squares of the modulus of elements n+1:m in that column.

 LDB
```

```
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,M,N).

S
  S is DOUBLE PRECISION array, dimension (min(M,N))
  The singular values of A in decreasing order.
  The condition number of A in the 2-norm = S(1)/S(min(m,n)).

RCOND
  RCOND is DOUBLE PRECISION
  RCOND is used to determine the effective rank of A.
  Singular values S(i) <= RCOND*S(1) are treated as zero.
  If RCOND < 0, machine precision is used instead.

RANK
  RANK is INTEGER
  The effective rank of A, i.e., the number of singular values
  which are greater than RCOND*S(1).

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK >= 1, and also:
  LWORK >=  2*min(M,N) + max(M,N,NRHS)
  For good performance, LWORK should generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (5*min(M,N))

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  the algorithm for computing the SVD failed to converge;
        if INFO = i, i off-diagonal elements of an intermediate
        bidiagonal form did not converge to zero.
```

———-

23.4.3 fortran code

— zgelss.f —

```
*  =====================================================================
      SUBROUTINE ZGELSS( M, N, NRHS, A, LDA, B, LDB, S, RCOND, RANK,
     $                   WORK, LWORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDB, LWORK, M, N, NRHS, RANK
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   RWORK( * ), S( * )
      COMPLEX*16         A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      COMPLEX*16         CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0D+0, 0.0D+0 ),
     $                   CONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            BL, CHUNK, I, IASCL, IBSCL, IE, IL, IRWORK,
     $                   ITAU, ITAUP, ITAUQ, IWORK, LDWORK, MAXMN,
     $                   MAXWRK, MINMN, MINWRK, MM, MNTHR
      INTEGER            LWORK_ZGEQRF, LWORK_ZUNMQR, LWORK_ZGEBRD,
     $                   LWORK_ZUNMBR, LWORK_ZUNGBR, LWORK_ZUNMLQ,
     $                   LWORK_ZGELQF
      DOUBLE PRECISION   ANRM, BIGNUM, BNRM, EPS, SFMIN, SMLNUM, THR
*     ..
*     .. Local Arrays ..
      COMPLEX*16         DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, DLASCL, DLASET, XERBLA, ZBDSQR, ZCOPY,
     $                   ZDRSCL, ZGEBRD, ZGELQF, ZGEMM, ZGEMV, ZGEQRF,
     $                   ZLACPY, ZLASCL, ZLASET, ZUNGBR, ZUNMBR, ZUNMLQ,
     $                   ZUNMQR
```

```
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, ZLANGE
      EXTERNAL           ILAENV, DLAMCH, ZLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      MINMN = MIN( M, N )
      MAXMN = MAX( M, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, MAXMN ) ) THEN
         INFO = -7
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       CWorkspace refers to complex workspace, and RWorkspace refers
*       to real workspace. NB refers to the optimal block size for the
*       immediately following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = 1
         MAXWRK = 1
         IF( MINMN.GT.0 ) THEN
            MM = M
            MNTHR = ILAENV( 6, 'ZGELSS', ' ', M, N, NRHS, -1 )
            IF( M.GE.N .AND. M.GE.MNTHR ) THEN
*
*              Path 1a - overdetermined, with many more rows than
*                        columns
*
*              Compute space needed for ZGEQRF
               CALL ZGEQRF( M, N, A, LDA, DUM(1), DUM(1), -1, INFO )
```

```
               LWORK_ZGEQRF=DUM(1)
*              Compute space needed for ZUNMQR
               CALL ZUNMQR( 'L', 'C', M, NRHS, N, A, LDA, DUM(1), B,
     $                   LDB, DUM(1), -1, INFO )
               LWORK_ZUNMQR=DUM(1)
               MM = N
               MAXWRK = MAX( MAXWRK, N + N*ILAENV( 1, 'ZGEQRF', ' ', M,
     $                       N, -1, -1 ) )
               MAXWRK = MAX( MAXWRK, N + NRHS*ILAENV( 1, 'ZUNMQR', 'LC',
     $                       M, NRHS, N, -1 ) )
            END IF
            IF( M.GE.N ) THEN
*
*              Path 1 - overdetermined or exactly determined
*
*              Compute space needed for ZGEBRD
               CALL ZGEBRD( MM, N, A, LDA, S, DUM(1), DUM(1),
     $                      DUM(1), DUM(1), -1, INFO )
               LWORK_ZGEBRD=DUM(1)
*              Compute space needed for ZUNMBR
               CALL ZUNMBR( 'Q', 'L', 'C', MM, NRHS, N, A, LDA, DUM(1),
     $                 B, LDB, DUM(1), -1, INFO )
               LWORK_ZUNMBR=DUM(1)
*              Compute space needed for ZUNGBR
               CALL ZUNGBR( 'P', N, N, N, A, LDA, DUM(1),
     $                   DUM(1), -1, INFO )
               LWORK_ZUNGBR=DUM(1)
*              Compute total workspace needed
               MAXWRK = MAX( MAXWRK, 2*N + LWORK_ZGEBRD )
               MAXWRK = MAX( MAXWRK, 2*N + LWORK_ZUNMBR )
               MAXWRK = MAX( MAXWRK, 2*N + LWORK_ZUNGBR )
               MAXWRK = MAX( MAXWRK, N*NRHS )
               MINWRK = 2*N + MAX( NRHS, M )
            END IF
            IF( N.GT.M ) THEN
               MINWRK = 2*M + MAX( NRHS, N )
               IF( N.GE.MNTHR ) THEN
*
*                 Path 2a - underdetermined, with many more columns
*                 than rows
*
*                 Compute space needed for ZGELQF
                  CALL ZGELQF( M, N, A, LDA, DUM(1), DUM(1),
     $                 -1, INFO )
                  LWORK_ZGELQF=DUM(1)
*                 Compute space needed for ZGEBRD
                  CALL ZGEBRD( M, M, A, LDA, S, DUM(1), DUM(1),
     $                       DUM(1), DUM(1), -1, INFO )
                  LWORK_ZGEBRD=DUM(1)
*                 Compute space needed for ZUNMBR
```

```
                 CALL ZUNMBR( 'Q', 'L', 'C', M, NRHS, N, A, LDA,
     $               DUM(1), B, LDB, DUM(1), -1, INFO )
                 LWORK_ZUNMBR=DUM(1)
*                Compute space needed for ZUNGBR
                 CALL ZUNGBR( 'P', M, M, M, A, LDA, DUM(1),
     $                  DUM(1), -1, INFO )
                 LWORK_ZUNGBR=DUM(1)
*                Compute space needed for ZUNMLQ
                 CALL ZUNMLQ( 'L', 'C', N, NRHS, M, A, LDA, DUM(1),
     $                B, LDB, DUM(1), -1, INFO )
                 LWORK_ZUNMLQ=DUM(1)
*                Compute total workspace needed
                 MAXWRK = M + LWORK_ZGELQF
                 MAXWRK = MAX( MAXWRK, 3*M + M*M + LWORK_ZGEBRD )
                 MAXWRK = MAX( MAXWRK, 3*M + M*M + LWORK_ZUNMBR )
                 MAXWRK = MAX( MAXWRK, 3*M + M*M + LWORK_ZUNGBR )
                 IF( NRHS.GT.1 ) THEN
                    MAXWRK = MAX( MAXWRK, M*M + M + M*NRHS )
                 ELSE
                    MAXWRK = MAX( MAXWRK, M*M + 2*M )
                 END IF
                 MAXWRK = MAX( MAXWRK, M + LWORK_ZUNMLQ )
              ELSE
*
*                Path 2 - underdetermined
*
*                Compute space needed for ZGEBRD
                 CALL ZGEBRD( M, N, A, LDA, S, DUM(1), DUM(1),
     $                    DUM(1), DUM(1), -1, INFO )
                 LWORK_ZGEBRD=DUM(1)
*                Compute space needed for ZUNMBR
                 CALL ZUNMBR( 'Q', 'L', 'C', M, NRHS, M, A, LDA,
     $               DUM(1), B, LDB, DUM(1), -1, INFO )
                 LWORK_ZUNMBR=DUM(1)
*                Compute space needed for ZUNGBR
                 CALL ZUNGBR( 'P', M, N, M, A, LDA, DUM(1),
     $                  DUM(1), -1, INFO )
                 LWORK_ZUNGBR=DUM(1)
                 MAXWRK = 2*M + LWORK_ZGEBRD
                 MAXWRK = MAX( MAXWRK, 2*M + LWORK_ZUNMBR )
                 MAXWRK = MAX( MAXWRK, 2*M + LWORK_ZUNGBR )
                 MAXWRK = MAX( MAXWRK, N*NRHS )
              END IF
           END IF
           MAXWRK = MAX( MINWRK, MAXWRK )
        END IF
        WORK( 1 ) = MAXWRK
*
        IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY )
     $     INFO = -12
```

```
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGELSS', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         RANK = 0
         RETURN
      END IF
*
*     Get machine parameters
*
      EPS = DLAMCH( 'P' )
      SFMIN = DLAMCH( 'S' )
      SMLNUM = SFMIN / EPS
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = ZLANGE( 'M', M, N, A, LDA, RWORK )
      IASCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL ZLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
         IASCL = 1
      ELSE IF( ANRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL ZLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
         IASCL = 2
      ELSE IF( ANRM.EQ.ZERO ) THEN
*
*        Matrix all zero. Return zero solution.
*
         CALL ZLASET( 'F', MAX( M, N ), NRHS, CZERO, CZERO, B, LDB )
         CALL DLASET( 'F', MINMN, 1, ZERO, ZERO, S, MINMN )
         RANK = 0
         GO TO 70
      END IF
*
```

```
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = ZLANGE( 'M', M, NRHS, B, LDB, RWORK )
      IBSCL = 0
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL ZLASCL( 'G', 0, 0, BNRM, SMLNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 1
      ELSE IF( BNRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL ZLASCL( 'G', 0, 0, BNRM, BIGNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 2
      END IF
*
*     Overdetermined case
*
      IF( M.GE.N ) THEN
*
*        Path 1 - overdetermined or exactly determined
*
         MM = M
         IF( M.GE.MNTHR ) THEN
*
*           Path 1a - overdetermined, with many more rows than columns
*
            MM = N
            ITAU = 1
            IWORK = ITAU + N
*
*           Compute A=Q*R
*           (CWorkspace: need 2*N, prefer N+N*NB)
*           (RWorkspace: none)
*
            CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
     $                   LWORK-IWORK+1, INFO )
*
*           Multiply B by transpose(Q)
*           (CWorkspace: need N+NRHS, prefer N+NRHS*NB)
*           (RWorkspace: none)
*
            CALL ZUNMQR( 'L', 'C', M, NRHS, N, A, LDA, WORK( ITAU ), B,
     $                   LDB, WORK( IWORK ), LWORK-IWORK+1, INFO )
*
*           Zero out below R
*
            IF( N.GT.1 )
```

```
     $         CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO, A( 2, 1 ),
     $                      LDA )
         END IF
*
         IE = 1
         ITAUQ = 1
         ITAUP = ITAUQ + N
         IWORK = ITAUP + N
*
*        Bidiagonalize R in A
*        (CWorkspace: need 2*N+MM, prefer 2*N+(MM+N)*NB)
*        (RWorkspace: need N)
*
         CALL ZGEBRD( MM, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors of R
*        (CWorkspace: need 2*N+NRHS, prefer 2*N+NRHS*NB)
*        (RWorkspace: none)
*
         CALL ZUNMBR( 'Q', 'L', 'C', MM, NRHS, N, A, LDA, WORK( ITAUQ ),
     $                B, LDB, WORK( IWORK ), LWORK-IWORK+1, INFO )
*
*        Generate right bidiagonalizing vectors of R in A
*        (CWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*        (RWorkspace: none)
*
         CALL ZUNGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                WORK( IWORK ), LWORK-IWORK+1, INFO )
         IRWORK = IE + N
*
*        Perform bidiagonal QR iteration
*          multiply B by transpose of left singular vectors
*          compute right singular vectors in A
*        (CWorkspace: none)
*        (RWorkspace: need BDSPAC)
*
         CALL ZBDSQR( 'U', N, N, 0, NRHS, S, RWORK( IE ), A, LDA, DUM,
     $                1, B, LDB, RWORK( IRWORK ), INFO )
         IF( INFO.NE.0 )
     $      GO TO 70
*
*        Multiply B by reciprocals of singular values
*
         THR = MAX( RCOND*S( 1 ), SFMIN )
         IF( RCOND.LT.ZERO )
     $      THR = MAX( EPS*S( 1 ), SFMIN )
         RANK = 0
         DO 10 I = 1, N
```

```
            IF( S( I ).GT.THR ) THEN
               CALL ZDRSCL( NRHS, S( I ), B( I, 1 ), LDB )
               RANK = RANK + 1
            ELSE
               CALL ZLASET( 'F', 1, NRHS, CZERO, CZERO, B( I, 1 ), LDB )
            END IF
   10    CONTINUE
*
*        Multiply B by right singular vectors
*        (CWorkspace: need N, prefer N*NRHS)
*        (RWorkspace: none)
*
         IF( LWORK.GE.LDB*NRHS .AND. NRHS.GT.1 ) THEN
            CALL ZGEMM( 'C', 'N', N, NRHS, N, CONE, A, LDA, B, LDB,
     $                  CZERO, WORK, LDB )
            CALL ZLACPY( 'G', N, NRHS, WORK, LDB, B, LDB )
         ELSE IF( NRHS.GT.1 ) THEN
            CHUNK = LWORK / N
            DO 20 I = 1, NRHS, CHUNK
               BL = MIN( NRHS-I+1, CHUNK )
               CALL ZGEMM( 'C', 'N', N, BL, N, CONE, A, LDA, B( 1, I ),
     $                     LDB, CZERO, WORK, N )
               CALL ZLACPY( 'G', N, BL, WORK, N, B( 1, I ), LDB )
   20       CONTINUE
         ELSE
            CALL ZGEMV( 'C', N, N, CONE, A, LDA, B, 1, CZERO, WORK, 1 )
            CALL ZCOPY( N, WORK, 1, B, 1 )
         END IF
*
      ELSE IF( N.GE.MNTHR .AND. LWORK.GE.3*M+M*M+MAX( M, NRHS, N-2*M ) )
     $          THEN
*
*        Underdetermined case, M much less than N
*
*        Path 2a - underdetermined, with many more columns than rows
*        and sufficient workspace for an efficient algorithm
*
         LDWORK = M
         IF( LWORK.GE.3*M+M*LDA+MAX( M, NRHS, N-2*M ) )
     $      LDWORK = LDA
         ITAU = 1
         IWORK = M + 1
*
*        Compute A=L*Q
*        (CWorkspace: need 2*M, prefer M+M*NB)
*        (RWorkspace: none)
*
         CALL ZGELQF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
     $                LWORK-IWORK+1, INFO )
         IL = IWORK
```

```
*
*        Copy L to WORK(IL), zeroing out above it
*
         CALL ZLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWORK )
         CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO, WORK( IL+LDWORK ),
     $                LDWORK )
         IE = 1
         ITAUQ = IL + LDWORK*M
         ITAUP = ITAUQ + M
         IWORK = ITAUP + M
*
*        Bidiagonalize L in WORK(IL)
*        (CWorkspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*        (RWorkspace: need M)
*
         CALL ZGEBRD( M, M, WORK( IL ), LDWORK, S, RWORK( IE ),
     $                WORK( ITAUQ ), WORK( ITAUP ), WORK( IWORK ),
     $                LWORK-IWORK+1, INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors of L
*        (CWorkspace: need M*M+3*M+NRHS, prefer M*M+3*M+NRHS*NB)
*        (RWorkspace: none)
*
         CALL ZUNMBR( 'Q', 'L', 'C', M, NRHS, M, WORK( IL ), LDWORK,
     $                WORK( ITAUQ ), B, LDB, WORK( IWORK ),
     $                LWORK-IWORK+1, INFO )
*
*        Generate right bidiagonalizing vectors of R in WORK(IL)
*        (CWorkspace: need M*M+4*M-1, prefer M*M+3*M+(M-1)*NB)
*        (RWorkspace: none)
*
         CALL ZUNGBR( 'P', M, M, M, WORK( IL ), LDWORK, WORK( ITAUP ),
     $                WORK( IWORK ), LWORK-IWORK+1, INFO )
         IRWORK = IE + M
*
*        Perform bidiagonal QR iteration, computing right singular
*        vectors of L in WORK(IL) and multiplying B by transpose of
*        left singular vectors
*        (CWorkspace: need M*M)
*        (RWorkspace: need BDSPAC)
*
         CALL ZBDSQR( 'U', M, M, 0, NRHS, S, RWORK( IE ), WORK( IL ),
     $                LDWORK, A, LDA, B, LDB, RWORK( IRWORK ), INFO )
         IF( INFO.NE.0 )
     $      GO TO 70
*
*        Multiply B by reciprocals of singular values
*
         THR = MAX( RCOND*S( 1 ), SFMIN )
         IF( RCOND.LT.ZERO )
```

```
     $      THR = MAX( EPS*S( 1 ), SFMIN )
         RANK = 0
         DO 30 I = 1, M
            IF( S( I ).GT.THR ) THEN
               CALL ZDRSCL( NRHS, S( I ), B( I, 1 ), LDB )
               RANK = RANK + 1
            ELSE
               CALL ZLASET( 'F', 1, NRHS, CZERO, CZERO, B( I, 1 ), LDB )
            END IF
   30    CONTINUE
         IWORK = IL + M*LDWORK
*
*        Multiply B by right singular vectors of L in WORK(IL)
*        (CWorkspace: need M*M+2*M, prefer M*M+M+M*NRHS)
*        (RWorkspace: none)
*
         IF( LWORK.GE.LDB*NRHS+IWORK-1 .AND. NRHS.GT.1 ) THEN
            CALL ZGEMM( 'C', 'N', M, NRHS, M, CONE, WORK( IL ), LDWORK,
     $                  B, LDB, CZERO, WORK( IWORK ), LDB )
            CALL ZLACPY( 'G', M, NRHS, WORK( IWORK ), LDB, B, LDB )
         ELSE IF( NRHS.GT.1 ) THEN
            CHUNK = ( LWORK-IWORK+1 ) / M
            DO 40 I = 1, NRHS, CHUNK
               BL = MIN( NRHS-I+1, CHUNK )
               CALL ZGEMM( 'C', 'N', M, BL, M, CONE, WORK( IL ), LDWORK,
     $                     B( 1, I ), LDB, CZERO, WORK( IWORK ), M )
               CALL ZLACPY( 'G', M, BL, WORK( IWORK ), M, B( 1, I ),
     $                      LDB )
   40       CONTINUE
         ELSE
            CALL ZGEMV( 'C', M, M, CONE, WORK( IL ), LDWORK, B( 1, 1 ),
     $                  1, CZERO, WORK( IWORK ), 1 )
            CALL ZCOPY( M, WORK( IWORK ), 1, B( 1, 1 ), 1 )
         END IF
*
*        Zero out below first M rows of B
*
         CALL ZLASET( 'F', N-M, NRHS, CZERO, CZERO, B( M+1, 1 ), LDB )
         IWORK = ITAU + M
*
*        Multiply transpose(Q) by B
*        (CWorkspace: need M+NRHS, prefer M+NHRS*NB)
*        (RWorkspace: none)
*
         CALL ZUNMLQ( 'L', 'C', N, NRHS, M, A, LDA, WORK( ITAU ), B,
     $                LDB, WORK( IWORK ), LWORK-IWORK+1, INFO )
*
      ELSE
*
*        Path 2 - remaining underdetermined cases
```

```
*
         IE = 1
         ITAUQ = 1
         ITAUP = ITAUQ + M
         IWORK = ITAUP + M
*
*        Bidiagonalize A
*        (CWorkspace: need 3*M, prefer 2*M+(M+N)*NB)
*        (RWorkspace: need N)
*
         CALL ZGEBRD( M, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                INFO )
*
*        Multiply B by transpose of left bidiagonalizing vectors
*        (CWorkspace: need 2*M+NRHS, prefer 2*M+NRHS*NB)
*        (RWorkspace: none)
*
         CALL ZUNMBR( 'Q', 'L', 'C', M, NRHS, N, A, LDA, WORK( ITAUQ ),
     $                B, LDB, WORK( IWORK ), LWORK-IWORK+1, INFO )
*
*        Generate right bidiagonalizing vectors in A
*        (CWorkspace: need 3*M, prefer 2*M+M*NB)
*        (RWorkspace: none)
*
         CALL ZUNGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                WORK( IWORK ), LWORK-IWORK+1, INFO )
         IRWORK = IE + M
*
*        Perform bidiagonal QR iteration,
*           computing right singular vectors of A in A and
*           multiplying B by transpose of left singular vectors
*        (CWorkspace: none)
*        (RWorkspace: need BDSPAC)
*
         CALL ZBDSQR( 'L', M, N, 0, NRHS, S, RWORK( IE ), A, LDA, DUM,
     $                1, B, LDB, RWORK( IRWORK ), INFO )
         IF( INFO.NE.0 )
     $      GO TO 70
*
*        Multiply B by reciprocals of singular values
*
         THR = MAX( RCOND*S( 1 ), SFMIN )
         IF( RCOND.LT.ZERO )
     $      THR = MAX( EPS*S( 1 ), SFMIN )
         RANK = 0
         DO 50 I = 1, M
            IF( S( I ).GT.THR ) THEN
               CALL ZDRSCL( NRHS, S( I ), B( I, 1 ), LDB )
               RANK = RANK + 1
```

```
            ELSE
               CALL ZLASET( 'F', 1, NRHS, CZERO, CZERO, B( I, 1 ), LDB )
            END IF
   50    CONTINUE
*
*        Multiply B by right singular vectors of A
*        (CWorkspace: need N, prefer N*NRHS)
*        (RWorkspace: none)
*
         IF( LWORK.GE.LDB*NRHS .AND. NRHS.GT.1 ) THEN
            CALL ZGEMM( 'C', 'N', N, NRHS, M, CONE, A, LDA, B, LDB,
     $                  CZERO, WORK, LDB )
            CALL ZLACPY( 'G', N, NRHS, WORK, LDB, B, LDB )
         ELSE IF( NRHS.GT.1 ) THEN
            CHUNK = LWORK / N
            DO 60 I = 1, NRHS, CHUNK
               BL = MIN( NRHS-I+1, CHUNK )
               CALL ZGEMM( 'C', 'N', N, BL, M, CONE, A, LDA, B( 1, I ),
     $                     LDB, CZERO, WORK, N )
               CALL ZLACPY( 'F', N, BL, WORK, N, B( 1, I ), LDB )
   60       CONTINUE
         ELSE
            CALL ZGEMV( 'C', M, N, CONE, A, LDA, B, 1, CZERO, WORK, 1 )
            CALL ZCOPY( N, WORK, 1, B, 1 )
         END IF
      END IF
*
*     Undo scaling
*
      IF( IASCL.EQ.1 ) THEN
         CALL ZLASCL( 'G', 0, 0, ANRM, SMLNUM, N, NRHS, B, LDB, INFO )
         CALL DLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
     $                INFO )
      ELSE IF( IASCL.EQ.2 ) THEN
         CALL ZLASCL( 'G', 0, 0, ANRM, BIGNUM, N, NRHS, B, LDB, INFO )
         CALL DLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                INFO )
      END IF
      IF( IBSCL.EQ.1 ) THEN
         CALL ZLASCL( 'G', 0, 0, SMLNUM, BNRM, N, NRHS, B, LDB, INFO )
      ELSE IF( IBSCL.EQ.2 ) THEN
         CALL ZLASCL( 'G', 0, 0, BIGNUM, BNRM, N, NRHS, B, LDB, INFO )
      END IF
   70 CONTINUE
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of ZGELSS
*
      END
```

———-

23.4.4 lisp code

— zgelss.lisp —

```
(let* ((zero 0.0d0)
       (one 1.0d0)
       (czero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (cone (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (f2cl-lib:complex16) czero)
           (type (f2cl-lib:complex16) cone)
           (ignorable zero one czero cone))
  (defun zgelss (m n nrhs a lda b ldb$ s rcond rank work lwork rwork info)
    (declare (type (double-float) rcond)
             (type (array double-float (*)) rwork s)
             (type (array f2cl-lib:complex16 (*)) work b a)
             (type (f2cl-lib:integer4) info lwork rank ldb$ lda nrhs n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (s double-float s-%data% s-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (prog ((dum (make-array 1 :element-type 'f2cl-lib:complex16))
             (anrm 0.0d0) (bignum 0.0d0) (bnrm 0.0d0) (eps 0.0d0) (sfmin 0.0d0)
             (smlnum 0.0d0) (thr 0.0d0) (lwork_zgeqrf 0) (lwork_zunmqr 0)
             (lwork_zgebrd 0) (lwork_zunmbr 0) (lwork_zungbr 0)
             (lwork_zunmlq 0) (lwork_zgelqf 0) (bl 0) (chunk 0) (i 0) (iascl 0)
             (ibscl 0) (ie 0) (il 0) (irwork 0) (itau 0) (itaup 0) (itauq 0)
             (iwork 0) (ldwork 0) (maxmn 0) (maxwrk 0) (minmn 0) (minwrk 0)
             (mm 0) (mnthr 0) (lquery nil))
        (declare (type (array f2cl-lib:complex16 (1)) dum)
                 (type (double-float) anrm bignum bnrm eps sfmin smlnum thr)
                 (type (f2cl-lib:integer4) lwork_zgeqrf lwork_zunmqr
                  lwork_zgebrd lwork_zunmbr lwork_zungbr lwork_zunmlq
                  lwork_zgelqf bl chunk i iascl ibscl ie il irwork itau itaup
                  itauq iwork ldwork maxmn maxwrk minmn minwrk mm mnthr)
                 (type f2cl-lib:logical lquery))
        (setf info 0)
        (setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (setf maxmn (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< nrhs 0) (setf info -3))
              ((< lda
```

```
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
        (setf info -5))
       ((< ldb$
            (max (the f2cl-lib:integer4 1)
                 (the f2cl-lib:integer4 maxmn)))
        (setf info -7)))
 (cond
  ((= info 0) (setf minwrk 1) (setf maxwrk 1)
   (cond
    ((> minmn 0) (setf mm m)
     (setf mnthr
             (multiple-value-bind
                 (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                 (ilaenv 6 "ZGELSS" " " m n nrhs -1)
               (declare (ignore var-0 var-1 var-2 var-6))
               (setf m var-3)
               (setf n var-4)
               (setf nrhs var-5)
               ret-val))
     (cond
      ((and (>= m n) (>= m mnthr))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
           (zgeqrf m n a lda
            (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
            (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1))) -1
            info)
         (declare (ignore var-2 var-4 var-5 var-6))
         (when var-0 (setf m var-0))
         (when var-1 (setf n var-1))
         (when var-3 (setf lda var-3))
         (when var-7 (setf info var-7)))
       (setf lwork_zgeqrf
               (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10 var-11 var-12)
           (zunmqr "L" "C" m nrhs n a lda
            (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1))) b
            ldb$
            (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1))) -1
            info)
         (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
         (when var-2 (setf m var-2))
         (when var-3 (setf nrhs var-3))
         (when var-4 (setf n var-4))
         (when var-6 (setf lda var-6))
         (when var-9 (setf ldb$ var-9))
         (when var-12 (setf info var-12)))
       (setf lwork_zunmqr
```

```
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(setf mm n)
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add n
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "ZGEQRF"
                                                           " " m
                                                           n -1
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-5
                                                          var-6))
                                                        (setf m
                                                                var-3)
                                                        (setf n
                                                                var-4)
                                                        ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add n
                                    (f2cl-lib:int-mul nrhs
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
```

```
                                                                    "ZUNMQR"
                                                                    "LC" m
                                                                    nrhs n
                                                                    -1)
                                                                 (declare
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-6))
                                                                 (setf m
                                                                         var-3)
                                                                 (setf nrhs
                                                                         var-4)
                                                                 (setf n
                                                                         var-5)
                                                                 ret-val))))))))
                 (cond
                  ((>= m n)
                   (multiple-value-bind
                       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                        var-10)
                       (zgebrd mm n a lda s
                        (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
                        (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
                        (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
                        (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1))) -1
                        info)
                     (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
                     (setf mm var-0)
                     (setf n var-1)
                     (setf lda var-3)
                     (setf info var-10))
                   (setf lwork_zgebrd
                           (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
                   (multiple-value-bind
                       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                        var-10 var-11 var-12 var-13)
                       (zunmbr "Q" "L" "C" mm nrhs n a lda
                        (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1))) b
                        ldb$
                        (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1))) -1
                        info)
                     (declare
                      (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
                     (when var-3 (setf mm var-3))
                     (when var-4 (setf nrhs var-4))
                     (when var-5 (setf n var-5))
                     (when var-7 (setf lda var-7))
                     (when var-10 (setf ldb$ var-10))
```

```
      (when var-13 (setf info var-13)))
    (setf lwork_zunmbr
            (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (zungbr "P" n n n a lda
         (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
         (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1))) -1
         info)
      (declare (ignore var-0 var-4 var-6 var-7 var-8))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf n var-3))
      (when var-5 (setf lda var-5))
      (when var-9 (setf info var-9)))
    (setf lwork_zungbr
            (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        lwork_zgebrd))))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        lwork_zunmbr))))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        lwork_zungbr))))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4 (f2cl-lib:int-mul n nrhs))))
    (setf minwrk
            (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                              (max (the f2cl-lib:integer4 nrhs)
                                   (the f2cl-lib:integer4 m))))))
  (cond
   ((> n m)
    (setf minwrk
            (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                              (max (the f2cl-lib:integer4 nrhs)
                                   (the f2cl-lib:integer4 n))))
    (cond
     ((>= n mnthr)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (zgelqf m n a lda
```

```
     (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
     (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
     -1 info)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-7))
(setf lwork_zgelqf
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd m m a lda s
     (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
     (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
     (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
     (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
     -1 info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf info var-10))
(setf lwork_zgebrd
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (zunmbr "Q" "L" "C" m nrhs n a lda
     (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
     b ldb$
     (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
     -1 info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(setf lwork_zunmbr
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "P" m m m a lda
     (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
     (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
```

```
     -1 info)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf info var-9)))
(setf lwork_zungbr
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (zunmlq "L" "C" n nrhs m a lda
     (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
     b ldb$
     (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
     -1 info)
  (declare
   (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf n var-2))
  (when var-3 (setf nrhs var-3))
  (when var-4 (setf m var-4))
  (when var-6 (setf lda var-6))
  (when var-9 (setf ldb$ var-9))
  (when var-12 (setf info var-12)))
(setf lwork_zunmlq
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(setf maxwrk (f2cl-lib:int-add m lwork_zgelqf))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    (f2cl-lib:int-mul m m)
                                    lwork_zgebrd))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    (f2cl-lib:int-mul m m)
                                    lwork_zunmbr))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    (f2cl-lib:int-mul m m)
                                    lwork_zungbr))))
(cond
 ((> nrhs 1)
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
```

```
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul m m) m
                                       (f2cl-lib:int-mul m
                                                         nrhs))))))
  (t
   (setf maxwrk
           (max (the f2cl-lib:integer4 maxwrk)
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                       (f2cl-lib:int-mul 2
                                                         m)))))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add m lwork_zunmlq)))))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10)
     (zgebrd m n a lda s
      (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
      (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
      (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
      (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
      -1 info)
   (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf info var-10))
 (setf lwork_zgebrd
         (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12 var-13)
     (zunmbr "Q" "L" "C" m nrhs m a lda
      (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
      b ldb$
      (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
      -1 info)
   (declare
    (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
   (when var-3 (setf m var-3))
   (when var-4 (setf nrhs var-4))
   (when var-5 (setf m var-5))
   (when var-7 (setf lda var-7))
   (when var-10 (setf ldb$ var-10))
   (when var-13 (setf info var-13)))
 (setf lwork_zunmbr
         (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
```

```
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9)
                (zungbr "P" m n m a lda
                 (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
                 (f2cl-lib:array-slice dum f2cl-lib:complex16 (1) ((1 1)))
                 -1 info)
              (declare (ignore var-0 var-4 var-6 var-7 var-8))
              (when var-1 (setf m var-1))
              (when var-2 (setf n var-2))
              (when var-3 (setf m var-3))
              (when var-5 (setf lda var-5))
              (when var-9 (setf info var-9)))
            (setf lwork_zungbr
                    (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
            (setf maxwrk
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) lwork_zgebrd))
            (setf maxwrk
                    (max (the f2cl-lib:integer4 maxwrk)
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                lwork_zunmbr))))
            (setf maxwrk
                    (max (the f2cl-lib:integer4 maxwrk)
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                lwork_zungbr))))
            (setf maxwrk
                    (max (the f2cl-lib:integer4 maxwrk)
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-mul n nrhs))))))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 minwrk)
                     (the f2cl-lib:integer4 maxwrk)))))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce maxwrk 'f2cl-lib:complex16))
      (if (and (< lwork minwrk) (not lquery))
          (setf info -12))))
    (cond
     ((/= info 0) (xerbla "ZGELSS" (f2cl-lib:int-sub info)) (go end_label))
     (lquery (go end_label)))
    (cond ((or (= m 0) (= n 0)) (setf rank 0) (go end_label)))
    (setf eps (dlamch "P"))
    (setf sfmin (dlamch "S"))
    (setf smlnum (/ sfmin eps))
    (setf bignum (/ one smlnum))
    (multiple-value-bind (var-0 var-1)
        (dlabad smlnum bignum)
      (declare (ignore))
      (when var-0 (setf smlnum var-0))
```

```
  (when var-1 (setf bignum var-1)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "M" m n a lda rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iascl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 anrm smlnum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 1))
 ((> anrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 anrm bignum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 2))
 ((= anrm zero)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "F"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       czero czero b ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldb$ var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "F" minmn 1 zero zero s minmn)
    (declare (ignore var-0 var-2 var-5))
    (when var-1 (setf minmn var-1))
```

```
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf minmn var-6)))
  (setf rank 0) (go label70)))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "M" m nrhs b ldb$ rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ibscl 0)
(cond
 ((and (> bnrm zero) (< bnrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 bnrm smlnum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 1))
 ((> bnrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 bnrm bignum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 2)))
(cond
 ((>= m n) (setf mm m)
  (cond
   ((>= m mnthr) (setf mm n) (setf itau 1)
    (setf iwork (f2cl-lib:int-add itau n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (zgeqrf m n a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
```

```
                              ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
        (declare (ignore var-2 var-4 var-5 var-6))
        (when var-0 (setf m var-0))
        (when var-1 (setf n var-1))
        (when var-3 (setf lda var-3))
        (when var-7 (setf info var-7)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (zunmqr "L" "C" m nrhs n a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                              ((1 *)) work-%offset%)
           b ldb$
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                              ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf nrhs var-3))
        (when var-4 (setf n var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldb$ var-9))
        (when var-12 (setf info var-12)))
      (if (> n 1)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (zlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
               czero czero
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (2 1)
                                     ((1 lda) (1 *)) a-%offset%)
               lda)
            (declare (ignore var-0 var-1 var-2 var-5))
            (when var-3 (setf czero var-3))
            (when var-4 (setf czero var-4))
            (when var-6 (setf lda var-6))))))
    (setf ie 1) (setf itauq 1) (setf itaup (f2cl-lib:int-add itauq n))
    (setf iwork (f2cl-lib:int-add itaup n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (zgebrd mm n a lda s
         (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                               rwork-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                               ((1 *)) work-%offset%)
```

```
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf mm var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "Q" "L" "C" mm nrhs n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf mm var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (zungbr "P" n n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf info var-9)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" n n 0 nrhs s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     a lda dum 1 b ldb$
     (f2cl-lib:array-slice rwork-%data% double-float (irwork) ((1 *))
                           rwork-%offset%)
     info)
  (declare
```

```
   (ignore var-0 var-3 var-5 var-6 var-7 var-9 var-10 var-11 var-13))
  (setf n var-1)
  (setf n var-2)
  (setf nrhs var-4)
  (setf lda var-8)
  (setf ldb$ var-12)
  (setf info var-14))
(if (/= info 0)
    (go label70))
(setf thr
        (max
         (* rcond (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
         sfmin))
(if (< rcond zero)
    (setf thr
            (max
             (* eps (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
             sfmin)))
(setf rank 0)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref s (i) ((1 *))) thr)
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (zdrscl nrhs
           (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-2))
        (setf nrhs var-0)
        (setf ldb$ var-3))
      (setf rank (f2cl-lib:int-add rank 1)))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlaset "F" 1 nrhs czero czero
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-1 var-5))
        (when var-2 (setf nrhs var-2))
        (when var-3 (setf czero var-3))
        (when var-4 (setf czero var-4))
        (when var-6 (setf ldb$ var-6)))))
   label10))
(cond
 ((and (>= lwork (f2cl-lib:int-mul ldb$ nrhs)) (> nrhs 1))
  (multiple-value-bind
```

```
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (zgemm "C" "N" n nrhs n cone a lda b ldb$ czero work ldb$)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldb$ var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf ldb$ var-12)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "G" n nrhs work ldb$ b ldb$)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf nrhs var-2))
    (when var-4 (setf ldb$ var-4))
    (when var-6 (setf ldb$ var-6))))
 ((> nrhs 1) (setf chunk (the f2cl-lib:integer4 (truncate lwork n)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
                ((> i nrhs) nil)
    (tagbody
      (setf bl
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub nrhs i) 1))
               (the f2cl-lib:integer4 chunk)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (zgemm "C" "N" n bl n cone a lda
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 i)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$ czero work n)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf n var-2))
        (when var-3 (setf bl var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf cone var-5))
        (when var-7 (setf lda var-7))
        (when var-9 (setf ldb$ var-9))
        (when var-10 (setf czero var-10))
        (when var-12 (setf n var-12)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlacpy "G" n bl work n
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 i)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
```

```
            (declare (ignore var-0 var-3 var-5))
            (when var-1 (setf n var-1))
            (when var-2 (setf bl var-2))
            (when var-4 (setf n var-4))
            (when var-6 (setf ldb$ var-6)))
        label20)))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (zgemv "C" n n cone a lda b 1 czero work 1)
       (declare (ignore var-0 var-4 var-6 var-7 var-9 var-10))
       (when var-1 (setf n var-1))
       (when var-2 (setf n var-2))
       (when var-3 (setf cone var-3))
       (when var-5 (setf lda var-5))
       (when var-8 (setf czero var-8)))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (zcopy n work 1 b 1)
       (declare (ignore var-1 var-2 var-3 var-4))
       (when var-0 (setf n var-0))))))
  ((and (>= n mnthr)
        (>= lwork
            (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                              (f2cl-lib:int-mul m m)
                              (max (the f2cl-lib:integer4 m)
                                   (the f2cl-lib:integer4 nrhs)
                                   (the f2cl-lib:integer4
                                        (f2cl-lib:int-add n
                                                          (f2cl-lib:int-mul
                                                           -1 2 m)))))))
   (setf ldwork m)
   (if (>= lwork
           (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                             (f2cl-lib:int-mul m lda)
                             (max (the f2cl-lib:integer4 m)
                                  (the f2cl-lib:integer4 nrhs)
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-sub n
                                                         (f2cl-lib:int-mul
                                                          2 m))))))
       (setf ldwork lda))
   (setf itau 1) (setf iwork (f2cl-lib:int-add m 1))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (zgelqf m n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                              ((1 *)) work-%offset%)
```

```
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-7))
(setf il iwork)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                           ((1 *)) work-%offset%)
     ldwork)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwork var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1) czero
     czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ il ldwork)) ((1 *)) work-%offset%)
     ldwork)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwork var-6)))
(setf ie 1)
(setf itauq (f2cl-lib:int-add il (f2cl-lib:int-mul ldwork m)))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                           ((1 *)) work-%offset%)
     ldwork s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
```

```
  (setf ldwork var-3)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "Q" "L" "C" m nrhs m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                           ((1 *)) work-%offset%)
     ldwork
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwork var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (zungbr "P" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                           ((1 *)) work-%offset%)
     ldwork
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwork var-5))
  (when var-9 (setf info var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" m m 0 nrhs s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                           ((1 *)) work-%offset%)
     ldwork a lda b ldb$
```

```
     (f2cl-lib:array-slice rwork-%data% double-float (irwork) ((1 *))
                           rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-3 var-5 var-6 var-7 var-9 var-11 var-13))
  (setf m var-1)
  (setf m var-2)
  (setf nrhs var-4)
  (setf ldwork var-8)
  (setf lda var-10)
  (setf ldb$ var-12)
  (setf info var-14))
(if (/= info 0)
    (go label70))
(setf thr
        (max
         (* rcond (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
         sfmin))
(if (< rcond zero)
    (setf thr
            (max
             (* eps (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
             sfmin)))
(setf rank 0)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref s (i) ((1 *))) thr)
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (zdrscl nrhs
           (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-2))
        (setf nrhs var-0)
        (setf ldb$ var-3))
      (setf rank (f2cl-lib:int-add rank 1)))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlaset "F" 1 nrhs czero czero
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-1 var-5))
        (when var-2 (setf nrhs var-2))
        (when var-3 (setf czero var-3))
        (when var-4 (setf czero var-4))
```

```
        (when var-6 (setf ldb$ var-6)))))
   label30))
(setf iwork (f2cl-lib:int-add il (f2cl-lib:int-mul m ldwork)))
(cond
 ((and
   (>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul ldb$ nrhs) iwork
                         (f2cl-lib:int-sub 1)))
   (> nrhs 1))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (zgemm "C" "N" m nrhs m cone
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                             ((1 *)) work-%offset%)
       ldwork b ldb$ czero
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                             ((1 *)) work-%offset%)
       ldb$)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf ldwork var-7))
    (when var-9 (setf ldb$ var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf ldb$ var-12)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "G" m nrhs
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                             ((1 *)) work-%offset%)
       ldb$ b ldb$)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf nrhs var-2))
    (when var-4 (setf ldb$ var-4))
    (when var-6 (setf ldb$ var-6))))
 ((> nrhs 1)
  (setf chunk
          (the f2cl-lib:integer4 (truncate (+ (- lwork iwork) 1) m)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
                ((> i nrhs) nil)
    (tagbody
      (setf bl
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub nrhs i) 1))
               (the f2cl-lib:integer4 chunk)))
      (multiple-value-bind
```

```
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12)
            (zgemm "C" "N" m bl m cone
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                                   ((1 *)) work-%offset%)
             ldwork
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 i)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$ czero
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                   (iwork) ((1 *)) work-%offset%)
             m)
          (declare (ignore var-0 var-1 var-6 var-8 var-11))
          (when var-2 (setf m var-2))
          (when var-3 (setf bl var-3))
          (when var-4 (setf m var-4))
          (when var-5 (setf cone var-5))
          (when var-7 (setf ldwork var-7))
          (when var-9 (setf ldb$ var-9))
          (when var-10 (setf czero var-10))
          (when var-12 (setf m var-12)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zlacpy "G" m bl
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                   (iwork) ((1 *)) work-%offset%)
             m
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 i)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf bl var-2))
          (when var-4 (setf m var-4))
          (when var-6 (setf ldb$ var-6)))
       label40)))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (zgemv "C" m m cone
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                                ((1 *)) work-%offset%)
          ldwork
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          1 czero
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                                ((1 *)) work-%offset%)
          1)
```

```
        (declare (ignore var-0 var-4 var-6 var-7 var-9 var-10))
        (when var-1 (setf m var-1))
        (when var-2 (setf m var-2))
        (when var-3 (setf cone var-3))
        (when var-5 (setf ldwork var-5))
        (when var-8 (setf czero var-8)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (zcopy m
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                                 ((1 *)) work-%offset%)
           1
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf m var-0)))))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlaset "F" (f2cl-lib:int-sub n m) nrhs czero czero
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 ((+ m 1) 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf nrhs var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldb$ var-6)))
    (setf iwork (f2cl-lib:int-add itau m))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (zunmlq "L" "C" n nrhs m a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                               ((1 *)) work-%offset%)
         b ldb$
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf n var-2))
      (when var-3 (setf nrhs var-3))
      (when var-4 (setf m var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldb$ var-9))
      (when var-12 (setf info var-12))))
  (t (setf ie 1) (setf itauq 1) (setf itaup (f2cl-lib:int-add itauq m))
   (setf iwork (f2cl-lib:int-add itaup m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (zgebrd m n a lda s
```

```
      (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                            rwork-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "Q" "L" "C" m nrhs n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     b ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf nrhs var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (zungbr "P" m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) info)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf info var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14)
```

```
    (zbdsqr "L" m n 0 nrhs s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     a lda dum 1 b ldb$
     (f2cl-lib:array-slice rwork-%data% double-float (irwork) ((1 *))
                           rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-3 var-5 var-6 var-7 var-9 var-10 var-11 var-13))
  (setf m var-1)
  (setf n var-2)
  (setf nrhs var-4)
  (setf lda var-8)
  (setf ldb$ var-12)
  (setf info var-14))
(if (/= info 0)
    (go label70))
(setf thr
        (max
         (* rcond (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
         sfmin))
(if (< rcond zero)
    (setf thr
            (max
             (* eps (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
             sfmin)))
(setf rank 0)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref s (i) ((1 *))) thr)
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (zdrscl nrhs
           (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-2))
        (setf nrhs var-0)
        (setf ldb$ var-3))
      (setf rank (f2cl-lib:int-add rank 1)))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlaset "F" 1 nrhs czero czero
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-1 var-5))
```

```
        (when var-2 (setf nrhs var-2))
        (when var-3 (setf czero var-3))
        (when var-4 (setf czero var-4))
        (when var-6 (setf ldb$ var-6)))))
   label50))
(cond
 ((and (>= lwork (f2cl-lib:int-mul ldb$ nrhs)) (> nrhs 1))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (zgemm "C" "N" n nrhs m cone a lda b ldb$ czero work ldb$)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldb$ var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf ldb$ var-12)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "G" n nrhs work ldb$ b ldb$)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf nrhs var-2))
    (when var-4 (setf ldb$ var-4))
    (when var-6 (setf ldb$ var-6))))
 ((> nrhs 1) (setf chunk (the f2cl-lib:integer4 (truncate lwork n)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
                ((> i nrhs) nil)
    (tagbody
      (setf bl
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub nrhs i) 1))
               (the f2cl-lib:integer4 chunk)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (zgemm "C" "N" n bl m cone a lda
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 i)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$ czero work n)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf n var-2))
        (when var-3 (setf bl var-3))
        (when var-4 (setf m var-4))
        (when var-5 (setf cone var-5))
        (when var-7 (setf lda var-7))
        (when var-9 (setf ldb$ var-9))
```

```
            (when var-10 (setf czero var-10))
            (when var-12 (setf n var-12)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (zlacpy "F" n bl work n
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 i)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-0 var-3 var-5))
            (when var-1 (setf n var-1))
            (when var-2 (setf bl var-2))
            (when var-4 (setf n var-4))
            (when var-6 (setf ldb$ var-6)))
        label60)))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (zgemv "C" m n cone a lda b 1 czero work 1)
       (declare (ignore var-0 var-4 var-6 var-7 var-9 var-10))
       (when var-1 (setf m var-1))
       (when var-2 (setf n var-2))
       (when var-3 (setf cone var-3))
       (when var-5 (setf lda var-5))
       (when var-8 (setf czero var-8)))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (zcopy n work 1 b 1)
       (declare (ignore var-1 var-2 var-3 var-4))
       (when var-0 (setf n var-0)))))))
(cond
 ((= iascl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 anrm smlnum n nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf n var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 smlnum anrm minmn 1 s minmn info)
    (declare (ignore var-0 var-1 var-2 var-6 var-7))
    (when var-3 (setf smlnum var-3))
    (when var-4 (setf anrm var-4))
    (when var-5 (setf minmn var-5))
    (when var-8 (setf minmn var-8))
    (when var-9 (setf info var-9))))
```

```
   ((= iascl 2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (zlascl "G" 0 0 anrm bignum n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf bignum var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 bignum anrm minmn 1 s minmn info)
      (declare (ignore var-0 var-1 var-2 var-6 var-7))
      (when var-3 (setf bignum var-3))
      (when var-4 (setf anrm var-4))
      (when var-5 (setf minmn var-5))
      (when var-8 (setf minmn var-8))
      (when var-9 (setf info var-9)))))
  (cond
   ((= ibscl 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (zlascl "G" 0 0 smlnum bnrm n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf smlnum var-3))
      (when var-4 (setf bnrm var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9))))
   ((= ibscl 2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (zlascl "G" 0 0 bignum bnrm n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bignum var-3))
      (when var-4 (setf bnrm var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))))
 label70
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce maxwrk 'f2cl-lib:complex16))
  (go end_label)
 end_label
  (return
   (values m n nrhs nil lda nil ldb$ nil nil rank nil nil nil info))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgelss
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (double-float)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$ nil nil
                                              fortran-to-lisp::rank nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::zdrscl
                                              fortran-to-lisp::zbdsqr
                                              fortran-to-lisp::xerbla
                                              fortran-to-lisp::zgelqf
                                              fortran-to-lisp::zgebrd
                                              fortran-to-lisp::ilaenv))))
```

23.5 zgelsx - DEPRECATED use zgelsy

23.6 zgelsy

23.6.1 Axiom unit tests

— **zgelsy.input** —

```
)set break resume
)sys rm -f zgelsy.output
)spool zgelsy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

23.6.2 Axiom help page

— **zgelsy.help** —

```
======================================================================
zgelsy examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgelsy (M, N, NRHS, A, LDA, B, LDB, JPVT, RCOND, RANK, WORK,
           LWORK, RWORK, INFO)
            ZGELSY solves overdetermined or underdetermined systems for GE
           matrices

Function/Subroutine Documentation
   subroutine zgelsy (integer M, integer N, integer NRHS, complex*16,
       dimension( lda, * ) A, integer LDA, complex*16, dimension( ldb, * ) B,
       integer LDB, integer, dimension( * ) JPVT, double precision RCOND,
       integer RANK, complex*16, dimension( * ) WORK, integer LWORK, double
       precision, dimension( * ) RWORK, integer INFO)
```

```
       ZGELSY solves overdetermined or underdetermined systems for GE
      matrices

Purpose:
   ZGELSY computes the minimum-norm solution to a complex linear least
   squares problem:
       minimize || A * X - B ||
   using a complete orthogonal factorization of A.  A is an M-by-N
   matrix which may be rank-deficient.

   Several right hand side vectors b and solution vectors x can be
   handled in a single call; they are stored as the columns of the
   M-by-NRHS right hand side matrix B and the N-by-NRHS solution matrix X.

   The routine first computes a QR factorization with column pivoting:
          A * P = Q * [ R11 R12 ]
                      [  0  R22 ]
   with R11 defined as the largest leading submatrix whose estimated
   condition number is less than 1/RCOND.  The order of R11, RANK,
   is the effective rank of A.

   Then, R22 is considered to be negligible, and R12 is annihilated
   by unitary transformations from the right, arriving at the
   complete orthogonal factorization:
         A * P = Q * [ T11 0 ] * Z
                     [  0  0 ]
   The minimum-norm solution is then
         X = P * Z**H [ inv(T11)*Q1**H*B ]
                      [        0         ]
   where Q1 consists of the first RANK columns of Q.

   This routine is basically identical to the original xGELSX except
   three differences:
   o The permutation of matrix B (the right hand side) is faster and
     more simple.
   o The call to the subroutine xGEQPF has been substituted by the
     the call to the subroutine xGEQP3. This subroutine is a Blas-3
     version of the QR factorization with column pivoting.
   o Matrix B (the right hand side) is updated with Blas-3.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.
```

```
NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of
  columns of matrices B and X. NRHS >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit, A has been overwritten by details of its
  complete orthogonal factorization.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  On entry, the M-by-NRHS right hand side matrix B.
  On exit, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= max(1,M,N).

JPVT
  JPVT is INTEGER array, dimension (N)
  On entry, if JPVT(i) .ne. 0, the i-th column of A is permuted
  to the front of AP, otherwise column i is a free column.
  On exit, if JPVT(i) = k, then the i-th column of A*P
  was the k-th column of A.

RCOND
  RCOND is DOUBLE PRECISION
  RCOND is used to determine the effective rank of A, which
  is defined as the order of the largest leading triangular
  submatrix R11 in the QR factorization with pivoting of A,
  whose estimated condition number < 1/RCOND.

RANK
  RANK is INTEGER
  The effective rank of A, i.e., the order of the submatrix
  R11.  This is the same as the order of the submatrix T11
  in the complete orthogonal factorization of A.

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
```

```
  The dimension of the array WORK.
  The unblocked strategy requires that:
  LWORK >= MN + MAX( 2*MN, N+1, MN+NRHS )
  where MN = min(M,N).
  The block algorithm requires that:
  LWORK >= MN + MAX( 2*MN, NB*(N+1), MN+MN*NB, MN+NB*NRHS )
  where NB is an upper bound on the blocksize returned
  by ILAENV for the routines ZGEQP3, ZTZRZF, CTZRQF, ZUNMQR,
  and ZUNMRZ.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

 RWORK
  RWORK is DOUBLE PRECISION array, dimension (2*N)

 INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
```

———-

23.6.3 fortran code

— **zgelsy.f** —

```
*  ======================================================================
      SUBROUTINE ZGELSY( M, N, NRHS, A, LDA, B, LDB, JPVT, RCOND, RANK,
     $                   WORK, LWORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDB, LWORK, M, N, NRHS, RANK
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            JPVT( * )
      DOUBLE PRECISION   RWORK( * )
      COMPLEX*16         A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
```

```
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            IMAX, IMIN
      PARAMETER          ( IMAX = 1, IMIN = 2 )
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      COMPLEX*16         CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0D+0, 0.0D+0 ),
     $                   CONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IASCL, IBSCL, ISMAX, ISMIN, J, LWKOPT, MN,
     $                   NB, NB1, NB2, NB3, NB4
      DOUBLE PRECISION   ANRM, BIGNUM, BNRM, SMAX, SMAXPR, SMIN, SMINPR,
     $                   SMLNUM, WSIZE
      COMPLEX*16         C1, C2, S1, S2
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, XERBLA, ZCOPY, ZGEQP3, ZLAIC1, ZLASCL,
     $                   ZLASET, ZTRSM, ZTZRZF, ZUNMQR, ZUNMRZ
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, ZLANGE
      EXTERNAL           ILAENV, DLAMCH, ZLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DCMPLX, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      MN = MIN( M, N )
      ISMIN = MN + 1
      ISMAX = 2*MN + 1
*
*     Test the input arguments.
*
      INFO = 0
      NB1 = ILAENV( 1, 'ZGEQRF', ' ', M, N, -1, -1 )
      NB2 = ILAENV( 1, 'ZGERQF', ' ', M, N, -1, -1 )
      NB3 = ILAENV( 1, 'ZUNMQR', ' ', M, N, NRHS, -1 )
      NB4 = ILAENV( 1, 'ZUNMRQ', ' ', M, N, NRHS, -1 )
      NB = MAX( NB1, NB2, NB3, NB4 )
      LWKOPT = MAX( 1, MN+2*N+NB*( N+1 ), 2*MN+NB*NRHS )
      WORK( 1 ) = DCMPLX( LWKOPT )
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
```

```
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, M, N ) ) THEN
         INFO = -7
      ELSE IF( LWORK.LT.( MN+MAX( 2*MN, N+1, MN+NRHS ) ) .AND. .NOT.
     $         LQUERY ) THEN
         INFO = -12
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGELSY', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( MIN( M, N, NRHS ).EQ.0 ) THEN
         RANK = 0
         RETURN
      END IF
*
*     Get machine parameters
*
      SMLNUM = DLAMCH( 'S' ) / DLAMCH( 'P' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
*
*     Scale A, B if max entries outside range [SMLNUM,BIGNUM]
*
      ANRM = ZLANGE( 'M', M, N, A, LDA, RWORK )
      IASCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL ZLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
         IASCL = 1
      ELSE IF( ANRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL ZLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
         IASCL = 2
```

```
      ELSE IF( ANRM.EQ.ZERO ) THEN
*
*        Matrix all zero. Return zero solution.
*
         CALL ZLASET( 'F', MAX( M, N ), NRHS, CZERO, CZERO, B, LDB )
         RANK = 0
         GO TO 70
      END IF
*
      BNRM = ZLANGE( 'M', M, NRHS, B, LDB, RWORK )
      IBSCL = 0
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
*
*        Scale matrix norm up to SMLNUM
*
         CALL ZLASCL( 'G', 0, 0, BNRM, SMLNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 1
      ELSE IF( BNRM.GT.BIGNUM ) THEN
*
*        Scale matrix norm down to BIGNUM
*
         CALL ZLASCL( 'G', 0, 0, BNRM, BIGNUM, M, NRHS, B, LDB, INFO )
         IBSCL = 2
      END IF
*
*     Compute QR factorization with column pivoting of A:
*        A * P = Q * R
*
      CALL ZGEQP3( M, N, A, LDA, JPVT, WORK( 1 ), WORK( MN+1 ),
     $             LWORK-MN, RWORK, INFO )
      WSIZE = MN + DBLE( WORK( MN+1 ) )
*
*     complex workspace: MN+NB*(N+1). real workspace 2*N.
*     Details of Householder rotations stored in WORK(1:MN).
*
*     Determine RANK using incremental condition estimation
*
      WORK( ISMIN ) = CONE
      WORK( ISMAX ) = CONE
      SMAX = ABS( A( 1, 1 ) )
      SMIN = SMAX
      IF( ABS( A( 1, 1 ) ).EQ.ZERO ) THEN
         RANK = 0
         CALL ZLASET( 'F', MAX( M, N ), NRHS, CZERO, CZERO, B, LDB )
         GO TO 70
      ELSE
         RANK = 1
      END IF
*
   10 CONTINUE
```

```
      IF( RANK.LT.MN ) THEN
         I = RANK + 1
         CALL ZLAIC1( IMIN, RANK, WORK( ISMIN ), SMIN, A( 1, I ),
     $                A( I, I ), SMINPR, S1, C1 )
         CALL ZLAIC1( IMAX, RANK, WORK( ISMAX ), SMAX, A( 1, I ),
     $                A( I, I ), SMAXPR, S2, C2 )
*
         IF( SMAXPR*RCOND.LE.SMINPR ) THEN
            DO 20 I = 1, RANK
               WORK( ISMIN+I-1 ) = S1*WORK( ISMIN+I-1 )
               WORK( ISMAX+I-1 ) = S2*WORK( ISMAX+I-1 )
   20       CONTINUE
            WORK( ISMIN+RANK ) = C1
            WORK( ISMAX+RANK ) = C2
            SMIN = SMINPR
            SMAX = SMAXPR
            RANK = RANK + 1
            GO TO 10
         END IF
      END IF
*
*     complex workspace: 3*MN.
*
*     Logically partition R = [ R11 R12 ]
*                             [  0  R22 ]
*     where R11 = R(1:RANK,1:RANK)
*
*     [R11,R12] = [ T11, 0 ] * Y
*
      IF( RANK.LT.N )
     $   CALL ZTZRZF( RANK, N, A, LDA, WORK( MN+1 ), WORK( 2*MN+1 ),
     $                LWORK-2*MN, INFO )
*
*     complex workspace: 2*MN.
*     Details of Householder rotations stored in WORK(MN+1:2*MN)
*
*     B(1:M,1:NRHS) := Q**H * B(1:M,1:NRHS)
*
      CALL ZUNMQR( 'Left', 'Conjugate transpose', M, NRHS, MN, A, LDA,
     $             WORK( 1 ), B, LDB, WORK( 2*MN+1 ), LWORK-2*MN, INFO )
      WSIZE = MAX( WSIZE, 2*MN+DBLE( WORK( 2*MN+1 ) ) )
*
*     complex workspace: 2*MN+NB*NRHS.
*
*     B(1:RANK,1:NRHS) := inv(T11) * B(1:RANK,1:NRHS)
*
      CALL ZTRSM( 'Left', 'Upper', 'No transpose', 'Non-unit', RANK,
     $            NRHS, CONE, A, LDA, B, LDB )
*
      DO 40 J = 1, NRHS
```

```
         DO 30 I = RANK + 1, N
            B( I, J ) = CZERO
   30    CONTINUE
   40 CONTINUE
*
*     B(1:N,1:NRHS) := Y**H * B(1:N,1:NRHS)
*
      IF( RANK.LT.N ) THEN
         CALL ZUNMRZ( 'Left', 'Conjugate transpose', N, NRHS, RANK,
     $                N-RANK, A, LDA, WORK( MN+1 ), B, LDB,
     $                WORK( 2*MN+1 ), LWORK-2*MN, INFO )
      END IF
*
*     complex workspace: 2*MN+NRHS.
*
*     B(1:N,1:NRHS) := P * B(1:N,1:NRHS)
*
      DO 60 J = 1, NRHS
         DO 50 I = 1, N
            WORK( JPVT( I ) ) = B( I, J )
   50    CONTINUE
         CALL ZCOPY( N, WORK( 1 ), 1, B( 1, J ), 1 )
   60 CONTINUE
*
*     complex workspace: N.
*
*     Undo scaling
*
      IF( IASCL.EQ.1 ) THEN
         CALL ZLASCL( 'G', 0, 0, ANRM, SMLNUM, N, NRHS, B, LDB, INFO )
         CALL ZLASCL( 'U', 0, 0, SMLNUM, ANRM, RANK, RANK, A, LDA,
     $                INFO )
      ELSE IF( IASCL.EQ.2 ) THEN
         CALL ZLASCL( 'G', 0, 0, ANRM, BIGNUM, N, NRHS, B, LDB, INFO )
         CALL ZLASCL( 'U', 0, 0, BIGNUM, ANRM, RANK, RANK, A, LDA,
     $                INFO )
      END IF
      IF( IBSCL.EQ.1 ) THEN
         CALL ZLASCL( 'G', 0, 0, SMLNUM, BNRM, N, NRHS, B, LDB, INFO )
      ELSE IF( IBSCL.EQ.2 ) THEN
         CALL ZLASCL( 'G', 0, 0, BIGNUM, BNRM, N, NRHS, B, LDB, INFO )
      END IF
*
   70 CONTINUE
      WORK( 1 ) = DCMPLX( LWKOPT )
*
      RETURN
*
*     End of ZGELSY
*
```

```
      END
```

———-

23.6.4 lisp code

— **zgelsy.lisp** —

```
(let* ((imax 1)
       (imin 2)
       (zero 0.0d0)
       (one 1.0d0)
       (czero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (cone (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:integer4 1 1) imax)
           (type (f2cl-lib:integer4 2 2) imin)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (f2cl-lib:complex16) czero)
           (type (f2cl-lib:complex16) cone)
           (ignorable imax imin zero one czero cone))
  (defun zgelsy (m n nrhs a lda b ldb$ jpvt rcond rank work lwork rwork info)
    (declare (type (array double-float (*)) rwork)
             (type (double-float) rcond)
             (type (array f2cl-lib:integer4 (*)) jpvt)
             (type (array f2cl-lib:complex16 (*)) work b a)
             (type (f2cl-lib:integer4) info lwork rank ldb$ lda nrhs n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (jpvt f2cl-lib:integer4 jpvt-%data% jpvt-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (prog ((c1 #C(0.0d0 0.0d0)) (c2 #C(0.0d0 0.0d0)) (s1 #C(0.0d0 0.0d0))
             (s2 #C(0.0d0 0.0d0)) (anrm 0.0d0) (bignum 0.0d0) (bnrm 0.0d0)
             (smax 0.0d0) (smaxpr 0.0d0) (smin 0.0d0) (sminpr 0.0d0)
             (smlnum 0.0d0) (wsize 0.0d0) (i 0) (iascl 0) (ibscl 0) (ismax 0)
             (ismin 0) (j 0) (lwkopt 0) (mn 0) (nb 0) (nb1 0) (nb2 0) (nb3 0)
             (nb4 0) (lquery nil))
        (declare (type (f2cl-lib:complex16) c1 c2 s1 s2)
                 (type (double-float) anrm bignum bnrm smax smaxpr smin sminpr
                  smlnum wsize)
                 (type (f2cl-lib:integer4) i iascl ibscl ismax ismin j lwkopt
                  mn nb nb1 nb2 nb3 nb4)
                 (type f2cl-lib:logical lquery))
        (setf mn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (setf ismin (f2cl-lib:int-add mn 1))
        (setf ismax (f2cl-lib:int-add (f2cl-lib:int-mul 2 mn) 1))
```

```
(setf info 0)
(setf nb1
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 1 "ZGEQRF" " " m n -1 -1)
          (declare (ignore var-0 var-1 var-2 var-5 var-6))
          (setf m var-3)
          (setf n var-4)
          ret-val))
(setf nb2
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 1 "ZGERQF" " " m n -1 -1)
          (declare (ignore var-0 var-1 var-2 var-5 var-6))
          (setf m var-3)
          (setf n var-4)
          ret-val))
(setf nb3
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 1 "ZUNMQR" " " m n nrhs -1)
          (declare (ignore var-0 var-1 var-2 var-6))
          (setf m var-3)
          (setf n var-4)
          (setf nrhs var-5)
          ret-val))
(setf nb4
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 1 "ZUNMRQ" " " m n nrhs -1)
          (declare (ignore var-0 var-1 var-2 var-6))
          (setf m var-3)
          (setf n var-4)
          (setf nrhs var-5)
          ret-val))
(setf nb
        (max (the f2cl-lib:integer4 nb1) (the f2cl-lib:integer4 nb2)
             (the f2cl-lib:integer4 nb3) (the f2cl-lib:integer4 nb4)))
(setf lwkopt
        (max (the f2cl-lib:integer4 1)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add mn (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul nb
                                                      (f2cl-lib:int-add
                                                       n 1))))
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 mn)
                                    (f2cl-lib:int-mul nb nrhs)))))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (f2cl-lib:dcmplx lwkopt))
```

```
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
      ((< nrhs 0) (setf info -3))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -5))
      ((< ldb$
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)
               (the f2cl-lib:integer4 n)))
       (setf info -7))
      ((and
        (< lwork
           (f2cl-lib:int-add mn
                             (max
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-mul 2 mn))
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add n 1))
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add mn nrhs)))))
        (not lquery))
       (setf info -12)))
(cond
 ((/= info 0) (xerbla "ZGELSY" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond
 ((=
   (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)
        (the f2cl-lib:integer4 nrhs))
   0)
  (setf rank 0) (go end_label)))
(setf smlnum (/ (dlamch "S") (dlamch "P")))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "M" m n a lda rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iascl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum))
```

```
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 anrm smlnum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 1))
 ((> anrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 anrm bignum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))
  (setf iascl 2))
 ((= anrm zero)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "F"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       czero czero b ldb$)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nrhs var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldb$ var-6)))
  (setf rank 0) (go label70)))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "M" m nrhs b ldb$ rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ibscl 0)
(cond
 ((and (> bnrm zero) (< bnrm smlnum))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 bnrm smlnum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
```

```
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 1))
 ((> bnrm bignum)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 bnrm bignum m nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf bnrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (setf ibscl 2)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (zgeqp3 m n a lda jpvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (1) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 ((+ mn 1))
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-sub lwork mn) rwork info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-9 (setf info var-9)))
(setf wsize
        (+ mn
           (f2cl-lib:dble
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-add mn 1))
                           ((1 *)) work-%offset%))))
(setf (f2cl-lib:fref work-%data% (ismin) ((1 *)) work-%offset%) cone)
(setf (f2cl-lib:fref work-%data% (ismax) ((1 *)) work-%offset%) cone)
(setf smax
        (abs
         (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)))
(setf smin smax)
(cond
 ((= (abs (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))) zero) (setf rank 0)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "F"
       (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) nrhs
       czero czero b ldb$)
    (declare (ignore var-0 var-1 var-5))
```

```
        (when var-2 (setf nrhs var-2))
        (when var-3 (setf czero var-3))
        (when var-4 (setf czero var-4))
        (when var-6 (setf ldb$ var-6)))
      (go label70))
    (t (setf rank 1)))
  label10
   (cond
    ((< rank mn) (setf i (f2cl-lib:int-add rank 1))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (zlaic1 imin rank
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ismin)
                                ((1 *)) work-%offset%)
          smin
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 i)
                                ((1 lda) (1 *)) a-%offset%)
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                ((1 lda) (1 *)) a-%offset%)
          sminpr s1 c1)
        (declare (ignore var-2 var-4 var-5))
        (when var-0 (setf imin var-0))
        (when var-1 (setf rank var-1))
        (when var-3 (setf smin var-3))
        (when var-6 (setf sminpr var-6))
        (when var-7 (setf s1 var-7))
        (when var-8 (setf c1 var-8)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (zlaic1 imax rank
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ismax)
                                ((1 *)) work-%offset%)
          smax
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 i)
                                ((1 lda) (1 *)) a-%offset%)
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                ((1 lda) (1 *)) a-%offset%)
          smaxpr s2 c2)
        (declare (ignore var-2 var-4 var-5))
        (when var-0 (setf imax var-0))
        (when var-1 (setf rank var-1))
        (when var-3 (setf smax var-3))
        (when var-6 (setf smaxpr var-6))
        (when var-7 (setf s2 var-7))
        (when var-8 (setf c2 var-8)))
     (cond
      ((<= (* smaxpr rcond) sminpr)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i rank) nil)
         (tagbody
```

```
            (setf (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-add ismin i) 1))
                                 ((1 *)) work-%offset%)
                  (* s1
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-sub
                                      (f2cl-lib:int-add ismin i) 1))
                                    ((1 *)) work-%offset%)))
            (setf (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-add ismax i) 1))
                                 ((1 *)) work-%offset%)
                  (* s2
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-sub
                                      (f2cl-lib:int-add ismax i) 1))
                                    ((1 *)) work-%offset%)))
          label20))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ismin rank))
                             ((1 *)) work-%offset%)
              c1)
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ismax rank))
                             ((1 *)) work-%offset%)
              c2)
        (setf smin sminpr) (setf smax smaxpr)
        (setf rank (f2cl-lib:int-add rank 1)) (go label10)))))
    (if (< rank n)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (ztzrzf rank n a lda
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                   ((+ mn 1)) ((1 *)) work-%offset%)
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                   ((+ (f2cl-lib:int-mul 2 mn) 1)) ((1 *))
                                   work-%offset%)
             (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 mn)) info)
          (declare (ignore var-2 var-4 var-5 var-6))
          (when var-0 (setf rank var-0))
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-7 (setf info var-7))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
         var-11 var-12)
        (zunmqr "Left" "Conjugate transpose" m nrhs mn a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (1) ((1 *))
                               work-%offset%)
         b ldb$
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
```

```
                              ((+ (f2cl-lib:int-mul 2 mn) 1)) ((1 *))
                              work-%offset%)
      (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 mn)) info)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf nrhs var-3))
  (when var-4 (setf mn var-4))
  (when var-6 (setf lda var-6))
  (when var-9 (setf ldb$ var-9))
  (when var-12 (setf info var-12)))
(setf wsize
        (max wsize
             (+ (f2cl-lib:int-mul 2 mn)
                (f2cl-lib:dble
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add
                                  (f2cl-lib:int-mul 2 mn) 1))
                                ((1 *)) work-%offset%)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrsm "Left" "Upper" "No transpose" "Non-unit" rank nrhs cone a
     lda b ldb$)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf rank var-4))
  (when var-5 (setf nrhs var-5))
  (when var-6 (setf cone var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf ldb$ var-10)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (f2cl-lib:fdo (i (f2cl-lib:int-add rank 1) (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)
                czero)
       label30))
   label40))
(cond
 ((< rank n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (zunmrz "Left" "Conjugate transpose" n nrhs rank
       (f2cl-lib:int-sub n rank) a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 ((+ mn 1))
                             ((1 *)) work-%offset%)
       b ldb$
```

```
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              ((+ (f2cl-lib:int-mul 2 mn) 1)) ((1 *))
                              work-%offset%)
        (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 mn)) info)
    (declare
     (ignore var-0 var-1 var-5 var-6 var-8 var-9 var-11 var-12))
    (when var-2 (setf n var-2))
    (when var-3 (setf nrhs var-3))
    (when var-4 (setf rank var-4))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldb$ var-10))
    (when var-13 (setf info var-13)))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data%
                              ((f2cl-lib:fref jpvt (i) ((1 *)))) ((1 *))
                              work-%offset%)
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%))
       label50))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zcopy n
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (1)
                               ((1 *)) work-%offset%)
         1
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
   label60))
(cond
 ((= iascl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 anrm smlnum n nrhs b ldb$ info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf n var-5))
    (when var-6 (setf nrhs var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-9 (setf info var-9)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "U" 0 0 smlnum anrm rank rank a lda info)
```

```
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf smlnum var-3))
      (when var-4 (setf anrm var-4))
      (when var-5 (setf rank var-5))
      (when var-6 (setf rank var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf info var-9))))
    ((= iascl 2)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (zlascl "G" 0 0 anrm bignum n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf bignum var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (zlascl "U" 0 0 bignum anrm rank rank a lda info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bignum var-3))
      (when var-4 (setf anrm var-4))
      (when var-5 (setf rank var-5))
      (when var-6 (setf rank var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf info var-9)))))
  (cond
    ((= ibscl 1)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (zlascl "G" 0 0 smlnum bnrm n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf smlnum var-3))
      (when var-4 (setf bnrm var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf info var-9))))
    ((= ibscl 2)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (zlascl "G" 0 0 bignum bnrm n nrhs b ldb$ info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bignum var-3))
      (when var-4 (setf bnrm var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf nrhs var-6))
      (when var-8 (setf ldb$ var-8))
```

```
            (when var-9 (setf info var-9)))))
       label70
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (f2cl-lib:dcmplx lwkopt))
        (go end_label)
       end_label
        (return
         (values m n nrhs nil lda nil ldb$ nil nil rank nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgelsy
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (double-float)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$ nil nil
                                              fortran-to-lisp::rank nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::xerbla
                                              fortran-to-lisp::ilaenv))))
```

23.7 zgesv

23.7.1 Axiom unit tests

— **zgesv.input** —

```
)set break resume
)sys rm -f zgesv.output
)spool zgesv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

23.7.2 Axiom help page

— **zgesv.help** —

```
====================================================================
zgesv examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgesv (N, NRHS, A, LDA, IPIV, B, LDB, INFO)
            ZGESV computes the solution to system of linear equations A * X =
           B for GE matrices (simple driver)

Function/Subroutine Documentation
   subroutine zgesv (integer N, integer NRHS, complex*16, dimension( lda, * )
       A, integer LDA, integer, dimension( * ) IPIV, complex*16, dimension(
       ldb, * ) B, integer LDB, integer INFO)
        ZGESV computes the solution to system of linear equations A * X = B
       for GE matrices (simple driver)

Purpose:
   ZGESV computes the solution to a complex system of linear equations
               A * X = B,
```

```
  where A is an N-by-N matrix and X and B are N-by-NRHS matrices.

  The LU decomposition with partial pivoting and row interchanges is
  used to factor A as
              A = P * L * U,
  where P is a permutation matrix, L is unit lower triangular, and U is
  upper triangular.  The factored form of A is then used to solve the
  system of equations A * X = B.

Parameters:

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the N-by-N coefficient matrix A.
   On exit, the factors L and U from the factorization
   A = P*L*U; the unit diagonal elements of L are not stored.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   The pivot indices that define the permutation matrix P;
   row i of the matrix was interchanged with row IPIV(i).

 B
   B is COMPLEX*16 array, dimension (LDB,NRHS)
   On entry, the N-by-NRHS matrix of right hand side matrix B.
   On exit, if INFO = 0, the N-by-NRHS solution matrix X.

 LDB
   LDB is INTEGER
   The leading dimension of the array B.  LDB >= max(1,N).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, U(i,i) is exactly zero.  The factorization
         has been completed, but the factor U is exactly
```

```
singular, so the solution could not be computed.
```

———-

23.7.3 fortran code

— **zgesv.f** —

```
*  =====================================================================
      SUBROUTINE ZGESV( N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGETRF, ZGETRS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
```

```
         CALL XERBLA( 'ZGESV ', -INFO )
         RETURN
      END IF
*
*     Compute the LU factorization of A.
*
      CALL ZGETRF( N, N, A, LDA, IPIV, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
         CALL ZGETRS( 'No transpose', N, NRHS, A, LDA, IPIV, B, LDB,
     $                INFO )
      END IF
      RETURN
*
*     End of ZGESV
*
      END
```

———-

23.7.4 lisp code

— **zgesv.lisp** —

```
(defun zgesv (n nrhs a lda ipiv b ldb$ info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex16 (*)) b a)
           (type (f2cl-lib:integer4) info ldb$ lda nrhs n))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex16 a-%data% a-%offset%)
       (b f2cl-lib:complex16 b-%data% b-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ()
      (declare)
      (setf info 0)
      (cond ((< n 0) (setf info -1)) ((< nrhs 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -4))
            ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -7)))
      (cond
       ((/= info 0) (xerbla "ZGESV " (f2cl-lib:int-sub info)) (go end_label)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (zgetrf n n a lda ipiv info)
        (declare (ignore var-2 var-4))
        (when var-0 (setf n var-0))
```

```
        (when var-1 (setf n var-1))
        (when var-3 (setf lda var-3))
        (when var-5 (setf info var-5)))
      (cond
       ((= info 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (zgetrs "No transpose" n nrhs a lda ipiv b ldb$ info)
          (declare (ignore var-0 var-3 var-5 var-6))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf lda var-4))
          (when var-7 (setf ldb$ var-7))
          (when var-8 (setf info var-8)))))
      (go end_label)
     end_label
      (return (values n nrhs nil lda nil nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgesv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

23.8 zgesvx

23.8.1 Axiom unit tests

— **zgesvx.input** —

```
)set break resume
)sys rm -f zgesvx.output
)spool zgesvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

23.8.2 Axiom help page

— **zgesvx.help** —

```
======================================================================
zgesvx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgesvx (FACT, TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV, EQUED,
           R, C, B, LDB, X, LDX, RCOND, FERR, BERR, WORK, RWORK, INFO)
            ZGESVX computes the solution to system of linear equations A * X =
           B for GE matrices

Function/Subroutine Documentation
   subroutine zgesvx (character FACT, character TRANS, integer N, integer
       NRHS, complex*16, dimension( lda, * ) A, integer LDA, complex*16,
       dimension( ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV,
       character EQUED, double precision, dimension( * ) R, double precision,
       dimension( * ) C, complex*16, dimension( ldb, * ) B, integer LDB,
       complex*16, dimension( ldx, * ) X, integer LDX, double precision RCOND,
       double precision, dimension( * ) FERR, double precision, dimension( * )
       BERR, complex*16, dimension( * ) WORK, double precision, dimension( * )
```

```
       RWORK, integer INFO)
        ZGESVX computes the solution to system of linear equations A * X = B
       for GE matrices

Purpose:
   ZGESVX uses the LU factorization to compute the solution to a complex
   system of linear equations
               A * X = B,
   where A is an N-by-N matrix and X and B are N-by-NRHS matrices.

   Error bounds on the solution and a condition estimate are also provided.

Description:
   The following steps are performed:

   1. If FACT = 'E', real scaling factors are computed to equilibrate
      the system:
      TRANS = 'N':  diag(R)*A*diag(C)     *inv(diag(C))*X = diag(R)*B
      TRANS = 'T': (diag(R)*A*diag(C))**T *inv(diag(R))*X = diag(C)*B
      TRANS = 'C': (diag(R)*A*diag(C))**H *inv(diag(R))*X = diag(C)*B
      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(R)*A*diag(C) and B by diag(R)*B (if TRANS='N')
      or diag(C)*B (if TRANS = 'T' or 'C').

   2. If FACT = 'N' or 'E', the LU decomposition is used to factor the
      matrix A (after equilibration if FACT = 'E') as
         A = P * L * U,
      where P is a permutation matrix, L is a unit lower triangular
      matrix, and U is upper triangular.

   3. If some U(i,i)=0, so that U is exactly singular, then the routine
      returns with INFO = i. Otherwise, the factored form of A is used
      to estimate the condition number of the matrix A.  If the
      reciprocal of the condition number is less than machine precision,
      INFO = N+1 is returned as a warning, but the routine still goes on
      to solve for X and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(C) (if TRANS = 'N') or diag(R) (if TRANS = 'T' or 'C') so
      that it solves the original system before equilibration.

Parameters:

 FACT
```

```
  FACT is CHARACTER*1
  Specifies whether or not the factored form of the matrix A is
  supplied on entry, and if not, whether the matrix A should be
  equilibrated before it is factored.
  = 'F':  On entry, AF and IPIV contain the factored form of A.
          If EQUED is not 'N', the matrix A has been
          equilibrated with scaling factors given by R and C.
          A, AF, and IPIV are not modified.
  = 'N':  The matrix A will be copied to AF and factored.
  = 'E':  The matrix A will be equilibrated if necessary, then
          copied to AF and factored.

TRANS
  TRANS is CHARACTER*1
  Specifies the form of the system of equations:
  = 'N':  A * X = B     (No transpose)
  = 'T':  A**T * X = B  (Transpose)
  = 'C':  A**H * X = B  (Conjugate transpose)

N
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the N-by-N matrix A.  If FACT = 'F' and EQUED is
  not 'N', then A must have been equilibrated by the scaling
  factors in R and/or C.  A is not modified if FACT = 'F' or
  'N', or if FACT = 'E' and EQUED = 'N' on exit.

  On exit, if EQUED .ne. 'N', A is scaled as follows:
  EQUED = 'R':  A := diag(R) * A
  EQUED = 'C':  A := A * diag(C)
  EQUED = 'B':  A := diag(R) * A * diag(C).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX*16 array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the factors L and U from the factorization
  A = P*L*U as computed by ZGETRF.  If EQUED .ne. 'N', then
  AF is the factored form of the equilibrated matrix A.
```

```
  If FACT = 'N', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the equilibrated matrix A (see the description of A for
  the form of the equilibrated matrix).

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains the pivot indices from the factorization A = P*L*U
  as computed by ZGETRF; row i of the matrix was interchanged
  with row IPIV(i).

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the equilibrated matrix A.

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
  = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
  = 'B':  Both row and column equilibration, i.e., A has been
          replaced by diag(R) * A * diag(C).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

R
  R is DOUBLE PRECISION array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
  EQUED = 'R' or 'B', each element of R must be positive.

C
  C is DOUBLE PRECISION array, dimension (N)
```

```
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if TRANS = 'N' and EQUED = 'R' or 'B', B is overwritten by
  diag(R)*B;
  if TRANS = 'T' or 'C' and EQUED = 'C' or 'B', B is
  overwritten by diag(C)*B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X
  to the original system of equations.  Note that A and B are
  modified on exit if EQUED .ne. 'N', and the solution to the
  equilibrated system is inv(diag(C))*X if TRANS = 'N' and
  EQUED = 'C' or 'B', or inv(diag(R))*X if TRANS = 'T' or 'C'
  and EQUED = 'R' or 'B'.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  The estimate of the reciprocal condition number of the matrix
  A after equilibration (if done).  If RCOND is less than the
  machine precision (in particular, if RCOND = 0), the matrix
  is singular to working precision.  This condition is
  indicated by a return code of INFO > 0.

FERR
  FERR is DOUBLE PRECISION array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
```

```
  overestimate of the true error.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is COMPLEX*16 array, dimension (2*N)

RWORK
  RWORK is DOUBLE PRECISION array, dimension (2*N)
  On exit, RWORK(1) contains the reciprocal pivot growth
  factor norm(A)/norm(U). The "max absolute element" norm is
  used. If RWORK(1) is much less than 1, then the stability
  of the LU factorization of the (equilibrated) matrix A
  could be poor. This also means that the solution X, condition
  estimator RCOND, and forward error bound FERR could be
  unreliable. If factorization fails with 0<INFO<=N, then
  RWORK(1) contains the reciprocal pivot growth factor for the
  leading INFO columns of A.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, and i is
    <= N:  U(i,i) is exactly zero.  The factorization has
      been completed, but the factor U is exactly
      singular, so the solution and error bounds
      could not be computed. RCOND = 0 is returned.
    = N+1: U is nonsingular, but RCOND is less than machine
      precision, meaning that the matrix is singular
      to working precision.  Nevertheless, the
      solution and error bounds are computed because
      there are a number of situations where the
      computed solution can be more accurate than the
      value of RCOND would suggest.
```

———-

23.8.3 fortran code

— zgesvx.f —

```
*  ==================================================================
```

```
      SUBROUTINE ZGESVX( FACT, TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $                   EQUED, R, C, B, LDB, X, LDX, RCOND, FERR, BERR,
     $                   WORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, TRANS
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   BERR( * ), C( * ), FERR( * ), R( * ),
     $                   RWORK( * )
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            COLEQU, EQUIL, NOFACT, NOTRAN, ROWEQU
      CHARACTER          NORM
      INTEGER            I, INFEQU, J
      DOUBLE PRECISION   AMAX, ANORM, BIGNUM, COLCND, RCMAX, RCMIN,
     $                   ROWCND, RPVGRW, SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, ZLANGE, ZLANTR
      EXTERNAL           LSAME, DLAMCH, ZLANGE, ZLANTR
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGECON, ZGEEQU, ZGERFS, ZGETRF, ZGETRS,
     $                   ZLACPY, ZLAQGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
```

```
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      NOTRAN = LSAME( TRANS, 'N' )
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         ROWEQU = .FALSE.
         COLEQU = .FALSE.
      ELSE
         ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
         COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         SMLNUM = DLAMCH( 'Safe minimum' )
         BIGNUM = ONE / SMLNUM
      END IF
*
*     Test the input parameters.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.LSAME( FACT, 'F' ) )
     $     THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $         LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $         ( ROWEQU .OR. COLEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -10
      ELSE
         IF( ROWEQU ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 10 J = 1, N
               RCMIN = MIN( RCMIN, R( J ) )
               RCMAX = MAX( RCMAX, R( J ) )
   10       CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -11
            ELSE IF( N.GT.0 ) THEN
               ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               ROWCND = ONE
            END IF
         END IF
         IF( COLEQU .AND. INFO.EQ.0 ) THEN
```

```
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 20 J = 1, N
               RCMIN = MIN( RCMIN, C( J ) )
               RCMAX = MAX( RCMAX, C( J ) )
   20       CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -12
            ELSE IF( N.GT.0 ) THEN
               COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               COLCND = ONE
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -14
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -16
            END IF
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGESVX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*        Compute row and column scalings to equilibrate the matrix A.
*
         CALL ZGEEQU( N, N, A, LDA, R, C, ROWCND, COLCND, AMAX, INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*           Equilibrate the matrix.
*
            CALL ZLAQGE( N, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $                   EQUED )
            ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
            COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         END IF
      END IF
*
*     Scale the right hand side.
*
      IF( NOTRAN ) THEN
         IF( ROWEQU ) THEN
            DO 40 J = 1, NRHS
               DO 30 I = 1, N
```

```
                B( I, J ) = R( I )*B( I, J )
   30         CONTINUE
   40      CONTINUE
         END IF
      ELSE IF( COLEQU ) THEN
         DO 60 J = 1, NRHS
            DO 50 I = 1, N
               B( I, J ) = C( I )*B( I, J )
   50       CONTINUE
   60    CONTINUE
      END IF
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the LU factorization of A.
*
         CALL ZLACPY( 'Full', N, N, A, LDA, AF, LDAF )
         CALL ZGETRF( N, N, AF, LDAF, IPIV, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 ) THEN
*
*           Compute the reciprocal pivot growth factor of the
*           leading rank-deficient INFO columns of A.
*
            RPVGRW = ZLANTR( 'M', 'U', 'N', INFO, INFO, AF, LDAF,
     $               RWORK )
            IF( RPVGRW.EQ.ZERO ) THEN
               RPVGRW = ONE
            ELSE
               RPVGRW = ZLANGE( 'M', N, INFO, A, LDA, RWORK ) /
     $                  RPVGRW
            END IF
            RWORK( 1 ) = RPVGRW
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A and the
*     reciprocal pivot growth factor RPVGRW.
*
      IF( NOTRAN ) THEN
         NORM = '1'
      ELSE
         NORM = 'I'
      END IF
      ANORM = ZLANGE( NORM, N, N, A, LDA, RWORK )
      RPVGRW = ZLANTR( 'M', 'U', 'N', N, N, AF, LDAF, RWORK )
```

```
      IF( RPVGRW.EQ.ZERO ) THEN
         RPVGRW = ONE
      ELSE
         RPVGRW = ZLANGE( 'M', N, N, A, LDA, RWORK ) / RPVGRW
      END IF
*
*     Compute the reciprocal of the condition number of A.
*
      CALL ZGECON( NORM, N, AF, LDAF, ANORM, RCOND, WORK, RWORK, INFO )
*
*     Compute the solution matrix X.
*
      CALL ZLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL ZGETRS( TRANS, N, NRHS, AF, LDAF, IPIV, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL ZGERFS( TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB, X,
     $             LDX, FERR, BERR, WORK, RWORK, INFO )
*
*     Transform the solution matrix X to a solution of the original
*     system.
*
      IF( NOTRAN ) THEN
         IF( COLEQU ) THEN
            DO 80 J = 1, NRHS
               DO 70 I = 1, N
                  X( I, J ) = C( I )*X( I, J )
   70          CONTINUE
   80       CONTINUE
            DO 90 J = 1, NRHS
               FERR( J ) = FERR( J ) / COLCND
   90       CONTINUE
         END IF
      ELSE IF( ROWEQU ) THEN
         DO 110 J = 1, NRHS
            DO 100 I = 1, N
               X( I, J ) = R( I )*X( I, J )
  100       CONTINUE
  110    CONTINUE
         DO 120 J = 1, NRHS
            FERR( J ) = FERR( J ) / ROWCND
  120    CONTINUE
      END IF
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.DLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
```

```
*
      RWORK( 1 ) = RPVGRW
      RETURN
*
*     End of ZGESVX
*
      END
```

———-

23.8.4 lisp code

— **zgesvx.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun zgesvx
         (fact trans n nrhs a lda af ldaf ipiv equed r c b ldb$ x ldx rcond
          ferr berr work rwork info)
    (declare (type (double-float) rcond)
             (type (array double-float (*)) rwork berr ferr c r)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) work x b af a)
             (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
             (type (string *) equed trans fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (trans character trans-%data% trans-%offset%)
         (equed character equed-%data% equed-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (af f2cl-lib:complex16 af-%data% af-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (x f2cl-lib:complex16 x-%data% x-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (r double-float r-%data% r-%offset%)
         (c double-float c-%data% c-%offset%)
         (ferr double-float ferr-%data% ferr-%offset%)
         (berr double-float berr-%data% berr-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (prog ((amax 0.0d0) (anorm 0.0d0) (bignum 0.0d0) (colcnd 0.0d0)
             (rcmax 0.0d0) (rcmin 0.0d0) (rowcnd 0.0d0) (rpvgrw 0.0d0)
             (smlnum 0.0d0) (i 0) (infequ 0) (j 0)
             (norm
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (colequ nil) (equil nil) (nofact nil) (notran nil) (rowequ nil))
```

```
(declare
 (type (double-float) amax anorm bignum colcnd rcmax rcmin rowcnd
  rpvgrw smlnum)
 (type (f2cl-lib:integer4) i infequ j)
 (type (string 1) norm)
 (type f2cl-lib:logical colequ equil nofact notran rowequ))
(setf info 0)
(setf nofact
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "N")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val))
(setf equil
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "E")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val))
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
  (setf rowequ f2cl-lib:%false%) (setf colequ f2cl-lib:%false%))
 (t
  (setf rowequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "R")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)))
  (setf colequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "C")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
```

```
                (declare (ignore var-1))
                (when var-0 (setf equed var-0))
                ret-val)))
  (setf smlnum (dlamch "Safe minimum")) (setf bignum (/ one smlnum))))
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame fact "F")
     (declare (ignore var-1))
     (when var-0 (setf fact var-0))
     ret-val)
   (not
    (or rowequ colequ
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame equed "N")
          (declare (ignore var-1))
          (when var-0 (setf equed var-0))
          ret-val))))
  (setf info -10))
 (t
  (cond
   (rowequ (setf rcmin bignum) (setf rcmax zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                    ((> j n) nil)
          (tagbody
            (setf rcmin
                    (min rcmin
                         (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
            (setf rcmax
                    (max rcmax
                         (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
           label10))
        (cond ((<= rcmin zero) (setf info -11))
              ((> n 0)
               (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum))))
              (t (setf rowcnd one)))))
      (cond
       ((and colequ (= info 0)) (setf rcmin bignum) (setf rcmax zero)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf rcmin
                    (min rcmin
                         (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
            (setf rcmax
                    (max rcmax
                         (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
           label20))
        (cond ((<= rcmin zero) (setf info -12))
              ((> n 0)
               (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum))))
              (t (setf colcnd one)))))
      (cond
       ((= info 0)
        (cond
         ((< ldb$
             (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -14))
         ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -16)))))))
    (cond
     ((/= info 0) (xerbla "ZGESVX" (f2cl-lib:int-sub info))
      (go end_label)))
    (cond
     (equil
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (zgeequ n n a lda r c rowcnd colcnd amax infequ)
        (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5))
        (setf rowcnd var-6)
        (setf colcnd var-7)
        (setf amax var-8)
        (setf infequ var-9))
```

```
  (cond
   ((= infequ 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (zlaqge n n a lda r c rowcnd colcnd amax equed)
      (declare (ignore var-2 var-4 var-5))
      (when var-0 (setf n var-0))
      (when var-1 (setf n var-1))
      (when var-3 (setf lda var-3))
      (when var-6 (setf rowcnd var-6))
      (when var-7 (setf colcnd var-7))
      (when var-8 (setf amax var-8))
      (when var-9 (setf equed var-9)))
    (setf rowequ
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "R")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "B")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)))
    (setf colequ
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "C")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "B")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)))))))
(cond
 (notran
  (cond
   (rowequ
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
```

```
                        (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)))
            label30))
        label40)))))
  (colequ
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j nrhs) nil)
     (tagbody
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                   (* (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)
                      (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                     b-%offset%)))
         label50))
     label60))))
 (cond
  ((or nofact equil)
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (zlacpy "Full" n n a lda af ldaf)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf n var-1))
     (when var-2 (setf n var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf ldaf var-6)))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
       (zgetrf n n af ldaf ipiv info)
     (declare (ignore var-2 var-4))
     (setf n var-0)
     (setf n var-1)
     (setf ldaf var-3)
     (setf info var-5))
   (cond
    ((> info 0)
     (setf rpvgrw
             (multiple-value-bind
                 (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6
                  var-7)
                 (zlantr "M" "U" "N" info info af ldaf rwork)
               (declare (ignore var-0 var-1 var-2 var-5 var-7))
               (when var-3 (setf info var-3))
               (when var-4 (setf info var-4))
               (when var-6 (setf ldaf var-6))
               ret-val))
     (cond ((= rpvgrw zero) (setf rpvgrw one))
           (t
            (setf rpvgrw
                    (/
```

```
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                          (zlange "M" n info a lda rwork)
                        (declare (ignore var-0 var-3 var-5))
                        (when var-1 (setf n var-1))
                        (when var-2 (setf info var-2))
                        (when var-4 (setf lda var-4))
                        ret-val)
                      rpvgrw))))
    (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
            rpvgrw)
    (setf rcond zero) (go end_label)))))
(cond (notran (f2cl-lib:f2cl-set-string norm "1" (string 1)))
      (t (f2cl-lib:f2cl-set-string norm "I" (string 1))))
(setf anorm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange norm n n a lda rwork)
          (declare (ignore var-3 var-5))
          (when var-0 (setf norm var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf rpvgrw
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (zlantr "M" "U" "N" n n af ldaf rwork)
          (declare (ignore var-0 var-1 var-2 var-5 var-7))
          (when var-3 (setf n var-3))
          (when var-4 (setf n var-4))
          (when var-6 (setf ldaf var-6))
          ret-val))
(cond ((= rpvgrw zero) (setf rpvgrw one))
      (t
       (setf rpvgrw
               (/
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                    (zlange "M" n n a lda rwork)
                  (declare (ignore var-0 var-3 var-5))
                  (when var-1 (setf n var-1))
                  (when var-2 (setf n var-2))
                  (when var-4 (setf lda var-4))
                  ret-val)
                rpvgrw))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zgecon norm n af ldaf anorm rcond work rwork info)
  (declare (ignore var-2 var-4 var-6 var-7))
```

```
  (setf norm var-0)
  (setf n var-1)
  (setf ldaf var-3)
  (setf rcond var-5)
  (setf info var-8))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "Full" n nrhs b ldb$ x ldx)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf ldx var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zgetrs trans n nrhs af ldaf ipiv x ldx info)
  (declare (ignore var-3 var-5 var-6))
  (setf trans var-0)
  (setf n var-1)
  (setf nrhs var-2)
  (setf ldaf var-4)
  (setf ldx var-7)
  (setf info var-8))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16)
    (zgerfs trans n nrhs a lda af ldaf ipiv b ldb$ x ldx ferr berr work
     rwork info)
  (declare
   (ignore var-2 var-3 var-5 var-7 var-8 var-9 var-10 var-11 var-12
    var-13 var-14 var-15))
  (setf trans var-0)
  (setf n var-1)
  (setf lda var-4)
  (setf ldaf var-6)
  (setf info var-16))
(cond
 (notran
  (cond
   (colequ
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                 x-%offset%)
                  (* (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)
                     (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                    x-%offset%)))
```

```
                 label70))
              label80))
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j nrhs) nil)
              (tagbody
                (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                        (/
                         (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                         colcnd))
              label90)))))
          (rowequ
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j nrhs) nil)
             (tagbody
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i n) nil)
                 (tagbody
                   (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                        x-%offset%)
                         (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                            (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                           x-%offset%)))
                label100))
             label110))
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j nrhs) nil)
             (tagbody
               (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                       (/ (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                          rowcnd))
             label120))))
         (if (< rcond (dlamch "Epsilon"))
             (setf info (f2cl-lib:int-add n 1)))
         (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%) rpvgrw)
         (go end_label)
        end_label
         (return
          (values fact trans n nrhs nil lda nil ldaf nil equed nil nil nil ldb$
                  nil ldx rcond nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgesvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
```

```
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (string) (array double-float (*))
  (array double-float (*))
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (double-float)
  (array double-float (*))
  (array double-float (*))
  (array fortran-to-lisp::complex16
   (*))
  (array double-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::fact
  fortran-to-lisp::trans
  fortran-to-lisp::n
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldaf nil
  fortran-to-lisp::equed nil nil
  nil fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond nil nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zgetrs
  fortran-to-lisp::zgetrf
  fortran-to-lisp::zgerfs
  fortran-to-lisp::zgecon
  fortran-to-lisp::zgeequ
  fortran-to-lisp::xerbla))))
```

———-

23.9 zgesvxx

23.9.1 Axiom unit tests

— **zgesvxx.input** —

```
)set break resume
)sys rm -f zgesvxx.output
)spool zgesvxx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

23.9.2 Axiom help page [Demm05]

— **zgesvxx.help** —

```
====================================================================
zgesvxx examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgesvxx (FACT, TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV,
           EQUED, R, C, B, LDB, X, LDX, RCOND, RPVGRW, BERR, N_ERR_BNDS,
           ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, RWORK, INFO)
            ZGESVXX computes the solution to system of linear equations A * X
           = B for GE matrices

Function/Subroutine Documentation
   subroutine zgesvxx (character FACT, character TRANS, integer N, integer
       NRHS, complex*16, dimension( lda, * ) A, integer LDA, complex*16,
       dimension( ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV,
       character EQUED, double precision, dimension( * ) R, double precision,
       dimension( * ) C, complex*16, dimension( ldb, * ) B, integer LDB,
       complex*16, dimension( ldx , * ) X, integer LDX, double precision
       RCOND, double precision RPVGRW, double precision, dimension( * ) BERR,
```

```
      integer N_ERR_BNDS, double precision, dimension( nrhs, * )
      ERR_BNDS_NORM, double precision, dimension( nrhs, * ) ERR_BNDS_COMP,
      integer NPARAMS, double precision, dimension( * ) PARAMS, complex*16,
      dimension( * ) WORK, double precision, dimension( * ) RWORK, integer
      INFO)
       ZGESVXX computes the solution to system of linear equations A * X = B
      for GE matrices

Purpose:
   ZGESVXX uses the LU factorization to compute the solution to a
   complex*16 system of linear equations  A * X = B,  where A is an
   N-by-N matrix and X and B are N-by-NRHS matrices.

   If requested, both normwise and maximum componentwise error bounds
   are returned. ZGESVXX will return a solution with a tiny
   guaranteed error (O(eps) where eps is the working machine
   precision) unless the matrix is very ill-conditioned, in which
   case a warning is returned. Relevant condition numbers also are
   calculated and returned.

   ZGESVXX accepts user-provided factorizations and equilibration
   factors; see the definitions of the FACT and EQUED options.
   Solving with refinement and using a factorization from a previous
   ZGESVXX call will also produce a solution with either O(eps)
   errors or warnings, but we cannot make that claim for general
   user-provided factorizations and equilibration factors if they
   differ from what ZGESVXX would itself produce.

Description:
   The following steps are performed:

   1. If FACT = 'E', double precision scaling factors are computed to
      equilibrate the system:
   TRANS = 'N':  diag(R)*A*diag(C)     *inv(diag(C))*X = diag(R)*B
   TRANS = 'T': (diag(R)*A*diag(C))**T *inv(diag(R))*X = diag(C)*B
   TRANS = 'C': (diag(R)*A*diag(C))**H *inv(diag(R))*X = diag(C)*B

   Whether or not the system will be equilibrated depends on the
   scaling of the matrix A, but if equilibration is used, A is
   overwritten by diag(R)*A*diag(C) and B by diag(R)*B (if TRANS='N')
   or diag(C)*B (if TRANS = 'T' or 'C').

   2. If FACT = 'N' or 'E', the LU decomposition is used to factor
   the matrix A (after equilibration if FACT = 'E') as
           A = P * L * U,
   where P is a permutation matrix, L is a unit lower triangular
   matrix, and U is upper triangular.

   3. If some U(i,i)=0, so that U is exactly singular, then the
   routine returns with INFO = i. Otherwise, the factored form of A
```

```
   is used to estimate the condition number of the matrix A (see
   argument RCOND). If the reciprocal of the condition number is less
   than machine precision, the routine still goes on to solve for X
   and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. By default (unless PARAMS(LA_LINRX_ITREF_I) is set to zero),
   the routine will use iterative refinement to try to get a small
   error and error bounds.  Refinement calculates the residual to at
   least twice the working precision.

   6. If equilibration was used, the matrix X is premultiplied by
   diag(C) (if TRANS = 'N') or diag(R) (if TRANS = 'T' or 'C') so
   that it solves the original system before equilibration.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AF and IPIV contain the factored form of A.
           If EQUED is not 'N', the matrix A has been
           equilibrated with scaling factors given by R and C.
           A, AF, and IPIV are not modified.
   = 'N':  The matrix A will be copied to AF and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
           copied to AF and factored.

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate Transpose)

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 NRHS
```

```
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the N-by-N matrix A.  If FACT = 'F' and EQUED is
  not 'N', then A must have been equilibrated by the scaling
  factors in R and/or C.  A is not modified if FACT = 'F' or
  'N', or if FACT = 'E' and EQUED = 'N' on exit.

  On exit, if EQUED .ne. 'N', A is scaled as follows:
  EQUED = 'R':  A := diag(R) * A
  EQUED = 'C':  A := A * diag(C)
  EQUED = 'B':  A := diag(R) * A * diag(C).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX*16 array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the factors L and U from the factorization
  A = P*L*U as computed by ZGETRF.  If EQUED .ne. 'N', then
  AF is the factored form of the equilibrated matrix A.

  If FACT = 'N', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the equilibrated matrix A (see the description of A for
  the form of the equilibrated matrix).

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains the pivot indices from the factorization A = P*L*U
  as computed by ZGETRF; row i of the matrix was interchanged
  with row IPIV(i).

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the original matrix A.
```

```
  If FACT = 'E', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the equilibrated matrix A.

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
  = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
  = 'B':  Both row and column equilibration, i.e., A has been
          replaced by diag(R) * A * diag(C).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

R
  R is DOUBLE PRECISION array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
  EQUED = 'R' or 'B', each element of R must be positive.
  If R is output, each element of R is a power of the radix.
  If R is input, each element of R should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

C
  C is DOUBLE PRECISION array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.
  If C is output, each element of C is a power of the radix.
  If C is input, each element of C should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit,
```

```
  if EQUED = 'N', B is not modified;
  if TRANS = 'N' and EQUED = 'R' or 'B', B is overwritten by diag(R)*B;
  if TRANS = 'T' or 'C' and EQUED = 'C' or 'B', B is overwritten by diag(C)*B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  If INFO = 0, the N-by-NRHS solution matrix X to the original
  system of equations.  Note that A and B are modified on exit
  if EQUED .ne. 'N', and the solution to the equilibrated system is
  inv(diag(C))*X if TRANS = 'N' and EQUED = 'C' or 'B', or
  inv(diag(R))*X if TRANS = 'T' or 'C' and EQUED = 'R' or 'B'.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

RPVGRW
  RPVGRW is DOUBLE PRECISION
  Reciprocal pivot growth.  On exit, this contains the reciprocal
  pivot growth factor norm(A)/norm(U). The "max absolute element"
  norm is used.  If this is much less than 1, then the stability of
  the LU factorization of the (equilibrated) matrix A could be poor.
  This also means that the solution X, estimated condition numbers,
  and error bounds could be unreliable. If factorization fails with
  0<INFO<=N, then this contains the reciprocal pivot growth factor
  for the leading INFO columns of A.  In ZGESVX, this quantity is
  returned in WORK(1).

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
```

```
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
            max_j (abs(XTRUE(j,i) - X(j,i)))
            ------------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
          reciprocal condition number is less than the threshold
          sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
          almost certainly within a factor of 10 of the true error
          so long as the next entry is greater than the threshold
          sqrt(n) * dlamch('Epsilon'). This error bound should only
          be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
           reciprocal condition number.  Compared with the threshold
           sqrt(n) * dlamch('Epsilon') to determine if the error
           estimate is "guaranteed". These reciprocal condition
           numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
           appropriately scaled matrix Z.
           Let Z = S*A, where S scales each row by a power of the
           radix so all absolute row sums of Z are approximately 1.

ERR_BNDS_COMP
  ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
```

```
                    abs(XTRUE(j,i) - X(j,i))
            max_j ----------------------
                          abs(X(j,i))
```

The array is indexed by the right-hand side i (on which the componentwise relative error depends), and the type of error information as described below. There currently are up to three pieces of information returned for each right-hand side. If componentwise accuracy is not requested (PARAMS(3) = 0.0), then ERR_BNDS_COMP is not accessed. If N_ERR_BNDS .LT. 3, then at most the first (:,N_ERR_BNDS) entries are returned.

The first index in ERR_BNDS_COMP(i,:) corresponds to the ith right-hand side.

The second index in ERR_BNDS_COMP(:,err) contains the following three fields:

```
err = 1 "Trust/don't trust" boolean. Trust the answer if the
         reciprocal condition number is less than the threshold
         sqrt(n) * dlamch('Epsilon').

err = 2 "Guaranteed" error bound: The estimated forward error,
         almost certainly within a factor of 10 of the true error
         so long as the next entry is greater than the threshold
         sqrt(n) * dlamch('Epsilon'). This error bound should only
         be trusted if the previous boolean is true.

err = 3  Reciprocal condition number: Estimated componentwise
         reciprocal condition number.  Compared with the threshold
         sqrt(n) * dlamch('Epsilon') to determine if the error
         estimate is "guaranteed". These reciprocal condition
         numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
         appropriately scaled matrix Z.
         Let Z = S*(A*diag(x)), where x is the solution for the
         current right-hand side and S scales each row of
         A*diag(x) by a power of the radix so all absolute row
         sums of Z are approximately 1.
```

NPARAMS

NPARAMS is INTEGER

Specifies the number of parameters set in PARAMS. If .LE. 0, the PARAMS array is never referenced and default values are used.

PARAMS

PARAMS is DOUBLE PRECISION array, dimension NPARAMS

Specifies algorithm parameters. If an entry is .LT. 0.0, then that entry will be filled with default value used for that parameter. Only positions up to NPARAMS are accessed; defaults are used for higher-numbered parameters.

```
  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0D+0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the extra-precise refinement algorithm.
          (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be
              trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is COMPLEX*16 array, dimension (2*N)

RWORK
  RWORK is DOUBLE PRECISION array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is guaranteed.
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
      has been completed, but the factor U is exactly singular, so
      the solution and error bounds could not be computed. RCOND = 0
      is returned.
  = N+J: The solution corresponding to the Jth right-hand side is
         not guaranteed. The solutions corresponding to other right-
         hand sides K with K > J may not be guaranteed as well, but
         only the first such right-hand side is reported. If a small
         componentwise error is not requested (PARAMS(3) = 0.0) then
         the Jth right-hand side is the first with a normwise error
         bound that is not guaranteed (the smallest J such
         that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
         the Jth right-hand side is the first with either a normwise or
         componentwise error bound that is not guaranteed (the smallest
         J such that either ERR_BNDS_NORM(J,1) = 0.0 or
         ERR_BNDS_COMP(J,1) = 0.0). See the definition of
         ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
```

```
about all of the right-hand sides check ERR_BNDS_NORM or
ERR_BNDS_COMP.
```

———-

23.9.3 fortran code

— **zgesvxx.f** —

```
*  ==================================================================
      SUBROUTINE ZGESVXX( FACT, TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $                    EQUED, R, C, B, LDB, X, LDX, RCOND, RPVGRW,
     $                    BERR, N_ERR_BNDS, ERR_BNDS_NORM,
     $                    ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, RWORK,
     $                    INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, TRANS
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      DOUBLE PRECISION   RCOND, RPVGRW
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX , * ),WORK( * )
      DOUBLE PRECISION   R( * ), C( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * ), RWORK( * )
*     ..
*
*  ================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
```

```
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
*     ..
*     .. Local Scalars ..
      LOGICAL            COLEQU, EQUIL, NOFACT, NOTRAN, ROWEQU
      INTEGER            INFEQU, J
      DOUBLE PRECISION   AMAX, BIGNUM, COLCND, RCMAX, RCMIN,
     $                   ROWCND, SMLNUM
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, DLAMCH, ZLA_GERPVGRW
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, ZLA_GERPVGRW
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZGEEQUB, ZGETRF, ZGETRS, ZLACPY, ZLAQGE,
     $                   XERBLA, ZLASCL2, ZGERFSX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      NOTRAN = LSAME( TRANS, 'N' )
      SMLNUM = DLAMCH( 'Safe minimum' )
      BIGNUM = ONE / SMLNUM
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         ROWEQU = .FALSE.
         COLEQU = .FALSE.
      ELSE
         ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
         COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
      END IF
*
*     Default is failure.  If an input parameter is wrong or
*     factorization fails, make everything look horrible.  Only the
*     pivot growth is set here, the rest is initialized in ZGERFSX.
*
      RPVGRW = ZERO
*
*     Test the input parameters.  PARAMS is not tested until ZGERFSX.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.
     $     LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
```

```
     $        LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $        ( ROWEQU .OR. COLEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -10
      ELSE
         IF( ROWEQU ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 10 J = 1, N
               RCMIN = MIN( RCMIN, R( J ) )
               RCMAX = MAX( RCMAX, R( J ) )
10          CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -11
            ELSE IF( N.GT.0 ) THEN
               ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               ROWCND = ONE
            END IF
         END IF
         IF( COLEQU .AND. INFO.EQ.0 ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 20 J = 1, N
               RCMIN = MIN( RCMIN, C( J ) )
               RCMAX = MAX( RCMAX, C( J ) )
20          CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -12
            ELSE IF( N.GT.0 ) THEN
               COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               COLCND = ONE
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -14
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -16
            END IF
```

```
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGESVXX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*     Compute row and column scalings to equilibrate the matrix A.
*
         CALL ZGEEQUB( N, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $        INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*     Equilibrate the matrix.
*
            CALL ZLAQGE( N, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $           EQUED )
            ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
            COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         END IF
*
*     If the scaling factors are not applied, set them to 1.0.
*
         IF ( .NOT.ROWEQU ) THEN
            DO J = 1, N
               R( J ) = 1.0D+0
            END DO
         END IF
         IF ( .NOT.COLEQU ) THEN
            DO J = 1, N
               C( J ) = 1.0D+0
            END DO
         END IF
      END IF
*
*     Scale the right-hand side.
*
      IF( NOTRAN ) THEN
         IF( ROWEQU ) CALL ZLASCL2( N, NRHS, R, B, LDB )
      ELSE
         IF( COLEQU ) CALL ZLASCL2( N, NRHS, C, B, LDB )
      END IF
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the LU factorization of A.
*
```

```
         CALL ZLACPY( 'Full', N, N, A, LDA, AF, LDAF )
         CALL ZGETRF( N, N, AF, LDAF, IPIV, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 ) THEN
*
*           Pivot in column INFO is exactly 0
*           Compute the reciprocal pivot growth factor of the
*           leading rank-deficient INFO columns of A.
*
            RPVGRW = ZLA_GERPVGRW( N, INFO, A, LDA, AF, LDAF )
            RETURN
         END IF
      END IF
*
*     Compute the reciprocal pivot growth factor RPVGRW.
*
      RPVGRW = ZLA_GERPVGRW( N, N, A, LDA, AF, LDAF )
*
*     Compute the solution matrix X.
*
      CALL ZLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL ZGETRS( TRANS, N, NRHS, AF, LDAF, IPIV, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL ZGERFSX( TRANS, EQUED, N, NRHS, A, LDA, AF, LDAF,
     $     IPIV, R, C, B, LDB, X, LDX, RCOND, BERR,
     $     N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $     WORK, RWORK, INFO )
*
*     Scale solutions.
*
      IF ( COLEQU .AND. NOTRAN ) THEN
         CALL ZLASCL2 ( N, NRHS, C, X, LDX )
      ELSE IF ( ROWEQU .AND. .NOT.NOTRAN ) THEN
         CALL ZLASCL2 ( N, NRHS, R, X, LDX )
      END IF
*
      RETURN
*
*     End of ZGESVXX
*
      END
```

———-

23.9.4 lisp code

— zgesvxx.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun zgesvxx
         (fact trans n nrhs a lda af ldaf ipiv equed r c b ldb$ x ldx rcond
          rpvgrw berr n_err_bnds err_bnds_norm err_bnds_comp nparams params
          work rwork info)
    (declare (type (double-float) rpvgrw rcond)
             (type (array double-float (*)) rwork params err_bnds_comp
              err_bnds_norm berr c r)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) work x b af a)
             (type (f2cl-lib:integer4) info nparams n_err_bnds ldx ldb$ ldaf
              lda nrhs n)
             (type (string *) equed trans fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (trans character trans-%data% trans-%offset%)
         (equed character equed-%data% equed-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (af f2cl-lib:complex16 af-%data% af-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (x f2cl-lib:complex16 x-%data% x-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (r double-float r-%data% r-%offset%)
         (c double-float c-%data% c-%offset%)
         (berr double-float berr-%data% berr-%offset%)
         (err_bnds_norm double-float err_bnds_norm-%data%
          err_bnds_norm-%offset%)
         (err_bnds_comp double-float err_bnds_comp-%data%
          err_bnds_comp-%offset%)
         (params double-float params-%data% params-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (prog ((amax 0.0d0) (bignum 0.0d0) (colcnd 0.0d0) (rcmax 0.0d0)
             (rcmin 0.0d0) (rowcnd 0.0d0) (smlnum 0.0d0) (infequ 0) (j 0)
             (colequ nil) (equil nil) (nofact nil) (notran nil) (rowequ nil))
        (declare
         (type (double-float) amax bignum colcnd rcmax rcmin rowcnd smlnum)
         (type (f2cl-lib:integer4) infequ j)
         (type f2cl-lib:logical colequ equil nofact notran rowequ))
        (setf info 0)
        (setf nofact
                (multiple-value-bind (ret-val var-0 var-1)
```

```
            (lsame fact "N")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val))
(setf equil
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "E")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val))
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(setf smlnum (dlamch "Safe minimum"))
(setf bignum (/ one smlnum))
(cond
 ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
  (setf rowequ f2cl-lib:%false%) (setf colequ f2cl-lib:%false%))
 (t
  (setf rowequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "R")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)))
  (setf colequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "C")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)))))
(setf rpvgrw zero)
(cond
 ((and (not nofact) (not equil)
       (not
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
 (setf info -1))
((and (not notran)
      (not
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame trans "T")
         (declare (ignore var-1))
         (when var-0 (setf trans var-0))
         ret-val))
      (not
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame trans "C")
         (declare (ignore var-1))
         (when var-0 (setf trans var-0))
         ret-val)))
 (setf info -2))
((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
 (setf info -6))
((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
 (setf info -8))
((and
  (multiple-value-bind (ret-val var-0 var-1)
      (lsame fact "F")
    (declare (ignore var-1))
    (when var-0 (setf fact var-0))
    ret-val)
  (not
   (or rowequ colequ
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame equed "N")
         (declare (ignore var-1))
         (when var-0 (setf equed var-0))
         ret-val))))
 (setf info -10))
(t
 (cond
  (rowequ (setf rcmin bignum) (setf rcmax zero)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf rcmin
               (min rcmin
                    (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
       (setf rcmax
               (max rcmax
```

```
                        (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
          label10))
       (cond ((<= rcmin zero) (setf info -11))
             ((> n 0)
              (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum))))
             (t (setf rowcnd one)))))
     (cond
      ((and colequ (= info 0)) (setf rcmin bignum) (setf rcmax zero)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (setf rcmin
                   (min rcmin
                        (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
           (setf rcmax
                   (max rcmax
                        (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
          label20))
       (cond ((<= rcmin zero) (setf info -12))
             ((> n 0)
              (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum))))
             (t (setf colcnd one)))))
     (cond
      ((= info 0)
       (cond
        ((< ldb$
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -14))
        ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -16)))))))
   (cond
    ((/= info 0) (xerbla "ZGESVXX" (f2cl-lib:int-sub info))
     (go end_label)))
   (cond
    (equil
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (zgeequb n n a lda r c rowcnd colcnd amax infequ)
       (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5))
       (setf rowcnd var-6)
       (setf colcnd var-7)
       (setf amax var-8)
       (setf infequ var-9))
     (cond
      ((= infequ 0)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
           (zlaqge n n a lda r c rowcnd colcnd amax equed)
         (declare (ignore var-2 var-4 var-5))
         (when var-0 (setf n var-0))
```

```
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-6 (setf rowcnd var-6))
          (when var-7 (setf colcnd var-7))
          (when var-8 (setf amax var-8))
          (when var-9 (setf equed var-9)))
        (setf rowequu
                (or
                 (multiple-value-bind (ret-val var-0 var-1)
                     (lsame equed "R")
                   (declare (ignore var-1))
                   (when var-0 (setf equed var-0))
                   ret-val)
                 (multiple-value-bind (ret-val var-0 var-1)
                     (lsame equed "B")
                   (declare (ignore var-1))
                   (when var-0 (setf equed var-0))
                   ret-val)))
        (setf colequ
                (or
                 (multiple-value-bind (ret-val var-0 var-1)
                     (lsame equed "C")
                   (declare (ignore var-1))
                   (when var-0 (setf equed var-0))
                   ret-val)
                 (multiple-value-bind (ret-val var-0 var-1)
                     (lsame equed "B")
                   (declare (ignore var-1))
                   (when var-0 (setf equed var-0))
                   ret-val)))))
      (cond
       ((not rowequu)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%) 1.0d0)
           label100000))))
      (cond
       ((not colequ)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) 1.0d0)
           label100001))))))
    (cond
     (notran
      (if rowequu
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (zlascl2 n nrhs r b ldb$)
            (declare (ignore var-2 var-3))
```

```
            (when var-0 (setf n var-0))
            (when var-1 (setf nrhs var-1))
            (when var-4 (setf ldb$ var-4)))))
     (t
      (if colequ
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (zlascl2 n nrhs c b ldb$)
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf nrhs var-1))
            (when var-4 (setf ldb$ var-4))))))
    (cond
     ((or nofact equil)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlacpy "Full" n n a lda af ldaf)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldaf var-6)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (zgetrf n n af ldaf ipiv info)
        (declare (ignore var-2 var-4))
        (setf n var-0)
        (setf n var-1)
        (setf ldaf var-3)
        (setf info var-5))
      (cond
       ((> info 0)
        (setf rpvgrw
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                    (zla_gerpvgrw n info a lda af ldaf)
                  (declare (ignore var-2 var-4))
                  (when var-0 (setf n var-0))
                  (when var-1 (setf info var-1))
                  (when var-3 (setf lda var-3))
                  (when var-5 (setf ldaf var-5))
                  ret-val))
        (go end_label)))))
    (setf rpvgrw
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                (zla_gerpvgrw n n a lda af ldaf)
              (declare (ignore var-2 var-4))
              (when var-0 (setf n var-0))
              (when var-1 (setf n var-1))
              (when var-3 (setf lda var-3))
              (when var-5 (setf ldaf var-5))
              ret-val))
```

```
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "Full" n nrhs b ldb$ x ldx)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf ldx var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zgetrs trans n nrhs af ldaf ipiv x ldx info)
  (declare (ignore var-3 var-5 var-6))
  (setf trans var-0)
  (setf n var-1)
  (setf nrhs var-2)
  (setf ldaf var-4)
  (setf ldx var-7)
  (setf info var-8))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19
     var-20 var-21 var-22 var-23 var-24)
    (zgerfsx trans equed n nrhs a lda af ldaf ipiv r c b ldb$ x ldx
     rcond berr n_err_bnds err_bnds_norm err_bnds_comp nparams params
     work rwork info)
  (declare
   (ignore var-4 var-6 var-8 var-9 var-10 var-11 var-13 var-16 var-18
    var-19 var-21 var-22 var-23))
  (when var-0 (setf trans var-0))
  (when var-1 (setf equed var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf nrhs var-3))
  (when var-5 (setf lda var-5))
  (when var-7 (setf ldaf var-7))
  (when var-12 (setf ldb$ var-12))
  (when var-14 (setf ldx var-14))
  (when var-15 (setf rcond var-15))
  (when var-17 (setf n_err_bnds var-17))
  (when var-20 (setf nparams var-20))
  (when var-24 (setf info var-24)))
(cond
 ((and colequ notran)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (zlascl2 n nrhs c x ldx)
    (declare (ignore var-2 var-3))
    (when var-0 (setf n var-0))
    (when var-1 (setf nrhs var-1))
    (when var-4 (setf ldx var-4))))
 ((and rowequ (not notran))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (zlascl2 n nrhs r x ldx)
```

```
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf nrhs var-1))
            (when var-4 (setf ldx var-4)))))
        (go end_label)
       end_label
        (return
         (values fact trans n nrhs nil lda nil ldaf nil equed nil nil nil ldb$
                 nil ldx rcond rpvgrw nil n_err_bnds nil nil nparams nil nil
                 nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgesvxx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (string) (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::fact
                                              fortran-to-lisp::trans
```

```
  fortran-to-lisp::n
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldaf nil
  fortran-to-lisp::equed nil nil
  nil fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond
  fortran-to-lisp::rpvgrw nil
  fortran-to-lisp::n_err_bnds nil
  nil fortran-to-lisp::nparams nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zgetrs
  fortran-to-lisp::zgetrf
  fortran-to-lisp::zgeequb
  fortran-to-lisp::xerbla))))
```

Chapter 24

LAPACK - General Matrices, Eigenvalue , Double

24.1 dgees

24.1.1 Axiom unit tests

— dgees.input —

```
)set break resume
)sys rm -f dgees.output
)spool dgees.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

24.1.2 Axiom help page

— dgees.help —

```
======================================================================
dgees examples
======================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgees (JOBVS, SORT, SELECT, N, A, LDA, SDIM, WR, WI, VS,
           LDVS, WORK, LWORK, BWORK, INFO)
            DGEES computes the eigenvalues, the Schur form, and, optionally,
           the matrix of Schur vectors for GE matrices

Function/Subroutine Documentation
   subroutine dgees (character JOBVS, character SORT, external SELECT, integer
       N, double precision, dimension( lda, * ) A, integer LDA, integer SDIM,
       double precision, dimension( * ) WR, double precision, dimension( * )
       WI, double precision, dimension( ldvs, * ) VS, integer LDVS, double
       precision, dimension( * ) WORK, integer LWORK, logical, dimension( * )
       BWORK, integer INFO)
        DGEES computes the eigenvalues, the Schur form, and, optionally, the
       matrix of Schur vectors for GE matrices

Purpose:
   DGEES computes for an N-by-N real nonsymmetric matrix A, the
   eigenvalues, the real Schur form T, and, optionally, the matrix of
   Schur vectors Z.  This gives the Schur factorization A = Z*T*(Z**T).

   Optionally, it also orders the eigenvalues on the diagonal of the
   real Schur form so that selected eigenvalues are at the top left.
   The leading columns of Z then form an orthonormal basis for the
   invariant subspace corresponding to the selected eigenvalues.

   A matrix is in real Schur form if it is upper quasi-triangular with
   1-by-1 and 2-by-2 blocks. 2-by-2 blocks will be standardized in the form
          [  a  b  ]
          [  c  a  ]

   where b*c < 0. The eigenvalues of such a block are a +- sqrt(bc).

Parameters:

 JOBVS
   JOBVS is CHARACTER*1
   = 'N': Schur vectors are not computed;
   = 'V': Schur vectors are computed.

 SORT
   SORT is CHARACTER*1
   Specifies whether or not to order the eigenvalues on the
   diagonal of the Schur form.
```

```
  = 'N': Eigenvalues are not ordered;
  = 'S': Eigenvalues are ordered (see SELECT).

SELECT
  SELECT is a LOGICAL FUNCTION of two DOUBLE PRECISION arguments
  SELECT must be declared EXTERNAL in the calling subroutine.
  If SORT = 'S', SELECT is used to select eigenvalues to sort
  to the top left of the Schur form.
  If SORT = 'N', SELECT is not referenced.
  An eigenvalue WR(j)+sqrt(-1)*WI(j) is selected if
  SELECT(WR(j),WI(j)) is true; i.e., if either one of a complex
  conjugate pair of eigenvalues is selected, then both complex
  eigenvalues are selected.
  Note that a selected complex eigenvalue may no longer
  satisfy SELECT(WR(j),WI(j)) = .TRUE. after ordering, since
  ordering may change the value of complex eigenvalues
  (especially if the eigenvalue is ill-conditioned); in this
  case INFO is set to N+2 (see INFO below).

N
  N is INTEGER
  The order of the matrix A. N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the N-by-N matrix A.
  On exit, A has been overwritten by its real Schur form T.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

SDIM
  SDIM is INTEGER
  If SORT = 'N', SDIM = 0.
  If SORT = 'S', SDIM = number of eigenvalues (after sorting)
        for which SELECT is true. (Complex conjugate
        pairs for which SELECT is true for either
        eigenvalue count as 2.)

WR
  WR is DOUBLE PRECISION array, dimension (N)

WI
  WI is DOUBLE PRECISION array, dimension (N)
  WR and WI contain the real and imaginary parts,
  respectively, of the computed eigenvalues in the same order
  that they appear on the diagonal of the output Schur form T.
  Complex conjugate pairs of eigenvalues will appear
  consecutively with the eigenvalue having the positive
```

```
  imaginary part first.

VS
  VS is DOUBLE PRECISION array, dimension (LDVS,N)
  If JOBVS = 'V', VS contains the orthogonal matrix Z of Schur vectors.
  If JOBVS = 'N', VS is not referenced.

LDVS
  LDVS is INTEGER
  The leading dimension of the array VS.  LDVS >= 1; if
  JOBVS = 'V', LDVS >= N.

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) contains the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,3*N).
  For good performance, LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

BWORK
  BWORK is LOGICAL array, dimension (N)
  Not referenced if SORT = 'N'.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value.
  > 0: if INFO = i, and i is
    <= N: the QR algorithm failed to compute all the
          eigenvalues; elements 1:ILO-1 and i+1:N of WR and WI
          contain those eigenvalues which have converged; if
          JOBVS = 'V', VS contains the matrix which reduces A
          to its partially converged Schur form.
    = N+1: the eigenvalues could not be reordered because some
           eigenvalues were too close to separate (the problem
           is very ill-conditioned);
    = N+2: after reordering, roundoff changed values of some
           complex eigenvalues so that leading eigenvalues in
           the Schur form no longer satisfy SELECT=.TRUE.  This
           could also be caused by underflow due to scaling.


          ======
```

24.1.3 fortran code

— dgees.f —

```
*  =====================================================================
      SUBROUTINE DGEES( JOBVS, SORT, SELECT, N, A, LDA, SDIM, WR, WI,
     $                  VS, LDVS, WORK, LWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVS, SORT
      INTEGER            INFO, LDA, LDVS, LWORK, N, SDIM
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      DOUBLE PRECISION   A( LDA, * ), VS( LDVS, * ), WI( * ), WORK( * ),
     $                   WR( * )
*     ..
*     .. Function Arguments ..
      LOGICAL            SELECT
      EXTERNAL           SELECT
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            CURSL, LASTSL, LQUERY, LST2SL, SCALEA, WANTST,
     $                   WANTVS
      INTEGER            HSWORK, I, I1, I2, IBAL, ICOND, IERR, IEVAL,
     $                   IHI, ILO, INXT, IP, ITAU, IWRK, MAXWRK, MINWRK
      DOUBLE PRECISION   ANRM, BIGNUM, CSCALE, EPS, S, SEP, SMLNUM
*     ..
*     .. Local Arrays ..
      INTEGER            IDUM( 1 )
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DGEBAK, DGEBAL, DGEHRD, DHSEQR, DLACPY,
     $                   DLABAD, DLASCL, DORGHR, DSWAP, DTRSEN, XERBLA
*     ..
*     .. External Functions ..
```

```
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           LSAME, ILAENV, DLAMCH, DLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      WANTVS = LSAME( JOBVS, 'V' )
      WANTST = LSAME( SORT, 'S' )
      IF( ( .NOT.WANTVS ) .AND. ( .NOT.LSAME( JOBVS, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDVS.LT.1 .OR. ( WANTVS .AND. LDVS.LT.N ) ) THEN
         INFO = -11
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by DHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            MAXWRK = 1
         ELSE
            MAXWRK = 2*N + N*ILAENV( 1, 'DGEHRD', ' ', N, 1, N, 0 )
            MINWRK = 3*N
*
            CALL DHSEQR( 'S', JOBVS, N, 1, N, A, LDA, WR, WI, VS, LDVS,
     $             WORK, -1, IEVAL )
            HSWORK = WORK( 1 )
*
```

```
            IF( .NOT.WANTVS ) THEN
               MAXWRK = MAX( MAXWRK, N + HSWORK )
            ELSE
               MAXWRK = MAX( MAXWRK, 2*N + ( N - 1 )*ILAENV( 1,
     $                       'DORGHR', ' ', N, 1, N, -1 ) )
               MAXWRK = MAX( MAXWRK, N + HSWORK )
            END IF
         END IF
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -13
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEES ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = DLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
```

```
     $   CALL DLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*     Permute the matrix to make it more nearly triangular
*     (Workspace: need N)
*
      IBAL = 1
      CALL DGEBAL( 'P', N, A, LDA, ILO, IHI, WORK( IBAL ), IERR )
*
*     Reduce to upper Hessenberg form
*     (Workspace: need 3*N, prefer 2*N+N*NB)
*
      ITAU = N + IBAL
      IWRK = N + ITAU
      CALL DGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
      IF( WANTVS ) THEN
*
*        Copy Householder vectors to VS
*
         CALL DLACPY( 'L', N, N, A, LDA, VS, LDVS )
*
*        Generate orthogonal matrix in VS
*        (Workspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*
         CALL DORGHR( N, ILO, IHI, VS, LDVS, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
      END IF
*
      SDIM = 0
*
*     Perform QR iteration, accumulating Schur vectors in VS if desired
*     (Workspace: need N+1, prefer N+HSWORK (see comments) )
*
      IWRK = ITAU
      CALL DHSEQR( 'S', JOBVS, N, ILO, IHI, A, LDA, WR, WI, VS, LDVS,
     $             WORK( IWRK ), LWORK-IWRK+1, IEVAL )
      IF( IEVAL.GT.0 )
     $   INFO = IEVAL
*
*     Sort eigenvalues if desired
*
      IF( WANTST .AND. INFO.EQ.0 ) THEN
         IF( SCALEA ) THEN
            CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N, 1, WR, N, IERR )
            CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N, 1, WI, N, IERR )
         END IF
         DO 10 I = 1, N
            BWORK( I ) = SELECT( WR( I ), WI( I ) )
   10    CONTINUE
```

```
*
*        Reorder eigenvalues and transform Schur vectors
*        (Workspace: none needed)
*
         CALL DTRSEN( 'N', JOBVS, BWORK, N, A, LDA, VS, LDVS, WR, WI,
     $                SDIM, S, SEP, WORK( IWRK ), LWORK-IWRK+1, IDUM, 1,
     $                ICOND )
         IF( ICOND.GT.0 )
     $      INFO = N + ICOND
      END IF
*
      IF( WANTVS ) THEN
*
*        Undo balancing
*        (Workspace: need N)
*
         CALL DGEBAK( 'P', 'R', N, ILO, IHI, WORK( IBAL ), N, VS, LDVS,
     $                IERR )
      END IF
*
      IF( SCALEA ) THEN
*
*        Undo scaling for the Schur form of A
*
         CALL DLASCL( 'H', 0, 0, CSCALE, ANRM, N, N, A, LDA, IERR )
         CALL DCOPY( N, A, LDA+1, WR, 1 )
         IF( CSCALE.EQ.SMLNUM ) THEN
*
*           If scaling back towards underflow, adjust WI if an
*           offdiagonal element of a 2-by-2 block in the Schur form
*           underflows.
*
            IF( IEVAL.GT.0 ) THEN
               I1 = IEVAL + 1
               I2 = IHI - 1
               CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WI,
     $                      MAX( ILO-1, 1 ), IERR )
            ELSE IF( WANTST ) THEN
               I1 = 1
               I2 = N - 1
            ELSE
               I1 = ILO
               I2 = IHI - 1
            END IF
            INXT = I1 - 1
            DO 20 I = I1, I2
               IF( I.LT.INXT )
     $            GO TO 20
               IF( WI( I ).EQ.ZERO ) THEN
                  INXT = I + 1
```

```
            ELSE
               IF( A( I+1, I ).EQ.ZERO ) THEN
                  WI( I ) = ZERO
                  WI( I+1 ) = ZERO
               ELSE IF( A( I+1, I ).NE.ZERO .AND. A( I, I+1 ).EQ.
     $                 ZERO ) THEN
                  WI( I ) = ZERO
                  WI( I+1 ) = ZERO
                  IF( I.GT.1 )
     $               CALL DSWAP( I-1, A( 1, I ), 1, A( 1, I+1 ), 1 )
                  IF( N.GT.I+1 )
     $               CALL DSWAP( N-I-1, A( I, I+2 ), LDA,
     $                           A( I+1, I+2 ), LDA )
                  IF( WANTVS ) THEN
                     CALL DSWAP( N, VS( 1, I ), 1, VS( 1, I+1 ), 1 )
                  END IF
                  A( I, I+1 ) = A( I+1, I )
                  A( I+1, I ) = ZERO
               END IF
               INXT = I + 2
            END IF
   20       CONTINUE
         END IF
*
*        Undo scaling for the imaginary part of the eigenvalues
*
         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N-IEVAL, 1,
     $                WI( IEVAL+1 ), MAX( N-IEVAL, 1 ), IERR )
      END IF
*
      IF( WANTST .AND. INFO.EQ.0 ) THEN
*
*        Check if reordering successful
*
         LASTSL = .TRUE.
         LST2SL = .TRUE.
         SDIM = 0
         IP = 0
         DO 30 I = 1, N
            CURSL = SELECT( WR( I ), WI( I ) )
            IF( WI( I ).EQ.ZERO ) THEN
               IF( CURSL )
     $            SDIM = SDIM + 1
               IP = 0
               IF( CURSL .AND. .NOT.LASTSL )
     $            INFO = N + 2
            ELSE
               IF( IP.EQ.1 ) THEN
*
*                 Last eigenvalue of conjugate pair
```

```
*
                 CURSL = CURSL .OR. LASTSL
                 LASTSL = CURSL
                 IF( CURSL )
     $              SDIM = SDIM + 2
                 IP = -1
                 IF( CURSL .AND. .NOT.LST2SL )
     $              INFO = N + 2
              ELSE
*
*                First eigenvalue of conjugate pair
*
                 IP = 1
              END IF
           END IF
           LST2SL = LASTSL
           LASTSL = CURSL
   30    CONTINUE
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of DGEES
*
      END
```

———-

24.1.4 lisp code

— **dgees.lisp** —

———-

24.2 dgeesx

24.2.1 Axiom unit tests

— **dgeesx.input** —

```
)set break resume
```

```
)sys rm -f dgeesx.output
)spool dgeesx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

24.2.2 Axiom help page

— dgeesx.help —

```
=====================================================================
dgeesx examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgeesx (JOBVS, SORT, SELECT, SENSE, N, A, LDA, SDIM, WR, WI,
           VS, LDVS, RCONDE, RCONDV, WORK, LWORK, IWORK, LIWORK, BWORK, INFO)
            DGEESX computes the eigenvalues, the Schur form, and, optionally,
           the matrix of Schur vectors for GE matrices

Function/Subroutine Documentation
   subroutine dgeesx (character JOBVS, character SORT, external SELECT,
       character SENSE, integer N, double precision, dimension( lda, * ) A,
       integer LDA, integer SDIM, double precision, dimension( * ) WR, double
       precision, dimension( * ) WI, double precision, dimension( ldvs, * )
       VS, integer LDVS, double precision RCONDE, double precision RCONDV,
       double precision, dimension( * ) WORK, integer LWORK, integer,
       dimension( * ) IWORK, integer LIWORK, logical, dimension( * ) BWORK,
       integer INFO)
        DGEESX computes the eigenvalues, the Schur form, and, optionally, the
       matrix of Schur vectors for GE matrices

Purpose:
   DGEESX computes for an N-by-N real nonsymmetric matrix A, the
   eigenvalues, the real Schur form T, and, optionally, the matrix of
   Schur vectors Z.  This gives the Schur factorization A = Z*T*(Z**T).

   Optionally, it also orders the eigenvalues on the diagonal of the
```

```
  real Schur form so that selected eigenvalues are at the top left;
  computes a reciprocal condition number for the average of the
  selected eigenvalues (RCONDE); and computes a reciprocal condition
  number for the right invariant subspace corresponding to the
  selected eigenvalues (RCONDV).  The leading columns of Z form an
  orthonormal basis for this invariant subspace.

  For further explanation of the reciprocal condition numbers RCONDE
  and RCONDV, see Section 4.10 of the LAPACK Users' Guide (where
  these quantities are called s and sep respectively).

  A real matrix is in real Schur form if it is upper quasi-triangular
  with 1-by-1 and 2-by-2 blocks. 2-by-2 blocks will be standardized in
  the form
           [  a  b  ]
           [  c  a  ]

  where b*c < 0. The eigenvalues of such a block are a +- sqrt(bc).

Parameters:

 JOBVS
  JOBVS is CHARACTER*1
  = 'N': Schur vectors are not computed;
  = 'V': Schur vectors are computed.

 SORT
  SORT is CHARACTER*1
  Specifies whether or not to order the eigenvalues on the
  diagonal of the Schur form.
  = 'N': Eigenvalues are not ordered;
  = 'S': Eigenvalues are ordered (see SELECT).

 SELECT
  SELECT is procedure) LOGICAL FUNCTION of two DOUBLE PRECISION arguments
  SELECT must be declared EXTERNAL in the calling subroutine.
  If SORT = 'S', SELECT is used to select eigenvalues to sort
  to the top left of the Schur form.
  If SORT = 'N', SELECT is not referenced.
  An eigenvalue WR(j)+sqrt(-1)*WI(j) is selected if
  SELECT(WR(j),WI(j)) is true; i.e., if either one of a
  complex conjugate pair of eigenvalues is selected, then both
  are.  Note that a selected complex eigenvalue may no longer
  satisfy SELECT(WR(j),WI(j)) = .TRUE. after ordering, since
  ordering may change the value of complex eigenvalues
  (especially if the eigenvalue is ill-conditioned); in this
  case INFO may be set to N+3 (see INFO below).

 SENSE
  SENSE is CHARACTER*1
```

```
  Determines which reciprocal condition numbers are computed.
  = 'N': None are computed;
  = 'E': Computed for average of selected eigenvalues only;
  = 'V': Computed for selected right invariant subspace only;
  = 'B': Computed for both.
  If SENSE = 'E', 'V' or 'B', SORT must equal 'S'.

N
  N is INTEGER
  The order of the matrix A. N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA, N)
  On entry, the N-by-N matrix A.
  On exit, A is overwritten by its real Schur form T.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

SDIM
  SDIM is INTEGER
  If SORT = 'N', SDIM = 0.
  If SORT = 'S', SDIM = number of eigenvalues (after sorting)
  for which SELECT is true. (Complex conjugate
  pairs for which SELECT is true for either
  eigenvalue count as 2.)

WR
  WR is DOUBLE PRECISION array, dimension (N)

WI
  WI is DOUBLE PRECISION array, dimension (N)
  WR and WI contain the real and imaginary parts, respectively,
  of the computed eigenvalues, in the same order that they
  appear on the diagonal of the output Schur form T.  Complex
  conjugate pairs of eigenvalues appear consecutively with the
  eigenvalue having the positive imaginary part first.

VS
  VS is DOUBLE PRECISION array, dimension (LDVS,N)
  If JOBVS = 'V', VS contains the orthogonal matrix Z of Schur vectors.
  If JOBVS = 'N', VS is not referenced.

LDVS
  LDVS is INTEGER
  The leading dimension of the array VS.  LDVS >= 1, and if
  JOBVS = 'V', LDVS >= N.

RCONDE
```

```
  RCONDE is DOUBLE PRECISION
  If SENSE = 'E' or 'B', RCONDE contains the reciprocal
  condition number for the average of the selected eigenvalues.
  Not referenced if SENSE = 'N' or 'V'.

RCONDV
  RCONDV is DOUBLE PRECISION
  If SENSE = 'V' or 'B', RCONDV contains the reciprocal
  condition number for the selected right invariant subspace.
  Not referenced if SENSE = 'N' or 'E'.

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,3*N).
  Also, if SENSE = 'E' or 'V' or 'B',
  LWORK >= N+2*SDIM*(N-SDIM), where SDIM is the number of
  selected eigenvalues computed by this routine.  Note that
  N+2*SDIM*(N-SDIM) <= N+N*N/2. Note also that an error is only
  returned if LWORK < max(1,3*N), but if SENSE = 'E' or 'V' or
  'B' this may not be large enough.
  For good performance, LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates upper bounds on the optimal sizes of the
  arrays WORK and IWORK, returns these values as the first
  entries of the WORK and IWORK arrays, and no error messages
  related to LWORK or LIWORK are issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (MAX(1,LIWORK))
  On exit, if INFO = 0, IWORK(1) returns the optimal LIWORK.

LIWORK
  LIWORK is INTEGER
  The dimension of the array IWORK.
  LIWORK >= 1; if SENSE = 'V' or 'B', LIWORK >= SDIM*(N-SDIM).
  Note that SDIM*(N-SDIM) <= N*N/4. Note also that an error is
  only returned if LIWORK < 1, but if SENSE = 'V' or 'B' this
  may not be large enough.

  If LIWORK = -1, then a workspace query is assumed; the
  routine only calculates upper bounds on the optimal sizes of
  the arrays WORK and IWORK, returns these values as the first
  entries of the WORK and IWORK arrays, and no error messages
  related to LWORK or LIWORK are issued by XERBLA.
```

```
BWORK
  BWORK is LOGICAL array, dimension (N)
  Not referenced if SORT = 'N'.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value.
  > 0: if INFO = i, and i is
     <= N: the QR algorithm failed to compute all the
           eigenvalues; elements 1:ILO-1 and i+1:N of WR and WI
           contain those eigenvalues which have converged; if
           JOBVS = 'V', VS contains the transformation which
           reduces A to its partially converged Schur form.
     = N+1: the eigenvalues could not be reordered because some
            eigenvalues were too close to separate (the problem
            is very ill-conditioned);
     = N+2: after reordering, roundoff changed values of some
            complex eigenvalues so that leading eigenvalues in
            the Schur form no longer satisfy SELECT=.TRUE.  This
            could also be caused by underflow due to scaling.
```

———-

24.2.3 fortran code

— dgeesx.f —

```
*  =====================================================================
      SUBROUTINE DGEESX( JOBVS, SORT, SELECT, SENSE, N, A, LDA, SDIM,
     $                   WR, WI, VS, LDVS, RCONDE, RCONDV, WORK, LWORK,
     $                   IWORK, LIWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVS, SENSE, SORT
      INTEGER            INFO, LDA, LDVS, LIWORK, LWORK, N, SDIM
      DOUBLE PRECISION   RCONDE, RCONDV
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), VS( LDVS, * ), WI( * ), WORK( * ),
```

```
     $                   WR( * )
*     ..
*     .. Function Arguments ..
      LOGICAL            SELECT
      EXTERNAL           SELECT
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            CURSL, LASTSL, LQUERY, LST2SL, SCALEA, WANTSB,
     $                   WANTSE, WANTSN, WANTST, WANTSV, WANTVS
      INTEGER            HSWORK, I, I1, I2, IBAL, ICOND, IERR, IEVAL,
     $                   IHI, ILO, INXT, IP, ITAU, IWRK, LIWRK, LWRK,
     $                   MAXWRK, MINWRK
      DOUBLE PRECISION   ANRM, BIGNUM, CSCALE, EPS, SMLNUM
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DGEBAK, DGEBAL, DGEHRD, DHSEQR, DLACPY,
     $                   DLASCL, DORGHR, DSWAP, DTRSEN, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           LSAME, ILAENV, DLABAD, DLAMCH, DLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      WANTVS = LSAME( JOBVS, 'V' )
      WANTST = LSAME( SORT, 'S' )
      WANTSN = LSAME( SENSE, 'N' )
      WANTSE = LSAME( SENSE, 'E' )
      WANTSV = LSAME( SENSE, 'V' )
      WANTSB = LSAME( SENSE, 'B' )
      LQUERY = ( LWORK.EQ.-1 .OR. LIWORK.EQ.-1 )
*
```

```
      IF( ( .NOT.WANTVS ) .AND. ( .NOT.LSAME( JOBVS, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( .NOT.( WANTSN .OR. WANTSE .OR. WANTSV .OR. WANTSB ) .OR.
     $         ( .NOT.WANTST .AND. .NOT.WANTSN ) ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDVS.LT.1 .OR. ( WANTVS .AND. LDVS.LT.N ) ) THEN
         INFO = -12
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "RWorkspace:" describe the
*       minimal amount of real workspace needed at that point in the
*       code, as well as the preferred amount for good performance.
*       IWorkspace refers to integer workspace.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by DHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.
*       If SENSE = 'E', 'V' or 'B', then the amount of workspace needed
*       depends on SDIM, which is computed by the routine DTRSEN later
*       in the code.)
*
      IF( INFO.EQ.0 ) THEN
         LIWRK = 1
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            LWRK = 1
         ELSE
            MAXWRK = 2*N + N*ILAENV( 1, 'DGEHRD', ' ', N, 1, N, 0 )
            MINWRK = 3*N
*
            CALL DHSEQR( 'S', JOBVS, N, 1, N, A, LDA, WR, WI, VS, LDVS,
     $             WORK, -1, IEVAL )
            HSWORK = WORK( 1 )
*
            IF( .NOT.WANTVS ) THEN
               MAXWRK = MAX( MAXWRK, N + HSWORK )
            ELSE
               MAXWRK = MAX( MAXWRK, 2*N + ( N - 1 )*ILAENV( 1,
     $                       'DORGHR', ' ', N, 1, N, -1 ) )
               MAXWRK = MAX( MAXWRK, N + HSWORK )
            END IF
            LWRK = MAXWRK
```

```
            IF( .NOT.WANTSN )
     $         LWRK = MAX( LWRK, N + ( N*N )/2 )
            IF( WANTSV .OR. WANTSB )
     $         LIWRK = ( N*N )/4
         END IF
         IWORK( 1 ) = LIWRK
         WORK( 1 ) = LWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -16
         ELSE IF( LIWORK.LT.1 .AND. .NOT.LQUERY ) THEN
            INFO = -18
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEESX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = DLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
```

```
     $   CALL DLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*     Permute the matrix to make it more nearly triangular
*     (RWorkspace: need N)
*
      IBAL = 1
      CALL DGEBAL( 'P', N, A, LDA, ILO, IHI, WORK( IBAL ), IERR )
*
*     Reduce to upper Hessenberg form
*     (RWorkspace: need 3*N, prefer 2*N+N*NB)
*
      ITAU = N + IBAL
      IWRK = N + ITAU
      CALL DGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
      IF( WANTVS ) THEN
*
*        Copy Householder vectors to VS
*
         CALL DLACPY( 'L', N, N, A, LDA, VS, LDVS )
*
*        Generate orthogonal matrix in VS
*        (RWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*
         CALL DORGHR( N, ILO, IHI, VS, LDVS, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
      END IF
*
      SDIM = 0
*
*     Perform QR iteration, accumulating Schur vectors in VS if desired
*     (RWorkspace: need N+1, prefer N+HSWORK (see comments) )
*
      IWRK = ITAU
      CALL DHSEQR( 'S', JOBVS, N, ILO, IHI, A, LDA, WR, WI, VS, LDVS,
     $             WORK( IWRK ), LWORK-IWRK+1, IEVAL )
      IF( IEVAL.GT.0 )
     $   INFO = IEVAL
*
*     Sort eigenvalues if desired
*
      IF( WANTST .AND. INFO.EQ.0 ) THEN
         IF( SCALEA ) THEN
            CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N, 1, WR, N, IERR )
            CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N, 1, WI, N, IERR )
         END IF
         DO 10 I = 1, N
            BWORK( I ) = SELECT( WR( I ), WI( I ) )
   10    CONTINUE
```

```
*
*        Reorder eigenvalues, transform Schur vectors, and compute
*        reciprocal condition numbers
*        (RWorkspace: if SENSE is not 'N', need N+2*SDIM*(N-SDIM)
*                     otherwise, need N )
*        (IWorkspace: if SENSE is 'V' or 'B', need SDIM*(N-SDIM)
*                     otherwise, need 0 )
*
         CALL DTRSEN( SENSE, JOBVS, BWORK, N, A, LDA, VS, LDVS, WR, WI,
     $                SDIM, RCONDE, RCONDV, WORK( IWRK ), LWORK-IWRK+1,
     $                IWORK, LIWORK, ICOND )
         IF( .NOT.WANTSN )
     $      MAXWRK = MAX( MAXWRK, N+2*SDIM*( N-SDIM ) )
         IF( ICOND.EQ.-15 ) THEN
*
*           Not enough real workspace
*
            INFO = -16
         ELSE IF( ICOND.EQ.-17 ) THEN
*
*           Not enough integer workspace
*
            INFO = -18
         ELSE IF( ICOND.GT.0 ) THEN
*
*           DTRSEN failed to reorder or to restore standard Schur form
*
            INFO = ICOND + N
         END IF
      END IF
*
      IF( WANTVS ) THEN
*
*        Undo balancing
*        (RWorkspace: need N)
*
         CALL DGEBAK( 'P', 'R', N, ILO, IHI, WORK( IBAL ), N, VS, LDVS,
     $                IERR )
      END IF
*
      IF( SCALEA ) THEN
*
*        Undo scaling for the Schur form of A
*
         CALL DLASCL( 'H', 0, 0, CSCALE, ANRM, N, N, A, LDA, IERR )
         CALL DCOPY( N, A, LDA+1, WR, 1 )
         IF( ( WANTSV .OR. WANTSB ) .AND. INFO.EQ.0 ) THEN
            DUM( 1 ) = RCONDV
            CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, 1, 1, DUM, 1, IERR )
            RCONDV = DUM( 1 )
```

```
         END IF
         IF( CSCALE.EQ.SMLNUM ) THEN
*
*           If scaling back towards underflow, adjust WI if an
*           offdiagonal element of a 2-by-2 block in the Schur form
*           underflows.
*
            IF( IEVAL.GT.0 ) THEN
               I1 = IEVAL + 1
               I2 = IHI - 1
               CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WI, N,
     $                      IERR )
            ELSE IF( WANTST ) THEN
               I1 = 1
               I2 = N - 1
            ELSE
               I1 = ILO
               I2 = IHI - 1
            END IF
            INXT = I1 - 1
            DO 20 I = I1, I2
               IF( I.LT.INXT )
     $            GO TO 20
               IF( WI( I ).EQ.ZERO ) THEN
                  INXT = I + 1
               ELSE
                  IF( A( I+1, I ).EQ.ZERO ) THEN
                     WI( I ) = ZERO
                     WI( I+1 ) = ZERO
                  ELSE IF( A( I+1, I ).NE.ZERO .AND. A( I, I+1 ).EQ.
     $                    ZERO ) THEN
                     WI( I ) = ZERO
                     WI( I+1 ) = ZERO
                     IF( I.GT.1 )
     $                  CALL DSWAP( I-1, A( 1, I ), 1, A( 1, I+1 ), 1 )
                     IF( N.GT.I+1 )
     $                  CALL DSWAP( N-I-1, A( I, I+2 ), LDA,
     $                              A( I+1, I+2 ), LDA )
                     CALL DSWAP( N, VS( 1, I ), 1, VS( 1, I+1 ), 1 )
                     A( I, I+1 ) = A( I+1, I )
                     A( I+1, I ) = ZERO
                  END IF
                  INXT = I + 2
               END IF
   20       CONTINUE
         END IF
         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N-IEVAL, 1,
     $                WI( IEVAL+1 ), MAX( N-IEVAL, 1 ), IERR )
      END IF
*
```

```
      IF( WANTST .AND. INFO.EQ.0 ) THEN
*
*        Check if reordering successful
*
         LASTSL = .TRUE.
         LST2SL = .TRUE.
         SDIM = 0
         IP = 0
         DO 30 I = 1, N
            CURSL = SELECT( WR( I ), WI( I ) )
            IF( WI( I ).EQ.ZERO ) THEN
               IF( CURSL )
     $            SDIM = SDIM + 1
               IP = 0
               IF( CURSL .AND. .NOT.LASTSL )
     $            INFO = N + 2
            ELSE
               IF( IP.EQ.1 ) THEN
*
*                 Last eigenvalue of conjugate pair
*
                  CURSL = CURSL .OR. LASTSL
                  LASTSL = CURSL
                  IF( CURSL )
     $               SDIM = SDIM + 2
                  IP = -1
                  IF( CURSL .AND. .NOT.LST2SL )
     $               INFO = N + 2
               ELSE
*
*                 First eigenvalue of conjugate pair
*
                  IP = 1
               END IF
            END IF
            LST2SL = LASTSL
            LASTSL = CURSL
   30    CONTINUE
      END IF
*
      WORK( 1 ) = MAXWRK
      IF( WANTSV .OR. WANTSB ) THEN
         IWORK( 1 ) = MAX( 1, SDIM*( N-SDIM ) )
      ELSE
         IWORK( 1 ) = 1
      END IF
*
      RETURN
*
*     End of DGEESX
```

```
*
      END
```

———-

24.2.4 lisp code

— **dgeesx.lisp** —

———-

24.3 dgeev

24.3.1 Axiom unit tests

— **dgeev.input** —

```
)set break resume
)sys rm -f dgeev.output
)spool dgeev.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

24.3.2 Axiom help page

— **dgeev.help** —

```
=====================================================================
dgeev examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine dgeev (JOBVL, JOBVR, N, A, LDA, WR, WI, VL, LDVL, VR, LDVR,
           WORK, LWORK, INFO)
            DGEEV computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for GE matrices

Function/Subroutine Documentation
   subroutine dgeev (character JOBVL, character JOBVR, integer N, double
       precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( * ) WR, double precision, dimension( * ) WI, double
       precision, dimension( ldvl, * ) VL, integer LDVL, double precision,
       dimension( ldvr, * ) VR, integer LDVR, double precision, dimension( * )
       WORK, integer LWORK, integer INFO)
        DGEEV computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for GE matrices

Purpose:
   DGEEV computes for an N-by-N real nonsymmetric matrix A, the
   eigenvalues and, optionally, the left and/or right eigenvectors.

   The right eigenvector v(j) of A satisfies
   A * v(j) = lambda(j) * v(j)
   where lambda(j) is its eigenvalue.
   The left eigenvector u(j) of A satisfies
   u(j)**H * A = lambda(j) * u(j)**H
   where u(j)**H denotes the conjugate-transpose of u(j).

   The computed eigenvectors are normalized to have Euclidean norm
   equal to 1 and largest component real.

Parameters:

 JOBVL
   JOBVL is CHARACTER*1
   = 'N': left eigenvectors of A are not computed;
   = 'V': left eigenvectors of A are computed.

 JOBVR
   JOBVR is CHARACTER*1
   = 'N': right eigenvectors of A are not computed;
   = 'V': right eigenvectors of A are computed.

 N
   N is INTEGER
   The order of the matrix A. N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
```

```
  On entry, the N-by-N matrix A.
  On exit, A has been overwritten.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

WR
  WR is DOUBLE PRECISION array, dimension (N)

WI
  WI is DOUBLE PRECISION array, dimension (N)
  WR and WI contain the real and imaginary parts,
  respectively, of the computed eigenvalues.  Complex
  conjugate pairs of eigenvalues appear consecutively
  with the eigenvalue having the positive imaginary part first.

VL
  VL is DOUBLE PRECISION array, dimension (LDVL,N)
  If JOBVL = 'V', the left eigenvectors u(j) are stored one
  after another in the columns of VL, in the same order
  as their eigenvalues.
  If JOBVL = 'N', VL is not referenced.
  If the j-th eigenvalue is real, then u(j) = VL(:,j),
  the j-th column of VL.
  If the j-th and (j+1)-st eigenvalues form a complex
  conjugate pair, then u(j) = VL(:,j) + i*VL(:,j+1) and
  u(j+1) = VL(:,j) - i*VL(:,j+1).

LDVL
  LDVL is INTEGER
  The leading dimension of the array VL.  LDVL >= 1; if
  JOBVL = 'V', LDVL >= N.

VR
  VR is DOUBLE PRECISION array, dimension (LDVR,N)
  If JOBVR = 'V', the right eigenvectors v(j) are stored one
  after another in the columns of VR, in the same order
  as their eigenvalues.
  If JOBVR = 'N', VR is not referenced.
  If the j-th eigenvalue is real, then v(j) = VR(:,j),
  the j-th column of VR.
  If the j-th and (j+1)-st eigenvalues form a complex
  conjugate pair, then v(j) = VR(:,j) + i*VR(:,j+1) and
  v(j+1) = VR(:,j) - i*VR(:,j+1).

LDVR
  LDVR is INTEGER
  The leading dimension of the array VR.  LDVR >= 1; if
  JOBVR = 'V', LDVR >= N.
```

```
 WORK
   WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.  LWORK >= max(1,3*N), and
   if JOBVL = 'V' or JOBVR = 'V', LWORK >= 4*N.  For good
   performance, LWORK must generally be larger.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value.
   > 0:  if INFO = i, the QR algorithm failed to compute all the
         eigenvalues, and no eigenvectors have been computed;
         elements i+1:N of WR and WI contain eigenvalues which
         have converged.
```

———-

24.3.3 fortran code

— dgeev.f —

```
*  =====================================================================
      SUBROUTINE DGEEV( JOBVL, JOBVR, N, A, LDA, WR, WI, VL, LDVL, VR,
     $                  LDVR, WORK, LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVL, JOBVR
      INTEGER            INFO, LDA, LDVL, LDVR, LWORK, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   WI( * ), WORK( * ), WR( * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, SCALEA, WANTVL, WANTVR
      CHARACTER          SIDE
      INTEGER            HSWORK, I, IBAL, IERR, IHI, ILO, ITAU, IWRK, K,
     $                   MAXWRK, MINWRK, NOUT
      DOUBLE PRECISION   ANRM, BIGNUM, CS, CSCALE, EPS, R, SCL, SMLNUM,
     $                   SN
*     ..
*     .. Local Arrays ..
      LOGICAL            SELECT( 1 )
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEBAK, DGEBAL, DGEHRD, DHSEQR, DLABAD, DLACPY,
     $                   DLARTG, DLASCL, DORGHR, DROT, DSCAL, DTREVC,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX, ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE, DLAPY2, DNRM2
      EXTERNAL           LSAME, IDAMAX, ILAENV, DLAMCH, DLANGE, DLAPY2,
     $                   DNRM2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      WANTVL = LSAME( JOBVL, 'V' )
      WANTVR = LSAME( JOBVR, 'V' )
      IF( ( .NOT.WANTVL ) .AND. ( .NOT.LSAME( JOBVL, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTVR ) .AND. ( .NOT.LSAME( JOBVR, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
```

```
         INFO = -5
      ELSE IF( LDVL.LT.1 .OR. ( WANTVL .AND. LDVL.LT.N ) ) THEN
         INFO = -9
      ELSE IF( LDVR.LT.1 .OR. ( WANTVR .AND. LDVR.LT.N ) ) THEN
         INFO = -11
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by DHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            MAXWRK = 1
         ELSE
            MAXWRK = 2*N + N*ILAENV( 1, 'DGEHRD', ' ', N, 1, N, 0 )
            IF( WANTVL ) THEN
               MINWRK = 4*N
               MAXWRK = MAX( MAXWRK, 2*N + ( N - 1 )*ILAENV( 1,
     $                       'DORGHR', ' ', N, 1, N, -1 ) )
               CALL DHSEQR( 'S', 'V', N, 1, N, A, LDA, WR, WI, VL, LDVL,
     $                WORK, -1, INFO )
               HSWORK = WORK( 1 )
               MAXWRK = MAX( MAXWRK, N + 1, N + HSWORK )
               MAXWRK = MAX( MAXWRK, 4*N )
            ELSE IF( WANTVR ) THEN
               MINWRK = 4*N
               MAXWRK = MAX( MAXWRK, 2*N + ( N - 1 )*ILAENV( 1,
     $                       'DORGHR', ' ', N, 1, N, -1 ) )
               CALL DHSEQR( 'S', 'V', N, 1, N, A, LDA, WR, WI, VR, LDVR,
     $                WORK, -1, INFO )
               HSWORK = WORK( 1 )
               MAXWRK = MAX( MAXWRK, N + 1, N + HSWORK )
               MAXWRK = MAX( MAXWRK, 4*N )
            ELSE
               MINWRK = 3*N
               CALL DHSEQR( 'E', 'N', N, 1, N, A, LDA, WR, WI, VR, LDVR,
     $                WORK, -1, INFO )
               HSWORK = WORK( 1 )
               MAXWRK = MAX( MAXWRK, N + 1, N + HSWORK )
            END IF
            MAXWRK = MAX( MAXWRK, MINWRK )
         END IF
```

```
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -13
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEEV ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = DLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
     $   CALL DLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*     Balance the matrix
*     (Workspace: need N)
*
      IBAL = 1
      CALL DGEBAL( 'B', N, A, LDA, ILO, IHI, WORK( IBAL ), IERR )
*
*     Reduce to upper Hessenberg form
*     (Workspace: need 3*N, prefer 2*N+N*NB)
```

```
*
      ITAU = IBAL + N
      IWRK = ITAU + N
      CALL DGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
      IF( WANTVL ) THEN
*
*        Want left eigenvectors
*        Copy Householder vectors to VL
*
         SIDE = 'L'
         CALL DLACPY( 'L', N, N, A, LDA, VL, LDVL )
*
*        Generate orthogonal matrix in VL
*        (Workspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*
         CALL DORGHR( N, ILO, IHI, VL, LDVL, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VL
*        (Workspace: need N+1, prefer N+HSWORK (see comments) )
*
         IWRK = ITAU
         CALL DHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, WR, WI, VL, LDVL,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
         IF( WANTVR ) THEN
*
*           Want left and right eigenvectors
*           Copy Schur vectors to VR
*
            SIDE = 'B'
            CALL DLACPY( 'F', N, N, VL, LDVL, VR, LDVR )
         END IF
*
      ELSE IF( WANTVR ) THEN
*
*        Want right eigenvectors
*        Copy Householder vectors to VR
*
         SIDE = 'R'
         CALL DLACPY( 'L', N, N, A, LDA, VR, LDVR )
*
*        Generate orthogonal matrix in VR
*        (Workspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*
         CALL DORGHR( N, ILO, IHI, VR, LDVR, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
```

```
*        Perform QR iteration, accumulating Schur vectors in VR
*        (Workspace: need N+1, prefer N+HSWORK (see comments) )
*
         IWRK = ITAU
         CALL DHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, WR, WI, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
      ELSE
*
*        Compute eigenvalues only
*        (Workspace: need N+1, prefer N+HSWORK (see comments) )
*
         IWRK = ITAU
         CALL DHSEQR( 'E', 'N', N, ILO, IHI, A, LDA, WR, WI, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
      END IF
*
*     If INFO > 0 from DHSEQR, then quit
*
      IF( INFO.GT.0 )
     $   GO TO 50
*
      IF( WANTVL .OR. WANTVR ) THEN
*
*        Compute left and/or right eigenvectors
*        (Workspace: need 4*N)
*
         CALL DTREVC( SIDE, 'B', SELECT, N, A, LDA, VL, LDVL, VR, LDVR,
     $                N, NOUT, WORK( IWRK ), IERR )
      END IF
*
      IF( WANTVL ) THEN
*
*        Undo balancing of left eigenvectors
*        (Workspace: need N)
*
         CALL DGEBAK( 'B', 'L', N, ILO, IHI, WORK( IBAL ), N, VL, LDVL,
     $                IERR )
*
*        Normalize left eigenvectors and make largest component real
*
         DO 20 I = 1, N
            IF( WI( I ).EQ.ZERO ) THEN
               SCL = ONE / DNRM2( N, VL( 1, I ), 1 )
               CALL DSCAL( N, SCL, VL( 1, I ), 1 )
            ELSE IF( WI( I ).GT.ZERO ) THEN
               SCL = ONE / DLAPY2( DNRM2( N, VL( 1, I ), 1 ),
     $               DNRM2( N, VL( 1, I+1 ), 1 ) )
               CALL DSCAL( N, SCL, VL( 1, I ), 1 )
               CALL DSCAL( N, SCL, VL( 1, I+1 ), 1 )
```

```
               DO 10 K = 1, N
                  WORK( IWRK+K-1 ) = VL( K, I )**2 + VL( K, I+1 )**2
   10          CONTINUE
               K = IDAMAX( N, WORK( IWRK ), 1 )
               CALL DLARTG( VL( K, I ), VL( K, I+1 ), CS, SN, R )
               CALL DROT( N, VL( 1, I ), 1, VL( 1, I+1 ), 1, CS, SN )
               VL( K, I+1 ) = ZERO
            END IF
   20    CONTINUE
      END IF
*
      IF( WANTVR ) THEN
*
*        Undo balancing of right eigenvectors
*        (Workspace: need N)
*
         CALL DGEBAK( 'B', 'R', N, ILO, IHI, WORK( IBAL ), N, VR, LDVR,
     $                IERR )
*
*        Normalize right eigenvectors and make largest component real
*
         DO 40 I = 1, N
            IF( WI( I ).EQ.ZERO ) THEN
               SCL = ONE / DNRM2( N, VR( 1, I ), 1 )
               CALL DSCAL( N, SCL, VR( 1, I ), 1 )
            ELSE IF( WI( I ).GT.ZERO ) THEN
               SCL = ONE / DLAPY2( DNRM2( N, VR( 1, I ), 1 ),
     $               DNRM2( N, VR( 1, I+1 ), 1 ) )
               CALL DSCAL( N, SCL, VR( 1, I ), 1 )
               CALL DSCAL( N, SCL, VR( 1, I+1 ), 1 )
               DO 30 K = 1, N
                  WORK( IWRK+K-1 ) = VR( K, I )**2 + VR( K, I+1 )**2
   30          CONTINUE
               K = IDAMAX( N, WORK( IWRK ), 1 )
               CALL DLARTG( VR( K, I ), VR( K, I+1 ), CS, SN, R )
               CALL DROT( N, VR( 1, I ), 1, VR( 1, I+1 ), 1, CS, SN )
               VR( K, I+1 ) = ZERO
            END IF
   40    CONTINUE
      END IF
*
*     Undo scaling if necessary
*
   50 CONTINUE
      IF( SCALEA ) THEN
         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, WR( INFO+1 ),
     $                MAX( N-INFO, 1 ), IERR )
         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, WI( INFO+1 ),
     $                MAX( N-INFO, 1 ), IERR )
         IF( INFO.GT.0 ) THEN
```

```
            CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WR, N,
     $                   IERR )
            CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WI, N,
     $                   IERR )
         END IF
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of DGEEV
*
      END
```

———-

24.3.4 lisp code

— **dgeev.lisp** —

———-

24.4 dgeevx

24.4.1 Axiom unit tests

— **dgeevx.input** —

```
)set break resume
)sys rm -f dgeevx.output
)spool dgeevx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

24.4.2 Axiom help page

— dgeevx.help —

```
==================================================================
dgeevx examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgeevx (BALANC, JOBVL, JOBVR, SENSE, N, A, LDA, WR, WI, VL,
           LDVL, VR, LDVR, ILO, IHI, SCALE, ABNRM, RCONDE, RCONDV, WORK,
           LWORK, IWORK, INFO)
            DGEEVX computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for GE matrices

Function/Subroutine Documentation
   subroutine dgeevx (character BALANC, character JOBVL, character JOBVR,
       character SENSE, integer N, double precision, dimension( lda, * ) A,
       integer LDA, double precision, dimension( * ) WR, double precision,
       dimension( * ) WI, double precision, dimension( ldvl, * ) VL, integer
       LDVL, double precision, dimension( ldvr, * ) VR, integer LDVR, integer
       ILO, integer IHI, double precision, dimension( * ) SCALE, double
       precision ABNRM, double precision, dimension( * ) RCONDE, double
       precision, dimension( * ) RCONDV, double precision, dimension( * )
       WORK, integer LWORK, integer, dimension( * ) IWORK, integer INFO)
        DGEEVX computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for GE matrices

Purpose:
   DGEEVX computes for an N-by-N real nonsymmetric matrix A, the
   eigenvalues and, optionally, the left and/or right eigenvectors.

   Optionally also, it computes a balancing transformation to improve
   the conditioning of the eigenvalues and eigenvectors (ILO, IHI,
   SCALE, and ABNRM), reciprocal condition numbers for the eigenvalues
   (RCONDE), and reciprocal condition numbers for the right
   eigenvectors (RCONDV).

   The right eigenvector v(j) of A satisfies
            A * v(j) = lambda(j) * v(j)
   where lambda(j) is its eigenvalue.
   The left eigenvector u(j) of A satisfies
            u(j)**H * A = lambda(j) * u(j)**H
   where u(j)**H denotes the conjugate-transpose of u(j).
```

```
  The computed eigenvectors are normalized to have Euclidean norm
  equal to 1 and largest component real.

  Balancing a matrix means permuting the rows and columns to make it
  more nearly upper triangular, and applying a diagonal similarity
  transformation D * A * D**(-1), where D is a diagonal matrix, to
  make its rows and columns closer in norm and the condition numbers
  of its eigenvalues and eigenvectors smaller.  The computed
  reciprocal condition numbers correspond to the balanced matrix.
  Permuting rows and columns will not change the condition numbers
  (in exact arithmetic) but diagonal scaling will.  For further
  explanation of balancing, see section 4.10.2 of the LAPACK Users' Guide.

Parameters:

 BALANC
  BALANC is CHARACTER*1
  Indicates how the input matrix should be diagonally scaled
  and/or permuted to improve the conditioning of its
  eigenvalues.
  = 'N': Do not diagonally scale or permute;
  = 'P': Perform permutations to make the matrix more nearly
         upper triangular. Do not diagonally scale;
  = 'S': Diagonally scale the matrix, i.e. replace A by
         D*A*D**(-1), where D is a diagonal matrix chosen
         to make the rows and columns of A more equal in
         norm. Do not permute;
  = 'B': Both diagonally scale and permute A.
  Computed reciprocal condition numbers will be for the matrix
  after balancing and/or permuting. Permuting does not change
  condition numbers (in exact arithmetic), but balancing does.

 JOBVL
  JOBVL is CHARACTER*1
  = 'N': left eigenvectors of A are not computed;
  = 'V': left eigenvectors of A are computed.
  If SENSE = 'E' or 'B', JOBVL must = 'V'.

 JOBVR
  JOBVR is CHARACTER*1
  = 'N': right eigenvectors of A are not computed;
  = 'V': right eigenvectors of A are computed.
  If SENSE = 'E' or 'B', JOBVR must = 'V'.

 SENSE
  SENSE is CHARACTER*1
  Determines which reciprocal condition numbers are computed.
  = 'N': None are computed;
  = 'E': Computed for eigenvalues only;
```

```
  = 'V': Computed for right eigenvectors only;
  = 'B': Computed for eigenvalues and right eigenvectors.

  If SENSE = 'E' or 'B', both left and right eigenvectors
  must also be computed (JOBVL = 'V' and JOBVR = 'V').

N
  N is INTEGER
  The order of the matrix A. N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the N-by-N matrix A.
  On exit, A has been overwritten.  If JOBVL = 'V' or
  JOBVR = 'V', A contains the real Schur form of the balanced
  version of the input matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

WR
  WR is DOUBLE PRECISION array, dimension (N)

WI
  WI is DOUBLE PRECISION array, dimension (N)
  WR and WI contain the real and imaginary parts,
  respectively, of the computed eigenvalues.  Complex
  conjugate pairs of eigenvalues will appear consecutively
  with the eigenvalue having the positive imaginary part first.

VL
  VL is DOUBLE PRECISION array, dimension (LDVL,N)
  If JOBVL = 'V', the left eigenvectors u(j) are stored one
  after another in the columns of VL, in the same order
  as their eigenvalues.
  If JOBVL = 'N', VL is not referenced.
  If the j-th eigenvalue is real, then u(j) = VL(:,j),
  the j-th column of VL.
  If the j-th and (j+1)-st eigenvalues form a complex
  conjugate pair, then u(j) = VL(:,j) + i*VL(:,j+1) and
  u(j+1) = VL(:,j) - i*VL(:,j+1).

LDVL
  LDVL is INTEGER
  The leading dimension of the array VL.  LDVL >= 1; if
  JOBVL = 'V', LDVL >= N.

VR
  VR is DOUBLE PRECISION array, dimension (LDVR,N)
```

```
  If JOBVR = 'V', the right eigenvectors v(j) are stored one
  after another in the columns of VR, in the same order
  as their eigenvalues.
  If JOBVR = 'N', VR is not referenced.
  If the j-th eigenvalue is real, then v(j) = VR(:,j),
  the j-th column of VR.
  If the j-th and (j+1)-st eigenvalues form a complex
  conjugate pair, then v(j) = VR(:,j) + i*VR(:,j+1) and
  v(j+1) = VR(:,j) - i*VR(:,j+1).

LDVR
  LDVR is INTEGER
  The leading dimension of the array VR.  LDVR >= 1, and if
  JOBVR = 'V', LDVR >= N.

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER
  ILO and IHI are integer values determined when A was
  balanced.  The balanced A(i,j) = 0 if I > J and
  J = 1,...,ILO-1 or I = IHI+1,...,N.

SCALE
  SCALE is DOUBLE PRECISION array, dimension (N)
  Details of the permutations and scaling factors applied
  when balancing A.  If P(j) is the index of the row and column
  interchanged with row and column j, and D(j) is the scaling
  factor applied to row and column j, then
  SCALE(J) = P(J),    for J = 1,...,ILO-1
           = D(J),    for J = ILO,...,IHI
           = P(J)     for J = IHI+1,...,N.
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

ABNRM
  ABNRM is DOUBLE PRECISION
  The one-norm of the balanced matrix (the maximum
  of the sum of absolute values of elements of any column).

RCONDE
  RCONDE is DOUBLE PRECISION array, dimension (N)
  RCONDE(j) is the reciprocal condition number of the j-th eigenvalue.

RCONDV
  RCONDV is DOUBLE PRECISION array, dimension (N)
  RCONDV(j) is the reciprocal condition number of the j-th
  right eigenvector.
```

```
 WORK
   WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.   If SENSE = 'N' or 'E',
   LWORK >= max(1,2*N), and if JOBVL = 'V' or JOBVR = 'V',
   LWORK >= 3*N.  If SENSE = 'V' or 'B', LWORK >= N*(N+6).
   For good performance, LWORK must generally be larger.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 IWORK
   IWORK is INTEGER array, dimension (2*N-2)
   If SENSE = 'N' or 'E', not referenced.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value.
   > 0:  if INFO = i, the QR algorithm failed to compute all the
         eigenvalues, and no eigenvectors or condition numbers
         have been computed; elements 1:ILO-1 and i+1:N of WR
         and WI contain eigenvalues which have converged.
```

———-

24.4.3 fortran code

— **dgeevx.f** —

```
*  =====================================================================
      SUBROUTINE DGEEVX( BALANC, JOBVL, JOBVR, SENSE, N, A, LDA, WR, WI,
     $                   VL, LDVL, VR, LDVR, ILO, IHI, SCALE, ABNRM,
     $                   RCONDE, RCONDV, WORK, LWORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          BALANC, JOBVL, JOBVR, SENSE
```

```
      INTEGER            IHI, ILO, INFO, LDA, LDVL, LDVR, LWORK, N
      DOUBLE PRECISION   ABNRM
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), RCONDE( * ), RCONDV( * ),
     $                   SCALE( * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   WI( * ), WORK( * ), WR( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, SCALEA, WANTVL, WANTVR, WNTSNB, WNTSNE,
     $                   WNTSNN, WNTSNV
      CHARACTER          JOB, SIDE
      INTEGER            HSWORK, I, ICOND, IERR, ITAU, IWRK, K, MAXWRK,
     $                   MINWRK, NOUT
      DOUBLE PRECISION   ANRM, BIGNUM, CS, CSCALE, EPS, R, SCL, SMLNUM,
     $                   SN
*     ..
*     .. Local Arrays ..
      LOGICAL            SELECT( 1 )
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEBAK, DGEBAL, DGEHRD, DHSEQR, DLABAD, DLACPY,
     $                   DLARTG, DLASCL, DORGHR, DROT, DSCAL, DTREVC,
     $                   DTRSNA, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX, ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE, DLAPY2, DNRM2
      EXTERNAL           LSAME, IDAMAX, ILAENV, DLAMCH, DLANGE, DLAPY2,
     $                   DNRM2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
```

```
      WANTVL = LSAME( JOBVL, 'V' )
      WANTVR = LSAME( JOBVR, 'V' )
      WNTSNN = LSAME( SENSE, 'N' )
      WNTSNE = LSAME( SENSE, 'E' )
      WNTSNV = LSAME( SENSE, 'V' )
      WNTSNB = LSAME( SENSE, 'B' )
      IF( .NOT.( LSAME( BALANC, 'N' ) .OR. LSAME( BALANC,
     $    'S' ) .OR. LSAME( BALANC, 'P' ) .OR. LSAME( BALANC, 'B' ) ) )
     $     THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTVL ) .AND. ( .NOT.LSAME( JOBVL, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( ( .NOT.WANTVR ) .AND. ( .NOT.LSAME( JOBVR, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( .NOT.( WNTSNN .OR. WNTSNE .OR. WNTSNB .OR. WNTSNV ) .OR.
     $         ( ( WNTSNE .OR. WNTSNB ) .AND. .NOT.( WANTVL .AND.
     $         WANTVR ) ) ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDVL.LT.1 .OR. ( WANTVL .AND. LDVL.LT.N ) ) THEN
         INFO = -11
      ELSE IF( LDVR.LT.1 .OR. ( WANTVR .AND. LDVR.LT.N ) ) THEN
         INFO = -13
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by DHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            MAXWRK = 1
         ELSE
            MAXWRK = N + N*ILAENV( 1, 'DGEHRD', ' ', N, 1, N, 0 )
*
            IF( WANTVL ) THEN
               CALL DHSEQR( 'S', 'V', N, 1, N, A, LDA, WR, WI, VL, LDVL,
     $                 WORK, -1, INFO )
            ELSE IF( WANTVR ) THEN
               CALL DHSEQR( 'S', 'V', N, 1, N, A, LDA, WR, WI, VR, LDVR,
```

```
     $               WORK, -1, INFO )
            ELSE
               IF( WNTSNN ) THEN
                  CALL DHSEQR( 'E', 'N', N, 1, N, A, LDA, WR, WI, VR,
     $                LDVR, WORK, -1, INFO )
               ELSE
                  CALL DHSEQR( 'S', 'N', N, 1, N, A, LDA, WR, WI, VR,
     $                LDVR, WORK, -1, INFO )
               END IF
            END IF
            HSWORK = WORK( 1 )
*
            IF( ( .NOT.WANTVL ) .AND. ( .NOT.WANTVR ) ) THEN
               MINWRK = 2*N
               IF( .NOT.WNTSNN )
     $            MINWRK = MAX( MINWRK, N*N+6*N )
               MAXWRK = MAX( MAXWRK, HSWORK )
               IF( .NOT.WNTSNN )
     $            MAXWRK = MAX( MAXWRK, N*N + 6*N )
            ELSE
               MINWRK = 3*N
               IF( ( .NOT.WNTSNN ) .AND. ( .NOT.WNTSNE ) )
     $            MINWRK = MAX( MINWRK, N*N + 6*N )
               MAXWRK = MAX( MAXWRK, HSWORK )
               MAXWRK = MAX( MAXWRK, N + ( N - 1 )*ILAENV( 1, 'DORGHR',
     $                       ' ', N, 1, N, -1 ) )
               IF( ( .NOT.WNTSNN ) .AND. ( .NOT.WNTSNE ) )
     $            MAXWRK = MAX( MAXWRK, N*N + 6*N )
               MAXWRK = MAX( MAXWRK, 3*N )
            END IF
            MAXWRK = MAX( MAXWRK, MINWRK )
         END IF
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -21
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEEVX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
```

```
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ICOND = 0
      ANRM = DLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
     $   CALL DLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*     Balance the matrix and compute ABNRM
*
      CALL DGEBAL( BALANC, N, A, LDA, ILO, IHI, SCALE, IERR )
      ABNRM = DLANGE( '1', N, N, A, LDA, DUM )
      IF( SCALEA ) THEN
         DUM( 1 ) = ABNRM
         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, 1, 1, DUM, 1, IERR )
         ABNRM = DUM( 1 )
      END IF
*
*     Reduce to upper Hessenberg form
*     (Workspace: need 2*N, prefer N+N*NB)
*
      ITAU = 1
      IWRK = ITAU + N
      CALL DGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
      IF( WANTVL ) THEN
*
*        Want left eigenvectors
*        Copy Householder vectors to VL
*
         SIDE = 'L'
         CALL DLACPY( 'L', N, N, A, LDA, VL, LDVL )
```

```
*
*        Generate orthogonal matrix in VL
*        (Workspace: need 2*N-1, prefer N+(N-1)*NB)
*
         CALL DORGHR( N, ILO, IHI, VL, LDVL, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VL
*        (Workspace: need 1, prefer HSWORK (see comments) )
*
         IWRK = ITAU
         CALL DHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, WR, WI, VL, LDVL,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
         IF( WANTVR ) THEN
*
*           Want left and right eigenvectors
*           Copy Schur vectors to VR
*
            SIDE = 'B'
            CALL DLACPY( 'F', N, N, VL, LDVL, VR, LDVR )
         END IF
*
      ELSE IF( WANTVR ) THEN
*
*        Want right eigenvectors
*        Copy Householder vectors to VR
*
         SIDE = 'R'
         CALL DLACPY( 'L', N, N, A, LDA, VR, LDVR )
*
*        Generate orthogonal matrix in VR
*        (Workspace: need 2*N-1, prefer N+(N-1)*NB)
*
         CALL DORGHR( N, ILO, IHI, VR, LDVR, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VR
*        (Workspace: need 1, prefer HSWORK (see comments) )
*
         IWRK = ITAU
         CALL DHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, WR, WI, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
      ELSE
*
*        Compute eigenvalues only
*        If condition numbers desired, compute Schur form
*
         IF( WNTSNN ) THEN
```

```
            JOB = 'E'
         ELSE
            JOB = 'S'
         END IF
*
*        (Workspace: need 1, prefer HSWORK (see comments) )
*
         IWRK = ITAU
         CALL DHSEQR( JOB, 'N', N, ILO, IHI, A, LDA, WR, WI, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
      END IF
*
*     If INFO > 0 from DHSEQR, then quit
*
      IF( INFO.GT.0 )
     $   GO TO 50
*
      IF( WANTVL .OR. WANTVR ) THEN
*
*        Compute left and/or right eigenvectors
*        (Workspace: need 3*N)
*
         CALL DTREVC( SIDE, 'B', SELECT, N, A, LDA, VL, LDVL, VR, LDVR,
     $                N, NOUT, WORK( IWRK ), IERR )
      END IF
*
*     Compute condition numbers if desired
*     (Workspace: need N*N+6*N unless SENSE = 'E')
*
      IF( .NOT.WNTSNN ) THEN
         CALL DTRSNA( SENSE, 'A', SELECT, N, A, LDA, VL, LDVL, VR, LDVR,
     $                RCONDE, RCONDV, N, NOUT, WORK( IWRK ), N, IWORK,
     $                ICOND )
      END IF
*
      IF( WANTVL ) THEN
*
*        Undo balancing of left eigenvectors
*
         CALL DGEBAK( BALANC, 'L', N, ILO, IHI, SCALE, N, VL, LDVL,
     $                IERR )
*
*        Normalize left eigenvectors and make largest component real
*
         DO 20 I = 1, N
            IF( WI( I ).EQ.ZERO ) THEN
               SCL = ONE / DNRM2( N, VL( 1, I ), 1 )
               CALL DSCAL( N, SCL, VL( 1, I ), 1 )
            ELSE IF( WI( I ).GT.ZERO ) THEN
               SCL = ONE / DLAPY2( DNRM2( N, VL( 1, I ), 1 ),
```

```
     $              DNRM2( N, VL( 1, I+1 ), 1 ) )
                CALL DSCAL( N, SCL, VL( 1, I ), 1 )
                CALL DSCAL( N, SCL, VL( 1, I+1 ), 1 )
                DO 10 K = 1, N
                   WORK( K ) = VL( K, I )**2 + VL( K, I+1 )**2
   10           CONTINUE
                K = IDAMAX( N, WORK, 1 )
                CALL DLARTG( VL( K, I ), VL( K, I+1 ), CS, SN, R )
                CALL DROT( N, VL( 1, I ), 1, VL( 1, I+1 ), 1, CS, SN )
                VL( K, I+1 ) = ZERO
             END IF
   20    CONTINUE
      END IF
*
      IF( WANTVR ) THEN
*
*        Undo balancing of right eigenvectors
*
         CALL DGEBAK( BALANC, 'R', N, ILO, IHI, SCALE, N, VR, LDVR,
     $                IERR )
*
*        Normalize right eigenvectors and make largest component real
*
         DO 40 I = 1, N
            IF( WI( I ).EQ.ZERO ) THEN
               SCL = ONE / DNRM2( N, VR( 1, I ), 1 )
               CALL DSCAL( N, SCL, VR( 1, I ), 1 )
            ELSE IF( WI( I ).GT.ZERO ) THEN
               SCL = ONE / DLAPY2( DNRM2( N, VR( 1, I ), 1 ),
     $              DNRM2( N, VR( 1, I+1 ), 1 ) )
               CALL DSCAL( N, SCL, VR( 1, I ), 1 )
               CALL DSCAL( N, SCL, VR( 1, I+1 ), 1 )
               DO 30 K = 1, N
                  WORK( K ) = VR( K, I )**2 + VR( K, I+1 )**2
   30          CONTINUE
               K = IDAMAX( N, WORK, 1 )
               CALL DLARTG( VR( K, I ), VR( K, I+1 ), CS, SN, R )
               CALL DROT( N, VR( 1, I ), 1, VR( 1, I+1 ), 1, CS, SN )
               VR( K, I+1 ) = ZERO
            END IF
   40    CONTINUE
      END IF
*
*     Undo scaling if necessary
*
   50 CONTINUE
      IF( SCALEA ) THEN
         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, WR( INFO+1 ),
     $                MAX( N-INFO, 1 ), IERR )
         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, WI( INFO+1 ),
```

```
     $                MAX( N-INFO, 1 ), IERR )
         IF( INFO.EQ.0 ) THEN
            IF( ( WNTSNV .OR. WNTSNB ) .AND. ICOND.EQ.0 )
     $         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N, 1, RCONDV, N,
     $                      IERR )
         ELSE
            CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WR, N,
     $                   IERR )
            CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WI, N,
     $                   IERR )
         END IF
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of DGEEVX
*
      END
```

———-

24.4.4 lisp code

— **dgeevx.lisp** —

———-

24.5 dgegs - DEPRECATED use dgges

24.6 dgegv - DEPRECATED use dggev

24.7 dgges

24.7.1 Axiom unit tests

— **dgges.input** —

```
)set break resume
)sys rm -f dgges.output
```

```
)spool dgges.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

24.7.2 Axiom help page

— dgges.help —

```
=====================================================================
dgges examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgges (JOBVSL, JOBVSR, SORT, SELCTG, N, A, LDA, B, LDB,
           SDIM, ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR, WORK, LWORK,
           BWORK, INFO)
            DGGES computes the eigenvalues, the Schur form, and, optionally,
           the matrix of Schur vectors for GE matrices

Function/Subroutine Documentation
   subroutine dgges (character JOBVSL, character JOBVSR, character SORT,
       external SELCTG, integer N, double precision, dimension( lda, * ) A,
       integer LDA, double precision, dimension( ldb, * ) B, integer LDB,
       integer SDIM, double precision, dimension( * ) ALPHAR, double
       precision, dimension( * ) ALPHAI, double precision, dimension( * )
       BETA, double precision, dimension( ldvsl, * ) VSL, integer LDVSL,
       double precision, dimension( ldvsr, * ) VSR, integer LDVSR, double
       precision, dimension( * ) WORK, integer LWORK, logical, dimension( * )
       BWORK, integer INFO)
        DGGES computes the eigenvalues, the Schur form, and, optionally, the
       matrix of Schur vectors for GE matrices

Purpose:
   DGGES computes for a pair of N-by-N real nonsymmetric matrices (A,B),
   the generalized eigenvalues, the generalized real Schur form (S,T),
   optionally, the left and/or right matrices of Schur vectors (VSL and
   VSR). This gives the generalized Schur factorization
```

```
          (A,B) = ( (VSL)*S*(VSR)**T, (VSL)*T*(VSR)**T )

   Optionally, it also orders the eigenvalues so that a selected cluster
   of eigenvalues appears in the leading diagonal blocks of the upper
   quasi-triangular matrix S and the upper triangular matrix T.The
   leading columns of VSL and VSR then form an orthonormal basis for the
   corresponding left and right eigenspaces (deflating subspaces).

   (If only the generalized eigenvalues are needed, use the driver
   DGGEV instead, which is faster.)

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar w
   or a ratio alpha/beta = w, such that  A - w*B is singular.  It is
   usually represented as the pair (alpha,beta), as there is a
   reasonable interpretation for beta=0 or both being zero.

   A pair of matrices (S,T) is in generalized real Schur form if T is
   upper triangular with non-negative diagonal and S is block upper
   triangular with 1-by-1 and 2-by-2 blocks.  1-by-1 blocks correspond
   to real generalized eigenvalues, while 2-by-2 blocks of S will be
   "standardized" by making the corresponding elements of T have the form:
          [  a  0  ]
          [  0  b  ]
   and the pair of corresponding 2-by-2 blocks in S and T will have a
   complex conjugate pair of generalized eigenvalues.

Parameters:

 JOBVSL
   JOBVSL is CHARACTER*1
   = 'N':  do not compute the left Schur vectors;
   = 'V':  compute the left Schur vectors.

 JOBVSR
   JOBVSR is CHARACTER*1
   = 'N':  do not compute the right Schur vectors;
   = 'V':  compute the right Schur vectors.

 SORT
   SORT is CHARACTER*1
   Specifies whether or not to order the eigenvalues on the
   diagonal of the generalized Schur form.
   = 'N':  Eigenvalues are not ordered;
   = 'S':  Eigenvalues are ordered (see SELCTG);

 SELCTG
   SELCTG is a LOGICAL FUNCTION of three DOUBLE PRECISION arguments
   SELCTG must be declared EXTERNAL in the calling subroutine.
   If SORT = 'N', SELCTG is not referenced.
```

```
  If SORT = 'S', SELCTG is used to select eigenvalues to sort
  to the top left of the Schur form.
  An eigenvalue (ALPHAR(j)+ALPHAI(j))/BETA(j) is selected if
  SELCTG(ALPHAR(j),ALPHAI(j),BETA(j)) is true; i.e. if either
  one of a complex conjugate pair of eigenvalues is selected,
  then both complex eigenvalues are selected.

  Note that in the ill-conditioned case, a selected complex
  eigenvalue may no longer satisfy SELCTG(ALPHAR(j),ALPHAI(j),
  BETA(j)) = .TRUE. after ordering. INFO is to be set to N+2
  in this case.

N
  N is INTEGER
  The order of the matrices A, B, VSL, and VSR.  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA, N)
  On entry, the first of the pair of matrices.
  On exit, A has been overwritten by its generalized Schur form S.

LDA
  LDA is INTEGER
  The leading dimension of A.  LDA >= max(1,N).

B
  B is DOUBLE PRECISION array, dimension (LDB, N)
  On entry, the second of the pair of matrices.
  On exit, B has been overwritten by its generalized Schur form T.

LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).

SDIM
  SDIM is INTEGER
  If SORT = 'N', SDIM = 0.
  If SORT = 'S', SDIM = number of eigenvalues (after sorting)
  for which SELCTG is true.  (Complex conjugate pairs for which
  SELCTG is true for either eigenvalue count as 2.)

ALPHAR
  ALPHAR is DOUBLE PRECISION array, dimension (N)

ALPHAI
  ALPHAI is DOUBLE PRECISION array, dimension (N)

BETA
  BETA is DOUBLE PRECISION array, dimension (N)
  On exit, (ALPHAR(j) + ALPHAI(j)*i)/BETA(j), j=1,...,N, will
```

```
  be the generalized eigenvalues.  ALPHAR(j) + ALPHAI(j)*i,
  and  BETA(j),j=1,...,N are the diagonals of the complex Schur
  form (S,T) that would result if the 2-by-2 diagonal blocks of
  the real Schur form of (A,B) were further reduced to
  triangular form using 2-by-2 complex unitary transformations.
  If ALPHAI(j) is zero, then the j-th eigenvalue is real; if
  positive, then the j-th and (j+1)-st eigenvalues are a
  complex conjugate pair, with ALPHAI(j+1) negative.

  Note: the quotients ALPHAR(j)/BETA(j) and ALPHAI(j)/BETA(j)
  may easily over- or underflow, and BETA(j) may even be zero.
  Thus, the user should avoid naively computing the ratio.
  However, ALPHAR and ALPHAI will be always less than and
  usually comparable with norm(A) in magnitude, and BETA always
  less than and usually comparable with norm(B).

VSL
  VSL is DOUBLE PRECISION array, dimension (LDVSL,N)
  If JOBVSL = 'V', VSL will contain the left Schur vectors.
  Not referenced if JOBVSL = 'N'.

LDVSL
  LDVSL is INTEGER
  The leading dimension of the matrix VSL. LDVSL >=1, and
  if JOBVSL = 'V', LDVSL >= N.

VSR
  VSR is DOUBLE PRECISION array, dimension (LDVSR,N)
  If JOBVSR = 'V', VSR will contain the right Schur vectors.
  Not referenced if JOBVSR = 'N'.

LDVSR
  LDVSR is INTEGER
  The leading dimension of the matrix VSR. LDVSR >= 1, and
  if JOBVSR = 'V', LDVSR >= N.

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  If N = 0, LWORK >= 1, else LWORK >= 8*N+16.
  For good performance , LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.
```

```
 BWORK
   BWORK is LOGICAL array, dimension (N)
   Not referenced if SORT = 'N'.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value.
   = 1,...,N:
         The QZ iteration failed.  (A,B) are not in Schur
         form, but ALPHAR(j), ALPHAI(j), and BETA(j) should
         be correct for j=INFO+1,...,N.
   > N:
    =N+1: other than QZ iteration failed in DHGEQZ.
    =N+2: after reordering, roundoff changed values of
          some complex eigenvalues so that leading
          eigenvalues in the Generalized Schur form no
          longer satisfy SELCTG=.TRUE.  This could also
          be caused due to scaling.
    =N+3: reordering failed in DTGSEN.
```

———-

24.7.3 fortran code

— dgges.f —

```
*  =====================================================================
      SUBROUTINE DGGES( JOBVSL, JOBVSR, SORT, SELCTG, N, A, LDA, B, LDB,
     $                  SDIM, ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR,
     $                  LDVSR, WORK, LWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVSL, JOBVSR, SORT
      INTEGER            INFO, LDA, LDB, LDVSL, LDVSR, LWORK, N, SDIM
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      DOUBLE PRECISION   A( LDA, * ), ALPHAI( * ), ALPHAR( * ),
     $                   B( LDB, * ), BETA( * ), VSL( LDVSL, * ),
     $                   VSR( LDVSR, * ), WORK( * )
```

```
*     ..
*     .. Function Arguments ..
      LOGICAL            SELCTG
      EXTERNAL           SELCTG
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            CURSL, ILASCL, ILBSCL, ILVSL, ILVSR, LASTSL,
     $                   LQUERY, LST2SL, WANTST
      INTEGER            I, ICOLS, IERR, IHI, IJOBVL, IJOBVR, ILEFT,
     $                   ILO, IP, IRIGHT, IROWS, ITAU, IWRK, MAXWRK,
     $                   MINWRK
      DOUBLE PRECISION   ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS, PVSL,
     $                   PVSR, SAFMAX, SAFMIN, SMLNUM
*     ..
*     .. Local Arrays ..
      INTEGER            IDUM( 1 )
      DOUBLE PRECISION   DIF( 2 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEQRF, DGGBAK, DGGBAL, DGGHRD, DHGEQZ, DLABAD,
     $                   DLACPY, DLASCL, DLASET, DORGQR, DORMQR, DTGSEN,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           LSAME, ILAENV, DLAMCH, DLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVSL, 'N' ) ) THEN
         IJOBVL = 1
         ILVSL = .FALSE.
      ELSE IF( LSAME( JOBVSL, 'V' ) ) THEN
         IJOBVL = 2
         ILVSL = .TRUE.
      ELSE
```

```
         IJOBVL = -1
         ILVSL = .FALSE.
      END IF
*
      IF( LSAME( JOBVSR, 'N' ) ) THEN
         IJOBVR = 1
         ILVSR = .FALSE.
      ELSE IF( LSAME( JOBVSR, 'V' ) ) THEN
         IJOBVR = 2
         ILVSR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVSR = .FALSE.
      END IF
*
      WANTST = LSAME( SORT, 'S' )
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -2
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDVSL.LT.1 .OR. ( ILVSL .AND. LDVSL.LT.N ) ) THEN
         INFO = -15
      ELSE IF( LDVSR.LT.1 .OR. ( ILVSR .AND. LDVSR.LT.N ) ) THEN
         INFO = -17
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.GT.0 )THEN
            MINWRK = MAX( 8*N, 6*N + 16 )
            MAXWRK = MINWRK - N +
```

```
     $               N*ILAENV( 1, 'DGEQRF', ' ', N, 1, N, 0 )
            MAXWRK = MAX( MAXWRK, MINWRK - N +
     $                    N*ILAENV( 1, 'DORMQR', ' ', N, 1, N, -1 ) )
            IF( ILVSL ) THEN
               MAXWRK = MAX( MAXWRK, MINWRK - N +
     $                       N*ILAENV( 1, 'DORGQR', ' ', N, 1, N, -1 ) )
            END IF
         ELSE
            MINWRK = 1
            MAXWRK = 1
         END IF
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY )
     $      INFO = -19
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGGES ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SAFMIN = DLAMCH( 'S' )
      SAFMAX = ONE / SAFMIN
      CALL DLABAD( SAFMIN, SAFMAX )
      SMLNUM = SQRT( SAFMIN ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = DLANGE( 'M', N, N, A, LDA, WORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
```

```
      END IF
      IF( ILASCL )
     $   CALL DLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = DLANGE( 'M', N, N, B, LDB, WORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
     $   CALL DLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrix to make it more nearly triangular
*     (Workspace: need 6*N + 2*N space for storing balancing factors)
*
      ILEFT = 1
      IRIGHT = N + 1
      IWRK = IRIGHT + N
      CALL DGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, WORK( ILEFT ),
     $             WORK( IRIGHT ), WORK( IWRK ), IERR )
*
*     Reduce B to triangular form (QR decomposition of B)
*     (Workspace: need N, prefer N*NB)
*
      IROWS = IHI + 1 - ILO
      ICOLS = N + 1 - ILO
      ITAU = IWRK
      IWRK = ITAU + IROWS
      CALL DGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to matrix A
*     (Workspace: need N, prefer N*NB)
*
      CALL DORMQR( 'L', 'T', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VSL
*     (Workspace: need N, prefer N*NB)
*
      IF( ILVSL ) THEN
         CALL DLASET( 'Full', N, N, ZERO, ONE, VSL, LDVSL )
         IF( IROWS.GT.1 ) THEN
```

```
            CALL DLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VSL( ILO+1, ILO ), LDVSL )
         END IF
         CALL DORGQR( IROWS, IROWS, IROWS, VSL( ILO, ILO ), LDVSL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Initialize VSR
*
      IF( ILVSR )
     $   CALL DLASET( 'Full', N, N, ZERO, ONE, VSR, LDVSR )
*
*     Reduce to generalized Hessenberg form
*     (Workspace: none needed)
*
      CALL DGGHRD( JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB, VSL,
     $             LDVSL, VSR, LDVSR, IERR )
*
*     Perform QZ algorithm, computing Schur vectors if desired
*     (Workspace: need N)
*
      IWRK = ITAU
      CALL DHGEQZ( 'S', JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR,
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 50
      END IF
*
*     Sort eigenvalues ALPHA/BETA if desired
*     (Workspace: need 4*N+16 )
*
      SDIM = 0
      IF( WANTST ) THEN
*
*        Undo scaling on eigenvalues before SELCTGing
*
         IF( ILASCL ) THEN
            CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAR, N,
     $                   IERR )
            CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAI, N,
     $                   IERR )
         END IF
```

```
         IF( ILBSCL )
     $      CALL DLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
*
*        Select eigenvalues
*
         DO 10 I = 1, N
            BWORK( I ) = SELCTG( ALPHAR( I ), ALPHAI( I ), BETA( I ) )
   10    CONTINUE
*
         CALL DTGSEN( 0, ILVSL, ILVSR, BWORK, N, A, LDA, B, LDB, ALPHAR,
     $                ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR, SDIM, PVSL,
     $                PVSR, DIF, WORK( IWRK ), LWORK-IWRK+1, IDUM, 1,
     $                IERR )
         IF( IERR.EQ.1 )
     $      INFO = N + 3
*
      END IF
*
*     Apply back-permutation to VSL and VSR
*     (Workspace: none needed)
*
      IF( ILVSL )
     $   CALL DGGBAK( 'P', 'L', N, ILO, IHI, WORK( ILEFT ),
     $                WORK( IRIGHT ), N, VSL, LDVSL, IERR )
*
      IF( ILVSR )
     $   CALL DGGBAK( 'P', 'R', N, ILO, IHI, WORK( ILEFT ),
     $                WORK( IRIGHT ), N, VSR, LDVSR, IERR )
*
*     Check if unscaling would cause over/underflow, if so, rescale
*     (ALPHAR(I),ALPHAI(I),BETA(I)) so BETA(I) is on the order of
*     B(I,I) and ALPHAR(I) and ALPHAI(I) are on the order of A(I,I)
*
      IF( ILASCL ) THEN
         DO 20 I = 1, N
            IF( ALPHAI( I ).NE.ZERO ) THEN
               IF( ( ALPHAR( I ) / SAFMAX ).GT.( ANRMTO / ANRM ) .OR.
     $             ( SAFMIN / ALPHAR( I ) ).GT.( ANRM / ANRMTO ) ) THEN
                  WORK( 1 ) = ABS( A( I, I ) / ALPHAR( I ) )
                  BETA( I ) = BETA( I )*WORK( 1 )
                  ALPHAR( I ) = ALPHAR( I )*WORK( 1 )
                  ALPHAI( I ) = ALPHAI( I )*WORK( 1 )
               ELSE IF( ( ALPHAI( I ) / SAFMAX ).GT.
     $                  ( ANRMTO / ANRM ) .OR.
     $                  ( SAFMIN / ALPHAI( I ) ).GT.( ANRM / ANRMTO ) )
     $                   THEN
                  WORK( 1 ) = ABS( A( I, I+1 ) / ALPHAI( I ) )
                  BETA( I ) = BETA( I )*WORK( 1 )
                  ALPHAR( I ) = ALPHAR( I )*WORK( 1 )
                  ALPHAI( I ) = ALPHAI( I )*WORK( 1 )
```

```
            END IF
         END IF
   20    CONTINUE
      END IF
*
      IF( ILBSCL ) THEN
         DO 30 I = 1, N
            IF( ALPHAI( I ).NE.ZERO ) THEN
               IF( ( BETA( I ) / SAFMAX ).GT.( BNRMTO / BNRM ) .OR.
     $             ( SAFMIN / BETA( I ) ).GT.( BNRM / BNRMTO ) ) THEN
                  WORK( 1 ) = ABS( B( I, I ) / BETA( I ) )
                  BETA( I ) = BETA( I )*WORK( 1 )
                  ALPHAR( I ) = ALPHAR( I )*WORK( 1 )
                  ALPHAI( I ) = ALPHAI( I )*WORK( 1 )
               END IF
            END IF
   30    CONTINUE
      END IF
*
*     Undo scaling
*
      IF( ILASCL ) THEN
         CALL DLASCL( 'H', 0, 0, ANRMTO, ANRM, N, N, A, LDA, IERR )
         CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAR, N, IERR )
         CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAI, N, IERR )
      END IF
*
      IF( ILBSCL ) THEN
         CALL DLASCL( 'U', 0, 0, BNRMTO, BNRM, N, N, B, LDB, IERR )
         CALL DLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
      END IF
*
      IF( WANTST ) THEN
*
*        Check if reordering is correct
*
         LASTSL = .TRUE.
         LST2SL = .TRUE.
         SDIM = 0
         IP = 0
         DO 40 I = 1, N
            CURSL = SELCTG( ALPHAR( I ), ALPHAI( I ), BETA( I ) )
            IF( ALPHAI( I ).EQ.ZERO ) THEN
               IF( CURSL )
     $            SDIM = SDIM + 1
               IP = 0
               IF( CURSL .AND. .NOT.LASTSL )
     $            INFO = N + 2
            ELSE
               IF( IP.EQ.1 ) THEN
```

```
*
*                Last eigenvalue of conjugate pair
*
                 CURSL = CURSL .OR. LASTSL
                 LASTSL = CURSL
                 IF( CURSL )
     $              SDIM = SDIM + 2
                 IP = -1
                 IF( CURSL .AND. .NOT.LST2SL )
     $              INFO = N + 2
              ELSE
*
*                First eigenvalue of conjugate pair
*
                 IP = 1
              END IF
           END IF
           LST2SL = LASTSL
           LASTSL = CURSL
   40    CONTINUE
*
      END IF
*
   50 CONTINUE
*
      WORK( 1 ) = MAXWRK
*
      RETURN
*
*     End of DGGES
*
      END
```

———-

24.7.4 lisp code

— **dgges.lisp** —

———-

24.8 dgges3

24.8.1 Axiom unit tests

— dgges3.input —

```
)set break resume
)sys rm -f dgges3.output
)spool dgges3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

24.8.2 Axiom help page

— dgges3.help —

```
======================================================================
dgges3 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgges3 (JOBVSL, JOBVSR, SORT, SELCTG, N, A, LDA, B, LDB,
           SDIM, ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR, WORK, LWORK,
           BWORK, INFO)
            DGGES3 computes the eigenvalues, the Schur form, and, optionally,
           the matrix of Schur vectors for GE matrices (blocked algorithm)

Function/Subroutine Documentation
   subroutine dgges3 (character JOBVSL, character JOBVSR, character SORT,
       external SELCTG, integer N, double precision, dimension( lda, * ) A,
       integer LDA, double precision, dimension( ldb, * ) B, integer LDB,
       integer SDIM, double precision, dimension( * ) ALPHAR, double
       precision, dimension( * ) ALPHAI, double precision, dimension( * )
       BETA, double precision, dimension( ldvsl, * ) VSL, integer LDVSL,
       double precision, dimension( ldvsr, * ) VSR, integer LDVSR, double
```

```
      precision, dimension( * ) WORK, integer LWORK, logical, dimension( * )
      BWORK, integer INFO)
       DGGES3 computes the eigenvalues, the Schur form, and, optionally, the
      matrix of Schur vectors for GE matrices (blocked algorithm)

Purpose:
   DGGES3 computes for a pair of N-by-N real nonsymmetric matrices (A,B),
   the generalized eigenvalues, the generalized real Schur form (S,T),
   optionally, the left and/or right matrices of Schur vectors (VSL and
   VSR). This gives the generalized Schur factorization

        (A,B) = ( (VSL)*S*(VSR)**T, (VSL)*T*(VSR)**T )

   Optionally, it also orders the eigenvalues so that a selected cluster
   of eigenvalues appears in the leading diagonal blocks of the upper
   quasi-triangular matrix S and the upper triangular matrix T.The
   leading columns of VSL and VSR then form an orthonormal basis for the
   corresponding left and right eigenspaces (deflating subspaces).

   (If only the generalized eigenvalues are needed, use the driver
   DGGEV instead, which is faster.)

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar w
   or a ratio alpha/beta = w, such that  A - w*B is singular.  It is
   usually represented as the pair (alpha,beta), as there is a
   reasonable interpretation for beta=0 or both being zero.

   A pair of matrices (S,T) is in generalized real Schur form if T is
   upper triangular with non-negative diagonal and S is block upper
   triangular with 1-by-1 and 2-by-2 blocks.  1-by-1 blocks correspond
   to real generalized eigenvalues, while 2-by-2 blocks of S will be
   "standardized" by making the corresponding elements of T have the form:
          [  a  0  ]
          [  0  b  ]
   and the pair of corresponding 2-by-2 blocks in S and T will have a
   complex conjugate pair of generalized eigenvalues.

Parameters:

 JOBVSL
   JOBVSL is CHARACTER*1
   = 'N':  do not compute the left Schur vectors;
   = 'V':  compute the left Schur vectors.

 JOBVSR
   JOBVSR is CHARACTER*1
   = 'N':  do not compute the right Schur vectors;
   = 'V':  compute the right Schur vectors.

 SORT
```

```
  SORT is CHARACTER*1
  Specifies whether or not to order the eigenvalues on the
  diagonal of the generalized Schur form.
  = 'N':  Eigenvalues are not ordered;
  = 'S':  Eigenvalues are ordered (see SELCTG);

SELCTG
  SELCTG is a LOGICAL FUNCTION of three DOUBLE PRECISION arguments
  SELCTG must be declared EXTERNAL in the calling subroutine.
  If SORT = 'N', SELCTG is not referenced.
  If SORT = 'S', SELCTG is used to select eigenvalues to sort
  to the top left of the Schur form.
  An eigenvalue (ALPHAR(j)+ALPHAI(j))/BETA(j) is selected if
  SELCTG(ALPHAR(j),ALPHAI(j),BETA(j)) is true; i.e. if either
  one of a complex conjugate pair of eigenvalues is selected,
  then both complex eigenvalues are selected.

  Note that in the ill-conditioned case, a selected complex
  eigenvalue may no longer satisfy SELCTG(ALPHAR(j),ALPHAI(j),
  BETA(j)) = .TRUE. after ordering. INFO is to be set to N+2
  in this case.

N
  N is INTEGER
  The order of the matrices A, B, VSL, and VSR.  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA, N)
  On entry, the first of the pair of matrices.
  On exit, A has been overwritten by its generalized Schur form S.

LDA
  LDA is INTEGER
  The leading dimension of A.  LDA >= max(1,N).

B
  B is DOUBLE PRECISION array, dimension (LDB, N)
  On entry, the second of the pair of matrices.
  On exit, B has been overwritten by its generalized Schur form T.

LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).

SDIM
  SDIM is INTEGER
  If SORT = 'N', SDIM = 0.
  If SORT = 'S', SDIM = number of eigenvalues (after sorting)
  for which SELCTG is true.  (Complex conjugate pairs for which
  SELCTG is true for either eigenvalue count as 2.)
```

```
ALPHAR
  ALPHAR is DOUBLE PRECISION array, dimension (N)

ALPHAI
  ALPHAI is DOUBLE PRECISION array, dimension (N)

BETA
  BETA is DOUBLE PRECISION array, dimension (N)
  On exit, (ALPHAR(j) + ALPHAI(j)*i)/BETA(j), j=1,...,N, will
  be the generalized eigenvalues.  ALPHAR(j) + ALPHAI(j)*i,
  and  BETA(j),j=1,...,N are the diagonals of the complex Schur
  form (S,T) that would result if the 2-by-2 diagonal blocks of
  the real Schur form of (A,B) were further reduced to
  triangular form using 2-by-2 complex unitary transformations.
  If ALPHAI(j) is zero, then the j-th eigenvalue is real; if
  positive, then the j-th and (j+1)-st eigenvalues are a
  complex conjugate pair, with ALPHAI(j+1) negative.

  Note: the quotients ALPHAR(j)/BETA(j) and ALPHAI(j)/BETA(j)
  may easily over- or underflow, and BETA(j) may even be zero.
  Thus, the user should avoid naively computing the ratio.
  However, ALPHAR and ALPHAI will be always less than and
  usually comparable with norm(A) in magnitude, and BETA always
  less than and usually comparable with norm(B).

VSL
  VSL is DOUBLE PRECISION array, dimension (LDVSL,N)
  If JOBVSL = 'V', VSL will contain the left Schur vectors.
  Not referenced if JOBVSL = 'N'.

LDVSL
  LDVSL is INTEGER
  The leading dimension of the matrix VSL. LDVSL >=1, and
  if JOBVSL = 'V', LDVSL >= N.

VSR
  VSR is DOUBLE PRECISION array, dimension (LDVSR,N)
  If JOBVSR = 'V', VSR will contain the right Schur vectors.
  Not referenced if JOBVSR = 'N'.

LDVSR
  LDVSR is INTEGER
  The leading dimension of the matrix VSR. LDVSR >= 1, and
  if JOBVSR = 'V', LDVSR >= N.

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.
```

```
 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 BWORK
   BWORK is LOGICAL array, dimension (N)
   Not referenced if SORT = 'N'.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value.
   = 1,...,N:
         The QZ iteration failed.  (A,B) are not in Schur
         form, but ALPHAR(j), ALPHAI(j), and BETA(j) should
         be correct for j=INFO+1,...,N.
   > N:  =N+1: other than QZ iteration failed in DHGEQZ.
         =N+2: after reordering, roundoff changed values of
               some complex eigenvalues so that leading
               eigenvalues in the Generalized Schur form no
               longer satisfy SELCTG=.TRUE.  This could also
               be caused due to scaling.
        =N+3: reordering failed in DTGSEN.
```

———-

24.8.3 fortran code

— dgges3.f —

```
*  =====================================================================
      SUBROUTINE DGGES3( JOBVSL, JOBVSR, SORT, SELCTG, N, A, LDA, B,
     $                   LDB, SDIM, ALPHAR, ALPHAI, BETA, VSL, LDVSL,
     $                   VSR, LDVSR, WORK, LWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     January 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVSL, JOBVSR, SORT
```

```
      INTEGER            INFO, LDA, LDB, LDVSL, LDVSR, LWORK, N, SDIM
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      DOUBLE PRECISION   A( LDA, * ), ALPHAI( * ), ALPHAR( * ),
     $                   B( LDB, * ), BETA( * ), VSL( LDVSL, * ),
     $                   VSR( LDVSR, * ), WORK( * )
*     ..
*     .. Function Arguments ..
      LOGICAL            SELCTG
      EXTERNAL           SELCTG
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            CURSL, ILASCL, ILBSCL, ILVSL, ILVSR, LASTSL,
     $                   LQUERY, LST2SL, WANTST
      INTEGER            I, ICOLS, IERR, IHI, IJOBVL, IJOBVR, ILEFT,
     $                   ILO, IP, IRIGHT, IROWS, ITAU, IWRK, LWKOPT
      DOUBLE PRECISION   ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS, PVSL,
     $                   PVSR, SAFMAX, SAFMIN, SMLNUM
*     ..
*     .. Local Arrays ..
      INTEGER            IDUM( 1 )
      DOUBLE PRECISION   DIF( 2 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEQRF, DGGBAK, DGGBAL, DGGHD3, DHGEQZ, DLABAD,
     $                   DLACPY, DLASCL, DLASET, DORGQR, DORMQR, DTGSEN,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           LSAME, DLAMCH, DLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVSL, 'N' ) ) THEN
         IJOBVL = 1
```

```
         ILVSL = .FALSE.
      ELSE IF( LSAME( JOBVSL, 'V' ) ) THEN
         IJOBVL = 2
         ILVSL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVSL = .FALSE.
      END IF
*
      IF( LSAME( JOBVSR, 'N' ) ) THEN
         IJOBVR = 1
         ILVSR = .FALSE.
      ELSE IF( LSAME( JOBVSR, 'V' ) ) THEN
         IJOBVR = 2
         ILVSR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVSR = .FALSE.
      END IF
*
      WANTST = LSAME( SORT, 'S' )
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -2
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDVSL.LT.1 .OR. ( ILVSL .AND. LDVSL.LT.N ) ) THEN
         INFO = -15
      ELSE IF( LDVSR.LT.1 .OR. ( ILVSR .AND. LDVSR.LT.N ) ) THEN
         INFO = -17
      ELSE IF( LWORK.LT.6*N+16 .AND. .NOT.LQUERY ) THEN
         INFO = -19
      END IF
*
*     Compute workspace
*
      IF( INFO.EQ.0 ) THEN
         CALL DGEQRF( N, N, B, LDB, WORK, WORK, -1, IERR )
```

```
         LWKOPT = MAX( 6*N+16, 3*N+INT( WORK ( 1 ) ) )
         CALL DORMQR( 'L', 'T', N, N, N, B, LDB, WORK, A, LDA, WORK,
     $                -1, IERR )
         LWKOPT = MAX( LWKOPT, 3*N+INT( WORK ( 1 ) ) )
         IF( ILVSL ) THEN
            CALL DORGQR( N, N, N, VSL, LDVSL, WORK, WORK, -1, IERR )
            LWKOPT = MAX( LWKOPT, 3*N+INT( WORK ( 1 ) ) )
         END IF
         CALL DGGHD3( JOBVSL, JOBVSR, N, 1, N, A, LDA, B, LDB, VSL,
     $                LDVSL, VSR, LDVSR, WORK, -1, IERR )
         LWKOPT = MAX( LWKOPT, 3*N+INT( WORK ( 1 ) ) )
         CALL DHGEQZ( 'S', JOBVSL, JOBVSR, N, 1, N, A, LDA, B, LDB,
     $                ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR,
     $                WORK, -1, IERR )
         LWKOPT = MAX( LWKOPT, 2*N+INT( WORK ( 1 ) ) )
         IF( WANTST ) THEN
            CALL DTGSEN( 0, ILVSL, ILVSR, BWORK, N, A, LDA, B, LDB,
     $                   ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR,
     $                   SDIM, PVSL, PVSR, DIF, WORK, -1, IDUM, 1,
     $                   IERR )
            LWKOPT = MAX( LWKOPT, 2*N+INT( WORK ( 1 ) ) )
         END IF
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGGES3 ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SAFMIN = DLAMCH( 'S' )
      SAFMAX = ONE / SAFMIN
      CALL DLABAD( SAFMIN, SAFMAX )
      SMLNUM = SQRT( SAFMIN ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
```

```
      ANRM = DLANGE( 'M', N, N, A, LDA, WORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
      IF( ILASCL )
     $   CALL DLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = DLANGE( 'M', N, N, B, LDB, WORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
     $   CALL DLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrix to make it more nearly triangular
*
      ILEFT = 1
      IRIGHT = N + 1
      IWRK = IRIGHT + N
      CALL DGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, WORK( ILEFT ),
     $             WORK( IRIGHT ), WORK( IWRK ), IERR )
*
*     Reduce B to triangular form (QR decomposition of B)
*
      IROWS = IHI + 1 - ILO
      ICOLS = N + 1 - ILO
      ITAU = IWRK
      IWRK = ITAU + IROWS
      CALL DGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to matrix A
*
      CALL DORMQR( 'L', 'T', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VSL
```

```
*
      IF( ILVSL ) THEN
         CALL DLASET( 'Full', N, N, ZERO, ONE, VSL, LDVSL )
         IF( IROWS.GT.1 ) THEN
            CALL DLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VSL( ILO+1, ILO ), LDVSL )
         END IF
         CALL DORGQR( IROWS, IROWS, IROWS, VSL( ILO, ILO ), LDVSL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Initialize VSR
*
      IF( ILVSR )
     $   CALL DLASET( 'Full', N, N, ZERO, ONE, VSR, LDVSR )
*
*     Reduce to generalized Hessenberg form
*
      CALL DGGHD3( JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB, VSL,
     $             LDVSL, VSR, LDVSR, WORK( IWRK ), LWORK+1-IWRK,
     $             IERR )
*
*     Perform QZ algorithm, computing Schur vectors if desired
*
      IWRK = ITAU
      CALL DHGEQZ( 'S', JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR,
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 50
      END IF
*
*     Sort eigenvalues ALPHA/BETA if desired
*
      SDIM = 0
      IF( WANTST ) THEN
*
*        Undo scaling on eigenvalues before SELCTGing
*
         IF( ILASCL ) THEN
            CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAR, N,
     $                   IERR )
            CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAI, N,
```

```
     $                  IERR )
         END IF
         IF( ILBSCL )
     $      CALL DLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
*
*        Select eigenvalues
*
         DO 10 I = 1, N
            BWORK( I ) = SELCTG( ALPHAR( I ), ALPHAI( I ), BETA( I ) )
   10    CONTINUE
*
         CALL DTGSEN( 0, ILVSL, ILVSR, BWORK, N, A, LDA, B, LDB, ALPHAR,
     $                ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR, SDIM, PVSL,
     $                PVSR, DIF, WORK( IWRK ), LWORK-IWRK+1, IDUM, 1,
     $                IERR )
         IF( IERR.EQ.1 )
     $      INFO = N + 3
*
      END IF
*
*     Apply back-permutation to VSL and VSR
*
      IF( ILVSL )
     $   CALL DGGBAK( 'P', 'L', N, ILO, IHI, WORK( ILEFT ),
     $                WORK( IRIGHT ), N, VSL, LDVSL, IERR )
*
      IF( ILVSR )
     $   CALL DGGBAK( 'P', 'R', N, ILO, IHI, WORK( ILEFT ),
     $                WORK( IRIGHT ), N, VSR, LDVSR, IERR )
*
*     Check if unscaling would cause over/underflow, if so, rescale
*     (ALPHAR(I),ALPHAI(I),BETA(I)) so BETA(I) is on the order of
*     B(I,I) and ALPHAR(I) and ALPHAI(I) are on the order of A(I,I)
*
      IF( ILASCL ) THEN
         DO 20 I = 1, N
            IF( ALPHAI( I ).NE.ZERO ) THEN
               IF( ( ALPHAR( I ) / SAFMAX ).GT.( ANRMTO / ANRM ) .OR.
     $             ( SAFMIN / ALPHAR( I ) ).GT.( ANRM / ANRMTO ) ) THEN
                  WORK( 1 ) = ABS( A( I, I ) / ALPHAR( I ) )
                  BETA( I ) = BETA( I )*WORK( 1 )
                  ALPHAR( I ) = ALPHAR( I )*WORK( 1 )
                  ALPHAI( I ) = ALPHAI( I )*WORK( 1 )
               ELSE IF( ( ALPHAI( I ) / SAFMAX ).GT.
     $                  ( ANRMTO / ANRM ) .OR.
     $                  ( SAFMIN / ALPHAI( I ) ).GT.( ANRM / ANRMTO ) )
     $                   THEN
                  WORK( 1 ) = ABS( A( I, I+1 ) / ALPHAI( I ) )
                  BETA( I ) = BETA( I )*WORK( 1 )
                  ALPHAR( I ) = ALPHAR( I )*WORK( 1 )
```

```
                ALPHAI( I ) = ALPHAI( I )*WORK( 1 )
             END IF
          END IF
   20    CONTINUE
      END IF
*
      IF( ILBSCL ) THEN
         DO 30 I = 1, N
            IF( ALPHAI( I ).NE.ZERO ) THEN
               IF( ( BETA( I ) / SAFMAX ).GT.( BNRMTO / BNRM ) .OR.
     $             ( SAFMIN / BETA( I ) ).GT.( BNRM / BNRMTO ) ) THEN
                  WORK( 1 ) = ABS( B( I, I ) / BETA( I ) )
                  BETA( I ) = BETA( I )*WORK( 1 )
                  ALPHAR( I ) = ALPHAR( I )*WORK( 1 )
                  ALPHAI( I ) = ALPHAI( I )*WORK( 1 )
               END IF
            END IF
   30    CONTINUE
      END IF
*
*     Undo scaling
*
      IF( ILASCL ) THEN
         CALL DLASCL( 'H', 0, 0, ANRMTO, ANRM, N, N, A, LDA, IERR )
         CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAR, N, IERR )
         CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAI, N, IERR )
      END IF
*
      IF( ILBSCL ) THEN
         CALL DLASCL( 'U', 0, 0, BNRMTO, BNRM, N, N, B, LDB, IERR )
         CALL DLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
      END IF
*
      IF( WANTST ) THEN
*
*        Check if reordering is correct
*
         LASTSL = .TRUE.
         LST2SL = .TRUE.
         SDIM = 0
         IP = 0
         DO 40 I = 1, N
            CURSL = SELCTG( ALPHAR( I ), ALPHAI( I ), BETA( I ) )
            IF( ALPHAI( I ).EQ.ZERO ) THEN
               IF( CURSL )
     $            SDIM = SDIM + 1
               IP = 0
               IF( CURSL .AND. .NOT.LASTSL )
     $            INFO = N + 2
            ELSE
```

```
               IF( IP.EQ.1 ) THEN
*
*                 Last eigenvalue of conjugate pair
*
                  CURSL = CURSL .OR. LASTSL
                  LASTSL = CURSL
                  IF( CURSL )
     $               SDIM = SDIM + 2
                  IP = -1
                  IF( CURSL .AND. .NOT.LST2SL )
     $               INFO = N + 2
               ELSE
*
*                 First eigenvalue of conjugate pair
*
                  IP = 1
               END IF
            END IF
            LST2SL = LASTSL
            LASTSL = CURSL
   40    CONTINUE
*
      END IF
*
   50 CONTINUE
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of DGGES3
*
      END
```

———-

24.8.4 lisp code

— **dgges3.lisp** —

———-

24.9 dggesx

24.9.1 Axiom unit tests

— dggesx.input —

```
)set break resume
)sys rm -f dggesx.output
)spool dggesx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

24.9.2 Axiom help page

— dggesx.help —

```
======================================================================
dggesx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dggesx (JOBVSL, JOBVSR, SORT, SELCTG, SENSE, N, A, LDA, B,
           LDB, SDIM, ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR, RCONDE,
           RCONDV, WORK, LWORK, IWORK, LIWORK, BWORK, INFO)
            DGGESX computes the eigenvalues, the Schur form, and, optionally,
           the matrix of Schur vectors for GE matrices

Function/Subroutine Documentation
   subroutine dggesx (character JOBVSL, character JOBVSR, character SORT,
       external SELCTG, character SENSE, integer N, double precision,
       dimension( lda, * ) A, integer LDA, double precision, dimension( ldb, *
       ) B, integer LDB, integer SDIM, double precision, dimension( * )
       ALPHAR, double precision, dimension( * ) ALPHAI, double precision,
       dimension( * ) BETA, double precision, dimension( ldvsl, * ) VSL,
       integer LDVSL, double precision, dimension( ldvsr, * ) VSR, integer
```

```
      LDVSR, double precision, dimension( 2 ) RCONDE, double precision,
      dimension( 2 ) RCONDV, double precision, dimension( * ) WORK, integer
      LWORK, integer, dimension( * ) IWORK, integer LIWORK, logical,
      dimension( * ) BWORK, integer INFO)
       DGGESX computes the eigenvalues, the Schur form, and, optionally, the
      matrix of Schur vectors for GE matrices

Purpose:
   DGGESX computes for a pair of N-by-N real nonsymmetric matrices
   (A,B), the generalized eigenvalues, the real Schur form (S,T), and,
   optionally, the left and/or right matrices of Schur vectors (VSL and
   VSR).  This gives the generalized Schur factorization

          (A,B) = ( (VSL) S (VSR)**T, (VSL) T (VSR)**T )

   Optionally, it also orders the eigenvalues so that a selected cluster
   of eigenvalues appears in the leading diagonal blocks of the upper
   quasi-triangular matrix S and the upper triangular matrix T; computes
   a reciprocal condition number for the average of the selected
   eigenvalues (RCONDE); and computes a reciprocal condition number for
   the right and left deflating subspaces corresponding to the selected
   eigenvalues (RCONDV). The leading columns of VSL and VSR then form
   an orthonormal basis for the corresponding left and right eigenspaces
   (deflating subspaces).

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar w
   or a ratio alpha/beta = w, such that  A - w*B is singular.  It is
   usually represented as the pair (alpha,beta), as there is a
   reasonable interpretation for beta=0 or for both being zero.

   A pair of matrices (S,T) is in generalized real Schur form if T is
   upper triangular with non-negative diagonal and S is block upper
   triangular with 1-by-1 and 2-by-2 blocks.  1-by-1 blocks correspond
   to real generalized eigenvalues, while 2-by-2 blocks of S will be
   "standardized" by making the corresponding elements of T have the form:
          [  a  0  ]
          [  0  b  ]
   and the pair of corresponding 2-by-2 blocks in S and T will have a
   complex conjugate pair of generalized eigenvalues.

Parameters:

 JOBVSL
   JOBVSL is CHARACTER*1
   = 'N':  do not compute the left Schur vectors;
   = 'V':  compute the left Schur vectors.

 JOBVSR
   JOBVSR is CHARACTER*1
   = 'N':  do not compute the right Schur vectors;
```

```
  = 'V':  compute the right Schur vectors.

SORT
  SORT is CHARACTER*1
  Specifies whether or not to order the eigenvalues on the
  diagonal of the generalized Schur form.
  = 'N':  Eigenvalues are not ordered;
  = 'S':  Eigenvalues are ordered (see SELCTG).

SELCTG
  SELCTG is procedure) LOGICAL FUNCTION of three DOUBLE PRECISION arguments
  SELCTG must be declared EXTERNAL in the calling subroutine.
  If SORT = 'N', SELCTG is not referenced.
  If SORT = 'S', SELCTG is used to select eigenvalues to sort
  to the top left of the Schur form.
  An eigenvalue (ALPHAR(j)+ALPHAI(j))/BETA(j) is selected if
  SELCTG(ALPHAR(j),ALPHAI(j),BETA(j)) is true; i.e. if either
  one of a complex conjugate pair of eigenvalues is selected,
  then both complex eigenvalues are selected.
  Note that a selected complex eigenvalue may no longer satisfy
  SELCTG(ALPHAR(j),ALPHAI(j),BETA(j)) = .TRUE. after ordering,
  since ordering may change the value of complex eigenvalues
  (especially if the eigenvalue is ill-conditioned), in this
  case INFO is set to N+3.

SENSE
  SENSE is CHARACTER*1
  Determines which reciprocal condition numbers are computed.
  = 'N' : None are computed;
  = 'E' : Computed for average of selected eigenvalues only;
  = 'V' : Computed for selected deflating subspaces only;
  = 'B' : Computed for both.
  If SENSE = 'E', 'V', or 'B', SORT must equal 'S'.

N
  N is INTEGER
  The order of the matrices A, B, VSL, and VSR.  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA, N)
  On entry, the first of the pair of matrices.
  On exit, A has been overwritten by its generalized Schur form S.

LDA
  LDA is INTEGER
  The leading dimension of A.  LDA >= max(1,N).

B
  B is DOUBLE PRECISION array, dimension (LDB, N)
  On entry, the second of the pair of matrices.
```

```
  On exit, B has been overwritten by its generalized Schur form T.

LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).

SDIM
  SDIM is INTEGER
  If SORT = 'N', SDIM = 0.
  If SORT = 'S', SDIM = number of eigenvalues (after sorting)
  for which SELCTG is true.  (Complex conjugate pairs for which
  SELCTG is true for either eigenvalue count as 2.)

ALPHAR
  ALPHAR is DOUBLE PRECISION array, dimension (N)

ALPHAI
  ALPHAI is DOUBLE PRECISION array, dimension (N)

BETA
  BETA is DOUBLE PRECISION array, dimension (N)
  On exit, (ALPHAR(j) + ALPHAI(j)*i)/BETA(j), j=1,...,N, will
  be the generalized eigenvalues.  ALPHAR(j) + ALPHAI(j)*i
  and BETA(j),j=1,...,N  are the diagonals of the complex Schur
  form (S,T) that would result if the 2-by-2 diagonal blocks of
  the real Schur form of (A,B) were further reduced to
  triangular form using 2-by-2 complex unitary transformations.
  If ALPHAI(j) is zero, then the j-th eigenvalue is real; if
  positive, then the j-th and (j+1)-st eigenvalues are a
  complex conjugate pair, with ALPHAI(j+1) negative.

  Note: the quotients ALPHAR(j)/BETA(j) and ALPHAI(j)/BETA(j)
  may easily over- or underflow, and BETA(j) may even be zero.
  Thus, the user should avoid naively computing the ratio.
  However, ALPHAR and ALPHAI will be always less than and
  usually comparable with norm(A) in magnitude, and BETA always
  less than and usually comparable with norm(B).

VSL
  VSL is DOUBLE PRECISION array, dimension (LDVSL,N)
  If JOBVSL = 'V', VSL will contain the left Schur vectors.
  Not referenced if JOBVSL = 'N'.

LDVSL
  LDVSL is INTEGER
  The leading dimension of the matrix VSL. LDVSL >=1, and
  if JOBVSL = 'V', LDVSL >= N.

VSR
  VSR is DOUBLE PRECISION array, dimension (LDVSR,N)
```

```
  If JOBVSR = 'V', VSR will contain the right Schur vectors.
  Not referenced if JOBVSR = 'N'.

LDVSR
  LDVSR is INTEGER
  The leading dimension of the matrix VSR. LDVSR >= 1, and
  if JOBVSR = 'V', LDVSR >= N.

RCONDE
  RCONDE is DOUBLE PRECISION array, dimension ( 2 )
  If SENSE = 'E' or 'B', RCONDE(1) and RCONDE(2) contain the
  reciprocal condition numbers for the average of the selected eigenvalues.
  Not referenced if SENSE = 'N' or 'V'.

RCONDV
  RCONDV is DOUBLE PRECISION array, dimension ( 2 )
  If SENSE = 'V' or 'B', RCONDV(1) and RCONDV(2) contain the
  reciprocal condition numbers for the selected deflating subspaces.
  Not referenced if SENSE = 'N' or 'E'.

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  If N = 0, LWORK >= 1, else if SENSE = 'E', 'V', or 'B',
  LWORK >= max( 8*N, 6*N+16, 2*SDIM*(N-SDIM) ), else
  LWORK >= max( 8*N, 6*N+16 ).
  Note that 2*SDIM*(N-SDIM) <= N*N/2.
  Note also that an error is only returned if
  LWORK < max( 8*N, 6*N+16), but if SENSE = 'E' or 'V' or 'B'
  this may not be large enough.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the bound on the optimal size of the WORK
  array and the minimum size of the IWORK array, returns these
  values as the first entries of the WORK and IWORK arrays, and
  no error message related to LWORK or LIWORK is issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (MAX(1,LIWORK))
  On exit, if INFO = 0, IWORK(1) returns the minimum LIWORK.

LIWORK
  LIWORK is INTEGER
  The dimension of the array IWORK.
  If SENSE = 'N' or N = 0, LIWORK >= 1, otherwise
  LIWORK >= N+6.
```

```
   If LIWORK = -1, then a workspace query is assumed; the
   routine only calculates the bound on the optimal size of the
   WORK array and the minimum size of the IWORK array, returns
   these values as the first entries of the WORK and IWORK
   arrays, and no error message related to LWORK or LIWORK is issued by XERBLA.

 BWORK
   BWORK is LOGICAL array, dimension (N)
   Not referenced if SORT = 'N'.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value.
   = 1,...,N:
         The QZ iteration failed.  (A,B) are not in Schur
         form, but ALPHAR(j), ALPHAI(j), and BETA(j) should
         be correct for j=INFO+1,...,N.
   > N:  =N+1: other than QZ iteration failed in DHGEQZ
         =N+2: after reordering, roundoff changed values of
               some complex eigenvalues so that leading
               eigenvalues in the Generalized Schur form no
               longer satisfy SELCTG=.TRUE.  This could also
               be caused due to scaling.
         =N+3: reordering failed in DTGSEN.

Further Details:

   An approximate (asymptotic) bound on the average absolute error of
   the selected eigenvalues is

   EPS * norm((A, B)) / RCONDE( 1 ).

   An approximate (asymptotic) bound on the maximum angular error in
   the computed deflating subspaces is

   EPS * norm((A, B)) / RCONDV( 2 ).
```

———-

24.9.3 fortran code

— **dggesx.f** —

```
*  =====================================================================
      SUBROUTINE DGGESX( JOBVSL, JOBVSR, SORT, SELCTG, SENSE, N, A, LDA,
```

```
     $                   B, LDB, SDIM, ALPHAR, ALPHAI, BETA, VSL, LDVSL,
     $                   VSR, LDVSR, RCONDE, RCONDV, WORK, LWORK, IWORK,
     $                   LIWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVSL, JOBVSR, SENSE, SORT
      INTEGER            INFO, LDA, LDB, LDVSL, LDVSR, LIWORK, LWORK, N,
     $                   SDIM
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), ALPHAI( * ), ALPHAR( * ),
     $                   B( LDB, * ), BETA( * ), RCONDE( 2 ),
     $                   RCONDV( 2 ), VSL( LDVSL, * ), VSR( LDVSR, * ),
     $                   WORK( * )
*     ..
*     .. Function Arguments ..
      LOGICAL            SELCTG
      EXTERNAL           SELCTG
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            CURSL, ILASCL, ILBSCL, ILVSL, ILVSR, LASTSL,
     $                   LQUERY, LST2SL, WANTSB, WANTSE, WANTSN, WANTST,
     $                   WANTSV
      INTEGER            I, ICOLS, IERR, IHI, IJOB, IJOBVL, IJOBVR,
     $                   ILEFT, ILO, IP, IRIGHT, IROWS, ITAU, IWRK,
     $                   LIWMIN, LWRK, MAXWRK, MINWRK
      DOUBLE PRECISION   ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS, PL,
     $                   PR, SAFMAX, SAFMIN, SMLNUM
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   DIF( 2 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEQRF, DGGBAK, DGGBAL, DGGHRD, DHGEQZ, DLABAD,
     $                   DLACPY, DLASCL, DLASET, DORGQR, DORMQR, DTGSEN,
     $                   XERBLA
```

```
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           LSAME, ILAENV, DLAMCH, DLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVSL, 'N' ) ) THEN
         IJOBVL = 1
         ILVSL = .FALSE.
      ELSE IF( LSAME( JOBVSL, 'V' ) ) THEN
         IJOBVL = 2
         ILVSL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVSL = .FALSE.
      END IF
*
      IF( LSAME( JOBVSR, 'N' ) ) THEN
         IJOBVR = 1
         ILVSR = .FALSE.
      ELSE IF( LSAME( JOBVSR, 'V' ) ) THEN
         IJOBVR = 2
         ILVSR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVSR = .FALSE.
      END IF
*
      WANTST = LSAME( SORT, 'S' )
      WANTSN = LSAME( SENSE, 'N' )
      WANTSE = LSAME( SENSE, 'E' )
      WANTSV = LSAME( SENSE, 'V' )
      WANTSB = LSAME( SENSE, 'B' )
      LQUERY = ( LWORK.EQ.-1 .OR. LIWORK.EQ.-1 )
      IF( WANTSN ) THEN
         IJOB = 0
      ELSE IF( WANTSE ) THEN
         IJOB = 1
      ELSE IF( WANTSV ) THEN
         IJOB = 2
      ELSE IF( WANTSB ) THEN
         IJOB = 4
```

```
      END IF
*
*     Test the input arguments
*
      INFO = 0
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -2
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( .NOT.( WANTSN .OR. WANTSE .OR. WANTSV .OR. WANTSB ) .OR.
     $         ( .NOT.WANTST .AND. .NOT.WANTSN ) ) THEN
         INFO = -5
      ELSE IF( N.LT.0 ) THEN
         INFO = -6
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -10
      ELSE IF( LDVSL.LT.1 .OR. ( ILVSL .AND. LDVSL.LT.N ) ) THEN
         INFO = -16
      ELSE IF( LDVSR.LT.1 .OR. ( ILVSR .AND. LDVSR.LT.N ) ) THEN
         INFO = -18
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.GT.0) THEN
            MINWRK = MAX( 8*N, 6*N + 16 )
            MAXWRK = MINWRK - N +
     $               N*ILAENV( 1, 'DGEQRF', ' ', N, 1, N, 0 )
            MAXWRK = MAX( MAXWRK, MINWRK - N +
     $               N*ILAENV( 1, 'DORMQR', ' ', N, 1, N, -1 ) )
            IF( ILVSL ) THEN
               MAXWRK = MAX( MAXWRK, MINWRK - N +
     $                  N*ILAENV( 1, 'DORGQR', ' ', N, 1, N, -1 ) )
            END IF
            LWRK = MAXWRK
            IF( IJOB.GE.1 )
     $         LWRK = MAX( LWRK, N*N/2 )
         ELSE
            MINWRK = 1
            MAXWRK = 1
```

```
            LWRK   = 1
         END IF
         WORK( 1 ) = LWRK
         IF( WANTSN .OR. N.EQ.0 ) THEN
            LIWMIN = 1
         ELSE
            LIWMIN = N + 6
         END IF
         IWORK( 1 ) = LIWMIN
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -22
         ELSE IF( LIWORK.LT.LIWMIN  .AND. .NOT.LQUERY ) THEN
            INFO = -24
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGGESX', -INFO )
         RETURN
      ELSE IF (LQUERY) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SAFMIN = DLAMCH( 'S' )
      SAFMAX = ONE / SAFMIN
      CALL DLABAD( SAFMIN, SAFMAX )
      SMLNUM = SQRT( SAFMIN ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = DLANGE( 'M', N, N, A, LDA, WORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
```

```
      END IF
      IF( ILASCL )
     $   CALL DLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = DLANGE( 'M', N, N, B, LDB, WORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
     $   CALL DLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrix to make it more nearly triangular
*     (Workspace: need 6*N + 2*N for permutation parameters)
*
      ILEFT = 1
      IRIGHT = N + 1
      IWRK = IRIGHT + N
      CALL DGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, WORK( ILEFT ),
     $             WORK( IRIGHT ), WORK( IWRK ), IERR )
*
*     Reduce B to triangular form (QR decomposition of B)
*     (Workspace: need N, prefer N*NB)
*
      IROWS = IHI + 1 - ILO
      ICOLS = N + 1 - ILO
      ITAU = IWRK
      IWRK = ITAU + IROWS
      CALL DGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to matrix A
*     (Workspace: need N, prefer N*NB)
*
      CALL DORMQR( 'L', 'T', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VSL
*     (Workspace: need N, prefer N*NB)
*
      IF( ILVSL ) THEN
         CALL DLASET( 'Full', N, N, ZERO, ONE, VSL, LDVSL )
         IF( IROWS.GT.1 ) THEN
```

```
            CALL DLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VSL( ILO+1, ILO ), LDVSL )
         END IF
         CALL DORGQR( IROWS, IROWS, IROWS, VSL( ILO, ILO ), LDVSL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Initialize VSR
*
      IF( ILVSR )
     $   CALL DLASET( 'Full', N, N, ZERO, ONE, VSR, LDVSR )
*
*     Reduce to generalized Hessenberg form
*     (Workspace: none needed)
*
      CALL DGGHRD( JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB, VSL,
     $             LDVSL, VSR, LDVSR, IERR )
*
      SDIM = 0
*
*     Perform QZ algorithm, computing Schur vectors if desired
*     (Workspace: need N)
*
      IWRK = ITAU
      CALL DHGEQZ( 'S', JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR,
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 60
      END IF
*
*     Sort eigenvalues ALPHA/BETA and compute the reciprocal of
*     condition number(s)
*     (Workspace: If IJOB >= 1, need MAX( 8*(N+1), 2*SDIM*(N-SDIM) )
*                 otherwise, need 8*(N+1) )
*
      IF( WANTST ) THEN
*
*        Undo scaling on eigenvalues before SELCTGing
*
         IF( ILASCL ) THEN
            CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAR, N,
     $                   IERR )
```

```
            CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAI, N,
     $                   IERR )
         END IF
         IF( ILBSCL )
     $      CALL DLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
*
*        Select eigenvalues
*
         DO 10 I = 1, N
            BWORK( I ) = SELCTG( ALPHAR( I ), ALPHAI( I ), BETA( I ) )
   10    CONTINUE
*
*        Reorder eigenvalues, transform Generalized Schur vectors, and
*        compute reciprocal condition numbers
*
         CALL DTGSEN( IJOB, ILVSL, ILVSR, BWORK, N, A, LDA, B, LDB,
     $                ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR,
     $                SDIM, PL, PR, DIF, WORK( IWRK ), LWORK-IWRK+1,
     $                IWORK, LIWORK, IERR )
*
         IF( IJOB.GE.1 )
     $      MAXWRK = MAX( MAXWRK, 2*SDIM*( N-SDIM ) )
         IF( IERR.EQ.-22 ) THEN
*
*            not enough real workspace
*
            INFO = -22
         ELSE
            IF( IJOB.EQ.1 .OR. IJOB.EQ.4 ) THEN
               RCONDE( 1 ) = PL
               RCONDE( 2 ) = PR
            END IF
            IF( IJOB.EQ.2 .OR. IJOB.EQ.4 ) THEN
               RCONDV( 1 ) = DIF( 1 )
               RCONDV( 2 ) = DIF( 2 )
            END IF
            IF( IERR.EQ.1 )
     $         INFO = N + 3
         END IF
*
      END IF
*
*     Apply permutation to VSL and VSR
*     (Workspace: none needed)
*
      IF( ILVSL )
     $   CALL DGGBAK( 'P', 'L', N, ILO, IHI, WORK( ILEFT ),
     $                WORK( IRIGHT ), N, VSL, LDVSL, IERR )
*
      IF( ILVSR )
```

```
     $   CALL DGGBAK( 'P', 'R', N, ILO, IHI, WORK( ILEFT ),
     $                WORK( IRIGHT ), N, VSR, LDVSR, IERR )
*
*     Check if unscaling would cause over/underflow, if so, rescale
*     (ALPHAR(I),ALPHAI(I),BETA(I)) so BETA(I) is on the order of
*     B(I,I) and ALPHAR(I) and ALPHAI(I) are on the order of A(I,I)
*
      IF( ILASCL ) THEN
         DO 20 I = 1, N
            IF( ALPHAI( I ).NE.ZERO ) THEN
               IF( ( ALPHAR( I ) / SAFMAX ).GT.( ANRMTO / ANRM ) .OR.
     $             ( SAFMIN / ALPHAR( I ) ).GT.( ANRM / ANRMTO ) ) THEN
                  WORK( 1 ) = ABS( A( I, I ) / ALPHAR( I ) )
                  BETA( I ) = BETA( I )*WORK( 1 )
                  ALPHAR( I ) = ALPHAR( I )*WORK( 1 )
                  ALPHAI( I ) = ALPHAI( I )*WORK( 1 )
               ELSE IF( ( ALPHAI( I ) / SAFMAX ).GT.
     $                  ( ANRMTO / ANRM ) .OR.
     $                  ( SAFMIN / ALPHAI( I ) ).GT.( ANRM / ANRMTO ) )
     $                   THEN
                  WORK( 1 ) = ABS( A( I, I+1 ) / ALPHAI( I ) )
                  BETA( I ) = BETA( I )*WORK( 1 )
                  ALPHAR( I ) = ALPHAR( I )*WORK( 1 )
                  ALPHAI( I ) = ALPHAI( I )*WORK( 1 )
               END IF
            END IF
   20    CONTINUE
      END IF
*
      IF( ILBSCL ) THEN
         DO 30 I = 1, N
            IF( ALPHAI( I ).NE.ZERO ) THEN
               IF( ( BETA( I ) / SAFMAX ).GT.( BNRMTO / BNRM ) .OR.
     $             ( SAFMIN / BETA( I ) ).GT.( BNRM / BNRMTO ) ) THEN
                  WORK( 1 ) = ABS( B( I, I ) / BETA( I ) )
                  BETA( I ) = BETA( I )*WORK( 1 )
                  ALPHAR( I ) = ALPHAR( I )*WORK( 1 )
                  ALPHAI( I ) = ALPHAI( I )*WORK( 1 )
               END IF
            END IF
   30    CONTINUE
      END IF
*
*     Undo scaling
*
      IF( ILASCL ) THEN
         CALL DLASCL( 'H', 0, 0, ANRMTO, ANRM, N, N, A, LDA, IERR )
         CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAR, N, IERR )
         CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAI, N, IERR )
      END IF
```

```
*
      IF( ILBSCL ) THEN
         CALL DLASCL( 'U', 0, 0, BNRMTO, BNRM, N, N, B, LDB, IERR )
         CALL DLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
      END IF
*
      IF( WANTST ) THEN
*
*        Check if reordering is correct
*
         LASTSL = .TRUE.
         LST2SL = .TRUE.
         SDIM = 0
         IP = 0
         DO 50 I = 1, N
            CURSL = SELCTG( ALPHAR( I ), ALPHAI( I ), BETA( I ) )
            IF( ALPHAI( I ).EQ.ZERO ) THEN
               IF( CURSL )
     $            SDIM = SDIM + 1
               IP = 0
               IF( CURSL .AND. .NOT.LASTSL )
     $            INFO = N + 2
            ELSE
               IF( IP.EQ.1 ) THEN
*
*                Last eigenvalue of conjugate pair
*
                 CURSL = CURSL .OR. LASTSL
                 LASTSL = CURSL
                 IF( CURSL )
     $              SDIM = SDIM + 2
                 IP = -1
                 IF( CURSL .AND. .NOT.LST2SL )
     $              INFO = N + 2
               ELSE
*
*                First eigenvalue of conjugate pair
*
                 IP = 1
               END IF
            END IF
            LST2SL = LASTSL
            LASTSL = CURSL
   50    CONTINUE
*
      END IF
*
   60 CONTINUE
*
      WORK( 1 ) = MAXWRK
```

```
      IWORK( 1 ) = LIWMIN
*
      RETURN
*
*     End of DGGESX
*
      END
```

———-

24.9.4 lisp code

— **dggesx.lisp** —

———-

24.10 dggev

24.10.1 Axiom unit tests

— **dggev.input** —

```
)set break resume
)sys rm -f dggev.output
)spool dggev.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

24.10.2 Axiom help page

— **dggev.help** —

```
=====================================================================
dggev examples
```

```
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dggev (JOBVL, JOBVR, N, A, LDA, B, LDB, ALPHAR, ALPHAI,
           BETA, VL, LDVL, VR, LDVR, WORK, LWORK, INFO)
            DGGEV computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for GE matrices

Function/Subroutine Documentation
   subroutine dggev (character JOBVL, character JOBVR, integer N, double
       precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldb, * ) B, integer LDB, double precision, dimension( * )
       ALPHAR, double precision, dimension( * ) ALPHAI, double precision,
       dimension( * ) BETA, double precision, dimension( ldvl, * ) VL, integer
       LDVL, double precision, dimension( ldvr, * ) VR, integer LDVR, double
       precision, dimension( * ) WORK, integer LWORK, integer INFO)
        DGGEV computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for GE matrices

Purpose:
   DGGEV computes for a pair of N-by-N real nonsymmetric matrices (A,B)
   the generalized eigenvalues, and optionally, the left and/or right
   generalized eigenvectors.

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar
   lambda or a ratio alpha/beta = lambda, such that A - lambda*B is
   singular. It is usually represented as the pair (alpha,beta), as
   there is a reasonable interpretation for beta=0, and even for both
   being zero.

   The right eigenvector v(j) corresponding to the eigenvalue lambda(j)
   of (A,B) satisfies

           A * v(j) = lambda(j) * B * v(j).

   The left eigenvector u(j) corresponding to the eigenvalue lambda(j)
   of (A,B) satisfies

           u(j)**H * A  = lambda(j) * u(j)**H * B .

   where u(j)**H is the conjugate-transpose of u(j).

Parameters:

 JOBVL
```

```
  JOBVL is CHARACTER*1
  = 'N':  do not compute the left generalized eigenvectors;
  = 'V':  compute the left generalized eigenvectors.

JOBVR
  JOBVR is CHARACTER*1
  = 'N':  do not compute the right generalized eigenvectors;
  = 'V':  compute the right generalized eigenvectors.

N
  N is INTEGER
  The order of the matrices A, B, VL, and VR.  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA, N)
  On entry, the matrix A in the pair (A,B).
  On exit, A has been overwritten.

LDA
  LDA is INTEGER
  The leading dimension of A.  LDA >= max(1,N).

B
  B is DOUBLE PRECISION array, dimension (LDB, N)
  On entry, the matrix B in the pair (A,B).
  On exit, B has been overwritten.

LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).

ALPHAR
  ALPHAR is DOUBLE PRECISION array, dimension (N)

ALPHAI
  ALPHAI is DOUBLE PRECISION array, dimension (N)

BETA
  BETA is DOUBLE PRECISION array, dimension (N)
  On exit, (ALPHAR(j) + ALPHAI(j)*i)/BETA(j), j=1,...,N, will
  be the generalized eigenvalues.  If ALPHAI(j) is zero, then
  the j-th eigenvalue is real; if positive, then the j-th and
  (j+1)-st eigenvalues are a complex conjugate pair, with
  ALPHAI(j+1) negative.

  Note: the quotients ALPHAR(j)/BETA(j) and ALPHAI(j)/BETA(j)
  may easily over- or underflow, and BETA(j) may even be zero.
  Thus, the user should avoid naively computing the ratio
  alpha/beta.  However, ALPHAR and ALPHAI will be always less
  than and usually comparable with norm(A) in magnitude, and
```

```
  BETA always less than and usually comparable with norm(B).

VL
  VL is DOUBLE PRECISION array, dimension (LDVL,N)
  If JOBVL = 'V', the left eigenvectors u(j) are stored one
  after another in the columns of VL, in the same order as
  their eigenvalues. If the j-th eigenvalue is real, then
  u(j) = VL(:,j), the j-th column of VL. If the j-th and
  (j+1)-th eigenvalues form a complex conjugate pair, then
  u(j) = VL(:,j)+i*VL(:,j+1) and u(j+1) = VL(:,j)-i*VL(:,j+1).
  Each eigenvector is scaled so the largest component has
  abs(real part)+abs(imag. part)=1.
  Not referenced if JOBVL = 'N'.

LDVL
  LDVL is INTEGER
  The leading dimension of the matrix VL. LDVL >= 1, and
  if JOBVL = 'V', LDVL >= N.

VR
  VR is DOUBLE PRECISION array, dimension (LDVR,N)
  If JOBVR = 'V', the right eigenvectors v(j) are stored one
  after another in the columns of VR, in the same order as
  their eigenvalues. If the j-th eigenvalue is real, then
  v(j) = VR(:,j), the j-th column of VR. If the j-th and
  (j+1)-th eigenvalues form a complex conjugate pair, then
  v(j) = VR(:,j)+i*VR(:,j+1) and v(j+1) = VR(:,j)-i*VR(:,j+1).
  Each eigenvector is scaled so the largest component has
  abs(real part)+abs(imag. part)=1.
  Not referenced if JOBVR = 'N'.

LDVR
  LDVR is INTEGER
  The leading dimension of the matrix VR. LDVR >= 1, and
  if JOBVR = 'V', LDVR >= N.

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,8*N).
  For good performance, LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.
```

```
 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value.
   = 1,...,N:
         The QZ iteration failed.  No eigenvectors have been
         calculated, but ALPHAR(j), ALPHAI(j), and BETA(j)
         should be correct for j=INFO+1,...,N.
   > N:  =N+1: other than QZ iteration failed in DHGEQZ.
         =N+2: error return from DTGEVC.
```

———-

24.10.3 fortran code

— dggev.f —

```
*  =====================================================================
      SUBROUTINE DGGEV( JOBVL, JOBVR, N, A, LDA, B, LDB, ALPHAR, ALPHAI,
     $                  BETA, VL, LDVL, VR, LDVR, WORK, LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVL, JOBVR
      INTEGER            INFO, LDA, LDB, LDVL, LDVR, LWORK, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), ALPHAI( * ), ALPHAR( * ),
     $                   B( LDB, * ), BETA( * ), VL( LDVL, * ),
     $                   VR( LDVR, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ILASCL, ILBSCL, ILV, ILVL, ILVR, LQUERY
      CHARACTER          CHTEMP
      INTEGER            ICOLS, IERR, IHI, IJOBVL, IJOBVR, ILEFT, ILO,
     $                   IN, IRIGHT, IROWS, ITAU, IWRK, JC, JR, MAXWRK,
```

```
     $                   MINWRK
      DOUBLE PRECISION   ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS,
     $                   SMLNUM, TEMP
*     ..
*     .. Local Arrays ..
      LOGICAL            LDUMMA( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEQRF, DGGBAK, DGGBAL, DGGHRD, DHGEQZ, DLABAD,
     $                   DLACPY,DLASCL, DLASET, DORGQR, DORMQR, DTGEVC,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           LSAME, ILAENV, DLAMCH, DLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVL, 'N' ) ) THEN
         IJOBVL = 1
         ILVL = .FALSE.
      ELSE IF( LSAME( JOBVL, 'V' ) ) THEN
         IJOBVL = 2
         ILVL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVL = .FALSE.
      END IF
*
      IF( LSAME( JOBVR, 'N' ) ) THEN
         IJOBVR = 1
         ILVR = .FALSE.
      ELSE IF( LSAME( JOBVR, 'V' ) ) THEN
         IJOBVR = 2
         ILVR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVR = .FALSE.
      END IF
      ILV = ILVL .OR. ILVR
*
*     Test the input arguments
*
```

```
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDVL.LT.1 .OR. ( ILVL .AND. LDVL.LT.N ) ) THEN
         INFO = -12
      ELSE IF( LDVR.LT.1 .OR. ( ILVR .AND. LDVR.LT.N ) ) THEN
         INFO = -14
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV. The workspace is
*       computed assuming ILO = 1 and IHI = N, the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = MAX( 1, 8*N )
         MAXWRK = MAX( 1, N*( 7 +
     $                 ILAENV( 1, 'DGEQRF', ' ', N, 1, N, 0 ) ) )
         MAXWRK = MAX( MAXWRK, N*( 7 +
     $                 ILAENV( 1, 'DORMQR', ' ', N, 1, N, 0 ) ) )
         IF( ILVL ) THEN
            MAXWRK = MAX( MAXWRK, N*( 7 +
     $                 ILAENV( 1, 'DORGQR', ' ', N, 1, N, -1 ) ) )
         END IF
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY )
     $      INFO = -16
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGGEV ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
```

```
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = DLANGE( 'M', N, N, A, LDA, WORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
      IF( ILASCL )
     $   CALL DLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = DLANGE( 'M', N, N, B, LDB, WORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
     $   CALL DLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrices A, B to isolate eigenvalues if possible
*     (Workspace: need 6*N)
*
      ILEFT = 1
      IRIGHT = N + 1
      IWRK = IRIGHT + N
      CALL DGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, WORK( ILEFT ),
     $             WORK( IRIGHT ), WORK( IWRK ), IERR )
*
```

```
*     Reduce B to triangular form (QR decomposition of B)
*     (Workspace: need N, prefer N*NB)
*
      IROWS = IHI + 1 - ILO
      IF( ILV ) THEN
         ICOLS = N + 1 - ILO
      ELSE
         ICOLS = IROWS
      END IF
      ITAU = IWRK
      IWRK = ITAU + IROWS
      CALL DGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to matrix A
*     (Workspace: need N, prefer N*NB)
*
      CALL DORMQR( 'L', 'T', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VL
*     (Workspace: need N, prefer N*NB)
*
      IF( ILVL ) THEN
         CALL DLASET( 'Full', N, N, ZERO, ONE, VL, LDVL )
         IF( IROWS.GT.1 ) THEN
            CALL DLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VL( ILO+1, ILO ), LDVL )
         END IF
         CALL DORGQR( IROWS, IROWS, IROWS, VL( ILO, ILO ), LDVL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Initialize VR
*
      IF( ILVR )
     $   CALL DLASET( 'Full', N, N, ZERO, ONE, VR, LDVR )
*
*     Reduce to generalized Hessenberg form
*     (Workspace: none needed)
*
      IF( ILV ) THEN
*
*        Eigenvectors requested -- work on whole matrix.
*
         CALL DGGHRD( JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB, VL,
     $                LDVL, VR, LDVR, IERR )
      ELSE
         CALL DGGHRD( 'N', 'N', IROWS, 1, IROWS, A( ILO, ILO ), LDA,
```

```
     $                B( ILO, ILO ), LDB, VL, LDVL, VR, LDVR, IERR )
      END IF
*
*     Perform QZ algorithm (Compute eigenvalues, and optionally, the
*     Schur forms and Schur vectors)
*     (Workspace: need N)
*
      IWRK = ITAU
      IF( ILV ) THEN
         CHTEMP = 'S'
      ELSE
         CHTEMP = 'E'
      END IF
      CALL DHGEQZ( CHTEMP, JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHAR, ALPHAI, BETA, VL, LDVL, VR, LDVR,
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 110
      END IF
*
*     Compute Eigenvectors
*     (Workspace: need 6*N)
*
      IF( ILV ) THEN
         IF( ILVL ) THEN
            IF( ILVR ) THEN
               CHTEMP = 'B'
            ELSE
               CHTEMP = 'L'
            END IF
         ELSE
            CHTEMP = 'R'
         END IF
         CALL DTGEVC( CHTEMP, 'B', LDUMMA, N, A, LDA, B, LDB, VL, LDVL,
     $                VR, LDVR, N, IN, WORK( IWRK ), IERR )
         IF( IERR.NE.0 ) THEN
            INFO = N + 2
            GO TO 110
         END IF
*
*        Undo balancing on VL and VR and normalization
*        (Workspace: none needed)
*
```

```
      IF( ILVL ) THEN
         CALL DGGBAK( 'P', 'L', N, ILO, IHI, WORK( ILEFT ),
  $                   WORK( IRIGHT ), N, VL, LDVL, IERR )
         DO 50 JC = 1, N
            IF( ALPHAI( JC ).LT.ZERO )
  $            GO TO 50
            TEMP = ZERO
            IF( ALPHAI( JC ).EQ.ZERO ) THEN
               DO 10 JR = 1, N
                  TEMP = MAX( TEMP, ABS( VL( JR, JC ) ) )
10             CONTINUE
            ELSE
               DO 20 JR = 1, N
                  TEMP = MAX( TEMP, ABS( VL( JR, JC ) )+
  $                       ABS( VL( JR, JC+1 ) ) )
20             CONTINUE
            END IF
            IF( TEMP.LT.SMLNUM )
  $            GO TO 50
            TEMP = ONE / TEMP
            IF( ALPHAI( JC ).EQ.ZERO ) THEN
               DO 30 JR = 1, N
                  VL( JR, JC ) = VL( JR, JC )*TEMP
30             CONTINUE
            ELSE
               DO 40 JR = 1, N
                  VL( JR, JC ) = VL( JR, JC )*TEMP
                  VL( JR, JC+1 ) = VL( JR, JC+1 )*TEMP
40             CONTINUE
            END IF
50       CONTINUE
      END IF
      IF( ILVR ) THEN
         CALL DGGBAK( 'P', 'R', N, ILO, IHI, WORK( ILEFT ),
  $                   WORK( IRIGHT ), N, VR, LDVR, IERR )
         DO 100 JC = 1, N
            IF( ALPHAI( JC ).LT.ZERO )
  $            GO TO 100
            TEMP = ZERO
            IF( ALPHAI( JC ).EQ.ZERO ) THEN
               DO 60 JR = 1, N
                  TEMP = MAX( TEMP, ABS( VR( JR, JC ) ) )
60             CONTINUE
            ELSE
               DO 70 JR = 1, N
                  TEMP = MAX( TEMP, ABS( VR( JR, JC ) )+
  $                       ABS( VR( JR, JC+1 ) ) )
70             CONTINUE
            END IF
            IF( TEMP.LT.SMLNUM )
```

```
     $            GO TO 100
               TEMP = ONE / TEMP
               IF( ALPHAI( JC ).EQ.ZERO ) THEN
                  DO 80 JR = 1, N
                     VR( JR, JC ) = VR( JR, JC )*TEMP
   80             CONTINUE
               ELSE
                  DO 90 JR = 1, N
                     VR( JR, JC ) = VR( JR, JC )*TEMP
                     VR( JR, JC+1 ) = VR( JR, JC+1 )*TEMP
   90             CONTINUE
               END IF
  100       CONTINUE
         END IF
*
*        End of eigenvector calculation
*
      END IF
*
*     Undo scaling if necessary
*
  110 CONTINUE
*
      IF( ILASCL ) THEN
         CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAR, N, IERR )
         CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAI, N, IERR )
      END IF
*
      IF( ILBSCL ) THEN
         CALL DLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of DGGEV
*
      END
```

———-

24.10.4 lisp code

— dggev.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
```

```
          (ignorable zero one))
 (defun dggev
        (jobvl jobvr n a lda b ldb$ alphar alphai beta vl ldvl vr ldvr work
         lwork info)
   (declare (type (array double-float (*)) work vr vl beta alphai alphar b a)
            (type (f2cl-lib:integer4) info lwork ldvr ldvl ldb$ lda n)
            (type (string *) jobvr jobvl))
   (f2cl-lib:with-multi-array-data
       ((jobvl character jobvl-%data% jobvl-%offset%)
        (jobvr character jobvr-%data% jobvr-%offset%)
        (a double-float a-%data% a-%offset%)
        (b double-float b-%data% b-%offset%)
        (alphar double-float alphar-%data% alphar-%offset%)
        (alphai double-float alphai-%data% alphai-%offset%)
        (beta double-float beta-%data% beta-%offset%)
        (vl double-float vl-%data% vl-%offset%)
        (vr double-float vr-%data% vr-%offset%)
        (work double-float work-%data% work-%offset%))
     (prog ((ldumma (make-array 1 :element-type 't)) (anrm 0.0d0)
            (anrmto 0.0d0) (bignum 0.0d0) (bnrm 0.0d0) (bnrmto 0.0d0)
            (eps 0.0d0) (smlnum 0.0d0) (temp 0.0d0) (icols 0) (ierr 0) (ihi 0)
            (ijobvl 0) (ijobvr 0) (ileft 0) (ilo 0) (in 0) (iright 0)
            (irows 0) (itau 0) (iwrk 0) (jc 0) (jr 0) (maxwrk 0) (minwrk 0)
            (chtemp
             (make-array '(1) :element-type 'character :initial-element #\ ))
            (ilascl nil) (ilbscl nil) (ilv nil) (ilvl nil) (ilvr nil)
            (lquery nil))
       (declare (type (array f2cl-lib:logical (1)) ldumma)
                (type (double-float) anrm anrmto bignum bnrm bnrmto eps smlnum
                 temp)
                (type (f2cl-lib:integer4) icols ierr ihi ijobvl ijobvr ileft
                 ilo in iright irows itau iwrk jc jr maxwrk minwrk)
                (type (string 1) chtemp)
                (type f2cl-lib:logical ilascl ilbscl ilv ilvl ilvr lquery))
       (cond
        ((multiple-value-bind (ret-val var-0 var-1)
             (lsame jobvl "N")
           (declare (ignore var-1))
           (when var-0 (setf jobvl var-0))
           ret-val)
         (setf ijobvl 1) (setf ilvl f2cl-lib:%false%))
        ((multiple-value-bind (ret-val var-0 var-1)
             (lsame jobvl "V")
           (declare (ignore var-1))
           (when var-0 (setf jobvl var-0))
           ret-val)
         (setf ijobvl 2) (setf ilvl f2cl-lib:%true%))
        (t (setf ijobvl -1) (setf ilvl f2cl-lib:%false%)))
       (cond
        ((multiple-value-bind (ret-val var-0 var-1)
```

```
      (lsame jobvr "N")
    (declare (ignore var-1))
    (when var-0 (setf jobvr var-0))
    ret-val)
  (setf ijobvr 1) (setf ilvr f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvr "V")
    (declare (ignore var-1))
    (when var-0 (setf jobvr var-0))
    ret-val)
  (setf ijobvr 2) (setf ilvr f2cl-lib:%true%))
 (t (setf ijobvr -1) (setf ilvr f2cl-lib:%false%)))
(setf ilv (or ilvl ilvr))
(setf info 0)
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((<= ijobvl 0) (setf info -1)) ((<= ijobvr 0) (setf info -2))
      ((< n 0) (setf info -3))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -5))
      ((< ldb$
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -7))
      ((or (< ldvl 1) (and ilvl (< ldvl n))) (setf info -12))
      ((or (< ldvr 1) (and ilvr (< ldvr n))) (setf info -14)))
(cond
 ((= info 0)
  (setf minwrk
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul 8 n))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-mul n
                                      (f2cl-lib:int-add 7
                                                        (multiple-value-bind
                                                            (ret-val
                                                             var-0
                                                             var-1
                                                             var-2
                                                             var-3
                                                             var-4
                                                             var-5
                                                             var-6)
                                                            (ilaenv 1
                                                             "DGEQRF"
                                                             " " n 1 n
                                                             0)
                                                          (declare
                                                           (ignore
```

```
                                                                  var-0 var-1
                                                                  var-2 var-4
                                                                  var-6))
                                                                (when var-3
                                                                  (setf n
                                                                          var-3))
                                                                (when var-5
                                                                  (setf n
                                                                          var-5))
                                                                ret-val))))))
            (setf maxwrk
                    (max (the f2cl-lib:integer4 maxwrk)
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-mul n
                                               (f2cl-lib:int-add 7
                                                                (multiple-value-bind
                                                                   (ret-val
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-3
                                                                    var-4
                                                                    var-5
                                                                    var-6)
                                                                   (ilaenv 1
                                                                    "DORMQR"
                                                                    " " n 1 n
                                                                    0)
                                                                (declare
                                                                 (ignore
                                                                  var-0 var-1
                                                                  var-2 var-4
                                                                  var-6))
                                                                (when var-3
                                                                  (setf n
                                                                          var-3))
                                                                (when var-5
                                                                  (setf n
                                                                          var-5))
                                                                ret-val))))))
            (cond
             (ilvl
              (setf maxwrk
                      (max (the f2cl-lib:integer4 maxwrk)
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-mul n
                                                 (f2cl-lib:int-add 7
                                                                  (multiple-value-bind
                                                                     (ret-val
                                                                      var-0
```

```
                                                                    var-1
                                                                    var-2
                                                                    var-3
                                                                    var-4
                                                                    var-5
                                                                    var-6)
                                                                   (ilaenv 1
                                                                    "DORGQR"
                                                                    " " n 1
                                                                    n -1)
                                                                 (declare
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-4
                                                                   var-6))
                                                                 (when var-3
                                                                   (setf n
                                                                           var-3))
                                                                 (when var-5
                                                                   (setf n
                                                                           var-5))
                                                                ret-val))))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 maxwrk) 'double-float))
  (if (and (< lwork minwrk) (not lquery))
      (setf info -16))))
(cond
 ((/= info 0) (xerbla "DGGEV " (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(setf eps (dlamch "P"))
(setf smlnum (dlamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "M" n n a lda work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
```

```
          (when var-4 (setf lda var-4))
          ret-val))
(setf ilascl f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf anrmto smlnum)
  (setf ilascl f2cl-lib:%true%))
 ((> anrm bignum) (setf anrmto bignum) (setf ilascl f2cl-lib:%true%)))
(if ilascl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 anrm anrmto n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf anrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "M" n n b ldb$ work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ilbscl f2cl-lib:%false%)
(cond
 ((and (> bnrm zero) (< bnrm smlnum)) (setf bnrmto smlnum)
  (setf ilbscl f2cl-lib:%true%))
 ((> bnrm bignum) (setf bnrmto bignum) (setf ilbscl f2cl-lib:%true%)))
(if ilbscl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 bnrm bnrmto n n b ldb$ ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bnrm var-3))
      (when var-4 (setf bnrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf ierr var-9))))
(setf ileft 1)
(setf iright (f2cl-lib:int-add n 1))
(setf iwrk (f2cl-lib:int-add iright n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11)
    (dggbal "P" n a lda b ldb$ ilo ihi
```

```
     (f2cl-lib:array-slice work-%data% double-float (ileft) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iright) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                           work-%offset%)
     ierr)
  (declare (ignore var-0 var-1 var-2 var-4 var-8 var-9 var-10))
  (setf lda var-3)
  (setf ldb$ var-5)
  (setf ilo var-6)
  (setf ihi var-7)
  (setf ierr var-11))
(setf irows (f2cl-lib:int-sub (f2cl-lib:int-add ihi 1) ilo))
(cond (ilv (setf icols (f2cl-lib:int-sub (f2cl-lib:int-add n 1) ilo)))
      (t (setf icols irows)))
(setf itau iwrk)
(setf iwrk (f2cl-lib:int-add itau irows))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dgeqrf irows icols
     (f2cl-lib:array-slice b-%data% double-float (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf irows var-0)
  (setf icols var-1)
  (setf ldb$ var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (dormqr "L" "T" irows icols irows
     (f2cl-lib:array-slice b-%data% double-float (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice a-%data% double-float (ilo ilo)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf irows var-2))
```

```
  (when var-3 (setf icols var-3))
  (when var-4 (setf irows var-4))
  (when var-6 (setf ldb$ var-6))
  (when var-9 (setf lda var-9))
  (when var-12 (setf ierr var-12)))
(cond
 (ilvl
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "Full" n n zero one vl ldvl)
    (declare (ignore var-0 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldvl var-6)))
  (cond
   ((> irows 1)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "L" (f2cl-lib:int-sub irows 1)
         (f2cl-lib:int-sub irows 1)
         (f2cl-lib:array-slice b-%data% double-float ((+ ilo 1) ilo)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice vl-%data% double-float ((+ ilo 1) ilo)
                               ((1 ldvl) (1 *)) vl-%offset%)
         ldvl)
      (declare (ignore var-0 var-1 var-2 var-3 var-5))
      (when var-4 (setf ldb$ var-4))
      (when var-6 (setf ldvl var-6)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dorgqr irows irows irows
       (f2cl-lib:array-slice vl-%data% double-float (ilo ilo)
                             ((1 ldvl) (1 *)) vl-%offset%)
       ldvl
       (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf irows var-0))
    (when var-1 (setf irows var-1))
    (when var-2 (setf irows var-2))
    (when var-4 (setf ldvl var-4))
    (when var-8 (setf ierr var-8)))))
(if ilvr
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "Full" n n zero one vr ldvr)
      (declare (ignore var-0 var-5))
```

```
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldvr var-6))))
(cond
 (ilv
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dgghrd jobvl jobvr n ilo ihi a lda b ldb$ vl ldvl vr ldvr ierr)
    (declare (ignore var-5 var-7 var-9 var-11))
    (when var-0 (setf jobvl var-0))
    (when var-1 (setf jobvr var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-13 (setf ierr var-13))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dgghrd "N" "N" irows 1 irows
       (f2cl-lib:array-slice a-%data% double-float (ilo ilo)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice b-%data% double-float (ilo ilo)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$ vl ldvl vr ldvr ierr)
    (declare (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11))
    (when var-2 (setf irows var-2))
    (when var-4 (setf irows var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-13 (setf ierr var-13)))))
(setf iwrk itau)
(cond (ilv (f2cl-lib:f2cl-set-string chtemp "S" (string 1)))
      (t (f2cl-lib:f2cl-set-string chtemp "E" (string 1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19)
    (dhgeqz chtemp jobvl jobvr n ilo ihi a lda b ldb$ alphar alphai
     beta vl ldvl vr ldvr
     (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
```

```
                          work-%offset%)
      (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare
   (ignore var-6 var-8 var-10 var-11 var-12 var-13 var-15 var-17
    var-18))
  (when var-0 (setf chtemp var-0))
  (when var-1 (setf jobvl var-1))
  (when var-2 (setf jobvr var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf ilo var-4))
  (when var-5 (setf ihi var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldb$ var-9))
  (when var-14 (setf ldvl var-14))
  (when var-16 (setf ldvr var-16))
  (when var-19 (setf ierr var-19)))
(cond
 ((/= ierr 0)
  (cond ((and (> ierr 0) (<= ierr n)) (setf info ierr))
        ((and (> ierr n) (<= ierr (f2cl-lib:int-mul 2 n)))
         (setf info (f2cl-lib:int-sub ierr n)))
        (t (setf info (f2cl-lib:int-add n 1))))
  (go label110)))
(cond
 (ilv
  (cond
   (ilvl
    (cond (ilvr (f2cl-lib:f2cl-set-string chtemp "B" (string 1)))
          (t (f2cl-lib:f2cl-set-string chtemp "L" (string 1)))))
   (t (f2cl-lib:f2cl-set-string chtemp "R" (string 1))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (dtgevc chtemp "B" ldumma n a lda b ldb$ vl ldvl vr ldvr n in
       (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                             work-%offset%)
       ierr)
    (declare (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-14))
    (when var-0 (setf chtemp var-0))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldb$ var-7))
    (when var-9 (setf ldvl var-9))
    (when var-11 (setf ldvr var-11))
    (when var-12 (setf n var-12))
    (when var-13 (setf in var-13))
    (when var-15 (setf ierr var-15)))
  (cond ((/= ierr 0) (setf info (f2cl-lib:int-add n 2)) (go label110)))
  (cond
   (ilvl
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dggbak "P" "L" n ilo ihi
     (f2cl-lib:array-slice work-%data% double-float (ileft) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iright)
                           ((1 *)) work-%offset%)
     n vl ldvl ierr)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
  (setf n var-7)
  (setf ldvl var-9)
  (setf ierr var-10))
(f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
              ((> jc n) nil)
  (tagbody
    (if (<
         (f2cl-lib:fref alphai-%data% (jc) ((1 *)) alphai-%offset%)
         zero)
        (go label50))
    (setf temp zero)
    (cond
     ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf temp
                  (max temp
                       (abs
                        (f2cl-lib:fref vl-%data% (jr jc)
                                       ((1 ldvl) (1 *))
                                       vl-%offset%))))
         label10)))
     (t
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf temp
                  (max temp
                       (+
                        (abs
                         (f2cl-lib:fref vl-%data% (jr jc)
                                        ((1 ldvl) (1 *))
                                        vl-%offset%))
                        (abs
                         (f2cl-lib:fref vl-%data%
                                        (jr
                                         (f2cl-lib:int-add jc 1))
                                        ((1 ldvl) (1 *))
```

```
                                        vl-%offset%)))))
            label20))))
        (if (< temp smlnum)
            (go label50))
        (setf temp (/ one temp))
        (cond
         ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                        ((> jr n) nil)
            (tagbody
              (setf (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                                   vl-%offset%)
                    (*
                     (f2cl-lib:fref vl-%data% (jr jc)
                                    ((1 ldvl) (1 *)) vl-%offset%)
                     temp))
            label30)))
         (t
          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                        ((> jr n) nil)
            (tagbody
              (setf (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                                   vl-%offset%)
                    (*
                     (f2cl-lib:fref vl-%data% (jr jc)
                                    ((1 ldvl) (1 *)) vl-%offset%)
                     temp))
              (setf (f2cl-lib:fref vl-%data%
                                   (jr (f2cl-lib:int-add jc 1))
                                   ((1 ldvl) (1 *)) vl-%offset%)
                    (*
                     (f2cl-lib:fref vl-%data%
                                    (jr (f2cl-lib:int-add jc 1))
                                    ((1 ldvl) (1 *)) vl-%offset%)
                     temp))
            label40))))
    label50))))
(cond
 (ilvr
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dggbak "P" "R" n ilo ihi
       (f2cl-lib:array-slice work-%data% double-float (ileft) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iright)
                             ((1 *)) work-%offset%)
       n vr ldvr ierr)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
```

```
    (setf n var-7)
    (setf ldvr var-9)
    (setf ierr var-10))
  (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                ((> jc n) nil)
    (tagbody
      (if (<
           (f2cl-lib:fref alphai-%data% (jc) ((1 *)) alphai-%offset%)
           zero)
          (go label100))
      (setf temp zero)
      (cond
       ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr n) nil)
          (tagbody
            (setf temp
                    (max temp
                         (abs
                          (f2cl-lib:fref vr-%data% (jr jc)
                                         ((1 ldvr) (1 *))
                                         vr-%offset%))))
           label60)))
       (t
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr n) nil)
          (tagbody
            (setf temp
                    (max temp
                         (+
                          (abs
                           (f2cl-lib:fref vr-%data% (jr jc)
                                          ((1 ldvr) (1 *))
                                          vr-%offset%))
                          (abs
                           (f2cl-lib:fref vr-%data%
                                          (jr
                                           (f2cl-lib:int-add jc 1))
                                          ((1 ldvr) (1 *))
                                          vr-%offset%)))))
           label70))))
      (if (< temp smlnum)
          (go label100))
      (setf temp (/ one temp))
      (cond
       ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr n) nil)
          (tagbody
            (setf (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
```

```
                                        vr-%offset%)
                                (*
                                 (f2cl-lib:fref vr-%data% (jr jc)
                                                ((1 ldvr) (1 *)) vr-%offset%)
                                 temp))
                     label80)))
                  (t
                   (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                                 ((> jr n) nil)
                     (tagbody
                       (setf (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                            vr-%offset%)
                                (*
                                 (f2cl-lib:fref vr-%data% (jr jc)
                                                ((1 ldvr) (1 *)) vr-%offset%)
                                 temp))
                       (setf (f2cl-lib:fref vr-%data%
                                            (jr (f2cl-lib:int-add jc 1))
                                            ((1 ldvr) (1 *)) vr-%offset%)
                                (*
                                 (f2cl-lib:fref vr-%data%
                                                (jr (f2cl-lib:int-add jc 1))
                                                ((1 ldvr) (1 *)) vr-%offset%)
                                 temp))
                     label90))))
               label100))))))
        label110
         (cond
          (ilascl
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
               (dlascl "G" 0 0 anrmto anrm n 1 alphar n ierr)
             (declare (ignore var-0 var-1 var-2 var-6 var-7))
             (when var-3 (setf anrmto var-3))
             (when var-4 (setf anrm var-4))
             (when var-5 (setf n var-5))
             (when var-8 (setf n var-8))
             (when var-9 (setf ierr var-9)))
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
               (dlascl "G" 0 0 anrmto anrm n 1 alphai n ierr)
             (declare (ignore var-0 var-1 var-2 var-6 var-7))
             (when var-3 (setf anrmto var-3))
             (when var-4 (setf anrm var-4))
             (when var-5 (setf n var-5))
             (when var-8 (setf n var-8))
             (when var-9 (setf ierr var-9)))))
         (cond
          (ilbscl
           (multiple-value-bind
```

```
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (dlascl "G" 0 0 bnrmto bnrm n 1 beta n ierr)
            (declare (ignore var-0 var-1 var-2 var-6 var-7))
            (when var-3 (setf bnrmto var-3))
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf n var-5))
            (when var-8 (setf n var-8))
            (when var-9 (setf ierr var-9)))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 maxwrk) 'double-float))
        (go end_label)
       end_label
        (return
         (values jobvl jobvr n nil lda nil ldb$ nil nil nil nil ldvl nil ldvr
                 nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dggev fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::jobvl
                                              fortran-to-lisp::jobvr
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$ nil nil nil
                                              nil fortran-to-lisp::ldvl nil
                                              fortran-to-lisp::ldvr nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::dggbak
                                              fortran-to-lisp::dgeqrf
                                              fortran-to-lisp::dggbal))))
```

———-

24.11 dggev3

24.11.1 Axiom unit tests

— **dggev3.input** —

```
)set break resume
)sys rm -f dggev3.output
)spool dggev3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

24.11.2 Axiom help page

— **dggev3.help** —

```
======================================================================
dggev3 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dggev3 (JOBVL, JOBVR, N, A, LDA, B, LDB, ALPHAR, ALPHAI,
           BETA, VL, LDVL, VR, LDVR, WORK, LWORK, INFO)
            DGGEV3 computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for GE matrices (blocked algorithm)

Function/Subroutine Documentation
   subroutine dggev3 (character JOBVL, character JOBVR, integer N, double
```

```
      precision, dimension( lda, * ) A, integer LDA, double precision,
      dimension( ldb, * ) B, integer LDB, double precision, dimension( * )
      ALPHAR, double precision, dimension( * ) ALPHAI, double precision,
      dimension( * ) BETA, double precision, dimension( ldvl, * ) VL, integer
      LDVL, double precision, dimension( ldvr, * ) VR, integer LDVR, double
      precision, dimension( * ) WORK, integer LWORK, integer INFO)
       DGGEV3 computes the eigenvalues and, optionally, the left and/or right
      eigenvectors for GE matrices (blocked algorithm)

Purpose:
   DGGEV3 computes for a pair of N-by-N real nonsymmetric matrices (A,B)
   the generalized eigenvalues, and optionally, the left and/or right
   generalized eigenvectors.

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar
   lambda or a ratio alpha/beta = lambda, such that A - lambda*B is
   singular. It is usually represented as the pair (alpha,beta), as
   there is a reasonable interpretation for beta=0, and even for both
   being zero.

   The right eigenvector v(j) corresponding to the eigenvalue lambda(j)
   of (A,B) satisfies

   A * v(j) = lambda(j) * B * v(j).

   The left eigenvector u(j) corresponding to the eigenvalue lambda(j)
   of (A,B) satisfies

   u(j)**H * A  = lambda(j) * u(j)**H * B .

   where u(j)**H is the conjugate-transpose of u(j).

Parameters:

 JOBVL
   JOBVL is CHARACTER*1
   = 'N':  do not compute the left generalized eigenvectors;
   = 'V':  compute the left generalized eigenvectors.

 JOBVR
   JOBVR is CHARACTER*1
   = 'N':  do not compute the right generalized eigenvectors;
   = 'V':  compute the right generalized eigenvectors.

 N
   N is INTEGER
   The order of the matrices A, B, VL, and VR.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA, N)
```

```
  On entry, the matrix A in the pair (A,B).
  On exit, A has been overwritten.

LDA
  LDA is INTEGER
  The leading dimension of A.  LDA >= max(1,N).

B
  B is DOUBLE PRECISION array, dimension (LDB, N)
  On entry, the matrix B in the pair (A,B).
  On exit, B has been overwritten.

LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).

ALPHAR
  ALPHAR is DOUBLE PRECISION array, dimension (N)

ALPHAI
  ALPHAI is DOUBLE PRECISION array, dimension (N)

BETA
  BETA is DOUBLE PRECISION array, dimension (N)
  On exit, (ALPHAR(j) + ALPHAI(j)*i)/BETA(j), j=1,...,N, will
  be the generalized eigenvalues.  If ALPHAI(j) is zero, then
  the j-th eigenvalue is real; if positive, then the j-th and
  (j+1)-st eigenvalues are a complex conjugate pair, with
  ALPHAI(j+1) negative.

  Note: the quotients ALPHAR(j)/BETA(j) and ALPHAI(j)/BETA(j)
  may easily over- or underflow, and BETA(j) may even be zero.
  Thus, the user should avoid naively computing the ratio
  alpha/beta.  However, ALPHAR and ALPHAI will be always less
  than and usually comparable with norm(A) in magnitude, and
  BETA always less than and usually comparable with norm(B).

VL
  VL is DOUBLE PRECISION array, dimension (LDVL,N)
  If JOBVL = 'V', the left eigenvectors u(j) are stored one
  after another in the columns of VL, in the same order as
  their eigenvalues. If the j-th eigenvalue is real, then
  u(j) = VL(:,j), the j-th column of VL. If the j-th and
  (j+1)-th eigenvalues form a complex conjugate pair, then
  u(j) = VL(:,j)+i*VL(:,j+1) and u(j+1) = VL(:,j)-i*VL(:,j+1).
  Each eigenvector is scaled so the largest component has
  abs(real part)+abs(imag. part)=1.
  Not referenced if JOBVL = 'N'.

LDVL
```

```
  LDVL is INTEGER
  The leading dimension of the matrix VL. LDVL >= 1, and
  if JOBVL = 'V', LDVL >= N.

VR
  VR is DOUBLE PRECISION array, dimension (LDVR,N)
  If JOBVR = 'V', the right eigenvectors v(j) are stored one
  after another in the columns of VR, in the same order as
  their eigenvalues. If the j-th eigenvalue is real, then
  v(j) = VR(:,j), the j-th column of VR. If the j-th and
  (j+1)-th eigenvalues form a complex conjugate pair, then
  v(j) = VR(:,j)+i*VR(:,j+1) and v(j+1) = VR(:,j)-i*VR(:,j+1).
  Each eigenvector is scaled so the largest component has
  abs(real part)+abs(imag. part)=1.
  Not referenced if JOBVR = 'N'.

LDVR
  LDVR is INTEGER
  The leading dimension of the matrix VR. LDVR >= 1, and
  if JOBVR = 'V', LDVR >= N.

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  = 1,...,N:
        The QZ iteration failed.  No eigenvectors have been
        calculated, but ALPHAR(j), ALPHAI(j), and BETA(j)
        should be correct for j=INFO+1,...,N.
  > N:  =N+1: other than QZ iteration failed in DHGEQZ.
        =N+2: error return from DTGEVC.
```

———-

24.11.3 fortran code

— dggev3.f —

```
*  =====================================================================
      SUBROUTINE DGGEV3( JOBVL, JOBVR, N, A, LDA, B, LDB, ALPHAR,
     $                   ALPHAI, BETA, VL, LDVL, VR, LDVR, WORK, LWORK,
     $                   INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     January 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVL, JOBVR
      INTEGER            INFO, LDA, LDB, LDVL, LDVR, LWORK, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), ALPHAI( * ), ALPHAR( * ),
     $                   B( LDB, * ), BETA( * ), VL( LDVL, * ),
     $                   VR( LDVR, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ILASCL, ILBSCL, ILV, ILVL, ILVR, LQUERY
      CHARACTER          CHTEMP
      INTEGER            ICOLS, IERR, IHI, IJOBVL, IJOBVR, ILEFT, ILO,
     $                   IN, IRIGHT, IROWS, ITAU, IWRK, JC, JR, LWKOPT
      DOUBLE PRECISION   ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS,
     $                   SMLNUM, TEMP
*     ..
*     .. Local Arrays ..
      LOGICAL            LDUMMA( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEQRF, DGGBAK, DGGBAL, DGGHD3, DHGEQZ, DLABAD,
     $                   DLACPY, DLASCL, DLASET, DORGQR, DORMQR, DTGEVC,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, DLANGE
```

```
      EXTERNAL           LSAME, DLAMCH, DLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVL, 'N' ) ) THEN
         IJOBVL = 1
         ILVL = .FALSE.
      ELSE IF( LSAME( JOBVL, 'V' ) ) THEN
         IJOBVL = 2
         ILVL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVL = .FALSE.
      END IF
*
      IF( LSAME( JOBVR, 'N' ) ) THEN
         IJOBVR = 1
         ILVR = .FALSE.
      ELSE IF( LSAME( JOBVR, 'V' ) ) THEN
         IJOBVR = 2
         ILVR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVR = .FALSE.
      END IF
      ILV = ILVL .OR. ILVR
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDVL.LT.1 .OR. ( ILVL .AND. LDVL.LT.N ) ) THEN
         INFO = -12
      ELSE IF( LDVR.LT.1 .OR. ( ILVR .AND. LDVR.LT.N ) ) THEN
         INFO = -14
```

```
      ELSE IF( LWORK.LT.MAX( 1, 8*N ) .AND. .NOT.LQUERY ) THEN
         INFO = -16
      END IF
*
*     Compute workspace
*
      IF( INFO.EQ.0 ) THEN
         CALL DGEQRF( N, N, B, LDB, WORK, WORK, -1, IERR )
         LWKOPT = MAX(1, 8*N, 3*N+INT( WORK( 1 ) ) )
         CALL DORMQR( 'L', 'T', N, N, N, B, LDB, WORK, A, LDA, WORK, -1,
     $                IERR )
         LWKOPT = MAX( LWKOPT, 3*N+INT( WORK ( 1 ) ) )
         IF( ILVL ) THEN
            CALL DORGQR( N, N, N, VL, LDVL, WORK, WORK, -1, IERR )
            LWKOPT = MAX( LWKOPT, 3*N+INT( WORK ( 1 ) ) )
         END IF
         IF( ILV ) THEN
            CALL DGGHD3( JOBVL, JOBVR, N, 1, N, A, LDA, B, LDB, VL,
     $                   LDVL, VR, LDVR, WORK, -1, IERR )
            LWKOPT = MAX( LWKOPT, 3*N+INT( WORK ( 1 ) ) )
            CALL DHGEQZ( 'S', JOBVL, JOBVR, N, 1, N, A, LDA, B, LDB,
     $                   ALPHAR, ALPHAI, BETA, VL, LDVL, VR, LDVR,
     $                   WORK, -1, IERR )
            LWKOPT = MAX( LWKOPT, 2*N+INT( WORK ( 1 ) ) )
         ELSE
            CALL DGGHD3( 'N', 'N', N, 1, N, A, LDA, B, LDB, VL, LDVL,
     $                   VR, LDVR, WORK, -1, IERR )
            LWKOPT = MAX( LWKOPT, 3*N+INT( WORK ( 1 ) ) )
            CALL DHGEQZ( 'E', JOBVL, JOBVR, N, 1, N, A, LDA, B, LDB,
     $                   ALPHAR, ALPHAI, BETA, VL, LDVL, VR, LDVR,
     $                   WORK, -1, IERR )
            LWKOPT = MAX( LWKOPT, 2*N+INT( WORK ( 1 ) ) )
         END IF

         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGGEV3 ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
```

```
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = DLANGE( 'M', N, N, A, LDA, WORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
      IF( ILASCL )
     $   CALL DLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = DLANGE( 'M', N, N, B, LDB, WORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
     $   CALL DLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrices A, B to isolate eigenvalues if possible
*
      ILEFT = 1
      IRIGHT = N + 1
      IWRK = IRIGHT + N
      CALL DGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, WORK( ILEFT ),
     $             WORK( IRIGHT ), WORK( IWRK ), IERR )
*
*     Reduce B to triangular form (QR decomposition of B)
*
      IROWS = IHI + 1 - ILO
      IF( ILV ) THEN
         ICOLS = N + 1 - ILO
      ELSE
```

```
         ICOLS = IROWS
      END IF
      ITAU = IWRK
      IWRK = ITAU + IROWS
      CALL DGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to matrix A
*
      CALL DORMQR( 'L', 'T', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VL
*
      IF( ILVL ) THEN
         CALL DLASET( 'Full', N, N, ZERO, ONE, VL, LDVL )
         IF( IROWS.GT.1 ) THEN
            CALL DLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VL( ILO+1, ILO ), LDVL )
         END IF
         CALL DORGQR( IROWS, IROWS, IROWS, VL( ILO, ILO ), LDVL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Initialize VR
*
      IF( ILVR )
     $   CALL DLASET( 'Full', N, N, ZERO, ONE, VR, LDVR )
*
*     Reduce to generalized Hessenberg form
*
      IF( ILV ) THEN
*
*        Eigenvectors requested -- work on whole matrix.
*
         CALL DGGHD3( JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB, VL,
     $                LDVL, VR, LDVR, WORK( IWRK ), LWORK+1-IWRK, IERR )
      ELSE
         CALL DGGHD3( 'N', 'N', IROWS, 1, IROWS, A( ILO, ILO ), LDA,
     $                B( ILO, ILO ), LDB, VL, LDVL, VR, LDVR,
     $                WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Perform QZ algorithm (Compute eigenvalues, and optionally, the
*     Schur forms and Schur vectors)
*
      IWRK = ITAU
      IF( ILV ) THEN
         CHTEMP = 'S'
```

```
      ELSE
         CHTEMP = 'E'
      END IF
      CALL DHGEQZ( CHTEMP, JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHAR, ALPHAI, BETA, VL, LDVL, VR, LDVR,
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 110
      END IF
*
*     Compute Eigenvectors
*
      IF( ILV ) THEN
         IF( ILVL ) THEN
            IF( ILVR ) THEN
               CHTEMP = 'B'
            ELSE
               CHTEMP = 'L'
            END IF
         ELSE
            CHTEMP = 'R'
         END IF
         CALL DTGEVC( CHTEMP, 'B', LDUMMA, N, A, LDA, B, LDB, VL, LDVL,
     $                VR, LDVR, N, IN, WORK( IWRK ), IERR )
         IF( IERR.NE.0 ) THEN
            INFO = N + 2
            GO TO 110
         END IF
*
*        Undo balancing on VL and VR and normalization
*
         IF( ILVL ) THEN
            CALL DGGBAK( 'P', 'L', N, ILO, IHI, WORK( ILEFT ),
     $                   WORK( IRIGHT ), N, VL, LDVL, IERR )
            DO 50 JC = 1, N
               IF( ALPHAI( JC ).LT.ZERO )
     $            GO TO 50
               TEMP = ZERO
               IF( ALPHAI( JC ).EQ.ZERO ) THEN
                  DO 10 JR = 1, N
                     TEMP = MAX( TEMP, ABS( VL( JR, JC ) ) )
   10             CONTINUE
               ELSE
```

```
               DO 20 JR = 1, N
                  TEMP = MAX( TEMP, ABS( VL( JR, JC ) )+
  $                      ABS( VL( JR, JC+1 ) ) )
20             CONTINUE
            END IF
            IF( TEMP.LT.SMLNUM )
  $            GO TO 50
            TEMP = ONE / TEMP
            IF( ALPHAI( JC ).EQ.ZERO ) THEN
               DO 30 JR = 1, N
                  VL( JR, JC ) = VL( JR, JC )*TEMP
30             CONTINUE
            ELSE
               DO 40 JR = 1, N
                  VL( JR, JC ) = VL( JR, JC )*TEMP
                  VL( JR, JC+1 ) = VL( JR, JC+1 )*TEMP
40             CONTINUE
            END IF
50       CONTINUE
      END IF
      IF( ILVR ) THEN
         CALL DGGBAK( 'P', 'R', N, ILO, IHI, WORK( ILEFT ),
  $                   WORK( IRIGHT ), N, VR, LDVR, IERR )
         DO 100 JC = 1, N
            IF( ALPHAI( JC ).LT.ZERO )
  $            GO TO 100
            TEMP = ZERO
            IF( ALPHAI( JC ).EQ.ZERO ) THEN
               DO 60 JR = 1, N
                  TEMP = MAX( TEMP, ABS( VR( JR, JC ) ) )
60             CONTINUE
            ELSE
               DO 70 JR = 1, N
                  TEMP = MAX( TEMP, ABS( VR( JR, JC ) )+
  $                      ABS( VR( JR, JC+1 ) ) )
70             CONTINUE
            END IF
            IF( TEMP.LT.SMLNUM )
  $            GO TO 100
            TEMP = ONE / TEMP
            IF( ALPHAI( JC ).EQ.ZERO ) THEN
               DO 80 JR = 1, N
                  VR( JR, JC ) = VR( JR, JC )*TEMP
80             CONTINUE
            ELSE
               DO 90 JR = 1, N
                  VR( JR, JC ) = VR( JR, JC )*TEMP
                  VR( JR, JC+1 ) = VR( JR, JC+1 )*TEMP
90             CONTINUE
            END IF
```

```
  100       CONTINUE
         END IF
*
*        End of eigenvector calculation
*
      END IF
*
*     Undo scaling if necessary
*
  110 CONTINUE
*
      IF( ILASCL ) THEN
         CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAR, N, IERR )
         CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAI, N, IERR )
      END IF
*
      IF( ILBSCL ) THEN
         CALL DLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
      END IF
*
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of DGGEV3
*
      END
```

———-

24.11.4 lisp code

— dggev3.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dggev3
         (jobvl jobvr n a lda b ldb$ alphar alphai beta vl ldvl vr ldvr work
          lwork info)
    (declare (type (array double-float (*)) work vr vl beta alphai alphar b a)
             (type (f2cl-lib:integer4) info lwork ldvr ldvl ldb$ lda n)
             (type (string *) jobvr jobvl))
    (f2cl-lib:with-multi-array-data
        ((jobvl character jobvl-%data% jobvl-%offset%)
         (jobvr character jobvr-%data% jobvr-%offset%)
         (a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%)
```

```
    (alphar double-float alphar-%data% alphar-%offset%)
    (alphai double-float alphai-%data% alphai-%offset%)
    (beta double-float beta-%data% beta-%offset%)
    (vl double-float vl-%data% vl-%offset%)
    (vr double-float vr-%data% vr-%offset%)
    (work double-float work-%data% work-%offset%))
  (prog ((ldumma (make-array 1 :element-type 't)) (anrm 0.0d0)
         (anrmto 0.0d0) (bignum 0.0d0) (bnrm 0.0d0) (bnrmto 0.0d0)
         (eps 0.0d0) (smlnum 0.0d0) (temp 0.0d0) (icols 0) (ierr 0) (ihi 0)
         (ijobvl 0) (ijobvr 0) (ileft 0) (ilo 0) (in 0) (iright 0)
         (irows 0) (itau 0) (iwrk 0) (jc 0) (jr 0) (lwkopt 0)
         (chtemp
          (make-array '(1) :element-type 'character :initial-element #\ ))
         (ilascl nil) (ilbscl nil) (ilv nil) (ilvl nil) (ilvr nil)
         (lquery nil))
    (declare (type (array f2cl-lib:logical (1)) ldumma)
             (type (double-float) anrm anrmto bignum bnrm bnrmto eps smlnum
              temp)
             (type (f2cl-lib:integer4) icols ierr ihi ijobvl ijobvr ileft
              ilo in iright irows itau iwrk jc jr lwkopt)
             (type (string 1) chtemp)
             (type f2cl-lib:logical ilascl ilbscl ilv ilvl ilvr lquery))
    (cond
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame jobvl "N")
        (declare (ignore var-1))
        (when var-0 (setf jobvl var-0))
        ret-val)
      (setf ijobvl 1) (setf ilvl f2cl-lib:%false%))
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame jobvl "V")
        (declare (ignore var-1))
        (when var-0 (setf jobvl var-0))
        ret-val)
      (setf ijobvl 2) (setf ilvl f2cl-lib:%true%))
     (t (setf ijobvl -1) (setf ilvl f2cl-lib:%false%)))
    (cond
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame jobvr "N")
        (declare (ignore var-1))
        (when var-0 (setf jobvr var-0))
        ret-val)
      (setf ijobvr 1) (setf ilvr f2cl-lib:%false%))
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame jobvr "V")
        (declare (ignore var-1))
        (when var-0 (setf jobvr var-0))
        ret-val)
      (setf ijobvr 2) (setf ilvr f2cl-lib:%true%))
     (t (setf ijobvr -1) (setf ilvr f2cl-lib:%false%)))
```

```
(setf ilv (or ilvl ilvr))
(setf info 0)
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((<= ijobvl 0) (setf info -1)) ((<= ijobvr 0) (setf info -2))
      ((< n 0) (setf info -3))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -5))
      ((< ldb$
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -7))
      ((or (< ldvl 1) (and ilvl (< ldvl n))) (setf info -12))
      ((or (< ldvr 1) (and ilvr (< ldvr n))) (setf info -14))
      ((and
        (< lwork
           (max (the f2cl-lib:integer4 1)
                (the f2cl-lib:integer4 (f2cl-lib:int-mul 8 n))))
        (not lquery))
       (setf info -16)))
(cond
 ((= info 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dgeqrf n n b ldb$ work work -1 ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf n var-0)
    (setf n var-1)
    (setf ldb$ var-3)
    (setf ierr var-7))
  (setf lwkopt
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul 8 n))
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                      (f2cl-lib:int
                                       (f2cl-lib:fref work-%data% (1)
                                                      ((1 *))
                                                      work-%offset%))))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dormqr "L" "T" n n n b ldb$ work a lda work -1 ierr)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-6 (setf ldb$ var-6))
    (when var-9 (setf lda var-9))
    (when var-12 (setf ierr var-12)))
  (setf lwkopt
```

```
        (max (the f2cl-lib:integer4 lwkopt)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int
                                     (f2cl-lib:fref work-%data% (1)
                                                    ((1 *))
                                                    work-%offset%))))))
(cond
 (ilvl
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dorgqr n n n vl ldvl work work -1 ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf n var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldvl var-4))
    (when var-8 (setf ierr var-8)))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkopt)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                      (f2cl-lib:int
                                       (f2cl-lib:fref work-%data% (1)
                                                      ((1 *))
                                                      work-%offset%))))))))
(cond
 (ilv
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (dgghd3 jobvl jobvr n 1 n a lda b ldb$ vl ldvl vr ldvr work -1
       ierr)
    (declare (ignore var-3 var-5 var-7 var-9 var-11 var-13 var-14))
    (when var-0 (setf jobvl var-0))
    (when var-1 (setf jobvr var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf n var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-15 (setf ierr var-15)))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkopt)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                      (f2cl-lib:int
                                       (f2cl-lib:fref work-%data% (1)
                                                      ((1 *))
```

```
                                                        work-%offset%))))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18
      var-19)
     (dhgeqz "S" jobvl jobvr n 1 n a lda b ldb$ alphar alphai beta
      vl ldvl vr ldvr work -1 ierr)
   (declare
    (ignore var-0 var-4 var-6 var-8 var-10 var-11 var-12 var-13
     var-15 var-17 var-18))
   (when var-1 (setf jobvl var-1))
   (when var-2 (setf jobvr var-2))
   (when var-3 (setf n var-3))
   (when var-5 (setf n var-5))
   (when var-7 (setf lda var-7))
   (when var-9 (setf ldb$ var-9))
   (when var-14 (setf ldvl var-14))
   (when var-16 (setf ldvr var-16))
   (when var-19 (setf ierr var-19)))
 (setf lwkopt
         (max (the f2cl-lib:integer4 lwkopt)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     (f2cl-lib:int
                                      (f2cl-lib:fref work-%data% (1)
                                                     ((1 *))
                                                     work-%offset%)))))))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13 var-14 var-15)
     (dgghd3 "N" "N" n 1 n a lda b ldb$ vl ldvl vr ldvr work -1
      ierr)
   (declare
    (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-13
     var-14))
   (when var-2 (setf n var-2))
   (when var-4 (setf n var-4))
   (when var-6 (setf lda var-6))
   (when var-8 (setf ldb$ var-8))
   (when var-10 (setf ldvl var-10))
   (when var-12 (setf ldvr var-12))
   (when var-15 (setf ierr var-15)))
 (setf lwkopt
         (max (the f2cl-lib:integer4 lwkopt)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     (f2cl-lib:int
                                      (f2cl-lib:fref work-%data% (1)
                                                     ((1 *))
```

```
                                                        work-%offset%))))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18
         var-19)
        (dhgeqz "E" jobvl jobvr n 1 n a lda b ldb$ alphar alphai beta
         vl ldvl vr ldvr work -1 ierr)
      (declare
       (ignore var-0 var-4 var-6 var-8 var-10 var-11 var-12 var-13
        var-15 var-17 var-18))
      (when var-1 (setf jobvl var-1))
      (when var-2 (setf jobvr var-2))
      (when var-3 (setf n var-3))
      (when var-5 (setf n var-5))
      (when var-7 (setf lda var-7))
      (when var-9 (setf ldb$ var-9))
      (when var-14 (setf ldvl var-14))
      (when var-16 (setf ldvr var-16))
      (when var-19 (setf ierr var-19)))
    (setf lwkopt
            (max (the f2cl-lib:integer4 lwkopt)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        (f2cl-lib:int
                                         (f2cl-lib:fref work-%data% (1)
                                                        ((1 *))
                                                        work-%offset%))))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))))
(cond
 ((/= info 0) (xerbla "DGGEV3 " (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(setf eps (dlamch "P"))
(setf smlnum (dlamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "M" n n a lda work)
          (declare (ignore var-0 var-3 var-5))
```

```
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf ilascl f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf anrmto smlnum)
  (setf ilascl f2cl-lib:%true%))
 ((> anrm bignum) (setf anrmto bignum) (setf ilascl f2cl-lib:%true%)))
(if ilascl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 anrm anrmto n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf anrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "M" n n b ldb$ work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ilbscl f2cl-lib:%false%)
(cond
 ((and (> bnrm zero) (< bnrm smlnum)) (setf bnrmto smlnum)
  (setf ilbscl f2cl-lib:%true%))
 ((> bnrm bignum) (setf bnrmto bignum) (setf ilbscl f2cl-lib:%true%)))
(if ilbscl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 bnrm bnrmto n n b ldb$ ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bnrm var-3))
      (when var-4 (setf bnrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf ierr var-9))))
(setf ileft 1)
(setf iright (f2cl-lib:int-add n 1))
(setf iwrk (f2cl-lib:int-add iright n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
```

```
     var-11)
    (dggbal "P" n a lda b ldb$ ilo ihi
     (f2cl-lib:array-slice work-%data% double-float (ileft) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iright) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                           work-%offset%)
     ierr)
  (declare (ignore var-0 var-1 var-2 var-4 var-8 var-9 var-10))
  (setf lda var-3)
  (setf ldb$ var-5)
  (setf ilo var-6)
  (setf ihi var-7)
  (setf ierr var-11))
(setf irows (f2cl-lib:int-sub (f2cl-lib:int-add ihi 1) ilo))
(cond (ilv (setf icols (f2cl-lib:int-sub (f2cl-lib:int-add n 1) ilo)))
      (t (setf icols irows)))
(setf itau iwrk)
(setf iwrk (f2cl-lib:int-add itau irows))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dgeqrf irows icols
     (f2cl-lib:array-slice b-%data% double-float (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf irows var-0)
  (setf icols var-1)
  (setf ldb$ var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (dormqr "L" "T" irows icols irows
     (f2cl-lib:array-slice b-%data% double-float (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice a-%data% double-float (ilo ilo)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
```

```
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf irows var-2))
  (when var-3 (setf icols var-3))
  (when var-4 (setf irows var-4))
  (when var-6 (setf ldb$ var-6))
  (when var-9 (setf lda var-9))
  (when var-12 (setf ierr var-12)))
(cond
 (ilvl
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "Full" n n zero one vl ldvl)
    (declare (ignore var-0 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldvl var-6)))
  (cond
   ((> irows 1)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "L" (f2cl-lib:int-sub irows 1)
         (f2cl-lib:int-sub irows 1)
         (f2cl-lib:array-slice b-%data% double-float ((+ ilo 1) ilo)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice vl-%data% double-float ((+ ilo 1) ilo)
                               ((1 ldvl) (1 *)) vl-%offset%)
         ldvl)
      (declare (ignore var-0 var-1 var-2 var-3 var-5))
      (when var-4 (setf ldb$ var-4))
      (when var-6 (setf ldvl var-6)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dorgqr irows irows irows
       (f2cl-lib:array-slice vl-%data% double-float (ilo ilo)
                             ((1 ldvl) (1 *)) vl-%offset%)
       ldvl
       (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf irows var-0))
    (when var-1 (setf irows var-1))
    (when var-2 (setf irows var-2))
    (when var-4 (setf ldvl var-4))
    (when var-8 (setf ierr var-8)))))
(if ilvr
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
          (dlaset "Full" n n zero one vr ldvr)
        (declare (ignore var-0 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf zero var-3))
        (when var-4 (setf one var-4))
        (when var-6 (setf ldvr var-6))))
  (cond
   (ilv
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14 var-15)
        (dgghd3 jobvl jobvr n ilo ihi a lda b ldb$ vl ldvl vr ldvr
         (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                               work-%offset%)
         (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
      (declare (ignore var-5 var-7 var-9 var-11 var-13 var-14))
      (when var-0 (setf jobvl var-0))
      (when var-1 (setf jobvr var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf ilo var-3))
      (when var-4 (setf ihi var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-10 (setf ldvl var-10))
      (when var-12 (setf ldvr var-12))
      (when var-15 (setf ierr var-15))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14 var-15)
        (dgghd3 "N" "N" irows 1 irows
         (f2cl-lib:array-slice a-%data% double-float (ilo ilo)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice b-%data% double-float (ilo ilo)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$ vl ldvl vr ldvr
         (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                               work-%offset%)
         (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
      (declare
       (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-13 var-14))
      (when var-2 (setf irows var-2))
      (when var-4 (setf irows var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-10 (setf ldvl var-10))
      (when var-12 (setf ldvr var-12))
      (when var-15 (setf ierr var-15)))))
```

```
(setf iwrk itau)
(cond (ilv (f2cl-lib:f2cl-set-string chtemp "S" (string 1)))
      (t (f2cl-lib:f2cl-set-string chtemp "E" (string 1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19)
    (dhgeqz chtemp jobvl jobvr n ilo ihi a lda b ldb$ alphar alphai
     beta vl ldvl vr ldvr
     (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare
   (ignore var-6 var-8 var-10 var-11 var-12 var-13 var-15 var-17
    var-18))
  (when var-0 (setf chtemp var-0))
  (when var-1 (setf jobvl var-1))
  (when var-2 (setf jobvr var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf ilo var-4))
  (when var-5 (setf ihi var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldb$ var-9))
  (when var-14 (setf ldvl var-14))
  (when var-16 (setf ldvr var-16))
  (when var-19 (setf ierr var-19)))
(cond
 ((/= ierr 0)
  (cond ((and (> ierr 0) (<= ierr n)) (setf info ierr))
        ((and (> ierr n) (<= ierr (f2cl-lib:int-mul 2 n)))
         (setf info (f2cl-lib:int-sub ierr n)))
        (t (setf info (f2cl-lib:int-add n 1))))
  (go label110)))
(cond
 (ilv
  (cond
   (ilvl
    (cond (ilvr (f2cl-lib:f2cl-set-string chtemp "B" (string 1)))
          (t (f2cl-lib:f2cl-set-string chtemp "L" (string 1)))))
   (t (f2cl-lib:f2cl-set-string chtemp "R" (string 1))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (dtgevc chtemp "B" ldumma n a lda b ldb$ vl ldvl vr ldvr n in
       (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                             work-%offset%)
       ierr)
    (declare (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-14))
    (when var-0 (setf chtemp var-0))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
```

```
  (when var-7 (setf ldb$ var-7))
  (when var-9 (setf ldvl var-9))
  (when var-11 (setf ldvr var-11))
  (when var-12 (setf n var-12))
  (when var-13 (setf in var-13))
  (when var-15 (setf ierr var-15)))
(cond ((/= ierr 0) (setf info (f2cl-lib:int-add n 2)) (go label110)))
(cond
 (ilvl
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dggbak "P" "L" n ilo ihi
       (f2cl-lib:array-slice work-%data% double-float (ileft) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iright)
                             ((1 *)) work-%offset%)
       n vl ldvl ierr)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
    (setf n var-7)
    (setf ldvl var-9)
    (setf ierr var-10))
  (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                ((> jc n) nil)
    (tagbody
      (if (<
           (f2cl-lib:fref alphai-%data% (jc) ((1 *)) alphai-%offset%)
           zero)
          (go label50))
      (setf temp zero)
      (cond
       ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr n) nil)
          (tagbody
            (setf temp
                    (max temp
                         (abs
                          (f2cl-lib:fref vl-%data% (jr jc)
                                         ((1 ldvl) (1 *))
                                         vl-%offset%))))
           label10)))
       (t
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr n) nil)
          (tagbody
            (setf temp
                    (max temp
                         (+
```

```
                              (abs
                               (f2cl-lib:fref vl-%data% (jr jc)
                                              ((1 ldvl) (1 *))
                                              vl-%offset%))
                              (abs
                               (f2cl-lib:fref vl-%data%
                                              (jr
                                               (f2cl-lib:int-add jc 1))
                                              ((1 ldvl) (1 *))
                                              vl-%offset%)))))
                 label20))))
            (if (< temp smlnum)
                (go label50))
            (setf temp (/ one temp))
            (cond
             ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
              (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                            ((> jr n) nil)
                (tagbody
                  (setf (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                                       vl-%offset%)
                        (*
                         (f2cl-lib:fref vl-%data% (jr jc)
                                        ((1 ldvl) (1 *)) vl-%offset%)
                         temp))
                 label30)))
             (t
              (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                            ((> jr n) nil)
                (tagbody
                  (setf (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                                       vl-%offset%)
                        (*
                         (f2cl-lib:fref vl-%data% (jr jc)
                                        ((1 ldvl) (1 *)) vl-%offset%)
                         temp))
                  (setf (f2cl-lib:fref vl-%data%
                                       (jr (f2cl-lib:int-add jc 1))
                                       ((1 ldvl) (1 *)) vl-%offset%)
                        (*
                         (f2cl-lib:fref vl-%data%
                                        (jr (f2cl-lib:int-add jc 1))
                                        ((1 ldvl) (1 *)) vl-%offset%)
                         temp))
                 label40))))
        label50))))
    (cond
     (ilvr
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
      var-10)
     (dggbak "P" "R" n ilo ihi
      (f2cl-lib:array-slice work-%data% double-float (ileft) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (iright)
                            ((1 *)) work-%offset%)
      n vr ldvr ierr)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
    (setf n var-7)
    (setf ldvr var-9)
    (setf ierr var-10))
  (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                ((> jc n) nil)
    (tagbody
      (if (<
           (f2cl-lib:fref alphai-%data% (jc) ((1 *)) alphai-%offset%)
           zero)
          (go label100))
      (setf temp zero)
      (cond
       ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr n) nil)
          (tagbody
            (setf temp
                    (max temp
                         (abs
                          (f2cl-lib:fref vr-%data% (jr jc)
                                         ((1 ldvr) (1 *))
                                         vr-%offset%))))
           label60)))
       (t
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr n) nil)
          (tagbody
            (setf temp
                    (max temp
                         (+
                          (abs
                           (f2cl-lib:fref vr-%data% (jr jc)
                                          ((1 ldvr) (1 *))
                                          vr-%offset%))
                          (abs
                           (f2cl-lib:fref vr-%data%
                                          (jr
                                           (f2cl-lib:int-add jc 1))
                                          ((1 ldvr) (1 *))
                                          vr-%offset%)))))
           label70))))
```

```
            (if (< temp smlnum)
                (go label100))
            (setf temp (/ one temp))
            (cond
             ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
              (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                            ((> jr n) nil)
                (tagbody
                  (setf (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                       vr-%offset%)
                        (*
                         (f2cl-lib:fref vr-%data% (jr jc)
                                        ((1 ldvr) (1 *)) vr-%offset%)
                         temp))
                label80)))
             (t
              (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                            ((> jr n) nil)
                (tagbody
                  (setf (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                       vr-%offset%)
                        (*
                         (f2cl-lib:fref vr-%data% (jr jc)
                                        ((1 ldvr) (1 *)) vr-%offset%)
                         temp))
                  (setf (f2cl-lib:fref vr-%data%
                                       (jr (f2cl-lib:int-add jc 1))
                                       ((1 ldvr) (1 *)) vr-%offset%)
                        (*
                         (f2cl-lib:fref vr-%data%
                                        (jr (f2cl-lib:int-add jc 1))
                                        ((1 ldvr) (1 *)) vr-%offset%)
                         temp))
                label90))))
        label100))))))
label110
 (cond
  (ilascl
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (dlascl "G" 0 0 anrmto anrm n 1 alphar n ierr)
     (declare (ignore var-0 var-1 var-2 var-6 var-7))
     (when var-3 (setf anrmto var-3))
     (when var-4 (setf anrm var-4))
     (when var-5 (setf n var-5))
     (when var-8 (setf n var-8))
     (when var-9 (setf ierr var-9)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (dlascl "G" 0 0 anrmto anrm n 1 alphai n ierr)
```

```
            (declare (ignore var-0 var-1 var-2 var-6 var-7))
            (when var-3 (setf anrmto var-3))
            (when var-4 (setf anrm var-4))
            (when var-5 (setf n var-5))
            (when var-8 (setf n var-8))
            (when var-9 (setf ierr var-9)))))
        (cond
         (ilbscl
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (dlascl "G" 0 0 bnrmto bnrm n 1 beta n ierr)
            (declare (ignore var-0 var-1 var-2 var-6 var-7))
            (when var-3 (setf bnrmto var-3))
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf n var-5))
            (when var-8 (setf n var-8))
            (when var-9 (setf ierr var-9)))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
        (go end_label)
       end_label
        (return
         (values jobvl jobvr n nil lda nil ldb$ nil nil nil nil ldvl nil ldvr
                 nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dggev3
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::jobvl
                                              fortran-to-lisp::jobvr
```

```
                                        fortran-to-lisp::n nil
                                        fortran-to-lisp::lda nil
                                        fortran-to-lisp::ldb$ nil nil nil
                                        nil fortran-to-lisp::ldvl nil
                                        fortran-to-lisp::ldvr nil nil
                                        fortran-to-lisp::info)
                                      :calls
                                      '(fortran-to-lisp::dggbak
                                        fortran-to-lisp::dggbal
                                        fortran-to-lisp::dgeqrf))))
```

———-

24.12 dggevx

24.12.1 Axiom unit tests

— dggevx.input —

```
)set break resume
)sys rm -f dggevx.output
)spool dggevx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

24.12.2 Axiom help page

— dggevx.help —

```
======================================================================
dggevx examples
======================================================================

======================================================================
Man Page Details
======================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine dggevx (BALANC, JOBVL, JOBVR, SENSE, N, A, LDA, B, LDB,
           ALPHAR, ALPHAI, BETA, VL, LDVL, VR, LDVR, ILO, IHI, LSCALE, RSCALE,
           ABNRM, BBNRM, RCONDE, RCONDV, WORK, LWORK, IWORK, BWORK, INFO)
            DGGEVX computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for GE matrices

Function/Subroutine Documentation
   subroutine dggevx (character BALANC, character JOBVL, character JOBVR,
       character SENSE, integer N, double precision, dimension( lda, * ) A,
       integer LDA, double precision, dimension( ldb, * ) B, integer LDB,
       double precision, dimension( * ) ALPHAR, double precision, dimension( *
       ) ALPHAI, double precision, dimension( * ) BETA, double precision,
       dimension( ldvl, * ) VL, integer LDVL, double precision, dimension(
       ldvr, * ) VR, integer LDVR, integer ILO, integer IHI, double precision,
       dimension( * ) LSCALE, double precision, dimension( * ) RSCALE, double
       precision ABNRM, double precision BBNRM, double precision, dimension( *
       ) RCONDE, double precision, dimension( * ) RCONDV, double precision,
       dimension( * ) WORK, integer LWORK, integer, dimension( * ) IWORK,
       logical, dimension( * ) BWORK, integer INFO)
        DGGEVX computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for GE matrices

Purpose:
   DGGEVX computes for a pair of N-by-N real nonsymmetric matrices (A,B)
   the generalized eigenvalues, and optionally, the left and/or right
   generalized eigenvectors.

   Optionally also, it computes a balancing transformation to improve
   the conditioning of the eigenvalues and eigenvectors (ILO, IHI,
   LSCALE, RSCALE, ABNRM, and BBNRM), reciprocal condition numbers for
   the eigenvalues (RCONDE), and reciprocal condition numbers for the
   right eigenvectors (RCONDV).

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar
   lambda or a ratio alpha/beta = lambda, such that A - lambda*B is
   singular. It is usually represented as the pair (alpha,beta), as
   there is a reasonable interpretation for beta=0, and even for both
   being zero.

   The right eigenvector v(j) corresponding to the eigenvalue lambda(j)
   of (A,B) satisfies

   A * v(j) = lambda(j) * B * v(j) .

   The left eigenvector u(j) corresponding to the eigenvalue lambda(j)
   of (A,B) satisfies
```

```
  u(j)**H * A  = lambda(j) * u(j)**H * B.

  where u(j)**H is the conjugate-transpose of u(j).

Parameters:

 BALANC
   BALANC is CHARACTER*1
   Specifies the balance option to be performed.
   = 'N':  do not diagonally scale or permute;
   = 'P':  permute only;
   = 'S':  scale only;
   = 'B':  both permute and scale.
   Computed reciprocal condition numbers will be for the
   matrices after permuting and/or balancing. Permuting does
   not change condition numbers (in exact arithmetic), but balancing does.

 JOBVL
   JOBVL is CHARACTER*1
   = 'N':  do not compute the left generalized eigenvectors;
   = 'V':  compute the left generalized eigenvectors.

 JOBVR
   JOBVR is CHARACTER*1
   = 'N':  do not compute the right generalized eigenvectors;
   = 'V':  compute the right generalized eigenvectors.

 SENSE
   SENSE is CHARACTER*1
   Determines which reciprocal condition numbers are computed.
   = 'N': none are computed;
   = 'E': computed for eigenvalues only;
   = 'V': computed for eigenvectors only;
   = 'B': computed for eigenvalues and eigenvectors.

 N
   N is INTEGER
   The order of the matrices A, B, VL, and VR.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA, N)
   On entry, the matrix A in the pair (A,B).
   On exit, A has been overwritten. If JOBVL='V' or JOBVR='V'
   or both, then A contains the first part of the real Schur
   form of the "balanced" versions of the input A and B.

 LDA
   LDA is INTEGER
   The leading dimension of A.  LDA >= max(1,N).
```

```
B
  B is DOUBLE PRECISION array, dimension (LDB, N)
  On entry, the matrix B in the pair (A,B).
  On exit, B has been overwritten. If JOBVL='V' or JOBVR='V'
  or both, then B contains the second part of the real Schur
  form of the "balanced" versions of the input A and B.

LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).

ALPHAR
  ALPHAR is DOUBLE PRECISION array, dimension (N)

ALPHAI
  ALPHAI is DOUBLE PRECISION array, dimension (N)

BETA
  BETA is DOUBLE PRECISION array, dimension (N)
  On exit, (ALPHAR(j) + ALPHAI(j)*i)/BETA(j), j=1,...,N, will
  be the generalized eigenvalues.  If ALPHAI(j) is zero, then
  the j-th eigenvalue is real; if positive, then the j-th and
  (j+1)-st eigenvalues are a complex conjugate pair, with
  ALPHAI(j+1) negative.

  Note: the quotients ALPHAR(j)/BETA(j) and ALPHAI(j)/BETA(j)
  may easily over- or underflow, and BETA(j) may even be zero.
  Thus, the user should avoid naively computing the ratio
  ALPHA/BETA. However, ALPHAR and ALPHAI will be always less
  than and usually comparable with norm(A) in magnitude, and
  BETA always less than and usually comparable with norm(B).

VL
  VL is DOUBLE PRECISION array, dimension (LDVL,N)
  If JOBVL = 'V', the left eigenvectors u(j) are stored one
  after another in the columns of VL, in the same order as
  their eigenvalues. If the j-th eigenvalue is real, then
  u(j) = VL(:,j), the j-th column of VL. If the j-th and
  (j+1)-th eigenvalues form a complex conjugate pair, then
  u(j) = VL(:,j)+i*VL(:,j+1) and u(j+1) = VL(:,j)-i*VL(:,j+1).
  Each eigenvector will be scaled so the largest component have
  abs(real part) + abs(imag. part) = 1.
  Not referenced if JOBVL = 'N'.

LDVL
  LDVL is INTEGER
  The leading dimension of the matrix VL. LDVL >= 1, and
  if JOBVL = 'V', LDVL >= N.

VR
```

```
  VR is DOUBLE PRECISION array, dimension (LDVR,N)
  If JOBVR = 'V', the right eigenvectors v(j) are stored one
  after another in the columns of VR, in the same order as
  their eigenvalues. If the j-th eigenvalue is real, then
  v(j) = VR(:,j), the j-th column of VR. If the j-th and
  (j+1)-th eigenvalues form a complex conjugate pair, then
  v(j) = VR(:,j)+i*VR(:,j+1) and v(j+1) = VR(:,j)-i*VR(:,j+1).
  Each eigenvector will be scaled so the largest component have
  abs(real part) + abs(imag. part) = 1.
  Not referenced if JOBVR = 'N'.

LDVR
  LDVR is INTEGER
  The leading dimension of the matrix VR. LDVR >= 1, and
  if JOBVR = 'V', LDVR >= N.

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER
  ILO and IHI are integer values such that on exit
  A(i,j) = 0 and B(i,j) = 0 if i > j and
  j = 1,...,ILO-1 or i = IHI+1,...,N.
  If BALANC = 'N' or 'S', ILO = 1 and IHI = N.

LSCALE
  LSCALE is DOUBLE PRECISION array, dimension (N)
  Details of the permutations and scaling factors applied
  to the left side of A and B.  If PL(j) is the index of the
  row interchanged with row j, and DL(j) is the scaling
  factor applied to row j, then
  LSCALE(j) = PL(j)  for j = 1,...,ILO-1
            = DL(j)  for j = ILO,...,IHI
            = PL(j)  for j = IHI+1,...,N.
            The order in which the interchanges are made is N to IHI+1,
            then 1 to ILO-1.

RSCALE
  RSCALE is DOUBLE PRECISION array, dimension (N)
  Details of the permutations and scaling factors applied
  to the right side of A and B.  If PR(j) is the index of the
  column interchanged with column j, and DR(j) is the scaling
  factor applied to column j, then
  RSCALE(j) = PR(j)  for j = 1,...,ILO-1
            = DR(j)  for j = ILO,...,IHI
            = PR(j)  for j = IHI+1,...,N
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.
```

```
ABNRM
  ABNRM is DOUBLE PRECISION
  The one-norm of the balanced matrix A.

BBNRM
  BBNRM is DOUBLE PRECISION
  The one-norm of the balanced matrix B.

RCONDE
  RCONDE is DOUBLE PRECISION array, dimension (N)
  If SENSE = 'E' or 'B', the reciprocal condition numbers of
  the eigenvalues, stored in consecutive elements of the array.
  For a complex conjugate pair of eigenvalues two consecutive
  elements of RCONDE are set to the same value. Thus RCONDE(j),
  RCONDV(j), and the j-th columns of VL and VR all correspond
  to the j-th eigenpair.
  If SENSE = 'N or 'V', RCONDE is not referenced.

RCONDV
  RCONDV is DOUBLE PRECISION array, dimension (N)
  If SENSE = 'V' or 'B', the estimated reciprocal condition
  numbers of the eigenvectors, stored in consecutive elements
  of the array. For a complex eigenvector two consecutive
  elements of RCONDV are set to the same value. If the
  eigenvalues cannot be reordered to compute RCONDV(j),
  RCONDV(j) is set to 0; this can only occur when the true
  value would be very small anyway.
  If SENSE = 'N' or 'E', RCONDV is not referenced.

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK >= max(1,2*N).
  If BALANC = 'S' or 'B', or JOBVL = 'V', or JOBVR = 'V',
  LWORK >= max(1,6*N).
  If SENSE = 'E' or 'B', LWORK >= max(1,10*N).
  If SENSE = 'V' or 'B', LWORK >= 2*N*N+8*N+16.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (N+6)
  If SENSE = 'E', IWORK is not referenced.
```

```
 BWORK
   BWORK is LOGICAL array, dimension (N)
   If SENSE = 'N', BWORK is not referenced.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value.
   = 1,...,N:
         The QZ iteration failed.  No eigenvectors have been
         calculated, but ALPHAR(j), ALPHAI(j), and BETA(j)
         should be correct for j=INFO+1,...,N.
   > N:  =N+1: other than QZ iteration failed in DHGEQZ.
         =N+2: error return from DTGEVC.

Further Details:
   Balancing a matrix pair (A,B) includes, first, permuting rows and
   columns to isolate eigenvalues, second, applying diagonal similarity
   transformation to the rows and columns to make the rows and columns
   as close in norm as possible. The computed reciprocal condition
   numbers correspond to the balanced matrix. Permuting rows and columns
   will not change the condition numbers (in exact arithmetic) but
   diagonal scaling will.  For further explanation of balancing, see
   section 4.11.1.2 of LAPACK Users' Guide.

   An approximate error bound on the chordal distance between the i-th
   computed generalized eigenvalue w and the corresponding exact
   eigenvalue lambda is

   chord(w, lambda) <= EPS * norm(ABNRM, BBNRM) / RCONDE(I)

   An approximate error bound for the angle between the i-th computed
   eigenvector VL(i) or VR(i) is given by

   EPS * norm(ABNRM, BBNRM) / DIF(i).

   For further explanation of the reciprocal condition numbers RCONDE
   and RCONDV, see section 4.11 of LAPACK User's Guide.
```

———-

24.12.3 fortran code

— dggevx.f —

```
*  =====================================================================
      SUBROUTINE DGGEVX( BALANC, JOBVL, JOBVR, SENSE, N, A, LDA, B, LDB,
```

```
     $                   ALPHAR, ALPHAI, BETA, VL, LDVL, VR, LDVR, ILO,
     $                   IHI, LSCALE, RSCALE, ABNRM, BBNRM, RCONDE,
     $                   RCONDV, WORK, LWORK, IWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          BALANC, JOBVL, JOBVR, SENSE
      INTEGER            IHI, ILO, INFO, LDA, LDB, LDVL, LDVR, LWORK, N
      DOUBLE PRECISION   ABNRM, BBNRM
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), ALPHAI( * ), ALPHAR( * ),
     $                   B( LDB, * ), BETA( * ), LSCALE( * ),
     $                   RCONDE( * ), RCONDV( * ), RSCALE( * ),
     $                   VL( LDVL, * ), VR( LDVR, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ILASCL, ILBSCL, ILV, ILVL, ILVR, LQUERY, NOSCL,
     $                   PAIR, WANTSB, WANTSE, WANTSN, WANTSV
      CHARACTER          CHTEMP
      INTEGER            I, ICOLS, IERR, IJOBVL, IJOBVR, IN, IROWS,
     $                   ITAU, IWRK, IWRK1, J, JC, JR, M, MAXWRK,
     $                   MINWRK, MM
      DOUBLE PRECISION   ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS,
     $                   SMLNUM, TEMP
*     ..
*     .. Local Arrays ..
      LOGICAL            LDUMMA( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEQRF, DGGBAK, DGGBAL, DGGHRD, DHGEQZ, DLABAD,
     $                   DLACPY, DLASCL, DLASET, DORGQR, DORMQR, DTGEVC,
     $                   DTGSNA, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
```

```
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           LSAME, ILAENV, DLAMCH, DLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVL, 'N' ) ) THEN
         IJOBVL = 1
         ILVL = .FALSE.
      ELSE IF( LSAME( JOBVL, 'V' ) ) THEN
         IJOBVL = 2
         ILVL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVL = .FALSE.
      END IF
*
      IF( LSAME( JOBVR, 'N' ) ) THEN
         IJOBVR = 1
         ILVR = .FALSE.
      ELSE IF( LSAME( JOBVR, 'V' ) ) THEN
         IJOBVR = 2
         ILVR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVR = .FALSE.
      END IF
      ILV = ILVL .OR. ILVR
*
      NOSCL  = LSAME( BALANC, 'N' ) .OR. LSAME( BALANC, 'P' )
      WANTSN = LSAME( SENSE, 'N' )
      WANTSE = LSAME( SENSE, 'E' )
      WANTSV = LSAME( SENSE, 'V' )
      WANTSB = LSAME( SENSE, 'B' )
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.( LSAME( BALANC, 'N' ) .OR. LSAME( BALANC,
     $    'S' ) .OR. LSAME( BALANC, 'P' ) .OR. LSAME( BALANC, 'B' ) ) )
     $     THEN
         INFO = -1
      ELSE IF( IJOBVL.LE.0 ) THEN
         INFO = -2
      ELSE IF( IJOBVR.LE.0 ) THEN
```

```
         INFO = -3
      ELSE IF( .NOT.( WANTSN .OR. WANTSE .OR. WANTSB .OR. WANTSV ) )
     $          THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDVL.LT.1 .OR. ( ILVL .AND. LDVL.LT.N ) ) THEN
         INFO = -14
      ELSE IF( LDVR.LT.1 .OR. ( ILVR .AND. LDVR.LT.N ) ) THEN
         INFO = -16
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV. The workspace is
*       computed assuming ILO = 1 and IHI = N, the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            MAXWRK = 1
         ELSE
            IF( NOSCL .AND. .NOT.ILV ) THEN
               MINWRK = 2*N
            ELSE
               MINWRK = 6*N
            END IF
            IF( WANTSE .OR. WANTSB ) THEN
               MINWRK = 10*N
            END IF
            IF( WANTSV .OR. WANTSB ) THEN
               MINWRK = MAX( MINWRK, 2*N*( N + 4 ) + 16 )
            END IF
            MAXWRK = MINWRK
            MAXWRK = MAX( MAXWRK,
     $                    N + N*ILAENV( 1, 'DGEQRF', ' ', N, 1, N, 0 ) )
            MAXWRK = MAX( MAXWRK,
     $                    N + N*ILAENV( 1, 'DORMQR', ' ', N, 1, N, 0 ) )
            IF( ILVL ) THEN
               MAXWRK = MAX( MAXWRK, N +
     $                       N*ILAENV( 1, 'DORGQR', ' ', N, 1, N, 0 ) )
            END IF
         END IF
```

```
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -26
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGGEVX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = DLANGE( 'M', N, N, A, LDA, WORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
      IF( ILASCL )
     $   CALL DLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = DLANGE( 'M', N, N, B, LDB, WORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
```

```
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
     $   CALL DLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute and/or balance the matrix pair (A,B)
*     (Workspace: need 6*N if BALANC = 'S' or 'B', 1 otherwise)
*
      CALL DGGBAL( BALANC, N, A, LDA, B, LDB, ILO, IHI, LSCALE, RSCALE,
     $             WORK, IERR )
*
*     Compute ABNRM and BBNRM
*
      ABNRM = DLANGE( '1', N, N, A, LDA, WORK( 1 ) )
      IF( ILASCL ) THEN
         WORK( 1 ) = ABNRM
         CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, 1, 1, WORK( 1 ), 1,
     $                IERR )
         ABNRM = WORK( 1 )
      END IF
*
      BBNRM = DLANGE( '1', N, N, B, LDB, WORK( 1 ) )
      IF( ILBSCL ) THEN
         WORK( 1 ) = BBNRM
         CALL DLASCL( 'G', 0, 0, BNRMTO, BNRM, 1, 1, WORK( 1 ), 1,
     $                IERR )
         BBNRM = WORK( 1 )
      END IF
*
*     Reduce B to triangular form (QR decomposition of B)
*     (Workspace: need N, prefer N*NB )
*
      IROWS = IHI + 1 - ILO
      IF( ILV .OR. .NOT.WANTSN ) THEN
         ICOLS = N + 1 - ILO
      ELSE
         ICOLS = IROWS
      END IF
      ITAU = 1
      IWRK = ITAU + IROWS
      CALL DGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to A
*     (Workspace: need N, prefer N*NB)
*
      CALL DORMQR( 'L', 'T', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
```

```
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VL and/or VR
*     (Workspace: need N, prefer N*NB)
*
      IF( ILVL ) THEN
         CALL DLASET( 'Full', N, N, ZERO, ONE, VL, LDVL )
         IF( IROWS.GT.1 ) THEN
            CALL DLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VL( ILO+1, ILO ), LDVL )
         END IF
         CALL DORGQR( IROWS, IROWS, IROWS, VL( ILO, ILO ), LDVL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
      IF( ILVR )
     $   CALL DLASET( 'Full', N, N, ZERO, ONE, VR, LDVR )
*
*     Reduce to generalized Hessenberg form
*     (Workspace: none needed)
*
      IF( ILV .OR. .NOT.WANTSN ) THEN
*
*        Eigenvectors requested -- work on whole matrix.
*
         CALL DGGHRD( JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB, VL,
     $                LDVL, VR, LDVR, IERR )
      ELSE
         CALL DGGHRD( 'N', 'N', IROWS, 1, IROWS, A( ILO, ILO ), LDA,
     $                B( ILO, ILO ), LDB, VL, LDVL, VR, LDVR, IERR )
      END IF
*
*     Perform QZ algorithm (Compute eigenvalues, and optionally, the
*     Schur forms and Schur vectors)
*     (Workspace: need N)
*
      IF( ILV .OR. .NOT.WANTSN ) THEN
         CHTEMP = 'S'
      ELSE
         CHTEMP = 'E'
      END IF
*
      CALL DHGEQZ( CHTEMP, JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHAR, ALPHAI, BETA, VL, LDVL, VR, LDVR, WORK,
     $             LWORK, IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
```

```
         ELSE
            INFO = N + 1
         END IF
         GO TO 130
      END IF
*
*     Compute Eigenvectors and estimate condition numbers if desired
*     (Workspace: DTGEVC: need 6*N
*                 DTGSNA: need 2*N*(N+2)+16 if SENSE = 'V' or 'B',
*                         need N otherwise )
*
      IF( ILV .OR. .NOT.WANTSN ) THEN
         IF( ILV ) THEN
            IF( ILVL ) THEN
               IF( ILVR ) THEN
                  CHTEMP = 'B'
               ELSE
                  CHTEMP = 'L'
               END IF
            ELSE
               CHTEMP = 'R'
            END IF
*
            CALL DTGEVC( CHTEMP, 'B', LDUMMA, N, A, LDA, B, LDB, VL,
     $                   LDVL, VR, LDVR, N, IN, WORK, IERR )
            IF( IERR.NE.0 ) THEN
               INFO = N + 2
               GO TO 130
            END IF
         END IF
*
         IF( .NOT.WANTSN ) THEN
*
*           compute eigenvectors (DTGEVC) and estimate condition
*           numbers (DTGSNA). Note that the definition of the condition
*           number is not invariant under transformation (u,v) to
*           (Q*u, Z*v), where (u,v) are eigenvectors of the generalized
*           Schur form (S,T), Q and Z are orthogonal matrices. In order
*           to avoid using extra 2*N*N workspace, we have to recalculate
*           eigenvectors and estimate one condition numbers at a time.
*
            PAIR = .FALSE.
            DO 20 I = 1, N
*
               IF( PAIR ) THEN
                  PAIR = .FALSE.
                  GO TO 20
               END IF
               MM = 1
               IF( I.LT.N ) THEN
```

```
                 IF( A( I+1, I ).NE.ZERO ) THEN
                    PAIR = .TRUE.
                    MM = 2
                 END IF
              END IF
*
              DO 10 J = 1, N
                 BWORK( J ) = .FALSE.
   10         CONTINUE
              IF( MM.EQ.1 ) THEN
                 BWORK( I ) = .TRUE.
              ELSE IF( MM.EQ.2 ) THEN
                 BWORK( I ) = .TRUE.
                 BWORK( I+1 ) = .TRUE.
              END IF
*
              IWRK = MM*N + 1
              IWRK1 = IWRK + MM*N
*
*             Compute a pair of left and right eigenvectors.
*             (compute workspace: need up to 4*N + 6*N)
*
              IF( WANTSE .OR. WANTSB ) THEN
                 CALL DTGEVC( 'B', 'S', BWORK, N, A, LDA, B, LDB,
     $                        WORK( 1 ), N, WORK( IWRK ), N, MM, M,
     $                        WORK( IWRK1 ), IERR )
                 IF( IERR.NE.0 ) THEN
                    INFO = N + 2
                    GO TO 130
                 END IF
              END IF
*
              CALL DTGSNA( SENSE, 'S', BWORK, N, A, LDA, B, LDB,
     $                     WORK( 1 ), N, WORK( IWRK ), N, RCONDE( I ),
     $                     RCONDV( I ), MM, M, WORK( IWRK1 ),
     $                     LWORK-IWRK1+1, IWORK, IERR )
*
   20       CONTINUE
         END IF
      END IF
*
*     Undo balancing on VL and VR and normalization
*     (Workspace: none needed)
*
      IF( ILVL ) THEN
         CALL DGGBAK( BALANC, 'L', N, ILO, IHI, LSCALE, RSCALE, N, VL,
     $                LDVL, IERR )
*
         DO 70 JC = 1, N
            IF( ALPHAI( JC ).LT.ZERO )
```

```
  $         GO TO 70
         TEMP = ZERO
         IF( ALPHAI( JC ).EQ.ZERO ) THEN
            DO 30 JR = 1, N
               TEMP = MAX( TEMP, ABS( VL( JR, JC ) ) )
30          CONTINUE
         ELSE
            DO 40 JR = 1, N
               TEMP = MAX( TEMP, ABS( VL( JR, JC ) )+
  $                   ABS( VL( JR, JC+1 ) ) )
40          CONTINUE
         END IF
         IF( TEMP.LT.SMLNUM )
  $         GO TO 70
         TEMP = ONE / TEMP
         IF( ALPHAI( JC ).EQ.ZERO ) THEN
            DO 50 JR = 1, N
               VL( JR, JC ) = VL( JR, JC )*TEMP
50          CONTINUE
         ELSE
            DO 60 JR = 1, N
               VL( JR, JC ) = VL( JR, JC )*TEMP
               VL( JR, JC+1 ) = VL( JR, JC+1 )*TEMP
60          CONTINUE
         END IF
70    CONTINUE
   END IF
   IF( ILVR ) THEN
      CALL DGGBAK( BALANC, 'R', N, ILO, IHI, LSCALE, RSCALE, N, VR,
  $                LDVR, IERR )
      DO 120 JC = 1, N
         IF( ALPHAI( JC ).LT.ZERO )
  $         GO TO 120
         TEMP = ZERO
         IF( ALPHAI( JC ).EQ.ZERO ) THEN
            DO 80 JR = 1, N
               TEMP = MAX( TEMP, ABS( VR( JR, JC ) ) )
80          CONTINUE
         ELSE
            DO 90 JR = 1, N
               TEMP = MAX( TEMP, ABS( VR( JR, JC ) )+
  $                   ABS( VR( JR, JC+1 ) ) )
90          CONTINUE
         END IF
         IF( TEMP.LT.SMLNUM )
  $         GO TO 120
         TEMP = ONE / TEMP
         IF( ALPHAI( JC ).EQ.ZERO ) THEN
            DO 100 JR = 1, N
               VR( JR, JC ) = VR( JR, JC )*TEMP
```

```
  100          CONTINUE
            ELSE
               DO 110 JR = 1, N
                  VR( JR, JC ) = VR( JR, JC )*TEMP
                  VR( JR, JC+1 ) = VR( JR, JC+1 )*TEMP
  110          CONTINUE
            END IF
  120    CONTINUE
      END IF
*
*     Undo scaling if necessary
*
  130 CONTINUE
*
      IF( ILASCL ) THEN
         CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAR, N, IERR )
         CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAI, N, IERR )
      END IF
*
      IF( ILBSCL ) THEN
         CALL DLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of DGGEVX
*
      END
```

———-

24.12.4 lisp code

— **dggevx.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dggevx
         (balanc jobvl jobvr sense n a lda b ldb$ alphar alphai beta vl ldvl vr
          ldvr ilo ihi lscale rscale abnrm bbnrm rconde rcondv work lwork iwork
          bwork info)
    (declare (type (array f2cl-lib:logical (*)) bwork)
             (type (array f2cl-lib:integer4 (*)) iwork)
             (type (double-float) bbnrm abnrm)
             (type (array double-float (*)) work rcondv rconde rscale lscale vr
```

```
            vl beta alphai alphar b a)
           (type (f2cl-lib:integer4) info lwork ihi ilo ldvr ldvl ldb$ lda n)
           (type (string *) sense jobvr jobvl balanc))
  (f2cl-lib:with-multi-array-data
      ((balanc character balanc-%data% balanc-%offset%)
       (jobvl character jobvl-%data% jobvl-%offset%)
       (jobvr character jobvr-%data% jobvr-%offset%)
       (sense character sense-%data% sense-%offset%)
       (a double-float a-%data% a-%offset%)
       (b double-float b-%data% b-%offset%)
       (alphar double-float alphar-%data% alphar-%offset%)
       (alphai double-float alphai-%data% alphai-%offset%)
       (beta double-float beta-%data% beta-%offset%)
       (vl double-float vl-%data% vl-%offset%)
       (vr double-float vr-%data% vr-%offset%)
       (lscale double-float lscale-%data% lscale-%offset%)
       (rscale double-float rscale-%data% rscale-%offset%)
       (rconde double-float rconde-%data% rconde-%offset%)
       (rcondv double-float rcondv-%data% rcondv-%offset%)
       (work double-float work-%data% work-%offset%)
       (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%)
       (bwork f2cl-lib:logical bwork-%data% bwork-%offset%))
    (prog ((ldumma (make-array 1 :element-type 't)) (anrm 0.0d0)
           (anrmto 0.0d0) (bignum 0.0d0) (bnrm 0.0d0) (bnrmto 0.0d0)
           (eps 0.0d0) (smlnum 0.0d0) (temp 0.0d0) (i 0) (icols 0) (ierr 0)
           (ijobvl 0) (ijobvr 0) (in 0) (irows 0) (itau 0) (iwrk 0) (iwrk1 0)
           (j 0) (jc 0) (jr 0) (m 0) (maxwrk 0) (minwrk 0) (mm 0)
           (chtemp
            (make-array '(1) :element-type 'character :initial-element #\ ))
           (ilascl nil) (ilbscl nil) (ilv nil) (ilvl nil) (ilvr nil)
           (lquery nil) (noscl nil) (pair nil) (wantsb nil) (wantse nil)
           (wantsn nil) (wantsv nil))
      (declare (type (array f2cl-lib:logical (1)) ldumma)
               (type (double-float) anrm anrmto bignum bnrm bnrmto eps smlnum
                temp)
               (type (f2cl-lib:integer4) i icols ierr ijobvl ijobvr in irows
                itau iwrk iwrk1 j jc jr m maxwrk minwrk mm)
               (type (string 1) chtemp)
               (type f2cl-lib:logical ilascl ilbscl ilv ilvl ilvr lquery
                noscl pair wantsb wantse wantsn wantsv))
      (cond
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvl "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvl var-0))
          ret-val)
        (setf ijobvl 1) (setf ilvl f2cl-lib:%false%))
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvl "V")
          (declare (ignore var-1))
```

```
    (when var-0 (setf jobvl var-0))
    ret-val)
  (setf ijobvl 2) (setf ilvl f2cl-lib:%true%))
 (t (setf ijobvl -1) (setf ilvl f2cl-lib:%false%)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvr "N")
    (declare (ignore var-1))
    (when var-0 (setf jobvr var-0))
    ret-val)
  (setf ijobvr 1) (setf ilvr f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvr "V")
    (declare (ignore var-1))
    (when var-0 (setf jobvr var-0))
    ret-val)
  (setf ijobvr 2) (setf ilvr f2cl-lib:%true%))
 (t (setf ijobvr -1) (setf ilvr f2cl-lib:%false%)))
(setf ilv (or ilvl ilvr))
(setf noscl
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame balanc "N")
           (declare (ignore var-1))
           (when var-0 (setf balanc var-0))
           ret-val)
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame balanc "P")
           (declare (ignore var-1))
           (when var-0 (setf balanc var-0))
           ret-val)))
(setf wantsn
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "N")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wantse
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "E")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wantsv
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "V")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wantsb
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "B")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf info 0)
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond
 ((not
   (or
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "N")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "S")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "P")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "B")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)))
  (setf info -1))
 ((<= ijobvl 0) (setf info -2)) ((<= ijobvr 0) (setf info -3))
 ((not (or wantsn wantse wantsb wantsv)) (setf info -4))
 ((< n 0) (setf info -5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9))
 ((or (< ldvl 1) (and ilvl (< ldvl n))) (setf info -14))
 ((or (< ldvr 1) (and ilvr (< ldvr n))) (setf info -16)))
(cond
 ((= info 0)
  (cond ((= n 0) (setf minwrk 1) (setf maxwrk 1))
        (t
         (cond
          ((and noscl (not ilv)) (setf minwrk (f2cl-lib:int-mul 2 n)))
          (t (setf minwrk (f2cl-lib:int-mul 6 n))))
         (cond
          ((or wantse wantsb) (setf minwrk (f2cl-lib:int-mul 10 n))))
         (cond
```

```
 ((or wantsv wantsb)
  (setf minwrk
          (max (the f2cl-lib:integer4 minwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add
                     (f2cl-lib:int-mul 2 n
                                       (f2cl-lib:int-add n 4))
                     16))))))
(setf maxwrk minwrk)
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add n
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "DGEQRF"
                                                           " "
                                                           n 1
                                                           n
                                                           0)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-4
                                                          var-6))
                                                        (when
                                                            var-3
                                                          (setf n
                                                                var-3))
                                                        (when
                                                            var-5
                                                          (setf n
                                                                var-5))
                                                        ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add n
```

```
                                                    (f2cl-lib:int-mul n
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                         (ilaenv
                                                                          1
                                                                          "DORMQR"
                                                                          " "
                                                                          n 1
                                                                          n
                                                                          0)
                                                                        (declare
                                                                         (ignore
                                                                         var-0
                                                                         var-1
                                                                         var-2
                                                                         var-4
                                                                         var-6))
                                                                        (when
                                                                            var-3
                                                                         (setf n
                                                                                 var-3))
                                                                        (when
                                                                            var-5
                                                                         (setf n
                                                                                 var-5))
                                                                        ret-val))))))
                                  (cond
                                   (ilvl
                                    (setf maxwrk
                                            (max (the f2cl-lib:integer4 maxwrk)
                                                 (the f2cl-lib:integer4
                                                      (f2cl-lib:int-add n
                                                                        (f2cl-lib:int-mul n
                                                                                          (multiple-value-bind
                                                                                              (ret-val
                                                                                               var-0
                                                                                               var-1
                                                                                               var-2
                                                                                               var-3
                                                                                               var-4
                                                                                               var-5
                                                                                               var-6)
                                                                                              (ilaenv
```

```
                                                                    1
                                                                    "DORGQR"
                                                                    " "
                                                                    n
                                                                    1
                                                                    n
                                                                    0)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-4
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf n
                                                                            var-3))
                                                                  (when
                                                                      var-5
                                                                    (setf n
                                                                            var-5))
                                                                  ret-val)))))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 maxwrk) 'double-float))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -26)))))
(cond
 ((/= info 0) (xerbla "DGGEVX" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(setf eps (dlamch "P"))
(setf smlnum (dlamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "M" n n a lda work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
```

```
(setf ilascl f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf anrmto smlnum)
  (setf ilascl f2cl-lib:%true%))
 ((> anrm bignum) (setf anrmto bignum) (setf ilascl f2cl-lib:%true%)))
(if ilascl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 anrm anrmto n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf anrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "M" n n b ldb$ work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ilbscl f2cl-lib:%false%)
(cond
 ((and (> bnrm zero) (< bnrm smlnum)) (setf bnrmto smlnum)
  (setf ilbscl f2cl-lib:%true%))
 ((> bnrm bignum) (setf bnrmto bignum) (setf ilbscl f2cl-lib:%true%)))
(if ilbscl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 bnrm bnrmto n n b ldb$ ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bnrm var-3))
      (when var-4 (setf bnrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf ierr var-9))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11)
    (dggbal balanc n a lda b ldb$ ilo ihi lscale rscale work ierr)
  (declare (ignore var-1 var-2 var-4 var-8 var-9 var-10))
  (setf balanc var-0)
  (setf lda var-3)
  (setf ldb$ var-5)
  (setf ilo var-6)
```

```
  (setf ihi var-7)
  (setf ierr var-11))
(setf abnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "1" n n a lda
             (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                                   work-%offset%))
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(cond
 (ilascl
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) abnrm)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 anrmto anrm 1 1
       (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                             work-%offset%)
       1 ierr)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
    (when var-3 (setf anrmto var-3))
    (when var-4 (setf anrm var-4))
    (when var-9 (setf ierr var-9)))
  (setf abnrm (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%))))
(setf bbnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "1" n n b ldb$
             (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                                   work-%offset%))
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(cond
 (ilbscl
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) bbnrm)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 bnrmto bnrm 1 1
       (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                             work-%offset%)
       1 ierr)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
    (when var-3 (setf bnrmto var-3))
    (when var-4 (setf bnrm var-4))
```

```
    (when var-9 (setf ierr var-9)))
  (setf bbnrm (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%))))
(setf irows (f2cl-lib:int-sub (f2cl-lib:int-add ihi 1) ilo))
(cond
 ((or ilv (not wantsn))
  (setf icols (f2cl-lib:int-sub (f2cl-lib:int-add n 1) ilo)))
 (t (setf icols irows)))
(setf itau 1)
(setf iwrk (f2cl-lib:int-add itau irows))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dgeqrf irows icols
     (f2cl-lib:array-slice b-%data% double-float (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf irows var-0)
  (setf icols var-1)
  (setf ldb$ var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (dormqr "L" "T" irows icols irows
     (f2cl-lib:array-slice b-%data% double-float (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice a-%data% double-float (ilo ilo)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf irows var-2))
  (when var-3 (setf icols var-3))
  (when var-4 (setf irows var-4))
  (when var-6 (setf ldb$ var-6))
  (when var-9 (setf lda var-9))
  (when var-12 (setf ierr var-12)))
(cond
 (ilvl
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "Full" n n zero one vl ldvl)
```

```
    (declare (ignore var-0 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldvl var-6)))
  (cond
   ((> irows 1)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "L" (f2cl-lib:int-sub irows 1)
         (f2cl-lib:int-sub irows 1)
         (f2cl-lib:array-slice b-%data% double-float ((+ ilo 1) ilo)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice vl-%data% double-float ((+ ilo 1) ilo)
                               ((1 ldvl) (1 *)) vl-%offset%)
         ldvl)
      (declare (ignore var-0 var-1 var-2 var-3 var-5))
      (when var-4 (setf ldb$ var-4))
      (when var-6 (setf ldvl var-6)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dorgqr irows irows irows
       (f2cl-lib:array-slice vl-%data% double-float (ilo ilo)
                             ((1 ldvl) (1 *)) vl-%offset%)
       ldvl
       (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf irows var-0))
    (when var-1 (setf irows var-1))
    (when var-2 (setf irows var-2))
    (when var-4 (setf ldvl var-4))
    (when var-8 (setf ierr var-8)))))
(if ilvr
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "Full" n n zero one vr ldvr)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldvr var-6))))
(cond
 ((or ilv (not wantsn))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
        var-10 var-11 var-12 var-13)
       (dgghrd jobvl jobvr n ilo ihi a lda b ldb$ vl ldvl vr ldvr ierr)
     (declare (ignore var-5 var-7 var-9 var-11))
     (when var-0 (setf jobvl var-0))
     (when var-1 (setf jobvr var-1))
     (when var-2 (setf n var-2))
     (when var-3 (setf ilo var-3))
     (when var-4 (setf ihi var-4))
     (when var-6 (setf lda var-6))
     (when var-8 (setf ldb$ var-8))
     (when var-10 (setf ldvl var-10))
     (when var-12 (setf ldvr var-12))
     (when var-13 (setf ierr var-13))))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13)
       (dgghrd "N" "N" irows 1 irows
        (f2cl-lib:array-slice a-%data% double-float (ilo ilo)
                              ((1 lda) (1 *)) a-%offset%)
        lda
        (f2cl-lib:array-slice b-%data% double-float (ilo ilo)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$ vl ldvl vr ldvr ierr)
     (declare (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11))
     (when var-2 (setf irows var-2))
     (when var-4 (setf irows var-4))
     (when var-6 (setf lda var-6))
     (when var-8 (setf ldb$ var-8))
     (when var-10 (setf ldvl var-10))
     (when var-12 (setf ldvr var-12))
     (when var-13 (setf ierr var-13)))))
 (cond
  ((or ilv (not wantsn))
   (f2cl-lib:f2cl-set-string chtemp "S" (string 1)))
  (t (f2cl-lib:f2cl-set-string chtemp "E" (string 1))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
      var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19)
     (dhgeqz chtemp jobvl jobvr n ilo ihi a lda b ldb$ alphar alphai
      beta vl ldvl vr ldvr work lwork ierr)
   (declare
    (ignore var-6 var-8 var-10 var-11 var-12 var-13 var-15 var-17))
   (when var-0 (setf chtemp var-0))
   (when var-1 (setf jobvl var-1))
   (when var-2 (setf jobvr var-2))
   (when var-3 (setf n var-3))
   (when var-4 (setf ilo var-4))
   (when var-5 (setf ihi var-5))
   (when var-7 (setf lda var-7))
```

```
  (when var-9 (setf ldb$ var-9))
  (when var-14 (setf ldvl var-14))
  (when var-16 (setf ldvr var-16))
  (when var-18 (setf lwork var-18))
  (when var-19 (setf ierr var-19)))
(cond
 ((/= ierr 0)
  (cond ((and (> ierr 0) (<= ierr n)) (setf info ierr))
        ((and (> ierr n) (<= ierr (f2cl-lib:int-mul 2 n)))
         (setf info (f2cl-lib:int-sub ierr n)))
        (t (setf info (f2cl-lib:int-add n 1))))
  (go label130)))
(cond
 ((or ilv (not wantsn))
  (cond
   (ilv
    (cond
     (ilvl
      (cond (ilvr (f2cl-lib:f2cl-set-string chtemp "B" (string 1)))
            (t (f2cl-lib:f2cl-set-string chtemp "L" (string 1)))))
     (t (f2cl-lib:f2cl-set-string chtemp "R" (string 1))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14 var-15)
        (dtgevc chtemp "B" ldumma n a lda b ldb$ vl ldvl vr ldvr n in
         work ierr)
      (declare (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-14))
      (when var-0 (setf chtemp var-0))
      (when var-3 (setf n var-3))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldb$ var-7))
      (when var-9 (setf ldvl var-9))
      (when var-11 (setf ldvr var-11))
      (when var-12 (setf n var-12))
      (when var-13 (setf in var-13))
      (when var-15 (setf ierr var-15)))
    (cond
     ((/= ierr 0) (setf info (f2cl-lib:int-add n 2)) (go label130)))))
  (cond
   ((not wantsn) (setf pair f2cl-lib:%false%)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (cond (pair (setf pair f2cl-lib:%false%) (go label20)))
        (setf mm 1)
        (cond
         ((< i n)
          (cond
           ((/=
             (f2cl-lib:fref a ((f2cl-lib:int-add i 1) i)
```

```
                    ((1 lda) (1 *)))
     zero)
    (setf pair f2cl-lib:%true%) (setf mm 2)))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref bwork-%data% (j) ((1 *))
                         bwork-%offset%)
            f2cl-lib:%false%)
   label10))
(cond
 ((= mm 1)
  (setf (f2cl-lib:fref bwork-%data% (i) ((1 *)) bwork-%offset%)
          f2cl-lib:%true%))
 ((= mm 2)
  (setf (f2cl-lib:fref bwork-%data% (i) ((1 *)) bwork-%offset%)
          f2cl-lib:%true%)
  (setf (f2cl-lib:fref bwork-%data% ((f2cl-lib:int-add i 1))
                       ((1 *)) bwork-%offset%)
          f2cl-lib:%true%)))
(setf iwrk (f2cl-lib:int-add (f2cl-lib:int-mul mm n) 1))
(setf iwrk1 (f2cl-lib:int-add iwrk (f2cl-lib:int-mul mm n)))
(cond
 ((or wantse wantsb)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13 var-14 var-15)
      (dtgevc "B" "S" bwork n a lda b ldb$
       (f2cl-lib:array-slice work-%data% double-float (1)
                             ((1 *)) work-%offset%)
       n
       (f2cl-lib:array-slice work-%data% double-float (iwrk)
                             ((1 *)) work-%offset%)
       n mm m
       (f2cl-lib:array-slice work-%data% double-float (iwrk1)
                             ((1 *)) work-%offset%)
       ierr)
    (declare
     (ignore var-0 var-1 var-2 var-4 var-6 var-8 var-10
      var-14))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldb$ var-7))
    (when var-9 (setf n var-9))
    (when var-11 (setf n var-11))
    (when var-12 (setf mm var-12))
    (when var-13 (setf m var-13))
    (when var-15 (setf ierr var-15)))
  (cond
   ((/= ierr 0) (setf info (f2cl-lib:int-add n 2))
```

```
            (go label130)))))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
             var-17 var-18 var-19)
            (dtgsna sense "S" bwork n a lda b ldb$
             (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                                   work-%offset%)
             n
             (f2cl-lib:array-slice work-%data% double-float (iwrk)
                                   ((1 *)) work-%offset%)
             n
             (f2cl-lib:array-slice rconde-%data% double-float (i)
                                   ((1 *)) rconde-%offset%)
             (f2cl-lib:array-slice rcondv-%data% double-float (i)
                                   ((1 *)) rcondv-%offset%)
             mm m
             (f2cl-lib:array-slice work-%data% double-float (iwrk1)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk1) 1) iwork
             ierr)
          (declare
           (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-12 var-13
            var-16 var-17 var-18))
          (when var-0 (setf sense var-0))
          (when var-3 (setf n var-3))
          (when var-5 (setf lda var-5))
          (when var-7 (setf ldb$ var-7))
          (when var-9 (setf n var-9))
          (when var-11 (setf n var-11))
          (when var-14 (setf mm var-14))
          (when var-15 (setf m var-15))
          (when var-19 (setf ierr var-19)))
      label20))))))
(cond
 (ilvl
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dggbak balanc "L" n ilo ihi lscale rscale n vl ldvl ierr)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-8))
    (setf balanc var-0)
    (setf n var-7)
    (setf ldvl var-9)
    (setf ierr var-10))
  (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                ((> jc n) nil)
    (tagbody
      (if (< (f2cl-lib:fref alphai-%data% (jc) ((1 *)) alphai-%offset%)
             zero)
```

```
    (go label70))
(setf temp zero)
(cond
 ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
      (setf temp
              (max temp
                   (abs
                    (f2cl-lib:fref vl-%data% (jr jc)
                                   ((1 ldvl) (1 *))
                                   vl-%offset%))))
     label30)))
 (t
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
      (setf temp
              (max temp
                   (+
                    (abs
                     (f2cl-lib:fref vl-%data% (jr jc)
                                    ((1 ldvl) (1 *))
                                    vl-%offset%))
                    (abs
                     (f2cl-lib:fref vl-%data%
                                    (jr (f2cl-lib:int-add jc 1))
                                    ((1 ldvl) (1 *))
                                    vl-%offset%)))))
     label40))))
(if (< temp smlnum)
    (go label70))
(setf temp (/ one temp))
(cond
 ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
      (setf (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                           vl-%offset%)
              (*
               (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                              vl-%offset%)
               temp))
     label50)))
 (t
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
```

```
            (setf (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                                 vl-%offset%)
                  (*
                   (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                                  vl-%offset%)
                   temp))
            (setf (f2cl-lib:fref vl-%data% (jr (f2cl-lib:int-add jc 1))
                                 ((1 ldvl) (1 *)) vl-%offset%)
                  (*
                   (f2cl-lib:fref vl-%data%
                                  (jr (f2cl-lib:int-add jc 1))
                                  ((1 ldvl) (1 *)) vl-%offset%)
                   temp))
          label60))))
     label70))))
(cond
 (ilvr
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dggbak balanc "R" n ilo ihi lscale rscale n vr ldvr ierr)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-8))
    (setf balanc var-0)
    (setf n var-7)
    (setf ldvr var-9)
    (setf ierr var-10))
  (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                ((> jc n) nil)
    (tagbody
      (if (< (f2cl-lib:fref alphai-%data% (jc) ((1 *)) alphai-%offset%)
             zero)
          (go label120))
      (setf temp zero)
      (cond
       ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr n) nil)
          (tagbody
            (setf temp
                  (max temp
                       (abs
                        (f2cl-lib:fref vr-%data% (jr jc)
                                       ((1 ldvr) (1 *))
                                       vr-%offset%))))
          label80)))
       (t
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr n) nil)
          (tagbody
            (setf temp
```

```
                    (max temp
                         (+
                          (abs
                           (f2cl-lib:fref vr-%data% (jr jc)
                                          ((1 ldvr) (1 *))
                                          vr-%offset%))
                          (abs
                           (f2cl-lib:fref vr-%data%
                                          (jr (f2cl-lib:int-add jc 1))
                                          ((1 ldvr) (1 *))
                                          vr-%offset%)))))
             label90))))
        (if (< temp smlnum)
            (go label120))
        (setf temp (/ one temp))
        (cond
         ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                        ((> jr n) nil)
            (tagbody
              (setf (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                   vr-%offset%)
                    (*
                     (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                    vr-%offset%)
                     temp))
             label100)))
         (t
          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                        ((> jr n) nil)
            (tagbody
              (setf (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                   vr-%offset%)
                    (*
                     (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                    vr-%offset%)
                     temp))
              (setf (f2cl-lib:fref vr-%data% (jr (f2cl-lib:int-add jc 1))
                                   ((1 ldvr) (1 *)) vr-%offset%)
                    (*
                     (f2cl-lib:fref vr-%data%
                                    (jr (f2cl-lib:int-add jc 1))
                                    ((1 ldvr) (1 *)) vr-%offset%)
                     temp))
             label110))))
      label120))))
label130
 (cond
  (ilascl
   (multiple-value-bind
```

```
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (dlascl "G" 0 0 anrmto anrm n 1 alphar n ierr)
            (declare (ignore var-0 var-1 var-2 var-6 var-7))
            (when var-3 (setf anrmto var-3))
            (when var-4 (setf anrm var-4))
            (when var-5 (setf n var-5))
            (when var-8 (setf n var-8))
            (when var-9 (setf ierr var-9)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (dlascl "G" 0 0 anrmto anrm n 1 alphai n ierr)
            (declare (ignore var-0 var-1 var-2 var-6 var-7))
            (when var-3 (setf anrmto var-3))
            (when var-4 (setf anrm var-4))
            (when var-5 (setf n var-5))
            (when var-8 (setf n var-8))
            (when var-9 (setf ierr var-9)))))
        (cond
         (ilbscl
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (dlascl "G" 0 0 bnrmto bnrm n 1 beta n ierr)
            (declare (ignore var-0 var-1 var-2 var-6 var-7))
            (when var-3 (setf bnrmto var-3))
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf n var-5))
            (when var-8 (setf n var-8))
            (when var-9 (setf ierr var-9)))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 maxwrk) 'double-float))
        (go end_label)
       end_label
        (return
         (values balanc jobvl jobvr sense n nil lda nil ldb$ nil nil nil nil
                 ldvl nil ldvr ilo ihi nil nil abnrm bbnrm nil nil nil lwork
                 nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dggevx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
```

```
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (double-float) (double-float)
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (array fortran-to-lisp::logical
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::balanc
  fortran-to-lisp::jobvl
  fortran-to-lisp::jobvr
  fortran-to-lisp::sense
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$ nil nil nil
  nil fortran-to-lisp::ldvl nil
  fortran-to-lisp::ldvr
  fortran-to-lisp::ilo
  fortran-to-lisp::ihi nil nil
  fortran-to-lisp::abnrm
  fortran-to-lisp::bbnrm nil nil
  nil fortran-to-lisp::lwork nil
  nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::dggbak
  fortran-to-lisp::dgeqrf
  fortran-to-lisp::dggbal))))
```

Chapter 25

LAPACK - General Matrices, Eigenvalue , Real

25.1 sgees

25.1.1 Axiom unit tests

— sgees.input —

```
)set break resume
)sys rm -f sgees.output
)spool sgees.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

25.1.2 Axiom help page

— sgees.help —

```
==========================================================================
sgees examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgees (JOBVS, SORT, SELECT, N, A, LDA, SDIM, WR, WI, VS,
           LDVS, WORK, LWORK, BWORK, INFO)
            SGEES computes the eigenvalues, the Schur form, and, optionally,
           the matrix of Schur vectors for GE matrices

Function/Subroutine Documentation
   subroutine sgees (character JOBVS, character SORT, external SELECT, integer
       N, real, dimension( lda, * ) A, integer LDA, integer SDIM, real,
       dimension( * ) WR, real, dimension( * ) WI, real, dimension( ldvs, * )
       VS, integer LDVS, real, dimension( * ) WORK, integer LWORK, logical,
       dimension( * ) BWORK, integer INFO)
        SGEES computes the eigenvalues, the Schur form, and, optionally, the
       matrix of Schur vectors for GE matrices

Purpose:
   SGEES computes for an N-by-N real nonsymmetric matrix A, the
   eigenvalues, the real Schur form T, and, optionally, the matrix of
   Schur vectors Z.  This gives the Schur factorization A = Z*T*(Z**T).

   Optionally, it also orders the eigenvalues on the diagonal of the
   real Schur form so that selected eigenvalues are at the top left.
   The leading columns of Z then form an orthonormal basis for the
   invariant subspace corresponding to the selected eigenvalues.

   A matrix is in real Schur form if it is upper quasi-triangular with
   1-by-1 and 2-by-2 blocks. 2-by-2 blocks will be standardized in the form
         [  a  b  ]
         [  c  a  ]
   where b*c < 0. The eigenvalues of such a block are a +- sqrt(bc).

Parameters:

 JOBVS
   JOBVS is CHARACTER*1
   = 'N': Schur vectors are not computed;
   = 'V': Schur vectors are computed.

 SORT
   SORT is CHARACTER*1
   Specifies whether or not to order the eigenvalues on the
   diagonal of the Schur form.
   = 'N': Eigenvalues are not ordered;
   = 'S': Eigenvalues are ordered (see SELECT).
```

```
SELECT
  SELECT is LOGICAL FUNCTION of two REAL arguments
  SELECT must be declared EXTERNAL in the calling subroutine.
  If SORT = 'S', SELECT is used to select eigenvalues to sort
  to the top left of the Schur form.
  If SORT = 'N', SELECT is not referenced.
  An eigenvalue WR(j)+sqrt(-1)*WI(j) is selected if
  SELECT(WR(j),WI(j)) is true; i.e., if either one of a complex
  conjugate pair of eigenvalues is selected, then both complex
  eigenvalues are selected.
  Note that a selected complex eigenvalue may no longer
  satisfy SELECT(WR(j),WI(j)) = .TRUE. after ordering, since
  ordering may change the value of complex eigenvalues
  (especially if the eigenvalue is ill-conditioned); in this
  case INFO is set to N+2 (see INFO below).

N
  N is INTEGER
  The order of the matrix A. N >= 0.

A
  A is REAL array, dimension (LDA,N)
  On entry, the N-by-N matrix A.
  On exit, A has been overwritten by its real Schur form T.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

SDIM
  SDIM is INTEGER
  If SORT = 'N', SDIM = 0.
  If SORT = 'S', SDIM = number of eigenvalues (after sorting)
          for which SELECT is true. (Complex conjugate
          pairs for which SELECT is true for either
          eigenvalue count as 2.)

WR
  WR is REAL array, dimension (N)

WI
  WI is REAL array, dimension (N)
  WR and WI contain the real and imaginary parts,
  respectively, of the computed eigenvalues in the same order
  that they appear on the diagonal of the output Schur form T.
  Complex conjugate pairs of eigenvalues will appear
  consecutively with the eigenvalue having the positive
  imaginary part first.
```

```
VS
  VS is REAL array, dimension (LDVS,N)
  If JOBVS = 'V', VS contains the orthogonal matrix Z of Schur vectors.
  If JOBVS = 'N', VS is not referenced.

LDVS
  LDVS is INTEGER
  The leading dimension of the array VS.  LDVS >= 1; if
  JOBVS = 'V', LDVS >= N.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) contains the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,3*N).
  For good performance, LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

BWORK
  BWORK is LOGICAL array, dimension (N)
  Not referenced if SORT = 'N'.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value.
  > 0: if INFO = i, and i is
    <= N: the QR algorithm failed to compute all the
          eigenvalues; elements 1:ILO-1 and i+1:N of WR and WI
          contain those eigenvalues which have converged; if
          JOBVS = 'V', VS contains the matrix which reduces A
          to its partially converged Schur form.
      = N+1: the eigenvalues could not be reordered because some
             eigenvalues were too close to separate (the problem
             is very ill-conditioned);
      = N+2: after reordering, roundoff changed values of some
             complex eigenvalues so that leading eigenvalues in
             the Schur form no longer satisfy SELECT=.TRUE.  This
             could also be caused by underflow due to scaling.
```

———-

25.1.3 fortran code

— sgees.f —

```
*  =====================================================================
      SUBROUTINE SGEES( JOBVS, SORT, SELECT, N, A, LDA, SDIM, WR, WI,
     $                  VS, LDVS, WORK, LWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVS, SORT
      INTEGER            INFO, LDA, LDVS, LWORK, N, SDIM
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      REAL               A( LDA, * ), VS( LDVS, * ), WI( * ), WORK( * ),
     $                   WR( * )
*     ..
*     .. Function Arguments ..
      LOGICAL            SELECT
      EXTERNAL           SELECT
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            CURSL, LASTSL, LQUERY, LST2SL, SCALEA, WANTST,
     $                   WANTVS
      INTEGER            HSWORK, I, I1, I2, IBAL, ICOND, IERR, IEVAL,
     $                   IHI, ILO, INXT, IP, ITAU, IWRK, MAXWRK, MINWRK
      REAL               ANRM, BIGNUM, CSCALE, EPS, S, SEP, SMLNUM
*     ..
*     .. Local Arrays ..
      INTEGER            IDUM( 1 )
      REAL               DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SGEBAK, SGEBAL, SGEHRD, SHSEQR, SLABAD,
     $                   SLACPY, SLASCL, SORGHR, SSWAP, STRSEN, XERBLA
*     ..
*     .. External Functions ..
```

```
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               SLAMCH, SLANGE
      EXTERNAL           LSAME, ILAENV, SLAMCH, SLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      WANTVS = LSAME( JOBVS, 'V' )
      WANTST = LSAME( SORT, 'S' )
      IF( ( .NOT.WANTVS ) .AND. ( .NOT.LSAME( JOBVS, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDVS.LT.1 .OR. ( WANTVS .AND. LDVS.LT.N ) ) THEN
         INFO = -11
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by SHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            MAXWRK = 1
         ELSE
            MAXWRK = 2*N + N*ILAENV( 1, 'SGEHRD', ' ', N, 1, N, 0 )
            MINWRK = 3*N
*
            CALL SHSEQR( 'S', JOBVS, N, 1, N, A, LDA, WR, WI, VS, LDVS,
     $             WORK, -1, IEVAL )
            HSWORK = WORK( 1 )
*
```

```
            IF( .NOT.WANTVS ) THEN
               MAXWRK = MAX( MAXWRK, N + HSWORK )
            ELSE
               MAXWRK = MAX( MAXWRK, 2*N + ( N - 1 )*ILAENV( 1,
     $                       'SORGHR', ' ', N, 1, N, -1 ) )
               MAXWRK = MAX( MAXWRK, N + HSWORK )
            END IF
         END IF
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -13
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEES ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = SLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
```

```
     $   CALL SLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*     Permute the matrix to make it more nearly triangular
*     (Workspace: need N)
*
      IBAL = 1
      CALL SGEBAL( 'P', N, A, LDA, ILO, IHI, WORK( IBAL ), IERR )
*
*     Reduce to upper Hessenberg form
*     (Workspace: need 3*N, prefer 2*N+N*NB)
*
      ITAU = N + IBAL
      IWRK = N + ITAU
      CALL SGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
      IF( WANTVS ) THEN
*
*        Copy Householder vectors to VS
*
         CALL SLACPY( 'L', N, N, A, LDA, VS, LDVS )
*
*        Generate orthogonal matrix in VS
*        (Workspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*
         CALL SORGHR( N, ILO, IHI, VS, LDVS, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
      END IF
*
      SDIM = 0
*
*     Perform QR iteration, accumulating Schur vectors in VS if desired
*     (Workspace: need N+1, prefer N+HSWORK (see comments) )
*
      IWRK = ITAU
      CALL SHSEQR( 'S', JOBVS, N, ILO, IHI, A, LDA, WR, WI, VS, LDVS,
     $             WORK( IWRK ), LWORK-IWRK+1, IEVAL )
      IF( IEVAL.GT.0 )
     $   INFO = IEVAL
*
*     Sort eigenvalues if desired
*
      IF( WANTST .AND. INFO.EQ.0 ) THEN
         IF( SCALEA ) THEN
            CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, N, 1, WR, N, IERR )
            CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, N, 1, WI, N, IERR )
         END IF
         DO 10 I = 1, N
            BWORK( I ) = SELECT( WR( I ), WI( I ) )
   10    CONTINUE
```

```
*
*        Reorder eigenvalues and transform Schur vectors
*        (Workspace: none needed)
*
         CALL STRSEN( 'N', JOBVS, BWORK, N, A, LDA, VS, LDVS, WR, WI,
     $                SDIM, S, SEP, WORK( IWRK ), LWORK-IWRK+1, IDUM, 1,
     $                ICOND )
         IF( ICOND.GT.0 )
     $      INFO = N + ICOND
      END IF
*
      IF( WANTVS ) THEN
*
*        Undo balancing
*        (Workspace: need N)
*
         CALL SGEBAK( 'P', 'R', N, ILO, IHI, WORK( IBAL ), N, VS, LDVS,
     $                IERR )
      END IF
*
      IF( SCALEA ) THEN
*
*        Undo scaling for the Schur form of A
*
         CALL SLASCL( 'H', 0, 0, CSCALE, ANRM, N, N, A, LDA, IERR )
         CALL SCOPY( N, A, LDA+1, WR, 1 )
         IF( CSCALE.EQ.SMLNUM ) THEN
*
*           If scaling back towards underflow, adjust WI if an
*           offdiagonal element of a 2-by-2 block in the Schur form
*           underflows.
*
            IF( IEVAL.GT.0 ) THEN
               I1 = IEVAL + 1
               I2 = IHI - 1
               CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WI,
     $                      MAX( ILO-1, 1 ), IERR )
            ELSE IF( WANTST ) THEN
               I1 = 1
               I2 = N - 1
            ELSE
               I1 = ILO
               I2 = IHI - 1
            END IF
            INXT = I1 - 1
            DO 20 I = I1, I2
               IF( I.LT.INXT )
     $            GO TO 20
               IF( WI( I ).EQ.ZERO ) THEN
                  INXT = I + 1
```

```
            ELSE
               IF( A( I+1, I ).EQ.ZERO ) THEN
                  WI( I ) = ZERO
                  WI( I+1 ) = ZERO
               ELSE IF( A( I+1, I ).NE.ZERO .AND. A( I, I+1 ).EQ.
     $                 ZERO ) THEN
                  WI( I ) = ZERO
                  WI( I+1 ) = ZERO
                  IF( I.GT.1 )
     $               CALL SSWAP( I-1, A( 1, I ), 1, A( 1, I+1 ), 1 )
                  IF( N.GT.I+1 )
     $               CALL SSWAP( N-I-1, A( I, I+2 ), LDA,
     $                           A( I+1, I+2 ), LDA )
                  IF( WANTVS ) THEN
                     CALL SSWAP( N, VS( 1, I ), 1, VS( 1, I+1 ), 1 )
                  END IF
                  A( I, I+1 ) = A( I+1, I )
                  A( I+1, I ) = ZERO
               END IF
               INXT = I + 2
            END IF
   20       CONTINUE
         END IF
*
*        Undo scaling for the imaginary part of the eigenvalues
*
         CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, N-IEVAL, 1,
     $                WI( IEVAL+1 ), MAX( N-IEVAL, 1 ), IERR )
      END IF
*
      IF( WANTST .AND. INFO.EQ.0 ) THEN
*
*        Check if reordering successful
*
         LASTSL = .TRUE.
         LST2SL = .TRUE.
         SDIM = 0
         IP = 0
         DO 30 I = 1, N
            CURSL = SELECT( WR( I ), WI( I ) )
            IF( WI( I ).EQ.ZERO ) THEN
               IF( CURSL )
     $            SDIM = SDIM + 1
               IP = 0
               IF( CURSL .AND. .NOT.LASTSL )
     $            INFO = N + 2
            ELSE
               IF( IP.EQ.1 ) THEN
*
*                 Last eigenvalue of conjugate pair
```

```
*
                 CURSL = CURSL .OR. LASTSL
                 LASTSL = CURSL
                 IF( CURSL )
     $              SDIM = SDIM + 2
                 IP = -1
                 IF( CURSL .AND. .NOT.LST2SL )
     $              INFO = N + 2
              ELSE
*
*                First eigenvalue of conjugate pair
*
                 IP = 1
              END IF
           END IF
           LST2SL = LASTSL
           LASTSL = CURSL
   30    CONTINUE
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of SGEES
*
      END
```

———-

25.1.4 lisp code

— **sgees.lisp** —

———-

25.2 sgeesx

25.2.1 Axiom unit tests

— **sgeesx.input** —

```
)set break resume
```

```
)sys rm -f sgeesx.output
)spool sgeesx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

25.2.2 Axiom help page

— sgeesx.help —

```
=====================================================================
sgeesx examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgeesx (JOBVS, SORT, SELECT, SENSE, N, A, LDA, SDIM, WR, WI,
           VS, LDVS, RCONDE, RCONDV, WORK, LWORK, IWORK, LIWORK, BWORK, INFO)
            SGEESX computes the eigenvalues, the Schur form, and, optionally,
           the matrix of Schur vectors for GE matrices

Function/Subroutine Documentation
   subroutine sgeesx (character JOBVS, character SORT, external SELECT,
       character SENSE, integer N, real, dimension( lda, * ) A, integer LDA,
       integer SDIM, real, dimension( * ) WR, real, dimension( * ) WI, real,
       dimension( ldvs, * ) VS, integer LDVS, real RCONDE, real RCONDV, real,
       dimension( * ) WORK, integer LWORK, integer, dimension( * ) IWORK,
       integer LIWORK, logical, dimension( * ) BWORK, integer INFO)
        SGEESX computes the eigenvalues, the Schur form, and, optionally, the
       matrix of Schur vectors for GE matrices

Purpose:
   SGEESX computes for an N-by-N real nonsymmetric matrix A, the
   eigenvalues, the real Schur form T, and, optionally, the matrix of
   Schur vectors Z.  This gives the Schur factorization A = Z*T*(Z**T).

   Optionally, it also orders the eigenvalues on the diagonal of the
   real Schur form so that selected eigenvalues are at the top left;
   computes a reciprocal condition number for the average of the
```

```
  selected eigenvalues (RCONDE); and computes a reciprocal condition
  number for the right invariant subspace corresponding to the
  selected eigenvalues (RCONDV).  The leading columns of Z form an
  orthonormal basis for this invariant subspace.

  For further explanation of the reciprocal condition numbers RCONDE
  and RCONDV, see Section 4.10 of the LAPACK Users' Guide (where
  these quantities are called s and sep respectively).

  A real matrix is in real Schur form if it is upper quasi-triangular
  with 1-by-1 and 2-by-2 blocks. 2-by-2 blocks will be standardized in
  the form
          [  a  b  ]
          [  c  a  ]
  where b*c < 0. The eigenvalues of such a block are a +- sqrt(bc).

Parameters:

 JOBVS
   JOBVS is CHARACTER*1
   = 'N': Schur vectors are not computed;
   = 'V': Schur vectors are computed.

 SORT
   SORT is CHARACTER*1
   Specifies whether or not to order the eigenvalues on the
   diagonal of the Schur form.
   = 'N': Eigenvalues are not ordered;
   = 'S': Eigenvalues are ordered (see SELECT).

 SELECT
   SELECT is procedure) LOGICAL FUNCTION of two REAL arguments
   SELECT must be declared EXTERNAL in the calling subroutine.
   If SORT = 'S', SELECT is used to select eigenvalues to sort
   to the top left of the Schur form.
   If SORT = 'N', SELECT is not referenced.
   An eigenvalue WR(j)+sqrt(-1)*WI(j) is selected if
   SELECT(WR(j),WI(j)) is true; i.e., if either one of a
   complex conjugate pair of eigenvalues is selected, then both
   are.  Note that a selected complex eigenvalue may no longer
   satisfy SELECT(WR(j),WI(j)) = .TRUE. after ordering, since
   ordering may change the value of complex eigenvalues
   (especially if the eigenvalue is ill-conditioned); in this
   case INFO may be set to N+3 (see INFO below).

 SENSE
   SENSE is CHARACTER*1
   Determines which reciprocal condition numbers are computed.
   = 'N': None are computed;
   = 'E': Computed for average of selected eigenvalues only;
```

```
  = 'V': Computed for selected right invariant subspace only;
  = 'B': Computed for both.
  If SENSE = 'E', 'V' or 'B', SORT must equal 'S'.

N
  N is INTEGER
  The order of the matrix A. N >= 0.

A
  A is REAL array, dimension (LDA, N)
  On entry, the N-by-N matrix A.
  On exit, A is overwritten by its real Schur form T.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

SDIM
  SDIM is INTEGER
  If SORT = 'N', SDIM = 0.
  If SORT = 'S', SDIM = number of eigenvalues (after sorting)
          for which SELECT is true. (Complex conjugate
          pairs for which SELECT is true for either
          eigenvalue count as 2.)

WR
  WR is REAL array, dimension (N)

WI
  WI is REAL array, dimension (N)
  WR and WI contain the real and imaginary parts, respectively,
  of the computed eigenvalues, in the same order that they
  appear on the diagonal of the output Schur form T.  Complex
  conjugate pairs of eigenvalues appear consecutively with the
  eigenvalue having the positive imaginary part first.

VS
  VS is REAL array, dimension (LDVS,N)
  If JOBVS = 'V', VS contains the orthogonal matrix Z of Schur vectors.
  If JOBVS = 'N', VS is not referenced.

LDVS
  LDVS is INTEGER
  The leading dimension of the array VS.  LDVS >= 1, and if
  JOBVS = 'V', LDVS >= N.

RCONDE
  RCONDE is REAL
  If SENSE = 'E' or 'B', RCONDE contains the reciprocal
  condition number for the average of the selected eigenvalues.
```

```
  Not referenced if SENSE = 'N' or 'V'.

RCONDV
  RCONDV is REAL
  If SENSE = 'V' or 'B', RCONDV contains the reciprocal
  condition number for the selected right invariant subspace.
  Not referenced if SENSE = 'N' or 'E'.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,3*N).
  Also, if SENSE = 'E' or 'V' or 'B',
  LWORK >= N+2*SDIM*(N-SDIM), where SDIM is the number of
  selected eigenvalues computed by this routine.  Note that
  N+2*SDIM*(N-SDIM) <= N+N*N/2. Note also that an error is only
  returned if LWORK < max(1,3*N), but if SENSE = 'E' or 'V' or
  'B' this may not be large enough.
  For good performance, LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates upper bounds on the optimal sizes of the
  arrays WORK and IWORK, returns these values as the first
  entries of the WORK and IWORK arrays, and no error messages
  related to LWORK or LIWORK are issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (MAX(1,LIWORK))
  On exit, if INFO = 0, IWORK(1) returns the optimal LIWORK.

LIWORK
  LIWORK is INTEGER
  The dimension of the array IWORK.
  LIWORK >= 1; if SENSE = 'V' or 'B', LIWORK >= SDIM*(N-SDIM).
  Note that SDIM*(N-SDIM) <= N*N/4. Note also that an error is
  only returned if LIWORK < 1, but if SENSE = 'V' or 'B' this
  may not be large enough.

  If LIWORK = -1, then a workspace query is assumed; the
  routine only calculates upper bounds on the optimal sizes of
  the arrays WORK and IWORK, returns these values as the first
  entries of the WORK and IWORK arrays, and no error messages
  related to LWORK or LIWORK are issued by XERBLA.

BWORK
  BWORK is LOGICAL array, dimension (N)
  Not referenced if SORT = 'N'.
```

```
 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value.
   > 0: if INFO = i, and i is
     <= N: the QR algorithm failed to compute all the
           eigenvalues; elements 1:ILO-1 and i+1:N of WR and WI
           contain those eigenvalues which have converged; if
           JOBVS = 'V', VS contains the transformation which
           reduces A to its partially converged Schur form.
        = N+1: the eigenvalues could not be reordered because some
               eigenvalues were too close to separate (the problem
               is very ill-conditioned);
        = N+2: after reordering, roundoff changed values of some
               complex eigenvalues so that leading eigenvalues in
               the Schur form no longer satisfy SELECT=.TRUE.  This
               could also be caused by underflow due to scaling.
```

25.2.3 fortran code

— sgeesx.f —

```
*  ===================================================================
      SUBROUTINE SGEESX( JOBVS, SORT, SELECT, SENSE, N, A, LDA, SDIM,
     $                   WR, WI, VS, LDVS, RCONDE, RCONDV, WORK, LWORK,
     $                   IWORK, LIWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVS, SENSE, SORT
      INTEGER            INFO, LDA, LDVS, LIWORK, LWORK, N, SDIM
      REAL               RCONDE, RCONDV
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      INTEGER            IWORK( * )
      REAL               A( LDA, * ), VS( LDVS, * ), WI( * ), WORK( * ),
     $                   WR( * )
*     ..
*     .. Function Arguments ..
```

```
      LOGICAL            SELECT
      EXTERNAL           SELECT
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            CURSL, LASTSL, LQUERY, LST2SL, SCALEA, WANTSB,
     $                   WANTSE, WANTSN, WANTST, WANTSV, WANTVS
      INTEGER            HSWORK, I, I1, I2, IBAL, ICOND, IERR, IEVAL,
     $                   IHI, ILO, INXT, IP, ITAU, IWRK, LWRK, LIWRK,
     $                   MAXWRK, MINWRK
      REAL               ANRM, BIGNUM, CSCALE, EPS, SMLNUM
*     ..
*     .. Local Arrays ..
      REAL               DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SGEBAK, SGEBAL, SGEHRD, SHSEQR, SLABAD,
     $                   SLACPY, SLASCL, SORGHR, SSWAP, STRSEN, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               SLAMCH, SLANGE
      EXTERNAL           LSAME, ILAENV, SLAMCH, SLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      WANTVS = LSAME( JOBVS, 'V' )
      WANTST = LSAME( SORT, 'S' )
      WANTSN = LSAME( SENSE, 'N' )
      WANTSE = LSAME( SENSE, 'E' )
      WANTSV = LSAME( SENSE, 'V' )
      WANTSB = LSAME( SENSE, 'B' )
      LQUERY = ( LWORK.EQ.-1 .OR. LIWORK.EQ.-1 )
*
      IF( ( .NOT.WANTVS ) .AND. ( .NOT.LSAME( JOBVS, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
```

```
         INFO = -2
      ELSE IF( .NOT.( WANTSN .OR. WANTSE .OR. WANTSV .OR. WANTSB ) .OR.
     $         ( .NOT.WANTST .AND. .NOT.WANTSN ) ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDVS.LT.1 .OR. ( WANTVS .AND. LDVS.LT.N ) ) THEN
         INFO = -12
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "RWorkspace:" describe the
*       minimal amount of real workspace needed at that point in the
*       code, as well as the preferred amount for good performance.
*       IWorkspace refers to integer workspace.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by SHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.
*       If SENSE = 'E', 'V' or 'B', then the amount of workspace needed
*       depends on SDIM, which is computed by the routine STRSEN later
*       in the code.)
*
      IF( INFO.EQ.0 ) THEN
         LIWRK = 1
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            LWRK = 1
         ELSE
            MAXWRK = 2*N + N*ILAENV( 1, 'SGEHRD', ' ', N, 1, N, 0 )
            MINWRK = 3*N
*
            CALL SHSEQR( 'S', JOBVS, N, 1, N, A, LDA, WR, WI, VS, LDVS,
     $             WORK, -1, IEVAL )
            HSWORK = WORK( 1 )
*
            IF( .NOT.WANTVS ) THEN
               MAXWRK = MAX( MAXWRK, N + HSWORK )
            ELSE
               MAXWRK = MAX( MAXWRK, 2*N + ( N - 1 )*ILAENV( 1,
     $                       'SORGHR', ' ', N, 1, N, -1 ) )
               MAXWRK = MAX( MAXWRK, N + HSWORK )
            END IF
            LWRK = MAXWRK
            IF( .NOT.WANTSN )
     $         LWRK = MAX( LWRK, N + ( N*N )/2 )
            IF( WANTSV .OR. WANTSB )
```

```
     $         LIWRK = ( N*N )/4
         END IF
         IWORK( 1 ) = LIWRK
         WORK( 1 ) = LWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -16
         ELSE IF( LIWORK.LT.1 .AND. .NOT.LQUERY ) THEN
            INFO = -18
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEESX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = SLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
     $   CALL SLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*     Permute the matrix to make it more nearly triangular
```

```
*     (RWorkspace: need N)
*
      IBAL = 1
      CALL SGEBAL( 'P', N, A, LDA, ILO, IHI, WORK( IBAL ), IERR )
*
*     Reduce to upper Hessenberg form
*     (RWorkspace: need 3*N, prefer 2*N+N*NB)
*
      ITAU = N + IBAL
      IWRK = N + ITAU
      CALL SGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
      IF( WANTVS ) THEN
*
*        Copy Householder vectors to VS
*
         CALL SLACPY( 'L', N, N, A, LDA, VS, LDVS )
*
*        Generate orthogonal matrix in VS
*        (RWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*
         CALL SORGHR( N, ILO, IHI, VS, LDVS, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
      END IF
*
      SDIM = 0
*
*     Perform QR iteration, accumulating Schur vectors in VS if desired
*     (RWorkspace: need N+1, prefer N+HSWORK (see comments) )
*
      IWRK = ITAU
      CALL SHSEQR( 'S', JOBVS, N, ILO, IHI, A, LDA, WR, WI, VS, LDVS,
     $             WORK( IWRK ), LWORK-IWRK+1, IEVAL )
      IF( IEVAL.GT.0 )
     $   INFO = IEVAL
*
*     Sort eigenvalues if desired
*
      IF( WANTST .AND. INFO.EQ.0 ) THEN
         IF( SCALEA ) THEN
            CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, N, 1, WR, N, IERR )
            CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, N, 1, WI, N, IERR )
         END IF
         DO 10 I = 1, N
            BWORK( I ) = SELECT( WR( I ), WI( I ) )
   10    CONTINUE
*
*        Reorder eigenvalues, transform Schur vectors, and compute
*        reciprocal condition numbers
```

```
*        (RWorkspace: if SENSE is not 'N', need N+2*SDIM*(N-SDIM)
*                     otherwise, need N )
*        (IWorkspace: if SENSE is 'V' or 'B', need SDIM*(N-SDIM)
*                     otherwise, need 0 )
*
         CALL STRSEN( SENSE, JOBVS, BWORK, N, A, LDA, VS, LDVS, WR, WI,
     $                SDIM, RCONDE, RCONDV, WORK( IWRK ), LWORK-IWRK+1,
     $                IWORK, LIWORK, ICOND )
         IF( .NOT.WANTSN )
     $      MAXWRK = MAX( MAXWRK, N+2*SDIM*( N-SDIM ) )
         IF( ICOND.EQ.-15 ) THEN
*
*           Not enough real workspace
*
            INFO = -16
         ELSE IF( ICOND.EQ.-17 ) THEN
*
*           Not enough integer workspace
*
            INFO = -18
         ELSE IF( ICOND.GT.0 ) THEN
*
*           STRSEN failed to reorder or to restore standard Schur form
*
            INFO = ICOND + N
         END IF
      END IF
*
      IF( WANTVS ) THEN
*
*        Undo balancing
*        (RWorkspace: need N)
*
         CALL SGEBAK( 'P', 'R', N, ILO, IHI, WORK( IBAL ), N, VS, LDVS,
     $                IERR )
      END IF
*
      IF( SCALEA ) THEN
*
*        Undo scaling for the Schur form of A
*
         CALL SLASCL( 'H', 0, 0, CSCALE, ANRM, N, N, A, LDA, IERR )
         CALL SCOPY( N, A, LDA+1, WR, 1 )
         IF( ( WANTSV .OR. WANTSB ) .AND. INFO.EQ.0 ) THEN
            DUM( 1 ) = RCONDV
            CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, 1, 1, DUM, 1, IERR )
            RCONDV = DUM( 1 )
         END IF
         IF( CSCALE.EQ.SMLNUM ) THEN
*
```

```
*          If scaling back towards underflow, adjust WI if an
*          offdiagonal element of a 2-by-2 block in the Schur form
*          underflows.
*
           IF( IEVAL.GT.0 ) THEN
              I1 = IEVAL + 1
              I2 = IHI - 1
              CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WI, N,
     $                     IERR )
           ELSE IF( WANTST ) THEN
              I1 = 1
              I2 = N - 1
           ELSE
              I1 = ILO
              I2 = IHI - 1
           END IF
           INXT = I1 - 1
           DO 20 I = I1, I2
              IF( I.LT.INXT )
     $           GO TO 20
              IF( WI( I ).EQ.ZERO ) THEN
                 INXT = I + 1
              ELSE
                 IF( A( I+1, I ).EQ.ZERO ) THEN
                    WI( I ) = ZERO
                    WI( I+1 ) = ZERO
                 ELSE IF( A( I+1, I ).NE.ZERO .AND. A( I, I+1 ).EQ.
     $                   ZERO ) THEN
                    WI( I ) = ZERO
                    WI( I+1 ) = ZERO
                    IF( I.GT.1 )
     $                 CALL SSWAP( I-1, A( 1, I ), 1, A( 1, I+1 ), 1 )
                    IF( N.GT.I+1 )
     $                 CALL SSWAP( N-I-1, A( I, I+2 ), LDA,
     $                             A( I+1, I+2 ), LDA )
                    CALL SSWAP( N, VS( 1, I ), 1, VS( 1, I+1 ), 1 )
                    A( I, I+1 ) = A( I+1, I )
                    A( I+1, I ) = ZERO
                 END IF
                 INXT = I + 2
              END IF
   20      CONTINUE
        END IF
        CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, N-IEVAL, 1,
     $               WI( IEVAL+1 ), MAX( N-IEVAL, 1 ), IERR )
     END IF
*
     IF( WANTST .AND. INFO.EQ.0 ) THEN
*
*       Check if reordering successful
```

```
*
         LASTSL = .TRUE.
         LST2SL = .TRUE.
         SDIM = 0
         IP = 0
         DO 30 I = 1, N
            CURSL = SELECT( WR( I ), WI( I ) )
            IF( WI( I ).EQ.ZERO ) THEN
               IF( CURSL )
     $            SDIM = SDIM + 1
               IP = 0
               IF( CURSL .AND. .NOT.LASTSL )
     $            INFO = N + 2
            ELSE
               IF( IP.EQ.1 ) THEN
*
*                 Last eigenvalue of conjugate pair
*
                  CURSL = CURSL .OR. LASTSL
                  LASTSL = CURSL
                  IF( CURSL )
     $               SDIM = SDIM + 2
                  IP = -1
                  IF( CURSL .AND. .NOT.LST2SL )
     $               INFO = N + 2
               ELSE
*
*                 First eigenvalue of conjugate pair
*
                  IP = 1
               END IF
            END IF
            LST2SL = LASTSL
            LASTSL = CURSL
   30    CONTINUE
      END IF
*
      WORK( 1 ) = MAXWRK
      IF( WANTSV .OR. WANTSB ) THEN
         IWORK( 1 ) = SDIM*(N-SDIM)
      ELSE
         IWORK( 1 ) = 1
      END IF
*
      RETURN
*
*     End of SGEESX
*
      END
```

———-

25.2.4 lisp code

— **sgeesx.lisp** —

———-

25.3 sgeev

25.3.1 Axiom unit tests

— **sgeev.input** —

```
)set break resume
)sys rm -f sgeev.output
)spool sgeev.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

25.3.2 Axiom help page

— **sgeev.help** —

```
======================================================================
sgeev examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
```

```
      subroutine sgeev (JOBVL, JOBVR, N, A, LDA, WR, WI, VL, LDVL, VR, LDVR,
          WORK, LWORK, INFO)
           SGEEV computes the eigenvalues and, optionally, the left and/or
          right eigenvectors for GE matrices

Function/Subroutine Documentation
   subroutine sgeev (character JOBVL, character JOBVR, integer N, real,
       dimension( lda, * ) A, integer LDA, real, dimension( * ) WR, real,
       dimension( * ) WI, real, dimension( ldvl, * ) VL, integer LDVL, real,
       dimension( ldvr, * ) VR, integer LDVR, real, dimension( * ) WORK,
       integer LWORK, integer INFO)
        SGEEV computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for GE matrices

Purpose:
   SGEEV computes for an N-by-N real nonsymmetric matrix A, the
   eigenvalues and, optionally, the left and/or right eigenvectors.

   The right eigenvector v(j) of A satisfies
            A * v(j) = lambda(j) * v(j)
   where lambda(j) is its eigenvalue.
   The left eigenvector u(j) of A satisfies
            u(j)**H * A = lambda(j) * u(j)**H
   where u(j)**H denotes the conjugate-transpose of u(j).

   The computed eigenvectors are normalized to have Euclidean norm
   equal to 1 and largest component real.

Parameters:

 JOBVL
   JOBVL is CHARACTER*1
   = 'N': left eigenvectors of A are not computed;
   = 'V': left eigenvectors of A are computed.

 JOBVR
   JOBVR is CHARACTER*1
   = 'N': right eigenvectors of A are not computed;
   = 'V': right eigenvectors of A are computed.

 N
   N is INTEGER
   The order of the matrix A. N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the N-by-N matrix A.
   On exit, A has been overwritten.

 LDA
```

```
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

WR
  WR is REAL array, dimension (N)

WI
  WI is REAL array, dimension (N)
  WR and WI contain the real and imaginary parts,
  respectively, of the computed eigenvalues.  Complex
  conjugate pairs of eigenvalues appear consecutively
  with the eigenvalue having the positive imaginary part first.

VL
  VL is REAL array, dimension (LDVL,N)
  If JOBVL = 'V', the left eigenvectors u(j) are stored one
  after another in the columns of VL, in the same order
  as their eigenvalues.
  If JOBVL = 'N', VL is not referenced.
  If the j-th eigenvalue is real, then u(j) = VL(:,j),
  the j-th column of VL.
  If the j-th and (j+1)-st eigenvalues form a complex
  conjugate pair, then u(j) = VL(:,j) + i*VL(:,j+1) and
  u(j+1) = VL(:,j) - i*VL(:,j+1).

LDVL
  LDVL is INTEGER
  The leading dimension of the array VL.  LDVL >= 1; if
  JOBVL = 'V', LDVL >= N.

VR
  VR is REAL array, dimension (LDVR,N)
  If JOBVR = 'V', the right eigenvectors v(j) are stored one
  after another in the columns of VR, in the same order
  as their eigenvalues.
  If JOBVR = 'N', VR is not referenced.
  If the j-th eigenvalue is real, then v(j) = VR(:,j),
  the j-th column of VR.
  If the j-th and (j+1)-st eigenvalues form a complex
  conjugate pair, then v(j) = VR(:,j) + i*VR(:,j+1) and
  v(j+1) = VR(:,j) - i*VR(:,j+1).

LDVR
  LDVR is INTEGER
  The leading dimension of the array VR.  LDVR >= 1; if
  JOBVR = 'V', LDVR >= N.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.
```

```
 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.  LWORK >= max(1,3*N), and
   if JOBVL = 'V' or JOBVR = 'V', LWORK >= 4*N.  For good
   performance, LWORK must generally be larger.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value.
   > 0:  if INFO = i, the QR algorithm failed to compute all the
         eigenvalues, and no eigenvectors have been computed;
         elements i+1:N of WR and WI contain eigenvalues which
         have converged.
```

———-

25.3.3 fortran code

— **sgeev.f** —

```
*  =====================================================================
      SUBROUTINE SGEEV( JOBVL, JOBVR, N, A, LDA, WR, WI, VL, LDVL, VR,
     $                  LDVR, WORK, LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVL, JOBVR
      INTEGER            INFO, LDA, LDVL, LDVR, LWORK, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   WI( * ), WORK( * ), WR( * )
*     ..
*
*  =====================================================================
*
```

```
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, SCALEA, WANTVL, WANTVR
      CHARACTER          SIDE
      INTEGER            HSWORK, I, IBAL, IERR, IHI, ILO, ITAU, IWRK, K,
     $                   MAXWRK, MINWRK, NOUT
      REAL               ANRM, BIGNUM, CS, CSCALE, EPS, R, SCL, SMLNUM,
     $                   SN
*     ..
*     .. Local Arrays ..
      LOGICAL            SELECT( 1 )
      REAL               DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEBAK, SGEBAL, SGEHRD, SHSEQR, SLABAD, SLACPY,
     $                   SLARTG, SLASCL, SORGHR, SROT, SSCAL, STREVC,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV, ISAMAX
      REAL               SLAMCH, SLANGE, SLAPY2, SNRM2
      EXTERNAL           LSAME, ILAENV, ISAMAX, SLAMCH, SLANGE, SLAPY2,
     $                   SNRM2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      WANTVL = LSAME( JOBVL, 'V' )
      WANTVR = LSAME( JOBVR, 'V' )
      IF( ( .NOT.WANTVL ) .AND. ( .NOT.LSAME( JOBVL, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTVR ) .AND. ( .NOT.LSAME( JOBVR, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDVL.LT.1 .OR. ( WANTVL .AND. LDVL.LT.N ) ) THEN
         INFO = -9
      ELSE IF( LDVR.LT.1 .OR. ( WANTVR .AND. LDVR.LT.N ) ) THEN
```

```
         INFO = -11
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by SHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            MAXWRK = 1
         ELSE
            MAXWRK = 2*N + N*ILAENV( 1, 'SGEHRD', ' ', N, 1, N, 0 )
            IF( WANTVL ) THEN
               MINWRK = 4*N
               MAXWRK = MAX( MAXWRK, 2*N + ( N - 1 )*ILAENV( 1,
     $                       'SORGHR', ' ', N, 1, N, -1 ) )
               CALL SHSEQR( 'S', 'V', N, 1, N, A, LDA, WR, WI, VL, LDVL,
     $                WORK, -1, INFO )
               HSWORK = WORK( 1 )
               MAXWRK = MAX( MAXWRK, N + 1, N + HSWORK )
               MAXWRK = MAX( MAXWRK, 4*N )
            ELSE IF( WANTVR ) THEN
               MINWRK = 4*N
               MAXWRK = MAX( MAXWRK, 2*N + ( N - 1 )*ILAENV( 1,
     $                       'SORGHR', ' ', N, 1, N, -1 ) )
               CALL SHSEQR( 'S', 'V', N, 1, N, A, LDA, WR, WI, VR, LDVR,
     $                WORK, -1, INFO )
               HSWORK = WORK( 1 )
               MAXWRK = MAX( MAXWRK, N + 1, N + HSWORK )
               MAXWRK = MAX( MAXWRK, 4*N )
            ELSE
               MINWRK = 3*N
               CALL SHSEQR( 'E', 'N', N, 1, N, A, LDA, WR, WI, VR, LDVR,
     $                WORK, -1, INFO )
               HSWORK = WORK( 1 )
               MAXWRK = MAX( MAXWRK, N + 1, N + HSWORK )
            END IF
            MAXWRK = MAX( MAXWRK, MINWRK )
         END IF
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -13
```

```
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEEV ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = SLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
     $   CALL SLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*     Balance the matrix
*     (Workspace: need N)
*
      IBAL = 1
      CALL SGEBAL( 'B', N, A, LDA, ILO, IHI, WORK( IBAL ), IERR )
*
*     Reduce to upper Hessenberg form
*     (Workspace: need 3*N, prefer 2*N+N*NB)
*
      ITAU = IBAL + N
      IWRK = ITAU + N
      CALL SGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
```

```
     $             LWORK-IWRK+1, IERR )
*
      IF( WANTVL ) THEN
*
*        Want left eigenvectors
*        Copy Householder vectors to VL
*
         SIDE = 'L'
         CALL SLACPY( 'L', N, N, A, LDA, VL, LDVL )
*
*        Generate orthogonal matrix in VL
*        (Workspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*
         CALL SORGHR( N, ILO, IHI, VL, LDVL, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VL
*        (Workspace: need N+1, prefer N+HSWORK (see comments) )
*
         IWRK = ITAU
         CALL SHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, WR, WI, VL, LDVL,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
         IF( WANTVR ) THEN
*
*           Want left and right eigenvectors
*           Copy Schur vectors to VR
*
            SIDE = 'B'
            CALL SLACPY( 'F', N, N, VL, LDVL, VR, LDVR )
         END IF
*
      ELSE IF( WANTVR ) THEN
*
*        Want right eigenvectors
*        Copy Householder vectors to VR
*
         SIDE = 'R'
         CALL SLACPY( 'L', N, N, A, LDA, VR, LDVR )
*
*        Generate orthogonal matrix in VR
*        (Workspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*
         CALL SORGHR( N, ILO, IHI, VR, LDVR, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VR
*        (Workspace: need N+1, prefer N+HSWORK (see comments) )
*
         IWRK = ITAU
```

```
         CALL SHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, WR, WI, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
      ELSE
*
*        Compute eigenvalues only
*        (Workspace: need N+1, prefer N+HSWORK (see comments) )
*
         IWRK = ITAU
         CALL SHSEQR( 'E', 'N', N, ILO, IHI, A, LDA, WR, WI, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
      END IF
*
*     If INFO > 0 from SHSEQR, then quit
*
      IF( INFO.GT.0 )
     $   GO TO 50
*
      IF( WANTVL .OR. WANTVR ) THEN
*
*        Compute left and/or right eigenvectors
*        (Workspace: need 4*N)
*
         CALL STREVC( SIDE, 'B', SELECT, N, A, LDA, VL, LDVL, VR, LDVR,
     $                N, NOUT, WORK( IWRK ), IERR )
      END IF
*
      IF( WANTVL ) THEN
*
*        Undo balancing of left eigenvectors
*        (Workspace: need N)
*
         CALL SGEBAK( 'B', 'L', N, ILO, IHI, WORK( IBAL ), N, VL, LDVL,
     $                IERR )
*
*        Normalize left eigenvectors and make largest component real
*
         DO 20 I = 1, N
            IF( WI( I ).EQ.ZERO ) THEN
               SCL = ONE / SNRM2( N, VL( 1, I ), 1 )
               CALL SSCAL( N, SCL, VL( 1, I ), 1 )
            ELSE IF( WI( I ).GT.ZERO ) THEN
               SCL = ONE / SLAPY2( SNRM2( N, VL( 1, I ), 1 ),
     $               SNRM2( N, VL( 1, I+1 ), 1 ) )
               CALL SSCAL( N, SCL, VL( 1, I ), 1 )
               CALL SSCAL( N, SCL, VL( 1, I+1 ), 1 )
               DO 10 K = 1, N
                  WORK( IWRK+K-1 ) = VL( K, I )**2 + VL( K, I+1 )**2
   10          CONTINUE
               K = ISAMAX( N, WORK( IWRK ), 1 )
```

```
               CALL SLARTG( VL( K, I ), VL( K, I+1 ), CS, SN, R )
               CALL SROT( N, VL( 1, I ), 1, VL( 1, I+1 ), 1, CS, SN )
               VL( K, I+1 ) = ZERO
            END IF
   20    CONTINUE
      END IF
*
      IF( WANTVR ) THEN
*
*        Undo balancing of right eigenvectors
*        (Workspace: need N)
*
         CALL SGEBAK( 'B', 'R', N, ILO, IHI, WORK( IBAL ), N, VR, LDVR,
     $                IERR )
*
*        Normalize right eigenvectors and make largest component real
*
         DO 40 I = 1, N
            IF( WI( I ).EQ.ZERO ) THEN
               SCL = ONE / SNRM2( N, VR( 1, I ), 1 )
               CALL SSCAL( N, SCL, VR( 1, I ), 1 )
            ELSE IF( WI( I ).GT.ZERO ) THEN
               SCL = ONE / SLAPY2( SNRM2( N, VR( 1, I ), 1 ),
     $               SNRM2( N, VR( 1, I+1 ), 1 ) )
               CALL SSCAL( N, SCL, VR( 1, I ), 1 )
               CALL SSCAL( N, SCL, VR( 1, I+1 ), 1 )
               DO 30 K = 1, N
                  WORK( IWRK+K-1 ) = VR( K, I )**2 + VR( K, I+1 )**2
   30          CONTINUE
               K = ISAMAX( N, WORK( IWRK ), 1 )
               CALL SLARTG( VR( K, I ), VR( K, I+1 ), CS, SN, R )
               CALL SROT( N, VR( 1, I ), 1, VR( 1, I+1 ), 1, CS, SN )
               VR( K, I+1 ) = ZERO
            END IF
   40    CONTINUE
      END IF
*
*     Undo scaling if necessary
*
   50 CONTINUE
      IF( SCALEA ) THEN
         CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, WR( INFO+1 ),
     $                MAX( N-INFO, 1 ), IERR )
         CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, WI( INFO+1 ),
     $                MAX( N-INFO, 1 ), IERR )
         IF( INFO.GT.0 ) THEN
            CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WR, N,
     $                   IERR )
            CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WI, N,
     $                   IERR )
```

```
         END IF
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of SGEEV
*
      END
```

———-

25.3.4 lisp code

— **sgeev.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sgeev (jobvl jobvr n a lda wr wi vl ldvl vr ldvr work lwork info)
    (declare (type (array single-float (*)) work vr vl wi wr a)
             (type (f2cl-lib:integer4) info lwork ldvr ldvl lda n)
             (type (string *) jobvr jobvl))
    (f2cl-lib:with-multi-array-data
        ((jobvl character jobvl-%data% jobvl-%offset%)
         (jobvr character jobvr-%data% jobvr-%offset%)
         (a single-float a-%data% a-%offset%)
         (wr single-float wr-%data% wr-%offset%)
         (wi single-float wi-%data% wi-%offset%)
         (vl single-float vl-%data% vl-%offset%)
         (vr single-float vr-%data% vr-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((dum (make-array 1 :element-type 'single-float))
             (select (make-array 1 :element-type 't)) (anrm 0.0) (bignum 0.0)
             (cs 0.0) (cscale 0.0) (eps 0.0) (r 0.0) (scl 0.0) (smlnum 0.0)
             (sn 0.0) (hswork 0) (i 0) (ibal 0) (ierr 0) (ihi 0) (ilo 0)
             (itau 0) (iwrk 0) (k 0) (maxwrk 0) (minwrk 0) (nout 0)
             (side
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (lquery nil) (scalea nil) (wantvl nil) (wantvr nil))
        (declare (type (array single-float (1)) dum)
                 (type (array f2cl-lib:logical (1)) select)
                 (type (single-float) anrm bignum cs cscale eps r scl smlnum
                  sn)
                 (type (f2cl-lib:integer4) hswork i ibal ierr ihi ilo itau iwrk
                  k maxwrk minwrk nout)
                 (type (string 1) side)
```

```
         (type f2cl-lib:logical lquery scalea wantvl wantvr))
(setf info 0)
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(setf wantvl
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvl "V")
          (declare (ignore var-1))
          (when var-0 (setf jobvl var-0))
          ret-val))
(setf wantvr
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvr "V")
          (declare (ignore var-1))
          (when var-0 (setf jobvr var-0))
          ret-val))
(cond
 ((and (not wantvl)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvl "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvl var-0))
          ret-val)))
  (setf info -1))
 ((and (not wantvr)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvr "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvr var-0))
          ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((or (< ldvl 1) (and wantvl (< ldvl n))) (setf info -9))
 ((or (< ldvr 1) (and wantvr (< ldvr n))) (setf info -11)))
(cond
 ((= info 0)
  (cond ((= n 0) (setf minwrk 1) (setf maxwrk 1))
        (t
         (setf maxwrk
                 (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n
                                                     (multiple-value-bind
                                                         (ret-val var-0
                                                          var-1 var-2
                                                          var-3 var-4
                                                          var-5 var-6)
                                                         (ilaenv 1
```

```
                                                        "SGEHRD" " "
                                                        n 1 n 0)
                                                    (declare
                                                     (ignore var-0
                                                      var-1 var-2
                                                      var-4 var-6))
                                                    (setf n var-3)
                                                    (setf n var-5)
                                                    ret-val))))
(cond
 (wantvl (setf minwrk (f2cl-lib:int-mul 4 n))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                      (f2cl-lib:int-mul
                                       (f2cl-lib:int-sub n 1)
                                       (multiple-value-bind
                                           (ret-val var-0
                                            var-1 var-2 var-3
                                            var-4 var-5 var-6)
                                           (ilaenv 1 "SORGHR"
                                            " " n 1 n -1)
                                         (declare
                                          (ignore var-0 var-1
                                           var-2 var-4 var-6))
                                         (setf n var-3)
                                         (setf n var-5)
                                         ret-val))))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13)
      (shseqr "S" "V" n 1 n a lda wr wi vl ldvl work -1 info)
    (declare
     (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-9 var-11
      var-12))
    (when var-2 (setf n var-2))
    (when var-4 (setf n var-4))
    (when var-6 (setf lda var-6))
    (when var-10 (setf ldvl var-10))
    (when var-13 (setf info var-13)))
  (setf hswork
          (f2cl-lib:int
           (f2cl-lib:fref work-%data% (1) ((1 *))
                          work-%offset%)))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4 (f2cl-lib:int-add n 1))
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add n hswork))))
```

```
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-mul 4 n)))))
(wantvr (setf minwrk (f2cl-lib:int-mul 4 n))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     (f2cl-lib:int-mul
                                      (f2cl-lib:int-sub n 1)
                                      (multiple-value-bind
                                          (ret-val var-0
                                           var-1 var-2 var-3
                                           var-4 var-5 var-6)
                                          (ilaenv 1 "SORGHR"
                                           " " n 1 n -1)
                                        (declare
                                         (ignore var-0 var-1
                                          var-2 var-4 var-6))
                                        (setf n var-3)
                                        (setf n var-5)
                                        ret-val))))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12 var-13)
     (shseqr "S" "V" n 1 n a lda wr wi vr ldvr work -1 info)
   (declare
    (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-9 var-11
     var-12))
   (when var-2 (setf n var-2))
   (when var-4 (setf n var-4))
   (when var-6 (setf lda var-6))
   (when var-10 (setf ldvr var-10))
   (when var-13 (setf info var-13)))
 (setf hswork
         (f2cl-lib:int
          (f2cl-lib:fref work-%data% (1) ((1 *))
                         work-%offset%)))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4 (f2cl-lib:int-add n 1))
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n hswork))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-mul 4 n)))))
(t (setf minwrk (f2cl-lib:int-mul 3 n))
 (multiple-value-bind
```

```
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10 var-11 var-12 var-13)
                (shseqr "E" "N" n 1 n a lda wr wi vr ldvr work -1 info)
              (declare
               (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-9 var-11
                var-12))
              (when var-2 (setf n var-2))
              (when var-4 (setf n var-4))
              (when var-6 (setf lda var-6))
              (when var-10 (setf ldvr var-10))
              (when var-13 (setf info var-13)))
            (setf hswork
                    (f2cl-lib:int
                     (f2cl-lib:fref work-%data% (1) ((1 *))
                                    work-%offset%)))
            (setf maxwrk
                    (max (the f2cl-lib:integer4 maxwrk)
                         (the f2cl-lib:integer4 (f2cl-lib:int-add n 1))
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add n hswork))))))
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4 minwrk)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 maxwrk) 'single-float))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -13)))))
(cond
 ((/= info 0) (xerbla "SGEEV " (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(setf eps (slamch "P"))
(setf smlnum (slamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (slabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "M" n n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
```

```
(setf scalea f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf scalea f2cl-lib:%true%)
  (setf cscale smlnum))
 ((> anrm bignum) (setf scalea f2cl-lib:%true%) (setf cscale bignum)))
(if scalea
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl "G" 0 0 anrm cscale n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf cscale var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(setf ibal 1)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (sgebal "B" n a lda ilo ihi
     (f2cl-lib:array-slice work-%data% single-float (ibal) ((1 *))
                           work-%offset%)
     ierr)
  (declare (ignore var-0 var-1 var-2 var-6))
  (setf lda var-3)
  (setf ilo var-4)
  (setf ihi var-5)
  (setf ierr var-7))
(setf itau (f2cl-lib:int-add ibal n))
(setf iwrk (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sgehrd n ilo ihi a lda
     (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf ilo var-1))
  (when var-2 (setf ihi var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(cond
 (wantvl (f2cl-lib:f2cl-set-string side "L" (string 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "L" n n a lda vl ldvl)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
```

```
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvl var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (sorghr n ilo ihi vl ldvl
       (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf n var-0))
    (when var-1 (setf ilo var-1))
    (when var-2 (setf ihi var-2))
    (when var-4 (setf ldvl var-4))
    (when var-8 (setf ierr var-8)))
  (setf iwrk itau)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (shseqr "S" "V" n ilo ihi a lda wr wi vl ldvl
       (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
    (declare
     (ignore var-0 var-1 var-5 var-7 var-8 var-9 var-11 var-12))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-10 (setf ldvl var-10))
    (when var-13 (setf info var-13)))
  (cond
   (wantvr (f2cl-lib:f2cl-set-string side "B" (string 1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slacpy "F" n n vl ldvl vr ldvr)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf ldvl var-4))
      (when var-6 (setf ldvr var-6))))))
 (wantvr (f2cl-lib:f2cl-set-string side "R" (string 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "L" n n a lda vr ldvr)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvr var-6)))
  (multiple-value-bind
```

```
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (sorghr n ilo ihi vr ldvr
         (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                               work-%offset%)
         (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                               work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
      (declare (ignore var-3 var-5 var-6 var-7))
      (when var-0 (setf n var-0))
      (when var-1 (setf ilo var-1))
      (when var-2 (setf ihi var-2))
      (when var-4 (setf ldvr var-4))
      (when var-8 (setf ierr var-8)))
    (setf iwrk itau)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (shseqr "S" "V" n ilo ihi a lda wr wi vr ldvr
         (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                               work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
      (declare
       (ignore var-0 var-1 var-5 var-7 var-8 var-9 var-11 var-12))
      (when var-2 (setf n var-2))
      (when var-3 (setf ilo var-3))
      (when var-4 (setf ihi var-4))
      (when var-6 (setf lda var-6))
      (when var-10 (setf ldvr var-10))
      (when var-13 (setf info var-13))))
   (t (setf iwrk itau)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (shseqr "E" "N" n ilo ihi a lda wr wi vr ldvr
         (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                               work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
      (declare
       (ignore var-0 var-1 var-5 var-7 var-8 var-9 var-11 var-12))
      (when var-2 (setf n var-2))
      (when var-3 (setf ilo var-3))
      (when var-4 (setf ihi var-4))
      (when var-6 (setf lda var-6))
      (when var-10 (setf ldvr var-10))
      (when var-13 (setf info var-13)))))
  (if (> info 0)
      (go label50))
  (cond
   ((or wantvl wantvr)
    (multiple-value-bind
```

```
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (strevc side "B" select n a lda vl ldvl vr ldvr n nout
         (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                               work-%offset%)
         ierr)
      (declare (ignore var-1 var-2 var-4 var-6 var-8 var-12))
      (when var-0 (setf side var-0))
      (when var-3 (setf n var-3))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldvl var-7))
      (when var-9 (setf ldvr var-9))
      (when var-10 (setf n var-10))
      (when var-11 (setf nout var-11))
      (when var-13 (setf ierr var-13)))))
  (cond
   (wantvl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (sgebak "B" "L" n ilo ihi
         (f2cl-lib:array-slice work-%data% single-float (ibal) ((1 *))
                               work-%offset%)
         n vl ldvl ierr)
      (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7))
      (setf n var-6)
      (setf ldvl var-8)
      (setf ierr var-9))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (cond
         ((= (f2cl-lib:fref wi (i) ((1 *))) zero)
          (setf scl
                  (/ one
                     (multiple-value-bind (ret-val var-0 var-1 var-2)
                         (snrm2 n
                          (f2cl-lib:array-slice vl-%data% single-float
                                                (1 i) ((1 ldvl) (1 *))
                                                vl-%offset%)
                          1)
                       (declare (ignore var-1 var-2))
                       (when var-0 (setf n var-0))
                       ret-val)))
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (sscal n scl
               (f2cl-lib:array-slice vl-%data% single-float (1 i)
                                     ((1 ldvl) (1 *)) vl-%offset%)
               1)
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
```

```
  (when var-1 (setf scl var-1))))
((> (f2cl-lib:fref wi (i) ((1 *))) zero)
 (setf scl
         (/ one
            (slapy2
             (multiple-value-bind (ret-val var-0 var-1 var-2)
                 (snrm2 n
                  (f2cl-lib:array-slice vl-%data% single-float
                                        (1 i) ((1 ldvl) (1 *))
                                        vl-%offset%)
                  1)
               (declare (ignore var-1 var-2))
               (when var-0 (setf n var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1 var-2)
                 (snrm2 n
                  (f2cl-lib:array-slice vl-%data% single-float
                                        (1
                                         (f2cl-lib:int-add i 1))
                                        ((1 ldvl) (1 *))
                                        vl-%offset%)
                  1)
               (declare (ignore var-1 var-2))
               (when var-0 (setf n var-0))
               ret-val))))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (sscal n scl
      (f2cl-lib:array-slice vl-%data% single-float (1 i)
                            ((1 ldvl) (1 *)) vl-%offset%)
      1)
   (declare (ignore var-2 var-3))
   (when var-0 (setf n var-0))
   (when var-1 (setf scl var-1)))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (sscal n scl
      (f2cl-lib:array-slice vl-%data% single-float
                            (1 (f2cl-lib:int-add i 1))
                            ((1 ldvl) (1 *)) vl-%offset%)
      1)
   (declare (ignore var-2 var-3))
   (when var-0 (setf n var-0))
   (when var-1 (setf scl var-1)))
 (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
               ((> k n) nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add iwrk k) 1))
                          ((1 *)) work-%offset%)
             (+
```

```
                (expt
                 (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                                vl-%offset%)
                 2)
                (expt
                 (f2cl-lib:fref vl-%data%
                                (k (f2cl-lib:int-add i 1))
                                ((1 ldvl) (1 *)) vl-%offset%)
                 2)))
      label10))
   (setf k
           (multiple-value-bind (ret-val var-0 var-1 var-2)
               (isamax n
                (f2cl-lib:array-slice work-%data% single-float
                                      (iwrk) ((1 *))
                                      work-%offset%)
                1)
             (declare (ignore var-1 var-2))
             (when var-0 (setf n var-0))
             ret-val))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
       (slartg
        (f2cl-lib:array-slice vl-%data% single-float (k i)
                              ((1 ldvl) (1 *)) vl-%offset%)
        (f2cl-lib:array-slice vl-%data% single-float
                              (k (f2cl-lib:int-add i 1))
                              ((1 ldvl) (1 *)) vl-%offset%)
        cs sn r)
     (declare (ignore var-0 var-1))
     (when var-2 (setf cs var-2))
     (when var-3 (setf sn var-3))
     (when var-4 (setf r var-4)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (srot n
        (f2cl-lib:array-slice vl-%data% single-float (1 i)
                              ((1 ldvl) (1 *)) vl-%offset%)
        1
        (f2cl-lib:array-slice vl-%data% single-float
                              (1 (f2cl-lib:int-add i 1))
                              ((1 ldvl) (1 *)) vl-%offset%)
        1 cs sn)
     (declare (ignore var-1 var-2 var-3 var-4))
     (when var-0 (setf n var-0))
     (when var-5 (setf cs var-5))
     (when var-6 (setf sn var-6)))
   (setf (f2cl-lib:fref vl-%data% (k (f2cl-lib:int-add i 1))
                        ((1 ldvl) (1 *)) vl-%offset%)
           zero)))
label20))))
```

```
(cond
 (wantvr
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (sgebak "B" "R" n ilo ihi
       (f2cl-lib:array-slice work-%data% single-float (ibal) ((1 *))
                             work-%offset%)
       n vr ldvr ierr)
    (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7))
    (setf n var-6)
    (setf ldvr var-8)
    (setf ierr var-9))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref wi (i) ((1 *))) zero)
        (setf scl
                (/ one
                   (multiple-value-bind (ret-val var-0 var-1 var-2)
                       (snrm2 n
                        (f2cl-lib:array-slice vr-%data% single-float
                                              (1 i) ((1 ldvr) (1 *))
                                              vr-%offset%)
                        1)
                     (declare (ignore var-1 var-2))
                     (when var-0 (setf n var-0))
                     ret-val)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal n scl
             (f2cl-lib:array-slice vr-%data% single-float (1 i)
                                   ((1 ldvr) (1 *)) vr-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-1 (setf scl var-1))))
       ((> (f2cl-lib:fref wi (i) ((1 *))) zero)
        (setf scl
                (/ one
                   (slapy2
                    (multiple-value-bind (ret-val var-0 var-1 var-2)
                        (snrm2 n
                         (f2cl-lib:array-slice vr-%data% single-float
                                               (1 i) ((1 ldvr) (1 *))
                                               vr-%offset%)
                         1)
                      (declare (ignore var-1 var-2))
                      (when var-0 (setf n var-0))
                      ret-val)
                    (multiple-value-bind (ret-val var-0 var-1 var-2)
```

```
                  (snrm2 n
                   (f2cl-lib:array-slice vr-%data% single-float
                                         (1
                                          (f2cl-lib:int-add i 1))
                                         ((1 ldvr) (1 *))
                                         vr-%offset%)
                   1)
                (declare (ignore var-1 var-2))
                (when var-0 (setf n var-0))
                ret-val))))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (sscal n scl
     (f2cl-lib:array-slice vr-%data% single-float (1 i)
                           ((1 ldvr) (1 *)) vr-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf scl var-1)))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (sscal n scl
     (f2cl-lib:array-slice vr-%data% single-float
                           (1 (f2cl-lib:int-add i 1))
                           ((1 ldvr) (1 *)) vr-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf scl var-1)))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iwrk k) 1))
                         ((1 *)) work-%offset%)
            (+
             (expt
              (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                             vr-%offset%)
              2)
             (expt
              (f2cl-lib:fref vr-%data%
                             (k (f2cl-lib:int-add i 1))
                             ((1 ldvr) (1 *)) vr-%offset%)
              2)))
   label30))
(setf k
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (isamax n
             (f2cl-lib:array-slice work-%data% single-float
                                   (iwrk) ((1 *))
```

```
                                          work-%offset%)
                      1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf n var-0))
                  ret-val))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (slartg
               (f2cl-lib:array-slice vr-%data% single-float (k i)
                                     ((1 ldvr) (1 *)) vr-%offset%)
               (f2cl-lib:array-slice vr-%data% single-float
                                     (k (f2cl-lib:int-add i 1))
                                     ((1 ldvr) (1 *)) vr-%offset%)
               cs sn r)
            (declare (ignore var-0 var-1))
            (when var-2 (setf cs var-2))
            (when var-3 (setf sn var-3))
            (when var-4 (setf r var-4)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (srot n
               (f2cl-lib:array-slice vr-%data% single-float (1 i)
                                     ((1 ldvr) (1 *)) vr-%offset%)
               1
               (f2cl-lib:array-slice vr-%data% single-float
                                     (1 (f2cl-lib:int-add i 1))
                                     ((1 ldvr) (1 *)) vr-%offset%)
               1 cs sn)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf n var-0))
            (when var-5 (setf cs var-5))
            (when var-6 (setf sn var-6)))
          (setf (f2cl-lib:fref vr-%data% (k (f2cl-lib:int-add i 1))
                               ((1 ldvr) (1 *)) vr-%offset%)
                zero)))
      label40))))
label50
 (cond
  (scalea
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (slascl "G" 0 0 cscale anrm (f2cl-lib:int-sub n info) 1
        (f2cl-lib:array-slice wr-%data% single-float ((+ info 1))
                              ((1 *)) wr-%offset%)
        (max (the f2cl-lib:integer4 (f2cl-lib:int-sub n info))
             (the f2cl-lib:integer4 1))
        ierr)
     (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
     (when var-3 (setf cscale var-3))
     (when var-4 (setf anrm var-4))
     (when var-9 (setf ierr var-9)))
```

```
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (slascl "G" 0 0 cscale anrm (f2cl-lib:int-sub n info) 1
               (f2cl-lib:array-slice wi-%data% single-float ((+ info 1))
                                     ((1 *)) wi-%offset%)
               (max (the f2cl-lib:integer4 (f2cl-lib:int-sub n info))
                    (the f2cl-lib:integer4 1))
               ierr)
            (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
            (when var-3 (setf cscale var-3))
            (when var-4 (setf anrm var-4))
            (when var-9 (setf ierr var-9)))
          (cond
           ((> info 0)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                (slascl "G" 0 0 cscale anrm (f2cl-lib:int-sub ilo 1) 1 wr n
                 ierr)
              (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
              (when var-3 (setf cscale var-3))
              (when var-4 (setf anrm var-4))
              (when var-8 (setf n var-8))
              (when var-9 (setf ierr var-9)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                (slascl "G" 0 0 cscale anrm (f2cl-lib:int-sub ilo 1) 1 wi n
                 ierr)
              (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
              (when var-3 (setf cscale var-3))
              (when var-4 (setf anrm var-4))
              (when var-8 (setf n var-8))
              (when var-9 (setf ierr var-9)))))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 maxwrk) 'single-float))
        (go end_label)
       end_label
        (return
         (values jobvl jobvr n nil lda nil nil nil ldvl nil ldvr nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgeev fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
```

```
  (array single-float (*))
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::jobvl
  fortran-to-lisp::jobvr
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil nil
  fortran-to-lisp::ldvl nil
  fortran-to-lisp::ldvr nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::sgebak
  fortran-to-lisp::sgebal
  fortran-to-lisp::ilaenv))))
```

———-

25.4 sgeevx

25.4.1 Axiom unit tests

— **sgeevx.input** —

```
)set break resume
)sys rm -f sgeevx.output
)spool sgeevx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

25.4.2 Axiom help page

— sgeevx.help —

```
=====================================================================
sgeevx examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgeevx (BALANC, JOBVL, JOBVR, SENSE, N, A, LDA, WR, WI, VL,
           LDVL, VR, LDVR, ILO, IHI, SCALE, ABNRM, RCONDE, RCONDV, WORK,
           LWORK, IWORK, INFO)
            SGEEVX computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for GE matrices

Function/Subroutine Documentation
   subroutine sgeevx (character BALANC, character JOBVL, character JOBVR,
       character SENSE, integer N, real, dimension( lda, * ) A, integer LDA,
       real, dimension( * ) WR, real, dimension( * ) WI, real, dimension(
       ldvl, * ) VL, integer LDVL, real, dimension( ldvr, * ) VR, integer
       LDVR, integer ILO, integer IHI, real, dimension( * ) SCALE, real ABNRM,
       real, dimension( * ) RCONDE, real, dimension( * ) RCONDV, real,
       dimension( * ) WORK, integer LWORK, integer, dimension( * ) IWORK,
       integer INFO)
        SGEEVX computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for GE matrices

Purpose:
   SGEEVX computes for an N-by-N real nonsymmetric matrix A, the
   eigenvalues and, optionally, the left and/or right eigenvectors.

   Optionally also, it computes a balancing transformation to improve
   the conditioning of the eigenvalues and eigenvectors (ILO, IHI,
   SCALE, and ABNRM), reciprocal condition numbers for the eigenvalues
   (RCONDE), and reciprocal condition numbers for the right
   eigenvectors (RCONDV).

   The right eigenvector v(j) of A satisfies
      A * v(j) = lambda(j) * v(j)
   where lambda(j) is its eigenvalue.
   The left eigenvector u(j) of A satisfies
      u(j)**H * A = lambda(j) * u(j)**H
   where u(j)**H denotes the conjugate-transpose of u(j).

   The computed eigenvectors are normalized to have Euclidean norm
   equal to 1 and largest component real.
```

```
  Balancing a matrix means permuting the rows and columns to make it
  more nearly upper triangular, and applying a diagonal similarity
  transformation D * A * D**(-1), where D is a diagonal matrix, to
  make its rows and columns closer in norm and the condition numbers
  of its eigenvalues and eigenvectors smaller.  The computed
  reciprocal condition numbers correspond to the balanced matrix.
  Permuting rows and columns will not change the condition numbers
  (in exact arithmetic) but diagonal scaling will.  For further
  explanation of balancing, see section 4.10.2 of the LAPACK Users' Guide.

Parameters:

 BALANC
   BALANC is CHARACTER*1
   Indicates how the input matrix should be diagonally scaled
   and/or permuted to improve the conditioning of its
   eigenvalues.
   = 'N': Do not diagonally scale or permute;
   = 'P': Perform permutations to make the matrix more nearly
          upper triangular. Do not diagonally scale;
   = 'S': Diagonally scale the matrix, i.e. replace A by
          D*A*D**(-1), where D is a diagonal matrix chosen
          to make the rows and columns of A more equal in
          norm. Do not permute;
   = 'B': Both diagonally scale and permute A.

   Computed reciprocal condition numbers will be for the matrix
   after balancing and/or permuting. Permuting does not change
   condition numbers (in exact arithmetic), but balancing does.

 JOBVL
   JOBVL is CHARACTER*1
   = 'N': left eigenvectors of A are not computed;
   = 'V': left eigenvectors of A are computed.
   If SENSE = 'E' or 'B', JOBVL must = 'V'.

 JOBVR
   JOBVR is CHARACTER*1
   = 'N': right eigenvectors of A are not computed;
   = 'V': right eigenvectors of A are computed.
   If SENSE = 'E' or 'B', JOBVR must = 'V'.

 SENSE
   SENSE is CHARACTER*1
   Determines which reciprocal condition numbers are computed.
   = 'N': None are computed;
   = 'E': Computed for eigenvalues only;
   = 'V': Computed for right eigenvectors only;
   = 'B': Computed for eigenvalues and right eigenvectors.
```

```
  If SENSE = 'E' or 'B', both left and right eigenvectors
  must also be computed (JOBVL = 'V' and JOBVR = 'V').

N
  N is INTEGER
  The order of the matrix A. N >= 0.

A
  A is REAL array, dimension (LDA,N)
  On entry, the N-by-N matrix A.
  On exit, A has been overwritten.  If JOBVL = 'V' or
  JOBVR = 'V', A contains the real Schur form of the balanced
  version of the input matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

WR
  WR is REAL array, dimension (N)

WI
  WI is REAL array, dimension (N)
  WR and WI contain the real and imaginary parts,
  respectively, of the computed eigenvalues.  Complex
  conjugate pairs of eigenvalues will appear consecutively
  with the eigenvalue having the positive imaginary part first.

VL
  VL is REAL array, dimension (LDVL,N)
  If JOBVL = 'V', the left eigenvectors u(j) are stored one
  after another in the columns of VL, in the same order
  as their eigenvalues.
  If JOBVL = 'N', VL is not referenced.
  If the j-th eigenvalue is real, then u(j) = VL(:,j),
  the j-th column of VL.
  If the j-th and (j+1)-st eigenvalues form a complex
  conjugate pair, then u(j) = VL(:,j) + i*VL(:,j+1) and
  u(j+1) = VL(:,j) - i*VL(:,j+1).

LDVL
  LDVL is INTEGER
  The leading dimension of the array VL.  LDVL >= 1; if
  JOBVL = 'V', LDVL >= N.

VR
  VR is REAL array, dimension (LDVR,N)
  If JOBVR = 'V', the right eigenvectors v(j) are stored one
  after another in the columns of VR, in the same order
  as their eigenvalues.
```

```
  If JOBVR = 'N', VR is not referenced.
  If the j-th eigenvalue is real, then v(j) = VR(:,j),
  the j-th column of VR.
  If the j-th and (j+1)-st eigenvalues form a complex
  conjugate pair, then v(j) = VR(:,j) + i*VR(:,j+1) and
  v(j+1) = VR(:,j) - i*VR(:,j+1).

LDVR
  LDVR is INTEGER
  The leading dimension of the array VR.  LDVR >= 1, and if
  JOBVR = 'V', LDVR >= N.

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER
  ILO and IHI are integer values determined when A was
  balanced.  The balanced A(i,j) = 0 if I > J and
  J = 1,...,ILO-1 or I = IHI+1,...,N.

SCALE
  SCALE is REAL array, dimension (N)
  Details of the permutations and scaling factors applied
  when balancing A.  If P(j) is the index of the row and column
  interchanged with row and column j, and D(j) is the scaling
  factor applied to row and column j, then
  SCALE(J) = P(J),    for J = 1,...,ILO-1
           = D(J),    for J = ILO,...,IHI
           = P(J)     for J = IHI+1,...,N.
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

ABNRM
  ABNRM is REAL
  The one-norm of the balanced matrix (the maximum
  of the sum of absolute values of elements of any column).

RCONDE
  RCONDE is REAL array, dimension (N)
  RCONDE(j) is the reciprocal condition number of the j-th eigenvalue.

RCONDV
  RCONDV is REAL array, dimension (N)
  RCONDV(j) is the reciprocal condition number of the j-th
  right eigenvector.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.
```

```
LWORK
  LWORK is INTEGER
  The dimension of the array WORK.   If SENSE = 'N' or 'E',
  LWORK >= max(1,2*N), and if JOBVL = 'V' or JOBVR = 'V',
  LWORK >= 3*N.  If SENSE = 'V' or 'B', LWORK >= N*(N+6).
  For good performance, LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (2*N-2)
  If SENSE = 'N' or 'E', not referenced.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if INFO = i, the QR algorithm failed to compute all the
        eigenvalues, and no eigenvectors or condition numbers
        have been computed; elements 1:ILO-1 and i+1:N of WR
        and WI contain eigenvalues which have converged.
```

———-

25.4.3 fortran code

— **sgeevx.f** —

```
*  =====================================================================
      SUBROUTINE SGEEVX( BALANC, JOBVL, JOBVR, SENSE, N, A, LDA, WR, WI,
     $                   VL, LDVL, VR, LDVR, ILO, IHI, SCALE, ABNRM,
     $                   RCONDE, RCONDV, WORK, LWORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          BALANC, JOBVL, JOBVR, SENSE
      INTEGER            IHI, ILO, INFO, LDA, LDVL, LDVR, LWORK, N
      REAL               ABNRM
*     ..
```

```
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      REAL               A( LDA, * ), RCONDE( * ), RCONDV( * ),
     $                   SCALE( * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   WI( * ), WORK( * ), WR( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, SCALEA, WANTVL, WANTVR, WNTSNB, WNTSNE,
     $                   WNTSNN, WNTSNV
      CHARACTER          JOB, SIDE
      INTEGER            HSWORK, I, ICOND, IERR, ITAU, IWRK, K, MAXWRK,
     $                   MINWRK, NOUT
      REAL               ANRM, BIGNUM, CS, CSCALE, EPS, R, SCL, SMLNUM,
     $                   SN
*     ..
*     .. Local Arrays ..
      LOGICAL            SELECT( 1 )
      REAL               DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEBAK, SGEBAL, SGEHRD, SHSEQR, SLABAD, SLACPY,
     $                   SLARTG, SLASCL, SORGHR, SROT, SSCAL, STREVC,
     $                   STRSNA, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV, ISAMAX
      REAL               SLAMCH, SLANGE, SLAPY2, SNRM2
      EXTERNAL           LSAME, ILAENV, ISAMAX, SLAMCH, SLANGE, SLAPY2,
     $                   SNRM2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      WANTVL = LSAME( JOBVL, 'V' )
      WANTVR = LSAME( JOBVR, 'V' )
      WNTSNN = LSAME( SENSE, 'N' )
```

```
      WNTSNE = LSAME( SENSE, 'E' )
      WNTSNV = LSAME( SENSE, 'V' )
      WNTSNB = LSAME( SENSE, 'B' )
      IF( .NOT.( LSAME( BALANC, 'N' ) .OR. LSAME( BALANC, 'S' ) .OR.
     $    LSAME( BALANC, 'P' ) .OR. LSAME( BALANC, 'B' ) ) ) THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTVL ) .AND. ( .NOT.LSAME( JOBVL, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( ( .NOT.WANTVR ) .AND. ( .NOT.LSAME( JOBVR, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( .NOT.( WNTSNN .OR. WNTSNE .OR. WNTSNB .OR. WNTSNV ) .OR.
     $         ( ( WNTSNE .OR. WNTSNB ) .AND. .NOT.( WANTVL .AND.
     $         WANTVR ) ) ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDVL.LT.1 .OR. ( WANTVL .AND. LDVL.LT.N ) ) THEN
         INFO = -11
      ELSE IF( LDVR.LT.1 .OR. ( WANTVR .AND. LDVR.LT.N ) ) THEN
         INFO = -13
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by SHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            MAXWRK = 1
         ELSE
            MAXWRK = N + N*ILAENV( 1, 'SGEHRD', ' ', N, 1, N, 0 )
*
            IF( WANTVL ) THEN
               CALL SHSEQR( 'S', 'V', N, 1, N, A, LDA, WR, WI, VL, LDVL,
     $                 WORK, -1, INFO )
            ELSE IF( WANTVR ) THEN
               CALL SHSEQR( 'S', 'V', N, 1, N, A, LDA, WR, WI, VR, LDVR,
     $                 WORK, -1, INFO )
            ELSE
               IF( WNTSNN ) THEN
                  CALL SHSEQR( 'E', 'N', N, 1, N, A, LDA, WR, WI, VR,
```

```
     $                LDVR, WORK, -1, INFO )
              ELSE
                 CALL SHSEQR( 'S', 'N', N, 1, N, A, LDA, WR, WI, VR,
     $                LDVR, WORK, -1, INFO )
              END IF
           END IF
           HSWORK = WORK( 1 )
*
           IF( ( .NOT.WANTVL ) .AND. ( .NOT.WANTVR ) ) THEN
              MINWRK = 2*N
              IF( .NOT.WNTSNN )
     $           MINWRK = MAX( MINWRK, N*N+6*N )
              MAXWRK = MAX( MAXWRK, HSWORK )
              IF( .NOT.WNTSNN )
     $           MAXWRK = MAX( MAXWRK, N*N + 6*N )
           ELSE
              MINWRK = 3*N
              IF( ( .NOT.WNTSNN ) .AND. ( .NOT.WNTSNE ) )
     $           MINWRK = MAX( MINWRK, N*N + 6*N )
              MAXWRK = MAX( MAXWRK, HSWORK )
              MAXWRK = MAX( MAXWRK, N + ( N - 1 )*ILAENV( 1, 'SORGHR',
     $                       ' ', N, 1, N, -1 ) )
              IF( ( .NOT.WNTSNN ) .AND. ( .NOT.WNTSNE ) )
     $           MAXWRK = MAX( MAXWRK, N*N + 6*N )
              MAXWRK = MAX( MAXWRK, 3*N )
           END IF
           MAXWRK = MAX( MAXWRK, MINWRK )
        END IF
        WORK( 1 ) = MAXWRK
*
        IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
           INFO = -21
        END IF
     END IF
*
     IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'SGEEVX', -INFO )
        RETURN
     ELSE IF( LQUERY ) THEN
        RETURN
     END IF
*
*    Quick return if possible
*
     IF( N.EQ.0 )
     $   RETURN
*
*    Get machine constants
*
     EPS = SLAMCH( 'P' )
```

```
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ICOND = 0
      ANRM = SLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
     $   CALL SLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*     Balance the matrix and compute ABNRM
*
      CALL SGEBAL( BALANC, N, A, LDA, ILO, IHI, SCALE, IERR )
      ABNRM = SLANGE( '1', N, N, A, LDA, DUM )
      IF( SCALEA ) THEN
         DUM( 1 ) = ABNRM
         CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, 1, 1, DUM, 1, IERR )
         ABNRM = DUM( 1 )
      END IF
*
*     Reduce to upper Hessenberg form
*     (Workspace: need 2*N, prefer N+N*NB)
*
      ITAU = 1
      IWRK = ITAU + N
      CALL SGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
      IF( WANTVL ) THEN
*
*        Want left eigenvectors
*        Copy Householder vectors to VL
*
         SIDE = 'L'
         CALL SLACPY( 'L', N, N, A, LDA, VL, LDVL )
*
*        Generate orthogonal matrix in VL
*        (Workspace: need 2*N-1, prefer N+(N-1)*NB)
*
```

```
         CALL SORGHR( N, ILO, IHI, VL, LDVL, WORK( ITAU ), WORK( IWRK ),
     $               LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VL
*        (Workspace: need 1, prefer HSWORK (see comments) )
*
         IWRK = ITAU
         CALL SHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, WR, WI, VL, LDVL,
     $               WORK( IWRK ), LWORK-IWRK+1, INFO )
*
         IF( WANTVR ) THEN
*
*           Want left and right eigenvectors
*           Copy Schur vectors to VR
*
            SIDE = 'B'
            CALL SLACPY( 'F', N, N, VL, LDVL, VR, LDVR )
         END IF
*
      ELSE IF( WANTVR ) THEN
*
*        Want right eigenvectors
*        Copy Householder vectors to VR
*
         SIDE = 'R'
         CALL SLACPY( 'L', N, N, A, LDA, VR, LDVR )
*
*        Generate orthogonal matrix in VR
*        (Workspace: need 2*N-1, prefer N+(N-1)*NB)
*
         CALL SORGHR( N, ILO, IHI, VR, LDVR, WORK( ITAU ), WORK( IWRK ),
     $               LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VR
*        (Workspace: need 1, prefer HSWORK (see comments) )
*
         IWRK = ITAU
         CALL SHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, WR, WI, VR, LDVR,
     $               WORK( IWRK ), LWORK-IWRK+1, INFO )
*
      ELSE
*
*        Compute eigenvalues only
*        If condition numbers desired, compute Schur form
*
         IF( WNTSNN ) THEN
            JOB = 'E'
         ELSE
            JOB = 'S'
         END IF
```

```
*
*        (Workspace: need 1, prefer HSWORK (see comments) )
*
         IWRK = ITAU
         CALL SHSEQR( JOB, 'N', N, ILO, IHI, A, LDA, WR, WI, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
      END IF
*
*     If INFO > 0 from SHSEQR, then quit
*
      IF( INFO.GT.0 )
     $   GO TO 50
*
      IF( WANTVL .OR. WANTVR ) THEN
*
*        Compute left and/or right eigenvectors
*        (Workspace: need 3*N)
*
         CALL STREVC( SIDE, 'B', SELECT, N, A, LDA, VL, LDVL, VR, LDVR,
     $                N, NOUT, WORK( IWRK ), IERR )
      END IF
*
*     Compute condition numbers if desired
*     (Workspace: need N*N+6*N unless SENSE = 'E')
*
      IF( .NOT.WNTSNN ) THEN
         CALL STRSNA( SENSE, 'A', SELECT, N, A, LDA, VL, LDVL, VR, LDVR,
     $                RCONDE, RCONDV, N, NOUT, WORK( IWRK ), N, IWORK,
     $                ICOND )
      END IF
*
      IF( WANTVL ) THEN
*
*        Undo balancing of left eigenvectors
*
         CALL SGEBAK( BALANC, 'L', N, ILO, IHI, SCALE, N, VL, LDVL,
     $                IERR )
*
*        Normalize left eigenvectors and make largest component real
*
         DO 20 I = 1, N
            IF( WI( I ).EQ.ZERO ) THEN
               SCL = ONE / SNRM2( N, VL( 1, I ), 1 )
               CALL SSCAL( N, SCL, VL( 1, I ), 1 )
            ELSE IF( WI( I ).GT.ZERO ) THEN
               SCL = ONE / SLAPY2( SNRM2( N, VL( 1, I ), 1 ),
     $               SNRM2( N, VL( 1, I+1 ), 1 ) )
               CALL SSCAL( N, SCL, VL( 1, I ), 1 )
               CALL SSCAL( N, SCL, VL( 1, I+1 ), 1 )
               DO 10 K = 1, N
```

```
                 WORK( K ) = VL( K, I )**2 + VL( K, I+1 )**2
   10          CONTINUE
               K = ISAMAX( N, WORK, 1 )
               CALL SLARTG( VL( K, I ), VL( K, I+1 ), CS, SN, R )
               CALL SROT( N, VL( 1, I ), 1, VL( 1, I+1 ), 1, CS, SN )
               VL( K, I+1 ) = ZERO
            END IF
   20    CONTINUE
      END IF
*
      IF( WANTVR ) THEN
*
*        Undo balancing of right eigenvectors
*
         CALL SGEBAK( BALANC, 'R', N, ILO, IHI, SCALE, N, VR, LDVR,
     $                IERR )
*
*        Normalize right eigenvectors and make largest component real
*
         DO 40 I = 1, N
            IF( WI( I ).EQ.ZERO ) THEN
               SCL = ONE / SNRM2( N, VR( 1, I ), 1 )
               CALL SSCAL( N, SCL, VR( 1, I ), 1 )
            ELSE IF( WI( I ).GT.ZERO ) THEN
               SCL = ONE / SLAPY2( SNRM2( N, VR( 1, I ), 1 ),
     $               SNRM2( N, VR( 1, I+1 ), 1 ) )
               CALL SSCAL( N, SCL, VR( 1, I ), 1 )
               CALL SSCAL( N, SCL, VR( 1, I+1 ), 1 )
               DO 30 K = 1, N
                  WORK( K ) = VR( K, I )**2 + VR( K, I+1 )**2
   30          CONTINUE
               K = ISAMAX( N, WORK, 1 )
               CALL SLARTG( VR( K, I ), VR( K, I+1 ), CS, SN, R )
               CALL SROT( N, VR( 1, I ), 1, VR( 1, I+1 ), 1, CS, SN )
               VR( K, I+1 ) = ZERO
            END IF
   40    CONTINUE
      END IF
*
*     Undo scaling if necessary
*
   50 CONTINUE
      IF( SCALEA ) THEN
         CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, WR( INFO+1 ),
     $                MAX( N-INFO, 1 ), IERR )
         CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, WI( INFO+1 ),
     $                MAX( N-INFO, 1 ), IERR )
         IF( INFO.EQ.0 ) THEN
            IF( ( WNTSNV .OR. WNTSNB ) .AND. ICOND.EQ.0 )
     $         CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, N, 1, RCONDV, N,
```

```
     $                      IERR )
         ELSE
            CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WR, N,
     $                   IERR )
            CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WI, N,
     $                   IERR )
         END IF
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of SGEEVX
*
      END
```

———-

25.4.4 lisp code

— **sgeevx.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sgeevx
         (balanc jobvl jobvr sense n a lda wr wi vl ldvl vr ldvr ilo ihi scale
          abnrm rconde rcondv work lwork iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (single-float) abnrm)
             (type (array single-float (*)) work rcondv rconde scale vr vl wi
              wr a)
             (type (f2cl-lib:integer4) info lwork ihi ilo ldvr ldvl lda n)
             (type (string *) sense jobvr jobvl balanc))
    (f2cl-lib:with-multi-array-data
        ((balanc character balanc-%data% balanc-%offset%)
         (jobvl character jobvl-%data% jobvl-%offset%)
         (jobvr character jobvr-%data% jobvr-%offset%)
         (sense character sense-%data% sense-%offset%)
         (a single-float a-%data% a-%offset%)
         (wr single-float wr-%data% wr-%offset%)
         (wi single-float wi-%data% wi-%offset%)
         (vl single-float vl-%data% vl-%offset%)
         (vr single-float vr-%data% vr-%offset%)
         (scale single-float scale-%data% scale-%offset%)
         (rconde single-float rconde-%data% rconde-%offset%)
         (rcondv single-float rcondv-%data% rcondv-%offset%)
```

```
   (work single-float work-%data% work-%offset%)
   (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
(prog ((dum (make-array 1 :element-type 'single-float))
       (select (make-array 1 :element-type 't)) (anrm 0.0) (bignum 0.0)
       (cs 0.0) (cscale 0.0) (eps 0.0) (r 0.0) (scl 0.0) (smlnum 0.0)
       (sn 0.0) (hswork 0) (i 0) (icond 0) (ierr 0) (itau 0) (iwrk 0)
       (k 0) (maxwrk 0) (minwrk 0) (nout 0)
       (job
        (make-array '(1) :element-type 'character :initial-element #\ ))
       (side
        (make-array '(1) :element-type 'character :initial-element #\ ))
       (lquery nil) (scalea nil) (wantvl nil) (wantvr nil) (wntsnb nil)
       (wntsne nil) (wntsnn nil) (wntsnv nil))
  (declare (type (array single-float (1)) dum)
           (type (array f2cl-lib:logical (1)) select)
           (type (single-float) anrm bignum cs cscale eps r scl smlnum
            sn)
           (type (f2cl-lib:integer4) hswork i icond ierr itau iwrk k
            maxwrk minwrk nout)
           (type (string 1) job side)
           (type f2cl-lib:logical lquery scalea wantvl wantvr wntsnb
            wntsne wntsnn wntsnv))
  (setf info 0)
  (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
  (setf wantvl
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame jobvl "V")
            (declare (ignore var-1))
            (when var-0 (setf jobvl var-0))
            ret-val))
  (setf wantvr
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame jobvr "V")
            (declare (ignore var-1))
            (when var-0 (setf jobvr var-0))
            ret-val))
  (setf wntsnn
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame sense "N")
            (declare (ignore var-1))
            (when var-0 (setf sense var-0))
            ret-val))
  (setf wntsne
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame sense "E")
            (declare (ignore var-1))
            (when var-0 (setf sense var-0))
            ret-val))
  (setf wntsnv
          (multiple-value-bind (ret-val var-0 var-1)
```

```
            (lsame sense "V")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wntsnb
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "B")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(cond
 ((not
   (or
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "N")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "S")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "P")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "B")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)))
  (setf info -1))
 ((and (not wantvl)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvl "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvl var-0))
          ret-val)))
  (setf info -2))
 ((and (not wantvr)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvr "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvr var-0))
          ret-val)))
  (setf info -3))
```

```
 ((or (not (or wntsnn wntsne wntsnb wntsnv))
      (and (or wntsne wntsnb) (not (and wantvl wantvr))))
  (setf info -4))
 ((< n 0) (setf info -5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7))
 ((or (< ldvl 1) (and wantvl (< ldvl n))) (setf info -11))
 ((or (< ldvr 1) (and wantvr (< ldvr n))) (setf info -13)))
(cond
 ((= info 0)
  (cond ((= n 0) (setf minwrk 1) (setf maxwrk 1))
        (t
         (setf maxwrk
                 (f2cl-lib:int-add n
                                   (f2cl-lib:int-mul n
                                                     (multiple-value-bind
                                                         (ret-val var-0
                                                          var-1 var-2
                                                          var-3 var-4
                                                          var-5 var-6)
                                                         (ilaenv 1
                                                          "SGEHRD" " "
                                                          n 1 n 0)
                                                       (declare
                                                        (ignore var-0
                                                         var-1 var-2
                                                         var-4 var-6))
                                                       (setf n var-3)
                                                       (setf n var-5)
                                                       ret-val))))
         (cond
          (wantvl
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                var-9 var-10 var-11 var-12 var-13)
               (shseqr "S" "V" n 1 n a lda wr wi vl ldvl work -1 info)
             (declare
              (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-9 var-11
               var-12))
             (when var-2 (setf n var-2))
             (when var-4 (setf n var-4))
             (when var-6 (setf lda var-6))
             (when var-10 (setf ldvl var-10))
             (when var-13 (setf info var-13))))
          (wantvr
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                var-9 var-10 var-11 var-12 var-13)
               (shseqr "S" "V" n 1 n a lda wr wi vr ldvr work -1 info)
             (declare
```

```
      (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-9 var-11
       var-12))
     (when var-2 (setf n var-2))
     (when var-4 (setf n var-4))
     (when var-6 (setf lda var-6))
     (when var-10 (setf ldvr var-10))
     (when var-13 (setf info var-13))))
  (t
   (cond
    (wntsnn
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
          var-9 var-10 var-11 var-12 var-13)
         (shseqr "E" "N" n 1 n a lda wr wi vr ldvr work -1
          info)
       (declare
        (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-9
         var-11 var-12))
       (when var-2 (setf n var-2))
       (when var-4 (setf n var-4))
       (when var-6 (setf lda var-6))
       (when var-10 (setf ldvr var-10))
       (when var-13 (setf info var-13))))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
          var-9 var-10 var-11 var-12 var-13)
         (shseqr "S" "N" n 1 n a lda wr wi vr ldvr work -1
          info)
       (declare
        (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-9
         var-11 var-12))
       (when var-2 (setf n var-2))
       (when var-4 (setf n var-4))
       (when var-6 (setf lda var-6))
       (when var-10 (setf ldvr var-10))
       (when var-13 (setf info var-13)))))))
(setf hswork
        (f2cl-lib:int
         (f2cl-lib:fref work-%data% (1) ((1 *))
                        work-%offset%)))
(cond
 ((and (not wantvl) (not wantvr))
  (setf minwrk (f2cl-lib:int-mul 2 n))
  (if (not wntsnn)
      (setf minwrk
              (max (the f2cl-lib:integer4 minwrk)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add
                         (f2cl-lib:int-mul n n)
```

```
                        (f2cl-lib:int-mul 6 n))))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4 hswork)))
 (if (not wntsnn)
     (setf maxwrk
             (max (the f2cl-lib:integer4 maxwrk)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add
                        (f2cl-lib:int-mul n n)
                        (f2cl-lib:int-mul 6 n)))))))
(t (setf minwrk (f2cl-lib:int-mul 3 n))
 (if (and (not wntsnn) (not wntsne))
     (setf minwrk
             (max (the f2cl-lib:integer4 minwrk)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add
                        (f2cl-lib:int-mul n n)
                        (f2cl-lib:int-mul 6 n))))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4 hswork)))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n
                                     (f2cl-lib:int-mul
                                      (f2cl-lib:int-sub n 1)
                                      (multiple-value-bind
                                          (ret-val var-0
                                           var-1 var-2 var-3
                                           var-4 var-5 var-6)
                                          (ilaenv 1 "SORGHR"
                                           " " n 1 n -1)
                                        (declare
                                         (ignore var-0 var-1
                                          var-2 var-4 var-6))
                                        (setf n var-3)
                                        (setf n var-5)
                                        ret-val))))))
 (if (and (not wntsnn) (not wntsne))
     (setf maxwrk
             (max (the f2cl-lib:integer4 maxwrk)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add
                        (f2cl-lib:int-mul n n)
                        (f2cl-lib:int-mul 6 n))))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
```

```
                                (f2cl-lib:int-mul 3 n))))))
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4 minwrk)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 maxwrk) 'single-float))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -21)))))
(cond
 ((/= info 0) (xerbla "SGEEVX" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(setf eps (slamch "P"))
(setf smlnum (slamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (slabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf icond 0)
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "M" n n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf scalea f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf scalea f2cl-lib:%true%)
  (setf cscale smlnum))
 ((> anrm bignum) (setf scalea f2cl-lib:%true%) (setf cscale bignum)))
(if scalea
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl "G" 0 0 anrm cscale n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf cscale var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (sgebal balanc n a lda ilo ihi scale ierr)
```

```
  (declare (ignore var-1 var-2 var-6))
  (setf balanc var-0)
  (setf lda var-3)
  (setf ilo var-4)
  (setf ihi var-5)
  (setf ierr var-7))
(setf abnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "1" n n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(cond
 (scalea (setf (f2cl-lib:fref dum (1) ((1 1))) abnrm)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 cscale anrm 1 1 dum 1 ierr)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
    (when var-3 (setf cscale var-3))
    (when var-4 (setf anrm var-4))
    (when var-9 (setf ierr var-9)))
  (setf abnrm (f2cl-lib:fref dum (1) ((1 1))))))
(setf itau 1)
(setf iwrk (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sgehrd n ilo ihi a lda
     (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf ilo var-1))
  (when var-2 (setf ihi var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(cond
 (wantvl (f2cl-lib:f2cl-set-string side "L" (string 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "L" n n a lda vl ldvl)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvl var-6)))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorghr n ilo ihi vl ldvl
     (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf ilo var-1))
  (when var-2 (setf ihi var-2))
  (when var-4 (setf ldvl var-4))
  (when var-8 (setf ierr var-8)))
(setf iwrk itau)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (shseqr "S" "V" n ilo ihi a lda wr wi vl ldvl
     (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
  (declare
   (ignore var-0 var-1 var-5 var-7 var-8 var-9 var-11 var-12))
  (when var-2 (setf n var-2))
  (when var-3 (setf ilo var-3))
  (when var-4 (setf ihi var-4))
  (when var-6 (setf lda var-6))
  (when var-10 (setf ldvl var-10))
  (when var-13 (setf info var-13)))
(cond
 (wantvr (f2cl-lib:f2cl-set-string side "B" (string 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "F" n n vl ldvl vr ldvr)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldvl var-4))
    (when var-6 (setf ldvr var-6))))))
(wantvr (f2cl-lib:f2cl-set-string side "R" (string 1))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slacpy "L" n n a lda vr ldvr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvr var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (sorghr n ilo ihi vr ldvr
```

```
          (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                                work-%offset%)
          (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                                work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
      (declare (ignore var-3 var-5 var-6 var-7))
      (when var-0 (setf n var-0))
      (when var-1 (setf ilo var-1))
      (when var-2 (setf ihi var-2))
      (when var-4 (setf ldvr var-4))
      (when var-8 (setf ierr var-8)))
    (setf iwrk itau)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (shseqr "S" "V" n ilo ihi a lda wr wi vr ldvr
         (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                               work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
      (declare
       (ignore var-0 var-1 var-5 var-7 var-8 var-9 var-11 var-12))
      (when var-2 (setf n var-2))
      (when var-3 (setf ilo var-3))
      (when var-4 (setf ihi var-4))
      (when var-6 (setf lda var-6))
      (when var-10 (setf ldvr var-10))
      (when var-13 (setf info var-13))))
   (t
    (cond (wntsnn (f2cl-lib:f2cl-set-string job "E" (string 1)))
          (t (f2cl-lib:f2cl-set-string job "S" (string 1))))
    (setf iwrk itau)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (shseqr job "N" n ilo ihi a lda wr wi vr ldvr
         (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                               work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
      (declare (ignore var-1 var-5 var-7 var-8 var-9 var-11 var-12))
      (when var-0 (setf job var-0))
      (when var-2 (setf n var-2))
      (when var-3 (setf ilo var-3))
      (when var-4 (setf ihi var-4))
      (when var-6 (setf lda var-6))
      (when var-10 (setf ldvr var-10))
      (when var-13 (setf info var-13)))))
  (if (> info 0)
      (go label50))
  (cond
   ((or wantvl wantvr)
```

```
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (strevc side "B" select n a lda vl ldvl vr ldvr n nout
       (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                             work-%offset%)
       ierr)
    (declare (ignore var-1 var-2 var-4 var-6 var-8 var-12))
    (when var-0 (setf side var-0))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldvl var-7))
    (when var-9 (setf ldvr var-9))
    (when var-10 (setf n var-10))
    (when var-11 (setf nout var-11))
    (when var-13 (setf ierr var-13)))))
(cond
 ((not wntsnn)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17)
      (strsna sense "A" select n a lda vl ldvl vr ldvr rconde rcondv n
       nout
       (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                             work-%offset%)
       n iwork icond)
    (declare
     (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-11 var-14
      var-16))
    (when var-0 (setf sense var-0))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldvl var-7))
    (when var-9 (setf ldvr var-9))
    (when var-12 (setf n var-12))
    (when var-13 (setf nout var-13))
    (when var-15 (setf n var-15))
    (when var-17 (setf icond var-17)))))
(cond
 (wantvl
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (sgebak balanc "L" n ilo ihi scale n vl ldvl ierr)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-7))
    (setf balanc var-0)
    (setf n var-6)
    (setf ldvl var-8)
    (setf ierr var-9))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
```

```
(tagbody
  (cond
   ((= (f2cl-lib:fref wi (i) ((1 *))) zero)
    (setf scl
            (/ one
               (multiple-value-bind (ret-val var-0 var-1 var-2)
                   (snrm2 n
                    (f2cl-lib:array-slice vl-%data% single-float
                                          (1 i) ((1 ldvl) (1 *))
                                          vl-%offset%)
                    1)
                 (declare (ignore var-1 var-2))
                 (when var-0 (setf n var-0))
                 ret-val)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal n scl
         (f2cl-lib:array-slice vl-%data% single-float (1 i)
                               ((1 ldvl) (1 *)) vl-%offset%)
         1)
      (declare (ignore var-2 var-3))
      (when var-0 (setf n var-0))
      (when var-1 (setf scl var-1))))
   ((> (f2cl-lib:fref wi (i) ((1 *))) zero)
    (setf scl
            (/ one
               (slapy2
                (multiple-value-bind (ret-val var-0 var-1 var-2)
                    (snrm2 n
                     (f2cl-lib:array-slice vl-%data% single-float
                                           (1 i) ((1 ldvl) (1 *))
                                           vl-%offset%)
                     1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf n var-0))
                  ret-val)
                (multiple-value-bind (ret-val var-0 var-1 var-2)
                    (snrm2 n
                     (f2cl-lib:array-slice vl-%data% single-float
                                           (1
                                            (f2cl-lib:int-add i 1))
                                           ((1 ldvl) (1 *))
                                           vl-%offset%)
                     1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf n var-0))
                  ret-val))))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal n scl
         (f2cl-lib:array-slice vl-%data% single-float (1 i)
                               ((1 ldvl) (1 *)) vl-%offset%)
```

```
      1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf scl var-1)))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (sscal n scl
     (f2cl-lib:array-slice vl-%data% single-float
                           (1 (f2cl-lib:int-add i 1))
                           ((1 ldvl) (1 *)) vl-%offset%)
      1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf scl var-1)))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
            (+
             (expt
              (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                             vl-%offset%)
              2)
             (expt
              (f2cl-lib:fref vl-%data%
                             (k (f2cl-lib:int-add i 1))
                             ((1 ldvl) (1 *)) vl-%offset%)
              2)))
   label10))
(setf k
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (isamax n work 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (slartg
     (f2cl-lib:array-slice vl-%data% single-float (k i)
                           ((1 ldvl) (1 *)) vl-%offset%)
     (f2cl-lib:array-slice vl-%data% single-float
                           (k (f2cl-lib:int-add i 1))
                           ((1 ldvl) (1 *)) vl-%offset%)
     cs sn r)
  (declare (ignore var-0 var-1))
  (when var-2 (setf cs var-2))
  (when var-3 (setf sn var-3))
  (when var-4 (setf r var-4)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (srot n
     (f2cl-lib:array-slice vl-%data% single-float (1 i)
```

```
                                    ((1 ldvl) (1 *)) vl-%offset%)
                1
                (f2cl-lib:array-slice vl-%data% single-float
                                    (1 (f2cl-lib:int-add i 1))
                                    ((1 ldvl) (1 *)) vl-%offset%)
                1 cs sn)
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf n var-0))
              (when var-5 (setf cs var-5))
              (when var-6 (setf sn var-6)))
            (setf (f2cl-lib:fref vl-%data% (k (f2cl-lib:int-add i 1))
                                ((1 ldvl) (1 *)) vl-%offset%)
                  zero)))
        label20))))
  (cond
   (wantvr
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (sgebak balanc "R" n ilo ihi scale n vr ldvr ierr)
      (declare (ignore var-1 var-2 var-3 var-4 var-5 var-7))
      (setf balanc var-0)
      (setf n var-6)
      (setf ldvr var-8)
      (setf ierr var-9))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (cond
         ((= (f2cl-lib:fref wi (i) ((1 *))) zero)
          (setf scl
                  (/ one
                     (multiple-value-bind (ret-val var-0 var-1 var-2)
                         (snrm2 n
                          (f2cl-lib:array-slice vr-%data% single-float
                                                (1 i) ((1 ldvr) (1 *))
                                                vr-%offset%)
                          1)
                       (declare (ignore var-1 var-2))
                       (when var-0 (setf n var-0))
                       ret-val)))
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (sscal n scl
                (f2cl-lib:array-slice vr-%data% single-float (1 i)
                                    ((1 ldvr) (1 *)) vr-%offset%)
                1)
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf scl var-1))))
         ((> (f2cl-lib:fref wi (i) ((1 *))) zero)
          (setf scl
```

```
          (/ one
             (slapy2
              (multiple-value-bind (ret-val var-0 var-1 var-2)
                  (snrm2 n
                   (f2cl-lib:array-slice vr-%data% single-float
                                         (1 i) ((1 ldvr) (1 *))
                                         vr-%offset%)
                   1)
                (declare (ignore var-1 var-2))
                (when var-0 (setf n var-0))
                ret-val)
              (multiple-value-bind (ret-val var-0 var-1 var-2)
                  (snrm2 n
                   (f2cl-lib:array-slice vr-%data% single-float
                                         (1
                                          (f2cl-lib:int-add i 1))
                                         ((1 ldvr) (1 *))
                                         vr-%offset%)
                   1)
                (declare (ignore var-1 var-2))
                (when var-0 (setf n var-0))
                ret-val))))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (sscal n scl
     (f2cl-lib:array-slice vr-%data% single-float (1 i)
                           ((1 ldvr) (1 *)) vr-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf scl var-1)))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (sscal n scl
     (f2cl-lib:array-slice vr-%data% single-float
                           (1 (f2cl-lib:int-add i 1))
                           ((1 ldvr) (1 *)) vr-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf scl var-1)))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
            (+
             (expt
              (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                             vr-%offset%)
              2)
             (expt
              (f2cl-lib:fref vr-%data%
```

```
                                        (k (f2cl-lib:int-add i 1))
                                        ((1 ldvr) (1 *)) vr-%offset%)
                          2)))
                label30))
            (setf k
                    (multiple-value-bind (ret-val var-0 var-1 var-2)
                        (isamax n work 1)
                      (declare (ignore var-1 var-2))
                      (when var-0 (setf n var-0))
                      ret-val))
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (slartg
                 (f2cl-lib:array-slice vr-%data% single-float (k i)
                                       ((1 ldvr) (1 *)) vr-%offset%)
                 (f2cl-lib:array-slice vr-%data% single-float
                                       (k (f2cl-lib:int-add i 1))
                                       ((1 ldvr) (1 *)) vr-%offset%)
                 cs sn r)
              (declare (ignore var-0 var-1))
              (when var-2 (setf cs var-2))
              (when var-3 (setf sn var-3))
              (when var-4 (setf r var-4)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (srot n
                 (f2cl-lib:array-slice vr-%data% single-float (1 i)
                                       ((1 ldvr) (1 *)) vr-%offset%)
                 1
                 (f2cl-lib:array-slice vr-%data% single-float
                                       (1 (f2cl-lib:int-add i 1))
                                       ((1 ldvr) (1 *)) vr-%offset%)
                 1 cs sn)
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf n var-0))
              (when var-5 (setf cs var-5))
              (when var-6 (setf sn var-6)))
            (setf (f2cl-lib:fref vr-%data% (k (f2cl-lib:int-add i 1))
                                 ((1 ldvr) (1 *)) vr-%offset%)
                  zero)))
        label40))))
  label50
   (cond
    (scalea
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (slascl "G" 0 0 cscale anrm (f2cl-lib:int-sub n info) 1
          (f2cl-lib:array-slice wr-%data% single-float ((+ info 1))
                                ((1 *)) wr-%offset%)
          (max (the f2cl-lib:integer4 (f2cl-lib:int-sub n info))
               (the f2cl-lib:integer4 1))
```

```
        ierr)
      (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
      (when var-3 (setf cscale var-3))
      (when var-4 (setf anrm var-4))
      (when var-9 (setf ierr var-9)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl "G" 0 0 cscale anrm (f2cl-lib:int-sub n info) 1
         (f2cl-lib:array-slice wi-%data% single-float ((+ info 1))
                               ((1 *)) wi-%offset%)
         (max (the f2cl-lib:integer4 (f2cl-lib:int-sub n info))
              (the f2cl-lib:integer4 1))
        ierr)
      (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
      (when var-3 (setf cscale var-3))
      (when var-4 (setf anrm var-4))
      (when var-9 (setf ierr var-9)))
    (cond
     ((= info 0)
      (if (and (or wntsnv wntsnb) (= icond 0))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9)
              (slascl "G" 0 0 cscale anrm n 1 rcondv n ierr)
            (declare (ignore var-0 var-1 var-2 var-6 var-7))
            (when var-3 (setf cscale var-3))
            (when var-4 (setf anrm var-4))
            (when var-5 (setf n var-5))
            (when var-8 (setf n var-8))
            (when var-9 (setf ierr var-9)))))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (slascl "G" 0 0 cscale anrm (f2cl-lib:int-sub ilo 1) 1 wr n
           ierr)
        (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
        (when var-3 (setf cscale var-3))
        (when var-4 (setf anrm var-4))
        (when var-8 (setf n var-8))
        (when var-9 (setf ierr var-9)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (slascl "G" 0 0 cscale anrm (f2cl-lib:int-sub ilo 1) 1 wi n
           ierr)
        (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
        (when var-3 (setf cscale var-3))
        (when var-4 (setf anrm var-4))
        (when var-8 (setf n var-8))
        (when var-9 (setf ierr var-9)))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
```

```
                (coerce (the f2cl-lib:integer4 maxwrk) 'single-float))
        (go end_label)
       end_label
        (return
         (values balanc jobvl jobvr sense n nil lda nil nil nil ldvl nil ldvr
                 ilo ihi nil abnrm nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgeevx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (single-float)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::balanc
                                              fortran-to-lisp::jobvl
                                              fortran-to-lisp::jobvr
                                              fortran-to-lisp::sense
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::ldvl nil
                                              fortran-to-lisp::ldvr
                                              fortran-to-lisp::ilo
                                              fortran-to-lisp::ihi nil
                                              fortran-to-lisp::abnrm nil nil
                                              nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
```

```
                                        '(fortran-to_lisp::sgebak
                                          fortran-to_lisp::sgebal
                                          fortran-to_lisp::ilaenv))))
```

———-

25.5 sgegs - DEPRECATED use sgges

25.6 sgegv - DEPRECATED use sggev

25.7 sgges

25.7.1 Axiom unit tests

— **sgges.input** —

```
)set break resume
)sys rm -f sgges.output
)spool sgges.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

25.7.2 Axiom help page

— **sgges.help** —

```
==========================================================================
sgges examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine sgges (JOBVSL, JOBVSR, SORT, SELCTG, N, A, LDA, B, LDB,
           SDIM, ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR, WORK, LWORK,
           BWORK, INFO)
            SGGES computes the eigenvalues, the Schur form, and, optionally,
           the matrix of Schur vectors for GE matrices

Function/Subroutine Documentation
   subroutine sgges (character JOBVSL, character JOBVSR, character SORT,
       external SELCTG, integer N, real, dimension( lda, * ) A, integer LDA,
       real, dimension( ldb, * ) B, integer LDB, integer SDIM, real,
       dimension( * ) ALPHAR, real, dimension( * ) ALPHAI, real, dimension( *
       ) BETA, real, dimension( ldvsl, * ) VSL, integer LDVSL, real,
       dimension( ldvsr, * ) VSR, integer LDVSR, real, dimension( * ) WORK,
       integer LWORK, logical, dimension( * ) BWORK, integer INFO)
        SGGES computes the eigenvalues, the Schur form, and, optionally, the
       matrix of Schur vectors for GE matrices

Purpose:
   SGGES computes for a pair of N-by-N real nonsymmetric matrices (A,B),
   the generalized eigenvalues, the generalized real Schur form (S,T),
   optionally, the left and/or right matrices of Schur vectors (VSL and
   VSR). This gives the generalized Schur factorization

   (A,B) = ( (VSL)*S*(VSR)**T, (VSL)*T*(VSR)**T )

   Optionally, it also orders the eigenvalues so that a selected cluster
   of eigenvalues appears in the leading diagonal blocks of the upper
   quasi-triangular matrix S and the upper triangular matrix T.The
   leading columns of VSL and VSR then form an orthonormal basis for the
   corresponding left and right eigenspaces (deflating subspaces).

   (If only the generalized eigenvalues are needed, use the driver
   SGGEV instead, which is faster.)

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar w
   or a ratio alpha/beta = w, such that  A - w*B is singular.  It is
   usually represented as the pair (alpha,beta), as there is a
   reasonable interpretation for beta=0 or both being zero.

   A pair of matrices (S,T) is in generalized real Schur form if T is
   upper triangular with non-negative diagonal and S is block upper
   triangular with 1-by-1 and 2-by-2 blocks.  1-by-1 blocks correspond
   to real generalized eigenvalues, while 2-by-2 blocks of S will be
   "standardized" by making the corresponding elements of T have the form:
          [  a  0  ]
          [  0  b  ]

   and the pair of corresponding 2-by-2 blocks in S and T will have a
```

```
   complex conjugate pair of generalized eigenvalues.

Parameters:

 JOBVSL
   JOBVSL is CHARACTER*1
   = 'N':  do not compute the left Schur vectors;
   = 'V':  compute the left Schur vectors.

 JOBVSR
   JOBVSR is CHARACTER*1
   = 'N':  do not compute the right Schur vectors;
   = 'V':  compute the right Schur vectors.

 SORT
   SORT is CHARACTER*1
   Specifies whether or not to order the eigenvalues on the
   diagonal of the generalized Schur form.
   = 'N':  Eigenvalues are not ordered;
   = 'S':  Eigenvalues are ordered (see SELCTG);

 SELCTG
   SELCTG is a LOGICAL FUNCTION of three REAL arguments
   SELCTG must be declared EXTERNAL in the calling subroutine.
   If SORT = 'N', SELCTG is not referenced.
   If SORT = 'S', SELCTG is used to select eigenvalues to sort
   to the top left of the Schur form.
   An eigenvalue (ALPHAR(j)+ALPHAI(j))/BETA(j) is selected if
   SELCTG(ALPHAR(j),ALPHAI(j),BETA(j)) is true; i.e. if either
   one of a complex conjugate pair of eigenvalues is selected,
   then both complex eigenvalues are selected.

   Note that in the ill-conditioned case, a selected complex
   eigenvalue may no longer satisfy SELCTG(ALPHAR(j),ALPHAI(j),
   BETA(j)) = .TRUE. after ordering. INFO is to be set to N+2
   in this case.

 N
   N is INTEGER
   The order of the matrices A, B, VSL, and VSR.  N >= 0.

 A
   A is REAL array, dimension (LDA, N)
   On entry, the first of the pair of matrices.
   On exit, A has been overwritten by its generalized Schur form S.

 LDA
   LDA is INTEGER
   The leading dimension of A.  LDA >= max(1,N).
```

```
B
  B is REAL array, dimension (LDB, N)
  On entry, the second of the pair of matrices.
  On exit, B has been overwritten by its generalized Schur form T.

LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).

SDIM
  SDIM is INTEGER
  If SORT = 'N', SDIM = 0.
  If SORT = 'S', SDIM = number of eigenvalues (after sorting)
  for which SELCTG is true.  (Complex conjugate pairs for which
  SELCTG is true for either eigenvalue count as 2.)

ALPHAR
  ALPHAR is REAL array, dimension (N)

ALPHAI
  ALPHAI is REAL array, dimension (N)

BETA
  BETA is REAL array, dimension (N)
  On exit, (ALPHAR(j) + ALPHAI(j)*i)/BETA(j), j=1,...,N, will
  be the generalized eigenvalues.  ALPHAR(j) + ALPHAI(j)*i,
  and  BETA(j),j=1,...,N are the diagonals of the complex Schur
  form (S,T) that would result if the 2-by-2 diagonal blocks of
  the real Schur form of (A,B) were further reduced to
  triangular form using 2-by-2 complex unitary transformations.
  If ALPHAI(j) is zero, then the j-th eigenvalue is real; if
  positive, then the j-th and (j+1)-st eigenvalues are a
  complex conjugate pair, with ALPHAI(j+1) negative.

  Note: the quotients ALPHAR(j)/BETA(j) and ALPHAI(j)/BETA(j)
  may easily over- or underflow, and BETA(j) may even be zero.
  Thus, the user should avoid naively computing the ratio.
  However, ALPHAR and ALPHAI will be always less than and
  usually comparable with norm(A) in magnitude, and BETA always
  less than and usually comparable with norm(B).

VSL
  VSL is REAL array, dimension (LDVSL,N)
  If JOBVSL = 'V', VSL will contain the left Schur vectors.
  Not referenced if JOBVSL = 'N'.

LDVSL
  LDVSL is INTEGER
  The leading dimension of the matrix VSL. LDVSL >=1, and
  if JOBVSL = 'V', LDVSL >= N.
```

```
VSR
  VSR is REAL array, dimension (LDVSR,N)
  If JOBVSR = 'V', VSR will contain the right Schur vectors.
  Not referenced if JOBVSR = 'N'.

LDVSR
  LDVSR is INTEGER
  The leading dimension of the matrix VSR. LDVSR >= 1, and
  if JOBVSR = 'V', LDVSR >= N.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  If N = 0, LWORK >= 1, else LWORK >= max(8*N,6*N+16).
  For good performance , LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

BWORK
  BWORK is LOGICAL array, dimension (N)
  Not referenced if SORT = 'N'.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  = 1,...,N:
        The QZ iteration failed.  (A,B) are not in Schur
        form, but ALPHAR(j), ALPHAI(j), and BETA(j) should
        be correct for j=INFO+1,...,N.
  > N:  =N+1: other than QZ iteration failed in SHGEQZ.
        =N+2: after reordering, roundoff changed values of
              some complex eigenvalues so that leading
              eigenvalues in the Generalized Schur form no
              longer satisfy SELCTG=.TRUE.  This could also
              be caused due to scaling.
        =N+3: reordering failed in STGSEN.
```

25.7.3 fortran code

— **sgges.f** —

```
*  =====================================================================
      SUBROUTINE SGGES( JOBVSL, JOBVSR, SORT, SELCTG, N, A, LDA, B, LDB,
     $                  SDIM, ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR,
     $                  LDVSR, WORK, LWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVSL, JOBVSR, SORT
      INTEGER            INFO, LDA, LDB, LDVSL, LDVSR, LWORK, N, SDIM
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      REAL               A( LDA, * ), ALPHAI( * ), ALPHAR( * ),
     $                   B( LDB, * ), BETA( * ), VSL( LDVSL, * ),
     $                   VSR( LDVSR, * ), WORK( * )
*     ..
*     .. Function Arguments ..
      LOGICAL            SELCTG
      EXTERNAL           SELCTG
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            CURSL, ILASCL, ILBSCL, ILVSL, ILVSR, LASTSL,
     $                   LQUERY, LST2SL, WANTST
      INTEGER            I, ICOLS, IERR, IHI, IJOBVL, IJOBVR, ILEFT,
     $                   ILO, IP, IRIGHT, IROWS, ITAU, IWRK, MAXWRK,
     $                   MINWRK
      REAL               ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS, PVSL,
     $                   PVSR, SAFMAX, SAFMIN, SMLNUM
*     ..
*     .. Local Arrays ..
      INTEGER            IDUM( 1 )
      REAL               DIF( 2 )
*     ..
*     .. External Subroutines ..
```

```
      EXTERNAL           SGEQRF, SGGBAK, SGGBAL, SGGHRD, SHGEQZ, SLABAD,
     $                   SLACPY, SLASCL, SLASET, SORGQR, SORMQR, STGSEN,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               SLAMCH, SLANGE
      EXTERNAL           LSAME, ILAENV, SLAMCH, SLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVSL, 'N' ) ) THEN
         IJOBVL = 1
         ILVSL = .FALSE.
      ELSE IF( LSAME( JOBVSL, 'V' ) ) THEN
         IJOBVL = 2
         ILVSL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVSL = .FALSE.
      END IF
*
      IF( LSAME( JOBVSR, 'N' ) ) THEN
         IJOBVR = 1
         ILVSR = .FALSE.
      ELSE IF( LSAME( JOBVSR, 'V' ) ) THEN
         IJOBVR = 2
         ILVSR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVSR = .FALSE.
      END IF
*
      WANTST = LSAME( SORT, 'S' )
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -2
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
```

```
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDVSL.LT.1 .OR. ( ILVSL .AND. LDVSL.LT.N ) ) THEN
         INFO = -15
      ELSE IF( LDVSR.LT.1 .OR. ( ILVSR .AND. LDVSR.LT.N ) ) THEN
         INFO = -17
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.GT.0 )THEN
            MINWRK = MAX( 8*N, 6*N + 16 )
            MAXWRK = MINWRK - N +
     $               N*ILAENV( 1, 'SGEQRF', ' ', N, 1, N, 0 )
            MAXWRK = MAX( MAXWRK, MINWRK - N +
     $                    N*ILAENV( 1, 'SORMQR', ' ', N, 1, N, -1 ) )
            IF( ILVSL ) THEN
               MAXWRK = MAX( MAXWRK, MINWRK - N +
     $                       N*ILAENV( 1, 'SORGQR', ' ', N, 1, N, -1 ) )
            END IF
         ELSE
            MINWRK = 1
            MAXWRK = 1
         END IF
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY )
     $      INFO = -19
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGGES ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
```

```
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SAFMIN = SLAMCH( 'S' )
      SAFMAX = ONE / SAFMIN
      CALL SLABAD( SAFMIN, SAFMAX )
      SMLNUM = SQRT( SAFMIN ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = SLANGE( 'M', N, N, A, LDA, WORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
      IF( ILASCL )
     $   CALL SLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = SLANGE( 'M', N, N, B, LDB, WORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
     $   CALL SLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrix to make it more nearly triangular
*     (Workspace: need 6*N + 2*N space for storing balancing factors)
*
      ILEFT = 1
      IRIGHT = N + 1
      IWRK = IRIGHT + N
      CALL SGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, WORK( ILEFT ),
     $             WORK( IRIGHT ), WORK( IWRK ), IERR )
```

```
*
*     Reduce B to triangular form (QR decomposition of B)
*     (Workspace: need N, prefer N*NB)
*
      IROWS = IHI + 1 - ILO
      ICOLS = N + 1 - ILO
      ITAU = IWRK
      IWRK = ITAU + IROWS
      CALL SGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to matrix A
*     (Workspace: need N, prefer N*NB)
*
      CALL SORMQR( 'L', 'T', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VSL
*     (Workspace: need N, prefer N*NB)
*
      IF( ILVSL ) THEN
         CALL SLASET( 'Full', N, N, ZERO, ONE, VSL, LDVSL )
         IF( IROWS.GT.1 ) THEN
            CALL SLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VSL( ILO+1, ILO ), LDVSL )
         END IF
         CALL SORGQR( IROWS, IROWS, IROWS, VSL( ILO, ILO ), LDVSL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Initialize VSR
*
      IF( ILVSR )
     $   CALL SLASET( 'Full', N, N, ZERO, ONE, VSR, LDVSR )
*
*     Reduce to generalized Hessenberg form
*     (Workspace: none needed)
*
      CALL SGGHRD( JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB, VSL,
     $             LDVSL, VSR, LDVSR, IERR )
*
*     Perform QZ algorithm, computing Schur vectors if desired
*     (Workspace: need N)
*
      IWRK = ITAU
      CALL SHGEQZ( 'S', JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR,
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
      IF( IERR.NE.0 ) THEN
```

```
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 40
      END IF
*
*     Sort eigenvalues ALPHA/BETA if desired
*     (Workspace: need 4*N+16 )
*
      SDIM = 0
      IF( WANTST ) THEN
*
*        Undo scaling on eigenvalues before SELCTGing
*
         IF( ILASCL ) THEN
            CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAR, N,
     $                   IERR )
            CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAI, N,
     $                   IERR )
         END IF
         IF( ILBSCL )
     $      CALL SLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
*
*        Select eigenvalues
*
         DO 10 I = 1, N
            BWORK( I ) = SELCTG( ALPHAR( I ), ALPHAI( I ), BETA( I ) )
   10    CONTINUE
*
         CALL STGSEN( 0, ILVSL, ILVSR, BWORK, N, A, LDA, B, LDB, ALPHAR,
     $                ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR, SDIM, PVSL,
     $                PVSR, DIF, WORK( IWRK ), LWORK-IWRK+1, IDUM, 1,
     $                IERR )
         IF( IERR.EQ.1 )
     $      INFO = N + 3
*
      END IF
*
*     Apply back-permutation to VSL and VSR
*     (Workspace: none needed)
*
      IF( ILVSL )
     $   CALL SGGBAK( 'P', 'L', N, ILO, IHI, WORK( ILEFT ),
     $                WORK( IRIGHT ), N, VSL, LDVSL, IERR )
*
      IF( ILVSR )
```

```
     $   CALL SGGBAK( 'P', 'R', N, ILO, IHI, WORK( ILEFT ),
     $                WORK( IRIGHT ), N, VSR, LDVSR, IERR )
*
*     Check if unscaling would cause over/underflow, if so, rescale
*     (ALPHAR(I),ALPHAI(I),BETA(I)) so BETA(I) is on the order of
*     B(I,I) and ALPHAR(I) and ALPHAI(I) are on the order of A(I,I)
*
      IF( ILASCL )THEN
         DO 50 I = 1, N
            IF( ALPHAI( I ).NE.ZERO ) THEN
               IF( ( ALPHAR( I )/SAFMAX ).GT.( ANRMTO/ANRM ) .OR.
     $             ( SAFMIN/ALPHAR( I ) ).GT.( ANRM/ANRMTO ) ) THEN
                  WORK( 1 ) = ABS( A( I, I )/ALPHAR( I ) )
                  BETA( I ) = BETA( I )*WORK( 1 )
                  ALPHAR( I ) = ALPHAR( I )*WORK( 1 )
                  ALPHAI( I ) = ALPHAI( I )*WORK( 1 )
               ELSE IF( ( ALPHAI( I )/SAFMAX ).GT.( ANRMTO/ANRM ) .OR.
     $             ( SAFMIN/ALPHAI( I ) ).GT.( ANRM/ANRMTO ) ) THEN
                  WORK( 1 ) = ABS( A( I, I+1 )/ALPHAI( I ) )
                  BETA( I ) = BETA( I )*WORK( 1 )
                  ALPHAR( I ) = ALPHAR( I )*WORK( 1 )
                  ALPHAI( I ) = ALPHAI( I )*WORK( 1 )
               END IF
            END IF
   50    CONTINUE
      END IF
*
      IF( ILBSCL )THEN
         DO 60 I = 1, N
            IF( ALPHAI( I ).NE.ZERO ) THEN
                IF( ( BETA( I )/SAFMAX ).GT.( BNRMTO/BNRM ) .OR.
     $              ( SAFMIN/BETA( I ) ).GT.( BNRM/BNRMTO ) ) THEN
                  WORK( 1 ) = ABS(B( I, I )/BETA( I ))
                  BETA( I ) = BETA( I )*WORK( 1 )
                  ALPHAR( I ) = ALPHAR( I )*WORK( 1 )
                  ALPHAI( I ) = ALPHAI( I )*WORK( 1 )
                END IF
             END IF
   60    CONTINUE
      END IF
*
*     Undo scaling
*
      IF( ILASCL ) THEN
         CALL SLASCL( 'H', 0, 0, ANRMTO, ANRM, N, N, A, LDA, IERR )
         CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAR, N, IERR )
         CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAI, N, IERR )
      END IF
*
      IF( ILBSCL ) THEN
```

```
         CALL SLASCL( 'U', 0, 0, BNRMTO, BNRM, N, N, B, LDB, IERR )
         CALL SLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
      END IF
*
      IF( WANTST ) THEN
*
*        Check if reordering is correct
*
         LASTSL = .TRUE.
         LST2SL = .TRUE.
         SDIM = 0
         IP = 0
         DO 30 I = 1, N
            CURSL = SELCTG( ALPHAR( I ), ALPHAI( I ), BETA( I ) )
            IF( ALPHAI( I ).EQ.ZERO ) THEN
               IF( CURSL )
     $            SDIM = SDIM + 1
               IP = 0
               IF( CURSL .AND. .NOT.LASTSL )
     $            INFO = N + 2
            ELSE
               IF( IP.EQ.1 ) THEN
*
*                Last eigenvalue of conjugate pair
*
                 CURSL = CURSL .OR. LASTSL
                 LASTSL = CURSL
                 IF( CURSL )
     $              SDIM = SDIM + 2
                 IP = -1
                 IF( CURSL .AND. .NOT.LST2SL )
     $              INFO = N + 2
               ELSE
*
*                First eigenvalue of conjugate pair
*
                 IP = 1
               END IF
            END IF
            LST2SL = LASTSL
            LASTSL = CURSL
   30    CONTINUE
*
      END IF
*
   40 CONTINUE
*
      WORK( 1 ) = MAXWRK
*
      RETURN
```

```
*
*     End of SGGES
*
      END
```

———-

25.7.4 lisp code

— **sgges.lisp** —

———-

25.8 sgges3

25.8.1 Axiom unit tests

— **sgges3.input** —

```
)set break resume
)sys rm -f sgges3.output
)spool sgges3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

25.8.2 Axiom help page

— **sgges3.help** —

```
==================================================================
sgges3 examples
==================================================================

==================================================================
```

```
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgges3 (JOBVSL, JOBVSR, SORT, SELCTG, N, A, LDA, B, LDB,
           SDIM, ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR, WORK, LWORK,
           BWORK, INFO)
            SGGES3 computes the eigenvalues, the Schur form, and, optionally,
           the matrix of Schur vectors for GE matrices (blocked algorithm)

Function/Subroutine Documentation
   subroutine sgges3 (character JOBVSL, character JOBVSR, character SORT,
       external SELCTG, integer N, real, dimension( lda, * ) A, integer LDA,
       real, dimension( ldb, * ) B, integer LDB, integer SDIM, real,
       dimension( * ) ALPHAR, real, dimension( * ) ALPHAI, real, dimension( *
       ) BETA, real, dimension( ldvsl, * ) VSL, integer LDVSL, real,
       dimension( ldvsr, * ) VSR, integer LDVSR, real, dimension( * ) WORK,
       integer LWORK, logical, dimension( * ) BWORK, integer INFO)
        SGGES3 computes the eigenvalues, the Schur form, and, optionally, the
       matrix of Schur vectors for GE matrices (blocked algorithm)

Purpose:
   SGGES3 computes for a pair of N-by-N real nonsymmetric matrices (A,B),
   the generalized eigenvalues, the generalized real Schur form (S,T),
   optionally, the left and/or right matrices of Schur vectors (VSL and
   VSR). This gives the generalized Schur factorization

   (A,B) = ( (VSL)*S*(VSR)**T, (VSL)*T*(VSR)**T )

   Optionally, it also orders the eigenvalues so that a selected cluster
   of eigenvalues appears in the leading diagonal blocks of the upper
   quasi-triangular matrix S and the upper triangular matrix T.The
   leading columns of VSL and VSR then form an orthonormal basis for the
   corresponding left and right eigenspaces (deflating subspaces).

   (If only the generalized eigenvalues are needed, use the driver
   SGGEV instead, which is faster.)

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar w
   or a ratio alpha/beta = w, such that  A - w*B is singular.  It is
   usually represented as the pair (alpha,beta), as there is a
   reasonable interpretation for beta=0 or both being zero.

   A pair of matrices (S,T) is in generalized real Schur form if T is
   upper triangular with non-negative diagonal and S is block upper
   triangular with 1-by-1 and 2-by-2 blocks.  1-by-1 blocks correspond
   to real generalized eigenvalues, while 2-by-2 blocks of S will be
   "standardized" by making the corresponding elements of T have the form:
          [  a  0  ]
```

```
         [  0  b  ]

  and the pair of corresponding 2-by-2 blocks in S and T will have a
  complex conjugate pair of generalized eigenvalues.

Parameters:

 JOBVSL
   JOBVSL is CHARACTER*1
   = 'N':  do not compute the left Schur vectors;
   = 'V':  compute the left Schur vectors.

 JOBVSR
   JOBVSR is CHARACTER*1
   = 'N':  do not compute the right Schur vectors;
   = 'V':  compute the right Schur vectors.

 SORT
   SORT is CHARACTER*1
   Specifies whether or not to order the eigenvalues on the
   diagonal of the generalized Schur form.
   = 'N':  Eigenvalues are not ordered;
   = 'S':  Eigenvalues are ordered (see SELCTG);

 SELCTG
   SELCTG is a LOGICAL FUNCTION of three REAL arguments
   SELCTG must be declared EXTERNAL in the calling subroutine.
   If SORT = 'N', SELCTG is not referenced.
   If SORT = 'S', SELCTG is used to select eigenvalues to sort
   to the top left of the Schur form.
   An eigenvalue (ALPHAR(j)+ALPHAI(j))/BETA(j) is selected if
   SELCTG(ALPHAR(j),ALPHAI(j),BETA(j)) is true; i.e. if either
   one of a complex conjugate pair of eigenvalues is selected,
   then both complex eigenvalues are selected.

   Note that in the ill-conditioned case, a selected complex
   eigenvalue may no longer satisfy SELCTG(ALPHAR(j),ALPHAI(j),
   BETA(j)) = .TRUE. after ordering. INFO is to be set to N+2
   in this case.

 N
   N is INTEGER
   The order of the matrices A, B, VSL, and VSR.  N >= 0.

 A
   A is REAL array, dimension (LDA, N)
   On entry, the first of the pair of matrices.
   On exit, A has been overwritten by its generalized Schur form S.

 LDA
```

```
  LDA is INTEGER
  The leading dimension of A.  LDA >= max(1,N).

B
  B is REAL array, dimension (LDB, N)
  On entry, the second of the pair of matrices.
  On exit, B has been overwritten by its generalized Schur form T.

LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).

SDIM
  SDIM is INTEGER
  If SORT = 'N', SDIM = 0.
  If SORT = 'S', SDIM = number of eigenvalues (after sorting)
  for which SELCTG is true.  (Complex conjugate pairs for which
  SELCTG is true for either eigenvalue count as 2.)

ALPHAR
  ALPHAR is REAL array, dimension (N)

ALPHAI
  ALPHAI is REAL array, dimension (N)

BETA
  BETA is REAL array, dimension (N)
  On exit, (ALPHAR(j) + ALPHAI(j)*i)/BETA(j), j=1,...,N, will
  be the generalized eigenvalues.  ALPHAR(j) + ALPHAI(j)*i,
  and  BETA(j),j=1,...,N are the diagonals of the complex Schur
  form (S,T) that would result if the 2-by-2 diagonal blocks of
  the real Schur form of (A,B) were further reduced to
  triangular form using 2-by-2 complex unitary transformations.
  If ALPHAI(j) is zero, then the j-th eigenvalue is real; if
  positive, then the j-th and (j+1)-st eigenvalues are a
  complex conjugate pair, with ALPHAI(j+1) negative.

  Note: the quotients ALPHAR(j)/BETA(j) and ALPHAI(j)/BETA(j)
  may easily over- or underflow, and BETA(j) may even be zero.
  Thus, the user should avoid naively computing the ratio.
  However, ALPHAR and ALPHAI will be always less than and
  usually comparable with norm(A) in magnitude, and BETA always
  less than and usually comparable with norm(B).

VSL
  VSL is REAL array, dimension (LDVSL,N)
  If JOBVSL = 'V', VSL will contain the left Schur vectors.
  Not referenced if JOBVSL = 'N'.

LDVSL
```

```
  LDVSL is INTEGER
  The leading dimension of the matrix VSL. LDVSL >=1, and
  if JOBVSL = 'V', LDVSL >= N.

VSR
  VSR is REAL array, dimension (LDVSR,N)
  If JOBVSR = 'V', VSR will contain the right Schur vectors.
  Not referenced if JOBVSR = 'N'.

LDVSR
  LDVSR is INTEGER
  The leading dimension of the matrix VSR. LDVSR >= 1, and
  if JOBVSR = 'V', LDVSR >= N.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

BWORK
  BWORK is LOGICAL array, dimension (N)
  Not referenced if SORT = 'N'.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  = 1,...,N:
        The QZ iteration failed.  (A,B) are not in Schur
        form, but ALPHAR(j), ALPHAI(j), and BETA(j) should
        be correct for j=INFO+1,...,N.
  > N:  =N+1: other than QZ iteration failed in SHGEQZ.
        =N+2: after reordering, roundoff changed values of
              some complex eigenvalues so that leading
              eigenvalues in the Generalized Schur form no
              longer satisfy SELCTG=.TRUE.  This could also
              be caused due to scaling.
        =N+3: reordering failed in STGSEN.
```

25.8.3 fortran code

— sgges3.f —

```
*  =====================================================================
      SUBROUTINE SGGES3( JOBVSL, JOBVSR, SORT, SELCTG, N, A, LDA, B,
     $                   LDB, SDIM, ALPHAR, ALPHAI, BETA, VSL, LDVSL,
     $                   VSR, LDVSR, WORK, LWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     January 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVSL, JOBVSR, SORT
      INTEGER            INFO, LDA, LDB, LDVSL, LDVSR, LWORK, N, SDIM
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      REAL               A( LDA, * ), ALPHAI( * ), ALPHAR( * ),
     $                   B( LDB, * ), BETA( * ), VSL( LDVSL, * ),
     $                   VSR( LDVSR, * ), WORK( * )
*     ..
*     .. Function Arguments ..
      LOGICAL            SELCTG
      EXTERNAL           SELCTG
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            CURSL, ILASCL, ILBSCL, ILVSL, ILVSR, LASTSL,
     $                   LQUERY, LST2SL, WANTST
      INTEGER            I, ICOLS, IERR, IHI, IJOBVL, IJOBVR, ILEFT,
     $                   ILO, IP, IRIGHT, IROWS, ITAU, IWRK, LWKOPT
      REAL               ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS, PVSL,
     $                   PVSR, SAFMAX, SAFMIN, SMLNUM
*     ..
*     .. Local Arrays ..
      INTEGER            IDUM( 1 )
      REAL               DIF( 2 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEQRF, SGGBAK, SGGBAL, SGGHD3, SHGEQZ, SLABAD,
```

```
     $                   SLACPY, SLASCL, SLASET, SORGQR, SORMQR, STGSEN,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH, SLANGE
      EXTERNAL           LSAME, SLAMCH, SLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVSL, 'N' ) ) THEN
         IJOBVL = 1
         ILVSL = .FALSE.
      ELSE IF( LSAME( JOBVSL, 'V' ) ) THEN
         IJOBVL = 2
         ILVSL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVSL = .FALSE.
      END IF
*
      IF( LSAME( JOBVSR, 'N' ) ) THEN
         IJOBVR = 1
         ILVSR = .FALSE.
      ELSE IF( LSAME( JOBVSR, 'V' ) ) THEN
         IJOBVR = 2
         ILVSR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVSR = .FALSE.
      END IF
*
      WANTST = LSAME( SORT, 'S' )
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -2
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
```

```
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDVSL.LT.1 .OR. ( ILVSL .AND. LDVSL.LT.N ) ) THEN
         INFO = -15
      ELSE IF( LDVSR.LT.1 .OR. ( ILVSR .AND. LDVSR.LT.N ) ) THEN
         INFO = -17
      ELSE IF( LWORK.LT.6*N+16 .AND. .NOT.LQUERY ) THEN
         INFO = -19
      END IF
*
*     Compute workspace
*
      IF( INFO.EQ.0 ) THEN
         CALL SGEQRF( N, N, B, LDB, WORK, WORK, -1, IERR )
         LWKOPT = MAX( 6*N+16, 3*N+INT( WORK( 1 ) ) )
         CALL SORMQR( 'L', 'T', N, N, N, B, LDB, WORK, A, LDA, WORK,
     $                -1, IERR )
         LWKOPT = MAX( LWKOPT, 3*N+INT( WORK( 1 ) ) )
         IF( ILVSL ) THEN
            CALL SORGQR( N, N, N, VSL, LDVSL, WORK, WORK, -1, IERR )
            LWKOPT = MAX( LWKOPT, 3*N+INT( WORK( 1 ) ) )
         END IF
         CALL SGGHD3( JOBVSL, JOBVSR, N, 1, N, A, LDA, B, LDB, VSL,
     $                LDVSL, VSR, LDVSR, WORK, -1, IERR )
         LWKOPT = MAX( LWKOPT, 3*N+INT( WORK( 1 ) ) )
         CALL SHGEQZ( 'S', JOBVSL, JOBVSR, N, 1, N, A, LDA, B, LDB,
     $                ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR,
     $                WORK, -1, IERR )
         LWKOPT = MAX( LWKOPT, 2*N+INT( WORK( 1 ) ) )
         IF( WANTST ) THEN
            CALL STGSEN( 0, ILVSL, ILVSR, BWORK, N, A, LDA, B, LDB,
     $                   ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR,
     $                   SDIM, PVSL, PVSR, DIF, WORK, -1, IDUM, 1,
     $                   IERR )
            LWKOPT = MAX( LWKOPT, 2*N+INT( WORK( 1 ) ) )
         END IF
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGGES3 ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
```

```
*
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SAFMIN = SLAMCH( 'S' )
      SAFMAX = ONE / SAFMIN
      CALL SLABAD( SAFMIN, SAFMAX )
      SMLNUM = SQRT( SAFMIN ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = SLANGE( 'M', N, N, A, LDA, WORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
      IF( ILASCL )
     $   CALL SLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = SLANGE( 'M', N, N, B, LDB, WORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
     $   CALL SLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrix to make it more nearly triangular
*
      ILEFT = 1
      IRIGHT = N + 1
      IWRK = IRIGHT + N
      CALL SGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, WORK( ILEFT ),
     $             WORK( IRIGHT ), WORK( IWRK ), IERR )
```

```
*
*     Reduce B to triangular form (QR decomposition of B)
*
      IROWS = IHI + 1 - ILO
      ICOLS = N + 1 - ILO
      ITAU = IWRK
      IWRK = ITAU + IROWS
      CALL SGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to matrix A
*
      CALL SORMQR( 'L', 'T', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VSL
*
      IF( ILVSL ) THEN
         CALL SLASET( 'Full', N, N, ZERO, ONE, VSL, LDVSL )
         IF( IROWS.GT.1 ) THEN
            CALL SLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VSL( ILO+1, ILO ), LDVSL )
         END IF
         CALL SORGQR( IROWS, IROWS, IROWS, VSL( ILO, ILO ), LDVSL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Initialize VSR
*
      IF( ILVSR )
     $   CALL SLASET( 'Full', N, N, ZERO, ONE, VSR, LDVSR )
*
*     Reduce to generalized Hessenberg form
*
      CALL SGGHD3( JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB, VSL,
     $             LDVSL, VSR, LDVSR, WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Perform QZ algorithm, computing Schur vectors if desired
*
      IWRK = ITAU
      CALL SHGEQZ( 'S', JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR,
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
```

```
            INFO = N + 1
         END IF
         GO TO 40
      END IF
*
*     Sort eigenvalues ALPHA/BETA if desired
*
      SDIM = 0
      IF( WANTST ) THEN
*
*        Undo scaling on eigenvalues before SELCTGing
*
         IF( ILASCL ) THEN
            CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAR, N,
     $                   IERR )
            CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAI, N,
     $                   IERR )
         END IF
         IF( ILBSCL )
     $      CALL SLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
*
*        Select eigenvalues
*
         DO 10 I = 1, N
            BWORK( I ) = SELCTG( ALPHAR( I ), ALPHAI( I ), BETA( I ) )
   10    CONTINUE
*
         CALL STGSEN( 0, ILVSL, ILVSR, BWORK, N, A, LDA, B, LDB, ALPHAR,
     $                ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR, SDIM, PVSL,
     $                PVSR, DIF, WORK( IWRK ), LWORK-IWRK+1, IDUM, 1,
     $                IERR )
         IF( IERR.EQ.1 )
     $      INFO = N + 3
*
      END IF
*
*     Apply back-permutation to VSL and VSR
*
      IF( ILVSL )
     $   CALL SGGBAK( 'P', 'L', N, ILO, IHI, WORK( ILEFT ),
     $                WORK( IRIGHT ), N, VSL, LDVSL, IERR )
*
      IF( ILVSR )
     $   CALL SGGBAK( 'P', 'R', N, ILO, IHI, WORK( ILEFT ),
     $                WORK( IRIGHT ), N, VSR, LDVSR, IERR )
*
*     Check if unscaling would cause over/underflow, if so, rescale
*     (ALPHAR(I),ALPHAI(I),BETA(I)) so BETA(I) is on the order of
*     B(I,I) and ALPHAR(I) and ALPHAI(I) are on the order of A(I,I)
*
```

```
      IF( ILASCL )THEN
         DO 50 I = 1, N
            IF( ALPHAI( I ).NE.ZERO ) THEN
               IF( ( ALPHAR( I )/SAFMAX ).GT.( ANRMTO/ANRM ) .OR.
     $             ( SAFMIN/ALPHAR( I ) ).GT.( ANRM/ANRMTO ) ) THEN
                  WORK( 1 ) = ABS( A( I, I )/ALPHAR( I ) )
                  BETA( I ) = BETA( I )*WORK( 1 )
                  ALPHAR( I ) = ALPHAR( I )*WORK( 1 )
                  ALPHAI( I ) = ALPHAI( I )*WORK( 1 )
               ELSE IF( ( ALPHAI( I )/SAFMAX ).GT.( ANRMTO/ANRM ) .OR.
     $             ( SAFMIN/ALPHAI( I ) ).GT.( ANRM/ANRMTO ) ) THEN
                  WORK( 1 ) = ABS( A( I, I+1 )/ALPHAI( I ) )
                  BETA( I ) = BETA( I )*WORK( 1 )
                  ALPHAR( I ) = ALPHAR( I )*WORK( 1 )
                  ALPHAI( I ) = ALPHAI( I )*WORK( 1 )
               END IF
            END IF
   50    CONTINUE
      END IF
*
      IF( ILBSCL )THEN
         DO 60 I = 1, N
            IF( ALPHAI( I ).NE.ZERO ) THEN
                IF( ( BETA( I )/SAFMAX ).GT.( BNRMTO/BNRM ) .OR.
     $              ( SAFMIN/BETA( I ) ).GT.( BNRM/BNRMTO ) ) THEN
                  WORK( 1 ) = ABS(B( I, I )/BETA( I ))
                  BETA( I ) = BETA( I )*WORK( 1 )
                  ALPHAR( I ) = ALPHAR( I )*WORK( 1 )
                  ALPHAI( I ) = ALPHAI( I )*WORK( 1 )
                END IF
             END IF
   60    CONTINUE
      END IF
*
*     Undo scaling
*
      IF( ILASCL ) THEN
         CALL SLASCL( 'H', 0, 0, ANRMTO, ANRM, N, N, A, LDA, IERR )
         CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAR, N, IERR )
         CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAI, N, IERR )
      END IF
*
      IF( ILBSCL ) THEN
         CALL SLASCL( 'U', 0, 0, BNRMTO, BNRM, N, N, B, LDB, IERR )
         CALL SLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
      END IF
*
      IF( WANTST ) THEN
*
*        Check if reordering is correct
```

```
*
         LASTSL = .TRUE.
         LST2SL = .TRUE.
         SDIM = 0
         IP = 0
         DO 30 I = 1, N
            CURSL = SELCTG( ALPHAR( I ), ALPHAI( I ), BETA( I ) )
            IF( ALPHAI( I ).EQ.ZERO ) THEN
               IF( CURSL )
     $            SDIM = SDIM + 1
               IP = 0
               IF( CURSL .AND. .NOT.LASTSL )
     $            INFO = N + 2
            ELSE
               IF( IP.EQ.1 ) THEN
*
*                 Last eigenvalue of conjugate pair
*
                  CURSL = CURSL .OR. LASTSL
                  LASTSL = CURSL
                  IF( CURSL )
     $               SDIM = SDIM + 2
                  IP = -1
                  IF( CURSL .AND. .NOT.LST2SL )
     $               INFO = N + 2
               ELSE
*
*                 First eigenvalue of conjugate pair
*
                  IP = 1
               END IF
            END IF
            LST2SL = LASTSL
            LASTSL = CURSL
   30    CONTINUE
*
      END IF
*
   40 CONTINUE
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of SGGES3
*
      END
```

———-

25.8.4 lisp code

— sgges3.lisp —

———-

25.9 sggesx

25.9.1 Axiom unit tests

— sggesx.input —

```
)set break resume
)sys rm -f sggesx.output
)spool sggesx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

25.9.2 Axiom help page

— sggesx.help —

```
======================================================================
sggesx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sggesx (JOBVSL, JOBVSR, SORT, SELCTG, SENSE, N, A, LDA, B,
           LDB, SDIM, ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR, RCONDE,
           RCONDV, WORK, LWORK, IWORK, LIWORK, BWORK, INFO)
```

```
           SGGESX computes the eigenvalues, the Schur form, and, optionally,
          the matrix of Schur vectors for GE matrices

Function/Subroutine Documentation
   subroutine sggesx (character JOBVSL, character JOBVSR, character SORT,
       external SELCTG, character SENSE, integer N, real, dimension( lda, * )
       A, integer LDA, real, dimension( ldb, * ) B, integer LDB, integer SDIM,
       real, dimension( * ) ALPHAR, real, dimension( * ) ALPHAI, real,
       dimension( * ) BETA, real, dimension( ldvsl, * ) VSL, integer LDVSL,
       real, dimension( ldvsr, * ) VSR, integer LDVSR, real, dimension( 2 )
       RCONDE, real, dimension( 2 ) RCONDV, real, dimension( * ) WORK, integer
       LWORK, integer, dimension( * ) IWORK, integer LIWORK, logical,
       dimension( * ) BWORK, integer INFO)
        SGGESX computes the eigenvalues, the Schur form, and, optionally, the
       matrix of Schur vectors for GE matrices

Purpose:
   SGGESX computes for a pair of N-by-N real nonsymmetric matrices
   (A,B), the generalized eigenvalues, the real Schur form (S,T), and,
   optionally, the left and/or right matrices of Schur vectors (VSL and
   VSR).  This gives the generalized Schur factorization

   (A,B) = ( (VSL) S (VSR)**T, (VSL) T (VSR)**T )

   Optionally, it also orders the eigenvalues so that a selected cluster
   of eigenvalues appears in the leading diagonal blocks of the upper
   quasi-triangular matrix S and the upper triangular matrix T; computes
   a reciprocal condition number for the average of the selected
   eigenvalues (RCONDE); and computes a reciprocal condition number for
   the right and left deflating subspaces corresponding to the selected
   eigenvalues (RCONDV). The leading columns of VSL and VSR then form
   an orthonormal basis for the corresponding left and right eigenspaces
   (deflating subspaces).

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar w
   or a ratio alpha/beta = w, such that  A - w*B is singular.  It is
   usually represented as the pair (alpha,beta), as there is a
   reasonable interpretation for beta=0 or for both being zero.

   A pair of matrices (S,T) is in generalized real Schur form if T is
   upper triangular with non-negative diagonal and S is block upper
   triangular with 1-by-1 and 2-by-2 blocks.  1-by-1 blocks correspond
   to real generalized eigenvalues, while 2-by-2 blocks of S will be
   "standardized" by making the corresponding elements of T have the form:
          [  a  0  ]
          [  0  b  ]

   and the pair of corresponding 2-by-2 blocks in S and T will have a
   complex conjugate pair of generalized eigenvalues.
```

```
Parameters:

 JOBVSL
   JOBVSL is CHARACTER*1
   = 'N':  do not compute the left Schur vectors;
   = 'V':  compute the left Schur vectors.

 JOBVSR
   JOBVSR is CHARACTER*1
   = 'N':  do not compute the right Schur vectors;
   = 'V':  compute the right Schur vectors.

 SORT
   SORT is CHARACTER*1
   Specifies whether or not to order the eigenvalues on the
   diagonal of the generalized Schur form.
   = 'N':  Eigenvalues are not ordered;
   = 'S':  Eigenvalues are ordered (see SELCTG).

 SELCTG
   SELCTG is procedure) LOGICAL FUNCTION of three REAL arguments
   SELCTG must be declared EXTERNAL in the calling subroutine.
   If SORT = 'N', SELCTG is not referenced.
   If SORT = 'S', SELCTG is used to select eigenvalues to sort
   to the top left of the Schur form.
   An eigenvalue (ALPHAR(j)+ALPHAI(j))/BETA(j) is selected if
   SELCTG(ALPHAR(j),ALPHAI(j),BETA(j)) is true; i.e. if either
   one of a complex conjugate pair of eigenvalues is selected,
   then both complex eigenvalues are selected.
   Note that a selected complex eigenvalue may no longer satisfy
   SELCTG(ALPHAR(j),ALPHAI(j),BETA(j)) = .TRUE. after ordering,
   since ordering may change the value of complex eigenvalues
   (especially if the eigenvalue is ill-conditioned), in this
   case INFO is set to N+3.

 SENSE
   SENSE is CHARACTER*1
   Determines which reciprocal condition numbers are computed.
   = 'N' : None are computed;
   = 'E' : Computed for average of selected eigenvalues only;
   = 'V' : Computed for selected deflating subspaces only;
   = 'B' : Computed for both.
   If SENSE = 'E', 'V', or 'B', SORT must equal 'S'.

 N
   N is INTEGER
   The order of the matrices A, B, VSL, and VSR.  N >= 0.

 A
   A is REAL array, dimension (LDA, N)
```

```
  On entry, the first of the pair of matrices.
  On exit, A has been overwritten by its generalized Schur form S.

LDA
  LDA is INTEGER
  The leading dimension of A.  LDA >= max(1,N).

B
  B is REAL array, dimension (LDB, N)
  On entry, the second of the pair of matrices.
  On exit, B has been overwritten by its generalized Schur form T.

LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).

SDIM
  SDIM is INTEGER
  If SORT = 'N', SDIM = 0.
  If SORT = 'S', SDIM = number of eigenvalues (after sorting)
  for which SELCTG is true.  (Complex conjugate pairs for which
  SELCTG is true for either eigenvalue count as 2.)

ALPHAR
  ALPHAR is REAL array, dimension (N)

ALPHAI
  ALPHAI is REAL array, dimension (N)

BETA
  BETA is REAL array, dimension (N)
  On exit, (ALPHAR(j) + ALPHAI(j)*i)/BETA(j), j=1,...,N, will
  be the generalized eigenvalues.  ALPHAR(j) + ALPHAI(j)*i
  and BETA(j),j=1,...,N  are the diagonals of the complex Schur
  form (S,T) that would result if the 2-by-2 diagonal blocks of
  the real Schur form of (A,B) were further reduced to
  triangular form using 2-by-2 complex unitary transformations.
  If ALPHAI(j) is zero, then the j-th eigenvalue is real; if
  positive, then the j-th and (j+1)-st eigenvalues are a
  complex conjugate pair, with ALPHAI(j+1) negative.

  Note: the quotients ALPHAR(j)/BETA(j) and ALPHAI(j)/BETA(j)
  may easily over- or underflow, and BETA(j) may even be zero.
  Thus, the user should avoid naively computing the ratio.
  However, ALPHAR and ALPHAI will be always less than and
  usually comparable with norm(A) in magnitude, and BETA always
  less than and usually comparable with norm(B).

VSL
  VSL is REAL array, dimension (LDVSL,N)
```

```
  If JOBVSL = 'V', VSL will contain the left Schur vectors.
  Not referenced if JOBVSL = 'N'.

LDVSL
  LDVSL is INTEGER
  The leading dimension of the matrix VSL. LDVSL >=1, and
  if JOBVSL = 'V', LDVSL >= N.

VSR
  VSR is REAL array, dimension (LDVSR,N)
  If JOBVSR = 'V', VSR will contain the right Schur vectors.
  Not referenced if JOBVSR = 'N'.

LDVSR
  LDVSR is INTEGER
  The leading dimension of the matrix VSR. LDVSR >= 1, and
  if JOBVSR = 'V', LDVSR >= N.

RCONDE
  RCONDE is REAL array, dimension ( 2 )
  If SENSE = 'E' or 'B', RCONDE(1) and RCONDE(2) contain the
  reciprocal condition numbers for the average of the selected
  eigenvalues.
  Not referenced if SENSE = 'N' or 'V'.

RCONDV
  RCONDV is REAL array, dimension ( 2 )
  If SENSE = 'V' or 'B', RCONDV(1) and RCONDV(2) contain the
  reciprocal condition numbers for the selected deflating subspaces.
  Not referenced if SENSE = 'N' or 'E'.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  If N = 0, LWORK >= 1, else if SENSE = 'E', 'V', or 'B',
  LWORK >= max( 8*N, 6*N+16, 2*SDIM*(N-SDIM) ), else
  LWORK >= max( 8*N, 6*N+16 ).
  Note that 2*SDIM*(N-SDIM) <= N*N/2.
  Note also that an error is only returned if
  LWORK < max( 8*N, 6*N+16), but if SENSE = 'E' or 'V' or 'B'
  this may not be large enough.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the bound on the optimal size of the WORK
  array and the minimum size of the IWORK array, returns these
  values as the first entries of the WORK and IWORK arrays, and
```

```
   no error message related to LWORK or LIWORK is issued by XERBLA.

 IWORK
   IWORK is INTEGER array, dimension (MAX(1,LIWORK))
   On exit, if INFO = 0, IWORK(1) returns the minimum LIWORK.

 LIWORK
   LIWORK is INTEGER
   The dimension of the array IWORK.
   If SENSE = 'N' or N = 0, LIWORK >= 1, otherwise
   LIWORK >= N+6.

   If LIWORK = -1, then a workspace query is assumed; the
   routine only calculates the bound on the optimal size of the
   WORK array and the minimum size of the IWORK array, returns
   these values as the first entries of the WORK and IWORK
   arrays, and no error message related to LWORK or LIWORK is
   issued by XERBLA.

 BWORK
   BWORK is LOGICAL array, dimension (N)
   Not referenced if SORT = 'N'.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value.
   = 1,...,N:
         The QZ iteration failed.  (A,B) are not in Schur
         form, but ALPHAR(j), ALPHAI(j), and BETA(j) should
         be correct for j=INFO+1,...,N.
   > N:  =N+1: other than QZ iteration failed in SHGEQZ
         =N+2: after reordering, roundoff changed values of
               some complex eigenvalues so that leading
               eigenvalues in the Generalized Schur form no
               longer satisfy SELCTG=.TRUE.  This could also
               be caused due to scaling.
         =N+3: reordering failed in STGSEN.

Further Details:
   An approximate (asymptotic) bound on the average absolute error of
   the selected eigenvalues is

   EPS * norm((A, B)) / RCONDE( 1 ).

   An approximate (asymptotic) bound on the maximum angular error in
   the computed deflating subspaces is

   EPS * norm((A, B)) / RCONDV( 2 ).
```

———-

25.9.3 fortran code

— **sggesx.f** —

```
*  =====================================================================
      SUBROUTINE SGGESX( JOBVSL, JOBVSR, SORT, SELCTG, SENSE, N, A, LDA,
     $                   B, LDB, SDIM, ALPHAR, ALPHAI, BETA, VSL, LDVSL,
     $                   VSR, LDVSR, RCONDE, RCONDV, WORK, LWORK, IWORK,
     $                   LIWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVSL, JOBVSR, SENSE, SORT
      INTEGER            INFO, LDA, LDB, LDVSL, LDVSR, LIWORK, LWORK, N,
     $                   SDIM
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      INTEGER            IWORK( * )
      REAL               A( LDA, * ), ALPHAI( * ), ALPHAR( * ),
     $                   B( LDB, * ), BETA( * ), RCONDE( 2 ),
     $                   RCONDV( 2 ), VSL( LDVSL, * ), VSR( LDVSR, * ),
     $                   WORK( * )
*     ..
*     .. Function Arguments ..
      LOGICAL            SELCTG
      EXTERNAL           SELCTG
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            CURSL, ILASCL, ILBSCL, ILVSL, ILVSR, LASTSL,
     $                   LQUERY, LST2SL, WANTSB, WANTSE, WANTSN, WANTST,
     $                   WANTSV
      INTEGER            I, ICOLS, IERR, IHI, IJOB, IJOBVL, IJOBVR,
     $                   ILEFT, ILO, IP, IRIGHT, IROWS, ITAU, IWRK,
     $                   LIWMIN, LWRK, MAXWRK, MINWRK
```

```
      REAL               ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS, PL,
     $                   PR, SAFMAX, SAFMIN, SMLNUM
*     ..
*     .. Local Arrays ..
      REAL               DIF( 2 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEQRF, SGGBAK, SGGBAL, SGGHRD, SHGEQZ, SLABAD,
     $                   SLACPY, SLASCL, SLASET, SORGQR, SORMQR, STGSEN,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               SLAMCH, SLANGE
      EXTERNAL           LSAME, ILAENV, SLAMCH, SLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVSL, 'N' ) ) THEN
         IJOBVL = 1
         ILVSL = .FALSE.
      ELSE IF( LSAME( JOBVSL, 'V' ) ) THEN
         IJOBVL = 2
         ILVSL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVSL = .FALSE.
      END IF
*
      IF( LSAME( JOBVSR, 'N' ) ) THEN
         IJOBVR = 1
         ILVSR = .FALSE.
      ELSE IF( LSAME( JOBVSR, 'V' ) ) THEN
         IJOBVR = 2
         ILVSR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVSR = .FALSE.
      END IF
*
      WANTST = LSAME( SORT, 'S' )
      WANTSN = LSAME( SENSE, 'N' )
      WANTSE = LSAME( SENSE, 'E' )
      WANTSV = LSAME( SENSE, 'V' )
```

```
      WANTSB = LSAME( SENSE, 'B' )
      LQUERY = ( LWORK.EQ.-1 .OR. LIWORK.EQ.-1 )
      IF( WANTSN ) THEN
         IJOB = 0
      ELSE IF( WANTSE ) THEN
         IJOB = 1
      ELSE IF( WANTSV ) THEN
         IJOB = 2
      ELSE IF( WANTSB ) THEN
         IJOB = 4
      END IF
*
*     Test the input arguments
*
      INFO = 0
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -2
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( .NOT.( WANTSN .OR. WANTSE .OR. WANTSV .OR. WANTSB ) .OR.
     $         ( .NOT.WANTST .AND. .NOT.WANTSN ) ) THEN
         INFO = -5
      ELSE IF( N.LT.0 ) THEN
         INFO = -6
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -10
      ELSE IF( LDVSL.LT.1 .OR. ( ILVSL .AND. LDVSL.LT.N ) ) THEN
         INFO = -16
      ELSE IF( LDVSR.LT.1 .OR. ( ILVSR .AND. LDVSR.LT.N ) ) THEN
         INFO = -18
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.GT.0) THEN
            MINWRK = MAX( 8*N, 6*N + 16 )
            MAXWRK = MINWRK - N +
     $               N*ILAENV( 1, 'SGEQRF', ' ', N, 1, N, 0 )
            MAXWRK = MAX( MAXWRK, MINWRK - N +
     $               N*ILAENV( 1, 'SORMQR', ' ', N, 1, N, -1 ) )
```

```
            IF( ILVSL ) THEN
               MAXWRK = MAX( MAXWRK, MINWRK - N +
     $                   N*ILAENV( 1, 'SORGQR', ' ', N, 1, N, -1 ) )
            END IF
            LWRK = MAXWRK
            IF( IJOB.GE.1 )
     $         LWRK = MAX( LWRK, N*N/2 )
         ELSE
            MINWRK = 1
            MAXWRK = 1
            LWRK   = 1
         END IF
         WORK( 1 ) = LWRK
         IF( WANTSN .OR. N.EQ.0 ) THEN
            LIWMIN = 1
         ELSE
            LIWMIN = N + 6
         END IF
         IWORK( 1 ) = LIWMIN
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -22
         ELSE IF( LIWORK.LT.LIWMIN  .AND. .NOT.LQUERY ) THEN
            INFO = -24
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGGESX', -INFO )
         RETURN
      ELSE IF (LQUERY) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SAFMIN = SLAMCH( 'S' )
      SAFMAX = ONE / SAFMIN
      CALL SLABAD( SAFMIN, SAFMAX )
      SMLNUM = SQRT( SAFMIN ) / EPS
      BIGNUM = ONE / SMLNUM
*
```

```
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = SLANGE( 'M', N, N, A, LDA, WORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
      IF( ILASCL )
     $   CALL SLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = SLANGE( 'M', N, N, B, LDB, WORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
     $   CALL SLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrix to make it more nearly triangular
*     (Workspace: need 6*N + 2*N for permutation parameters)
*
      ILEFT = 1
      IRIGHT = N + 1
      IWRK = IRIGHT + N
      CALL SGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, WORK( ILEFT ),
     $             WORK( IRIGHT ), WORK( IWRK ), IERR )
*
*     Reduce B to triangular form (QR decomposition of B)
*     (Workspace: need N, prefer N*NB)
*
      IROWS = IHI + 1 - ILO
      ICOLS = N + 1 - ILO
      ITAU = IWRK
      IWRK = ITAU + IROWS
      CALL SGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to matrix A
*     (Workspace: need N, prefer N*NB)
*
```

```
      CALL SORMQR( 'L', 'T', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VSL
*     (Workspace: need N, prefer N*NB)
*
      IF( ILVSL ) THEN
         CALL SLASET( 'Full', N, N, ZERO, ONE, VSL, LDVSL )
         IF( IROWS.GT.1 ) THEN
            CALL SLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VSL( ILO+1, ILO ), LDVSL )
         END IF
         CALL SORGQR( IROWS, IROWS, IROWS, VSL( ILO, ILO ), LDVSL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Initialize VSR
*
      IF( ILVSR )
     $   CALL SLASET( 'Full', N, N, ZERO, ONE, VSR, LDVSR )
*
*     Reduce to generalized Hessenberg form
*     (Workspace: none needed)
*
      CALL SGGHRD( JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB, VSL,
     $             LDVSL, VSR, LDVSR, IERR )
*
      SDIM = 0
*
*     Perform QZ algorithm, computing Schur vectors if desired
*     (Workspace: need N)
*
      IWRK = ITAU
      CALL SHGEQZ( 'S', JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR,
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 50
      END IF
*
*     Sort eigenvalues ALPHA/BETA and compute the reciprocal of
*     condition number(s)
```

```
*     (Workspace: If IJOB >= 1, need MAX( 8*(N+1), 2*SDIM*(N-SDIM) )
*                 otherwise, need 8*(N+1) )
*
      IF( WANTST ) THEN
*
*        Undo scaling on eigenvalues before SELCTGing
*
         IF( ILASCL ) THEN
            CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAR, N,
     $                   IERR )
            CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAI, N,
     $                   IERR )
         END IF
         IF( ILBSCL )
     $      CALL SLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
*
*        Select eigenvalues
*
         DO 10 I = 1, N
            BWORK( I ) = SELCTG( ALPHAR( I ), ALPHAI( I ), BETA( I ) )
   10    CONTINUE
*
*        Reorder eigenvalues, transform Generalized Schur vectors, and
*        compute reciprocal condition numbers
*
         CALL STGSEN( IJOB, ILVSL, ILVSR, BWORK, N, A, LDA, B, LDB,
     $                ALPHAR, ALPHAI, BETA, VSL, LDVSL, VSR, LDVSR,
     $                SDIM, PL, PR, DIF, WORK( IWRK ), LWORK-IWRK+1,
     $                IWORK, LIWORK, IERR )
*
         IF( IJOB.GE.1 )
     $      MAXWRK = MAX( MAXWRK, 2*SDIM*( N-SDIM ) )
         IF( IERR.EQ.-22 ) THEN
*
*           not enough real workspace
*
            INFO = -22
         ELSE
            IF( IJOB.EQ.1 .OR. IJOB.EQ.4 ) THEN
               RCONDE( 1 ) = PL
               RCONDE( 2 ) = PR
            END IF
            IF( IJOB.EQ.2 .OR. IJOB.EQ.4 ) THEN
               RCONDV( 1 ) = DIF( 1 )
               RCONDV( 2 ) = DIF( 2 )
            END IF
            IF( IERR.EQ.1 )
     $         INFO = N + 3
         END IF
*
```

```
      END IF
*
*     Apply permutation to VSL and VSR
*     (Workspace: none needed)
*
      IF( ILVSL )
     $   CALL SGGBAK( 'P', 'L', N, ILO, IHI, WORK( ILEFT ),
     $                WORK( IRIGHT ), N, VSL, LDVSL, IERR )
*
      IF( ILVSR )
     $   CALL SGGBAK( 'P', 'R', N, ILO, IHI, WORK( ILEFT ),
     $                WORK( IRIGHT ), N, VSR, LDVSR, IERR )
*
*     Check if unscaling would cause over/underflow, if so, rescale
*     (ALPHAR(I),ALPHAI(I),BETA(I)) so BETA(I) is on the order of
*     B(I,I) and ALPHAR(I) and ALPHAI(I) are on the order of A(I,I)
*
      IF( ILASCL ) THEN
         DO 20 I = 1, N
            IF( ALPHAI( I ).NE.ZERO ) THEN
               IF( ( ALPHAR( I ) / SAFMAX ).GT.( ANRMTO / ANRM ) .OR.
     $             ( SAFMIN / ALPHAR( I ) ).GT.( ANRM / ANRMTO ) )
     $            THEN
                  WORK( 1 ) = ABS( A( I, I ) / ALPHAR( I ) )
                  BETA( I ) = BETA( I )*WORK( 1 )
                  ALPHAR( I ) = ALPHAR( I )*WORK( 1 )
                  ALPHAI( I ) = ALPHAI( I )*WORK( 1 )
               ELSE IF( ( ALPHAI( I ) / SAFMAX ).GT.( ANRMTO / ANRM )
     $            .OR. ( SAFMIN / ALPHAI( I ) ).GT.( ANRM / ANRMTO ) )
     $            THEN
                  WORK( 1 ) = ABS( A( I, I+1 ) / ALPHAI( I ) )
                  BETA( I ) = BETA( I )*WORK( 1 )
                  ALPHAR( I ) = ALPHAR( I )*WORK( 1 )
                  ALPHAI( I ) = ALPHAI( I )*WORK( 1 )
               END IF
            END IF
   20    CONTINUE
      END IF
*
      IF( ILBSCL ) THEN
         DO 25 I = 1, N
            IF( ALPHAI( I ).NE.ZERO ) THEN
               IF( ( BETA( I ) / SAFMAX ).GT.( BNRMTO / BNRM ) .OR.
     $             ( SAFMIN / BETA( I ) ).GT.( BNRM / BNRMTO ) ) THEN
                  WORK( 1 ) = ABS( B( I, I ) / BETA( I ) )
                  BETA( I ) = BETA( I )*WORK( 1 )
                  ALPHAR( I ) = ALPHAR( I )*WORK( 1 )
                  ALPHAI( I ) = ALPHAI( I )*WORK( 1 )
               END IF
            END IF
```

```
   25    CONTINUE
      END IF
*
*     Undo scaling
*
      IF( ILASCL ) THEN
         CALL SLASCL( 'H', 0, 0, ANRMTO, ANRM, N, N, A, LDA, IERR )
         CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAR, N, IERR )
         CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAI, N, IERR )
      END IF
*
      IF( ILBSCL ) THEN
         CALL SLASCL( 'U', 0, 0, BNRMTO, BNRM, N, N, B, LDB, IERR )
         CALL SLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
      END IF
*
      IF( WANTST ) THEN
*
*        Check if reordering is correct
*
         LASTSL = .TRUE.
         LST2SL = .TRUE.
         SDIM = 0
         IP = 0
         DO 40 I = 1, N
            CURSL = SELCTG( ALPHAR( I ), ALPHAI( I ), BETA( I ) )
            IF( ALPHAI( I ).EQ.ZERO ) THEN
               IF( CURSL )
     $            SDIM = SDIM + 1
               IP = 0
               IF( CURSL .AND. .NOT.LASTSL )
     $            INFO = N + 2
            ELSE
               IF( IP.EQ.1 ) THEN
*
*                 Last eigenvalue of conjugate pair
*
                  CURSL = CURSL .OR. LASTSL
                  LASTSL = CURSL
                  IF( CURSL )
     $               SDIM = SDIM + 2
                  IP = -1
                  IF( CURSL .AND. .NOT.LST2SL )
     $               INFO = N + 2
               ELSE
*
*                 First eigenvalue of conjugate pair
*
                  IP = 1
               END IF
```

```
            END IF
            LST2SL = LASTSL
            LASTSL = CURSL
   40    CONTINUE
*
      END IF
*
   50 CONTINUE
*
      WORK( 1 ) = MAXWRK
      IWORK( 1 ) = LIWMIN
*
      RETURN
*
*     End of SGGESX
*
      END
```

———-

25.9.4 lisp code

— sggesx.lisp —

———-

25.10 sggev

25.10.1 Axiom unit tests

— sggev.input —

```
)set break resume
)sys rm -f sggev.output
)spool sggev.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

25.10.2 Axiom help page

— sggev.help —

```
=====================================================================
sggev examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sggev (JOBVL, JOBVR, N, A, LDA, B, LDB, ALPHAR, ALPHAI,
           BETA, VL, LDVL, VR, LDVR, WORK, LWORK, INFO)
            SGGEV computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for GE matrices

Function/Subroutine Documentation
   subroutine sggev (character JOBVL, character JOBVR, integer N, real,
       dimension( lda, * ) A, integer LDA, real, dimension( ldb, * ) B,
       integer LDB, real, dimension( * ) ALPHAR, real, dimension( * ) ALPHAI,
       real, dimension( * ) BETA, real, dimension( ldvl, * ) VL, integer LDVL,
       real, dimension( ldvr, * ) VR, integer LDVR, real, dimension( * ) WORK,
       integer LWORK, integer INFO)
        SGGEV computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for GE matrices

Purpose:
   SGGEV computes for a pair of N-by-N real nonsymmetric matrices (A,B)
   the generalized eigenvalues, and optionally, the left and/or right
   generalized eigenvectors.

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar
   lambda or a ratio alpha/beta = lambda, such that A - lambda*B is
   singular. It is usually represented as the pair (alpha,beta), as
   there is a reasonable interpretation for beta=0, and even for both
   being zero.

   The right eigenvector v(j) corresponding to the eigenvalue lambda(j)
   of (A,B) satisfies

   A * v(j) = lambda(j) * B * v(j).

   The left eigenvector u(j) corresponding to the eigenvalue lambda(j)
   of (A,B) satisfies

   u(j)**H * A  = lambda(j) * u(j)**H * B .
```

```
  where u(j)**H is the conjugate-transpose of u(j).

Parameters:

 JOBVL
   JOBVL is CHARACTER*1
   = 'N':  do not compute the left generalized eigenvectors;
   = 'V':  compute the left generalized eigenvectors.

 JOBVR
   JOBVR is CHARACTER*1
   = 'N':  do not compute the right generalized eigenvectors;
   = 'V':  compute the right generalized eigenvectors.

 N
   N is INTEGER
   The order of the matrices A, B, VL, and VR.  N >= 0.

 A
   A is REAL array, dimension (LDA, N)
   On entry, the matrix A in the pair (A,B).
   On exit, A has been overwritten.

 LDA
   LDA is INTEGER
   The leading dimension of A.  LDA >= max(1,N).

 B
   B is REAL array, dimension (LDB, N)
   On entry, the matrix B in the pair (A,B).
   On exit, B has been overwritten.

 LDB
   LDB is INTEGER
   The leading dimension of B.  LDB >= max(1,N).

 ALPHAR
   ALPHAR is REAL array, dimension (N)

 ALPHAI
   ALPHAI is REAL array, dimension (N)

 BETA
   BETA is REAL array, dimension (N)
   On exit, (ALPHAR(j) + ALPHAI(j)*i)/BETA(j), j=1,...,N, will
   be the generalized eigenvalues.  If ALPHAI(j) is zero, then
   the j-th eigenvalue is real; if positive, then the j-th and
   (j+1)-st eigenvalues are a complex conjugate pair, with
   ALPHAI(j+1) negative.
```

```
  Note: the quotients ALPHAR(j)/BETA(j) and ALPHAI(j)/BETA(j)
  may easily over- or underflow, and BETA(j) may even be zero.
  Thus, the user should avoid naively computing the ratio
  alpha/beta.  However, ALPHAR and ALPHAI will be always less
  than and usually comparable with norm(A) in magnitude, and
  BETA always less than and usually comparable with norm(B).

VL
  VL is REAL array, dimension (LDVL,N)
  If JOBVL = 'V', the left eigenvectors u(j) are stored one
  after another in the columns of VL, in the same order as
  their eigenvalues. If the j-th eigenvalue is real, then
  u(j) = VL(:,j), the j-th column of VL. If the j-th and
  (j+1)-th eigenvalues form a complex conjugate pair, then
  u(j) = VL(:,j)+i*VL(:,j+1) and u(j+1) = VL(:,j)-i*VL(:,j+1).
  Each eigenvector is scaled so the largest component has
  abs(real part)+abs(imag. part)=1.
  Not referenced if JOBVL = 'N'.

LDVL
  LDVL is INTEGER
  The leading dimension of the matrix VL. LDVL >= 1, and
  if JOBVL = 'V', LDVL >= N.

VR
  VR is REAL array, dimension (LDVR,N)
  If JOBVR = 'V', the right eigenvectors v(j) are stored one
  after another in the columns of VR, in the same order as
  their eigenvalues. If the j-th eigenvalue is real, then
  v(j) = VR(:,j), the j-th column of VR. If the j-th and
  (j+1)-th eigenvalues form a complex conjugate pair, then
  v(j) = VR(:,j)+i*VR(:,j+1) and v(j+1) = VR(:,j)-i*VR(:,j+1).
  Each eigenvector is scaled so the largest component has
  abs(real part)+abs(imag. part)=1.
  Not referenced if JOBVR = 'N'.

LDVR
  LDVR is INTEGER
  The leading dimension of the matrix VR. LDVR >= 1, and
  if JOBVR = 'V', LDVR >= N.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,8*N).
  For good performance, LWORK must generally be larger.
```

```
  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  = 1,...,N:
        The QZ iteration failed.  No eigenvectors have been
        calculated, but ALPHAR(j), ALPHAI(j), and BETA(j)
        should be correct for j=INFO+1,...,N.
  > N:  =N+1: other than QZ iteration failed in SHGEQZ.
        =N+2: error return from STGEVC.
```

———-

25.10.3 fortran code

— **sggev.f** —

```
*  =====================================================================
      SUBROUTINE SGGEV( JOBVL, JOBVR, N, A, LDA, B, LDB, ALPHAR, ALPHAI,
     $                  BETA, VL, LDVL, VR, LDVR, WORK, LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVL, JOBVR
      INTEGER            INFO, LDA, LDB, LDVL, LDVR, LWORK, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), ALPHAI( * ), ALPHAR( * ),
     $                   B( LDB, * ), BETA( * ), VL( LDVL, * ),
     $                   VR( LDVR, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
```

```
*     ..
*     .. Local Scalars ..
      LOGICAL            ILASCL, ILBSCL, ILV, ILVL, ILVR, LQUERY
      CHARACTER          CHTEMP
      INTEGER            ICOLS, IERR, IHI, IJOBVL, IJOBVR, ILEFT, ILO,
     $                   IN, IRIGHT, IROWS, ITAU, IWRK, JC, JR, MAXWRK,
     $                   MINWRK
      REAL               ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS,
     $                   SMLNUM, TEMP
*     ..
*     .. Local Arrays ..
      LOGICAL            LDUMMA( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEQRF, SGGBAK, SGGBAL, SGGHRD, SHGEQZ, SLABAD,
     $                   SLACPY, SLASCL, SLASET, SORGQR, SORMQR, STGEVC,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               SLAMCH, SLANGE
      EXTERNAL           LSAME, ILAENV, SLAMCH, SLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVL, 'N' ) ) THEN
         IJOBVL = 1
         ILVL = .FALSE.
      ELSE IF( LSAME( JOBVL, 'V' ) ) THEN
         IJOBVL = 2
         ILVL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVL = .FALSE.
      END IF
*
      IF( LSAME( JOBVR, 'N' ) ) THEN
         IJOBVR = 1
         ILVR = .FALSE.
      ELSE IF( LSAME( JOBVR, 'V' ) ) THEN
         IJOBVR = 2
         ILVR = .TRUE.
      ELSE
         IJOBVR = -1
```

```
         ILVR = .FALSE.
      END IF
      ILV = ILVL .OR. ILVR
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDVL.LT.1 .OR. ( ILVL .AND. LDVL.LT.N ) ) THEN
         INFO = -12
      ELSE IF( LDVR.LT.1 .OR. ( ILVR .AND. LDVR.LT.N ) ) THEN
         INFO = -14
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV. The workspace is
*       computed assuming ILO = 1 and IHI = N, the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = MAX( 1, 8*N )
         MAXWRK = MAX( 1, N*( 7 +
     $                 ILAENV( 1, 'SGEQRF', ' ', N, 1, N, 0 ) ) )
         MAXWRK = MAX( MAXWRK, N*( 7 +
     $                 ILAENV( 1, 'SORMQR', ' ', N, 1, N, 0 ) ) )
         IF( ILVL ) THEN
            MAXWRK = MAX( MAXWRK, N*( 7 +
     $                 ILAENV( 1, 'SORGQR', ' ', N, 1, N, -1 ) ) )
         END IF
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY )
     $      INFO = -16
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGGEV ', -INFO )
```

```
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = SLANGE( 'M', N, N, A, LDA, WORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
      IF( ILASCL )
     $   CALL SLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = SLANGE( 'M', N, N, B, LDB, WORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
     $   CALL SLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrices A, B to isolate eigenvalues if possible
*     (Workspace: need 6*N)
*
```

```
      ILEFT = 1
      IRIGHT = N + 1
      IWRK = IRIGHT + N
      CALL SGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, WORK( ILEFT ),
     $             WORK( IRIGHT ), WORK( IWRK ), IERR )
*
*     Reduce B to triangular form (QR decomposition of B)
*     (Workspace: need N, prefer N*NB)
*
      IROWS = IHI + 1 - ILO
      IF( ILV ) THEN
         ICOLS = N + 1 - ILO
      ELSE
         ICOLS = IROWS
      END IF
      ITAU = IWRK
      IWRK = ITAU + IROWS
      CALL SGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to matrix A
*     (Workspace: need N, prefer N*NB)
*
      CALL SORMQR( 'L', 'T', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VL
*     (Workspace: need N, prefer N*NB)
*
      IF( ILVL ) THEN
         CALL SLASET( 'Full', N, N, ZERO, ONE, VL, LDVL )
         IF( IROWS.GT.1 ) THEN
            CALL SLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VL( ILO+1, ILO ), LDVL )
         END IF
         CALL SORGQR( IROWS, IROWS, IROWS, VL( ILO, ILO ), LDVL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Initialize VR
*
      IF( ILVR )
     $   CALL SLASET( 'Full', N, N, ZERO, ONE, VR, LDVR )
*
*     Reduce to generalized Hessenberg form
*     (Workspace: none needed)
*
      IF( ILV ) THEN
*
```

```
*        Eigenvectors requested -- work on whole matrix.
*
         CALL SGGHRD( JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB, VL,
     $                LDVL, VR, LDVR, IERR )
      ELSE
         CALL SGGHRD( 'N', 'N', IROWS, 1, IROWS, A( ILO, ILO ), LDA,
     $                B( ILO, ILO ), LDB, VL, LDVL, VR, LDVR, IERR )
      END IF
*
*     Perform QZ algorithm (Compute eigenvalues, and optionally, the
*     Schur forms and Schur vectors)
*     (Workspace: need N)
*
      IWRK = ITAU
      IF( ILV ) THEN
         CHTEMP = 'S'
      ELSE
         CHTEMP = 'E'
      END IF
      CALL SHGEQZ( CHTEMP, JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHAR, ALPHAI, BETA, VL, LDVL, VR, LDVR,
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 110
      END IF
*
*     Compute Eigenvectors
*     (Workspace: need 6*N)
*
      IF( ILV ) THEN
         IF( ILVL ) THEN
            IF( ILVR ) THEN
               CHTEMP = 'B'
            ELSE
               CHTEMP = 'L'
            END IF
         ELSE
            CHTEMP = 'R'
         END IF
         CALL STGEVC( CHTEMP, 'B', LDUMMA, N, A, LDA, B, LDB, VL, LDVL,
     $                VR, LDVR, N, IN, WORK( IWRK ), IERR )
         IF( IERR.NE.0 ) THEN
            INFO = N + 2
```

```
            GO TO 110
         END IF
*
*        Undo balancing on VL and VR and normalization
*        (Workspace: none needed)
*
         IF( ILVL ) THEN
            CALL SGGBAK( 'P', 'L', N, ILO, IHI, WORK( ILEFT ),
     $                   WORK( IRIGHT ), N, VL, LDVL, IERR )
            DO 50 JC = 1, N
               IF( ALPHAI( JC ).LT.ZERO )
     $            GO TO 50
               TEMP = ZERO
               IF( ALPHAI( JC ).EQ.ZERO ) THEN
                  DO 10 JR = 1, N
                     TEMP = MAX( TEMP, ABS( VL( JR, JC ) ) )
   10             CONTINUE
               ELSE
                  DO 20 JR = 1, N
                     TEMP = MAX( TEMP, ABS( VL( JR, JC ) )+
     $                       ABS( VL( JR, JC+1 ) ) )
   20             CONTINUE
               END IF
               IF( TEMP.LT.SMLNUM )
     $            GO TO 50
               TEMP = ONE / TEMP
               IF( ALPHAI( JC ).EQ.ZERO ) THEN
                  DO 30 JR = 1, N
                     VL( JR, JC ) = VL( JR, JC )*TEMP
   30             CONTINUE
               ELSE
                  DO 40 JR = 1, N
                     VL( JR, JC ) = VL( JR, JC )*TEMP
                     VL( JR, JC+1 ) = VL( JR, JC+1 )*TEMP
   40             CONTINUE
               END IF
   50       CONTINUE
         END IF
         IF( ILVR ) THEN
            CALL SGGBAK( 'P', 'R', N, ILO, IHI, WORK( ILEFT ),
     $                   WORK( IRIGHT ), N, VR, LDVR, IERR )
            DO 100 JC = 1, N
               IF( ALPHAI( JC ).LT.ZERO )
     $            GO TO 100
               TEMP = ZERO
               IF( ALPHAI( JC ).EQ.ZERO ) THEN
                  DO 60 JR = 1, N
                     TEMP = MAX( TEMP, ABS( VR( JR, JC ) ) )
   60             CONTINUE
               ELSE
```

```
                 DO 70 JR = 1, N
                    TEMP = MAX( TEMP, ABS( VR( JR, JC ) )+
     $                     ABS( VR( JR, JC+1 ) ) )
   70            CONTINUE
              END IF
              IF( TEMP.LT.SMLNUM )
     $           GO TO 100
              TEMP = ONE / TEMP
              IF( ALPHAI( JC ).EQ.ZERO ) THEN
                 DO 80 JR = 1, N
                    VR( JR, JC ) = VR( JR, JC )*TEMP
   80            CONTINUE
              ELSE
                 DO 90 JR = 1, N
                    VR( JR, JC ) = VR( JR, JC )*TEMP
                    VR( JR, JC+1 ) = VR( JR, JC+1 )*TEMP
   90            CONTINUE
              END IF
  100       CONTINUE
         END IF
*
*        End of eigenvector calculation
*
      END IF
*
*     Undo scaling if necessary
*
  110 CONTINUE
*
      IF( ILASCL ) THEN
         CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAR, N, IERR )
         CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAI, N, IERR )
      END IF
*
      IF( ILBSCL ) THEN
         CALL SLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of SGGEV
*
      END
```

———-

25.10.4 lisp code

— **sggev.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sggev
         (jobvl jobvr n a lda b ldb$ alphar alphai beta vl ldvl vr ldvr work
          lwork info)
    (declare (type (array single-float (*)) work vr vl beta alphai alphar b a)
             (type (f2cl-lib:integer4) info lwork ldvr ldvl ldb$ lda n)
             (type (string *) jobvr jobvl))
    (f2cl-lib:with-multi-array-data
        ((jobvl character jobvl-%data% jobvl-%offset%)
         (jobvr character jobvr-%data% jobvr-%offset%)
         (a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%)
         (alphar single-float alphar-%data% alphar-%offset%)
         (alphai single-float alphai-%data% alphai-%offset%)
         (beta single-float beta-%data% beta-%offset%)
         (vl single-float vl-%data% vl-%offset%)
         (vr single-float vr-%data% vr-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((ldumma (make-array 1 :element-type 't)) (anrm 0.0) (anrmto 0.0)
             (bignum 0.0) (bnrm 0.0) (bnrmto 0.0) (eps 0.0) (smlnum 0.0)
             (temp 0.0) (icols 0) (ierr 0) (ihi 0) (ijobvl 0) (ijobvr 0)
             (ileft 0) (ilo 0) (in 0) (iright 0) (irows 0) (itau 0) (iwrk 0)
             (jc 0) (jr 0) (maxwrk 0) (minwrk 0)
             (chtemp
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (ilascl nil) (ilbscl nil) (ilv nil) (ilvl nil) (ilvr nil)
             (lquery nil))
        (declare (type (array f2cl-lib:logical (1)) ldumma)
                 (type (single-float) anrm anrmto bignum bnrm bnrmto eps smlnum
                  temp)
                 (type (f2cl-lib:integer4) icols ierr ihi ijobvl ijobvr ileft
                  ilo in iright irows itau iwrk jc jr maxwrk minwrk)
                 (type (string 1) chtemp)
                 (type f2cl-lib:logical ilascl ilbscl ilv ilvl ilvr lquery))
        (cond
         ((multiple-value-bind (ret-val var-0 var-1)
              (lsame jobvl "N")
            (declare (ignore var-1))
            (when var-0 (setf jobvl var-0))
            ret-val)
          (setf ijobvl 1) (setf ilvl f2cl-lib:%false%))
         ((multiple-value-bind (ret-val var-0 var-1)
```

```
      (lsame jobvl "V")
    (declare (ignore var-1))
    (when var-0 (setf jobvl var-0))
    ret-val)
  (setf ijobvl 2) (setf ilvl f2cl-lib:%true%))
 (t (setf ijobvl -1) (setf ilvl f2cl-lib:%false%)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvr "N")
    (declare (ignore var-1))
    (when var-0 (setf jobvr var-0))
    ret-val)
  (setf ijobvr 1) (setf ilvr f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvr "V")
    (declare (ignore var-1))
    (when var-0 (setf jobvr var-0))
    ret-val)
  (setf ijobvr 2) (setf ilvr f2cl-lib:%true%))
 (t (setf ijobvr -1) (setf ilvr f2cl-lib:%false%)))
(setf ilv (or ilvl ilvr))
(setf info 0)
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((<= ijobvl 0) (setf info -1)) ((<= ijobvr 0) (setf info -2))
      ((< n 0) (setf info -3))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -5))
      ((< ldb$
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -7))
      ((or (< ldvl 1) (and ilvl (< ldvl n))) (setf info -12))
      ((or (< ldvr 1) (and ilvr (< ldvr n))) (setf info -14)))
(cond
 ((= info 0)
  (setf minwrk
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul 8 n))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-mul n
                                      (f2cl-lib:int-add 7
                                                        (multiple-value-bind
                                                            (ret-val
                                                             var-0
                                                             var-1
                                                             var-2
                                                             var-3
                                                             var-4
```

```
                                                                var-5
                                                                var-6)
                                                               (ilaenv 1
                                                                "SGEQRF"
                                                                " " n 1 n
                                                                0)
                                                             (declare
                                                              (ignore
                                                               var-0 var-1
                                                               var-2 var-4
                                                               var-6))
                                                             (setf n
                                                                     var-3)
                                                             (setf n
                                                                     var-5)
                                                             ret-val))))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-mul n
                                            (f2cl-lib:int-add 7
                                                              (multiple-value-bind
                                                                (ret-val
                                                                 var-0
                                                                 var-1
                                                                 var-2
                                                                 var-3
                                                                 var-4
                                                                 var-5
                                                                 var-6)
                                                                (ilaenv 1
                                                                 "SORMQR"
                                                                 " " n 1 n
                                                                 0)
                                                             (declare
                                                              (ignore
                                                               var-0 var-1
                                                               var-2 var-4
                                                               var-6))
                                                             (setf n
                                                                     var-3)
                                                             (setf n
                                                                     var-5)
                                                             ret-val))))))
        (cond
         (ilvl
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-mul n
```

```
                                                    (f2cl-lib:int-add 7
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv 1
                                                                           "SORGQR"
                                                                           " " n 1
                                                                           n -1)
                                                                        (declare
                                                                         (ignore
                                                                          var-0
                                                                          var-1
                                                                          var-2
                                                                          var-4
                                                                          var-6))
                                                                        (setf n
                                                                                var-3)
                                                                        (setf n
                                                                                var-5)
                                                                        ret-val))))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 maxwrk) 'single-float))
  (if (and (< lwork minwrk) (not lquery))
      (setf info -16))))
(cond
 ((/= info 0) (xerbla "SGGEV " (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(setf eps (slamch "P"))
(setf smlnum (slamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (slabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "M" n n a lda work)
          (declare (ignore var-0 var-3 var-5))
```

```
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf ilascl f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf anrmto smlnum)
  (setf ilascl f2cl-lib:%true%))
 ((> anrm bignum) (setf anrmto bignum) (setf ilascl f2cl-lib:%true%)))
(if ilascl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl "G" 0 0 anrm anrmto n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf anrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "M" n n b ldb$ work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ilbscl f2cl-lib:%false%)
(cond
 ((and (> bnrm zero) (< bnrm smlnum)) (setf bnrmto smlnum)
  (setf ilbscl f2cl-lib:%true%))
 ((> bnrm bignum) (setf bnrmto bignum) (setf ilbscl f2cl-lib:%true%)))
(if ilbscl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl "G" 0 0 bnrm bnrmto n n b ldb$ ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bnrm var-3))
      (when var-4 (setf bnrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf ierr var-9))))
(setf ileft 1)
(setf iright (f2cl-lib:int-add n 1))
(setf iwrk (f2cl-lib:int-add iright n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
```

```
     var-11)
    (sggbal "P" n a lda b ldb$ ilo ihi
     (f2cl-lib:array-slice work-%data% single-float (ileft) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iright) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                           work-%offset%)
     ierr)
  (declare (ignore var-0 var-1 var-2 var-4 var-8 var-9 var-10))
  (setf lda var-3)
  (setf ldb$ var-5)
  (setf ilo var-6)
  (setf ihi var-7)
  (setf ierr var-11))
(setf irows (f2cl-lib:int-sub (f2cl-lib:int-add ihi 1) ilo))
(cond (ilv (setf icols (f2cl-lib:int-sub (f2cl-lib:int-add n 1) ilo)))
      (t (setf icols irows)))
(setf itau iwrk)
(setf iwrk (f2cl-lib:int-add itau irows))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (sgeqrf irows icols
     (f2cl-lib:array-slice b-%data% single-float (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf irows var-0)
  (setf icols var-1)
  (setf ldb$ var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (sormqr "L" "T" irows icols irows
     (f2cl-lib:array-slice b-%data% single-float (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice a-%data% single-float (ilo ilo)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
```

```
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf irows var-2))
  (when var-3 (setf icols var-3))
  (when var-4 (setf irows var-4))
  (when var-6 (setf ldb$ var-6))
  (when var-9 (setf lda var-9))
  (when var-12 (setf ierr var-12)))
(cond
 (ilvl
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "Full" n n zero one vl ldvl)
    (declare (ignore var-0 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldvl var-6)))
  (cond
   ((> irows 1)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slacpy "L" (f2cl-lib:int-sub irows 1)
         (f2cl-lib:int-sub irows 1)
         (f2cl-lib:array-slice b-%data% single-float ((+ ilo 1) ilo)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice vl-%data% single-float ((+ ilo 1) ilo)
                               ((1 ldvl) (1 *)) vl-%offset%)
         ldvl)
      (declare (ignore var-0 var-1 var-2 var-3 var-5))
      (when var-4 (setf ldb$ var-4))
      (when var-6 (setf ldvl var-6)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (sorgqr irows irows irows
       (f2cl-lib:array-slice vl-%data% single-float (ilo ilo)
                             ((1 ldvl) (1 *)) vl-%offset%)
       ldvl
       (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf irows var-0))
    (when var-1 (setf irows var-1))
    (when var-2 (setf irows var-2))
    (when var-4 (setf ldvl var-4))
    (when var-8 (setf ierr var-8)))))
(if ilvr
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
        (slaset "Full" n n zero one vr ldvr)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldvr var-6))))
(cond
 (ilv
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (sgghrd jobvl jobvr n ilo ihi a lda b ldb$ vl ldvl vr ldvr ierr)
    (declare (ignore var-5 var-7 var-9 var-11))
    (when var-0 (setf jobvl var-0))
    (when var-1 (setf jobvr var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-13 (setf ierr var-13))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (sgghrd "N" "N" irows 1 irows
       (f2cl-lib:array-slice a-%data% single-float (ilo ilo)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice b-%data% single-float (ilo ilo)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$ vl ldvl vr ldvr ierr)
    (declare (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11))
    (when var-2 (setf irows var-2))
    (when var-4 (setf irows var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-13 (setf ierr var-13)))))
(setf iwrk itau)
(cond (ilv (f2cl-lib:f2cl-set-string chtemp "S" (string 1)))
      (t (f2cl-lib:f2cl-set-string chtemp "E" (string 1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19)
    (shgeqz chtemp jobvl jobvr n ilo ihi a lda b ldb$ alphar alphai
```

```
      beta vl ldvl vr ldvr
      (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                            work-%offset%)
      (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare
   (ignore var-6 var-8 var-10 var-11 var-12 var-13 var-15 var-17
    var-18))
  (when var-0 (setf chtemp var-0))
  (when var-1 (setf jobvl var-1))
  (when var-2 (setf jobvr var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf ilo var-4))
  (when var-5 (setf ihi var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldb$ var-9))
  (when var-14 (setf ldvl var-14))
  (when var-16 (setf ldvr var-16))
  (when var-19 (setf ierr var-19)))
(cond
 ((/= ierr 0)
  (cond ((and (> ierr 0) (<= ierr n)) (setf info ierr))
        ((and (> ierr n) (<= ierr (f2cl-lib:int-mul 2 n)))
         (setf info (f2cl-lib:int-sub ierr n)))
        (t (setf info (f2cl-lib:int-add n 1))))
  (go label110)))
(cond
 (ilv
  (cond
   (ilvl
    (cond (ilvr (f2cl-lib:f2cl-set-string chtemp "B" (string 1)))
          (t (f2cl-lib:f2cl-set-string chtemp "L" (string 1)))))
   (t (f2cl-lib:f2cl-set-string chtemp "R" (string 1))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (stgevc chtemp "B" ldumma n a lda b ldb$ vl ldvl vr ldvr n in
       (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                             work-%offset%)
       ierr)
    (declare (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-14))
    (when var-0 (setf chtemp var-0))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldb$ var-7))
    (when var-9 (setf ldvl var-9))
    (when var-11 (setf ldvr var-11))
    (when var-12 (setf n var-12))
    (when var-13 (setf in var-13))
    (when var-15 (setf ierr var-15)))
  (cond ((/= ierr 0) (setf info (f2cl-lib:int-add n 2)) (go label110)))
```

```
(cond
 (ilvl
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (sggbak "P" "L" n ilo ihi
       (f2cl-lib:array-slice work-%data% single-float (ileft) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iright)
                             ((1 *)) work-%offset%)
       n vl ldvl ierr)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
    (setf n var-7)
    (setf ldvl var-9)
    (setf ierr var-10))
  (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                ((> jc n) nil)
    (tagbody
      (if (<
           (f2cl-lib:fref alphai-%data% (jc) ((1 *)) alphai-%offset%)
           zero)
          (go label50))
      (setf temp zero)
      (cond
       ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr n) nil)
          (tagbody
            (setf temp
                    (max temp
                         (abs
                          (f2cl-lib:fref vl-%data% (jr jc)
                                         ((1 ldvl) (1 *))
                                         vl-%offset%))))
           label10)))
       (t
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr n) nil)
          (tagbody
            (setf temp
                    (max temp
                         (+
                          (abs
                           (f2cl-lib:fref vl-%data% (jr jc)
                                          ((1 ldvl) (1 *))
                                          vl-%offset%))
                          (abs
                           (f2cl-lib:fref vl-%data%
                                          (jr
```

```
                                          (f2cl-lib:int-add jc 1))
                                         ((1 ldvl) (1 *))
                                         vl-%offset%)))))
            label20))))
        (if (< temp smlnum)
            (go label50))
        (setf temp (/ one temp))
        (cond
         ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                        ((> jr n) nil)
            (tagbody
              (setf (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                                   vl-%offset%)
                    (*
                     (f2cl-lib:fref vl-%data% (jr jc)
                                    ((1 ldvl) (1 *)) vl-%offset%)
                     temp))
            label30)))
         (t
          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                        ((> jr n) nil)
            (tagbody
              (setf (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                                   vl-%offset%)
                    (*
                     (f2cl-lib:fref vl-%data% (jr jc)
                                    ((1 ldvl) (1 *)) vl-%offset%)
                     temp))
              (setf (f2cl-lib:fref vl-%data%
                                   (jr (f2cl-lib:int-add jc 1))
                                   ((1 ldvl) (1 *)) vl-%offset%)
                    (*
                     (f2cl-lib:fref vl-%data%
                                    (jr (f2cl-lib:int-add jc 1))
                                    ((1 ldvl) (1 *)) vl-%offset%)
                     temp))
            label40))))
    label50))))
(cond
 (ilvr
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (sggbak "P" "R" n ilo ihi
       (f2cl-lib:array-slice work-%data% single-float (ileft) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iright)
                             ((1 *)) work-%offset%)
       n vr ldvr ierr)
```

```
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
  (setf n var-7)
  (setf ldvr var-9)
  (setf ierr var-10))
(f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
              ((> jc n) nil)
  (tagbody
    (if (<
         (f2cl-lib:fref alphai-%data% (jc) ((1 *)) alphai-%offset%)
         zero)
        (go label100))
    (setf temp zero)
    (cond
     ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf temp
                  (max temp
                       (abs
                        (f2cl-lib:fref vr-%data% (jr jc)
                                       ((1 ldvr) (1 *))
                                       vr-%offset%))))
         label60)))
     (t
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf temp
                  (max temp
                       (+
                        (abs
                         (f2cl-lib:fref vr-%data% (jr jc)
                                        ((1 ldvr) (1 *))
                                        vr-%offset%))
                        (abs
                         (f2cl-lib:fref vr-%data%
                                        (jr
                                         (f2cl-lib:int-add jc 1))
                                        ((1 ldvr) (1 *))
                                        vr-%offset%)))))
         label70))))
    (if (< temp smlnum)
        (go label100))
    (setf temp (/ one temp))
    (cond
     ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
```

```
                (tagbody
                  (setf (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                       vr-%offset%)
                        (*
                         (f2cl-lib:fref vr-%data% (jr jc)
                                        ((1 ldvr) (1 *)) vr-%offset%)
                         temp))
                 label80)))
            (t
             (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                           ((> jr n) nil)
                (tagbody
                  (setf (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                       vr-%offset%)
                        (*
                         (f2cl-lib:fref vr-%data% (jr jc)
                                        ((1 ldvr) (1 *)) vr-%offset%)
                         temp))
                  (setf (f2cl-lib:fref vr-%data%
                                       (jr (f2cl-lib:int-add jc 1))
                                       ((1 ldvr) (1 *)) vr-%offset%)
                        (*
                         (f2cl-lib:fref vr-%data%
                                        (jr (f2cl-lib:int-add jc 1))
                                        ((1 ldvr) (1 *)) vr-%offset%)
                         temp))
                 label90))))
        label100))))))
label110
 (cond
  (ilascl
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (slascl "G" 0 0 anrmto anrm n 1 alphar n ierr)
     (declare (ignore var-0 var-1 var-2 var-6 var-7))
     (when var-3 (setf anrmto var-3))
     (when var-4 (setf anrm var-4))
     (when var-5 (setf n var-5))
     (when var-8 (setf n var-8))
     (when var-9 (setf ierr var-9)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (slascl "G" 0 0 anrmto anrm n 1 alphai n ierr)
     (declare (ignore var-0 var-1 var-2 var-6 var-7))
     (when var-3 (setf anrmto var-3))
     (when var-4 (setf anrm var-4))
     (when var-5 (setf n var-5))
     (when var-8 (setf n var-8))
     (when var-9 (setf ierr var-9)))))
 (cond
```

```
         (ilbscl
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (slascl "G" 0 0 bnrmto bnrm n 1 beta n ierr)
            (declare (ignore var-0 var-1 var-2 var-6 var-7))
            (when var-3 (setf bnrmto var-3))
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf n var-5))
            (when var-8 (setf n var-8))
            (when var-9 (setf ierr var-9)))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 maxwrk) 'single-float))
        (go end_label)
       end_label
        (return
         (values jobvl jobvr n nil lda nil ldb$ nil nil nil nil ldvl nil ldvr
                 nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sggev fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::jobvl
                                              fortran-to-lisp::jobvr
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$ nil nil nil
                                              nil fortran-to-lisp::ldvl nil
                                              fortran-to-lisp::ldvr nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::sggbak
```

```
                                        fortran-to-lisp::sgeqrf
                                        fortran-to-lisp::sggbal
                                        fortran-to-lisp::ilaenv))))
```

———-

25.11 sggev3

25.11.1 Axiom unit tests

— **sggev3.input** —

```
)set break resume
)sys rm -f sggev3.output
)spool sggev3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

25.11.2 Axiom help page

— **sggev3.help** —

```
======================================================================
sggev3 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sggev3 (JOBVL, JOBVR, N, A, LDA, B, LDB, ALPHAR, ALPHAI,
           BETA, VL, LDVL, VR, LDVR, WORK, LWORK, INFO)
            SGGEV3 computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for GE matrices (blocked algorithm)
```

```
Function/Subroutine Documentation
   subroutine sggev3 (character JOBVL, character JOBVR, integer N, real,
       dimension( lda, * ) A, integer LDA, real, dimension( ldb, * ) B,
       integer LDB, real, dimension( * ) ALPHAR, real, dimension( * ) ALPHAI,
       real, dimension( * ) BETA, real, dimension( ldvl, * ) VL, integer LDVL,
       real, dimension( ldvr, * ) VR, integer LDVR, real, dimension( * ) WORK,
       integer LWORK, integer INFO)
        SGGEV3 computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for GE matrices (blocked algorithm)

Purpose:
   SGGEV3 computes for a pair of N-by-N real nonsymmetric matrices (A,B)
   the generalized eigenvalues, and optionally, the left and/or right
   generalized eigenvectors.

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar
   lambda or a ratio alpha/beta = lambda, such that A - lambda*B is
   singular. It is usually represented as the pair (alpha,beta), as
   there is a reasonable interpretation for beta=0, and even for both
   being zero.

   The right eigenvector v(j) corresponding to the eigenvalue lambda(j)
   of (A,B) satisfies

   A * v(j) = lambda(j) * B * v(j).

   The left eigenvector u(j) corresponding to the eigenvalue lambda(j)
   of (A,B) satisfies

   u(j)**H * A  = lambda(j) * u(j)**H * B .

   where u(j)**H is the conjugate-transpose of u(j).

Parameters:

 JOBVL
   JOBVL is CHARACTER*1
   = 'N':  do not compute the left generalized eigenvectors;
   = 'V':  compute the left generalized eigenvectors.

 JOBVR
   JOBVR is CHARACTER*1
   = 'N':  do not compute the right generalized eigenvectors;
   = 'V':  compute the right generalized eigenvectors.

 N
   N is INTEGER
   The order of the matrices A, B, VL, and VR.  N >= 0.
```

```
A
  A is REAL array, dimension (LDA, N)
  On entry, the matrix A in the pair (A,B).
  On exit, A has been overwritten.

LDA
  LDA is INTEGER
  The leading dimension of A.  LDA >= max(1,N).

B
  B is REAL array, dimension (LDB, N)
  On entry, the matrix B in the pair (A,B).
  On exit, B has been overwritten.

LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).

ALPHAR
  ALPHAR is REAL array, dimension (N)

ALPHAI
  ALPHAI is REAL array, dimension (N)

BETA
  BETA is REAL array, dimension (N)
  On exit, (ALPHAR(j) + ALPHAI(j)*i)/BETA(j), j=1,...,N, will
  be the generalized eigenvalues.  If ALPHAI(j) is zero, then
  the j-th eigenvalue is real; if positive, then the j-th and
  (j+1)-st eigenvalues are a complex conjugate pair, with
  ALPHAI(j+1) negative.

  Note: the quotients ALPHAR(j)/BETA(j) and ALPHAI(j)/BETA(j)
  may easily over- or underflow, and BETA(j) may even be zero.
  Thus, the user should avoid naively computing the ratio
  alpha/beta.  However, ALPHAR and ALPHAI will be always less
  than and usually comparable with norm(A) in magnitude, and
  BETA always less than and usually comparable with norm(B).

VL
  VL is REAL array, dimension (LDVL,N)
  If JOBVL = 'V', the left eigenvectors u(j) are stored one
  after another in the columns of VL, in the same order as
  their eigenvalues. If the j-th eigenvalue is real, then
  u(j) = VL(:,j), the j-th column of VL. If the j-th and
  (j+1)-th eigenvalues form a complex conjugate pair, then
  u(j) = VL(:,j)+i*VL(:,j+1) and u(j+1) = VL(:,j)-i*VL(:,j+1).
  Each eigenvector is scaled so the largest component has
  abs(real part)+abs(imag. part)=1.
  Not referenced if JOBVL = 'N'.
```

```
LDVL
  LDVL is INTEGER
  The leading dimension of the matrix VL. LDVL >= 1, and
  if JOBVL = 'V', LDVL >= N.

VR
  VR is REAL array, dimension (LDVR,N)
  If JOBVR = 'V', the right eigenvectors v(j) are stored one
  after another in the columns of VR, in the same order as
  their eigenvalues. If the j-th eigenvalue is real, then
  v(j) = VR(:,j), the j-th column of VR. If the j-th and
  (j+1)-th eigenvalues form a complex conjugate pair, then
  v(j) = VR(:,j)+i*VR(:,j+1) and v(j+1) = VR(:,j)-i*VR(:,j+1).
  Each eigenvector is scaled so the largest component has
  abs(real part)+abs(imag. part)=1.
  Not referenced if JOBVR = 'N'.

LDVR
  LDVR is INTEGER
  The leading dimension of the matrix VR. LDVR >= 1, and
  if JOBVR = 'V', LDVR >= N.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  = 1,...,N:
        The QZ iteration failed.  No eigenvectors have been
        calculated, but ALPHAR(j), ALPHAI(j), and BETA(j)
        should be correct for j=INFO+1,...,N.
  > N:  =N+1: other than QZ iteration failed in SHGEQZ.
        =N+2: error return from STGEVC.
```

———-

25.11.3 fortran code

— sggev3.f —

```
*  =====================================================================
      SUBROUTINE SGGEV3( JOBVL, JOBVR, N, A, LDA, B, LDB, ALPHAR,
     $                   ALPHAI, BETA, VL, LDVL, VR, LDVR, WORK, LWORK,
     $                   INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     January 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVL, JOBVR
      INTEGER            INFO, LDA, LDB, LDVL, LDVR, LWORK, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), ALPHAI( * ), ALPHAR( * ),
     $                   B( LDB, * ), BETA( * ), VL( LDVL, * ),
     $                   VR( LDVR, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ILASCL, ILBSCL, ILV, ILVL, ILVR, LQUERY
      CHARACTER          CHTEMP
      INTEGER            ICOLS, IERR, IHI, IJOBVL, IJOBVR, ILEFT, ILO,
     $                   IN, IRIGHT, IROWS, ITAU, IWRK, JC, JR, LWKOPT
      REAL               ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS,
     $                   SMLNUM, TEMP
*     ..
*     .. Local Arrays ..
      LOGICAL            LDUMMA( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEQRF, SGGBAK, SGGBAL, SGGHD3, SHGEQZ, SLABAD,
     $                   SLACPY, SLASCL, SLASET, SORGQR, SORMQR, STGEVC,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH, SLANGE
```

```
      EXTERNAL           LSAME, SLAMCH, SLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVL, 'N' ) ) THEN
         IJOBVL = 1
         ILVL = .FALSE.
      ELSE IF( LSAME( JOBVL, 'V' ) ) THEN
         IJOBVL = 2
         ILVL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVL = .FALSE.
      END IF
*
      IF( LSAME( JOBVR, 'N' ) ) THEN
         IJOBVR = 1
         ILVR = .FALSE.
      ELSE IF( LSAME( JOBVR, 'V' ) ) THEN
         IJOBVR = 2
         ILVR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVR = .FALSE.
      END IF
      ILV = ILVL .OR. ILVR
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDVL.LT.1 .OR. ( ILVL .AND. LDVL.LT.N ) ) THEN
         INFO = -12
      ELSE IF( LDVR.LT.1 .OR. ( ILVR .AND. LDVR.LT.N ) ) THEN
         INFO = -14
```

```
      ELSE IF( LWORK.LT.MAX( 1, 8*N ) .AND. .NOT.LQUERY ) THEN
         INFO = -16
      END IF
*
*     Compute workspace
*
      IF( INFO.EQ.0 ) THEN
         CALL SGEQRF( N, N, B, LDB, WORK, WORK, -1, IERR )
         LWKOPT = MAX( 1, 8*N, 3*N+INT ( WORK( 1 ) ) )
         CALL SORMQR( 'L', 'T', N, N, N, B, LDB, WORK, A, LDA, WORK,
     $               -1, IERR )
         LWKOPT = MAX( LWKOPT, 3*N+INT ( WORK( 1 ) ) )
         CALL SGGHD3( JOBVL, JOBVR, N, 1, N, A, LDA, B, LDB, VL, LDVL,
     $               VR, LDVR, WORK, -1, IERR )
         LWKOPT = MAX( LWKOPT, 3*N+INT ( WORK( 1 ) ) )
         IF( ILVL ) THEN
            CALL SORGQR( N, N, N, VL, LDVL, WORK, WORK, -1, IERR )
            LWKOPT = MAX( LWKOPT, 3*N+INT ( WORK( 1 ) ) )
            CALL SHGEQZ( 'S', JOBVL, JOBVR, N, 1, N, A, LDA, B, LDB,
     $                   ALPHAR, ALPHAI, BETA, VL, LDVL, VR, LDVR,
     $                   WORK, -1, IERR )
            LWKOPT = MAX( LWKOPT, 2*N+INT ( WORK( 1 ) ) )
         ELSE
            CALL SHGEQZ( 'E', JOBVL, JOBVR, N, 1, N, A, LDA, B, LDB,
     $                   ALPHAR, ALPHAI, BETA, VL, LDVL, VR, LDVR,
     $                   WORK, -1, IERR )
            LWKOPT = MAX( LWKOPT, 2*N+INT ( WORK( 1 ) ) )
         END IF
         WORK( 1 ) = REAL( LWKOPT )
*
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGGEV3 ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
```

```
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = SLANGE( 'M', N, N, A, LDA, WORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
      IF( ILASCL )
     $   CALL SLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = SLANGE( 'M', N, N, B, LDB, WORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
     $   CALL SLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrices A, B to isolate eigenvalues if possible
*
      ILEFT = 1
      IRIGHT = N + 1
      IWRK = IRIGHT + N
      CALL SGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, WORK( ILEFT ),
     $             WORK( IRIGHT ), WORK( IWRK ), IERR )
*
*     Reduce B to triangular form (QR decomposition of B)
*
      IROWS = IHI + 1 - ILO
      IF( ILV ) THEN
         ICOLS = N + 1 - ILO
      ELSE
         ICOLS = IROWS
      END IF
      ITAU = IWRK
      IWRK = ITAU + IROWS
      CALL SGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
```

```
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to matrix A
*
      CALL SORMQR( 'L', 'T', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VL
*
      IF( ILVL ) THEN
         CALL SLASET( 'Full', N, N, ZERO, ONE, VL, LDVL )
         IF( IROWS.GT.1 ) THEN
            CALL SLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VL( ILO+1, ILO ), LDVL )
         END IF
         CALL SORGQR( IROWS, IROWS, IROWS, VL( ILO, ILO ), LDVL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Initialize VR
*
      IF( ILVR )
     $   CALL SLASET( 'Full', N, N, ZERO, ONE, VR, LDVR )
*
*     Reduce to generalized Hessenberg form
*
      IF( ILV ) THEN
*
*        Eigenvectors requested -- work on whole matrix.
*
         CALL SGGHD3( JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB, VL,
     $                LDVL, VR, LDVR, WORK( IWRK ), LWORK+1-IWRK, IERR )
      ELSE
         CALL SGGHD3( 'N', 'N', IROWS, 1, IROWS, A( ILO, ILO ), LDA,
     $                B( ILO, ILO ), LDB, VL, LDVL, VR, LDVR,
     $                WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Perform QZ algorithm (Compute eigenvalues, and optionally, the
*     Schur forms and Schur vectors)
*
      IWRK = ITAU
      IF( ILV ) THEN
         CHTEMP = 'S'
      ELSE
         CHTEMP = 'E'
      END IF
      CALL SHGEQZ( CHTEMP, JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHAR, ALPHAI, BETA, VL, LDVL, VR, LDVR,
```

```
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 110
      END IF
*
*     Compute Eigenvectors
*
      IF( ILV ) THEN
         IF( ILVL ) THEN
            IF( ILVR ) THEN
               CHTEMP = 'B'
            ELSE
               CHTEMP = 'L'
            END IF
         ELSE
            CHTEMP = 'R'
         END IF
         CALL STGEVC( CHTEMP, 'B', LDUMMA, N, A, LDA, B, LDB, VL, LDVL,
     $                VR, LDVR, N, IN, WORK( IWRK ), IERR )
         IF( IERR.NE.0 ) THEN
            INFO = N + 2
            GO TO 110
         END IF
*
*        Undo balancing on VL and VR and normalization
*
         IF( ILVL ) THEN
            CALL SGGBAK( 'P', 'L', N, ILO, IHI, WORK( ILEFT ),
     $                   WORK( IRIGHT ), N, VL, LDVL, IERR )
            DO 50 JC = 1, N
               IF( ALPHAI( JC ).LT.ZERO )
     $            GO TO 50
               TEMP = ZERO
               IF( ALPHAI( JC ).EQ.ZERO ) THEN
                  DO 10 JR = 1, N
                     TEMP = MAX( TEMP, ABS( VL( JR, JC ) ) )
   10             CONTINUE
               ELSE
                  DO 20 JR = 1, N
                     TEMP = MAX( TEMP, ABS( VL( JR, JC ) )+
     $                       ABS( VL( JR, JC+1 ) ) )
   20             CONTINUE
               END IF
```

```
               IF( TEMP.LT.SMLNUM )
     $            GO TO 50
               TEMP = ONE / TEMP
               IF( ALPHAI( JC ).EQ.ZERO ) THEN
                  DO 30 JR = 1, N
                     VL( JR, JC ) = VL( JR, JC )*TEMP
   30             CONTINUE
               ELSE
                  DO 40 JR = 1, N
                     VL( JR, JC ) = VL( JR, JC )*TEMP
                     VL( JR, JC+1 ) = VL( JR, JC+1 )*TEMP
   40             CONTINUE
               END IF
   50       CONTINUE
         END IF
         IF( ILVR ) THEN
            CALL SGGBAK( 'P', 'R', N, ILO, IHI, WORK( ILEFT ),
     $                   WORK( IRIGHT ), N, VR, LDVR, IERR )
            DO 100 JC = 1, N
               IF( ALPHAI( JC ).LT.ZERO )
     $            GO TO 100
               TEMP = ZERO
               IF( ALPHAI( JC ).EQ.ZERO ) THEN
                  DO 60 JR = 1, N
                     TEMP = MAX( TEMP, ABS( VR( JR, JC ) ) )
   60             CONTINUE
               ELSE
                  DO 70 JR = 1, N
                     TEMP = MAX( TEMP, ABS( VR( JR, JC ) )+
     $                     ABS( VR( JR, JC+1 ) ) )
   70             CONTINUE
               END IF
               IF( TEMP.LT.SMLNUM )
     $            GO TO 100
               TEMP = ONE / TEMP
               IF( ALPHAI( JC ).EQ.ZERO ) THEN
                  DO 80 JR = 1, N
                     VR( JR, JC ) = VR( JR, JC )*TEMP
   80             CONTINUE
               ELSE
                  DO 90 JR = 1, N
                     VR( JR, JC ) = VR( JR, JC )*TEMP
                     VR( JR, JC+1 ) = VR( JR, JC+1 )*TEMP
   90             CONTINUE
               END IF
  100       CONTINUE
         END IF
*
*        End of eigenvector calculation
*
```

```
      END IF
*
*     Undo scaling if necessary
*
  110 CONTINUE
*
      IF( ILASCL ) THEN
         CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAR, N, IERR )
         CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAI, N, IERR )
      END IF
*
      IF( ILBSCL ) THEN
         CALL SLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
      END IF
*
      WORK( 1 ) = REAL( LWKOPT )
      RETURN
*
*     End of SGGEV3
*
      END
```

———-

25.11.4 lisp code

— **sggev3.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sggev3
         (jobvl jobvr n a lda b ldb$ alphar alphai beta vl ldvl vr ldvr work
          lwork info)
    (declare (type (array single-float (*)) work vr vl beta alphai alphar b a)
             (type (f2cl-lib:integer4) info lwork ldvr ldvl ldb$ lda n)
             (type (string *) jobvr jobvl))
    (f2cl-lib:with-multi-array-data
        ((jobvl character jobvl-%data% jobvl-%offset%)
         (jobvr character jobvr-%data% jobvr-%offset%)
         (a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%)
         (alphar single-float alphar-%data% alphar-%offset%)
         (alphai single-float alphai-%data% alphai-%offset%)
         (beta single-float beta-%data% beta-%offset%)
         (vl single-float vl-%data% vl-%offset%)
         (vr single-float vr-%data% vr-%offset%)
```

```
    (work single-float work-%data% work-%offset%))
  (prog ((ldumma (make-array 1 :element-type 't)) (anrm 0.0) (anrmto 0.0)
         (bignum 0.0) (bnrm 0.0) (bnrmto 0.0) (eps 0.0) (smlnum 0.0)
         (temp 0.0) (icols 0) (ierr 0) (ihi 0) (ijobvl 0) (ijobvr 0)
         (ileft 0) (ilo 0) (in 0) (iright 0) (irows 0) (itau 0) (iwrk 0)
         (jc 0) (jr 0) (lwkopt 0)
         (chtemp
          (make-array '(1) :element-type 'character :initial-element #\ ))
         (ilascl nil) (ilbscl nil) (ilv nil) (ilvl nil) (ilvr nil)
         (lquery nil))
    (declare (type (array f2cl-lib:logical (1)) ldumma)
             (type (single-float) anrm anrmto bignum bnrm bnrmto eps smlnum
              temp)
             (type (f2cl-lib:integer4) icols ierr ihi ijobvl ijobvr ileft
              ilo in iright irows itau iwrk jc jr lwkopt)
             (type (string 1) chtemp)
             (type f2cl-lib:logical ilascl ilbscl ilv ilvl ilvr lquery))
    (cond
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame jobvl "N")
        (declare (ignore var-1))
        (when var-0 (setf jobvl var-0))
        ret-val)
      (setf ijobvl 1) (setf ilvl f2cl-lib:%false%))
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame jobvl "V")
        (declare (ignore var-1))
        (when var-0 (setf jobvl var-0))
        ret-val)
      (setf ijobvl 2) (setf ilvl f2cl-lib:%true%))
     (t (setf ijobvl -1) (setf ilvl f2cl-lib:%false%)))
    (cond
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame jobvr "N")
        (declare (ignore var-1))
        (when var-0 (setf jobvr var-0))
        ret-val)
      (setf ijobvr 1) (setf ilvr f2cl-lib:%false%))
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame jobvr "V")
        (declare (ignore var-1))
        (when var-0 (setf jobvr var-0))
        ret-val)
      (setf ijobvr 2) (setf ilvr f2cl-lib:%true%))
     (t (setf ijobvr -1) (setf ilvr f2cl-lib:%false%)))
    (setf ilv (or ilvl ilvr))
    (setf info 0)
    (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
    (cond ((<= ijobvl 0) (setf info -1)) ((<= ijobvr 0) (setf info -2))
          ((< n 0) (setf info -3))
```

```
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -5))
      ((< ldb$
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -7))
      ((or (< ldvl 1) (and ilvl (< ldvl n))) (setf info -12))
      ((or (< ldvr 1) (and ilvr (< ldvr n))) (setf info -14))
      ((and
        (< lwork
           (max (the f2cl-lib:integer4 1)
                (the f2cl-lib:integer4 (f2cl-lib:int-mul 8 n))))
        (not lquery))
       (setf info -16)))
(cond
 ((= info 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (sgeqrf n n b ldb$ work work -1 ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf n var-0)
    (setf n var-1)
    (setf ldb$ var-3)
    (setf ierr var-7))
  (setf lwkopt
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul 8 n))
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                      (f2cl-lib:int
                                       (f2cl-lib:fref work-%data% (1)
                                                      ((1 *))
                                                      work-%offset%))))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sormqr "L" "T" n n n b ldb$ work a lda work -1 ierr)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-6 (setf ldb$ var-6))
    (when var-9 (setf lda var-9))
    (when var-12 (setf ierr var-12)))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkopt)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                      (f2cl-lib:int
                                       (f2cl-lib:fref work-%data% (1)
```

```
                                                        ((1 *))
                                                        work-%offset%))))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15)
    (sgghd3 jobvl jobvr n 1 n a lda b ldb$ vl ldvl vr ldvr work -1
     ierr)
  (declare (ignore var-3 var-5 var-7 var-9 var-11 var-13 var-14))
  (when var-0 (setf jobvl var-0))
  (when var-1 (setf jobvr var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-10 (setf ldvl var-10))
  (when var-12 (setf ldvr var-12))
  (when var-15 (setf ierr var-15)))
(setf lwkopt
        (max (the f2cl-lib:integer4 lwkopt)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int
                                     (f2cl-lib:fref work-%data% (1)
                                                    ((1 *))
                                                    work-%offset%))))))
(cond
 (ilvl
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (sorgqr n n n vl ldvl work work -1 ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf n var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldvl var-4))
    (when var-8 (setf ierr var-8)))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkopt)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                      (f2cl-lib:int
                                       (f2cl-lib:fref work-%data% (1)
                                                      ((1 *))
                                                      work-%offset%))))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18
       var-19)
      (shgeqz "S" jobvl jobvr n 1 n a lda b ldb$ alphar alphai beta
       vl ldvl vr ldvr work -1 ierr)
```

```
        (declare
         (ignore var-0 var-4 var-6 var-8 var-10 var-11 var-12 var-13
          var-15 var-17 var-18))
        (when var-1 (setf jobvl var-1))
        (when var-2 (setf jobvr var-2))
        (when var-3 (setf n var-3))
        (when var-5 (setf n var-5))
        (when var-7 (setf lda var-7))
        (when var-9 (setf ldb$ var-9))
        (when var-14 (setf ldvl var-14))
        (when var-16 (setf ldvr var-16))
        (when var-19 (setf ierr var-19)))
      (setf lwkopt
              (max (the f2cl-lib:integer4 lwkopt)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                          (f2cl-lib:int
                                           (f2cl-lib:fref work-%data% (1)
                                                          ((1 *))
                                                          work-%offset%)))))))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18
          var-19)
         (shgeqz "E" jobvl jobvr n 1 n a lda b ldb$ alphar alphai beta
          vl ldvl vr ldvr work -1 ierr)
        (declare
         (ignore var-0 var-4 var-6 var-8 var-10 var-11 var-12 var-13
          var-15 var-17 var-18))
        (when var-1 (setf jobvl var-1))
        (when var-2 (setf jobvr var-2))
        (when var-3 (setf n var-3))
        (when var-5 (setf n var-5))
        (when var-7 (setf lda var-7))
        (when var-9 (setf ldb$ var-9))
        (when var-14 (setf ldvl var-14))
        (when var-16 (setf ldvr var-16))
        (when var-19 (setf ierr var-19)))
      (setf lwkopt
              (max (the f2cl-lib:integer4 lwkopt)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                          (f2cl-lib:int
                                           (f2cl-lib:fref work-%data% (1)
                                                          ((1 *))
                                                          work-%offset%))))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:freal lwkopt))))
(cond
```

```
 ((/= info 0) (xerbla "SGGEV3 " (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(setf eps (slamch "P"))
(setf smlnum (slamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (slabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "M" n n a lda work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf ilascl f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf anrmto smlnum)
  (setf ilascl f2cl-lib:%true%))
 ((> anrm bignum) (setf anrmto bignum) (setf ilascl f2cl-lib:%true%)))
(if ilascl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl "G" 0 0 anrm anrmto n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf anrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "M" n n b ldb$ work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ilbscl f2cl-lib:%false%)
```

```
(cond
 ((and (> bnrm zero) (< bnrm smlnum)) (setf bnrmto smlnum)
  (setf ilbscl f2cl-lib:%true%))
 ((> bnrm bignum) (setf bnrmto bignum) (setf ilbscl f2cl-lib:%true%)))
(if ilbscl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl "G" 0 0 bnrm bnrmto n n b ldb$ ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bnrm var-3))
      (when var-4 (setf bnrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf ierr var-9))))
(setf ileft 1)
(setf iright (f2cl-lib:int-add n 1))
(setf iwrk (f2cl-lib:int-add iright n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11)
    (sggbal "P" n a lda b ldb$ ilo ihi
     (f2cl-lib:array-slice work-%data% single-float (ileft) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iright) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                           work-%offset%)
     ierr)
  (declare (ignore var-0 var-1 var-2 var-4 var-8 var-9 var-10))
  (setf lda var-3)
  (setf ldb$ var-5)
  (setf ilo var-6)
  (setf ihi var-7)
  (setf ierr var-11))
(setf irows (f2cl-lib:int-sub (f2cl-lib:int-add ihi 1) ilo))
(cond (ilv (setf icols (f2cl-lib:int-sub (f2cl-lib:int-add n 1) ilo)))
      (t (setf icols irows)))
(setf itau iwrk)
(setf iwrk (f2cl-lib:int-add itau irows))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (sgeqrf irows icols
     (f2cl-lib:array-slice b-%data% single-float (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
```

```
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf irows var-0)
  (setf icols var-1)
  (setf ldb$ var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (sormqr "L" "T" irows icols irows
     (f2cl-lib:array-slice b-%data% single-float (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice a-%data% single-float (ilo ilo)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf irows var-2))
  (when var-3 (setf icols var-3))
  (when var-4 (setf irows var-4))
  (when var-6 (setf ldb$ var-6))
  (when var-9 (setf lda var-9))
  (when var-12 (setf ierr var-12)))
(cond
 (ilvl
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "Full" n n zero one vl ldvl)
    (declare (ignore var-0 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldvl var-6)))
  (cond
   ((> irows 1)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slacpy "L" (f2cl-lib:int-sub irows 1)
         (f2cl-lib:int-sub irows 1)
         (f2cl-lib:array-slice b-%data% single-float ((+ ilo 1) ilo)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice vl-%data% single-float ((+ ilo 1) ilo)
                               ((1 ldvl) (1 *)) vl-%offset%)
         ldvl)
      (declare (ignore var-0 var-1 var-2 var-3 var-5))
      (when var-4 (setf ldb$ var-4))
```

```
      (when var-6 (setf ldvl var-6)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (sorgqr irows irows irows
       (f2cl-lib:array-slice vl-%data% single-float (ilo ilo)
                             ((1 ldvl) (1 *)) vl-%offset%)
       ldvl
       (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf irows var-0))
    (when var-1 (setf irows var-1))
    (when var-2 (setf irows var-2))
    (when var-4 (setf ldvl var-4))
    (when var-8 (setf ierr var-8)))))
(if ilvr
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "Full" n n zero one vr ldvr)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldvr var-6))))
(cond
 (ilv
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (sgghd3 jobvl jobvr n ilo ihi a lda b ldb$ vl ldvl vr ldvr
       (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
    (declare (ignore var-5 var-7 var-9 var-11 var-13 var-14))
    (when var-0 (setf jobvl var-0))
    (when var-1 (setf jobvr var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-15 (setf ierr var-15))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
        var-10 var-11 var-12 var-13 var-14 var-15)
       (sgghd3 "N" "N" irows 1 irows
        (f2cl-lib:array-slice a-%data% single-float (ilo ilo)
                              ((1 lda) (1 *)) a-%offset%)
        lda
        (f2cl-lib:array-slice b-%data% single-float (ilo ilo)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$ vl ldvl vr ldvr
        (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                              work-%offset%)
        (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
    (declare
     (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-13 var-14))
    (when var-2 (setf irows var-2))
    (when var-4 (setf irows var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-15 (setf ierr var-15)))))
(setf iwrk itau)
(cond (ilv (f2cl-lib:f2cl-set-string chtemp "S" (string 1)))
      (t (f2cl-lib:f2cl-set-string chtemp "E" (string 1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19)
    (shgeqz chtemp jobvl jobvr n ilo ihi a lda b ldb$ alphar alphai
     beta vl ldvl vr ldvr
     (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare
   (ignore var-6 var-8 var-10 var-11 var-12 var-13 var-15 var-17
    var-18))
  (when var-0 (setf chtemp var-0))
  (when var-1 (setf jobvl var-1))
  (when var-2 (setf jobvr var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf ilo var-4))
  (when var-5 (setf ihi var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldb$ var-9))
  (when var-14 (setf ldvl var-14))
  (when var-16 (setf ldvr var-16))
  (when var-19 (setf ierr var-19)))
(cond
 ((/= ierr 0)
  (cond ((and (> ierr 0) (<= ierr n)) (setf info ierr))
        ((and (> ierr n) (<= ierr (f2cl-lib:int-mul 2 n)))
         (setf info (f2cl-lib:int-sub ierr n)))
```

```
        (t (setf info (f2cl-lib:int-add n 1))))
  (go label110)))
(cond
 (ilv
  (cond
   (ilvl
    (cond (ilvr (f2cl-lib:f2cl-set-string chtemp "B" (string 1)))
          (t (f2cl-lib:f2cl-set-string chtemp "L" (string 1)))))
   (t (f2cl-lib:f2cl-set-string chtemp "R" (string 1))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (stgevc chtemp "B" ldumma n a lda b ldb$ vl ldvl vr ldvr n in
       (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                             work-%offset%)
       ierr)
    (declare (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-14))
    (when var-0 (setf chtemp var-0))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldb$ var-7))
    (when var-9 (setf ldvl var-9))
    (when var-11 (setf ldvr var-11))
    (when var-12 (setf n var-12))
    (when var-13 (setf in var-13))
    (when var-15 (setf ierr var-15)))
  (cond ((/= ierr 0) (setf info (f2cl-lib:int-add n 2)) (go label110)))
  (cond
   (ilvl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (sggbak "P" "L" n ilo ihi
         (f2cl-lib:array-slice work-%data% single-float (ileft) ((1 *))
                               work-%offset%)
         (f2cl-lib:array-slice work-%data% single-float (iright)
                               ((1 *)) work-%offset%)
         n vl ldvl ierr)
      (declare
       (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
      (setf n var-7)
      (setf ldvl var-9)
      (setf ierr var-10))
    (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                  ((> jc n) nil)
      (tagbody
        (if (<
             (f2cl-lib:fref alphai-%data% (jc) ((1 *)) alphai-%offset%)
             zero)
            (go label50))
```

```
(setf temp zero)
(cond
 ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
      (setf temp
              (max temp
                   (abs
                    (f2cl-lib:fref vl-%data% (jr jc)
                                   ((1 ldvl) (1 *))
                                   vl-%offset%))))
     label10)))
 (t
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
      (setf temp
              (max temp
                   (+
                    (abs
                     (f2cl-lib:fref vl-%data% (jr jc)
                                    ((1 ldvl) (1 *))
                                    vl-%offset%))
                    (abs
                     (f2cl-lib:fref vl-%data%
                                    (jr
                                     (f2cl-lib:int-add jc 1))
                                    ((1 ldvl) (1 *))
                                    vl-%offset%)))))
     label20))))
(if (< temp smlnum)
    (go label50))
(setf temp (/ one temp))
(cond
 ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
      (setf (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                           vl-%offset%)
              (*
               (f2cl-lib:fref vl-%data% (jr jc)
                              ((1 ldvl) (1 *)) vl-%offset%)
               temp))
     label30)))
 (t
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
```

```
                (setf (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                                     vl-%offset%)
                        (*
                         (f2cl-lib:fref vl-%data% (jr jc)
                                        ((1 ldvl) (1 *)) vl-%offset%)
                         temp))
                (setf (f2cl-lib:fref vl-%data%
                                     (jr (f2cl-lib:int-add jc 1))
                                     ((1 ldvl) (1 *)) vl-%offset%)
                        (*
                         (f2cl-lib:fref vl-%data%
                                        (jr (f2cl-lib:int-add jc 1))
                                        ((1 ldvl) (1 *)) vl-%offset%)
                         temp))
              label40))))
          label50))))
     (cond
      (ilvr
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10)
           (sggbak "P" "R" n ilo ihi
            (f2cl-lib:array-slice work-%data% single-float (ileft) ((1 *))
                                  work-%offset%)
            (f2cl-lib:array-slice work-%data% single-float (iright)
                                  ((1 *)) work-%offset%)
            n vr ldvr ierr)
         (declare
          (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
         (setf n var-7)
         (setf ldvr var-9)
         (setf ierr var-10))
       (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                     ((> jc n) nil)
         (tagbody
           (if (<
                (f2cl-lib:fref alphai-%data% (jc) ((1 *)) alphai-%offset%)
                zero)
               (go label100))
           (setf temp zero)
           (cond
            ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
             (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                           ((> jr n) nil)
               (tagbody
                 (setf temp
                         (max temp
                              (abs
                               (f2cl-lib:fref vr-%data% (jr jc)
                                              ((1 ldvr) (1 *))
```

```
                                    vr-%offset%))))
     label60)))
 (t
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
      (setf temp
              (max temp
                   (+
                    (abs
                     (f2cl-lib:fref vr-%data% (jr jc)
                                    ((1 ldvr) (1 *))
                                    vr-%offset%))
                    (abs
                     (f2cl-lib:fref vr-%data%
                                    (jr
                                     (f2cl-lib:int-add jc 1))
                                    ((1 ldvr) (1 *))
                                    vr-%offset%)))))
     label70))))
(if (< temp smlnum)
    (go label100))
(setf temp (/ one temp))
(cond
 ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
      (setf (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                           vr-%offset%)
              (*
               (f2cl-lib:fref vr-%data% (jr jc)
                              ((1 ldvr) (1 *)) vr-%offset%)
               temp))
     label80)))
 (t
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
      (setf (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                           vr-%offset%)
              (*
               (f2cl-lib:fref vr-%data% (jr jc)
                              ((1 ldvr) (1 *)) vr-%offset%)
               temp))
      (setf (f2cl-lib:fref vr-%data%
                           (jr (f2cl-lib:int-add jc 1))
                           ((1 ldvr) (1 *)) vr-%offset%)
              (*
               (f2cl-lib:fref vr-%data%
```

```
                                                (jr (f2cl-lib:int-add jc 1))
                                                ((1 ldvr) (1 *)) vr-%offset%)
                                temp))
                        label90))))
                 label100))))))
        label110
         (cond
          (ilascl
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
               (slascl "G" 0 0 anrmto anrm n 1 alphar n ierr)
             (declare (ignore var-0 var-1 var-2 var-6 var-7))
             (when var-3 (setf anrmto var-3))
             (when var-4 (setf anrm var-4))
             (when var-5 (setf n var-5))
             (when var-8 (setf n var-8))
             (when var-9 (setf ierr var-9)))
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
               (slascl "G" 0 0 anrmto anrm n 1 alphai n ierr)
             (declare (ignore var-0 var-1 var-2 var-6 var-7))
             (when var-3 (setf anrmto var-3))
             (when var-4 (setf anrm var-4))
             (when var-5 (setf n var-5))
             (when var-8 (setf n var-8))
             (when var-9 (setf ierr var-9)))))
         (cond
          (ilbscl
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
               (slascl "G" 0 0 bnrmto bnrm n 1 beta n ierr)
             (declare (ignore var-0 var-1 var-2 var-6 var-7))
             (when var-3 (setf bnrmto var-3))
             (when var-4 (setf bnrm var-4))
             (when var-5 (setf n var-5))
             (when var-8 (setf n var-8))
             (when var-9 (setf ierr var-9)))))
         (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                 (f2cl-lib:freal lwkopt))
         (go end_label)
        end_label
         (return
          (values jobvl jobvr n nil lda nil ldb$ nil nil nil nil ldvl nil ldvr
                  nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sggev3
                 fortran-to-lisp::*f2cl-function-info*)
```

```
(fortran-to-lisp::make-f2cl-finfo :arg-types
                                  '((string) (string)
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (array single-float (*))
                                    (array single-float (*))
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (fortran-to-lisp::integer4))
                                  :return-values
                                  '(fortran-to-lisp::jobvl
                                    fortran-to-lisp::jobvr
                                    fortran-to-lisp::n nil
                                    fortran-to-lisp::lda nil
                                    fortran-to-lisp::ldb$ nil nil nil
                                    nil fortran-to-lisp::ldvl nil
                                    fortran-to-lisp::ldvr nil nil
                                    fortran-to-lisp::info)
                                  :calls
                                  '(fortran-to-lisp::sggbak
                                    fortran-to-lisp::sggbal
                                    fortran-to-lisp::sgeqrf))))
```

———-

25.12 sggevx

25.12.1 Axiom unit tests

— sggevx.input —

```
)set break resume
)sys rm -f sggevx.output
)spool sggevx.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

25.12.2 Axiom help page

— **sggevx.help** —

```
==========================================================================
sggevx examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sggevx (BALANC, JOBVL, JOBVR, SENSE, N, A, LDA, B, LDB,
           ALPHAR, ALPHAI, BETA, VL, LDVL, VR, LDVR, ILO, IHI, LSCALE, RSCALE,
           ABNRM, BBNRM, RCONDE, RCONDV, WORK, LWORK, IWORK, BWORK, INFO)
            SGGEVX computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for GE matrices

Function/Subroutine Documentation
   subroutine sggevx (character BALANC, character JOBVL, character JOBVR,
       character SENSE, integer N, real, dimension( lda, * ) A, integer LDA,
       real, dimension( ldb, * ) B, integer LDB, real, dimension( * ) ALPHAR,
       real, dimension( * ) ALPHAI, real, dimension( * ) BETA, real,
       dimension( ldvl, * ) VL, integer LDVL, real, dimension( ldvr, * ) VR,
       integer LDVR, integer ILO, integer IHI, real, dimension( * ) LSCALE,
       real, dimension( * ) RSCALE, real ABNRM, real BBNRM, real, dimension( *
       ) RCONDE, real, dimension( * ) RCONDV, real, dimension( * ) WORK,
       integer LWORK, integer, dimension( * ) IWORK, logical, dimension( * )
       BWORK, integer INFO)
        SGGEVX computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for GE matrices

Purpose:
   SGGEVX computes for a pair of N-by-N real nonsymmetric matrices (A,B)
   the generalized eigenvalues, and optionally, the left and/or right
   generalized eigenvectors.

   Optionally also, it computes a balancing transformation to improve
   the conditioning of the eigenvalues and eigenvectors (ILO, IHI,
   LSCALE, RSCALE, ABNRM, and BBNRM), reciprocal condition numbers for
```

```
   the eigenvalues (RCONDE), and reciprocal condition numbers for the
   right eigenvectors (RCONDV).

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar
   lambda or a ratio alpha/beta = lambda, such that A - lambda*B is
   singular. It is usually represented as the pair (alpha,beta), as
   there is a reasonable interpretation for beta=0, and even for both
   being zero.

   The right eigenvector v(j) corresponding to the eigenvalue lambda(j)
   of (A,B) satisfies

   A * v(j) = lambda(j) * B * v(j) .

   The left eigenvector u(j) corresponding to the eigenvalue lambda(j)
   of (A,B) satisfies

   u(j)**H * A  = lambda(j) * u(j)**H * B.

   where u(j)**H is the conjugate-transpose of u(j).

Parameters:

 BALANC
   BALANC is CHARACTER*1
   Specifies the balance option to be performed.
   = 'N':  do not diagonally scale or permute;
   = 'P':  permute only;
   = 'S':  scale only;
   = 'B':  both permute and scale.
   Computed reciprocal condition numbers will be for the
   matrices after permuting and/or balancing. Permuting does
   not change condition numbers (in exact arithmetic), but
   balancing does.

 JOBVL
   JOBVL is CHARACTER*1
   = 'N':  do not compute the left generalized eigenvectors;
   = 'V':  compute the left generalized eigenvectors.

 JOBVR
   JOBVR is CHARACTER*1
   = 'N':  do not compute the right generalized eigenvectors;
   = 'V':  compute the right generalized eigenvectors.

 SENSE
   SENSE is CHARACTER*1
   Determines which reciprocal condition numbers are computed.
   = 'N': none are computed;
   = 'E': computed for eigenvalues only;
```

```
  = 'V': computed for eigenvectors only;
  = 'B': computed for eigenvalues and eigenvectors.

N
  N is INTEGER
  The order of the matrices A, B, VL, and VR.  N >= 0.

A
  A is REAL array, dimension (LDA, N)
  On entry, the matrix A in the pair (A,B).
  On exit, A has been overwritten. If JOBVL='V' or JOBVR='V'
  or both, then A contains the first part of the real Schur
  form of the "balanced" versions of the input A and B.

LDA
  LDA is INTEGER
  The leading dimension of A.  LDA >= max(1,N).

B
  B is REAL array, dimension (LDB, N)
  On entry, the matrix B in the pair (A,B).
  On exit, B has been overwritten. If JOBVL='V' or JOBVR='V'
  or both, then B contains the second part of the real Schur
  form of the "balanced" versions of the input A and B.

LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).

ALPHAR
  ALPHAR is REAL array, dimension (N)

ALPHAI
  ALPHAI is REAL array, dimension (N)

BETA
  BETA is REAL array, dimension (N)
  On exit, (ALPHAR(j) + ALPHAI(j)*i)/BETA(j), j=1,...,N, will
  be the generalized eigenvalues.  If ALPHAI(j) is zero, then
  the j-th eigenvalue is real; if positive, then the j-th and
  (j+1)-st eigenvalues are a complex conjugate pair, with
  ALPHAI(j+1) negative.

  Note: the quotients ALPHAR(j)/BETA(j) and ALPHAI(j)/BETA(j)
  may easily over- or underflow, and BETA(j) may even be zero.
  Thus, the user should avoid naively computing the ratio
  ALPHA/BETA. However, ALPHAR and ALPHAI will be always less
  than and usually comparable with norm(A) in magnitude, and
  BETA always less than and usually comparable with norm(B).
```

```
VL
  VL is REAL array, dimension (LDVL,N)
  If JOBVL = 'V', the left eigenvectors u(j) are stored one
  after another in the columns of VL, in the same order as
  their eigenvalues. If the j-th eigenvalue is real, then
  u(j) = VL(:,j), the j-th column of VL. If the j-th and
  (j+1)-th eigenvalues form a complex conjugate pair, then
  u(j) = VL(:,j)+i*VL(:,j+1) and u(j+1) = VL(:,j)-i*VL(:,j+1).
  Each eigenvector will be scaled so the largest component have
  abs(real part) + abs(imag. part) = 1.
  Not referenced if JOBVL = 'N'.

LDVL
  LDVL is INTEGER
  The leading dimension of the matrix VL. LDVL >= 1, and
  if JOBVL = 'V', LDVL >= N.

VR
  VR is REAL array, dimension (LDVR,N)
  If JOBVR = 'V', the right eigenvectors v(j) are stored one
  after another in the columns of VR, in the same order as
  their eigenvalues. If the j-th eigenvalue is real, then
  v(j) = VR(:,j), the j-th column of VR. If the j-th and
  (j+1)-th eigenvalues form a complex conjugate pair, then
  v(j) = VR(:,j)+i*VR(:,j+1) and v(j+1) = VR(:,j)-i*VR(:,j+1).
  Each eigenvector will be scaled so the largest component have
  abs(real part) + abs(imag. part) = 1.
  Not referenced if JOBVR = 'N'.

LDVR
  LDVR is INTEGER
  The leading dimension of the matrix VR. LDVR >= 1, and
  if JOBVR = 'V', LDVR >= N.

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER
  ILO and IHI are integer values such that on exit
  A(i,j) = 0 and B(i,j) = 0 if i > j and
  j = 1,...,ILO-1 or i = IHI+1,...,N.
  If BALANC = 'N' or 'S', ILO = 1 and IHI = N.

LSCALE
  LSCALE is REAL array, dimension (N)
  Details of the permutations and scaling factors applied
  to the left side of A and B.  If PL(j) is the index of the
  row interchanged with row j, and DL(j) is the scaling
  factor applied to row j, then
```

```
  LSCALE(j) = PL(j)  for j = 1,...,ILO-1
            = DL(j)  for j = ILO,...,IHI
            = PL(j)  for j = IHI+1,...,N.
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

RSCALE
  RSCALE is REAL array, dimension (N)
  Details of the permutations and scaling factors applied
  to the right side of A and B.  If PR(j) is the index of the
  column interchanged with column j, and DR(j) is the scaling
  factor applied to column j, then
  RSCALE(j) = PR(j)  for j = 1,...,ILO-1
            = DR(j)  for j = ILO,...,IHI
            = PR(j)  for j = IHI+1,...,N
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

ABNRM
  ABNRM is REAL
  The one-norm of the balanced matrix A.

BBNRM
  BBNRM is REAL
  The one-norm of the balanced matrix B.

RCONDE
  RCONDE is REAL array, dimension (N)
  If SENSE = 'E' or 'B', the reciprocal condition numbers of
  the eigenvalues, stored in consecutive elements of the array.
  For a complex conjugate pair of eigenvalues two consecutive
  elements of RCONDE are set to the same value. Thus RCONDE(j),
  RCONDV(j), and the j-th columns of VL and VR all correspond
  to the j-th eigenpair.
  If SENSE = 'N' or 'V', RCONDE is not referenced.

RCONDV
  RCONDV is REAL array, dimension (N)
  If SENSE = 'V' or 'B', the estimated reciprocal condition
  numbers of the eigenvectors, stored in consecutive elements
  of the array. For a complex eigenvector two consecutive
  elements of RCONDV are set to the same value. If the
  eigenvalues cannot be reordered to compute RCONDV(j),
  RCONDV(j) is set to 0; this can only occur when the true
  value would be very small anyway.
  If SENSE = 'N' or 'E', RCONDV is not referenced.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.
```

```
 LWORK
   LWORK is INTEGER
   The dimension of the array WORK. LWORK >= max(1,2*N).
   If BALANC = 'S' or 'B', or JOBVL = 'V', or JOBVR = 'V',
   LWORK >= max(1,6*N).
   If SENSE = 'E', LWORK >= max(1,10*N).
   If SENSE = 'V' or 'B', LWORK >= 2*N*N+8*N+16.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 IWORK
   IWORK is INTEGER array, dimension (N+6)
   If SENSE = 'E', IWORK is not referenced.

 BWORK
   BWORK is LOGICAL array, dimension (N)
   If SENSE = 'N', BWORK is not referenced.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value.
   = 1,...,N:
         The QZ iteration failed.  No eigenvectors have been
         calculated, but ALPHAR(j), ALPHAI(j), and BETA(j)
         should be correct for j=INFO+1,...,N.
   > N:  =N+1: other than QZ iteration failed in SHGEQZ.
         =N+2: error return from STGEVC.

Further Details:
   Balancing a matrix pair (A,B) includes, first, permuting rows and
   columns to isolate eigenvalues, second, applying diagonal similarity
   transformation to the rows and columns to make the rows and columns
   as close in norm as possible. The computed reciprocal condition
   numbers correspond to the balanced matrix. Permuting rows and columns
   will not change the condition numbers (in exact arithmetic) but
   diagonal scaling will.  For further explanation of balancing, see
   section 4.11.1.2 of LAPACK Users' Guide.

   An approximate error bound on the chordal distance between the i-th
   computed generalized eigenvalue w and the corresponding exact
   eigenvalue lambda is

   chord(w, lambda) <= EPS * norm(ABNRM, BBNRM) / RCONDE(I)

   An approximate error bound for the angle between the i-th computed
```

```
eigenvector VL(i) or VR(i) is given by

EPS * norm(ABNRM, BBNRM) / DIF(i).

For further explanation of the reciprocal condition numbers RCONDE
and RCONDV, see section 4.11 of LAPACK User's Guide.
```

———-

25.12.3 fortran code

— **sggevx.f** —

```
*  =====================================================================
      SUBROUTINE SGGEVX( BALANC, JOBVL, JOBVR, SENSE, N, A, LDA, B, LDB,
     $                   ALPHAR, ALPHAI, BETA, VL, LDVL, VR, LDVR, ILO,
     $                   IHI, LSCALE, RSCALE, ABNRM, BBNRM, RCONDE,
     $                   RCONDV, WORK, LWORK, IWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          BALANC, JOBVL, JOBVR, SENSE
      INTEGER            IHI, ILO, INFO, LDA, LDB, LDVL, LDVR, LWORK, N
      REAL               ABNRM, BBNRM
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      INTEGER            IWORK( * )
      REAL               A( LDA, * ), ALPHAI( * ), ALPHAR( * ),
     $                   B( LDB, * ), BETA( * ), LSCALE( * ),
     $                   RCONDE( * ), RCONDV( * ), RSCALE( * ),
     $                   VL( LDVL, * ), VR( LDVR, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ILASCL, ILBSCL, ILV, ILVL, ILVR, LQUERY, NOSCL,
     $                   PAIR, WANTSB, WANTSE, WANTSN, WANTSV
```

```
      CHARACTER          CHTEMP
      INTEGER            I, ICOLS, IERR, IJOBVL, IJOBVR, IN, IROWS,
     $                   ITAU, IWRK, IWRK1, J, JC, JR, M, MAXWRK,
     $                   MINWRK, MM
      REAL               ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS,
     $                   SMLNUM, TEMP
*     ..
*     .. Local Arrays ..
      LOGICAL            LDUMMA( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEQRF, SGGBAK, SGGBAL, SGGHRD, SHGEQZ, SLABAD,
     $                   SLACPY, SLASCL, SLASET, SORGQR, SORMQR, STGEVC,
     $                   STGSNA, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               SLAMCH, SLANGE
      EXTERNAL           LSAME, ILAENV, SLAMCH, SLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVL, 'N' ) ) THEN
         IJOBVL = 1
         ILVL = .FALSE.
      ELSE IF( LSAME( JOBVL, 'V' ) ) THEN
         IJOBVL = 2
         ILVL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVL = .FALSE.
      END IF
*
      IF( LSAME( JOBVR, 'N' ) ) THEN
         IJOBVR = 1
         ILVR = .FALSE.
      ELSE IF( LSAME( JOBVR, 'V' ) ) THEN
         IJOBVR = 2
         ILVR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVR = .FALSE.
      END IF
      ILV = ILVL .OR. ILVR
```

```
*
      NOSCL  = LSAME( BALANC, 'N' ) .OR. LSAME( BALANC, 'P' )
      WANTSN = LSAME( SENSE, 'N' )
      WANTSE = LSAME( SENSE, 'E' )
      WANTSV = LSAME( SENSE, 'V' )
      WANTSB = LSAME( SENSE, 'B' )
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.( NOSCL .OR. LSAME( BALANC, 'S' ) .OR.
     $    LSAME( BALANC, 'B' ) ) ) THEN
         INFO = -1
      ELSE IF( IJOBVL.LE.0 ) THEN
         INFO = -2
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -3
      ELSE IF( .NOT.( WANTSN .OR. WANTSE .OR. WANTSB .OR. WANTSV ) )
     $          THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDVL.LT.1 .OR. ( ILVL .AND. LDVL.LT.N ) ) THEN
         INFO = -14
      ELSE IF( LDVR.LT.1 .OR. ( ILVR .AND. LDVR.LT.N ) ) THEN
         INFO = -16
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV. The workspace is
*       computed assuming ILO = 1 and IHI = N, the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            MAXWRK = 1
         ELSE
            IF( NOSCL .AND. .NOT.ILV ) THEN
               MINWRK = 2*N
            ELSE
               MINWRK = 6*N
```

```
            END IF
            IF( WANTSE ) THEN
               MINWRK = 10*N
            ELSE IF( WANTSV .OR. WANTSB ) THEN
               MINWRK = 2*N*( N + 4 ) + 16
            END IF
            MAXWRK = MINWRK
            MAXWRK = MAX( MAXWRK,
     $                    N + N*ILAENV( 1, 'SGEQRF', ' ', N, 1, N, 0 ) )
            MAXWRK = MAX( MAXWRK,
     $                    N + N*ILAENV( 1, 'SORMQR', ' ', N, 1, N, 0 ) )
            IF( ILVL ) THEN
               MAXWRK = MAX( MAXWRK, N +
     $                       N*ILAENV( 1, 'SORGQR', ' ', N, 1, N, 0 ) )
            END IF
         END IF
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -26
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGGEVX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = SLANGE( 'M', N, N, A, LDA, WORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
```

```
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
      IF( ILASCL )
     $   CALL SLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = SLANGE( 'M', N, N, B, LDB, WORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
     $   CALL SLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute and/or balance the matrix pair (A,B)
*     (Workspace: need 6*N if BALANC = 'S' or 'B', 1 otherwise)
*
      CALL SGGBAL( BALANC, N, A, LDA, B, LDB, ILO, IHI, LSCALE, RSCALE,
     $             WORK, IERR )
*
*     Compute ABNRM and BBNRM
*
      ABNRM = SLANGE( '1', N, N, A, LDA, WORK( 1 ) )
      IF( ILASCL ) THEN
         WORK( 1 ) = ABNRM
         CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, 1, 1, WORK( 1 ), 1,
     $                IERR )
         ABNRM = WORK( 1 )
      END IF
*
      BBNRM = SLANGE( '1', N, N, B, LDB, WORK( 1 ) )
      IF( ILBSCL ) THEN
         WORK( 1 ) = BBNRM
         CALL SLASCL( 'G', 0, 0, BNRMTO, BNRM, 1, 1, WORK( 1 ), 1,
     $                IERR )
         BBNRM = WORK( 1 )
      END IF
*
*     Reduce B to triangular form (QR decomposition of B)
*     (Workspace: need N, prefer N*NB )
*
```

```
      IROWS = IHI + 1 - ILO
      IF( ILV .OR. .NOT.WANTSN ) THEN
         ICOLS = N + 1 - ILO
      ELSE
         ICOLS = IROWS
      END IF
      ITAU = 1
      IWRK = ITAU + IROWS
      CALL SGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to A
*     (Workspace: need N, prefer N*NB)
*
      CALL SORMQR( 'L', 'T', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VL and/or VR
*     (Workspace: need N, prefer N*NB)
*
      IF( ILVL ) THEN
         CALL SLASET( 'Full', N, N, ZERO, ONE, VL, LDVL )
         IF( IROWS.GT.1 ) THEN
            CALL SLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VL( ILO+1, ILO ), LDVL )
         END IF
         CALL SORGQR( IROWS, IROWS, IROWS, VL( ILO, ILO ), LDVL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
      IF( ILVR )
     $   CALL SLASET( 'Full', N, N, ZERO, ONE, VR, LDVR )
*
*     Reduce to generalized Hessenberg form
*     (Workspace: none needed)
*
      IF( ILV .OR. .NOT.WANTSN ) THEN
*
*        Eigenvectors requested -- work on whole matrix.
*
         CALL SGGHRD( JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB, VL,
     $                LDVL, VR, LDVR, IERR )
      ELSE
         CALL SGGHRD( 'N', 'N', IROWS, 1, IROWS, A( ILO, ILO ), LDA,
     $                B( ILO, ILO ), LDB, VL, LDVL, VR, LDVR, IERR )
      END IF
*
*     Perform QZ algorithm (Compute eigenvalues, and optionally, the
*     Schur forms and Schur vectors)
```

```
*     (Workspace: need N)
*
      IF( ILV .OR. .NOT.WANTSN ) THEN
         CHTEMP = 'S'
      ELSE
         CHTEMP = 'E'
      END IF
*
      CALL SHGEQZ( CHTEMP, JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHAR, ALPHAI, BETA, VL, LDVL, VR, LDVR, WORK,
     $             LWORK, IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 130
      END IF
*
*     Compute Eigenvectors and estimate condition numbers if desired
*     (Workspace: STGEVC: need 6*N
*                 STGSNA: need 2*N*(N+2)+16 if SENSE = 'V' or 'B',
*                         need N otherwise )
*
      IF( ILV .OR. .NOT.WANTSN ) THEN
         IF( ILV ) THEN
            IF( ILVL ) THEN
               IF( ILVR ) THEN
                  CHTEMP = 'B'
               ELSE
                  CHTEMP = 'L'
               END IF
            ELSE
               CHTEMP = 'R'
            END IF
*
            CALL STGEVC( CHTEMP, 'B', LDUMMA, N, A, LDA, B, LDB, VL,
     $                   LDVL, VR, LDVR, N, IN, WORK, IERR )
            IF( IERR.NE.0 ) THEN
               INFO = N + 2
               GO TO 130
            END IF
         END IF
*
         IF( .NOT.WANTSN ) THEN
*
*           compute eigenvectors (STGEVC) and estimate condition
```

```
*           numbers (STGSNA). Note that the definition of the condition
*           number is not invariant under transformation (u,v) to
*           (Q*u, Z*v), where (u,v) are eigenvectors of the generalized
*           Schur form (S,T), Q and Z are orthogonal matrices. In order
*           to avoid using extra 2*N*N workspace, we have to recalculate
*           eigenvectors and estimate one condition numbers at a time.
*
            PAIR = .FALSE.
            DO 20 I = 1, N
*
               IF( PAIR ) THEN
                  PAIR = .FALSE.
                  GO TO 20
               END IF
               MM = 1
               IF( I.LT.N ) THEN
                  IF( A( I+1, I ).NE.ZERO ) THEN
                     PAIR = .TRUE.
                     MM = 2
                  END IF
               END IF
*
               DO 10 J = 1, N
                  BWORK( J ) = .FALSE.
   10          CONTINUE
               IF( MM.EQ.1 ) THEN
                  BWORK( I ) = .TRUE.
               ELSE IF( MM.EQ.2 ) THEN
                  BWORK( I ) = .TRUE.
                  BWORK( I+1 ) = .TRUE.
               END IF
*
               IWRK = MM*N + 1
               IWRK1 = IWRK + MM*N
*
*              Compute a pair of left and right eigenvectors.
*              (compute workspace: need up to 4*N + 6*N)
*
               IF( WANTSE .OR. WANTSB ) THEN
                  CALL STGEVC( 'B', 'S', BWORK, N, A, LDA, B, LDB,
     $                         WORK( 1 ), N, WORK( IWRK ), N, MM, M,
     $                         WORK( IWRK1 ), IERR )
                  IF( IERR.NE.0 ) THEN
                     INFO = N + 2
                     GO TO 130
                  END IF
               END IF
*
               CALL STGSNA( SENSE, 'S', BWORK, N, A, LDA, B, LDB,
     $                      WORK( 1 ), N, WORK( IWRK ), N, RCONDE( I ),
```

```
     $                      RCONDV( I ), MM, M, WORK( IWRK1 ),
     $                      LWORK-IWRK1+1, IWORK, IERR )
*
   20       CONTINUE
         END IF
      END IF
*
*     Undo balancing on VL and VR and normalization
*     (Workspace: none needed)
*
      IF( ILVL ) THEN
         CALL SGGBAK( BALANC, 'L', N, ILO, IHI, LSCALE, RSCALE, N, VL,
     $                LDVL, IERR )
*
         DO 70 JC = 1, N
            IF( ALPHAI( JC ).LT.ZERO )
     $         GO TO 70
            TEMP = ZERO
            IF( ALPHAI( JC ).EQ.ZERO ) THEN
               DO 30 JR = 1, N
                  TEMP = MAX( TEMP, ABS( VL( JR, JC ) ) )
   30          CONTINUE
            ELSE
               DO 40 JR = 1, N
                  TEMP = MAX( TEMP, ABS( VL( JR, JC ) )+
     $                   ABS( VL( JR, JC+1 ) ) )
   40          CONTINUE
            END IF
            IF( TEMP.LT.SMLNUM )
     $         GO TO 70
            TEMP = ONE / TEMP
            IF( ALPHAI( JC ).EQ.ZERO ) THEN
               DO 50 JR = 1, N
                  VL( JR, JC ) = VL( JR, JC )*TEMP
   50          CONTINUE
            ELSE
               DO 60 JR = 1, N
                  VL( JR, JC ) = VL( JR, JC )*TEMP
                  VL( JR, JC+1 ) = VL( JR, JC+1 )*TEMP
   60          CONTINUE
            END IF
   70    CONTINUE
      END IF
      IF( ILVR ) THEN
         CALL SGGBAK( BALANC, 'R', N, ILO, IHI, LSCALE, RSCALE, N, VR,
     $                LDVR, IERR )
         DO 120 JC = 1, N
            IF( ALPHAI( JC ).LT.ZERO )
     $         GO TO 120
            TEMP = ZERO
```

```
            IF( ALPHAI( JC ).EQ.ZERO ) THEN
               DO 80 JR = 1, N
                  TEMP = MAX( TEMP, ABS( VR( JR, JC ) ) )
   80          CONTINUE
            ELSE
               DO 90 JR = 1, N
                  TEMP = MAX( TEMP, ABS( VR( JR, JC ) )+
     $                   ABS( VR( JR, JC+1 ) ) )
   90          CONTINUE
            END IF
            IF( TEMP.LT.SMLNUM )
     $         GO TO 120
            TEMP = ONE / TEMP
            IF( ALPHAI( JC ).EQ.ZERO ) THEN
               DO 100 JR = 1, N
                  VR( JR, JC ) = VR( JR, JC )*TEMP
  100          CONTINUE
            ELSE
               DO 110 JR = 1, N
                  VR( JR, JC ) = VR( JR, JC )*TEMP
                  VR( JR, JC+1 ) = VR( JR, JC+1 )*TEMP
  110          CONTINUE
            END IF
  120    CONTINUE
      END IF
*
*     Undo scaling if necessary
*
  130 CONTINUE
*
      IF( ILASCL ) THEN
         CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAR, N, IERR )
         CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHAI, N, IERR )
      END IF
*
      IF( ILBSCL ) THEN
         CALL SLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of SGGEVX
*
      END
```

———-

25.12.4 lisp code

— sggevx.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sggevx
         (balanc jobvl jobvr sense n a lda b ldb$ alphar alphai beta vl ldvl vr
          ldvr ilo ihi lscale rscale abnrm bbnrm rconde rcondv work lwork iwork
          bwork info)
    (declare (type (array f2cl-lib:logical (*)) bwork)
             (type (array f2cl-lib:integer4 (*)) iwork)
             (type (single-float) bbnrm abnrm)
             (type (array single-float (*)) work rcondv rconde rscale lscale vr
              vl beta alphai alphar b a)
             (type (f2cl-lib:integer4) info lwork ihi ilo ldvr ldvl ldb$ lda n)
             (type (string *) sense jobvr jobvl balanc))
    (f2cl-lib:with-multi-array-data
        ((balanc character balanc-%data% balanc-%offset%)
         (jobvl character jobvl-%data% jobvl-%offset%)
         (jobvr character jobvr-%data% jobvr-%offset%)
         (sense character sense-%data% sense-%offset%)
         (a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%)
         (alphar single-float alphar-%data% alphar-%offset%)
         (alphai single-float alphai-%data% alphai-%offset%)
         (beta single-float beta-%data% beta-%offset%)
         (vl single-float vl-%data% vl-%offset%)
         (vr single-float vr-%data% vr-%offset%)
         (lscale single-float lscale-%data% lscale-%offset%)
         (rscale single-float rscale-%data% rscale-%offset%)
         (rconde single-float rconde-%data% rconde-%offset%)
         (rcondv single-float rcondv-%data% rcondv-%offset%)
         (work single-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%)
         (bwork f2cl-lib:logical bwork-%data% bwork-%offset%))
      (prog ((ldumma (make-array 1 :element-type 't)) (anrm 0.0) (anrmto 0.0)
             (bignum 0.0) (bnrm 0.0) (bnrmto 0.0) (eps 0.0) (smlnum 0.0)
             (temp 0.0) (i 0) (icols 0) (ierr 0) (ijobvl 0) (ijobvr 0) (in 0)
             (irows 0) (itau 0) (iwrk 0) (iwrk1 0) (j 0) (jc 0) (jr 0) (m 0)
             (maxwrk 0) (minwrk 0) (mm 0)
             (chtemp
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (ilascl nil) (ilbscl nil) (ilv nil) (ilvl nil) (ilvr nil)
             (lquery nil) (noscl nil) (pair nil) (wantsb nil) (wantse nil)
             (wantsn nil) (wantsv nil))
        (declare (type (array f2cl-lib:logical (1)) ldumma)
```

```
          (type (single-float) anrm anrmto bignum bnrm bnrmto eps smlnum
           temp)
          (type (f2cl-lib:integer4) i icols ierr ijobvl ijobvr in irows
           itau iwrk iwrk1 j jc jr m maxwrk minwrk mm)
          (type (string 1) chtemp)
          (type f2cl-lib:logical ilascl ilbscl ilv ilvl ilvr lquery
           noscl pair wantsb wantse wantsn wantsv))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvl "N")
    (declare (ignore var-1))
    (when var-0 (setf jobvl var-0))
    ret-val)
  (setf ijobvl 1) (setf ilvl f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvl "V")
    (declare (ignore var-1))
    (when var-0 (setf jobvl var-0))
    ret-val)
  (setf ijobvl 2) (setf ilvl f2cl-lib:%true%))
 (t (setf ijobvl -1) (setf ilvl f2cl-lib:%false%)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvr "N")
    (declare (ignore var-1))
    (when var-0 (setf jobvr var-0))
    ret-val)
  (setf ijobvr 1) (setf ilvr f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvr "V")
    (declare (ignore var-1))
    (when var-0 (setf jobvr var-0))
    ret-val)
  (setf ijobvr 2) (setf ilvr f2cl-lib:%true%))
 (t (setf ijobvr -1) (setf ilvr f2cl-lib:%false%)))
(setf ilv (or ilvl ilvr))
(setf noscl
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame balanc "N")
           (declare (ignore var-1))
           (when var-0 (setf balanc var-0))
           ret-val)
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame balanc "P")
           (declare (ignore var-1))
           (when var-0 (setf balanc var-0))
           ret-val)))
(setf wantsn
        (multiple-value-bind (ret-val var-0 var-1)
```

```
            (lsame sense "N")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wantse
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "E")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wantsv
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "V")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wantsb
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "B")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf info 0)
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond
 ((not
   (or noscl
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame balanc "S")
         (declare (ignore var-1))
         (when var-0 (setf balanc var-0))
         ret-val)
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame balanc "B")
         (declare (ignore var-1))
         (when var-0 (setf balanc var-0))
         ret-val)))
  (setf info -1))
 ((<= ijobvl 0) (setf info -2)) ((<= ijobvr 0) (setf info -3))
 ((not (or wantsn wantse wantsb wantsv)) (setf info -4))
 ((< n 0) (setf info -5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9))
 ((or (< ldvl 1) (and ilvl (< ldvl n))) (setf info -14))
 ((or (< ldvr 1) (and ilvr (< ldvr n))) (setf info -16)))
(cond
 ((= info 0)
  (cond ((= n 0) (setf minwrk 1) (setf maxwrk 1))
```

```
(t
 (cond
  ((and noscl (not ilv)) (setf minwrk (f2cl-lib:int-mul 2 n)))
  (t (setf minwrk (f2cl-lib:int-mul 6 n))))
 (cond (wantse (setf minwrk (f2cl-lib:int-mul 10 n)))
       ((or wantsv wantsb)
        (setf minwrk
                (f2cl-lib:int-add
                 (f2cl-lib:int-mul 2 n (f2cl-lib:int-add n 4))
                 16))))
 (setf maxwrk minwrk)
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n
                                     (f2cl-lib:int-mul n
                                                       (multiple-value-bind
                                                           (ret-val
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-3
                                                            var-4
                                                            var-5
                                                            var-6)
                                                           (ilaenv
                                                            1
                                                            "SGEQRF"
                                                            " "
                                                            n 1
                                                            n
                                                            0)
                                                         (declare
                                                          (ignore
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-4
                                                           var-6))
                                                         (setf n
                                                                 var-3)
                                                         (setf n
                                                                 var-5)
                                                         ret-val))))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n
                                     (f2cl-lib:int-mul n
                                                       (multiple-value-bind
```

```
                                                                  (ret-val
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-3
                                                                   var-4
                                                                   var-5
                                                                   var-6)
                                                                  (ilaenv
                                                                   1
                                                                   "SORMQR"
                                                                   " "
                                                                   n 1
                                                                   n
                                                                   0)
                                                                (declare
                                                                 (ignore
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-4
                                                                  var-6))
                                                                (setf n
                                                                        var-3)
                                                                (setf n
                                                                        var-5)
                                                                ret-val))))))
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
(cond
 (ilvl
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add n
                                      (f2cl-lib:int-mul n
                                                        (multiple-value-bind
                                                            (ret-val
                                                             var-0
                                                             var-1
                                                             var-2
                                                             var-3
                                                             var-4
                                                             var-5
                                                             var-6)
                                                            (ilaenv
                                                             1
                                                             "SORGQR"
                                                             " "
                                                             n
                                                             1
                                                             n
```

```
                                                                    0)
                                                                (declare
                                                                 (ignore
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-4
                                                                  var-6))
                                                                (setf n
                                                                        var-3)
                                                                (setf n
                                                                        var-5)
                                                               ret-val)))))))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 maxwrk) 'single-float))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -26)))))
(cond
 ((/= info 0) (xerbla "SGGEVX" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(setf eps (slamch "P"))
(setf smlnum (slamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (slabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "M" n n a lda work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf ilascl f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf anrmto smlnum)
  (setf ilascl f2cl-lib:%true%))
 ((> anrm bignum) (setf anrmto bignum) (setf ilascl f2cl-lib:%true%)))
(if ilascl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl "G" 0 0 anrm anrmto n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
```

```
      (when var-3 (setf anrm var-3))
      (when var-4 (setf anrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "M" n n b ldb$ work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ilbscl f2cl-lib:%false%)
(cond
 ((and (> bnrm zero) (< bnrm smlnum)) (setf bnrmto smlnum)
  (setf ilbscl f2cl-lib:%true%))
 ((> bnrm bignum) (setf bnrmto bignum) (setf ilbscl f2cl-lib:%true%)))
(if ilbscl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl "G" 0 0 bnrm bnrmto n n b ldb$ ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bnrm var-3))
      (when var-4 (setf bnrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf ierr var-9))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11)
    (sggbal balanc n a lda b ldb$ ilo ihi lscale rscale work ierr)
  (declare (ignore var-1 var-2 var-4 var-8 var-9 var-10))
  (setf balanc var-0)
  (setf lda var-3)
  (setf ldb$ var-5)
  (setf ilo var-6)
  (setf ihi var-7)
  (setf ierr var-11))
(setf abnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "1" n n a lda
             (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                                   work-%offset%))
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
```

```
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(cond
 (ilascl
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) abnrm)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 anrmto anrm 1 1
       (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                             work-%offset%)
       1 ierr)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
    (when var-3 (setf anrmto var-3))
    (when var-4 (setf anrm var-4))
    (when var-9 (setf ierr var-9)))
  (setf abnrm (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%))))
(setf bbnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "1" n n b ldb$
             (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                                   work-%offset%))
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(cond
 (ilbscl
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) bbnrm)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 bnrmto bnrm 1 1
       (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                             work-%offset%)
       1 ierr)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
    (when var-3 (setf bnrmto var-3))
    (when var-4 (setf bnrm var-4))
    (when var-9 (setf ierr var-9)))
  (setf bbnrm (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%))))
(setf irows (f2cl-lib:int-sub (f2cl-lib:int-add ihi 1) ilo))
(cond
 ((or ilv (not wantsn))
  (setf icols (f2cl-lib:int-sub (f2cl-lib:int-add n 1) ilo)))
 (t (setf icols irows)))
(setf itau 1)
(setf iwrk (f2cl-lib:int-add itau irows))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
```

```
    (sgeqrf irows icols
     (f2cl-lib:array-slice b-%data% single-float (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf irows var-0)
  (setf icols var-1)
  (setf ldb$ var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (sormqr "L" "T" irows icols irows
     (f2cl-lib:array-slice b-%data% single-float (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice a-%data% single-float (ilo ilo)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf irows var-2))
  (when var-3 (setf icols var-3))
  (when var-4 (setf irows var-4))
  (when var-6 (setf ldb$ var-6))
  (when var-9 (setf lda var-9))
  (when var-12 (setf ierr var-12)))
(cond
 (ilvl
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "Full" n n zero one vl ldvl)
    (declare (ignore var-0 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldvl var-6)))
  (cond
   ((> irows 1)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slacpy "L" (f2cl-lib:int-sub irows 1)
```

```
            (f2cl-lib:int-sub irows 1)
            (f2cl-lib:array-slice b-%data% single-float ((+ ilo 1) ilo)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$
            (f2cl-lib:array-slice vl-%data% single-float ((+ ilo 1) ilo)
                                  ((1 ldvl) (1 *)) vl-%offset%)
            ldvl)
        (declare (ignore var-0 var-1 var-2 var-3 var-5))
        (when var-4 (setf ldb$ var-4))
        (when var-6 (setf ldvl var-6)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (sorgqr irows irows irows
         (f2cl-lib:array-slice vl-%data% single-float (ilo ilo)
                               ((1 ldvl) (1 *)) vl-%offset%)
         ldvl
         (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                               work-%offset%)
         (f2cl-lib:array-slice work-%data% single-float (iwrk) ((1 *))
                               work-%offset%)
         (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
      (declare (ignore var-3 var-5 var-6 var-7))
      (when var-0 (setf irows var-0))
      (when var-1 (setf irows var-1))
      (when var-2 (setf irows var-2))
      (when var-4 (setf ldvl var-4))
      (when var-8 (setf ierr var-8)))))
  (if ilvr
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slaset "Full" n n zero one vr ldvr)
        (declare (ignore var-0 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf zero var-3))
        (when var-4 (setf one var-4))
        (when var-6 (setf ldvr var-6))))
  (cond
   ((or ilv (not wantsn))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (sgghrd jobvl jobvr n ilo ihi a lda b ldb$ vl ldvl vr ldvr ierr)
      (declare (ignore var-5 var-7 var-9 var-11))
      (when var-0 (setf jobvl var-0))
      (when var-1 (setf jobvr var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf ilo var-3))
      (when var-4 (setf ihi var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf ldb$ var-8))
```

```
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-13 (setf ierr var-13))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (sgghrd "N" "N" irows 1 irows
       (f2cl-lib:array-slice a-%data% single-float (ilo ilo)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice b-%data% single-float (ilo ilo)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$ vl ldvl vr ldvr ierr)
    (declare (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11))
    (when var-2 (setf irows var-2))
    (when var-4 (setf irows var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-13 (setf ierr var-13)))))
(cond
 ((or ilv (not wantsn))
  (f2cl-lib:f2cl-set-string chtemp "S" (string 1)))
 (t (f2cl-lib:f2cl-set-string chtemp "E" (string 1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19)
    (shgeqz chtemp jobvl jobvr n ilo ihi a lda b ldb$ alphar alphai
     beta vl ldvl vr ldvr work lwork ierr)
  (declare
   (ignore var-6 var-8 var-10 var-11 var-12 var-13 var-15 var-17))
  (when var-0 (setf chtemp var-0))
  (when var-1 (setf jobvl var-1))
  (when var-2 (setf jobvr var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf ilo var-4))
  (when var-5 (setf ihi var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldb$ var-9))
  (when var-14 (setf ldvl var-14))
  (when var-16 (setf ldvr var-16))
  (when var-18 (setf lwork var-18))
  (when var-19 (setf ierr var-19)))
(cond
 ((/= ierr 0)
  (cond ((and (> ierr 0) (<= ierr n)) (setf info ierr))
        ((and (> ierr n) (<= ierr (f2cl-lib:int-mul 2 n)))
         (setf info (f2cl-lib:int-sub ierr n)))
```

```
        (t (setf info (f2cl-lib:int-add n 1))))
  (go label130)))
(cond
 ((or ilv (not wantsn))
  (cond
   (ilv
    (cond
     (ilvl
      (cond (ilvr (f2cl-lib:f2cl-set-string chtemp "B" (string 1)))
            (t (f2cl-lib:f2cl-set-string chtemp "L" (string 1)))))
     (t (f2cl-lib:f2cl-set-string chtemp "R" (string 1))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14 var-15)
        (stgevc chtemp "B" ldumma n a lda b ldb$ vl ldvl vr ldvr n in
         work ierr)
      (declare (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-14))
      (when var-0 (setf chtemp var-0))
      (when var-3 (setf n var-3))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldb$ var-7))
      (when var-9 (setf ldvl var-9))
      (when var-11 (setf ldvr var-11))
      (when var-12 (setf n var-12))
      (when var-13 (setf in var-13))
      (when var-15 (setf ierr var-15)))
    (cond
     ((/= ierr 0) (setf info (f2cl-lib:int-add n 2)) (go label130)))))
  (cond
   ((not wantsn) (setf pair f2cl-lib:%false%)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (cond (pair (setf pair f2cl-lib:%false%) (go label20)))
        (setf mm 1)
        (cond
         ((< i n)
          (cond
           ((/=
             (f2cl-lib:fref a ((f2cl-lib:int-add i 1) i)
                            ((1 lda) (1 *)))
             zero)
            (setf pair f2cl-lib:%true%) (setf mm 2)))))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf (f2cl-lib:fref bwork-%data% (j) ((1 *))
                                 bwork-%offset%)
                  f2cl-lib:%false%)
           label10))
```

```
(cond
 ((= mm 1)
  (setf (f2cl-lib:fref bwork-%data% (i) ((1 *)) bwork-%offset%)
          f2cl-lib:%true%))
 ((= mm 2)
  (setf (f2cl-lib:fref bwork-%data% (i) ((1 *)) bwork-%offset%)
          f2cl-lib:%true%)
  (setf (f2cl-lib:fref bwork-%data% ((f2cl-lib:int-add i 1))
                       ((1 *)) bwork-%offset%)
          f2cl-lib:%true%)))
(setf iwrk (f2cl-lib:int-add (f2cl-lib:int-mul mm n) 1))
(setf iwrk1 (f2cl-lib:int-add iwrk (f2cl-lib:int-mul mm n)))
(cond
 ((or wantse wantsb)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13 var-14 var-15)
      (stgevc "B" "S" bwork n a lda b ldb$
       (f2cl-lib:array-slice work-%data% single-float (1)
                             ((1 *)) work-%offset%)
       n
       (f2cl-lib:array-slice work-%data% single-float (iwrk)
                             ((1 *)) work-%offset%)
       n mm m
       (f2cl-lib:array-slice work-%data% single-float (iwrk1)
                             ((1 *)) work-%offset%)
       ierr)
    (declare
     (ignore var-0 var-1 var-2 var-4 var-6 var-8 var-10
      var-14))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldb$ var-7))
    (when var-9 (setf n var-9))
    (when var-11 (setf n var-11))
    (when var-12 (setf mm var-12))
    (when var-13 (setf m var-13))
    (when var-15 (setf ierr var-15)))
  (cond
   ((/= ierr 0) (setf info (f2cl-lib:int-add n 2))
    (go label130)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17 var-18 var-19)
    (stgsna sense "S" bwork n a lda b ldb$
     (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                           work-%offset%)
     n
     (f2cl-lib:array-slice work-%data% single-float (iwrk)
```

```
                                  ((1 *)) work-%offset%)
                n
                (f2cl-lib:array-slice rconde-%data% single-float (i)
                                  ((1 *)) rconde-%offset%)
                (f2cl-lib:array-slice rcondv-%data% single-float (i)
                                  ((1 *)) rcondv-%offset%)
                mm m
                (f2cl-lib:array-slice work-%data% single-float (iwrk1)
                                  ((1 *)) work-%offset%)
                (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk1) 1) iwork
                ierr)
              (declare
               (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-12 var-13
                var-16 var-17 var-18))
              (when var-0 (setf sense var-0))
              (when var-3 (setf n var-3))
              (when var-5 (setf lda var-5))
              (when var-7 (setf ldb$ var-7))
              (when var-9 (setf n var-9))
              (when var-11 (setf n var-11))
              (when var-14 (setf mm var-14))
              (when var-15 (setf m var-15))
              (when var-19 (setf ierr var-19)))
          label20))))))
  (cond
   (ilvl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (sggbak balanc "L" n ilo ihi lscale rscale n vl ldvl ierr)
      (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-8))
      (setf balanc var-0)
      (setf n var-7)
      (setf ldvl var-9)
      (setf ierr var-10))
    (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                  ((> jc n) nil)
      (tagbody
        (if (< (f2cl-lib:fref alphai-%data% (jc) ((1 *)) alphai-%offset%)
               zero)
              (go label70))
        (setf temp zero)
        (cond
         ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                        ((> jr n) nil)
              (tagbody
                (setf temp
                        (max temp
                             (abs
```

```
                    (f2cl-lib:fref vl-%data% (jr jc)
                                   ((1 ldvl) (1 *))
                                   vl-%offset%))))
     label30)))
 (t
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
      (setf temp
              (max temp
                   (+
                    (abs
                     (f2cl-lib:fref vl-%data% (jr jc)
                                    ((1 ldvl) (1 *))
                                    vl-%offset%))
                    (abs
                     (f2cl-lib:fref vl-%data%
                                    (jr (f2cl-lib:int-add jc 1))
                                    ((1 ldvl) (1 *))
                                    vl-%offset%)))))
     label40))))
(if (< temp smlnum)
    (go label70))
(setf temp (/ one temp))
(cond
 ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
      (setf (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                           vl-%offset%)
              (*
               (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                              vl-%offset%)
               temp))
     label50)))
 (t
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
      (setf (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                           vl-%offset%)
              (*
               (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                              vl-%offset%)
               temp))
      (setf (f2cl-lib:fref vl-%data% (jr (f2cl-lib:int-add jc 1))
                           ((1 ldvl) (1 *)) vl-%offset%)
              (*
               (f2cl-lib:fref vl-%data%
```

```
                                        (jr (f2cl-lib:int-add jc 1))
                                        ((1 ldvl) (1 *)) vl-%offset%)
                        temp))
              label60))))
         label70))))
    (cond
     (ilvr
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (sggbak balanc "R" n ilo ihi lscale rscale n vr ldvr ierr)
        (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-8))
        (setf balanc var-0)
        (setf n var-7)
        (setf ldvr var-9)
        (setf ierr var-10))
      (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                    ((> jc n) nil)
        (tagbody
          (if (< (f2cl-lib:fref alphai-%data% (jc) ((1 *)) alphai-%offset%)
                 zero)
              (go label120))
          (setf temp zero)
          (cond
           ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
            (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                          ((> jr n) nil)
              (tagbody
                (setf temp
                        (max temp
                             (abs
                              (f2cl-lib:fref vr-%data% (jr jc)
                                             ((1 ldvr) (1 *))
                                             vr-%offset%))))
              label80)))
           (t
            (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                          ((> jr n) nil)
              (tagbody
                (setf temp
                        (max temp
                             (+
                              (abs
                               (f2cl-lib:fref vr-%data% (jr jc)
                                              ((1 ldvr) (1 *))
                                              vr-%offset%))
                              (abs
                               (f2cl-lib:fref vr-%data%
                                              (jr (f2cl-lib:int-add jc 1))
                                              ((1 ldvr) (1 *))
```

```
                                                vr-%offset%)))))
                 label90))))
            (if (< temp smlnum)
                (go label120))
            (setf temp (/ one temp))
            (cond
             ((= (f2cl-lib:fref alphai (jc) ((1 *))) zero)
              (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                            ((> jr n) nil)
                (tagbody
                  (setf (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                       vr-%offset%)
                        (*
                         (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                        vr-%offset%)
                         temp))
                 label100)))
             (t
              (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                            ((> jr n) nil)
                (tagbody
                  (setf (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                       vr-%offset%)
                        (*
                         (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                        vr-%offset%)
                         temp))
                  (setf (f2cl-lib:fref vr-%data% (jr (f2cl-lib:int-add jc 1))
                                       ((1 ldvr) (1 *)) vr-%offset%)
                        (*
                         (f2cl-lib:fref vr-%data%
                                        (jr (f2cl-lib:int-add jc 1))
                                        ((1 ldvr) (1 *)) vr-%offset%)
                         temp))
                 label110))))
          label120))))
      label130
       (cond
        (ilascl
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
             (slascl "G" 0 0 anrmto anrm n 1 alphar n ierr)
           (declare (ignore var-0 var-1 var-2 var-6 var-7))
           (when var-3 (setf anrmto var-3))
           (when var-4 (setf anrm var-4))
           (when var-5 (setf n var-5))
           (when var-8 (setf n var-8))
           (when var-9 (setf ierr var-9)))
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
```

```
              (slascl "G" 0 0 anrmto anrm n 1 alphai n ierr)
            (declare (ignore var-0 var-1 var-2 var-6 var-7))
            (when var-3 (setf anrmto var-3))
            (when var-4 (setf anrm var-4))
            (when var-5 (setf n var-5))
            (when var-8 (setf n var-8))
            (when var-9 (setf ierr var-9)))))
        (cond
         (ilbscl
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (slascl "G" 0 0 bnrmto bnrm n 1 beta n ierr)
            (declare (ignore var-0 var-1 var-2 var-6 var-7))
            (when var-3 (setf bnrmto var-3))
            (when var-4 (setf bnrm var-4))
            (when var-5 (setf n var-5))
            (when var-8 (setf n var-8))
            (when var-9 (setf ierr var-9)))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 maxwrk) 'single-float))
        (go end_label)
       end_label
        (return
         (values balanc jobvl jobvr sense n nil lda nil ldb$ nil nil nil nil
                 ldvl nil ldvr ilo ihi nil nil abnrm bbnrm nil nil nil lwork
                 nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sggevx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
```

```
  (array single-float (*))
  (single-float) (single-float)
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (array fortran-to-lisp::logical
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::balanc
  fortran-to-lisp::jobvl
  fortran-to-lisp::jobvr
  fortran-to-lisp::sense
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$ nil nil nil
  nil fortran-to-lisp::ldvl nil
  fortran-to-lisp::ldvr
  fortran-to-lisp::ilo
  fortran-to-lisp::ihi nil nil
  fortran-to-lisp::abnrm
  fortran-to-lisp::bbnrm nil nil
  nil fortran-to-lisp::lwork nil
  nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::sggbak
  fortran-to-lisp::sgeqrf
  fortran-to-lisp::sggbal
  fortran-to-lisp::ilaenv))))
```

———-

Chapter 26

LAPACK - General Matrices, Eigenvalue , Complex

26.1 cgees

26.1.1 Axiom unit tests

— cgees.input —

```
)set break resume
)sys rm -f cgees.output
)spool cgees.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

26.1.2 Axiom help page

— cgees.help —

```
========================================================================
cgees examples
========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgees (JOBVS, SORT, SELECT, N, A, LDA, SDIM, W, VS, LDVS,
           WORK, LWORK, RWORK, BWORK, INFO)
            CGEES computes the eigenvalues, the Schur form, and, optionally,
           the matrix of Schur vectors for GE matrices

Function/Subroutine Documentation
   subroutine cgees (character JOBVS, character SORT, external SELECT, integer
       N, complex, dimension( lda, * ) A, integer LDA, integer SDIM, complex,
       dimension( * ) W, complex, dimension( ldvs, * ) VS, integer LDVS,
       complex, dimension( * ) WORK, integer LWORK, real, dimension( * )
       RWORK, logical, dimension( * ) BWORK, integer INFO)
        CGEES computes the eigenvalues, the Schur form, and, optionally, the
       matrix of Schur vectors for GE matrices

Purpose:
   CGEES computes for an N-by-N complex nonsymmetric matrix A, the
   eigenvalues, the Schur form T, and, optionally, the matrix of Schur
   vectors Z.  This gives the Schur factorization A = Z*T*(Z**H).

   Optionally, it also orders the eigenvalues on the diagonal of the
   Schur form so that selected eigenvalues are at the top left.
   The leading columns of Z then form an orthonormal basis for the
   invariant subspace corresponding to the selected eigenvalues.

   A complex matrix is in Schur form if it is upper triangular.

Parameters:

 JOBVS
   JOBVS is CHARACTER*1
   = 'N': Schur vectors are not computed;
   = 'V': Schur vectors are computed.

 SORT
   SORT is CHARACTER*1
   Specifies whether or not to order the eigenvalues on the
   diagonal of the Schur form.
   = 'N': Eigenvalues are not ordered:
   = 'S': Eigenvalues are ordered (see SELECT).

 SELECT
   SELECT is a LOGICAL FUNCTION of one COMPLEX argument
   SELECT must be declared EXTERNAL in the calling subroutine.
```

```
  If SORT = 'S', SELECT is used to select eigenvalues to order
  to the top left of the Schur form.
  IF SORT = 'N', SELECT is not referenced.
  The eigenvalue W(j) is selected if SELECT(W(j)) is true.

N
  N is INTEGER
  The order of the matrix A. N >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the N-by-N matrix A.
  On exit, A has been overwritten by its Schur form T.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

SDIM
  SDIM is INTEGER
  If SORT = 'N', SDIM = 0.
  If SORT = 'S', SDIM = number of eigenvalues for which
                 SELECT is true.

W
  W is COMPLEX array, dimension (N)
  W contains the computed eigenvalues, in the same order that
  they appear on the diagonal of the output Schur form T.

VS
  VS is COMPLEX array, dimension (LDVS,N)
  If JOBVS = 'V', VS contains the unitary matrix Z of Schur vectors.
  If JOBVS = 'N', VS is not referenced.

LDVS
  LDVS is INTEGER
  The leading dimension of the array VS.  LDVS >= 1; if
  JOBVS = 'V', LDVS >= N.

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,2*N).
  For good performance, LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
```

```
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is REAL array, dimension (N)

BWORK
  BWORK is LOGICAL array, dimension (N)
  Not referenced if SORT = 'N'.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value.
  > 0: if INFO = i, and i is
       <= N:  the QR algorithm failed to compute all the
              eigenvalues; elements 1:ILO-1 and i+1:N of W
              contain those eigenvalues which have converged;
              if JOBVS = 'V', VS contains the matrix which
              reduces A to its partially converged Schur form.
       = N+1: the eigenvalues could not be reordered because
              some eigenvalues were too close to separate (the
              problem is very ill-conditioned);
       = N+2: after reordering, roundoff changed values of
              some complex eigenvalues so that leading
              eigenvalues in the Schur form no longer satisfy
              SELECT = .TRUE..  This could also be caused by
              underflow due to scaling.
```

———-

26.1.3 fortran code

— **cgees.f** —

```
*  ===================================================================
      SUBROUTINE CGEES( JOBVS, SORT, SELECT, N, A, LDA, SDIM, W, VS,
     $                  LDVS, WORK, LWORK, RWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVS, SORT
      INTEGER            INFO, LDA, LDVS, LWORK, N, SDIM
```

```
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      REAL               RWORK( * )
      COMPLEX            A( LDA, * ), VS( LDVS, * ), W( * ), WORK( * )
*     ..
*     .. Function Arguments ..
      LOGICAL            SELECT
      EXTERNAL           SELECT
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, SCALEA, WANTST, WANTVS
      INTEGER            HSWORK, I, IBAL, ICOND, IERR, IEVAL, IHI, ILO,
     $                   ITAU, IWRK, MAXWRK, MINWRK
      REAL               ANRM, BIGNUM, CSCALE, EPS, S, SEP, SMLNUM
*     ..
*     .. Local Arrays ..
      REAL               DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           CCOPY, CGEBAK, CGEBAL, CGEHRD, CHSEQR, CLACPY,
     $                   CLASCL, CTRSEN, CUNGHR, SLABAD, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               CLANGE, SLAMCH
      EXTERNAL           LSAME, ILAENV, CLANGE, SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      WANTVS = LSAME( JOBVS, 'V' )
      WANTST = LSAME( SORT, 'S' )
      IF( ( .NOT.WANTVS ) .AND. ( .NOT.LSAME( JOBVS, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
```

```
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDVS.LT.1 .OR. ( WANTVS .AND. LDVS.LT.N ) ) THEN
         INFO = -10
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       CWorkspace refers to complex workspace, and RWorkspace to real
*       workspace. NB refers to the optimal block size for the
*       immediately following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by CHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            MAXWRK = 1
         ELSE
            MAXWRK = N + N*ILAENV( 1, 'CGEHRD', ' ', N, 1, N, 0 )
            MINWRK = 2*N
*
            CALL CHSEQR( 'S', JOBVS, N, 1, N, A, LDA, W, VS, LDVS,
     $             WORK, -1, IEVAL )
            HSWORK = WORK( 1 )
*
            IF( .NOT.WANTVS ) THEN
               MAXWRK = MAX( MAXWRK, HSWORK )
            ELSE
               MAXWRK = MAX( MAXWRK, N + ( N - 1 )*ILAENV( 1, 'CUNGHR',
     $                       ' ', N, 1, N, -1 ) )
               MAXWRK = MAX( MAXWRK, HSWORK )
            END IF
         END IF
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -12
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEES ', -INFO )
         RETURN
```

```
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = CLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
     $   CALL CLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*     Permute the matrix to make it more nearly triangular
*     (CWorkspace: none)
*     (RWorkspace: need N)
*
      IBAL = 1
      CALL CGEBAL( 'P', N, A, LDA, ILO, IHI, RWORK( IBAL ), IERR )
*
*     Reduce to upper Hessenberg form
*     (CWorkspace: need 2*N, prefer N+N*NB)
*     (RWorkspace: none)
*
      ITAU = 1
      IWRK = N + ITAU
      CALL CGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
```

```
      IF( WANTVS ) THEN
*
*        Copy Householder vectors to VS
*
         CALL CLACPY( 'L', N, N, A, LDA, VS, LDVS )
*
*        Generate unitary matrix in VS
*        (CWorkspace: need 2*N-1, prefer N+(N-1)*NB)
*        (RWorkspace: none)
*
         CALL CUNGHR( N, ILO, IHI, VS, LDVS, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
      END IF
*
      SDIM = 0
*
*     Perform QR iteration, accumulating Schur vectors in VS if desired
*     (CWorkspace: need 1, prefer HSWORK (see comments) )
*     (RWorkspace: none)
*
      IWRK = ITAU
      CALL CHSEQR( 'S', JOBVS, N, ILO, IHI, A, LDA, W, VS, LDVS,
     $             WORK( IWRK ), LWORK-IWRK+1, IEVAL )
      IF( IEVAL.GT.0 )
     $   INFO = IEVAL
*
*     Sort eigenvalues if desired
*
      IF( WANTST .AND. INFO.EQ.0 ) THEN
         IF( SCALEA )
     $      CALL CLASCL( 'G', 0, 0, CSCALE, ANRM, N, 1, W, N, IERR )
         DO 10 I = 1, N
            BWORK( I ) = SELECT( W( I ) )
   10    CONTINUE
*
*        Reorder eigenvalues and transform Schur vectors
*        (CWorkspace: none)
*        (RWorkspace: none)
*
         CALL CTRSEN( 'N', JOBVS, BWORK, N, A, LDA, VS, LDVS, W, SDIM,
     $                S, SEP, WORK( IWRK ), LWORK-IWRK+1, ICOND )
      END IF
*
      IF( WANTVS ) THEN
*
*        Undo balancing
*        (CWorkspace: none)
*        (RWorkspace: need N)
*
         CALL CGEBAK( 'P', 'R', N, ILO, IHI, RWORK( IBAL ), N, VS, LDVS,
```

```
     $                IERR )
      END IF
*
      IF( SCALEA ) THEN
*
*        Undo scaling for the Schur form of A
*
         CALL CLASCL( 'U', 0, 0, CSCALE, ANRM, N, N, A, LDA, IERR )
         CALL CCOPY( N, A, LDA+1, W, 1 )
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of CGEES
*
      END
```

———-

26.1.4 lisp code

— **cgees.lisp** —

———-

26.2 cgeesx

26.2.1 Axiom unit tests

— **cgeesx.input** —

```
)set break resume
)sys rm -f cgeesx.output
)spool cgeesx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

26.2.2 Axiom help page

— cgeesx.help —

```
====================================================================
cgeesx examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgeesx (JOBVS, SORT, SELECT, SENSE, N, A, LDA, SDIM, W, VS,
           LDVS, RCONDE, RCONDV, WORK, LWORK, RWORK, BWORK, INFO)
            CGEESX computes the eigenvalues, the Schur form, and, optionally,
           the matrix of Schur vectors for GE matrices

Function/Subroutine Documentation
   subroutine cgeesx (character JOBVS, character SORT, external SELECT,
       character SENSE, integer N, complex, dimension( lda, * ) A, integer
       LDA, integer SDIM, complex, dimension( * ) W, complex, dimension( ldvs,
       * ) VS, integer LDVS, real RCONDE, real RCONDV, complex, dimension( * )
       WORK, integer LWORK, real, dimension( * ) RWORK, logical, dimension( *
       ) BWORK, integer INFO)
        CGEESX computes the eigenvalues, the Schur form, and, optionally, the
       matrix of Schur vectors for GE matrices

Purpose:
   CGEESX computes for an N-by-N complex nonsymmetric matrix A, the
   eigenvalues, the Schur form T, and, optionally, the matrix of Schur
   vectors Z.  This gives the Schur factorization A = Z*T*(Z**H).

   Optionally, it also orders the eigenvalues on the diagonal of the
   Schur form so that selected eigenvalues are at the top left;
   computes a reciprocal condition number for the average of the
   selected eigenvalues (RCONDE); and computes a reciprocal condition
   number for the right invariant subspace corresponding to the
   selected eigenvalues (RCONDV).  The leading columns of Z form an
   orthonormal basis for this invariant subspace.

   For further explanation of the reciprocal condition numbers RCONDE
   and RCONDV, see Section 4.10 of the LAPACK Users' Guide (where
   these quantities are called s and sep respectively).

   A complex matrix is in Schur form if it is upper triangular.

Parameters:
```

```
JOBVS
  JOBVS is CHARACTER*1
  = 'N': Schur vectors are not computed;
  = 'V': Schur vectors are computed.

SORT
  SORT is CHARACTER*1
  Specifies whether or not to order the eigenvalues on the
  diagonal of the Schur form.
  = 'N': Eigenvalues are not ordered;
  = 'S': Eigenvalues are ordered (see SELECT).

SELECT
  SELECT is procedure) LOGICAL FUNCTION of one COMPLEX argument
  SELECT must be declared EXTERNAL in the calling subroutine.
  If SORT = 'S', SELECT is used to select eigenvalues to order
  to the top left of the Schur form.
  If SORT = 'N', SELECT is not referenced.
  An eigenvalue W(j) is selected if SELECT(W(j)) is true.

SENSE
  SENSE is CHARACTER*1
  Determines which reciprocal condition numbers are computed.
  = 'N': None are computed;
  = 'E': Computed for average of selected eigenvalues only;
  = 'V': Computed for selected right invariant subspace only;
  = 'B': Computed for both.
  If SENSE = 'E', 'V' or 'B', SORT must equal 'S'.

N
  N is INTEGER
  The order of the matrix A. N >= 0.

A
  A is COMPLEX array, dimension (LDA, N)
  On entry, the N-by-N matrix A.
  On exit, A is overwritten by its Schur form T.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

SDIM
  SDIM is INTEGER
  If SORT = 'N', SDIM = 0.
  If SORT = 'S', SDIM = number of eigenvalues for which SELECT is true.

W
  W is COMPLEX array, dimension (N)
```

```
  W contains the computed eigenvalues, in the same order
  that they appear on the diagonal of the output Schur form T.

VS
  VS is COMPLEX array, dimension (LDVS,N)
  If JOBVS = 'V', VS contains the unitary matrix Z of Schur vectors.
  If JOBVS = 'N', VS is not referenced.

LDVS
  LDVS is INTEGER
  The leading dimension of the array VS.  LDVS >= 1, and if
  JOBVS = 'V', LDVS >= N.

RCONDE
  RCONDE is REAL
  If SENSE = 'E' or 'B', RCONDE contains the reciprocal
  condition number for the average of the selected eigenvalues.
  Not referenced if SENSE = 'N' or 'V'.

RCONDV
  RCONDV is REAL
  If SENSE = 'V' or 'B', RCONDV contains the reciprocal
  condition number for the selected right invariant subspace.
  Not referenced if SENSE = 'N' or 'E'.

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,2*N).
  Also, if SENSE = 'E' or 'V' or 'B', LWORK >= 2*SDIM*(N-SDIM),
  where SDIM is the number of selected eigenvalues computed by
  this routine.  Note that 2*SDIM*(N-SDIM) <= N*N/2. Note also
  that an error is only returned if LWORK < max(1,2*N), but if
  SENSE = 'E' or 'V' or 'B' this may not be large enough.
  For good performance, LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates upper bound on the optimal size of the
  array WORK, returns this value as the first entry of the WORK
  array, and no error message related to LWORK is issued by XERBLA.

RWORK
  RWORK is REAL array, dimension (N)

BWORK
  BWORK is LOGICAL array, dimension (N)
  Not referenced if SORT = 'N'.
```

```
 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value.
   > 0: if INFO = i, and i is
        <= N: the QR algorithm failed to compute all the
              eigenvalues; elements 1:ILO-1 and i+1:N of W
              contain those eigenvalues which have converged; if
              JOBVS = 'V', VS contains the transformation which
              reduces A to its partially converged Schur form.
        = N+1: the eigenvalues could not be reordered because some
               eigenvalues were too close to separate (the problem
               is very ill-conditioned);
        = N+2: after reordering, roundoff changed values of some
               complex eigenvalues so that leading eigenvalues in
               the Schur form no longer satisfy SELECT=.TRUE.  This
               could also be caused by underflow due to scaling.
```

26.2.3 fortran code

— cgeesx.f —

```
*  ===================================================================
      SUBROUTINE CGEESX( JOBVS, SORT, SELECT, SENSE, N, A, LDA, SDIM, W,
     $                   VS, LDVS, RCONDE, RCONDV, WORK, LWORK, RWORK,
     $                   BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVS, SENSE, SORT
      INTEGER            INFO, LDA, LDVS, LWORK, N, SDIM
      REAL               RCONDE, RCONDV
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      REAL               RWORK( * )
      COMPLEX            A( LDA, * ), VS( LDVS, * ), W( * ), WORK( * )
*     ..
*     .. Function Arguments ..
      LOGICAL            SELECT
```

```
      EXTERNAL           SELECT
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, SCALEA, WANTSB, WANTSE, WANTSN, WANTST,
     $                   WANTSV, WANTVS
      INTEGER            HSWORK, I, IBAL, ICOND, IERR, IEVAL, IHI, ILO,
     $                   ITAU, IWRK, LWRK, MAXWRK, MINWRK
      REAL               ANRM, BIGNUM, CSCALE, EPS, SMLNUM
*     ..
*     .. Local Arrays ..
      REAL               DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           CCOPY, CGEBAK, CGEBAL, CGEHRD, CHSEQR, CLACPY,
     $                   CLASCL, CTRSEN, CUNGHR, SLABAD, SLASCL, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               CLANGE, SLAMCH
      EXTERNAL           LSAME, ILAENV, CLANGE, SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      WANTVS = LSAME( JOBVS, 'V' )
      WANTST = LSAME( SORT, 'S' )
      WANTSN = LSAME( SENSE, 'N' )
      WANTSE = LSAME( SENSE, 'E' )
      WANTSV = LSAME( SENSE, 'V' )
      WANTSB = LSAME( SENSE, 'B' )
      LQUERY = ( LWORK.EQ.-1 )
*
      IF( ( .NOT.WANTVS ) .AND. ( .NOT.LSAME( JOBVS, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( .NOT.( WANTSN .OR. WANTSE .OR. WANTSV .OR. WANTSB ) .OR.
```

```
     $         ( .NOT.WANTST .AND. .NOT.WANTSN ) ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDVS.LT.1 .OR. ( WANTVS .AND. LDVS.LT.N ) ) THEN
         INFO = -11
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of real workspace needed at that point in the
*       code, as well as the preferred amount for good performance.
*       CWorkspace refers to complex workspace, and RWorkspace to real
*       workspace. NB refers to the optimal block size for the
*       immediately following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by CHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.
*       If SENSE = 'E', 'V' or 'B', then the amount of workspace needed
*       depends on SDIM, which is computed by the routine CTRSEN later
*       in the code.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            LWRK = 1
         ELSE
            MAXWRK = N + N*ILAENV( 1, 'CGEHRD', ' ', N, 1, N, 0 )
            MINWRK = 2*N
*
            CALL CHSEQR( 'S', JOBVS, N, 1, N, A, LDA, W, VS, LDVS,
     $              WORK, -1, IEVAL )
            HSWORK = WORK( 1 )
*
            IF( .NOT.WANTVS ) THEN
               MAXWRK = MAX( MAXWRK, HSWORK )
            ELSE
               MAXWRK = MAX( MAXWRK, N + ( N - 1 )*ILAENV( 1, 'CUNGHR',
     $                        ' ', N, 1, N, -1 ) )
               MAXWRK = MAX( MAXWRK, HSWORK )
            END IF
            LWRK = MAXWRK
            IF( .NOT.WANTSN )
     $         LWRK = MAX( LWRK, ( N*N )/2 )
         END IF
         WORK( 1 ) = LWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
```

```
            INFO = -15
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEESX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = CLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
     $   CALL CLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*
*     Permute the matrix to make it more nearly triangular
*     (CWorkspace: none)
*     (RWorkspace: need N)
*
      IBAL = 1
      CALL CGEBAL( 'P', N, A, LDA, ILO, IHI, RWORK( IBAL ), IERR )
*
*     Reduce to upper Hessenberg form
```

```
*     (CWorkspace: need 2*N, prefer N+N*NB)
*     (RWorkspace: none)
*
      ITAU = 1
      IWRK = N + ITAU
      CALL CGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
      IF( WANTVS ) THEN
*
*        Copy Householder vectors to VS
*
         CALL CLACPY( 'L', N, N, A, LDA, VS, LDVS )
*
*        Generate unitary matrix in VS
*        (CWorkspace: need 2*N-1, prefer N+(N-1)*NB)
*        (RWorkspace: none)
*
         CALL CUNGHR( N, ILO, IHI, VS, LDVS, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
      END IF
*
      SDIM = 0
*
*     Perform QR iteration, accumulating Schur vectors in VS if desired
*     (CWorkspace: need 1, prefer HSWORK (see comments) )
*     (RWorkspace: none)
*
      IWRK = ITAU
      CALL CHSEQR( 'S', JOBVS, N, ILO, IHI, A, LDA, W, VS, LDVS,
     $             WORK( IWRK ), LWORK-IWRK+1, IEVAL )
      IF( IEVAL.GT.0 )
     $   INFO = IEVAL
*
*     Sort eigenvalues if desired
*
      IF( WANTST .AND. INFO.EQ.0 ) THEN
         IF( SCALEA )
     $      CALL CLASCL( 'G', 0, 0, CSCALE, ANRM, N, 1, W, N, IERR )
         DO 10 I = 1, N
            BWORK( I ) = SELECT( W( I ) )
   10    CONTINUE
*
*        Reorder eigenvalues, transform Schur vectors, and compute
*        reciprocal condition numbers
*        (CWorkspace: if SENSE is not 'N', need 2*SDIM*(N-SDIM)
*                     otherwise, need none )
*        (RWorkspace: none)
*
         CALL CTRSEN( SENSE, JOBVS, BWORK, N, A, LDA, VS, LDVS, W, SDIM,
```

```
     $                RCONDE, RCONDV, WORK( IWRK ), LWORK-IWRK+1,
     $                ICOND )
         IF( .NOT.WANTSN )
     $      MAXWRK = MAX( MAXWRK, 2*SDIM*( N-SDIM ) )
         IF( ICOND.EQ.-14 ) THEN
*
*           Not enough complex workspace
*
            INFO = -15
         END IF
      END IF
*
      IF( WANTVS ) THEN
*
*        Undo balancing
*        (CWorkspace: none)
*        (RWorkspace: need N)
*
         CALL CGEBAK( 'P', 'R', N, ILO, IHI, RWORK( IBAL ), N, VS, LDVS,
     $                IERR )
      END IF
*
      IF( SCALEA ) THEN
*
*        Undo scaling for the Schur form of A
*
         CALL CLASCL( 'U', 0, 0, CSCALE, ANRM, N, N, A, LDA, IERR )
         CALL CCOPY( N, A, LDA+1, W, 1 )
         IF( ( WANTSV .OR. WANTSB ) .AND. INFO.EQ.0 ) THEN
            DUM( 1 ) = RCONDV
            CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, 1, 1, DUM, 1, IERR )
            RCONDV = DUM( 1 )
         END IF
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of CGEESX
*
      END
```

———-

26.2.4 lisp code

— cgeesx.lisp —

———-

26.3 cgeev

26.3.1 Axiom unit tests

— **cgeev.input** —

```
)set break resume
)sys rm -f cgeev.output
)spool cgeev.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

26.3.2 Axiom help page

— **cgeev.help** —

```
======================================================================
cgeev examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgeev (JOBVL, JOBVR, N, A, LDA, W, VL, LDVL, VR, LDVR, WORK,
           LWORK, RWORK, INFO)
            CGEEV computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for GE matrices

Function/Subroutine Documentation
   subroutine cgeev (character JOBVL, character JOBVR, integer N, complex,
       dimension( lda, * ) A, integer LDA, complex, dimension( * ) W, complex,
```

```
      dimension( ldvl, * ) VL, integer LDVL, complex, dimension( ldvr, * )
      VR, integer LDVR, complex, dimension( * ) WORK, integer LWORK, real,
      dimension( * ) RWORK, integer INFO)
       CGEEV computes the eigenvalues and, optionally, the left and/or right
      eigenvectors for GE matrices

Purpose:
   CGEEV computes for an N-by-N complex nonsymmetric matrix A, the
   eigenvalues and, optionally, the left and/or right eigenvectors.

   The right eigenvector v(j) of A satisfies
      A * v(j) = lambda(j) * v(j)
   where lambda(j) is its eigenvalue.

   The left eigenvector u(j) of A satisfies
      u(j)**H * A = lambda(j) * u(j)**H
   where u(j)**H denotes the conjugate transpose of u(j).

   The computed eigenvectors are normalized to have Euclidean norm
   equal to 1 and largest component real.

Parameters:

 JOBVL
   JOBVL is CHARACTER*1
   = 'N': left eigenvectors of A are not computed;
   = 'V': left eigenvectors of are computed.

 JOBVR
   JOBVR is CHARACTER*1
   = 'N': right eigenvectors of A are not computed;
   = 'V': right eigenvectors of A are computed.

 N
   N is INTEGER
   The order of the matrix A. N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the N-by-N matrix A.
   On exit, A has been overwritten.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 W
   W is COMPLEX array, dimension (N)
   W contains the computed eigenvalues.
```

```
VL
  VL is COMPLEX array, dimension (LDVL,N)
  If JOBVL = 'V', the left eigenvectors u(j) are stored one
  after another in the columns of VL, in the same order
  as their eigenvalues.
  If JOBVL = 'N', VL is not referenced.
  u(j) = VL(:,j), the j-th column of VL.

LDVL
  LDVL is INTEGER
  The leading dimension of the array VL.  LDVL >= 1; if
  JOBVL = 'V', LDVL >= N.

VR
  VR is COMPLEX array, dimension (LDVR,N)
  If JOBVR = 'V', the right eigenvectors v(j) are stored one
  after another in the columns of VR, in the same order
  as their eigenvalues.
  If JOBVR = 'N', VR is not referenced.
  v(j) = VR(:,j), the j-th column of VR.

LDVR
  LDVR is INTEGER
  The leading dimension of the array VR.  LDVR >= 1; if
  JOBVR = 'V', LDVR >= N.

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,2*N).
  For good performance, LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is REAL array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if INFO = i, the QR algorithm failed to compute all the
        eigenvalues, and no eigenvectors have been computed;
        elements and i+1:N of W contain eigenvalues which have converged.
```

———-

26.3.3 fortran code

— **cgeev.f** —

```
*  ======================================================================
      SUBROUTINE CGEEV( JOBVL, JOBVR, N, A, LDA, W, VL, LDVL, VR, LDVR,
     $                  WORK, LWORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVL, JOBVR
      INTEGER            INFO, LDA, LDVL, LDVR, LWORK, N
*     ..
*     .. Array Arguments ..
      REAL               RWORK( * )
      COMPLEX            A( LDA, * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   W( * ), WORK( * )
*     ..
*
*  ======================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, SCALEA, WANTVL, WANTVR
      CHARACTER          SIDE
      INTEGER            HSWORK, I, IBAL, IERR, IHI, ILO, IRWORK, ITAU,
     $                   IWRK, K, MAXWRK, MINWRK, NOUT
      REAL               ANRM, BIGNUM, CSCALE, EPS, SCL, SMLNUM
      COMPLEX            TMP
*     ..
*     .. Local Arrays ..
      LOGICAL            SELECT( 1 )
      REAL               DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEBAK, CGEBAL, CGEHRD, CHSEQR, CLACPY, CLASCL,
     $                   CSCAL, CSSCAL, CTREVC, CUNGHR, SLABAD, XERBLA
```

```
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV, ISAMAX
      REAL               CLANGE, SCNRM2, SLAMCH
      EXTERNAL           LSAME, ILAENV, ISAMAX, CLANGE, SCNRM2, SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          AIMAG, CMPLX, CONJG, MAX, REAL, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      WANTVL = LSAME( JOBVL, 'V' )
      WANTVR = LSAME( JOBVR, 'V' )
      IF( ( .NOT.WANTVL ) .AND. ( .NOT.LSAME( JOBVL, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTVR ) .AND. ( .NOT.LSAME( JOBVR, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDVL.LT.1 .OR. ( WANTVL .AND. LDVL.LT.N ) ) THEN
         INFO = -8
      ELSE IF( LDVR.LT.1 .OR. ( WANTVR .AND. LDVR.LT.N ) ) THEN
         INFO = -10
      END IF

*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       CWorkspace refers to complex workspace, and RWorkspace to real
*       workspace. NB refers to the optimal block size for the
*       immediately following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by CHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            MAXWRK = 1
         ELSE
            MAXWRK = N + N*ILAENV( 1, 'CGEHRD', ' ', N, 1, N, 0 )
```

```
            MINWRK = 2*N
            IF( WANTVL ) THEN
               MAXWRK = MAX( MAXWRK, N + ( N - 1 )*ILAENV( 1, 'CUNGHR',
     $                       ' ', N, 1, N, -1 ) )
               CALL CHSEQR( 'S', 'V', N, 1, N, A, LDA, W, VL, LDVL,
     $                 WORK, -1, INFO )
            ELSE IF( WANTVR ) THEN
               MAXWRK = MAX( MAXWRK, N + ( N - 1 )*ILAENV( 1, 'CUNGHR',
     $                       ' ', N, 1, N, -1 ) )
               CALL CHSEQR( 'S', 'V', N, 1, N, A, LDA, W, VR, LDVR,
     $                 WORK, -1, INFO )
            ELSE
               CALL CHSEQR( 'E', 'N', N, 1, N, A, LDA, W, VR, LDVR,
     $                 WORK, -1, INFO )
            END IF
            HSWORK = WORK( 1 )
            MAXWRK = MAX( MAXWRK, HSWORK, MINWRK )
         END IF
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -12
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEEV ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = CLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
```

```
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
     $   CALL CLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*     Balance the matrix
*     (CWorkspace: none)
*     (RWorkspace: need N)
*
      IBAL = 1
      CALL CGEBAL( 'B', N, A, LDA, ILO, IHI, RWORK( IBAL ), IERR )
*
*     Reduce to upper Hessenberg form
*     (CWorkspace: need 2*N, prefer N+N*NB)
*     (RWorkspace: none)
*
      ITAU = 1
      IWRK = ITAU + N
      CALL CGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
      IF( WANTVL ) THEN
*
*        Want left eigenvectors
*        Copy Householder vectors to VL
*
         SIDE = 'L'
         CALL CLACPY( 'L', N, N, A, LDA, VL, LDVL )
*
*        Generate unitary matrix in VL
*        (CWorkspace: need 2*N-1, prefer N+(N-1)*NB)
*        (RWorkspace: none)
*
         CALL CUNGHR( N, ILO, IHI, VL, LDVL, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VL
*        (CWorkspace: need 1, prefer HSWORK (see comments) )
*        (RWorkspace: none)
*
         IWRK = ITAU
         CALL CHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, W, VL, LDVL,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
         IF( WANTVR ) THEN
```

```
*
*          Want left and right eigenvectors
*          Copy Schur vectors to VR
*
           SIDE = 'B'
           CALL CLACPY( 'F', N, N, VL, LDVL, VR, LDVR )
        END IF
*
      ELSE IF( WANTVR ) THEN
*
*       Want right eigenvectors
*       Copy Householder vectors to VR
*
        SIDE = 'R'
        CALL CLACPY( 'L', N, N, A, LDA, VR, LDVR )
*
*       Generate unitary matrix in VR
*       (CWorkspace: need 2*N-1, prefer N+(N-1)*NB)
*       (RWorkspace: none)
*
        CALL CUNGHR( N, ILO, IHI, VR, LDVR, WORK( ITAU ), WORK( IWRK ),
     $               LWORK-IWRK+1, IERR )
*
*       Perform QR iteration, accumulating Schur vectors in VR
*       (CWorkspace: need 1, prefer HSWORK (see comments) )
*       (RWorkspace: none)
*
        IWRK = ITAU
        CALL CHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, W, VR, LDVR,
     $               WORK( IWRK ), LWORK-IWRK+1, INFO )
*
      ELSE
*
*       Compute eigenvalues only
*       (CWorkspace: need 1, prefer HSWORK (see comments) )
*       (RWorkspace: none)
*
        IWRK = ITAU
        CALL CHSEQR( 'E', 'N', N, ILO, IHI, A, LDA, W, VR, LDVR,
     $               WORK( IWRK ), LWORK-IWRK+1, INFO )
      END IF
*
      If INFO > 0 from CHSEQR, then quit
*
      IF( INFO.GT.0 )
     $   GO TO 50
*
      IF( WANTVL .OR. WANTVR ) THEN
*
*       Compute left and/or right eigenvectors
```

```
*        (CWorkspace: need 2*N)
*        (RWorkspace: need 2*N)
*
         IRWORK = IBAL + N
         CALL CTREVC( SIDE, 'B', SELECT, N, A, LDA, VL, LDVL, VR, LDVR,
     $                N, NOUT, WORK( IWRK ), RWORK( IRWORK ), IERR )
      END IF
*
      IF( WANTVL ) THEN
*
*        Undo balancing of left eigenvectors
*        (CWorkspace: none)
*        (RWorkspace: need N)
*
         CALL CGEBAK( 'B', 'L', N, ILO, IHI, RWORK( IBAL ), N, VL, LDVL,
     $                IERR )
*
*        Normalize left eigenvectors and make largest component real
*
         DO 20 I = 1, N
            SCL = ONE / SCNRM2( N, VL( 1, I ), 1 )
            CALL CSSCAL( N, SCL, VL( 1, I ), 1 )
            DO 10 K = 1, N
               RWORK( IRWORK+K-1 ) = REAL( VL( K, I ) )**2 +
     $                               AIMAG( VL( K, I ) )**2
   10       CONTINUE
            K = ISAMAX( N, RWORK( IRWORK ), 1 )
            TMP = CONJG( VL( K, I ) ) / SQRT( RWORK( IRWORK+K-1 ) )
            CALL CSCAL( N, TMP, VL( 1, I ), 1 )
            VL( K, I ) = CMPLX( REAL( VL( K, I ) ), ZERO )
   20    CONTINUE
      END IF
*
      IF( WANTVR ) THEN
*
*        Undo balancing of right eigenvectors
*        (CWorkspace: none)
*        (RWorkspace: need N)
*
         CALL CGEBAK( 'B', 'R', N, ILO, IHI, RWORK( IBAL ), N, VR, LDVR,
     $                IERR )
*
*        Normalize right eigenvectors and make largest component real
*
         DO 40 I = 1, N
            SCL = ONE / SCNRM2( N, VR( 1, I ), 1 )
            CALL CSSCAL( N, SCL, VR( 1, I ), 1 )
            DO 30 K = 1, N
               RWORK( IRWORK+K-1 ) = REAL( VR( K, I ) )**2 +
     $                               AIMAG( VR( K, I ) )**2
```

```
   30       CONTINUE
            K = ISAMAX( N, RWORK( IRWORK ), 1 )
            TMP = CONJG( VR( K, I ) ) / SQRT( RWORK( IRWORK+K-1 ) )
            CALL CSCAL( N, TMP, VR( 1, I ), 1 )
            VR( K, I ) = CMPLX( REAL( VR( K, I ) ), ZERO )
   40    CONTINUE
      END IF
*
*     Undo scaling if necessary
*
   50 CONTINUE
      IF( SCALEA ) THEN
         CALL CLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, W( INFO+1 ),
     $                MAX( N-INFO, 1 ), IERR )
         IF( INFO.GT.0 ) THEN
            CALL CLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, W, N, IERR )
         END IF
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of CGEEV
*
      END
```

———-

26.3.4 lisp code

— **cgeev.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun cgeev (jobvl jobvr n a lda w vl ldvl vr ldvr work lwork rwork info)
    (declare (type (array single-float (*)) rwork)
             (type (array f2cl-lib:complex8 (*)) work vr vl w a)
             (type (f2cl-lib:integer4) info lwork ldvr ldvl lda n)
             (type (string *) jobvr jobvl))
    (f2cl-lib:with-multi-array-data
        ((jobvl character jobvl-%data% jobvl-%offset%)
         (jobvr character jobvr-%data% jobvr-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (w f2cl-lib:complex8 w-%data% w-%offset%)
         (vl f2cl-lib:complex8 vl-%data% vl-%offset%)
         (vr f2cl-lib:complex8 vr-%data% vr-%offset%)
```

```
   (work f2cl-lib:complex8 work-%data% work-%offset%)
   (rwork single-float rwork-%data% rwork-%offset%))
(prog ((dum (make-array 1 :element-type 'single-float))
       (select (make-array 1 :element-type 't)) (tmp #C(0.0 0.0))
       (anrm 0.0) (bignum 0.0) (cscale 0.0) (eps 0.0) (scl 0.0)
       (smlnum 0.0) (hswork 0) (i 0) (ibal 0) (ierr 0) (ihi 0) (ilo 0)
       (irwork 0) (itau 0) (iwrk 0) (k 0) (maxwrk 0) (minwrk 0) (nout 0)
       (side
        (make-array '(1) :element-type 'character :initial-element #\ ))
       (lquery nil) (scalea nil) (wantvl nil) (wantvr nil))
  (declare (type (array single-float (1)) dum)
           (type (array f2cl-lib:logical (1)) select)
           (type (f2cl-lib:complex8) tmp)
           (type (single-float) anrm bignum cscale eps scl smlnum)
           (type (f2cl-lib:integer4) hswork i ibal ierr ihi ilo irwork
            itau iwrk k maxwrk minwrk nout)
           (type (string 1) side)
           (type f2cl-lib:logical lquery scalea wantvl wantvr))
  (setf info 0)
  (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
  (setf wantvl
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame jobvl "V")
            (declare (ignore var-1))
            (when var-0 (setf jobvl var-0))
            ret-val))
  (setf wantvr
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame jobvr "V")
            (declare (ignore var-1))
            (when var-0 (setf jobvr var-0))
            ret-val))
  (cond
   ((and (not wantvl)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame jobvl "N")
            (declare (ignore var-1))
            (when var-0 (setf jobvl var-0))
            ret-val)))
    (setf info -1))
   ((and (not wantvr)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame jobvr "N")
            (declare (ignore var-1))
            (when var-0 (setf jobvr var-0))
            ret-val)))
    (setf info -2))
   ((< n 0) (setf info -3))
```

```
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((or (< ldvl 1) (and wantvl (< ldvl n))) (setf info -8))
 ((or (< ldvr 1) (and wantvr (< ldvr n))) (setf info -10)))
(cond
 ((= info 0)
  (cond ((= n 0) (setf minwrk 1) (setf maxwrk 1))
        (t
         (setf maxwrk
                 (f2cl-lib:int-add n
                                   (f2cl-lib:int-mul n
                                                     (multiple-value-bind
                                                         (ret-val var-0
                                                          var-1 var-2
                                                          var-3 var-4
                                                          var-5 var-6)
                                                         (ilaenv 1
                                                          "CGEHRD" " "
                                                          n 1 n 0)
                                                       (declare
                                                        (ignore var-0
                                                         var-1 var-2
                                                         var-4 var-6))
                                                       (when var-3
                                                         (setf n
                                                                 var-3))
                                                       (when var-5
                                                         (setf n
                                                                 var-5))
                                                       ret-val))))
         (setf minwrk (f2cl-lib:int-mul 2 n))
         (cond
          (wantvl
           (setf maxwrk
                   (max (the f2cl-lib:integer4 maxwrk)
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add n
                                               (f2cl-lib:int-mul
                                                (f2cl-lib:int-sub n 1)
                                                (multiple-value-bind
                                                    (ret-val var-0
                                                     var-1 var-2 var-3
                                                     var-4 var-5 var-6)
                                                    (ilaenv 1 "CUNGHR"
                                                     " " n 1 n -1)
                                                  (declare
                                                   (ignore var-0 var-1
                                                    var-2 var-4 var-6))
                                                  (when var-3
                                                    (setf n var-3))
```

```
                                        (when var-5
                                          (setf n var-5))
                                        ret-val))))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12)
     (chseqr "S" "V" n 1 n a lda w vl ldvl work -1 info)
   (declare
    (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10
     var-11))
   (when var-2 (setf n var-2))
   (when var-4 (setf n var-4))
   (when var-6 (setf lda var-6))
   (when var-9 (setf ldvl var-9))
   (when var-12 (setf info var-12))))
(wantvr
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n
                                     (f2cl-lib:int-mul
                                      (f2cl-lib:int-sub n 1)
                                      (multiple-value-bind
                                          (ret-val var-0
                                           var-1 var-2 var-3
                                           var-4 var-5 var-6)
                                          (ilaenv 1 "CUNGHR"
                                           " " n 1 n -1)
                                        (declare
                                         (ignore var-0 var-1
                                          var-2 var-4 var-6))
                                        (when var-3
                                          (setf n var-3))
                                        (when var-5
                                          (setf n var-5))
                                        ret-val))))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12)
     (chseqr "S" "V" n 1 n a lda w vr ldvr work -1 info)
   (declare
    (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10
     var-11))
   (when var-2 (setf n var-2))
   (when var-4 (setf n var-4))
   (when var-6 (setf lda var-6))
   (when var-9 (setf ldvr var-9))
   (when var-12 (setf info var-12))))
(t
 (multiple-value-bind
```

```
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10 var-11 var-12)
                (chseqr "E" "N" n 1 n a lda w vr ldvr work -1 info)
              (declare
               (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10
                var-11))
              (when var-2 (setf n var-2))
              (when var-4 (setf n var-4))
              (when var-6 (setf lda var-6))
              (when var-9 (setf ldvr var-9))
              (when var-12 (setf info var-12)))))
          (setf hswork
                  (f2cl-lib:int
                   (f2cl-lib:fref work-%data% (1) ((1 *))
                                  work-%offset%)))
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4 hswork)
                       (the f2cl-lib:integer4 minwrk)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce maxwrk 'f2cl-lib:complex8))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -12)))))
(cond
 ((/= info 0) (xerbla "CGEEV " (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(setf eps (slamch "P"))
(setf smlnum (slamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (slabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "M" n n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf scalea f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf scalea f2cl-lib:%true%)
  (setf cscale smlnum))
```

```
 ((> anrm bignum) (setf scalea f2cl-lib:%true%) (setf cscale bignum)))
(if scalea
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (clascl "G" 0 0 anrm cscale n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf cscale var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(setf ibal 1)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgebal "B" n a lda ilo ihi
     (f2cl-lib:array-slice rwork-%data% single-float (ibal) ((1 *))
                           rwork-%offset%)
     ierr)
  (declare (ignore var-0 var-1 var-2 var-6))
  (setf lda var-3)
  (setf ilo var-4)
  (setf ihi var-5)
  (setf ierr var-7))
(setf itau 1)
(setf iwrk (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cgehrd n ilo ihi a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf ilo var-1))
  (when var-2 (setf ihi var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(cond
 (wantvl (f2cl-lib:f2cl-set-string side "L" (string 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "L" n n a lda vl ldvl)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvl var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
```

```
      (cunghr n ilo ihi vl ldvl
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf n var-0))
    (when var-1 (setf ilo var-1))
    (when var-2 (setf ihi var-2))
    (when var-4 (setf ldvl var-4))
    (when var-8 (setf ierr var-8)))
  (setf iwrk itau)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (chseqr "S" "V" n ilo ihi a lda w vl ldvl
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldvl var-9))
    (when var-12 (setf info var-12)))
  (cond
   (wantvr (f2cl-lib:f2cl-set-string side "B" (string 1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (clacpy "F" n n vl ldvl vr ldvr)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf ldvl var-4))
      (when var-6 (setf ldvr var-6))))))
 (wantvr (f2cl-lib:f2cl-set-string side "R" (string 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "L" n n a lda vr ldvr)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvr var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (cunghr n ilo ihi vr ldvr
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
```

```
                              ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
      (declare (ignore var-3 var-5 var-6 var-7))
      (when var-0 (setf n var-0))
      (when var-1 (setf ilo var-1))
      (when var-2 (setf ihi var-2))
      (when var-4 (setf ldvr var-4))
      (when var-8 (setf ierr var-8)))
    (setf iwrk itau)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (chseqr "S" "V" n ilo ihi a lda w vr ldvr
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                              ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf n var-2))
      (when var-3 (setf ilo var-3))
      (when var-4 (setf ihi var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldvr var-9))
      (when var-12 (setf info var-12))))
   (t (setf iwrk itau)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (chseqr "E" "N" n ilo ihi a lda w vr ldvr
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                              ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf n var-2))
      (when var-3 (setf ilo var-3))
      (when var-4 (setf ihi var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldvr var-9))
      (when var-12 (setf info var-12)))))
  (if (> info 0)
      (go label50))
  (cond
   ((or wantvl wantvr) (setf irwork (f2cl-lib:int-add ibal n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14)
        (ctrevc side "B" select n a lda vl ldvl vr ldvr n nout
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                              ((1 *)) work-%offset%)
         (f2cl-lib:array-slice rwork-%data% single-float (irwork) ((1 *))
                              rwork-%offset%)
```

```
        ierr)
    (declare (ignore var-1 var-2 var-4 var-6 var-8 var-12 var-13))
    (when var-0 (setf side var-0))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldvl var-7))
    (when var-9 (setf ldvr var-9))
    (when var-10 (setf n var-10))
    (when var-11 (setf nout var-11))
    (when var-14 (setf ierr var-14)))))
(cond
 (wantvl
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (cgebak "B" "L" n ilo ihi
       (f2cl-lib:array-slice rwork-%data% single-float (ibal) ((1 *))
                             rwork-%offset%)
       n vl ldvl ierr)
    (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7))
    (setf n var-6)
    (setf ldvl var-8)
    (setf ierr var-9))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf scl
              (/ one
                 (multiple-value-bind (ret-val var-0 var-1 var-2)
                     (scnrm2 n
                      (f2cl-lib:array-slice vl-%data% f2cl-lib:complex8
                                            (1 i) ((1 ldvl) (1 *))
                                            vl-%offset%)
                      1)
                   (declare (ignore var-1 var-2))
                   (when var-0 (setf n var-0))
                   ret-val)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (csscal n scl
           (f2cl-lib:array-slice vl-%data% f2cl-lib:complex8 (1 i)
                                 ((1 ldvl) (1 *)) vl-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf n var-0))
        (when var-1 (setf scl var-1)))
      (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                    ((> k n) nil)
        (tagbody
          (setf (f2cl-lib:fref rwork-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add irwork k) 1))
```

```
                                ((1 *)) rwork-%offset%)
                        (+
                         (expt
                          (f2cl-lib:freal
                           (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                                          vl-%offset%))
                          2)
                         (expt
                          (f2cl-lib:aimag
                           (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                                          vl-%offset%))
                          2)))
                label10))
            (setf k
                    (multiple-value-bind (ret-val var-0 var-1 var-2)
                        (isamax n
                         (f2cl-lib:array-slice rwork-%data% single-float
                                               (irwork) ((1 *))
                                               rwork-%offset%)
                         1)
                      (declare (ignore var-1 var-2))
                      (when var-0 (setf n var-0))
                      ret-val))
            (setf tmp
                    (/
                     (f2cl-lib:conjg
                      (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                                     vl-%offset%))
                     (f2cl-lib:fsqrt
                      (f2cl-lib:fref rwork-%data%
                                     ((f2cl-lib:int-sub
                                       (f2cl-lib:int-add irwork k) 1))
                                     ((1 *)) rwork-%offset%))))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (cscal n tmp
                 (f2cl-lib:array-slice vl-%data% f2cl-lib:complex8 (1 i)
                                       ((1 ldvl) (1 *)) vl-%offset%)
                 1)
              (declare (ignore var-2 var-3))
              (when var-0 (setf n var-0))
              (when var-1 (setf tmp var-1)))
            (setf (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                                 vl-%offset%)
                    (f2cl-lib:cmplx
                     (f2cl-lib:freal
                      (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                                     vl-%offset%))
                     zero))
          label20))))
      (cond
```

```
(wantvr
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
     (cgebak "B" "R" n ilo ihi
      (f2cl-lib:array-slice rwork-%data% single-float (ibal) ((1 *))
                            rwork-%offset%)
      n vr ldvr ierr)
   (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7))
   (setf n var-6)
   (setf ldvr var-8)
   (setf ierr var-9))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf scl
             (/ one
                (multiple-value-bind (ret-val var-0 var-1 var-2)
                    (scnrm2 n
                     (f2cl-lib:array-slice vr-%data% f2cl-lib:complex8
                                           (1 i) ((1 ldvr) (1 *))
                                           vr-%offset%)
                     1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf n var-0))
                  ret-val)))
     (multiple-value-bind (var-0 var-1 var-2 var-3)
         (csscal n scl
          (f2cl-lib:array-slice vr-%data% f2cl-lib:complex8 (1 i)
                                ((1 ldvr) (1 *)) vr-%offset%)
          1)
       (declare (ignore var-2 var-3))
       (when var-0 (setf n var-0))
       (when var-1 (setf scl var-1)))
     (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                   ((> k n) nil)
       (tagbody
         (setf (f2cl-lib:fref rwork-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add irwork k) 1))
                              ((1 *)) rwork-%offset%)
                 (+
                  (expt
                   (f2cl-lib:freal
                    (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                                   vr-%offset%))
                   2)
                  (expt
                   (f2cl-lib:aimag
                    (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                                   vr-%offset%))
```

```
                    2)))
          label30))
        (setf k
                (multiple-value-bind (ret-val var-0 var-1 var-2)
                    (isamax n
                     (f2cl-lib:array-slice rwork-%data% single-float
                                           (irwork) ((1 *))
                                           rwork-%offset%)
                     1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf n var-0))
                  ret-val))
        (setf tmp
                (/
                 (f2cl-lib:conjg
                  (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                                 vr-%offset%))
                 (f2cl-lib:fsqrt
                  (f2cl-lib:fref rwork-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-add irwork k) 1))
                                 ((1 *)) rwork-%offset%))))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (cscal n tmp
             (f2cl-lib:array-slice vr-%data% f2cl-lib:complex8 (1 i)
                                   ((1 ldvr) (1 *)) vr-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-1 (setf tmp var-1)))
        (setf (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                             vr-%offset%)
                (f2cl-lib:cmplx
                 (f2cl-lib:freal
                  (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                                 vr-%offset%))
                 zero))
      label40))))
label50
 (cond
  (scalea
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (clascl "G" 0 0 cscale anrm (f2cl-lib:int-sub n info) 1
        (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 ((+ info 1))
                              ((1 *)) w-%offset%)
        (max (the f2cl-lib:integer4 (f2cl-lib:int-sub n info))
             (the f2cl-lib:integer4 1))
        ierr)
     (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
```

```
            (when var-3 (setf cscale var-3))
            (when var-4 (setf anrm var-4))
            (when var-9 (setf ierr var-9)))
          (cond
           ((> info 0)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                (clascl "G" 0 0 cscale anrm (f2cl-lib:int-sub ilo 1) 1 w n
                 ierr)
              (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
              (when var-3 (setf cscale var-3))
              (when var-4 (setf anrm var-4))
              (when var-8 (setf n var-8))
              (when var-9 (setf ierr var-9)))))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce maxwrk 'f2cl-lib:complex8))
        (go end_label)
       end_label
        (return
         (values jobvl jobvr n nil lda nil nil ldvl nil ldvr nil nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgeev fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::jobvl
                                              fortran-to-lisp::jobvr
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil
```

```
                                                  fortran-to-lisp::ldvl nil
                                                  fortran-to-lisp::ldvr nil nil nil
                                                  fortran-to-lisp::info)
                                                :calls
                                                '(fortran-to-lisp::cgebak
                                                  fortran-to-lisp::cgebal))))
```

———-

26.4 cgeevx

26.4.1 Axiom unit tests

— **cgeevx.input** —

```
)set break resume
)sys rm -f cgeevx.output
)spool cgeevx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

26.4.2 Axiom help page

— **cgeevx.help** —

```
=====================================================================
cgeevx examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgeevx (BALANC, JOBVL, JOBVR, SENSE, N, A, LDA, W, VL, LDVL,
```

```
          VR, LDVR, ILO, IHI, SCALE, ABNRM, RCONDE, RCONDV, WORK, LWORK,
          RWORK, INFO)
           CGEEVX computes the eigenvalues and, optionally, the left and/or
          right eigenvectors for GE matrices

Function/Subroutine Documentation
   subroutine cgeevx (character BALANC, character JOBVL, character JOBVR,
       character SENSE, integer N, complex, dimension( lda, * ) A, integer
       LDA, complex, dimension( * ) W, complex, dimension( ldvl, * ) VL,
       integer LDVL, complex, dimension( ldvr, * ) VR, integer LDVR, integer
       ILO, integer IHI, real, dimension( * ) SCALE, real ABNRM, real,
       dimension( * ) RCONDE, real, dimension( * ) RCONDV, complex, dimension(
       * ) WORK, integer LWORK, real, dimension( * ) RWORK, integer INFO)
        CGEEVX computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for GE matrices

Purpose:
   CGEEVX computes for an N-by-N complex nonsymmetric matrix A, the
   eigenvalues and, optionally, the left and/or right eigenvectors.

   Optionally also, it computes a balancing transformation to improve
   the conditioning of the eigenvalues and eigenvectors (ILO, IHI,
   SCALE, and ABNRM), reciprocal condition numbers for the eigenvalues
   (RCONDE), and reciprocal condition numbers for the right
   eigenvectors (RCONDV).

   The right eigenvector v(j) of A satisfies
       A * v(j) = lambda(j) * v(j)
   where lambda(j) is its eigenvalue.

   The left eigenvector u(j) of A satisfies
       u(j)**H * A = lambda(j) * u(j)**H
   where u(j)**H denotes the conjugate transpose of u(j).

   The computed eigenvectors are normalized to have Euclidean norm
   equal to 1 and largest component real.

   Balancing a matrix means permuting the rows and columns to make it
   more nearly upper triangular, and applying a diagonal similarity
   transformation D * A * D**(-1), where D is a diagonal matrix, to
   make its rows and columns closer in norm and the condition numbers
   of its eigenvalues and eigenvectors smaller.  The computed
   reciprocal condition numbers correspond to the balanced matrix.
   Permuting rows and columns will not change the condition numbers
   (in exact arithmetic) but diagonal scaling will.  For further
   explanation of balancing, see section 4.10.2 of the LAPACK Users' Guide.

Parameters:

 BALANC
```

```
  BALANC is CHARACTER*1
  Indicates how the input matrix should be diagonally scaled
  and/or permuted to improve the conditioning of its eigenvalues.
  = 'N': Do not diagonally scale or permute;
  = 'P': Perform permutations to make the matrix more nearly
         upper triangular. Do not diagonally scale;
  = 'S': Diagonally scale the matrix, ie. replace A by
         D*A*D**(-1), where D is a diagonal matrix chosen
         to make the rows and columns of A more equal in
         norm. Do not permute;
  = 'B': Both diagonally scale and permute A.

  Computed reciprocal condition numbers will be for the matrix
  after balancing and/or permuting. Permuting does not change
  condition numbers (in exact arithmetic), but balancing does.

JOBVL
  JOBVL is CHARACTER*1
  = 'N': left eigenvectors of A are not computed;
  = 'V': left eigenvectors of A are computed.
  If SENSE = 'E' or 'B', JOBVL must = 'V'.

JOBVR
  JOBVR is CHARACTER*1
  = 'N': right eigenvectors of A are not computed;
  = 'V': right eigenvectors of A are computed.
  If SENSE = 'E' or 'B', JOBVR must = 'V'.

SENSE
  SENSE is CHARACTER*1
  Determines which reciprocal condition numbers are computed.
  = 'N': None are computed;
  = 'E': Computed for eigenvalues only;
  = 'V': Computed for right eigenvectors only;
  = 'B': Computed for eigenvalues and right eigenvectors.

  If SENSE = 'E' or 'B', both left and right eigenvectors
  must also be computed (JOBVL = 'V' and JOBVR = 'V').

N
  N is INTEGER
  The order of the matrix A. N >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the N-by-N matrix A.
  On exit, A has been overwritten.  If JOBVL = 'V' or
  JOBVR = 'V', A contains the Schur form of the balanced
  version of the matrix A.
```

```
LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

W
  W is COMPLEX array, dimension (N)
  W contains the computed eigenvalues.

VL
  VL is COMPLEX array, dimension (LDVL,N)
  If JOBVL = 'V', the left eigenvectors u(j) are stored one
  after another in the columns of VL, in the same order
  as their eigenvalues.
  If JOBVL = 'N', VL is not referenced.
  u(j) = VL(:,j), the j-th column of VL.

LDVL
  LDVL is INTEGER
  The leading dimension of the array VL.  LDVL >= 1; if
  JOBVL = 'V', LDVL >= N.

VR
  VR is COMPLEX array, dimension (LDVR,N)
  If JOBVR = 'V', the right eigenvectors v(j) are stored one
  after another in the columns of VR, in the same order
  as their eigenvalues.
  If JOBVR = 'N', VR is not referenced.
  v(j) = VR(:,j), the j-th column of VR.

LDVR
  LDVR is INTEGER
  The leading dimension of the array VR.  LDVR >= 1; if
  JOBVR = 'V', LDVR >= N.

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER
  ILO and IHI are integer values determined when A was
  balanced.  The balanced A(i,j) = 0 if I > J and
  J = 1,...,ILO-1 or I = IHI+1,...,N.

SCALE
  SCALE is REAL array, dimension (N)
  Details of the permutations and scaling factors applied
  when balancing A.  If P(j) is the index of the row and column
  interchanged with row and column j, and D(j) is the scaling
  factor applied to row and column j, then
  SCALE(J) = P(J),    for J = 1,...,ILO-1
```

```
          = D(J),    for J = ILO,...,IHI
          = P(J)     for J = IHI+1,...,N.
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

ABNRM
  ABNRM is REAL
  The one-norm of the balanced matrix (the maximum
  of the sum of absolute values of elements of any column).

RCONDE
  RCONDE is REAL array, dimension (N)
  RCONDE(j) is the reciprocal condition number of the j-th eigenvalue.

RCONDV
  RCONDV is REAL array, dimension (N)
  RCONDV(j) is the reciprocal condition number of the j-th
  right eigenvector.

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  If SENSE = 'N' or 'E',
  LWORK >= max(1,2*N), and if SENSE = 'V' or 'B',
  LWORK >= N*N+2*N.
  For good performance, LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is REAL array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if INFO = i, the QR algorithm failed to compute all the
        eigenvalues, and no eigenvectors or condition numbers
        have been computed; elements 1:ILO-1 and i+1:N of W
        contain eigenvalues which have converged.
```

———-

26.4.3 fortran code

— cgeevx.f —

```
*  =====================================================================
      SUBROUTINE CGEEVX( BALANC, JOBVL, JOBVR, SENSE, N, A, LDA, W, VL,
     $                   LDVL, VR, LDVR, ILO, IHI, SCALE, ABNRM, RCONDE,
     $                   RCONDV, WORK, LWORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          BALANC, JOBVL, JOBVR, SENSE
      INTEGER            IHI, ILO, INFO, LDA, LDVL, LDVR, LWORK, N
      REAL               ABNRM
*     ..
*     .. Array Arguments ..
      REAL               RCONDE( * ), RCONDV( * ), RWORK( * ),
     $                   SCALE( * )
      COMPLEX            A( LDA, * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   W( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, SCALEA, WANTVL, WANTVR, WNTSNB, WNTSNE,
     $                   WNTSNN, WNTSNV
      CHARACTER          JOB, SIDE
      INTEGER            HSWORK, I, ICOND, IERR, ITAU, IWRK, K, MAXWRK,
     $                   MINWRK, NOUT
      REAL               ANRM, BIGNUM, CSCALE, EPS, SCL, SMLNUM
      COMPLEX            TMP
*     ..
*     .. Local Arrays ..
      LOGICAL            SELECT( 1 )
      REAL               DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEBAK, CGEBAL, CGEHRD, CHSEQR, CLACPY, CLASCL,
     $                   CSCAL, CSSCAL, CTREVC, CTRSNA, CUNGHR, SLABAD,
     $                   SLASCL, XERBLA
```

```
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV, ISAMAX
      REAL               CLANGE, SCNRM2, SLAMCH
      EXTERNAL           LSAME, ILAENV, ISAMAX, CLANGE, SCNRM2, SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          AIMAG, CMPLX, CONJG, MAX, REAL, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      WANTVL = LSAME( JOBVL, 'V' )
      WANTVR = LSAME( JOBVR, 'V' )
      WNTSNN = LSAME( SENSE, 'N' )
      WNTSNE = LSAME( SENSE, 'E' )
      WNTSNV = LSAME( SENSE, 'V' )
      WNTSNB = LSAME( SENSE, 'B' )
      IF( .NOT.( LSAME( BALANC, 'N' ) .OR. LSAME( BALANC, 'S' ) .OR.
     $    LSAME( BALANC, 'P' ) .OR. LSAME( BALANC, 'B' ) ) ) THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTVL ) .AND. ( .NOT.LSAME( JOBVL, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( ( .NOT.WANTVR ) .AND. ( .NOT.LSAME( JOBVR, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( .NOT.( WNTSNN .OR. WNTSNE .OR. WNTSNB .OR. WNTSNV ) .OR.
     $         ( ( WNTSNE .OR. WNTSNB ) .AND. .NOT.( WANTVL .AND.
     $         WANTVR ) ) ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDVL.LT.1 .OR. ( WANTVL .AND. LDVL.LT.N ) ) THEN
         INFO = -10
      ELSE IF( LDVR.LT.1 .OR. ( WANTVR .AND. LDVR.LT.N ) ) THEN
         INFO = -12
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       CWorkspace refers to complex workspace, and RWorkspace to real
*       workspace. NB refers to the optimal block size for the
*       immediately following subroutine, as returned by ILAENV.
```

```
*       HSWORK refers to the workspace preferred by CHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            MAXWRK = 1
         ELSE
            MAXWRK = N + N*ILAENV( 1, 'CGEHRD', ' ', N, 1, N, 0 )
*
            IF( WANTVL ) THEN
               CALL CHSEQR( 'S', 'V', N, 1, N, A, LDA, W, VL, LDVL,
     $                WORK, -1, INFO )
            ELSE IF( WANTVR ) THEN
               CALL CHSEQR( 'S', 'V', N, 1, N, A, LDA, W, VR, LDVR,
     $                WORK, -1, INFO )
            ELSE
               IF( WNTSNN ) THEN
                  CALL CHSEQR( 'E', 'N', N, 1, N, A, LDA, W, VR, LDVR,
     $                WORK, -1, INFO )
               ELSE
                  CALL CHSEQR( 'S', 'N', N, 1, N, A, LDA, W, VR, LDVR,
     $                WORK, -1, INFO )
               END IF
            END IF
            HSWORK = WORK( 1 )
*
            IF( ( .NOT.WANTVL ) .AND. ( .NOT.WANTVR ) ) THEN
               MINWRK = 2*N
               IF( .NOT.( WNTSNN .OR. WNTSNE ) )
     $            MINWRK = MAX( MINWRK, N*N + 2*N )
               MAXWRK = MAX( MAXWRK, HSWORK )
               IF( .NOT.( WNTSNN .OR. WNTSNE ) )
     $            MAXWRK = MAX( MAXWRK, N*N + 2*N )
            ELSE
               MINWRK = 2*N
               IF( .NOT.( WNTSNN .OR. WNTSNE ) )
     $            MINWRK = MAX( MINWRK, N*N + 2*N )
               MAXWRK = MAX( MAXWRK, HSWORK )
               MAXWRK = MAX( MAXWRK, N + ( N - 1 )*ILAENV( 1, 'CUNGHR',
     $                       ' ', N, 1, N, -1 ) )
               IF( .NOT.( WNTSNN .OR. WNTSNE ) )
     $            MAXWRK = MAX( MAXWRK, N*N + 2*N )
               MAXWRK = MAX( MAXWRK, 2*N )
            END IF
            MAXWRK = MAX( MAXWRK, MINWRK )
         END IF
         WORK( 1 ) = MAXWRK
*
```

```
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -20
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEEVX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ICOND = 0
      ANRM = CLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
     $   CALL CLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*     Balance the matrix and compute ABNRM
*
      CALL CGEBAL( BALANC, N, A, LDA, ILO, IHI, SCALE, IERR )
      ABNRM = CLANGE( '1', N, N, A, LDA, DUM )
      IF( SCALEA ) THEN
         DUM( 1 ) = ABNRM
         CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, 1, 1, DUM, 1, IERR )
         ABNRM = DUM( 1 )
      END IF
```

```
*
*     Reduce to upper Hessenberg form
*     (CWorkspace: need 2*N, prefer N+N*NB)
*     (RWorkspace: none)
*
      ITAU = 1
      IWRK = ITAU + N
      CALL CGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
      IF( WANTVL ) THEN
*
*        Want left eigenvectors
*        Copy Householder vectors to VL
*
         SIDE = 'L'
         CALL CLACPY( 'L', N, N, A, LDA, VL, LDVL )
*
*        Generate unitary matrix in VL
*        (CWorkspace: need 2*N-1, prefer N+(N-1)*NB)
*        (RWorkspace: none)
*
         CALL CUNGHR( N, ILO, IHI, VL, LDVL, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VL
*        (CWorkspace: need 1, prefer HSWORK (see comments) )
*        (RWorkspace: none)
*
         IWRK = ITAU
         CALL CHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, W, VL, LDVL,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
         IF( WANTVR ) THEN
*
*           Want left and right eigenvectors
*           Copy Schur vectors to VR
*
            SIDE = 'B'
            CALL CLACPY( 'F', N, N, VL, LDVL, VR, LDVR )
         END IF
*
      ELSE IF( WANTVR ) THEN
*
*        Want right eigenvectors
*        Copy Householder vectors to VR
*
         SIDE = 'R'
         CALL CLACPY( 'L', N, N, A, LDA, VR, LDVR )
*
```

```
*        Generate unitary matrix in VR
*        (CWorkspace: need 2*N-1, prefer N+(N-1)*NB)
*        (RWorkspace: none)
*
         CALL CUNGHR( N, ILO, IHI, VR, LDVR, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VR
*        (CWorkspace: need 1, prefer HSWORK (see comments) )
*        (RWorkspace: none)
*
         IWRK = ITAU
         CALL CHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, W, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
      ELSE
*
*        Compute eigenvalues only
*        If condition numbers desired, compute Schur form
*
         IF( WNTSNN ) THEN
            JOB = 'E'
         ELSE
            JOB = 'S'
         END IF
*
*        (CWorkspace: need 1, prefer HSWORK (see comments) )
*        (RWorkspace: none)
*
         IWRK = ITAU
         CALL CHSEQR( JOB, 'N', N, ILO, IHI, A, LDA, W, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
      END IF
*
*     If INFO > 0 from CHSEQR, then quit
*
      IF( INFO.GT.0 )
     $   GO TO 50
*
      IF( WANTVL .OR. WANTVR ) THEN
*
*        Compute left and/or right eigenvectors
*        (CWorkspace: need 2*N)
*        (RWorkspace: need N)
*
         CALL CTREVC( SIDE, 'B', SELECT, N, A, LDA, VL, LDVL, VR, LDVR,
     $                N, NOUT, WORK( IWRK ), RWORK, IERR )
      END IF
*
*     Compute condition numbers if desired
```

```
*     (CWorkspace: need N*N+2*N unless SENSE = 'E')
*     (RWorkspace: need 2*N unless SENSE = 'E')
*
      IF( .NOT.WNTSNN ) THEN
         CALL CTRSNA( SENSE, 'A', SELECT, N, A, LDA, VL, LDVL, VR, LDVR,
     $                RCONDE, RCONDV, N, NOUT, WORK( IWRK ), N, RWORK,
     $                ICOND )
      END IF
*
      IF( WANTVL ) THEN
*
*        Undo balancing of left eigenvectors
*
         CALL CGEBAK( BALANC, 'L', N, ILO, IHI, SCALE, N, VL, LDVL,
     $                IERR )
*
*        Normalize left eigenvectors and make largest component real
*
         DO 20 I = 1, N
            SCL = ONE / SCNRM2( N, VL( 1, I ), 1 )
            CALL CSSCAL( N, SCL, VL( 1, I ), 1 )
            DO 10 K = 1, N
               RWORK( K ) = REAL( VL( K, I ) )**2 +
     $                      AIMAG( VL( K, I ) )**2
   10       CONTINUE
            K = ISAMAX( N, RWORK, 1 )
            TMP = CONJG( VL( K, I ) ) / SQRT( RWORK( K ) )
            CALL CSCAL( N, TMP, VL( 1, I ), 1 )
            VL( K, I ) = CMPLX( REAL( VL( K, I ) ), ZERO )
   20    CONTINUE
      END IF
*
      IF( WANTVR ) THEN
*
*        Undo balancing of right eigenvectors
*
         CALL CGEBAK( BALANC, 'R', N, ILO, IHI, SCALE, N, VR, LDVR,
     $                IERR )
*
*        Normalize right eigenvectors and make largest component real
*
         DO 40 I = 1, N
            SCL = ONE / SCNRM2( N, VR( 1, I ), 1 )
            CALL CSSCAL( N, SCL, VR( 1, I ), 1 )
            DO 30 K = 1, N
               RWORK( K ) = REAL( VR( K, I ) )**2 +
     $                      AIMAG( VR( K, I ) )**2
   30       CONTINUE
            K = ISAMAX( N, RWORK, 1 )
            TMP = CONJG( VR( K, I ) ) / SQRT( RWORK( K ) )
```

```
            CALL CSCAL( N, TMP, VR( 1, I ), 1 )
            VR( K, I ) = CMPLX( REAL( VR( K, I ) ), ZERO )
   40    CONTINUE
      END IF
*
*     Undo scaling if necessary
*
   50 CONTINUE
      IF( SCALEA ) THEN
         CALL CLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, W( INFO+1 ),
     $                MAX( N-INFO, 1 ), IERR )
         IF( INFO.EQ.0 ) THEN
            IF( ( WNTSNV .OR. WNTSNB ) .AND. ICOND.EQ.0 )
     $         CALL SLASCL( 'G', 0, 0, CSCALE, ANRM, N, 1, RCONDV, N,
     $                      IERR )
         ELSE
            CALL CLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, W, N, IERR )
         END IF
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of CGEEVX
*
      END
```

———-

26.4.4 lisp code

— cgeevx.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun cgeevx
         (balanc jobvl jobvr sense n a lda w vl ldvl vr ldvr ilo ihi scale
          abnrm rconde rcondv work lwork rwork info)
    (declare (type (single-float) abnrm)
             (type (array single-float (*)) rwork rcondv rconde scale)
             (type (array f2cl-lib:complex8 (*)) work vr vl w a)
             (type (f2cl-lib:integer4) info lwork ihi ilo ldvr ldvl lda n)
             (type (string *) sense jobvr jobvl balanc))
    (f2cl-lib:with-multi-array-data
        ((balanc character balanc-%data% balanc-%offset%)
         (jobvl character jobvl-%data% jobvl-%offset%)
```

```
   (jobvr character jobvr-%data% jobvr-%offset%)
   (sense character sense-%data% sense-%offset%)
   (a f2cl-lib:complex8 a-%data% a-%offset%)
   (w f2cl-lib:complex8 w-%data% w-%offset%)
   (vl f2cl-lib:complex8 vl-%data% vl-%offset%)
   (vr f2cl-lib:complex8 vr-%data% vr-%offset%)
   (work f2cl-lib:complex8 work-%data% work-%offset%)
   (scale single-float scale-%data% scale-%offset%)
   (rconde single-float rconde-%data% rconde-%offset%)
   (rcondv single-float rcondv-%data% rcondv-%offset%)
   (rwork single-float rwork-%data% rwork-%offset%))
(prog ((dum (make-array 1 :element-type 'single-float))
       (select (make-array 1 :element-type 't)) (tmp #C(0.0 0.0))
       (anrm 0.0) (bignum 0.0) (cscale 0.0) (eps 0.0) (scl 0.0)
       (smlnum 0.0) (hswork 0) (i 0) (icond 0) (ierr 0) (itau 0) (iwrk 0)
       (k 0) (maxwrk 0) (minwrk 0) (nout 0)
       (job
        (make-array '(1) :element-type 'character :initial-element #\ ))
       (side
        (make-array '(1) :element-type 'character :initial-element #\ ))
       (lquery nil) (scalea nil) (wantvl nil) (wantvr nil) (wntsnb nil)
       (wntsne nil) (wntsnn nil) (wntsnv nil))
  (declare (type (array single-float (1)) dum)
           (type (array f2cl-lib:logical (1)) select)
           (type (f2cl-lib:complex8) tmp)
           (type (single-float) anrm bignum cscale eps scl smlnum)
           (type (f2cl-lib:integer4) hswork i icond ierr itau iwrk k
            maxwrk minwrk nout)
           (type (string 1) job side)
           (type f2cl-lib:logical lquery scalea wantvl wantvr wntsnb
            wntsne wntsnn wntsnv))
  (setf info 0)
  (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
  (setf wantvl
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame jobvl "V")
            (declare (ignore var-1))
            (when var-0 (setf jobvl var-0))
            ret-val))
  (setf wantvr
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame jobvr "V")
            (declare (ignore var-1))
            (when var-0 (setf jobvr var-0))
            ret-val))
  (setf wntsnn
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame sense "N")
            (declare (ignore var-1))
            (when var-0 (setf sense var-0))
```

```
          ret-val))
(setf wntsne
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "E")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wntsnv
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "V")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wntsnb
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "B")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(cond
 ((not
   (or
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "N")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "S")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "P")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "B")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)))
  (setf info -1))
 ((and (not wantvl)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvl "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvl var-0))
          ret-val)))
```

```
  (setf info -2))
 ((and (not wantvr)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvr "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvr var-0))
          ret-val)))
  (setf info -3))
 ((or (not (or wntsnn wntsne wntsnb wntsnv))
      (and (or wntsne wntsnb) (not (and wantvl wantvr))))
  (setf info -4))
 ((< n 0) (setf info -5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7))
 ((or (< ldvl 1) (and wantvl (< ldvl n))) (setf info -10))
 ((or (< ldvr 1) (and wantvr (< ldvr n))) (setf info -12)))
(cond
 ((= info 0)
  (cond ((= n 0) (setf minwrk 1) (setf maxwrk 1))
        (t
         (setf maxwrk
                 (f2cl-lib:int-add n
                                   (f2cl-lib:int-mul n
                                                     (multiple-value-bind
                                                         (ret-val var-0
                                                          var-1 var-2
                                                          var-3 var-4
                                                          var-5 var-6)
                                                         (ilaenv 1
                                                          "CGEHRD" " "
                                                          n 1 n 0)
                                                       (declare
                                                        (ignore var-0
                                                         var-1 var-2
                                                         var-4 var-6))
                                                       (when var-3
                                                         (setf n
                                                                 var-3))
                                                       (when var-5
                                                         (setf n
                                                                 var-5))
                                                       ret-val))))
         (cond
          (wantvl
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                var-9 var-10 var-11 var-12)
               (chseqr "S" "V" n 1 n a lda w vl ldvl work -1 info)
             (declare
```

```
      (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10
       var-11))
     (when var-2 (setf n var-2))
     (when var-4 (setf n var-4))
     (when var-6 (setf lda var-6))
     (when var-9 (setf ldvl var-9))
     (when var-12 (setf info var-12))))
   (wantvr
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (chseqr "S" "V" n 1 n a lda w vr ldvr work -1 info)
     (declare
      (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10
       var-11))
     (when var-2 (setf n var-2))
     (when var-4 (setf n var-4))
     (when var-6 (setf lda var-6))
     (when var-9 (setf ldvr var-9))
     (when var-12 (setf info var-12))))
   (t
    (cond
     (wntsnn
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (chseqr "E" "N" n 1 n a lda w vr ldvr work -1 info)
        (declare
         (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10
          var-11))
        (when var-2 (setf n var-2))
        (when var-4 (setf n var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldvr var-9))
        (when var-12 (setf info var-12))))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (chseqr "S" "N" n 1 n a lda w vr ldvr work -1 info)
        (declare
         (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10
          var-11))
        (when var-2 (setf n var-2))
        (when var-4 (setf n var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldvr var-9))
        (when var-12 (setf info var-12)))))))
  (setf hswork
          (f2cl-lib:int
```

```
            (f2cl-lib:fref work-%data% (1) ((1 *))
                           work-%offset%)))
    (cond
     ((and (not wantvl) (not wantvr))
      (setf minwrk (f2cl-lib:int-mul 2 n))
      (if (not (or wntsnn wntsne))
          (setf minwrk
                  (max (the f2cl-lib:integer4 minwrk)
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add
                             (f2cl-lib:int-mul n n)
                             (f2cl-lib:int-mul 2 n))))))
      (setf maxwrk
              (max (the f2cl-lib:integer4 maxwrk)
                   (the f2cl-lib:integer4 hswork)))
      (if (not (or wntsnn wntsne))
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add
                             (f2cl-lib:int-mul n n)
                             (f2cl-lib:int-mul 2 n)))))))
     (t (setf minwrk (f2cl-lib:int-mul 2 n))
      (if (not (or wntsnn wntsne))
          (setf minwrk
                  (max (the f2cl-lib:integer4 minwrk)
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add
                             (f2cl-lib:int-mul n n)
                             (f2cl-lib:int-mul 2 n))))))
      (setf maxwrk
              (max (the f2cl-lib:integer4 maxwrk)
                   (the f2cl-lib:integer4 hswork)))
      (setf maxwrk
              (max (the f2cl-lib:integer4 maxwrk)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add n
                                          (f2cl-lib:int-mul
                                           (f2cl-lib:int-sub n 1)
                                           (multiple-value-bind
                                               (ret-val var-0
                                                var-1 var-2 var-3
                                                var-4 var-5 var-6)
                                               (ilaenv 1 "CUNGHR"
                                                " " n 1 n -1)
                                             (declare
                                              (ignore var-0 var-1
                                               var-2 var-4 var-6))
                                             (when var-3
                                               (setf n var-3))
```

```
                                                    (when var-5
                                                      (setf n var-5))
                                                    ret-val))))))
            (if (not (or wntsnn wntsne))
                (setf maxwrk
                        (max (the f2cl-lib:integer4 maxwrk)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-mul n n)
                                   (f2cl-lib:int-mul 2 n))))))
            (setf maxwrk
                    (max (the f2cl-lib:integer4 maxwrk)
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-mul 2 n))))))
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4 minwrk)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce maxwrk 'f2cl-lib:complex8))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -20)))))
(cond
 ((/= info 0) (xerbla "CGEEVX" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(setf eps (slamch "P"))
(setf smlnum (slamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (slabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf icond 0)
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "M" n n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf scalea f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf scalea f2cl-lib:%true%)
  (setf cscale smlnum))
 ((> anrm bignum) (setf scalea f2cl-lib:%true%) (setf cscale bignum)))
```

```
(if scalea
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (clascl "G" 0 0 anrm cscale n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf cscale var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgebal balanc n a lda ilo ihi scale ierr)
  (declare (ignore var-1 var-2 var-6))
  (setf balanc var-0)
  (setf lda var-3)
  (setf ilo var-4)
  (setf ihi var-5)
  (setf ierr var-7))
(setf abnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "1" n n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(cond
 (scalea (setf (f2cl-lib:fref dum (1) ((1 1))) abnrm)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 cscale anrm 1 1 dum 1 ierr)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
    (when var-3 (setf cscale var-3))
    (when var-4 (setf anrm var-4))
    (when var-9 (setf ierr var-9)))
  (setf abnrm (f2cl-lib:fref dum (1) ((1 1))))))
(setf itau 1)
(setf iwrk (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cgehrd n ilo ihi a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
```

```
    (when var-1 (setf ilo var-1))
    (when var-2 (setf ihi var-2))
    (when var-4 (setf lda var-4))
    (when var-8 (setf ierr var-8)))
  (cond
   (wantvl (f2cl-lib:f2cl-set-string side "L" (string 1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (clacpy "L" n n a lda vl ldvl)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-6 (setf ldvl var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (cunghr n ilo ihi vl ldvl
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
      (declare (ignore var-3 var-5 var-6 var-7))
      (when var-0 (setf n var-0))
      (when var-1 (setf ilo var-1))
      (when var-2 (setf ihi var-2))
      (when var-4 (setf ldvl var-4))
      (when var-8 (setf ierr var-8)))
    (setf iwrk itau)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (chseqr "S" "V" n ilo ihi a lda w vl ldvl
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf n var-2))
      (when var-3 (setf ilo var-3))
      (when var-4 (setf ihi var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldvl var-9))
      (when var-12 (setf info var-12)))
    (cond
     (wantvr (f2cl-lib:f2cl-set-string side "B" (string 1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (clacpy "F" n n vl ldvl vr ldvr)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf ldvl var-4))
```

```
      (when var-6 (setf ldvr var-6))))))
 (wantvr (f2cl-lib:f2cl-set-string side "R" (string 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "L" n n a lda vr ldvr)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvr var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (cunghr n ilo ihi vr ldvr
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf n var-0))
    (when var-1 (setf ilo var-1))
    (when var-2 (setf ihi var-2))
    (when var-4 (setf ldvr var-4))
    (when var-8 (setf ierr var-8)))
  (setf iwrk itau)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (chseqr "S" "V" n ilo ihi a lda w vr ldvr
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldvr var-9))
    (when var-12 (setf info var-12))))
(t
 (cond (wntsnn (f2cl-lib:f2cl-set-string job "E" (string 1)))
       (t (f2cl-lib:f2cl-set-string job "S" (string 1))))
 (setf iwrk itau)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12)
     (chseqr job "N" n ilo ihi a lda w vr ldvr
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
   (declare (ignore var-1 var-5 var-7 var-8 var-10 var-11))
```

```
    (when var-0 (setf job var-0))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldvr var-9))
    (when var-12 (setf info var-12)))))
(if (> info 0)
    (go label50))
(cond
 ((or wantvl wantvr)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (ctrevc side "B" select n a lda vl ldvl vr ldvr n nout
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                             ((1 *)) work-%offset%)
       rwork ierr)
    (declare (ignore var-1 var-2 var-4 var-6 var-8 var-12 var-13))
    (when var-0 (setf side var-0))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldvl var-7))
    (when var-9 (setf ldvr var-9))
    (when var-10 (setf n var-10))
    (when var-11 (setf nout var-11))
    (when var-14 (setf ierr var-14)))))
(cond
 ((not wntsnn)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17)
      (ctrsna sense "A" select n a lda vl ldvl vr ldvr rconde rcondv n
       nout
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                             ((1 *)) work-%offset%)
       n rwork icond)
    (declare
     (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-11 var-14
      var-16))
    (when var-0 (setf sense var-0))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldvl var-7))
    (when var-9 (setf ldvr var-9))
    (when var-12 (setf n var-12))
    (when var-13 (setf nout var-13))
    (when var-15 (setf n var-15))
    (when var-17 (setf icond var-17)))))
(cond
```

```
(wantvl
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
     (cgebak balanc "L" n ilo ihi scale n vl ldvl ierr)
   (declare (ignore var-1 var-2 var-3 var-4 var-5 var-7))
   (setf balanc var-0)
   (setf n var-6)
   (setf ldvl var-8)
   (setf ierr var-9))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf scl
             (/ one
                (multiple-value-bind (ret-val var-0 var-1 var-2)
                    (scnrm2 n
                     (f2cl-lib:array-slice vl-%data% f2cl-lib:complex8
                                           (1 i) ((1 ldvl) (1 *))
                                           vl-%offset%)
                     1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf n var-0))
                  ret-val)))
     (multiple-value-bind (var-0 var-1 var-2 var-3)
         (csscal n scl
          (f2cl-lib:array-slice vl-%data% f2cl-lib:complex8 (1 i)
                                ((1 ldvl) (1 *)) vl-%offset%)
          1)
       (declare (ignore var-2 var-3))
       (when var-0 (setf n var-0))
       (when var-1 (setf scl var-1)))
     (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                   ((> k n) nil)
       (tagbody
         (setf (f2cl-lib:fref rwork-%data% (k) ((1 *)) rwork-%offset%)
                 (+
                  (expt
                   (f2cl-lib:freal
                    (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                                   vl-%offset%))
                   2)
                  (expt
                   (f2cl-lib:aimag
                    (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                                   vl-%offset%))
                   2)))
        label10))
     (setf k
             (multiple-value-bind (ret-val var-0 var-1 var-2)
                 (isamax n rwork 1)
```

```
                 (declare (ignore var-1 var-2))
                 (when var-0 (setf n var-0))
                 ret-val))
        (setf tmp
                (/
                 (f2cl-lib:conjg
                  (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                                 vl-%offset%))
                 (f2cl-lib:fsqrt
                  (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                 rwork-%offset%))))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (cscal n tmp
             (f2cl-lib:array-slice vl-%data% f2cl-lib:complex8 (1 i)
                                   ((1 ldvl) (1 *)) vl-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-1 (setf tmp var-1)))
        (setf (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                             vl-%offset%)
                (f2cl-lib:cmplx
                 (f2cl-lib:freal
                  (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                                 vl-%offset%))
                 zero))
      label20))))
  (cond
   (wantvr
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (cgebak balanc "R" n ilo ihi scale n vr ldvr ierr)
      (declare (ignore var-1 var-2 var-3 var-4 var-5 var-7))
      (setf balanc var-0)
      (setf n var-6)
      (setf ldvr var-8)
      (setf ierr var-9))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf scl
                (/ one
                   (multiple-value-bind (ret-val var-0 var-1 var-2)
                       (scnrm2 n
                        (f2cl-lib:array-slice vr-%data% f2cl-lib:complex8
                                              (1 i) ((1 ldvr) (1 *))
                                              vr-%offset%)
                        1)
                     (declare (ignore var-1 var-2))
                     (when var-0 (setf n var-0))
```

```
             ret-val)))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (csscal n scl
     (f2cl-lib:array-slice vr-%data% f2cl-lib:complex8 (1 i)
                           ((1 ldvr) (1 *)) vr-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf scl var-1)))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (k) ((1 *)) rwork-%offset%)
            (+
             (expt
              (f2cl-lib:freal
               (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                              vr-%offset%))
              2)
             (expt
              (f2cl-lib:aimag
               (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                              vr-%offset%))
              2)))
   label30))
(setf k
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (isamax n rwork 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(setf tmp
        (/
         (f2cl-lib:conjg
          (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                         vr-%offset%))
         (f2cl-lib:fsqrt
          (f2cl-lib:fref rwork-%data% (k) ((1 *))
                         rwork-%offset%))))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (cscal n tmp
     (f2cl-lib:array-slice vr-%data% f2cl-lib:complex8 (1 i)
                           ((1 ldvr) (1 *)) vr-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf tmp var-1)))
(setf (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                     vr-%offset%)
        (f2cl-lib:cmplx
```

```
                 (f2cl-lib:freal
                  (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                                 vr-%offset%))
                 zero))
      label40))))
label50
 (cond
  (scalea
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (clascl "G" 0 0 cscale anrm (f2cl-lib:int-sub n info) 1
        (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 ((+ info 1))
                              ((1 *)) w-%offset%)
        (max (the f2cl-lib:integer4 (f2cl-lib:int-sub n info))
             (the f2cl-lib:integer4 1))
        ierr)
     (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
     (when var-3 (setf cscale var-3))
     (when var-4 (setf anrm var-4))
     (when var-9 (setf ierr var-9)))
   (cond
    ((= info 0)
     (if (and (or wntsnv wntsnb) (= icond 0))
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9)
             (slascl "G" 0 0 cscale anrm n 1 rcondv n ierr)
           (declare (ignore var-0 var-1 var-2 var-6 var-7))
           (when var-3 (setf cscale var-3))
           (when var-4 (setf anrm var-4))
           (when var-5 (setf n var-5))
           (when var-8 (setf n var-8))
           (when var-9 (setf ierr var-9)))))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (clascl "G" 0 0 cscale anrm (f2cl-lib:int-sub ilo 1) 1 w n
          ierr)
       (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
       (when var-3 (setf cscale var-3))
       (when var-4 (setf anrm var-4))
       (when var-8 (setf n var-8))
       (when var-9 (setf ierr var-9)))))))
 (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
         (coerce maxwrk 'f2cl-lib:complex8))
 (go end_label)
end_label
 (return
  (values balanc jobvl jobvr sense n nil lda nil nil ldvl nil ldvr ilo
          ihi nil abnrm nil nil nil nil nil info))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgeevx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (single-float)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::balanc
                                              fortran-to-lisp::jobvl
                                              fortran-to-lisp::jobvr
                                              fortran-to-lisp::sense
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::ldvl nil
                                              fortran-to-lisp::ldvr
                                              fortran-to-lisp::ilo
                                              fortran-to-lisp::ihi nil
                                              fortran-to-lisp::abnrm nil nil
                                              nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::cgebak
                                              fortran-to-lisp::cgebal))))
```

———-

26.5 cgegs - DEPRECATED use cgges

26.6 cgegv - DEPRECATED use cggev

26.7 cgges

26.7.1 Axiom unit tests

— **cgges.input** —

```
)set break resume
)sys rm -f cgges.output
)spool cgges.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

26.7.2 Axiom help page

— **cgges.help** —

```
======================================================================
cgges examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgges (JOBVSL, JOBVSR, SORT, SELCTG, N, A, LDA, B, LDB,
           SDIM, ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, WORK, LWORK, RWORK,
```

```
          BWORK, INFO)
           CGGES computes the eigenvalues, the Schur form, and, optionally,
          the matrix of Schur vectors for GE matrices

Function/Subroutine Documentation
   subroutine cgges (character JOBVSL, character JOBVSR, character SORT,
       external SELCTG, integer N, complex, dimension( lda, * ) A, integer
       LDA, complex, dimension( ldb, * ) B, integer LDB, integer SDIM,
       complex, dimension( * ) ALPHA, complex, dimension( * ) BETA, complex,
       dimension( ldvsl, * ) VSL, integer LDVSL, complex, dimension( ldvsr, *
       ) VSR, integer LDVSR, complex, dimension( * ) WORK, integer LWORK,
       real, dimension( * ) RWORK, logical, dimension( * ) BWORK, integer
       INFO)
        CGGES computes the eigenvalues, the Schur form, and, optionally, the
       matrix of Schur vectors for GE matrices

Purpose:
   CGGES computes for a pair of N-by-N complex nonsymmetric matrices
   (A,B), the generalized eigenvalues, the generalized complex Schur
   form (S, T), and optionally left and/or right Schur vectors (VSL
   and VSR). This gives the generalized Schur factorization

      (A,B) = ( (VSL)*S*(VSR)**H, (VSL)*T*(VSR)**H )

   where (VSR)**H is the conjugate-transpose of VSR.

   Optionally, it also orders the eigenvalues so that a selected cluster
   of eigenvalues appears in the leading diagonal blocks of the upper
   triangular matrix S and the upper triangular matrix T. The leading
   columns of VSL and VSR then form an unitary basis for the
   corresponding left and right eigenspaces (deflating subspaces).

   (If only the generalized eigenvalues are needed, use the driver
   CGGEV instead, which is faster.)

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar w
   or a ratio alpha/beta = w, such that  A - w*B is singular.  It is
   usually represented as the pair (alpha,beta), as there is a
   reasonable interpretation for beta=0, and even for both being zero.

   A pair of matrices (S,T) is in generalized complex Schur form if S
   and T are upper triangular and, in addition, the diagonal elements
   of T are non-negative real numbers.

Parameters:

 JOBVSL
   JOBVSL is CHARACTER*1
   = 'N':  do not compute the left Schur vectors;
   = 'V':  compute the left Schur vectors.
```

```
JOBVSR
  JOBVSR is CHARACTER*1
  = 'N':  do not compute the right Schur vectors;
  = 'V':  compute the right Schur vectors.

SORT
  SORT is CHARACTER*1
  Specifies whether or not to order the eigenvalues on the
  diagonal of the generalized Schur form.
  = 'N':  Eigenvalues are not ordered;
  = 'S':  Eigenvalues are ordered (see SELCTG).

SELCTG
  SELCTG is a LOGICAL FUNCTION of two COMPLEX arguments
  SELCTG must be declared EXTERNAL in the calling subroutine.
  If SORT = 'N', SELCTG is not referenced.
  If SORT = 'S', SELCTG is used to select eigenvalues to sort
  to the top left of the Schur form.
  An eigenvalue ALPHA(j)/BETA(j) is selected if
  SELCTG(ALPHA(j),BETA(j)) is true.

  Note that a selected complex eigenvalue may no longer satisfy
  SELCTG(ALPHA(j),BETA(j)) = .TRUE. after ordering, since
  ordering may change the value of complex eigenvalues
  (especially if the eigenvalue is ill-conditioned), in this
  case INFO is set to N+2 (See INFO below).

N
  N is INTEGER
  The order of the matrices A, B, VSL, and VSR.  N >= 0.

A
  A is COMPLEX array, dimension (LDA, N)
  On entry, the first of the pair of matrices.
  On exit, A has been overwritten by its generalized Schur form S.

LDA
  LDA is INTEGER
  The leading dimension of A.  LDA >= max(1,N).

B
  B is COMPLEX array, dimension (LDB, N)
  On entry, the second of the pair of matrices.
  On exit, B has been overwritten by its generalized Schur form T.

LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).
```

```
SDIM
  SDIM is INTEGER
  If SORT = 'N', SDIM = 0.
  If SORT = 'S', SDIM = number of eigenvalues (after sorting)
  for which SELCTG is true.

ALPHA
  ALPHA is COMPLEX array, dimension (N)

BETA
  BETA is COMPLEX array, dimension (N)
  On exit,  ALPHA(j)/BETA(j), j=1,...,N, will be the
  generalized eigenvalues.  ALPHA(j), j=1,...,N  and  BETA(j),
  j=1,...,N  are the diagonals of the complex Schur form (A,B)
  output by CGGES. The  BETA(j) will be non-negative real.

  Note: the quotients ALPHA(j)/BETA(j) may easily over- or
  underflow, and BETA(j) may even be zero.  Thus, the user
  should avoid naively computing the ratio alpha/beta.
  However, ALPHA will be always less than and usually
  comparable with norm(A) in magnitude, and BETA always less
  than and usually comparable with norm(B).

VSL
  VSL is COMPLEX array, dimension (LDVSL,N)
  If JOBVSL = 'V', VSL will contain the left Schur vectors.
  Not referenced if JOBVSL = 'N'.

LDVSL
  LDVSL is INTEGER
  The leading dimension of the matrix VSL. LDVSL >= 1, and
  if JOBVSL = 'V', LDVSL >= N.

VSR
  VSR is COMPLEX array, dimension (LDVSR,N)
  If JOBVSR = 'V', VSR will contain the right Schur vectors.
  Not referenced if JOBVSR = 'N'.

LDVSR
  LDVSR is INTEGER
  The leading dimension of the matrix VSR. LDVSR >= 1, and
  if JOBVSR = 'V', LDVSR >= N.

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,2*N).
```

```
  For good performance, LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is REAL array, dimension (8*N)

BWORK
  BWORK is LOGICAL array, dimension (N)
  Not referenced if SORT = 'N'.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  =1,...,N:
     The QZ iteration failed.  (A,B) are not in Schur
     form, but ALPHA(j) and BETA(j) should be correct for
     j=INFO+1,...,N.
  > N:  =N+1: other than QZ iteration failed in CHGEQZ
        =N+2: after reordering, roundoff changed values of
              some complex eigenvalues so that leading
              eigenvalues in the Generalized Schur form no
              longer satisfy SELCTG=.TRUE.  This could also
              be caused due to scaling.
        =N+3: reordering failed in CTGSEN.
```

———-

26.7.3 fortran code

— **cgges.f** —

```
*  =====================================================================
      SUBROUTINE CGGES( JOBVSL, JOBVSR, SORT, SELCTG, N, A, LDA, B, LDB,
     $                  SDIM, ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, WORK,
     $                  LWORK, RWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
```

```
      CHARACTER          JOBVSL, JOBVSR, SORT
      INTEGER            INFO, LDA, LDB, LDVSL, LDVSR, LWORK, N, SDIM
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      REAL               RWORK( * )
      COMPLEX            A( LDA, * ), ALPHA( * ), B( LDB, * ),
     $                   BETA( * ), VSL( LDVSL, * ), VSR( LDVSR, * ),
     $                   WORK( * )
*     ..
*     .. Function Arguments ..
      LOGICAL            SELCTG
      EXTERNAL           SELCTG
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
      COMPLEX            CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0E0, 0.0E0 ),
     $                   CONE = ( 1.0E0, 0.0E0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            CURSL, ILASCL, ILBSCL, ILVSL, ILVSR, LASTSL,
     $                   LQUERY, WANTST
      INTEGER            I, ICOLS, IERR, IHI, IJOBVL, IJOBVR, ILEFT,
     $                   ILO, IRIGHT, IROWS, IRWRK, ITAU, IWRK, LWKMIN,
     $                   LWKOPT
      REAL               ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS, PVSL,
     $                   PVSR, SMLNUM
*     ..
*     .. Local Arrays ..
      INTEGER            IDUM( 1 )
      REAL               DIF( 2 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEQRF, CGGBAK, CGGBAL, CGGHRD, CHGEQZ, CLACPY,
     $                   CLASCL, CLASET, CTGSEN, CUNGQR, CUNMQR, SLABAD,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               CLANGE, SLAMCH
      EXTERNAL           LSAME, ILAENV, CLANGE, SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
```

```
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVSL, 'N' ) ) THEN
         IJOBVL = 1
         ILVSL = .FALSE.
      ELSE IF( LSAME( JOBVSL, 'V' ) ) THEN
         IJOBVL = 2
         ILVSL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVSL = .FALSE.
      END IF
*
      IF( LSAME( JOBVSR, 'N' ) ) THEN
         IJOBVR = 1
         ILVSR = .FALSE.
      ELSE IF( LSAME( JOBVSR, 'V' ) ) THEN
         IJOBVR = 2
         ILVSR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVSR = .FALSE.
      END IF
*
      WANTST = LSAME( SORT, 'S' )
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -2
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDVSL.LT.1 .OR. ( ILVSL .AND. LDVSL.LT.N ) ) THEN
         INFO = -14
      ELSE IF( LDVSR.LT.1 .OR. ( ILVSR .AND. LDVSR.LT.N ) ) THEN
         INFO = -16
      END IF
```

```
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         LWKMIN = MAX( 1, 2*N )
         LWKOPT = MAX( 1, N + N*ILAENV( 1, 'CGEQRF', ' ', N, 1, N, 0 ) )
         LWKOPT = MAX( LWKOPT, N +
     $                 N*ILAENV( 1, 'CUNMQR', ' ', N, 1, N, -1 ) )
         IF( ILVSL ) THEN
            LWKOPT = MAX( LWKOPT, N +
     $                    N*ILAENV( 1, 'CUNGQR', ' ', N, 1, N, -1 ) )
         END IF
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.LWKMIN .AND. .NOT.LQUERY )
     $      INFO = -18
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGGES ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = CLANGE( 'M', N, N, A, LDA, RWORK )
      ILASCL = .FALSE.
```

```
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
*
      IF( ILASCL )
     $   CALL CLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = CLANGE( 'M', N, N, B, LDB, RWORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
*
      IF( ILBSCL )
     $   CALL CLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrix to make it more nearly triangular
*     (Real Workspace: need 6*N)
*
      ILEFT = 1
      IRIGHT = N + 1
      IRWRK = IRIGHT + N
      CALL CGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, RWORK( ILEFT ),
     $             RWORK( IRIGHT ), RWORK( IRWRK ), IERR )
*
*     Reduce B to triangular form (QR decomposition of B)
*     (Complex Workspace: need N, prefer N*NB)
*
      IROWS = IHI + 1 - ILO
      ICOLS = N + 1 - ILO
      ITAU = 1
      IWRK = ITAU + IROWS
      CALL CGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to matrix A
*     (Complex Workspace: need N, prefer N*NB)
*
      CALL CUNMQR( 'L', 'C', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
```

```
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VSL
*     (Complex Workspace: need N, prefer N*NB)
*
      IF( ILVSL ) THEN
         CALL CLASET( 'Full', N, N, CZERO, CONE, VSL, LDVSL )
         IF( IROWS.GT.1 ) THEN
            CALL CLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VSL( ILO+1, ILO ), LDVSL )
         END IF
         CALL CUNGQR( IROWS, IROWS, IROWS, VSL( ILO, ILO ), LDVSL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Initialize VSR
*
      IF( ILVSR )
     $   CALL CLASET( 'Full', N, N, CZERO, CONE, VSR, LDVSR )
*
*     Reduce to generalized Hessenberg form
*     (Workspace: none needed)
*
      CALL CGGHRD( JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB, VSL,
     $             LDVSL, VSR, LDVSR, IERR )
*
      SDIM = 0
*
*     Perform QZ algorithm, computing Schur vectors if desired
*     (Complex Workspace: need N)
*     (Real Workspace: need N)
*
      IWRK = ITAU
      CALL CHGEQZ( 'S', JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, WORK( IWRK ),
     $             LWORK+1-IWRK, RWORK( IRWRK ), IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 30
      END IF
*
*     Sort eigenvalues ALPHA/BETA if desired
*     (Workspace: none needed)
*
```

```
      IF( WANTST ) THEN
*
*        Undo scaling on eigenvalues before selecting
*
         IF( ILASCL )
     $      CALL CLASCL( 'G', 0, 0, ANRM, ANRMTO, N, 1, ALPHA, N, IERR )
         IF( ILBSCL )
     $      CALL CLASCL( 'G', 0, 0, BNRM, BNRMTO, N, 1, BETA, N, IERR )
*
*        Select eigenvalues
*
         DO 10 I = 1, N
            BWORK( I ) = SELCTG( ALPHA( I ), BETA( I ) )
   10    CONTINUE
*
         CALL CTGSEN( 0, ILVSL, ILVSR, BWORK, N, A, LDA, B, LDB, ALPHA,
     $                BETA, VSL, LDVSL, VSR, LDVSR, SDIM, PVSL, PVSR,
     $                DIF, WORK( IWRK ), LWORK-IWRK+1, IDUM, 1, IERR )
         IF( IERR.EQ.1 )
     $      INFO = N + 3
*
      END IF
*
*     Apply back-permutation to VSL and VSR
*     (Workspace: none needed)
*
      IF( ILVSL )
     $   CALL CGGBAK( 'P', 'L', N, ILO, IHI, RWORK( ILEFT ),
     $                RWORK( IRIGHT ), N, VSL, LDVSL, IERR )
      IF( ILVSR )
     $   CALL CGGBAK( 'P', 'R', N, ILO, IHI, RWORK( ILEFT ),
     $                RWORK( IRIGHT ), N, VSR, LDVSR, IERR )
*
*     Undo scaling
*
      IF( ILASCL ) THEN
         CALL CLASCL( 'U', 0, 0, ANRMTO, ANRM, N, N, A, LDA, IERR )
         CALL CLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHA, N, IERR )
      END IF
*
      IF( ILBSCL ) THEN
         CALL CLASCL( 'U', 0, 0, BNRMTO, BNRM, N, N, B, LDB, IERR )
         CALL CLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
      END IF
*
      IF( WANTST ) THEN
*
*        Check if reordering is correct
*
         LASTSL = .TRUE.
```

```
         SDIM = 0
         DO 20 I = 1, N
            CURSL = SELCTG( ALPHA( I ), BETA( I ) )
            IF( CURSL )
     $         SDIM = SDIM + 1
            IF( CURSL .AND. .NOT.LASTSL )
     $         INFO = N + 2
            LASTSL = CURSL
   20    CONTINUE
*
      END IF
*
   30 CONTINUE
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of CGGES
*
      END
```

———-

26.7.4 lisp code

— **cgges.lisp** —

———-

26.8 cgges3

26.8.1 Axiom unit tests

— **cgges3.input** —

```
)set break resume
)sys rm -f cgges3.output
)spool cgges3.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

26.8.2 Axiom help page

— **cgges3.help** —

```
=====================================================================
cgges3 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgges3 (JOBVSL, JOBVSR, SORT, SELCTG, N, A, LDA, B, LDB,
           SDIM, ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, WORK, LWORK, RWORK,
           BWORK, INFO)
            CGGES3 computes the eigenvalues, the Schur form, and, optionally,
           the matrix of Schur vectors for GE matrices (blocked algorithm)

Function/Subroutine Documentation
   subroutine cgges3 (character JOBVSL, character JOBVSR, character SORT,
       external SELCTG, integer N, complex, dimension( lda, * ) A, integer
       LDA, complex, dimension( ldb, * ) B, integer LDB, integer SDIM,
       complex, dimension( * ) ALPHA, complex, dimension( * ) BETA, complex,
       dimension( ldvsl, * ) VSL, integer LDVSL, complex, dimension( ldvsr, *
       ) VSR, integer LDVSR, complex, dimension( * ) WORK, integer LWORK,
       real, dimension( * ) RWORK, logical, dimension( * ) BWORK, integer
       INFO)
        CGGES3 computes the eigenvalues, the Schur form, and, optionally, the
       matrix of Schur vectors for GE matrices (blocked algorithm)

Purpose:
   CGGES3 computes for a pair of N-by-N complex nonsymmetric matrices
   (A,B), the generalized eigenvalues, the generalized complex Schur
   form (S, T), and optionally left and/or right Schur vectors (VSL
   and VSR). This gives the generalized Schur factorization

      (A,B) = ( (VSL)*S*(VSR)**H, (VSL)*T*(VSR)**H )

   where (VSR)**H is the conjugate-transpose of VSR.
```

```
   Optionally, it also orders the eigenvalues so that a selected cluster
   of eigenvalues appears in the leading diagonal blocks of the upper
   triangular matrix S and the upper triangular matrix T. The leading
   columns of VSL and VSR then form an unitary basis for the
   corresponding left and right eigenspaces (deflating subspaces).

   (If only the generalized eigenvalues are needed, use the driver
   CGGEV instead, which is faster.)

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar w
   or a ratio alpha/beta = w, such that  A - w*B is singular.  It is
   usually represented as the pair (alpha,beta), as there is a
   reasonable interpretation for beta=0, and even for both being zero.

   A pair of matrices (S,T) is in generalized complex Schur form if S
   and T are upper triangular and, in addition, the diagonal elements
   of T are non-negative real numbers.

Parameters:

 JOBVSL
   JOBVSL is CHARACTER*1
   = 'N':  do not compute the left Schur vectors;
   = 'V':  compute the left Schur vectors.

 JOBVSR
   JOBVSR is CHARACTER*1
   = 'N':  do not compute the right Schur vectors;
   = 'V':  compute the right Schur vectors.

 SORT
   SORT is CHARACTER*1
   Specifies whether or not to order the eigenvalues on the
   diagonal of the generalized Schur form.
   = 'N':  Eigenvalues are not ordered;
   = 'S':  Eigenvalues are ordered (see SELCTG).

 SELCTG
   SELCTG is a LOGICAL FUNCTION of two COMPLEX arguments
   SELCTG must be declared EXTERNAL in the calling subroutine.
   If SORT = 'N', SELCTG is not referenced.
   If SORT = 'S', SELCTG is used to select eigenvalues to sort
   to the top left of the Schur form.
   An eigenvalue ALPHA(j)/BETA(j) is selected if
   SELCTG(ALPHA(j),BETA(j)) is true.

   Note that a selected complex eigenvalue may no longer satisfy
   SELCTG(ALPHA(j),BETA(j)) = .TRUE. after ordering, since
   ordering may change the value of complex eigenvalues
   (especially if the eigenvalue is ill-conditioned), in this
```

```
  case INFO is set to N+2 (See INFO below).

N
  N is INTEGER
  The order of the matrices A, B, VSL, and VSR.  N >= 0.

A
  A is COMPLEX array, dimension (LDA, N)
  On entry, the first of the pair of matrices.
  On exit, A has been overwritten by its generalized Schur form S.

LDA
  LDA is INTEGER
  The leading dimension of A.  LDA >= max(1,N).

B
  B is COMPLEX array, dimension (LDB, N)
  On entry, the second of the pair of matrices.
  On exit, B has been overwritten by its generalized Schur form T.

LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).

SDIM
  SDIM is INTEGER
  If SORT = 'N', SDIM = 0.
  If SORT = 'S', SDIM = number of eigenvalues (after sorting)
  for which SELCTG is true.

ALPHA
  ALPHA is COMPLEX array, dimension (N)

BETA
  BETA is COMPLEX array, dimension (N)
  On exit,  ALPHA(j)/BETA(j), j=1,...,N, will be the
  generalized eigenvalues.  ALPHA(j), j=1,...,N  and  BETA(j),
  j=1,...,N  are the diagonals of the complex Schur form (A,B)
  output by CGGES3. The  BETA(j) will be non-negative real.

  Note: the quotients ALPHA(j)/BETA(j) may easily over- or
  underflow, and BETA(j) may even be zero.  Thus, the user
  should avoid naively computing the ratio alpha/beta.
  However, ALPHA will be always less than and usually
  comparable with norm(A) in magnitude, and BETA always less
  than and usually comparable with norm(B).

VSL
  VSL is COMPLEX array, dimension (LDVSL,N)
  If JOBVSL = 'V', VSL will contain the left Schur vectors.
```

```
  Not referenced if JOBVSL = 'N'.

LDVSL
  LDVSL is INTEGER
  The leading dimension of the matrix VSL. LDVSL >= 1, and
  if JOBVSL = 'V', LDVSL >= N.

VSR
  VSR is COMPLEX array, dimension (LDVSR,N)
  If JOBVSR = 'V', VSR will contain the right Schur vectors.
  Not referenced if JOBVSR = 'N'.

LDVSR
  LDVSR is INTEGER
  The leading dimension of the matrix VSR. LDVSR >= 1, and
  if JOBVSR = 'V', LDVSR >= N.

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is REAL array, dimension (8*N)

BWORK
  BWORK is LOGICAL array, dimension (N)
  Not referenced if SORT = 'N'.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  =1,...,N:
        The QZ iteration failed.  (A,B) are not in Schur
        form, but ALPHA(j) and BETA(j) should be correct for
        j=INFO+1,...,N.
  > N:  =N+1: other than QZ iteration failed in CHGEQZ
        =N+2: after reordering, roundoff changed values of
              some complex eigenvalues so that leading
              eigenvalues in the Generalized Schur form no
              longer satisfy SELCTG=.TRUE.  This could also
```

```
            be caused due to scaling.
        =N+3: reordering failed in CTGSEN.
```

———-

26.8.3 fortran code

— **cgges3.f** —

```
*  =====================================================================
      SUBROUTINE CGGES3( JOBVSL, JOBVSR, SORT, SELCTG, N, A, LDA, B,
     $                   LDB, SDIM, ALPHA, BETA, VSL, LDVSL, VSR, LDVSR,
     $                   WORK, LWORK, RWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     January 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVSL, JOBVSR, SORT
      INTEGER            INFO, LDA, LDB, LDVSL, LDVSR, LWORK, N, SDIM
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      REAL               RWORK( * )
      COMPLEX            A( LDA, * ), ALPHA( * ), B( LDB, * ),
     $                   BETA( * ), VSL( LDVSL, * ), VSR( LDVSR, * ),
     $                   WORK( * )
*     ..
*     .. Function Arguments ..
      LOGICAL            SELCTG
      EXTERNAL           SELCTG
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
      COMPLEX            CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0E0, 0.0E0 ),
     $                   CONE = ( 1.0E0, 0.0E0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            CURSL, ILASCL, ILBSCL, ILVSL, ILVSR, LASTSL,
     $                   LQUERY, WANTST
```

```
      INTEGER            I, ICOLS, IERR, IHI, IJOBVL, IJOBVR, ILEFT,
     $                   ILO, IRIGHT, IROWS, IRWRK, ITAU, IWRK, LWKOPT
      REAL               ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS, PVSL,
     $                   PVSR, SMLNUM
*     ..
*     .. Local Arrays ..
      INTEGER            IDUM( 1 )
      REAL               DIF( 2 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEQRF, CGGBAK, CGGBAL, CGGHD3, CHGEQZ, CLACPY,
     $                   CLASCL, CLASET, CTGSEN, CUNGQR, CUNMQR, SLABAD,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               CLANGE, SLAMCH
      EXTERNAL           LSAME, CLANGE, SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVSL, 'N' ) ) THEN
         IJOBVL = 1
         ILVSL = .FALSE.
      ELSE IF( LSAME( JOBVSL, 'V' ) ) THEN
         IJOBVL = 2
         ILVSL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVSL = .FALSE.
      END IF
*
      IF( LSAME( JOBVSR, 'N' ) ) THEN
         IJOBVR = 1
         ILVSR = .FALSE.
      ELSE IF( LSAME( JOBVSR, 'V' ) ) THEN
         IJOBVR = 2
         ILVSR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVSR = .FALSE.
      END IF
*
      WANTST = LSAME( SORT, 'S' )
*
```

```
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -2
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDVSL.LT.1 .OR. ( ILVSL .AND. LDVSL.LT.N ) ) THEN
         INFO = -14
      ELSE IF( LDVSR.LT.1 .OR. ( ILVSR .AND. LDVSR.LT.N ) ) THEN
         INFO = -16
      ELSE IF( LWORK.LT.MAX( 1, 2*N ) .AND. .NOT.LQUERY ) THEN
         INFO = -18
      END IF
*
*     Compute workspace
*
      IF( INFO.EQ.0 ) THEN
         CALL CGEQRF( N, N, B, LDB, WORK, WORK, -1, IERR )
         LWKOPT = MAX( 1,  N + INT ( WORK( 1 ) ) )
         CALL CUNMQR( 'L', 'C', N, N, N, B, LDB, WORK, A, LDA, WORK,
     $                -1, IERR )
         LWKOPT = MAX( LWKOPT, N + INT ( WORK( 1 ) ) )
         IF( ILVSL ) THEN
            CALL CUNGQR( N, N, N, VSL, LDVSL, WORK, WORK, -1,
     $                   IERR )
            LWKOPT = MAX( LWKOPT, N + INT ( WORK( 1 ) ) )
         END IF
         CALL CGGHD3( JOBVSL, JOBVSR, N, 1, N, A, LDA, B, LDB, VSL,
     $                LDVSL, VSR, LDVSR, WORK, -1, IERR )
         LWKOPT = MAX( LWKOPT, N + INT ( WORK( 1 ) ) )
         CALL CHGEQZ( 'S', JOBVSL, JOBVSR, N, 1, N, A, LDA, B, LDB,
     $                ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, WORK, -1,
     $                WORK, IERR )
         LWKOPT = MAX( LWKOPT, INT ( WORK( 1 ) ) )
         IF( WANTST ) THEN
            CALL CTGSEN( 0, ILVSL, ILVSR, BWORK, N, A, LDA, B, LDB,
     $                   ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, SDIM,
     $                   PVSL, PVSR, DIF, WORK, -1, IDUM, 1, IERR )
            LWKOPT = MAX( LWKOPT, INT ( WORK( 1 ) ) )
         END IF
```

```
         WORK( 1 ) = CMPLX( LWKOPT )
      END IF

*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGGES3 ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = CLANGE( 'M', N, N, A, LDA, RWORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
*
      IF( ILASCL )
     $   CALL CLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = CLANGE( 'M', N, N, B, LDB, RWORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
```

```
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
*
      IF( ILBSCL )
     $   CALL CLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrix to make it more nearly triangular
*
      ILEFT = 1
      IRIGHT = N + 1
      IRWRK = IRIGHT + N
      CALL CGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, RWORK( ILEFT ),
     $             RWORK( IRIGHT ), RWORK( IRWRK ), IERR )
*
*     Reduce B to triangular form (QR decomposition of B)
*
      IROWS = IHI + 1 - ILO
      ICOLS = N + 1 - ILO
      ITAU = 1
      IWRK = ITAU + IROWS
      CALL CGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to matrix A
*
      CALL CUNMQR( 'L', 'C', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VSL
*
      IF( ILVSL ) THEN
         CALL CLASET( 'Full', N, N, CZERO, CONE, VSL, LDVSL )
         IF( IROWS.GT.1 ) THEN
            CALL CLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VSL( ILO+1, ILO ), LDVSL )
         END IF
         CALL CUNGQR( IROWS, IROWS, IROWS, VSL( ILO, ILO ), LDVSL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Initialize VSR
*
      IF( ILVSR )
     $   CALL CLASET( 'Full', N, N, CZERO, CONE, VSR, LDVSR )
*
*     Reduce to generalized Hessenberg form
*
      CALL CGGHD3( JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB, VSL,
```

```
     $             LDVSL, VSR, LDVSR, WORK( IWRK ), LWORK+1-IWRK, IERR )
*
      SDIM = 0
*
*     Perform QZ algorithm, computing Schur vectors if desired
*
      IWRK = ITAU
      CALL CHGEQZ( 'S', JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, WORK( IWRK ),
     $             LWORK+1-IWRK, RWORK( IRWRK ), IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 30
      END IF
*
*     Sort eigenvalues ALPHA/BETA if desired
*
      IF( WANTST ) THEN
*
*        Undo scaling on eigenvalues before selecting
*
         IF( ILASCL )
     $      CALL CLASCL( 'G', 0, 0, ANRM, ANRMTO, N, 1, ALPHA, N, IERR )
         IF( ILBSCL )
     $      CALL CLASCL( 'G', 0, 0, BNRM, BNRMTO, N, 1, BETA, N, IERR )
*
*        Select eigenvalues
*
         DO 10 I = 1, N
            BWORK( I ) = SELCTG( ALPHA( I ), BETA( I ) )
   10    CONTINUE
*
         CALL CTGSEN( 0, ILVSL, ILVSR, BWORK, N, A, LDA, B, LDB, ALPHA,
     $                BETA, VSL, LDVSL, VSR, LDVSR, SDIM, PVSL, PVSR,
     $                DIF, WORK( IWRK ), LWORK-IWRK+1, IDUM, 1, IERR )
         IF( IERR.EQ.1 )
     $      INFO = N + 3
*
      END IF
*
*     Apply back-permutation to VSL and VSR
*
      IF( ILVSL )
     $   CALL CGGBAK( 'P', 'L', N, ILO, IHI, RWORK( ILEFT ),
```

```
     $                RWORK( IRIGHT ), N, VSL, LDVSL, IERR )
      IF( ILVSR )
     $   CALL CGGBAK( 'P', 'R', N, ILO, IHI, RWORK( ILEFT ),
     $                RWORK( IRIGHT ), N, VSR, LDVSR, IERR )
*
*     Undo scaling
*
      IF( ILASCL ) THEN
         CALL CLASCL( 'U', 0, 0, ANRMTO, ANRM, N, N, A, LDA, IERR )
         CALL CLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHA, N, IERR )
      END IF
*
      IF( ILBSCL ) THEN
         CALL CLASCL( 'U', 0, 0, BNRMTO, BNRM, N, N, B, LDB, IERR )
         CALL CLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
      END IF
*
      IF( WANTST ) THEN
*
*        Check if reordering is correct
*
         LASTSL = .TRUE.
         SDIM = 0
         DO 20 I = 1, N
            CURSL = SELCTG( ALPHA( I ), BETA( I ) )
            IF( CURSL )
     $         SDIM = SDIM + 1
            IF( CURSL .AND. .NOT.LASTSL )
     $         INFO = N + 2
            LASTSL = CURSL
   20    CONTINUE
*
      END IF
*
   30 CONTINUE
*
      WORK( 1 ) = CMPLX( LWKOPT )
*
      RETURN
*
*     End of CGGES3
*
      END
```

———-

26.8.4 lisp code

— **cgges3.lisp** —

———-

26.9 cggesx

26.9.1 Axiom unit tests

— **cggesx.input** —

```
)set break resume
)sys rm -f cggesx.output
)spool cggesx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

26.9.2 Axiom help page

— **cggesx.help** —

```
======================================================================
cggesx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cggesx (JOBVSL, JOBVSR, SORT, SELCTG, SENSE, N, A, LDA, B,
           LDB, SDIM, ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, RCONDE, RCONDV,
           WORK, LWORK, RWORK, IWORK, LIWORK, BWORK, INFO)
            CGGESX computes the eigenvalues, the Schur form, and, optionally,
           the matrix of Schur vectors for GE matrices
```

```
Function/Subroutine Documentation
   subroutine cggesx (character JOBVSL, character JOBVSR, character SORT,
       external SELCTG, character SENSE, integer N, complex, dimension( lda, *
       ) A, integer LDA, complex, dimension( ldb, * ) B, integer LDB, integer
       SDIM, complex, dimension( * ) ALPHA, complex, dimension( * ) BETA,
       complex, dimension( ldvsl, * ) VSL, integer LDVSL, complex, dimension(
       ldvsr, * ) VSR, integer LDVSR, real, dimension( 2 ) RCONDE, real,
       dimension( 2 ) RCONDV, complex, dimension( * ) WORK, integer LWORK,
       real, dimension( * ) RWORK, integer, dimension( * ) IWORK, integer
       LIWORK, logical, dimension( * ) BWORK, integer INFO)
        CGGESX computes the eigenvalues, the Schur form, and, optionally, the
       matrix of Schur vectors for GE matrices

Purpose:
   CGGESX computes for a pair of N-by-N complex nonsymmetric matrices
   (A,B), the generalized eigenvalues, the complex Schur form (S,T),
   and, optionally, the left and/or right matrices of Schur vectors (VSL
   and VSR).  This gives the generalized Schur factorization

     (A,B) = ( (VSL) S (VSR)**H, (VSL) T (VSR)**H )

   where (VSR)**H is the conjugate-transpose of VSR.

   Optionally, it also orders the eigenvalues so that a selected cluster
   of eigenvalues appears in the leading diagonal blocks of the upper
   triangular matrix S and the upper triangular matrix T; computes
   a reciprocal condition number for the average of the selected
   eigenvalues (RCONDE); and computes a reciprocal condition number for
   the right and left deflating subspaces corresponding to the selected
   eigenvalues (RCONDV). The leading columns of VSL and VSR then form
   an orthonormal basis for the corresponding left and right eigenspaces
   (deflating subspaces).

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar w
   or a ratio alpha/beta = w, such that  A - w*B is singular.  It is
   usually represented as the pair (alpha,beta), as there is a
   reasonable interpretation for beta=0 or for both being zero.

   A pair of matrices (S,T) is in generalized complex Schur form if T is
   upper triangular with non-negative diagonal and S is upper
   triangular.

Parameters:

 JOBVSL
   JOBVSL is CHARACTER*1
   = 'N':  do not compute the left Schur vectors;
   = 'V':  compute the left Schur vectors.

 JOBVSR
```

```
  JOBVSR is CHARACTER*1
  = 'N':  do not compute the right Schur vectors;
  = 'V':  compute the right Schur vectors.

SORT
  SORT is CHARACTER*1
  Specifies whether or not to order the eigenvalues on the
  diagonal of the generalized Schur form.
  = 'N':  Eigenvalues are not ordered;
  = 'S':  Eigenvalues are ordered (see SELCTG).

SELCTG
  SELCTG is procedure) LOGICAL FUNCTION of two COMPLEX arguments
  SELCTG must be declared EXTERNAL in the calling subroutine.
  If SORT = 'N', SELCTG is not referenced.
  If SORT = 'S', SELCTG is used to select eigenvalues to sort
  to the top left of the Schur form.
  Note that a selected complex eigenvalue may no longer satisfy
  SELCTG(ALPHA(j),BETA(j)) = .TRUE. after ordering, since
  ordering may change the value of complex eigenvalues
  (especially if the eigenvalue is ill-conditioned), in this
  case INFO is set to N+3 see INFO below).

SENSE
  SENSE is CHARACTER*1
  Determines which reciprocal condition numbers are computed.
  = 'N' : None are computed;
  = 'E' : Computed for average of selected eigenvalues only;
  = 'V' : Computed for selected deflating subspaces only;
  = 'B' : Computed for both.
  If SENSE = 'E', 'V', or 'B', SORT must equal 'S'.

N
  N is INTEGER
  The order of the matrices A, B, VSL, and VSR.  N >= 0.

A
  A is COMPLEX array, dimension (LDA, N)
  On entry, the first of the pair of matrices.
  On exit, A has been overwritten by its generalized Schur form S.

LDA
  LDA is INTEGER
  The leading dimension of A.  LDA >= max(1,N).

B
  B is COMPLEX array, dimension (LDB, N)
  On entry, the second of the pair of matrices.
  On exit, B has been overwritten by its generalized Schur form T.
```

```
LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).

SDIM
  SDIM is INTEGER
  If SORT = 'N', SDIM = 0.
  If SORT = 'S', SDIM = number of eigenvalues (after sorting)
  for which SELCTG is true.

ALPHA
  ALPHA is COMPLEX array, dimension (N)

BETA
  BETA is COMPLEX array, dimension (N)
  On exit, ALPHA(j)/BETA(j), j=1,...,N, will be the
  generalized eigenvalues.  ALPHA(j) and BETA(j),j=1,...,N  are
  the diagonals of the complex Schur form (S,T).  BETA(j) will
  be non-negative real.

  Note: the quotients ALPHA(j)/BETA(j) may easily over- or
  underflow, and BETA(j) may even be zero.  Thus, the user
  should avoid naively computing the ratio alpha/beta.
  However, ALPHA will be always less than and usually
  comparable with norm(A) in magnitude, and BETA always less
  than and usually comparable with norm(B).

VSL
  VSL is COMPLEX array, dimension (LDVSL,N)
  If JOBVSL = 'V', VSL will contain the left Schur vectors.
  Not referenced if JOBVSL = 'N'.

LDVSL
  LDVSL is INTEGER
  The leading dimension of the matrix VSL. LDVSL >=1, and
  if JOBVSL = 'V', LDVSL >= N.

VSR
  VSR is COMPLEX array, dimension (LDVSR,N)
  If JOBVSR = 'V', VSR will contain the right Schur vectors.
  Not referenced if JOBVSR = 'N'.

LDVSR
  LDVSR is INTEGER
  The leading dimension of the matrix VSR. LDVSR >= 1, and
  if JOBVSR = 'V', LDVSR >= N.

RCONDE
  RCONDE is REAL array, dimension ( 2 )
  If SENSE = 'E' or 'B', RCONDE(1) and RCONDE(2) contain the
```

```
  reciprocal condition numbers for the average of the selected
  eigenvalues.
  Not referenced if SENSE = 'N' or 'V'.

RCONDV
  RCONDV is REAL array, dimension ( 2 )
  If SENSE = 'V' or 'B', RCONDV(1) and RCONDV(2) contain the
  reciprocal condition number for the selected deflating
  subspaces.
  Not referenced if SENSE = 'N' or 'E'.

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  If N = 0, LWORK >= 1, else if SENSE = 'E', 'V', or 'B',
  LWORK >= MAX(1,2*N,2*SDIM*(N-SDIM)), else
  LWORK >= MAX(1,2*N).  Note that 2*SDIM*(N-SDIM) <= N*N/2.
  Note also that an error is only returned if
  LWORK < MAX(1,2*N), but if SENSE = 'E' or 'V' or 'B' this may
  not be large enough.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the bound on the optimal size of the WORK
  array and the minimum size of the IWORK array, returns these
  values as the first entries of the WORK and IWORK arrays, and
  no error message related to LWORK or LIWORK is issued by XERBLA.

RWORK
  RWORK is REAL array, dimension ( 8*N )
  Real workspace.

IWORK
  IWORK is INTEGER array, dimension (MAX(1,LIWORK))
  On exit, if INFO = 0, IWORK(1) returns the minimum LIWORK.

LIWORK
  LIWORK is INTEGER
  The dimension of the array WORK.
  If SENSE = 'N' or N = 0, LIWORK >= 1, otherwise
  LIWORK >= N+2.

  If LIWORK = -1, then a workspace query is assumed; the
  routine only calculates the bound on the optimal size of the
  WORK array and the minimum size of the IWORK array, returns
  these values as the first entries of the WORK and IWORK
  arrays, and no error message related to LWORK or LIWORK is
```

```
  issued by XERBLA.

 BWORK
  BWORK is LOGICAL array, dimension (N)
  Not referenced if SORT = 'N'.

 INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  = 1,...,N:
              The QZ iteration failed.  (A,B) are not in Schur
              form, but ALPHA(j) and BETA(j) should be correct for
              j=INFO+1,...,N.
  > N:  =N+1: other than QZ iteration failed in CHGEQZ
        =N+2: after reordering, roundoff changed values of
              some complex eigenvalues so that leading
              eigenvalues in the Generalized Schur form no
              longer satisfy SELCTG=.TRUE.  This could also
              be caused due to scaling.
        =N+3: reordering failed in CTGSEN.
```

———-

26.9.3 fortran code

— **cggesx.f** —

```
*  ====================================================================
      SUBROUTINE CGGESX( JOBVSL, JOBVSR, SORT, SELCTG, SENSE, N, A, LDA,
     $                   B, LDB, SDIM, ALPHA, BETA, VSL, LDVSL, VSR,
     $                   LDVSR, RCONDE, RCONDV, WORK, LWORK, RWORK,
     $                   IWORK, LIWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVSL, JOBVSR, SENSE, SORT
      INTEGER            INFO, LDA, LDB, LDVSL, LDVSR, LIWORK, LWORK, N,
     $                   SDIM
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      INTEGER            IWORK( * )
```

```
      REAL               RCONDE( 2 ), RCONDV( 2 ), RWORK( * )
      COMPLEX            A( LDA, * ), ALPHA( * ), B( LDB, * ),
     $                   BETA( * ), VSL( LDVSL, * ), VSR( LDVSR, * ),
     $                   WORK( * )
*     ..
*     .. Function Arguments ..
      LOGICAL            SELCTG
      EXTERNAL           SELCTG
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      COMPLEX            CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0E+0, 0.0E+0 ),
     $                   CONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            CURSL, ILASCL, ILBSCL, ILVSL, ILVSR, LASTSL,
     $                   LQUERY, WANTSB, WANTSE, WANTSN, WANTST, WANTSV
      INTEGER            I, ICOLS, IERR, IHI, IJOB, IJOBVL, IJOBVR,
     $                   ILEFT, ILO, IRIGHT, IROWS, IRWRK, ITAU, IWRK,
     $                   LIWMIN, LWRK, MAXWRK, MINWRK
      REAL               ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS, PL,
     $                   PR, SMLNUM
*     ..
*     .. Local Arrays ..
      REAL               DIF( 2 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEQRF, CGGBAK, CGGBAL, CGGHRD, CHGEQZ, CLACPY,
     $                   CLASCL, CLASET, CTGSEN, CUNGQR, CUNMQR, SLABAD,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               CLANGE, SLAMCH
      EXTERNAL           LSAME, ILAENV, CLANGE, SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVSL, 'N' ) ) THEN
```

```
         IJOBVL = 1
         ILVSL = .FALSE.
      ELSE IF( LSAME( JOBVSL, 'V' ) ) THEN
         IJOBVL = 2
         ILVSL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVSL = .FALSE.
      END IF
*
      IF( LSAME( JOBVSR, 'N' ) ) THEN
         IJOBVR = 1
         ILVSR = .FALSE.
      ELSE IF( LSAME( JOBVSR, 'V' ) ) THEN
         IJOBVR = 2
         ILVSR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVSR = .FALSE.
      END IF
*
      WANTST = LSAME( SORT, 'S' )
      WANTSN = LSAME( SENSE, 'N' )
      WANTSE = LSAME( SENSE, 'E' )
      WANTSV = LSAME( SENSE, 'V' )
      WANTSB = LSAME( SENSE, 'B' )
      LQUERY = ( LWORK.EQ.-1 .OR. LIWORK.EQ.-1 )
      IF( WANTSN ) THEN
         IJOB = 0
      ELSE IF( WANTSE ) THEN
         IJOB = 1
      ELSE IF( WANTSV ) THEN
         IJOB = 2
      ELSE IF( WANTSB ) THEN
         IJOB = 4
      END IF
*
*     Test the input arguments
*
      INFO = 0
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -2
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( .NOT.( WANTSN .OR. WANTSE .OR. WANTSV .OR. WANTSB ) .OR.
     $         ( .NOT.WANTST .AND. .NOT.WANTSN ) ) THEN
         INFO = -5
      ELSE IF( N.LT.0 ) THEN
```

```
         INFO = -6
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -10
      ELSE IF( LDVSL.LT.1 .OR. ( ILVSL .AND. LDVSL.LT.N ) ) THEN
         INFO = -15
      ELSE IF( LDVSR.LT.1 .OR. ( ILVSR .AND. LDVSR.LT.N ) ) THEN
         INFO = -17
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.GT.0) THEN
            MINWRK = 2*N
            MAXWRK = N*(1 + ILAENV( 1, 'CGEQRF', ' ', N, 1, N, 0 ) )
            MAXWRK = MAX( MAXWRK, N*( 1 +
     $                    ILAENV( 1, 'CUNMQR', ' ', N, 1, N, -1 ) ) )
            IF( ILVSL ) THEN
               MAXWRK = MAX( MAXWRK, N*( 1 +
     $                       ILAENV( 1, 'CUNGQR', ' ', N, 1, N, -1 ) ) )
            END IF
            LWRK = MAXWRK
            IF( IJOB.GE.1 )
     $         LWRK = MAX( LWRK, N*N/2 )
         ELSE
            MINWRK = 1
            MAXWRK = 1
            LWRK   = 1
         END IF
         WORK( 1 ) = LWRK
         IF( WANTSN .OR. N.EQ.0 ) THEN
            LIWMIN = 1
         ELSE
            LIWMIN = N + 2
         END IF
         IWORK( 1 ) = LIWMIN
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -21
         ELSE IF( LIWORK.LT.LIWMIN  .AND. .NOT.LQUERY) THEN
            INFO = -24
         END IF
      END IF
```

```
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGGESX', -INFO )
         RETURN
      ELSE IF (LQUERY) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = CLANGE( 'M', N, N, A, LDA, RWORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
      IF( ILASCL )
     $   CALL CLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = CLANGE( 'M', N, N, B, LDB, RWORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
```

```
     $   CALL CLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrix to make it more nearly triangular
*     (Real Workspace: need 6*N)
*
      ILEFT = 1
      IRIGHT = N + 1
      IRWRK = IRIGHT + N
      CALL CGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, RWORK( ILEFT ),
     $             RWORK( IRIGHT ), RWORK( IRWRK ), IERR )
*
*     Reduce B to triangular form (QR decomposition of B)
*     (Complex Workspace: need N, prefer N*NB)
*
      IROWS = IHI + 1 - ILO
      ICOLS = N + 1 - ILO
      ITAU = 1
      IWRK = ITAU + IROWS
      CALL CGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the unitary transformation to matrix A
*     (Complex Workspace: need N, prefer N*NB)
*
      CALL CUNMQR( 'L', 'C', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VSL
*     (Complex Workspace: need N, prefer N*NB)
*
      IF( ILVSL ) THEN
         CALL CLASET( 'Full', N, N, CZERO, CONE, VSL, LDVSL )
         IF( IROWS.GT.1 ) THEN
            CALL CLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VSL( ILO+1, ILO ), LDVSL )
         END IF
         CALL CUNGQR( IROWS, IROWS, IROWS, VSL( ILO, ILO ), LDVSL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Initialize VSR
*
      IF( ILVSR )
     $   CALL CLASET( 'Full', N, N, CZERO, CONE, VSR, LDVSR )
*
*     Reduce to generalized Hessenberg form
*     (Workspace: none needed)
*
      CALL CGGHRD( JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB, VSL,
```

```
     $             LDVSL, VSR, LDVSR, IERR )
*
      SDIM = 0
*
*     Perform QZ algorithm, computing Schur vectors if desired
*     (Complex Workspace: need N)
*     (Real Workspace:    need N)
*
      IWRK = ITAU
      CALL CHGEQZ( 'S', JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, WORK( IWRK ),
     $             LWORK+1-IWRK, RWORK( IRWRK ), IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 40
      END IF
*
*     Sort eigenvalues ALPHA/BETA and compute the reciprocal of
*     condition number(s)
*
      IF( WANTST ) THEN
*
*        Undo scaling on eigenvalues before SELCTGing
*
         IF( ILASCL )
     $      CALL CLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHA, N, IERR )
         IF( ILBSCL )
     $      CALL CLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
*
*        Select eigenvalues
*
         DO 10 I = 1, N
            BWORK( I ) = SELCTG( ALPHA( I ), BETA( I ) )
   10    CONTINUE
*
*        Reorder eigenvalues, transform Generalized Schur vectors, and
*        compute reciprocal condition numbers
*        (Complex Workspace: If IJOB >= 1, need MAX(1, 2*SDIM*(N-SDIM))
*                            otherwise, need 1 )
*
         CALL CTGSEN( IJOB, ILVSL, ILVSR, BWORK, N, A, LDA, B, LDB,
     $                ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, SDIM, PL, PR,
     $                DIF, WORK( IWRK ), LWORK-IWRK+1, IWORK, LIWORK,
     $                IERR )
```

```
*
         IF( IJOB.GE.1 )
     $      MAXWRK = MAX( MAXWRK, 2*SDIM*( N-SDIM ) )
         IF( IERR.EQ.-21 ) THEN
*
*           not enough complex workspace
*
            INFO = -21
         ELSE
            IF( IJOB.EQ.1 .OR. IJOB.EQ.4 ) THEN
               RCONDE( 1 ) = PL
               RCONDE( 2 ) = PR
            END IF
            IF( IJOB.EQ.2 .OR. IJOB.EQ.4 ) THEN
               RCONDV( 1 ) = DIF( 1 )
               RCONDV( 2 ) = DIF( 2 )
            END IF
            IF( IERR.EQ.1 )
     $         INFO = N + 3
         END IF
*
      END IF
*
*     Apply permutation to VSL and VSR
*     (Workspace: none needed)
*
      IF( ILVSL )
     $   CALL CGGBAK( 'P', 'L', N, ILO, IHI, RWORK( ILEFT ),
     $                RWORK( IRIGHT ), N, VSL, LDVSL, IERR )
*
      IF( ILVSR )
     $   CALL CGGBAK( 'P', 'R', N, ILO, IHI, RWORK( ILEFT ),
     $                RWORK( IRIGHT ), N, VSR, LDVSR, IERR )
*
*     Undo scaling
*
      IF( ILASCL ) THEN
         CALL CLASCL( 'U', 0, 0, ANRMTO, ANRM, N, N, A, LDA, IERR )
         CALL CLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHA, N, IERR )
      END IF
*
      IF( ILBSCL ) THEN
         CALL CLASCL( 'U', 0, 0, BNRMTO, BNRM, N, N, B, LDB, IERR )
         CALL CLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
      END IF
*
      IF( WANTST ) THEN
*
*        Check if reordering is correct
*
```

```
         LASTSL = .TRUE.
         SDIM = 0
         DO 30 I = 1, N
            CURSL = SELCTG( ALPHA( I ), BETA( I ) )
            IF( CURSL )
     $         SDIM = SDIM + 1
            IF( CURSL .AND. .NOT.LASTSL )
     $         INFO = N + 2
            LASTSL = CURSL
   30    CONTINUE
*
      END IF
*
   40 CONTINUE
*
      WORK( 1 ) = MAXWRK
      IWORK( 1 ) = LIWMIN
*
      RETURN
*
*     End of CGGESX
*
      END
```

———-

26.9.4 lisp code

— cggesx.lisp —

———-

26.10 cggev

26.10.1 Axiom unit tests

— cggev.input —

```
)set break resume
)sys rm -f cggev.output
)spool cggev.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

26.10.2 Axiom help page

— **cggev.help** —

```
=====================================================================
cggev examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cggev (JOBVL, JOBVR, N, A, LDA, B, LDB, ALPHA, BETA, VL,
           LDVL, VR, LDVR, WORK, LWORK, RWORK, INFO)
            CGGEV computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for GE matrices

Function/Subroutine Documentation
   subroutine cggev (character JOBVL, character JOBVR, integer N, complex,
       dimension( lda, * ) A, integer LDA, complex, dimension( ldb, * ) B,
       integer LDB, complex, dimension( * ) ALPHA, complex, dimension( * )
       BETA, complex, dimension( ldvl, * ) VL, integer LDVL, complex,
       dimension( ldvr, * ) VR, integer LDVR, complex, dimension( * ) WORK,
       integer LWORK, real, dimension( * ) RWORK, integer INFO)
        CGGEV computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for GE matrices

Purpose:
   CGGEV computes for a pair of N-by-N complex nonsymmetric matrices
   (A,B), the generalized eigenvalues, and optionally, the left and/or
   right generalized eigenvectors.

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar
   lambda or a ratio alpha/beta = lambda, such that A - lambda*B is
   singular. It is usually represented as the pair (alpha,beta), as
   there is a reasonable interpretation for beta=0, and even for both
   being zero.
```

```
   The right generalized eigenvector v(j) corresponding to the
   generalized eigenvalue lambda(j) of (A,B) satisfies

      A * v(j) = lambda(j) * B * v(j).

   The left generalized eigenvector u(j) corresponding to the
   generalized eigenvalues lambda(j) of (A,B) satisfies

      u(j)**H * A = lambda(j) * u(j)**H * B

   where u(j)**H is the conjugate-transpose of u(j).

Parameters:

 JOBVL
   JOBVL is CHARACTER*1
   = 'N':  do not compute the left generalized eigenvectors;
   = 'V':  compute the left generalized eigenvectors.

 JOBVR
   JOBVR is CHARACTER*1
   = 'N':  do not compute the right generalized eigenvectors;
   = 'V':  compute the right generalized eigenvectors.

 N
   N is INTEGER
   The order of the matrices A, B, VL, and VR.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA, N)
   On entry, the matrix A in the pair (A,B).
   On exit, A has been overwritten.

 LDA
   LDA is INTEGER
   The leading dimension of A.  LDA >= max(1,N).

 B
   B is COMPLEX array, dimension (LDB, N)
   On entry, the matrix B in the pair (A,B).
   On exit, B has been overwritten.

 LDB
   LDB is INTEGER
   The leading dimension of B.  LDB >= max(1,N).

 ALPHA
   ALPHA is COMPLEX array, dimension (N)

 BETA
```

```
  BETA is COMPLEX array, dimension (N)
  On exit, ALPHA(j)/BETA(j), j=1,...,N, will be the
  generalized eigenvalues.

  Note: the quotients ALPHA(j)/BETA(j) may easily over- or
  underflow, and BETA(j) may even be zero.  Thus, the user
  should avoid naively computing the ratio alpha/beta.
  However, ALPHA will be always less than and usually
  comparable with norm(A) in magnitude, and BETA always less
  than and usually comparable with norm(B).

VL
  VL is COMPLEX array, dimension (LDVL,N)
  If JOBVL = 'V', the left generalized eigenvectors u(j) are
  stored one after another in the columns of VL, in the same
  order as their eigenvalues.
  Each eigenvector is scaled so the largest component has
  abs(real part) + abs(imag. part) = 1.
  Not referenced if JOBVL = 'N'.

LDVL
  LDVL is INTEGER
  The leading dimension of the matrix VL. LDVL >= 1, and
  if JOBVL = 'V', LDVL >= N.

VR
  VR is COMPLEX array, dimension (LDVR,N)
  If JOBVR = 'V', the right generalized eigenvectors v(j) are
  stored one after another in the columns of VR, in the same
  order as their eigenvalues.
  Each eigenvector is scaled so the largest component has
  abs(real part) + abs(imag. part) = 1.
  Not referenced if JOBVR = 'N'.

LDVR
  LDVR is INTEGER
  The leading dimension of the matrix VR. LDVR >= 1, and
  if JOBVR = 'V', LDVR >= N.

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,2*N).
  For good performance, LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
```

```
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is REAL array, dimension (8*N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  =1,...,N:
        The QZ iteration failed.  No eigenvectors have been
        calculated, but ALPHA(j) and BETA(j) should be
        correct for j=INFO+1,...,N.
  > N:  =N+1: other then QZ iteration failed in SHGEQZ,
        =N+2: error return from STGEVC.
```

26.10.3 fortran code

— cggev.f —

```
*  =====================================================================
      SUBROUTINE CGGEV( JOBVL, JOBVR, N, A, LDA, B, LDB, ALPHA, BETA,
     $                  VL, LDVL, VR, LDVR, WORK, LWORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVL, JOBVR
      INTEGER            INFO, LDA, LDB, LDVL, LDVR, LWORK, N
*     ..
*     .. Array Arguments ..
      REAL               RWORK( * )
      COMPLEX            A( LDA, * ), ALPHA( * ), B( LDB, * ),
     $                   BETA( * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
```

```
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
      COMPLEX            CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0E0, 0.0E0 ),
     $                   CONE = ( 1.0E0, 0.0E0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            ILASCL, ILBSCL, ILV, ILVL, ILVR, LQUERY
      CHARACTER          CHTEMP
      INTEGER            ICOLS, IERR, IHI, IJOBVL, IJOBVR, ILEFT, ILO,
     $                   IN, IRIGHT, IROWS, IRWRK, ITAU, IWRK, JC, JR,
     $                   LWKMIN, LWKOPT
      REAL               ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS,
     $                   SMLNUM, TEMP
      COMPLEX            X
*     ..
*     .. Local Arrays ..
      LOGICAL            LDUMMA( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEQRF, CGGBAK, CGGBAL, CGGHRD, CHGEQZ, CLACPY,
     $                   CLASCL, CLASET, CTGEVC, CUNGQR, CUNMQR, SLABAD,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               CLANGE, SLAMCH
      EXTERNAL           LSAME, ILAENV, CLANGE, SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, MAX, REAL, SQRT
*     ..
*     .. Statement Functions ..
      REAL               ABS1
*     ..
*     .. Statement Function definitions ..
      ABS1( X ) = ABS( REAL( X ) ) + ABS( AIMAG( X ) )
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVL, 'N' ) ) THEN
         IJOBVL = 1
         ILVL = .FALSE.
      ELSE IF( LSAME( JOBVL, 'V' ) ) THEN
         IJOBVL = 2
         ILVL = .TRUE.
      ELSE
         IJOBVL = -1
```

```
         ILVL = .FALSE.
      END IF
*
      IF( LSAME( JOBVR, 'N' ) ) THEN
         IJOBVR = 1
         ILVR = .FALSE.
      ELSE IF( LSAME( JOBVR, 'V' ) ) THEN
         IJOBVR = 2
         ILVR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVR = .FALSE.
      END IF
      ILV = ILVL .OR. ILVR
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDVL.LT.1 .OR. ( ILVL .AND. LDVL.LT.N ) ) THEN
         INFO = -11
      ELSE IF( LDVR.LT.1 .OR. ( ILVR .AND. LDVR.LT.N ) ) THEN
         INFO = -13
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV. The workspace is
*       computed assuming ILO = 1 and IHI = N, the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         LWKMIN = MAX( 1, 2*N )
         LWKOPT = MAX( 1, N + N*ILAENV( 1, 'CGEQRF', ' ', N, 1, N, 0 ) )
         LWKOPT = MAX( LWKOPT, N +
     $                 N*ILAENV( 1, 'CUNMQR', ' ', N, 1, N, 0 ) )
         IF( ILVL ) THEN
            LWKOPT = MAX( LWKOPT, N +
```

```
     $                 N*ILAENV( 1, 'CUNGQR', ' ', N, 1, N, -1 ) )
         END IF
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.LWKMIN .AND. .NOT.LQUERY )
     $      INFO = -15
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGGEV ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
*
      EPS = SLAMCH( 'E' )*SLAMCH( 'B' )
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = CLANGE( 'M', N, N, A, LDA, RWORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
      IF( ILASCL )
     $   CALL CLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = CLANGE( 'M', N, N, B, LDB, RWORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
```

```
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
     $   CALL CLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrices A, B to isolate eigenvalues if possible
*     (Real Workspace: need 6*N)
*
      ILEFT = 1
      IRIGHT = N + 1
      IRWRK = IRIGHT + N
      CALL CGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, RWORK( ILEFT ),
     $             RWORK( IRIGHT ), RWORK( IRWRK ), IERR )
*
*     Reduce B to triangular form (QR decomposition of B)
*     (Complex Workspace: need N, prefer N*NB)
*
      IROWS = IHI + 1 - ILO
      IF( ILV ) THEN
         ICOLS = N + 1 - ILO
      ELSE
         ICOLS = IROWS
      END IF
      ITAU = 1
      IWRK = ITAU + IROWS
      CALL CGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to matrix A
*     (Complex Workspace: need N, prefer N*NB)
*
      CALL CUNMQR( 'L', 'C', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VL
*     (Complex Workspace: need N, prefer N*NB)
*
      IF( ILVL ) THEN
         CALL CLASET( 'Full', N, N, CZERO, CONE, VL, LDVL )
         IF( IROWS.GT.1 ) THEN
            CALL CLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VL( ILO+1, ILO ), LDVL )
         END IF
         CALL CUNGQR( IROWS, IROWS, IROWS, VL( ILO, ILO ), LDVL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
```

```
*     Initialize VR
*
      IF( ILVR )
     $   CALL CLASET( 'Full', N, N, CZERO, CONE, VR, LDVR )
*
*     Reduce to generalized Hessenberg form
*
      IF( ILV ) THEN
*
*        Eigenvectors requested -- work on whole matrix.
*
         CALL CGGHRD( JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB, VL,
     $                LDVL, VR, LDVR, IERR )
      ELSE
         CALL CGGHRD( 'N', 'N', IROWS, 1, IROWS, A( ILO, ILO ), LDA,
     $                B( ILO, ILO ), LDB, VL, LDVL, VR, LDVR, IERR )
      END IF
*
*     Perform QZ algorithm (Compute eigenvalues, and optionally, the
*     Schur form and Schur vectors)
*     (Complex Workspace: need N)
*     (Real Workspace: need N)
*
      IWRK = ITAU
      IF( ILV ) THEN
         CHTEMP = 'S'
      ELSE
         CHTEMP = 'E'
      END IF
      CALL CHGEQZ( CHTEMP, JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHA, BETA, VL, LDVL, VR, LDVR, WORK( IWRK ),
     $             LWORK+1-IWRK, RWORK( IRWRK ), IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 70
      END IF
*
*     Compute Eigenvectors
*     (Real Workspace: need 2*N)
*     (Complex Workspace: need 2*N)
*
      IF( ILV ) THEN
         IF( ILVL ) THEN
            IF( ILVR ) THEN
```

```
            CHTEMP = 'B'
         ELSE
            CHTEMP = 'L'
         END IF
      ELSE
         CHTEMP = 'R'
      END IF
*
      CALL CTGEVC( CHTEMP, 'B', LDUMMA, N, A, LDA, B, LDB, VL, LDVL,
     $               VR, LDVR, N, IN, WORK( IWRK ), RWORK( IRWRK ),
     $               IERR )
      IF( IERR.NE.0 ) THEN
         INFO = N + 2
         GO TO 70
      END IF
*
*     Undo balancing on VL and VR and normalization
*     (Workspace: none needed)
*
      IF( ILVL ) THEN
         CALL CGGBAK( 'P', 'L', N, ILO, IHI, RWORK( ILEFT ),
     $                  RWORK( IRIGHT ), N, VL, LDVL, IERR )
         DO 30 JC = 1, N
            TEMP = ZERO
            DO 10 JR = 1, N
               TEMP = MAX( TEMP, ABS1( VL( JR, JC ) ) )
   10       CONTINUE
            IF( TEMP.LT.SMLNUM )
     $         GO TO 30
            TEMP = ONE / TEMP
            DO 20 JR = 1, N
               VL( JR, JC ) = VL( JR, JC )*TEMP
   20       CONTINUE
   30    CONTINUE
      END IF
      IF( ILVR ) THEN
         CALL CGGBAK( 'P', 'R', N, ILO, IHI, RWORK( ILEFT ),
     $                  RWORK( IRIGHT ), N, VR, LDVR, IERR )
         DO 60 JC = 1, N
            TEMP = ZERO
            DO 40 JR = 1, N
               TEMP = MAX( TEMP, ABS1( VR( JR, JC ) ) )
   40       CONTINUE
            IF( TEMP.LT.SMLNUM )
     $         GO TO 60
            TEMP = ONE / TEMP
            DO 50 JR = 1, N
               VR( JR, JC ) = VR( JR, JC )*TEMP
   50       CONTINUE
   60    CONTINUE
```

```
         END IF
      END IF
*
*     Undo scaling if necessary
*
   70 CONTINUE
*
      IF( ILASCL )
     $   CALL CLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHA, N, IERR )
*
      IF( ILBSCL )
     $   CALL CLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
*
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of CGGEV
*
      END
```

———-

26.10.4 lisp code

— **cggev.lisp** —

```
(let* ((zero 0.0)
       (one 1.0)
       (czero (f2cl-lib:cmplx 0.0 0.0))
       (cone (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (f2cl-lib:complex8) czero)
           (type (f2cl-lib:complex8) cone)
           (ignorable zero one czero cone))
  (defun cggev
         (jobvl jobvr n a lda b ldb$ alpha beta vl ldvl vr ldvr work lwork
          rwork info)
    (declare (type (array single-float (*)) rwork)
             (type (array f2cl-lib:complex8 (*)) work vr vl beta alpha b a)
             (type (f2cl-lib:integer4) info lwork ldvr ldvl ldb$ lda n)
             (type (string *) jobvr jobvl))
    (f2cl-lib:with-multi-array-data
        ((jobvl character jobvl-%data% jobvl-%offset%)
         (jobvr character jobvr-%data% jobvr-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (alpha f2cl-lib:complex8 alpha-%data% alpha-%offset%)
```

```
    (beta f2cl-lib:complex8 beta-%data% beta-%offset%)
    (vl f2cl-lib:complex8 vl-%data% vl-%offset%)
    (vr f2cl-lib:complex8 vr-%data% vr-%offset%)
    (work f2cl-lib:complex8 work-%data% work-%offset%)
    (rwork single-float rwork-%data% rwork-%offset%))
  (labels ((abs1 (x)
             (+ (abs (f2cl-lib:freal x)) (abs (f2cl-lib:aimag x)))))
    (declare
     (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
      abs1))
    (prog ((ldumma (make-array 1 :element-type 't)) (x #C(0.0 0.0))
           (anrm 0.0) (anrmto 0.0) (bignum 0.0) (bnrm 0.0) (bnrmto 0.0)
           (eps 0.0) (smlnum 0.0) (temp 0.0) (icols 0) (ierr 0) (ihi 0)
           (ijobvl 0) (ijobvr 0) (ileft 0) (ilo 0) (in 0) (iright 0)
           (irows 0) (irwrk 0) (itau 0) (iwrk 0) (jc 0) (jr 0) (lwkmin 0)
           (lwkopt 0)
           (chtemp
            (make-array '(1) :element-type 'character :initial-element
                        #\ ))
           (ilascl nil) (ilbscl nil) (ilv nil) (ilvl nil) (ilvr nil)
           (lquery nil))
      (declare (type (array f2cl-lib:logical (1)) ldumma)
               (type (f2cl-lib:complex8) x)
               (type (single-float) anrm anrmto bignum bnrm bnrmto eps
                smlnum temp)
               (type (f2cl-lib:integer4) icols ierr ihi ijobvl ijobvr ileft
                ilo in iright irows irwrk itau iwrk jc jr lwkmin lwkopt)
               (type (string 1) chtemp)
               (type f2cl-lib:logical ilascl ilbscl ilv ilvl ilvr lquery))
      (cond
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvl "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvl var-0))
          ret-val)
        (setf ijobvl 1) (setf ilvl f2cl-lib:%false%))
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvl "V")
          (declare (ignore var-1))
          (when var-0 (setf jobvl var-0))
          ret-val)
        (setf ijobvl 2) (setf ilvl f2cl-lib:%true%))
       (t (setf ijobvl -1) (setf ilvl f2cl-lib:%false%)))
      (cond
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvr "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvr var-0))
          ret-val)
        (setf ijobvr 1) (setf ilvr f2cl-lib:%false%))
```

```
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvr "V")
    (declare (ignore var-1))
    (when var-0 (setf jobvr var-0))
    ret-val)
  (setf ijobvr 2) (setf ilvr f2cl-lib:%true%))
 (t (setf ijobvr -1) (setf ilvr f2cl-lib:%false%)))
(setf ilv (or ilvl ilvr))
(setf info 0)
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((<= ijobvl 0) (setf info -1)) ((<= ijobvr 0) (setf info -2))
      ((< n 0) (setf info -3))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -5))
      ((< ldb$
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -7))
      ((or (< ldvl 1) (and ilvl (< ldvl n))) (setf info -11))
      ((or (< ldvr 1) (and ilvr (< ldvr n))) (setf info -13)))
(cond
 ((= info 0)
  (setf lwkmin
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul 2 n))))
  (setf lwkopt
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add n
                                      (f2cl-lib:int-mul n
                                                        (multiple-value-bind
                                                            (ret-val
                                                             var-0
                                                             var-1
                                                             var-2
                                                             var-3
                                                             var-4
                                                             var-5
                                                             var-6)
                                                            (ilaenv 1
                                                             "CGEQRF"
                                                             " " n 1
                                                             n 0)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-4
                                                            var-6))
```

```
                                                        (when var-3
                                                          (setf n
                                                                  var-3))
                                                        (when var-5
                                                          (setf n
                                                                  var-5))
                                                        ret-val))))))
            (setf lwkopt
                    (max (the f2cl-lib:integer4 lwkopt)
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add n
                                                (f2cl-lib:int-mul n
                                                                  (multiple-value-bind
                                                                      (ret-val
                                                                       var-0
                                                                       var-1
                                                                       var-2
                                                                       var-3
                                                                       var-4
                                                                       var-5
                                                                       var-6)
                                                                      (ilaenv 1
                                                                       "CUNMQR"
                                                                       " " n 1
                                                                       n 0)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-4
                                                          var-6))
                                                        (when var-3
                                                          (setf n
                                                                  var-3))
                                                        (when var-5
                                                          (setf n
                                                                  var-5))
                                                        ret-val))))))
            (cond
             (ilvl
              (setf lwkopt
                      (max (the f2cl-lib:integer4 lwkopt)
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add n
                                                  (f2cl-lib:int-mul n
                                                                    (multiple-value-bind
                                                                        (ret-val
                                                                         var-0
                                                                         var-1
```

```
                                                                    var-2
                                                                    var-3
                                                                    var-4
                                                                    var-5
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "CUNGQR"
                                                                    " " n
                                                                    1 n
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-4
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf n
                                                                            var-3))
                                                                  (when
                                                                      var-5
                                                                    (setf n
                                                                            var-5))
                                                                  ret-val))))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwkopt 'f2cl-lib:complex8))
  (if (and (< lwork lwkmin) (not lquery))
      (setf info -15))))
(cond
 ((/= info 0) (xerbla "CGGEV " (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(setf eps (* (slamch "E") (slamch "B")))
(setf smlnum (slamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (slabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
```

```
            (clange "M" n n a lda rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf ilascl f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf anrmto smlnum)
  (setf ilascl f2cl-lib:%true%))
 ((> anrm bignum) (setf anrmto bignum)
  (setf ilascl f2cl-lib:%true%)))
(if ilascl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (clascl "G" 0 0 anrm anrmto n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf anrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "M" n n b ldb$ rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ilbscl f2cl-lib:%false%)
(cond
 ((and (> bnrm zero) (< bnrm smlnum)) (setf bnrmto smlnum)
  (setf ilbscl f2cl-lib:%true%))
 ((> bnrm bignum) (setf bnrmto bignum)
  (setf ilbscl f2cl-lib:%true%)))
(if ilbscl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (clascl "G" 0 0 bnrm bnrmto n n b ldb$ ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bnrm var-3))
      (when var-4 (setf bnrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf ierr var-9))))
(setf ileft 1)
```

```
(setf iright (f2cl-lib:int-add n 1))
(setf irwrk (f2cl-lib:int-add iright n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (cggbal "P" n a lda b ldb$ ilo ihi
     (f2cl-lib:array-slice rwork-%data% single-float (ileft) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (iright) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (irwrk) ((1 *))
                           rwork-%offset%)
     ierr)
  (declare (ignore var-0 var-1 var-2 var-4 var-8 var-9 var-10))
  (setf lda var-3)
  (setf ldb$ var-5)
  (setf ilo var-6)
  (setf ihi var-7)
  (setf ierr var-11))
(setf irows (f2cl-lib:int-sub (f2cl-lib:int-add ihi 1) ilo))
(cond
 (ilv (setf icols (f2cl-lib:int-sub (f2cl-lib:int-add n 1) ilo)))
 (t (setf icols irows)))
(setf itau 1)
(setf iwrk (f2cl-lib:int-add itau irows))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgeqrf irows icols
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf irows var-0)
  (setf icols var-1)
  (setf ldb$ var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (cunmqr "L" "C" irows icols irows
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
```

```
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (ilo ilo)
                            ((1 lda) (1 *)) a-%offset%)
      lda
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf irows var-2))
  (when var-3 (setf icols var-3))
  (when var-4 (setf irows var-4))
  (when var-6 (setf ldb$ var-6))
  (when var-9 (setf lda var-9))
  (when var-12 (setf ierr var-12)))
(cond
 (ilvl
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "Full" n n czero cone vl ldvl)
    (declare (ignore var-0 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf cone var-4))
    (when var-6 (setf ldvl var-6)))
  (cond
   ((> irows 1)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (clacpy "L" (f2cl-lib:int-sub irows 1)
         (f2cl-lib:int-sub irows 1)
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                               ((+ ilo 1) ilo) ((1 ldb$) (1 *))
                               b-%offset%)
         ldb$
         (f2cl-lib:array-slice vl-%data% f2cl-lib:complex8
                               ((+ ilo 1) ilo) ((1 ldvl) (1 *))
                               vl-%offset%)
         ldvl)
      (declare (ignore var-0 var-1 var-2 var-3 var-5))
      (when var-4 (setf ldb$ var-4))
      (when var-6 (setf ldvl var-6)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (cungqr irows irows irows
       (f2cl-lib:array-slice vl-%data% f2cl-lib:complex8 (ilo ilo)
                             ((1 ldvl) (1 *)) vl-%offset%)
       ldvl
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
```

```
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf irows var-0))
    (when var-1 (setf irows var-1))
    (when var-2 (setf irows var-2))
    (when var-4 (setf ldvl var-4))
    (when var-8 (setf ierr var-8)))))
(if ilvr
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "Full" n n czero cone vr ldvr)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldvr var-6))))
(cond
 (ilv
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (cgghrd jobvl jobvr n ilo ihi a lda b ldb$ vl ldvl vr ldvr
       ierr)
    (declare (ignore var-5 var-7 var-9 var-11))
    (when var-0 (setf jobvl var-0))
    (when var-1 (setf jobvr var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-13 (setf ierr var-13))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (cgghrd "N" "N" irows 1 irows
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (ilo ilo)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (ilo ilo)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$ vl ldvl vr ldvr ierr)
    (declare (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11))
    (when var-2 (setf irows var-2))
    (when var-4 (setf irows var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
```

```
    (when var-12 (setf ldvr var-12))
    (when var-13 (setf ierr var-13)))))
(setf iwrk itau)
(cond (ilv (f2cl-lib:f2cl-set-string chtemp "S" (string 1)))
      (t (f2cl-lib:f2cl-set-string chtemp "E" (string 1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18
     var-19)
    (chgeqz chtemp jobvl jobvr n ilo ihi a lda b ldb$ alpha beta vl
     ldvl vr ldvr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk)
     (f2cl-lib:array-slice rwork-%data% single-float (irwrk) ((1 *))
                           rwork-%offset%)
     ierr)
  (declare
   (ignore var-6 var-8 var-10 var-11 var-12 var-14 var-16 var-17
    var-18))
  (when var-0 (setf chtemp var-0))
  (when var-1 (setf jobvl var-1))
  (when var-2 (setf jobvr var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf ilo var-4))
  (when var-5 (setf ihi var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldb$ var-9))
  (when var-13 (setf ldvl var-13))
  (when var-15 (setf ldvr var-15))
  (when var-19 (setf ierr var-19)))
(cond
 ((/= ierr 0)
  (cond ((and (> ierr 0) (<= ierr n)) (setf info ierr))
        ((and (> ierr n) (<= ierr (f2cl-lib:int-mul 2 n)))
         (setf info (f2cl-lib:int-sub ierr n)))
        (t (setf info (f2cl-lib:int-add n 1))))
  (go label70)))
(cond
 (ilv
  (cond
   (ilvl
    (cond (ilvr (f2cl-lib:f2cl-set-string chtemp "B" (string 1)))
          (t (f2cl-lib:f2cl-set-string chtemp "L" (string 1)))))
   (t (f2cl-lib:f2cl-set-string chtemp "R" (string 1))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16)
      (ctgevc chtemp "B" ldumma n a lda b ldb$ vl ldvl vr ldvr n in
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
```

```
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice rwork-%data% single-float (irwrk)
                              ((1 *)) rwork-%offset%)
        ierr)
    (declare
     (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-14 var-15))
    (when var-0 (setf chtemp var-0))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldb$ var-7))
    (when var-9 (setf ldvl var-9))
    (when var-11 (setf ldvr var-11))
    (when var-12 (setf n var-12))
    (when var-13 (setf in var-13))
    (when var-16 (setf ierr var-16)))
  (cond
   ((/= ierr 0) (setf info (f2cl-lib:int-add n 2)) (go label70)))
  (cond
   (ilvl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (cggbak "P" "L" n ilo ihi
         (f2cl-lib:array-slice rwork-%data% single-float (ileft)
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice rwork-%data% single-float (iright)
                               ((1 *)) rwork-%offset%)
         n vl ldvl ierr)
      (declare
       (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
      (setf n var-7)
      (setf ldvl var-9)
      (setf ierr var-10))
    (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                  ((> jc n) nil)
      (tagbody
        (setf temp zero)
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr n) nil)
          (tagbody
            (setf temp
                    (max temp
                         (abs1
                          (f2cl-lib:fref vl-%data% (jr jc)
                                         ((1 ldvl) (1 *))
                                         vl-%offset%))))
           label10))
        (if (< temp smlnum)
            (go label30))
        (setf temp (/ one temp))
```

```
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr n) nil)
          (tagbody
            (setf (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                                 vl-%offset%)
                  (*
                   (f2cl-lib:fref vl-%data% (jr jc)
                                  ((1 ldvl) (1 *)) vl-%offset%)
                   temp))
           label20))
       label30))))
    (cond
     (ilvr
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (cggbak "P" "R" n ilo ihi
           (f2cl-lib:array-slice rwork-%data% single-float (ileft)
                                 ((1 *)) rwork-%offset%)
           (f2cl-lib:array-slice rwork-%data% single-float (iright)
                                 ((1 *)) rwork-%offset%)
           n vr ldvr ierr)
        (declare
         (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
        (setf n var-7)
        (setf ldvr var-9)
        (setf ierr var-10))
      (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                    ((> jc n) nil)
        (tagbody
          (setf temp zero)
          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                        ((> jr n) nil)
            (tagbody
              (setf temp
                    (max temp
                         (abs1
                          (f2cl-lib:fref vr-%data% (jr jc)
                                         ((1 ldvr) (1 *))
                                         vr-%offset%))))
             label40))
          (if (< temp smlnum)
              (go label60))
          (setf temp (/ one temp))
          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                        ((> jr n) nil)
            (tagbody
              (setf (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                   vr-%offset%)
                    (*
```

```
                              (f2cl-lib:fref vr-%data% (jr jc)
                                             ((1 ldvr) (1 *)) vr-%offset%)
                              temp))
                        label50))
                    label60))))))
          label70
           (if ilascl
               (multiple-value-bind
                   (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                   (clascl "G" 0 0 anrmto anrm n 1 alpha n ierr)
                 (declare (ignore var-0 var-1 var-2 var-6 var-7))
                 (when var-3 (setf anrmto var-3))
                 (when var-4 (setf anrm var-4))
                 (when var-5 (setf n var-5))
                 (when var-8 (setf n var-8))
                 (when var-9 (setf ierr var-9))))
           (if ilbscl
               (multiple-value-bind
                   (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                   (clascl "G" 0 0 bnrmto bnrm n 1 beta n ierr)
                 (declare (ignore var-0 var-1 var-2 var-6 var-7))
                 (when var-3 (setf bnrmto var-3))
                 (when var-4 (setf bnrm var-4))
                 (when var-5 (setf n var-5))
                 (when var-8 (setf n var-8))
                 (when var-9 (setf ierr var-9))))
           (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                   (coerce lwkopt 'f2cl-lib:complex8))
           (go end_label)
          end_label
           (return
            (values jobvl jobvr n nil lda nil ldb$ nil nil nil ldvl nil ldvr nil
                    nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cggev fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
```

```
   (*))
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::jobvl
  fortran-to-lisp::jobvr
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$ nil nil nil
  fortran-to-lisp::ldvl nil
  fortran-to-lisp::ldvr nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::cggbak
  fortran-to-lisp::cgeqrf
  fortran-to-lisp::cggbal))))
```

———-

26.11 cggev3

26.11.1 Axiom unit tests

— **cggev3.input** —

```
)set break resume
)sys rm -f cggev3.output
)spool cggev3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

26.11.2 Axiom help page

— cggev3.help —

```
==================================================================
cggev3 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cggev3 (JOBVL, JOBVR, N, A, LDA, B, LDB, ALPHA, BETA, VL,
           LDVL, VR, LDVR, WORK, LWORK, RWORK, INFO)
            CGGEV3 computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for GE matrices (blocked algorithm)

Function/Subroutine Documentation
   subroutine cggev3 (character JOBVL, character JOBVR, integer N, complex,
       dimension( lda, * ) A, integer LDA, complex, dimension( ldb, * ) B,
       integer LDB, complex, dimension( * ) ALPHA, complex, dimension( * )
       BETA, complex, dimension( ldvl, * ) VL, integer LDVL, complex,
       dimension( ldvr, * ) VR, integer LDVR, complex, dimension( * ) WORK,
       integer LWORK, real, dimension( * ) RWORK, integer INFO)
        CGGEV3 computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for GE matrices (blocked algorithm)

Purpose:
   CGGEV3 computes for a pair of N-by-N complex nonsymmetric matrices
   (A,B), the generalized eigenvalues, and optionally, the left and/or
   right generalized eigenvectors.

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar
   lambda or a ratio alpha/beta = lambda, such that A - lambda*B is
   singular. It is usually represented as the pair (alpha,beta), as
   there is a reasonable interpretation for beta=0, and even for both
   being zero.

   The right generalized eigenvector v(j) corresponding to the
   generalized eigenvalue lambda(j) of (A,B) satisfies

      A * v(j) = lambda(j) * B * v(j).

   The left generalized eigenvector u(j) corresponding to the
   generalized eigenvalues lambda(j) of (A,B) satisfies

     u(j)**H * A = lambda(j) * u(j)**H * B
```

```
   where u(j)**H is the conjugate-transpose of u(j).

Parameters:

 JOBVL
   JOBVL is CHARACTER*1
   = 'N':  do not compute the left generalized eigenvectors;
   = 'V':  compute the left generalized eigenvectors.

 JOBVR
e   JOBVR is CHARACTER*1
   = 'N':  do not compute the right generalized eigenvectors;
   = 'V':  compute the right generalized eigenvectors.

 N
   N is INTEGER
   The order of the matrices A, B, VL, and VR.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA, N)
   On entry, the matrix A in the pair (A,B).
   On exit, A has been overwritten.

 LDA
   LDA is INTEGER
   The leading dimension of A.  LDA >= max(1,N).

 B
   B is COMPLEX array, dimension (LDB, N)
   On entry, the matrix B in the pair (A,B).
   On exit, B has been overwritten.

 LDB
   LDB is INTEGER
   The leading dimension of B.  LDB >= max(1,N).

 ALPHA
   ALPHA is COMPLEX array, dimension (N)

 BETA
   BETA is COMPLEX array, dimension (N)
   On exit, ALPHA(j)/BETA(j), j=1,...,N, will be the
   generalized eigenvalues.

   Note: the quotients ALPHA(j)/BETA(j) may easily over- or
   underflow, and BETA(j) may even be zero.  Thus, the user
   should avoid naively computing the ratio alpha/beta.
   However, ALPHA will be always less than and usually
   comparable with norm(A) in magnitude, and BETA always less
```

```
  than and usually comparable with norm(B).

VL
  VL is COMPLEX array, dimension (LDVL,N)
  If JOBVL = 'V', the left generalized eigenvectors u(j) are
  stored one after another in the columns of VL, in the same
  order as their eigenvalues.
  Each eigenvector is scaled so the largest component has
  abs(real part) + abs(imag. part) = 1.
  Not referenced if JOBVL = 'N'.

LDVL
  LDVL is INTEGER
  The leading dimension of the matrix VL. LDVL >= 1, and
  if JOBVL = 'V', LDVL >= N.

VR
  VR is COMPLEX array, dimension (LDVR,N)
  If JOBVR = 'V', the right generalized eigenvectors v(j) are
  stored one after another in the columns of VR, in the same
  order as their eigenvalues.
  Each eigenvector is scaled so the largest component has
  abs(real part) + abs(imag. part) = 1.
  Not referenced if JOBVR = 'N'.

LDVR
  LDVR is INTEGER
  The leading dimension of the matrix VR. LDVR >= 1, and
  if JOBVR = 'V', LDVR >= N.

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is REAL array, dimension (8*N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

```
=1,...,N:
      The QZ iteration failed.  No eigenvectors have been
      calculated, but ALPHA(j) and BETA(j) should be
      correct for j=INFO+1,...,N.
> N:  =N+1: other then QZ iteration failed in SHGEQZ,
      =N+2: error return from STGEVC.
```

———-

26.11.3 fortran code

— **cggev3.f** —

```
*  =====================================================================
      SUBROUTINE CGGEV3( JOBVL, JOBVR, N, A, LDA, B, LDB, ALPHA, BETA,
     $                   VL, LDVL, VR, LDVR, WORK, LWORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     January 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVL, JOBVR
      INTEGER            INFO, LDA, LDB, LDVL, LDVR, LWORK, N
*     ..
*     .. Array Arguments ..
      REAL               RWORK( * )
      COMPLEX            A( LDA, * ), ALPHA( * ), B( LDB, * ),
     $                   BETA( * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
      COMPLEX            CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0E0, 0.0E0 ),
     $                   CONE = ( 1.0E0, 0.0E0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            ILASCL, ILBSCL, ILV, ILVL, ILVR, LQUERY
      CHARACTER          CHTEMP
      INTEGER            ICOLS, IERR, IHI, IJOBVL, IJOBVR, ILEFT, ILO,
     $                   IN, IRIGHT, IROWS, IRWRK, ITAU, IWRK, JC, JR,
```

```
     $                   LWKOPT
      REAL               ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS,
     $                   SMLNUM, TEMP
      COMPLEX            X
*     ..
*     .. Local Arrays ..
      LOGICAL            LDUMMA( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEQRF, CGGBAK, CGGBAL, CGGHD3, CHGEQZ, CLACPY,
     $                   CLASCL, CLASET, CTGEVC, CUNGQR, CUNMQR, SLABAD,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               CLANGE, SLAMCH
      EXTERNAL           LSAME, CLANGE, SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, MAX, REAL, SQRT
*     ..
*     .. Statement Functions ..
      REAL               ABS1
*     ..
*     .. Statement Function definitions ..
      ABS1( X ) = ABS( REAL( X ) ) + ABS( AIMAG( X ) )
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVL, 'N' ) ) THEN
         IJOBVL = 1
         ILVL = .FALSE.
      ELSE IF( LSAME( JOBVL, 'V' ) ) THEN
         IJOBVL = 2
         ILVL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVL = .FALSE.
      END IF
*
      IF( LSAME( JOBVR, 'N' ) ) THEN
         IJOBVR = 1
         ILVR = .FALSE.
      ELSE IF( LSAME( JOBVR, 'V' ) ) THEN
         IJOBVR = 2
         ILVR = .TRUE.
      ELSE
         IJOBVR = -1
```

```
         ILVR = .FALSE.
      END IF
      ILV = ILVL .OR. ILVR
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDVL.LT.1 .OR. ( ILVL .AND. LDVL.LT.N ) ) THEN
         INFO = -11
      ELSE IF( LDVR.LT.1 .OR. ( ILVR .AND. LDVR.LT.N ) ) THEN
         INFO = -13
      ELSE IF( LWORK.LT.MAX( 1, 2*N ) .AND. .NOT.LQUERY ) THEN
         INFO = -15
      END IF
*
*     Compute workspace
*
      IF( INFO.EQ.0 ) THEN
         CALL CGEQRF( N, N, B, LDB, WORK, WORK, -1, IERR )
         LWKOPT = MAX( N,  N+INT( WORK( 1 ) ) )
         CALL CUNMQR( 'L', 'C', N, N, N, B, LDB, WORK, A, LDA, WORK,
     $                -1, IERR )
         LWKOPT = MAX( LWKOPT, N+INT( WORK( 1 ) ) )
         IF( ILVL ) THEN
            CALL CUNGQR( N, N, N, VL, LDVL, WORK, WORK, -1, IERR )
            LWKOPT = MAX( LWKOPT, N+INT( WORK( 1 ) ) )
         END IF
         IF( ILV ) THEN
            CALL CGGHD3( JOBVL, JOBVR, N, 1, N, A, LDA, B, LDB, VL,
     $                   LDVL, VR, LDVR, WORK, -1, IERR )
            LWKOPT = MAX( LWKOPT, N+INT( WORK( 1 ) ) )
            CALL CHGEQZ( 'S', JOBVL, JOBVR, N, 1, N, A, LDA, B, LDB,
     $                   ALPHA, BETA, VL, LDVL, VR, LDVR, WORK,
     $                   -1, WORK, IERR )
            LWKOPT = MAX( LWKOPT, N+INT( WORK( 1 ) ) )
         ELSE
            CALL CGGHD3( 'N', 'N', N, 1, N, A, LDA, B, LDB, VL, LDVL,
     $                   VR, LDVR, WORK, -1, IERR )
            LWKOPT = MAX( LWKOPT, N+INT( WORK( 1 ) ) )
```

```
            CALL CHGEQZ( 'E', JOBVL, JOBVR, N, 1, N, A, LDA, B, LDB,
     $                   ALPHA, BETA, VL, LDVL, VR, LDVR, WORK,
     $                   -1, WORK, IERR )
            LWKOPT = MAX( LWKOPT, N+INT( WORK( 1 ) ) )
         END IF
         WORK( 1 ) = CMPLX( LWKOPT )
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGGEV3 ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
*
      EPS = SLAMCH( 'E' )*SLAMCH( 'B' )
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = CLANGE( 'M', N, N, A, LDA, RWORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
      IF( ILASCL )
     $   CALL CLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = CLANGE( 'M', N, N, B, LDB, RWORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
```

```
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
     $   CALL CLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrices A, B to isolate eigenvalues if possible
*
      ILEFT = 1
      IRIGHT = N + 1
      IRWRK = IRIGHT + N
      CALL CGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, RWORK( ILEFT ),
     $             RWORK( IRIGHT ), RWORK( IRWRK ), IERR )
*
*     Reduce B to triangular form (QR decomposition of B)
*
      IROWS = IHI + 1 - ILO
      IF( ILV ) THEN
         ICOLS = N + 1 - ILO
      ELSE
         ICOLS = IROWS
      END IF
      ITAU = 1
      IWRK = ITAU + IROWS
      CALL CGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to matrix A
*
      CALL CUNMQR( 'L', 'C', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VL
*
      IF( ILVL ) THEN
         CALL CLASET( 'Full', N, N, CZERO, CONE, VL, LDVL )
         IF( IROWS.GT.1 ) THEN
            CALL CLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VL( ILO+1, ILO ), LDVL )
         END IF
         CALL CUNGQR( IROWS, IROWS, IROWS, VL( ILO, ILO ), LDVL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Initialize VR
*
      IF( ILVR )
     $   CALL CLASET( 'Full', N, N, CZERO, CONE, VR, LDVR )
```

```
*
*     Reduce to generalized Hessenberg form
*
      IF( ILV ) THEN
*
*        Eigenvectors requested -- work on whole matrix.
*
         CALL CGGHD3( JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB, VL,
     $                LDVL, VR, LDVR, WORK( IWRK ), LWORK+1-IWRK,
     $                IERR )
      ELSE
         CALL CGGHD3( 'N', 'N', IROWS, 1, IROWS, A( ILO, ILO ), LDA,
     $                B( ILO, ILO ), LDB, VL, LDVL, VR, LDVR,
     $                WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Perform QZ algorithm (Compute eigenvalues, and optionally, the
*     Schur form and Schur vectors)
*
      IWRK = ITAU
      IF( ILV ) THEN
         CHTEMP = 'S'
      ELSE
         CHTEMP = 'E'
      END IF
      CALL CHGEQZ( CHTEMP, JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHA, BETA, VL, LDVL, VR, LDVR, WORK( IWRK ),
     $             LWORK+1-IWRK, RWORK( IRWRK ), IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 70
      END IF
*
*     Compute Eigenvectors
*
      IF( ILV ) THEN
         IF( ILVL ) THEN
            IF( ILVR ) THEN
               CHTEMP = 'B'
            ELSE
               CHTEMP = 'L'
            END IF
         ELSE
            CHTEMP = 'R'
```

```
         END IF
*
         CALL CTGEVC( CHTEMP, 'B', LDUMMA, N, A, LDA, B, LDB, VL, LDVL,
     $               VR, LDVR, N, IN, WORK( IWRK ), RWORK( IRWRK ),
     $               IERR )
         IF( IERR.NE.0 ) THEN
            INFO = N + 2
            GO TO 70
         END IF
*
*        Undo balancing on VL and VR and normalization
*
         IF( ILVL ) THEN
            CALL CGGBAK( 'P', 'L', N, ILO, IHI, RWORK( ILEFT ),
     $                   RWORK( IRIGHT ), N, VL, LDVL, IERR )
            DO 30 JC = 1, N
               TEMP = ZERO
               DO 10 JR = 1, N
                  TEMP = MAX( TEMP, ABS1( VL( JR, JC ) ) )
   10          CONTINUE
               IF( TEMP.LT.SMLNUM )
     $            GO TO 30
               TEMP = ONE / TEMP
               DO 20 JR = 1, N
                  VL( JR, JC ) = VL( JR, JC )*TEMP
   20          CONTINUE
   30       CONTINUE
         END IF
         IF( ILVR ) THEN
            CALL CGGBAK( 'P', 'R', N, ILO, IHI, RWORK( ILEFT ),
     $                   RWORK( IRIGHT ), N, VR, LDVR, IERR )
            DO 60 JC = 1, N
               TEMP = ZERO
               DO 40 JR = 1, N
                  TEMP = MAX( TEMP, ABS1( VR( JR, JC ) ) )
   40          CONTINUE
               IF( TEMP.LT.SMLNUM )
     $            GO TO 60
               TEMP = ONE / TEMP
               DO 50 JR = 1, N
                  VR( JR, JC ) = VR( JR, JC )*TEMP
   50          CONTINUE
   60       CONTINUE
         END IF
      END IF
*
*     Undo scaling if necessary
*
   70 CONTINUE
*
```

```
      IF( ILASCL )
     $   CALL CLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHA, N, IERR )
*
      IF( ILBSCL )
     $   CALL CLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
*
      WORK( 1 ) = CMPLX( LWKOPT )
      RETURN
*
*     End of CGGEV3
*
      END
```

———-

26.11.4 lisp code

— **cggev3.lisp** —

```
(let* ((zero 0.0)
       (one 1.0)
       (czero (f2cl-lib:cmplx 0.0 0.0))
       (cone (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (f2cl-lib:complex8) czero)
           (type (f2cl-lib:complex8) cone)
           (ignorable zero one czero cone))
  (defun cggev3
         (jobvl jobvr n a lda b ldb$ alpha beta vl ldvl vr ldvr work lwork
          rwork info)
    (declare (type (array single-float (*)) rwork)
             (type (array f2cl-lib:complex8 (*)) work vr vl beta alpha b a)
             (type (f2cl-lib:integer4) info lwork ldvr ldvl ldb$ lda n)
             (type (string *) jobvr jobvl))
    (f2cl-lib:with-multi-array-data
        ((jobvl character jobvl-%data% jobvl-%offset%)
         (jobvr character jobvr-%data% jobvr-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (alpha f2cl-lib:complex8 alpha-%data% alpha-%offset%)
         (beta f2cl-lib:complex8 beta-%data% beta-%offset%)
         (vl f2cl-lib:complex8 vl-%data% vl-%offset%)
         (vr f2cl-lib:complex8 vr-%data% vr-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%))
      (labels ((abs1 (x)
                 (+ (abs (f2cl-lib:freal x)) (abs (f2cl-lib:aimag x)))))
```

```
(declare
 (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
  abs1))
(prog ((ldumma (make-array 1 :element-type 't)) (x #C(0.0 0.0))
       (anrm 0.0) (anrmto 0.0) (bignum 0.0) (bnrm 0.0) (bnrmto 0.0)
       (eps 0.0) (smlnum 0.0) (temp 0.0) (icols 0) (ierr 0) (ihi 0)
       (ijobvl 0) (ijobvr 0) (ileft 0) (ilo 0) (in 0) (iright 0)
       (irows 0) (irwrk 0) (itau 0) (iwrk 0) (jc 0) (jr 0) (lwkopt 0)
       (chtemp
        (make-array '(1) :element-type 'character :initial-element
                    #\ ))
       (ilascl nil) (ilbscl nil) (ilv nil) (ilvl nil) (ilvr nil)
       (lquery nil))
  (declare (type (array f2cl-lib:logical (1)) ldumma)
           (type (f2cl-lib:complex8) x)
           (type (single-float) anrm anrmto bignum bnrm bnrmto eps
            smlnum temp)
           (type (f2cl-lib:integer4) icols ierr ihi ijobvl ijobvr ileft
            ilo in iright irows irwrk itau iwrk jc jr lwkopt)
           (type (string 1) chtemp)
           (type f2cl-lib:logical ilascl ilbscl ilv ilvl ilvr lquery))
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame jobvl "N")
      (declare (ignore var-1))
      (when var-0 (setf jobvl var-0))
      ret-val)
    (setf ijobvl 1) (setf ilvl f2cl-lib:%false%))
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame jobvl "V")
      (declare (ignore var-1))
      (when var-0 (setf jobvl var-0))
      ret-val)
    (setf ijobvl 2) (setf ilvl f2cl-lib:%true%))
   (t (setf ijobvl -1) (setf ilvl f2cl-lib:%false%)))
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame jobvr "N")
      (declare (ignore var-1))
      (when var-0 (setf jobvr var-0))
      ret-val)
    (setf ijobvr 1) (setf ilvr f2cl-lib:%false%))
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame jobvr "V")
      (declare (ignore var-1))
      (when var-0 (setf jobvr var-0))
      ret-val)
    (setf ijobvr 2) (setf ilvr f2cl-lib:%true%))
   (t (setf ijobvr -1) (setf ilvr f2cl-lib:%false%)))
  (setf ilv (or ilvl ilvr))
```

```
(setf info 0)
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((<= ijobvl 0) (setf info -1)) ((<= ijobvr 0) (setf info -2))
      ((< n 0) (setf info -3))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -5))
      ((< ldb$
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -7))
      ((or (< ldvl 1) (and ilvl (< ldvl n))) (setf info -11))
      ((or (< ldvr 1) (and ilvr (< ldvr n))) (setf info -13))
      ((and
        (< lwork
           (max (the f2cl-lib:integer4 1)
                (the f2cl-lib:integer4 (f2cl-lib:int-mul 2 n))))
        (not lquery))
       (setf info -15)))
(cond
 ((= info 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (cgeqrf n n b ldb$ work work -1 ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf n var-0)
    (setf n var-1)
    (setf ldb$ var-3)
    (setf ierr var-7))
  (setf lwkopt
          (max (the f2cl-lib:integer4 n)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add n
                                      (f2cl-lib:int
                                       (f2cl-lib:fref work-%data% (1)
                                                      ((1 *))
                                                      work-%offset%))))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (cunmqr "L" "C" n n n b ldb$ work a lda work -1 ierr)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-6 (setf ldb$ var-6))
    (when var-9 (setf lda var-9))
    (when var-12 (setf ierr var-12)))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkopt)
               (the f2cl-lib:integer4
```

```
                    (f2cl-lib:int-add n
                                      (f2cl-lib:int
                                       (f2cl-lib:fref work-%data% (1)
                                                      ((1 *))
                                                      work-%offset%))))))
(cond
 (ilvl
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (cungqr n n n vl ldvl work work -1 ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf n var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldvl var-4))
    (when var-8 (setf ierr var-8)))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkopt)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add n
                                      (f2cl-lib:int
                                       (f2cl-lib:fref work-%data%
                                                      (1) ((1 *))
                                                      work-%offset%))))))))
(cond
 (ilv
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (cgghd3 jobvl jobvr n 1 n a lda b ldb$ vl ldvl vr ldvr work
       -1 ierr)
    (declare (ignore var-3 var-5 var-7 var-9 var-11 var-13 var-14))
    (when var-0 (setf jobvl var-0))
    (when var-1 (setf jobvr var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf n var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-15 (setf ierr var-15)))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkopt)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add n
                                      (f2cl-lib:int
                                       (f2cl-lib:fref work-%data%
                                                      (1) ((1 *))
                                                      work-%offset%))))))
  (multiple-value-bind
```

```
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17
       var-18 var-19)
      (chgeqz "S" jobvl jobvr n 1 n a lda b ldb$ alpha beta vl ldvl
       vr ldvr work -1 work ierr)
    (declare
     (ignore var-0 var-4 var-6 var-8 var-10 var-11 var-12 var-14
      var-16 var-17 var-18))
    (when var-1 (setf jobvl var-1))
    (when var-2 (setf jobvr var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf n var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldb$ var-9))
    (when var-13 (setf ldvl var-13))
    (when var-15 (setf ldvr var-15))
    (when var-19 (setf ierr var-19)))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkopt)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add n
                                      (f2cl-lib:int
                                       (f2cl-lib:fref work-%data%
                                                      (1) ((1 *))
                                                      work-%offset%)))))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (cgghd3 "N" "N" n 1 n a lda b ldb$ vl ldvl vr ldvr work -1
       ierr)
    (declare
     (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-13
      var-14))
    (when var-2 (setf n var-2))
    (when var-4 (setf n var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-15 (setf ierr var-15)))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkopt)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add n
                                      (f2cl-lib:int
                                       (f2cl-lib:fref work-%data%
                                                      (1) ((1 *))
                                                      work-%offset%))))))
  (multiple-value-bind
```

```
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17
           var-18 var-19)
          (chgeqz "E" jobvl jobvr n 1 n a lda b ldb$ alpha beta vl ldvl
           vr ldvr work -1 work ierr)
        (declare
         (ignore var-0 var-4 var-6 var-8 var-10 var-11 var-12 var-14
          var-16 var-17 var-18))
        (when var-1 (setf jobvl var-1))
        (when var-2 (setf jobvr var-2))
        (when var-3 (setf n var-3))
        (when var-5 (setf n var-5))
        (when var-7 (setf lda var-7))
        (when var-9 (setf ldb$ var-9))
        (when var-13 (setf ldvl var-13))
        (when var-15 (setf ldvr var-15))
        (when var-19 (setf ierr var-19)))
      (setf lwkopt
              (max (the f2cl-lib:integer4 lwkopt)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add n
                                          (f2cl-lib:int
                                           (f2cl-lib:fref work-%data%
                                                          (1) ((1 *))
                                                          work-%offset%))))))))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (f2cl-lib:cmplx lwkopt))))
  (cond
   ((/= info 0) (xerbla "CGGEV3 " (f2cl-lib:int-sub info))
    (go end_label))
   (lquery (go end_label)))
  (if (= n 0)
      (go end_label))
  (setf eps (* (slamch "E") (slamch "B")))
  (setf smlnum (slamch "S"))
  (setf bignum (/ one smlnum))
  (multiple-value-bind (var-0 var-1)
      (slabad smlnum bignum)
    (declare (ignore))
    (when var-0 (setf smlnum var-0))
    (when var-1 (setf bignum var-1)))
  (setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
  (setf bignum (/ one smlnum))
  (setf anrm
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
              (clange "M" n n a lda rwork)
            (declare (ignore var-0 var-3 var-5))
            (when var-1 (setf n var-1))
            (when var-2 (setf n var-2))
```

```
          (when var-4 (setf lda var-4))
          ret-val))
(setf ilascl f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf anrmto smlnum)
  (setf ilascl f2cl-lib:%true%))
 ((> anrm bignum) (setf anrmto bignum)
  (setf ilascl f2cl-lib:%true%)))
(if ilascl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (clascl "G" 0 0 anrm anrmto n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf anrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "M" n n b ldb$ rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ilbscl f2cl-lib:%false%)
(cond
 ((and (> bnrm zero) (< bnrm smlnum)) (setf bnrmto smlnum)
  (setf ilbscl f2cl-lib:%true%))
 ((> bnrm bignum) (setf bnrmto bignum)
  (setf ilbscl f2cl-lib:%true%)))
(if ilbscl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (clascl "G" 0 0 bnrm bnrmto n n b ldb$ ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bnrm var-3))
      (when var-4 (setf bnrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf ierr var-9))))
(setf ileft 1)
(setf iright (f2cl-lib:int-add n 1))
(setf irwrk (f2cl-lib:int-add iright n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
     var-10 var-11)
    (cggbal "P" n a lda b ldb$ ilo ihi
     (f2cl-lib:array-slice rwork-%data% single-float (ileft) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (iright) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (irwrk) ((1 *))
                           rwork-%offset%)
     ierr)
  (declare (ignore var-0 var-1 var-2 var-4 var-8 var-9 var-10))
  (setf lda var-3)
  (setf ldb$ var-5)
  (setf ilo var-6)
  (setf ihi var-7)
  (setf ierr var-11))
(setf irows (f2cl-lib:int-sub (f2cl-lib:int-add ihi 1) ilo))
(cond
 (ilv (setf icols (f2cl-lib:int-sub (f2cl-lib:int-add n 1) ilo)))
 (t (setf icols irows)))
(setf itau 1)
(setf iwrk (f2cl-lib:int-add itau irows))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgeqrf irows icols
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf irows var-0)
  (setf icols var-1)
  (setf ldb$ var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (cunmqr "L" "C" irows icols irows
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (ilo ilo)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
```

```
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf irows var-2))
    (when var-3 (setf icols var-3))
    (when var-4 (setf irows var-4))
    (when var-6 (setf ldb$ var-6))
    (when var-9 (setf lda var-9))
    (when var-12 (setf ierr var-12)))
  (cond
   (ilvl
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "Full" n n czero cone vl ldvl)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldvl var-6)))
    (cond
     ((> irows 1)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (clacpy "L" (f2cl-lib:int-sub irows 1)
           (f2cl-lib:int-sub irows 1)
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                 ((+ ilo 1) ilo) ((1 ldb$) (1 *))
                                 b-%offset%)
           ldb$
           (f2cl-lib:array-slice vl-%data% f2cl-lib:complex8
                                 ((+ ilo 1) ilo) ((1 ldvl) (1 *))
                                 vl-%offset%)
           ldvl)
        (declare (ignore var-0 var-1 var-2 var-3 var-5))
        (when var-4 (setf ldb$ var-4))
        (when var-6 (setf ldvl var-6)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (cungqr irows irows irows
         (f2cl-lib:array-slice vl-%data% f2cl-lib:complex8 (ilo ilo)
                               ((1 ldvl) (1 *)) vl-%offset%)
         ldvl
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
      (declare (ignore var-3 var-5 var-6 var-7))
      (when var-0 (setf irows var-0))
      (when var-1 (setf irows var-1))
      (when var-2 (setf irows var-2))
```

```
    (when var-4 (setf ldvl var-4))
    (when var-8 (setf ierr var-8)))))
(if ilvr
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "Full" n n czero cone vr ldvr)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldvr var-6))))
(cond
 (ilv
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (cgghd3 jobvl jobvr n ilo ihi a lda b ldb$ vl ldvl vr ldvr
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
    (declare (ignore var-5 var-7 var-9 var-11 var-13 var-14))
    (when var-0 (setf jobvl var-0))
    (when var-1 (setf jobvr var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-15 (setf ierr var-15))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (cgghd3 "N" "N" irows 1 irows
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (ilo ilo)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (ilo ilo)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$ vl ldvl vr ldvr
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
    (declare
     (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-13
      var-14))
    (when var-2 (setf irows var-2))
    (when var-4 (setf irows var-4))
```

```
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-15 (setf ierr var-15)))))
(setf iwrk itau)
(cond (ilv (f2cl-lib:f2cl-set-string chtemp "S" (string 1)))
      (t (f2cl-lib:f2cl-set-string chtemp "E" (string 1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18
     var-19)
    (chgeqz chtemp jobvl jobvr n ilo ihi a lda b ldb$ alpha beta vl
     ldvl vr ldvr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk)
     (f2cl-lib:array-slice rwork-%data% single-float (irwrk) ((1 *))
                           rwork-%offset%)
     ierr)
  (declare
   (ignore var-6 var-8 var-10 var-11 var-12 var-14 var-16 var-17
    var-18))
  (when var-0 (setf chtemp var-0))
  (when var-1 (setf jobvl var-1))
  (when var-2 (setf jobvr var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf ilo var-4))
  (when var-5 (setf ihi var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldb$ var-9))
  (when var-13 (setf ldvl var-13))
  (when var-15 (setf ldvr var-15))
  (when var-19 (setf ierr var-19)))
(cond
 ((/= ierr 0)
  (cond ((and (> ierr 0) (<= ierr n)) (setf info ierr))
        ((and (> ierr n) (<= ierr (f2cl-lib:int-mul 2 n)))
         (setf info (f2cl-lib:int-sub ierr n)))
        (t (setf info (f2cl-lib:int-add n 1))))
  (go label70)))
(cond
 (ilv
  (cond
   (ilvl
    (cond (ilvr (f2cl-lib:f2cl-set-string chtemp "B" (string 1)))
          (t (f2cl-lib:f2cl-set-string chtemp "L" (string 1)))))
   (t (f2cl-lib:f2cl-set-string chtemp "R" (string 1))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
      var-10 var-11 var-12 var-13 var-14 var-15 var-16)
     (ctgevc chtemp "B" ldumma n a lda b ldb$ vl ldvl vr ldvr n in
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice rwork-%data% single-float (irwrk)
                            ((1 *)) rwork-%offset%)
      ierr)
   (declare
    (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-14 var-15))
   (when var-0 (setf chtemp var-0))
   (when var-3 (setf n var-3))
   (when var-5 (setf lda var-5))
   (when var-7 (setf ldb$ var-7))
   (when var-9 (setf ldvl var-9))
   (when var-11 (setf ldvr var-11))
   (when var-12 (setf n var-12))
   (when var-13 (setf in var-13))
   (when var-16 (setf ierr var-16)))
 (cond
  ((/= ierr 0) (setf info (f2cl-lib:int-add n 2)) (go label70)))
 (cond
  (ilvl
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (cggbak "P" "L" n ilo ihi
        (f2cl-lib:array-slice rwork-%data% single-float (ileft)
                              ((1 *)) rwork-%offset%)
        (f2cl-lib:array-slice rwork-%data% single-float (iright)
                              ((1 *)) rwork-%offset%)
        n vl ldvl ierr)
     (declare
      (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
     (setf n var-7)
     (setf ldvl var-9)
     (setf ierr var-10))
   (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                 ((> jc n) nil)
     (tagbody
       (setf temp zero)
       (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                     ((> jr n) nil)
         (tagbody
           (setf temp
                   (max temp
                        (abs1
                         (f2cl-lib:fref vl-%data% (jr jc)
                                        ((1 ldvl) (1 *))
                                        vl-%offset%))))
          label10))
```

```
      (if (< temp smlnum)
          (go label30))
      (setf temp (/ one temp))
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                               vl-%offset%)
                (*
                 (f2cl-lib:fref vl-%data% (jr jc)
                                ((1 ldvl) (1 *)) vl-%offset%)
                 temp))
         label20))
     label30))))
(cond
 (ilvr
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (cggbak "P" "R" n ilo ihi
       (f2cl-lib:array-slice rwork-%data% single-float (ileft)
                             ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float (iright)
                             ((1 *)) rwork-%offset%)
       n vr ldvr ierr)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
    (setf n var-7)
    (setf ldvr var-9)
    (setf ierr var-10))
  (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                ((> jc n) nil)
    (tagbody
      (setf temp zero)
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf temp
                  (max temp
                       (abs1
                        (f2cl-lib:fref vr-%data% (jr jc)
                                       ((1 ldvr) (1 *))
                                       vr-%offset%))))
         label40))
      (if (< temp smlnum)
          (go label60))
      (setf temp (/ one temp))
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
```

```
                    (setf (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                         vr-%offset%)
                            (*
                             (f2cl-lib:fref vr-%data% (jr jc)
                                            ((1 ldvr) (1 *)) vr-%offset%)
                             temp))
                       label50))
                 label60))))))
         label70
          (if ilascl
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (clascl "G" 0 0 anrmto anrm n 1 alpha n ierr)
                (declare (ignore var-0 var-1 var-2 var-6 var-7))
                (when var-3 (setf anrmto var-3))
                (when var-4 (setf anrm var-4))
                (when var-5 (setf n var-5))
                (when var-8 (setf n var-8))
                (when var-9 (setf ierr var-9))))
          (if ilbscl
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (clascl "G" 0 0 bnrmto bnrm n 1 beta n ierr)
                (declare (ignore var-0 var-1 var-2 var-6 var-7))
                (when var-3 (setf bnrmto var-3))
                (when var-4 (setf bnrm var-4))
                (when var-5 (setf n var-5))
                (when var-8 (setf n var-8))
                (when var-9 (setf ierr var-9))))
          (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                  (f2cl-lib:cmplx lwkopt))
          (go end_label)
         end_label
          (return
           (values jobvl jobvr n nil lda nil ldb$ nil nil nil ldvl nil ldvr nil
                   nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cggev3
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
```

```
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (array fortran-to-lisp::complex8
   (*))
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::jobvl
  fortran-to-lisp::jobvr
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$ nil nil nil
  fortran-to-lisp::ldvl nil
  fortran-to-lisp::ldvr nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::cggbak
  fortran-to-lisp::cggbal
  fortran-to-lisp::cgeqrf))))
```

———-

26.12 cggevx

26.12.1 Axiom unit tests

— cggevx.input —

```
)set break resume
)sys rm -f cggevx.output
)spool cggevx.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

26.12.2 Axiom help page

— **cggevx.help** —

```
=====================================================================
cggevx examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cggevx (BALANC, JOBVL, JOBVR, SENSE, N, A, LDA, B, LDB,
           ALPHA, BETA, VL, LDVL, VR, LDVR, ILO, IHI, LSCALE, RSCALE, ABNRM,
           BBNRM, RCONDE, RCONDV, WORK, LWORK, RWORK, IWORK, BWORK, INFO)
            CGGEVX computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for GE matrices

Function/Subroutine Documentation
   subroutine cggevx (character BALANC, character JOBVL, character JOBVR,
       character SENSE, integer N, complex, dimension( lda, * ) A, integer
       LDA, complex, dimension( ldb, * ) B, integer LDB, complex, dimension( *
       ) ALPHA, complex, dimension( * ) BETA, complex, dimension( ldvl, * )
       VL, integer LDVL, complex, dimension( ldvr, * ) VR, integer LDVR,
       integer ILO, integer IHI, real, dimension( * ) LSCALE, real, dimension(
       * ) RSCALE, real ABNRM, real BBNRM, real, dimension( * ) RCONDE, real,
       dimension( * ) RCONDV, complex, dimension( * ) WORK, integer LWORK,
       real, dimension( * ) RWORK, integer, dimension( * ) IWORK, logical,
       dimension( * ) BWORK, integer INFO)
        CGGEVX computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for GE matrices

Purpose:
   CGGEVX computes for a pair of N-by-N complex nonsymmetric matrices
   (A,B) the generalized eigenvalues, and optionally, the left and/or
   right generalized eigenvectors.

   Optionally, it also computes a balancing transformation to improve
   the conditioning of the eigenvalues and eigenvectors (ILO, IHI,
   LSCALE, RSCALE, ABNRM, and BBNRM), reciprocal condition numbers for
   the eigenvalues (RCONDE), and reciprocal condition numbers for the
```

```
   right eigenvectors (RCONDV).

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar
   lambda or a ratio alpha/beta = lambda, such that A - lambda*B is
   singular. It is usually represented as the pair (alpha,beta), as
   there is a reasonable interpretation for beta=0, and even for both
   being zero.

   The right eigenvector v(j) corresponding to the eigenvalue lambda(j)
   of (A,B) satisfies
      A * v(j) = lambda(j) * B * v(j) .
   The left eigenvector u(j) corresponding to the eigenvalue lambda(j)
   of (A,B) satisfies
      u(j)**H * A  = lambda(j) * u(j)**H * B.
   where u(j)**H is the conjugate-transpose of u(j).

Parameters:

 BALANC
   BALANC is CHARACTER*1
   Specifies the balance option to be performed:
   = 'N':  do not diagonally scale or permute;
   = 'P':  permute only;
   = 'S':  scale only;
   = 'B':  both permute and scale.
   Computed reciprocal condition numbers will be for the
   matrices after permuting and/or balancing. Permuting does
   not change condition numbers (in exact arithmetic), but balancing does.

 JOBVL
   JOBVL is CHARACTER*1
   = 'N':  do not compute the left generalized eigenvectors;
   = 'V':  compute the left generalized eigenvectors.

 JOBVR
   JOBVR is CHARACTER*1
   = 'N':  do not compute the right generalized eigenvectors;
   = 'V':  compute the right generalized eigenvectors.

 SENSE
   SENSE is CHARACTER*1
   Determines which reciprocal condition numbers are computed.
   = 'N': none are computed;
   = 'E': computed for eigenvalues only;
   = 'V': computed for eigenvectors only;
   = 'B': computed for eigenvalues and eigenvectors.

 N
   N is INTEGER
   The order of the matrices A, B, VL, and VR.  N >= 0.
```

```
A
  A is COMPLEX array, dimension (LDA, N)
  On entry, the matrix A in the pair (A,B).
  On exit, A has been overwritten. If JOBVL='V' or JOBVR='V'
  or both, then A contains the first part of the complex Schur
  form of the "balanced" versions of the input A and B.

LDA
  LDA is INTEGER
  The leading dimension of A.  LDA >= max(1,N).

B
  B is COMPLEX array, dimension (LDB, N)
  On entry, the matrix B in the pair (A,B).
  On exit, B has been overwritten. If JOBVL='V' or JOBVR='V'
  or both, then B contains the second part of the complex
  Schur form of the "balanced" versions of the input A and B.

LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).

ALPHA
  ALPHA is COMPLEX array, dimension (N)

BETA
  BETA is COMPLEX array, dimension (N)
  On exit, ALPHA(j)/BETA(j), j=1,...,N, will be the generalized
  eigenvalues.

  Note: the quotient ALPHA(j)/BETA(j) ) may easily over- or
  underflow, and BETA(j) may even be zero.  Thus, the user
  should avoid naively computing the ratio ALPHA/BETA.
  However, ALPHA will be always less than and usually
  comparable with norm(A) in magnitude, and BETA always less
  than and usually comparable with norm(B).

VL
  VL is COMPLEX array, dimension (LDVL,N)
  If JOBVL = 'V', the left generalized eigenvectors u(j) are
  stored one after another in the columns of VL, in the same
  order as their eigenvalues.
  Each eigenvector will be scaled so the largest component
  will have abs(real part) + abs(imag. part) = 1.
  Not referenced if JOBVL = 'N'.

LDVL
  LDVL is INTEGER
  The leading dimension of the matrix VL. LDVL >= 1, and
```

```
  if JOBVL = 'V', LDVL >= N.

VR
  VR is COMPLEX array, dimension (LDVR,N)
  If JOBVR = 'V', the right generalized eigenvectors v(j) are
  stored one after another in the columns of VR, in the same
  order as their eigenvalues.
  Each eigenvector will be scaled so the largest component
  will have abs(real part) + abs(imag. part) = 1.
  Not referenced if JOBVR = 'N'.

LDVR
  LDVR is INTEGER
  The leading dimension of the matrix VR. LDVR >= 1, and
  if JOBVR = 'V', LDVR >= N.

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER
  ILO and IHI are integer values such that on exit
  A(i,j) = 0 and B(i,j) = 0 if i > j and
  j = 1,...,ILO-1 or i = IHI+1,...,N.
  If BALANC = 'N' or 'S', ILO = 1 and IHI = N.

LSCALE
  LSCALE is REAL array, dimension (N)
  Details of the permutations and scaling factors applied
  to the left side of A and B.  If PL(j) is the index of the
  row interchanged with row j, and DL(j) is the scaling
  factor applied to row j, then
     LSCALE(j) = PL(j)  for j = 1,...,ILO-1
               = DL(j)  for j = ILO,...,IHI
               = PL(j)  for j = IHI+1,...,N.
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

RSCALE
  RSCALE is REAL array, dimension (N)
  Details of the permutations and scaling factors applied
  to the right side of A and B.  If PR(j) is the index of the
  column interchanged with column j, and DR(j) is the scaling
  factor applied to column j, then
     RSCALE(j) = PR(j)  for j = 1,...,ILO-1
               = DR(j)  for j = ILO,...,IHI
               = PR(j)  for j = IHI+1,...,N
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.
```

```
ABNRM
  ABNRM is REAL
  The one-norm of the balanced matrix A.

BBNRM
  BBNRM is REAL
  The one-norm of the balanced matrix B.

RCONDE
  RCONDE is REAL array, dimension (N)
  If SENSE = 'E' or 'B', the reciprocal condition numbers of
  the eigenvalues, stored in consecutive elements of the array.
  If SENSE = 'N' or 'V', RCONDE is not referenced.

RCONDV
  RCONDV is REAL array, dimension (N)
  If SENSE = 'V' or 'B', the estimated reciprocal condition
  numbers of the eigenvectors, stored in consecutive elements
  of the array. If the eigenvalues cannot be reordered to
  compute RCONDV(j), RCONDV(j) is set to 0; this can only occur
  when the true value would be very small anyway.
  If SENSE = 'N' or 'E', RCONDV is not referenced.

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK >= max(1,2*N).
  If SENSE = 'E', LWORK >= max(1,4*N).
  If SENSE = 'V' or 'B', LWORK >= max(1,2*N*N+2*N).

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is REAL array, dimension (lrwork)
  lrwork must be at least max(1,6*N) if BALANC = 'S' or 'B',
  and at least max(1,2*N) otherwise.
  Real workspace.

IWORK
  IWORK is INTEGER array, dimension (N+2)
  If SENSE = 'E', IWORK is not referenced.

BWORK
  BWORK is LOGICAL array, dimension (N)
```

```
  If SENSE = 'N', BWORK is not referenced.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  = 1,...,N:
        The QZ iteration failed.  No eigenvectors have been
        calculated, but ALPHA(j) and BETA(j) should be correct
        for j=INFO+1,...,N.
  > N:  =N+1: other than QZ iteration failed in CHGEQZ.
        =N+2: error return from CTGEVC.

  Balancing a matrix pair (A,B) includes, first, permuting rows and
  columns to isolate eigenvalues, second, applying diagonal similarity
  transformation to the rows and columns to make the rows and columns
  as close in norm as possible. The computed reciprocal condition
  numbers correspond to the balanced matrix. Permuting rows and columns
  will not change the condition numbers (in exact arithmetic) but
  diagonal scaling will.  For further explanation of balancing, see
  section 4.11.1.2 of LAPACK Users' Guide.

  An approximate error bound on the chordal distance between the i-th
  computed generalized eigenvalue w and the corresponding exact
  eigenvalue lambda is

     chord(w, lambda) <= EPS * norm(ABNRM, BBNRM) / RCONDE(I)

  An approximate error bound for the angle between the i-th computed
  eigenvector VL(i) or VR(i) is given by

     EPS * norm(ABNRM, BBNRM) / DIF(i).

  For further explanation of the reciprocal condition numbers RCONDE
  and RCONDV, see section 4.11 of LAPACK User's Guide.
```

———-

26.12.3 fortran code

— **cggevx.f** —

```
*  =====================================================================
      SUBROUTINE CGGEVX( BALANC, JOBVL, JOBVR, SENSE, N, A, LDA, B, LDB,
     $                   ALPHA, BETA, VL, LDVL, VR, LDVR, ILO, IHI,
     $                   LSCALE, RSCALE, ABNRM, BBNRM, RCONDE, RCONDV,
     $                   WORK, LWORK, RWORK, IWORK, BWORK, INFO )
```

```
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          BALANC, JOBVL, JOBVR, SENSE
      INTEGER            IHI, ILO, INFO, LDA, LDB, LDVL, LDVR, LWORK, N
      REAL               ABNRM, BBNRM
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      INTEGER            IWORK( * )
      REAL               LSCALE( * ), RCONDE( * ), RCONDV( * ),
     $                   RSCALE( * ), RWORK( * )
      COMPLEX            A( LDA, * ), ALPHA( * ), B( LDB, * ),
     $                   BETA( * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      COMPLEX            CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0E+0, 0.0E+0 ),
     $                   CONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            ILASCL, ILBSCL, ILV, ILVL, ILVR, LQUERY, NOSCL,
     $                   WANTSB, WANTSE, WANTSN, WANTSV
      CHARACTER          CHTEMP
      INTEGER            I, ICOLS, IERR, IJOBVL, IJOBVR, IN, IROWS,
     $                   ITAU, IWRK, IWRK1, J, JC, JR, M, MAXWRK, MINWRK
      REAL               ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS,
     $                   SMLNUM, TEMP
      COMPLEX            X
*     ..
*     .. Local Arrays ..
      LOGICAL            LDUMMA( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEQRF, CGGBAK, CGGBAL, CGGHRD, CHGEQZ, CLACPY,
     $                   CLASCL, CLASET, CTGEVC, CTGSNA, CUNGQR, CUNMQR,
     $                   SLABAD, SLASCL, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
```

```
      INTEGER            ILAENV
      REAL               CLANGE, SLAMCH
      EXTERNAL           LSAME, ILAENV, CLANGE, SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, MAX, REAL, SQRT
*     ..
*     .. Statement Functions ..
      REAL               ABS1
*     ..
*     .. Statement Function definitions ..
      ABS1( X ) = ABS( REAL( X ) ) + ABS( AIMAG( X ) )
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVL, 'N' ) ) THEN
         IJOBVL = 1
         ILVL = .FALSE.
      ELSE IF( LSAME( JOBVL, 'V' ) ) THEN
         IJOBVL = 2
         ILVL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVL = .FALSE.
      END IF
*
      IF( LSAME( JOBVR, 'N' ) ) THEN
         IJOBVR = 1
         ILVR = .FALSE.
      ELSE IF( LSAME( JOBVR, 'V' ) ) THEN
         IJOBVR = 2
         ILVR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVR = .FALSE.
      END IF
      ILV = ILVL .OR. ILVR
*
      NOSCL  = LSAME( BALANC, 'N' ) .OR. LSAME( BALANC, 'P' )
      WANTSN = LSAME( SENSE, 'N' )
      WANTSE = LSAME( SENSE, 'E' )
      WANTSV = LSAME( SENSE, 'V' )
      WANTSB = LSAME( SENSE, 'B' )
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
```

```
      IF( .NOT.( NOSCL .OR. LSAME( BALANC,'S' ) .OR.
     $    LSAME( BALANC, 'B' ) ) ) THEN
         INFO = -1
      ELSE IF( IJOBVL.LE.0 ) THEN
         INFO = -2
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -3
      ELSE IF( .NOT.( WANTSN .OR. WANTSE .OR. WANTSB .OR. WANTSV ) )
     $          THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDVL.LT.1 .OR. ( ILVL .AND. LDVL.LT.N ) ) THEN
         INFO = -13
      ELSE IF( LDVR.LT.1 .OR. ( ILVR .AND. LDVR.LT.N ) ) THEN
         INFO = -15
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV. The workspace is
*       computed assuming ILO = 1 and IHI = N, the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            MAXWRK = 1
         ELSE
            MINWRK = 2*N
            IF( WANTSE ) THEN
               MINWRK = 4*N
            ELSE IF( WANTSV .OR. WANTSB ) THEN
               MINWRK = 2*N*( N + 1)
            END IF
            MAXWRK = MINWRK
            MAXWRK = MAX( MAXWRK,
     $                    N + N*ILAENV( 1, 'CGEQRF', ' ', N, 1, N, 0 ) )
            MAXWRK = MAX( MAXWRK,
     $                    N + N*ILAENV( 1, 'CUNMQR', ' ', N, 1, N, 0 ) )
            IF( ILVL ) THEN
               MAXWRK = MAX( MAXWRK, N +
     $                       N*ILAENV( 1, 'CUNGQR', ' ', N, 1, N, 0 ) )
            END IF
```

```
         END IF
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -25
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGGEVX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = CLANGE( 'M', N, N, A, LDA, RWORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
      IF( ILASCL )
     $   CALL CLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = CLANGE( 'M', N, N, B, LDB, RWORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
```

```
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
     $   CALL CLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute and/or balance the matrix pair (A,B)
*     (Real Workspace: need 6*N if BALANC = 'S' or 'B', 1 otherwise)
*
      CALL CGGBAL( BALANC, N, A, LDA, B, LDB, ILO, IHI, LSCALE, RSCALE,
     $             RWORK, IERR )
*
*     Compute ABNRM and BBNRM
*
      ABNRM = CLANGE( '1', N, N, A, LDA, RWORK( 1 ) )
      IF( ILASCL ) THEN
         RWORK( 1 ) = ABNRM
         CALL SLASCL( 'G', 0, 0, ANRMTO, ANRM, 1, 1, RWORK( 1 ), 1,
     $                IERR )
         ABNRM = RWORK( 1 )
      END IF
*
      BBNRM = CLANGE( '1', N, N, B, LDB, RWORK( 1 ) )
      IF( ILBSCL ) THEN
         RWORK( 1 ) = BBNRM
         CALL SLASCL( 'G', 0, 0, BNRMTO, BNRM, 1, 1, RWORK( 1 ), 1,
     $                IERR )
         BBNRM = RWORK( 1 )
      END IF
*
*     Reduce B to triangular form (QR decomposition of B)
*     (Complex Workspace: need N, prefer N*NB )
*
      IROWS = IHI + 1 - ILO
      IF( ILV .OR. .NOT.WANTSN ) THEN
         ICOLS = N + 1 - ILO
      ELSE
         ICOLS = IROWS
      END IF
      ITAU = 1
      IWRK = ITAU + IROWS
      CALL CGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the unitary transformation to A
*     (Complex Workspace: need N, prefer N*NB)
*
      CALL CUNMQR( 'L', 'C', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
```

```
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VL and/or VR
*     (Workspace: need N, prefer N*NB)
*
      IF( ILVL ) THEN
         CALL CLASET( 'Full', N, N, CZERO, CONE, VL, LDVL )
         IF( IROWS.GT.1 ) THEN
            CALL CLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VL( ILO+1, ILO ), LDVL )
         END IF
         CALL CUNGQR( IROWS, IROWS, IROWS, VL( ILO, ILO ), LDVL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
      IF( ILVR )
     $   CALL CLASET( 'Full', N, N, CZERO, CONE, VR, LDVR )
*
*     Reduce to generalized Hessenberg form
*     (Workspace: none needed)
*
      IF( ILV .OR. .NOT.WANTSN ) THEN
*
*        Eigenvectors requested -- work on whole matrix.
*
         CALL CGGHRD( JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB, VL,
     $                LDVL, VR, LDVR, IERR )
      ELSE
         CALL CGGHRD( 'N', 'N', IROWS, 1, IROWS, A( ILO, ILO ), LDA,
     $                B( ILO, ILO ), LDB, VL, LDVL, VR, LDVR, IERR )
      END IF
*
*     Perform QZ algorithm (Compute eigenvalues, and optionally, the
*     Schur forms and Schur vectors)
*     (Complex Workspace: need N)
*     (Real Workspace: need N)
*
      IWRK = ITAU
      IF( ILV .OR. .NOT.WANTSN ) THEN
         CHTEMP = 'S'
      ELSE
         CHTEMP = 'E'
      END IF
*
      CALL CHGEQZ( CHTEMP, JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHA, BETA, VL, LDVL, VR, LDVR, WORK( IWRK ),
     $             LWORK+1-IWRK, RWORK, IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
```

```
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 90
      END IF
*
*     Compute Eigenvectors and estimate condition numbers if desired
*     CTGEVC: (Complex Workspace: need 2*N )
*             (Real Workspace:    need 2*N )
*     CTGSNA: (Complex Workspace: need 2*N*N if SENSE='V' or 'B')
*             (Integer Workspace: need N+2 )
*
      IF( ILV .OR. .NOT.WANTSN ) THEN
         IF( ILV ) THEN
            IF( ILVL ) THEN
               IF( ILVR ) THEN
                  CHTEMP = 'B'
               ELSE
                  CHTEMP = 'L'
               END IF
            ELSE
               CHTEMP = 'R'
            END IF
*
            CALL CTGEVC( CHTEMP, 'B', LDUMMA, N, A, LDA, B, LDB, VL,
     $                   LDVL, VR, LDVR, N, IN, WORK( IWRK ), RWORK,
     $                   IERR )
            IF( IERR.NE.0 ) THEN
               INFO = N + 2
               GO TO 90
            END IF
         END IF
*
         IF( .NOT.WANTSN ) THEN
*
*           compute eigenvectors (STGEVC) and estimate condition
*           numbers (STGSNA). Note that the definition of the condition
*           number is not invariant under transformation (u,v) to
*           (Q*u, Z*v), where (u,v) are eigenvectors of the generalized
*           Schur form (S,T), Q and Z are orthogonal matrices. In order
*           to avoid using extra 2*N*N workspace, we have to
*           re-calculate eigenvectors and estimate the condition numbers
*           one at a time.
*
            DO 20 I = 1, N
*
               DO 10 J = 1, N
                  BWORK( J ) = .FALSE.
```

```
   10          CONTINUE
               BWORK( I ) = .TRUE.
*
               IWRK = N + 1
               IWRK1 = IWRK + N
*
               IF( WANTSE .OR. WANTSB ) THEN
                  CALL CTGEVC( 'B', 'S', BWORK, N, A, LDA, B, LDB,
     $                         WORK( 1 ), N, WORK( IWRK ), N, 1, M,
     $                         WORK( IWRK1 ), RWORK, IERR )
                  IF( IERR.NE.0 ) THEN
                     INFO = N + 2
                     GO TO 90
                  END IF
               END IF
*
               CALL CTGSNA( SENSE, 'S', BWORK, N, A, LDA, B, LDB,
     $                      WORK( 1 ), N, WORK( IWRK ), N, RCONDE( I ),
     $                      RCONDV( I ), 1, M, WORK( IWRK1 ),
     $                      LWORK-IWRK1+1, IWORK, IERR )
*
   20       CONTINUE
         END IF
      END IF
*
*     Undo balancing on VL and VR and normalization
*     (Workspace: none needed)
*
      IF( ILVL ) THEN
         CALL CGGBAK( BALANC, 'L', N, ILO, IHI, LSCALE, RSCALE, N, VL,
     $                LDVL, IERR )
*
         DO 50 JC = 1, N
            TEMP = ZERO
            DO 30 JR = 1, N
               TEMP = MAX( TEMP, ABS1( VL( JR, JC ) ) )
   30       CONTINUE
            IF( TEMP.LT.SMLNUM )
     $         GO TO 50
            TEMP = ONE / TEMP
            DO 40 JR = 1, N
               VL( JR, JC ) = VL( JR, JC )*TEMP
   40       CONTINUE
   50    CONTINUE
      END IF
*
      IF( ILVR ) THEN
         CALL CGGBAK( BALANC, 'R', N, ILO, IHI, LSCALE, RSCALE, N, VR,
     $                LDVR, IERR )
         DO 80 JC = 1, N
```

```
            TEMP = ZERO
            DO 60 JR = 1, N
               TEMP = MAX( TEMP, ABS1( VR( JR, JC ) ) )
   60       CONTINUE
            IF( TEMP.LT.SMLNUM )
     $         GO TO 80
            TEMP = ONE / TEMP
            DO 70 JR = 1, N
               VR( JR, JC ) = VR( JR, JC )*TEMP
   70       CONTINUE
   80    CONTINUE
      END IF
*
*     Undo scaling if necessary
*
   90 CONTINUE
*
      IF( ILASCL )
     $   CALL CLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHA, N, IERR )
*
      IF( ILBSCL )
     $   CALL CLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of CGGEVX
*
      END
```

———-

26.12.4 lisp code

— **cggevx.lisp** —

```
(let* ((zero 0.0)
       (one 1.0)
       (czero (f2cl-lib:cmplx 0.0 0.0))
       (cone (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (f2cl-lib:complex8) czero)
           (type (f2cl-lib:complex8) cone)
           (ignorable zero one czero cone))
  (defun cggevx
         (balanc jobvl jobvr sense n a lda b ldb$ alpha beta vl ldvl vr ldvr
          ilo ihi lscale rscale abnrm bbnrm rconde rcondv work lwork rwork
```

```
      iwork bwork info)
(declare (type (array f2cl-lib:logical (*)) bwork)
         (type (array f2cl-lib:integer4 (*)) iwork)
         (type (single-float) bbnrm abnrm)
         (type (array single-float (*)) rwork rcondv rconde rscale lscale)
         (type (array f2cl-lib:complex8 (*)) work vr vl beta alpha b a)
         (type (f2cl-lib:integer4) info lwork ihi ilo ldvr ldvl ldb$ lda n)
         (type (string *) sense jobvr jobvl balanc))
(f2cl-lib:with-multi-array-data
    ((balanc character balanc-%data% balanc-%offset%)
     (jobvl character jobvl-%data% jobvl-%offset%)
     (jobvr character jobvr-%data% jobvr-%offset%)
     (sense character sense-%data% sense-%offset%)
     (a f2cl-lib:complex8 a-%data% a-%offset%)
     (b f2cl-lib:complex8 b-%data% b-%offset%)
     (alpha f2cl-lib:complex8 alpha-%data% alpha-%offset%)
     (beta f2cl-lib:complex8 beta-%data% beta-%offset%)
     (vl f2cl-lib:complex8 vl-%data% vl-%offset%)
     (vr f2cl-lib:complex8 vr-%data% vr-%offset%)
     (work f2cl-lib:complex8 work-%data% work-%offset%)
     (lscale single-float lscale-%data% lscale-%offset%)
     (rscale single-float rscale-%data% rscale-%offset%)
     (rconde single-float rconde-%data% rconde-%offset%)
     (rcondv single-float rcondv-%data% rcondv-%offset%)
     (rwork single-float rwork-%data% rwork-%offset%)
     (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%)
     (bwork f2cl-lib:logical bwork-%data% bwork-%offset%))
  (labels ((abs1 (x)
             (+ (abs (f2cl-lib:freal x)) (abs (f2cl-lib:aimag x)))))
    (declare
     (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
      abs1))
    (prog ((ldumma (make-array 1 :element-type 't)) (x #C(0.0 0.0))
           (anrm 0.0) (anrmto 0.0) (bignum 0.0) (bnrm 0.0) (bnrmto 0.0)
           (eps 0.0) (smlnum 0.0) (temp 0.0) (i 0) (icols 0) (ierr 0)
           (ijobvl 0) (ijobvr 0) (in 0) (irows 0) (itau 0) (iwrk 0)
           (iwrk1 0) (j 0) (jc 0) (jr 0) (m 0) (maxwrk 0) (minwrk 0)
           (chtemp
            (make-array '(1) :element-type 'character :initial-element
                        #\ ))
           (ilascl nil) (ilbscl nil) (ilv nil) (ilvl nil) (ilvr nil)
           (lquery nil) (noscl nil) (wantsb nil) (wantse nil) (wantsn nil)
           (wantsv nil))
     (declare (type (array f2cl-lib:logical (1)) ldumma)
              (type (f2cl-lib:complex8) x)
              (type (single-float) anrm anrmto bignum bnrm bnrmto eps
               smlnum temp)
              (type (f2cl-lib:integer4) i icols ierr ijobvl ijobvr in
               irows itau iwrk iwrk1 j jc jr m maxwrk minwrk)
              (type (string 1) chtemp)
```

```
         (type f2cl-lib:logical ilascl ilbscl ilv ilvl ilvr lquery
          noscl wantsb wantse wantsn wantsv))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvl "N")
    (declare (ignore var-1))
    (when var-0 (setf jobvl var-0))
    ret-val)
  (setf ijobvl 1) (setf ilvl f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvl "V")
    (declare (ignore var-1))
    (when var-0 (setf jobvl var-0))
    ret-val)
  (setf ijobvl 2) (setf ilvl f2cl-lib:%true%))
 (t (setf ijobvl -1) (setf ilvl f2cl-lib:%false%)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvr "N")
    (declare (ignore var-1))
    (when var-0 (setf jobvr var-0))
    ret-val)
  (setf ijobvr 1) (setf ilvr f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvr "V")
    (declare (ignore var-1))
    (when var-0 (setf jobvr var-0))
    ret-val)
  (setf ijobvr 2) (setf ilvr f2cl-lib:%true%))
 (t (setf ijobvr -1) (setf ilvr f2cl-lib:%false%)))
(setf ilv (or ilvl ilvr))
(setf noscl
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame balanc "N")
           (declare (ignore var-1))
           (when var-0 (setf balanc var-0))
           ret-val)
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame balanc "P")
           (declare (ignore var-1))
           (when var-0 (setf balanc var-0))
           ret-val)))
(setf wantsn
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "N")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wantse
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "E")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wantsv
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "V")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wantsb
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "B")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf info 0)
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond
 ((not
   (or noscl
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame balanc "S")
         (declare (ignore var-1))
         (when var-0 (setf balanc var-0))
         ret-val)
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame balanc "B")
         (declare (ignore var-1))
         (when var-0 (setf balanc var-0))
         ret-val)))
  (setf info -1))
 ((<= ijobvl 0) (setf info -2)) ((<= ijobvr 0) (setf info -3))
 ((not (or wantsn wantse wantsb wantsv)) (setf info -4))
 ((< n 0) (setf info -5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9))
 ((or (< ldvl 1) (and ilvl (< ldvl n))) (setf info -13))
 ((or (< ldvr 1) (and ilvr (< ldvr n))) (setf info -15)))
(cond
 ((= info 0)
  (cond ((= n 0) (setf minwrk 1) (setf maxwrk 1))
        (t (setf minwrk (f2cl-lib:int-mul 2 n))
         (cond (wantse (setf minwrk (f2cl-lib:int-mul 4 n)))
               ((or wantsv wantsb)
                (setf minwrk
                        (f2cl-lib:int-mul 2 n
```

```
                                    (f2cl-lib:int-add n 1)))))
(setf maxwrk minwrk)
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add n
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "CGEQRF"
                                                           " "
                                                           n
                                                           1
                                                           n
                                                           0)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-4
                                                          var-6))
                                                        (when
                                                            var-3
                                                          (setf n
                                                                var-3))
                                                        (when
                                                            var-5
                                                          (setf n
                                                                var-5))
                                                        ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add n
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
```

```
                                                                    var-3
                                                                    var-4
                                                                    var-5
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "CUNMQR"
                                                                    " "
                                                                    n
                                                                    1
                                                                    n
                                                                    0)
                                                                 (declare
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-4
                                                                   var-6))
                                                                 (when
                                                                     var-3
                                                                   (setf n
                                                                           var-3))
                                                                 (when
                                                                     var-5
                                                                   (setf n
                                                                           var-5))
                                                                 ret-val))))))
                                                 (cond
                                                  (ilvl
                                                   (setf maxwrk
                                                           (max (the f2cl-lib:integer4 maxwrk)
                                                                (the f2cl-lib:integer4
                                                                     (f2cl-lib:int-add n
                                                                                       (f2cl-lib:int-mul n
                                                                                                         (multiple-value-bind
                                                                                                             (ret-val
                                                                                                              var-0
                                                                                                              var-1
                                                                                                              var-2
                                                                                                              var-3
                                                                                                              var-4
                                                                                                              var-5
                                                                                                              var-6)
                                                                                                             (ilaenv
                                                                                                              1
                                                                                                              "CUNGQR"
                                                                                                              " "
                                                                                                              n
                                                                                                              1
```

```
                                                                    n
                                                                    0)
                                                                (declare
                                                                 (ignore
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-4
                                                                  var-6))
                                                                (when
                                                                    var-3
                                                                  (setf n
                                                                          var-3))
                                                                (when
                                                                    var-5
                                                                  (setf n
                                                                          var-5))
                                                               ret-val))))))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce maxwrk 'f2cl-lib:complex8))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -25)))))
(cond
 ((/= info 0) (xerbla "CGGEVX" (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(setf eps (slamch "P"))
(setf smlnum (slamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (slabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "M" n n a lda rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf ilascl f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf anrmto smlnum)
  (setf ilascl f2cl-lib:%true%))
```

```
 ((> anrm bignum) (setf anrmto bignum)
  (setf ilascl f2cl-lib:%true%)))
(if ilascl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (clascl "G" 0 0 anrm anrmto n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf anrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "M" n n b ldb$ rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ilbscl f2cl-lib:%false%)
(cond
 ((and (> bnrm zero) (< bnrm smlnum)) (setf bnrmto smlnum)
  (setf ilbscl f2cl-lib:%true%))
 ((> bnrm bignum) (setf bnrmto bignum)
  (setf ilbscl f2cl-lib:%true%)))
(if ilbscl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (clascl "G" 0 0 bnrm bnrmto n n b ldb$ ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bnrm var-3))
      (when var-4 (setf bnrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf ierr var-9))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (cggbal balanc n a lda b ldb$ ilo ihi lscale rscale rwork ierr)
  (declare (ignore var-1 var-2 var-4 var-8 var-9 var-10))
  (setf balanc var-0)
  (setf lda var-3)
  (setf ldb$ var-5)
  (setf ilo var-6)
  (setf ihi var-7)
  (setf ierr var-11))
```

```
(setf abnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "1" n n a lda
             (f2cl-lib:array-slice rwork-%data% single-float (1)
                                   ((1 *)) rwork-%offset%))
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(cond
 (ilascl
  (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
          abnrm)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 anrmto anrm 1 1
       (f2cl-lib:array-slice rwork-%data% single-float (1) ((1 *))
                             rwork-%offset%)
       1 ierr)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
    (when var-3 (setf anrmto var-3))
    (when var-4 (setf anrm var-4))
    (when var-9 (setf ierr var-9)))
  (setf abnrm
          (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%))))
(setf bbnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "1" n n b ldb$
             (f2cl-lib:array-slice rwork-%data% single-float (1)
                                   ((1 *)) rwork-%offset%))
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(cond
 (ilbscl
  (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
          bbnrm)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 bnrmto bnrm 1 1
       (f2cl-lib:array-slice rwork-%data% single-float (1) ((1 *))
                             rwork-%offset%)
       1 ierr)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
    (when var-3 (setf bnrmto var-3))
```

```
    (when var-4 (setf bnrm var-4))
    (when var-9 (setf ierr var-9)))
  (setf bbnrm
          (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%))))
(setf irows (f2cl-lib:int-sub (f2cl-lib:int-add ihi 1) ilo))
(cond
 ((or ilv (not wantsn))
  (setf icols (f2cl-lib:int-sub (f2cl-lib:int-add n 1) ilo)))
 (t (setf icols irows)))
(setf itau 1)
(setf iwrk (f2cl-lib:int-add itau irows))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgeqrf irows icols
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf irows var-0)
  (setf icols var-1)
  (setf ldb$ var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (cunmqr "L" "C" irows icols irows
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (ilo ilo)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf irows var-2))
  (when var-3 (setf icols var-3))
  (when var-4 (setf irows var-4))
  (when var-6 (setf ldb$ var-6))
  (when var-9 (setf lda var-9))
  (when var-12 (setf ierr var-12)))
(cond
```

```
(ilvl
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (claset "Full" n n czero cone vl ldvl)
   (declare (ignore var-0 var-5))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-3 (setf czero var-3))
   (when var-4 (setf cone var-4))
   (when var-6 (setf ldvl var-6)))
 (cond
  ((> irows 1)
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (clacpy "L" (f2cl-lib:int-sub irows 1)
        (f2cl-lib:int-sub irows 1)
        (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                              ((+ ilo 1) ilo) ((1 ldb$) (1 *))
                              b-%offset%)
        ldb$
        (f2cl-lib:array-slice vl-%data% f2cl-lib:complex8
                              ((+ ilo 1) ilo) ((1 ldvl) (1 *))
                              vl-%offset%)
        ldvl)
     (declare (ignore var-0 var-1 var-2 var-3 var-5))
     (when var-4 (setf ldb$ var-4))
     (when var-6 (setf ldvl var-6)))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (cungqr irows irows irows
      (f2cl-lib:array-slice vl-%data% f2cl-lib:complex8 (ilo ilo)
                            ((1 ldvl) (1 *)) vl-%offset%)
      ldvl
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
   (declare (ignore var-3 var-5 var-6 var-7))
   (when var-0 (setf irows var-0))
   (when var-1 (setf irows var-1))
   (when var-2 (setf irows var-2))
   (when var-4 (setf ldvl var-4))
   (when var-8 (setf ierr var-8)))))
(if ilvr
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (claset "Full" n n czero cone vr ldvr)
     (declare (ignore var-0 var-5))
     (when var-1 (setf n var-1))
     (when var-2 (setf n var-2))
     (when var-3 (setf czero var-3))
     (when var-4 (setf cone var-4))
```

```
      (when var-6 (setf ldvr var-6))))
(cond
 ((or ilv (not wantsn))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (cgghrd jobvl jobvr n ilo ihi a lda b ldb$ vl ldvl vr ldvr
       ierr)
    (declare (ignore var-5 var-7 var-9 var-11))
    (when var-0 (setf jobvl var-0))
    (when var-1 (setf jobvr var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-13 (setf ierr var-13))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (cgghrd "N" "N" irows 1 irows
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (ilo ilo)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (ilo ilo)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$ vl ldvl vr ldvr ierr)
    (declare (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11))
    (when var-2 (setf irows var-2))
    (when var-4 (setf irows var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-13 (setf ierr var-13)))))
(setf iwrk itau)
(cond
 ((or ilv (not wantsn))
  (f2cl-lib:f2cl-set-string chtemp "S" (string 1)))
 (t (f2cl-lib:f2cl-set-string chtemp "E" (string 1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18
     var-19)
    (chgeqz chtemp jobvl jobvr n ilo ihi a lda b ldb$ alpha beta vl
     ldvl vr ldvr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
```

```
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) rwork ierr)
    (declare
     (ignore var-6 var-8 var-10 var-11 var-12 var-14 var-16 var-17
      var-18))
    (when var-0 (setf chtemp var-0))
    (when var-1 (setf jobvl var-1))
    (when var-2 (setf jobvr var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf ilo var-4))
    (when var-5 (setf ihi var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldb$ var-9))
    (when var-13 (setf ldvl var-13))
    (when var-15 (setf ldvr var-15))
    (when var-19 (setf ierr var-19)))
  (cond
   ((/= ierr 0)
    (cond ((and (> ierr 0) (<= ierr n)) (setf info ierr))
          ((and (> ierr n) (<= ierr (f2cl-lib:int-mul 2 n)))
           (setf info (f2cl-lib:int-sub ierr n)))
          (t (setf info (f2cl-lib:int-add n 1))))
    (go label90)))
  (cond
   ((or ilv (not wantsn))
    (cond
     (ilv
      (cond
       (ilvl
        (cond (ilvr (f2cl-lib:f2cl-set-string chtemp "B" (string 1)))
              (t (f2cl-lib:f2cl-set-string chtemp "L" (string 1)))))
       (t (f2cl-lib:f2cl-set-string chtemp "R" (string 1))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13 var-14 var-15 var-16)
          (ctgevc chtemp "B" ldumma n a lda b ldb$ vl ldvl vr ldvr n in
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwrk)
                                 ((1 *)) work-%offset%)
           rwork ierr)
        (declare
         (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-14 var-15))
        (when var-0 (setf chtemp var-0))
        (when var-3 (setf n var-3))
        (when var-5 (setf lda var-5))
        (when var-7 (setf ldb$ var-7))
        (when var-9 (setf ldvl var-9))
        (when var-11 (setf ldvr var-11))
        (when var-12 (setf n var-12))
        (when var-13 (setf in var-13))
        (when var-16 (setf ierr var-16)))
```

```
  (cond
   ((/= ierr 0) (setf info (f2cl-lib:int-add n 2)) (go label90)))))
(cond
 ((not wantsn)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf (f2cl-lib:fref bwork-%data% (j) ((1 *))
                               bwork-%offset%)
                  f2cl-lib:%false%)
         label10))
      (setf (f2cl-lib:fref bwork-%data% (i) ((1 *)) bwork-%offset%)
              f2cl-lib:%true%)
      (setf iwrk (f2cl-lib:int-add n 1))
      (setf iwrk1 (f2cl-lib:int-add iwrk n))
      (cond
       ((or wantse wantsb)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12 var-13 var-14 var-15
             var-16)
            (ctgevc "B" "S" bwork n a lda b ldb$
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                   (1) ((1 *)) work-%offset%)
             n
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                   (iwrk) ((1 *)) work-%offset%)
             n 1 m
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                   (iwrk1) ((1 *)) work-%offset%)
             rwork ierr)
          (declare
           (ignore var-0 var-1 var-2 var-4 var-6 var-8 var-10
            var-12 var-14 var-15))
          (when var-3 (setf n var-3))
          (when var-5 (setf lda var-5))
          (when var-7 (setf ldb$ var-7))
          (when var-9 (setf n var-9))
          (when var-11 (setf n var-11))
          (when var-13 (setf m var-13))
          (when var-16 (setf ierr var-16)))
        (cond
         ((/= ierr 0) (setf info (f2cl-lib:int-add n 2))
          (go label90)))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
```

```
                var-17 var-18 var-19)
               (ctgsna sense "S" bwork n a lda b ldb$
                (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (1)
                                      ((1 *)) work-%offset%)
                n
                (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                      (iwrk) ((1 *)) work-%offset%)
                n
                (f2cl-lib:array-slice rconde-%data% single-float (i)
                                      ((1 *)) rconde-%offset%)
                (f2cl-lib:array-slice rcondv-%data% single-float (i)
                                      ((1 *)) rcondv-%offset%)
                1 m
                (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                      (iwrk1) ((1 *)) work-%offset%)
                (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk1) 1)
                iwork ierr)
            (declare
             (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-12 var-13
              var-14 var-16 var-17 var-18))
            (when var-0 (setf sense var-0))
            (when var-3 (setf n var-3))
            (when var-5 (setf lda var-5))
            (when var-7 (setf ldb$ var-7))
            (when var-9 (setf n var-9))
            (when var-11 (setf n var-11))
            (when var-15 (setf m var-15))
            (when var-19 (setf ierr var-19)))
        label20))))))
(cond
 (ilvl
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (cggbak balanc "L" n ilo ihi lscale rscale n vl ldvl ierr)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-8))
    (setf balanc var-0)
    (setf n var-7)
    (setf ldvl var-9)
    (setf ierr var-10))
  (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                ((> jc n) nil)
    (tagbody
      (setf temp zero)
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf temp
                  (max temp
                       (abs1
```

```
                              (f2cl-lib:fref vl-%data% (jr jc)
                                             ((1 ldvl) (1 *))
                                             vl-%offset%))))
           label30))
        (if (< temp smlnum)
            (go label50))
        (setf temp (/ one temp))
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr n) nil)
          (tagbody
            (setf (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                                 vl-%offset%)
                  (*
                   (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                                  vl-%offset%)
                   temp))
           label40))
       label50))))
(cond
 (ilvr
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (cggbak balanc "R" n ilo ihi lscale rscale n vr ldvr ierr)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-8))
    (setf balanc var-0)
    (setf n var-7)
    (setf ldvr var-9)
    (setf ierr var-10))
  (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                ((> jc n) nil)
    (tagbody
      (setf temp zero)
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf temp
                  (max temp
                       (abs1
                        (f2cl-lib:fref vr-%data% (jr jc)
                                       ((1 ldvr) (1 *))
                                       vr-%offset%))))
           label60))
      (if (< temp smlnum)
          (go label80))
      (setf temp (/ one temp))
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
```

```
                                        vr-%offset%)
                              (*
                               (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                              vr-%offset%)
                               temp))
                        label70))
                     label80))))
               label90
                (if ilascl
                    (multiple-value-bind
                        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                        (clascl "G" 0 0 anrmto anrm n 1 alpha n ierr)
                      (declare (ignore var-0 var-1 var-2 var-6 var-7))
                      (when var-3 (setf anrmto var-3))
                      (when var-4 (setf anrm var-4))
                      (when var-5 (setf n var-5))
                      (when var-8 (setf n var-8))
                      (when var-9 (setf ierr var-9))))
                (if ilbscl
                    (multiple-value-bind
                        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                        (clascl "G" 0 0 bnrmto bnrm n 1 beta n ierr)
                      (declare (ignore var-0 var-1 var-2 var-6 var-7))
                      (when var-3 (setf bnrmto var-3))
                      (when var-4 (setf bnrm var-4))
                      (when var-5 (setf n var-5))
                      (when var-8 (setf n var-8))
                      (when var-9 (setf ierr var-9))))
                (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                        (coerce maxwrk 'f2cl-lib:complex8))
                (go end_label)
               end_label
                (return
                 (values balanc jobvl jobvr sense n nil lda nil ldb$ nil nil nil ldvl
                         nil ldvr ilo ihi nil nil abnrm bbnrm nil nil nil nil nil nil
                         nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cggevx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
```

```
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (array fortran-to-lisp::complex8
   (*))
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (single-float) (single-float)
  (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (array fortran-to-lisp::logical
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::balanc
  fortran-to-lisp::jobvl
  fortran-to-lisp::jobvr
  fortran-to-lisp::sense
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$ nil nil nil
  fortran-to-lisp::ldvl nil
  fortran-to-lisp::ldvr
  fortran-to-lisp::ilo
  fortran-to-lisp::ihi nil nil
  fortran-to-lisp::abnrm
  fortran-to-lisp::bbnrm nil nil
  nil nil nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::cggbak
  fortran-to-lisp::cgeqrf
  fortran-to-lisp::cggbal))))
```

Chapter 27

LAPACK - General Matrices, Eigenvalue , Complex16

27.1 zgees

27.1.1 Axiom unit tests

— zgees.input —

```
)set break resume
)sys rm -f zgees.output
)spool zgees.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

27.1.2 Axiom help page

— zgees.help —

```
========================================================================
zgees examples
========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgees (JOBVS, SORT, SELECT, N, A, LDA, SDIM, W, VS, LDVS,
           WORK, LWORK, RWORK, BWORK, INFO)
            ZGEES computes the eigenvalues, the Schur form, and, optionally,
           the matrix of Schur vectors for GE matrices

Function/Subroutine Documentation
   subroutine zgees (character JOBVS, character SORT, external SELECT, integer
       N, complex*16, dimension( lda, * ) A, integer LDA, integer SDIM,
       complex*16, dimension( * ) W, complex*16, dimension( ldvs, * ) VS,
       integer LDVS, complex*16, dimension( * ) WORK, integer LWORK, double
       precision, dimension( * ) RWORK, logical, dimension( * ) BWORK, integer
       INFO)
        ZGEES computes the eigenvalues, the Schur form, and, optionally, the
       matrix of Schur vectors for GE matrices

Purpose:
   ZGEES computes for an N-by-N complex nonsymmetric matrix A, the
   eigenvalues, the Schur form T, and, optionally, the matrix of Schur
   vectors Z.  This gives the Schur factorization A = Z*T*(Z**H).

   Optionally, it also orders the eigenvalues on the diagonal of the
   Schur form so that selected eigenvalues are at the top left.
   The leading columns of Z then form an orthonormal basis for the
   invariant subspace corresponding to the selected eigenvalues.

   A complex matrix is in Schur form if it is upper triangular.

Parameters:

 JOBVS
   JOBVS is CHARACTER*1
   = 'N': Schur vectors are not computed;
   = 'V': Schur vectors are computed.

 SORT
   SORT is CHARACTER*1
   Specifies whether or not to order the eigenvalues on the
   diagonal of the Schur form.
   = 'N': Eigenvalues are not ordered:
   = 'S': Eigenvalues are ordered (see SELECT).

 SELECT
   SELECT is a LOGICAL FUNCTION of one COMPLEX*16 argument
```

```
  SELECT must be declared EXTERNAL in the calling subroutine.
  If SORT = 'S', SELECT is used to select eigenvalues to order
  to the top left of the Schur form.
  IF SORT = 'N', SELECT is not referenced.
  The eigenvalue W(j) is selected if SELECT(W(j)) is true.

N
  N is INTEGER
  The order of the matrix A. N >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the N-by-N matrix A.
  On exit, A has been overwritten by its Schur form T.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

SDIM
  SDIM is INTEGER
  If SORT = 'N', SDIM = 0.
  If SORT = 'S', SDIM = number of eigenvalues for which SELECT is true.

W
  W is COMPLEX*16 array, dimension (N)
  W contains the computed eigenvalues, in the same order that
  they appear on the diagonal of the output Schur form T.

VS
  VS is COMPLEX*16 array, dimension (LDVS,N)
  If JOBVS = 'V', VS contains the unitary matrix Z of Schur vectors.
  If JOBVS = 'N', VS is not referenced.

LDVS
  LDVS is INTEGER
  The leading dimension of the array VS.  LDVS >= 1; if
  JOBVS = 'V', LDVS >= N.

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,2*N).
  For good performance, LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
```

```
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (N)

BWORK
  BWORK is LOGICAL array, dimension (N)
  Not referenced if SORT = 'N'.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value.
  > 0: if INFO = i, and i is
   <= N:  the QR algorithm failed to compute all the
          eigenvalues; elements 1:ILO-1 and i+1:N of W
          contain those eigenvalues which have converged;
          if JOBVS = 'V', VS contains the matrix which
          reduces A to its partially converged Schur form.
   = N+1: the eigenvalues could not be reordered because
          some eigenvalues were too close to separate (the
          problem is very ill-conditioned);
   = N+2: after reordering, roundoff changed values of
          some complex eigenvalues so that leading
          eigenvalues in the Schur form no longer satisfy
          SELECT = .TRUE..  This could also be caused by
          underflow due to scaling.
```

27.1.3 fortran code

— zgees.f —

```
*  =====================================================================
      SUBROUTINE ZGEES( JOBVS, SORT, SELECT, N, A, LDA, SDIM, W, VS,
     $                  LDVS, WORK, LWORK, RWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVS, SORT
      INTEGER            INFO, LDA, LDVS, LWORK, N, SDIM
```

```
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      DOUBLE PRECISION   RWORK( * )
      COMPLEX*16         A( LDA, * ), VS( LDVS, * ), W( * ), WORK( * )
*     ..
*     .. Function Arguments ..
      LOGICAL            SELECT
      EXTERNAL           SELECT
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, SCALEA, WANTST, WANTVS
      INTEGER            HSWORK, I, IBAL, ICOND, IERR, IEVAL, IHI, ILO,
     $                   ITAU, IWRK, MAXWRK, MINWRK
      DOUBLE PRECISION   ANRM, BIGNUM, CSCALE, EPS, S, SEP, SMLNUM
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, XERBLA, ZCOPY, ZGEBAK, ZGEBAL, ZGEHRD,
     $                   ZHSEQR, ZLACPY, ZLASCL, ZTRSEN, ZUNGHR
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, ZLANGE
      EXTERNAL           LSAME, ILAENV, DLAMCH, ZLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      WANTVS = LSAME( JOBVS, 'V' )
      WANTST = LSAME( SORT, 'S' )
      IF( ( .NOT.WANTVS ) .AND. ( .NOT.LSAME( JOBVS, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
```

```
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDVS.LT.1 .OR. ( WANTVS .AND. LDVS.LT.N ) ) THEN
         INFO = -10
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       CWorkspace refers to complex workspace, and RWorkspace to real
*       workspace. NB refers to the optimal block size for the
*       immediately following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by ZHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            MAXWRK = 1
         ELSE
            MAXWRK = N + N*ILAENV( 1, 'ZGEHRD', ' ', N, 1, N, 0 )
            MINWRK = 2*N
*
            CALL ZHSEQR( 'S', JOBVS, N, 1, N, A, LDA, W, VS, LDVS,
     $              WORK, -1, IEVAL )
            HSWORK = WORK( 1 )
*
            IF( .NOT.WANTVS ) THEN
               MAXWRK = MAX( MAXWRK, HSWORK )
            ELSE
               MAXWRK = MAX( MAXWRK, N + ( N - 1 )*ILAENV( 1, 'ZUNGHR',
     $                        ' ', N, 1, N, -1 ) )
               MAXWRK = MAX( MAXWRK, HSWORK )
            END IF
         END IF
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -12
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEES ', -INFO )
         RETURN
```

```
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = ZLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
     $   CALL ZLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*     Permute the matrix to make it more nearly triangular
*     (CWorkspace: none)
*     (RWorkspace: need N)
*
      IBAL = 1
      CALL ZGEBAL( 'P', N, A, LDA, ILO, IHI, RWORK( IBAL ), IERR )
*
*     Reduce to upper Hessenberg form
*     (CWorkspace: need 2*N, prefer N+N*NB)
*     (RWorkspace: none)
*
      ITAU = 1
      IWRK = N + ITAU
      CALL ZGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
```

```
      IF( WANTVS ) THEN
*
*        Copy Householder vectors to VS
*
         CALL ZLACPY( 'L', N, N, A, LDA, VS, LDVS )
*
*        Generate unitary matrix in VS
*        (CWorkspace: need 2*N-1, prefer N+(N-1)*NB)
*        (RWorkspace: none)
*
         CALL ZUNGHR( N, ILO, IHI, VS, LDVS, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
      END IF
*
      SDIM = 0
*
*     Perform QR iteration, accumulating Schur vectors in VS if desired
*     (CWorkspace: need 1, prefer HSWORK (see comments) )
*     (RWorkspace: none)
*
      IWRK = ITAU
      CALL ZHSEQR( 'S', JOBVS, N, ILO, IHI, A, LDA, W, VS, LDVS,
     $             WORK( IWRK ), LWORK-IWRK+1, IEVAL )
      IF( IEVAL.GT.0 )
     $   INFO = IEVAL
*
*     Sort eigenvalues if desired
*
      IF( WANTST .AND. INFO.EQ.0 ) THEN
         IF( SCALEA )
     $      CALL ZLASCL( 'G', 0, 0, CSCALE, ANRM, N, 1, W, N, IERR )
         DO 10 I = 1, N
            BWORK( I ) = SELECT( W( I ) )
   10    CONTINUE
*
*        Reorder eigenvalues and transform Schur vectors
*        (CWorkspace: none)
*        (RWorkspace: none)
*
         CALL ZTRSEN( 'N', JOBVS, BWORK, N, A, LDA, VS, LDVS, W, SDIM,
     $                S, SEP, WORK( IWRK ), LWORK-IWRK+1, ICOND )
      END IF
*
      IF( WANTVS ) THEN
*
*        Undo balancing
*        (CWorkspace: none)
*        (RWorkspace: need N)
*
         CALL ZGEBAK( 'P', 'R', N, ILO, IHI, RWORK( IBAL ), N, VS, LDVS,
```

```
     $                IERR )
      END IF
*
      IF( SCALEA ) THEN
*
*        Undo scaling for the Schur form of A
*
         CALL ZLASCL( 'U', 0, 0, CSCALE, ANRM, N, N, A, LDA, IERR )
         CALL ZCOPY( N, A, LDA+1, W, 1 )
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of ZGEES
*
      END
```

———-

27.1.4 lisp code

— **zgees.lisp** —

———-

27.2 zgeesx

27.2.1 Axiom unit tests

— **zgeesx.input** —

```
)set break resume
)sys rm -f zgeesx.output
)spool zgeesx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

27.2.2 Axiom help page

— zgeesx.help —

```
====================================================================
zgeesx examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgeesx (JOBVS, SORT, SELECT, SENSE, N, A, LDA, SDIM, W, VS,
           LDVS, RCONDE, RCONDV, WORK, LWORK, RWORK, BWORK, INFO)
            ZGEESX computes the eigenvalues, the Schur form, and, optionally,
           the matrix of Schur vectors for GE matrices

Function/Subroutine Documentation
   subroutine zgeesx (character JOBVS, character SORT, external SELECT,
       character SENSE, integer N, complex*16, dimension( lda, * ) A, integer
       LDA, integer SDIM, complex*16, dimension( * ) W, complex*16, dimension(
       ldvs, * ) VS, integer LDVS, double precision RCONDE, double precision
       RCONDV, complex*16, dimension( * ) WORK, integer LWORK, double
       precision, dimension( * ) RWORK, logical, dimension( * ) BWORK, integer
       INFO)
        ZGEESX computes the eigenvalues, the Schur form, and, optionally, the
       matrix of Schur vectors for GE matrices

Purpose:
   ZGEESX computes for an N-by-N complex nonsymmetric matrix A, the
   eigenvalues, the Schur form T, and, optionally, the matrix of Schur
   vectors Z.  This gives the Schur factorization A = Z*T*(Z**H).

   Optionally, it also orders the eigenvalues on the diagonal of the
   Schur form so that selected eigenvalues are at the top left;
   computes a reciprocal condition number for the average of the
   selected eigenvalues (RCONDE); and computes a reciprocal condition
   number for the right invariant subspace corresponding to the
   selected eigenvalues (RCONDV).  The leading columns of Z form an
   orthonormal basis for this invariant subspace.

   For further explanation of the reciprocal condition numbers RCONDE
   and RCONDV, see Section 4.10 of the LAPACK Users' Guide (where
   these quantities are called s and sep respectively).

   A complex matrix is in Schur form if it is upper triangular.
```

```
Parameters:

 JOBVS
   JOBVS is CHARACTER*1
   = 'N': Schur vectors are not computed;
   = 'V': Schur vectors are computed.

 SORT
   SORT is CHARACTER*1
   Specifies whether or not to order the eigenvalues on the
   diagonal of the Schur form.
   = 'N': Eigenvalues are not ordered;
   = 'S': Eigenvalues are ordered (see SELECT).

 SELECT
   SELECT is procedure) LOGICAL FUNCTION of one COMPLEX*16 argument
   SELECT must be declared EXTERNAL in the calling subroutine.
   If SORT = 'S', SELECT is used to select eigenvalues to order
   to the top left of the Schur form.
   If SORT = 'N', SELECT is not referenced.
   An eigenvalue W(j) is selected if SELECT(W(j)) is true.

 SENSE
   SENSE is CHARACTER*1
   Determines which reciprocal condition numbers are computed.
   = 'N': None are computed;
   = 'E': Computed for average of selected eigenvalues only;
   = 'V': Computed for selected right invariant subspace only;
   = 'B': Computed for both.
   If SENSE = 'E', 'V' or 'B', SORT must equal 'S'.

 N
   N is INTEGER
   The order of the matrix A. N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA, N)
   On entry, the N-by-N matrix A.
   On exit, A is overwritten by its Schur form T.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 SDIM
   SDIM is INTEGER
   If SORT = 'N', SDIM = 0.
   If SORT = 'S', SDIM = number of eigenvalues for which SELECT is true.

 W
```

```
  W is COMPLEX*16 array, dimension (N)
  W contains the computed eigenvalues, in the same order
  that they appear on the diagonal of the output Schur form T.

VS
  VS is COMPLEX*16 array, dimension (LDVS,N)
  If JOBVS = 'V', VS contains the unitary matrix Z of Schur vectors.
  If JOBVS = 'N', VS is not referenced.

LDVS
  LDVS is INTEGER
  The leading dimension of the array VS.  LDVS >= 1, and if
  JOBVS = 'V', LDVS >= N.

RCONDE
  RCONDE is DOUBLE PRECISION
  If SENSE = 'E' or 'B', RCONDE contains the reciprocal
  condition number for the average of the selected eigenvalues.
  Not referenced if SENSE = 'N' or 'V'.

RCONDV
  RCONDV is DOUBLE PRECISION
  If SENSE = 'V' or 'B', RCONDV contains the reciprocal
  condition number for the selected right invariant subspace.
  Not referenced if SENSE = 'N' or 'E'.

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,2*N).
  Also, if SENSE = 'E' or 'V' or 'B', LWORK >= 2*SDIM*(N-SDIM),
  where SDIM is the number of selected eigenvalues computed by
  this routine.  Note that 2*SDIM*(N-SDIM) <= N*N/2. Note also
  that an error is only returned if LWORK < max(1,2*N), but if
  SENSE = 'E' or 'V' or 'B' this may not be large enough.
  For good performance, LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates upper bound on the optimal size of the
  array WORK, returns this value as the first entry of the WORK
  array, and no error message related to LWORK is issued by XERBLA.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (N)

BWORK
  BWORK is LOGICAL array, dimension (N)
```

```
  Not referenced if SORT = 'N'.

 INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value.
  > 0: if INFO = i, and i is
    <= N: the QR algorithm failed to compute all the
          eigenvalues; elements 1:ILO-1 and i+1:N of W
          contain those eigenvalues which have converged; if
          JOBVS = 'V', VS contains the transformation which
          reduces A to its partially converged Schur form.
   = N+1: the eigenvalues could not be reordered because some
          eigenvalues were too close to separate (the problem
          is very ill-conditioned);
   = N+2: after reordering, roundoff changed values of some
          complex eigenvalues so that leading eigenvalues in
          the Schur form no longer satisfy SELECT=.TRUE.  This
          could also be caused by underflow due to scaling.
```

27.2.3 fortran code

— zgeesx.f —

```
*  =====================================================================
      SUBROUTINE ZGEESX( JOBVS, SORT, SELECT, SENSE, N, A, LDA, SDIM, W,
     $                   VS, LDVS, RCONDE, RCONDV, WORK, LWORK, RWORK,
     $                   BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVS, SENSE, SORT
      INTEGER            INFO, LDA, LDVS, LWORK, N, SDIM
      DOUBLE PRECISION   RCONDE, RCONDV
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      DOUBLE PRECISION   RWORK( * )
      COMPLEX*16         A( LDA, * ), VS( LDVS, * ), W( * ), WORK( * )
*     ..
*     .. Function Arguments ..
```

```
      LOGICAL            SELECT
      EXTERNAL           SELECT
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, SCALEA, WANTSB, WANTSE, WANTSN, WANTST,
     $                   WANTSV, WANTVS
      INTEGER            HSWORK, I, IBAL, ICOND, IERR, IEVAL, IHI, ILO,
     $                   ITAU, IWRK, LWRK, MAXWRK, MINWRK
      DOUBLE PRECISION   ANRM, BIGNUM, CSCALE, EPS, SMLNUM
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, DLASCL, XERBLA, ZCOPY, ZGEBAK, ZGEBAL,
     $                   ZGEHRD, ZHSEQR, ZLACPY, ZLASCL, ZTRSEN, ZUNGHR
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, ZLANGE
      EXTERNAL           LSAME, ILAENV, DLAMCH, ZLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      WANTVS = LSAME( JOBVS, 'V' )
      WANTST = LSAME( SORT, 'S' )
      WANTSN = LSAME( SENSE, 'N' )
      WANTSE = LSAME( SENSE, 'E' )
      WANTSV = LSAME( SENSE, 'V' )
      WANTSB = LSAME( SENSE, 'B' )
      LQUERY = ( LWORK.EQ.-1 )
*
      IF( ( .NOT.WANTVS ) .AND. ( .NOT.LSAME( JOBVS, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
         INFO = -2
```

```
      ELSE IF( .NOT.( WANTSN .OR. WANTSE .OR. WANTSV .OR. WANTSB ) .OR.
     $         ( .NOT.WANTST .AND. .NOT.WANTSN ) ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDVS.LT.1 .OR. ( WANTVS .AND. LDVS.LT.N ) ) THEN
         INFO = -11
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of real workspace needed at that point in the
*       code, as well as the preferred amount for good performance.
*       CWorkspace refers to complex workspace, and RWorkspace to real
*       workspace. NB refers to the optimal block size for the
*       immediately following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by ZHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.
*       If SENSE = 'E', 'V' or 'B', then the amount of workspace needed
*       depends on SDIM, which is computed by the routine ZTRSEN later
*       in the code.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            LWRK = 1
         ELSE
            MAXWRK = N + N*ILAENV( 1, 'ZGEHRD', ' ', N, 1, N, 0 )
            MINWRK = 2*N
*
            CALL ZHSEQR( 'S', JOBVS, N, 1, N, A, LDA, W, VS, LDVS,
     $              WORK, -1, IEVAL )
            HSWORK = WORK( 1 )
*
            IF( .NOT.WANTVS ) THEN
               MAXWRK = MAX( MAXWRK, HSWORK )
            ELSE
               MAXWRK = MAX( MAXWRK, N + ( N - 1 )*ILAENV( 1, 'ZUNGHR',
     $                        ' ', N, 1, N, -1 ) )
               MAXWRK = MAX( MAXWRK, HSWORK )
            END IF
            LWRK = MAXWRK
            IF( .NOT.WANTSN )
     $         LWRK = MAX( LWRK, ( N*N )/2 )
         END IF
         WORK( 1 ) = LWRK
*
```

```
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -15
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEESX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = ZLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
     $   CALL ZLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*
*     Permute the matrix to make it more nearly triangular
*     (CWorkspace: none)
*     (RWorkspace: need N)
*
      IBAL = 1
      CALL ZGEBAL( 'P', N, A, LDA, ILO, IHI, RWORK( IBAL ), IERR )
*
```

```
*     Reduce to upper Hessenberg form
*     (CWorkspace: need 2*N, prefer N+N*NB)
*     (RWorkspace: none)
*
      ITAU = 1
      IWRK = N + ITAU
      CALL ZGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
      IF( WANTVS ) THEN
*
*        Copy Householder vectors to VS
*
         CALL ZLACPY( 'L', N, N, A, LDA, VS, LDVS )
*
*        Generate unitary matrix in VS
*        (CWorkspace: need 2*N-1, prefer N+(N-1)*NB)
*        (RWorkspace: none)
*
         CALL ZUNGHR( N, ILO, IHI, VS, LDVS, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
      END IF
*
      SDIM = 0
*
*     Perform QR iteration, accumulating Schur vectors in VS if desired
*     (CWorkspace: need 1, prefer HSWORK (see comments) )
*     (RWorkspace: none)
*
      IWRK = ITAU
      CALL ZHSEQR( 'S', JOBVS, N, ILO, IHI, A, LDA, W, VS, LDVS,
     $             WORK( IWRK ), LWORK-IWRK+1, IEVAL )
      IF( IEVAL.GT.0 )
     $   INFO = IEVAL
*
*     Sort eigenvalues if desired
*
      IF( WANTST .AND. INFO.EQ.0 ) THEN
         IF( SCALEA )
     $      CALL ZLASCL( 'G', 0, 0, CSCALE, ANRM, N, 1, W, N, IERR )
         DO 10 I = 1, N
            BWORK( I ) = SELECT( W( I ) )
   10    CONTINUE
*
*        Reorder eigenvalues, transform Schur vectors, and compute
*        reciprocal condition numbers
*        (CWorkspace: if SENSE is not 'N', need 2*SDIM*(N-SDIM)
*                     otherwise, need none )
*        (RWorkspace: none)
*
```

```
         CALL ZTRSEN( SENSE, JOBVS, BWORK, N, A, LDA, VS, LDVS, W, SDIM,
     $                RCONDE, RCONDV, WORK( IWRK ), LWORK-IWRK+1,
     $                ICOND )
         IF( .NOT.WANTSN )
     $      MAXWRK = MAX( MAXWRK, 2*SDIM*( N-SDIM ) )
         IF( ICOND.EQ.-14 ) THEN
*
*           Not enough complex workspace
*
            INFO = -15
         END IF
      END IF
*
      IF( WANTVS ) THEN
*
*        Undo balancing
*        (CWorkspace: none)
*        (RWorkspace: need N)
*
         CALL ZGEBAK( 'P', 'R', N, ILO, IHI, RWORK( IBAL ), N, VS, LDVS,
     $                IERR )
      END IF
*
      IF( SCALEA ) THEN
*
*        Undo scaling for the Schur form of A
*
         CALL ZLASCL( 'U', 0, 0, CSCALE, ANRM, N, N, A, LDA, IERR )
         CALL ZCOPY( N, A, LDA+1, W, 1 )
         IF( ( WANTSV .OR. WANTSB ) .AND. INFO.EQ.0 ) THEN
            DUM( 1 ) = RCONDV
            CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, 1, 1, DUM, 1, IERR )
            RCONDV = DUM( 1 )
         END IF
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of ZGEESX
*
      END
```

———-

27.2.4 lisp code

— zgeesx.lisp —

———-

27.3 zgeev

27.3.1 Axiom unit tests

— **zgeev.input** —

```
)set break resume
)sys rm -f zgeev.output
)spool zgeev.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

27.3.2 Axiom help page

— **zgeev.help** —

```
====================================================================
zgeev examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgeev (JOBVL, JOBVR, N, A, LDA, W, VL, LDVL, VR, LDVR, WORK,
           LWORK, RWORK, INFO)
            ZGEEV computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for GE matrices

Function/Subroutine Documentation
   subroutine zgeev (character JOBVL, character JOBVR, integer N, complex*16,
       dimension( lda, * ) A, integer LDA, complex*16, dimension( * ) W,
```

```
      complex*16, dimension( ldvl, * ) VL, integer LDVL, complex*16,
      dimension( ldvr, * ) VR, integer LDVR, complex*16, dimension( * ) WORK,
      integer LWORK, double precision, dimension( * ) RWORK, integer INFO)
       ZGEEV computes the eigenvalues and, optionally, the left and/or right
      eigenvectors for GE matrices

Purpose:
   ZGEEV computes for an N-by-N complex nonsymmetric matrix A, the
   eigenvalues and, optionally, the left and/or right eigenvectors.

   The right eigenvector v(j) of A satisfies
        A * v(j) = lambda(j) * v(j)
   where lambda(j) is its eigenvalue.
   The left eigenvector u(j) of A satisfies
        u(j)**H * A = lambda(j) * u(j)**H
   where u(j)**H denotes the conjugate transpose of u(j).

   The computed eigenvectors are normalized to have Euclidean norm
   equal to 1 and largest component real.

Parameters:

 JOBVL
   JOBVL is CHARACTER*1
   = 'N': left eigenvectors of A are not computed;
   = 'V': left eigenvectors of are computed.

 JOBVR
   JOBVR is CHARACTER*1
   = 'N': right eigenvectors of A are not computed;
   = 'V': right eigenvectors of A are computed.

 N
   N is INTEGER
   The order of the matrix A. N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the N-by-N matrix A.
   On exit, A has been overwritten.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 W
   W is COMPLEX*16 array, dimension (N)
   W contains the computed eigenvalues.

 VL
```

```
  VL is COMPLEX*16 array, dimension (LDVL,N)
  If JOBVL = 'V', the left eigenvectors u(j) are stored one
  after another in the columns of VL, in the same order
  as their eigenvalues.
  If JOBVL = 'N', VL is not referenced.
  u(j) = VL(:,j), the j-th column of VL.

LDVL
  LDVL is INTEGER
  The leading dimension of the array VL.  LDVL >= 1; if
  JOBVL = 'V', LDVL >= N.

VR
  VR is COMPLEX*16 array, dimension (LDVR,N)
  If JOBVR = 'V', the right eigenvectors v(j) are stored one
  after another in the columns of VR, in the same order
  as their eigenvalues.
  If JOBVR = 'N', VR is not referenced.
  v(j) = VR(:,j), the j-th column of VR.

LDVR
  LDVR is INTEGER
  The leading dimension of the array VR.  LDVR >= 1; if
  JOBVR = 'V', LDVR >= N.

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,2*N).
  For good performance, LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if INFO = i, the QR algorithm failed to compute all the
        eigenvalues, and no eigenvectors have been computed;
        elements and i+1:N of W contain eigenvalues which have converged.
```

———-

27.3.3 fortran code

— **zgeev.f** —

```
*  ======================================================================
      SUBROUTINE ZGEEV( JOBVL, JOBVR, N, A, LDA, W, VL, LDVL, VR, LDVR,
     $                  WORK, LWORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVL, JOBVR
      INTEGER            INFO, LDA, LDVL, LDVR, LWORK, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   RWORK( * )
      COMPLEX*16         A( LDA, * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   W( * ), WORK( * )
*     ..
*
*  ======================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, SCALEA, WANTVL, WANTVR
      CHARACTER          SIDE
      INTEGER            HSWORK, I, IBAL, IERR, IHI, ILO, IRWORK, ITAU,
     $                   IWRK, K, MAXWRK, MINWRK, NOUT
      DOUBLE PRECISION   ANRM, BIGNUM, CSCALE, EPS, SCL, SMLNUM
      COMPLEX*16         TMP
*     ..
*     .. Local Arrays ..
      LOGICAL            SELECT( 1 )
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, XERBLA, ZDSCAL, ZGEBAK, ZGEBAL, ZGEHRD,
     $                   ZHSEQR, ZLACPY, ZLASCL, ZSCAL, ZTREVC, ZUNGHR
*     ..
*     .. External Functions ..
```

```
      LOGICAL            LSAME
      INTEGER            IDAMAX, ILAENV
      DOUBLE PRECISION   DLAMCH, DZNRM2, ZLANGE
      EXTERNAL           LSAME, IDAMAX, ILAENV, DLAMCH, DZNRM2, ZLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, DCMPLX, DCONJG, DIMAG, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      WANTVL = LSAME( JOBVL, 'V' )
      WANTVR = LSAME( JOBVR, 'V' )
      IF( ( .NOT.WANTVL ) .AND. ( .NOT.LSAME( JOBVL, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTVR ) .AND. ( .NOT.LSAME( JOBVR, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDVL.LT.1 .OR. ( WANTVL .AND. LDVL.LT.N ) ) THEN
         INFO = -8
      ELSE IF( LDVR.LT.1 .OR. ( WANTVR .AND. LDVR.LT.N ) ) THEN
         INFO = -10
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       CWorkspace refers to complex workspace, and RWorkspace to real
*       workspace. NB refers to the optimal block size for the
*       immediately following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by ZHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            MAXWRK = 1
         ELSE
            MAXWRK = N + N*ILAENV( 1, 'ZGEHRD', ' ', N, 1, N, 0 )
            MINWRK = 2*N
            IF( WANTVL ) THEN
               MAXWRK = MAX( MAXWRK, N + ( N - 1 )*ILAENV( 1, 'ZUNGHR',
```

```
     $                       ' ', N, 1, N, -1 ) )
              CALL ZHSEQR( 'S', 'V', N, 1, N, A, LDA, W, VL, LDVL,
     $                WORK, -1, INFO )
           ELSE IF( WANTVR ) THEN
              MAXWRK = MAX( MAXWRK, N + ( N - 1 )*ILAENV( 1, 'ZUNGHR',
     $                       ' ', N, 1, N, -1 ) )
              CALL ZHSEQR( 'S', 'V', N, 1, N, A, LDA, W, VR, LDVR,
     $                WORK, -1, INFO )
           ELSE
              CALL ZHSEQR( 'E', 'N', N, 1, N, A, LDA, W, VR, LDVR,
     $                WORK, -1, INFO )
           END IF
           HSWORK = WORK( 1 )
           MAXWRK = MAX( MAXWRK, HSWORK, MINWRK )
        END IF
        WORK( 1 ) = MAXWRK
*
        IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
           INFO = -12
        END IF
     END IF
*
     IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'ZGEEV ', -INFO )
        RETURN
     ELSE IF( LQUERY ) THEN
        RETURN
     END IF
*
*    Quick return if possible
*
     IF( N.EQ.0 )
     $   RETURN
*
*    Get machine constants
*
     EPS = DLAMCH( 'P' )
     SMLNUM = DLAMCH( 'S' )
     BIGNUM = ONE / SMLNUM
     CALL DLABAD( SMLNUM, BIGNUM )
     SMLNUM = SQRT( SMLNUM ) / EPS
     BIGNUM = ONE / SMLNUM
*
*    Scale A if max element outside range [SMLNUM,BIGNUM]
*
     ANRM = ZLANGE( 'M', N, N, A, LDA, DUM )
     SCALEA = .FALSE.
     IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
        SCALEA = .TRUE.
        CSCALE = SMLNUM
```

```
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
     $   CALL ZLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*     Balance the matrix
*     (CWorkspace: none)
*     (RWorkspace: need N)
*
      IBAL = 1
      CALL ZGEBAL( 'B', N, A, LDA, ILO, IHI, RWORK( IBAL ), IERR )
*
*     Reduce to upper Hessenberg form
*     (CWorkspace: need 2*N, prefer N+N*NB)
*     (RWorkspace: none)
*
      ITAU = 1
      IWRK = ITAU + N
      CALL ZGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
      IF( WANTVL ) THEN
*
*        Want left eigenvectors
*        Copy Householder vectors to VL
*
         SIDE = 'L'
         CALL ZLACPY( 'L', N, N, A, LDA, VL, LDVL )
*
*        Generate unitary matrix in VL
*        (CWorkspace: need 2*N-1, prefer N+(N-1)*NB)
*        (RWorkspace: none)
*
         CALL ZUNGHR( N, ILO, IHI, VL, LDVL, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VL
*        (CWorkspace: need 1, prefer HSWORK (see comments) )
*        (RWorkspace: none)
*
         IWRK = ITAU
         CALL ZHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, W, VL, LDVL,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
         IF( WANTVR ) THEN
*
*           Want left and right eigenvectors
*           Copy Schur vectors to VR
```

```
*
            SIDE = 'B'
            CALL ZLACPY( 'F', N, N, VL, LDVL, VR, LDVR )
         END IF
*
      ELSE IF( WANTVR ) THEN
*
*        Want right eigenvectors
*        Copy Householder vectors to VR
*
         SIDE = 'R'
         CALL ZLACPY( 'L', N, N, A, LDA, VR, LDVR )
*
*        Generate unitary matrix in VR
*        (CWorkspace: need 2*N-1, prefer N+(N-1)*NB)
*        (RWorkspace: none)
*
         CALL ZUNGHR( N, ILO, IHI, VR, LDVR, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VR
*        (CWorkspace: need 1, prefer HSWORK (see comments) )
*        (RWorkspace: none)
*
         IWRK = ITAU
         CALL ZHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, W, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
      ELSE
*
*        Compute eigenvalues only
*        (CWorkspace: need 1, prefer HSWORK (see comments) )
*        (RWorkspace: none)
*
         IWRK = ITAU
         CALL ZHSEQR( 'E', 'N', N, ILO, IHI, A, LDA, W, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
      END IF
*
*     If INFO > 0 from ZHSEQR, then quit
*
      IF( INFO.GT.0 )
     $   GO TO 50
*
      IF( WANTVL .OR. WANTVR ) THEN
*
*        Compute left and/or right eigenvectors
*        (CWorkspace: need 2*N)
*        (RWorkspace: need 2*N)
*
```

```
         IRWORK = IBAL + N
         CALL ZTREVC( SIDE, 'B', SELECT, N, A, LDA, VL, LDVL, VR, LDVR,
     $                N, NOUT, WORK( IWRK ), RWORK( IRWORK ), IERR )
      END IF
*
      IF( WANTVL ) THEN
*
*        Undo balancing of left eigenvectors
*        (CWorkspace: none)
*        (RWorkspace: need N)
*
         CALL ZGEBAK( 'B', 'L', N, ILO, IHI, RWORK( IBAL ), N, VL, LDVL,
     $                IERR )
*
*        Normalize left eigenvectors and make largest component real
*
         DO 20 I = 1, N
            SCL = ONE / DZNRM2( N, VL( 1, I ), 1 )
            CALL ZDSCAL( N, SCL, VL( 1, I ), 1 )
            DO 10 K = 1, N
               RWORK( IRWORK+K-1 ) = DBLE( VL( K, I ) )**2 +
     $                               DIMAG( VL( K, I ) )**2
   10       CONTINUE
            K = IDAMAX( N, RWORK( IRWORK ), 1 )
            TMP = DCONJG( VL( K, I ) ) / SQRT( RWORK( IRWORK+K-1 ) )
            CALL ZSCAL( N, TMP, VL( 1, I ), 1 )
            VL( K, I ) = DCMPLX( DBLE( VL( K, I ) ), ZERO )
   20    CONTINUE
      END IF
*
      IF( WANTVR ) THEN
*
*        Undo balancing of right eigenvectors
*        (CWorkspace: none)
*        (RWorkspace: need N)
*
         CALL ZGEBAK( 'B', 'R', N, ILO, IHI, RWORK( IBAL ), N, VR, LDVR,
     $                IERR )
*
*        Normalize right eigenvectors and make largest component real
*
         DO 40 I = 1, N
            SCL = ONE / DZNRM2( N, VR( 1, I ), 1 )
            CALL ZDSCAL( N, SCL, VR( 1, I ), 1 )
            DO 30 K = 1, N
               RWORK( IRWORK+K-1 ) = DBLE( VR( K, I ) )**2 +
     $                               DIMAG( VR( K, I ) )**2
   30       CONTINUE
            K = IDAMAX( N, RWORK( IRWORK ), 1 )
            TMP = DCONJG( VR( K, I ) ) / SQRT( RWORK( IRWORK+K-1 ) )
```

```
            CALL ZSCAL( N, TMP, VR( 1, I ), 1 )
            VR( K, I ) = DCMPLX( DBLE( VR( K, I ) ), ZERO )
   40    CONTINUE
      END IF
*
*     Undo scaling if necessary
*
   50 CONTINUE
      IF( SCALEA ) THEN
         CALL ZLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, W( INFO+1 ),
     $                MAX( N-INFO, 1 ), IERR )
         IF( INFO.GT.0 ) THEN
            CALL ZLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, W, N, IERR )
         END IF
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of ZGEEV
*
      END
```

———-

27.3.4 lisp code

— **zgeev.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun zgeevx
         (balanc jobvl jobvr sense n a lda w vl ldvl vr ldvr ilo ihi scale
          abnrm rconde rcondv work lwork rwork info)
    (declare (type (double-float) abnrm)
             (type (array double-float (*)) rwork rcondv rconde scale)
             (type (array f2cl-lib:complex16 (*)) work vr vl w a)
             (type (f2cl-lib:integer4) info lwork ihi ilo ldvr ldvl lda n)
             (type (string *) sense jobvr jobvl balanc))
    (f2cl-lib:with-multi-array-data
        ((balanc character balanc-%data% balanc-%offset%)
         (jobvl character jobvl-%data% jobvl-%offset%)
         (jobvr character jobvr-%data% jobvr-%offset%)
         (sense character sense-%data% sense-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (w f2cl-lib:complex16 w-%data% w-%offset%)
```

```
    (vl f2cl-lib:complex16 vl-%data% vl-%offset%)
    (vr f2cl-lib:complex16 vr-%data% vr-%offset%)
    (work f2cl-lib:complex16 work-%data% work-%offset%)
    (scale double-float scale-%data% scale-%offset%)
    (rconde double-float rconde-%data% rconde-%offset%)
    (rcondv double-float rcondv-%data% rcondv-%offset%)
    (rwork double-float rwork-%data% rwork-%offset%))
 (prog ((dum (make-array 1 :element-type 'double-float))
        (select (make-array 1 :element-type 't)) (tmp #C(0.0d0 0.0d0))
        (anrm 0.0d0) (bignum 0.0d0) (cscale 0.0d0) (eps 0.0d0) (scl 0.0d0)
        (smlnum 0.0d0) (hswork 0) (i 0) (icond 0) (ierr 0) (itau 0)
        (iwrk 0) (k 0) (maxwrk 0) (minwrk 0) (nout 0)
        (job
         (make-array '(1) :element-type 'character :initial-element #\ ))
        (side
         (make-array '(1) :element-type 'character :initial-element #\ ))
        (lquery nil) (scalea nil) (wantvl nil) (wantvr nil) (wntsnb nil)
        (wntsne nil) (wntsnn nil) (wntsnv nil))
   (declare (type (array double-float (1)) dum)
            (type (array f2cl-lib:logical (1)) select)
            (type (f2cl-lib:complex16) tmp)
            (type (double-float) anrm bignum cscale eps scl smlnum)
            (type (f2cl-lib:integer4) hswork i icond ierr itau iwrk k
             maxwrk minwrk nout)
            (type (string 1) job side)
            (type f2cl-lib:logical lquery scalea wantvl wantvr wntsnb
             wntsne wntsnn wntsnv))
   (setf info 0)
   (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
   (setf wantvl
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame jobvl "V")
             (declare (ignore var-1))
             (when var-0 (setf jobvl var-0))
             ret-val))
   (setf wantvr
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame jobvr "V")
             (declare (ignore var-1))
             (when var-0 (setf jobvr var-0))
             ret-val))
   (setf wntsnn
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame sense "N")
             (declare (ignore var-1))
             (when var-0 (setf sense var-0))
             ret-val))
   (setf wntsne
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame sense "E")
```

```
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wntsnv
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "V")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wntsnb
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "B")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(cond
 ((not
   (or
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "N")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "S")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "P")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "B")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)))
  (setf info -1))
 ((and (not wantvl)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvl "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvl var-0))
          ret-val)))
  (setf info -2))
 ((and (not wantvr)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
```

```
              (lsame jobvr "N")
            (declare (ignore var-1))
            (when var-0 (setf jobvr var-0))
            ret-val)))
   (setf info -3))
  ((or (not (or wntsnn wntsne wntsnb wntsnv))
       (and (or wntsne wntsnb) (not (and wantvl wantvr))))
   (setf info -4))
  ((< n 0) (setf info -5))
  ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
   (setf info -7))
  ((or (< ldvl 1) (and wantvl (< ldvl n))) (setf info -10))
  ((or (< ldvr 1) (and wantvr (< ldvr n))) (setf info -12)))
 (cond
  ((= info 0)
   (cond ((= n 0) (setf minwrk 1) (setf maxwrk 1))
         (t
          (setf maxwrk
                  (f2cl-lib:int-add n
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val var-0
                                                           var-1 var-2
                                                           var-3 var-4
                                                           var-5 var-6)
                                                          (ilaenv 1
                                                           "ZGEHRD" " "
                                                           n 1 n 0)
                                                        (declare
                                                         (ignore var-0
                                                          var-1 var-2
                                                          var-4 var-6))
                                                        (setf n var-3)
                                                        (setf n var-5)
                                                        ret-val))))
          (cond
           (wantvl
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10 var-11 var-12)
                (zhseqr "S" "V" n 1 n a lda w vl ldvl work -1 info)
              (declare
               (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10
                var-11))
              (when var-2 (setf n var-2))
              (when var-4 (setf n var-4))
              (when var-6 (setf lda var-6))
              (when var-9 (setf ldvl var-9))
              (when var-12 (setf info var-12))))
           (wantvr
```

```
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zhseqr "S" "V" n 1 n a lda w vr ldvr work -1 info)
    (declare
     (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10
      var-11))
    (when var-2 (setf n var-2))
    (when var-4 (setf n var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldvr var-9))
    (when var-12 (setf info var-12))))
 (t
  (cond
   (wntsnn
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (zhseqr "E" "N" n 1 n a lda w vr ldvr work -1 info)
      (declare
       (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10
        var-11))
      (when var-2 (setf n var-2))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldvr var-9))
      (when var-12 (setf info var-12))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (zhseqr "S" "N" n 1 n a lda w vr ldvr work -1 info)
      (declare
       (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10
        var-11))
      (when var-2 (setf n var-2))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldvr var-9))
      (when var-12 (setf info var-12)))))))
(setf hswork
        (f2cl-lib:int
         (f2cl-lib:fref work-%data% (1) ((1 *))
                        work-%offset%)))
(cond
 ((and (not wantvl) (not wantvr))
  (setf minwrk (f2cl-lib:int-mul 2 n))
  (if (not (or wntsnn wntsne))
      (setf minwrk
              (max (the f2cl-lib:integer4 minwrk)
```

```
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add
                          (f2cl-lib:int-mul n n)
                          (f2cl-lib:int-mul 2 n))))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4 hswork)))
 (if (not (or wntsnn wntsne))
     (setf maxwrk
             (max (the f2cl-lib:integer4 maxwrk)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add
                        (f2cl-lib:int-mul n n)
                        (f2cl-lib:int-mul 2 n)))))))
(t (setf minwrk (f2cl-lib:int-mul 2 n))
 (if (not (or wntsnn wntsne))
     (setf minwrk
             (max (the f2cl-lib:integer4 minwrk)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add
                        (f2cl-lib:int-mul n n)
                        (f2cl-lib:int-mul 2 n))))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4 hswork)))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n
                                     (f2cl-lib:int-mul
                                      (f2cl-lib:int-sub n 1)
                                      (multiple-value-bind
                                          (ret-val var-0
                                           var-1 var-2 var-3
                                           var-4 var-5 var-6)
                                          (ilaenv 1 "ZUNGHR"
                                           " " n 1 n -1)
                                        (declare
                                         (ignore var-0 var-1
                                          var-2 var-4 var-6))
                                        (setf n var-3)
                                        (setf n var-5)
                                        ret-val))))))
 (if (not (or wntsnn wntsne))
     (setf maxwrk
             (max (the f2cl-lib:integer4 maxwrk)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add
                        (f2cl-lib:int-mul n n)
                        (f2cl-lib:int-mul 2 n))))))
```

```
            (setf maxwrk
                    (max (the f2cl-lib:integer4 maxwrk)
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-mul 2 n))))))
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4 minwrk)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce maxwrk 'f2cl-lib:complex16))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -20)))))
(cond
 ((/= info 0) (xerbla "ZGEEVX" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(setf eps (dlamch "P"))
(setf smlnum (dlamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf icond 0)
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "M" n n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf scalea f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf scalea f2cl-lib:%true%)
  (setf cscale smlnum))
 ((> anrm bignum) (setf scalea f2cl-lib:%true%) (setf cscale bignum)))
(if scalea
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (zlascl "G" 0 0 anrm cscale n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf cscale var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
```

```
      (when var-9 (setf ierr var-9))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgebal balanc n a lda ilo ihi scale ierr)
  (declare (ignore var-1 var-2 var-6))
  (setf balanc var-0)
  (setf lda var-3)
  (setf ilo var-4)
  (setf ihi var-5)
  (setf ierr var-7))
(setf abnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "1" n n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(cond
 (scalea (setf (f2cl-lib:fref dum (1) ((1 1))) abnrm)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 cscale anrm 1 1 dum 1 ierr)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
    (when var-3 (setf cscale var-3))
    (when var-4 (setf anrm var-4))
    (when var-9 (setf ierr var-9)))
  (setf abnrm (f2cl-lib:fref dum (1) ((1 1))))))
(setf itau 1)
(setf iwrk (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zgehrd n ilo ihi a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf ilo var-1))
  (when var-2 (setf ihi var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(cond
 (wantvl (f2cl-lib:f2cl-set-string side "L" (string 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "L" n n a lda vl ldvl)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
```

```
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvl var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zunghr n ilo ihi vl ldvl
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf n var-0))
    (when var-1 (setf ilo var-1))
    (when var-2 (setf ihi var-2))
    (when var-4 (setf ldvl var-4))
    (when var-8 (setf ierr var-8)))
  (setf iwrk itau)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (zhseqr "S" "V" n ilo ihi a lda w vl ldvl
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldvl var-9))
    (when var-12 (setf info var-12)))
  (cond
   (wantvr (f2cl-lib:f2cl-set-string side "B" (string 1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlacpy "F" n n vl ldvl vr ldvr)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf ldvl var-4))
      (when var-6 (setf ldvr var-6))))))
 (wantvr (f2cl-lib:f2cl-set-string side "R" (string 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "L" n n a lda vr ldvr)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvr var-6)))
  (multiple-value-bind
```

```
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (zunghr n ilo ihi vr ldvr
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
      (declare (ignore var-3 var-5 var-6 var-7))
      (when var-0 (setf n var-0))
      (when var-1 (setf ilo var-1))
      (when var-2 (setf ihi var-2))
      (when var-4 (setf ldvr var-4))
      (when var-8 (setf ierr var-8)))
    (setf iwrk itau)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (zhseqr "S" "V" n ilo ihi a lda w vr ldvr
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf n var-2))
      (when var-3 (setf ilo var-3))
      (when var-4 (setf ihi var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldvr var-9))
      (when var-12 (setf info var-12))))
   (t
    (cond (wntsnn (f2cl-lib:f2cl-set-string job "E" (string 1)))
          (t (f2cl-lib:f2cl-set-string job "S" (string 1))))
    (setf iwrk itau)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (zhseqr job "N" n ilo ihi a lda w vr ldvr
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
      (declare (ignore var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-0 (setf job var-0))
      (when var-2 (setf n var-2))
      (when var-3 (setf ilo var-3))
      (when var-4 (setf ihi var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldvr var-9))
      (when var-12 (setf info var-12)))))
  (if (> info 0)
      (go label50))
  (cond
```

```
 ((or wantvl wantvr)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (ztrevc side "B" select n a lda vl ldvl vr ldvr n nout
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                             ((1 *)) work-%offset%)
       rwork ierr)
    (declare (ignore var-1 var-2 var-4 var-6 var-8 var-12 var-13))
    (when var-0 (setf side var-0))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldvl var-7))
    (when var-9 (setf ldvr var-9))
    (when var-10 (setf n var-10))
    (when var-11 (setf nout var-11))
    (when var-14 (setf ierr var-14)))))
(cond
 ((not wntsnn)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17)
      (ztrsna sense "A" select n a lda vl ldvl vr ldvr rconde rcondv n
       nout
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                             ((1 *)) work-%offset%)
       n rwork icond)
    (declare
     (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-11 var-14
      var-16))
    (when var-0 (setf sense var-0))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldvl var-7))
    (when var-9 (setf ldvr var-9))
    (when var-12 (setf n var-12))
    (when var-13 (setf nout var-13))
    (when var-15 (setf n var-15))
    (when var-17 (setf icond var-17)))))
(cond
 (wantvl
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zgebak balanc "L" n ilo ihi scale n vl ldvl ierr)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-7))
    (setf balanc var-0)
    (setf n var-6)
    (setf ldvl var-8)
    (setf ierr var-9))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
            ((> i n) nil)
(tagbody
  (setf scl
          (/ one
             (multiple-value-bind (ret-val var-0 var-1 var-2)
                 (dznrm2 n
                  (f2cl-lib:array-slice vl-%data%
                                        f2cl-lib:complex16 (1 i)
                                        ((1 ldvl) (1 *))
                                        vl-%offset%)
                  1)
               (declare (ignore var-1 var-2))
               (when var-0 (setf n var-0))
               ret-val)))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (zdscal n scl
       (f2cl-lib:array-slice vl-%data% f2cl-lib:complex16 (1 i)
                             ((1 ldvl) (1 *)) vl-%offset%)
       1)
    (declare (ignore var-2 var-3))
    (when var-0 (setf n var-0))
    (when var-1 (setf scl var-1)))
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (setf (f2cl-lib:fref rwork-%data% (k) ((1 *)) rwork-%offset%)
              (+
               (expt
                (f2cl-lib:dble
                 (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                                vl-%offset%))
                2)
               (expt
                (f2cl-lib:dimag
                 (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                                vl-%offset%))
                2)))
     label10))
  (setf k
          (multiple-value-bind (ret-val var-0 var-1 var-2)
              (idamax n rwork 1)
            (declare (ignore var-1 var-2))
            (when var-0 (setf n var-0))
            ret-val))
  (setf tmp
          (coerce
           (/
            (f2cl-lib:dconjg
             (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                            vl-%offset%))
```

```
                    (f2cl-lib:fsqrt
                     (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                    rwork-%offset%)))
                  'f2cl-lib:complex16))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (zscal n tmp
             (f2cl-lib:array-slice vl-%data% f2cl-lib:complex16 (1 i)
                                   ((1 ldvl) (1 *)) vl-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-1 (setf tmp var-1)))
        (setf (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                             vl-%offset%)
                (f2cl-lib:dcmplx
                 (f2cl-lib:dble
                  (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                                 vl-%offset%))
                 zero))
      label20))))
(cond
 (wantvr
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zgebak balanc "R" n ilo ihi scale n vr ldvr ierr)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-7))
    (setf balanc var-0)
    (setf n var-6)
    (setf ldvr var-8)
    (setf ierr var-9))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf scl
              (/ one
                 (multiple-value-bind (ret-val var-0 var-1 var-2)
                     (dznrm2 n
                      (f2cl-lib:array-slice vr-%data%
                                            f2cl-lib:complex16 (1 i)
                                            ((1 ldvr) (1 *))
                                            vr-%offset%)
                      1)
                   (declare (ignore var-1 var-2))
                   (when var-0 (setf n var-0))
                   ret-val)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (zdscal n scl
           (f2cl-lib:array-slice vr-%data% f2cl-lib:complex16 (1 i)
                                 ((1 ldvr) (1 *)) vr-%offset%)
           1)
```

```
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf scl var-1)))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (k) ((1 *)) rwork-%offset%)
            (+
             (expt
              (f2cl-lib:dble
               (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                              vr-%offset%))
              2)
             (expt
              (f2cl-lib:dimag
               (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                              vr-%offset%))
              2)))
   label30))
(setf k
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (idamax n rwork 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(setf tmp
        (coerce
         (/
          (f2cl-lib:dconjg
           (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                          vr-%offset%))
          (f2cl-lib:fsqrt
           (f2cl-lib:fref rwork-%data% (k) ((1 *))
                          rwork-%offset%)))
         'f2cl-lib:complex16))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (zscal n tmp
     (f2cl-lib:array-slice vr-%data% f2cl-lib:complex16 (1 i)
                           ((1 ldvr) (1 *)) vr-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf tmp var-1)))
(setf (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                     vr-%offset%)
        (f2cl-lib:dcmplx
         (f2cl-lib:dble
          (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                         vr-%offset%))
         zero))
```

```
            label40))))
      label50
       (cond
        (scalea
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
             (zlascl "G" 0 0 cscale anrm (f2cl-lib:int-sub n info) 1
              (f2cl-lib:array-slice w-%data% f2cl-lib:complex16 ((+ info 1))
                                    ((1 *)) w-%offset%)
              (max (the f2cl-lib:integer4 (f2cl-lib:int-sub n info))
                   (the f2cl-lib:integer4 1))
              ierr)
           (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
           (when var-3 (setf cscale var-3))
           (when var-4 (setf anrm var-4))
           (when var-9 (setf ierr var-9)))
         (cond
          ((= info 0)
           (if (and (or wntsnv wntsnb) (= icond 0))
               (multiple-value-bind
                   (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                    var-9)
                   (dlascl "G" 0 0 cscale anrm n 1 rcondv n ierr)
                 (declare (ignore var-0 var-1 var-2 var-6 var-7))
                 (when var-3 (setf cscale var-3))
                 (when var-4 (setf anrm var-4))
                 (when var-5 (setf n var-5))
                 (when var-8 (setf n var-8))
                 (when var-9 (setf ierr var-9)))))
          (t
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
               (zlascl "G" 0 0 cscale anrm (f2cl-lib:int-sub ilo 1) 1 w n
                ierr)
             (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
             (when var-3 (setf cscale var-3))
             (when var-4 (setf anrm var-4))
             (when var-8 (setf n var-8))
             (when var-9 (setf ierr var-9)))))))
       (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
               (coerce maxwrk 'f2cl-lib:complex16))
       (go end_label)
      end_label
       (return
        (values balanc jobvl jobvr sense n nil lda nil nil ldvl nil ldvr ilo
                ihi nil abnrm nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
```

```
(setf (gethash 'fortran-to-lisp::zgeevx
               fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((string) (string) (string)
                                            (string)
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex16
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex16
                                             (*))
                                            (array fortran-to-lisp::complex16
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex16
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (array double-float (*))
                                            (double-float)
                                            (array double-float (*))
                                            (array double-float (*))
                                            (array fortran-to-lisp::complex16
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array double-float (*))
                                            (fortran-to-lisp::integer4))
                                          :return-values
                                          '(fortran-to-lisp::balanc
                                            fortran-to-lisp::jobvl
                                            fortran-to-lisp::jobvr
                                            fortran-to-lisp::sense
                                            fortran-to-lisp::n nil
                                            fortran-to-lisp::lda nil nil
                                            fortran-to-lisp::ldvl nil
                                            fortran-to-lisp::ldvr
                                            fortran-to-lisp::ilo
                                            fortran-to-lisp::ihi nil
                                            fortran-to-lisp::abnrm nil nil
                                            nil nil nil
                                            fortran-to-lisp::info)
                                          :calls
                                          '(fortran-to-lisp::zgebak
                                            fortran-to-lisp::zgebal
                                            fortran-to-lisp::xerbla
                                            fortran-to-lisp::ilaenv))))
```

———-

27.4 zgeevx

27.4.1 Axiom unit tests

— zgeevx.input —

```
)set break resume
)sys rm -f zgeevx.output
)spool zgeevx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

27.4.2 Axiom help page

— zgeevx.help —

```
======================================================================
zgeevx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgeevx (BALANC, JOBVL, JOBVR, SENSE, N, A, LDA, W, VL, LDVL,
           VR, LDVR, ILO, IHI, SCALE, ABNRM, RCONDE, RCONDV, WORK, LWORK,
           RWORK, INFO)
            ZGEEVX computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for GE matrices

Function/Subroutine Documentation
   subroutine zgeevx (character BALANC, character JOBVL, character JOBVR,
       character SENSE, integer N, complex*16, dimension( lda, * ) A, integer
       LDA, complex*16, dimension( * ) W, complex*16, dimension( ldvl, * ) VL,
```

```
      integer LDVL, complex*16, dimension( ldvr, * ) VR, integer LDVR,
      integer ILO, integer IHI, double precision, dimension( * ) SCALE,
      double precision ABNRM, double precision, dimension( * ) RCONDE, double
      precision, dimension( * ) RCONDV, complex*16, dimension( * ) WORK,
      integer LWORK, double precision, dimension( * ) RWORK, integer INFO)
       ZGEEVX computes the eigenvalues and, optionally, the left and/or right
      eigenvectors for GE matrices

Purpose:
   ZGEEVX computes for an N-by-N complex nonsymmetric matrix A, the
   eigenvalues and, optionally, the left and/or right eigenvectors.

   Optionally also, it computes a balancing transformation to improve
   the conditioning of the eigenvalues and eigenvectors (ILO, IHI,
   SCALE, and ABNRM), reciprocal condition numbers for the eigenvalues
   (RCONDE), and reciprocal condition numbers for the right
   eigenvectors (RCONDV).

   The right eigenvector v(j) of A satisfies
        A * v(j) = lambda(j) * v(j)
   where lambda(j) is its eigenvalue.
   The left eigenvector u(j) of A satisfies
        u(j)**H * A = lambda(j) * u(j)**H
   where u(j)**H denotes the conjugate transpose of u(j).

   The computed eigenvectors are normalized to have Euclidean norm
   equal to 1 and largest component real.

   Balancing a matrix means permuting the rows and columns to make it
   more nearly upper triangular, and applying a diagonal similarity
   transformation D * A * D**(-1), where D is a diagonal matrix, to
   make its rows and columns closer in norm and the condition numbers
   of its eigenvalues and eigenvectors smaller.  The computed
   reciprocal condition numbers correspond to the balanced matrix.
   Permuting rows and columns will not change the condition numbers
   (in exact arithmetic) but diagonal scaling will.  For further
   explanation of balancing, see section 4.10.2 of the LAPACK Users' Guide.

Parameters:

 BALANC
   BALANC is CHARACTER*1
   Indicates how the input matrix should be diagonally scaled
   and/or permuted to improve the conditioning of its
   eigenvalues.
   = 'N': Do not diagonally scale or permute;
   = 'P': Perform permutations to make the matrix more nearly
          upper triangular. Do not diagonally scale;
   = 'S': Diagonally scale the matrix, ie. replace A by
          D*A*D**(-1), where D is a diagonal matrix chosen
```

```
          to make the rows and columns of A more equal in
          norm. Do not permute;
  = 'B': Both diagonally scale and permute A.

  Computed reciprocal condition numbers will be for the matrix
  after balancing and/or permuting. Permuting does not change
  condition numbers (in exact arithmetic), but balancing does.

JOBVL
  JOBVL is CHARACTER*1
  = 'N': left eigenvectors of A are not computed;
  = 'V': left eigenvectors of A are computed.
  If SENSE = 'E' or 'B', JOBVL must = 'V'.

JOBVR
  JOBVR is CHARACTER*1
  = 'N': right eigenvectors of A are not computed;
  = 'V': right eigenvectors of A are computed.
  If SENSE = 'E' or 'B', JOBVR must = 'V'.

SENSE
  SENSE is CHARACTER*1
  Determines which reciprocal condition numbers are computed.
  = 'N': None are computed;
  = 'E': Computed for eigenvalues only;
  = 'V': Computed for right eigenvectors only;
  = 'B': Computed for eigenvalues and right eigenvectors.

  If SENSE = 'E' or 'B', both left and right eigenvectors
  must also be computed (JOBVL = 'V' and JOBVR = 'V').

N
  N is INTEGER
  The order of the matrix A. N >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the N-by-N matrix A.
  On exit, A has been overwritten.  If JOBVL = 'V' or
  JOBVR = 'V', A contains the Schur form of the balanced
  version of the matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

W
  W is COMPLEX*16 array, dimension (N)
  W contains the computed eigenvalues.
```

```
VL
  VL is COMPLEX*16 array, dimension (LDVL,N)
  If JOBVL = 'V', the left eigenvectors u(j) are stored one
  after another in the columns of VL, in the same order
  as their eigenvalues.
  If JOBVL = 'N', VL is not referenced.
  u(j) = VL(:,j), the j-th column of VL.

LDVL
  LDVL is INTEGER
  The leading dimension of the array VL.  LDVL >= 1; if
  JOBVL = 'V', LDVL >= N.

VR
  VR is COMPLEX*16 array, dimension (LDVR,N)
  If JOBVR = 'V', the right eigenvectors v(j) are stored one
  after another in the columns of VR, in the same order
  as their eigenvalues.
  If JOBVR = 'N', VR is not referenced.
  v(j) = VR(:,j), the j-th column of VR.

LDVR
  LDVR is INTEGER
  The leading dimension of the array VR.  LDVR >= 1; if
  JOBVR = 'V', LDVR >= N.

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER
  ILO and IHI are integer values determined when A was
  balanced.  The balanced A(i,j) = 0 if I > J and
  J = 1,...,ILO-1 or I = IHI+1,...,N.

SCALE
  SCALE is DOUBLE PRECISION array, dimension (N)
  Details of the permutations and scaling factors applied
  when balancing A.  If P(j) is the index of the row and column
  interchanged with row and column j, and D(j) is the scaling
  factor applied to row and column j, then
  SCALE(J) = P(J),    for J = 1,...,ILO-1
           = D(J),    for J = ILO,...,IHI
           = P(J)     for J = IHI+1,...,N.
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

ABNRM
  ABNRM is DOUBLE PRECISION
  The one-norm of the balanced matrix (the maximum
```

```
  of the sum of absolute values of elements of any column).

RCONDE
  RCONDE is DOUBLE PRECISION array, dimension (N)
  RCONDE(j) is the reciprocal condition number of the j-th eigenvalue.

RCONDV
  RCONDV is DOUBLE PRECISION array, dimension (N)
  RCONDV(j) is the reciprocal condition number of the j-th
  right eigenvector.

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  If SENSE = 'N' or 'E',
  LWORK >= max(1,2*N), and if SENSE = 'V' or 'B',
  LWORK >= N*N+2*N.
  For good performance, LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if INFO = i, the QR algorithm failed to compute all the
        eigenvalues, and no eigenvectors or condition numbers
        have been computed; elements 1:ILO-1 and i+1:N of W
        contain eigenvalues which have converged.
```

———-

27.4.3 fortran code

— zgeevx.f —

```
*  =====================================================================
      SUBROUTINE ZGEEVX( BALANC, JOBVL, JOBVR, SENSE, N, A, LDA, W, VL,
```

```
     $                   LDVL, VR, LDVR, ILO, IHI, SCALE, ABNRM, RCONDE,
     $                   RCONDV, WORK, LWORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          BALANC, JOBVL, JOBVR, SENSE
      INTEGER            IHI, ILO, INFO, LDA, LDVL, LDVR, LWORK, N
      DOUBLE PRECISION   ABNRM
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   RCONDE( * ), RCONDV( * ), RWORK( * ),
     $                   SCALE( * )
      COMPLEX*16         A( LDA, * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   W( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, SCALEA, WANTVL, WANTVR, WNTSNB, WNTSNE,
     $                   WNTSNN, WNTSNV
      CHARACTER          JOB, SIDE
      INTEGER            HSWORK, I, ICOND, IERR, ITAU, IWRK, K, MAXWRK,
     $                   MINWRK, NOUT
      DOUBLE PRECISION   ANRM, BIGNUM, CSCALE, EPS, SCL, SMLNUM
      COMPLEX*16         TMP
*     ..
*     .. Local Arrays ..
      LOGICAL            SELECT( 1 )
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, DLASCL, XERBLA, ZDSCAL, ZGEBAK, ZGEBAL,
     $                   ZGEHRD, ZHSEQR, ZLACPY, ZLASCL, ZSCAL, ZTREVC,
     $                   ZTRSNA, ZUNGHR
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX, ILAENV
      DOUBLE PRECISION   DLAMCH, DZNRM2, ZLANGE
      EXTERNAL           LSAME, IDAMAX, ILAENV, DLAMCH, DZNRM2, ZLANGE
*     ..
```

```
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, DCMPLX, DCONJG, DIMAG, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      WANTVL = LSAME( JOBVL, 'V' )
      WANTVR = LSAME( JOBVR, 'V' )
      WNTSNN = LSAME( SENSE, 'N' )
      WNTSNE = LSAME( SENSE, 'E' )
      WNTSNV = LSAME( SENSE, 'V' )
      WNTSNB = LSAME( SENSE, 'B' )
      IF( .NOT.( LSAME( BALANC, 'N' ) .OR. LSAME( BALANC, 'S' ) .OR.
     $    LSAME( BALANC, 'P' ) .OR. LSAME( BALANC, 'B' ) ) ) THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTVL ) .AND. ( .NOT.LSAME( JOBVL, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( ( .NOT.WANTVR ) .AND. ( .NOT.LSAME( JOBVR, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( .NOT.( WNTSNN .OR. WNTSNE .OR. WNTSNB .OR. WNTSNV ) .OR.
     $         ( ( WNTSNE .OR. WNTSNB ) .AND. .NOT.( WANTVL .AND.
     $         WANTVR ) ) ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDVL.LT.1 .OR. ( WANTVL .AND. LDVL.LT.N ) ) THEN
         INFO = -10
      ELSE IF( LDVR.LT.1 .OR. ( WANTVR .AND. LDVR.LT.N ) ) THEN
         INFO = -12
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       CWorkspace refers to complex workspace, and RWorkspace to real
*       workspace. NB refers to the optimal block size for the
*       immediately following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by ZHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
```

```
            MAXWRK = 1
         ELSE
            MAXWRK = N + N*ILAENV( 1, 'ZGEHRD', ' ', N, 1, N, 0 )
*
            IF( WANTVL ) THEN
               CALL ZHSEQR( 'S', 'V', N, 1, N, A, LDA, W, VL, LDVL,
     $                 WORK, -1, INFO )
            ELSE IF( WANTVR ) THEN
               CALL ZHSEQR( 'S', 'V', N, 1, N, A, LDA, W, VR, LDVR,
     $                 WORK, -1, INFO )
            ELSE
               IF( WNTSNN ) THEN
                  CALL ZHSEQR( 'E', 'N', N, 1, N, A, LDA, W, VR, LDVR,
     $                 WORK, -1, INFO )
               ELSE
                  CALL ZHSEQR( 'S', 'N', N, 1, N, A, LDA, W, VR, LDVR,
     $                 WORK, -1, INFO )
               END IF
            END IF
            HSWORK = WORK( 1 )
*
            IF( ( .NOT.WANTVL ) .AND. ( .NOT.WANTVR ) ) THEN
               MINWRK = 2*N
               IF( .NOT.( WNTSNN .OR. WNTSNE ) )
     $            MINWRK = MAX( MINWRK, N*N + 2*N )
               MAXWRK = MAX( MAXWRK, HSWORK )
               IF( .NOT.( WNTSNN .OR. WNTSNE ) )
     $            MAXWRK = MAX( MAXWRK, N*N + 2*N )
            ELSE
               MINWRK = 2*N
               IF( .NOT.( WNTSNN .OR. WNTSNE ) )
     $            MINWRK = MAX( MINWRK, N*N + 2*N )
               MAXWRK = MAX( MAXWRK, HSWORK )
               MAXWRK = MAX( MAXWRK, N + ( N - 1 )*ILAENV( 1, 'ZUNGHR',
     $                       ' ', N, 1, N, -1 ) )
               IF( .NOT.( WNTSNN .OR. WNTSNE ) )
     $            MAXWRK = MAX( MAXWRK, N*N + 2*N )
               MAXWRK = MAX( MAXWRK, 2*N )
            END IF
            MAXWRK = MAX( MAXWRK, MINWRK )
         END IF
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -20
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEEVX', -INFO )
```

```
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ICOND = 0
      ANRM = ZLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
     $   CALL ZLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*     Balance the matrix and compute ABNRM
*
      CALL ZGEBAL( BALANC, N, A, LDA, ILO, IHI, SCALE, IERR )
      ABNRM = ZLANGE( '1', N, N, A, LDA, DUM )
      IF( SCALEA ) THEN
         DUM( 1 ) = ABNRM
         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, 1, 1, DUM, 1, IERR )
         ABNRM = DUM( 1 )
      END IF
*
*     Reduce to upper Hessenberg form
*     (CWorkspace: need 2*N, prefer N+N*NB)
*     (RWorkspace: none)
*
      ITAU = 1
      IWRK = ITAU + N
```

```
      CALL ZGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
      IF( WANTVL ) THEN
*
*        Want left eigenvectors
*        Copy Householder vectors to VL
*
         SIDE = 'L'
         CALL ZLACPY( 'L', N, N, A, LDA, VL, LDVL )
*
*        Generate unitary matrix in VL
*        (CWorkspace: need 2*N-1, prefer N+(N-1)*NB)
*        (RWorkspace: none)
*
         CALL ZUNGHR( N, ILO, IHI, VL, LDVL, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VL
*        (CWorkspace: need 1, prefer HSWORK (see comments) )
*        (RWorkspace: none)
*
         IWRK = ITAU
         CALL ZHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, W, VL, LDVL,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
         IF( WANTVR ) THEN
*
*           Want left and right eigenvectors
*           Copy Schur vectors to VR
*
            SIDE = 'B'
            CALL ZLACPY( 'F', N, N, VL, LDVL, VR, LDVR )
         END IF
*
      ELSE IF( WANTVR ) THEN
*
*        Want right eigenvectors
*        Copy Householder vectors to VR
*
         SIDE = 'R'
         CALL ZLACPY( 'L', N, N, A, LDA, VR, LDVR )
*
*        Generate unitary matrix in VR
*        (CWorkspace: need 2*N-1, prefer N+(N-1)*NB)
*        (RWorkspace: none)
*
         CALL ZUNGHR( N, ILO, IHI, VR, LDVR, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
```

```
*        Perform QR iteration, accumulating Schur vectors in VR
*        (CWorkspace: need 1, prefer HSWORK (see comments) )
*        (RWorkspace: none)
*
         IWRK = ITAU
         CALL ZHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, W, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
      ELSE
*
*        Compute eigenvalues only
*        If condition numbers desired, compute Schur form
*
         IF( WNTSNN ) THEN
            JOB = 'E'
         ELSE
            JOB = 'S'
         END IF
*
*        (CWorkspace: need 1, prefer HSWORK (see comments) )
*        (RWorkspace: none)
*
         IWRK = ITAU
         CALL ZHSEQR( JOB, 'N', N, ILO, IHI, A, LDA, W, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
      END IF
*
*     If INFO > 0 from ZHSEQR, then quit
*
      IF( INFO.GT.0 )
     $   GO TO 50
*
      IF( WANTVL .OR. WANTVR ) THEN
*
*        Compute left and/or right eigenvectors
*        (CWorkspace: need 2*N)
*        (RWorkspace: need N)
*
         CALL ZTREVC( SIDE, 'B', SELECT, N, A, LDA, VL, LDVL, VR, LDVR,
     $                N, NOUT, WORK( IWRK ), RWORK, IERR )
      END IF
*
*     Compute condition numbers if desired
*     (CWorkspace: need N*N+2*N unless SENSE = 'E')
*     (RWorkspace: need 2*N unless SENSE = 'E')
*
      IF( .NOT.WNTSNN ) THEN
         CALL ZTRSNA( SENSE, 'A', SELECT, N, A, LDA, VL, LDVL, VR, LDVR,
     $                RCONDE, RCONDV, N, NOUT, WORK( IWRK ), N, RWORK,
     $                ICOND )
```

```
      END IF
*
      IF( WANTVL ) THEN
*
*        Undo balancing of left eigenvectors
*
         CALL ZGEBAK( BALANC, 'L', N, ILO, IHI, SCALE, N, VL, LDVL,
     $                IERR )
*
*        Normalize left eigenvectors and make largest component real
*
         DO 20 I = 1, N
            SCL = ONE / DZNRM2( N, VL( 1, I ), 1 )
            CALL ZDSCAL( N, SCL, VL( 1, I ), 1 )
            DO 10 K = 1, N
               RWORK( K ) = DBLE( VL( K, I ) )**2 +
     $                      DIMAG( VL( K, I ) )**2
   10       CONTINUE
            K = IDAMAX( N, RWORK, 1 )
            TMP = DCONJG( VL( K, I ) ) / SQRT( RWORK( K ) )
            CALL ZSCAL( N, TMP, VL( 1, I ), 1 )
            VL( K, I ) = DCMPLX( DBLE( VL( K, I ) ), ZERO )
   20    CONTINUE
      END IF
*
      IF( WANTVR ) THEN
*
*        Undo balancing of right eigenvectors
*
         CALL ZGEBAK( BALANC, 'R', N, ILO, IHI, SCALE, N, VR, LDVR,
     $                IERR )
*
*        Normalize right eigenvectors and make largest component real
*
         DO 40 I = 1, N
            SCL = ONE / DZNRM2( N, VR( 1, I ), 1 )
            CALL ZDSCAL( N, SCL, VR( 1, I ), 1 )
            DO 30 K = 1, N
               RWORK( K ) = DBLE( VR( K, I ) )**2 +
     $                      DIMAG( VR( K, I ) )**2
   30       CONTINUE
            K = IDAMAX( N, RWORK, 1 )
            TMP = DCONJG( VR( K, I ) ) / SQRT( RWORK( K ) )
            CALL ZSCAL( N, TMP, VR( 1, I ), 1 )
            VR( K, I ) = DCMPLX( DBLE( VR( K, I ) ), ZERO )
   40    CONTINUE
      END IF
*
*     Undo scaling if necessary
*
```

```
   50 CONTINUE
      IF( SCALEA ) THEN
         CALL ZLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, W( INFO+1 ),
     $                MAX( N-INFO, 1 ), IERR )
         IF( INFO.EQ.0 ) THEN
            IF( ( WNTSNV .OR. WNTSNB ) .AND. ICOND.EQ.0 )
     $         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N, 1, RCONDV, N,
     $                      IERR )
         ELSE
            CALL ZLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, W, N, IERR )
         END IF
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of ZGEEVX
*
      END
```

———-

27.4.4 lisp code

— **zgeevx.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun zgeevx
         (balanc jobvl jobvr sense n a lda w vl ldvl vr ldvr ilo ihi scale
          abnrm rconde rcondv work lwork rwork info)
    (declare (type (double-float) abnrm)
             (type (array double-float (*)) rwork rcondv rconde scale)
             (type (array f2cl-lib:complex16 (*)) work vr vl w a)
             (type (f2cl-lib:integer4) info lwork ihi ilo ldvr ldvl lda n)
             (type (string *) sense jobvr jobvl balanc))
    (f2cl-lib:with-multi-array-data
        ((balanc character balanc-%data% balanc-%offset%)
         (jobvl character jobvl-%data% jobvl-%offset%)
         (jobvr character jobvr-%data% jobvr-%offset%)
         (sense character sense-%data% sense-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (w f2cl-lib:complex16 w-%data% w-%offset%)
         (vl f2cl-lib:complex16 vl-%data% vl-%offset%)
         (vr f2cl-lib:complex16 vr-%data% vr-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
```

```
   (scale double-float scale-%data% scale-%offset%)
   (rconde double-float rconde-%data% rconde-%offset%)
   (rcondv double-float rcondv-%data% rcondv-%offset%)
   (rwork double-float rwork-%data% rwork-%offset%))
(prog ((dum (make-array 1 :element-type 'double-float))
       (select (make-array 1 :element-type 't)) (tmp #C(0.0d0 0.0d0))
       (anrm 0.0d0) (bignum 0.0d0) (cscale 0.0d0) (eps 0.0d0) (scl 0.0d0)
       (smlnum 0.0d0) (hswork 0) (i 0) (icond 0) (ierr 0) (itau 0)
       (iwrk 0) (k 0) (maxwrk 0) (minwrk 0) (nout 0)
       (job
        (make-array '(1) :element-type 'character :initial-element #\ ))
       (side
        (make-array '(1) :element-type 'character :initial-element #\ ))
       (lquery nil) (scalea nil) (wantvl nil) (wantvr nil) (wntsnb nil)
       (wntsne nil) (wntsnn nil) (wntsnv nil))
  (declare (type (array double-float (1)) dum)
           (type (array f2cl-lib:logical (1)) select)
           (type (f2cl-lib:complex16) tmp)
           (type (double-float) anrm bignum cscale eps scl smlnum)
           (type (f2cl-lib:integer4) hswork i icond ierr itau iwrk k
            maxwrk minwrk nout)
           (type (string 1) job side)
           (type f2cl-lib:logical lquery scalea wantvl wantvr wntsnb
            wntsne wntsnn wntsnv))
  (setf info 0)
  (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
  (setf wantvl
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame jobvl "V")
            (declare (ignore var-1))
            (when var-0 (setf jobvl var-0))
            ret-val))
  (setf wantvr
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame jobvr "V")
            (declare (ignore var-1))
            (when var-0 (setf jobvr var-0))
            ret-val))
  (setf wntsnn
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame sense "N")
            (declare (ignore var-1))
            (when var-0 (setf sense var-0))
            ret-val))
  (setf wntsne
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame sense "E")
            (declare (ignore var-1))
            (when var-0 (setf sense var-0))
            ret-val))
```

```
(setf wntsnv
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "V")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wntsnb
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "B")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(cond
 ((not
   (or
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "N")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "S")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "P")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "B")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)))
  (setf info -1))
 ((and (not wantvl)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvl "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvl var-0))
          ret-val)))
  (setf info -2))
 ((and (not wantvr)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvr "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvr var-0))
```

```
            ret-val)))
     (setf info -3))
    ((or (not (or wntsnn wntsne wntsnb wntsnv))
         (and (or wntsne wntsnb) (not (and wantvl wantvr))))
     (setf info -4))
    ((< n 0) (setf info -5))
    ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
     (setf info -7))
    ((or (< ldvl 1) (and wantvl (< ldvl n))) (setf info -10))
    ((or (< ldvr 1) (and wantvr (< ldvr n))) (setf info -12)))
   (cond
    ((= info 0)
     (cond ((= n 0) (setf minwrk 1) (setf maxwrk 1))
           (t
            (setf maxwrk
                    (f2cl-lib:int-add n
                                      (f2cl-lib:int-mul n
                                                        (multiple-value-bind
                                                            (ret-val var-0
                                                             var-1 var-2
                                                             var-3 var-4
                                                             var-5 var-6)
                                                            (ilaenv 1
                                                             "ZGEHRD" " "
                                                             n 1 n 0)
                                                          (declare
                                                           (ignore var-0
                                                            var-1 var-2
                                                            var-4 var-6))
                                                          (setf n var-3)
                                                          (setf n var-5)
                                                          ret-val))))
            (cond
             (wantvl
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                   var-9 var-10 var-11 var-12)
                  (zhseqr "S" "V" n 1 n a lda w vl ldvl work -1 info)
                (declare
                 (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10
                  var-11))
                (when var-2 (setf n var-2))
                (when var-4 (setf n var-4))
                (when var-6 (setf lda var-6))
                (when var-9 (setf ldvl var-9))
                (when var-12 (setf info var-12))))
             (wantvr
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                   var-9 var-10 var-11 var-12)
```

```
      (zhseqr "S" "V" n 1 n a lda w vr ldvr work -1 info)
    (declare
     (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10
      var-11))
    (when var-2 (setf n var-2))
    (when var-4 (setf n var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldvr var-9))
    (when var-12 (setf info var-12))))
 (t
  (cond
   (wntsnn
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (zhseqr "E" "N" n 1 n a lda w vr ldvr work -1 info)
      (declare
       (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10
        var-11))
      (when var-2 (setf n var-2))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldvr var-9))
      (when var-12 (setf info var-12))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (zhseqr "S" "N" n 1 n a lda w vr ldvr work -1 info)
      (declare
       (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10
        var-11))
      (when var-2 (setf n var-2))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldvr var-9))
      (when var-12 (setf info var-12)))))))
(setf hswork
        (f2cl-lib:int
         (f2cl-lib:fref work-%data% (1) ((1 *))
                        work-%offset%)))
(cond
 ((and (not wantvl) (not wantvr))
  (setf minwrk (f2cl-lib:int-mul 2 n))
  (if (not (or wntsnn wntsne))
      (setf minwrk
              (max (the f2cl-lib:integer4 minwrk)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add
                         (f2cl-lib:int-mul n n)
```

```
                        (f2cl-lib:int-mul 2 n))))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4 hswork)))
 (if (not (or wntsnn wntsne))
     (setf maxwrk
             (max (the f2cl-lib:integer4 maxwrk)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add
                        (f2cl-lib:int-mul n n)
                        (f2cl-lib:int-mul 2 n)))))))
(t (setf minwrk (f2cl-lib:int-mul 2 n))
 (if (not (or wntsnn wntsne))
     (setf minwrk
             (max (the f2cl-lib:integer4 minwrk)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add
                        (f2cl-lib:int-mul n n)
                        (f2cl-lib:int-mul 2 n))))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4 hswork)))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n
                                     (f2cl-lib:int-mul
                                      (f2cl-lib:int-sub n 1)
                                      (multiple-value-bind
                                          (ret-val var-0
                                           var-1 var-2 var-3
                                           var-4 var-5 var-6)
                                          (ilaenv 1 "ZUNGHR"
                                           " " n 1 n -1)
                                        (declare
                                         (ignore var-0 var-1
                                          var-2 var-4 var-6))
                                        (setf n var-3)
                                        (setf n var-5)
                                        ret-val))))))
 (if (not (or wntsnn wntsne))
     (setf maxwrk
             (max (the f2cl-lib:integer4 maxwrk)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add
                        (f2cl-lib:int-mul n n)
                        (f2cl-lib:int-mul 2 n))))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
```

```
                                (f2cl-lib:int-mul 2 n))))))
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4 minwrk)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce maxwrk 'f2cl-lib:complex16))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -20)))))
(cond
 ((/= info 0) (xerbla "ZGEEVX" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(setf eps (dlamch "P"))
(setf smlnum (dlamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf icond 0)
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "M" n n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf scalea f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf scalea f2cl-lib:%true%)
  (setf cscale smlnum))
 ((> anrm bignum) (setf scalea f2cl-lib:%true%) (setf cscale bignum)))
(if scalea
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (zlascl "G" 0 0 anrm cscale n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf cscale var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgebal balanc n a lda ilo ihi scale ierr)
```

```
  (declare (ignore var-1 var-2 var-6))
  (setf balanc var-0)
  (setf lda var-3)
  (setf ilo var-4)
  (setf ihi var-5)
  (setf ierr var-7))
(setf abnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "1" n n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(cond
 (scalea (setf (f2cl-lib:fref dum (1) ((1 1))) abnrm)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 cscale anrm 1 1 dum 1 ierr)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
    (when var-3 (setf cscale var-3))
    (when var-4 (setf anrm var-4))
    (when var-9 (setf ierr var-9)))
  (setf abnrm (f2cl-lib:fref dum (1) ((1 1))))))
(setf itau 1)
(setf iwrk (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zgehrd n ilo ihi a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf ilo var-1))
  (when var-2 (setf ihi var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(cond
 (wantvl (f2cl-lib:f2cl-set-string side "L" (string 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "L" n n a lda vl ldvl)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvl var-6)))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zunghr n ilo ihi vl ldvl
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf ilo var-1))
  (when var-2 (setf ihi var-2))
  (when var-4 (setf ldvl var-4))
  (when var-8 (setf ierr var-8)))
(setf iwrk itau)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (zhseqr "S" "V" n ilo ihi a lda w vl ldvl
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf n var-2))
  (when var-3 (setf ilo var-3))
  (when var-4 (setf ihi var-4))
  (when var-6 (setf lda var-6))
  (when var-9 (setf ldvl var-9))
  (when var-12 (setf info var-12)))
(cond
 (wantvr (f2cl-lib:f2cl-set-string side "B" (string 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "F" n n vl ldvl vr ldvr)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldvl var-4))
    (when var-6 (setf ldvr var-6))))))
(wantvr (f2cl-lib:f2cl-set-string side "R" (string 1))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (zlacpy "L" n n a lda vr ldvr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvr var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (zunghr n ilo ihi vr ldvr
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
```

```
                                ((1 *)) work-%offset%)
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                                ((1 *)) work-%offset%)
            (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
        (declare (ignore var-3 var-5 var-6 var-7))
        (when var-0 (setf n var-0))
        (when var-1 (setf ilo var-1))
        (when var-2 (setf ihi var-2))
        (when var-4 (setf ldvr var-4))
        (when var-8 (setf ierr var-8)))
      (setf iwrk itau)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (zhseqr "S" "V" n ilo ihi a lda w vr ldvr
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                                ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf n var-2))
        (when var-3 (setf ilo var-3))
        (when var-4 (setf ihi var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldvr var-9))
        (when var-12 (setf info var-12))))
     (t
      (cond (wntsnn (f2cl-lib:f2cl-set-string job "E" (string 1)))
            (t (f2cl-lib:f2cl-set-string job "S" (string 1))))
      (setf iwrk itau)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (zhseqr job "N" n ilo ihi a lda w vr ldvr
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                                ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
        (declare (ignore var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-0 (setf job var-0))
        (when var-2 (setf n var-2))
        (when var-3 (setf ilo var-3))
        (when var-4 (setf ihi var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldvr var-9))
        (when var-12 (setf info var-12)))))
    (if (> info 0)
        (go label50))
    (cond
     ((or wantvl wantvr)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
        var-10 var-11 var-12 var-13 var-14)
       (ztrevc side "B" select n a lda vl ldvl vr ldvr n nout
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                              ((1 *)) work-%offset%)
        rwork ierr)
     (declare (ignore var-1 var-2 var-4 var-6 var-8 var-12 var-13))
     (when var-0 (setf side var-0))
     (when var-3 (setf n var-3))
     (when var-5 (setf lda var-5))
     (when var-7 (setf ldvl var-7))
     (when var-9 (setf ldvr var-9))
     (when var-10 (setf n var-10))
     (when var-11 (setf nout var-11))
     (when var-14 (setf ierr var-14)))))
(cond
 ((not wntsnn)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17)
      (ztrsna sense "A" select n a lda vl ldvl vr ldvr rconde rcondv n
       nout
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                              ((1 *)) work-%offset%)
       n rwork icond)
     (declare
      (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-11 var-14
       var-16))
     (when var-0 (setf sense var-0))
     (when var-3 (setf n var-3))
     (when var-5 (setf lda var-5))
     (when var-7 (setf ldvl var-7))
     (when var-9 (setf ldvr var-9))
     (when var-12 (setf n var-12))
     (when var-13 (setf nout var-13))
     (when var-15 (setf n var-15))
     (when var-17 (setf icond var-17)))))
(cond
 (wantvl
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zgebak balanc "L" n ilo ihi scale n vl ldvl ierr)
     (declare (ignore var-1 var-2 var-3 var-4 var-5 var-7))
     (setf balanc var-0)
     (setf n var-6)
     (setf ldvl var-8)
     (setf ierr var-9))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
     (tagbody
       (setf scl
```

```
        (/ one
           (multiple-value-bind (ret-val var-0 var-1 var-2)
               (dznrm2 n
                (f2cl-lib:array-slice vl-%data%
                                      f2cl-lib:complex16 (1 i)
                                      ((1 ldvl) (1 *))
                                      vl-%offset%)
                1)
             (declare (ignore var-1 var-2))
             (when var-0 (setf n var-0))
             ret-val)))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (zdscal n scl
     (f2cl-lib:array-slice vl-%data% f2cl-lib:complex16 (1 i)
                           ((1 ldvl) (1 *)) vl-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf scl var-1)))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (k) ((1 *)) rwork-%offset%)
            (+
             (expt
              (f2cl-lib:dble
               (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                              vl-%offset%))
              2)
             (expt
              (f2cl-lib:dimag
               (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                              vl-%offset%))
              2)))
   label10))
(setf k
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (idamax n rwork 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(setf tmp
        (coerce
         (/
          (f2cl-lib:dconjg
           (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                          vl-%offset%))
          (f2cl-lib:fsqrt
           (f2cl-lib:fref rwork-%data% (k) ((1 *))
                          rwork-%offset%)))
```

```
                 'f2cl-lib:complex16))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (zscal n tmp
             (f2cl-lib:array-slice vl-%data% f2cl-lib:complex16 (1 i)
                                   ((1 ldvl) (1 *)) vl-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-1 (setf tmp var-1)))
        (setf (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                             vl-%offset%)
                (f2cl-lib:dcmplx
                 (f2cl-lib:dble
                  (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                                 vl-%offset%))
                 zero))
      label20))))
(cond
 (wantvr
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zgebak balanc "R" n ilo ihi scale n vr ldvr ierr)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-7))
    (setf balanc var-0)
    (setf n var-6)
    (setf ldvr var-8)
    (setf ierr var-9))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf scl
              (/ one
                 (multiple-value-bind (ret-val var-0 var-1 var-2)
                     (dznrm2 n
                      (f2cl-lib:array-slice vr-%data%
                                            f2cl-lib:complex16 (1 i)
                                            ((1 ldvr) (1 *))
                                            vr-%offset%)
                      1)
                   (declare (ignore var-1 var-2))
                   (when var-0 (setf n var-0))
                   ret-val)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (zdscal n scl
           (f2cl-lib:array-slice vr-%data% f2cl-lib:complex16 (1 i)
                                 ((1 ldvr) (1 *)) vr-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf n var-0))
        (when var-1 (setf scl var-1)))
```

```
        (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                      ((> k n) nil)
          (tagbody
            (setf (f2cl-lib:fref rwork-%data% (k) ((1 *)) rwork-%offset%)
                    (+
                     (expt
                      (f2cl-lib:dble
                       (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                                      vr-%offset%))
                      2)
                     (expt
                      (f2cl-lib:dimag
                       (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                                      vr-%offset%))
                      2)))
           label30))
        (setf k
                (multiple-value-bind (ret-val var-0 var-1 var-2)
                    (idamax n rwork 1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf n var-0))
                  ret-val))
        (setf tmp
                (coerce
                 (/
                  (f2cl-lib:dconjg
                   (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                                  vr-%offset%))
                  (f2cl-lib:fsqrt
                   (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                  rwork-%offset%)))
                 'f2cl-lib:complex16))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (zscal n tmp
             (f2cl-lib:array-slice vr-%data% f2cl-lib:complex16 (1 i)
                                   ((1 ldvr) (1 *)) vr-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-1 (setf tmp var-1)))
        (setf (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                             vr-%offset%)
                (f2cl-lib:dcmplx
                 (f2cl-lib:dble
                  (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                                 vr-%offset%))
                 zero))
      label40))))
label50
 (cond
```

```
          (scalea
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
               (zlascl "G" 0 0 cscale anrm (f2cl-lib:int-sub n info) 1
                (f2cl-lib:array-slice w-%data% f2cl-lib:complex16 ((+ info 1))
                                      ((1 *)) w-%offset%)
                (max (the f2cl-lib:integer4 (f2cl-lib:int-sub n info))
                     (the f2cl-lib:integer4 1))
                ierr)
             (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
             (when var-3 (setf cscale var-3))
             (when var-4 (setf anrm var-4))
             (when var-9 (setf ierr var-9)))
           (cond
            ((= info 0)
             (if (and (or wntsnv wntsnb) (= icond 0))
                 (multiple-value-bind
                     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                      var-9)
                     (dlascl "G" 0 0 cscale anrm n 1 rcondv n ierr)
                   (declare (ignore var-0 var-1 var-2 var-6 var-7))
                   (when var-3 (setf cscale var-3))
                   (when var-4 (setf anrm var-4))
                   (when var-5 (setf n var-5))
                   (when var-8 (setf n var-8))
                   (when var-9 (setf ierr var-9)))))
            (t
             (multiple-value-bind
                 (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                 (zlascl "G" 0 0 cscale anrm (f2cl-lib:int-sub ilo 1) 1 w n
                  ierr)
               (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
               (when var-3 (setf cscale var-3))
               (when var-4 (setf anrm var-4))
               (when var-8 (setf n var-8))
               (when var-9 (setf ierr var-9)))))))
          (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce maxwrk 'f2cl-lib:complex16))
          (go end_label)
         end_label
          (return
           (values balanc jobvl jobvr sense n nil lda nil nil ldvl nil ldvr ilo
                   ihi nil abnrm nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgeevx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
```

```
'((string) (string) (string)
  (string)
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (double-float)
  (array double-float (*))
  (array double-float (*))
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::balanc
  fortran-to-lisp::jobvl
  fortran-to-lisp::jobvr
  fortran-to-lisp::sense
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil
  fortran-to-lisp::ldvl nil
  fortran-to-lisp::ldvr
  fortran-to-lisp::ilo
  fortran-to-lisp::ihi nil
  fortran-to-lisp::abnrm nil nil
  nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zgebak
  fortran-to-lisp::zgebal
  fortran-to-lisp::xerbla
  fortran-to-lisp::ilaenv))))
```

27.5 zgegs - DEPRECATED use zgges

27.6 zgegv - DEPRECATED use zggev

27.7 zgges

27.7.1 Axiom unit tests

— **zgges.input** —

```
)set break resume
)sys rm -f zgges.output
)spool zgges.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

27.7.2 Axiom help page

— **zgges.help** —

```
======================================================================
zgges examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgges (JOBVSL, JOBVSR, SORT, SELCTG, N, A, LDA, B, LDB,
           SDIM, ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, WORK, LWORK, RWORK,
           BWORK, INFO)
            ZGGES computes the eigenvalues, the Schur form, and, optionally,
           the matrix of Schur vectors for GE matrices

Function/Subroutine Documentation
   subroutine zgges (character JOBVSL, character JOBVSR, character SORT,
```

```
      external SELCTG, integer N, complex*16, dimension( lda, * ) A, integer
      LDA, complex*16, dimension( ldb, * ) B, integer LDB, integer SDIM,
      complex*16, dimension( * ) ALPHA, complex*16, dimension( * ) BETA,
      complex*16, dimension( ldvsl, * ) VSL, integer LDVSL, complex*16,
      dimension( ldvsr, * ) VSR, integer LDVSR, complex*16, dimension( * )
      WORK, integer LWORK, double precision, dimension( * ) RWORK, logical,
      dimension( * ) BWORK, integer INFO)
       ZGGES computes the eigenvalues, the Schur form, and, optionally, the
      matrix of Schur vectors for GE matrices

Purpose:
   ZGGES computes for a pair of N-by-N complex nonsymmetric matrices
   (A,B), the generalized eigenvalues, the generalized complex Schur
   form (S, T), and optionally left and/or right Schur vectors (VSL
   and VSR). This gives the generalized Schur factorization

   (A,B) = ( (VSL)*S*(VSR)**H, (VSL)*T*(VSR)**H )

   where (VSR)**H is the conjugate-transpose of VSR.

   Optionally, it also orders the eigenvalues so that a selected cluster
   of eigenvalues appears in the leading diagonal blocks of the upper
   triangular matrix S and the upper triangular matrix T. The leading
   columns of VSL and VSR then form an unitary basis for the
   corresponding left and right eigenspaces (deflating subspaces).

   (If only the generalized eigenvalues are needed, use the driver
   ZGGEV instead, which is faster.)

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar w
   or a ratio alpha/beta = w, such that  A - w*B is singular.  It is
   usually represented as the pair (alpha,beta), as there is a
   reasonable interpretation for beta=0, and even for both being zero.

   A pair of matrices (S,T) is in generalized complex Schur form if S
   and T are upper triangular and, in addition, the diagonal elements
   of T are non-negative real numbers.

Parameters:

 JOBVSL
   JOBVSL is CHARACTER*1
   = 'N':  do not compute the left Schur vectors;
   = 'V':  compute the left Schur vectors.

 JOBVSR
   JOBVSR is CHARACTER*1
   = 'N':  do not compute the right Schur vectors;
   = 'V':  compute the right Schur vectors.
```

```
SORT
  SORT is CHARACTER*1
  Specifies whether or not to order the eigenvalues on the
  diagonal of the generalized Schur form.
  = 'N':  Eigenvalues are not ordered;
  = 'S':  Eigenvalues are ordered (see SELCTG).

SELCTG
  SELCTG is a LOGICAL FUNCTION of two COMPLEX*16 arguments
  SELCTG must be declared EXTERNAL in the calling subroutine.
  If SORT = 'N', SELCTG is not referenced.
  If SORT = 'S', SELCTG is used to select eigenvalues to sort
  to the top left of the Schur form.
  An eigenvalue ALPHA(j)/BETA(j) is selected if
  SELCTG(ALPHA(j),BETA(j)) is true.

  Note that a selected complex eigenvalue may no longer satisfy
  SELCTG(ALPHA(j),BETA(j)) = .TRUE. after ordering, since
  ordering may change the value of complex eigenvalues
  (especially if the eigenvalue is ill-conditioned), in this
  case INFO is set to N+2 (See INFO below).

N
  N is INTEGER
  The order of the matrices A, B, VSL, and VSR.  N >= 0.

A
  A is COMPLEX*16 array, dimension (LDA, N)
  On entry, the first of the pair of matrices.
  On exit, A has been overwritten by its generalized Schur form S.

LDA
  LDA is INTEGER
  The leading dimension of A.  LDA >= max(1,N).

B
  B is COMPLEX*16 array, dimension (LDB, N)
  On entry, the second of the pair of matrices.
  On exit, B has been overwritten by its generalized Schur form T.

LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).

SDIM
  SDIM is INTEGER
  If SORT = 'N', SDIM = 0.
  If SORT = 'S', SDIM = number of eigenvalues (after sorting)
  for which SELCTG is true.
```

```
ALPHA
  ALPHA is COMPLEX*16 array, dimension (N)

BETA
  BETA is COMPLEX*16 array, dimension (N)
  On exit,  ALPHA(j)/BETA(j), j=1,...,N, will be the
  generalized eigenvalues.  ALPHA(j), j=1,...,N  and  BETA(j),
  j=1,...,N  are the diagonals of the complex Schur form (A,B)
  output by ZGGES. The  BETA(j) will be non-negative real.

  Note: the quotients ALPHA(j)/BETA(j) may easily over- or
  underflow, and BETA(j) may even be zero.  Thus, the user
  should avoid naively computing the ratio alpha/beta.
  However, ALPHA will be always less than and usually
  comparable with norm(A) in magnitude, and BETA always less
  than and usually comparable with norm(B).

VSL
  VSL is COMPLEX*16 array, dimension (LDVSL,N)
  If JOBVSL = 'V', VSL will contain the left Schur vectors.
  Not referenced if JOBVSL = 'N'.

LDVSL
  LDVSL is INTEGER
  The leading dimension of the matrix VSL. LDVSL >= 1, and
  if JOBVSL = 'V', LDVSL >= N.

VSR
  VSR is COMPLEX*16 array, dimension (LDVSR,N)
  If JOBVSR = 'V', VSR will contain the right Schur vectors.
  Not referenced if JOBVSR = 'N'.

LDVSR
  LDVSR is INTEGER
  The leading dimension of the matrix VSR. LDVSR >= 1, and
  if JOBVSR = 'V', LDVSR >= N.

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,2*N).
  For good performance, LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.
```

```
 RWORK
   RWORK is DOUBLE PRECISION array, dimension (8*N)

 BWORK
   BWORK is LOGICAL array, dimension (N)
   Not referenced if SORT = 'N'.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value.
   =1,...,N:
         The QZ iteration failed.  (A,B) are not in Schur
         form, but ALPHA(j) and BETA(j) should be correct for
         j=INFO+1,...,N.
   > N:  =N+1: other than QZ iteration failed in ZHGEQZ
         =N+2: after reordering, roundoff changed values of
               some complex eigenvalues so that leading
               eigenvalues in the Generalized Schur form no
               longer satisfy SELCTG=.TRUE.  This could also
               be caused due to scaling.
         =N+3: reordering failed in ZTGSEN.
```

———-

27.7.3 fortran code

— **zgges.f** —

```
*  ===================================================================
      SUBROUTINE ZGGES( JOBVSL, JOBVSR, SORT, SELCTG, N, A, LDA, B, LDB,
     $                  SDIM, ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, WORK,
     $                  LWORK, RWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVSL, JOBVSR, SORT
      INTEGER            INFO, LDA, LDB, LDVSL, LDVSR, LWORK, N, SDIM
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      DOUBLE PRECISION   RWORK( * )
```

```
      COMPLEX*16         A( LDA, * ), ALPHA( * ), B( LDB, * ),
     $                   BETA( * ), VSL( LDVSL, * ), VSR( LDVSR, * ),
     $                   WORK( * )
*     ..
*     .. Function Arguments ..
      LOGICAL            SELCTG
      EXTERNAL           SELCTG
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
      COMPLEX*16         CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0D0, 0.0D0 ),
     $                   CONE = ( 1.0D0, 0.0D0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            CURSL, ILASCL, ILBSCL, ILVSL, ILVSR, LASTSL,
     $                   LQUERY, WANTST
      INTEGER            I, ICOLS, IERR, IHI, IJOBVL, IJOBVR, ILEFT,
     $                   ILO, IRIGHT, IROWS, IRWRK, ITAU, IWRK, LWKMIN,
     $                   LWKOPT
      DOUBLE PRECISION   ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS, PVSL,
     $                   PVSR, SMLNUM
*     ..
*     .. Local Arrays ..
      INTEGER            IDUM( 1 )
      DOUBLE PRECISION   DIF( 2 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, XERBLA, ZGEQRF, ZGGBAK, ZGGBAL, ZGGHRD,
     $                   ZHGEQZ, ZLACPY, ZLASCL, ZLASET, ZTGSEN, ZUNGQR,
     $                   ZUNMQR
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, ZLANGE
      EXTERNAL           LSAME, ILAENV, DLAMCH, ZLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVSL, 'N' ) ) THEN
```

```
         IJOBVL = 1
         ILVSL = .FALSE.
      ELSE IF( LSAME( JOBVSL, 'V' ) ) THEN
         IJOBVL = 2
         ILVSL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVSL = .FALSE.
      END IF
*
      IF( LSAME( JOBVSR, 'N' ) ) THEN
         IJOBVR = 1
         ILVSR = .FALSE.
      ELSE IF( LSAME( JOBVSR, 'V' ) ) THEN
         IJOBVR = 2
         ILVSR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVSR = .FALSE.
      END IF
*
      WANTST = LSAME( SORT, 'S' )
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -2
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDVSL.LT.1 .OR. ( ILVSL .AND. LDVSL.LT.N ) ) THEN
         INFO = -14
      ELSE IF( LDVSR.LT.1 .OR. ( ILVSR .AND. LDVSR.LT.N ) ) THEN
         INFO = -16
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
```

```
*       following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         LWKMIN = MAX( 1, 2*N )
         LWKOPT = MAX( 1, N + N*ILAENV( 1, 'ZGEQRF', ' ', N, 1, N, 0 ) )
         LWKOPT = MAX( LWKOPT, N +
     $                 N*ILAENV( 1, 'ZUNMQR', ' ', N, 1, N, -1 ) )
         IF( ILVSL ) THEN
            LWKOPT = MAX( LWKOPT, N +
     $                    N*ILAENV( 1, 'ZUNGQR', ' ', N, 1, N, -1 ) )
         END IF
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.LWKMIN .AND. .NOT.LQUERY )
     $      INFO = -18
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGGES ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = ZLANGE( 'M', N, N, A, LDA, RWORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
```

```
      END IF
*
      IF( ILASCL )
     $   CALL ZLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = ZLANGE( 'M', N, N, B, LDB, RWORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
*
      IF( ILBSCL )
     $   CALL ZLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrix to make it more nearly triangular
*     (Real Workspace: need 6*N)
*
      ILEFT = 1
      IRIGHT = N + 1
      IRWRK = IRIGHT + N
      CALL ZGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, RWORK( ILEFT ),
     $             RWORK( IRIGHT ), RWORK( IRWRK ), IERR )
*
*     Reduce B to triangular form (QR decomposition of B)
*     (Complex Workspace: need N, prefer N*NB)
*
      IROWS = IHI + 1 - ILO
      ICOLS = N + 1 - ILO
      ITAU = 1
      IWRK = ITAU + IROWS
      CALL ZGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to matrix A
*     (Complex Workspace: need N, prefer N*NB)
*
      CALL ZUNMQR( 'L', 'C', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VSL
*     (Complex Workspace: need N, prefer N*NB)
*
      IF( ILVSL ) THEN
```

```
         CALL ZLASET( 'Full', N, N, CZERO, CONE, VSL, LDVSL )
         IF( IROWS.GT.1 ) THEN
            CALL ZLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VSL( ILO+1, ILO ), LDVSL )
         END IF
         CALL ZUNGQR( IROWS, IROWS, IROWS, VSL( ILO, ILO ), LDVSL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Initialize VSR
*
      IF( ILVSR )
     $   CALL ZLASET( 'Full', N, N, CZERO, CONE, VSR, LDVSR )
*
*     Reduce to generalized Hessenberg form
*     (Workspace: none needed)
*
      CALL ZGGHRD( JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB, VSL,
     $             LDVSL, VSR, LDVSR, IERR )
*
      SDIM = 0
*
*     Perform QZ algorithm, computing Schur vectors if desired
*     (Complex Workspace: need N)
*     (Real Workspace: need N)
*
      IWRK = ITAU
      CALL ZHGEQZ( 'S', JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, WORK( IWRK ),
     $             LWORK+1-IWRK, RWORK( IRWRK ), IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 30
      END IF
*
*     Sort eigenvalues ALPHA/BETA if desired
*     (Workspace: none needed)
*
      IF( WANTST ) THEN
*
*        Undo scaling on eigenvalues before selecting
*
         IF( ILASCL )
     $      CALL ZLASCL( 'G', 0, 0, ANRM, ANRMTO, N, 1, ALPHA, N, IERR )
```

```
         IF( ILBSCL )
     $      CALL ZLASCL( 'G', 0, 0, BNRM, BNRMTO, N, 1, BETA, N, IERR )
*
*        Select eigenvalues
*
         DO 10 I = 1, N
            BWORK( I ) = SELCTG( ALPHA( I ), BETA( I ) )
   10    CONTINUE
*
         CALL ZTGSEN( 0, ILVSL, ILVSR, BWORK, N, A, LDA, B, LDB, ALPHA,
     $                BETA, VSL, LDVSL, VSR, LDVSR, SDIM, PVSL, PVSR,
     $                DIF, WORK( IWRK ), LWORK-IWRK+1, IDUM, 1, IERR )
         IF( IERR.EQ.1 )
     $      INFO = N + 3
*
      END IF
*
*     Apply back-permutation to VSL and VSR
*     (Workspace: none needed)
*
      IF( ILVSL )
     $   CALL ZGGBAK( 'P', 'L', N, ILO, IHI, RWORK( ILEFT ),
     $                RWORK( IRIGHT ), N, VSL, LDVSL, IERR )
      IF( ILVSR )
     $   CALL ZGGBAK( 'P', 'R', N, ILO, IHI, RWORK( ILEFT ),
     $                RWORK( IRIGHT ), N, VSR, LDVSR, IERR )
*
*     Undo scaling
*
      IF( ILASCL ) THEN
         CALL ZLASCL( 'U', 0, 0, ANRMTO, ANRM, N, N, A, LDA, IERR )
         CALL ZLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHA, N, IERR )
      END IF
*
      IF( ILBSCL ) THEN
         CALL ZLASCL( 'U', 0, 0, BNRMTO, BNRM, N, N, B, LDB, IERR )
         CALL ZLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
      END IF
*
      IF( WANTST ) THEN
*
         Check if reordering is correct
*
         LASTSL = .TRUE.
         SDIM = 0
         DO 20 I = 1, N
            CURSL = SELCTG( ALPHA( I ), BETA( I ) )
            IF( CURSL )
     $         SDIM = SDIM + 1
            IF( CURSL .AND. .NOT.LASTSL )
```

```
     $         INFO = N + 2
            LASTSL = CURSL
   20    CONTINUE
*
      END IF
*
   30 CONTINUE
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of ZGGES
*
      END
```

———-

27.7.4 lisp code

— **zgges.lisp** —

———-

27.8 zgges3

27.8.1 Axiom unit tests

— **zgges3.input** —

```
)set break resume
)sys rm -f zgges3.output
)spool zgges3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

27.8.2 Axiom help page

— zgges3.help —

```
====================================================================
zgges3 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgges3 (JOBVSL, JOBVSR, SORT, SELCTG, N, A, LDA, B, LDB,
           SDIM, ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, WORK, LWORK, RWORK,
           BWORK, INFO)
            ZGGES3 computes the eigenvalues, the Schur form, and, optionally,
           the matrix of Schur vectors for GE matrices (blocked algorithm)

Function/Subroutine Documentation
   subroutine zgges3 (character JOBVSL, character JOBVSR, character SORT,
       external SELCTG, integer N, complex*16, dimension( lda, * ) A, integer
       LDA, complex*16, dimension( ldb, * ) B, integer LDB, integer SDIM,
       complex*16, dimension( * ) ALPHA, complex*16, dimension( * ) BETA,
       complex*16, dimension( ldvsl, * ) VSL, integer LDVSL, complex*16,
       dimension( ldvsr, * ) VSR, integer LDVSR, complex*16, dimension( * )
       WORK, integer LWORK, double precision, dimension( * ) RWORK, logical,
       dimension( * ) BWORK, integer INFO)
        ZGGES3 computes the eigenvalues, the Schur form, and, optionally, the
       matrix of Schur vectors for GE matrices (blocked algorithm)

Purpose:
   ZGGES3 computes for a pair of N-by-N complex nonsymmetric matrices
   (A,B), the generalized eigenvalues, the generalized complex Schur
   form (S, T), and optionally left and/or right Schur vectors (VSL
   and VSR). This gives the generalized Schur factorization

   (A,B) = ( (VSL)*S*(VSR)**H, (VSL)*T*(VSR)**H )

   where (VSR)**H is the conjugate-transpose of VSR.

   Optionally, it also orders the eigenvalues so that a selected cluster
   of eigenvalues appears in the leading diagonal blocks of the upper
   triangular matrix S and the upper triangular matrix T. The leading
   columns of VSL and VSR then form an unitary basis for the
   corresponding left and right eigenspaces (deflating subspaces).

   (If only the generalized eigenvalues are needed, use the driver
```

```
   ZGGEV instead, which is faster.)

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar w
   or a ratio alpha/beta = w, such that  A - w*B is singular.  It is
   usually represented as the pair (alpha,beta), as there is a
   reasonable interpretation for beta=0, and even for both being zero.

   A pair of matrices (S,T) is in generalized complex Schur form if S
   and T are upper triangular and, in addition, the diagonal elements
   of T are non-negative real numbers.

Parameters:

 JOBVSL
   JOBVSL is CHARACTER*1
   = 'N':  do not compute the left Schur vectors;
   = 'V':  compute the left Schur vectors.

 JOBVSR
   JOBVSR is CHARACTER*1
   = 'N':  do not compute the right Schur vectors;
   = 'V':  compute the right Schur vectors.

 SORT
   SORT is CHARACTER*1
   Specifies whether or not to order the eigenvalues on the
   diagonal of the generalized Schur form.
   = 'N':  Eigenvalues are not ordered;
   = 'S':  Eigenvalues are ordered (see SELCTG).

 SELCTG
   SELCTG is a LOGICAL FUNCTION of two COMPLEX*16 arguments
   SELCTG must be declared EXTERNAL in the calling subroutine.
   If SORT = 'N', SELCTG is not referenced.
   If SORT = 'S', SELCTG is used to select eigenvalues to sort
   to the top left of the Schur form.
   An eigenvalue ALPHA(j)/BETA(j) is selected if
   SELCTG(ALPHA(j),BETA(j)) is true.

   Note that a selected complex eigenvalue may no longer satisfy
   SELCTG(ALPHA(j),BETA(j)) = .TRUE. after ordering, since
   ordering may change the value of complex eigenvalues
   (especially if the eigenvalue is ill-conditioned), in this
   case INFO is set to N+2 (See INFO below).

 N
   N is INTEGER
   The order of the matrices A, B, VSL, and VSR.  N >= 0.

 A
```

```
  A is COMPLEX*16 array, dimension (LDA, N)
  On entry, the first of the pair of matrices.
  On exit, A has been overwritten by its generalized Schur form S.

LDA
  LDA is INTEGER
  The leading dimension of A.  LDA >= max(1,N).

B
  B is COMPLEX*16 array, dimension (LDB, N)
  On entry, the second of the pair of matrices.
  On exit, B has been overwritten by its generalized Schur form T.

LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).

SDIM
  SDIM is INTEGER
  If SORT = 'N', SDIM = 0.
  If SORT = 'S', SDIM = number of eigenvalues (after sorting)
  for which SELCTG is true.

ALPHA
  ALPHA is COMPLEX*16 array, dimension (N)

BETA
  BETA is COMPLEX*16 array, dimension (N)
  On exit,  ALPHA(j)/BETA(j), j=1,...,N, will be the
  generalized eigenvalues.  ALPHA(j), j=1,...,N  and  BETA(j),
  j=1,...,N  are the diagonals of the complex Schur form (A,B)
  output by ZGGES3. The  BETA(j) will be non-negative real.

  Note: the quotients ALPHA(j)/BETA(j) may easily over- or
  underflow, and BETA(j) may even be zero.  Thus, the user
  should avoid naively computing the ratio alpha/beta.
  However, ALPHA will be always less than and usually
  comparable with norm(A) in magnitude, and BETA always less
  than and usually comparable with norm(B).

VSL
  VSL is COMPLEX*16 array, dimension (LDVSL,N)
  If JOBVSL = 'V', VSL will contain the left Schur vectors.
  Not referenced if JOBVSL = 'N'.

LDVSL
  LDVSL is INTEGER
  The leading dimension of the matrix VSL. LDVSL >= 1, and
  if JOBVSL = 'V', LDVSL >= N.
```

```
VSR
  VSR is COMPLEX*16 array, dimension (LDVSR,N)
  If JOBVSR = 'V', VSR will contain the right Schur vectors.
  Not referenced if JOBVSR = 'N'.

LDVSR
  LDVSR is INTEGER
  The leading dimension of the matrix VSR. LDVSR >= 1, and
  if JOBVSR = 'V', LDVSR >= N.

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (8*N)

BWORK
  BWORK is LOGICAL array, dimension (N)
  Not referenced if SORT = 'N'.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  =1,...,N:
        The QZ iteration failed.  (A,B) are not in Schur
        form, but ALPHA(j) and BETA(j) should be correct for
        j=INFO+1,...,N.
  > N:  =N+1: other than QZ iteration failed in ZHGEQZ
        =N+2: after reordering, roundoff changed values of
              some complex eigenvalues so that leading
              eigenvalues in the Generalized Schur form no
              longer satisfy SELCTG=.TRUE.  This could also
              be caused due to scaling.
        =N+3: reordering failed in ZTGSEN.
```

———-

27.8.3 fortran code

— zgges3.f —

```
*  =====================================================================
      SUBROUTINE ZGGES3( JOBVSL, JOBVSR, SORT, SELCTG, N, A, LDA, B,
     $                   LDB, SDIM, ALPHA, BETA, VSL, LDVSL, VSR, LDVSR,
     $                   WORK, LWORK, RWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     January 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVSL, JOBVSR, SORT
      INTEGER            INFO, LDA, LDB, LDVSL, LDVSR, LWORK, N, SDIM
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      DOUBLE PRECISION   RWORK( * )
      COMPLEX*16         A( LDA, * ), ALPHA( * ), B( LDB, * ),
     $                   BETA( * ), VSL( LDVSL, * ), VSR( LDVSR, * ),
     $                   WORK( * )
*     ..
*     .. Function Arguments ..
      LOGICAL            SELCTG
      EXTERNAL           SELCTG
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
      COMPLEX*16         CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0D0, 0.0D0 ),
     $                   CONE = ( 1.0D0, 0.0D0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            CURSL, ILASCL, ILBSCL, ILVSL, ILVSR, LASTSL,
     $                   LQUERY, WANTST
      INTEGER            I, ICOLS, IERR, IHI, IJOBVL, IJOBVR, ILEFT,
     $                   ILO, IRIGHT, IROWS, IRWRK, ITAU, IWRK, LWKOPT
      DOUBLE PRECISION   ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS, PVSL,
     $                   PVSR, SMLNUM
*     ..
*     .. Local Arrays ..
      INTEGER            IDUM( 1 )
```

```
      DOUBLE PRECISION   DIF( 2 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, XERBLA, ZGEQRF, ZGGBAK, ZGGBAL, ZGGHD3,
     $                   ZHGEQZ, ZLACPY, ZLASCL, ZLASET, ZTGSEN, ZUNGQR,
     $                   ZUNMQR
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, ZLANGE
      EXTERNAL           LSAME, DLAMCH, ZLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVSL, 'N' ) ) THEN
         IJOBVL = 1
         ILVSL = .FALSE.
      ELSE IF( LSAME( JOBVSL, 'V' ) ) THEN
         IJOBVL = 2
         ILVSL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVSL = .FALSE.
      END IF
*
      IF( LSAME( JOBVSR, 'N' ) ) THEN
         IJOBVR = 1
         ILVSR = .FALSE.
      ELSE IF( LSAME( JOBVSR, 'V' ) ) THEN
         IJOBVR = 2
         ILVSR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVSR = .FALSE.
      END IF
*
      WANTST = LSAME( SORT, 'S' )
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
```

```
         INFO = -2
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDVSL.LT.1 .OR. ( ILVSL .AND. LDVSL.LT.N ) ) THEN
         INFO = -14
      ELSE IF( LDVSR.LT.1 .OR. ( ILVSR .AND. LDVSR.LT.N ) ) THEN
         INFO = -16
      ELSE IF( LWORK.LT.MAX( 1, 2*N ) .AND. .NOT.LQUERY ) THEN
         INFO = -18
      END IF
*
*     Compute workspace
*
      IF( INFO.EQ.0 ) THEN
         CALL ZGEQRF( N, N, B, LDB, WORK, WORK, -1, IERR )
         LWKOPT = MAX( 1,  N + INT ( WORK( 1 ) ) )
         CALL ZUNMQR( 'L', 'C', N, N, N, B, LDB, WORK, A, LDA, WORK,
     $                -1, IERR )
         LWKOPT = MAX( LWKOPT, N + INT ( WORK( 1 ) ) )
         IF( ILVSL ) THEN
            CALL ZUNGQR( N, N, N, VSL, LDVSL, WORK, WORK, -1, IERR )
            LWKOPT = MAX( LWKOPT, N + INT ( WORK( 1 ) ) )
         END IF
         CALL ZGGHD3( JOBVSL, JOBVSR, N, 1, N, A, LDA, B, LDB, VSL,
     $                LDVSL, VSR, LDVSR, WORK, -1, IERR )
         LWKOPT = MAX( LWKOPT, N + INT ( WORK( 1 ) ) )
         CALL ZHGEQZ( 'S', JOBVSL, JOBVSR, N, 1, N, A, LDA, B, LDB,
     $                ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, WORK,
     $        -1, RWORK, IERR )
         LWKOPT = MAX( LWKOPT, INT ( WORK( 1 ) ) )
         IF( WANTST ) THEN
            CALL ZTGSEN( 0, ILVSL, ILVSR, BWORK, N, A, LDA, B, LDB,
     $                   ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, SDIM,
     $                   PVSL, PVSR, DIF, WORK, -1, IDUM, 1, IERR )
            LWKOPT = MAX( LWKOPT, INT ( WORK( 1 ) ) )
         END IF
         WORK( 1 ) = DCMPLX( LWKOPT )
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGGES3 ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
```

```
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = ZLANGE( 'M', N, N, A, LDA, RWORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
*
      IF( ILASCL )
     $   CALL ZLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = ZLANGE( 'M', N, N, B, LDB, RWORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
*
      IF( ILBSCL )
     $   CALL ZLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrix to make it more nearly triangular
*
```

```
      ILEFT = 1
      IRIGHT = N + 1
      IRWRK = IRIGHT + N
      CALL ZGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, RWORK( ILEFT ),
     $             RWORK( IRIGHT ), RWORK( IRWRK ), IERR )
*
*     Reduce B to triangular form (QR decomposition of B)
*
      IROWS = IHI + 1 - ILO
      ICOLS = N + 1 - ILO
      ITAU = 1
      IWRK = ITAU + IROWS
      CALL ZGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to matrix A
*
      CALL ZUNMQR( 'L', 'C', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VSL
*
      IF( ILVSL ) THEN
         CALL ZLASET( 'Full', N, N, CZERO, CONE, VSL, LDVSL )
         IF( IROWS.GT.1 ) THEN
            CALL ZLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VSL( ILO+1, ILO ), LDVSL )
         END IF
         CALL ZUNGQR( IROWS, IROWS, IROWS, VSL( ILO, ILO ), LDVSL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Initialize VSR
*
      IF( ILVSR )
     $   CALL ZLASET( 'Full', N, N, CZERO, CONE, VSR, LDVSR )
*
*     Reduce to generalized Hessenberg form
*
      CALL ZGGHD3( JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB, VSL,
     $             LDVSL, VSR, LDVSR, WORK( IWRK ), LWORK+1-IWRK, IERR )
*
      SDIM = 0
*
*     Perform QZ algorithm, computing Schur vectors if desired
*
      IWRK = ITAU
      CALL ZHGEQZ( 'S', JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, WORK( IWRK ),
```

```
     $             LWORK+1-IWRK, RWORK( IRWRK ), IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 30
      END IF
*
*     Sort eigenvalues ALPHA/BETA if desired
*
      IF( WANTST ) THEN
*
*        Undo scaling on eigenvalues before selecting
*
         IF( ILASCL )
     $      CALL ZLASCL( 'G', 0, 0, ANRM, ANRMTO, N, 1, ALPHA, N, IERR )
         IF( ILBSCL )
     $      CALL ZLASCL( 'G', 0, 0, BNRM, BNRMTO, N, 1, BETA, N, IERR )
*
*        Select eigenvalues
*
         DO 10 I = 1, N
            BWORK( I ) = SELCTG( ALPHA( I ), BETA( I ) )
   10    CONTINUE
*
         CALL ZTGSEN( 0, ILVSL, ILVSR, BWORK, N, A, LDA, B, LDB, ALPHA,
     $                BETA, VSL, LDVSL, VSR, LDVSR, SDIM, PVSL, PVSR,
     $                DIF, WORK( IWRK ), LWORK-IWRK+1, IDUM, 1, IERR )
         IF( IERR.EQ.1 )
     $      INFO = N + 3
*
      END IF
*
*     Apply back-permutation to VSL and VSR
*
      IF( ILVSL )
     $   CALL ZGGBAK( 'P', 'L', N, ILO, IHI, RWORK( ILEFT ),
     $                RWORK( IRIGHT ), N, VSL, LDVSL, IERR )
      IF( ILVSR )
     $   CALL ZGGBAK( 'P', 'R', N, ILO, IHI, RWORK( ILEFT ),
     $                RWORK( IRIGHT ), N, VSR, LDVSR, IERR )
*
*     Undo scaling
*
      IF( ILASCL ) THEN
         CALL ZLASCL( 'U', 0, 0, ANRMTO, ANRM, N, N, A, LDA, IERR )
```

```
         CALL ZLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHA, N, IERR )
      END IF
*
      IF( ILBSCL ) THEN
         CALL ZLASCL( 'U', 0, 0, BNRMTO, BNRM, N, N, B, LDB, IERR )
         CALL ZLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
      END IF
*
      IF( WANTST ) THEN
*
*        Check if reordering is correct
*
         LASTSL = .TRUE.
         SDIM = 0
         DO 20 I = 1, N
            CURSL = SELCTG( ALPHA( I ), BETA( I ) )
            IF( CURSL )
     $         SDIM = SDIM + 1
            IF( CURSL .AND. .NOT.LASTSL )
     $         INFO = N + 2
            LASTSL = CURSL
   20    CONTINUE
*
      END IF
*
   30 CONTINUE
*
      WORK( 1 ) = DCMPLX( LWKOPT )
*
      RETURN
*
*     End of ZGGES3
*
      END
```

———-

27.8.4 lisp code

— **zgges3.lisp** —

———-

27.9 zggesx

27.9.1 Axiom unit tests

— **zggesx.input** —

```
)set break resume
)sys rm -f zggesx.output
)spool zggesx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

27.9.2 Axiom help page

— **zggesx.help** —

```
======================================================================
zggesx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zggesx (JOBVSL, JOBVSR, SORT, SELCTG, SENSE, N, A, LDA, B,
           LDB, SDIM, ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, RCONDE, RCONDV,
           WORK, LWORK, RWORK, IWORK, LIWORK, BWORK, INFO)
            ZGGESX computes the eigenvalues, the Schur form, and, optionally,
           the matrix of Schur vectors for GE matrices

Function/Subroutine Documentation
   subroutine zggesx (character JOBVSL, character JOBVSR, character SORT,
       external SELCTG, character SENSE, integer N, complex*16, dimension(
       lda, * ) A, integer LDA, complex*16, dimension( ldb, * ) B, integer
       LDB, integer SDIM, complex*16, dimension( * ) ALPHA, complex*16,
       dimension( * ) BETA, complex*16, dimension( ldvsl, * ) VSL, integer
       LDVSL, complex*16, dimension( ldvsr, * ) VSR, integer LDVSR, double
       precision, dimension( 2 ) RCONDE, double precision, dimension( 2 )
```

```
      RCONDV, complex*16, dimension( * ) WORK, integer LWORK, double
      precision, dimension( * ) RWORK, integer, dimension( * ) IWORK, integer
      LIWORK, logical, dimension( * ) BWORK, integer INFO)
       ZGGESX computes the eigenvalues, the Schur form, and, optionally, the
      matrix of Schur vectors for GE matrices

Purpose:
   ZGGESX computes for a pair of N-by-N complex nonsymmetric matrices
   (A,B), the generalized eigenvalues, the complex Schur form (S,T),
   and, optionally, the left and/or right matrices of Schur vectors (VSL
   and VSR).  This gives the generalized Schur factorization

   (A,B) = ( (VSL) S (VSR)**H, (VSL) T (VSR)**H )

   where (VSR)**H is the conjugate-transpose of VSR.

   Optionally, it also orders the eigenvalues so that a selected cluster
   of eigenvalues appears in the leading diagonal blocks of the upper
   triangular matrix S and the upper triangular matrix T; computes
   a reciprocal condition number for the average of the selected
   eigenvalues (RCONDE); and computes a reciprocal condition number for
   the right and left deflating subspaces corresponding to the selected
   eigenvalues (RCONDV). The leading columns of VSL and VSR then form
   an orthonormal basis for the corresponding left and right eigenspaces
   (deflating subspaces).

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar w
   or a ratio alpha/beta = w, such that  A - w*B is singular.  It is
   usually represented as the pair (alpha,beta), as there is a
   reasonable interpretation for beta=0 or for both being zero.

   A pair of matrices (S,T) is in generalized complex Schur form if T is
   upper triangular with non-negative diagonal and S is upper
   triangular.

Parameters:

 JOBVSL
   JOBVSL is CHARACTER*1
   = 'N':  do not compute the left Schur vectors;
   = 'V':  compute the left Schur vectors.

 JOBVSR
   JOBVSR is CHARACTER*1
   = 'N':  do not compute the right Schur vectors;
   = 'V':  compute the right Schur vectors.

 SORT
   SORT is CHARACTER*1
   Specifies whether or not to order the eigenvalues on the
```

```
  diagonal of the generalized Schur form.
  = 'N':  Eigenvalues are not ordered;
  = 'S':  Eigenvalues are ordered (see SELCTG).

SELCTG
  SELCTG is procedure) LOGICAL FUNCTION of two COMPLEX*16 arguments
  SELCTG must be declared EXTERNAL in the calling subroutine.
  If SORT = 'N', SELCTG is not referenced.
  If SORT = 'S', SELCTG is used to select eigenvalues to sort
  to the top left of the Schur form.
  Note that a selected complex eigenvalue may no longer satisfy
  SELCTG(ALPHA(j),BETA(j)) = .TRUE. after ordering, since
  ordering may change the value of complex eigenvalues
  (especially if the eigenvalue is ill-conditioned), in this
  case INFO is set to N+3 see INFO below).

SENSE
  SENSE is CHARACTER*1
  Determines which reciprocal condition numbers are computed.
  = 'N' : None are computed;
  = 'E' : Computed for average of selected eigenvalues only;
  = 'V' : Computed for selected deflating subspaces only;
  = 'B' : Computed for both.
  If SENSE = 'E', 'V', or 'B', SORT must equal 'S'.

N
  N is INTEGER
  The order of the matrices A, B, VSL, and VSR.  N >= 0.

A
  A is COMPLEX*16 array, dimension (LDA, N)
  On entry, the first of the pair of matrices.
  On exit, A has been overwritten by its generalized Schur form S.

LDA
  LDA is INTEGER
  The leading dimension of A.  LDA >= max(1,N).

B
  B is COMPLEX*16 array, dimension (LDB, N)
  On entry, the second of the pair of matrices.
  On exit, B has been overwritten by its generalized Schur form T.

LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).

SDIM
  SDIM is INTEGER
  If SORT = 'N', SDIM = 0.
```

```
  If SORT = 'S', SDIM = number of eigenvalues (after sorting)
  for which SELCTG is true.

ALPHA
  ALPHA is COMPLEX*16 array, dimension (N)

BETA
  BETA is COMPLEX*16 array, dimension (N)
  On exit, ALPHA(j)/BETA(j), j=1,...,N, will be the
  generalized eigenvalues.  ALPHA(j) and BETA(j),j=1,...,N  are
  the diagonals of the complex Schur form (S,T).  BETA(j) will
  be non-negative real.

  Note: the quotients ALPHA(j)/BETA(j) may easily over- or
  underflow, and BETA(j) may even be zero.  Thus, the user
  should avoid naively computing the ratio alpha/beta.
  However, ALPHA will be always less than and usually
  comparable with norm(A) in magnitude, and BETA always less
  than and usually comparable with norm(B).

VSL
  VSL is COMPLEX*16 array, dimension (LDVSL,N)
  If JOBVSL = 'V', VSL will contain the left Schur vectors.
  Not referenced if JOBVSL = 'N'.

LDVSL
  LDVSL is INTEGER
  The leading dimension of the matrix VSL. LDVSL >=1, and
  if JOBVSL = 'V', LDVSL >= N.

VSR
  VSR is COMPLEX*16 array, dimension (LDVSR,N)
  If JOBVSR = 'V', VSR will contain the right Schur vectors.
  Not referenced if JOBVSR = 'N'.

LDVSR
  LDVSR is INTEGER
  The leading dimension of the matrix VSR. LDVSR >= 1, and
  if JOBVSR = 'V', LDVSR >= N.

RCONDE
  RCONDE is DOUBLE PRECISION array, dimension ( 2 )
  If SENSE = 'E' or 'B', RCONDE(1) and RCONDE(2) contain the
  reciprocal condition numbers for the average of the selected eigenvalues.
  Not referenced if SENSE = 'N' or 'V'.

RCONDV
  RCONDV is DOUBLE PRECISION array, dimension ( 2 )
  If SENSE = 'V' or 'B', RCONDV(1) and RCONDV(2) contain the
  reciprocal condition number for the selected deflating subspaces.
```

```
  Not referenced if SENSE = 'N' or 'E'.

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  If N = 0, LWORK >= 1, else if SENSE = 'E', 'V', or 'B',
  LWORK >= MAX(1,2*N,2*SDIM*(N-SDIM)), else
  LWORK >= MAX(1,2*N).  Note that 2*SDIM*(N-SDIM) <= N*N/2.
  Note also that an error is only returned if
  LWORK < MAX(1,2*N), but if SENSE = 'E' or 'V' or 'B' this may
  not be large enough.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the bound on the optimal size of the WORK
  array and the minimum size of the IWORK array, returns these
  values as the first entries of the WORK and IWORK arrays, and
  no error message related to LWORK or LIWORK is issued by XERBLA.

RWORK
  RWORK is DOUBLE PRECISION array, dimension ( 8*N )
  Real workspace.

IWORK
  IWORK is INTEGER array, dimension (MAX(1,LIWORK))
  On exit, if INFO = 0, IWORK(1) returns the minimum LIWORK.

LIWORK
  LIWORK is INTEGER
  The dimension of the array IWORK.
  If SENSE = 'N' or N = 0, LIWORK >= 1, otherwise
  LIWORK >= N+2.

  If LIWORK = -1, then a workspace query is assumed; the
  routine only calculates the bound on the optimal size of the
  WORK array and the minimum size of the IWORK array, returns
  these values as the first entries of the WORK and IWORK
  arrays, and no error message related to LWORK or LIWORK is
  issued by XERBLA.

BWORK
  BWORK is LOGICAL array, dimension (N)
  Not referenced if SORT = 'N'.

INFO
  INFO is INTEGER
  = 0:  successful exit
```

```
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  = 1,...,N:
        The QZ iteration failed.  (A,B) are not in Schur
        form, but ALPHA(j) and BETA(j) should be correct for
        j=INFO+1,...,N.
   > N:  =N+1: other than QZ iteration failed in ZHGEQZ
         =N+2: after reordering, roundoff changed values of
               some complex eigenvalues so that leading
               eigenvalues in the Generalized Schur form no
               longer satisfy SELCTG=.TRUE.  This could also
               be caused due to scaling.
         =N+3: reordering failed in ZTGSEN.
```

———-

27.9.3 fortran code

— **zggesx.f** —

```
*  =====================================================================
      SUBROUTINE ZGGESX( JOBVSL, JOBVSR, SORT, SELCTG, SENSE, N, A, LDA,
     $                   B, LDB, SDIM, ALPHA, BETA, VSL, LDVSL, VSR,
     $                   LDVSR, RCONDE, RCONDV, WORK, LWORK, RWORK,
     $                   IWORK, LIWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVSL, JOBVSR, SENSE, SORT
      INTEGER            INFO, LDA, LDB, LDVSL, LDVSR, LIWORK, LWORK, N,
     $                   SDIM
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      INTEGER            IWORK( * )
      DOUBLE PRECISION   RCONDE( 2 ), RCONDV( 2 ), RWORK( * )
      COMPLEX*16         A( LDA, * ), ALPHA( * ), B( LDB, * ),
     $                   BETA( * ), VSL( LDVSL, * ), VSR( LDVSR, * ),
     $                   WORK( * )
*     ..
*     .. Function Arguments ..
      LOGICAL            SELCTG
      EXTERNAL           SELCTG
*     ..
```

```
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      COMPLEX*16         CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0D+0, 0.0D+0 ),
     $                   CONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            CURSL, ILASCL, ILBSCL, ILVSL, ILVSR, LASTSL,
     $                   LQUERY, WANTSB, WANTSE, WANTSN, WANTST, WANTSV
      INTEGER            I, ICOLS, IERR, IHI, IJOB, IJOBVL, IJOBVR,
     $                   ILEFT, ILO, IRIGHT, IROWS, IRWRK, ITAU, IWRK,
     $                   LIWMIN, LWRK, MAXWRK, MINWRK
      DOUBLE PRECISION   ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS, PL,
     $                   PR, SMLNUM
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   DIF( 2 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, XERBLA, ZGEQRF, ZGGBAK, ZGGBAL, ZGGHRD,
     $                   ZHGEQZ, ZLACPY, ZLASCL, ZLASET, ZTGSEN, ZUNGQR,
     $                   ZUNMQR
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, ZLANGE
      EXTERNAL           LSAME, ILAENV, DLAMCH, ZLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVSL, 'N' ) ) THEN
         IJOBVL = 1
         ILVSL = .FALSE.
      ELSE IF( LSAME( JOBVSL, 'V' ) ) THEN
         IJOBVL = 2
         ILVSL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVSL = .FALSE.
      END IF
```

```
*
      IF( LSAME( JOBVSR, 'N' ) ) THEN
         IJOBVR = 1
         ILVSR = .FALSE.
      ELSE IF( LSAME( JOBVSR, 'V' ) ) THEN
         IJOBVR = 2
         ILVSR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVSR = .FALSE.
      END IF
*
      WANTST = LSAME( SORT, 'S' )
      WANTSN = LSAME( SENSE, 'N' )
      WANTSE = LSAME( SENSE, 'E' )
      WANTSV = LSAME( SENSE, 'V' )
      WANTSB = LSAME( SENSE, 'B' )
      LQUERY = ( LWORK.EQ.-1 .OR. LIWORK.EQ.-1 )
      IF( WANTSN ) THEN
         IJOB = 0
      ELSE IF( WANTSE ) THEN
         IJOB = 1
      ELSE IF( WANTSV ) THEN
         IJOB = 2
      ELSE IF( WANTSB ) THEN
         IJOB = 4
      END IF
*
*     Test the input arguments
*
      INFO = 0
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -2
      ELSE IF( ( .NOT.WANTST ) .AND. ( .NOT.LSAME( SORT, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( .NOT.( WANTSN .OR. WANTSE .OR. WANTSV .OR. WANTSB ) .OR.
     $         ( .NOT.WANTST .AND. .NOT.WANTSN ) ) THEN
         INFO = -5
      ELSE IF( N.LT.0 ) THEN
         INFO = -6
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -10
      ELSE IF( LDVSL.LT.1 .OR. ( ILVSL .AND. LDVSL.LT.N ) ) THEN
         INFO = -15
      ELSE IF( LDVSR.LT.1 .OR. ( ILVSR .AND. LDVSR.LT.N ) ) THEN
         INFO = -17
```

```
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.GT.0) THEN
            MINWRK = 2*N
            MAXWRK = N*(1 + ILAENV( 1, 'ZGEQRF', ' ', N, 1, N, 0 ) )
            MAXWRK = MAX( MAXWRK, N*( 1 +
     $                    ILAENV( 1, 'ZUNMQR', ' ', N, 1, N, -1 ) ) )
            IF( ILVSL ) THEN
               MAXWRK = MAX( MAXWRK, N*( 1 +
     $                       ILAENV( 1, 'ZUNGQR', ' ', N, 1, N, -1 ) ) )
            END IF
            LWRK = MAXWRK
            IF( IJOB.GE.1 )
     $         LWRK = MAX( LWRK, N*N/2 )
         ELSE
            MINWRK = 1
            MAXWRK = 1
            LWRK   = 1
         END IF
         WORK( 1 ) = LWRK
         IF( WANTSN .OR. N.EQ.0 ) THEN
            LIWMIN = 1
         ELSE
            LIWMIN = N + 2
         END IF
         IWORK( 1 ) = LIWMIN
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -21
         ELSE IF( LIWORK.LT.LIWMIN  .AND. .NOT.LQUERY) THEN
            INFO = -24
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGGESX', -INFO )
         RETURN
      ELSE IF (LQUERY) THEN
         RETURN
      END IF
*
*     Quick return if possible
```

```
*
      IF( N.EQ.0 ) THEN
         SDIM = 0
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = ZLANGE( 'M', N, N, A, LDA, RWORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
      IF( ILASCL )
     $   CALL ZLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = ZLANGE( 'M', N, N, B, LDB, RWORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
     $   CALL ZLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrix to make it more nearly triangular
*     (Real Workspace: need 6*N)
*
      ILEFT = 1
      IRIGHT = N + 1
      IRWRK = IRIGHT + N
      CALL ZGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, RWORK( ILEFT ),
```

```
     $             RWORK( IRIGHT ), RWORK( IRWRK ), IERR )
*
*     Reduce B to triangular form (QR decomposition of B)
*     (Complex Workspace: need N, prefer N*NB)
*
      IROWS = IHI + 1 - ILO
      ICOLS = N + 1 - ILO
      ITAU = 1
      IWRK = ITAU + IROWS
      CALL ZGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the unitary transformation to matrix A
*     (Complex Workspace: need N, prefer N*NB)
*
      CALL ZUNMQR( 'L', 'C', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VSL
*     (Complex Workspace: need N, prefer N*NB)
*
      IF( ILVSL ) THEN
         CALL ZLASET( 'Full', N, N, CZERO, CONE, VSL, LDVSL )
         IF( IROWS.GT.1 ) THEN
            CALL ZLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VSL( ILO+1, ILO ), LDVSL )
         END IF
         CALL ZUNGQR( IROWS, IROWS, IROWS, VSL( ILO, ILO ), LDVSL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Initialize VSR
*
      IF( ILVSR )
     $   CALL ZLASET( 'Full', N, N, CZERO, CONE, VSR, LDVSR )
*
*     Reduce to generalized Hessenberg form
*     (Workspace: none needed)
*
      CALL ZGGHRD( JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB, VSL,
     $             LDVSL, VSR, LDVSR, IERR )
*
      SDIM = 0
*
*     Perform QZ algorithm, computing Schur vectors if desired
*     (Complex Workspace: need N)
*     (Real Workspace:    need N)
*
      IWRK = ITAU
```

```
      CALL ZHGEQZ( 'S', JOBVSL, JOBVSR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, WORK( IWRK ),
     $             LWORK+1-IWRK, RWORK( IRWRK ), IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 40
      END IF
*
*     Sort eigenvalues ALPHA/BETA and compute the reciprocal of
*     condition number(s)
*
      IF( WANTST ) THEN
*
*        Undo scaling on eigenvalues before SELCTGing
*
         IF( ILASCL )
     $      CALL ZLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHA, N, IERR )
         IF( ILBSCL )
     $      CALL ZLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
*
*        Select eigenvalues
*
         DO 10 I = 1, N
            BWORK( I ) = SELCTG( ALPHA( I ), BETA( I ) )
   10    CONTINUE
*
*        Reorder eigenvalues, transform Generalized Schur vectors, and
*        compute reciprocal condition numbers
*        (Complex Workspace: If IJOB >= 1, need MAX(1, 2*SDIM*(N-SDIM))
*                            otherwise, need 1 )
*
         CALL ZTGSEN( IJOB, ILVSL, ILVSR, BWORK, N, A, LDA, B, LDB,
     $                ALPHA, BETA, VSL, LDVSL, VSR, LDVSR, SDIM, PL, PR,
     $                DIF, WORK( IWRK ), LWORK-IWRK+1, IWORK, LIWORK,
     $                IERR )
*
         IF( IJOB.GE.1 )
     $      MAXWRK = MAX( MAXWRK, 2*SDIM*( N-SDIM ) )
         IF( IERR.EQ.-21 ) THEN
*
*            not enough complex workspace
*
            INFO = -21
         ELSE
```

```
            IF( IJOB.EQ.1 .OR. IJOB.EQ.4 ) THEN
               RCONDE( 1 ) = PL
               RCONDE( 2 ) = PR
            END IF
            IF( IJOB.EQ.2 .OR. IJOB.EQ.4 ) THEN
               RCONDV( 1 ) = DIF( 1 )
               RCONDV( 2 ) = DIF( 2 )
            END IF
            IF( IERR.EQ.1 )
     $         INFO = N + 3
         END IF
*
      END IF
*
*     Apply permutation to VSL and VSR
*     (Workspace: none needed)
*
      IF( ILVSL )
     $   CALL ZGGBAK( 'P', 'L', N, ILO, IHI, RWORK( ILEFT ),
     $                RWORK( IRIGHT ), N, VSL, LDVSL, IERR )
*
      IF( ILVSR )
     $   CALL ZGGBAK( 'P', 'R', N, ILO, IHI, RWORK( ILEFT ),
     $                RWORK( IRIGHT ), N, VSR, LDVSR, IERR )
*
*     Undo scaling
*
      IF( ILASCL ) THEN
         CALL ZLASCL( 'U', 0, 0, ANRMTO, ANRM, N, N, A, LDA, IERR )
         CALL ZLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHA, N, IERR )
      END IF
*
      IF( ILBSCL ) THEN
         CALL ZLASCL( 'U', 0, 0, BNRMTO, BNRM, N, N, B, LDB, IERR )
         CALL ZLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
      END IF
*
      IF( WANTST ) THEN
*
*        Check if reordering is correct
*
         LASTSL = .TRUE.
         SDIM = 0
         DO 30 I = 1, N
            CURSL = SELCTG( ALPHA( I ), BETA( I ) )
            IF( CURSL )
     $         SDIM = SDIM + 1
            IF( CURSL .AND. .NOT.LASTSL )
     $         INFO = N + 2
            LASTSL = CURSL
```

```
   30    CONTINUE
*
      END IF
*
   40 CONTINUE
*
      WORK( 1 ) = MAXWRK
      IWORK( 1 ) = LIWMIN
*
      RETURN
*
*     End of ZGGESX
*
      END
```

———-

27.9.4 lisp code

— **zggesx.lisp** —

———-

27.10 zggev

27.10.1 Axiom unit tests

— **zggev.input** —

```
)set break resume
)sys rm -f zggev.output
)spool zggev.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

27.10.2 Axiom help page

— zggev.help —

```
=====================================================================
zggev examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zggev (JOBVL, JOBVR, N, A, LDA, B, LDB, ALPHA, BETA, VL,
           LDVL, VR, LDVR, WORK, LWORK, RWORK, INFO)
            ZGGEV computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for GE matrices

Function/Subroutine Documentation
   subroutine zggev (character JOBVL, character JOBVR, integer N, complex*16,
       dimension( lda, * ) A, integer LDA, complex*16, dimension( ldb, * ) B,
       integer LDB, complex*16, dimension( * ) ALPHA, complex*16, dimension( *
       ) BETA, complex*16, dimension( ldvl, * ) VL, integer LDVL, complex*16,
       dimension( ldvr, * ) VR, integer LDVR, complex*16, dimension( * ) WORK,
       integer LWORK, double precision, dimension( * ) RWORK, integer INFO)
        ZGGEV computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for GE matrices

Purpose:
   ZGGEV computes for a pair of N-by-N complex nonsymmetric matrices
   (A,B), the generalized eigenvalues, and optionally, the left and/or
   right generalized eigenvectors.

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar
   lambda or a ratio alpha/beta = lambda, such that A - lambda*B is
   singular. It is usually represented as the pair (alpha,beta), as
   there is a reasonable interpretation for beta=0, and even for both
   being zero.

   The right generalized eigenvector v(j) corresponding to the
   generalized eigenvalue lambda(j) of (A,B) satisfies

   A * v(j) = lambda(j) * B * v(j).

   The left generalized eigenvector u(j) corresponding to the
   generalized eigenvalues lambda(j) of (A,B) satisfies

   u(j)**H * A = lambda(j) * u(j)**H * B
```

```
   where u(j)**H is the conjugate-transpose of u(j).

Parameters:

 JOBVL
   JOBVL is CHARACTER*1
   = 'N':  do not compute the left generalized eigenvectors;
   = 'V':  compute the left generalized eigenvectors.

 JOBVR
   JOBVR is CHARACTER*1
   = 'N':  do not compute the right generalized eigenvectors;
   = 'V':  compute the right generalized eigenvectors.

 N
   N is INTEGER
   The order of the matrices A, B, VL, and VR.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA, N)
   On entry, the matrix A in the pair (A,B).
   On exit, A has been overwritten.

 LDA
   LDA is INTEGER
   The leading dimension of A.  LDA >= max(1,N).

 B
   B is COMPLEX*16 array, dimension (LDB, N)
   On entry, the matrix B in the pair (A,B).
   On exit, B has been overwritten.

 LDB
   LDB is INTEGER
   The leading dimension of B.  LDB >= max(1,N).

 ALPHA
   ALPHA is COMPLEX*16 array, dimension (N)

 BETA
   BETA is COMPLEX*16 array, dimension (N)
   On exit, ALPHA(j)/BETA(j), j=1,...,N, will be the
   generalized eigenvalues.

   Note: the quotients ALPHA(j)/BETA(j) may easily over- or
   underflow, and BETA(j) may even be zero.  Thus, the user
   should avoid naively computing the ratio alpha/beta.
   However, ALPHA will be always less than and usually
   comparable with norm(A) in magnitude, and BETA always less
```

```
  than and usually comparable with norm(B).

VL
  VL is COMPLEX*16 array, dimension (LDVL,N)
  If JOBVL = 'V', the left generalized eigenvectors u(j) are
  stored one after another in the columns of VL, in the same
  order as their eigenvalues.
  Each eigenvector is scaled so the largest component has
  abs(real part) + abs(imag. part) = 1.
  Not referenced if JOBVL = 'N'.

LDVL
  LDVL is INTEGER
  The leading dimension of the matrix VL. LDVL >= 1, and
  if JOBVL = 'V', LDVL >= N.

VR
  VR is COMPLEX*16 array, dimension (LDVR,N)
  If JOBVR = 'V', the right generalized eigenvectors v(j) are
  stored one after another in the columns of VR, in the same
  order as their eigenvalues.
  Each eigenvector is scaled so the largest component has
  abs(real part) + abs(imag. part) = 1.
  Not referenced if JOBVR = 'N'.

LDVR
  LDVR is INTEGER
  The leading dimension of the matrix VR. LDVR >= 1, and
  if JOBVR = 'V', LDVR >= N.

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,2*N).
  For good performance, LWORK must generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (8*N)

INFO
  INFO is INTEGER
  = 0:  successful exit
```

```
< 0:  if INFO = -i, the i-th argument had an illegal value.
=1,...,N:
      The QZ iteration failed.  No eigenvectors have been
      calculated, but ALPHA(j) and BETA(j) should be
      correct for j=INFO+1,...,N.
> N:  =N+1: other then QZ iteration failed in DHGEQZ,
      =N+2: error return from DTGEVC.
```

27.10.3 fortran code

— zggev.f —

```
*  =====================================================================
      SUBROUTINE ZGGEV( JOBVL, JOBVR, N, A, LDA, B, LDB, ALPHA, BETA,
     $                  VL, LDVL, VR, LDVR, WORK, LWORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVL, JOBVR
      INTEGER            INFO, LDA, LDB, LDVL, LDVR, LWORK, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   RWORK( * )
      COMPLEX*16         A( LDA, * ), ALPHA( * ), B( LDB, * ),
     $                   BETA( * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
      COMPLEX*16         CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0D0, 0.0D0 ),
     $                   CONE = ( 1.0D0, 0.0D0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            ILASCL, ILBSCL, ILV, ILVL, ILVR, LQUERY
      CHARACTER          CHTEMP
      INTEGER            ICOLS, IERR, IHI, IJOBVL, IJOBVR, ILEFT, ILO,
```

```
     $                   IN, IRIGHT, IROWS, IRWRK, ITAU, IWRK, JC, JR,
     $                   LWKMIN, LWKOPT
      DOUBLE PRECISION   ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS,
     $                   SMLNUM, TEMP
      COMPLEX*16         X
*     ..
*     .. Local Arrays ..
      LOGICAL            LDUMMA( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, XERBLA, ZGEQRF, ZGGBAK, ZGGBAL, ZGGHRD,
     $                   ZHGEQZ, ZLACPY, ZLASCL, ZLASET, ZTGEVC, ZUNGQR,
     $                   ZUNMQR
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, ZLANGE
      EXTERNAL           LSAME, ILAENV, DLAMCH, ZLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, MAX, SQRT
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   ABS1
*     ..
*     .. Statement Function definitions ..
      ABS1( X ) = ABS( DBLE( X ) ) + ABS( DIMAG( X ) )
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVL, 'N' ) ) THEN
         IJOBVL = 1
         ILVL = .FALSE.
      ELSE IF( LSAME( JOBVL, 'V' ) ) THEN
         IJOBVL = 2
         ILVL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVL = .FALSE.
      END IF
*
      IF( LSAME( JOBVR, 'N' ) ) THEN
         IJOBVR = 1
         ILVR = .FALSE.
      ELSE IF( LSAME( JOBVR, 'V' ) ) THEN
         IJOBVR = 2
         ILVR = .TRUE.
```

```
      ELSE
         IJOBVR = -1
         ILVR = .FALSE.
      END IF
      ILV = ILVL .OR. ILVR
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDVL.LT.1 .OR. ( ILVL .AND. LDVL.LT.N ) ) THEN
         INFO = -11
      ELSE IF( LDVR.LT.1 .OR. ( ILVR .AND. LDVR.LT.N ) ) THEN
         INFO = -13
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV. The workspace is
*       computed assuming ILO = 1 and IHI = N, the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         LWKMIN = MAX( 1, 2*N )
         LWKOPT = MAX( 1, N + N*ILAENV( 1, 'ZGEQRF', ' ', N, 1, N, 0 ) )
         LWKOPT = MAX( LWKOPT, N +
     $                 N*ILAENV( 1, 'ZUNMQR', ' ', N, 1, N, 0 ) )
         IF( ILVL ) THEN
            LWKOPT = MAX( LWKOPT, N +
     $                    N*ILAENV( 1, 'ZUNGQR', ' ', N, 1, N, -1 ) )
         END IF
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.LWKMIN .AND. .NOT.LQUERY )
     $      INFO = -15
      END IF
*
      IF( INFO.NE.0 ) THEN
```

```
         CALL XERBLA( 'ZGGEV ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
*
      EPS = DLAMCH( 'E' )*DLAMCH( 'B' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = ZLANGE( 'M', N, N, A, LDA, RWORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
      IF( ILASCL )
     $   CALL ZLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = ZLANGE( 'M', N, N, B, LDB, RWORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
     $   CALL ZLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrices A, B to isolate eigenvalues if possible
*     (Real Workspace: need 6*N)
```

```
*
      ILEFT = 1
      IRIGHT = N + 1
      IRWRK = IRIGHT + N
      CALL ZGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, RWORK( ILEFT ),
     $             RWORK( IRIGHT ), RWORK( IRWRK ), IERR )
*
*     Reduce B to triangular form (QR decomposition of B)
*     (Complex Workspace: need N, prefer N*NB)
*
      IROWS = IHI + 1 - ILO
      IF( ILV ) THEN
         ICOLS = N + 1 - ILO
      ELSE
         ICOLS = IROWS
      END IF
      ITAU = 1
      IWRK = ITAU + IROWS
      CALL ZGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to matrix A
*     (Complex Workspace: need N, prefer N*NB)
*
      CALL ZUNMQR( 'L', 'C', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VL
*     (Complex Workspace: need N, prefer N*NB)
*
      IF( ILVL ) THEN
         CALL ZLASET( 'Full', N, N, CZERO, CONE, VL, LDVL )
         IF( IROWS.GT.1 ) THEN
            CALL ZLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VL( ILO+1, ILO ), LDVL )
         END IF
         CALL ZUNGQR( IROWS, IROWS, IROWS, VL( ILO, ILO ), LDVL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Initialize VR
*
      IF( ILVR )
     $   CALL ZLASET( 'Full', N, N, CZERO, CONE, VR, LDVR )
*
*     Reduce to generalized Hessenberg form
*
      IF( ILV ) THEN
*
```

```
*        Eigenvectors requested -- work on whole matrix.
*
         CALL ZGGHRD( JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB, VL,
     $                LDVL, VR, LDVR, IERR )
      ELSE
         CALL ZGGHRD( 'N', 'N', IROWS, 1, IROWS, A( ILO, ILO ), LDA,
     $                B( ILO, ILO ), LDB, VL, LDVL, VR, LDVR, IERR )
      END IF
*
*     Perform QZ algorithm (Compute eigenvalues, and optionally, the
*     Schur form and Schur vectors)
*     (Complex Workspace: need N)
*     (Real Workspace: need N)
*
      IWRK = ITAU
      IF( ILV ) THEN
         CHTEMP = 'S'
      ELSE
         CHTEMP = 'E'
      END IF
      CALL ZHGEQZ( CHTEMP, JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHA, BETA, VL, LDVL, VR, LDVR, WORK( IWRK ),
     $             LWORK+1-IWRK, RWORK( IRWRK ), IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 70
      END IF
*
*     Compute Eigenvectors
*     (Real Workspace: need 2*N)
*     (Complex Workspace: need 2*N)
*
      IF( ILV ) THEN
         IF( ILVL ) THEN
            IF( ILVR ) THEN
               CHTEMP = 'B'
            ELSE
               CHTEMP = 'L'
            END IF
         ELSE
            CHTEMP = 'R'
         END IF
*
         CALL ZTGEVC( CHTEMP, 'B', LDUMMA, N, A, LDA, B, LDB, VL, LDVL,
```

```
     $                VR, LDVR, N, IN, WORK( IWRK ), RWORK( IRWRK ),
     $                IERR )
         IF( IERR.NE.0 ) THEN
            INFO = N + 2
            GO TO 70
         END IF
*
*        Undo balancing on VL and VR and normalization
*        (Workspace: none needed)
*
         IF( ILVL ) THEN
            CALL ZGGBAK( 'P', 'L', N, ILO, IHI, RWORK( ILEFT ),
     $                   RWORK( IRIGHT ), N, VL, LDVL, IERR )
            DO 30 JC = 1, N
               TEMP = ZERO
               DO 10 JR = 1, N
                  TEMP = MAX( TEMP, ABS1( VL( JR, JC ) ) )
   10          CONTINUE
               IF( TEMP.LT.SMLNUM )
     $            GO TO 30
               TEMP = ONE / TEMP
               DO 20 JR = 1, N
                  VL( JR, JC ) = VL( JR, JC )*TEMP
   20          CONTINUE
   30       CONTINUE
         END IF
         IF( ILVR ) THEN
            CALL ZGGBAK( 'P', 'R', N, ILO, IHI, RWORK( ILEFT ),
     $                   RWORK( IRIGHT ), N, VR, LDVR, IERR )
            DO 60 JC = 1, N
               TEMP = ZERO
               DO 40 JR = 1, N
                  TEMP = MAX( TEMP, ABS1( VR( JR, JC ) ) )
   40          CONTINUE
               IF( TEMP.LT.SMLNUM )
     $            GO TO 60
               TEMP = ONE / TEMP
               DO 50 JR = 1, N
                  VR( JR, JC ) = VR( JR, JC )*TEMP
   50          CONTINUE
   60       CONTINUE
         END IF
      END IF
*
*     Undo scaling if necessary
*
   70 CONTINUE
*
      IF( ILASCL )
     $   CALL ZLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHA, N, IERR )
```

```
*
      IF( ILBSCL )
     $   CALL ZLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
*
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of ZGGEV
*
      END
```

———-

27.10.4 lisp code

— **zggev.lisp** —

```
(let* ((zero 0.0d0)
       (one 1.0d0)
       (czero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (cone (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (f2cl-lib:complex16) czero)
           (type (f2cl-lib:complex16) cone)
           (ignorable zero one czero cone))
  (defun zggev
         (jobvl jobvr n a lda b ldb$ alpha beta vl ldvl vr ldvr work lwork
          rwork info)
    (declare (type (array double-float (*)) rwork)
             (type (array f2cl-lib:complex16 (*)) work vr vl beta alpha b a)
             (type (f2cl-lib:integer4) info lwork ldvr ldvl ldb$ lda n)
             (type (string *) jobvr jobvl))
    (f2cl-lib:with-multi-array-data
        ((jobvl character jobvl-%data% jobvl-%offset%)
         (jobvr character jobvr-%data% jobvr-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (alpha f2cl-lib:complex16 alpha-%data% alpha-%offset%)
         (beta f2cl-lib:complex16 beta-%data% beta-%offset%)
         (vl f2cl-lib:complex16 vl-%data% vl-%offset%)
         (vr f2cl-lib:complex16 vr-%data% vr-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (labels ((abs1 (x)
                 (+ (abs (f2cl-lib:dble x)) (abs (f2cl-lib:dimag x)))))
        (declare
         (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
```

```
  abs1))
(prog ((ldumma (make-array 1 :element-type 't)) (x #C(0.0d0 0.0d0))
       (anrm 0.0d0) (anrmto 0.0d0) (bignum 0.0d0) (bnrm 0.0d0)
       (bnrmto 0.0d0) (eps 0.0d0) (smlnum 0.0d0) (temp 0.0d0) (icols 0)
       (ierr 0) (ihi 0) (ijobvl 0) (ijobvr 0) (ileft 0) (ilo 0) (in 0)
       (iright 0) (irows 0) (irwrk 0) (itau 0) (iwrk 0) (jc 0) (jr 0)
       (lwkmin 0) (lwkopt 0)
       (chtemp
        (make-array '(1) :element-type 'character :initial-element
                    #\ ))
       (ilascl nil) (ilbscl nil) (ilv nil) (ilvl nil) (ilvr nil)
       (lquery nil))
  (declare (type (array f2cl-lib:logical (1)) ldumma)
           (type (f2cl-lib:complex16) x)
           (type (double-float) anrm anrmto bignum bnrm bnrmto eps
            smlnum temp)
           (type (f2cl-lib:integer4) icols ierr ihi ijobvl ijobvr ileft
            ilo in iright irows irwrk itau iwrk jc jr lwkmin lwkopt)
           (type (string 1) chtemp)
           (type f2cl-lib:logical ilascl ilbscl ilv ilvl ilvr lquery))
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame jobvl "N")
      (declare (ignore var-1))
      (when var-0 (setf jobvl var-0))
      ret-val)
    (setf ijobvl 1) (setf ilvl f2cl-lib:%false%))
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame jobvl "V")
      (declare (ignore var-1))
      (when var-0 (setf jobvl var-0))
      ret-val)
    (setf ijobvl 2) (setf ilvl f2cl-lib:%true%))
   (t (setf ijobvl -1) (setf ilvl f2cl-lib:%false%)))
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame jobvr "N")
      (declare (ignore var-1))
      (when var-0 (setf jobvr var-0))
      ret-val)
    (setf ijobvr 1) (setf ilvr f2cl-lib:%false%))
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame jobvr "V")
      (declare (ignore var-1))
      (when var-0 (setf jobvr var-0))
      ret-val)
    (setf ijobvr 2) (setf ilvr f2cl-lib:%true%))
   (t (setf ijobvr -1) (setf ilvr f2cl-lib:%false%)))
  (setf ilv (or ilvl ilvr))
  (setf info 0)
```

```
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((<= ijobvl 0) (setf info -1)) ((<= ijobvr 0) (setf info -2))
      ((< n 0) (setf info -3))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -5))
      ((< ldb$
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -7))
      ((or (< ldvl 1) (and ilvl (< ldvl n))) (setf info -11))
      ((or (< ldvr 1) (and ilvr (< ldvr n))) (setf info -13)))
(cond
 ((= info 0)
  (setf lwkmin
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul 2 n))))
  (setf lwkopt
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add n
                                      (f2cl-lib:int-mul n
                                                        (multiple-value-bind
                                                            (ret-val
                                                             var-0
                                                             var-1
                                                             var-2
                                                             var-3
                                                             var-4
                                                             var-5
                                                             var-6)
                                                            (ilaenv 1
                                                             "ZGEQRF"
                                                             " " n 1
                                                             n 0)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-4
                                                            var-6))
                                                          (setf n
                                                                  var-3)
                                                          (setf n
                                                                  var-5)
                                                          ret-val))))))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkopt)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add n
```

```
                                                    (f2cl-lib:int-mul n
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv 1
                                                                           "ZUNMQR"
                                                                           " " n 1
                                                                           n 0)
                                                                        (declare
                                                                         (ignore
                                                                          var-0
                                                                          var-1
                                                                          var-2
                                                                          var-4
                                                                          var-6))
                                                                        (setf n
                                                                                var-3)
                                                                        (setf n
                                                                                var-5)
                                                                        ret-val))))))
                                                 (cond
                                                  (ilvl
                                                   (setf lwkopt
                                                           (max (the f2cl-lib:integer4 lwkopt)
                                                                (the f2cl-lib:integer4
                                                                     (f2cl-lib:int-add n
                                                                                       (f2cl-lib:int-mul n
                                                                                                         (multiple-value-bind
                                                                                                             (ret-val
                                                                                                              var-0
                                                                                                              var-1
                                                                                                              var-2
                                                                                                              var-3
                                                                                                              var-4
                                                                                                              var-5
                                                                                                              var-6)
                                                                                                             (ilaenv
                                                                                                              1
                                                                                                              "ZUNGQR"
                                                                                                              " " n
                                                                                                              1 n
                                                                                                              -1)
                                                                                                            (declare
                                                                                                             (ignore
```

```
                                                                var-0
                                                                var-1
                                                                var-2
                                                                var-4
                                                                var-6))
                                                              (setf n
                                                                      var-3)
                                                              (setf n
                                                                      var-5)
                                                              ret-val))))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwkopt 'f2cl-lib:complex16))
  (if (and (< lwork lwkmin) (not lquery))
      (setf info -15))))
(cond
 ((/= info 0) (xerbla "ZGGEV " (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(setf eps (* (dlamch "E") (dlamch "B")))
(setf smlnum (dlamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "M" n n a lda rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf ilascl f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf anrmto smlnum)
  (setf ilascl f2cl-lib:%true%))
 ((> anrm bignum) (setf anrmto bignum)
  (setf ilascl f2cl-lib:%true%)))
(if ilascl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (zlascl "G" 0 0 anrm anrmto n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
```

```
      (when var-3 (setf anrm var-3))
      (when var-4 (setf anrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "M" n n b ldb$ rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ilbscl f2cl-lib:%false%)
(cond
 ((and (> bnrm zero) (< bnrm smlnum)) (setf bnrmto smlnum)
  (setf ilbscl f2cl-lib:%true%))
 ((> bnrm bignum) (setf bnrmto bignum)
  (setf ilbscl f2cl-lib:%true%)))
(if ilbscl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (zlascl "G" 0 0 bnrm bnrmto n n b ldb$ ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bnrm var-3))
      (when var-4 (setf bnrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf ierr var-9))))
(setf ileft 1)
(setf iright (f2cl-lib:int-add n 1))
(setf irwrk (f2cl-lib:int-add iright n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (zggbal "P" n a lda b ldb$ ilo ihi
     (f2cl-lib:array-slice rwork-%data% double-float (ileft) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (iright) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (irwrk) ((1 *))
                           rwork-%offset%)
     ierr)
  (declare (ignore var-0 var-1 var-2 var-4 var-8 var-9 var-10))
  (setf lda var-3)
  (setf ldb$ var-5)
  (setf ilo var-6)
```

```
  (setf ihi var-7)
  (setf ierr var-11))
(setf irows (f2cl-lib:int-sub (f2cl-lib:int-add ihi 1) ilo))
(cond
 (ilv (setf icols (f2cl-lib:int-sub (f2cl-lib:int-add n 1) ilo)))
 (t (setf icols irows)))
(setf itau 1)
(setf iwrk (f2cl-lib:int-add itau irows))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgeqrf irows icols
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf irows var-0)
  (setf icols var-1)
  (setf ldb$ var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (zunmqr "L" "C" irows icols irows
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (ilo ilo)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf irows var-2))
  (when var-3 (setf icols var-3))
  (when var-4 (setf irows var-4))
  (when var-6 (setf ldb$ var-6))
  (when var-9 (setf lda var-9))
  (when var-12 (setf ierr var-12)))
(cond
 (ilvl
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "Full" n n czero cone vl ldvl)
```

```
    (declare (ignore var-0 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf cone var-4))
    (when var-6 (setf ldvl var-6)))
  (cond
   ((> irows 1)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlacpy "L" (f2cl-lib:int-sub irows 1)
         (f2cl-lib:int-sub irows 1)
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                               ((+ ilo 1) ilo) ((1 ldb$) (1 *))
                               b-%offset%)
         ldb$
         (f2cl-lib:array-slice vl-%data% f2cl-lib:complex16
                               ((+ ilo 1) ilo) ((1 ldvl) (1 *))
                               vl-%offset%)
         ldvl)
      (declare (ignore var-0 var-1 var-2 var-3 var-5))
      (when var-4 (setf ldb$ var-4))
      (when var-6 (setf ldvl var-6)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zungqr irows irows irows
       (f2cl-lib:array-slice vl-%data% f2cl-lib:complex16 (ilo ilo)
                             ((1 ldvl) (1 *)) vl-%offset%)
       ldvl
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf irows var-0))
    (when var-1 (setf irows var-1))
    (when var-2 (setf irows var-2))
    (when var-4 (setf ldvl var-4))
    (when var-8 (setf ierr var-8)))))
(if ilvr
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlaset "Full" n n czero cone vr ldvr)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldvr var-6))))
(cond
 (ilv
```

```
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (zgghrd jobvl jobvr n ilo ihi a lda b ldb$ vl ldvl vr ldvr
       ierr)
    (declare (ignore var-5 var-7 var-9 var-11))
    (when var-0 (setf jobvl var-0))
    (when var-1 (setf jobvr var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-13 (setf ierr var-13))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (zgghrd "N" "N" irows 1 irows
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (ilo ilo)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (ilo ilo)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$ vl ldvl vr ldvr ierr)
    (declare (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11))
    (when var-2 (setf irows var-2))
    (when var-4 (setf irows var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-13 (setf ierr var-13)))))
(setf iwrk itau)
(cond (ilv (f2cl-lib:f2cl-set-string chtemp "S" (string 1)))
      (t (f2cl-lib:f2cl-set-string chtemp "E" (string 1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18
     var-19)
    (zhgeqz chtemp jobvl jobvr n ilo ihi a lda b ldb$ alpha beta vl
     ldvl vr ldvr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk)
     (f2cl-lib:array-slice rwork-%data% double-float (irwrk) ((1 *))
                           rwork-%offset%)
     ierr)
```

```
  (declare
   (ignore var-6 var-8 var-10 var-11 var-12 var-14 var-16 var-17
    var-18))
  (when var-0 (setf chtemp var-0))
  (when var-1 (setf jobvl var-1))
  (when var-2 (setf jobvr var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf ilo var-4))
  (when var-5 (setf ihi var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldb$ var-9))
  (when var-13 (setf ldvl var-13))
  (when var-15 (setf ldvr var-15))
  (when var-19 (setf ierr var-19)))
(cond
 ((/= ierr 0)
  (cond ((and (> ierr 0) (<= ierr n)) (setf info ierr))
        ((and (> ierr n) (<= ierr (f2cl-lib:int-mul 2 n)))
         (setf info (f2cl-lib:int-sub ierr n)))
        (t (setf info (f2cl-lib:int-add n 1))))
  (go label70)))
(cond
 (ilv
  (cond
   (ilvl
    (cond (ilvr (f2cl-lib:f2cl-set-string chtemp "B" (string 1)))
          (t (f2cl-lib:f2cl-set-string chtemp "L" (string 1)))))
   (t (f2cl-lib:f2cl-set-string chtemp "R" (string 1))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16)
      (ztgevc chtemp "B" ldumma n a lda b ldb$ vl ldvl vr ldvr n in
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice rwork-%data% double-float (irwrk)
                             ((1 *)) rwork-%offset%)
       ierr)
    (declare
     (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-14 var-15))
    (when var-0 (setf chtemp var-0))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldb$ var-7))
    (when var-9 (setf ldvl var-9))
    (when var-11 (setf ldvr var-11))
    (when var-12 (setf n var-12))
    (when var-13 (setf in var-13))
    (when var-16 (setf ierr var-16)))
  (cond
   ((/= ierr 0) (setf info (f2cl-lib:int-add n 2)) (go label70)))
```

```
(cond
 (ilvl
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (zggbak "P" "L" n ilo ihi
       (f2cl-lib:array-slice rwork-%data% double-float (ileft)
                             ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% double-float (iright)
                             ((1 *)) rwork-%offset%)
       n vl ldvl ierr)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
    (setf n var-7)
    (setf ldvl var-9)
    (setf ierr var-10))
  (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                ((> jc n) nil)
    (tagbody
      (setf temp zero)
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf temp
                  (max temp
                       (abs1
                        (f2cl-lib:fref vl-%data% (jr jc)
                                       ((1 ldvl) (1 *))
                                       vl-%offset%))))
         label10))
      (if (< temp smlnum)
          (go label30))
      (setf temp (/ one temp))
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                               vl-%offset%)
                  (*
                   (f2cl-lib:fref vl-%data% (jr jc)
                                  ((1 ldvl) (1 *)) vl-%offset%)
                   temp))
         label20))
     label30))))
(cond
 (ilvr
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (zggbak "P" "R" n ilo ihi
```

```
              (f2cl-lib:array-slice rwork-%data% double-float (ileft)
                                    ((1 *)) rwork-%offset%)
              (f2cl-lib:array-slice rwork-%data% double-float (iright)
                                    ((1 *)) rwork-%offset%)
              n vr ldvr ierr)
            (declare
             (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
            (setf n var-7)
            (setf ldvr var-9)
            (setf ierr var-10))
          (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                        ((> jc n) nil)
            (tagbody
              (setf temp zero)
              (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                            ((> jr n) nil)
                (tagbody
                  (setf temp
                          (max temp
                               (abs1
                                (f2cl-lib:fref vr-%data% (jr jc)
                                               ((1 ldvr) (1 *))
                                               vr-%offset%))))
                 label40))
              (if (< temp smlnum)
                  (go label60))
              (setf temp (/ one temp))
              (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                            ((> jr n) nil)
                (tagbody
                  (setf (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                       vr-%offset%)
                          (*
                           (f2cl-lib:fref vr-%data% (jr jc)
                                          ((1 ldvr) (1 *)) vr-%offset%)
                           temp))
                 label50))
             label60))))))
label70
 (if ilascl
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (zlascl "G" 0 0 anrmto anrm n 1 alpha n ierr)
       (declare (ignore var-0 var-1 var-2 var-6 var-7))
       (when var-3 (setf anrmto var-3))
       (when var-4 (setf anrm var-4))
       (when var-5 (setf n var-5))
       (when var-8 (setf n var-8))
       (when var-9 (setf ierr var-9))))
 (if ilbscl
```

```
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (zlascl "G" 0 0 bnrmto bnrm n 1 beta n ierr)
                (declare (ignore var-0 var-1 var-2 var-6 var-7))
                (when var-3 (setf bnrmto var-3))
                (when var-4 (setf bnrm var-4))
                (when var-5 (setf n var-5))
                (when var-8 (setf n var-8))
                (when var-9 (setf ierr var-9))))
           (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                   (coerce lwkopt 'f2cl-lib:complex16))
           (go end_label)
          end_label
           (return
            (values jobvl jobvr n nil lda nil ldb$ nil nil nil ldvl nil ldvr nil
                    nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zggev fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::jobvl
                                              fortran-to-lisp::jobvr
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
```

```
                                                  fortran-to-lisp::ldb$ nil nil nil
                                                  fortran-to-lisp::ldvl nil
                                                  fortran-to-lisp::ldvr nil nil nil
                                                  fortran-to-lisp::info)
                                                :calls
                                                '(fortran-to-lisp::zggbak
                                                  fortran-to-lisp::zgeqrf
                                                  fortran-to-lisp::zggbal
                                                  fortran-to-lisp::xerbla
                                                  fortran-to-lisp::ilaenv))))
```

———-

27.11 zggev3

27.11.1 Axiom unit tests

— **zggev3.input** —

```
)set break resume
)sys rm -f zggev3.output
)spool zggev3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

27.11.2 Axiom help page

— **zggev3.help** —

```
====================================================================
zggev3 examples
====================================================================

====================================================================
Man Page Details
====================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine zggev3 (JOBVL, JOBVR, N, A, LDA, B, LDB, ALPHA, BETA, VL,
           LDVL, VR, LDVR, WORK, LWORK, RWORK, INFO)
            ZGGEV3 computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for GE matrices (blocked algorithm)

Function/Subroutine Documentation
   subroutine zggev3 (character JOBVL, character JOBVR, integer N, complex*16,
       dimension( lda, * ) A, integer LDA, complex*16, dimension( ldb, * ) B,
       integer LDB, complex*16, dimension( * ) ALPHA, complex*16, dimension( *
       ) BETA, complex*16, dimension( ldvl, * ) VL, integer LDVL, complex*16,
       dimension( ldvr, * ) VR, integer LDVR, complex*16, dimension( * ) WORK,
       integer LWORK, double precision, dimension( * ) RWORK, integer INFO)
        ZGGEV3 computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for GE matrices (blocked algorithm)

Purpose:
   ZGGEV3 computes for a pair of N-by-N complex nonsymmetric matrices
   (A,B), the generalized eigenvalues, and optionally, the left and/or
   right generalized eigenvectors.

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar
   lambda or a ratio alpha/beta = lambda, such that A - lambda*B is
   singular. It is usually represented as the pair (alpha,beta), as
   there is a reasonable interpretation for beta=0, and even for both
   being zero.

   The right generalized eigenvector v(j) corresponding to the
   generalized eigenvalue lambda(j) of (A,B) satisfies

   A * v(j) = lambda(j) * B * v(j).

   The left generalized eigenvector u(j) corresponding to the
   generalized eigenvalues lambda(j) of (A,B) satisfies

   u(j)**H * A = lambda(j) * u(j)**H * B

   where u(j)**H is the conjugate-transpose of u(j).

Parameters:

 JOBVL
   JOBVL is CHARACTER*1
   = 'N':  do not compute the left generalized eigenvectors;
   = 'V':  compute the left generalized eigenvectors.

 JOBVR
   JOBVR is CHARACTER*1
```

```
  = 'N':  do not compute the right generalized eigenvectors;
  = 'V':  compute the right generalized eigenvectors.

N
  N is INTEGER
  The order of the matrices A, B, VL, and VR.  N >= 0.

A
  A is COMPLEX*16 array, dimension (LDA, N)
  On entry, the matrix A in the pair (A,B).
  On exit, A has been overwritten.

LDA
  LDA is INTEGER
  The leading dimension of A.  LDA >= max(1,N).

B
  B is COMPLEX*16 array, dimension (LDB, N)
  On entry, the matrix B in the pair (A,B).
  On exit, B has been overwritten.

LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).

ALPHA
  ALPHA is COMPLEX*16 array, dimension (N)

BETA
  BETA is COMPLEX*16 array, dimension (N)
  On exit, ALPHA(j)/BETA(j), j=1,...,N, will be the
  generalized eigenvalues.

  Note: the quotients ALPHA(j)/BETA(j) may easily over- or
  underflow, and BETA(j) may even be zero.  Thus, the user
  should avoid naively computing the ratio alpha/beta.
  However, ALPHA will be always less than and usually
  comparable with norm(A) in magnitude, and BETA always less
  than and usually comparable with norm(B).

VL
  VL is COMPLEX*16 array, dimension (LDVL,N)
  If JOBVL = 'V', the left generalized eigenvectors u(j) are
  stored one after another in the columns of VL, in the same
  order as their eigenvalues.
  Each eigenvector is scaled so the largest component has
  abs(real part) + abs(imag. part) = 1.
  Not referenced if JOBVL = 'N'.

LDVL
```

```
  LDVL is INTEGER
  The leading dimension of the matrix VL. LDVL >= 1, and
  if JOBVL = 'V', LDVL >= N.

VR
  VR is COMPLEX*16 array, dimension (LDVR,N)
  If JOBVR = 'V', the right generalized eigenvectors v(j) are
  stored one after another in the columns of VR, in the same
  order as their eigenvalues.
  Each eigenvector is scaled so the largest component has
  abs(real part) + abs(imag. part) = 1.
  Not referenced if JOBVR = 'N'.

LDVR
  LDVR is INTEGER
  The leading dimension of the matrix VR. LDVR >= 1, and
  if JOBVR = 'V', LDVR >= N.

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (8*N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  =1,...,N:
        The QZ iteration failed.  No eigenvectors have been
        calculated, but ALPHA(j) and BETA(j) should be
        correct for j=INFO+1,...,N.
  > N:  =N+1: other then QZ iteration failed in DHGEQZ,
        =N+2: error return from DTGEVC.
```

———-

27.11.3 fortran code

— zggev3.f —

```
*  =====================================================================
      SUBROUTINE ZGGEV3( JOBVL, JOBVR, N, A, LDA, B, LDB, ALPHA, BETA,
     $                   VL, LDVL, VR, LDVR, WORK, LWORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     January 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVL, JOBVR
      INTEGER            INFO, LDA, LDB, LDVL, LDVR, LWORK, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   RWORK( * )
      COMPLEX*16         A( LDA, * ), ALPHA( * ), B( LDB, * ),
     $                   BETA( * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
      COMPLEX*16         CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0D0, 0.0D0 ),
     $                   CONE = ( 1.0D0, 0.0D0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            ILASCL, ILBSCL, ILV, ILVL, ILVR, LQUERY
      CHARACTER          CHTEMP
      INTEGER            ICOLS, IERR, IHI, IJOBVL, IJOBVR, ILEFT, ILO,
     $                   IN, IRIGHT, IROWS, IRWRK, ITAU, IWRK, JC, JR,
     $                   LWKOPT
      DOUBLE PRECISION   ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS,
     $                   SMLNUM, TEMP
      COMPLEX*16         X
*     ..
*     .. Local Arrays ..
      LOGICAL            LDUMMA( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, XERBLA, ZGEQRF, ZGGBAK, ZGGBAL, ZGGHD3,
     $                   ZHGEQZ, ZLACPY, ZLASCL, ZLASET, ZTGEVC, ZUNGQR,
```

```
     $                   ZUNMQR
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, ZLANGE
      EXTERNAL           LSAME, DLAMCH, ZLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, MAX, SQRT
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   ABS1
*     ..
*     .. Statement Function definitions ..
      ABS1( X ) = ABS( DBLE( X ) ) + ABS( DIMAG( X ) )
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVL, 'N' ) ) THEN
         IJOBVL = 1
         ILVL = .FALSE.
      ELSE IF( LSAME( JOBVL, 'V' ) ) THEN
         IJOBVL = 2
         ILVL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVL = .FALSE.
      END IF
*
      IF( LSAME( JOBVR, 'N' ) ) THEN
         IJOBVR = 1
         ILVR = .FALSE.
      ELSE IF( LSAME( JOBVR, 'V' ) ) THEN
         IJOBVR = 2
         ILVR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVR = .FALSE.
      END IF
      ILV = ILVL .OR. ILVR
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( IJOBVL.LE.0 ) THEN
         INFO = -1
      ELSE IF( IJOBVR.LE.0 ) THEN
```

```
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDVL.LT.1 .OR. ( ILVL .AND. LDVL.LT.N ) ) THEN
         INFO = -11
      ELSE IF( LDVR.LT.1 .OR. ( ILVR .AND. LDVR.LT.N ) ) THEN
         INFO = -13
      ELSE IF( LWORK.LT.MAX( 1, 2*N ) .AND. .NOT.LQUERY ) THEN
         INFO = -15
      END IF
*
*     Compute workspace
*
      IF( INFO.EQ.0 ) THEN
         CALL ZGEQRF( N, N, B, LDB, WORK, WORK, -1, IERR )
         LWKOPT = MAX( 1,  N+INT( WORK( 1 ) ) )
         CALL ZUNMQR( 'L', 'C', N, N, N, B, LDB, WORK, A, LDA, WORK,
     $                -1, IERR )
         LWKOPT = MAX( LWKOPT, N+INT( WORK( 1 ) ) )
         IF( ILVL ) THEN
            CALL ZUNGQR( N, N, N, VL, LDVL, WORK, WORK, -1, IERR )
            LWKOPT = MAX( LWKOPT, N+INT( WORK( 1 ) ) )
         END IF
         IF( ILV ) THEN
            CALL ZGGHD3( JOBVL, JOBVR, N, 1, N, A, LDA, B, LDB, VL,
     $                   LDVL, VR, LDVR, WORK, -1, IERR )
            LWKOPT = MAX( LWKOPT, N+INT( WORK( 1 ) ) )
            CALL ZHGEQZ( 'S', JOBVL, JOBVR, N, 1, N, A, LDA, B, LDB,
     $                   ALPHA, BETA, VL, LDVL, VR, LDVR, WORK, -1,
     $                   WORK, IERR )
            LWKOPT = MAX( LWKOPT, N+INT( WORK( 1 ) ) )
         ELSE
            CALL ZGGHD3( JOBVL, JOBVR, N, 1, N, A, LDA, B, LDB, VL,
     $                   LDVL, VR, LDVR, WORK, -1, IERR )
            LWKOPT = MAX( LWKOPT, N+INT( WORK( 1 ) ) )
            CALL ZHGEQZ( 'E', JOBVL, JOBVR, N, 1, N, A, LDA, B, LDB,
     $                   ALPHA, BETA, VL, LDVL, VR, LDVR, WORK, -1,
     $                   WORK, IERR )
            LWKOPT = MAX( LWKOPT, N+INT( WORK( 1 ) ) )
         END IF
         WORK( 1 ) = DCMPLX( LWKOPT )
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGGEV3 ', -INFO )
         RETURN
```

```
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
*
      EPS = DLAMCH( 'E' )*DLAMCH( 'B' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = ZLANGE( 'M', N, N, A, LDA, RWORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
      IF( ILASCL )
     $   CALL ZLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = ZLANGE( 'M', N, N, B, LDB, RWORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
     $   CALL ZLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute the matrices A, B to isolate eigenvalues if possible
*
      ILEFT = 1
      IRIGHT = N + 1
```

```
      IRWRK = IRIGHT + N
      CALL ZGGBAL( 'P', N, A, LDA, B, LDB, ILO, IHI, RWORK( ILEFT ),
     $             RWORK( IRIGHT ), RWORK( IRWRK ), IERR )
*
*     Reduce B to triangular form (QR decomposition of B)
*
      IROWS = IHI + 1 - ILO
      IF( ILV ) THEN
         ICOLS = N + 1 - ILO
      ELSE
         ICOLS = IROWS
      END IF
      ITAU = 1
      IWRK = ITAU + IROWS
      CALL ZGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the orthogonal transformation to matrix A
*
      CALL ZUNMQR( 'L', 'C', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VL
*
      IF( ILVL ) THEN
         CALL ZLASET( 'Full', N, N, CZERO, CONE, VL, LDVL )
         IF( IROWS.GT.1 ) THEN
            CALL ZLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VL( ILO+1, ILO ), LDVL )
         END IF
         CALL ZUNGQR( IROWS, IROWS, IROWS, VL( ILO, ILO ), LDVL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Initialize VR
*
      IF( ILVR )
     $   CALL ZLASET( 'Full', N, N, CZERO, CONE, VR, LDVR )
*
*     Reduce to generalized Hessenberg form
*
      IF( ILV ) THEN
*
*        Eigenvectors requested -- work on whole matrix.
*
         CALL ZGGHD3( JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB, VL,
     $                LDVL, VR, LDVR, WORK( IWRK ), LWORK+1-IWRK, IERR )
      ELSE
         CALL ZGGHD3( 'N', 'N', IROWS, 1, IROWS, A( ILO, ILO ), LDA,
```

```
     $                B( ILO, ILO ), LDB, VL, LDVL, VR, LDVR,
     $                WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
*     Perform QZ algorithm (Compute eigenvalues, and optionally, the
*     Schur form and Schur vectors)
*
      IWRK = ITAU
      IF( ILV ) THEN
         CHTEMP = 'S'
      ELSE
         CHTEMP = 'E'
      END IF
      CALL ZHGEQZ( CHTEMP, JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHA, BETA, VL, LDVL, VR, LDVR, WORK( IWRK ),
     $             LWORK+1-IWRK, RWORK( IRWRK ), IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 70
      END IF
*
*     Compute Eigenvectors
*
      IF( ILV ) THEN
         IF( ILVL ) THEN
            IF( ILVR ) THEN
               CHTEMP = 'B'
            ELSE
               CHTEMP = 'L'
            END IF
         ELSE
            CHTEMP = 'R'
         END IF
*
         CALL ZTGEVC( CHTEMP, 'B', LDUMMA, N, A, LDA, B, LDB, VL, LDVL,
     $                VR, LDVR, N, IN, WORK( IWRK ), RWORK( IRWRK ),
     $                IERR )
         IF( IERR.NE.0 ) THEN
            INFO = N + 2
            GO TO 70
         END IF
*
*        Undo balancing on VL and VR and normalization
*
```

```
         IF( ILVL ) THEN
            CALL ZGGBAK( 'P', 'L', N, ILO, IHI, RWORK( ILEFT ),
     $                   RWORK( IRIGHT ), N, VL, LDVL, IERR )
            DO 30 JC = 1, N
               TEMP = ZERO
               DO 10 JR = 1, N
                  TEMP = MAX( TEMP, ABS1( VL( JR, JC ) ) )
   10          CONTINUE
               IF( TEMP.LT.SMLNUM )
     $            GO TO 30
               TEMP = ONE / TEMP
               DO 20 JR = 1, N
                  VL( JR, JC ) = VL( JR, JC )*TEMP
   20          CONTINUE
   30       CONTINUE
         END IF
         IF( ILVR ) THEN
            CALL ZGGBAK( 'P', 'R', N, ILO, IHI, RWORK( ILEFT ),
     $                   RWORK( IRIGHT ), N, VR, LDVR, IERR )
            DO 60 JC = 1, N
               TEMP = ZERO
               DO 40 JR = 1, N
                  TEMP = MAX( TEMP, ABS1( VR( JR, JC ) ) )
   40          CONTINUE
               IF( TEMP.LT.SMLNUM )
     $            GO TO 60
               TEMP = ONE / TEMP
               DO 50 JR = 1, N
                  VR( JR, JC ) = VR( JR, JC )*TEMP
   50          CONTINUE
   60       CONTINUE
         END IF
      END IF
*
*     Undo scaling if necessary
*
   70 CONTINUE
*
      IF( ILASCL )
     $   CALL ZLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHA, N, IERR )
*
      IF( ILBSCL )
     $   CALL ZLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
*
      WORK( 1 ) = DCMPLX( LWKOPT )
      RETURN
*
*     End of ZGGEV3
*
      END
```

———-

27.11.4 lisp code

— **zggev3.lisp** —

```
(let* ((zero 0.0d0)
       (one 1.0d0)
       (czero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (cone (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (f2cl-lib:complex16) czero)
           (type (f2cl-lib:complex16) cone)
           (ignorable zero one czero cone))
  (defun zggev3
         (jobvl jobvr n a lda b ldb$ alpha beta vl ldvl vr ldvr work lwork
          rwork info)
    (declare (type (array double-float (*)) rwork)
             (type (array f2cl-lib:complex16 (*)) work vr vl beta alpha b a)
             (type (f2cl-lib:integer4) info lwork ldvr ldvl ldb$ lda n)
             (type (string *) jobvr jobvl))
    (f2cl-lib:with-multi-array-data
        ((jobvl character jobvl-%data% jobvl-%offset%)
         (jobvr character jobvr-%data% jobvr-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (alpha f2cl-lib:complex16 alpha-%data% alpha-%offset%)
         (beta f2cl-lib:complex16 beta-%data% beta-%offset%)
         (vl f2cl-lib:complex16 vl-%data% vl-%offset%)
         (vr f2cl-lib:complex16 vr-%data% vr-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (labels ((abs1 (x)
                 (+ (abs (f2cl-lib:dble x)) (abs (f2cl-lib:dimag x)))))
        (declare
         (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
          abs1))
        (prog ((ldumma (make-array 1 :element-type 't)) (x #C(0.0d0 0.0d0))
               (anrm 0.0d0) (anrmto 0.0d0) (bignum 0.0d0) (bnrm 0.0d0)
               (bnrmto 0.0d0) (eps 0.0d0) (smlnum 0.0d0) (temp 0.0d0) (icols 0)
               (ierr 0) (ihi 0) (ijobvl 0) (ijobvr 0) (ileft 0) (ilo 0) (in 0)
               (iright 0) (irows 0) (irwrk 0) (itau 0) (iwrk 0) (jc 0) (jr 0)
               (lwkopt 0)
               (chtemp
                (make-array '(1) :element-type 'character :initial-element
                            #\ ))
               (ilascl nil) (ilbscl nil) (ilv nil) (ilvl nil) (ilvr nil)
```

```
     (lquery nil))
(declare (type (array f2cl-lib:logical (1)) ldumma)
         (type (f2cl-lib:complex16) x)
         (type (double-float) anrm anrmto bignum bnrm bnrmto eps
          smlnum temp)
         (type (f2cl-lib:integer4) icols ierr ihi ijobvl ijobvr ileft
          ilo in iright irows irwrk itau iwrk jc jr lwkopt)
         (type (string 1) chtemp)
         (type f2cl-lib:logical ilascl ilbscl ilv ilvl ilvr lquery))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvl "N")
    (declare (ignore var-1))
    (when var-0 (setf jobvl var-0))
    ret-val)
  (setf ijobvl 1) (setf ilvl f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvl "V")
    (declare (ignore var-1))
    (when var-0 (setf jobvl var-0))
    ret-val)
  (setf ijobvl 2) (setf ilvl f2cl-lib:%true%))
 (t (setf ijobvl -1) (setf ilvl f2cl-lib:%false%)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvr "N")
    (declare (ignore var-1))
    (when var-0 (setf jobvr var-0))
    ret-val)
  (setf ijobvr 1) (setf ilvr f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvr "V")
    (declare (ignore var-1))
    (when var-0 (setf jobvr var-0))
    ret-val)
  (setf ijobvr 2) (setf ilvr f2cl-lib:%true%))
 (t (setf ijobvr -1) (setf ilvr f2cl-lib:%false%)))
(setf ilv (or ilvl ilvr))
(setf info 0)
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((<= ijobvl 0) (setf info -1)) ((<= ijobvr 0) (setf info -2))
      ((< n 0) (setf info -3))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -5))
      ((< ldb$
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -7))
      ((or (< ldvl 1) (and ilvl (< ldvl n))) (setf info -11))
      ((or (< ldvr 1) (and ilvr (< ldvr n))) (setf info -13))
```

```
        ((and
          (< lwork
             (max (the f2cl-lib:integer4 1)
                  (the f2cl-lib:integer4 (f2cl-lib:int-mul 2 n))))
          (not lquery))
         (setf info -15)))
    (cond
     ((= info 0)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (zgeqrf n n b ldb$ work work -1 ierr)
        (declare (ignore var-2 var-4 var-5 var-6))
        (setf n var-0)
        (setf n var-1)
        (setf ldb$ var-3)
        (setf ierr var-7))
      (setf lwkopt
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add n
                                          (f2cl-lib:int
                                           (f2cl-lib:fref work-%data% (1)
                                                          ((1 *))
                                                          work-%offset%))))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (zunmqr "L" "C" n n n b ldb$ work a lda work -1 ierr)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf n var-2))
        (when var-3 (setf n var-3))
        (when var-4 (setf n var-4))
        (when var-6 (setf ldb$ var-6))
        (when var-9 (setf lda var-9))
        (when var-12 (setf ierr var-12)))
      (setf lwkopt
              (max (the f2cl-lib:integer4 lwkopt)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add n
                                          (f2cl-lib:int
                                           (f2cl-lib:fref work-%data% (1)
                                                          ((1 *))
                                                          work-%offset%))))))
      (cond
       (ilvl
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (zungqr n n n vl ldvl work work -1 ierr)
          (declare (ignore var-3 var-5 var-6 var-7))
          (when var-0 (setf n var-0))
```

```
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldvl var-4))
    (when var-8 (setf ierr var-8)))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkopt)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add n
                                      (f2cl-lib:int
                                       (f2cl-lib:fref work-%data%
                                                      (1) ((1 *))
                                                      work-%offset%))))))))
(cond
 (ilv
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (zgghd3 jobvl jobvr n 1 n a lda b ldb$ vl ldvl vr ldvr work
       -1 ierr)
    (declare (ignore var-3 var-5 var-7 var-9 var-11 var-13 var-14))
    (when var-0 (setf jobvl var-0))
    (when var-1 (setf jobvr var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf n var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-15 (setf ierr var-15)))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkopt)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add n
                                      (f2cl-lib:int
                                       (f2cl-lib:fref work-%data%
                                                      (1) ((1 *))
                                                      work-%offset%))))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17
       var-18 var-19)
      (zhgeqz "S" jobvl jobvr n 1 n a lda b ldb$ alpha beta vl ldvl
       vr ldvr work -1 work ierr)
    (declare
     (ignore var-0 var-4 var-6 var-8 var-10 var-11 var-12 var-14
      var-16 var-17 var-18))
    (when var-1 (setf jobvl var-1))
    (when var-2 (setf jobvr var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf n var-5))
```

```
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldb$ var-9))
    (when var-13 (setf ldvl var-13))
    (when var-15 (setf ldvr var-15))
    (when var-19 (setf ierr var-19)))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkopt)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add n
                                      (f2cl-lib:int
                                       (f2cl-lib:fref work-%data%
                                                      (1) ((1 *))
                                                      work-%offset%)))))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (zgghd3 jobvl jobvr n 1 n a lda b ldb$ vl ldvl vr ldvr work
       -1 ierr)
    (declare (ignore var-3 var-5 var-7 var-9 var-11 var-13 var-14))
    (when var-0 (setf jobvl var-0))
    (when var-1 (setf jobvr var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf n var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-15 (setf ierr var-15)))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkopt)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add n
                                      (f2cl-lib:int
                                       (f2cl-lib:fref work-%data%
                                                      (1) ((1 *))
                                                      work-%offset%))))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17
       var-18 var-19)
      (zhgeqz "E" jobvl jobvr n 1 n a lda b ldb$ alpha beta vl ldvl
       vr ldvr work -1 work ierr)
    (declare
     (ignore var-0 var-4 var-6 var-8 var-10 var-11 var-12 var-14
      var-16 var-17 var-18))
    (when var-1 (setf jobvl var-1))
    (when var-2 (setf jobvr var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf n var-5))
```

```
      (when var-7 (setf lda var-7))
      (when var-9 (setf ldb$ var-9))
      (when var-13 (setf ldvl var-13))
      (when var-15 (setf ldvr var-15))
      (when var-19 (setf ierr var-19)))
    (setf lwkopt
            (max (the f2cl-lib:integer4 lwkopt)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add n
                                        (f2cl-lib:int
                                         (f2cl-lib:fref work-%data%
                                                        (1) ((1 *))
                                                        work-%offset%))))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:dcmplx lwkopt))))
(cond
 ((/= info 0) (xerbla "ZGGEV3 " (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(setf eps (* (dlamch "E") (dlamch "B")))
(setf smlnum (dlamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "M" n n a lda rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf ilascl f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf anrmto smlnum)
  (setf ilascl f2cl-lib:%true%))
 ((> anrm bignum) (setf anrmto bignum)
  (setf ilascl f2cl-lib:%true%)))
(if ilascl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (zlascl "G" 0 0 anrm anrmto n n a lda ierr)
```

```
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf anrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "M" n n b ldb$ rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ilbscl f2cl-lib:%false%)
(cond
 ((and (> bnrm zero) (< bnrm smlnum)) (setf bnrmto smlnum)
  (setf ilbscl f2cl-lib:%true%))
 ((> bnrm bignum) (setf bnrmto bignum)
  (setf ilbscl f2cl-lib:%true%)))
(if ilbscl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (zlascl "G" 0 0 bnrm bnrmto n n b ldb$ ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bnrm var-3))
      (when var-4 (setf bnrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf ierr var-9))))
(setf ileft 1)
(setf iright (f2cl-lib:int-add n 1))
(setf irwrk (f2cl-lib:int-add iright n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (zggbal "P" n a lda b ldb$ ilo ihi
     (f2cl-lib:array-slice rwork-%data% double-float (ileft) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (iright) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (irwrk) ((1 *))
                           rwork-%offset%)
     ierr)
  (declare (ignore var-0 var-1 var-2 var-4 var-8 var-9 var-10))
  (setf lda var-3)
  (setf ldb$ var-5)
```

```
  (setf ilo var-6)
  (setf ihi var-7)
  (setf ierr var-11))
(setf irows (f2cl-lib:int-sub (f2cl-lib:int-add ihi 1) ilo))
(cond
 (ilv (setf icols (f2cl-lib:int-sub (f2cl-lib:int-add n 1) ilo)))
 (t (setf icols irows)))
(setf itau 1)
(setf iwrk (f2cl-lib:int-add itau irows))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgeqrf irows icols
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf irows var-0)
  (setf icols var-1)
  (setf ldb$ var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (zunmqr "L" "C" irows icols irows
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (ilo ilo)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf irows var-2))
  (when var-3 (setf icols var-3))
  (when var-4 (setf irows var-4))
  (when var-6 (setf ldb$ var-6))
  (when var-9 (setf lda var-9))
  (when var-12 (setf ierr var-12)))
(cond
 (ilvl
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
        (zlaset "Full" n n czero cone vl ldvl)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldvl var-6)))
    (cond
     ((> irows 1)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlacpy "L" (f2cl-lib:int-sub irows 1)
           (f2cl-lib:int-sub irows 1)
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                 ((+ ilo 1) ilo) ((1 ldb$) (1 *))
                                 b-%offset%)
           ldb$
           (f2cl-lib:array-slice vl-%data% f2cl-lib:complex16
                                 ((+ ilo 1) ilo) ((1 ldvl) (1 *))
                                 vl-%offset%)
           ldvl)
        (declare (ignore var-0 var-1 var-2 var-3 var-5))
        (when var-4 (setf ldb$ var-4))
        (when var-6 (setf ldvl var-6)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (zungqr irows irows irows
         (f2cl-lib:array-slice vl-%data% f2cl-lib:complex16 (ilo ilo)
                               ((1 ldvl) (1 *)) vl-%offset%)
         ldvl
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
      (declare (ignore var-3 var-5 var-6 var-7))
      (when var-0 (setf irows var-0))
      (when var-1 (setf irows var-1))
      (when var-2 (setf irows var-2))
      (when var-4 (setf ldvl var-4))
      (when var-8 (setf ierr var-8)))))
  (if ilvr
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlaset "Full" n n czero cone vr ldvr)
        (declare (ignore var-0 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf czero var-3))
        (when var-4 (setf cone var-4))
        (when var-6 (setf ldvr var-6))))
  (cond
```

```
 (ilv
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (zgghd3 jobvl jobvr n ilo ihi a lda b ldb$ vl ldvl vr ldvr
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
    (declare (ignore var-5 var-7 var-9 var-11 var-13 var-14))
    (when var-0 (setf jobvl var-0))
    (when var-1 (setf jobvr var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-15 (setf ierr var-15))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (zgghd3 "N" "N" irows 1 irows
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (ilo ilo)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (ilo ilo)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$ vl ldvl vr ldvr
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
    (declare
     (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-13
      var-14))
    (when var-2 (setf irows var-2))
    (when var-4 (setf irows var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-15 (setf ierr var-15)))))
(setf iwrk itau)
(cond (ilv (f2cl-lib:f2cl-set-string chtemp "S" (string 1)))
      (t (f2cl-lib:f2cl-set-string chtemp "E" (string 1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18
     var-19)
```

```
      (zhgeqz chtemp jobvl jobvr n ilo ihi a lda b ldb$ alpha beta vl
       ldvl vr ldvr
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk)
       (f2cl-lib:array-slice rwork-%data% double-float (irwrk) ((1 *))
                             rwork-%offset%)
       ierr)
    (declare
     (ignore var-6 var-8 var-10 var-11 var-12 var-14 var-16 var-17
      var-18))
    (when var-0 (setf chtemp var-0))
    (when var-1 (setf jobvl var-1))
    (when var-2 (setf jobvr var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf ilo var-4))
    (when var-5 (setf ihi var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldb$ var-9))
    (when var-13 (setf ldvl var-13))
    (when var-15 (setf ldvr var-15))
    (when var-19 (setf ierr var-19)))
  (cond
   ((/= ierr 0)
    (cond ((and (> ierr 0) (<= ierr n)) (setf info ierr))
          ((and (> ierr n) (<= ierr (f2cl-lib:int-mul 2 n)))
           (setf info (f2cl-lib:int-sub ierr n)))
          (t (setf info (f2cl-lib:int-add n 1))))
    (go label70)))
  (cond
   (ilv
    (cond
     (ilvl
      (cond (ilvr (f2cl-lib:f2cl-set-string chtemp "B" (string 1)))
            (t (f2cl-lib:f2cl-set-string chtemp "L" (string 1)))))
     (t (f2cl-lib:f2cl-set-string chtemp "R" (string 1))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14 var-15 var-16)
        (ztgevc chtemp "B" ldumma n a lda b ldb$ vl ldvl vr ldvr n in
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice rwork-%data% double-float (irwrk)
                               ((1 *)) rwork-%offset%)
         ierr)
      (declare
       (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-14 var-15))
      (when var-0 (setf chtemp var-0))
      (when var-3 (setf n var-3))
      (when var-5 (setf lda var-5))
```

```
  (when var-7 (setf ldb$ var-7))
  (when var-9 (setf ldvl var-9))
  (when var-11 (setf ldvr var-11))
  (when var-12 (setf n var-12))
  (when var-13 (setf in var-13))
  (when var-16 (setf ierr var-16)))
(cond
 ((/= ierr 0) (setf info (f2cl-lib:int-add n 2)) (go label70)))
(cond
 (ilvl
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (zggbak "P" "L" n ilo ihi
       (f2cl-lib:array-slice rwork-%data% double-float (ileft)
                             ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% double-float (iright)
                             ((1 *)) rwork-%offset%)
       n vl ldvl ierr)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
    (setf n var-7)
    (setf ldvl var-9)
    (setf ierr var-10))
  (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                ((> jc n) nil)
    (tagbody
      (setf temp zero)
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf temp
                  (max temp
                       (abs1
                        (f2cl-lib:fref vl-%data% (jr jc)
                                       ((1 ldvl) (1 *))
                                       vl-%offset%))))
         label10))
      (if (< temp smlnum)
          (go label30))
      (setf temp (/ one temp))
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                               vl-%offset%)
                  (*
                   (f2cl-lib:fref vl-%data% (jr jc)
                                  ((1 ldvl) (1 *)) vl-%offset%)
                   temp))
```

```
            label20))
        label30))))
    (cond
     (ilvr
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (zggbak "P" "R" n ilo ihi
           (f2cl-lib:array-slice rwork-%data% double-float (ileft)
                                 ((1 *)) rwork-%offset%)
           (f2cl-lib:array-slice rwork-%data% double-float (iright)
                                 ((1 *)) rwork-%offset%)
           n vr ldvr ierr)
        (declare
         (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
        (setf n var-7)
        (setf ldvr var-9)
        (setf ierr var-10))
      (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                    ((> jc n) nil)
        (tagbody
          (setf temp zero)
          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                        ((> jr n) nil)
            (tagbody
              (setf temp
                      (max temp
                           (abs1
                            (f2cl-lib:fref vr-%data% (jr jc)
                                           ((1 ldvr) (1 *))
                                           vr-%offset%))))
            label40))
          (if (< temp smlnum)
              (go label60))
          (setf temp (/ one temp))
          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                        ((> jr n) nil)
            (tagbody
              (setf (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                   vr-%offset%)
                      (*
                       (f2cl-lib:fref vr-%data% (jr jc)
                                      ((1 ldvr) (1 *)) vr-%offset%)
                       temp))
            label50))
        label60))))))
label70
 (if ilascl
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
```

```
                (zlascl "G" 0 0 anrmto anrm n 1 alpha n ierr)
              (declare (ignore var-0 var-1 var-2 var-6 var-7))
              (when var-3 (setf anrmto var-3))
              (when var-4 (setf anrm var-4))
              (when var-5 (setf n var-5))
              (when var-8 (setf n var-8))
              (when var-9 (setf ierr var-9))))
         (if ilbscl
             (multiple-value-bind
                 (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                 (zlascl "G" 0 0 bnrmto bnrm n 1 beta n ierr)
              (declare (ignore var-0 var-1 var-2 var-6 var-7))
              (when var-3 (setf bnrmto var-3))
              (when var-4 (setf bnrm var-4))
              (when var-5 (setf n var-5))
              (when var-8 (setf n var-8))
              (when var-9 (setf ierr var-9))))
         (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                 (f2cl-lib:dcmplx lwkopt))
         (go end_label)
        end_label
         (return
          (values jobvl jobvr n nil lda nil ldb$ nil nil nil ldvl nil ldvr nil
                  nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zggev3
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string) (string)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
```

```
   (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::jobvl
  fortran-to-lisp::jobvr
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$ nil nil nil
  fortran-to-lisp::ldvl nil
  fortran-to-lisp::ldvr nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zggbak
  fortran-to-lisp::zggbal
  fortran-to-lisp::xerbla
  fortran-to-lisp::zgeqrf))))
```

———-

27.12 zggevx

27.12.1 Axiom unit tests

— zggevx.input —

```
)set break resume
)sys rm -f zggevx.output
)spool zggevx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

27.12.2 Axiom help page

— zggevx.help —

```
====================================================================
zggevx examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zggevx (BALANC, JOBVL, JOBVR, SENSE, N, A, LDA, B, LDB,
           ALPHA, BETA, VL, LDVL, VR, LDVR, ILO, IHI, LSCALE, RSCALE, ABNRM,
           BBNRM, RCONDE, RCONDV, WORK, LWORK, RWORK, IWORK, BWORK, INFO)
            ZGGEVX computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for GE matrices

Function/Subroutine Documentation
   subroutine zggevx (character BALANC, character JOBVL, character JOBVR,
       character SENSE, integer N, complex*16, dimension( lda, * ) A, integer
       LDA, complex*16, dimension( ldb, * ) B, integer LDB, complex*16,
       dimension( * ) ALPHA, complex*16, dimension( * ) BETA, complex*16,
       dimension( ldvl, * ) VL, integer LDVL, complex*16, dimension( ldvr, * )
       VR, integer LDVR, integer ILO, integer IHI, double precision,
       dimension( * ) LSCALE, double precision, dimension( * ) RSCALE, double
       precision ABNRM, double precision BBNRM, double precision, dimension( *
       ) RCONDE, double precision, dimension( * ) RCONDV, complex*16,
       dimension( * ) WORK, integer LWORK, double precision, dimension( * )
       RWORK, integer, dimension( * ) IWORK, logical, dimension( * ) BWORK,
       integer INFO)
        ZGGEVX computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for GE matrices

Purpose:
   ZGGEVX computes for a pair of N-by-N complex nonsymmetric matrices
   (A,B) the generalized eigenvalues, and optionally, the left and/or
   right generalized eigenvectors.

   Optionally, it also computes a balancing transformation to improve
   the conditioning of the eigenvalues and eigenvectors (ILO, IHI,
   LSCALE, RSCALE, ABNRM, and BBNRM), reciprocal condition numbers for
   the eigenvalues (RCONDE), and reciprocal condition numbers for the
   right eigenvectors (RCONDV).

   A generalized eigenvalue for a pair of matrices (A,B) is a scalar
   lambda or a ratio alpha/beta = lambda, such that A - lambda*B is
   singular. It is usually represented as the pair (alpha,beta), as
   there is a reasonable interpretation for beta=0, and even for both
   being zero.

   The right eigenvector v(j) corresponding to the eigenvalue lambda(j)
```

```
   of (A,B) satisfies
   A * v(j) = lambda(j) * B * v(j) .
   The left eigenvector u(j) corresponding to the eigenvalue lambda(j)
   of (A,B) satisfies
   u(j)**H * A  = lambda(j) * u(j)**H * B.
   where u(j)**H is the conjugate-transpose of u(j).

Parameters:

 BALANC
   BALANC is CHARACTER*1
   Specifies the balance option to be performed:
   = 'N':  do not diagonally scale or permute;
   = 'P':  permute only;
   = 'S':  scale only;
   = 'B':  both permute and scale.
   Computed reciprocal condition numbers will be for the
   matrices after permuting and/or balancing. Permuting does
   not change condition numbers (in exact arithmetic), but balancing does.

 JOBVL
   JOBVL is CHARACTER*1
   = 'N':  do not compute the left generalized eigenvectors;
   = 'V':  compute the left generalized eigenvectors.

 JOBVR
   JOBVR is CHARACTER*1
   = 'N':  do not compute the right generalized eigenvectors;
   = 'V':  compute the right generalized eigenvectors.

 SENSE
   SENSE is CHARACTER*1
   Determines which reciprocal condition numbers are computed.
   = 'N': none are computed;
   = 'E': computed for eigenvalues only;
   = 'V': computed for eigenvectors only;
   = 'B': computed for eigenvalues and eigenvectors.

 N
   N is INTEGER
   The order of the matrices A, B, VL, and VR.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA, N)
   On entry, the matrix A in the pair (A,B).
   On exit, A has been overwritten. If JOBVL='V' or JOBVR='V'
   or both, then A contains the first part of the complex Schur
   form of the "balanced" versions of the input A and B.

 LDA
```

```
  LDA is INTEGER
  The leading dimension of A.  LDA >= max(1,N).

B
  B is COMPLEX*16 array, dimension (LDB, N)
  On entry, the matrix B in the pair (A,B).
  On exit, B has been overwritten. If JOBVL='V' or JOBVR='V'
  or both, then B contains the second part of the complex
  Schur form of the "balanced" versions of the input A and B.

LDB
  LDB is INTEGER
  The leading dimension of B.  LDB >= max(1,N).

ALPHA
  ALPHA is COMPLEX*16 array, dimension (N)

BETA
  BETA is COMPLEX*16 array, dimension (N)
  On exit, ALPHA(j)/BETA(j), j=1,...,N, will be the generalized
  eigenvalues.

  Note: the quotient ALPHA(j)/BETA(j) ) may easily over- or
  underflow, and BETA(j) may even be zero.  Thus, the user
  should avoid naively computing the ratio ALPHA/BETA.
  However, ALPHA will be always less than and usually
  comparable with norm(A) in magnitude, and BETA always less
  than and usually comparable with norm(B).

VL
  VL is COMPLEX*16 array, dimension (LDVL,N)
  If JOBVL = 'V', the left generalized eigenvectors u(j) are
  stored one after another in the columns of VL, in the same
  order as their eigenvalues.
  Each eigenvector will be scaled so the largest component
  will have abs(real part) + abs(imag. part) = 1.
  Not referenced if JOBVL = 'N'.

LDVL
  LDVL is INTEGER
  The leading dimension of the matrix VL. LDVL >= 1, and
  if JOBVL = 'V', LDVL >= N.

VR
  VR is COMPLEX*16 array, dimension (LDVR,N)
  If JOBVR = 'V', the right generalized eigenvectors v(j) are
  stored one after another in the columns of VR, in the same
  order as their eigenvalues.
  Each eigenvector will be scaled so the largest component
  will have abs(real part) + abs(imag. part) = 1.
```

```
  Not referenced if JOBVR = 'N'.

LDVR
  LDVR is INTEGER
  The leading dimension of the matrix VR. LDVR >= 1, and
  if JOBVR = 'V', LDVR >= N.

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER
  ILO and IHI are integer values such that on exit
  A(i,j) = 0 and B(i,j) = 0 if i > j and
  j = 1,...,ILO-1 or i = IHI+1,...,N.
  If BALANC = 'N' or 'S', ILO = 1 and IHI = N.

LSCALE
  LSCALE is DOUBLE PRECISION array, dimension (N)
  Details of the permutations and scaling factors applied
  to the left side of A and B.  If PL(j) is the index of the
  row interchanged with row j, and DL(j) is the scaling
  factor applied to row j, then
  LSCALE(j) = PL(j)  for j = 1,...,ILO-1
            = DL(j)  for j = ILO,...,IHI
            = PL(j)  for j = IHI+1,...,N.
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

RSCALE
  RSCALE is DOUBLE PRECISION array, dimension (N)
  Details of the permutations and scaling factors applied
  to the right side of A and B.  If PR(j) is the index of the
  column interchanged with column j, and DR(j) is the scaling
  factor applied to column j, then
  RSCALE(j) = PR(j)  for j = 1,...,ILO-1
            = DR(j)  for j = ILO,...,IHI
            = PR(j)  for j = IHI+1,...,N
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

ABNRM
  ABNRM is DOUBLE PRECISION
  The one-norm of the balanced matrix A.

BBNRM
  BBNRM is DOUBLE PRECISION
  The one-norm of the balanced matrix B.

RCONDE
```

```
  RCONDE is DOUBLE PRECISION array, dimension (N)
  If SENSE = 'E' or 'B', the reciprocal condition numbers of
  the eigenvalues, stored in consecutive elements of the array.
  If SENSE = 'N' or 'V', RCONDE is not referenced.

RCONDV
  RCONDV is DOUBLE PRECISION array, dimension (N)
  If JOB = 'V' or 'B', the estimated reciprocal condition
  numbers of the eigenvectors, stored in consecutive elements
  of the array. If the eigenvalues cannot be reordered to
  compute RCONDV(j), RCONDV(j) is set to 0; this can only occur
  when the true value would be very small anyway.
  If SENSE = 'N' or 'E', RCONDV is not referenced.

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK >= max(1,2*N).
  If SENSE = 'E', LWORK >= max(1,4*N).
  If SENSE = 'V' or 'B', LWORK >= max(1,2*N*N+2*N).

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (lrwork)
  lrwork must be at least max(1,6*N) if BALANC = 'S' or 'B',
  and at least max(1,2*N) otherwise.
  Real workspace.

IWORK
  IWORK is INTEGER array, dimension (N+2)
  If SENSE = 'E', IWORK is not referenced.

BWORK
  BWORK is LOGICAL array, dimension (N)
  If SENSE = 'N', BWORK is not referenced.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  = 1,...,N:
         The QZ iteration failed.  No eigenvectors have been
         calculated, but ALPHA(j) and BETA(j) should be correct
```

```
          for j=INFO+1,...,N.
   > N:  =N+1: other than QZ iteration failed in ZHGEQZ.
         =N+2: error return from ZTGEVC.

Further Details:
   Balancing a matrix pair (A,B) includes, first, permuting rows and
   columns to isolate eigenvalues, second, applying diagonal similarity
   transformation to the rows and columns to make the rows and columns
   as close in norm as possible. The computed reciprocal condition
   numbers correspond to the balanced matrix. Permuting rows and columns
   will not change the condition numbers (in exact arithmetic) but
   diagonal scaling will.  For further explanation of balancing, see
   section 4.11.1.2 of LAPACK Users' Guide.

   An approximate error bound on the chordal distance between the i-th
   computed generalized eigenvalue w and the corresponding exact
   eigenvalue lambda is

   chord(w, lambda) <= EPS * norm(ABNRM, BBNRM) / RCONDE(I)

   An approximate error bound for the angle between the i-th computed
   eigenvector VL(i) or VR(i) is given by

   EPS * norm(ABNRM, BBNRM) / DIF(i).

   For further explanation of the reciprocal condition numbers RCONDE
   and RCONDV, see section 4.11 of LAPACK User's Guide.
```

———-

27.12.3 fortran code

— zggevx.f —

```
*  ===================================================================
      SUBROUTINE ZGGEVX( BALANC, JOBVL, JOBVR, SENSE, N, A, LDA, B, LDB,
     $                   ALPHA, BETA, VL, LDVL, VR, LDVR, ILO, IHI,
     $                   LSCALE, RSCALE, ABNRM, BBNRM, RCONDE, RCONDV,
     $                   WORK, LWORK, RWORK, IWORK, BWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          BALANC, JOBVL, JOBVR, SENSE
```

```
      INTEGER            IHI, ILO, INFO, LDA, LDB, LDVL, LDVR, LWORK, N
      DOUBLE PRECISION   ABNRM, BBNRM
*     ..
*     .. Array Arguments ..
      LOGICAL            BWORK( * )
      INTEGER            IWORK( * )
      DOUBLE PRECISION   LSCALE( * ), RCONDE( * ), RCONDV( * ),
     $                   RSCALE( * ), RWORK( * )
      COMPLEX*16         A( LDA, * ), ALPHA( * ), B( LDB, * ),
     $                   BETA( * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      COMPLEX*16         CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0D+0, 0.0D+0 ),
     $                   CONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            ILASCL, ILBSCL, ILV, ILVL, ILVR, LQUERY, NOSCL,
     $                   WANTSB, WANTSE, WANTSN, WANTSV
      CHARACTER          CHTEMP
      INTEGER            I, ICOLS, IERR, IJOBVL, IJOBVR, IN, IROWS,
     $                   ITAU, IWRK, IWRK1, J, JC, JR, M, MAXWRK, MINWRK
      DOUBLE PRECISION   ANRM, ANRMTO, BIGNUM, BNRM, BNRMTO, EPS,
     $                   SMLNUM, TEMP
      COMPLEX*16         X
*     ..
*     .. Local Arrays ..
      LOGICAL            LDUMMA( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, DLASCL, XERBLA, ZGEQRF, ZGGBAK, ZGGBAL,
     $                   ZGGHRD, ZHGEQZ, ZLACPY, ZLASCL, ZLASET, ZTGEVC,
     $                   ZTGSNA, ZUNGQR, ZUNMQR
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, ZLANGE
      EXTERNAL           LSAME, ILAENV, DLAMCH, ZLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, MAX, SQRT
*     ..
*     .. Statement Functions ..
```

```
      DOUBLE PRECISION   ABS1
*     ..
*     .. Statement Function definitions ..
      ABS1( X ) = ABS( DBLE( X ) ) + ABS( DIMAG( X ) )
*     ..
*     .. Executable Statements ..
*
*     Decode the input arguments
*
      IF( LSAME( JOBVL, 'N' ) ) THEN
         IJOBVL = 1
         ILVL = .FALSE.
      ELSE IF( LSAME( JOBVL, 'V' ) ) THEN
         IJOBVL = 2
         ILVL = .TRUE.
      ELSE
         IJOBVL = -1
         ILVL = .FALSE.
      END IF
*
      IF( LSAME( JOBVR, 'N' ) ) THEN
         IJOBVR = 1
         ILVR = .FALSE.
      ELSE IF( LSAME( JOBVR, 'V' ) ) THEN
         IJOBVR = 2
         ILVR = .TRUE.
      ELSE
         IJOBVR = -1
         ILVR = .FALSE.
      END IF
      ILV = ILVL .OR. ILVR
*
      NOSCL  = LSAME( BALANC, 'N' ) .OR. LSAME( BALANC, 'P' )
      WANTSN = LSAME( SENSE, 'N' )
      WANTSE = LSAME( SENSE, 'E' )
      WANTSV = LSAME( SENSE, 'V' )
      WANTSB = LSAME( SENSE, 'B' )
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.( NOSCL .OR. LSAME( BALANC,'S' ) .OR.
     $    LSAME( BALANC, 'B' ) ) ) THEN
         INFO = -1
      ELSE IF( IJOBVL.LE.0 ) THEN
         INFO = -2
      ELSE IF( IJOBVR.LE.0 ) THEN
         INFO = -3
      ELSE IF( .NOT.( WANTSN .OR. WANTSE .OR. WANTSB .OR. WANTSV ) )
```

```
     $          THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDVL.LT.1 .OR. ( ILVL .AND. LDVL.LT.N ) ) THEN
         INFO = -13
      ELSE IF( LDVR.LT.1 .OR. ( ILVR .AND. LDVR.LT.N ) ) THEN
         INFO = -15
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV. The workspace is
*       computed assuming ILO = 1 and IHI = N, the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            MAXWRK = 1
         ELSE
            MINWRK = 2*N
            IF( WANTSE ) THEN
               MINWRK = 4*N
            ELSE IF( WANTSV .OR. WANTSB ) THEN
               MINWRK = 2*N*( N + 1)
            END IF
            MAXWRK = MINWRK
            MAXWRK = MAX( MAXWRK,
     $                    N + N*ILAENV( 1, 'ZGEQRF', ' ', N, 1, N, 0 ) )
            MAXWRK = MAX( MAXWRK,
     $                    N + N*ILAENV( 1, 'ZUNMQR', ' ', N, 1, N, 0 ) )
            IF( ILVL ) THEN
               MAXWRK = MAX( MAXWRK, N +
     $                       N*ILAENV( 1, 'ZUNGQR', ' ', N, 1, N, 0 ) )
            END IF
         END IF
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -25
         END IF
      END IF
*
```

```
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGGEVX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = ZLANGE( 'M', N, N, A, LDA, RWORK )
      ILASCL = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ANRMTO = SMLNUM
         ILASCL = .TRUE.
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ANRMTO = BIGNUM
         ILASCL = .TRUE.
      END IF
      IF( ILASCL )
     $   CALL ZLASCL( 'G', 0, 0, ANRM, ANRMTO, N, N, A, LDA, IERR )
*
*     Scale B if max element outside range [SMLNUM,BIGNUM]
*
      BNRM = ZLANGE( 'M', N, N, B, LDB, RWORK )
      ILBSCL = .FALSE.
      IF( BNRM.GT.ZERO .AND. BNRM.LT.SMLNUM ) THEN
         BNRMTO = SMLNUM
         ILBSCL = .TRUE.
      ELSE IF( BNRM.GT.BIGNUM ) THEN
         BNRMTO = BIGNUM
         ILBSCL = .TRUE.
      END IF
      IF( ILBSCL )
     $   CALL ZLASCL( 'G', 0, 0, BNRM, BNRMTO, N, N, B, LDB, IERR )
*
*     Permute and/or balance the matrix pair (A,B)
```

```
*     (Real Workspace: need 6*N if BALANC = 'S' or 'B', 1 otherwise)
*
      CALL ZGGBAL( BALANC, N, A, LDA, B, LDB, ILO, IHI, LSCALE, RSCALE,
     $             RWORK, IERR )
*
*     Compute ABNRM and BBNRM
*
      ABNRM = ZLANGE( '1', N, N, A, LDA, RWORK( 1 ) )
      IF( ILASCL ) THEN
         RWORK( 1 ) = ABNRM
         CALL DLASCL( 'G', 0, 0, ANRMTO, ANRM, 1, 1, RWORK( 1 ), 1,
     $                IERR )
         ABNRM = RWORK( 1 )
      END IF
*
      BBNRM = ZLANGE( '1', N, N, B, LDB, RWORK( 1 ) )
      IF( ILBSCL ) THEN
         RWORK( 1 ) = BBNRM
         CALL DLASCL( 'G', 0, 0, BNRMTO, BNRM, 1, 1, RWORK( 1 ), 1,
     $                IERR )
         BBNRM = RWORK( 1 )
      END IF
*
*     Reduce B to triangular form (QR decomposition of B)
*     (Complex Workspace: need N, prefer N*NB )
*
      IROWS = IHI + 1 - ILO
      IF( ILV .OR. .NOT.WANTSN ) THEN
         ICOLS = N + 1 - ILO
      ELSE
         ICOLS = IROWS
      END IF
      ITAU = 1
      IWRK = ITAU + IROWS
      CALL ZGEQRF( IROWS, ICOLS, B( ILO, ILO ), LDB, WORK( ITAU ),
     $             WORK( IWRK ), LWORK+1-IWRK, IERR )
*
*     Apply the unitary transformation to A
*     (Complex Workspace: need N, prefer N*NB)
*
      CALL ZUNMQR( 'L', 'C', IROWS, ICOLS, IROWS, B( ILO, ILO ), LDB,
     $             WORK( ITAU ), A( ILO, ILO ), LDA, WORK( IWRK ),
     $             LWORK+1-IWRK, IERR )
*
*     Initialize VL and/or VR
*     (Workspace: need N, prefer N*NB)
*
      IF( ILVL ) THEN
         CALL ZLASET( 'Full', N, N, CZERO, CONE, VL, LDVL )
         IF( IROWS.GT.1 ) THEN
```

```
            CALL ZLACPY( 'L', IROWS-1, IROWS-1, B( ILO+1, ILO ), LDB,
     $                   VL( ILO+1, ILO ), LDVL )
         END IF
         CALL ZUNGQR( IROWS, IROWS, IROWS, VL( ILO, ILO ), LDVL,
     $                WORK( ITAU ), WORK( IWRK ), LWORK+1-IWRK, IERR )
      END IF
*
      IF( ILVR )
     $   CALL ZLASET( 'Full', N, N, CZERO, CONE, VR, LDVR )
*
*     Reduce to generalized Hessenberg form
*     (Workspace: none needed)
*
      IF( ILV .OR. .NOT.WANTSN ) THEN
*
*        Eigenvectors requested -- work on whole matrix.
*
         CALL ZGGHRD( JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB, VL,
     $                LDVL, VR, LDVR, IERR )
      ELSE
         CALL ZGGHRD( 'N', 'N', IROWS, 1, IROWS, A( ILO, ILO ), LDA,
     $                B( ILO, ILO ), LDB, VL, LDVL, VR, LDVR, IERR )
      END IF
*
*     Perform QZ algorithm (Compute eigenvalues, and optionally, the
*     Schur forms and Schur vectors)
*     (Complex Workspace: need N)
*     (Real Workspace: need N)
*
      IWRK = ITAU
      IF( ILV .OR. .NOT.WANTSN ) THEN
         CHTEMP = 'S'
      ELSE
         CHTEMP = 'E'
      END IF
*
      CALL ZHGEQZ( CHTEMP, JOBVL, JOBVR, N, ILO, IHI, A, LDA, B, LDB,
     $             ALPHA, BETA, VL, LDVL, VR, LDVR, WORK( IWRK ),
     $             LWORK+1-IWRK, RWORK, IERR )
      IF( IERR.NE.0 ) THEN
         IF( IERR.GT.0 .AND. IERR.LE.N ) THEN
            INFO = IERR
         ELSE IF( IERR.GT.N .AND. IERR.LE.2*N ) THEN
            INFO = IERR - N
         ELSE
            INFO = N + 1
         END IF
         GO TO 90
      END IF
*
```

```
*     Compute Eigenvectors and estimate condition numbers if desired
*     ZTGEVC: (Complex Workspace: need 2*N )
*             (Real Workspace:    need 2*N )
*     ZTGSNA: (Complex Workspace: need 2*N*N if SENSE='V' or 'B')
*             (Integer Workspace: need N+2 )
*
      IF( ILV .OR. .NOT.WANTSN ) THEN
         IF( ILV ) THEN
            IF( ILVL ) THEN
               IF( ILVR ) THEN
                  CHTEMP = 'B'
               ELSE
                  CHTEMP = 'L'
               END IF
            ELSE
               CHTEMP = 'R'
            END IF
*
            CALL ZTGEVC( CHTEMP, 'B', LDUMMA, N, A, LDA, B, LDB, VL,
     $                   LDVL, VR, LDVR, N, IN, WORK( IWRK ), RWORK,
     $                   IERR )
            IF( IERR.NE.0 ) THEN
               INFO = N + 2
               GO TO 90
            END IF
         END IF
*
         IF( .NOT.WANTSN ) THEN
*
*           compute eigenvectors (DTGEVC) and estimate condition
*           numbers (DTGSNA). Note that the definition of the condition
*           number is not invariant under transformation (u,v) to
*           (Q*u, Z*v), where (u,v) are eigenvectors of the generalized
*           Schur form (S,T), Q and Z are orthogonal matrices. In order
*           to avoid using extra 2*N*N workspace, we have to
*           re-calculate eigenvectors and estimate the condition numbers
*           one at a time.
*
            DO 20 I = 1, N
*
               DO 10 J = 1, N
                  BWORK( J ) = .FALSE.
   10          CONTINUE
               BWORK( I ) = .TRUE.
*
               IWRK = N + 1
               IWRK1 = IWRK + N
*
               IF( WANTSE .OR. WANTSB ) THEN
                  CALL ZTGEVC( 'B', 'S', BWORK, N, A, LDA, B, LDB,
```

```
     $                        WORK( 1 ), N, WORK( IWRK ), N, 1, M,
     $                        WORK( IWRK1 ), RWORK, IERR )
                  IF( IERR.NE.0 ) THEN
                     INFO = N + 2
                     GO TO 90
                  END IF
               END IF
*
               CALL ZTGSNA( SENSE, 'S', BWORK, N, A, LDA, B, LDB,
     $                      WORK( 1 ), N, WORK( IWRK ), N, RCONDE( I ),
     $                      RCONDV( I ), 1, M, WORK( IWRK1 ),
     $                      LWORK-IWRK1+1, IWORK, IERR )
*
   20       CONTINUE
         END IF
      END IF
*
*     Undo balancing on VL and VR and normalization
*     (Workspace: none needed)
*
      IF( ILVL ) THEN
         CALL ZGGBAK( BALANC, 'L', N, ILO, IHI, LSCALE, RSCALE, N, VL,
     $                LDVL, IERR )
*
         DO 50 JC = 1, N
            TEMP = ZERO
            DO 30 JR = 1, N
               TEMP = MAX( TEMP, ABS1( VL( JR, JC ) ) )
   30       CONTINUE
            IF( TEMP.LT.SMLNUM )
     $         GO TO 50
            TEMP = ONE / TEMP
            DO 40 JR = 1, N
               VL( JR, JC ) = VL( JR, JC )*TEMP
   40       CONTINUE
   50    CONTINUE
      END IF
*
      IF( ILVR ) THEN
         CALL ZGGBAK( BALANC, 'R', N, ILO, IHI, LSCALE, RSCALE, N, VR,
     $                LDVR, IERR )
         DO 80 JC = 1, N
            TEMP = ZERO
            DO 60 JR = 1, N
               TEMP = MAX( TEMP, ABS1( VR( JR, JC ) ) )
   60       CONTINUE
            IF( TEMP.LT.SMLNUM )
     $         GO TO 80
            TEMP = ONE / TEMP
            DO 70 JR = 1, N
```

```
              VR( JR, JC ) = VR( JR, JC )*TEMP
   70       CONTINUE
   80    CONTINUE
      END IF
*
*     Undo scaling if necessary
*
   90 CONTINUE
*
      IF( ILASCL )
     $   CALL ZLASCL( 'G', 0, 0, ANRMTO, ANRM, N, 1, ALPHA, N, IERR )
*
      IF( ILBSCL )
     $   CALL ZLASCL( 'G', 0, 0, BNRMTO, BNRM, N, 1, BETA, N, IERR )
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of ZGGEVX
*
      END
```

———-

27.12.4 lisp code

— **zggevx.lisp** —

```
(let* ((zero 0.0d0)
       (one 1.0d0)
       (czero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (cone (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (f2cl-lib:complex16) czero)
           (type (f2cl-lib:complex16) cone)
           (ignorable zero one czero cone))
  (defun zggevx
         (balanc jobvl jobvr sense n a lda b ldb$ alpha beta vl ldvl vr ldvr
          ilo ihi lscale rscale abnrm bbnrm rconde rcondv work lwork rwork
          iwork bwork info)
    (declare (type (array f2cl-lib:logical (*)) bwork)
             (type (array f2cl-lib:integer4 (*)) iwork)
             (type (double-float) bbnrm abnrm)
             (type (array double-float (*)) rwork rcondv rconde rscale lscale)
             (type (array f2cl-lib:complex16 (*)) work vr vl beta alpha b a)
             (type (f2cl-lib:integer4) info lwork ihi ilo ldvr ldvl ldb$ lda n)
             (type (string *) sense jobvr jobvl balanc))
```

```
(f2cl-lib:with-multi-array-data
    ((balanc character balanc-%data% balanc-%offset%)
     (jobvl character jobvl-%data% jobvl-%offset%)
     (jobvr character jobvr-%data% jobvr-%offset%)
     (sense character sense-%data% sense-%offset%)
     (a f2cl-lib:complex16 a-%data% a-%offset%)
     (b f2cl-lib:complex16 b-%data% b-%offset%)
     (alpha f2cl-lib:complex16 alpha-%data% alpha-%offset%)
     (beta f2cl-lib:complex16 beta-%data% beta-%offset%)
     (vl f2cl-lib:complex16 vl-%data% vl-%offset%)
     (vr f2cl-lib:complex16 vr-%data% vr-%offset%)
     (work f2cl-lib:complex16 work-%data% work-%offset%)
     (lscale double-float lscale-%data% lscale-%offset%)
     (rscale double-float rscale-%data% rscale-%offset%)
     (rconde double-float rconde-%data% rconde-%offset%)
     (rcondv double-float rcondv-%data% rcondv-%offset%)
     (rwork double-float rwork-%data% rwork-%offset%)
     (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%)
     (bwork f2cl-lib:logical bwork-%data% bwork-%offset%))
  (labels ((abs1 (x)
             (+ (abs (f2cl-lib:dble x)) (abs (f2cl-lib:dimag x)))))
    (declare
     (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
      abs1))
    (prog ((ldumma (make-array 1 :element-type 't)) (x #C(0.0d0 0.0d0))
           (anrm 0.0d0) (anrmto 0.0d0) (bignum 0.0d0) (bnrm 0.0d0)
           (bnrmto 0.0d0) (eps 0.0d0) (smlnum 0.0d0) (temp 0.0d0) (i 0)
           (icols 0) (ierr 0) (ijobvl 0) (ijobvr 0) (in 0) (irows 0)
           (itau 0) (iwrk 0) (iwrk1 0) (j 0) (jc 0) (jr 0) (m 0) (maxwrk 0)
           (minwrk 0)
           (chtemp
            (make-array '(1) :element-type 'character :initial-element
                        #\ ))
           (ilascl nil) (ilbscl nil) (ilv nil) (ilvl nil) (ilvr nil)
           (lquery nil) (noscl nil) (wantsb nil) (wantse nil) (wantsn nil)
           (wantsv nil))
      (declare (type (array f2cl-lib:logical (1)) ldumma)
               (type (f2cl-lib:complex16) x)
               (type (double-float) anrm anrmto bignum bnrm bnrmto eps
                smlnum temp)
               (type (f2cl-lib:integer4) i icols ierr ijobvl ijobvr in
                irows itau iwrk iwrk1 j jc jr m maxwrk minwrk)
               (type (string 1) chtemp)
               (type f2cl-lib:logical ilascl ilbscl ilv ilvl ilvr lquery
                noscl wantsb wantse wantsn wantsv))
      (cond
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvl "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvl var-0))
```

```
    ret-val)
  (setf ijobvl 1) (setf ilvl f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvl "V")
    (declare (ignore var-1))
    (when var-0 (setf jobvl var-0))
    ret-val)
  (setf ijobvl 2) (setf ilvl f2cl-lib:%true%))
 (t (setf ijobvl -1) (setf ilvl f2cl-lib:%false%)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvr "N")
    (declare (ignore var-1))
    (when var-0 (setf jobvr var-0))
    ret-val)
  (setf ijobvr 1) (setf ilvr f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame jobvr "V")
    (declare (ignore var-1))
    (when var-0 (setf jobvr var-0))
    ret-val)
  (setf ijobvr 2) (setf ilvr f2cl-lib:%true%))
 (t (setf ijobvr -1) (setf ilvr f2cl-lib:%false%)))
(setf ilv (or ilvl ilvr))
(setf noscl
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame balanc "N")
           (declare (ignore var-1))
           (when var-0 (setf balanc var-0))
           ret-val)
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame balanc "P")
           (declare (ignore var-1))
           (when var-0 (setf balanc var-0))
           ret-val)))
(setf wantsn
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "N")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wantse
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "E")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wantsv
        (multiple-value-bind (ret-val var-0 var-1)
```

```
            (lsame sense "V")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wantsb
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "B")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf info 0)
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond
 ((not
   (or noscl
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame balanc "S")
         (declare (ignore var-1))
         (when var-0 (setf balanc var-0))
         ret-val)
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame balanc "B")
         (declare (ignore var-1))
         (when var-0 (setf balanc var-0))
         ret-val)))
  (setf info -1))
 ((<= ijobvl 0) (setf info -2)) ((<= ijobvr 0) (setf info -3))
 ((not (or wantsn wantse wantsb wantsv)) (setf info -4))
 ((< n 0) (setf info -5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9))
 ((or (< ldvl 1) (and ilvl (< ldvl n))) (setf info -13))
 ((or (< ldvr 1) (and ilvr (< ldvr n))) (setf info -15)))
(cond
 ((= info 0)
  (cond ((= n 0) (setf minwrk 1) (setf maxwrk 1))
        (t (setf minwrk (f2cl-lib:int-mul 2 n))
         (cond (wantse (setf minwrk (f2cl-lib:int-mul 4 n)))
               ((or wantsv wantsb)
                (setf minwrk
                        (f2cl-lib:int-mul 2 n
                                          (f2cl-lib:int-add n 1)))))
         (setf maxwrk minwrk)
         (setf maxwrk
                 (max (the f2cl-lib:integer4 maxwrk)
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add n
                                             (f2cl-lib:int-mul n
```

```
                                                                    (multiple-value-bind
                                                                        (ret-val
                                                                         var-0
                                                                         var-1
                                                                         var-2
                                                                         var-3
                                                                         var-4
                                                                         var-5
                                                                         var-6)
                                                                        (ilaenv
                                                                         1
                                                                         "ZGEQRF"
                                                                         " "
                                                                         n
                                                                         1
                                                                         n
                                                                         0)
                                                                      (declare
                                                                       (ignore
                                                                        var-0
                                                                        var-1
                                                                        var-2
                                                                        var-4
                                                                        var-6))
                                                                      (setf n
                                                                              var-3)
                                                                      (setf n
                                                                              var-5)
                                                                      ret-val))))))
                                  (setf maxwrk
                                          (max (the f2cl-lib:integer4 maxwrk)
                                               (the f2cl-lib:integer4
                                                    (f2cl-lib:int-add n
                                                                      (f2cl-lib:int-mul n
                                                                    (multiple-value-bind
                                                                        (ret-val
                                                                         var-0
                                                                         var-1
                                                                         var-2
                                                                         var-3
                                                                         var-4
                                                                         var-5
                                                                         var-6)
                                                                        (ilaenv
                                                                         1
                                                                         "ZUNMQR"
                                                                         " "
                                                                         n
                                                                         1
                                                                         n
```

```
                                                                  0)
                                                                (declare
                                                                 (ignore
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-4
                                                                  var-6))
                                                                (setf n
                                                                        var-3)
                                                                (setf n
                                                                        var-5)
                                                                ret-val))))))
                    (cond
                     (ilvl
                      (setf maxwrk
                              (max (the f2cl-lib:integer4 maxwrk)
                                   (the f2cl-lib:integer4
                                        (f2cl-lib:int-add n
                                                          (f2cl-lib:int-mul n
                                                                            (multiple-value-bind
                                                                                (ret-val
                                                                                 var-0
                                                                                 var-1
                                                                                 var-2
                                                                                 var-3
                                                                                 var-4
                                                                                 var-5
                                                                                 var-6)
                                                                                (ilaenv
                                                                                 1
                                                                                 "ZUNGQR"
                                                                                 " "
                                                                                 n
                                                                                 1
                                                                                 n
                                                                                 0)
                                                                              (declare
                                                                               (ignore
                                                                                var-0
                                                                                var-1
                                                                                var-2
                                                                                var-4
                                                                                var-6))
                                                                              (setf n
                                                                                      var-3)
                                                                              (setf n
                                                                                      var-5)
                                                                              ret-val))))))))))
            (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
```

```
          (coerce maxwrk 'f2cl-lib:complex16))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -25)))))
(cond
 ((/= info 0) (xerbla "ZGGEVX" (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(setf eps (dlamch "P"))
(setf smlnum (dlamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "M" n n a lda rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf ilascl f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf anrmto smlnum)
  (setf ilascl f2cl-lib:%true%))
 ((> anrm bignum) (setf anrmto bignum)
  (setf ilascl f2cl-lib:%true%)))
(if ilascl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (zlascl "G" 0 0 anrm anrmto n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf anrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(setf bnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "M" n n b ldb$ rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
```

```
          (when var-2 (setf n var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf ilbscl f2cl-lib:%false%)
(cond
 ((and (> bnrm zero) (< bnrm smlnum)) (setf bnrmto smlnum)
  (setf ilbscl f2cl-lib:%true%))
 ((> bnrm bignum) (setf bnrmto bignum)
  (setf ilbscl f2cl-lib:%true%)))
(if ilbscl
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (zlascl "G" 0 0 bnrm bnrmto n n b ldb$ ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf bnrm var-3))
      (when var-4 (setf bnrmto var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-9 (setf ierr var-9))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (zggbal balanc n a lda b ldb$ ilo ihi lscale rscale rwork ierr)
  (declare (ignore var-1 var-2 var-4 var-8 var-9 var-10))
  (setf balanc var-0)
  (setf lda var-3)
  (setf ldb$ var-5)
  (setf ilo var-6)
  (setf ihi var-7)
  (setf ierr var-11))
(setf abnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "1" n n a lda
             (f2cl-lib:array-slice rwork-%data% double-float (1)
                                   ((1 *)) rwork-%offset%))
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(cond
 (ilascl
  (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
          abnrm)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 anrmto anrm 1 1
       (f2cl-lib:array-slice rwork-%data% double-float (1) ((1 *))
```

```
                              rwork-%offset%)
       1 ierr)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
    (when var-3 (setf anrmto var-3))
    (when var-4 (setf anrm var-4))
    (when var-9 (setf ierr var-9)))
  (setf abnrm
          (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%))))
(setf bbnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "1" n n b ldb$
             (f2cl-lib:array-slice rwork-%data% double-float (1)
                                   ((1 *)) rwork-%offset%))
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(cond
 (ilbscl
  (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
          bbnrm)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 bnrmto bnrm 1 1
       (f2cl-lib:array-slice rwork-%data% double-float (1) ((1 *))
                              rwork-%offset%)
       1 ierr)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
    (when var-3 (setf bnrmto var-3))
    (when var-4 (setf bnrm var-4))
    (when var-9 (setf ierr var-9)))
  (setf bbnrm
          (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%))))
(setf irows (f2cl-lib:int-sub (f2cl-lib:int-add ihi 1) ilo))
(cond
 ((or ilv (not wantsn))
  (setf icols (f2cl-lib:int-sub (f2cl-lib:int-add n 1) ilo)))
 (t (setf icols irows)))
(setf itau 1)
(setf iwrk (f2cl-lib:int-add itau irows))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgeqrf irows icols
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                           ((1 *)) work-%offset%)
```

```
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf irows var-0)
  (setf icols var-1)
  (setf ldb$ var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (zunmqr "L" "C" irows icols irows
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (ilo ilo)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (ilo ilo)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf irows var-2))
  (when var-3 (setf icols var-3))
  (when var-4 (setf irows var-4))
  (when var-6 (setf ldb$ var-6))
  (when var-9 (setf lda var-9))
  (when var-12 (setf ierr var-12)))
(cond
 (ilvl
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "Full" n n czero cone vl ldvl)
    (declare (ignore var-0 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf cone var-4))
    (when var-6 (setf ldvl var-6)))
  (cond
   ((> irows 1)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlacpy "L" (f2cl-lib:int-sub irows 1)
         (f2cl-lib:int-sub irows 1)
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                               ((+ ilo 1) ilo) ((1 ldb$) (1 *))
                               b-%offset%)
         ldb$
         (f2cl-lib:array-slice vl-%data% f2cl-lib:complex16
```

```
                                  ((+ ilo 1) ilo) ((1 ldvl) (1 *))
                                  vl-%offset%)
            ldvl)
        (declare (ignore var-0 var-1 var-2 var-3 var-5))
        (when var-4 (setf ldb$ var-4))
        (when var-6 (setf ldvl var-6)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (zungqr irows irows irows
         (f2cl-lib:array-slice vl-%data% f2cl-lib:complex16 (ilo ilo)
                               ((1 ldvl) (1 *)) vl-%offset%)
         ldvl
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) ierr)
      (declare (ignore var-3 var-5 var-6 var-7))
      (when var-0 (setf irows var-0))
      (when var-1 (setf irows var-1))
      (when var-2 (setf irows var-2))
      (when var-4 (setf ldvl var-4))
      (when var-8 (setf ierr var-8)))))
  (if ilvr
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlaset "Full" n n czero cone vr ldvr)
        (declare (ignore var-0 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf czero var-3))
        (when var-4 (setf cone var-4))
        (when var-6 (setf ldvr var-6))))
  (cond
   ((or ilv (not wantsn))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (zgghrd jobvl jobvr n ilo ihi a lda b ldb$ vl ldvl vr ldvr
         ierr)
      (declare (ignore var-5 var-7 var-9 var-11))
      (when var-0 (setf jobvl var-0))
      (when var-1 (setf jobvr var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf ilo var-3))
      (when var-4 (setf ihi var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-10 (setf ldvl var-10))
      (when var-12 (setf ldvr var-12))
      (when var-13 (setf ierr var-13))))
```

```
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (zgghrd "N" "N" irows 1 irows
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (ilo ilo)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (ilo ilo)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$ vl ldvl vr ldvr ierr)
    (declare (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11))
    (when var-2 (setf irows var-2))
    (when var-4 (setf irows var-4))
    (when var-6 (setf lda var-6))
    (when var-8 (setf ldb$ var-8))
    (when var-10 (setf ldvl var-10))
    (when var-12 (setf ldvr var-12))
    (when var-13 (setf ierr var-13)))))
(setf iwrk itau)
(cond
 ((or ilv (not wantsn))
  (f2cl-lib:f2cl-set-string chtemp "S" (string 1)))
 (t (f2cl-lib:f2cl-set-string chtemp "E" (string 1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18
     var-19)
    (zhgeqz chtemp jobvl jobvr n ilo ihi a lda b ldb$ alpha beta vl
     ldvl vr ldvr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-sub (f2cl-lib:int-add lwork 1) iwrk) rwork ierr)
  (declare
   (ignore var-6 var-8 var-10 var-11 var-12 var-14 var-16 var-17
    var-18))
  (when var-0 (setf chtemp var-0))
  (when var-1 (setf jobvl var-1))
  (when var-2 (setf jobvr var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf ilo var-4))
  (when var-5 (setf ihi var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldb$ var-9))
  (when var-13 (setf ldvl var-13))
  (when var-15 (setf ldvr var-15))
  (when var-19 (setf ierr var-19)))
(cond
 ((/= ierr 0)
  (cond ((and (> ierr 0) (<= ierr n)) (setf info ierr))
```

```
        ((and (> ierr n) (<= ierr (f2cl-lib:int-mul 2 n)))
         (setf info (f2cl-lib:int-sub ierr n)))
        (t (setf info (f2cl-lib:int-add n 1))))
  (go label90)))
(cond
 ((or ilv (not wantsn))
  (cond
   (ilv
    (cond
     (ilvl
      (cond (ilvr (f2cl-lib:f2cl-set-string chtemp "B" (string 1)))
            (t (f2cl-lib:f2cl-set-string chtemp "L" (string 1)))))
     (t (f2cl-lib:f2cl-set-string chtemp "R" (string 1))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14 var-15 var-16)
        (ztgevc chtemp "B" ldumma n a lda b ldb$ vl ldvl vr ldvr n in
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                               ((1 *)) work-%offset%)
         rwork ierr)
      (declare
       (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-14 var-15))
      (when var-0 (setf chtemp var-0))
      (when var-3 (setf n var-3))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldb$ var-7))
      (when var-9 (setf ldvl var-9))
      (when var-11 (setf ldvr var-11))
      (when var-12 (setf n var-12))
      (when var-13 (setf in var-13))
      (when var-16 (setf ierr var-16)))
    (cond
     ((/= ierr 0) (setf info (f2cl-lib:int-add n 2)) (go label90)))))
  (cond
   ((not wantsn)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf (f2cl-lib:fref bwork-%data% (j) ((1 *))
                                 bwork-%offset%)
                  f2cl-lib:%false%)
          label10))
        (setf (f2cl-lib:fref bwork-%data% (i) ((1 *)) bwork-%offset%)
              f2cl-lib:%true%)
        (setf iwrk (f2cl-lib:int-add n 1))
        (setf iwrk1 (f2cl-lib:int-add iwrk n))
        (cond
```

```
 ((or wantse wantsb)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13 var-14 var-15
       var-16)
      (ztgevc "B" "S" bwork n a lda b ldb$
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (1) ((1 *)) work-%offset%)
       n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (iwrk) ((1 *)) work-%offset%)
       n 1 m
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (iwrk1) ((1 *)) work-%offset%)
       rwork ierr)
    (declare
     (ignore var-0 var-1 var-2 var-4 var-6 var-8 var-10
      var-12 var-14 var-15))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldb$ var-7))
    (when var-9 (setf n var-9))
    (when var-11 (setf n var-11))
    (when var-13 (setf m var-13))
    (when var-16 (setf ierr var-16)))
  (cond
   ((/= ierr 0) (setf info (f2cl-lib:int-add n 2))
    (go label90)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17 var-18 var-19)
    (ztgsna sense "S" bwork n a lda b ldb$
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (1)
                           ((1 *)) work-%offset%)
     n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwrk) ((1 *)) work-%offset%)
     n
     (f2cl-lib:array-slice rconde-%data% double-float (i)
                           ((1 *)) rconde-%offset%)
     (f2cl-lib:array-slice rcondv-%data% double-float (i)
                           ((1 *)) rcondv-%offset%)
     1 m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwrk1) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk1) 1)
     iwork ierr)
  (declare
   (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-12 var-13
```

```
              var-14 var-16 var-17 var-18))
            (when var-0 (setf sense var-0))
            (when var-3 (setf n var-3))
            (when var-5 (setf lda var-5))
            (when var-7 (setf ldb$ var-7))
            (when var-9 (setf n var-9))
            (when var-11 (setf n var-11))
            (when var-15 (setf m var-15))
            (when var-19 (setf ierr var-19)))
        label20))))))
(cond
 (ilvl
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (zggbak balanc "L" n ilo ihi lscale rscale n vl ldvl ierr)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-8))
    (setf balanc var-0)
    (setf n var-7)
    (setf ldvl var-9)
    (setf ierr var-10))
  (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                ((> jc n) nil)
    (tagbody
      (setf temp zero)
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf temp
                  (max temp
                       (abs1
                        (f2cl-lib:fref vl-%data% (jr jc)
                                       ((1 ldvl) (1 *))
                                       vl-%offset%))))
         label30))
      (if (< temp smlnum)
          (go label50))
      (setf temp (/ one temp))
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                               vl-%offset%)
                  (*
                   (f2cl-lib:fref vl-%data% (jr jc) ((1 ldvl) (1 *))
                                  vl-%offset%)
                   temp))
         label40))
     label50))))
(cond
```

```
  (ilvr
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (zggbak balanc "R" n ilo ihi lscale rscale n vr ldvr ierr)
     (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-8))
     (setf balanc var-0)
     (setf n var-7)
     (setf ldvr var-9)
     (setf ierr var-10))
   (f2cl-lib:fdo (jc 1 (f2cl-lib:int-add jc 1))
                 ((> jc n) nil)
     (tagbody
       (setf temp zero)
       (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                     ((> jr n) nil)
         (tagbody
           (setf temp
                   (max temp
                        (abs1
                         (f2cl-lib:fref vr-%data% (jr jc)
                                        ((1 ldvr) (1 *))
                                        vr-%offset%))))
          label60))
       (if (< temp smlnum)
           (go label80))
       (setf temp (/ one temp))
       (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                     ((> jr n) nil)
         (tagbody
           (setf (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                vr-%offset%)
                   (*
                    (f2cl-lib:fref vr-%data% (jr jc) ((1 ldvr) (1 *))
                                   vr-%offset%)
                    temp))
          label70))
      label80))))
label90
 (if ilascl
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (zlascl "G" 0 0 anrmto anrm n 1 alpha n ierr)
       (declare (ignore var-0 var-1 var-2 var-6 var-7))
       (when var-3 (setf anrmto var-3))
       (when var-4 (setf anrm var-4))
       (when var-5 (setf n var-5))
       (when var-8 (setf n var-8))
       (when var-9 (setf ierr var-9))))
 (if ilbscl
```

```
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (zlascl "G" 0 0 bnrmto bnrm n 1 beta n ierr)
                (declare (ignore var-0 var-1 var-2 var-6 var-7))
                (when var-3 (setf bnrmto var-3))
                (when var-4 (setf bnrm var-4))
                (when var-5 (setf n var-5))
                (when var-8 (setf n var-8))
                (when var-9 (setf ierr var-9))))
           (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                   (coerce maxwrk 'f2cl-lib:complex16))
           (go end_label)
          end_label
           (return
            (values balanc jobvl jobvr sense n nil lda nil ldb$ nil nil nil ldvl
                    nil ldvr ilo ihi nil nil abnrm bbnrm nil nil nil nil nil nil
                    nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zggevx
                 fortran-to-lisp::*f2cl-function-info*)
           (fortran-to-lisp::make-f2cl-finfo :arg-types
                                             '((string) (string) (string)
                                               (string)
                                               (fortran-to-lisp::integer4)
                                               (array fortran-to-lisp::complex16
                                                (*))
                                               (fortran-to-lisp::integer4)
                                               (array fortran-to-lisp::complex16
                                                (*))
                                               (fortran-to-lisp::integer4)
                                               (array fortran-to-lisp::complex16
                                                (*))
                                               (array fortran-to-lisp::complex16
                                                (*))
                                               (array fortran-to-lisp::complex16
                                                (*))
                                               (fortran-to-lisp::integer4)
                                               (array fortran-to-lisp::complex16
                                                (*))
                                               (fortran-to-lisp::integer4)
                                               (fortran-to-lisp::integer4)
                                               (fortran-to-lisp::integer4)
                                               (array double-float (*))
                                               (array double-float (*))
                                               (double-float) (double-float)
                                               (array double-float (*))
                                               (array double-float (*))
```

```
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (array fortran-to-lisp::logical
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::balanc
  fortran-to-lisp::jobvl
  fortran-to-lisp::jobvr
  fortran-to-lisp::sense
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$ nil nil nil
  fortran-to-lisp::ldvl nil
  fortran-to-lisp::ldvr
  fortran-to-lisp::ilo
  fortran-to-lisp::ihi nil nil
  fortran-to-lisp::abnrm
  fortran-to-lisp::bbnrm nil nil
  nil nil nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zggbak
  fortran-to-lisp::zgeqrf
  fortran-to-lisp::zggbal
  fortran-to-lisp::xerbla
  fortran-to-lisp::ilaenv))))
```

———-

Chapter 28

LAPACK - General Matrices, Singular Value, Double

28.1 dgejsv

28.1.1 Axiom unit tests

— dgejsv.input —

```
)set break resume
)sys rm -f dgejsv.output
)spool dgejsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

28.1.2 Axiom help page

— dgejsv.help —

```
==========================================================================
dgejsv examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgejsv (JOBA, JOBU, JOBV, JOBR, JOBT, JOBP, M, N, A, LDA,
           SVA, U, LDU, V, LDV, WORK, LWORK, IWORK, INFO)
           DGEJSV

Function/Subroutine Documentation
   subroutine dgejsv (character*1 JOBA, character*1 JOBU, character*1 JOBV,
       character*1 JOBR, character*1 JOBT, character*1 JOBP, integer M,
       integer N, double precision, dimension( lda, * ) A, integer LDA, double
       precision, dimension( n ) SVA, double precision, dimension( ldu, * ) U,
       integer LDU, double precision, dimension( ldv, * ) V, integer LDV,
       double precision, dimension( lwork ) WORK, integer LWORK, integer,
       dimension( * ) IWORK, integer INFO)
       DGEJSV

Purpose:
   DGEJSV computes the singular value decomposition (SVD) of a real M-by-N
   matrix [A], where M >= N. The SVD of [A] is written as

         [A] = [U] * [SIGMA] * [V]^t,

   where [SIGMA] is an N-by-N (M-by-N) matrix which is zero except for its N
   diagonal elements, [U] is an M-by-N (or M-by-M) orthonormal matrix, and
   [V] is an N-by-N orthogonal matrix. The diagonal elements of [SIGMA] are
   the singular values of [A]. The columns of [U] and [V] are the left and
   the right singular vectors of [A], respectively. The matrices [U] and [V]
   are computed and stored in the arrays U and V, respectively. The diagonal
   of [SIGMA] is computed and stored in the array SVA.
   DGEJSV can sometimes compute tiny singular values and their singular
   vectors much more accurately than other SVD routines,
   see below under Further Details.*>

Parameters:

 JOBA
   JOBA is CHARACTER*1
   Specifies the level of accuracy:
   = 'C': This option works well (high relative accuracy) if A = B * D,
          with well-conditioned B and arbitrary diagonal matrix D.
          The accuracy cannot be spoiled by COLUMN scaling. The
          accuracy of the computed output depends on the condition of
          B, and the procedure aims at the best theoretical accuracy.
          The relative error max_{i=1:N}|d sigma_i| / sigma_i is
          bounded by f(M,N)*epsilon* cond(B), independent of D.
```

```
         The input matrix is preprocessed with the QRF with column
         pivoting. This initial preprocessing and preconditioning by
         a rank revealing QR factorization is common for all values of
         JOBA. Additional actions are specified as follows:
  = 'E': Computation as with 'C' with an additional estimate of the
         condition number of B. It provides a realistic error bound.
  = 'F': If A = D1 * C * D2 with ill-conditioned diagonal scalings
         D1, D2, and well-conditioned matrix C, this option gives
         higher accuracy than the 'C' option. If the structure of the
         input matrix is not known, and relative accuracy is
         desirable, then this option is advisable. The input matrix A
         is preprocessed with QR factorization with FULL (row and
         column) pivoting.
  = 'G'  Computation as with 'F' with an additional estimate of the
         condition number of B, where A=D*B. If A has heavily weighted
         rows, then using this condition number gives too pessimistic
         error bound.
  = 'A': Small singular values are the noise and the matrix is treated
         as numerically rank defficient. The error in the computed
         singular values is bounded by f(m,n)*epsilon*||A||.
         The computed SVD A = U * S * V^t restores A up to
         f(m,n)*epsilon*||A||.
         This gives the procedure the licence to discard (set to zero)
         all singular values below N*epsilon*||A||.
  = 'R': Similar as in 'A'. Rank revealing property of the initial
         QR factorization is used do reveal (using triangular factor)
         a gap sigma_{r+1} < epsilon * sigma_r in which case the
         numerical RANK is declared to be r. The SVD is computed with
         absolute error bounds, but more accurately than with 'A'.

JOBU
  JOBU is CHARACTER*1
  Specifies whether to compute the columns of U:
  = 'U': N columns of U are returned in the array U.
  = 'F': full set of M left sing. vectors is returned in the array U.
  = 'W': U may be used as workspace of length M*N. See the description of U.
  = 'N': U is not computed.

JOBV
  JOBV is CHARACTER*1
  Specifies whether to compute the matrix V:
  = 'V': N columns of V are returned in the array V; Jacobi rotations
         are not explicitly accumulated.
  = 'J': N columns of V are returned in the array V, but they are
         computed as the product of Jacobi rotations. This option is
         allowed only if JOBU .NE. 'N', i.e. in computing the full SVD.
  = 'W': V may be used as workspace of length N*N. See the description of V.
  = 'N': V is not computed.

JOBR
```

```
  JOBR is CHARACTER*1
  Specifies the RANGE for the singular values. Issues the licence to
  set to zero small positive singular values if they are outside
  specified range. If A .NE. 0 is scaled so that the largest singular
  value of c*A is around DSQRT(BIG), BIG=SLAMCH('O'), then JOBR issues
  the licence to kill columns of A whose norm in c*A is less than
  DSQRT(SFMIN) (for JOBR.EQ.'R'), or less than SMALL=SFMIN/EPSLN,
  where SFMIN=SLAMCH('S'), EPSLN=SLAMCH('E').
  = 'N': Do not kill small columns of c*A. This option assumes that
         BLAS and QR factorizations and triangular solvers are
         implemented to work in that range. If the condition of A
         is greater than BIG, use DGESVJ.
  = 'R': RESTRICTED range for sigma(c*A) is [DSQRT(SFMIN), DSQRT(BIG)]
         (roughly, as described above). This option is recommended.
                         ~~~~~~~~~~~~~~~~~~~~~~~~~~~
         For computing the singular values in the FULL range [SFMIN,BIG]
         use DGESVJ.

JOBT
  JOBT is CHARACTER*1
  If the matrix is square then the procedure may determine to use
  transposed A if A^t seems to be better with respect to convergence.
  If the matrix is not square, JOBT is ignored. This is subject to
  changes in the future.
  The decision is based on two values of entropy over the adjoint
  orbit of A^t * A. See the descriptions of WORK(6) and WORK(7).
  = 'T': transpose if entropy test indicates possibly faster
  convergence of Jacobi process if A^t is taken as input. If A is
  replaced with A^t, then the row pivoting is included automatically.
  = 'N': do not speculate.
  This option can be used to compute only the singular values, or the
  full SVD (U, SIGMA and V). For only one set of singular vectors
  (U or V), the caller should provide both U and V, as one of the
  matrices is used as workspace if the matrix A is transposed.
  The implementer can easily remove this constraint and make the
  code more complicated. See the descriptions of U and V.

JOBP
  JOBP is CHARACTER*1
  Issues the licence to introduce structured perturbations to drown
  denormalized numbers. This licence should be active if the
  denormals are poorly implemented, causing slow computation,
  especially in cases of fast convergence (!). For details see [1,2].
  For the sake of simplicity, this perturbations are included only
  when the full SVD or only the singular values are requested. The
  implementer/user can easily add the perturbation for the cases of
  computing one set of singular vectors.
  = 'P': introduce perturbation
  = 'N': do not perturb
```

```
M
  M is INTEGER
  The number of rows of the input matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the input matrix A. M >= N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the M-by-N matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

SVA
  SVA is DOUBLE PRECISION array, dimension (N)
  On exit,
  - For WORK(1)/WORK(2) = ONE: The singular values of A. During the
    computation SVA contains Euclidean column norms of the
    iterated matrices in the array A.
  - For WORK(1) .NE. WORK(2): The singular values of A are
    (WORK(1)/WORK(2)) * SVA(1:N). This factored form is used if
    sigma_max(A) overflows or if small singular values have been
    saved from underflow by scaling the input matrix A.
  - If JOBR='R' then some of the singular values may be returned
    as exact zeros obtained by "set to zero" because they are
    below the numerical rank threshold or are denormalized numbers.

U
  U is DOUBLE PRECISION array, dimension ( LDU, N )
  If JOBU = 'U', then U contains on exit the M-by-N matrix of
     the left singular vectors.
  If JOBU = 'F', then U contains on exit the M-by-M matrix of
     the left singular vectors, including an ONB
     of the orthogonal complement of the Range(A).
  If JOBU = 'W'  .AND. (JOBV.EQ.'V' .AND. JOBT.EQ.'T' .AND. M.EQ.N),
     then U is used as workspace if the procedure
     replaces A with A^t. In that case, [V] is computed
     in U as left singular vectors of A^t and then
     copied back to the V array. This 'W' option is just
     a reminder to the caller that in this case U is
     reserved as workspace of length N*N.
  If JOBU = 'N'  U is not referenced.

LDU
  LDU is INTEGER
  The leading dimension of the array U,  LDU >= 1.
  IF  JOBU = 'U' or 'F' or 'W',  then LDU >= M.
```

```
V
  V is DOUBLE PRECISION array, dimension ( LDV, N )
  If JOBV = 'V', 'J' then V contains on exit the N-by-N matrix of
     the right singular vectors;
  If JOBV = 'W', AND (JOBU.EQ.'U' AND JOBT.EQ.'T' AND M.EQ.N),
     then V is used as workspace if the pprocedure
     replaces A with A^t. In that case, [U] is computed
     in V as right singular vectors of A^t and then
     copied back to the U array. This 'W' option is just
     a reminder to the caller that in this case V is
     reserved as workspace of length N*N.
  If JOBV = 'N'  V is not referenced.

LDV
  LDV is INTEGER
  The leading dimension of the array V,  LDV >= 1.
  If JOBV = 'V' or 'J' or 'W', then LDV >= N.

WORK
  WORK is DOUBLE PRECISION array, dimension at least LWORK.
  On exit, if N.GT.0 .AND. M.GT.0 (else not referenced),
  WORK(1) = SCALE = WORK(2) / WORK(1) is the scaling factor such
      that SCALE*SVA(1:N) are the computed singular values
      of A. (See the description of SVA().)
  WORK(2) = See the description of WORK(1).
  WORK(3) = SCONDA is an estimate for the condition number of
      column equilibrated A. (If JOBA .EQ. 'E' or 'G')
      SCONDA is an estimate of DSQRT(||(R^t * R)^(-1)||_1).
      It is computed using DPOCON. It holds
      N^(-1/4) * SCONDA <= ||R^(-1)||_2 <= N^(1/4) * SCONDA
      where R is the triangular factor from the QRF of A.
      However, if R is truncated and the numerical rank is
      determined to be strictly smaller than N, SCONDA is
      returned as -1, thus indicating that the smallest
      singular values might be lost.

  If full SVD is needed, the following two condition numbers are
  useful for the analysis of the algorithm. They are provied for
  a developer/implementer who is familiar with the details of
  the method.

  WORK(4) = an estimate of the scaled condition number of the
      triangular factor in the first QR factorization.
  WORK(5) = an estimate of the scaled condition number of the
      triangular factor in the second QR factorization.

  The following two parameters are computed if JOBT .EQ. 'T'.
  They are provided for a developer/implementer who is familiar
  with the details of the method.
```

```
  WORK(6) = the entropy of A^t*A :: this is the Shannon entropy
      of diag(A^t*A) / Trace(A^t*A) taken as point in the
      probability simplex.
  WORK(7) = the entropy of A*A^t.

LWORK
  LWORK is INTEGER
  Length of WORK to confirm proper allocation of work space.
  LWORK depends on the job:

  If only SIGMA is needed ( JOBU.EQ.'N', JOBV.EQ.'N' ) and
  -> .. no scaled condition estimate required (JOBE.EQ.'N'):
        LWORK >= max(2*M+N,4*N+1,7). This is the minimal requirement.
    ->> For optimal performance (blocked code) the optimal value
        is LWORK >= max(2*M+N,3*N+(N+1)*NB,7). Here NB is the optimal
        block size for DGEQP3 and DGEQRF.
        In general, optimal LWORK is computed as
        LWORK >= max(2*M+N,N+LWORK(DGEQP3),N+LWORK(DGEQRF), 7).
  -> .. an estimate of the scaled condition number of A is
        required (JOBA='E', 'G'). In this case, LWORK is the maximum
        of the above and N*N+4*N, i.e. LWORK >= max(2*M+N,N*N+4*N,7).
    ->> For optimal performance (blocked code) the optimal value
        is LWORK >= max(2*M+N,3*N+(N+1)*NB, N*N+4*N, 7).
        In general, the optimal length LWORK is computed as
        LWORK >= max(2*M+N,N+LWORK(DGEQP3),N+LWORK(DGEQRF),
                 N+N*N+LWORK(DPOCON),7).

  If SIGMA and the right singular vectors are needed (JOBV.EQ.'V'),
  -> the minimal requirement is LWORK >= max(2*M+N,4*N+1,7).
  -> For optimal performance, LWORK >= max(2*M+N,3*N+(N+1)*NB,7),
     where NB is the optimal block size for DGEQP3, DGEQRF, DGELQ,
     DORMLQ. In general, the optimal length LWORK is computed as
     LWORK >= max(2*M+N,N+LWORK(DGEQP3), N+LWORK(DPOCON),
              N+LWORK(DGELQ), 2*N+LWORK(DGEQRF), N+LWORK(DORMLQ)).

  If SIGMA and the left singular vectors are needed
  -> the minimal requirement is LWORK >= max(2*M+N,4*N+1,7).
  -> For optimal performance:
     if JOBU.EQ.'U' :: LWORK >= max(2*M+N,3*N+(N+1)*NB,7),
     if JOBU.EQ.'F' :: LWORK >= max(2*M+N,3*N+(N+1)*NB,N+M*NB,7),
     where NB is the optimal block size for DGEQP3, DGEQRF, DORMQR.
     In general, the optimal length LWORK is computed as
     LWORK >= max(2*M+N,N+LWORK(DGEQP3),N+LWORK(DPOCON),
                  2*N+LWORK(DGEQRF), N+LWORK(DORMQR)).
     Here LWORK(DORMQR) equals N*NB (for JOBU.EQ.'U') or
           M*NB (for JOBU.EQ.'F').

  If the full SVD is needed: (JOBU.EQ.'U' or JOBU.EQ.'F') and
  -> if JOBV.EQ.'V'
```

```
     the minimal requirement is LWORK >= max(2*M+N,6*N+2*N*N).
  -> if JOBV.EQ.'J' the minimal requirement is
     LWORK >= max(2*M+N, 4*N+N*N,2*N+N*N+6).
  -> For optimal performance, LWORK should be additionally
     larger than N+M*NB, where NB is the optimal block size for DORMQR.

IWORK
  IWORK is INTEGER array, dimension M+3*N.
  On exit,
  IWORK(1) = the numerical rank determined after the initial
             QR factorization with pivoting. See the descriptions
             of JOBA and JOBR.
  IWORK(2) = the number of the computed nonzero singular values
  IWORK(3) = if nonzero, a warning message:
             If IWORK(3).EQ.1 then some of the column norms of A
             were denormalized floats. The requested high accuracy
             is not warranted by the data.

INFO
  INFO is INTEGER
  < 0  : if INFO = -i, then the i-th argument had an illegal value.
  = 0 :  successfull exit;
  > 0 :  DGEJSV  did not converge in the maximal allowed number
         of sweeps. The computed values may be inaccurate.

Further Details:
  DGEJSV implements a preconditioned Jacobi SVD algorithm. It uses DGEQP3,
  DGEQRF, and DGELQF as preprocessors and preconditioners. Optionally, an
  additional row pivoting can be used as a preprocessor, which in some
  cases results in much higher accuracy. An example is matrix A with the
  structure A = D1 * C * D2, where D1, D2 are arbitrarily ill-conditioned
  diagonal matrices and C is well-conditioned matrix. In that case, complete
  pivoting in the first QR factorizations provides accuracy dependent on the
  condition number of C, and independent of D1, D2. Such higher accuracy is
  not completely understood theoretically, but it works well in practice.
  Further, if A can be written as A = B*D, with well-conditioned B and some
  diagonal D, then the high accuracy is guaranteed, both theoretically and
  in software, independent of D. For more details see [1], [2].
  The computational range for the singular values can be the full range
  ( UNDERFLOW,OVERFLOW ), provided that the machine arithmetic and the BLAS
  & LAPACK routines called by DGEJSV are implemented to work in that range.
  If that is not the case, then the restriction for safe computation with
  the singular values in the range of normalized IEEE numbers is that the
  spectral condition number kappa(A)=sigma_max(A)/sigma_min(A) does not
  overflow. This code (DGEJSV) is best used in this restricted range,
  meaning that singular values of magnitude below ||A||_2 / DLAMCH('O') are
  returned as zeros. See JOBR for details on this.
  Further, this implementation is somewhat slower than the one described
  in [1,2] due to replacement of some non-LAPACK components, and because
  the choice of some tuning parameters in the iterative part (DGESVJ) is
```

```
left to the implementer on a particular machine.
The rank revealing QR factorization (in this code: DGEQP3) should be
implemented as in [3]. We have a new version of DGEQP3 under development
that is more robust than the current one in LAPACK, with a cleaner cut in
rank defficient cases. It will be available in the SIGMA library [4].
If M is much larger than N, it is obvious that the inital QRF with
column pivoting can be preprocessed by the QRF without pivoting. That
well known trick is not used in DGEJSV because in some cases heavy row
weighting can be treated with complete pivoting. The overhead in cases
M much larger than N is then only due to pivoting, but the benefits in
terms of accuracy have prevailed. The implementer/user can incorporate
this extra QRF step easily. The implementer can also improve data movement
(matrix transpose, matrix copy, matrix transposed copy) - this
implementation of DGEJSV uses only the simplest, naive data movement.
```

———-

28.1.3 fortran code

— **dgejsv.f** —

```
*  ===================================================================
      SUBROUTINE DGEJSV( JOBA, JOBU, JOBV, JOBR, JOBT, JOBP,
     $                   M, N, A, LDA, SVA, U, LDU, V, LDV,
     $                   WORK, LWORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      IMPLICIT    NONE
      INTEGER     INFO, LDA, LDU, LDV, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A( LDA, * ), SVA( N ), U( LDU, * ), V( LDV, * ),
     $            WORK( LWORK )
      INTEGER     IWORK( * )
      CHARACTER*1 JOBA, JOBP, JOBR, JOBT, JOBU, JOBV
*     ..
*
*  ======================================================================
*
*     .. Local Parameters ..
      DOUBLE PRECISION   ZERO,  ONE
      PARAMETER ( ZERO = 0.0D0, ONE = 1.0D0 )
```

```
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION AAPP, AAQQ, AATMAX, AATMIN, BIG, BIG1, COND_OK,
     $        CONDR1, CONDR2, ENTRA,  ENTRAT, EPSLN,  MAXPRJ, SCALEM,
     $        SCONDA, SFMIN,  SMALL,  TEMP1,  USCAL1, USCAL2, XSC
      INTEGER IERR,   N1,     NR,     NUMRANK,        p, q,   WARNING
      LOGICAL ALMORT, DEFR,   ERREST, GOSCAL, JRACC,  KILL,   LSVEC,
     $        L2ABER, L2KILL, L2PERT, L2RANK, L2TRAN,
     $        NOSCAL, ROWPIV, RSVEC,  TRANSP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC DABS, DLOG, MAX, MIN, DBLE, IDNINT, DSIGN, DSQRT
*     ..
*     .. External Functions ..
      DOUBLE PRECISION  DLAMCH, DNRM2
      INTEGER   IDAMAX
      LOGICAL   LSAME
      EXTERNAL  IDAMAX, LSAME, DLAMCH, DNRM2
*     ..
*     .. External Subroutines ..
      EXTERNAL  DCOPY,  DGELQF, DGEQP3, DGEQRF, DLACPY, DLASCL,
     $          DLASET, DLASSQ, DLASWP, DORGQR, DORMLQ,
     $          DORMQR, DPOCON, DSCAL,  DSWAP,  DTRSM,  XERBLA
*
      EXTERNAL  DGESVJ
*     ..
*
*     Test the input arguments
*
      LSVEC  = LSAME( JOBU, 'U' ) .OR. LSAME( JOBU, 'F' )
      JRACC  = LSAME( JOBV, 'J' )
      RSVEC  = LSAME( JOBV, 'V' ) .OR. JRACC
      ROWPIV = LSAME( JOBA, 'F' ) .OR. LSAME( JOBA, 'G' )
      L2RANK = LSAME( JOBA, 'R' )
      L2ABER = LSAME( JOBA, 'A' )
      ERREST = LSAME( JOBA, 'E' ) .OR. LSAME( JOBA, 'G' )
      L2TRAN = LSAME( JOBT, 'T' )
      L2KILL = LSAME( JOBR, 'R' )
      DEFR   = LSAME( JOBR, 'N' )
      L2PERT = LSAME( JOBP, 'P' )
*
      IF ( .NOT.(ROWPIV .OR. L2RANK .OR. L2ABER .OR.
     $     ERREST .OR. LSAME( JOBA, 'C' ) )) THEN
         INFO = - 1
      ELSE IF ( .NOT.( LSVEC  .OR. LSAME( JOBU, 'N' ) .OR.
     $                             LSAME( JOBU, 'W' )) ) THEN
         INFO = - 2
      ELSE IF ( .NOT.( RSVEC .OR. LSAME( JOBV, 'N' ) .OR.
     $   LSAME( JOBV, 'W' )) .OR. ( JRACC .AND. (.NOT.LSVEC) ) ) THEN
         INFO = - 3
```

```
      ELSE IF ( .NOT. ( L2KILL .OR. DEFR ) )    THEN
         INFO = - 4
      ELSE IF ( .NOT. ( L2TRAN .OR. LSAME( JOBT, 'N' ) ) ) THEN
         INFO = - 5
      ELSE IF ( .NOT. ( L2PERT .OR. LSAME( JOBP, 'N' ) ) ) THEN
         INFO = - 6
      ELSE IF ( M .LT. 0 ) THEN
         INFO = - 7
      ELSE IF ( ( N .LT. 0 ) .OR. ( N .GT. M ) ) THEN
         INFO = - 8
      ELSE IF ( LDA .LT. M ) THEN
         INFO = - 10
      ELSE IF ( LSVEC .AND. ( LDU .LT. M ) ) THEN
         INFO = - 13
      ELSE IF ( RSVEC .AND. ( LDV .LT. N ) ) THEN
         INFO = - 14
      ELSE IF ( (.NOT.(LSVEC .OR. RSVEC .OR. ERREST).AND.
     &                           (LWORK .LT. MAX(7,4*N+1,2*M+N))) .OR.
     & (.NOT.(LSVEC .OR. RSVEC) .AND. ERREST .AND.
     &                         (LWORK .LT. MAX(7,4*N+N*N,2*M+N))) .OR.
     & (LSVEC .AND. (.NOT.RSVEC) .AND. (LWORK .LT. MAX(7,2*M+N,4*N+1)))
     & .OR.
     & (RSVEC .AND. (.NOT.LSVEC) .AND. (LWORK .LT. MAX(7,2*M+N,4*N+1)))
     & .OR.
     & (LSVEC .AND. RSVEC .AND. (.NOT.JRACC) .AND.
     &                          (LWORK.LT.MAX(2*M+N,6*N+2*N*N)))
     & .OR. (LSVEC .AND. RSVEC .AND. JRACC .AND.
     &                          LWORK.LT.MAX(2*M+N,4*N+N*N,2*N+N*N+6)))
     &   THEN
         INFO = - 17
      ELSE
*        #:)
         INFO = 0
      END IF
*
      IF ( INFO .NE. 0 ) THEN
*       #:(
         CALL XERBLA( 'DGEJSV', - INFO )
         RETURN
      END IF
*
*     Quick return for void matrix (Y3K safe)
* #:)
      IF ( ( M .EQ. 0 ) .OR. ( N .EQ. 0 ) ) RETURN
*
*     Determine whether the matrix U should be M x N or M x M
*
      IF ( LSVEC ) THEN
         N1 = N
         IF ( LSAME( JOBU, 'F' ) ) N1 = M
```

```
      END IF
*
*     Set numerical parameters
*
*!    NOTE: Make sure DLAMCH() does not fail on the target architecture.
*
      EPSLN = DLAMCH('Epsilon')
      SFMIN = DLAMCH('SafeMinimum')
      SMALL = SFMIN / EPSLN
      BIG   = DLAMCH('O')
*     BIG   = ONE / SFMIN
*
*     Initialize SVA(1:N) = diag( ||A e_i||_2 )_1^N
*
*(!)  If necessary, scale SVA() to protect the largest norm from
*     overflow. It is possible that this scaling pushes the smallest
*     column norm left from the underflow threshold (extreme case).
*
      SCALEM  = ONE / DSQRT(DBLE(M)*DBLE(N))
      NOSCAL  = .TRUE.
      GOSCAL  = .TRUE.
      DO 1874 p = 1, N
         AAPP = ZERO
         AAQQ = ONE
         CALL DLASSQ( M, A(1,p), 1, AAPP, AAQQ )
         IF ( AAPP .GT. BIG ) THEN
            INFO = - 9
            CALL XERBLA( 'DGEJSV', -INFO )
            RETURN
         END IF
         AAQQ = DSQRT(AAQQ)
         IF ( ( AAPP .LT. (BIG / AAQQ) ) .AND. NOSCAL  ) THEN
            SVA(p)  = AAPP * AAQQ
         ELSE
            NOSCAL  = .FALSE.
            SVA(p)  = AAPP * ( AAQQ * SCALEM )
            IF ( GOSCAL ) THEN
               GOSCAL = .FALSE.
               CALL DSCAL( p-1, SCALEM, SVA, 1 )
            END IF
         END IF
 1874 CONTINUE
*
      IF ( NOSCAL ) SCALEM = ONE
*
      AAPP = ZERO
      AAQQ = BIG
      DO 4781 p = 1, N
         AAPP = MAX( AAPP, SVA(p) )
         IF ( SVA(p) .NE. ZERO ) AAQQ = MIN( AAQQ, SVA(p) )
```

```
 4781 CONTINUE
*
*     Quick return for zero M x N matrix
* #:)
      IF ( AAPP .EQ. ZERO ) THEN
         IF ( LSVEC ) CALL DLASET( 'G', M, N1, ZERO, ONE, U, LDU )
         IF ( RSVEC ) CALL DLASET( 'G', N, N,  ZERO, ONE, V, LDV )
         WORK(1) = ONE
         WORK(2) = ONE
         IF ( ERREST ) WORK(3) = ONE
         IF ( LSVEC .AND. RSVEC ) THEN
            WORK(4) = ONE
            WORK(5) = ONE
         END IF
         IF ( L2TRAN ) THEN
            WORK(6) = ZERO
            WORK(7) = ZERO
         END IF
         IWORK(1) = 0
         IWORK(2) = 0
         IWORK(3) = 0
         RETURN
      END IF
*
*     Issue warning if denormalized column norms detected. Override the
*     high relative accuracy request. Issue licence to kill columns
*     (set them to zero) whose norm is less than sigma_max / BIG (roughly).
* #:(
      WARNING = 0
      IF ( AAQQ .LE. SFMIN ) THEN
         L2RANK = .TRUE.
         L2KILL = .TRUE.
         WARNING = 1
      END IF
*
*     Quick return for one-column matrix
* #:)
      IF ( N .EQ. 1 ) THEN
*
         IF ( LSVEC ) THEN
            CALL DLASCL( 'G',0,0,SVA(1),SCALEM, M,1,A(1,1),LDA,IERR )
            CALL DLACPY( 'A', M, 1, A, LDA, U, LDU )
*           computing all M left singular vectors of the M x 1 matrix
            IF ( N1 .NE. N  ) THEN
               CALL DGEQRF( M, N, U,LDU, WORK, WORK(N+1),LWORK-N,IERR )
               CALL DORGQR( M,N1,1, U,LDU,WORK,WORK(N+1),LWORK-N,IERR )
               CALL DCOPY( M, A(1,1), 1, U(1,1), 1 )
            END IF
         END IF
         IF ( RSVEC ) THEN
```

```
            V(1,1) = ONE
         END IF
         IF ( SVA(1) .LT. (BIG*SCALEM) ) THEN
            SVA(1)  = SVA(1) / SCALEM
            SCALEM  = ONE
         END IF
         WORK(1) = ONE / SCALEM
         WORK(2) = ONE
         IF ( SVA(1) .NE. ZERO ) THEN
            IWORK(1) = 1
            IF ( ( SVA(1) / SCALEM) .GE. SFMIN ) THEN
               IWORK(2) = 1
            ELSE
               IWORK(2) = 0
            END IF
         ELSE
            IWORK(1) = 0
            IWORK(2) = 0
         END IF
         IF ( ERREST ) WORK(3) = ONE
         IF ( LSVEC .AND. RSVEC ) THEN
            WORK(4) = ONE
            WORK(5) = ONE
         END IF
         IF ( L2TRAN ) THEN
            WORK(6) = ZERO
            WORK(7) = ZERO
         END IF
         RETURN
*
      END IF
*
      TRANSP = .FALSE.
      L2TRAN = L2TRAN .AND. ( M .EQ. N )
*
      AATMAX = -ONE
      AATMIN =  BIG
      IF ( ROWPIV .OR. L2TRAN ) THEN
*
*     Compute the row norms, needed to determine row pivoting sequence
*     (in the case of heavily row weighted A, row pivoting is strongly
*     advised) and to collect information needed to compare the
*     structures of A * A^t and A^t * A (in the case L2TRAN.EQ..TRUE.).
*
         IF ( L2TRAN ) THEN
            DO 1950 p = 1, M
               XSC   = ZERO
               TEMP1 = ONE
               CALL DLASSQ( N, A(p,1), LDA, XSC, TEMP1 )
*              DLASSQ gets both the ell_2 and the ell_infinity norm
```

```
*              in one pass through the vector
               WORK(M+N+p)  = XSC * SCALEM
               WORK(N+p)    = XSC * (SCALEM*DSQRT(TEMP1))
               AATMAX = MAX( AATMAX, WORK(N+p) )
               IF (WORK(N+p) .NE. ZERO) AATMIN = MIN(AATMIN,WORK(N+p))
 1950       CONTINUE
         ELSE
            DO 1904 p = 1, M
               WORK(M+N+p) = SCALEM*DABS( A(p,IDAMAX(N,A(p,1),LDA)) )
               AATMAX = MAX( AATMAX, WORK(M+N+p) )
               AATMIN = MIN( AATMIN, WORK(M+N+p) )
 1904       CONTINUE
         END IF
*
      END IF
*
*     For square matrix A try to determine whether A^t  would be  better
*     input for the preconditioned Jacobi SVD, with faster convergence.
*     The decision is based on an O(N) function of the vector of column
*     and row norms of A, based on the Shannon entropy. This should give
*     the right choice in most cases when the difference actually matters.
*     It may fail and pick the slower converging side.
*
      ENTRA  = ZERO
      ENTRAT = ZERO
      IF ( L2TRAN ) THEN
*
         XSC   = ZERO
         TEMP1 = ONE
         CALL DLASSQ( N, SVA, 1, XSC, TEMP1 )
         TEMP1 = ONE / TEMP1
*
         ENTRA = ZERO
         DO 1113 p = 1, N
            BIG1  = ( ( SVA(p) / XSC )**2 ) * TEMP1
            IF ( BIG1 .NE. ZERO ) ENTRA = ENTRA + BIG1 * DLOG(BIG1)
 1113    CONTINUE
         ENTRA = - ENTRA / DLOG(DBLE(N))
*
*        Now, SVA().^2/Trace(A^t * A) is a point in the probability simplex.
*        It is derived from the diagonal of  A^t * A.  Do the same with the
*        diagonal of A * A^t, compute the entropy of the corresponding
*        probability distribution. Note that A * A^t and A^t * A have the
*        same trace.
*
         ENTRAT = ZERO
         DO 1114 p = N+1, N+M
            BIG1 = ( ( WORK(p) / XSC )**2 ) * TEMP1
            IF ( BIG1 .NE. ZERO ) ENTRAT = ENTRAT + BIG1 * DLOG(BIG1)
 1114    CONTINUE
```

```
         ENTRAT = - ENTRAT / DLOG(DBLE(M))
*
*        Analyze the entropies and decide A or A^t. Smaller entropy
*        usually means better input for the algorithm.
*
         TRANSP = ( ENTRAT .LT. ENTRA )
*
*        If A^t is better than A, transpose A.
*
         IF ( TRANSP ) THEN
*           In an optimal implementation, this trivial transpose
*           should be replaced with faster transpose.
            DO 1115 p = 1, N - 1
               DO 1116 q = p + 1, N
                   TEMP1 = A(q,p)
                  A(q,p) = A(p,q)
                  A(p,q) = TEMP1
 1116          CONTINUE
 1115       CONTINUE
            DO 1117 p = 1, N
               WORK(M+N+p) = SVA(p)
               SVA(p)      = WORK(N+p)
 1117       CONTINUE
            TEMP1  = AAPP
            AAPP   = AATMAX
            AATMAX = TEMP1
            TEMP1  = AAQQ
            AAQQ   = AATMIN
            AATMIN = TEMP1
            KILL   = LSVEC
            LSVEC  = RSVEC
            RSVEC  = KILL
            IF ( LSVEC ) N1 = N
*
            ROWPIV = .TRUE.
         END IF
*
      END IF
*     END IF L2TRAN
*
*     Scale the matrix so that its maximal singular value remains less
*     than DSQRT(BIG) -- the matrix is scaled so that its maximal column
*     has Euclidean norm equal to DSQRT(BIG/N). The only reason to keep
*     DSQRT(BIG) instead of BIG is the fact that DGEJSV uses LAPACK and
*     BLAS routines that, in some implementations, are not capable of
*     working in the full interval [SFMIN,BIG] and that they may provoke
*     overflows in the intermediate results. If the singular values spread
*     from SFMIN to BIG, then DGESVJ will compute them. So, in that case,
*     one should use DGESVJ instead of DGEJSV.
*
```

```
      BIG1   = DSQRT( BIG )
      TEMP1  = DSQRT( BIG / DBLE(N) )
*
      CALL DLASCL( 'G', 0, 0, AAPP, TEMP1, N, 1, SVA, N, IERR )
      IF ( AAQQ .GT. (AAPP * SFMIN) ) THEN
          AAQQ = ( AAQQ / AAPP ) * TEMP1
      ELSE
          AAQQ = ( AAQQ * TEMP1 ) / AAPP
      END IF
      TEMP1 = TEMP1 * SCALEM
      CALL DLASCL( 'G', 0, 0, AAPP, TEMP1, M, N, A, LDA, IERR )
*
*     To undo scaling at the end of this procedure, multiply the
*     computed singular values with USCAL2 / USCAL1.
*
      USCAL1 = TEMP1
      USCAL2 = AAPP
*
      IF ( L2KILL ) THEN
*        L2KILL enforces computation of nonzero singular values in
*        the restricted range of condition number of the initial A,
*        sigma_max(A) / sigma_min(A) approx. DSQRT(BIG)/DSQRT(SFMIN).
         XSC = DSQRT( SFMIN )
      ELSE
         XSC = SMALL
*
*        Now, if the condition number of A is too big,
*        sigma_max(A) / sigma_min(A) .GT. DSQRT(BIG/N) * EPSLN / SFMIN,
*        as a precaution measure, the full SVD is computed using DGESVJ
*        with accumulated Jacobi rotations. This provides numerically
*        more robust computation, at the cost of slightly increased run
*        time. Depending on the concrete implementation of BLAS and LAPACK
*        (i.e. how they behave in presence of extreme ill-conditioning) the
*        implementor may decide to remove this switch.
         IF ( ( AAQQ.LT.DSQRT(SFMIN) ) .AND. LSVEC .AND. RSVEC ) THEN
            JRACC = .TRUE.
         END IF
*
      END IF
      IF ( AAQQ .LT. XSC ) THEN
         DO 700 p = 1, N
            IF ( SVA(p) .LT. XSC ) THEN
               CALL DLASET( 'A', M, 1, ZERO, ZERO, A(1,p), LDA )
               SVA(p) = ZERO
            END IF
 700     CONTINUE
      END IF
*
*     Preconditioning using QR factorization with pivoting
*
```

```
      IF ( ROWPIV ) THEN
*        Optional row permutation (Bjoerck row pivoting):
*        A result by Cox and Higham shows that the Bjoerck's
*        row pivoting combined with standard column pivoting
*        has similar effect as Powell-Reid complete pivoting.
*        The ell-infinity norms of A are made nonincreasing.
         DO 1952 p = 1, M - 1
            q = IDAMAX( M-p+1, WORK(M+N+p), 1 ) + p - 1
            IWORK(2*N+p) = q
            IF ( p .NE. q ) THEN
               TEMP1       = WORK(M+N+p)
               WORK(M+N+p) = WORK(M+N+q)
               WORK(M+N+q) = TEMP1
            END IF
 1952    CONTINUE
         CALL DLASWP( N, A, LDA, 1, M-1, IWORK(2*N+1), 1 )
      END IF
*
*     End of the preparation phase (scaling, optional sorting and
*     transposing, optional flushing of small columns).
*
*     Preconditioning
*
*     If the full SVD is needed, the right singular vectors are computed
*     from a matrix equation, and for that we need theoretical analysis
*     of the Businger-Golub pivoting. So we use DGEQP3 as the first RR QRF.
*     In all other cases the first RR QRF can be chosen by other criteria
*     (eg speed by replacing global with restricted window pivoting, such
*     as in SGEQPX from TOMS # 782). Good results will be obtained using
*     SGEQPX with properly (!) chosen numerical parameters.
*     Any improvement of DGEQP3 improves overal performance of DGEJSV.
*
*     A * P1 = Q1 * [ R1^t 0]^t:
      DO 1963 p = 1, N
*        .. all columns are free columns
         IWORK(p) = 0
 1963 CONTINUE
      CALL DGEQP3( M,N,A,LDA, IWORK,WORK, WORK(N+1),LWORK-N, IERR )
*
*     The upper triangular matrix R1 from the first QRF is inspected for
*     rank deficiency and possibilities for deflation, or possible
*     ill-conditioning. Depending on the user specified flag L2RANK,
*     the procedure explores possibilities to reduce the numerical
*     rank by inspecting the computed upper triangular factor. If
*     L2RANK or L2ABER are up, then DGEJSV will compute the SVD of
*     A + dA, where ||dA|| <= f(M,N)*EPSLN.
*
      NR = 1
      IF ( L2ABER ) THEN
*        Standard absolute error bound suffices. All sigma_i with
```

```
*        sigma_i < N*EPSLN*||A|| are flushed to zero. This is an
*        agressive enforcement of lower numerical rank by introducing a
*        backward error of the order of N*EPSLN*||A||.
         TEMP1 = DSQRT(DBLE(N))*EPSLN
         DO 3001 p = 2, N
            IF ( DABS(A(p,p)) .GE. (TEMP1*DABS(A(1,1))) ) THEN
               NR = NR + 1
            ELSE
               GO TO 3002
            END IF
 3001    CONTINUE
 3002    CONTINUE
      ELSE IF ( L2RANK ) THEN
*        .. similarly as above, only slightly more gentle (less agressive).
*        Sudden drop on the diagonal of R1 is used as the criterion for
*        close-to-rank-defficient.
         TEMP1 = DSQRT(SFMIN)
         DO 3401 p = 2, N
            IF ( ( DABS(A(p,p)) .LT. (EPSLN*DABS(A(p-1,p-1))) ) .OR.
     $           ( DABS(A(p,p)) .LT. SMALL ) .OR.
     $           ( L2KILL .AND. (DABS(A(p,p)) .LT. TEMP1) ) ) GO TO 3402
            NR = NR + 1
 3401    CONTINUE
 3402    CONTINUE
*
      ELSE
*        The goal is high relative accuracy. However, if the matrix
*        has high scaled condition number the relative accuracy is in
*        general not feasible. Later on, a condition number estimator
*        will be deployed to estimate the scaled condition number.
*        Here we just remove the underflowed part of the triangular
*        factor. This prevents the situation in which the code is
*        working hard to get the accuracy not warranted by the data.
         TEMP1  = DSQRT(SFMIN)
         DO 3301 p = 2, N
            IF ( ( DABS(A(p,p)) .LT. SMALL ) .OR.
     $          ( L2KILL .AND. (DABS(A(p,p)) .LT. TEMP1) ) ) GO TO 3302
            NR = NR + 1
 3301    CONTINUE
 3302    CONTINUE
*
      END IF
*
      ALMORT = .FALSE.
      IF ( NR .EQ. N ) THEN
         MAXPRJ = ONE
         DO 3051 p = 2, N
            TEMP1  = DABS(A(p,p)) / SVA(IWORK(p))
            MAXPRJ = MIN( MAXPRJ, TEMP1 )
 3051    CONTINUE
```

```
         IF ( MAXPRJ**2 .GE. ONE - DBLE(N)*EPSLN ) ALMORT = .TRUE.
      END IF
*
*
      SCONDA = - ONE
      CONDR1 = - ONE
      CONDR2 = - ONE
*
      IF ( ERREST ) THEN
         IF ( N .EQ. NR ) THEN
            IF ( RSVEC ) THEN
*              .. V is available as workspace
               CALL DLACPY( 'U', N, N, A, LDA, V, LDV )
               DO 3053 p = 1, N
                  TEMP1 = SVA(IWORK(p))
                  CALL DSCAL( p, ONE/TEMP1, V(1,p), 1 )
 3053          CONTINUE
               CALL DPOCON( 'U', N, V, LDV, ONE, TEMP1,
     $              WORK(N+1), IWORK(2*N+M+1), IERR )
            ELSE IF ( LSVEC ) THEN
*              .. U is available as workspace
               CALL DLACPY( 'U', N, N, A, LDA, U, LDU )
               DO 3054 p = 1, N
                  TEMP1 = SVA(IWORK(p))
                  CALL DSCAL( p, ONE/TEMP1, U(1,p), 1 )
 3054          CONTINUE
               CALL DPOCON( 'U', N, U, LDU, ONE, TEMP1,
     $              WORK(N+1), IWORK(2*N+M+1), IERR )
            ELSE
               CALL DLACPY( 'U', N, N, A, LDA, WORK(N+1), N )
               DO 3052 p = 1, N
                  TEMP1 = SVA(IWORK(p))
                  CALL DSCAL( p, ONE/TEMP1, WORK(N+(p-1)*N+1), 1 )
 3052          CONTINUE
*           .. the columns of R are scaled to have unit Euclidean lengths.
               CALL DPOCON( 'U', N, WORK(N+1), N, ONE, TEMP1,
     $              WORK(N+N*N+1), IWORK(2*N+M+1), IERR )
            END IF
            SCONDA = ONE / DSQRT(TEMP1)
*           SCONDA is an estimate of DSQRT(||(R^t * R)^(-1)||_1).
*           N^(-1/4) * SCONDA <= ||R^(-1)||_2 <= N^(1/4) * SCONDA
         ELSE
            SCONDA = - ONE
         END IF
      END IF
*
      L2PERT = L2PERT .AND. ( DABS( A(1,1)/A(NR,NR) ) .GT. DSQRT(BIG1) )
*     If there is no violent scaling, artificial perturbation is not needed.
*
*     Phase 3:
```

```
*
      IF ( .NOT. ( RSVEC .OR. LSVEC ) ) THEN
*
*         Singular Values only
*
*         .. transpose A(1:NR,1:N)
         DO 1946 p = 1, MIN( N-1, NR )
            CALL DCOPY( N-p, A(p,p+1), LDA, A(p+1,p), 1 )
 1946    CONTINUE
*
*        The following two DO-loops introduce small relative perturbation
*        into the strict upper triangle of the lower triangular matrix.
*        Small entries below the main diagonal are also changed.
*        This modification is useful if the computing environment does not
*        provide/allow FLUSH TO ZERO underflow, for it prevents many
*        annoying denormalized numbers in case of strongly scaled matrices.
*        The perturbation is structured so that it does not introduce any
*        new perturbation of the singular values, and it does not destroy
*        the job done by the preconditioner.
*        The licence for this perturbation is in the variable L2PERT, which
*        should be .FALSE. if FLUSH TO ZERO underflow is active.
*
         IF ( .NOT. ALMORT ) THEN
*
            IF ( L2PERT ) THEN
*              XSC = DSQRT(SMALL)
               XSC = EPSLN / DBLE(N)
               DO 4947 q = 1, NR
                  TEMP1 = XSC*DABS(A(q,q))
                  DO 4949 p = 1, N
                     IF ( ( (p.GT.q) .AND. (DABS(A(p,q)).LE.TEMP1) )
     $                    .OR. ( p .LT. q ) )
     $                     A(p,q) = DSIGN( TEMP1, A(p,q) )
 4949             CONTINUE
 4947          CONTINUE
            ELSE
               CALL DLASET( 'U', NR-1,NR-1, ZERO,ZERO, A(1,2),LDA )
            END IF
*
*            .. second preconditioning using the QR factorization
*
            CALL DGEQRF( N,NR, A,LDA, WORK, WORK(N+1),LWORK-N, IERR )
*
*           .. and transpose upper to lower triangular
            DO 1948 p = 1, NR - 1
               CALL DCOPY( NR-p, A(p,p+1), LDA, A(p+1,p), 1 )
 1948       CONTINUE
*
         END IF
*
```

```
*          Row-cyclic Jacobi SVD algorithm with column pivoting
*
*          .. again some perturbation (a "background noise") is added
*          to drown denormals
           IF ( L2PERT ) THEN
*             XSC = DSQRT(SMALL)
              XSC = EPSLN / DBLE(N)
              DO 1947 q = 1, NR
                 TEMP1 = XSC*DABS(A(q,q))
                 DO 1949 p = 1, NR
                    IF ( ( (p.GT.q) .AND. (DABS(A(p,q)).LE.TEMP1) )
     $                      .OR. ( p .LT. q ) )
     $                   A(p,q) = DSIGN( TEMP1, A(p,q) )
 1949            CONTINUE
 1947         CONTINUE
           ELSE
              CALL DLASET( 'U', NR-1, NR-1, ZERO, ZERO, A(1,2), LDA )
           END IF
*
*          .. and one-sided Jacobi rotations are started on a lower
*          triangular matrix (plus perturbation which is ignored in
*          the part which destroys triangular form (confusing?!))
*
           CALL DGESVJ( 'L', 'NoU', 'NoV', NR, NR, A, LDA, SVA,
     $                     N, V, LDV, WORK, LWORK, INFO )
*
           SCALEM  = WORK(1)
           NUMRANK = IDNINT(WORK(2))
*
*
      ELSE IF ( RSVEC .AND. ( .NOT. LSVEC ) ) THEN
*
*        -> Singular Values and Right Singular Vectors <-
*
         IF ( ALMORT ) THEN
*
*          .. in this case NR equals N
           DO 1998 p = 1, NR
              CALL DCOPY( N-p+1, A(p,p), LDA, V(p,p), 1 )
 1998      CONTINUE
           CALL DLASET( 'Upper', NR-1, NR-1, ZERO, ZERO, V(1,2), LDV )
*
           CALL DGESVJ( 'L','U','N', N, NR, V,LDV, SVA, NR, A,LDA,
     $                WORK, LWORK, INFO )
           SCALEM  = WORK(1)
           NUMRANK = IDNINT(WORK(2))

         ELSE
*
*        .. two more QR factorizations ( one QRF is not enough, two require
```

```
*        accumulated product of Jacobi rotations, three are perfect )
*
            CALL DLASET( 'Lower', NR-1, NR-1, ZERO, ZERO, A(2,1), LDA )
            CALL DGELQF( NR, N, A, LDA, WORK, WORK(N+1), LWORK-N, IERR)
            CALL DLACPY( 'Lower', NR, NR, A, LDA, V, LDV )
            CALL DLASET( 'Upper', NR-1, NR-1, ZERO, ZERO, V(1,2), LDV )
            CALL DGEQRF( NR, NR, V, LDV, WORK(N+1), WORK(2*N+1),
     $                   LWORK-2*N, IERR )
            DO 8998 p = 1, NR
               CALL DCOPY( NR-p+1, V(p,p), LDV, V(p,p), 1 )
 8998       CONTINUE
            CALL DLASET( 'Upper', NR-1, NR-1, ZERO, ZERO, V(1,2), LDV )
*
            CALL DGESVJ( 'Lower', 'U','N', NR, NR, V,LDV, SVA, NR, U,
     $                  LDU, WORK(N+1), LWORK, INFO )
            SCALEM  = WORK(N+1)
            NUMRANK = IDNINT(WORK(N+2))
            IF ( NR .LT. N ) THEN
               CALL DLASET( 'A',N-NR, NR, ZERO,ZERO, V(NR+1,1),   LDV )
               CALL DLASET( 'A',NR, N-NR, ZERO,ZERO, V(1,NR+1),   LDV )
               CALL DLASET( 'A',N-NR,N-NR,ZERO,ONE, V(NR+1,NR+1), LDV )
            END IF
*
         CALL DORMLQ( 'Left', 'Transpose', N, N, NR, A, LDA, WORK,
     $               V, LDV, WORK(N+1), LWORK-N, IERR )
*
         END IF
*
         DO 8991 p = 1, N
            CALL DCOPY( N, V(p,1), LDV, A(IWORK(p),1), LDA )
 8991    CONTINUE
         CALL DLACPY( 'All', N, N, A, LDA, V, LDV )
*
         IF ( TRANSP ) THEN
            CALL DLACPY( 'All', N, N, V, LDV, U, LDU )
         END IF
*
      ELSE IF ( LSVEC .AND. ( .NOT. RSVEC ) ) THEN
*
*        .. Singular Values and Left Singular Vectors                 ..
*
*        .. second preconditioning step to avoid need to accumulate
*        Jacobi rotations in the Jacobi iterations.
         DO 1965 p = 1, NR
            CALL DCOPY( N-p+1, A(p,p), LDA, U(p,p), 1 )
 1965    CONTINUE
         CALL DLASET( 'Upper', NR-1, NR-1, ZERO, ZERO, U(1,2), LDU )
*
         CALL DGEQRF( N, NR, U, LDU, WORK(N+1), WORK(2*N+1),
     $              LWORK-2*N, IERR )
```

```
*
         DO 1967 p = 1, NR - 1
            CALL DCOPY( NR-p, U(p,p+1), LDU, U(p+1,p), 1 )
 1967    CONTINUE
         CALL DLASET( 'Upper', NR-1, NR-1, ZERO, ZERO, U(1,2), LDU )
*
         CALL DGESVJ( 'Lower', 'U', 'N', NR,NR, U, LDU, SVA, NR, A,
     $        LDA, WORK(N+1), LWORK-N, INFO )
         SCALEM  = WORK(N+1)
         NUMRANK = IDNINT(WORK(N+2))
*
         IF ( NR .LT. M ) THEN
            CALL DLASET( 'A',  M-NR, NR,ZERO, ZERO, U(NR+1,1), LDU )
            IF ( NR .LT. N1 ) THEN
               CALL DLASET( 'A',NR, N1-NR, ZERO, ZERO, U(1,NR+1), LDU )
               CALL DLASET( 'A',M-NR,N1-NR,ZERO,ONE,U(NR+1,NR+1), LDU )
            END IF
         END IF
*
         CALL DORMQR( 'Left', 'No Tr', M, N1, N, A, LDA, WORK, U,
     $               LDU, WORK(N+1), LWORK-N, IERR )
*
         IF ( ROWPIV )
     $       CALL DLASWP( N1, U, LDU, 1, M-1, IWORK(2*N+1), -1 )
*
         DO 1974 p = 1, N1
            XSC = ONE / DNRM2( M, U(1,p), 1 )
            CALL DSCAL( M, XSC, U(1,p), 1 )
 1974    CONTINUE
*
         IF ( TRANSP ) THEN
            CALL DLACPY( 'All', N, N, U, LDU, V, LDV )
         END IF
*
      ELSE
*
*        .. Full SVD ..
*
         IF ( .NOT. JRACC ) THEN
*
         IF ( .NOT. ALMORT ) THEN
*
*           Second Preconditioning Step (QRF [with pivoting])
*           Note that the composition of TRANSPOSE, QRF and TRANSPOSE is
*           equivalent to an LQF CALL. Since in many libraries the QRF
*           seems to be better optimized than the LQF, we do explicit
*           transpose and use the QRF. This is subject to changes in an
*           optimized implementation of DGEJSV.
*
            DO 1968 p = 1, NR
```

```
               CALL DCOPY( N-p+1, A(p,p), LDA, V(p,p), 1 )
 1968       CONTINUE
*
*           .. the following two loops perturb small entries to avoid
*           denormals in the second QR factorization, where they are
*           as good as zeros. This is done to avoid painfully slow
*           computation with denormals. The relative size of the perturbation
*           is a parameter that can be changed by the implementer.
*           This perturbation device will be obsolete on machines with
*           properly implemented arithmetic.
*           To switch it off, set L2PERT=.FALSE. To remove it from  the
*           code, remove the action under L2PERT=.TRUE., leave the ELSE part.
*           The following two loops should be blocked and fused with the
*           transposed copy above.
*
            IF ( L2PERT ) THEN
               XSC = DSQRT(SMALL)
               DO 2969 q = 1, NR
                  TEMP1 = XSC*DABS( V(q,q) )
                  DO 2968 p = 1, N
                     IF ( ( p .GT. q ) .AND. ( DABS(V(p,q)) .LE. TEMP1 )
     $                   .OR. ( p .LT. q ) )
     $                   V(p,q) = DSIGN( TEMP1, V(p,q) )
                     IF ( p .LT. q ) V(p,q) = - V(p,q)
 2968             CONTINUE
 2969          CONTINUE
            ELSE
               CALL DLASET( 'U', NR-1, NR-1, ZERO, ZERO, V(1,2), LDV )
            END IF
*
*           Estimate the row scaled condition number of R1
*           (If R1 is rectangular, N > NR, then the condition number
*           of the leading NR x NR submatrix is estimated.)
*
            CALL DLACPY( 'L', NR, NR, V, LDV, WORK(2*N+1), NR )
            DO 3950 p = 1, NR
               TEMP1 = DNRM2(NR-p+1,WORK(2*N+(p-1)*NR+p),1)
               CALL DSCAL(NR-p+1,ONE/TEMP1,WORK(2*N+(p-1)*NR+p),1)
 3950       CONTINUE
            CALL DPOCON('Lower',NR,WORK(2*N+1),NR,ONE,TEMP1,
     $                  WORK(2*N+NR*NR+1),IWORK(M+2*N+1),IERR)
            CONDR1 = ONE / DSQRT(TEMP1)
*           .. here need a second oppinion on the condition number
*           .. then assume worst case scenario
*           R1 is OK for inverse <=> CONDR1 .LT. DBLE(N)
*           more conservative    <=> CONDR1 .LT. DSQRT(DBLE(N))
*
            COND_OK = DSQRT(DBLE(NR))
*[TP]       COND_OK is a tuning parameter.
```

```
            IF ( CONDR1 .LT. COND_OK ) THEN
*              .. the second QRF without pivoting. Note: in an optimized
*              implementation, this QRF should be implemented as the QRF
*              of a lower triangular matrix.
*              R1^t = Q2 * R2
               CALL DGEQRF( N, NR, V, LDV, WORK(N+1), WORK(2*N+1),
     $              LWORK-2*N, IERR )
*
               IF ( L2PERT ) THEN
                  XSC = DSQRT(SMALL)/EPSLN
                  DO 3959 p = 2, NR
                     DO 3958 q = 1, p - 1
                        TEMP1 = XSC * MIN(DABS(V(p,p)),DABS(V(q,q)))
                        IF ( DABS(V(q,p)) .LE. TEMP1 )
     $                     V(q,p) = DSIGN( TEMP1, V(q,p) )
 3958                CONTINUE
 3959             CONTINUE
               END IF
*
               IF ( NR .NE. N )
     $         CALL DLACPY( 'A', N, NR, V, LDV, WORK(2*N+1), N )
*              .. save ...
*
*           .. this transposed copy should be better than naive
               DO 1969 p = 1, NR - 1
                  CALL DCOPY( NR-p, V(p,p+1), LDV, V(p+1,p), 1 )
 1969          CONTINUE
*
               CONDR2 = CONDR1
*
            ELSE
*
*              .. ill-conditioned case: second QRF with pivoting
*              Note that windowed pivoting would be equaly good
*              numerically, and more run-time efficient. So, in
*              an optimal implementation, the next call to DGEQP3
*              should be replaced with eg. CALL SGEQPX (ACM TOMS #782)
*              with properly (carefully) chosen parameters.
*
*              R1^t * P2 = Q2 * R2
               DO 3003 p = 1, NR
                  IWORK(N+p) = 0
 3003          CONTINUE
               CALL DGEQP3( N, NR, V, LDV, IWORK(N+1), WORK(N+1),
     $                 WORK(2*N+1), LWORK-2*N, IERR )
**               CALL DGEQRF( N, NR, V, LDV, WORK(N+1), WORK(2*N+1),
**     $              LWORK-2*N, IERR )
               IF ( L2PERT ) THEN
                  XSC = DSQRT(SMALL)
                  DO 3969 p = 2, NR
```

```
                  DO 3968 q = 1, p - 1
                     TEMP1 = XSC * MIN(DABS(V(p,p)),DABS(V(q,q)))
                     IF ( DABS(V(q,p)) .LE. TEMP1 )
     $                  V(q,p) = DSIGN( TEMP1, V(q,p) )
 3968             CONTINUE
 3969          CONTINUE
            END IF
*
            CALL DLACPY( 'A', N, NR, V, LDV, WORK(2*N+1), N )
*
            IF ( L2PERT ) THEN
               XSC = DSQRT(SMALL)
               DO 8970 p = 2, NR
                  DO 8971 q = 1, p - 1
                     TEMP1 = XSC * MIN(DABS(V(p,p)),DABS(V(q,q)))
                     V(p,q) = - DSIGN( TEMP1, V(q,p) )
 8971             CONTINUE
 8970          CONTINUE
            ELSE
               CALL DLASET( 'L',NR-1,NR-1,ZERO,ZERO,V(2,1),LDV )
            END IF
*           Now, compute R2 = L3 * Q3, the LQ factorization.
            CALL DGELQF( NR, NR, V, LDV, WORK(2*N+N*NR+1),
     $            WORK(2*N+N*NR+NR+1), LWORK-2*N-N*NR-NR, IERR )
*           .. and estimate the condition number
            CALL DLACPY( 'L',NR,NR,V,LDV,WORK(2*N+N*NR+NR+1),NR )
            DO 4950 p = 1, NR
               TEMP1 = DNRM2( p, WORK(2*N+N*NR+NR+p), NR )
               CALL DSCAL( p, ONE/TEMP1, WORK(2*N+N*NR+NR+p), NR )
 4950       CONTINUE
            CALL DPOCON( 'L',NR,WORK(2*N+N*NR+NR+1),NR,ONE,TEMP1,
     $           WORK(2*N+N*NR+NR+NR*NR+1),IWORK(M+2*N+1),IERR )
            CONDR2 = ONE / DSQRT(TEMP1)
*
            IF ( CONDR2 .GE. COND_OK ) THEN
*              .. save the Householder vectors used for Q3
*              (this overwrittes the copy of R2, as it will not be
*              needed in this branch, but it does not overwritte the
*              Huseholder vectors of Q2.).
               CALL DLACPY( 'U', NR, NR, V, LDV, WORK(2*N+1), N )
*              .. and the rest of the information on Q3 is in
*              WORK(2*N+N*NR+1:2*N+N*NR+N)
            END IF
*
         END IF
*
         IF ( L2PERT ) THEN
            XSC = DSQRT(SMALL)
            DO 4968 q = 2, NR
               TEMP1 = XSC * V(q,q)
```

```
                 DO 4969 p = 1, q - 1
*                   V(p,q) = - DSIGN( TEMP1, V(q,p) )
                    V(p,q) = - DSIGN( TEMP1, V(p,q) )
 4969            CONTINUE
 4968         CONTINUE
           ELSE
              CALL DLASET( 'U', NR-1,NR-1, ZERO,ZERO, V(1,2), LDV )
           END IF
*
*        Second preconditioning finished; continue with Jacobi SVD
*        The input matrix is lower trinagular.
*
*        Recover the right singular vectors as solution of a well
*        conditioned triangular matrix equation.
*
           IF ( CONDR1 .LT. COND_OK ) THEN
*
              CALL DGESVJ( 'L','U','N',NR,NR,V,LDV,SVA,NR,U,
     $             LDU,WORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,INFO )
              SCALEM  = WORK(2*N+N*NR+NR+1)
              NUMRANK = IDNINT(WORK(2*N+N*NR+NR+2))
              DO 3970 p = 1, NR
                 CALL DCOPY( NR, V(1,p), 1, U(1,p), 1 )
                 CALL DSCAL( NR, SVA(p),    V(1,p), 1 )
 3970         CONTINUE

*        .. pick the right matrix equation and solve it
*
              IF ( NR .EQ. N ) THEN
* :))             .. best case, R1 is inverted. The solution of this matrix
*                 equation is Q2*V2 = the product of the Jacobi rotations
*                 used in DGESVJ, premultiplied with the orthogonal matrix
*                 from the second QR factorization.
                 CALL DTRSM( 'L','U','N','N', NR,NR,ONE, A,LDA, V,LDV )
              ELSE
*                .. R1 is well conditioned, but non-square. Transpose(R2)
*                 is inverted to get the product of the Jacobi rotations
*                 used in DGESVJ. The Q-factor from the second QR
*                 factorization is then built in explicitly.
                 CALL DTRSM('L','U','T','N',NR,NR,ONE,WORK(2*N+1),
     $                N,V,LDV)
                 IF ( NR .LT. N ) THEN
                   CALL DLASET('A',N-NR,NR,ZERO,ZERO,V(NR+1,1),LDV)
                   CALL DLASET('A',NR,N-NR,ZERO,ZERO,V(1,NR+1),LDV)
                   CALL DLASET('A',N-NR,N-NR,ZERO,ONE,V(NR+1,NR+1),LDV)
                 END IF
                 CALL DORMQR('L','N',N,N,NR,WORK(2*N+1),N,WORK(N+1),
     $                V,LDV,WORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,IERR)
              END IF
*
```

```
            ELSE IF ( CONDR2 .LT. COND_OK ) THEN
*
* :)           .. the input matrix A is very likely a relative of
*              the Kahan matrix :)
*              The matrix R2 is inverted. The solution of the matrix equation
*              is Q3^T*V3 = the product of the Jacobi rotations (appplied to
*              the lower triangular L3 from the LQ factorization of
*              R2=L3*Q3), pre-multiplied with the transposed Q3.
               CALL DGESVJ( 'L', 'U', 'N', NR, NR, V, LDV, SVA, NR, U,
     $              LDU, WORK(2*N+N*NR+NR+1), LWORK-2*N-N*NR-NR, INFO )
               SCALEM  = WORK(2*N+N*NR+NR+1)
               NUMRANK = IDNINT(WORK(2*N+N*NR+NR+2))
               DO 3870 p = 1, NR
                  CALL DCOPY( NR, V(1,p), 1, U(1,p), 1 )
                  CALL DSCAL( NR, SVA(p),    U(1,p), 1 )
 3870          CONTINUE
               CALL DTRSM('L','U','N','N',NR,NR,ONE,WORK(2*N+1),N,U,LDU)
*              .. apply the permutation from the second QR factorization
               DO 873 q = 1, NR
                  DO 872 p = 1, NR
                     WORK(2*N+N*NR+NR+IWORK(N+p)) = U(p,q)
 872              CONTINUE
                  DO 874 p = 1, NR
                     U(p,q) = WORK(2*N+N*NR+NR+p)
 874              CONTINUE
 873           CONTINUE
               IF ( NR .LT. N ) THEN
                  CALL DLASET( 'A',N-NR,NR,ZERO,ZERO,V(NR+1,1),LDV )
                  CALL DLASET( 'A',NR,N-NR,ZERO,ZERO,V(1,NR+1),LDV )
                  CALL DLASET( 'A',N-NR,N-NR,ZERO,ONE,V(NR+1,NR+1),LDV )
               END IF
               CALL DORMQR( 'L','N',N,N,NR,WORK(2*N+1),N,WORK(N+1),
     $              V,LDV,WORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,IERR )
            ELSE
*              Last line of defense.
* #:(          This is a rather pathological case: no scaled condition
*              improvement after two pivoted QR factorizations. Other
*              possibility is that the rank revealing QR factorization
*              or the condition estimator has failed, or the COND_OK
*              is set very close to ONE (which is unnecessary). Normally,
*              this branch should never be executed, but in rare cases of
*              failure of the RRQR or condition estimator, the last line of
*              defense ensures that DGEJSV completes the task.
*              Compute the full SVD of L3 using DGESVJ with explicit
*              accumulation of Jacobi rotations.
               CALL DGESVJ( 'L', 'U', 'V', NR, NR, V, LDV, SVA, NR, U,
     $              LDU, WORK(2*N+N*NR+NR+1), LWORK-2*N-N*NR-NR, INFO )
               SCALEM  = WORK(2*N+N*NR+NR+1)
               NUMRANK = IDNINT(WORK(2*N+N*NR+NR+2))
               IF ( NR .LT. N ) THEN
```

```
                 CALL DLASET( 'A',N-NR,NR,ZERO,ZERO,V(NR+1,1),LDV )
                 CALL DLASET( 'A',NR,N-NR,ZERO,ZERO,V(1,NR+1),LDV )
                 CALL DLASET( 'A',N-NR,N-NR,ZERO,ONE,V(NR+1,NR+1),LDV )
              END IF
              CALL DORMQR( 'L','N',N,N,NR,WORK(2*N+1),N,WORK(N+1),
     $             V,LDV,WORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,IERR )
*
              CALL DORMLQ( 'L', 'T', NR, NR, NR, WORK(2*N+1), N,
     $             WORK(2*N+N*NR+1), U, LDU, WORK(2*N+N*NR+NR+1),
     $             LWORK-2*N-N*NR-NR, IERR )
              DO 773 q = 1, NR
                 DO 772 p = 1, NR
                    WORK(2*N+N*NR+NR+IWORK(N+p)) = U(p,q)
 772             CONTINUE
                 DO 774 p = 1, NR
                    U(p,q) = WORK(2*N+N*NR+NR+p)
 774             CONTINUE
 773          CONTINUE
*
           END IF
*
*          Permute the rows of V using the (column) permutation from the
*          first QRF. Also, scale the columns to make them unit in
*          Euclidean norm. This applies to all cases.
*
           TEMP1 = DSQRT(DBLE(N)) * EPSLN
           DO 1972 q = 1, N
              DO 972 p = 1, N
                 WORK(2*N+N*NR+NR+IWORK(p)) = V(p,q)
  972         CONTINUE
              DO 973 p = 1, N
                 V(p,q) = WORK(2*N+N*NR+NR+p)
  973         CONTINUE
              XSC = ONE / DNRM2( N, V(1,q), 1 )
              IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $          CALL DSCAL( N, XSC, V(1,q), 1 )
 1972      CONTINUE
*          At this moment, V contains the right singular vectors of A.
*          Next, assemble the left singular vector matrix U (M x N).
           IF ( NR .LT. M ) THEN
              CALL DLASET( 'A', M-NR, NR, ZERO, ZERO, U(NR+1,1), LDU )
              IF ( NR .LT. N1 ) THEN
                 CALL DLASET('A',NR,N1-NR,ZERO,ZERO,U(1,NR+1),LDU)
                 CALL DLASET('A',M-NR,N1-NR,ZERO,ONE,U(NR+1,NR+1),LDU)
              END IF
           END IF
*
*          The Q matrix from the first QRF is built into the left singular
*          matrix U. This applies to all cases.
*
```

```
            CALL DORMQR( 'Left', 'No_Tr', M, N1, N, A, LDA, WORK, U,
     $           LDU, WORK(N+1), LWORK-N, IERR )

*           The columns of U are normalized. The cost is O(M*N) flops.
            TEMP1 = DSQRT(DBLE(M)) * EPSLN
            DO 1973 p = 1, NR
               XSC = ONE / DNRM2( M, U(1,p), 1 )
               IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $          CALL DSCAL( M, XSC, U(1,p), 1 )
 1973       CONTINUE
*
*           If the initial QRF is computed with row pivoting, the left
*           singular vectors must be adjusted.
*
            IF ( ROWPIV )
     $          CALL DLASWP( N1, U, LDU, 1, M-1, IWORK(2*N+1), -1 )
*
         ELSE
*
*        .. the initial matrix A has almost orthogonal columns and
*        the second QRF is not needed
*
            CALL DLACPY( 'Upper', N, N, A, LDA, WORK(N+1), N )
            IF ( L2PERT ) THEN
               XSC = DSQRT(SMALL)
               DO 5970 p = 2, N
                  TEMP1 = XSC * WORK( N + (p-1)*N + p )
                  DO 5971 q = 1, p - 1
                     WORK(N+(q-1)*N+p)=-DSIGN(TEMP1,WORK(N+(p-1)*N+q))
 5971             CONTINUE
 5970          CONTINUE
            ELSE
               CALL DLASET( 'Lower',N-1,N-1,ZERO,ZERO,WORK(N+2),N )
            END IF
*
            CALL DGESVJ( 'Upper', 'U', 'N', N, N, WORK(N+1), N, SVA,
     $           N, U, LDU, WORK(N+N*N+1), LWORK-N-N*N, INFO )
*
            SCALEM  = WORK(N+N*N+1)
            NUMRANK = IDNINT(WORK(N+N*N+2))
            DO 6970 p = 1, N
               CALL DCOPY( N, WORK(N+(p-1)*N+1), 1, U(1,p), 1 )
               CALL DSCAL( N, SVA(p), WORK(N+(p-1)*N+1), 1 )
 6970       CONTINUE
*
            CALL DTRSM( 'Left', 'Upper', 'NoTrans', 'No UD', N, N,
     $           ONE, A, LDA, WORK(N+1), N )
            DO 6972 p = 1, N
               CALL DCOPY( N, WORK(N+p), N, V(IWORK(p),1), LDV )
 6972       CONTINUE
```

```
            TEMP1 = DSQRT(DBLE(N))*EPSLN
            DO 6971 p = 1, N
               XSC = ONE / DNRM2( N, V(1,p), 1 )
               IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $            CALL DSCAL( N, XSC, V(1,p), 1 )
 6971       CONTINUE
*
*           Assemble the left singular vector matrix U (M x N).
*
            IF ( N .LT. M ) THEN
               CALL DLASET( 'A',  M-N, N, ZERO, ZERO, U(N+1,1), LDU )
               IF ( N .LT. N1 ) THEN
                  CALL DLASET( 'A',N,  N1-N, ZERO, ZERO,  U(1,N+1),LDU )
                  CALL DLASET( 'A',M-N,N1-N, ZERO, ONE,U(N+1,N+1),LDU )
               END IF
            END IF
            CALL DORMQR( 'Left', 'No Tr', M, N1, N, A, LDA, WORK, U,
     $           LDU, WORK(N+1), LWORK-N, IERR )
            TEMP1 = DSQRT(DBLE(M))*EPSLN
            DO 6973 p = 1, N1
               XSC = ONE / DNRM2( M, U(1,p), 1 )
               IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $            CALL DSCAL( M, XSC, U(1,p), 1 )
 6973       CONTINUE
*
            IF ( ROWPIV )
     $         CALL DLASWP( N1, U, LDU, 1, M-1, IWORK(2*N+1), -1 )
*
         END IF
*
*        end of the  >> almost orthogonal case <<  in the full SVD
*
         ELSE
*
*        This branch deploys a preconditioned Jacobi SVD with explicitly
*        accumulated rotations. It is included as optional, mainly for
*        experimental purposes. It does perfom well, and can also be used.
*        In this implementation, this branch will be automatically activated
*        if the  condition number sigma_max(A) / sigma_min(A) is predicted
*        to be greater than the overflow threshold. This is because the
*        a posteriori computation of the singular vectors assumes robust
*        implementation of BLAS and some LAPACK procedures, capable of working
*        in presence of extreme values. Since that is not always the case, ...
*
         DO 7968 p = 1, NR
            CALL DCOPY( N-p+1, A(p,p), LDA, V(p,p), 1 )
 7968    CONTINUE
*
         IF ( L2PERT ) THEN
            XSC = DSQRT(SMALL/EPSLN)
```

```
            DO 5969 q = 1, NR
               TEMP1 = XSC*DABS( V(q,q) )
               DO 5968 p = 1, N
                  IF ( ( p .GT. q ) .AND. ( DABS(V(p,q)) .LE. TEMP1 )
     $                .OR. ( p .LT. q ) )
     $                V(p,q) = DSIGN( TEMP1, V(p,q) )
                  IF ( p .LT. q ) V(p,q) = - V(p,q)
 5968          CONTINUE
 5969       CONTINUE
         ELSE
            CALL DLASET( 'U', NR-1, NR-1, ZERO, ZERO, V(1,2), LDV )
         END IF

         CALL DGEQRF( N, NR, V, LDV, WORK(N+1), WORK(2*N+1),
     $        LWORK-2*N, IERR )
         CALL DLACPY( 'L', N, NR, V, LDV, WORK(2*N+1), N )
*
         DO 7969 p = 1, NR
            CALL DCOPY( NR-p+1, V(p,p), LDV, U(p,p), 1 )
 7969    CONTINUE

         IF ( L2PERT ) THEN
            XSC = DSQRT(SMALL/EPSLN)
            DO 9970 q = 2, NR
               DO 9971 p = 1, q - 1
                  TEMP1 = XSC * MIN(DABS(U(p,p)),DABS(U(q,q)))
                  U(p,q) = - DSIGN( TEMP1, U(q,p) )
 9971          CONTINUE
 9970       CONTINUE
         ELSE
            CALL DLASET('U', NR-1, NR-1, ZERO, ZERO, U(1,2), LDU )
         END IF

         CALL DGESVJ( 'G', 'U', 'V', NR, NR, U, LDU, SVA,
     $        N, V, LDV, WORK(2*N+N*NR+1), LWORK-2*N-N*NR, INFO )
         SCALEM  = WORK(2*N+N*NR+1)
         NUMRANK = IDNINT(WORK(2*N+N*NR+2))

         IF ( NR .LT. N ) THEN
            CALL DLASET( 'A',N-NR,NR,ZERO,ZERO,V(NR+1,1),LDV )
            CALL DLASET( 'A',NR,N-NR,ZERO,ZERO,V(1,NR+1),LDV )
            CALL DLASET( 'A',N-NR,N-NR,ZERO,ONE,V(NR+1,NR+1),LDV )
         END IF

         CALL DORMQR( 'L','N',N,N,NR,WORK(2*N+1),N,WORK(N+1),
     $        V,LDV,WORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,IERR )
*
*           Permute the rows of V using the (column) permutation from the
*           first QRF. Also, scale the columns to make them unit in
*           Euclidean norm. This applies to all cases.
```

```
*
            TEMP1 = DSQRT(DBLE(N)) * EPSLN
            DO 7972 q = 1, N
               DO 8972 p = 1, N
                  WORK(2*N+N*NR+NR+IWORK(p)) = V(p,q)
 8972          CONTINUE
               DO 8973 p = 1, N
                  V(p,q) = WORK(2*N+N*NR+NR+p)
 8973          CONTINUE
               XSC = ONE / DNRM2( N, V(1,q), 1 )
               IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $           CALL DSCAL( N, XSC, V(1,q), 1 )
 7972       CONTINUE
*
*           At this moment, V contains the right singular vectors of A.
*           Next, assemble the left singular vector matrix U (M x N).
*
         IF ( NR .LT. M ) THEN
            CALL DLASET( 'A',  M-NR, NR, ZERO, ZERO, U(NR+1,1), LDU )
            IF ( NR .LT. N1 ) THEN
               CALL DLASET( 'A',NR,  N1-NR, ZERO, ZERO,  U(1,NR+1),LDU )
               CALL DLASET( 'A',M-NR,N1-NR, ZERO, ONE,U(NR+1,NR+1),LDU )
            END IF
         END IF
*
         CALL DORMQR( 'Left', 'No Tr', M, N1, N, A, LDA, WORK, U,
     $       LDU, WORK(N+1), LWORK-N, IERR )
*
            IF ( ROWPIV )
     $         CALL DLASWP( N1, U, LDU, 1, M-1, IWORK(2*N+1), -1 )
*
*
         END IF
         IF ( TRANSP ) THEN
*           .. swap U and V because the procedure worked on A^t
            DO 6974 p = 1, N
               CALL DSWAP( N, U(1,p), 1, V(1,p), 1 )
 6974       CONTINUE
         END IF
*
      END IF
*     end of the full SVD
*
*     Undo scaling, if necessary (and possible)
*
      IF ( USCAL2 .LE. (BIG/SVA(1))*USCAL1 ) THEN
         CALL DLASCL( 'G', 0, 0, USCAL1, USCAL2, NR, 1, SVA, N, IERR )
         USCAL1 = ONE
         USCAL2 = ONE
      END IF
```

```
*
      IF ( NR .LT. N ) THEN
         DO 3004 p = NR+1, N
            SVA(p) = ZERO
 3004    CONTINUE
      END IF
*
      WORK(1) = USCAL2 * SCALEM
      WORK(2) = USCAL1
      IF ( ERREST ) WORK(3) = SCONDA
      IF ( LSVEC .AND. RSVEC ) THEN
         WORK(4) = CONDR1
         WORK(5) = CONDR2
      END IF
      IF ( L2TRAN ) THEN
         WORK(6) = ENTRA
         WORK(7) = ENTRAT
      END IF
*
      IWORK(1) = NR
      IWORK(2) = NUMRANK
      IWORK(3) = WARNING
*
      RETURN
*     ..
*     .. END OF DGEJSV
*     ..
      END
*
```

———-

28.1.4 lisp code

— **dgejsv.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dgejsv
         (joba jobu jobv jobr jobt jobp m n a lda sva u ldu v ldv work lwork
          iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (array double-float (*)) work v u sva a)
             (type (f2cl-lib:integer4) info lwork ldv ldu lda n m)
             (type (string *) jobp jobt jobr jobv jobu joba))
    (f2cl-lib:with-multi-array-data
```

```
  ((joba character joba-%data% joba-%offset%)
   (jobu character jobu-%data% jobu-%offset%)
   (jobv character jobv-%data% jobv-%offset%)
   (jobr character jobr-%data% jobr-%offset%)
   (jobt character jobt-%data% jobt-%offset%)
   (jobp character jobp-%data% jobp-%offset%)
   (a double-float a-%data% a-%offset%)
   (sva double-float sva-%data% sva-%offset%)
   (u double-float u-%data% u-%offset%)
   (v double-float v-%data% v-%offset%)
   (work double-float work-%data% work-%offset%)
   (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
 (prog ((almort nil) (defr nil) (errest nil) (goscal nil) (jracc nil)
        (kill nil) (lsvec nil) (l2aber nil) (l2kill nil) (l2pert nil)
        (l2rank nil) (l2tran nil) (noscal nil) (rowpiv nil) (rsvec nil)
        (transp nil) (ierr 0) (n1 0) (nr 0) (numrank 0) (p 0) (q 0)
        (warning 0) (aapp 0.0d0) (aaqq 0.0d0) (aatmax 0.0d0)
        (aatmin 0.0d0) (big 0.0d0) (big1 0.0d0) (cond_ok 0.0d0)
        (condr1 0.0d0) (condr2 0.0d0) (entra 0.0d0) (entrat 0.0d0)
        (epsln 0.0d0) (maxprj 0.0d0) (scalem 0.0d0) (sconda 0.0d0)
        (sfmin 0.0d0) (small 0.0d0) (temp1 0.0d0) (uscal1 0.0d0)
        (uscal2 0.0d0) (xsc 0.0d0))
  (declare
   (type f2cl-lib:logical almort defr errest goscal jracc kill lsvec
    l2aber l2kill l2pert l2rank l2tran noscal rowpiv rsvec transp)
   (type (f2cl-lib:integer4) ierr n1 nr numrank p q warning)
   (type (double-float) aapp aaqq aatmax aatmin big big1 cond_ok condr1
    condr2 entra entrat epsln maxprj scalem sconda sfmin small temp1
    uscal1 uscal2 xsc))
  (setf lsvec
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame jobu "U")
             (declare (ignore var-1))
             (when var-0 (setf jobu var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame jobu "F")
             (declare (ignore var-1))
             (when var-0 (setf jobu var-0))
             ret-val)))
  (setf jracc
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame jobv "J")
            (declare (ignore var-1))
            (when var-0 (setf jobv var-0))
            ret-val))
  (setf rsvec
          (or
           (multiple-value-bind (ret-val var-0 var-1)
```

```
                (lsame jobv "V")
              (declare (ignore var-1))
              (when var-0 (setf jobv var-0))
              ret-val)
            jracc))
(setf rowpiv
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame joba "F")
           (declare (ignore var-1))
           (when var-0 (setf joba var-0))
           ret-val)
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame joba "G")
           (declare (ignore var-1))
           (when var-0 (setf joba var-0))
           ret-val)))
(setf l2rank
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame joba "R")
          (declare (ignore var-1))
          (when var-0 (setf joba var-0))
          ret-val))
(setf l2aber
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame joba "A")
          (declare (ignore var-1))
          (when var-0 (setf joba var-0))
          ret-val))
(setf errest
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame joba "E")
           (declare (ignore var-1))
           (when var-0 (setf joba var-0))
           ret-val)
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame joba "G")
           (declare (ignore var-1))
           (when var-0 (setf joba var-0))
           ret-val)))
(setf l2tran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobt "T")
          (declare (ignore var-1))
          (when var-0 (setf jobt var-0))
          ret-val))
(setf l2kill
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobr "R")
```

```
          (declare (ignore var-1))
          (when var-0 (setf jobr var-0))
          ret-val))
(setf defr
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobr "N")
          (declare (ignore var-1))
          (when var-0 (setf jobr var-0))
          ret-val))
(setf l2pert
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobp "P")
          (declare (ignore var-1))
          (when var-0 (setf jobp var-0))
          ret-val))
(cond
 ((not
   (or rowpiv l2rank l2aber errest
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame joba "C")
         (declare (ignore var-1))
         (when var-0 (setf joba var-0))
         ret-val)))
  (setf info -1))
 ((not
   (or lsvec
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobu "N")
         (declare (ignore var-1))
         (when var-0 (setf jobu var-0))
         ret-val)
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobu "W")
         (declare (ignore var-1))
         (when var-0 (setf jobu var-0))
         ret-val)))
  (setf info -2))
 ((or
   (not
    (or rsvec
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobv "N")
          (declare (ignore var-1))
          (when var-0 (setf jobv var-0))
          ret-val)
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobv "W")
          (declare (ignore var-1))
          (when var-0 (setf jobv var-0))
          ret-val)))
```

```
    (and jracc (not lsvec)))
   (setf info -3))
  ((not (or l2kill defr)) (setf info -4))
  ((not
    (or l2tran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobt "N")
          (declare (ignore var-1))
          (when var-0 (setf jobt var-0))
          ret-val)))
   (setf info -5))
  ((not
    (or l2pert
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobp "N")
          (declare (ignore var-1))
          (when var-0 (setf jobp var-0))
          ret-val)))
   (setf info -6))
  ((< m 0) (setf info -7)) ((or (< n 0) (> n m)) (setf info -8))
  ((< lda m) (setf info -10)) ((and lsvec (< ldu m)) (setf info -13))
  ((and rsvec (< ldv n)) (setf info -14))
  ((or
    (and (not (or lsvec rsvec errest))
         (< lwork
            (max (the f2cl-lib:integer4 7)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 4 n) 1))
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))))
    (and (not (or lsvec rsvec)) errest
         (< lwork
            (max (the f2cl-lib:integer4 7)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 4 n)
                                        (f2cl-lib:int-mul n n)))
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))))
    (and lsvec (not rsvec)
         (< lwork
            (max (the f2cl-lib:integer4 7)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n))
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 4 n) 1)))))
    (and rsvec (not lsvec)
         (< lwork
            (max (the f2cl-lib:integer4 7)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n))
```

```
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul 4 n) 1)))))
   (and lsvec rsvec (not jracc)
        (< lwork
           (max
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n))
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-mul 6 n)
                                   (f2cl-lib:int-mul 2 n n))))))
   (and lsvec rsvec jracc
        (< lwork
           (max
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n))
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-mul 4 n)
                                   (f2cl-lib:int-mul n n)))
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n n) 6))))))
  (setf info -17))
 (t (setf info 0)))
(cond
 ((/= info 0) (xerbla "DGEJSV" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= m 0) (= n 0))
    (go end_label))
(cond
 (lsvec (setf n1 n)
  (if (multiple-value-bind (ret-val var-0 var-1)
          (lsame jobu "F")
        (declare (ignore var-1))
        (when var-0 (setf jobu var-0))
        ret-val)
      (setf n1 m))))
(setf epsln (dlamch "Epsilon"))
(setf sfmin (dlamch "SafeMinimum"))
(setf small (/ sfmin epsln))
(setf big (dlamch "O"))
(setf scalem
        (/ one
           (f2cl-lib:dsqrt (* (f2cl-lib:dble m) (f2cl-lib:dble n)))))
(setf noscal f2cl-lib:%true%)
(setf goscal f2cl-lib:%true%)
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (setf aapp zero)
    (setf aaqq one)
```

```
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dlassq m
         (f2cl-lib:array-slice a-%data% double-float (1 p)
                               ((1 lda) (1 *)) a-%offset%)
         1 aapp aaqq)
      (declare (ignore var-1 var-2))
      (when var-0 (setf m var-0))
      (when var-3 (setf aapp var-3))
      (when var-4 (setf aaqq var-4)))
    (cond
     ((> aapp big) (setf info -9)
      (xerbla "DGEJSV" (f2cl-lib:int-sub info)) (go end_label)))
    (setf aaqq (f2cl-lib:dsqrt aaqq))
    (cond
     ((and (< aapp (f2cl-lib:f2cl/ big aaqq)) noscal)
      (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
              (* aapp aaqq)))
     (t (setf noscal f2cl-lib:%false%)
      (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
              (* aapp (* aaqq scalem)))
      (cond
       (goscal (setf goscal f2cl-lib:%false%)
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal (f2cl-lib:int-sub p 1) scalem sva 1)
          (declare (ignore var-0 var-2 var-3))
          (when var-1 (setf scalem var-1)))))))
   label1874))
(if noscal
    (setf scalem one))
(setf aapp zero)
(setf aaqq big)
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (setf aapp
            (max aapp
                 (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)))
    (if (/= (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%) zero)
        (setf aaqq
                (min aaqq
                     (f2cl-lib:fref sva-%data% (p) ((1 n))
                                    sva-%offset%))))
   label4781))
(cond
 ((= aapp zero)
  (if lsvec
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlaset "G" m n1 zero one u ldu)
        (declare (ignore var-0 var-5))
        (when var-1 (setf m var-1))
```

```
            (when var-2 (setf n1 var-2))
            (when var-3 (setf zero var-3))
            (when var-4 (setf one var-4))
            (when var-6 (setf ldu var-6))))
    (if rsvec
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dlaset "G" n n zero one v ldv)
            (declare (ignore var-0 var-5))
            (when var-1 (setf n var-1))
            (when var-2 (setf n var-2))
            (when var-3 (setf zero var-3))
            (when var-4 (setf one var-4))
            (when var-6 (setf ldv var-6))))
    (setf (f2cl-lib:fref work-%data% (1) ((1 lwork)) work-%offset%) one)
    (setf (f2cl-lib:fref work-%data% (2) ((1 lwork)) work-%offset%) one)
    (if errest
        (setf (f2cl-lib:fref work-%data% (3) ((1 lwork)) work-%offset%)
              one))
    (cond
     ((and lsvec rsvec)
      (setf (f2cl-lib:fref work-%data% (4) ((1 lwork)) work-%offset%)
            one)
      (setf (f2cl-lib:fref work-%data% (5) ((1 lwork)) work-%offset%)
            one)))
    (cond
     (l2tran
      (setf (f2cl-lib:fref work-%data% (6) ((1 lwork)) work-%offset%)
            zero)
      (setf (f2cl-lib:fref work-%data% (7) ((1 lwork)) work-%offset%)
            zero)))
    (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) 0)
    (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%) 0)
    (setf (f2cl-lib:fref iwork-%data% (3) ((1 *)) iwork-%offset%) 0)
    (go end_label)))
  (setf warning 0)
  (cond
   ((<= aaqq sfmin) (setf l2rank f2cl-lib:%true%)
    (setf l2kill f2cl-lib:%true%) (setf warning 1)))
  (cond
   ((= n 1)
    (cond
     (lsvec
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (dlascl "G" 0 0
           (f2cl-lib:array-slice sva-%data% double-float (1) ((1 n))
                                 sva-%offset%)
           scalem m 1
           (f2cl-lib:array-slice a-%data% double-float (1 1)
                                 ((1 lda) (1 *)) a-%offset%)
```

```
        lda ierr)
      (declare (ignore var-0 var-1 var-2 var-3 var-6 var-7))
      (when var-4 (setf scalem var-4))
      (when var-5 (setf m var-5))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "A" m 1 a lda u ldu)
      (declare (ignore var-0 var-2 var-3 var-5))
      (when var-1 (setf m var-1))
      (when var-4 (setf lda var-4))
      (when var-6 (setf ldu var-6)))
    (cond
     ((/= n1 n)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (dgeqrf m n u ldu work
           (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                                 ((1 lwork)) work-%offset%)
           (f2cl-lib:int-sub lwork n) ierr)
        (declare (ignore var-2 var-4 var-5 var-6))
        (when var-0 (setf m var-0))
        (when var-1 (setf n var-1))
        (when var-3 (setf ldu var-3))
        (when var-7 (setf ierr var-7)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dorgqr m n1 1 u ldu work
           (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                                 ((1 lwork)) work-%offset%)
           (f2cl-lib:int-sub lwork n) ierr)
        (declare (ignore var-2 var-3 var-5 var-6 var-7))
        (when var-0 (setf m var-0))
        (when var-1 (setf n1 var-1))
        (when var-4 (setf ldu var-4))
        (when var-8 (setf ierr var-8)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy m
           (f2cl-lib:array-slice a-%data% double-float (1 1)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice u-%data% double-float (1 1)
                                 ((1 ldu) (1 *)) u-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf m var-0)))))))
(cond
 (rsvec
  (setf (f2cl-lib:fref v-%data% (1 1) ((1 ldv) (1 *)) v-%offset%)
          one)))
```

```
  (cond
   ((< (f2cl-lib:fref sva (1) ((1 n))) (* big scalem))
    (setf (f2cl-lib:fref sva-%data% (1) ((1 n)) sva-%offset%)
            (/ (f2cl-lib:fref sva-%data% (1) ((1 n)) sva-%offset%)
               scalem))
    (setf scalem one)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 lwork)) work-%offset%)
          (/ one scalem))
  (setf (f2cl-lib:fref work-%data% (2) ((1 lwork)) work-%offset%) one)
  (cond
   ((/= (f2cl-lib:fref sva (1) ((1 n))) zero)
    (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) 1)
    (cond
     ((>= (f2cl-lib:f2cl/ (f2cl-lib:fref sva (1) ((1 n))) scalem)
          sfmin)
      (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%) 1))
     (t
      (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%)
              0))))
   (t (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) 0)
    (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%) 0)))
  (if errest
      (setf (f2cl-lib:fref work-%data% (3) ((1 lwork)) work-%offset%)
              one))
  (cond
   ((and lsvec rsvec)
    (setf (f2cl-lib:fref work-%data% (4) ((1 lwork)) work-%offset%)
            one)
    (setf (f2cl-lib:fref work-%data% (5) ((1 lwork)) work-%offset%)
            one)))
  (cond
   (l2tran
    (setf (f2cl-lib:fref work-%data% (6) ((1 lwork)) work-%offset%)
            zero)
    (setf (f2cl-lib:fref work-%data% (7) ((1 lwork)) work-%offset%)
            zero)))
  (go end_label)))
(setf transp f2cl-lib:%false%)
(setf l2tran (and l2tran (= m n)))
(setf aatmax (- one))
(setf aatmin big)
(cond
 ((or rowpiv l2tran)
  (cond
   (l2tran
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p m) nil)
      (tagbody
        (setf xsc zero)
        (setf temp1 one)
```

```
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dlassq n
             (f2cl-lib:array-slice a-%data% double-float (p 1)
                                   ((1 lda) (1 *)) a-%offset%)
             lda xsc temp1)
          (declare (ignore var-1))
          (when var-0 (setf n var-0))
          (when var-2 (setf lda var-2))
          (when var-3 (setf xsc var-3))
          (when var-4 (setf temp1 var-4)))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add m n p))
                             ((1 lwork)) work-%offset%)
                (* xsc scalem))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n p))
                             ((1 lwork)) work-%offset%)
                (* xsc (* scalem (f2cl-lib:dsqrt temp1))))
        (setf aatmax
                (max aatmax
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add n p))
                                    ((1 lwork)) work-%offset%)))
        (if (/=
             (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n p))
                            ((1 lwork)) work-%offset%)
             zero)
            (setf aatmin
                    (min aatmin
                         (f2cl-lib:fref work-%data%
                                        ((f2cl-lib:int-add n p))
                                        ((1 lwork)) work-%offset%))))
      label1950)))
    (t
     (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                   ((> p m) nil)
       (tagbody
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add m n p))
                             ((1 lwork)) work-%offset%)
                (* scalem
                   (f2cl-lib:dabs
                    (f2cl-lib:fref a-%data%
                                   (p
                                    (multiple-value-bind
                                        (ret-val var-0 var-1 var-2)
                                        (idamax n
                                         (f2cl-lib:array-slice a
                                                               double-float
                                                               (p 1)
                                                               ((1 lda)
                                                                (1 *)))
                                         lda)
```

```
                                        (declare (ignore var-1))
                                        (when var-0 (setf n var-0))
                                        (when var-2 (setf lda var-2))
                                        ret-val))
                                    ((1 lda) (1 *)) a-%offset%))))
        (setf aatmax
                (max aatmax
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add m n p))
                                    ((1 lwork)) work-%offset%)))
        (setf aatmin
                (min aatmin
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add m n p))
                                    ((1 lwork)) work-%offset%)))
       label1904))))))
(setf entra zero)
(setf entrat zero)
(cond
 (l2tran (setf xsc zero) (setf temp1 one)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dlassq n sva 1 xsc temp1)
    (declare (ignore var-1 var-2))
    (when var-0 (setf n var-0))
    (when var-3 (setf xsc var-3))
    (when var-4 (setf temp1 var-4)))
  (setf temp1 (/ one temp1)) (setf entra zero)
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (setf big1
              (*
               (expt
                (/ (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                   xsc)
                2)
               temp1))
      (if (/= big1 zero)
          (setf entra (+ entra (* big1 (f2cl-lib:dlog big1)))))
     label1113))
  (setf entra (/ (- entra) (f2cl-lib:dlog (f2cl-lib:dble n))))
  (setf entrat zero)
  (f2cl-lib:fdo (p (f2cl-lib:int-add n 1) (f2cl-lib:int-add p 1))
                ((> p (f2cl-lib:int-add n m)) nil)
    (tagbody
      (setf big1
              (*
               (expt
                (/
                 (f2cl-lib:fref work-%data% (p) ((1 lwork))
```

```
                                  work-%offset%)
                   xsc)
                  2)
                 temp1))
        (if (/= big1 zero)
            (setf entrat (+ entrat (* big1 (f2cl-lib:dlog big1)))))
       label1114))
    (setf entrat (/ (- entrat) (f2cl-lib:dlog (f2cl-lib:dble m))))
    (setf transp (< entrat entra))
    (cond
     (transp
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
        (tagbody
          (f2cl-lib:fdo (q (f2cl-lib:int-add p 1) (f2cl-lib:int-add q 1))
                        ((> q n) nil)
            (tagbody
              (setf temp1
                      (f2cl-lib:fref a-%data% (q p) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (q p) ((1 lda) (1 *))
                                    a-%offset%)
                      (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                    a-%offset%)
                      temp1)
             label1116))
         label1115))
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add m n p))
                               ((1 lwork)) work-%offset%)
                  (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%))
          (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n p))
                                 ((1 lwork)) work-%offset%))
         label1117))
      (setf temp1 aapp) (setf aapp aatmax) (setf aatmax temp1)
      (setf temp1 aaqq) (setf aaqq aatmin) (setf aatmin temp1)
      (setf kill lsvec) (setf lsvec rsvec) (setf rsvec kill)
      (if lsvec
          (setf n1 n))
      (setf rowpiv f2cl-lib:%true%)))))
  (setf big1 (f2cl-lib:dsqrt big))
  (setf temp1 (f2cl-lib:dsqrt (/ big (f2cl-lib:dble n))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 aapp temp1 n 1 sva n ierr)
```

```
  (declare (ignore var-0 var-1 var-2 var-6 var-7))
  (when var-3 (setf aapp var-3))
  (when var-4 (setf temp1 var-4))
  (when var-5 (setf n var-5))
  (when var-8 (setf n var-8))
  (when var-9 (setf ierr var-9)))
(cond ((> aaqq (* aapp sfmin)) (setf aaqq (* (/ aaqq aapp) temp1)))
      (t (setf aaqq (/ (* aaqq temp1) aapp))))
(setf temp1 (* temp1 scalem))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dlascl "G" 0 0 aapp temp1 m n a lda ierr)
  (declare (ignore var-0 var-1 var-2 var-7))
  (when var-3 (setf aapp var-3))
  (when var-4 (setf temp1 var-4))
  (when var-5 (setf m var-5))
  (when var-6 (setf n var-6))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(setf uscal1 temp1)
(setf uscal2 aapp)
(cond (l2kill (setf xsc (f2cl-lib:dsqrt sfmin)))
      (t (setf xsc small)
       (cond
        ((and (< aaqq (f2cl-lib:dsqrt sfmin)) lsvec rsvec)
         (setf jracc f2cl-lib:%true%)))))
(cond
 ((< aaqq xsc)
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (cond
       ((< (f2cl-lib:fref sva (p) ((1 n))) xsc)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dlaset "A" m 1 zero zero
             (f2cl-lib:array-slice a-%data% double-float (1 p)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-2 var-5))
          (when var-1 (setf m var-1))
          (when var-3 (setf zero var-3))
          (when var-4 (setf zero var-4))
          (when var-6 (setf lda var-6)))
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
              zero)))
     label700))))
(cond
 (rowpiv
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
```

```
                ((> p (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (setf q
                (f2cl-lib:int-sub
                 (f2cl-lib:int-add
                  (idamax (f2cl-lib:int-add (f2cl-lib:int-sub m p) 1)
                   (f2cl-lib:array-slice work-%data% double-float
                                         ((+ m n p)) ((1 lwork))
                                         work-%offset%)
                   1)
                  p)
                 1))
        (setf (f2cl-lib:fref iwork-%data%
                             ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                p))
                             ((1 *)) iwork-%offset%)
                q)
        (cond
         ((/= p q)
          (setf temp1
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add m n p))
                                 ((1 lwork)) work-%offset%))
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add m n p))
                               ((1 lwork)) work-%offset%)
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add m n q))
                                 ((1 lwork)) work-%offset%))
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add m n q))
                               ((1 lwork)) work-%offset%)
                  temp1)))
       label1952))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaswp n a lda 1 (f2cl-lib:int-sub m 1)
         (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                               ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                               iwork-%offset%)
         1)
      (declare (ignore var-1 var-3 var-4 var-5 var-6))
      (when var-0 (setf n var-0))
      (when var-2 (setf lda var-2)))))
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (setf (f2cl-lib:fref iwork-%data% (p) ((1 *)) iwork-%offset%) 0)
     label1963))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dgeqp3 m n a lda iwork work
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 lwork)) work-%offset%)
       (f2cl-lib:int-sub lwork n) ierr)
```

```
  (declare (ignore var-2 var-4 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-8 (setf ierr var-8)))
(setf nr 1)
(cond
 (l2aber
  (tagbody
    (setf temp1 (* (f2cl-lib:dsqrt (f2cl-lib:dble n)) epsln))
    (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (cond
         ((>= (f2cl-lib:dabs (f2cl-lib:fref a (p p) ((1 lda) (1 *))))
              (* temp1
                 (f2cl-lib:dabs
                  (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))))
          (setf nr (f2cl-lib:int-add nr 1)))
         (t (go label3002)))
       label3001))
   label3002))
 (l2rank
  (tagbody
    (setf temp1 (f2cl-lib:dsqrt sfmin))
    (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (if (or
             (<
              (f2cl-lib:dabs
               (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                              a-%offset%))
              (* epsln
                 (f2cl-lib:dabs
                  (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-sub p 1)
                                  (f2cl-lib:int-sub p 1))
                                 ((1 lda) (1 *)) a-%offset%))))
             (<
              (f2cl-lib:dabs
               (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                              a-%offset%))
              small)
             (and l2kill
                  (<
                   (f2cl-lib:dabs
                    (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                                   a-%offset%))
                   temp1)))
```

```
            (go label3402))
        (setf nr (f2cl-lib:int-add nr 1))
       label3401))
   label3402))
 (t
  (tagbody
    (setf temp1 (f2cl-lib:dsqrt sfmin))
    (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (if (or
             (<
              (f2cl-lib:dabs
               (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                              a-%offset%))
              small)
             (and l2kill
                  (<
                   (f2cl-lib:dabs
                    (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                                   a-%offset%))
                   temp1)))
            (go label3302))
        (setf nr (f2cl-lib:int-add nr 1))
       label3301))
   label3302)))
(setf almort f2cl-lib:%false%)
(cond
 ((= nr n) (setf maxprj one)
  (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (setf temp1
              (/
               (f2cl-lib:dabs
                (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                               a-%offset%))
               (f2cl-lib:fref sva-%data%
                              ((f2cl-lib:fref iwork (p) ((1 *))))
                              ((1 n)) sva-%offset%)))
      (setf maxprj (min maxprj temp1))
     label3051))
  (if (>= (expt maxprj 2) (- one (* (f2cl-lib:dble n) epsln)))
      (setf almort f2cl-lib:%true%))))
(setf sconda (- one))
(setf condr1 (- one))
(setf condr2 (- one))
(cond
 (errest
  (cond
```

```
((= n nr)
 (cond
  (rsvec
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlacpy "U" n n a lda v ldv)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf n var-1))
     (when var-2 (setf n var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf ldv var-6)))
   (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                 ((> p n) nil)
     (tagbody
       (setf temp1
               (f2cl-lib:fref sva-%data%
                              ((f2cl-lib:fref iwork (p) ((1 *))))
                              ((1 n)) sva-%offset%))
       (multiple-value-bind (var-0 var-1 var-2 var-3)
           (dscal p (/ one temp1)
            (f2cl-lib:array-slice v-%data% double-float (1 p)
                                  ((1 ldv) (1 *)) v-%offset%)
            1)
         (declare (ignore var-1 var-2 var-3))
         (when var-0 (setf p var-0)))
      label3053))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (dpocon "U" n v ldv one temp1
        (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                              ((1 lwork)) work-%offset%)
        (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                              ((+ (f2cl-lib:int-mul 2 n) m 1))
                              ((1 *)) iwork-%offset%)
        ierr)
     (declare (ignore var-0 var-2 var-6 var-7))
     (when var-1 (setf n var-1))
     (when var-3 (setf ldv var-3))
     (when var-4 (setf one var-4))
     (when var-5 (setf temp1 var-5))
     (when var-8 (setf ierr var-8))))
  (lsvec
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlacpy "U" n n a lda u ldu)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf n var-1))
     (when var-2 (setf n var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf ldu var-6)))
   (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                 ((> p n) nil)
```

```
      (tagbody
        (setf temp1
                (f2cl-lib:fref sva-%data%
                               ((f2cl-lib:fref iwork (p) ((1 *))))
                               ((1 n)) sva-%offset%))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal p (/ one temp1)
             (f2cl-lib:array-slice u-%data% double-float (1 p)
                                   ((1 ldu) (1 *)) u-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3))
          (when var-0 (setf p var-0)))
       label3054))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dpocon "U" n u ldu one temp1
         (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                               ((1 lwork)) work-%offset%)
         (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                               ((+ (f2cl-lib:int-mul 2 n) m 1))
                               ((1 *)) iwork-%offset%)
         ierr)
      (declare (ignore var-0 var-2 var-6 var-7))
      (when var-1 (setf n var-1))
      (when var-3 (setf ldu var-3))
      (when var-4 (setf one var-4))
      (when var-5 (setf temp1 var-5))
      (when var-8 (setf ierr var-8))))
  (t
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlacpy "U" n n a lda
        (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                              ((1 lwork)) work-%offset%)
        n)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-6 (setf n var-6)))
   (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                 ((> p n) nil)
      (tagbody
        (setf temp1
                (f2cl-lib:fref sva-%data%
                               ((f2cl-lib:fref iwork (p) ((1 *))))
                               ((1 n)) sva-%offset%))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal p (/ one temp1)
             (f2cl-lib:array-slice work-%data% double-float
                                   ((+ n
```

```
                                                (f2cl-lib:int-mul
                                                 (f2cl-lib:int-add p
                                                                   (f2cl-lib:int-sub
                                                                    1))
                                                 n)
                                                1))
                                          ((1 lwork)) work-%offset%)
                    1)
                 (declare (ignore var-1 var-2 var-3))
                 (when var-0 (setf p var-0)))
             label3052))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (dpocon "U" n
               (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                                     ((1 lwork)) work-%offset%)
               n one temp1
               (f2cl-lib:array-slice work-%data% double-float
                                     ((+ n (f2cl-lib:int-mul n n) 1))
                                     ((1 lwork)) work-%offset%)
               (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                                     ((+ (f2cl-lib:int-mul 2 n) m 1))
                                     ((1 *)) iwork-%offset%)
               ierr)
            (declare (ignore var-0 var-2 var-6 var-7))
            (when var-1 (setf n var-1))
            (when var-3 (setf n var-3))
            (when var-4 (setf one var-4))
            (when var-5 (setf temp1 var-5))
            (when var-8 (setf ierr var-8)))))
        (setf sconda (/ one (f2cl-lib:dsqrt temp1))))
       (t (setf sconda (- one))))))
    (setf l2pert
            (and l2pert
                 (>
                  (f2cl-lib:dabs
                   (/
                    (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *))
                                   a-%offset%)
                    (f2cl-lib:fref a-%data% (nr nr) ((1 lda) (1 *))
                                   a-%offset%)))
                  (f2cl-lib:dsqrt big1))))
    (cond
     ((not (or rsvec lsvec))
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p
                        (min
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                         (the f2cl-lib:integer4 nr)))
```

```
                nil)
          (tagbody
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (dcopy (f2cl-lib:int-sub n p)
                 (f2cl-lib:array-slice a-%data% double-float
                                       (p (f2cl-lib:int-add p 1))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice a-%data% double-float ((+ p 1) p)
                                       ((1 lda) (1 *)) a-%offset%)
                 1)
              (declare (ignore var-0 var-1 var-3 var-4))
              (when var-2 (setf lda var-2)))
           label1946))
        (cond
         ((not almort)
          (cond
           (l2pert (setf xsc (/ epsln (f2cl-lib:dble n)))
            (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                          ((> q nr) nil)
              (tagbody
                (setf temp1
                        (* xsc
                           (f2cl-lib:dabs
                            (f2cl-lib:fref a-%data% (q q) ((1 lda) (1 *))
                                           a-%offset%))))
                (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                              ((> p n) nil)
                  (tagbody
                    (if (or
                         (and (> p q)
                              (<=
                               (f2cl-lib:dabs
                                (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                               a-%offset%))
                               temp1))
                         (< p q))
                        (setf (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                             a-%offset%)
                              (coerce
                               (f2cl-lib:dsign temp1
                                               (f2cl-lib:fref a-%data%
                                                              (p q)
                                                              ((1 lda)
                                                               (1 *))
                                                              a-%offset%))
                               'double-float)))
                    label4949))
              label4947)))
           (t
```

```
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlaset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
           zero zero
           (f2cl-lib:array-slice a-%data% double-float (1 2)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-5))
        (when var-3 (setf zero var-3))
        (when var-4 (setf zero var-4))
        (when var-6 (setf lda var-6)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (dgeqrf n nr a lda work
         (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                               ((1 lwork)) work-%offset%)
         (f2cl-lib:int-sub lwork n) ierr)
      (declare (ignore var-2 var-4 var-5 var-6))
      (when var-0 (setf n var-0))
      (when var-1 (setf nr var-1))
      (when var-3 (setf lda var-3))
      (when var-7 (setf ierr var-7)))
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p (f2cl-lib:int-add nr (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dcopy (f2cl-lib:int-sub nr p)
             (f2cl-lib:array-slice a-%data% double-float
                                   (p (f2cl-lib:int-add p 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice a-%data% double-float ((+ p 1) p)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf lda var-2)))
       label1948))))
  (cond
   (l2pert (setf xsc (/ epsln (f2cl-lib:dble n)))
    (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                  ((> q nr) nil)
      (tagbody
        (setf temp1
                (* xsc
                   (f2cl-lib:dabs
                    (f2cl-lib:fref a-%data% (q q) ((1 lda) (1 *))
                                   a-%offset%))))
        (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                      ((> p nr) nil)
          (tagbody
            (if (or
```

```
                    (and (> p q)
                         (<=
                          (f2cl-lib:dabs
                           (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                          a-%offset%))
                          temp1))
                    (< p q))
                   (setf (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                        a-%offset%)
                           (coerce
                            (f2cl-lib:dsign temp1
                                            (f2cl-lib:fref a-%data% (p q)
                                                           ((1 lda) (1 *))
                                                           a-%offset%))
                            'double-float)))
            label1949))
        label1947)))
    (t
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (dlaset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
          zero zero
          (f2cl-lib:array-slice a-%data% double-float (1 2)
                                ((1 lda) (1 *)) a-%offset%)
          lda)
       (declare (ignore var-0 var-1 var-2 var-5))
       (when var-3 (setf zero var-3))
       (when var-4 (setf zero var-4))
       (when var-6 (setf lda var-6)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (dgesvj "L" "NoU" "NoV" nr nr a lda sva n v ldv work lwork info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11))
      (when var-3 (setf nr var-3))
      (when var-4 (setf nr var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf n var-8))
      (when var-10 (setf ldv var-10))
      (when var-12 (setf lwork var-12))
      (when var-13 (setf info var-13)))
    (setf scalem
            (f2cl-lib:fref work-%data% (1) ((1 lwork)) work-%offset%))
    (setf numrank
            (f2cl-lib:idnint
             (f2cl-lib:fref work-%data% (2) ((1 lwork)) work-%offset%))))
   ((and rsvec (not lsvec))
    (cond
     (almort
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p nr) nil)
```

```
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
           (f2cl-lib:array-slice a-%data% double-float (p p)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice v-%data% double-float (p p)
                                 ((1 ldv) (1 *)) v-%offset%)
           1)
        (declare (ignore var-0 var-1 var-3 var-4))
        (when var-2 (setf lda var-2)))
     label1998))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
       zero zero
       (f2cl-lib:array-slice v-%data% double-float (1 2)
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dgesvj "L" "U" "N" n nr v ldv sva nr a lda work lwork info)
    (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11))
    (when var-3 (setf n var-3))
    (when var-4 (setf nr var-4))
    (when var-6 (setf ldv var-6))
    (when var-8 (setf nr var-8))
    (when var-10 (setf lda var-10))
    (when var-12 (setf lwork var-12))
    (when var-13 (setf info var-13)))
  (setf scalem
          (f2cl-lib:fref work-%data% (1) ((1 lwork)) work-%offset%))
  (setf numrank
          (f2cl-lib:idnint
           (f2cl-lib:fref work-%data% (2) ((1 lwork))
                          work-%offset%))))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "Lower" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
       zero zero
       (f2cl-lib:array-slice a-%data% double-float (2 1)
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
```

```
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dgelqf nr n a lda work
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (when var-0 (setf nr var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-7 (setf ierr var-7)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "Lower" nr nr a lda v ldv)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf nr var-1))
  (when var-2 (setf nr var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldv var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
     zero zero
     (f2cl-lib:array-slice v-%data% double-float (1 2)
                           ((1 ldv) (1 *)) v-%offset%)
     ldv)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldv var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dgeqrf nr nr v ldv
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (when var-0 (setf nr var-0))
  (when var-1 (setf nr var-1))
  (when var-3 (setf ldv var-3))
  (when var-7 (setf ierr var-7)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
         (f2cl-lib:array-slice v-%data% double-float (p p)
                               ((1 ldv) (1 *)) v-%offset%)
```

```
          ldv
          (f2cl-lib:array-slice v-%data% double-float (p p)
                                ((1 ldv) (1 *)) v-%offset%)
          1)
       (declare (ignore var-0 var-1 var-3 var-4))
       (when var-2 (setf ldv var-2)))
   label8998))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
     zero zero
     (f2cl-lib:array-slice v-%data% double-float (1 2)
                           ((1 ldv) (1 *)) v-%offset%)
     ldv)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldv var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dgesvj "Lower" "U" "N" nr nr v ldv sva nr u ldu
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     lwork info)
  (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11))
  (when var-3 (setf nr var-3))
  (when var-4 (setf nr var-4))
  (when var-6 (setf ldv var-6))
  (when var-8 (setf nr var-8))
  (when var-10 (setf ldu var-10))
  (when var-12 (setf lwork var-12))
  (when var-13 (setf info var-13)))
(setf scalem
        (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 1))
                       ((1 lwork)) work-%offset%))
(setf numrank
        (f2cl-lib:idnint
         (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 2))
                        ((1 lwork)) work-%offset%)))
(cond
 ((< nr n)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" (f2cl-lib:int-sub n nr) nr zero zero
       (f2cl-lib:array-slice v-%data% double-float ((+ nr 1) 1)
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
```

```
        (when var-6 (setf ldv var-6)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlaset "A" nr (f2cl-lib:int-sub n nr) zero zero
           (f2cl-lib:array-slice v-%data% double-float
                                 (1 (f2cl-lib:int-add nr 1))
                                 ((1 ldv) (1 *)) v-%offset%)
           ldv)
        (declare (ignore var-0 var-2 var-5))
        (when var-1 (setf nr var-1))
        (when var-3 (setf zero var-3))
        (when var-4 (setf zero var-4))
        (when var-6 (setf ldv var-6)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlaset "A" (f2cl-lib:int-sub n nr) (f2cl-lib:int-sub n nr)
           zero one
           (f2cl-lib:array-slice v-%data% double-float
                                 ((+ nr 1) (f2cl-lib:int-add nr 1))
                                 ((1 ldv) (1 *)) v-%offset%)
           ldv)
        (declare (ignore var-0 var-1 var-2 var-5))
        (when var-3 (setf zero var-3))
        (when var-4 (setf one var-4))
        (when var-6 (setf ldv var-6)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (dormlq "Left" "Transpose" n n nr a lda work v ldv
         (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                               ((1 lwork)) work-%offset%)
         (f2cl-lib:int-sub lwork n) ierr)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf n var-2))
      (when var-3 (setf n var-3))
      (when var-4 (setf nr var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldv var-9))
      (when var-12 (setf ierr var-12)))))
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy n
           (f2cl-lib:array-slice v-%data% double-float (p 1)
                                 ((1 ldv) (1 *)) v-%offset%)
           ldv
           (f2cl-lib:array-slice a-%data% double-float
                                 ((f2cl-lib:fref iwork (p) ((1 *))) 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-1 var-3))
```

```
        (when var-0 (setf n var-0))
        (when var-2 (setf ldv var-2))
        (when var-4 (setf lda var-4)))
    label8991))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlacpy "All" n n a lda v ldv)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldv var-6)))
 (cond
  (transp
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlacpy "All" n n v ldv u ldu)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf n var-1))
     (when var-2 (setf n var-2))
     (when var-4 (setf ldv var-4))
     (when var-6 (setf ldu var-6))))))
((and lsvec (not rsvec))
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p nr) nil)
   (tagbody
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
          (f2cl-lib:array-slice a-%data% double-float (p p)
                                ((1 lda) (1 *)) a-%offset%)
          lda
          (f2cl-lib:array-slice u-%data% double-float (p p)
                                ((1 ldu) (1 *)) u-%offset%)
          1)
       (declare (ignore var-0 var-1 var-3 var-4))
       (when var-2 (setf lda var-2)))
    label1965))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlaset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
      zero zero
      (f2cl-lib:array-slice u-%data% double-float (1 2)
                            ((1 ldu) (1 *)) u-%offset%)
      ldu)
   (declare (ignore var-0 var-1 var-2 var-5))
   (when var-3 (setf zero var-3))
   (when var-4 (setf zero var-4))
   (when var-6 (setf ldu var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (dgeqrf n nr u ldu
      (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                            ((1 lwork)) work-%offset%)
```

```
      (f2cl-lib:array-slice work-%data% double-float
                            ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 lwork))
                            work-%offset%)
      (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (when var-0 (setf n var-0))
  (when var-1 (setf nr var-1))
  (when var-3 (setf ldu var-3))
  (when var-7 (setf ierr var-7)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p (f2cl-lib:int-add nr (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy (f2cl-lib:int-sub nr p)
         (f2cl-lib:array-slice u-%data% double-float
                               (p (f2cl-lib:int-add p 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu
         (f2cl-lib:array-slice u-%data% double-float ((+ p 1) p)
                               ((1 ldu) (1 *)) u-%offset%)
         1)
      (declare (ignore var-0 var-1 var-3 var-4))
      (when var-2 (setf ldu var-2)))
   label1967))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
     zero zero
     (f2cl-lib:array-slice u-%data% double-float (1 2)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dgesvj "Lower" "U" "N" nr nr u ldu sva nr a lda
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork n) info)
  (declare
   (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12))
  (when var-3 (setf nr var-3))
  (when var-4 (setf nr var-4))
  (when var-6 (setf ldu var-6))
  (when var-8 (setf nr var-8))
  (when var-10 (setf lda var-10))
  (when var-13 (setf info var-13)))
(setf scalem
```

```
        (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 1))
                       ((1 lwork)) work-%offset%))
(setf numrank
        (f2cl-lib:idnint
         (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 2))
                        ((1 lwork)) work-%offset%)))
(cond
 ((< nr m)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" (f2cl-lib:int-sub m nr) nr zero zero
       (f2cl-lib:array-slice u-%data% double-float ((+ nr 1) 1)
                             ((1 ldu) (1 *)) u-%offset%)
       ldu)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldu var-6)))
  (cond
   ((< nr n1)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "A" nr (f2cl-lib:int-sub n1 nr) zero zero
         (f2cl-lib:array-slice u-%data% double-float
                               (1 (f2cl-lib:int-add nr 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-2 var-5))
      (when var-1 (setf nr var-1))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldu var-6)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "A" (f2cl-lib:int-sub m nr) (f2cl-lib:int-sub n1 nr)
         zero one
         (f2cl-lib:array-slice u-%data% double-float
                               ((+ nr 1) (f2cl-lib:int-add nr 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldu var-6)))))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dormqr "Left" "No Tr" m n1 n a lda work u ldu
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
```

```
    (when var-2 (setf m var-2))
    (when var-3 (setf n1 var-3))
    (when var-4 (setf n var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldu var-9))
    (when var-12 (setf ierr var-12)))
  (if rowpiv
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlaswp n1 u ldu 1 (f2cl-lib:int-sub m 1)
           (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                                 ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                                 iwork-%offset%)
           -1)
        (declare (ignore var-1 var-3 var-4 var-5 var-6))
        (when var-0 (setf n1 var-0))
        (when var-2 (setf ldu var-2))))
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n1) nil)
    (tagbody
      (setf xsc
              (/ one
                 (multiple-value-bind (ret-val var-0 var-1 var-2)
                     (dnrm2 m
                      (f2cl-lib:array-slice u-%data% double-float (1 p)
                                            ((1 ldu) (1 *)) u-%offset%)
                      1)
                   (declare (ignore var-1 var-2))
                   (when var-0 (setf m var-0))
                   ret-val)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal m xsc
           (f2cl-lib:array-slice u-%data% double-float (1 p)
                                 ((1 ldu) (1 *)) u-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf xsc var-1)))
     label1974))
  (cond
   (transp
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "All" n n u ldu v ldv)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf ldu var-4))
      (when var-6 (setf ldv var-6))))))
 (t
  (cond
   ((not jracc)
```

```
(cond
 ((not almort)
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p nr) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
           (f2cl-lib:array-slice a-%data% double-float (p p)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice v-%data% double-float (p p)
                                 ((1 ldv) (1 *)) v-%offset%)
           1)
        (declare (ignore var-0 var-1 var-3 var-4))
        (when var-2 (setf lda var-2)))
     label1968))
  (cond
   (l2pert (setf xsc (f2cl-lib:dsqrt small))
    (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                  ((> q nr) nil)
      (tagbody
        (setf temp1
                (* xsc
                   (f2cl-lib:dabs
                    (f2cl-lib:fref v-%data% (q q) ((1 ldv) (1 *))
                                   v-%offset%))))
        (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                      ((> p n) nil)
          (tagbody
            (if (or
                 (and (> p q)
                      (<=
                       (f2cl-lib:dabs
                        (f2cl-lib:fref v-%data% (p q)
                                       ((1 ldv) (1 *)) v-%offset%))
                       temp1))
                 (< p q))
                (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                     v-%offset%)
                      (coerce
                       (f2cl-lib:dsign temp1
                                       (f2cl-lib:fref v-%data%
                                                      (p q)
                                                      ((1 ldv)
                                                       (1 *))
                                                      v-%offset%))
                       'double-float)))
            (if (< p q)
                (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                     v-%offset%)
```

```
                        (-
                         (f2cl-lib:fref v-%data% (p q)
                                        ((1 ldv) (1 *))
                                        v-%offset%))))
          label2968))
      label2969)))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
       zero zero
       (f2cl-lib:array-slice v-%data% double-float (1 2)
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldv var-6)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" nr nr v ldv
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) work-%offset%)
     nr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf nr var-1))
  (when var-2 (setf nr var-2))
  (when var-4 (setf ldv var-4))
  (when var-6 (setf nr var-6)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (setf temp1
            (dnrm2 (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
             (f2cl-lib:array-slice work-%data% double-float
                                   ((+ (f2cl-lib:int-mul 2 n)
                                       (f2cl-lib:int-mul
                                        (f2cl-lib:int-add p
                                                          (f2cl-lib:int-sub
                                                           1))
                                        nr)
                                       p))
                                   ((1 lwork)) work-%offset%)
             1))
    (dscal (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
     (/ one temp1)
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul
                                (f2cl-lib:int-add p
```

```
                                                        (f2cl-lib:int-sub
                                                         1))
                                  nr)
                                 p))
                            ((1 lwork)) work-%offset%)
     1)
   label3950))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dpocon "Lower" nr
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) work-%offset%)
     nr one temp1
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul nr nr) 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                           ((+ m (f2cl-lib:int-mul 2 n) 1))
                           ((1 *)) iwork-%offset%)
     ierr)
  (declare (ignore var-0 var-2 var-6 var-7))
  (when var-1 (setf nr var-1))
  (when var-3 (setf nr var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf temp1 var-5))
  (when var-8 (setf ierr var-8)))
(setf condr1 (/ one (f2cl-lib:dsqrt temp1)))
(setf cond_ok (f2cl-lib:dsqrt (f2cl-lib:dble nr)))
(cond
 ((< condr1 cond_ok)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dgeqrf n nr v ldv
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 lwork)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ (f2cl-lib:int-mul 2 n) 1))
                             ((1 lwork)) work-%offset%)
       (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (when var-0 (setf n var-0))
    (when var-1 (setf nr var-1))
    (when var-3 (setf ldv var-3))
    (when var-7 (setf ierr var-7)))
  (cond
   (l2pert (setf xsc (/ (f2cl-lib:dsqrt small) epsln))
    (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                  ((> p nr) nil)
```

```
    (tagbody
      (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                    ((> q
                        (f2cl-lib:int-add p
                                          (f2cl-lib:int-sub
                                           1)))
                     nil)
        (tagbody
          (setf temp1
                  (* xsc
                     (min
                      (f2cl-lib:dabs
                       (f2cl-lib:fref v-%data% (p p)
                                      ((1 ldv) (1 *))
                                      v-%offset%))
                      (f2cl-lib:dabs
                       (f2cl-lib:fref v-%data% (q q)
                                      ((1 ldv) (1 *))
                                      v-%offset%)))))
          (if (<=
               (f2cl-lib:dabs
                (f2cl-lib:fref v-%data% (q p) ((1 ldv) (1 *))
                               v-%offset%))
               temp1)
              (setf (f2cl-lib:fref v-%data% (q p)
                                   ((1 ldv) (1 *)) v-%offset%)
                      (coerce
                       (f2cl-lib:dsign temp1
                                       (f2cl-lib:fref v-%data%
                                                      (q p)
                                                      ((1 ldv)
                                                       (1 *))
                                                      v-%offset%))
                       'double-float)))
           label3958))
     label3959))))
(if (/= nr n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "A" n nr v ldv
         (f2cl-lib:array-slice work-%data% double-float
                               ((+ (f2cl-lib:int-mul 2 n) 1))
                               ((1 lwork)) work-%offset%)
         n)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf nr var-2))
      (when var-4 (setf ldv var-4))
      (when var-6 (setf n var-6))))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
```

```
                  ((> p (f2cl-lib:int-add nr (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dcopy (f2cl-lib:int-sub nr p)
             (f2cl-lib:array-slice v-%data% double-float
                                   (p (f2cl-lib:int-add p 1))
                                   ((1 ldv) (1 *)) v-%offset%)
             ldv
             (f2cl-lib:array-slice v-%data% double-float
                                   ((+ p 1) p) ((1 ldv) (1 *))
                                   v-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldv var-2)))
       label1969))
    (setf condr2 condr1))
   (t
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p nr) nil)
      (tagbody
        (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add n p))
                             ((1 *)) iwork-%offset%)
              0)
       label3003))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dgeqp3 n nr v ldv
         (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                               ((+ n 1)) ((1 *)) iwork-%offset%)
         (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                               ((1 lwork)) work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float
                               ((+ (f2cl-lib:int-mul 2 n) 1))
                               ((1 lwork)) work-%offset%)
         (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) ierr)
      (declare (ignore var-2 var-4 var-5 var-6 var-7))
      (when var-0 (setf n var-0))
      (when var-1 (setf nr var-1))
      (when var-3 (setf ldv var-3))
      (when var-8 (setf ierr var-8)))
    (cond
     (l2pert (setf xsc (f2cl-lib:dsqrt small))
      (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                    ((> p nr) nil)
        (tagbody
          (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                        ((> q
                            (f2cl-lib:int-add p
                                              (f2cl-lib:int-sub
```

```
                                          1)))
                      nil)
          (tagbody
            (setf temp1
                    (* xsc
                       (min
                        (f2cl-lib:dabs
                         (f2cl-lib:fref v-%data% (p p)
                                        ((1 ldv) (1 *))
                                        v-%offset%))
                        (f2cl-lib:dabs
                         (f2cl-lib:fref v-%data% (q q)
                                        ((1 ldv) (1 *))
                                        v-%offset%)))))
            (if (<=
                 (f2cl-lib:dabs
                  (f2cl-lib:fref v-%data% (q p) ((1 ldv) (1 *))
                                 v-%offset%))
                 temp1)
                (setf (f2cl-lib:fref v-%data% (q p)
                                     ((1 ldv) (1 *)) v-%offset%)
                        (coerce
                         (f2cl-lib:dsign temp1
                                         (f2cl-lib:fref v-%data%
                                                        (q p)
                                                        ((1 ldv)
                                                         (1 *))
                                                        v-%offset%))
                         'double-float)))
          label3968))
      label3969))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "A" n nr v ldv
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) work-%offset%)
     n)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nr var-2))
  (when var-4 (setf ldv var-4))
  (when var-6 (setf n var-6)))
(cond
 (l2pert (setf xsc (f2cl-lib:dsqrt small))
  (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                ((> p nr) nil)
    (tagbody
      (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                    ((> q
```

```
                                (f2cl-lib:int-add p
                                                  (f2cl-lib:int-sub
                                                   1)))
                            nil)
                (tagbody
                  (setf temp1
                          (* xsc
                             (min
                              (f2cl-lib:dabs
                               (f2cl-lib:fref v-%data% (p p)
                                              ((1 ldv) (1 *))
                                              v-%offset%))
                              (f2cl-lib:dabs
                               (f2cl-lib:fref v-%data% (q q)
                                              ((1 ldv) (1 *))
                                              v-%offset%)))))
                  (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                       v-%offset%)
                          (coerce
                           (-
                            (f2cl-lib:dsign temp1
                                            (f2cl-lib:fref v-%data%
                                                           (q p)
                                                           ((1 ldv)
                                                            (1 *))
                                                           v-%offset%)))
                           'double-float))
                label8971))
            label8970)))
        (t
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
             (dlaset "L" (f2cl-lib:int-sub nr 1)
              (f2cl-lib:int-sub nr 1) zero zero
              (f2cl-lib:array-slice v-%data% double-float (2 1)
                                    ((1 ldv) (1 *)) v-%offset%)
              ldv)
           (declare (ignore var-0 var-1 var-2 var-5))
           (when var-3 (setf zero var-3))
           (when var-4 (setf zero var-4))
           (when var-6 (setf ldv var-6)))))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
           (dgelqf nr nr v ldv
            (f2cl-lib:array-slice work-%data% double-float
                                  ((+ (f2cl-lib:int-mul 2 n)
                                      (f2cl-lib:int-mul n nr) 1))
                                  ((1 lwork)) work-%offset%)
            (f2cl-lib:array-slice work-%data% double-float
                                  ((+ (f2cl-lib:int-mul 2 n)
```

```
                                (f2cl-lib:int-mul n nr) nr 1))
                            ((1 lwork)) work-%offset%)
      (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                        (f2cl-lib:int-mul n nr) nr)
      ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (when var-0 (setf nr var-0))
  (when var-1 (setf nr var-1))
  (when var-3 (setf ldv var-3))
  (when var-7 (setf ierr var-7)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" nr nr v ldv
     (f2cl-lib:array-slice work-%data% double-float
                            ((+ (f2cl-lib:int-mul 2 n)
                                (f2cl-lib:int-mul n nr) nr 1))
                            ((1 lwork)) work-%offset%)
     nr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf nr var-1))
  (when var-2 (setf nr var-2))
  (when var-4 (setf ldv var-4))
  (when var-6 (setf nr var-6)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (setf temp1
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (dnrm2 p
                 (f2cl-lib:array-slice work-%data% double-float
                                       ((+
                                         (f2cl-lib:int-mul 2 n)
                                         (f2cl-lib:int-mul n
                                                           nr)
                                         nr p))
                                       ((1 lwork))
                                       work-%offset%)
                 nr)
              (declare (ignore var-1))
              (when var-0 (setf p var-0))
              (when var-2 (setf nr var-2))
              ret-val))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal p (/ one temp1)
         (f2cl-lib:array-slice work-%data% double-float
                               ((+ (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n nr) nr
                                   p))
                               ((1 lwork)) work-%offset%)
         nr)
```

```
            (declare (ignore var-1 var-2))
            (when var-0 (setf p var-0))
            (when var-3 (setf nr var-3)))
        label4950))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dpocon "L" nr
         (f2cl-lib:array-slice work-%data% double-float
                               ((+ (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n nr) nr 1))
                               ((1 lwork)) work-%offset%)
         nr one temp1
         (f2cl-lib:array-slice work-%data% double-float
                               ((+ (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n nr) nr
                                   (f2cl-lib:int-mul nr nr) 1))
                               ((1 lwork)) work-%offset%)
         (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                               ((+ m (f2cl-lib:int-mul 2 n) 1))
                               ((1 *)) iwork-%offset%)
         ierr)
      (declare (ignore var-0 var-2 var-6 var-7))
      (when var-1 (setf nr var-1))
      (when var-3 (setf nr var-3))
      (when var-4 (setf one var-4))
      (when var-5 (setf temp1 var-5))
      (when var-8 (setf ierr var-8)))
    (setf condr2 (/ one (f2cl-lib:dsqrt temp1)))
    (cond
     ((>= condr2 cond_ok)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlacpy "U" nr nr v ldv
           (f2cl-lib:array-slice work-%data% double-float
                                 ((+ (f2cl-lib:int-mul 2 n) 1))
                                 ((1 lwork)) work-%offset%)
           n)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf nr var-1))
        (when var-2 (setf nr var-2))
        (when var-4 (setf ldv var-4))
        (when var-6 (setf n var-6)))))))
  (cond
   (l2pert (setf xsc (f2cl-lib:dsqrt small))
    (f2cl-lib:fdo (q 2 (f2cl-lib:int-add q 1))
                  ((> q nr) nil)
      (tagbody
        (setf temp1
                (* xsc
                   (f2cl-lib:fref v-%data% (q q) ((1 ldv) (1 *))
```

```
                                v-%offset%)))
        (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                      ((> p
                          (f2cl-lib:int-add q
                                            (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                 v-%offset%)
                  (coerce
                   (-
                    (f2cl-lib:dsign temp1
                                    (f2cl-lib:fref v-%data% (p q)
                                                   ((1 ldv)
                                                    (1 *))
                                                   v-%offset%)))
                   'double-float))
           label4969))
      label4968)))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlaset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
        zero zero
        (f2cl-lib:array-slice v-%data% double-float (1 2)
                              ((1 ldv) (1 *)) v-%offset%)
        ldv)
     (declare (ignore var-0 var-1 var-2 var-5))
     (when var-3 (setf zero var-3))
     (when var-4 (setf zero var-4))
     (when var-6 (setf ldv var-6)))))
 (cond
  ((< condr1 cond_ok)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10 var-11 var-12 var-13)
       (dgesvj "L" "U" "N" nr nr v ldv sva nr u ldu
        (f2cl-lib:array-slice work-%data% double-float
                              ((+ (f2cl-lib:int-mul 2 n)
                                  (f2cl-lib:int-mul n nr) nr 1))
                              ((1 lwork)) work-%offset%)
        (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                          (f2cl-lib:int-mul n nr) nr)
        info)
     (declare
      (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12))
     (when var-3 (setf nr var-3))
     (when var-4 (setf nr var-4))
     (when var-6 (setf ldv var-6))
     (when var-8 (setf nr var-8))
```

```
  (when var-10 (setf ldu var-10))
  (when var-13 (setf info var-13)))
(setf scalem
        (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 2 n)
                         (f2cl-lib:int-mul n nr) nr 1))
                       ((1 lwork)) work-%offset%))
(setf numrank
        (f2cl-lib:idnint
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add
                          (f2cl-lib:int-mul 2 n)
                          (f2cl-lib:int-mul n nr) nr 2))
                        ((1 lwork)) work-%offset%)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy nr
         (f2cl-lib:array-slice v-%data% double-float (1 p)
                               ((1 ldv) (1 *)) v-%offset%)
         1
         (f2cl-lib:array-slice u-%data% double-float (1 p)
                               ((1 ldu) (1 *)) u-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf nr var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal nr
         (f2cl-lib:array-slice sva-%data% double-float (p)
                               ((1 n)) sva-%offset%)
         (f2cl-lib:array-slice v-%data% double-float (1 p)
                               ((1 ldv) (1 *)) v-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf nr var-0)))
   label3970))
(cond
 ((= nr n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (dtrsm "L" "U" "N" "N" nr nr one a lda v ldv)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf nr var-4))
    (when var-5 (setf nr var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldv var-10))))
```

```
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10)
     (dtrsm "L" "U" "T" "N" nr nr one
      (f2cl-lib:array-slice work-%data% double-float
                            ((+ (f2cl-lib:int-mul 2 n) 1))
                            ((1 lwork)) work-%offset%)
      n v ldv)
   (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
   (when var-4 (setf nr var-4))
   (when var-5 (setf nr var-5))
   (when var-6 (setf one var-6))
   (when var-8 (setf n var-8))
   (when var-10 (setf ldv var-10)))
 (cond
  ((< nr n)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlaset "A" (f2cl-lib:int-sub n nr) nr zero zero
        (f2cl-lib:array-slice v-%data% double-float
                              ((+ nr 1) 1) ((1 ldv) (1 *))
                              v-%offset%)
        ldv)
     (declare (ignore var-0 var-1 var-5))
     (when var-2 (setf nr var-2))
     (when var-3 (setf zero var-3))
     (when var-4 (setf zero var-4))
     (when var-6 (setf ldv var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlaset "A" nr (f2cl-lib:int-sub n nr) zero zero
        (f2cl-lib:array-slice v-%data% double-float
                              (1 (f2cl-lib:int-add nr 1))
                              ((1 ldv) (1 *)) v-%offset%)
        ldv)
     (declare (ignore var-0 var-2 var-5))
     (when var-1 (setf nr var-1))
     (when var-3 (setf zero var-3))
     (when var-4 (setf zero var-4))
     (when var-6 (setf ldv var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlaset "A" (f2cl-lib:int-sub n nr)
        (f2cl-lib:int-sub n nr) zero one
        (f2cl-lib:array-slice v-%data% double-float
                              ((+ nr 1)
                               (f2cl-lib:int-add nr 1))
                              ((1 ldv) (1 *)) v-%offset%)
        ldv)
```

```
            (declare (ignore var-0 var-1 var-2 var-5))
            (when var-3 (setf zero var-3))
            (when var-4 (setf one var-4))
            (when var-6 (setf ldv var-6)))))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12)
            (dormqr "L" "N" n n nr
             (f2cl-lib:array-slice work-%data% double-float
                                   ((+ (f2cl-lib:int-mul 2 n) 1))
                                   ((1 lwork)) work-%offset%)
             n
             (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                                   ((1 lwork)) work-%offset%)
             v ldv
             (f2cl-lib:array-slice work-%data% double-float
                                   ((+ (f2cl-lib:int-mul 2 n)
                                       (f2cl-lib:int-mul n nr) nr 1))
                                   ((1 lwork)) work-%offset%)
             (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) nr)
             ierr)
          (declare
           (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
          (when var-2 (setf n var-2))
          (when var-3 (setf n var-3))
          (when var-4 (setf nr var-4))
          (when var-6 (setf n var-6))
          (when var-9 (setf ldv var-9))
          (when var-12 (setf ierr var-12))))))
    ((< condr2 cond_ok)
     (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12 var-13)
          (dgesvj "L" "U" "N" nr nr v ldv sva nr u ldu
           (f2cl-lib:array-slice work-%data% double-float
                                 ((+ (f2cl-lib:int-mul 2 n)
                                     (f2cl-lib:int-mul n nr) nr 1))
                                 ((1 lwork)) work-%offset%)
           (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                             (f2cl-lib:int-mul n nr) nr)
           info)
        (declare
         (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12))
        (when var-3 (setf nr var-3))
        (when var-4 (setf nr var-4))
        (when var-6 (setf ldv var-6))
        (when var-8 (setf nr var-8))
        (when var-10 (setf ldu var-10))
        (when var-13 (setf info var-13)))
```

```
(setf scalem
        (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 2 n)
                         (f2cl-lib:int-mul n nr) nr 1))
                       ((1 lwork)) work-%offset%))
(setf numrank
        (f2cl-lib:idnint
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add
                          (f2cl-lib:int-mul 2 n)
                          (f2cl-lib:int-mul n nr) nr 2))
                        ((1 lwork)) work-%offset%)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy nr
         (f2cl-lib:array-slice v-%data% double-float (1 p)
                               ((1 ldv) (1 *)) v-%offset%)
         1
         (f2cl-lib:array-slice u-%data% double-float (1 p)
                               ((1 ldu) (1 *)) u-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf nr var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal nr
         (f2cl-lib:array-slice sva-%data% double-float (p)
                               ((1 n)) sva-%offset%)
         (f2cl-lib:array-slice u-%data% double-float (1 p)
                               ((1 ldu) (1 *)) u-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf nr var-0)))
   label3870))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dtrsm "L" "U" "N" "N" nr nr one
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) work-%offset%)
     n u ldu)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf nr var-4))
  (when var-5 (setf nr var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf n var-8))
  (when var-10 (setf ldu var-10)))
```

```
(f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
              ((> q nr) nil)
  (tagbody
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p nr) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) nr
                               (f2cl-lib:fref iwork
                                              ((f2cl-lib:int-add
                                                n p))
                                              ((1 *)))))
                             ((1 lwork)) work-%offset%)
                  (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                                 u-%offset%))
       label872))
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p nr) nil)
      (tagbody
        (setf (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                             u-%offset%)
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n nr) nr p))
                                 ((1 lwork)) work-%offset%))
       label874))
   label873))
(cond
 ((< nr n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" (f2cl-lib:int-sub n nr) nr zero zero
       (f2cl-lib:array-slice v-%data% double-float ((+ nr 1) 1)
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" nr (f2cl-lib:int-sub n nr) zero zero
       (f2cl-lib:array-slice v-%data% double-float
                             (1 (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
```

```
      (declare (ignore var-0 var-2 var-5))
      (when var-1 (setf nr var-1))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldv var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "A" (f2cl-lib:int-sub n nr)
         (f2cl-lib:int-sub n nr) zero one
         (f2cl-lib:array-slice v-%data% double-float
                               ((+ nr 1) (f2cl-lib:int-add nr 1))
                               ((1 ldv) (1 *)) v-%offset%)
         ldv)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldv var-6)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dormqr "L" "N" n n nr
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ (f2cl-lib:int-mul 2 n) 1))
                             ((1 lwork)) work-%offset%)
       n
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 lwork)) work-%offset%)
       v ldv
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ (f2cl-lib:int-mul 2 n)
                                 (f2cl-lib:int-mul n nr) nr 1))
                             ((1 lwork)) work-%offset%)
       (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                         (f2cl-lib:int-mul n nr) nr)
       ierr)
    (declare
     (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf nr var-4))
    (when var-6 (setf n var-6))
    (when var-9 (setf ldv var-9))
    (when var-12 (setf ierr var-12))))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12 var-13)
     (dgesvj "L" "U" "V" nr nr v ldv sva nr u ldu
      (f2cl-lib:array-slice work-%data% double-float
                            ((+ (f2cl-lib:int-mul 2 n)
```

```
                                (f2cl-lib:int-mul n nr) nr 1))
                            ((1 lwork)) work-%offset%)
      (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                        (f2cl-lib:int-mul n nr) nr)
      info)
  (declare
   (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12))
  (when var-3 (setf nr var-3))
  (when var-4 (setf nr var-4))
  (when var-6 (setf ldv var-6))
  (when var-8 (setf nr var-8))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf info var-13)))
(setf scalem
        (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 2 n)
                         (f2cl-lib:int-mul n nr) nr 1))
                       ((1 lwork)) work-%offset%))
(setf numrank
        (f2cl-lib:idnint
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add
                          (f2cl-lib:int-mul 2 n)
                          (f2cl-lib:int-mul n nr) nr 2))
                        ((1 lwork)) work-%offset%)))
(cond
 ((< nr n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" (f2cl-lib:int-sub n nr) nr zero zero
       (f2cl-lib:array-slice v-%data% double-float ((+ nr 1) 1)
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" nr (f2cl-lib:int-sub n nr) zero zero
       (f2cl-lib:array-slice v-%data% double-float
                             (1 (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-2 var-5))
    (when var-1 (setf nr var-1))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
```

```
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" (f2cl-lib:int-sub n nr)
       (f2cl-lib:int-sub n nr) zero one
       (f2cl-lib:array-slice v-%data% double-float
                             ((+ nr 1) (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldv var-6)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (dormqr "L" "N" n n nr
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) work-%offset%)
     n
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     v ldv
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) nr 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                       (f2cl-lib:int-mul n nr) nr)
     ierr)
  (declare
   (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf nr var-4))
  (when var-6 (setf n var-6))
  (when var-9 (setf ldv var-9))
  (when var-12 (setf ierr var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (dormlq "L" "T" nr nr nr
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) work-%offset%)
     n
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) 1))
```

```
                                ((1 lwork)) work-%offset%)
        u ldu
        (f2cl-lib:array-slice work-%data% double-float
                                ((+ (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul n nr) nr 1))
                                ((1 lwork)) work-%offset%)
        (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                          (f2cl-lib:int-mul n nr) nr)
        ierr)
      (declare
       (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf nr var-2))
      (when var-3 (setf nr var-3))
      (when var-4 (setf nr var-4))
      (when var-6 (setf n var-6))
      (when var-9 (setf ldu var-9))
      (when var-12 (setf ierr var-12)))
    (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                  ((> q nr) nil)
      (tagbody
        (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                      ((> p nr) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n nr) nr
                                   (f2cl-lib:fref iwork
                                                  ((f2cl-lib:int-add
                                                    n p))
                                                  ((1 *)))))
                                 ((1 lwork)) work-%offset%)
                  (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                                 u-%offset%))
          label772))
        (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                      ((> p nr) nil)
          (tagbody
            (setf (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                                 u-%offset%)
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n nr) nr p))
                                 ((1 lwork)) work-%offset%))
          label774))
      label773))))
  (setf temp1 (* (f2cl-lib:dsqrt (f2cl-lib:dble n)) epsln))
  (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                ((> q n) nil)
```

```
  (tagbody
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) nr
                               (f2cl-lib:fref iwork (p)
                                              ((1 *)))))
                             ((1 lwork)) work-%offset%)
              (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                               v-%offset%))
       label972))
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                             v-%offset%)
              (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add
                                 (f2cl-lib:int-mul 2 n)
                                 (f2cl-lib:int-mul n nr) nr p))
                               ((1 lwork)) work-%offset%))
       label973))
    (setf xsc
            (/ one
               (multiple-value-bind (ret-val var-0 var-1 var-2)
                   (dnrm2 n
                    (f2cl-lib:array-slice v-%data% double-float
                                          (1 q) ((1 ldv) (1 *))
                                          v-%offset%)
                    1)
                 (declare (ignore var-1 var-2))
                 (when var-0 (setf n var-0))
                 ret-val)))
    (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal n xsc
             (f2cl-lib:array-slice v-%data% double-float (1 q)
                                   ((1 ldv) (1 *)) v-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-1 (setf xsc var-1))))
   label1972))
(cond
 ((< nr m)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
        (dlaset "A" (f2cl-lib:int-sub m nr) nr zero zero
         (f2cl-lib:array-slice u-%data% double-float ((+ nr 1) 1)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf nr var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldu var-6)))
    (cond
     ((< nr n1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlaset "A" nr (f2cl-lib:int-sub n1 nr) zero zero
           (f2cl-lib:array-slice u-%data% double-float
                                 (1 (f2cl-lib:int-add nr 1))
                                 ((1 ldu) (1 *)) u-%offset%)
           ldu)
        (declare (ignore var-0 var-2 var-5))
        (when var-1 (setf nr var-1))
        (when var-3 (setf zero var-3))
        (when var-4 (setf zero var-4))
        (when var-6 (setf ldu var-6)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlaset "A" (f2cl-lib:int-sub m nr)
           (f2cl-lib:int-sub n1 nr) zero one
           (f2cl-lib:array-slice u-%data% double-float
                                 ((+ nr 1) (f2cl-lib:int-add nr 1))
                                 ((1 ldu) (1 *)) u-%offset%)
           ldu)
        (declare (ignore var-0 var-1 var-2 var-5))
        (when var-3 (setf zero var-3))
        (when var-4 (setf one var-4))
        (when var-6 (setf ldu var-6)))))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dormqr "Left" "No_Tr" m n1 n a lda work u ldu
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 lwork)) work-%offset%)
       (f2cl-lib:int-sub lwork n) ierr)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n1 var-3))
    (when var-4 (setf n var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldu var-9))
    (when var-12 (setf ierr var-12)))
  (setf temp1 (* (f2cl-lib:dsqrt (f2cl-lib:dble m)) epsln))
```

```
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (setf xsc
            (/ one
               (multiple-value-bind (ret-val var-0 var-1 var-2)
                   (dnrm2 m
                    (f2cl-lib:array-slice u-%data% double-float
                                          (1 p) ((1 ldu) (1 *))
                                          u-%offset%)
                    1)
                 (declare (ignore var-1 var-2))
                 (when var-0 (setf m var-0))
                 ret-val)))
    (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal m xsc
             (f2cl-lib:array-slice u-%data% double-float (1 p)
                                   ((1 ldu) (1 *)) u-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf xsc var-1))))
   label1973))
(if rowpiv
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaswp n1 u ldu 1 (f2cl-lib:int-sub m 1)
         (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                               ((+ (f2cl-lib:int-mul 2 n) 1))
                               ((1 *)) iwork-%offset%)
         -1)
      (declare (ignore var-1 var-3 var-4 var-5 var-6))
      (when var-0 (setf n1 var-0))
      (when var-2 (setf ldu var-2)))))
(t
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlacpy "Upper" n n a lda
      (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                            ((1 lwork)) work-%offset%)
      n)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf n var-6)))
 (cond
  (l2pert (setf xsc (f2cl-lib:dsqrt small))
   (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                 ((> p n) nil)
```

```
                (tagbody
                  (setf temp1
                          (* xsc
                             (f2cl-lib:fref work-%data%
                                            ((f2cl-lib:int-add n
                                                               (f2cl-lib:int-mul
                                                                (f2cl-lib:int-sub
                                                                 p 1)
                                                                n)
                                                               p))
                                            ((1 lwork)) work-%offset%)))
                  (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                                ((> q
                                    (f2cl-lib:int-add p
                                                      (f2cl-lib:int-sub 1)))
                                 nil)
                    (tagbody
                      (setf (f2cl-lib:fref work-%data%
                                           ((f2cl-lib:int-add n
                                                              (f2cl-lib:int-mul
                                                               (f2cl-lib:int-sub
                                                                q 1)
                                                               n)
                                                              p))
                                           ((1 lwork)) work-%offset%)
                            (coerce
                             (-
                              (f2cl-lib:dsign temp1
                                              (f2cl-lib:fref work-%data%
                                                             ((f2cl-lib:int-add
                                                               n
                                                               (f2cl-lib:int-mul
                                                                (f2cl-lib:int-sub
                                                                 p 1)
                                                                n)
                                                               q))
                                                             ((1 lwork))
                                                             work-%offset%)))
                             'double-float))
                      label5971))
                  label5970)))
              (t
               (multiple-value-bind
                   (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                   (dlaset "Lower" (f2cl-lib:int-sub n 1)
                    (f2cl-lib:int-sub n 1) zero zero
                    (f2cl-lib:array-slice work-%data% double-float ((+ n 2))
                                          ((1 lwork)) work-%offset%)
                    n)
                 (declare (ignore var-0 var-1 var-2 var-5))
```

```
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf n var-6)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dgesvj "Upper" "U" "N" n n
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     n sva n u ldu
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ n (f2cl-lib:int-mul n n) 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork n (f2cl-lib:int-mul n n)) info)
  (declare
   (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-6 (setf n var-6))
  (when var-8 (setf n var-8))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf info var-13)))
(setf scalem
        (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add n
                                          (f2cl-lib:int-mul n n)
                                          1))
                       ((1 lwork)) work-%offset%))
(setf numrank
        (f2cl-lib:idnint
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add n
                                           (f2cl-lib:int-mul n n)
                                           2))
                        ((1 lwork)) work-%offset%)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy n
         (f2cl-lib:array-slice work-%data% double-float
                               ((+ n
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-add p
                                                      (f2cl-lib:int-sub
                                                       1))
                                    n)
                                   1))
                               ((1 lwork)) work-%offset%)
         1
```

```
            (f2cl-lib:array-slice u-%data% double-float (1 p)
                                  ((1 ldu) (1 *)) u-%offset%)
            1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf n var-0)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal n
            (f2cl-lib:array-slice sva-%data% double-float (p)
                                  ((1 n)) sva-%offset%)
            (f2cl-lib:array-slice work-%data% double-float
                                  ((+ n
                                      (f2cl-lib:int-mul
                                       (f2cl-lib:int-add p
                                                         (f2cl-lib:int-sub
                                                          1))
                                       n)
                                      1))
                                  ((1 lwork)) work-%offset%)
            1)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf n var-0)))
    label6970))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dtrsm "Left" "Upper" "NoTrans" "No UD" n n one a lda
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 lwork)) work-%offset%)
       n)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf n var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf n var-10)))
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy n
            (f2cl-lib:array-slice work-%data% double-float ((+ n p))
                                  ((1 lwork)) work-%offset%)
            n
            (f2cl-lib:array-slice v-%data% double-float
                                  ((f2cl-lib:fref iwork (p) ((1 *)))
                                   1)
                                  ((1 ldv) (1 *)) v-%offset%)
            ldv)
        (declare (ignore var-1 var-3))
        (when var-0 (setf n var-0))
```

```
      (when var-2 (setf n var-2))
      (when var-4 (setf ldv var-4)))
   label6972))
(setf temp1 (* (f2cl-lib:dsqrt (f2cl-lib:dble n)) epsln))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (setf xsc
            (/ one
               (multiple-value-bind (ret-val var-0 var-1 var-2)
                   (dnrm2 n
                    (f2cl-lib:array-slice v-%data% double-float
                                          (1 p) ((1 ldv) (1 *))
                                          v-%offset%)
                    1)
                 (declare (ignore var-1 var-2))
                 (when var-0 (setf n var-0))
                 ret-val)))
    (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal n xsc
             (f2cl-lib:array-slice v-%data% double-float (1 p)
                                   ((1 ldv) (1 *)) v-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-1 (setf xsc var-1))))
   label6971))
(cond
 ((< n m)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" (f2cl-lib:int-sub m n) n zero zero
       (f2cl-lib:array-slice u-%data% double-float ((+ n 1) 1)
                             ((1 ldu) (1 *)) u-%offset%)
       ldu)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf n var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldu var-6)))
  (cond
   ((< n n1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "A" n (f2cl-lib:int-sub n1 n) zero zero
         (f2cl-lib:array-slice u-%data% double-float
                               (1 (f2cl-lib:int-add n 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
```

```
      (declare (ignore var-0 var-2 var-5))
      (when var-1 (setf n var-1))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldu var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "A" (f2cl-lib:int-sub m n)
         (f2cl-lib:int-sub n1 n) zero one
         (f2cl-lib:array-slice u-%data% double-float
                               ((+ n 1) (f2cl-lib:int-add n 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldu var-6)))))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dormqr "Left" "No Tr" m n1 n a lda work u ldu
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n1 var-3))
  (when var-4 (setf n var-4))
  (when var-6 (setf lda var-6))
  (when var-9 (setf ldu var-9))
  (when var-12 (setf ierr var-12)))
(setf temp1 (* (f2cl-lib:dsqrt (f2cl-lib:dble m)) epsln))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n1) nil)
  (tagbody
    (setf xsc
            (/ one
               (multiple-value-bind (ret-val var-0 var-1 var-2)
                   (dnrm2 m
                    (f2cl-lib:array-slice u-%data% double-float
                                          (1 p) ((1 ldu) (1 *))
                                          u-%offset%)
                    1)
                 (declare (ignore var-1 var-2))
                 (when var-0 (setf m var-0))
                 ret-val)))
    (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal m xsc
             (f2cl-lib:array-slice u-%data% double-float (1 p)
```

```
                                        ((1 ldu) (1 *)) u-%offset%)
                    1)
                 (declare (ignore var-2 var-3))
                 (when var-0 (setf m var-0))
                 (when var-1 (setf xsc var-1))))
          label6973))
      (if rowpiv
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (dlaswp n1 u ldu 1 (f2cl-lib:int-sub m 1)
               (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                                     ((+ (f2cl-lib:int-mul 2 n) 1))
                                     ((1 *)) iwork-%offset%)
               -1)
            (declare (ignore var-1 var-3 var-4 var-5 var-6))
            (when var-0 (setf n1 var-0))
            (when var-2 (setf ldu var-2)))))))
  (t
   (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                 ((> p nr) nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
            (f2cl-lib:array-slice a-%data% double-float (p p)
                                  ((1 lda) (1 *)) a-%offset%)
            lda
            (f2cl-lib:array-slice v-%data% double-float (p p)
                                  ((1 ldv) (1 *)) v-%offset%)
            1)
         (declare (ignore var-0 var-1 var-3 var-4))
         (when var-2 (setf lda var-2)))
      label7968))
   (cond
    (l2pert (setf xsc (f2cl-lib:dsqrt (/ small epsln)))
     (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                   ((> q nr) nil)
       (tagbody
         (setf temp1
                 (* xsc
                    (f2cl-lib:dabs
                     (f2cl-lib:fref v-%data% (q q) ((1 ldv) (1 *))
                                    v-%offset%))))
         (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                       ((> p n) nil)
           (tagbody
             (if (or
                  (and (> p q)
                       (<=
                        (f2cl-lib:dabs
                         (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
```

```
                                        v-%offset%))
                          temp1))
                     (< p q))
                    (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                         v-%offset%)
                            (coerce
                             (f2cl-lib:dsign temp1
                                             (f2cl-lib:fref v-%data%
                                                            (p q)
                                                            ((1 ldv)
                                                             (1 *))
                                                            v-%offset%))
                             'double-float)))
                (if (< p q)
                    (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                         v-%offset%)
                            (-
                             (f2cl-lib:fref v-%data% (p q)
                                            ((1 ldv) (1 *))
                                            v-%offset%))))
              label5968))
          label5969)))
      (t
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
           (dlaset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
            zero zero
            (f2cl-lib:array-slice v-%data% double-float (1 2)
                                  ((1 ldv) (1 *)) v-%offset%)
            ldv)
         (declare (ignore var-0 var-1 var-2 var-5))
         (when var-3 (setf zero var-3))
         (when var-4 (setf zero var-4))
         (when var-6 (setf ldv var-6)))))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (dgeqrf n nr v ldv
          (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                                ((1 lwork)) work-%offset%)
          (f2cl-lib:array-slice work-%data% double-float
                                ((+ (f2cl-lib:int-mul 2 n) 1))
                                ((1 lwork)) work-%offset%)
          (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) ierr)
       (declare (ignore var-2 var-4 var-5 var-6))
       (when var-0 (setf n var-0))
       (when var-1 (setf nr var-1))
       (when var-3 (setf ldv var-3))
       (when var-7 (setf ierr var-7)))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (dlacpy "L" n nr v ldv
          (f2cl-lib:array-slice work-%data% double-float
```

```
                              ((+ (f2cl-lib:int-mul 2 n) 1))
                              ((1 lwork)) work-%offset%)
      n)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf nr var-2))
    (when var-4 (setf ldv var-4))
    (when var-6 (setf n var-6)))
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p nr) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
           (f2cl-lib:array-slice v-%data% double-float (p p)
                                 ((1 ldv) (1 *)) v-%offset%)
           ldv
           (f2cl-lib:array-slice u-%data% double-float (p p)
                                 ((1 ldu) (1 *)) u-%offset%)
           1)
        (declare (ignore var-0 var-1 var-3 var-4))
        (when var-2 (setf ldv var-2)))
     label7969))
  (cond
   (l2pert (setf xsc (f2cl-lib:dsqrt (/ small epsln)))
    (f2cl-lib:fdo (q 2 (f2cl-lib:int-add q 1))
                  ((> q nr) nil)
      (tagbody
        (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                      ((> p
                          (f2cl-lib:int-add q (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf temp1
                    (* xsc
                       (min
                        (f2cl-lib:dabs
                         (f2cl-lib:fref u-%data% (p p)
                                        ((1 ldu) (1 *)) u-%offset%))
                        (f2cl-lib:dabs
                         (f2cl-lib:fref u-%data% (q q)
                                        ((1 ldu) (1 *))
                                        u-%offset%)))))
            (setf (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                                 u-%offset%)
                    (coerce
                     (-
                      (f2cl-lib:dsign temp1
                                      (f2cl-lib:fref u-%data% (q p)
                                                     ((1 ldu) (1 *))
                                                     u-%offset%)))
```

```
                    'double-float))
          label9971))
      label9970)))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
       zero zero
       (f2cl-lib:array-slice u-%data% double-float (1 2)
                             ((1 ldu) (1 *)) u-%offset%)
       ldu)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldu var-6)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dgesvj "G" "U" "V" nr nr u ldu sva n v ldv
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                       (f2cl-lib:int-mul n nr))
     info)
  (declare
   (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12))
  (when var-3 (setf nr var-3))
  (when var-4 (setf nr var-4))
  (when var-6 (setf ldu var-6))
  (when var-8 (setf n var-8))
  (when var-10 (setf ldv var-10))
  (when var-13 (setf info var-13)))
(setf scalem
        (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                          (f2cl-lib:int-mul n nr)
                                          1))
                       ((1 lwork)) work-%offset%))
(setf numrank
        (f2cl-lib:idnint
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                           (f2cl-lib:int-mul n nr)
                                           2))
                        ((1 lwork)) work-%offset%)))
(cond
 ((< nr n)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" (f2cl-lib:int-sub n nr) nr zero zero
```

```
        (f2cl-lib:array-slice v-%data% double-float ((+ nr 1) 1)
                              ((1 ldv) (1 *)) v-%offset%)
        ldv)
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf nr var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldv var-6)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "A" nr (f2cl-lib:int-sub n nr) zero zero
        (f2cl-lib:array-slice v-%data% double-float
                              (1 (f2cl-lib:int-add nr 1))
                              ((1 ldv) (1 *)) v-%offset%)
        ldv)
      (declare (ignore var-0 var-2 var-5))
      (when var-1 (setf nr var-1))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldv var-6)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "A" (f2cl-lib:int-sub n nr) (f2cl-lib:int-sub n nr)
        zero one
        (f2cl-lib:array-slice v-%data% double-float
                              ((+ nr 1) (f2cl-lib:int-add nr 1))
                              ((1 ldv) (1 *)) v-%offset%)
        ldv)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldv var-6)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dormqr "L" "N" n n nr
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ (f2cl-lib:int-mul 2 n) 1))
                             ((1 lwork)) work-%offset%)
       n
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 lwork)) work-%offset%)
       v ldv
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ (f2cl-lib:int-mul 2 n)
                                 (f2cl-lib:int-mul n nr) nr 1))
                             ((1 lwork)) work-%offset%)
       (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                         (f2cl-lib:int-mul n nr) nr)
       ierr)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
```

```
  (when var-3 (setf n var-3))
  (when var-4 (setf nr var-4))
  (when var-6 (setf n var-6))
  (when var-9 (setf ldv var-9))
  (when var-12 (setf ierr var-12)))
(setf temp1 (* (f2cl-lib:dsqrt (f2cl-lib:dble n)) epsln))
(f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
              ((> q n) nil)
  (tagbody
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) nr
                               (f2cl-lib:fref iwork (p) ((1 *)))))
                             ((1 lwork)) work-%offset%)
              (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                               v-%offset%))
       label8972))
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                             v-%offset%)
              (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add
                                 (f2cl-lib:int-mul 2 n)
                                 (f2cl-lib:int-mul n nr) nr p))
                               ((1 lwork)) work-%offset%))
       label8973))
    (setf xsc
            (/ one
               (multiple-value-bind (ret-val var-0 var-1 var-2)
                   (dnrm2 n
                    (f2cl-lib:array-slice v-%data% double-float
                                          (1 q) ((1 ldv) (1 *))
                                          v-%offset%)
                    1)
                 (declare (ignore var-1 var-2))
                 (when var-0 (setf n var-0))
                 ret-val)))
    (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal n xsc
             (f2cl-lib:array-slice v-%data% double-float (1 q)
                                   ((1 ldv) (1 *)) v-%offset%)
             1)
          (declare (ignore var-2 var-3))
```

```
          (when var-0 (setf n var-0))
          (when var-1 (setf xsc var-1))))
   label7972))
(cond
 ((< nr m)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" (f2cl-lib:int-sub m nr) nr zero zero
       (f2cl-lib:array-slice u-%data% double-float ((+ nr 1) 1)
                             ((1 ldu) (1 *)) u-%offset%)
       ldu)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldu var-6)))
  (cond
   ((< nr n1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "A" nr (f2cl-lib:int-sub n1 nr) zero zero
         (f2cl-lib:array-slice u-%data% double-float
                               (1 (f2cl-lib:int-add nr 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-2 var-5))
      (when var-1 (setf nr var-1))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldu var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "A" (f2cl-lib:int-sub m nr)
         (f2cl-lib:int-sub n1 nr) zero one
         (f2cl-lib:array-slice u-%data% double-float
                               ((+ nr 1) (f2cl-lib:int-add nr 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldu var-6)))))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dormqr "Left" "No Tr" m n1 n a lda work u ldu
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf m var-2))
```

```
        (when var-3 (setf n1 var-3))
        (when var-4 (setf n var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldu var-9))
        (when var-12 (setf ierr var-12)))
      (if rowpiv
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (dlaswp n1 u ldu 1 (f2cl-lib:int-sub m 1)
               (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                                     ((+ (f2cl-lib:int-mul 2 n) 1))
                                     ((1 *)) iwork-%offset%)
               -1)
            (declare (ignore var-1 var-3 var-4 var-5 var-6))
            (when var-0 (setf n1 var-0))
            (when var-2 (setf ldu var-2))))))
  (cond
   (transp
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dswap n
             (f2cl-lib:array-slice u-%data% double-float (1 p)
                                   ((1 ldu) (1 *)) u-%offset%)
             1
             (f2cl-lib:array-slice v-%data% double-float (1 p)
                                   ((1 ldv) (1 *)) v-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf n var-0)))
       label6974))))))
(cond
 ((<= uscal2
      (* (f2cl-lib:f2cl/ big (f2cl-lib:fref sva (1) ((1 n)))) uscal1))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 uscal1 uscal2 nr 1 sva n ierr)
    (declare (ignore var-0 var-1 var-2 var-6 var-7))
    (when var-3 (setf uscal1 var-3))
    (when var-4 (setf uscal2 var-4))
    (when var-5 (setf nr var-5))
    (when var-8 (setf n var-8))
    (when var-9 (setf ierr var-9)))
  (setf uscal1 one) (setf uscal2 one)))
(cond
 ((< nr n)
  (f2cl-lib:fdo (p (f2cl-lib:int-add nr 1) (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
```

```
              (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%) zero)
             label3004))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 lwork)) work-%offset%)
                (* uscal2 scalem))
        (setf (f2cl-lib:fref work-%data% (2) ((1 lwork)) work-%offset%) uscal1)
        (if errest
            (setf (f2cl-lib:fref work-%data% (3) ((1 lwork)) work-%offset%)
                    sconda))
        (cond
         ((and lsvec rsvec)
          (setf (f2cl-lib:fref work-%data% (4) ((1 lwork)) work-%offset%)
                  condr1)
          (setf (f2cl-lib:fref work-%data% (5) ((1 lwork)) work-%offset%)
                  condr2)))
        (cond
         (l2tran
          (setf (f2cl-lib:fref work-%data% (6) ((1 lwork)) work-%offset%)
                  entra)
          (setf (f2cl-lib:fref work-%data% (7) ((1 lwork)) work-%offset%)
                  entrat)))
        (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) nr)
        (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%) numrank)
        (setf (f2cl-lib:fref iwork-%data% (3) ((1 *)) iwork-%offset%) warning)
        (go end_label)
       end_label
        (return
         (values joba jobu jobv jobr jobt jobp m n nil lda nil nil ldu nil ldv
                 nil lwork nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgejsv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
```

```
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::joba
  fortran-to-lisp::jobu
  fortran-to-lisp::jobv
  fortran-to-lisp::jobr
  fortran-to-lisp::jobt
  fortran-to-lisp::jobp
  fortran-to-lisp::m
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil
  fortran-to-lisp::ldu nil
  fortran-to-lisp::ldv nil
  fortran-to-lisp::lwork nil
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

28.2 dgesdd

28.2.1 Axiom unit tests

— **dgesdd.input** —

```
)set break resume
)sys rm -f dgesdd.output
)spool dgesdd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

28.2.2 Axiom help page

— **dgesdd.help** —

```
====================================================================
```

```
dgesdd examples
===================================================================

===================================================================
Man Page Details
===================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgesdd (JOBZ, M, N, A, LDA, S, U, LDU, VT, LDVT, WORK,
           LWORK, IWORK, INFO)
           DGESDD

Function/Subroutine Documentation
   subroutine dgesdd (character JOBZ, integer M, integer N, double precision,
       dimension( lda, * ) A, integer LDA, double precision, dimension( * ) S,
       double precision, dimension( ldu, * ) U, integer LDU, double precision,
       dimension( ldvt, * ) VT, integer LDVT, double precision, dimension( * )
       WORK, integer LWORK, integer, dimension( * ) IWORK, integer INFO)
       DGESDD

Purpose:
   DGESDD computes the singular value decomposition (SVD) of a real
   M-by-N matrix A, optionally computing the left and right singular
   vectors.  If singular vectors are desired, it uses a
   divide-and-conquer algorithm.

   The SVD is written

   A = U * SIGMA * transpose(V)

   where SIGMA is an M-by-N matrix which is zero except for its
   min(m,n) diagonal elements, U is an M-by-M orthogonal matrix, and
   V is an N-by-N orthogonal matrix.  The diagonal elements of SIGMA
   are the singular values of A; they are real and non-negative, and
   are returned in descending order.  The first min(m,n) columns of
   U and V are the left and right singular vectors of A.

   Note that the routine returns VT = V**T, not V.

   The divide and conquer algorithm makes very mild assumptions about
   floating point arithmetic. It will work on machines with a guard
   digit in add/subtract, or on those binary machines without guard
   digits which subtract like the Cray X-MP, Cray Y-MP, Cray C-90, or
   Cray-2. It could conceivably fail on hexadecimal or decimal machines
   without guard digits, but we know of none.

Parameters:

 JOBZ
```

```
  JOBZ is CHARACTER*1
  Specifies options for computing all or part of the matrix U:
  = 'A':  all M columns of U and all N rows of V**T are
          returned in the arrays U and VT;
  = 'S':  the first min(M,N) columns of U and the first
          min(M,N) rows of V**T are returned in the arrays U and VT;
  = 'O':  If M >= N, the first N columns of U are overwritten
          on the array A and all rows of V**T are returned in
          the array VT;
          otherwise, all columns of U are returned in the
          array U and the first M rows of V**T are overwritten
          in the array A;
  = 'N':  no columns of U or rows of V**T are computed.

M
  M is INTEGER
  The number of rows of the input matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the input matrix A.  N >= 0.

A

  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit,
  if JOBZ = 'O',  A is overwritten with the first N columns
                  of U (the left singular vectors, stored
                  columnwise) if M >= N;
                  A is overwritten with the first M rows
                  of V**T (the right singular vectors, stored
                  rowwise) otherwise.
  if JOBZ .ne. 'O', the contents of A are destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

S
  S is DOUBLE PRECISION array, dimension (min(M,N))
  The singular values of A, sorted so that S(i) >= S(i+1).

U
  U is DOUBLE PRECISION array, dimension (LDU,UCOL)
  UCOL = M if JOBZ = 'A' or JOBZ = 'O' and M < N;
  UCOL = min(M,N) if JOBZ = 'S'.
  If JOBZ = 'A' or JOBZ = 'O' and M < N, U contains the M-by-M
  orthogonal matrix U;
  if JOBZ = 'S', U contains the first min(M,N) columns of U
```

```
  (the left singular vectors, stored columnwise);
  if JOBZ = 'O' and M >= N, or JOBZ = 'N', U is not referenced.

LDU
  LDU is INTEGER
  The leading dimension of the array U.  LDU >= 1; if
  JOBZ = 'S' or 'A' or JOBZ = 'O' and M < N, LDU >= M.

VT
  VT is DOUBLE PRECISION array, dimension (LDVT,N)
  If JOBZ = 'A' or JOBZ = 'O' and M >= N, VT contains the
  N-by-N orthogonal matrix V**T;
  if JOBZ = 'S', VT contains the first min(M,N) rows of
  V**T (the right singular vectors, stored rowwise);
  if JOBZ = 'O' and M < N, or JOBZ = 'N', VT is not referenced.

LDVT
  LDVT is INTEGER
  The leading dimension of the array VT.  LDVT >= 1; if
  JOBZ = 'A' or JOBZ = 'O' and M >= N, LDVT >= N;
  if JOBZ = 'S', LDVT >= min(M,N).

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK;

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK >= 1.
  If JOBZ = 'N', LWORK >= 3*min(M,N) + max(max(M,N),7*min(M,N)).
  If JOBZ = 'O', LWORK >= 3*min(M,N) +
                 max(max(M,N),5*min(M,N)*min(M,N)+4*min(M,N)).
  If JOBZ = 'S' or 'A' LWORK >= min(M,N)*(7+4*min(M,N))
  For good performance, LWORK should generally be larger.
  If LWORK = -1 but other input arguments are legal, WORK(1)
  returns the optimal LWORK.

IWORK
  IWORK is INTEGER array, dimension (8*min(M,N))

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  DBDSDC did not converge, updating process failed.
```

———-

28.2.3 fortran code

— dgesdd.f —

```
*  =====================================================================
      SUBROUTINE DGESDD( JOBZ, M, N, A, LDA, S, U, LDU, VT, LDVT, WORK,
     $                   LWORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBZ
      INTEGER            INFO, LDA, LDU, LDVT, LWORK, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), S( * ), U( LDU, * ),
     $                   VT( LDVT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, WNTQA, WNTQAS, WNTQN, WNTQO, WNTQS
      INTEGER            BDSPAC, BLK, CHUNK, I, IE, IERR, IL,
     $                   IR, ISCL, ITAU, ITAUP, ITAUQ, IU, IVT, LDWKVT,
     $                   LDWRKL, LDWRKR, LDWRKU, MAXWRK, MINMN, MINWRK,
     $                   MNTHR, NWORK, WRKBL
      DOUBLE PRECISION   ANRM, BIGNUM, EPS, SMLNUM
*     ..
*     .. Local Arrays ..
      INTEGER            IDUM( 1 )
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DBDSDC, DGEBRD, DGELQF, DGEMM, DGEQRF, DLACPY,
     $                   DLASCL, DLASET, DORGBR, DORGLQ, DORGQR, DORMBR,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
```

```
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           DLAMCH, DLANGE, ILAENV, LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          INT, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      MINMN = MIN( M, N )
      WNTQA = LSAME( JOBZ, 'A' )
      WNTQS = LSAME( JOBZ, 'S' )
      WNTQAS = WNTQA .OR. WNTQS
      WNTQO = LSAME( JOBZ, 'O' )
      WNTQN = LSAME( JOBZ, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
*
      IF( .NOT.( WNTQA .OR. WNTQS .OR. WNTQO .OR. WNTQN ) ) THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDU.LT.1 .OR. ( WNTQAS .AND. LDU.LT.M ) .OR.
     $         ( WNTQO .AND. M.LT.N .AND. LDU.LT.M ) ) THEN
         INFO = -8
      ELSE IF( LDVT.LT.1 .OR. ( WNTQA .AND. LDVT.LT.N ) .OR.
     $         ( WNTQS .AND. LDVT.LT.MINMN ) .OR.
     $         ( WNTQO .AND. M.GE.N .AND. LDVT.LT.N ) ) THEN
         INFO = -10
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = 1
         MAXWRK = 1
         IF( M.GE.N .AND. MINMN.GT.0 ) THEN
*
*           Compute space needed for DBDSDC
*
```

```
            MNTHR = INT( MINMN*11.0D0 / 6.0D0 )
            IF( WNTQN ) THEN
               BDSPAC = 7*N
            ELSE
               BDSPAC = 3*N*N + 4*N
            END IF
            IF( M.GE.MNTHR ) THEN
               IF( WNTQN ) THEN
*
*                 Path 1 (M much larger than N, JOBZ='N')
*
                  WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1,
     $                    -1 )
                  WRKBL = MAX( WRKBL, 3*N+2*N*
     $                    ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                  MAXWRK = MAX( WRKBL, BDSPAC+N )
                  MINWRK = BDSPAC + N
               ELSE IF( WNTQO ) THEN
*
*                 Path 2 (M much larger than N, JOBZ='O')
*
                  WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
                  WRKBL = MAX( WRKBL, N+N*ILAENV( 1, 'DORGQR', ' ', M,
     $                    N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 3*N+2*N*
     $                    ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                  WRKBL = MAX( WRKBL, 3*N+N*
     $                    ILAENV( 1, 'DORMBR', 'QLN', N, N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 3*N+N*
     $                    ILAENV( 1, 'DORMBR', 'PRT', N, N, N, -1 ) )
                  WRKBL = MAX( WRKBL, BDSPAC+3*N )
                  MAXWRK = WRKBL + 2*N*N
                  MINWRK = BDSPAC + 2*N*N + 3*N
               ELSE IF( WNTQS ) THEN
*
*                 Path 3 (M much larger than N, JOBZ='S')
*
                  WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
                  WRKBL = MAX( WRKBL, N+N*ILAENV( 1, 'DORGQR', ' ', M,
     $                    N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 3*N+2*N*
     $                    ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                  WRKBL = MAX( WRKBL, 3*N+N*
     $                    ILAENV( 1, 'DORMBR', 'QLN', N, N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 3*N+N*
     $                    ILAENV( 1, 'DORMBR', 'PRT', N, N, N, -1 ) )
                  WRKBL = MAX( WRKBL, BDSPAC+3*N )
                  MAXWRK = WRKBL + N*N
                  MINWRK = BDSPAC + N*N + 3*N
               ELSE IF( WNTQA ) THEN
```

```
*
*                Path 4 (M much larger than N, JOBZ='A')
*
                 WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, N+M*ILAENV( 1, 'DORGQR', ' ', M,
     $                   M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+2*N*
     $                   ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'QLN', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'PRT', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*N )
                 MAXWRK = WRKBL + N*N
                 MINWRK = BDSPAC + N*N + 2*N + M
              END IF
           ELSE
*
*             Path 5 (M at least N, but not much larger)
*
              WRKBL = 3*N + ( M+N )*ILAENV( 1, 'DGEBRD', ' ', M, N, -1,
     $                -1 )
              IF( WNTQN ) THEN
                 MAXWRK = MAX( WRKBL, BDSPAC+3*N )
                 MINWRK = 3*N + MAX( M, BDSPAC )
              ELSE IF( WNTQO ) THEN
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'PRT', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*N )
                 MAXWRK = WRKBL + M*N
                 MINWRK = 3*N + MAX( M, N*N+BDSPAC )
              ELSE IF( WNTQS ) THEN
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'PRT', N, N, N, -1 ) )
                 MAXWRK = MAX( WRKBL, BDSPAC+3*N )
                 MINWRK = 3*N + MAX( M, BDSPAC )
              ELSE IF( WNTQA ) THEN
                 WRKBL = MAX( WRKBL, 3*N+M*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'PRT', N, N, N, -1 ) )
                 MAXWRK = MAX( MAXWRK, BDSPAC+3*N )
                 MINWRK = 3*N + MAX( M, BDSPAC )
              END IF
           END IF
        ELSE IF( MINMN.GT.0 ) THEN
```

```
*
*          Compute space needed for DBDSDC
*
           MNTHR = INT( MINMN*11.0D0 / 6.0D0 )
           IF( WNTQN ) THEN
              BDSPAC = 7*M
           ELSE
              BDSPAC = 3*M*M + 4*M
           END IF
           IF( N.GE.MNTHR ) THEN
              IF( WNTQN ) THEN
*
*                Path 1t (N much larger than M, JOBZ='N')
*
                 WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1,
     $                   -1 )
                 WRKBL = MAX( WRKBL, 3*M+2*M*
     $                   ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
                 MAXWRK = MAX( WRKBL, BDSPAC+M )
                 MINWRK = BDSPAC + M
              ELSE IF( WNTQO ) THEN
*
*                Path 2t (N much larger than M, JOBZ='O')
*
                 WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, M+M*ILAENV( 1, 'DORGLQ', ' ', M,
     $                   N, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+2*M*
     $                   ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'PRT', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*M )
                 MAXWRK = WRKBL + 2*M*M
                 MINWRK = BDSPAC + 2*M*M + 3*M
              ELSE IF( WNTQS ) THEN
*
*                Path 3t (N much larger than M, JOBZ='S')
*
                 WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, M+M*ILAENV( 1, 'DORGLQ', ' ', M,
     $                   N, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+2*M*
     $                   ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'PRT', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*M )
```

```
                 MAXWRK = WRKBL + M*M
                 MINWRK = BDSPAC + M*M + 3*M
              ELSE IF( WNTQA ) THEN
*
*                Path 4t (N much larger than M, JOBZ='A')
*
                 WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, M+N*ILAENV( 1, 'DORGLQ', ' ', N,
     $                   N, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+2*M*
     $                   ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'PRT', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*M )
                 MAXWRK = WRKBL + M*M
                 MINWRK = BDSPAC + M*M + 3*M
              END IF
           ELSE
*
*             Path 5t (N greater than M, but not much larger)
*
              WRKBL = 3*M + ( M+N )*ILAENV( 1, 'DGEBRD', ' ', M, N, -1,
     $                -1 )
              IF( WNTQN ) THEN
                 MAXWRK = MAX( WRKBL, BDSPAC+3*M )
                 MINWRK = 3*M + MAX( N, BDSPAC )
              ELSE IF( WNTQO ) THEN
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'PRT', M, N, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*M )
                 MAXWRK = WRKBL + M*N
                 MINWRK = 3*M + MAX( N, M*M+BDSPAC )
              ELSE IF( WNTQS ) THEN
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'PRT', M, N, M, -1 ) )
                 MAXWRK = MAX( WRKBL, BDSPAC+3*M )
                 MINWRK = 3*M + MAX( N, BDSPAC )
              ELSE IF( WNTQA ) THEN
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'PRT', N, N, M, -1 ) )
                 MAXWRK = MAX( WRKBL, BDSPAC+3*M )
                 MINWRK = 3*M + MAX( N, BDSPAC )
```

```
            END IF
         END IF
      END IF
      MAXWRK = MAX( MAXWRK, MINWRK )
      WORK( 1 ) = MAXWRK
*
      IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
         INFO = -12
      END IF
   END IF
*
   IF( INFO.NE.0 ) THEN
      CALL XERBLA( 'DGESDD', -INFO )
      RETURN
   ELSE IF( LQUERY ) THEN
      RETURN
   END IF
*
*     Quick return if possible
*
   IF( M.EQ.0 .OR. N.EQ.0 ) THEN
      RETURN
   END IF
*
*     Get machine constants
*
   EPS = DLAMCH( 'P' )
   SMLNUM = SQRT( DLAMCH( 'S' ) ) / EPS
   BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
   ANRM = DLANGE( 'M', M, N, A, LDA, DUM )
   ISCL = 0
   IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
      ISCL = 1
      CALL DLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, IERR )
   ELSE IF( ANRM.GT.BIGNUM ) THEN
      ISCL = 1
      CALL DLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, IERR )
   END IF
*
   IF( M.GE.N ) THEN
*
*        A has at least as many rows as columns. If A has sufficiently
*        more rows than columns, first reduce using the QR
*        decomposition (if sufficient workspace available)
*
      IF( M.GE.MNTHR ) THEN
*
```

```
            IF( WNTQN ) THEN
*
*              Path 1 (M much larger than N, JOBZ='N')
*              No singular vectors to be computed
*
               ITAU = 1
               NWORK = ITAU + N
*
*              Compute A=Q*R
*              (Workspace: need 2*N, prefer N+N*NB)
*
               CALL DGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Zero out below R
*
               CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ), LDA )
               IE = 1
               ITAUQ = IE + N
               ITAUP = ITAUQ + N
               NWORK = ITAUP + N
*
*              Bidiagonalize R in A
*              (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
               CALL DGEBRD( N, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                      IERR )
               NWORK = IE + N
*
*              Perform bidiagonal SVD, computing singular values only
*              (Workspace: need N+BDSPAC)
*
               CALL DBDSDC( 'U', 'N', N, S, WORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, WORK( NWORK ), IWORK, INFO )
*
            ELSE IF( WNTQO ) THEN
*
*              Path 2 (M much larger than N, JOBZ = 'O')
*              N left singular vectors to be overwritten on A and
*              N right singular vectors to be computed in VT
*
               IR = 1
*
*              WORK(IR) is LDWRKR by N
*
               IF( LWORK.GE.LDA*N+N*N+3*N+BDSPAC ) THEN
                  LDWRKR = LDA
               ELSE
                  LDWRKR = ( LWORK-N*N-3*N-BDSPAC ) / N
```

```
               END IF
               ITAU = IR + LDWRKR*N
               NWORK = ITAU + N
*
*              Compute A=Q*R
*              (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
               CALL DGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy R to WORK(IR), zeroing out below it
*
               CALL DLACPY( 'U', N, N, A, LDA, WORK( IR ), LDWRKR )
               CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, WORK( IR+1 ),
     $                      LDWRKR )
*
*              Generate Q in A
*              (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
               CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
               IE = ITAU
               ITAUQ = IE + N
               ITAUP = ITAUQ + N
               NWORK = ITAUP + N
*
*              Bidiagonalize R in VT, copying result to WORK(IR)
*              (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
               CALL DGEBRD( N, N, WORK( IR ), LDWRKR, S, WORK( IE ),
     $                      WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              WORK(IU) is N by N
*
               IU = NWORK
               NWORK = IU + N*N
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in WORK(IU) and computing right
*              singular vectors of bidiagonal matrix in VT
*              (Workspace: need N+N*N+BDSPAC)
*
               CALL DBDSDC( 'U', 'I', N, S, WORK( IE ), WORK( IU ), N,
     $                      VT, LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                      INFO )
*
*              Overwrite WORK(IU) by left singular vectors of R
*              and VT by right singular vectors of R
*              (Workspace: need 2*N*N+3*N, prefer 2*N*N+2*N+N*NB)
```

```
*
               CALL DORMBR( 'Q', 'L', 'N', N, N, N, WORK( IR ), LDWRKR,
     $                      WORK( ITAUQ ), WORK( IU ), N, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
               CALL DORMBR( 'P', 'R', 'T', N, N, N, WORK( IR ), LDWRKR,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Multiply Q in A by left singular vectors of R in
*              WORK(IU), storing result in WORK(IR) and copying to A
*              (Workspace: need 2*N*N, prefer N*N+M*N)
*
               DO 10 I = 1, M, LDWRKR
                  CHUNK = MIN( M-I+1, LDWRKR )
                  CALL DGEMM( 'N', 'N', CHUNK, N, N, ONE, A( I, 1 ),
     $                        LDA, WORK( IU ), N, ZERO, WORK( IR ),
     $                        LDWRKR )
                  CALL DLACPY( 'F', CHUNK, N, WORK( IR ), LDWRKR,
     $                         A( I, 1 ), LDA )
   10          CONTINUE
*
            ELSE IF( WNTQS ) THEN
*
               Path 3 (M much larger than N, JOBZ='S')
*              N left singular vectors to be computed in U and
*              N right singular vectors to be computed in VT
*
               IR = 1
*
*              WORK(IR) is N by N
*
               LDWRKR = N
               ITAU = IR + LDWRKR*N
               NWORK = ITAU + N
*
*              Compute A=Q*R
*              (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
               CALL DGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy R to WORK(IR), zeroing out below it
*
               CALL DLACPY( 'U', N, N, A, LDA, WORK( IR ), LDWRKR )
               CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, WORK( IR+1 ),
     $                      LDWRKR )
*
*              Generate Q in A
*              (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
```

```
            CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                   WORK( NWORK ), LWORK-NWORK+1, IERR )
            IE = ITAU
            ITAUQ = IE + N
            ITAUP = ITAUQ + N
            NWORK = ITAUP + N
*
*           Bidiagonalize R in WORK(IR)
*           (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
            CALL DGEBRD( N, N, WORK( IR ), LDWRKR, S, WORK( IE ),
     $                   WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                   LWORK-NWORK+1, IERR )
*
*           Perform bidiagonal SVD, computing left singular vectors
*           of bidiagoal matrix in U and computing right singular
*           vectors of bidiagonal matrix in VT
*           (Workspace: need N+BDSPAC)
*
            CALL DBDSDC( 'U', 'I', N, S, WORK( IE ), U, LDU, VT,
     $                   LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                   INFO )
*
*           Overwrite U by left singular vectors of R and VT
*           by right singular vectors of R
*           (Workspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*
            CALL DORMBR( 'Q', 'L', 'N', N, N, N, WORK( IR ), LDWRKR,
     $                   WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                   LWORK-NWORK+1, IERR )
*
            CALL DORMBR( 'P', 'R', 'T', N, N, N, WORK( IR ), LDWRKR,
     $                   WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                   LWORK-NWORK+1, IERR )
*
*           Multiply Q in A by left singular vectors of R in
*           WORK(IR), storing result in U
*           (Workspace: need N*N)
*
            CALL DLACPY( 'F', N, N, U, LDU, WORK( IR ), LDWRKR )
            CALL DGEMM( 'N', 'N', M, N, N, ONE, A, LDA, WORK( IR ),
     $                  LDWRKR, ZERO, U, LDU )
*
         ELSE IF( WNTQA ) THEN
*
*           Path 4 (M much larger than N, JOBZ='A')
*           M left singular vectors to be computed in U and
*           N right singular vectors to be computed in VT
*
            IU = 1
```

```
*
*               WORK(IU) is N by N
*
                LDWRKU = N
                ITAU = IU + LDWRKU*N
                NWORK = ITAU + N
*
*               Compute A=Q*R, copying result to U
*               (Workspace: need N*N+N+M, prefer N*N+N+M*NB)
*
                CALL DGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                       LWORK-NWORK+1, IERR )
                CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*               Generate Q in U
*               (Workspace: need N*N+N+M, prefer N*N+N+M*NB)
                CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                       WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*               Produce R in A, zeroing out other entries
*
                CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ), LDA )
                IE = ITAU
                ITAUQ = IE + N
                ITAUP = ITAUQ + N
                NWORK = ITAUP + N
*
*               Bidiagonalize R in A
*               (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                CALL DGEBRD( N, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                       WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                       IERR )
*
*               Perform bidiagonal SVD, computing left singular vectors
*               of bidiagonal matrix in WORK(IU) and computing right
*               singular vectors of bidiagonal matrix in VT
*               (Workspace: need N+N*N+BDSPAC)
*
                CALL DBDSDC( 'U', 'I', N, S, WORK( IE ), WORK( IU ), N,
     $                       VT, LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                       INFO )
*
*               Overwrite WORK(IU) by left singular vectors of R and VT
*               by right singular vectors of R
*               (Workspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*
                CALL DORMBR( 'Q', 'L', 'N', N, N, N, A, LDA,
     $                       WORK( ITAUQ ), WORK( IU ), LDWRKU,
     $                       WORK( NWORK ), LWORK-NWORK+1, IERR )
```

```
               CALL DORMBR( 'P', 'R', 'T', N, N, N, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Multiply Q in U by left singular vectors of R in
*              WORK(IU), storing result in A
*              (Workspace: need N*N)
*
               CALL DGEMM( 'N', 'N', M, N, N, ONE, U, LDU, WORK( IU ),
     $                     LDWRKU, ZERO, A, LDA )
*
*              Copy left singular vectors of A from A to U
*
               CALL DLACPY( 'F', M, N, A, LDA, U, LDU )
*
            END IF
*
         ELSE
*
            M .LT. MNTHR
*
            Path 5 (M at least N, but not much larger)
            Reduce to bidiagonal form without QR decomposition
*
            IE = 1
            ITAUQ = IE + N
            ITAUP = ITAUQ + N
            NWORK = ITAUP + N
*
            Bidiagonalize A
            (Workspace: need 3*N+M, prefer 3*N+(M+N)*NB)
*
            CALL DGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                   WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                   IERR )
            IF( WNTQN ) THEN
*
               Perform bidiagonal SVD, only computing singular values
               (Workspace: need N+BDSPAC)
*
               CALL DBDSDC( 'U', 'N', N, S, WORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, WORK( NWORK ), IWORK, INFO )
            ELSE IF( WNTQO ) THEN
               IU = NWORK
               IF( LWORK.GE.M*N+3*N+BDSPAC ) THEN
*
                  WORK( IU ) is M by N
*
                  LDWRKU = M
                  NWORK = IU + LDWRKU*N
```

```
                 CALL DLASET( 'F', M, N, ZERO, ZERO, WORK( IU ),
     $                        LDWRKU )
              ELSE
*
*                WORK( IU ) is N by N
*
                 LDWRKU = N
                 NWORK = IU + LDWRKU*N
*
*                WORK(IR) is LDWRKR by N
*
                 IR = NWORK
                 LDWRKR = ( LWORK-N*N-3*N ) / N
              END IF
              NWORK = IU + LDWRKU*N
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in WORK(IU) and computing right
*             singular vectors of bidiagonal matrix in VT
*             (Workspace: need N+N*N+BDSPAC)
*
              CALL DBDSDC( 'U', 'I', N, S, WORK( IE ), WORK( IU ),
     $                     LDWRKU, VT, LDVT, DUM, IDUM, WORK( NWORK ),
     $                     IWORK, INFO )
*
*             Overwrite VT by right singular vectors of A
*             (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
              CALL DORMBR( 'P', 'R', 'T', N, N, N, A, LDA,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
              IF( LWORK.GE.M*N+3*N+BDSPAC ) THEN
*
*                Overwrite WORK(IU) by left singular vectors of A
*                (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                 CALL DORMBR( 'Q', 'L', 'N', M, N, N, A, LDA,
     $                        WORK( ITAUQ ), WORK( IU ), LDWRKU,
     $                        WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                Copy left singular vectors of A from WORK(IU) to A
*
                 CALL DLACPY( 'F', M, N, WORK( IU ), LDWRKU, A, LDA )
              ELSE
*
*                Generate Q in A
*                (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                 CALL DORGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
```

```
     $                        WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                Multiply Q in A by left singular vectors of
*                bidiagonal matrix in WORK(IU), storing result in
*                WORK(IR) and copying to A
*                (Workspace: need 2*N*N, prefer N*N+M*N)
*
                 DO 20 I = 1, M, LDWRKR
                    CHUNK = MIN( M-I+1, LDWRKR )
                    CALL DGEMM( 'N', 'N', CHUNK, N, N, ONE, A( I, 1 ),
     $                          LDA, WORK( IU ), LDWRKU, ZERO,
     $                          WORK( IR ), LDWRKR )
                    CALL DLACPY( 'F', CHUNK, N, WORK( IR ), LDWRKR,
     $                           A( I, 1 ), LDA )
   20            CONTINUE
              END IF
*
           ELSE IF( WNTQS ) THEN
*
              Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in U and computing right singular
*             vectors of bidiagonal matrix in VT
*             (Workspace: need N+BDSPAC)
*
              CALL DLASET( 'F', M, N, ZERO, ZERO, U, LDU )
              CALL DBDSDC( 'U', 'I', N, S, WORK( IE ), U, LDU, VT,
     $                     LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                     INFO )
*
*             Overwrite U by left singular vectors of A and VT
*             by right singular vectors of A
*             (Workspace: need 3*N, prefer 2*N+N*NB)
*
              CALL DORMBR( 'Q', 'L', 'N', M, N, N, A, LDA,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
              CALL DORMBR( 'P', 'R', 'T', N, N, N, A, LDA,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
           ELSE IF( WNTQA ) THEN
*
              Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in U and computing right singular
*             vectors of bidiagonal matrix in VT
*             (Workspace: need N+BDSPAC)
*
              CALL DLASET( 'F', M, M, ZERO, ZERO, U, LDU )
              CALL DBDSDC( 'U', 'I', N, S, WORK( IE ), U, LDU, VT,
     $                     LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                     INFO )
```

```
*
*             Set the right corner of U to identity matrix
*
              IF( M.GT.N ) THEN
                 CALL DLASET( 'F', M-N, M-N, ZERO, ONE, U( N+1, N+1 ),
     $                        LDU )
              END IF
*
*             Overwrite U by left singular vectors of A and VT
*             by right singular vectors of A
*             (Workspace: need N*N+2*N+M, prefer N*N+2*N+M*NB)
*
              CALL DORMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
              CALL DORMBR( 'P', 'R', 'T', N, N, M, A, LDA,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
           END IF
*
        END IF
*
      ELSE
*
*       A has more columns than rows. If A has sufficiently more
*       columns than rows, first reduce using the LQ decomposition (if
*       sufficient workspace available)
*
        IF( N.GE.MNTHR ) THEN
*
           IF( WNTQN ) THEN
*
*             Path 1t (N much larger than M, JOBZ='N')
*             No singular vectors to be computed
*
              ITAU = 1
              NWORK = ITAU + M
*
*             Compute A=L*Q
*             (Workspace: need 2*M, prefer M+M*NB)
*
              CALL DGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*             Zero out above L
*
              CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ), LDA )
              IE = 1
              ITAUQ = IE + M
              ITAUP = ITAUQ + M
```

```
               NWORK = ITAUP + M
*
*              Bidiagonalize L in A
*              (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
               CALL DGEBRD( M, M, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                      IERR )
               NWORK = IE + M
*
*              Perform bidiagonal SVD, computing singular values only
*              (Workspace: need M+BDSPAC)
*
               CALL DBDSDC( 'U', 'N', M, S, WORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, WORK( NWORK ), IWORK, INFO )
*
            ELSE IF( WNTQO ) THEN
*
*              Path 2t (N much larger than M, JOBZ='O')
*              M right singular vectors to be overwritten on A and
*              M left singular vectors to be computed in U
*
               IVT = 1
*
*              IVT is M by M
*
               IL = IVT + M*M
               IF( LWORK.GE.M*N+M*M+3*M+BDSPAC ) THEN
*
*                 WORK(IL) is M by N
*
                  LDWRKL = M
                  CHUNK = N
               ELSE
                  LDWRKL = M
                  CHUNK = ( LWORK-M*M ) / M
               END IF
               ITAU = IL + LDWRKL*M
               NWORK = ITAU + M
*
*              Compute A=L*Q
*              (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
               CALL DGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy L to WORK(IL), zeroing about above it
*
               CALL DLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWRKL )
               CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
```

```
     $                     WORK( IL+LDWRKL ), LDWRKL )
*
*              Generate Q in A
*              (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
               CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
               IE = ITAU
               ITAUQ = IE + M
               ITAUP = ITAUQ + M
               NWORK = ITAUP + M
*
*              Bidiagonalize L in WORK(IL)
*              (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
               CALL DGEBRD( M, M, WORK( IL ), LDWRKL, S, WORK( IE ),
     $                      WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in U, and computing right singular
*              vectors of bidiagonal matrix in WORK(IVT)
*              (Workspace: need M+M*M+BDSPAC)
*
               CALL DBDSDC( 'U', 'I', M, S, WORK( IE ), U, LDU,
     $                      WORK( IVT ), M, DUM, IDUM, WORK( NWORK ),
     $                      IWORK, INFO )
*
*              Overwrite U by left singular vectors of L and WORK(IVT)
*              by right singular vectors of L
*              (Workspace: need 2*M*M+3*M, prefer 2*M*M+2*M+M*NB)
*
               CALL DORMBR( 'Q', 'L', 'N', M, M, M, WORK( IL ), LDWRKL,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
               CALL DORMBR( 'P', 'R', 'T', M, M, M, WORK( IL ), LDWRKL,
     $                      WORK( ITAUP ), WORK( IVT ), M,
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Multiply right singular vectors of L in WORK(IVT) by Q
*              in A, storing result in WORK(IL) and copying to A
*              (Workspace: need 2*M*M, prefer M*M+M*N)
*
               DO 30 I = 1, N, CHUNK
                  BLK = MIN( N-I+1, CHUNK )
                  CALL DGEMM( 'N', 'N', M, BLK, M, ONE, WORK( IVT ), M,
     $                        A( 1, I ), LDA, ZERO, WORK( IL ), LDWRKL )
                  CALL DLACPY( 'F', M, BLK, WORK( IL ), LDWRKL,
     $                         A( 1, I ), LDA )
   30          CONTINUE
```

```
*
            ELSE IF( WNTQS ) THEN
*
*              Path 3t (N much larger than M, JOBZ='S')
*              M right singular vectors to be computed in VT and
*              M left singular vectors to be computed in U
*
               IL = 1
*
*              WORK(IL) is M by M
*
               LDWRKL = M
               ITAU = IL + LDWRKL*M
               NWORK = ITAU + M
*
*              Compute A=L*Q
*              (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
               CALL DGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy L to WORK(IL), zeroing out above it
*
               CALL DLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWRKL )
               CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                      WORK( IL+LDWRKL ), LDWRKL )
*
*              Generate Q in A
*              (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
               CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
               IE = ITAU
               ITAUQ = IE + M
               ITAUP = ITAUQ + M
               NWORK = ITAUP + M
*
*              Bidiagonalize L in WORK(IU), copying result to U
*              (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
               CALL DGEBRD( M, M, WORK( IL ), LDWRKL, S, WORK( IE ),
     $                      WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in U and computing right singular
*              vectors of bidiagonal matrix in VT
*              (Workspace: need M+BDSPAC)
*
               CALL DBDSDC( 'U', 'I', M, S, WORK( IE ), U, LDU, VT,
```

```
     $                     LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                     INFO )
*
*              Overwrite U by left singular vectors of L and VT
*              by right singular vectors of L
*              (Workspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*
               CALL DORMBR( 'Q', 'L', 'N', M, M, M, WORK( IL ), LDWRKL,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
               CALL DORMBR( 'P', 'R', 'T', M, M, M, WORK( IL ), LDWRKL,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*              Multiply right singular vectors of L in WORK(IL) by
*              Q in A, storing result in VT
*              (Workspace: need M*M)
*
               CALL DLACPY( 'F', M, M, VT, LDVT, WORK( IL ), LDWRKL )
               CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IL ), LDWRKL,
     $                    A, LDA, ZERO, VT, LDVT )
*
            ELSE IF( WNTQA ) THEN
*
*              Path 4t (N much larger than M, JOBZ='A')
*              N right singular vectors to be computed in VT and
*              M left singular vectors to be computed in U
*
               IVT = 1
*
*              WORK(IVT) is M by M
*
               LDWKVT = M
               ITAU = IVT + LDWKVT*M
               NWORK = ITAU + M
*
*              Compute A=L*Q, copying result to VT
*              (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
               CALL DGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
               CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*              Generate Q in VT
*              (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
               CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Produce L in A, zeroing out other entries
```

```
*
               CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ), LDA )
               IE = ITAU
               ITAUQ = IE + M
               ITAUP = ITAUQ + M
               NWORK = ITAUP + M
*
*              Bidiagonalize L in A
*              (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
               CALL DGEBRD( M, M, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                      IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in U and computing right singular
*              vectors of bidiagonal matrix in WORK(IVT)
*              (Workspace: need M+M*M+BDSPAC)
*
               CALL DBDSDC( 'U', 'I', M, S, WORK( IE ), U, LDU,
     $                      WORK( IVT ), LDWKVT, DUM, IDUM,
     $                      WORK( NWORK ), IWORK, INFO )
*
*              Overwrite U by left singular vectors of L and WORK(IVT)
*              by right singular vectors of L
*              (Workspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*
               CALL DORMBR( 'Q', 'L', 'N', M, M, M, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
               CALL DORMBR( 'P', 'R', 'T', M, M, M, A, LDA,
     $                      WORK( ITAUP ), WORK( IVT ), LDWKVT,
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Multiply right singular vectors of L in WORK(IVT) by
*              Q in VT, storing result in A
*              (Workspace: need M*M)
*
               CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IVT ), LDWKVT,
     $                     VT, LDVT, ZERO, A, LDA )
*
*              Copy right singular vectors of A from A to VT
*
               CALL DLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
            END IF
*
         ELSE
*
*           N .LT. MNTHR
```

```
*
*           Path 5t (N greater than M, but not much larger)
*           Reduce to bidiagonal form without LQ decomposition
*
            IE = 1
            ITAUQ = IE + M
            ITAUP = ITAUQ + M
            NWORK = ITAUP + M
*
*           Bidiagonalize A
*           (Workspace: need 3*M+N, prefer 3*M+(M+N)*NB)
*
            CALL DGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                   WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                   IERR )
            IF( WNTQN ) THEN
*
*              Perform bidiagonal SVD, only computing singular values
*              (Workspace: need M+BDSPAC)
*
               CALL DBDSDC( 'L', 'N', M, S, WORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, WORK( NWORK ), IWORK, INFO )
            ELSE IF( WNTQO ) THEN
               LDWKVT = M
               IVT = NWORK
               IF( LWORK.GE.M*N+3*M+BDSPAC ) THEN
*
*                 WORK( IVT ) is M by N
*
                  CALL DLASET( 'F', M, N, ZERO, ZERO, WORK( IVT ),
     $                         LDWKVT )
                  NWORK = IVT + LDWKVT*N
               ELSE
*
*                 WORK( IVT ) is M by M
*
                  NWORK = IVT + LDWKVT*M
                  IL = NWORK
*
*                 WORK(IL) is M by CHUNK
*
                  CHUNK = ( LWORK-M*M-3*M ) / M
               END IF
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in U and computing right singular
*              vectors of bidiagonal matrix in WORK(IVT)
*              (Workspace: need M*M+BDSPAC)
*
               CALL DBDSDC( 'L', 'I', M, S, WORK( IE ), U, LDU,
```

```
     $                     WORK( IVT ), LDWKVT, DUM, IDUM,
     $                     WORK( NWORK ), IWORK, INFO )
*
*             Overwrite U by left singular vectors of A
*             (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
              CALL DORMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
              IF( LWORK.GE.M*N+3*M+BDSPAC ) THEN
*
*                Overwrite WORK(IVT) by left singular vectors of A
*                (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                 CALL DORMBR( 'P', 'R', 'T', M, N, M, A, LDA,
     $                        WORK( ITAUP ), WORK( IVT ), LDWKVT,
     $                        WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                Copy right singular vectors of A from WORK(IVT) to A
*
                 CALL DLACPY( 'F', M, N, WORK( IVT ), LDWKVT, A, LDA )
              ELSE
*
*                Generate P**T in A
*                (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                 CALL DORGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                        WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                Multiply Q in A by right singular vectors of
*                bidiagonal matrix in WORK(IVT), storing result in
*                WORK(IL) and copying to A
*                (Workspace: need 2*M*M, prefer M*M+M*N)
*
                 DO 40 I = 1, N, CHUNK
                    BLK = MIN( N-I+1, CHUNK )
                    CALL DGEMM( 'N', 'N', M, BLK, M, ONE, WORK( IVT ),
     $                          LDWKVT, A( 1, I ), LDA, ZERO,
     $                          WORK( IL ), M )
                    CALL DLACPY( 'F', M, BLK, WORK( IL ), M, A( 1, I ),
     $                           LDA )
   40            CONTINUE
              END IF
           ELSE IF( WNTQS ) THEN
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in U and computing right singular
*             vectors of bidiagonal matrix in VT
*             (Workspace: need M+BDSPAC)
```

```
*
               CALL DLASET( 'F', M, N, ZERO, ZERO, VT, LDVT )
               CALL DBDSDC( 'L', 'I', M, S, WORK( IE ), U, LDU, VT,
     $                      LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                      INFO )
*
*              Overwrite U by left singular vectors of A and VT
*              by right singular vectors of A
*              (Workspace: need 3*M, prefer 2*M+M*NB)
*
               CALL DORMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
               CALL DORMBR( 'P', 'R', 'T', M, N, M, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
            ELSE IF( WNTQA ) THEN
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in U and computing right singular
*              vectors of bidiagonal matrix in VT
*              (Workspace: need M+BDSPAC)
*
               CALL DLASET( 'F', N, N, ZERO, ZERO, VT, LDVT )
               CALL DBDSDC( 'L', 'I', M, S, WORK( IE ), U, LDU, VT,
     $                      LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                      INFO )
*
*              Set the right corner of VT to identity matrix
*
               IF( N.GT.M ) THEN
                  CALL DLASET( 'F', N-M, N-M, ZERO, ONE, VT( M+1, M+1 ),
     $                         LDVT )
               END IF
*
*              Overwrite U by left singular vectors of A and VT
*              by right singular vectors of A
*              (Workspace: need 2*M+N, prefer 2*M+N*NB)
*
               CALL DORMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
               CALL DORMBR( 'P', 'R', 'T', N, N, M, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
            END IF
*
         END IF
*
      END IF
```

```
*
*     Undo scaling if necessary
*
      IF( ISCL.EQ.1 ) THEN
         IF( ANRM.GT.BIGNUM )
     $      CALL DLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( ANRM.LT.SMLNUM )
     $      CALL DLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
      END IF
*
*     Return optimal workspace in WORK(1)
*
      WORK( 1 ) = MAXWRK
*
      RETURN
*
*     End of DGESDD
*
      END
```

———-

28.2.4 lisp code

— **dgesdd.lisp** —

———-

28.3 dgesvd

28.3.1 Axiom unit tests

— **dgesvd.input** —

```
)set break resume
)sys rm -f dgesvd.output
)spool dgesvd.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

28.3.2 Axiom help page

— **dgesvd.help** —

```
=====================================================================
dgesvd examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgesvd (JOBU, JOBVT, M, N, A, LDA, S, U, LDU, VT, LDVT,
           WORK, LWORK, INFO)
            DGESVD computes the singular value decomposition (SVD) for GE
           matrices

Function/Subroutine Documentation
   subroutine dgesvd (character JOBU, character JOBVT, integer M, integer N,
       double precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( * ) S, double precision, dimension( ldu, * ) U, integer LDU,
       double precision, dimension( ldvt, * ) VT, integer LDVT, double
       precision, dimension( * ) WORK, integer LWORK, integer INFO)
        DGESVD computes the singular value decomposition (SVD) for GE matrices

Purpose:
   DGESVD computes the singular value decomposition (SVD) of a real
   M-by-N matrix A, optionally computing the left and/or right singular
   vectors. The SVD is written

   A = U * SIGMA * transpose(V)

   where SIGMA is an M-by-N matrix which is zero except for its
   min(m,n) diagonal elements, U is an M-by-M orthogonal matrix, and
   V is an N-by-N orthogonal matrix.  The diagonal elements of SIGMA
   are the singular values of A; they are real and non-negative, and
   are returned in descending order.  The first min(m,n) columns of
   U and V are the left and right singular vectors of A.

   Note that the routine returns V**T, not V.
```

```
Parameters:

 JOBU
   JOBU is CHARACTER*1
   Specifies options for computing all or part of the matrix U:
   = 'A':  all M columns of U are returned in array U:
   = 'S':  the first min(m,n) columns of U (the left singular
           vectors) are returned in the array U;
   = 'O':  the first min(m,n) columns of U (the left singular
           vectors) are overwritten on the array A;
   = 'N':  no columns of U (no left singular vectors) are computed.

 JOBVT
   JOBVT is CHARACTER*1
   Specifies options for computing all or part of the matrix V**T:
   = 'A':  all N rows of V**T are returned in the array VT;
   = 'S':  the first min(m,n) rows of V**T (the right singular
           vectors) are returned in the array VT;
   = 'O':  the first min(m,n) rows of V**T (the right singular
           vectors) are overwritten on the array A;
   = 'N':  no rows of V**T (no right singular vectors) are computed.

   JOBVT and JOBU cannot both be 'O'.

 M
   M is INTEGER
   The number of rows of the input matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the input matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit,
   if JOBU = 'O',  A is overwritten with the first min(m,n)
                   columns of U (the left singular vectors,
                   stored columnwise);
   if JOBVT = 'O', A is overwritten with the first min(m,n)
                   rows of V**T (the right singular vectors,
                   stored rowwise);
   if JOBU .ne. 'O' and JOBVT .ne. 'O', the contents of A are destroyed.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 S
```

```
  S is DOUBLE PRECISION array, dimension (min(M,N))
  The singular values of A, sorted so that S(i) >= S(i+1).

U
  U is DOUBLE PRECISION array, dimension (LDU,UCOL)
  (LDU,M) if JOBU = 'A' or (LDU,min(M,N)) if JOBU = 'S'.
  If JOBU = 'A', U contains the M-by-M orthogonal matrix U;
  if JOBU = 'S', U contains the first min(m,n) columns of U
  (the left singular vectors, stored columnwise);
  if JOBU = 'N' or 'O', U is not referenced.

LDU
  LDU is INTEGER
  The leading dimension of the array U.  LDU >= 1; if
  JOBU = 'S' or 'A', LDU >= M.

VT
  VT is DOUBLE PRECISION array, dimension (LDVT,N)
  If JOBVT = 'A', VT contains the N-by-N orthogonal matrix V**T;
  if JOBVT = 'S', VT contains the first min(m,n) rows of
  V**T (the right singular vectors, stored rowwise);
  if JOBVT = 'N' or 'O', VT is not referenced.

LDVT
  LDVT is INTEGER
  The leading dimension of the array VT.  LDVT >= 1; if
  JOBVT = 'A', LDVT >= N; if JOBVT = 'S', LDVT >= min(M,N).

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK;
  if INFO > 0, WORK(2:MIN(M,N)) contains the unconverged
  superdiagonal elements of an upper bidiagonal matrix B
  whose diagonal is in S (not necessarily sorted). B
  satisfies A = U * B * VT, so it has the same singular values
  as A, and singular vectors related by U and VT.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  LWORK >= MAX(1,5*MIN(M,N)) for the paths (see comments inside code):
  - PATH 1  (M much larger than N, JOBU='N')
  - PATH 1t (N much larger than M, JOBVT='N')
  LWORK >= MAX(1,3*MIN(M,N)+MAX(M,N),5*MIN(M,N)) for the other paths
  For good performance, LWORK should generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.
```

```
 INFO
   INFO is INTEGER
   = 0:  successful exit.
   < 0:  if INFO = -i, the i-th argument had an illegal value.
   > 0:  if DBDSQR did not converge, INFO specifies how many
         superdiagonals of an intermediate bidiagonal form B
         did not converge to zero. See the description of WORK
         above for details.
```

———-

28.3.3 fortran code

— **dgesvd.f** —

```
*  =====================================================================
      SUBROUTINE DGESVD( JOBU, JOBVT, M, N, A, LDA, S, U, LDU,
     $                   VT, LDVT, WORK, LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          JOBU, JOBVT
      INTEGER            INFO, LDA, LDU, LDVT, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), S( * ), U( LDU, * ),
     $                   VT( LDVT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, WNTUA, WNTUAS, WNTUN, WNTUO, WNTUS,
     $                   WNTVA, WNTVAS, WNTVN, WNTVO, WNTVS
      INTEGER            BDSPAC, BLK, CHUNK, I, IE, IERR, IR, ISCL,
     $                   ITAU, ITAUP, ITAUQ, IU, IWORK, LDWRKR, LDWRKU,
     $                   MAXWRK, MINMN, MINWRK, MNTHR, NCU, NCVT, NRU,
     $                   NRVT, WRKBL
```

```
      INTEGER            LWORK_DGEQRF, LWORK_DORGQR_N, LWORK_DORGQR_M,
     $                   LWORK_DGEBRD, LWORK_DORGBR_P, LWORK_DORGBR_Q,
     $                   LWORK_DGELQF, LWORK_DORGLQ_N, LWORK_DORGLQ_M
      DOUBLE PRECISION   ANRM, BIGNUM, EPS, SMLNUM
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DBDSQR, DGEBRD, DGELQF, DGEMM, DGEQRF, DLACPY,
     $                   DLASCL, DLASET, DORGBR, DORGLQ, DORGQR, DORMBR,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           LSAME, ILAENV, DLAMCH, DLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      MINMN = MIN( M, N )
      WNTUA = LSAME( JOBU, 'A' )
      WNTUS = LSAME( JOBU, 'S' )
      WNTUAS = WNTUA .OR. WNTUS
      WNTUO = LSAME( JOBU, 'O' )
      WNTUN = LSAME( JOBU, 'N' )
      WNTVA = LSAME( JOBVT, 'A' )
      WNTVS = LSAME( JOBVT, 'S' )
      WNTVAS = WNTVA .OR. WNTVS
      WNTVO = LSAME( JOBVT, 'O' )
      WNTVN = LSAME( JOBVT, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
*
      IF( .NOT.( WNTUA .OR. WNTUS .OR. WNTUO .OR. WNTUN ) ) THEN
         INFO = -1
      ELSE IF( .NOT.( WNTVA .OR. WNTVS .OR. WNTVO .OR. WNTVN ) .OR.
     $         ( WNTVO .AND. WNTUO ) ) THEN
         INFO = -2
      ELSE IF( M.LT.0 ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
```

```
         INFO = -6
      ELSE IF( LDU.LT.1 .OR. ( WNTUAS .AND. LDU.LT.M ) ) THEN
         INFO = -9
      ELSE IF( LDVT.LT.1 .OR. ( WNTVA .AND. LDVT.LT.N ) .OR.
     $         ( WNTVS .AND. LDVT.LT.MINMN ) ) THEN
         INFO = -11
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = 1
         MAXWRK = 1
         IF( M.GE.N .AND. MINMN.GT.0 ) THEN
*
*           Compute space needed for DBDSQR
*
            MNTHR = ILAENV( 6, 'DGESVD', JOBU // JOBVT, M, N, 0, 0 )
            BDSPAC = 5*N
*           Compute space needed for DGEQRF
            CALL DGEQRF( M, N, A, LDA, DUM(1), DUM(1), -1, IERR )
            LWORK_DGEQRF=DUM(1)
*           Compute space needed for DORGQR
            CALL DORGQR( M, N, N, A, LDA, DUM(1), DUM(1), -1, IERR )
            LWORK_DORGQR_N=DUM(1)
            CALL DORGQR( M, M, N, A, LDA, DUM(1), DUM(1), -1, IERR )
            LWORK_DORGQR_M=DUM(1)
*           Compute space needed for DGEBRD
            CALL DGEBRD( N, N, A, LDA, S, DUM(1), DUM(1),
     $                   DUM(1), DUM(1), -1, IERR )
            LWORK_DGEBRD=DUM(1)
*           Compute space needed for DORGBR P
            CALL DORGBR( 'P', N, N, N, A, LDA, DUM(1),
     $                   DUM(1), -1, IERR )
            LWORK_DORGBR_P=DUM(1)
*           Compute space needed for DORGBR Q
            CALL DORGBR( 'Q', N, N, N, A, LDA, DUM(1),
     $                   DUM(1), -1, IERR )
            LWORK_DORGBR_Q=DUM(1)
*
            IF( M.GE.MNTHR ) THEN
               IF( WNTUN ) THEN
*
*                 Path 1 (M much larger than N, JOBU='N')
*
```

```
                 MAXWRK = N + LWORK_DGEQRF
                 MAXWRK = MAX( MAXWRK, 3*N+LWORK_DGEBRD )
                 IF( WNTVO .OR. WNTVAS )
     $              MAXWRK = MAX( MAXWRK, 3*N+LWORK_DORGBR_P )
                 MAXWRK = MAX( MAXWRK, BDSPAC )
                 MINWRK = MAX( 4*N, BDSPAC )
              ELSE IF( WNTUO .AND. WNTVN ) THEN
*
*                Path 2 (M much larger than N, JOBU='O', JOBVT='N')
*
                 WRKBL = N + LWORK_DGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_DORGQR_N )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = MAX( N*N+WRKBL, N*N+M*N+N )
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUO .AND. WNTVAS ) THEN
*
*                Path 3 (M much larger than N, JOBU='O', JOBVT='S' or
*                'A')
*
                 WRKBL = N + LWORK_DGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_DORGQR_N )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = MAX( N*N+WRKBL, N*N+M*N+N )
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUS .AND. WNTVN ) THEN
*
*                Path 4 (M much larger than N, JOBU='S', JOBVT='N')
*
                 WRKBL = N + LWORK_DGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_DORGQR_N )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUS .AND. WNTVO ) THEN
*
*                Path 5 (M much larger than N, JOBU='S', JOBVT='O')
*
                 WRKBL = N + LWORK_DGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_DORGQR_N )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_P )
```

```
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = 2*N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUS .AND. WNTVAS ) THEN
*
*                Path 6 (M much larger than N, JOBU='S', JOBVT='S' or
*                'A')
*
                 WRKBL = N + LWORK_DGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_DORGQR_N )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUA .AND. WNTVN ) THEN
*
*                Path 7 (M much larger than N, JOBU='A', JOBVT='N')
*
                 WRKBL = N + LWORK_DGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_DORGQR_M )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUA .AND. WNTVO ) THEN
*
*                Path 8 (M much larger than N, JOBU='A', JOBVT='O')
*
                 WRKBL = N + LWORK_DGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_DORGQR_M )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = 2*N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUA .AND. WNTVAS ) THEN
*
*                Path 9 (M much larger than N, JOBU='A', JOBVT='S' or
*                'A')
*
                 WRKBL = N + LWORK_DGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_DORGQR_M )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, BDSPAC )
```

```
                 MAXWRK = N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
              END IF
           ELSE
*
*             Path 10 (M at least N, but not much larger)
*
              CALL DGEBRD( M, N, A, LDA, S, DUM(1), DUM(1),
     $                     DUM(1), DUM(1), -1, IERR )
              LWORK_DGEBRD=DUM(1)
              MAXWRK = 3*N + LWORK_DGEBRD
              IF( WNTUS .OR. WNTUO ) THEN
                 CALL DORGBR( 'Q', M, N, N, A, LDA, DUM(1),
     $                        DUM(1), -1, IERR )
                 LWORK_DORGBR_Q=DUM(1)
                 MAXWRK = MAX( MAXWRK, 3*N+LWORK_DORGBR_Q )
              END IF
              IF( WNTUA ) THEN
                 CALL DORGBR( 'Q', M, M, N, A, LDA, DUM(1),
     $                        DUM(1), -1, IERR )
                 LWORK_DORGBR_Q=DUM(1)
                 MAXWRK = MAX( MAXWRK, 3*N+LWORK_DORGBR_Q )
              END IF
              IF( .NOT.WNTVN ) THEN
                MAXWRK = MAX( MAXWRK, 3*N+LWORK_DORGBR_P )
              END IF
              MAXWRK = MAX( MAXWRK, BDSPAC )
              MINWRK = MAX( 3*N+M, BDSPAC )
           END IF
        ELSE IF( MINMN.GT.0 ) THEN
*
*          Compute space needed for DBDSQR
*
           MNTHR = ILAENV( 6, 'DGESVD', JOBU // JOBVT, M, N, 0, 0 )
           BDSPAC = 5*M
*          Compute space needed for DGELQF
           CALL DGELQF( M, N, A, LDA, DUM(1), DUM(1), -1, IERR )
           LWORK_DGELQF=DUM(1)
*          Compute space needed for DORGLQ
           CALL DORGLQ( N, N, M, DUM(1), N, DUM(1), DUM(1), -1, IERR )
           LWORK_DORGLQ_N=DUM(1)
           CALL DORGLQ( M, N, M, A, LDA, DUM(1), DUM(1), -1, IERR )
           LWORK_DORGLQ_M=DUM(1)
*          Compute space needed for DGEBRD
           CALL DGEBRD( M, M, A, LDA, S, DUM(1), DUM(1),
     $                  DUM(1), DUM(1), -1, IERR )
           LWORK_DGEBRD=DUM(1)
*           Compute space needed for DORGBR P
           CALL DORGBR( 'P', M, M, M, A, N, DUM(1),
     $                  DUM(1), -1, IERR )
```

```
            LWORK_DORGBR_P=DUM(1)
*           Compute space needed for DORGBR Q
            CALL DORGBR( 'Q', M, M, M, A, N, DUM(1),
     $                   DUM(1), -1, IERR )
            LWORK_DORGBR_Q=DUM(1)
            IF( N.GE.MNTHR ) THEN
               IF( WNTVN ) THEN
*
*                 Path 1t(N much larger than M, JOBVT='N')
*
                  MAXWRK = M + LWORK_DGELQF
                  MAXWRK = MAX( MAXWRK, 3*M+LWORK_DGEBRD )
                  IF( WNTUO .OR. WNTUAS )
     $               MAXWRK = MAX( MAXWRK, 3*M+LWORK_DORGBR_Q )
                  MAXWRK = MAX( MAXWRK, BDSPAC )
                  MINWRK = MAX( 4*M, BDSPAC )
               ELSE IF( WNTVO .AND. WNTUN ) THEN
*
*                 Path 2t(N much larger than M, JOBU='N', JOBVT='O')
*
                  WRKBL = M + LWORK_DGELQF
                  WRKBL = MAX( WRKBL, M+LWORK_DORGLQ_M )
                  WRKBL = MAX( WRKBL, 3*M+LWORK_DGEBRD )
                  WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_P )
                  WRKBL = MAX( WRKBL, BDSPAC )
                  MAXWRK = MAX( M*M+WRKBL, M*M+M*N+M )
                  MINWRK = MAX( 3*M+N, BDSPAC )
               ELSE IF( WNTVO .AND. WNTUAS ) THEN
*
*                 Path 3t(N much larger than M, JOBU='S' or 'A',
*                 JOBVT='O')
*
                  WRKBL = M + LWORK_DGELQF
                  WRKBL = MAX( WRKBL, M+LWORK_DORGLQ_M )
                  WRKBL = MAX( WRKBL, 3*M+LWORK_DGEBRD )
                  WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_P )
                  WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_Q )
                  WRKBL = MAX( WRKBL, BDSPAC )
                  MAXWRK = MAX( M*M+WRKBL, M*M+M*N+M )
                  MINWRK = MAX( 3*M+N, BDSPAC )
               ELSE IF( WNTVS .AND. WNTUN ) THEN
*
*                 Path 4t(N much larger than M, JOBU='N', JOBVT='S')
*
                  WRKBL = M + LWORK_DGELQF
                  WRKBL = MAX( WRKBL, M+LWORK_DORGLQ_M )
                  WRKBL = MAX( WRKBL, 3*M+LWORK_DGEBRD )
                  WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_P )
                  WRKBL = MAX( WRKBL, BDSPAC )
                  MAXWRK = M*M + WRKBL
```

```
                 MINWRK = MAX( 3*M+N, BDSPAC )
              ELSE IF( WNTVS .AND. WNTUO ) THEN
*
*                Path 5t(N much larger than M, JOBU='O', JOBVT='S')
*
                 WRKBL = M + LWORK_DGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_DORGLQ_M )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = 2*M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
              ELSE IF( WNTVS .AND. WNTUAS ) THEN
*
*                Path 6t(N much larger than M, JOBU='S' or 'A',
*                JOBVT='S')
*
                 WRKBL = M + LWORK_DGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_DORGLQ_M )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
              ELSE IF( WNTVA .AND. WNTUN ) THEN
*
*                Path 7t(N much larger than M, JOBU='N', JOBVT='A')
*
                 WRKBL = M + LWORK_DGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_DORGLQ_N )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
              ELSE IF( WNTVA .AND. WNTUO ) THEN
*
*                Path 8t(N much larger than M, JOBU='O', JOBVT='A')
*
                 WRKBL = M + LWORK_DGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_DORGLQ_N )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = 2*M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
              ELSE IF( WNTVA .AND. WNTUAS ) THEN
```

```
*
*                Path 9t(N much larger than M, JOBU='S' or 'A',
*                JOBVT='A')
*
                 WRKBL = M + LWORK_DGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_DORGLQ_N )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
              END IF
           ELSE
*
*             Path 10t(N greater than M, but not much larger)
*
              CALL DGEBRD( M, N, A, LDA, S, DUM(1), DUM(1),
     $                     DUM(1), DUM(1), -1, IERR )
              LWORK_DGEBRD=DUM(1)
              MAXWRK = 3*M + LWORK_DGEBRD
              IF( WNTVS .OR. WNTVO ) THEN
*               Compute space needed for DORGBR P
                CALL DORGBR( 'P', M, N, M, A, N, DUM(1),
     $                       DUM(1), -1, IERR )
                LWORK_DORGBR_P=DUM(1)
                MAXWRK = MAX( MAXWRK, 3*M+LWORK_DORGBR_P )
              END IF
              IF( WNTVA ) THEN
                CALL DORGBR( 'P', N, N, M, A, N, DUM(1),
     $                       DUM(1), -1, IERR )
                LWORK_DORGBR_P=DUM(1)
                MAXWRK = MAX( MAXWRK, 3*M+LWORK_DORGBR_P )
              END IF
              IF( .NOT.WNTUN ) THEN
                 MAXWRK = MAX( MAXWRK, 3*M+LWORK_DORGBR_Q )
              END IF
              MAXWRK = MAX( MAXWRK, BDSPAC )
              MINWRK = MAX( 3*M+N, BDSPAC )
           END IF
        END IF
        MAXWRK = MAX( MAXWRK, MINWRK )
        WORK( 1 ) = MAXWRK
*
        IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
           INFO = -13
        END IF
     END IF
*
     IF( INFO.NE.0 ) THEN
```

```
         CALL XERBLA( 'DGESVD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = SQRT( DLAMCH( 'S' ) ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = DLANGE( 'M', M, N, A, LDA, DUM )
      ISCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ISCL = 1
         CALL DLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, IERR )
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ISCL = 1
         CALL DLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, IERR )
      END IF
*
      IF( M.GE.N ) THEN
*
*        A has at least as many rows as columns. If A has sufficiently
*        more rows than columns, first reduce using the QR
*        decomposition (if sufficient workspace available)
*
         IF( M.GE.MNTHR ) THEN
*
            IF( WNTUN ) THEN
*
*              Path 1 (M much larger than N, JOBU='N')
*              No left singular vectors to be computed
*
               ITAU = 1
               IWORK = ITAU + N
*
*              Compute A=Q*R
*              (Workspace: need 2*N, prefer N+N*NB)
*
               CALL DGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
```

```
     $                     LWORK-IWORK+1, IERR )
*
*              Zero out below R
*
               CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ), LDA )
               IE = 1
               ITAUQ = IE + N
               ITAUP = ITAUQ + N
               IWORK = ITAUP + N
*
*              Bidiagonalize R in A
*              (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
               CALL DGEBRD( N, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                      IERR )
               NCVT = 0
               IF( WNTVO .OR. WNTVAS ) THEN
*
*                 If right singular vectors desired, generate P'.
*                 (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                  CALL DORGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  NCVT = N
               END IF
               IWORK = IE + N
*
*              Perform bidiagonal QR iteration, computing right
*              singular vectors of A in A if desired
*              (Workspace: need BDSPAC)
*
               CALL DBDSQR( 'U', N, NCVT, 0, 0, S, WORK( IE ), A, LDA,
     $                      DUM, 1, DUM, 1, WORK( IWORK ), INFO )
*
*              If right singular vectors desired in VT, copy them there
*
               IF( WNTVAS )
     $            CALL DLACPY( 'F', N, N, A, LDA, VT, LDVT )
*
            ELSE IF( WNTUO .AND. WNTVN ) THEN
*
*              Path 2 (M much larger than N, JOBU='O', JOBVT='N')
*              N left singular vectors to be overwritten on A and
*              no right singular vectors to be computed
*
               IF( LWORK.GE.N*N+MAX( 4*N, BDSPAC ) ) THEN
*
*                 Sufficient workspace for a fast algorithm
*
```

```
                IR = 1
                IF( LWORK.GE.MAX( WRKBL, LDA*N+N )+LDA*N ) THEN
*
*                  WORK(IU) is LDA by N, WORK(IR) is LDA by N
*
                   LDWRKU = LDA
                   LDWRKR = LDA
                ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N+N )+N*N ) THEN
*
*                  WORK(IU) is LDA by N, WORK(IR) is N by N
*
                   LDWRKU = LDA
                   LDWRKR = N
                ELSE
*
*                  WORK(IU) is LDWRKU by N, WORK(IR) is N by N
*
                   LDWRKU = ( LWORK-N*N-N ) / N
                   LDWRKR = N
                END IF
                ITAU = IR + LDWRKR*N
                IWORK = ITAU + N
*
*               Compute A=Q*R
*               (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                       WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*               Copy R to WORK(IR) and zero out below it
*
                CALL DLACPY( 'U', N, N, A, LDA, WORK( IR ), LDWRKR )
                CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, WORK( IR+1 ),
     $                       LDWRKR )
*
*               Generate Q in A
*               (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                       WORK( IWORK ), LWORK-IWORK+1, IERR )
                IE = ITAU
                ITAUQ = IE + N
                ITAUP = ITAUQ + N
                IWORK = ITAUP + N
*
*               Bidiagonalize R in WORK(IR)
*               (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                CALL DGEBRD( N, N, WORK( IR ), LDWRKR, S, WORK( IE ),
     $                       WORK( ITAUQ ), WORK( ITAUP ),
```

```
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Generate left vectors bidiagonalizing R
*                 (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                  CALL DORGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                        WORK( ITAUQ ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
                  IWORK = IE + N
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of R in WORK(IR)
*                 (Workspace: need N*N+BDSPAC)
*
                  CALL DBDSQR( 'U', N, 0, N, 0, S, WORK( IE ), DUM, 1,
     $                        WORK( IR ), LDWRKR, DUM, 1,
     $                        WORK( IWORK ), INFO )
                  IU = IE + N
*
*                 Multiply Q in A by left singular vectors of R in
*                 WORK(IR), storing result in WORK(IU) and copying to A
*                 (Workspace: need N*N+2*N, prefer N*N+M*N+N)
*
                  DO 10 I = 1, M, LDWRKU
                     CHUNK = MIN( M-I+1, LDWRKU )
                     CALL DGEMM( 'N', 'N', CHUNK, N, N, ONE, A( I, 1 ),
     $                           LDA, WORK( IR ), LDWRKR, ZERO,
     $                           WORK( IU ), LDWRKU )
                     CALL DLACPY( 'F', CHUNK, N, WORK( IU ), LDWRKU,
     $                            A( I, 1 ), LDA )
   10             CONTINUE
*
               ELSE
*
*                 Insufficient workspace for a fast algorithm
*
                  IE = 1
                  ITAUQ = IE + N
                  ITAUP = ITAUQ + N
                  IWORK = ITAUP + N
*
*                 Bidiagonalize A
*                 (Workspace: need 3*N+M, prefer 3*N+(M+N)*NB)
*
                  CALL DGEBRD( M, N, A, LDA, S, WORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Generate left vectors bidiagonalizing A
*                 (Workspace: need 4*N, prefer 3*N+N*NB)
```

```
*
                 CALL DORGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IWORK = IE + N
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of A in A
*                (Workspace: need BDSPAC)
*
                 CALL DBDSQR( 'U', N, 0, M, 0, S, WORK( IE ), DUM, 1,
     $                        A, LDA, DUM, 1, WORK( IWORK ), INFO )
*
              END IF
*
           ELSE IF( WNTUO .AND. WNTVAS ) THEN
*
              Path 3 (M much larger than N, JOBU='O', JOBVT='S' or 'A')
*             N left singular vectors to be overwritten on A and
*             N right singular vectors to be computed in VT
*
              IF( LWORK.GE.N*N+MAX( 4*N, BDSPAC ) ) THEN
*
*                Sufficient workspace for a fast algorithm
*
                 IR = 1
                 IF( LWORK.GE.MAX( WRKBL, LDA*N+N )+LDA*N ) THEN
*
*                   WORK(IU) is LDA by N and WORK(IR) is LDA by N
*
                    LDWRKU = LDA
                    LDWRKR = LDA
                 ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N+N )+N*N ) THEN
*
*                   WORK(IU) is LDA by N and WORK(IR) is N by N
*
                    LDWRKU = LDA
                    LDWRKR = N
                 ELSE
*
*                   WORK(IU) is LDWRKU by N and WORK(IR) is N by N
*
                    LDWRKU = ( LWORK-N*N-N ) / N
                    LDWRKR = N
                 END IF
                 ITAU = IR + LDWRKR*N
                 IWORK = ITAU + N
*
*                Compute A=Q*R
*                (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
```

```
                 CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy R to VT, zeroing out below it
*
                 CALL DLACPY( 'U', N, N, A, LDA, VT, LDVT )
                 IF( N.GT.1 )
     $              CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                           VT( 2, 1 ), LDVT )
*
*                Generate Q in A
*                (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                 CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = ITAU
                 ITAUQ = IE + N
                 ITAUP = ITAUQ + N
                 IWORK = ITAUP + N
*
*                Bidiagonalize R in VT, copying result to WORK(IR)
*                (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                 CALL DGEBRD( N, N, VT, LDVT, S, WORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL DLACPY( 'L', N, N, VT, LDVT, WORK( IR ), LDWRKR )
*
*                Generate left vectors bidiagonalizing R in WORK(IR)
*                (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                 CALL DORGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                        WORK( ITAUQ ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate right vectors bidiagonalizing R in VT
*                (Workspace: need N*N+4*N-1, prefer N*N+3*N+(N-1)*NB)
*
                 CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IWORK = IE + N
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of R in WORK(IR) and computing right
*                singular vectors of R in VT
*                (Workspace: need N*N+BDSPAC)
*
                 CALL DBDSQR( 'U', N, N, N, 0, S, WORK( IE ), VT, LDVT,
     $                        WORK( IR ), LDWRKR, DUM, 1,
     $                        WORK( IWORK ), INFO )
```

```
                 IU = IE + N
*
*                Multiply Q in A by left singular vectors of R in
*                WORK(IR), storing result in WORK(IU) and copying to A
*                (Workspace: need N*N+2*N, prefer N*N+M*N+N)
*
                 DO 20 I = 1, M, LDWRKU
                    CHUNK = MIN( M-I+1, LDWRKU )
                    CALL DGEMM( 'N', 'N', CHUNK, N, N, ONE, A( I, 1 ),
     $                          LDA, WORK( IR ), LDWRKR, ZERO,
     $                          WORK( IU ), LDWRKU )
                    CALL DLACPY( 'F', CHUNK, N, WORK( IU ), LDWRKU,
     $                           A( I, 1 ), LDA )
   20            CONTINUE
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + N
*
*                Compute A=Q*R
*                (Workspace: need 2*N, prefer N+N*NB)
*
                 CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy R to VT, zeroing out below it
*
                 CALL DLACPY( 'U', N, N, A, LDA, VT, LDVT )
                 IF( N.GT.1 )
     $              CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                           VT( 2, 1 ), LDVT )
*
*                Generate Q in A
*                (Workspace: need 2*N, prefer N+N*NB)
*
                 CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = ITAU
                 ITAUQ = IE + N
                 ITAUP = ITAUQ + N
                 IWORK = ITAUP + N
*
*                Bidiagonalize R in VT
*                (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                 CALL DGEBRD( N, N, VT, LDVT, S, WORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
```

```
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Multiply Q in A by left vectors bidiagonalizing R
*                (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                 CALL DORMBR( 'Q', 'R', 'N', M, N, N, VT, LDVT,
     $                        WORK( ITAUQ ), A, LDA, WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate right vectors bidiagonalizing R in VT
*                (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                 CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IWORK = IE + N
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of A in A and computing right
*                singular vectors of A in VT
*                (Workspace: need BDSPAC)
*
                 CALL DBDSQR( 'U', N, N, M, 0, S, WORK( IE ), VT, LDVT,
     $                        A, LDA, DUM, 1, WORK( IWORK ), INFO )
*
              END IF
*
           ELSE IF( WNTUS ) THEN
*
              IF( WNTVN ) THEN
*
*                Path 4 (M much larger than N, JOBU='S', JOBVT='N')
*                N left singular vectors to be computed in U and
*                no right singular vectors to be computed
*
                 IF( LWORK.GE.N*N+MAX( 4*N, BDSPAC ) ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IR = 1
                    IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                      WORK(IR) is LDA by N
*
                       LDWRKR = LDA
                    ELSE
*
*                      WORK(IR) is N by N
*
                       LDWRKR = N
                    END IF
```

```
                    ITAU = IR + LDWRKR*N
                    IWORK = ITAU + N
*
*                   Compute A=Q*R
*                   (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                    CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy R to WORK(IR), zeroing out below it
*
                    CALL DLACPY( 'U', N, N, A, LDA, WORK( IR ),
     $                           LDWRKR )
                    CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                           WORK( IR+1 ), LDWRKR )
*
*                   Generate Q in A
*                   (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                    CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = ITAU
                    ITAUQ = IE + N
                    ITAUP = ITAUQ + N
                    IWORK = ITAUP + N
*
*                   Bidiagonalize R in WORK(IR)
*                   (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                    CALL DGEBRD( N, N, WORK( IR ), LDWRKR, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate left vectors bidiagonalizing R in WORK(IR)
*                   (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                    CALL DORGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    IWORK = IE + N
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of R in WORK(IR)
*                   (Workspace: need N*N+BDSPAC)
*
                    CALL DBDSQR( 'U', N, 0, N, 0, S, WORK( IE ), DUM,
     $                           1, WORK( IR ), LDWRKR, DUM, 1,
     $                           WORK( IWORK ), INFO )
*
```

```
*                    Multiply Q in A by left singular vectors of R in
*                    WORK(IR), storing result in U
*                    (Workspace: need N*N)
*
                     CALL DGEMM( 'N', 'N', M, N, N, ONE, A, LDA,
     $                           WORK( IR ), LDWRKR, ZERO, U, LDU )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (Workspace: need 2*N, prefer N+N*NB)
*
                     CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                    Generate Q in U
*                    (Workspace: need 2*N, prefer N+N*NB)
*
                     CALL DORGQR( M, N, N, U, LDU, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Zero out below R in A
*
                     CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ),
     $                            LDA )
*
*                    Bidiagonalize R in A
*                    (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                     CALL DGEBRD( N, N, A, LDA, S, WORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply Q in U by left vectors bidiagonalizing R
*                    (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                     CALL DORMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                            WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IWORK = IE + N
```

```
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', N, 0, M, 0, S, WORK( IE ), DUM,
     $                            1, U, LDU, DUM, 1, WORK( IWORK ),
     $                            INFO )
*
                  END IF
*
               ELSE IF( WNTVO ) THEN
*
                  Path 5 (M much larger than N, JOBU='S', JOBVT='O')
                  N left singular vectors to be computed in U and
                  N right singular vectors to be overwritten on A
*
                  IF( LWORK.GE.2*N*N+MAX( 4*N, BDSPAC ) ) THEN
*
                     Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+2*LDA*N ) THEN
*
                        WORK(IU) is LDA by N and WORK(IR) is LDA by N
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*N
                        LDWRKR = LDA
                     ELSE IF( LWORK.GE.WRKBL+( LDA+N )*N ) THEN
*
                        WORK(IU) is LDA by N and WORK(IR) is N by N
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*N
                        LDWRKR = N
                     ELSE
*
                        WORK(IU) is N by N and WORK(IR) is N by N
*
                        LDWRKU = N
                        IR = IU + LDWRKU*N
                        LDWRKR = N
                     END IF
                     ITAU = IR + LDWRKR*N
                     IWORK = ITAU + N
*
                     Compute A=Q*R
                     (Workspace: need 2*N*N+2*N, prefer 2*N*N+N+N*NB)
*
```

```
                    CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy R to WORK(IU), zeroing out below it
*
                    CALL DLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                           LDWRKU )
                    CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                           WORK( IU+1 ), LDWRKU )
*
*                   Generate Q in A
*                   (Workspace: need 2*N*N+2*N, prefer 2*N*N+N+N*NB)
*
                    CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = ITAU
                    ITAUQ = IE + N
                    ITAUP = ITAUQ + N
                    IWORK = ITAUP + N
*
*                   Bidiagonalize R in WORK(IU), copying result to
*                   WORK(IR)
*                   (Workspace: need 2*N*N+4*N,
*                               prefer 2*N*N+3*N+2*N*NB)
*
                    CALL DGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    CALL DLACPY( 'U', N, N, WORK( IU ), LDWRKU,
     $                           WORK( IR ), LDWRKR )
*
*                   Generate left bidiagonalizing vectors in WORK(IU)
*                   (Workspace: need 2*N*N+4*N, prefer 2*N*N+3*N+N*NB)
*
                    CALL DORGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate right bidiagonalizing vectors in WORK(IR)
*                   (Workspace: need 2*N*N+4*N-1,
*                               prefer 2*N*N+3*N+(N-1)*NB)
*
                    CALL DORGBR( 'P', N, N, N, WORK( IR ), LDWRKR,
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    IWORK = IE + N
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of R in WORK(IU) and computing
```

```
*                    right singular vectors of R in WORK(IR)
*                    (Workspace: need 2*N*N+BDSPAC)
*
                     CALL DBDSQR( 'U', N, N, N, 0, S, WORK( IE ),
     $                            WORK( IR ), LDWRKR, WORK( IU ),
     $                            LDWRKU, DUM, 1, WORK( IWORK ), INFO )
*
*                    Multiply Q in A by left singular vectors of R in
*                    WORK(IU), storing result in U
*                    (Workspace: need N*N)
*
                     CALL DGEMM( 'N', 'N', M, N, N, ONE, A, LDA,
     $                           WORK( IU ), LDWRKU, ZERO, U, LDU )
*
*                    Copy right singular vectors of R to A
*                    (Workspace: need N*N)
*
                     CALL DLACPY( 'F', N, N, WORK( IR ), LDWRKR, A,
     $                            LDA )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (Workspace: need 2*N, prefer N+N*NB)
*
                     CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                    Generate Q in U
*                    (Workspace: need 2*N, prefer N+N*NB)
*
                     CALL DORGQR( M, N, N, U, LDU, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Zero out below R in A
*
                     CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ),
     $                            LDA )
*
*                    Bidiagonalize R in A
```

```
*                    (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                     CALL DGEBRD( N, N, A, LDA, S, WORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply Q in U by left vectors bidiagonalizing R
*                    (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                     CALL DORMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                            WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right vectors bidiagonalizing R in A
*                    (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                     CALL DORGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in A
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', N, N, M, 0, S, WORK( IE ), A,
     $                            LDA, U, LDU, DUM, 1, WORK( IWORK ),
     $                            INFO )
*
                  END IF
*
               ELSE IF( WNTVAS ) THEN
*
*                 Path 6 (M much larger than N, JOBU='S', JOBVT='S'
*                         or 'A')
*                 N left singular vectors to be computed in U and
*                 N right singular vectors to be computed in VT
*
                  IF( LWORK.GE.N*N+MAX( 4*N, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                       WORK(IU) is LDA by N
*
                        LDWRKU = LDA
                     ELSE
*
```

```
*                       WORK(IU) is N by N
*
                        LDWRKU = N
                     END IF
                     ITAU = IU + LDWRKU*N
                     IWORK = ITAU + N
*
*                    Compute A=Q*R
*                    (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                     CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy R to WORK(IU), zeroing out below it
*
                     CALL DLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                            LDWRKU )
                     CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                            WORK( IU+1 ), LDWRKU )
*
*                    Generate Q in A
*                    (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                     CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Bidiagonalize R in WORK(IU), copying result to VT
*                    (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                     CALL DGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                            WORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'U', N, N, WORK( IU ), LDWRKU, VT,
     $                            LDVT )
*
*                    Generate left bidiagonalizing vectors in WORK(IU)
*                    (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                     CALL DORGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                            WORK( ITAUQ ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in VT
*                    (Workspace: need N*N+4*N-1,
*                                prefer N*N+3*N+(N-1)*NB)
```

```
*
                 CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IWORK = IE + N
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of R in WORK(IU) and computing
*                right singular vectors of R in VT
*                (Workspace: need N*N+BDSPAC)
*
                 CALL DBDSQR( 'U', N, N, N, 0, S, WORK( IE ), VT,
     $                        LDVT, WORK( IU ), LDWRKU, DUM, 1,
     $                        WORK( IWORK ), INFO )
*
*                Multiply Q in A by left singular vectors of R in
*                WORK(IU), storing result in U
*                (Workspace: need N*N)
*
                 CALL DGEMM( 'N', 'N', M, N, N, ONE, A, LDA,
     $                       WORK( IU ), LDWRKU, ZERO, U, LDU )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + N
*
*                Compute A=Q*R, copying result to U
*                (Workspace: need 2*N, prefer N+N*NB)
*
                 CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                Generate Q in U
*                (Workspace: need 2*N, prefer N+N*NB)
*
                 CALL DORGQR( M, N, N, U, LDU, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy R to VT, zeroing out below it
*
                 CALL DLACPY( 'U', N, N, A, LDA, VT, LDVT )
                 IF( N.GT.1 )
     $              CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                           VT( 2, 1 ), LDVT )
                 IE = ITAU
                 ITAUQ = IE + N
                 ITAUP = ITAUQ + N
```

```
                    IWORK = ITAUP + N
*
*                   Bidiagonalize R in VT
*                   (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                    CALL DGEBRD( N, N, VT, LDVT, S, WORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Multiply Q in U by left bidiagonalizing vectors
*                   in VT
*                   (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                    CALL DORMBR( 'Q', 'R', 'N', M, N, N, VT, LDVT,
     $                           WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate right bidiagonalizing vectors in VT
*                   (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                    CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IWORK = IE + N
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of A in U and computing right
*                   singular vectors of A in VT
*                   (Workspace: need BDSPAC)
*
                    CALL DBDSQR( 'U', N, N, M, 0, S, WORK( IE ), VT,
     $                           LDVT, U, LDU, DUM, 1, WORK( IWORK ),
     $                           INFO )
*
                 END IF
*
              END IF
*
           ELSE IF( WNTUA ) THEN
*
              IF( WNTVN ) THEN
*
*                Path 7 (M much larger than N, JOBU='A', JOBVT='N')
*                M left singular vectors to be computed in U and
*                no right singular vectors to be computed
*
                 IF( LWORK.GE.N*N+MAX( N+M, 4*N, BDSPAC ) ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IR = 1
```

```
                    IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                      WORK(IR) is LDA by N
*
                       LDWRKR = LDA
                    ELSE
*
*                      WORK(IR) is N by N
*
                       LDWRKR = N
                    END IF
                    ITAU = IR + LDWRKR*N
                    IWORK = ITAU + N
*
*                   Compute A=Q*R, copying result to U
*                   (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                    CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                   Copy R to WORK(IR), zeroing out below it
*
                    CALL DLACPY( 'U', N, N, A, LDA, WORK( IR ),
     $                           LDWRKR )
                    CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                           WORK( IR+1 ), LDWRKR )
*
*                   Generate Q in U
*                   (Workspace: need N*N+N+M, prefer N*N+N+M*NB)
*
                    CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = ITAU
                    ITAUQ = IE + N
                    ITAUP = ITAUQ + N
                    IWORK = ITAUP + N
*
*                   Bidiagonalize R in WORK(IR)
*                   (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                    CALL DGEBRD( N, N, WORK( IR ), LDWRKR, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate left bidiagonalizing vectors in WORK(IR)
*                   (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                    CALL DORGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
```

```
     $                          WORK( ITAUQ ), WORK( IWORK ),
     $                          LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IR)
*                    (Workspace: need N*N+BDSPAC)
*
                     CALL DBDSQR( 'U', N, 0, N, 0, S, WORK( IE ), DUM,
     $                            1, WORK( IR ), LDWRKR, DUM, 1,
     $                          WORK( IWORK ), INFO )
*
*                    Multiply Q in U by left singular vectors of R in
*                    WORK(IR), storing result in A
*                    (Workspace: need N*N)
*
                     CALL DGEMM( 'N', 'N', M, N, N, ONE, U, LDU,
     $                           WORK( IR ), LDWRKR, ZERO, A, LDA )
*
*                    Copy left singular vectors of A from A to U
*
                     CALL DLACPY( 'F', M, N, A, LDA, U, LDU )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (Workspace: need 2*N, prefer N+N*NB)
*
                     CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                    Generate Q in U
*                    (Workspace: need N+M, prefer N+M*NB)
*
                     CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Zero out below R in A
*
                     CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ),
```

```
     $                          LDA )
*
*                    Bidiagonalize R in A
*                    (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                     CALL DGEBRD( N, N, A, LDA, S, WORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply Q in U by left bidiagonalizing vectors
*                    in A
*                    (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                     CALL DORMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                           WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', N, 0, M, 0, S, WORK( IE ), DUM,
     $                           1, U, LDU, DUM, 1, WORK( IWORK ),
     $                           INFO )
*
                  END IF
*
               ELSE IF( WNTVO ) THEN
*
                  Path 8 (M much larger than N, JOBU='A', JOBVT='O')
*                 M left singular vectors to be computed in U and
*                 N right singular vectors to be overwritten on A
*
                  IF( LWORK.GE.2*N*N+MAX( N+M, 4*N, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+2*LDA*N ) THEN
*
*                       WORK(IU) is LDA by N and WORK(IR) is LDA by N
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*N
                        LDWRKR = LDA
                     ELSE IF( LWORK.GE.WRKBL+( LDA+N )*N ) THEN
*
*                       WORK(IU) is LDA by N and WORK(IR) is N by N
*
```

```
                      LDWRKU = LDA
                      IR = IU + LDWRKU*N
                      LDWRKR = N
                   ELSE
*
*                     WORK(IU) is N by N and WORK(IR) is N by N
*
                      LDWRKU = N
                      IR = IU + LDWRKU*N
                      LDWRKR = N
                   END IF
                   ITAU = IR + LDWRKR*N
                   IWORK = ITAU + N
*
*                  Compute A=Q*R, copying result to U
*                  (Workspace: need 2*N*N+2*N, prefer 2*N*N+N+N*NB)
*
                   CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                   CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                  Generate Q in U
*                  (Workspace: need 2*N*N+N+M, prefer 2*N*N+N+M*NB)
*
                   CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                  Copy R to WORK(IU), zeroing out below it
*
                   CALL DLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                           LDWRKU )
                   CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                           WORK( IU+1 ), LDWRKU )
                   IE = ITAU
                   ITAUQ = IE + N
                   ITAUP = ITAUQ + N
                   IWORK = ITAUP + N
*
*                  Bidiagonalize R in WORK(IU), copying result to
*                  WORK(IR)
*                  (Workspace: need 2*N*N+4*N,
*                              prefer 2*N*N+3*N+2*N*NB)
*
                   CALL DGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                   CALL DLACPY( 'U', N, N, WORK( IU ), LDWRKU,
     $                           WORK( IR ), LDWRKR )
*
```

```
*                    Generate left bidiagonalizing vectors in WORK(IU)
*                    (Workspace: need 2*N*N+4*N, prefer 2*N*N+3*N+N*NB)
*
                     CALL DORGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                            WORK( ITAUQ ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in WORK(IR)
*                    (Workspace: need 2*N*N+4*N-1,
*                                prefer 2*N*N+3*N+(N-1)*NB)
*
                     CALL DORGBR( 'P', N, N, N, WORK( IR ), LDWRKR,
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IU) and computing
*                    right singular vectors of R in WORK(IR)
*                    (Workspace: need 2*N*N+BDSPAC)
*
                     CALL DBDSQR( 'U', N, N, N, 0, S, WORK( IE ),
     $                            WORK( IR ), LDWRKR, WORK( IU ),
     $                            LDWRKU, DUM, 1, WORK( IWORK ), INFO )
*
*                    Multiply Q in U by left singular vectors of R in
*                    WORK(IU), storing result in A
*                    (Workspace: need N*N)
*
                     CALL DGEMM( 'N', 'N', M, N, N, ONE, U, LDU,
     $                           WORK( IU ), LDWRKU, ZERO, A, LDA )
*
*                    Copy left singular vectors of A from A to U
*
                     CALL DLACPY( 'F', M, N, A, LDA, U, LDU )
*
*                    Copy right singular vectors of R from WORK(IR) to A
*
                     CALL DLACPY( 'F', N, N, WORK( IR ), LDWRKR, A,
     $                            LDA )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (Workspace: need 2*N, prefer N+N*NB)
```

```
*
                    CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                   Generate Q in U
*                   (Workspace: need N+M, prefer N+M*NB)
*
                    CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = ITAU
                    ITAUQ = IE + N
                    ITAUP = ITAUQ + N
                    IWORK = ITAUP + N
*
*                   Zero out below R in A
*
                    CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ),
     $                           LDA )
*
*                   Bidiagonalize R in A
*                   (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                    CALL DGEBRD( N, N, A, LDA, S, WORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Multiply Q in U by left bidiagonalizing vectors
*                   in A
*                   (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                    CALL DORMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                           WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate right bidiagonalizing vectors in A
*                   (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                    CALL DORGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IWORK = IE + N
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of A in U and computing right
*                   singular vectors of A in A
*                   (Workspace: need BDSPAC)
*
                    CALL DBDSQR( 'U', N, N, M, 0, S, WORK( IE ), A,
     $                           LDA, U, LDU, DUM, 1, WORK( IWORK ),
     $                           INFO )
```

```
*
                 END IF
*
              ELSE IF( WNTVAS ) THEN
*
*                Path 9 (M much larger than N, JOBU='A', JOBVT='S'
*                       or 'A')
*                M left singular vectors to be computed in U and
*                N right singular vectors to be computed in VT
*
                 IF( LWORK.GE.N*N+MAX( N+M, 4*N, BDSPAC ) ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IU = 1
                    IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                      WORK(IU) is LDA by N
*
                       LDWRKU = LDA
                    ELSE
*
*                      WORK(IU) is N by N
*
                       LDWRKU = N
                    END IF
                    ITAU = IU + LDWRKU*N
                    IWORK = ITAU + N
*
*                   Compute A=Q*R, copying result to U
*                   (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                    CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                   Generate Q in U
*                   (Workspace: need N*N+N+M, prefer N*N+N+M*NB)
*
                    CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy R to WORK(IU), zeroing out below it
*
                    CALL DLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                           LDWRKU )
                    CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                           WORK( IU+1 ), LDWRKU )
                    IE = ITAU
                    ITAUQ = IE + N
```

```
                    ITAUP = ITAUQ + N
                    IWORK = ITAUP + N
*
*                   Bidiagonalize R in WORK(IU), copying result to VT
*                   (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                    CALL DGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    CALL DLACPY( 'U', N, N, WORK( IU ), LDWRKU, VT,
     $                           LDVT )
*
*                   Generate left bidiagonalizing vectors in WORK(IU)
*                   (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                    CALL DORGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate right bidiagonalizing vectors in VT
*                   (Workspace: need N*N+4*N-1,
*                               prefer N*N+3*N+(N-1)*NB)
*
                    CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IWORK = IE + N
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of R in WORK(IU) and computing
*                   right singular vectors of R in VT
*                   (Workspace: need N*N+BDSPAC)
*
                    CALL DBDSQR( 'U', N, N, N, 0, S, WORK( IE ), VT,
     $                           LDVT, WORK( IU ), LDWRKU, DUM, 1,
     $                           WORK( IWORK ), INFO )
*
*                   Multiply Q in U by left singular vectors of R in
*                   WORK(IU), storing result in A
*                   (Workspace: need N*N)
*
                    CALL DGEMM( 'N', 'N', M, N, N, ONE, U, LDU,
     $                          WORK( IU ), LDWRKU, ZERO, A, LDA )
*
*                   Copy left singular vectors of A from A to U
*
                    CALL DLACPY( 'F', M, N, A, LDA, U, LDU )
*
                 ELSE
*
```

```
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (Workspace: need 2*N, prefer N+N*NB)
*
                     CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                    Generate Q in U
*                    (Workspace: need N+M, prefer N+M*NB)
*
                     CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy R from A to VT, zeroing out below it
*
                     CALL DLACPY( 'U', N, N, A, LDA, VT, LDVT )
                     IF( N.GT.1 )
     $                  CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                               VT( 2, 1 ), LDVT )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Bidiagonalize R in VT
*                    (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                     CALL DGEBRD( N, N, VT, LDVT, S, WORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply Q in U by left bidiagonalizing vectors
*                    in VT
*                    (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                     CALL DORMBR( 'Q', 'R', 'N', M, N, N, VT, LDVT,
     $                            WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in VT
*                    (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                     CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + N
```

```
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', N, N, M, 0, S, WORK( IE ), VT,
     $                            LDVT, U, LDU, DUM, 1, WORK( IWORK ),
     $                            INFO )
*
                  END IF
*
               END IF
*
            END IF
*
         ELSE
*
            M .LT. MNTHR
*
            Path 10 (M at least N, but not much larger)
            Reduce to bidiagonal form without QR decomposition
*
            IE = 1
            ITAUQ = IE + N
            ITAUP = ITAUQ + N
            IWORK = ITAUP + N
*
            Bidiagonalize A
            (Workspace: need 3*N+M, prefer 3*N+(M+N)*NB)
*
            CALL DGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                   WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                   IERR )
            IF( WNTUAS ) THEN
*
               If left singular vectors desired in U, copy result to U
               and generate left bidiagonalizing vectors in U
               (Workspace: need 3*N+NCU, prefer 3*N+NCU*NB)
*
               CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
               IF( WNTUS )
     $            NCU = N
               IF( WNTUA )
     $            NCU = M
               CALL DORGBR( 'Q', M, NCU, N, U, LDU, WORK( ITAUQ ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IF( WNTVAS ) THEN
*
```

```
*              If right singular vectors desired in VT, copy result to
*              VT and generate right bidiagonalizing vectors in VT
*              (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
               CALL DLACPY( 'U', N, N, A, LDA, VT, LDVT )
               CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IF( WNTUO ) THEN
*
*              If left singular vectors desired in A, generate left
*              bidiagonalizing vectors in A
*              (Workspace: need 4*N, prefer 3*N+N*NB)
*
               CALL DORGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IF( WNTVO ) THEN
*
*              If right singular vectors desired in A, generate right
*              bidiagonalizing vectors in A
*              (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
               CALL DORGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IWORK = IE + N
            IF( WNTUAS .OR. WNTUO )
     $         NRU = M
            IF( WNTUN )
     $         NRU = 0
            IF( WNTVAS .OR. WNTVO )
     $         NCVT = N
            IF( WNTVN )
     $         NCVT = 0
            IF( ( .NOT.WNTUO ) .AND. ( .NOT.WNTVO ) ) THEN
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in U and computing right singular
*              vectors in VT
*              (Workspace: need BDSPAC)
*
               CALL DBDSQR( 'U', N, NCVT, NRU, 0, S, WORK( IE ), VT,
     $                      LDVT, U, LDU, DUM, 1, WORK( IWORK ), INFO )
            ELSE IF( ( .NOT.WNTUO ) .AND. WNTVO ) THEN
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in U and computing right singular
*              vectors in A
*              (Workspace: need BDSPAC)
```

```
*
               CALL DBDSQR( 'U', N, NCVT, NRU, 0, S, WORK( IE ), A, LDA,
     $                      U, LDU, DUM, 1, WORK( IWORK ), INFO )
            ELSE
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in A and computing right singular
*              vectors in VT
*              (Workspace: need BDSPAC)
*
               CALL DBDSQR( 'U', N, NCVT, NRU, 0, S, WORK( IE ), VT,
     $                      LDVT, A, LDA, DUM, 1, WORK( IWORK ), INFO )
            END IF
*
         END IF
*
      ELSE
*
*        A has more columns than rows. If A has sufficiently more
*        columns than rows, first reduce using the LQ decomposition (if
*        sufficient workspace available)
*
         IF( N.GE.MNTHR ) THEN
*
            IF( WNTVN ) THEN
*
*              Path 1t(N much larger than M, JOBVT='N')
*              No right singular vectors to be computed
*
               ITAU = 1
               IWORK = ITAU + M
*
*              Compute A=L*Q
*              (Workspace: need 2*M, prefer M+M*NB)
*
               CALL DGELQF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
     $                      LWORK-IWORK+1, IERR )
*
*              Zero out above L
*
               CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ), LDA )
               IE = 1
               ITAUQ = IE + M
               ITAUP = ITAUQ + M
               IWORK = ITAUP + M
*
*              Bidiagonalize L in A
*              (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
               CALL DGEBRD( M, M, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
```

```
     $                     WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                     IERR )
               IF( WNTUO .OR. WNTUAS ) THEN
*
*                 If left singular vectors desired, generate Q
*                 (Workspace: need 4*M, prefer 3*M+M*NB)
*
                  CALL DORGBR( 'Q', M, M, M, A, LDA, WORK( ITAUQ ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
               END IF
               IWORK = IE + M
               NRU = 0
               IF( WNTUO .OR. WNTUAS )
     $            NRU = M
*
*              Perform bidiagonal QR iteration, computing left singular
*              vectors of A in A if desired
*              (Workspace: need BDSPAC)
*
               CALL DBDSQR( 'U', M, 0, NRU, 0, S, WORK( IE ), DUM, 1, A,
     $                      LDA, DUM, 1, WORK( IWORK ), INFO )
*
*              If left singular vectors desired in U, copy them there
*
               IF( WNTUAS )
     $            CALL DLACPY( 'F', M, M, A, LDA, U, LDU )
*
            ELSE IF( WNTVO .AND. WNTUN ) THEN
*
*              Path 2t(N much larger than M, JOBU='N', JOBVT='O')
*              M right singular vectors to be overwritten on A and
*              no left singular vectors to be computed
*
               IF( LWORK.GE.M*M+MAX( 4*M, BDSPAC ) ) THEN
*
*                 Sufficient workspace for a fast algorithm
*
                  IR = 1
                  IF( LWORK.GE.MAX( WRKBL, LDA*N+M )+LDA*M ) THEN
*
*                    WORK(IU) is LDA by N and WORK(IR) is LDA by M
*
                     LDWRKU = LDA
                     CHUNK = N
                     LDWRKR = LDA
                  ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N+M )+M*M ) THEN
*
*                    WORK(IU) is LDA by N and WORK(IR) is M by M
*
                     LDWRKU = LDA
```

```
                 CHUNK = N
                 LDWRKR = M
              ELSE
*
*                WORK(IU) is M by CHUNK and WORK(IR) is M by M
*
                 LDWRKU = M
                 CHUNK = ( LWORK-M*M-M ) / M
                 LDWRKR = M
              END IF
              ITAU = IR + LDWRKR*M
              IWORK = ITAU + M
*
*             Compute A=L*Q
*             (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
              CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*             Copy L to WORK(IR) and zero out above it
*
              CALL DLACPY( 'L', M, M, A, LDA, WORK( IR ), LDWRKR )
              CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                     WORK( IR+LDWRKR ), LDWRKR )
*
*             Generate Q in A
*             (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
              CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
              IE = ITAU
              ITAUQ = IE + M
              ITAUP = ITAUQ + M
              IWORK = ITAUP + M
*
*             Bidiagonalize L in WORK(IR)
*             (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
              CALL DGEBRD( M, M, WORK( IR ), LDWRKR, S, WORK( IE ),
     $                     WORK( ITAUQ ), WORK( ITAUP ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*             Generate right vectors bidiagonalizing L
*             (Workspace: need M*M+4*M-1, prefer M*M+3*M+(M-1)*NB)
*
              CALL DORGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                     WORK( ITAUP ), WORK( IWORK ),
     $                     LWORK-IWORK+1, IERR )
              IWORK = IE + M
*
```

```
*                 Perform bidiagonal QR iteration, computing right
*                 singular vectors of L in WORK(IR)
*                 (Workspace: need M*M+BDSPAC)
*
                  CALL DBDSQR( 'U', M, M, 0, 0, S, WORK( IE ),
     $                         WORK( IR ), LDWRKR, DUM, 1, DUM, 1,
     $                         WORK( IWORK ), INFO )
                  IU = IE + M
*
*                 Multiply right singular vectors of L in WORK(IR) by Q
*                 in A, storing result in WORK(IU) and copying to A
*                 (Workspace: need M*M+2*M, prefer M*M+M*N+M)
*
                  DO 30 I = 1, N, CHUNK
                     BLK = MIN( N-I+1, CHUNK )
                     CALL DGEMM( 'N', 'N', M, BLK, M, ONE, WORK( IR ),
     $                           LDWRKR, A( 1, I ), LDA, ZERO,
     $                           WORK( IU ), LDWRKU )
                     CALL DLACPY( 'F', M, BLK, WORK( IU ), LDWRKU,
     $                            A( 1, I ), LDA )
   30             CONTINUE
*
               ELSE
*
*                 Insufficient workspace for a fast algorithm
*
                  IE = 1
                  ITAUQ = IE + M
                  ITAUP = ITAUQ + M
                  IWORK = ITAUP + M
*
*                 Bidiagonalize A
*                 (Workspace: need 3*M+N, prefer 3*M+(M+N)*NB)
*
                  CALL DGEBRD( M, N, A, LDA, S, WORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Generate right vectors bidiagonalizing A
*                 (Workspace: need 4*M, prefer 3*M+M*NB)
*
                  CALL DORGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IWORK = IE + M
*
*                 Perform bidiagonal QR iteration, computing right
*                 singular vectors of A in A
*                 (Workspace: need BDSPAC)
*
                  CALL DBDSQR( 'L', M, N, 0, 0, S, WORK( IE ), A, LDA,
```

```
     $                        DUM, 1, DUM, 1, WORK( IWORK ), INFO )
*
               END IF
*
            ELSE IF( WNTVO .AND. WNTUAS ) THEN
*
*              Path 3t(N much larger than M, JOBU='S' or 'A', JOBVT='O')
*              M right singular vectors to be overwritten on A and
*              M left singular vectors to be computed in U
*
               IF( LWORK.GE.M*M+MAX( 4*M, BDSPAC ) ) THEN
*
*                 Sufficient workspace for a fast algorithm
*
                  IR = 1
                  IF( LWORK.GE.MAX( WRKBL, LDA*N+M )+LDA*M ) THEN
*
*                    WORK(IU) is LDA by N and WORK(IR) is LDA by M
*
                     LDWRKU = LDA
                     CHUNK = N
                     LDWRKR = LDA
                  ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N+M )+M*M ) THEN
*
*                    WORK(IU) is LDA by N and WORK(IR) is M by M
*
                     LDWRKU = LDA
                     CHUNK = N
                     LDWRKR = M
                  ELSE
*
*                    WORK(IU) is M by CHUNK and WORK(IR) is M by M
*
                     LDWRKU = M
                     CHUNK = ( LWORK-M*M-M ) / M
                     LDWRKR = M
                  END IF
                  ITAU = IR + LDWRKR*M
                  IWORK = ITAU + M
*
*                 Compute A=L*Q
*                 (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                  CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Copy L to U, zeroing about above it
*
                  CALL DLACPY( 'L', M, M, A, LDA, U, LDU )
                  CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, U( 1, 2 ),
```

```
     $                        LDU )
*
*                 Generate Q in A
*                 (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                  CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IE = ITAU
                  ITAUQ = IE + M
                  ITAUP = ITAUQ + M
                  IWORK = ITAUP + M
*
*                 Bidiagonalize L in U, copying result to WORK(IR)
*                 (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                  CALL DGEBRD( M, M, U, LDU, S, WORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  CALL DLACPY( 'U', M, M, U, LDU, WORK( IR ), LDWRKR )
*
*                 Generate right vectors bidiagonalizing L in WORK(IR)
*                 (Workspace: need M*M+4*M-1, prefer M*M+3*M+(M-1)*NB)
*
                  CALL DORGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                         WORK( ITAUP ), WORK( IWORK ),
     $                         LWORK-IWORK+1, IERR )
*
*                 Generate left vectors bidiagonalizing L in U
*                 (Workspace: need M*M+4*M, prefer M*M+3*M+M*NB)
*
                  CALL DORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IWORK = IE + M
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of L in U, and computing right
*                 singular vectors of L in WORK(IR)
*                 (Workspace: need M*M+BDSPAC)
*
                  CALL DBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                         WORK( IR ), LDWRKR, U, LDU, DUM, 1,
     $                         WORK( IWORK ), INFO )
                  IU = IE + M
*
*                 Multiply right singular vectors of L in WORK(IR) by Q
*                 in A, storing result in WORK(IU) and copying to A
*                 (Workspace: need M*M+2*M, prefer M*M+M*N+M))
*
                  DO 40 I = 1, N, CHUNK
                     BLK = MIN( N-I+1, CHUNK )
```

```
                 CALL DGEMM( 'N', 'N', M, BLK, M, ONE, WORK( IR ),
     $                       LDWRKR, A( 1, I ), LDA, ZERO,
     $                       WORK( IU ), LDWRKU )
                 CALL DLACPY( 'F', M, BLK, WORK( IU ), LDWRKU,
     $                        A( 1, I ), LDA )
   40         CONTINUE
*
            ELSE
*
*              Insufficient workspace for a fast algorithm
*
               ITAU = 1
               IWORK = ITAU + M
*
*              Compute A=L*Q
*              (Workspace: need 2*M, prefer M+M*NB)
*
               CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*              Copy L to U, zeroing out above it
*
               CALL DLACPY( 'L', M, M, A, LDA, U, LDU )
               CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, U( 1, 2 ),
     $                      LDU )
*
*              Generate Q in A
*              (Workspace: need 2*M, prefer M+M*NB)
*
               CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
               IE = ITAU
               ITAUQ = IE + M
               ITAUP = ITAUQ + M
               IWORK = ITAUP + M
*
*              Bidiagonalize L in U
*              (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
               CALL DGEBRD( M, M, U, LDU, S, WORK( IE ),
     $                      WORK( ITAUQ ), WORK( ITAUP ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*              Multiply right vectors bidiagonalizing L by Q in A
*              (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
               CALL DORMBR( 'P', 'L', 'T', M, N, M, U, LDU,
     $                      WORK( ITAUP ), A, LDA, WORK( IWORK ),
     $                      LWORK-IWORK+1, IERR )
*
```

```
*                 Generate left vectors bidiagonalizing L in U
*                 (Workspace: need 4*M, prefer 3*M+M*NB)
*
                  CALL DORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IWORK = IE + M
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of A in U and computing right
*                 singular vectors of A in A
*                 (Workspace: need BDSPAC)
*
                  CALL DBDSQR( 'U', M, N, M, 0, S, WORK( IE ), A, LDA,
     $                         U, LDU, DUM, 1, WORK( IWORK ), INFO )
*
               END IF
*
            ELSE IF( WNTVS ) THEN
*
               IF( WNTUN ) THEN
*
*                 Path 4t(N much larger than M, JOBU='N', JOBVT='S')
*                 M right singular vectors to be computed in VT and
*                 no left singular vectors to be computed
*
                  IF( LWORK.GE.M*M+MAX( 4*M, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IR = 1
                     IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                       WORK(IR) is LDA by M
*
                        LDWRKR = LDA
                     ELSE
*
*                       WORK(IR) is M by M
*
                        LDWRKR = M
                     END IF
                     ITAU = IR + LDWRKR*M
                     IWORK = ITAU + M
*
*                    Compute A=L*Q
*                    (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                     CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
```

```
*                    Copy L to WORK(IR), zeroing out above it
*
                     CALL DLACPY( 'L', M, M, A, LDA, WORK( IR ),
     $                            LDWRKR )
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                            WORK( IR+LDWRKR ), LDWRKR )
*
*                    Generate Q in A
*                    (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                     CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Bidiagonalize L in WORK(IR)
*                    (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                     CALL DGEBRD( M, M, WORK( IR ), LDWRKR, S,
     $                            WORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right vectors bidiagonalizing L in
*                    WORK(IR)
*                    (Workspace: need M*M+4*M, prefer M*M+3*M+(M-1)*NB)
*
                     CALL DORGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing right
*                    singular vectors of L in WORK(IR)
*                    (Workspace: need M*M+BDSPAC)
*
                     CALL DBDSQR( 'U', M, M, 0, 0, S, WORK( IE ),
     $                            WORK( IR ), LDWRKR, DUM, 1, DUM, 1,
     $                            WORK( IWORK ), INFO )
*
*                    Multiply right singular vectors of L in WORK(IR) by
*                    Q in A, storing result in VT
*                    (Workspace: need M*M)
*
                     CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IR ),
     $                           LDWRKR, A, LDA, ZERO, VT, LDVT )
*
                  ELSE
```

```
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + M
*
*                    Compute A=L*Q
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy result to VT
*
                     CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL DORGLQ( M, N, M, VT, LDVT, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Zero out above L in A
*
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ),
     $                            LDA )
*
*                    Bidiagonalize L in A
*                    (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                     CALL DGEBRD( M, M, A, LDA, S, WORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply right vectors bidiagonalizing L by Q in VT
*                    (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                     CALL DORMBR( 'P', 'L', 'T', M, N, M, A, LDA,
     $                            WORK( ITAUP ), VT, LDVT,
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing right
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
```

```
                  CALL DBDSQR( 'U', M, N, 0, 0, S, WORK( IE ), VT,
     $                         LDVT, DUM, 1, DUM, 1, WORK( IWORK ),
     $                         INFO )
*
                END IF
*
             ELSE IF( WNTUO ) THEN
*
                Path 5t(N much larger than M, JOBU='O', JOBVT='S')
*               M right singular vectors to be computed in VT and
*               M left singular vectors to be overwritten on A
*
                IF( LWORK.GE.2*M*M+MAX( 4*M, BDSPAC ) ) THEN
*
                   Sufficient workspace for a fast algorithm
*
                   IU = 1
                   IF( LWORK.GE.WRKBL+2*LDA*M ) THEN
*
                      WORK(IU) is LDA by M and WORK(IR) is LDA by M
*
                      LDWRKU = LDA
                      IR = IU + LDWRKU*M
                      LDWRKR = LDA
                   ELSE IF( LWORK.GE.WRKBL+( LDA+M )*M ) THEN
*
                      WORK(IU) is LDA by M and WORK(IR) is M by M
*
                      LDWRKU = LDA
                      IR = IU + LDWRKU*M
                      LDWRKR = M
                   ELSE
*
                      WORK(IU) is M by M and WORK(IR) is M by M
*
                      LDWRKU = M
                      IR = IU + LDWRKU*M
                      LDWRKR = M
                   END IF
                   ITAU = IR + LDWRKR*M
                   IWORK = ITAU + M
*
                   Compute A=L*Q
*                  (Workspace: need 2*M*M+2*M, prefer 2*M*M+M+M*NB)
*
                   CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                  Copy L to WORK(IU), zeroing out below it
*
```

```
                    CALL DLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                           LDWRKU )
                    CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                           WORK( IU+LDWRKU ), LDWRKU )
*
*                   Generate Q in A
*                   (Workspace: need 2*M*M+2*M, prefer 2*M*M+M+M*NB)
*
                    CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = ITAU
                    ITAUQ = IE + M
                    ITAUP = ITAUQ + M
                    IWORK = ITAUP + M
*
*                   Bidiagonalize L in WORK(IU), copying result to
*                   WORK(IR)
*                   (Workspace: need 2*M*M+4*M,
*                               prefer 2*M*M+3*M+2*M*NB)
*
                    CALL DGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    CALL DLACPY( 'L', M, M, WORK( IU ), LDWRKU,
     $                           WORK( IR ), LDWRKR )
*
*                   Generate right bidiagonalizing vectors in WORK(IU)
*                   (Workspace: need 2*M*M+4*M-1,
*                               prefer 2*M*M+3*M+(M-1)*NB)
*
                    CALL DORGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate left bidiagonalizing vectors in WORK(IR)
*                   (Workspace: need 2*M*M+4*M, prefer 2*M*M+3*M+M*NB)
*
                    CALL DORGBR( 'Q', M, M, M, WORK( IR ), LDWRKR,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    IWORK = IE + M
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of L in WORK(IR) and computing
*                   right singular vectors of L in WORK(IU)
*                   (Workspace: need 2*M*M+BDSPAC)
*
                    CALL DBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                           WORK( IU ), LDWRKU, WORK( IR ),
```

```
     $                          LDWRKR, DUM, 1, WORK( IWORK ), INFO )
*
*                    Multiply right singular vectors of L in WORK(IU) by
*                    Q in A, storing result in VT
*                    (Workspace: need M*M)
*
                     CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IU ),
     $                           LDWRKU, A, LDA, ZERO, VT, LDVT )
*
*                    Copy left singular vectors of L to A
*                    (Workspace: need M*M)
*
                     CALL DLACPY( 'F', M, M, WORK( IR ), LDWRKR, A,
     $                           LDA )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL DORGLQ( M, N, M, VT, LDVT, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Zero out above L in A
*
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ),
     $                           LDA )
*
*                    Bidiagonalize L in A
*                    (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                     CALL DGEBRD( M, M, A, LDA, S, WORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
```

```
*
*                    Multiply right vectors bidiagonalizing L by Q in VT
*                    (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                     CALL DORMBR( 'P', 'L', 'T', M, N, M, A, LDA,
     $                            WORK( ITAUP ), VT, LDVT,
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors of L in A
*                    (Workspace: need 4*M, prefer 3*M+M*NB)
*
                     CALL DORGBR( 'Q', M, M, M, A, LDA, WORK( ITAUQ ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, compute left
*                    singular vectors of A in A and compute right
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', M, N, M, 0, S, WORK( IE ), VT,
     $                            LDVT, A, LDA, DUM, 1, WORK( IWORK ),
     $                            INFO )
*
                  END IF
*
               ELSE IF( WNTUAS ) THEN
*
                  Path 6t(N much larger than M, JOBU='S' or 'A',
*                         JOBVT='S')
*                 M right singular vectors to be computed in VT and
*                 M left singular vectors to be computed in U
*
                  IF( LWORK.GE.M*M+MAX( 4*M, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                       WORK(IU) is LDA by N
*
                        LDWRKU = LDA
                     ELSE
*
*                       WORK(IU) is LDA by M
*
                        LDWRKU = M
                     END IF
                     ITAU = IU + LDWRKU*M
```

```
                    IWORK = ITAU + M
*
*                   Compute A=L*Q
*                   (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                    CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy L to WORK(IU), zeroing out above it
*
                    CALL DLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                           LDWRKU )
                    CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                           WORK( IU+LDWRKU ), LDWRKU )
*
*                   Generate Q in A
*                   (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                    CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = ITAU
                    ITAUQ = IE + M
                    ITAUP = ITAUQ + M
                    IWORK = ITAUP + M
*
*                   Bidiagonalize L in WORK(IU), copying result to U
*                   (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                    CALL DGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    CALL DLACPY( 'L', M, M, WORK( IU ), LDWRKU, U,
     $                           LDU )
*
*                   Generate right bidiagonalizing vectors in WORK(IU)
*                   (Workspace: need M*M+4*M-1,
*                               prefer M*M+3*M+(M-1)*NB)
*
                    CALL DORGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate left bidiagonalizing vectors in U
*                   (Workspace: need M*M+4*M, prefer M*M+3*M+M*NB)
*
                    CALL DORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IWORK = IE + M
*
```

```
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of L in U and computing right
*                    singular vectors of L in WORK(IU)
*                    (Workspace: need M*M+BDSPAC)
*
                     CALL DBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                            WORK( IU ), LDWRKU, U, LDU, DUM, 1,
     $                            WORK( IWORK ), INFO )
*
*                    Multiply right singular vectors of L in WORK(IU) by
*                    Q in A, storing result in VT
*                    (Workspace: need M*M)
*
                     CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IU ),
     $                           LDWRKU, A, LDA, ZERO, VT, LDVT )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL DORGLQ( M, N, M, VT, LDVT, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to U, zeroing out above it
*
                     CALL DLACPY( 'L', M, M, A, LDA, U, LDU )
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, U( 1, 2 ),
     $                            LDU )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Bidiagonalize L in U
*                    (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                     CALL DGEBRD( M, M, U, LDU, S, WORK( IE ),
```

```
     $                          WORK( ITAUQ ), WORK( ITAUP ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply right bidiagonalizing vectors in U by Q
*                    in VT
*                    (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                     CALL DORMBR( 'P', 'L', 'T', M, N, M, U, LDU,
     $                          WORK( ITAUP ), VT, LDVT,
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors in U
*                    (Workspace: need 4*M, prefer 3*M+M*NB)
*
                     CALL DORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', M, N, M, 0, S, WORK( IE ), VT,
     $                          LDVT, U, LDU, DUM, 1, WORK( IWORK ),
     $                          INFO )
*
                  END IF
*
               END IF
*
            ELSE IF( WNTVA ) THEN
*
               IF( WNTUN ) THEN
*
                  Path 7t(N much larger than M, JOBU='N', JOBVT='A')
                  N right singular vectors to be computed in VT and
                  no left singular vectors to be computed
*
                  IF( LWORK.GE.M*M+MAX( N+M, 4*M, BDSPAC ) ) THEN
*
                     Sufficient workspace for a fast algorithm
*
                     IR = 1
                     IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
                        WORK(IR) is LDA by M
*
                        LDWRKR = LDA
                     ELSE
```

```
*
*                       WORK(IR) is M by M
*
                        LDWRKR = M
                     END IF
                     ITAU = IR + LDWRKR*M
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                     CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Copy L to WORK(IR), zeroing out above it
*
                     CALL DLACPY( 'L', M, M, A, LDA, WORK( IR ),
     $                            LDWRKR )
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                            WORK( IR+LDWRKR ), LDWRKR )
*
*                    Generate Q in VT
*                    (Workspace: need M*M+M+N, prefer M*M+M+N*NB)
*
                     CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Bidiagonalize L in WORK(IR)
*                    (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                     CALL DGEBRD( M, M, WORK( IR ), LDWRKR, S,
     $                            WORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in WORK(IR)
*                    (Workspace: need M*M+4*M-1,
*                                prefer M*M+3*M+(M-1)*NB)
*
                     CALL DORGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing right
```

```
*                    singular vectors of L in WORK(IR)
*                    (Workspace: need M*M+BDSPAC)
*
                     CALL DBDSQR( 'U', M, M, 0, 0, S, WORK( IE ),
     $                            WORK( IR ), LDWRKR, DUM, 1, DUM, 1,
     $                            WORK( IWORK ), INFO )
*
*                    Multiply right singular vectors of L in WORK(IR) by
*                    Q in VT, storing result in A
*                    (Workspace: need M*M)
*
                     CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IR ),
     $                           LDWRKR, VT, LDVT, ZERO, A, LDA )
*
*                    Copy right singular vectors of A from A to VT
*
                     CALL DLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (Workspace: need M+N, prefer M+N*NB)
*
                     CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Zero out above L in A
*
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ),
     $                            LDA )
*
*                    Bidiagonalize L in A
*                    (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
```

```
                 CALL DGEBRD( M, M, A, LDA, S, WORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Multiply right bidiagonalizing vectors in A by Q
*                in VT
*                (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                 CALL DORMBR( 'P', 'L', 'T', M, N, M, A, LDA,
     $                        WORK( ITAUP ), VT, LDVT,
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing right
*                singular vectors of A in VT
*                (Workspace: need BDSPAC)
*
                 CALL DBDSQR( 'U', M, N, 0, 0, S, WORK( IE ), VT,
     $                        LDVT, DUM, 1, DUM, 1, WORK( IWORK ),
     $                        INFO )
*
              END IF
*
           ELSE IF( WNTUO ) THEN
*
              Path 8t(N much larger than M, JOBU='O', JOBVT='A')
*             N right singular vectors to be computed in VT and
*             M left singular vectors to be overwritten on A
*
              IF( LWORK.GE.2*M*M+MAX( N+M, 4*M, BDSPAC ) ) THEN
*
*                Sufficient workspace for a fast algorithm
*
                 IU = 1
                 IF( LWORK.GE.WRKBL+2*LDA*M ) THEN
*
*                   WORK(IU) is LDA by M and WORK(IR) is LDA by M
*
                    LDWRKU = LDA
                    IR = IU + LDWRKU*M
                    LDWRKR = LDA
                 ELSE IF( LWORK.GE.WRKBL+( LDA+M )*M ) THEN
*
*                   WORK(IU) is LDA by M and WORK(IR) is M by M
*
                    LDWRKU = LDA
                    IR = IU + LDWRKU*M
                    LDWRKR = M
                 ELSE
*
```

```
*                       WORK(IU) is M by M and WORK(IR) is M by M
*
                        LDWRKU = M
                        IR = IU + LDWRKU*M
                        LDWRKR = M
                     END IF
                     ITAU = IR + LDWRKR*M
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (Workspace: need 2*M*M+2*M, prefer 2*M*M+M+M*NB)
*
                     CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (Workspace: need 2*M*M+M+N, prefer 2*M*M+M+N*NB)
*
                     CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to WORK(IU), zeroing out above it
*
                     CALL DLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                            LDWRKU )
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                            WORK( IU+LDWRKU ), LDWRKU )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Bidiagonalize L in WORK(IU), copying result to
*                    WORK(IR)
*                    (Workspace: need 2*M*M+4*M,
*                                prefer 2*M*M+3*M+2*M*NB)
*
                     CALL DGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                            WORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'L', M, M, WORK( IU ), LDWRKU,
     $                            WORK( IR ), LDWRKR )
*
*                    Generate right bidiagonalizing vectors in WORK(IU)
*                    (Workspace: need 2*M*M+4*M-1,
*                                prefer 2*M*M+3*M+(M-1)*NB)
*
                     CALL DORGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
```

```
     $                          WORK( ITAUP ), WORK( IWORK ),
     $                          LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors in WORK(IR)
*                    (Workspace: need 2*M*M+4*M, prefer 2*M*M+3*M+M*NB)
*
                     CALL DORGBR( 'Q', M, M, M, WORK( IR ), LDWRKR,
     $                          WORK( ITAUQ ), WORK( IWORK ),
     $                          LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of L in WORK(IR) and computing
*                    right singular vectors of L in WORK(IU)
*                    (Workspace: need 2*M*M+BDSPAC)
*
                     CALL DBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                          WORK( IU ), LDWRKU, WORK( IR ),
     $                          LDWRKR, DUM, 1, WORK( IWORK ), INFO )
*
*                    Multiply right singular vectors of L in WORK(IU) by
*                    Q in VT, storing result in A
*                    (Workspace: need M*M)
*
                     CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IU ),
     $                         LDWRKU, VT, LDVT, ZERO, A, LDA )
*
*                    Copy right singular vectors of A from A to VT
*
                     CALL DLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
*                    Copy left singular vectors of A from WORK(IR) to A
*
                     CALL DLACPY( 'F', M, M, WORK( IR ), LDWRKR, A,
     $                          LDA )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
```

```
*                    Generate Q in VT
*                    (Workspace: need M+N, prefer M+N*NB)
*
                     CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Zero out above L in A
*
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ),
     $                            LDA )
*
*                    Bidiagonalize L in A
*                    (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                     CALL DGEBRD( M, M, A, LDA, S, WORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply right bidiagonalizing vectors in A by Q
*                    in VT
*                    (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                     CALL DORMBR( 'P', 'L', 'T', M, N, M, A, LDA,
     $                            WORK( ITAUP ), VT, LDVT,
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors in A
*                    (Workspace: need 4*M, prefer 3*M+M*NB)
*
                     CALL DORGBR( 'Q', M, M, M, A, LDA, WORK( ITAUQ ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in A and computing right
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', M, N, M, 0, S, WORK( IE ), VT,
     $                            LDVT, A, LDA, DUM, 1, WORK( IWORK ),
     $                            INFO )
*
                  END IF
*
               ELSE IF( WNTUAS ) THEN
*
```

```
*                 Path 9t(N much larger than M, JOBU='S' or 'A',
*                         JOBVT='A')
*                 N right singular vectors to be computed in VT and
*                 M left singular vectors to be computed in U
*
                  IF( LWORK.GE.M*M+MAX( N+M, 4*M, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                       WORK(IU) is LDA by M
*
                        LDWRKU = LDA
                     ELSE
*
*                       WORK(IU) is M by M
*
                        LDWRKU = M
                     END IF
                     ITAU = IU + LDWRKU*M
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                     CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (Workspace: need M*M+M+N, prefer M*M+M+N*NB)
*
                     CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to WORK(IU), zeroing out above it
*
                     CALL DLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                            LDWRKU )
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                            WORK( IU+LDWRKU ), LDWRKU )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Bidiagonalize L in WORK(IU), copying result to U
*                    (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
```

```
*
                 CALL DGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                        WORK( IE ), WORK( ITAUQ ),
     $                        WORK( ITAUP ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
                 CALL DLACPY( 'L', M, M, WORK( IU ), LDWRKU, U,
     $                        LDU )
*
*                Generate right bidiagonalizing vectors in WORK(IU)
*                (Workspace: need M*M+4*M, prefer M*M+3*M+(M-1)*NB)
*
                 CALL DORGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                        WORK( ITAUP ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate left bidiagonalizing vectors in U
*                (Workspace: need M*M+4*M, prefer M*M+3*M+M*NB)
*
                 CALL DORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of L in U and computing right
*                singular vectors of L in WORK(IU)
*                (Workspace: need M*M+BDSPAC)
*
                 CALL DBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                        WORK( IU ), LDWRKU, U, LDU, DUM, 1,
     $                        WORK( IWORK ), INFO )
*
*                Multiply right singular vectors of L in WORK(IU) by
*                Q in VT, storing result in A
*                (Workspace: need M*M)
*
                 CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IU ),
     $                       LDWRKU, VT, LDVT, ZERO, A, LDA )
*
*                Copy right singular vectors of A from A to VT
*
                 CALL DLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + M
*
*                Compute A=L*Q, copying result to VT
```

```
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (Workspace: need M+N, prefer M+N*NB)
*
                     CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to U, zeroing out above it
*
                     CALL DLACPY( 'L', M, M, A, LDA, U, LDU )
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, U( 1, 2 ),
     $                            LDU )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Bidiagonalize L in U
*                    (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                     CALL DGEBRD( M, M, U, LDU, S, WORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply right bidiagonalizing vectors in U by Q
*                    in VT
*                    (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                     CALL DORMBR( 'P', 'L', 'T', M, N, M, U, LDU,
     $                            WORK( ITAUP ), VT, LDVT,
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors in U
*                    (Workspace: need 4*M, prefer 3*M+M*NB)
*
                     CALL DORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', M, N, M, 0, S, WORK( IE ), VT,
```

```
     $                           LDVT, U, LDU, DUM, 1, WORK( IWORK ),
     $                           INFO )
*
                 END IF
*
              END IF
*
           END IF
*
        ELSE
*
*          N .LT. MNTHR
*
*          Path 10t(N greater than M, but not much larger)
*          Reduce to bidiagonal form without LQ decomposition
*
           IE = 1
           ITAUQ = IE + M
           ITAUP = ITAUQ + M
           IWORK = ITAUP + M
*
*          Bidiagonalize A
*          (Workspace: need 3*M+N, prefer 3*M+(M+N)*NB)
*
           CALL DGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                  WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                  IERR )
           IF( WNTUAS ) THEN
*
*             If left singular vectors desired in U, copy result to U
*             and generate left bidiagonalizing vectors in U
*             (Workspace: need 4*M-1, prefer 3*M+(M-1)*NB)
*
              CALL DLACPY( 'L', M, M, A, LDA, U, LDU )
              CALL DORGBR( 'Q', M, M, N, U, LDU, WORK( ITAUQ ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
           IF( WNTVAS ) THEN
*
*             If right singular vectors desired in VT, copy result to
*             VT and generate right bidiagonalizing vectors in VT
*             (Workspace: need 3*M+NRVT, prefer 3*M+NRVT*NB)
*
              CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
              IF( WNTVA )
     $           NRVT = N
              IF( WNTVS )
     $           NRVT = M
              CALL DORGBR( 'P', NRVT, N, M, VT, LDVT, WORK( ITAUP ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
```

```
            END IF
            IF( WNTUO ) THEN
*
*              If left singular vectors desired in A, generate left
*              bidiagonalizing vectors in A
*              (Workspace: need 4*M-1, prefer 3*M+(M-1)*NB)
*
               CALL DORGBR( 'Q', M, M, N, A, LDA, WORK( ITAUQ ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IF( WNTVO ) THEN
*
*              If right singular vectors desired in A, generate right
*              bidiagonalizing vectors in A
*              (Workspace: need 4*M, prefer 3*M+M*NB)
*
               CALL DORGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IWORK = IE + M
            IF( WNTUAS .OR. WNTUO )
     $         NRU = M
            IF( WNTUN )
     $         NRU = 0
            IF( WNTVAS .OR. WNTVO )
     $         NCVT = N
            IF( WNTVN )
     $         NCVT = 0
            IF( ( .NOT.WNTUO ) .AND. ( .NOT.WNTVO ) ) THEN
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in U and computing right singular
*              vectors in VT
*              (Workspace: need BDSPAC)
*
               CALL DBDSQR( 'L', M, NCVT, NRU, 0, S, WORK( IE ), VT,
     $                      LDVT, U, LDU, DUM, 1, WORK( IWORK ), INFO )
            ELSE IF( ( .NOT.WNTUO ) .AND. WNTVO ) THEN
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in U and computing right singular
*              vectors in A
*              (Workspace: need BDSPAC)
*
               CALL DBDSQR( 'L', M, NCVT, NRU, 0, S, WORK( IE ), A, LDA,
     $                      U, LDU, DUM, 1, WORK( IWORK ), INFO )
            ELSE
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in A and computing right singular
```

```
*              vectors in VT
*              (Workspace: need BDSPAC)
*
               CALL DBDSQR( 'L', M, NCVT, NRU, 0, S, WORK( IE ), VT,
     $                      LDVT, A, LDA, DUM, 1, WORK( IWORK ), INFO )
            END IF
*
         END IF
*
      END IF
*
*     If DBDSQR failed to converge, copy unconverged superdiagonals
*     to WORK( 2:MINMN )
*
      IF( INFO.NE.0 ) THEN
         IF( IE.GT.2 ) THEN
            DO 50 I = 1, MINMN - 1
               WORK( I+1 ) = WORK( I+IE-1 )
   50       CONTINUE
         END IF
         IF( IE.LT.2 ) THEN
            DO 60 I = MINMN - 1, 1, -1
               WORK( I+1 ) = WORK( I+IE-1 )
   60       CONTINUE
         END IF
      END IF
*
*     Undo scaling if necessary
*
      IF( ISCL.EQ.1 ) THEN
         IF( ANRM.GT.BIGNUM )
     $      CALL DLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( INFO.NE.0 .AND. ANRM.GT.BIGNUM )
     $      CALL DLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN-1, 1, WORK( 2 ),
     $                   MINMN, IERR )
         IF( ANRM.LT.SMLNUM )
     $      CALL DLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( INFO.NE.0 .AND. ANRM.LT.SMLNUM )
     $      CALL DLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN-1, 1, WORK( 2 ),
     $                   MINMN, IERR )
      END IF
*
*     Return optimal workspace in WORK(1)
*
      WORK( 1 ) = MAXWRK
*
      RETURN
*
```

```
*     End of DGESVD
*
      END
```

———-

28.3.4 lisp code

— **dgesvd.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dgesvdx
         (jobu jobvt range m n a lda vl vu il iu ns s u ldu vt ldvt work lwork
          iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (double-float) vu vl)
             (type (array double-float (*)) work vt u s a)
             (type (f2cl-lib:integer4) info lwork ldvt ldu ns iu il lda n m)
             (type (string *) range jobvt jobu))
    (f2cl-lib:with-multi-array-data
        ((jobu character jobu-%data% jobu-%offset%)
         (jobvt character jobvt-%data% jobvt-%offset%)
         (range character range-%data% range-%offset%)
         (a double-float a-%data% a-%offset%)
         (s double-float s-%data% s-%offset%)
         (u double-float u-%data% u-%offset%)
         (vt double-float vt-%data% vt-%offset%)
         (work double-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((dum (make-array 1 :element-type 'double-float)) (abstol 0.0d0)
             (anrm 0.0d0) (bignum 0.0d0) (eps 0.0d0) (smlnum 0.0d0) (i 0)
             (id 0) (ie 0) (ierr 0) (ilqf 0) (iltgk 0) (iqrf 0) (iscl 0)
             (itau 0) (itaup 0) (itauq 0) (itemp 0) (itgkz 0) (iutgk 0) (j 0)
             (maxwrk 0) (minmn 0) (minwrk 0) (mnthr 0) (alls nil) (inds nil)
             (lquery nil) (vals nil) (wantu nil) (wantvt nil)
             (jobz
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (rngtgk
              (make-array '(1) :element-type 'character :initial-element #\ )))
        (declare (type (array double-float (1)) dum)
                 (type (double-float) abstol anrm bignum eps smlnum)
                 (type (f2cl-lib:integer4) i id ie ierr ilqf iltgk iqrf iscl
                  itau itaup itauq itemp itgkz iutgk j maxwrk minmn minwrk
                  mnthr)
                 (type f2cl-lib:logical alls inds lquery vals wantu wantvt)
```

```
         (type (string 1) jobz rngtgk))
(setf ns 0)
(setf info 0)
(setf abstol (* 2 (dlamch "S")))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(setf wantu
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobu "V")
          (declare (ignore var-1))
          (when var-0 (setf jobu var-0))
          ret-val))
(setf wantvt
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvt "V")
          (declare (ignore var-1))
          (when var-0 (setf jobvt var-0))
          ret-val))
(cond
 ((or wantu wantvt) (f2cl-lib:f2cl-set-string jobz "V" (string 1)))
 (t (f2cl-lib:f2cl-set-string jobz "N" (string 1))))
(setf alls
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "A")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf vals
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "V")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf inds
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "I")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf info 0)
(cond
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame jobu "V")
      (declare (ignore var-1))
      (when var-0 (setf jobu var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
```

```
          (lsame jobu "N")
        (declare (ignore var-1))
        (when var-0 (setf jobu var-0))
        ret-val)))
   (setf info -1))
  ((and
    (not
     (multiple-value-bind (ret-val var-0 var-1)
         (lsame jobvt "V")
       (declare (ignore var-1))
       (when var-0 (setf jobvt var-0))
       ret-val))
    (not
     (multiple-value-bind (ret-val var-0 var-1)
         (lsame jobvt "N")
       (declare (ignore var-1))
       (when var-0 (setf jobvt var-0))
       ret-val)))
   (setf info -2))
  ((not (or alls vals inds)) (setf info -3)) ((< m 0) (setf info -4))
  ((< n 0) (setf info -5)) ((> m lda) (setf info -7))
  ((> minmn 0)
   (cond
    (vals
     (cond ((< vl zero) (setf info -8)) ((<= vu vl) (setf info -9))))
    (inds
     (cond
      ((or (< il 1)
           (> il
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 minmn))))
       (setf info -10))
      ((or
        (< iu
           (min (the f2cl-lib:integer4 minmn)
                (the f2cl-lib:integer4 il)))
        (> iu minmn))
       (setf info -11)))))
   (cond
    ((= info 0)
     (cond ((and wantu (< ldu m)) (setf info -15))
           ((and wantvt (< ldvt minmn)) (setf info -16)))))))
(cond
 ((= info 0) (setf minwrk 1) (setf maxwrk 1)
  (cond
   ((> minmn 0)
    (cond
     ((>= m n)
      (setf mnthr
              (multiple-value-bind
```

```
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 6 "DGESVD" (f2cl-lib:f2cl-// jobu jobvt) m n
                 0 0)
              (declare (ignore var-0 var-1 var-2 var-5 var-6))
              (when var-3 (setf m var-3))
              (when var-4 (setf n var-4))
              ret-val))
    (cond
     ((>= m mnthr)
      (setf maxwrk
              (f2cl-lib:int-add
               (f2cl-lib:int-mul n
                                 (f2cl-lib:int-add
                                  (f2cl-lib:int-mul n 2) 16))
               (f2cl-lib:int-mul n
                                 (multiple-value-bind
                                     (ret-val var-0 var-1 var-2 var-3
                                      var-4 var-5 var-6)
                                     (ilaenv 1 "DGEQRF" " " m n -1
                                      -1)
                                   (declare
                                    (ignore var-0 var-1 var-2 var-5
                                     var-6))
                                   (when var-3 (setf m var-3))
                                   (when var-4 (setf n var-4))
                                   ret-val))))
      (setf maxwrk
              (max (the f2cl-lib:integer4 maxwrk)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add
                         (f2cl-lib:int-mul n
                                           (f2cl-lib:int-add
                                            (f2cl-lib:int-mul n 2)
                                            20))
                         (f2cl-lib:int-mul 2 n
                                           (multiple-value-bind
                                               (ret-val var-0 var-1
                                                var-2 var-3 var-4
                                                var-5 var-6)
                                               (ilaenv 1 "DGEBRD" " "
                                                n n -1 -1)
                                             (declare
                                              (ignore var-0 var-1
                                               var-2 var-5 var-6))
                                             (when var-3
                                               (setf n var-3))
                                             (when var-4
                                               (setf n var-4))
                                             ret-val))))))
      (setf minwrk
```

```
                (f2cl-lib:int-mul n
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-mul n 2) 21))))
        (t
         (setf maxwrk
                (f2cl-lib:int-add
                 (f2cl-lib:int-mul n
                                   (f2cl-lib:int-add
                                    (f2cl-lib:int-mul n 2) 19))
                 (f2cl-lib:int-mul (f2cl-lib:int-add m n)
                                   (multiple-value-bind
                                       (ret-val var-0 var-1 var-2 var-3
                                        var-4 var-5 var-6)
                                       (ilaenv 1 "DGEBRD" " " m n -1
                                        -1)
                                     (declare
                                      (ignore var-0 var-1 var-2 var-5
                                       var-6))
                                     (when var-3 (setf m var-3))
                                     (when var-4 (setf n var-4))
                                     ret-val))))
         (setf minwrk
                (f2cl-lib:int-add
                 (f2cl-lib:int-mul n
                                   (f2cl-lib:int-add
                                    (f2cl-lib:int-mul n 2) 20))
                 m)))))
       (t
        (setf mnthr
               (multiple-value-bind
                   (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                   (ilaenv 6 "DGESVD" (f2cl-lib:f2cl-// jobu jobvt) m n
                    0 0)
                (declare (ignore var-0 var-1 var-2 var-5 var-6))
                (when var-3 (setf m var-3))
                (when var-4 (setf n var-4))
                ret-val))
        (cond
         ((>= n mnthr)
          (setf maxwrk
                (f2cl-lib:int-add
                 (f2cl-lib:int-mul m
                                   (f2cl-lib:int-add
                                    (f2cl-lib:int-mul m 2) 16))
                 (f2cl-lib:int-mul m
                                   (multiple-value-bind
                                       (ret-val var-0 var-1 var-2 var-3
                                        var-4 var-5 var-6)
                                       (ilaenv 1 "DGELQF" " " m n -1
                                        -1)
```

```
                                (declare
                                 (ignore var-0 var-1 var-2 var-5
                                  var-6))
                                (when var-3 (setf m var-3))
                                (when var-4 (setf n var-4))
                                ret-val))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add
                    (f2cl-lib:int-mul m
                                      (f2cl-lib:int-add
                                       (f2cl-lib:int-mul m 2)
                                       20))
                    (f2cl-lib:int-mul 2 m
                                      (multiple-value-bind
                                          (ret-val var-0 var-1
                                           var-2 var-3 var-4
                                           var-5 var-6)
                                          (ilaenv 1 "DGEBRD" " "
                                           m m -1 -1)
                                        (declare
                                         (ignore var-0 var-1
                                          var-2 var-5 var-6))
                                        (when var-3
                                          (setf m var-3))
                                        (when var-4
                                          (setf m var-4))
                                        ret-val))))))
 (setf minwrk
         (f2cl-lib:int-mul m
                           (f2cl-lib:int-add
                            (f2cl-lib:int-mul m 2) 21))))
(t
 (setf maxwrk
         (f2cl-lib:int-add
          (f2cl-lib:int-mul m
                            (f2cl-lib:int-add
                             (f2cl-lib:int-mul m 2) 19))
          (f2cl-lib:int-mul (f2cl-lib:int-add m n)
                            (multiple-value-bind
                                (ret-val var-0 var-1 var-2 var-3
                                 var-4 var-5 var-6)
                                (ilaenv 1 "DGEBRD" " " m n -1
                                 -1)
                              (declare
                               (ignore var-0 var-1 var-2 var-5
                                var-6))
                              (when var-3 (setf m var-3))
                              (when var-4 (setf n var-4))
```

```
                                    ret-val))))
        (setf minwrk
                (f2cl-lib:int-add
                 (f2cl-lib:int-mul m
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-mul m 2) 20))
                 n)))))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4 minwrk)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:dble maxwrk))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -19)))))
(cond
 ((/= info 0) (xerbla "DGESVDX" (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(cond ((or (= m 0) (= n 0)) (go end_label)))
(cond
 (alls (f2cl-lib:f2cl-set-string rngtgk "I" (string 1)) (setf iltgk 1)
  (setf iutgk
          (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n))))
 (inds (f2cl-lib:f2cl-set-string rngtgk "I" (string 1)) (setf iltgk il)
  (setf iutgk iu))
 (t (f2cl-lib:f2cl-set-string rngtgk "V" (string 1)) (setf iltgk 0)
  (setf iutgk 0)))
(setf eps (dlamch "P"))
(setf smlnum (/ (f2cl-lib:fsqrt (dlamch "S")) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "M" m n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iscl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf iscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 anrm smlnum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
```

```
      (when var-9 (setf info var-9))))
    ((> anrm bignum) (setf iscl 1)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (dlascl "G" 0 0 anrm bignum m n a lda info)
       (declare (ignore var-0 var-1 var-2 var-7))
       (when var-3 (setf anrm var-3))
       (when var-4 (setf bignum var-4))
       (when var-5 (setf m var-5))
       (when var-6 (setf n var-6))
       (when var-8 (setf lda var-8))
       (when var-9 (setf info var-9)))))
  (cond
   ((>= m n)
    (cond
     ((>= m mnthr) (setf itau 1) (setf itemp (f2cl-lib:int-add itau n))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (dgeqrf m n a lda
           (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                                 work-%offset%)
           (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                                 work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
        (declare (ignore var-2 var-4 var-5 var-6))
        (setf m var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf info var-7))
      (setf iqrf itemp)
      (setf id (f2cl-lib:int-add iqrf (f2cl-lib:int-mul n n)))
      (setf ie (f2cl-lib:int-add id n))
      (setf itauq (f2cl-lib:int-add ie n))
      (setf itaup (f2cl-lib:int-add itauq n))
      (setf itemp (f2cl-lib:int-add itaup n))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlacpy "U" n n a lda
           (f2cl-lib:array-slice work-%data% double-float (iqrf) ((1 *))
                                 work-%offset%)
           n)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf n var-6)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1) zero
           zero
           (f2cl-lib:array-slice work-%data% double-float ((+ iqrf 1))
                                 ((1 *)) work-%offset%)
```

```
      n)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf n var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgebrd n n
     (f2cl-lib:array-slice work-%data% double-float (iqrf) ((1 *))
                           work-%offset%)
     n
     (f2cl-lib:array-slice work-%data% double-float (id) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf n var-3)
  (setf info var-10))
(setf itgkz itemp)
(setf itemp
        (f2cl-lib:int-add itgkz
                          (f2cl-lib:int-mul n
                                            (f2cl-lib:int-add
                                             (f2cl-lib:int-mul n 2)
                                             1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (dbdsvdx "U" jobz rngtgk n
     (f2cl-lib:array-slice work-%data% double-float (id) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     vl vu iltgk iutgk ns s
     (f2cl-lib:array-slice work-%data% double-float (itgkz) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-mul n 2)
     (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                           work-%offset%)
     iwork info)
```

```
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-8 var-9 var-11 var-12
    var-13 var-14 var-15))
  (setf jobz var-1)
  (setf rngtgk var-2)
  (setf n var-3)
  (setf ns var-10)
  (setf info var-16))
(cond
 (wantu (setf j itgkz)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy n
           (f2cl-lib:array-slice work-%data% double-float (j)
                                 ((1 *)) work-%offset%)
           1
           (f2cl-lib:array-slice u-%data% double-float (1 i)
                                 ((1 ldu) (1 *)) u-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf n var-0)))
      (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul n 2)))
     label100000))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" (f2cl-lib:int-sub m n) n zero zero
       (f2cl-lib:array-slice u-%data% double-float ((+ n 1) 1)
                             ((1 ldu) (1 *)) u-%offset%)
       ldu)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf n var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dormbr "Q" "L" "N" n ns n
       (f2cl-lib:array-slice work-%data% double-float (iqrf)
                             ((1 *)) work-%offset%)
       n
       (f2cl-lib:array-slice work-%data% double-float (itauq)
                             ((1 *)) work-%offset%)
       u ldu
       (f2cl-lib:array-slice work-%data% double-float (itemp)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
```

```
      (when var-3 (setf n var-3))
      (when var-4 (setf ns var-4))
      (when var-5 (setf n var-5))
      (when var-7 (setf n var-7))
      (when var-10 (setf ldu var-10))
      (when var-13 (setf info var-13)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (dormqr "L" "N" m ns n a lda
         (f2cl-lib:array-slice work-%data% double-float (itau)
                               ((1 *)) work-%offset%)
         u ldu
         (f2cl-lib:array-slice work-%data% double-float (itemp)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf ns var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldu var-9))
      (when var-12 (setf info var-12)))))
  (cond
   (wantvt (setf j (f2cl-lib:int-add itgkz n))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i ns) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dcopy n
             (f2cl-lib:array-slice work-%data% double-float (j)
                                   ((1 *)) work-%offset%)
             1
             (f2cl-lib:array-slice vt-%data% double-float (i 1)
                                   ((1 ldvt) (1 *)) vt-%offset%)
             ldvt)
          (declare (ignore var-1 var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-4 (setf ldvt var-4)))
        (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul n 2)))
       label100001))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (dormbr "P" "R" "T" ns n n
         (f2cl-lib:array-slice work-%data% double-float (iqrf)
                               ((1 *)) work-%offset%)
         n
         (f2cl-lib:array-slice work-%data% double-float (itaup)
                               ((1 *)) work-%offset%)
```

```
        vt ldvt
        (f2cl-lib:array-slice work-%data% double-float (itemp)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
      (when var-3 (setf ns var-3))
      (when var-4 (setf n var-4))
      (when var-5 (setf n var-5))
      (when var-7 (setf n var-7))
      (when var-10 (setf ldvt var-10))
      (when var-13 (setf info var-13))))))
 (t (setf id 1) (setf ie (f2cl-lib:int-add id n))
  (setf itauq (f2cl-lib:int-add ie n))
  (setf itaup (f2cl-lib:int-add itauq n))
  (setf itemp (f2cl-lib:int-add itaup n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dgebrd m n a lda
       (f2cl-lib:array-slice work-%data% double-float (id) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
    (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf info var-10))
  (setf itgkz itemp)
  (setf itemp
          (f2cl-lib:int-add itgkz
                            (f2cl-lib:int-mul n
                                              (f2cl-lib:int-add
                                               (f2cl-lib:int-mul n 2)
                                               1))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16)
      (dbdsvdx "U" jobz rngtgk n
       (f2cl-lib:array-slice work-%data% double-float (id) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
```

```
                              work-%offset%)
        vl vu iltgk iutgk ns s
        (f2cl-lib:array-slice work-%data% double-float (itgkz) ((1 *))
                              work-%offset%)
        (f2cl-lib:int-mul n 2)
        (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                              work-%offset%)
        iwork info)
    (declare
     (ignore var-0 var-4 var-5 var-6 var-7 var-8 var-9 var-11 var-12
      var-13 var-14 var-15))
    (setf jobz var-1)
    (setf rngtgk var-2)
    (setf n var-3)
    (setf ns var-10)
    (setf info var-16))
  (cond
   (wantu (setf j itgkz)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i ns) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dcopy n
             (f2cl-lib:array-slice work-%data% double-float (j)
                                   ((1 *)) work-%offset%)
             1
             (f2cl-lib:array-slice u-%data% double-float (1 i)
                                   ((1 ldu) (1 *)) u-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf n var-0)))
        (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul n 2)))
       label100002))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "A" (f2cl-lib:int-sub m n) n zero zero
         (f2cl-lib:array-slice u-%data% double-float ((+ n 1) 1)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldu var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (dormbr "Q" "L" "N" m ns n a lda
         (f2cl-lib:array-slice work-%data% double-float (itauq)
                               ((1 *)) work-%offset%)
         u ldu
```

```
            (f2cl-lib:array-slice work-%data% double-float (itemp)
                                  ((1 *)) work-%offset%)
            (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) ierr)
          (declare
           (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
          (when var-3 (setf m var-3))
          (when var-4 (setf ns var-4))
          (when var-5 (setf n var-5))
          (when var-7 (setf lda var-7))
          (when var-10 (setf ldu var-10))
          (when var-13 (setf ierr var-13)))))
      (cond
       (wantvt (setf j (f2cl-lib:int-add itgkz n))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i ns) nil)
          (tagbody
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (dcopy n
                 (f2cl-lib:array-slice work-%data% double-float (j)
                                       ((1 *)) work-%offset%)
                 1
                 (f2cl-lib:array-slice vt-%data% double-float (i 1)
                                       ((1 ldvt) (1 *)) vt-%offset%)
                 ldvt)
              (declare (ignore var-1 var-2 var-3))
              (when var-0 (setf n var-0))
              (when var-4 (setf ldvt var-4)))
            (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul n 2)))
           label100003))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11 var-12 var-13)
            (dormbr "P" "R" "T" ns n n a lda
             (f2cl-lib:array-slice work-%data% double-float (itaup)
                                   ((1 *)) work-%offset%)
             vt ldvt
             (f2cl-lib:array-slice work-%data% double-float (itemp)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) ierr)
          (declare
           (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
          (when var-3 (setf ns var-3))
          (when var-4 (setf n var-4))
          (when var-5 (setf n var-5))
          (when var-7 (setf lda var-7))
          (when var-10 (setf ldvt var-10))
          (when var-13 (setf ierr var-13))))))))
    (t
     (cond
      ((>= n mnthr) (setf itau 1) (setf itemp (f2cl-lib:int-add itau m))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dgelqf m n a lda
     (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-7))
(setf ilqf itemp)
(setf id (f2cl-lib:int-add ilqf (f2cl-lib:int-mul m m)))
(setf ie (f2cl-lib:int-add id m))
(setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf itemp (f2cl-lib:int-add itaup m))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% double-float (ilqf) ((1 *))
                           work-%offset%)
     m)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf m var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1) zero
     zero
     (f2cl-lib:array-slice work-%data% double-float ((+ ilqf m))
                           ((1 *)) work-%offset%)
     m)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf m var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgebrd m m
     (f2cl-lib:array-slice work-%data% double-float (ilqf) ((1 *))
                           work-%offset%)
     m
     (f2cl-lib:array-slice work-%data% double-float (id) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
```

```
      (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                            work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf m var-3)
  (setf info var-10))
(setf itgkz itemp)
(setf itemp
        (f2cl-lib:int-add itgkz
                          (f2cl-lib:int-mul m
                                            (f2cl-lib:int-add
                                             (f2cl-lib:int-mul m 2)
                                             1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (dbdsvdx "U" jobz rngtgk m
     (f2cl-lib:array-slice work-%data% double-float (id) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     vl vu iltgk iutgk ns s
     (f2cl-lib:array-slice work-%data% double-float (itgkz) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-mul m 2)
     (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                           work-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-8 var-9 var-11 var-12
    var-13 var-14 var-15))
  (setf jobz var-1)
  (setf rngtgk var-2)
  (setf m var-3)
  (setf ns var-10)
  (setf info var-16))
(cond
 (wantu (setf j itgkz)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy m
           (f2cl-lib:array-slice work-%data% double-float (j)
```

```
                                    ((1 *)) work-%offset%)
                1
                (f2cl-lib:array-slice u-%data% double-float (1 i)
                                    ((1 ldu) (1 *)) u-%offset%)
                1)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf m var-0)))
          (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul m 2)))
         label100004))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13)
          (dormbr "Q" "L" "N" m ns m
           (f2cl-lib:array-slice work-%data% double-float (ilqf)
                                 ((1 *)) work-%offset%)
           m
           (f2cl-lib:array-slice work-%data% double-float (itauq)
                                 ((1 *)) work-%offset%)
           u ldu
           (f2cl-lib:array-slice work-%data% double-float (itemp)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
        (declare
         (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
        (when var-3 (setf m var-3))
        (when var-4 (setf ns var-4))
        (when var-5 (setf m var-5))
        (when var-7 (setf m var-7))
        (when var-10 (setf ldu var-10))
        (when var-13 (setf info var-13)))))
    (cond
     (wantvt (setf j (f2cl-lib:int-add itgkz m))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i ns) nil)
        (tagbody
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dcopy m
               (f2cl-lib:array-slice work-%data% double-float (j)
                                     ((1 *)) work-%offset%)
               1
               (f2cl-lib:array-slice vt-%data% double-float (i 1)
                                     ((1 ldvt) (1 *)) vt-%offset%)
               ldvt)
            (declare (ignore var-1 var-2 var-3))
            (when var-0 (setf m var-0))
            (when var-4 (setf ldvt var-4)))
          (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul m 2)))
         label100005))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlaset "A" m (f2cl-lib:int-sub n m) zero zero
```

```
          (f2cl-lib:array-slice vt-%data% double-float
                                (1 (f2cl-lib:int-add m 1))
                                ((1 ldvt) (1 *)) vt-%offset%)
          ldvt)
        (declare (ignore var-0 var-2 var-5))
        (when var-1 (setf m var-1))
        (when var-3 (setf zero var-3))
        (when var-4 (setf zero var-4))
        (when var-6 (setf ldvt var-6)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13)
          (dormbr "P" "R" "T" ns m m
           (f2cl-lib:array-slice work-%data% double-float (ilqf)
                                 ((1 *)) work-%offset%)
           m
           (f2cl-lib:array-slice work-%data% double-float (itaup)
                                 ((1 *)) work-%offset%)
           vt ldvt
           (f2cl-lib:array-slice work-%data% double-float (itemp)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
        (declare
         (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
        (when var-3 (setf ns var-3))
        (when var-4 (setf m var-4))
        (when var-5 (setf m var-5))
        (when var-7 (setf m var-7))
        (when var-10 (setf ldvt var-10))
        (when var-13 (setf info var-13)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (dormlq "R" "N" ns n m a lda
           (f2cl-lib:array-slice work-%data% double-float (itau)
                                 ((1 *)) work-%offset%)
           vt ldvt
           (f2cl-lib:array-slice work-%data% double-float (itemp)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf ns var-2))
        (when var-3 (setf n var-3))
        (when var-4 (setf m var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldvt var-9))
        (when var-12 (setf info var-12))))))
  (t (setf id 1) (setf ie (f2cl-lib:int-add id m))
   (setf itauq (f2cl-lib:int-add ie m))
   (setf itaup (f2cl-lib:int-add itauq m))
```

```
(setf itemp (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgebrd m n a lda
     (f2cl-lib:array-slice work-%data% double-float (id) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-10))
(setf itgkz itemp)
(setf itemp
        (f2cl-lib:int-add itgkz
                          (f2cl-lib:int-mul m
                                            (f2cl-lib:int-add
                                             (f2cl-lib:int-mul m 2)
                                             1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (dbdsvdx "L" jobz rngtgk m
     (f2cl-lib:array-slice work-%data% double-float (id) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     vl vu iltgk iutgk ns s
     (f2cl-lib:array-slice work-%data% double-float (itgkz) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-mul m 2)
     (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                           work-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-8 var-9 var-11 var-12
    var-13 var-14 var-15))
  (setf jobz var-1)
  (setf rngtgk var-2)
  (setf m var-3)
  (setf ns var-10)
```

```
  (setf info var-16))
(cond
 (wantu (setf j itgkz)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy m
           (f2cl-lib:array-slice work-%data% double-float (j)
                                 ((1 *)) work-%offset%)
           1
           (f2cl-lib:array-slice u-%data% double-float (1 i)
                                 ((1 ldu) (1 *)) u-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf m var-0)))
      (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul m 2)))
     label100006))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dormbr "Q" "L" "N" m ns n a lda
       (f2cl-lib:array-slice work-%data% double-float (itauq)
                             ((1 *)) work-%offset%)
       u ldu
       (f2cl-lib:array-slice work-%data% double-float (itemp)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf ns var-4))
    (when var-5 (setf n var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldu var-10))
    (when var-13 (setf info var-13)))))
(cond
 (wantvt (setf j (f2cl-lib:int-add itgkz m))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy m
           (f2cl-lib:array-slice work-%data% double-float (j)
                                 ((1 *)) work-%offset%)
           1
           (f2cl-lib:array-slice vt-%data% double-float (i 1)
                                 ((1 ldvt) (1 *)) vt-%offset%)
           ldvt)
        (declare (ignore var-1 var-2 var-3))
```

```
                (when var-0 (setf m var-0))
                (when var-4 (setf ldvt var-4)))
              (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul m 2)))
            label100007))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dlaset "A" m (f2cl-lib:int-sub n m) zero zero
             (f2cl-lib:array-slice vt-%data% double-float
                                   (1 (f2cl-lib:int-add m 1))
                                   ((1 ldvt) (1 *)) vt-%offset%)
             ldvt)
          (declare (ignore var-0 var-2 var-5))
          (when var-1 (setf m var-1))
          (when var-3 (setf zero var-3))
          (when var-4 (setf zero var-4))
          (when var-6 (setf ldvt var-6)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11 var-12 var-13)
            (dormbr "P" "R" "T" ns n m a lda
             (f2cl-lib:array-slice work-%data% double-float (itaup)
                                   ((1 *)) work-%offset%)
             vt ldvt
             (f2cl-lib:array-slice work-%data% double-float (itemp)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
          (declare
           (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
          (when var-3 (setf ns var-3))
          (when var-4 (setf n var-4))
          (when var-5 (setf m var-5))
          (when var-7 (setf lda var-7))
          (when var-10 (setf ldvt var-10))
          (when var-13 (setf info var-13)))))))))
(cond
 ((= iscl 1)
  (if (> anrm bignum)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (dlascl "G" 0 0 bignum anrm minmn 1 s minmn info)
        (declare (ignore var-0 var-1 var-2 var-6 var-7))
        (when var-3 (setf bignum var-3))
        (when var-4 (setf anrm var-4))
        (when var-5 (setf minmn var-5))
        (when var-8 (setf minmn var-8))
        (when var-9 (setf info var-9))))
  (if (< anrm smlnum)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (dlascl "G" 0 0 smlnum anrm minmn 1 s minmn info)
        (declare (ignore var-0 var-1 var-2 var-6 var-7))
```

```
                (when var-3 (setf smlnum var-3))
                (when var-4 (setf anrm var-4))
                (when var-5 (setf minmn var-5))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf info var-9))))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (f2cl-lib:dble maxwrk))
        (go end_label)
       end_label
        (return
         (values jobu jobvt range m n nil lda nil nil nil nil ns nil nil ldu
                 nil ldvt nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgesvdx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::jobu
                                              fortran-to-lisp::jobvt
                                              fortran-to-lisp::range
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              nil fortran-to-lisp::ns nil nil
                                              fortran-to-lisp::ldu nil
                                              fortran-to-lisp::ldvt nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
```

```
                                        '(fortran-to_lisp::dgelqf
                                          fortran-to_lisp::dbdsvdx
                                          fortran-to_lisp::dgebrd
                                          fortran-to_lisp::dgeqrf))))
```

———-

28.4 dgesvdx

28.4.1 Axiom unit tests

— **dgesvdx.input** —

```
)set break resume
)sys rm -f dgesvdx.output
)spool dgesvdx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

28.4.2 Axiom help page

— **dgesvdx.help** —

```
====================================================================
dgesvdx examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgesvdx (JOBU, JOBVT, RANGE, M, N, A, LDA, VL, VU, IL, IU,
           NS, S, U, LDU, VT, LDVT, WORK, LWORK, IWORK, INFO)
            DGESVDX computes the singular value decomposition (SVD) for GE
```

```
          matrices

Function/Subroutine Documentation
   subroutine dgesvdx (character JOBU, character JOBVT, character RANGE,
       integer M, integer N, double precision, dimension( lda, * ) A, integer
       LDA, double precision VL, double precision VU, integer IL, integer IU,
       integer NS, double precision, dimension( * ) S, double precision,
       dimension( ldu, * ) U, integer LDU, double precision, dimension( ldvt,
       * ) VT, integer LDVT, double precision, dimension( * ) WORK, integer
       LWORK, integer, dimension( * ) IWORK, integer INFO)
        DGESVDX computes the singular value decomposition (SVD) for GE
       matrices

Purpose:
   DGESVDX computes the singular value decomposition (SVD) of a real
   M-by-N matrix A, optionally computing the left and/or right singular
   vectors. The SVD is written

   A = U * SIGMA * transpose(V)

   where SIGMA is an M-by-N matrix which is zero except for its
   min(m,n) diagonal elements, U is an M-by-M orthogonal matrix, and
   V is an N-by-N orthogonal matrix.  The diagonal elements of SIGMA
   are the singular values of A; they are real and non-negative, and
   are returned in descending order.  The first min(m,n) columns of
   U and V are the left and right singular vectors of A.

   DGESVDX uses an eigenvalue problem for obtaining the SVD, which
   allows for the computation of a subset of singular values and
   vectors. See DBDSVDX for details.

   Note that the routine returns V**T, not V.

Parameters:

 JOBU
   JOBU is CHARACTER*1
   Specifies options for computing all or part of the matrix U:
   = 'V':  the first min(m,n) columns of U (the left singular
           vectors) or as specified by RANGE are returned in
           the array U;
   = 'N':  no columns of U (no left singular vectors) are computed.

 JOBVT
   JOBVT is CHARACTER*1
   Specifies options for computing all or part of the matrix V**T:
   = 'V':  the first min(m,n) rows of V**T (the right singular
           vectors) or as specified by RANGE are returned in
           the array VT;
   = 'N':  no rows of V**T (no right singular vectors) are computed.
```

```
RANGE
  RANGE is CHARACTER*1
  = 'A': all singular values will be found.
  = 'V': all singular values in the half-open interval (VL,VU]
         will be found.
  = 'I': the IL-th through IU-th singular values will be found.

M
  M is INTEGER
  The number of rows of the input matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the input matrix A.  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit, the contents of A are destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

VL
  VL is DOUBLE PRECISION
  VL >=0.

VU
  VU is DOUBLE PRECISION
  If RANGE='V', the lower and upper bounds of the interval to
  be searched for singular values. VU > VL.
  Not referenced if RANGE = 'A' or 'I'.

IL
  IL is INTEGER

IU
  IU is INTEGER
  If RANGE='I', the indices (in ascending order) of the
  smallest and largest singular values to be returned.
  1 <= IL <= IU <= min(M,N), if min(M,N) > 0.
  Not referenced if RANGE = 'A' or 'V'.

NS
  NS is INTEGER
  The total number of singular values found,
  0 <= NS <= min(M,N).
  If RANGE = 'A', NS = min(M,N); if RANGE = 'I', NS = IU-IL+1.
```

```
S
  S is DOUBLE PRECISION array, dimension (min(M,N))
  The singular values of A, sorted so that S(i) >= S(i+1).

U
  U is DOUBLE PRECISION array, dimension (LDU,UCOL)
  If JOBU = 'V', U contains columns of U (the left singular
  vectors, stored columnwise) as specified by RANGE; if
  JOBU = 'N', U is not referenced.
  Note: The user must ensure that UCOL >= NS; if RANGE = 'V',
  the exact value of NS is not known ILQFin advance and an upper
  bound must be used.

LDU
  LDU is INTEGER
  The leading dimension of the array U.  LDU >= 1; if
  JOBU = 'V', LDU >= M.

VT
  VT is DOUBLE PRECISION array, dimension (LDVT,N)
  If JOBVT = 'V', VT contains the rows of V**T (the right singular
  vectors, stored rowwise) as specified by RANGE; if JOBVT = 'N',
  VT is not referenced.
  Note: The user must ensure that LDVT >= NS; if RANGE = 'V',
  the exact value of NS is not known in advance and an upper
  bound must be used.

LDVT
  LDVT is INTEGER
  The leading dimension of the array VT.  LDVT >= 1; if
  JOBVT = 'V', LDVT >= NS (see above).

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK;

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  LWORK >= MAX(1,MIN(M,N)*(MIN(M,N)+4)) for the paths (see
  comments inside the code):
  - PATH 1  (M much larger than N)
  - PATH 1t (N much larger than M)
  LWORK >= MAX(1,MIN(M,N)*2+MAX(M,N)) for the other paths.
  For good performance, LWORK should generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
```

```
  message related to LWORK is issued by XERBLA.

 IWORK
  IWORK is INTEGER array, dimension (12*MIN(M,N))
  If INFO = 0, the first NS elements of IWORK are zero. If INFO > 0,
  then IWORK contains the indices of the eigenvectors that failed
  to converge in DBDSVDX/DSTEVX.

 INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, then i eigenvectors failed to converge
        in DBDSVDX/DSTEVX.
        if INFO = N*2 + 1, an internal error occurred in DBDSVDX
```

———-

28.4.3 fortran code

— dgesvdx.f —

```
*  =====================================================================
      SUBROUTINE DGESVDX( JOBU, JOBVT, RANGE, M, N, A, LDA, VL, VU,
     $                    IL, IU, NS, S, U, LDU, VT, LDVT, WORK,
     $                    LWORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBU, JOBVT, RANGE
      INTEGER            IL, INFO, IU, LDA, LDU, LDVT, LWORK, M, N, NS
      DOUBLE PRECISION   VL, VU
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), S( * ), U( LDU, * ),
     $                   VT( LDVT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
```

```
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      CHARACTER          JOBZ, RNGTGK
      LOGICAL            ALLS, INDS, LQUERY, VALS, WANTU, WANTVT
      INTEGER            I, ID, IE, IERR, ILQF, ILTGK, IQRF, ISCL,
     $                   ITAU, ITAUP, ITAUQ, ITEMP, ITGKZ, IUTGK,
     $                   J, MAXWRK, MINMN, MINWRK, MNTHR
      DOUBLE PRECISION   ABSTOL, ANRM, BIGNUM, EPS, SMLNUM
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DBDSVDX, DGEBRD, DGELQF, DGEQRF, DLACPY,
     $                   DLASCL, DLASET, DORMBR, DORMLQ, DORMQR,
     $                   DSCAL, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE, DNRM2
      EXTERNAL           LSAME, ILAENV, DLAMCH, DLANGE, DNRM2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      NS = 0
      INFO = 0
      ABSTOL = 2*DLAMCH('S')
      LQUERY = ( LWORK.EQ.-1 )
      MINMN = MIN( M, N )

      WANTU = LSAME( JOBU, 'V' )
      WANTVT = LSAME( JOBVT, 'V' )
      IF( WANTU .OR. WANTVT ) THEN
         JOBZ = 'V'
      ELSE
         JOBZ = 'N'
      END IF
      ALLS = LSAME( RANGE, 'A' )
      VALS = LSAME( RANGE, 'V' )
      INDS = LSAME( RANGE, 'I' )
*
      INFO = 0
      IF( .NOT.LSAME( JOBU, 'V' ) .AND.
```

```
     $    .NOT.LSAME( JOBU, 'N' ) ) THEN
          INFO = -1
      ELSE IF( .NOT.LSAME( JOBVT, 'V' ) .AND.
     $         .NOT.LSAME( JOBVT, 'N' ) ) THEN
          INFO = -2
      ELSE IF( .NOT.( ALLS .OR. VALS .OR. INDS ) ) THEN
          INFO = -3
      ELSE IF( M.LT.0 ) THEN
          INFO = -4
      ELSE IF( N.LT.0 ) THEN
          INFO = -5
      ELSE IF( M.GT.LDA ) THEN
          INFO = -7
      ELSE IF( MINMN.GT.0 ) THEN
          IF( VALS ) THEN
             IF( VL.LT.ZERO ) THEN
                INFO = -8
             ELSE IF( VU.LE.VL ) THEN
                INFO = -9
             END IF
          ELSE IF( INDS ) THEN
             IF( IL.LT.1 .OR. IL.GT.MAX( 1, MINMN ) ) THEN
                INFO = -10
             ELSE IF( IU.LT.MIN( MINMN, IL ) .OR. IU.GT.MINMN ) THEN
                INFO = -11
             END IF
          END IF
          IF( INFO.EQ.0 ) THEN
             IF( WANTU .AND. LDU.LT.M ) THEN
                INFO = -15
             ELSE IF( WANTVT .AND. LDVT.LT.MINMN ) THEN
                INFO = -16
             END IF
          END IF
      END IF
*
*     Compute workspace
*     (Note: Comments in the code beginning "Workspace:" describe the
*     minimal amount of workspace needed at that point in the code,
*     as well as the preferred amount for good performance.
*     NB refers to the optimal block size for the immediately
*     following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
          MINWRK = 1
          MAXWRK = 1
          IF( MINMN.GT.0 ) THEN
             IF( M.GE.N ) THEN
                MNTHR = ILAENV( 6, 'DGESVD', JOBU // JOBVT, M, N, 0, 0 )
                IF( M.GE.MNTHR ) THEN
```

```
*
*                Path 1 (M much larger than N)
*
                 MAXWRK = N*(N*2+16) +
     $                   N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
                 MAXWRK = MAX( MAXWRK, N*(N*2+20) + 2*N*
     $                   ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                 MINWRK = N*(N*2+21)
              ELSE
*
*                Path 2 (M at least N, but not much larger)
*
                 MAXWRK = N*(N*2+19) + ( M+N )*
     $                   ILAENV( 1, 'DGEBRD', ' ', M, N, -1, -1 )
                 MINWRK = N*(N*2+20) + M
              END IF
           ELSE
              MNTHR = ILAENV( 6, 'DGESVD', JOBU // JOBVT, M, N, 0, 0 )
              IF( N.GE.MNTHR ) THEN
*
*                Path 1t (N much larger than M)
*
                 MAXWRK = M*(M*2+16) +
     $                   M*ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
                 MAXWRK = MAX( MAXWRK, M*(M*2+20) + 2*M*
     $                   ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
                 MINWRK = M*(M*2+21)
              ELSE
*
*                Path 2t (N greater than M, but not much larger)
*
                 MAXWRK = M*(M*2+19) + ( M+N )*
     $                   ILAENV( 1, 'DGEBRD', ' ', M, N, -1, -1 )
                 MINWRK = M*(M*2+20) + N
              END IF
           END IF
        END IF
        MAXWRK = MAX( MAXWRK, MINWRK )
        WORK( 1 ) = DBLE( MAXWRK )
*
        IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -19
        END IF
     END IF
*
     IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'DGESVDX', -INFO )
        RETURN
     ELSE IF( LQUERY ) THEN
        RETURN
```

```
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         RETURN
      END IF
*
*     Set singular values indices accord to RANGE.
*
      IF( ALLS ) THEN
         RNGTGK = 'I'
         ILTGK = 1
         IUTGK = MIN( M, N )
      ELSE IF( INDS ) THEN
         RNGTGK = 'I'
         ILTGK = IL
         IUTGK = IU
      ELSE
         RNGTGK = 'V'
         ILTGK = 0
         IUTGK = 0
      END IF
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = SQRT( DLAMCH( 'S' ) ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = DLANGE( 'M', M, N, A, LDA, DUM )
      ISCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ISCL = 1
         CALL DLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ISCL = 1
         CALL DLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
      END IF
*
      IF( M.GE.N ) THEN
*
*        A has at least as many rows as columns. If A has sufficiently
*        more rows than columns, first reduce A using the QR
*        decomposition.
*
         IF( M.GE.MNTHR ) THEN
*
```

```
*           Path 1 (M much larger than N):
*           A = Q * R = Q * ( QB * B * PB**T )
*                     = Q * ( QB * ( UB * S * VB**T ) * PB**T )
*           U = Q * QB * UB; V**T = VB**T * PB**T
*
*           Compute A=Q*R
*           (Workspace: need 2*N, prefer N+N*NB)
*
            ITAU = 1
            ITEMP = ITAU + N
            CALL DGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( ITEMP ),
     $                   LWORK-ITEMP+1, INFO )
*
*           Copy R into WORK and bidiagonalize it:
*           (Workspace: need N*N+5*N, prefer N*N+4*N+2*N*NB)
*
            IQRF = ITEMP
            ID = IQRF + N*N
            IE = ID + N
            ITAUQ = IE + N
            ITAUP = ITAUQ + N
            ITEMP = ITAUP + N
            CALL DLACPY( 'U', N, N, A, LDA, WORK( IQRF ), N )
            CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, WORK( IQRF+1 ), N )
            CALL DGEBRD( N, N, WORK( IQRF ), N, WORK( ID ), WORK( IE ),
     $                   WORK( ITAUQ ), WORK( ITAUP ), WORK( ITEMP ),
     $                   LWORK-ITEMP+1, INFO )
*
*           Solve eigenvalue problem TGK*Z=Z*S.
*           (Workspace: need 14*N + 2*N*(N+1))
*
            ITGKZ = ITEMP
            ITEMP = ITGKZ + N*(N*2+1)
            CALL DBDSVDX( 'U', JOBZ, RNGTGK, N, WORK( ID ), WORK( IE ),
     $                    VL, VU, ILTGK, IUTGK, NS, S, WORK( ITGKZ ),
     $                    N*2, WORK( ITEMP ), IWORK, INFO)
*
*           If needed, compute left singular vectors.
*
            IF( WANTU ) THEN
               J = ITGKZ
               DO I = 1, NS
                  CALL DCOPY( N, WORK( J ), 1, U( 1,I ), 1 )
                  J = J + N*2
               END DO
               CALL DLASET( 'A', M-N, N, ZERO, ZERO, U( N+1,1 ), LDU )
*
*              Call DORMBR to compute QB*UB.
*              (Workspace in WORK( ITEMP ): need N, prefer N*NB)
*
```

```
               CALL DORMBR( 'Q', 'L', 'N', N, NS, N, WORK( IQRF ), N,
     $                      WORK( ITAUQ ), U, LDU, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
*
*              Call DORMQR to compute Q*(QB*UB).
*              (Workspace in WORK( ITEMP ): need N, prefer N*NB)
*
               CALL DORMQR( 'L', 'N', M, NS, N, A, LDA,
     $                      WORK( ITAU ), U, LDU, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
            END IF
*
*           If needed, compute right singular vectors.
*
            IF( WANTVT) THEN
               J = ITGKZ + N
               DO I = 1, NS
                  CALL DCOPY( N, WORK( J ), 1, VT( I,1 ), LDVT )
                  J = J + N*2
               END DO
*
               Call DORMBR to compute VB**T * PB**T
               (Workspace in WORK( ITEMP ): need N, prefer N*NB)
*
               CALL DORMBR( 'P', 'R', 'T', NS, N, N, WORK( IQRF ), N,
     $                      WORK( ITAUP ), VT, LDVT, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
            END IF
         ELSE
*
*           Path 2 (M at least N, but not much larger)
*           Reduce A to bidiagonal form without QR decomposition
*           A = QB * B * PB**T = QB * ( UB * S * VB**T ) * PB**T
*           U = QB * UB; V**T = VB**T * PB**T
*
*           Bidiagonalize A
*           (Workspace: need 4*N+M, prefer 4*N+(M+N)*NB)
*
            ID = 1
            IE = ID + N
            ITAUQ = IE + N
            ITAUP = ITAUQ + N
            ITEMP = ITAUP + N
            CALL DGEBRD( M, N, A, LDA, WORK( ID ), WORK( IE ),
     $                   WORK( ITAUQ ), WORK( ITAUP ), WORK( ITEMP ),
     $                   LWORK-ITEMP+1, INFO )
*
*           Solve eigenvalue problem TGK*Z=Z*S.
*           (Workspace: need 14*N + 2*N*(N+1))
*
```

```
            ITGKZ = ITEMP
            ITEMP = ITGKZ + N*(N*2+1)
            CALL DBDSVDX( 'U', JOBZ, RNGTGK, N, WORK( ID ), WORK( IE ),
     $                    VL, VU, ILTGK, IUTGK, NS, S, WORK( ITGKZ ),
     $                    N*2, WORK( ITEMP ), IWORK, INFO)
*
*           If needed, compute left singular vectors.
*
            IF( WANTU ) THEN
               J = ITGKZ
               DO I = 1, NS
                  CALL DCOPY( N, WORK( J ), 1, U( 1,I ), 1 )
                  J = J + N*2
               END DO
               CALL DLASET( 'A', M-N, N, ZERO, ZERO, U( N+1,1 ), LDU )
*
*              Call DORMBR to compute QB*UB.
*              (Workspace in WORK( ITEMP ): need N, prefer N*NB)
*
               CALL DORMBR( 'Q', 'L', 'N', M, NS, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, IERR )
            END IF
*
*           If needed, compute right singular vectors.
*
            IF( WANTVT) THEN
               J = ITGKZ + N
               DO I = 1, NS
                  CALL DCOPY( N, WORK( J ), 1, VT( I,1 ), LDVT )
                  J = J + N*2
               END DO
*
*              Call DORMBR to compute VB**T * PB**T
*              (Workspace in WORK( ITEMP ): need N, prefer N*NB)
*
               CALL DORMBR( 'P', 'R', 'T', NS, N, N, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, IERR )
            END IF
         END IF
      ELSE
*
*        A has more columns than rows. If A has sufficiently more
*        columns than rows, first reduce A using the LQ decomposition.
*
         IF( N.GE.MNTHR ) THEN
*
*           Path 1t (N much larger than M):
*           A = L * Q = ( QB * B * PB**T ) * Q
```

```
*                    = ( QB * ( UB * S * VB**T ) * PB**T ) * Q
*          U = QB * UB ; V**T = VB**T * PB**T * Q
*
*          Compute A=L*Q
*          (Workspace: need 2*M, prefer M+M*NB)
*
           ITAU = 1
           ITEMP = ITAU + M
           CALL DGELQF( M, N, A, LDA, WORK( ITAU ), WORK( ITEMP ),
     $                  LWORK-ITEMP+1, INFO )

*          Copy L into WORK and bidiagonalize it:
*          (Workspace in WORK( ITEMP ): need M*M+5*N, prefer M*M+4*M+2*M*NB)
*
           ILQF = ITEMP
           ID = ILQF + M*M
           IE = ID + M
           ITAUQ = IE + M
           ITAUP = ITAUQ + M
           ITEMP = ITAUP + M
           CALL DLACPY( 'L', M, M, A, LDA, WORK( ILQF ), M )
           CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, WORK( ILQF+M ), M )
           CALL DGEBRD( M, M, WORK( ILQF ), M, WORK( ID ), WORK( IE ),
     $                  WORK( ITAUQ ), WORK( ITAUP ), WORK( ITEMP ),
     $                  LWORK-ITEMP+1, INFO )
*
*          Solve eigenvalue problem TGK*Z=Z*S.
*          (Workspace: need 2*M*M+14*M)
*
           ITGKZ = ITEMP
           ITEMP = ITGKZ + M*(M*2+1)
           CALL DBDSVDX( 'U', JOBZ, RNGTGK, M, WORK( ID ), WORK( IE ),
     $                   VL, VU, ILTGK, IUTGK, NS, S, WORK( ITGKZ ),
     $                   M*2, WORK( ITEMP ), IWORK, INFO)
*
*          If needed, compute left singular vectors.
*
           IF( WANTU ) THEN
              J = ITGKZ
              DO I = 1, NS
                 CALL DCOPY( M, WORK( J ), 1, U( 1,I ), 1 )
                 J = J + M*2
              END DO
*
              Call DORMBR to compute QB*UB.
*             (Workspace in WORK( ITEMP ): need M, prefer M*NB)
*
              CALL DORMBR( 'Q', 'L', 'N', M, NS, M, WORK( ILQF ), M,
     $                     WORK( ITAUQ ), U, LDU, WORK( ITEMP ),
     $                     LWORK-ITEMP+1, INFO )
```

```
            END IF
*
*           If needed, compute right singular vectors.
*
            IF( WANTVT) THEN
               J = ITGKZ + M
               DO I = 1, NS
                  CALL DCOPY( M, WORK( J ), 1, VT( I,1 ), LDVT )
                  J = J + M*2
               END DO
               CALL DLASET( 'A', M, N-M, ZERO, ZERO, VT( 1,M+1 ), LDVT )
*
*              Call DORMBR to compute (VB**T)*(PB**T)
*              (Workspace in WORK( ITEMP ): need M, prefer M*NB)
*
               CALL DORMBR( 'P', 'R', 'T', NS, M, M, WORK( ILQF ), M,
     $                      WORK( ITAUP ), VT, LDVT, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
*
*              Call DORMLQ to compute ((VB**T)*(PB**T))*Q.
*              (Workspace in WORK( ITEMP ): need M, prefer M*NB)
*
               CALL DORMLQ( 'R', 'N', NS, N, M, A, LDA,
     $                      WORK( ITAU ), VT, LDVT, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
            END IF
         ELSE
*
*           Path 2t (N greater than M, but not much larger)
*           Reduce to bidiagonal form without LQ decomposition
*           A = QB * B * PB**T = QB * ( UB * S * VB**T ) * PB**T
*           U = QB * UB; V**T = VB**T * PB**T
*
*           Bidiagonalize A
*           (Workspace: need 4*M+N, prefer 4*M+(M+N)*NB)
*
            ID = 1
            IE = ID + M
            ITAUQ = IE + M
            ITAUP = ITAUQ + M
            ITEMP = ITAUP + M
            CALL DGEBRD( M, N, A, LDA, WORK( ID ), WORK( IE ),
     $                   WORK( ITAUQ ), WORK( ITAUP ), WORK( ITEMP ),
     $                   LWORK-ITEMP+1, INFO )
*
*           Solve eigenvalue problem TGK*Z=Z*S.
*           (Workspace: need 2*M*M+14*M)
*
            ITGKZ = ITEMP
            ITEMP = ITGKZ + M*(M*2+1)
```

```
            CALL DBDSVDX( 'L', JOBZ, RNGTGK, M, WORK( ID ), WORK( IE ),
     $                    VL, VU, ILTGK, IUTGK, NS, S, WORK( ITGKZ ),
     $                    M*2, WORK( ITEMP ), IWORK, INFO)
*
*           If needed, compute left singular vectors.
*
            IF( WANTU ) THEN
               J = ITGKZ
               DO I = 1, NS
                  CALL DCOPY( M, WORK( J ), 1, U( 1,I ), 1 )
                  J = J + M*2
               END DO
*
*              Call DORMBR to compute QB*UB.
*              (Workspace in WORK( ITEMP ): need M, prefer M*NB)
*
               CALL DORMBR( 'Q', 'L', 'N', M, NS, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
            END IF
*
*           If needed, compute right singular vectors.
*
            IF( WANTVT) THEN
               J = ITGKZ + M
               DO I = 1, NS
                  CALL DCOPY( M, WORK( J ), 1, VT( I,1 ), LDVT )
                  J = J + M*2
               END DO
               CALL DLASET( 'A', M, N-M, ZERO, ZERO, VT( 1,M+1 ), LDVT )
*
*              Call DORMBR to compute VB**T * PB**T
*              (Workspace in WORK( ITEMP ): need M, prefer M*NB)
*
               CALL DORMBR( 'P', 'R', 'T', NS, N, M, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
            END IF
         END IF
      END IF
*
*     Undo scaling if necessary
*
      IF( ISCL.EQ.1 ) THEN
         IF( ANRM.GT.BIGNUM )
     $      CALL DLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1,
     $                   S, MINMN, INFO )
         IF( ANRM.LT.SMLNUM )
     $      CALL DLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1,
     $                   S, MINMN, INFO )
```

```
      END IF
*
*     Return optimal workspace in WORK(1)
*
      WORK( 1 ) = DBLE( MAXWRK )
*
      RETURN
*
*     End of DGESVDX
*
      END
```

———-

28.4.4 lisp code

— **dgesvdx.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dgesvdx
         (jobu jobvt range m n a lda vl vu il iu ns s u ldu vt ldvt work lwork
          iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (double-float) vu vl)
             (type (array double-float (*)) work vt u s a)
             (type (f2cl-lib:integer4) info lwork ldvt ldu ns iu il lda n m)
             (type (string *) range jobvt jobu))
    (f2cl-lib:with-multi-array-data
        ((jobu character jobu-%data% jobu-%offset%)
         (jobvt character jobvt-%data% jobvt-%offset%)
         (range character range-%data% range-%offset%)
         (a double-float a-%data% a-%offset%)
         (s double-float s-%data% s-%offset%)
         (u double-float u-%data% u-%offset%)
         (vt double-float vt-%data% vt-%offset%)
         (work double-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((dum (make-array 1 :element-type 'double-float)) (abstol 0.0d0)
             (anrm 0.0d0) (bignum 0.0d0) (eps 0.0d0) (smlnum 0.0d0) (i 0)
             (id 0) (ie 0) (ierr 0) (ilqf 0) (iltgk 0) (iqrf 0) (iscl 0)
             (itau 0) (itaup 0) (itauq 0) (itemp 0) (itgkz 0) (iutgk 0) (j 0)
             (maxwrk 0) (minmn 0) (minwrk 0) (mnthr 0) (alls nil) (inds nil)
             (lquery nil) (vals nil) (wantu nil) (wantvt nil)
             (jobz
              (make-array '(1) :element-type 'character :initial-element #\ ))
```

```
     (rngtgk
      (make-array '(1) :element-type 'character :initial-element #\ )))
(declare (type (array double-float (1)) dum)
         (type (double-float) abstol anrm bignum eps smlnum)
         (type (f2cl-lib:integer4) i id ie ierr ilqf iltgk iqrf iscl
          itau itaup itauq itemp itgkz iutgk j maxwrk minmn minwrk
          mnthr)
         (type f2cl-lib:logical alls inds lquery vals wantu wantvt)
         (type (string 1) jobz rngtgk))
(setf ns 0)
(setf info 0)
(setf abstol (* 2 (dlamch "S")))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(setf wantu
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobu "V")
          (declare (ignore var-1))
          (when var-0 (setf jobu var-0))
          ret-val))
(setf wantvt
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvt "V")
          (declare (ignore var-1))
          (when var-0 (setf jobvt var-0))
          ret-val))
(cond
 ((or wantu wantvt) (f2cl-lib:f2cl-set-string jobz "V" (string 1)))
 (t (f2cl-lib:f2cl-set-string jobz "N" (string 1))))
(setf alls
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "A")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf vals
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "V")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf inds
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "I")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf info 0)
(cond
 ((and
```

```
  (not
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame jobu "V")
     (declare (ignore var-1))
     (when var-0 (setf jobu var-0))
     ret-val))
  (not
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame jobu "N")
     (declare (ignore var-1))
     (when var-0 (setf jobu var-0))
     ret-val)))
 (setf info -1))
((and
  (not
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame jobvt "V")
     (declare (ignore var-1))
     (when var-0 (setf jobvt var-0))
     ret-val))
  (not
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame jobvt "N")
     (declare (ignore var-1))
     (when var-0 (setf jobvt var-0))
     ret-val)))
 (setf info -2))
((not (or alls vals inds)) (setf info -3)) ((< m 0) (setf info -4))
((< n 0) (setf info -5)) ((> m lda) (setf info -7))
((> minmn 0)
 (cond
  (vals
   (cond ((< vl zero) (setf info -8)) ((<= vu vl) (setf info -9))))
  (inds
   (cond
    ((or (< il 1)
         (> il
            (max (the f2cl-lib:integer4 1)
                 (the f2cl-lib:integer4 minmn))))
     (setf info -10))
    ((or
      (< iu
         (min (the f2cl-lib:integer4 minmn)
              (the f2cl-lib:integer4 il)))
      (> iu minmn))
     (setf info -11)))))
 (cond
  ((= info 0)
   (cond ((and wantu (< ldu m)) (setf info -15))
         ((and wantvt (< ldvt minmn)) (setf info -16)))))))
```

```
(cond
 ((= info 0) (setf minwrk 1) (setf maxwrk 1)
  (cond
   ((> minmn 0)
    (cond
     ((>= m n)
      (setf mnthr
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 6 "DGESVD" (f2cl-lib:f2cl-// jobu jobvt) m n
                   0 0)
                (declare (ignore var-0 var-1 var-2 var-5 var-6))
                (when var-3 (setf m var-3))
                (when var-4 (setf n var-4))
                ret-val))
      (cond
       ((>= m mnthr)
        (setf maxwrk
                (f2cl-lib:int-add
                 (f2cl-lib:int-mul n
                                   (f2cl-lib:int-add
                                    (f2cl-lib:int-mul n 2) 16))
                 (f2cl-lib:int-mul n
                                   (multiple-value-bind
                                       (ret-val var-0 var-1 var-2 var-3
                                        var-4 var-5 var-6)
                                       (ilaenv 1 "DGEQRF" " " m n -1
                                        -1)
                                     (declare
                                      (ignore var-0 var-1 var-2 var-5
                                       var-6))
                                     (when var-3 (setf m var-3))
                                     (when var-4 (setf n var-4))
                                     ret-val))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add
                           (f2cl-lib:int-mul n
                                             (f2cl-lib:int-add
                                              (f2cl-lib:int-mul n 2)
                                              20))
                           (f2cl-lib:int-mul 2 n
                                             (multiple-value-bind
                                                 (ret-val var-0 var-1
                                                  var-2 var-3 var-4
                                                  var-5 var-6)
                                                 (ilaenv 1 "DGEBRD" " "
                                                  n n -1 -1)
                                               (declare
```

```
                                                (ignore var-0 var-1
                                                 var-2 var-5 var-6))
                                               (when var-3
                                                 (setf n var-3))
                                               (when var-4
                                                 (setf n var-4))
                                               ret-val))))))
    (setf minwrk
            (f2cl-lib:int-mul n
                              (f2cl-lib:int-add
                               (f2cl-lib:int-mul n 2) 21))))
   (t
    (setf maxwrk
            (f2cl-lib:int-add
             (f2cl-lib:int-mul n
                               (f2cl-lib:int-add
                                (f2cl-lib:int-mul n 2) 19))
             (f2cl-lib:int-mul (f2cl-lib:int-add m n)
                               (multiple-value-bind
                                   (ret-val var-0 var-1 var-2 var-3
                                    var-4 var-5 var-6)
                                   (ilaenv 1 "DGEBRD" " " m n -1
                                    -1)
                                 (declare
                                  (ignore var-0 var-1 var-2 var-5
                                   var-6))
                                 (when var-3 (setf m var-3))
                                 (when var-4 (setf n var-4))
                                 ret-val))))
    (setf minwrk
            (f2cl-lib:int-add
             (f2cl-lib:int-mul n
                               (f2cl-lib:int-add
                                (f2cl-lib:int-mul n 2) 20))
             m)))))
  (t
   (setf mnthr
           (multiple-value-bind
               (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
               (ilaenv 6 "DGESVD" (f2cl-lib:f2cl-// jobu jobvt) m n
                0 0)
             (declare (ignore var-0 var-1 var-2 var-5 var-6))
             (when var-3 (setf m var-3))
             (when var-4 (setf n var-4))
             ret-val))
   (cond
    ((>= n mnthr)
     (setf maxwrk
             (f2cl-lib:int-add
              (f2cl-lib:int-mul m
```

```
                                (f2cl-lib:int-add
                                 (f2cl-lib:int-mul m 2) 16))
                 (f2cl-lib:int-mul m
                                (multiple-value-bind
                                    (ret-val var-0 var-1 var-2 var-3
                                     var-4 var-5 var-6)
                                    (ilaenv 1 "DGELQF" " " m n -1
                                     -1)
                                  (declare
                                   (ignore var-0 var-1 var-2 var-5
                                    var-6))
                                  (when var-3 (setf m var-3))
                                  (when var-4 (setf n var-4))
                                  ret-val))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add
                           (f2cl-lib:int-mul m
                                             (f2cl-lib:int-add
                                              (f2cl-lib:int-mul m 2)
                                              20))
                           (f2cl-lib:int-mul 2 m
                                             (multiple-value-bind
                                                 (ret-val var-0 var-1
                                                  var-2 var-3 var-4
                                                  var-5 var-6)
                                                 (ilaenv 1 "DGEBRD" " "
                                                  m m -1 -1)
                                               (declare
                                                (ignore var-0 var-1
                                                 var-2 var-5 var-6))
                                               (when var-3
                                                 (setf m var-3))
                                               (when var-4
                                                 (setf m var-4))
                                               ret-val))))))
        (setf minwrk
                (f2cl-lib:int-mul m
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-mul m 2) 21))))
       (t
        (setf maxwrk
                (f2cl-lib:int-add
                 (f2cl-lib:int-mul m
                                   (f2cl-lib:int-add
                                    (f2cl-lib:int-mul m 2) 19))
                 (f2cl-lib:int-mul (f2cl-lib:int-add m n)
                                   (multiple-value-bind
                                       (ret-val var-0 var-1 var-2 var-3
```

```
                                        var-4 var-5 var-6)
                                       (ilaenv 1 "DGEBRD" " " m n -1
                                        -1)
                                     (declare
                                      (ignore var-0 var-1 var-2 var-5
                                       var-6))
                                     (when var-3 (setf m var-3))
                                     (when var-4 (setf n var-4))
                                     ret-val))))
        (setf minwrk
                (f2cl-lib:int-add
                 (f2cl-lib:int-mul m
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-mul m 2) 20))
                 n))))))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4 minwrk)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:dble maxwrk))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -19)))))
(cond
 ((/= info 0) (xerbla "DGESVDX" (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(cond ((or (= m 0) (= n 0)) (go end_label)))
(cond
 (alls (f2cl-lib:f2cl-set-string rngtgk "I" (string 1)) (setf iltgk 1)
  (setf iutgk
          (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n))))
 (inds (f2cl-lib:f2cl-set-string rngtgk "I" (string 1)) (setf iltgk il)
  (setf iutgk iu))
 (t (f2cl-lib:f2cl-set-string rngtgk "V" (string 1)) (setf iltgk 0)
  (setf iutgk 0)))
(setf eps (dlamch "P"))
(setf smlnum (/ (f2cl-lib:fsqrt (dlamch "S")) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "M" m n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iscl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf iscl 1)
  (multiple-value-bind
```

```
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 anrm smlnum m n a lda info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf smlnum var-4))
      (when var-5 (setf m var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf info var-9))))
   ((> anrm bignum) (setf iscl 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 anrm bignum m n a lda info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf bignum var-4))
      (when var-5 (setf m var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf info var-9)))))
  (cond
   ((>= m n)
    (cond
     ((>= m mnthr) (setf itau 1) (setf itemp (f2cl-lib:int-add itau n))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (dgeqrf m n a lda
           (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                                 work-%offset%)
           (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                                 work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
        (declare (ignore var-2 var-4 var-5 var-6))
        (setf m var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf info var-7))
      (setf iqrf itemp)
      (setf id (f2cl-lib:int-add iqrf (f2cl-lib:int-mul n n)))
      (setf ie (f2cl-lib:int-add id n))
      (setf itauq (f2cl-lib:int-add ie n))
      (setf itaup (f2cl-lib:int-add itauq n))
      (setf itemp (f2cl-lib:int-add itaup n))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlacpy "U" n n a lda
           (f2cl-lib:array-slice work-%data% double-float (iqrf) ((1 *))
                                 work-%offset%)
           n)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf n var-1))
```

```
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf n var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1) zero
     zero
     (f2cl-lib:array-slice work-%data% double-float ((+ iqrf 1))
                           ((1 *)) work-%offset%)
     n)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf n var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgebrd n n
     (f2cl-lib:array-slice work-%data% double-float (iqrf) ((1 *))
                           work-%offset%)
     n
     (f2cl-lib:array-slice work-%data% double-float (id) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf n var-3)
  (setf info var-10))
(setf itgkz itemp)
(setf itemp
        (f2cl-lib:int-add itgkz
                          (f2cl-lib:int-mul n
                                            (f2cl-lib:int-add
                                             (f2cl-lib:int-mul n 2)
                                             1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (dbdsvdx "U" jobz rngtgk n
     (f2cl-lib:array-slice work-%data% double-float (id) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
```

```
                              work-%offset%)
        vl vu iltgk iutgk ns s
        (f2cl-lib:array-slice work-%data% double-float (itgkz) ((1 *))
                              work-%offset%)
        (f2cl-lib:int-mul n 2)
        (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                              work-%offset%)
        iwork info)
    (declare
     (ignore var-0 var-4 var-5 var-6 var-7 var-8 var-9 var-11 var-12
      var-13 var-14 var-15))
    (setf jobz var-1)
    (setf rngtgk var-2)
    (setf n var-3)
    (setf ns var-10)
    (setf info var-16))
  (cond
   (wantu (setf j itgkz)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i ns) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dcopy n
             (f2cl-lib:array-slice work-%data% double-float (j)
                                   ((1 *)) work-%offset%)
             1
             (f2cl-lib:array-slice u-%data% double-float (1 i)
                                   ((1 ldu) (1 *)) u-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf n var-0)))
        (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul n 2)))
       label100000))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "A" (f2cl-lib:int-sub m n) n zero zero
         (f2cl-lib:array-slice u-%data% double-float ((+ n 1) 1)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldu var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (dormbr "Q" "L" "N" n ns n
         (f2cl-lib:array-slice work-%data% double-float (iqrf)
                               ((1 *)) work-%offset%)
         n
```

```
          (f2cl-lib:array-slice work-%data% double-float (itauq)
                                ((1 *)) work-%offset%)
          u ldu
          (f2cl-lib:array-slice work-%data% double-float (itemp)
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
        (declare
         (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
        (when var-3 (setf n var-3))
        (when var-4 (setf ns var-4))
        (when var-5 (setf n var-5))
        (when var-7 (setf n var-7))
        (when var-10 (setf ldu var-10))
        (when var-13 (setf info var-13)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (dormqr "L" "N" m ns n a lda
           (f2cl-lib:array-slice work-%data% double-float (itau)
                                 ((1 *)) work-%offset%)
           u ldu
           (f2cl-lib:array-slice work-%data% double-float (itemp)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf ns var-3))
        (when var-4 (setf n var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldu var-9))
        (when var-12 (setf info var-12)))))
    (cond
     (wantvt (setf j (f2cl-lib:int-add itgkz n))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i ns) nil)
        (tagbody
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dcopy n
               (f2cl-lib:array-slice work-%data% double-float (j)
                                     ((1 *)) work-%offset%)
               1
               (f2cl-lib:array-slice vt-%data% double-float (i 1)
                                     ((1 ldvt) (1 *)) vt-%offset%)
               ldvt)
            (declare (ignore var-1 var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-4 (setf ldvt var-4)))
          (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul n 2)))
         label100001))
      (multiple-value-bind
```

```
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13)
          (dormbr "P" "R" "T" ns n n
           (f2cl-lib:array-slice work-%data% double-float (iqrf)
                                 ((1 *)) work-%offset%)
           n
           (f2cl-lib:array-slice work-%data% double-float (itaup)
                                 ((1 *)) work-%offset%)
           vt ldvt
           (f2cl-lib:array-slice work-%data% double-float (itemp)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
        (declare
         (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
        (when var-3 (setf ns var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf n var-5))
        (when var-7 (setf n var-7))
        (when var-10 (setf ldvt var-10))
        (when var-13 (setf info var-13))))))
    (t (setf id 1) (setf ie (f2cl-lib:int-add id n))
     (setf itauq (f2cl-lib:int-add ie n))
     (setf itaup (f2cl-lib:int-add itauq n))
     (setf itemp (f2cl-lib:int-add itaup n))
     (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (dgebrd m n a lda
         (f2cl-lib:array-slice work-%data% double-float (id) ((1 *))
                               work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                               work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                               work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                               work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                               work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
      (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
      (setf m var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf info var-10))
     (setf itgkz itemp)
     (setf itemp
             (f2cl-lib:int-add itgkz
                               (f2cl-lib:int-mul n
                                                 (f2cl-lib:int-add
                                                  (f2cl-lib:int-mul n 2)
```

```
                                                1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (dbdsvdx "U" jobz rngtgk n
     (f2cl-lib:array-slice work-%data% double-float (id) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     vl vu iltgk iutgk ns s
     (f2cl-lib:array-slice work-%data% double-float (itgkz) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-mul n 2)
     (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                           work-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-8 var-9 var-11 var-12
    var-13 var-14 var-15))
  (setf jobz var-1)
  (setf rngtgk var-2)
  (setf n var-3)
  (setf ns var-10)
  (setf info var-16))
(cond
 (wantu (setf j itgkz)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy n
           (f2cl-lib:array-slice work-%data% double-float (j)
                                 ((1 *)) work-%offset%)
           1
           (f2cl-lib:array-slice u-%data% double-float (1 i)
                                 ((1 ldu) (1 *)) u-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf n var-0)))
      (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul n 2)))
     label100002))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" (f2cl-lib:int-sub m n) n zero zero
       (f2cl-lib:array-slice u-%data% double-float ((+ n 1) 1)
                             ((1 ldu) (1 *)) u-%offset%)
       ldu)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf n var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
```

```
      (when var-6 (setf ldu var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (dormbr "Q" "L" "N" m ns n a lda
         (f2cl-lib:array-slice work-%data% double-float (itauq)
                               ((1 *)) work-%offset%)
         u ldu
         (f2cl-lib:array-slice work-%data% double-float (itemp)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) ierr)
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
      (when var-3 (setf m var-3))
      (when var-4 (setf ns var-4))
      (when var-5 (setf n var-5))
      (when var-7 (setf lda var-7))
      (when var-10 (setf ldu var-10))
      (when var-13 (setf ierr var-13)))))
  (cond
   (wantvt (setf j (f2cl-lib:int-add itgkz n))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i ns) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dcopy n
             (f2cl-lib:array-slice work-%data% double-float (j)
                                   ((1 *)) work-%offset%)
             1
             (f2cl-lib:array-slice vt-%data% double-float (i 1)
                                   ((1 ldvt) (1 *)) vt-%offset%)
             ldvt)
          (declare (ignore var-1 var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-4 (setf ldvt var-4)))
        (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul n 2)))
       label100003))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (dormbr "P" "R" "T" ns n n a lda
         (f2cl-lib:array-slice work-%data% double-float (itaup)
                               ((1 *)) work-%offset%)
         vt ldvt
         (f2cl-lib:array-slice work-%data% double-float (itemp)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) ierr)
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
      (when var-3 (setf ns var-3))
```

```
            (when var-4 (setf n var-4))
            (when var-5 (setf n var-5))
            (when var-7 (setf lda var-7))
            (when var-10 (setf ldvt var-10))
            (when var-13 (setf ierr var-13))))))))
        (t
         (cond
          ((>= n mnthr) (setf itau 1) (setf itemp (f2cl-lib:int-add itau m))
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
               (dgelqf m n a lda
                (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                                      work-%offset%)
                (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                                      work-%offset%)
                (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
             (declare (ignore var-2 var-4 var-5 var-6))
             (setf m var-0)
             (setf n var-1)
             (setf lda var-3)
             (setf info var-7))
           (setf ilqf itemp)
           (setf id (f2cl-lib:int-add ilqf (f2cl-lib:int-mul m m)))
           (setf ie (f2cl-lib:int-add id m))
           (setf itauq (f2cl-lib:int-add ie m))
           (setf itaup (f2cl-lib:int-add itauq m))
           (setf itemp (f2cl-lib:int-add itaup m))
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
               (dlacpy "L" m m a lda
                (f2cl-lib:array-slice work-%data% double-float (ilqf) ((1 *))
                                      work-%offset%)
                m)
             (declare (ignore var-0 var-3 var-5))
             (when var-1 (setf m var-1))
             (when var-2 (setf m var-2))
             (when var-4 (setf lda var-4))
             (when var-6 (setf m var-6)))
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
               (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1) zero
                zero
                (f2cl-lib:array-slice work-%data% double-float ((+ ilqf m))
                                      ((1 *)) work-%offset%)
                m)
             (declare (ignore var-0 var-1 var-2 var-5))
             (when var-3 (setf zero var-3))
             (when var-4 (setf zero var-4))
             (when var-6 (setf m var-6)))
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                var-10)
```

```
    (dgebrd m m
     (f2cl-lib:array-slice work-%data% double-float (ilqf) ((1 *))
                           work-%offset%)
     m
     (f2cl-lib:array-slice work-%data% double-float (id) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf m var-3)
  (setf info var-10))
(setf itgkz itemp)
(setf itemp
        (f2cl-lib:int-add itgkz
                          (f2cl-lib:int-mul m
                                            (f2cl-lib:int-add
                                             (f2cl-lib:int-mul m 2)
                                             1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (dbdsvdx "U" jobz rngtgk m
     (f2cl-lib:array-slice work-%data% double-float (id) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     vl vu iltgk iutgk ns s
     (f2cl-lib:array-slice work-%data% double-float (itgkz) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-mul m 2)
     (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                           work-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-8 var-9 var-11 var-12
    var-13 var-14 var-15))
  (setf jobz var-1)
  (setf rngtgk var-2)
  (setf m var-3)
  (setf ns var-10)
  (setf info var-16))
```

```
(cond
 (wantu (setf j itgkz)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy m
           (f2cl-lib:array-slice work-%data% double-float (j)
                                 ((1 *)) work-%offset%)
           1
           (f2cl-lib:array-slice u-%data% double-float (1 i)
                                 ((1 ldu) (1 *)) u-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf m var-0)))
      (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul m 2)))
     label100004))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dormbr "Q" "L" "N" m ns m
       (f2cl-lib:array-slice work-%data% double-float (ilqf)
                             ((1 *)) work-%offset%)
       m
       (f2cl-lib:array-slice work-%data% double-float (itauq)
                             ((1 *)) work-%offset%)
       u ldu
       (f2cl-lib:array-slice work-%data% double-float (itemp)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf ns var-4))
    (when var-5 (setf m var-5))
    (when var-7 (setf m var-7))
    (when var-10 (setf ldu var-10))
    (when var-13 (setf info var-13)))))
(cond
 (wantvt (setf j (f2cl-lib:int-add itgkz m))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy m
           (f2cl-lib:array-slice work-%data% double-float (j)
                                 ((1 *)) work-%offset%)
           1
           (f2cl-lib:array-slice vt-%data% double-float (i 1)
                                 ((1 ldvt) (1 *)) vt-%offset%)
```

```
          ldvt)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-4 (setf ldvt var-4)))
      (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul m 2)))
    label100005))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "A" m (f2cl-lib:int-sub n m) zero zero
     (f2cl-lib:array-slice vt-%data% double-float
                           (1 (f2cl-lib:int-add m 1))
                           ((1 ldvt) (1 *)) vt-%offset%)
     ldvt)
  (declare (ignore var-0 var-2 var-5))
  (when var-1 (setf m var-1))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "P" "R" "T" ns m m
     (f2cl-lib:array-slice work-%data% double-float (ilqf)
                           ((1 *)) work-%offset%)
     m
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (itemp)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf ns var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf m var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dormlq "R" "N" ns n m a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (itemp)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf ns var-2))
```

```
        (when var-3 (setf n var-3))
        (when var-4 (setf m var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldvt var-9))
        (when var-12 (setf info var-12))))))
    (t (setf id 1) (setf ie (f2cl-lib:int-add id m))
     (setf itauq (f2cl-lib:int-add ie m))
     (setf itaup (f2cl-lib:int-add itauq m))
     (setf itemp (f2cl-lib:int-add itaup m))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (dgebrd m n a lda
          (f2cl-lib:array-slice work-%data% double-float (id) ((1 *))
                                work-%offset%)
          (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                                work-%offset%)
          (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                                work-%offset%)
          (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                                work-%offset%)
          (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                                work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
       (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
       (setf m var-0)
       (setf n var-1)
       (setf lda var-3)
       (setf info var-10))
     (setf itgkz itemp)
     (setf itemp
             (f2cl-lib:int-add itgkz
                               (f2cl-lib:int-mul m
                                                 (f2cl-lib:int-add
                                                  (f2cl-lib:int-mul m 2)
                                                  1))))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12 var-13 var-14 var-15 var-16)
         (dbdsvdx "L" jobz rngtgk m
          (f2cl-lib:array-slice work-%data% double-float (id) ((1 *))
                                work-%offset%)
          (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                                work-%offset%)
          vl vu iltgk iutgk ns s
          (f2cl-lib:array-slice work-%data% double-float (itgkz) ((1 *))
                                work-%offset%)
          (f2cl-lib:int-mul m 2)
          (f2cl-lib:array-slice work-%data% double-float (itemp) ((1 *))
                                work-%offset%)
```

```
      iwork info)
   (declare
    (ignore var-0 var-4 var-5 var-6 var-7 var-8 var-9 var-11 var-12
     var-13 var-14 var-15))
   (setf jobz var-1)
   (setf rngtgk var-2)
   (setf m var-3)
   (setf ns var-10)
   (setf info var-16))
 (cond
  (wantu (setf j itgkz)
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i ns) nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (dcopy m
            (f2cl-lib:array-slice work-%data% double-float (j)
                                  ((1 *)) work-%offset%)
            1
            (f2cl-lib:array-slice u-%data% double-float (1 i)
                                  ((1 ldu) (1 *)) u-%offset%)
            1)
         (declare (ignore var-1 var-2 var-3 var-4))
         (when var-0 (setf m var-0)))
       (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul m 2)))
      label100006))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13)
       (dormbr "Q" "L" "N" m ns n a lda
        (f2cl-lib:array-slice work-%data% double-float (itauq)
                              ((1 *)) work-%offset%)
        u ldu
        (f2cl-lib:array-slice work-%data% double-float (itemp)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
     (declare
      (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
     (when var-3 (setf m var-3))
     (when var-4 (setf ns var-4))
     (when var-5 (setf n var-5))
     (when var-7 (setf lda var-7))
     (when var-10 (setf ldu var-10))
     (when var-13 (setf info var-13)))))
 (cond
  (wantvt (setf j (f2cl-lib:int-add itgkz m))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i ns) nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
                (dcopy m
                 (f2cl-lib:array-slice work-%data% double-float (j)
                                       ((1 *)) work-%offset%)
                 1
                 (f2cl-lib:array-slice vt-%data% double-float (i 1)
                                       ((1 ldvt) (1 *)) vt-%offset%)
                 ldvt)
               (declare (ignore var-1 var-2 var-3))
               (when var-0 (setf m var-0))
               (when var-4 (setf ldvt var-4)))
             (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul m 2)))
            label100007))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dlaset "A" m (f2cl-lib:int-sub n m) zero zero
             (f2cl-lib:array-slice vt-%data% double-float
                                   (1 (f2cl-lib:int-add m 1))
                                   ((1 ldvt) (1 *)) vt-%offset%)
             ldvt)
          (declare (ignore var-0 var-2 var-5))
          (when var-1 (setf m var-1))
          (when var-3 (setf zero var-3))
          (when var-4 (setf zero var-4))
          (when var-6 (setf ldvt var-6)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11 var-12 var-13)
            (dormbr "P" "R" "T" ns n m a lda
             (f2cl-lib:array-slice work-%data% double-float (itaup)
                                   ((1 *)) work-%offset%)
             vt ldvt
             (f2cl-lib:array-slice work-%data% double-float (itemp)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
          (declare
           (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
          (when var-3 (setf ns var-3))
          (when var-4 (setf n var-4))
          (when var-5 (setf m var-5))
          (when var-7 (setf lda var-7))
          (when var-10 (setf ldvt var-10))
          (when var-13 (setf info var-13)))))))))
  (cond
   ((= iscl 1)
    (if (> anrm bignum)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
            (dlascl "G" 0 0 bignum anrm minmn 1 s minmn info)
          (declare (ignore var-0 var-1 var-2 var-6 var-7))
          (when var-3 (setf bignum var-3))
          (when var-4 (setf anrm var-4))
```

```
                (when var-5 (setf minmn var-5))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf info var-9))))
          (if (< anrm smlnum)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (dlascl "G" 0 0 smlnum anrm minmn 1 s minmn info)
                (declare (ignore var-0 var-1 var-2 var-6 var-7))
                (when var-3 (setf smlnum var-3))
                (when var-4 (setf anrm var-4))
                (when var-5 (setf minmn var-5))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf info var-9))))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (f2cl-lib:dble maxwrk))
        (go end_label)
       end_label
        (return
         (values jobu jobvt range m n nil lda nil nil nil nil ns nil nil ldu
                 nil ldvt nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgesvdx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::jobu
                                              fortran-to-lisp::jobvt
                                              fortran-to-lisp::range
```

```
  fortran-to-lisp::m
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil nil
  nil fortran-to-lisp::ns nil nil
  fortran-to-lisp::ldu nil
  fortran-to-lisp::ldvt nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::dgelqf
  fortran-to-lisp::dbdsvdx
  fortran-to-lisp::dgebrd
  fortran-to-lisp::dgeqrf))))
```

———-

Chapter 29

LAPACK - General Matrices, Singular Value, Real

29.1 sgejsv

29.1.1 Axiom unit tests

— sgejsv.input —

```
)set break resume
)sys rm -f sgejsv.output
)spool sgejsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

29.1.2 Axiom help page

— sgejsv.help —

```
======================================================================
sgejsv examples
======================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgejsv (JOBA, JOBU, JOBV, JOBR, JOBT, JOBP, M, N, A, LDA,
           SVA, U, LDU, V, LDV, WORK, LWORK, IWORK, INFO)
           SGEJSV

Function/Subroutine Documentation
   subroutine sgejsv (character*1 JOBA, character*1 JOBU, character*1 JOBV,
       character*1 JOBR, character*1 JOBT, character*1 JOBP, integer M,
       integer N, real, dimension( lda, * ) A, integer LDA, real, dimension( n
       ) SVA, real, dimension( ldu, * ) U, integer LDU, real, dimension( ldv,
       * ) V, integer LDV, real, dimension( lwork ) WORK, integer LWORK,
       integer, dimension( * ) IWORK, integer INFO)
       SGEJSV

Purpose:
   SGEJSV computes the singular value decomposition (SVD) of a real M-by-N
   matrix [A], where M >= N. The SVD of [A] is written as

   [A] = [U] * [SIGMA] * [V]^t,

   where [SIGMA] is an N-by-N (M-by-N) matrix which is zero except for its N
   diagonal elements, [U] is an M-by-N (or M-by-M) orthonormal matrix, and
   [V] is an N-by-N orthogonal matrix. The diagonal elements of [SIGMA] are
   the singular values of [A]. The columns of [U] and [V] are the left and
   the right singular vectors of [A], respectively. The matrices [U] and [V]
   are computed and stored in the arrays U and V, respectively. The diagonal
   of [SIGMA] is computed and stored in the array SVA.
   SGEJSV can sometimes compute tiny singular values and their singular
   vectors much more accurately than other SVD routines,
   see below under Further Details.

Parameters:

 JOBA
   JOBA is CHARACTER*1
   Specifies the level of accuracy:
   = 'C': This option works well (high relative accuracy) if A = B * D,
          with well-conditioned B and arbitrary diagonal matrix D.
          The accuracy cannot be spoiled by COLUMN scaling. The
          accuracy of the computed output depends on the condition of
          B, and the procedure aims at the best theoretical accuracy.
          The relative error max_{i=1:N}|d sigma_i| / sigma_i is
          bounded by f(M,N)*epsilon* cond(B), independent of D.
          The input matrix is preprocessed with the QRF with column
```

```
          pivoting. This initial preprocessing and preconditioning by
          a rank revealing QR factorization is common for all values of
          JOBA. Additional actions are specified as follows:
  = 'E': Computation as with 'C' with an additional estimate of the
          condition number of B. It provides a realistic error bound.
  = 'F': If A = D1 * C * D2 with ill-conditioned diagonal scalings
          D1, D2, and well-conditioned matrix C, this option gives
          higher accuracy than the 'C' option. If the structure of the
          input matrix is not known, and relative accuracy is
          desirable, then this option is advisable. The input matrix A
          is preprocessed with QR factorization with FULL (row and
          column) pivoting.
  = 'G'  Computation as with 'F' with an additional estimate of the
          condition number of B, where A=D*B. If A has heavily weighted
          rows, then using this condition number gives too pessimistic
          error bound.
  = 'A': Small singular values are the noise and the matrix is treated
          as numerically rank defficient. The error in the computed
          singular values is bounded by f(m,n)*epsilon*||A||.
          The computed SVD A = U * S * V^t restores A up to
          f(m,n)*epsilon*||A||.
          This gives the procedure the licence to discard (set to zero)
          all singular values below N*epsilon*||A||.
  = 'R': Similar as in 'A'. Rank revealing property of the initial
          QR factorization is used do reveal (using triangular factor)
          a gap sigma_{r+1} < epsilon * sigma_r in which case the
          numerical RANK is declared to be r. The SVD is computed with
          absolute error bounds, but more accurately than with 'A'.

JOBU
  JOBU is CHARACTER*1
  Specifies whether to compute the columns of U:
  = 'U': N columns of U are returned in the array U.
  = 'F': full set of M left sing. vectors is returned in the array U.
  = 'W': U may be used as workspace of length M*N. See the description of U.
  = 'N': U is not computed.

JOBV
  JOBV is CHARACTER*1
  Specifies whether to compute the matrix V:
  = 'V': N columns of V are returned in the array V; Jacobi rotations
          are not explicitly accumulated.
  = 'J': N columns of V are returned in the array V, but they are
          computed as the product of Jacobi rotations. This option is
          allowed only if JOBU .NE. 'N', i.e. in computing the full SVD.
  = 'W': V may be used as workspace of length N*N. See the description of V.
  = 'N': V is not computed.

JOBR
  JOBR is CHARACTER*1
```

```
  Specifies the RANGE for the singular values. Issues the licence to
  set to zero small positive singular values if they are outside
  specified range. If A .NE. 0 is scaled so that the largest singular
  value of c*A is around SQRT(BIG), BIG=SLAMCH('O'), then JOBR issues
  the licence to kill columns of A whose norm in c*A is less than
  SQRT(SFMIN) (for JOBR.EQ.'R'), or less than SMALL=SFMIN/EPSLN,
  where SFMIN=SLAMCH('S'), EPSLN=SLAMCH('E').
  = 'N': Do not kill small columns of c*A. This option assumes that
         BLAS and QR factorizations and triangular solvers are
         implemented to work in that range. If the condition of A
         is greater than BIG, use SGESVJ.
  = 'R': RESTRICTED range for sigma(c*A) is [SQRT(SFMIN), SQRT(BIG)]
         (roughly, as described above). This option is recommended.
                           ===========================
  For computing the singular values in the FULL range [SFMIN,BIG]
  use SGESVJ.

JOBT
  JOBT is CHARACTER*1
  If the matrix is square then the procedure may determine to use
  transposed A if A^t seems to be better with respect to convergence.
  If the matrix is not square, JOBT is ignored. This is subject to
  changes in the future.
  The decision is based on two values of entropy over the adjoint
  orbit of A^t * A. See the descriptions of WORK(6) and WORK(7).
  = 'T': transpose if entropy test indicates possibly faster
         convergence of Jacobi process if A^t is taken as input. If A is
         replaced with A^t, then the row pivoting is included automatically.
  = 'N': do not speculate.
  This option can be used to compute only the singular values, or the
  full SVD (U, SIGMA and V). For only one set of singular vectors
  (U or V), the caller should provide both U and V, as one of the
  matrices is used as workspace if the matrix A is transposed.
  The implementer can easily remove this constraint and make the
  code more complicated. See the descriptions of U and V.

JOBP
  JOBP is CHARACTER*1
  Issues the licence to introduce structured perturbations to drown
  denormalized numbers. This licence should be active if the
  denormals are poorly implemented, causing slow computation,
  especially in cases of fast convergence (!). For details see [1,2].
  For the sake of simplicity, this perturbations are included only
  when the full SVD or only the singular values are requested. The
  implementer/user can easily add the perturbation for the cases of
  computing one set of singular vectors.
  = 'P': introduce perturbation
  = 'N': do not perturb

M
```

```
  M is INTEGER
  The number of rows of the input matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the input matrix A. M >= N >= 0.

A
  A is REAL array, dimension (LDA,N)
  On entry, the M-by-N matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

SVA
  SVA is REAL array, dimension (N)
  On exit,
  - For WORK(1)/WORK(2) = ONE: The singular values of A. During the
    computation SVA contains Euclidean column norms of the
    iterated matrices in the array A.
  - For WORK(1) .NE. WORK(2): The singular values of A are
    (WORK(1)/WORK(2)) * SVA(1:N). This factored form is used if
    sigma_max(A) overflows or if small singular values have been
    saved from underflow by scaling the input matrix A.
  - If JOBR='R' then some of the singular values may be returned
    as exact zeros obtained by "set to zero" because they are
    below the numerical rank threshold or are denormalized numbers.

U
  U is REAL array, dimension ( LDU, N )
  If JOBU = 'U', then U contains on exit the M-by-N matrix of
       the left singular vectors.
  If JOBU = 'F', then U contains on exit the M-by-M matrix of
       the left singular vectors, including an ONB
       of the orthogonal complement of the Range(A).
  If JOBU = 'W'  .AND. (JOBV.EQ.'V' .AND. JOBT.EQ.'T' .AND. M.EQ.N),
       then U is used as workspace if the procedure
       replaces A with A^t. In that case, [V] is computed
       in U as left singular vectors of A^t and then
       copied back to the V array. This 'W' option is just
       a reminder to the caller that in this case U is
       reserved as workspace of length N*N.
  If JOBU = 'N'  U is not referenced.

LDU
  LDU is INTEGER
  The leading dimension of the array U,  LDU >= 1.
  IF  JOBU = 'U' or 'F' or 'W',  then LDU >= M.
```

```
V
  V is REAL array, dimension ( LDV, N )
  If JOBV = 'V', 'J' then V contains on exit the N-by-N matrix of
       the right singular vectors;
  If JOBV = 'W', AND (JOBU.EQ.'U' AND JOBT.EQ.'T' AND M.EQ.N),
       then V is used as workspace if the pprocedure
       replaces A with A^t. In that case, [U] is computed
       in V as right singular vectors of A^t and then
       copied back to the U array. This 'W' option is just
       a reminder to the caller that in this case V is
       reserved as workspace of length N*N.
  If JOBV = 'N'  V is not referenced.

LDV
  LDV is INTEGER
  The leading dimension of the array V,  LDV >= 1.
  If JOBV = 'V' or 'J' or 'W', then LDV >= N.

WORK
  WORK is REAL array, dimension at least LWORK.
  On exit,
  WORK(1) = SCALE = WORK(2) / WORK(1) is the scaling factor such
      that SCALE*SVA(1:N) are the computed singular values
      of A. (See the description of SVA().)
  WORK(2) = See the description of WORK(1).
  WORK(3) = SCONDA is an estimate for the condition number of
      column equilibrated A. (If JOBA .EQ. 'E' or 'G')
      SCONDA is an estimate of SQRT(||(R^t * R)^(-1)||_1).
      It is computed using SPOCON. It holds
      N^(-1/4) * SCONDA <= ||R^(-1)||_2 <= N^(1/4) * SCONDA
      where R is the triangular factor from the QRF of A.
      However, if R is truncated and the numerical rank is
      determined to be strictly smaller than N, SCONDA is
      returned as -1, thus indicating that the smallest
      singular values might be lost.

  If full SVD is needed, the following two condition numbers are
  useful for the analysis of the algorithm. They are provied for
  a developer/implementer who is familiar with the details of
  the method.

  WORK(4) = an estimate of the scaled condition number of the
      triangular factor in the first QR factorization.
  WORK(5) = an estimate of the scaled condition number of the
      triangular factor in the second QR factorization.

  The following two parameters are computed if JOBT .EQ. 'T'.
  They are provided for a developer/implementer who is familiar
  with the details of the method.
```

```
  WORK(6) = the entropy of A^t*A :: this is the Shannon entropy
      of diag(A^t*A) / Trace(A^t*A) taken as point in the
      probability simplex.
  WORK(7) = the entropy of A*A^t.

LWORK
  LWORK is INTEGER
  Length of WORK to confirm proper allocation of work space.
  LWORK depends on the job:

  If only SIGMA is needed ( JOBU.EQ.'N', JOBV.EQ.'N' ) and
  -> .. no scaled condition estimate required (JOBE.EQ.'N'):
        LWORK >= max(2*M+N,4*N+1,7). This is the minimal requirement.
    ->> For optimal performance (blocked code) the optimal value
        is LWORK >= max(2*M+N,3*N+(N+1)*NB,7). Here NB is the optimal
        block size for DGEQP3 and DGEQRF.
        In general, optimal LWORK is computed as
        LWORK >= max(2*M+N,N+LWORK(DGEQP3),N+LWORK(DGEQRF), 7).
  -> .. an estimate of the scaled condition number of A is
        required (JOBA='E', 'G'). In this case, LWORK is the maximum
        of the above and N*N+4*N, i.e. LWORK >= max(2*M+N,N*N+4*N,7).
    ->> For optimal performance (blocked code) the optimal value
        is LWORK >= max(2*M+N,3*N+(N+1)*NB, N*N+4*N, 7).
        In general, the optimal length LWORK is computed as
         LWORK >= max(2*M+N,N+LWORK(DGEQP3),N+LWORK(DGEQRF),
                  N+N*N+LWORK(DPOCON),7).

  If SIGMA and the right singular vectors are needed (JOBV.EQ.'V'),
  -> the minimal requirement is LWORK >= max(2*M+N,4*N+1,7).
  -> For optimal performance, LWORK >= max(2*M+N,3*N+(N+1)*NB,7),
     where NB is the optimal block size for DGEQP3, DGEQRF, DGELQ,
     DORMLQ. In general, the optimal length LWORK is computed as
     LWORK >= max(2*M+N,N+LWORK(DGEQP3), N+LWORK(DPOCON),
              N+LWORK(DGELQ), 2*N+LWORK(DGEQRF), N+LWORK(DORMLQ)).

  If SIGMA and the left singular vectors are needed
  -> the minimal requirement is LWORK >= max(2*M+N,4*N+1,7).
  -> For optimal performance:
     if JOBU.EQ.'U' :: LWORK >= max(2*M+N,3*N+(N+1)*NB,7),
     if JOBU.EQ.'F' :: LWORK >= max(2*M+N,3*N+(N+1)*NB,N+M*NB,7),
         where NB is the optimal block size for DGEQP3, DGEQRF, DORMQR.
         In general, the optimal length LWORK is computed as
         LWORK >= max(2*M+N,N+LWORK(DGEQP3),N+LWORK(DPOCON),
                  2*N+LWORK(DGEQRF), N+LWORK(DORMQR)).
         Here LWORK(DORMQR) equals N*NB (for JOBU.EQ.'U') or
                  M*NB (for JOBU.EQ.'F').

  If the full SVD is needed: (JOBU.EQ.'U' or JOBU.EQ.'F') and
  -> if JOBV.EQ.'V' the minimal requirement is
        LWORK >= max(2*M+N,6*N+2*N*N).
```

```
   -> if JOBV.EQ.'J' the minimal requirement is
         LWORK >= max(2*M+N, 4*N+N*N,2*N+N*N+6).
   -> For optimal performance, LWORK should be additionally
   larger than N+M*NB, where NB is the optimal block size for DORMQR.

 IWORK
   IWORK is INTEGER array, dimension M+3*N.
   On exit,
   IWORK(1) = the numerical rank determined after the initial
        QR factorization with pivoting. See the descriptions
        of JOBA and JOBR.
   IWORK(2) = the number of the computed nonzero singular values
   IWORK(3) = if nonzero, a warning message:
        If IWORK(3).EQ.1 then some of the column norms of A
        were denormalized floats. The requested high accuracy
        is not warranted by the data.

 INFO
   INFO is INTEGER
   < 0  : if INFO = -i, then the i-th argument had an illegal value.
   = 0 :  successfull exit;
   > 0 :  SGEJSV  did not converge in the maximal allowed number
          of sweeps. The computed values may be inaccurate.

Further Details:
   SGEJSV implements a preconditioned Jacobi SVD algorithm. It uses SGEQP3,
   SGEQRF, and SGELQF as preprocessors and preconditioners. Optionally, an
   additional row pivoting can be used as a preprocessor, which in some
   cases results in much higher accuracy. An example is matrix A with the
   structure A = D1 * C * D2, where D1, D2 are arbitrarily ill-conditioned
   diagonal matrices and C is well-conditioned matrix. In that case, complete
   pivoting in the first QR factorizations provides accuracy dependent on the
   condition number of C, and independent of D1, D2. Such higher accuracy is
   not completely understood theoretically, but it works well in practice.
   Further, if A can be written as A = B*D, with well-conditioned B and some
   diagonal D, then the high accuracy is guaranteed, both theoretically and
   in software, independent of D. For more details see [1], [2].
   The computational range for the singular values can be the full range
   ( UNDERFLOW,OVERFLOW ), provided that the machine arithmetic and the BLAS
   & LAPACK routines called by SGEJSV are implemented to work in that range.
   If that is not the case, then the restriction for safe computation with
   the singular values in the range of normalized IEEE numbers is that the
   spectral condition number kappa(A)=sigma_max(A)/sigma_min(A) does not
   overflow. This code (SGEJSV) is best used in this restricted range,
   meaning that singular values of magnitude below ||A||_2 / SLAMCH('O') are
   returned as zeros. See JOBR for details on this.
   Further, this implementation is somewhat slower than the one described
   in [1,2] due to replacement of some non-LAPACK components, and because
   the choice of some tuning parameters in the iterative part (SGESVJ) is
   left to the implementer on a particular machine.
```

```
The rank revealing QR factorization (in this code: SGEQP3) should be
implemented as in [3]. We have a new version of SGEQP3 under development
that is more robust than the current one in LAPACK, with a cleaner cut in
rank defficient cases. It will be available in the SIGMA library [4].
If M is much larger than N, it is obvious that the inital QRF with
column pivoting can be preprocessed by the QRF without pivoting. That
well known trick is not used in SGEJSV because in some cases heavy row
weighting can be treated with complete pivoting. The overhead in cases
M much larger than N is then only due to pivoting, but the benefits in
terms of accuracy have prevailed. The implementer/user can incorporate
this extra QRF step easily. The implementer can also improve data movement
(matrix transpose, matrix copy, matrix transposed copy) - this
implementation of SGEJSV uses only the simplest, naive data movement.
```

———-

29.1.3 fortran code

— **sgejsv.f** —

```
*  ===================================================================
      SUBROUTINE SGEJSV( JOBA, JOBU, JOBV, JOBR, JOBT, JOBP,
     $                   M, N, A, LDA, SVA, U, LDU, V, LDV,
     $                   WORK, LWORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      IMPLICIT    NONE
      INTEGER     INFO, LDA, LDU, LDV, LWORK, M, N
*     ..
*     .. Array Arguments ..
      REAL        A( LDA, * ), SVA( N ), U( LDU, * ), V( LDV, * ),
     $            WORK( LWORK )
      INTEGER     IWORK( * )
      CHARACTER*1 JOBA, JOBP, JOBR, JOBT, JOBU, JOBV
*     ..
*
*  ======================================================================
*
*     .. Local Parameters ..
      REAL        ZERO,         ONE
      PARAMETER ( ZERO = 0.0E0, ONE = 1.0E0 )
*     ..
```

```
*     .. Local Scalars ..
      REAL    AAPP,   AAQQ,   AATMAX, AATMIN, BIG,    BIG1,   COND_OK,
     $        CONDR1, CONDR2, ENTRA,  ENTRAT, EPSLN,  MAXPRJ, SCALEM,
     $        SCONDA, SFMIN,  SMALL,  TEMP1,  USCAL1, USCAL2, XSC
      INTEGER IERR,   N1,     NR,     NUMRANK,        p, q,   WARNING
      LOGICAL ALMORT, DEFR,   ERREST, GOSCAL, JRACC,  KILL,   LSVEC,
     $        L2ABER, L2KILL, L2PERT, L2RANK, L2TRAN,
     $        NOSCAL, ROWPIV, RSVEC,  TRANSP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC ABS, ALOG, MAX, MIN, FLOAT, NINT, SIGN, SQRT
*     ..
*     .. External Functions ..
      REAL      SLAMCH, SNRM2
      INTEGER   ISAMAX
      LOGICAL   LSAME
      EXTERNAL  ISAMAX, LSAME, SLAMCH, SNRM2
*     ..
*     .. External Subroutines ..
      EXTERNAL  SCOPY,  SGELQF, SGEQP3, SGEQRF, SLACPY, SLASCL,
     $          SLASET, SLASSQ, SLASWP, SORGQR, SORMLQ,
     $          SORMQR, SPOCON, SSCAL,  SSWAP,  STRSM,  XERBLA
*
      EXTERNAL  SGESVJ
*     ..
*
*     Test the input arguments
*
      LSVEC  = LSAME( JOBU, 'U' ) .OR. LSAME( JOBU, 'F' )
      JRACC  = LSAME( JOBV, 'J' )
      RSVEC  = LSAME( JOBV, 'V' ) .OR. JRACC
      ROWPIV = LSAME( JOBA, 'F' ) .OR. LSAME( JOBA, 'G' )
      L2RANK = LSAME( JOBA, 'R' )
      L2ABER = LSAME( JOBA, 'A' )
      ERREST = LSAME( JOBA, 'E' ) .OR. LSAME( JOBA, 'G' )
      L2TRAN = LSAME( JOBT, 'T' )
      L2KILL = LSAME( JOBR, 'R' )
      DEFR   = LSAME( JOBR, 'N' )
      L2PERT = LSAME( JOBP, 'P' )
*
      IF ( .NOT.(ROWPIV .OR. L2RANK .OR. L2ABER .OR.
     $     ERREST .OR. LSAME( JOBA, 'C' ) )) THEN
         INFO = - 1
      ELSE IF ( .NOT.( LSVEC  .OR. LSAME( JOBU, 'N' ) .OR.
     $                             LSAME( JOBU, 'W' )) ) THEN
         INFO = - 2
      ELSE IF ( .NOT.( RSVEC .OR. LSAME( JOBV, 'N' ) .OR.
     $   LSAME( JOBV, 'W' )) .OR. ( JRACC .AND. (.NOT.LSVEC) ) ) THEN
         INFO = - 3
      ELSE IF ( .NOT. ( L2KILL .OR. DEFR ) )    THEN
```

```
         INFO = - 4
      ELSE IF ( .NOT. ( L2TRAN .OR. LSAME( JOBT, 'N' ) ) ) THEN
         INFO = - 5
      ELSE IF ( .NOT. ( L2PERT .OR. LSAME( JOBP, 'N' ) ) ) THEN
         INFO = - 6
      ELSE IF ( M .LT. 0 ) THEN
         INFO = - 7
      ELSE IF ( ( N .LT. 0 ) .OR. ( N .GT. M ) ) THEN
         INFO = - 8
      ELSE IF ( LDA .LT. M ) THEN
         INFO = - 10
      ELSE IF ( LSVEC .AND. ( LDU .LT. M ) ) THEN
         INFO = - 13
      ELSE IF ( RSVEC .AND. ( LDV .LT. N ) ) THEN
         INFO = - 14
      ELSE IF ( (.NOT.(LSVEC .OR. RSVEC .OR. ERREST).AND.
     $                           (LWORK .LT. MAX(7,4*N+1,2*M+N))) .OR.
     $ (.NOT.(LSVEC .OR. RSVEC) .AND. ERREST .AND.
     $                         (LWORK .LT. MAX(7,4*N+N*N,2*M+N))) .OR.
     $ (LSVEC .AND. (.NOT.RSVEC) .AND. (LWORK .LT. MAX(7,2*M+N,4*N+1)))
     $ .OR.
     $ (RSVEC .AND. (.NOT.LSVEC) .AND. (LWORK .LT. MAX(7,2*M+N,4*N+1)))
     $ .OR.
     $ (LSVEC .AND. RSVEC .AND. (.NOT.JRACC) .AND.
     $                          (LWORK.LT.MAX(2*M+N,6*N+2*N*N)))
     $ .OR. (LSVEC .AND. RSVEC .AND. JRACC .AND.
     $                          LWORK.LT.MAX(2*M+N,4*N+N*N,2*N+N*N+6)))
     $   THEN
         INFO = - 17
      ELSE
*        #:)
         INFO = 0
      END IF
*
      IF ( INFO .NE. 0 ) THEN
*       #:(
         CALL XERBLA( 'SGEJSV', - INFO )
         RETURN
      END IF
*
*     Quick return for void matrix (Y3K safe)
* #:)
      IF ( ( M .EQ. 0 ) .OR. ( N .EQ. 0 ) ) RETURN
*
*     Determine whether the matrix U should be M x N or M x M
*
      IF ( LSVEC ) THEN
         N1 = N
         IF ( LSAME( JOBU, 'F' ) ) N1 = M
      END IF
```

```
*
*     Set numerical parameters
*
*!    NOTE: Make sure SLAMCH() does not fail on the target architecture.
*
      EPSLN = SLAMCH('Epsilon')
      SFMIN = SLAMCH('SafeMinimum')
      SMALL = SFMIN / EPSLN
      BIG   = SLAMCH('O')
*     BIG   = ONE / SFMIN
*
*     Initialize SVA(1:N) = diag( ||A e_i||_2 )_1^N
*
*(!)  If necessary, scale SVA() to protect the largest norm from
*     overflow. It is possible that this scaling pushes the smallest
*     column norm left from the underflow threshold (extreme case).
*
      SCALEM  = ONE / SQRT(FLOAT(M)*FLOAT(N))
      NOSCAL  = .TRUE.
      GOSCAL  = .TRUE.
      DO 1874 p = 1, N
         AAPP = ZERO
         AAQQ = ONE
         CALL SLASSQ( M, A(1,p), 1, AAPP, AAQQ )
         IF ( AAPP .GT. BIG ) THEN
            INFO = - 9
            CALL XERBLA( 'SGEJSV', -INFO )
            RETURN
         END IF
         AAQQ = SQRT(AAQQ)
         IF ( ( AAPP .LT. (BIG / AAQQ) ) .AND. NOSCAL  ) THEN
            SVA(p)  = AAPP * AAQQ
         ELSE
            NOSCAL  = .FALSE.
            SVA(p)  = AAPP * ( AAQQ * SCALEM )
            IF ( GOSCAL ) THEN
               GOSCAL = .FALSE.
               CALL SSCAL( p-1, SCALEM, SVA, 1 )
            END IF
         END IF
 1874 CONTINUE
*
      IF ( NOSCAL ) SCALEM = ONE
*
      AAPP = ZERO
      AAQQ = BIG
      DO 4781 p = 1, N
         AAPP = MAX( AAPP, SVA(p) )
         IF ( SVA(p) .NE. ZERO ) AAQQ = MIN( AAQQ, SVA(p) )
 4781 CONTINUE
```

```
*
*     Quick return for zero M x N matrix
* #:)
      IF ( AAPP .EQ. ZERO ) THEN
         IF ( LSVEC ) CALL SLASET( 'G', M, N1, ZERO, ONE, U, LDU )
         IF ( RSVEC ) CALL SLASET( 'G', N, N,  ZERO, ONE, V, LDV )
         WORK(1) = ONE
         WORK(2) = ONE
         IF ( ERREST ) WORK(3) = ONE
         IF ( LSVEC .AND. RSVEC ) THEN
            WORK(4) = ONE
            WORK(5) = ONE
         END IF
         IF ( L2TRAN ) THEN
            WORK(6) = ZERO
            WORK(7) = ZERO
         END IF
         IWORK(1) = 0
         IWORK(2) = 0
         IWORK(3) = 0
         RETURN
      END IF
*
*     Issue warning if denormalized column norms detected. Override the
*     high relative accuracy request. Issue licence to kill columns
*     (set them to zero) whose norm is less than sigma_max / BIG (roughly).
* #:(
      WARNING = 0
      IF ( AAQQ .LE. SFMIN ) THEN
         L2RANK = .TRUE.
         L2KILL = .TRUE.
         WARNING = 1
      END IF
*
*     Quick return for one-column matrix
* #:)
      IF ( N .EQ. 1 ) THEN
*
         IF ( LSVEC ) THEN
            CALL SLASCL( 'G',0,0,SVA(1),SCALEM, M,1,A(1,1),LDA,IERR )
            CALL SLACPY( 'A', M, 1, A, LDA, U, LDU )
*           computing all M left singular vectors of the M x 1 matrix
            IF ( N1 .NE. N  ) THEN
               CALL SGEQRF( M, N, U,LDU, WORK, WORK(N+1),LWORK-N,IERR )
               CALL SORGQR( M,N1,1, U,LDU,WORK,WORK(N+1),LWORK-N,IERR )
               CALL SCOPY( M, A(1,1), 1, U(1,1), 1 )
            END IF
         END IF
         IF ( RSVEC ) THEN
             V(1,1) = ONE
```

```
         END IF
         IF ( SVA(1) .LT. (BIG*SCALEM) ) THEN
            SVA(1)  = SVA(1) / SCALEM
            SCALEM  = ONE
         END IF
         WORK(1) = ONE / SCALEM
         WORK(2) = ONE
         IF ( SVA(1) .NE. ZERO ) THEN
            IWORK(1) = 1
            IF ( ( SVA(1) / SCALEM) .GE. SFMIN ) THEN
               IWORK(2) = 1
            ELSE
               IWORK(2) = 0
            END IF
         ELSE
            IWORK(1) = 0
            IWORK(2) = 0
         END IF
         IF ( ERREST ) WORK(3) = ONE
         IF ( LSVEC .AND. RSVEC ) THEN
            WORK(4) = ONE
            WORK(5) = ONE
         END IF
         IF ( L2TRAN ) THEN
            WORK(6) = ZERO
            WORK(7) = ZERO
         END IF
         RETURN
*
      END IF
*
      TRANSP = .FALSE.
      L2TRAN = L2TRAN .AND. ( M .EQ. N )
*
      AATMAX = -ONE
      AATMIN =  BIG
      IF ( ROWPIV .OR. L2TRAN ) THEN
*
*     Compute the row norms, needed to determine row pivoting sequence
*     (in the case of heavily row weighted A, row pivoting is strongly
*     advised) and to collect information needed to compare the
*     structures of A * A^t and A^t * A (in the case L2TRAN.EQ..TRUE.).
*
         IF ( L2TRAN ) THEN
            DO 1950 p = 1, M
               XSC   = ZERO
               TEMP1 = ONE
               CALL SLASSQ( N, A(p,1), LDA, XSC, TEMP1 )
*              SLASSQ gets both the ell_2 and the ell_infinity norm
*              in one pass through the vector
```

```
               WORK(M+N+p)  = XSC * SCALEM
               WORK(N+p)    = XSC * (SCALEM*SQRT(TEMP1))
               AATMAX = MAX( AATMAX, WORK(N+p) )
               IF (WORK(N+p) .NE. ZERO) AATMIN = MIN(AATMIN,WORK(N+p))
 1950       CONTINUE
         ELSE
            DO 1904 p = 1, M
               WORK(M+N+p) = SCALEM*ABS( A(p,ISAMAX(N,A(p,1),LDA)) )
               AATMAX = MAX( AATMAX, WORK(M+N+p) )
               AATMIN = MIN( AATMIN, WORK(M+N+p) )
 1904       CONTINUE
         END IF
*
      END IF
*
*     For square matrix A try to determine whether A^t  would be  better
*     input for the preconditioned Jacobi SVD, with faster convergence.
*     The decision is based on an O(N) function of the vector of column
*     and row norms of A, based on the Shannon entropy. This should give
*     the right choice in most cases when the difference actually matters.
*     It may fail and pick the slower converging side.
*
      ENTRA  = ZERO
      ENTRAT = ZERO
      IF ( L2TRAN ) THEN
*
         XSC   = ZERO
         TEMP1 = ONE
         CALL SLASSQ( N, SVA, 1, XSC, TEMP1 )
         TEMP1 = ONE / TEMP1
*
         ENTRA = ZERO
         DO 1113 p = 1, N
            BIG1  = ( ( SVA(p) / XSC )**2 ) * TEMP1
            IF ( BIG1 .NE. ZERO ) ENTRA = ENTRA + BIG1 * ALOG(BIG1)
 1113    CONTINUE
         ENTRA = - ENTRA / ALOG(FLOAT(N))
*
*        Now, SVA().^2/Trace(A^t * A) is a point in the probability simplex.
*        It is derived from the diagonal of  A^t * A.  Do the same with the
*        diagonal of A * A^t, compute the entropy of the corresponding
*        probability distribution. Note that A * A^t and A^t * A have the
*        same trace.
*
         ENTRAT = ZERO
         DO 1114 p = N+1, N+M
            BIG1 = ( ( WORK(p) / XSC )**2 ) * TEMP1
            IF ( BIG1 .NE. ZERO ) ENTRAT = ENTRAT + BIG1 * ALOG(BIG1)
 1114    CONTINUE
         ENTRAT = - ENTRAT / ALOG(FLOAT(M))
```

```
*
*        Analyze the entropies and decide A or A^t. Smaller entropy
*        usually means better input for the algorithm.
*
         TRANSP = ( ENTRAT .LT. ENTRA )
*
*        If A^t is better than A, transpose A.
*
         IF ( TRANSP ) THEN
*           In an optimal implementation, this trivial transpose
*           should be replaced with faster transpose.
            DO 1115 p = 1, N - 1
               DO 1116 q = p + 1, N
                   TEMP1 = A(q,p)
                  A(q,p) = A(p,q)
                  A(p,q) = TEMP1
 1116          CONTINUE
 1115       CONTINUE
            DO 1117 p = 1, N
               WORK(M+N+p) = SVA(p)
               SVA(p)      = WORK(N+p)
 1117       CONTINUE
            TEMP1  = AAPP
            AAPP   = AATMAX
            AATMAX = TEMP1
            TEMP1  = AAQQ
            AAQQ   = AATMIN
            AATMIN = TEMP1
            KILL   = LSVEC
            LSVEC  = RSVEC
            RSVEC  = KILL
            IF ( LSVEC ) N1 = N
*
            ROWPIV = .TRUE.
         END IF
*
      END IF
*     END IF L2TRAN
*
*     Scale the matrix so that its maximal singular value remains less
*     than SQRT(BIG) -- the matrix is scaled so that its maximal column
*     has Euclidean norm equal to SQRT(BIG/N). The only reason to keep
*     SQRT(BIG) instead of BIG is the fact that SGEJSV uses LAPACK and
*     BLAS routines that, in some implementations, are not capable of
*     working in the full interval [SFMIN,BIG] and that they may provoke
*     overflows in the intermediate results. If the singular values spread
*     from SFMIN to BIG, then SGESVJ will compute them. So, in that case,
*     one should use SGESVJ instead of SGEJSV.
*
      BIG1   = SQRT( BIG )
```

```
      TEMP1  = SQRT( BIG / FLOAT(N) )
*
      CALL SLASCL( 'G', 0, 0, AAPP, TEMP1, N, 1, SVA, N, IERR )
      IF ( AAQQ .GT. (AAPP * SFMIN) ) THEN
          AAQQ = ( AAQQ / AAPP ) * TEMP1
      ELSE
          AAQQ = ( AAQQ * TEMP1 ) / AAPP
      END IF
      TEMP1 = TEMP1 * SCALEM
      CALL SLASCL( 'G', 0, 0, AAPP, TEMP1, M, N, A, LDA, IERR )
*
*     To undo scaling at the end of this procedure, multiply the
*     computed singular values with USCAL2 / USCAL1.
*
      USCAL1 = TEMP1
      USCAL2 = AAPP
*
      IF ( L2KILL ) THEN
*        L2KILL enforces computation of nonzero singular values in
*        the restricted range of condition number of the initial A,
*        sigma_max(A) / sigma_min(A) approx. SQRT(BIG)/SQRT(SFMIN).
         XSC = SQRT( SFMIN )
      ELSE
         XSC = SMALL
*
*        Now, if the condition number of A is too big,
*        sigma_max(A) / sigma_min(A) .GT. SQRT(BIG/N) * EPSLN / SFMIN,
*        as a precaution measure, the full SVD is computed using SGESVJ
*        with accumulated Jacobi rotations. This provides numerically
*        more robust computation, at the cost of slightly increased run
*        time. Depending on the concrete implementation of BLAS and LAPACK
*        (i.e. how they behave in presence of extreme ill-conditioning) the
*        implementor may decide to remove this switch.
         IF ( ( AAQQ.LT.SQRT(SFMIN) ) .AND. LSVEC .AND. RSVEC ) THEN
            JRACC = .TRUE.
         END IF
*
      END IF
      IF ( AAQQ .LT. XSC ) THEN
         DO 700 p = 1, N
            IF ( SVA(p) .LT. XSC ) THEN
               CALL SLASET( 'A', M, 1, ZERO, ZERO, A(1,p), LDA )
               SVA(p) = ZERO
            END IF
 700     CONTINUE
      END IF
*
*     Preconditioning using QR factorization with pivoting
*
      IF ( ROWPIV ) THEN
```

```
*        Optional row permutation (Bjoerck row pivoting):
*        A result by Cox and Higham shows that the Bjoerck's
*        row pivoting combined with standard column pivoting
*        has similar effect as Powell-Reid complete pivoting.
*        The ell-infinity norms of A are made nonincreasing.
         DO 1952 p = 1, M - 1
            q = ISAMAX( M-p+1, WORK(M+N+p), 1 ) + p - 1
            IWORK(2*N+p) = q
            IF ( p .NE. q ) THEN
               TEMP1       = WORK(M+N+p)
               WORK(M+N+p) = WORK(M+N+q)
               WORK(M+N+q) = TEMP1
            END IF
 1952    CONTINUE
         CALL SLASWP( N, A, LDA, 1, M-1, IWORK(2*N+1), 1 )
      END IF
*
*     End of the preparation phase (scaling, optional sorting and
*     transposing, optional flushing of small columns).
*
*     Preconditioning
*
*     If the full SVD is needed, the right singular vectors are computed
*     from a matrix equation, and for that we need theoretical analysis
*     of the Businger-Golub pivoting. So we use SGEQP3 as the first RR QRF.
*     In all other cases the first RR QRF can be chosen by other criteria
*     (eg speed by replacing global with restricted window pivoting, such
*     as in SGEQPX from TOMS # 782). Good results will be obtained using
*     SGEQPX with properly (!) chosen numerical parameters.
*     Any improvement of SGEQP3 improves overal performance of SGEJSV.
*
*     A * P1 = Q1 * [ R1^t 0]^t:
      DO 1963 p = 1, N
*        .. all columns are free columns
         IWORK(p) = 0
 1963 CONTINUE
      CALL SGEQP3( M,N,A,LDA, IWORK,WORK, WORK(N+1),LWORK-N, IERR )
*
*     The upper triangular matrix R1 from the first QRF is inspected for
*     rank deficiency and possibilities for deflation, or possible
*     ill-conditioning. Depending on the user specified flag L2RANK,
*     the procedure explores possibilities to reduce the numerical
*     rank by inspecting the computed upper triangular factor. If
*     L2RANK or L2ABER are up, then SGEJSV will compute the SVD of
*     A + dA, where ||dA|| <= f(M,N)*EPSLN.
*
      NR = 1
      IF ( L2ABER ) THEN
*        Standard absolute error bound suffices. All sigma_i with
*        sigma_i < N*EPSLN*||A|| are flushed to zero. This is an
```

```
*        agressive enforcement of lower numerical rank by introducing a
*        backward error of the order of N*EPSLN*||A||.
         TEMP1 = SQRT(FLOAT(N))*EPSLN
         DO 3001 p = 2, N
            IF ( ABS(A(p,p)) .GE. (TEMP1*ABS(A(1,1))) ) THEN
               NR = NR + 1
            ELSE
               GO TO 3002
            END IF
 3001    CONTINUE
 3002    CONTINUE
      ELSE IF ( L2RANK ) THEN
*        .. similarly as above, only slightly more gentle (less agressive).
*        Sudden drop on the diagonal of R1 is used as the criterion for
*        close-to-rank-defficient.
         TEMP1 = SQRT(SFMIN)
         DO 3401 p = 2, N
            IF ( ( ABS(A(p,p)) .LT. (EPSLN*ABS(A(p-1,p-1))) ) .OR.
     $           ( ABS(A(p,p)) .LT. SMALL ) .OR.
     $           ( L2KILL .AND. (ABS(A(p,p)) .LT. TEMP1) ) ) GO TO 3402
            NR = NR + 1
 3401    CONTINUE
 3402    CONTINUE
*
      ELSE
*        The goal is high relative accuracy. However, if the matrix
*        has high scaled condition number the relative accuracy is in
*        general not feasible. Later on, a condition number estimator
*        will be deployed to estimate the scaled condition number.
*        Here we just remove the underflowed part of the triangular
*        factor. This prevents the situation in which the code is
*        working hard to get the accuracy not warranted by the data.
         TEMP1  = SQRT(SFMIN)
         DO 3301 p = 2, N
            IF ( ( ABS(A(p,p)) .LT. SMALL ) .OR.
     $           ( L2KILL .AND. (ABS(A(p,p)) .LT. TEMP1) ) ) GO TO 3302
            NR = NR + 1
 3301    CONTINUE
 3302    CONTINUE
*
      END IF
*
      ALMORT = .FALSE.
      IF ( NR .EQ. N ) THEN
         MAXPRJ = ONE
         DO 3051 p = 2, N
            TEMP1  = ABS(A(p,p)) / SVA(IWORK(p))
            MAXPRJ = MIN( MAXPRJ, TEMP1 )
 3051    CONTINUE
         IF ( MAXPRJ**2 .GE. ONE - FLOAT(N)*EPSLN ) ALMORT = .TRUE.
```

```
      END IF
*
*
      SCONDA = - ONE
      CONDR1 = - ONE
      CONDR2 = - ONE
*
      IF ( ERREST ) THEN
         IF ( N .EQ. NR ) THEN
            IF ( RSVEC ) THEN
*              .. V is available as workspace
               CALL SLACPY( 'U', N, N, A, LDA, V, LDV )
               DO 3053 p = 1, N
                  TEMP1 = SVA(IWORK(p))
                  CALL SSCAL( p, ONE/TEMP1, V(1,p), 1 )
 3053          CONTINUE
               CALL SPOCON( 'U', N, V, LDV, ONE, TEMP1,
     $              WORK(N+1), IWORK(2*N+M+1), IERR )
            ELSE IF ( LSVEC ) THEN
*              .. U is available as workspace
               CALL SLACPY( 'U', N, N, A, LDA, U, LDU )
               DO 3054 p = 1, N
                  TEMP1 = SVA(IWORK(p))
                  CALL SSCAL( p, ONE/TEMP1, U(1,p), 1 )
 3054          CONTINUE
               CALL SPOCON( 'U', N, U, LDU, ONE, TEMP1,
     $              WORK(N+1), IWORK(2*N+M+1), IERR )
            ELSE
               CALL SLACPY( 'U', N, N, A, LDA, WORK(N+1), N )
               DO 3052 p = 1, N
                  TEMP1 = SVA(IWORK(p))
                  CALL SSCAL( p, ONE/TEMP1, WORK(N+(p-1)*N+1), 1 )
 3052          CONTINUE
*           .. the columns of R are scaled to have unit Euclidean lengths.
               CALL SPOCON( 'U', N, WORK(N+1), N, ONE, TEMP1,
     $              WORK(N+N*N+1), IWORK(2*N+M+1), IERR )
            END IF
            SCONDA = ONE / SQRT(TEMP1)
*           SCONDA is an estimate of SQRT(||(R^t * R)^(-1)||_1).
*           N^(-1/4) * SCONDA <= ||R^(-1)||_2 <= N^(1/4) * SCONDA
         ELSE
            SCONDA = - ONE
         END IF
      END IF
*
      L2PERT = L2PERT .AND. ( ABS( A(1,1)/A(NR,NR) ) .GT. SQRT(BIG1) )
*     If there is no violent scaling, artificial perturbation is not needed.
*
*     Phase 3:
*
```

```
      IF ( .NOT. ( RSVEC .OR. LSVEC ) ) THEN
*
*         Singular Values only
*
*         .. transpose A(1:NR,1:N)
         DO 1946 p = 1, MIN( N-1, NR )
            CALL SCOPY( N-p, A(p,p+1), LDA, A(p+1,p), 1 )
 1946    CONTINUE
*
*        The following two DO-loops introduce small relative perturbation
*        into the strict upper triangle of the lower triangular matrix.
*        Small entries below the main diagonal are also changed.
*        This modification is useful if the computing environment does not
*        provide/allow FLUSH TO ZERO underflow, for it prevents many
*        annoying denormalized numbers in case of strongly scaled matrices.
*        The perturbation is structured so that it does not introduce any
*        new perturbation of the singular values, and it does not destroy
*        the job done by the preconditioner.
*        The licence for this perturbation is in the variable L2PERT, which
*        should be .FALSE. if FLUSH TO ZERO underflow is active.
*
         IF ( .NOT. ALMORT ) THEN
*
            IF ( L2PERT ) THEN
*              XSC = SQRT(SMALL)
               XSC = EPSLN / FLOAT(N)
               DO 4947 q = 1, NR
                  TEMP1 = XSC*ABS(A(q,q))
                  DO 4949 p = 1, N
                     IF ( ( (p.GT.q) .AND. (ABS(A(p,q)).LE.TEMP1) )
     $                    .OR. ( p .LT. q ) )
     $                     A(p,q) = SIGN( TEMP1, A(p,q) )
 4949             CONTINUE
 4947          CONTINUE
            ELSE
               CALL SLASET( 'U', NR-1,NR-1, ZERO,ZERO, A(1,2),LDA )
            END IF
*
*            .. second preconditioning using the QR factorization
*
            CALL SGEQRF( N,NR, A,LDA, WORK, WORK(N+1),LWORK-N, IERR )
*
*           .. and transpose upper to lower triangular
            DO 1948 p = 1, NR - 1
               CALL SCOPY( NR-p, A(p,p+1), LDA, A(p+1,p), 1 )
 1948       CONTINUE
*
         END IF
*
*           Row-cyclic Jacobi SVD algorithm with column pivoting
```

```
*
*          .. again some perturbation (a "background noise") is added
*          to drown denormals
           IF ( L2PERT ) THEN
*             XSC = SQRT(SMALL)
              XSC = EPSLN / FLOAT(N)
              DO 1947 q = 1, NR
                 TEMP1 = XSC*ABS(A(q,q))
                 DO 1949 p = 1, NR
                    IF ( ( (p.GT.q) .AND. (ABS(A(p,q)).LE.TEMP1) )
     $                      .OR. ( p .LT. q ) )
     $                 A(p,q) = SIGN( TEMP1, A(p,q) )
 1949            CONTINUE
 1947         CONTINUE
           ELSE
              CALL SLASET( 'U', NR-1, NR-1, ZERO, ZERO, A(1,2), LDA )
           END IF
*
*          .. and one-sided Jacobi rotations are started on a lower
*          triangular matrix (plus perturbation which is ignored in
*          the part which destroys triangular form (confusing?!))
*
           CALL SGESVJ( 'L', 'NoU', 'NoV', NR, NR, A, LDA, SVA,
     $                    N, V, LDV, WORK, LWORK, INFO )
*
           SCALEM  = WORK(1)
           NUMRANK = NINT(WORK(2))
*
*
      ELSE IF ( RSVEC .AND. ( .NOT. LSVEC ) ) THEN
*
*        -> Singular Values and Right Singular Vectors <-
*
         IF ( ALMORT ) THEN
*
*          .. in this case NR equals N
           DO 1998 p = 1, NR
              CALL SCOPY( N-p+1, A(p,p), LDA, V(p,p), 1 )
 1998      CONTINUE
           CALL SLASET( 'Upper', NR-1, NR-1, ZERO, ZERO, V(1,2), LDV )
*
           CALL SGESVJ( 'L','U','N', N, NR, V,LDV, SVA, NR, A,LDA,
     $                 WORK, LWORK, INFO )
           SCALEM  = WORK(1)
           NUMRANK = NINT(WORK(2))

         ELSE
*
*        .. two more QR factorizations ( one QRF is not enough, two require
*        accumulated product of Jacobi rotations, three are perfect )
```

```
*
            CALL SLASET( 'Lower', NR-1, NR-1, ZERO, ZERO, A(2,1), LDA )
            CALL SGELQF( NR, N, A, LDA, WORK, WORK(N+1), LWORK-N, IERR)
            CALL SLACPY( 'Lower', NR, NR, A, LDA, V, LDV )
            CALL SLASET( 'Upper', NR-1, NR-1, ZERO, ZERO, V(1,2), LDV )
            CALL SGEQRF( NR, NR, V, LDV, WORK(N+1), WORK(2*N+1),
     $                   LWORK-2*N, IERR )
            DO 8998 p = 1, NR
               CALL SCOPY( NR-p+1, V(p,p), LDV, V(p,p), 1 )
 8998       CONTINUE
            CALL SLASET( 'Upper', NR-1, NR-1, ZERO, ZERO, V(1,2), LDV )
*
            CALL SGESVJ( 'Lower', 'U','N', NR, NR, V,LDV, SVA, NR, U,
     $                   LDU, WORK(N+1), LWORK-N, INFO )
            SCALEM  = WORK(N+1)
            NUMRANK = NINT(WORK(N+2))
            IF ( NR .LT. N ) THEN
               CALL SLASET( 'A',N-NR, NR, ZERO,ZERO, V(NR+1,1),   LDV )
               CALL SLASET( 'A',NR, N-NR, ZERO,ZERO, V(1,NR+1),   LDV )
               CALL SLASET( 'A',N-NR,N-NR,ZERO,ONE, V(NR+1,NR+1), LDV )
            END IF
*
         CALL SORMLQ( 'Left', 'Transpose', N, N, NR, A, LDA, WORK,
     $               V, LDV, WORK(N+1), LWORK-N, IERR )
*
         END IF
*
         DO 8991 p = 1, N
            CALL SCOPY( N, V(p,1), LDV, A(IWORK(p),1), LDA )
 8991    CONTINUE
         CALL SLACPY( 'All', N, N, A, LDA, V, LDV )
*
         IF ( TRANSP ) THEN
            CALL SLACPY( 'All', N, N, V, LDV, U, LDU )
         END IF
*
      ELSE IF ( LSVEC .AND. ( .NOT. RSVEC ) ) THEN
*
*        .. Singular Values and Left Singular Vectors                 ..
*
*        .. second preconditioning step to avoid need to accumulate
*        Jacobi rotations in the Jacobi iterations.
         DO 1965 p = 1, NR
            CALL SCOPY( N-p+1, A(p,p), LDA, U(p,p), 1 )
 1965    CONTINUE
         CALL SLASET( 'Upper', NR-1, NR-1, ZERO, ZERO, U(1,2), LDU )
*
         CALL SGEQRF( N, NR, U, LDU, WORK(N+1), WORK(2*N+1),
     $              LWORK-2*N, IERR )
*
```

```
         DO 1967 p = 1, NR - 1
            CALL SCOPY( NR-p, U(p,p+1), LDU, U(p+1,p), 1 )
 1967    CONTINUE
         CALL SLASET( 'Upper', NR-1, NR-1, ZERO, ZERO, U(1,2), LDU )
*
         CALL SGESVJ( 'Lower', 'U', 'N', NR,NR, U, LDU, SVA, NR, A,
     $        LDA, WORK(N+1), LWORK-N, INFO )
         SCALEM  = WORK(N+1)
         NUMRANK = NINT(WORK(N+2))
*
         IF ( NR .LT. M ) THEN
            CALL SLASET( 'A',  M-NR, NR,ZERO, ZERO, U(NR+1,1), LDU )
            IF ( NR .LT. N1 ) THEN
               CALL SLASET( 'A',NR, N1-NR, ZERO, ZERO, U(1,NR+1), LDU )
               CALL SLASET( 'A',M-NR,N1-NR,ZERO,ONE,U(NR+1,NR+1), LDU )
            END IF
         END IF
*
         CALL SORMQR( 'Left', 'No Tr', M, N1, N, A, LDA, WORK, U,
     $               LDU, WORK(N+1), LWORK-N, IERR )
*
         IF ( ROWPIV )
     $       CALL SLASWP( N1, U, LDU, 1, M-1, IWORK(2*N+1), -1 )
*
         DO 1974 p = 1, N1
            XSC = ONE / SNRM2( M, U(1,p), 1 )
            CALL SSCAL( M, XSC, U(1,p), 1 )
 1974    CONTINUE
*
         IF ( TRANSP ) THEN
            CALL SLACPY( 'All', N, N, U, LDU, V, LDV )
         END IF
*
      ELSE
*
*        .. Full SVD ..
*
         IF ( .NOT. JRACC ) THEN
*
         IF ( .NOT. ALMORT ) THEN
*
*           Second Preconditioning Step (QRF [with pivoting])
*           Note that the composition of TRANSPOSE, QRF and TRANSPOSE is
*           equivalent to an LQF CALL. Since in many libraries the QRF
*           seems to be better optimized than the LQF, we do explicit
*           transpose and use the QRF. This is subject to changes in an
*           optimized implementation of SGEJSV.
*
            DO 1968 p = 1, NR
               CALL SCOPY( N-p+1, A(p,p), LDA, V(p,p), 1 )
```

```
 1968       CONTINUE
*
*           .. the following two loops perturb small entries to avoid
*           denormals in the second QR factorization, where they are
*           as good as zeros. This is done to avoid painfully slow
*           computation with denormals. The relative size of the perturbation
*           is a parameter that can be changed by the implementer.
*           This perturbation device will be obsolete on machines with
*           properly implemented arithmetic.
*           To switch it off, set L2PERT=.FALSE. To remove it from  the
*           code, remove the action under L2PERT=.TRUE., leave the ELSE part.
*           The following two loops should be blocked and fused with the
*           transposed copy above.
*
            IF ( L2PERT ) THEN
               XSC = SQRT(SMALL)
               DO 2969 q = 1, NR
                  TEMP1 = XSC*ABS( V(q,q) )
                  DO 2968 p = 1, N
                     IF ( ( p .GT. q ) .AND. ( ABS(V(p,q)) .LE. TEMP1 )
     $                   .OR. ( p .LT. q ) )
     $                   V(p,q) = SIGN( TEMP1, V(p,q) )
                     IF ( p .LT. q ) V(p,q) = - V(p,q)
 2968             CONTINUE
 2969          CONTINUE
            ELSE
               CALL SLASET( 'U', NR-1, NR-1, ZERO, ZERO, V(1,2), LDV )
            END IF
*
*           Estimate the row scaled condition number of R1
*           (If R1 is rectangular, N > NR, then the condition number
*           of the leading NR x NR submatrix is estimated.)
*
            CALL SLACPY( 'L', NR, NR, V, LDV, WORK(2*N+1), NR )
            DO 3950 p = 1, NR
               TEMP1 = SNRM2(NR-p+1,WORK(2*N+(p-1)*NR+p),1)
               CALL SSCAL(NR-p+1,ONE/TEMP1,WORK(2*N+(p-1)*NR+p),1)
 3950       CONTINUE
            CALL SPOCON('Lower',NR,WORK(2*N+1),NR,ONE,TEMP1,
     $                  WORK(2*N+NR*NR+1),IWORK(M+2*N+1),IERR)
            CONDR1 = ONE / SQRT(TEMP1)
*           .. here need a second oppinion on the condition number
*           .. then assume worst case scenario
*           R1 is OK for inverse <=> CONDR1 .LT. FLOAT(N)
*           more conservative    <=> CONDR1 .LT. SQRT(FLOAT(N))
*
            COND_OK = SQRT(FLOAT(NR))
*[TP]       COND_OK is a tuning parameter.

            IF ( CONDR1 .LT. COND_OK ) THEN
```

```
*              .. the second QRF without pivoting. Note: in an optimized
*              implementation, this QRF should be implemented as the QRF
*              of a lower triangular matrix.
*              R1^t = Q2 * R2
               CALL SGEQRF( N, NR, V, LDV, WORK(N+1), WORK(2*N+1),
     $              LWORK-2*N, IERR )
*
               IF ( L2PERT ) THEN
                  XSC = SQRT(SMALL)/EPSLN
                  DO 3959 p = 2, NR
                     DO 3958 q = 1, p - 1
                        TEMP1 = XSC * MIN(ABS(V(p,p)),ABS(V(q,q)))
                        IF ( ABS(V(q,p)) .LE. TEMP1 )
     $                     V(q,p) = SIGN( TEMP1, V(q,p) )
 3958                CONTINUE
 3959             CONTINUE
               END IF
*
               IF ( NR .NE. N )
     $         CALL SLACPY( 'A', N, NR, V, LDV, WORK(2*N+1), N )
*              .. save ...
*
*           .. this transposed copy should be better than naive
               DO 1969 p = 1, NR - 1
                  CALL SCOPY( NR-p, V(p,p+1), LDV, V(p+1,p), 1 )
 1969          CONTINUE
*
               CONDR2 = CONDR1
*
            ELSE
*
*              .. ill-conditioned case: second QRF with pivoting
*              Note that windowed pivoting would be equaly good
*              numerically, and more run-time efficient. So, in
*              an optimal implementation, the next call to SGEQP3
*              should be replaced with eg. CALL SGEQPX (ACM TOMS #782)
*              with properly (carefully) chosen parameters.
*
*              R1^t * P2 = Q2 * R2
               DO 3003 p = 1, NR
                  IWORK(N+p) = 0
 3003          CONTINUE
               CALL SGEQP3( N, NR, V, LDV, IWORK(N+1), WORK(N+1),
     $                 WORK(2*N+1), LWORK-2*N, IERR )
**               CALL SGEQRF( N, NR, V, LDV, WORK(N+1), WORK(2*N+1),
**     $              LWORK-2*N, IERR )
               IF ( L2PERT ) THEN
                  XSC = SQRT(SMALL)
                  DO 3969 p = 2, NR
                     DO 3968 q = 1, p - 1
```

```
                     TEMP1 = XSC * MIN(ABS(V(p,p)),ABS(V(q,q)))
                     IF ( ABS(V(q,p)) .LE. TEMP1 )
     $                  V(q,p) = SIGN( TEMP1, V(q,p) )
 3968              CONTINUE
 3969           CONTINUE
             END IF
*
             CALL SLACPY( 'A', N, NR, V, LDV, WORK(2*N+1), N )
*
             IF ( L2PERT ) THEN
                XSC = SQRT(SMALL)
                DO 8970 p = 2, NR
                   DO 8971 q = 1, p - 1
                      TEMP1 = XSC * MIN(ABS(V(p,p)),ABS(V(q,q)))
                      V(p,q) = - SIGN( TEMP1, V(q,p) )
 8971              CONTINUE
 8970           CONTINUE
             ELSE
                CALL SLASET( 'L',NR-1,NR-1,ZERO,ZERO,V(2,1),LDV )
             END IF
*            Now, compute R2 = L3 * Q3, the LQ factorization.
             CALL SGELQF( NR, NR, V, LDV, WORK(2*N+N*NR+1),
     $             WORK(2*N+N*NR+NR+1), LWORK-2*N-N*NR-NR, IERR )
*            .. and estimate the condition number
             CALL SLACPY( 'L',NR,NR,V,LDV,WORK(2*N+N*NR+NR+1),NR )
             DO 4950 p = 1, NR
                TEMP1 = SNRM2( p, WORK(2*N+N*NR+NR+p), NR )
                CALL SSCAL( p, ONE/TEMP1, WORK(2*N+N*NR+NR+p), NR )
 4950        CONTINUE
             CALL SPOCON( 'L',NR,WORK(2*N+N*NR+NR+1),NR,ONE,TEMP1,
     $            WORK(2*N+N*NR+NR+NR*NR+1),IWORK(M+2*N+1),IERR )
             CONDR2 = ONE / SQRT(TEMP1)
*
             IF ( CONDR2 .GE. COND_OK ) THEN
*               .. save the Householder vectors used for Q3
*               (this overwrittes the copy of R2, as it will not be
*               needed in this branch, but it does not overwritte the
*               Huseholder vectors of Q2.).
                CALL SLACPY( 'U', NR, NR, V, LDV, WORK(2*N+1), N )
*               .. and the rest of the information on Q3 is in
*               WORK(2*N+N*NR+1:2*N+N*NR+N)
             END IF
*
          END IF
*
          IF ( L2PERT ) THEN
             XSC = SQRT(SMALL)
             DO 4968 q = 2, NR
                TEMP1 = XSC * V(q,q)
                DO 4969 p = 1, q - 1
```

```
*                  V(p,q) = - SIGN( TEMP1, V(q,p) )
                   V(p,q) = - SIGN( TEMP1, V(p,q) )
 4969            CONTINUE
 4968         CONTINUE
           ELSE
              CALL SLASET( 'U', NR-1,NR-1, ZERO,ZERO, V(1,2), LDV )
           END IF
*
*        Second preconditioning finished; continue with Jacobi SVD
*        The input matrix is lower trinagular.
*
*        Recover the right singular vectors as solution of a well
*        conditioned triangular matrix equation.
*
           IF ( CONDR1 .LT. COND_OK ) THEN
*
              CALL SGESVJ( 'L','U','N',NR,NR,V,LDV,SVA,NR,U,
     $             LDU,WORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,INFO )
              SCALEM  = WORK(2*N+N*NR+NR+1)
              NUMRANK = NINT(WORK(2*N+N*NR+NR+2))
              DO 3970 p = 1, NR
                 CALL SCOPY( NR, V(1,p), 1, U(1,p), 1 )
                 CALL SSCAL( NR, SVA(p),    V(1,p), 1 )
 3970         CONTINUE

*        .. pick the right matrix equation and solve it
*
              IF ( NR .EQ. N ) THEN
* :))             .. best case, R1 is inverted. The solution of this matrix
*                 equation is Q2*V2 = the product of the Jacobi rotations
*                 used in SGESVJ, premultiplied with the orthogonal matrix
*                 from the second QR factorization.
                 CALL STRSM( 'L','U','N','N', NR,NR,ONE, A,LDA, V,LDV )
              ELSE
*                .. R1 is well conditioned, but non-square. Transpose(R2)
*                 is inverted to get the product of the Jacobi rotations
*                 used in SGESVJ. The Q-factor from the second QR
*                 factorization is then built in explicitly.
                 CALL STRSM('L','U','T','N',NR,NR,ONE,WORK(2*N+1),
     $                N,V,LDV)
                 IF ( NR .LT. N ) THEN
                   CALL SLASET('A',N-NR,NR,ZERO,ZERO,V(NR+1,1),LDV)
                   CALL SLASET('A',NR,N-NR,ZERO,ZERO,V(1,NR+1),LDV)
                   CALL SLASET('A',N-NR,N-NR,ZERO,ONE,V(NR+1,NR+1),LDV)
                 END IF
                 CALL SORMQR('L','N',N,N,NR,WORK(2*N+1),N,WORK(N+1),
     $                V,LDV,WORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,IERR)
              END IF
*
           ELSE IF ( CONDR2 .LT. COND_OK ) THEN
```

```
*
* :)           .. the input matrix A is very likely a relative of
*              the Kahan matrix :)
*              The matrix R2 is inverted. The solution of the matrix equation
*              is Q3^T*V3 = the product of the Jacobi rotations (appplied to
*              the lower triangular L3 from the LQ factorization of
*              R2=L3*Q3), pre-multiplied with the transposed Q3.
               CALL SGESVJ( 'L', 'U', 'N', NR, NR, V, LDV, SVA, NR, U,
     $              LDU, WORK(2*N+N*NR+NR+1), LWORK-2*N-N*NR-NR, INFO )
               SCALEM  = WORK(2*N+N*NR+NR+1)
               NUMRANK = NINT(WORK(2*N+N*NR+NR+2))
               DO 3870 p = 1, NR
                  CALL SCOPY( NR, V(1,p), 1, U(1,p), 1 )
                  CALL SSCAL( NR, SVA(p),    U(1,p), 1 )
 3870          CONTINUE
               CALL STRSM('L','U','N','N',NR,NR,ONE,WORK(2*N+1),N,U,LDU)
*              .. apply the permutation from the second QR factorization
               DO 873 q = 1, NR
                  DO 872 p = 1, NR
                     WORK(2*N+N*NR+NR+IWORK(N+p)) = U(p,q)
 872              CONTINUE
                  DO 874 p = 1, NR
                     U(p,q) = WORK(2*N+N*NR+NR+p)
 874              CONTINUE
 873           CONTINUE
               IF ( NR .LT. N ) THEN
                  CALL SLASET( 'A',N-NR,NR,ZERO,ZERO,V(NR+1,1),LDV )
                  CALL SLASET( 'A',NR,N-NR,ZERO,ZERO,V(1,NR+1),LDV )
                  CALL SLASET( 'A',N-NR,N-NR,ZERO,ONE,V(NR+1,NR+1),LDV )
               END IF
               CALL SORMQR( 'L','N',N,N,NR,WORK(2*N+1),N,WORK(N+1),
     $              V,LDV,WORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,IERR )
            ELSE
*              Last line of defense.
* #:(          This is a rather pathological case: no scaled condition
*              improvement after two pivoted QR factorizations. Other
*              possibility is that the rank revealing QR factorization
*              or the condition estimator has failed, or the COND_OK
*              is set very close to ONE (which is unnecessary). Normally,
*              this branch should never be executed, but in rare cases of
*              failure of the RRQR or condition estimator, the last line of
*              defense ensures that SGEJSV completes the task.
*              Compute the full SVD of L3 using SGESVJ with explicit
*              accumulation of Jacobi rotations.
               CALL SGESVJ( 'L', 'U', 'V', NR, NR, V, LDV, SVA, NR, U,
     $              LDU, WORK(2*N+N*NR+NR+1), LWORK-2*N-N*NR-NR, INFO )
               SCALEM  = WORK(2*N+N*NR+NR+1)
               NUMRANK = NINT(WORK(2*N+N*NR+NR+2))
               IF ( NR .LT. N ) THEN
                  CALL SLASET( 'A',N-NR,NR,ZERO,ZERO,V(NR+1,1),LDV )
```

```
                 CALL SLASET( 'A',NR,N-NR,ZERO,ZERO,V(1,NR+1),LDV )
                 CALL SLASET( 'A',N-NR,N-NR,ZERO,ONE,V(NR+1,NR+1),LDV )
              END IF
              CALL SORMQR( 'L','N',N,N,NR,WORK(2*N+1),N,WORK(N+1),
     $             V,LDV,WORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,IERR )
*
              CALL SORMLQ( 'L', 'T', NR, NR, NR, WORK(2*N+1), N,
     $             WORK(2*N+N*NR+1), U, LDU, WORK(2*N+N*NR+NR+1),
     $             LWORK-2*N-N*NR-NR, IERR )
              DO 773 q = 1, NR
                 DO 772 p = 1, NR
                    WORK(2*N+N*NR+NR+IWORK(N+p)) = U(p,q)
 772             CONTINUE
                 DO 774 p = 1, NR
                    U(p,q) = WORK(2*N+N*NR+NR+p)
 774             CONTINUE
 773          CONTINUE
*
           END IF
*
*          Permute the rows of V using the (column) permutation from the
*          first QRF. Also, scale the columns to make them unit in
*          Euclidean norm. This applies to all cases.
*
           TEMP1 = SQRT(FLOAT(N)) * EPSLN
           DO 1972 q = 1, N
              DO 972 p = 1, N
                 WORK(2*N+N*NR+NR+IWORK(p)) = V(p,q)
  972         CONTINUE
              DO 973 p = 1, N
                 V(p,q) = WORK(2*N+N*NR+NR+p)
  973         CONTINUE
              XSC = ONE / SNRM2( N, V(1,q), 1 )
              IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $          CALL SSCAL( N, XSC, V(1,q), 1 )
 1972      CONTINUE
*          At this moment, V contains the right singular vectors of A.
*          Next, assemble the left singular vector matrix U (M x N).
           IF ( NR .LT. M ) THEN
              CALL SLASET( 'A', M-NR, NR, ZERO, ZERO, U(NR+1,1), LDU )
              IF ( NR .LT. N1 ) THEN
                 CALL SLASET('A',NR,N1-NR,ZERO,ZERO,U(1,NR+1),LDU)
                 CALL SLASET('A',M-NR,N1-NR,ZERO,ONE,U(NR+1,NR+1),LDU)
              END IF
           END IF
*
*          The Q matrix from the first QRF is built into the left singular
*          matrix U. This applies to all cases.
*
           CALL SORMQR( 'Left', 'No_Tr', M, N1, N, A, LDA, WORK, U,
```

```
     $           LDU, WORK(N+1), LWORK-N, IERR )

*           The columns of U are normalized. The cost is O(M*N) flops.
            TEMP1 = SQRT(FLOAT(M)) * EPSLN
            DO 1973 p = 1, NR
               XSC = ONE / SNRM2( M, U(1,p), 1 )
               IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $          CALL SSCAL( M, XSC, U(1,p), 1 )
 1973       CONTINUE
*
*           If the initial QRF is computed with row pivoting, the left
*           singular vectors must be adjusted.
*
            IF ( ROWPIV )
     $          CALL SLASWP( N1, U, LDU, 1, M-1, IWORK(2*N+1), -1 )
*
         ELSE
*
*        .. the initial matrix A has almost orthogonal columns and
*        the second QRF is not needed
*
            CALL SLACPY( 'Upper', N, N, A, LDA, WORK(N+1), N )
            IF ( L2PERT ) THEN
               XSC = SQRT(SMALL)
               DO 5970 p = 2, N
                  TEMP1 = XSC * WORK( N + (p-1)*N + p )
                  DO 5971 q = 1, p - 1
                     WORK(N+(q-1)*N+p)=-SIGN(TEMP1,WORK(N+(p-1)*N+q))
 5971             CONTINUE
 5970          CONTINUE
            ELSE
               CALL SLASET( 'Lower',N-1,N-1,ZERO,ZERO,WORK(N+2),N )
            END IF
*
            CALL SGESVJ( 'Upper', 'U', 'N', N, N, WORK(N+1), N, SVA,
     $           N, U, LDU, WORK(N+N*N+1), LWORK-N-N*N, INFO )
*
            SCALEM  = WORK(N+N*N+1)
            NUMRANK = NINT(WORK(N+N*N+2))
            DO 6970 p = 1, N
               CALL SCOPY( N, WORK(N+(p-1)*N+1), 1, U(1,p), 1 )
               CALL SSCAL( N, SVA(p), WORK(N+(p-1)*N+1), 1 )
 6970       CONTINUE
*
            CALL STRSM( 'Left', 'Upper', 'NoTrans', 'No UD', N, N,
     $           ONE, A, LDA, WORK(N+1), N )
            DO 6972 p = 1, N
               CALL SCOPY( N, WORK(N+p), N, V(IWORK(p),1), LDV )
 6972       CONTINUE
            TEMP1 = SQRT(FLOAT(N))*EPSLN
```

```
            DO 6971 p = 1, N
               XSC = ONE / SNRM2( N, V(1,p), 1 )
               IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $            CALL SSCAL( N, XSC, V(1,p), 1 )
 6971       CONTINUE
*
*           Assemble the left singular vector matrix U (M x N).
*
            IF ( N .LT. M ) THEN
               CALL SLASET( 'A',  M-N, N, ZERO, ZERO, U(N+1,1), LDU )
               IF ( N .LT. N1 ) THEN
                  CALL SLASET( 'A',N,  N1-N, ZERO, ZERO,  U(1,N+1),LDU )
                  CALL SLASET( 'A',M-N,N1-N, ZERO, ONE,U(N+1,N+1),LDU )
               END IF
            END IF
            CALL SORMQR( 'Left', 'No Tr', M, N1, N, A, LDA, WORK, U,
     $           LDU, WORK(N+1), LWORK-N, IERR )
            TEMP1 = SQRT(FLOAT(M))*EPSLN
            DO 6973 p = 1, N1
               XSC = ONE / SNRM2( M, U(1,p), 1 )
               IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $            CALL SSCAL( M, XSC, U(1,p), 1 )
 6973       CONTINUE
*
            IF ( ROWPIV )
     $         CALL SLASWP( N1, U, LDU, 1, M-1, IWORK(2*N+1), -1 )
*
         END IF
*
*        end of the  >> almost orthogonal case <<  in the full SVD
*
         ELSE
*
*        This branch deploys a preconditioned Jacobi SVD with explicitly
*        accumulated rotations. It is included as optional, mainly for
*        experimental purposes. It does perfom well, and can also be used.
*        In this implementation, this branch will be automatically activated
*        if the  condition number sigma_max(A) / sigma_min(A) is predicted
*        to be greater than the overflow threshold. This is because the
*        a posteriori computation of the singular vectors assumes robust
*        implementation of BLAS and some LAPACK procedures, capable of working
*        in presence of extreme values. Since that is not always the case, ...
*
         DO 7968 p = 1, NR
            CALL SCOPY( N-p+1, A(p,p), LDA, V(p,p), 1 )
 7968    CONTINUE
*
         IF ( L2PERT ) THEN
            XSC = SQRT(SMALL/EPSLN)
            DO 5969 q = 1, NR
```

```
               TEMP1 = XSC*ABS( V(q,q) )
               DO 5968 p = 1, N
                  IF ( ( p .GT. q ) .AND. ( ABS(V(p,q)) .LE. TEMP1 )
     $                .OR. ( p .LT. q ) )
     $                V(p,q) = SIGN( TEMP1, V(p,q) )
                  IF ( p .LT. q ) V(p,q) = - V(p,q)
 5968          CONTINUE
 5969       CONTINUE
         ELSE
            CALL SLASET( 'U', NR-1, NR-1, ZERO, ZERO, V(1,2), LDV )
         END IF

         CALL SGEQRF( N, NR, V, LDV, WORK(N+1), WORK(2*N+1),
     $        LWORK-2*N, IERR )
         CALL SLACPY( 'L', N, NR, V, LDV, WORK(2*N+1), N )
*
         DO 7969 p = 1, NR
            CALL SCOPY( NR-p+1, V(p,p), LDV, U(p,p), 1 )
 7969    CONTINUE

         IF ( L2PERT ) THEN
            XSC = SQRT(SMALL/EPSLN)
            DO 9970 q = 2, NR
               DO 9971 p = 1, q - 1
                  TEMP1 = XSC * MIN(ABS(U(p,p)),ABS(U(q,q)))
                  U(p,q) = - SIGN( TEMP1, U(q,p) )
 9971          CONTINUE
 9970       CONTINUE
         ELSE
            CALL SLASET('U', NR-1, NR-1, ZERO, ZERO, U(1,2), LDU )
         END IF

         CALL SGESVJ( 'L', 'U', 'V', NR, NR, U, LDU, SVA,
     $        N, V, LDV, WORK(2*N+N*NR+1), LWORK-2*N-N*NR, INFO )
         SCALEM  = WORK(2*N+N*NR+1)
         NUMRANK = NINT(WORK(2*N+N*NR+2))

         IF ( NR .LT. N ) THEN
            CALL SLASET( 'A',N-NR,NR,ZERO,ZERO,V(NR+1,1),LDV )
            CALL SLASET( 'A',NR,N-NR,ZERO,ZERO,V(1,NR+1),LDV )
            CALL SLASET( 'A',N-NR,N-NR,ZERO,ONE,V(NR+1,NR+1),LDV )
         END IF

         CALL SORMQR( 'L','N',N,N,NR,WORK(2*N+1),N,WORK(N+1),
     $        V,LDV,WORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,IERR )
*
*           Permute the rows of V using the (column) permutation from the
*           first QRF. Also, scale the columns to make them unit in
*           Euclidean norm. This applies to all cases.
*
```

```
            TEMP1 = SQRT(FLOAT(N)) * EPSLN
            DO 7972 q = 1, N
               DO 8972 p = 1, N
                  WORK(2*N+N*NR+NR+IWORK(p)) = V(p,q)
 8972          CONTINUE
               DO 8973 p = 1, N
                  V(p,q) = WORK(2*N+N*NR+NR+p)
 8973          CONTINUE
               XSC = ONE / SNRM2( N, V(1,q), 1 )
               IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $           CALL SSCAL( N, XSC, V(1,q), 1 )
 7972       CONTINUE
*
*           At this moment, V contains the right singular vectors of A.
*           Next, assemble the left singular vector matrix U (M x N).
*
         IF ( NR .LT. M ) THEN
            CALL SLASET( 'A',  M-NR, NR, ZERO, ZERO, U(NR+1,1), LDU )
            IF ( NR .LT. N1 ) THEN
               CALL SLASET( 'A',NR,  N1-NR, ZERO, ZERO,  U(1,NR+1),LDU )
               CALL SLASET( 'A',M-NR,N1-NR, ZERO, ONE,U(NR+1,NR+1),LDU )
            END IF
         END IF
*
         CALL SORMQR( 'Left', 'No Tr', M, N1, N, A, LDA, WORK, U,
     $       LDU, WORK(N+1), LWORK-N, IERR )
*
            IF ( ROWPIV )
     $         CALL SLASWP( N1, U, LDU, 1, M-1, IWORK(2*N+1), -1 )
*
*
         END IF
         IF ( TRANSP ) THEN
*           .. swap U and V because the procedure worked on A^t
            DO 6974 p = 1, N
               CALL SSWAP( N, U(1,p), 1, V(1,p), 1 )
 6974       CONTINUE
         END IF
*
      END IF
*     end of the full SVD
*
*     Undo scaling, if necessary (and possible)
*
      IF ( USCAL2 .LE. (BIG/SVA(1))*USCAL1 ) THEN
         CALL SLASCL( 'G', 0, 0, USCAL1, USCAL2, NR, 1, SVA, N, IERR )
         USCAL1 = ONE
         USCAL2 = ONE
      END IF
*
```

```
      IF ( NR .LT. N ) THEN
         DO 3004 p = NR+1, N
            SVA(p) = ZERO
 3004    CONTINUE
      END IF
*
      WORK(1) = USCAL2 * SCALEM
      WORK(2) = USCAL1
      IF ( ERREST ) WORK(3) = SCONDA
      IF ( LSVEC .AND. RSVEC ) THEN
         WORK(4) = CONDR1
         WORK(5) = CONDR2
      END IF
      IF ( L2TRAN ) THEN
         WORK(6) = ENTRA
         WORK(7) = ENTRAT
      END IF
*
      IWORK(1) = NR
      IWORK(2) = NUMRANK
      IWORK(3) = WARNING
*
      RETURN
*     ..
*     .. END OF SGEJSV
*     ..
      END
*
```

———-

29.1.4 lisp code

— **sgejsv.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sgejsv
         (joba jobu jobv jobr jobt jobp m n a lda sva u ldu v ldv work lwork
          iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (array single-float (*)) work v u sva a)
             (type (f2cl-lib:integer4) info lwork ldv ldu lda n m)
             (type (string *) jobp jobt jobr jobv jobu joba))
    (f2cl-lib:with-multi-array-data
        ((joba character joba-%data% joba-%offset%)
```

```
    (jobu character jobu-%data% jobu-%offset%)
    (jobv character jobv-%data% jobv-%offset%)
    (jobr character jobr-%data% jobr-%offset%)
    (jobt character jobt-%data% jobt-%offset%)
    (jobp character jobp-%data% jobp-%offset%)
    (a single-float a-%data% a-%offset%)
    (sva single-float sva-%data% sva-%offset%)
    (u single-float u-%data% u-%offset%)
    (v single-float v-%data% v-%offset%)
    (work single-float work-%data% work-%offset%)
    (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((almort nil) (defr nil) (errest nil) (goscal nil) (jracc nil)
         (kill nil) (lsvec nil) (l2aber nil) (l2kill nil) (l2pert nil)
         (l2rank nil) (l2tran nil) (noscal nil) (rowpiv nil) (rsvec nil)
         (transp nil) (ierr 0) (n1 0) (nr 0) (numrank 0) (p 0) (q 0)
         (warning 0) (aapp 0.0) (aaqq 0.0) (aatmax 0.0) (aatmin 0.0)
         (big 0.0) (big1 0.0) (cond_ok 0.0) (condr1 0.0) (condr2 0.0)
         (entra 0.0) (entrat 0.0) (epsln 0.0) (maxprj 0.0) (scalem 0.0)
         (sconda 0.0) (sfmin 0.0) (small 0.0) (temp1 0.0) (uscal1 0.0)
         (uscal2 0.0) (xsc 0.0))
    (declare
     (type f2cl-lib:logical almort defr errest goscal jracc kill lsvec
      l2aber l2kill l2pert l2rank l2tran noscal rowpiv rsvec transp)
     (type (f2cl-lib:integer4) ierr n1 nr numrank p q warning)
     (type (single-float) aapp aaqq aatmax aatmin big big1 cond_ok condr1
      condr2 entra entrat epsln maxprj scalem sconda sfmin small temp1
      uscal1 uscal2 xsc))
    (setf lsvec
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame jobu "U")
               (declare (ignore var-1))
               (when var-0 (setf jobu var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame jobu "F")
               (declare (ignore var-1))
               (when var-0 (setf jobu var-0))
               ret-val)))
    (setf jracc
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobv "J")
              (declare (ignore var-1))
              (when var-0 (setf jobv var-0))
              ret-val))
    (setf rsvec
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame jobv "V")
               (declare (ignore var-1))
```

```
            (when var-0 (setf jobv var-0))
            ret-val)
          jracc))
(setf rowpiv
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame joba "F")
            (declare (ignore var-1))
            (when var-0 (setf joba var-0))
            ret-val)
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame joba "G")
            (declare (ignore var-1))
            (when var-0 (setf joba var-0))
            ret-val)))
(setf l2rank
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame joba "R")
          (declare (ignore var-1))
          (when var-0 (setf joba var-0))
          ret-val))
(setf l2aber
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame joba "A")
          (declare (ignore var-1))
          (when var-0 (setf joba var-0))
          ret-val))
(setf errest
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame joba "E")
            (declare (ignore var-1))
            (when var-0 (setf joba var-0))
            ret-val)
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame joba "G")
            (declare (ignore var-1))
            (when var-0 (setf joba var-0))
            ret-val)))
(setf l2tran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobt "T")
          (declare (ignore var-1))
          (when var-0 (setf jobt var-0))
          ret-val))
(setf l2kill
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobr "R")
          (declare (ignore var-1))
          (when var-0 (setf jobr var-0))
```

```
          ret-val))
(setf defr
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobr "N")
          (declare (ignore var-1))
          (when var-0 (setf jobr var-0))
          ret-val))
(setf l2pert
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobp "P")
          (declare (ignore var-1))
          (when var-0 (setf jobp var-0))
          ret-val))
(cond
 ((not
   (or rowpiv l2rank l2aber errest
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame joba "C")
         (declare (ignore var-1))
         (when var-0 (setf joba var-0))
         ret-val)))
  (setf info -1))
 ((not
   (or lsvec
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobu "N")
         (declare (ignore var-1))
         (when var-0 (setf jobu var-0))
         ret-val)
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobu "W")
         (declare (ignore var-1))
         (when var-0 (setf jobu var-0))
         ret-val)))
  (setf info -2))
 ((or
   (not
    (or rsvec
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobv "N")
          (declare (ignore var-1))
          (when var-0 (setf jobv var-0))
          ret-val)
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobv "W")
          (declare (ignore var-1))
          (when var-0 (setf jobv var-0))
          ret-val)))
   (and jracc (not lsvec)))
  (setf info -3))
```

```
((not (or l2kill defr)) (setf info -4))
((not
  (or l2tran
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame jobt "N")
        (declare (ignore var-1))
        (when var-0 (setf jobt var-0))
        ret-val)))
 (setf info -5))
((not
  (or l2pert
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame jobp "N")
        (declare (ignore var-1))
        (when var-0 (setf jobp var-0))
        ret-val)))
 (setf info -6))
((< m 0) (setf info -7)) ((or (< n 0) (> n m)) (setf info -8))
((< lda m) (setf info -10)) ((and lsvec (< ldu m)) (setf info -13))
((and rsvec (< ldv n)) (setf info -14))
((or
  (and (not (or lsvec rsvec errest))
       (< lwork
          (max (the f2cl-lib:integer4 7)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 4 n) 1))
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))))
  (and (not (or lsvec rsvec)) errest
       (< lwork
          (max (the f2cl-lib:integer4 7)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 4 n)
                                      (f2cl-lib:int-mul n n)))
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))))
  (and lsvec (not rsvec)
       (< lwork
          (max (the f2cl-lib:integer4 7)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n))
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 4 n) 1)))))
  (and rsvec (not lsvec)
       (< lwork
          (max (the f2cl-lib:integer4 7)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n))
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 4 n) 1)))))
```

```
   (and lsvec rsvec (not jracc)
        (< lwork
           (max
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n))
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-mul 6 n)
                                   (f2cl-lib:int-mul 2 n n))))))
   (and lsvec rsvec jracc
        (< lwork
           (max
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n))
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-mul 4 n)
                                   (f2cl-lib:int-mul n n)))
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n n) 6))))))
  (setf info -17))
 (t (setf info 0)))
(cond
 ((/= info 0) (xerbla "SGEJSV" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= m 0) (= n 0))
    (go end_label))
(cond
 (lsvec (setf n1 n)
  (if (multiple-value-bind (ret-val var-0 var-1)
          (lsame jobu "F")
        (declare (ignore var-1))
        (when var-0 (setf jobu var-0))
        ret-val)
      (setf n1 m))))
(setf epsln (slamch "Epsilon"))
(setf sfmin (slamch "SafeMinimum"))
(setf small (/ sfmin epsln))
(setf big (slamch "O"))
(setf scalem
        (/ one
           (f2cl-lib:fsqrt
            (* (f2cl-lib:ffloat m) (f2cl-lib:ffloat n)))))
(setf noscal f2cl-lib:%true%)
(setf goscal f2cl-lib:%true%)
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (setf aapp zero)
    (setf aaqq one)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
          (slassq m
           (f2cl-lib:array-slice a-%data% single-float (1 p)
                                 ((1 lda) (1 *)) a-%offset%)
           1 aapp aaqq)
        (declare (ignore var-1 var-2))
        (when var-0 (setf m var-0))
        (when var-3 (setf aapp var-3))
        (when var-4 (setf aaqq var-4)))
      (cond
       ((> aapp big) (setf info -9)
        (xerbla "SGEJSV" (f2cl-lib:int-sub info)) (go end_label)))
      (setf aaqq (f2cl-lib:fsqrt aaqq))
      (cond
       ((and (< aapp (f2cl-lib:f2cl/ big aaqq)) noscal)
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                (* aapp aaqq)))
       (t (setf noscal f2cl-lib:%false%)
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                (* aapp (* aaqq scalem)))
        (cond
         (goscal (setf goscal f2cl-lib:%false%)
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (sscal (f2cl-lib:int-sub p 1) scalem sva 1)
            (declare (ignore var-0 var-2 var-3))
            (when var-1 (setf scalem var-1)))))))
   label1874))
(if noscal
    (setf scalem one))
(setf aapp zero)
(setf aaqq big)
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (setf aapp
            (max aapp
                 (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)))
    (if (/= (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%) zero)
        (setf aaqq
                (min aaqq
                     (f2cl-lib:fref sva-%data% (p) ((1 n))
                                    sva-%offset%))))
   label4781))
(cond
 ((= aapp zero)
  (if lsvec
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slaset "G" m n1 zero one u ldu)
        (declare (ignore var-0 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf n1 var-2))
```

```
        (when var-3 (setf zero var-3))
        (when var-4 (setf one var-4))
        (when var-6 (setf ldu var-6))))
  (if rsvec
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slaset "G" n n zero one v ldv)
        (declare (ignore var-0 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf zero var-3))
        (when var-4 (setf one var-4))
        (when var-6 (setf ldv var-6))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 lwork)) work-%offset%) one)
  (setf (f2cl-lib:fref work-%data% (2) ((1 lwork)) work-%offset%) one)
  (if errest
      (setf (f2cl-lib:fref work-%data% (3) ((1 lwork)) work-%offset%)
            one))
  (cond
   ((and lsvec rsvec)
    (setf (f2cl-lib:fref work-%data% (4) ((1 lwork)) work-%offset%)
          one)
    (setf (f2cl-lib:fref work-%data% (5) ((1 lwork)) work-%offset%)
          one)))
  (cond
   (l2tran
    (setf (f2cl-lib:fref work-%data% (6) ((1 lwork)) work-%offset%)
          zero)
    (setf (f2cl-lib:fref work-%data% (7) ((1 lwork)) work-%offset%)
          zero)))
  (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) 0)
  (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%) 0)
  (setf (f2cl-lib:fref iwork-%data% (3) ((1 *)) iwork-%offset%) 0)
  (go end_label)))
(setf warning 0)
(cond
 ((<= aaqq sfmin) (setf l2rank f2cl-lib:%true%)
  (setf l2kill f2cl-lib:%true%) (setf warning 1)))
(cond
 ((= n 1)
  (cond
   (lsvec
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl "G" 0 0
         (f2cl-lib:array-slice sva-%data% single-float (1) ((1 n))
                               sva-%offset%)
         scalem m 1
         (f2cl-lib:array-slice a-%data% single-float (1 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda ierr)
```

```
      (declare (ignore var-0 var-1 var-2 var-3 var-6 var-7))
      (when var-4 (setf scalem var-4))
      (when var-5 (setf m var-5))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slacpy "A" m 1 a lda u ldu)
      (declare (ignore var-0 var-2 var-3 var-5))
      (when var-1 (setf m var-1))
      (when var-4 (setf lda var-4))
      (when var-6 (setf ldu var-6)))
    (cond
     ((/= n1 n)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (sgeqrf m n u ldu work
           (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                                 ((1 lwork)) work-%offset%)
           (f2cl-lib:int-sub lwork n) ierr)
        (declare (ignore var-2 var-4 var-5 var-6))
        (when var-0 (setf m var-0))
        (when var-1 (setf n var-1))
        (when var-3 (setf ldu var-3))
        (when var-7 (setf ierr var-7)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (sorgqr m n1 1 u ldu work
           (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                                 ((1 lwork)) work-%offset%)
           (f2cl-lib:int-sub lwork n) ierr)
        (declare (ignore var-2 var-3 var-5 var-6 var-7))
        (when var-0 (setf m var-0))
        (when var-1 (setf n1 var-1))
        (when var-4 (setf ldu var-4))
        (when var-8 (setf ierr var-8)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (scopy m
           (f2cl-lib:array-slice a-%data% single-float (1 1)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice u-%data% single-float (1 1)
                                 ((1 ldu) (1 *)) u-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf m var-0)))))))
  (cond
   (rsvec
    (setf (f2cl-lib:fref v-%data% (1 1) ((1 ldv) (1 *)) v-%offset%)
          one)))
  (cond
```

```
   ((< (f2cl-lib:fref sva (1) ((1 n))) (* big scalem))
    (setf (f2cl-lib:fref sva-%data% (1) ((1 n)) sva-%offset%)
            (/ (f2cl-lib:fref sva-%data% (1) ((1 n)) sva-%offset%)
               scalem))
    (setf scalem one)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 lwork)) work-%offset%)
          (/ one scalem))
  (setf (f2cl-lib:fref work-%data% (2) ((1 lwork)) work-%offset%) one)
  (cond
   ((/= (f2cl-lib:fref sva (1) ((1 n))) zero)
    (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) 1)
    (cond
     ((>= (f2cl-lib:f2cl/ (f2cl-lib:fref sva (1) ((1 n))) scalem)
          sfmin)
      (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%) 1))
     (t
      (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%)
              0))))
   (t (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) 0)
    (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%) 0)))
  (if errest
      (setf (f2cl-lib:fref work-%data% (3) ((1 lwork)) work-%offset%)
              one))
  (cond
   ((and lsvec rsvec)
    (setf (f2cl-lib:fref work-%data% (4) ((1 lwork)) work-%offset%)
            one)
    (setf (f2cl-lib:fref work-%data% (5) ((1 lwork)) work-%offset%)
            one)))
  (cond
   (l2tran
    (setf (f2cl-lib:fref work-%data% (6) ((1 lwork)) work-%offset%)
            zero)
    (setf (f2cl-lib:fref work-%data% (7) ((1 lwork)) work-%offset%)
            zero)))
  (go end_label)))
(setf transp f2cl-lib:%false%)
(setf l2tran (and l2tran (= m n)))
(setf aatmax (- one))
(setf aatmin big)
(cond
 ((or rowpiv l2tran)
  (cond
   (l2tran
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p m) nil)
      (tagbody
        (setf xsc zero)
        (setf temp1 one)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
            (slassq n
             (f2cl-lib:array-slice a-%data% single-float (p 1)
                                   ((1 lda) (1 *)) a-%offset%)
             lda xsc temp1)
          (declare (ignore var-1))
          (when var-0 (setf n var-0))
          (when var-2 (setf lda var-2))
          (when var-3 (setf xsc var-3))
          (when var-4 (setf temp1 var-4)))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add m n p))
                             ((1 lwork)) work-%offset%)
                (* xsc scalem))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n p))
                             ((1 lwork)) work-%offset%)
                (* xsc (* scalem (f2cl-lib:fsqrt temp1))))
        (setf aatmax
                (max aatmax
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add n p))
                                    ((1 lwork)) work-%offset%)))
        (if (/=
             (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n p))
                            ((1 lwork)) work-%offset%)
             zero)
            (setf aatmin
                    (min aatmin
                         (f2cl-lib:fref work-%data%
                                        ((f2cl-lib:int-add n p))
                                        ((1 lwork)) work-%offset%))))
      label1950)))
  (t
   (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                 ((> p m) nil)
     (tagbody
       (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add m n p))
                            ((1 lwork)) work-%offset%)
               (* scalem
                  (abs
                   (f2cl-lib:fref a-%data%
                                  (p
                                   (multiple-value-bind
                                       (ret-val var-0 var-1 var-2)
                                       (isamax n
                                        (f2cl-lib:array-slice a
                                                              single-float
                                                              (p 1)
                                                              ((1 lda)
                                                               (1 *)))
                                        lda)
                                     (declare (ignore var-1))
```

```
                                        (when var-0 (setf n var-0))
                                        (when var-2 (setf lda var-2))
                                        ret-val))
                                    ((1 lda) (1 *)) a-%offset%))))
        (setf aatmax
                (max aatmax
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add m n p))
                                    ((1 lwork)) work-%offset%)))
        (setf aatmin
                (min aatmin
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add m n p))
                                    ((1 lwork)) work-%offset%)))
       label1904))))))
(setf entra zero)
(setf entrat zero)
(cond
 (l2tran (setf xsc zero) (setf temp1 one)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slassq n sva 1 xsc temp1)
    (declare (ignore var-1 var-2))
    (when var-0 (setf n var-0))
    (when var-3 (setf xsc var-3))
    (when var-4 (setf temp1 var-4)))
  (setf temp1 (/ one temp1)) (setf entra zero)
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (setf big1
              (*
               (expt
                (/ (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                   xsc)
                2)
               temp1))
      (if (/= big1 zero)
          (setf entra (+ entra (* big1 (f2cl-lib:alog big1)))))
     label1113))
  (setf entra (/ (- entra) (f2cl-lib:alog (f2cl-lib:ffloat n))))
  (setf entrat zero)
  (f2cl-lib:fdo (p (f2cl-lib:int-add n 1) (f2cl-lib:int-add p 1))
                ((> p (f2cl-lib:int-add n m)) nil)
    (tagbody
      (setf big1
              (*
               (expt
                (/
                 (f2cl-lib:fref work-%data% (p) ((1 lwork))
                                work-%offset%)
```

```
                 xsc)
                2)
               temp1))
        (if (/= big1 zero)
            (setf entrat (+ entrat (* big1 (f2cl-lib:alog big1)))))
       label1114))
    (setf entrat (/ (- entrat) (f2cl-lib:alog (f2cl-lib:ffloat m))))
    (setf transp (< entrat entra))
    (cond
     (transp
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
        (tagbody
          (f2cl-lib:fdo (q (f2cl-lib:int-add p 1) (f2cl-lib:int-add q 1))
                        ((> q n) nil)
            (tagbody
              (setf temp1
                      (f2cl-lib:fref a-%data% (q p) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (q p) ((1 lda) (1 *))
                                    a-%offset%)
                      (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                    a-%offset%)
                      temp1)
            label1116))
       label1115))
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add m n p))
                               ((1 lwork)) work-%offset%)
                  (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%))
          (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n p))
                                 ((1 lwork)) work-%offset%))
       label1117))
      (setf temp1 aapp) (setf aapp aatmax) (setf aatmax temp1)
      (setf temp1 aaqq) (setf aaqq aatmin) (setf aatmin temp1)
      (setf kill lsvec) (setf lsvec rsvec) (setf rsvec kill)
      (if lsvec
          (setf n1 n))
      (setf rowpiv f2cl-lib:%true%)))))
  (setf big1 (f2cl-lib:fsqrt big))
  (setf temp1 (f2cl-lib:fsqrt (/ big (f2cl-lib:ffloat n))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 aapp temp1 n 1 sva n ierr)
    (declare (ignore var-0 var-1 var-2 var-6 var-7))
```

```
  (when var-3 (setf aapp var-3))
  (when var-4 (setf temp1 var-4))
  (when var-5 (setf n var-5))
  (when var-8 (setf n var-8))
  (when var-9 (setf ierr var-9)))
(cond ((> aaqq (* aapp sfmin)) (setf aaqq (* (/ aaqq aapp) temp1)))
      (t (setf aaqq (/ (* aaqq temp1) aapp))))
(setf temp1 (* temp1 scalem))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (slascl "G" 0 0 aapp temp1 m n a lda ierr)
  (declare (ignore var-0 var-1 var-2 var-7))
  (when var-3 (setf aapp var-3))
  (when var-4 (setf temp1 var-4))
  (when var-5 (setf m var-5))
  (when var-6 (setf n var-6))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(setf uscal1 temp1)
(setf uscal2 aapp)
(cond (l2kill (setf xsc (f2cl-lib:fsqrt sfmin)))
      (t (setf xsc small)
       (cond
        ((and (< aaqq (f2cl-lib:fsqrt sfmin)) lsvec rsvec)
         (setf jracc f2cl-lib:%true%)))))
(cond
 ((< aaqq xsc)
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (cond
       ((< (f2cl-lib:fref sva (p) ((1 n))) xsc)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (slaset "A" m 1 zero zero
             (f2cl-lib:array-slice a-%data% single-float (1 p)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-2 var-5))
          (when var-1 (setf m var-1))
          (when var-3 (setf zero var-3))
          (when var-4 (setf zero var-4))
          (when var-6 (setf lda var-6)))
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                zero)))
     label700))))
(cond
 (rowpiv
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
```

```
      (tagbody
        (setf q
                (f2cl-lib:int-sub
                 (f2cl-lib:int-add
                  (isamax (f2cl-lib:int-add (f2cl-lib:int-sub m p) 1)
                   (f2cl-lib:array-slice work-%data% single-float
                                         ((+ m n p)) ((1 lwork))
                                         work-%offset%)
                   1)
                  p)
                 1))
        (setf (f2cl-lib:fref iwork-%data%
                             ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                p))
                             ((1 *)) iwork-%offset%)
                q)
        (cond
         ((/= p q)
          (setf temp1
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add m n p))
                                 ((1 lwork)) work-%offset%))
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add m n p))
                               ((1 lwork)) work-%offset%)
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add m n q))
                                 ((1 lwork)) work-%offset%))
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add m n q))
                               ((1 lwork)) work-%offset%)
                  temp1)))
       label1952))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaswp n a lda 1 (f2cl-lib:int-sub m 1)
         (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                               ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                               iwork-%offset%)
         1)
      (declare (ignore var-1 var-3 var-4 var-5 var-6))
      (when var-0 (setf n var-0))
      (when var-2 (setf lda var-2)))))
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (setf (f2cl-lib:fref iwork-%data% (p) ((1 *)) iwork-%offset%) 0)
     label1963))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (sgeqp3 m n a lda iwork work
       (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                             ((1 lwork)) work-%offset%)
       (f2cl-lib:int-sub lwork n) ierr)
    (declare (ignore var-2 var-4 var-5 var-6 var-7))
```

```
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-8 (setf ierr var-8)))
(setf nr 1)
(cond
 (l2aber
  (tagbody
    (setf temp1 (* (f2cl-lib:fsqrt (f2cl-lib:ffloat n)) epsln))
    (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (cond
         ((>= (abs (f2cl-lib:fref a (p p) ((1 lda) (1 *))))
              (* temp1 (abs (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))))
          (setf nr (f2cl-lib:int-add nr 1)))
         (t (go label3002)))
       label3001))
   label3002))
 (l2rank
  (tagbody
    (setf temp1 (f2cl-lib:fsqrt sfmin))
    (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (if (or
             (<
              (abs
               (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                              a-%offset%))
              (* epsln
                 (abs
                  (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-sub p 1)
                                  (f2cl-lib:int-sub p 1))
                                 ((1 lda) (1 *)) a-%offset%))))
             (<
              (abs
               (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                              a-%offset%))
              small)
             (and l2kill
                  (<
                   (abs
                    (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                                   a-%offset%))
                   temp1)))
            (go label3402))
        (setf nr (f2cl-lib:int-add nr 1))
       label3401))
```

```
   label3402))
 (t
  (tagbody
    (setf temp1 (f2cl-lib:fsqrt sfmin))
    (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (if (or
             (<
              (abs
               (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                              a-%offset%))
              small)
             (and l2kill
                  (<
                   (abs
                    (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                                   a-%offset%))
                   temp1)))
            (go label3302))
        (setf nr (f2cl-lib:int-add nr 1))
       label3301))
   label3302)))
(setf almort f2cl-lib:%false%)
(cond
 ((= nr n) (setf maxprj one)
  (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (setf temp1
              (/
               (abs
                (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                               a-%offset%))
               (f2cl-lib:fref sva-%data%
                              ((f2cl-lib:fref iwork (p) ((1 *))))
                              ((1 n)) sva-%offset%)))
      (setf maxprj (min maxprj temp1))
     label3051))
  (if (>= (expt maxprj 2) (- one (* (f2cl-lib:ffloat n) epsln)))
      (setf almort f2cl-lib:%true%))))
(setf sconda (- one))
(setf condr1 (- one))
(setf condr2 (- one))
(cond
 (errest
  (cond
   ((= n nr)
    (cond
     (rsvec
```

```
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "U" n n a lda v ldv)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldv var-6)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (setf temp1
            (f2cl-lib:fref sva-%data%
                           ((f2cl-lib:fref iwork (p) ((1 *))))
                           ((1 n)) sva-%offset%))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal p (/ one temp1)
         (f2cl-lib:array-slice v-%data% single-float (1 p)
                               ((1 ldv) (1 *)) v-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf p var-0)))
   label3053))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (spocon "U" n v ldv one temp1
     (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                           ((+ (f2cl-lib:int-mul 2 n) m 1))
                           ((1 *)) iwork-%offset%)
     ierr)
  (declare (ignore var-0 var-2 var-6 var-7))
  (when var-1 (setf n var-1))
  (when var-3 (setf ldv var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf temp1 var-5))
  (when var-8 (setf ierr var-8))))
(lsvec
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slacpy "U" n n a lda u ldu)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldu var-6)))
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p n) nil)
   (tagbody
     (setf temp1
             (f2cl-lib:fref sva-%data%
```

```
                              ((f2cl-lib:fref iwork (p) ((1 *))))
                              ((1 n)) sva-%offset%))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal p (/ one temp1)
             (f2cl-lib:array-slice u-%data% single-float (1 p)
                                   ((1 ldu) (1 *)) u-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3))
          (when var-0 (setf p var-0)))
       label3054))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (spocon "U" n u ldu one temp1
         (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                              ((1 lwork)) work-%offset%)
         (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                              ((+ (f2cl-lib:int-mul 2 n) m 1))
                              ((1 *)) iwork-%offset%)
         ierr)
      (declare (ignore var-0 var-2 var-6 var-7))
      (when var-1 (setf n var-1))
      (when var-3 (setf ldu var-3))
      (when var-4 (setf one var-4))
      (when var-5 (setf temp1 var-5))
      (when var-8 (setf ierr var-8))))
   (t
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slacpy "U" n n a lda
         (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                              ((1 lwork)) work-%offset%)
         n)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-6 (setf n var-6)))
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (setf temp1
                (f2cl-lib:fref sva-%data%
                              ((f2cl-lib:fref iwork (p) ((1 *))))
                              ((1 n)) sva-%offset%))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal p (/ one temp1)
             (f2cl-lib:array-slice work-%data% single-float
                                   ((+ n
                                       (f2cl-lib:int-mul
                                        (f2cl-lib:int-add p
                                                          (f2cl-lib:int-sub
```

```
                                                                  1))
                                                 n)
                                                1))
                                          ((1 lwork)) work-%offset%)
                  1)
               (declare (ignore var-1 var-2 var-3))
               (when var-0 (setf p var-0)))
            label3052))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (spocon "U" n
             (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                                   ((1 lwork)) work-%offset%)
             n one temp1
             (f2cl-lib:array-slice work-%data% single-float
                                   ((+ n (f2cl-lib:int-mul n n) 1))
                                   ((1 lwork)) work-%offset%)
             (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                                   ((+ (f2cl-lib:int-mul 2 n) m 1))
                                   ((1 *)) iwork-%offset%)
             ierr)
          (declare (ignore var-0 var-2 var-6 var-7))
          (when var-1 (setf n var-1))
          (when var-3 (setf n var-3))
          (when var-4 (setf one var-4))
          (when var-5 (setf temp1 var-5))
          (when var-8 (setf ierr var-8)))))
      (setf sconda (/ one (f2cl-lib:fsqrt temp1))))
     (t (setf sconda (- one))))))
  (setf l2pert
          (and l2pert
               (>
                (abs
                 (/
                  (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *))
                                 a-%offset%)
                  (f2cl-lib:fref a-%data% (nr nr) ((1 lda) (1 *))
                                 a-%offset%)))
                (f2cl-lib:fsqrt big1))))
  (cond
   ((not (or rsvec lsvec))
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p
                      (min
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                       (the f2cl-lib:integer4 nr)))
                   nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
            (scopy (f2cl-lib:int-sub n p)
             (f2cl-lib:array-slice a-%data% single-float
                                   (p (f2cl-lib:int-add p 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice a-%data% single-float ((+ p 1) p)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
           (declare (ignore var-0 var-1 var-3 var-4))
           (when var-2 (setf lda var-2)))
         label1946))
       (cond
        ((not almort)
         (cond
          (l2pert (setf xsc (/ epsln (f2cl-lib:ffloat n)))
           (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                         ((> q nr) nil)
             (tagbody
               (setf temp1
                       (* xsc
                          (abs
                           (f2cl-lib:fref a-%data% (q q) ((1 lda) (1 *))
                                          a-%offset%))))
               (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                             ((> p n) nil)
                 (tagbody
                   (if (or
                        (and (> p q)
                             (<=
                              (abs
                               (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                              a-%offset%))
                              temp1))
                        (< p q))
                       (setf (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                            a-%offset%)
                             (f2cl-lib:sign temp1
                                            (f2cl-lib:fref a-%data% (p q)
                                                           ((1 lda) (1 *))
                                                           a-%offset%))))
                   label4949))
               label4947)))
          (t
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
               (slaset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
                zero zero
                (f2cl-lib:array-slice a-%data% single-float (1 2)
                                      ((1 lda) (1 *)) a-%offset%)
                lda)
             (declare (ignore var-0 var-1 var-2 var-5))
```

```
        (when var-3 (setf zero var-3))
        (when var-4 (setf zero var-4))
        (when var-6 (setf lda var-6)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (sgeqrf n nr a lda work
         (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                               ((1 lwork)) work-%offset%)
         (f2cl-lib:int-sub lwork n) ierr)
      (declare (ignore var-2 var-4 var-5 var-6))
      (when var-0 (setf n var-0))
      (when var-1 (setf nr var-1))
      (when var-3 (setf lda var-3))
      (when var-7 (setf ierr var-7)))
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p (f2cl-lib:int-add nr (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (scopy (f2cl-lib:int-sub nr p)
             (f2cl-lib:array-slice a-%data% single-float
                                   (p (f2cl-lib:int-add p 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice a-%data% single-float ((+ p 1) p)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf lda var-2)))
       label1948))))
  (cond
   (l2pert (setf xsc (/ epsln (f2cl-lib:ffloat n)))
    (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                  ((> q nr) nil)
      (tagbody
        (setf temp1
                (* xsc
                   (abs
                    (f2cl-lib:fref a-%data% (q q) ((1 lda) (1 *))
                                   a-%offset%))))
        (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                      ((> p nr) nil)
          (tagbody
            (if (or
                 (and (> p q)
                      (<=
                       (abs
                        (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                       a-%offset%))
                       temp1))
                 (< p q))
```

```
                (setf (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                     a-%offset%)
                      (f2cl-lib:sign temp1
                                     (f2cl-lib:fref a-%data% (p q)
                                                    ((1 lda) (1 *))
                                                    a-%offset%))))
          label1949))
      label1947)))
  (t
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slaset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
        zero zero
        (f2cl-lib:array-slice a-%data% single-float (1 2)
                              ((1 lda) (1 *)) a-%offset%)
        lda)
     (declare (ignore var-0 var-1 var-2 var-5))
     (when var-3 (setf zero var-3))
     (when var-4 (setf zero var-4))
     (when var-6 (setf lda var-6)))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13)
     (sgesvj "L" "NoU" "NoV" nr nr a lda sva n v ldv work lwork info)
   (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11))
   (when var-3 (setf nr var-3))
   (when var-4 (setf nr var-4))
   (when var-6 (setf lda var-6))
   (when var-8 (setf n var-8))
   (when var-10 (setf ldv var-10))
   (when var-12 (setf lwork var-12))
   (when var-13 (setf info var-13)))
 (setf scalem
         (f2cl-lib:fref work-%data% (1) ((1 lwork)) work-%offset%))
 (setf numrank
         (f2cl-lib:nint
          (f2cl-lib:fref work-%data% (2) ((1 lwork)) work-%offset%))))
((and rsvec (not lsvec))
 (cond
  (almort
   (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                 ((> p nr) nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (scopy (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
            (f2cl-lib:array-slice a-%data% single-float (p p)
                                  ((1 lda) (1 *)) a-%offset%)
            lda
            (f2cl-lib:array-slice v-%data% single-float (p p)
                                  ((1 ldv) (1 *)) v-%offset%)
            1)
```

```
        (declare (ignore var-0 var-1 var-3 var-4))
        (when var-2 (setf lda var-2)))
    label1998))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
       zero zero
       (f2cl-lib:array-slice v-%data% single-float (1 2)
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (sgesvj "L" "U" "N" n nr v ldv sva nr a lda work lwork info)
    (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11))
    (when var-3 (setf n var-3))
    (when var-4 (setf nr var-4))
    (when var-6 (setf ldv var-6))
    (when var-8 (setf nr var-8))
    (when var-10 (setf lda var-10))
    (when var-12 (setf lwork var-12))
    (when var-13 (setf info var-13)))
  (setf scalem
          (f2cl-lib:fref work-%data% (1) ((1 lwork)) work-%offset%))
  (setf numrank
          (f2cl-lib:nint
           (f2cl-lib:fref work-%data% (2) ((1 lwork))
                          work-%offset%))))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "Lower" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
       zero zero
       (f2cl-lib:array-slice a-%data% single-float (2 1)
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf lda var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (sgelqf nr n a lda work
       (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                             ((1 lwork)) work-%offset%)
       (f2cl-lib:int-sub lwork n) ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (when var-0 (setf nr var-0))
```

```
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-7 (setf ierr var-7)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "Lower" nr nr a lda v ldv)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf nr var-1))
  (when var-2 (setf nr var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldv var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
     zero zero
     (f2cl-lib:array-slice v-%data% single-float (1 2)
                           ((1 ldv) (1 *)) v-%offset%)
     ldv)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldv var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (sgeqrf nr nr v ldv
     (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (when var-0 (setf nr var-0))
  (when var-1 (setf nr var-1))
  (when var-3 (setf ldv var-3))
  (when var-7 (setf ierr var-7)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
         (f2cl-lib:array-slice v-%data% single-float (p p)
                               ((1 ldv) (1 *)) v-%offset%)
         ldv
         (f2cl-lib:array-slice v-%data% single-float (p p)
                               ((1 ldv) (1 *)) v-%offset%)
         1)
      (declare (ignore var-0 var-1 var-3 var-4))
      (when var-2 (setf ldv var-2)))
   label8998))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
```

```
      zero zero
      (f2cl-lib:array-slice v-%data% single-float (1 2)
                            ((1 ldv) (1 *)) v-%offset%)
      ldv)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldv var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sgesvj "Lower" "U" "N" nr nr v ldv sva nr u ldu
     (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork n) info)
  (declare
   (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12))
  (when var-3 (setf nr var-3))
  (when var-4 (setf nr var-4))
  (when var-6 (setf ldv var-6))
  (when var-8 (setf nr var-8))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf info var-13)))
(setf scalem
        (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 1))
                       ((1 lwork)) work-%offset%))
(setf numrank
        (f2cl-lib:nint
         (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 2))
                        ((1 lwork)) work-%offset%)))
(cond
 ((< nr n)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "A" (f2cl-lib:int-sub n nr) nr zero zero
       (f2cl-lib:array-slice v-%data% single-float ((+ nr 1) 1)
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "A" nr (f2cl-lib:int-sub n nr) zero zero
       (f2cl-lib:array-slice v-%data% single-float
                             (1 (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-2 var-5))
    (when var-1 (setf nr var-1))
```

```
        (when var-3 (setf zero var-3))
        (when var-4 (setf zero var-4))
        (when var-6 (setf ldv var-6)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slaset "A" (f2cl-lib:int-sub n nr) (f2cl-lib:int-sub n nr)
           zero one
           (f2cl-lib:array-slice v-%data% single-float
                                 ((+ nr 1) (f2cl-lib:int-add nr 1))
                                 ((1 ldv) (1 *)) v-%offset%)
           ldv)
        (declare (ignore var-0 var-1 var-2 var-5))
        (when var-3 (setf zero var-3))
        (when var-4 (setf one var-4))
        (when var-6 (setf ldv var-6)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (sormlq "Left" "Transpose" n n nr a lda work v ldv
         (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                               ((1 lwork)) work-%offset%)
         (f2cl-lib:int-sub lwork n) ierr)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf n var-2))
      (when var-3 (setf n var-3))
      (when var-4 (setf nr var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldv var-9))
      (when var-12 (setf ierr var-12)))))
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (scopy n
           (f2cl-lib:array-slice v-%data% single-float (p 1)
                                 ((1 ldv) (1 *)) v-%offset%)
           ldv
           (f2cl-lib:array-slice a-%data% single-float
                                 ((f2cl-lib:fref iwork (p) ((1 *))) 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-1 var-3))
        (when var-0 (setf n var-0))
        (when var-2 (setf ldv var-2))
        (when var-4 (setf lda var-4)))
     label8991))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "All" n n a lda v ldv)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
```

```
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldv var-6)))
  (cond
   (transp
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slacpy "All" n n v ldv u ldu)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf ldv var-4))
      (when var-6 (setf ldu var-6))))))
  ((and lsvec (not rsvec))
   (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                 ((> p nr) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (scopy (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
           (f2cl-lib:array-slice a-%data% single-float (p p)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice u-%data% single-float (p p)
                                 ((1 ldu) (1 *)) u-%offset%)
           1)
        (declare (ignore var-0 var-1 var-3 var-4))
        (when var-2 (setf lda var-2)))
     label1965))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
       zero zero
       (f2cl-lib:array-slice u-%data% single-float (1 2)
                             ((1 ldu) (1 *)) u-%offset%)
       ldu)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldu var-6)))
   (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (sgeqrf n nr u ldu
       (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                             ((1 lwork)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float
                             ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 lwork))
                             work-%offset%)
       (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (when var-0 (setf n var-0))
    (when var-1 (setf nr var-1))
    (when var-3 (setf ldu var-3))
    (when var-7 (setf ierr var-7)))
```

```
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p (f2cl-lib:int-add nr (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy (f2cl-lib:int-sub nr p)
         (f2cl-lib:array-slice u-%data% single-float
                               (p (f2cl-lib:int-add p 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu
         (f2cl-lib:array-slice u-%data% single-float ((+ p 1) p)
                               ((1 ldu) (1 *)) u-%offset%)
         1)
      (declare (ignore var-0 var-1 var-3 var-4))
      (when var-2 (setf ldu var-2)))
   label1967))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
     zero zero
     (f2cl-lib:array-slice u-%data% single-float (1 2)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sgesvj "Lower" "U" "N" nr nr u ldu sva nr a lda
     (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork n) info)
  (declare
   (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12))
  (when var-3 (setf nr var-3))
  (when var-4 (setf nr var-4))
  (when var-6 (setf ldu var-6))
  (when var-8 (setf nr var-8))
  (when var-10 (setf lda var-10))
  (when var-13 (setf info var-13)))
(setf scalem
        (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 1))
                       ((1 lwork)) work-%offset%))
(setf numrank
        (f2cl-lib:nint
         (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 2))
                        ((1 lwork)) work-%offset%)))
(cond
 ((< nr m)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
        (slaset "A" (f2cl-lib:int-sub m nr) nr zero zero
         (f2cl-lib:array-slice u-%data% single-float ((+ nr 1) 1)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf nr var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldu var-6)))
    (cond
     ((< nr n1)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slaset "A" nr (f2cl-lib:int-sub n1 nr) zero zero
           (f2cl-lib:array-slice u-%data% single-float
                                 (1 (f2cl-lib:int-add nr 1))
                                 ((1 ldu) (1 *)) u-%offset%)
           ldu)
        (declare (ignore var-0 var-2 var-5))
        (when var-1 (setf nr var-1))
        (when var-3 (setf zero var-3))
        (when var-4 (setf zero var-4))
        (when var-6 (setf ldu var-6)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slaset "A" (f2cl-lib:int-sub m nr) (f2cl-lib:int-sub n1 nr)
           zero one
           (f2cl-lib:array-slice u-%data% single-float
                                 ((+ nr 1) (f2cl-lib:int-add nr 1))
                                 ((1 ldu) (1 *)) u-%offset%)
           ldu)
        (declare (ignore var-0 var-1 var-2 var-5))
        (when var-3 (setf zero var-3))
        (when var-4 (setf one var-4))
        (when var-6 (setf ldu var-6)))))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sormqr "Left" "No Tr" m n1 n a lda work u ldu
       (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                             ((1 lwork)) work-%offset%)
       (f2cl-lib:int-sub lwork n) ierr)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n1 var-3))
    (when var-4 (setf n var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldu var-9))
    (when var-12 (setf ierr var-12)))
  (if rowpiv
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slaswp n1 u ldu 1 (f2cl-lib:int-sub m 1)
```

```
            (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                                  ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                                  iwork-%offset%)
            -1)
          (declare (ignore var-1 var-3 var-4 var-5 var-6))
          (when var-0 (setf n1 var-0))
          (when var-2 (setf ldu var-2))))
   (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                 ((> p n1) nil)
     (tagbody
       (setf xsc
               (/ one
                  (multiple-value-bind (ret-val var-0 var-1 var-2)
                      (snrm2 m
                       (f2cl-lib:array-slice u-%data% single-float (1 p)
                                             ((1 ldu) (1 *)) u-%offset%)
                       1)
                    (declare (ignore var-1 var-2))
                    (when var-0 (setf m var-0))
                    ret-val)))
       (multiple-value-bind (var-0 var-1 var-2 var-3)
           (sscal m xsc
            (f2cl-lib:array-slice u-%data% single-float (1 p)
                                  ((1 ldu) (1 *)) u-%offset%)
            1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf xsc var-1)))
     label1974))
   (cond
    (transp
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (slacpy "All" n n u ldu v ldv)
       (declare (ignore var-0 var-3 var-5))
       (when var-1 (setf n var-1))
       (when var-2 (setf n var-2))
       (when var-4 (setf ldu var-4))
       (when var-6 (setf ldv var-6))))))
  (t
   (cond
    ((not jracc)
     (cond
      ((not almort)
       (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                     ((> p nr) nil)
         (tagbody
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
               (scopy (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
                (f2cl-lib:array-slice a-%data% single-float (p p)
                                      ((1 lda) (1 *)) a-%offset%)
```

```
            lda
            (f2cl-lib:array-slice v-%data% single-float (p p)
                                  ((1 ldv) (1 *)) v-%offset%)
            1)
        (declare (ignore var-0 var-1 var-3 var-4))
        (when var-2 (setf lda var-2)))
    label1968))
  (cond
   (l2pert (setf xsc (f2cl-lib:fsqrt small))
    (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                  ((> q nr) nil)
      (tagbody
        (setf temp1
                (* xsc
                   (abs
                    (f2cl-lib:fref v-%data% (q q) ((1 ldv) (1 *))
                                   v-%offset%))))
        (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                      ((> p n) nil)
          (tagbody
            (if (or
                 (and (> p q)
                      (<=
                       (abs
                        (f2cl-lib:fref v-%data% (p q)
                                       ((1 ldv) (1 *)) v-%offset%))
                       temp1))
                 (< p q))
                (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                     v-%offset%)
                      (f2cl-lib:sign temp1
                                     (f2cl-lib:fref v-%data%
                                                    (p q)
                                                    ((1 ldv)
                                                     (1 *))
                                                    v-%offset%))))
            (if (< p q)
                (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                     v-%offset%)
                      (-
                       (f2cl-lib:fref v-%data% (p q)
                                      ((1 ldv) (1 *))
                                      v-%offset%))))
            label2968))
        label2969)))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
         zero zero
```

```
        (f2cl-lib:array-slice v-%data% single-float (1 2)
                              ((1 ldv) (1 *)) v-%offset%)
        ldv)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldv var-6)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" nr nr v ldv
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) work-%offset%)
     nr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf nr var-1))
  (when var-2 (setf nr var-2))
  (when var-4 (setf ldv var-4))
  (when var-6 (setf nr var-6)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (setf temp1
            (snrm2 (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
             (f2cl-lib:array-slice work-%data% single-float
                                   ((+ (f2cl-lib:int-mul 2 n)
                                       (f2cl-lib:int-mul
                                        (f2cl-lib:int-add p
                                                          (f2cl-lib:int-sub
                                                           1))
                                        nr)
                                       p))
                                   ((1 lwork)) work-%offset%)
             1))
    (sscal (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
     (/ one temp1)
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul
                                (f2cl-lib:int-add p
                                                  (f2cl-lib:int-sub
                                                   1))
                                nr)
                               p))
                           ((1 lwork)) work-%offset%)
     1)
   label3950))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (spocon "Lower" nr
     (f2cl-lib:array-slice work-%data% single-float
```

```
                              ((+ (f2cl-lib:int-mul 2 n) 1))
                              ((1 lwork)) work-%offset%)
        nr one temp1
        (f2cl-lib:array-slice work-%data% single-float
                              ((+ (f2cl-lib:int-mul 2 n)
                                  (f2cl-lib:int-mul nr nr) 1))
                              ((1 lwork)) work-%offset%)
        (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                              ((+ m (f2cl-lib:int-mul 2 n) 1))
                              ((1 *)) iwork-%offset%)
        ierr)
    (declare (ignore var-0 var-2 var-6 var-7))
    (when var-1 (setf nr var-1))
    (when var-3 (setf nr var-3))
    (when var-4 (setf one var-4))
    (when var-5 (setf temp1 var-5))
    (when var-8 (setf ierr var-8)))
  (setf condr1 (/ one (f2cl-lib:fsqrt temp1)))
  (setf cond_ok (f2cl-lib:fsqrt (f2cl-lib:ffloat nr)))
  (cond
   ((< condr1 cond_ok)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (sgeqrf n nr v ldv
         (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                               ((1 lwork)) work-%offset%)
         (f2cl-lib:array-slice work-%data% single-float
                               ((+ (f2cl-lib:int-mul 2 n) 1))
                               ((1 lwork)) work-%offset%)
         (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) ierr)
      (declare (ignore var-2 var-4 var-5 var-6))
      (when var-0 (setf n var-0))
      (when var-1 (setf nr var-1))
      (when var-3 (setf ldv var-3))
      (when var-7 (setf ierr var-7)))
    (cond
     (l2pert (setf xsc (/ (f2cl-lib:fsqrt small) epsln))
      (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                    ((> p nr) nil)
        (tagbody
          (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                        ((> q
                            (f2cl-lib:int-add p
                                              (f2cl-lib:int-sub
                                               1)))
                         nil)
            (tagbody
              (setf temp1
                      (* xsc
                         (min
```

```
                        (abs
                         (f2cl-lib:fref v-%data% (p p)
                                        ((1 ldv) (1 *))
                                        v-%offset%))
                        (abs
                         (f2cl-lib:fref v-%data% (q q)
                                        ((1 ldv) (1 *))
                                        v-%offset%)))))
            (if (<=
                 (abs
                  (f2cl-lib:fref v-%data% (q p) ((1 ldv) (1 *))
                                 v-%offset%))
                 temp1)
                (setf (f2cl-lib:fref v-%data% (q p)
                                     ((1 ldv) (1 *)) v-%offset%)
                      (f2cl-lib:sign temp1
                                     (f2cl-lib:fref v-%data%
                                                    (q p)
                                                    ((1 ldv)
                                                     (1 *))
                                                    v-%offset%))))
            label3958))
        label3959))))
    (if (/= nr n)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (slacpy "A" n nr v ldv
             (f2cl-lib:array-slice work-%data% single-float
                                   ((+ (f2cl-lib:int-mul 2 n) 1))
                                   ((1 lwork)) work-%offset%)
             n)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf nr var-2))
          (when var-4 (setf ldv var-4))
          (when var-6 (setf n var-6))))
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p (f2cl-lib:int-add nr (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (scopy (f2cl-lib:int-sub nr p)
             (f2cl-lib:array-slice v-%data% single-float
                                   (p (f2cl-lib:int-add p 1))
                                   ((1 ldv) (1 *)) v-%offset%)
             ldv
             (f2cl-lib:array-slice v-%data% single-float
                                   ((+ p 1) p) ((1 ldv) (1 *))
                                   v-%offset%)
             1)
```

```
        (declare (ignore var-0 var-1 var-3 var-4))
        (when var-2 (setf ldv var-2)))
    label1969))
 (setf condr2 condr1))
(t
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p nr) nil)
   (tagbody
     (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add n p))
                          ((1 *)) iwork-%offset%)
           0)
    label3003))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (sgeqp3 n nr v ldv
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                            ((+ n 1)) ((1 *)) iwork-%offset%)
      (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                            ((1 lwork)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float
                            ((+ (f2cl-lib:int-mul 2 n) 1))
                            ((1 lwork)) work-%offset%)
      (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) ierr)
   (declare (ignore var-2 var-4 var-5 var-6 var-7))
   (when var-0 (setf n var-0))
   (when var-1 (setf nr var-1))
   (when var-3 (setf ldv var-3))
   (when var-8 (setf ierr var-8)))
 (cond
  (l2pert (setf xsc (f2cl-lib:fsqrt small))
   (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                 ((> p nr) nil)
     (tagbody
       (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                     ((> q
                         (f2cl-lib:int-add p
                                           (f2cl-lib:int-sub
                                            1)))
                      nil)
         (tagbody
           (setf temp1
                   (* xsc
                      (min
                       (abs
                        (f2cl-lib:fref v-%data% (p p)
                                       ((1 ldv) (1 *))
                                       v-%offset%))
                       (abs
                        (f2cl-lib:fref v-%data% (q q)
                                       ((1 ldv) (1 *))
```

```
                                        v-%offset%)))))
            (if (<=
                 (abs
                  (f2cl-lib:fref v-%data% (q p) ((1 ldv) (1 *))
                                 v-%offset%))
                 temp1)
                (setf (f2cl-lib:fref v-%data% (q p)
                                     ((1 ldv) (1 *)) v-%offset%)
                      (f2cl-lib:sign temp1
                                     (f2cl-lib:fref v-%data%
                                                    (q p)
                                                    ((1 ldv)
                                                     (1 *))
                                                    v-%offset%))))
        label3968))
    label3969))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "A" n nr v ldv
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) work-%offset%)
     n)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nr var-2))
  (when var-4 (setf ldv var-4))
  (when var-6 (setf n var-6)))
(cond
 (l2pert (setf xsc (f2cl-lib:fsqrt small))
  (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                ((> p nr) nil)
    (tagbody
      (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                    ((> q
                        (f2cl-lib:int-add p
                                          (f2cl-lib:int-sub
                                           1)))
                     nil)
        (tagbody
          (setf temp1
                  (* xsc
                     (min
                      (abs
                       (f2cl-lib:fref v-%data% (p p)
                                      ((1 ldv) (1 *))
                                      v-%offset%))
                      (abs
                       (f2cl-lib:fref v-%data% (q q)
                                      ((1 ldv) (1 *))
```

```
                                        v-%offset%)))))
                (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                     v-%offset%)
                        (-
                         (f2cl-lib:sign temp1
                                        (f2cl-lib:fref v-%data% (q p)
                                                       ((1 ldv)
                                                        (1 *))
                                                       v-%offset%))))
            label8971))
        label8970)))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (slaset "L" (f2cl-lib:int-sub nr 1)
          (f2cl-lib:int-sub nr 1) zero zero
          (f2cl-lib:array-slice v-%data% single-float (2 1)
                                ((1 ldv) (1 *)) v-%offset%)
          ldv)
       (declare (ignore var-0 var-1 var-2 var-5))
       (when var-3 (setf zero var-3))
       (when var-4 (setf zero var-4))
       (when var-6 (setf ldv var-6)))))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (sgelqf nr nr v ldv
        (f2cl-lib:array-slice work-%data% single-float
                              ((+ (f2cl-lib:int-mul 2 n)
                                  (f2cl-lib:int-mul n nr) 1))
                              ((1 lwork)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float
                              ((+ (f2cl-lib:int-mul 2 n)
                                  (f2cl-lib:int-mul n nr) nr 1))
                              ((1 lwork)) work-%offset%)
        (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                          (f2cl-lib:int-mul n nr) nr)
        ierr)
     (declare (ignore var-2 var-4 var-5 var-6))
     (when var-0 (setf nr var-0))
     (when var-1 (setf nr var-1))
     (when var-3 (setf ldv var-3))
     (when var-7 (setf ierr var-7)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slacpy "L" nr nr v ldv
        (f2cl-lib:array-slice work-%data% single-float
                              ((+ (f2cl-lib:int-mul 2 n)
                                  (f2cl-lib:int-mul n nr) nr 1))
                              ((1 lwork)) work-%offset%)
        nr)
```

```
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf nr var-1))
  (when var-2 (setf nr var-2))
  (when var-4 (setf ldv var-4))
  (when var-6 (setf nr var-6)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (setf temp1
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (snrm2 p
                 (f2cl-lib:array-slice work-%data% single-float
                                       ((+
                                         (f2cl-lib:int-mul 2 n)
                                         (f2cl-lib:int-mul n
                                                           nr)
                                         nr p))
                                       ((1 lwork))
                                       work-%offset%)
                 nr)
              (declare (ignore var-1))
              (when var-0 (setf p var-0))
              (when var-2 (setf nr var-2))
              ret-val))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal p (/ one temp1)
         (f2cl-lib:array-slice work-%data% single-float
                               ((+ (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n nr) nr
                                   p))
                               ((1 lwork)) work-%offset%)
         nr)
      (declare (ignore var-1 var-2))
      (when var-0 (setf p var-0))
      (when var-3 (setf nr var-3)))
   label4950))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (spocon "L" nr
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) nr 1))
                           ((1 lwork)) work-%offset%)
     nr one temp1
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) nr
                               (f2cl-lib:int-mul nr nr) 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
```

```
                                  ((+ m (f2cl-lib:int-mul 2 n) 1))
                                  ((1 *)) iwork-%offset%)
                ierr)
              (declare (ignore var-0 var-2 var-6 var-7))
              (when var-1 (setf nr var-1))
              (when var-3 (setf nr var-3))
              (when var-4 (setf one var-4))
              (when var-5 (setf temp1 var-5))
              (when var-8 (setf ierr var-8)))
            (setf condr2 (/ one (f2cl-lib:fsqrt temp1)))
            (cond
             ((>= condr2 cond_ok)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (slacpy "U" nr nr v ldv
                   (f2cl-lib:array-slice work-%data% single-float
                                         ((+ (f2cl-lib:int-mul 2 n) 1))
                                         ((1 lwork)) work-%offset%)
                   n)
                (declare (ignore var-0 var-3 var-5))
                (when var-1 (setf nr var-1))
                (when var-2 (setf nr var-2))
                (when var-4 (setf ldv var-4))
                (when var-6 (setf n var-6)))))))
          (cond
           (l2pert (setf xsc (f2cl-lib:fsqrt small))
            (f2cl-lib:fdo (q 2 (f2cl-lib:int-add q 1))
                          ((> q nr) nil)
              (tagbody
                (setf temp1
                        (* xsc
                           (f2cl-lib:fref v-%data% (q q) ((1 ldv) (1 *))
                                          v-%offset%)))
                (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                              ((> p
                                  (f2cl-lib:int-add q
                                                    (f2cl-lib:int-sub 1)))
                               nil)
                  (tagbody
                    (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                         v-%offset%)
                          (-
                           (f2cl-lib:sign temp1
                                          (f2cl-lib:fref v-%data% (p q)
                                                         ((1 ldv) (1 *))
                                                         v-%offset%))))
                    label4969))
                label4968)))
           (t
            (multiple-value-bind
```

```
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
         zero zero
         (f2cl-lib:array-slice v-%data% single-float (1 2)
                               ((1 ldv) (1 *)) v-%offset%)
         ldv)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldv var-6)))))
  (cond
   ((< condr1 cond_ok)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13)
        (sgesvj "L" "U" "N" nr nr v ldv sva nr u ldu
         (f2cl-lib:array-slice work-%data% single-float
                               ((+ (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n nr) nr 1))
                               ((1 lwork)) work-%offset%)
         (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                           (f2cl-lib:int-mul n nr) nr)
         info)
      (declare
       (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12))
      (when var-3 (setf nr var-3))
      (when var-4 (setf nr var-4))
      (when var-6 (setf ldv var-6))
      (when var-8 (setf nr var-8))
      (when var-10 (setf ldu var-10))
      (when var-13 (setf info var-13)))
    (setf scalem
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add
                             (f2cl-lib:int-mul 2 n)
                             (f2cl-lib:int-mul n nr) nr 1))
                           ((1 lwork)) work-%offset%))
    (setf numrank
            (f2cl-lib:nint
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add
                              (f2cl-lib:int-mul 2 n)
                              (f2cl-lib:int-mul n nr) nr 2))
                            ((1 lwork)) work-%offset%)))
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p nr) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (scopy nr
             (f2cl-lib:array-slice v-%data% single-float (1 p)
```

```
                                ((1 ldv) (1 *)) v-%offset%)
          1
          (f2cl-lib:array-slice u-%data% single-float (1 p)
                                ((1 ldu) (1 *)) u-%offset%)
          1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf nr var-0)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal nr
          (f2cl-lib:array-slice sva-%data% single-float (p)
                                ((1 n)) sva-%offset%)
          (f2cl-lib:array-slice v-%data% single-float (1 p)
                                ((1 ldv) (1 *)) v-%offset%)
          1)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf nr var-0)))
    label3970))
  (cond
   ((= nr n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10)
        (strsm "L" "U" "N" "N" nr nr one a lda v ldv)
      (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
      (when var-4 (setf nr var-4))
      (when var-5 (setf nr var-5))
      (when var-6 (setf one var-6))
      (when var-8 (setf lda var-8))
      (when var-10 (setf ldv var-10))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10)
        (strsm "L" "U" "T" "N" nr nr one
         (f2cl-lib:array-slice work-%data% single-float
                               ((+ (f2cl-lib:int-mul 2 n) 1))
                               ((1 lwork)) work-%offset%)
         n v ldv)
      (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
      (when var-4 (setf nr var-4))
      (when var-5 (setf nr var-5))
      (when var-6 (setf one var-6))
      (when var-8 (setf n var-8))
      (when var-10 (setf ldv var-10)))
    (cond
     ((< nr n)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slaset "A" (f2cl-lib:int-sub n nr) nr zero zero
           (f2cl-lib:array-slice v-%data% single-float
```

```
                                ((+ nr 1) 1) ((1 ldv) (1 *))
                                v-%offset%)
          ldv)
        (declare (ignore var-0 var-1 var-5))
        (when var-2 (setf nr var-2))
        (when var-3 (setf zero var-3))
        (when var-4 (setf zero var-4))
        (when var-6 (setf ldv var-6)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slaset "A" nr (f2cl-lib:int-sub n nr) zero zero
           (f2cl-lib:array-slice v-%data% single-float
                                (1 (f2cl-lib:int-add nr 1))
                                ((1 ldv) (1 *)) v-%offset%)
           ldv)
        (declare (ignore var-0 var-2 var-5))
        (when var-1 (setf nr var-1))
        (when var-3 (setf zero var-3))
        (when var-4 (setf zero var-4))
        (when var-6 (setf ldv var-6)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slaset "A" (f2cl-lib:int-sub n nr)
           (f2cl-lib:int-sub n nr) zero one
           (f2cl-lib:array-slice v-%data% single-float
                                ((+ nr 1)
                                 (f2cl-lib:int-add nr 1))
                                ((1 ldv) (1 *)) v-%offset%)
           ldv)
        (declare (ignore var-0 var-1 var-2 var-5))
        (when var-3 (setf zero var-3))
        (when var-4 (setf one var-4))
        (when var-6 (setf ldv var-6)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (sormqr "L" "N" n n nr
         (f2cl-lib:array-slice work-%data% single-float
                              ((+ (f2cl-lib:int-mul 2 n) 1))
                              ((1 lwork)) work-%offset%)
         n
         (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                              ((1 lwork)) work-%offset%)
         v ldv
         (f2cl-lib:array-slice work-%data% single-float
                              ((+ (f2cl-lib:int-mul 2 n)
                                  (f2cl-lib:int-mul n nr) nr 1))
                              ((1 lwork)) work-%offset%)
         (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                           (f2cl-lib:int-mul n nr) nr)
```

```
          ierr)
        (declare
         (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf n var-2))
        (when var-3 (setf n var-3))
        (when var-4 (setf nr var-4))
        (when var-6 (setf n var-6))
        (when var-9 (setf ldv var-9))
        (when var-12 (setf ierr var-12))))))
((< condr2 cond_ok)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12 var-13)
     (sgesvj "L" "U" "N" nr nr v ldv sva nr u ldu
      (f2cl-lib:array-slice work-%data% single-float
                            ((+ (f2cl-lib:int-mul 2 n)
                                (f2cl-lib:int-mul n nr) nr 1))
                            ((1 lwork)) work-%offset%)
      (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                        (f2cl-lib:int-mul n nr) nr)
      info)
   (declare
    (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12))
   (when var-3 (setf nr var-3))
   (when var-4 (setf nr var-4))
   (when var-6 (setf ldv var-6))
   (when var-8 (setf nr var-8))
   (when var-10 (setf ldu var-10))
   (when var-13 (setf info var-13)))
 (setf scalem
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add
                          (f2cl-lib:int-mul 2 n)
                          (f2cl-lib:int-mul n nr) nr 1))
                        ((1 lwork)) work-%offset%))
 (setf numrank
         (f2cl-lib:nint
          (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add
                           (f2cl-lib:int-mul 2 n)
                           (f2cl-lib:int-mul n nr) nr 2))
                         ((1 lwork)) work-%offset%)))
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p nr) nil)
   (tagbody
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (scopy nr
          (f2cl-lib:array-slice v-%data% single-float (1 p)
                                ((1 ldv) (1 *)) v-%offset%)
          1
```

```
          (f2cl-lib:array-slice u-%data% single-float (1 p)
                                ((1 ldu) (1 *)) u-%offset%)
          1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf nr var-0)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal nr
          (f2cl-lib:array-slice sva-%data% single-float (p)
                                ((1 n)) sva-%offset%)
          (f2cl-lib:array-slice u-%data% single-float (1 p)
                                ((1 ldu) (1 *)) u-%offset%)
          1)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf nr var-0)))
    label3870))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (strsm "L" "U" "N" "N" nr nr one
       (f2cl-lib:array-slice work-%data% single-float
                             ((+ (f2cl-lib:int-mul 2 n) 1))
                             ((1 lwork)) work-%offset%)
       n u ldu)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf nr var-4))
    (when var-5 (setf nr var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf n var-8))
    (when var-10 (setf ldu var-10)))
  (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                ((> q nr) nil)
    (tagbody
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p nr) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add
                                 (f2cl-lib:int-mul 2 n)
                                 (f2cl-lib:int-mul n nr) nr
                                 (f2cl-lib:fref iwork
                                                ((f2cl-lib:int-add
                                                  n p))
                                                ((1 *)))))
                               ((1 lwork)) work-%offset%)
                (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                               u-%offset%))
         label872))
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p nr) nil)
        (tagbody
```

```
            (setf (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                                  u-%offset%)
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n nr) nr p))
                                 ((1 lwork)) work-%offset%))
        label874))
    label873))
(cond
 ((< nr n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "A" (f2cl-lib:int-sub n nr) nr zero zero
       (f2cl-lib:array-slice v-%data% single-float ((+ nr 1) 1)
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "A" nr (f2cl-lib:int-sub n nr) zero zero
       (f2cl-lib:array-slice v-%data% single-float
                             (1 (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-2 var-5))
    (when var-1 (setf nr var-1))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "A" (f2cl-lib:int-sub n nr)
       (f2cl-lib:int-sub n nr) zero one
       (f2cl-lib:array-slice v-%data% single-float
                             ((+ nr 1) (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldv var-6)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (sormqr "L" "N" n n nr
```

```
      (f2cl-lib:array-slice work-%data% single-float
                            ((+ (f2cl-lib:int-mul 2 n) 1))
                            ((1 lwork)) work-%offset%)
      n
      (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                            ((1 lwork)) work-%offset%)
      v ldv
      (f2cl-lib:array-slice work-%data% single-float
                            ((+ (f2cl-lib:int-mul 2 n)
                                (f2cl-lib:int-mul n nr) nr 1))
                            ((1 lwork)) work-%offset%)
      (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                        (f2cl-lib:int-mul n nr) nr)
      ierr)
    (declare
     (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf nr var-4))
    (when var-6 (setf n var-6))
    (when var-9 (setf ldv var-9))
    (when var-12 (setf ierr var-12))))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12 var-13)
     (sgesvj "L" "U" "V" nr nr v ldv sva nr u ldu
      (f2cl-lib:array-slice work-%data% single-float
                            ((+ (f2cl-lib:int-mul 2 n)
                                (f2cl-lib:int-mul n nr) nr 1))
                            ((1 lwork)) work-%offset%)
      (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                        (f2cl-lib:int-mul n nr) nr)
      info)
    (declare
     (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12))
    (when var-3 (setf nr var-3))
    (when var-4 (setf nr var-4))
    (when var-6 (setf ldv var-6))
    (when var-8 (setf nr var-8))
    (when var-10 (setf ldu var-10))
    (when var-13 (setf info var-13)))
 (setf scalem
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add
                          (f2cl-lib:int-mul 2 n)
                          (f2cl-lib:int-mul n nr) nr 1))
                        ((1 lwork)) work-%offset%))
 (setf numrank
         (f2cl-lib:nint
```

```
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add
                             (f2cl-lib:int-mul 2 n)
                             (f2cl-lib:int-mul n nr) nr 2))
                           ((1 lwork)) work-%offset%)))
(cond
 ((< nr n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "A" (f2cl-lib:int-sub n nr) nr zero zero
       (f2cl-lib:array-slice v-%data% single-float ((+ nr 1) 1)
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "A" nr (f2cl-lib:int-sub n nr) zero zero
       (f2cl-lib:array-slice v-%data% single-float
                             (1 (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-2 var-5))
    (when var-1 (setf nr var-1))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "A" (f2cl-lib:int-sub n nr)
       (f2cl-lib:int-sub n nr) zero one
       (f2cl-lib:array-slice v-%data% single-float
                             ((+ nr 1) (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldv var-6)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (sormqr "L" "N" n n nr
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) work-%offset%)
     n
```

```
      (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                            ((1 lwork)) work-%offset%)
      v ldv
      (f2cl-lib:array-slice work-%data% single-float
                            ((+ (f2cl-lib:int-mul 2 n)
                                (f2cl-lib:int-mul n nr) nr 1))
                            ((1 lwork)) work-%offset%)
      (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                        (f2cl-lib:int-mul n nr) nr)
      ierr)
    (declare
     (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf nr var-4))
    (when var-6 (setf n var-6))
    (when var-9 (setf ldv var-9))
    (when var-12 (setf ierr var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (sormlq "L" "T" nr nr nr
       (f2cl-lib:array-slice work-%data% single-float
                             ((+ (f2cl-lib:int-mul 2 n) 1))
                             ((1 lwork)) work-%offset%)
       n
       (f2cl-lib:array-slice work-%data% single-float
                             ((+ (f2cl-lib:int-mul 2 n)
                                 (f2cl-lib:int-mul n nr) 1))
                             ((1 lwork)) work-%offset%)
       u ldu
       (f2cl-lib:array-slice work-%data% single-float
                             ((+ (f2cl-lib:int-mul 2 n)
                                 (f2cl-lib:int-mul n nr) nr 1))
                             ((1 lwork)) work-%offset%)
       (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                         (f2cl-lib:int-mul n nr) nr)
       ierr)
    (declare
     (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf nr var-2))
    (when var-3 (setf nr var-3))
    (when var-4 (setf nr var-4))
    (when var-6 (setf n var-6))
    (when var-9 (setf ldu var-9))
    (when var-12 (setf ierr var-12)))
  (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                ((> q nr) nil)
    (tagbody
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
```

```
                    ((> p nr) nil)
            (tagbody
              (setf (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-add
                                     (f2cl-lib:int-mul 2 n)
                                     (f2cl-lib:int-mul n nr) nr
                                     (f2cl-lib:fref iwork
                                                    ((f2cl-lib:int-add
                                                      n p))
                                                    ((1 *)))))
                                   ((1 lwork)) work-%offset%)
                    (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                                   u-%offset%))
             label772))
          (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                        ((> p nr) nil)
            (tagbody
              (setf (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                                   u-%offset%)
                    (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-add
                                     (f2cl-lib:int-mul 2 n)
                                     (f2cl-lib:int-mul n nr) nr p))
                                   ((1 lwork)) work-%offset%))
             label774))
         label773))))
(setf temp1 (* (f2cl-lib:fsqrt (f2cl-lib:ffloat n)) epsln))
(f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
              ((> q n) nil)
  (tagbody
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) nr
                               (f2cl-lib:fref iwork (p)
                                              ((1 *)))))
                             ((1 lwork)) work-%offset%)
              (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                             v-%offset%))
       label972))
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                             v-%offset%)
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add
```

```
                                  (f2cl-lib:int-mul 2 n)
                                  (f2cl-lib:int-mul n nr) nr p))
                                ((1 lwork)) work-%offset%))
        label973))
    (setf xsc
            (/ one
               (multiple-value-bind (ret-val var-0 var-1 var-2)
                   (snrm2 n
                    (f2cl-lib:array-slice v-%data% single-float
                                          (1 q) ((1 ldv) (1 *))
                                          v-%offset%)
                    1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf n var-0))
                  ret-val)))
    (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal n xsc
             (f2cl-lib:array-slice v-%data% single-float (1 q)
                                   ((1 ldv) (1 *)) v-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-1 (setf xsc var-1))))
   label1972))
(cond
 ((< nr m)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "A" (f2cl-lib:int-sub m nr) nr zero zero
       (f2cl-lib:array-slice u-%data% single-float ((+ nr 1) 1)
                             ((1 ldu) (1 *)) u-%offset%)
       ldu)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldu var-6)))
  (cond
   ((< nr n1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "A" nr (f2cl-lib:int-sub n1 nr) zero zero
         (f2cl-lib:array-slice u-%data% single-float
                               (1 (f2cl-lib:int-add nr 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-2 var-5))
      (when var-1 (setf nr var-1))
      (when var-3 (setf zero var-3))
```

```
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldu var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "A" (f2cl-lib:int-sub m nr)
         (f2cl-lib:int-sub n1 nr) zero one
         (f2cl-lib:array-slice u-%data% single-float
                               ((+ nr 1) (f2cl-lib:int-add nr 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldu var-6)))))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (sormqr "Left" "No_Tr" m n1 n a lda work u ldu
     (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n1 var-3))
  (when var-4 (setf n var-4))
  (when var-6 (setf lda var-6))
  (when var-9 (setf ldu var-9))
  (when var-12 (setf ierr var-12)))
(setf temp1 (* (f2cl-lib:fsqrt (f2cl-lib:ffloat m)) epsln))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (setf xsc
            (/ one
               (multiple-value-bind (ret-val var-0 var-1 var-2)
                   (snrm2 m
                    (f2cl-lib:array-slice u-%data% single-float
                                          (1 p) ((1 ldu) (1 *))
                                          u-%offset%)
                    1)
                 (declare (ignore var-1 var-2))
                 (when var-0 (setf m var-0))
                 ret-val)))
    (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal m xsc
             (f2cl-lib:array-slice u-%data% single-float (1 p)
                                   ((1 ldu) (1 *)) u-%offset%)
             1)
          (declare (ignore var-2 var-3))
```

```
            (when var-0 (setf m var-0))
            (when var-1 (setf xsc var-1))))
    label1973))
  (if rowpiv
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slaswp n1 u ldu 1 (f2cl-lib:int-sub m 1)
           (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                                 ((+ (f2cl-lib:int-mul 2 n) 1))
                                 ((1 *)) iwork-%offset%)
           -1)
        (declare (ignore var-1 var-3 var-4 var-5 var-6))
        (when var-0 (setf n1 var-0))
        (when var-2 (setf ldu var-2)))))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "Upper" n n a lda
       (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                             ((1 lwork)) work-%offset%)
       n)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf n var-6)))
  (cond
   (l2pert (setf xsc (f2cl-lib:fsqrt small))
    (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (setf temp1
                (* xsc
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add n
                                                     (f2cl-lib:int-mul
                                                      (f2cl-lib:int-sub
                                                       p 1)
                                                      n)
                                                     p))
                                  ((1 lwork)) work-%offset%)))
        (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                      ((> q
                          (f2cl-lib:int-add p
                                            (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add n
                                                    (f2cl-lib:int-mul
                                                     (f2cl-lib:int-sub
```

```
                                                        q 1)
                                                       n)
                                                      p))
                                  ((1 lwork)) work-%offset%)
                        (-
                         (f2cl-lib:sign temp1
                                        (f2cl-lib:fref work-%data%
                                                       ((f2cl-lib:int-add
                                                         n
                                                         (f2cl-lib:int-mul
                                                          (f2cl-lib:int-sub
                                                           p 1)
                                                          n)
                                                         q))
                                                       ((1 lwork))
                                                       work-%offset%))))
                 label5971))
             label5970)))
         (t
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (slaset "Lower" (f2cl-lib:int-sub n 1)
               (f2cl-lib:int-sub n 1) zero zero
               (f2cl-lib:array-slice work-%data% single-float ((+ n 2))
                                     ((1 lwork)) work-%offset%)
               n)
            (declare (ignore var-0 var-1 var-2 var-5))
            (when var-3 (setf zero var-3))
            (when var-4 (setf zero var-4))
            (when var-6 (setf n var-6)))))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11 var-12 var-13)
            (sgesvj "Upper" "U" "N" n n
             (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                                   ((1 lwork)) work-%offset%)
             n sva n u ldu
             (f2cl-lib:array-slice work-%data% single-float
                                   ((+ n (f2cl-lib:int-mul n n) 1))
                                   ((1 lwork)) work-%offset%)
             (f2cl-lib:int-sub lwork n (f2cl-lib:int-mul n n)) info)
          (declare
           (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12))
          (when var-3 (setf n var-3))
          (when var-4 (setf n var-4))
          (when var-6 (setf n var-6))
          (when var-8 (setf n var-8))
          (when var-10 (setf ldu var-10))
          (when var-13 (setf info var-13)))
        (setf scalem
```

```
        (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add n
                                          (f2cl-lib:int-mul n n)
                                          1))
                       ((1 lwork)) work-%offset%))
(setf numrank
        (f2cl-lib:nint
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add n
                                           (f2cl-lib:int-mul n n)
                                           2))
                        ((1 lwork)) work-%offset%)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy n
         (f2cl-lib:array-slice work-%data% single-float
                               ((+ n
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-add p
                                                      (f2cl-lib:int-sub
                                                       1))
                                    n)
                                   1))
                               ((1 lwork)) work-%offset%)
         1
         (f2cl-lib:array-slice u-%data% single-float (1 p)
                               ((1 ldu) (1 *)) u-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal n
         (f2cl-lib:array-slice sva-%data% single-float (p)
                               ((1 n)) sva-%offset%)
         (f2cl-lib:array-slice work-%data% single-float
                               ((+ n
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-add p
                                                      (f2cl-lib:int-sub
                                                       1))
                                    n)
                                   1))
                               ((1 lwork)) work-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf n var-0)))
   label6970))
(multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (strsm "Left" "Upper" "NoTrans" "No UD" n n one a lda
     (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     n)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf n var-10)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy n
         (f2cl-lib:array-slice work-%data% single-float ((+ n p))
                               ((1 lwork)) work-%offset%)
         n
         (f2cl-lib:array-slice v-%data% single-float
                               ((f2cl-lib:fref iwork (p) ((1 *)))
                                1)
                               ((1 ldv) (1 *)) v-%offset%)
         ldv)
      (declare (ignore var-1 var-3))
      (when var-0 (setf n var-0))
      (when var-2 (setf n var-2))
      (when var-4 (setf ldv var-4)))
   label6972))
(setf temp1 (* (f2cl-lib:fsqrt (f2cl-lib:ffloat n)) epsln))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (setf xsc
            (/ one
               (multiple-value-bind (ret-val var-0 var-1 var-2)
                   (snrm2 n
                    (f2cl-lib:array-slice v-%data% single-float
                                          (1 p) ((1 ldv) (1 *))
                                          v-%offset%)
                    1)
                 (declare (ignore var-1 var-2))
                 (when var-0 (setf n var-0))
                 ret-val)))
    (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal n xsc
             (f2cl-lib:array-slice v-%data% single-float (1 p)
                                   ((1 ldv) (1 *)) v-%offset%)
```

```
                1)
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf xsc var-1))))
    label6971))
 (cond
  ((< n m)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slaset "A" (f2cl-lib:int-sub m n) n zero zero
        (f2cl-lib:array-slice u-%data% single-float ((+ n 1) 1)
                              ((1 ldu) (1 *)) u-%offset%)
        ldu)
     (declare (ignore var-0 var-1 var-5))
     (when var-2 (setf n var-2))
     (when var-3 (setf zero var-3))
     (when var-4 (setf zero var-4))
     (when var-6 (setf ldu var-6)))
   (cond
    ((< n n1)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (slaset "A" n (f2cl-lib:int-sub n1 n) zero zero
          (f2cl-lib:array-slice u-%data% single-float
                                (1 (f2cl-lib:int-add n 1))
                                ((1 ldu) (1 *)) u-%offset%)
          ldu)
       (declare (ignore var-0 var-2 var-5))
       (when var-1 (setf n var-1))
       (when var-3 (setf zero var-3))
       (when var-4 (setf zero var-4))
       (when var-6 (setf ldu var-6)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (slaset "A" (f2cl-lib:int-sub m n)
          (f2cl-lib:int-sub n1 n) zero one
          (f2cl-lib:array-slice u-%data% single-float
                                ((+ n 1) (f2cl-lib:int-add n 1))
                                ((1 ldu) (1 *)) u-%offset%)
          ldu)
       (declare (ignore var-0 var-1 var-2 var-5))
       (when var-3 (setf zero var-3))
       (when var-4 (setf one var-4))
       (when var-6 (setf ldu var-6)))))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12)
     (sormqr "Left" "No Tr" m n1 n a lda work u ldu
      (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                            ((1 lwork)) work-%offset%)
```

```
          (f2cl-lib:int-sub lwork n) ierr)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf n1 var-3))
        (when var-4 (setf n var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldu var-9))
        (when var-12 (setf ierr var-12)))
      (setf temp1 (* (f2cl-lib:fsqrt (f2cl-lib:ffloat m)) epsln))
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p n1) nil)
        (tagbody
          (setf xsc
                  (/ one
                     (multiple-value-bind (ret-val var-0 var-1 var-2)
                         (snrm2 m
                          (f2cl-lib:array-slice u-%data% single-float
                                                (1 p) ((1 ldu) (1 *))
                                                u-%offset%)
                          1)
                       (declare (ignore var-1 var-2))
                       (when var-0 (setf m var-0))
                       ret-val)))
          (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
              (multiple-value-bind (var-0 var-1 var-2 var-3)
                  (sscal m xsc
                   (f2cl-lib:array-slice u-%data% single-float (1 p)
                                         ((1 ldu) (1 *)) u-%offset%)
                   1)
                (declare (ignore var-2 var-3))
                (when var-0 (setf m var-0))
                (when var-1 (setf xsc var-1))))
         label6973))
      (if rowpiv
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (slaswp n1 u ldu 1 (f2cl-lib:int-sub m 1)
               (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                                     ((+ (f2cl-lib:int-mul 2 n) 1))
                                     ((1 *)) iwork-%offset%)
               -1)
            (declare (ignore var-1 var-3 var-4 var-5 var-6))
            (when var-0 (setf n1 var-0))
            (when var-2 (setf ldu var-2)))))))
    (t
     (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                   ((> p nr) nil)
       (tagbody
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (scopy (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
```

```
            (f2cl-lib:array-slice a-%data% single-float (p p)
                                  ((1 lda) (1 *)) a-%offset%)
            lda
            (f2cl-lib:array-slice v-%data% single-float (p p)
                                  ((1 ldv) (1 *)) v-%offset%)
            1)
        (declare (ignore var-0 var-1 var-3 var-4))
        (when var-2 (setf lda var-2)))
      label7968))
   (cond
    (l2pert (setf xsc (f2cl-lib:fsqrt (/ small epsln)))
     (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                   ((> q nr) nil)
       (tagbody
         (setf temp1
                 (* xsc
                    (abs
                     (f2cl-lib:fref v-%data% (q q) ((1 ldv) (1 *))
                                    v-%offset%))))
         (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                       ((> p n) nil)
           (tagbody
             (if (or
                  (and (> p q)
                       (<=
                        (abs
                         (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                        v-%offset%))
                        temp1))
                  (< p q))
                 (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                      v-%offset%)
                       (f2cl-lib:sign temp1
                                      (f2cl-lib:fref v-%data% (p q)
                                                     ((1 ldv) (1 *))
                                                     v-%offset%))))
             (if (< p q)
                 (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                      v-%offset%)
                       (-
                        (f2cl-lib:fref v-%data% (p q)
                                       ((1 ldv) (1 *))
                                       v-%offset%))))
            label5968))
        label5969)))
    (t
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (slaset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
          zero zero
          (f2cl-lib:array-slice v-%data% single-float (1 2)
```

```
                                ((1 ldv) (1 *)) v-%offset%)
        ldv)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldv var-6)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (sgeqrf n nr v ldv
     (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (when var-0 (setf n var-0))
  (when var-1 (setf nr var-1))
  (when var-3 (setf ldv var-3))
  (when var-7 (setf ierr var-7)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" n nr v ldv
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) work-%offset%)
     n)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nr var-2))
  (when var-4 (setf ldv var-4))
  (when var-6 (setf n var-6)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
         (f2cl-lib:array-slice v-%data% single-float (p p)
                               ((1 ldv) (1 *)) v-%offset%)
         ldv
         (f2cl-lib:array-slice u-%data% single-float (p p)
                               ((1 ldu) (1 *)) u-%offset%)
         1)
      (declare (ignore var-0 var-1 var-3 var-4))
      (when var-2 (setf ldv var-2)))
   label7969))
(cond
 (l2pert (setf xsc (f2cl-lib:fsqrt (/ small epsln)))
  (f2cl-lib:fdo (q 2 (f2cl-lib:int-add q 1))
                ((> q nr) nil)
    (tagbody
```

```
        (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                      ((> p
                          (f2cl-lib:int-add q (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf temp1
                    (* xsc
                       (min
                        (abs
                         (f2cl-lib:fref u-%data% (p p)
                                        ((1 ldu) (1 *)) u-%offset%))
                        (abs
                         (f2cl-lib:fref u-%data% (q q)
                                        ((1 ldu) (1 *))
                                        u-%offset%)))))
            (setf (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                                 u-%offset%)
                    (-
                     (f2cl-lib:sign temp1
                                    (f2cl-lib:fref u-%data% (q p)
                                                   ((1 ldu) (1 *))
                                                   u-%offset%))))
           label9971))
      label9970)))
  (t
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slaset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
        zero zero
        (f2cl-lib:array-slice u-%data% single-float (1 2)
                              ((1 ldu) (1 *)) u-%offset%)
        ldu)
     (declare (ignore var-0 var-1 var-2 var-5))
     (when var-3 (setf zero var-3))
     (when var-4 (setf zero var-4))
     (when var-6 (setf ldu var-6)))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13)
     (sgesvj "L" "U" "V" nr nr u ldu sva n v ldv
      (f2cl-lib:array-slice work-%data% single-float
                            ((+ (f2cl-lib:int-mul 2 n)
                                (f2cl-lib:int-mul n nr) 1))
                            ((1 lwork)) work-%offset%)
      (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                        (f2cl-lib:int-mul n nr))
      info)
   (declare
    (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12))
   (when var-3 (setf nr var-3))
   (when var-4 (setf nr var-4))
```

```
  (when var-6 (setf ldu var-6))
  (when var-8 (setf n var-8))
  (when var-10 (setf ldv var-10))
  (when var-13 (setf info var-13)))
(setf scalem
        (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                          (f2cl-lib:int-mul n nr)
                                          1))
                       ((1 lwork)) work-%offset%))
(setf numrank
        (f2cl-lib:nint
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                           (f2cl-lib:int-mul n nr)
                                           2))
                        ((1 lwork)) work-%offset%)))
(cond
 ((< nr n)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "A" (f2cl-lib:int-sub n nr) nr zero zero
       (f2cl-lib:array-slice v-%data% single-float ((+ nr 1) 1)
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "A" nr (f2cl-lib:int-sub n nr) zero zero
       (f2cl-lib:array-slice v-%data% single-float
                             (1 (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-2 var-5))
    (when var-1 (setf nr var-1))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "A" (f2cl-lib:int-sub n nr) (f2cl-lib:int-sub n nr)
       zero one
       (f2cl-lib:array-slice v-%data% single-float
                             ((+ nr 1) (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
```

```
    (when var-6 (setf ldv var-6)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (sormqr "L" "N" n n nr
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) work-%offset%)
     n
     (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     v ldv
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) nr 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                       (f2cl-lib:int-mul n nr) nr)
     ierr)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf nr var-4))
  (when var-6 (setf n var-6))
  (when var-9 (setf ldv var-9))
  (when var-12 (setf ierr var-12)))
(setf temp1 (* (f2cl-lib:fsqrt (f2cl-lib:ffloat n)) epsln))
(f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
              ((> q n) nil)
  (tagbody
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) nr
                               (f2cl-lib:fref iwork (p) ((1 *)))))
                             ((1 lwork)) work-%offset%)
              (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                             v-%offset%))
       label8972))
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                             v-%offset%)
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-mul 2 n)
```

```
                                  (f2cl-lib:int-mul n nr) nr p))
                                ((1 lwork)) work-%offset%))
        label8973))
    (setf xsc
            (/ one
               (multiple-value-bind (ret-val var-0 var-1 var-2)
                   (snrm2 n
                    (f2cl-lib:array-slice v-%data% single-float
                                          (1 q) ((1 ldv) (1 *))
                                          v-%offset%)
                    1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf n var-0))
                  ret-val)))
    (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal n xsc
             (f2cl-lib:array-slice v-%data% single-float (1 q)
                                   ((1 ldv) (1 *)) v-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-1 (setf xsc var-1))))
   label7972))
(cond
 ((< nr m)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "A" (f2cl-lib:int-sub m nr) nr zero zero
       (f2cl-lib:array-slice u-%data% single-float ((+ nr 1) 1)
                             ((1 ldu) (1 *)) u-%offset%)
       ldu)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldu var-6)))
  (cond
   ((< nr n1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "A" nr (f2cl-lib:int-sub n1 nr) zero zero
         (f2cl-lib:array-slice u-%data% single-float
                               (1 (f2cl-lib:int-add nr 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-2 var-5))
      (when var-1 (setf nr var-1))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldu var-6)))
```

```
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (slaset "A" (f2cl-lib:int-sub m nr)
             (f2cl-lib:int-sub n1 nr) zero one
             (f2cl-lib:array-slice u-%data% single-float
                                   ((+ nr 1) (f2cl-lib:int-add nr 1))
                                   ((1 ldu) (1 *)) u-%offset%)
             ldu)
          (declare (ignore var-0 var-1 var-2 var-5))
          (when var-3 (setf zero var-3))
          (when var-4 (setf one var-4))
          (when var-6 (setf ldu var-6)))))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (sormqr "Left" "No Tr" m n1 n a lda work u ldu
         (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                               ((1 lwork)) work-%offset%)
         (f2cl-lib:int-sub lwork n) ierr)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf n1 var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldu var-9))
      (when var-12 (setf ierr var-12)))
    (if rowpiv
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (slaswp n1 u ldu 1 (f2cl-lib:int-sub m 1)
             (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                                   ((+ (f2cl-lib:int-mul 2 n) 1))
                                   ((1 *)) iwork-%offset%)
             -1)
          (declare (ignore var-1 var-3 var-4 var-5 var-6))
          (when var-0 (setf n1 var-0))
          (when var-2 (setf ldu var-2))))))
  (cond
   (transp
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (sswap n
             (f2cl-lib:array-slice u-%data% single-float (1 p)
                                   ((1 ldu) (1 *)) u-%offset%)
             1
             (f2cl-lib:array-slice v-%data% single-float (1 p)
                                   ((1 ldv) (1 *)) v-%offset%)
             1)
```

```
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf n var-0)))
       label6974))))))
 (cond
  ((<= uscal2
       (* (f2cl-lib:f2cl/ big (f2cl-lib:fref sva (1) ((1 n)))) uscal1))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (slascl "G" 0 0 uscal1 uscal2 nr 1 sva n ierr)
     (declare (ignore var-0 var-1 var-2 var-6 var-7))
     (when var-3 (setf uscal1 var-3))
     (when var-4 (setf uscal2 var-4))
     (when var-5 (setf nr var-5))
     (when var-8 (setf n var-8))
     (when var-9 (setf ierr var-9)))
   (setf uscal1 one) (setf uscal2 one)))
 (cond
  ((< nr n)
   (f2cl-lib:fdo (p (f2cl-lib:int-add nr 1) (f2cl-lib:int-add p 1))
                 ((> p n) nil)
     (tagbody
       (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%) zero)
      label3004))))
 (setf (f2cl-lib:fref work-%data% (1) ((1 lwork)) work-%offset%)
        (* uscal2 scalem))
 (setf (f2cl-lib:fref work-%data% (2) ((1 lwork)) work-%offset%) uscal1)
 (if errest
     (setf (f2cl-lib:fref work-%data% (3) ((1 lwork)) work-%offset%)
           sconda))
 (cond
  ((and lsvec rsvec)
   (setf (f2cl-lib:fref work-%data% (4) ((1 lwork)) work-%offset%)
         condr1)
   (setf (f2cl-lib:fref work-%data% (5) ((1 lwork)) work-%offset%)
         condr2)))
 (cond
  (l2tran
   (setf (f2cl-lib:fref work-%data% (6) ((1 lwork)) work-%offset%)
         entra)
   (setf (f2cl-lib:fref work-%data% (7) ((1 lwork)) work-%offset%)
         entrat)))
 (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) nr)
 (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%) numrank)
 (setf (f2cl-lib:fref iwork-%data% (3) ((1 *)) iwork-%offset%) warning)
 (go end_label)
end_label
 (return
  (values joba jobu jobv jobr jobt jobp m n nil lda nil nil ldu nil ldv
          nil lwork nil info))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgejsv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::joba
                                              fortran-to-lisp::jobu
                                              fortran-to-lisp::jobv
                                              fortran-to-lisp::jobr
                                              fortran-to-lisp::jobt
                                              fortran-to-lisp::jobp
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::ldu nil
                                              fortran-to-lisp::ldv nil
                                              fortran-to-lisp::lwork nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

29.2 sgesdd

29.2.1 Axiom unit tests

— sgesdd.input —

```
)set break resume
)sys rm -f sgesdd.output
)spool sgesdd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

29.2.2 Axiom help page

— sgesdd.help —

```
==================================================================
sgesdd examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgesdd (JOBZ, M, N, A, LDA, S, U, LDU, VT, LDVT, WORK,
           LWORK, IWORK, INFO)
           SGESDD

Function/Subroutine Documentation
   subroutine sgesdd (character JOBZ, integer M, integer N, real, dimension(
       lda, * ) A, integer LDA, real, dimension( * ) S, real, dimension( ldu,
       * ) U, integer LDU, real, dimension( ldvt, * ) VT, integer LDVT, real,
       dimension( * ) WORK, integer LWORK, integer, dimension( * ) IWORK,
       integer INFO)
       SGESDD

Purpose:
   SGESDD computes the singular value decomposition (SVD) of a real
   M-by-N matrix A, optionally computing the left and right singular
   vectors.  If singular vectors are desired, it uses a
   divide-and-conquer algorithm.

   The SVD is written

   A = U * SIGMA * transpose(V)
```

```
   where SIGMA is an M-by-N matrix which is zero except for its
   min(m,n) diagonal elements, U is an M-by-M orthogonal matrix, and
   V is an N-by-N orthogonal matrix.  The diagonal elements of SIGMA
   are the singular values of A; they are real and non-negative, and
   are returned in descending order.  The first min(m,n) columns of
   U and V are the left and right singular vectors of A.

   Note that the routine returns VT = V**T, not V.

   The divide and conquer algorithm makes very mild assumptions about
   floating point arithmetic. It will work on machines with a guard
   digit in add/subtract, or on those binary machines without guard
   digits which subtract like the Cray X-MP, Cray Y-MP, Cray C-90, or
   Cray-2. It could conceivably fail on hexadecimal or decimal machines
   without guard digits, but we know of none.

Parameters:

 JOBZ
   JOBZ is CHARACTER*1
   Specifies options for computing all or part of the matrix U:
   = 'A':  all M columns of U and all N rows of V**T are
           returned in the arrays U and VT;
   = 'S':  the first min(M,N) columns of U and the first
           min(M,N) rows of V**T are returned in the arrays U and VT;
   = 'O':  If M >= N, the first N columns of U are overwritten
           on the array A and all rows of V**T are returned in the array VT;
           otherwise, all columns of U are returned in the
           array U and the first M rows of V**T are overwritten
           in the array A;
   = 'N':  no columns of U or rows of V**T are computed.

 M
   M is INTEGER
   The number of rows of the input matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the input matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit,
   if JOBZ = 'O',  A is overwritten with the first N columns
        of U (the left singular vectors, stored columnwise) if M >= N;
        A is overwritten with the first M rows
        of V**T (the right singular vectors, stored rowwise) otherwise.
   if JOBZ .ne. 'O', the contents of A are destroyed.
```

```
LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

S
  S is REAL array, dimension (min(M,N))
  The singular values of A, sorted so that S(i) >= S(i+1).

U
  U is REAL array, dimension (LDU,UCOL)
  UCOL = M if JOBZ = 'A' or JOBZ = 'O' and M < N;
  UCOL = min(M,N) if JOBZ = 'S'.
  If JOBZ = 'A' or JOBZ = 'O' and M < N, U contains the M-by-M
       orthogonal matrix U;
  if JOBZ = 'S', U contains the first min(M,N) columns of U
       (the left singular vectors, stored columnwise);
  if JOBZ = 'O' and M >= N, or JOBZ = 'N', U is not referenced.

LDU
  LDU is INTEGER
  The leading dimension of the array U.  LDU >= 1; if
  JOBZ = 'S' or 'A' or JOBZ = 'O' and M < N, LDU >= M.

VT
  VT is REAL array, dimension (LDVT,N)
  If JOBZ = 'A' or JOBZ = 'O' and M >= N, VT contains the
  N-by-N orthogonal matrix V**T;
  if JOBZ = 'S', VT contains the first min(M,N) rows of
  V**T (the right singular vectors, stored rowwise);
  if JOBZ = 'O' and M < N, or JOBZ = 'N', VT is not referenced.

LDVT
  LDVT is INTEGER
  The leading dimension of the array VT.  LDVT >= 1; if
  JOBZ = 'A' or JOBZ = 'O' and M >= N, LDVT >= N;
  if JOBZ = 'S', LDVT >= min(M,N).

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK;

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK >= 1.
  If JOBZ = 'N', LWORK >= 3*min(M,N) + max(max(M,N),6*min(M,N)).
  If JOBZ = 'O', LWORK >= 3*min(M,N) +
                         max(max(M,N),5*min(M,N)*min(M,N)+4*min(M,N)).
  If JOBZ = 'S' or 'A' LWORK >= min(M,N)*(7+4*min(M,N))
  For good performance, LWORK should generally be larger.
  If LWORK = -1 but other input arguments are legal, WORK(1)
```

```
  returns the optimal LWORK.

 IWORK
  IWORK is INTEGER array, dimension (8*min(M,N))

 INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  SBDSDC did not converge, updating process failed.
```

———-

29.2.3 fortran code

— **sgesdd.f** —

```
*  =====================================================================
      SUBROUTINE SGESDD( JOBZ, M, N, A, LDA, S, U, LDU, VT, LDVT, WORK,
     $                   LWORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBZ
      INTEGER            INFO, LDA, LDU, LDVT, LWORK, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      REAL               A( LDA, * ), S( * ), U( LDU, * ),
     $                   VT( LDVT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, WNTQA, WNTQAS, WNTQN, WNTQO, WNTQS
      INTEGER            BDSPAC, BLK, CHUNK, I, IE, IERR, IL,
     $                   IR, ISCL, ITAU, ITAUP, ITAUQ, IU, IVT, LDWKVT,
     $                   LDWRKL, LDWRKR, LDWRKU, MAXWRK, MINMN, MINWRK,
```

```
     $                   MNTHR, NWORK, WRKBL
      REAL               ANRM, BIGNUM, EPS, SMLNUM
*     ..
*     .. Local Arrays ..
      INTEGER            IDUM( 1 )
      REAL               DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SBDSDC, SGEBRD, SGELQF, SGEMM, SGEQRF, SLACPY,
     $                   SLASCL, SLASET, SORGBR, SORGLQ, SORGQR, SORMBR,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               SLAMCH, SLANGE
      EXTERNAL           ILAENV, LSAME, SLAMCH, SLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          INT, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      MINMN = MIN( M, N )
      WNTQA = LSAME( JOBZ, 'A' )
      WNTQS = LSAME( JOBZ, 'S' )
      WNTQAS = WNTQA .OR. WNTQS
      WNTQO = LSAME( JOBZ, 'O' )
      WNTQN = LSAME( JOBZ, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
*
      IF( .NOT.( WNTQA .OR. WNTQS .OR. WNTQO .OR. WNTQN ) ) THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDU.LT.1 .OR. ( WNTQAS .AND. LDU.LT.M ) .OR.
     $         ( WNTQO .AND. M.LT.N .AND. LDU.LT.M ) ) THEN
         INFO = -8
      ELSE IF( LDVT.LT.1 .OR. ( WNTQA .AND. LDVT.LT.N ) .OR.
     $         ( WNTQS .AND. LDVT.LT.MINMN ) .OR.
     $         ( WNTQO .AND. M.GE.N .AND. LDVT.LT.N ) ) THEN
         INFO = -10
      END IF
```

```
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = 1
         MAXWRK = 1
         IF( M.GE.N .AND. MINMN.GT.0 ) THEN
*
*           Compute space needed for SBDSDC
*
            MNTHR = INT( MINMN*11.0E0 / 6.0E0 )
            IF( WNTQN ) THEN
               BDSPAC = 7*N
            ELSE
               BDSPAC = 3*N*N + 4*N
            END IF
            IF( M.GE.MNTHR ) THEN
               IF( WNTQN ) THEN
*
*                 Path 1 (M much larger than N, JOBZ='N')
*
                  WRKBL = N + N*ILAENV( 1, 'SGEQRF', ' ', M, N, -1,
     $                    -1 )
                  WRKBL = MAX( WRKBL, 3*N+2*N*
     $                    ILAENV( 1, 'SGEBRD', ' ', N, N, -1, -1 ) )
                  MAXWRK = MAX( WRKBL, BDSPAC+N )
                  MINWRK = BDSPAC + N
               ELSE IF( WNTQO ) THEN
*
*                 Path 2 (M much larger than N, JOBZ='O')
*
                  WRKBL = N + N*ILAENV( 1, 'SGEQRF', ' ', M, N, -1, -1 )
                  WRKBL = MAX( WRKBL, N+N*ILAENV( 1, 'SORGQR', ' ', M,
     $                    N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 3*N+2*N*
     $                    ILAENV( 1, 'SGEBRD', ' ', N, N, -1, -1 ) )
                  WRKBL = MAX( WRKBL, 3*N+N*
     $                    ILAENV( 1, 'SORMBR', 'QLN', N, N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 3*N+N*
     $                    ILAENV( 1, 'SORMBR', 'PRT', N, N, N, -1 ) )
                  WRKBL = MAX( WRKBL, BDSPAC+3*N )
                  MAXWRK = WRKBL + 2*N*N
                  MINWRK = BDSPAC + 2*N*N + 3*N
               ELSE IF( WNTQS ) THEN
*
```

```
*                Path 3 (M much larger than N, JOBZ='S')
*
                 WRKBL = N + N*ILAENV( 1, 'SGEQRF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, N+N*ILAENV( 1, 'SORGQR', ' ', M,
     $                   N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+2*N*
     $                   ILAENV( 1, 'SGEBRD', ' ', N, N, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'SORMBR', 'QLN', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'SORMBR', 'PRT', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*N )
                 MAXWRK = WRKBL + N*N
                 MINWRK = BDSPAC + N*N + 3*N
              ELSE IF( WNTQA ) THEN
*
*                Path 4 (M much larger than N, JOBZ='A')
*
                 WRKBL = N + N*ILAENV( 1, 'SGEQRF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, N+M*ILAENV( 1, 'SORGQR', ' ', M,
     $                   M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+2*N*
     $                   ILAENV( 1, 'SGEBRD', ' ', N, N, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'SORMBR', 'QLN', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'SORMBR', 'PRT', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*N )
                 MAXWRK = WRKBL + N*N
                 MINWRK = BDSPAC + N*N + 2*N + M
              END IF
           ELSE
*
*             Path 5 (M at least N, but not much larger)
*
              WRKBL = 3*N + ( M+N )*ILAENV( 1, 'SGEBRD', ' ', M, N, -1,
     $                -1 )
              IF( WNTQN ) THEN
                 MAXWRK = MAX( WRKBL, BDSPAC+3*N )
                 MINWRK = 3*N + MAX( M, BDSPAC )
              ELSE IF( WNTQO ) THEN
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'SORMBR', 'QLN', M, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'SORMBR', 'PRT', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*N )
                 MAXWRK = WRKBL + M*N
                 MINWRK = 3*N + MAX( M, N*N+BDSPAC )
              ELSE IF( WNTQS ) THEN
                 WRKBL = MAX( WRKBL, 3*N+N*
```

```
     $                  ILAENV( 1, 'SORMBR', 'QLN', M, N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 3*N+N*
     $                  ILAENV( 1, 'SORMBR', 'PRT', N, N, N, -1 ) )
                  MAXWRK = MAX( WRKBL, BDSPAC+3*N )
                  MINWRK = 3*N + MAX( M, BDSPAC )
               ELSE IF( WNTQA ) THEN
                  WRKBL = MAX( WRKBL, 3*N+M*
     $                  ILAENV( 1, 'SORMBR', 'QLN', M, M, N, -1 ) )
                  WRKBL = MAX( WRKBL, 3*N+N*
     $                  ILAENV( 1, 'SORMBR', 'PRT', N, N, N, -1 ) )
                  MAXWRK = MAX( MAXWRK, BDSPAC+3*N )
                  MINWRK = 3*N + MAX( M, BDSPAC )
               END IF
            END IF
         ELSE IF ( MINMN.GT.0 ) THEN
*
*           Compute space needed for SBDSDC
*
            MNTHR = INT( MINMN*11.0E0 / 6.0E0 )
            IF( WNTQN ) THEN
               BDSPAC = 7*M
            ELSE
               BDSPAC = 3*M*M + 4*M
            END IF
            IF( N.GE.MNTHR ) THEN
               IF( WNTQN ) THEN
*
*                 Path 1t (N much larger than M, JOBZ='N')
*
                  WRKBL = M + M*ILAENV( 1, 'SGELQF', ' ', M, N, -1,
     $                  -1 )
                  WRKBL = MAX( WRKBL, 3*M+2*M*
     $                  ILAENV( 1, 'SGEBRD', ' ', M, M, -1, -1 ) )
                  MAXWRK = MAX( WRKBL, BDSPAC+M )
                  MINWRK = BDSPAC + M
               ELSE IF( WNTQO ) THEN
*
*                 Path 2t (N much larger than M, JOBZ='O')
*
                  WRKBL = M + M*ILAENV( 1, 'SGELQF', ' ', M, N, -1, -1 )
                  WRKBL = MAX( WRKBL, M+M*ILAENV( 1, 'SORGLQ', ' ', M,
     $                  N, M, -1 ) )
                  WRKBL = MAX( WRKBL, 3*M+2*M*
     $                  ILAENV( 1, 'SGEBRD', ' ', M, M, -1, -1 ) )
                  WRKBL = MAX( WRKBL, 3*M+M*
     $                  ILAENV( 1, 'SORMBR', 'QLN', M, M, M, -1 ) )
                  WRKBL = MAX( WRKBL, 3*M+M*
     $                  ILAENV( 1, 'SORMBR', 'PRT', M, M, M, -1 ) )
                  WRKBL = MAX( WRKBL, BDSPAC+3*M )
                  MAXWRK = WRKBL + 2*M*M
```

```
                 MINWRK = BDSPAC + 2*M*M + 3*M
              ELSE IF( WNTQS ) THEN
*
*                Path 3t (N much larger than M, JOBZ='S')
*
                 WRKBL = M + M*ILAENV( 1, 'SGELQF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, M+M*ILAENV( 1, 'SORGLQ', ' ', M,
     $                   N, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+2*M*
     $                   ILAENV( 1, 'SGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'SORMBR', 'QLN', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'SORMBR', 'PRT', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*M )
                 MAXWRK = WRKBL + M*M
                 MINWRK = BDSPAC + M*M + 3*M
              ELSE IF( WNTQA ) THEN
*
*                Path 4t (N much larger than M, JOBZ='A')
*
                 WRKBL = M + M*ILAENV( 1, 'SGELQF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, M+N*ILAENV( 1, 'SORGLQ', ' ', N,
     $                   N, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+2*M*
     $                   ILAENV( 1, 'SGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'SORMBR', 'QLN', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'SORMBR', 'PRT', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*M )
                 MAXWRK = WRKBL + M*M
                 MINWRK = BDSPAC + M*M + 3*M
              END IF
           ELSE
*
*             Path 5t (N greater than M, but not much larger)
*
              WRKBL = 3*M + ( M+N )*ILAENV( 1, 'SGEBRD', ' ', M, N, -1,
     $                -1 )
              IF( WNTQN ) THEN
                 MAXWRK = MAX( WRKBL, BDSPAC+3*M )
                 MINWRK = 3*M + MAX( N, BDSPAC )
              ELSE IF( WNTQO ) THEN
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'SORMBR', 'QLN', M, M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'SORMBR', 'PRT', M, N, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*M )
                 MAXWRK = WRKBL + M*N
```

```
                 MINWRK = 3*M + MAX( N, M*M+BDSPAC )
              ELSE IF( WNTQS ) THEN
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'SORMBR', 'QLN', M, M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'SORMBR', 'PRT', M, N, M, -1 ) )
                 MAXWRK = MAX( WRKBL, BDSPAC+3*M )
                 MINWRK = 3*M + MAX( N, BDSPAC )
              ELSE IF( WNTQA ) THEN
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'SORMBR', 'QLN', M, M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'SORMBR', 'PRT', N, N, M, -1 ) )
                 MAXWRK = MAX( WRKBL, BDSPAC+3*M )
                 MINWRK = 3*M + MAX( N, BDSPAC )
              END IF
           END IF
        END IF
        MAXWRK = MAX( MAXWRK, MINWRK )
        WORK( 1 ) = MAXWRK
*
        IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
           INFO = -12
        END IF
     END IF
*
     IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'SGESDD', -INFO )
        RETURN
     ELSE IF( LQUERY ) THEN
        RETURN
     END IF
*
*    Quick return if possible
*
     IF( M.EQ.0 .OR. N.EQ.0 ) THEN
        RETURN
     END IF
*
*    Get machine constants
*
     EPS = SLAMCH( 'P' )
     SMLNUM = SQRT( SLAMCH( 'S' ) ) / EPS
     BIGNUM = ONE / SMLNUM
*
*    Scale A if max element outside range [SMLNUM,BIGNUM]
*
     ANRM = SLANGE( 'M', M, N, A, LDA, DUM )
     ISCL = 0
     IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
```

```
         ISCL = 1
         CALL SLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, IERR )
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ISCL = 1
         CALL SLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, IERR )
      END IF
*
      IF( M.GE.N ) THEN
*
*        A has at least as many rows as columns. If A has sufficiently
*        more rows than columns, first reduce using the QR
*        decomposition (if sufficient workspace available)
*
         IF( M.GE.MNTHR ) THEN
*
            IF( WNTQN ) THEN
*
*              Path 1 (M much larger than N, JOBZ='N')
*              No singular vectors to be computed
*
               ITAU = 1
               NWORK = ITAU + N
*
*              Compute A=Q*R
*              (Workspace: need 2*N, prefer N+N*NB)
*
               CALL SGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Zero out below R
*
               CALL SLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ), LDA )
               IE = 1
               ITAUQ = IE + N
               ITAUP = ITAUQ + N
               NWORK = ITAUP + N
*
*              Bidiagonalize R in A
*              (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
               CALL SGEBRD( N, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                      IERR )
               NWORK = IE + N
*
*              Perform bidiagonal SVD, computing singular values only
*              (Workspace: need N+BDSPAC)
*
               CALL SBDSDC( 'U', 'N', N, S, WORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, WORK( NWORK ), IWORK, INFO )
```

```
*
            ELSE IF( WNTQO ) THEN
*
*              Path 2 (M much larger than N, JOBZ = 'O')
*              N left singular vectors to be overwritten on A and
*              N right singular vectors to be computed in VT
*
               IR = 1
*
*              WORK(IR) is LDWRKR by N
*
               IF( LWORK.GE.LDA*N+N*N+3*N+BDSPAC ) THEN
                  LDWRKR = LDA
               ELSE
                  LDWRKR = ( LWORK-N*N-3*N-BDSPAC ) / N
               END IF
               ITAU = IR + LDWRKR*N
               NWORK = ITAU + N
*
*              Compute A=Q*R
*              (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
               CALL SGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy R to WORK(IR), zeroing out below it
*
               CALL SLACPY( 'U', N, N, A, LDA, WORK( IR ), LDWRKR )
               CALL SLASET( 'L', N-1, N-1, ZERO, ZERO, WORK( IR+1 ),
     $                      LDWRKR )
*
*              Generate Q in A
*              (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
               CALL SORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
               IE = ITAU
               ITAUQ = IE + N
               ITAUP = ITAUQ + N
               NWORK = ITAUP + N
*
*              Bidiagonalize R in VT, copying result to WORK(IR)
*              (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
               CALL SGEBRD( N, N, WORK( IR ), LDWRKR, S, WORK( IE ),
     $                      WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              WORK(IU) is N by N
*
```

```
               IU = NWORK
               NWORK = IU + N*N
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in WORK(IU) and computing right
*              singular vectors of bidiagonal matrix in VT
*              (Workspace: need N+N*N+BDSPAC)
*
               CALL SBDSDC( 'U', 'I', N, S, WORK( IE ), WORK( IU ), N,
     $                      VT, LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                      INFO )
*
*              Overwrite WORK(IU) by left singular vectors of R
*              and VT by right singular vectors of R
*              (Workspace: need 2*N*N+3*N, prefer 2*N*N+2*N+N*NB)
*
               CALL SORMBR( 'Q', 'L', 'N', N, N, N, WORK( IR ), LDWRKR,
     $                      WORK( ITAUQ ), WORK( IU ), N, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
               CALL SORMBR( 'P', 'R', 'T', N, N, N, WORK( IR ), LDWRKR,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Multiply Q in A by left singular vectors of R in
*              WORK(IU), storing result in WORK(IR) and copying to A
*              (Workspace: need 2*N*N, prefer N*N+M*N)
*
               DO 10 I = 1, M, LDWRKR
                  CHUNK = MIN( M-I+1, LDWRKR )
                  CALL SGEMM( 'N', 'N', CHUNK, N, N, ONE, A( I, 1 ),
     $                        LDA, WORK( IU ), N, ZERO, WORK( IR ),
     $                        LDWRKR )
                  CALL SLACPY( 'F', CHUNK, N, WORK( IR ), LDWRKR,
     $                         A( I, 1 ), LDA )
   10          CONTINUE
*
            ELSE IF( WNTQS ) THEN
*
               Path 3 (M much larger than N, JOBZ='S')
*              N left singular vectors to be computed in U and
*              N right singular vectors to be computed in VT
*
               IR = 1
*
*              WORK(IR) is N by N
*
               LDWRKR = N
               ITAU = IR + LDWRKR*N
               NWORK = ITAU + N
*
```

```
*               Compute A=Q*R
*               (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                CALL SGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                       LWORK-NWORK+1, IERR )
*
*               Copy R to WORK(IR), zeroing out below it
*
                CALL SLACPY( 'U', N, N, A, LDA, WORK( IR ), LDWRKR )
                CALL SLASET( 'L', N-1, N-1, ZERO, ZERO, WORK( IR+1 ),
     $                       LDWRKR )
*
*               Generate Q in A
*               (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                CALL SORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                       WORK( NWORK ), LWORK-NWORK+1, IERR )
                IE = ITAU
                ITAUQ = IE + N
                ITAUP = ITAUQ + N
                NWORK = ITAUP + N
*
*               Bidiagonalize R in WORK(IR)
*               (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                CALL SGEBRD( N, N, WORK( IR ), LDWRKR, S, WORK( IE ),
     $                       WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                       LWORK-NWORK+1, IERR )
*
*               Perform bidiagonal SVD, computing left singular vectors
*               of bidiagoal matrix in U and computing right singular
*               vectors of bidiagonal matrix in VT
*               (Workspace: need N+BDSPAC)
*
                CALL SBDSDC( 'U', 'I', N, S, WORK( IE ), U, LDU, VT,
     $                       LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                       INFO )
*
*               Overwrite U by left singular vectors of R and VT
*               by right singular vectors of R
*               (Workspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*
                CALL SORMBR( 'Q', 'L', 'N', N, N, N, WORK( IR ), LDWRKR,
     $                       WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                       LWORK-NWORK+1, IERR )
*
                CALL SORMBR( 'P', 'R', 'T', N, N, N, WORK( IR ), LDWRKR,
     $                       WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                       LWORK-NWORK+1, IERR )
*
```

```
*              Multiply Q in A by left singular vectors of R in
*              WORK(IR), storing result in U
*              (Workspace: need N*N)
*
               CALL SLACPY( 'F', N, N, U, LDU, WORK( IR ), LDWRKR )
               CALL SGEMM( 'N', 'N', M, N, N, ONE, A, LDA, WORK( IR ),
     $                     LDWRKR, ZERO, U, LDU )
*
            ELSE IF( WNTQA ) THEN
*
*              Path 4 (M much larger than N, JOBZ='A')
*              M left singular vectors to be computed in U and
*              N right singular vectors to be computed in VT
*
               IU = 1
*
*              WORK(IU) is N by N
*
               LDWRKU = N
               ITAU = IU + LDWRKU*N
               NWORK = ITAU + N
*
*              Compute A=Q*R, copying result to U
*              (Workspace: need N*N+N+M, prefer N*N+N+M*NB)
*
               CALL SGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
               CALL SLACPY( 'L', M, N, A, LDA, U, LDU )
*
*              Generate Q in U
*              (Workspace: need N*N+N+M, prefer N*N+N+M*NB)
               CALL SORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Produce R in A, zeroing out other entries
*
               CALL SLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ), LDA )
               IE = ITAU
               ITAUQ = IE + N
               ITAUP = ITAUQ + N
               NWORK = ITAUP + N
*
*              Bidiagonalize R in A
*              (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
               CALL SGEBRD( N, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                      IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
```

```
*              of bidiagonal matrix in WORK(IU) and computing right
*              singular vectors of bidiagonal matrix in VT
*              (Workspace: need N+N*N+BDSPAC)
*
               CALL SBDSDC( 'U', 'I', N, S, WORK( IE ), WORK( IU ), N,
     $                      VT, LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                      INFO )
*
*              Overwrite WORK(IU) by left singular vectors of R and VT
*              by right singular vectors of R
*              (Workspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*
               CALL SORMBR( 'Q', 'L', 'N', N, N, N, A, LDA,
     $                      WORK( ITAUQ ), WORK( IU ), LDWRKU,
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
               CALL SORMBR( 'P', 'R', 'T', N, N, N, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Multiply Q in U by left singular vectors of R in
*              WORK(IU), storing result in A
*              (Workspace: need N*N)
*
               CALL SGEMM( 'N', 'N', M, N, N, ONE, U, LDU, WORK( IU ),
     $                     LDWRKU, ZERO, A, LDA )
*
*              Copy left singular vectors of A from A to U
*
               CALL SLACPY( 'F', M, N, A, LDA, U, LDU )
*
            END IF
*
         ELSE
*
*           M .LT. MNTHR
*
*           Path 5 (M at least N, but not much larger)
*           Reduce to bidiagonal form without QR decomposition
*
            IE = 1
            ITAUQ = IE + N
            ITAUP = ITAUQ + N
            NWORK = ITAUP + N
*
*           Bidiagonalize A
*           (Workspace: need 3*N+M, prefer 3*N+(M+N)*NB)
*
            CALL SGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                   WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                   IERR )
```

```
            IF( WNTQN ) THEN
*
*              Perform bidiagonal SVD, only computing singular values
*              (Workspace: need N+BDSPAC)
*
               CALL SBDSDC( 'U', 'N', N, S, WORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, WORK( NWORK ), IWORK, INFO )
            ELSE IF( WNTQO ) THEN
               IU = NWORK
               IF( LWORK.GE.M*N+3*N+BDSPAC ) THEN
*
*                 WORK( IU ) is M by N
*
                  LDWRKU = M
                  NWORK = IU + LDWRKU*N
                  CALL SLASET( 'F', M, N, ZERO, ZERO, WORK( IU ),
     $                         LDWRKU )
               ELSE
*
*                 WORK( IU ) is N by N
*
                  LDWRKU = N
                  NWORK = IU + LDWRKU*N
*
*                 WORK(IR) is LDWRKR by N
*
                  IR = NWORK
                  LDWRKR = ( LWORK-N*N-3*N ) / N
               END IF
               NWORK = IU + LDWRKU*N
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in WORK(IU) and computing right
*              singular vectors of bidiagonal matrix in VT
*              (Workspace: need N+N*N+BDSPAC)
*
               CALL SBDSDC( 'U', 'I', N, S, WORK( IE ), WORK( IU ),
     $                      LDWRKU, VT, LDVT, DUM, IDUM, WORK( NWORK ),
     $                      IWORK, INFO )
*
*              Overwrite VT by right singular vectors of A
*              (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
               CALL SORMBR( 'P', 'R', 'T', N, N, N, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
               IF( LWORK.GE.M*N+3*N+BDSPAC ) THEN
*
*                 Overwrite WORK(IU) by left singular vectors of A
```

```
*                 (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                  CALL SORMBR( 'Q', 'L', 'N', M, N, N, A, LDA,
     $                         WORK( ITAUQ ), WORK( IU ), LDWRKU,
     $                         WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                 Copy left singular vectors of A from WORK(IU) to A
*
                  CALL SLACPY( 'F', M, N, WORK( IU ), LDWRKU, A, LDA )
               ELSE
*
*                 Generate Q in A
*                 (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                  CALL SORGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
     $                         WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                 Multiply Q in A by left singular vectors of
*                 bidiagonal matrix in WORK(IU), storing result in
*                 WORK(IR) and copying to A
*                 (Workspace: need 2*N*N, prefer N*N+M*N)
*
                  DO 20 I = 1, M, LDWRKR
                     CHUNK = MIN( M-I+1, LDWRKR )
                     CALL SGEMM( 'N', 'N', CHUNK, N, N, ONE, A( I, 1 ),
     $                           LDA, WORK( IU ), LDWRKU, ZERO,
     $                           WORK( IR ), LDWRKR )
                     CALL SLACPY( 'F', CHUNK, N, WORK( IR ), LDWRKR,
     $                            A( I, 1 ), LDA )
   20             CONTINUE
               END IF
*
            ELSE IF( WNTQS ) THEN
*
               Perform bidiagonal SVD, computing left singular vectors
               of bidiagonal matrix in U and computing right singular
               vectors of bidiagonal matrix in VT
               (Workspace: need N+BDSPAC)
*
               CALL SLASET( 'F', M, N, ZERO, ZERO, U, LDU )
               CALL SBDSDC( 'U', 'I', N, S, WORK( IE ), U, LDU, VT,
     $                      LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                      INFO )
*
               Overwrite U by left singular vectors of A and VT
               by right singular vectors of A
               (Workspace: need 3*N, prefer 2*N+N*NB)
*
               CALL SORMBR( 'Q', 'L', 'N', M, N, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
```

```
     $                     LWORK-NWORK+1, IERR )
              CALL SORMBR( 'P', 'R', 'T', N, N, N, A, LDA,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
           ELSE IF( WNTQA ) THEN
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in U and computing right singular
*             vectors of bidiagonal matrix in VT
*             (Workspace: need N+BDSPAC)
*
              CALL SLASET( 'F', M, M, ZERO, ZERO, U, LDU )
              CALL SBDSDC( 'U', 'I', N, S, WORK( IE ), U, LDU, VT,
     $                     LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                     INFO )
*
*             Set the right corner of U to identity matrix
*
              IF( M.GT.N ) THEN
                 CALL SLASET( 'F', M-N, M-N, ZERO, ONE, U( N+1, N+1 ),
     $                        LDU )
              END IF
*
*             Overwrite U by left singular vectors of A and VT
*             by right singular vectors of A
*             (Workspace: need N*N+2*N+M, prefer N*N+2*N+M*NB)
*
              CALL SORMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
              CALL SORMBR( 'P', 'R', 'T', N, N, M, A, LDA,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
           END IF
*
        END IF
*
     ELSE
*
*       A has more columns than rows. If A has sufficiently more
*       columns than rows, first reduce using the LQ decomposition (if
*       sufficient workspace available)
*
        IF( N.GE.MNTHR ) THEN
*
           IF( WNTQN ) THEN
*
*             Path 1t (N much larger than M, JOBZ='N')
*             No singular vectors to be computed
*
```

```
               ITAU = 1
               NWORK = ITAU + M
*
*              Compute A=L*Q
*              (Workspace: need 2*M, prefer M+M*NB)
*
               CALL SGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Zero out above L
*
               CALL SLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ), LDA )
               IE = 1
               ITAUQ = IE + M
               ITAUP = ITAUQ + M
               NWORK = ITAUP + M
*
*              Bidiagonalize L in A
*              (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
               CALL SGEBRD( M, M, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                      IERR )
               NWORK = IE + M
*
*              Perform bidiagonal SVD, computing singular values only
*              (Workspace: need M+BDSPAC)
*
               CALL SBDSDC( 'U', 'N', M, S, WORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, WORK( NWORK ), IWORK, INFO )
*
            ELSE IF( WNTQO ) THEN
*
*              Path 2t (N much larger than M, JOBZ='O')
*              M right singular vectors to be overwritten on A and
*              M left singular vectors to be computed in U
*
               IVT = 1
*
*              IVT is M by M
*
               IL = IVT + M*M
               IF( LWORK.GE.M*N+M*M+3*M+BDSPAC ) THEN
*
*                 WORK(IL) is M by N
*
                  LDWRKL = M
                  CHUNK = N
               ELSE
                  LDWRKL = M
```

```
               CHUNK = ( LWORK-M*M ) / M
            END IF
            ITAU = IL + LDWRKL*M
            NWORK = ITAU + M
*
*           Compute A=L*Q
*           (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
            CALL SGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                   LWORK-NWORK+1, IERR )
*
*           Copy L to WORK(IL), zeroing about above it
*
            CALL SLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWRKL )
            CALL SLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                   WORK( IL+LDWRKL ), LDWRKL )
*
*           Generate Q in A
*           (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
            CALL SORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                   WORK( NWORK ), LWORK-NWORK+1, IERR )
            IE = ITAU
            ITAUQ = IE + M
            ITAUP = ITAUQ + M
            NWORK = ITAUP + M
*
*           Bidiagonalize L in WORK(IL)
*           (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
            CALL SGEBRD( M, M, WORK( IL ), LDWRKL, S, WORK( IE ),
     $                   WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                   LWORK-NWORK+1, IERR )
*
*           Perform bidiagonal SVD, computing left singular vectors
*           of bidiagonal matrix in U, and computing right singular
*           vectors of bidiagonal matrix in WORK(IVT)
*           (Workspace: need M+M*M+BDSPAC)
*
            CALL SBDSDC( 'U', 'I', M, S, WORK( IE ), U, LDU,
     $                   WORK( IVT ), M, DUM, IDUM, WORK( NWORK ),
     $                   IWORK, INFO )
*
*           Overwrite U by left singular vectors of L and WORK(IVT)
*           by right singular vectors of L
*           (Workspace: need 2*M*M+3*M, prefer 2*M*M+2*M+M*NB)
*
            CALL SORMBR( 'Q', 'L', 'N', M, M, M, WORK( IL ), LDWRKL,
     $                   WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                   LWORK-NWORK+1, IERR )
```

```
               CALL SORMBR( 'P', 'R', 'T', M, M, M, WORK( IL ), LDWRKL,
     $                      WORK( ITAUP ), WORK( IVT ), M,
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Multiply right singular vectors of L in WORK(IVT) by Q
*              in A, storing result in WORK(IL) and copying to A
*              (Workspace: need 2*M*M, prefer M*M+M*N)
*
               DO 30 I = 1, N, CHUNK
                  BLK = MIN( N-I+1, CHUNK )
                  CALL SGEMM( 'N', 'N', M, BLK, M, ONE, WORK( IVT ), M,
     $                        A( 1, I ), LDA, ZERO, WORK( IL ), LDWRKL )
                  CALL SLACPY( 'F', M, BLK, WORK( IL ), LDWRKL,
     $                         A( 1, I ), LDA )
   30          CONTINUE
*
            ELSE IF( WNTQS ) THEN
*
*              Path 3t (N much larger than M, JOBZ='S')
*              M right singular vectors to be computed in VT and
*              M left singular vectors to be computed in U
*
               IL = 1
*
*              WORK(IL) is M by M
*
               LDWRKL = M
               ITAU = IL + LDWRKL*M
               NWORK = ITAU + M
*
*              Compute A=L*Q
*              (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
               CALL SGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy L to WORK(IL), zeroing out above it
*
               CALL SLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWRKL )
               CALL SLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                      WORK( IL+LDWRKL ), LDWRKL )
*
*              Generate Q in A
*              (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
               CALL SORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
               IE = ITAU
               ITAUQ = IE + M
               ITAUP = ITAUQ + M
```

```
               NWORK = ITAUP + M
*
*              Bidiagonalize L in WORK(IU), copying result to U
*              (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
               CALL SGEBRD( M, M, WORK( IL ), LDWRKL, S, WORK( IE ),
     $                      WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in U and computing right singular
*              vectors of bidiagonal matrix in VT
*              (Workspace: need M+BDSPAC)
*
               CALL SBDSDC( 'U', 'I', M, S, WORK( IE ), U, LDU, VT,
     $                      LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                      INFO )
*
*              Overwrite U by left singular vectors of L and VT
*              by right singular vectors of L
*              (Workspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*
               CALL SORMBR( 'Q', 'L', 'N', M, M, M, WORK( IL ), LDWRKL,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
               CALL SORMBR( 'P', 'R', 'T', M, M, M, WORK( IL ), LDWRKL,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Multiply right singular vectors of L in WORK(IL) by
*              Q in A, storing result in VT
*              (Workspace: need M*M)
*
               CALL SLACPY( 'F', M, M, VT, LDVT, WORK( IL ), LDWRKL )
               CALL SGEMM( 'N', 'N', M, N, M, ONE, WORK( IL ), LDWRKL,
     $                     A, LDA, ZERO, VT, LDVT )
*
            ELSE IF( WNTQA ) THEN
*
*              Path 4t (N much larger than M, JOBZ='A')
*              N right singular vectors to be computed in VT and
*              M left singular vectors to be computed in U
*
               IVT = 1
*
*              WORK(IVT) is M by M
*
               LDWKVT = M
               ITAU = IVT + LDWKVT*M
               NWORK = ITAU + M
```

```
*
*               Compute A=L*Q, copying result to VT
*               (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                CALL SGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                       LWORK-NWORK+1, IERR )
                CALL SLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*               Generate Q in VT
*               (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                CALL SORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                       WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*               Produce L in A, zeroing out other entries
*
                CALL SLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ), LDA )
                IE = ITAU
                ITAUQ = IE + M
                ITAUP = ITAUQ + M
                NWORK = ITAUP + M
*
*               Bidiagonalize L in A
*               (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                CALL SGEBRD( M, M, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                       WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                       IERR )
*
*               Perform bidiagonal SVD, computing left singular vectors
*               of bidiagonal matrix in U and computing right singular
*               vectors of bidiagonal matrix in WORK(IVT)
*               (Workspace: need M+M*M+BDSPAC)
*
                CALL SBDSDC( 'U', 'I', M, S, WORK( IE ), U, LDU,
     $                       WORK( IVT ), LDWKVT, DUM, IDUM,
     $                       WORK( NWORK ), IWORK, INFO )
*
*               Overwrite U by left singular vectors of L and WORK(IVT)
*               by right singular vectors of L
*               (Workspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*
                CALL SORMBR( 'Q', 'L', 'N', M, M, M, A, LDA,
     $                       WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                       LWORK-NWORK+1, IERR )
                CALL SORMBR( 'P', 'R', 'T', M, M, M, A, LDA,
     $                       WORK( ITAUP ), WORK( IVT ), LDWKVT,
     $                       WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*               Multiply right singular vectors of L in WORK(IVT) by
```

```
*              Q in VT, storing result in A
*              (Workspace: need M*M)
*
               CALL SGEMM( 'N', 'N', M, N, M, ONE, WORK( IVT ), LDWKVT,
     $                     VT, LDVT, ZERO, A, LDA )
*
*              Copy right singular vectors of A from A to VT
*
               CALL SLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
            END IF
*
         ELSE
*
            N .LT. MNTHR
*
            Path 5t (N greater than M, but not much larger)
            Reduce to bidiagonal form without LQ decomposition
*
            IE = 1
            ITAUQ = IE + M
            ITAUP = ITAUQ + M
            NWORK = ITAUP + M
*
            Bidiagonalize A
            (Workspace: need 3*M+N, prefer 3*M+(M+N)*NB)
*
            CALL SGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                   WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                   IERR )
            IF( WNTQN ) THEN
*
               Perform bidiagonal SVD, only computing singular values
               (Workspace: need M+BDSPAC)
*
               CALL SBDSDC( 'L', 'N', M, S, WORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, WORK( NWORK ), IWORK, INFO )
            ELSE IF( WNTQO ) THEN
               LDWKVT = M
               IVT = NWORK
               IF( LWORK.GE.M*N+3*M+BDSPAC ) THEN
*
                  WORK( IVT ) is M by N
*
                  CALL SLASET( 'F', M, N, ZERO, ZERO, WORK( IVT ),
     $                         LDWKVT )
                  NWORK = IVT + LDWKVT*N
               ELSE
*
                  WORK( IVT ) is M by M
```

```
*
                 NWORK = IVT + LDWKVT*M
                 IL = NWORK
*
*                WORK(IL) is M by CHUNK
*
                 CHUNK = ( LWORK-M*M-3*M ) / M
              END IF
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in U and computing right singular
*             vectors of bidiagonal matrix in WORK(IVT)
*             (Workspace: need M*M+BDSPAC)
*
              CALL SBDSDC( 'L', 'I', M, S, WORK( IE ), U, LDU,
     $                     WORK( IVT ), LDWKVT, DUM, IDUM,
     $                     WORK( NWORK ), IWORK, INFO )
*
*             Overwrite U by left singular vectors of A
*             (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
              CALL SORMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
              IF( LWORK.GE.M*N+3*M+BDSPAC ) THEN
*
*                Overwrite WORK(IVT) by left singular vectors of A
*                (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                 CALL SORMBR( 'P', 'R', 'T', M, N, M, A, LDA,
     $                        WORK( ITAUP ), WORK( IVT ), LDWKVT,
     $                        WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                Copy right singular vectors of A from WORK(IVT) to A
*
                 CALL SLACPY( 'F', M, N, WORK( IVT ), LDWKVT, A, LDA )
              ELSE
*
*                Generate P**T in A
*                (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                 CALL SORGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                        WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                Multiply Q in A by right singular vectors of
*                bidiagonal matrix in WORK(IVT), storing result in
*                WORK(IL) and copying to A
*                (Workspace: need 2*M*M, prefer M*M+M*N)
*
```

```
                 DO 40 I = 1, N, CHUNK
                    BLK = MIN( N-I+1, CHUNK )
                    CALL SGEMM( 'N', 'N', M, BLK, M, ONE, WORK( IVT ),
     $                          LDWKVT, A( 1, I ), LDA, ZERO,
     $                          WORK( IL ), M )
                    CALL SLACPY( 'F', M, BLK, WORK( IL ), M, A( 1, I ),
     $                           LDA )
   40            CONTINUE
              END IF
           ELSE IF( WNTQS ) THEN
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in U and computing right singular
*             vectors of bidiagonal matrix in VT
*             (Workspace: need M+BDSPAC)
*
              CALL SLASET( 'F', M, N, ZERO, ZERO, VT, LDVT )
              CALL SBDSDC( 'L', 'I', M, S, WORK( IE ), U, LDU, VT,
     $                     LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                     INFO )
*
*             Overwrite U by left singular vectors of A and VT
*             by right singular vectors of A
*             (Workspace: need 3*M, prefer 2*M+M*NB)
*
              CALL SORMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
              CALL SORMBR( 'P', 'R', 'T', M, N, M, A, LDA,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
           ELSE IF( WNTQA ) THEN
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in U and computing right singular
*             vectors of bidiagonal matrix in VT
*             (Workspace: need M+BDSPAC)
*
              CALL SLASET( 'F', N, N, ZERO, ZERO, VT, LDVT )
              CALL SBDSDC( 'L', 'I', M, S, WORK( IE ), U, LDU, VT,
     $                     LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                     INFO )
*
*             Set the right corner of VT to identity matrix
*
              IF( N.GT.M ) THEN
                 CALL SLASET( 'F', N-M, N-M, ZERO, ONE, VT( M+1, M+1 ),
     $                        LDVT )
              END IF
*
```

```
*              Overwrite U by left singular vectors of A and VT
*              by right singular vectors of A
*              (Workspace: need 2*M+N, prefer 2*M+N*NB)
*
               CALL SORMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
               CALL SORMBR( 'P', 'R', 'T', N, N, M, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
            END IF
*
         END IF
*
      END IF
*
*     Undo scaling if necessary
*
      IF( ISCL.EQ.1 ) THEN
         IF( ANRM.GT.BIGNUM )
     $      CALL SLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( ANRM.LT.SMLNUM )
     $      CALL SLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
      END IF
*
*     Return optimal workspace in WORK(1)
*
      WORK( 1 ) = MAXWRK
*
      RETURN
*
*     End of SGESDD
*
      END
```

———-

29.2.4 lisp code

— **sgesdd.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sgesdd (jobz m n a lda s u ldu vt ldvt work lwork iwork info)
```

```
(declare (type (array f2cl-lib:integer4 (*)) iwork)
         (type (array single-float (*)) work vt u s a)
         (type (f2cl-lib:integer4) info lwork ldvt ldu lda n m)
         (type (string *) jobz))
(f2cl-lib:with-multi-array-data
    ((jobz character jobz-%data% jobz-%offset%)
     (a single-float a-%data% a-%offset%)
     (s single-float s-%data% s-%offset%)
     (u single-float u-%data% u-%offset%)
     (vt single-float vt-%data% vt-%offset%)
     (work single-float work-%data% work-%offset%)
     (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((dum (make-array 1 :element-type 'single-float))
         (idum (make-array 1 :element-type 'f2cl-lib:integer4)) (anrm 0.0)
         (bignum 0.0) (eps 0.0) (smlnum 0.0) (bdspac 0) (blk 0) (chunk 0)
         (i 0) (ie 0) (ierr 0) (il 0) (ir 0) (iscl 0) (itau 0) (itaup 0)
         (itauq 0) (iu 0) (ivt 0) (ldwkvt 0) (ldwrkl 0) (ldwrkr 0)
         (ldwrku 0) (maxwrk 0) (minmn 0) (minwrk 0) (mnthr 0) (nwork 0)
         (wrkbl 0) (lquery nil) (wntqa nil) (wntqas nil) (wntqn nil)
         (wntqo nil) (wntqs nil))
    (declare (type (array single-float (1)) dum)
             (type (array f2cl-lib:integer4 (1)) idum)
             (type (single-float) anrm bignum eps smlnum)
             (type (f2cl-lib:integer4) bdspac blk chunk i ie ierr il ir
              iscl itau itaup itauq iu ivt ldwkvt ldwrkl ldwrkr ldwrku
              maxwrk minmn minwrk mnthr nwork wrkbl)
             (type f2cl-lib:logical lquery wntqa wntqas wntqn wntqo wntqs))
    (setf info 0)
    (setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
    (setf wntqa
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobz "A")
              (declare (ignore var-1))
              (when var-0 (setf jobz var-0))
              ret-val))
    (setf wntqs
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobz "S")
              (declare (ignore var-1))
              (when var-0 (setf jobz var-0))
              ret-val))
    (setf wntqas (or wntqa wntqs))
    (setf wntqo
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobz "O")
              (declare (ignore var-1))
              (when var-0 (setf jobz var-0))
              ret-val))
    (setf wntqn
            (multiple-value-bind (ret-val var-0 var-1)
```

```
              (lsame jobz "N")
            (declare (ignore var-1))
            (when var-0 (setf jobz var-0))
            ret-val))
  (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
  (cond ((not (or wntqa wntqs wntqo wntqn)) (setf info -1))
        ((< m 0) (setf info -2)) ((< n 0) (setf info -3))
        ((< lda
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -5))
        ((or (< ldu 1) (and wntqas (< ldu m))
             (and wntqo (< m n) (< ldu m)))
         (setf info -8))
        ((or (< ldvt 1) (and wntqa (< ldvt n)) (and wntqs (< ldvt minmn))
             (and wntqo (>= m n) (< ldvt n)))
         (setf info -10)))
  (cond
   ((= info 0) (setf minwrk 1) (setf maxwrk 1)
    (cond
     ((and (>= m n) (> minmn 0))
      (setf mnthr (f2cl-lib:int (/ (* minmn 11.0) 6.0)))
      (cond (wntqn (setf bdspac (f2cl-lib:int-mul 7 n)))
            (t
             (setf bdspac
                     (f2cl-lib:int-add (f2cl-lib:int-mul 3 n n)
                                       (f2cl-lib:int-mul 4 n)))))
      (cond
       ((>= m mnthr)
        (cond
         (wntqn
          (setf wrkbl
                  (f2cl-lib:int-add n
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val var-0
                                                           var-1 var-2
                                                           var-3 var-4
                                                           var-5 var-6)
                                                          (ilaenv 1
                                                           "SGEQRF" " " m
                                                           n -1 -1)
                                                        (declare
                                                         (ignore var-0
                                                          var-1 var-2
                                                          var-5 var-6))
                                                        (setf m var-3)
                                                        (setf n var-4)
                                                        ret-val))))
          (setf wrkbl
                  (max (the f2cl-lib:integer4 wrkbl)
```

```
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                           (f2cl-lib:int-mul 2 n
                                                             (multiple-value-bind
                                                                 (ret-val
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-3
                                                                  var-4
                                                                  var-5
                                                                  var-6)
                                                                 (ilaenv
                                                                  1
                                                                  "SGEBRD"
                                                                  " "
                                                                  n n
                                                                  -1
                                                                  -1)
                                                               (declare
                                                                (ignore
                                                                 var-0
                                                                 var-1
                                                                 var-2
                                                                 var-5
                                                                 var-6))
                                                               (setf n
                                                                       var-3)
                                                               (setf n
                                                                       var-4)
                                                               ret-val))))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add bdspac n))))
 (setf minwrk (f2cl-lib:int-add bdspac n)))
(wntqo
 (setf wrkbl
         (f2cl-lib:int-add n
                           (f2cl-lib:int-mul n
                                             (multiple-value-bind
                                                 (ret-val var-0
                                                  var-1 var-2
                                                  var-3 var-4
                                                  var-5 var-6)
                                                 (ilaenv 1
                                                  "SGEQRF" " " m
                                                  n -1 -1)
                                               (declare
                                                (ignore var-0
```

```
                                                    var-1 var-2
                                                    var-5 var-6))
                                                  (setf m var-3)
                                                  (setf n var-4)
                                                  ret-val))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add n
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "SORGQR"
                                                           " "
                                                           m n
                                                           n
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (setf m
                                                                var-3)
                                                        (setf n
                                                                var-4)
                                                        (setf n
                                                                var-5)
                                                        ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul 2 n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
```

```
                                                                    var-4
                                                                    var-5
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "SGEBRD"
                                                                    " "
                                                                    n n
                                                                    -1
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-5
                                                                    var-6))
                                                                  (setf n
                                                                          var-3)
                                                                  (setf n
                                                                          var-4)
                                                                  ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "SORMBR"
                                                           "QLN"
                                                           n n
                                                           n
                                                           -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-6))
                                                          (setf n
```

```
                                                                    var-3)
                                                            (setf n
                                                                    var-4)
                                                            (setf n
                                                                    var-5)
                                                            ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                                    (f2cl-lib:int-mul n
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "SORMBR"
                                                                           "PRT"
                                                                           n n
                                                                           n
                                                                           -1)
                                                            (declare
                                                             (ignore
                                                              var-0
                                                              var-1
                                                              var-2
                                                              var-6))
                                                            (setf n
                                                                    var-3)
                                                            (setf n
                                                                    var-4)
                                                            (setf n
                                                                    var-5)
                                                            ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add bdspac
                                                    (f2cl-lib:int-mul 3 n)))))
                (setf maxwrk (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 n n)))
                (setf minwrk
                        (f2cl-lib:int-add bdspac (f2cl-lib:int-mul 2 n n)
                                          (f2cl-lib:int-mul 3 n))))
               (wntqs
```

```
(setf wrkbl
        (f2cl-lib:int-add n
                          (f2cl-lib:int-mul n
                                            (multiple-value-bind
                                                (ret-val var-0
                                                 var-1 var-2
                                                 var-3 var-4
                                                 var-5 var-6)
                                                (ilaenv 1
                                                 "SGEQRF" " " m
                                                 n -1 -1)
                                              (declare
                                               (ignore var-0
                                                var-1 var-2
                                                var-5 var-6))
                                              (setf m var-3)
                                              (setf n var-4)
                                              ret-val))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add n
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "SORGQR"
                                                           " "
                                                           m n
                                                           n
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (setf m
                                                                var-3)
                                                        (setf n
                                                                var-4)
                                                        (setf n
```

```
                                                                    var-5)
                                                              ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul 2 n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "SGEBRD"
                                                           " "
                                                           n n
                                                           -1
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-5
                                                          var-6))
                                                        (setf n
                                                                var-3)
                                                        (setf n
                                                                var-4)
                                                        ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
```

```
                                                                    1
                                                                    "SORMBR"
                                                                    "QLN"
                                                                    n n
                                                                    n
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (setf n
                                                                          var-3)
                                                                  (setf n
                                                                          var-4)
                                                                  (setf n
                                                                          var-5)
                                                                  ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "SORMBR"
                                                           "PRT"
                                                           n n
                                                           n
                                                           -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-6))
                                                          (setf n
                                                                  var-3)
                                                          (setf n
                                                                  var-4)
```

```
                                                                (setf n
                                                                        var-5)
                                                                ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add bdspac
                                                    (f2cl-lib:int-mul 3 n)))))
                (setf maxwrk (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul n n)))
                (setf minwrk
                        (f2cl-lib:int-add bdspac (f2cl-lib:int-mul n n)
                                          (f2cl-lib:int-mul 3 n))))
               (wntqa
                (setf wrkbl
                        (f2cl-lib:int-add n
                                          (f2cl-lib:int-mul n
                                                            (multiple-value-bind
                                                                (ret-val var-0
                                                                 var-1 var-2
                                                                 var-3 var-4
                                                                 var-5 var-6)
                                                                (ilaenv 1
                                                                 "SGEQRF" " " m
                                                                 n -1 -1)
                                                              (declare
                                                               (ignore var-0
                                                                var-1 var-2
                                                                var-5 var-6))
                                                              (setf m var-3)
                                                              (setf n var-4)
                                                              ret-val))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add n
                                                    (f2cl-lib:int-mul m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "SORGQR"
                                                                           " "
                                                                           m m
```

```
                                                                    n
                                                                    -1)
                                                                 (declare
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-6))
                                                                 (setf m
                                                                         var-3)
                                                                 (setf m
                                                                         var-4)
                                                                 (setf n
                                                                         var-5)
                                                                 ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul 2 n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "SGEBRD"
                                                           " "
                                                           n n
                                                           -1
                                                           -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-5
                                                            var-6))
                                                          (setf n
                                                                  var-3)
                                                          (setf n
                                                                  var-4)
                                                          ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
```

```
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                           (f2cl-lib:int-mul n
                                                             (multiple-value-bind
                                                                 (ret-val
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-3
                                                                  var-4
                                                                  var-5
                                                                  var-6)
                                                                 (ilaenv
                                                                  1
                                                                  "SORMBR"
                                                                  "QLN"
                                                                  n n
                                                                  n
                                                                  -1)
                                                               (declare
                                                                (ignore
                                                                 var-0
                                                                 var-1
                                                                 var-2
                                                                 var-6))
                                                               (setf n
                                                                       var-3)
                                                               (setf n
                                                                       var-4)
                                                               (setf n
                                                                       var-5)
                                                               ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                           (f2cl-lib:int-mul n
                                                             (multiple-value-bind
                                                                 (ret-val
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-3
                                                                  var-4
                                                                  var-5
                                                                  var-6)
                                                                 (ilaenv
                                                                  1
                                                                  "SORMBR"
                                                                  "PRT"
```

```
                                                                n n
                                                                n
                                                                -1)
                                                              (declare
                                                               (ignore
                                                                var-0
                                                                var-1
                                                                var-2
                                                                var-6))
                                                              (setf n
                                                                      var-3)
                                                              (setf n
                                                                      var-4)
                                                              (setf n
                                                                      var-5)
                                                              ret-val))))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add bdspac
                                        (f2cl-lib:int-mul 3 n)))))
    (setf maxwrk (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul n n)))
    (setf minwrk
            (f2cl-lib:int-add bdspac (f2cl-lib:int-mul n n)
                              (f2cl-lib:int-mul 2 n) m)))))
(t
 (setf wrkbl
         (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-add m n)
                            (multiple-value-bind
                                (ret-val var-0 var-1 var-2 var-3
                                 var-4 var-5 var-6)
                                (ilaenv 1 "SGEBRD" " " m n -1 -1)
                              (declare
                               (ignore var-0 var-1 var-2 var-5
                                var-6))
                              (setf m var-3)
                              (setf n var-4)
                              ret-val))))
 (cond
  (wntqn
   (setf maxwrk
           (max (the f2cl-lib:integer4 wrkbl)
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add bdspac
                                       (f2cl-lib:int-mul 3 n)))))
   (setf minwrk
           (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                             (max (the f2cl-lib:integer4 m)
```

```
                                    (the f2cl-lib:integer4
                                         bdspac)))))
                (wntqo
                 (setf wrkbl
                         (max (the f2cl-lib:integer4 wrkbl)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                                     (f2cl-lib:int-mul n
                                                                       (multiple-value-bind
                                                                           (ret-val
                                                                            var-0
                                                                            var-1
                                                                            var-2
                                                                            var-3
                                                                            var-4
                                                                            var-5
                                                                            var-6)
                                                                           (ilaenv
                                                                            1
                                                                            "SORMBR"
                                                                            "QLN"
                                                                            m n
                                                                            n
                                                                            -1)
                                                                         (declare
                                                                          (ignore
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-6))
                                                                         (setf m
                                                                                 var-3)
                                                                         (setf n
                                                                                 var-4)
                                                                         (setf n
                                                                                 var-5)
                                                                         ret-val))))))
                 (setf wrkbl
                         (max (the f2cl-lib:integer4 wrkbl)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                                     (f2cl-lib:int-mul n
                                                                       (multiple-value-bind
                                                                           (ret-val
                                                                            var-0
                                                                            var-1
                                                                            var-2
                                                                            var-3
                                                                            var-4
                                                                            var-5
```

```
                                                                var-6)
                                                               (ilaenv
                                                                1
                                                                "SORMBR"
                                                                "PRT"
                                                                n n
                                                                n
                                                                -1)
                                                              (declare
                                                               (ignore
                                                                var-0
                                                                var-1
                                                                var-2
                                                                var-6))
                                                              (setf n
                                                                      var-3)
                                                              (setf n
                                                                      var-4)
                                                              (setf n
                                                                      var-5)
                                                              ret-val))))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add bdspac
                                     (f2cl-lib:int-mul 3 n)))))
 (setf maxwrk (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul m n)))
 (setf minwrk
         (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                           (max (the f2cl-lib:integer4 m)
                                (the f2cl-lib:integer4
                                     (f2cl-lib:int-add
                                      (f2cl-lib:int-mul n n)
                                      bdspac))))))
(wntqs
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     (f2cl-lib:int-mul n
                                                       (multiple-value-bind
                                                           (ret-val
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-3
                                                            var-4
                                                            var-5
                                                            var-6)
                                                           (ilaenv
```

```
                                                                    1
                                                                    "SORMBR"
                                                                    "QLN"
                                                                    m n
                                                                    n
                                                                    -1)
                                                                 (declare
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-6))
                                                                 (setf m
                                                                         var-3)
                                                                 (setf n
                                                                         var-4)
                                                                 (setf n
                                                                         var-5)
                                                                 ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "SORMBR"
                                                           "PRT"
                                                           n n
                                                           n
                                                           -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-6))
                                                          (setf n
                                                                  var-3)
                                                          (setf n
                                                                  var-4)
```

```
                                                                (setf n
                                                                        var-5)
                                                                ret-val))))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add bdspac
                                            (f2cl-lib:int-mul 3 n)))))
        (setf minwrk
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                  (max (the f2cl-lib:integer4 m)
                                       (the f2cl-lib:integer4
                                            bdspac)))))
       (wntqa
        (setf wrkbl
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                            (f2cl-lib:int-mul m
                                                              (multiple-value-bind
                                                                  (ret-val
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-3
                                                                   var-4
                                                                   var-5
                                                                   var-6)
                                                                  (ilaenv
                                                                   1
                                                                   "SORMBR"
                                                                   "QLN"
                                                                   m m
                                                                   n
                                                                   -1)
                                                                (declare
                                                                 (ignore
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-6))
                                                                (setf m
                                                                        var-3)
                                                                (setf m
                                                                        var-4)
                                                                (setf n
                                                                        var-5)
                                                                ret-val))))))
        (setf wrkbl
                (max (the f2cl-lib:integer4 wrkbl)
```

```
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                           (f2cl-lib:int-mul n
                                                             (multiple-value-bind
                                                                 (ret-val
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-3
                                                                  var-4
                                                                  var-5
                                                                  var-6)
                                                                 (ilaenv
                                                                  1
                                                                  "SORMBR"
                                                                  "PRT"
                                                                  n n
                                                                  n
                                                                  -1)
                                                               (declare
                                                                (ignore
                                                                 var-0
                                                                 var-1
                                                                 var-2
                                                                 var-6))
                                                               (setf n
                                                                       var-3)
                                                               (setf n
                                                                       var-4)
                                                               (setf n
                                                                       var-5)
                                                               ret-val))))))
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add bdspac
                                              (f2cl-lib:int-mul 3 n)))))
          (setf minwrk
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (max (the f2cl-lib:integer4 m)
                                         (the f2cl-lib:integer4
                                              bdspac)))))))))
      ((> minmn 0) (setf mnthr (f2cl-lib:int (/ (* minmn 11.0) 6.0)))
       (cond (wntqn (setf bdspac (f2cl-lib:int-mul 7 m)))
             (t
              (setf bdspac
                      (f2cl-lib:int-add (f2cl-lib:int-mul 3 m m)
                                        (f2cl-lib:int-mul 4 m)))))
       (cond
        ((>= n mnthr)
```

```
(cond
 (wntqn
  (setf wrkbl
          (f2cl-lib:int-add m
                            (f2cl-lib:int-mul m
                                              (multiple-value-bind
                                                  (ret-val var-0
                                                   var-1 var-2
                                                   var-3 var-4
                                                   var-5 var-6)
                                                  (ilaenv 1
                                                   "SGELQF" " " m
                                                   n -1 -1)
                                                (declare
                                                 (ignore var-0
                                                  var-1 var-2
                                                  var-5 var-6))
                                                (setf m var-3)
                                                (setf n var-4)
                                                ret-val))))
  (setf wrkbl
          (max (the f2cl-lib:integer4 wrkbl)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                      (f2cl-lib:int-mul 2 m
                                                        (multiple-value-bind
                                                            (ret-val
                                                             var-0
                                                             var-1
                                                             var-2
                                                             var-3
                                                             var-4
                                                             var-5
                                                             var-6)
                                                            (ilaenv
                                                             1
                                                             "SGEBRD"
                                                             " "
                                                             m m
                                                             -1
                                                             -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-5
                                                            var-6))
                                                          (setf m
                                                                  var-3)
```

```
                                                                  (setf m
                                                                          var-4)
                                                                 ret-val))))))
                (setf maxwrk
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add bdspac m))))
                (setf minwrk (f2cl-lib:int-add bdspac m)))
               (wntqo
                (setf wrkbl
                        (f2cl-lib:int-add m
                                          (f2cl-lib:int-mul m
                                                            (multiple-value-bind
                                                                (ret-val var-0
                                                                 var-1 var-2
                                                                 var-3 var-4
                                                                 var-5 var-6)
                                                                (ilaenv 1
                                                                 "SGELQF" " " m
                                                                 n -1 -1)
                                                              (declare
                                                               (ignore var-0
                                                                var-1 var-2
                                                                var-5 var-6))
                                                              (setf m var-3)
                                                              (setf n var-4)
                                                              ret-val))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add m
                                                    (f2cl-lib:int-mul m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "SORGLQ"
                                                                           " "
                                                                           m n
                                                                           m
                                                                           -1)
                                                                         (declare
                                                                          (ignore
```

```
                                                                var-0
                                                                var-1
                                                                var-2
                                                                var-6))
                                                              (setf m
                                                                      var-3)
                                                              (setf n
                                                                      var-4)
                                                              (setf m
                                                                      var-5)
                                                              ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    (f2cl-lib:int-mul 2 m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "SGEBRD"
                                                           " "
                                                           m m
                                                           -1
                                                           -1)
                                                              (declare
                                                               (ignore
                                                                var-0
                                                                var-1
                                                                var-2
                                                                var-5
                                                                var-6))
                                                              (setf m
                                                                      var-3)
                                                              (setf m
                                                                      var-4)
                                                              ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
```

```
                                                                    (ret-val
                                                                     var-0
                                                                     var-1
                                                                     var-2
                                                                     var-3
                                                                     var-4
                                                                     var-5
                                                                     var-6)
                                                                    (ilaenv
                                                                     1
                                                                     "SORMBR"
                                                                     "QLN"
                                                                     m m
                                                                     m
                                                                     -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (setf m
                                                                          var-3)
                                                                  (setf m
                                                                          var-4)
                                                                  (setf m
                                                                          var-5)
                                                                  ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                                    (ret-val
                                                                     var-0
                                                                     var-1
                                                                     var-2
                                                                     var-3
                                                                     var-4
                                                                     var-5
                                                                     var-6)
                                                                    (ilaenv
                                                                     1
                                                                     "SORMBR"
                                                                     "PRT"
                                                                     m m
                                                                     m
                                                                     -1)
                                                                  (declare
```

```
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-6))
                                                                 (setf m
                                                                         var-3)
                                                                 (setf m
                                                                         var-4)
                                                                 (setf m
                                                                         var-5)
                                                                 ret-val))))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add bdspac
                                     (f2cl-lib:int-mul 3 m)))))
 (setf maxwrk (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 m m)))
 (setf minwrk
         (f2cl-lib:int-add bdspac (f2cl-lib:int-mul 2 m m)
                           (f2cl-lib:int-mul 3 m))))
(wntqs
 (setf wrkbl
         (f2cl-lib:int-add m
                           (f2cl-lib:int-mul m
                                             (multiple-value-bind
                                                 (ret-val var-0
                                                  var-1 var-2
                                                  var-3 var-4
                                                  var-5 var-6)
                                                 (ilaenv 1
                                                  "SGELQF" " " m
                                                  n -1 -1)
                                               (declare
                                                (ignore var-0
                                                 var-1 var-2
                                                 var-5 var-6))
                                               (setf m var-3)
                                               (setf n var-4)
                                               ret-val))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add m
                                     (f2cl-lib:int-mul m
                                                       (multiple-value-bind
                                                           (ret-val
                                                            var-0
                                                            var-1
                                                            var-2
```

```
                                                                var-3
                                                                var-4
                                                                var-5
                                                                var-6)
                                                               (ilaenv
                                                                1
                                                                "SORGLQ"
                                                                " "
                                                                m n
                                                                m
                                                                -1)
                                                             (declare
                                                              (ignore
                                                               var-0
                                                               var-1
                                                               var-2
                                                               var-6))
                                                             (setf m
                                                                     var-3)
                                                             (setf n
                                                                     var-4)
                                                             (setf m
                                                                     var-5)
                                                             ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    (f2cl-lib:int-mul 2 m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "SGEBRD"
                                                           " "
                                                           m m
                                                           -1
                                                           -1)
                                                             (declare
                                                              (ignore
                                                               var-0
                                                               var-1
                                                               var-2
```

```
                                                                var-5
                                                                var-6))
                                                              (setf m
                                                                      var-3)
                                                              (setf m
                                                                      var-4)
                                                              ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                                    (f2cl-lib:int-mul m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "SORMBR"
                                                                           "QLN"
                                                                           m m
                                                                           m
                                                                           -1)
                                                                          (declare
                                                                           (ignore
                                                                            var-0
                                                                            var-1
                                                                            var-2
                                                                            var-6))
                                                                          (setf m
                                                                                  var-3)
                                                                          (setf m
                                                                                  var-4)
                                                                          (setf m
                                                                                  var-5)
                                                                          ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                                    (f2cl-lib:int-mul m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
```

```
                                                                    var-2
                                                                    var-3
                                                                    var-4
                                                                    var-5
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "SORMBR"
                                                                    "PRT"
                                                                    m m
                                                                    m
                                                                    -1)
                                                                 (declare
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-6))
                                                                 (setf m
                                                                         var-3)
                                                                 (setf m
                                                                         var-4)
                                                                 (setf m
                                                                         var-5)
                                                                 ret-val))))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add bdspac
                                     (f2cl-lib:int-mul 3 m)))))
 (setf maxwrk (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul m m)))
 (setf minwrk
         (f2cl-lib:int-add bdspac (f2cl-lib:int-mul m m)
                           (f2cl-lib:int-mul 3 m))))
(wntqa
 (setf wrkbl
         (f2cl-lib:int-add m
                           (f2cl-lib:int-mul m
                                             (multiple-value-bind
                                                 (ret-val var-0
                                                  var-1 var-2
                                                  var-3 var-4
                                                  var-5 var-6)
                                                 (ilaenv 1
                                                  "SGELQF" " " m
                                                  n -1 -1)
                                                (declare
                                                 (ignore var-0
                                                  var-1 var-2
                                                  var-5 var-6))
```

```
                                                    (setf m var-3)
                                                    (setf n var-4)
                                                    ret-val))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add m
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "SORGLQ"
                                                           " "
                                                           n n
                                                           m
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (setf n
                                                                var-3)
                                                        (setf n
                                                                var-4)
                                                        (setf m
                                                                var-5)
                                                        ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    (f2cl-lib:int-mul 2 m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
```

```
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "SGEBRD"
                                                                    " "
                                                                    m m
                                                                    -1
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-5
                                                                    var-6))
                                                                  (setf m
                                                                          var-3)
                                                                  (setf m
                                                                          var-4)
                                                                  ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                                    (f2cl-lib:int-mul m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "SORMBR"
                                                                           "QLN"
                                                                           m m
                                                                           m
                                                                           -1)
                                                                          (declare
                                                                           (ignore
                                                                            var-0
                                                                            var-1
                                                                            var-2
                                                                            var-6))
                                                                          (setf m
                                                                                  var-3)
                                                                          (setf m
```

```
                                                                        var-4)
                                                                (setf m
                                                                        var-5)
                                                                ret-val))))))
        (setf wrkbl
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                            (f2cl-lib:int-mul m
                                                              (multiple-value-bind
                                                                  (ret-val
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-3
                                                                   var-4
                                                                   var-5
                                                                   var-6)
                                                                  (ilaenv
                                                                   1
                                                                   "SORMBR"
                                                                   "PRT"
                                                                   m m
                                                                   m
                                                                   -1)
                                                                (declare
                                                                 (ignore
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-6))
                                                                (setf m
                                                                        var-3)
                                                                (setf m
                                                                        var-4)
                                                                (setf m
                                                                        var-5)
                                                                ret-val))))))
        (setf wrkbl
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add bdspac
                                            (f2cl-lib:int-mul 3 m)))))
        (setf maxwrk (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul m m)))
        (setf minwrk
                (f2cl-lib:int-add bdspac (f2cl-lib:int-mul m m)
                                  (f2cl-lib:int-mul 3 m))))))
    (t
     (setf wrkbl
             (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
```

```
                                (f2cl-lib:int-mul
                                 (f2cl-lib:int-add m n)
                                 (multiple-value-bind
                                     (ret-val var-0 var-1 var-2 var-3
                                      var-4 var-5 var-6)
                                     (ilaenv 1 "SGEBRD" " " m n -1 -1)
                                   (declare
                                    (ignore var-0 var-1 var-2 var-5
                                     var-6))
                                   (setf m var-3)
                                   (setf n var-4)
                                   ret-val))))
                (cond
                 (wntqn
                  (setf maxwrk
                          (max (the f2cl-lib:integer4 wrkbl)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add bdspac
                                                      (f2cl-lib:int-mul 3 m)))))
                  (setf minwrk
                          (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                            (max (the f2cl-lib:integer4 n)
                                                 (the f2cl-lib:integer4
                                                      bdspac)))))
                 (wntqo
                  (setf wrkbl
                          (max (the f2cl-lib:integer4 wrkbl)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                                      (f2cl-lib:int-mul m
                                                                        (multiple-value-bind
                                                                            (ret-val
                                                                             var-0
                                                                             var-1
                                                                             var-2
                                                                             var-3
                                                                             var-4
                                                                             var-5
                                                                             var-6)
                                                                            (ilaenv
                                                                             1
                                                                             "SORMBR"
                                                                             "QLN"
                                                                             m m
                                                                             n
                                                                             -1)
                                                                          (declare
                                                                           (ignore
                                                                            var-0
                                                                            var-1
```

```
                                                                var-2
                                                                var-6))
                                                              (setf m
                                                                      var-3)
                                                              (setf m
                                                                      var-4)
                                                              (setf n
                                                                      var-5)
                                                              ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "SORMBR"
                                                           "PRT"
                                                           m n
                                                           m
                                                           -1)
                                                              (declare
                                                               (ignore
                                                                var-0
                                                                var-1
                                                                var-2
                                                                var-6))
                                                              (setf m
                                                                      var-3)
                                                              (setf n
                                                                      var-4)
                                                              (setf m
                                                                      var-5)
                                                              ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add bdspac
                                    (f2cl-lib:int-mul 3 m)))))
(setf maxwrk (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul m n)))
(setf minwrk
```

```
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                  (max (the f2cl-lib:integer4 n)
                                       (the f2cl-lib:integer4
                                            (f2cl-lib:int-add
                                             (f2cl-lib:int-mul m m)
                                             bdspac))))))
        (wntqs
         (setf wrkbl
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                            (f2cl-lib:int-mul m
                                                              (multiple-value-bind
                                                                  (ret-val
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-3
                                                                   var-4
                                                                   var-5
                                                                   var-6)
                                                                  (ilaenv
                                                                   1
                                                                   "SORMBR"
                                                                   "QLN"
                                                                   m m
                                                                   n
                                                                   -1)
                                                                (declare
                                                                 (ignore
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-6))
                                                                (setf m
                                                                        var-3)
                                                                (setf m
                                                                        var-4)
                                                                (setf n
                                                                        var-5)
                                                                ret-val))))))
         (setf wrkbl
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                            (f2cl-lib:int-mul m
                                                              (multiple-value-bind
                                                                  (ret-val
                                                                   var-0
                                                                   var-1
```

```
                                                                var-2
                                                                var-3
                                                                var-4
                                                                var-5
                                                                var-6)
                                                               (ilaenv
                                                                1
                                                                "SORMBR"
                                                                "PRT"
                                                                m n
                                                                m
                                                                -1)
                                                              (declare
                                                               (ignore
                                                                var-0
                                                                var-1
                                                                var-2
                                                                var-6))
                                                              (setf m
                                                                      var-3)
                                                              (setf n
                                                                      var-4)
                                                              (setf m
                                                                      var-5)
                                                              ret-val))))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add bdspac
                                     (f2cl-lib:int-mul 3 m)))))
 (setf minwrk
         (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                           (max (the f2cl-lib:integer4 n)
                                (the f2cl-lib:integer4
                                     bdspac)))))
(wntqa
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     (f2cl-lib:int-mul m
                                                       (multiple-value-bind
                                                           (ret-val
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-3
                                                            var-4
                                                            var-5
                                                            var-6)
```

```
                                                                    (ilaenv
                                                                     1
                                                                     "SORMBR"
                                                                     "QLN"
                                                                     m m
                                                                     n
                                                                     -1)
                                                                   (declare
                                                                    (ignore
                                                                     var-0
                                                                     var-1
                                                                     var-2
                                                                     var-6))
                                                                   (setf m
                                                                           var-3)
                                                                   (setf m
                                                                           var-4)
                                                                   (setf n
                                                                           var-5)
                                                                   ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "SORMBR"
                                                           "PRT"
                                                           n n
                                                           m
                                                           -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-6))
                                                          (setf n
                                                                  var-3)
                                                          (setf n
```

```
                                                                                var-4)
                                                                          (setf m
                                                                                var-5)
                                                                          ret-val))))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add bdspac
                                            (f2cl-lib:int-mul 3 m)))))
        (setf minwrk
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                  (max (the f2cl-lib:integer4 n)
                                       (the f2cl-lib:integer4
                                            bdspac))))))))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4 minwrk)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 maxwrk) 'single-float))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -12)))))
(cond
 ((/= info 0) (xerbla "SGESDD" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond ((or (= m 0) (= n 0)) (go end_label)))
(setf eps (slamch "P"))
(setf smlnum (/ (f2cl-lib:fsqrt (slamch "S")) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "M" m n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iscl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf iscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 anrm smlnum m n a lda ierr)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf ierr var-9))))
 ((> anrm bignum) (setf iscl 1)
```

```
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl "G" 0 0 anrm bignum m n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf bignum var-4))
      (when var-5 (setf m var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9)))))
  (cond
   ((>= m n)
    (cond
     ((>= m mnthr)
      (cond
       (wntqn (setf itau 1) (setf nwork (f2cl-lib:int-add itau n))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (sgeqrf m n a lda
             (f2cl-lib:array-slice work-%data% single-float (itau)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:array-slice work-%data% single-float (nwork)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
          (declare (ignore var-2 var-4 var-5 var-6))
          (setf m var-0)
          (setf n var-1)
          (setf lda var-3)
          (setf ierr var-7))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (slaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
             zero zero
             (f2cl-lib:array-slice a-%data% single-float (2 1)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-1 var-2 var-5))
          (when var-3 (setf zero var-3))
          (when var-4 (setf zero var-4))
          (when var-6 (setf lda var-6)))
        (setf ie 1) (setf itauq (f2cl-lib:int-add ie n))
        (setf itaup (f2cl-lib:int-add itauq n))
        (setf nwork (f2cl-lib:int-add itaup n))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10)
            (sgebrd n n a lda s
             (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                                   work-%offset%)
             (f2cl-lib:array-slice work-%data% single-float (itauq)
                                   ((1 *)) work-%offset%)
```

```
      (f2cl-lib:array-slice work-%data% single-float (itaup)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
    (setf n var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-10))
  (setf nwork (f2cl-lib:int-add ie n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (sbdsdc "U" "N" n s
       (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                             work-%offset%)
       dum 1 dum 1 dum idum
       (f2cl-lib:array-slice work-%data% single-float (nwork)
                             ((1 *)) work-%offset%)
       iwork info)
    (declare
     (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12))
    (setf n var-2)
    (setf info var-13)))
 (wntqo (setf ir 1)
  (cond
   ((>= lwork
        (f2cl-lib:int-add (f2cl-lib:int-mul lda n)
                          (f2cl-lib:int-mul n n)
                          (f2cl-lib:int-mul 3 n) bdspac))
    (setf ldwrkr lda))
   (t
    (setf ldwrkr
            (the f2cl-lib:integer4
                 (truncate (- lwork (* n n) (* 3 n) bdspac) n)))))
  (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
  (setf nwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (sgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% single-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
```

```
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% single-float (ir) ((1 *))
                           work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice work-%data% single-float ((+ ir 1))
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorgqr m n n a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf nwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (sgebrd n n
     (f2cl-lib:array-slice work-%data% single-float (ir) ((1 *))
                           work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
```

```
      (f2cl-lib:array-slice work-%data% single-float (itaup)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(setf iu nwork)
(setf nwork (f2cl-lib:int-add iu (f2cl-lib:int-mul n n)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sbdsdc "U" "I" n s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iu) ((1 *))
                           work-%offset%)
     n vt ldvt dum idum
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (setf n var-2)
  (setf n var-6)
  (setf ldvt var-8)
  (setf info var-13))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "Q" "L" "N" n n n
     (f2cl-lib:array-slice work-%data% single-float (ir) ((1 *))
                           work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iu) ((1 *))
                           work-%offset%)
     n
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
```

```
  (when var-5 (setf n var-5))
  (when var-7 (setf ldwrkr var-7))
  (when var-10 (setf n var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "P" "R" "T" n n n
     (f2cl-lib:array-slice work-%data% single-float (ir) ((1 *))
                           work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldwrkr var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i ldwrkr))
              ((> i m) nil)
  (tagbody
    (setf chunk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
             (the f2cl-lib:integer4 ldwrkr)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (sgemm "N" "N" chunk n n one
         (f2cl-lib:array-slice a-%data% single-float (i 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice work-%data% single-float (iu)
                               ((1 *)) work-%offset%)
         n zero
         (f2cl-lib:array-slice work-%data% single-float (ir)
                               ((1 *)) work-%offset%)
         ldwrkr)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf chunk var-2))
      (when var-3 (setf n var-3))
      (when var-4 (setf n var-4))
```

```
          (when var-5 (setf one var-5))
          (when var-7 (setf lda var-7))
          (when var-9 (setf n var-9))
          (when var-10 (setf zero var-10))
          (when var-12 (setf ldwrkr var-12)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (slacpy "F" chunk n
             (f2cl-lib:array-slice work-%data% single-float (ir)
                                   ((1 *)) work-%offset%)
             ldwrkr
             (f2cl-lib:array-slice a-%data% single-float (i 1)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf chunk var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf ldwrkr var-4))
          (when var-6 (setf lda var-6)))
      label10)))
  (wntqs (setf ir 1) (setf ldwrkr n)
   (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
   (setf nwork (f2cl-lib:int-add itau n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (sgeqrf m n a lda
        (f2cl-lib:array-slice work-%data% single-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-7))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slacpy "U" n n a lda
        (f2cl-lib:array-slice work-%data% single-float (ir) ((1 *))
                              work-%offset%)
        ldwrkr)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf n var-1))
     (when var-2 (setf n var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf ldwrkr var-6)))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
        zero zero
        (f2cl-lib:array-slice work-%data% single-float ((+ ir 1))
```

```
                          ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorgqr m n n a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf nwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (sgebrd n n
     (f2cl-lib:array-slice work-%data% single-float (ir) ((1 *))
                           work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sbdsdc "U" "I" n s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
```

```
      u ldu vt ldvt dum idum
      (f2cl-lib:array-slice work-%data% single-float (nwork)
                            ((1 *)) work-%offset%)
      iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (setf n var-2)
  (setf ldu var-6)
  (setf ldvt var-8)
  (setf info var-13))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "Q" "L" "N" n n n
     (f2cl-lib:array-slice work-%data% single-float (ir) ((1 *))
                           work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldwrkr var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "P" "R" "T" n n n
     (f2cl-lib:array-slice work-%data% single-float (ir) ((1 *))
                           work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
```

```
    (when var-7 (setf ldwrkr var-7))
    (when var-10 (setf ldvt var-10))
    (when var-13 (setf ierr var-13)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "F" n n u ldu
       (f2cl-lib:array-slice work-%data% single-float (ir) ((1 *))
                             work-%offset%)
       ldwrkr)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldu var-4))
    (when var-6 (setf ldwrkr var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sgemm "N" "N" m n n one a lda
       (f2cl-lib:array-slice work-%data% single-float (ir) ((1 *))
                             work-%offset%)
       ldwrkr zero u ldu)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldwrkr var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldu var-12))))
 (wntqa (setf iu 1) (setf ldwrku n)
  (setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
  (setf nwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (sgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% single-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
```

```
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorgqr m m n u ldu
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice a-%data% single-float (2 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf nwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (sgebrd n n a lda s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
```

```
    (sbdsdc "U" "I" n s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iu) ((1 *))
                           work-%offset%)
     n vt ldvt dum idum
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (setf n var-2)
  (setf n var-6)
  (setf ldvt var-8)
  (setf info var-13))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "Q" "L" "N" n n n a lda
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iu) ((1 *))
                           work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldwrku var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "P" "R" "T" n n n a lda
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
```

```
        (when var-5 (setf n var-5))
        (when var-7 (setf lda var-7))
        (when var-10 (setf ldvt var-10))
        (when var-13 (setf ierr var-13)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (sgemm "N" "N" m n n one u ldu
           (f2cl-lib:array-slice work-%data% single-float (iu) ((1 *))
                                 work-%offset%)
           ldwrku zero a lda)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf n var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf one var-5))
        (when var-7 (setf ldu var-7))
        (when var-9 (setf ldwrku var-9))
        (when var-10 (setf zero var-10))
        (when var-12 (setf lda var-12)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slacpy "F" m n a lda u ldu)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldu var-6))))))
    (t (setf ie 1) (setf itauq (f2cl-lib:int-add ie n))
     (setf itaup (f2cl-lib:int-add itauq n))
     (setf nwork (f2cl-lib:int-add itaup n))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (sgebrd m n a lda s
          (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                                work-%offset%)
          (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
                                work-%offset%)
          (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                                work-%offset%)
          (f2cl-lib:array-slice work-%data% single-float (nwork) ((1 *))
                                work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
       (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
       (setf m var-0)
       (setf n var-1)
       (setf lda var-3)
       (setf ierr var-10))
     (cond
      (wntqn
```

```
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13)
     (sbdsdc "U" "N" n s
      (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                            work-%offset%)
      dum 1 dum 1 dum idum
      (f2cl-lib:array-slice work-%data% single-float (nwork)
                            ((1 *)) work-%offset%)
      iwork info)
   (declare
    (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12))
   (setf n var-2)
   (setf info var-13)))
(wntqo (setf iu nwork)
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                         (f2cl-lib:int-mul 3 n) bdspac))
   (setf ldwrku m)
   (setf nwork (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slaset "F" m n zero zero
        (f2cl-lib:array-slice work-%data% single-float (iu)
                              ((1 *)) work-%offset%)
        ldwrku)
     (declare (ignore var-0 var-5))
     (when var-1 (setf m var-1))
     (when var-2 (setf n var-2))
     (when var-3 (setf zero var-3))
     (when var-4 (setf zero var-4))
     (when var-6 (setf ldwrku var-6))))
  (t (setf ldwrku n)
   (setf nwork (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
   (setf ir nwork)
   (setf ldwrkr
           (the f2cl-lib:integer4
                (truncate (- lwork (* n n) (* 3 n)) n)))))
 (setf nwork (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13)
     (sbdsdc "U" "I" n s
      (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (iu) ((1 *))
                            work-%offset%)
      ldwrku vt ldvt dum idum
```

```
      (f2cl-lib:array-slice work-%data% single-float (nwork)
                            ((1 *)) work-%offset%)
      iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (setf n var-2)
  (setf ldwrku var-6)
  (setf ldvt var-8)
  (setf info var-13))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "P" "R" "T" n n n a lda
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(cond
 ((>= lwork
      (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                        (f2cl-lib:int-mul 3 n) bdspac))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13)
      (sormbr "Q" "L" "N" m n n a lda
       (f2cl-lib:array-slice work-%data% single-float (itauq)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iu)
                             ((1 *)) work-%offset%)
       ldwrku
       (f2cl-lib:array-slice work-%data% single-float (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf n var-5))
    (when var-7 (setf lda var-7))
```

```
    (when var-10 (setf ldwrku var-10))
    (when var-13 (setf ierr var-13)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "F" m n
       (f2cl-lib:array-slice work-%data% single-float (iu)
                             ((1 *)) work-%offset%)
       ldwrku a lda)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldwrku var-4))
    (when var-6 (setf lda var-6))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (sorgbr "Q" m n n a lda
       (f2cl-lib:array-slice work-%data% single-float (itauq)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i ldwrkr))
                ((> i m) nil)
    (tagbody
      (setf chunk
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
               (the f2cl-lib:integer4 ldwrkr)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (sgemm "N" "N" chunk n n one
           (f2cl-lib:array-slice a-%data% single-float (i 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice work-%data% single-float (iu)
                                 ((1 *)) work-%offset%)
           ldwrku zero
           (f2cl-lib:array-slice work-%data% single-float (ir)
                                 ((1 *)) work-%offset%)
           ldwrkr)
```

```
            (declare (ignore var-0 var-1 var-6 var-8 var-11))
            (when var-2 (setf chunk var-2))
            (when var-3 (setf n var-3))
            (when var-4 (setf n var-4))
            (when var-5 (setf one var-5))
            (when var-7 (setf lda var-7))
            (when var-9 (setf ldwrku var-9))
            (when var-10 (setf zero var-10))
            (when var-12 (setf ldwrkr var-12)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (slacpy "F" chunk n
               (f2cl-lib:array-slice work-%data% single-float (ir)
                                     ((1 *)) work-%offset%)
               ldwrkr
               (f2cl-lib:array-slice a-%data% single-float (i 1)
                                     ((1 lda) (1 *)) a-%offset%)
               lda)
            (declare (ignore var-0 var-3 var-5))
            (when var-1 (setf chunk var-1))
            (when var-2 (setf n var-2))
            (when var-4 (setf ldwrkr var-4))
            (when var-6 (setf lda var-6)))
        label20)))))
  (wntqs
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slaset "F" m n zero zero u ldu)
     (declare (ignore var-0 var-5))
     (when var-1 (setf m var-1))
     (when var-2 (setf n var-2))
     (when var-3 (setf zero var-3))
     (when var-4 (setf zero var-4))
     (when var-6 (setf ldu var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13)
       (sbdsdc "U" "I" n s
        (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                              work-%offset%)
        u ldu vt ldvt dum idum
        (f2cl-lib:array-slice work-%data% single-float (nwork)
                              ((1 *)) work-%offset%)
        iwork info)
     (declare
      (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
       var-11 var-12))
     (setf n var-2)
     (setf ldu var-6)
     (setf ldvt var-8)
     (setf info var-13))
```

```
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13)
     (sormbr "Q" "L" "N" m n n a lda
      (f2cl-lib:array-slice work-%data% single-float (itauq)
                            ((1 *)) work-%offset%)
      u ldu
      (f2cl-lib:array-slice work-%data% single-float (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
   (declare
    (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
   (when var-3 (setf m var-3))
   (when var-4 (setf n var-4))
   (when var-5 (setf n var-5))
   (when var-7 (setf lda var-7))
   (when var-10 (setf ldu var-10))
   (when var-13 (setf ierr var-13)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13)
     (sormbr "P" "R" "T" n n n a lda
      (f2cl-lib:array-slice work-%data% single-float (itaup)
                            ((1 *)) work-%offset%)
      vt ldvt
      (f2cl-lib:array-slice work-%data% single-float (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
   (declare
    (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
   (when var-3 (setf n var-3))
   (when var-4 (setf n var-4))
   (when var-5 (setf n var-5))
   (when var-7 (setf lda var-7))
   (when var-10 (setf ldvt var-10))
   (when var-13 (setf ierr var-13))))
(wntqa
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slaset "F" m m zero zero u ldu)
   (declare (ignore var-0 var-5))
   (when var-1 (setf m var-1))
   (when var-2 (setf m var-2))
   (when var-3 (setf zero var-3))
   (when var-4 (setf zero var-4))
   (when var-6 (setf ldu var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13)
     (sbdsdc "U" "I" n s
      (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
```

```
                              work-%offset%)
        u ldu vt ldvt dum idum
        (f2cl-lib:array-slice work-%data% single-float (nwork)
                              ((1 *)) work-%offset%)
        iwork info)
    (declare
     (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
      var-11 var-12))
    (setf n var-2)
    (setf ldu var-6)
    (setf ldvt var-8)
    (setf info var-13))
  (cond
   ((> m n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "F" (f2cl-lib:int-sub m n) (f2cl-lib:int-sub m n)
         zero one
         (f2cl-lib:array-slice u-%data% single-float
                               ((+ n 1) (f2cl-lib:int-add n 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldu var-6)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (sormbr "Q" "L" "N" m m n a lda
       (f2cl-lib:array-slice work-%data% single-float (itauq)
                             ((1 *)) work-%offset%)
       u ldu
       (f2cl-lib:array-slice work-%data% single-float (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf n var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldu var-10))
    (when var-13 (setf ierr var-13)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (sormbr "P" "R" "T" n n m a lda
       (f2cl-lib:array-slice work-%data% single-float (itaup)
                             ((1 *)) work-%offset%)
```

```
            vt ldvt
            (f2cl-lib:array-slice work-%data% single-float (nwork)
                                  ((1 *)) work-%offset%)
            (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
          (declare
           (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
          (when var-3 (setf n var-3))
          (when var-4 (setf n var-4))
          (when var-5 (setf m var-5))
          (when var-7 (setf lda var-7))
          (when var-10 (setf ldvt var-10))
          (when var-13 (setf ierr var-13))))))))
  (t
   (cond
    ((>= n mnthr)
     (cond
      (wntqn (setf itau 1) (setf nwork (f2cl-lib:int-add itau m))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
           (sgelqf m n a lda
            (f2cl-lib:array-slice work-%data% single-float (itau)
                                  ((1 *)) work-%offset%)
            (f2cl-lib:array-slice work-%data% single-float (nwork)
                                  ((1 *)) work-%offset%)
            (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
         (declare (ignore var-2 var-4 var-5 var-6))
         (setf m var-0)
         (setf n var-1)
         (setf lda var-3)
         (setf ierr var-7))
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
           (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
            zero zero
            (f2cl-lib:array-slice a-%data% single-float (1 2)
                                  ((1 lda) (1 *)) a-%offset%)
            lda)
         (declare (ignore var-0 var-1 var-2 var-5))
         (when var-3 (setf zero var-3))
         (when var-4 (setf zero var-4))
         (when var-6 (setf lda var-6)))
       (setf ie 1) (setf itauq (f2cl-lib:int-add ie m))
       (setf itaup (f2cl-lib:int-add itauq m))
       (setf nwork (f2cl-lib:int-add itaup m))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10)
           (sgebrd m m a lda s
            (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                                  work-%offset%)
            (f2cl-lib:array-slice work-%data% single-float (itauq)
```

```
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% single-float (itaup)
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% single-float (nwork)
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
      (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
      (setf m var-0)
      (setf m var-1)
      (setf lda var-3)
      (setf ierr var-10))
    (setf nwork (f2cl-lib:int-add ie m))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (sbdsdc "U" "N" m s
         (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                               work-%offset%)
         dum 1 dum 1 dum idum
         (f2cl-lib:array-slice work-%data% single-float (nwork)
                               ((1 *)) work-%offset%)
         iwork info)
      (declare
       (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12))
      (setf m var-2)
      (setf info var-13)))
  (wntqo (setf ivt 1)
    (setf il (f2cl-lib:int-add ivt (f2cl-lib:int-mul m m)))
    (cond
     ((>= lwork
          (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                            (f2cl-lib:int-mul m m)
                            (f2cl-lib:int-mul 3 m) bdspac))
      (setf ldwrkl m) (setf chunk n))
     (t (setf ldwrkl m)
      (setf chunk
              (the f2cl-lib:integer4
                   (truncate (- lwork (* m m)) m)))))
    (setf itau (f2cl-lib:int-add il (f2cl-lib:int-mul ldwrkl m)))
    (setf nwork (f2cl-lib:int-add itau m))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (sgelqf m n a lda
         (f2cl-lib:array-slice work-%data% single-float (itau)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% single-float (nwork)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
      (declare (ignore var-2 var-4 var-5 var-6))
```

```
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                           work-%offset%)
     ldwrkl)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkl var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ il ldwrkl)) ((1 *)) work-%offset%)
     ldwrkl)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrkl var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorglq m n m a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf nwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (sgebrd m m
     (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                           work-%offset%)
     ldwrkl s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
```

```
      (f2cl-lib:array-slice work-%data% single-float (itauq)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (itaup)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkl var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sbdsdc "U" "I" m s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% single-float (ivt) ((1 *))
                           work-%offset%)
     m dum idum
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (setf m var-2)
  (setf ldu var-6)
  (setf m var-8)
  (setf info var-13))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "Q" "L" "N" m m m
     (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                           work-%offset%)
     ldwrkl
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
```

```
  (when var-7 (setf ldwrkl var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "P" "R" "T" m m m
     (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                           work-%offset%)
     ldwrkl
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (ivt) ((1 *))
                           work-%offset%)
     m
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwrkl var-7))
  (when var-10 (setf m var-10))
  (when var-13 (setf ierr var-13)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
              ((> i n) nil)
  (tagbody
    (setf blk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
             (the f2cl-lib:integer4 chunk)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (sgemm "N" "N" m blk m one
         (f2cl-lib:array-slice work-%data% single-float (ivt)
                               ((1 *)) work-%offset%)
         m
         (f2cl-lib:array-slice a-%data% single-float (1 i)
                               ((1 lda) (1 *)) a-%offset%)
         lda zero
         (f2cl-lib:array-slice work-%data% single-float (il)
                               ((1 *)) work-%offset%)
         ldwrkl)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf blk var-3))
```

```
          (when var-4 (setf m var-4))
          (when var-5 (setf one var-5))
          (when var-7 (setf m var-7))
          (when var-9 (setf lda var-9))
          (when var-10 (setf zero var-10))
          (when var-12 (setf ldwrkl var-12)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (slacpy "F" m blk
             (f2cl-lib:array-slice work-%data% single-float (il)
                                   ((1 *)) work-%offset%)
             ldwrkl
             (f2cl-lib:array-slice a-%data% single-float (1 i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf blk var-2))
          (when var-4 (setf ldwrkl var-4))
          (when var-6 (setf lda var-6)))
      label30)))
  (wntqs (setf il 1) (setf ldwrkl m)
   (setf itau (f2cl-lib:int-add il (f2cl-lib:int-mul ldwrkl m)))
   (setf nwork (f2cl-lib:int-add itau m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (sgelqf m n a lda
        (f2cl-lib:array-slice work-%data% single-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-7))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slacpy "L" m m a lda
        (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                              work-%offset%)
        ldwrkl)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf m var-1))
     (when var-2 (setf m var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf ldwrkl var-6)))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
        zero zero
```

```
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ il ldwrkl)) ((1 *)) work-%offset%)
     ldwrkl)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrkl var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorglq m n m a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf nwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (sgebrd m m
     (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                           work-%offset%)
     ldwrkl s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkl var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sbdsdc "U" "I" m s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
```

```
                          work-%offset%)
      u ldu vt ldvt dum idum
      (f2cl-lib:array-slice work-%data% single-float (nwork)
                          ((1 *)) work-%offset%)
      iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (setf m var-2)
  (setf ldu var-6)
  (setf ldvt var-8)
  (setf info var-13))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "Q" "L" "N" m m m
     (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                          work-%offset%)
     ldwrkl
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                          ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                          ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwrkl var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "P" "R" "T" m m m
     (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                          work-%offset%)
     ldwrkl
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                          ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                          ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
```

```
    (when var-5 (setf m var-5))
    (when var-7 (setf ldwrkl var-7))
    (when var-10 (setf ldvt var-10))
    (when var-13 (setf ierr var-13)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "F" m m vt ldvt
       (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                             work-%offset%)
       ldwrkl)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf ldvt var-4))
    (when var-6 (setf ldwrkl var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sgemm "N" "N" m n m one
       (f2cl-lib:array-slice work-%data% single-float (il) ((1 *))
                             work-%offset%)
       ldwrkl a lda zero vt ldvt)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldwrkl var-7))
    (when var-9 (setf lda var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldvt var-12))))
 (wntqa (setf ivt 1) (setf ldwkvt m)
  (setf itau (f2cl-lib:int-add ivt (f2cl-lib:int-mul ldwkvt m)))
  (setf nwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (sgelqf m n a lda
       (f2cl-lib:array-slice work-%data% single-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
```

```
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorglq n n m vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice a-%data% single-float (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf nwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (sgebrd m m a lda s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
     var-10 var-11 var-12 var-13)
    (sbdsdc "U" "I" m s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% single-float (ivt) ((1 *))
                           work-%offset%)
     ldwkvt dum idum
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (setf m var-2)
  (setf ldu var-6)
  (setf ldwkvt var-8)
  (setf info var-13))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "Q" "L" "N" m m m a lda
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "P" "R" "T" m m m a lda
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (ivt) ((1 *))
                           work-%offset%)
     ldwkvt
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
```

```
        (when var-3 (setf m var-3))
        (when var-4 (setf m var-4))
        (when var-5 (setf m var-5))
        (when var-7 (setf lda var-7))
        (when var-10 (setf ldwkvt var-10))
        (when var-13 (setf ierr var-13)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (sgemm "N" "N" m n m one
           (f2cl-lib:array-slice work-%data% single-float (ivt) ((1 *))
                                 work-%offset%)
           ldwkvt vt ldvt zero a lda)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf n var-3))
        (when var-4 (setf m var-4))
        (when var-5 (setf one var-5))
        (when var-7 (setf ldwkvt var-7))
        (when var-9 (setf ldvt var-9))
        (when var-10 (setf zero var-10))
        (when var-12 (setf lda var-12)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slacpy "F" m n a lda vt ldvt)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldvt var-6))))))
    (t (setf ie 1) (setf itauq (f2cl-lib:int-add ie m))
     (setf itaup (f2cl-lib:int-add itauq m))
     (setf nwork (f2cl-lib:int-add itaup m))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (sgebrd m n a lda s
          (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                                work-%offset%)
          (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
                                work-%offset%)
          (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                                work-%offset%)
          (f2cl-lib:array-slice work-%data% single-float (nwork) ((1 *))
                                work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
      (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
      (setf m var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf ierr var-10))
```

```
(cond
 (wntqn
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (sbdsdc "L" "N" m s
       (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                             work-%offset%)
       dum 1 dum 1 dum idum
       (f2cl-lib:array-slice work-%data% single-float (nwork)
                             ((1 *)) work-%offset%)
       iwork info)
    (declare
     (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12))
    (setf m var-2)
    (setf info var-13)))
 (wntqo (setf ldwkvt m) (setf ivt nwork)
  (cond
   ((>= lwork
        (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                          (f2cl-lib:int-mul 3 m) bdspac))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "F" m n zero zero
         (f2cl-lib:array-slice work-%data% single-float (ivt)
                               ((1 *)) work-%offset%)
         ldwkvt)
      (declare (ignore var-0 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldwkvt var-6)))
    (setf nwork
            (f2cl-lib:int-add ivt (f2cl-lib:int-mul ldwkvt n))))
   (t
    (setf nwork (f2cl-lib:int-add ivt (f2cl-lib:int-mul ldwkvt m)))
    (setf il nwork)
    (setf chunk
            (the f2cl-lib:integer4
                 (truncate (- lwork (* m m) (* 3 m)) m)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (sbdsdc "L" "I" m s
       (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                             work-%offset%)
       u ldu
       (f2cl-lib:array-slice work-%data% single-float (ivt) ((1 *))
```

```
                              work-%offset%)
      ldwkvt dum idum
      (f2cl-lib:array-slice work-%data% single-float (nwork)
                              ((1 *)) work-%offset%)
      iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (setf m var-2)
  (setf ldu var-6)
  (setf ldwkvt var-8)
  (setf info var-13))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "Q" "L" "N" m m n a lda
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                              ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                              ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(cond
 ((>= lwork
      (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                        (f2cl-lib:int-mul 3 m) bdspac))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13)
      (sormbr "P" "R" "T" m n m a lda
       (f2cl-lib:array-slice work-%data% single-float (itaup)
                               ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (ivt)
                               ((1 *)) work-%offset%)
       ldwkvt
       (f2cl-lib:array-slice work-%data% single-float (nwork)
                               ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf n var-4))
```

```
    (when var-5 (setf m var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldwkvt var-10))
    (when var-13 (setf ierr var-13)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "F" m n
       (f2cl-lib:array-slice work-%data% single-float (ivt)
                             ((1 *)) work-%offset%)
       ldwkvt a lda)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldwkvt var-4))
    (when var-6 (setf lda var-6))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (sorgbr "P" m n m a lda
       (f2cl-lib:array-slice work-%data% single-float (itaup)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
                ((> i n) nil)
    (tagbody
      (setf blk
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
               (the f2cl-lib:integer4 chunk)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (sgemm "N" "N" m blk m one
           (f2cl-lib:array-slice work-%data% single-float (ivt)
                                 ((1 *)) work-%offset%)
           ldwkvt
           (f2cl-lib:array-slice a-%data% single-float (1 i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda zero
           (f2cl-lib:array-slice work-%data% single-float (il)
```

```
                                    ((1 *)) work-%offset%)
                m)
              (declare (ignore var-0 var-1 var-6 var-8 var-11))
              (when var-2 (setf m var-2))
              (when var-3 (setf blk var-3))
              (when var-4 (setf m var-4))
              (when var-5 (setf one var-5))
              (when var-7 (setf ldwkvt var-7))
              (when var-9 (setf lda var-9))
              (when var-10 (setf zero var-10))
              (when var-12 (setf m var-12)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (slacpy "F" m blk
                 (f2cl-lib:array-slice work-%data% single-float (il)
                                    ((1 *)) work-%offset%)
                 m
                 (f2cl-lib:array-slice a-%data% single-float (1 i)
                                    ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare (ignore var-0 var-3 var-5))
              (when var-1 (setf m var-1))
              (when var-2 (setf blk var-2))
              (when var-4 (setf m var-4))
              (when var-6 (setf lda var-6)))
          label40)))))
    (wntqs
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (slaset "F" m n zero zero vt ldvt)
       (declare (ignore var-0 var-5))
       (when var-1 (setf m var-1))
       (when var-2 (setf n var-2))
       (when var-3 (setf zero var-3))
       (when var-4 (setf zero var-4))
       (when var-6 (setf ldvt var-6)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12 var-13)
         (sbdsdc "L" "I" m s
          (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                                work-%offset%)
          u ldu vt ldvt dum idum
          (f2cl-lib:array-slice work-%data% single-float (nwork)
                                ((1 *)) work-%offset%)
          iwork info)
       (declare
        (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
         var-11 var-12))
       (setf m var-2)
       (setf ldu var-6)
```

```
    (setf ldvt var-8)
    (setf info var-13))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (sormbr "Q" "L" "N" m m n a lda
       (f2cl-lib:array-slice work-%data% single-float (itauq)
                             ((1 *)) work-%offset%)
       u ldu
       (f2cl-lib:array-slice work-%data% single-float (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf n var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldu var-10))
    (when var-13 (setf ierr var-13)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (sormbr "P" "R" "T" m n m a lda
       (f2cl-lib:array-slice work-%data% single-float (itaup)
                             ((1 *)) work-%offset%)
       vt ldvt
       (f2cl-lib:array-slice work-%data% single-float (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf m var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldvt var-10))
    (when var-13 (setf ierr var-13))))
 (wntqa
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "F" n n zero zero vt ldvt)
    (declare (ignore var-0 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
```

```
    (sbdsdc "L" "I" m s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     u ldu vt ldvt dum idum
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (setf m var-2)
  (setf ldu var-6)
  (setf ldvt var-8)
  (setf info var-13))
(cond
 ((> n m)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "F" (f2cl-lib:int-sub n m) (f2cl-lib:int-sub n m)
       zero one
       (f2cl-lib:array-slice vt-%data% single-float
                             ((+ m 1) (f2cl-lib:int-add m 1))
                             ((1 ldvt) (1 *)) vt-%offset%)
       ldvt)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldvt var-6)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "Q" "L" "N" m m n a lda
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% single-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sormbr "P" "R" "T" n n m a lda
```

```
                  (f2cl-lib:array-slice work-%data% single-float (itaup)
                                        ((1 *)) work-%offset%)
                  vt ldvt
                  (f2cl-lib:array-slice work-%data% single-float (nwork)
                                        ((1 *)) work-%offset%)
                  (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
                (declare
                 (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
                (when var-3 (setf n var-3))
                (when var-4 (setf n var-4))
                (when var-5 (setf m var-5))
                (when var-7 (setf lda var-7))
                (when var-10 (setf ldvt var-10))
                (when var-13 (setf ierr var-13)))))))))
        (cond
         ((= iscl 1)
          (if (> anrm bignum)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (slascl "G" 0 0 bignum anrm minmn 1 s minmn ierr)
                (declare (ignore var-0 var-1 var-2 var-6 var-7))
                (when var-3 (setf bignum var-3))
                (when var-4 (setf anrm var-4))
                (when var-5 (setf minmn var-5))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf ierr var-9))))
          (if (< anrm smlnum)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (slascl "G" 0 0 smlnum anrm minmn 1 s minmn ierr)
                (declare (ignore var-0 var-1 var-2 var-6 var-7))
                (when var-3 (setf smlnum var-3))
                (when var-4 (setf anrm var-4))
                (when var-5 (setf minmn var-5))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf ierr var-9))))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 maxwrk) 'single-float))
        (go end_label)
       end_label
        (return
         (values jobz m n nil lda nil nil ldu nil ldvt nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgesdd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
```

```
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::jobz
  fortran-to-lisp::m
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil
  fortran-to-lisp::ldu nil
  fortran-to-lisp::ldvt nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::sgelqf
  fortran-to-lisp::sbdsdc
  fortran-to-lisp::sgebrd
  fortran-to-lisp::sgeqrf
  fortran-to-lisp::ilaenv))))
```

———-

29.3 sgesvd

29.3.1 Axiom unit tests

— sgesvd.input —

```
)set break resume
)sys rm -f sgesvd.output
)spool sgesvd.output
)set message test on
)set message auto off
)clear all

)spool
```

```
)lisp (bye)
```

———-

29.3.2 Axiom help page

— **sgesvd.help** —

```
=====================================================================
sgesvd examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgesvd (JOBU, JOBVT, M, N, A, LDA, S, U, LDU, VT, LDVT,
           WORK, LWORK, INFO)
            SGESVD computes the singular value decomposition (SVD) for GE
           matrices

Function/Subroutine Documentation
   subroutine sgesvd (character JOBU, character JOBVT, integer M, integer N,
       real, dimension( lda, * ) A, integer LDA, real, dimension( * ) S, real,
       dimension( ldu, * ) U, integer LDU, real, dimension( ldvt, * ) VT,
       integer LDVT, real, dimension( * ) WORK, integer LWORK, integer INFO)
        SGESVD computes the singular value decomposition (SVD) for GE matrices

Purpose:
   SGESVD computes the singular value decomposition (SVD) of a real
   M-by-N matrix A, optionally computing the left and/or right singular
   vectors. The SVD is written

   A = U * SIGMA * transpose(V)

   where SIGMA is an M-by-N matrix which is zero except for its
   min(m,n) diagonal elements, U is an M-by-M orthogonal matrix, and
   V is an N-by-N orthogonal matrix.  The diagonal elements of SIGMA
   are the singular values of A; they are real and non-negative, and
   are returned in descending order.  The first min(m,n) columns of
   U and V are the left and right singular vectors of A.

   Note that the routine returns V**T, not V.

Parameters:
```

```
JOBU
  JOBU is CHARACTER*1
  Specifies options for computing all or part of the matrix U:
  = 'A':  all M columns of U are returned in array U:
  = 'S':  the first min(m,n) columns of U (the left singular
          vectors) are returned in the array U;
  = 'O':  the first min(m,n) columns of U (the left singular
          vectors) are overwritten on the array A;
  = 'N':  no columns of U (no left singular vectors) are computed.

JOBVT
  JOBVT is CHARACTER*1
  Specifies options for computing all or part of the matrix
  V**T:
  = 'A':  all N rows of V**T are returned in the array VT;
  = 'S':  the first min(m,n) rows of V**T (the right singular
          vectors) are returned in the array VT;
  = 'O':  the first min(m,n) rows of V**T (the right singular
          vectors) are overwritten on the array A;
  = 'N':  no rows of V**T (no right singular vectors) are computed.

JOBVT and JOBU cannot both be 'O'.

M
  M is INTEGER
  The number of rows of the input matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the input matrix A.  N >= 0.

A
  A is REAL array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit,
  if JOBU = 'O',  A is overwritten with the first min(m,n)
       columns of U (the left singular vectors, stored columnwise);
  if JOBVT = 'O', A is overwritten with the first min(m,n)
       rows of V**T (the right singular vectors, stored rowwise);
  if JOBU .ne. 'O' and JOBVT .ne. 'O', the contents of A are destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

S
  S is REAL array, dimension (min(M,N))
  The singular values of A, sorted so that S(i) >= S(i+1).

U
```

```
  U is REAL array, dimension (LDU,UCOL)
  (LDU,M) if JOBU = 'A' or (LDU,min(M,N)) if JOBU = 'S'.
  If JOBU = 'A', U contains the M-by-M orthogonal matrix U;
  if JOBU = 'S', U contains the first min(m,n) columns of U
     (the left singular vectors, stored columnwise);
  if JOBU = 'N' or 'O', U is not referenced.

LDU
  LDU is INTEGER
  The leading dimension of the array U.  LDU >= 1; if
  JOBU = 'S' or 'A', LDU >= M.

VT
  VT is REAL array, dimension (LDVT,N)
  If JOBVT = 'A', VT contains the N-by-N orthogonal matrix V**T;
  if JOBVT = 'S', VT contains the first min(m,n) rows of
     V**T (the right singular vectors, stored rowwise);
  if JOBVT = 'N' or 'O', VT is not referenced.

LDVT
  LDVT is INTEGER
  The leading dimension of the array VT.  LDVT >= 1; if
  JOBVT = 'A', LDVT >= N; if JOBVT = 'S', LDVT >= min(M,N).

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK;
  if INFO > 0, WORK(2:MIN(M,N)) contains the unconverged
  superdiagonal elements of an upper bidiagonal matrix B
  whose diagonal is in S (not necessarily sorted). B
  satisfies A = U * B * VT, so it has the same singular values
  as A, and singular vectors related by U and VT.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  LWORK >= MAX(1,5*MIN(M,N)) for the paths (see comments inside code):
  - PATH 1  (M much larger than N, JOBU='N')
  - PATH 1t (N much larger than M, JOBVT='N')
  LWORK >= MAX(1,3*MIN(M,N)+MAX(M,N),5*MIN(M,N)) for the other paths
  For good performance, LWORK should generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit.
```

```
< 0:  if INFO = -i, the i-th argument had an illegal value.
> 0:  if SBDSQR did not converge, INFO specifies how many
      superdiagonals of an intermediate bidiagonal form B
      did not converge to zero. See the description of WORK
      above for details.
```

———-

29.3.3 fortran code

— **sgesvd.f** —

```
*  =====================================================================
      SUBROUTINE SGESVD( JOBU, JOBVT, M, N, A, LDA, S, U, LDU, VT, LDVT,
     $                   WORK, LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          JOBU, JOBVT
      INTEGER            INFO, LDA, LDU, LDVT, LWORK, M, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), S( * ), U( LDU, * ),
     $                   VT( LDVT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, WNTUA, WNTUAS, WNTUN, WNTUO, WNTUS,
     $                   WNTVA, WNTVAS, WNTVN, WNTVO, WNTVS
      INTEGER            BDSPAC, BLK, CHUNK, I, IE, IERR, IR, ISCL,
     $                   ITAU, ITAUP, ITAUQ, IU, IWORK, LDWRKR, LDWRKU,
     $                   MAXWRK, MINMN, MINWRK, MNTHR, NCU, NCVT, NRU,
     $                   NRVT, WRKBL
      INTEGER            LWORK_SGEQRF, LWORK_SORGQR_N, LWORK_SORGQR_M,
     $                   LWORK_SGEBRD, LWORK_SORGBR_P, LWORK_SORGBR_Q,
     $                   LWORK_SGELQF, LWORK_SORGLQ_N, LWORK_SORGLQ_M
      REAL               ANRM, BIGNUM, EPS, SMLNUM
```

```
*     ..
*     .. Local Arrays ..
      REAL               DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SBDSQR, SGEBRD, SGELQF, SGEMM, SGEQRF, SLACPY,
     $                   SLASCL, SLASET, SORGBR, SORGLQ, SORGQR, SORMBR,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               SLAMCH, SLANGE
      EXTERNAL           LSAME, ILAENV, SLAMCH, SLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      MINMN = MIN( M, N )
      WNTUA = LSAME( JOBU, 'A' )
      WNTUS = LSAME( JOBU, 'S' )
      WNTUAS = WNTUA .OR. WNTUS
      WNTUO = LSAME( JOBU, 'O' )
      WNTUN = LSAME( JOBU, 'N' )
      WNTVA = LSAME( JOBVT, 'A' )
      WNTVS = LSAME( JOBVT, 'S' )
      WNTVAS = WNTVA .OR. WNTVS
      WNTVO = LSAME( JOBVT, 'O' )
      WNTVN = LSAME( JOBVT, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
*
      IF( .NOT.( WNTUA .OR. WNTUS .OR. WNTUO .OR. WNTUN ) ) THEN
         INFO = -1
      ELSE IF( .NOT.( WNTVA .OR. WNTVS .OR. WNTVO .OR. WNTVN ) .OR.
     $         ( WNTVO .AND. WNTUO ) ) THEN
         INFO = -2
      ELSE IF( M.LT.0 ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -6
      ELSE IF( LDU.LT.1 .OR. ( WNTUAS .AND. LDU.LT.M ) ) THEN
         INFO = -9
      ELSE IF( LDVT.LT.1 .OR. ( WNTVA .AND. LDVT.LT.N ) .OR.
```

```
     $         ( WNTVS .AND. LDVT.LT.MINMN ) ) THEN
         INFO = -11
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = 1
         MAXWRK = 1
         IF( M.GE.N .AND. MINMN.GT.0 ) THEN
*
*           Compute space needed for SBDSQR
*
            MNTHR = ILAENV( 6, 'SGESVD', JOBU // JOBVT, M, N, 0, 0 )
            BDSPAC = 5*N
*           Compute space needed for SGEQRF
            CALL SGEQRF( M, N, A, LDA, DUM(1), DUM(1), -1, IERR )
            LWORK_SGEQRF=DUM(1)
*           Compute space needed for SORGQR
            CALL SORGQR( M, N, N, A, LDA, DUM(1), DUM(1), -1, IERR )
            LWORK_SORGQR_N=DUM(1)
            CALL SORGQR( M, M, N, A, LDA, DUM(1), DUM(1), -1, IERR )
            LWORK_SORGQR_M=DUM(1)
*           Compute space needed for SGEBRD
            CALL SGEBRD( N, N, A, LDA, S, DUM(1), DUM(1),
     $                   DUM(1), DUM(1), -1, IERR )
            LWORK_SGEBRD=DUM(1)
*           Compute space needed for SORGBR P
            CALL SORGBR( 'P', N, N, N, A, LDA, DUM(1),
     $                   DUM(1), -1, IERR )
            LWORK_SORGBR_P=DUM(1)
*           Compute space needed for SORGBR Q
            CALL SORGBR( 'Q', N, N, N, A, LDA, DUM(1),
     $                   DUM(1), -1, IERR )
            LWORK_SORGBR_Q=DUM(1)
*
            IF( M.GE.MNTHR ) THEN
               IF( WNTUN ) THEN
*
*                 Path 1 (M much larger than N, JOBU='N')
*
                  MAXWRK = N + LWORK_SGEQRF
                  MAXWRK = MAX( MAXWRK, 3*N+LWORK_SGEBRD )
                  IF( WNTVO .OR. WNTVAS )
     $               MAXWRK = MAX( MAXWRK, 3*N+LWORK_SORGBR_P )
```

```
                 MAXWRK = MAX( MAXWRK, BDSPAC )
                 MINWRK = MAX( 4*N, BDSPAC )
              ELSE IF( WNTUO .AND. WNTVN ) THEN
*
*                Path 2 (M much larger than N, JOBU='O', JOBVT='N')
*
                 WRKBL = N + LWORK_SGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_SORGQR_N )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = MAX( N*N+WRKBL, N*N+M*N+N )
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUO .AND. WNTVAS ) THEN
*
*                Path 3 (M much larger than N, JOBU='O', JOBVT='S' or
*                'A')
*
                 WRKBL = N + LWORK_SGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_SORGQR_N )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SORGBR_Q )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SORGBR_P )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = MAX( N*N+WRKBL, N*N+M*N+N )
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUS .AND. WNTVN ) THEN
*
*                Path 4 (M much larger than N, JOBU='S', JOBVT='N')
*
                 WRKBL = N + LWORK_SGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_SORGQR_N )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUS .AND. WNTVO ) THEN
*
*                Path 5 (M much larger than N, JOBU='S', JOBVT='O')
*
                 WRKBL = N + LWORK_SGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_SORGQR_N )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SORGBR_Q )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SORGBR_P )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = 2*N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUS .AND. WNTVAS ) THEN
```

```
*
*                Path 6 (M much larger than N, JOBU='S', JOBVT='S' or
*                'A')
*
                 WRKBL = N + LWORK_SGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_SORGQR_N )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SORGBR_Q )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SORGBR_P )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUA .AND. WNTVN ) THEN
*
*                Path 7 (M much larger than N, JOBU='A', JOBVT='N')
*
                 WRKBL = N + LWORK_SGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_SORGQR_M )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUA .AND. WNTVO ) THEN
*
*                Path 8 (M much larger than N, JOBU='A', JOBVT='O')
*
                 WRKBL = N + LWORK_SGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_SORGQR_M )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SORGBR_Q )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SORGBR_P )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = 2*N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUA .AND. WNTVAS ) THEN
*
*                Path 9 (M much larger than N, JOBU='A', JOBVT='S' or
*                'A')
*
                 WRKBL = N + LWORK_SGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_SORGQR_M )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SORGBR_Q )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_SORGBR_P )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
              END IF
           ELSE
```

```
*
*             Path 10 (M at least N, but not much larger)
*
              CALL SGEBRD( M, N, A, LDA, S, DUM(1), DUM(1),
     $                     DUM(1), DUM(1), -1, IERR )
              LWORK_SGEBRD=DUM(1)
              MAXWRK = 3*N + LWORK_SGEBRD
              IF( WNTUS .OR. WNTUO ) THEN
                 CALL SORGBR( 'Q', M, N, N, A, LDA, DUM(1),
     $                        DUM(1), -1, IERR )
                 LWORK_SORGBR_Q=DUM(1)
                 MAXWRK = MAX( MAXWRK, 3*N+LWORK_SORGBR_Q )
              END IF
              IF( WNTUA ) THEN
                 CALL SORGBR( 'Q', M, M, N, A, LDA, DUM(1),
     $                        DUM(1), -1, IERR )
                 LWORK_SORGBR_Q=DUM(1)
                 MAXWRK = MAX( MAXWRK, 3*N+LWORK_SORGBR_Q )
              END IF
              IF( .NOT.WNTVN ) THEN
                MAXWRK = MAX( MAXWRK, 3*N+LWORK_SORGBR_P )
              END IF
              MAXWRK = MAX( MAXWRK, BDSPAC )
              MINWRK = MAX( 3*N+M, BDSPAC )
           END IF
        ELSE IF( MINMN.GT.0 ) THEN
*
*          Compute space needed for SBDSQR
*
           MNTHR = ILAENV( 6, 'SGESVD', JOBU // JOBVT, M, N, 0, 0 )
           BDSPAC = 5*M
*          Compute space needed for SGELQF
           CALL SGELQF( M, N, A, LDA, DUM(1), DUM(1), -1, IERR )
           LWORK_SGELQF=DUM(1)
*          Compute space needed for SORGLQ
           CALL SORGLQ( N, N, M, DUM(1), N, DUM(1), DUM(1), -1, IERR )
           LWORK_SORGLQ_N=DUM(1)
           CALL SORGLQ( M, N, M, A, LDA, DUM(1), DUM(1), -1, IERR )
           LWORK_SORGLQ_M=DUM(1)
*          Compute space needed for SGEBRD
           CALL SGEBRD( M, M, A, LDA, S, DUM(1), DUM(1),
     $                  DUM(1), DUM(1), -1, IERR )
           LWORK_SGEBRD=DUM(1)
*           Compute space needed for SORGBR P
           CALL SORGBR( 'P', M, M, M, A, N, DUM(1),
     $                  DUM(1), -1, IERR )
           LWORK_SORGBR_P=DUM(1)
*          Compute space needed for SORGBR Q
           CALL SORGBR( 'Q', M, M, M, A, N, DUM(1),
     $                  DUM(1), -1, IERR )
```

```
            LWORK_SORGBR_Q=DUM(1)
            IF( N.GE.MNTHR ) THEN
               IF( WNTVN ) THEN
*
*                 Path 1t(N much larger than M, JOBVT='N')
*
                  MAXWRK = M + LWORK_SGELQF
                  MAXWRK = MAX( MAXWRK, 3*M+LWORK_SGEBRD )
                  IF( WNTUO .OR. WNTUAS )
     $               MAXWRK = MAX( MAXWRK, 3*M+LWORK_SORGBR_Q )
                  MAXWRK = MAX( MAXWRK, BDSPAC )
                  MINWRK = MAX( 4*M, BDSPAC )
               ELSE IF( WNTVO .AND. WNTUN ) THEN
*
*                 Path 2t(N much larger than M, JOBU='N', JOBVT='O')
*
                  WRKBL = M + LWORK_SGELQF
                  WRKBL = MAX( WRKBL, M+LWORK_SORGLQ_M )
                  WRKBL = MAX( WRKBL, 3*M+LWORK_SGEBRD )
                  WRKBL = MAX( WRKBL, 3*M+LWORK_SORGBR_P )
                  WRKBL = MAX( WRKBL, BDSPAC )
                  MAXWRK = MAX( M*M+WRKBL, M*M+M*N+M )
                  MINWRK = MAX( 3*M+N, BDSPAC )
               ELSE IF( WNTVO .AND. WNTUAS ) THEN
*
*                 Path 3t(N much larger than M, JOBU='S' or 'A',
*                 JOBVT='O')
*
                  WRKBL = M + LWORK_SGELQF
                  WRKBL = MAX( WRKBL, M+LWORK_SORGLQ_M )
                  WRKBL = MAX( WRKBL, 3*M+LWORK_SGEBRD )
                  WRKBL = MAX( WRKBL, 3*M+LWORK_SORGBR_P )
                  WRKBL = MAX( WRKBL, 3*M+LWORK_SORGBR_Q )
                  WRKBL = MAX( WRKBL, BDSPAC )
                  MAXWRK = MAX( M*M+WRKBL, M*M+M*N+M )
                  MINWRK = MAX( 3*M+N, BDSPAC )
               ELSE IF( WNTVS .AND. WNTUN ) THEN
*
*                 Path 4t(N much larger than M, JOBU='N', JOBVT='S')
*
                  WRKBL = M + LWORK_SGELQF
                  WRKBL = MAX( WRKBL, M+LWORK_SORGLQ_M )
                  WRKBL = MAX( WRKBL, 3*M+LWORK_SGEBRD )
                  WRKBL = MAX( WRKBL, 3*M+LWORK_SORGBR_P )
                  WRKBL = MAX( WRKBL, BDSPAC )
                  MAXWRK = M*M + WRKBL
                  MINWRK = MAX( 3*M+N, BDSPAC )
               ELSE IF( WNTVS .AND. WNTUO ) THEN
*
*                 Path 5t(N much larger than M, JOBU='O', JOBVT='S')
```

```
*
                 WRKBL = M + LWORK_SGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_SORGLQ_M )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_SGEBRD )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_SORGBR_P )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_SORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = 2*M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
                 MAXWRK = MAX( MAXWRK, MINWRK )
              ELSE IF( WNTVS .AND. WNTUAS ) THEN
*
*                Path 6t(N much larger than M, JOBU='S' or 'A',
*                JOBVT='S')
*
                 WRKBL = M + LWORK_SGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_SORGLQ_M )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_SGEBRD )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_SORGBR_P )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_SORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
              ELSE IF( WNTVA .AND. WNTUN ) THEN
*
*                Path 7t(N much larger than M, JOBU='N', JOBVT='A')
*
                 WRKBL = M + LWORK_SGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_SORGLQ_N )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_SGEBRD )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_SORGBR_P )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
              ELSE IF( WNTVA .AND. WNTUO ) THEN
*
*                Path 8t(N much larger than M, JOBU='O', JOBVT='A')
*
                 WRKBL = M + LWORK_SGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_SORGLQ_N )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_SGEBRD )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_SORGBR_P )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_SORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = 2*M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
              ELSE IF( WNTVA .AND. WNTUAS ) THEN
*
*                Path 9t(N much larger than M, JOBU='S' or 'A',
*                JOBVT='A')
```

```
*
                 WRKBL = M + LWORK_SGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_SORGLQ_N )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_SGEBRD )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_SORGBR_P )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_SORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
              END IF
           ELSE
*
*             Path 10t(N greater than M, but not much larger)
*
              CALL SGEBRD( M, N, A, LDA, S, DUM(1), DUM(1),
     $                     DUM(1), DUM(1), -1, IERR )
              LWORK_SGEBRD=DUM(1)
              MAXWRK = 3*M + LWORK_SGEBRD
              IF( WNTVS .OR. WNTVO ) THEN
*               Compute space needed for SORGBR P
                CALL SORGBR( 'P', M, N, M, A, N, DUM(1),
     $                     DUM(1), -1, IERR )
                LWORK_SORGBR_P=DUM(1)
                MAXWRK = MAX( MAXWRK, 3*M+LWORK_SORGBR_P )
              END IF
              IF( WNTVA ) THEN
                CALL SORGBR( 'P', N, N, M, A, N, DUM(1),
     $                     DUM(1), -1, IERR )
                LWORK_SORGBR_P=DUM(1)
                MAXWRK = MAX( MAXWRK, 3*M+LWORK_SORGBR_P )
              END IF
              IF( .NOT.WNTUN ) THEN
                 MAXWRK = MAX( MAXWRK, 3*M+LWORK_SORGBR_Q )
              END IF
              MAXWRK = MAX( MAXWRK, BDSPAC )
              MINWRK = MAX( 3*M+N, BDSPAC )
           END IF
        END IF
        MAXWRK = MAX( MAXWRK, MINWRK )
        WORK( 1 ) = MAXWRK
*
        IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
           INFO = -13
        END IF
     END IF
*
     IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'SGESVD', -INFO )
        RETURN
     ELSE IF( LQUERY ) THEN
```

```
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SQRT( SLAMCH( 'S' ) ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = SLANGE( 'M', M, N, A, LDA, DUM )
      ISCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ISCL = 1
         CALL SLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, IERR )
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ISCL = 1
         CALL SLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, IERR )
      END IF
*
      IF( M.GE.N ) THEN
*
*        A has at least as many rows as columns. If A has sufficiently
*        more rows than columns, first reduce using the QR
*        decomposition (if sufficient workspace available)
*
         IF( M.GE.MNTHR ) THEN
*
            IF( WNTUN ) THEN
*
*              Path 1 (M much larger than N, JOBU='N')
*              No left singular vectors to be computed
*
               ITAU = 1
               IWORK = ITAU + N
*
*              Compute A=Q*R
*              (Workspace: need 2*N, prefer N+N*NB)
*
               CALL SGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
     $                      LWORK-IWORK+1, IERR )
*
*              Zero out below R
```

```
*
               CALL SLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ), LDA )
               IE = 1
               ITAUQ = IE + N
               ITAUP = ITAUQ + N
               IWORK = ITAUP + N
*
*              Bidiagonalize R in A
*              (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
               CALL SGEBRD( N, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                      IERR )
               NCVT = 0
               IF( WNTVO .OR. WNTVAS ) THEN
*
*                 If right singular vectors desired, generate P'.
*                 (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                  CALL SORGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  NCVT = N
               END IF
               IWORK = IE + N
*
*              Perform bidiagonal QR iteration, computing right
*              singular vectors of A in A if desired
*              (Workspace: need BDSPAC)
*
               CALL SBDSQR( 'U', N, NCVT, 0, 0, S, WORK( IE ), A, LDA,
     $                      DUM, 1, DUM, 1, WORK( IWORK ), INFO )
*
*              If right singular vectors desired in VT, copy them there
*
               IF( WNTVAS )
     $            CALL SLACPY( 'F', N, N, A, LDA, VT, LDVT )
*
            ELSE IF( WNTUO .AND. WNTVN ) THEN
*
*              Path 2 (M much larger than N, JOBU='O', JOBVT='N')
*              N left singular vectors to be overwritten on A and
*              no right singular vectors to be computed
*
               IF( LWORK.GE.N*N+MAX( 4*N, BDSPAC ) ) THEN
*
*                 Sufficient workspace for a fast algorithm
*
                  IR = 1
                  IF( LWORK.GE.MAX( WRKBL, LDA*N+N )+LDA*N ) THEN
*
```

```
*                    WORK(IU) is LDA by N, WORK(IR) is LDA by N
*
                     LDWRKU = LDA
                     LDWRKR = LDA
                  ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N+N )+N*N ) THEN
*
*                    WORK(IU) is LDA by N, WORK(IR) is N by N
*
                     LDWRKU = LDA
                     LDWRKR = N
                  ELSE
*
*                    WORK(IU) is LDWRKU by N, WORK(IR) is N by N
*
                     LDWRKU = ( LWORK-N*N-N ) / N
                     LDWRKR = N
                  END IF
                  ITAU = IR + LDWRKR*N
                  IWORK = ITAU + N
*
*                 Compute A=Q*R
*                 (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                  CALL SGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Copy R to WORK(IR) and zero out below it
*
                  CALL SLACPY( 'U', N, N, A, LDA, WORK( IR ), LDWRKR )
                  CALL SLASET( 'L', N-1, N-1, ZERO, ZERO, WORK( IR+1 ),
     $                         LDWRKR )
*
*                 Generate Q in A
*                 (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                  CALL SORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IE = ITAU
                  ITAUQ = IE + N
                  ITAUP = ITAUQ + N
                  IWORK = ITAUP + N
*
*                 Bidiagonalize R in WORK(IR)
*                 (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                  CALL SGEBRD( N, N, WORK( IR ), LDWRKR, S, WORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Generate left vectors bidiagonalizing R
```

```
*                 (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                  CALL SORGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                         WORK( ITAUQ ), WORK( IWORK ),
     $                         LWORK-IWORK+1, IERR )
                  IWORK = IE + N
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of R in WORK(IR)
*                 (Workspace: need N*N+BDSPAC)
*
                  CALL SBDSQR( 'U', N, 0, N, 0, S, WORK( IE ), DUM, 1,
     $                         WORK( IR ), LDWRKR, DUM, 1,
     $                         WORK( IWORK ), INFO )
                  IU = IE + N
*
*                 Multiply Q in A by left singular vectors of R in
*                 WORK(IR), storing result in WORK(IU) and copying to A
*                 (Workspace: need N*N+2*N, prefer N*N+M*N+N)
*
                  DO 10 I = 1, M, LDWRKU
                     CHUNK = MIN( M-I+1, LDWRKU )
                     CALL SGEMM( 'N', 'N', CHUNK, N, N, ONE, A( I, 1 ),
     $                           LDA, WORK( IR ), LDWRKR, ZERO,
     $                           WORK( IU ), LDWRKU )
                     CALL SLACPY( 'F', CHUNK, N, WORK( IU ), LDWRKU,
     $                            A( I, 1 ), LDA )
   10             CONTINUE
*
               ELSE
*
*                 Insufficient workspace for a fast algorithm
*
                  IE = 1
                  ITAUQ = IE + N
                  ITAUP = ITAUQ + N
                  IWORK = ITAUP + N
*
*                 Bidiagonalize A
*                 (Workspace: need 3*N+M, prefer 3*N+(M+N)*NB)
*
                  CALL SGEBRD( M, N, A, LDA, S, WORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Generate left vectors bidiagonalizing A
*                 (Workspace: need 4*N, prefer 3*N+N*NB)
*
                  CALL SORGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
```

```
                 IWORK = IE + N
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of A in A
*                (Workspace: need BDSPAC)
*
                 CALL SBDSQR( 'U', N, 0, M, 0, S, WORK( IE ), DUM, 1,
     $                        A, LDA, DUM, 1, WORK( IWORK ), INFO )
*
              END IF
*
           ELSE IF( WNTUO .AND. WNTVAS ) THEN
*
              Path 3 (M much larger than N, JOBU='O', JOBVT='S' or 'A')
              N left singular vectors to be overwritten on A and
              N right singular vectors to be computed in VT
*
              IF( LWORK.GE.N*N+MAX( 4*N, BDSPAC ) ) THEN
*
                 Sufficient workspace for a fast algorithm
*
                 IR = 1
                 IF( LWORK.GE.MAX( WRKBL, LDA*N+N )+LDA*N ) THEN
*
                    WORK(IU) is LDA by N and WORK(IR) is LDA by N
*
                    LDWRKU = LDA
                    LDWRKR = LDA
                 ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N+N )+N*N ) THEN
*
                    WORK(IU) is LDA by N and WORK(IR) is N by N
*
                    LDWRKU = LDA
                    LDWRKR = N
                 ELSE
*
                    WORK(IU) is LDWRKU by N and WORK(IR) is N by N
*
                    LDWRKU = ( LWORK-N*N-N ) / N
                    LDWRKR = N
                 END IF
                 ITAU = IR + LDWRKR*N
                 IWORK = ITAU + N
*
                 Compute A=Q*R
*                (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                 CALL SGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
```

```
*                 Copy R to VT, zeroing out below it
*
                  CALL SLACPY( 'U', N, N, A, LDA, VT, LDVT )
                  IF( N.GT.1 )
     $               CALL SLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                            VT( 2, 1 ), LDVT )
*
*                 Generate Q in A
*                 (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                  CALL SORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IE = ITAU
                  ITAUQ = IE + N
                  ITAUP = ITAUQ + N
                  IWORK = ITAUP + N
*
*                 Bidiagonalize R in VT, copying result to WORK(IR)
*                 (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                  CALL SGEBRD( N, N, VT, LDVT, S, WORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  CALL SLACPY( 'L', N, N, VT, LDVT, WORK( IR ), LDWRKR )
*
*                 Generate left vectors bidiagonalizing R in WORK(IR)
*                 (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                  CALL SORGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                         WORK( ITAUQ ), WORK( IWORK ),
     $                         LWORK-IWORK+1, IERR )
*
*                 Generate right vectors bidiagonalizing R in VT
*                 (Workspace: need N*N+4*N-1, prefer N*N+3*N+(N-1)*NB)
*
                  CALL SORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IWORK = IE + N
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of R in WORK(IR) and computing right
*                 singular vectors of R in VT
*                 (Workspace: need N*N+BDSPAC)
*
                  CALL SBDSQR( 'U', N, N, N, 0, S, WORK( IE ), VT, LDVT,
     $                         WORK( IR ), LDWRKR, DUM, 1,
     $                         WORK( IWORK ), INFO )
                  IU = IE + N
*
*                 Multiply Q in A by left singular vectors of R in
```

```
*                 WORK(IR), storing result in WORK(IU) and copying to A
*                 (Workspace: need N*N+2*N, prefer N*N+M*N+N)
*
                  DO 20 I = 1, M, LDWRKU
                     CHUNK = MIN( M-I+1, LDWRKU )
                     CALL SGEMM( 'N', 'N', CHUNK, N, N, ONE, A( I, 1 ),
     $                           LDA, WORK( IR ), LDWRKR, ZERO,
     $                           WORK( IU ), LDWRKU )
                     CALL SLACPY( 'F', CHUNK, N, WORK( IU ), LDWRKU,
     $                            A( I, 1 ), LDA )
   20             CONTINUE
*
               ELSE
*
*                 Insufficient workspace for a fast algorithm
*
                  ITAU = 1
                  IWORK = ITAU + N
*
*                 Compute A=Q*R
*                 (Workspace: need 2*N, prefer N+N*NB)
*
                  CALL SGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Copy R to VT, zeroing out below it
*
                  CALL SLACPY( 'U', N, N, A, LDA, VT, LDVT )
                  IF( N.GT.1 )
     $               CALL SLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                            VT( 2, 1 ), LDVT )
*
*                 Generate Q in A
*                 (Workspace: need 2*N, prefer N+N*NB)
*
                  CALL SORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IE = ITAU
                  ITAUQ = IE + N
                  ITAUP = ITAUQ + N
                  IWORK = ITAUP + N
*
*                 Bidiagonalize R in VT
*                 (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                  CALL SGEBRD( N, N, VT, LDVT, S, WORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Multiply Q in A by left vectors bidiagonalizing R
```

```
*                 (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                  CALL SORMBR( 'Q', 'R', 'N', M, N, N, VT, LDVT,
     $                         WORK( ITAUQ ), A, LDA, WORK( IWORK ),
     $                         LWORK-IWORK+1, IERR )
*
*                 Generate right vectors bidiagonalizing R in VT
*                 (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                  CALL SORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IWORK = IE + N
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of A in A and computing right
*                 singular vectors of A in VT
*                 (Workspace: need BDSPAC)
*
                  CALL SBDSQR( 'U', N, N, M, 0, S, WORK( IE ), VT, LDVT,
     $                         A, LDA, DUM, 1, WORK( IWORK ), INFO )
*
               END IF
*
            ELSE IF( WNTUS ) THEN
*
               IF( WNTVN ) THEN
*
*                 Path 4 (M much larger than N, JOBU='S', JOBVT='N')
*                 N left singular vectors to be computed in U and
*                 no right singular vectors to be computed
*
                  IF( LWORK.GE.N*N+MAX( 4*N, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IR = 1
                     IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                       WORK(IR) is LDA by N
*
                        LDWRKR = LDA
                     ELSE
*
*                       WORK(IR) is N by N
*
                        LDWRKR = N
                     END IF
                     ITAU = IR + LDWRKR*N
                     IWORK = ITAU + N
*
```

```
*                    Compute A=Q*R
*                    (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                     CALL SGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy R to WORK(IR), zeroing out below it
*
                     CALL SLACPY( 'U', N, N, A, LDA, WORK( IR ),
     $                            LDWRKR )
                     CALL SLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                            WORK( IR+1 ), LDWRKR )
*
*                    Generate Q in A
*                    (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                     CALL SORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Bidiagonalize R in WORK(IR)
*                    (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                     CALL SGEBRD( N, N, WORK( IR ), LDWRKR, S,
     $                            WORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate left vectors bidiagonalizing R in WORK(IR)
*                    (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                     CALL SORGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                            WORK( ITAUQ ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IR)
*                    (Workspace: need N*N+BDSPAC)
*
                     CALL SBDSQR( 'U', N, 0, N, 0, S, WORK( IE ), DUM,
     $                            1, WORK( IR ), LDWRKR, DUM, 1,
     $                            WORK( IWORK ), INFO )
*
*                    Multiply Q in A by left singular vectors of R in
*                    WORK(IR), storing result in U
*                    (Workspace: need N*N)
```

```
*
                  CALL SGEMM( 'N', 'N', M, N, N, ONE, A, LDA,
     $                        WORK( IR ), LDWRKR, ZERO, U, LDU )
*
               ELSE
*
*                 Insufficient workspace for a fast algorithm
*
                  ITAU = 1
                  IWORK = ITAU + N
*
*                 Compute A=Q*R, copying result to U
*                 (Workspace: need 2*N, prefer N+N*NB)
*
                  CALL SGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  CALL SLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                 Generate Q in U
*                 (Workspace: need 2*N, prefer N+N*NB)
*
                  CALL SORGQR( M, N, N, U, LDU, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IE = ITAU
                  ITAUQ = IE + N
                  ITAUP = ITAUQ + N
                  IWORK = ITAUP + N
*
*                 Zero out below R in A
*
                  CALL SLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ),
     $                         LDA )
*
*                 Bidiagonalize R in A
*                 (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                  CALL SGEBRD( N, N, A, LDA, S, WORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Multiply Q in U by left vectors bidiagonalizing R
*                 (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                  CALL SORMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                         WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                         LWORK-IWORK+1, IERR )
                  IWORK = IE + N
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of A in U
```

```
*                    (Workspace: need BDSPAC)
*
                     CALL SBDSQR( 'U', N, 0, M, 0, S, WORK( IE ), DUM,
     $                            1, U, LDU, DUM, 1, WORK( IWORK ),
     $                            INFO )
*
                  END IF
*
               ELSE IF( WNTVO ) THEN
*
                  Path 5 (M much larger than N, JOBU='S', JOBVT='O')
*                 N left singular vectors to be computed in U and
*                 N right singular vectors to be overwritten on A
*
                  IF( LWORK.GE.2*N*N+MAX( 4*N, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+2*LDA*N ) THEN
*
*                       WORK(IU) is LDA by N and WORK(IR) is LDA by N
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*N
                        LDWRKR = LDA
                     ELSE IF( LWORK.GE.WRKBL+( LDA+N )*N ) THEN
*
*                       WORK(IU) is LDA by N and WORK(IR) is N by N
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*N
                        LDWRKR = N
                     ELSE
*
*                       WORK(IU) is N by N and WORK(IR) is N by N
*
                        LDWRKU = N
                        IR = IU + LDWRKU*N
                        LDWRKR = N
                     END IF
                     ITAU = IR + LDWRKR*N
                     IWORK = ITAU + N
*
*                    Compute A=Q*R
*                    (Workspace: need 2*N*N+2*N, prefer 2*N*N+N+N*NB)
*
                     CALL SGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
```

```
*                    Copy R to WORK(IU), zeroing out below it
*
                     CALL SLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                            LDWRKU )
                     CALL SLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                            WORK( IU+1 ), LDWRKU )
*
*                    Generate Q in A
*                    (Workspace: need 2*N*N+2*N, prefer 2*N*N+N+N*NB)
*
                     CALL SORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Bidiagonalize R in WORK(IU), copying result to
*                    WORK(IR)
*                    (Workspace: need 2*N*N+4*N,
*                                prefer 2*N*N+3*N+2*N*NB)
*
                     CALL SGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                            WORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     CALL SLACPY( 'U', N, N, WORK( IU ), LDWRKU,
     $                            WORK( IR ), LDWRKR )
*
*                    Generate left bidiagonalizing vectors in WORK(IU)
*                    (Workspace: need 2*N*N+4*N, prefer 2*N*N+3*N+N*NB)
*
                     CALL SORGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                            WORK( ITAUQ ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in WORK(IR)
*                    (Workspace: need 2*N*N+4*N-1,
*                                prefer 2*N*N+3*N+(N-1)*NB)
*
                     CALL SORGBR( 'P', N, N, N, WORK( IR ), LDWRKR,
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IU) and computing
*                    right singular vectors of R in WORK(IR)
*                    (Workspace: need 2*N*N+BDSPAC)
*
```

```
                 CALL SBDSQR( 'U', N, N, N, 0, S, WORK( IE ),
     $                        WORK( IR ), LDWRKR, WORK( IU ),
     $                        LDWRKU, DUM, 1, WORK( IWORK ), INFO )
*
*                Multiply Q in A by left singular vectors of R in
*                WORK(IU), storing result in U
*                (Workspace: need N*N)
*
                 CALL SGEMM( 'N', 'N', M, N, N, ONE, A, LDA,
     $                       WORK( IU ), LDWRKU, ZERO, U, LDU )
*
*                Copy right singular vectors of R to A
*                (Workspace: need N*N)
*
                 CALL SLACPY( 'F', N, N, WORK( IR ), LDWRKR, A,
     $                        LDA )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + N
*
*                Compute A=Q*R, copying result to U
*                (Workspace: need 2*N, prefer N+N*NB)
*
                 CALL SGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL SLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                Generate Q in U
*                (Workspace: need 2*N, prefer N+N*NB)
*
                 CALL SORGQR( M, N, N, U, LDU, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = ITAU
                 ITAUQ = IE + N
                 ITAUP = ITAUQ + N
                 IWORK = ITAUP + N
*
*                Zero out below R in A
*
                 CALL SLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ),
     $                        LDA )
*
*                Bidiagonalize R in A
*                (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                 CALL SGEBRD( N, N, A, LDA, S, WORK( IE ),
```

```
     $                          WORK( ITAUQ ), WORK( ITAUP ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply Q in U by left vectors bidiagonalizing R
*                    (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                     CALL SORMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                          WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                          LWORK-IWORK+1, IERR )
*
*                    Generate right vectors bidiagonalizing R in A
*                    (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                     CALL SORGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in A
*                    (Workspace: need BDSPAC)
*
                     CALL SBDSQR( 'U', N, N, M, 0, S, WORK( IE ), A,
     $                          LDA, U, LDU, DUM, 1, WORK( IWORK ),
     $                          INFO )
*
                  END IF
*
               ELSE IF( WNTVAS ) THEN
*
                  Path 6 (M much larger than N, JOBU='S', JOBVT='S'
*                         or 'A')
                  N left singular vectors to be computed in U and
*                 N right singular vectors to be computed in VT
*
                  IF( LWORK.GE.N*N+MAX( 4*N, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                       WORK(IU) is LDA by N
*
                        LDWRKU = LDA
                     ELSE
*
*                       WORK(IU) is N by N
*
                        LDWRKU = N
```

```
                    END IF
                    ITAU = IU + LDWRKU*N
                    IWORK = ITAU + N
*
*                   Compute A=Q*R
*                   (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                    CALL SGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy R to WORK(IU), zeroing out below it
*
                    CALL SLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                           LDWRKU )
                    CALL SLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                           WORK( IU+1 ), LDWRKU )
*
*                   Generate Q in A
*                   (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                    CALL SORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = ITAU
                    ITAUQ = IE + N
                    ITAUP = ITAUQ + N
                    IWORK = ITAUP + N
*
*                   Bidiagonalize R in WORK(IU), copying result to VT
*                   (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                    CALL SGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    CALL SLACPY( 'U', N, N, WORK( IU ), LDWRKU, VT,
     $                           LDVT )
*
*                   Generate left bidiagonalizing vectors in WORK(IU)
*                   (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                    CALL SORGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate right bidiagonalizing vectors in VT
*                   (Workspace: need N*N+4*N-1,
*                              prefer N*N+3*N+(N-1)*NB)
*
                    CALL SORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
```

```
                 IWORK = IE + N
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of R in WORK(IU) and computing
*                right singular vectors of R in VT
*                (Workspace: need N*N+BDSPAC)
*
                 CALL SBDSQR( 'U', N, N, N, 0, S, WORK( IE ), VT,
     $                        LDVT, WORK( IU ), LDWRKU, DUM, 1,
     $                        WORK( IWORK ), INFO )
*
*                Multiply Q in A by left singular vectors of R in
*                WORK(IU), storing result in U
*                (Workspace: need N*N)
*
                 CALL SGEMM( 'N', 'N', M, N, N, ONE, A, LDA,
     $                       WORK( IU ), LDWRKU, ZERO, U, LDU )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + N
*
*                Compute A=Q*R, copying result to U
*                (Workspace: need 2*N, prefer N+N*NB)
*
                 CALL SGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL SLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                Generate Q in U
*                (Workspace: need 2*N, prefer N+N*NB)
*
                 CALL SORGQR( M, N, N, U, LDU, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy R to VT, zeroing out below it
*
                 CALL SLACPY( 'U', N, N, A, LDA, VT, LDVT )
                 IF( N.GT.1 )
     $              CALL SLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                           VT( 2, 1 ), LDVT )
                 IE = ITAU
                 ITAUQ = IE + N
                 ITAUP = ITAUQ + N
                 IWORK = ITAUP + N
*
*                Bidiagonalize R in VT
```

```
*                    (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                     CALL SGEBRD( N, N, VT, LDVT, S, WORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply Q in U by left bidiagonalizing vectors
*                    in VT
*                    (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                     CALL SORMBR( 'Q', 'R', 'N', M, N, N, VT, LDVT,
     $                            WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in VT
*                    (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                     CALL SORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
                     CALL SBDSQR( 'U', N, N, M, 0, S, WORK( IE ), VT,
     $                            LDVT, U, LDU, DUM, 1, WORK( IWORK ),
     $                            INFO )
*
                  END IF
*
               END IF
*
            ELSE IF( WNTUA ) THEN
*
               IF( WNTVN ) THEN
*
*                 Path 7 (M much larger than N, JOBU='A', JOBVT='N')
*                 M left singular vectors to be computed in U and
*                 no right singular vectors to be computed
*
                  IF( LWORK.GE.N*N+MAX( N+M, 4*N, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IR = 1
                     IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                       WORK(IR) is LDA by N
```

```
*
                      LDWRKR = LDA
                   ELSE
*
*                     WORK(IR) is N by N
*
                      LDWRKR = N
                   END IF
                   ITAU = IR + LDWRKR*N
                   IWORK = ITAU + N
*
*                  Compute A=Q*R, copying result to U
*                  (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                   CALL SGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                   CALL SLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                  Copy R to WORK(IR), zeroing out below it
*
                   CALL SLACPY( 'U', N, N, A, LDA, WORK( IR ),
     $                           LDWRKR )
                   CALL SLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                           WORK( IR+1 ), LDWRKR )
*
*                  Generate Q in U
*                  (Workspace: need N*N+N+M, prefer N*N+N+M*NB)
*
                   CALL SORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                   IE = ITAU
                   ITAUQ = IE + N
                   ITAUP = ITAUQ + N
                   IWORK = ITAUP + N
*
*                  Bidiagonalize R in WORK(IR)
*                  (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                   CALL SGEBRD( N, N, WORK( IR ), LDWRKR, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                  Generate left bidiagonalizing vectors in WORK(IR)
*                  (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                   CALL SORGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                   IWORK = IE + N
```

```
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IR)
*                    (Workspace: need N*N+BDSPAC)
*
                     CALL SBDSQR( 'U', N, 0, N, 0, S, WORK( IE ), DUM,
     $                            1, WORK( IR ), LDWRKR, DUM, 1,
     $                            WORK( IWORK ), INFO )
*
*                    Multiply Q in U by left singular vectors of R in
*                    WORK(IR), storing result in A
*                    (Workspace: need N*N)
*
                     CALL SGEMM( 'N', 'N', M, N, N, ONE, U, LDU,
     $                           WORK( IR ), LDWRKR, ZERO, A, LDA )
*
*                    Copy left singular vectors of A from A to U
*
                     CALL SLACPY( 'F', M, N, A, LDA, U, LDU )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (Workspace: need 2*N, prefer N+N*NB)
*
                     CALL SGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL SLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                    Generate Q in U
*                    (Workspace: need N+M, prefer N+M*NB)
*
                     CALL SORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Zero out below R in A
*
                     CALL SLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ),
     $                            LDA )
*
*                    Bidiagonalize R in A
```

```
*                    (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                     CALL SGEBRD( N, N, A, LDA, S, WORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply Q in U by left bidiagonalizing vectors
*                    in A
*                    (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                     CALL SORMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                            WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U
*                    (Workspace: need BDSPAC)
*
                     CALL SBDSQR( 'U', N, 0, M, 0, S, WORK( IE ), DUM,
     $                            1, U, LDU, DUM, 1, WORK( IWORK ),
     $                            INFO )
*
                  END IF
*
               ELSE IF( WNTVO ) THEN
*
*                 Path 8 (M much larger than N, JOBU='A', JOBVT='O')
*                 M left singular vectors to be computed in U and
*                 N right singular vectors to be overwritten on A
*
                  IF( LWORK.GE.2*N*N+MAX( N+M, 4*N, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+2*LDA*N ) THEN
*
*                       WORK(IU) is LDA by N and WORK(IR) is LDA by N
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*N
                        LDWRKR = LDA
                     ELSE IF( LWORK.GE.WRKBL+( LDA+N )*N ) THEN
*
*                       WORK(IU) is LDA by N and WORK(IR) is N by N
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*N
                        LDWRKR = N
```

```
                    ELSE
*
*                      WORK(IU) is N by N and WORK(IR) is N by N
*
                       LDWRKU = N
                       IR = IU + LDWRKU*N
                       LDWRKR = N
                    END IF
                    ITAU = IR + LDWRKR*N
                    IWORK = ITAU + N
*
*                   Compute A=Q*R, copying result to U
*                   (Workspace: need 2*N*N+2*N, prefer 2*N*N+N+N*NB)
*
                    CALL SGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    CALL SLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                   Generate Q in U
*                   (Workspace: need 2*N*N+N+M, prefer 2*N*N+N+M*NB)
*
                    CALL SORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy R to WORK(IU), zeroing out below it
*
                    CALL SLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                           LDWRKU )
                    CALL SLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                           WORK( IU+1 ), LDWRKU )
                    IE = ITAU
                    ITAUQ = IE + N
                    ITAUP = ITAUQ + N
                    IWORK = ITAUP + N
*
*                   Bidiagonalize R in WORK(IU), copying result to
*                   WORK(IR)
*                   (Workspace: need 2*N*N+4*N,
*                               prefer 2*N*N+3*N+2*N*NB)
*
                    CALL SGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    CALL SLACPY( 'U', N, N, WORK( IU ), LDWRKU,
     $                           WORK( IR ), LDWRKR )
*
*                   Generate left bidiagonalizing vectors in WORK(IU)
*                   (Workspace: need 2*N*N+4*N, prefer 2*N*N+3*N+N*NB)
*
```

```
                 CALL SORGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                        WORK( ITAUQ ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate right bidiagonalizing vectors in WORK(IR)
*                (Workspace: need 2*N*N+4*N-1,
*                           prefer 2*N*N+3*N+(N-1)*NB)
*
                 CALL SORGBR( 'P', N, N, N, WORK( IR ), LDWRKR,
     $                        WORK( ITAUP ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
                 IWORK = IE + N
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of R in WORK(IU) and computing
*                right singular vectors of R in WORK(IR)
*                (Workspace: need 2*N*N+BDSPAC)
*
                 CALL SBDSQR( 'U', N, N, N, 0, S, WORK( IE ),
     $                        WORK( IR ), LDWRKR, WORK( IU ),
     $                        LDWRKU, DUM, 1, WORK( IWORK ), INFO )
*
*                Multiply Q in U by left singular vectors of R in
*                WORK(IU), storing result in A
*                (Workspace: need N*N)
*
                 CALL SGEMM( 'N', 'N', M, N, N, ONE, U, LDU,
     $                       WORK( IU ), LDWRKU, ZERO, A, LDA )
*
*                Copy left singular vectors of A from A to U
*
                 CALL SLACPY( 'F', M, N, A, LDA, U, LDU )
*
*                Copy right singular vectors of R from WORK(IR) to A
*
                 CALL SLACPY( 'F', N, N, WORK( IR ), LDWRKR, A,
     $                        LDA )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + N
*
*                Compute A=Q*R, copying result to U
*                (Workspace: need 2*N, prefer N+N*NB)
*
                 CALL SGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
```

```
                    CALL SLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                   Generate Q in U
*                   (Workspace: need N+M, prefer N+M*NB)
*
                    CALL SORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = ITAU
                    ITAUQ = IE + N
                    ITAUP = ITAUQ + N
                    IWORK = ITAUP + N
*
*                   Zero out below R in A
*
                    CALL SLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ),
     $                           LDA )
*
*                   Bidiagonalize R in A
*                   (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                    CALL SGEBRD( N, N, A, LDA, S, WORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Multiply Q in U by left bidiagonalizing vectors
*                   in A
*                   (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                    CALL SORMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                           WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate right bidiagonalizing vectors in A
*                   (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                    CALL SORGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IWORK = IE + N
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of A in U and computing right
*                   singular vectors of A in A
*                   (Workspace: need BDSPAC)
*
                    CALL SBDSQR( 'U', N, N, M, 0, S, WORK( IE ), A,
     $                           LDA, U, LDU, DUM, 1, WORK( IWORK ),
     $                           INFO )
*
                  END IF
*
```

```
               ELSE IF( WNTVAS ) THEN
*
*                 Path 9 (M much larger than N, JOBU='A', JOBVT='S'
*                         or 'A')
*                 M left singular vectors to be computed in U and
*                 N right singular vectors to be computed in VT
*
                  IF( LWORK.GE.N*N+MAX( N+M, 4*N, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                       WORK(IU) is LDA by N
*
                        LDWRKU = LDA
                     ELSE
*
*                       WORK(IU) is N by N
*
                        LDWRKU = N
                     END IF
                     ITAU = IU + LDWRKU*N
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                     CALL SGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL SLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                    Generate Q in U
*                    (Workspace: need N*N+N+M, prefer N*N+N+M*NB)
*
                     CALL SORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy R to WORK(IU), zeroing out below it
*
                     CALL SLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                            LDWRKU )
                     CALL SLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                            WORK( IU+1 ), LDWRKU )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
```

```
*                    Bidiagonalize R in WORK(IU), copying result to VT
*                    (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                     CALL SGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                            WORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     CALL SLACPY( 'U', N, N, WORK( IU ), LDWRKU, VT,
     $                            LDVT )
*
*                    Generate left bidiagonalizing vectors in WORK(IU)
*                    (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                     CALL SORGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                            WORK( ITAUQ ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in VT
*                    (Workspace: need N*N+4*N-1,
*                                prefer N*N+3*N+(N-1)*NB)
*
                     CALL SORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IU) and computing
*                    right singular vectors of R in VT
*                    (Workspace: need N*N+BDSPAC)
*
                     CALL SBDSQR( 'U', N, N, N, 0, S, WORK( IE ), VT,
     $                            LDVT, WORK( IU ), LDWRKU, DUM, 1,
     $                            WORK( IWORK ), INFO )
*
*                    Multiply Q in U by left singular vectors of R in
*                    WORK(IU), storing result in A
*                    (Workspace: need N*N)
*
                     CALL SGEMM( 'N', 'N', M, N, N, ONE, U, LDU,
     $                           WORK( IU ), LDWRKU, ZERO, A, LDA )
*
*                    Copy left singular vectors of A from A to U
*
                     CALL SLACPY( 'F', M, N, A, LDA, U, LDU )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
```

```
                  IWORK = ITAU + N
*
*                 Compute A=Q*R, copying result to U
*                 (Workspace: need 2*N, prefer N+N*NB)
*
                  CALL SGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  CALL SLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                 Generate Q in U
*                 (Workspace: need N+M, prefer N+M*NB)
*
                  CALL SORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Copy R from A to VT, zeroing out below it
*
                  CALL SLACPY( 'U', N, N, A, LDA, VT, LDVT )
                  IF( N.GT.1 )
     $               CALL SLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                            VT( 2, 1 ), LDVT )
                  IE = ITAU
                  ITAUQ = IE + N
                  ITAUP = ITAUQ + N
                  IWORK = ITAUP + N
*
*                 Bidiagonalize R in VT
*                 (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                  CALL SGEBRD( N, N, VT, LDVT, S, WORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Multiply Q in U by left bidiagonalizing vectors
*                 in VT
*                 (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                  CALL SORMBR( 'Q', 'R', 'N', M, N, N, VT, LDVT,
     $                         WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                         LWORK-IWORK+1, IERR )
*
*                 Generate right bidiagonalizing vectors in VT
*                 (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                  CALL SORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IWORK = IE + N
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of A in U and computing right
```

```
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
                     CALL SBDSQR( 'U', N, N, M, 0, S, WORK( IE ), VT,
     $                            LDVT, U, LDU, DUM, 1, WORK( IWORK ),
     $                            INFO )
*
                  END IF
*
               END IF
*
            END IF
*
         ELSE
*
*           M .LT. MNTHR
*
*           Path 10 (M at least N, but not much larger)
*           Reduce to bidiagonal form without QR decomposition
*
            IE = 1
            ITAUQ = IE + N
            ITAUP = ITAUQ + N
            IWORK = ITAUP + N
*
*           Bidiagonalize A
*           (Workspace: need 3*N+M, prefer 3*N+(M+N)*NB)
*
            CALL SGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                   WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                   IERR )
            IF( WNTUAS ) THEN
*
*              If left singular vectors desired in U, copy result to U
*              and generate left bidiagonalizing vectors in U
*              (Workspace: need 3*N+NCU, prefer 3*N+NCU*NB)
*
               CALL SLACPY( 'L', M, N, A, LDA, U, LDU )
               IF( WNTUS )
     $            NCU = N
               IF( WNTUA )
     $            NCU = M
               CALL SORGBR( 'Q', M, NCU, N, U, LDU, WORK( ITAUQ ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IF( WNTVAS ) THEN
*
*              If right singular vectors desired in VT, copy result to
*              VT and generate right bidiagonalizing vectors in VT
*              (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
```

```
*
            CALL SLACPY( 'U', N, N, A, LDA, VT, LDVT )
            CALL SORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                   WORK( IWORK ), LWORK-IWORK+1, IERR )
         END IF
         IF( WNTUO ) THEN
*
*           If left singular vectors desired in A, generate left
*           bidiagonalizing vectors in A
*           (Workspace: need 4*N, prefer 3*N+N*NB)
*
            CALL SORGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
     $                   WORK( IWORK ), LWORK-IWORK+1, IERR )
         END IF
         IF( WNTVO ) THEN
*
*           If right singular vectors desired in A, generate right
*           bidiagonalizing vectors in A
*           (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
            CALL SORGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                   WORK( IWORK ), LWORK-IWORK+1, IERR )
         END IF
         IWORK = IE + N
         IF( WNTUAS .OR. WNTUO )
     $      NRU = M
         IF( WNTUN )
     $      NRU = 0
         IF( WNTVAS .OR. WNTVO )
     $      NCVT = N
         IF( WNTVN )
     $      NCVT = 0
         IF( ( .NOT.WNTUO ) .AND. ( .NOT.WNTVO ) ) THEN
*
*           Perform bidiagonal QR iteration, if desired, computing
*           left singular vectors in U and computing right singular
*           vectors in VT
*           (Workspace: need BDSPAC)
*
            CALL SBDSQR( 'U', N, NCVT, NRU, 0, S, WORK( IE ), VT,
     $                   LDVT, U, LDU, DUM, 1, WORK( IWORK ), INFO )
         ELSE IF( ( .NOT.WNTUO ) .AND. WNTVO ) THEN
*
*           Perform bidiagonal QR iteration, if desired, computing
*           left singular vectors in U and computing right singular
*           vectors in A
*           (Workspace: need BDSPAC)
*
            CALL SBDSQR( 'U', N, NCVT, NRU, 0, S, WORK( IE ), A, LDA,
     $                   U, LDU, DUM, 1, WORK( IWORK ), INFO )
```

```
            ELSE
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in A and computing right singular
*              vectors in VT
*              (Workspace: need BDSPAC)
*
               CALL SBDSQR( 'U', N, NCVT, NRU, 0, S, WORK( IE ), VT,
     $                      LDVT, A, LDA, DUM, 1, WORK( IWORK ), INFO )
            END IF
*
         END IF
*
      ELSE
*
         A has more columns than rows. If A has sufficiently more
         columns than rows, first reduce using the LQ decomposition (if
         sufficient workspace available)
*
         IF( N.GE.MNTHR ) THEN
*
            IF( WNTVN ) THEN
*
*              Path 1t(N much larger than M, JOBVT='N')
*              No right singular vectors to be computed
*
               ITAU = 1
               IWORK = ITAU + M
*
*              Compute A=L*Q
*              (Workspace: need 2*M, prefer M+M*NB)
*
               CALL SGELQF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
     $                      LWORK-IWORK+1, IERR )
*
*              Zero out above L
*
               CALL SLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ), LDA )
               IE = 1
               ITAUQ = IE + M
               ITAUP = ITAUQ + M
               IWORK = ITAUP + M
*
*              Bidiagonalize L in A
*              (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
               CALL SGEBRD( M, M, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                      IERR )
               IF( WNTUO .OR. WNTUAS ) THEN
```

```
*
*                 If left singular vectors desired, generate Q
*                 (Workspace: need 4*M, prefer 3*M+M*NB)
*
                  CALL SORGBR( 'Q', M, M, M, A, LDA, WORK( ITAUQ ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
               END IF
               IWORK = IE + M
               NRU = 0
               IF( WNTUO .OR. WNTUAS )
     $            NRU = M
*
*              Perform bidiagonal QR iteration, computing left singular
*              vectors of A in A if desired
*              (Workspace: need BDSPAC)
*
               CALL SBDSQR( 'U', M, 0, NRU, 0, S, WORK( IE ), DUM, 1, A,
     $                      LDA, DUM, 1, WORK( IWORK ), INFO )
*
*              If left singular vectors desired in U, copy them there
*
               IF( WNTUAS )
     $            CALL SLACPY( 'F', M, M, A, LDA, U, LDU )
*
            ELSE IF( WNTVO .AND. WNTUN ) THEN
*
*              Path 2t(N much larger than M, JOBU='N', JOBVT='O')
*              M right singular vectors to be overwritten on A and
*              no left singular vectors to be computed
*
               IF( LWORK.GE.M*M+MAX( 4*M, BDSPAC ) ) THEN
*
*                 Sufficient workspace for a fast algorithm
*
                  IR = 1
                  IF( LWORK.GE.MAX( WRKBL, LDA*N+M )+LDA*M ) THEN
*
*                    WORK(IU) is LDA by N and WORK(IR) is LDA by M
*
                     LDWRKU = LDA
                     CHUNK = N
                     LDWRKR = LDA
                  ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N+M )+M*M ) THEN
*
*                    WORK(IU) is LDA by N and WORK(IR) is M by M
*
                     LDWRKU = LDA
                     CHUNK = N
                     LDWRKR = M
                  ELSE
```

```
*
*                   WORK(IU) is M by CHUNK and WORK(IR) is M by M
*
                    LDWRKU = M
                    CHUNK = ( LWORK-M*M-M ) / M
                    LDWRKR = M
                 END IF
                 ITAU = IR + LDWRKR*M
                 IWORK = ITAU + M
*
*                Compute A=L*Q
*                (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                 CALL SGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy L to WORK(IR) and zero out above it
*
                 CALL SLACPY( 'L', M, M, A, LDA, WORK( IR ), LDWRKR )
                 CALL SLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                        WORK( IR+LDWRKR ), LDWRKR )
*
*                Generate Q in A
*                (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                 CALL SORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = ITAU
                 ITAUQ = IE + M
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Bidiagonalize L in WORK(IR)
*                (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                 CALL SGEBRD( M, M, WORK( IR ), LDWRKR, S, WORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Generate right vectors bidiagonalizing L
*                (Workspace: need M*M+4*M-1, prefer M*M+3*M+(M-1)*NB)
*
                 CALL SORGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                        WORK( ITAUP ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
                 IWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing right
*                singular vectors of L in WORK(IR)
*                (Workspace: need M*M+BDSPAC)
```

```
*
                 CALL SBDSQR( 'U', M, M, 0, 0, S, WORK( IE ),
     $                        WORK( IR ), LDWRKR, DUM, 1, DUM, 1,
     $                        WORK( IWORK ), INFO )
                 IU = IE + M
*
*                Multiply right singular vectors of L in WORK(IR) by Q
*                in A, storing result in WORK(IU) and copying to A
*                (Workspace: need M*M+2*M, prefer M*M+M*N+M)
*
                 DO 30 I = 1, N, CHUNK
                    BLK = MIN( N-I+1, CHUNK )
                    CALL SGEMM( 'N', 'N', M, BLK, M, ONE, WORK( IR ),
     $                          LDWRKR, A( 1, I ), LDA, ZERO,
     $                          WORK( IU ), LDWRKU )
                    CALL SLACPY( 'F', M, BLK, WORK( IU ), LDWRKU,
     $                           A( 1, I ), LDA )
   30            CONTINUE
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 IE = 1
                 ITAUQ = IE + M
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Bidiagonalize A
*                (Workspace: need 3*M+N, prefer 3*M+(M+N)*NB)
*
                 CALL SGEBRD( M, N, A, LDA, S, WORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Generate right vectors bidiagonalizing A
*                (Workspace: need 4*M, prefer 3*M+M*NB)
*
                 CALL SORGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing right
*                singular vectors of A in A
*                (Workspace: need BDSPAC)
*
                 CALL SBDSQR( 'L', M, N, 0, 0, S, WORK( IE ), A, LDA,
     $                        DUM, 1, DUM, 1, WORK( IWORK ), INFO )
*
              END IF
```

```
*
            ELSE IF( WNTVO .AND. WNTUAS ) THEN
*
*              Path 3t(N much larger than M, JOBU='S' or 'A', JOBVT='O')
*              M right singular vectors to be overwritten on A and
*              M left singular vectors to be computed in U
*
               IF( LWORK.GE.M*M+MAX( 4*M, BDSPAC ) ) THEN
*
*                 Sufficient workspace for a fast algorithm
*
                  IR = 1
                  IF( LWORK.GE.MAX( WRKBL, LDA*N+M )+LDA*M ) THEN
*
*                    WORK(IU) is LDA by N and WORK(IR) is LDA by M
*
                     LDWRKU = LDA
                     CHUNK = N
                     LDWRKR = LDA
                  ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N+M )+M*M ) THEN
*
*                    WORK(IU) is LDA by N and WORK(IR) is M by M
*
                     LDWRKU = LDA
                     CHUNK = N
                     LDWRKR = M
                  ELSE
*
*                    WORK(IU) is M by CHUNK and WORK(IR) is M by M
*
                     LDWRKU = M
                     CHUNK = ( LWORK-M*M-M ) / M
                     LDWRKR = M
                  END IF
                  ITAU = IR + LDWRKR*M
                  IWORK = ITAU + M
*
*                 Compute A=L*Q
*                 (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                  CALL SGELQF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Copy L to U, zeroing about above it
*
                  CALL SLACPY( 'L', M, M, A, LDA, U, LDU )
                  CALL SLASET( 'U', M-1, M-1, ZERO, ZERO, U( 1, 2 ),
     $                         LDU )
*
*                 Generate Q in A
```

```
*                 (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                  CALL SORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IE = ITAU
                  ITAUQ = IE + M
                  ITAUP = ITAUQ + M
                  IWORK = ITAUP + M
*
*                 Bidiagonalize L in U, copying result to WORK(IR)
*                 (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                  CALL SGEBRD( M, M, U, LDU, S, WORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  CALL SLACPY( 'U', M, M, U, LDU, WORK( IR ), LDWRKR )
*
*                 Generate right vectors bidiagonalizing L in WORK(IR)
*                 (Workspace: need M*M+4*M-1, prefer M*M+3*M+(M-1)*NB)
*
                  CALL SORGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                         WORK( ITAUP ), WORK( IWORK ),
     $                         LWORK-IWORK+1, IERR )
*
*                 Generate left vectors bidiagonalizing L in U
*                 (Workspace: need M*M+4*M, prefer M*M+3*M+M*NB)
*
                  CALL SORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IWORK = IE + M
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of L in U, and computing right
*                 singular vectors of L in WORK(IR)
*                 (Workspace: need M*M+BDSPAC)
*
                  CALL SBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                         WORK( IR ), LDWRKR, U, LDU, DUM, 1,
     $                         WORK( IWORK ), INFO )
                  IU = IE + M
*
*                 Multiply right singular vectors of L in WORK(IR) by Q
*                 in A, storing result in WORK(IU) and copying to A
*                 (Workspace: need M*M+2*M, prefer M*M+M*N+M))
*
                  DO 40 I = 1, N, CHUNK
                     BLK = MIN( N-I+1, CHUNK )
                     CALL SGEMM( 'N', 'N', M, BLK, M, ONE, WORK( IR ),
     $                           LDWRKR, A( 1, I ), LDA, ZERO,
     $                           WORK( IU ), LDWRKU )
```

```
                 CALL SLACPY( 'F', M, BLK, WORK( IU ), LDWRKU,
     $                        A( 1, I ), LDA )
   40         CONTINUE
*
            ELSE
*
*              Insufficient workspace for a fast algorithm
*
               ITAU = 1
               IWORK = ITAU + M
*
*              Compute A=L*Q
*              (Workspace: need 2*M, prefer M+M*NB)
*
               CALL SGELQF( M, N, A, LDA, WORK( ITAU ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*              Copy L to U, zeroing out above it
*
               CALL SLACPY( 'L', M, M, A, LDA, U, LDU )
               CALL SLASET( 'U', M-1, M-1, ZERO, ZERO, U( 1, 2 ),
     $                      LDU )
*
*              Generate Q in A
*              (Workspace: need 2*M, prefer M+M*NB)
*
               CALL SORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
               IE = ITAU
               ITAUQ = IE + M
               ITAUP = ITAUQ + M
               IWORK = ITAUP + M
*
*              Bidiagonalize L in U
*              (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
               CALL SGEBRD( M, M, U, LDU, S, WORK( IE ),
     $                      WORK( ITAUQ ), WORK( ITAUP ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*              Multiply right vectors bidiagonalizing L by Q in A
*              (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
               CALL SORMBR( 'P', 'L', 'T', M, N, M, U, LDU,
     $                      WORK( ITAUP ), A, LDA, WORK( IWORK ),
     $                      LWORK-IWORK+1, IERR )
*
*              Generate left vectors bidiagonalizing L in U
*              (Workspace: need 4*M, prefer 3*M+M*NB)
*
```

```
                 CALL SORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of A in U and computing right
*                singular vectors of A in A
*                (Workspace: need BDSPAC)
*
                 CALL SBDSQR( 'U', M, N, M, 0, S, WORK( IE ), A, LDA,
     $                        U, LDU, DUM, 1, WORK( IWORK ), INFO )
*
              END IF
*
           ELSE IF( WNTVS ) THEN
*
              IF( WNTUN ) THEN
*
*                Path 4t(N much larger than M, JOBU='N', JOBVT='S')
*                M right singular vectors to be computed in VT and
*                no left singular vectors to be computed
*
                 IF( LWORK.GE.M*M+MAX( 4*M, BDSPAC ) ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IR = 1
                    IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                      WORK(IR) is LDA by M
*
                       LDWRKR = LDA
                    ELSE
*
*                      WORK(IR) is M by M
*
                       LDWRKR = M
                    END IF
                    ITAU = IR + LDWRKR*M
                    IWORK = ITAU + M
*
*                   Compute A=L*Q
*                   (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                    CALL SGELQF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy L to WORK(IR), zeroing out above it
*
                    CALL SLACPY( 'L', M, M, A, LDA, WORK( IR ),
```

```
     $                          LDWRKR )
                      CALL SLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                          WORK( IR+LDWRKR ), LDWRKR )
*
*                     Generate Q in A
*                     (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                      CALL SORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                      IE = ITAU
                      ITAUQ = IE + M
                      ITAUP = ITAUQ + M
                      IWORK = ITAUP + M
*
*                     Bidiagonalize L in WORK(IR)
*                     (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                      CALL SGEBRD( M, M, WORK( IR ), LDWRKR, S,
     $                          WORK( IE ), WORK( ITAUQ ),
     $                          WORK( ITAUP ), WORK( IWORK ),
     $                          LWORK-IWORK+1, IERR )
*
*                     Generate right vectors bidiagonalizing L in
*                     WORK(IR)
*                     (Workspace: need M*M+4*M, prefer M*M+3*M+(M-1)*NB)
*
                      CALL SORGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                          WORK( ITAUP ), WORK( IWORK ),
     $                          LWORK-IWORK+1, IERR )
                      IWORK = IE + M
*
*                     Perform bidiagonal QR iteration, computing right
*                     singular vectors of L in WORK(IR)
*                     (Workspace: need M*M+BDSPAC)
*
                      CALL SBDSQR( 'U', M, M, 0, 0, S, WORK( IE ),
     $                          WORK( IR ), LDWRKR, DUM, 1, DUM, 1,
     $                          WORK( IWORK ), INFO )
*
*                     Multiply right singular vectors of L in WORK(IR) by
*                     Q in A, storing result in VT
*                     (Workspace: need M*M)
*
                      CALL SGEMM( 'N', 'N', M, N, M, ONE, WORK( IR ),
     $                         LDWRKR, A, LDA, ZERO, VT, LDVT )
*
                   ELSE
*
*                     Insufficient workspace for a fast algorithm
*
```

```
                  ITAU = 1
                  IWORK = ITAU + M
*
*                 Compute A=L*Q
*                 (Workspace: need 2*M, prefer M+M*NB)
*
                  CALL SGELQF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Copy result to VT
*
                  CALL SLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                 Generate Q in VT
*                 (Workspace: need 2*M, prefer M+M*NB)
*
                  CALL SORGLQ( M, N, M, VT, LDVT, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IE = ITAU
                  ITAUQ = IE + M
                  ITAUP = ITAUQ + M
                  IWORK = ITAUP + M
*
*                 Zero out above L in A
*
                  CALL SLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ),
     $                         LDA )
*
*                 Bidiagonalize L in A
*                 (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                  CALL SGEBRD( M, M, A, LDA, S, WORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Multiply right vectors bidiagonalizing L by Q in VT
*                 (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                  CALL SORMBR( 'P', 'L', 'T', M, N, M, A, LDA,
     $                         WORK( ITAUP ), VT, LDVT,
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IWORK = IE + M
*
*                 Perform bidiagonal QR iteration, computing right
*                 singular vectors of A in VT
*                 (Workspace: need BDSPAC)
*
                  CALL SBDSQR( 'U', M, N, 0, 0, S, WORK( IE ), VT,
     $                         LDVT, DUM, 1, DUM, 1, WORK( IWORK ),
     $                         INFO )
```

```
*
                 END IF
*
              ELSE IF( WNTUO ) THEN
*
*                Path 5t(N much larger than M, JOBU='O', JOBVT='S')
*                M right singular vectors to be computed in VT and
*                M left singular vectors to be overwritten on A
*
                 IF( LWORK.GE.2*M*M+MAX( 4*M, BDSPAC ) ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IU = 1
                    IF( LWORK.GE.WRKBL+2*LDA*M ) THEN
*
*                      WORK(IU) is LDA by M and WORK(IR) is LDA by M
*
                       LDWRKU = LDA
                       IR = IU + LDWRKU*M
                       LDWRKR = LDA
                    ELSE IF( LWORK.GE.WRKBL+( LDA+M )*M ) THEN
*
*                      WORK(IU) is LDA by M and WORK(IR) is M by M
*
                       LDWRKU = LDA
                       IR = IU + LDWRKU*M
                       LDWRKR = M
                    ELSE
*
*                      WORK(IU) is M by M and WORK(IR) is M by M
*
                       LDWRKU = M
                       IR = IU + LDWRKU*M
                       LDWRKR = M
                    END IF
                    ITAU = IR + LDWRKR*M
                    IWORK = ITAU + M
*
*                   Compute A=L*Q
*                   (Workspace: need 2*M*M+2*M, prefer 2*M*M+M+M*NB)
*
                    CALL SGELQF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy L to WORK(IU), zeroing out below it
*
                    CALL SLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                           LDWRKU )
                    CALL SLASET( 'U', M-1, M-1, ZERO, ZERO,
```

```
     $                          WORK( IU+LDWRKU ), LDWRKU )
*
*                    Generate Q in A
*                    (Workspace: need 2*M*M+2*M, prefer 2*M*M+M+M*NB)
*
                     CALL SORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Bidiagonalize L in WORK(IU), copying result to
*                    WORK(IR)
*                    (Workspace: need 2*M*M+4*M,
*                                prefer 2*M*M+3*M+2*M*NB)
*
                     CALL SGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                     CALL SLACPY( 'L', M, M, WORK( IU ), LDWRKU,
     $                           WORK( IR ), LDWRKR )
*
*                    Generate right bidiagonalizing vectors in WORK(IU)
*                    (Workspace: need 2*M*M+4*M-1,
*                                prefer 2*M*M+3*M+(M-1)*NB)
*
                     CALL SORGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors in WORK(IR)
*                    (Workspace: need 2*M*M+4*M, prefer 2*M*M+3*M+M*NB)
*
                     CALL SORGBR( 'Q', M, M, M, WORK( IR ), LDWRKR,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of L in WORK(IR) and computing
*                    right singular vectors of L in WORK(IU)
*                    (Workspace: need 2*M*M+BDSPAC)
*
                     CALL SBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                           WORK( IU ), LDWRKU, WORK( IR ),
     $                           LDWRKR, DUM, 1, WORK( IWORK ), INFO )
*
*                    Multiply right singular vectors of L in WORK(IU) by
```

```
*                    Q in A, storing result in VT
*                    (Workspace: need M*M)
*
                     CALL SGEMM( 'N', 'N', M, N, M, ONE, WORK( IU ),
     $                           LDWRKU, A, LDA, ZERO, VT, LDVT )
*
*                    Copy left singular vectors of L to A
*                    (Workspace: need M*M)
*
                     CALL SLACPY( 'F', M, M, WORK( IR ), LDWRKR, A,
     $                            LDA )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL SGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL SLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL SORGLQ( M, N, M, VT, LDVT, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Zero out above L in A
*
                     CALL SLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ),
     $                            LDA )
*
*                    Bidiagonalize L in A
*                    (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                     CALL SGEBRD( M, M, A, LDA, S, WORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply right vectors bidiagonalizing L by Q in VT
*                    (Workspace: need 3*M+N, prefer 3*M+N*NB)
```

```
*
                    CALL SORMBR( 'P', 'L', 'T', M, N, M, A, LDA,
     $                           WORK( ITAUP ), VT, LDVT,
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Generate left bidiagonalizing vectors of L in A
*                   (Workspace: need 4*M, prefer 3*M+M*NB)
*
                    CALL SORGBR( 'Q', M, M, M, A, LDA, WORK( ITAUQ ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IWORK = IE + M
*
*                   Perform bidiagonal QR iteration, compute left
*                   singular vectors of A in A and compute right
*                   singular vectors of A in VT
*                   (Workspace: need BDSPAC)
*
                    CALL SBDSQR( 'U', M, N, M, 0, S, WORK( IE ), VT,
     $                           LDVT, A, LDA, DUM, 1, WORK( IWORK ),
     $                           INFO )
*
                 END IF
*
              ELSE IF( WNTUAS ) THEN
*
*                Path 6t(N much larger than M, JOBU='S' or 'A',
*                        JOBVT='S')
*                M right singular vectors to be computed in VT and
*                M left singular vectors to be computed in U
*
                 IF( LWORK.GE.M*M+MAX( 4*M, BDSPAC ) ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IU = 1
                    IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                      WORK(IU) is LDA by N
*
                       LDWRKU = LDA
                    ELSE
*
*                      WORK(IU) is LDA by M
*
                       LDWRKU = M
                    END IF
                    ITAU = IU + LDWRKU*M
                    IWORK = ITAU + M
*
*                   Compute A=L*Q
```

```
*                    (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                     CALL SGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to WORK(IU), zeroing out above it
*
                     CALL SLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                            LDWRKU )
                     CALL SLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                            WORK( IU+LDWRKU ), LDWRKU )
*
*                    Generate Q in A
*                    (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                     CALL SORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Bidiagonalize L in WORK(IU), copying result to U
*                    (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                     CALL SGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                            WORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     CALL SLACPY( 'L', M, M, WORK( IU ), LDWRKU, U,
     $                            LDU )
*
*                    Generate right bidiagonalizing vectors in WORK(IU)
*                    (Workspace: need M*M+4*M-1,
*                                prefer M*M+3*M+(M-1)*NB)
*
                     CALL SORGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors in U
*                    (Workspace: need M*M+4*M, prefer M*M+3*M+M*NB)
*
                     CALL SORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of L in U and computing right
*                    singular vectors of L in WORK(IU)
```

```
*                    (Workspace: need M*M+BDSPAC)
*
                     CALL SBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                            WORK( IU ), LDWRKU, U, LDU, DUM, 1,
     $                            WORK( IWORK ), INFO )
*
*                    Multiply right singular vectors of L in WORK(IU) by
*                    Q in A, storing result in VT
*                    (Workspace: need M*M)
*
                     CALL SGEMM( 'N', 'N', M, N, M, ONE, WORK( IU ),
     $                           LDWRKU, A, LDA, ZERO, VT, LDVT )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL SGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL SLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL SORGLQ( M, N, M, VT, LDVT, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to U, zeroing out above it
*
                     CALL SLACPY( 'L', M, M, A, LDA, U, LDU )
                     CALL SLASET( 'U', M-1, M-1, ZERO, ZERO, U( 1, 2 ),
     $                            LDU )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Bidiagonalize L in U
*                    (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                     CALL SGEBRD( M, M, U, LDU, S, WORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
```

```
*                    Multiply right bidiagonalizing vectors in U by Q
*                    in VT
*                    (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                     CALL SORMBR( 'P', 'L', 'T', M, N, M, U, LDU,
     $                            WORK( ITAUP ), VT, LDVT,
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors in U
*                    (Workspace: need 4*M, prefer 3*M+M*NB)
*
                     CALL SORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
                     CALL SBDSQR( 'U', M, N, M, 0, S, WORK( IE ), VT,
     $                            LDVT, U, LDU, DUM, 1, WORK( IWORK ),
     $                            INFO )
*
                  END IF
*
               END IF
*
            ELSE IF( WNTVA ) THEN
*
               IF( WNTUN ) THEN
*
                  Path 7t(N much larger than M, JOBU='N', JOBVT='A')
*                 N right singular vectors to be computed in VT and
*                 no left singular vectors to be computed
*
                  IF( LWORK.GE.M*M+MAX( N+M, 4*M, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IR = 1
                     IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                       WORK(IR) is LDA by M
*
                        LDWRKR = LDA
                     ELSE
*
*                       WORK(IR) is M by M
*
```

```
                 LDWRKR = M
              END IF
              ITAU = IR + LDWRKR*M
              IWORK = ITAU + M
*
*             Compute A=L*Q, copying result to VT
*             (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
              CALL SGELQF( M, N, A, LDA, WORK( ITAU ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
              CALL SLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*             Copy L to WORK(IR), zeroing out above it
*
              CALL SLACPY( 'L', M, M, A, LDA, WORK( IR ),
     $                     LDWRKR )
              CALL SLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                     WORK( IR+LDWRKR ), LDWRKR )
*
*             Generate Q in VT
*             (Workspace: need M*M+M+N, prefer M*M+M+N*NB)
*
              CALL SORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
              IE = ITAU
              ITAUQ = IE + M
              ITAUP = ITAUQ + M
              IWORK = ITAUP + M
*
*             Bidiagonalize L in WORK(IR)
*             (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
              CALL SGEBRD( M, M, WORK( IR ), LDWRKR, S,
     $                     WORK( IE ), WORK( ITAUQ ),
     $                     WORK( ITAUP ), WORK( IWORK ),
     $                     LWORK-IWORK+1, IERR )
*
*             Generate right bidiagonalizing vectors in WORK(IR)
*             (Workspace: need M*M+4*M-1,
*                         prefer M*M+3*M+(M-1)*NB)
*
              CALL SORGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                     WORK( ITAUP ), WORK( IWORK ),
     $                     LWORK-IWORK+1, IERR )
              IWORK = IE + M
*
*             Perform bidiagonal QR iteration, computing right
*             singular vectors of L in WORK(IR)
*             (Workspace: need M*M+BDSPAC)
*
```

```
                 CALL SBDSQR( 'U', M, M, 0, 0, S, WORK( IE ),
     $                        WORK( IR ), LDWRKR, DUM, 1, DUM, 1,
     $                        WORK( IWORK ), INFO )
*
*                Multiply right singular vectors of L in WORK(IR) by
*                Q in VT, storing result in A
*                (Workspace: need M*M)
*
                 CALL SGEMM( 'N', 'N', M, N, M, ONE, WORK( IR ),
     $                      LDWRKR, VT, LDVT, ZERO, A, LDA )
*
*                Copy right singular vectors of A from A to VT
*
                 CALL SLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + M
*
*                Compute A=L*Q, copying result to VT
*                (Workspace: need 2*M, prefer M+M*NB)
*
                 CALL SGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL SLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                Generate Q in VT
*                (Workspace: need M+N, prefer M+N*NB)
*
                 CALL SORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = ITAU
                 ITAUQ = IE + M
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Zero out above L in A
*
                 CALL SLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ),
     $                        LDA )
*
*                Bidiagonalize L in A
*                (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                 CALL SGEBRD( M, M, A, LDA, S, WORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
```

```
*
*                    Multiply right bidiagonalizing vectors in A by Q
*                    in VT
*                    (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                     CALL SORMBR( 'P', 'L', 'T', M, N, M, A, LDA,
     $                            WORK( ITAUP ), VT, LDVT,
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing right
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
                     CALL SBDSQR( 'U', M, N, 0, 0, S, WORK( IE ), VT,
     $                            LDVT, DUM, 1, DUM, 1, WORK( IWORK ),
     $                            INFO )
*
                  END IF
*
               ELSE IF( WNTUO ) THEN
*
                  Path 8t(N much larger than M, JOBU='O', JOBVT='A')
*                 N right singular vectors to be computed in VT and
*                 M left singular vectors to be overwritten on A
*
                  IF( LWORK.GE.2*M*M+MAX( N+M, 4*M, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+2*LDA*M ) THEN
*
*                       WORK(IU) is LDA by M and WORK(IR) is LDA by M
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*M
                        LDWRKR = LDA
                     ELSE IF( LWORK.GE.WRKBL+( LDA+M )*M ) THEN
*
*                       WORK(IU) is LDA by M and WORK(IR) is M by M
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*M
                        LDWRKR = M
                     ELSE
*
*                       WORK(IU) is M by M and WORK(IR) is M by M
*
                        LDWRKU = M
```

```
                      IR = IU + LDWRKU*M
                      LDWRKR = M
                   END IF
                   ITAU = IR + LDWRKR*M
                   IWORK = ITAU + M
*
*                  Compute A=L*Q, copying result to VT
*                  (Workspace: need 2*M*M+2*M, prefer 2*M*M+M+M*NB)
*
                   CALL SGELQF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                   CALL SLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                  Generate Q in VT
*                  (Workspace: need 2*M*M+M+N, prefer 2*M*M+M+N*NB)
*
                   CALL SORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                  Copy L to WORK(IU), zeroing out above it
*
                   CALL SLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                           LDWRKU )
                   CALL SLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                           WORK( IU+LDWRKU ), LDWRKU )
                   IE = ITAU
                   ITAUQ = IE + M
                   ITAUP = ITAUQ + M
                   IWORK = ITAUP + M
*
*                  Bidiagonalize L in WORK(IU), copying result to
*                  WORK(IR)
*                  (Workspace: need 2*M*M+4*M,
*                              prefer 2*M*M+3*M+2*M*NB)
*
                   CALL SGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                   CALL SLACPY( 'L', M, M, WORK( IU ), LDWRKU,
     $                           WORK( IR ), LDWRKR )
*
*                  Generate right bidiagonalizing vectors in WORK(IU)
*                  (Workspace: need 2*M*M+4*M-1,
*                              prefer 2*M*M+3*M+(M-1)*NB)
*
                   CALL SORGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
```

```
*                    Generate left bidiagonalizing vectors in WORK(IR)
*                    (Workspace: need 2*M*M+4*M, prefer 2*M*M+3*M+M*NB)
*
                     CALL SORGBR( 'Q', M, M, M, WORK( IR ), LDWRKR,
     $                            WORK( ITAUQ ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of L in WORK(IR) and computing
*                    right singular vectors of L in WORK(IU)
*                    (Workspace: need 2*M*M+BDSPAC)
*
                     CALL SBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                            WORK( IU ), LDWRKU, WORK( IR ),
     $                            LDWRKR, DUM, 1, WORK( IWORK ), INFO )
*
*                    Multiply right singular vectors of L in WORK(IU) by
*                    Q in VT, storing result in A
*                    (Workspace: need M*M)
*
                     CALL SGEMM( 'N', 'N', M, N, M, ONE, WORK( IU ),
     $                           LDWRKU, VT, LDVT, ZERO, A, LDA )
*
*                    Copy right singular vectors of A from A to VT
*
                     CALL SLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
*                    Copy left singular vectors of A from WORK(IR) to A
*
                     CALL SLACPY( 'F', M, M, WORK( IR ), LDWRKR, A,
     $                            LDA )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL SGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL SLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (Workspace: need M+N, prefer M+N*NB)
*
```

```
                 CALL SORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = ITAU
                 ITAUQ = IE + M
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Zero out above L in A
*
                 CALL SLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ),
     $                        LDA )
*
*                Bidiagonalize L in A
*                (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                 CALL SGEBRD( M, M, A, LDA, S, WORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Multiply right bidiagonalizing vectors in A by Q
*                in VT
*                (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                 CALL SORMBR( 'P', 'L', 'T', M, N, M, A, LDA,
     $                        WORK( ITAUP ), VT, LDVT,
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Generate left bidiagonalizing vectors in A
*                (Workspace: need 4*M, prefer 3*M+M*NB)
*
                 CALL SORGBR( 'Q', M, M, M, A, LDA, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of A in A and computing right
*                singular vectors of A in VT
*                (Workspace: need BDSPAC)
*
                 CALL SBDSQR( 'U', M, N, M, 0, S, WORK( IE ), VT,
     $                        LDVT, A, LDA, DUM, 1, WORK( IWORK ),
     $                        INFO )
*
              END IF
*
           ELSE IF( WNTUAS ) THEN
*
              Path 9t(N much larger than M, JOBU='S' or 'A',
*                     JOBVT='A')
*             N right singular vectors to be computed in VT and
```

```
*                M left singular vectors to be computed in U
*
                 IF( LWORK.GE.M*M+MAX( N+M, 4*M, BDSPAC ) ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IU = 1
                    IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                      WORK(IU) is LDA by M
*
                       LDWRKU = LDA
                    ELSE
*
*                      WORK(IU) is M by M
*
                       LDWRKU = M
                    END IF
                    ITAU = IU + LDWRKU*M
                    IWORK = ITAU + M
*
*                   Compute A=L*Q, copying result to VT
*                   (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                    CALL SGELQF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    CALL SLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                   Generate Q in VT
*                   (Workspace: need M*M+M+N, prefer M*M+M+N*NB)
*
                    CALL SORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy L to WORK(IU), zeroing out above it
*
                    CALL SLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                           LDWRKU )
                    CALL SLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                           WORK( IU+LDWRKU ), LDWRKU )
                    IE = ITAU
                    ITAUQ = IE + M
                    ITAUP = ITAUQ + M
                    IWORK = ITAUP + M
*
*                   Bidiagonalize L in WORK(IU), copying result to U
*                   (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                    CALL SGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                           WORK( IE ), WORK( ITAUQ ),
```

```
     $                          WORK( ITAUP ), WORK( IWORK ),
     $                          LWORK-IWORK+1, IERR )
                     CALL SLACPY( 'L', M, M, WORK( IU ), LDWRKU, U,
     $                          LDU )
*
*                    Generate right bidiagonalizing vectors in WORK(IU)
*                    (Workspace: need M*M+4*M, prefer M*M+3*M+(M-1)*NB)
*
                     CALL SORGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                          WORK( ITAUP ), WORK( IWORK ),
     $                          LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors in U
*                    (Workspace: need M*M+4*M, prefer M*M+3*M+M*NB)
*
                     CALL SORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of L in U and computing right
*                    singular vectors of L in WORK(IU)
*                    (Workspace: need M*M+BDSPAC)
*
                     CALL SBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                          WORK( IU ), LDWRKU, U, LDU, DUM, 1,
     $                          WORK( IWORK ), INFO )
*
*                    Multiply right singular vectors of L in WORK(IU) by
*                    Q in VT, storing result in A
*                    (Workspace: need M*M)
*
                     CALL SGEMM( 'N', 'N', M, N, M, ONE, WORK( IU ),
     $                         LDWRKU, VT, LDVT, ZERO, A, LDA )
*
*                    Copy right singular vectors of A from A to VT
*
                     CALL SLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL SGELQF( M, N, A, LDA, WORK( ITAU ),
```

```
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                      CALL SLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                     Generate Q in VT
*                     (Workspace: need M+N, prefer M+N*NB)
*
                      CALL SORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                     Copy L to U, zeroing out above it
*
                      CALL SLACPY( 'L', M, M, A, LDA, U, LDU )
                      CALL SLASET( 'U', M-1, M-1, ZERO, ZERO, U( 1, 2 ),
     $                          LDU )
                      IE = ITAU
                      ITAUQ = IE + M
                      ITAUP = ITAUQ + M
                      IWORK = ITAUP + M
*
*                     Bidiagonalize L in U
*                     (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                      CALL SGEBRD( M, M, U, LDU, S, WORK( IE ),
     $                          WORK( ITAUQ ), WORK( ITAUP ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                     Multiply right bidiagonalizing vectors in U by Q
*                     in VT
*                     (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                      CALL SORMBR( 'P', 'L', 'T', M, N, M, U, LDU,
     $                          WORK( ITAUP ), VT, LDVT,
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                     Generate left bidiagonalizing vectors in U
*                     (Workspace: need 4*M, prefer 3*M+M*NB)
*
                      CALL SORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                      IWORK = IE + M
*
*                     Perform bidiagonal QR iteration, computing left
*                     singular vectors of A in U and computing right
*                     singular vectors of A in VT
*                     (Workspace: need BDSPAC)
*
                      CALL SBDSQR( 'U', M, N, M, 0, S, WORK( IE ), VT,
     $                          LDVT, U, LDU, DUM, 1, WORK( IWORK ),
     $                          INFO )
*
```

```
                 END IF
*
              END IF
*
           END IF
*
        ELSE
*
*          N .LT. MNTHR
*
*          Path 10t(N greater than M, but not much larger)
*          Reduce to bidiagonal form without LQ decomposition
*
           IE = 1
           ITAUQ = IE + M
           ITAUP = ITAUQ + M
           IWORK = ITAUP + M
*
*          Bidiagonalize A
*          (Workspace: need 3*M+N, prefer 3*M+(M+N)*NB)
*
           CALL SGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                  WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                  IERR )
           IF( WNTUAS ) THEN
*
*             If left singular vectors desired in U, copy result to U
*             and generate left bidiagonalizing vectors in U
*             (Workspace: need 4*M-1, prefer 3*M+(M-1)*NB)
*
              CALL SLACPY( 'L', M, M, A, LDA, U, LDU )
              CALL SORGBR( 'Q', M, M, N, U, LDU, WORK( ITAUQ ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
           IF( WNTVAS ) THEN
*
*             If right singular vectors desired in VT, copy result to
*             VT and generate right bidiagonalizing vectors in VT
*             (Workspace: need 3*M+NRVT, prefer 3*M+NRVT*NB)
*
              CALL SLACPY( 'U', M, N, A, LDA, VT, LDVT )
              IF( WNTVA )
     $           NRVT = N
              IF( WNTVS )
     $           NRVT = M
              CALL SORGBR( 'P', NRVT, N, M, VT, LDVT, WORK( ITAUP ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
           IF( WNTUO ) THEN
*
```

```
*             If left singular vectors desired in A, generate left
*             bidiagonalizing vectors in A
*             (Workspace: need 4*M-1, prefer 3*M+(M-1)*NB)
*
              CALL SORGBR( 'Q', M, M, N, A, LDA, WORK( ITAUQ ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
           IF( WNTVO ) THEN
*
*             If right singular vectors desired in A, generate right
*             bidiagonalizing vectors in A
*             (Workspace: need 4*M, prefer 3*M+M*NB)
*
              CALL SORGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
           IWORK = IE + M
           IF( WNTUAS .OR. WNTUO )
     $        NRU = M
           IF( WNTUN )
     $        NRU = 0
           IF( WNTVAS .OR. WNTVO )
     $        NCVT = N
           IF( WNTVN )
     $        NCVT = 0
           IF( ( .NOT.WNTUO ) .AND. ( .NOT.WNTVO ) ) THEN
*
*             Perform bidiagonal QR iteration, if desired, computing
*             left singular vectors in U and computing right singular
*             vectors in VT
*             (Workspace: need BDSPAC)
*
              CALL SBDSQR( 'L', M, NCVT, NRU, 0, S, WORK( IE ), VT,
     $                     LDVT, U, LDU, DUM, 1, WORK( IWORK ), INFO )
           ELSE IF( ( .NOT.WNTUO ) .AND. WNTVO ) THEN
*
*             Perform bidiagonal QR iteration, if desired, computing
*             left singular vectors in U and computing right singular
*             vectors in A
*             (Workspace: need BDSPAC)
*
              CALL SBDSQR( 'L', M, NCVT, NRU, 0, S, WORK( IE ), A, LDA,
     $                     U, LDU, DUM, 1, WORK( IWORK ), INFO )
           ELSE
*
*             Perform bidiagonal QR iteration, if desired, computing
*             left singular vectors in A and computing right singular
*             vectors in VT
*             (Workspace: need BDSPAC)
*
```

```
               CALL SBDSQR( 'L', M, NCVT, NRU, 0, S, WORK( IE ), VT,
     $                      LDVT, A, LDA, DUM, 1, WORK( IWORK ), INFO )
            END IF
*
         END IF
*
      END IF
*
*     If SBDSQR failed to converge, copy unconverged superdiagonals
*     to WORK( 2:MINMN )
*
      IF( INFO.NE.0 ) THEN
         IF( IE.GT.2 ) THEN
            DO 50 I = 1, MINMN - 1
               WORK( I+1 ) = WORK( I+IE-1 )
   50       CONTINUE
         END IF
         IF( IE.LT.2 ) THEN
            DO 60 I = MINMN - 1, 1, -1
               WORK( I+1 ) = WORK( I+IE-1 )
   60       CONTINUE
         END IF
      END IF
*
*     Undo scaling if necessary
*
      IF( ISCL.EQ.1 ) THEN
         IF( ANRM.GT.BIGNUM )
     $      CALL SLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( INFO.NE.0 .AND. ANRM.GT.BIGNUM )
     $      CALL SLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN-1, 1, WORK( 2 ),
     $                   MINMN, IERR )
         IF( ANRM.LT.SMLNUM )
     $      CALL SLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( INFO.NE.0 .AND. ANRM.LT.SMLNUM )
     $      CALL SLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN-1, 1, WORK( 2 ),
     $                   MINMN, IERR )
      END IF
*
*     Return optimal workspace in WORK(1)
*
      WORK( 1 ) = MAXWRK
*
      RETURN
*
*     End of SGESVD
*
      END
```

———-

29.3.4 lisp code

— sgesvd.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sgesvd (jobu jobvt m n a lda s u ldu vt ldvt work lwork info)
    (declare (type (array single-float (*)) work vt u s a)
             (type (f2cl-lib:integer4) info lwork ldvt ldu lda n m)
             (type (string *) jobvt jobu))
    (f2cl-lib:with-multi-array-data
        ((jobu character jobu-%data% jobu-%offset%)
         (jobvt character jobvt-%data% jobvt-%offset%)
         (a single-float a-%data% a-%offset%)
         (s single-float s-%data% s-%offset%)
         (u single-float u-%data% u-%offset%)
         (vt single-float vt-%data% vt-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((dum (make-array 1 :element-type 'single-float)) (anrm 0.0)
             (bignum 0.0) (eps 0.0) (smlnum 0.0) (lwork_sgeqrf 0)
             (lwork_sorgqr_n 0) (lwork_sorgqr_m 0) (lwork_sgebrd 0)
             (lwork_sorgbr_p 0) (lwork_sorgbr_q 0) (lwork_sgelqf 0)
             (lwork_sorglq_n 0) (lwork_sorglq_m 0) (bdspac 0) (blk 0) (chunk 0)
             (i 0) (ie 0) (ierr 0) (ir 0) (iscl 0) (itau 0) (itaup 0) (itauq 0)
             (iu 0) (iwork 0) (ldwrkr 0) (ldwrku 0) (maxwrk 0) (minmn 0)
             (minwrk 0) (mnthr 0) (ncu 0) (ncvt 0) (nru 0) (nrvt 0) (wrkbl 0)
             (lquery nil) (wntua nil) (wntuas nil) (wntun nil) (wntuo nil)
             (wntus nil) (wntva nil) (wntvas nil) (wntvn nil) (wntvo nil)
             (wntvs nil))
        (declare (type (array single-float (1)) dum)
                 (type (single-float) anrm bignum eps smlnum)
                 (type (f2cl-lib:integer4) lwork_sgeqrf lwork_sorgqr_n
                  lwork_sorgqr_m lwork_sgebrd lwork_sorgbr_p lwork_sorgbr_q
                  lwork_sgelqf lwork_sorglq_n lwork_sorglq_m bdspac blk chunk i
                  ie ierr ir iscl itau itaup itauq iu iwork ldwrkr ldwrku
                  maxwrk minmn minwrk mnthr ncu ncvt nru nrvt wrkbl)
                 (type f2cl-lib:logical lquery wntua wntuas wntun wntuo wntus
                  wntva wntvas wntvn wntvo wntvs))
        (setf info 0)
        (setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (setf wntua
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame jobu "A")
                  (declare (ignore var-1))
```

```
          (when var-0 (setf jobu var-0))
          ret-val))
(setf wntus
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobu "S")
          (declare (ignore var-1))
          (when var-0 (setf jobu var-0))
          ret-val))
(setf wntuas (or wntua wntus))
(setf wntuo
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobu "O")
          (declare (ignore var-1))
          (when var-0 (setf jobu var-0))
          ret-val))
(setf wntun
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobu "N")
          (declare (ignore var-1))
          (when var-0 (setf jobu var-0))
          ret-val))
(setf wntva
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvt "A")
          (declare (ignore var-1))
          (when var-0 (setf jobvt var-0))
          ret-val))
(setf wntvs
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvt "S")
          (declare (ignore var-1))
          (when var-0 (setf jobvt var-0))
          ret-val))
(setf wntvas (or wntva wntvs))
(setf wntvo
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvt "O")
          (declare (ignore var-1))
          (when var-0 (setf jobvt var-0))
          ret-val))
(setf wntvn
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvt "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvt var-0))
          ret-val))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((not (or wntua wntus wntuo wntun)) (setf info -1))
      ((or (not (or wntva wntvs wntvo wntvn)) (and wntvo wntuo))
       (setf info -2))
```

```
      ((< m 0) (setf info -3)) ((< n 0) (setf info -4))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -6))
      ((or (< ldu 1) (and wntuas (< ldu m))) (setf info -9))
      ((or (< ldvt 1) (and wntva (< ldvt n))
           (and wntvs (< ldvt minmn)))
       (setf info -11)))
(cond
 ((= info 0) (setf minwrk 1) (setf maxwrk 1)
  (cond
   ((and (>= m n) (> minmn 0))
    (setf mnthr
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 6 "SGESVD" (f2cl-lib:f2cl-// jobu jobvt) m n 0
                 0)
              (declare (ignore var-0 var-1 var-2 var-5 var-6))
              (setf m var-3)
              (setf n var-4)
              ret-val))
    (setf bdspac (f2cl-lib:int-mul 5 n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (sgeqrf m n a lda
         (f2cl-lib:array-slice dum single-float (1) ((1 1)))
         (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1 ierr)
      (declare (ignore var-2 var-4 var-5 var-6))
      (setf m var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf ierr var-7))
    (setf lwork_sgeqrf (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (sorgqr m n n a lda
         (f2cl-lib:array-slice dum single-float (1) ((1 1)))
         (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1 ierr)
      (declare (ignore var-3 var-5 var-6 var-7))
      (when var-0 (setf m var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-8 (setf ierr var-8)))
    (setf lwork_sorgqr_n
            (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (sorgqr m m n a lda
         (f2cl-lib:array-slice dum single-float (1) ((1 1)))
```

```
      (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1 ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf lwork_sorgqr_m
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (sgebrd n n a lda s
     (f2cl-lib:array-slice dum single-float (1) ((1 1)))
     (f2cl-lib:array-slice dum single-float (1) ((1 1)))
     (f2cl-lib:array-slice dum single-float (1) ((1 1)))
     (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1 ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(setf lwork_sgebrd (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (sorgbr "P" n n n a lda
     (f2cl-lib:array-slice dum single-float (1) ((1 1)))
     (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1 ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf ierr var-9)))
(setf lwork_sorgbr_p
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (sorgbr "Q" n n n a lda
     (f2cl-lib:array-slice dum single-float (1) ((1 1)))
     (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1 ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf ierr var-9)))
(setf lwork_sorgbr_q
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(cond
```

```
((>= m mnthr)
 (cond
  (wntun (setf maxwrk (f2cl-lib:int-add n lwork_sgeqrf))
   (setf maxwrk
           (max (the f2cl-lib:integer4 maxwrk)
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                       lwork_sgebrd))))
   (if (or wntvo wntvas)
       (setf maxwrk
               (max (the f2cl-lib:integer4 maxwrk)
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                           lwork_sorgbr_p)))))
   (setf maxwrk
           (max (the f2cl-lib:integer4 maxwrk)
                (the f2cl-lib:integer4 bdspac)))
   (setf minwrk
           (max (the f2cl-lib:integer4 (f2cl-lib:int-mul 4 n))
                (the f2cl-lib:integer4 bdspac))))
  ((and wntuo wntvn)
   (setf wrkbl (f2cl-lib:int-add n lwork_sgeqrf))
   (setf wrkbl
           (max (the f2cl-lib:integer4 wrkbl)
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add n lwork_sorgqr_n))))
   (setf wrkbl
           (max (the f2cl-lib:integer4 wrkbl)
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                       lwork_sgebrd))))
   (setf wrkbl
           (max (the f2cl-lib:integer4 wrkbl)
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                       lwork_sorgbr_q))))
   (setf wrkbl
           (max (the f2cl-lib:integer4 wrkbl)
                (the f2cl-lib:integer4 bdspac)))
   (setf maxwrk
           (max
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                                   (f2cl-lib:int-mul m n) n))))
   (setf minwrk
           (max
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) m))
```

```
          (the f2cl-lib:integer4 bdspac))))
((and wntuo wntvas)
 (setf wrkbl (f2cl-lib:int-add n lwork_sgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_sorgqr_n))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_sgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_sorgbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_sorgbr_p))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                                 (f2cl-lib:int-mul m n) n))))
 (setf minwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) m))
          (the f2cl-lib:integer4 bdspac))))
((and wntus wntvn)
 (setf wrkbl (f2cl-lib:int-add n lwork_sgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_sorgqr_n))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_sgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
```

```
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                       lwork_sorgbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
 (setf minwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) m))
          (the f2cl-lib:integer4 bdspac))))
((and wntus wntvo)
 (setf wrkbl (f2cl-lib:int-add n lwork_sgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_sorgqr_n))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_sgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_sorgbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_sorgbr_p))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n n) wrkbl))
 (setf minwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) m))
          (the f2cl-lib:integer4 bdspac))))
((and wntus wntvas)
 (setf wrkbl (f2cl-lib:int-add n lwork_sgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_sorgqr_n))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
```

```
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                       lwork_sgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_sorgbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_sorgbr_p))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
 (setf minwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) m))
          (the f2cl-lib:integer4 bdspac))))
((and wntua wntvn)
 (setf wrkbl (f2cl-lib:int-add n lwork_sgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_sorgqr_m))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_sgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_sorgbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
 (setf minwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) m))
          (the f2cl-lib:integer4 bdspac))))
((and wntua wntvo)
 (setf wrkbl (f2cl-lib:int-add n lwork_sgeqrf))
 (setf wrkbl
```

```
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add n lwork_sorgqr_m))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    lwork_sgebrd))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    lwork_sorgbr_q))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    lwork_sorgbr_p))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4 bdspac)))
(setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n n) wrkbl))
(setf minwrk
        (max
         (the f2cl-lib:integer4
              (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) m))
         (the f2cl-lib:integer4 bdspac))))
((and wntua wntvas)
 (setf wrkbl (f2cl-lib:int-add n lwork_sgeqrf))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add n lwork_sorgqr_m))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    lwork_sgebrd))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    lwork_sorgbr_q))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    lwork_sorgbr_p))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
```

```
                (the f2cl-lib:integer4 bdspac)))
    (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
    (setf minwrk
            (max
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) m))
             (the f2cl-lib:integer4 bdspac))))))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (sgebrd m n a lda s
        (f2cl-lib:array-slice dum single-float (1) ((1 1)))
        (f2cl-lib:array-slice dum single-float (1) ((1 1)))
        (f2cl-lib:array-slice dum single-float (1) ((1 1)))
        (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1 ierr)
    (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-10))
   (setf lwork_sgebrd
           (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
   (setf maxwrk
           (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) lwork_sgebrd))
   (cond
    ((or wntus wntuo)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
          var-9)
         (sorgbr "Q" m n n a lda
          (f2cl-lib:array-slice dum single-float (1) ((1 1)))
          (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1
          ierr)
       (declare (ignore var-0 var-4 var-6 var-7 var-8))
       (when var-1 (setf m var-1))
       (when var-2 (setf n var-2))
       (when var-3 (setf n var-3))
       (when var-5 (setf lda var-5))
       (when var-9 (setf ierr var-9)))
     (setf lwork_sorgbr_q
             (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
     (setf maxwrk
             (max (the f2cl-lib:integer4 maxwrk)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                         lwork_sorgbr_q))))))
   (cond
    (wntua
     (multiple-value-bind
```

```
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9)
            (sorgbr "Q" m m n a lda
             (f2cl-lib:array-slice dum single-float (1) ((1 1)))
             (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1
             ierr)
          (declare (ignore var-0 var-4 var-6 var-7 var-8))
          (when var-1 (setf m var-1))
          (when var-2 (setf m var-2))
          (when var-3 (setf n var-3))
          (when var-5 (setf lda var-5))
          (when var-9 (setf ierr var-9)))
        (setf lwork_sorgbr_q
                (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                            lwork_sorgbr_q))))))
      (cond
       ((not wntvn)
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                            lwork_sorgbr_p))))))
      (setf maxwrk
              (max (the f2cl-lib:integer4 maxwrk)
                   (the f2cl-lib:integer4 bdspac)))
      (setf minwrk
              (max
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) m))
               (the f2cl-lib:integer4 bdspac))))))
    ((> minmn 0)
     (setf mnthr
             (multiple-value-bind
                 (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                 (ilaenv 6 "SGESVD" (f2cl-lib:f2cl-// jobu jobvt) m n 0
                  0)
               (declare (ignore var-0 var-1 var-2 var-5 var-6))
               (setf m var-3)
               (setf n var-4)
               ret-val))
     (setf bdspac (f2cl-lib:int-mul 5 m))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (sgelqf m n a lda
          (f2cl-lib:array-slice dum single-float (1) ((1 1)))
          (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1 ierr)
```

```
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(setf lwork_sgelqf (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorglq n n m
     (f2cl-lib:array-slice dum single-float (1) ((1 1))) n
     (f2cl-lib:array-slice dum single-float (1) ((1 1)))
     (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1 ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf n var-4))
  (when var-8 (setf ierr var-8)))
(setf lwork_sorglq_n
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorglq m n m a lda
     (f2cl-lib:array-slice dum single-float (1) ((1 1)))
     (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1 ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf lwork_sorglq_m
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (sgebrd m m a lda s
     (f2cl-lib:array-slice dum single-float (1) ((1 1)))
     (f2cl-lib:array-slice dum single-float (1) ((1 1)))
     (f2cl-lib:array-slice dum single-float (1) ((1 1)))
     (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1 ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(setf lwork_sgebrd (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (sorgbr "P" m m m a n
```

```
     (f2cl-lib:array-slice dum single-float (1) ((1 1)))
     (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1 ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf n var-5))
  (when var-9 (setf ierr var-9)))
(setf lwork_sorgbr_p
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (sorgbr "Q" m m m a n
     (f2cl-lib:array-slice dum single-float (1) ((1 1)))
     (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1 ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf n var-5))
  (when var-9 (setf ierr var-9)))
(setf lwork_sorgbr_q
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(cond
 ((>= n mnthr)
  (cond
   (wntvn (setf maxwrk (f2cl-lib:int-add m lwork_sgelqf))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                        lwork_sgebrd))))
    (if (or wntuo wntuas)
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                            lwork_sorgbr_q)))))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4 bdspac)))
    (setf minwrk
            (max (the f2cl-lib:integer4 (f2cl-lib:int-mul 4 m))
                 (the f2cl-lib:integer4 bdspac))))
   ((and wntvo wntun)
    (setf wrkbl (f2cl-lib:int-add m lwork_sgelqf))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add m lwork_sorglq_m))))
```

```
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     lwork_sgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     lwork_sorgbr_p))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                 (f2cl-lib:int-mul m n) m))))
 (setf minwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
          (the f2cl-lib:integer4 bdspac))))
((and wntvo wntuas)
 (setf wrkbl (f2cl-lib:int-add m lwork_sgelqf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add m lwork_sorglq_m))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     lwork_sgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     lwork_sorgbr_p))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     lwork_sorgbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk
```

```
        (max
         (the f2cl-lib:integer4
              (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
         (the f2cl-lib:integer4
              (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                (f2cl-lib:int-mul m n) m))))
 (setf minwrk
        (max
         (the f2cl-lib:integer4
              (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
         (the f2cl-lib:integer4 bdspac))))
((and wntvs wntun)
 (setf wrkbl (f2cl-lib:int-add m lwork_sgelqf))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add m lwork_sorglq_m))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    lwork_sgebrd))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    lwork_sorgbr_p))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
 (setf minwrk
        (max
         (the f2cl-lib:integer4
              (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
         (the f2cl-lib:integer4 bdspac))))
((and wntvs wntuo)
 (setf wrkbl (f2cl-lib:int-add m lwork_sgelqf))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add m lwork_sorglq_m))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    lwork_sgebrd))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
```

```
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                      lwork_sorgbr_p))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     lwork_sorgbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m m) wrkbl))
 (setf minwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
          (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4 minwrk))))
((and wntvs wntuas)
 (setf wrkbl (f2cl-lib:int-add m lwork_sgelqf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add m lwork_sorglq_m))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     lwork_sgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     lwork_sorgbr_p))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     lwork_sorgbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
 (setf minwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
          (the f2cl-lib:integer4 bdspac))))
((and wntva wntun)
```

```
        (setf wrkbl (f2cl-lib:int-add m lwork_sgelqf))
        (setf wrkbl
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add m lwork_sorglq_n))))
        (setf wrkbl
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                            lwork_sgebrd))))
        (setf wrkbl
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                            lwork_sorgbr_p))))
        (setf wrkbl
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4 bdspac)))
        (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
        (setf minwrk
                (max
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
                 (the f2cl-lib:integer4 bdspac))))
       ((and wntva wntuo)
        (setf wrkbl (f2cl-lib:int-add m lwork_sgelqf))
        (setf wrkbl
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add m lwork_sorglq_n))))
        (setf wrkbl
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                            lwork_sgebrd))))
        (setf wrkbl
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                            lwork_sorgbr_p))))
        (setf wrkbl
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                            lwork_sorgbr_q))))
        (setf wrkbl
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4 bdspac)))
        (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m m) wrkbl))
        (setf minwrk
```

```
            (max
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
             (the f2cl-lib:integer4 bdspac))))
      ((and wntva wntuas)
       (setf wrkbl (f2cl-lib:int-add m lwork_sgelqf))
       (setf wrkbl
               (max (the f2cl-lib:integer4 wrkbl)
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add m lwork_sorglq_n))))
       (setf wrkbl
               (max (the f2cl-lib:integer4 wrkbl)
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                           lwork_sgebrd))))
       (setf wrkbl
               (max (the f2cl-lib:integer4 wrkbl)
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                           lwork_sorgbr_p))))
       (setf wrkbl
               (max (the f2cl-lib:integer4 wrkbl)
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                           lwork_sorgbr_q))))
       (setf wrkbl
               (max (the f2cl-lib:integer4 wrkbl)
                    (the f2cl-lib:integer4 bdspac)))
       (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
       (setf minwrk
               (max
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
                (the f2cl-lib:integer4 bdspac))))))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (sgebrd m n a lda s
          (f2cl-lib:array-slice dum single-float (1) ((1 1)))
          (f2cl-lib:array-slice dum single-float (1) ((1 1)))
          (f2cl-lib:array-slice dum single-float (1) ((1 1)))
          (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1 ierr)
       (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
       (setf m var-0)
       (setf n var-1)
       (setf lda var-3)
       (setf ierr var-10))
     (setf lwork_sgebrd
             (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
```

```
(setf maxwrk
        (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) lwork_sgebrd))
(cond
 ((or wntvs wntvo)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (sorgbr "P" m n m a n
       (f2cl-lib:array-slice dum single-float (1) ((1 1)))
       (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1
       ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf n var-5))
    (when var-9 (setf ierr var-9)))
  (setf lwork_sorgbr_p
          (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                      lwork_sorgbr_p))))))
(cond
 (wntva
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (sorgbr "P" n n m a n
       (f2cl-lib:array-slice dum single-float (1) ((1 1)))
       (f2cl-lib:array-slice dum single-float (1) ((1 1))) -1
       ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf n var-5))
    (when var-9 (setf ierr var-9)))
  (setf lwork_sorgbr_p
          (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                      lwork_sorgbr_p))))))
(cond
 ((not wntun)
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
```

```
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                             lwork_sorgbr_q))))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4 bdspac)))
        (setf minwrk
                (max
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
                 (the f2cl-lib:integer4 bdspac)))))))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4 minwrk)))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce (the f2cl-lib:integer4 maxwrk) 'single-float))
    (cond ((and (< lwork minwrk) (not lquery)) (setf info -13)))))
  (cond
   ((/= info 0) (xerbla "SGESVD" (f2cl-lib:int-sub info)) (go end_label))
   (lquery (go end_label)))
  (cond ((or (= m 0) (= n 0)) (go end_label)))
  (setf eps (slamch "P"))
  (setf smlnum (/ (f2cl-lib:fsqrt (slamch "S")) eps))
  (setf bignum (/ one smlnum))
  (setf anrm
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
              (slange "M" m n a lda dum)
            (declare (ignore var-0 var-3 var-5))
            (when var-1 (setf m var-1))
            (when var-2 (setf n var-2))
            (when var-4 (setf lda var-4))
            ret-val))
  (setf iscl 0)
  (cond
   ((and (> anrm zero) (< anrm smlnum)) (setf iscl 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl "G" 0 0 anrm smlnum m n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf smlnum var-4))
      (when var-5 (setf m var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
   ((> anrm bignum) (setf iscl 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl "G" 0 0 anrm bignum m n a lda ierr)
```

```
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf ierr var-9)))))
(cond
 ((>= m n)
  (cond
   ((>= m mnthr)
    (cond
     (wntun (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (sgeqrf m n a lda
           (f2cl-lib:array-slice work-%data% single-float (itau)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% single-float (iwork)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
        (declare (ignore var-2 var-4 var-5 var-6))
        (setf m var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf ierr var-7))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
           zero zero
           (f2cl-lib:array-slice a-%data% single-float (2 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-5))
        (when var-3 (setf zero var-3))
        (when var-4 (setf zero var-4))
        (when var-6 (setf lda var-6)))
      (setf ie 1) (setf itauq (f2cl-lib:int-add ie n))
      (setf itaup (f2cl-lib:int-add itauq n))
      (setf iwork (f2cl-lib:int-add itaup n))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (sgebrd n n a lda s
           (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                                 work-%offset%)
           (f2cl-lib:array-slice work-%data% single-float (itauq)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% single-float (itaup)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% single-float (iwork)
```

```
                          ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(setf ncvt 0)
(cond
 ((or wntvo wntvas)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (sorgbr "P" n n n a lda
       (f2cl-lib:array-slice work-%data% single-float (itaup)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))
  (setf ncvt n)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" n ncvt 0 0 s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     a lda dum 1 dum 1
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
    var-11 var-12 var-13))
  (setf n var-1)
  (setf ncvt var-2)
  (setf lda var-8)
  (setf info var-14))
(if wntvas
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slacpy "F" n n a lda vt ldvt)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
```

```
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldvt var-6)))))
((and wntuo wntvn)
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-mul 4 n))
                          (the f2cl-lib:integer4 bdspac))))
   (setf ir 1)
   (cond
    ((>= lwork
         (f2cl-lib:int-add
          (max (the f2cl-lib:integer4 wrkbl)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul lda n)
                                      n)))
          (f2cl-lib:int-mul lda n)))
     (setf ldwrku lda) (setf ldwrkr lda))
    ((>= lwork
         (f2cl-lib:int-add
          (max (the f2cl-lib:integer4 wrkbl)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul lda n)
                                      n)))
          (f2cl-lib:int-mul n n)))
     (setf ldwrku lda) (setf ldwrkr n))
    (t
     (setf ldwrku
             (the f2cl-lib:integer4
                  (truncate (- lwork (* n n) n) n)))
     (setf ldwrkr n)))
   (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
   (setf iwork (f2cl-lib:int-add itau n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (sgeqrf m n a lda
        (f2cl-lib:array-slice work-%data% single-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-7))
   (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice work-%data% single-float ((+ ir 1))
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorgqr m n n a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd n n
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
```

```
                              ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "Q" n n n
     (f2cl-lib:array-slice work-%data% single-float (ir)
                              ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                              ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" n 0 n 0 s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                              ((1 *)) work-%offset%)
     dum 1
     (f2cl-lib:array-slice work-%data% single-float (ir)
                              ((1 *)) work-%offset%)
     ldwrkr dum 1
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
    var-11 var-12 var-13))
  (setf n var-1)
  (setf n var-3)
  (setf ldwrkr var-10)
  (setf info var-14))
(setf iu (f2cl-lib:int-add ie n))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i ldwrku))
```

```
                ((> i m) nil)
      (tagbody
        (setf chunk
                (min
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
                 (the f2cl-lib:integer4 ldwrku)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12)
            (sgemm "N" "N" chunk n n one
             (f2cl-lib:array-slice a-%data% single-float (i 1)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice work-%data% single-float (ir)
                                   ((1 *)) work-%offset%)
             ldwrkr zero
             (f2cl-lib:array-slice work-%data% single-float (iu)
                                   ((1 *)) work-%offset%)
             ldwrku)
          (declare (ignore var-0 var-1 var-6 var-8 var-11))
          (when var-2 (setf chunk var-2))
          (when var-3 (setf n var-3))
          (when var-4 (setf n var-4))
          (when var-5 (setf one var-5))
          (when var-7 (setf lda var-7))
          (when var-9 (setf ldwrkr var-9))
          (when var-10 (setf zero var-10))
          (when var-12 (setf ldwrku var-12)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (slacpy "F" chunk n
             (f2cl-lib:array-slice work-%data% single-float (iu)
                                   ((1 *)) work-%offset%)
             ldwrku
             (f2cl-lib:array-slice a-%data% single-float (i 1)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf chunk var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf ldwrku var-4))
          (when var-6 (setf lda var-6)))
      label10)))
  (t (setf ie 1) (setf itauq (f2cl-lib:int-add ie n))
   (setf itaup (f2cl-lib:int-add itauq n))
   (setf iwork (f2cl-lib:int-add itaup n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
```

```
            (sgebrd m n a lda s
             (f2cl-lib:array-slice work-%data% single-float (ie)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:array-slice work-%data% single-float (itauq)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:array-slice work-%data% single-float (itaup)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:array-slice work-%data% single-float (iwork)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
          (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
          (setf m var-0)
          (setf n var-1)
          (setf lda var-3)
          (setf ierr var-10))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9)
            (sorgbr "Q" m n n a lda
             (f2cl-lib:array-slice work-%data% single-float (itauq)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:array-slice work-%data% single-float (iwork)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
          (declare (ignore var-0 var-4 var-6 var-7 var-8))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-3 (setf n var-3))
          (when var-5 (setf lda var-5))
          (when var-9 (setf ierr var-9)))
        (setf iwork (f2cl-lib:int-add ie n))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12 var-13 var-14)
            (sbdsqr "U" n 0 m 0 s
             (f2cl-lib:array-slice work-%data% single-float (ie)
                                   ((1 *)) work-%offset%)
             dum 1 a lda dum 1
             (f2cl-lib:array-slice work-%data% single-float (iwork)
                                   ((1 *)) work-%offset%)
             info)
          (declare
           (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
            var-11 var-12 var-13))
          (setf n var-1)
          (setf m var-3)
          (setf lda var-10)
          (setf info var-14)))))
      ((and wntuo wntvas)
       (cond
```

```
((>= lwork
     (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                       (max
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-mul 4 n))
                        (the f2cl-lib:integer4 bdspac))))
 (setf ir 1)
 (cond
  ((>= lwork
       (f2cl-lib:int-add
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul lda n)
                                    n)))
        (f2cl-lib:int-mul lda n)))
   (setf ldwrku lda) (setf ldwrkr lda))
  ((>= lwork
       (f2cl-lib:int-add
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul lda n)
                                    n)))
        (f2cl-lib:int-mul n n)))
   (setf ldwrku lda) (setf ldwrkr n))
  (t
   (setf ldwrku
           (the f2cl-lib:integer4
                (truncate (- lwork (* n n) n) n)))
   (setf ldwrkr n)))
 (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
 (setf iwork (f2cl-lib:int-add itau n))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (sgeqrf m n a lda
      (f2cl-lib:array-slice work-%data% single-float (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-7))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slacpy "U" n n a lda vt ldvt)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
```

```
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(if (> n 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "L" (f2cl-lib:int-sub n 1)
         (f2cl-lib:int-sub n 1) zero zero
         (f2cl-lib:array-slice vt-%data% single-float (2 1)
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldvt var-6))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorgqr m n n a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd n n vt ldvt s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldvt var-3)
  (setf ierr var-10))
(multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" n n vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldvt var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "Q" n n n
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "P" n n n vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" n n n 0 s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
```

```
      vt ldvt
      (f2cl-lib:array-slice work-%data% single-float (ir)
                            ((1 *)) work-%offset%)
      ldwrkr dum 1
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
      info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf n var-1)
  (setf n var-2)
  (setf n var-3)
  (setf ldvt var-8)
  (setf ldwrkr var-10)
  (setf info var-14))
(setf iu (f2cl-lib:int-add ie n))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i ldwrku))
              ((> i m) nil)
  (tagbody
    (setf chunk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
             (the f2cl-lib:integer4 ldwrku)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (sgemm "N" "N" chunk n n one
         (f2cl-lib:array-slice a-%data% single-float (i 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice work-%data% single-float (ir)
                               ((1 *)) work-%offset%)
         ldwrkr zero
         (f2cl-lib:array-slice work-%data% single-float (iu)
                               ((1 *)) work-%offset%)
         ldwrku)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf chunk var-2))
      (when var-3 (setf n var-3))
      (when var-4 (setf n var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf lda var-7))
      (when var-9 (setf ldwrkr var-9))
      (when var-10 (setf zero var-10))
      (when var-12 (setf ldwrku var-12)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slacpy "F" chunk n
```

```
            (f2cl-lib:array-slice work-%data% single-float (iu)
                                  ((1 *)) work-%offset%)
            ldwrku
            (f2cl-lib:array-slice a-%data% single-float (i 1)
                                  ((1 lda) (1 *)) a-%offset%)
            lda)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf chunk var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf ldwrku var-4))
        (when var-6 (setf lda var-6)))
    label20)))
(t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (sgeqrf m n a lda
      (f2cl-lib:array-slice work-%data% single-float (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-7))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slacpy "U" n n a lda vt ldvt)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldvt var-6)))
 (if (> n 1)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (slaset "L" (f2cl-lib:int-sub n 1)
          (f2cl-lib:int-sub n 1) zero zero
          (f2cl-lib:array-slice vt-%data% single-float (2 1)
                                ((1 ldvt) (1 *)) vt-%offset%)
          ldvt)
       (declare (ignore var-0 var-1 var-2 var-5))
       (when var-3 (setf zero var-3))
       (when var-4 (setf zero var-4))
       (when var-6 (setf ldvt var-6))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (sorgqr m n n a lda
      (f2cl-lib:array-slice work-%data% single-float (itau)
```

```
                              ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd n n vt ldvt s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                              ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                              ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                              ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldvt var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (sormbr "Q" "R" "N" m n n vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                              ((1 *)) work-%offset%)
     a lda
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldvt var-7))
  (when var-10 (setf lda var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
```

```
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9)
          (sorgbr "P" n n n vt ldvt
           (f2cl-lib:array-slice work-%data% single-float (itaup)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% single-float (iwork)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
        (declare (ignore var-0 var-4 var-6 var-7 var-8))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf n var-3))
        (when var-5 (setf ldvt var-5))
        (when var-9 (setf ierr var-9)))
      (setf iwork (f2cl-lib:int-add ie n))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12 var-13 var-14)
          (sbdsqr "U" n n m 0 s
           (f2cl-lib:array-slice work-%data% single-float (ie)
                                 ((1 *)) work-%offset%)
           vt ldvt a lda dum 1
           (f2cl-lib:array-slice work-%data% single-float (iwork)
                                 ((1 *)) work-%offset%)
           info)
        (declare
         (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
          var-13))
        (setf n var-1)
        (setf n var-2)
        (setf m var-3)
        (setf ldvt var-8)
        (setf lda var-10)
        (setf info var-14)))))
  (wntus
   (cond
    (wntvn
     (cond
      ((>= lwork
           (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                             (max
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-mul 4 n))
                              (the f2cl-lib:integer4 bdspac))))
      (setf ir 1)
      (cond
       ((>= lwork
            (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda n)))
        (setf ldwrkr lda))
       (t (setf ldwrkr n)))
```

```
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
(setf iwork (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (sgeqrf m n a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ ir 1)) ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorgqr m n n a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
```

```
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd n n
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "Q" n n n
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
```

```
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13 var-14)
      (sbdsqr "U" n 0 n 0 s
       (f2cl-lib:array-slice work-%data% single-float (ie)
                             ((1 *)) work-%offset%)
       dum 1
       (f2cl-lib:array-slice work-%data% single-float (ir)
                             ((1 *)) work-%offset%)
       ldwrkr dum 1
       (f2cl-lib:array-slice work-%data% single-float (iwork)
                             ((1 *)) work-%offset%)
       info)
    (declare
     (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
      var-11 var-12 var-13))
    (setf n var-1)
    (setf n var-3)
    (setf ldwrkr var-10)
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (sgemm "N" "N" m n n one a lda
       (f2cl-lib:array-slice work-%data% single-float (ir)
                             ((1 *)) work-%offset%)
       ldwrkr zero u ldu)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldwrkr var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldu var-12))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (sgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% single-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorgqr m n n u ldu
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice a-%data% single-float (2 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd n n a lda s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
```

```
            ierr)
          (declare
           (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
          (setf n var-0)
          (setf n var-1)
          (setf lda var-3)
          (setf ierr var-10))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12 var-13)
            (sormbr "Q" "R" "N" m n n a lda
             (f2cl-lib:array-slice work-%data% single-float (itauq)
                                   ((1 *)) work-%offset%)
             u ldu
             (f2cl-lib:array-slice work-%data% single-float (iwork)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
             ierr)
          (declare
           (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
            var-12))
          (when var-3 (setf m var-3))
          (when var-4 (setf n var-4))
          (when var-5 (setf n var-5))
          (when var-7 (setf lda var-7))
          (when var-10 (setf ldu var-10))
          (when var-13 (setf ierr var-13)))
        (setf iwork (f2cl-lib:int-add ie n))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12 var-13 var-14)
            (sbdsqr "U" n 0 m 0 s
             (f2cl-lib:array-slice work-%data% single-float (ie)
                                   ((1 *)) work-%offset%)
             dum 1 u ldu dum 1
             (f2cl-lib:array-slice work-%data% single-float (iwork)
                                   ((1 *)) work-%offset%)
             info)
          (declare
           (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
            var-11 var-12 var-13))
          (setf n var-1)
          (setf m var-3)
          (setf ldu var-10)
          (setf info var-14)))))
    (wntvo
     (cond
      ((>= lwork
           (f2cl-lib:int-add (f2cl-lib:int-mul 2 n n)
                             (max
```

```
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-mul 4 n))
                        (the f2cl-lib:integer4 bdspac))))
(setf iu 1)
(cond
 ((>= lwork
      (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 lda n)))
  (setf ldwrku lda)
  (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
  (setf ldwrkr lda))
 ((>= lwork
      (f2cl-lib:int-add wrkbl
                         (f2cl-lib:int-mul
                          (f2cl-lib:int-add lda n) n)))
  (setf ldwrku lda)
  (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
  (setf ldwrkr n))
 (t (setf ldwrku n)
  (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
  (setf ldwrkr n)))
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
(setf iwork (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (sgeqrf m n a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
```

```
      zero zero
      (f2cl-lib:array-slice work-%data% single-float
                            ((+ iu 1)) ((1 *)) work-%offset%)
      ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorgqr m n n a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd n n
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "U" n n
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "Q" n n n
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "P" n n n
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
```

```
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" n n n 0 s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku dum 1
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf n var-1)
  (setf n var-2)
  (setf n var-3)
  (setf ldwrkr var-8)
  (setf ldwrku var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (sgemm "N" "N" m n n one a lda
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku zero u ldu)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldwrku var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf ldu var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "F" n n
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr a lda)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
```

```
    (when var-2 (setf n var-2))
    (when var-4 (setf ldwrkr var-4))
    (when var-6 (setf lda var-6))))
  (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (sgeqrf m n a lda
        (f2cl-lib:array-slice work-%data% single-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (sorgqr m n n u ldu
        (f2cl-lib:array-slice work-%data% single-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf m var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldu var-4))
    (when var-8 (setf ierr var-8)))
   (setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
   (setf itaup (f2cl-lib:int-add itauq n))
   (setf iwork (f2cl-lib:int-add itaup n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
        zero zero
        (f2cl-lib:array-slice a-%data% single-float (2 1)
                              ((1 lda) (1 *)) a-%offset%)
```

```
      lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd n n a lda s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (sormbr "Q" "R" "N" m n n a lda
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "P" n n n a lda
     (f2cl-lib:array-slice work-%data% single-float (itaup)
```

```
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% single-float (iwork)
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
          ierr)
      (declare (ignore var-0 var-4 var-6 var-7 var-8))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf n var-3))
      (when var-5 (setf lda var-5))
      (when var-9 (setf ierr var-9)))
    (setf iwork (f2cl-lib:int-add ie n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14)
        (sbdsqr "U" n n m 0 s
         (f2cl-lib:array-slice work-%data% single-float (ie)
                               ((1 *)) work-%offset%)
         a lda u ldu dum 1
         (f2cl-lib:array-slice work-%data% single-float (iwork)
                               ((1 *)) work-%offset%)
         info)
      (declare
       (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
        var-13))
      (setf n var-1)
      (setf n var-2)
      (setf m var-3)
      (setf lda var-8)
      (setf ldu var-10)
      (setf info var-14)))))
  (wntvas
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-mul 4 n))
                            (the f2cl-lib:integer4 bdspac))))
    (setf iu 1)
    (cond
     ((>= lwork
          (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda n)))
      (setf ldwrku lda))
     (t (setf ldwrku n)))
    (setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
    (setf iwork (f2cl-lib:int-add itau n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (sgeqrf m n a lda
```

```
      (f2cl-lib:array-slice work-%data% single-float (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ iu 1)) ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorgqr m n n a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
```

```
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd n n
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "U" n n
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "Q" n n n
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
```

```
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "P" n n n vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" n n n 0 s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku dum 1
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf n var-1)
  (setf n var-2)
  (setf n var-3)
  (setf ldvt var-8)
  (setf ldwrku var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (sgemm "N" "N" m n n one a lda
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku zero u ldu)
```

```
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldwrku var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldu var-12))))
  (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (sgeqrf m n a lda
        (f2cl-lib:array-slice work-%data% single-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (sorgqr m n n u ldu
        (f2cl-lib:array-slice work-%data% single-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf m var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldu var-4))
    (when var-8 (setf ierr var-8)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slacpy "U" n n a lda vt ldvt)
```

```
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(if (> n 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "L" (f2cl-lib:int-sub n 1)
         (f2cl-lib:int-sub n 1) zero zero
         (f2cl-lib:array-slice vt-%data% single-float (2 1)
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldvt var-6))))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd n n vt ldvt s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldvt var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (sormbr "Q" "R" "N" m n n vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
```

```
            ierr)
          (declare
           (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
            var-12))
          (when var-3 (setf m var-3))
          (when var-4 (setf n var-4))
          (when var-5 (setf n var-5))
          (when var-7 (setf ldvt var-7))
          (when var-10 (setf ldu var-10))
          (when var-13 (setf ierr var-13)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9)
            (sorgbr "P" n n n vt ldvt
             (f2cl-lib:array-slice work-%data% single-float (itaup)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:array-slice work-%data% single-float (iwork)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
             ierr)
          (declare (ignore var-0 var-4 var-6 var-7 var-8))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-3 (setf n var-3))
          (when var-5 (setf ldvt var-5))
          (when var-9 (setf ierr var-9)))
        (setf iwork (f2cl-lib:int-add ie n))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12 var-13 var-14)
            (sbdsqr "U" n n m 0 s
             (f2cl-lib:array-slice work-%data% single-float (ie)
                                   ((1 *)) work-%offset%)
             vt ldvt u ldu dum 1
             (f2cl-lib:array-slice work-%data% single-float (iwork)
                                   ((1 *)) work-%offset%)
             info)
          (declare
           (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
            var-13))
          (setf n var-1)
          (setf n var-2)
          (setf m var-3)
          (setf ldvt var-8)
          (setf ldu var-10)
          (setf info var-14)))))))
  (wntua
   (cond
    (wntvn
     (cond
```

```
((>= lwork
     (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                       (max
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add n m))
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-mul 4 n))
                        (the f2cl-lib:integer4 bdspac))))
 (setf ir 1)
 (cond
  ((>= lwork
       (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda n)))
   (setf ldwrkr lda))
  (t (setf ldwrkr n)))
 (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
 (setf iwork (f2cl-lib:int-add itau n))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (sgeqrf m n a lda
      (f2cl-lib:array-slice work-%data% single-float (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
   (declare (ignore var-2 var-4 var-5 var-6))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-7))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slacpy "L" m n a lda u ldu)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf m var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldu var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slacpy "U" n n a lda
      (f2cl-lib:array-slice work-%data% single-float (ir)
                            ((1 *)) work-%offset%)
      ldwrkr)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldwrkr var-6)))
 (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ ir 1)) ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorgqr m m n u ldu
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd n n
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrkr var-3)
```

```
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "Q" n n n
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" n 0 n 0 s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     dum 1
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr dum 1
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
    var-11 var-12 var-13))
  (setf n var-1)
  (setf n var-3)
  (setf ldwrkr var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (sgemm "N" "N" m n n one u ldu
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr zero a lda)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
```

```
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldu var-7))
    (when var-9 (setf ldwrkr var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf lda var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "F" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (sgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% single-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (sorgqr m m n u ldu
       (f2cl-lib:array-slice work-%data% single-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf m var-0))
    (when var-1 (setf m var-1))
```

```
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice a-%data% single-float (2 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd n n a lda s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (sormbr "Q" "R" "N" m n n a lda
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
```

```
        var-12))
      (when var-3 (setf m var-3))
      (when var-4 (setf n var-4))
      (when var-5 (setf n var-5))
      (when var-7 (setf lda var-7))
      (when var-10 (setf ldu var-10))
      (when var-13 (setf ierr var-13)))
    (setf iwork (f2cl-lib:int-add ie n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14)
        (sbdsqr "U" n 0 m 0 s
         (f2cl-lib:array-slice work-%data% single-float (ie)
                               ((1 *)) work-%offset%)
         dum 1 u ldu dum 1
         (f2cl-lib:array-slice work-%data% single-float (iwork)
                               ((1 *)) work-%offset%)
         info)
      (declare
       (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
        var-11 var-12 var-13))
      (setf n var-1)
      (setf m var-3)
      (setf ldu var-10)
      (setf info var-14)))))
(wntvo
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul 2 n n)
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add n m))
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-mul 4 n))
                          (the f2cl-lib:integer4 bdspac))))
   (setf iu 1)
   (cond
    ((>= lwork
         (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 lda n)))
     (setf ldwrku lda)
     (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
     (setf ldwrkr lda))
    ((>= lwork
         (f2cl-lib:int-add wrkbl
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-add lda n) n)))
     (setf ldwrku lda)
     (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
     (setf ldwrkr n))
    (t (setf ldwrku n)
```

```
  (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
  (setf ldwrkr n)))
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
(setf iwork (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (sgeqrf m n a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorgqr m m n u ldu
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
```

```
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ iu 1)) ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrku var-6)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd n n
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "U" n n
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
```

```
  (when var-2 (setf n var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "Q" n n n
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "P" n n n
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" n n n 0 s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
```

```
      ldwrkr
      (f2cl-lib:array-slice work-%data% single-float (iu)
                            ((1 *)) work-%offset%)
      ldwrku dum 1
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
      info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf n var-1)
  (setf n var-2)
  (setf n var-3)
  (setf ldwrkr var-8)
  (setf ldwrku var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (sgemm "N" "N" m n n one u ldu
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku zero a lda)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldu var-7))
  (when var-9 (setf ldwrku var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf lda var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "F" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "F" n n
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr a lda)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldwrkr var-4))
```

```
    (when var-6 (setf lda var-6))))
  (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (sgeqrf m n a lda
        (f2cl-lib:array-slice work-%data% single-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
     (declare (ignore var-2 var-4 var-5 var-6))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-7))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slacpy "L" m n a lda u ldu)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf m var-1))
     (when var-2 (setf n var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf ldu var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (sorgqr m m n u ldu
        (f2cl-lib:array-slice work-%data% single-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
     (declare (ignore var-3 var-5 var-6 var-7))
     (when var-0 (setf m var-0))
     (when var-1 (setf m var-1))
     (when var-2 (setf n var-2))
     (when var-4 (setf ldu var-4))
     (when var-8 (setf ierr var-8)))
   (setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
   (setf itaup (f2cl-lib:int-add itauq n))
   (setf iwork (f2cl-lib:int-add itaup n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
        zero zero
        (f2cl-lib:array-slice a-%data% single-float (2 1)
                              ((1 lda) (1 *)) a-%offset%)
        lda)
     (declare (ignore var-0 var-1 var-2 var-5))
```

```
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd n n a lda s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (sormbr "Q" "R" "N" m n n a lda
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "P" n n n a lda
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
```

```
                                ((1 *)) work-%offset%)
                 (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
                 ierr)
            (declare (ignore var-0 var-4 var-6 var-7 var-8))
            (when var-1 (setf n var-1))
            (when var-2 (setf n var-2))
            (when var-3 (setf n var-3))
            (when var-5 (setf lda var-5))
            (when var-9 (setf ierr var-9)))
          (setf iwork (f2cl-lib:int-add ie n))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10 var-11 var-12 var-13 var-14)
              (sbdsqr "U" n n m 0 s
               (f2cl-lib:array-slice work-%data% single-float (ie)
                                ((1 *)) work-%offset%)
               a lda u ldu dum 1
               (f2cl-lib:array-slice work-%data% single-float (iwork)
                                ((1 *)) work-%offset%)
               info)
            (declare
             (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
              var-13))
            (setf n var-1)
            (setf n var-2)
            (setf m var-3)
            (setf lda var-8)
            (setf ldu var-10)
            (setf info var-14)))))
        (wntvas
         (cond
          ((>= lwork
               (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                                 (max
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-add n m))
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-mul 4 n))
                                  (the f2cl-lib:integer4 bdspac))))
          (setf iu 1)
          (cond
           ((>= lwork
                (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda n)))
            (setf ldwrku lda))
           (t (setf ldwrku n)))
          (setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
          (setf iwork (f2cl-lib:int-add itau n))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
              (sgeqrf m n a lda
```

```
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorgqr m m n u ldu
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice work-%data% single-float
```

```
                              ((+ iu 1)) ((1 *)) work-%offset%)
      ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrku var-6)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd n n
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "U" n n
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "Q" n n n
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
```

```
      (f2cl-lib:array-slice work-%data% single-float (itauq)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "P" n n n vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                            ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" n n n 0 s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                            ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (iu)
                            ((1 *)) work-%offset%)
     ldwrku dum 1
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf n var-1)
  (setf n var-2)
  (setf n var-3)
  (setf ldvt var-8)
  (setf ldwrku var-10)
```

```
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (sgemm "N" "N" m n n one u ldu
       (f2cl-lib:array-slice work-%data% single-float (iu)
                             ((1 *)) work-%offset%)
       ldwrku zero a lda)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldu var-7))
    (when var-9 (setf ldwrku var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf lda var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "F" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (sgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% single-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
```

```
    (sorgqr m m n u ldu
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "U" n n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(if (> n 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "L" (f2cl-lib:int-sub n 1)
         (f2cl-lib:int-sub n 1) zero zero
         (f2cl-lib:array-slice vt-%data% single-float (2 1)
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldvt var-6))))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd n n vt ldvt s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
```

```
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldvt var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (sormbr "Q" "R" "N" m n n vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldvt var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "P" n n n vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" n n m 0 s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     vt ldvt u ldu dum 1
```

```
                (f2cl-lib:array-slice work-%data% single-float (iwork)
                                      ((1 *)) work-%offset%)
                info)
            (declare
             (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
              var-13))
            (setf n var-1)
            (setf n var-2)
            (setf m var-3)
            (setf ldvt var-8)
            (setf ldu var-10)
            (setf info var-14)))))))))
  (t (setf ie 1) (setf itauq (f2cl-lib:int-add ie n))
   (setf itaup (f2cl-lib:int-add itauq n))
   (setf iwork (f2cl-lib:int-add itaup n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (sgebrd m n a lda s
        (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
                              work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-10))
   (cond
    (wntuas
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (slacpy "L" m n a lda u ldu)
       (declare (ignore var-0 var-3 var-5))
       (when var-1 (setf m var-1))
       (when var-2 (setf n var-2))
       (when var-4 (setf lda var-4))
       (when var-6 (setf ldu var-6)))
     (if wntus
         (setf ncu n))
     (if wntua
         (setf ncu m))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (sorgbr "Q" m ncu n u ldu
          (f2cl-lib:array-slice work-%data% single-float (itauq)
```

```
                                ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (iwork)
                                ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf ncu var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf ldu var-5))
    (when var-9 (setf ierr var-9)))))
(cond
 (wntvas
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "U" n n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (sorgbr "P" n n n vt ldvt
       (f2cl-lib:array-slice work-%data% single-float (itaup)
                                ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwork)
                                ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf ldvt var-5))
    (when var-9 (setf ierr var-9)))))
(cond
 (wntuo
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (sorgbr "Q" m n n a lda
       (f2cl-lib:array-slice work-%data% single-float (itauq)
                                ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwork)
                                ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))))
(cond
```

```
 (wntvo
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (sorgbr "P" n n n a lda
       (f2cl-lib:array-slice work-%data% single-float (itaup)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))))
(setf iwork (f2cl-lib:int-add ie n))
(if (or wntuas wntuo)
    (setf nru m))
(if wntun
    (setf nru 0))
(if (or wntvas wntvo)
    (setf ncvt n))
(if wntvn
    (setf ncvt 0))
(cond
 ((and (not wntuo) (not wntvo))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (sbdsqr "U" n ncvt nru 0 s
       (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                             work-%offset%)
       vt ldvt u ldu dum 1
       (f2cl-lib:array-slice work-%data% single-float (iwork)
                             ((1 *)) work-%offset%)
       info)
    (declare
     (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
      var-13))
    (setf n var-1)
    (setf ncvt var-2)
    (setf nru var-3)
    (setf ldvt var-8)
    (setf ldu var-10)
    (setf info var-14)))
 ((and (not wntuo) wntvo)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (sbdsqr "U" n ncvt nru 0 s
```

```
            (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                                  work-%offset%)
            a lda u ldu dum 1
            (f2cl-lib:array-slice work-%data% single-float (iwork)
                                  ((1 *)) work-%offset%)
            info)
          (declare
           (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
            var-13))
          (setf n var-1)
          (setf ncvt var-2)
          (setf nru var-3)
          (setf lda var-8)
          (setf ldu var-10)
          (setf info var-14)))
        (t
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
              var-10 var-11 var-12 var-13 var-14)
             (sbdsqr "U" n ncvt nru 0 s
              (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                                    work-%offset%)
              vt ldvt a lda dum 1
              (f2cl-lib:array-slice work-%data% single-float (iwork)
                                    ((1 *)) work-%offset%)
              info)
          (declare
           (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
            var-13))
          (setf n var-1)
          (setf ncvt var-2)
          (setf nru var-3)
          (setf ldvt var-8)
          (setf lda var-10)
          (setf info var-14)))))))
    (t
     (cond
      ((>= n mnthr)
       (cond
        (wntvn (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
             (sgelqf m n a lda
              (f2cl-lib:array-slice work-%data% single-float (itau)
                                    ((1 *)) work-%offset%)
              (f2cl-lib:array-slice work-%data% single-float (iwork)
                                    ((1 *)) work-%offset%)
              (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
          (declare (ignore var-2 var-4 var-5 var-6))
          (setf m var-0)
```

```
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice a-%data% single-float (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(setf ie 1) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (sgebrd m m a lda s
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(cond
 ((or wntuo wntuas)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (sorgbr "Q" m m m a lda
       (f2cl-lib:array-slice work-%data% single-float (itauq)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf lda var-5))
```

```
        (when var-9 (setf ierr var-9)))))
    (setf iwork (f2cl-lib:int-add ie m)) (setf nru 0)
    (if (or wntuo wntuas)
        (setf nru m))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14)
        (sbdsqr "U" m 0 nru 0 s
         (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                               work-%offset%)
         dum 1 a lda dum 1
         (f2cl-lib:array-slice work-%data% single-float (iwork)
                               ((1 *)) work-%offset%)
         info)
      (declare
       (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9 var-11
        var-12 var-13))
      (setf m var-1)
      (setf nru var-3)
      (setf lda var-10)
      (setf info var-14))
    (if wntuas
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (slacpy "F" m m a lda u ldu)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf m var-2))
          (when var-4 (setf lda var-4))
          (when var-6 (setf ldu var-6)))))
  ((and wntvo wntun)
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-mul 4 m))
                            (the f2cl-lib:integer4 bdspac))))
      (setf ir 1)
      (cond
       ((>= lwork
            (f2cl-lib:int-add
             (max (the f2cl-lib:integer4 wrkbl)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add (f2cl-lib:int-mul lda n)
                                         m)))
             (f2cl-lib:int-mul lda m)))
        (setf ldwrku lda) (setf chunk n) (setf ldwrkr lda))
       ((>= lwork
            (f2cl-lib:int-add
```

```
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul lda n)
                                    m)))
        (f2cl-lib:int-mul m m)))
  (setf ldwrku lda) (setf chunk n) (setf ldwrkr m))
 (t (setf ldwrku m)
  (setf chunk
          (the f2cl-lib:integer4
               (truncate (- lwork (* m m) m) m)))
  (setf ldwrkr m)))
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
(setf iwork (f2cl-lib:int-add itau m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (sgelqf m n a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ ir ldwrkr)) ((1 *))
                           work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorglq m n m a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd m m
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "P" m m m
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
```

```
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" m m 0 0 s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr dum 1 dum 1
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
    var-11 var-12 var-13))
  (setf m var-1)
  (setf m var-2)
  (setf ldwrkr var-8)
  (setf info var-14))
(setf iu (f2cl-lib:int-add ie m))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
              ((> i n) nil)
  (tagbody
    (setf blk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
             (the f2cl-lib:integer4 chunk)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (sgemm "N" "N" m blk m one
         (f2cl-lib:array-slice work-%data% single-float (ir)
                               ((1 *)) work-%offset%)
         ldwrkr
         (f2cl-lib:array-slice a-%data% single-float (1 i)
                               ((1 lda) (1 *)) a-%offset%)
         lda zero
         (f2cl-lib:array-slice work-%data% single-float (iu)
                               ((1 *)) work-%offset%)
         ldwrku)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf blk var-3))
```

```
          (when var-4 (setf m var-4))
          (when var-5 (setf one var-5))
          (when var-7 (setf ldwrkr var-7))
          (when var-9 (setf lda var-9))
          (when var-10 (setf zero var-10))
          (when var-12 (setf ldwrku var-12)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (slacpy "F" m blk
             (f2cl-lib:array-slice work-%data% single-float (iu)
                                   ((1 *)) work-%offset%)
             ldwrku
             (f2cl-lib:array-slice a-%data% single-float (1 i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf blk var-2))
          (when var-4 (setf ldwrku var-4))
          (when var-6 (setf lda var-6)))
      label30)))
  (t (setf ie 1) (setf itauq (f2cl-lib:int-add ie m))
   (setf itaup (f2cl-lib:int-add itauq m))
   (setf iwork (f2cl-lib:int-add itaup m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (sgebrd m n a lda s
        (f2cl-lib:array-slice work-%data% single-float (ie)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (itauq)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (itaup)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-10))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9)
       (sorgbr "P" m n m a lda
        (f2cl-lib:array-slice work-%data% single-float (itaup)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
```

```
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
        (declare (ignore var-0 var-4 var-6 var-7 var-8))
        (when var-1 (setf m var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf m var-3))
        (when var-5 (setf lda var-5))
        (when var-9 (setf ierr var-9)))
      (setf iwork (f2cl-lib:int-add ie m))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12 var-13 var-14)
          (sbdsqr "L" m n 0 0 s
           (f2cl-lib:array-slice work-%data% single-float (ie)
                                 ((1 *)) work-%offset%)
           a lda dum 1 dum 1
           (f2cl-lib:array-slice work-%data% single-float (iwork)
                                 ((1 *)) work-%offset%)
           info)
        (declare
         (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
          var-11 var-12 var-13))
        (setf m var-1)
        (setf n var-2)
        (setf lda var-8)
        (setf info var-14)))))
  ((and wntvo wntuas)
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-mul 4 m))
                            (the f2cl-lib:integer4 bdspac))))
      (setf ir 1)
      (cond
       ((>= lwork
            (f2cl-lib:int-add
             (max (the f2cl-lib:integer4 wrkbl)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add (f2cl-lib:int-mul lda n)
                                         m)))
             (f2cl-lib:int-mul lda m)))
        (setf ldwrku lda) (setf chunk n) (setf ldwrkr lda))
       ((>= lwork
            (f2cl-lib:int-add
             (max (the f2cl-lib:integer4 wrkbl)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add (f2cl-lib:int-mul lda n)
                                         m)))
             (f2cl-lib:int-mul m m)))
```

```
  (setf ldwrku lda) (setf chunk n) (setf ldwrkr m))
 (t (setf ldwrku m)
  (setf chunk
          (the f2cl-lib:integer4
               (truncate (- lwork (* m m) m) m)))
  (setf ldwrkr m)))
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
(setf iwork (f2cl-lib:int-add itau m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (sgelqf m n a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" m m a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice u-%data% single-float (1 2)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorglq m n m a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
```

```
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd m m u ldu s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldu var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "U" m m u ldu
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldu var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "P" m m m
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
```

```
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" m m m 0 s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr u ldu dum 1
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf m var-1)
  (setf m var-2)
  (setf m var-3)
  (setf ldwrkr var-8)
  (setf ldu var-10)
  (setf info var-14))
(setf iu (f2cl-lib:int-add ie m))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
              ((> i n) nil)
  (tagbody
    (setf blk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
             (the f2cl-lib:integer4 chunk)))
```

```
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (sgemm "N" "N" m blk m one
           (f2cl-lib:array-slice work-%data% single-float (ir)
                                 ((1 *)) work-%offset%)
           ldwrkr
           (f2cl-lib:array-slice a-%data% single-float (1 i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda zero
           (f2cl-lib:array-slice work-%data% single-float (iu)
                                 ((1 *)) work-%offset%)
           ldwrku)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf blk var-3))
        (when var-4 (setf m var-4))
        (when var-5 (setf one var-5))
        (when var-7 (setf ldwrkr var-7))
        (when var-9 (setf lda var-9))
        (when var-10 (setf zero var-10))
        (when var-12 (setf ldwrku var-12)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slacpy "F" m blk
           (f2cl-lib:array-slice work-%data% single-float (iu)
                                 ((1 *)) work-%offset%)
           ldwrku
           (f2cl-lib:array-slice a-%data% single-float (1 i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf blk var-2))
        (when var-4 (setf ldwrku var-4))
        (when var-6 (setf lda var-6)))
    label40)))
(t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (sgelqf m n a lda
      (f2cl-lib:array-slice work-%data% single-float (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
```

```
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" m m a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice u-%data% single-float (1 2)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorglq m n m a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd m m u ldu s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
```

```
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldu var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (sormbr "P" "L" "T" m n m u ldu
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     a lda
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldu var-7))
  (when var-10 (setf lda var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" m n m 0 s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     a lda u ldu dum 1
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
```

```
        (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
         var-13))
       (setf m var-1)
       (setf n var-2)
       (setf m var-3)
       (setf lda var-8)
       (setf ldu var-10)
       (setf info var-14)))))
(wntvs
 (cond
  (wntun
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-mul 4 m))
                            (the f2cl-lib:integer4 bdspac))))
     (setf ir 1)
     (cond
      ((>= lwork
           (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda m)))
       (setf ldwrkr lda))
      (t (setf ldwrkr m)))
     (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
     (setf iwork (f2cl-lib:int-add itau m))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (sgelqf m n a lda
          (f2cl-lib:array-slice work-%data% single-float (itau)
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% single-float (iwork)
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
          ierr)
       (declare (ignore var-2 var-4 var-5 var-6))
       (setf m var-0)
       (setf n var-1)
       (setf lda var-3)
       (setf ierr var-7))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (slacpy "L" m m a lda
          (f2cl-lib:array-slice work-%data% single-float (ir)
                                ((1 *)) work-%offset%)
          ldwrkr)
       (declare (ignore var-0 var-3 var-5))
       (when var-1 (setf m var-1))
       (when var-2 (setf m var-2))
       (when var-4 (setf lda var-4))
```

```
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ ir ldwrkr)) ((1 *))
                           work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorglq m n m a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd m m
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
```

```
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "P" m m m
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" m m 0 0 s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr dum 1 dum 1
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
    var-11 var-12 var-13))
  (setf m var-1)
  (setf m var-2)
  (setf ldwrkr var-8)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (sgemm "N" "N" m n m one
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr a lda zero vt ldvt)
```

```
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldwrkr var-7))
    (when var-9 (setf lda var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldvt var-12))))
  (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (sgelqf m n a lda
        (f2cl-lib:array-slice work-%data% single-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (sorglq m n m vt ldvt
        (f2cl-lib:array-slice work-%data% single-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf m var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf ldvt var-4))
    (when var-8 (setf ierr var-8)))
   (setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
   (setf itaup (f2cl-lib:int-add itauq m))
   (setf iwork (f2cl-lib:int-add itaup m))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice a-%data% single-float (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd m m a lda s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (sormbr "P" "L" "T" m n m a lda
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldvt var-10))
```

```
      (when var-13 (setf ierr var-13)))
    (setf iwork (f2cl-lib:int-add ie m))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14)
        (sbdsqr "U" m n 0 0 s
         (f2cl-lib:array-slice work-%data% single-float (ie)
                               ((1 *)) work-%offset%)
         vt ldvt dum 1 dum 1
         (f2cl-lib:array-slice work-%data% single-float (iwork)
                               ((1 *)) work-%offset%)
         info)
      (declare
       (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
        var-11 var-12 var-13))
      (setf m var-1)
      (setf n var-2)
      (setf ldvt var-8)
      (setf info var-14)))))
(wntuo
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul 2 m m)
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-mul 4 m))
                          (the f2cl-lib:integer4 bdspac))))
    (setf iu 1)
    (cond
     ((>= lwork
          (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 lda m)))
      (setf ldwrku lda)
      (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
      (setf ldwrkr lda))
     ((>= lwork
          (f2cl-lib:int-add wrkbl
                            (f2cl-lib:int-mul
                             (f2cl-lib:int-add lda m) m)))
      (setf ldwrku lda)
      (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
      (setf ldwrkr m))
     (t (setf ldwrku m)
      (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
      (setf ldwrkr m)))
    (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
    (setf iwork (f2cl-lib:int-add itau m))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (sgelqf m n a lda
         (f2cl-lib:array-slice work-%data% single-float (itau)
```

```
                              ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% single-float (iu)
                              ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice work-%data% single-float
                              ((+ iu ldwrku)) ((1 *))
                              work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorglq m n m a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                              ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
```

```
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd m m
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" m m
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "P" m m m
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
```

```
      ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "Q" m m m
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" m m m 0 s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr dum 1
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf m var-1)
  (setf m var-2)
  (setf m var-3)
  (setf ldwrku var-8)
  (setf ldwrkr var-10)
```

```
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (sgemm "N" "N" m n m one
       (f2cl-lib:array-slice work-%data% single-float (iu)
                             ((1 *)) work-%offset%)
       ldwrku a lda zero vt ldvt)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldwrku var-7))
    (when var-9 (setf lda var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldvt var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "F" m m
       (f2cl-lib:array-slice work-%data% single-float (ir)
                             ((1 *)) work-%offset%)
       ldwrkr a lda)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf ldwrkr var-4))
    (when var-6 (setf lda var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (sgelqf m n a lda
       (f2cl-lib:array-slice work-%data% single-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
```

```
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorglq m n m vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice a-%data% single-float (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd m m a lda s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (sormbr "P" "L" "T" m n m a lda
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "Q" m m m a lda
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" m n m 0 s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     vt ldvt a lda dum 1
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
```

```
      (setf m var-1)
      (setf n var-2)
      (setf m var-3)
      (setf ldvt var-8)
      (setf lda var-10)
      (setf info var-14)))))
(wntuas
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-mul 4 m))
                          (the f2cl-lib:integer4 bdspac))))
   (setf iu 1)
   (cond
    ((>= lwork
         (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda m)))
     (setf ldwrku lda))
    (t (setf ldwrku m)))
   (setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
   (setf iwork (f2cl-lib:int-add itau m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (sgelqf m n a lda
        (f2cl-lib:array-slice work-%data% single-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
     (declare (ignore var-2 var-4 var-5 var-6))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-7))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slacpy "L" m m a lda
        (f2cl-lib:array-slice work-%data% single-float (iu)
                              ((1 *)) work-%offset%)
        ldwrku)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf m var-1))
     (when var-2 (setf m var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf ldwrku var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
```

```
      zero zero
      (f2cl-lib:array-slice work-%data% single-float
                            ((+ iu ldwrku)) ((1 *))
                            work-%offset%)
      ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorglq m n m a lda
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd m m
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" m m
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "P" m m m
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
```

```
      (sbdsqr "U" m m m 0 s
       (f2cl-lib:array-slice work-%data% single-float (ie)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iu)
                             ((1 *)) work-%offset%)
       ldwrku u ldu dum 1
       (f2cl-lib:array-slice work-%data% single-float (iwork)
                             ((1 *)) work-%offset%)
       info)
    (declare
     (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
      var-13))
    (setf m var-1)
    (setf m var-2)
    (setf m var-3)
    (setf ldwrku var-8)
    (setf ldu var-10)
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (sgemm "N" "N" m n m one
       (f2cl-lib:array-slice work-%data% single-float (iu)
                             ((1 *)) work-%offset%)
       ldwrku a lda zero vt ldvt)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldwrku var-7))
    (when var-9 (setf lda var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldvt var-12))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (sgelqf m n a lda
       (f2cl-lib:array-slice work-%data% single-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "U" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorglq m n m vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" m m a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice u-%data% single-float (1 2)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldu var-6)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd m m u ldu s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
```

```
      (f2cl-lib:array-slice work-%data% single-float (itauq)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (itaup)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldu var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (sormbr "P" "L" "T" m n m u ldu
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldu var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
```

```
        (setf iwork (f2cl-lib:int-add ie m))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12 var-13 var-14)
            (sbdsqr "U" m n m 0 s
             (f2cl-lib:array-slice work-%data% single-float (ie)
                                   ((1 *)) work-%offset%)
             vt ldvt u ldu dum 1
             (f2cl-lib:array-slice work-%data% single-float (iwork)
                                   ((1 *)) work-%offset%)
             info)
          (declare
           (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
            var-13))
          (setf m var-1)
          (setf n var-2)
          (setf m var-3)
          (setf ldvt var-8)
          (setf ldu var-10)
          (setf info var-14)))))))
(wntva
 (cond
  (wntun
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add n m))
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-mul 4 m))
                            (the f2cl-lib:integer4 bdspac))))
        (setf ir 1)
        (cond
         ((>= lwork
              (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda m)))
          (setf ldwrkr lda))
         (t (setf ldwrkr m)))
        (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
        (setf iwork (f2cl-lib:int-add itau m))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (sgelqf m n a lda
             (f2cl-lib:array-slice work-%data% single-float (itau)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:array-slice work-%data% single-float (iwork)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
             ierr)
          (declare (ignore var-2 var-4 var-5 var-6))
```

```
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "U" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ ir ldwrkr)) ((1 *))
                           work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorglq n n m vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
```

```
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd m m
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "P" m m m
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" m m 0 0 s
```

```
      (f2cl-lib:array-slice work-%data% single-float (ie)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (ir)
                            ((1 *)) work-%offset%)
      ldwrkr dum 1 dum 1
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
      info)
    (declare
     (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
      var-11 var-12 var-13))
    (setf m var-1)
    (setf m var-2)
    (setf ldwrkr var-8)
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (sgemm "N" "N" m n m one
       (f2cl-lib:array-slice work-%data% single-float (ir)
                            ((1 *)) work-%offset%)
       ldwrkr vt ldvt zero a lda)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldwrkr var-7))
    (when var-9 (setf ldvt var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf lda var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "F" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (sgelqf m n a lda
       (f2cl-lib:array-slice work-%data% single-float (itau)
                            ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
```

```
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "U" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorglq n n m vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice a-%data% single-float (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd m m a lda s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
```

```
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% single-float (iwork)
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
          ierr)
        (declare
         (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
        (setf m var-0)
        (setf m var-1)
        (setf lda var-3)
        (setf ierr var-10))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12 var-13)
          (sormbr "P" "L" "T" m n m a lda
           (f2cl-lib:array-slice work-%data% single-float (itaup)
                                ((1 *)) work-%offset%)
           vt ldvt
           (f2cl-lib:array-slice work-%data% single-float (iwork)
                                ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
           ierr)
        (declare
         (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
          var-12))
        (when var-3 (setf m var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf m var-5))
        (when var-7 (setf lda var-7))
        (when var-10 (setf ldvt var-10))
        (when var-13 (setf ierr var-13)))
      (setf iwork (f2cl-lib:int-add ie m))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12 var-13 var-14)
          (sbdsqr "U" m n 0 0 s
           (f2cl-lib:array-slice work-%data% single-float (ie)
                                ((1 *)) work-%offset%)
           vt ldvt dum 1 dum 1
           (f2cl-lib:array-slice work-%data% single-float (iwork)
                                ((1 *)) work-%offset%)
           info)
        (declare
         (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
          var-11 var-12 var-13))
        (setf m var-1)
        (setf n var-2)
        (setf ldvt var-8)
        (setf info var-14)))))
    (wntuo
```

```
(cond
 ((>= lwork
      (f2cl-lib:int-add (f2cl-lib:int-mul 2 m m)
                        (max
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add n m))
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-mul 4 m))
                         (the f2cl-lib:integer4 bdspac))))
  (setf iu 1)
  (cond
   ((>= lwork
        (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 lda m)))
    (setf ldwrku lda)
    (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
    (setf ldwrkr lda))
   ((>= lwork
        (f2cl-lib:int-add wrkbl
                          (f2cl-lib:int-mul
                           (f2cl-lib:int-add lda m) m)))
    (setf ldwrku lda)
    (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
    (setf ldwrkr m))
   (t (setf ldwrku m)
    (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
    (setf ldwrkr m)))
  (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
  (setf iwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (sgelqf m n a lda
       (f2cl-lib:array-slice work-%data% single-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorglq n n m vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ iu ldwrku)) ((1 *))
                           work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrku var-6)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd m m
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
```

```
      (f2cl-lib:array-slice work-%data% single-float (itauq)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (itaup)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" m m
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "P" m m m
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
```

```
    (sorgbr "Q" m m m
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" m m m 0 s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% single-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr dum 1
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf m var-1)
  (setf m var-2)
  (setf m var-3)
  (setf ldwrku var-8)
  (setf ldwrkr var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (sgemm "N" "N" m n m one
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku vt ldvt zero a lda)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
```

```
    (when var-3 (setf n var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldwrku var-7))
    (when var-9 (setf ldvt var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf lda var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "F" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "F" m m
       (f2cl-lib:array-slice work-%data% single-float (ir)
                             ((1 *)) work-%offset%)
       ldwrkr a lda)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf ldwrkr var-4))
    (when var-6 (setf lda var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (sgelqf m n a lda
       (f2cl-lib:array-slice work-%data% single-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sorglq n n m vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice a-%data% single-float (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd m m a lda s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
      var-9 var-10 var-11 var-12 var-13)
     (sormbr "P" "L" "T" m n m a lda
      (f2cl-lib:array-slice work-%data% single-float (itaup)
                            ((1 *)) work-%offset%)
      vt ldvt
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
   (declare
    (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
     var-12))
   (when var-3 (setf m var-3))
   (when var-4 (setf n var-4))
   (when var-5 (setf m var-5))
   (when var-7 (setf lda var-7))
   (when var-10 (setf ldvt var-10))
   (when var-13 (setf ierr var-13)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9)
     (sorgbr "Q" m m m a lda
      (f2cl-lib:array-slice work-%data% single-float (itauq)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
   (declare (ignore var-0 var-4 var-6 var-7 var-8))
   (when var-1 (setf m var-1))
   (when var-2 (setf m var-2))
   (when var-3 (setf m var-3))
   (when var-5 (setf lda var-5))
   (when var-9 (setf ierr var-9)))
 (setf iwork (f2cl-lib:int-add ie m))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12 var-13 var-14)
     (sbdsqr "U" m n m 0 s
      (f2cl-lib:array-slice work-%data% single-float (ie)
                            ((1 *)) work-%offset%)
      vt ldvt a lda dum 1
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
      info)
   (declare
    (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
     var-13))
   (setf m var-1)
   (setf n var-2)
```

```
      (setf m var-3)
      (setf ldvt var-8)
      (setf lda var-10)
      (setf info var-14)))))
(wntuas
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add n m))
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-mul 4 m))
                          (the f2cl-lib:integer4 bdspac))))
   (setf iu 1)
   (cond
    ((>= lwork
         (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda m)))
     (setf ldwrku lda))
    (t (setf ldwrku m)))
   (setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
   (setf iwork (f2cl-lib:int-add itau m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (sgelqf m n a lda
        (f2cl-lib:array-slice work-%data% single-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
     (declare (ignore var-2 var-4 var-5 var-6))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-7))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slacpy "U" m n a lda vt ldvt)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf m var-1))
     (when var-2 (setf n var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf ldvt var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (sorglq n n m vt ldvt
        (f2cl-lib:array-slice work-%data% single-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (iwork)
```

```
                          ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% single-float (iu)
                          ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice work-%data% single-float
                          ((+ iu ldwrku)) ((1 *))
                          work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrku var-6)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd m m
     (f2cl-lib:array-slice work-%data% single-float (iu)
                          ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                          ((1 *)) work-%offset%)
```

```
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "L" m m
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "P" m m m
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (sorgbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
```

```
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (sbdsqr "U" m m m 0 s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku u ldu dum 1
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf m var-1)
  (setf m var-2)
  (setf m var-3)
  (setf ldwrku var-8)
  (setf ldu var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (sgemm "N" "N" m n m one
     (f2cl-lib:array-slice work-%data% single-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku vt ldvt zero a lda)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldwrku var-7))
  (when var-9 (setf ldvt var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf lda var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "F" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6))))
```

```
(t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (sgelqf m n a lda
      (f2cl-lib:array-slice work-%data% single-float (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
   (declare (ignore var-2 var-4 var-5 var-6))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-7))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slacpy "U" m n a lda vt ldvt)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf m var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldvt var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (sorglq n n m vt ldvt
      (f2cl-lib:array-slice work-%data% single-float (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
   (declare (ignore var-3 var-5 var-6 var-7))
   (when var-0 (setf n var-0))
   (when var-1 (setf n var-1))
   (when var-2 (setf m var-2))
   (when var-4 (setf ldvt var-4))
   (when var-8 (setf ierr var-8)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slacpy "L" m m a lda u ldu)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf m var-1))
   (when var-2 (setf m var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldu var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
      zero zero
```

```
      (f2cl-lib:array-slice u-%data% single-float (1 2)
                            ((1 ldu) (1 *)) u-%offset%)
      ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldu var-6)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgebrd m m u ldu s
     (f2cl-lib:array-slice work-%data% single-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldu var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (sormbr "P" "L" "T" m n m u ldu
     (f2cl-lib:array-slice work-%data% single-float (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% single-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldu var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
```

```
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9)
            (sorgbr "Q" m m m u ldu
             (f2cl-lib:array-slice work-%data% single-float (itauq)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:array-slice work-%data% single-float (iwork)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
             ierr)
          (declare (ignore var-0 var-4 var-6 var-7 var-8))
          (when var-1 (setf m var-1))
          (when var-2 (setf m var-2))
          (when var-3 (setf m var-3))
          (when var-5 (setf ldu var-5))
          (when var-9 (setf ierr var-9)))
        (setf iwork (f2cl-lib:int-add ie m))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12 var-13 var-14)
            (sbdsqr "U" m n m 0 s
             (f2cl-lib:array-slice work-%data% single-float (ie)
                                   ((1 *)) work-%offset%)
             vt ldvt u ldu dum 1
             (f2cl-lib:array-slice work-%data% single-float (iwork)
                                   ((1 *)) work-%offset%)
             info)
          (declare
           (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
            var-13))
          (setf m var-1)
          (setf n var-2)
          (setf m var-3)
          (setf ldvt var-8)
          (setf ldu var-10)
          (setf info var-14)))))))))
(t (setf ie 1) (setf itauq (f2cl-lib:int-add ie m))
 (setf itaup (f2cl-lib:int-add itauq m))
 (setf iwork (f2cl-lib:int-add itaup m))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (sgebrd m n a lda s
      (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (iwork) ((1 *))
```

```
                              work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
      (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
      (setf m var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf ierr var-10))
    (cond
     (wntuas
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slacpy "L" m m a lda u ldu)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf m var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldu var-6)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (sorgbr "Q" m m n u ldu
           (f2cl-lib:array-slice work-%data% single-float (itauq)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% single-float (iwork)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
        (declare (ignore var-0 var-4 var-6 var-7 var-8))
        (when var-1 (setf m var-1))
        (when var-2 (setf m var-2))
        (when var-3 (setf n var-3))
        (when var-5 (setf ldu var-5))
        (when var-9 (setf ierr var-9)))))
    (cond
     (wntvas
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slacpy "U" m n a lda vt ldvt)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldvt var-6)))
      (if wntva
          (setf nrvt n))
      (if wntvs
          (setf nrvt m))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (sorgbr "P" nrvt n m vt ldvt
           (f2cl-lib:array-slice work-%data% single-float (itaup)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% single-float (iwork)
                                 ((1 *)) work-%offset%)
```

```
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf nrvt var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf ldvt var-5))
    (when var-9 (setf ierr var-9)))))
(cond
 (wntuo
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (sorgbr "Q" m m n a lda
       (f2cl-lib:array-slice work-%data% single-float (itauq)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))))
(cond
 (wntvo
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (sorgbr "P" m n m a lda
       (f2cl-lib:array-slice work-%data% single-float (itaup)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))))
(setf iwork (f2cl-lib:int-add ie m))
(if (or wntuas wntuo)
    (setf nru m))
(if wntun
    (setf nru 0))
(if (or wntvas wntvo)
    (setf ncvt n))
(if wntvn
    (setf ncvt 0))
(cond
 ((and (not wntuo) (not wntvo))
```

```
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13 var-14)
     (sbdsqr "L" m ncvt nru 0 s
      (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                            work-%offset%)
      vt ldvt u ldu dum 1
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
      info)
   (declare
    (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
     var-13))
   (setf m var-1)
   (setf ncvt var-2)
   (setf nru var-3)
   (setf ldvt var-8)
   (setf ldu var-10)
   (setf info var-14)))
((and (not wntuo) wntvo)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13 var-14)
     (sbdsqr "L" m ncvt nru 0 s
      (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                            work-%offset%)
      a lda u ldu dum 1
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
      info)
   (declare
    (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
     var-13))
   (setf m var-1)
   (setf ncvt var-2)
   (setf nru var-3)
   (setf lda var-8)
   (setf ldu var-10)
   (setf info var-14)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13 var-14)
     (sbdsqr "L" m ncvt nru 0 s
      (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                            work-%offset%)
      vt ldvt a lda dum 1
      (f2cl-lib:array-slice work-%data% single-float (iwork)
                            ((1 *)) work-%offset%)
      info)
```

```
        (declare
         (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
          var-13))
        (setf m var-1)
        (setf ncvt var-2)
        (setf nru var-3)
        (setf ldvt var-8)
        (setf lda var-10)
        (setf info var-14)))))))
(cond
 ((/= info 0)
  (cond
   ((> ie 2)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add minmn (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1))
                             ((1 *)) work-%offset%)
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add i ie) 1))
                               ((1 *)) work-%offset%))
       label50))))
  (cond
   ((< ie 2)
    (f2cl-lib:fdo (i (f2cl-lib:int-add minmn (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1))
                             ((1 *)) work-%offset%)
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add i ie) 1))
                               ((1 *)) work-%offset%))
       label60))))))
(cond
 ((= iscl 1)
  (if (> anrm bignum)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (slascl "G" 0 0 bignum anrm minmn 1 s minmn ierr)
        (declare (ignore var-0 var-1 var-2 var-6 var-7))
        (when var-3 (setf bignum var-3))
        (when var-4 (setf anrm var-4))
        (when var-5 (setf minmn var-5))
        (when var-8 (setf minmn var-8))
        (when var-9 (setf ierr var-9))))
  (if (and (/= info 0) (> anrm bignum))
```

```
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (slascl "G" 0 0 bignum anrm (f2cl-lib:int-sub minmn 1) 1
                   (f2cl-lib:array-slice work-%data% single-float (2) ((1 *))
                                         work-%offset%)
                   minmn ierr)
                (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
                (when var-3 (setf bignum var-3))
                (when var-4 (setf anrm var-4))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf ierr var-9))))
          (if (< anrm smlnum)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (slascl "G" 0 0 smlnum anrm minmn 1 s minmn ierr)
                (declare (ignore var-0 var-1 var-2 var-6 var-7))
                (when var-3 (setf smlnum var-3))
                (when var-4 (setf anrm var-4))
                (when var-5 (setf minmn var-5))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf ierr var-9))))
          (if (and (/= info 0) (< anrm smlnum))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (slascl "G" 0 0 smlnum anrm (f2cl-lib:int-sub minmn 1) 1
                   (f2cl-lib:array-slice work-%data% single-float (2) ((1 *))
                                         work-%offset%)
                   minmn ierr)
                (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
                (when var-3 (setf smlnum var-3))
                (when var-4 (setf anrm var-4))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf ierr var-9))))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 maxwrk) 'single-float))
        (go end_label)
       end_label
        (return
         (values jobu jobvt m n nil lda nil nil ldu nil ldvt nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgesvd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
```

```
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::jobu
  fortran-to-lisp::jobvt
  fortran-to-lisp::m
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil
  fortran-to-lisp::ldu nil
  fortran-to-lisp::ldvt nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::sbdsqr
  fortran-to-lisp::sgelqf
  fortran-to-lisp::sgebrd
  fortran-to-lisp::sgeqrf
  fortran-to-lisp::ilaenv))))
```

———-

29.4 sgesvdx

29.4.1 Axiom unit tests

— sgesvdx.input —

```
)set break resume
)sys rm -f sgesvdx.output
)spool sgesvdx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

29.4.2 Axiom help page

— sgesvdx.help —

```
==================================================================
sgesvdx examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgesvdx (JOBU, JOBVT, RANGE, M, N, A, LDA, VL, VU, IL, IU,
           NS, S, U, LDU, VT, LDVT, WORK, LWORK, IWORK, INFO)
            SGESVDX computes the singular value decomposition (SVD) for GE
           matrices

Function/Subroutine Documentation
   subroutine sgesvdx (character JOBU, character JOBVT, character RANGE,
       integer M, integer N, real, dimension( lda, * ) A, integer LDA, real
       VL, real VU, integer IL, integer IU, integer NS, real, dimension( * )
       S, real, dimension( ldu, * ) U, integer LDU, real, dimension( ldvt, * )
       VT, integer LDVT, real, dimension( * ) WORK, integer LWORK, integer,
       dimension( * ) IWORK, integer INFO)
        SGESVDX computes the singular value decomposition (SVD) for GE
       matrices

Purpose:
   SGESVDX computes the singular value decomposition (SVD) of a real
   M-by-N matrix A, optionally computing the left and/or right singular
   vectors. The SVD is written

   A = U * SIGMA * transpose(V)

   where SIGMA is an M-by-N matrix which is zero except for its
   min(m,n) diagonal elements, U is an M-by-M orthogonal matrix, and
   V is an N-by-N orthogonal matrix.  The diagonal elements of SIGMA
   are the singular values of A; they are real and non-negative, and
   are returned in descending order.  The first min(m,n) columns of
   U and V are the left and right singular vectors of A.

   SGESVDX uses an eigenvalue problem for obtaining the SVD, which
   allows for the computation of a subset of singular values and
   vectors. See SBDSVDX for details.

   Note that the routine returns V**T, not V.
```

```
Parameters:

 JOBU
   JOBU is CHARACTER*1
   Specifies options for computing all or part of the matrix U:
   = 'V':  the first min(m,n) columns of U (the left singular
           vectors) or as specified by RANGE are returned in the array U;
   = 'N':  no columns of U (no left singular vectors) are computed.

 JOBVT
   JOBVT is CHARACTER*1
   Specifies options for computing all or part of the matrix
   V**T:
   = 'V':  the first min(m,n) rows of V**T (the right singular
           vectors) or as specified by RANGE are returned in the array VT;
   = 'N':  no rows of V**T (no right singular vectors) are computed.

 RANGE
   RANGE is CHARACTER*1
   = 'A': all singular values will be found.
   = 'V': all singular values in the half-open interval (VL,VU] will be found.
   = 'I': the IL-th through IU-th singular values will be found.

 M
   M is INTEGER
   The number of rows of the input matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the input matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, the contents of A are destroyed.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 VL
   VL is REAL
   VL >=0.

 VU
   VU is REAL
   If RANGE='V', the lower and upper bounds of the interval to
   be searched for singular values. VU > VL.
   Not referenced if RANGE = 'A' or 'I'.
```

```
IL
  IL is INTEGER

IU
  IU is INTEGER
  If RANGE='I', the indices (in ascending order) of the
  smallest and largest singular values to be returned.
  1 <= IL <= IU <= min(M,N), if min(M,N) > 0.
  Not referenced if RANGE = 'A' or 'V'.

NS
  NS is INTEGER
  The total number of singular values found,
  0 <= NS <= min(M,N).
  If RANGE = 'A', NS = min(M,N); if RANGE = 'I', NS = IU-IL+1.

S
  S is REAL array, dimension (min(M,N))
  The singular values of A, sorted so that S(i) >= S(i+1).

U
  U is REAL array, dimension (LDU,UCOL)
  If JOBU = 'V', U contains columns of U (the left singular
  vectors, stored columnwise) as specified by RANGE; if
  JOBU = 'N', U is not referenced.
  Note: The user must ensure that UCOL >= NS; if RANGE = 'V',
  the exact value of NS is not known ILQFin advance and an upper
  bound must be used.

LDU
  LDU is INTEGER
  The leading dimension of the array U.  LDU >= 1; if
  JOBU = 'V', LDU >= M.

VT
  VT is REAL array, dimension (LDVT,N)
  If JOBVT = 'V', VT contains the rows of V**T (the right singular
  vectors, stored rowwise) as specified by RANGE; if JOBVT = 'N',
  VT is not referenced.
  Note: The user must ensure that LDVT >= NS; if RANGE = 'V',
  the exact value of NS is not known in advance and an upper
  bound must be used.

LDVT
  LDVT is INTEGER
  The leading dimension of the array VT.  LDVT >= 1; if
  JOBVT = 'V', LDVT >= NS (see above).

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
```

```
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK;

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  LWORK >= MAX(1,MIN(M,N)*(MIN(M,N)+4)) for the paths (see
  comments inside the code):
  - PATH 1  (M much larger than N)
  - PATH 1t (N much larger than M)
  LWORK >= MAX(1,MIN(M,N)*2+MAX(M,N)) for the other paths.
  For good performance, LWORK should generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (12*MIN(M,N))
  If INFO = 0, the first NS elements of IWORK are zero. If INFO > 0,
  then IWORK contains the indices of the eigenvectors that failed
  to converge in SBDSVDX/SSTEVX.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, then i eigenvectors failed to converge
        in SBDSVDX/SSTEVX.
  if INFO = N*2 + 1, an internal error occurred in SBDSVDX
```

———-

29.4.3 fortran code

— sgesvdx.f —

```
*  =====================================================================
      SUBROUTINE SGESVDX( JOBU, JOBVT, RANGE, M, N, A, LDA, VL, VU,
     $                    IL, IU, NS, S, U, LDU, VT, LDVT, WORK,
     $                    LWORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
```

```
*     .. Scalar Arguments ..
      CHARACTER          JOBU, JOBVT, RANGE
      INTEGER            IL, INFO, IU, LDA, LDU, LDVT, LWORK, M, N, NS
      REAL               VL, VU
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      REAL               A( LDA, * ), S( * ), U( LDU, * ),
     $                   VT( LDVT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
*     ..
*     .. Local Scalars ..
      CHARACTER          JOBZ, RNGTGK
      LOGICAL            ALLS, INDS, LQUERY, VALS, WANTU, WANTVT
      INTEGER            I, ID, IE, IERR, ILQF, ILTGK, IQRF, ISCL,
     $                   ITAU, ITAUP, ITAUQ, ITEMP, ITGKZ, IUTGK,
     $                   J, MAXWRK, MINMN, MINWRK, MNTHR
      REAL               ABSTOL, ANRM, BIGNUM, EPS, SMLNUM
*     ..
*     .. Local Arrays ..
      REAL               DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SBDSVDX, SGEBRD, SGELQF, SGEQRF, SLACPY,
     $                   SLASCL, SLASET, SORMBR, SORMLQ, SORMQR,
     $                   SSCAL, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               SLAMCH, SLANGE, SNRM2
      EXTERNAL           LSAME, ILAENV, SLAMCH, SLANGE, SNRM2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      NS = 0
      INFO = 0
      ABSTOL = 2*SLAMCH('S')
      LQUERY = ( LWORK.EQ.-1 )
```

```
      MINMN = MIN( M, N )

      WANTU = LSAME( JOBU, 'V' )
      WANTVT = LSAME( JOBVT, 'V' )
      IF( WANTU .OR. WANTVT ) THEN
         JOBZ = 'V'
      ELSE
         JOBZ = 'N'
      END IF
      ALLS = LSAME( RANGE, 'A' )
      VALS = LSAME( RANGE, 'V' )
      INDS = LSAME( RANGE, 'I' )
*
      INFO = 0
      IF( .NOT.LSAME( JOBU, 'V' ) .AND.
     $    .NOT.LSAME( JOBU, 'N' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.LSAME( JOBVT, 'V' ) .AND.
     $         .NOT.LSAME( JOBVT, 'N' ) ) THEN
         INFO = -2
      ELSE IF( .NOT.( ALLS .OR. VALS .OR. INDS ) ) THEN
         INFO = -3
      ELSE IF( M.LT.0 ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( M.GT.LDA ) THEN
         INFO = -7
      ELSE IF( MINMN.GT.0 ) THEN
         IF( VALS ) THEN
            IF( VL.LT.ZERO ) THEN
               INFO = -8
            ELSE IF( VU.LE.VL ) THEN
               INFO = -9
            END IF
         ELSE IF( INDS ) THEN
            IF( IL.LT.1 .OR. IL.GT.MAX( 1, MINMN ) ) THEN
               INFO = -10
            ELSE IF( IU.LT.MIN( MINMN, IL ) .OR. IU.GT.MINMN ) THEN
               INFO = -11
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
            IF( WANTU .AND. LDU.LT.M ) THEN
               INFO = -15
            ELSE IF( WANTVT .AND. LDVT.LT.MINMN ) THEN
               INFO = -16
            END IF
         END IF
      END IF
```

```
*
*     Compute workspace
*     (Note: Comments in the code beginning "Workspace:" describe the
*     minimal amount of workspace needed at that point in the code,
*     as well as the preferred amount for good performance.
*     NB refers to the optimal block size for the immediately
*     following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = 1
         MAXWRK = 1
         IF( MINMN.GT.0 ) THEN
            IF( M.GE.N ) THEN
               MNTHR = ILAENV( 6, 'SGESVD', JOBU // JOBVT, M, N, 0, 0 )
               IF( M.GE.MNTHR ) THEN
*
*                Path 1 (M much larger than N)
*
                MAXWRK = N*(N*2+16) +
     $                   N*ILAENV( 1, 'SGEQRF', ' ', M, N, -1, -1 )
                MAXWRK = MAX( MAXWRK, N*(N*2+20) + 2*N*
     $                   ILAENV( 1, 'SGEBRD', ' ', N, N, -1, -1 ) )
                MINWRK = N*(N*2+21)
               ELSE
*
*                Path 2 (M at least N, but not much larger)
*
                MAXWRK = N*(N*2+19) + ( M+N )*
     $                   ILAENV( 1, 'SGEBRD', ' ', M, N, -1, -1 )
                MINWRK = N*(N*2+20) + M
               END IF
            ELSE
               MNTHR = ILAENV( 6, 'SGESVD', JOBU // JOBVT, M, N, 0, 0 )
               IF( N.GE.MNTHR ) THEN
*
*                Path 1t (N much larger than M)
*
                MAXWRK = M*(M*2+16) +
     $                   M*ILAENV( 1, 'SGELQF', ' ', M, N, -1, -1 )
                MAXWRK = MAX( MAXWRK, M*(M*2+20) + 2*M*
     $                   ILAENV( 1, 'SGEBRD', ' ', M, M, -1, -1 ) )
                MINWRK = M*(M*2+21)
               ELSE
*
*                Path 2t (N greater than M, but not much larger)
*
                MAXWRK = M*(M*2+19) + ( M+N )*
     $                   ILAENV( 1, 'SGEBRD', ' ', M, N, -1, -1 )
                MINWRK = M*(M*2+20) + N
               END IF
```

```
            END IF
         END IF
         MAXWRK = MAX( MAXWRK, MINWRK )
         WORK( 1 ) = REAL( MAXWRK )
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
             INFO = -19
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGESVDX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         RETURN
      END IF
*
*     Set singular values indices accord to RANGE.
*
      IF( ALLS ) THEN
         RNGTGK = 'I'
         ILTGK = 1
         IUTGK = MIN( M, N )
      ELSE IF( INDS ) THEN
         RNGTGK = 'I'
         ILTGK = IL
         IUTGK = IU
      ELSE
         RNGTGK = 'V'
         ILTGK = 0
         IUTGK = 0
      END IF
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SQRT( SLAMCH( 'S' ) ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = SLANGE( 'M', M, N, A, LDA, DUM )
      ISCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
```

```
         ISCL = 1
         CALL SLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ISCL = 1
         CALL SLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
      END IF
*
      IF( M.GE.N ) THEN
*
*        A has at least as many rows as columns. If A has sufficiently
*        more rows than columns, first reduce A using the QR
*        decomposition.
*
         IF( M.GE.MNTHR ) THEN
*
*           Path 1 (M much larger than N):
*           A = Q * R = Q * ( QB * B * PB**T )
*                     = Q * ( QB * ( UB * S * VB**T ) * PB**T )
*           U = Q * QB * UB; V**T = VB**T * PB**T
*
*           Compute A=Q*R
*           (Workspace: need 2*N, prefer N+N*NB)
*
            ITAU = 1
            ITEMP = ITAU + N
            CALL SGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( ITEMP ),
     $                   LWORK-ITEMP+1, INFO )
*
*           Copy R into WORK and bidiagonalize it:
*           (Workspace: need N*N+5*N, prefer N*N+4*N+2*N*NB)
*
            IQRF = ITEMP
            ID = IQRF + N*N
            IE = ID + N
            ITAUQ = IE + N
            ITAUP = ITAUQ + N
            ITEMP = ITAUP + N
            CALL SLACPY( 'U', N, N, A, LDA, WORK( IQRF ), N )
            CALL SLASET( 'L', N-1, N-1, ZERO, ZERO, WORK( IQRF+1 ), N )
            CALL SGEBRD( N, N, WORK( IQRF ), N, WORK( ID ), WORK( IE ),
     $                   WORK( ITAUQ ), WORK( ITAUP ), WORK( ITEMP ),
     $                   LWORK-ITEMP+1, INFO )
*
*           Solve eigenvalue problem TGK*Z=Z*S.
*           (Workspace: need 14*N + 2*N*(N+1))
*
            ITGKZ = ITEMP
            ITEMP = ITGKZ + N*(N*2+1)
            CALL SBDSVDX( 'U', JOBZ, RNGTGK, N, WORK( ID ), WORK( IE ),
     $                    VL, VU, ILTGK, IUTGK, NS, S, WORK( ITGKZ ),
```

```
     $                   N*2, WORK( ITEMP ), IWORK, INFO)
*
*          If needed, compute left singular vectors.
*
           IF( WANTU ) THEN
              J = ITGKZ
              DO I = 1, NS
                 CALL SCOPY( N, WORK( J ), 1, U( 1,I ), 1 )
                 J = J + N*2
              END DO
              CALL SLASET( 'A', M-N, N, ZERO, ZERO, U( N+1,1 ), LDU )
*
*             Call SORMBR to compute QB*UB.
*             (Workspace in WORK( ITEMP ): need N, prefer N*NB)
*
              CALL SORMBR( 'Q', 'L', 'N', N, NS, N, WORK( IQRF ), N,
     $                     WORK( ITAUQ ), U, LDU, WORK( ITEMP ),
     $                     LWORK-ITEMP+1, INFO )
*
*             Call SORMQR to compute Q*(QB*UB).
*             (Workspace in WORK( ITEMP ): need N, prefer N*NB)
*
              CALL SORMQR( 'L', 'N', M, NS, N, A, LDA,
     $                     WORK( ITAU ), U, LDU, WORK( ITEMP ),
     $                     LWORK-ITEMP+1, INFO )
           END IF
*
*          If needed, compute right singular vectors.
*
           IF( WANTVT) THEN
              J = ITGKZ + N
              DO I = 1, NS
                 CALL SCOPY( N, WORK( J ), 1, VT( I,1 ), LDVT )
                 J = J + N*2
              END DO
*
*             Call SORMBR to compute VB**T * PB**T
*             (Workspace in WORK( ITEMP ): need N, prefer N*NB)
*
              CALL SORMBR( 'P', 'R', 'T', NS, N, N, WORK( IQRF ), N,
     $                     WORK( ITAUP ), VT, LDVT, WORK( ITEMP ),
     $                     LWORK-ITEMP+1, INFO )
           END IF
        ELSE
*
*          Path 2 (M at least N, but not much larger)
*          Reduce A to bidiagonal form without QR decomposition
*          A = QB * B * PB**T = QB * ( UB * S * VB**T ) * PB**T
*          U = QB * UB; V**T = VB**T * PB**T
*
```

```
*           Bidiagonalize A
*           (Workspace: need 4*N+M, prefer 4*N+(M+N)*NB)
*
            ID = 1
            IE = ID + N
            ITAUQ = IE + N
            ITAUP = ITAUQ + N
            ITEMP = ITAUP + N
            CALL SGEBRD( M, N, A, LDA, WORK( ID ), WORK( IE ),
     $                   WORK( ITAUQ ), WORK( ITAUP ), WORK( ITEMP ),
     $                   LWORK-ITEMP+1, INFO )
*
*           Solve eigenvalue problem TGK*Z=Z*S.
*           (Workspace: need 14*N + 2*N*(N+1))
*
            ITGKZ = ITEMP
            ITEMP = ITGKZ + N*(N*2+1)
            CALL SBDSVDX( 'U', JOBZ, RNGTGK, N, WORK( ID ), WORK( IE ),
     $                    VL, VU, ILTGK, IUTGK, NS, S, WORK( ITGKZ ),
     $                    N*2, WORK( ITEMP ), IWORK, INFO)
*
*           If needed, compute left singular vectors.
*
            IF( WANTU ) THEN
               J = ITGKZ
               DO I = 1, NS
                  CALL SCOPY( N, WORK( J ), 1, U( 1,I ), 1 )
                  J = J + N*2
               END DO
               CALL SLASET( 'A', M-N, N, ZERO, ZERO, U( N+1,1 ), LDU )
*
*              Call SORMBR to compute QB*UB.
*              (Workspace in WORK( ITEMP ): need N, prefer N*NB)
*
               CALL SORMBR( 'Q', 'L', 'N', M, NS, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, IERR )
            END IF
*
*           If needed, compute right singular vectors.
*
            IF( WANTVT) THEN
               J = ITGKZ + N
               DO I = 1, NS
                  CALL SCOPY( N, WORK( J ), 1, VT( I,1 ), LDVT )
                  J = J + N*2
               END DO
*
*              Call SORMBR to compute VB**T * PB**T
*              (Workspace in WORK( ITEMP ): need N, prefer N*NB)
```

```
*
               CALL SORMBR( 'P', 'R', 'T', NS, N, N, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, IERR )
            END IF
         END IF
      ELSE
*
*        A has more columns than rows. If A has sufficiently more
*        columns than rows, first reduce A using the LQ decomposition.
*
         IF( N.GE.MNTHR ) THEN
*
*           Path 1t (N much larger than M):
*           A = L * Q = ( QB * B * PB**T ) * Q
*                     = ( QB * ( UB * S * VB**T ) * PB**T ) * Q
*           U = QB * UB ; V**T = VB**T * PB**T * Q
*
*           Compute A=L*Q
*           (Workspace: need 2*M, prefer M+M*NB)
*
            ITAU = 1
            ITEMP = ITAU + M
            CALL SGELQF( M, N, A, LDA, WORK( ITAU ), WORK( ITEMP ),
     $                   LWORK-ITEMP+1, INFO )

*           Copy L into WORK and bidiagonalize it:
*           (Workspace in WORK( ITEMP ): need M*M+5*N, prefer M*M+4*M+2*M*NB)
*
            ILQF = ITEMP
            ID = ILQF + M*M
            IE = ID + M
            ITAUQ = IE + M
            ITAUP = ITAUQ + M
            ITEMP = ITAUP + M
            CALL SLACPY( 'L', M, M, A, LDA, WORK( ILQF ), M )
            CALL SLASET( 'U', M-1, M-1, ZERO, ZERO, WORK( ILQF+M ), M )
            CALL SGEBRD( M, M, WORK( ILQF ), M, WORK( ID ), WORK( IE ),
     $                   WORK( ITAUQ ), WORK( ITAUP ), WORK( ITEMP ),
     $                   LWORK-ITEMP+1, INFO )
*
*           Solve eigenvalue problem TGK*Z=Z*S.
*           (Workspace: need 2*M*M+14*M)
*
            ITGKZ = ITEMP
            ITEMP = ITGKZ + M*(M*2+1)
            CALL SBDSVDX( 'U', JOBZ, RNGTGK, M, WORK( ID ), WORK( IE ),
     $                    VL, VU, ILTGK, IUTGK, NS, S, WORK( ITGKZ ),
     $                    M*2, WORK( ITEMP ), IWORK, INFO)
*
```

```
*          If needed, compute left singular vectors.
*
           IF( WANTU ) THEN
              J = ITGKZ
              DO I = 1, NS
                 CALL SCOPY( M, WORK( J ), 1, U( 1,I ), 1 )
                 J = J + M*2
              END DO
*
*             Call SORMBR to compute QB*UB.
*             (Workspace in WORK( ITEMP ): need M, prefer M*NB)
*
              CALL SORMBR( 'Q', 'L', 'N', M, NS, M, WORK( ILQF ), M,
     $                     WORK( ITAUQ ), U, LDU, WORK( ITEMP ),
     $                     LWORK-ITEMP+1, INFO )
           END IF
*
*          If needed, compute right singular vectors.
*
           IF( WANTVT) THEN
              J = ITGKZ + M
              DO I = 1, NS
                 CALL SCOPY( M, WORK( J ), 1, VT( I,1 ), LDVT )
                 J = J + M*2
              END DO
              CALL SLASET( 'A', M, N-M, ZERO, ZERO, VT( 1,M+1 ), LDVT )
*
*             Call SORMBR to compute (VB**T)*(PB**T)
*             (Workspace in WORK( ITEMP ): need M, prefer M*NB)
*
              CALL SORMBR( 'P', 'R', 'T', NS, M, M, WORK( ILQF ), M,
     $                     WORK( ITAUP ), VT, LDVT, WORK( ITEMP ),
     $                     LWORK-ITEMP+1, INFO )
*
*             Call SORMLQ to compute ((VB**T)*(PB**T))*Q.
*             (Workspace in WORK( ITEMP ): need M, prefer M*NB)
*
              CALL SORMLQ( 'R', 'N', NS, N, M, A, LDA,
     $                     WORK( ITAU ), VT, LDVT, WORK( ITEMP ),
     $                     LWORK-ITEMP+1, INFO )
           END IF
        ELSE
*
*          Path 2t (N greater than M, but not much larger)
*          Reduce to bidiagonal form without LQ decomposition
*          A = QB * B * PB**T = QB * ( UB * S * VB**T ) * PB**T
*          U = QB * UB; V**T = VB**T * PB**T
*
*          Bidiagonalize A
*          (Workspace: need 4*M+N, prefer 4*M+(M+N)*NB)
```

```
*
            ID = 1
            IE = ID + M
            ITAUQ = IE + M
            ITAUP = ITAUQ + M
            ITEMP = ITAUP + M
            CALL SGEBRD( M, N, A, LDA, WORK( ID ), WORK( IE ),
     $                   WORK( ITAUQ ), WORK( ITAUP ), WORK( ITEMP ),
     $                   LWORK-ITEMP+1, INFO )
*
*           Solve eigenvalue problem TGK*Z=Z*S.
*           (Workspace: need 2*M*M+14*M)
*
            ITGKZ = ITEMP
            ITEMP = ITGKZ + M*(M*2+1)
            CALL SBDSVDX( 'L', JOBZ, RNGTGK, M, WORK( ID ), WORK( IE ),
     $                    VL, VU, ILTGK, IUTGK, NS, S, WORK( ITGKZ ),
     $                    M*2, WORK( ITEMP ), IWORK, INFO)
*
*           If needed, compute left singular vectors.
*
            IF( WANTU ) THEN
               J = ITGKZ
               DO I = 1, NS
                  CALL SCOPY( M, WORK( J ), 1, U( 1,I ), 1 )
                  J = J + M*2
               END DO
*
*              Call SORMBR to compute QB*UB.
*              (Workspace in WORK( ITEMP ): need M, prefer M*NB)
*
               CALL SORMBR( 'Q', 'L', 'N', M, NS, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
            END IF
*
*           If needed, compute right singular vectors.
*
            IF( WANTVT) THEN
               J = ITGKZ + M
               DO I = 1, NS
                  CALL SCOPY( M, WORK( J ), 1, VT( I,1 ), LDVT )
                  J = J + M*2
               END DO
               CALL SLASET( 'A', M, N-M, ZERO, ZERO, VT( 1,M+1 ), LDVT )
*
*              Call SORMBR to compute VB**T * PB**T
*              (Workspace in WORK( ITEMP ): need M, prefer M*NB)
*
               CALL SORMBR( 'P', 'R', 'T', NS, N, M, A, LDA,
```

```
     $                      WORK( ITAUP ), VT, LDVT, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
            END IF
         END IF
      END IF
*
*     Undo scaling if necessary
*
      IF( ISCL.EQ.1 ) THEN
         IF( ANRM.GT.BIGNUM )
     $      CALL SLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1,
     $                   S, MINMN, INFO )
         IF( ANRM.LT.SMLNUM )
     $      CALL SLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1,
     $                   S, MINMN, INFO )
      END IF
*
*     Return optimal workspace in WORK(1)
*
      WORK( 1 ) = REAL( MAXWRK )
*
      RETURN
*
*     End of SGESVDX
*
      END
```

———-

29.4.4 lisp code

— sgesvdx.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sgesvdx
         (jobu jobvt range m n a lda vl vu il iu ns s u ldu vt ldvt work lwork
          iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (single-float) vu vl)
             (type (array single-float (*)) work vt u s a)
             (type (f2cl-lib:integer4) info lwork ldvt ldu ns iu il lda n m)
             (type (string *) range jobvt jobu))
    (f2cl-lib:with-multi-array-data
        ((jobu character jobu-%data% jobu-%offset%)
         (jobvt character jobvt-%data% jobvt-%offset%)
```

```
    (range character range-%data% range-%offset%)
    (a single-float a-%data% a-%offset%)
    (s single-float s-%data% s-%offset%)
    (u single-float u-%data% u-%offset%)
    (vt single-float vt-%data% vt-%offset%)
    (work single-float work-%data% work-%offset%)
    (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((dum (make-array 1 :element-type 'single-float)) (abstol 0.0)
         (anrm 0.0) (bignum 0.0) (eps 0.0) (smlnum 0.0) (i 0) (id 0) (ie 0)
         (ierr 0) (ilqf 0) (iltgk 0) (iqrf 0) (iscl 0) (itau 0) (itaup 0)
         (itauq 0) (itemp 0) (itgkz 0) (iutgk 0) (j 0) (maxwrk 0) (minmn 0)
         (minwrk 0) (mnthr 0) (alls nil) (inds nil) (lquery nil) (vals nil)
         (wantu nil) (wantvt nil)
         (jobz
          (make-array '(1) :element-type 'character :initial-element #\ ))
         (rngtgk
          (make-array '(1) :element-type 'character :initial-element #\ )))
    (declare (type (array single-float (1)) dum)
             (type (single-float) abstol anrm bignum eps smlnum)
             (type (f2cl-lib:integer4) i id ie ierr ilqf iltgk iqrf iscl
              itau itaup itauq itemp itgkz iutgk j maxwrk minmn minwrk
              mnthr)
             (type f2cl-lib:logical alls inds lquery vals wantu wantvt)
             (type (string 1) jobz rngtgk))
    (setf ns 0)
    (setf info 0)
    (setf abstol (* 2 (slamch "S")))
    (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
    (setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
    (setf wantu
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobu "V")
              (declare (ignore var-1))
              (when var-0 (setf jobu var-0))
              ret-val))
    (setf wantvt
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobvt "V")
              (declare (ignore var-1))
              (when var-0 (setf jobvt var-0))
              ret-val))
    (cond
     ((or wantu wantvt) (f2cl-lib:f2cl-set-string jobz "V" (string 1)))
     (t (f2cl-lib:f2cl-set-string jobz "N" (string 1))))
    (setf alls
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame range "A")
              (declare (ignore var-1))
              (when var-0 (setf range var-0))
              ret-val))
```

```
(setf vals
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "V")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf inds
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "I")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf info 0)
(cond
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame jobu "V")
      (declare (ignore var-1))
      (when var-0 (setf jobu var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame jobu "N")
      (declare (ignore var-1))
      (when var-0 (setf jobu var-0))
      ret-val)))
  (setf info -1))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame jobvt "V")
      (declare (ignore var-1))
      (when var-0 (setf jobvt var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame jobvt "N")
      (declare (ignore var-1))
      (when var-0 (setf jobvt var-0))
      ret-val)))
  (setf info -2))
 ((not (or alls vals inds)) (setf info -3)) ((< m 0) (setf info -4))
 ((< n 0) (setf info -5)) ((> m lda) (setf info -7))
 ((> minmn 0)
  (cond
   (vals
    (cond ((< vl zero) (setf info -8)) ((<= vu vl) (setf info -9))))
   (inds
    (cond
```

```
      ((or (< il 1)
           (> il
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 minmn))))
       (setf info -10))
      ((or
        (< iu
           (min (the f2cl-lib:integer4 minmn)
                (the f2cl-lib:integer4 il)))
        (> iu minmn))
       (setf info -11)))))
  (cond
   ((= info 0)
    (cond ((and wantu (< ldu m)) (setf info -15))
          ((and wantvt (< ldvt minmn)) (setf info -16)))))))
(cond
 ((= info 0) (setf minwrk 1) (setf maxwrk 1)
  (cond
   ((> minmn 0)
    (cond
     ((>= m n)
      (setf mnthr
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 6 "SGESVD" (f2cl-lib:f2cl-// jobu jobvt) m n
                   0 0)
                (declare (ignore var-0 var-1 var-2 var-5 var-6))
                (setf m var-3)
                (setf n var-4)
                ret-val))
      (cond
       ((>= m mnthr)
        (setf maxwrk
                (f2cl-lib:int-add
                 (f2cl-lib:int-mul n
                                   (f2cl-lib:int-add
                                    (f2cl-lib:int-mul n 2) 16))
                 (f2cl-lib:int-mul n
                                   (multiple-value-bind
                                       (ret-val var-0 var-1 var-2 var-3
                                        var-4 var-5 var-6)
                                       (ilaenv 1 "SGEQRF" " " m n -1
                                        -1)
                                     (declare
                                      (ignore var-0 var-1 var-2 var-5
                                       var-6))
                                     (setf m var-3)
                                     (setf n var-4)
                                     ret-val))))
        (setf maxwrk
```

```
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add
                           (f2cl-lib:int-mul n
                                             (f2cl-lib:int-add
                                              (f2cl-lib:int-mul n 2)
                                              20))
                           (f2cl-lib:int-mul 2 n
                                             (multiple-value-bind
                                                 (ret-val var-0 var-1
                                                  var-2 var-3 var-4
                                                  var-5 var-6)
                                                 (ilaenv 1 "SGEBRD" " "
                                                  n n -1 -1)
                                               (declare
                                                (ignore var-0 var-1
                                                 var-2 var-5 var-6))
                                               (setf n var-3)
                                               (setf n var-4)
                                               ret-val))))))
    (setf minwrk
            (f2cl-lib:int-mul n
                              (f2cl-lib:int-add
                               (f2cl-lib:int-mul n 2) 21))))
   (t
    (setf maxwrk
            (f2cl-lib:int-add
             (f2cl-lib:int-mul n
                               (f2cl-lib:int-add
                                (f2cl-lib:int-mul n 2) 19))
             (f2cl-lib:int-mul (f2cl-lib:int-add m n)
                               (multiple-value-bind
                                   (ret-val var-0 var-1 var-2 var-3
                                    var-4 var-5 var-6)
                                   (ilaenv 1 "SGEBRD" " " m n -1
                                    -1)
                                 (declare
                                  (ignore var-0 var-1 var-2 var-5
                                   var-6))
                                 (setf m var-3)
                                 (setf n var-4)
                                 ret-val))))
    (setf minwrk
            (f2cl-lib:int-add
             (f2cl-lib:int-mul n
                               (f2cl-lib:int-add
                                (f2cl-lib:int-mul n 2) 20))
             m)))))
  (t
   (setf mnthr
```

```
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 6 "SGESVD" (f2cl-lib:f2cl-// jobu jobvt) m n
             0 0)
          (declare (ignore var-0 var-1 var-2 var-5 var-6))
          (setf m var-3)
          (setf n var-4)
          ret-val))
(cond
 ((>= n mnthr)
  (setf maxwrk
          (f2cl-lib:int-add
           (f2cl-lib:int-mul m
                             (f2cl-lib:int-add
                              (f2cl-lib:int-mul m 2) 16))
           (f2cl-lib:int-mul m
                             (multiple-value-bind
                                 (ret-val var-0 var-1 var-2 var-3
                                  var-4 var-5 var-6)
                                 (ilaenv 1 "SGELQF" " " m n -1
                                  -1)
                               (declare
                                (ignore var-0 var-1 var-2 var-5
                                 var-6))
                               (setf m var-3)
                               (setf n var-4)
                               ret-val))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add
                     (f2cl-lib:int-mul m
                                       (f2cl-lib:int-add
                                        (f2cl-lib:int-mul m 2)
                                        20))
                     (f2cl-lib:int-mul 2 m
                                       (multiple-value-bind
                                           (ret-val var-0 var-1
                                            var-2 var-3 var-4
                                            var-5 var-6)
                                           (ilaenv 1 "SGEBRD" " "
                                            m m -1 -1)
                                         (declare
                                          (ignore var-0 var-1
                                           var-2 var-5 var-6))
                                         (setf m var-3)
                                         (setf m var-4)
                                         ret-val))))))
  (setf minwrk
          (f2cl-lib:int-mul m
```

```
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-mul m 2) 21))))
        (t
         (setf maxwrk
                 (f2cl-lib:int-add
                  (f2cl-lib:int-mul m
                                   (f2cl-lib:int-add
                                    (f2cl-lib:int-mul m 2) 19))
                  (f2cl-lib:int-mul (f2cl-lib:int-add m n)
                                   (multiple-value-bind
                                       (ret-val var-0 var-1 var-2 var-3
                                        var-4 var-5 var-6)
                                       (ilaenv 1 "SGEBRD" " " m n -1
                                        -1)
                                     (declare
                                      (ignore var-0 var-1 var-2 var-5
                                       var-6))
                                     (setf m var-3)
                                     (setf n var-4)
                                     ret-val))))
         (setf minwrk
                 (f2cl-lib:int-add
                  (f2cl-lib:int-mul m
                                   (f2cl-lib:int-add
                                    (f2cl-lib:int-mul m 2) 20))
                  n))))))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4 minwrk)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:freal maxwrk))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -19)))))
(cond
 ((/= info 0) (xerbla "SGESVDX" (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(cond ((or (= m 0) (= n 0)) (go end_label)))
(cond
 (alls (f2cl-lib:f2cl-set-string rngtgk "I" (string 1)) (setf iltgk 1)
  (setf iutgk
          (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n))))
 (inds (f2cl-lib:f2cl-set-string rngtgk "I" (string 1)) (setf iltgk il)
  (setf iutgk iu))
 (t (f2cl-lib:f2cl-set-string rngtgk "V" (string 1)) (setf iltgk 0)
  (setf iutgk 0)))
(setf eps (slamch "P"))
(setf smlnum (/ (f2cl-lib:fsqrt (slamch "S")) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
```

```
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slange "M" m n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iscl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf iscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 anrm smlnum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9))))
 ((> anrm bignum) (setf iscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 anrm bignum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))))
(cond
 ((>= m n)
  (cond
   ((>= m mnthr) (setf itau 1) (setf itemp (f2cl-lib:int-add itau n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (sgeqrf m n a lda
         (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                               work-%offset%)
         (f2cl-lib:array-slice work-%data% single-float (itemp) ((1 *))
                               work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
      (declare (ignore var-2 var-4 var-5 var-6))
      (setf m var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf info var-7))
    (setf iqrf itemp)
    (setf id (f2cl-lib:int-add iqrf (f2cl-lib:int-mul n n)))
```

```
(setf ie (f2cl-lib:int-add id n))
(setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf itemp (f2cl-lib:int-add itaup n))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% single-float (iqrf) ((1 *))
                           work-%offset%)
     n)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf n var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1) zero
     zero
     (f2cl-lib:array-slice work-%data% single-float ((+ iqrf 1))
                           ((1 *)) work-%offset%)
     n)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf n var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (sgebrd n n
     (f2cl-lib:array-slice work-%data% single-float (iqrf) ((1 *))
                           work-%offset%)
     n
     (f2cl-lib:array-slice work-%data% single-float (id) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itemp) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf n var-3)
  (setf info var-10))
(setf itgkz itemp)
(setf itemp
        (f2cl-lib:int-add itgkz
```

```
                              (f2cl-lib:int-mul n
                                                (f2cl-lib:int-add
                                                 (f2cl-lib:int-mul n 2)
                                                 1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (sbdsvdx "U" jobz rngtgk n
     (f2cl-lib:array-slice work-%data% single-float (id) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     vl vu iltgk iutgk ns s
     (f2cl-lib:array-slice work-%data% single-float (itgkz) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-mul n 2)
     (f2cl-lib:array-slice work-%data% single-float (itemp) ((1 *))
                           work-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-8 var-9 var-11 var-12
    var-13 var-14 var-15))
  (setf jobz var-1)
  (setf rngtgk var-2)
  (setf n var-3)
  (setf ns var-10)
  (setf info var-16))
(cond
 (wantu (setf j itgkz)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (scopy n
           (f2cl-lib:array-slice work-%data% single-float (j)
                                 ((1 *)) work-%offset%)
           1
           (f2cl-lib:array-slice u-%data% single-float (1 i)
                                 ((1 ldu) (1 *)) u-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf n var-0)))
      (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul n 2)))
     label100000))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "A" (f2cl-lib:int-sub m n) n zero zero
       (f2cl-lib:array-slice u-%data% single-float ((+ n 1) 1)
                             ((1 ldu) (1 *)) u-%offset%)
       ldu)
    (declare (ignore var-0 var-1 var-5))
```

```
        (when var-2 (setf n var-2))
        (when var-3 (setf zero var-3))
        (when var-4 (setf zero var-4))
        (when var-6 (setf ldu var-6)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13)
          (sormbr "Q" "L" "N" n ns n
           (f2cl-lib:array-slice work-%data% single-float (iqrf)
                                 ((1 *)) work-%offset%)
           n
           (f2cl-lib:array-slice work-%data% single-float (itauq)
                                 ((1 *)) work-%offset%)
           u ldu
           (f2cl-lib:array-slice work-%data% single-float (itemp)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
        (declare
         (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
        (when var-3 (setf n var-3))
        (when var-4 (setf ns var-4))
        (when var-5 (setf n var-5))
        (when var-7 (setf n var-7))
        (when var-10 (setf ldu var-10))
        (when var-13 (setf info var-13)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (sormqr "L" "N" m ns n a lda
           (f2cl-lib:array-slice work-%data% single-float (itau)
                                 ((1 *)) work-%offset%)
           u ldu
           (f2cl-lib:array-slice work-%data% single-float (itemp)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf ns var-3))
        (when var-4 (setf n var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldu var-9))
        (when var-12 (setf info var-12)))))
    (cond
     (wantvt (setf j (f2cl-lib:int-add itgkz n))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i ns) nil)
        (tagbody
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (scopy n
               (f2cl-lib:array-slice work-%data% single-float (j)
```

```
                                  ((1 *)) work-%offset%)
                1
                (f2cl-lib:array-slice vt-%data% single-float (i 1)
                                  ((1 ldvt) (1 *)) vt-%offset%)
                ldvt)
            (declare (ignore var-1 var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-4 (setf ldvt var-4)))
          (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul n 2)))
         label100001))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13)
          (sormbr "P" "R" "T" ns n n
           (f2cl-lib:array-slice work-%data% single-float (iqrf)
                                 ((1 *)) work-%offset%)
           n
           (f2cl-lib:array-slice work-%data% single-float (itaup)
                                 ((1 *)) work-%offset%)
           vt ldvt
           (f2cl-lib:array-slice work-%data% single-float (itemp)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
        (declare
         (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
        (when var-3 (setf ns var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf n var-5))
        (when var-7 (setf n var-7))
        (when var-10 (setf ldvt var-10))
        (when var-13 (setf info var-13))))))
  (t (setf id 1) (setf ie (f2cl-lib:int-add id n))
   (setf itauq (f2cl-lib:int-add ie n))
   (setf itaup (f2cl-lib:int-add itauq n))
   (setf itemp (f2cl-lib:int-add itaup n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (sgebrd m n a lda
        (f2cl-lib:array-slice work-%data% single-float (id) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (itemp) ((1 *))
                              work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
```

```
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-10))
(setf itgkz itemp)
(setf itemp
        (f2cl-lib:int-add itgkz
                          (f2cl-lib:int-mul n
                                            (f2cl-lib:int-add
                                             (f2cl-lib:int-mul n 2)
                                             1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (sbdsvdx "U" jobz rngtgk n
     (f2cl-lib:array-slice work-%data% single-float (id) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     vl vu iltgk iutgk ns s
     (f2cl-lib:array-slice work-%data% single-float (itgkz) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-mul n 2)
     (f2cl-lib:array-slice work-%data% single-float (itemp) ((1 *))
                           work-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-8 var-9 var-11 var-12
    var-13 var-14 var-15))
  (setf jobz var-1)
  (setf rngtgk var-2)
  (setf n var-3)
  (setf ns var-10)
  (setf info var-16))
(cond
 (wantu (setf j itgkz)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (scopy n
           (f2cl-lib:array-slice work-%data% single-float (j)
                                 ((1 *)) work-%offset%)
           1
           (f2cl-lib:array-slice u-%data% single-float (1 i)
                                 ((1 ldu) (1 *)) u-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf n var-0)))
```

```
        (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul n 2)))
       label100002))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "A" (f2cl-lib:int-sub m n) n zero zero
         (f2cl-lib:array-slice u-%data% single-float ((+ n 1) 1)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldu var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (sormbr "Q" "L" "N" m ns n a lda
         (f2cl-lib:array-slice work-%data% single-float (itauq)
                               ((1 *)) work-%offset%)
         u ldu
         (f2cl-lib:array-slice work-%data% single-float (itemp)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) ierr)
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
      (when var-3 (setf m var-3))
      (when var-4 (setf ns var-4))
      (when var-5 (setf n var-5))
      (when var-7 (setf lda var-7))
      (when var-10 (setf ldu var-10))
      (when var-13 (setf ierr var-13)))))
  (cond
   (wantvt (setf j (f2cl-lib:int-add itgkz n))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i ns) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (scopy n
             (f2cl-lib:array-slice work-%data% single-float (j)
                                   ((1 *)) work-%offset%)
             1
             (f2cl-lib:array-slice vt-%data% single-float (i 1)
                                   ((1 ldvt) (1 *)) vt-%offset%)
             ldvt)
          (declare (ignore var-1 var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-4 (setf ldvt var-4)))
        (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul n 2)))
       label100003))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
          var-10 var-11 var-12 var-13)
         (sormbr "P" "R" "T" ns n n a lda
          (f2cl-lib:array-slice work-%data% single-float (itaup)
                                ((1 *)) work-%offset%)
          vt ldvt
          (f2cl-lib:array-slice work-%data% single-float (itemp)
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) ierr)
       (declare
        (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
       (when var-3 (setf ns var-3))
       (when var-4 (setf n var-4))
       (when var-5 (setf n var-5))
       (when var-7 (setf lda var-7))
       (when var-10 (setf ldvt var-10))
       (when var-13 (setf ierr var-13))))))))
(t
 (cond
  ((>= n mnthr) (setf itau 1) (setf itemp (f2cl-lib:int-add itau m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (sgelqf m n a lda
        (f2cl-lib:array-slice work-%data% single-float (itau) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (itemp) ((1 *))
                              work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
     (declare (ignore var-2 var-4 var-5 var-6))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf info var-7))
   (setf ilqf itemp)
   (setf id (f2cl-lib:int-add ilqf (f2cl-lib:int-mul m m)))
   (setf ie (f2cl-lib:int-add id m))
   (setf itauq (f2cl-lib:int-add ie m))
   (setf itaup (f2cl-lib:int-add itauq m))
   (setf itemp (f2cl-lib:int-add itaup m))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slacpy "L" m m a lda
        (f2cl-lib:array-slice work-%data% single-float (ilqf) ((1 *))
                              work-%offset%)
        m)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf m var-1))
     (when var-2 (setf m var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf m var-6)))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (slaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1) zero
```

```
      zero
      (f2cl-lib:array-slice work-%data% single-float ((+ ilqf m))
                            ((1 *)) work-%offset%)
      m)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf m var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (sgebrd m m
     (f2cl-lib:array-slice work-%data% single-float (ilqf) ((1 *))
                           work-%offset%)
     m
     (f2cl-lib:array-slice work-%data% single-float (id) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (itemp) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf m var-3)
  (setf info var-10))
(setf itgkz itemp)
(setf itemp
        (f2cl-lib:int-add itgkz
                          (f2cl-lib:int-mul m
                                            (f2cl-lib:int-add
                                             (f2cl-lib:int-mul m 2)
                                             1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (sbdsvdx "U" jobz rngtgk m
     (f2cl-lib:array-slice work-%data% single-float (id) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                           work-%offset%)
     vl vu iltgk iutgk ns s
     (f2cl-lib:array-slice work-%data% single-float (itgkz) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-mul m 2)
```

```
      (f2cl-lib:array-slice work-%data% single-float (itemp) ((1 *))
                            work-%offset%)
      iwork info)
    (declare
     (ignore var-0 var-4 var-5 var-6 var-7 var-8 var-9 var-11 var-12
      var-13 var-14 var-15))
    (setf jobz var-1)
    (setf rngtgk var-2)
    (setf m var-3)
    (setf ns var-10)
    (setf info var-16))
  (cond
   (wantu (setf j itgkz)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i ns) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (scopy m
             (f2cl-lib:array-slice work-%data% single-float (j)
                                   ((1 *)) work-%offset%)
             1
             (f2cl-lib:array-slice u-%data% single-float (1 i)
                                   ((1 ldu) (1 *)) u-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf m var-0)))
        (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul m 2)))
       label100004))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (sormbr "Q" "L" "N" m ns m
         (f2cl-lib:array-slice work-%data% single-float (ilqf)
                               ((1 *)) work-%offset%)
         m
         (f2cl-lib:array-slice work-%data% single-float (itauq)
                               ((1 *)) work-%offset%)
         u ldu
         (f2cl-lib:array-slice work-%data% single-float (itemp)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
      (when var-3 (setf m var-3))
      (when var-4 (setf ns var-4))
      (when var-5 (setf m var-5))
      (when var-7 (setf m var-7))
      (when var-10 (setf ldu var-10))
      (when var-13 (setf info var-13)))))
  (cond
```

```
(wantvt (setf j (f2cl-lib:int-add itgkz m))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i ns) nil)
   (tagbody
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (scopy m
          (f2cl-lib:array-slice work-%data% single-float (j)
                                ((1 *)) work-%offset%)
          1
          (f2cl-lib:array-slice vt-%data% single-float (i 1)
                                ((1 ldvt) (1 *)) vt-%offset%)
          ldvt)
       (declare (ignore var-1 var-2 var-3))
       (when var-0 (setf m var-0))
       (when var-4 (setf ldvt var-4)))
     (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul m 2)))
    label100005))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slaset "A" m (f2cl-lib:int-sub n m) zero zero
      (f2cl-lib:array-slice vt-%data% single-float
                            (1 (f2cl-lib:int-add m 1))
                            ((1 ldvt) (1 *)) vt-%offset%)
      ldvt)
   (declare (ignore var-0 var-2 var-5))
   (when var-1 (setf m var-1))
   (when var-3 (setf zero var-3))
   (when var-4 (setf zero var-4))
   (when var-6 (setf ldvt var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13)
     (sormbr "P" "R" "T" ns m m
      (f2cl-lib:array-slice work-%data% single-float (ilqf)
                            ((1 *)) work-%offset%)
      m
      (f2cl-lib:array-slice work-%data% single-float (itaup)
                            ((1 *)) work-%offset%)
      vt ldvt
      (f2cl-lib:array-slice work-%data% single-float (itemp)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
   (declare
    (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
   (when var-3 (setf ns var-3))
   (when var-4 (setf m var-4))
   (when var-5 (setf m var-5))
   (when var-7 (setf m var-7))
   (when var-10 (setf ldvt var-10))
   (when var-13 (setf info var-13)))
 (multiple-value-bind
```

```
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (sormlq "R" "N" ns n m a lda
         (f2cl-lib:array-slice work-%data% single-float (itau)
                               ((1 *)) work-%offset%)
         vt ldvt
         (f2cl-lib:array-slice work-%data% single-float (itemp)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf ns var-2))
      (when var-3 (setf n var-3))
      (when var-4 (setf m var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldvt var-9))
      (when var-12 (setf info var-12))))))
  (t (setf id 1) (setf ie (f2cl-lib:int-add id m))
   (setf itauq (f2cl-lib:int-add ie m))
   (setf itaup (f2cl-lib:int-add itauq m))
   (setf itemp (f2cl-lib:int-add itaup m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (sgebrd m n a lda
        (f2cl-lib:array-slice work-%data% single-float (id) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (itauq) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (itaup) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (itemp) ((1 *))
                              work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
     (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf info var-10))
   (setf itgkz itemp)
   (setf itemp
           (f2cl-lib:int-add itgkz
                             (f2cl-lib:int-mul m
                                               (f2cl-lib:int-add
                                                (f2cl-lib:int-mul m 2)
                                                1))))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13 var-14 var-15 var-16)
```

```
      (sbdsvdx "L" jobz rngtgk m
       (f2cl-lib:array-slice work-%data% single-float (id) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (ie) ((1 *))
                             work-%offset%)
       vl vu iltgk iutgk ns s
       (f2cl-lib:array-slice work-%data% single-float (itgkz) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-mul m 2)
       (f2cl-lib:array-slice work-%data% single-float (itemp) ((1 *))
                             work-%offset%)
       iwork info)
    (declare
     (ignore var-0 var-4 var-5 var-6 var-7 var-8 var-9 var-11 var-12
      var-13 var-14 var-15))
    (setf jobz var-1)
    (setf rngtgk var-2)
    (setf m var-3)
    (setf ns var-10)
    (setf info var-16))
  (cond
   (wantu (setf j itgkz)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i ns) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (scopy m
             (f2cl-lib:array-slice work-%data% single-float (j)
                                   ((1 *)) work-%offset%)
             1
             (f2cl-lib:array-slice u-%data% single-float (1 i)
                                   ((1 ldu) (1 *)) u-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf m var-0)))
        (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul m 2)))
       label100006))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (sormbr "Q" "L" "N" m ns n a lda
         (f2cl-lib:array-slice work-%data% single-float (itauq)
                               ((1 *)) work-%offset%)
         u ldu
         (f2cl-lib:array-slice work-%data% single-float (itemp)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
      (when var-3 (setf m var-3))
```

```
    (when var-4 (setf ns var-4))
    (when var-5 (setf n var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldu var-10))
    (when var-13 (setf info var-13)))))
(cond
 (wantvt (setf j (f2cl-lib:int-add itgkz m))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (scopy m
           (f2cl-lib:array-slice work-%data% single-float (j)
                                 ((1 *)) work-%offset%)
           1
           (f2cl-lib:array-slice vt-%data% single-float (i 1)
                                 ((1 ldvt) (1 *)) vt-%offset%)
           ldvt)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-4 (setf ldvt var-4)))
      (setf j (f2cl-lib:int-add j (f2cl-lib:int-mul m 2)))
     label100007))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "A" m (f2cl-lib:int-sub n m) zero zero
       (f2cl-lib:array-slice vt-%data% single-float
                             (1 (f2cl-lib:int-add m 1))
                             ((1 ldvt) (1 *)) vt-%offset%)
       ldvt)
    (declare (ignore var-0 var-2 var-5))
    (when var-1 (setf m var-1))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (sormbr "P" "R" "T" ns n m a lda
       (f2cl-lib:array-slice work-%data% single-float (itaup)
                             ((1 *)) work-%offset%)
       vt ldvt
       (f2cl-lib:array-slice work-%data% single-float (itemp)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf ns var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf m var-5))
    (when var-7 (setf lda var-7))
```

```
                (when var-10 (setf ldvt var-10))
                (when var-13 (setf info var-13)))))))))
        (cond
         ((= iscl 1)
          (if (> anrm bignum)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (slascl "G" 0 0 bignum anrm minmn 1 s minmn info)
                (declare (ignore var-0 var-1 var-2 var-6 var-7))
                (when var-3 (setf bignum var-3))
                (when var-4 (setf anrm var-4))
                (when var-5 (setf minmn var-5))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf info var-9))))
          (if (< anrm smlnum)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (slascl "G" 0 0 smlnum anrm minmn 1 s minmn info)
                (declare (ignore var-0 var-1 var-2 var-6 var-7))
                (when var-3 (setf smlnum var-3))
                (when var-4 (setf anrm var-4))
                (when var-5 (setf minmn var-5))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf info var-9))))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (f2cl-lib:freal maxwrk))
        (go end_label)
       end_label
        (return
         (values jobu jobvt range m n nil lda nil nil nil nil ns nil nil ldu
                 nil ldvt nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgesvdx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
```

```
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::jobu
  fortran-to-lisp::jobvt
  fortran-to-lisp::range
  fortran-to-lisp::m
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil nil
  nil fortran-to-lisp::ns nil nil
  fortran-to-lisp::ldu nil
  fortran-to-lisp::ldvt nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::sgelqf
  fortran-to-lisp::sbdsvdx
  fortran-to-lisp::sgebrd
  fortran-to-lisp::sgeqrf
  fortran-to-lisp::ilaenv))))
```

Chapter 30

LAPACK - General Matrices, Singular Value, Complex

30.1 cgejsv

30.1.1 Axiom unit tests

— cgejsv.input —

```
)set break resume
)sys rm -f cgejsv.output
)spool cgejsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

30.1.2 Axiom help page

— cgejsv.help —

```
========================================================================
cgejsv examples
========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgejsv (JOBA, JOBU, JOBV, JOBR, JOBT, JOBP, M, N, A, LDA,
           SVA, U, LDU, V, LDV, CWORK, LWORK, RWORK, LRWORK, IWORK, INFO)
           CGEJSV

Function/Subroutine Documentation
   subroutine cgejsv (character*1 JOBA, character*1 JOBU, character*1 JOBV,
       character*1 JOBR, character*1 JOBT, character*1 JOBP, integer M,
       integer N, complex, dimension( lda, * ) A, integer LDA, real,
       dimension( n ) SVA, complex, dimension( ldu, * ) U, integer LDU,
       complex, dimension( ldv, * ) V, integer LDV, complex, dimension( lwork
       ) CWORK, integer LWORK, real, dimension( * ) RWORK, integer LRWORK,
       integer, dimension( * ) IWORK, integer INFO)
       CGEJSV

Purpose:
   CGEJSV computes the singular value decomposition (SVD) of a real M-by-N
   matrix [A], where M >= N. The SVD of [A] is written as

      [A] = [U] * [SIGMA] * [V]^*,

   where [SIGMA] is an N-by-N (M-by-N) matrix which is zero except for its N
   diagonal elements, [U] is an M-by-N (or M-by-M) orthonormal matrix, and
   [V] is an N-by-N orthogonal matrix. The diagonal elements of [SIGMA] are
   the singular values of [A]. The columns of [U] and [V] are the left and
   the right singular vectors of [A], respectively. The matrices [U] and [V]
   are computed and stored in the arrays U and V, respectively. The diagonal
   of [SIGMA] is computed and stored in the array SVA.

   Arguments:
   ==========

 JOBA
   JOBA is CHARACTER*1
   Specifies the level of accuracy:
   = 'C': This option works well (high relative accuracy) if A = B * D,
          with well-conditioned B and arbitrary diagonal matrix D.
          The accuracy cannot be spoiled by COLUMN scaling. The
          accuracy of the computed output depends on the condition of
          B, and the procedure aims at the best theoretical accuracy.
          The relative error max_{i=1:N}|d sigma_i| / sigma_i is
          bounded by f(M,N)*epsilon* cond(B), independent of D.
          The input matrix is preprocessed with the QRF with column
          pivoting. This initial preprocessing and preconditioning by
```

```
          a rank revealing QR factorization is common for all values of
          JOBA. Additional actions are specified as follows:

   = 'E': Computation as with 'C' with an additional estimate of the
          condition number of B. It provides a realistic error bound.

   = 'F':
     If A = D1 * C * D2 with ill-conditioned diagonal scalings
            D1, D2, and well-conditioned matrix C, this option gives
            higher accuracy than the 'C' option. If the structure of the
            input matrix is not known, and relative accuracy is
            desirable, then this option is advisable. The input matrix A
            is preprocessed with QR factorization with FULL (row and
            column) pivoting.
          = 'G'  Computation as with 'F' with an additional estimate of the
            condition number of B, where A=D*B. If A has heavily weighted
            rows, then using this condition number gives too pessimistic
            error bound.
          = 'A': Small singular values are the noise and the matrix is treated
             as numerically rank defficient. The error in the computed
             singular values is bounded by f(m,n)*epsilon*||A||.
             The computed SVD A = U * S * V^* restores A up to
             f(m,n)*epsilon*||A||.
             This gives the procedure the licence to discard (set to zero)
             all singular values below N*epsilon*||A||.
           = 'R': Similar as in 'A'. Rank revealing property of the initial
             QR factorization is used do reveal (using triangular factor)
             a gap sigma_{r+1} < epsilon * sigma_r in which case the
             numerical RANK is declared to be r. The SVD is computed with
             absolute error bounds, but more accurately than with 'A'.

Parameters:

 JOBU
   JOBU is CHARACTER*1
   Specifies whether to compute the columns of U:
   = 'U': N columns of U are returned in the array U.
   = 'F': full set of M left sing. vectors is returned in the array U.
   = 'W': U may be used as workspace of length M*N. See the description of U.
   = 'N': U is not computed.

 JOBV
   JOBV is CHARACTER*1
   Specifies whether to compute the matrix V:
   = 'V': N columns of V are returned in the array V; Jacobi rotations
          are not explicitly accumulated.
   = 'J': N columns of V are returned in the array V, but they are
          computed as the product of Jacobi rotations. This option is
          allowed only if JOBU .NE. 'N', i.e. in computing the full SVD.
   = 'W': V may be used as workspace of length N*N. See the description of V.
```

```
  = 'N': V is not computed.

JOBR
  JOBR is CHARACTER*1
  Specifies the RANGE for the singular values. Issues the licence to
  set to zero small positive singular values if they are outside
  specified range. If A .NE. 0 is scaled so that the largest singular
  value of c*A is around SQRT(BIG), BIG=SLAMCH('O'), then JOBR issues
  the licence to kill columns of A whose norm in c*A is less than
  SQRT(SFMIN) (for JOBR.EQ.'R'), or less than SMALL=SFMIN/EPSLN,
  where SFMIN=SLAMCH('S'), EPSLN=SLAMCH('E').
  = 'N': Do not kill small columns of c*A. This option assumes that
         BLAS and QR factorizations and triangular solvers are
         implemented to work in that range. If the condition of A
         is greater than BIG, use CGESVJ.
  = 'R': RESTRICTED range for sigma(c*A) is [SQRT(SFMIN), SQRT(BIG)]
         (roughly, as described above). This option is recommended.

                         ===========================
  For computing the singular values in the FULL range [SFMIN,BIG]
  use CGESVJ.

JOBT
  JOBT is CHARACTER*1
  If the matrix is square then the procedure may determine to use
  transposed A if A^* seems to be better with respect to convergence.
  If the matrix is not square, JOBT is ignored. This is subject to
  changes in the future.
  The decision is based on two values of entropy over the adjoint
  orbit of A^* * A. See the descriptions of WORK(6) and WORK(7).
  = 'T': transpose if entropy test indicates possibly faster
  convergence of Jacobi process if A^* is taken as input. If A is
  replaced with A^*, then the row pivoting is included automatically.
  = 'N': do not speculate.
         This option can be used to compute only the singular values, or the
         full SVD (U, SIGMA and V). For only one set of singular vectors
         (U or V), the caller should provide both U and V, as one of the
         matrices is used as workspace if the matrix A is transposed.
         The implementer can easily remove this constraint and make the
         code more complicated. See the descriptions of U and V.

JOBP
  JOBP is CHARACTER*1
  Issues the licence to introduce structured perturbations to drown
  denormalized numbers. This licence should be active if the
  denormals are poorly implemented, causing slow computation,
  especially in cases of fast convergence (!). For details see [1,2].
  For the sake of simplicity, this perturbations are included only
  when the full SVD or only the singular values are requested. The
  implementer/user can easily add the perturbation for the cases of
```

```
  computing one set of singular vectors.
  = 'P': introduce perturbation
  = 'N': do not perturb

M
  M is INTEGER
  The number of rows of the input matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the input matrix A. M >= N >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the M-by-N matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

SVA
  SVA is REAL array, dimension (N)
  On exit,
  - For WORK(1)/WORK(2) = ONE: The singular values of A. During the
    computation SVA contains Euclidean column norms of the
    iterated matrices in the array A.
  - For WORK(1) .NE. WORK(2): The singular values of A are
    (WORK(1)/WORK(2)) * SVA(1:N). This factored form is used if
    sigma_max(A) overflows or if small singular values have been
    saved from underflow by scaling the input matrix A.
  - If JOBR='R' then some of the singular values may be returned
    as exact zeros obtained by "set to zero" because they are
    below the numerical rank threshold or are denormalized numbers.

U
  U is COMPLEX array, dimension ( LDU, N )
  If JOBU = 'U', then U contains on exit the M-by-N matrix of
       the left singular vectors.
  If JOBU = 'F', then U contains on exit the M-by-M matrix of
       the left singular vectors, including an ONB
       of the orthogonal complement of the Range(A).
  If JOBU = 'W'  .AND. (JOBV.EQ.'V' .AND. JOBT.EQ.'T' .AND. M.EQ.N),
       then U is used as workspace if the procedure
       replaces A with A^*. In that case, [V] is computed
       in U as left singular vectors of A^* and then
       copied back to the V array. This 'W' option is just
       a reminder to the caller that in this case U is
       reserved as workspace of length N*N.
  If JOBU = 'N'  U is not referenced.
```

```
LDU
  LDU is INTEGER
  The leading dimension of the array U,  LDU >= 1.
  IF  JOBU = 'U' or 'F' or 'W',  then LDU >= M.

V
  V is COMPLEX array, dimension ( LDV, N )
  If JOBV = 'V', 'J' then V contains on exit the N-by-N matrix of
        the right singular vectors;
  If JOBV = 'W', AND (JOBU.EQ.'U' AND JOBT.EQ.'T' AND M.EQ.N),
        then V is used as workspace if the pprocedure
        replaces A with A^*. In that case, [U] is computed
        in V as right singular vectors of A^* and then
        copied back to the U array. This 'W' option is just
        a reminder to the caller that in this case V is
        reserved as workspace of length N*N.
  If JOBV = 'N'  V is not referenced.

LDV
  LDV is INTEGER
  The leading dimension of the array V,  LDV >= 1.
  If JOBV = 'V' or 'J' or 'W', then LDV >= N.

CWORK
  CWORK is COMPLEX array, dimension at least LWORK.

LWORK
  LWORK is INTEGER
  Length of CWORK to confirm proper allocation of workspace.
  LWORK depends on the job:

  1. If only SIGMA is needed ( JOBU.EQ.'N', JOBV.EQ.'N' ) and
   1.1 .. no scaled condition estimate required (JOBE.EQ.'N'):
       LWORK >= 2*N+1. This is the minimal requirement.
       ->> For optimal performance (blocked code) the optimal value
       is LWORK >= N + (N+1)*NB. Here NB is the optimal
       block size for CGEQP3 and CGEQRF.
       In general, optimal LWORK is computed as
       LWORK >= max(N+LWORK(CGEQP3),N+LWORK(CGEQRF)).
   1.2. .. an estimate of the scaled condition number of A is
       required (JOBA='E', or 'G'). In this case, LWORK the minimal
       requirement is LWORK >= N*N + 3*N.
       ->> For optimal performance (blocked code) the optimal value
       is LWORK >= max(N+(N+1)*NB, N*N+3*N).
       In general, the optimal length LWORK is computed as
       LWORK >= max(N+LWORK(CGEQP3),N+LWORK(CGEQRF), N+N*N+LWORK(CPOCON)).

  2. If SIGMA and the right singular vectors are needed (JOBV.EQ.'V'),
       (JOBU.EQ.'N')
       -> the minimal requirement is LWORK >= 3*N.
```

```
      -> For optimal performance, LWORK >= max(N+(N+1)*NB, 3*N,2*N+N*NB),
         where NB is the optimal block size for CGEQP3, CGEQRF, CGELQ,
         CUNMLQ. In general, the optimal length LWORK is computed as
         LWORK >= max(N+LWORK(CGEQP3), N+LWORK(CPOCON), N+LWORK(CGESVJ),
                  N+LWORK(CGELQF), 2*N+LWORK(CGEQRF), N+LWORK(CUNMLQ)).

  3. If SIGMA and the left singular vectors are needed
      -> the minimal requirement is LWORK >= 3*N.
      -> For optimal performance:
      if JOBU.EQ.'U' :: LWORK >= max(3*N, N+(N+1)*NB, 2*N+N*NB),
      where NB is the optimal block size for CGEQP3, CGEQRF, CUNMQR.
      In general, the optimal length LWORK is computed as
      LWORK >= max(N+LWORK(CGEQP3),N+LWORK(CPOCON),
               2*N+LWORK(CGEQRF), N+LWORK(CUNMQR)).

  4. If the full SVD is needed: (JOBU.EQ.'U' or JOBU.EQ.'F') and
   4.1. if JOBV.EQ.'V' the minimal requirement is LWORK >= 5*N+2*N*N.
   4.2. if JOBV.EQ.'J' the minimal requirement is LWORK >= 4*N+N*N.
   In both cases, the allocated CWORK can accomodate blocked runs
   of CGEQP3, CGEQRF, CGELQF, SUNMQR, CUNMLQ.

RWORK
  RWORK is REAL array, dimension at least LRWORK.
  On exit,

  RWORK(1) = Determines the scaling factor SCALE = RWORK(2) / RWORK(1)
  such that SCALE*SVA(1:N) are the computed singular values
  of A. (See the description of SVA().)

  RWORK(2) = See the description of RWORK(1).

  RWORK(3) = SCONDA is an estimate for the condition number of
  column equilibrated A. (If JOBA .EQ. 'E' or 'G')
  SCONDA is an estimate of SQRT(||(R^* * R)^(-1)||_1).
  It is computed using SPOCON. It holds
  N^(-1/4) * SCONDA <= ||R^(-1)||_2 <= N^(1/4) * SCONDA
  where R is the triangular factor from the QRF of A.
  However, if R is truncated and the numerical rank is
  determined to be strictly smaller than N, SCONDA is
  returned as -1, thus indicating that the smallest
  singular values might be lost.

  If full SVD is needed, the following two condition numbers are
  useful for the analysis of the algorithm. They are provied for
  a developer/implementer who is familiar with the details of
  the method.

  RWORK(4) = an estimate of the scaled condition number of the
  triangular factor in the first QR factorization.
```

```
  RWORK(5) = an estimate of the scaled condition number of the
  triangular factor in the second QR factorization.
  The following two parameters are computed if JOBT .EQ. 'T'.
  They are provided for a developer/implementer who is familiar
  with the details of the method.

  RWORK(6) = the entropy of A^* * A :: this is the Shannon entropy
  of diag(A^* * A) / Trace(A^* * A) taken as point in the
  probability simplex.

  RWORK(7) = the entropy of A * A^*. (See the description of RWORK(6).)

LRWORK
  LRWORK is INTEGER
  Length of RWORK to confirm proper allocation of workspace.
  LRWORK depends on the job:

  1. If only singular values are requested i.e. if
     LSAME(JOBU,'N') .AND. LSAME(JOBV,'N') then:
    1.1. If LSAME(JOBT,'T') .OR. LSAME(JOBA,'F') .OR. LSAME(JOBA,'G'),
         then LRWORK = max( 7, N + 2 * M ).
    1.2. Otherwise, LRWORK  = max( 7, 2 * N ).

   2. If singular values with the right singular vectors are requested
      i.e. if
             (LSAME(JOBV,'V').OR.LSAME(JOBV,'J')) .AND.
              .NOT.(LSAME(JOBU,'U').OR.LSAME(JOBU,'F'))
            then:
    2.1. If LSAME(JOBT,'T') .OR. LSAME(JOBA,'F') .OR. LSAME(JOBA,'G'),
            then LRWORK = max( 7, N + 2 * M ).
    2.2. Otherwise, LRWORK  = max( 7, 2 * N ).

   3. If singular values with the left singular vectors are requested, i.e. if
        (LSAME(JOBU,'U').OR.LSAME(JOBU,'F')) .AND.
        .NOT.(LSAME(JOBV,'V').OR.LSAME(JOBV,'J'))
      then:
    3.1. If LSAME(JOBT,'T') .OR. LSAME(JOBA,'F') .OR. LSAME(JOBA,'G'),
          then LRWORK = max( 7, N + 2 * M ).
    3.2. Otherwise, LRWORK  = max( 7, 2 * N ).

   4. If singular values with both the left and the right singular vectors
      are requested, i.e. if
       (LSAME(JOBU,'U').OR.LSAME(JOBU,'F')) .AND.
       (LSAME(JOBV,'V').OR.LSAME(JOBV,'J'))
      then:
    4.1. If LSAME(JOBT,'T') .OR. LSAME(JOBA,'F') .OR. LSAME(JOBA,'G'),
         then LRWORK = max( 7, N + 2 * M ).
    4.2. Otherwise, LRWORK  = max( 7, 2 * N ).

IWORK
```

```
  IWORK is INTEGER array, of dimension:
  If LSAME(JOBA,'F') .OR. LSAME(JOBA,'G'), then
  the dimension of IWORK is max( 3, 2 * N + M ).
  Otherwise, the dimension of IWORK is
  -> max( 3, 2*N ) for full SVD
  -> max( 3, N ) for singular values only or singular
     values with one set of singular vectors (left or right)
  On exit,

  IWORK(1) = the numerical rank determined after the initial
             QR factorization with pivoting. See the descriptions
             of JOBA and JOBR.

  IWORK(2) = the number of the computed nonzero singular values

  IWORK(3) = if nonzero, a warning message:
             If IWORK(3).EQ.1 then some of the column norms of A
             were denormalized floats. The requested high accuracy
             is not warranted by the data.

INFO
  INFO is INTEGER
  < 0  : if INFO = -i, then the i-th argument had an illegal value.
  = 0 :  successfull exit;
  > 0 :  CGEJSV  did not converge in the maximal allowed number
         of sweeps. The computed values may be inaccurate.

  CGEJSV implements a preconditioned Jacobi SVD algorithm. It uses CGEQP3,
  CGEQRF, and CGELQF as preprocessors and preconditioners. Optionally, an
  additional row pivoting can be used as a preprocessor, which in some
  cases results in much higher accuracy. An example is matrix A with the
  structure A = D1 * C * D2, where D1, D2 are arbitrarily ill-conditioned
  diagonal matrices and C is well-conditioned matrix. In that case, complete
  pivoting in the first QR factorizations provides accuracy dependent on the
  condition number of C, and independent of D1, D2. Such higher accuracy is
  not completely understood theoretically, but it works well in practice.
  Further, if A can be written as A = B*D, with well-conditioned B and some
  diagonal D, then the high accuracy is guaranteed, both theoretically and
  in software, independent of D. For more details see [1], [2].

  The computational range for the singular values can be the full range
  ( UNDERFLOW,OVERFLOW ), provided that the machine arithmetic and the BLAS
  & LAPACK routines called by CGEJSV are implemented to work in that range.
  If that is not the case, then the restriction for safe computation with
  the singular values in the range of normalized IEEE numbers is that the
  spectral condition number kappa(A)=sigma_max(A)/sigma_min(A) does not
  overflow. This code (CGEJSV) is best used in this restricted range,
  meaning that singular values of magnitude below ||A||_2 / SLAMCH('O') are
  returned as zeros. See JOBR for details on this.
```

```
Further, this implementation is somewhat slower than the one described
in [1,2] due to replacement of some non-LAPACK components, and because
the choice of some tuning parameters in the iterative part (CGESVJ) is
left to the implementer on a particular machine.

The rank revealing QR factorization (in this code: CGEQP3) should be
implemented as in [3]. We have a new version of CGEQP3 under development
that is more robust than the current one in LAPACK, with a cleaner cut in
rank defficient cases. It will be available in the SIGMA library [4].

If M is much larger than N, it is obvious that the inital QRF with
column pivoting can be preprocessed by the QRF without pivoting. That
well known trick is not used in CGEJSV because in some cases heavy row
weighting can be treated with complete pivoting. The overhead in cases
M much larger than N is then only due to pivoting, but the benefits in
terms of accuracy have prevailed. The implementer/user can incorporate
this extra QRF step easily. The implementer can also improve data movement
(matrix transpose, matrix copy, matrix transposed copy) - this
implementation of CGEJSV uses only the simplest, naive data movement.
```

———-

30.1.3 fortran code

— cgejsv.f —

```
*  ===================================================================
      SUBROUTINE CGEJSV( JOBA, JOBU, JOBV, JOBR, JOBT, JOBP,
     $                   M, N, A, LDA, SVA, U, LDU, V, LDV,
     $                   CWORK, LWORK, RWORK, LRWORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      IMPLICIT    NONE
      INTEGER     INFO, LDA, LDU, LDV, LWORK, LRWORK, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX     A( LDA, * ), U( LDU, * ), V( LDV, * ), CWORK( LWORK )
      REAL        SVA( N ), RWORK( * )
      INTEGER     IWORK( * )
      CHARACTER*1 JOBA, JOBP, JOBR, JOBT, JOBU, JOBV
*     ..
*
```

```
*  ============================================================================
*
*     .. Local Parameters ..
      REAL        ZERO,         ONE
      PARAMETER ( ZERO = 0.0E0, ONE = 1.0E0 )
      COMPLEX     CZERO,                    CONE
      PARAMETER ( CZERO = ( 0.0E0, 0.0E0 ), CONE = ( 1.0E0, 0.0E0 ) )
*     ..
*     .. Local Scalars ..
      COMPLEX CTEMP
      REAL    AAPP,   AAQQ,   AATMAX, AATMIN, BIG,    BIG1,   COND_OK,
     $        CONDR1, CONDR2, ENTRA,  ENTRAT, EPSLN,  MAXPRJ, SCALEM,
     $        SCONDA, SFMIN,  SMALL,  TEMP1,  USCAL1, USCAL2, XSC
      INTEGER IERR,   N1,     NR,     NUMRANK,        p, q,   WARNING
      LOGICAL ALMORT, DEFR,   ERREST, GOSCAL, JRACC,  KILL,   LSVEC,
     $        L2ABER, L2KILL, L2PERT, L2RANK, L2TRAN,
     $        NOSCAL, ROWPIV, RSVEC,  TRANSP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC ABS,  CONJG, ALOG, AMAX1, AMIN1, CMPLX, FLOAT,
     $          MAX0, MIN0, NINT,  SIGN,  SQRT
*     ..
*     .. External Functions ..
      REAL      SLAMCH, SCNRM2
      INTEGER   ISAMAX
      LOGICAL   LSAME
      EXTERNAL  ISAMAX, LSAME, SLAMCH, SCNRM2
*     ..
*     .. External Subroutines ..
      EXTERNAL  CCOPY,  CGELQF, CGEQP3, CGEQRF, CLACPY, CLASCL,
     $          CLASET, CLASSQ, CLASWP, CUNGQR, CUNMLQ,
     $          CUNMQR, CPOCON, SSCAL, CSSCAL, CSWAP,  CTRSM,  XERBLA
*
      EXTERNAL  CGESVJ
*     ..
*
*     Test the input arguments
*
      LSVEC  = LSAME( JOBU, 'U' ) .OR. LSAME( JOBU, 'F' )
      JRACC  = LSAME( JOBV, 'J' )
      RSVEC  = LSAME( JOBV, 'V' ) .OR. JRACC
      ROWPIV = LSAME( JOBA, 'F' ) .OR. LSAME( JOBA, 'G' )
      L2RANK = LSAME( JOBA, 'R' )
      L2ABER = LSAME( JOBA, 'A' )
      ERREST = LSAME( JOBA, 'E' ) .OR. LSAME( JOBA, 'G' )
      L2TRAN = LSAME( JOBT, 'T' )
      L2KILL = LSAME( JOBR, 'R' )
      DEFR   = LSAME( JOBR, 'N' )
      L2PERT = LSAME( JOBP, 'P' )
*
```

```
      IF ( .NOT.(ROWPIV .OR. L2RANK .OR. L2ABER .OR.
     $     ERREST .OR. LSAME( JOBA, 'C' ) )) THEN
         INFO = - 1
      ELSE IF ( .NOT.( LSVEC  .OR. LSAME( JOBU, 'N' ) .OR.
     $                            LSAME( JOBU, 'W' )) ) THEN
         INFO = - 2
      ELSE IF ( .NOT.( RSVEC .OR. LSAME( JOBV, 'N' ) .OR.
     $   LSAME( JOBV, 'W' )) .OR. ( JRACC .AND. (.NOT.LSVEC) ) ) THEN
         INFO = - 3
      ELSE IF ( .NOT. ( L2KILL .OR. DEFR ) )    THEN
         INFO = - 4
      ELSE IF ( .NOT. ( L2TRAN .OR. LSAME( JOBT, 'N' ) ) ) THEN
         INFO = - 5
      ELSE IF ( .NOT. ( L2PERT .OR. LSAME( JOBP, 'N' ) ) ) THEN
         INFO = - 6
      ELSE IF ( M .LT. 0 ) THEN
         INFO = - 7
      ELSE IF ( ( N .LT. 0 ) .OR. ( N .GT. M ) ) THEN
         INFO = - 8
      ELSE IF ( LDA .LT. M ) THEN
         INFO = - 10
      ELSE IF ( LSVEC .AND. ( LDU .LT. M ) ) THEN
         INFO = - 13
      ELSE IF ( RSVEC .AND. ( LDV .LT. N ) ) THEN
         INFO = - 15
      ELSE IF ( (.NOT.(LSVEC .OR. RSVEC .OR. ERREST).AND.
     $                           (LWORK .LT. 2*N+1)) .OR.
     $ (.NOT.(LSVEC .OR. RSVEC) .AND. ERREST .AND.
     $                         (LWORK .LT. N*N+3*N)) .OR.
     $ (LSVEC .AND. (.NOT.RSVEC) .AND. (LWORK .LT. 3*N))
     $ .OR.
     $ (RSVEC .AND. (.NOT.LSVEC) .AND. (LWORK .LT. 3*N))
     $ .OR.
     $ (LSVEC .AND. RSVEC .AND. (.NOT.JRACC) .AND.
     $                          (LWORK.LT.5*N+2*N*N))
     $ .OR. (LSVEC .AND. RSVEC .AND. JRACC .AND.
     $                          LWORK.LT.4*N+N*N))
     $   THEN
         INFO = - 17
      ELSE IF ( LRWORK.LT. MAX0(N+2*M,7)) THEN
         INFO = -19
      ELSE
*        #:)
         INFO = 0
      END IF
*
      IF ( INFO .NE. 0 ) THEN
*       #:(
         CALL XERBLA( 'CGEJSV', - INFO )
         RETURN
```

```
      END IF
*
*     Quick return for void matrix (Y3K safe)
* #:)
      IF ( ( M .EQ. 0 ) .OR. ( N .EQ. 0 ) ) RETURN
*
*     Determine whether the matrix U should be M x N or M x M
*
      IF ( LSVEC ) THEN
         N1 = N
         IF ( LSAME( JOBU, 'F' ) ) N1 = M
      END IF
*
*     Set numerical parameters
*
*!    NOTE: Make sure SLAMCH() does not fail on the target architecture.
*
      EPSLN = SLAMCH('Epsilon')
      SFMIN = SLAMCH('SafeMinimum')
      SMALL = SFMIN / EPSLN
      BIG   = SLAMCH('O')
*     BIG   = ONE / SFMIN
*
*     Initialize SVA(1:N) = diag( ||A e_i||_2 )_1^N
*
*(!)  If necessary, scale SVA() to protect the largest norm from
*     overflow. It is possible that this scaling pushes the smallest
*     column norm left from the underflow threshold (extreme case).
*
      SCALEM  = ONE / SQRT(FLOAT(M)*FLOAT(N))
      NOSCAL  = .TRUE.
      GOSCAL  = .TRUE.
      DO 1874 p = 1, N
         AAPP = ZERO
         AAQQ = ONE
         CALL CLASSQ( M, A(1,p), 1, AAPP, AAQQ )
         IF ( AAPP .GT. BIG ) THEN
            INFO = - 9
            CALL XERBLA( 'CGEJSV', -INFO )
            RETURN
         END IF
         AAQQ = SQRT(AAQQ)
         IF ( ( AAPP .LT. (BIG / AAQQ) ) .AND. NOSCAL  ) THEN
            SVA(p)  = AAPP * AAQQ
         ELSE
            NOSCAL  = .FALSE.
            SVA(p)  = AAPP * ( AAQQ * SCALEM )
            IF ( GOSCAL ) THEN
               GOSCAL = .FALSE.
               CALL SSCAL( p-1, SCALEM, SVA, 1 )
```

```
            END IF
         END IF
 1874 CONTINUE
*
      IF ( NOSCAL ) SCALEM = ONE
*
      AAPP = ZERO
      AAQQ = BIG
      DO 4781 p = 1, N
         AAPP = AMAX1( AAPP, SVA(p) )
         IF ( SVA(p) .NE. ZERO ) AAQQ = AMIN1( AAQQ, SVA(p) )
 4781 CONTINUE
*
*     Quick return for zero M x N matrix
* #:)
      IF ( AAPP .EQ. ZERO ) THEN
         IF ( LSVEC ) CALL CLASET( 'G', M, N1, CZERO, CONE, U, LDU )
         IF ( RSVEC ) CALL CLASET( 'G', N, N,  CZERO, CONE, V, LDV )
         RWORK(1) = ONE
         RWORK(2) = ONE
         IF ( ERREST ) RWORK(3) = ONE
         IF ( LSVEC .AND. RSVEC ) THEN
            RWORK(4) = ONE
            RWORK(5) = ONE
         END IF
         IF ( L2TRAN ) THEN
            RWORK(6) = ZERO
            RWORK(7) = ZERO
         END IF
         IWORK(1) = 0
         IWORK(2) = 0
         IWORK(3) = 0
         RETURN
      END IF
*
*     Issue warning if denormalized column norms detected. Override the
*     high relative accuracy request. Issue licence to kill columns
*     (set them to zero) whose norm is less than sigma_max / BIG (roughly).
* #:(
      WARNING = 0
      IF ( AAQQ .LE. SFMIN ) THEN
         L2RANK = .TRUE.
         L2KILL = .TRUE.
         WARNING = 1
      END IF
*
*     Quick return for one-column matrix
* #:)
      IF ( N .EQ. 1 ) THEN
*
```

```
         IF ( LSVEC ) THEN
            CALL CLASCL( 'G',0,0,SVA(1),SCALEM, M,1,A(1,1),LDA,IERR )
            CALL CLACPY( 'A', M, 1, A, LDA, U, LDU )
*           computing all M left singular vectors of the M x 1 matrix
            IF ( N1 .NE. N  ) THEN
              CALL CGEQRF( M, N, U,LDU, CWORK, CWORK(N+1),LWORK-N,IERR )
              CALL CUNGQR( M,N1,1, U,LDU,CWORK,CWORK(N+1),LWORK-N,IERR )
              CALL CCOPY( M, A(1,1), 1, U(1,1), 1 )
            END IF
         END IF
         IF ( RSVEC ) THEN
             V(1,1) = CONE
         END IF
         IF ( SVA(1) .LT. (BIG*SCALEM) ) THEN
            SVA(1)  = SVA(1) / SCALEM
            SCALEM  = ONE
         END IF
         RWORK(1) = ONE / SCALEM
         RWORK(2) = ONE
         IF ( SVA(1) .NE. ZERO ) THEN
            IWORK(1) = 1
            IF ( ( SVA(1) / SCALEM) .GE. SFMIN ) THEN
               IWORK(2) = 1
            ELSE
               IWORK(2) = 0
            END IF
         ELSE
            IWORK(1) = 0
            IWORK(2) = 0
         END IF
         IWORK(3) = 0
         IF ( ERREST ) RWORK(3) = ONE
         IF ( LSVEC .AND. RSVEC ) THEN
            RWORK(4) = ONE
            RWORK(5) = ONE
         END IF
         IF ( L2TRAN ) THEN
            RWORK(6) = ZERO
            RWORK(7) = ZERO
         END IF
         RETURN
*
      END IF
*
      TRANSP = .FALSE.
      L2TRAN = L2TRAN .AND. ( M .EQ. N )
*
      AATMAX = -ONE
      AATMIN =  BIG
      IF ( ROWPIV .OR. L2TRAN ) THEN
```

```
*
*     Compute the row norms, needed to determine row pivoting sequence
*     (in the case of heavily row weighted A, row pivoting is strongly
*     advised) and to collect information needed to compare the
*     structures of A * A^* and A^* * A (in the case L2TRAN.EQ..TRUE.).
*
         IF ( L2TRAN ) THEN
            DO 1950 p = 1, M
               XSC   = ZERO
               TEMP1 = ONE
               CALL CLASSQ( N, A(p,1), LDA, XSC, TEMP1 )
*              CLASSQ gets both the ell_2 and the ell_infinity norm
*              in one pass through the vector
               RWORK(M+N+p)  = XSC * SCALEM
               RWORK(N+p)    = XSC * (SCALEM*SQRT(TEMP1))
               AATMAX = AMAX1( AATMAX, RWORK(N+p) )
               IF (RWORK(N+p) .NE. ZERO)
     $            AATMIN = AMIN1(AATMIN,RWORK(N+p))
 1950       CONTINUE
         ELSE
            DO 1904 p = 1, M
               RWORK(M+N+p) = SCALEM*ABS( A(p,ISAMAX(N,A(p,1),LDA)) )
               AATMAX = AMAX1( AATMAX, RWORK(M+N+p) )
               AATMIN = AMIN1( AATMIN, RWORK(M+N+p) )
 1904       CONTINUE
         END IF
*
      END IF
*
*     For square matrix A try to determine whether A^*  would be  better
*     input for the preconditioned Jacobi SVD, with faster convergence.
*     The decision is based on an O(N) function of the vector of column
*     and row norms of A, based on the Shannon entropy. This should give
*     the right choice in most cases when the difference actually matters.
*     It may fail and pick the slower converging side.
*
      ENTRA  = ZERO
      ENTRAT = ZERO
      IF ( L2TRAN ) THEN
*
         XSC   = ZERO
         TEMP1 = ONE
         CALL CLASSQ( N, SVA, 1, XSC, TEMP1 )
         TEMP1 = ONE / TEMP1
*
         ENTRA = ZERO
         DO 1113 p = 1, N
            BIG1  = ( ( SVA(p) / XSC )**2 ) * TEMP1
            IF ( BIG1 .NE. ZERO ) ENTRA = ENTRA + BIG1 * ALOG(BIG1)
 1113    CONTINUE
```

```
         ENTRA = - ENTRA / ALOG(FLOAT(N))
*
*        Now, SVA().^2/Trace(A^* * A) is a point in the probability simplex.
*        It is derived from the diagonal of  A^* * A.  Do the same with the
*        diagonal of A * A^*, compute the entropy of the corresponding
*        probability distribution. Note that A * A^* and A^* * A have the
*        same trace.
*
         ENTRAT = ZERO
         DO 1114 p = N+1, N+M
            BIG1 = ( ( RWORK(p) / XSC )**2 ) * TEMP1
            IF ( BIG1 .NE. ZERO ) ENTRAT = ENTRAT + BIG1 * ALOG(BIG1)
 1114    CONTINUE
         ENTRAT = - ENTRAT / ALOG(FLOAT(M))
*
*        Analyze the entropies and decide A or A^*. Smaller entropy
*        usually means better input for the algorithm.
*
         TRANSP = ( ENTRAT .LT. ENTRA )
         TRANSP = .TRUE.
*
*        If A^* is better than A, take the adjoint of A.
*
         IF ( TRANSP ) THEN
*           In an optimal implementation, this trivial transpose
*           should be replaced with faster transpose.
            DO 1115 p = 1, N - 1
               A(p,p) = CONJG(A(p,p))
               DO 1116 q = p + 1, N
                   CTEMP = CONJG(A(q,p))
                  A(q,p) = CONJG(A(p,q))
                  A(p,q) = CTEMP
 1116          CONTINUE
 1115       CONTINUE
            A(N,N) = CONJG(A(N,N))
            DO 1117 p = 1, N
               RWORK(M+N+p) = SVA(p)
               SVA(p)      = RWORK(N+p)
*              previously computed row 2-norms are now column 2-norms
*              of the transposed matrix
 1117       CONTINUE
            TEMP1  = AAPP
            AAPP   = AATMAX
            AATMAX = TEMP1
            TEMP1  = AAQQ
            AAQQ   = AATMIN
            AATMIN = TEMP1
            KILL   = LSVEC
            LSVEC  = RSVEC
            RSVEC  = KILL
```

```
            IF ( LSVEC ) N1 = N
*
            ROWPIV = .TRUE.
         END IF
*
      END IF
*     END IF L2TRAN
*
*     Scale the matrix so that its maximal singular value remains less
*     than SQRT(BIG) -- the matrix is scaled so that its maximal column
*     has Euclidean norm equal to SQRT(BIG/N). The only reason to keep
*     SQRT(BIG) instead of BIG is the fact that CGEJSV uses LAPACK and
*     BLAS routines that, in some implementations, are not capable of
*     working in the full interval [SFMIN,BIG] and that they may provoke
*     overflows in the intermediate results. If the singular values spread
*     from SFMIN to BIG, then CGESVJ will compute them. So, in that case,
*     one should use CGESVJ instead of CGEJSV.
*
      BIG1   = SQRT( BIG )
      TEMP1  = SQRT( BIG / FLOAT(N) )
*
      CALL CLASCL( 'G', 0, 0, AAPP, TEMP1, N, 1, SVA, N, IERR )
      IF ( AAQQ .GT. (AAPP * SFMIN) ) THEN
          AAQQ = ( AAQQ / AAPP ) * TEMP1
      ELSE
          AAQQ = ( AAQQ * TEMP1 ) / AAPP
      END IF
      TEMP1 = TEMP1 * SCALEM
      CALL CLASCL( 'G', 0, 0, AAPP, TEMP1, M, N, A, LDA, IERR )
*
*     To undo scaling at the end of this procedure, multiply the
*     computed singular values with USCAL2 / USCAL1.
*
      USCAL1 = TEMP1
      USCAL2 = AAPP
*
      IF ( L2KILL ) THEN
*        L2KILL enforces computation of nonzero singular values in
*        the restricted range of condition number of the initial A,
*        sigma_max(A) / sigma_min(A) approx. SQRT(BIG)/SQRT(SFMIN).
         XSC = SQRT( SFMIN )
      ELSE
         XSC = SMALL
*
*        Now, if the condition number of A is too big,
*        sigma_max(A) / sigma_min(A) .GT. SQRT(BIG/N) * EPSLN / SFMIN,
*        as a precaution measure, the full SVD is computed using CGESVJ
*        with accumulated Jacobi rotations. This provides numerically
*        more robust computation, at the cost of slightly increased run
*        time. Depending on the concrete implementation of BLAS and LAPACK
```

```
*        (i.e. how they behave in presence of extreme ill-conditioning) the
*        implementor may decide to remove this switch.
         IF ( ( AAQQ.LT.SQRT(SFMIN) ) .AND. LSVEC .AND. RSVEC ) THEN
            JRACC = .TRUE.
         END IF
*
      END IF
      IF ( AAQQ .LT. XSC ) THEN
         DO 700 p = 1, N
            IF ( SVA(p) .LT. XSC ) THEN
               CALL CLASET( 'A', M, 1, CZERO, CZERO, A(1,p), LDA )
               SVA(p) = ZERO
            END IF
 700     CONTINUE
      END IF
*
*     Preconditioning using QR factorization with pivoting
*
      IF ( ROWPIV ) THEN
*        Optional row permutation (Bjoerck row pivoting):
*        A result by Cox and Higham shows that the Bjoerck's
*        row pivoting combined with standard column pivoting
*        has similar effect as Powell-Reid complete pivoting.
*        The ell-infinity norms of A are made nonincreasing.
         DO 1952 p = 1, M - 1
            q = ISAMAX( M-p+1, RWORK(M+N+p), 1 ) + p - 1
            IWORK(2*N+p) = q
            IF ( p .NE. q ) THEN
               TEMP1        = RWORK(M+N+p)
               RWORK(M+N+p) = RWORK(M+N+q)
               RWORK(M+N+q) = TEMP1
            END IF
 1952    CONTINUE
         CALL CLASWP( N, A, LDA, 1, M-1, IWORK(2*N+1), 1 )
      END IF
*
*     End of the preparation phase (scaling, optional sorting and
*     transposing, optional flushing of small columns).
*
*     Preconditioning
*
*     If the full SVD is needed, the right singular vectors are computed
*     from a matrix equation, and for that we need theoretical analysis
*     of the Businger-Golub pivoting. So we use CGEQP3 as the first RR QRF.
*     In all other cases the first RR QRF can be chosen by other criteria
*     (eg speed by replacing global with restricted window pivoting, such
*     as in xGEQPX from TOMS # 782). Good results will be obtained using
*     xGEQPX with properly (!) chosen numerical parameters.
*     Any improvement of CGEQP3 improves overal performance of CGEJSV.
*
```

```
*     A * P1 = Q1 * [ R1^* 0]^*:
      DO 1963 p = 1, N
*        .. all columns are free columns
         IWORK(p) = 0
 1963 CONTINUE
      CALL CGEQP3( M, N, A, LDA, IWORK, CWORK, CWORK(N+1), LWORK-N,
     $             RWORK, IERR )
*
*     The upper triangular matrix R1 from the first QRF is inspected for
*     rank deficiency and possibilities for deflation, or possible
*     ill-conditioning. Depending on the user specified flag L2RANK,
*     the procedure explores possibilities to reduce the numerical
*     rank by inspecting the computed upper triangular factor. If
*     L2RANK or L2ABER are up, then CGEJSV will compute the SVD of
*     A + dA, where ||dA|| <= f(M,N)*EPSLN.
*
      NR = 1
      IF ( L2ABER ) THEN
*        Standard absolute error bound suffices. All sigma_i with
*        sigma_i < N*EPSLN*||A|| are flushed to zero. This is an
*        agressive enforcement of lower numerical rank by introducing a
*        backward error of the order of N*EPSLN*||A||.
         TEMP1 = SQRT(FLOAT(N))*EPSLN
         DO 3001 p = 2, N
            IF ( ABS(A(p,p)) .GE. (TEMP1*ABS(A(1,1))) ) THEN
               NR = NR + 1
            ELSE
               GO TO 3002
            END IF
 3001    CONTINUE
 3002    CONTINUE
      ELSE IF ( L2RANK ) THEN
*        .. similarly as above, only slightly more gentle (less agressive).
*        Sudden drop on the diagonal of R1 is used as the criterion for
*        close-to-rank-defficient.
         TEMP1 = SQRT(SFMIN)
         DO 3401 p = 2, N
            IF ( ( ABS(A(p,p)) .LT. (EPSLN*ABS(A(p-1,p-1))) ) .OR.
     $           ( ABS(A(p,p)) .LT. SMALL ) .OR.
     $           ( L2KILL .AND. (ABS(A(p,p)) .LT. TEMP1) ) ) GO TO 3402
            NR = NR + 1
 3401    CONTINUE
 3402    CONTINUE
*
      ELSE
*        The goal is high relative accuracy. However, if the matrix
*        has high scaled condition number the relative accuracy is in
*        general not feasible. Later on, a condition number estimator
*        will be deployed to estimate the scaled condition number.
*        Here we just remove the underflowed part of the triangular
```

```
*        factor. This prevents the situation in which the code is
*        working hard to get the accuracy not warranted by the data.
         TEMP1  = SQRT(SFMIN)
         DO 3301 p = 2, N
            IF ( ( ABS(A(p,p)) .LT. SMALL ) .OR.
     $           ( L2KILL .AND. (ABS(A(p,p)) .LT. TEMP1) ) ) GO TO 3302
            NR = NR + 1
 3301    CONTINUE
 3302    CONTINUE
*
      END IF
*
      ALMORT = .FALSE.
      IF ( NR .EQ. N ) THEN
         MAXPRJ = ONE
         DO 3051 p = 2, N
            TEMP1  = ABS(A(p,p)) / SVA(IWORK(p))
            MAXPRJ = AMIN1( MAXPRJ, TEMP1 )
 3051    CONTINUE
         IF ( MAXPRJ**2 .GE. ONE - FLOAT(N)*EPSLN ) ALMORT = .TRUE.
      END IF
*
*
      SCONDA = - ONE
      CONDR1 = - ONE
      CONDR2 = - ONE
*
      IF ( ERREST ) THEN
         IF ( N .EQ. NR ) THEN
            IF ( RSVEC ) THEN
*              .. V is available as workspace
               CALL CLACPY( 'U', N, N, A, LDA, V, LDV )
               DO 3053 p = 1, N
                  TEMP1 = SVA(IWORK(p))
                  CALL CSSCAL( p, ONE/TEMP1, V(1,p), 1 )
 3053          CONTINUE
               CALL CPOCON( 'U', N, V, LDV, ONE, TEMP1,
     $              CWORK(N+1), RWORK, IERR )
*
            ELSE IF ( LSVEC ) THEN
*              .. U is available as workspace
               CALL CLACPY( 'U', N, N, A, LDA, U, LDU )
               DO 3054 p = 1, N
                  TEMP1 = SVA(IWORK(p))
                  CALL CSSCAL( p, ONE/TEMP1, U(1,p), 1 )
 3054          CONTINUE
               CALL CPOCON( 'U', N, U, LDU, ONE, TEMP1,
     $              CWORK(N+1), RWORK, IERR )
            ELSE
               CALL CLACPY( 'U', N, N, A, LDA, CWORK(N+1), N )
```

```
               DO 3052 p = 1, N
                  TEMP1 = SVA(IWORK(p))
                  CALL CSSCAL( p, ONE/TEMP1, CWORK(N+(p-1)*N+1), 1 )
 3052          CONTINUE
*           .. the columns of R are scaled to have unit Euclidean lengths.
               CALL CPOCON( 'U', N, CWORK(N+1), N, ONE, TEMP1,
     $              CWORK(N+N*N+1), RWORK, IERR )
*
            END IF
            SCONDA = ONE / SQRT(TEMP1)
*           SCONDA is an estimate of SQRT(||(R^* * R)^(-1)||_1).
*           N^(-1/4) * SCONDA <= ||R^(-1)||_2 <= N^(1/4) * SCONDA
         ELSE
            SCONDA = - ONE
         END IF
      END IF
*
      L2PERT = L2PERT .AND. ( ABS( A(1,1)/A(NR,NR) ) .GT. SQRT(BIG1) )
*     If there is no violent scaling, artificial perturbation is not needed.
*
*     Phase 3:
*
      IF ( .NOT. ( RSVEC .OR. LSVEC ) ) THEN
*
*         Singular Values only
*
*         .. transpose A(1:NR,1:N)
         DO 1946 p = 1, MIN0( N-1, NR )
            CALL CCOPY( N-p, A(p,p+1), LDA, A(p+1,p), 1 )
            CALL CLACGV( N-p+1, A(p,p), 1 )
 1946    CONTINUE
         IF ( NR .EQ. N ) A(N,N) = CONJG(A(N,N))
*
         The following two DO-loops introduce small relative perturbation
*        into the strict upper triangle of the lower triangular matrix.
*        Small entries below the main diagonal are also changed.
*        This modification is useful if the computing environment does not
*        provide/allow FLUSH TO ZERO underflow, for it prevents many
*        annoying denormalized numbers in case of strongly scaled matrices.
*        The perturbation is structured so that it does not introduce any
*        new perturbation of the singular values, and it does not destroy
*        the job done by the preconditioner.
*        The licence for this perturbation is in the variable L2PERT, which
*        should be .FALSE. if FLUSH TO ZERO underflow is active.
*
         IF ( .NOT. ALMORT ) THEN
*
            IF ( L2PERT ) THEN
*              XSC = SQRT(SMALL)
               XSC = EPSLN / FLOAT(N)
```

```
               DO 4947 q = 1, NR
                  CTEMP = CMPLX(XSC*ABS(A(q,q)),ZERO)
                  DO 4949 p = 1, N
                     IF ( ( (p.GT.q) .AND. (ABS(A(p,q)).LE.TEMP1) )
     $                    .OR. ( p .LT. q ) )
*     $                     A(p,q) = TEMP1 * ( A(p,q) / ABS(A(p,q)) )
     $                     A(p,q) = CTEMP
 4949             CONTINUE
 4947          CONTINUE
            ELSE
               CALL CLASET( 'U', NR-1,NR-1, CZERO,CZERO, A(1,2),LDA )
            END IF
*
*            .. second preconditioning using the QR factorization
*
            CALL CGEQRF( N,NR, A,LDA, CWORK, CWORK(N+1),LWORK-N, IERR )
*
*           .. and transpose upper to lower triangular
            DO 1948 p = 1, NR - 1
               CALL CCOPY( NR-p, A(p,p+1), LDA, A(p+1,p), 1 )
               CALL CLACGV( NR-p+1, A(p,p), 1 )
 1948       CONTINUE
*
         END IF
*
*           Row-cyclic Jacobi SVD algorithm with column pivoting
*
*           .. again some perturbation (a "background noise") is added
*           to drown denormals
            IF ( L2PERT ) THEN
*              XSC = SQRT(SMALL)
               XSC = EPSLN / FLOAT(N)
               DO 1947 q = 1, NR
                  CTEMP = CMPLX(XSC*ABS(A(q,q)),ZERO)
                  DO 1949 p = 1, NR
                     IF ( ( (p.GT.q) .AND. (ABS(A(p,q)).LE.TEMP1) )
     $                       .OR. ( p .LT. q ) )
*     $                    A(p,q) = TEMP1 * ( A(p,q) / ABS(A(p,q)) )
     $                    A(p,q) = CTEMP
 1949             CONTINUE
 1947          CONTINUE
            ELSE
               CALL CLASET( 'U', NR-1, NR-1, CZERO, CZERO, A(1,2), LDA )
            END IF
*
*           .. and one-sided Jacobi rotations are started on a lower
*           triangular matrix (plus perturbation which is ignored in
*           the part which destroys triangular form (confusing?!))
*
            CALL CGESVJ( 'L', 'NoU', 'NoV', NR, NR, A, LDA, SVA,
```

```
     $                N, V, LDV, CWORK, LWORK, RWORK, LRWORK, INFO )
*
            SCALEM  = RWORK(1)
            NUMRANK = NINT(RWORK(2))
*
*
      ELSE IF ( RSVEC .AND. ( .NOT. LSVEC ) ) THEN
*
*        -> Singular Values and Right Singular Vectors <-
*
         IF ( ALMORT ) THEN
*
*           .. in this case NR equals N
            DO 1998 p = 1, NR
               CALL CCOPY( N-p+1, A(p,p), LDA, V(p,p), 1 )
               CALL CLACGV( N-p+1, V(p,p), 1 )
 1998       CONTINUE
            CALL CLASET( 'Upper', NR-1,NR-1, CZERO, CZERO, V(1,2), LDV )
*
            CALL CGESVJ( 'L','U','N', N, NR, V,LDV, SVA, NR, A,LDA,
     $                   CWORK, LWORK, RWORK, LRWORK, INFO )
            SCALEM  = RWORK(1)
            NUMRANK = NINT(RWORK(2))

         ELSE
*
*        .. two more QR factorizations ( one QRF is not enough, two require
*        accumulated product of Jacobi rotations, three are perfect )
*
            CALL CLASET( 'Lower', NR-1,NR-1, CZERO, CZERO, A(2,1), LDA )
            CALL CGELQF( NR,N, A, LDA, CWORK, CWORK(N+1), LWORK-N, IERR)
            CALL CLACPY( 'Lower', NR, NR, A, LDA, V, LDV )
            CALL CLASET( 'Upper', NR-1,NR-1, CZERO, CZERO, V(1,2), LDV )
            CALL CGEQRF( NR, NR, V, LDV, CWORK(N+1), CWORK(2*N+1),
     $                   LWORK-2*N, IERR )
            DO 8998 p = 1, NR
               CALL CCOPY( NR-p+1, V(p,p), LDV, V(p,p), 1 )
               CALL CLACGV( NR-p+1, V(p,p), 1 )
 8998       CONTINUE
            CALL CLASET( 'Upper', NR-1, NR-1, ZERO, ZERO, V(1,2), LDV )
*
            CALL CGESVJ( 'Lower', 'U','N', NR, NR, V,LDV, SVA, NR, U,
     $                   LDU, CWORK(N+1), LWORK-N, RWORK, LRWORK, INFO )
            SCALEM  = RWORK(1)
            NUMRANK = NINT(RWORK(2))
            IF ( NR .LT. N ) THEN
               CALL CLASET( 'A',N-NR, NR, CZERO,CZERO, V(NR+1,1),  LDV )
               CALL CLASET( 'A',NR, N-NR, CZERO,CZERO, V(1,NR+1),  LDV )
               CALL CLASET( 'A',N-NR,N-NR,CZERO,CONE, V(NR+1,NR+1),LDV )
            END IF
```

```
*
         CALL CUNMLQ( 'Left', 'C', N, N, NR, A, LDA, CWORK,
     $               V, LDV, CWORK(N+1), LWORK-N, IERR )
*
         END IF
*
         DO 8991 p = 1, N
            CALL CCOPY( N, V(p,1), LDV, A(IWORK(p),1), LDA )
 8991    CONTINUE
         CALL CLACPY( 'All', N, N, A, LDA, V, LDV )
*
         IF ( TRANSP ) THEN
            CALL CLACPY( 'All', N, N, V, LDV, U, LDU )
         END IF
*
      ELSE IF ( LSVEC .AND. ( .NOT. RSVEC ) ) THEN
*
*        .. Singular Values and Left Singular Vectors                 ..
*
*        .. second preconditioning step to avoid need to accumulate
*        Jacobi rotations in the Jacobi iterations.
         DO 1965 p = 1, NR
            CALL CCOPY( N-p+1, A(p,p), LDA, U(p,p), 1 )
            CALL CLACGV( N-p+1, U(p,p), 1 )
 1965    CONTINUE
         CALL CLASET( 'Upper', NR-1, NR-1, CZERO, CZERO, U(1,2), LDU )
*
         CALL CGEQRF( N, NR, U, LDU, CWORK(N+1), CWORK(2*N+1),
     $              LWORK-2*N, IERR )
*
         DO 1967 p = 1, NR - 1
            CALL CCOPY( NR-p, U(p,p+1), LDU, U(p+1,p), 1 )
            CALL CLACGV( N-p+1, U(p,p), 1 )
 1967    CONTINUE
         CALL CLASET( 'Upper', NR-1, NR-1, CZERO, CZERO, U(1,2), LDU )
*
         CALL CGESVJ( 'Lower', 'U', 'N', NR,NR, U, LDU, SVA, NR, A,
     $        LDA, CWORK(N+1), LWORK-N, RWORK, LRWORK, INFO )
         SCALEM  = RWORK(1)
         NUMRANK = NINT(RWORK(2))
*
         IF ( NR .LT. M ) THEN
            CALL CLASET( 'A',  M-NR, NR,CZERO, CZERO, U(NR+1,1), LDU )
            IF ( NR .LT. N1 ) THEN
               CALL CLASET( 'A',NR, N1-NR, CZERO, CZERO, U(1,NR+1),LDU )
               CALL CLASET( 'A',M-NR,N1-NR,CZERO,CONE,U(NR+1,NR+1),LDU )
            END IF
         END IF
*
         CALL CUNMQR( 'Left', 'No Tr', M, N1, N, A, LDA, CWORK, U,
```

```
     $               LDU, CWORK(N+1), LWORK-N, IERR )
*
         IF ( ROWPIV )
     $       CALL CLASWP( N1, U, LDU, 1, M-1, IWORK(2*N+1), -1 )
*
         DO 1974 p = 1, N1
            XSC = ONE / SCNRM2( M, U(1,p), 1 )
            CALL CSSCAL( M, XSC, U(1,p), 1 )
 1974    CONTINUE
*
         IF ( TRANSP ) THEN
            CALL CLACPY( 'All', N, N, U, LDU, V, LDV )
         END IF
*
      ELSE
*
*        .. Full SVD ..
*
         IF ( .NOT. JRACC ) THEN
*
         IF ( .NOT. ALMORT ) THEN
*
*           Second Preconditioning Step (QRF [with pivoting])
*           Note that the composition of TRANSPOSE, QRF and TRANSPOSE is
*           equivalent to an LQF CALL. Since in many libraries the QRF
*           seems to be better optimized than the LQF, we do explicit
*           transpose and use the QRF. This is subject to changes in an
*           optimized implementation of CGEJSV.
*
            DO 1968 p = 1, NR
               CALL CCOPY( N-p+1, A(p,p), LDA, V(p,p), 1 )
               CALL CLACGV( N-p+1, V(p,p), 1 )
 1968       CONTINUE
*
*           .. the following two loops perturb small entries to avoid
*           denormals in the second QR factorization, where they are
*           as good as zeros. This is done to avoid painfully slow
*           computation with denormals. The relative size of the perturbation
*           is a parameter that can be changed by the implementer.
*           This perturbation device will be obsolete on machines with
*           properly implemented arithmetic.
*           To switch it off, set L2PERT=.FALSE. To remove it from  the
*           code, remove the action under L2PERT=.TRUE., leave the ELSE part.
*           The following two loops should be blocked and fused with the
*           transposed copy above.
*
            IF ( L2PERT ) THEN
               XSC = SQRT(SMALL)
               DO 2969 q = 1, NR
                  CTEMP = CMPLX(XSC*ABS( V(q,q) ),ZERO)
```

```
                 DO 2968 p = 1, N
                    IF ( ( p .GT. q ) .AND. ( ABS(V(p,q)) .LE. TEMP1 )
     $                  .OR. ( p .LT. q ) )
*     $                  V(p,q) = TEMP1 * ( V(p,q) / ABS(V(p,q)) )
     $                 V(p,q) = CTEMP
                    IF ( p .LT. q ) V(p,q) = - V(p,q)
 2968            CONTINUE
 2969         CONTINUE
           ELSE
              CALL CLASET( 'U', NR-1, NR-1, CZERO, CZERO, V(1,2), LDV )
           END IF
*
*          Estimate the row scaled condition number of R1
*          (If R1 is rectangular, N > NR, then the condition number
*          of the leading NR x NR submatrix is estimated.)
*
           CALL CLACPY( 'L', NR, NR, V, LDV, CWORK(2*N+1), NR )
           DO 3950 p = 1, NR
              TEMP1 = SCNRM2(NR-p+1,CWORK(2*N+(p-1)*NR+p),1)
              CALL CSSCAL(NR-p+1,ONE/TEMP1,CWORK(2*N+(p-1)*NR+p),1)
 3950      CONTINUE
           CALL CPOCON('Lower',NR,CWORK(2*N+1),NR,ONE,TEMP1,
     $                 CWORK(2*N+NR*NR+1),RWORK,IERR)
           CONDR1 = ONE / SQRT(TEMP1)
*          .. here need a second oppinion on the condition number
*          .. then assume worst case scenario
*          R1 is OK for inverse <=> CONDR1 .LT. FLOAT(N)
*          more conservative    <=> CONDR1 .LT. SQRT(FLOAT(N))
*
           COND_OK = SQRT(SQRT(FLOAT(NR)))
*[TP]      COND_OK is a tuning parameter.
*
           IF ( CONDR1 .LT. COND_OK ) THEN
*             .. the second QRF without pivoting. Note: in an optimized
*             implementation, this QRF should be implemented as the QRF
*             of a lower triangular matrix.
*             R1^* = Q2 * R2
              CALL CGEQRF( N, NR, V, LDV, CWORK(N+1), CWORK(2*N+1),
     $             LWORK-2*N, IERR )
*
              IF ( L2PERT ) THEN
                 XSC = SQRT(SMALL)/EPSLN
                 DO 3959 p = 2, NR
                    DO 3958 q = 1, p - 1
                       CTEMP=CMPLX(XSC*AMIN1(ABS(V(p,p)),ABS(V(q,q))),
     $                           ZERO)
                       IF ( ABS(V(q,p)) .LE. TEMP1 )
*     $                    V(q,p) = TEMP1 * ( V(q,p) / ABS(V(q,p)) )
     $                    V(q,p) = CTEMP
 3958               CONTINUE
```

```
 3959            CONTINUE
               END IF
*
               IF ( NR .NE. N )
     $         CALL CLACPY( 'A', N, NR, V, LDV, CWORK(2*N+1), N )
*              .. save ...
*
*           .. this transposed copy should be better than naive
               DO 1969 p = 1, NR - 1
                  CALL CCOPY( NR-p, V(p,p+1), LDV, V(p+1,p), 1 )
                  CALL CLACGV(NR-p+1, V(p,p), 1 )
 1969          CONTINUE
               V(NR,NR)=CONJG(V(NR,NR))
*
               CONDR2 = CONDR1
*
            ELSE
*
               .. ill-conditioned case: second QRF with pivoting
*              Note that windowed pivoting would be equaly good
*              numerically, and more run-time efficient. So, in
*              an optimal implementation, the next call to CGEQP3
*              should be replaced with eg. CALL CGEQPX (ACM TOMS #782)
*              with properly (carefully) chosen parameters.
*
*              R1^* * P2 = Q2 * R2
               DO 3003 p = 1, NR
                  IWORK(N+p) = 0
 3003          CONTINUE
               CALL CGEQP3( N, NR, V, LDV, IWORK(N+1), CWORK(N+1),
     $                 CWORK(2*N+1), LWORK-2*N, RWORK, IERR )
**               CALL CGEQRF( N, NR, V, LDV, CWORK(N+1), CWORK(2*N+1),
**     $              LWORK-2*N, IERR )
               IF ( L2PERT ) THEN
                  XSC = SQRT(SMALL)
                  DO 3969 p = 2, NR
                     DO 3968 q = 1, p - 1
                        CTEMP=CMPLX(XSC*AMIN1(ABS(V(p,p)),ABS(V(q,q))),
     $                              ZERO)
                        IF ( ABS(V(q,p)) .LE. TEMP1 )
*     $                     V(q,p) = TEMP1 * ( V(q,p) / ABS(V(q,p)) )
     $                     V(q,p) = CTEMP
 3968                CONTINUE
 3969             CONTINUE
               END IF
*
               CALL CLACPY( 'A', N, NR, V, LDV, CWORK(2*N+1), N )
*
               IF ( L2PERT ) THEN
                  XSC = SQRT(SMALL)
```

```
                 DO 8970 p = 2, NR
                    DO 8971 q = 1, p - 1
                       CTEMP=CMPLX(XSC*AMIN1(ABS(V(p,p)),ABS(V(q,q))),
     $                              ZERO)
*                       V(p,q) = - TEMP1*( V(q,p) / ABS(V(q,p)) )
                       V(p,q) = - CTEMP
 8971                CONTINUE
 8970             CONTINUE
               ELSE
                  CALL CLASET( 'L',NR-1,NR-1,CZERO,CZERO,V(2,1),LDV )
               END IF
*              Now, compute R2 = L3 * Q3, the LQ factorization.
               CALL CGELQF( NR, NR, V, LDV, CWORK(2*N+N*NR+1),
     $               CWORK(2*N+N*NR+NR+1), LWORK-2*N-N*NR-NR, IERR )
*              .. and estimate the condition number
               CALL CLACPY( 'L',NR,NR,V,LDV,CWORK(2*N+N*NR+NR+1),NR )
               DO 4950 p = 1, NR
                  TEMP1 = SCNRM2( p, CWORK(2*N+N*NR+NR+p), NR )
                  CALL CSSCAL( p, ONE/TEMP1, CWORK(2*N+N*NR+NR+p), NR )
 4950          CONTINUE
               CALL CPOCON( 'L',NR,CWORK(2*N+N*NR+NR+1),NR,ONE,TEMP1,
     $              CWORK(2*N+N*NR+NR+NR*NR+1),RWORK,IERR )
               CONDR2 = ONE / SQRT(TEMP1)
*
*
               IF ( CONDR2 .GE. COND_OK ) THEN
*                 .. save the Householder vectors used for Q3
*                 (this overwrittes the copy of R2, as it will not be
*                 needed in this branch, but it does not overwritte the
*                 Huseholder vectors of Q2.).
                  CALL CLACPY( 'U', NR, NR, V, LDV, CWORK(2*N+1), N )
*                 .. and the rest of the information on Q3 is in
*                 WORK(2*N+N*NR+1:2*N+N*NR+N)
               END IF
*
            END IF
*
            IF ( L2PERT ) THEN
               XSC = SQRT(SMALL)
               DO 4968 q = 2, NR
                  CTEMP = XSC * V(q,q)
                  DO 4969 p = 1, q - 1
*                    V(p,q) = - SIGN( TEMP1, V(q,p) )
*                     V(p,q) = - TEMP1*( V(p,q) / ABS(V(p,q)) )
                     V(p,q) = - CTEMP
 4969             CONTINUE
 4968          CONTINUE
            ELSE
               CALL CLASET( 'U', NR-1,NR-1, CZERO,CZERO, V(1,2), LDV )
            END IF
```

```
*
*        Second preconditioning finished; continue with Jacobi SVD
*        The input matrix is lower trinagular.
*
*        Recover the right singular vectors as solution of a well
*        conditioned triangular matrix equation.
*
            IF ( CONDR1 .LT. COND_OK ) THEN
*
               CALL CGESVJ( 'L','U','N',NR,NR,V,LDV,SVA,NR,U, LDU,
     $              CWORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,RWORK,
     $              LRWORK, INFO )
               SCALEM  = RWORK(1)
               NUMRANK = NINT(RWORK(2))
               DO 3970 p = 1, NR
                  CALL CCOPY(  NR, V(1,p), 1, U(1,p), 1 )
                  CALL CSSCAL( NR, SVA(p),    V(1,p), 1 )
 3970          CONTINUE

*        .. pick the right matrix equation and solve it
*
               IF ( NR .EQ. N ) THEN
* :))             .. best case, R1 is inverted. The solution of this matrix
*                  equation is Q2*V2 = the product of the Jacobi rotations
*                  used in CGESVJ, premultiplied with the orthogonal matrix
*                  from the second QR factorization.
                  CALL CTRSM('L','U','N','N', NR,NR,CONE, A,LDA, V,LDV)
               ELSE
*                  .. R1 is well conditioned, but non-square. Adjoint of R2
*                  is inverted to get the product of the Jacobi rotations
*                  used in CGESVJ. The Q-factor from the second QR
*                  factorization is then built in explicitly.
                  CALL CTRSM('L','U','C','N',NR,NR,CONE,CWORK(2*N+1),
     $                 N,V,LDV)
                  IF ( NR .LT. N ) THEN
                   CALL CLASET('A',N-NR,NR,ZERO,CZERO,V(NR+1,1),LDV)
                   CALL CLASET('A',NR,N-NR,ZERO,CZERO,V(1,NR+1),LDV)
                   CALL CLASET('A',N-NR,N-NR,ZERO,CONE,V(NR+1,NR+1),LDV)
                  END IF
                  CALL CUNMQR('L','N',N,N,NR,CWORK(2*N+1),N,CWORK(N+1),
     $                V,LDV,CWORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,IERR)
               END IF
*
            ELSE IF ( CONDR2 .LT. COND_OK ) THEN
*
*              The matrix R2 is inverted. The solution of the matrix equation
*              is Q3^* * V3 = the product of the Jacobi rotations (appplied to
*              the lower triangular L3 from the LQ factorization of
*              R2=L3*Q3), pre-multiplied with the transposed Q3.
               CALL CGESVJ( 'L', 'U', 'N', NR, NR, V, LDV, SVA, NR, U,
```

```
     $              LDU, CWORK(2*N+N*NR+NR+1), LWORK-2*N-N*NR-NR,
     $           RWORK, LRWORK, INFO )
               SCALEM  = RWORK(1)
               NUMRANK = NINT(RWORK(2))
               DO 3870 p = 1, NR
                  CALL CCOPY( NR, V(1,p), 1, U(1,p), 1 )
                  CALL CSSCAL( NR, SVA(p),    U(1,p), 1 )
 3870          CONTINUE
               CALL CTRSM('L','U','N','N',NR,NR,CONE,CWORK(2*N+1),N,
     $                    U,LDU)
*              .. apply the permutation from the second QR factorization
               DO 873 q = 1, NR
                  DO 872 p = 1, NR
                     CWORK(2*N+N*NR+NR+IWORK(N+p)) = U(p,q)
 872              CONTINUE
                  DO 874 p = 1, NR
                     U(p,q) = CWORK(2*N+N*NR+NR+p)
 874              CONTINUE
 873           CONTINUE
               IF ( NR .LT. N ) THEN
                  CALL CLASET( 'A',N-NR,NR,CZERO,CZERO,V(NR+1,1),LDV )
                  CALL CLASET( 'A',NR,N-NR,CZERO,CZERO,V(1,NR+1),LDV )
                  CALL CLASET('A',N-NR,N-NR,CZERO,CONE,V(NR+1,NR+1),LDV)
               END IF
               CALL CUNMQR( 'L','N',N,N,NR,CWORK(2*N+1),N,CWORK(N+1),
     $              V,LDV,CWORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,IERR )
            ELSE
*              Last line of defense.
* #:(          This is a rather pathological case: no scaled condition
*              improvement after two pivoted QR factorizations. Other
*              possibility is that the rank revealing QR factorization
*              or the condition estimator has failed, or the COND_OK
*              is set very close to ONE (which is unnecessary). Normally,
*              this branch should never be executed, but in rare cases of
*              failure of the RRQR or condition estimator, the last line of
*              defense ensures that CGEJSV completes the task.
*              Compute the full SVD of L3 using CGESVJ with explicit
*              accumulation of Jacobi rotations.
               CALL CGESVJ( 'L', 'U', 'V', NR, NR, V, LDV, SVA, NR, U,
     $              LDU, CWORK(2*N+N*NR+NR+1), LWORK-2*N-N*NR-NR,
     $                         RWORK, LRWORK, INFO )
               SCALEM  = RWORK(1)
               NUMRANK = NINT(RWORK(2))
               IF ( NR .LT. N ) THEN
                  CALL CLASET( 'A',N-NR,NR,CZERO,CZERO,V(NR+1,1),LDV )
                  CALL CLASET( 'A',NR,N-NR,CZERO,CZERO,V(1,NR+1),LDV )
                  CALL CLASET('A',N-NR,N-NR,CZERO,CONE,V(NR+1,NR+1),LDV)
               END IF
               CALL CUNMQR( 'L','N',N,N,NR,CWORK(2*N+1),N,CWORK(N+1),
     $              V,LDV,CWORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,IERR )
```

```
*
               CALL CUNMLQ( 'L', 'C', NR, NR, NR, CWORK(2*N+1), N,
     $              CWORK(2*N+N*NR+1), U, LDU, CWORK(2*N+N*NR+NR+1),
     $              LWORK-2*N-N*NR-NR, IERR )
               DO 773 q = 1, NR
                  DO 772 p = 1, NR
                     CWORK(2*N+N*NR+NR+IWORK(N+p)) = U(p,q)
 772              CONTINUE
                  DO 774 p = 1, NR
                     U(p,q) = CWORK(2*N+N*NR+NR+p)
 774              CONTINUE
 773           CONTINUE
*
            END IF
*
*           Permute the rows of V using the (column) permutation from the
*           first QRF. Also, scale the columns to make them unit in
*           Euclidean norm. This applies to all cases.
*
            TEMP1 = SQRT(FLOAT(N)) * EPSLN
            DO 1972 q = 1, N
               DO 972 p = 1, N
                  CWORK(2*N+N*NR+NR+IWORK(p)) = V(p,q)
  972          CONTINUE
               DO 973 p = 1, N
                  V(p,q) = CWORK(2*N+N*NR+NR+p)
  973          CONTINUE
               XSC = ONE / SCNRM2( N, V(1,q), 1 )
               IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $           CALL CSSCAL( N, XSC, V(1,q), 1 )
 1972       CONTINUE
*           At this moment, V contains the right singular vectors of A.
*           Next, assemble the left singular vector matrix U (M x N).
            IF ( NR .LT. M ) THEN
               CALL CLASET('A', M-NR, NR, CZERO, CZERO, U(NR+1,1), LDU)
               IF ( NR .LT. N1 ) THEN
                  CALL CLASET('A',NR,N1-NR,CZERO,CZERO,U(1,NR+1),LDU)
                  CALL CLASET('A',M-NR,N1-NR,CZERO,CONE,
     $                       U(NR+1,NR+1),LDU)
               END IF
            END IF
*
*           The Q matrix from the first QRF is built into the left singular
*           matrix U. This applies to all cases.
*
            CALL CUNMQR( 'Left', 'No_Tr', M, N1, N, A, LDA, CWORK, U,
     $           LDU, CWORK(N+1), LWORK-N, IERR )

*           The columns of U are normalized. The cost is O(M*N) flops.
            TEMP1 = SQRT(FLOAT(M)) * EPSLN
```

```
            DO 1973 p = 1, NR
               XSC = ONE / SCNRM2( M, U(1,p), 1 )
               IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $          CALL CSSCAL( M, XSC, U(1,p), 1 )
 1973       CONTINUE
*
*           If the initial QRF is computed with row pivoting, the left
*           singular vectors must be adjusted.
*
            IF ( ROWPIV )
     $          CALL CLASWP( N1, U, LDU, 1, M-1, IWORK(2*N+1), -1 )
*
         ELSE
*
*        .. the initial matrix A has almost orthogonal columns and
*        the second QRF is not needed
*
            CALL CLACPY( 'Upper', N, N, A, LDA, CWORK(N+1), N )
            IF ( L2PERT ) THEN
               XSC = SQRT(SMALL)
               DO 5970 p = 2, N
                  CTEMP = XSC * CWORK( N + (p-1)*N + p )
                  DO 5971 q = 1, p - 1
*                     CWORK(N+(q-1)*N+p)=-TEMP1 * ( CWORK(N+(p-1)*N+q) /
*     $                                     ABS(CWORK(N+(p-1)*N+q)) )
                     CWORK(N+(q-1)*N+p)=-CTEMP
 5971             CONTINUE
 5970          CONTINUE
            ELSE
               CALL CLASET( 'Lower',N-1,N-1,CZERO,CZERO,CWORK(N+2),N )
            END IF
*
            CALL CGESVJ( 'Upper', 'U', 'N', N, N, CWORK(N+1), N, SVA,
     $           N, U, LDU, CWORK(N+N*N+1), LWORK-N-N*N, RWORK, LRWORK,
     $       INFO )
*
            SCALEM  = RWORK(1)
            NUMRANK = NINT(RWORK(2))
            DO 6970 p = 1, N
               CALL CCOPY( N, CWORK(N+(p-1)*N+1), 1, U(1,p), 1 )
               CALL CSSCAL( N, SVA(p), CWORK(N+(p-1)*N+1), 1 )
 6970       CONTINUE
*
            CALL CTRSM( 'Left', 'Upper', 'NoTrans', 'No UD', N, N,
     $           CONE, A, LDA, CWORK(N+1), N )
            DO 6972 p = 1, N
               CALL CCOPY( N, CWORK(N+p), N, V(IWORK(p),1), LDV )
 6972       CONTINUE
            TEMP1 = SQRT(FLOAT(N))*EPSLN
            DO 6971 p = 1, N
```

```
               XSC = ONE / SCNRM2( N, V(1,p), 1 )
               IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $            CALL CSSCAL( N, XSC, V(1,p), 1 )
 6971       CONTINUE
*
*           Assemble the left singular vector matrix U (M x N).
*
            IF ( N .LT. M ) THEN
               CALL CLASET( 'A',  M-N, N, CZERO, CZERO, U(N+1,1), LDU )
               IF ( N .LT. N1 ) THEN
                  CALL CLASET('A',N,  N1-N, CZERO, CZERO,  U(1,N+1),LDU)
                  CALL CLASET( 'A',M-N,N1-N, CZERO, CONE,U(N+1,N+1),LDU)
               END IF
            END IF
            CALL CUNMQR( 'Left', 'No Tr', M, N1, N, A, LDA, CWORK, U,
     $           LDU, CWORK(N+1), LWORK-N, IERR )
            TEMP1 = SQRT(FLOAT(M))*EPSLN
            DO 6973 p = 1, N1
               XSC = ONE / SCNRM2( M, U(1,p), 1 )
               IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $            CALL CSSCAL( M, XSC, U(1,p), 1 )
 6973       CONTINUE
*
            IF ( ROWPIV )
     $         CALL CLASWP( N1, U, LDU, 1, M-1, IWORK(2*N+1), -1 )
*
         END IF
*
*        end of the  >> almost orthogonal case <<  in the full SVD
*
         ELSE
*
*        This branch deploys a preconditioned Jacobi SVD with explicitly
*        accumulated rotations. It is included as optional, mainly for
*        experimental purposes. It does perfom well, and can also be used.
*        In this implementation, this branch will be automatically activated
*        if the  condition number sigma_max(A) / sigma_min(A) is predicted
*        to be greater than the overflow threshold. This is because the
*        a posteriori computation of the singular vectors assumes robust
*        implementation of BLAS and some LAPACK procedures, capable of working
*        in presence of extreme values. Since that is not always the case, ...
*
         DO 7968 p = 1, NR
            CALL CCOPY( N-p+1, A(p,p), LDA, V(p,p), 1 )
            CALL CLACGV( N-p+1, V(p,p), 1 )
 7968    CONTINUE
*
         IF ( L2PERT ) THEN
            XSC = SQRT(SMALL/EPSLN)
            DO 5969 q = 1, NR
```

```
               CTEMP = CMPLX(XSC*ABS( V(q,q) ),ZERO)
               DO 5968 p = 1, N
                  IF ( ( p .GT. q ) .AND. ( ABS(V(p,q)) .LE. TEMP1 )
     $                .OR. ( p .LT. q ) )
*     $                V(p,q) = TEMP1 * ( V(p,q) / ABS(V(p,q)) )
     $                V(p,q) = CTEMP
                  IF ( p .LT. q ) V(p,q) = - V(p,q)
 5968          CONTINUE
 5969       CONTINUE
         ELSE
            CALL CLASET( 'U', NR-1, NR-1, CZERO, CZERO, V(1,2), LDV )
         END IF

         CALL CGEQRF( N, NR, V, LDV, CWORK(N+1), CWORK(2*N+1),
     $        LWORK-2*N, IERR )
         CALL CLACPY( 'L', N, NR, V, LDV, CWORK(2*N+1), N )
*
         DO 7969 p = 1, NR
            CALL CCOPY( NR-p+1, V(p,p), LDV, U(p,p), 1 )
            CALL CLACGV( NR-p+1, U(p,p), 1 )
 7969    CONTINUE

         IF ( L2PERT ) THEN
            XSC = SQRT(SMALL/EPSLN)
            DO 9970 q = 2, NR
               DO 9971 p = 1, q - 1
                  CTEMP = CMPLX(XSC * AMIN1(ABS(U(p,p)),ABS(U(q,q))),
     $                           ZERO)
*                  U(p,q) = - TEMP1 * ( U(q,p) / ABS(U(q,p)) )
                  U(p,q) = - CTEMP
 9971          CONTINUE
 9970       CONTINUE
         ELSE
            CALL CLASET('U', NR-1, NR-1, CZERO, CZERO, U(1,2), LDU )
         END IF

         CALL CGESVJ( 'L', 'U', 'V', NR, NR, U, LDU, SVA,
     $        N, V, LDV, CWORK(2*N+N*NR+1), LWORK-2*N-N*NR,
     $         RWORK, LRWORK, INFO )
         SCALEM  = RWORK(1)
         NUMRANK = NINT(RWORK(2))

         IF ( NR .LT. N ) THEN
            CALL CLASET( 'A',N-NR,NR,ZERO,ZERO,V(NR+1,1),LDV )
            CALL CLASET( 'A',NR,N-NR,ZERO,ZERO,V(1,NR+1),LDV )
            CALL CLASET( 'A',N-NR,N-NR,ZERO,ONE,V(NR+1,NR+1),LDV )
         END IF

         CALL CUNMQR( 'L','N',N,N,NR,CWORK(2*N+1),N,CWORK(N+1),
     $        V,LDV,CWORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,IERR )
```

```
*
*          Permute the rows of V using the (column) permutation from the
*          first QRF. Also, scale the columns to make them unit in
*          Euclidean norm. This applies to all cases.
*
           TEMP1 = SQRT(FLOAT(N)) * EPSLN
           DO 7972 q = 1, N
              DO 8972 p = 1, N
                 CWORK(2*N+N*NR+NR+IWORK(p)) = V(p,q)
 8972         CONTINUE
              DO 8973 p = 1, N
                 V(p,q) = CWORK(2*N+N*NR+NR+p)
 8973         CONTINUE
              XSC = ONE / SCNRM2( N, V(1,q), 1 )
              IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $          CALL CSSCAL( N, XSC, V(1,q), 1 )
 7972      CONTINUE
*
*          At this moment, V contains the right singular vectors of A.
*          Next, assemble the left singular vector matrix U (M x N).
*
        IF ( NR .LT. M ) THEN
           CALL CLASET( 'A',  M-NR, NR, CZERO, CZERO, U(NR+1,1), LDU )
           IF ( NR .LT. N1 ) THEN
              CALL CLASET('A',NR,  N1-NR, CZERO, CZERO,  U(1,NR+1),LDU)
              CALL CLASET('A',M-NR,N1-NR, CZERO, CONE,U(NR+1,NR+1),LDU)
           END IF
        END IF
*
        CALL CUNMQR( 'Left', 'No Tr', M, N1, N, A, LDA, CWORK, U,
     $       LDU, CWORK(N+1), LWORK-N, IERR )
*
           IF ( ROWPIV )
     $        CALL CLASWP( N1, U, LDU, 1, M-1, IWORK(2*N+1), -1 )
*
*
        END IF
        IF ( TRANSP ) THEN
*          .. swap U and V because the procedure worked on A^*
           DO 6974 p = 1, N
              CALL CSWAP( N, U(1,p), 1, V(1,p), 1 )
 6974      CONTINUE
        END IF
*
      END IF
*     end of the full SVD
*
*     Undo scaling, if necessary (and possible)
*
      IF ( USCAL2 .LE. (BIG/SVA(1))*USCAL1 ) THEN
```

```
         CALL CLASCL( 'G', 0, 0, USCAL1, USCAL2, NR, 1, SVA, N, IERR )
         USCAL1 = ONE
         USCAL2 = ONE
      END IF
*
      IF ( NR .LT. N ) THEN
         DO 3004 p = NR+1, N
            SVA(p) = ZERO
 3004    CONTINUE
      END IF
*
      RWORK(1) = USCAL2 * SCALEM
      RWORK(2) = USCAL1
      IF ( ERREST ) RWORK(3) = SCONDA
      IF ( LSVEC .AND. RSVEC ) THEN
         RWORK(4) = CONDR1
         RWORK(5) = CONDR2
      END IF
      IF ( L2TRAN ) THEN
         RWORK(6) = ENTRA
         RWORK(7) = ENTRAT
      END IF
*
      IWORK(1) = NR
      IWORK(2) = NUMRANK
      IWORK(3) = WARNING
*
      RETURN
*     ..
*     .. END OF CGEJSV
*     ..
      END
*
```

———-

30.1.4 lisp code

— cgejsv.lisp —

```
(let* ((zero 0.0)
       (one 1.0)
       (czero (f2cl-lib:cmplx 0.0 0.0))
       (cone (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (f2cl-lib:complex8) czero)
           (type (f2cl-lib:complex8) cone)
```

```
          (ignorable zero one czero cone))
(defun cgejsv
       (joba jobu jobv jobr jobt jobp m n a lda sva u ldu v ldv cwork lwork
        rwork lrwork iwork info)
  (declare (type (array f2cl-lib:integer4 (*)) iwork)
           (type (array single-float (*)) rwork sva)
           (type (array f2cl-lib:complex8 (*)) cwork v u a)
           (type (f2cl-lib:integer4) info lrwork lwork ldv ldu lda n m)
           (type (string *) jobp jobt jobr jobv jobu joba))
  (f2cl-lib:with-multi-array-data
      ((joba character joba-%data% joba-%offset%)
       (jobu character jobu-%data% jobu-%offset%)
       (jobv character jobv-%data% jobv-%offset%)
       (jobr character jobr-%data% jobr-%offset%)
       (jobt character jobt-%data% jobt-%offset%)
       (jobp character jobp-%data% jobp-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (u f2cl-lib:complex8 u-%data% u-%offset%)
       (v f2cl-lib:complex8 v-%data% v-%offset%)
       (cwork f2cl-lib:complex8 cwork-%data% cwork-%offset%)
       (sva single-float sva-%data% sva-%offset%)
       (rwork single-float rwork-%data% rwork-%offset%)
       (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
    (prog ((almort nil) (defr nil) (errest nil) (goscal nil) (jracc nil)
           (kill nil) (lsvec nil) (l2aber nil) (l2kill nil) (l2pert nil)
           (l2rank nil) (l2tran nil) (noscal nil) (rowpiv nil) (rsvec nil)
           (transp nil) (ierr 0) (n1 0) (nr 0) (numrank 0) (p 0) (q 0)
           (warning 0) (aapp 0.0) (aaqq 0.0) (aatmax 0.0) (aatmin 0.0)
           (big 0.0) (big1 0.0) (cond_ok 0.0) (condr1 0.0) (condr2 0.0)
           (entra 0.0) (entrat 0.0) (epsln 0.0) (maxprj 0.0) (scalem 0.0)
           (sconda 0.0) (sfmin 0.0) (small 0.0) (temp1 0.0) (uscal1 0.0)
           (uscal2 0.0) (xsc 0.0) (ctemp #C(0.0 0.0)))
      (declare
       (type f2cl-lib:logical almort defr errest goscal jracc kill lsvec
        l2aber l2kill l2pert l2rank l2tran noscal rowpiv rsvec transp)
       (type (f2cl-lib:integer4) ierr n1 nr numrank p q warning)
       (type (single-float) aapp aaqq aatmax aatmin big big1 cond_ok condr1
        condr2 entra entrat epsln maxprj scalem sconda sfmin small temp1
        uscal1 uscal2 xsc)
       (type (f2cl-lib:complex8) ctemp))
      (setf lsvec
              (or
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame jobu "U")
                 (declare (ignore var-1))
                 (when var-0 (setf jobu var-0))
                 ret-val)
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame jobu "F")
                 (declare (ignore var-1))
```

```
            (when var-0 (setf jobu var-0))
            ret-val)))
(setf jracc
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobv "J")
          (declare (ignore var-1))
          (when var-0 (setf jobv var-0))
          ret-val))
(setf rsvec
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame jobv "V")
           (declare (ignore var-1))
           (when var-0 (setf jobv var-0))
           ret-val)
         jracc))
(setf rowpiv
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame joba "F")
           (declare (ignore var-1))
           (when var-0 (setf joba var-0))
           ret-val)
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame joba "G")
           (declare (ignore var-1))
           (when var-0 (setf joba var-0))
           ret-val)))
(setf l2rank
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame joba "R")
          (declare (ignore var-1))
          (when var-0 (setf joba var-0))
          ret-val))
(setf l2aber
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame joba "A")
          (declare (ignore var-1))
          (when var-0 (setf joba var-0))
          ret-val))
(setf errest
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame joba "E")
           (declare (ignore var-1))
           (when var-0 (setf joba var-0))
           ret-val)
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame joba "G")
           (declare (ignore var-1))
```

```
            (when var-0 (setf joba var-0))
            ret-val)))
(setf l2tran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobt "T")
          (declare (ignore var-1))
          (when var-0 (setf jobt var-0))
          ret-val))
(setf l2kill
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobr "R")
          (declare (ignore var-1))
          (when var-0 (setf jobr var-0))
          ret-val))
(setf defr
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobr "N")
          (declare (ignore var-1))
          (when var-0 (setf jobr var-0))
          ret-val))
(setf l2pert
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobp "P")
          (declare (ignore var-1))
          (when var-0 (setf jobp var-0))
          ret-val))
(cond
 ((not
   (or rowpiv l2rank l2aber errest
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame joba "C")
         (declare (ignore var-1))
         (when var-0 (setf joba var-0))
         ret-val)))
  (setf info -1))
 ((not
   (or lsvec
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobu "N")
         (declare (ignore var-1))
         (when var-0 (setf jobu var-0))
         ret-val)
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobu "W")
         (declare (ignore var-1))
         (when var-0 (setf jobu var-0))
         ret-val)))
  (setf info -2))
 ((or
   (not
```

```
    (or rsvec
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobv "N")
          (declare (ignore var-1))
          (when var-0 (setf jobv var-0))
          ret-val)
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobv "W")
          (declare (ignore var-1))
          (when var-0 (setf jobv var-0))
          ret-val)))
  (and jracc (not lsvec)))
 (setf info -3))
((not (or l2kill defr)) (setf info -4))
((not
  (or l2tran
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame jobt "N")
        (declare (ignore var-1))
        (when var-0 (setf jobt var-0))
        ret-val)))
 (setf info -5))
((not
  (or l2pert
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame jobp "N")
        (declare (ignore var-1))
        (when var-0 (setf jobp var-0))
        ret-val)))
 (setf info -6))
((< m 0) (setf info -7)) ((or (< n 0) (> n m)) (setf info -8))
((< lda m) (setf info -10)) ((and lsvec (< ldu m)) (setf info -13))
((and rsvec (< ldv n)) (setf info -15))
((or
  (and (not (or lsvec rsvec errest))
       (< lwork (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) 1)))
  (and (not (or lsvec rsvec)) errest
       (< lwork
          (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                            (f2cl-lib:int-mul 3 n))))
  (and lsvec (not rsvec) (< lwork (f2cl-lib:int-mul 3 n)))
  (and rsvec (not lsvec) (< lwork (f2cl-lib:int-mul 3 n)))
  (and lsvec rsvec (not jracc)
       (< lwork
          (f2cl-lib:int-add (f2cl-lib:int-mul 5 n)
                            (f2cl-lib:int-mul 2 n n))))
  (and lsvec rsvec jracc
       (< lwork
          (f2cl-lib:int-add (f2cl-lib:int-mul 4 n)
                            (f2cl-lib:int-mul n n)))))
```

```
  (setf info -17))
 ((< lrwork
     (f2cl-lib:max0 (f2cl-lib:int-add n (f2cl-lib:int-mul 2 m)) 7))
  (setf info -19))
 (t (setf info 0)))
(cond
 ((/= info 0) (xerbla "CGEJSV" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= m 0) (= n 0))
    (go end_label))
(cond
 (lsvec (setf n1 n)
  (if (multiple-value-bind (ret-val var-0 var-1)
          (lsame jobu "F")
        (declare (ignore var-1))
        (when var-0 (setf jobu var-0))
        ret-val)
      (setf n1 m))))
(setf epsln (slamch "Epsilon"))
(setf sfmin (slamch "SafeMinimum"))
(setf small (/ sfmin epsln))
(setf big (slamch "O"))
(setf scalem
        (/ one
           (f2cl-lib:fsqrt
            (* (f2cl-lib:ffloat m) (f2cl-lib:ffloat n)))))
(setf noscal f2cl-lib:%true%)
(setf goscal f2cl-lib:%true%)
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (setf aapp zero)
    (setf aaqq one)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (classq m
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 p)
                               ((1 lda) (1 *)) a-%offset%)
         1 aapp aaqq)
      (declare (ignore var-1 var-2))
      (when var-0 (setf m var-0))
      (when var-3 (setf aapp var-3))
      (when var-4 (setf aaqq var-4)))
    (cond
     ((> aapp big) (setf info -9)
      (xerbla "CGEJSV" (f2cl-lib:int-sub info)) (go end_label)))
    (setf aaqq (f2cl-lib:fsqrt aaqq))
    (cond
     ((and (< aapp (f2cl-lib:f2cl/ big aaqq)) noscal)
      (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
              (* aapp aaqq)))
```

```
      (t (setf noscal f2cl-lib:%false%)
       (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
               (* aapp (* aaqq scalem)))
       (cond
        (goscal (setf goscal f2cl-lib:%false%)
         (multiple-value-bind (var-0 var-1 var-2 var-3)
             (sscal (f2cl-lib:int-sub p 1) scalem sva 1)
           (declare (ignore var-0 var-2 var-3))
           (when var-1 (setf scalem var-1)))))))
   label1874))
(if noscal
    (setf scalem one))
(setf aapp zero)
(setf aaqq big)
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (setf aapp
            (f2cl-lib:amax1 aapp
                            (f2cl-lib:fref sva-%data% (p) ((1 n))
                                           sva-%offset%)))
    (if (/= (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%) zero)
        (setf aaqq
                (f2cl-lib:amin1 aaqq
                                (f2cl-lib:fref sva-%data% (p) ((1 n))
                                               sva-%offset%))))
   label4781))
(cond
 ((= aapp zero)
  (if lsvec
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (claset "G" m n1 czero cone u ldu)
        (declare (ignore var-0 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf n1 var-2))
        (when var-3 (setf czero var-3))
        (when var-4 (setf cone var-4))
        (when var-6 (setf ldu var-6))))
  (if rsvec
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (claset "G" n n czero cone v ldv)
        (declare (ignore var-0 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf czero var-3))
        (when var-4 (setf cone var-4))
        (when var-6 (setf ldv var-6))))
  (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%) one)
  (setf (f2cl-lib:fref rwork-%data% (2) ((1 *)) rwork-%offset%) one)
  (if errest
```

```
        (setf (f2cl-lib:fref rwork-%data% (3) ((1 *)) rwork-%offset%)
                one))
    (cond
     ((and lsvec rsvec)
      (setf (f2cl-lib:fref rwork-%data% (4) ((1 *)) rwork-%offset%) one)
      (setf (f2cl-lib:fref rwork-%data% (5) ((1 *)) rwork-%offset%)
              one)))
    (cond
     (l2tran
      (setf (f2cl-lib:fref rwork-%data% (6) ((1 *)) rwork-%offset%) zero)
      (setf (f2cl-lib:fref rwork-%data% (7) ((1 *)) rwork-%offset%)
              zero)))
    (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) 0)
    (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%) 0)
    (setf (f2cl-lib:fref iwork-%data% (3) ((1 *)) iwork-%offset%) 0)
    (go end_label)))
  (setf warning 0)
  (cond
   ((<= aaqq sfmin) (setf l2rank f2cl-lib:%true%)
    (setf l2kill f2cl-lib:%true%) (setf warning 1)))
  (cond
   ((= n 1)
    (cond
     (lsvec
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (clascl "G" 0 0
           (f2cl-lib:array-slice sva-%data% single-float (1) ((1 n))
                                 sva-%offset%)
           scalem m 1
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda ierr)
        (declare (ignore var-0 var-1 var-2 var-3 var-6 var-7))
        (when var-4 (setf scalem var-4))
        (when var-5 (setf m var-5))
        (when var-8 (setf lda var-8))
        (when var-9 (setf ierr var-9)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (clacpy "A" m 1 a lda u ldu)
        (declare (ignore var-0 var-2 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldu var-6)))
      (cond
       ((/= n1 n)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (cgeqrf m n u ldu cwork
             (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
```

```
                                ((+ n 1)) ((1 lwork)) cwork-%offset%)
            (f2cl-lib:int-sub lwork n) ierr)
        (declare (ignore var-2 var-4 var-5 var-6))
        (when var-0 (setf m var-0))
        (when var-1 (setf n var-1))
        (when var-3 (setf ldu var-3))
        (when var-7 (setf ierr var-7)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (cungqr m n1 1 u ldu cwork
           (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                                ((+ n 1)) ((1 lwork)) cwork-%offset%)
           (f2cl-lib:int-sub lwork n) ierr)
        (declare (ignore var-2 var-3 var-5 var-6 var-7))
        (when var-0 (setf m var-0))
        (when var-1 (setf n1 var-1))
        (when var-4 (setf ldu var-4))
        (when var-8 (setf ierr var-8)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (ccopy m
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 1)
                                ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 1)
                                ((1 ldu) (1 *)) u-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf m var-0)))))))
(cond
 (rsvec
  (setf (f2cl-lib:fref v-%data% (1 1) ((1 ldv) (1 *)) v-%offset%)
          cone)))
(cond
 ((< (f2cl-lib:fref sva (1) ((1 n))) (* big scalem))
  (setf (f2cl-lib:fref sva-%data% (1) ((1 n)) sva-%offset%)
          (/ (f2cl-lib:fref sva-%data% (1) ((1 n)) sva-%offset%)
             scalem))
  (setf scalem one)))
(setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
        (/ one scalem))
(setf (f2cl-lib:fref rwork-%data% (2) ((1 *)) rwork-%offset%) one)
(cond
 ((/= (f2cl-lib:fref sva (1) ((1 n))) zero)
  (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) 1)
  (cond
   ((>= (f2cl-lib:f2cl/ (f2cl-lib:fref sva (1) ((1 n))) scalem)
        sfmin)
    (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%) 1))
   (t
    (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%)
```

```
                0))))
     (t (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) 0)
      (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%) 0)))
    (setf (f2cl-lib:fref iwork-%data% (3) ((1 *)) iwork-%offset%) 0)
    (if errest
        (setf (f2cl-lib:fref rwork-%data% (3) ((1 *)) rwork-%offset%)
                one))
    (cond
     ((and lsvec rsvec)
      (setf (f2cl-lib:fref rwork-%data% (4) ((1 *)) rwork-%offset%) one)
      (setf (f2cl-lib:fref rwork-%data% (5) ((1 *)) rwork-%offset%)
              one)))
    (cond
     (l2tran
      (setf (f2cl-lib:fref rwork-%data% (6) ((1 *)) rwork-%offset%) zero)
      (setf (f2cl-lib:fref rwork-%data% (7) ((1 *)) rwork-%offset%)
              zero)))
    (go end_label)))
  (setf transp f2cl-lib:%false%)
  (setf l2tran (and l2tran (= m n)))
  (setf aatmax (- one))
  (setf aatmin big)
  (cond
   ((or rowpiv l2tran)
    (cond
     (l2tran
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p m) nil)
        (tagbody
          (setf xsc zero)
          (setf temp1 one)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (classq n
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (p 1)
                                     ((1 lda) (1 *)) a-%offset%)
               lda xsc temp1)
            (declare (ignore var-1))
            (when var-0 (setf n var-0))
            (when var-2 (setf lda var-2))
            (when var-3 (setf xsc var-3))
            (when var-4 (setf temp1 var-4)))
          (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add m n p))
                               ((1 *)) rwork-%offset%)
                  (* xsc scalem))
          (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add n p))
                               ((1 *)) rwork-%offset%)
                  (* xsc (* scalem (f2cl-lib:fsqrt temp1))))
          (setf aatmax
                  (f2cl-lib:amax1 aatmax
                                  (f2cl-lib:fref rwork-%data%
```

```
                                                ((f2cl-lib:int-add n p))
                                                ((1 *))
                                                rwork-%offset%)))
            (if (/=
                 (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add n p))
                                ((1 *)) rwork-%offset%)
                 zero)
                (setf aatmin
                        (f2cl-lib:amin1 aatmin
                                        (f2cl-lib:fref rwork-%data%
                                                       ((f2cl-lib:int-add n
                                                                          p))
                                                       ((1 *))
                                                       rwork-%offset%))))
           label1950)))
        (t
         (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                       ((> p m) nil)
           (tagbody
             (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add m n p))
                                  ((1 *)) rwork-%offset%)
                     (* scalem
                        (abs
                         (f2cl-lib:fref a-%data%
                                        (p
                                         (multiple-value-bind
                                             (ret-val var-0 var-1 var-2)
                                             (isamax n
                                              (f2cl-lib:array-slice a
                                                                    f2cl-lib:complex8
                                                                    (p 1)
                                                                    ((1 lda)
                                                                     (1 *)))
                                              lda)
                                           (declare (ignore var-1))
                                           (when var-0 (setf n var-0))
                                           (when var-2 (setf lda var-2))
                                           ret-val))
                                        ((1 lda) (1 *)) a-%offset%))))
             (setf aatmax
                     (f2cl-lib:amax1 aatmax
                                     (f2cl-lib:fref rwork-%data%
                                                    ((f2cl-lib:int-add m n
                                                                       p))
                                                    ((1 *))
                                                    rwork-%offset%)))
             (setf aatmin
                     (f2cl-lib:amin1 aatmin
                                     (f2cl-lib:fref rwork-%data%
                                                    ((f2cl-lib:int-add m n
```

```
                                                                    p))
                                                  ((1 *))
                                                  rwork-%offset%)))
        label1904))))))
(setf entra zero)
(setf entrat zero)
(cond
 (l2tran (setf xsc zero) (setf temp1 one)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (classq n sva 1 xsc temp1)
    (declare (ignore var-1 var-2))
    (when var-0 (setf n var-0))
    (when var-3 (setf xsc var-3))
    (when var-4 (setf temp1 var-4)))
  (setf temp1 (/ one temp1)) (setf entra zero)
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (setf big1
              (*
               (expt
                (/ (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                   xsc)
                2)
               temp1))
      (if (/= big1 zero)
          (setf entra (+ entra (* big1 (f2cl-lib:alog big1)))))
     label1113))
  (setf entra (/ (- entra) (f2cl-lib:alog (f2cl-lib:ffloat n))))
  (setf entrat zero)
  (f2cl-lib:fdo (p (f2cl-lib:int-add n 1) (f2cl-lib:int-add p 1))
                ((> p (f2cl-lib:int-add n m)) nil)
    (tagbody
      (setf big1
              (*
               (expt
                (/
                 (f2cl-lib:fref rwork-%data% (p) ((1 *))
                                rwork-%offset%)
                 xsc)
                2)
               temp1))
      (if (/= big1 zero)
          (setf entrat (+ entrat (* big1 (f2cl-lib:alog big1)))))
     label1114))
  (setf entrat (/ (- entrat) (f2cl-lib:alog (f2cl-lib:ffloat m))))
  (setf transp (< entrat entra)) (setf transp f2cl-lib:%true%)
  (cond
   (transp
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
```

```
                    ((> p (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *)) a-%offset%)
                  (f2cl-lib:conjg
                   (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                                  a-%offset%)))
          (f2cl-lib:fdo (q (f2cl-lib:int-add p 1) (f2cl-lib:int-add q 1))
                        ((> q n) nil)
            (tagbody
              (setf ctemp
                      (f2cl-lib:conjg
                       (f2cl-lib:fref a-%data% (q p) ((1 lda) (1 *))
                                      a-%offset%)))
              (setf (f2cl-lib:fref a-%data% (q p) ((1 lda) (1 *))
                                    a-%offset%)
                      (f2cl-lib:conjg
                       (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                      a-%offset%)))
              (setf (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                    a-%offset%)
                      ctemp)
             label1116))
         label1115))
      (setf (f2cl-lib:fref a-%data% (n n) ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:conjg
               (f2cl-lib:fref a-%data% (n n) ((1 lda) (1 *))
                              a-%offset%)))
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p n) nil)
        (tagbody
          (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add m n p))
                                ((1 *)) rwork-%offset%)
                  (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%))
          (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                  (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add n p))
                                 ((1 *)) rwork-%offset%))
         label1117))
      (setf temp1 aapp) (setf aapp aatmax) (setf aatmax temp1)
      (setf temp1 aaqq) (setf aaqq aatmin) (setf aatmin temp1)
      (setf kill lsvec) (setf lsvec rsvec) (setf rsvec kill)
      (if lsvec
          (setf n1 n))
      (setf rowpiv f2cl-lib:%true%)))))
  (setf big1 (f2cl-lib:fsqrt big))
  (setf temp1 (f2cl-lib:fsqrt (/ big (f2cl-lib:ffloat n))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 aapp temp1 n 1 sva n ierr)
    (declare (ignore var-0 var-1 var-2 var-6 var-7))
    (when var-3 (setf aapp var-3))
```

```
  (when var-4 (setf temp1 var-4))
  (when var-5 (setf n var-5))
  (when var-8 (setf n var-8))
  (when var-9 (setf ierr var-9)))
(cond ((> aaqq (* aapp sfmin)) (setf aaqq (* (/ aaqq aapp) temp1)))
      (t (setf aaqq (/ (* aaqq temp1) aapp))))
(setf temp1 (* temp1 scalem))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (clascl "G" 0 0 aapp temp1 m n a lda ierr)
  (declare (ignore var-0 var-1 var-2 var-7))
  (when var-3 (setf aapp var-3))
  (when var-4 (setf temp1 var-4))
  (when var-5 (setf m var-5))
  (when var-6 (setf n var-6))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(setf uscal1 temp1)
(setf uscal2 aapp)
(cond (l2kill (setf xsc (f2cl-lib:fsqrt sfmin)))
      (t (setf xsc small)
       (cond
        ((and (< aaqq (f2cl-lib:fsqrt sfmin)) lsvec rsvec)
         (setf jracc f2cl-lib:%true%)))))
(cond
 ((< aaqq xsc)
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (cond
       ((< (f2cl-lib:fref sva (p) ((1 n))) xsc)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (claset "A" m 1 czero czero
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 p)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-2 var-5))
          (when var-1 (setf m var-1))
          (when var-3 (setf czero var-3))
          (when var-4 (setf czero var-4))
          (when var-6 (setf lda var-6)))
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                zero)))
     label700))))
(cond
 (rowpiv
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
    (tagbody
```

```
        (setf q
                (f2cl-lib:int-sub
                 (f2cl-lib:int-add
                  (isamax (f2cl-lib:int-add (f2cl-lib:int-sub m p) 1)
                   (f2cl-lib:array-slice rwork-%data% single-float
                                         ((+ m n p)) ((1 *))
                                         rwork-%offset%)
                   1)
                  p)
                 1))
        (setf (f2cl-lib:fref iwork-%data%
                             ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                p))
                             ((1 *)) iwork-%offset%)
                q)
        (cond
         ((/= p q)
          (setf temp1
                  (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add m n p))
                                 ((1 *)) rwork-%offset%))
          (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add m n p))
                               ((1 *)) rwork-%offset%)
                  (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add m n q))
                                 ((1 *)) rwork-%offset%))
          (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add m n q))
                               ((1 *)) rwork-%offset%)
                  temp1)))
      label1952))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claswp n a lda 1 (f2cl-lib:int-sub m 1)
         (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                               ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                               iwork-%offset%)
         1)
      (declare (ignore var-1 var-3 var-4 var-5 var-6))
      (when var-0 (setf n var-0))
      (when var-2 (setf lda var-2)))))
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (setf (f2cl-lib:fref iwork-%data% (p) ((1 *)) iwork-%offset%) 0)
     label1963))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (cgeqp3 m n a lda iwork cwork
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                             ((1 lwork)) cwork-%offset%)
       (f2cl-lib:int-sub lwork n) rwork ierr)
    (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8))
    (when var-0 (setf m var-0))
```

```
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-9 (setf ierr var-9)))
(setf nr 1)
(cond
 (l2aber
  (tagbody
    (setf temp1 (* (f2cl-lib:fsqrt (f2cl-lib:ffloat n)) epsln))
    (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (cond
         ((>= (abs (f2cl-lib:fref a (p p) ((1 lda) (1 *))))
              (* temp1 (abs (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))))
          (setf nr (f2cl-lib:int-add nr 1)))
         (t (go label3002)))
       label3001))
   label3002))
 (l2rank
  (tagbody
    (setf temp1 (f2cl-lib:fsqrt sfmin))
    (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (if (or
             (<
              (abs
               (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                              a-%offset%))
              (* epsln
                 (abs
                  (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-sub p 1)
                                  (f2cl-lib:int-sub p 1))
                                 ((1 lda) (1 *)) a-%offset%))))
             (<
              (abs
               (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                              a-%offset%))
              small)
             (and l2kill
                  (<
                   (abs
                    (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                                   a-%offset%))
                   temp1)))
            (go label3402))
        (setf nr (f2cl-lib:int-add nr 1))
       label3401))
   label3402))
```

```
 (t
  (tagbody
    (setf temp1 (f2cl-lib:fsqrt sfmin))
    (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (if (or
             (<
              (abs
               (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                              a-%offset%))
              small)
             (and l2kill
                  (<
                   (abs
                    (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                                   a-%offset%))
                   temp1)))
            (go label3302))
        (setf nr (f2cl-lib:int-add nr 1))
       label3301))
   label3302)))
(setf almort f2cl-lib:%false%)
(cond
 ((= nr n) (setf maxprj one)
  (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (setf temp1
              (/
               (abs
                (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                               a-%offset%))
               (f2cl-lib:fref sva-%data%
                              ((f2cl-lib:fref iwork (p) ((1 *))))
                              ((1 n)) sva-%offset%)))
      (setf maxprj (f2cl-lib:amin1 maxprj temp1))
     label3051))
  (if (>= (expt maxprj 2) (- one (* (f2cl-lib:ffloat n) epsln)))
      (setf almort f2cl-lib:%true%))))
(setf sconda (- one))
(setf condr1 (- one))
(setf condr2 (- one))
(cond
 (errest
  (cond
   ((= n nr)
    (cond
     (rsvec
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
      (clacpy "U" n n a lda v ldv)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldv var-6)))
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (setf temp1
              (f2cl-lib:fref sva-%data%
                             ((f2cl-lib:fref iwork (p) ((1 *))))
                             ((1 n)) sva-%offset%))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (csscal p (/ one temp1)
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (1 p)
                                 ((1 ldv) (1 *)) v-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf p var-0)))
    label3053))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (cpocon "U" n v ldv one temp1
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                             ((+ n 1)) ((1 lwork)) cwork-%offset%)
       rwork ierr)
    (declare (ignore var-0 var-2 var-6 var-7))
    (when var-1 (setf n var-1))
    (when var-3 (setf ldv var-3))
    (when var-4 (setf one var-4))
    (when var-5 (setf temp1 var-5))
    (when var-8 (setf ierr var-8))))
 (lsvec
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "U" n n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (setf temp1
              (f2cl-lib:fref sva-%data%
                             ((f2cl-lib:fref iwork (p) ((1 *))))
                             ((1 n)) sva-%offset%))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (csscal p (/ one temp1)
```

```
          (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 p)
                                ((1 ldu) (1 *)) u-%offset%)
          1)
       (declare (ignore var-1 var-2 var-3))
       (when var-0 (setf p var-0)))
    label3054))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (cpocon "U" n u ldu one temp1
      (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                            ((+ n 1)) ((1 lwork)) cwork-%offset%)
      rwork ierr)
   (declare (ignore var-0 var-2 var-6 var-7))
   (when var-1 (setf n var-1))
   (when var-3 (setf ldu var-3))
   (when var-4 (setf one var-4))
   (when var-5 (setf temp1 var-5))
   (when var-8 (setf ierr var-8))))
(t
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (clacpy "U" n n a lda
      (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                            ((+ n 1)) ((1 lwork)) cwork-%offset%)
      n)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf n var-6)))
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p n) nil)
   (tagbody
     (setf temp1
             (f2cl-lib:fref sva-%data%
                            ((f2cl-lib:fref iwork (p) ((1 *))))
                            ((1 n)) sva-%offset%))
     (multiple-value-bind (var-0 var-1 var-2 var-3)
         (csscal p (/ one temp1)
          (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                                ((+ n
                                    (f2cl-lib:int-mul
                                     (f2cl-lib:int-add p
                                                       (f2cl-lib:int-sub
                                                        1))
                                     n)
                                    1))
                                ((1 lwork)) cwork-%offset%)
          1)
       (declare (ignore var-1 var-2 var-3))
       (when var-0 (setf p var-0)))
```

```
          label3052))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (cpocon "U" n
             (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                                   ((+ n 1)) ((1 lwork)) cwork-%offset%)
             n one temp1
             (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                                   ((+ n (f2cl-lib:int-mul n n) 1))
                                   ((1 lwork)) cwork-%offset%)
             rwork ierr)
          (declare (ignore var-0 var-2 var-6 var-7))
          (when var-1 (setf n var-1))
          (when var-3 (setf n var-3))
          (when var-4 (setf one var-4))
          (when var-5 (setf temp1 var-5))
          (when var-8 (setf ierr var-8)))))
     (setf sconda (/ one (f2cl-lib:fsqrt temp1))))
    (t (setf sconda (- one))))))
(setf l2pert
        (and l2pert
             (>
              (abs
               (/
                (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *))
                               a-%offset%)
                (f2cl-lib:fref a-%data% (nr nr) ((1 lda) (1 *))
                               a-%offset%)))
              (f2cl-lib:fsqrt big1))))
(cond
 ((not (or rsvec lsvec))
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p
                    (f2cl-lib:min0
                     (f2cl-lib:int-add n (f2cl-lib:int-sub 1)) nr))
                 nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (ccopy (f2cl-lib:int-sub n p)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 (p (f2cl-lib:int-add p 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 ((+ p 1) p)
                                 ((1 lda) (1 *)) a-%offset%)
           1)
        (declare (ignore var-0 var-1 var-3 var-4))
        (when var-2 (setf lda var-2)))
      (clacgv (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (p p)
```

```
                              ((1 lda) (1 *)) a-%offset%)
     1)
   label1946))
(if (= nr n)
    (setf (f2cl-lib:fref a-%data% (n n) ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:conjg
             (f2cl-lib:fref a-%data% (n n) ((1 lda) (1 *))
                            a-%offset%))))
(cond
 ((not almort)
  (cond
   (l2pert (setf xsc (/ epsln (f2cl-lib:ffloat n)))
    (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                  ((> q nr) nil)
      (tagbody
        (setf ctemp
                (f2cl-lib:cmplx
                 (* xsc
                    (abs
                     (f2cl-lib:fref a-%data% (q q) ((1 lda) (1 *))
                                    a-%offset%)))
                 zero))
        (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                      ((> p n) nil)
          (tagbody
            (if (or
                 (and (> p q)
                      (<=
                       (abs
                        (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                       a-%offset%))
                       temp1))
                 (< p q))
                (setf (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                     a-%offset%)
                      ctemp))
           label4949))
       label4947)))
   (t
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
         czero czero
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 2)
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf lda var-6)))))
  (multiple-value-bind
```

```
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (cgeqrf n nr a lda cwork
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                               ((1 lwork)) cwork-%offset%)
         (f2cl-lib:int-sub lwork n) ierr)
      (declare (ignore var-2 var-4 var-5 var-6))
      (when var-0 (setf n var-0))
      (when var-1 (setf nr var-1))
      (when var-3 (setf lda var-3))
      (when var-7 (setf ierr var-7)))
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p (f2cl-lib:int-add nr (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (ccopy (f2cl-lib:int-sub nr p)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   (p (f2cl-lib:int-add p 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   ((+ p 1) p) ((1 lda) (1 *))
                                   a-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf lda var-2)))
        (clacgv (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (p p)
                               ((1 lda) (1 *)) a-%offset%)
         1)
       label1948))))
  (cond
   (l2pert (setf xsc (/ epsln (f2cl-lib:ffloat n)))
    (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                  ((> q nr) nil)
      (tagbody
        (setf ctemp
                (f2cl-lib:cmplx
                 (* xsc
                    (abs
                     (f2cl-lib:fref a-%data% (q q) ((1 lda) (1 *))
                                    a-%offset%)))
                 zero))
        (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                      ((> p nr) nil)
          (tagbody
            (if (or
                 (and (> p q)
                      (<=
                       (abs
                        (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
```

```
                                        a-%offset%))
                          temp1))
                    (< p q))
                   (setf (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                        a-%offset%)
                          ctemp))
             label1949))
         label1947)))
     (t
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (claset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
           czero czero
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 2)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-5))
        (when var-3 (setf czero var-3))
        (when var-4 (setf czero var-4))
        (when var-6 (setf lda var-6)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14 var-15)
        (cgesvj "L" "NoU" "NoV" nr nr a lda sva n v ldv cwork lwork rwork
         lrwork info)
      (declare
       (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-13))
      (when var-3 (setf nr var-3))
      (when var-4 (setf nr var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf n var-8))
      (when var-10 (setf ldv var-10))
      (when var-12 (setf lwork var-12))
      (when var-14 (setf lrwork var-14))
      (when var-15 (setf info var-15)))
    (setf scalem (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%))
    (setf numrank
            (f2cl-lib:nint
             (f2cl-lib:fref rwork-%data% (2) ((1 *)) rwork-%offset%))))
   ((and rsvec (not lsvec))
    (cond
     (almort
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p nr) nil)
        (tagbody
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (ccopy (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (p p)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (p p)
```

```
                                    ((1 ldv) (1 *)) v-%offset%)
          1)
       (declare (ignore var-0 var-1 var-3 var-4))
       (when var-2 (setf lda var-2)))
     (clacgv (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
      (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (p p)
                            ((1 ldv) (1 *)) v-%offset%)
      1)
    label1998))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (claset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
      czero czero
      (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (1 2)
                            ((1 ldv) (1 *)) v-%offset%)
      ldv)
   (declare (ignore var-0 var-1 var-2 var-5))
   (when var-3 (setf czero var-3))
   (when var-4 (setf czero var-4))
   (when var-6 (setf ldv var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13 var-14 var-15)
     (cgesvj "L" "U" "N" n nr v ldv sva nr a lda cwork lwork rwork
      lrwork info)
   (declare
    (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-13))
   (when var-3 (setf n var-3))
   (when var-4 (setf nr var-4))
   (when var-6 (setf ldv var-6))
   (when var-8 (setf nr var-8))
   (when var-10 (setf lda var-10))
   (when var-12 (setf lwork var-12))
   (when var-14 (setf lrwork var-14))
   (when var-15 (setf info var-15)))
 (setf scalem
         (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%))
 (setf numrank
         (f2cl-lib:nint
          (f2cl-lib:fref rwork-%data% (2) ((1 *)) rwork-%offset%))))
(t
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (claset "Lower" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
      czero czero
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (2 1)
                            ((1 lda) (1 *)) a-%offset%)
      lda)
   (declare (ignore var-0 var-1 var-2 var-5))
   (when var-3 (setf czero var-3))
   (when var-4 (setf czero var-4))
   (when var-6 (setf lda var-6)))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgelqf nr n a lda cwork
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (when var-0 (setf nr var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-7 (setf ierr var-7)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "Lower" nr nr a lda v ldv)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf nr var-1))
  (when var-2 (setf nr var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldv var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
     czero czero
     (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (1 2)
                           ((1 ldv) (1 *)) v-%offset%)
     ldv)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldv var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgeqrf nr nr v ldv
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (when var-0 (setf nr var-0))
  (when var-1 (setf nr var-1))
  (when var-3 (setf ldv var-3))
  (when var-7 (setf ierr var-7)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (ccopy (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (p p)
                               ((1 ldv) (1 *)) v-%offset%)
         ldv
```

```
          (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (p p)
                                ((1 ldv) (1 *)) v-%offset%)
          1)
       (declare (ignore var-0 var-1 var-3 var-4))
       (when var-2 (setf ldv var-2)))
     (clacgv (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
      (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (p p)
                            ((1 ldv) (1 *)) v-%offset%)
      1)
    label8998))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
     zero zero
     (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (1 2)
                           ((1 ldv) (1 *)) v-%offset%)
     ldv)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldv var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15)
    (cgesvj "Lower" "U" "N" nr nr v ldv sva nr u ldu
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork n) rwork lrwork info)
  (declare
   (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12
    var-13))
  (when var-3 (setf nr var-3))
  (when var-4 (setf nr var-4))
  (when var-6 (setf ldv var-6))
  (when var-8 (setf nr var-8))
  (when var-10 (setf ldu var-10))
  (when var-14 (setf lrwork var-14))
  (when var-15 (setf info var-15)))
(setf scalem
        (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%))
(setf numrank
        (f2cl-lib:nint
         (f2cl-lib:fref rwork-%data% (2) ((1 *)) rwork-%offset%)))
(cond
 ((< nr n)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "A" (f2cl-lib:int-sub n nr) nr czero czero
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                             ((+ nr 1) 1) ((1 ldv) (1 *))
                             v-%offset%)
       ldv)
```

```
          (declare (ignore var-0 var-1 var-5))
          (when var-2 (setf nr var-2))
          (when var-3 (setf czero var-3))
          (when var-4 (setf czero var-4))
          (when var-6 (setf ldv var-6)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (claset "A" nr (f2cl-lib:int-sub n nr) czero czero
             (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                                   (1 (f2cl-lib:int-add nr 1))
                                   ((1 ldv) (1 *)) v-%offset%)
             ldv)
          (declare (ignore var-0 var-2 var-5))
          (when var-1 (setf nr var-1))
          (when var-3 (setf czero var-3))
          (when var-4 (setf czero var-4))
          (when var-6 (setf ldv var-6)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (claset "A" (f2cl-lib:int-sub n nr) (f2cl-lib:int-sub n nr)
             czero cone
             (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                                   ((+ nr 1) (f2cl-lib:int-add nr 1))
                                   ((1 ldv) (1 *)) v-%offset%)
             ldv)
          (declare (ignore var-0 var-1 var-2 var-5))
          (when var-3 (setf czero var-3))
          (when var-4 (setf cone var-4))
          (when var-6 (setf ldv var-6)))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (cunmlq "Left" "C" n n nr a lda cwork v ldv
           (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                                 ((1 lwork)) cwork-%offset%)
           (f2cl-lib:int-sub lwork n) ierr)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf n var-2))
        (when var-3 (setf n var-3))
        (when var-4 (setf nr var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldv var-9))
        (when var-12 (setf ierr var-12)))))
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (ccopy n
             (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (p 1)
                                   ((1 ldv) (1 *)) v-%offset%)
             ldv
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
```

```
                                    ((f2cl-lib:fref iwork (p) ((1 *))) 1)
                                    ((1 lda) (1 *)) a-%offset%)
            lda)
          (declare (ignore var-1 var-3))
          (when var-0 (setf n var-0))
          (when var-2 (setf ldv var-2))
          (when var-4 (setf lda var-4)))
      label8991))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (clacpy "All" n n a lda v ldv)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf n var-1))
     (when var-2 (setf n var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf ldv var-6)))
   (cond
    (transp
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (clacpy "All" n n v ldv u ldu)
       (declare (ignore var-0 var-3 var-5))
       (when var-1 (setf n var-1))
       (when var-2 (setf n var-2))
       (when var-4 (setf ldv var-4))
       (when var-6 (setf ldu var-6))))))
  ((and lsvec (not rsvec))
   (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                 ((> p nr) nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (ccopy (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (p p)
                                    ((1 lda) (1 *)) a-%offset%)
            lda
            (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (p p)
                                    ((1 ldu) (1 *)) u-%offset%)
            1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf lda var-2)))
       (clacgv (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
        (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (p p)
                                ((1 ldu) (1 *)) u-%offset%)
        1)
      label1965))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (claset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
        czero czero
        (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 2)
                                ((1 ldu) (1 *)) u-%offset%)
        ldu)
     (declare (ignore var-0 var-1 var-2 var-5))
```

```
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgeqrf n nr u ldu
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 lwork))
                           cwork-%offset%)
     (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (when var-0 (setf n var-0))
  (when var-1 (setf nr var-1))
  (when var-3 (setf ldu var-3))
  (when var-7 (setf ierr var-7)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p (f2cl-lib:int-add nr (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (ccopy (f2cl-lib:int-sub nr p)
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8
                               (p (f2cl-lib:int-add p 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 ((+ p 1) p)
                               ((1 ldu) (1 *)) u-%offset%)
         1)
      (declare (ignore var-0 var-1 var-3 var-4))
      (when var-2 (setf ldu var-2)))
    (clacgv (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
     (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (p p)
                           ((1 ldu) (1 *)) u-%offset%)
     1)
   label1967))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
     czero czero
     (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 2)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15)
    (cgesvj "Lower" "U" "N" nr nr u ldu sva nr a lda
```

```
      (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                            ((1 lwork)) cwork-%offset%)
      (f2cl-lib:int-sub lwork n) rwork lrwork info)
  (declare
   (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12 var-13))
  (when var-3 (setf nr var-3))
  (when var-4 (setf nr var-4))
  (when var-6 (setf ldu var-6))
  (when var-8 (setf nr var-8))
  (when var-10 (setf lda var-10))
  (when var-14 (setf lrwork var-14))
  (when var-15 (setf info var-15)))
(setf scalem (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%))
(setf numrank
        (f2cl-lib:nint
         (f2cl-lib:fref rwork-%data% (2) ((1 *)) rwork-%offset%)))
(cond
 ((< nr m)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "A" (f2cl-lib:int-sub m nr) nr czero czero
       (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 ((+ nr 1) 1)
                             ((1 ldu) (1 *)) u-%offset%)
       ldu)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldu var-6)))
  (cond
   ((< nr n1)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "A" nr (f2cl-lib:int-sub n1 nr) czero czero
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8
                               (1 (f2cl-lib:int-add nr 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-2 var-5))
      (when var-1 (setf nr var-1))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldu var-6)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "A" (f2cl-lib:int-sub m nr) (f2cl-lib:int-sub n1 nr)
         czero cone
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8
                               ((+ nr 1) (f2cl-lib:int-add nr 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf czero var-3))
```

```
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldu var-6)))))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (cunmqr "Left" "No Tr" m n1 n a lda cwork u ldu
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n1 var-3))
  (when var-4 (setf n var-4))
  (when var-6 (setf lda var-6))
  (when var-9 (setf ldu var-9))
  (when var-12 (setf ierr var-12)))
(if rowpiv
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claswp n1 u ldu 1 (f2cl-lib:int-sub m 1)
         (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                               ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                               iwork-%offset%)
         -1)
      (declare (ignore var-1 var-3 var-4 var-5 var-6))
      (when var-0 (setf n1 var-0))
      (when var-2 (setf ldu var-2))))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n1) nil)
  (tagbody
    (setf xsc
            (/ one
               (multiple-value-bind (ret-val var-0 var-1 var-2)
                   (scnrm2 m
                    (f2cl-lib:array-slice u-%data% f2cl-lib:complex8
                                          (1 p) ((1 ldu) (1 *))
                                          u-%offset%)
                    1)
                 (declare (ignore var-1 var-2))
                 (when var-0 (setf m var-0))
                 ret-val)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (csscal m xsc
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 p)
                               ((1 ldu) (1 *)) u-%offset%)
         1)
      (declare (ignore var-2 var-3))
      (when var-0 (setf m var-0))
      (when var-1 (setf xsc var-1)))
   label1974))
(cond
```

```
  (transp
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (clacpy "All" n n u ldu v ldv)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf n var-1))
     (when var-2 (setf n var-2))
     (when var-4 (setf ldu var-4))
     (when var-6 (setf ldv var-6))))))
(t
 (cond
  ((not jracc)
   (cond
    ((not almort)
     (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                   ((> p nr) nil)
       (tagbody
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (ccopy (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (p p)
                                    ((1 lda) (1 *)) a-%offset%)
              lda
              (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (p p)
                                    ((1 ldv) (1 *)) v-%offset%)
              1)
           (declare (ignore var-0 var-1 var-3 var-4))
           (when var-2 (setf lda var-2)))
         (clacgv (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
          (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (p p)
                                ((1 ldv) (1 *)) v-%offset%)
          1)
        label1968))
     (cond
      (l2pert (setf xsc (f2cl-lib:fsqrt small))
       (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                     ((> q nr) nil)
         (tagbody
           (setf ctemp
                   (f2cl-lib:cmplx
                    (* xsc
                       (abs
                        (f2cl-lib:fref v-%data% (q q) ((1 ldv) (1 *))
                                       v-%offset%)))
                    zero))
           (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                         ((> p n) nil)
             (tagbody
               (if (or
                    (and (> p q)
                         (<=
                          (abs
```

```
                        (f2cl-lib:fref v-%data% (p q)
                                       ((1 ldv) (1 *)) v-%offset%))
                      temp1))
                 (< p q))
                (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                     v-%offset%)
                      ctemp))
            (if (< p q)
                (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                     v-%offset%)
                      (-
                       (f2cl-lib:fref v-%data% (p q)
                                      ((1 ldv) (1 *))
                                      v-%offset%))))
          label2968))
      label2969)))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (claset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
        czero czero
        (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (1 2)
                              ((1 ldv) (1 *)) v-%offset%)
        ldv)
     (declare (ignore var-0 var-1 var-2 var-5))
     (when var-3 (setf czero var-3))
     (when var-4 (setf czero var-4))
     (when var-6 (setf ldv var-6)))))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (clacpy "L" nr nr v ldv
      (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                            ((+ (f2cl-lib:int-mul 2 n) 1))
                            ((1 lwork)) cwork-%offset%)
      nr)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf nr var-1))
   (when var-2 (setf nr var-2))
   (when var-4 (setf ldv var-4))
   (when var-6 (setf nr var-6)))
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p nr) nil)
   (tagbody
     (setf temp1
             (scnrm2 (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
              (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                                    ((+ (f2cl-lib:int-mul 2 n)
                                        (f2cl-lib:int-mul
                                         (f2cl-lib:int-add p
                                                           (f2cl-lib:int-sub
                                                            1))
```

```
                                        nr)
                                       p))
                                  ((1 lwork)) cwork-%offset%)
              1))
    (csscal (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
     (/ one temp1)
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul
                                (f2cl-lib:int-add p
                                                  (f2cl-lib:int-sub
                                                   1))
                                nr)
                               p))
                           ((1 lwork)) cwork-%offset%)
     1)
   label3950))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cpocon "Lower" nr
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) cwork-%offset%)
     nr one temp1
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul nr nr) 1))
                           ((1 lwork)) cwork-%offset%)
     rwork ierr)
  (declare (ignore var-0 var-2 var-6 var-7))
  (when var-1 (setf nr var-1))
  (when var-3 (setf nr var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf temp1 var-5))
  (when var-8 (setf ierr var-8)))
(setf condr1 (/ one (f2cl-lib:fsqrt temp1)))
(setf cond_ok
        (f2cl-lib:fsqrt (f2cl-lib:fsqrt (f2cl-lib:ffloat nr))))
(cond
 ((< condr1 cond_ok)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (cgeqrf n nr v ldv
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                             ((+ n 1)) ((1 lwork))
                             cwork-%offset%)
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                             ((+ (f2cl-lib:int-mul 2 n) 1))
                             ((1 lwork)) cwork-%offset%)
       (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) ierr)
```

```
  (declare (ignore var-2 var-4 var-5 var-6))
  (when var-0 (setf n var-0))
  (when var-1 (setf nr var-1))
  (when var-3 (setf ldv var-3))
  (when var-7 (setf ierr var-7)))
(cond
 (l2pert (setf xsc (/ (f2cl-lib:fsqrt small) epsln))
  (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                ((> p nr) nil)
    (tagbody
      (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                    ((> q
                        (f2cl-lib:int-add p
                                          (f2cl-lib:int-sub
                                           1)))
                     nil)
        (tagbody
          (setf ctemp
                  (f2cl-lib:cmplx
                   (* xsc
                      (f2cl-lib:amin1
                       (abs
                        (f2cl-lib:fref v-%data% (p p)
                                       ((1 ldv) (1 *))
                                       v-%offset%))
                       (abs
                        (f2cl-lib:fref v-%data% (q q)
                                       ((1 ldv) (1 *))
                                       v-%offset%))))
                   zero))
          (if (<=
               (abs
                (f2cl-lib:fref v-%data% (q p) ((1 ldv) (1 *))
                               v-%offset%))
               temp1)
              (setf (f2cl-lib:fref v-%data% (q p)
                                   ((1 ldv) (1 *)) v-%offset%)
                    ctemp))
         label3958))
     label3959))))
(if (/= nr n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (clacpy "A" n nr v ldv
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                               ((+ (f2cl-lib:int-mul 2 n) 1))
                               ((1 lwork)) cwork-%offset%)
         n)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
```

```
        (when var-2 (setf nr var-2))
        (when var-4 (setf ldv var-4))
        (when var-6 (setf n var-6))))
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p (f2cl-lib:int-add nr (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (ccopy (f2cl-lib:int-sub nr p)
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                                 (p (f2cl-lib:int-add p 1))
                                 ((1 ldv) (1 *)) v-%offset%)
           ldv
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                                 ((+ p 1) p) ((1 ldv) (1 *))
                                 v-%offset%)
           1)
        (declare (ignore var-0 var-1 var-3 var-4))
        (when var-2 (setf ldv var-2)))
      (clacgv (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (p p)
                             ((1 ldv) (1 *)) v-%offset%)
       1)
     label1969))
  (setf (f2cl-lib:fref v-%data% (nr nr) ((1 ldv) (1 *))
                       v-%offset%)
          (f2cl-lib:conjg
           (f2cl-lib:fref v-%data% (nr nr) ((1 ldv) (1 *))
                          v-%offset%)))
  (setf condr2 condr1))
 (t
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p nr) nil)
    (tagbody
      (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add n p))
                           ((1 *)) iwork-%offset%)
            0)
     label3003))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (cgeqp3 n nr v ldv
       (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                             ((+ n 1)) ((1 *)) iwork-%offset%)
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                             ((+ n 1)) ((1 lwork))
                             cwork-%offset%)
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                             ((+ (f2cl-lib:int-mul 2 n) 1))
                             ((1 lwork)) cwork-%offset%)
```

```
      (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) rwork
      ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8))
  (when var-0 (setf n var-0))
  (when var-1 (setf nr var-1))
  (when var-3 (setf ldv var-3))
  (when var-9 (setf ierr var-9)))
(cond
 (l2pert (setf xsc (f2cl-lib:fsqrt small))
  (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                ((> p nr) nil)
    (tagbody
      (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                    ((> q
                        (f2cl-lib:int-add p
                                          (f2cl-lib:int-sub
                                           1)))
                     nil)
        (tagbody
          (setf ctemp
                  (f2cl-lib:cmplx
                   (* xsc
                      (f2cl-lib:amin1
                       (abs
                        (f2cl-lib:fref v-%data% (p p)
                                       ((1 ldv) (1 *))
                                       v-%offset%))
                       (abs
                        (f2cl-lib:fref v-%data% (q q)
                                       ((1 ldv) (1 *))
                                       v-%offset%))))
                   zero))
          (if (<=
               (abs
                (f2cl-lib:fref v-%data% (q p) ((1 ldv) (1 *))
                               v-%offset%))
               temp1)
              (setf (f2cl-lib:fref v-%data% (q p)
                                   ((1 ldv) (1 *)) v-%offset%)
                    ctemp))
          label3968))
      label3969))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "A" n nr v ldv
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) cwork-%offset%)
     n)
  (declare (ignore var-0 var-3 var-5))
```

```
  (when var-1 (setf n var-1))
  (when var-2 (setf nr var-2))
  (when var-4 (setf ldv var-4))
  (when var-6 (setf n var-6)))
(cond
 (l2pert (setf xsc (f2cl-lib:fsqrt small))
  (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                ((> p nr) nil)
    (tagbody
      (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                    ((> q
                        (f2cl-lib:int-add p
                                          (f2cl-lib:int-sub
                                           1)))
                     nil)
        (tagbody
          (setf ctemp
                  (f2cl-lib:cmplx
                   (* xsc
                      (f2cl-lib:amin1
                       (abs
                        (f2cl-lib:fref v-%data% (p p)
                                       ((1 ldv) (1 *))
                                       v-%offset%))
                       (abs
                        (f2cl-lib:fref v-%data% (q q)
                                       ((1 ldv) (1 *))
                                       v-%offset%))))
                   zero))
          (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                               v-%offset%)
                  (- ctemp))
         label8971))
     label8970)))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "L" (f2cl-lib:int-sub nr 1)
       (f2cl-lib:int-sub nr 1) czero czero
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (2 1)
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldv var-6)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgelqf nr nr v ldv
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
```

```
                              ((+ (f2cl-lib:int-mul 2 n)
                                  (f2cl-lib:int-mul n nr) 1))
                              ((1 lwork)) cwork-%offset%)
      (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                              ((+ (f2cl-lib:int-mul 2 n)
                                  (f2cl-lib:int-mul n nr) nr 1))
                              ((1 lwork)) cwork-%offset%)
      (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                        (f2cl-lib:int-mul n nr) nr)
      ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (when var-0 (setf nr var-0))
  (when var-1 (setf nr var-1))
  (when var-3 (setf ldv var-3))
  (when var-7 (setf ierr var-7)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" nr nr v ldv
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                              ((+ (f2cl-lib:int-mul 2 n)
                                  (f2cl-lib:int-mul n nr) nr 1))
                              ((1 lwork)) cwork-%offset%)
     nr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf nr var-1))
  (when var-2 (setf nr var-2))
  (when var-4 (setf ldv var-4))
  (when var-6 (setf nr var-6)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (setf temp1
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (scnrm2 p
                 (f2cl-lib:array-slice cwork-%data%
                                       f2cl-lib:complex8
                                       ((+
                                         (f2cl-lib:int-mul 2 n)
                                         (f2cl-lib:int-mul n
                                                           nr)
                                         nr p))
                                       ((1 lwork))
                                       cwork-%offset%)
                 nr)
              (declare (ignore var-1))
              (when var-0 (setf p var-0))
              (when var-2 (setf nr var-2))
              ret-val))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (csscal p (/ one temp1)
```

```
                (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                                      ((+ (f2cl-lib:int-mul 2 n)
                                          (f2cl-lib:int-mul n nr) nr
                                          p))
                                      ((1 lwork)) cwork-%offset%)
                nr)
            (declare (ignore var-1 var-2))
            (when var-0 (setf p var-0))
            (when var-3 (setf nr var-3)))
        label4950))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (cpocon "L" nr
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                               ((+ (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n nr) nr 1))
                               ((1 lwork)) cwork-%offset%)
         nr one temp1
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                               ((+ (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n nr) nr
                                   (f2cl-lib:int-mul nr nr) 1))
                               ((1 lwork)) cwork-%offset%)
         rwork ierr)
      (declare (ignore var-0 var-2 var-6 var-7))
      (when var-1 (setf nr var-1))
      (when var-3 (setf nr var-3))
      (when var-4 (setf one var-4))
      (when var-5 (setf temp1 var-5))
      (when var-8 (setf ierr var-8)))
    (setf condr2 (/ one (f2cl-lib:fsqrt temp1)))
    (cond
     ((>= condr2 cond_ok)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (clacpy "U" nr nr v ldv
           (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                                 ((+ (f2cl-lib:int-mul 2 n) 1))
                                 ((1 lwork)) cwork-%offset%)
           n)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf nr var-1))
        (when var-2 (setf nr var-2))
        (when var-4 (setf ldv var-4))
        (when var-6 (setf n var-6)))))))
  (cond
   (l2pert (setf xsc (f2cl-lib:fsqrt small))
    (f2cl-lib:fdo (q 2 (f2cl-lib:int-add q 1))
                  ((> q nr) nil)
      (tagbody
```

```
        (setf ctemp
                (* xsc
                   (f2cl-lib:fref v-%data% (q q) ((1 ldv) (1 *))
                                   v-%offset%)))
        (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                      ((> p
                          (f2cl-lib:int-add q
                                            (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                 v-%offset%)
                  (- ctemp))
           label4969))
      label4968)))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (claset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
        czero czero
        (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (1 2)
                              ((1 ldv) (1 *)) v-%offset%)
        ldv)
     (declare (ignore var-0 var-1 var-2 var-5))
     (when var-3 (setf czero var-3))
     (when var-4 (setf czero var-4))
     (when var-6 (setf ldv var-6)))))
 (cond
  ((< condr1 cond_ok)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10 var-11 var-12 var-13 var-14 var-15)
       (cgesvj "L" "U" "N" nr nr v ldv sva nr u ldu
        (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                              ((+ (f2cl-lib:int-mul 2 n)
                                  (f2cl-lib:int-mul n nr) nr 1))
                              ((1 lwork)) cwork-%offset%)
        (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                          (f2cl-lib:int-mul n nr) nr)
        rwork lrwork info)
     (declare
      (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12
       var-13))
     (when var-3 (setf nr var-3))
     (when var-4 (setf nr var-4))
     (when var-6 (setf ldv var-6))
     (when var-8 (setf nr var-8))
     (when var-10 (setf ldu var-10))
     (when var-14 (setf lrwork var-14))
     (when var-15 (setf info var-15)))
```

```
(setf scalem
        (f2cl-lib:fref rwork-%data% (1) ((1 *))
                       rwork-%offset%))
(setf numrank
        (f2cl-lib:nint
         (f2cl-lib:fref rwork-%data% (2) ((1 *))
                        rwork-%offset%)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (ccopy nr
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (1 p)
                               ((1 ldv) (1 *)) v-%offset%)
         1
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 p)
                               ((1 ldu) (1 *)) u-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf nr var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (csscal nr
         (f2cl-lib:array-slice sva-%data% single-float (p)
                               ((1 n)) sva-%offset%)
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (1 p)
                               ((1 ldv) (1 *)) v-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf nr var-0)))
   label3970))
(cond
 ((= nr n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (ctrsm "L" "U" "N" "N" nr nr cone a lda v ldv)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf nr var-4))
    (when var-5 (setf nr var-5))
    (when var-6 (setf cone var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldv var-10))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (ctrsm "L" "U" "C" "N" nr nr cone
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                             ((+ (f2cl-lib:int-mul 2 n) 1))
                             ((1 lwork)) cwork-%offset%)
```

```
      n v ldv)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf nr var-4))
  (when var-5 (setf nr var-5))
  (when var-6 (setf cone var-6))
  (when var-8 (setf n var-8))
  (when var-10 (setf ldv var-10)))
(cond
 ((< nr n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "A" (f2cl-lib:int-sub n nr) nr zero czero
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                             ((+ nr 1) 1) ((1 ldv) (1 *))
                             v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "A" nr (f2cl-lib:int-sub n nr) zero czero
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                             (1 (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-2 var-5))
    (when var-1 (setf nr var-1))
    (when var-3 (setf zero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "A" (f2cl-lib:int-sub n nr)
       (f2cl-lib:int-sub n nr) zero cone
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                             ((+ nr 1)
                              (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf zero var-3))
    (when var-4 (setf cone var-4))
    (when var-6 (setf ldv var-6)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (cunmqr "L" "N" n n nr
```

```
          (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                                ((+ (f2cl-lib:int-mul 2 n) 1))
                                ((1 lwork)) cwork-%offset%)
          n
          (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                                ((+ n 1)) ((1 lwork))
                                cwork-%offset%)
          v ldv
          (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                                ((+ (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul n nr) nr 1))
                                ((1 lwork)) cwork-%offset%)
          (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                            (f2cl-lib:int-mul n nr) nr)
          ierr)
        (declare
         (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf n var-2))
        (when var-3 (setf n var-3))
        (when var-4 (setf nr var-4))
        (when var-6 (setf n var-6))
        (when var-9 (setf ldv var-9))
        (when var-12 (setf ierr var-12))))))
((< condr2 cond_ok)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12 var-13 var-14 var-15)
     (cgesvj "L" "U" "N" nr nr v ldv sva nr u ldu
      (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                            ((+ (f2cl-lib:int-mul 2 n)
                                (f2cl-lib:int-mul n nr) nr 1))
                            ((1 lwork)) cwork-%offset%)
      (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                        (f2cl-lib:int-mul n nr) nr)
      rwork lrwork info)
   (declare
    (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12
     var-13))
   (when var-3 (setf nr var-3))
   (when var-4 (setf nr var-4))
   (when var-6 (setf ldv var-6))
   (when var-8 (setf nr var-8))
   (when var-10 (setf ldu var-10))
   (when var-14 (setf lrwork var-14))
   (when var-15 (setf info var-15)))
 (setf scalem
         (f2cl-lib:fref rwork-%data% (1) ((1 *))
                        rwork-%offset%))
 (setf numrank
         (f2cl-lib:nint
```

```
          (f2cl-lib:fref rwork-%data% (2) ((1 *))
                         rwork-%offset%)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (ccopy nr
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (1 p)
                               ((1 ldv) (1 *)) v-%offset%)
         1
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 p)
                               ((1 ldu) (1 *)) u-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf nr var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (csscal nr
         (f2cl-lib:array-slice sva-%data% single-float (p)
                               ((1 n)) sva-%offset%)
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 p)
                               ((1 ldu) (1 *)) u-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf nr var-0)))
   label3870))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (ctrsm "L" "U" "N" "N" nr nr cone
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) cwork-%offset%)
     n u ldu)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf nr var-4))
  (when var-5 (setf nr var-5))
  (when var-6 (setf cone var-6))
  (when var-8 (setf n var-8))
  (when var-10 (setf ldu var-10)))
(f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
              ((> q nr) nil)
  (tagbody
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p nr) nil)
      (tagbody
        (setf (f2cl-lib:fref cwork-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) nr
                               (f2cl-lib:fref iwork
```

```
                                                    ((f2cl-lib:int-add
                                                      n p))
                                                    ((1 *)))))
                                  ((1 lwork)) cwork-%offset%)
                     (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                                    u-%offset%))
          label872))
        (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                      ((> p nr) nil)
          (tagbody
            (setf (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                                 u-%offset%)
                     (f2cl-lib:fref cwork-%data%
                                    ((f2cl-lib:int-add
                                      (f2cl-lib:int-mul 2 n)
                                      (f2cl-lib:int-mul n nr) nr p))
                                    ((1 lwork)) cwork-%offset%))
          label874))
      label873))
    (cond
     ((< nr n)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (claset "A" (f2cl-lib:int-sub n nr) nr czero czero
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                                 ((+ nr 1) 1) ((1 ldv) (1 *))
                                 v-%offset%)
           ldv)
        (declare (ignore var-0 var-1 var-5))
        (when var-2 (setf nr var-2))
        (when var-3 (setf czero var-3))
        (when var-4 (setf czero var-4))
        (when var-6 (setf ldv var-6)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (claset "A" nr (f2cl-lib:int-sub n nr) czero czero
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                                 (1 (f2cl-lib:int-add nr 1))
                                 ((1 ldv) (1 *)) v-%offset%)
           ldv)
        (declare (ignore var-0 var-2 var-5))
        (when var-1 (setf nr var-1))
        (when var-3 (setf czero var-3))
        (when var-4 (setf czero var-4))
        (when var-6 (setf ldv var-6)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (claset "A" (f2cl-lib:int-sub n nr)
           (f2cl-lib:int-sub n nr) czero cone
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
```

```
                                ((+ nr 1) (f2cl-lib:int-add nr 1))
                                ((1 ldv) (1 *)) v-%offset%)
        ldv)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf czero var-3))
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldv var-6)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (cunmqr "L" "N" n n nr
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                             ((+ (f2cl-lib:int-mul 2 n) 1))
                             ((1 lwork)) cwork-%offset%)
       n
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                             ((+ n 1)) ((1 lwork))
                             cwork-%offset%)
       v ldv
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                             ((+ (f2cl-lib:int-mul 2 n)
                                 (f2cl-lib:int-mul n nr) nr 1))
                             ((1 lwork)) cwork-%offset%)
       (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                         (f2cl-lib:int-mul n nr) nr)
       ierr)
    (declare
     (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf nr var-4))
    (when var-6 (setf n var-6))
    (when var-9 (setf ldv var-9))
    (when var-12 (setf ierr var-12))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13 var-14 var-15)
      (cgesvj "L" "U" "V" nr nr v ldv sva nr u ldu
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                             ((+ (f2cl-lib:int-mul 2 n)
                                 (f2cl-lib:int-mul n nr) nr 1))
                             ((1 lwork)) cwork-%offset%)
       (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                         (f2cl-lib:int-mul n nr) nr)
       rwork lrwork info)
    (declare
     (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12
      var-13))
    (when var-3 (setf nr var-3))
```

```
  (when var-4 (setf nr var-4))
  (when var-6 (setf ldv var-6))
  (when var-8 (setf nr var-8))
  (when var-10 (setf ldu var-10))
  (when var-14 (setf lrwork var-14))
  (when var-15 (setf info var-15)))
(setf scalem
        (f2cl-lib:fref rwork-%data% (1) ((1 *))
                       rwork-%offset%))
(setf numrank
        (f2cl-lib:nint
         (f2cl-lib:fref rwork-%data% (2) ((1 *))
                        rwork-%offset%)))
(cond
 ((< nr n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "A" (f2cl-lib:int-sub n nr) nr czero czero
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                             ((+ nr 1) 1) ((1 ldv) (1 *))
                             v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "A" nr (f2cl-lib:int-sub n nr) czero czero
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                             (1 (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-2 var-5))
    (when var-1 (setf nr var-1))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "A" (f2cl-lib:int-sub n nr)
       (f2cl-lib:int-sub n nr) czero cone
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                             ((+ nr 1) (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf czero var-3))
    (when var-4 (setf cone var-4))
```

```
    (when var-6 (setf ldv var-6)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (cunmqr "L" "N" n n nr
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) cwork-%offset%)
     n
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ n 1)) ((1 lwork))
                           cwork-%offset%)
     v ldv
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) nr 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                       (f2cl-lib:int-mul n nr) nr)
     ierr)
  (declare
   (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf nr var-4))
  (when var-6 (setf n var-6))
  (when var-9 (setf ldv var-9))
  (when var-12 (setf ierr var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (cunmlq "L" "C" nr nr nr
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) cwork-%offset%)
     n
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) 1))
                           ((1 lwork)) cwork-%offset%)
     u ldu
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) nr 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                       (f2cl-lib:int-mul n nr) nr)
     ierr)
  (declare
   (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
```

```
    (when var-2 (setf nr var-2))
    (when var-3 (setf nr var-3))
    (when var-4 (setf nr var-4))
    (when var-6 (setf n var-6))
    (when var-9 (setf ldu var-9))
    (when var-12 (setf ierr var-12)))
  (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                ((> q nr) nil)
    (tagbody
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p nr) nil)
        (tagbody
          (setf (f2cl-lib:fref cwork-%data%
                               ((f2cl-lib:int-add
                                 (f2cl-lib:int-mul 2 n)
                                 (f2cl-lib:int-mul n nr) nr
                                 (f2cl-lib:fref iwork
                                                ((f2cl-lib:int-add
                                                  n p))
                                                ((1 *)))))
                               ((1 lwork)) cwork-%offset%)
                  (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                                 u-%offset%))
        label772))
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p nr) nil)
        (tagbody
          (setf (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                               u-%offset%)
                  (f2cl-lib:fref cwork-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n nr) nr p))
                                 ((1 lwork)) cwork-%offset%))
        label774))
     label773))))
(setf temp1 (* (f2cl-lib:fsqrt (f2cl-lib:ffloat n)) epsln))
(f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
              ((> q n) nil)
  (tagbody
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (setf (f2cl-lib:fref cwork-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) nr
                               (f2cl-lib:fref iwork (p)
                                              ((1 *)))))
                             ((1 lwork)) cwork-%offset%)
```

```
                (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                               v-%offset%))
        label972))
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p n) nil)
        (tagbody
          (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                               v-%offset%)
                (f2cl-lib:fref cwork-%data%
                               ((f2cl-lib:int-add
                                 (f2cl-lib:int-mul 2 n)
                                 (f2cl-lib:int-mul n nr) nr p))
                               ((1 lwork)) cwork-%offset%))
        label973))
      (setf xsc
              (/ one
                 (multiple-value-bind (ret-val var-0 var-1 var-2)
                     (scnrm2 n
                      (f2cl-lib:array-slice v-%data%
                                            f2cl-lib:complex8 (1 q)
                                            ((1 ldv) (1 *))
                                            v-%offset%)
                      1)
                   (declare (ignore var-1 var-2))
                   (when var-0 (setf n var-0))
                   ret-val)))
      (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (csscal n xsc
               (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                                     (1 q) ((1 ldv) (1 *))
                                     v-%offset%)
               1)
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf xsc var-1))))
    label1972))
  (cond
   ((< nr m)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "A" (f2cl-lib:int-sub m nr) nr czero czero
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8
                               ((+ nr 1) 1) ((1 ldu) (1 *))
                               u-%offset%)
        ldu)
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf nr var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
```

```
    (when var-6 (setf ldu var-6)))
  (cond
   ((< nr n1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "A" nr (f2cl-lib:int-sub n1 nr) czero czero
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8
                               (1 (f2cl-lib:int-add nr 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-2 var-5))
      (when var-1 (setf nr var-1))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldu var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "A" (f2cl-lib:int-sub m nr)
         (f2cl-lib:int-sub n1 nr) czero cone
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8
                               ((+ nr 1) (f2cl-lib:int-add nr 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf czero var-3))
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldu var-6)))))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (cunmqr "Left" "No_Tr" m n1 n a lda cwork u ldu
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ n 1)) ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n1 var-3))
  (when var-4 (setf n var-4))
  (when var-6 (setf lda var-6))
  (when var-9 (setf ldu var-9))
  (when var-12 (setf ierr var-12)))
(setf temp1 (* (f2cl-lib:fsqrt (f2cl-lib:ffloat m)) epsln))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (setf xsc
            (/ one
               (multiple-value-bind (ret-val var-0 var-1 var-2)
                   (scnrm2 m
                    (f2cl-lib:array-slice u-%data%
```

```
                                                f2cl-lib:complex8 (1 p)
                                                ((1 ldu) (1 *))
                                                u-%offset%)
                          1)
                       (declare (ignore var-1 var-2))
                       (when var-0 (setf m var-0))
                       ret-val)))
         (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
             (multiple-value-bind (var-0 var-1 var-2 var-3)
                 (csscal m xsc
                  (f2cl-lib:array-slice u-%data% f2cl-lib:complex8
                                        (1 p) ((1 ldu) (1 *))
                                        u-%offset%)
                  1)
               (declare (ignore var-2 var-3))
               (when var-0 (setf m var-0))
               (when var-1 (setf xsc var-1))))
       label1973))
     (if rowpiv
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
             (claswp n1 u ldu 1 (f2cl-lib:int-sub m 1)
              (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                                    ((+ (f2cl-lib:int-mul 2 n) 1))
                                    ((1 *)) iwork-%offset%)
              -1)
           (declare (ignore var-1 var-3 var-4 var-5 var-6))
           (when var-0 (setf n1 var-0))
           (when var-2 (setf ldu var-2)))))
    (t
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (clacpy "Upper" n n a lda
          (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                                ((+ n 1)) ((1 lwork)) cwork-%offset%)
          n)
       (declare (ignore var-0 var-3 var-5))
       (when var-1 (setf n var-1))
       (when var-2 (setf n var-2))
       (when var-4 (setf lda var-4))
       (when var-6 (setf n var-6)))
     (cond
      (l2pert (setf xsc (f2cl-lib:fsqrt small))
       (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                     ((> p n) nil)
         (tagbody
           (setf ctemp
                 (* xsc
                    (f2cl-lib:fref cwork-%data%
                                   ((f2cl-lib:int-add n
                                                      (f2cl-lib:int-mul
```

```
                                                        (f2cl-lib:int-sub
                                                         p 1)
                                                        n)
                                                       p))
                                  ((1 lwork)) cwork-%offset%)))
        (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                      ((> q
                          (f2cl-lib:int-add p
                                            (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref cwork-%data%
                                 ((f2cl-lib:int-add n
                                                    (f2cl-lib:int-mul
                                                     (f2cl-lib:int-sub
                                                      q 1)
                                                     n)
                                                    p))
                                 ((1 lwork)) cwork-%offset%)
                  (- ctemp))
           label5971))
      label5970)))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (claset "Lower" (f2cl-lib:int-sub n 1)
        (f2cl-lib:int-sub n 1) czero czero
        (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                              ((+ n 2)) ((1 lwork))
                              cwork-%offset%)
        n)
     (declare (ignore var-0 var-1 var-2 var-5))
     (when var-3 (setf czero var-3))
     (when var-4 (setf czero var-4))
     (when var-6 (setf n var-6)))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13 var-14 var-15)
     (cgesvj "Upper" "U" "N" n n
      (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                            ((+ n 1)) ((1 lwork)) cwork-%offset%)
      n sva n u ldu
      (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                            ((+ n (f2cl-lib:int-mul n n) 1))
                            ((1 lwork)) cwork-%offset%)
      (f2cl-lib:int-sub lwork n (f2cl-lib:int-mul n n)) rwork
      lrwork info)
   (declare
    (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12
     var-13))
```

```
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-6 (setf n var-6))
  (when var-8 (setf n var-8))
  (when var-10 (setf ldu var-10))
  (when var-14 (setf lrwork var-14))
  (when var-15 (setf info var-15)))
(setf scalem
        (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%))
(setf numrank
        (f2cl-lib:nint
         (f2cl-lib:fref rwork-%data% (2) ((1 *))
                        rwork-%offset%)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (ccopy n
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                               ((+ n
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-add p
                                                      (f2cl-lib:int-sub
                                                       1))
                                    n)
                                   1))
                               ((1 lwork)) cwork-%offset%)
         1
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 p)
                               ((1 ldu) (1 *)) u-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (csscal n
         (f2cl-lib:array-slice sva-%data% single-float (p)
                               ((1 n)) sva-%offset%)
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                               ((+ n
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-add p
                                                      (f2cl-lib:int-sub
                                                       1))
                                    n)
                                   1))
                               ((1 lwork)) cwork-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf n var-0)))
   label6970))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ctrsm "Left" "Upper" "NoTrans" "No UD" n n cone a lda
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ n 1)) ((1 lwork)) cwork-%offset%)
     n)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-6 (setf cone var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf n var-10)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (ccopy n
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                               ((+ n p)) ((1 lwork))
                               cwork-%offset%)
         n
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                               ((f2cl-lib:fref iwork (p) ((1 *)))
                                1)
                               ((1 ldv) (1 *)) v-%offset%)
         ldv)
      (declare (ignore var-1 var-3))
      (when var-0 (setf n var-0))
      (when var-2 (setf n var-2))
      (when var-4 (setf ldv var-4)))
   label6972))
(setf temp1 (* (f2cl-lib:fsqrt (f2cl-lib:ffloat n)) epsln))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (setf xsc
            (/ one
               (multiple-value-bind (ret-val var-0 var-1 var-2)
                   (scnrm2 n
                    (f2cl-lib:array-slice v-%data%
                                          f2cl-lib:complex8 (1 p)
                                          ((1 ldv) (1 *))
                                          v-%offset%)
                    1)
                 (declare (ignore var-1 var-2))
                 (when var-0 (setf n var-0))
                 ret-val)))
    (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
```

```
            (csscal n xsc
             (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                                   (1 p) ((1 ldv) (1 *))
                                   v-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-1 (setf xsc var-1))))
   label6971))
(cond
 ((< n m)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "A" (f2cl-lib:int-sub m n) n czero czero
       (f2cl-lib:array-slice u-%data% f2cl-lib:complex8
                             ((+ n 1) 1) ((1 ldu) (1 *))
                             u-%offset%)
       ldu)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf n var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldu var-6)))
  (cond
   ((< n n1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "A" n (f2cl-lib:int-sub n1 n) czero czero
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8
                               (1 (f2cl-lib:int-add n 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-2 var-5))
      (when var-1 (setf n var-1))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldu var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "A" (f2cl-lib:int-sub m n)
         (f2cl-lib:int-sub n1 n) czero cone
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8
                               ((+ n 1) (f2cl-lib:int-add n 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf czero var-3))
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldu var-6)))))))
(multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (cunmqr "Left" "No Tr" m n1 n a lda cwork u ldu
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ n 1)) ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n1 var-3))
  (when var-4 (setf n var-4))
  (when var-6 (setf lda var-6))
  (when var-9 (setf ldu var-9))
  (when var-12 (setf ierr var-12)))
(setf temp1 (* (f2cl-lib:fsqrt (f2cl-lib:ffloat m)) epsln))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n1) nil)
  (tagbody
    (setf xsc
            (/ one
               (multiple-value-bind (ret-val var-0 var-1 var-2)
                   (scnrm2 m
                    (f2cl-lib:array-slice u-%data%
                                          f2cl-lib:complex8 (1 p)
                                          ((1 ldu) (1 *))
                                          u-%offset%)
                    1)
                 (declare (ignore var-1 var-2))
                 (when var-0 (setf m var-0))
                 ret-val)))
    (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (csscal m xsc
             (f2cl-lib:array-slice u-%data% f2cl-lib:complex8
                                   (1 p) ((1 ldu) (1 *))
                                   u-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf xsc var-1))))
   label6973))
(if rowpiv
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claswp n1 u ldu 1 (f2cl-lib:int-sub m 1)
         (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                               ((+ (f2cl-lib:int-mul 2 n) 1))
                               ((1 *)) iwork-%offset%)
         -1)
      (declare (ignore var-1 var-3 var-4 var-5 var-6))
      (when var-0 (setf n1 var-0))
```

```
          (when var-2 (setf ldu var-2)))))))
      (t
       (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                     ((> p nr) nil)
         (tagbody
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
               (ccopy (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (p p)
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (p p)
                                      ((1 ldv) (1 *)) v-%offset%)
                1)
             (declare (ignore var-0 var-1 var-3 var-4))
             (when var-2 (setf lda var-2)))
           (clacgv (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
            (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (p p)
                                  ((1 ldv) (1 *)) v-%offset%)
            1)
          label7968))
       (cond
        (l2pert (setf xsc (f2cl-lib:fsqrt (/ small epsln)))
         (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                       ((> q nr) nil)
           (tagbody
             (setf ctemp
                     (f2cl-lib:cmplx
                      (* xsc
                         (abs
                          (f2cl-lib:fref v-%data% (q q) ((1 ldv) (1 *))
                                         v-%offset%)))
                      zero))
             (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                           ((> p n) nil)
               (tagbody
                 (if (or
                      (and (> p q)
                           (<=
                            (abs
                             (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                            v-%offset%))
                            temp1))
                      (< p q))
                     (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                          v-%offset%)
                            ctemp))
                 (if (< p q)
                     (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                          v-%offset%)
                            (-
```

```
                        (f2cl-lib:fref v-%data% (p q)
                                       ((1 ldv) (1 *))
                                       v-%offset%))))
         label5968))
     label5969)))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
       czero czero
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (1 2)
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldv var-6)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgeqrf n nr v ldv
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (when var-0 (setf n var-0))
  (when var-1 (setf nr var-1))
  (when var-3 (setf ldv var-3))
  (when var-7 (setf ierr var-7)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" n nr v ldv
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) cwork-%offset%)
     n)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nr var-2))
  (when var-4 (setf ldv var-4))
  (when var-6 (setf n var-6)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (ccopy (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (p p)
                               ((1 ldv) (1 *)) v-%offset%)
         ldv
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (p p)
```

```
                                ((1 ldu) (1 *)) u-%offset%)
          1)
        (declare (ignore var-0 var-1 var-3 var-4))
        (when var-2 (setf ldv var-2)))
      (clacgv (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
       (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (p p)
                             ((1 ldu) (1 *)) u-%offset%)
       1)
     label7969))
  (cond
   (l2pert (setf xsc (f2cl-lib:fsqrt (/ small epsln)))
    (f2cl-lib:fdo (q 2 (f2cl-lib:int-add q 1))
                  ((> q nr) nil)
      (tagbody
        (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                      ((> p
                          (f2cl-lib:int-add q (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf ctemp
                    (f2cl-lib:cmplx
                     (* xsc
                        (f2cl-lib:amin1
                         (abs
                          (f2cl-lib:fref u-%data% (p p)
                                         ((1 ldu) (1 *)) u-%offset%))
                         (abs
                          (f2cl-lib:fref u-%data% (q q)
                                         ((1 ldu) (1 *))
                                         u-%offset%))))
                     zero))
            (setf (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                                 u-%offset%)
                    (- ctemp))
          label9971))
      label9970)))
   (t
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
         czero czero
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 2)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldu var-6)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
```

```
    (cgesvj "L" "U" "V" nr nr u ldu sva n v ldv
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                       (f2cl-lib:int-mul n nr))
     rwork lrwork info)
  (declare
   (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12
    var-13))
  (when var-3 (setf nr var-3))
  (when var-4 (setf nr var-4))
  (when var-6 (setf ldu var-6))
  (when var-8 (setf n var-8))
  (when var-10 (setf ldv var-10))
  (when var-14 (setf lrwork var-14))
  (when var-15 (setf info var-15)))
(setf scalem
        (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%))
(setf numrank
        (f2cl-lib:nint
         (f2cl-lib:fref rwork-%data% (2) ((1 *)) rwork-%offset%)))
(cond
 ((< nr n)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "A" (f2cl-lib:int-sub n nr) nr zero zero
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                             ((+ nr 1) 1) ((1 ldv) (1 *))
                             v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "A" nr (f2cl-lib:int-sub n nr) zero zero
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                             (1 (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-2 var-5))
    (when var-1 (setf nr var-1))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "A" (f2cl-lib:int-sub n nr) (f2cl-lib:int-sub n nr)
       zero one
```

```
        (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                              ((+ nr 1) (f2cl-lib:int-add nr 1))
                              ((1 ldv) (1 *)) v-%offset%)
        ldv)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldv var-6)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (cunmqr "L" "N" n n nr
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) cwork-%offset%)
     n
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                           ((1 lwork)) cwork-%offset%)
     v ldv
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) nr 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                       (f2cl-lib:int-mul n nr) nr)
     ierr)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf nr var-4))
  (when var-6 (setf n var-6))
  (when var-9 (setf ldv var-9))
  (when var-12 (setf ierr var-12)))
(setf temp1 (* (f2cl-lib:fsqrt (f2cl-lib:ffloat n)) epsln))
(f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
              ((> q n) nil)
  (tagbody
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (setf (f2cl-lib:fref cwork-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) nr
                               (f2cl-lib:fref iwork (p) ((1 *)))))
                             ((1 lwork)) cwork-%offset%)
              (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                             v-%offset%))
        label8972))
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
```

```
                    ((> p n) nil)
          (tagbody
            (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                 v-%offset%)
                  (f2cl-lib:fref cwork-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n nr) nr p))
                                 ((1 lwork)) cwork-%offset%))
           label8973))
        (setf xsc
                (/ one
                   (multiple-value-bind (ret-val var-0 var-1 var-2)
                       (scnrm2 n
                        (f2cl-lib:array-slice v-%data%
                                              f2cl-lib:complex8 (1 q)
                                              ((1 ldv) (1 *))
                                              v-%offset%)
                        1)
                      (declare (ignore var-1 var-2))
                      (when var-0 (setf n var-0))
                      ret-val)))
        (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (csscal n xsc
                 (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (1 q)
                                       ((1 ldv) (1 *)) v-%offset%)
                 1)
              (declare (ignore var-2 var-3))
              (when var-0 (setf n var-0))
              (when var-1 (setf xsc var-1))))
       label7972))
    (cond
     ((< nr m)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (claset "A" (f2cl-lib:int-sub m nr) nr czero czero
           (f2cl-lib:array-slice u-%data% f2cl-lib:complex8
                                 ((+ nr 1) 1) ((1 ldu) (1 *))
                                 u-%offset%)
           ldu)
        (declare (ignore var-0 var-1 var-5))
        (when var-2 (setf nr var-2))
        (when var-3 (setf czero var-3))
        (when var-4 (setf czero var-4))
        (when var-6 (setf ldu var-6)))
      (cond
       ((< nr n1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (claset "A" nr (f2cl-lib:int-sub n1 nr) czero czero
```

```
              (f2cl-lib:array-slice u-%data% f2cl-lib:complex8
                                    (1 (f2cl-lib:int-add nr 1))
                                    ((1 ldu) (1 *)) u-%offset%)
              ldu)
            (declare (ignore var-0 var-2 var-5))
            (when var-1 (setf nr var-1))
            (when var-3 (setf czero var-3))
            (when var-4 (setf czero var-4))
            (when var-6 (setf ldu var-6)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (claset "A" (f2cl-lib:int-sub m nr)
               (f2cl-lib:int-sub n1 nr) czero cone
               (f2cl-lib:array-slice u-%data% f2cl-lib:complex8
                                    ((+ nr 1) (f2cl-lib:int-add nr 1))
                                    ((1 ldu) (1 *)) u-%offset%)
              ldu)
            (declare (ignore var-0 var-1 var-2 var-5))
            (when var-3 (setf czero var-3))
            (when var-4 (setf cone var-4))
            (when var-6 (setf ldu var-6)))))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (cunmqr "Left" "No Tr" m n1 n a lda cwork u ldu
           (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                                 ((1 lwork)) cwork-%offset%)
           (f2cl-lib:int-sub lwork n) ierr)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf n1 var-3))
        (when var-4 (setf n var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldu var-9))
        (when var-12 (setf ierr var-12)))
      (if rowpiv
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (claswp n1 u ldu 1 (f2cl-lib:int-sub m 1)
               (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                                    ((+ (f2cl-lib:int-mul 2 n) 1))
                                    ((1 *)) iwork-%offset%)
              -1)
            (declare (ignore var-1 var-3 var-4 var-5 var-6))
            (when var-0 (setf n1 var-0))
            (when var-2 (setf ldu var-2))))))
    (cond
     (transp
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p n) nil)
```

```
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (cswap n
             (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 p)
                                   ((1 ldu) (1 *)) u-%offset%)
             1
             (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (1 p)
                                   ((1 ldv) (1 *)) v-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf n var-0)))
       label6974))))))
(cond
 ((<= uscal2
      (* (f2cl-lib:f2cl/ big (f2cl-lib:fref sva (1) ((1 n)))) uscal1))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 uscal1 uscal2 nr 1 sva n ierr)
    (declare (ignore var-0 var-1 var-2 var-6 var-7))
    (when var-3 (setf uscal1 var-3))
    (when var-4 (setf uscal2 var-4))
    (when var-5 (setf nr var-5))
    (when var-8 (setf n var-8))
    (when var-9 (setf ierr var-9)))
  (setf uscal1 one) (setf uscal2 one)))
(cond
 ((< nr n)
  (f2cl-lib:fdo (p (f2cl-lib:int-add nr 1) (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%) zero)
     label3004))))
(setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
        (* uscal2 scalem))
(setf (f2cl-lib:fref rwork-%data% (2) ((1 *)) rwork-%offset%) uscal1)
(if errest
    (setf (f2cl-lib:fref rwork-%data% (3) ((1 *)) rwork-%offset%)
            sconda))
(cond
 ((and lsvec rsvec)
  (setf (f2cl-lib:fref rwork-%data% (4) ((1 *)) rwork-%offset%) condr1)
  (setf (f2cl-lib:fref rwork-%data% (5) ((1 *)) rwork-%offset%)
          condr2)))
(cond
 (l2tran
  (setf (f2cl-lib:fref rwork-%data% (6) ((1 *)) rwork-%offset%) entra)
  (setf (f2cl-lib:fref rwork-%data% (7) ((1 *)) rwork-%offset%)
          entrat)))
(setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) nr)
(setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%) numrank)
```

```
        (setf (f2cl-lib:fref iwork-%data% (3) ((1 *)) iwork-%offset%) warning)
        (go end_label)
       end_label
        (return
         (values joba jobu jobv jobr jobt jobp m n nil lda nil nil ldu nil ldv
                 nil lwork nil lrwork nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgejsv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::joba
                                              fortran-to-lisp::jobu
                                              fortran-to-lisp::jobv
                                              fortran-to-lisp::jobr
                                              fortran-to-lisp::jobt
                                              fortran-to-lisp::jobp
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::ldu nil
                                              fortran-to-lisp::ldv nil
                                              fortran-to-lisp::lwork nil
                                              fortran-to-lisp::lrwork nil
                                              fortran-to-lisp::info)
```

```
                                        :calls 'nil)))
```

———-

30.2 cgesdd

30.2.1 Axiom unit tests

— **cgesdd.input** —

```
)set break resume
)sys rm -f cgesdd.output
)spool cgesdd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

30.2.2 Axiom help page

— **cgesdd.help** —

```
======================================================================
cgesdd examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgesdd (JOBZ, M, N, A, LDA, S, U, LDU, VT, LDVT, WORK,
           LWORK, RWORK, IWORK, INFO)
           CGESDD

Function/Subroutine Documentation
   subroutine cgesdd (character JOBZ, integer M, integer N, complex,
```

```
      dimension( lda, * ) A, integer LDA, real, dimension( * ) S, complex,
      dimension( ldu, * ) U, integer LDU, complex, dimension( ldvt, * ) VT,
      integer LDVT, complex, dimension( * ) WORK, integer LWORK, real,
      dimension( * ) RWORK, integer, dimension( * ) IWORK, integer INFO)
      CGESDD

Purpose:
   CGESDD computes the singular value decomposition (SVD) of a complex
   M-by-N matrix A, optionally computing the left and/or right singular
   vectors, by using divide-and-conquer method. The SVD is written

      A = U * SIGMA * conjugate-transpose(V)

   where SIGMA is an M-by-N matrix which is zero except for its
   min(m,n) diagonal elements, U is an M-by-M unitary matrix, and
   V is an N-by-N unitary matrix.  The diagonal elements of SIGMA
   are the singular values of A; they are real and non-negative, and
   are returned in descending order.  The first min(m,n) columns of
   U and V are the left and right singular vectors of A.

   Note that the routine returns VT = V**H, not V.

   The divide and conquer algorithm makes very mild assumptions about
   floating point arithmetic. It will work on machines with a guard
   digit in add/subtract, or on those binary machines without guard
   digits which subtract like the Cray X-MP, Cray Y-MP, Cray C-90, or
   Cray-2. It could conceivably fail on hexadecimal or decimal machines
   without guard digits, but we know of none.

Parameters:

 JOBZ
   JOBZ is CHARACTER*1
   Specifies options for computing all or part of the matrix U:
   = 'A':  all M columns of U and all N rows of V**H are
           returned in the arrays U and VT;
   = 'S':  the first min(M,N) columns of U and the first
           min(M,N) rows of V**H are returned in the arrays U and VT;
   = 'O':  If M >= N, the first N columns of U are overwritten
           in the array A and all rows of V**H are returned in the array VT;
           otherwise, all columns of U are returned in the
           array U and the first M rows of V**H are overwritten
           in the array A;
   = 'N':  no columns of U or rows of V**H are computed.

 M
   M is INTEGER
   The number of rows of the input matrix A.  M >= 0.

 N
```

```
  N is INTEGER
  The number of columns of the input matrix A.  N >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit,
  if JOBZ = 'O',  A is overwritten with the first N columns
      of U (the left singular vectors, stored columnwise) if M >= N;
      A is overwritten with the first M rows
      of V**H (the right singular vectors, stored rowwise) otherwise.
      if JOBZ .ne. 'O', the contents of A are destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

S
  S is REAL array, dimension (min(M,N))
  The singular values of A, sorted so that S(i) >= S(i+1).

U
  U is COMPLEX array, dimension (LDU,UCOL)
  UCOL = M if JOBZ = 'A' or JOBZ = 'O' and M < N;
  UCOL = min(M,N) if JOBZ = 'S'.
  If JOBZ = 'A' or JOBZ = 'O' and M < N, U contains the M-by-M
  unitary matrix U;
  if JOBZ = 'S', U contains the first min(M,N) columns of U
  (the left singular vectors, stored columnwise);
  if JOBZ = 'O' and M >= N, or JOBZ = 'N', U is not referenced.

LDU
  LDU is INTEGER
  The leading dimension of the array U.  LDU >= 1; if
  JOBZ = 'S' or 'A' or JOBZ = 'O' and M < N, LDU >= M.

VT
  VT is COMPLEX array, dimension (LDVT,N)
  If JOBZ = 'A' or JOBZ = 'O' and M >= N, VT contains the
  N-by-N unitary matrix V**H;
  if JOBZ = 'S', VT contains the first min(M,N) rows of
  V**H (the right singular vectors, stored rowwise);
  if JOBZ = 'O' and M < N, or JOBZ = 'N', VT is not referenced.

LDVT
  LDVT is INTEGER
  The leading dimension of the array VT.  LDVT >= 1; if
  JOBZ = 'A' or JOBZ = 'O' and M >= N, LDVT >= N;
  if JOBZ = 'S', LDVT >= min(M,N).
```

```
 WORK
   WORK is COMPLEX array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK. LWORK >= 1.
   if JOBZ = 'N', LWORK >= 2*min(M,N)+max(M,N).
   if JOBZ = 'O',
       LWORK >= 2*min(M,N)*min(M,N)+2*min(M,N)+max(M,N).
   if JOBZ = 'S' or 'A',
       LWORK >= min(M,N)*min(M,N)+2*min(M,N)+max(M,N).
   For good performance, LWORK should generally be larger.

   If LWORK = -1, a workspace query is assumed.  The optimal
   size for the WORK array is calculated and stored in WORK(1),
   and no other work except argument checking is performed.

   RWORK
   RWORK is REAL array, dimension (MAX(1,LRWORK))
   If JOBZ = 'N', LRWORK >= 7*min(M,N).
   Otherwise,
   LRWORK >= min(M,N)*max(5*min(M,N)+7,2*max(M,N)+2*min(M,N)+1)

   IWORK
   IWORK is INTEGER array, dimension (8*min(M,N))

   INFO
   INFO is INTEGER
   = 0:  successful exit.
   < 0:  if INFO = -i, the i-th argument had an illegal value.
   > 0:  The updating process of SBDSDC did not converge.
```

———-

30.2.3 fortran code

— **cgesdd.f** —

```
*  ===================================================================
      SUBROUTINE CGESDD( JOBZ, M, N, A, LDA, S, U, LDU, VT, LDVT,
     $                   WORK, LWORK, RWORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
```

```
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBZ
      INTEGER            INFO, LDA, LDU, LDVT, LWORK, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      REAL               RWORK( * ), S( * )
      COMPLEX            A( LDA, * ), U( LDU, * ), VT( LDVT, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            LQUERV
      PARAMETER          ( LQUERV = -1 )
      COMPLEX            CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0E+0, 0.0E+0 ),
     $                   CONE = ( 1.0E+0, 0.0E+0 ) )
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            WNTQA, WNTQAS, WNTQN, WNTQO, WNTQS
      INTEGER            BLK, CHUNK, I, IE, IERR, IL, IR, IRU, IRVT,
     $                   ISCL, ITAU, ITAUP, ITAUQ, IU, IVT, LDWKVT,
     $                   LDWRKL, LDWRKR, LDWRKU, MAXWRK, MINMN, MINWRK,
     $                   MNTHR1, MNTHR2, NRWORK, NWORK, WRKBL
      REAL               ANRM, BIGNUM, EPS, SMLNUM
*     ..
*     .. Local Arrays ..
      INTEGER            IDUM( 1 )
      REAL               DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEBRD, CGELQF, CGEMM, CGEQRF, CLACP2, CLACPY,
     $                   CLACRM, CLARCM, CLASCL, CLASET, CUNGBR, CUNGLQ,
     $                   CUNGQR, CUNMBR, SBDSDC, SLASCL, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               CLANGE, SLAMCH
      EXTERNAL           CLANGE, SLAMCH, ILAENV, LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          INT, MAX, MIN, SQRT
*     ..
```

```
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      MINMN = MIN( M, N )
      MNTHR1 = INT( MINMN*17.0 / 9.0 )
      MNTHR2 = INT( MINMN*5.0 / 3.0 )
      WNTQA = LSAME( JOBZ, 'A' )
      WNTQS = LSAME( JOBZ, 'S' )
      WNTQAS = WNTQA .OR. WNTQS
      WNTQO = LSAME( JOBZ, 'O' )
      WNTQN = LSAME( JOBZ, 'N' )
      MINWRK = 1
      MAXWRK = 1
*
      IF( .NOT.( WNTQA .OR. WNTQS .OR. WNTQO .OR. WNTQN ) ) THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDU.LT.1 .OR. ( WNTQAS .AND. LDU.LT.M ) .OR.
     $         ( WNTQO .AND. M.LT.N .AND. LDU.LT.M ) ) THEN
         INFO = -8
      ELSE IF( LDVT.LT.1 .OR. ( WNTQA .AND. LDVT.LT.N ) .OR.
     $         ( WNTQS .AND. LDVT.LT.MINMN ) .OR.
     $         ( WNTQO .AND. M.GE.N .AND. LDVT.LT.N ) ) THEN
         INFO = -10
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       CWorkspace refers to complex workspace, and RWorkspace to
*       real workspace. NB refers to the optimal block size for the
*       immediately following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 .AND. M.GT.0 .AND. N.GT.0 ) THEN
         IF( M.GE.N ) THEN
*
*           There is no complex work space needed for bidiagonal SVD
*           The real work space needed for bidiagonal SVD is BDSPAC
*           for computing singular values and singular vectors; BDSPAN
*           for computing singular values only.
*           BDSPAC = 5*N*N + 7*N
*           BDSPAN = MAX(7*N+4, 3*N+2+SMLSIZ*(SMLSIZ+8))
```

```
*
            IF( M.GE.MNTHR1 ) THEN
               IF( WNTQN ) THEN
*
*                 Path 1 (M much larger than N, JOBZ='N')
*
                  MAXWRK = N + N*ILAENV( 1, 'CGEQRF', ' ', M, N, -1,
     $                     -1 )
                  MAXWRK = MAX( MAXWRK, 2*N+2*N*
     $                     ILAENV( 1, 'CGEBRD', ' ', N, N, -1, -1 ) )
                  MINWRK = 3*N
               ELSE IF( WNTQO ) THEN
*
*                 Path 2 (M much larger than N, JOBZ='O')
*
                  WRKBL = N + N*ILAENV( 1, 'CGEQRF', ' ', M, N, -1, -1 )
                  WRKBL = MAX( WRKBL, N+N*ILAENV( 1, 'CUNGQR', ' ', M,
     $                    N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 2*N+2*N*
     $                    ILAENV( 1, 'CGEBRD', ' ', N, N, -1, -1 ) )
                  WRKBL = MAX( WRKBL, 2*N+N*
     $                    ILAENV( 1, 'CUNMBR', 'QLN', N, N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 2*N+N*
     $                    ILAENV( 1, 'CUNMBR', 'PRC', N, N, N, -1 ) )
                  MAXWRK = M*N + N*N + WRKBL
                  MINWRK = 2*N*N + 3*N
               ELSE IF( WNTQS ) THEN
*
*                 Path 3 (M much larger than N, JOBZ='S')
*
                  WRKBL = N + N*ILAENV( 1, 'CGEQRF', ' ', M, N, -1, -1 )
                  WRKBL = MAX( WRKBL, N+N*ILAENV( 1, 'CUNGQR', ' ', M,
     $                    N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 2*N+2*N*
     $                    ILAENV( 1, 'CGEBRD', ' ', N, N, -1, -1 ) )
                  WRKBL = MAX( WRKBL, 2*N+N*
     $                    ILAENV( 1, 'CUNMBR', 'QLN', N, N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 2*N+N*
     $                    ILAENV( 1, 'CUNMBR', 'PRC', N, N, N, -1 ) )
                  MAXWRK = N*N + WRKBL
                  MINWRK = N*N + 3*N
               ELSE IF( WNTQA ) THEN
*
*                 Path 4 (M much larger than N, JOBZ='A')
*
                  WRKBL = N + N*ILAENV( 1, 'CGEQRF', ' ', M, N, -1, -1 )
                  WRKBL = MAX( WRKBL, N+M*ILAENV( 1, 'CUNGQR', ' ', M,
     $                    M, N, -1 ) )
                  WRKBL = MAX( WRKBL, 2*N+2*N*
     $                    ILAENV( 1, 'CGEBRD', ' ', N, N, -1, -1 ) )
```

```
                 WRKBL = MAX( WRKBL, 2*N+N*
     $                   ILAENV( 1, 'CUNMBR', 'QLN', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 2*N+N*
     $                   ILAENV( 1, 'CUNMBR', 'PRC', N, N, N, -1 ) )
                 MAXWRK = N*N + WRKBL
                 MINWRK = N*N + 2*N + M
              END IF
           ELSE IF( M.GE.MNTHR2 ) THEN
*
*             Path 5 (M much larger than N, but not as much as MNTHR1)
*
              MAXWRK = 2*N + ( M+N )*ILAENV( 1, 'CGEBRD', ' ', M, N,
     $                 -1, -1 )
              MINWRK = 2*N + M
              IF( WNTQO ) THEN
                 MAXWRK = MAX( MAXWRK, 2*N+N*
     $                    ILAENV( 1, 'CUNGBR', 'P', N, N, N, -1 ) )
                 MAXWRK = MAX( MAXWRK, 2*N+N*
     $                    ILAENV( 1, 'CUNGBR', 'Q', M, N, N, -1 ) )
                 MAXWRK = MAXWRK + M*N
                 MINWRK = MINWRK + N*N
              ELSE IF( WNTQS ) THEN
                 MAXWRK = MAX( MAXWRK, 2*N+N*
     $                    ILAENV( 1, 'CUNGBR', 'P', N, N, N, -1 ) )
                 MAXWRK = MAX( MAXWRK, 2*N+N*
     $                    ILAENV( 1, 'CUNGBR', 'Q', M, N, N, -1 ) )
              ELSE IF( WNTQA ) THEN
                 MAXWRK = MAX( MAXWRK, 2*N+N*
     $                    ILAENV( 1, 'CUNGBR', 'P', N, N, N, -1 ) )
                 MAXWRK = MAX( MAXWRK, 2*N+M*
     $                    ILAENV( 1, 'CUNGBR', 'Q', M, M, N, -1 ) )
              END IF
           ELSE
*
*             Path 6 (M at least N, but not much larger)
*
              MAXWRK = 2*N + ( M+N )*ILAENV( 1, 'CGEBRD', ' ', M, N,
     $                 -1, -1 )
              MINWRK = 2*N + M
              IF( WNTQO ) THEN
                 MAXWRK = MAX( MAXWRK, 2*N+N*
     $                    ILAENV( 1, 'CUNMBR', 'PRC', N, N, N, -1 ) )
                 MAXWRK = MAX( MAXWRK, 2*N+N*
     $                    ILAENV( 1, 'CUNMBR', 'QLN', M, N, N, -1 ) )
                 MAXWRK = MAXWRK + M*N
                 MINWRK = MINWRK + N*N
              ELSE IF( WNTQS ) THEN
                 MAXWRK = MAX( MAXWRK, 2*N+N*
     $                    ILAENV( 1, 'CUNMBR', 'PRC', N, N, N, -1 ) )
                 MAXWRK = MAX( MAXWRK, 2*N+N*
```

```
     $                    ILAENV( 1, 'CUNMBR', 'QLN', M, N, N, -1 ) )
               ELSE IF( WNTQA ) THEN
                  MAXWRK = MAX( MAXWRK, 2*N+N*
     $                    ILAENV( 1, 'CUNGBR', 'PRC', N, N, N, -1 ) )
                  MAXWRK = MAX( MAXWRK, 2*N+M*
     $                    ILAENV( 1, 'CUNGBR', 'QLN', M, M, N, -1 ) )
               END IF
            END IF
         ELSE
*
*           There is no complex work space needed for bidiagonal SVD
*           The real work space needed for bidiagonal SVD is BDSPAC
*           for computing singular values and singular vectors; BDSPAN
*           for computing singular values only.
*           BDSPAC = 5*M*M + 7*M
*           BDSPAN = MAX(7*M+4, 3*M+2+SMLSIZ*(SMLSIZ+8))
*
            IF( N.GE.MNTHR1 ) THEN
               IF( WNTQN ) THEN
*
*                 Path 1t (N much larger than M, JOBZ='N')
*
                  MAXWRK = M + M*ILAENV( 1, 'CGELQF', ' ', M, N, -1,
     $                    -1 )
                  MAXWRK = MAX( MAXWRK, 2*M+2*M*
     $                    ILAENV( 1, 'CGEBRD', ' ', M, M, -1, -1 ) )
                  MINWRK = 3*M
               ELSE IF( WNTQO ) THEN
*
*                 Path 2t (N much larger than M, JOBZ='O')
*
                  WRKBL = M + M*ILAENV( 1, 'CGELQF', ' ', M, N, -1, -1 )
                  WRKBL = MAX( WRKBL, M+M*ILAENV( 1, 'CUNGLQ', ' ', M,
     $                    N, M, -1 ) )
                  WRKBL = MAX( WRKBL, 2*M+2*M*
     $                    ILAENV( 1, 'CGEBRD', ' ', M, M, -1, -1 ) )
                  WRKBL = MAX( WRKBL, 2*M+M*
     $                    ILAENV( 1, 'CUNMBR', 'PRC', M, M, M, -1 ) )
                  WRKBL = MAX( WRKBL, 2*M+M*
     $                    ILAENV( 1, 'CUNMBR', 'QLN', M, M, M, -1 ) )
                  MAXWRK = M*N + M*M + WRKBL
                  MINWRK = 2*M*M + 3*M
               ELSE IF( WNTQS ) THEN
*
*                 Path 3t (N much larger than M, JOBZ='S')
*
                  WRKBL = M + M*ILAENV( 1, 'CGELQF', ' ', M, N, -1, -1 )
                  WRKBL = MAX( WRKBL, M+M*ILAENV( 1, 'CUNGLQ', ' ', M,
     $                    N, M, -1 ) )
                  WRKBL = MAX( WRKBL, 2*M+2*M*
```

```
     $                  ILAENV( 1, 'CGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 2*M+M*
     $                  ILAENV( 1, 'CUNMBR', 'PRC', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, 2*M+M*
     $                  ILAENV( 1, 'CUNMBR', 'QLN', M, M, M, -1 ) )
                 MAXWRK = M*M + WRKBL
                 MINWRK = M*M + 3*M
              ELSE IF( WNTQA ) THEN
*
*                Path 4t (N much larger than M, JOBZ='A')
*
                 WRKBL = M + M*ILAENV( 1, 'CGELQF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, M+N*ILAENV( 1, 'CUNGLQ', ' ', N,
     $                   N, M, -1 ) )
                 WRKBL = MAX( WRKBL, 2*M+2*M*
     $                  ILAENV( 1, 'CGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 2*M+M*
     $                  ILAENV( 1, 'CUNMBR', 'PRC', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, 2*M+M*
     $                  ILAENV( 1, 'CUNMBR', 'QLN', M, M, M, -1 ) )
                 MAXWRK = M*M + WRKBL
                 MINWRK = M*M + 2*M + N
              END IF
           ELSE IF( N.GE.MNTHR2 ) THEN
*
*             Path 5t (N much larger than M, but not as much as MNTHR1)
*
              MAXWRK = 2*M + ( M+N )*ILAENV( 1, 'CGEBRD', ' ', M, N,
     $                 -1, -1 )
              MINWRK = 2*M + N
              IF( WNTQO ) THEN
                 MAXWRK = MAX( MAXWRK, 2*M+M*
     $                   ILAENV( 1, 'CUNGBR', 'P', M, N, M, -1 ) )
                 MAXWRK = MAX( MAXWRK, 2*M+M*
     $                   ILAENV( 1, 'CUNGBR', 'Q', M, M, N, -1 ) )
                 MAXWRK = MAXWRK + M*N
                 MINWRK = MINWRK + M*M
              ELSE IF( WNTQS ) THEN
                 MAXWRK = MAX( MAXWRK, 2*M+M*
     $                   ILAENV( 1, 'CUNGBR', 'P', M, N, M, -1 ) )
                 MAXWRK = MAX( MAXWRK, 2*M+M*
     $                   ILAENV( 1, 'CUNGBR', 'Q', M, M, N, -1 ) )
              ELSE IF( WNTQA ) THEN
                 MAXWRK = MAX( MAXWRK, 2*M+N*
     $                   ILAENV( 1, 'CUNGBR', 'P', N, N, M, -1 ) )
                 MAXWRK = MAX( MAXWRK, 2*M+M*
     $                   ILAENV( 1, 'CUNGBR', 'Q', M, M, N, -1 ) )
              END IF
           ELSE
*
```

```
*              Path 6t (N greater than M, but not much larger)
*
               MAXWRK = 2*M + ( M+N )*ILAENV( 1, 'CGEBRD', ' ', M, N,
     $                  -1, -1 )
               MINWRK = 2*M + N
               IF( WNTQO ) THEN
                  MAXWRK = MAX( MAXWRK, 2*M+M*
     $                     ILAENV( 1, 'CUNMBR', 'PRC', M, N, M, -1 ) )
                  MAXWRK = MAX( MAXWRK, 2*M+M*
     $                     ILAENV( 1, 'CUNMBR', 'QLN', M, M, N, -1 ) )
                  MAXWRK = MAXWRK + M*N
                  MINWRK = MINWRK + M*M
               ELSE IF( WNTQS ) THEN
                  MAXWRK = MAX( MAXWRK, 2*M+M*
     $                     ILAENV( 1, 'CUNGBR', 'PRC', M, N, M, -1 ) )
                  MAXWRK = MAX( MAXWRK, 2*M+M*
     $                     ILAENV( 1, 'CUNGBR', 'QLN', M, M, N, -1 ) )
               ELSE IF( WNTQA ) THEN
                  MAXWRK = MAX( MAXWRK, 2*M+N*
     $                     ILAENV( 1, 'CUNGBR', 'PRC', N, N, M, -1 ) )
                  MAXWRK = MAX( MAXWRK, 2*M+M*
     $                     ILAENV( 1, 'CUNGBR', 'QLN', M, M, N, -1 ) )
               END IF
            END IF
         END IF
         MAXWRK = MAX( MAXWRK, MINWRK )
      END IF
      IF( INFO.EQ.0 ) THEN
         WORK( 1 ) = MAXWRK
         IF( LWORK.LT.MINWRK .AND. LWORK.NE.LQUERV )
     $      INFO = -13
      END IF
*
*     Quick returns
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGESDD', -INFO )
         RETURN
      END IF
      IF( LWORK.EQ.LQUERV )
     $   RETURN
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SQRT( SLAMCH( 'S' ) ) / EPS
      BIGNUM = ONE / SMLNUM
```

```
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = CLANGE( 'M', M, N, A, LDA, DUM )
      ISCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ISCL = 1
         CALL CLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, IERR )
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ISCL = 1
         CALL CLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, IERR )
      END IF
*
      IF( M.GE.N ) THEN
*
*        A has at least as many rows as columns. If A has sufficiently
*        more rows than columns, first reduce using the QR
*        decomposition (if sufficient workspace available)
*
         IF( M.GE.MNTHR1 ) THEN
*
            IF( WNTQN ) THEN
*
*              Path 1 (M much larger than N, JOBZ='N')
*              No singular vectors to be computed
*
               ITAU = 1
               NWORK = ITAU + N
*
*              Compute A=Q*R
*              (CWorkspace: need 2*N, prefer N+N*NB)
*              (RWorkspace: need 0)
*
               CALL CGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Zero out below R
*
               CALL CLASET( 'L', N-1, N-1, CZERO, CZERO, A( 2, 1 ),
     $                      LDA )
               IE = 1
               ITAUQ = 1
               ITAUP = ITAUQ + N
               NWORK = ITAUP + N
*
*              Bidiagonalize R in A
*              (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
*              (RWorkspace: need N)
*
               CALL CGEBRD( N, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
```

```
     $                     WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                     IERR )
               NRWORK = IE + N
*
*              Perform bidiagonal SVD, compute singular values only
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAN)
*
               CALL SBDSDC( 'U', 'N', N, S, RWORK( IE ), DUM, 1, DUM, 1,
     $                     DUM, IDUM, RWORK( NRWORK ), IWORK, INFO )
*
            ELSE IF( WNTQO ) THEN
*
*              Path 2 (M much larger than N, JOBZ='O')
*              N left singular vectors to be overwritten on A and
*              N right singular vectors to be computed in VT
*
               IU = 1
*
*              WORK(IU) is N by N
*
               LDWRKU = N
               IR = IU + LDWRKU*N
               IF( LWORK.GE.M*N+N*N+3*N ) THEN
*
*                 WORK(IR) is M by N
*
                  LDWRKR = M
               ELSE
                  LDWRKR = ( LWORK-N*N-3*N ) / N
               END IF
               ITAU = IR + LDWRKR*N
               NWORK = ITAU + N
*
*              Compute A=Q*R
*              (CWorkspace: need N*N+2*N, prefer M*N+N+N*NB)
*              (RWorkspace: 0)
*
               CALL CGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*              Copy R to WORK( IR ), zeroing out below it
*
               CALL CLACPY( 'U', N, N, A, LDA, WORK( IR ), LDWRKR )
               CALL CLASET( 'L', N-1, N-1, CZERO, CZERO, WORK( IR+1 ),
     $                     LDWRKR )
*
*              Generate Q in A
*              (CWorkspace: need 2*N, prefer N+N*NB)
*              (RWorkspace: 0)
```

```
*
               CALL CUNGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
               IE = 1
               ITAUQ = ITAU
               ITAUP = ITAUQ + N
               NWORK = ITAUP + N
*
*              Bidiagonalize R in WORK(IR)
*              (CWorkspace: need N*N+3*N, prefer M*N+2*N+2*N*NB)
*              (RWorkspace: need N)
*
               CALL CGEBRD( N, N, WORK( IR ), LDWRKR, S, RWORK( IE ),
     $                      WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of R in WORK(IRU) and computing right singular vectors
*              of R in WORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               IRU = IE + N
               IRVT = IRU + N*N
               NRWORK = IRVT + N*N
               CALL SBDSDC( 'U', 'I', N, S, RWORK( IE ), RWORK( IRU ),
     $                      N, RWORK( IRVT ), N, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Copy real matrix RWORK(IRU) to complex matrix WORK(IU)
*              Overwrite WORK(IU) by the left singular vectors of R
*              (CWorkspace: need 2*N*N+3*N, prefer M*N+N*N+2*N+N*NB)
*              (RWorkspace: 0)
*
               CALL CLACP2( 'F', N, N, RWORK( IRU ), N, WORK( IU ),
     $                      LDWRKU )
               CALL CUNMBR( 'Q', 'L', 'N', N, N, N, WORK( IR ), LDWRKR,
     $                      WORK( ITAUQ ), WORK( IU ), LDWRKU,
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Copy real matrix RWORK(IRVT) to complex matrix VT
*              Overwrite VT by the right singular vectors of R
*              (CWorkspace: need N*N+3*N, prefer M*N+2*N+N*NB)
*              (RWorkspace: 0)
*
               CALL CLACP2( 'F', N, N, RWORK( IRVT ), N, VT, LDVT )
               CALL CUNMBR( 'P', 'R', 'C', N, N, N, WORK( IR ), LDWRKR,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
```

```
*              Multiply Q in A by left singular vectors of R in
*              WORK(IU), storing result in WORK(IR) and copying to A
*              (CWorkspace: need 2*N*N, prefer N*N+M*N)
*              (RWorkspace: 0)
*
               DO 10 I = 1, M, LDWRKR
                  CHUNK = MIN( M-I+1, LDWRKR )
                  CALL CGEMM( 'N', 'N', CHUNK, N, N, CONE, A( I, 1 ),
     $                        LDA, WORK( IU ), LDWRKU, CZERO,
     $                        WORK( IR ), LDWRKR )
                  CALL CLACPY( 'F', CHUNK, N, WORK( IR ), LDWRKR,
     $                         A( I, 1 ), LDA )
   10          CONTINUE
*
            ELSE IF( WNTQS ) THEN
*
*              Path 3 (M much larger than N, JOBZ='S')
*              N left singular vectors to be computed in U and
*              N right singular vectors to be computed in VT
*
               IR = 1
*
*              WORK(IR) is N by N
*
               LDWRKR = N
               ITAU = IR + LDWRKR*N
               NWORK = ITAU + N
*
*              Compute A=Q*R
*              (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*              (RWorkspace: 0)
*
               CALL CGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy R to WORK(IR), zeroing out below it
*
               CALL CLACPY( 'U', N, N, A, LDA, WORK( IR ), LDWRKR )
               CALL CLASET( 'L', N-1, N-1, CZERO, CZERO, WORK( IR+1 ),
     $                      LDWRKR )
*
*              Generate Q in A
*              (CWorkspace: need 2*N, prefer N+N*NB)
*              (RWorkspace: 0)
*
               CALL CUNGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
               IE = 1
               ITAUQ = ITAU
               ITAUP = ITAUQ + N
```

```
               NWORK = ITAUP + N
*
*              Bidiagonalize R in WORK(IR)
*              (CWorkspace: need N*N+3*N, prefer N*N+2*N+2*N*NB)
*              (RWorkspace: need N)
*
               CALL CGEBRD( N, N, WORK( IR ), LDWRKR, S, RWORK( IE ),
     $                      WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               IRU = IE + N
               IRVT = IRU + N*N
               NRWORK = IRVT + N*N
               CALL SBDSDC( 'U', 'I', N, S, RWORK( IE ), RWORK( IRU ),
     $                      N, RWORK( IRVT ), N, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Copy real matrix RWORK(IRU) to complex matrix U
*              Overwrite U by left singular vectors of R
*              (CWorkspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*              (RWorkspace: 0)
*
               CALL CLACP2( 'F', N, N, RWORK( IRU ), N, U, LDU )
               CALL CUNMBR( 'Q', 'L', 'N', N, N, N, WORK( IR ), LDWRKR,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy real matrix RWORK(IRVT) to complex matrix VT
*              Overwrite VT by right singular vectors of R
*              (CWorkspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*              (RWorkspace: 0)
*
               CALL CLACP2( 'F', N, N, RWORK( IRVT ), N, VT, LDVT )
               CALL CUNMBR( 'P', 'R', 'C', N, N, N, WORK( IR ), LDWRKR,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Multiply Q in A by left singular vectors of R in
*              WORK(IR), storing result in U
*              (CWorkspace: need N*N)
*              (RWorkspace: 0)
*
               CALL CLACPY( 'F', N, N, U, LDU, WORK( IR ), LDWRKR )
               CALL CGEMM( 'N', 'N', M, N, N, CONE, A, LDA, WORK( IR ),
```

```
     $                    LDWRKR, CZERO, U, LDU )
*
            ELSE IF( WNTQA ) THEN
*
*              Path 4 (M much larger than N, JOBZ='A')
*              M left singular vectors to be computed in U and
*              N right singular vectors to be computed in VT
*
               IU = 1
*
*              WORK(IU) is N by N
*
               LDWRKU = N
               ITAU = IU + LDWRKU*N
               NWORK = ITAU + N
*
*              Compute A=Q*R, copying result to U
*              (CWorkspace: need 2*N, prefer N+N*NB)
*              (RWorkspace: 0)
*
               CALL CGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
               CALL CLACPY( 'L', M, N, A, LDA, U, LDU )
*
*              Generate Q in U
*              (CWorkspace: need N+M, prefer N+M*NB)
*              (RWorkspace: 0)
*
               CALL CUNGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Produce R in A, zeroing out below it
*
               CALL CLASET( 'L', N-1, N-1, CZERO, CZERO, A( 2, 1 ),
     $                      LDA )
               IE = 1
               ITAUQ = ITAU
               ITAUP = ITAUQ + N
               NWORK = ITAUP + N
*
*              Bidiagonalize R in A
*              (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
*              (RWorkspace: need N)
*
               CALL CGEBRD( N, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                      IERR )
               IRU = IE + N
               IRVT = IRU + N*N
               NRWORK = IRVT + N*N
```

```
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in RWORK(IRU) and computing right
*             singular vectors of bidiagonal matrix in RWORK(IRVT)
*             (CWorkspace: need 0)
*             (RWorkspace: need BDSPAC)
*
              CALL SBDSDC( 'U', 'I', N, S, RWORK( IE ), RWORK( IRU ),
     $                     N, RWORK( IRVT ), N, DUM, IDUM,
     $                     RWORK( NRWORK ), IWORK, INFO )
*
*             Copy real matrix RWORK(IRU) to complex matrix WORK(IU)
*             Overwrite WORK(IU) by left singular vectors of R
*             (CWorkspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*             (RWorkspace: 0)
*
              CALL CLACP2( 'F', N, N, RWORK( IRU ), N, WORK( IU ),
     $                     LDWRKU )
              CALL CUNMBR( 'Q', 'L', 'N', N, N, N, A, LDA,
     $                     WORK( ITAUQ ), WORK( IU ), LDWRKU,
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*             Copy real matrix RWORK(IRVT) to complex matrix VT
*             Overwrite VT by right singular vectors of R
*             (CWorkspace: need 3*N, prefer 2*N+N*NB)
*             (RWorkspace: 0)
*
              CALL CLACP2( 'F', N, N, RWORK( IRVT ), N, VT, LDVT )
              CALL CUNMBR( 'P', 'R', 'C', N, N, N, A, LDA,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*             Multiply Q in U by left singular vectors of R in
*             WORK(IU), storing result in A
*             (CWorkspace: need N*N)
*             (RWorkspace: 0)
*
              CALL CGEMM( 'N', 'N', M, N, N, CONE, U, LDU, WORK( IU ),
     $                    LDWRKU, CZERO, A, LDA )
*
*             Copy left singular vectors of A from A to U
*
              CALL CLACPY( 'F', M, N, A, LDA, U, LDU )
*
           END IF
*
        ELSE IF( M.GE.MNTHR2 ) THEN
*
*          MNTHR2 <= M < MNTHR1
*
```

```
*           Path 5 (M much larger than N, but not as much as MNTHR1)
*           Reduce to bidiagonal form without QR decomposition, use
*           CUNGBR and matrix multiplication to compute singular vectors
*
            IE = 1
            NRWORK = IE + N
            ITAUQ = 1
            ITAUP = ITAUQ + N
            NWORK = ITAUP + N
*
*           Bidiagonalize A
*           (CWorkspace: need 2*N+M, prefer 2*N+(M+N)*NB)
*           (RWorkspace: need N)
*
            CALL CGEBRD( M, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                   WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                   IERR )
            IF( WNTQN ) THEN
*
*              Compute singular values only
*              (Cworkspace: 0)
*              (Rworkspace: need BDSPAN)
*
               CALL SBDSDC( 'U', 'N', N, S, RWORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, RWORK( NRWORK ), IWORK, INFO )
            ELSE IF( WNTQO ) THEN
               IU = NWORK
               IRU = NRWORK
               IRVT = IRU + N*N
               NRWORK = IRVT + N*N
*
*              Copy A to VT, generate P**H
*              (Cworkspace: need 2*N, prefer N+N*NB)
*              (Rworkspace: 0)
*
               CALL CLACPY( 'U', N, N, A, LDA, VT, LDVT )
               CALL CUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Generate Q in A
*              (CWorkspace: need 2*N, prefer N+N*NB)
*              (RWorkspace: 0)
*
               CALL CUNGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
               IF( LWORK.GE.M*N+3*N ) THEN
*
*                 WORK( IU ) is M by N
*
```

```
                 LDWRKU = M
              ELSE
*
*                WORK(IU) is LDWRKU by N
*
                 LDWRKU = ( LWORK-3*N ) / N
              END IF
              NWORK = IU + LDWRKU*N
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in RWORK(IRU) and computing right
*             singular vectors of bidiagonal matrix in RWORK(IRVT)
*             (CWorkspace: need 0)
*             (RWorkspace: need BDSPAC)
*
              CALL SBDSDC( 'U', 'I', N, S, RWORK( IE ), RWORK( IRU ),
     $                     N, RWORK( IRVT ), N, DUM, IDUM,
     $                     RWORK( NRWORK ), IWORK, INFO )
*
*             Multiply real matrix RWORK(IRVT) by P**H in VT,
*             storing the result in WORK(IU), copying to VT
*             (Cworkspace: need 0)
*             (Rworkspace: need 3*N*N)
*
              CALL CLARCM( N, N, RWORK( IRVT ), N, VT, LDVT,
     $                     WORK( IU ), LDWRKU, RWORK( NRWORK ) )
              CALL CLACPY( 'F', N, N, WORK( IU ), LDWRKU, VT, LDVT )
*
*             Multiply Q in A by real matrix RWORK(IRU), storing the
*             result in WORK(IU), copying to A
*             (CWorkspace: need N*N, prefer M*N)
*             (Rworkspace: need 3*N*N, prefer N*N+2*M*N)
*
              NRWORK = IRVT
              DO 20 I = 1, M, LDWRKU
                 CHUNK = MIN( M-I+1, LDWRKU )
                 CALL CLACRM( CHUNK, N, A( I, 1 ), LDA, RWORK( IRU ),
     $                        N, WORK( IU ), LDWRKU, RWORK( NRWORK ) )
                 CALL CLACPY( 'F', CHUNK, N, WORK( IU ), LDWRKU,
     $                        A( I, 1 ), LDA )
   20         CONTINUE
*
           ELSE IF( WNTQS ) THEN
*
*             Copy A to VT, generate P**H
*             (Cworkspace: need 2*N, prefer N+N*NB)
*             (Rworkspace: 0)
*
              CALL CLACPY( 'U', N, N, A, LDA, VT, LDVT )
              CALL CUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
```

```
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*             Copy A to U, generate Q
*             (Cworkspace: need 2*N, prefer N+N*NB)
*             (Rworkspace: 0)
*
              CALL CLACPY( 'L', M, N, A, LDA, U, LDU )
              CALL CUNGBR( 'Q', M, N, N, U, LDU, WORK( ITAUQ ),
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in RWORK(IRU) and computing right
*             singular vectors of bidiagonal matrix in RWORK(IRVT)
*             (CWorkspace: need 0)
*             (RWorkspace: need BDSPAC)
*
              IRU = NRWORK
              IRVT = IRU + N*N
              NRWORK = IRVT + N*N
              CALL SBDSDC( 'U', 'I', N, S, RWORK( IE ), RWORK( IRU ),
     $                     N, RWORK( IRVT ), N, DUM, IDUM,
     $                     RWORK( NRWORK ), IWORK, INFO )
*
*             Multiply real matrix RWORK(IRVT) by P**H in VT,
*             storing the result in A, copying to VT
*             (Cworkspace: need 0)
*             (Rworkspace: need 3*N*N)
*
              CALL CLARCM( N, N, RWORK( IRVT ), N, VT, LDVT, A, LDA,
     $                     RWORK( NRWORK ) )
              CALL CLACPY( 'F', N, N, A, LDA, VT, LDVT )
*
*             Multiply Q in U by real matrix RWORK(IRU), storing the
*             result in A, copying to U
*             (CWorkspace: need 0)
*             (Rworkspace: need N*N+2*M*N)
*
              NRWORK = IRVT
              CALL CLACRM( M, N, U, LDU, RWORK( IRU ), N, A, LDA,
     $                     RWORK( NRWORK ) )
              CALL CLACPY( 'F', M, N, A, LDA, U, LDU )
           ELSE
*
*             Copy A to VT, generate P**H
*             (Cworkspace: need 2*N, prefer N+N*NB)
*             (Rworkspace: 0)
*
              CALL CLACPY( 'U', N, N, A, LDA, VT, LDVT )
              CALL CUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
```

```
*
*             Copy A to U, generate Q
*             (Cworkspace: need 2*N, prefer N+N*NB)
*             (Rworkspace: 0)
*
              CALL CLACPY( 'L', M, N, A, LDA, U, LDU )
              CALL CUNGBR( 'Q', M, M, N, U, LDU, WORK( ITAUQ ),
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in RWORK(IRU) and computing right
*             singular vectors of bidiagonal matrix in RWORK(IRVT)
*             (CWorkspace: need 0)
*             (RWorkspace: need BDSPAC)
*
              IRU = NRWORK
              IRVT = IRU + N*N
              NRWORK = IRVT + N*N
              CALL SBDSDC( 'U', 'I', N, S, RWORK( IE ), RWORK( IRU ),
     $                     N, RWORK( IRVT ), N, DUM, IDUM,
     $                     RWORK( NRWORK ), IWORK, INFO )
*
*             Multiply real matrix RWORK(IRVT) by P**H in VT,
*             storing the result in A, copying to VT
*             (Cworkspace: need 0)
*             (Rworkspace: need 3*N*N)
*
              CALL CLARCM( N, N, RWORK( IRVT ), N, VT, LDVT, A, LDA,
     $                     RWORK( NRWORK ) )
              CALL CLACPY( 'F', N, N, A, LDA, VT, LDVT )
*
*             Multiply Q in U by real matrix RWORK(IRU), storing the
*             result in A, copying to U
*             (CWorkspace: 0)
*             (Rworkspace: need 3*N*N)
*
              NRWORK = IRVT
              CALL CLACRM( M, N, U, LDU, RWORK( IRU ), N, A, LDA,
     $                     RWORK( NRWORK ) )
              CALL CLACPY( 'F', M, N, A, LDA, U, LDU )
           END IF
*
        ELSE
*
*          M .LT. MNTHR2
*
*          Path 6 (M at least N, but not much larger)
*          Reduce to bidiagonal form without QR decomposition
*          Use CUNMBR to compute singular vectors
*
```

```
            IE = 1
            NRWORK = IE + N
            ITAUQ = 1
            ITAUP = ITAUQ + N
            NWORK = ITAUP + N
*
*           Bidiagonalize A
*           (CWorkspace: need 2*N+M, prefer 2*N+(M+N)*NB)
*           (RWorkspace: need N)
*
            CALL CGEBRD( M, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                   WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                   IERR )
            IF( WNTQN ) THEN
*
*              Compute singular values only
*              (Cworkspace: 0)
*              (Rworkspace: need BDSPAN)
*
               CALL SBDSDC( 'U', 'N', N, S, RWORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, RWORK( NRWORK ), IWORK, INFO )
            ELSE IF( WNTQO ) THEN
               IU = NWORK
               IRU = NRWORK
               IRVT = IRU + N*N
               NRWORK = IRVT + N*N
               IF( LWORK.GE.M*N+3*N ) THEN
*
*                 WORK( IU ) is M by N
*
                  LDWRKU = M
               ELSE
*
*                 WORK( IU ) is LDWRKU by N
*
                  LDWRKU = ( LWORK-3*N ) / N
               END IF
               NWORK = IU + LDWRKU*N
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               CALL SBDSDC( 'U', 'I', N, S, RWORK( IE ), RWORK( IRU ),
     $                      N, RWORK( IRVT ), N, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Copy real matrix RWORK(IRVT) to complex matrix VT
```

```
*              Overwrite VT by right singular vectors of A
*              (Cworkspace: need 2*N, prefer N+N*NB)
*              (Rworkspace: need 0)
*
               CALL CLACP2( 'F', N, N, RWORK( IRVT ), N, VT, LDVT )
               CALL CUNMBR( 'P', 'R', 'C', N, N, N, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
               IF( LWORK.GE.M*N+3*N ) THEN
*
*              Copy real matrix RWORK(IRU) to complex matrix WORK(IU)
*              Overwrite WORK(IU) by left singular vectors of A, copying
*              to A
*              (Cworkspace: need M*N+2*N, prefer M*N+N+N*NB)
*              (Rworkspace: need 0)
*
                  CALL CLASET( 'F', M, N, CZERO, CZERO, WORK( IU ),
     $                         LDWRKU )
                  CALL CLACP2( 'F', N, N, RWORK( IRU ), N, WORK( IU ),
     $                         LDWRKU )
                  CALL CUNMBR( 'Q', 'L', 'N', M, N, N, A, LDA,
     $                         WORK( ITAUQ ), WORK( IU ), LDWRKU,
     $                         WORK( NWORK ), LWORK-NWORK+1, IERR )
                  CALL CLACPY( 'F', M, N, WORK( IU ), LDWRKU, A, LDA )
               ELSE
*
                  Generate Q in A
*                 (Cworkspace: need 2*N, prefer N+N*NB)
*                 (Rworkspace: need 0)
*
                  CALL CUNGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
     $                         WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                 Multiply Q in A by real matrix RWORK(IRU), storing the
*                 result in WORK(IU), copying to A
*                 (CWorkspace: need N*N, prefer M*N)
*                 (Rworkspace: need 3*N*N, prefer N*N+2*M*N)
*
                  NRWORK = IRVT
                  DO 30 I = 1, M, LDWRKU
                     CHUNK = MIN( M-I+1, LDWRKU )
                     CALL CLACRM( CHUNK, N, A( I, 1 ), LDA,
     $                            RWORK( IRU ), N, WORK( IU ), LDWRKU,
     $                            RWORK( NRWORK ) )
                     CALL CLACPY( 'F', CHUNK, N, WORK( IU ), LDWRKU,
     $                            A( I, 1 ), LDA )
   30             CONTINUE
               END IF
*
```

```
            ELSE IF( WNTQS ) THEN
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               IRU = NRWORK
               IRVT = IRU + N*N
               NRWORK = IRVT + N*N
               CALL SBDSDC( 'U', 'I', N, S, RWORK( IE ), RWORK( IRU ),
     $                      N, RWORK( IRVT ), N, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Copy real matrix RWORK(IRU) to complex matrix U
*              Overwrite U by left singular vectors of A
*              (CWorkspace: need 3*N, prefer 2*N+N*NB)
*              (RWorkspace: 0)
*
               CALL CLASET( 'F', M, N, CZERO, CZERO, U, LDU )
               CALL CLACP2( 'F', N, N, RWORK( IRU ), N, U, LDU )
               CALL CUNMBR( 'Q', 'L', 'N', M, N, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy real matrix RWORK(IRVT) to complex matrix VT
*              Overwrite VT by right singular vectors of A
*              (CWorkspace: need 3*N, prefer 2*N+N*NB)
*              (RWorkspace: 0)
*
               CALL CLACP2( 'F', N, N, RWORK( IRVT ), N, VT, LDVT )
               CALL CUNMBR( 'P', 'R', 'C', N, N, N, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
            ELSE
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               IRU = NRWORK
               IRVT = IRU + N*N
               NRWORK = IRVT + N*N
               CALL SBDSDC( 'U', 'I', N, S, RWORK( IE ), RWORK( IRU ),
     $                      N, RWORK( IRVT ), N, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
```

```
*             Set the right corner of U to identity matrix
*
              CALL CLASET( 'F', M, M, CZERO, CZERO, U, LDU )
              IF( M.GT.N ) THEN
                 CALL CLASET( 'F', M-N, M-N, CZERO, CONE,
     $                        U( N+1, N+1 ), LDU )
              END IF
*
*             Copy real matrix RWORK(IRU) to complex matrix U
*             Overwrite U by left singular vectors of A
*             (CWorkspace: need 2*N+M, prefer 2*N+M*NB)
*             (RWorkspace: 0)
*
              CALL CLACP2( 'F', N, N, RWORK( IRU ), N, U, LDU )
              CALL CUNMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*             Copy real matrix RWORK(IRVT) to complex matrix VT
*             Overwrite VT by right singular vectors of A
*             (CWorkspace: need 3*N, prefer 2*N+N*NB)
*             (RWorkspace: 0)
*
              CALL CLACP2( 'F', N, N, RWORK( IRVT ), N, VT, LDVT )
              CALL CUNMBR( 'P', 'R', 'C', N, N, N, A, LDA,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
           END IF
*
        END IF
*
     ELSE
*
        A has more columns than rows. If A has sufficiently more
        columns than rows, first reduce using the LQ decomposition (if
        sufficient workspace available)
*
        IF( N.GE.MNTHR1 ) THEN
*
           IF( WNTQN ) THEN
*
              Path 1t (N much larger than M, JOBZ='N')
              No singular vectors to be computed
*
              ITAU = 1
              NWORK = ITAU + M
*
              Compute A=L*Q
              (CWorkspace: need 2*M, prefer M+M*NB)
              (RWorkspace: 0)
```

```
*
               CALL CGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Zero out above L
*
               CALL CLASET( 'U', M-1, M-1, CZERO, CZERO, A( 1, 2 ),
     $                      LDA )
               IE = 1
               ITAUQ = 1
               ITAUP = ITAUQ + M
               NWORK = ITAUP + M
*
*              Bidiagonalize L in A
*              (CWorkspace: need 3*M, prefer 2*M+2*M*NB)
*              (RWorkspace: need M)
*
               CALL CGEBRD( M, M, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                      IERR )
               NRWORK = IE + M
*
*              Perform bidiagonal SVD, compute singular values only
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAN)
*
               CALL SBDSDC( 'U', 'N', M, S, RWORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, RWORK( NRWORK ), IWORK, INFO )
*
            ELSE IF( WNTQO ) THEN
*
*              Path 2t (N much larger than M, JOBZ='O')
*              M right singular vectors to be overwritten on A and
*              M left singular vectors to be computed in U
*
               IVT = 1
               LDWKVT = M
*
*              WORK(IVT) is M by M
*
               IL = IVT + LDWKVT*M
               IF( LWORK.GE.M*N+M*M+3*M ) THEN
*
*                 WORK(IL) M by N
*
                  LDWRKL = M
                  CHUNK = N
               ELSE
*
*                 WORK(IL) is M by CHUNK
```

```
*
                 LDWRKL = M
                 CHUNK = ( LWORK-M*M-3*M ) / M
              END IF
              ITAU = IL + LDWRKL*CHUNK
              NWORK = ITAU + M
*
*             Compute A=L*Q
*             (CWorkspace: need 2*M, prefer M+M*NB)
*             (RWorkspace: 0)
*
              CALL CGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*             Copy L to WORK(IL), zeroing about above it
*
              CALL CLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWRKL )
              CALL CLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                     WORK( IL+LDWRKL ), LDWRKL )
*
*             Generate Q in A
*             (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*             (RWorkspace: 0)
*
              CALL CUNGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
              IE = 1
              ITAUQ = ITAU
              ITAUP = ITAUQ + M
              NWORK = ITAUP + M
*
*             Bidiagonalize L in WORK(IL)
*             (CWorkspace: need M*M+3*M, prefer M*M+2*M+2*M*NB)
*             (RWorkspace: need M)
*
              CALL CGEBRD( M, M, WORK( IL ), LDWRKL, S, RWORK( IE ),
     $                     WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in RWORK(IRU) and computing right
*             singular vectors of bidiagonal matrix in RWORK(IRVT)
*             (CWorkspace: need 0)
*             (RWorkspace: need BDSPAC)
*
              IRU = IE + M
              IRVT = IRU + M*M
              NRWORK = IRVT + M*M
              CALL SBDSDC( 'U', 'I', M, S, RWORK( IE ), RWORK( IRU ),
     $                     M, RWORK( IRVT ), M, DUM, IDUM,
```

```
     $                     RWORK( NRWORK ), IWORK, INFO )
*
*             Copy real matrix RWORK(IRU) to complex matrix WORK(IU)
*             Overwrite WORK(IU) by the left singular vectors of L
*             (CWorkspace: need N*N+3*N, prefer M*N+2*N+N*NB)
*             (RWorkspace: 0)
*
              CALL CLACP2( 'F', M, M, RWORK( IRU ), M, U, LDU )
              CALL CUNMBR( 'Q', 'L', 'N', M, M, M, WORK( IL ), LDWRKL,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*             Copy real matrix RWORK(IRVT) to complex matrix WORK(IVT)
*             Overwrite WORK(IVT) by the right singular vectors of L
*             (CWorkspace: need N*N+3*N, prefer M*N+2*N+N*NB)
*             (RWorkspace: 0)
*
              CALL CLACP2( 'F', M, M, RWORK( IRVT ), M, WORK( IVT ),
     $                     LDWKVT )
              CALL CUNMBR( 'P', 'R', 'C', M, M, M, WORK( IL ), LDWRKL,
     $                     WORK( ITAUP ), WORK( IVT ), LDWKVT,
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*             Multiply right singular vectors of L in WORK(IL) by Q
*             in A, storing result in WORK(IL) and copying to A
*             (CWorkspace: need 2*M*M, prefer M*M+M*N))
*             (RWorkspace: 0)
*
              DO 40 I = 1, N, CHUNK
                 BLK = MIN( N-I+1, CHUNK )
                 CALL CGEMM( 'N', 'N', M, BLK, M, CONE, WORK( IVT ), M,
     $                       A( 1, I ), LDA, CZERO, WORK( IL ),
     $                       LDWRKL )
                 CALL CLACPY( 'F', M, BLK, WORK( IL ), LDWRKL,
     $                        A( 1, I ), LDA )
   40         CONTINUE
*
           ELSE IF( WNTQS ) THEN
*
            Path 3t (N much larger than M, JOBZ='S')
*           M right singular vectors to be computed in VT and
*           M left singular vectors to be computed in U
*
              IL = 1
*
*             WORK(IL) is M by M
*
              LDWRKL = M
              ITAU = IL + LDWRKL*M
              NWORK = ITAU + M
```

```
*
*             Compute A=L*Q
*             (CWorkspace: need 2*M, prefer M+M*NB)
*             (RWorkspace: 0)
*
              CALL CGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*             Copy L to WORK(IL), zeroing out above it
*
              CALL CLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWRKL )
              CALL CLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                     WORK( IL+LDWRKL ), LDWRKL )
*
*             Generate Q in A
*             (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*             (RWorkspace: 0)
*
              CALL CUNGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
              IE = 1
              ITAUQ = ITAU
              ITAUP = ITAUQ + M
              NWORK = ITAUP + M
*
*             Bidiagonalize L in WORK(IL)
*             (CWorkspace: need M*M+3*M, prefer M*M+2*M+2*M*NB)
*             (RWorkspace: need M)
*
              CALL CGEBRD( M, M, WORK( IL ), LDWRKL, S, RWORK( IE ),
     $                     WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in RWORK(IRU) and computing right
*             singular vectors of bidiagonal matrix in RWORK(IRVT)
*             (CWorkspace: need 0)
*             (RWorkspace: need BDSPAC)
*
              IRU = IE + M
              IRVT = IRU + M*M
              NRWORK = IRVT + M*M
              CALL SBDSDC( 'U', 'I', M, S, RWORK( IE ), RWORK( IRU ),
     $                     M, RWORK( IRVT ), M, DUM, IDUM,
     $                     RWORK( NRWORK ), IWORK, INFO )
*
*             Copy real matrix RWORK(IRU) to complex matrix U
*             Overwrite U by left singular vectors of L
*             (CWorkspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*             (RWorkspace: 0)
```

```
*
               CALL CLACP2( 'F', M, M, RWORK( IRU ), M, U, LDU )
               CALL CUNMBR( 'Q', 'L', 'N', M, M, M, WORK( IL ), LDWRKL,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy real matrix RWORK(IRVT) to complex matrix VT
*              Overwrite VT by left singular vectors of L
*              (CWorkspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*              (RWorkspace: 0)
*
               CALL CLACP2( 'F', M, M, RWORK( IRVT ), M, VT, LDVT )
               CALL CUNMBR( 'P', 'R', 'C', M, M, M, WORK( IL ), LDWRKL,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy VT to WORK(IL), multiply right singular vectors of L
*              in WORK(IL) by Q in A, storing result in VT
*              (CWorkspace: need M*M)
*              (RWorkspace: 0)
*
               CALL CLACPY( 'F', M, M, VT, LDVT, WORK( IL ), LDWRKL )
               CALL CGEMM( 'N', 'N', M, N, M, CONE, WORK( IL ), LDWRKL,
     $                     A, LDA, CZERO, VT, LDVT )
*
            ELSE IF( WNTQA ) THEN
*
*              Path 9t (N much larger than M, JOBZ='A')
*              N right singular vectors to be computed in VT and
*              M left singular vectors to be computed in U
*
               IVT = 1
*
*              WORK(IVT) is M by M
*
               LDWKVT = M
               ITAU = IVT + LDWKVT*M
               NWORK = ITAU + M
*
*              Compute A=L*Q, copying result to VT
*              (CWorkspace: need 2*M, prefer M+M*NB)
*              (RWorkspace: 0)
*
               CALL CGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
               CALL CLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*              Generate Q in VT
*              (CWorkspace: need M+N, prefer M+N*NB)
*              (RWorkspace: 0)
```

```
*
               CALL CUNGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Produce L in A, zeroing out above it
*
               CALL CLASET( 'U', M-1, M-1, CZERO, CZERO, A( 1, 2 ),
     $                      LDA )
               IE = 1
               ITAUQ = ITAU
               ITAUP = ITAUQ + M
               NWORK = ITAUP + M
*
*              Bidiagonalize L in A
*              (CWorkspace: need M*M+3*M, prefer M*M+2*M+2*M*NB)
*              (RWorkspace: need M)
*
               CALL CGEBRD( M, M, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                      IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               IRU = IE + M
               IRVT = IRU + M*M
               NRWORK = IRVT + M*M
               CALL SBDSDC( 'U', 'I', M, S, RWORK( IE ), RWORK( IRU ),
     $                      M, RWORK( IRVT ), M, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Copy real matrix RWORK(IRU) to complex matrix U
*              Overwrite U by left singular vectors of L
*              (CWorkspace: need 3*M, prefer 2*M+M*NB)
*              (RWorkspace: 0)
*
               CALL CLACP2( 'F', M, M, RWORK( IRU ), M, U, LDU )
               CALL CUNMBR( 'Q', 'L', 'N', M, M, M, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy real matrix RWORK(IRVT) to complex matrix WORK(IVT)
*              Overwrite WORK(IVT) by right singular vectors of L
*              (CWorkspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*              (RWorkspace: 0)
*
               CALL CLACP2( 'F', M, M, RWORK( IRVT ), M, WORK( IVT ),
```

```
     $                      LDWKVT )
               CALL CUNMBR( 'P', 'R', 'C', M, M, M, A, LDA,
     $                      WORK( ITAUP ), WORK( IVT ), LDWKVT,
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Multiply right singular vectors of L in WORK(IVT) by
*              Q in VT, storing result in A
*              (CWorkspace: need M*M)
*              (RWorkspace: 0)
*
               CALL CGEMM( 'N', 'N', M, N, M, CONE, WORK( IVT ),
     $                     LDWKVT, VT, LDVT, CZERO, A, LDA )
*
*              Copy right singular vectors of A from A to VT
*
               CALL CLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
            END IF
*
         ELSE IF( N.GE.MNTHR2 ) THEN
*
*           MNTHR2 <= N < MNTHR1
*
*           Path 5t (N much larger than M, but not as much as MNTHR1)
*           Reduce to bidiagonal form without QR decomposition, use
*           CUNGBR and matrix multiplication to compute singular vectors
*
*
            IE = 1
            NRWORK = IE + M
            ITAUQ = 1
            ITAUP = ITAUQ + M
            NWORK = ITAUP + M
*
*           Bidiagonalize A
*           (CWorkspace: need 2*M+N, prefer 2*M+(M+N)*NB)
*           (RWorkspace: M)
*
            CALL CGEBRD( M, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                   WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                   IERR )
*
            IF( WNTQN ) THEN
*
*              Compute singular values only
*              (Cworkspace: 0)
*              (Rworkspace: need BDSPAN)
*
               CALL SBDSDC( 'L', 'N', M, S, RWORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, RWORK( NRWORK ), IWORK, INFO )
```

```
            ELSE IF( WNTQO ) THEN
               IRVT = NRWORK
               IRU = IRVT + M*M
               NRWORK = IRU + M*M
               IVT = NWORK
*
*              Copy A to U, generate Q
*              (Cworkspace: need 2*M, prefer M+M*NB)
*              (Rworkspace: 0)
*
               CALL CLACPY( 'L', M, M, A, LDA, U, LDU )
               CALL CUNGBR( 'Q', M, M, N, U, LDU, WORK( ITAUQ ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Generate P**H in A
*              (Cworkspace: need 2*M, prefer M+M*NB)
*              (Rworkspace: 0)
*
               CALL CUNGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
               LDWKVT = M
               IF( LWORK.GE.M*N+3*M ) THEN
*
*                 WORK( IVT ) is M by N
*
                  NWORK = IVT + LDWKVT*N
                  CHUNK = N
               ELSE
*
*                 WORK( IVT ) is M by CHUNK
*
                  CHUNK = ( LWORK-3*M ) / M
                  NWORK = IVT + LDWKVT*CHUNK
               END IF
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               CALL SBDSDC( 'L', 'I', M, S, RWORK( IE ), RWORK( IRU ),
     $                      M, RWORK( IRVT ), M, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Multiply Q in U by real matrix RWORK(IRVT)
*              storing the result in WORK(IVT), copying to U
*              (Cworkspace: need 0)
*              (Rworkspace: need 2*M*M)
```

```
*
               CALL CLACRM( M, M, U, LDU, RWORK( IRU ), M, WORK( IVT ),
     $                      LDWKVT, RWORK( NRWORK ) )
               CALL CLACPY( 'F', M, M, WORK( IVT ), LDWKVT, U, LDU )
*
*              Multiply RWORK(IRVT) by P**H in A, storing the
*              result in WORK(IVT), copying to A
*              (CWorkspace: need M*M, prefer M*N)
*              (Rworkspace: need 2*M*M, prefer 2*M*N)
*
               NRWORK = IRU
               DO 50 I = 1, N, CHUNK
                  BLK = MIN( N-I+1, CHUNK )
                  CALL CLARCM( M, BLK, RWORK( IRVT ), M, A( 1, I ), LDA,
     $                         WORK( IVT ), LDWKVT, RWORK( NRWORK ) )
                  CALL CLACPY( 'F', M, BLK, WORK( IVT ), LDWKVT,
     $                         A( 1, I ), LDA )
   50          CONTINUE
            ELSE IF( WNTQS ) THEN
*
*              Copy A to U, generate Q
*              (Cworkspace: need 2*M, prefer M+M*NB)
*              (Rworkspace: 0)
*
               CALL CLACPY( 'L', M, M, A, LDA, U, LDU )
               CALL CUNGBR( 'Q', M, M, N, U, LDU, WORK( ITAUQ ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Copy A to VT, generate P**H
*              (Cworkspace: need 2*M, prefer M+M*NB)
*              (Rworkspace: 0)
*
               CALL CLACPY( 'U', M, N, A, LDA, VT, LDVT )
               CALL CUNGBR( 'P', M, N, M, VT, LDVT, WORK( ITAUP ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               IRVT = NRWORK
               IRU = IRVT + M*M
               NRWORK = IRU + M*M
               CALL SBDSDC( 'L', 'I', M, S, RWORK( IE ), RWORK( IRU ),
     $                      M, RWORK( IRVT ), M, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Multiply Q in U by real matrix RWORK(IRU), storing the
```

```
*             result in A, copying to U
*             (CWorkspace: need 0)
*             (Rworkspace: need 3*M*M)
*
              CALL CLACRM( M, M, U, LDU, RWORK( IRU ), M, A, LDA,
     $                     RWORK( NRWORK ) )
              CALL CLACPY( 'F', M, M, A, LDA, U, LDU )
*
*             Multiply real matrix RWORK(IRVT) by P**H in VT,
*             storing the result in A, copying to VT
*             (Cworkspace: need 0)
*             (Rworkspace: need M*M+2*M*N)
*
              NRWORK = IRU
              CALL CLARCM( M, N, RWORK( IRVT ), M, VT, LDVT, A, LDA,
     $                     RWORK( NRWORK ) )
              CALL CLACPY( 'F', M, N, A, LDA, VT, LDVT )
           ELSE
*
*             Copy A to U, generate Q
*             (Cworkspace: need 2*M, prefer M+M*NB)
*             (Rworkspace: 0)
*
              CALL CLACPY( 'L', M, M, A, LDA, U, LDU )
              CALL CUNGBR( 'Q', M, M, N, U, LDU, WORK( ITAUQ ),
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*             Copy A to VT, generate P**H
*             (Cworkspace: need 2*M, prefer M+M*NB)
*             (Rworkspace: 0)
*
              CALL CLACPY( 'U', M, N, A, LDA, VT, LDVT )
              CALL CUNGBR( 'P', N, N, M, VT, LDVT, WORK( ITAUP ),
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in RWORK(IRU) and computing right
*             singular vectors of bidiagonal matrix in RWORK(IRVT)
*             (CWorkspace: need 0)
*             (RWorkspace: need BDSPAC)
*
              IRVT = NRWORK
              IRU = IRVT + M*M
              NRWORK = IRU + M*M
              CALL SBDSDC( 'L', 'I', M, S, RWORK( IE ), RWORK( IRU ),
     $                     M, RWORK( IRVT ), M, DUM, IDUM,
     $                     RWORK( NRWORK ), IWORK, INFO )
*
*             Multiply Q in U by real matrix RWORK(IRU), storing the
*             result in A, copying to U
```

```
*              (CWorkspace: need 0)
*              (Rworkspace: need 3*M*M)
*
               CALL CLACRM( M, M, U, LDU, RWORK( IRU ), M, A, LDA,
     $                      RWORK( NRWORK ) )
               CALL CLACPY( 'F', M, M, A, LDA, U, LDU )
*
*              Multiply real matrix RWORK(IRVT) by P**H in VT,
*              storing the result in A, copying to VT
*              (Cworkspace: need 0)
*              (Rworkspace: need M*M+2*M*N)
*
               CALL CLARCM( M, N, RWORK( IRVT ), M, VT, LDVT, A, LDA,
     $                      RWORK( NRWORK ) )
               CALL CLACPY( 'F', M, N, A, LDA, VT, LDVT )
            END IF
*
         ELSE
*
            N .LT. MNTHR2
*
            Path 6t (N greater than M, but not much larger)
            Reduce to bidiagonal form without LQ decomposition
            Use CUNMBR to compute singular vectors
*
            IE = 1
            NRWORK = IE + M
            ITAUQ = 1
            ITAUP = ITAUQ + M
            NWORK = ITAUP + M
*
            Bidiagonalize A
            (CWorkspace: need 2*M+N, prefer 2*M+(M+N)*NB)
            (RWorkspace: M)
*
            CALL CGEBRD( M, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                   WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                   IERR )
            IF( WNTQN ) THEN
*
               Compute singular values only
               (Cworkspace: 0)
               (Rworkspace: need BDSPAN)
*
               CALL SBDSDC( 'L', 'N', M, S, RWORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, RWORK( NRWORK ), IWORK, INFO )
            ELSE IF( WNTQO ) THEN
               LDWKVT = M
               IVT = NWORK
               IF( LWORK.GE.M*N+3*M ) THEN
```

```
*
*                 WORK( IVT ) is M by N
*
                  CALL CLASET( 'F', M, N, CZERO, CZERO, WORK( IVT ),
     $                         LDWKVT )
                  NWORK = IVT + LDWKVT*N
               ELSE
*
*                 WORK( IVT ) is M by CHUNK
*
                  CHUNK = ( LWORK-3*M ) / M
                  NWORK = IVT + LDWKVT*CHUNK
               END IF
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               IRVT = NRWORK
               IRU = IRVT + M*M
               NRWORK = IRU + M*M
               CALL SBDSDC( 'L', 'I', M, S, RWORK( IE ), RWORK( IRU ),
     $                      M, RWORK( IRVT ), M, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Copy real matrix RWORK(IRU) to complex matrix U
*              Overwrite U by left singular vectors of A
*              (Cworkspace: need 2*M, prefer M+M*NB)
*              (Rworkspace: need 0)
*
               CALL CLACP2( 'F', M, M, RWORK( IRU ), M, U, LDU )
               CALL CUNMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
               IF( LWORK.GE.M*N+3*M ) THEN
*
*              Copy real matrix RWORK(IRVT) to complex matrix WORK(IVT)
*              Overwrite WORK(IVT) by right singular vectors of A,
*              copying to A
*              (Cworkspace: need M*N+2*M, prefer M*N+M+M*NB)
*              (Rworkspace: need 0)
*
                  CALL CLACP2( 'F', M, M, RWORK( IRVT ), M, WORK( IVT ),
     $                         LDWKVT )
                  CALL CUNMBR( 'P', 'R', 'C', M, N, M, A, LDA,
     $                         WORK( ITAUP ), WORK( IVT ), LDWKVT,
     $                         WORK( NWORK ), LWORK-NWORK+1, IERR )
```

```
                 CALL CLACPY( 'F', M, N, WORK( IVT ), LDWKVT, A, LDA )
              ELSE
*
*                Generate P**H in A
*                (Cworkspace: need 2*M, prefer M+M*NB)
*                (Rworkspace: need 0)
*
                 CALL CUNGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                        WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                Multiply Q in A by real matrix RWORK(IRU), storing the
*                result in WORK(IU), copying to A
*                (CWorkspace: need M*M, prefer M*N)
*                (Rworkspace: need 3*M*M, prefer M*M+2*M*N)
*
                 NRWORK = IRU
                 DO 60 I = 1, N, CHUNK
                    BLK = MIN( N-I+1, CHUNK )
                    CALL CLARCM( M, BLK, RWORK( IRVT ), M, A( 1, I ),
     $                           LDA, WORK( IVT ), LDWKVT,
     $                           RWORK( NRWORK ) )
                    CALL CLACPY( 'F', M, BLK, WORK( IVT ), LDWKVT,
     $                           A( 1, I ), LDA )
   60            CONTINUE
              END IF
           ELSE IF( WNTQS ) THEN
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in RWORK(IRU) and computing right
*             singular vectors of bidiagonal matrix in RWORK(IRVT)
*             (CWorkspace: need 0)
*             (RWorkspace: need BDSPAC)
*
              IRVT = NRWORK
              IRU = IRVT + M*M
              NRWORK = IRU + M*M
              CALL SBDSDC( 'L', 'I', M, S, RWORK( IE ), RWORK( IRU ),
     $                     M, RWORK( IRVT ), M, DUM, IDUM,
     $                     RWORK( NRWORK ), IWORK, INFO )
*
*             Copy real matrix RWORK(IRU) to complex matrix U
*             Overwrite U by left singular vectors of A
*             (CWorkspace: need 3*M, prefer 2*M+M*NB)
*             (RWorkspace: M*M)
*
              CALL CLACP2( 'F', M, M, RWORK( IRU ), M, U, LDU )
              CALL CUNMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
```

```
*              Copy real matrix RWORK(IRVT) to complex matrix VT
*              Overwrite VT by right singular vectors of A
*              (CWorkspace: need 3*M, prefer 2*M+M*NB)
*              (RWorkspace: M*M)
*
               CALL CLASET( 'F', M, N, CZERO, CZERO, VT, LDVT )
               CALL CLACP2( 'F', M, M, RWORK( IRVT ), M, VT, LDVT )
               CALL CUNMBR( 'P', 'R', 'C', M, N, M, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
            ELSE
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               IRVT = NRWORK
               IRU = IRVT + M*M
               NRWORK = IRU + M*M
*
               CALL SBDSDC( 'L', 'I', M, S, RWORK( IE ), RWORK( IRU ),
     $                      M, RWORK( IRVT ), M, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Copy real matrix RWORK(IRU) to complex matrix U
*              Overwrite U by left singular vectors of A
*              (CWorkspace: need 3*M, prefer 2*M+M*NB)
*              (RWorkspace: M*M)
*
               CALL CLACP2( 'F', M, M, RWORK( IRU ), M, U, LDU )
               CALL CUNMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Set all of VT to identity matrix
*
               CALL CLASET( 'F', N, N, CZERO, CONE, VT, LDVT )
*
*              Copy real matrix RWORK(IRVT) to complex matrix VT
*              Overwrite VT by right singular vectors of A
*              (CWorkspace: need 2*M+N, prefer 2*M+N*NB)
*              (RWorkspace: M*M)
*
               CALL CLACP2( 'F', M, M, RWORK( IRVT ), M, VT, LDVT )
               CALL CUNMBR( 'P', 'R', 'C', N, N, M, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
            END IF
```

```
*
         END IF
*
      END IF
*
*     Undo scaling if necessary
*
      IF( ISCL.EQ.1 ) THEN
         IF( ANRM.GT.BIGNUM )
     $      CALL SLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( INFO.NE.0 .AND. ANRM.GT.BIGNUM )
     $      CALL SLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN-1, 1,
     $                   RWORK( IE ), MINMN, IERR )
         IF( ANRM.LT.SMLNUM )
     $      CALL SLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( INFO.NE.0 .AND. ANRM.LT.SMLNUM )
     $      CALL SLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN-1, 1,
     $                   RWORK( IE ), MINMN, IERR )
      END IF
*
*     Return optimal workspace in WORK(1)
*
      WORK( 1 ) = MAXWRK
*
      RETURN
*
*     End of CGESDD
*
      END
```

———-

30.2.4 lisp code

— cgesdd.lisp —

```
(let* ((lquerv (- 1))
       (czero (f2cl-lib:cmplx 0.0 0.0))
       (cone (f2cl-lib:cmplx 1.0 0.0))
       (zero 0.0)
       (one 1.0))
  (declare (type (f2cl-lib:integer4) lquerv)
           (type (f2cl-lib:complex8) czero)
           (type (f2cl-lib:complex8) cone)
           (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
```

```
          (ignorable lquerv czero cone zero one))
 (defun cgesdd (jobz m n a lda s u ldu vt ldvt work lwork rwork iwork info)
   (declare (type (array f2cl-lib:integer4 (*)) iwork)
            (type (array single-float (*)) rwork s)
            (type (array f2cl-lib:complex8 (*)) work vt u a)
            (type (f2cl-lib:integer4) info lwork ldvt ldu lda n m)
            (type (string *) jobz))
   (f2cl-lib:with-multi-array-data
       ((jobz character jobz-%data% jobz-%offset%)
        (a f2cl-lib:complex8 a-%data% a-%offset%)
        (u f2cl-lib:complex8 u-%data% u-%offset%)
        (vt f2cl-lib:complex8 vt-%data% vt-%offset%)
        (work f2cl-lib:complex8 work-%data% work-%offset%)
        (s single-float s-%data% s-%offset%)
        (rwork single-float rwork-%data% rwork-%offset%)
        (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
     (prog ((dum (make-array 1 :element-type 'single-float))
            (idum (make-array 1 :element-type 'f2cl-lib:integer4)) (anrm 0.0)
            (bignum 0.0) (eps 0.0) (smlnum 0.0) (blk 0) (chunk 0) (i 0) (ie 0)
            (ierr 0) (il 0) (ir 0) (iru 0) (irvt 0) (iscl 0) (itau 0)
            (itaup 0) (itauq 0) (iu 0) (ivt 0) (ldwkvt 0) (ldwrkl 0)
            (ldwrkr 0) (ldwrku 0) (maxwrk 0) (minmn 0) (minwrk 0) (mnthr1 0)
            (mnthr2 0) (nrwork 0) (nwork 0) (wrkbl 0) (wntqa nil) (wntqas nil)
            (wntqn nil) (wntqo nil) (wntqs nil))
       (declare (type (array single-float (1)) dum)
                (type (array f2cl-lib:integer4 (1)) idum)
                (type (single-float) anrm bignum eps smlnum)
                (type (f2cl-lib:integer4) blk chunk i ie ierr il ir iru irvt
                 iscl itau itaup itauq iu ivt ldwkvt ldwrkl ldwrkr ldwrku
                 maxwrk minmn minwrk mnthr1 mnthr2 nrwork nwork wrkbl)
                (type f2cl-lib:logical wntqa wntqas wntqn wntqo wntqs))
       (setf info 0)
       (setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
       (setf mnthr1 (f2cl-lib:int (/ (* minmn 17.0) 9.0)))
       (setf mnthr2 (f2cl-lib:int (/ (* minmn 5.0) 3.0)))
       (setf wntqa
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame jobz "A")
                 (declare (ignore var-1))
                 (when var-0 (setf jobz var-0))
                 ret-val))
       (setf wntqs
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame jobz "S")
                 (declare (ignore var-1))
                 (when var-0 (setf jobz var-0))
                 ret-val))
       (setf wntqas (or wntqa wntqs))
       (setf wntqo
               (multiple-value-bind (ret-val var-0 var-1)
```

```
            (lsame jobz "O")
          (declare (ignore var-1))
          (when var-0 (setf jobz var-0))
          ret-val))
(setf wntqn
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobz "N")
          (declare (ignore var-1))
          (when var-0 (setf jobz var-0))
          ret-val))
(setf minwrk 1)
(setf maxwrk 1)
(cond ((not (or wntqa wntqs wntqo wntqn)) (setf info -1))
      ((< m 0) (setf info -2)) ((< n 0) (setf info -3))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -5))
      ((or (< ldu 1) (and wntqas (< ldu m))
           (and wntqo (< m n) (< ldu m)))
       (setf info -8))
      ((or (< ldvt 1) (and wntqa (< ldvt n)) (and wntqs (< ldvt minmn))
           (and wntqo (>= m n) (< ldvt n)))
       (setf info -10)))
(cond
 ((and (= info 0) (> m 0) (> n 0))
  (cond
   ((>= m n)
    (cond
     ((>= m mnthr1)
      (cond
       (wntqn
        (setf maxwrk
                (f2cl-lib:int-add n
                                  (f2cl-lib:int-mul n
                                                    (multiple-value-bind
                                                        (ret-val var-0
                                                         var-1 var-2
                                                         var-3 var-4
                                                         var-5 var-6)
                                                        (ilaenv 1
                                                         "CGEQRF" " " m
                                                         n -1 -1)
                                                      (declare
                                                       (ignore var-0
                                                        var-1 var-2
                                                        var-5 var-6))
                                                      (when var-3
                                                        (setf m var-3))
                                                      (when var-4
                                                        (setf n var-4))
```

```
                                                ret-val))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     (f2cl-lib:int-mul 2 n
                                                       (multiple-value-bind
                                                           (ret-val
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-3
                                                            var-4
                                                            var-5
                                                            var-6)
                                                           (ilaenv
                                                            1
                                                            "CGEBRD"
                                                            " "
                                                            n n
                                                            -1
                                                            -1)
                                                         (declare
                                                          (ignore
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-5
                                                           var-6))
                                                         (when
                                                             var-3
                                                           (setf n
                                                                   var-3))
                                                         (when
                                                             var-4
                                                           (setf n
                                                                   var-4))
                                                         ret-val))))))
 (setf minwrk (f2cl-lib:int-mul 3 n)))
(wntqo
 (setf wrkbl
         (f2cl-lib:int-add n
                           (f2cl-lib:int-mul n
                                             (multiple-value-bind
                                                 (ret-val var-0
                                                  var-1 var-2
                                                  var-3 var-4
                                                  var-5 var-6)
                                                 (ilaenv 1
                                                  "CGEQRF" " " m
```

```
                                                    n -1 -1)
                                                 (declare
                                                  (ignore var-0
                                                   var-1 var-2
                                                   var-5 var-6))
                                                 (when var-3
                                                   (setf m var-3))
                                                 (when var-4
                                                   (setf n var-4))
                                                 ret-val))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add n
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "CUNGQR"
                                                           " "
                                                           m n
                                                           n
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (when
                                                            var-3
                                                          (setf m
                                                                var-3))
                                                        (when
                                                            var-4
                                                          (setf n
                                                                var-4))
                                                        (when
                                                            var-5
                                                          (setf n
                                                                var-5))
                                                        ret-val))))))
```

```
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul 2 n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "CGEBRD"
                                                           " "
                                                           n n
                                                           -1
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-5
                                                          var-6))
                                                        (when
                                                            var-3
                                                          (setf n
                                                                  var-3))
                                                        (when
                                                            var-4
                                                          (setf n
                                                                  var-4))
                                                        ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
```

```
                                                                var-6)
                                                               (ilaenv
                                                                1
                                                                "CUNMBR"
                                                                "QLN"
                                                                n n
                                                                n
                                                                -1)
                                                              (declare
                                                               (ignore
                                                                var-0
                                                                var-1
                                                                var-2
                                                                var-6))
                                                              (when
                                                                  var-3
                                                               (setf n
                                                                        var-3))
                                                              (when
                                                                  var-4
                                                               (setf n
                                                                        var-4))
                                                              (when
                                                                  var-5
                                                               (setf n
                                                                        var-5))
                                                              ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "CUNMBR"
                                                           "PRC"
                                                           n n
                                                           n
                                                           -1)
                                                          (declare
                                                           (ignore
```

```
                                                                var-0
                                                                var-1
                                                                var-2
                                                                var-6))
                                                              (when
                                                                  var-3
                                                                (setf n
                                                                        var-3))
                                                              (when
                                                                  var-4
                                                                (setf n
                                                                        var-4))
                                                              (when
                                                                  var-5
                                                                (setf n
                                                                        var-5))
                                                              ret-val))))))
                (setf maxwrk
                        (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                                          (f2cl-lib:int-mul n n) wrkbl))
                (setf minwrk
                        (f2cl-lib:int-add (f2cl-lib:int-mul 2 n n)
                                          (f2cl-lib:int-mul 3 n))))
               (wntqs
                (setf wrkbl
                        (f2cl-lib:int-add n
                                          (f2cl-lib:int-mul n
                                                            (multiple-value-bind
                                                                (ret-val var-0
                                                                 var-1 var-2
                                                                 var-3 var-4
                                                                 var-5 var-6)
                                                                (ilaenv 1
                                                                 "CGEQRF" " " m
                                                                 n -1 -1)
                                                              (declare
                                                               (ignore var-0
                                                                var-1 var-2
                                                                var-5 var-6))
                                                              (when var-3
                                                                (setf m var-3))
                                                              (when var-4
                                                                (setf n var-4))
                                                              ret-val))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add n
                                                    (f2cl-lib:int-mul n
                                                                      (multiple-value-bind
```

```
                                                                    (ret-val
                                                                     var-0
                                                                     var-1
                                                                     var-2
                                                                     var-3
                                                                     var-4
                                                                     var-5
                                                                     var-6)
                                                                    (ilaenv
                                                                     1
                                                                     "CUNGQR"
                                                                     " "
                                                                     m n
                                                                     n
                                                                     -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf n
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf n
                                                                            var-5))
                                                                  ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul 2 n
                                                      (multiple-value-bind
                                                                    (ret-val
                                                                     var-0
                                                                     var-1
                                                                     var-2
                                                                     var-3
                                                                     var-4
                                                                     var-5
                                                                     var-6)
                                                                    (ilaenv
                                                                     1
```

```
                                                                    "CGEBRD"
                                                                    " "
                                                                    n n
                                                                    -1
                                                                    -1)
                                                                 (declare
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-5
                                                                   var-6))
                                                                 (when
                                                                     var-3
                                                                   (setf n
                                                                           var-3))
                                                                 (when
                                                                     var-4
                                                                   (setf n
                                                                           var-4))
                                                                 ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "CUNMBR"
                                                           "QLN"
                                                           n n
                                                           n
                                                           -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-6))
                                                          (when
                                                              var-3
```

```
                                                                    (setf n
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf n
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf n
                                                                            var-5))
                                                                  ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "CUNMBR"
                                                           "PRC"
                                                           n n
                                                           n
                                                           -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-6))
                                                          (when
                                                              var-3
                                                            (setf n
                                                                    var-3))
                                                          (when
                                                              var-4
                                                            (setf n
                                                                    var-4))
                                                          (when
                                                              var-5
                                                            (setf n
                                                                    var-5))
```

```
                                                          ret-val))))))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
 (setf minwrk
         (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                           (f2cl-lib:int-mul 3 n))))
(wntqa
 (setf wrkbl
         (f2cl-lib:int-add n
                           (f2cl-lib:int-mul n
                                             (multiple-value-bind
                                                 (ret-val var-0
                                                  var-1 var-2
                                                  var-3 var-4
                                                  var-5 var-6)
                                                 (ilaenv 1
                                                  "CGEQRF" " " m
                                                  n -1 -1)
                                               (declare
                                                (ignore var-0
                                                 var-1 var-2
                                                 var-5 var-6))
                                               (when var-3
                                                 (setf m var-3))
                                               (when var-4
                                                 (setf n var-4))
                                               ret-val))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n
                                     (f2cl-lib:int-mul m
                                                       (multiple-value-bind
                                                           (ret-val
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-3
                                                            var-4
                                                            var-5
                                                            var-6)
                                                           (ilaenv
                                                            1
                                                            "CUNGQR"
                                                            " "
                                                            m m
                                                            n
                                                            -1)
                                                         (declare
                                                          (ignore
                                                           var-0
```

```
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf m
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf n
                                                                            var-5))
                                                                  ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul 2 n
                                                                 (multiple-value-bind
                                                                     (ret-val
                                                                      var-0
                                                                      var-1
                                                                      var-2
                                                                      var-3
                                                                      var-4
                                                                      var-5
                                                                      var-6)
                                                                     (ilaenv
                                                                      1
                                                                      "CGEBRD"
                                                                      " "
                                                                      n n
                                                                      -1
                                                                      -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-5
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf n
                                                                            var-3))
                                                                  (when
                                                                      var-4
```

```
                                                                  (setf n
                                                                          var-4))
                                                                ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                    (f2cl-lib:int-mul n
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "CUNMBR"
                                                                           "QLN"
                                                                           n n
                                                                           n
                                                                           -1)
                                                                        (declare
                                                                         (ignore
                                                                          var-0
                                                                          var-1
                                                                          var-2
                                                                          var-6))
                                                                        (when
                                                                            var-3
                                                                          (setf n
                                                                                  var-3))
                                                                        (when
                                                                            var-4
                                                                          (setf n
                                                                                  var-4))
                                                                        (when
                                                                            var-5
                                                                          (setf n
                                                                                  var-5))
                                                                        ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                    (f2cl-lib:int-mul n
                                                                      (multiple-value-bind
                                                                          (ret-val
```

```
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-3
                                                                    var-4
                                                                    var-5
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "CUNMBR"
                                                                    "PRC"
                                                                    n n
                                                                    n
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf n
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf n
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf n
                                                                            var-5))
                                                                  ret-val))))))
                  (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
                  (setf minwrk
                          (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                                            (f2cl-lib:int-mul 2 n) m)))))
                ((>= m mnthr2)
                 (setf maxwrk
                         (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                           (f2cl-lib:int-mul
                                            (f2cl-lib:int-add m n)
                                            (multiple-value-bind
                                                (ret-val var-0 var-1 var-2 var-3
                                                 var-4 var-5 var-6)
                                                (ilaenv 1 "CGEBRD" " " m n -1 -1)
                                              (declare
                                               (ignore var-0 var-1 var-2 var-5
                                                var-6))
                                              (when var-3 (setf m var-3))
```

```
                              (when var-4 (setf n var-4))
                              ret-val))))
(setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m))
(cond
 (wntqo
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                      (f2cl-lib:int-mul n
                                                        (multiple-value-bind
                                                            (ret-val
                                                             var-0
                                                             var-1
                                                             var-2
                                                             var-3
                                                             var-4
                                                             var-5
                                                             var-6)
                                                            (ilaenv
                                                             1
                                                             "CUNGBR"
                                                             "P"
                                                             n n
                                                             n
                                                             -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-6))
                                                          (when
                                                              var-3
                                                            (setf n
                                                                    var-3))
                                                          (when
                                                              var-4
                                                            (setf n
                                                                    var-4))
                                                          (when
                                                              var-5
                                                            (setf n
                                                                    var-5))
                                                          ret-val))))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                      (f2cl-lib:int-mul n
```

```
                                                                    (multiple-value-bind
                                                                        (ret-val
                                                                         var-0
                                                                         var-1
                                                                         var-2
                                                                         var-3
                                                                         var-4
                                                                         var-5
                                                                         var-6)
                                                                        (ilaenv
                                                                         1
                                                                         "CUNGBR"
                                                                         "Q"
                                                                         m n
                                                                         n
                                                                         -1)
                                                                      (declare
                                                                       (ignore
                                                                        var-0
                                                                        var-1
                                                                        var-2
                                                                        var-6))
                                                                      (when
                                                                          var-3
                                                                        (setf m
                                                                                var-3))
                                                                      (when
                                                                          var-4
                                                                        (setf n
                                                                                var-4))
                                                                      (when
                                                                          var-5
                                                                        (setf n
                                                                                var-5))
                                                                      ret-val))))))
                                (setf maxwrk (f2cl-lib:int-add maxwrk (f2cl-lib:int-mul m n)))
                                (setf minwrk (f2cl-lib:int-add minwrk (f2cl-lib:int-mul n n))))
                               (wntqs
                                (setf maxwrk
                                        (max (the f2cl-lib:integer4 maxwrk)
                                             (the f2cl-lib:integer4
                                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                                    (f2cl-lib:int-mul n
                                                                    (multiple-value-bind
                                                                        (ret-val
                                                                         var-0
                                                                         var-1
                                                                         var-2
                                                                         var-3
                                                                         var-4
```

```
                                                                    var-5
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "CUNGBR"
                                                                    "P"
                                                                    n n
                                                                    n
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf n
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf n
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf n
                                                                            var-5))
                                                                  ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "CUNGBR"
                                                           "Q"
                                                           m n
                                                           n
                                                           -1)
                                                          (declare
```

```
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-6))
                                                                 (when
                                                                     var-3
                                                                   (setf m
                                                                           var-3))
                                                                 (when
                                                                     var-4
                                                                   (setf n
                                                                           var-4))
                                                                 (when
                                                                     var-5
                                                                   (setf n
                                                                           var-5))
                                                                 ret-val)))))))
                (wntqa
                 (setf maxwrk
                         (max (the f2cl-lib:integer4 maxwrk)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                     (f2cl-lib:int-mul n
                                                                       (multiple-value-bind
                                                                           (ret-val
                                                                            var-0
                                                                            var-1
                                                                            var-2
                                                                            var-3
                                                                            var-4
                                                                            var-5
                                                                            var-6)
                                                                           (ilaenv
                                                                            1
                                                                            "CUNGBR"
                                                                            "P"
                                                                            n n
                                                                            n
                                                                            -1)
                                                                         (declare
                                                                          (ignore
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-6))
                                                                         (when
                                                                             var-3
                                                                           (setf n
                                                                                   var-3))
```

```
                                                                  (when
                                                                      var-4
                                                                    (setf n
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf n
                                                                            var-5))
                                                                  ret-val))))))
                (setf maxwrk
                        (max (the f2cl-lib:integer4 maxwrk)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                    (f2cl-lib:int-mul m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "CUNGBR"
                                                                           "Q"
                                                                           m m
                                                                           n
                                                                           -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf m
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf n
                                                                            var-5))
                                                                  ret-val)))))))))
                (t
```

```
(setf maxwrk
        (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                          (f2cl-lib:int-mul
                           (f2cl-lib:int-add m n)
                           (multiple-value-bind
                               (ret-val var-0 var-1 var-2 var-3
                                var-4 var-5 var-6)
                               (ilaenv 1 "CGEBRD" " " m n -1 -1)
                             (declare
                              (ignore var-0 var-1 var-2 var-5
                              var-6))
                             (when var-3 (setf m var-3))
                             (when var-4 (setf n var-4))
                             ret-val))))
(setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m))
(cond
 (wntqo
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                      (f2cl-lib:int-mul n
                                                        (multiple-value-bind
                                                            (ret-val
                                                             var-0
                                                             var-1
                                                             var-2
                                                             var-3
                                                             var-4
                                                             var-5
                                                             var-6)
                                                            (ilaenv
                                                             1
                                                             "CUNMBR"
                                                             "PRC"
                                                             n n
                                                             n
                                                             -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-6))
                                                          (when
                                                              var-3
                                                            (setf n
                                                                    var-3))
                                                          (when
                                                              var-4
```

```
                                                                  (setf n
                                                                          var-4))
                                                                (when
                                                                    var-5
                                                                  (setf n
                                                                          var-5))
                                                                ret-val))))))
                (setf maxwrk
                        (max (the f2cl-lib:integer4 maxwrk)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                    (f2cl-lib:int-mul n
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "CUNMBR"
                                                                           "QLN"
                                                                           m n
                                                                           n
                                                                           -1)
                                                                        (declare
                                                                         (ignore
                                                                          var-0
                                                                          var-1
                                                                          var-2
                                                                          var-6))
                                                                        (when
                                                                            var-3
                                                                          (setf m
                                                                                  var-3))
                                                                        (when
                                                                            var-4
                                                                          (setf n
                                                                                  var-4))
                                                                        (when
                                                                            var-5
                                                                          (setf n
                                                                                  var-5))
                                                                        ret-val))))))
                (setf maxwrk (f2cl-lib:int-add maxwrk (f2cl-lib:int-mul m n)))
                (setf minwrk (f2cl-lib:int-add minwrk (f2cl-lib:int-mul n n))))
               (wntqs
```

```
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "CUNMBR"
                                                           "PRC"
                                                           n n
                                                           n
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (when
                                                            var-3
                                                          (setf n
                                                                  var-3))
                                                        (when
                                                            var-4
                                                          (setf n
                                                                  var-4))
                                                        (when
                                                            var-5
                                                          (setf n
                                                                  var-5))
                                                        ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
```

```
                                                                    var-3
                                                                    var-4
                                                                    var-5
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "CUNMBR"
                                                                    "QLN"
                                                                    m n
                                                                    n
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf n
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf n
                                                                            var-5))
                                                                  ret-val)))))))
                                                                 (wntqa
                                                                  (setf maxwrk
                                                                          (max (the f2cl-lib:integer4 maxwrk)
                                                                               (the f2cl-lib:integer4
                                                                                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                                                                      (f2cl-lib:int-mul n
                                                                                                                        (multiple-value-bind
                                                                                                                            (ret-val
                                                                                                                             var-0
                                                                                                                             var-1
                                                                                                                             var-2
                                                                                                                             var-3
                                                                                                                             var-4
                                                                                                                             var-5
                                                                                                                             var-6)
                                                                                                                            (ilaenv
                                                                                                                             1
                                                                                                                             "CUNGBR"
                                                                                                                             "PRC"
                                                                                                                             n n
```

```
                                                                    n
                                                                    -1)
                                                                (declare
                                                                 (ignore
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-6))
                                                                (when
                                                                    var-3
                                                                  (setf n
                                                                          var-3))
                                                                (when
                                                                    var-4
                                                                  (setf n
                                                                          var-4))
                                                                (when
                                                                    var-5
                                                                  (setf n
                                                                          var-5))
                                                                ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "CUNGBR"
                                                           "QLN"
                                                           m m
                                                           n
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (when
                                                            var-3
```

```
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf m
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf n
                                                                            var-5))
                                                                  ret-val)))))))))))
            (t
             (cond
              ((>= n mnthr1)
               (cond
                (wntqn
                 (setf maxwrk
                         (f2cl-lib:int-add m
                                           (f2cl-lib:int-mul m
                                                             (multiple-value-bind
                                                                 (ret-val var-0
                                                                  var-1 var-2
                                                                  var-3 var-4
                                                                  var-5 var-6)
                                                                 (ilaenv 1
                                                                  "CGELQF" " " m
                                                                  n -1 -1)
                                                               (declare
                                                                (ignore var-0
                                                                 var-1 var-2
                                                                 var-5 var-6))
                                                               (when var-3
                                                                 (setf m var-3))
                                                               (when var-4
                                                                 (setf n var-4))
                                                               ret-val))))
                 (setf maxwrk
                         (max (the f2cl-lib:integer4 maxwrk)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                     (f2cl-lib:int-mul 2 m
                                                                       (multiple-value-bind
                                                                           (ret-val
                                                                            var-0
                                                                            var-1
                                                                            var-2
                                                                            var-3
                                                                            var-4
                                                                            var-5
                                                                            var-6)
```

```
                                                                    (ilaenv
                                                                     1
                                                                     "CGEBRD"
                                                                     " "
                                                                     m m
                                                                     -1
                                                                     -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-5
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf m
                                                                            var-4))
                                                                  ret-val))))))
                 (setf minwrk (f2cl-lib:int-mul 3 m)))
                (wntqo
                 (setf wrkbl
                         (f2cl-lib:int-add m
                                           (f2cl-lib:int-mul m
                                                             (multiple-value-bind
                                                                 (ret-val var-0
                                                                  var-1 var-2
                                                                  var-3 var-4
                                                                  var-5 var-6)
                                                                 (ilaenv 1
                                                                  "CGELQF" " " m
                                                                  n -1 -1)
                                                               (declare
                                                                (ignore var-0
                                                                 var-1 var-2
                                                                 var-5 var-6))
                                                               (when var-3
                                                                 (setf m var-3))
                                                               (when var-4
                                                                 (setf n var-4))
                                                               ret-val))))
                 (setf wrkbl
                         (max (the f2cl-lib:integer4 wrkbl)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add m
                                                     (f2cl-lib:int-mul m
```

```
                                                                    (multiple-value-bind
                                                                        (ret-val
                                                                         var-0
                                                                         var-1
                                                                         var-2
                                                                         var-3
                                                                         var-4
                                                                         var-5
                                                                         var-6)
                                                                        (ilaenv
                                                                         1
                                                                         "CUNGLQ"
                                                                         " "
                                                                         m n
                                                                         m
                                                                         -1)
                                                                      (declare
                                                                       (ignore
                                                                        var-0
                                                                        var-1
                                                                        var-2
                                                                        var-6))
                                                                      (when
                                                                          var-3
                                                                        (setf m
                                                                                var-3))
                                                                      (when
                                                                          var-4
                                                                        (setf n
                                                                                var-4))
                                                                      (when
                                                                          var-5
                                                                        (setf m
                                                                                var-5))
                                                                      ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                    (f2cl-lib:int-mul 2 m
                                                                    (multiple-value-bind
                                                                        (ret-val
                                                                         var-0
                                                                         var-1
                                                                         var-2
                                                                         var-3
                                                                         var-4
                                                                         var-5
                                                                         var-6)
                                                                        (ilaenv
```

```
                                                                    1
                                                                    "CGEBRD"
                                                                    " "
                                                                    m m
                                                                    -1
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-5
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf m
                                                                            var-4))
                                                                  ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "CUNMBR"
                                                           "PRC"
                                                           m m
                                                           m
                                                           -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-6))
                                                          (when
```

```
                                                                    var-3
                                                                  (setf m
                                                                          var-3))
                                                                (when
                                                                    var-4
                                                                  (setf m
                                                                          var-4))
                                                                (when
                                                                    var-5
                                                                  (setf m
                                                                          var-5))
                                                                ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                    (f2cl-lib:int-mul m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "CUNMBR"
                                                                           "QLN"
                                                                           m m
                                                                           m
                                                                           -1)
                                                                        (declare
                                                                         (ignore
                                                                          var-0
                                                                          var-1
                                                                          var-2
                                                                          var-6))
                                                                        (when
                                                                            var-3
                                                                          (setf m
                                                                                  var-3))
                                                                        (when
                                                                            var-4
                                                                          (setf m
                                                                                  var-4))
                                                                        (when
                                                                            var-5
                                                                          (setf m
```

```
                                                                                var-5))
                                                                        ret-val))))))
 (setf maxwrk
         (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                           (f2cl-lib:int-mul m m) wrkbl))
 (setf minwrk
         (f2cl-lib:int-add (f2cl-lib:int-mul 2 m m)
                           (f2cl-lib:int-mul 3 m))))
(wntqs
 (setf wrkbl
         (f2cl-lib:int-add m
                           (f2cl-lib:int-mul m
                                             (multiple-value-bind
                                                 (ret-val var-0
                                                  var-1 var-2
                                                  var-3 var-4
                                                  var-5 var-6)
                                                 (ilaenv 1
                                                  "CGELQF" " " m
                                                  n -1 -1)
                                               (declare
                                                (ignore var-0
                                                 var-1 var-2
                                                 var-5 var-6))
                                               (when var-3
                                                 (setf m var-3))
                                               (when var-4
                                                 (setf n var-4))
                                               ret-val))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add m
                                     (f2cl-lib:int-mul m
                                                       (multiple-value-bind
                                                           (ret-val
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-3
                                                            var-4
                                                            var-5
                                                            var-6)
                                                           (ilaenv
                                                            1
                                                            "CUNGLQ"
                                                            " "
                                                            m n
                                                            m
                                                            -1)
```

```
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf n
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf m
                                                                            var-5))
                                                                  ret-val))))))
                  (setf wrkbl
                          (max (the f2cl-lib:integer4 wrkbl)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                      (f2cl-lib:int-mul 2 m
                                                                        (multiple-value-bind
                                                                            (ret-val
                                                                             var-0
                                                                             var-1
                                                                             var-2
                                                                             var-3
                                                                             var-4
                                                                             var-5
                                                                             var-6)
                                                                            (ilaenv
                                                                             1
                                                                             "CGEBRD"
                                                                             " "
                                                                             m m
                                                                             -1
                                                                             -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-5
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
```

```
                                                                                var-3))
                                                                          (when
                                                                              var-4
                                                                            (setf m
                                                                                     var-4))
                                                                          ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                    (f2cl-lib:int-mul m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "CUNMBR"
                                                                           "PRC"
                                                                           m m
                                                                           m
                                                                           -1)
                                                                          (declare
                                                                           (ignore
                                                                            var-0
                                                                            var-1
                                                                            var-2
                                                                            var-6))
                                                                          (when
                                                                              var-3
                                                                            (setf m
                                                                                     var-3))
                                                                          (when
                                                                              var-4
                                                                            (setf m
                                                                                     var-4))
                                                                          (when
                                                                              var-5
                                                                            (setf m
                                                                                     var-5))
                                                                          ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
```

```
                                                    (f2cl-lib:int-mul m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "CUNMBR"
                                                                           "QLN"
                                                                           m m
                                                                           m
                                                                           -1)
                                                                        (declare
                                                                         (ignore
                                                                          var-0
                                                                          var-1
                                                                          var-2
                                                                          var-6))
                                                                        (when
                                                                            var-3
                                                                          (setf m
                                                                                  var-3))
                                                                        (when
                                                                            var-4
                                                                          (setf m
                                                                                  var-4))
                                                                        (when
                                                                            var-5
                                                                          (setf m
                                                                                  var-5))
                                                                        ret-val))))))
                 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
                 (setf minwrk
                         (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                           (f2cl-lib:int-mul 3 m))))
                (wntqa
                 (setf wrkbl
                         (f2cl-lib:int-add m
                                           (f2cl-lib:int-mul m
                                                             (multiple-value-bind
                                                                 (ret-val var-0
                                                                  var-1 var-2
                                                                  var-3 var-4
                                                                  var-5 var-6)
                                                                 (ilaenv 1
```

```
                                                    "CGELQF" " " m
                                                    n -1 -1)
                                                 (declare
                                                  (ignore var-0
                                                   var-1 var-2
                                                   var-5 var-6))
                                                 (when var-3
                                                   (setf m var-3))
                                                 (when var-4
                                                   (setf n var-4))
                                                 ret-val))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add m
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "CUNGLQ"
                                                           " "
                                                           n n
                                                           m
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (when
                                                            var-3
                                                          (setf n
                                                                  var-3))
                                                        (when
                                                            var-4
                                                          (setf n
                                                                  var-4))
                                                        (when
                                                            var-5
                                                          (setf m
                                                                  var-5))
```

```
                                                                ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                    (f2cl-lib:int-mul 2 m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "CGEBRD"
                                                                           " "
                                                                           m m
                                                                           -1
                                                                           -1)
                                                                        (declare
                                                                         (ignore
                                                                          var-0
                                                                          var-1
                                                                          var-2
                                                                          var-5
                                                                          var-6))
                                                                        (when
                                                                            var-3
                                                                          (setf m
                                                                                  var-3))
                                                                        (when
                                                                            var-4
                                                                          (setf m
                                                                                  var-4))
                                                                        ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                    (f2cl-lib:int-mul m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
```

```
                                                                    var-5
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "CUNMBR"
                                                                    "PRC"
                                                                    m m
                                                                    m
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf m
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf m
                                                                            var-5))
                                                                  ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "CUNMBR"
                                                           "QLN"
                                                           m m
                                                           m
                                                           -1)
                                                          (declare
```

```
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-6))
                                                                 (when
                                                                     var-3
                                                                   (setf m
                                                                           var-3))
                                                                 (when
                                                                     var-4
                                                                   (setf m
                                                                           var-4))
                                                                 (when
                                                                     var-5
                                                                   (setf m
                                                                           var-5))
                                                                 ret-val))))))
    (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
    (setf minwrk
            (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                              (f2cl-lib:int-mul 2 m) n)))))
((>= n mnthr2)
 (setf maxwrk
         (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-add m n)
                            (multiple-value-bind
                                (ret-val var-0 var-1 var-2 var-3
                                 var-4 var-5 var-6)
                                (ilaenv 1 "CGEBRD" " " m n -1 -1)
                              (declare
                               (ignore var-0 var-1 var-2 var-5
                                var-6))
                              (when var-3 (setf m var-3))
                              (when var-4 (setf n var-4))
                              ret-val))))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n))
 (cond
  (wntqo
   (setf maxwrk
           (max (the f2cl-lib:integer4 maxwrk)
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                       (f2cl-lib:int-mul m
                                                         (multiple-value-bind
                                                             (ret-val
                                                              var-0
                                                              var-1
                                                              var-2
```

```
                                                                var-3
                                                                var-4
                                                                var-5
                                                                var-6)
                                                               (ilaenv
                                                                1
                                                                "CUNGBR"
                                                                "P"
                                                                m n
                                                                m
                                                                -1)
                                                              (declare
                                                               (ignore
                                                                var-0
                                                                var-1
                                                                var-2
                                                                var-6))
                                                              (when
                                                                  var-3
                                                                (setf m
                                                                        var-3))
                                                              (when
                                                                  var-4
                                                                (setf n
                                                                        var-4))
                                                              (when
                                                                  var-5
                                                                (setf m
                                                                        var-5))
                                                              ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "CUNGBR"
                                                           "Q"
                                                           m m
                                                           n
```

```
                                                                  -1)
                                                                (declare
                                                                 (ignore
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-6))
                                                                (when
                                                                    var-3
                                                                  (setf m
                                                                          var-3))
                                                                (when
                                                                    var-4
                                                                  (setf m
                                                                          var-4))
                                                                (when
                                                                    var-5
                                                                  (setf n
                                                                          var-5))
                                                                ret-val))))))
                 (setf maxwrk (f2cl-lib:int-add maxwrk (f2cl-lib:int-mul m n)))
                 (setf minwrk (f2cl-lib:int-add minwrk (f2cl-lib:int-mul m m))))
                (wntqs
                 (setf maxwrk
                         (max (the f2cl-lib:integer4 maxwrk)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                     (f2cl-lib:int-mul m
                                                                       (multiple-value-bind
                                                                           (ret-val
                                                                            var-0
                                                                            var-1
                                                                            var-2
                                                                            var-3
                                                                            var-4
                                                                            var-5
                                                                            var-6)
                                                                           (ilaenv
                                                                            1
                                                                            "CUNGBR"
                                                                            "P"
                                                                            m n
                                                                            m
                                                                            -1)
                                                                         (declare
                                                                          (ignore
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-6))
```

```
                                                                    (when
                                                                        var-3
                                                                      (setf m
                                                                              var-3))
                                                                    (when
                                                                        var-4
                                                                      (setf n
                                                                              var-4))
                                                                    (when
                                                                        var-5
                                                                      (setf m
                                                                              var-5))
                                                                    ret-val))))))
                (setf maxwrk
                        (max (the f2cl-lib:integer4 maxwrk)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                    (f2cl-lib:int-mul m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "CUNGBR"
                                                                           "Q"
                                                                           m m
                                                                           n
                                                                           -1)
                                                                    (declare
                                                                     (ignore
                                                                      var-0
                                                                      var-1
                                                                      var-2
                                                                      var-6))
                                                                    (when
                                                                        var-3
                                                                      (setf m
                                                                              var-3))
                                                                    (when
                                                                        var-4
                                                                      (setf m
                                                                              var-4))
                                                                    (when
                                                                        var-5
```

```
                                                                  (setf n
                                                                          var-5))
                                                                ret-val)))))))
                (wntqa
                 (setf maxwrk
                         (max (the f2cl-lib:integer4 maxwrk)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                     (f2cl-lib:int-mul n
                                                                       (multiple-value-bind
                                                                           (ret-val
                                                                            var-0
                                                                            var-1
                                                                            var-2
                                                                            var-3
                                                                            var-4
                                                                            var-5
                                                                            var-6)
                                                                           (ilaenv
                                                                            1
                                                                            "CUNGBR"
                                                                            "P"
                                                                            n n
                                                                            m
                                                                            -1)
                                                                         (declare
                                                                          (ignore
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-6))
                                                                         (when
                                                                             var-3
                                                                           (setf n
                                                                                   var-3))
                                                                         (when
                                                                             var-4
                                                                           (setf n
                                                                                   var-4))
                                                                         (when
                                                                             var-5
                                                                           (setf m
                                                                                   var-5))
                                                                         ret-val))))))
                 (setf maxwrk
                         (max (the f2cl-lib:integer4 maxwrk)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                     (f2cl-lib:int-mul m
                                                                       (multiple-value-bind
```

```
                                                                    (ret-val
                                                                     var-0
                                                                     var-1
                                                                     var-2
                                                                     var-3
                                                                     var-4
                                                                     var-5
                                                                     var-6)
                                                                    (ilaenv
                                                                     1
                                                                     "CUNGBR"
                                                                     "Q"
                                                                     m m
                                                                     n
                                                                     -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf m
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf n
                                                                            var-5))
                                                                  ret-val)))))))))
                                                          (t
                                                           (setf maxwrk
                                                                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                                                     (f2cl-lib:int-mul
                                                                                      (f2cl-lib:int-add m n)
                                                                                      (multiple-value-bind
                                                                                          (ret-val var-0 var-1 var-2 var-3
                                                                                           var-4 var-5 var-6)
                                                                                          (ilaenv 1 "CGEBRD" " " m n -1 -1)
                                                                                        (declare
                                                                                         (ignore var-0 var-1 var-2 var-5
                                                                                          var-6))
                                                                                        (when var-3 (setf m var-3))
                                                                                        (when var-4 (setf n var-4))
                                                                                        ret-val))))
                                                           (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n))
```

```
(cond
 (wntqo
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                      (f2cl-lib:int-mul m
                                                        (multiple-value-bind
                                                            (ret-val
                                                             var-0
                                                             var-1
                                                             var-2
                                                             var-3
                                                             var-4
                                                             var-5
                                                             var-6)
                                                            (ilaenv
                                                             1
                                                             "CUNMBR"
                                                             "PRC"
                                                             m n
                                                             m
                                                             -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-6))
                                                          (when
                                                              var-3
                                                            (setf m
                                                                    var-3))
                                                          (when
                                                              var-4
                                                            (setf n
                                                                    var-4))
                                                          (when
                                                              var-5
                                                            (setf m
                                                                    var-5))
                                                          ret-val))))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                      (f2cl-lib:int-mul m
                                                        (multiple-value-bind
                                                            (ret-val
                                                             var-0
```

```
                                                                var-1
                                                                var-2
                                                                var-3
                                                                var-4
                                                                var-5
                                                                var-6)
                                                               (ilaenv
                                                                1
                                                                "CUNMBR"
                                                                "QLN"
                                                                m m
                                                                n
                                                                -1)
                                                             (declare
                                                              (ignore
                                                               var-0
                                                               var-1
                                                               var-2
                                                               var-6))
                                                             (when
                                                                 var-3
                                                               (setf m
                                                                       var-3))
                                                             (when
                                                                 var-4
                                                               (setf m
                                                                       var-4))
                                                             (when
                                                                 var-5
                                                               (setf n
                                                                       var-5))
                                                             ret-val))))))
                (setf maxwrk (f2cl-lib:int-add maxwrk (f2cl-lib:int-mul m n)))
                (setf minwrk (f2cl-lib:int-add minwrk (f2cl-lib:int-mul m m))))
               (wntqs
                (setf maxwrk
                        (max (the f2cl-lib:integer4 maxwrk)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                    (f2cl-lib:int-mul m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
```

```
                                                                    1
                                                                    "CUNGBR"
                                                                    "PRC"
                                                                    m n
                                                                    m
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf n
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf m
                                                                            var-5))
                                                                  ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "CUNGBR"
                                                           "QLN"
                                                           m m
                                                           n
                                                           -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
```

```
                                                                    var-2
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf m
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf n
                                                                            var-5))
                                                                  ret-val)))))))
                (wntqa
                 (setf maxwrk
                         (max (the f2cl-lib:integer4 maxwrk)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                     (f2cl-lib:int-mul n
                                                                       (multiple-value-bind
                                                                           (ret-val
                                                                            var-0
                                                                            var-1
                                                                            var-2
                                                                            var-3
                                                                            var-4
                                                                            var-5
                                                                            var-6)
                                                                           (ilaenv
                                                                            1
                                                                            "CUNGBR"
                                                                            "PRC"
                                                                            n n
                                                                            m
                                                                            -1)
                                                                         (declare
                                                                          (ignore
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-6))
                                                                         (when
                                                                             var-3
                                                                           (setf n
                                                                                   var-3))
                                                                         (when
                                                                             var-4
                                                                           (setf n
```

```
                                                                    var-4))
                                                              (when
                                                                  var-5
                                                                (setf m
                                                                    var-5))
                                                              ret-val))))))
                (setf maxwrk
                        (max (the f2cl-lib:integer4 maxwrk)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                    (f2cl-lib:int-mul m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "CUNGBR"
                                                                           "QLN"
                                                                           m m
                                                                           n
                                                                           -1)
                                                                          (declare
                                                                           (ignore
                                                                            var-0
                                                                            var-1
                                                                            var-2
                                                                            var-6))
                                                                          (when
                                                                              var-3
                                                                            (setf m
                                                                                var-3))
                                                                          (when
                                                                              var-4
                                                                            (setf m
                                                                                var-4))
                                                                          (when
                                                                              var-5
                                                                            (setf n
                                                                                var-5))
                                                                          ret-val))))))))))))
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4 minwrk)))))
        (cond
```

```
 ((= info 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce maxwrk 'f2cl-lib:complex8))
  (if (and (< lwork minwrk) (/= lwork lquerv))
      (setf info -13))))
(cond
 ((/= info 0) (xerbla "CGESDD" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= lwork lquerv)
    (go end_label))
(cond ((or (= m 0) (= n 0)) (go end_label)))
(setf eps (slamch "P"))
(setf smlnum (/ (f2cl-lib:fsqrt (slamch "S")) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "M" m n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iscl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf iscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 anrm smlnum m n a lda ierr)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf ierr var-9))))
 ((> anrm bignum) (setf iscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 anrm bignum m n a lda ierr)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf ierr var-9)))))
(cond
 ((>= m n)
  (cond
```

```
((>= m mnthr1)
 (cond
  (wntqn (setf itau 1) (setf nwork (f2cl-lib:int-add itau n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (cgeqrf m n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-7))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (claset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
        czero czero
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (2 1)
                              ((1 lda) (1 *)) a-%offset%)
        lda)
     (declare (ignore var-0 var-1 var-2 var-5))
     (when var-3 (setf czero var-3))
     (when var-4 (setf czero var-4))
     (when var-6 (setf lda var-6)))
   (setf ie 1) (setf itauq 1)
   (setf itaup (f2cl-lib:int-add itauq n))
   (setf nwork (f2cl-lib:int-add itaup n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (cgebrd n n a lda s
        (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                              rwork-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
     (setf n var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-10))
   (setf nrwork (f2cl-lib:int-add ie n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
      var-10 var-11 var-12 var-13)
     (sbdsdc "U" "N" n s
      (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                            rwork-%offset%)
      dum 1 dum 1 dum idum
      (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                            ((1 *)) rwork-%offset%)
      iwork info)
    (declare
     (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12))
    (when var-2 (setf n var-2))
    (when var-13 (setf info var-13))))
  (wntqo (setf iu 1) (setf ldwrku n)
   (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                           (f2cl-lib:int-mul n n)
                           (f2cl-lib:int-mul 3 n)))
     (setf ldwrkr m))
    (t
     (setf ldwrkr
             (the f2cl-lib:integer4
                  (truncate (- lwork (* n n) (* 3 n)) n)))))
   (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
   (setf nwork (f2cl-lib:int-add itau n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (cgeqrf m n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-7))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (clacpy "U" n n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                              ((1 *)) work-%offset%)
        ldwrkr)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf n var-1))
     (when var-2 (setf n var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf ldwrkr var-6)))
```

```
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           ((+ ir 1)) ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cungqr m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf nwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(setf iru (f2cl-lib:int-add ie n))
(setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul n n)))
```

```
(setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul n n)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sbdsdc "U" "I" n s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     n
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     n dum idum
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf n var-2))
  (when var-6 (setf n var-6))
  (when var-8 (setf n var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" n n
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "Q" "L" "N" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
```

```
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldwrkr var-7))
  (when var-10 (setf ldwrku var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" n n
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     n vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "P" "R" "C" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldwrkr var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i ldwrkr))
              ((> i m) nil)
  (tagbody
    (setf chunk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
             (the f2cl-lib:integer4 ldwrkr)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
                var-9 var-10 var-11 var-12)
               (cgemm "N" "N" chunk n n cone
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i 1)
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                                      ((1 *)) work-%offset%)
                ldwrku czero
                (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                                      ((1 *)) work-%offset%)
                ldwrkr)
            (declare (ignore var-0 var-1 var-6 var-8 var-11))
            (when var-2 (setf chunk var-2))
            (when var-3 (setf n var-3))
            (when var-4 (setf n var-4))
            (when var-5 (setf cone var-5))
            (when var-7 (setf lda var-7))
            (when var-9 (setf ldwrku var-9))
            (when var-10 (setf czero var-10))
            (when var-12 (setf ldwrkr var-12)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (clacpy "F" chunk n
                (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                                      ((1 *)) work-%offset%)
                ldwrkr
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i 1)
                                      ((1 lda) (1 *)) a-%offset%)
                lda)
            (declare (ignore var-0 var-3 var-5))
            (when var-1 (setf chunk var-1))
            (when var-2 (setf n var-2))
            (when var-4 (setf ldwrkr var-4))
            (when var-6 (setf lda var-6)))
        label10)))
    (wntqs (setf ir 1) (setf ldwrkr n)
     (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
     (setf nwork (f2cl-lib:int-add itau n))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (cgeqrf m n a lda
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
       (declare (ignore var-2 var-4 var-5 var-6))
       (setf m var-0)
       (setf n var-1)
       (setf lda var-3)
```

```
  (setf ierr var-7))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           ((+ ir 1)) ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cungqr m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf nwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
```

```
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                          ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(setf iru (f2cl-lib:int-add ie n))
(setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul n n)))
(setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul n n)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sbdsdc "U" "I" n s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     n
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     n dum idum
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf n var-2))
  (when var-6 (setf n var-6))
  (when var-8 (setf n var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" n n
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     n u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "Q" "L" "N" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
```

```
      ldwrkr
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                            ((1 *)) work-%offset%)
      u ldu
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldwrkr var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" n n
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     n vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "P" "R" "C" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldwrkr var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "F" n n u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
```

```
      ldwrkr)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldu var-4))
    (when var-6 (setf ldwrkr var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (cgemm "N" "N" m n n cone a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                             ((1 *)) work-%offset%)
       ldwrkr czero u ldu)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldwrkr var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf ldu var-12))))
 (wntqa (setf iu 1) (setf ldwrku n)
  (setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
  (setf nwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (cgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (cungqr m m n u ldu
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                             ((1 *)) work-%offset%)
```

```
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (2 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf nwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgebrd n n a lda s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(setf iru (f2cl-lib:int-add ie n))
(setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul n n)))
(setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul n n)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sbdsdc "U" "I" n s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
```

```
                              ((1 *)) rwork-%offset%)
      n
      (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                              ((1 *)) rwork-%offset%)
      n dum idum
      (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                              ((1 *)) rwork-%offset%)
      iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf n var-2))
  (when var-6 (setf n var-6))
  (when var-8 (setf n var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" n n
      (f2cl-lib:array-slice rwork-%data% single-float (iru)
                              ((1 *)) rwork-%offset%)
      n
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                              ((1 *)) work-%offset%)
      ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "Q" "L" "N" n n n a lda
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                              ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                              ((1 *)) work-%offset%)
      ldwrku
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                              ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldwrku var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" n n
```

```
          (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                                ((1 *)) rwork-%offset%)
          n vt ldvt)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf n var-4))
        (when var-6 (setf ldvt var-6)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13)
          (cunmbr "P" "R" "C" n n n a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                                ((1 *)) work-%offset%)
           vt ldvt
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                                ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
        (declare
         (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
        (when var-3 (setf n var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf n var-5))
        (when var-7 (setf lda var-7))
        (when var-10 (setf ldvt var-10))
        (when var-13 (setf ierr var-13)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (cgemm "N" "N" m n n cone u ldu
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                                ((1 *)) work-%offset%)
           ldwrku czero a lda)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf n var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf cone var-5))
        (when var-7 (setf ldu var-7))
        (when var-9 (setf ldwrku var-9))
        (when var-10 (setf czero var-10))
        (when var-12 (setf lda var-12)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (clacpy "F" m n a lda u ldu)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldu var-6))))))
    ((>= m mnthr2) (setf ie 1) (setf nrwork (f2cl-lib:int-add ie n))
```

```
(setf itauq 1) (setf itaup (f2cl-lib:int-add itauq n))
(setf nwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgebrd m n a lda s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(cond
 (wntqn
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (sbdsdc "U" "N" n s
       (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                             rwork-%offset%)
       dum 1 dum 1 dum idum
       (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                             ((1 *)) rwork-%offset%)
       iwork info)
    (declare
     (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12))
    (when var-2 (setf n var-2))
    (when var-13 (setf info var-13))))
 (wntqo (setf iu nwork) (setf iru nrwork)
  (setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul n n)))
  (setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul n n)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "U" n n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (cungbr "P" n n n vt ldvt
```

```
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (cungbr "Q" m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                            ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                            ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf ierr var-9)))
(cond
 ((>= lwork
      (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                        (f2cl-lib:int-mul 3 n)))
  (setf ldwrku m))
 (t
  (setf ldwrku
          (the f2cl-lib:integer4
               (truncate (- lwork (* 3 n)) n)))))
(setf nwork (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sbdsdc "U" "I" n s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     n
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     n dum idum
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
```

```
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf n var-2))
  (when var-6 (setf n var-6))
  (when var-8 (setf n var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (clarcm n n
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     n vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                           ((1 *)) rwork-%offset%))
  (declare (ignore var-2 var-4 var-6 var-8))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-7 (setf ldwrku var-7)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "F" n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldvt var-6)))
(setf nrwork irvt)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i ldwrku))
              ((> i m) nil)
  (tagbody
    (setf chunk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
             (the f2cl-lib:integer4 ldwrku)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clacrm chunk n
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice rwork-%data% single-float (iru)
```

```
                                ((1 *)) rwork-%offset%)
            n
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                                ((1 *)) work-%offset%)
            ldwrku
            (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                                ((1 *)) rwork-%offset%))
        (declare (ignore var-2 var-4 var-6 var-8))
        (when var-0 (setf chunk var-0))
        (when var-1 (setf n var-1))
        (when var-3 (setf lda var-3))
        (when var-5 (setf n var-5))
        (when var-7 (setf ldwrku var-7)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (clacpy "F" chunk n
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                                ((1 *)) work-%offset%)
           ldwrku
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i 1)
                                ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf chunk var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf ldwrku var-4))
        (when var-6 (setf lda var-6)))
    label20)))
(wntqs
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (clacpy "U" n n a lda vt ldvt)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldvt var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
     (cungbr "P" n n n vt ldvt
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
   (declare (ignore var-0 var-4 var-6 var-7 var-8))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-3 (setf n var-3))
   (when var-5 (setf ldvt var-5))
   (when var-9 (setf ierr var-9)))
```

```
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (cungbr "Q" m n n u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf iru nrwork)
(setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul n n)))
(setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul n n)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sbdsdc "U" "I" n s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     n
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     n dum idum
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf n var-2))
  (when var-6 (setf n var-6))
  (when var-8 (setf n var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (clarcm n n
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
```

```
                              ((1 *)) rwork-%offset%)
        n vt ldvt a lda
        (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                              ((1 *)) rwork-%offset%))
      (declare (ignore var-2 var-4 var-6 var-8))
      (when var-0 (setf n var-0))
      (when var-1 (setf n var-1))
      (when var-3 (setf n var-3))
      (when var-5 (setf ldvt var-5))
      (when var-7 (setf lda var-7)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (clacpy "F" n n a lda vt ldvt)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-6 (setf ldvt var-6)))
    (setf nrwork irvt)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clacrm m n u ldu
        (f2cl-lib:array-slice rwork-%data% single-float (iru)
                              ((1 *)) rwork-%offset%)
        n a lda
        (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                              ((1 *)) rwork-%offset%))
      (declare (ignore var-2 var-4 var-6 var-8))
      (when var-0 (setf m var-0))
      (when var-1 (setf n var-1))
      (when var-3 (setf ldu var-3))
      (when var-5 (setf n var-5))
      (when var-7 (setf lda var-7)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (clacpy "F" m n a lda u ldu)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-6 (setf ldu var-6))))
  (t
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (clacpy "U" n n a lda vt ldvt)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-6 (setf ldvt var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (cungbr "P" n n n vt ldvt
```

```
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (cungbr "Q" m m n u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf iru nrwork)
(setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul n n)))
(setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul n n)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sbdsdc "U" "I" n s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     n
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     n dum idum
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
```

```
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf n var-2))
  (when var-6 (setf n var-6))
  (when var-8 (setf n var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (clarcm n n
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     n vt ldvt a lda
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                           ((1 *)) rwork-%offset%))
  (declare (ignore var-2 var-4 var-6 var-8))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-7 (setf lda var-7)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "F" n n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(setf nrwork irvt)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (clacrm m n u ldu
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     n a lda
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                           ((1 *)) rwork-%offset%))
  (declare (ignore var-2 var-4 var-6 var-8))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf ldu var-3))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "F" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6))))))
```

```
(t (setf ie 1) (setf nrwork (f2cl-lib:int-add ie n)) (setf itauq 1)
 (setf itaup (f2cl-lib:int-add itauq n))
 (setf nwork (f2cl-lib:int-add itaup n))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (cgebrd m n a lda s
      (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                            rwork-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-10))
 (cond
  (wntqn
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13)
       (sbdsdc "U" "N" n s
        (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                              rwork-%offset%)
        dum 1 dum 1 dum idum
        (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                              ((1 *)) rwork-%offset%)
        iwork info)
     (declare
      (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12))
     (when var-2 (setf n var-2))
     (when var-13 (setf info var-13))))
  (wntqo (setf iu nwork) (setf iru nrwork)
   (setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul n n)))
   (setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul n n)))
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                           (f2cl-lib:int-mul 3 n)))
     (setf ldwrku m))
    (t
     (setf ldwrku
             (the f2cl-lib:integer4
                  (truncate (- lwork (* 3 n)) n)))))
```

```
(setf nwork (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sbdsdc "U" "I" n s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     n
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     n dum idum
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf n var-2))
  (when var-6 (setf n var-6))
  (when var-8 (setf n var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" n n
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     n vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "P" "R" "C" n n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldvt var-10))
```

```
  (when var-13 (setf ierr var-13)))
(cond
 ((>= lwork
      (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                        (f2cl-lib:int-mul 3 n)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "F" m n czero czero
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                             ((1 *)) work-%offset%)
       ldwrku)
    (declare (ignore var-0 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldwrku var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacp2 "F" n n
       (f2cl-lib:array-slice rwork-%data% single-float (iru)
                             ((1 *)) rwork-%offset%)
       n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                             ((1 *)) work-%offset%)
       ldwrku)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf n var-4))
    (when var-6 (setf ldwrku var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13)
      (cunmbr "Q" "L" "N" m n n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (itauq) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                             ((1 *)) work-%offset%)
       ldwrku
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (nwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf n var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldwrku var-10))
```

```
    (when var-13 (setf ierr var-13)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "F" m n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                             ((1 *)) work-%offset%)
       ldwrku a lda)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldwrku var-4))
    (when var-6 (setf lda var-6))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (cungbr "Q" m n n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (itauq) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (nwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))
  (setf nrwork irvt)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i ldwrku))
                ((> i m) nil)
    (tagbody
      (setf chunk
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
               (the f2cl-lib:integer4 ldwrku)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (clacrm chunk n
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice rwork-%data% single-float (iru)
                                 ((1 *)) rwork-%offset%)
           n
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                 (iu) ((1 *)) work-%offset%)
           ldwrku
           (f2cl-lib:array-slice rwork-%data% single-float
```

```
                                (nrwork) ((1 *))
                                rwork-%offset%))
            (declare (ignore var-2 var-4 var-6 var-8))
            (when var-0 (setf chunk var-0))
            (when var-1 (setf n var-1))
            (when var-3 (setf lda var-3))
            (when var-5 (setf n var-5))
            (when var-7 (setf ldwrku var-7)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (clacpy "F" chunk n
               (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iu) ((1 *)) work-%offset%)
               ldwrku
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i 1)
                                ((1 lda) (1 *)) a-%offset%)
               lda)
            (declare (ignore var-0 var-3 var-5))
            (when var-1 (setf chunk var-1))
            (when var-2 (setf n var-2))
            (when var-4 (setf ldwrku var-4))
            (when var-6 (setf lda var-6)))
        label30)))))
  (wntqs (setf iru nrwork)
   (setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul n n)))
   (setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul n n)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13)
       (sbdsdc "U" "I" n s
        (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                              rwork-%offset%)
        (f2cl-lib:array-slice rwork-%data% single-float (iru)
                              ((1 *)) rwork-%offset%)
        n
        (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                              ((1 *)) rwork-%offset%)
        n dum idum
        (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                              ((1 *)) rwork-%offset%)
        iwork info)
     (declare
      (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
       var-11 var-12))
     (when var-2 (setf n var-2))
     (when var-6 (setf n var-6))
     (when var-8 (setf n var-8))
     (when var-13 (setf info var-13)))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (claset "F" m n czero czero u ldu)
```

```
  (declare (ignore var-0 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" n n
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     n u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "Q" "L" "N" m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" n n
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     n vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "P" "R" "C" n n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
```

```
        vt ldvt
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
      (when var-3 (setf n var-3))
      (when var-4 (setf n var-4))
      (when var-5 (setf n var-5))
      (when var-7 (setf lda var-7))
      (when var-10 (setf ldvt var-10))
      (when var-13 (setf ierr var-13))))
    (t (setf iru nrwork)
     (setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul n n)))
     (setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul n n)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12 var-13)
         (sbdsdc "U" "I" n s
          (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                                rwork-%offset%)
          (f2cl-lib:array-slice rwork-%data% single-float (iru)
                                ((1 *)) rwork-%offset%)
          n
          (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                                ((1 *)) rwork-%offset%)
          n dum idum
          (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                                ((1 *)) rwork-%offset%)
          iwork info)
      (declare
       (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
        var-11 var-12))
      (when var-2 (setf n var-2))
      (when var-6 (setf n var-6))
      (when var-8 (setf n var-8))
      (when var-13 (setf info var-13)))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (claset "F" m m czero czero u ldu)
      (declare (ignore var-0 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf m var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldu var-6)))
     (cond
      ((> m n)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
           (claset "F" (f2cl-lib:int-sub m n) (f2cl-lib:int-sub m n)
```

```
        czero cone
        (f2cl-lib:array-slice u-%data% f2cl-lib:complex8
                              ((+ n 1) (f2cl-lib:int-add n 1))
                              ((1 ldu) (1 *)) u-%offset%)
        ldu)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf czero var-3))
    (when var-4 (setf cone var-4))
    (when var-6 (setf ldu var-6)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" n n
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     n u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "Q" "L" "N" m m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" n n
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     n vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
```

```
            (cunmbr "P" "R" "C" n n n a lda
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                                   ((1 *)) work-%offset%)
             vt ldvt
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
          (declare
           (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
          (when var-3 (setf n var-3))
          (when var-4 (setf n var-4))
          (when var-5 (setf n var-5))
          (when var-7 (setf lda var-7))
          (when var-10 (setf ldvt var-10))
          (when var-13 (setf ierr var-13))))))))
(t
 (cond
  ((>= n mnthr1)
   (cond
    (wntqn (setf itau 1) (setf nwork (f2cl-lib:int-add itau m))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (cgelqf m n a lda
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
       (declare (ignore var-2 var-4 var-5 var-6))
       (setf m var-0)
       (setf n var-1)
       (setf lda var-3)
       (setf ierr var-7))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
          czero czero
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 2)
                                ((1 lda) (1 *)) a-%offset%)
          lda)
       (declare (ignore var-0 var-1 var-2 var-5))
       (when var-3 (setf czero var-3))
       (when var-4 (setf czero var-4))
       (when var-6 (setf lda var-6)))
     (setf ie 1) (setf itauq 1)
     (setf itaup (f2cl-lib:int-add itauq m))
     (setf nwork (f2cl-lib:int-add itaup m))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (cgebrd m m a lda s
```

```
      (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                            rwork-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
    (setf m var-0)
    (setf m var-1)
    (setf lda var-3)
    (setf ierr var-10))
  (setf nrwork (f2cl-lib:int-add ie m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (sbdsdc "U" "N" m s
       (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                             rwork-%offset%)
       dum 1 dum 1 dum idum
       (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                             ((1 *)) rwork-%offset%)
       iwork info)
    (declare
     (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12))
    (when var-2 (setf m var-2))
    (when var-13 (setf info var-13))))
 (wntqo (setf ivt 1) (setf ldwkvt m)
  (setf il (f2cl-lib:int-add ivt (f2cl-lib:int-mul ldwkvt m)))
  (cond
   ((>= lwork
        (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                          (f2cl-lib:int-mul m m)
                          (f2cl-lib:int-mul 3 m)))
    (setf ldwrkl m) (setf chunk n))
   (t (setf ldwrkl m)
    (setf chunk
            (the f2cl-lib:integer4
                 (truncate (- lwork (* m m) (* 3 m)) m)))))
  (setf itau (f2cl-lib:int-add il (f2cl-lib:int-mul ldwrkl chunk)))
  (setf nwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (cgelqf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
```

```
                              ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il)
                              ((1 *)) work-%offset%)
     ldwrkl)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkl var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              ((+ il ldwrkl)) ((1 *)) work-%offset%)
     ldwrkl)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrkl var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cunglq m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                              ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                              ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf nwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il)
                              ((1 *)) work-%offset%)
```

```
      ldwrkl s
      (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                            rwork-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkl var-3)
  (setf ierr var-10))
(setf iru (f2cl-lib:int-add ie m))
(setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul m m)))
(setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul m m)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sbdsdc "U" "I" m s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     m
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     m dum idum
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf m var-2))
  (when var-6 (setf m var-6))
  (when var-8 (setf m var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     m u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldu var-6)))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "Q" "L" "N" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il)
                           ((1 *)) work-%offset%)
     ldwrkl
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwrkl var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ivt)
                           ((1 *)) work-%offset%)
     ldwkvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldwkvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "P" "R" "C" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il)
                           ((1 *)) work-%offset%)
     ldwrkl
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ivt)
                           ((1 *)) work-%offset%)
     ldwkvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
```

```
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwrkl var-7))
  (when var-10 (setf ldwkvt var-10))
  (when var-13 (setf ierr var-13)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
              ((> i n) nil)
  (tagbody
    (setf blk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
             (the f2cl-lib:integer4 chunk)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (cgemm "N" "N" m blk m cone
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                               (ivt) ((1 *)) work-%offset%)
         m
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 i)
                               ((1 lda) (1 *)) a-%offset%)
         lda czero
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il)
                               ((1 *)) work-%offset%)
         ldwrkl)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf blk var-3))
      (when var-4 (setf m var-4))
      (when var-5 (setf cone var-5))
      (when var-7 (setf m var-7))
      (when var-9 (setf lda var-9))
      (when var-10 (setf czero var-10))
      (when var-12 (setf ldwrkl var-12)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (clacpy "F" m blk
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il)
                               ((1 *)) work-%offset%)
         ldwrkl
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 i)
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf blk var-2))
      (when var-4 (setf ldwrkl var-4))
```

```
        (when var-6 (setf lda var-6)))
    label40)))
(wntqs (setf il 1) (setf ldwrkl m)
 (setf itau (f2cl-lib:int-add il (f2cl-lib:int-mul ldwrkl m)))
 (setf nwork (f2cl-lib:int-add itau m))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (cgelqf m n a lda
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-7))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (clacpy "L" m m a lda
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il)
                            ((1 *)) work-%offset%)
      ldwrkl)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf m var-1))
   (when var-2 (setf m var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldwrkl var-6)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
      czero czero
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                            ((+ il ldwrkl)) ((1 *)) work-%offset%)
      ldwrkl)
   (declare (ignore var-0 var-1 var-2 var-5))
   (when var-3 (setf czero var-3))
   (when var-4 (setf czero var-4))
   (when var-6 (setf ldwrkl var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (cunglq m n m a lda
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
   (declare (ignore var-3 var-5 var-6 var-7))
   (when var-0 (setf m var-0))
   (when var-1 (setf n var-1))
   (when var-2 (setf m var-2))
```

```
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf nwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il)
                           ((1 *)) work-%offset%)
     ldwrkl s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkl var-3)
  (setf ierr var-10))
(setf iru (f2cl-lib:int-add ie m))
(setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul m m)))
(setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul m m)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sbdsdc "U" "I" m s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     m
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     m dum idum
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf m var-2))
  (when var-6 (setf m var-6))
  (when var-8 (setf m var-8))
```

```
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     m u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "Q" "L" "N" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il)
                           ((1 *)) work-%offset%)
     ldwrkl
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwrkl var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     m vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "P" "R" "C" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il)
                           ((1 *)) work-%offset%)
     ldwrkl
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
```

```
                              ((1 *)) work-%offset%)
        vt ldvt
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
      (when var-3 (setf m var-3))
      (when var-4 (setf m var-4))
      (when var-5 (setf m var-5))
      (when var-7 (setf ldwrkl var-7))
      (when var-10 (setf ldvt var-10))
      (when var-13 (setf ierr var-13)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (clacpy "F" m m vt ldvt
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il)
                              ((1 *)) work-%offset%)
        ldwrkl)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf m var-2))
      (when var-4 (setf ldvt var-4))
      (when var-6 (setf ldwrkl var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (cgemm "N" "N" m n m cone
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (il)
                              ((1 *)) work-%offset%)
        ldwrkl a lda czero vt ldvt)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf n var-3))
      (when var-4 (setf m var-4))
      (when var-5 (setf cone var-5))
      (when var-7 (setf ldwrkl var-7))
      (when var-9 (setf lda var-9))
      (when var-10 (setf czero var-10))
      (when var-12 (setf ldvt var-12))))
  (wntqa (setf ivt 1) (setf ldwkvt m)
   (setf itau (f2cl-lib:int-add ivt (f2cl-lib:int-mul ldwkvt m)))
   (setf nwork (f2cl-lib:int-add itau m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (cgelqf m n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
```

```
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "U" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cunglq n n m vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf nwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgebrd m m a lda s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
```

```
                          ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(setf iru (f2cl-lib:int-add ie m))
(setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul m m)))
(setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul m m)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sbdsdc "U" "I" m s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     m
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     m dum idum
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf m var-2))
  (when var-6 (setf m var-6))
  (when var-8 (setf m var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     m u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "Q" "L" "N" m m m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
```

```
                          ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                          ((1 *)) rwork-%offset%)
     m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ivt)
                          ((1 *)) work-%offset%)
     ldwkvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldwkvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "P" "R" "C" m m m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ivt)
                          ((1 *)) work-%offset%)
     ldwkvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                          ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldwkvt var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (cgemm "N" "N" m n m cone
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ivt)
                          ((1 *)) work-%offset%)
     ldwkvt vt ldvt czero a lda)
```

```
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf n var-3))
      (when var-4 (setf m var-4))
      (when var-5 (setf cone var-5))
      (when var-7 (setf ldwkvt var-7))
      (when var-9 (setf ldvt var-9))
      (when var-10 (setf czero var-10))
      (when var-12 (setf lda var-12)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (clacpy "F" m n a lda vt ldvt)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-6 (setf ldvt var-6))))))
((>= n mnthr2) (setf ie 1) (setf nrwork (f2cl-lib:int-add ie m))
 (setf itauq 1) (setf itaup (f2cl-lib:int-add itauq m))
 (setf nwork (f2cl-lib:int-add itaup m))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (cgebrd m n a lda s
      (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                            rwork-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-10))
 (cond
  (wntqn
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13)
       (sbdsdc "L" "N" m s
        (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                              rwork-%offset%)
        dum 1 dum 1 dum idum
        (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                              ((1 *)) rwork-%offset%)
        iwork info)
     (declare
```

```
      (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12))
    (when var-2 (setf m var-2))
    (when var-13 (setf info var-13))))
  (wntqo (setf irvt nrwork)
   (setf iru (f2cl-lib:int-add irvt (f2cl-lib:int-mul m m)))
   (setf nrwork (f2cl-lib:int-add iru (f2cl-lib:int-mul m m)))
   (setf ivt nwork)
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (clacpy "L" m m a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (cungbr "Q" m m n u ldu
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf ldu var-5))
    (when var-9 (setf ierr var-9)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (cungbr "P" m n m a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))
   (setf ldwkvt m)
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                           (f2cl-lib:int-mul 3 m)))
     (setf nwork (f2cl-lib:int-add ivt (f2cl-lib:int-mul ldwkvt n)))
     (setf chunk n))
```

```
 (t
  (setf chunk
          (the f2cl-lib:integer4 (truncate (- lwork (* 3 m)) m)))
  (setf nwork
          (f2cl-lib:int-add ivt
                            (f2cl-lib:int-mul ldwkvt chunk)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sbdsdc "L" "I" m s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     m
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     m dum idum
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf m var-2))
  (when var-6 (setf m var-6))
  (when var-8 (setf m var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (clacrm m m u ldu
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ivt)
                           ((1 *)) work-%offset%)
     ldwkvt
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                           ((1 *)) rwork-%offset%))
  (declare (ignore var-2 var-4 var-6 var-8))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-3 (setf ldu var-3))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwkvt var-7)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "F" m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ivt)
                           ((1 *)) work-%offset%)
     ldwkvt u ldu)
```

```
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf ldwkvt var-4))
    (when var-6 (setf ldu var-6)))
  (setf nrwork iru)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
                ((> i n) nil)
    (tagbody
      (setf blk
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
               (the f2cl-lib:integer4 chunk)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (clarcm m blk
           (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                                 ((1 *)) rwork-%offset%)
           m
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                 (ivt) ((1 *)) work-%offset%)
           ldwkvt
           (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                                 ((1 *)) rwork-%offset%))
        (declare (ignore var-2 var-4 var-6 var-8))
        (when var-0 (setf m var-0))
        (when var-1 (setf blk var-1))
        (when var-3 (setf m var-3))
        (when var-5 (setf lda var-5))
        (when var-7 (setf ldwkvt var-7)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (clacpy "F" m blk
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                 (ivt) ((1 *)) work-%offset%)
           ldwkvt
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf blk var-2))
        (when var-4 (setf ldwkvt var-4))
        (when var-6 (setf lda var-6)))
     label50)))
 (wntqs
```

```
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" m m a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (cungbr "Q" m m n u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "U" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (cungbr "P" m n m vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf irvt nrwork)
(setf iru (f2cl-lib:int-add irvt (f2cl-lib:int-mul m m)))
(setf nrwork (f2cl-lib:int-add iru (f2cl-lib:int-mul m m)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sbdsdc "L" "I" m s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
```

```
                                rwork-%offset%)
      (f2cl-lib:array-slice rwork-%data% single-float (iru)
                                ((1 *)) rwork-%offset%)
      m
      (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                                ((1 *)) rwork-%offset%)
      m dum idum
      (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                                ((1 *)) rwork-%offset%)
      iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf m var-2))
  (when var-6 (setf m var-6))
  (when var-8 (setf m var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (clacrm m m u ldu
      (f2cl-lib:array-slice rwork-%data% single-float (iru)
                                ((1 *)) rwork-%offset%)
      m a lda
      (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                                ((1 *)) rwork-%offset%))
  (declare (ignore var-2 var-4 var-6 var-8))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-3 (setf ldu var-3))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "F" m m a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(setf nrwork iru)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (clarcm m n
      (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                                ((1 *)) rwork-%offset%)
      m vt ldvt a lda
      (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                                ((1 *)) rwork-%offset%))
  (declare (ignore var-2 var-4 var-6 var-8))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
```

```
    (when var-3 (setf m var-3))
    (when var-5 (setf ldvt var-5))
    (when var-7 (setf lda var-7)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "F" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6))))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "L" m m a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (cungbr "Q" m m n u ldu
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf ldu var-5))
    (when var-9 (setf ierr var-9)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (cungbr "P" n n m vt ldvt
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
```

```
  (when var-3 (setf m var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf irvt nrwork)
(setf iru (f2cl-lib:int-add irvt (f2cl-lib:int-mul m m)))
(setf nrwork (f2cl-lib:int-add iru (f2cl-lib:int-mul m m)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sbdsdc "L" "I" m s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     m
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     m dum idum
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf m var-2))
  (when var-6 (setf m var-6))
  (when var-8 (setf m var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (clacrm m m u ldu
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     m a lda
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                           ((1 *)) rwork-%offset%))
  (declare (ignore var-2 var-4 var-6 var-8))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-3 (setf ldu var-3))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "F" m m a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
```

```
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clarcm m n
         (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                               ((1 *)) rwork-%offset%)
         m vt ldvt a lda
         (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                               ((1 *)) rwork-%offset%))
      (declare (ignore var-2 var-4 var-6 var-8))
      (when var-0 (setf m var-0))
      (when var-1 (setf n var-1))
      (when var-3 (setf m var-3))
      (when var-5 (setf ldvt var-5))
      (when var-7 (setf lda var-7)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (clacpy "F" m n a lda vt ldvt)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-6 (setf ldvt var-6))))))
  (t (setf ie 1) (setf nrwork (f2cl-lib:int-add ie m)) (setf itauq 1)
   (setf itaup (f2cl-lib:int-add itauq m))
   (setf nwork (f2cl-lib:int-add itaup m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (cgebrd m n a lda s
        (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                              rwork-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-10))
   (cond
    (wntqn
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12 var-13)
         (sbdsdc "L" "N" m s
          (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                                rwork-%offset%)
          dum 1 dum 1 dum idum
```

```
        (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                              ((1 *)) rwork-%offset%)
        iwork info)
    (declare
     (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12))
    (when var-2 (setf m var-2))
    (when var-13 (setf info var-13))))
  (wntqo (setf ldwkvt m) (setf ivt nwork)
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                           (f2cl-lib:int-mul 3 m)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (claset "F" m n czero czero
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ivt)
                                ((1 *)) work-%offset%)
          ldwkvt)
       (declare (ignore var-0 var-5))
       (when var-1 (setf m var-1))
       (when var-2 (setf n var-2))
       (when var-3 (setf czero var-3))
       (when var-4 (setf czero var-4))
       (when var-6 (setf ldwkvt var-6)))
     (setf nwork
             (f2cl-lib:int-add ivt (f2cl-lib:int-mul ldwkvt n))))
    (t
     (setf chunk
             (the f2cl-lib:integer4 (truncate (- lwork (* 3 m)) m)))
     (setf nwork
             (f2cl-lib:int-add ivt
                               (f2cl-lib:int-mul ldwkvt chunk)))))
   (setf irvt nrwork)
   (setf iru (f2cl-lib:int-add irvt (f2cl-lib:int-mul m m)))
   (setf nrwork (f2cl-lib:int-add iru (f2cl-lib:int-mul m m)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13)
       (sbdsdc "L" "I" m s
        (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                              rwork-%offset%)
        (f2cl-lib:array-slice rwork-%data% single-float (iru)
                              ((1 *)) rwork-%offset%)
        m
        (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                              ((1 *)) rwork-%offset%)
        m dum idum
        (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                              ((1 *)) rwork-%offset%)
```

```
      iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf m var-2))
  (when var-6 (setf m var-6))
  (when var-8 (setf m var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     m u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "Q" "L" "N" m m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(cond
 ((>= lwork
      (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                        (f2cl-lib:int-mul 3 m)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacp2 "F" m m
       (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                             ((1 *)) rwork-%offset%)
       m
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ivt)
                             ((1 *)) work-%offset%)
       ldwkvt)
    (declare (ignore var-0 var-3 var-5))
```

```
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf m var-4))
    (when var-6 (setf ldwkvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13)
      (cunmbr "P" "R" "C" m n m a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (itaup) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ivt)
                             ((1 *)) work-%offset%)
       ldwkvt
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (nwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf m var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldwkvt var-10))
    (when var-13 (setf ierr var-13)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "F" m n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ivt)
                             ((1 *)) work-%offset%)
       ldwkvt a lda)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldwkvt var-4))
    (when var-6 (setf lda var-6))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (cungbr "P" m n m a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (itaup) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (nwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf lda var-5))
```

```
      (when var-9 (setf ierr var-9)))
    (setf nrwork iru)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
                  ((> i n) nil)
      (tagbody
        (setf blk
                (min
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
                 (the f2cl-lib:integer4 chunk)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (clarcm m blk
             (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                                   ((1 *)) rwork-%offset%)
             m
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                   (ivt) ((1 *)) work-%offset%)
             ldwkvt
             (f2cl-lib:array-slice rwork-%data% single-float
                                   (nrwork) ((1 *))
                                   rwork-%offset%))
          (declare (ignore var-2 var-4 var-6 var-8))
          (when var-0 (setf m var-0))
          (when var-1 (setf blk var-1))
          (when var-3 (setf m var-3))
          (when var-5 (setf lda var-5))
          (when var-7 (setf ldwkvt var-7)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (clacpy "F" m blk
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                   (ivt) ((1 *)) work-%offset%)
             ldwkvt
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf blk var-2))
          (when var-4 (setf ldwkvt var-4))
          (when var-6 (setf lda var-6)))
      label60)))))
(wntqs (setf irvt nrwork)
 (setf iru (f2cl-lib:int-add irvt (f2cl-lib:int-mul m m)))
 (setf nrwork (f2cl-lib:int-add iru (f2cl-lib:int-mul m m)))
 (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (sbdsdc "L" "I" m s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     m
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
     m dum idum
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf m var-2))
  (when var-6 (setf m var-6))
  (when var-8 (setf m var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     m u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "Q" "L" "N" m m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
      (claset "F" m n czero czero vt ldvt)
    (declare (ignore var-0 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacp2 "F" m m
       (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                             ((1 *)) rwork-%offset%)
       m vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf m var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (cunmbr "P" "R" "C" m n m a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                             ((1 *)) work-%offset%)
       vt ldvt
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf m var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldvt var-10))
    (when var-13 (setf ierr var-13))))
 (t (setf irvt nrwork)
  (setf iru (f2cl-lib:int-add irvt (f2cl-lib:int-mul m m)))
  (setf nrwork (f2cl-lib:int-add iru (f2cl-lib:int-mul m m)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (sbdsdc "L" "I" m s
       (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                             rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float (iru)
                             ((1 *)) rwork-%offset%)
       m
       (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                             ((1 *)) rwork-%offset%)
       m dum idum
```

```
     (f2cl-lib:array-slice rwork-%data% single-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf m var-2))
  (when var-6 (setf m var-6))
  (when var-8 (setf m var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% single-float (iru)
                           ((1 *)) rwork-%offset%)
     m u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "Q" "L" "N" m m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "F" n n czero cone vt ldvt)
  (declare (ignore var-0 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf czero var-3))
  (when var-4 (setf cone var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% single-float (irvt)
                           ((1 *)) rwork-%offset%)
```

```
            m vt ldvt)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf m var-2))
          (when var-4 (setf m var-4))
          (when var-6 (setf ldvt var-6)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11 var-12 var-13)
            (cunmbr "P" "R" "C" n n m a lda
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                                   ((1 *)) work-%offset%)
             vt ldvt
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (nwork)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
          (declare
           (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
          (when var-3 (setf n var-3))
          (when var-4 (setf n var-4))
          (when var-5 (setf m var-5))
          (when var-7 (setf lda var-7))
          (when var-10 (setf ldvt var-10))
          (when var-13 (setf ierr var-13)))))))))
(cond
 ((= iscl 1)
  (if (> anrm bignum)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (slascl "G" 0 0 bignum anrm minmn 1 s minmn ierr)
        (declare (ignore var-0 var-1 var-2 var-6 var-7))
        (when var-3 (setf bignum var-3))
        (when var-4 (setf anrm var-4))
        (when var-5 (setf minmn var-5))
        (when var-8 (setf minmn var-8))
        (when var-9 (setf ierr var-9))))
  (if (and (/= info 0) (> anrm bignum))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (slascl "G" 0 0 bignum anrm (f2cl-lib:int-sub minmn 1) 1
           (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                                 rwork-%offset%)
           minmn ierr)
        (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
        (when var-3 (setf bignum var-3))
        (when var-4 (setf anrm var-4))
        (when var-8 (setf minmn var-8))
        (when var-9 (setf ierr var-9))))
  (if (< anrm smlnum)
      (multiple-value-bind
```

```
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (slascl "G" 0 0 smlnum anrm minmn 1 s minmn ierr)
                (declare (ignore var-0 var-1 var-2 var-6 var-7))
                (when var-3 (setf smlnum var-3))
                (when var-4 (setf anrm var-4))
                (when var-5 (setf minmn var-5))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf ierr var-9))))
          (if (and (/= info 0) (< anrm smlnum))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (slascl "G" 0 0 smlnum anrm (f2cl-lib:int-sub minmn 1) 1
                   (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                                         rwork-%offset%)
                   minmn ierr)
                (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
                (when var-3 (setf smlnum var-3))
                (when var-4 (setf anrm var-4))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf ierr var-9))))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce maxwrk 'f2cl-lib:complex8))
        (go end_label)
       end_label
        (return
         (values jobz m n nil lda nil nil ldu nil ldvt nil nil nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgesdd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
```

```
  (array single-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::jobz
  fortran-to-lisp::m
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil
  fortran-to-lisp::ldu nil
  fortran-to-lisp::ldvt nil nil nil
  nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::cgelqf
  fortran-to-lisp::cgebrd
  fortran-to-lisp::cgeqrf))))
```

———-

30.3 cgesvd

30.3.1 Axiom unit tests

— **cgesvd.input** —

```
)set break resume
)sys rm -f cgesvd.output
)spool cgesvd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

30.3.2 Axiom help page

— **cgesvd.help** —

```
==========================================================================
```

```
cgesvd examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgesvd (JOBU, JOBVT, M, N, A, LDA, S, U, LDU, VT, LDVT,
           WORK, LWORK, RWORK, INFO)
            CGESVD computes the singular value decomposition (SVD) for GE
           matrices

Function/Subroutine Documentation
   subroutine cgesvd (character JOBU, character JOBVT, integer M, integer N,
       complex, dimension( lda, * ) A, integer LDA, real, dimension( * ) S,
       complex, dimension( ldu, * ) U, integer LDU, complex, dimension( ldvt,
       * ) VT, integer LDVT, complex, dimension( * ) WORK, integer LWORK,
       real, dimension( * ) RWORK, integer INFO)
        CGESVD computes the singular value decomposition (SVD) for GE matrices

Purpose:
   CGESVD computes the singular value decomposition (SVD) of a complex
   M-by-N matrix A, optionally computing the left and/or right singular
   vectors. The SVD is written

      A = U * SIGMA * conjugate-transpose(V)

   where SIGMA is an M-by-N matrix which is zero except for its
   min(m,n) diagonal elements, U is an M-by-M unitary matrix, and
   V is an N-by-N unitary matrix.  The diagonal elements of SIGMA
   are the singular values of A; they are real and non-negative, and
   are returned in descending order.  The first min(m,n) columns of
   U and V are the left and right singular vectors of A.

   Note that the routine returns V**H, not V.

Parameters:

 JOBU
   JOBU is CHARACTER*1
   Specifies options for computing all or part of the matrix U:
   = 'A':  all M columns of U are returned in array U:
   = 'S':  the first min(m,n) columns of U (the left singular
           vectors) are returned in the array U;
   = 'O':  the first min(m,n) columns of U (the left singular
           vectors) are overwritten on the array A;
   = 'N':  no columns of U (no left singular vectors) are computed.
```

```
JOBVT
  JOBVT is CHARACTER*1
  Specifies options for computing all or part of the matrix
  V**H:
  = 'A':  all N rows of V**H are returned in the array VT;
  = 'S':  the first min(m,n) rows of V**H (the right singular
          vectors) are returned in the array VT;
  = 'O':  the first min(m,n) rows of V**H (the right singular
          vectors) are overwritten on the array A;
  = 'N':  no rows of V**H (no right singular vectors) are computed.

  JOBVT and JOBU cannot both be 'O'.

M
  M is INTEGER
  The number of rows of the input matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the input matrix A.  N >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit,
  if JOBU = 'O',  A is overwritten with the first min(m,n)
        columns of U (the left singular vectors, stored columnwise);
  if JOBVT = 'O', A is overwritten with the first min(m,n)
        rows of V**H (the right singular vectors, stored rowwise);
  if JOBU .ne. 'O' and JOBVT .ne. 'O', the contents of A are destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

S
  S is REAL array, dimension (min(M,N))
  The singular values of A, sorted so that S(i) >= S(i+1).

U
  U is COMPLEX array, dimension (LDU,UCOL)
  (LDU,M) if JOBU = 'A' or (LDU,min(M,N)) if JOBU = 'S'.
  If JOBU = 'A', U contains the M-by-M unitary matrix U;
  if JOBU = 'S', U contains the first min(m,n) columns of U
  (the left singular vectors, stored columnwise);
  if JOBU = 'N' or 'O', U is not referenced.

LDU
  LDU is INTEGER
  The leading dimension of the array U.  LDU >= 1; if
```

```
  JOBU = 'S' or 'A', LDU >= M.

VT
  VT is COMPLEX array, dimension (LDVT,N)
  If JOBVT = 'A', VT contains the N-by-N unitary matrix V**H;
  if JOBVT = 'S', VT contains the first min(m,n) rows of
  V**H (the right singular vectors, stored rowwise);
  if JOBVT = 'N' or 'O', VT is not referenced.

LDVT
  LDVT is INTEGER
  The leading dimension of the array VT.  LDVT >= 1; if
  JOBVT = 'A', LDVT >= N; if JOBVT = 'S', LDVT >= min(M,N).

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  LWORK >=  MAX(1,2*MIN(M,N)+MAX(M,N)).
  For good performance, LWORK should generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is REAL array, dimension (5*min(M,N))
  On exit, if INFO > 0, RWORK(1:MIN(M,N)-1) contains the
  unconverged superdiagonal elements of an upper bidiagonal
  matrix B whose diagonal is in S (not necessarily sorted).
  B satisfies A = U * B * VT, so it has the same singular
  values as A, and singular vectors related by U and VT.

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if CBDSQR did not converge, INFO specifies how many
        superdiagonals of an intermediate bidiagonal form B
        did not converge to zero. See the description of RWORK
        above for details.
```

———-

30.3.3 fortran code

— cgesvd.f —

```
*  =====================================================================
      SUBROUTINE CGESVD( JOBU, JOBVT, M, N, A, LDA, S, U, LDU, VT, LDVT,
     $                   WORK, LWORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          JOBU, JOBVT
      INTEGER            INFO, LDA, LDU, LDVT, LWORK, M, N
*     ..
*     .. Array Arguments ..
      REAL               RWORK( * ), S( * )
      COMPLEX            A( LDA, * ), U( LDU, * ), VT( LDVT, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0E0, 0.0E0 ),
     $                   CONE = ( 1.0E0, 0.0E0 ) )
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, WNTUA, WNTUAS, WNTUN, WNTUO, WNTUS,
     $                   WNTVA, WNTVAS, WNTVN, WNTVO, WNTVS
      INTEGER            BLK, CHUNK, I, IE, IERR, IR, IRWORK, ISCL,
     $                   ITAU, ITAUP, ITAUQ, IU, IWORK, LDWRKR, LDWRKU,
     $                   MAXWRK, MINMN, MINWRK, MNTHR, NCU, NCVT, NRU,
     $                   NRVT, WRKBL
      INTEGER            LWORK_CGEQRF, LWORK_CUNGQR_N, LWORK_CUNGQR_M,
     $                   LWORK_CGEBRD, LWORK_CUNGBR_P, LWORK_CUNGBR_Q,
     $                   LWORK_CGELQF, LWORK_CUNGLQ_N, LWORK_CUNGLQ_M
      REAL               ANRM, BIGNUM, EPS, SMLNUM
*     ..
*     .. Local Arrays ..
      REAL               DUM( 1 )
      COMPLEX            CDUM( 1 )
*     ..
*     .. External Subroutines ..
```

```
      EXTERNAL           CBDSQR, CGEBRD, CGELQF, CGEMM, CGEQRF, CLACPY,
     $                   CLASCL, CLASET, CUNGBR, CUNGLQ, CUNGQR, CUNMBR,
     $                   SLASCL, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               CLANGE, SLAMCH
      EXTERNAL           LSAME, ILAENV, CLANGE, SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      MINMN = MIN( M, N )
      WNTUA = LSAME( JOBU, 'A' )
      WNTUS = LSAME( JOBU, 'S' )
      WNTUAS = WNTUA .OR. WNTUS
      WNTUO = LSAME( JOBU, 'O' )
      WNTUN = LSAME( JOBU, 'N' )
      WNTVA = LSAME( JOBVT, 'A' )
      WNTVS = LSAME( JOBVT, 'S' )
      WNTVAS = WNTVA .OR. WNTVS
      WNTVO = LSAME( JOBVT, 'O' )
      WNTVN = LSAME( JOBVT, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
*
      IF( .NOT.( WNTUA .OR. WNTUS .OR. WNTUO .OR. WNTUN ) ) THEN
         INFO = -1
      ELSE IF( .NOT.( WNTVA .OR. WNTVS .OR. WNTVO .OR. WNTVN ) .OR.
     $         ( WNTVO .AND. WNTUO ) ) THEN
         INFO = -2
      ELSE IF( M.LT.0 ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -6
      ELSE IF( LDU.LT.1 .OR. ( WNTUAS .AND. LDU.LT.M ) ) THEN
         INFO = -9
      ELSE IF( LDVT.LT.1 .OR. ( WNTVA .AND. LDVT.LT.N ) .OR.
     $         ( WNTVS .AND. LDVT.LT.MINMN ) ) THEN
         INFO = -11
      END IF
*
*     Compute workspace
```

```
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       CWorkspace refers to complex workspace, and RWorkspace to
*       real workspace. NB refers to the optimal block size for the
*       immediately following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = 1
         MAXWRK = 1
         IF( M.GE.N .AND. MINMN.GT.0 ) THEN
*
*           Space needed for ZBDSQR is BDSPAC = 5*N
*
            MNTHR = ILAENV( 6, 'CGESVD', JOBU // JOBVT, M, N, 0, 0 )
*           Compute space needed for CGEQRF
            CALL CGEQRF( M, N, A, LDA, CDUM(1), CDUM(1), -1, IERR )
            LWORK_CGEQRF=CDUM(1)
*           Compute space needed for CUNGQR
            CALL CUNGQR( M, N, N, A, LDA, CDUM(1), CDUM(1), -1, IERR )
            LWORK_CUNGQR_N=CDUM(1)
            CALL CUNGQR( M, M, N, A, LDA, CDUM(1), CDUM(1), -1, IERR )
            LWORK_CUNGQR_M=CDUM(1)
*           Compute space needed for CGEBRD
            CALL CGEBRD( N, N, A, LDA, S, DUM(1), CDUM(1),
     $                   CDUM(1), CDUM(1), -1, IERR )
            LWORK_CGEBRD=CDUM(1)
*           Compute space needed for CUNGBR
            CALL CUNGBR( 'P', N, N, N, A, LDA, CDUM(1),
     $                   CDUM(1), -1, IERR )
            LWORK_CUNGBR_P=CDUM(1)
            CALL CUNGBR( 'Q', N, N, N, A, LDA, CDUM(1),
     $                   CDUM(1), -1, IERR )
            LWORK_CUNGBR_Q=CDUM(1)
*
            MNTHR = ILAENV( 6, 'CGESVD', JOBU // JOBVT, M, N, 0, 0 )
            IF( M.GE.MNTHR ) THEN
               IF( WNTUN ) THEN
*
*                 Path 1 (M much larger than N, JOBU='N')
*
                  MAXWRK = N + LWORK_CGEQRF
                  MAXWRK = MAX( MAXWRK, 2*N+LWORK_CGEBRD )
                  IF( WNTVO .OR. WNTVAS )
     $               MAXWRK = MAX( MAXWRK, 2*N+LWORK_CUNGBR_P )
                  MINWRK = 3*N
               ELSE IF( WNTUO .AND. WNTVN ) THEN
*
*                 Path 2 (M much larger than N, JOBU='O', JOBVT='N')
*
```

```
                 WRKBL = N + LWORK_CGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_CUNGQR_N )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CGEBRD )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CUNGBR_Q )
                 MAXWRK = MAX( N*N+WRKBL, N*N+M*N )
                 MINWRK = 2*N + M
              ELSE IF( WNTUO .AND. WNTVAS ) THEN
*
*                Path 3 (M much larger than N, JOBU='O', JOBVT='S' or
*                'A')
*
                 WRKBL = N + LWORK_CGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_CUNGQR_N )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CGEBRD )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CUNGBR_Q )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CUNGBR_P )
                 MAXWRK = MAX( N*N+WRKBL, N*N+M*N )
                 MINWRK = 2*N + M
              ELSE IF( WNTUS .AND. WNTVN ) THEN
*
*                Path 4 (M much larger than N, JOBU='S', JOBVT='N')
*
                 WRKBL = N + LWORK_CGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_CUNGQR_N )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CGEBRD )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CUNGBR_Q )
                 MAXWRK = N*N + WRKBL
                 MINWRK = 2*N + M
              ELSE IF( WNTUS .AND. WNTVO ) THEN
*
*                Path 5 (M much larger than N, JOBU='S', JOBVT='O')
*
                 WRKBL = N + LWORK_CGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_CUNGQR_N )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CGEBRD )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CUNGBR_Q )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CUNGBR_P )
                 MAXWRK = 2*N*N + WRKBL
                 MINWRK = 2*N + M
              ELSE IF( WNTUS .AND. WNTVAS ) THEN
*
*                Path 6 (M much larger than N, JOBU='S', JOBVT='S' or
*                'A')
*
                 WRKBL = N + LWORK_CGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_CUNGQR_N )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CGEBRD )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CUNGBR_Q )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CUNGBR_P )
                 MAXWRK = N*N + WRKBL
```

```
                 MINWRK = 2*N + M
              ELSE IF( WNTUA .AND. WNTVN ) THEN
*
*                Path 7 (M much larger than N, JOBU='A', JOBVT='N')
*
                 WRKBL = N + LWORK_CGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_CUNGQR_M )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CGEBRD )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CUNGBR_Q )
                 MAXWRK = N*N + WRKBL
                 MINWRK = 2*N + M
              ELSE IF( WNTUA .AND. WNTVO ) THEN
*
*                Path 8 (M much larger than N, JOBU='A', JOBVT='O')
*
                 WRKBL = N + LWORK_CGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_CUNGQR_M )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CGEBRD )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CUNGBR_Q )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CUNGBR_P )
                 MAXWRK = 2*N*N + WRKBL
                 MINWRK = 2*N + M
              ELSE IF( WNTUA .AND. WNTVAS ) THEN
*
*                Path 9 (M much larger than N, JOBU='A', JOBVT='S' or
*                'A')
*
                 WRKBL = N + LWORK_CGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_CUNGQR_M )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CGEBRD )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CUNGBR_Q )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_CUNGBR_P )
                 MAXWRK = N*N + WRKBL
                 MINWRK = 2*N + M
              END IF
           ELSE
*
*             Path 10 (M at least N, but not much larger)
*
              CALL CGEBRD( M, N, A, LDA, S, DUM(1), CDUM(1),
     $                     CDUM(1), CDUM(1), -1, IERR )
              LWORK_CGEBRD=CDUM(1)
              MAXWRK = 2*N + LWORK_CGEBRD
              IF( WNTUS .OR. WNTUO ) THEN
                 CALL CUNGBR( 'Q', M, N, N, A, LDA, CDUM(1),
     $                        CDUM(1), -1, IERR )
                 LWORK_CUNGBR_Q=CDUM(1)
                 MAXWRK = MAX( MAXWRK, 2*N+LWORK_CUNGBR_Q )
              END IF
              IF( WNTUA ) THEN
```

```
                 CALL CUNGBR( 'Q', M, M, N, A, LDA, CDUM(1),
     $                  CDUM(1), -1, IERR )
                 LWORK_CUNGBR_Q=CDUM(1)
                 MAXWRK = MAX( MAXWRK, 2*N+LWORK_CUNGBR_Q )
              END IF
              IF( .NOT.WNTVN ) THEN
                 MAXWRK = MAX( MAXWRK, 2*N+LWORK_CUNGBR_P )
              MINWRK = 2*N + M
              END IF
           END IF
        ELSE IF( MINMN.GT.0 ) THEN
*
*          Space needed for CBDSQR is BDSPAC = 5*M
*
           MNTHR = ILAENV( 6, 'CGESVD', JOBU // JOBVT, M, N, 0, 0 )
*          Compute space needed for CGELQF
           CALL CGELQF( M, N, A, LDA, CDUM(1), CDUM(1), -1, IERR )
           LWORK_CGELQF=CDUM(1)
*          Compute space needed for CUNGLQ
           CALL CUNGLQ( N, N, M, CDUM(1), N, CDUM(1), CDUM(1), -1,
     $                  IERR )
           LWORK_CUNGLQ_N=CDUM(1)
           CALL CUNGLQ( M, N, M, A, LDA, CDUM(1), CDUM(1), -1, IERR )
           LWORK_CUNGLQ_M=CDUM(1)
*          Compute space needed for CGEBRD
           CALL CGEBRD( M, M, A, LDA, S, DUM(1), CDUM(1),
     $                  CDUM(1), CDUM(1), -1, IERR )
           LWORK_CGEBRD=CDUM(1)
*           Compute space needed for CUNGBR P
           CALL CUNGBR( 'P', M, M, M, A, N, CDUM(1),
     $                  CDUM(1), -1, IERR )
           LWORK_CUNGBR_P=CDUM(1)
*          Compute space needed for CUNGBR Q
           CALL CUNGBR( 'Q', M, M, M, A, N, CDUM(1),
     $                  CDUM(1), -1, IERR )
           LWORK_CUNGBR_Q=CDUM(1)
           IF( N.GE.MNTHR ) THEN
              IF( WNTVN ) THEN
*
*                Path 1t(N much larger than M, JOBVT='N')
*
                 MAXWRK = M + LWORK_CGELQF
                 MAXWRK = MAX( MAXWRK, 2*M+LWORK_CGEBRD )
                 IF( WNTUO .OR. WNTUAS )
     $              MAXWRK = MAX( MAXWRK, 2*M+LWORK_CUNGBR_Q )
                 MINWRK = 3*M
              ELSE IF( WNTVO .AND. WNTUN ) THEN
*
*                Path 2t(N much larger than M, JOBU='N', JOBVT='O')
*
```

```
                 WRKBL = M + LWORK_CGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_CUNGLQ_M )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CGEBRD )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CUNGBR_P )
                 MAXWRK = MAX( M*M+WRKBL, M*M+M*N )
                 MINWRK = 2*M + N
              ELSE IF( WNTVO .AND. WNTUAS ) THEN
*
*                Path 3t(N much larger than M, JOBU='S' or 'A',
*                JOBVT='O')
*
                 WRKBL = M + LWORK_CGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_CUNGLQ_M )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CGEBRD )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CUNGBR_P )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CUNGBR_Q )
                 MAXWRK = MAX( M*M+WRKBL, M*M+M*N )
                 MINWRK = 2*M + N
              ELSE IF( WNTVS .AND. WNTUN ) THEN
*
*                Path 4t(N much larger than M, JOBU='N', JOBVT='S')
*
                 WRKBL = M + LWORK_CGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_CUNGLQ_M )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CGEBRD )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CUNGBR_P )
                 MAXWRK = M*M + WRKBL
                 MINWRK = 2*M + N
              ELSE IF( WNTVS .AND. WNTUO ) THEN
*
*                Path 5t(N much larger than M, JOBU='O', JOBVT='S')
*
                 WRKBL = M + LWORK_CGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_CUNGLQ_M )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CGEBRD )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CUNGBR_P )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CUNGBR_Q )
                 MAXWRK = 2*M*M + WRKBL
                 MINWRK = 2*M + N
              ELSE IF( WNTVS .AND. WNTUAS ) THEN
*
*                Path 6t(N much larger than M, JOBU='S' or 'A',
*                JOBVT='S')
*
                 WRKBL = M + LWORK_CGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_CUNGLQ_M )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CGEBRD )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CUNGBR_P )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CUNGBR_Q )
                 MAXWRK = M*M + WRKBL
```

```
                 MINWRK = 2*M + N
              ELSE IF( WNTVA .AND. WNTUN ) THEN
*
*                Path 7t(N much larger than M, JOBU='N', JOBVT='A')
*
                 WRKBL = M + LWORK_CGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_CUNGLQ_N )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CGEBRD )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CUNGBR_P )
                 MAXWRK = M*M + WRKBL
                 MINWRK = 2*M + N
              ELSE IF( WNTVA .AND. WNTUO ) THEN
*
*                Path 8t(N much larger than M, JOBU='O', JOBVT='A')
*
                 WRKBL = M + LWORK_CGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_CUNGLQ_N )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CGEBRD )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CUNGBR_P )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CUNGBR_Q )
                 MAXWRK = 2*M*M + WRKBL
                 MINWRK = 2*M + N
              ELSE IF( WNTVA .AND. WNTUAS ) THEN
*
*                Path 9t(N much larger than M, JOBU='S' or 'A',
*                JOBVT='A')
*
                 WRKBL = M + LWORK_CGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_CUNGLQ_N )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CGEBRD )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CUNGBR_P )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_CUNGBR_Q )
                 MAXWRK = M*M + WRKBL
                 MINWRK = 2*M + N
              END IF
           ELSE
*
*             Path 10t(N greater than M, but not much larger)
*
              CALL CGEBRD( M, N, A, LDA, S, DUM(1), CDUM(1),
     $                     CDUM(1), CDUM(1), -1, IERR )
              LWORK_CGEBRD=CDUM(1)
              MAXWRK = 2*M + LWORK_CGEBRD
              IF( WNTVS .OR. WNTVO ) THEN
*               Compute space needed for CUNGBR P
                CALL CUNGBR( 'P', M, N, M, A, N, CDUM(1),
     $                       CDUM(1), -1, IERR )
                LWORK_CUNGBR_P=CDUM(1)
                MAXWRK = MAX( MAXWRK, 2*M+LWORK_CUNGBR_P )
              END IF
```

```
               IF( WNTVA ) THEN
                 CALL CUNGBR( 'P', N,  N, M, A, N, CDUM(1),
     $                   CDUM(1), -1, IERR )
                 LWORK_CUNGBR_P=CDUM(1)
                 MAXWRK = MAX( MAXWRK, 2*M+LWORK_CUNGBR_P )
               END IF
               IF( .NOT.WNTUN ) THEN
                  MAXWRK = MAX( MAXWRK, 2*M+LWORK_CUNGBR_Q )
               MINWRK = 2*M + N
               END IF
            END IF
         END IF
         MAXWRK = MAX( MINWRK, MAXWRK )
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -13
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGESVD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SQRT( SLAMCH( 'S' ) ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = CLANGE( 'M', M, N, A, LDA, DUM )
      ISCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ISCL = 1
         CALL CLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, IERR )
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ISCL = 1
         CALL CLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, IERR )
      END IF
```

```
*
      IF( M.GE.N ) THEN
*
*        A has at least as many rows as columns. If A has sufficiently
*        more rows than columns, first reduce using the QR
*        decomposition (if sufficient workspace available)
*
         IF( M.GE.MNTHR ) THEN
*
            IF( WNTUN ) THEN
*
*              Path 1 (M much larger than N, JOBU='N')
*              No left singular vectors to be computed
*
               ITAU = 1
               IWORK = ITAU + N
*
*              Compute A=Q*R
*              (CWorkspace: need 2*N, prefer N+N*NB)
*              (RWorkspace: need 0)
*
               CALL CGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
     $                      LWORK-IWORK+1, IERR )
*
*              Zero out below R
*
               CALL CLASET( 'L', N-1, N-1, CZERO, CZERO, A( 2, 1 ),
     $                      LDA )
               IE = 1
               ITAUQ = 1
               ITAUP = ITAUQ + N
               IWORK = ITAUP + N
*
*              Bidiagonalize R in A
*              (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
*              (RWorkspace: need N)
*
               CALL CGEBRD( N, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                      IERR )
               NCVT = 0
               IF( WNTVO .OR. WNTVAS ) THEN
*
*                If right singular vectors desired, generate P'.
*                (CWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*                (RWorkspace: 0)
*
                 CALL CUNGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 NCVT = N
```

```
               END IF
               IRWORK = IE + N
*
*              Perform bidiagonal QR iteration, computing right
*              singular vectors of A in A if desired
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAC)
*
               CALL CBDSQR( 'U', N, NCVT, 0, 0, S, RWORK( IE ), A, LDA,
     $                      CDUM, 1, CDUM, 1, RWORK( IRWORK ), INFO )
*
*              If right singular vectors desired in VT, copy them there
*
               IF( WNTVAS )
     $            CALL CLACPY( 'F', N, N, A, LDA, VT, LDVT )
*
            ELSE IF( WNTUO .AND. WNTVN ) THEN
*
*              Path 2 (M much larger than N, JOBU='O', JOBVT='N')
*              N left singular vectors to be overwritten on A and
*              no right singular vectors to be computed
*
               IF( LWORK.GE.N*N+3*N ) THEN
*
*                Sufficient workspace for a fast algorithm
*
                 IR = 1
                 IF( LWORK.GE.MAX( WRKBL, LDA*N )+LDA*N ) THEN
*
*                  WORK(IU) is LDA by N, WORK(IR) is LDA by N
*
                   LDWRKU = LDA
                   LDWRKR = LDA
                 ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N )+N*N ) THEN
*
*                  WORK(IU) is LDA by N, WORK(IR) is N by N
*
                   LDWRKU = LDA
                   LDWRKR = N
                 ELSE
*
*                  WORK(IU) is LDWRKU by N, WORK(IR) is N by N
*
                   LDWRKU = ( LWORK-N*N ) / N
                   LDWRKR = N
                 END IF
                 ITAU = IR + LDWRKR*N
                 IWORK = ITAU + N
*
*                Compute A=Q*R
```

```
*                 (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*                 (RWorkspace: 0)
*
                  CALL CGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Copy R to WORK(IR) and zero out below it
*
                  CALL CLACPY( 'U', N, N, A, LDA, WORK( IR ), LDWRKR )
                  CALL CLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                         WORK( IR+1 ), LDWRKR )
*
*                 Generate Q in A
*                 (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*                 (RWorkspace: 0)
*
                  CALL CUNGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IE = 1
                  ITAUQ = ITAU
                  ITAUP = ITAUQ + N
                  IWORK = ITAUP + N
*
*                 Bidiagonalize R in WORK(IR)
*                 (CWorkspace: need N*N+3*N, prefer N*N+2*N+2*N*NB)
*                 (RWorkspace: need N)
*
                  CALL CGEBRD( N, N, WORK( IR ), LDWRKR, S, RWORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Generate left vectors bidiagonalizing R
*                 (CWorkspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*                 (RWorkspace: need 0)
*
                  CALL CUNGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                         WORK( ITAUQ ), WORK( IWORK ),
     $                         LWORK-IWORK+1, IERR )
                  IRWORK = IE + N
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of R in WORK(IR)
*                 (CWorkspace: need N*N)
*                 (RWorkspace: need BDSPAC)
*
                  CALL CBDSQR( 'U', N, 0, N, 0, S, RWORK( IE ), CDUM, 1,
     $                         WORK( IR ), LDWRKR, CDUM, 1,
     $                         RWORK( IRWORK ), INFO )
                  IU = ITAUQ
*
```

```
*                 Multiply Q in A by left singular vectors of R in
*                 WORK(IR), storing result in WORK(IU) and copying to A
*                 (CWorkspace: need N*N+N, prefer N*N+M*N)
*                 (RWorkspace: 0)
*
                  DO 10 I = 1, M, LDWRKU
                     CHUNK = MIN( M-I+1, LDWRKU )
                     CALL CGEMM( 'N', 'N', CHUNK, N, N, CONE, A( I, 1 ),
     $                           LDA, WORK( IR ), LDWRKR, CZERO,
     $                           WORK( IU ), LDWRKU )
                     CALL CLACPY( 'F', CHUNK, N, WORK( IU ), LDWRKU,
     $                            A( I, 1 ), LDA )
   10             CONTINUE
*
               ELSE
*
*                 Insufficient workspace for a fast algorithm
*
                  IE = 1
                  ITAUQ = 1
                  ITAUP = ITAUQ + N
                  IWORK = ITAUP + N
*
*                 Bidiagonalize A
*                 (CWorkspace: need 2*N+M, prefer 2*N+(M+N)*NB)
*                 (RWorkspace: N)
*
                  CALL CGEBRD( M, N, A, LDA, S, RWORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Generate left vectors bidiagonalizing A
*                 (CWorkspace: need 3*N, prefer 2*N+N*NB)
*                 (RWorkspace: 0)
*
                  CALL CUNGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IRWORK = IE + N
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of A in A
*                 (CWorkspace: need 0)
*                 (RWorkspace: need BDSPAC)
*
                  CALL CBDSQR( 'U', N, 0, M, 0, S, RWORK( IE ), CDUM, 1,
     $                         A, LDA, CDUM, 1, RWORK( IRWORK ), INFO )
*
               END IF
*
            ELSE IF( WNTUO .AND. WNTVAS ) THEN
```

```
*
*              Path 3 (M much larger than N, JOBU='O', JOBVT='S' or 'A')
*              N left singular vectors to be overwritten on A and
*              N right singular vectors to be computed in VT
*
               IF( LWORK.GE.N*N+3*N ) THEN
*
*                 Sufficient workspace for a fast algorithm
*
                  IR = 1
                  IF( LWORK.GE.MAX( WRKBL, LDA*N )+LDA*N ) THEN
*
*                    WORK(IU) is LDA by N and WORK(IR) is LDA by N
*
                     LDWRKU = LDA
                     LDWRKR = LDA
                  ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N )+N*N ) THEN
*
*                    WORK(IU) is LDA by N and WORK(IR) is N by N
*
                     LDWRKU = LDA
                     LDWRKR = N
                  ELSE
*
*                    WORK(IU) is LDWRKU by N and WORK(IR) is N by N
*
                     LDWRKU = ( LWORK-N*N ) / N
                     LDWRKR = N
                  END IF
                  ITAU = IR + LDWRKR*N
                  IWORK = ITAU + N
*
*                 Compute A=Q*R
*                 (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*                 (RWorkspace: 0)
*
                  CALL CGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Copy R to VT, zeroing out below it
*
                  CALL CLACPY( 'U', N, N, A, LDA, VT, LDVT )
                  IF( N.GT.1 )
     $               CALL CLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                            VT( 2, 1 ), LDVT )
*
*                 Generate Q in A
*                 (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*                 (RWorkspace: 0)
*
```

```
                 CALL CUNGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = 1
                 ITAUQ = ITAU
                 ITAUP = ITAUQ + N
                 IWORK = ITAUP + N
*
*                Bidiagonalize R in VT, copying result to WORK(IR)
*                (CWorkspace: need N*N+3*N, prefer N*N+2*N+2*N*NB)
*                (RWorkspace: need N)
*
                 CALL CGEBRD( N, N, VT, LDVT, S, RWORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL CLACPY( 'L', N, N, VT, LDVT, WORK( IR ), LDWRKR )
*
*                Generate left vectors bidiagonalizing R in WORK(IR)
*                (CWorkspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*                (RWorkspace: 0)
*
                 CALL CUNGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                        WORK( ITAUQ ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate right vectors bidiagonalizing R in VT
*                (CWorkspace: need N*N+3*N-1, prefer N*N+2*N+(N-1)*NB)
*                (RWorkspace: 0)
*
                 CALL CUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IRWORK = IE + N
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of R in WORK(IR) and computing right
*                singular vectors of R in VT
*                (CWorkspace: need N*N)
*                (RWorkspace: need BDSPAC)
*
                 CALL CBDSQR( 'U', N, N, N, 0, S, RWORK( IE ), VT,
     $                        LDVT, WORK( IR ), LDWRKR, CDUM, 1,
     $                        RWORK( IRWORK ), INFO )
                 IU = ITAUQ
*
*                Multiply Q in A by left singular vectors of R in
*                WORK(IR), storing result in WORK(IU) and copying to A
*                (CWorkspace: need N*N+N, prefer N*N+M*N)
*                (RWorkspace: 0)
*
                 DO 20 I = 1, M, LDWRKU
                    CHUNK = MIN( M-I+1, LDWRKU )
```

```
                 CALL CGEMM( 'N', 'N', CHUNK, N, N, CONE, A( I, 1 ),
     $                       LDA, WORK( IR ), LDWRKR, CZERO,
     $                       WORK( IU ), LDWRKU )
                 CALL CLACPY( 'F', CHUNK, N, WORK( IU ), LDWRKU,
     $                        A( I, 1 ), LDA )
   20         CONTINUE
*
            ELSE
*
*              Insufficient workspace for a fast algorithm
*
               ITAU = 1
               IWORK = ITAU + N
*
*              Compute A=Q*R
*              (CWorkspace: need 2*N, prefer N+N*NB)
*              (RWorkspace: 0)
*
               CALL CGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*              Copy R to VT, zeroing out below it
*
               CALL CLACPY( 'U', N, N, A, LDA, VT, LDVT )
               IF( N.GT.1 )
     $            CALL CLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                        VT( 2, 1 ), LDVT )
*
*              Generate Q in A
*              (CWorkspace: need 2*N, prefer N+N*NB)
*              (RWorkspace: 0)
*
               CALL CUNGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
               IE = 1
               ITAUQ = ITAU
               ITAUP = ITAUQ + N
               IWORK = ITAUP + N
*
*              Bidiagonalize R in VT
*              (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
*              (RWorkspace: N)
*
               CALL CGEBRD( N, N, VT, LDVT, S, RWORK( IE ),
     $                      WORK( ITAUQ ), WORK( ITAUP ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*              Multiply Q in A by left vectors bidiagonalizing R
*              (CWorkspace: need 2*N+M, prefer 2*N+M*NB)
*              (RWorkspace: 0)
```

```
*
                 CALL CUNMBR( 'Q', 'R', 'N', M, N, N, VT, LDVT,
     $                        WORK( ITAUQ ), A, LDA, WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate right vectors bidiagonalizing R in VT
*                (CWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*                (RWorkspace: 0)
*
                 CALL CUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IRWORK = IE + N
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of A in A and computing right
*                singular vectors of A in VT
*                (CWorkspace: 0)
*                (RWorkspace: need BDSPAC)
*
                 CALL CBDSQR( 'U', N, N, M, 0, S, RWORK( IE ), VT,
     $                        LDVT, A, LDA, CDUM, 1, RWORK( IRWORK ),
     $                        INFO )
*
              END IF
*
           ELSE IF( WNTUS ) THEN
*
              IF( WNTVN ) THEN
*
*                Path 4 (M much larger than N, JOBU='S', JOBVT='N')
*                N left singular vectors to be computed in U and
*                no right singular vectors to be computed
*
                 IF( LWORK.GE.N*N+3*N ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IR = 1
                    IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                      WORK(IR) is LDA by N
*
                       LDWRKR = LDA
                    ELSE
*
*                      WORK(IR) is N by N
*
                       LDWRKR = N
                    END IF
                    ITAU = IR + LDWRKR*N
```

```
                    IWORK = ITAU + N
*
*                   Compute A=Q*R
*                   (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*                   (RWorkspace: 0)
*
                    CALL CGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy R to WORK(IR), zeroing out below it
*
                    CALL CLACPY( 'U', N, N, A, LDA, WORK( IR ),
     $                           LDWRKR )
                    CALL CLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                           WORK( IR+1 ), LDWRKR )
*
*                   Generate Q in A
*                   (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*                   (RWorkspace: 0)
*
                    CALL CUNGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = 1
                    ITAUQ = ITAU
                    ITAUP = ITAUQ + N
                    IWORK = ITAUP + N
*
*                   Bidiagonalize R in WORK(IR)
*                   (CWorkspace: need N*N+3*N, prefer N*N+2*N+2*N*NB)
*                   (RWorkspace: need N)
*
                    CALL CGEBRD( N, N, WORK( IR ), LDWRKR, S,
     $                           RWORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate left vectors bidiagonalizing R in WORK(IR)
*                   (CWorkspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*                   (RWorkspace: 0)
*
                    CALL CUNGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    IRWORK = IE + N
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of R in WORK(IR)
*                   (CWorkspace: need N*N)
*                   (RWorkspace: need BDSPAC)
*
```

```
                 CALL CBDSQR( 'U', N, 0, N, 0, S, RWORK( IE ), CDUM,
     $                        1, WORK( IR ), LDWRKR, CDUM, 1,
     $                        RWORK( IRWORK ), INFO )
*
*                Multiply Q in A by left singular vectors of R in
*                WORK(IR), storing result in U
*                (CWorkspace: need N*N)
*                (RWorkspace: 0)
*
                 CALL CGEMM( 'N', 'N', M, N, N, CONE, A, LDA,
     $                       WORK( IR ), LDWRKR, CZERO, U, LDU )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + N
*
*                Compute A=Q*R, copying result to U
*                (CWorkspace: need 2*N, prefer N+N*NB)
*                (RWorkspace: 0)
*
                 CALL CGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL CLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                Generate Q in U
*                (CWorkspace: need 2*N, prefer N+N*NB)
*                (RWorkspace: 0)
*
                 CALL CUNGQR( M, N, N, U, LDU, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = 1
                 ITAUQ = ITAU
                 ITAUP = ITAUQ + N
                 IWORK = ITAUP + N
*
*                Zero out below R in A
*
                 CALL CLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                        A( 2, 1 ), LDA )
*
*                Bidiagonalize R in A
*                (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
*                (RWorkspace: need N)
*
                 CALL CGEBRD( N, N, A, LDA, S, RWORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
```

```
*
*                    Multiply Q in U by left vectors bidiagonalizing R
*                    (CWorkspace: need 2*N+M, prefer 2*N+M*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                            WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IRWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U
*                    (CWorkspace: 0)
*                    (RWorkspace: need BDSPAC)
*
                     CALL CBDSQR( 'U', N, 0, M, 0, S, RWORK( IE ), CDUM,
     $                            1, U, LDU, CDUM, 1, RWORK( IRWORK ),
     $                            INFO )
*
                  END IF
*
               ELSE IF( WNTVO ) THEN
*
*                 Path 5 (M much larger than N, JOBU='S', JOBVT='O')
*                 N left singular vectors to be computed in U and
*                 N right singular vectors to be overwritten on A
*
                  IF( LWORK.GE.2*N*N+3*N ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+2*LDA*N ) THEN
*
*                       WORK(IU) is LDA by N and WORK(IR) is LDA by N
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*N
                        LDWRKR = LDA
                     ELSE IF( LWORK.GE.WRKBL+( LDA+N )*N ) THEN
*
*                       WORK(IU) is LDA by N and WORK(IR) is N by N
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*N
                        LDWRKR = N
                     ELSE
*
*                       WORK(IU) is N by N and WORK(IR) is N by N
*
```

```
                      LDWRKU = N
                      IR = IU + LDWRKU*N
                      LDWRKR = N
                   END IF
                   ITAU = IR + LDWRKR*N
                   IWORK = ITAU + N
*
*                  Compute A=Q*R
*                  (CWorkspace: need 2*N*N+2*N, prefer 2*N*N+N+N*NB)
*                  (RWorkspace: 0)
*
                   CALL CGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                  Copy R to WORK(IU), zeroing out below it
*
                   CALL CLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                           LDWRKU )
                   CALL CLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                           WORK( IU+1 ), LDWRKU )
*
*                  Generate Q in A
*                  (CWorkspace: need 2*N*N+2*N, prefer 2*N*N+N+N*NB)
*                  (RWorkspace: 0)
*
                   CALL CUNGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                   IE = 1
                   ITAUQ = ITAU
                   ITAUP = ITAUQ + N
                   IWORK = ITAUP + N
*
*                  Bidiagonalize R in WORK(IU), copying result to
*                  WORK(IR)
*                  (CWorkspace: need   2*N*N+3*N,
*                               prefer 2*N*N+2*N+2*N*NB)
*                  (RWorkspace: need   N)
*
                   CALL CGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                           RWORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                   CALL CLACPY( 'U', N, N, WORK( IU ), LDWRKU,
     $                           WORK( IR ), LDWRKR )
*
*                  Generate left bidiagonalizing vectors in WORK(IU)
*                  (CWorkspace: need 2*N*N+3*N, prefer 2*N*N+2*N+N*NB)
*                  (RWorkspace: 0)
*
                   CALL CUNGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
```

```
     $                          WORK( ITAUQ ), WORK( IWORK ),
     $                          LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in WORK(IR)
*                    (CWorkspace: need   2*N*N+3*N-1,
*                                 prefer 2*N*N+2*N+(N-1)*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGBR( 'P', N, N, N, WORK( IR ), LDWRKR,
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                     IRWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IU) and computing
*                    right singular vectors of R in WORK(IR)
*                    (CWorkspace: need 2*N*N)
*                    (RWorkspace: need BDSPAC)
*
                     CALL CBDSQR( 'U', N, N, N, 0, S, RWORK( IE ),
     $                           WORK( IR ), LDWRKR, WORK( IU ),
     $                           LDWRKU, CDUM, 1, RWORK( IRWORK ),
     $                           INFO )
*
*                    Multiply Q in A by left singular vectors of R in
*                    WORK(IU), storing result in U
*                    (CWorkspace: need N*N)
*                    (RWorkspace: 0)
*
                     CALL CGEMM( 'N', 'N', M, N, N, CONE, A, LDA,
     $                          WORK( IU ), LDWRKU, CZERO, U, LDU )
*
*                    Copy right singular vectors of R to A
*                    (CWorkspace: need N*N)
*                    (RWorkspace: 0)
*
                     CALL CLACPY( 'F', N, N, WORK( IR ), LDWRKR, A,
     $                           LDA )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (CWorkspace: need 2*N, prefer N+N*NB)
*                    (RWorkspace: 0)
*
```

```
                    CALL CGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    CALL CLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                   Generate Q in U
*                   (CWorkspace: need 2*N, prefer N+N*NB)
*                   (RWorkspace: 0)
*
                    CALL CUNGQR( M, N, N, U, LDU, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = 1
                    ITAUQ = ITAU
                    ITAUP = ITAUQ + N
                    IWORK = ITAUP + N
*
*                   Zero out below R in A
*
                    CALL CLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                           A( 2, 1 ), LDA )
*
*                   Bidiagonalize R in A
*                   (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
*                   (RWorkspace: need N)
*
                    CALL CGEBRD( N, N, A, LDA, S, RWORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Multiply Q in U by left vectors bidiagonalizing R
*                   (CWorkspace: need 2*N+M, prefer 2*N+M*NB)
*                   (RWorkspace: 0)
*
                    CALL CUNMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                           WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate right vectors bidiagonalizing R in A
*                   (CWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*                   (RWorkspace: 0)
*
                    CALL CUNGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IRWORK = IE + N
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of A in U and computing right
*                   singular vectors of A in A
*                   (CWorkspace: 0)
*                   (RWorkspace: need BDSPAC)
*
```

```
                 CALL CBDSQR( 'U', N, N, M, 0, S, RWORK( IE ), A,
     $                        LDA, U, LDU, CDUM, 1, RWORK( IRWORK ),
     $                        INFO )
*
              END IF
*
           ELSE IF( WNTVAS ) THEN
*
              Path 6 (M much larger than N, JOBU='S', JOBVT='S'
*                     or 'A')
*             N left singular vectors to be computed in U and
*             N right singular vectors to be computed in VT
*
              IF( LWORK.GE.N*N+3*N ) THEN
*
                 Sufficient workspace for a fast algorithm
*
                 IU = 1
                 IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
                    WORK(IU) is LDA by N
*
                    LDWRKU = LDA
                 ELSE
*
                    WORK(IU) is N by N
*
                    LDWRKU = N
                 END IF
                 ITAU = IU + LDWRKU*N
                 IWORK = ITAU + N
*
                 Compute A=Q*R
*                (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*                (RWorkspace: 0)
*
                 CALL CGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy R to WORK(IU), zeroing out below it
*
                 CALL CLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                        LDWRKU )
                 CALL CLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                        WORK( IU+1 ), LDWRKU )
*
*                Generate Q in A
*                (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*                (RWorkspace: 0)
*
```

```
                    CALL CUNGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = 1
                    ITAUQ = ITAU
                    ITAUP = ITAUQ + N
                    IWORK = ITAUP + N
*
*                   Bidiagonalize R in WORK(IU), copying result to VT
*                   (CWorkspace: need N*N+3*N, prefer N*N+2*N+2*N*NB)
*                   (RWorkspace: need N)
*
                    CALL CGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                           RWORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    CALL CLACPY( 'U', N, N, WORK( IU ), LDWRKU, VT,
     $                           LDVT )
*
*                   Generate left bidiagonalizing vectors in WORK(IU)
*                   (CWorkspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*                   (RWorkspace: 0)
*
                    CALL CUNGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate right bidiagonalizing vectors in VT
*                   (CWorkspace: need   N*N+3*N-1,
*                                prefer N*N+2*N+(N-1)*NB)
*                   (RWorkspace: 0)
*
                    CALL CUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IRWORK = IE + N
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of R in WORK(IU) and computing
*                   right singular vectors of R in VT
*                   (CWorkspace: need N*N)
*                   (RWorkspace: need BDSPAC)
*
                    CALL CBDSQR( 'U', N, N, N, 0, S, RWORK( IE ), VT,
     $                           LDVT, WORK( IU ), LDWRKU, CDUM, 1,
     $                           RWORK( IRWORK ), INFO )
*
*                   Multiply Q in A by left singular vectors of R in
*                   WORK(IU), storing result in U
*                   (CWorkspace: need N*N)
*                   (RWorkspace: 0)
*
```

```
                 CALL CGEMM( 'N', 'N', M, N, N, CONE, A, LDA,
     $                       WORK( IU ), LDWRKU, CZERO, U, LDU )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + N
*
*                Compute A=Q*R, copying result to U
*                (CWorkspace: need 2*N, prefer N+N*NB)
*                (RWorkspace: 0)
*
                 CALL CGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL CLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                Generate Q in U
*                (CWorkspace: need 2*N, prefer N+N*NB)
*                (RWorkspace: 0)
*
                 CALL CUNGQR( M, N, N, U, LDU, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy R to VT, zeroing out below it
*
                 CALL CLACPY( 'U', N, N, A, LDA, VT, LDVT )
                 IF( N.GT.1 )
     $              CALL CLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                           VT( 2, 1 ), LDVT )
                 IE = 1
                 ITAUQ = ITAU
                 ITAUP = ITAUQ + N
                 IWORK = ITAUP + N
*
*                Bidiagonalize R in VT
*                (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
*                (RWorkspace: need N)
*
                 CALL CGEBRD( N, N, VT, LDVT, S, RWORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Multiply Q in U by left bidiagonalizing vectors
*                in VT
*                (CWorkspace: need 2*N+M, prefer 2*N+M*NB)
*                (RWorkspace: 0)
*
                 CALL CUNMBR( 'Q', 'R', 'N', M, N, N, VT, LDVT,
```

```
     $                          WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                          LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in VT
*                    (CWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IRWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in VT
*                    (CWorkspace: 0)
*                    (RWorkspace: need BDSPAC)
*
                     CALL CBDSQR( 'U', N, N, M, 0, S, RWORK( IE ), VT,
     $                          LDVT, U, LDU, CDUM, 1,
     $                          RWORK( IRWORK ), INFO )
*
                  END IF
*
               END IF
*
            ELSE IF( WNTUA ) THEN
*
               IF( WNTVN ) THEN
*
                  Path 7 (M much larger than N, JOBU='A', JOBVT='N')
*                 M left singular vectors to be computed in U and
*                 no right singular vectors to be computed
*
                  IF( LWORK.GE.N*N+MAX( N+M, 3*N ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IR = 1
                     IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                       WORK(IR) is LDA by N
*
                        LDWRKR = LDA
                     ELSE
*
*                       WORK(IR) is N by N
*
                        LDWRKR = N
                     END IF
                     ITAU = IR + LDWRKR*N
```

```
                    IWORK = ITAU + N
*
*                   Compute A=Q*R, copying result to U
*                   (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*                   (RWorkspace: 0)
*
                    CALL CGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    CALL CLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                   Copy R to WORK(IR), zeroing out below it
*
                    CALL CLACPY( 'U', N, N, A, LDA, WORK( IR ),
     $                           LDWRKR )
                    CALL CLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                           WORK( IR+1 ), LDWRKR )
*
*                   Generate Q in U
*                   (CWorkspace: need N*N+N+M, prefer N*N+N+M*NB)
*                   (RWorkspace: 0)
*
                    CALL CUNGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = 1
                    ITAUQ = ITAU
                    ITAUP = ITAUQ + N
                    IWORK = ITAUP + N
*
*                   Bidiagonalize R in WORK(IR)
*                   (CWorkspace: need N*N+3*N, prefer N*N+2*N+2*N*NB)
*                   (RWorkspace: need N)
*
                    CALL CGEBRD( N, N, WORK( IR ), LDWRKR, S,
     $                           RWORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate left bidiagonalizing vectors in WORK(IR)
*                   (CWorkspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*                   (RWorkspace: 0)
*
                    CALL CUNGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    IRWORK = IE + N
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of R in WORK(IR)
*                   (CWorkspace: need N*N)
*                   (RWorkspace: need BDSPAC)
```

```
*
                 CALL CBDSQR( 'U', N, 0, N, 0, S, RWORK( IE ), CDUM,
     $                        1, WORK( IR ), LDWRKR, CDUM, 1,
     $                        RWORK( IRWORK ), INFO )
*
*                Multiply Q in U by left singular vectors of R in
*                WORK(IR), storing result in A
*                (CWorkspace: need N*N)
*                (RWorkspace: 0)
*
                 CALL CGEMM( 'N', 'N', M, N, N, CONE, U, LDU,
     $                       WORK( IR ), LDWRKR, CZERO, A, LDA )
*
*                Copy left singular vectors of A from A to U
*
                 CALL CLACPY( 'F', M, N, A, LDA, U, LDU )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + N
*
*                Compute A=Q*R, copying result to U
*                (CWorkspace: need 2*N, prefer N+N*NB)
*                (RWorkspace: 0)
*
                 CALL CGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL CLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                Generate Q in U
*                (CWorkspace: need N+M, prefer N+M*NB)
*                (RWorkspace: 0)
*
                 CALL CUNGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = 1
                 ITAUQ = ITAU
                 ITAUP = ITAUQ + N
                 IWORK = ITAUP + N
*
*                Zero out below R in A
*
                 CALL CLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                        A( 2, 1 ), LDA )
*
*                Bidiagonalize R in A
*                (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
```

```
*                    (RWorkspace: need N)
*
                     CALL CGEBRD( N, N, A, LDA, S, RWORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply Q in U by left bidiagonalizing vectors
*                    in A
*                    (CWorkspace: need 2*N+M, prefer 2*N+M*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                            WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IRWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U
*                    (CWorkspace: 0)
*                    (RWorkspace: need BDSPAC)
*
                     CALL CBDSQR( 'U', N, 0, M, 0, S, RWORK( IE ), CDUM,
     $                            1, U, LDU, CDUM, 1, RWORK( IRWORK ),
     $                            INFO )
*
                  END IF
*
               ELSE IF( WNTVO ) THEN
*
*                 Path 8 (M much larger than N, JOBU='A', JOBVT='O')
*                 M left singular vectors to be computed in U and
*                 N right singular vectors to be overwritten on A
*
                  IF( LWORK.GE.2*N*N+MAX( N+M, 3*N ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+2*LDA*N ) THEN
*
*                       WORK(IU) is LDA by N and WORK(IR) is LDA by N
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*N
                        LDWRKR = LDA
                     ELSE IF( LWORK.GE.WRKBL+( LDA+N )*N ) THEN
*
*                       WORK(IU) is LDA by N and WORK(IR) is N by N
*
                        LDWRKU = LDA
```

```
                      IR = IU + LDWRKU*N
                      LDWRKR = N
                   ELSE
*
*                     WORK(IU) is N by N and WORK(IR) is N by N
*
                      LDWRKU = N
                      IR = IU + LDWRKU*N
                      LDWRKR = N
                   END IF
                   ITAU = IR + LDWRKR*N
                   IWORK = ITAU + N
*
*                  Compute A=Q*R, copying result to U
*                  (CWorkspace: need 2*N*N+2*N, prefer 2*N*N+N+N*NB)
*                  (RWorkspace: 0)
*
                   CALL CGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                   CALL CLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                  Generate Q in U
*                  (CWorkspace: need 2*N*N+N+M, prefer 2*N*N+N+M*NB)
*                  (RWorkspace: 0)
*
                   CALL CUNGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                  Copy R to WORK(IU), zeroing out below it
*
                   CALL CLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                           LDWRKU )
                   CALL CLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                           WORK( IU+1 ), LDWRKU )
                   IE = 1
                   ITAUQ = ITAU
                   ITAUP = ITAUQ + N
                   IWORK = ITAUP + N
*
*                  Bidiagonalize R in WORK(IU), copying result to
*                  WORK(IR)
*                  (CWorkspace: need   2*N*N+3*N,
*                               prefer 2*N*N+2*N+2*N*NB)
*                  (RWorkspace: need   N)
*
                   CALL CGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                           RWORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                   CALL CLACPY( 'U', N, N, WORK( IU ), LDWRKU,
```

```
     $                          WORK( IR ), LDWRKR )
*
*                    Generate left bidiagonalizing vectors in WORK(IU)
*                    (CWorkspace: need 2*N*N+3*N, prefer 2*N*N+2*N+N*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in WORK(IR)
*                    (CWorkspace: need   2*N*N+3*N-1,
*                                 prefer 2*N*N+2*N+(N-1)*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGBR( 'P', N, N, N, WORK( IR ), LDWRKR,
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                     IRWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IU) and computing
*                    right singular vectors of R in WORK(IR)
*                    (CWorkspace: need 2*N*N)
*                    (RWorkspace: need BDSPAC)
*
                     CALL CBDSQR( 'U', N, N, N, 0, S, RWORK( IE ),
     $                           WORK( IR ), LDWRKR, WORK( IU ),
     $                           LDWRKU, CDUM, 1, RWORK( IRWORK ),
     $                           INFO )
*
*                    Multiply Q in U by left singular vectors of R in
*                    WORK(IU), storing result in A
*                    (CWorkspace: need N*N)
*                    (RWorkspace: 0)
*
                     CALL CGEMM( 'N', 'N', M, N, N, CONE, U, LDU,
     $                          WORK( IU ), LDWRKU, CZERO, A, LDA )
*
*                    Copy left singular vectors of A from A to U
*
                     CALL CLACPY( 'F', M, N, A, LDA, U, LDU )
*
*                    Copy right singular vectors of R from WORK(IR) to A
*
                     CALL CLACPY( 'F', N, N, WORK( IR ), LDWRKR, A,
     $                           LDA )
*
                  ELSE
*
```

```
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (CWorkspace: need 2*N, prefer N+N*NB)
*                    (RWorkspace: 0)
*
                     CALL CGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL CLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                    Generate Q in U
*                    (CWorkspace: need N+M, prefer N+M*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = 1
                     ITAUQ = ITAU
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Zero out below R in A
*
                     CALL CLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                            A( 2, 1 ), LDA )
*
*                    Bidiagonalize R in A
*                    (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
*                    (RWorkspace: need N)
*
                     CALL CGEBRD( N, N, A, LDA, S, RWORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply Q in U by left bidiagonalizing vectors
*                    in A
*                    (CWorkspace: need 2*N+M, prefer 2*N+M*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                            WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in A
*                    (CWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*                    (RWorkspace: 0)
*
```

```
                 CALL CUNGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IRWORK = IE + N
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of A in U and computing right
*                singular vectors of A in A
*                (CWorkspace: 0)
*                (RWorkspace: need BDSPAC)
*
                 CALL CBDSQR( 'U', N, N, M, 0, S, RWORK( IE ), A,
     $                        LDA, U, LDU, CDUM, 1, RWORK( IRWORK ),
     $                        INFO )
*
              END IF
*
           ELSE IF( WNTVAS ) THEN
*
              Path 9 (M much larger than N, JOBU='A', JOBVT='S'
*                     or 'A')
*             M left singular vectors to be computed in U and
*             N right singular vectors to be computed in VT
*
              IF( LWORK.GE.N*N+MAX( N+M, 3*N ) ) THEN
*
*                Sufficient workspace for a fast algorithm
*
                 IU = 1
                 IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                   WORK(IU) is LDA by N
*
                    LDWRKU = LDA
                 ELSE
*
*                   WORK(IU) is N by N
*
                    LDWRKU = N
                 END IF
                 ITAU = IU + LDWRKU*N
                 IWORK = ITAU + N
*
*                Compute A=Q*R, copying result to U
*                (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*                (RWorkspace: 0)
*
                 CALL CGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL CLACPY( 'L', M, N, A, LDA, U, LDU )
*
```

```
*                    Generate Q in U
*                    (CWorkspace: need N*N+N+M, prefer N*N+N+M*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy R to WORK(IU), zeroing out below it
*
                     CALL CLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                            LDWRKU )
                     CALL CLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                            WORK( IU+1 ), LDWRKU )
                     IE = 1
                     ITAUQ = ITAU
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Bidiagonalize R in WORK(IU), copying result to VT
*                    (CWorkspace: need N*N+3*N, prefer N*N+2*N+2*N*NB)
*                    (RWorkspace: need N)
*
                     CALL CGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                            RWORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     CALL CLACPY( 'U', N, N, WORK( IU ), LDWRKU, VT,
     $                            LDVT )
*
*                    Generate left bidiagonalizing vectors in WORK(IU)
*                    (CWorkspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                            WORK( ITAUQ ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in VT
*                    (CWorkspace: need   N*N+3*N-1,
*                                 prefer N*N+2*N+(N-1)*NB)
*                    (RWorkspace: need   0)
*
                     CALL CUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IRWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IU) and computing
*                    right singular vectors of R in VT
*                    (CWorkspace: need N*N)
```

```
*                    (RWorkspace: need BDSPAC)
*
                     CALL CBDSQR( 'U', N, N, N, 0, S, RWORK( IE ), VT,
     $                            LDVT, WORK( IU ), LDWRKU, CDUM, 1,
     $                            RWORK( IRWORK ), INFO )
*
*                    Multiply Q in U by left singular vectors of R in
*                    WORK(IU), storing result in A
*                    (CWorkspace: need N*N)
*                    (RWorkspace: 0)
*
                     CALL CGEMM( 'N', 'N', M, N, N, CONE, U, LDU,
     $                           WORK( IU ), LDWRKU, CZERO, A, LDA )
*
*                    Copy left singular vectors of A from A to U
*
                     CALL CLACPY( 'F', M, N, A, LDA, U, LDU )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (CWorkspace: need 2*N, prefer N+N*NB)
*                    (RWorkspace: 0)
*
                     CALL CGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL CLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                    Generate Q in U
*                    (CWorkspace: need N+M, prefer N+M*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy R from A to VT, zeroing out below it
*
                     CALL CLACPY( 'U', N, N, A, LDA, VT, LDVT )
                     IF( N.GT.1 )
     $                  CALL CLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                               VT( 2, 1 ), LDVT )
                     IE = 1
                     ITAUQ = ITAU
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
```

```
*
*                    Bidiagonalize R in VT
*                    (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
*                    (RWorkspace: need N)
*
                     CALL CGEBRD( N, N, VT, LDVT, S, RWORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply Q in U by left bidiagonalizing vectors
*                    in VT
*                    (CWorkspace: need 2*N+M, prefer 2*N+M*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNMBR( 'Q', 'R', 'N', M, N, N, VT, LDVT,
     $                            WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in VT
*                    (CWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IRWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in VT
*                    (CWorkspace: 0)
*                    (RWorkspace: need BDSPAC)
*
                     CALL CBDSQR( 'U', N, N, M, 0, S, RWORK( IE ), VT,
     $                            LDVT, U, LDU, CDUM, 1,
     $                            RWORK( IRWORK ), INFO )
*
                  END IF
*
               END IF
*
            END IF
*
         ELSE
*
*           M .LT. MNTHR
*
*           Path 10 (M at least N, but not much larger)
*           Reduce to bidiagonal form without QR decomposition
*
            IE = 1
```

```
            ITAUQ = 1
            ITAUP = ITAUQ + N
            IWORK = ITAUP + N
*
*           Bidiagonalize A
*           (CWorkspace: need 2*N+M, prefer 2*N+(M+N)*NB)
*           (RWorkspace: need N)
*
            CALL CGEBRD( M, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                   WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                   IERR )
            IF( WNTUAS ) THEN
*
*              If left singular vectors desired in U, copy result to U
*              and generate left bidiagonalizing vectors in U
*              (CWorkspace: need 2*N+NCU, prefer 2*N+NCU*NB)
*              (RWorkspace: 0)
*
               CALL CLACPY( 'L', M, N, A, LDA, U, LDU )
               IF( WNTUS )
     $            NCU = N
               IF( WNTUA )
     $            NCU = M
               CALL CUNGBR( 'Q', M, NCU, N, U, LDU, WORK( ITAUQ ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IF( WNTVAS ) THEN
*
*              If right singular vectors desired in VT, copy result to
*              VT and generate right bidiagonalizing vectors in VT
*              (CWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*              (RWorkspace: 0)
*
               CALL CLACPY( 'U', N, N, A, LDA, VT, LDVT )
               CALL CUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IF( WNTUO ) THEN
*
*              If left singular vectors desired in A, generate left
*              bidiagonalizing vectors in A
*              (CWorkspace: need 3*N, prefer 2*N+N*NB)
*              (RWorkspace: 0)
*
               CALL CUNGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IF( WNTVO ) THEN
*
*              If right singular vectors desired in A, generate right
```

```
*              bidiagonalizing vectors in A
*              (CWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*              (RWorkspace: 0)
*
               CALL CUNGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IRWORK = IE + N
            IF( WNTUAS .OR. WNTUO )
     $         NRU = M
            IF( WNTUN )
     $         NRU = 0
            IF( WNTVAS .OR. WNTVO )
     $         NCVT = N
            IF( WNTVN )
     $         NCVT = 0
            IF( ( .NOT.WNTUO ) .AND. ( .NOT.WNTVO ) ) THEN
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in U and computing right singular
*              vectors in VT
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAC)
*
               CALL CBDSQR( 'U', N, NCVT, NRU, 0, S, RWORK( IE ), VT,
     $                      LDVT, U, LDU, CDUM, 1, RWORK( IRWORK ),
     $                      INFO )
            ELSE IF( ( .NOT.WNTUO ) .AND. WNTVO ) THEN
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in U and computing right singular
*              vectors in A
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAC)
*
               CALL CBDSQR( 'U', N, NCVT, NRU, 0, S, RWORK( IE ), A,
     $                      LDA, U, LDU, CDUM, 1, RWORK( IRWORK ),
     $                      INFO )
            ELSE
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in A and computing right singular
*              vectors in VT
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAC)
*
               CALL CBDSQR( 'U', N, NCVT, NRU, 0, S, RWORK( IE ), VT,
     $                      LDVT, A, LDA, CDUM, 1, RWORK( IRWORK ),
     $                      INFO )
            END IF
```

```
*
         END IF
*
      ELSE
*
*        A has more columns than rows. If A has sufficiently more
*        columns than rows, first reduce using the LQ decomposition (if
*        sufficient workspace available)
*
         IF( N.GE.MNTHR ) THEN
*
            IF( WNTVN ) THEN
*
*              Path 1t(N much larger than M, JOBVT='N')
*              No right singular vectors to be computed
*
               ITAU = 1
               IWORK = ITAU + M
*
*              Compute A=L*Q
*              (CWorkspace: need 2*M, prefer M+M*NB)
*              (RWorkspace: 0)
*
               CALL CGELQF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
     $                      LWORK-IWORK+1, IERR )
*
*              Zero out above L
*
               CALL CLASET( 'U', M-1, M-1, CZERO, CZERO, A( 1, 2 ),
     $                      LDA )
               IE = 1
               ITAUQ = 1
               ITAUP = ITAUQ + M
               IWORK = ITAUP + M
*
*              Bidiagonalize L in A
*              (CWorkspace: need 3*M, prefer 2*M+2*M*NB)
*              (RWorkspace: need M)
*
               CALL CGEBRD( M, M, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                      IERR )
               IF( WNTUO .OR. WNTUAS ) THEN
*
*                If left singular vectors desired, generate Q
*                (CWorkspace: need 3*M, prefer 2*M+M*NB)
*                (RWorkspace: 0)
*
                 CALL CUNGBR( 'Q', M, M, M, A, LDA, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
```

```
               END IF
               IRWORK = IE + M
               NRU = 0
               IF( WNTUO .OR. WNTUAS )
     $            NRU = M
*
*              Perform bidiagonal QR iteration, computing left singular
*              vectors of A in A if desired
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAC)
*
               CALL CBDSQR( 'U', M, 0, NRU, 0, S, RWORK( IE ), CDUM, 1,
     $                      A, LDA, CDUM, 1, RWORK( IRWORK ), INFO )
*
*              If left singular vectors desired in U, copy them there
*
               IF( WNTUAS )
     $            CALL CLACPY( 'F', M, M, A, LDA, U, LDU )
*
            ELSE IF( WNTVO .AND. WNTUN ) THEN
*
*              Path 2t(N much larger than M, JOBU='N', JOBVT='O')
*              M right singular vectors to be overwritten on A and
*              no left singular vectors to be computed
*
               IF( LWORK.GE.M*M+3*M ) THEN
*
*                Sufficient workspace for a fast algorithm
*
                 IR = 1
                 IF( LWORK.GE.MAX( WRKBL, LDA*N )+LDA*M ) THEN
*
*                  WORK(IU) is LDA by N and WORK(IR) is LDA by M
*
                   LDWRKU = LDA
                   CHUNK = N
                   LDWRKR = LDA
                 ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N )+M*M ) THEN
*
*                  WORK(IU) is LDA by N and WORK(IR) is M by M
*
                   LDWRKU = LDA
                   CHUNK = N
                   LDWRKR = M
                 ELSE
*
*                  WORK(IU) is M by CHUNK and WORK(IR) is M by M
*
                   LDWRKU = M
                   CHUNK = ( LWORK-M*M ) / M
```

```
                 LDWRKR = M
              END IF
              ITAU = IR + LDWRKR*M
              IWORK = ITAU + M
*
*             Compute A=L*Q
*             (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*             (RWorkspace: 0)
*
              CALL CGELQF( M, N, A, LDA, WORK( ITAU ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*             Copy L to WORK(IR) and zero out above it
*
              CALL CLACPY( 'L', M, M, A, LDA, WORK( IR ), LDWRKR )
              CALL CLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                     WORK( IR+LDWRKR ), LDWRKR )
*
*             Generate Q in A
*             (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*             (RWorkspace: 0)
*
              CALL CUNGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
              IE = 1
              ITAUQ = ITAU
              ITAUP = ITAUQ + M
              IWORK = ITAUP + M
*
*             Bidiagonalize L in WORK(IR)
*             (CWorkspace: need M*M+3*M, prefer M*M+2*M+2*M*NB)
*             (RWorkspace: need M)
*
              CALL CGEBRD( M, M, WORK( IR ), LDWRKR, S, RWORK( IE ),
     $                     WORK( ITAUQ ), WORK( ITAUP ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*             Generate right vectors bidiagonalizing L
*             (CWorkspace: need M*M+3*M-1, prefer M*M+2*M+(M-1)*NB)
*             (RWorkspace: 0)
*
              CALL CUNGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                     WORK( ITAUP ), WORK( IWORK ),
     $                     LWORK-IWORK+1, IERR )
              IRWORK = IE + M
*
*             Perform bidiagonal QR iteration, computing right
*             singular vectors of L in WORK(IR)
*             (CWorkspace: need M*M)
*             (RWorkspace: need BDSPAC)
```

```
*
                 CALL CBDSQR( 'U', M, M, 0, 0, S, RWORK( IE ),
     $                        WORK( IR ), LDWRKR, CDUM, 1, CDUM, 1,
     $                        RWORK( IRWORK ), INFO )
                 IU = ITAUQ
*
*                Multiply right singular vectors of L in WORK(IR) by Q
*                in A, storing result in WORK(IU) and copying to A
*                (CWorkspace: need M*M+M, prefer M*M+M*N)
*                (RWorkspace: 0)
*
                 DO 30 I = 1, N, CHUNK
                    BLK = MIN( N-I+1, CHUNK )
                    CALL CGEMM( 'N', 'N', M, BLK, M, CONE, WORK( IR ),
     $                          LDWRKR, A( 1, I ), LDA, CZERO,
     $                          WORK( IU ), LDWRKU )
                    CALL CLACPY( 'F', M, BLK, WORK( IU ), LDWRKU,
     $                           A( 1, I ), LDA )
   30            CONTINUE
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 IE = 1
                 ITAUQ = 1
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Bidiagonalize A
*                (CWorkspace: need 2*M+N, prefer 2*M+(M+N)*NB)
*                (RWorkspace: need M)
*
                 CALL CGEBRD( M, N, A, LDA, S, RWORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Generate right vectors bidiagonalizing A
*                (CWorkspace: need 3*M, prefer 2*M+M*NB)
*                (RWorkspace: 0)
*
                 CALL CUNGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IRWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing right
*                singular vectors of A in A
*                (CWorkspace: 0)
*                (RWorkspace: need BDSPAC)
*
```

```
                 CALL CBDSQR( 'L', M, N, 0, 0, S, RWORK( IE ), A, LDA,
     $                        CDUM, 1, CDUM, 1, RWORK( IRWORK ), INFO )
*
              END IF
*
           ELSE IF( WNTVO .AND. WNTUAS ) THEN
*
              Path 3t(N much larger than M, JOBU='S' or 'A', JOBVT='O')
*             M right singular vectors to be overwritten on A and
*             M left singular vectors to be computed in U
*
              IF( LWORK.GE.M*M+3*M ) THEN
*
                 Sufficient workspace for a fast algorithm
*
                 IR = 1
                 IF( LWORK.GE.MAX( WRKBL, LDA*N )+LDA*M ) THEN
*
                    WORK(IU) is LDA by N and WORK(IR) is LDA by M
*
                    LDWRKU = LDA
                    CHUNK = N
                    LDWRKR = LDA
                 ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N )+M*M ) THEN
*
                    WORK(IU) is LDA by N and WORK(IR) is M by M
*
                    LDWRKU = LDA
                    CHUNK = N
                    LDWRKR = M
                 ELSE
*
                    WORK(IU) is M by CHUNK and WORK(IR) is M by M
*
                    LDWRKU = M
                    CHUNK = ( LWORK-M*M ) / M
                    LDWRKR = M
                 END IF
                 ITAU = IR + LDWRKR*M
                 IWORK = ITAU + M
*
                 Compute A=L*Q
*                (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*                (RWorkspace: 0)
*
                 CALL CGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy L to U, zeroing about above it
*
```

```
                 CALL CLACPY( 'L', M, M, A, LDA, U, LDU )
                 CALL CLASET( 'U', M-1, M-1, CZERO, CZERO, U( 1, 2 ),
     $                        LDU )
*
*                Generate Q in A
*                (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*                (RWorkspace: 0)
*
                 CALL CUNGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = 1
                 ITAUQ = ITAU
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Bidiagonalize L in U, copying result to WORK(IR)
*                (CWorkspace: need M*M+3*M, prefer M*M+2*M+2*M*NB)
*                (RWorkspace: need M)
*
                 CALL CGEBRD( M, M, U, LDU, S, RWORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL CLACPY( 'U', M, M, U, LDU, WORK( IR ), LDWRKR )
*
*                Generate right vectors bidiagonalizing L in WORK(IR)
*                (CWorkspace: need M*M+3*M-1, prefer M*M+2*M+(M-1)*NB)
*                (RWorkspace: 0)
*
                 CALL CUNGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                        WORK( ITAUP ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate left vectors bidiagonalizing L in U
*                (CWorkspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*                (RWorkspace: 0)
*
                 CALL CUNGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IRWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of L in U, and computing right
*                singular vectors of L in WORK(IR)
*                (CWorkspace: need M*M)
*                (RWorkspace: need BDSPAC)
*
                 CALL CBDSQR( 'U', M, M, M, 0, S, RWORK( IE ),
     $                        WORK( IR ), LDWRKR, U, LDU, CDUM, 1,
     $                        RWORK( IRWORK ), INFO )
                 IU = ITAUQ
```

```
*
*                 Multiply right singular vectors of L in WORK(IR) by Q
*                 in A, storing result in WORK(IU) and copying to A
*                 (CWorkspace: need M*M+M, prefer M*M+M*N))
*                 (RWorkspace: 0)
*
                  DO 40 I = 1, N, CHUNK
                     BLK = MIN( N-I+1, CHUNK )
                     CALL CGEMM( 'N', 'N', M, BLK, M, CONE, WORK( IR ),
     $                           LDWRKR, A( 1, I ), LDA, CZERO,
     $                           WORK( IU ), LDWRKU )
                     CALL CLACPY( 'F', M, BLK, WORK( IU ), LDWRKU,
     $                            A( 1, I ), LDA )
   40             CONTINUE
*
               ELSE
*
*                 Insufficient workspace for a fast algorithm
*
                  ITAU = 1
                  IWORK = ITAU + M
*
*                 Compute A=L*Q
*                 (CWorkspace: need 2*M, prefer M+M*NB)
*                 (RWorkspace: 0)
*
                  CALL CGELQF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Copy L to U, zeroing out above it
*
                  CALL CLACPY( 'L', M, M, A, LDA, U, LDU )
                  CALL CLASET( 'U', M-1, M-1, CZERO, CZERO, U( 1, 2 ),
     $                         LDU )
*
*                 Generate Q in A
*                 (CWorkspace: need 2*M, prefer M+M*NB)
*                 (RWorkspace: 0)
*
                  CALL CUNGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IE = 1
                  ITAUQ = ITAU
                  ITAUP = ITAUQ + M
                  IWORK = ITAUP + M
*
*                 Bidiagonalize L in U
*                 (CWorkspace: need 3*M, prefer 2*M+2*M*NB)
*                 (RWorkspace: need M)
*
```

```
                 CALL CGEBRD( M, M, U, LDU, S, RWORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Multiply right vectors bidiagonalizing L by Q in A
*                (CWorkspace: need 2*M+N, prefer 2*M+N*NB)
*                (RWorkspace: 0)
*
                 CALL CUNMBR( 'P', 'L', 'C', M, N, M, U, LDU,
     $                        WORK( ITAUP ), A, LDA, WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate left vectors bidiagonalizing L in U
*                (CWorkspace: need 3*M, prefer 2*M+M*NB)
*                (RWorkspace: 0)
*
                 CALL CUNGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IRWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of A in U and computing right
*                singular vectors of A in A
*                (CWorkspace: 0)
*                (RWorkspace: need BDSPAC)
*
                 CALL CBDSQR( 'U', M, N, M, 0, S, RWORK( IE ), A, LDA,
     $                        U, LDU, CDUM, 1, RWORK( IRWORK ), INFO )
*
              END IF
*
           ELSE IF( WNTVS ) THEN
*
              IF( WNTUN ) THEN
*
*                Path 4t(N much larger than M, JOBU='N', JOBVT='S')
*                M right singular vectors to be computed in VT and
*                no left singular vectors to be computed
*
                 IF( LWORK.GE.M*M+3*M ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IR = 1
                    IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                      WORK(IR) is LDA by M
*
                       LDWRKR = LDA
                    ELSE
```

```
*
*                       WORK(IR) is M by M
*
                        LDWRKR = M
                     END IF
                     ITAU = IR + LDWRKR*M
                     IWORK = ITAU + M
*
*                    Compute A=L*Q
*                    (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL CGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to WORK(IR), zeroing out above it
*
                     CALL CLACPY( 'L', M, M, A, LDA, WORK( IR ),
     $                            LDWRKR )
                     CALL CLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                            WORK( IR+LDWRKR ), LDWRKR )
*
*                    Generate Q in A
*                    (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = 1
                     ITAUQ = ITAU
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Bidiagonalize L in WORK(IR)
*                    (CWorkspace: need M*M+3*M, prefer M*M+2*M+2*M*NB)
*                    (RWorkspace: need M)
*
                     CALL CGEBRD( M, M, WORK( IR ), LDWRKR, S,
     $                            RWORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right vectors bidiagonalizing L in
*                    WORK(IR)
*                    (CWorkspace: need M*M+3*M, prefer M*M+2*M+(M-1)*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
```

```
                 IRWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing right
*                singular vectors of L in WORK(IR)
*                (CWorkspace: need M*M)
*                (RWorkspace: need BDSPAC)
*
                 CALL CBDSQR( 'U', M, M, 0, 0, S, RWORK( IE ),
     $                        WORK( IR ), LDWRKR, CDUM, 1, CDUM, 1,
     $                        RWORK( IRWORK ), INFO )
*
*                Multiply right singular vectors of L in WORK(IR) by
*                Q in A, storing result in VT
*                (CWorkspace: need M*M)
*                (RWorkspace: 0)
*
                 CALL CGEMM( 'N', 'N', M, N, M, CONE, WORK( IR ),
     $                       LDWRKR, A, LDA, CZERO, VT, LDVT )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + M
*
*                Compute A=L*Q
*                (CWorkspace: need 2*M, prefer M+M*NB)
*                (RWorkspace: 0)
*
                 CALL CGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy result to VT
*
                 CALL CLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                Generate Q in VT
*                (CWorkspace: need 2*M, prefer M+M*NB)
*                (RWorkspace: 0)
*
                 CALL CUNGLQ( M, N, M, VT, LDVT, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = 1
                 ITAUQ = ITAU
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Zero out above L in A
*
```

```
                    CALL CLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                           A( 1, 2 ), LDA )
*
*                   Bidiagonalize L in A
*                   (CWorkspace: need 3*M, prefer 2*M+2*M*NB)
*                   (RWorkspace: need M)
*
                    CALL CGEBRD( M, M, A, LDA, S, RWORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Multiply right vectors bidiagonalizing L by Q in VT
*                   (CWorkspace: need 2*M+N, prefer 2*M+N*NB)
*                   (RWorkspace: 0)
*
                    CALL CUNMBR( 'P', 'L', 'C', M, N, M, A, LDA,
     $                           WORK( ITAUP ), VT, LDVT,
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IRWORK = IE + M
*
*                   Perform bidiagonal QR iteration, computing right
*                   singular vectors of A in VT
*                   (CWorkspace: 0)
*                   (RWorkspace: need BDSPAC)
*
                    CALL CBDSQR( 'U', M, N, 0, 0, S, RWORK( IE ), VT,
     $                           LDVT, CDUM, 1, CDUM, 1,
     $                           RWORK( IRWORK ), INFO )
*
                 END IF
*
              ELSE IF( WNTUO ) THEN
*
                 Path 5t(N much larger than M, JOBU='O', JOBVT='S')
*                M right singular vectors to be computed in VT and
*                M left singular vectors to be overwritten on A
*
                 IF( LWORK.GE.2*M*M+3*M ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IU = 1
                    IF( LWORK.GE.WRKBL+2*LDA*M ) THEN
*
*                      WORK(IU) is LDA by M and WORK(IR) is LDA by M
*
                       LDWRKU = LDA
                       IR = IU + LDWRKU*M
                       LDWRKR = LDA
                    ELSE IF( LWORK.GE.WRKBL+( LDA+M )*M ) THEN
```

```
*
*                       WORK(IU) is LDA by M and WORK(IR) is M by M
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*M
                        LDWRKR = M
                     ELSE
*
*                       WORK(IU) is M by M and WORK(IR) is M by M
*
                        LDWRKU = M
                        IR = IU + LDWRKU*M
                        LDWRKR = M
                     END IF
                     ITAU = IR + LDWRKR*M
                     IWORK = ITAU + M
*
*                    Compute A=L*Q
*                    (CWorkspace: need 2*M*M+2*M, prefer 2*M*M+M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL CGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to WORK(IU), zeroing out below it
*
                     CALL CLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                            LDWRKU )
                     CALL CLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                            WORK( IU+LDWRKU ), LDWRKU )
*
*                    Generate Q in A
*                    (CWorkspace: need 2*M*M+2*M, prefer 2*M*M+M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = 1
                     ITAUQ = ITAU
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Bidiagonalize L in WORK(IU), copying result to
*                    WORK(IR)
*                    (CWorkspace: need   2*M*M+3*M,
*                                 prefer 2*M*M+2*M+2*M*NB)
*                    (RWorkspace: need   M)
*
                     CALL CGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                            RWORK( IE ), WORK( ITAUQ ),
```

```
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                     CALL CLACPY( 'L', M, M, WORK( IU ), LDWRKU,
     $                           WORK( IR ), LDWRKR )
*
*                    Generate right bidiagonalizing vectors in WORK(IU)
*                    (CWorkspace: need   2*M*M+3*M-1,
*                                 prefer 2*M*M+2*M+(M-1)*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors in WORK(IR)
*                    (CWorkspace: need 2*M*M+3*M, prefer 2*M*M+2*M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGBR( 'Q', M, M, M, WORK( IR ), LDWRKR,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                     IRWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of L in WORK(IR) and computing
*                    right singular vectors of L in WORK(IU)
*                    (CWorkspace: need 2*M*M)
*                    (RWorkspace: need BDSPAC)
*
                     CALL CBDSQR( 'U', M, M, M, 0, S, RWORK( IE ),
     $                           WORK( IU ), LDWRKU, WORK( IR ),
     $                           LDWRKR, CDUM, 1, RWORK( IRWORK ),
     $                           INFO )
*
*                    Multiply right singular vectors of L in WORK(IU) by
*                    Q in A, storing result in VT
*                    (CWorkspace: need M*M)
*                    (RWorkspace: 0)
*
                     CALL CGEMM( 'N', 'N', M, N, M, CONE, WORK( IU ),
     $                          LDWRKU, A, LDA, CZERO, VT, LDVT )
*
*                    Copy left singular vectors of L to A
*                    (CWorkspace: need M*M)
*                    (RWorkspace: 0)
*
                     CALL CLACPY( 'F', M, M, WORK( IR ), LDWRKR, A,
     $                           LDA )
*
                 ELSE
```

```
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (CWorkspace: need 2*M, prefer M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL CGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL CLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (CWorkspace: need 2*M, prefer M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGLQ( M, N, M, VT, LDVT, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = 1
                     ITAUQ = ITAU
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Zero out above L in A
*
                     CALL CLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                            A( 1, 2 ), LDA )
*
*                    Bidiagonalize L in A
*                    (CWorkspace: need 3*M, prefer 2*M+2*M*NB)
*                    (RWorkspace: need M)
*
                     CALL CGEBRD( M, M, A, LDA, S, RWORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply right vectors bidiagonalizing L by Q in VT
*                    (CWorkspace: need 2*M+N, prefer 2*M+N*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNMBR( 'P', 'L', 'C', M, N, M, A, LDA,
     $                            WORK( ITAUP ), VT, LDVT,
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors of L in A
*                    (CWorkspace: need 3*M, prefer 2*M+M*NB)
*                    (RWorkspace: 0)
*
```

```
                 CALL CUNGBR( 'Q', M, M, M, A, LDA, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IRWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of A in A and computing right
*                singular vectors of A in VT
*                (CWorkspace: 0)
*                (RWorkspace: need BDSPAC)
*
                 CALL CBDSQR( 'U', M, N, M, 0, S, RWORK( IE ), VT,
     $                        LDVT, A, LDA, CDUM, 1,
     $                        RWORK( IRWORK ), INFO )
*
              END IF
*
           ELSE IF( WNTUAS ) THEN
*
              Path 6t(N much larger than M, JOBU='S' or 'A',
*                     JOBVT='S')
*             M right singular vectors to be computed in VT and
*             M left singular vectors to be computed in U
*
              IF( LWORK.GE.M*M+3*M ) THEN
*
*                Sufficient workspace for a fast algorithm
*
                 IU = 1
                 IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                   WORK(IU) is LDA by N
*
                    LDWRKU = LDA
                 ELSE
*
*                   WORK(IU) is LDA by M
*
                    LDWRKU = M
                 END IF
                 ITAU = IU + LDWRKU*M
                 IWORK = ITAU + M
*
*                Compute A=L*Q
*                (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*                (RWorkspace: 0)
*
                 CALL CGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy L to WORK(IU), zeroing out above it
```

```
*
                    CALL CLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                           LDWRKU )
                    CALL CLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                           WORK( IU+LDWRKU ), LDWRKU )
*
*                   Generate Q in A
*                   (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*                   (RWorkspace: 0)
*
                    CALL CUNGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = 1
                    ITAUQ = ITAU
                    ITAUP = ITAUQ + M
                    IWORK = ITAUP + M
*
*                   Bidiagonalize L in WORK(IU), copying result to U
*                   (CWorkspace: need M*M+3*M, prefer M*M+2*M+2*M*NB)
*                   (RWorkspace: need M)
*
                    CALL CGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                           RWORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    CALL CLACPY( 'L', M, M, WORK( IU ), LDWRKU, U,
     $                           LDU )
*
*                   Generate right bidiagonalizing vectors in WORK(IU)
*                   (CWorkspace: need   M*M+3*M-1,
*                                prefer M*M+2*M+(M-1)*NB)
*                   (RWorkspace: 0)
*
                    CALL CUNGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate left bidiagonalizing vectors in U
*                   (CWorkspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*                   (RWorkspace: 0)
*
                    CALL CUNGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IRWORK = IE + M
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of L in U and computing right
*                   singular vectors of L in WORK(IU)
*                   (CWorkspace: need M*M)
*                   (RWorkspace: need BDSPAC)
```

```
*
                 CALL CBDSQR( 'U', M, M, M, 0, S, RWORK( IE ),
     $                        WORK( IU ), LDWRKU, U, LDU, CDUM, 1,
     $                        RWORK( IRWORK ), INFO )
*
*                Multiply right singular vectors of L in WORK(IU) by
*                Q in A, storing result in VT
*                (CWorkspace: need M*M)
*                (RWorkspace: 0)
*
                 CALL CGEMM( 'N', 'N', M, N, M, CONE, WORK( IU ),
     $                       LDWRKU, A, LDA, CZERO, VT, LDVT )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + M
*
*                Compute A=L*Q, copying result to VT
*                (CWorkspace: need 2*M, prefer M+M*NB)
*                (RWorkspace: 0)
*
                 CALL CGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL CLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                Generate Q in VT
*                (CWorkspace: need 2*M, prefer M+M*NB)
*                (RWorkspace: 0)
*
                 CALL CUNGLQ( M, N, M, VT, LDVT, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy L to U, zeroing out above it
*
                 CALL CLACPY( 'L', M, M, A, LDA, U, LDU )
                 CALL CLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                        U( 1, 2 ), LDU )
                 IE = 1
                 ITAUQ = ITAU
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Bidiagonalize L in U
*                (CWorkspace: need 3*M, prefer 2*M+2*M*NB)
*                (RWorkspace: need M)
*
                 CALL CGEBRD( M, M, U, LDU, S, RWORK( IE ),
```

```
     $                          WORK( ITAUQ ), WORK( ITAUP ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply right bidiagonalizing vectors in U by Q
*                    in VT
*                    (CWorkspace: need 2*M+N, prefer 2*M+N*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNMBR( 'P', 'L', 'C', M, N, M, U, LDU,
     $                          WORK( ITAUP ), VT, LDVT,
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors in U
*                    (CWorkspace: need 3*M, prefer 2*M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IRWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in VT
*                    (CWorkspace: 0)
*                    (RWorkspace: need BDSPAC)
*
                     CALL CBDSQR( 'U', M, N, M, 0, S, RWORK( IE ), VT,
     $                          LDVT, U, LDU, CDUM, 1,
     $                          RWORK( IRWORK ), INFO )
*
                  END IF
*
               END IF
*
            ELSE IF( WNTVA ) THEN
*
               IF( WNTUN ) THEN
*
*                 Path 7t(N much larger than M, JOBU='N', JOBVT='A')
*                 N right singular vectors to be computed in VT and
*                 no left singular vectors to be computed
*
                  IF( LWORK.GE.M*M+MAX( N+M, 3*M ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IR = 1
                     IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                       WORK(IR) is LDA by M
```

```
*
                      LDWRKR = LDA
                   ELSE
*
*                     WORK(IR) is M by M
*
                      LDWRKR = M
                   END IF
                   ITAU = IR + LDWRKR*M
                   IWORK = ITAU + M
*
*                  Compute A=L*Q, copying result to VT
*                  (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*                  (RWorkspace: 0)
*
                   CALL CGELQF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                   CALL CLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                  Copy L to WORK(IR), zeroing out above it
*
                   CALL CLACPY( 'L', M, M, A, LDA, WORK( IR ),
     $                           LDWRKR )
                   CALL CLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                           WORK( IR+LDWRKR ), LDWRKR )
*
*                  Generate Q in VT
*                  (CWorkspace: need M*M+M+N, prefer M*M+M+N*NB)
*                  (RWorkspace: 0)
*
                   CALL CUNGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                   IE = 1
                   ITAUQ = ITAU
                   ITAUP = ITAUQ + M
                   IWORK = ITAUP + M
*
*                  Bidiagonalize L in WORK(IR)
*                  (CWorkspace: need M*M+3*M, prefer M*M+2*M+2*M*NB)
*                  (RWorkspace: need M)
*
                   CALL CGEBRD( M, M, WORK( IR ), LDWRKR, S,
     $                           RWORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                  Generate right bidiagonalizing vectors in WORK(IR)
*                  (CWorkspace: need   M*M+3*M-1,
*                               prefer M*M+2*M+(M-1)*NB)
*                  (RWorkspace: 0)
```

```
*
                 CALL CUNGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                        WORK( ITAUP ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
                 IRWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing right
*                singular vectors of L in WORK(IR)
*                (CWorkspace: need M*M)
*                (RWorkspace: need BDSPAC)
*
                 CALL CBDSQR( 'U', M, M, 0, 0, S, RWORK( IE ),
     $                        WORK( IR ), LDWRKR, CDUM, 1, CDUM, 1,
     $                        RWORK( IRWORK ), INFO )
*
*                Multiply right singular vectors of L in WORK(IR) by
*                Q in VT, storing result in A
*                (CWorkspace: need M*M)
*                (RWorkspace: 0)
*
                 CALL CGEMM( 'N', 'N', M, N, M, CONE, WORK( IR ),
     $                       LDWRKR, VT, LDVT, CZERO, A, LDA )
*
*                Copy right singular vectors of A from A to VT
*
                 CALL CLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + M
*
*                Compute A=L*Q, copying result to VT
*                (CWorkspace: need 2*M, prefer M+M*NB)
*                (RWorkspace: 0)
*
                 CALL CGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL CLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                Generate Q in VT
*                (CWorkspace: need M+N, prefer M+N*NB)
*                (RWorkspace: 0)
*
                 CALL CUNGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = 1
                 ITAUQ = ITAU
```

```
                    ITAUP = ITAUQ + M
                    IWORK = ITAUP + M
*
*                   Zero out above L in A
*
                    CALL CLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                           A( 1, 2 ), LDA )
*
*                   Bidiagonalize L in A
*                   (CWorkspace: need 3*M, prefer 2*M+2*M*NB)
*                   (RWorkspace: need M)
*
                    CALL CGEBRD( M, M, A, LDA, S, RWORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Multiply right bidiagonalizing vectors in A by Q
*                   in VT
*                   (CWorkspace: need 2*M+N, prefer 2*M+N*NB)
*                   (RWorkspace: 0)
*
                    CALL CUNMBR( 'P', 'L', 'C', M, N, M, A, LDA,
     $                           WORK( ITAUP ), VT, LDVT,
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IRWORK = IE + M
*
*                   Perform bidiagonal QR iteration, computing right
*                   singular vectors of A in VT
*                   (CWorkspace: 0)
*                   (RWorkspace: need BDSPAC)
*
                    CALL CBDSQR( 'U', M, N, 0, 0, S, RWORK( IE ), VT,
     $                           LDVT, CDUM, 1, CDUM, 1,
     $                           RWORK( IRWORK ), INFO )
*
                 END IF
*
              ELSE IF( WNTUO ) THEN
*
*                Path 8t(N much larger than M, JOBU='O', JOBVT='A')
*                N right singular vectors to be computed in VT and
*                M left singular vectors to be overwritten on A
*
                 IF( LWORK.GE.2*M*M+MAX( N+M, 3*M ) ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IU = 1
                    IF( LWORK.GE.WRKBL+2*LDA*M ) THEN
*
```

```
*                       WORK(IU) is LDA by M and WORK(IR) is LDA by M
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*M
                        LDWRKR = LDA
                     ELSE IF( LWORK.GE.WRKBL+( LDA+M )*M ) THEN
*
*                       WORK(IU) is LDA by M and WORK(IR) is M by M
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*M
                        LDWRKR = M
                     ELSE
*
*                       WORK(IU) is M by M and WORK(IR) is M by M
*
                        LDWRKU = M
                        IR = IU + LDWRKU*M
                        LDWRKR = M
                     END IF
                     ITAU = IR + LDWRKR*M
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (CWorkspace: need 2*M*M+2*M, prefer 2*M*M+M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL CGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL CLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (CWorkspace: need 2*M*M+M+N, prefer 2*M*M+M+N*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to WORK(IU), zeroing out above it
*
                     CALL CLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                            LDWRKU )
                     CALL CLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                            WORK( IU+LDWRKU ), LDWRKU )
                     IE = 1
                     ITAUQ = ITAU
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Bidiagonalize L in WORK(IU), copying result to
```

```
*                    WORK(IR)
*                    (CWorkspace: need   2*M*M+3*M,
*                                 prefer 2*M*M+2*M+2*M*NB)
*                    (RWorkspace: need   M)
*
                     CALL CGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                            RWORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     CALL CLACPY( 'L', M, M, WORK( IU ), LDWRKU,
     $                            WORK( IR ), LDWRKR )
*
*                    Generate right bidiagonalizing vectors in WORK(IU)
*                    (CWorkspace: need   2*M*M+3*M-1,
*                                 prefer 2*M*M+2*M+(M-1)*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors in WORK(IR)
*                    (CWorkspace: need 2*M*M+3*M, prefer 2*M*M+2*M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGBR( 'Q', M, M, M, WORK( IR ), LDWRKR,
     $                            WORK( ITAUQ ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IRWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of L in WORK(IR) and computing
*                    right singular vectors of L in WORK(IU)
*                    (CWorkspace: need 2*M*M)
*                    (RWorkspace: need BDSPAC)
*
                     CALL CBDSQR( 'U', M, M, M, 0, S, RWORK( IE ),
     $                            WORK( IU ), LDWRKU, WORK( IR ),
     $                            LDWRKR, CDUM, 1, RWORK( IRWORK ),
     $                            INFO )
*
*                    Multiply right singular vectors of L in WORK(IU) by
*                    Q in VT, storing result in A
*                    (CWorkspace: need M*M)
*                    (RWorkspace: 0)
*
                     CALL CGEMM( 'N', 'N', M, N, M, CONE, WORK( IU ),
     $                           LDWRKU, VT, LDVT, CZERO, A, LDA )
*
*                    Copy right singular vectors of A from A to VT
```

```
*
                 CALL CLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
*                Copy left singular vectors of A from WORK(IR) to A
*
                 CALL CLACPY( 'F', M, M, WORK( IR ), LDWRKR, A,
     $                        LDA )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + M
*
*                Compute A=L*Q, copying result to VT
*                (CWorkspace: need 2*M, prefer M+M*NB)
*                (RWorkspace: 0)
*
                 CALL CGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL CLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                Generate Q in VT
*                (CWorkspace: need M+N, prefer M+N*NB)
*                (RWorkspace: 0)
*
                 CALL CUNGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = 1
                 ITAUQ = ITAU
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Zero out above L in A
*
                 CALL CLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                        A( 1, 2 ), LDA )
*
*                Bidiagonalize L in A
*                (CWorkspace: need 3*M, prefer 2*M+2*M*NB)
*                (RWorkspace: need M)
*
                 CALL CGEBRD( M, M, A, LDA, S, RWORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Multiply right bidiagonalizing vectors in A by Q
*                in VT
*                (CWorkspace: need 2*M+N, prefer 2*M+N*NB)
```

```
*                    (RWorkspace: 0)
*
                     CALL CUNMBR( 'P', 'L', 'C', M, N, M, A, LDA,
     $                            WORK( ITAUP ), VT, LDVT,
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors in A
*                    (CWorkspace: need 3*M, prefer 2*M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL CUNGBR( 'Q', M, M, M, A, LDA, WORK( ITAUQ ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IRWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in A and computing right
*                    singular vectors of A in VT
*                    (CWorkspace: 0)
*                    (RWorkspace: need BDSPAC)
*
                     CALL CBDSQR( 'U', M, N, M, 0, S, RWORK( IE ), VT,
     $                            LDVT, A, LDA, CDUM, 1,
     $                            RWORK( IRWORK ), INFO )
*
                  END IF
*
               ELSE IF( WNTUAS ) THEN
*
                  Path 9t(N much larger than M, JOBU='S' or 'A',
*                         JOBVT='A')
                  N right singular vectors to be computed in VT and
                  M left singular vectors to be computed in U
*
                  IF( LWORK.GE.M*M+MAX( N+M, 3*M ) ) THEN
*
                     Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
                        WORK(IU) is LDA by M
*
                        LDWRKU = LDA
                     ELSE
*
                        WORK(IU) is M by M
*
                        LDWRKU = M
                     END IF
                     ITAU = IU + LDWRKU*M
```

```
                    IWORK = ITAU + M
*
*                   Compute A=L*Q, copying result to VT
*                   (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*                   (RWorkspace: 0)
*
                    CALL CGELQF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    CALL CLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                   Generate Q in VT
*                   (CWorkspace: need M*M+M+N, prefer M*M+M+N*NB)
*                   (RWorkspace: 0)
*
                    CALL CUNGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy L to WORK(IU), zeroing out above it
*
                    CALL CLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                           LDWRKU )
                    CALL CLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                           WORK( IU+LDWRKU ), LDWRKU )
                    IE = 1
                    ITAUQ = ITAU
                    ITAUP = ITAUQ + M
                    IWORK = ITAUP + M
*
*                   Bidiagonalize L in WORK(IU), copying result to U
*                   (CWorkspace: need M*M+3*M, prefer M*M+2*M+2*M*NB)
*                   (RWorkspace: need M)
*
                    CALL CGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                           RWORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    CALL CLACPY( 'L', M, M, WORK( IU ), LDWRKU, U,
     $                           LDU )
*
*                   Generate right bidiagonalizing vectors in WORK(IU)
*                   (CWorkspace: need M*M+3*M, prefer M*M+2*M+(M-1)*NB)
*                   (RWorkspace: 0)
*
                    CALL CUNGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate left bidiagonalizing vectors in U
*                   (CWorkspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*                   (RWorkspace: 0)
```

```
*
                 CALL CUNGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IRWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of L in U and computing right
*                singular vectors of L in WORK(IU)
*                (CWorkspace: need M*M)
*                (RWorkspace: need BDSPAC)
*
                 CALL CBDSQR( 'U', M, M, M, 0, S, RWORK( IE ),
     $                        WORK( IU ), LDWRKU, U, LDU, CDUM, 1,
     $                        RWORK( IRWORK ), INFO )
*
*                Multiply right singular vectors of L in WORK(IU) by
*                Q in VT, storing result in A
*                (CWorkspace: need M*M)
*                (RWorkspace: 0)
*
                 CALL CGEMM( 'N', 'N', M, N, M, CONE, WORK( IU ),
     $                       LDWRKU, VT, LDVT, CZERO, A, LDA )
*
*                Copy right singular vectors of A from A to VT
*
                 CALL CLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + M
*
*                Compute A=L*Q, copying result to VT
*                (CWorkspace: need 2*M, prefer M+M*NB)
*                (RWorkspace: 0)
*
                 CALL CGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL CLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                Generate Q in VT
*                (CWorkspace: need M+N, prefer M+N*NB)
*                (RWorkspace: 0)
*
                 CALL CUNGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy L to U, zeroing out above it
```

```
*
                    CALL CLACPY( 'L', M, M, A, LDA, U, LDU )
                    CALL CLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                           U( 1, 2 ), LDU )
                    IE = 1
                    ITAUQ = ITAU
                    ITAUP = ITAUQ + M
                    IWORK = ITAUP + M
*
*                   Bidiagonalize L in U
*                   (CWorkspace: need 3*M, prefer 2*M+2*M*NB)
*                   (RWorkspace: need M)
*
                    CALL CGEBRD( M, M, U, LDU, S, RWORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Multiply right bidiagonalizing vectors in U by Q
*                   in VT
*                   (CWorkspace: need 2*M+N, prefer 2*M+N*NB)
*                   (RWorkspace: 0)
*
                    CALL CUNMBR( 'P', 'L', 'C', M, N, M, U, LDU,
     $                           WORK( ITAUP ), VT, LDVT,
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Generate left bidiagonalizing vectors in U
*                   (CWorkspace: need 3*M, prefer 2*M+M*NB)
*                   (RWorkspace: 0)
*
                    CALL CUNGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IRWORK = IE + M
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of A in U and computing right
*                   singular vectors of A in VT
*                   (CWorkspace: 0)
*                   (RWorkspace: need BDSPAC)
*
                    CALL CBDSQR( 'U', M, N, M, 0, S, RWORK( IE ), VT,
     $                           LDVT, U, LDU, CDUM, 1,
     $                           RWORK( IRWORK ), INFO )
*
                 END IF
*
              END IF
*
           END IF
*
```

```
         ELSE
*
*           N .LT. MNTHR
*
*           Path 10t(N greater than M, but not much larger)
*           Reduce to bidiagonal form without LQ decomposition
*
            IE = 1
            ITAUQ = 1
            ITAUP = ITAUQ + M
            IWORK = ITAUP + M
*
*           Bidiagonalize A
*           (CWorkspace: need 2*M+N, prefer 2*M+(M+N)*NB)
*           (RWorkspace: M)
*
            CALL CGEBRD( M, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                   WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                   IERR )
            IF( WNTUAS ) THEN
*
*              If left singular vectors desired in U, copy result to U
*              and generate left bidiagonalizing vectors in U
*              (CWorkspace: need 3*M-1, prefer 2*M+(M-1)*NB)
*              (RWorkspace: 0)
*
               CALL CLACPY( 'L', M, M, A, LDA, U, LDU )
               CALL CUNGBR( 'Q', M, M, N, U, LDU, WORK( ITAUQ ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IF( WNTVAS ) THEN
*
*              If right singular vectors desired in VT, copy result to
*              VT and generate right bidiagonalizing vectors in VT
*              (CWorkspace: need 2*M+NRVT, prefer 2*M+NRVT*NB)
*              (RWorkspace: 0)
*
               CALL CLACPY( 'U', M, N, A, LDA, VT, LDVT )
               IF( WNTVA )
     $            NRVT = N
               IF( WNTVS )
     $            NRVT = M
               CALL CUNGBR( 'P', NRVT, N, M, VT, LDVT, WORK( ITAUP ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IF( WNTUO ) THEN
*
*              If left singular vectors desired in A, generate left
*              bidiagonalizing vectors in A
*              (CWorkspace: need 3*M-1, prefer 2*M+(M-1)*NB)
```

```
*              (RWorkspace: 0)
*
               CALL CUNGBR( 'Q', M, M, N, A, LDA, WORK( ITAUQ ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IF( WNTVO ) THEN
*
*              If right singular vectors desired in A, generate right
*              bidiagonalizing vectors in A
*              (CWorkspace: need 3*M, prefer 2*M+M*NB)
*              (RWorkspace: 0)
*
               CALL CUNGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IRWORK = IE + M
            IF( WNTUAS .OR. WNTUO )
     $         NRU = M
            IF( WNTUN )
     $         NRU = 0
            IF( WNTVAS .OR. WNTVO )
     $         NCVT = N
            IF( WNTVN )
     $         NCVT = 0
            IF( ( .NOT.WNTUO ) .AND. ( .NOT.WNTVO ) ) THEN
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in U and computing right singular
*              vectors in VT
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAC)
*
               CALL CBDSQR( 'L', M, NCVT, NRU, 0, S, RWORK( IE ), VT,
     $                      LDVT, U, LDU, CDUM, 1, RWORK( IRWORK ),
     $                      INFO )
            ELSE IF( ( .NOT.WNTUO ) .AND. WNTVO ) THEN
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in U and computing right singular
*              vectors in A
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAC)
*
               CALL CBDSQR( 'L', M, NCVT, NRU, 0, S, RWORK( IE ), A,
     $                      LDA, U, LDU, CDUM, 1, RWORK( IRWORK ),
     $                      INFO )
            ELSE
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in A and computing right singular
```

```
*              vectors in VT
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAC)
*
               CALL CBDSQR( 'L', M, NCVT, NRU, 0, S, RWORK( IE ), VT,
     $                      LDVT, A, LDA, CDUM, 1, RWORK( IRWORK ),
     $                      INFO )
            END IF
*
         END IF
*
      END IF
*
*     Undo scaling if necessary
*
      IF( ISCL.EQ.1 ) THEN
         IF( ANRM.GT.BIGNUM )
     $      CALL SLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( INFO.NE.0 .AND. ANRM.GT.BIGNUM )
     $      CALL SLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN-1, 1,
     $                   RWORK( IE ), MINMN, IERR )
         IF( ANRM.LT.SMLNUM )
     $      CALL SLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( INFO.NE.0 .AND. ANRM.LT.SMLNUM )
     $      CALL SLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN-1, 1,
     $                   RWORK( IE ), MINMN, IERR )
      END IF
*
*     Return optimal workspace in WORK(1)
*
      WORK( 1 ) = MAXWRK
*
      RETURN
*
*     End of CGESVD
*
      END
```

———-

30.3.4 lisp code

— cgesvd.lisp —

```
(let* ((czero (f2cl-lib:cmplx 0.0 0.0))
       (cone (f2cl-lib:cmplx 1.0 0.0))
```

```
      (zero 0.0)
      (one 1.0))
  (declare (type (f2cl-lib:complex8) czero)
           (type (f2cl-lib:complex8) cone)
           (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable czero cone zero one))
  (defun cgesvd (jobu jobvt m n a lda s u ldu vt ldvt work lwork rwork info)
    (declare (type (array single-float (*)) rwork s)
             (type (array f2cl-lib:complex8 (*)) work vt u a)
             (type (f2cl-lib:integer4) info lwork ldvt ldu lda n m)
             (type (string *) jobvt jobu))
    (f2cl-lib:with-multi-array-data
        ((jobu character jobu-%data% jobu-%offset%)
         (jobvt character jobvt-%data% jobvt-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (u f2cl-lib:complex8 u-%data% u-%offset%)
         (vt f2cl-lib:complex8 vt-%data% vt-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (s single-float s-%data% s-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%))
      (prog ((cdum (make-array 1 :element-type 'f2cl-lib:complex8))
             (dum (make-array 1 :element-type 'single-float)) (anrm 0.0)
             (bignum 0.0) (eps 0.0) (smlnum 0.0) (lwork_cgeqrf 0)
             (lwork_cungqr_n 0) (lwork_cungqr_m 0) (lwork_cgebrd 0)
             (lwork_cungbr_p 0) (lwork_cungbr_q 0) (lwork_cgelqf 0)
             (lwork_cunglq_n 0) (lwork_cunglq_m 0) (blk 0) (chunk 0) (i 0)
             (ie 0) (ierr 0) (ir 0) (irwork 0) (iscl 0) (itau 0) (itaup 0)
             (itauq 0) (iu 0) (iwork 0) (ldwrkr 0) (ldwrku 0) (maxwrk 0)
             (minmn 0) (minwrk 0) (mnthr 0) (ncu 0) (ncvt 0) (nru 0) (nrvt 0)
             (wrkbl 0) (lquery nil) (wntua nil) (wntuas nil) (wntun nil)
             (wntuo nil) (wntus nil) (wntva nil) (wntvas nil) (wntvn nil)
             (wntvo nil) (wntvs nil))
        (declare (type (array f2cl-lib:complex8 (1)) cdum)
                 (type (array single-float (1)) dum)
                 (type (single-float) anrm bignum eps smlnum)
                 (type (f2cl-lib:integer4) lwork_cgeqrf lwork_cungqr_n
                  lwork_cungqr_m lwork_cgebrd lwork_cungbr_p lwork_cungbr_q
                  lwork_cgelqf lwork_cunglq_n lwork_cunglq_m blk chunk i ie
                  ierr ir irwork iscl itau itaup itauq iu iwork ldwrkr ldwrku
                  maxwrk minmn minwrk mnthr ncu ncvt nru nrvt wrkbl)
                 (type f2cl-lib:logical lquery wntua wntuas wntun wntuo wntus
                  wntva wntvas wntvn wntvo wntvs))
        (setf info 0)
        (setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (setf wntua
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame jobu "A")
                  (declare (ignore var-1))
                  (when var-0 (setf jobu var-0))
```

```
          ret-val))
(setf wntus
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobu "S")
          (declare (ignore var-1))
          (when var-0 (setf jobu var-0))
          ret-val))
(setf wntuas (or wntua wntus))
(setf wntuo
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobu "O")
          (declare (ignore var-1))
          (when var-0 (setf jobu var-0))
          ret-val))
(setf wntun
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobu "N")
          (declare (ignore var-1))
          (when var-0 (setf jobu var-0))
          ret-val))
(setf wntva
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvt "A")
          (declare (ignore var-1))
          (when var-0 (setf jobvt var-0))
          ret-val))
(setf wntvs
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvt "S")
          (declare (ignore var-1))
          (when var-0 (setf jobvt var-0))
          ret-val))
(setf wntvas (or wntva wntvs))
(setf wntvo
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvt "O")
          (declare (ignore var-1))
          (when var-0 (setf jobvt var-0))
          ret-val))
(setf wntvn
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvt "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvt var-0))
          ret-val))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((not (or wntua wntus wntuo wntun)) (setf info -1))
      ((or (not (or wntva wntvs wntvo wntvn)) (and wntvo wntuo))
       (setf info -2))
      ((< m 0) (setf info -3)) ((< n 0) (setf info -4))
```

```
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -6))
      ((or (< ldu 1) (and wntuas (< ldu m))) (setf info -9))
      ((or (< ldvt 1) (and wntva (< ldvt n))
           (and wntvs (< ldvt minmn)))
       (setf info -11)))
(cond
 ((= info 0) (setf minwrk 1) (setf maxwrk 1)
  (cond
   ((and (>= m n) (> minmn 0))
    (setf mnthr
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 6 "CGESVD" (f2cl-lib:f2cl-// jobu jobvt) m n 0
                 0)
              (declare (ignore var-0 var-1 var-2 var-5 var-6))
              (when var-3 (setf m var-3))
              (when var-4 (setf n var-4))
              ret-val))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (cgeqrf m n a lda
         (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
         (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1))) -1
         ierr)
      (declare (ignore var-2 var-4 var-5 var-6))
      (setf m var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf ierr var-7))
    (setf lwork_cgeqrf (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (cungqr m n n a lda
         (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
         (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1))) -1
         ierr)
      (declare (ignore var-3 var-5 var-6 var-7))
      (when var-0 (setf m var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-8 (setf ierr var-8)))
    (setf lwork_cungqr_n
            (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (cungqr m m n a lda
         (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
```

```
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1))) -1
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf lwork_cungqr_m
        (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgebrd n n a lda s
     (f2cl-lib:array-slice dum single-float (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1))) -1
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(setf lwork_cgebrd (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (cungbr "P" n n n a lda
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1))) -1
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf ierr var-9)))
(setf lwork_cungbr_p
        (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (cungbr "Q" n n n a lda
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1))) -1
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf lda var-5))
```

```
  (when var-9 (setf ierr var-9)))
(setf lwork_cungbr_q
        (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
(setf mnthr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 6 "CGESVD" (f2cl-lib:f2cl-// jobu jobvt) m n 0
             0)
          (declare (ignore var-0 var-1 var-2 var-5 var-6))
          (when var-3 (setf m var-3))
          (when var-4 (setf n var-4))
          ret-val))
(cond
 ((>= m mnthr)
  (cond
   (wntun (setf maxwrk (f2cl-lib:int-add n lwork_cgeqrf))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        lwork_cgebrd))))
    (if (or wntvo wntvas)
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                            lwork_cungbr_p)))))
    (setf minwrk (f2cl-lib:int-mul 3 n)))
   ((and wntuo wntvn)
    (setf wrkbl (f2cl-lib:int-add n lwork_cgeqrf))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add n lwork_cungqr_n))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        lwork_cgebrd))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        lwork_cungbr_q))))
    (setf maxwrk
            (max
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul n n)
```

```
                                 (f2cl-lib:int-mul m n)))))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))
((and wntuo wntvas)
 (setf wrkbl (f2cl-lib:int-add n lwork_cgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_cungqr_n))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_cgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_cungbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_cungbr_p))))
 (setf maxwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                                 (f2cl-lib:int-mul m n)))))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))
((and wntus wntvn)
 (setf wrkbl (f2cl-lib:int-add n lwork_cgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_cungqr_n))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_cgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_cungbr_q))))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))
((and wntus wntvo)
```

```
 (setf wrkbl (f2cl-lib:int-add n lwork_cgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_cungqr_n))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_cgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_cungbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_cungbr_p))))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n n) wrkbl))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))
((and wntus wntvas)
 (setf wrkbl (f2cl-lib:int-add n lwork_cgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_cungqr_n))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_cgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_cungbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_cungbr_p))))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))
((and wntua wntvn)
 (setf wrkbl (f2cl-lib:int-add n lwork_cgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
```

```
                    (f2cl-lib:int-add n lwork_cungqr_m))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_cgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_cungbr_q))))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))
((and wntua wntvo)
 (setf wrkbl (f2cl-lib:int-add n lwork_cgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_cungqr_m))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_cgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_cungbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_cungbr_p))))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n n) wrkbl))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))
((and wntua wntvas)
 (setf wrkbl (f2cl-lib:int-add n lwork_cgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_cungqr_m))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_cgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
```

```
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                      lwork_cungbr_q))))
  (setf wrkbl
          (max (the f2cl-lib:integer4 wrkbl)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                      lwork_cungbr_p))))
  (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
  (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (cgebrd m n a lda s
      (f2cl-lib:array-slice dum single-float (1) ((1 1)))
      (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
      (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
      (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1))) -1
      ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
 (setf lwork_cgebrd
         (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
 (setf maxwrk
         (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) lwork_cgebrd))
 (cond
  ((or wntus wntuo)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9)
       (cungbr "Q" m n n a lda
        (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
        (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
        -1 ierr)
     (declare (ignore var-0 var-4 var-6 var-7 var-8))
     (when var-1 (setf m var-1))
     (when var-2 (setf n var-2))
     (when var-3 (setf n var-3))
     (when var-5 (setf lda var-5))
     (when var-9 (setf ierr var-9)))
   (setf lwork_cungbr_q
           (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
   (setf maxwrk
           (max (the f2cl-lib:integer4 maxwrk)
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                       lwork_cungbr_q))))))
```

```
    (cond
     (wntua
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9)
          (cungbr "Q" m m n a lda
           (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
           (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
           -1 ierr)
        (declare (ignore var-0 var-4 var-6 var-7 var-8))
        (when var-1 (setf m var-1))
        (when var-2 (setf m var-2))
        (when var-3 (setf n var-3))
        (when var-5 (setf lda var-5))
        (when var-9 (setf ierr var-9)))
      (setf lwork_cungbr_q
              (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
      (setf maxwrk
              (max (the f2cl-lib:integer4 maxwrk)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                          lwork_cungbr_q))))))
    (cond
     ((not wntvn)
      (setf maxwrk
              (max (the f2cl-lib:integer4 maxwrk)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                          lwork_cungbr_p))))
      (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))))))
((> minmn 0)
 (setf mnthr
         (multiple-value-bind
             (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
             (ilaenv 6 "CGESVD" (f2cl-lib:f2cl-// jobu jobvt) m n 0
              0)
           (declare (ignore var-0 var-1 var-2 var-5 var-6))
           (when var-3 (setf m var-3))
           (when var-4 (setf n var-4))
           ret-val))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (cgelqf m n a lda
      (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
      (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1))) -1
      ierr)
   (declare (ignore var-2 var-4 var-5 var-6))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
```

```
  (setf ierr var-7))
(setf lwork_cgelqf (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cunglq n n m
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1))) n
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1))) -1
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf n var-4))
  (when var-8 (setf ierr var-8)))
(setf lwork_cunglq_n
        (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cunglq m n m a lda
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1))) -1
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf lwork_cunglq_m
        (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgebrd m m a lda s
     (f2cl-lib:array-slice dum single-float (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1))) -1
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(setf lwork_cgebrd (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (cungbr "P" m m m a n
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
```

```
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1))) -1
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf n var-5))
  (when var-9 (setf ierr var-9)))
(setf lwork_cungbr_p
        (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (cungbr "Q" m m m a n
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1))) -1
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf n var-5))
  (when var-9 (setf ierr var-9)))
(setf lwork_cungbr_q
        (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
(cond
 ((>= n mnthr)
  (cond
   (wntvn (setf maxwrk (f2cl-lib:int-add m lwork_cgelqf))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                        lwork_cgebrd))))
    (if (or wntuo wntuas)
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                            lwork_cungbr_q)))))
    (setf minwrk (f2cl-lib:int-mul 3 m)))
   ((and wntvo wntun)
    (setf wrkbl (f2cl-lib:int-add m lwork_cgelqf))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add m lwork_cunglq_m))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
```

```
                                        lwork_cgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                     lwork_cungbr_p))))
 (setf maxwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                 (f2cl-lib:int-mul m n)))))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))
((and wntvo wntuas)
 (setf wrkbl (f2cl-lib:int-add m lwork_cgelqf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add m lwork_cunglq_m))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                     lwork_cgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                     lwork_cungbr_p))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                     lwork_cungbr_q))))
 (setf maxwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                 (f2cl-lib:int-mul m n)))))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))
((and wntvs wntun)
 (setf wrkbl (f2cl-lib:int-add m lwork_cgelqf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add m lwork_cunglq_m))))
 (setf wrkbl
```

```
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    lwork_cgebrd))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    lwork_cungbr_p))))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))
((and wntvs wntuo)
 (setf wrkbl (f2cl-lib:int-add m lwork_cgelqf))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add m lwork_cunglq_m))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    lwork_cgebrd))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    lwork_cungbr_p))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    lwork_cungbr_q))))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m m) wrkbl))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))
((and wntvs wntuas)
 (setf wrkbl (f2cl-lib:int-add m lwork_cgelqf))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add m lwork_cunglq_m))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    lwork_cgebrd))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    lwork_cungbr_p))))
```

```
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                     lwork_cungbr_q))))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))
((and wntva wntun)
 (setf wrkbl (f2cl-lib:int-add m lwork_cgelqf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add m lwork_cunglq_n))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                     lwork_cgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                     lwork_cungbr_p))))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))
((and wntva wntuo)
 (setf wrkbl (f2cl-lib:int-add m lwork_cgelqf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add m lwork_cunglq_n))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                     lwork_cgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                     lwork_cungbr_p))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                     lwork_cungbr_q))))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m m) wrkbl))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))
((and wntva wntuas)
 (setf wrkbl (f2cl-lib:int-add m lwork_cgelqf))
```

```
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add m lwork_cunglq_n))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                        lwork_cgebrd))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                        lwork_cungbr_p))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                        lwork_cungbr_q))))
    (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
    (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (cgebrd m n a lda s
        (f2cl-lib:array-slice dum single-float (1) ((1 1)))
        (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
        (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
        (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1))) -1
        ierr)
    (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-10))
   (setf lwork_cgebrd
           (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
   (setf maxwrk
           (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) lwork_cgebrd))
   (cond
    ((or wntvs wntvo)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
          var-9)
         (cungbr "P" m n m a n
          (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
          (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
          -1 ierr)
       (declare (ignore var-0 var-4 var-6 var-7 var-8))
```

```
            (when var-1 (setf m var-1))
            (when var-2 (setf n var-2))
            (when var-3 (setf m var-3))
            (when var-5 (setf n var-5))
            (when var-9 (setf ierr var-9)))
          (setf lwork_cungbr_p
                  (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                              lwork_cungbr_p))))))
        (cond
         (wntva
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9)
              (cungbr "P" n n m a n
               (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
               (f2cl-lib:array-slice cdum f2cl-lib:complex8 (1) ((1 1)))
               -1 ierr)
            (declare (ignore var-0 var-4 var-6 var-7 var-8))
            (when var-1 (setf n var-1))
            (when var-2 (setf n var-2))
            (when var-3 (setf m var-3))
            (when var-5 (setf n var-5))
            (when var-9 (setf ierr var-9)))
          (setf lwork_cungbr_p
                  (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                              lwork_cungbr_p))))))
        (cond
         ((not wntun)
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                              lwork_cungbr_q))))
          (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n))))))))
    (setf maxwrk
            (max (the f2cl-lib:integer4 minwrk)
                 (the f2cl-lib:integer4 maxwrk)))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce maxwrk 'f2cl-lib:complex8))
    (cond ((and (< lwork minwrk) (not lquery)) (setf info -13)))))
  (cond
   ((/= info 0) (xerbla "CGESVD" (f2cl-lib:int-sub info)) (go end_label))
```

```
 (lquery (go end_label)))
(cond ((or (= m 0) (= n 0)) (go end_label)))
(setf eps (slamch "P"))
(setf smlnum (/ (f2cl-lib:fsqrt (slamch "S")) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "M" m n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iscl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf iscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 anrm smlnum m n a lda ierr)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf ierr var-9))))
 ((> anrm bignum) (setf iscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 anrm bignum m n a lda ierr)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf ierr var-9)))))
(cond
 ((>= m n)
  (cond
   ((>= m mnthr)
    (cond
     (wntun (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (cgeqrf m n a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
```

```
                          ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (2 1)
                          ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(setf ie 1) (setf itauq 1)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgebrd n n a lda s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                          rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                          ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(setf ncvt 0)
(cond
 ((or wntvo wntvas)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (cungbr "P" n n n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                            (itaup) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                            (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
```

```
      (declare (ignore var-0 var-4 var-6 var-7 var-8))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf n var-3))
      (when var-5 (setf lda var-5))
      (when var-9 (setf ierr var-9)))
    (setf ncvt n)))
  (setf irwork (f2cl-lib:int-add ie n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (cbdsqr "U" n ncvt 0 0 s
       (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                             rwork-%offset%)
       a lda cdum 1 cdum 1
       (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                             ((1 *)) rwork-%offset%)
       info)
    (declare
     (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
      var-11 var-12 var-13))
    (setf n var-1)
    (setf ncvt var-2)
    (setf lda var-8)
    (setf info var-14))
  (if wntvas
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (clacpy "F" n n a lda vt ldvt)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldvt var-6)))))
 ((and wntuo wntvn)
  (cond
   ((>= lwork
        (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                          (f2cl-lib:int-mul 3 n)))
    (setf ir 1)
    (cond
     ((>= lwork
          (f2cl-lib:int-add
           (max (the f2cl-lib:integer4 wrkbl)
                (the f2cl-lib:integer4 (f2cl-lib:int-mul lda n)))
           (f2cl-lib:int-mul lda n)))
      (setf ldwrku lda) (setf ldwrkr lda))
     ((>= lwork
          (f2cl-lib:int-add
           (max (the f2cl-lib:integer4 wrkbl)
```

```
            (the f2cl-lib:integer4 (f2cl-lib:int-mul lda n)))
        (f2cl-lib:int-mul n n)))
  (setf ldwrku lda) (setf ldwrkr n))
 (t
  (setf ldwrku
          (the f2cl-lib:integer4
               (truncate (- lwork (* n n)) n)))
  (setf ldwrkr n)))
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
(setf iwork (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgeqrf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           ((+ ir 1)) ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cungqr m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
```

```
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "Q" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
```

```
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" n 0 n 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     cdum 1
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr cdum 1
     (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
    var-11 var-12 var-13))
  (setf n var-1)
  (setf n var-3)
  (setf ldwrkr var-10)
  (setf info var-14))
(setf iu itauq)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i ldwrku))
              ((> i m) nil)
  (tagbody
    (setf chunk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
             (the f2cl-lib:integer4 ldwrku)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (cgemm "N" "N" chunk n n cone
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                               (ir) ((1 *)) work-%offset%)
         ldwrkr czero
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                               (iu) ((1 *)) work-%offset%)
         ldwrku)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf chunk var-2))
      (when var-3 (setf n var-3))
      (when var-4 (setf n var-4))
      (when var-5 (setf cone var-5))
      (when var-7 (setf lda var-7))
```

```
          (when var-9 (setf ldwrkr var-9))
          (when var-10 (setf czero var-10))
          (when var-12 (setf ldwrku var-12)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (clacpy "F" chunk n
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                   (iu) ((1 *)) work-%offset%)
             ldwrku
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i 1)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf chunk var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf ldwrku var-4))
          (when var-6 (setf lda var-6)))
      label10)))
  (t (setf ie 1) (setf itauq 1)
   (setf itaup (f2cl-lib:int-add itauq n))
   (setf iwork (f2cl-lib:int-add itaup n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (cgebrd m n a lda s
        (f2cl-lib:array-slice rwork-%data% single-float (ie)
                              ((1 *)) rwork-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (itauq) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (itaup) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (iwork) ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-10))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9)
       (cungbr "Q" m n n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (itauq) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (iwork) ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
     (declare (ignore var-0 var-4 var-6 var-7 var-8))
     (when var-1 (setf m var-1))
```

```
      (when var-2 (setf n var-2))
      (when var-3 (setf n var-3))
      (when var-5 (setf lda var-5))
      (when var-9 (setf ierr var-9)))
    (setf irwork (f2cl-lib:int-add ie n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14)
        (cbdsqr "U" n 0 m 0 s
         (f2cl-lib:array-slice rwork-%data% single-float (ie)
                               ((1 *)) rwork-%offset%)
         cdum 1 a lda cdum 1
         (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                               ((1 *)) rwork-%offset%)
         info)
      (declare
       (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
        var-11 var-12 var-13))
      (setf n var-1)
      (setf m var-3)
      (setf lda var-10)
      (setf info var-14)))))
((and wntuo wntvas)
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                         (f2cl-lib:int-mul 3 n)))
   (setf ir 1)
   (cond
    ((>= lwork
         (f2cl-lib:int-add
          (max (the f2cl-lib:integer4 wrkbl)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul lda n)))
          (f2cl-lib:int-mul lda n)))
     (setf ldwrku lda) (setf ldwrkr lda))
    ((>= lwork
         (f2cl-lib:int-add
          (max (the f2cl-lib:integer4 wrkbl)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul lda n)))
          (f2cl-lib:int-mul n n)))
     (setf ldwrku lda) (setf ldwrkr n))
    (t
     (setf ldwrku
             (the f2cl-lib:integer4
                  (truncate (- lwork (* n n)) n)))
     (setf ldwrkr n)))
   (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
   (setf iwork (f2cl-lib:int-add itau n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
```

```
    (cgeqrf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "U" n n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(if (> n 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "L" (f2cl-lib:int-sub n 1)
         (f2cl-lib:int-sub n 1) czero czero
         (f2cl-lib:array-slice vt-%data% f2cl-lib:complex8
                               (2 1) ((1 ldvt) (1 *))
                               vt-%offset%)
         ldvt)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldvt var-6))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cungqr m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd n n vt ldvt s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldvt var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" n n vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldvt var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "Q" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "P" n n n vt ldvt
```

```
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                            (itaup) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                            (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" n n n 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr cdum 1
     (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf n var-1)
  (setf n var-2)
  (setf n var-3)
  (setf ldvt var-8)
  (setf ldwrkr var-10)
  (setf info var-14))
(setf iu itauq)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i ldwrku))
              ((> i m) nil)
  (tagbody
    (setf chunk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
             (the f2cl-lib:integer4 ldwrku)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (cgemm "N" "N" chunk n n cone
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
```

```
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                  (ir) ((1 *)) work-%offset%)
            ldwrkr czero
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                  (iu) ((1 *)) work-%offset%)
            ldwrku)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf chunk var-2))
        (when var-3 (setf n var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf cone var-5))
        (when var-7 (setf lda var-7))
        (when var-9 (setf ldwrkr var-9))
        (when var-10 (setf czero var-10))
        (when var-12 (setf ldwrku var-12)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (clacpy "F" chunk n
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                 (iu) ((1 *)) work-%offset%)
           ldwrku
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf chunk var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf ldwrku var-4))
        (when var-6 (setf lda var-6)))
     label20)))
  (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (cgeqrf m n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (iwork) ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-7))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (clacpy "U" n n a lda vt ldvt)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf n var-1))
     (when var-2 (setf n var-2))
```

```
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(if (> n 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "L" (f2cl-lib:int-sub n 1)
         (f2cl-lib:int-sub n 1) czero czero
         (f2cl-lib:array-slice vt-%data% f2cl-lib:complex8
                               (2 1) ((1 ldvt) (1 *))
                               vt-%offset%)
         ldvt)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldvt var-6))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cungqr m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd n n vt ldvt s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldvt var-3)
  (setf ierr var-10))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (cunmbr "Q" "R" "N" m n n vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldvt var-7))
  (when var-10 (setf lda var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "P" n n n vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" n n m 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     vt ldvt a lda cdum 1
     (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf n var-1)
  (setf n var-2)
  (setf m var-3)
```

```
      (setf ldvt var-8)
      (setf lda var-10)
      (setf info var-14)))))
 (wntus
  (cond
   (wntvn
    (cond
     ((>= lwork
          (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                            (f2cl-lib:int-mul 3 n)))
      (setf ir 1)
      (cond
       ((>= lwork
            (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda n)))
        (setf ldwrkr lda))
       (t (setf ldwrkr n)))
      (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
      (setf iwork (f2cl-lib:int-add itau n))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (cgeqrf m n a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                 (itau) ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                 (iwork) ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
           ierr)
        (declare (ignore var-2 var-4 var-5 var-6))
        (setf m var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf ierr var-7))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (clacpy "U" n n a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                                 ((1 *)) work-%offset%)
           ldwrkr)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldwrkr var-6)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (claset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
           czero czero
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                 ((+ ir 1)) ((1 *)) work-%offset%)
           ldwrkr)
```

```
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cungqr m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "Q" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
```

```
                              ((1 *)) work-%offset%)
      ldwrkr
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (itauq) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" n 0 n 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                              ((1 *)) rwork-%offset%)
     cdum 1
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                              ((1 *)) work-%offset%)
     ldwrkr cdum 1
     (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                              ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
    var-11 var-12 var-13))
  (setf n var-1)
  (setf n var-3)
  (setf ldwrkr var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (cgemm "N" "N" m n n cone a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                              ((1 *)) work-%offset%)
     ldwrkr czero u ldu)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf cone var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldwrkr var-9))
  (when var-10 (setf czero var-10))
```

```
    (when var-12 (setf ldu var-12))))
  (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (cgeqrf m n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (itau) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (iwork) ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (clacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (cungqr m n n u ldu
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (itau) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (iwork) ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf m var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldu var-4))
    (when var-8 (setf ierr var-8)))
   (setf ie 1) (setf itauq itau)
   (setf itaup (f2cl-lib:int-add itauq n))
   (setf iwork (f2cl-lib:int-add itaup n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (claset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
        czero czero
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (2 1)
                              ((1 lda) (1 *)) a-%offset%)
        lda)
    (declare (ignore var-0 var-1 var-2 var-5))
```

```
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd n n a lda s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (cunmbr "Q" "R" "N" m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" n 0 m 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
```

```
          cdum 1 u ldu cdum 1
          (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                                ((1 *)) rwork-%offset%)
          info)
        (declare
         (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
          var-11 var-12 var-13))
        (setf n var-1)
        (setf m var-3)
        (setf ldu var-10)
        (setf info var-14)))))
  (wntvo
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul 2 n n)
                           (f2cl-lib:int-mul 3 n)))
     (setf iu 1)
     (cond
      ((>= lwork
           (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 lda n)))
       (setf ldwrku lda)
       (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
       (setf ldwrkr lda))
      ((>= lwork
           (f2cl-lib:int-add wrkbl
                             (f2cl-lib:int-mul
                              (f2cl-lib:int-add lda n) n)))
       (setf ldwrku lda)
       (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
       (setf ldwrkr n))
      (t (setf ldwrku n)
       (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
       (setf ldwrkr n)))
     (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
     (setf iwork (f2cl-lib:int-add itau n))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (cgeqrf m n a lda
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (itau) ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
          ierr)
       (declare (ignore var-2 var-4 var-5 var-6))
       (setf m var-0)
       (setf n var-1)
       (setf lda var-3)
       (setf ierr var-7))
     (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           ((+ iu 1)) ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cungqr m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
```

```
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                            (itaup) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                            (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "U" n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "Q" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "P" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
```

```
                              ((1 *)) work-%offset%)
      ldwrkr
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (itaup) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" n n n 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                              ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                              ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                              ((1 *)) work-%offset%)
     ldwrku cdum 1
     (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                              ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf n var-1)
  (setf n var-2)
  (setf n var-3)
  (setf ldwrkr var-8)
  (setf ldwrku var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (cgemm "N" "N" m n n cone a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                              ((1 *)) work-%offset%)
     ldwrku czero u ldu)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
```

```
    (when var-5 (setf cone var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldwrku var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf ldu var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "F" n n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                             ((1 *)) work-%offset%)
       ldwrkr a lda)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldwrkr var-4))
    (when var-6 (setf lda var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (cgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (cungqr m n n u ldu
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf m var-0))
```

```
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (2 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd n n a lda s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (cunmbr "Q" "R" "N" m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
```

```
        (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
         var-12))
       (when var-3 (setf m var-3))
       (when var-4 (setf n var-4))
       (when var-5 (setf n var-5))
       (when var-7 (setf lda var-7))
       (when var-10 (setf ldu var-10))
       (when var-13 (setf ierr var-13)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
          var-9)
         (cungbr "P" n n n a lda
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (itaup) ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
          ierr)
       (declare (ignore var-0 var-4 var-6 var-7 var-8))
       (when var-1 (setf n var-1))
       (when var-2 (setf n var-2))
       (when var-3 (setf n var-3))
       (when var-5 (setf lda var-5))
       (when var-9 (setf ierr var-9)))
     (setf irwork (f2cl-lib:int-add ie n))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
          var-9 var-10 var-11 var-12 var-13 var-14)
         (cbdsqr "U" n n m 0 s
          (f2cl-lib:array-slice rwork-%data% single-float (ie)
                                ((1 *)) rwork-%offset%)
          a lda u ldu cdum 1
          (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                                ((1 *)) rwork-%offset%)
          info)
       (declare
        (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
         var-13))
       (setf n var-1)
       (setf n var-2)
       (setf m var-3)
       (setf lda var-8)
       (setf ldu var-10)
       (setf info var-14)))))
  (wntvas
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                           (f2cl-lib:int-mul 3 n)))
     (setf iu 1)
```

```
(cond
 ((>= lwork
      (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda n)))
  (setf ldwrku lda))
 (t (setf ldwrku n)))
(setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
(setf iwork (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgeqrf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           ((+ iu 1)) ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cungqr m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
```

```
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "U" n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "Q" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
```

```
                                ((1 *)) work-%offset%)
                ldwrku
                (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (itauq) ((1 *)) work-%offset%)
                (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
                (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
                ierr)
            (declare (ignore var-0 var-4 var-6 var-7 var-8))
            (when var-1 (setf n var-1))
            (when var-2 (setf n var-2))
            (when var-3 (setf n var-3))
            (when var-5 (setf ldwrku var-5))
            (when var-9 (setf ierr var-9)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9)
              (cungbr "P" n n n vt ldvt
               (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (itaup) ((1 *)) work-%offset%)
               (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
               (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
               ierr)
            (declare (ignore var-0 var-4 var-6 var-7 var-8))
            (when var-1 (setf n var-1))
            (when var-2 (setf n var-2))
            (when var-3 (setf n var-3))
            (when var-5 (setf ldvt var-5))
            (when var-9 (setf ierr var-9)))
          (setf irwork (f2cl-lib:int-add ie n))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10 var-11 var-12 var-13 var-14)
              (cbdsqr "U" n n n 0 s
               (f2cl-lib:array-slice rwork-%data% single-float (ie)
                                ((1 *)) rwork-%offset%)
               vt ldvt
               (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                                ((1 *)) work-%offset%)
               ldwrku cdum 1
               (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                                ((1 *)) rwork-%offset%)
               info)
            (declare
             (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
              var-13))
            (setf n var-1)
            (setf n var-2)
            (setf n var-3)
```

```
    (setf ldvt var-8)
    (setf ldwrku var-10)
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (cgemm "N" "N" m n n cone a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                             ((1 *)) work-%offset%)
       ldwrku czero u ldu)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldwrku var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf ldu var-12))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (cgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (cungqr m n n u ldu
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
```

```
      ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "U" n n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(if (> n 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "L" (f2cl-lib:int-sub n 1)
         (f2cl-lib:int-sub n 1) czero czero
         (f2cl-lib:array-slice vt-%data% f2cl-lib:complex8
                               (2 1) ((1 ldvt) (1 *))
                               vt-%offset%)
         ldvt)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldvt var-6))))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd n n vt ldvt s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldvt var-3)
```

```
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (cunmbr "Q" "R" "N" m n n vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldvt var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "P" n n n vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" n n m 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     vt ldvt u ldu cdum 1
     (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
```

```
          var-13))
        (setf n var-1)
        (setf n var-2)
        (setf m var-3)
        (setf ldvt var-8)
        (setf ldu var-10)
        (setf info var-14)))))))
(wntua
 (cond
  (wntvn
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add n m))
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-mul 3 n)))))
     (setf ir 1)
     (cond
      ((>= lwork
           (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda n)))
       (setf ldwrkr lda))
      (t (setf ldwrkr n)))
     (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
     (setf iwork (f2cl-lib:int-add itau n))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (cgeqrf m n a lda
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (itau) ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
          ierr)
       (declare (ignore var-2 var-4 var-5 var-6))
       (setf m var-0)
       (setf n var-1)
       (setf lda var-3)
       (setf ierr var-7))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (clacpy "L" m n a lda u ldu)
       (declare (ignore var-0 var-3 var-5))
       (when var-1 (setf m var-1))
       (when var-2 (setf n var-2))
       (when var-4 (setf lda var-4))
       (when var-6 (setf ldu var-6)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
    (clacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           ((+ ir 1)) ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cungqr m m n u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
```

```
                                (itaup) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "Q" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                                ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" n 0 n 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                                ((1 *)) rwork-%offset%)
     cdum 1
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                                ((1 *)) work-%offset%)
     ldwrkr cdum 1
     (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                                ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
    var-11 var-12 var-13))
  (setf n var-1)
  (setf n var-3)
  (setf ldwrkr var-10)
```

```
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (cgemm "N" "N" m n n cone u ldu
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                             ((1 *)) work-%offset%)
       ldwrkr czero a lda)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf ldu var-7))
    (when var-9 (setf ldwrkr var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf lda var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "F" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (cgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
```

```
    (cungqr m m n u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (2 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd n n a lda s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
```

```
          (cunmbr "Q" "R" "N" m n n a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                 (itauq) ((1 *)) work-%offset%)
           u ldu
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                 (iwork) ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
           ierr)
        (declare
         (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
          var-12))
        (when var-3 (setf m var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf n var-5))
        (when var-7 (setf lda var-7))
        (when var-10 (setf ldu var-10))
        (when var-13 (setf ierr var-13)))
      (setf irwork (f2cl-lib:int-add ie n))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12 var-13 var-14)
          (cbdsqr "U" n 0 m 0 s
           (f2cl-lib:array-slice rwork-%data% single-float (ie)
                                 ((1 *)) rwork-%offset%)
           cdum 1 u ldu cdum 1
           (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                                 ((1 *)) rwork-%offset%)
           info)
        (declare
         (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
          var-11 var-12 var-13))
        (setf n var-1)
        (setf m var-3)
        (setf ldu var-10)
        (setf info var-14)))))
    (wntvo
     (cond
      ((>= lwork
           (f2cl-lib:int-add (f2cl-lib:int-mul 2 n n)
                             (max
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add n m))
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-mul 3 n)))))
      (setf iu 1)
      (cond
       ((>= lwork
            (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 lda n)))
        (setf ldwrku lda)
        (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
```

```
  (setf ldwrkr lda))
 ((>= lwork
      (f2cl-lib:int-add wrkbl
                        (f2cl-lib:int-mul
                         (f2cl-lib:int-add lda n) n)))
  (setf ldwrku lda)
  (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
  (setf ldwrkr n))
 (t (setf ldwrku n)
  (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
  (setf ldwrkr n)))
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
(setf iwork (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgeqrf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cungqr m m n u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           ((+ iu 1)) ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrku var-6)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
    (clacpy "U" n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "Q" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "P" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
```

```
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" n n n 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku cdum 1
     (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf n var-1)
  (setf n var-2)
  (setf n var-3)
  (setf ldwrkr var-8)
  (setf ldwrku var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (cgemm "N" "N" m n n cone u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku czero a lda)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf cone var-5))
  (when var-7 (setf ldu var-7))
  (when var-9 (setf ldwrku var-9))
  (when var-10 (setf czero var-10))
  (when var-12 (setf lda var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "F" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
```

```
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "F" n n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                             ((1 *)) work-%offset%)
       ldwrkr a lda)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldwrkr var-4))
    (when var-6 (setf lda var-6))))
  (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
   (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (cgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
   (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
   (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (cungqr m m n u ldu
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf m var-0))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldu var-4))
    (when var-8 (setf ierr var-8)))
   (setf ie 1) (setf itauq itau)
   (setf itaup (f2cl-lib:int-add itauq n))
```

```
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (2 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd n n a lda s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (cunmbr "Q" "R" "N" m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
```

```
      (when var-10 (setf ldu var-10))
      (when var-13 (setf ierr var-13)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9)
        (cungbr "P" n n n a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                               (itaup) ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                               (iwork) ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
         ierr)
      (declare (ignore var-0 var-4 var-6 var-7 var-8))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf n var-3))
      (when var-5 (setf lda var-5))
      (when var-9 (setf ierr var-9)))
    (setf irwork (f2cl-lib:int-add ie n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14)
        (cbdsqr "U" n n m 0 s
         (f2cl-lib:array-slice rwork-%data% single-float (ie)
                               ((1 *)) rwork-%offset%)
         a lda u ldu cdum 1
         (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                               ((1 *)) rwork-%offset%)
         info)
      (declare
       (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
        var-13))
      (setf n var-1)
      (setf n var-2)
      (setf m var-3)
      (setf lda var-8)
      (setf ldu var-10)
      (setf info var-14)))))
  (wntvas
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add n m))
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-mul 3 n)))))
    (setf iu 1)
    (cond
     ((>= lwork
```

```
      (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda n)))
  (setf ldwrku lda))
 (t (setf ldwrku n)))
(setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
(setf iwork (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgeqrf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cungqr m m n u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
```

```
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           ((+ iu 1)) ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrku var-6)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "U" n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldwrku var-4))
```

```
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "Q" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "P" n n n vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" n n n 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku cdum 1
     (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
```

```
    (declare
     (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
      var-13))
    (setf n var-1)
    (setf n var-2)
    (setf n var-3)
    (setf ldvt var-8)
    (setf ldwrku var-10)
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (cgemm "N" "N" m n n cone u ldu
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                             ((1 *)) work-%offset%)
       ldwrku czero a lda)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf ldu var-7))
    (when var-9 (setf ldwrku var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf lda var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "F" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (cgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
    (clacpy "L" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cungqr m m n u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "U" n n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(if (> n 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "L" (f2cl-lib:int-sub n 1)
         (f2cl-lib:int-sub n 1) czero czero
         (f2cl-lib:array-slice vt-%data% f2cl-lib:complex8
                               (2 1) ((1 ldvt) (1 *))
                               vt-%offset%)
         ldvt)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldvt var-6))))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd n n vt ldvt s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
```

```
                                ((1 *)) rwork-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (itauq) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (itaup) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldvt var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (cunmbr "Q" "R" "N" m n n vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (itauq) ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldvt var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "P" n n n vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
```

```
          (when var-9 (setf ierr var-9)))
        (setf irwork (f2cl-lib:int-add ie n))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12 var-13 var-14)
            (cbdsqr "U" n n m 0 s
             (f2cl-lib:array-slice rwork-%data% single-float (ie)
                                   ((1 *)) rwork-%offset%)
             vt ldvt u ldu cdum 1
             (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                                   ((1 *)) rwork-%offset%)
             info)
          (declare
           (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
            var-13))
          (setf n var-1)
          (setf n var-2)
          (setf m var-3)
          (setf ldvt var-8)
          (setf ldu var-10)
          (setf info var-14)))))))))
(t (setf ie 1) (setf itauq 1)
 (setf itaup (f2cl-lib:int-add itauq n))
 (setf iwork (f2cl-lib:int-add itaup n))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (cgebrd m n a lda s
      (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                            rwork-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-10))
 (cond
  (wntuas
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (clacpy "L" m n a lda u ldu)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf m var-1))
     (when var-2 (setf n var-2))
     (when var-4 (setf lda var-4))
```

```
      (when var-6 (setf ldu var-6)))
    (if wntus
        (setf ncu n))
    (if wntua
        (setf ncu m))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (cungbr "Q" m ncu n u ldu
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
      (declare (ignore var-0 var-4 var-6 var-7 var-8))
      (when var-1 (setf m var-1))
      (when var-2 (setf ncu var-2))
      (when var-3 (setf n var-3))
      (when var-5 (setf ldu var-5))
      (when var-9 (setf ierr var-9)))))
  (cond
   (wntvas
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (clacpy "U" n n a lda vt ldvt)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-6 (setf ldvt var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (cungbr "P" n n n vt ldvt
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
      (declare (ignore var-0 var-4 var-6 var-7 var-8))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf n var-3))
      (when var-5 (setf ldvt var-5))
      (when var-9 (setf ierr var-9)))))
  (cond
   (wntuo
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (cungbr "Q" m n n a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
```

```
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))))
(cond
 (wntvo
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (cungbr "P" n n n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                              ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                              ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))))
(setf irwork (f2cl-lib:int-add ie n))
(if (or wntuas wntuo)
    (setf nru m))
(if wntun
    (setf nru 0))
(if (or wntvas wntvo)
    (setf ncvt n))
(if wntvn
    (setf ncvt 0))
(cond
 ((and (not wntuo) (not wntvo))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (cbdsqr "U" n ncvt nru 0 s
       (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                              rwork-%offset%)
       vt ldvt u ldu cdum 1
       (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                              ((1 *)) rwork-%offset%)
       info)
    (declare
     (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
      var-13))
    (setf n var-1)
    (setf ncvt var-2)
```

```
            (setf nru var-3)
            (setf ldvt var-8)
            (setf ldu var-10)
            (setf info var-14)))
        ((and (not wntuo) wntvo)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
              var-10 var-11 var-12 var-13 var-14)
             (cbdsqr "U" n ncvt nru 0 s
              (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                                    rwork-%offset%)
              a lda u ldu cdum 1
              (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                                    ((1 *)) rwork-%offset%)
              info)
            (declare
             (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
              var-13))
            (setf n var-1)
            (setf ncvt var-2)
            (setf nru var-3)
            (setf lda var-8)
            (setf ldu var-10)
            (setf info var-14)))
        (t
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
              var-10 var-11 var-12 var-13 var-14)
             (cbdsqr "U" n ncvt nru 0 s
              (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                                    rwork-%offset%)
              vt ldvt a lda cdum 1
              (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                                    ((1 *)) rwork-%offset%)
              info)
            (declare
             (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
              var-13))
            (setf n var-1)
            (setf ncvt var-2)
            (setf nru var-3)
            (setf ldvt var-8)
            (setf lda var-10)
            (setf info var-14)))))))
    (t
     (cond
      ((>= n mnthr)
       (cond
        (wntvn (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
         (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgelqf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(setf ie 1) (setf itauq 1)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgebrd m m a lda s
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(cond
 ((or wntuo wntuas)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (cungbr "Q" m m m a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
```

```
                                (itauq) ((1 *)) work-%offset%)
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
            (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
        (declare (ignore var-0 var-4 var-6 var-7 var-8))
        (when var-1 (setf m var-1))
        (when var-2 (setf m var-2))
        (when var-3 (setf m var-3))
        (when var-5 (setf lda var-5))
        (when var-9 (setf ierr var-9)))))
    (setf irwork (f2cl-lib:int-add ie m)) (setf nru 0)
    (if (or wntuo wntuas)
        (setf nru m))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14)
        (cbdsqr "U" m 0 nru 0 s
         (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                               rwork-%offset%)
         cdum 1 a lda cdum 1
         (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                               ((1 *)) rwork-%offset%)
         info)
      (declare
       (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9 var-11
        var-12 var-13))
      (setf m var-1)
      (setf nru var-3)
      (setf lda var-10)
      (setf info var-14))
    (if wntuas
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (clacpy "F" m m a lda u ldu)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf m var-2))
          (when var-4 (setf lda var-4))
          (when var-6 (setf ldu var-6)))))
  ((and wntvo wntun)
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                           (f2cl-lib:int-mul 3 m)))
      (setf ir 1)
      (cond
       ((>= lwork
            (f2cl-lib:int-add
             (max (the f2cl-lib:integer4 wrkbl)
                  (the f2cl-lib:integer4 (f2cl-lib:int-mul lda n)))
```

```
      (f2cl-lib:int-mul lda m)))
  (setf ldwrku lda) (setf chunk n) (setf ldwrkr lda))
 ((>= lwork
      (f2cl-lib:int-add
       (max (the f2cl-lib:integer4 wrkbl)
            (the f2cl-lib:integer4 (f2cl-lib:int-mul lda n)))
       (f2cl-lib:int-mul m m)))
  (setf ldwrku lda) (setf chunk n) (setf ldwrkr m))
 (t (setf ldwrku m)
  (setf chunk
          (the f2cl-lib:integer4
               (truncate (- lwork (* m m)) m)))
  (setf ldwrkr m)))
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
(setf iwork (f2cl-lib:int-add itau m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgelqf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           ((+ ir ldwrkr)) ((1 *))
                           work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
```

```
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cunglq m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "P" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
```

```
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" m m 0 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr cdum 1 cdum 1
     (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
    var-11 var-12 var-13))
  (setf m var-1)
  (setf m var-2)
  (setf ldwrkr var-8)
  (setf info var-14))
(setf iu itauq)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
              ((> i n) nil)
  (tagbody
    (setf blk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
             (the f2cl-lib:integer4 chunk)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (cgemm "N" "N" m blk m cone
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                               (ir) ((1 *)) work-%offset%)
         ldwrkr
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 i)
                               ((1 lda) (1 *)) a-%offset%)
         lda czero
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                               (iu) ((1 *)) work-%offset%)
         ldwrku)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
```

```
          (when var-2 (setf m var-2))
          (when var-3 (setf blk var-3))
          (when var-4 (setf m var-4))
          (when var-5 (setf cone var-5))
          (when var-7 (setf ldwrkr var-7))
          (when var-9 (setf lda var-9))
          (when var-10 (setf czero var-10))
          (when var-12 (setf ldwrku var-12)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (clacpy "F" m blk
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                   (iu) ((1 *)) work-%offset%)
             ldwrku
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf blk var-2))
          (when var-4 (setf ldwrku var-4))
          (when var-6 (setf lda var-6)))
      label30)))
  (t (setf ie 1) (setf itauq 1)
   (setf itaup (f2cl-lib:int-add itauq m))
   (setf iwork (f2cl-lib:int-add itaup m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (cgebrd m n a lda s
        (f2cl-lib:array-slice rwork-%data% single-float (ie)
                              ((1 *)) rwork-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (itauq) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (itaup) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (iwork) ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-10))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9)
       (cungbr "P" m n m a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (itaup) ((1 *)) work-%offset%)
```

```
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
        (declare (ignore var-0 var-4 var-6 var-7 var-8))
        (when var-1 (setf m var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf m var-3))
        (when var-5 (setf lda var-5))
        (when var-9 (setf ierr var-9)))
      (setf irwork (f2cl-lib:int-add ie m))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12 var-13 var-14)
          (cbdsqr "L" m n 0 0 s
           (f2cl-lib:array-slice rwork-%data% single-float (ie)
                                 ((1 *)) rwork-%offset%)
           a lda cdum 1 cdum 1
           (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                                 ((1 *)) rwork-%offset%)
           info)
        (declare
         (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
          var-11 var-12 var-13))
        (setf m var-1)
        (setf n var-2)
        (setf lda var-8)
        (setf info var-14)))))
  ((and wntvo wntuas)
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                           (f2cl-lib:int-mul 3 m)))
      (setf ir 1)
      (cond
       ((>= lwork
            (f2cl-lib:int-add
             (max (the f2cl-lib:integer4 wrkbl)
                  (the f2cl-lib:integer4 (f2cl-lib:int-mul lda n)))
             (f2cl-lib:int-mul lda m)))
        (setf ldwrku lda) (setf chunk n) (setf ldwrkr lda))
       ((>= lwork
            (f2cl-lib:int-add
             (max (the f2cl-lib:integer4 wrkbl)
                  (the f2cl-lib:integer4 (f2cl-lib:int-mul lda n)))
             (f2cl-lib:int-mul m m)))
        (setf ldwrku lda) (setf chunk n) (setf ldwrkr m))
       (t (setf ldwrku m)
        (setf chunk
                (the f2cl-lib:integer4
                     (truncate (- lwork (* m m)) m)))
```

```
  (setf ldwrkr m)))
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
(setf iwork (f2cl-lib:int-add itau m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgelqf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" m m a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 2)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cunglq m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
```

```
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd m m u ldu s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldu var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "U" m m u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldu var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "P" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" m m m 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr u ldu cdum 1
     (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf m var-1)
  (setf m var-2)
  (setf m var-3)
  (setf ldwrkr var-8)
  (setf ldu var-10)
  (setf info var-14))
(setf iu itauq)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
              ((> i n) nil)
  (tagbody
    (setf blk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
             (the f2cl-lib:integer4 chunk)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (cgemm "N" "N" m blk m cone
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
```

```
                                (ir) ((1 *)) work-%offset%)
            ldwrkr
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 i)
                                ((1 lda) (1 *)) a-%offset%)
            lda czero
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iu) ((1 *)) work-%offset%)
            ldwrku)
          (declare (ignore var-0 var-1 var-6 var-8 var-11))
          (when var-2 (setf m var-2))
          (when var-3 (setf blk var-3))
          (when var-4 (setf m var-4))
          (when var-5 (setf cone var-5))
          (when var-7 (setf ldwrkr var-7))
          (when var-9 (setf lda var-9))
          (when var-10 (setf czero var-10))
          (when var-12 (setf ldwrku var-12)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (clacpy "F" m blk
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iu) ((1 *)) work-%offset%)
             ldwrku
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 i)
                                ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf blk var-2))
          (when var-4 (setf ldwrku var-4))
          (when var-6 (setf lda var-6)))
      label40)))
  (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (cgelqf m n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (iwork) ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-7))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (clacpy "L" m m a lda u ldu)
     (declare (ignore var-0 var-3 var-5))
```

```
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 2)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cunglq m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd m m u ldu s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldu var-3)
  (setf ierr var-10))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (cunmbr "P" "L" "C" m n m u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldu var-7))
  (when var-10 (setf lda var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" m n m 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     a lda u ldu cdum 1
     (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf m var-1)
  (setf n var-2)
  (setf m var-3)
```

```
      (setf lda var-8)
      (setf ldu var-10)
      (setf info var-14)))))
 (wntvs
  (cond
   (wntun
    (cond
     ((>= lwork
          (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                            (f2cl-lib:int-mul 3 m)))
      (setf ir 1)
      (cond
       ((>= lwork
            (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda m)))
        (setf ldwrkr lda))
       (t (setf ldwrkr m)))
      (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
      (setf iwork (f2cl-lib:int-add itau m))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (cgelqf m n a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                 (itau) ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                 (iwork) ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
           ierr)
        (declare (ignore var-2 var-4 var-5 var-6))
        (setf m var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf ierr var-7))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (clacpy "L" m m a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                                 ((1 *)) work-%offset%)
           ldwrkr)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf m var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldwrkr var-6)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
           czero czero
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                 ((+ ir ldwrkr)) ((1 *))
                                 work-%offset%)
```

```
      ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cunglq m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "P" m m m
```

```
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                            ((1 *)) work-%offset%)
      ldwrkr
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                            (itaup) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                            (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" m m 0 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr cdum 1 cdum 1
     (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
    var-11 var-12 var-13))
  (setf m var-1)
  (setf m var-2)
  (setf ldwrkr var-8)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (cgemm "N" "N" m n m cone
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr a lda czero vt ldvt)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf cone var-5))
  (when var-7 (setf ldwrkr var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf czero var-10))
```

```
    (when var-12 (setf ldvt var-12))))
  (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (cgelqf m n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (itau) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (iwork) ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (clacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (cunglq m n m vt ldvt
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (itau) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (iwork) ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf m var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf ldvt var-4))
    (when var-8 (setf ierr var-8)))
   (setf ie 1) (setf itauq itau)
   (setf itaup (f2cl-lib:int-add itauq m))
   (setf iwork (f2cl-lib:int-add itaup m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
        czero czero
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 2)
                              ((1 lda) (1 *)) a-%offset%)
        lda)
    (declare (ignore var-0 var-1 var-2 var-5))
```

```
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd m m a lda s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (cunmbr "P" "L" "C" m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" m n 0 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
```

```
          vt ldvt cdum 1 cdum 1
          (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                                ((1 *)) rwork-%offset%)
          info)
        (declare
         (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
          var-11 var-12 var-13))
        (setf m var-1)
        (setf n var-2)
        (setf ldvt var-8)
        (setf info var-14)))))
  (wntuo
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul 2 m m)
                           (f2cl-lib:int-mul 3 m)))
     (setf iu 1)
     (cond
      ((>= lwork
           (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 lda m)))
       (setf ldwrku lda)
       (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
       (setf ldwrkr lda))
      ((>= lwork
           (f2cl-lib:int-add wrkbl
                             (f2cl-lib:int-mul
                              (f2cl-lib:int-add lda m) m)))
       (setf ldwrku lda)
       (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
       (setf ldwrkr m))
      (t (setf ldwrku m)
       (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
       (setf ldwrkr m)))
     (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
     (setf iwork (f2cl-lib:int-add itau m))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (cgelqf m n a lda
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (itau) ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
          ierr)
       (declare (ignore var-2 var-4 var-5 var-6))
       (setf m var-0)
       (setf n var-1)
       (setf lda var-3)
       (setf ierr var-7))
     (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           ((+ iu ldwrku)) ((1 *))
                           work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cunglq m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
```

```
                              (itauq) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (itaup) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                              ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                              ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "P" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                              ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "Q" m m m
```

```
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                            ((1 *)) work-%offset%)
      ldwrkr
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                            (itauq) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                            (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" m m m 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr cdum 1
     (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf m var-1)
  (setf m var-2)
  (setf m var-3)
  (setf ldwrku var-8)
  (setf ldwrkr var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (cgemm "N" "N" m n m cone
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku a lda czero vt ldvt)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
```

```
    (when var-4 (setf m var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf ldwrku var-7))
    (when var-9 (setf lda var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf ldvt var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "F" m m
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                             ((1 *)) work-%offset%)
       ldwrkr a lda)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf ldwrkr var-4))
    (when var-6 (setf lda var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (cgelqf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (cunglq m n m vt ldvt
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
```

```
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd m m a lda s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (cunmbr "P" "L" "C" m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
```

```
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
        var-12))
      (when var-3 (setf m var-3))
      (when var-4 (setf n var-4))
      (when var-5 (setf m var-5))
      (when var-7 (setf lda var-7))
      (when var-10 (setf ldvt var-10))
      (when var-13 (setf ierr var-13)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9)
        (cungbr "Q" m m m a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                               (itauq) ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                               (iwork) ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
         ierr)
      (declare (ignore var-0 var-4 var-6 var-7 var-8))
      (when var-1 (setf m var-1))
      (when var-2 (setf m var-2))
      (when var-3 (setf m var-3))
      (when var-5 (setf lda var-5))
      (when var-9 (setf ierr var-9)))
    (setf irwork (f2cl-lib:int-add ie m))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14)
        (cbdsqr "U" m n m 0 s
         (f2cl-lib:array-slice rwork-%data% single-float (ie)
                               ((1 *)) rwork-%offset%)
         vt ldvt a lda cdum 1
         (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                               ((1 *)) rwork-%offset%)
         info)
      (declare
       (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
        var-13))
      (setf m var-1)
      (setf n var-2)
      (setf m var-3)
      (setf ldvt var-8)
      (setf lda var-10)
      (setf info var-14)))))
  (wntuas
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                           (f2cl-lib:int-mul 3 m)))
```

```
(setf iu 1)
(cond
 ((>= lwork
      (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda m)))
  (setf ldwrku lda))
 (t (setf ldwrku m)))
(setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
(setf iwork (f2cl-lib:int-add itau m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgelqf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           ((+ iu ldwrku)) ((1 *))
                           work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cunglq m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
```

```
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                            (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
```

```
    (cungbr "P" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" m m m 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku u ldu cdum 1
     (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf m var-1)
  (setf m var-2)
```

```
    (setf m var-3)
    (setf ldwrku var-8)
    (setf ldu var-10)
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (cgemm "N" "N" m n m cone
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                             ((1 *)) work-%offset%)
       ldwrku a lda czero vt ldvt)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf ldwrku var-7))
    (when var-9 (setf lda var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf ldvt var-12))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (cgelqf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (cunglq m n m vt ldvt
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (iwork) ((1 *)) work-%offset%)
```

```
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" m m a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 2)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldu var-6)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd m m u ldu s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldu var-3)
  (setf ierr var-10))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (cunmbr "P" "L" "C" m n m u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldu var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" m n m 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     vt ldvt u ldu cdum 1
     (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
```

```
        (setf m var-1)
        (setf n var-2)
        (setf m var-3)
        (setf ldvt var-8)
        (setf ldu var-10)
        (setf info var-14)))))))
(wntva
 (cond
  (wntun
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add n m))
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-mul 3 m)))))
     (setf ir 1)
     (cond
      ((>= lwork
           (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda m)))
       (setf ldwrkr lda))
      (t (setf ldwrkr m)))
     (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
     (setf iwork (f2cl-lib:int-add itau m))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (cgelqf m n a lda
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (itau) ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
          ierr)
       (declare (ignore var-2 var-4 var-5 var-6))
       (setf m var-0)
       (setf n var-1)
       (setf lda var-3)
       (setf ierr var-7))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (clacpy "U" m n a lda vt ldvt)
       (declare (ignore var-0 var-3 var-5))
       (when var-1 (setf m var-1))
       (when var-2 (setf n var-2))
       (when var-4 (setf lda var-4))
       (when var-6 (setf ldvt var-6)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (clacpy "L" m m a lda
```

```
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           ((+ ir ldwrkr)) ((1 *))
                           work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cunglq n n m vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
```

```
                                (itaup) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "P" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                                ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" m m 0 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                                ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                                ((1 *)) work-%offset%)
     ldwrkr cdum 1 cdum 1
     (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                                ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
    var-11 var-12 var-13))
  (setf m var-1)
  (setf m var-2)
  (setf ldwrkr var-8)
  (setf info var-14))
```

```
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12)
     (cgemm "N" "N" m n m cone
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                            ((1 *)) work-%offset%)
      ldwrkr vt ldvt czero a lda)
   (declare (ignore var-0 var-1 var-6 var-8 var-11))
   (when var-2 (setf m var-2))
   (when var-3 (setf n var-3))
   (when var-4 (setf m var-4))
   (when var-5 (setf cone var-5))
   (when var-7 (setf ldwrkr var-7))
   (when var-9 (setf ldvt var-9))
   (when var-10 (setf czero var-10))
   (when var-12 (setf lda var-12)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (clacpy "F" m n a lda vt ldvt)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf m var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldvt var-6))))
(t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (cgelqf m n a lda
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                            (itau) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                            (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
   (declare (ignore var-2 var-4 var-5 var-6))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-7))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (clacpy "U" m n a lda vt ldvt)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf m var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldvt var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (cunglq n n m vt ldvt
```

```
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                            (itau) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                            (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd m m a lda s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (cunmbr "P" "L" "C" m n m a lda
```

```
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (itaup) ((1 *)) work-%offset%)
          vt ldvt
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
          ierr)
        (declare
         (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
          var-12))
        (when var-3 (setf m var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf m var-5))
        (when var-7 (setf lda var-7))
        (when var-10 (setf ldvt var-10))
        (when var-13 (setf ierr var-13)))
      (setf irwork (f2cl-lib:int-add ie m))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12 var-13 var-14)
          (cbdsqr "U" m n 0 0 s
           (f2cl-lib:array-slice rwork-%data% single-float (ie)
                                 ((1 *)) rwork-%offset%)
           vt ldvt cdum 1 cdum 1
           (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                                 ((1 *)) rwork-%offset%)
           info)
        (declare
         (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
          var-11 var-12 var-13))
        (setf m var-1)
        (setf n var-2)
        (setf ldvt var-8)
        (setf info var-14)))))
  (wntuo
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul 2 m m)
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add n m))
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-mul 3 m)))))
      (setf iu 1)
      (cond
       ((>= lwork
            (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 lda m)))
        (setf ldwrku lda)
        (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
        (setf ldwrkr lda))
```

```
 ((>= lwork
      (f2cl-lib:int-add wrkbl
                        (f2cl-lib:int-mul
                         (f2cl-lib:int-add lda m) m)))
  (setf ldwrku lda)
  (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
  (setf ldwrkr m))
 (t (setf ldwrku m)
  (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
  (setf ldwrkr m)))
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
(setf iwork (f2cl-lib:int-add itau m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgelqf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "U" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cunglq n n m vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           ((+ iu ldwrku)) ((1 *))
                           work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrku var-6)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
    (clacpy "L" m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "P" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "Q" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
```

```
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" m m m 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr cdum 1
     (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf m var-1)
  (setf m var-2)
  (setf m var-3)
  (setf ldwrku var-8)
  (setf ldwrkr var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (cgemm "N" "N" m n m cone
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku vt ldvt czero a lda)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf cone var-5))
  (when var-7 (setf ldwrku var-7))
  (when var-9 (setf ldvt var-9))
  (when var-10 (setf czero var-10))
  (when var-12 (setf lda var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "F" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
```

```
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "F" m m
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ir)
                             ((1 *)) work-%offset%)
       ldwrkr a lda)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf ldwrkr var-4))
    (when var-6 (setf lda var-6))))
  (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
   (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (cgelqf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
   (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
   (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (cunglq n n m vt ldvt
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf n var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf ldvt var-4))
    (when var-8 (setf ierr var-8)))
   (setf ie 1) (setf itauq itau)
   (setf itaup (f2cl-lib:int-add itauq m))
```

```
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd m m a lda s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (cunmbr "P" "L" "C" m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
```

```
      (when var-10 (setf ldvt var-10))
      (when var-13 (setf ierr var-13)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9)
        (cungbr "Q" m m m a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                               (itauq) ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                               (iwork) ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
         ierr)
      (declare (ignore var-0 var-4 var-6 var-7 var-8))
      (when var-1 (setf m var-1))
      (when var-2 (setf m var-2))
      (when var-3 (setf m var-3))
      (when var-5 (setf lda var-5))
      (when var-9 (setf ierr var-9)))
    (setf irwork (f2cl-lib:int-add ie m))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14)
        (cbdsqr "U" m n m 0 s
         (f2cl-lib:array-slice rwork-%data% single-float (ie)
                               ((1 *)) rwork-%offset%)
         vt ldvt a lda cdum 1
         (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                               ((1 *)) rwork-%offset%)
         info)
      (declare
       (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
        var-13))
      (setf m var-1)
      (setf n var-2)
      (setf m var-3)
      (setf ldvt var-8)
      (setf lda var-10)
      (setf info var-14)))))
(wntuas
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add n m))
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-mul 3 m)))))
   (setf iu 1)
   (cond
    ((>= lwork
```

```
      (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda m)))
  (setf ldwrku lda))
 (t (setf ldwrku m)))
(setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
(setf iwork (f2cl-lib:int-add itau m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cgelqf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "U" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cunglq n n m vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
```

```
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           ((+ iu ldwrku)) ((1 *))
                           work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrku var-6)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
```

```
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "P" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (cbdsqr "U" m m m 0 s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                           ((1 *)) work-%offset%)
     ldwrku u ldu cdum 1
     (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
```

```
    (declare
     (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
      var-13))
    (setf m var-1)
    (setf m var-2)
    (setf m var-3)
    (setf ldwrku var-8)
    (setf ldu var-10)
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (cgemm "N" "N" m n m cone
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iu)
                             ((1 *)) work-%offset%)
       ldwrku vt ldvt czero a lda)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf ldwrku var-7))
    (when var-9 (setf ldvt var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf lda var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "F" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (cgelqf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
    (clacpy "U" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cunglq n n m vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "L" m m a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 2)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldu var-6)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgebrd m m u ldu s
     (f2cl-lib:array-slice rwork-%data% single-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
```

```
                                (itauq) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (itaup) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldu var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (cunmbr "P" "L" "C" m n m u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (itaup) ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldu var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (cungbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
```

```
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12 var-13 var-14)
            (cbdsqr "U" m n m 0 s
             (f2cl-lib:array-slice rwork-%data% single-float (ie)
                                   ((1 *)) rwork-%offset%)
             vt ldvt u ldu cdum 1
             (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                                   ((1 *)) rwork-%offset%)
             info)
          (declare
           (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
            var-13))
          (setf m var-1)
          (setf n var-2)
          (setf m var-3)
          (setf ldvt var-8)
          (setf ldu var-10)
          (setf info var-14)))))))))
(t (setf ie 1) (setf itauq 1)
 (setf itaup (f2cl-lib:int-add itauq m))
 (setf iwork (f2cl-lib:int-add itaup m))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (cgebrd m n a lda s
      (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                            rwork-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-10))
 (cond
  (wntuas
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (clacpy "L" m m a lda u ldu)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf m var-1))
     (when var-2 (setf m var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf ldu var-6)))
   (multiple-value-bind
```

```
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (cungbr "Q" m m n u ldu
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf ldu var-5))
    (when var-9 (setf ierr var-9)))))
(cond
 (wntvas
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (if wntva
      (setf nrvt n))
  (if wntvs
      (setf nrvt m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (cungbr "P" nrvt n m vt ldvt
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf nrvt var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf ldvt var-5))
    (when var-9 (setf ierr var-9)))))
(cond
 (wntuo
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (cungbr "Q" m m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
```

```
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))))
(cond
 (wntvo
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (cungbr "P" m n m a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))))
(setf irwork (f2cl-lib:int-add ie m))
(if (or wntuas wntuo)
    (setf nru m))
(if wntun
    (setf nru 0))
(if (or wntvas wntvo)
    (setf ncvt n))
(if wntvn
    (setf ncvt 0))
(cond
 ((and (not wntuo) (not wntvo))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (cbdsqr "L" m ncvt nru 0 s
       (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                             rwork-%offset%)
       vt ldvt u ldu cdum 1
       (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                             ((1 *)) rwork-%offset%)
       info)
    (declare
     (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
      var-13))
    (setf m var-1)
    (setf ncvt var-2)
    (setf nru var-3)
    (setf ldvt var-8)
```

```
            (setf ldu var-10)
            (setf info var-14)))
        ((and (not wntuo) wntvo)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
              var-10 var-11 var-12 var-13 var-14)
             (cbdsqr "L" m ncvt nru 0 s
              (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                                    rwork-%offset%)
              a lda u ldu cdum 1
              (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                                    ((1 *)) rwork-%offset%)
              info)
            (declare
             (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
              var-13))
            (setf m var-1)
            (setf ncvt var-2)
            (setf nru var-3)
            (setf lda var-8)
            (setf ldu var-10)
            (setf info var-14)))
        (t
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
              var-10 var-11 var-12 var-13 var-14)
             (cbdsqr "L" m ncvt nru 0 s
              (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                                    rwork-%offset%)
              vt ldvt a lda cdum 1
              (f2cl-lib:array-slice rwork-%data% single-float (irwork)
                                    ((1 *)) rwork-%offset%)
              info)
            (declare
             (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
              var-13))
            (setf m var-1)
            (setf ncvt var-2)
            (setf nru var-3)
            (setf ldvt var-8)
            (setf lda var-10)
            (setf info var-14))))))))
    (cond
     ((= iscl 1)
      (if (> anrm bignum)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (slascl "G" 0 0 bignum anrm minmn 1 s minmn ierr)
            (declare (ignore var-0 var-1 var-2 var-6 var-7))
            (when var-3 (setf bignum var-3))
```

```
                (when var-4 (setf anrm var-4))
                (when var-5 (setf minmn var-5))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf ierr var-9))))
          (if (and (/= info 0) (> anrm bignum))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (slascl "G" 0 0 bignum anrm (f2cl-lib:int-sub minmn 1) 1
                   (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                                         rwork-%offset%)
                   minmn ierr)
                (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
                (when var-3 (setf bignum var-3))
                (when var-4 (setf anrm var-4))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf ierr var-9))))
          (if (< anrm smlnum)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (slascl "G" 0 0 smlnum anrm minmn 1 s minmn ierr)
                (declare (ignore var-0 var-1 var-2 var-6 var-7))
                (when var-3 (setf smlnum var-3))
                (when var-4 (setf anrm var-4))
                (when var-5 (setf minmn var-5))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf ierr var-9))))
          (if (and (/= info 0) (< anrm smlnum))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (slascl "G" 0 0 smlnum anrm (f2cl-lib:int-sub minmn 1) 1
                   (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                                         rwork-%offset%)
                   minmn ierr)
                (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
                (when var-3 (setf smlnum var-3))
                (when var-4 (setf anrm var-4))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf ierr var-9))))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce maxwrk 'f2cl-lib:complex8))
        (go end_label)
       end_label
        (return
         (values jobu jobvt m n nil lda nil nil ldu nil ldvt nil nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgesvd
```

```
        fortran-to-lisp::*f2cl-function-info*)
  (fortran-to-lisp::make-f2cl-finfo :arg-types
                                    '((string) (string)
                                      (fortran-to-lisp::integer4)
                                      (fortran-to-lisp::integer4)
                                      (array fortran-to-lisp::complex8
                                       (*))
                                      (fortran-to-lisp::integer4)
                                      (array single-float (*))
                                      (array fortran-to-lisp::complex8
                                       (*))
                                      (fortran-to-lisp::integer4)
                                      (array fortran-to-lisp::complex8
                                       (*))
                                      (fortran-to-lisp::integer4)
                                      (array fortran-to-lisp::complex8
                                       (*))
                                      (fortran-to-lisp::integer4)
                                      (array single-float (*))
                                      (fortran-to-lisp::integer4))
                                    :return-values
                                    '(fortran-to-lisp::jobu
                                      fortran-to-lisp::jobvt
                                      fortran-to-lisp::m
                                      fortran-to-lisp::n nil
                                      fortran-to-lisp::lda nil nil
                                      fortran-to-lisp::ldu nil
                                      fortran-to-lisp::ldvt nil nil nil
                                      fortran-to-lisp::info)
                                    :calls
                                    '(fortran-to-lisp::cbdsqr
                                      fortran-to-lisp::cgelqf
                                      fortran-to-lisp::cgebrd
                                      fortran-to-lisp::cgeqrf))))
```

———-

30.4 cgesvdx

30.4.1 Axiom unit tests

— cgesvdx.input —

```
)set break resume
)sys rm -f cgesvdx.output
```

```
)spool cgesvdx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

30.4.2 Axiom help page

— **cgesvdx.help** —

```
==================================================================
cgesvdx examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgesvdx (JOBU, JOBVT, RANGE, M, N, A, LDA, VL, VU, IL, IU,
           NS, S, U, LDU, VT, LDVT, WORK, LWORK, RWORK, IWORK, INFO)
            CGESVDX computes the singular value decomposition (SVD) for GE
           matrices

Function/Subroutine Documentation
   subroutine cgesvdx (character JOBU, character JOBVT, character RANGE,
       integer M, integer N, complex, dimension( lda, * ) A, integer LDA, real
       VL, real VU, integer IL, integer IU, integer NS, real, dimension( * )
       S, complex, dimension( ldu, * ) U, integer LDU, complex, dimension(
       ldvt, * ) VT, integer LDVT, complex, dimension( * ) WORK, integer
       LWORK, real, dimension( * ) RWORK, integer, dimension( * ) IWORK,
       integer INFO)
        CGESVDX computes the singular value decomposition (SVD) for GE
       matrices

Purpose:
   CGESVDX computes the singular value decomposition (SVD) of a complex
   M-by-N matrix A, optionally computing the left and/or right singular
   vectors. The SVD is written

    A = U * SIGMA * transpose(V)

   where SIGMA is an M-by-N matrix which is zero except for its
```

```
   min(m,n) diagonal elements, U is an M-by-M unitary matrix, and
   V is an N-by-N unitary matrix.  The diagonal elements of SIGMA
   are the singular values of A; they are real and non-negative, and
   are returned in descending order.  The first min(m,n) columns of
   U and V are the left and right singular vectors of A.

   CGESVDX uses an eigenvalue problem for obtaining the SVD, which
   allows for the computation of a subset of singular values and
   vectors. See SBDSVDX for details.

   Note that the routine returns V**T, not V.

Parameters:

   JOBU
   JOBU is CHARACTER*1
   Specifies options for computing all or part of the matrix U:
   = 'V':  the first min(m,n) columns of U (the left singular
        vectors) or as specified by RANGE are returned in the array U;
   = 'N':  no columns of U (no left singular vectors) are computed.

 JOBVT
   JOBVT is CHARACTER*1
   Specifies options for computing all or part of the matrix
   V**T:
   = 'V':  the first min(m,n) rows of V**T (the right singular
        vectors) or as specified by RANGE are returned in the array VT;
   = 'N':  no rows of V**T (no right singular vectors) are computed.

 RANGE
   RANGE is CHARACTER*1
   = 'A': all singular values will be found.
   = 'V': all singular values in the half-open interval (VL,VU] will be found.
   = 'I': the IL-th through IU-th singular values will be found.

 M
   M is INTEGER
   The number of rows of the input matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the input matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, the contents of A are destroyed.

 LDA
   LDA is INTEGER
```

```
  The leading dimension of the array A.  LDA >= max(1,M).

VL
  VL is REAL
  VL >=0.

VU
  VU is REAL
  If RANGE='V', the lower and upper bounds of the interval to
  be searched for singular values. VU > VL.
  Not referenced if RANGE = 'A' or 'I'.

IL
  IL is INTEGER

IU
  IU is INTEGER
  If RANGE='I', the indices (in ascending order) of the
  smallest and largest singular values to be returned.
  1 <= IL <= IU <= min(M,N), if min(M,N) > 0.
  Not referenced if RANGE = 'A' or 'V'.

NS
  NS is INTEGER
  The total number of singular values found,
  0 <= NS <= min(M,N).
  If RANGE = 'A', NS = min(M,N); if RANGE = 'I', NS = IU-IL+1.

S
  S is REAL array, dimension (min(M,N))
  The singular values of A, sorted so that S(i) >= S(i+1).

U
  U is COMPLEX array, dimension (LDU,UCOL)
  If JOBU = 'V', U contains columns of U (the left singular
  vectors, stored columnwise) as specified by RANGE; if
  JOBU = 'N', U is not referenced.
  Note: The user must ensure that UCOL >= NS; if RANGE = 'V',
  the exact value of NS is not known ILQFin advance and an upper
  bound must be used.

LDU
  LDU is INTEGER
  The leading dimension of the array U.  LDU >= 1; if
  JOBU = 'V', LDU >= M.

VT
  VT is COMPLEX array, dimension (LDVT,N)
  If JOBVT = 'V', VT contains the rows of V**T (the right singular
  vectors, stored rowwise) as specified by RANGE; if JOBVT = 'N',
```

```
  VT is not referenced.
  Note: The user must ensure that LDVT >= NS; if RANGE = 'V',
  the exact value of NS is not known in advance and an upper
  bound must be used.

LDVT
  LDVT is INTEGER
  The leading dimension of the array VT.  LDVT >= 1; if
  JOBVT = 'V', LDVT >= NS (see above).

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK;

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  LWORK >= MAX(1,MIN(M,N)*(MIN(M,N)+4)) for the paths (see
  comments inside the code):
  - PATH 1  (M much larger than N)
  - PATH 1t (N much larger than M)
  LWORK >= MAX(1,MIN(M,N)*2+MAX(M,N)) for the other paths.
  For good performance, LWORK should generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is REAL array, dimension (MAX(1,LRWORK))
  LRWORK >= MIN(M,N)*(MIN(M,N)*2+15*MIN(M,N)).

IWORK
  IWORK is INTEGER array, dimension (12*MIN(M,N))
  If INFO = 0, the first NS elements of IWORK are zero. If INFO > 0,
  then IWORK contains the indices of the eigenvectors that failed
  to converge in SBDSVDX/SSTEVX.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, then i eigenvectors failed to converge
        in SBDSVDX/SSTEVX. if INFO = N*2 + 1, an internal error occurred
        in SBDSVDX
```

———-

30.4.3 fortran code

— cgesvdx.f —

```
*  =====================================================================
      SUBROUTINE CGESVDX( JOBU, JOBVT, RANGE, M, N, A, LDA, VL, VU,
     $                    IL, IU, NS, S, U, LDU, VT, LDVT, WORK,
     $                    LWORK, RWORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBU, JOBVT, RANGE
      INTEGER            IL, INFO, IU, LDA, LDU, LDVT, LWORK, M, N, NS
      REAL               VL, VU
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      REAL               S( * ), RWORK( * )
      COMPLEX            A( LDA, * ), U( LDU, * ), VT( LDVT, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0E0, 0.0E0 ),
     $                   CONE = ( 1.0E0, 0.0E0 ) )
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
*     ..
*     .. Local Scalars ..
      CHARACTER          JOBZ, RNGTGK
      LOGICAL            ALLS, INDS, LQUERY, VALS, WANTU, WANTVT
      INTEGER            I, ID, IE, IERR, ILQF, ILTGK, IQRF, ISCL,
     $                   ITAU, ITAUP, ITAUQ, ITEMP, ITGKZ, IUTGK,
     $                   J, K, MAXWRK, MINMN, MINWRK, MNTHR
      REAL               ABSTOL, ANRM, BIGNUM, EPS, SMLNUM
*     ..
*     .. Local Arrays ..
      REAL               DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEBRD, CGELQF, CGEQRF, CLASCL, CLASET,
     $                   SLASCL, XERBLA
```

```
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               SLAMCH, CLANGE
      EXTERNAL           LSAME, ILAENV, SLAMCH, CLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      NS = 0
      INFO = 0
      ABSTOL = 2*SLAMCH('S')
      LQUERY = ( LWORK.EQ.-1 )
      MINMN = MIN( M, N )

      WANTU = LSAME( JOBU, 'V' )
      WANTVT = LSAME( JOBVT, 'V' )
      IF( WANTU .OR. WANTVT ) THEN
         JOBZ = 'V'
      ELSE
         JOBZ = 'N'
      END IF
      ALLS = LSAME( RANGE, 'A' )
      VALS = LSAME( RANGE, 'V' )
      INDS = LSAME( RANGE, 'I' )
*
      INFO = 0
      IF( .NOT.LSAME( JOBU, 'V' ) .AND.
     $    .NOT.LSAME( JOBU, 'N' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.LSAME( JOBVT, 'V' ) .AND.
     $         .NOT.LSAME( JOBVT, 'N' ) ) THEN
         INFO = -2
      ELSE IF( .NOT.( ALLS .OR. VALS .OR. INDS ) ) THEN
         INFO = -3
      ELSE IF( M.LT.0 ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( M.GT.LDA ) THEN
         INFO = -7
      ELSE IF( MINMN.GT.0 ) THEN
         IF( VALS ) THEN
            IF( VL.LT.ZERO ) THEN
               INFO = -8
```

```
            ELSE IF( VU.LE.VL ) THEN
               INFO = -9
            END IF
         ELSE IF( INDS ) THEN
            IF( IL.LT.1 .OR. IL.GT.MAX( 1, MINMN ) ) THEN
               INFO = -10
            ELSE IF( IU.LT.MIN( MINMN, IL ) .OR. IU.GT.MINMN ) THEN
               INFO = -11
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
            IF( WANTU .AND. LDU.LT.M ) THEN
               INFO = -15
            ELSE IF( WANTVT .AND. LDVT.LT.MINMN ) THEN
               INFO = -16
            END IF
         END IF
      END IF
*
*     Compute workspace
*     (Note: Comments in the code beginning "Workspace:" describe the
*     minimal amount of workspace needed at that point in the code,
*     as well as the preferred amount for good performance.
*     NB refers to the optimal block size for the immediately
*     following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = 1
         MAXWRK = 1
         IF( MINMN.GT.0 ) THEN
            IF( M.GE.N ) THEN
               MNTHR = ILAENV( 6, 'CGESVD', JOBU // JOBVT, M, N, 0, 0 )
               IF( M.GE.MNTHR ) THEN
*
*                Path 1 (M much larger than N)
*
                 MAXWRK = N + N*
     $                   ILAENV( 1, 'SGEQRF', ' ', M, N, -1, -1 )
                 MAXWRK = MAX( MAXWRK, N*N + N + 2*N*
     $                   ILAENV( 1, 'SGEBRD', ' ', N, N, -1, -1 ) )
                 MINWRK = N*(N+4)
               ELSE
*
*                Path 2 (M at least N, but not much larger)
*
                 MAXWRK = 2*N + ( M+N )*
     $                   ILAENV( 1, 'CGEBRD', ' ', M, N, -1, -1 )
                 MINWRK = 2*N + M
               END IF
            ELSE
```

```
               MNTHR = ILAENV( 6, 'CGESVD', JOBU // JOBVT, M, N, 0, 0 )
               IF( N.GE.MNTHR ) THEN
*
*                 Path 1t (N much larger than M)
*
                  MAXWRK = M + M*
     $                    ILAENV( 1, 'CGELQF', ' ', M, N, -1, -1 )
                  MAXWRK = MAX( MAXWRK, M*M + M + 2*M*
     $                    ILAENV( 1, 'CGEBRD', ' ', M, M, -1, -1 ) )
                  MINWRK = M*(M+4)
               ELSE
*
*                 Path 2t (N greater than M, but not much larger)
*
                  MAXWRK = M*(M*2+19) + ( M+N )*
     $                    ILAENV( 1, 'CGEBRD', ' ', M, N, -1, -1 )
                  MINWRK = 2*M + N
               END IF
            END IF
         END IF
         MAXWRK = MAX( MAXWRK, MINWRK )
         WORK( 1 ) = CMPLX( REAL( MAXWRK ), ZERO )
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -19
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGESVDX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         RETURN
      END IF
*
*     Set singular values indices accord to RANGE='A'.
*
      ALLS = LSAME( RANGE, 'A' )
      INDS = LSAME( RANGE, 'I' )
      IF( ALLS ) THEN
         RNGTGK = 'I'
         ILTGK = 1
         IUTGK = MIN( M, N )
      ELSE IF( INDS ) THEN
```

```
         RNGTGK = 'I'
         ILTGK = IL
         IUTGK = IU
      ELSE
         RNGTGK = 'V'
         ILTGK = 0
         IUTGK = 0
      END IF
*
*     Get machine constants
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SQRT( SLAMCH( 'S' ) ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = CLANGE( 'M', M, N, A, LDA, DUM )
      ISCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ISCL = 1
         CALL CLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ISCL = 1
         CALL CLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
      END IF
*
      IF( M.GE.N ) THEN
*
*        A has at least as many rows as columns. If A has sufficiently
*        more rows than columns, first reduce A using the QR
*        decomposition.
*
         IF( M.GE.MNTHR ) THEN
*
*           Path 1 (M much larger than N):
*           A = Q * R = Q * ( QB * B * PB**T )
*                     = Q * ( QB * ( UB * S * VB**T ) * PB**T )
*           U = Q * QB * UB; V**T = VB**T * PB**T
*
*           Compute A=Q*R
*           (Workspace: need 2*N, prefer N+N*NB)
*
            ITAU = 1
            ITEMP = ITAU + N
            CALL CGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( ITEMP ),
     $                   LWORK-ITEMP+1, INFO )
*
*           Copy R into WORK and bidiagonalize it:
*           (Workspace: need N*N+3*N, prefer N*N+N+2*N*NB)
```

```
*
            IQRF = ITEMP
            ITAUQ = ITEMP + N*N
            ITAUP = ITAUQ + N
            ITEMP = ITAUP + N
            ID = 1
            IE = ID + N
            ITGKZ = IE + N
            CALL CLACPY( 'U', N, N, A, LDA, WORK( IQRF ), N )
            CALL CLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                   WORK( IQRF+1 ), N )
            CALL CGEBRD( N, N, WORK( IQRF ), N, RWORK( ID ),
     $                   RWORK( IE ), WORK( ITAUQ ), WORK( ITAUP ),
     $                   WORK( ITEMP ), LWORK-ITEMP+1, INFO )
            ITEMP = ITGKZ + N*(N*2+1)
*
*           Solve eigenvalue problem TGK*Z=Z*S.
*           (Workspace: need 2*N*N+14*N)
*
            CALL SBDSVDX( 'U', JOBZ, RNGTGK, N, RWORK( ID ),
     $                    RWORK( IE ), VL, VU, ILTGK, IUTGK, NS, S,
     $                    RWORK( ITGKZ ), N*2, RWORK( ITEMP ),
     $                    IWORK, INFO)
*
*           If needed, compute left singular vectors.
*
            IF( WANTU ) THEN
               K = ITGKZ
               DO I = 1, NS
                  DO J = 1, N
                     U( J, I ) = CMPLX( RWORK( K ), ZERO )
                     K = K + 1
                  END DO
                  K = K + N
               END DO
               CALL CLASET( 'A', M-N, N, CZERO, CZERO, U( N+1,1 ), LDU )
*
*              Call CUNMBR to compute QB*UB.
*              (Workspace in WORK( ITEMP ): need N, prefer N*NB)
*
               CALL CUNMBR( 'Q', 'L', 'N', N, NS, N, WORK( IQRF ), N,
     $                      WORK( ITAUQ ), U, LDU, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
*
*              Call CUNMQR to compute Q*(QB*UB).
*              (Workspace in WORK( ITEMP ): need N, prefer N*NB)
*
               CALL CUNMQR( 'L', 'N', M, NS, N, A, LDA,
     $                      WORK( ITAU ), U, LDU, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
```

```
            END IF
*
*           If needed, compute right singular vectors.
*
            IF( WANTVT) THEN
               K = ITGKZ + N
               DO I = 1, NS
                  DO J = 1, N
                     VT( I, J ) = CMPLX( RWORK( K ), ZERO )
                     K = K + 1
                  END DO
                  K = K + N
               END DO
*
*              Call CUNMBR to compute VB**T * PB**T
*              (Workspace in WORK( ITEMP ): need N, prefer N*NB)
*
               CALL CUNMBR( 'P', 'R', 'C', NS, N, N, WORK( IQRF ), N,
     $                      WORK( ITAUP ), VT, LDVT, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
            END IF
         ELSE
*
*           Path 2 (M at least N, but not much larger)
*           Reduce A to bidiagonal form without QR decomposition
*           A = QB * B * PB**T = QB * ( UB * S * VB**T ) * PB**T
*           U = QB * UB; V**T = VB**T * PB**T
*
*           Bidiagonalize A
*           (Workspace: need 2*N+M, prefer 2*N+(M+N)*NB)
*
            ITAUQ = 1
            ITAUP = ITAUQ + N
            ITEMP = ITAUP + N
            ID = 1
            IE = ID + N
            ITGKZ = IE + N
            CALL CGEBRD( M, N, A, LDA, RWORK( ID ), RWORK( IE ),
     $                   WORK( ITAUQ ), WORK( ITAUP ), WORK( ITEMP ),
     $                   LWORK-ITEMP+1, INFO )
            ITEMP = ITGKZ + N*(N*2+1)
*
*           Solve eigenvalue problem TGK*Z=Z*S.
*           (Workspace: need 2*N*N+14*N)
*
            CALL SBDSVDX( 'U', JOBZ, RNGTGK, N, RWORK( ID ),
     $                    RWORK( IE ), VL, VU, ILTGK, IUTGK, NS, S,
     $                    RWORK( ITGKZ ), N*2, RWORK( ITEMP ),
     $                    IWORK, INFO)
*
```

```
*          If needed, compute left singular vectors.
*
           IF( WANTU ) THEN
              K = ITGKZ
              DO I = 1, NS
                 DO J = 1, N
                    U( J, I ) = CMPLX( RWORK( K ), ZERO )
                    K = K + 1
                 END DO
                 K = K + N
              END DO
              CALL CLASET( 'A', M-N, N, CZERO, CZERO, U( N+1,1 ), LDU )
*
*             Call CUNMBR to compute QB*UB.
*             (Workspace in WORK( ITEMP ): need N, prefer N*NB)
*
              CALL CUNMBR( 'Q', 'L', 'N', M, NS, N, A, LDA,
     $                     WORK( ITAUQ ), U, LDU, WORK( ITEMP ),
     $                     LWORK-ITEMP+1, IERR )
           END IF
*
*          If needed, compute right singular vectors.
*
           IF( WANTVT) THEN
              K = ITGKZ + N
              DO I = 1, NS
                 DO J = 1, N
                    VT( I, J ) = CMPLX( RWORK( K ), ZERO )
                    K = K + 1
                 END DO
                 K = K + N
              END DO
*
*             Call CUNMBR to compute VB**T * PB**T
*             (Workspace in WORK( ITEMP ): need N, prefer N*NB)
*
              CALL CUNMBR( 'P', 'R', 'C', NS, N, N, A, LDA,
     $                     WORK( ITAUP ), VT, LDVT, WORK( ITEMP ),
     $                     LWORK-ITEMP+1, IERR )
           END IF
        END IF
     ELSE
*
*       A has more columns than rows. If A has sufficiently more
*       columns than rows, first reduce A using the LQ decomposition.
*
        IF( N.GE.MNTHR ) THEN
*
*          Path 1t (N much larger than M):
*          A = L * Q = ( QB * B * PB**T ) * Q
```

```
*                    = ( QB * ( UB * S * VB**T ) * PB**T ) * Q
*          U = QB * UB ; V**T = VB**T * PB**T * Q
*
*          Compute A=L*Q
*          (Workspace: need 2*M, prefer M+M*NB)
*
           ITAU = 1
           ITEMP = ITAU + M
           CALL CGELQF( M, N, A, LDA, WORK( ITAU ), WORK( ITEMP ),
     $                  LWORK-ITEMP+1, INFO )

*          Copy L into WORK and bidiagonalize it:
*          (Workspace in WORK( ITEMP ): need M*M+3*M, prefer M*M+M+2*M*NB)
*
           ILQF = ITEMP
           ITAUQ = ILQF + M*M
           ITAUP = ITAUQ + M
           ITEMP = ITAUP + M
           ID = 1
           IE = ID + M
           ITGKZ = IE + M
           CALL CLACPY( 'L', M, M, A, LDA, WORK( ILQF ), M )
           CALL CLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                  WORK( ILQF+M ), M )
           CALL CGEBRD( M, M, WORK( ILQF ), M, RWORK( ID ),
     $                  RWORK( IE ), WORK( ITAUQ ), WORK( ITAUP ),
     $                  WORK( ITEMP ), LWORK-ITEMP+1, INFO )
           ITEMP = ITGKZ + M*(M*2+1)
*
*          Solve eigenvalue problem TGK*Z=Z*S.
*          (Workspace: need 2*M*M+14*M)
*
           CALL SBDSVDX( 'U', JOBZ, RNGTGK, M, RWORK( ID ),
     $                   RWORK( IE ), VL, VU, ILTGK, IUTGK, NS, S,
     $                   RWORK( ITGKZ ), M*2, RWORK( ITEMP ),
     $                   IWORK, INFO)
*
*          If needed, compute left singular vectors.
*
           IF( WANTU ) THEN
              K = ITGKZ
              DO I = 1, NS
                 DO J = 1, M
                    U( J, I ) = CMPLX( RWORK( K ), ZERO )
                    K = K + 1
                 END DO
                 K = K + M
              END DO
*
*             Call CUNMBR to compute QB*UB.
```

```
*              (Workspace in WORK( ITEMP ): need M, prefer M*NB)
*
               CALL CUNMBR( 'Q', 'L', 'N', M, NS, M, WORK( ILQF ), M,
     $                      WORK( ITAUQ ), U, LDU, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
            END IF
*
*           If needed, compute right singular vectors.
*
            IF( WANTVT) THEN
               K = ITGKZ + M
               DO I = 1, NS
                  DO J = 1, M
                     VT( I, J ) = CMPLX( RWORK( K ), ZERO )
                     K = K + 1
                  END DO
                  K = K + M
               END DO
               CALL CLASET( 'A', M, N-M, CZERO, CZERO,
     $                      VT( 1,M+1 ), LDVT )
*
*              Call CUNMBR to compute (VB**T)*(PB**T)
*              (Workspace in WORK( ITEMP ): need M, prefer M*NB)
*
               CALL CUNMBR( 'P', 'R', 'C', NS, M, M, WORK( ILQF ), M,
     $                      WORK( ITAUP ), VT, LDVT, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
*
*              Call CUNMLQ to compute ((VB**T)*(PB**T))*Q.
*              (Workspace in WORK( ITEMP ): need M, prefer M*NB)
*
               CALL CUNMLQ( 'R', 'N', NS, N, M, A, LDA,
     $                      WORK( ITAU ), VT, LDVT, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
            END IF
         ELSE
*
*           Path 2t (N greater than M, but not much larger)
*           Reduce to bidiagonal form without LQ decomposition
*           A = QB * B * PB**T = QB * ( UB * S * VB**T ) * PB**T
*           U = QB * UB; V**T = VB**T * PB**T
*
*           Bidiagonalize A
*           (Workspace: need 2*M+N, prefer 2*M+(M+N)*NB)
*
            ITAUQ = 1
            ITAUP = ITAUQ + M
            ITEMP = ITAUP + M
            ID = 1
            IE = ID + M
```

```
            ITGKZ = IE + M
            CALL CGEBRD( M, N, A, LDA, RWORK( ID ), RWORK( IE ),
     $                   WORK( ITAUQ ), WORK( ITAUP ), WORK( ITEMP ),
     $                   LWORK-ITEMP+1, INFO )
            ITEMP = ITGKZ + M*(M*2+1)
*
*           Solve eigenvalue problem TGK*Z=Z*S.
*           (Workspace: need 2*M*M+14*M)
*
            CALL SBDSVDX( 'L', JOBZ, RNGTGK, M, RWORK( ID ),
     $                    RWORK( IE ), VL, VU, ILTGK, IUTGK, NS, S,
     $                    RWORK( ITGKZ ), M*2, RWORK( ITEMP ),
     $                    IWORK, INFO)
*
*           If needed, compute left singular vectors.
*
            IF( WANTU ) THEN
               K = ITGKZ
               DO I = 1, NS
                  DO J = 1, M
                     U( J, I ) = CMPLX( RWORK( K ), ZERO )
                     K = K + 1
                  END DO
                  K = K + M
               END DO
*
*              Call CUNMBR to compute QB*UB.
*              (Workspace in WORK( ITEMP ): need M, prefer M*NB)
*
               CALL CUNMBR( 'Q', 'L', 'N', M, NS, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
            END IF
*
*           If needed, compute right singular vectors.
*
            IF( WANTVT) THEN
               K = ITGKZ + M
               DO I = 1, NS
                  DO J = 1, M
                     VT( I, J ) = CMPLX( RWORK( K ), ZERO )
                     K = K + 1
                  END DO
                  K = K + M
               END DO
               CALL CLASET( 'A', M, N-M, CZERO, CZERO,
     $                      VT( 1,M+1 ), LDVT )
*
*              Call CUNMBR to compute VB**T * PB**T
*              (Workspace in WORK( ITEMP ): need M, prefer M*NB)
```

```
*
               CALL CUNMBR( 'P', 'R', 'C', NS, N, M, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
            END IF
         END IF
      END IF
*
*     Undo scaling if necessary
*
      IF( ISCL.EQ.1 ) THEN
         IF( ANRM.GT.BIGNUM )
     $      CALL SLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1,
     $                   S, MINMN, INFO )
         IF( ANRM.LT.SMLNUM )
     $      CALL SLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1,
     $                   S, MINMN, INFO )
      END IF
*
*     Return optimal workspace in WORK(1)
*
      WORK( 1 ) = CMPLX( REAL( MAXWRK ), ZERO )
*
      RETURN
*
*     End of CGESVDX
*
      END
```

———-

30.4.4 lisp code

— **cgesvdx.lisp** —

```
(let* ((czero (f2cl-lib:cmplx 0.0 0.0)) (zero 0.0) (one 1.0))
  (declare (type (f2cl-lib:complex8) czero)
           (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable czero zero one))
  (defun cgesvdx
         (jobu jobvt range m n a lda vl vu il iu ns s u ldu vt ldvt work lwork
          rwork iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (array single-float (*)) rwork s)
             (type (single-float) vu vl)
             (type (array f2cl-lib:complex8 (*)) work vt u a)
             (type (f2cl-lib:integer4) info lwork ldvt ldu ns iu il lda n m)
```

```
          (type (string *) range jobvt jobu))
   (f2cl-lib:with-multi-array-data
       ((jobu character jobu-%data% jobu-%offset%)
        (jobvt character jobvt-%data% jobvt-%offset%)
        (range character range-%data% range-%offset%)
        (a f2cl-lib:complex8 a-%data% a-%offset%)
        (u f2cl-lib:complex8 u-%data% u-%offset%)
        (vt f2cl-lib:complex8 vt-%data% vt-%offset%)
        (work f2cl-lib:complex8 work-%data% work-%offset%)
        (s single-float s-%data% s-%offset%)
        (rwork single-float rwork-%data% rwork-%offset%)
        (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
     (prog ((dum (make-array 1 :element-type 'single-float)) (abstol 0.0)
            (anrm 0.0) (bignum 0.0) (eps 0.0) (smlnum 0.0) (i 0) (id 0) (ie 0)
            (ierr 0) (ilqf 0) (iltgk 0) (iqrf 0) (iscl 0) (itau 0) (itaup 0)
            (itauq 0) (itemp 0) (itgkz 0) (iutgk 0) (j 0) (k 0) (maxwrk 0)
            (minmn 0) (minwrk 0) (mnthr 0) (alls nil) (inds nil) (lquery nil)
            (vals nil) (wantu nil) (wantvt nil)
            (jobz
             (make-array '(1) :element-type 'character :initial-element #\ ))
            (rngtgk
             (make-array '(1) :element-type 'character :initial-element #\ )))
       (declare (type (array single-float (1)) dum)
                (type (single-float) abstol anrm bignum eps smlnum)
                (type (f2cl-lib:integer4) i id ie ierr ilqf iltgk iqrf iscl
                 itau itaup itauq itemp itgkz iutgk j k maxwrk minmn minwrk
                 mnthr)
                (type f2cl-lib:logical alls inds lquery vals wantu wantvt)
                (type (string 1) jobz rngtgk))
       (setf ns 0)
       (setf info 0)
       (setf abstol (* 2 (slamch "S")))
       (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
       (setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
       (setf wantu
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame jobu "V")
                 (declare (ignore var-1))
                 (when var-0 (setf jobu var-0))
                 ret-val))
       (setf wantvt
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame jobvt "V")
                 (declare (ignore var-1))
                 (when var-0 (setf jobvt var-0))
                 ret-val))
       (cond
        ((or wantu wantvt) (f2cl-lib:f2cl-set-string jobz "V" (string 1)))
        (t (f2cl-lib:f2cl-set-string jobz "N" (string 1))))
       (setf alls
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "A")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf vals
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "V")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf inds
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "I")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf info 0)
(cond
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame jobu "V")
      (declare (ignore var-1))
      (when var-0 (setf jobu var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame jobu "N")
      (declare (ignore var-1))
      (when var-0 (setf jobu var-0))
      ret-val)))
  (setf info -1))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame jobvt "V")
      (declare (ignore var-1))
      (when var-0 (setf jobvt var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame jobvt "N")
      (declare (ignore var-1))
      (when var-0 (setf jobvt var-0))
      ret-val)))
  (setf info -2))
 ((not (or alls vals inds)) (setf info -3)) ((< m 0) (setf info -4))
 ((< n 0) (setf info -5)) ((> m lda) (setf info -7))
 ((> minmn 0)
```

```
    (cond
     (vals
      (cond ((< vl zero) (setf info -8)) ((<= vu vl) (setf info -9))))
     (inds
      (cond
       ((or (< il 1)
            (> il
               (max (the f2cl-lib:integer4 1)
                    (the f2cl-lib:integer4 minmn))))
        (setf info -10))
       ((or
         (< iu
            (min (the f2cl-lib:integer4 minmn)
                 (the f2cl-lib:integer4 il)))
         (> iu minmn))
        (setf info -11)))))
    (cond
     ((= info 0)
      (cond ((and wantu (< ldu m)) (setf info -15))
            ((and wantvt (< ldvt minmn)) (setf info -16)))))))
  (cond
   ((= info 0) (setf minwrk 1) (setf maxwrk 1)
    (cond
     ((> minmn 0)
      (cond
       ((>= m n)
        (setf mnthr
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (ilaenv 6 "CGESVD" (f2cl-lib:f2cl-// jobu jobvt) m n
                     0 0)
                  (declare (ignore var-0 var-1 var-2 var-5 var-6))
                  (when var-3 (setf m var-3))
                  (when var-4 (setf n var-4))
                  ret-val))
        (cond
         ((>= m mnthr)
          (setf maxwrk
                  (f2cl-lib:int-add n
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val var-0
                                                           var-1 var-2
                                                           var-3 var-4
                                                           var-5 var-6)
                                                          (ilaenv 1
                                                           "SGEQRF" " " m
                                                           n -1 -1)
                                                        (declare
                                                         (ignore var-0
```

```
                                                        var-1 var-2
                                                        var-5 var-6))
                                                      (when var-3
                                                        (setf m var-3))
                                                      (when var-4
                                                        (setf n var-4))
                                                      ret-val))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul n n) n
                                     (f2cl-lib:int-mul 2 n
                                                       (multiple-value-bind
                                                           (ret-val
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-3
                                                            var-4
                                                            var-5
                                                            var-6)
                                                           (ilaenv
                                                            1
                                                            "SGEBRD"
                                                            " "
                                                            n n
                                                            -1
                                                            -1)
                                                         (declare
                                                          (ignore
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-5
                                                           var-6))
                                                         (when
                                                             var-3
                                                           (setf n
                                                                   var-3))
                                                         (when
                                                             var-4
                                                           (setf n
                                                                   var-4))
                                                         ret-val))))))
 (setf minwrk (f2cl-lib:int-mul n (f2cl-lib:int-add n 4))))
(t
 (setf maxwrk
         (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-add m n)
```

```
                                  (multiple-value-bind
                                      (ret-val var-0 var-1 var-2 var-3
                                       var-4 var-5 var-6)
                                      (ilaenv 1 "CGEBRD" " " m n -1
                                       -1)
                                    (declare
                                     (ignore var-0 var-1 var-2 var-5
                                      var-6))
                                    (when var-3 (setf m var-3))
                                    (when var-4 (setf n var-4))
                                    ret-val))))
    (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))))
  (t
   (setf mnthr
           (multiple-value-bind
               (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
               (ilaenv 6 "CGESVD" (f2cl-lib:f2cl-// jobu jobvt) m n
                0 0)
             (declare (ignore var-0 var-1 var-2 var-5 var-6))
             (when var-3 (setf m var-3))
             (when var-4 (setf n var-4))
             ret-val))
   (cond
    ((>= n mnthr)
     (setf maxwrk
             (f2cl-lib:int-add m
                               (f2cl-lib:int-mul m
                                                 (multiple-value-bind
                                                     (ret-val var-0
                                                      var-1 var-2
                                                      var-3 var-4
                                                      var-5 var-6)
                                                     (ilaenv 1
                                                      "CGELQF" " " m
                                                      n -1 -1)
                                                   (declare
                                                    (ignore var-0
                                                     var-1 var-2
                                                     var-5 var-6))
                                                   (when var-3
                                                     (setf m var-3))
                                                   (when var-4
                                                     (setf n var-4))
                                                   ret-val))))
     (setf maxwrk
             (max (the f2cl-lib:integer4 maxwrk)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add (f2cl-lib:int-mul m m) m
                                         (f2cl-lib:int-mul 2 m
                                                           (multiple-value-bind
```

```
                                                                    (ret-val
                                                                     var-0
                                                                     var-1
                                                                     var-2
                                                                     var-3
                                                                     var-4
                                                                     var-5
                                                                     var-6)
                                                                    (ilaenv
                                                                     1
                                                                     "CGEBRD"
                                                                     " "
                                                                     m m
                                                                     -1
                                                                     -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-5
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf m
                                                                            var-4))
                                                                  ret-val))))))
                                  (setf minwrk (f2cl-lib:int-mul m (f2cl-lib:int-add m 4))))
                                 (t
                                  (setf maxwrk
                                          (f2cl-lib:int-add
                                           (f2cl-lib:int-mul m
                                                             (f2cl-lib:int-add
                                                              (f2cl-lib:int-mul m 2) 19))
                                           (f2cl-lib:int-mul (f2cl-lib:int-add m n)
                                                             (multiple-value-bind
                                                                 (ret-val var-0 var-1 var-2 var-3
                                                                  var-4 var-5 var-6)
                                                                 (ilaenv 1 "CGEBRD" " " m n -1
                                                                  -1)
                                                               (declare
                                                                (ignore var-0 var-1 var-2 var-5
                                                                 var-6))
                                                               (when var-3 (setf m var-3))
                                                               (when var-4 (setf n var-4))
                                                               ret-val))))
```

```
        (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4 minwrk)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:cmplx (f2cl-lib:freal maxwrk) zero))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -19)))))
(cond
 ((/= info 0) (xerbla "CGESVDX" (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(cond ((or (= m 0) (= n 0)) (go end_label)))
(setf alls
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "A")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf inds
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "I")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(cond
 (alls (f2cl-lib:f2cl-set-string rngtgk "I" (string 1)) (setf iltgk 1)
  (setf iutgk
          (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n))))
 (inds (f2cl-lib:f2cl-set-string rngtgk "I" (string 1)) (setf iltgk il)
  (setf iutgk iu))
 (t (f2cl-lib:f2cl-set-string rngtgk "V" (string 1)) (setf iltgk 0)
  (setf iutgk 0)))
(setf eps (slamch "P"))
(setf smlnum (/ (f2cl-lib:fsqrt (slamch "S")) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clange "M" m n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iscl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf iscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl "G" 0 0 anrm smlnum m n a lda info)
```

```
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf smlnum var-4))
      (when var-5 (setf m var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf info var-9))))
   ((> anrm bignum) (setf iscl 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (clascl "G" 0 0 anrm bignum m n a lda info)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf bignum var-4))
      (when var-5 (setf m var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf info var-9)))))
  (cond
   ((>= m n)
    (cond
     ((>= m mnthr) (setf itau 1) (setf itemp (f2cl-lib:int-add itau n))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (cgeqrf m n a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itemp)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
        (declare (ignore var-2 var-4 var-5 var-6))
        (setf m var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf info var-7))
      (setf iqrf itemp)
      (setf itauq (f2cl-lib:int-add itemp (f2cl-lib:int-mul n n)))
      (setf itaup (f2cl-lib:int-add itauq n))
      (setf itemp (f2cl-lib:int-add itaup n)) (setf id 1)
      (setf ie (f2cl-lib:int-add id n))
      (setf itgkz (f2cl-lib:int-add ie n))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (clacpy "U" n n a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iqrf)
                                 ((1 *)) work-%offset%)
           n)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
```

```
  (when var-6 (setf n var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1) czero
     czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           ((+ iqrf 1)) ((1 *)) work-%offset%)
     n)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf n var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iqrf)
                           ((1 *)) work-%offset%)
     n
     (f2cl-lib:array-slice rwork-%data% single-float (id) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itemp)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf n var-3)
  (setf info var-10))
(setf itemp
        (f2cl-lib:int-add itgkz
                          (f2cl-lib:int-mul n
                                            (f2cl-lib:int-add
                                             (f2cl-lib:int-mul n 2)
                                             1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (sbdsvdx "U" jobz rngtgk n
     (f2cl-lib:array-slice rwork-%data% single-float (id) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     vl vu iltgk iutgk ns s
     (f2cl-lib:array-slice rwork-%data% single-float (itgkz)
```

```
                              ((1 *)) rwork-%offset%)
        (f2cl-lib:int-mul n 2)
        (f2cl-lib:array-slice rwork-%data% single-float (itemp)
                              ((1 *)) rwork-%offset%)
        iwork info)
    (declare
     (ignore var-0 var-4 var-5 var-11 var-12 var-13 var-14 var-15))
    (when var-1 (setf jobz var-1))
    (when var-2 (setf rngtgk var-2))
    (when var-3 (setf n var-3))
    (when var-6 (setf vl var-6))
    (when var-7 (setf vu var-7))
    (when var-8 (setf iltgk var-8))
    (when var-9 (setf iutgk var-9))
    (when var-10 (setf ns var-10))
    (when var-16 (setf info var-16)))
  (cond
   (wantu (setf k itgkz)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i ns) nil)
      (tagbody
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf (f2cl-lib:fref u-%data% (j i) ((1 ldu) (1 *))
                                 u-%offset%)
                  (f2cl-lib:cmplx
                   (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                  rwork-%offset%)
                   zero))
            (setf k (f2cl-lib:int-add k 1))
           label100001))
        (setf k (f2cl-lib:int-add k n))
       label100000))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "A" (f2cl-lib:int-sub m n) n czero czero
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 ((+ n 1) 1)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf n var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldu var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (cunmbr "Q" "L" "N" n ns n
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iqrf)
                               ((1 *)) work-%offset%)
```

```
          n
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                                ((1 *)) work-%offset%)
          u ldu
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itemp)
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
        (declare
         (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
        (when var-3 (setf n var-3))
        (when var-4 (setf ns var-4))
        (when var-5 (setf n var-5))
        (when var-7 (setf n var-7))
        (when var-10 (setf ldu var-10))
        (when var-13 (setf info var-13)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (cunmqr "L" "N" m ns n a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                                 ((1 *)) work-%offset%)
           u ldu
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itemp)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf ns var-3))
        (when var-4 (setf n var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldu var-9))
        (when var-12 (setf info var-12)))))
    (cond
     (wantvt (setf k (f2cl-lib:int-add itgkz n))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i ns) nil)
        (tagbody
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf (f2cl-lib:fref vt-%data% (i j) ((1 ldvt) (1 *))
                                   vt-%offset%)
                    (f2cl-lib:cmplx
                     (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                    rwork-%offset%)
                     zero))
              (setf k (f2cl-lib:int-add k 1))
             label100003))
          (setf k (f2cl-lib:int-add k n))
         label100002))
```

```
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13)
          (cunmbr "P" "R" "C" ns n n
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (iqrf)
                                 ((1 *)) work-%offset%)
           n
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                                 ((1 *)) work-%offset%)
           vt ldvt
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itemp)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
        (declare
         (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
        (when var-3 (setf ns var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf n var-5))
        (when var-7 (setf n var-7))
        (when var-10 (setf ldvt var-10))
        (when var-13 (setf info var-13))))))
    (t (setf itauq 1) (setf itaup (f2cl-lib:int-add itauq n))
     (setf itemp (f2cl-lib:int-add itaup n)) (setf id 1)
     (setf ie (f2cl-lib:int-add id n))
     (setf itgkz (f2cl-lib:int-add ie n))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (cgebrd m n a lda
          (f2cl-lib:array-slice rwork-%data% single-float (id) ((1 *))
                                rwork-%offset%)
          (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                                rwork-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itemp)
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
       (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
       (setf m var-0)
       (setf n var-1)
       (setf lda var-3)
       (setf info var-10))
     (setf itemp
             (f2cl-lib:int-add itgkz
                               (f2cl-lib:int-mul n
                                                 (f2cl-lib:int-add
                                                  (f2cl-lib:int-mul n 2)
```

```
                                                1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (sbdsvdx "U" jobz rngtgk n
     (f2cl-lib:array-slice rwork-%data% single-float (id) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     vl vu iltgk iutgk ns s
     (f2cl-lib:array-slice rwork-%data% single-float (itgkz)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:int-mul n 2)
     (f2cl-lib:array-slice rwork-%data% single-float (itemp)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-4 var-5 var-11 var-12 var-13 var-14 var-15))
  (when var-1 (setf jobz var-1))
  (when var-2 (setf rngtgk var-2))
  (when var-3 (setf n var-3))
  (when var-6 (setf vl var-6))
  (when var-7 (setf vu var-7))
  (when var-8 (setf iltgk var-8))
  (when var-9 (setf iutgk var-9))
  (when var-10 (setf ns var-10))
  (when var-16 (setf info var-16)))
(cond
 (wantu (setf k itgkz)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf (f2cl-lib:fref u-%data% (j i) ((1 ldu) (1 *))
                                u-%offset%)
                  (f2cl-lib:cmplx
                   (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                  rwork-%offset%)
                   zero))
          (setf k (f2cl-lib:int-add k 1))
         label100005))
      (setf k (f2cl-lib:int-add k n))
     label100004))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "A" (f2cl-lib:int-sub m n) n czero czero
       (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 ((+ n 1) 1)
                             ((1 ldu) (1 *)) u-%offset%)
       ldu)
```

```
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf n var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldu var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (cunmbr "Q" "L" "N" m ns n a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                               ((1 *)) work-%offset%)
         u ldu
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itemp)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) ierr)
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
      (when var-3 (setf m var-3))
      (when var-4 (setf ns var-4))
      (when var-5 (setf n var-5))
      (when var-7 (setf lda var-7))
      (when var-10 (setf ldu var-10))
      (when var-13 (setf ierr var-13)))))
  (cond
   (wantvt (setf k (f2cl-lib:int-add itgkz n))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i ns) nil)
      (tagbody
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf (f2cl-lib:fref vt-%data% (i j) ((1 ldvt) (1 *))
                                 vt-%offset%)
                  (f2cl-lib:cmplx
                   (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                  rwork-%offset%)
                   zero))
            (setf k (f2cl-lib:int-add k 1))
           label100007))
        (setf k (f2cl-lib:int-add k n))
       label100006))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (cunmbr "P" "R" "C" ns n n a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                               ((1 *)) work-%offset%)
         vt ldvt
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itemp)
                               ((1 *)) work-%offset%)
```

```
            (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) ierr)
        (declare
         (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
        (when var-3 (setf ns var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf n var-5))
        (when var-7 (setf lda var-7))
        (when var-10 (setf ldvt var-10))
        (when var-13 (setf ierr var-13))))))))
(t
 (cond
  ((>= n mnthr) (setf itau 1) (setf itemp (f2cl-lib:int-add itau m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (cgelqf m n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itemp)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
     (declare (ignore var-2 var-4 var-5 var-6))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf info var-7))
   (setf ilqf itemp)
   (setf itauq (f2cl-lib:int-add ilqf (f2cl-lib:int-mul m m)))
   (setf itaup (f2cl-lib:int-add itauq m))
   (setf itemp (f2cl-lib:int-add itaup m)) (setf id 1)
   (setf ie (f2cl-lib:int-add id m))
   (setf itgkz (f2cl-lib:int-add ie m))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (clacpy "L" m m a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ilqf)
                              ((1 *)) work-%offset%)
        m)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf m var-1))
     (when var-2 (setf m var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf m var-6)))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (claset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1) czero
        czero
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                              ((+ ilqf m)) ((1 *)) work-%offset%)
        m)
     (declare (ignore var-0 var-1 var-2 var-5))
     (when var-3 (setf czero var-3))
     (when var-4 (setf czero var-4))
```

```
  (when var-6 (setf m var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ilqf)
                           ((1 *)) work-%offset%)
     m
     (f2cl-lib:array-slice rwork-%data% single-float (id) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itemp)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf m var-3)
  (setf info var-10))
(setf itemp
        (f2cl-lib:int-add itgkz
                          (f2cl-lib:int-mul m
                                            (f2cl-lib:int-add
                                             (f2cl-lib:int-mul m 2)
                                             1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (sbdsvdx "U" jobz rngtgk m
     (f2cl-lib:array-slice rwork-%data% single-float (id) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                           rwork-%offset%)
     vl vu iltgk iutgk ns s
     (f2cl-lib:array-slice rwork-%data% single-float (itgkz)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:int-mul m 2)
     (f2cl-lib:array-slice rwork-%data% single-float (itemp)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-4 var-5 var-11 var-12 var-13 var-14 var-15))
  (when var-1 (setf jobz var-1))
  (when var-2 (setf rngtgk var-2))
  (when var-3 (setf m var-3))
```

```
  (when var-6 (setf vl var-6))
  (when var-7 (setf vu var-7))
  (when var-8 (setf iltgk var-8))
  (when var-9 (setf iutgk var-9))
  (when var-10 (setf ns var-10))
  (when var-16 (setf info var-16)))
(cond
 (wantu (setf k itgkz)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j m) nil)
        (tagbody
          (setf (f2cl-lib:fref u-%data% (j i) ((1 ldu) (1 *))
                                u-%offset%)
                  (f2cl-lib:cmplx
                   (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                  rwork-%offset%)
                   zero))
          (setf k (f2cl-lib:int-add k 1))
         label100009))
      (setf k (f2cl-lib:int-add k m))
     label100008))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (cunmbr "Q" "L" "N" m ns m
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ilqf)
                             ((1 *)) work-%offset%)
       m
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                             ((1 *)) work-%offset%)
       u ldu
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itemp)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf ns var-4))
    (when var-5 (setf m var-5))
    (when var-7 (setf m var-7))
    (when var-10 (setf ldu var-10))
    (when var-13 (setf info var-13)))))
(cond
 (wantvt (setf k (f2cl-lib:int-add itgkz m))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
```

```
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j m) nil)
      (tagbody
        (setf (f2cl-lib:fref vt-%data% (i j) ((1 ldvt) (1 *))
                             vt-%offset%)
              (f2cl-lib:cmplx
               (f2cl-lib:fref rwork-%data% (k) ((1 *))
                              rwork-%offset%)
               zero))
        (setf k (f2cl-lib:int-add k 1))
       label100011))
    (setf k (f2cl-lib:int-add k m))
   label100010))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claset "A" m (f2cl-lib:int-sub n m) czero czero
     (f2cl-lib:array-slice vt-%data% f2cl-lib:complex8
                           (1 (f2cl-lib:int-add m 1))
                           ((1 ldvt) (1 *)) vt-%offset%)
     ldvt)
  (declare (ignore var-0 var-2 var-5))
  (when var-1 (setf m var-1))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (cunmbr "P" "R" "C" ns m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (ilqf)
                           ((1 *)) work-%offset%)
     m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itemp)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf ns var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf m var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (cunmlq "R" "N" ns n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itau)
```

```
                                ((1 *)) work-%offset%)
            vt ldvt
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itemp)
                                ((1 *)) work-%offset%)
            (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf ns var-2))
        (when var-3 (setf n var-3))
        (when var-4 (setf m var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldvt var-9))
        (when var-12 (setf info var-12))))))
    (t (setf itauq 1) (setf itaup (f2cl-lib:int-add itauq m))
     (setf itemp (f2cl-lib:int-add itaup m)) (setf id 1)
     (setf ie (f2cl-lib:int-add id m))
     (setf itgkz (f2cl-lib:int-add ie m))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (cgebrd m n a lda
          (f2cl-lib:array-slice rwork-%data% single-float (id) ((1 *))
                                rwork-%offset%)
          (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                                rwork-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itemp)
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
       (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
       (setf m var-0)
       (setf n var-1)
       (setf lda var-3)
       (setf info var-10))
     (setf itemp
             (f2cl-lib:int-add itgkz
                               (f2cl-lib:int-mul m
                                                 (f2cl-lib:int-add
                                                  (f2cl-lib:int-mul m 2)
                                                  1))))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12 var-13 var-14 var-15 var-16)
         (sbdsvdx "L" jobz rngtgk m
          (f2cl-lib:array-slice rwork-%data% single-float (id) ((1 *))
                                rwork-%offset%)
          (f2cl-lib:array-slice rwork-%data% single-float (ie) ((1 *))
                                rwork-%offset%)
```

```
      vl vu iltgk iutgk ns s
      (f2cl-lib:array-slice rwork-%data% single-float (itgkz)
                            ((1 *)) rwork-%offset%)
      (f2cl-lib:int-mul m 2)
      (f2cl-lib:array-slice rwork-%data% single-float (itemp)
                            ((1 *)) rwork-%offset%)
      iwork info)
  (declare
   (ignore var-0 var-4 var-5 var-11 var-12 var-13 var-14 var-15))
  (when var-1 (setf jobz var-1))
  (when var-2 (setf rngtgk var-2))
  (when var-3 (setf m var-3))
  (when var-6 (setf vl var-6))
  (when var-7 (setf vu var-7))
  (when var-8 (setf iltgk var-8))
  (when var-9 (setf iutgk var-9))
  (when var-10 (setf ns var-10))
  (when var-16 (setf info var-16)))
(cond
 (wantu (setf k itgkz)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j m) nil)
        (tagbody
          (setf (f2cl-lib:fref u-%data% (j i) ((1 ldu) (1 *))
                                u-%offset%)
                  (f2cl-lib:cmplx
                   (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                  rwork-%offset%)
                   zero))
          (setf k (f2cl-lib:int-add k 1))
         label100013))
      (setf k (f2cl-lib:int-add k m))
     label100012))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (cunmbr "Q" "L" "N" m ns n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itauq)
                             ((1 *)) work-%offset%)
       u ldu
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itemp)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf ns var-4))
```

```
    (when var-5 (setf n var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldu var-10))
    (when var-13 (setf info var-13)))))
(cond
 (wantvt (setf k (f2cl-lib:int-add itgkz m))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j m) nil)
        (tagbody
          (setf (f2cl-lib:fref vt-%data% (i j) ((1 ldvt) (1 *))
                               vt-%offset%)
                  (f2cl-lib:cmplx
                   (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                  rwork-%offset%)
                   zero))
          (setf k (f2cl-lib:int-add k 1))
         label100015))
      (setf k (f2cl-lib:int-add k m))
     label100014))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "A" m (f2cl-lib:int-sub n m) czero czero
       (f2cl-lib:array-slice vt-%data% f2cl-lib:complex8
                             (1 (f2cl-lib:int-add m 1))
                             ((1 ldvt) (1 *)) vt-%offset%)
       ldvt)
    (declare (ignore var-0 var-2 var-5))
    (when var-1 (setf m var-1))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (cunmbr "P" "R" "C" ns n m a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itaup)
                             ((1 *)) work-%offset%)
       vt ldvt
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (itemp)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf ns var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf m var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldvt var-10))
```

```
                (when var-13 (setf info var-13)))))))))
        (cond
         ((= iscl 1)
          (if (> anrm bignum)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (slascl "G" 0 0 bignum anrm minmn 1 s minmn info)
                (declare (ignore var-0 var-1 var-2 var-6 var-7))
                (when var-3 (setf bignum var-3))
                (when var-4 (setf anrm var-4))
                (when var-5 (setf minmn var-5))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf info var-9))))
          (if (< anrm smlnum)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (slascl "G" 0 0 smlnum anrm minmn 1 s minmn info)
                (declare (ignore var-0 var-1 var-2 var-6 var-7))
                (when var-3 (setf smlnum var-3))
                (when var-4 (setf anrm var-4))
                (when var-5 (setf minmn var-5))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf info var-9))))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (f2cl-lib:cmplx (f2cl-lib:freal maxwrk) zero))
        (go end_label)
       end_label
        (return
         (values jobu jobvt range m n nil lda vl vu nil nil ns nil nil ldu nil
                 ldvt nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgesvdx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
```

```
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::jobu
  fortran-to-lisp::jobvt
  fortran-to-lisp::range
  fortran-to-lisp::m
  fortran-to-lisp::n nil
  fortran-to-lisp::lda
  fortran-to-lisp::vl
  fortran-to-lisp::vu nil nil
  fortran-to-lisp::ns nil nil
  fortran-to-lisp::ldu nil
  fortran-to-lisp::ldvt nil nil nil
  nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::cgelqf
  fortran-to-lisp::cgebrd
  fortran-to-lisp::cgeqrf))))
```

———-

Chapter 31

LAPACK - General Matrices, Singular Value, Complex16

31.1 zgejsv

31.1.1 Axiom unit tests

— zgejsv.input —

```
)set break resume
)sys rm -f zgejsv.output
)spool zgejsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

31.1.2 Axiom help page

— zgejsv.help —

```
========================================================================
zgejsv examples
========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgejsv (JOBA, JOBU, JOBV, JOBR, JOBT, JOBP, M, N, A, LDA,
           SVA, U, LDU, V, LDV, CWORK, LWORK, RWORK, LRWORK, IWORK, INFO)
           ZGEJSV

Function/Subroutine Documentation
   subroutine zgejsv (character*1 JOBA, character*1 JOBU, character*1 JOBV,
       character*1 JOBR, character*1 JOBT, character*1 JOBP, integer M,
       integer N, double complex, dimension( lda, * ) A, integer LDA, double
       precision, dimension( n ) SVA, double complex, dimension( ldu, * ) U,
       integer LDU, double complex, dimension( ldv, * ) V, integer LDV, double
       complex, dimension( lwork ) CWORK, integer LWORK, double precision,
       dimension( * ) RWORK, integer LRWORK, integer, dimension( * ) IWORK,
       integer INFO)
       ZGEJSV

Purpose:

 JOBA
   JOBA is CHARACTER*1
   Specifies the level of accuracy:
   = 'C': This option works well (high relative accuracy) if A = B * D,
          with well-conditioned B and arbitrary diagonal matrix D.
          The accuracy cannot be spoiled by COLUMN scaling. The
          accuracy of the computed output depends on the condition of
          B, and the procedure aims at the best theoretical accuracy.
          The relative error max_{i=1:N}|d sigma_i| / sigma_i is
          bounded by f(M,N)*epsilon* cond(B), independent of D.
          The input matrix is preprocessed with the QRF with column
          pivoting. This initial preprocessing and preconditioning by
          a rank revealing QR factorization is common for all values of
          JOBA. Additional actions are specified as follows:
   = 'E': Computation as with 'C' with an additional estimate of the
          condition number of B. It provides a realistic error bound.
   = 'F': If A = D1 * C * D2 with ill-conditioned diagonal scalings
          D1, D2, and well-conditioned matrix C, this option gives
          higher accuracy than the 'C' option. If the structure of the
          input matrix is not known, and relative accuracy is
          desirable, then this option is advisable. The input matrix A
          is preprocessed with QR factorization with FULL (row and
          column) pivoting.
   = 'G'  Computation as with 'F' with an additional estimate of the
          condition number of B, where A=D*B. If A has heavily weighted
          rows, then using this condition number gives too pessimistic
```

```
          error bound.
   = 'A': Small singular values are the noise and the matrix is treated
          as numerically rank defficient. The error in the computed
          singular values is bounded by f(m,n)*epsilon*||A||.
          The computed SVD A = U * S * V^* restores A up to
          f(m,n)*epsilon*||A||.
          This gives the procedure the licence to discard (set to zero)
          all singular values below N*epsilon*||A||.
   = 'R': Similar as in 'A'. Rank revealing property of the initial
          QR factorization is used do reveal (using triangular factor)
          a gap sigma_{r+1} < epsilon * sigma_r in which case the
          numerical RANK is declared to be r. The SVD is computed with
          absolute error bounds, but more accurately than with 'A'.

Parameters:

   JOBU
   JOBU is CHARACTER*1
   Specifies whether to compute the columns of U:
   = 'U': N columns of U are returned in the array U.
   = 'F': full set of M left sing. vectors is returned in the array U.
   = 'W': U may be used as workspace of length M*N. See the description of U.
   = 'N': U is not computed.

 JOBV
   JOBV is CHARACTER*1
   Specifies whether to compute the matrix V:
   = 'V': N columns of V are returned in the array V; Jacobi rotations
            are not explicitly accumulated.
   = 'J': N columns of V are returned in the array V, but they are
            computed as the product of Jacobi rotations. This option is
            allowed only if JOBU .NE. 'N', i.e. in computing the full SVD.
   = 'W': V may be used as workspace of length N*N. See the description of V.
   = 'N': V is not computed.

 JOBR
   JOBR is CHARACTER*1
   Specifies the RANGE for the singular values. Issues the licence to
   set to zero small positive singular values if they are outside
   specified range. If A .NE. 0 is scaled so that the largest singular
   value of c*A is around SQRT(BIG), BIG=DLAMCH('O'), then JOBR issues
   the licence to kill columns of A whose norm in c*A is less than
   SQRT(SFMIN) (for JOBR.EQ.'R'), or less than SMALL=SFMIN/EPSLN,
   where SFMIN=DLAMCH('S'), EPSLN=DLAMCH('E').
   = 'N': Do not kill small columns of c*A. This option assumes that
          BLAS and QR factorizations and triangular solvers are
          implemented to work in that range. If the condition of A
          is greater than BIG, use ZGESVJ.
   = 'R': RESTRICTED range for sigma(c*A) is [SQRT(SFMIN), SQRT(BIG)]
          (roughly, as described above). This option is recommended.
```

```
                    ===========================
  For computing the singular values in the FULL range [SFMIN,BIG]
  use ZGESVJ.

JOBT
  JOBT is CHARACTER*1
  If the matrix is square then the procedure may determine to use
  transposed A if A^* seems to be better with respect to convergence.
  If the matrix is not square, JOBT is ignored. This is subject to
  changes in the future.
  The decision is based on two values of entropy over the adjoint
  orbit of A^* * A. See the descriptions of WORK(6) and WORK(7).
  = 'T': transpose if entropy test indicates possibly faster
         convergence of Jacobi process if A^* is taken as input. If A is
         replaced with A^*, then the row pivoting is included automatically.
  = 'N': do not speculate.
  This option can be used to compute only the singular values, or the
  full SVD (U, SIGMA and V). For only one set of singular vectors
  (U or V), the caller should provide both U and V, as one of the
  matrices is used as workspace if the matrix A is transposed.
  The implementer can easily remove this constraint and make the
  code more complicated. See the descriptions of U and V.

JOBP
  JOBP is CHARACTER*1
  Issues the licence to introduce structured perturbations to drown
  denormalized numbers. This licence should be active if the
  denormals are poorly implemented, causing slow computation,
  especially in cases of fast convergence (!). For details see [1,2].
  For the sake of simplicity, this perturbations are included only
  when the full SVD or only the singular values are requested. The
  implementer/user can easily add the perturbation for the cases of
  computing one set of singular vectors.
  = 'P': introduce perturbation
  = 'N': do not perturb

M
  M is INTEGER
  The number of rows of the input matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the input matrix A. M >= N >= 0.

A
  A is DOUBLE COMPLEX array, dimension (LDA,N)
  On entry, the M-by-N matrix A.

LDA
  LDA is INTEGER
```

```
  The leading dimension of the array A.  LDA >= max(1,M).

SVA
  SVA is DOUBLE PRECISION array, dimension (N)
  On exit,
  - For WORK(1)/WORK(2) = ONE: The singular values of A. During the
    computation SVA contains Euclidean column norms of the
    iterated matrices in the array A.
  - For WORK(1) .NE. WORK(2): The singular values of A are
    (WORK(1)/WORK(2)) * SVA(1:N). This factored form is used if
    sigma_max(A) overflows or if small singular values have been
    saved from underflow by scaling the input matrix A.
  - If JOBR='R' then some of the singular values may be returned
    as exact zeros obtained by "set to zero" because they are
    below the numerical rank threshold or are denormalized numbers.

U
  U is DOUBLE COMPLEX array, dimension ( LDU, N )
  If JOBU = 'U', then U contains on exit the M-by-N matrix of
       the left singular vectors.
  If JOBU = 'F', then U contains on exit the M-by-M matrix of
       the left singular vectors, including an ONB
       of the orthogonal complement of the Range(A).
  If JOBU = 'W'  .AND. (JOBV.EQ.'V' .AND. JOBT.EQ.'T' .AND. M.EQ.N),
       then U is used as workspace if the procedure
       replaces A with A^*. In that case, [V] is computed
       in U as left singular vectors of A^* and then
       copied back to the V array. This 'W' option is just
       a reminder to the caller that in this case U is
       reserved as workspace of length N*N.
  If JOBU = 'N'  U is not referenced.

LDU
  LDU is INTEGER
  The leading dimension of the array U,  LDU >= 1.
  IF  JOBU = 'U' or 'F' or 'W',  then LDU >= M.

V
  V is DOUBLE COMPLEX array, dimension ( LDV, N )
  If JOBV = 'V', 'J' then V contains on exit the N-by-N matrix of
       the right singular vectors;
  If JOBV = 'W', AND (JOBU.EQ.'U' AND JOBT.EQ.'T' AND M.EQ.N),
       then V is used as workspace if the pprocedure
       replaces A with A^*. In that case, [U] is computed
       in V as right singular vectors of A^* and then
       copied back to the U array. This 'W' option is just
       a reminder to the caller that in this case V is
       reserved as workspace of length N*N.
  If JOBV = 'N'  V is not referenced.
```

```
LDV
  LDV is INTEGER
  The leading dimension of the array V,  LDV >= 1.
  If JOBV = 'V' or 'J' or 'W', then LDV >= N.

CWORK
  CWORK (workspace)
  CWORK is DOUBLE COMPLEX array, dimension at least LWORK.

LWORK
  LWORK is INTEGER
  Length of CWORK to confirm proper allocation of workspace.
  LWORK depends on the job:

  1. If only SIGMA is needed ( JOBU.EQ.'N', JOBV.EQ.'N' ) and
   1.1 .. no scaled condition estimate required (JOBE.EQ.'N'):
          LWORK >= 2*N+1. This is the minimal requirement.
      ->> For optimal performance (blocked code) the optimal value
          is LWORK >= N + (N+1)*NB. Here NB is the optimal
          block size for ZGEQP3 and ZGEQRF.
          In general, optimal LWORK is computed as
          LWORK >= max(N+LWORK(ZGEQP3),N+LWORK(ZGEQRF)).
   1.2. .. an estimate of the scaled condition number of A is
          required (JOBA='E', or 'G'). In this case, LWORK the minimal
          requirement is LWORK >= N*N + 3*N.
      ->> For optimal performance (blocked code) the optimal value
          is LWORK >= max(N+(N+1)*NB, N*N+3*N).
          In general, the optimal length LWORK is computed as
          LWORK >= max(N+LWORK(ZGEQP3),N+LWORK(ZGEQRF),
                   N+N*N+LWORK(CPOCON)).

  2. If SIGMA and the right singular vectors are needed (JOBV.EQ.'V'),
        (JOBU.EQ.'N')
    -> the minimal requirement is LWORK >= 3*N.
    -> For optimal performance, LWORK >= max(N+(N+1)*NB, 3*N,2*N+N*NB),
       where NB is the optimal block size for ZGEQP3, ZGEQRF, ZGELQ,
       CUNMLQ. In general, the optimal length LWORK is computed as
       LWORK >= max(N+LWORK(ZGEQP3), N+LWORK(CPOCON), N+LWORK(ZGESVJ),
                N+LWORK(ZGELQF), 2*N+LWORK(ZGEQRF), N+LWORK(CUNMLQ)).

  3. If SIGMA and the left singular vectors are needed
   -> the minimal requirement is LWORK >= 3*N.
   -> For optimal performance:
      if JOBU.EQ.'U' :: LWORK >= max(3*N, N+(N+1)*NB, 2*N+N*NB),
      where NB is the optimal block size for ZGEQP3, ZGEQRF, CUNMQR.
      In general, the optimal length LWORK is computed as
      LWORK >= max(N+LWORK(ZGEQP3),N+LWORK(CPOCON),
               2*N+LWORK(ZGEQRF), N+LWORK(CUNMQR)).

  4. If the full SVD is needed: (JOBU.EQ.'U' or JOBU.EQ.'F') and
```

```
   4.1. if JOBV.EQ.'V' the minimal requirement is LWORK >= 5*N+2*N*N.
   4.2. if JOBV.EQ.'J' the minimal requirement is LWORK >= 4*N+N*N.
  In both cases, the allocated CWORK can accomodate blocked runs
  of ZGEQP3, ZGEQRF, ZGELQF, SUNMQR, CUNMLQ.

RWORK
  RWORK is DOUBLE PRECISION array, dimension at least LRWORK.
  On exit,
  RWORK(1) = Determines the scaling factor SCALE = RWORK(2) / RWORK(1)
             such that SCALE*SVA(1:N) are the computed singular values
             of A. (See the description of SVA().)
  RWORK(2) = See the description of RWORK(1).
  RWORK(3) = SCONDA is an estimate for the condition number of
             column equilibrated A. (If JOBA .EQ. 'E' or 'G')
             SCONDA is an estimate of SQRT(||(R^* * R)^(-1)||_1).
             It is computed using SPOCON. It holds
             N^(-1/4) * SCONDA <= ||R^(-1)||_2 <= N^(1/4) * SCONDA
             where R is the triangular factor from the QRF of A.
             However, if R is truncated and the numerical rank is
             determined to be strictly smaller than N, SCONDA is
             returned as -1, thus indicating that the smallest
             singular values might be lost.

  If full SVD is needed, the following two condition numbers are
  useful for the analysis of the algorithm. They are provied for
  a developer/implementer who is familiar with the details of
  the method.

  RWORK(4) = an estimate of the scaled condition number of the
             triangular factor in the first QR factorization.
  RWORK(5) = an estimate of the scaled condition number of the
             triangular factor in the second QR factorization.

  The following two parameters are computed if JOBT .EQ. 'T'.
  They are provided for a developer/implementer who is familiar
  with the details of the method.
  RWORK(6) = the entropy of A^* * A :: this is the Shannon entropy
             of diag(A^* * A) / Trace(A^* * A) taken as point in the
             probability simplex.
  RWORK(7) = the entropy of A * A^*. (See the description of RWORK(6).)

LRWORK
  LRWORK is INTEGER
  Length of RWORK to confirm proper allocation of workspace.
  LRWORK depends on the job:

  1. If only singular values are requested i.e. if
     LSAME(JOBU,'N') .AND. LSAME(JOBV,'N')
     then:
   1.1. If LSAME(JOBT,'T') .OR. LSAME(JOBA,'F') .OR. LSAME(JOBA,'G'),
```

```
        then LRWORK = max( 7, N + 2 * M ).
   1.2. Otherwise, LRWORK  = max( 7, 2 * N ).
  2. If singular values with the right singular vectors are requested
     i.e. if
      (LSAME(JOBV,'V').OR.LSAME(JOBV,'J')) .AND.
       .NOT.(LSAME(JOBU,'U').OR.LSAME(JOBU,'F'))
     then:
   2.1. If LSAME(JOBT,'T') .OR. LSAME(JOBA,'F') .OR. LSAME(JOBA,'G'),
        then LRWORK = max( 7, N + 2 * M ).
   2.2. Otherwise, LRWORK  = max( 7, 2 * N ).
  3. If singular values with the left singular vectors are requested, i.e. if
      (LSAME(JOBU,'U').OR.LSAME(JOBU,'F')) .AND.
      .NOT.(LSAME(JOBV,'V').OR.LSAME(JOBV,'J'))
     then:
   3.1. If LSAME(JOBT,'T') .OR. LSAME(JOBA,'F') .OR. LSAME(JOBA,'G'),
        then LRWORK = max( 7, N + 2 * M ).
   3.2. Otherwise, LRWORK  = max( 7, 2 * N ).
  4. If singular values with both the left and the right singular vectors
     are requested, i.e. if
      (LSAME(JOBU,'U').OR.LSAME(JOBU,'F')) .AND.
      (LSAME(JOBV,'V').OR.LSAME(JOBV,'J'))
     then:
   4.1. If LSAME(JOBT,'T') .OR. LSAME(JOBA,'F') .OR. LSAME(JOBA,'G'),
        then LRWORK = max( 7, N + 2 * M ).
   4.2. Otherwise, LRWORK  = max( 7, 2 * N ).

IWORK
  IWORK is INTEGER array, of dimension:
  If LSAME(JOBA,'F') .OR. LSAME(JOBA,'G'), then
  the dimension of IWORK is max( 3, 2 * N + M ).
  Otherwise, the dimension of IWORK is
  -> max( 3, 2*N ) for full SVD
  -> max( 3, N ) for singular values only or singular
     values with one set of singular vectors (left or right)
  On exit,
  IWORK(1) = the numerical rank determined after the initial
             QR factorization with pivoting. See the descriptions
             of JOBA and JOBR.
  IWORK(2) = the number of the computed nonzero singular values
  IWORK(3) = if nonzero, a warning message:
             If IWORK(3).EQ.1 then some of the column norms of A
             were denormalized floats. The requested high accuracy
             is not warranted by the data.

INFO
  INFO is INTEGER
  < 0  : if INFO = -i, then the i-th argument had an illegal value.
  = 0 :  successfull exit;
  > 0 :  ZGEJSV  did not converge in the maximal allowed number
         of sweeps. The computed values may be inaccurate.
```

```
Further Details:
   ZGEJSV implements a preconditioned Jacobi SVD algorithm. It uses ZGEQP3,
   ZGEQRF, and ZGELQF as preprocessors and preconditioners. Optionally, an
   additional row pivoting can be used as a preprocessor, which in some
   cases results in much higher accuracy. An example is matrix A with the
   structure A = D1 * C * D2, where D1, D2 are arbitrarily ill-conditioned
   diagonal matrices and C is well-conditioned matrix. In that case, complete
   pivoting in the first QR factorizations provides accuracy dependent on the
   condition number of C, and independent of D1, D2. Such higher accuracy is
   not completely understood theoretically, but it works well in practice.
   Further, if A can be written as A = B*D, with well-conditioned B and some
   diagonal D, then the high accuracy is guaranteed, both theoretically and
   in software, independent of D. For more details see [1], [2].
   The computational range for the singular values can be the full range
   ( UNDERFLOW,OVERFLOW ), provided that the machine arithmetic and the BLAS
   & LAPACK routines called by ZGEJSV are implemented to work in that range.
   If that is not the case, then the restriction for safe computation with
   the singular values in the range of normalized IEEE numbers is that the
   spectral condition number kappa(A)=sigma_max(A)/sigma_min(A) does not
   overflow. This code (ZGEJSV) is best used in this restricted range,
   meaning that singular values of magnitude below ||A||_2 / DLAMCH('O') are
   returned as zeros. See JOBR for details on this.
   Further, this implementation is somewhat slower than the one described
   in [1,2] due to replacement of some non-LAPACK components, and because
   the choice of some tuning parameters in the iterative part (ZGESVJ) is
   left to the implementer on a particular machine.
   The rank revealing QR factorization (in this code: ZGEQP3) should be
   implemented as in [3]. We have a new version of ZGEQP3 under development
   that is more robust than the current one in LAPACK, with a cleaner cut in
   rank defficient cases. It will be available in the SIGMA library [4].
   If M is much larger than N, it is obvious that the inital QRF with
   column pivoting can be preprocessed by the QRF without pivoting. That
   well known trick is not used in ZGEJSV because in some cases heavy row
   weighting can be treated with complete pivoting. The overhead in cases
   M much larger than N is then only due to pivoting, but the benefits in
   terms of accuracy have prevailed. The implementer/user can incorporate
   this extra QRF step easily. The implementer can also improve data movement
   (matrix transpose, matrix copy, matrix transposed copy) - this
   implementation of ZGEJSV uses only the simplest, naive data movement.
```

———-

31.1.3 fortran code

— zgejsv.f —

```
*  =====================================================================
      SUBROUTINE ZGEJSV( JOBA, JOBU, JOBV, JOBR, JOBT, JOBP,
     $                   M, N, A, LDA, SVA, U, LDU, V, LDV,
     $                   CWORK, LWORK, RWORK, LRWORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      IMPLICIT    NONE
      INTEGER     INFO, LDA, LDU, LDV, LWORK, LRWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE COMPLEX   A( LDA, * ), U( LDU, * ), V( LDV, * ),
     $                 CWORK( LWORK )
      DOUBLE PRECISION SVA( N ), RWORK( * )
      INTEGER          IWORK( * )
      CHARACTER*1      JOBA, JOBP, JOBR, JOBT, JOBU, JOBV
*     ..
*
*  ==========================================================================
*
*     .. Local Parameters ..
      DOUBLE PRECISION ZERO,         ONE
      PARAMETER ( ZERO = 0.0D0, ONE = 1.0D0 )
      DOUBLE COMPLEX            CZERO,       CONE
      PARAMETER  ( CZERO = ( 0.0D0, 0.0D0 ), CONE = ( 1.0D0, 0.0D0 ) )
*     ..
*     .. Local Scalars ..
      DOUBLE COMPLEX   CTEMP
      DOUBLE PRECISION AAPP,    AAQQ,   AATMAX, AATMIN, BIG,    BIG1,
     $                 COND_OK, CONDR1, CONDR2, ENTRA,  ENTRAT, EPSLN,
     $                 MAXPRJ,  SCALEM, SCONDA, SFMIN,  SMALL,  TEMP1,
     $                 USCAL1,  USCAL2, XSC
      INTEGER IERR,   N1,     NR,     NUMRANK,        p, q,   WARNING
      LOGICAL ALMORT, DEFR,   ERREST, GOSCAL, JRACC,  KILL,   LSVEC,
     $        L2ABER, L2KILL, L2PERT, L2RANK, L2TRAN,
     $        NOSCAL, ROWPIV, RSVEC,  TRANSP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC ABS,  DCMPLX, DCONJG, DLOG, DMAX1, DMIN1, DFLOAT,
     $          MAX0, MIN0, NINT,  DSQRT
*     ..
*     .. External Functions ..
      DOUBLE PRECISION      DLAMCH, DZNRM2
      INTEGER   IDAMAX
      LOGICAL   LSAME
      EXTERNAL  IDAMAX, LSAME, DLAMCH, DZNRM2
```

```
*     ..
*     .. External Subroutines ..
      EXTERNAL  ZCOPY,  ZGELQF, ZGEQP3, ZGEQRF, ZLACPY, ZLASCL,
     $          ZLASET, ZLASSQ, ZLASWP, ZUNGQR, ZUNMLQ,
     $          ZUNMQR, ZPOCON, DSCAL, ZDSCAL, ZSWAP,  ZTRSM,  XERBLA
*
      EXTERNAL  ZGESVJ
*     ..
*
*     Test the input arguments
*

      LSVEC  = LSAME( JOBU, 'U' ) .OR. LSAME( JOBU, 'F' )
      JRACC  = LSAME( JOBV, 'J' )
      RSVEC  = LSAME( JOBV, 'V' ) .OR. JRACC
      ROWPIV = LSAME( JOBA, 'F' ) .OR. LSAME( JOBA, 'G' )
      L2RANK = LSAME( JOBA, 'R' )
      L2ABER = LSAME( JOBA, 'A' )
      ERREST = LSAME( JOBA, 'E' ) .OR. LSAME( JOBA, 'G' )
      L2TRAN = LSAME( JOBT, 'T' )
      L2KILL = LSAME( JOBR, 'R' )
      DEFR   = LSAME( JOBR, 'N' )
      L2PERT = LSAME( JOBP, 'P' )
*
      IF ( .NOT.(ROWPIV .OR. L2RANK .OR. L2ABER .OR.
     $     ERREST .OR. LSAME( JOBA, 'C' ) )) THEN
         INFO = - 1
      ELSE IF ( .NOT.( LSVEC  .OR. LSAME( JOBU, 'N' ) .OR.
     $                             LSAME( JOBU, 'W' )) ) THEN
         INFO = - 2
      ELSE IF ( .NOT.( RSVEC .OR. LSAME( JOBV, 'N' ) .OR.
     $   LSAME( JOBV, 'W' )) .OR. ( JRACC .AND. (.NOT.LSVEC) ) ) THEN
         INFO = - 3
      ELSE IF ( .NOT. ( L2KILL .OR. DEFR ) )    THEN
         INFO = - 4
      ELSE IF ( .NOT. ( L2TRAN .OR. LSAME( JOBT, 'N' ) ) ) THEN
         INFO = - 5
      ELSE IF ( .NOT. ( L2PERT .OR. LSAME( JOBP, 'N' ) ) ) THEN
         INFO = - 6
      ELSE IF ( M .LT. 0 ) THEN
         INFO = - 7
      ELSE IF ( ( N .LT. 0 ) .OR. ( N .GT. M ) ) THEN
         INFO = - 8
      ELSE IF ( LDA .LT. M ) THEN
         INFO = - 10
      ELSE IF ( LSVEC .AND. ( LDU .LT. M ) ) THEN
         INFO = - 13
      ELSE IF ( RSVEC .AND. ( LDV .LT. N ) ) THEN
         INFO = - 15
      ELSE IF ( (.NOT.(LSVEC .OR. RSVEC .OR. ERREST).AND.
```

```
     $                           (LWORK .LT. 2*N+1)) .OR.
     $ (.NOT.(LSVEC .OR. RSVEC) .AND. ERREST .AND.
     $                         (LWORK .LT. N*N+3*N)) .OR.
     $ (LSVEC .AND. (.NOT.RSVEC) .AND. (LWORK .LT. 3*N))
     $ .OR.
     $ (RSVEC .AND. (.NOT.LSVEC) .AND. (LWORK .LT. 3*N))
     $ .OR.
     $ (LSVEC .AND. RSVEC .AND. (.NOT.JRACC) .AND.
     $                          (LWORK.LT.5*N+2*N*N))
     $ .OR. (LSVEC .AND. RSVEC .AND. JRACC .AND.
     $                          LWORK.LT.4*N+N*N))
     $   THEN
         INFO = - 17
      ELSE IF ( LRWORK.LT. MAX0(N+2*M,7)) THEN
         INFO = -19
      ELSE
*        #:)
         INFO = 0
      END IF
*
      IF ( INFO .NE. 0 ) THEN
*       #:(
         CALL XERBLA( 'ZGEJSV', - INFO )
         RETURN
      END IF
*
*     Quick return for void matrix (Y3K safe)
* #:)
      IF ( ( M .EQ. 0 ) .OR. ( N .EQ. 0 ) ) RETURN
*
*     Determine whether the matrix U should be M x N or M x M
*
      IF ( LSVEC ) THEN
         N1 = N
         IF ( LSAME( JOBU, 'F' ) ) N1 = M
      END IF
*
*     Set numerical parameters
*
*!    NOTE: Make sure DLAMCH() does not fail on the target architecture.
*
      EPSLN = DLAMCH('Epsilon')
      SFMIN = DLAMCH('SafeMinimum')
      SMALL = SFMIN / EPSLN
      BIG   = DLAMCH('O')
*     BIG   = ONE / SFMIN
*
*     Initialize SVA(1:N) = diag( ||A e_i||_2 )_1^N
*
*(!)  If necessary, scale SVA() to protect the largest norm from
```

```
*     overflow. It is possible that this scaling pushes the smallest
*     column norm left from the underflow threshold (extreme case).
*
      SCALEM  = ONE / DSQRT(DFLOAT(M)*DFLOAT(N))
      NOSCAL  = .TRUE.
      GOSCAL  = .TRUE.
      DO 1874 p = 1, N
         AAPP = ZERO
         AAQQ = ONE
         CALL ZLASSQ( M, A(1,p), 1, AAPP, AAQQ )
         IF ( AAPP .GT. BIG ) THEN
            INFO = - 9
            CALL XERBLA( 'ZGEJSV', -INFO )
            RETURN
         END IF
         AAQQ = DSQRT(AAQQ)
         IF ( ( AAPP .LT. (BIG / AAQQ) ) .AND. NOSCAL  ) THEN
            SVA(p)  = AAPP * AAQQ
         ELSE
            NOSCAL  = .FALSE.
            SVA(p)  = AAPP * ( AAQQ * SCALEM )
            IF ( GOSCAL ) THEN
               GOSCAL = .FALSE.
               CALL DSCAL( p-1, SCALEM, SVA, 1 )
            END IF
         END IF
 1874 CONTINUE
*
      IF ( NOSCAL ) SCALEM = ONE
*
      AAPP = ZERO
      AAQQ = BIG
      DO 4781 p = 1, N
         AAPP = DMAX1( AAPP, SVA(p) )
         IF ( SVA(p) .NE. ZERO ) AAQQ = DMIN1( AAQQ, SVA(p) )
 4781 CONTINUE
*
*     Quick return for zero M x N matrix
* #:)
      IF ( AAPP .EQ. ZERO ) THEN
         IF ( LSVEC ) CALL ZLASET( 'G', M, N1, CZERO, CONE, U, LDU )
         IF ( RSVEC ) CALL ZLASET( 'G', N, N,  CZERO, CONE, V, LDV )
         RWORK(1) = ONE
         RWORK(2) = ONE
         IF ( ERREST ) RWORK(3) = ONE
         IF ( LSVEC .AND. RSVEC ) THEN
            RWORK(4) = ONE
            RWORK(5) = ONE
         END IF
         IF ( L2TRAN ) THEN
```

```
            RWORK(6) = ZERO
            RWORK(7) = ZERO
         END IF
         IWORK(1) = 0
         IWORK(2) = 0
         IWORK(3) = 0
         RETURN
      END IF
*
*     Issue warning if denormalized column norms detected. Override the
*     high relative accuracy request. Issue licence to kill columns
*     (set them to zero) whose norm is less than sigma_max / BIG (roughly).
* #:(
      WARNING = 0
      IF ( AAQQ .LE. SFMIN ) THEN
         L2RANK = .TRUE.
         L2KILL = .TRUE.
         WARNING = 1
      END IF
*
*     Quick return for one-column matrix
* #:)
      IF ( N .EQ. 1 ) THEN
*
         IF ( LSVEC ) THEN
            CALL ZLASCL( 'G',0,0,SVA(1),SCALEM, M,1,A(1,1),LDA,IERR )
            CALL ZLACPY( 'A', M, 1, A, LDA, U, LDU )
*           computing all M left singular vectors of the M x 1 matrix
            IF ( N1 .NE. N  ) THEN
              CALL ZGEQRF( M, N, U,LDU, CWORK, CWORK(N+1),LWORK-N,IERR )
              CALL ZUNGQR( M,N1,1, U,LDU,CWORK,CWORK(N+1),LWORK-N,IERR )
              CALL ZCOPY( M, A(1,1), 1, U(1,1), 1 )
            END IF
         END IF
         IF ( RSVEC ) THEN
             V(1,1) = CONE
         END IF
         IF ( SVA(1) .LT. (BIG*SCALEM) ) THEN
            SVA(1)  = SVA(1) / SCALEM
            SCALEM  = ONE
         END IF
         RWORK(1) = ONE / SCALEM
         RWORK(2) = ONE
         IF ( SVA(1) .NE. ZERO ) THEN
            IWORK(1) = 1
            IF ( ( SVA(1) / SCALEM) .GE. SFMIN ) THEN
               IWORK(2) = 1
            ELSE
               IWORK(2) = 0
            END IF
```

```
         ELSE
            IWORK(1) = 0
            IWORK(2) = 0
         END IF
         IWORK(3) = 0
         IF ( ERREST ) RWORK(3) = ONE
         IF ( LSVEC .AND. RSVEC ) THEN
            RWORK(4) = ONE
            RWORK(5) = ONE
         END IF
         IF ( L2TRAN ) THEN
            RWORK(6) = ZERO
            RWORK(7) = ZERO
         END IF
         RETURN
*
      END IF
*
      TRANSP = .FALSE.
      L2TRAN = L2TRAN .AND. ( M .EQ. N )
*
      AATMAX = -ONE
      AATMIN =  BIG
      IF ( ROWPIV .OR. L2TRAN ) THEN
*
*     Compute the row norms, needed to determine row pivoting sequence
*     (in the case of heavily row weighted A, row pivoting is strongly
*     advised) and to collect information needed to compare the
*     structures of A * A^* and A^* * A (in the case L2TRAN.EQ..TRUE.).
*
         IF ( L2TRAN ) THEN
            DO 1950 p = 1, M
               XSC   = ZERO
               TEMP1 = ONE
               CALL ZLASSQ( N, A(p,1), LDA, XSC, TEMP1 )
*              ZLASSQ gets both the ell_2 and the ell_infinity norm
*              in one pass through the vector
               RWORK(M+N+p)  = XSC * SCALEM
               RWORK(N+p)    = XSC * (SCALEM*DSQRT(TEMP1))
               AATMAX = DMAX1( AATMAX, RWORK(N+p) )
               IF (RWORK(N+p) .NE. ZERO)
     $            AATMIN = DMIN1(AATMIN,RWORK(N+p))
 1950       CONTINUE
         ELSE
            DO 1904 p = 1, M
               RWORK(M+N+p) = SCALEM*ABS( A(p,IDAMAX(N,A(p,1),LDA)) )
               AATMAX = DMAX1( AATMAX, RWORK(M+N+p) )
               AATMIN = DMIN1( AATMIN, RWORK(M+N+p) )
 1904       CONTINUE
         END IF
```

```
*
      END IF
*
*     For square matrix A try to determine whether A^*  would be  better
*     input for the preconditioned Jacobi SVD, with faster convergence.
*     The decision is based on an O(N) function of the vector of column
*     and row norms of A, based on the Shannon entropy. This should give
*     the right choice in most cases when the difference actually matters.
*     It may fail and pick the slower converging side.
*
      ENTRA  = ZERO
      ENTRAT = ZERO
      IF ( L2TRAN ) THEN
*
         XSC   = ZERO
         TEMP1 = ONE
         CALL ZLASSQ( N, SVA, 1, XSC, TEMP1 )
         TEMP1 = ONE / TEMP1
*
         ENTRA = ZERO
         DO 1113 p = 1, N
            BIG1  = ( ( SVA(p) / XSC )**2 ) * TEMP1
            IF ( BIG1 .NE. ZERO ) ENTRA = ENTRA + BIG1 * DLOG(BIG1)
 1113    CONTINUE
         ENTRA = - ENTRA / DLOG(DFLOAT(N))
*
*        Now, SVA().^2/Trace(A^* * A) is a point in the probability simplex.
*        It is derived from the diagonal of  A^* * A.  Do the same with the
*        diagonal of A * A^*, compute the entropy of the corresponding
*        probability distribution. Note that A * A^* and A^* * A have the
*        same trace.
*
         ENTRAT = ZERO
         DO 1114 p = N+1, N+M
            BIG1 = ( ( RWORK(p) / XSC )**2 ) * TEMP1
            IF ( BIG1 .NE. ZERO ) ENTRAT = ENTRAT + BIG1 * DLOG(BIG1)
 1114    CONTINUE
         ENTRAT = - ENTRAT / DLOG(DFLOAT(M))
*
*        Analyze the entropies and decide A or A^*. Smaller entropy
*        usually means better input for the algorithm.
*
         TRANSP = ( ENTRAT .LT. ENTRA )
         TRANSP = .TRUE.
*
*        If A^* is better than A, take the adjoint of A.
*
         IF ( TRANSP ) THEN
*           In an optimal implementation, this trivial transpose
*           should be replaced with faster transpose.
```

```
            DO 1115 p = 1, N - 1
               A(p,p) = DCONJG(A(p,p))
               DO 1116 q = p + 1, N
                   CTEMP = DCONJG(A(q,p))
                  A(q,p) = DCONJG(A(p,q))
                  A(p,q) = CTEMP
 1116          CONTINUE
 1115       CONTINUE
            A(N,N) = DCONJG(A(N,N))
            DO 1117 p = 1, N
               RWORK(M+N+p) = SVA(p)
               SVA(p)      = RWORK(N+p)
*              previously computed row 2-norms are now column 2-norms
*              of the transposed matrix
 1117       CONTINUE
            TEMP1  = AAPP
            AAPP   = AATMAX
            AATMAX = TEMP1
            TEMP1  = AAQQ
            AAQQ   = AATMIN
            AATMIN = TEMP1
            KILL   = LSVEC
            LSVEC  = RSVEC
            RSVEC  = KILL
            IF ( LSVEC ) N1 = N
*
            ROWPIV = .TRUE.
         END IF
*
      END IF
*     END IF L2TRAN
*
*     Scale the matrix so that its maximal singular value remains less
*     than SQRT(BIG) -- the matrix is scaled so that its maximal column
*     has Euclidean norm equal to SQRT(BIG/N). The only reason to keep
*     SQRT(BIG) instead of BIG is the fact that ZGEJSV uses LAPACK and
*     BLAS routines that, in some implementations, are not capable of
*     working in the full interval [SFMIN,BIG] and that they may provoke
*     overflows in the intermediate results. If the singular values spread
*     from SFMIN to BIG, then ZGESVJ will compute them. So, in that case,
*     one should use ZGESVJ instead of ZGEJSV.
*
      BIG1   = DSQRT( BIG )
      TEMP1  = DSQRT( BIG / DFLOAT(N) )
*
      CALL ZLASCL( 'G', 0, 0, AAPP, TEMP1, N, 1, SVA, N, IERR )
      IF ( AAQQ .GT. (AAPP * SFMIN) ) THEN
          AAQQ = ( AAQQ / AAPP ) * TEMP1
      ELSE
          AAQQ = ( AAQQ * TEMP1 ) / AAPP
```

```
      END IF
      TEMP1 = TEMP1 * SCALEM
      CALL ZLASCL( 'G', 0, 0, AAPP, TEMP1, M, N, A, LDA, IERR )
*
*     To undo scaling at the end of this procedure, multiply the
*     computed singular values with USCAL2 / USCAL1.
*
      USCAL1 = TEMP1
      USCAL2 = AAPP
*
      IF ( L2KILL ) THEN
*        L2KILL enforces computation of nonzero singular values in
*        the restricted range of condition number of the initial A,
*        sigma_max(A) / sigma_min(A) approx. SQRT(BIG)/SQRT(SFMIN).
         XSC = DSQRT( SFMIN )
      ELSE
         XSC = SMALL
*
*        Now, if the condition number of A is too big,
*        sigma_max(A) / sigma_min(A) .GT. SQRT(BIG/N) * EPSLN / SFMIN,
*        as a precaution measure, the full SVD is computed using ZGESVJ
*        with accumulated Jacobi rotations. This provides numerically
*        more robust computation, at the cost of slightly increased run
*        time. Depending on the concrete implementation of BLAS and LAPACK
*        (i.e. how they behave in presence of extreme ill-conditioning) the
*        implementor may decide to remove this switch.
         IF ( ( AAQQ.LT.DSQRT(SFMIN) ) .AND. LSVEC .AND. RSVEC ) THEN
            JRACC = .TRUE.
         END IF
*
      END IF
      IF ( AAQQ .LT. XSC ) THEN
         DO 700 p = 1, N
            IF ( SVA(p) .LT. XSC ) THEN
               CALL ZLASET( 'A', M, 1, CZERO, CZERO, A(1,p), LDA )
               SVA(p) = ZERO
            END IF
 700     CONTINUE
      END IF
*
*     Preconditioning using QR factorization with pivoting
*
      IF ( ROWPIV ) THEN
*        Optional row permutation (Bjoerck row pivoting):
*        A result by Cox and Higham shows that the Bjoerck's
*        row pivoting combined with standard column pivoting
*        has similar effect as Powell-Reid complete pivoting.
*        The ell-infinity norms of A are made nonincreasing.
         DO 1952 p = 1, M - 1
            q = IDAMAX( M-p+1, RWORK(M+N+p), 1 ) + p - 1
```

```
            IWORK(2*N+p) = q
            IF ( p .NE. q ) THEN
               TEMP1        = RWORK(M+N+p)
               RWORK(M+N+p) = RWORK(M+N+q)
               RWORK(M+N+q) = TEMP1
            END IF
 1952    CONTINUE
         CALL ZLASWP( N, A, LDA, 1, M-1, IWORK(2*N+1), 1 )
      END IF

*
*     End of the preparation phase (scaling, optional sorting and
*     transposing, optional flushing of small columns).
*
*     Preconditioning
*
*     If the full SVD is needed, the right singular vectors are computed
*     from a matrix equation, and for that we need theoretical analysis
*     of the Businger-Golub pivoting. So we use ZGEQP3 as the first RR QRF.
*     In all other cases the first RR QRF can be chosen by other criteria
*     (eg speed by replacing global with restricted window pivoting, such
*     as in xGEQPX from TOMS # 782). Good results will be obtained using
*     xGEQPX with properly (!) chosen numerical parameters.
*     Any improvement of ZGEQP3 improves overal performance of ZGEJSV.
*
*     A * P1 = Q1 * [ R1^* 0]^*:
      DO 1963 p = 1, N
*        .. all columns are free columns
         IWORK(p) = 0
 1963 CONTINUE
      CALL ZGEQP3( M, N, A, LDA, IWORK, CWORK, CWORK(N+1), LWORK-N,
     $             RWORK, IERR )
*
*     The upper triangular matrix R1 from the first QRF is inspected for
*     rank deficiency and possibilities for deflation, or possible
*     ill-conditioning. Depending on the user specified flag L2RANK,
*     the procedure explores possibilities to reduce the numerical
*     rank by inspecting the computed upper triangular factor. If
*     L2RANK or L2ABER are up, then ZGEJSV will compute the SVD of
*     A + dA, where ||dA|| <= f(M,N)*EPSLN.
*
      NR = 1
      IF ( L2ABER ) THEN
*        Standard absolute error bound suffices. All sigma_i with
*        sigma_i < N*EPSLN*||A|| are flushed to zero. This is an
*        agressive enforcement of lower numerical rank by introducing a
*        backward error of the order of N*EPSLN*||A||.
         TEMP1 = DSQRT(DFLOAT(N))*EPSLN
         DO 3001 p = 2, N
            IF ( ABS(A(p,p)) .GE. (TEMP1*ABS(A(1,1))) ) THEN
```

```
               NR = NR + 1
            ELSE
               GO TO 3002
            END IF
 3001    CONTINUE
 3002    CONTINUE
      ELSE IF ( L2RANK ) THEN
*        .. similarly as above, only slightly more gentle (less agressive).
*        Sudden drop on the diagonal of R1 is used as the criterion for
*        close-to-rank-defficient.
         TEMP1 = DSQRT(SFMIN)
         DO 3401 p = 2, N
            IF ( ( ABS(A(p,p)) .LT. (EPSLN*ABS(A(p-1,p-1))) ) .OR.
     $           ( ABS(A(p,p)) .LT. SMALL ) .OR.
     $           ( L2KILL .AND. (ABS(A(p,p)) .LT. TEMP1) ) ) GO TO 3402
            NR = NR + 1
 3401    CONTINUE
 3402    CONTINUE
*
      ELSE
*        The goal is high relative accuracy. However, if the matrix
*        has high scaled condition number the relative accuracy is in
*        general not feasible. Later on, a condition number estimator
*        will be deployed to estimate the scaled condition number.
*        Here we just remove the underflowed part of the triangular
*        factor. This prevents the situation in which the code is
*        working hard to get the accuracy not warranted by the data.
         TEMP1  = DSQRT(SFMIN)
         DO 3301 p = 2, N
            IF ( ( ABS(A(p,p)) .LT. SMALL ) .OR.
     $           ( L2KILL .AND. (ABS(A(p,p)) .LT. TEMP1) ) ) GO TO 3302
            NR = NR + 1
 3301    CONTINUE
 3302    CONTINUE
*
      END IF
*
      ALMORT = .FALSE.
      IF ( NR .EQ. N ) THEN
         MAXPRJ = ONE
         DO 3051 p = 2, N
            TEMP1  = ABS(A(p,p)) / SVA(IWORK(p))
            MAXPRJ = DMIN1( MAXPRJ, TEMP1 )
 3051    CONTINUE
         IF ( MAXPRJ**2 .GE. ONE - DFLOAT(N)*EPSLN ) ALMORT = .TRUE.
      END IF
*
*
      SCONDA = - ONE
      CONDR1 = - ONE
```

```
      CONDR2 = - ONE
*
      IF ( ERREST ) THEN
         IF ( N .EQ. NR ) THEN
            IF ( RSVEC ) THEN
*              .. V is available as workspace
               CALL ZLACPY( 'U', N, N, A, LDA, V, LDV )
               DO 3053 p = 1, N
                  TEMP1 = SVA(IWORK(p))
                  CALL ZDSCAL( p, ONE/TEMP1, V(1,p), 1 )
 3053          CONTINUE
               CALL ZPOCON( 'U', N, V, LDV, ONE, TEMP1,
     $              CWORK(N+1), RWORK, IERR )
*
            ELSE IF ( LSVEC ) THEN
*              .. U is available as workspace
               CALL ZLACPY( 'U', N, N, A, LDA, U, LDU )
               DO 3054 p = 1, N
                  TEMP1 = SVA(IWORK(p))
                  CALL ZDSCAL( p, ONE/TEMP1, U(1,p), 1 )
 3054          CONTINUE
               CALL ZPOCON( 'U', N, U, LDU, ONE, TEMP1,
     $              CWORK(N+1), RWORK, IERR )
            ELSE
               CALL ZLACPY( 'U', N, N, A, LDA, CWORK(N+1), N )
               DO 3052 p = 1, N
                  TEMP1 = SVA(IWORK(p))
                  CALL ZDSCAL( p, ONE/TEMP1, CWORK(N+(p-1)*N+1), 1 )
 3052          CONTINUE
*           .. the columns of R are scaled to have unit Euclidean lengths.
               CALL ZPOCON( 'U', N, CWORK(N+1), N, ONE, TEMP1,
     $              CWORK(N+N*N+1), RWORK, IERR )
*
            END IF
            SCONDA = ONE / DSQRT(TEMP1)
*           SCONDA is an estimate of SQRT(||(R^* * R)^(-1)||_1).
*           N^(-1/4) * SCONDA <= ||R^(-1)||_2 <= N^(1/4) * SCONDA
         ELSE
            SCONDA = - ONE
         END IF
      END IF
*
      L2PERT = L2PERT .AND. ( ABS( A(1,1)/A(NR,NR) ) .GT. DSQRT(BIG1) )
*     If there is no violent scaling, artificial perturbation is not needed.
*
*     Phase 3:
*
      IF ( .NOT. ( RSVEC .OR. LSVEC ) ) THEN
*
*         Singular Values only
```

```
*
*         .. transpose A(1:NR,1:N)
         DO 1946 p = 1, MIN0( N-1, NR )
            CALL ZCOPY( N-p, A(p,p+1), LDA, A(p+1,p), 1 )
            CALL ZLACGV( N-p+1, A(p,p), 1 )
 1946    CONTINUE
         IF ( NR .EQ. N ) A(N,N) = DCONJG(A(N,N))
*
*        The following two DO-loops introduce small relative perturbation
*        into the strict upper triangle of the lower triangular matrix.
*        Small entries below the main diagonal are also changed.
*        This modification is useful if the computing environment does not
*        provide/allow FLUSH TO ZERO underflow, for it prevents many
*        annoying denormalized numbers in case of strongly scaled matrices.
*        The perturbation is structured so that it does not introduce any
*        new perturbation of the singular values, and it does not destroy
*        the job done by the preconditioner.
*        The licence for this perturbation is in the variable L2PERT, which
*        should be .FALSE. if FLUSH TO ZERO underflow is active.
*
         IF ( .NOT. ALMORT ) THEN
*
            IF ( L2PERT ) THEN
*              XSC = SQRT(SMALL)
               XSC = EPSLN / DFLOAT(N)
               DO 4947 q = 1, NR
                  CTEMP = DCMPLX(XSC*ABS(A(q,q)),ZERO)
                  DO 4949 p = 1, N
                     IF ( ( (p.GT.q) .AND. (ABS(A(p,q)).LE.TEMP1) )
     $                    .OR. ( p .LT. q ) )
*     $                     A(p,q) = TEMP1 * ( A(p,q) / ABS(A(p,q)) )
     $                     A(p,q) = CTEMP
 4949             CONTINUE
 4947          CONTINUE
            ELSE
               CALL ZLASET( 'U', NR-1,NR-1, CZERO,CZERO, A(1,2),LDA )
            END IF
*
*           .. second preconditioning using the QR factorization
*
            CALL ZGEQRF( N,NR, A,LDA, CWORK, CWORK(N+1),LWORK-N, IERR )
*
*           .. and transpose upper to lower triangular
            DO 1948 p = 1, NR - 1
               CALL ZCOPY( NR-p, A(p,p+1), LDA, A(p+1,p), 1 )
               CALL ZLACGV( NR-p+1, A(p,p), 1 )
 1948       CONTINUE
*
      END IF
*
```

```
*           Row-cyclic Jacobi SVD algorithm with column pivoting
*
*           .. again some perturbation (a "background noise") is added
*           to drown denormals
            IF ( L2PERT ) THEN
*              XSC = SQRT(SMALL)
               XSC = EPSLN / DFLOAT(N)
               DO 1947 q = 1, NR
                  CTEMP = DCMPLX(XSC*ABS(A(q,q)),ZERO)
                  DO 1949 p = 1, NR
                     IF ( ( (p.GT.q) .AND. (ABS(A(p,q)).LE.TEMP1) )
     $                       .OR. ( p .LT. q ) )
*     $                   A(p,q) = TEMP1 * ( A(p,q) / ABS(A(p,q)) )
     $                   A(p,q) = CTEMP
 1949             CONTINUE
 1947          CONTINUE
            ELSE
               CALL ZLASET( 'U', NR-1, NR-1, CZERO, CZERO, A(1,2), LDA )
            END IF
*
*           .. and one-sided Jacobi rotations are started on a lower
*           triangular matrix (plus perturbation which is ignored in
*           the part which destroys triangular form (confusing?!))
*
            CALL ZGESVJ( 'L', 'NoU', 'NoV', NR, NR, A, LDA, SVA,
     $                N, V, LDV, CWORK, LWORK, RWORK, LRWORK, INFO )
*
            SCALEM  = RWORK(1)
            NUMRANK = NINT(RWORK(2))
*
*
      ELSE IF ( RSVEC .AND. ( .NOT. LSVEC ) ) THEN
*
*        -> Singular Values and Right Singular Vectors <-
*
         IF ( ALMORT ) THEN
*
*           .. in this case NR equals N
            DO 1998 p = 1, NR
               CALL ZCOPY( N-p+1, A(p,p), LDA, V(p,p), 1 )
               CALL ZLACGV( N-p+1, V(p,p), 1 )
 1998       CONTINUE
            CALL ZLASET( 'Upper', NR-1,NR-1, CZERO, CZERO, V(1,2), LDV )
*
            CALL ZGESVJ( 'L','U','N', N, NR, V,LDV, SVA, NR, A,LDA,
     $                  CWORK, LWORK, RWORK, LRWORK, INFO )
            SCALEM  = RWORK(1)
            NUMRANK = NINT(RWORK(2))

         ELSE
```

```
*
*        .. two more QR factorizations ( one QRF is not enough, two require
*        accumulated product of Jacobi rotations, three are perfect )
*
            CALL ZLASET( 'Lower', NR-1,NR-1, CZERO, CZERO, A(2,1), LDA )
            CALL ZGELQF( NR,N, A, LDA, CWORK, CWORK(N+1), LWORK-N, IERR)
            CALL ZLACPY( 'Lower', NR, NR, A, LDA, V, LDV )
            CALL ZLASET( 'Upper', NR-1,NR-1, CZERO, CZERO, V(1,2), LDV )
            CALL ZGEQRF( NR, NR, V, LDV, CWORK(N+1), CWORK(2*N+1),
     $                   LWORK-2*N, IERR )
            DO 8998 p = 1, NR
               CALL ZCOPY( NR-p+1, V(p,p), LDV, V(p,p), 1 )
               CALL ZLACGV( NR-p+1, V(p,p), 1 )
 8998       CONTINUE
            CALL ZLASET( 'Upper', NR-1, NR-1, ZERO, ZERO, V(1,2), LDV )
*
            CALL ZGESVJ( 'Lower', 'U','N', NR, NR, V,LDV, SVA, NR, U,
     $                  LDU, CWORK(N+1), LWORK-N, RWORK, LRWORK, INFO )
            SCALEM  = RWORK(1)
            NUMRANK = NINT(RWORK(2))
            IF ( NR .LT. N ) THEN
               CALL ZLASET( 'A',N-NR, NR, CZERO,CZERO, V(NR+1,1),  LDV )
               CALL ZLASET( 'A',NR, N-NR, CZERO,CZERO, V(1,NR+1),  LDV )
               CALL ZLASET( 'A',N-NR,N-NR,CZERO,CONE, V(NR+1,NR+1),LDV )
            END IF
*
         CALL ZUNMLQ( 'Left', 'C', N, N, NR, A, LDA, CWORK,
     $               V, LDV, CWORK(N+1), LWORK-N, IERR )
*
         END IF
*
         DO 8991 p = 1, N
            CALL ZCOPY( N, V(p,1), LDV, A(IWORK(p),1), LDA )
 8991    CONTINUE
         CALL ZLACPY( 'All', N, N, A, LDA, V, LDV )
*
         IF ( TRANSP ) THEN
            CALL ZLACPY( 'All', N, N, V, LDV, U, LDU )
         END IF
*
      ELSE IF ( LSVEC .AND. ( .NOT. RSVEC ) ) THEN
*
*        .. Singular Values and Left Singular Vectors                 ..
*
*        .. second preconditioning step to avoid need to accumulate
*        Jacobi rotations in the Jacobi iterations.
         DO 1965 p = 1, NR
            CALL ZCOPY( N-p+1, A(p,p), LDA, U(p,p), 1 )
            CALL ZLACGV( N-p+1, U(p,p), 1 )
 1965    CONTINUE
```

```
         CALL ZLASET( 'Upper', NR-1, NR-1, CZERO, CZERO, U(1,2), LDU )
*
         CALL ZGEQRF( N, NR, U, LDU, CWORK(N+1), CWORK(2*N+1),
     $              LWORK-2*N, IERR )
*
         DO 1967 p = 1, NR - 1
            CALL ZCOPY( NR-p, U(p,p+1), LDU, U(p+1,p), 1 )
            CALL ZLACGV( N-p+1, U(p,p), 1 )
 1967    CONTINUE
         CALL ZLASET( 'Upper', NR-1, NR-1, CZERO, CZERO, U(1,2), LDU )
*
         CALL ZGESVJ( 'Lower', 'U', 'N', NR,NR, U, LDU, SVA, NR, A,
     $        LDA, CWORK(N+1), LWORK-N, RWORK, LRWORK, INFO )
         SCALEM  = RWORK(1)
         NUMRANK = NINT(RWORK(2))
*
         IF ( NR .LT. M ) THEN
            CALL ZLASET( 'A',  M-NR, NR,CZERO, CZERO, U(NR+1,1), LDU )
            IF ( NR .LT. N1 ) THEN
               CALL ZLASET( 'A',NR, N1-NR, CZERO, CZERO, U(1,NR+1),LDU )
               CALL ZLASET( 'A',M-NR,N1-NR,CZERO,CONE,U(NR+1,NR+1),LDU )
            END IF
         END IF
*
         CALL ZUNMQR( 'Left', 'No Tr', M, N1, N, A, LDA, CWORK, U,
     $               LDU, CWORK(N+1), LWORK-N, IERR )
*
         IF ( ROWPIV )
     $       CALL ZLASWP( N1, U, LDU, 1, M-1, IWORK(2*N+1), -1 )
*
         DO 1974 p = 1, N1
            XSC = ONE / DZNRM2( M, U(1,p), 1 )
            CALL ZDSCAL( M, XSC, U(1,p), 1 )
 1974    CONTINUE
*
         IF ( TRANSP ) THEN
            CALL ZLACPY( 'All', N, N, U, LDU, V, LDV )
         END IF
*
      ELSE
*
*        .. Full SVD ..
*
         IF ( .NOT. JRACC ) THEN
*
         IF ( .NOT. ALMORT ) THEN
*
*           Second Preconditioning Step (QRF [with pivoting])
*           Note that the composition of TRANSPOSE, QRF and TRANSPOSE is
*           equivalent to an LQF CALL. Since in many libraries the QRF
```

```
*           seems to be better optimized than the LQF, we do explicit
*           transpose and use the QRF. This is subject to changes in an
*           optimized implementation of ZGEJSV.
*
            DO 1968 p = 1, NR
               CALL ZCOPY( N-p+1, A(p,p), LDA, V(p,p), 1 )
               CALL ZLACGV( N-p+1, V(p,p), 1 )
 1968       CONTINUE
*
*           .. the following two loops perturb small entries to avoid
*           denormals in the second QR factorization, where they are
*           as good as zeros. This is done to avoid painfully slow
*           computation with denormals. The relative size of the perturbation
*           is a parameter that can be changed by the implementer.
*           This perturbation device will be obsolete on machines with
*           properly implemented arithmetic.
*           To switch it off, set L2PERT=.FALSE. To remove it from  the
*           code, remove the action under L2PERT=.TRUE., leave the ELSE part.
*           The following two loops should be blocked and fused with the
*           transposed copy above.
*
            IF ( L2PERT ) THEN
               XSC = DSQRT(SMALL)
               DO 2969 q = 1, NR
                  CTEMP = DCMPLX(XSC*ABS( V(q,q) ),ZERO)
                  DO 2968 p = 1, N
                     IF ( ( p .GT. q ) .AND. ( ABS(V(p,q)) .LE. TEMP1 )
     $                   .OR. ( p .LT. q ) )
*     $                  V(p,q) = TEMP1 * ( V(p,q) / ABS(V(p,q)) )
     $                  V(p,q) = CTEMP
                     IF ( p .LT. q ) V(p,q) = - V(p,q)
 2968             CONTINUE
 2969          CONTINUE
            ELSE
               CALL ZLASET( 'U', NR-1, NR-1, CZERO, CZERO, V(1,2), LDV )
            END IF
*
*           Estimate the row scaled condition number of R1
*           (If R1 is rectangular, N > NR, then the condition number
*           of the leading NR x NR submatrix is estimated.)
*
            CALL ZLACPY( 'L', NR, NR, V, LDV, CWORK(2*N+1), NR )
            DO 3950 p = 1, NR
               TEMP1 = DZNRM2(NR-p+1,CWORK(2*N+(p-1)*NR+p),1)
               CALL ZDSCAL(NR-p+1,ONE/TEMP1,CWORK(2*N+(p-1)*NR+p),1)
 3950       CONTINUE
            CALL ZPOCON('Lower',NR,CWORK(2*N+1),NR,ONE,TEMP1,
     $                  CWORK(2*N+NR*NR+1),RWORK,IERR)
            CONDR1 = ONE / DSQRT(TEMP1)
*           .. here need a second oppinion on the condition number
```

```
*           .. then assume worst case scenario
*           R1 is OK for inverse <=> CONDR1 .LT. DFLOAT(N)
*           more conservative    <=> CONDR1 .LT. SQRT(DFLOAT(N))
*
            COND_OK = DSQRT(DSQRT(DFLOAT(NR)))
*[TP]       COND_OK is a tuning parameter.
*
            IF ( CONDR1 .LT. COND_OK ) THEN
*              .. the second QRF without pivoting. Note: in an optimized
*              implementation, this QRF should be implemented as the QRF
*              of a lower triangular matrix.
*              R1^* = Q2 * R2
               CALL ZGEQRF( N, NR, V, LDV, CWORK(N+1), CWORK(2*N+1),
     $              LWORK-2*N, IERR )
*
               IF ( L2PERT ) THEN
                  XSC = DSQRT(SMALL)/EPSLN
                  DO 3959 p = 2, NR
                     DO 3958 q = 1, p - 1
                        CTEMP=DCMPLX(XSC*DMIN1(ABS(V(p,p)),ABS(V(q,q))),
     $                            ZERO)
                        IF ( ABS(V(q,p)) .LE. TEMP1 )
*     $                     V(q,p) = TEMP1 * ( V(q,p) / ABS(V(q,p)) )
     $                     V(q,p) = CTEMP
 3958                CONTINUE
 3959             CONTINUE
               END IF
*
               IF ( NR .NE. N )
     $         CALL ZLACPY( 'A', N, NR, V, LDV, CWORK(2*N+1), N )
*              .. save ...
*
*           .. this transposed copy should be better than naive
               DO 1969 p = 1, NR - 1
                  CALL ZCOPY( NR-p, V(p,p+1), LDV, V(p+1,p), 1 )
                  CALL ZLACGV(NR-p+1, V(p,p), 1 )
 1969          CONTINUE
               V(NR,NR)=DCONJG(V(NR,NR))
*
               CONDR2 = CONDR1
*
            ELSE
*
*              .. ill-conditioned case: second QRF with pivoting
*              Note that windowed pivoting would be equaly good
*              numerically, and more run-time efficient. So, in
*              an optimal implementation, the next call to ZGEQP3
*              should be replaced with eg. CALL ZGEQPX (ACM TOMS #782)
*              with properly (carefully) chosen parameters.
*
```

```
*              R1^* * P2 = Q2 * R2
               DO 3003 p = 1, NR
                  IWORK(N+p) = 0
 3003          CONTINUE
               CALL ZGEQP3( N, NR, V, LDV, IWORK(N+1), CWORK(N+1),
     $                 CWORK(2*N+1), LWORK-2*N, RWORK, IERR )
**               CALL ZGEQRF( N, NR, V, LDV, CWORK(N+1), CWORK(2*N+1),
**     $              LWORK-2*N, IERR )
               IF ( L2PERT ) THEN
                  XSC = DSQRT(SMALL)
                  DO 3969 p = 2, NR
                     DO 3968 q = 1, p - 1
                        CTEMP=DCMPLX(XSC*DMIN1(ABS(V(p,p)),ABS(V(q,q))),
     $                                ZERO)
                        IF ( ABS(V(q,p)) .LE. TEMP1 )
*     $                     V(q,p) = TEMP1 * ( V(q,p) / ABS(V(q,p)) )
     $                     V(q,p) = CTEMP
 3968                CONTINUE
 3969             CONTINUE
               END IF
*
               CALL ZLACPY( 'A', N, NR, V, LDV, CWORK(2*N+1), N )
*
               IF ( L2PERT ) THEN
                  XSC = DSQRT(SMALL)
                  DO 8970 p = 2, NR
                     DO 8971 q = 1, p - 1
                        CTEMP=DCMPLX(XSC*DMIN1(ABS(V(p,p)),ABS(V(q,q))),
     $                                ZERO)
*                        V(p,q) = - TEMP1*( V(q,p) / ABS(V(q,p)) )
                        V(p,q) = - CTEMP
 8971                CONTINUE
 8970             CONTINUE
               ELSE
                  CALL ZLASET( 'L',NR-1,NR-1,CZERO,CZERO,V(2,1),LDV )
               END IF
*              Now, compute R2 = L3 * Q3, the LQ factorization.
               CALL ZGELQF( NR, NR, V, LDV, CWORK(2*N+N*NR+1),
     $              CWORK(2*N+N*NR+NR+1), LWORK-2*N-N*NR-NR, IERR )
*              .. and estimate the condition number
               CALL ZLACPY( 'L',NR,NR,V,LDV,CWORK(2*N+N*NR+NR+1),NR )
               DO 4950 p = 1, NR
                  TEMP1 = DZNRM2( p, CWORK(2*N+N*NR+NR+p), NR )
                  CALL ZDSCAL( p, ONE/TEMP1, CWORK(2*N+N*NR+NR+p), NR )
 4950          CONTINUE
               CALL ZPOCON( 'L',NR,CWORK(2*N+N*NR+NR+1),NR,ONE,TEMP1,
     $             CWORK(2*N+N*NR+NR+NR*NR+1),RWORK,IERR )
               CONDR2 = ONE / DSQRT(TEMP1)
*
*
```

```
               IF ( CONDR2 .GE. COND_OK ) THEN
*                 .. save the Householder vectors used for Q3
*                 (this overwrittes the copy of R2, as it will not be
*                 needed in this branch, but it does not overwritte the
*                 Huseholder vectors of Q2.).
                  CALL ZLACPY( 'U', NR, NR, V, LDV, CWORK(2*N+1), N )
*                 .. and the rest of the information on Q3 is in
*                 WORK(2*N+N*NR+1:2*N+N*NR+N)
               END IF
*
            END IF
*
            IF ( L2PERT ) THEN
               XSC = DSQRT(SMALL)
               DO 4968 q = 2, NR
                  CTEMP = XSC * V(q,q)
                  DO 4969 p = 1, q - 1
*                     V(p,q) = - TEMP1*( V(p,q) / ABS(V(p,q)) )
                     V(p,q) = - CTEMP
 4969             CONTINUE
 4968          CONTINUE
            ELSE
               CALL ZLASET( 'U', NR-1,NR-1, CZERO,CZERO, V(1,2), LDV )
            END IF
*
*        Second preconditioning finished; continue with Jacobi SVD
*        The input matrix is lower trinagular.
*
*        Recover the right singular vectors as solution of a well
*        conditioned triangular matrix equation.
*
            IF ( CONDR1 .LT. COND_OK ) THEN
*
               CALL ZGESVJ( 'L','U','N',NR,NR,V,LDV,SVA,NR,U, LDU,
     $              CWORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,RWORK,
     $              LRWORK, INFO )
               SCALEM  = RWORK(1)
               NUMRANK = NINT(RWORK(2))
               DO 3970 p = 1, NR
                  CALL ZCOPY(  NR, V(1,p), 1, U(1,p), 1 )
                  CALL ZDSCAL( NR, SVA(p),    V(1,p), 1 )
 3970          CONTINUE

*        .. pick the right matrix equation and solve it
*
               IF ( NR .EQ. N ) THEN
* :))              .. best case, R1 is inverted. The solution of this matrix
*                  equation is Q2*V2 = the product of the Jacobi rotations
*                  used in ZGESVJ, premultiplied with the orthogonal matrix
*                  from the second QR factorization.
```

```
                 CALL ZTRSM('L','U','N','N', NR,NR,CONE, A,LDA, V,LDV)
              ELSE
*                .. R1 is well conditioned, but non-square. Adjoint of R2
*                is inverted to get the product of the Jacobi rotations
*                used in ZGESVJ. The Q-factor from the second QR
*                factorization is then built in explicitly.
                 CALL ZTRSM('L','U','C','N',NR,NR,CONE,CWORK(2*N+1),
     $                N,V,LDV)
                 IF ( NR .LT. N ) THEN
                  CALL ZLASET('A',N-NR,NR,ZERO,CZERO,V(NR+1,1),LDV)
                  CALL ZLASET('A',NR,N-NR,ZERO,CZERO,V(1,NR+1),LDV)
                  CALL ZLASET('A',N-NR,N-NR,ZERO,CONE,V(NR+1,NR+1),LDV)
                 END IF
                 CALL ZUNMQR('L','N',N,N,NR,CWORK(2*N+1),N,CWORK(N+1),
     $               V,LDV,CWORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,IERR)
              END IF
*
           ELSE IF ( CONDR2 .LT. COND_OK ) THEN
*
*             The matrix R2 is inverted. The solution of the matrix equation
*             is Q3^* * V3 = the product of the Jacobi rotations (appplied to
*             the lower triangular L3 from the LQ factorization of
*             R2=L3*Q3), pre-multiplied with the transposed Q3.
              CALL ZGESVJ( 'L', 'U', 'N', NR, NR, V, LDV, SVA, NR, U,
     $             LDU, CWORK(2*N+N*NR+NR+1), LWORK-2*N-N*NR-NR,
     $         RWORK, LRWORK, INFO )
              SCALEM  = RWORK(1)
              NUMRANK = NINT(RWORK(2))
              DO 3870 p = 1, NR
                 CALL ZCOPY( NR, V(1,p), 1, U(1,p), 1 )
                 CALL ZDSCAL( NR, SVA(p),    U(1,p), 1 )
 3870         CONTINUE
              CALL ZTRSM('L','U','N','N',NR,NR,CONE,CWORK(2*N+1),N,
     $                   U,LDU)
*             .. apply the permutation from the second QR factorization
              DO 873 q = 1, NR
                 DO 872 p = 1, NR
                    CWORK(2*N+N*NR+NR+IWORK(N+p)) = U(p,q)
 872             CONTINUE
                 DO 874 p = 1, NR
                    U(p,q) = CWORK(2*N+N*NR+NR+p)
 874             CONTINUE
 873          CONTINUE
              IF ( NR .LT. N ) THEN
                 CALL ZLASET( 'A',N-NR,NR,CZERO,CZERO,V(NR+1,1),LDV )
                 CALL ZLASET( 'A',NR,N-NR,CZERO,CZERO,V(1,NR+1),LDV )
                 CALL ZLASET('A',N-NR,N-NR,CZERO,CONE,V(NR+1,NR+1),LDV)
              END IF
              CALL ZUNMQR( 'L','N',N,N,NR,CWORK(2*N+1),N,CWORK(N+1),
     $             V,LDV,CWORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,IERR )
```

```
            ELSE
*              Last line of defense.
* #:(          This is a rather pathological case: no scaled condition
*              improvement after two pivoted QR factorizations. Other
*              possibility is that the rank revealing QR factorization
*              or the condition estimator has failed, or the COND_OK
*              is set very close to ONE (which is unnecessary). Normally,
*              this branch should never be executed, but in rare cases of
*              failure of the RRQR or condition estimator, the last line of
*              defense ensures that ZGEJSV completes the task.
*              Compute the full SVD of L3 using ZGESVJ with explicit
*              accumulation of Jacobi rotations.
               CALL ZGESVJ( 'L', 'U', 'V', NR, NR, V, LDV, SVA, NR, U,
     $              LDU, CWORK(2*N+N*NR+NR+1), LWORK-2*N-N*NR-NR,
     $                         RWORK, LRWORK, INFO )
               SCALEM  = RWORK(1)
               NUMRANK = NINT(RWORK(2))
               IF ( NR .LT. N ) THEN
                  CALL ZLASET( 'A',N-NR,NR,CZERO,CZERO,V(NR+1,1),LDV )
                  CALL ZLASET( 'A',NR,N-NR,CZERO,CZERO,V(1,NR+1),LDV )
                  CALL ZLASET('A',N-NR,N-NR,CZERO,CONE,V(NR+1,NR+1),LDV)
               END IF
               CALL ZUNMQR( 'L','N',N,N,NR,CWORK(2*N+1),N,CWORK(N+1),
     $              V,LDV,CWORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,IERR )
*
               CALL ZUNMLQ( 'L', 'C', NR, NR, NR, CWORK(2*N+1), N,
     $              CWORK(2*N+N*NR+1), U, LDU, CWORK(2*N+N*NR+NR+1),
     $              LWORK-2*N-N*NR-NR, IERR )
               DO 773 q = 1, NR
                  DO 772 p = 1, NR
                     CWORK(2*N+N*NR+NR+IWORK(N+p)) = U(p,q)
 772              CONTINUE
                  DO 774 p = 1, NR
                     U(p,q) = CWORK(2*N+N*NR+NR+p)
 774              CONTINUE
 773           CONTINUE
*
            END IF
*
*           Permute the rows of V using the (column) permutation from the
*           first QRF. Also, scale the columns to make them unit in
*           Euclidean norm. This applies to all cases.
*
            TEMP1 = DSQRT(DFLOAT(N)) * EPSLN
            DO 1972 q = 1, N
               DO 972 p = 1, N
                  CWORK(2*N+N*NR+NR+IWORK(p)) = V(p,q)
  972          CONTINUE
               DO 973 p = 1, N
                  V(p,q) = CWORK(2*N+N*NR+NR+p)
```

```
 973          CONTINUE
              XSC = ONE / DZNRM2( N, V(1,q), 1 )
              IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $          CALL ZDSCAL( N, XSC, V(1,q), 1 )
 1972       CONTINUE
*           At this moment, V contains the right singular vectors of A.
*           Next, assemble the left singular vector matrix U (M x N).
            IF ( NR .LT. M ) THEN
              CALL ZLASET('A', M-NR, NR, CZERO, CZERO, U(NR+1,1), LDU)
              IF ( NR .LT. N1 ) THEN
                 CALL ZLASET('A',NR,N1-NR,CZERO,CZERO,U(1,NR+1),LDU)
                 CALL ZLASET('A',M-NR,N1-NR,CZERO,CONE,
     $                       U(NR+1,NR+1),LDU)
              END IF
            END IF
*
*           The Q matrix from the first QRF is built into the left singular
*           matrix U. This applies to all cases.
*
            CALL ZUNMQR( 'Left', 'No_Tr', M, N1, N, A, LDA, CWORK, U,
     $           LDU, CWORK(N+1), LWORK-N, IERR )

*           The columns of U are normalized. The cost is O(M*N) flops.
            TEMP1 = DSQRT(DFLOAT(M)) * EPSLN
            DO 1973 p = 1, NR
              XSC = ONE / DZNRM2( M, U(1,p), 1 )
              IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $          CALL ZDSCAL( M, XSC, U(1,p), 1 )
 1973       CONTINUE
*
*           If the initial QRF is computed with row pivoting, the left
*           singular vectors must be adjusted.
*
            IF ( ROWPIV )
     $          CALL ZLASWP( N1, U, LDU, 1, M-1, IWORK(2*N+1), -1 )
*
         ELSE
*
*        .. the initial matrix A has almost orthogonal columns and
*        the second QRF is not needed
*
            CALL ZLACPY( 'Upper', N, N, A, LDA, CWORK(N+1), N )
            IF ( L2PERT ) THEN
              XSC = DSQRT(SMALL)
              DO 5970 p = 2, N
                 CTEMP = XSC * CWORK( N + (p-1)*N + p )
                 DO 5971 q = 1, p - 1
*                    CWORK(N+(q-1)*N+p)=-TEMP1 * ( CWORK(N+(p-1)*N+q) /
*     $                                      ABS(CWORK(N+(p-1)*N+q)) )
                   CWORK(N+(q-1)*N+p)=-CTEMP
```

```
 5971            CONTINUE
 5970         CONTINUE
           ELSE
              CALL ZLASET( 'Lower',N-1,N-1,CZERO,CZERO,CWORK(N+2),N )
           END IF
*
           CALL ZGESVJ( 'Upper', 'U', 'N', N, N, CWORK(N+1), N, SVA,
     $           N, U, LDU, CWORK(N+N*N+1), LWORK-N-N*N, RWORK, LRWORK,
     $       INFO )
*
           SCALEM  = RWORK(1)
           NUMRANK = NINT(RWORK(2))
           DO 6970 p = 1, N
              CALL ZCOPY( N, CWORK(N+(p-1)*N+1), 1, U(1,p), 1 )
              CALL ZDSCAL( N, SVA(p), CWORK(N+(p-1)*N+1), 1 )
 6970      CONTINUE
*
           CALL ZTRSM( 'Left', 'Upper', 'NoTrans', 'No UD', N, N,
     $           CONE, A, LDA, CWORK(N+1), N )
           DO 6972 p = 1, N
              CALL ZCOPY( N, CWORK(N+p), N, V(IWORK(p),1), LDV )
 6972      CONTINUE
           TEMP1 = DSQRT(DFLOAT(N))*EPSLN
           DO 6971 p = 1, N
              XSC = ONE / DZNRM2( N, V(1,p), 1 )
              IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $           CALL ZDSCAL( N, XSC, V(1,p), 1 )
 6971      CONTINUE
*
*          Assemble the left singular vector matrix U (M x N).
*
           IF ( N .LT. M ) THEN
              CALL ZLASET( 'A',  M-N, N, CZERO, CZERO, U(N+1,1), LDU )
              IF ( N .LT. N1 ) THEN
                 CALL ZLASET('A',N,  N1-N, CZERO, CZERO,  U(1,N+1),LDU)
                 CALL ZLASET( 'A',M-N,N1-N, CZERO, CONE,U(N+1,N+1),LDU)
              END IF
           END IF
           CALL ZUNMQR( 'Left', 'No Tr', M, N1, N, A, LDA, CWORK, U,
     $           LDU, CWORK(N+1), LWORK-N, IERR )
           TEMP1 = DSQRT(DFLOAT(M))*EPSLN
           DO 6973 p = 1, N1
              XSC = ONE / DZNRM2( M, U(1,p), 1 )
              IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $           CALL ZDSCAL( M, XSC, U(1,p), 1 )
 6973      CONTINUE
*
           IF ( ROWPIV )
     $        CALL ZLASWP( N1, U, LDU, 1, M-1, IWORK(2*N+1), -1 )
*
```

```
         END IF
*
*        end of the  >> almost orthogonal case <<  in the full SVD
*
         ELSE
*
*        This branch deploys a preconditioned Jacobi SVD with explicitly
*        accumulated rotations. It is included as optional, mainly for
*        experimental purposes. It does perfom well, and can also be used.
*        In this implementation, this branch will be automatically activated
*        if the  condition number sigma_max(A) / sigma_min(A) is predicted
*        to be greater than the overflow threshold. This is because the
*        a posteriori computation of the singular vectors assumes robust
*        implementation of BLAS and some LAPACK procedures, capable of working
*        in presence of extreme values. Since that is not always the case, ...
*
         DO 7968 p = 1, NR
            CALL ZCOPY( N-p+1, A(p,p), LDA, V(p,p), 1 )
            CALL ZLACGV( N-p+1, V(p,p), 1 )
 7968    CONTINUE
*
         IF ( L2PERT ) THEN
            XSC = DSQRT(SMALL/EPSLN)
            DO 5969 q = 1, NR
               CTEMP = DCMPLX(XSC*ABS( V(q,q) ),ZERO)
               DO 5968 p = 1, N
                  IF ( ( p .GT. q ) .AND. ( ABS(V(p,q)) .LE. TEMP1 )
     $                .OR. ( p .LT. q ) )
*     $                V(p,q) = TEMP1 * ( V(p,q) / ABS(V(p,q)) )
     $                V(p,q) = CTEMP
                  IF ( p .LT. q ) V(p,q) = - V(p,q)
 5968          CONTINUE
 5969       CONTINUE
         ELSE
            CALL ZLASET( 'U', NR-1, NR-1, CZERO, CZERO, V(1,2), LDV )
         END IF

         CALL ZGEQRF( N, NR, V, LDV, CWORK(N+1), CWORK(2*N+1),
     $        LWORK-2*N, IERR )
         CALL ZLACPY( 'L', N, NR, V, LDV, CWORK(2*N+1), N )
*
         DO 7969 p = 1, NR
            CALL ZCOPY( NR-p+1, V(p,p), LDV, U(p,p), 1 )
            CALL ZLACGV( NR-p+1, U(p,p), 1 )
 7969    CONTINUE

         IF ( L2PERT ) THEN
            XSC = DSQRT(SMALL/EPSLN)
            DO 9970 q = 2, NR
               DO 9971 p = 1, q - 1
```

```
                 CTEMP = DCMPLX(XSC * DMIN1(ABS(U(p,p)),ABS(U(q,q))),
     $                          ZERO)
*                 U(p,q) = - TEMP1 * ( U(q,p) / ABS(U(q,p)) )
                 U(p,q) = - CTEMP
 9971          CONTINUE
 9970       CONTINUE
         ELSE
            CALL ZLASET('U', NR-1, NR-1, CZERO, CZERO, U(1,2), LDU )
         END IF

         CALL ZGESVJ( 'L', 'U', 'V', NR, NR, U, LDU, SVA,
     $        N, V, LDV, CWORK(2*N+N*NR+1), LWORK-2*N-N*NR,
     $         RWORK, LRWORK, INFO )
         SCALEM  = RWORK(1)
         NUMRANK = NINT(RWORK(2))

         IF ( NR .LT. N ) THEN
            CALL ZLASET( 'A',N-NR,NR,ZERO,ZERO,V(NR+1,1),LDV )
            CALL ZLASET( 'A',NR,N-NR,ZERO,ZERO,V(1,NR+1),LDV )
            CALL ZLASET( 'A',N-NR,N-NR,ZERO,ONE,V(NR+1,NR+1),LDV )
         END IF

         CALL ZUNMQR( 'L','N',N,N,NR,CWORK(2*N+1),N,CWORK(N+1),
     $        V,LDV,CWORK(2*N+N*NR+NR+1),LWORK-2*N-N*NR-NR,IERR )
*
*           Permute the rows of V using the (column) permutation from the
*           first QRF. Also, scale the columns to make them unit in
*           Euclidean norm. This applies to all cases.
*
            TEMP1 = DSQRT(DFLOAT(N)) * EPSLN
            DO 7972 q = 1, N
               DO 8972 p = 1, N
                  CWORK(2*N+N*NR+NR+IWORK(p)) = V(p,q)
 8972          CONTINUE
               DO 8973 p = 1, N
                  V(p,q) = CWORK(2*N+N*NR+NR+p)
 8973          CONTINUE
               XSC = ONE / DZNRM2( N, V(1,q), 1 )
               IF ( (XSC .LT. (ONE-TEMP1)) .OR. (XSC .GT. (ONE+TEMP1)) )
     $           CALL ZDSCAL( N, XSC, V(1,q), 1 )
 7972       CONTINUE
*
*           At this moment, V contains the right singular vectors of A.
*           Next, assemble the left singular vector matrix U (M x N).
*
         IF ( NR .LT. M ) THEN
            CALL ZLASET( 'A',  M-NR, NR, CZERO, CZERO, U(NR+1,1), LDU )
            IF ( NR .LT. N1 ) THEN
               CALL ZLASET('A',NR,  N1-NR, CZERO, CZERO,  U(1,NR+1),LDU)
               CALL ZLASET('A',M-NR,N1-NR, CZERO, CONE,U(NR+1,NR+1),LDU)
```

```
            END IF
         END IF
*
         CALL ZUNMQR( 'Left', 'No Tr', M, N1, N, A, LDA, CWORK, U,
     $        LDU, CWORK(N+1), LWORK-N, IERR )
*
            IF ( ROWPIV )
     $         CALL ZLASWP( N1, U, LDU, 1, M-1, IWORK(2*N+1), -1 )
*
*
         END IF
         IF ( TRANSP ) THEN
*           .. swap U and V because the procedure worked on A^*
            DO 6974 p = 1, N
               CALL ZSWAP( N, U(1,p), 1, V(1,p), 1 )
 6974       CONTINUE
         END IF
*
      END IF
*     end of the full SVD
*
*     Undo scaling, if necessary (and possible)
*
      IF ( USCAL2 .LE. (BIG/SVA(1))*USCAL1 ) THEN
         CALL ZLASCL( 'G', 0, 0, USCAL1, USCAL2, NR, 1, SVA, N, IERR )
         USCAL1 = ONE
         USCAL2 = ONE
      END IF
*
      IF ( NR .LT. N ) THEN
         DO 3004 p = NR+1, N
            SVA(p) = ZERO
 3004    CONTINUE
      END IF
*
      RWORK(1) = USCAL2 * SCALEM
      RWORK(2) = USCAL1
      IF ( ERREST ) RWORK(3) = SCONDA
      IF ( LSVEC .AND. RSVEC ) THEN
         RWORK(4) = CONDR1
         RWORK(5) = CONDR2
      END IF
      IF ( L2TRAN ) THEN
         RWORK(6) = ENTRA
         RWORK(7) = ENTRAT
      END IF
*
      IWORK(1) = NR
      IWORK(2) = NUMRANK
      IWORK(3) = WARNING
```

```
*
      RETURN
*     ..
*     .. END OF ZGEJSV
*     ..
      END
*
```

———-

31.1.4 lisp code

— **zgejsv.lisp** —

```
(let* ((zero 0.0d0)
       (one 1.0d0)
       (czero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (cone (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (f2cl-lib:complex16) czero)
           (type (f2cl-lib:complex16) cone)
           (ignorable zero one czero cone))
  (defun zgejsv
         (joba jobu jobv jobr jobt jobp m n a lda sva u ldu v ldv cwork lwork
          rwork lrwork iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (array double-float (*)) rwork sva)
             (type (array f2cl-lib:complex16 (*)) cwork v u a)
             (type (f2cl-lib:integer4) info lrwork lwork ldv ldu lda n m)
             (type (string *) jobp jobt jobr jobv jobu joba))
    (f2cl-lib:with-multi-array-data
        ((joba character joba-%data% joba-%offset%)
         (jobu character jobu-%data% jobu-%offset%)
         (jobv character jobv-%data% jobv-%offset%)
         (jobr character jobr-%data% jobr-%offset%)
         (jobt character jobt-%data% jobt-%offset%)
         (jobp character jobp-%data% jobp-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (u f2cl-lib:complex16 u-%data% u-%offset%)
         (v f2cl-lib:complex16 v-%data% v-%offset%)
         (cwork f2cl-lib:complex16 cwork-%data% cwork-%offset%)
         (sva double-float sva-%data% sva-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((almort nil) (defr nil) (errest nil) (goscal nil) (jracc nil)
             (kill nil) (lsvec nil) (l2aber nil) (l2kill nil) (l2pert nil)
             (l2rank nil) (l2tran nil) (noscal nil) (rowpiv nil) (rsvec nil)
```

```
     (transp nil) (ierr 0) (n1 0) (nr 0) (numrank 0) (p 0) (q 0)
     (warning 0) (aapp 0.0d0) (aaqq 0.0d0) (aatmax 0.0d0)
     (aatmin 0.0d0) (big 0.0d0) (big1 0.0d0) (cond_ok 0.0d0)
     (condr1 0.0d0) (condr2 0.0d0) (entra 0.0d0) (entrat 0.0d0)
     (epsln 0.0d0) (maxprj 0.0d0) (scalem 0.0d0) (sconda 0.0d0)
     (sfmin 0.0d0) (small 0.0d0) (temp1 0.0d0) (uscal1 0.0d0)
     (uscal2 0.0d0) (xsc 0.0d0) (ctemp #C(0.0d0 0.0d0)))
(declare
 (type f2cl-lib:logical almort defr errest goscal jracc kill lsvec
  l2aber l2kill l2pert l2rank l2tran noscal rowpiv rsvec transp)
 (type (f2cl-lib:integer4) ierr n1 nr numrank p q warning)
 (type (double-float) aapp aaqq aatmax aatmin big big1 cond_ok condr1
  condr2 entra entrat epsln maxprj scalem sconda sfmin small temp1
  uscal1 uscal2 xsc)
 (type (f2cl-lib:complex16) ctemp))
(setf lsvec
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame jobu "U")
           (declare (ignore var-1))
           (when var-0 (setf jobu var-0))
           ret-val)
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame jobu "F")
           (declare (ignore var-1))
           (when var-0 (setf jobu var-0))
           ret-val)))
(setf jracc
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobv "J")
          (declare (ignore var-1))
          (when var-0 (setf jobv var-0))
          ret-val))
(setf rsvec
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame jobv "V")
           (declare (ignore var-1))
           (when var-0 (setf jobv var-0))
           ret-val)
         jracc))
(setf rowpiv
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame joba "F")
           (declare (ignore var-1))
           (when var-0 (setf joba var-0))
           ret-val)
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame joba "G")
```

```
            (declare (ignore var-1))
            (when var-0 (setf joba var-0))
            ret-val)))
(setf l2rank
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame joba "R")
          (declare (ignore var-1))
          (when var-0 (setf joba var-0))
          ret-val))
(setf l2aber
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame joba "A")
          (declare (ignore var-1))
          (when var-0 (setf joba var-0))
          ret-val))
(setf errest
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame joba "E")
           (declare (ignore var-1))
           (when var-0 (setf joba var-0))
           ret-val)
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame joba "G")
           (declare (ignore var-1))
           (when var-0 (setf joba var-0))
           ret-val)))
(setf l2tran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobt "T")
          (declare (ignore var-1))
          (when var-0 (setf jobt var-0))
          ret-val))
(setf l2kill
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobr "R")
          (declare (ignore var-1))
          (when var-0 (setf jobr var-0))
          ret-val))
(setf defr
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobr "N")
          (declare (ignore var-1))
          (when var-0 (setf jobr var-0))
          ret-val))
(setf l2pert
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobp "P")
          (declare (ignore var-1))
          (when var-0 (setf jobp var-0))
```

```
          ret-val))
(cond
 ((not
   (or rowpiv l2rank l2aber errest
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame joba "C")
         (declare (ignore var-1))
         (when var-0 (setf joba var-0))
         ret-val)))
  (setf info -1))
 ((not
   (or lsvec
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobu "N")
         (declare (ignore var-1))
         (when var-0 (setf jobu var-0))
         ret-val)
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobu "W")
         (declare (ignore var-1))
         (when var-0 (setf jobu var-0))
         ret-val)))
  (setf info -2))
 ((or
   (not
    (or rsvec
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobv "N")
          (declare (ignore var-1))
          (when var-0 (setf jobv var-0))
          ret-val)
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobv "W")
          (declare (ignore var-1))
          (when var-0 (setf jobv var-0))
          ret-val)))
   (and jracc (not lsvec)))
  (setf info -3))
 ((not (or l2kill defr)) (setf info -4))
 ((not
   (or l2tran
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobt "N")
         (declare (ignore var-1))
         (when var-0 (setf jobt var-0))
         ret-val)))
  (setf info -5))
 ((not
   (or l2pert
       (multiple-value-bind (ret-val var-0 var-1)
```

```
            (lsame jobp "N")
          (declare (ignore var-1))
          (when var-0 (setf jobp var-0))
          ret-val)))
  (setf info -6))
 ((< m 0) (setf info -7)) ((or (< n 0) (> n m)) (setf info -8))
 ((< lda m) (setf info -10)) ((and lsvec (< ldu m)) (setf info -13))
 ((and rsvec (< ldv n)) (setf info -15))
 ((or
   (and (not (or lsvec rsvec errest))
        (< lwork (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) 1)))
   (and (not (or lsvec rsvec)) errest
        (< lwork
           (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                             (f2cl-lib:int-mul 3 n))))
   (and lsvec (not rsvec) (< lwork (f2cl-lib:int-mul 3 n)))
   (and rsvec (not lsvec) (< lwork (f2cl-lib:int-mul 3 n)))
   (and lsvec rsvec (not jracc)
        (< lwork
           (f2cl-lib:int-add (f2cl-lib:int-mul 5 n)
                             (f2cl-lib:int-mul 2 n n))))
   (and lsvec rsvec jracc
        (< lwork
           (f2cl-lib:int-add (f2cl-lib:int-mul 4 n)
                             (f2cl-lib:int-mul n n)))))
  (setf info -17))
 ((< lrwork
     (f2cl-lib:max0 (f2cl-lib:int-add n (f2cl-lib:int-mul 2 m)) 7))
  (setf info -19))
 (t (setf info 0)))
(cond
 ((/= info 0) (xerbla "ZGEJSV" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= m 0) (= n 0))
    (go end_label))
(cond
 (lsvec (setf n1 n)
  (if (multiple-value-bind (ret-val var-0 var-1)
          (lsame jobu "F")
        (declare (ignore var-1))
        (when var-0 (setf jobu var-0))
        ret-val)
      (setf n1 m))))
(setf epsln (dlamch "Epsilon"))
(setf sfmin (dlamch "SafeMinimum"))
(setf small (/ sfmin epsln))
(setf big (dlamch "O"))
(setf scalem
        (/ one
           (f2cl-lib:dsqrt
```

```
            (* (f2cl-lib:dfloat m) (f2cl-lib:dfloat n)))))
(setf noscal f2cl-lib:%true%)
(setf goscal f2cl-lib:%true%)
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (setf aapp zero)
    (setf aaqq one)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zlassq m
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 p)
                               ((1 lda) (1 *)) a-%offset%)
         1 aapp aaqq)
      (declare (ignore var-1 var-2))
      (when var-0 (setf m var-0))
      (when var-3 (setf aapp var-3))
      (when var-4 (setf aaqq var-4)))
    (cond
     ((> aapp big) (setf info -9)
      (xerbla "ZGEJSV" (f2cl-lib:int-sub info)) (go end_label)))
    (setf aaqq (f2cl-lib:dsqrt aaqq))
    (cond
     ((and (< aapp (f2cl-lib:f2cl/ big aaqq)) noscal)
      (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
              (* aapp aaqq)))
     (t (setf noscal f2cl-lib:%false%)
      (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
              (* aapp (* aaqq scalem)))
      (cond
       (goscal (setf goscal f2cl-lib:%false%)
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal (f2cl-lib:int-sub p 1) scalem sva 1)
          (declare (ignore var-0 var-2 var-3))
          (when var-1 (setf scalem var-1)))))))
   label1874))
(if noscal
    (setf scalem one))
(setf aapp zero)
(setf aaqq big)
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (setf aapp
            (f2cl-lib:dmax1 aapp
                            (f2cl-lib:fref sva-%data% (p) ((1 n))
                                           sva-%offset%)))
    (if (/= (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%) zero)
        (setf aaqq
                (f2cl-lib:dmin1 aaqq
                                (f2cl-lib:fref sva-%data% (p) ((1 n))
```

```
                                                    sva-%offset%))))
    label4781))
  (cond
   ((= aapp zero)
    (if lsvec
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zlaset "G" m n1 czero cone u ldu)
          (declare (ignore var-0 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n1 var-2))
          (when var-3 (setf czero var-3))
          (when var-4 (setf cone var-4))
          (when var-6 (setf ldu var-6))))
    (if rsvec
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zlaset "G" n n czero cone v ldv)
          (declare (ignore var-0 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-3 (setf czero var-3))
          (when var-4 (setf cone var-4))
          (when var-6 (setf ldv var-6))))
    (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%) one)
    (setf (f2cl-lib:fref rwork-%data% (2) ((1 *)) rwork-%offset%) one)
    (if errest
        (setf (f2cl-lib:fref rwork-%data% (3) ((1 *)) rwork-%offset%)
              one))
    (cond
     ((and lsvec rsvec)
      (setf (f2cl-lib:fref rwork-%data% (4) ((1 *)) rwork-%offset%) one)
      (setf (f2cl-lib:fref rwork-%data% (5) ((1 *)) rwork-%offset%)
              one)))
    (cond
     (l2tran
      (setf (f2cl-lib:fref rwork-%data% (6) ((1 *)) rwork-%offset%) zero)
      (setf (f2cl-lib:fref rwork-%data% (7) ((1 *)) rwork-%offset%)
              zero)))
    (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) 0)
    (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%) 0)
    (setf (f2cl-lib:fref iwork-%data% (3) ((1 *)) iwork-%offset%) 0)
    (go end_label)))
  (setf warning 0)
  (cond
   ((<= aaqq sfmin) (setf l2rank f2cl-lib:%true%)
    (setf l2kill f2cl-lib:%true%) (setf warning 1)))
  (cond
   ((= n 1)
    (cond
     (lsvec
      (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (zlascl "G" 0 0
     (f2cl-lib:array-slice sva-%data% double-float (1) ((1 n))
                           sva-%offset%)
     scalem m 1
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda ierr)
  (declare (ignore var-0 var-1 var-2 var-3 var-6 var-7))
  (when var-4 (setf scalem var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "A" m 1 a lda u ldu)
  (declare (ignore var-0 var-2 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(cond
 ((/= n1 n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgeqrf m n u ldu cwork
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ n 1)) ((1 lwork)) cwork-%offset%)
       (f2cl-lib:int-sub lwork n) ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (when var-0 (setf m var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf ldu var-3))
    (when var-7 (setf ierr var-7)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zungqr m n1 1 u ldu cwork
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ n 1)) ((1 lwork)) cwork-%offset%)
       (f2cl-lib:int-sub lwork n) ierr)
    (declare (ignore var-2 var-3 var-5 var-6 var-7))
    (when var-0 (setf m var-0))
    (when var-1 (setf n1 var-1))
    (when var-4 (setf ldu var-4))
    (when var-8 (setf ierr var-8)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (zcopy m
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 1)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice u-%data% f2cl-lib:complex16 (1 1)
                             ((1 ldu) (1 *)) u-%offset%)
```

```
            1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf m var-0)))))))
    (cond
     (rsvec
      (setf (f2cl-lib:fref v-%data% (1 1) ((1 ldv) (1 *)) v-%offset%)
              cone)))
    (cond
     ((< (f2cl-lib:fref sva (1) ((1 n))) (* big scalem))
      (setf (f2cl-lib:fref sva-%data% (1) ((1 n)) sva-%offset%)
              (/ (f2cl-lib:fref sva-%data% (1) ((1 n)) sva-%offset%)
                 scalem))
      (setf scalem one)))
    (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
            (/ one scalem))
    (setf (f2cl-lib:fref rwork-%data% (2) ((1 *)) rwork-%offset%) one)
    (cond
     ((/= (f2cl-lib:fref sva (1) ((1 n))) zero)
      (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) 1)
      (cond
       ((>= (f2cl-lib:f2cl/ (f2cl-lib:fref sva (1) ((1 n))) scalem)
            sfmin)
        (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%) 1))
       (t
        (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%)
                0))))
     (t (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) 0)
      (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%) 0)))
    (setf (f2cl-lib:fref iwork-%data% (3) ((1 *)) iwork-%offset%) 0)
    (if errest
        (setf (f2cl-lib:fref rwork-%data% (3) ((1 *)) rwork-%offset%)
                one))
    (cond
     ((and lsvec rsvec)
      (setf (f2cl-lib:fref rwork-%data% (4) ((1 *)) rwork-%offset%) one)
      (setf (f2cl-lib:fref rwork-%data% (5) ((1 *)) rwork-%offset%)
              one)))
    (cond
     (l2tran
      (setf (f2cl-lib:fref rwork-%data% (6) ((1 *)) rwork-%offset%) zero)
      (setf (f2cl-lib:fref rwork-%data% (7) ((1 *)) rwork-%offset%)
              zero)))
    (go end_label)))
  (setf transp f2cl-lib:%false%)
  (setf l2tran (and l2tran (= m n)))
  (setf aatmax (- one))
  (setf aatmin big)
  (cond
   ((or rowpiv l2tran)
    (cond
```

```
(l2tran
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p m) nil)
   (tagbody
     (setf xsc zero)
     (setf temp1 one)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (zlassq n
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (p 1)
                                ((1 lda) (1 *)) a-%offset%)
          lda xsc temp1)
       (declare (ignore var-1))
       (when var-0 (setf n var-0))
       (when var-2 (setf lda var-2))
       (when var-3 (setf xsc var-3))
       (when var-4 (setf temp1 var-4)))
     (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add m n p))
                          ((1 *)) rwork-%offset%)
             (* xsc scalem))
     (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add n p))
                          ((1 *)) rwork-%offset%)
             (* xsc (* scalem (f2cl-lib:dsqrt temp1))))
     (setf aatmax
             (f2cl-lib:dmax1 aatmax
                             (f2cl-lib:fref rwork-%data%
                                            ((f2cl-lib:int-add n p))
                                            ((1 *))
                                            rwork-%offset%)))
     (if (/=
          (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add n p))
                         ((1 *)) rwork-%offset%)
          zero)
         (setf aatmin
                 (f2cl-lib:dmin1 aatmin
                                 (f2cl-lib:fref rwork-%data%
                                                ((f2cl-lib:int-add n
                                                                   p))
                                                ((1 *))
                                                rwork-%offset%))))
    label1950)))
(t
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p m) nil)
   (tagbody
     (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add m n p))
                          ((1 *)) rwork-%offset%)
             (* scalem
                (abs
                 (f2cl-lib:fref a-%data%
                                (p
```

```
                                        (multiple-value-bind
                                            (ret-val var-0 var-1 var-2)
                                            (idamax n
                                             (f2cl-lib:array-slice a
                                                                   f2cl-lib:complex16
                                                                   (p 1)
                                                                   ((1 lda)
                                                                    (1 *)))
                                             lda)
                                          (declare (ignore var-1))
                                          (when var-0 (setf n var-0))
                                          (when var-2 (setf lda var-2))
                                          ret-val))
                                       ((1 lda) (1 *)) a-%offset%))))
          (setf aatmax
                  (f2cl-lib:dmax1 aatmax
                                  (f2cl-lib:fref rwork-%data%
                                                 ((f2cl-lib:int-add m n
                                                                    p))
                                                 ((1 *))
                                                 rwork-%offset%)))
          (setf aatmin
                  (f2cl-lib:dmin1 aatmin
                                  (f2cl-lib:fref rwork-%data%
                                                 ((f2cl-lib:int-add m n
                                                                    p))
                                                 ((1 *))
                                                 rwork-%offset%)))
        label1904))))))
  (setf entra zero)
  (setf entrat zero)
  (cond
   (l2tran (setf xsc zero) (setf temp1 one)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zlassq n sva 1 xsc temp1)
      (declare (ignore var-1 var-2))
      (when var-0 (setf n var-0))
      (when var-3 (setf xsc var-3))
      (when var-4 (setf temp1 var-4)))
    (setf temp1 (/ one temp1)) (setf entra zero)
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (setf big1
                (*
                 (expt
                  (/ (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                     xsc)
                  2)
                 temp1))
```

```
    (if (/= big1 zero)
        (setf entra (+ entra (* big1 (f2cl-lib:dlog big1)))))
   label1113))
(setf entra (/ (- entra) (f2cl-lib:dlog (f2cl-lib:dfloat n))))
(setf entrat zero)
(f2cl-lib:fdo (p (f2cl-lib:int-add n 1) (f2cl-lib:int-add p 1))
              ((> p (f2cl-lib:int-add n m)) nil)
  (tagbody
    (setf big1
            (*
             (expt
              (/
               (f2cl-lib:fref rwork-%data% (p) ((1 *))
                              rwork-%offset%)
               xsc)
              2)
             temp1))
    (if (/= big1 zero)
        (setf entrat (+ entrat (* big1 (f2cl-lib:dlog big1)))))
   label1114))
(setf entrat (/ (- entrat) (f2cl-lib:dlog (f2cl-lib:dfloat m))))
(setf transp (< entrat entra)) (setf transp f2cl-lib:%true%)
(cond
 (transp
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (setf (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *)) a-%offset%)
              (coerce
               (f2cl-lib:dconjg
                (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                               a-%offset%))
               'f2cl-lib:complex16))
      (f2cl-lib:fdo (q (f2cl-lib:int-add p 1) (f2cl-lib:int-add q 1))
                    ((> q n) nil)
        (tagbody
          (setf ctemp
                  (coerce
                   (f2cl-lib:dconjg
                    (f2cl-lib:fref a-%data% (q p) ((1 lda) (1 *))
                                   a-%offset%))
                   'f2cl-lib:complex16))
          (setf (f2cl-lib:fref a-%data% (q p) ((1 lda) (1 *))
                               a-%offset%)
                  (coerce
                   (f2cl-lib:dconjg
                    (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                   a-%offset%))
                   'f2cl-lib:complex16))
          (setf (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
```

```
                                a-%offset%)
                    ctemp)
            label1116))
        label1115))
    (setf (f2cl-lib:fref a-%data% (n n) ((1 lda) (1 *)) a-%offset%)
            (coerce
             (f2cl-lib:dconjg
              (f2cl-lib:fref a-%data% (n n) ((1 lda) (1 *))
                             a-%offset%))
             'f2cl-lib:complex16))
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add m n p))
                             ((1 *)) rwork-%offset%)
                (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%))
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add n p))
                               ((1 *)) rwork-%offset%))
        label1117))
    (setf temp1 aapp) (setf aapp aatmax) (setf aatmax temp1)
    (setf temp1 aaqq) (setf aaqq aatmin) (setf aatmin temp1)
    (setf kill lsvec) (setf lsvec rsvec) (setf rsvec kill)
    (if lsvec
        (setf n1 n))
    (setf rowpiv f2cl-lib:%true%)))))
(setf big1 (f2cl-lib:dsqrt big))
(setf temp1 (f2cl-lib:dsqrt (/ big (f2cl-lib:dfloat n))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (zlascl "G" 0 0 aapp temp1 n 1 sva n ierr)
  (declare (ignore var-0 var-1 var-2 var-6 var-7))
  (when var-3 (setf aapp var-3))
  (when var-4 (setf temp1 var-4))
  (when var-5 (setf n var-5))
  (when var-8 (setf n var-8))
  (when var-9 (setf ierr var-9)))
(cond ((> aaqq (* aapp sfmin)) (setf aaqq (* (/ aaqq aapp) temp1)))
      (t (setf aaqq (/ (* aaqq temp1) aapp))))
(setf temp1 (* temp1 scalem))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (zlascl "G" 0 0 aapp temp1 m n a lda ierr)
  (declare (ignore var-0 var-1 var-2 var-7))
  (when var-3 (setf aapp var-3))
  (when var-4 (setf temp1 var-4))
  (when var-5 (setf m var-5))
  (when var-6 (setf n var-6))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
```

```
(setf uscal1 temp1)
(setf uscal2 aapp)
(cond (l2kill (setf xsc (f2cl-lib:dsqrt sfmin)))
      (t (setf xsc small)
       (cond
        ((and (< aaqq (f2cl-lib:dsqrt sfmin)) lsvec rsvec)
         (setf jracc f2cl-lib:%true%)))))
(cond
 ((< aaqq xsc)
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (cond
       ((< (f2cl-lib:fref sva (p) ((1 n))) xsc)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zlaset "A" m 1 czero czero
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 p)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-2 var-5))
          (when var-1 (setf m var-1))
          (when var-3 (setf czero var-3))
          (when var-4 (setf czero var-4))
          (when var-6 (setf lda var-6)))
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
              zero)))
     label700))))
(cond
 (rowpiv
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (setf q
              (f2cl-lib:int-sub
               (f2cl-lib:int-add
                (idamax (f2cl-lib:int-add (f2cl-lib:int-sub m p) 1)
                 (f2cl-lib:array-slice rwork-%data% double-float
                                       ((+ m n p)) ((1 *))
                                       rwork-%offset%)
                 1)
                p)
               1))
      (setf (f2cl-lib:fref iwork-%data%
                           ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                              p))
                           ((1 *)) iwork-%offset%)
            q)
      (cond
       ((/= p q)
```

```
        (setf temp1
                (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add m n p))
                               ((1 *)) rwork-%offset%))
        (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add m n p))
                             ((1 *)) rwork-%offset%)
                (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add m n q))
                               ((1 *)) rwork-%offset%))
        (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add m n q))
                             ((1 *)) rwork-%offset%)
                temp1)))
     label1952))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaswp n a lda 1 (f2cl-lib:int-sub m 1)
       (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                             ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                             iwork-%offset%)
       1)
    (declare (ignore var-1 var-3 var-4 var-5 var-6))
    (when var-0 (setf n var-0))
    (when var-2 (setf lda var-2)))))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (setf (f2cl-lib:fref iwork-%data% (p) ((1 *)) iwork-%offset%) 0)
   label1963))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (zgeqp3 m n a lda iwork cwork
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16 ((+ n 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork n) rwork ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-9 (setf ierr var-9)))
(setf nr 1)
(cond
 (l2aber
  (tagbody
    (setf temp1 (* (f2cl-lib:dsqrt (f2cl-lib:dfloat n)) epsln))
    (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (cond
         ((>= (abs (f2cl-lib:fref a (p p) ((1 lda) (1 *))))
              (* temp1 (abs (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))))
          (setf nr (f2cl-lib:int-add nr 1)))
         (t (go label3002)))
       label3001))
```

```
  label3002))
(l2rank
 (tagbody
   (setf temp1 (f2cl-lib:dsqrt sfmin))
   (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                 ((> p n) nil)
     (tagbody
       (if (or
            (<
             (abs
              (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                             a-%offset%))
             (* epsln
                (abs
                 (f2cl-lib:fref a-%data%
                                ((f2cl-lib:int-sub p 1)
                                 (f2cl-lib:int-sub p 1))
                                ((1 lda) (1 *)) a-%offset%))))
            (<
             (abs
              (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                             a-%offset%))
             small)
            (and l2kill
                 (<
                  (abs
                   (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                                  a-%offset%))
                  temp1)))
           (go label3402))
       (setf nr (f2cl-lib:int-add nr 1))
      label3401))
  label3402))
(t
 (tagbody
   (setf temp1 (f2cl-lib:dsqrt sfmin))
   (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                 ((> p n) nil)
     (tagbody
       (if (or
            (<
             (abs
              (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                             a-%offset%))
             small)
            (and l2kill
                 (<
                  (abs
                   (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                                  a-%offset%))
```

```
                    temp1)))
            (go label3302))
        (setf nr (f2cl-lib:int-add nr 1))
       label3301))
   label3302)))
(setf almort f2cl-lib:%false%)
(cond
 ((= nr n) (setf maxprj one)
  (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (setf temp1
              (/
               (abs
                (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                               a-%offset%))
               (f2cl-lib:fref sva-%data%
                              ((f2cl-lib:fref iwork (p) ((1 *))))
                              ((1 n)) sva-%offset%)))
      (setf maxprj (f2cl-lib:dmin1 maxprj temp1))
     label3051))
  (if (>= (expt maxprj 2) (- one (* (f2cl-lib:dfloat n) epsln)))
      (setf almort f2cl-lib:%true%))))
(setf sconda (- one))
(setf condr1 (- one))
(setf condr2 (- one))
(cond
 (errest
  (cond
   ((= n nr)
    (cond
     (rsvec
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlacpy "U" n n a lda v ldv)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldv var-6)))
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p n) nil)
        (tagbody
          (setf temp1
                  (f2cl-lib:fref sva-%data%
                                 ((f2cl-lib:fref iwork (p) ((1 *))))
                                 ((1 n)) sva-%offset%))
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (zdscal p (/ one temp1)
               (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (1 p)
                                     ((1 ldv) (1 *)) v-%offset%)
```

```
          1)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf p var-0)))
    label3053))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (zpocon "U" n v ldv one temp1
      (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                            ((+ n 1)) ((1 lwork)) cwork-%offset%)
      rwork ierr)
   (declare (ignore var-0 var-2 var-6 var-7))
   (when var-1 (setf n var-1))
   (when var-3 (setf ldv var-3))
   (when var-4 (setf one var-4))
   (when var-5 (setf temp1 var-5))
   (when var-8 (setf ierr var-8))))
(lsvec
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (zlacpy "U" n n a lda u ldu)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldu var-6)))
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p n) nil)
   (tagbody
     (setf temp1
             (f2cl-lib:fref sva-%data%
                            ((f2cl-lib:fref iwork (p) ((1 *))))
                            ((1 n)) sva-%offset%))
     (multiple-value-bind (var-0 var-1 var-2 var-3)
         (zdscal p (/ one temp1)
          (f2cl-lib:array-slice u-%data% f2cl-lib:complex16 (1 p)
                                ((1 ldu) (1 *)) u-%offset%)
          1)
       (declare (ignore var-1 var-2 var-3))
       (when var-0 (setf p var-0)))
    label3054))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (zpocon "U" n u ldu one temp1
      (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                            ((+ n 1)) ((1 lwork)) cwork-%offset%)
      rwork ierr)
   (declare (ignore var-0 var-2 var-6 var-7))
   (when var-1 (setf n var-1))
   (when var-3 (setf ldu var-3))
   (when var-4 (setf one var-4))
   (when var-5 (setf temp1 var-5))
```

```
    (when var-8 (setf ierr var-8))))
  (t
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (zlacpy "U" n n a lda
        (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                              ((+ n 1)) ((1 lwork)) cwork-%offset%)
        n)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf n var-1))
     (when var-2 (setf n var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf n var-6)))
   (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                 ((> p n) nil)
     (tagbody
       (setf temp1
               (f2cl-lib:fref sva-%data%
                              ((f2cl-lib:fref iwork (p) ((1 *))))
                              ((1 n)) sva-%offset%))
       (multiple-value-bind (var-0 var-1 var-2 var-3)
           (zdscal p (/ one temp1)
            (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                                  ((+ n
                                      (f2cl-lib:int-mul
                                       (f2cl-lib:int-add p
                                                         (f2cl-lib:int-sub
                                                          1))
                                       n)
                                      1))
                                  ((1 lwork)) cwork-%offset%)
            1)
         (declare (ignore var-1 var-2 var-3))
         (when var-0 (setf p var-0)))
     label3052))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (zpocon "U" n
        (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                              ((+ n 1)) ((1 lwork)) cwork-%offset%)
        n one temp1
        (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                              ((+ n (f2cl-lib:int-mul n n) 1))
                              ((1 lwork)) cwork-%offset%)
        rwork ierr)
     (declare (ignore var-0 var-2 var-6 var-7))
     (when var-1 (setf n var-1))
     (when var-3 (setf n var-3))
     (when var-4 (setf one var-4))
     (when var-5 (setf temp1 var-5))
     (when var-8 (setf ierr var-8)))))
```

```
    (setf sconda (/ one (f2cl-lib:dsqrt temp1))))
   (t (setf sconda (- one))))))
(setf l2pert
        (and l2pert
             (>
              (abs
               (/
                (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *))
                               a-%offset%)
                (f2cl-lib:fref a-%data% (nr nr) ((1 lda) (1 *))
                               a-%offset%)))
              (f2cl-lib:dsqrt big1))))
(cond
 ((not (or rsvec lsvec))
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p
                    (f2cl-lib:min0
                     (f2cl-lib:int-add n (f2cl-lib:int-sub 1)) nr))
                 nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (zcopy (f2cl-lib:int-sub n p)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                 (p (f2cl-lib:int-add p 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                 ((+ p 1) p) ((1 lda) (1 *))
                                 a-%offset%)
           1)
        (declare (ignore var-0 var-1 var-3 var-4))
        (when var-2 (setf lda var-2)))
      (zlacgv (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (p p)
                             ((1 lda) (1 *)) a-%offset%)
       1)
     label1946))
  (if (= nr n)
      (setf (f2cl-lib:fref a-%data% (n n) ((1 lda) (1 *)) a-%offset%)
              (coerce
               (f2cl-lib:dconjg
                (f2cl-lib:fref a-%data% (n n) ((1 lda) (1 *))
                               a-%offset%))
               'f2cl-lib:complex16)))
  (cond
   ((not almort)
    (cond
     (l2pert (setf xsc (/ epsln (f2cl-lib:dfloat n)))
      (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                    ((> q nr) nil)
```

```
        (tagbody
          (setf ctemp
                  (f2cl-lib:dcmplx
                   (* xsc
                      (abs
                       (f2cl-lib:fref a-%data% (q q) ((1 lda) (1 *))
                                      a-%offset%)))
                   zero))
          (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                        ((> p n) nil)
            (tagbody
              (if (or
                   (and (> p q)
                        (<=
                         (abs
                          (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                         a-%offset%))
                         temp1))
                   (< p q))
                  (setf (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                       a-%offset%)
                        ctemp))
              label4949))
          label4947)))
      (t
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
           (zlaset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
            czero czero
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 2)
                                  ((1 lda) (1 *)) a-%offset%)
            lda)
         (declare (ignore var-0 var-1 var-2 var-5))
         (when var-3 (setf czero var-3))
         (when var-4 (setf czero var-4))
         (when var-6 (setf lda var-6)))))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (zgeqrf n nr a lda cwork
          (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                                ((+ n 1)) ((1 lwork)) cwork-%offset%)
          (f2cl-lib:int-sub lwork n) ierr)
       (declare (ignore var-2 var-4 var-5 var-6))
       (when var-0 (setf n var-0))
       (when var-1 (setf nr var-1))
       (when var-3 (setf lda var-3))
       (when var-7 (setf ierr var-7)))
     (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                   ((> p (f2cl-lib:int-add nr (f2cl-lib:int-sub 1))) nil)
       (tagbody
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
            (zcopy (f2cl-lib:int-sub nr p)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   (p (f2cl-lib:int-add p 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   ((+ p 1) p) ((1 lda) (1 *))
                                   a-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf lda var-2)))
        (zlacgv (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (p p)
                               ((1 lda) (1 *)) a-%offset%)
         1)
       label1948))))
  (cond
   (l2pert (setf xsc (/ epsln (f2cl-lib:dfloat n)))
    (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                  ((> q nr) nil)
      (tagbody
        (setf ctemp
                (f2cl-lib:dcmplx
                 (* xsc
                    (abs
                     (f2cl-lib:fref a-%data% (q q) ((1 lda) (1 *))
                                    a-%offset%)))
                 zero))
        (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                      ((> p nr) nil)
          (tagbody
            (if (or
                 (and (> p q)
                      (<=
                       (abs
                        (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                       a-%offset%))
                       temp1))
                 (< p q))
                (setf (f2cl-lib:fref a-%data% (p q) ((1 lda) (1 *))
                                     a-%offset%)
                      ctemp))
           label1949))
       label1947)))
   (t
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlaset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
         czero czero
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 2)
                               ((1 lda) (1 *)) a-%offset%)
```

```
          lda)
        (declare (ignore var-0 var-1 var-2 var-5))
        (when var-3 (setf czero var-3))
        (when var-4 (setf czero var-4))
        (when var-6 (setf lda var-6)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14 var-15)
        (zgesvj "L" "NoU" "NoV" nr nr a lda sva n v ldv cwork lwork rwork
         lrwork info)
      (declare
       (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-13))
      (when var-3 (setf nr var-3))
      (when var-4 (setf nr var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf n var-8))
      (when var-10 (setf ldv var-10))
      (when var-12 (setf lwork var-12))
      (when var-14 (setf lrwork var-14))
      (when var-15 (setf info var-15)))
    (setf scalem (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%))
    (setf numrank
            (f2cl-lib:nint
             (f2cl-lib:fref rwork-%data% (2) ((1 *)) rwork-%offset%))))
   ((and rsvec (not lsvec))
    (cond
     (almort
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p nr) nil)
        (tagbody
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (zcopy (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (p p)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (p p)
                                     ((1 ldv) (1 *)) v-%offset%)
               1)
            (declare (ignore var-0 var-1 var-3 var-4))
            (when var-2 (setf lda var-2)))
          (zlacgv (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (p p)
                                 ((1 ldv) (1 *)) v-%offset%)
           1)
          label1998))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlaset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
           czero czero
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (1 2)
                                 ((1 ldv) (1 *)) v-%offset%)
```

```
      ldv)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (zgesvj "L" "U" "N" n nr v ldv sva nr a lda cwork lwork rwork
       lrwork info)
    (declare
     (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-13))
    (when var-3 (setf n var-3))
    (when var-4 (setf nr var-4))
    (when var-6 (setf ldv var-6))
    (when var-8 (setf nr var-8))
    (when var-10 (setf lda var-10))
    (when var-12 (setf lwork var-12))
    (when var-14 (setf lrwork var-14))
    (when var-15 (setf info var-15)))
  (setf scalem
          (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%))
  (setf numrank
          (f2cl-lib:nint
           (f2cl-lib:fref rwork-%data% (2) ((1 *)) rwork-%offset%))))
(t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "Lower" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
       czero czero
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (2 1)
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf lda var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgelqf nr n a lda cwork
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ n 1)) ((1 lwork)) cwork-%offset%)
       (f2cl-lib:int-sub lwork n) ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (when var-0 (setf nr var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf lda var-3))
    (when var-7 (setf ierr var-7)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "Lower" nr nr a lda v ldv)
    (declare (ignore var-0 var-3 var-5))
```

```
  (when var-1 (setf nr var-1))
  (when var-2 (setf nr var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldv var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
     czero czero
     (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (1 2)
                           ((1 ldv) (1 *)) v-%offset%)
     ldv)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldv var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgeqrf nr nr v ldv
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ n 1)) ((1 lwork)) cwork-%offset%)
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (when var-0 (setf nr var-0))
  (when var-1 (setf nr var-1))
  (when var-3 (setf ldv var-3))
  (when var-7 (setf ierr var-7)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zcopy (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (p p)
                               ((1 ldv) (1 *)) v-%offset%)
         ldv
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (p p)
                               ((1 ldv) (1 *)) v-%offset%)
         1)
      (declare (ignore var-0 var-1 var-3 var-4))
      (when var-2 (setf ldv var-2)))
    (zlacgv (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
     (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (p p)
                           ((1 ldv) (1 *)) v-%offset%)
     1)
   label8998))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
     zero zero
     (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (1 2)
```

```
                              ((1 ldv) (1 *)) v-%offset%)
     ldv)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldv var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15)
    (zgesvj "Lower" "U" "N" nr nr v ldv sva nr u ldu
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                              ((+ n 1)) ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork n) rwork lrwork info)
  (declare
   (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12
    var-13))
  (when var-3 (setf nr var-3))
  (when var-4 (setf nr var-4))
  (when var-6 (setf ldv var-6))
  (when var-8 (setf nr var-8))
  (when var-10 (setf ldu var-10))
  (when var-14 (setf lrwork var-14))
  (when var-15 (setf info var-15)))
(setf scalem
        (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%))
(setf numrank
        (f2cl-lib:nint
         (f2cl-lib:fref rwork-%data% (2) ((1 *)) rwork-%offset%)))
(cond
 ((< nr n)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "A" (f2cl-lib:int-sub n nr) nr czero czero
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                              ((+ nr 1) 1) ((1 ldv) (1 *))
                              v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "A" nr (f2cl-lib:int-sub n nr) czero czero
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                              (1 (f2cl-lib:int-add nr 1))
                              ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-2 var-5))
    (when var-1 (setf nr var-1))
    (when var-3 (setf czero var-3))
```

```
        (when var-4 (setf czero var-4))
        (when var-6 (setf ldv var-6)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlaset "A" (f2cl-lib:int-sub n nr) (f2cl-lib:int-sub n nr)
           czero cone
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                                 ((+ nr 1) (f2cl-lib:int-add nr 1))
                                 ((1 ldv) (1 *)) v-%offset%)
           ldv)
        (declare (ignore var-0 var-1 var-2 var-5))
        (when var-3 (setf czero var-3))
        (when var-4 (setf cone var-4))
        (when var-6 (setf ldv var-6)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (zunmlq "Left" "C" n n nr a lda cwork v ldv
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                               ((+ n 1)) ((1 lwork)) cwork-%offset%)
         (f2cl-lib:int-sub lwork n) ierr)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf n var-2))
      (when var-3 (setf n var-3))
      (when var-4 (setf nr var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldv var-9))
      (when var-12 (setf ierr var-12)))))
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (zcopy n
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (p 1)
                                 ((1 ldv) (1 *)) v-%offset%)
           ldv
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                 ((f2cl-lib:fref iwork (p) ((1 *))) 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-1 var-3))
        (when var-0 (setf n var-0))
        (when var-2 (setf ldv var-2))
        (when var-4 (setf lda var-4)))
     label8991))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "All" n n a lda v ldv)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
```

```
    (when var-6 (setf ldv var-6)))
  (cond
   (transp
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlacpy "All" n n v ldv u ldu)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf ldv var-4))
      (when var-6 (setf ldu var-6))))))
 ((and lsvec (not rsvec))
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p nr) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (zcopy (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (p p)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice u-%data% f2cl-lib:complex16 (p p)
                                 ((1 ldu) (1 *)) u-%offset%)
           1)
        (declare (ignore var-0 var-1 var-3 var-4))
        (when var-2 (setf lda var-2)))
      (zlacgv (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
       (f2cl-lib:array-slice u-%data% f2cl-lib:complex16 (p p)
                             ((1 ldu) (1 *)) u-%offset%)
       1)
    label1965))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
       czero czero
       (f2cl-lib:array-slice u-%data% f2cl-lib:complex16 (1 2)
                             ((1 ldu) (1 *)) u-%offset%)
       ldu)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgeqrf n nr u ldu
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16 ((+ n 1))
                             ((1 lwork)) cwork-%offset%)
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 lwork))
                             cwork-%offset%)
       (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (when var-0 (setf n var-0))
```

```
  (when var-1 (setf nr var-1))
  (when var-3 (setf ldu var-3))
  (when var-7 (setf ierr var-7)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p (f2cl-lib:int-add nr (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zcopy (f2cl-lib:int-sub nr p)
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                               (p (f2cl-lib:int-add p 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                               ((+ p 1) p) ((1 ldu) (1 *))
                               u-%offset%)
         1)
      (declare (ignore var-0 var-1 var-3 var-4))
      (when var-2 (setf ldu var-2)))
    (zlacgv (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
     (f2cl-lib:array-slice u-%data% f2cl-lib:complex16 (p p)
                           ((1 ldu) (1 *)) u-%offset%)
     1)
   label1967))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "Upper" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
     czero czero
     (f2cl-lib:array-slice u-%data% f2cl-lib:complex16 (1 2)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15)
    (zgesvj "Lower" "U" "N" nr nr u ldu sva nr a lda
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16 ((+ n 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork n) rwork lrwork info)
  (declare
   (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12 var-13))
  (when var-3 (setf nr var-3))
  (when var-4 (setf nr var-4))
  (when var-6 (setf ldu var-6))
  (when var-8 (setf nr var-8))
  (when var-10 (setf lda var-10))
  (when var-14 (setf lrwork var-14))
  (when var-15 (setf info var-15)))
(setf scalem (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%))
```

```
(setf numrank
        (f2cl-lib:nint
         (f2cl-lib:fref rwork-%data% (2) ((1 *)) rwork-%offset%)))
(cond
 ((< nr m)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "A" (f2cl-lib:int-sub m nr) nr czero czero
       (f2cl-lib:array-slice u-%data% f2cl-lib:complex16 ((+ nr 1) 1)
                             ((1 ldu) (1 *)) u-%offset%)
       ldu)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldu var-6)))
  (cond
   ((< nr n1)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlaset "A" nr (f2cl-lib:int-sub n1 nr) czero czero
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                               (1 (f2cl-lib:int-add nr 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-2 var-5))
      (when var-1 (setf nr var-1))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldu var-6)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlaset "A" (f2cl-lib:int-sub m nr) (f2cl-lib:int-sub n1 nr)
         czero cone
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                               ((+ nr 1) (f2cl-lib:int-add nr 1))
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf czero var-3))
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldu var-6)))))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (zunmqr "Left" "No Tr" m n1 n a lda cwork u ldu
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16 ((+ n 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n1 var-3))
  (when var-4 (setf n var-4))
```

```
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldu var-9))
    (when var-12 (setf ierr var-12)))
  (if rowpiv
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlaswp n1 u ldu 1 (f2cl-lib:int-sub m 1)
           (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                                 ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                                 iwork-%offset%)
           -1)
        (declare (ignore var-1 var-3 var-4 var-5 var-6))
        (when var-0 (setf n1 var-0))
        (when var-2 (setf ldu var-2))))
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n1) nil)
    (tagbody
      (setf xsc
              (/ one
                 (multiple-value-bind (ret-val var-0 var-1 var-2)
                     (dznrm2 m
                      (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                                            (1 p) ((1 ldu) (1 *))
                                            u-%offset%)
                      1)
                   (declare (ignore var-1 var-2))
                   (when var-0 (setf m var-0))
                   ret-val)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (zdscal m xsc
           (f2cl-lib:array-slice u-%data% f2cl-lib:complex16 (1 p)
                                 ((1 ldu) (1 *)) u-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf xsc var-1)))
     label1974))
  (cond
   (transp
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlacpy "All" n n u ldu v ldv)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf ldu var-4))
      (when var-6 (setf ldv var-6))))))
 (t
  (cond
   ((not jracc)
    (cond
     ((not almort)
```

```
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zcopy (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (p p)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (p p)
                               ((1 ldv) (1 *)) v-%offset%)
         1)
      (declare (ignore var-0 var-1 var-3 var-4))
      (when var-2 (setf lda var-2)))
    (zlacgv (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
     (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (p p)
                           ((1 ldv) (1 *)) v-%offset%)
     1)
   label1968))
(cond
 (l2pert (setf xsc (f2cl-lib:dsqrt small))
  (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                ((> q nr) nil)
    (tagbody
      (setf ctemp
              (f2cl-lib:dcmplx
               (* xsc
                  (abs
                   (f2cl-lib:fref v-%data% (q q) ((1 ldv) (1 *))
                                  v-%offset%)))
               zero))
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p n) nil)
        (tagbody
          (if (or
               (and (> p q)
                    (<=
                     (abs
                      (f2cl-lib:fref v-%data% (p q)
                                     ((1 ldv) (1 *)) v-%offset%))
                     temp1))
               (< p q))
              (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                   v-%offset%)
                    ctemp))
          (if (< p q)
              (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                   v-%offset%)
                    (-
                     (f2cl-lib:fref v-%data% (p q)
                                    ((1 ldv) (1 *))
```

```
                                        v-%offset%))))
          label2968))
      label2969)))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
       czero czero
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (1 2)
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldv var-6)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" nr nr v ldv
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) cwork-%offset%)
     nr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf nr var-1))
  (when var-2 (setf nr var-2))
  (when var-4 (setf ldv var-4))
  (when var-6 (setf nr var-6)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (setf temp1
            (dznrm2 (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
             (f2cl-lib:array-slice cwork-%data%
                                   f2cl-lib:complex16
                                   ((+ (f2cl-lib:int-mul 2 n)
                                       (f2cl-lib:int-mul
                                        (f2cl-lib:int-add p
                                                          (f2cl-lib:int-sub
                                                           1))
                                        nr)
                                       p))
                                   ((1 lwork)) cwork-%offset%)
             1))
    (zdscal (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
     (/ one temp1)
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul
                                (f2cl-lib:int-add p
                                                  (f2cl-lib:int-sub
                                                   1))
```

```
                                 nr)
                                p))
                            ((1 lwork)) cwork-%offset%)
      1)
    label3950))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zpocon "Lower" nr
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) cwork-%offset%)
     nr one temp1
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul nr nr) 1))
                           ((1 lwork)) cwork-%offset%)
     rwork ierr)
  (declare (ignore var-0 var-2 var-6 var-7))
  (when var-1 (setf nr var-1))
  (when var-3 (setf nr var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf temp1 var-5))
  (when var-8 (setf ierr var-8)))
(setf condr1 (/ one (f2cl-lib:dsqrt temp1)))
(setf cond_ok
        (f2cl-lib:dsqrt (f2cl-lib:dsqrt (f2cl-lib:dfloat nr))))
(cond
 ((< condr1 cond_ok)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgeqrf n nr v ldv
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ n 1)) ((1 lwork))
                             cwork-%offset%)
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ (f2cl-lib:int-mul 2 n) 1))
                             ((1 lwork)) cwork-%offset%)
       (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (when var-0 (setf n var-0))
    (when var-1 (setf nr var-1))
    (when var-3 (setf ldv var-3))
    (when var-7 (setf ierr var-7)))
  (cond
   (l2pert (setf xsc (/ (f2cl-lib:dsqrt small) epsln))
    (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                  ((> p nr) nil)
      (tagbody
        (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                      ((> q
```

```
                              (f2cl-lib:int-add p
                                                (f2cl-lib:int-sub
                                                 1)))
                          nil)
            (tagbody
              (setf ctemp
                      (f2cl-lib:dcmplx
                       (* xsc
                          (f2cl-lib:dmin1
                           (abs
                            (f2cl-lib:fref v-%data% (p p)
                                           ((1 ldv) (1 *))
                                           v-%offset%))
                           (abs
                            (f2cl-lib:fref v-%data% (q q)
                                           ((1 ldv) (1 *))
                                           v-%offset%))))
                       zero))
              (if (<=
                   (abs
                    (f2cl-lib:fref v-%data% (q p) ((1 ldv) (1 *))
                                   v-%offset%))
                   temp1)
                  (setf (f2cl-lib:fref v-%data% (q p)
                                       ((1 ldv) (1 *)) v-%offset%)
                        ctemp))
             label3958))
         label3959))))
    (if (/= nr n)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zlacpy "A" n nr v ldv
             (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                                   ((+ (f2cl-lib:int-mul 2 n) 1))
                                   ((1 lwork)) cwork-%offset%)
             n)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf nr var-2))
          (when var-4 (setf ldv var-4))
          (when var-6 (setf n var-6))))
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p (f2cl-lib:int-add nr (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (zcopy (f2cl-lib:int-sub nr p)
             (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                                   (p (f2cl-lib:int-add p 1))
                                   ((1 ldv) (1 *)) v-%offset%)
```

```
          ldv
          (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                                ((+ p 1) p) ((1 ldv) (1 *))
                                v-%offset%)
          1)
       (declare (ignore var-0 var-1 var-3 var-4))
       (when var-2 (setf ldv var-2)))
     (zlacgv (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
      (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (p p)
                            ((1 ldv) (1 *)) v-%offset%)
      1)
    label1969))
 (setf (f2cl-lib:fref v-%data% (nr nr) ((1 ldv) (1 *))
                      v-%offset%)
        (coerce
         (f2cl-lib:dconjg
          (f2cl-lib:fref v-%data% (nr nr) ((1 ldv) (1 *))
                         v-%offset%))
         'f2cl-lib:complex16))
 (setf condr2 condr1))
(t
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p nr) nil)
   (tagbody
     (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add n p))
                          ((1 *)) iwork-%offset%)
           0)
    label3003))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9)
     (zgeqp3 n nr v ldv
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                            ((+ n 1)) ((1 *)) iwork-%offset%)
      (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                            ((+ n 1)) ((1 lwork))
                            cwork-%offset%)
      (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                            ((+ (f2cl-lib:int-mul 2 n) 1))
                            ((1 lwork)) cwork-%offset%)
      (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) rwork
      ierr)
   (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8))
   (when var-0 (setf n var-0))
   (when var-1 (setf nr var-1))
   (when var-3 (setf ldv var-3))
   (when var-9 (setf ierr var-9)))
 (cond
  (l2pert (setf xsc (f2cl-lib:dsqrt small))
   (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
```

```
                ((> p nr) nil)
      (tagbody
        (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                      ((> q
                          (f2cl-lib:int-add p
                                            (f2cl-lib:int-sub
                                             1)))
                       nil)
          (tagbody
            (setf ctemp
                    (f2cl-lib:dcmplx
                     (* xsc
                        (f2cl-lib:dmin1
                         (abs
                          (f2cl-lib:fref v-%data% (p p)
                                         ((1 ldv) (1 *))
                                         v-%offset%))
                         (abs
                          (f2cl-lib:fref v-%data% (q q)
                                         ((1 ldv) (1 *))
                                         v-%offset%))))
                     zero))
            (if (<=
                 (abs
                  (f2cl-lib:fref v-%data% (q p) ((1 ldv) (1 *))
                                 v-%offset%))
                 temp1)
                (setf (f2cl-lib:fref v-%data% (q p)
                                     ((1 ldv) (1 *)) v-%offset%)
                      ctemp))
          label3968))
      label3969))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "A" n nr v ldv
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) cwork-%offset%)
     n)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nr var-2))
  (when var-4 (setf ldv var-4))
  (when var-6 (setf n var-6)))
(cond
 (l2pert (setf xsc (f2cl-lib:dsqrt small))
  (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                ((> p nr) nil)
    (tagbody
      (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
```

```
                        ((> q
                            (f2cl-lib:int-add p
                                              (f2cl-lib:int-sub
                                               1)))
                         nil)
                (tagbody
                  (setf ctemp
                          (f2cl-lib:dcmplx
                           (* xsc
                              (f2cl-lib:dmin1
                               (abs
                                (f2cl-lib:fref v-%data% (p p)
                                               ((1 ldv) (1 *))
                                               v-%offset%))
                               (abs
                                (f2cl-lib:fref v-%data% (q q)
                                               ((1 ldv) (1 *))
                                               v-%offset%))))
                           zero))
                  (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                       v-%offset%)
                          (- ctemp))
                 label8971))
            label8970)))
        (t
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
             (zlaset "L" (f2cl-lib:int-sub nr 1)
              (f2cl-lib:int-sub nr 1) czero czero
              (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (2 1)
                                    ((1 ldv) (1 *)) v-%offset%)
              ldv)
           (declare (ignore var-0 var-1 var-2 var-5))
           (when var-3 (setf czero var-3))
           (when var-4 (setf czero var-4))
           (when var-6 (setf ldv var-6)))))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
           (zgelqf nr nr v ldv
            (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                                  ((+ (f2cl-lib:int-mul 2 n)
                                      (f2cl-lib:int-mul n nr) 1))
                                  ((1 lwork)) cwork-%offset%)
            (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                                  ((+ (f2cl-lib:int-mul 2 n)
                                      (f2cl-lib:int-mul n nr) nr 1))
                                  ((1 lwork)) cwork-%offset%)
            (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                              (f2cl-lib:int-mul n nr) nr)
            ierr)
```

```
  (declare (ignore var-2 var-4 var-5 var-6))
  (when var-0 (setf nr var-0))
  (when var-1 (setf nr var-1))
  (when var-3 (setf ldv var-3))
  (when var-7 (setf ierr var-7)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" nr nr v ldv
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) nr 1))
                           ((1 lwork)) cwork-%offset%)
     nr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf nr var-1))
  (when var-2 (setf nr var-2))
  (when var-4 (setf ldv var-4))
  (when var-6 (setf nr var-6)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (setf temp1
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (dznrm2 p
                 (f2cl-lib:array-slice cwork-%data%
                                       f2cl-lib:complex16
                                       ((+
                                         (f2cl-lib:int-mul 2 n)
                                         (f2cl-lib:int-mul n
                                                           nr)
                                         nr p))
                                       ((1 lwork))
                                       cwork-%offset%)
                 nr)
              (declare (ignore var-1))
              (when var-0 (setf p var-0))
              (when var-2 (setf nr var-2))
              ret-val))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zdscal p (/ one temp1)
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                               ((+ (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n nr) nr
                                   p))
                               ((1 lwork)) cwork-%offset%)
         nr)
      (declare (ignore var-1 var-2))
      (when var-0 (setf p var-0))
      (when var-3 (setf nr var-3)))
   label4950))
```

```
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zpocon "L" nr
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ (f2cl-lib:int-mul 2 n)
                                 (f2cl-lib:int-mul n nr) nr 1))
                             ((1 lwork)) cwork-%offset%)
       nr one temp1
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ (f2cl-lib:int-mul 2 n)
                                 (f2cl-lib:int-mul n nr) nr
                                 (f2cl-lib:int-mul nr nr) 1))
                             ((1 lwork)) cwork-%offset%)
       rwork ierr)
    (declare (ignore var-0 var-2 var-6 var-7))
    (when var-1 (setf nr var-1))
    (when var-3 (setf nr var-3))
    (when var-4 (setf one var-4))
    (when var-5 (setf temp1 var-5))
    (when var-8 (setf ierr var-8)))
  (setf condr2 (/ one (f2cl-lib:dsqrt temp1)))
  (cond
   ((>= condr2 cond_ok)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlacpy "U" nr nr v ldv
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                               ((+ (f2cl-lib:int-mul 2 n) 1))
                               ((1 lwork)) cwork-%offset%)
         n)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf nr var-1))
      (when var-2 (setf nr var-2))
      (when var-4 (setf ldv var-4))
      (when var-6 (setf n var-6)))))))
(cond
 (l2pert (setf xsc (f2cl-lib:dsqrt small))
  (f2cl-lib:fdo (q 2 (f2cl-lib:int-add q 1))
                ((> q nr) nil)
    (tagbody
      (setf ctemp
              (* xsc
                 (f2cl-lib:fref v-%data% (q q) ((1 ldv) (1 *))
                                v-%offset%)))
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p
                        (f2cl-lib:int-add q
                                          (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
```

```
            (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                 v-%offset%)
                  (- ctemp))
          label4969))
      label4968)))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (zlaset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
        czero czero
        (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (1 2)
                              ((1 ldv) (1 *)) v-%offset%)
        ldv)
     (declare (ignore var-0 var-1 var-2 var-5))
     (when var-3 (setf czero var-3))
     (when var-4 (setf czero var-4))
     (when var-6 (setf ldv var-6)))))
 (cond
  ((< condr1 cond_ok)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10 var-11 var-12 var-13 var-14 var-15)
       (zgesvj "L" "U" "N" nr nr v ldv sva nr u ldu
        (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                              ((+ (f2cl-lib:int-mul 2 n)
                                  (f2cl-lib:int-mul n nr) nr 1))
                              ((1 lwork)) cwork-%offset%)
        (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                          (f2cl-lib:int-mul n nr) nr)
        rwork lrwork info)
     (declare
      (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12
       var-13))
     (when var-3 (setf nr var-3))
     (when var-4 (setf nr var-4))
     (when var-6 (setf ldv var-6))
     (when var-8 (setf nr var-8))
     (when var-10 (setf ldu var-10))
     (when var-14 (setf lrwork var-14))
     (when var-15 (setf info var-15)))
   (setf scalem
           (f2cl-lib:fref rwork-%data% (1) ((1 *))
                          rwork-%offset%))
   (setf numrank
           (f2cl-lib:nint
            (f2cl-lib:fref rwork-%data% (2) ((1 *))
                           rwork-%offset%)))
   (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                 ((> p nr) nil)
     (tagbody
```

```
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (zcopy nr
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                                 (1 p) ((1 ldv) (1 *))
                                 v-%offset%)
           1
           (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                                 (1 p) ((1 ldu) (1 *))
                                 u-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf nr var-0)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (zdscal nr
           (f2cl-lib:array-slice sva-%data% double-float (p)
                                 ((1 n)) sva-%offset%)
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                                 (1 p) ((1 ldv) (1 *))
                                 v-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf nr var-0)))
    label3970))
  (cond
   ((= nr n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10)
        (ztrsm "L" "U" "N" "N" nr nr cone a lda v ldv)
      (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
      (when var-4 (setf nr var-4))
      (when var-5 (setf nr var-5))
      (when var-6 (setf cone var-6))
      (when var-8 (setf lda var-8))
      (when var-10 (setf ldv var-10))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10)
        (ztrsm "L" "U" "C" "N" nr nr cone
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                               ((+ (f2cl-lib:int-mul 2 n) 1))
                               ((1 lwork)) cwork-%offset%)
         n v ldv)
      (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
      (when var-4 (setf nr var-4))
      (when var-5 (setf nr var-5))
      (when var-6 (setf cone var-6))
      (when var-8 (setf n var-8))
      (when var-10 (setf ldv var-10)))
```

```
(cond
 ((< nr n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "A" (f2cl-lib:int-sub n nr) nr zero czero
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             ((+ nr 1) 1) ((1 ldv) (1 *))
                             v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "A" nr (f2cl-lib:int-sub n nr) zero czero
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             (1 (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-2 var-5))
    (when var-1 (setf nr var-1))
    (when var-3 (setf zero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "A" (f2cl-lib:int-sub n nr)
       (f2cl-lib:int-sub n nr) zero cone
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             ((+ nr 1)
                              (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf zero var-3))
    (when var-4 (setf cone var-4))
    (when var-6 (setf ldv var-6)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (zunmqr "L" "N" n n nr
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) cwork-%offset%)
     n
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ n 1)) ((1 lwork))
                           cwork-%offset%)
```

```
        v ldv
        (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                              ((+ (f2cl-lib:int-mul 2 n)
                                  (f2cl-lib:int-mul n nr) nr 1))
                              ((1 lwork)) cwork-%offset%)
        (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                          (f2cl-lib:int-mul n nr) nr)
        ierr)
      (declare
       (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf n var-2))
      (when var-3 (setf n var-3))
      (when var-4 (setf nr var-4))
      (when var-6 (setf n var-6))
      (when var-9 (setf ldv var-9))
      (when var-12 (setf ierr var-12))))))
((< condr2 cond_ok)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12 var-13 var-14 var-15)
     (zgesvj "L" "U" "N" nr nr v ldv sva nr u ldu
      (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                            ((+ (f2cl-lib:int-mul 2 n)
                                (f2cl-lib:int-mul n nr) nr 1))
                            ((1 lwork)) cwork-%offset%)
      (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                        (f2cl-lib:int-mul n nr) nr)
      rwork lrwork info)
   (declare
    (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12
     var-13))
   (when var-3 (setf nr var-3))
   (when var-4 (setf nr var-4))
   (when var-6 (setf ldv var-6))
   (when var-8 (setf nr var-8))
   (when var-10 (setf ldu var-10))
   (when var-14 (setf lrwork var-14))
   (when var-15 (setf info var-15)))
 (setf scalem
         (f2cl-lib:fref rwork-%data% (1) ((1 *))
                        rwork-%offset%))
 (setf numrank
         (f2cl-lib:nint
          (f2cl-lib:fref rwork-%data% (2) ((1 *))
                         rwork-%offset%)))
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p nr) nil)
   (tagbody
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (zcopy nr
```

```
          (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                                (1 p) ((1 ldv) (1 *))
                                v-%offset%)
          1
          (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                                (1 p) ((1 ldu) (1 *))
                                u-%offset%)
          1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf nr var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zdscal nr
          (f2cl-lib:array-slice sva-%data% double-float (p)
                                ((1 n)) sva-%offset%)
          (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                                (1 p) ((1 ldu) (1 *))
                                u-%offset%)
          1)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf nr var-0)))
   label3870))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (ztrsm "L" "U" "N" "N" nr nr cone
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) cwork-%offset%)
     n u ldu)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf nr var-4))
  (when var-5 (setf nr var-5))
  (when var-6 (setf cone var-6))
  (when var-8 (setf n var-8))
  (when var-10 (setf ldu var-10)))
(f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
              ((> q nr) nil)
  (tagbody
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p nr) nil)
      (tagbody
        (setf (f2cl-lib:fref cwork-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) nr
                               (f2cl-lib:fref iwork
                                              ((f2cl-lib:int-add
                                                n p))
                                              ((1 *)))))
                             ((1 lwork)) cwork-%offset%)
```

```
                    (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                                   u-%offset%))
          label872))
        (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                      ((> p nr) nil)
          (tagbody
            (setf (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                                 u-%offset%)
                    (f2cl-lib:fref cwork-%data%
                                   ((f2cl-lib:int-add
                                     (f2cl-lib:int-mul 2 n)
                                     (f2cl-lib:int-mul n nr) nr p))
                                   ((1 lwork)) cwork-%offset%))
          label874))
      label873))
    (cond
     ((< nr n)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlaset "A" (f2cl-lib:int-sub n nr) nr czero czero
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                                 ((+ nr 1) 1) ((1 ldv) (1 *))
                                 v-%offset%)
          ldv)
        (declare (ignore var-0 var-1 var-5))
        (when var-2 (setf nr var-2))
        (when var-3 (setf czero var-3))
        (when var-4 (setf czero var-4))
        (when var-6 (setf ldv var-6)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlaset "A" nr (f2cl-lib:int-sub n nr) czero czero
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                                 (1 (f2cl-lib:int-add nr 1))
                                 ((1 ldv) (1 *)) v-%offset%)
          ldv)
        (declare (ignore var-0 var-2 var-5))
        (when var-1 (setf nr var-1))
        (when var-3 (setf czero var-3))
        (when var-4 (setf czero var-4))
        (when var-6 (setf ldv var-6)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlaset "A" (f2cl-lib:int-sub n nr)
           (f2cl-lib:int-sub n nr) czero cone
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                                 ((+ nr 1) (f2cl-lib:int-add nr 1))
                                 ((1 ldv) (1 *)) v-%offset%)
          ldv)
        (declare (ignore var-0 var-1 var-2 var-5))
```

```
      (when var-3 (setf czero var-3))
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldv var-6)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zunmqr "L" "N" n n nr
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ (f2cl-lib:int-mul 2 n) 1))
                             ((1 lwork)) cwork-%offset%)
       n
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ n 1)) ((1 lwork))
                             cwork-%offset%)
       v ldv
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ (f2cl-lib:int-mul 2 n)
                                 (f2cl-lib:int-mul n nr) nr 1))
                             ((1 lwork)) cwork-%offset%)
       (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                         (f2cl-lib:int-mul n nr) nr)
       ierr)
    (declare
     (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf nr var-4))
    (when var-6 (setf n var-6))
    (when var-9 (setf ldv var-9))
    (when var-12 (setf ierr var-12))))
(t
 (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13 var-14 var-15)
      (zgesvj "L" "U" "V" nr nr v ldv sva nr u ldu
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ (f2cl-lib:int-mul 2 n)
                                 (f2cl-lib:int-mul n nr) nr 1))
                             ((1 lwork)) cwork-%offset%)
       (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                         (f2cl-lib:int-mul n nr) nr)
       rwork lrwork info)
    (declare
     (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12
      var-13))
    (when var-3 (setf nr var-3))
    (when var-4 (setf nr var-4))
    (when var-6 (setf ldv var-6))
    (when var-8 (setf nr var-8))
    (when var-10 (setf ldu var-10))
```

```
  (when var-14 (setf lrwork var-14))
  (when var-15 (setf info var-15)))
(setf scalem
        (f2cl-lib:fref rwork-%data% (1) ((1 *))
                       rwork-%offset%))
(setf numrank
        (f2cl-lib:nint
         (f2cl-lib:fref rwork-%data% (2) ((1 *))
                        rwork-%offset%)))
(cond
 ((< nr n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "A" (f2cl-lib:int-sub n nr) nr czero czero
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             ((+ nr 1) 1) ((1 ldv) (1 *))
                             v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "A" nr (f2cl-lib:int-sub n nr) czero czero
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             (1 (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-2 var-5))
    (when var-1 (setf nr var-1))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "A" (f2cl-lib:int-sub n nr)
       (f2cl-lib:int-sub n nr) czero cone
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             ((+ nr 1) (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf czero var-3))
    (when var-4 (setf cone var-4))
    (when var-6 (setf ldv var-6)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
```

```
      (zunmqr "L" "N" n n nr
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ (f2cl-lib:int-mul 2 n) 1))
                             ((1 lwork)) cwork-%offset%)
       n
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ n 1)) ((1 lwork))
                             cwork-%offset%)
       v ldv
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ (f2cl-lib:int-mul 2 n)
                                 (f2cl-lib:int-mul n nr) nr 1))
                             ((1 lwork)) cwork-%offset%)
       (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                         (f2cl-lib:int-mul n nr) nr)
       ierr)
    (declare
     (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf nr var-4))
    (when var-6 (setf n var-6))
    (when var-9 (setf ldv var-9))
    (when var-12 (setf ierr var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zunmlq "L" "C" nr nr nr
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ (f2cl-lib:int-mul 2 n) 1))
                             ((1 lwork)) cwork-%offset%)
       n
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ (f2cl-lib:int-mul 2 n)
                                 (f2cl-lib:int-mul n nr) 1))
                             ((1 lwork)) cwork-%offset%)
       u ldu
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ (f2cl-lib:int-mul 2 n)
                                 (f2cl-lib:int-mul n nr) nr 1))
                             ((1 lwork)) cwork-%offset%)
       (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                         (f2cl-lib:int-mul n nr) nr)
       ierr)
    (declare
     (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf nr var-2))
    (when var-3 (setf nr var-3))
    (when var-4 (setf nr var-4))
    (when var-6 (setf n var-6))
```

```
    (when var-9 (setf ldu var-9))
    (when var-12 (setf ierr var-12)))
  (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                ((> q nr) nil)
    (tagbody
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p nr) nil)
        (tagbody
          (setf (f2cl-lib:fref cwork-%data%
                               ((f2cl-lib:int-add
                                 (f2cl-lib:int-mul 2 n)
                                 (f2cl-lib:int-mul n nr) nr
                                 (f2cl-lib:fref iwork
                                                ((f2cl-lib:int-add
                                                  n p))
                                                ((1 *)))))
                               ((1 lwork)) cwork-%offset%)
                  (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                                 u-%offset%))
        label772))
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p nr) nil)
        (tagbody
          (setf (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                               u-%offset%)
                  (f2cl-lib:fref cwork-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n nr) nr p))
                                 ((1 lwork)) cwork-%offset%))
        label774))
    label773))))
(setf temp1 (* (f2cl-lib:dsqrt (f2cl-lib:dfloat n)) epsln))
(f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
              ((> q n) nil)
  (tagbody
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (setf (f2cl-lib:fref cwork-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-mul 2 n)
                               (f2cl-lib:int-mul n nr) nr
                               (f2cl-lib:fref iwork (p)
                                              ((1 *)))))
                             ((1 lwork)) cwork-%offset%)
                  (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                 v-%offset%))
      label972))
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
```

```
                  ((> p n) nil)
        (tagbody
          (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                v-%offset%)
                  (f2cl-lib:fref cwork-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n nr) nr p))
                                 ((1 lwork)) cwork-%offset%))
          label973))
      (setf xsc
              (/ one
                 (multiple-value-bind (ret-val var-0 var-1 var-2)
                     (dznrm2 n
                      (f2cl-lib:array-slice v-%data%
                                            f2cl-lib:complex16
                                            (1 q) ((1 ldv) (1 *))
                                            v-%offset%)
                      1)
                    (declare (ignore var-1 var-2))
                    (when var-0 (setf n var-0))
                    ret-val)))
      (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (zdscal n xsc
               (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                                     (1 q) ((1 ldv) (1 *))
                                     v-%offset%)
               1)
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf xsc var-1))))
     label1972))
  (cond
   ((< nr m)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlaset "A" (f2cl-lib:int-sub m nr) nr czero czero
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                               ((+ nr 1) 1) ((1 ldu) (1 *))
                               u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf nr var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldu var-6)))
    (cond
     ((< nr n1)
      (multiple-value-bind
```

```
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlaset "A" nr (f2cl-lib:int-sub n1 nr) czero czero
           (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                                 (1 (f2cl-lib:int-add nr 1))
                                 ((1 ldu) (1 *)) u-%offset%)
           ldu)
        (declare (ignore var-0 var-2 var-5))
        (when var-1 (setf nr var-1))
        (when var-3 (setf czero var-3))
        (when var-4 (setf czero var-4))
        (when var-6 (setf ldu var-6)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlaset "A" (f2cl-lib:int-sub m nr)
           (f2cl-lib:int-sub n1 nr) czero cone
           (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                                 ((+ nr 1) (f2cl-lib:int-add nr 1))
                                 ((1 ldu) (1 *)) u-%offset%)
           ldu)
        (declare (ignore var-0 var-1 var-2 var-5))
        (when var-3 (setf czero var-3))
        (when var-4 (setf cone var-4))
        (when var-6 (setf ldu var-6)))))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (zunmqr "Left" "No_Tr" m n1 n a lda cwork u ldu
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ n 1)) ((1 lwork)) cwork-%offset%)
       (f2cl-lib:int-sub lwork n) ierr)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n1 var-3))
    (when var-4 (setf n var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldu var-9))
    (when var-12 (setf ierr var-12)))
  (setf temp1 (* (f2cl-lib:dsqrt (f2cl-lib:dfloat m)) epsln))
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p nr) nil)
    (tagbody
      (setf xsc
              (/ one
                 (multiple-value-bind (ret-val var-0 var-1 var-2)
                     (dznrm2 m
                      (f2cl-lib:array-slice u-%data%
                                            f2cl-lib:complex16
                                            (1 p) ((1 ldu) (1 *))
                                            u-%offset%)
                      1)
```

```
                    (declare (ignore var-1 var-2))
                    (when var-0 (setf m var-0))
                    ret-val)))
        (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (zdscal m xsc
                 (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                                       (1 p) ((1 ldu) (1 *))
                                       u-%offset%)
                 1)
              (declare (ignore var-2 var-3))
              (when var-0 (setf m var-0))
              (when var-1 (setf xsc var-1))))
       label1973))
    (if rowpiv
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zlaswp n1 u ldu 1 (f2cl-lib:int-sub m 1)
             (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                                   ((+ (f2cl-lib:int-mul 2 n) 1))
                                   ((1 *)) iwork-%offset%)
             -1)
          (declare (ignore var-1 var-3 var-4 var-5 var-6))
          (when var-0 (setf n1 var-0))
          (when var-2 (setf ldu var-2)))))
   (t
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlacpy "Upper" n n a lda
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                               ((+ n 1)) ((1 lwork)) cwork-%offset%)
         n)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-6 (setf n var-6)))
    (cond
     (l2pert (setf xsc (f2cl-lib:dsqrt small))
      (f2cl-lib:fdo (p 2 (f2cl-lib:int-add p 1))
                    ((> p n) nil)
        (tagbody
          (setf ctemp
                  (* xsc
                     (f2cl-lib:fref cwork-%data%
                                    ((f2cl-lib:int-add n
                                                       (f2cl-lib:int-mul
                                                        (f2cl-lib:int-sub
                                                         p 1)
                                                        n)
                                                       p))
```

```
                                ((1 lwork)) cwork-%offset%)))
        (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                      ((> q
                          (f2cl-lib:int-add p
                                            (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref cwork-%data%
                                 ((f2cl-lib:int-add n
                                                    (f2cl-lib:int-mul
                                                     (f2cl-lib:int-sub
                                                      q 1)
                                                     n)
                                                    p))
                                 ((1 lwork)) cwork-%offset%)
                  (- ctemp))
           label5971))
      label5970)))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (zlaset "Lower" (f2cl-lib:int-sub n 1)
        (f2cl-lib:int-sub n 1) czero czero
        (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                              ((+ n 2)) ((1 lwork))
                              cwork-%offset%)
        n)
     (declare (ignore var-0 var-1 var-2 var-5))
     (when var-3 (setf czero var-3))
     (when var-4 (setf czero var-4))
     (when var-6 (setf n var-6)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15)
    (zgesvj "Upper" "U" "N" n n
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ n 1)) ((1 lwork)) cwork-%offset%)
     n sva n u ldu
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ n (f2cl-lib:int-mul n n) 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork n (f2cl-lib:int-mul n n)) rwork
     lrwork info)
  (declare
   (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12
    var-13))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-6 (setf n var-6))
  (when var-8 (setf n var-8))
```

```
  (when var-10 (setf ldu var-10))
  (when var-14 (setf lrwork var-14))
  (when var-15 (setf info var-15)))
(setf scalem
        (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%))
(setf numrank
        (f2cl-lib:nint
         (f2cl-lib:fref rwork-%data% (2) ((1 *))
                        rwork-%offset%)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zcopy n
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                               ((+ n
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-add p
                                                      (f2cl-lib:int-sub
                                                       1))
                                    n)
                                   1))
                               ((1 lwork)) cwork-%offset%)
         1
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex16 (1 p)
                               ((1 ldu) (1 *)) u-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zdscal n
         (f2cl-lib:array-slice sva-%data% double-float (p)
                               ((1 n)) sva-%offset%)
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                               ((+ n
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-add p
                                                      (f2cl-lib:int-sub
                                                       1))
                                    n)
                                   1))
                               ((1 lwork)) cwork-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf n var-0)))
   label6970))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrsm "Left" "Upper" "NoTrans" "No UD" n n cone a lda
```

```
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ n 1)) ((1 lwork)) cwork-%offset%)
     n)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-6 (setf cone var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf n var-10)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zcopy n
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                               ((+ n p)) ((1 lwork))
                               cwork-%offset%)
         n
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                               ((f2cl-lib:fref iwork (p) ((1 *)))
                                1)
                               ((1 ldv) (1 *)) v-%offset%)
         ldv)
      (declare (ignore var-1 var-3))
      (when var-0 (setf n var-0))
      (when var-2 (setf n var-2))
      (when var-4 (setf ldv var-4)))
   label6972))
(setf temp1 (* (f2cl-lib:dsqrt (f2cl-lib:dfloat n)) epsln))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (setf xsc
            (/ one
               (multiple-value-bind (ret-val var-0 var-1 var-2)
                   (dznrm2 n
                    (f2cl-lib:array-slice v-%data%
                                          f2cl-lib:complex16
                                          (1 p) ((1 ldv) (1 *))
                                          v-%offset%)
                    1)
                 (declare (ignore var-1 var-2))
                 (when var-0 (setf n var-0))
                 ret-val)))
    (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (zdscal n xsc
             (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                                   (1 p) ((1 ldv) (1 *))
                                   v-%offset%)
```

```
                1)
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf xsc var-1))))
    label6971))
 (cond
  ((< n m)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (zlaset "A" (f2cl-lib:int-sub m n) n czero czero
        (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                              ((+ n 1) 1) ((1 ldu) (1 *))
                              u-%offset%)
        ldu)
     (declare (ignore var-0 var-1 var-5))
     (when var-2 (setf n var-2))
     (when var-3 (setf czero var-3))
     (when var-4 (setf czero var-4))
     (when var-6 (setf ldu var-6)))
   (cond
    ((< n n1)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (zlaset "A" n (f2cl-lib:int-sub n1 n) czero czero
          (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                                (1 (f2cl-lib:int-add n 1))
                                ((1 ldu) (1 *)) u-%offset%)
          ldu)
       (declare (ignore var-0 var-2 var-5))
       (when var-1 (setf n var-1))
       (when var-3 (setf czero var-3))
       (when var-4 (setf czero var-4))
       (when var-6 (setf ldu var-6)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (zlaset "A" (f2cl-lib:int-sub m n)
          (f2cl-lib:int-sub n1 n) czero cone
          (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                                ((+ n 1) (f2cl-lib:int-add n 1))
                                ((1 ldu) (1 *)) u-%offset%)
          ldu)
       (declare (ignore var-0 var-1 var-2 var-5))
       (when var-3 (setf czero var-3))
       (when var-4 (setf cone var-4))
       (when var-6 (setf ldu var-6)))))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12)
     (zunmqr "Left" "No Tr" m n1 n a lda cwork u ldu
      (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
```

```
                                ((+ n 1)) ((1 lwork)) cwork-%offset%)
          (f2cl-lib:int-sub lwork n) ierr)
        (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf n1 var-3))
        (when var-4 (setf n var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldu var-9))
        (when var-12 (setf ierr var-12)))
      (setf temp1 (* (f2cl-lib:dsqrt (f2cl-lib:dfloat m)) epsln))
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p n1) nil)
        (tagbody
          (setf xsc
                  (/ one
                     (multiple-value-bind (ret-val var-0 var-1 var-2)
                         (dznrm2 m
                          (f2cl-lib:array-slice u-%data%
                                                f2cl-lib:complex16
                                                (1 p) ((1 ldu) (1 *))
                                                u-%offset%)
                          1)
                       (declare (ignore var-1 var-2))
                       (when var-0 (setf m var-0))
                       ret-val)))
          (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
              (multiple-value-bind (var-0 var-1 var-2 var-3)
                  (zdscal m xsc
                   (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                                         (1 p) ((1 ldu) (1 *))
                                         u-%offset%)
                   1)
                (declare (ignore var-2 var-3))
                (when var-0 (setf m var-0))
                (when var-1 (setf xsc var-1))))
         label6973))
      (if rowpiv
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (zlaswp n1 u ldu 1 (f2cl-lib:int-sub m 1)
               (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                                     ((+ (f2cl-lib:int-mul 2 n) 1))
                                     ((1 *)) iwork-%offset%)
               -1)
            (declare (ignore var-1 var-3 var-4 var-5 var-6))
            (when var-0 (setf n1 var-0))
            (when var-2 (setf ldu var-2)))))))
    (t
     (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                   ((> p nr) nil)
```

```
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zcopy (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (p p)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (p p)
                               ((1 ldv) (1 *)) v-%offset%)
         1)
      (declare (ignore var-0 var-1 var-3 var-4))
      (when var-2 (setf lda var-2)))
    (zlacgv (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
     (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (p p)
                           ((1 ldv) (1 *)) v-%offset%)
     1)
   label7968))
(cond
 (l2pert (setf xsc (f2cl-lib:dsqrt (/ small epsln)))
  (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                ((> q nr) nil)
    (tagbody
      (setf ctemp
              (f2cl-lib:dcmplx
               (* xsc
                  (abs
                   (f2cl-lib:fref v-%data% (q q) ((1 ldv) (1 *))
                                  v-%offset%)))
               zero))
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p n) nil)
        (tagbody
          (if (or
               (and (> p q)
                    (<=
                     (abs
                      (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                     v-%offset%))
                     temp1))
               (< p q))
              (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                   v-%offset%)
                    ctemp))
          (if (< p q)
              (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                   v-%offset%)
                    (-
                     (f2cl-lib:fref v-%data% (p q)
                                    ((1 ldv) (1 *))
                                    v-%offset%))))
         label5968))
```

```
    label5969)))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
       czero czero
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (1 2)
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldv var-6)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgeqrf n nr v ldv
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ n 1)) ((1 lwork)) cwork-%offset%)
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (when var-0 (setf n var-0))
  (when var-1 (setf nr var-1))
  (when var-3 (setf ldv var-3))
  (when var-7 (setf ierr var-7)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" n nr v ldv
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ (f2cl-lib:int-mul 2 n) 1))
                           ((1 lwork)) cwork-%offset%)
     n)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nr var-2))
  (when var-4 (setf ldv var-4))
  (when var-6 (setf n var-6)))
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p nr) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zcopy (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (p p)
                               ((1 ldv) (1 *)) v-%offset%)
         ldv
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex16 (p p)
                               ((1 ldu) (1 *)) u-%offset%)
         1)
      (declare (ignore var-0 var-1 var-3 var-4))
      (when var-2 (setf ldv var-2)))
```

```
        (zlacgv (f2cl-lib:int-add (f2cl-lib:int-sub nr p) 1)
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex16 (p p)
                               ((1 ldu) (1 *)) u-%offset%)
         1)
       label7969))
    (cond
     (l2pert (setf xsc (f2cl-lib:dsqrt (/ small epsln)))
      (f2cl-lib:fdo (q 2 (f2cl-lib:int-add q 1))
                    ((> q nr) nil)
        (tagbody
          (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                        ((> p
                            (f2cl-lib:int-add q (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf ctemp
                      (f2cl-lib:dcmplx
                       (* xsc
                          (f2cl-lib:dmin1
                           (abs
                            (f2cl-lib:fref u-%data% (p p)
                                           ((1 ldu) (1 *)) u-%offset%))
                           (abs
                            (f2cl-lib:fref u-%data% (q q)
                                           ((1 ldu) (1 *))
                                           u-%offset%))))
                       zero))
              (setf (f2cl-lib:fref u-%data% (p q) ((1 ldu) (1 *))
                                   u-%offset%)
                      (- ctemp))
             label9971))
         label9970)))
     (t
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlaset "U" (f2cl-lib:int-sub nr 1) (f2cl-lib:int-sub nr 1)
           czero czero
           (f2cl-lib:array-slice u-%data% f2cl-lib:complex16 (1 2)
                                 ((1 ldu) (1 *)) u-%offset%)
           ldu)
        (declare (ignore var-0 var-1 var-2 var-5))
        (when var-3 (setf czero var-3))
        (when var-4 (setf czero var-4))
        (when var-6 (setf ldu var-6)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14 var-15)
        (zgesvj "L" "U" "V" nr nr u ldu sva n v ldv
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                               ((+ (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n nr) 1))
```

```
                          ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                       (f2cl-lib:int-mul n nr))
     rwork lrwork info)
  (declare
   (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-11 var-12
    var-13))
  (when var-3 (setf nr var-3))
  (when var-4 (setf nr var-4))
  (when var-6 (setf ldu var-6))
  (when var-8 (setf n var-8))
  (when var-10 (setf ldv var-10))
  (when var-14 (setf lrwork var-14))
  (when var-15 (setf info var-15)))
(setf scalem
        (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%))
(setf numrank
        (f2cl-lib:nint
         (f2cl-lib:fref rwork-%data% (2) ((1 *)) rwork-%offset%)))
(cond
 ((< nr n)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "A" (f2cl-lib:int-sub n nr) nr zero zero
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             ((+ nr 1) 1) ((1 ldv) (1 *))
                             v-%offset%)
       ldv)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "A" nr (f2cl-lib:int-sub n nr) zero zero
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             (1 (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
    (declare (ignore var-0 var-2 var-5))
    (when var-1 (setf nr var-1))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldv var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "A" (f2cl-lib:int-sub n nr) (f2cl-lib:int-sub n nr)
       zero one
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             ((+ nr 1) (f2cl-lib:int-add nr 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv)
```

```
        (declare (ignore var-0 var-1 var-2 var-5))
        (when var-3 (setf zero var-3))
        (when var-4 (setf one var-4))
        (when var-6 (setf ldv var-6)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (zunmqr "L" "N" n n nr
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                               ((+ (f2cl-lib:int-mul 2 n) 1))
                               ((1 lwork)) cwork-%offset%)
         n
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                               ((+ n 1)) ((1 lwork)) cwork-%offset%)
         v ldv
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                               ((+ (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n nr) nr 1))
                               ((1 lwork)) cwork-%offset%)
         (f2cl-lib:int-sub lwork (f2cl-lib:int-mul 2 n)
                           (f2cl-lib:int-mul n nr) nr)
         ierr)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf n var-2))
      (when var-3 (setf n var-3))
      (when var-4 (setf nr var-4))
      (when var-6 (setf n var-6))
      (when var-9 (setf ldv var-9))
      (when var-12 (setf ierr var-12)))
    (setf temp1 (* (f2cl-lib:dsqrt (f2cl-lib:dfloat n)) epsln))
    (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                  ((> q n) nil)
      (tagbody
        (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                      ((> p n) nil)
          (tagbody
            (setf (f2cl-lib:fref cwork-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n nr) nr
                                   (f2cl-lib:fref iwork (p) ((1 *)))))
                                 ((1 lwork)) cwork-%offset%)
                  (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                 v-%offset%))
           label8972))
        (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                      ((> p n) nil)
          (tagbody
            (setf (f2cl-lib:fref v-%data% (p q) ((1 ldv) (1 *))
                                 v-%offset%)
```

```
                    (f2cl-lib:fref cwork-%data%
                                   ((f2cl-lib:int-add
                                     (f2cl-lib:int-mul 2 n)
                                     (f2cl-lib:int-mul n nr) nr p))
                                   ((1 lwork)) cwork-%offset%))
        label8973))
    (setf xsc
            (/ one
               (multiple-value-bind (ret-val var-0 var-1 var-2)
                   (dznrm2 n
                    (f2cl-lib:array-slice v-%data%
                                          f2cl-lib:complex16 (1 q)
                                          ((1 ldv) (1 *))
                                          v-%offset%)
                    1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf n var-0))
                  ret-val)))
    (if (or (< xsc (- one temp1)) (> xsc (+ one temp1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (zdscal n xsc
             (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                                   (1 q) ((1 ldv) (1 *))
                                   v-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-1 (setf xsc var-1))))
   label7972))
(cond
 ((< nr m)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "A" (f2cl-lib:int-sub m nr) nr czero czero
       (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                             ((+ nr 1) 1) ((1 ldu) (1 *))
                             u-%offset%)
       ldu)
    (declare (ignore var-0 var-1 var-5))
    (when var-2 (setf nr var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldu var-6)))
  (cond
   ((< nr n1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlaset "A" nr (f2cl-lib:int-sub n1 nr) czero czero
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                               (1 (f2cl-lib:int-add nr 1))
                               ((1 ldu) (1 *)) u-%offset%)
```

```
            ldu)
          (declare (ignore var-0 var-2 var-5))
          (when var-1 (setf nr var-1))
          (when var-3 (setf czero var-3))
          (when var-4 (setf czero var-4))
          (when var-6 (setf ldu var-6)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zlaset "A" (f2cl-lib:int-sub m nr)
             (f2cl-lib:int-sub n1 nr) czero cone
             (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                                   ((+ nr 1) (f2cl-lib:int-add nr 1))
                                   ((1 ldu) (1 *)) u-%offset%)
            ldu)
          (declare (ignore var-0 var-1 var-2 var-5))
          (when var-3 (setf czero var-3))
          (when var-4 (setf cone var-4))
          (when var-6 (setf ldu var-6)))))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (zunmqr "Left" "No Tr" m n1 n a lda cwork u ldu
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                               ((+ n 1)) ((1 lwork)) cwork-%offset%)
         (f2cl-lib:int-sub lwork n) ierr)
      (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf n1 var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldu var-9))
      (when var-12 (setf ierr var-12)))
    (if rowpiv
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zlaswp n1 u ldu 1 (f2cl-lib:int-sub m 1)
             (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                                   ((+ (f2cl-lib:int-mul 2 n) 1))
                                   ((1 *)) iwork-%offset%)
            -1)
          (declare (ignore var-1 var-3 var-4 var-5 var-6))
          (when var-0 (setf n1 var-0))
          (when var-2 (setf ldu var-2))))))
  (cond
   (transp
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (zswap n
```

```
                (f2cl-lib:array-slice u-%data% f2cl-lib:complex16 (1 p)
                                      ((1 ldu) (1 *)) u-%offset%)
                1
                (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (1 p)
                                      ((1 ldv) (1 *)) v-%offset%)
                1)
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf n var-0)))
          label6974))))))
  (cond
   ((<= uscal2
        (* (f2cl-lib:f2cl/ big (f2cl-lib:fref sva (1) ((1 n)))) uscal1))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (zlascl "G" 0 0 uscal1 uscal2 nr 1 sva n ierr)
      (declare (ignore var-0 var-1 var-2 var-6 var-7))
      (when var-3 (setf uscal1 var-3))
      (when var-4 (setf uscal2 var-4))
      (when var-5 (setf nr var-5))
      (when var-8 (setf n var-8))
      (when var-9 (setf ierr var-9)))
    (setf uscal1 one) (setf uscal2 one)))
  (cond
   ((< nr n)
    (f2cl-lib:fdo (p (f2cl-lib:int-add nr 1) (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%) zero)
      label3004))))
  (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
          (* uscal2 scalem))
  (setf (f2cl-lib:fref rwork-%data% (2) ((1 *)) rwork-%offset%) uscal1)
  (if errest
      (setf (f2cl-lib:fref rwork-%data% (3) ((1 *)) rwork-%offset%)
              sconda))
  (cond
   ((and lsvec rsvec)
    (setf (f2cl-lib:fref rwork-%data% (4) ((1 *)) rwork-%offset%) condr1)
    (setf (f2cl-lib:fref rwork-%data% (5) ((1 *)) rwork-%offset%)
            condr2)))
  (cond
   (l2tran
    (setf (f2cl-lib:fref rwork-%data% (6) ((1 *)) rwork-%offset%) entra)
    (setf (f2cl-lib:fref rwork-%data% (7) ((1 *)) rwork-%offset%)
            entrat)))
  (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) nr)
  (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%) numrank)
  (setf (f2cl-lib:fref iwork-%data% (3) ((1 *)) iwork-%offset%) warning)
  (go end_label)
end_label
```

```
        (return
         (values joba jobu jobv jobr jobt jobp m n nil lda nil nil ldu nil ldv
                 nil lwork nil lrwork nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgejsv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::joba
                                              fortran-to-lisp::jobu
                                              fortran-to-lisp::jobv
                                              fortran-to-lisp::jobr
                                              fortran-to-lisp::jobt
                                              fortran-to-lisp::jobp
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::ldu nil
                                              fortran-to-lisp::ldv nil
                                              fortran-to-lisp::lwork nil
                                              fortran-to-lisp::lrwork nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

31.2 zgesdd

31.2.1 Axiom unit tests

— zgesdd.input —

```
)set break resume
)sys rm -f zgesdd.output
)spool zgesdd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

31.2.2 Axiom help page

— zgesdd.help —

```
======================================================================
zgesdd examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgesdd (JOBZ, M, N, A, LDA, S, U, LDU, VT, LDVT, WORK,
           LWORK, RWORK, IWORK, INFO)
           ZGESDD

Function/Subroutine Documentation
   subroutine zgesdd (character JOBZ, integer M, integer N, complex*16,
       dimension( lda, * ) A, integer LDA, double precision, dimension( * ) S,
       complex*16, dimension( ldu, * ) U, integer LDU, complex*16, dimension(
       ldvt, * ) VT, integer LDVT, complex*16, dimension( * ) WORK, integer
       LWORK, double precision, dimension( * ) RWORK, integer, dimension( * )
```

```
       IWORK, integer INFO)
       ZGESDD

Purpose:
   ZGESDD computes the singular value decomposition (SVD) of a complex
   M-by-N matrix A, optionally computing the left and/or right singular
   vectors, by using divide-and-conquer method. The SVD is written

   A = U * SIGMA * conjugate-transpose(V)

   where SIGMA is an M-by-N matrix which is zero except for its
   min(m,n) diagonal elements, U is an M-by-M unitary matrix, and
   V is an N-by-N unitary matrix.  The diagonal elements of SIGMA
   are the singular values of A; they are real and non-negative, and
   are returned in descending order.  The first min(m,n) columns of
   U and V are the left and right singular vectors of A.

   Note that the routine returns VT = V**H, not V.

   The divide and conquer algorithm makes very mild assumptions about
   floating point arithmetic. It will work on machines with a guard
   digit in add/subtract, or on those binary machines without guard
   digits which subtract like the Cray X-MP, Cray Y-MP, Cray C-90, or
   Cray-2. It could conceivably fail on hexadecimal or decimal machines
   without guard digits, but we know of none.

Parameters:

 JOBZ
   JOBZ is CHARACTER*1
   Specifies options for computing all or part of the matrix U:
   = 'A':  all M columns of U and all N rows of V**H are
           returned in the arrays U and VT;
   = 'S':  the first min(M,N) columns of U and the first
           min(M,N) rows of V**H are returned in the arrays U and VT;
   = 'O':  If M >= N, the first N columns of U are overwritten
           in the array A and all rows of V**H are returned in the array VT;
           otherwise, all columns of U are returned in the
           array U and the first M rows of V**H are overwritten
           in the array A;
   = 'N':  no columns of U or rows of V**H are computed.

 M
   M is INTEGER
   The number of rows of the input matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the input matrix A.  N >= 0.
```

```
A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit,
  if JOBZ = 'O',  A is overwritten with the first N columns
       of U (the left singular vectors, stored columnwise) if M >= N;
       A is overwritten with the first M rows
       of V**H (the right singular vectors, stored rowwise) otherwise.
  if JOBZ .ne. 'O', the contents of A are destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

S
  S is DOUBLE PRECISION array, dimension (min(M,N))
  The singular values of A, sorted so that S(i) >= S(i+1).

U
  U is COMPLEX*16 array, dimension (LDU,UCOL)
  UCOL = M if JOBZ = 'A' or JOBZ = 'O' and M < N;
  UCOL = min(M,N) if JOBZ = 'S'.
  If JOBZ = 'A' or JOBZ = 'O' and M < N, U contains the M-by-M
  unitary matrix U;
  if JOBZ = 'S', U contains the first min(M,N) columns of U
  (the left singular vectors, stored columnwise);
  if JOBZ = 'O' and M >= N, or JOBZ = 'N', U is not referenced.

LDU
  LDU is INTEGER
  The leading dimension of the array U.  LDU >= 1; if
  JOBZ = 'S' or 'A' or JOBZ = 'O' and M < N, LDU >= M.

VT
  VT is COMPLEX*16 array, dimension (LDVT,N)
  If JOBZ = 'A' or JOBZ = 'O' and M >= N, VT contains the
  N-by-N unitary matrix V**H;
  if JOBZ = 'S', VT contains the first min(M,N) rows of
  V**H (the right singular vectors, stored rowwise);
  if JOBZ = 'O' and M < N, or JOBZ = 'N', VT is not referenced.

LDVT
  LDVT is INTEGER
  The leading dimension of the array VT.  LDVT >= 1; if
  JOBZ = 'A' or JOBZ = 'O' and M >= N, LDVT >= N;
  if JOBZ = 'S', LDVT >= min(M,N).

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.
```

```
 LWORK
   LWORK is INTEGER
   The dimension of the array WORK. LWORK >= 1.
   if JOBZ = 'N', LWORK >= 2*min(M,N)+max(M,N).
   if JOBZ = 'O',
      LWORK >= 2*min(M,N)*min(M,N)+2*min(M,N)+max(M,N).
   if JOBZ = 'S' or 'A',
      LWORK >= min(M,N)*min(M,N)+2*min(M,N)+max(M,N).
   For good performance, LWORK should generally be larger.

   If LWORK = -1, a workspace query is assumed.  The optimal
   size for the WORK array is calculated and stored in WORK(1),
   and no other work except argument checking is performed.

 RWORK
   RWORK is DOUBLE PRECISION array, dimension (MAX(1,LRWORK))
   If JOBZ = 'N', LRWORK >= 7*min(M,N).
   Otherwise,
   LRWORK >= min(M,N)*max(5*min(M,N)+7,2*max(M,N)+2*min(M,N)+1)

 IWORK
   IWORK is INTEGER array, dimension (8*min(M,N))

 INFO
   INFO is INTEGER
   = 0:  successful exit.
   < 0:  if INFO = -i, the i-th argument had an illegal value.
   > 0:  The updating process of DBDSDC did not converge.
```

———-

31.2.3 fortran code

— **zgesdd.f** —

```
*  =====================================================================
      SUBROUTINE ZGESDD( JOBZ, M, N, A, LDA, S, U, LDU, VT, LDVT, WORK,
     $                   LWORK, RWORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBZ
```

```
      INTEGER            INFO, LDA, LDU, LDVT, LWORK, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   RWORK( * ), S( * )
      COMPLEX*16         A( LDA, * ), U( LDU, * ), VT( LDVT, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            LQUERV
      PARAMETER          ( LQUERV = -1 )
      COMPLEX*16         CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0D+0, 0.0D+0 ),
     $                   CONE = ( 1.0D+0, 0.0D+0 ) )
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            WNTQA, WNTQAS, WNTQN, WNTQO, WNTQS
      INTEGER            BLK, CHUNK, I, IE, IERR, IL, IR, IRU, IRVT,
     $                   ISCL, ITAU, ITAUP, ITAUQ, IU, IVT, LDWKVT,
     $                   LDWRKL, LDWRKR, LDWRKU, MAXWRK, MINMN, MINWRK,
     $                   MNTHR1, MNTHR2, NRWORK, NWORK, WRKBL
      DOUBLE PRECISION   ANRM, BIGNUM, EPS, SMLNUM
*     ..
*     .. Local Arrays ..
      INTEGER            IDUM( 1 )
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DBDSDC, DLASCL, XERBLA, ZGEBRD, ZGELQF, ZGEMM,
     $                   ZGEQRF, ZLACP2, ZLACPY, ZLACRM, ZLARCM, ZLASCL,
     $                   ZLASET, ZUNGBR, ZUNGLQ, ZUNGQR, ZUNMBR
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, ZLANGE
      EXTERNAL           LSAME, ILAENV, DLAMCH, ZLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          INT, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
```

```
      INFO = 0
      MINMN = MIN( M, N )
      MNTHR1 = INT( MINMN*17.0D0 / 9.0D0 )
      MNTHR2 = INT( MINMN*5.0D0 / 3.0D0 )
      WNTQA = LSAME( JOBZ, 'A' )
      WNTQS = LSAME( JOBZ, 'S' )
      WNTQAS = WNTQA .OR. WNTQS
      WNTQO = LSAME( JOBZ, 'O' )
      WNTQN = LSAME( JOBZ, 'N' )
      MINWRK = 1
      MAXWRK = 1
*
      IF( .NOT.( WNTQA .OR. WNTQS .OR. WNTQO .OR. WNTQN ) ) THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDU.LT.1 .OR. ( WNTQAS .AND. LDU.LT.M ) .OR.
     $         ( WNTQO .AND. M.LT.N .AND. LDU.LT.M ) ) THEN
         INFO = -8
      ELSE IF( LDVT.LT.1 .OR. ( WNTQA .AND. LDVT.LT.N ) .OR.
     $         ( WNTQS .AND. LDVT.LT.MINMN ) .OR.
     $         ( WNTQO .AND. M.GE.N .AND. LDVT.LT.N ) ) THEN
         INFO = -10
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       CWorkspace refers to complex workspace, and RWorkspace to
*       real workspace. NB refers to the optimal block size for the
*       immediately following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 .AND. M.GT.0 .AND. N.GT.0 ) THEN
         IF( M.GE.N ) THEN
*
*           There is no complex work space needed for bidiagonal SVD
*           The real work space needed for bidiagonal SVD is BDSPAC
*           for computing singular values and singular vectors; BDSPAN
*           for computing singular values only.
*           BDSPAC = 5*N*N + 7*N
*           BDSPAN = MAX(7*N+4, 3*N+2+SMLSIZ*(SMLSIZ+8))
*
            IF( M.GE.MNTHR1 ) THEN
               IF( WNTQN ) THEN
*
```

```
*                 Path 1 (M much larger than N, JOBZ='N')
*
                  MAXWRK = N + N*ILAENV( 1, 'ZGEQRF', ' ', M, N, -1,
     $                     -1 )
                  MAXWRK = MAX( MAXWRK, 2*N+2*N*
     $                     ILAENV( 1, 'ZGEBRD', ' ', N, N, -1, -1 ) )
                  MINWRK = 3*N
               ELSE IF( WNTQO ) THEN
*
*                 Path 2 (M much larger than N, JOBZ='O')
*
                  WRKBL = N + N*ILAENV( 1, 'ZGEQRF', ' ', M, N, -1, -1 )
                  WRKBL = MAX( WRKBL, N+N*ILAENV( 1, 'ZUNGQR', ' ', M,
     $                    N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 2*N+2*N*
     $                    ILAENV( 1, 'ZGEBRD', ' ', N, N, -1, -1 ) )
                  WRKBL = MAX( WRKBL, 2*N+N*
     $                    ILAENV( 1, 'ZUNMBR', 'QLN', N, N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 2*N+N*
     $                    ILAENV( 1, 'ZUNMBR', 'PRC', N, N, N, -1 ) )
                  MAXWRK = M*N + N*N + WRKBL
                  MINWRK = 2*N*N + 3*N
               ELSE IF( WNTQS ) THEN
*
*                 Path 3 (M much larger than N, JOBZ='S')
*
                  WRKBL = N + N*ILAENV( 1, 'ZGEQRF', ' ', M, N, -1, -1 )
                  WRKBL = MAX( WRKBL, N+N*ILAENV( 1, 'ZUNGQR', ' ', M,
     $                    N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 2*N+2*N*
     $                    ILAENV( 1, 'ZGEBRD', ' ', N, N, -1, -1 ) )
                  WRKBL = MAX( WRKBL, 2*N+N*
     $                    ILAENV( 1, 'ZUNMBR', 'QLN', N, N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 2*N+N*
     $                    ILAENV( 1, 'ZUNMBR', 'PRC', N, N, N, -1 ) )
                  MAXWRK = N*N + WRKBL
                  MINWRK = N*N + 3*N
               ELSE IF( WNTQA ) THEN
*
*                 Path 4 (M much larger than N, JOBZ='A')
*
                  WRKBL = N + N*ILAENV( 1, 'ZGEQRF', ' ', M, N, -1, -1 )
                  WRKBL = MAX( WRKBL, N+M*ILAENV( 1, 'ZUNGQR', ' ', M,
     $                    M, N, -1 ) )
                  WRKBL = MAX( WRKBL, 2*N+2*N*
     $                    ILAENV( 1, 'ZGEBRD', ' ', N, N, -1, -1 ) )
                  WRKBL = MAX( WRKBL, 2*N+N*
     $                    ILAENV( 1, 'ZUNMBR', 'QLN', N, N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 2*N+N*
     $                    ILAENV( 1, 'ZUNMBR', 'PRC', N, N, N, -1 ) )
```

```
                 MAXWRK = N*N + WRKBL
                 MINWRK = N*N + 2*N + M
              END IF
           ELSE IF( M.GE.MNTHR2 ) THEN
*
*             Path 5 (M much larger than N, but not as much as MNTHR1)
*
              MAXWRK = 2*N + ( M+N )*ILAENV( 1, 'ZGEBRD', ' ', M, N,
     $                 -1, -1 )
              MINWRK = 2*N + M
              IF( WNTQO ) THEN
                 MAXWRK = MAX( MAXWRK, 2*N+N*
     $                   ILAENV( 1, 'ZUNGBR', 'P', N, N, N, -1 ) )
                 MAXWRK = MAX( MAXWRK, 2*N+N*
     $                   ILAENV( 1, 'ZUNGBR', 'Q', M, N, N, -1 ) )
                 MAXWRK = MAXWRK + M*N
                 MINWRK = MINWRK + N*N
              ELSE IF( WNTQS ) THEN
                 MAXWRK = MAX( MAXWRK, 2*N+N*
     $                   ILAENV( 1, 'ZUNGBR', 'P', N, N, N, -1 ) )
                 MAXWRK = MAX( MAXWRK, 2*N+N*
     $                   ILAENV( 1, 'ZUNGBR', 'Q', M, N, N, -1 ) )
              ELSE IF( WNTQA ) THEN
                 MAXWRK = MAX( MAXWRK, 2*N+N*
     $                   ILAENV( 1, 'ZUNGBR', 'P', N, N, N, -1 ) )
                 MAXWRK = MAX( MAXWRK, 2*N+M*
     $                   ILAENV( 1, 'ZUNGBR', 'Q', M, M, N, -1 ) )
              END IF
           ELSE
*
*             Path 6 (M at least N, but not much larger)
*
              MAXWRK = 2*N + ( M+N )*ILAENV( 1, 'ZGEBRD', ' ', M, N,
     $                 -1, -1 )
              MINWRK = 2*N + M
              IF( WNTQO ) THEN
                 MAXWRK = MAX( MAXWRK, 2*N+N*
     $                   ILAENV( 1, 'ZUNMBR', 'PRC', N, N, N, -1 ) )
                 MAXWRK = MAX( MAXWRK, 2*N+N*
     $                   ILAENV( 1, 'ZUNMBR', 'QLN', M, N, N, -1 ) )
                 MAXWRK = MAXWRK + M*N
                 MINWRK = MINWRK + N*N
              ELSE IF( WNTQS ) THEN
                 MAXWRK = MAX( MAXWRK, 2*N+N*
     $                   ILAENV( 1, 'ZUNMBR', 'PRC', N, N, N, -1 ) )
                 MAXWRK = MAX( MAXWRK, 2*N+N*
     $                   ILAENV( 1, 'ZUNMBR', 'QLN', M, N, N, -1 ) )
              ELSE IF( WNTQA ) THEN
                 MAXWRK = MAX( MAXWRK, 2*N+N*
     $                   ILAENV( 1, 'ZUNGBR', 'PRC', N, N, N, -1 ) )
```

```
                 MAXWRK = MAX( MAXWRK, 2*N+M*
     $                   ILAENV( 1, 'ZUNGBR', 'QLN', M, M, N, -1 ) )
              END IF
           END IF
        ELSE
*
*          There is no complex work space needed for bidiagonal SVD
*          The real work space needed for bidiagonal SVD is BDSPAC
*          for computing singular values and singular vectors; BDSPAN
*          for computing singular values only.
*          BDSPAC = 5*M*M + 7*M
*          BDSPAN = MAX(7*M+4, 3*M+2+SMLSIZ*(SMLSIZ+8))
*
           IF( N.GE.MNTHR1 ) THEN
              IF( WNTQN ) THEN
*
*                Path 1t (N much larger than M, JOBZ='N')
*
                 MAXWRK = M + M*ILAENV( 1, 'ZGELQF', ' ', M, N, -1,
     $                   -1 )
                 MAXWRK = MAX( MAXWRK, 2*M+2*M*
     $                   ILAENV( 1, 'ZGEBRD', ' ', M, M, -1, -1 ) )
                 MINWRK = 3*M
              ELSE IF( WNTQO ) THEN
*
*                Path 2t (N much larger than M, JOBZ='O')
*
                 WRKBL = M + M*ILAENV( 1, 'ZGELQF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, M+M*ILAENV( 1, 'ZUNGLQ', ' ', M,
     $                   N, M, -1 ) )
                 WRKBL = MAX( WRKBL, 2*M+2*M*
     $                   ILAENV( 1, 'ZGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 2*M+M*
     $                   ILAENV( 1, 'ZUNMBR', 'PRC', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, 2*M+M*
     $                   ILAENV( 1, 'ZUNMBR', 'QLN', M, M, M, -1 ) )
                 MAXWRK = M*N + M*M + WRKBL
                 MINWRK = 2*M*M + 3*M
              ELSE IF( WNTQS ) THEN
*
*                Path 3t (N much larger than M, JOBZ='S')
*
                 WRKBL = M + M*ILAENV( 1, 'ZGELQF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, M+M*ILAENV( 1, 'ZUNGLQ', ' ', M,
     $                   N, M, -1 ) )
                 WRKBL = MAX( WRKBL, 2*M+2*M*
     $                   ILAENV( 1, 'ZGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 2*M+M*
     $                   ILAENV( 1, 'ZUNMBR', 'PRC', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, 2*M+M*
```

```
     $                  ILAENV( 1, 'ZUNMBR', 'QLN', M, M, M, -1 ) )
                   MAXWRK = M*M + WRKBL
                   MINWRK = M*M + 3*M
                ELSE IF( WNTQA ) THEN
*
*                  Path 4t (N much larger than M, JOBZ='A')
*
                   WRKBL = M + M*ILAENV( 1, 'ZGELQF', ' ', M, N, -1, -1 )
                   WRKBL = MAX( WRKBL, M+N*ILAENV( 1, 'ZUNGLQ', ' ', N,
     $                  N, M, -1 ) )
                   WRKBL = MAX( WRKBL, 2*M+2*M*
     $                  ILAENV( 1, 'ZGEBRD', ' ', M, M, -1, -1 ) )
                   WRKBL = MAX( WRKBL, 2*M+M*
     $                  ILAENV( 1, 'ZUNMBR', 'PRC', M, M, M, -1 ) )
                   WRKBL = MAX( WRKBL, 2*M+M*
     $                  ILAENV( 1, 'ZUNMBR', 'QLN', M, M, M, -1 ) )
                   MAXWRK = M*M + WRKBL
                   MINWRK = M*M + 2*M + N
                END IF
             ELSE IF( N.GE.MNTHR2 ) THEN
*
*                Path 5t (N much larger than M, but not as much as MNTHR1)
*
                MAXWRK = 2*M + ( M+N )*ILAENV( 1, 'ZGEBRD', ' ', M, N,
     $                 -1, -1 )
                MINWRK = 2*M + N
                IF( WNTQO ) THEN
                   MAXWRK = MAX( MAXWRK, 2*M+M*
     $                    ILAENV( 1, 'ZUNGBR', 'P', M, N, M, -1 ) )
                   MAXWRK = MAX( MAXWRK, 2*M+M*
     $                    ILAENV( 1, 'ZUNGBR', 'Q', M, M, N, -1 ) )
                   MAXWRK = MAXWRK + M*N
                   MINWRK = MINWRK + M*M
                ELSE IF( WNTQS ) THEN
                   MAXWRK = MAX( MAXWRK, 2*M+M*
     $                    ILAENV( 1, 'ZUNGBR', 'P', M, N, M, -1 ) )
                   MAXWRK = MAX( MAXWRK, 2*M+M*
     $                    ILAENV( 1, 'ZUNGBR', 'Q', M, M, N, -1 ) )
                ELSE IF( WNTQA ) THEN
                   MAXWRK = MAX( MAXWRK, 2*M+N*
     $                    ILAENV( 1, 'ZUNGBR', 'P', N, N, M, -1 ) )
                   MAXWRK = MAX( MAXWRK, 2*M+M*
     $                    ILAENV( 1, 'ZUNGBR', 'Q', M, M, N, -1 ) )
                END IF
             ELSE
*
*                Path 6t (N greater than M, but not much larger)
*
                MAXWRK = 2*M + ( M+N )*ILAENV( 1, 'ZGEBRD', ' ', M, N,
     $                 -1, -1 )
```

```
               MINWRK = 2*M + N
               IF( WNTQO ) THEN
                  MAXWRK = MAX( MAXWRK, 2*M+M*
     $                    ILAENV( 1, 'ZUNMBR', 'PRC', M, N, M, -1 ) )
                  MAXWRK = MAX( MAXWRK, 2*M+M*
     $                    ILAENV( 1, 'ZUNMBR', 'QLN', M, M, N, -1 ) )
                  MAXWRK = MAXWRK + M*N
                  MINWRK = MINWRK + M*M
               ELSE IF( WNTQS ) THEN
                  MAXWRK = MAX( MAXWRK, 2*M+M*
     $                    ILAENV( 1, 'ZUNGBR', 'PRC', M, N, M, -1 ) )
                  MAXWRK = MAX( MAXWRK, 2*M+M*
     $                    ILAENV( 1, 'ZUNGBR', 'QLN', M, M, N, -1 ) )
               ELSE IF( WNTQA ) THEN
                  MAXWRK = MAX( MAXWRK, 2*M+N*
     $                    ILAENV( 1, 'ZUNGBR', 'PRC', N, N, M, -1 ) )
                  MAXWRK = MAX( MAXWRK, 2*M+M*
     $                    ILAENV( 1, 'ZUNGBR', 'QLN', M, M, N, -1 ) )
               END IF
            END IF
         END IF
         MAXWRK = MAX( MAXWRK, MINWRK )
      END IF
      IF( INFO.EQ.0 ) THEN
         WORK( 1 ) = MAXWRK
         IF( LWORK.LT.MINWRK .AND. LWORK.NE.LQUERV )
     $      INFO = -13
      END IF
*
*     Quick returns
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGESDD', -INFO )
         RETURN
      END IF
      IF( LWORK.EQ.LQUERV )
     $   RETURN
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = SQRT( DLAMCH( 'S' ) ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = ZLANGE( 'M', M, N, A, LDA, DUM )
```

```
      ISCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ISCL = 1
         CALL ZLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, IERR )
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ISCL = 1
         CALL ZLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, IERR )
      END IF
*
      IF( M.GE.N ) THEN
*
*        A has at least as many rows as columns. If A has sufficiently
*        more rows than columns, first reduce using the QR
*        decomposition (if sufficient workspace available)
*
         IF( M.GE.MNTHR1 ) THEN
*
            IF( WNTQN ) THEN
*
*              Path 1 (M much larger than N, JOBZ='N')
*              No singular vectors to be computed
*
               ITAU = 1
               NWORK = ITAU + N
*
*              Compute A=Q*R
*              (CWorkspace: need 2*N, prefer N+N*NB)
*              (RWorkspace: need 0)
*
               CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Zero out below R
*
               CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO, A( 2, 1 ),
     $                      LDA )
               IE = 1
               ITAUQ = 1
               ITAUP = ITAUQ + N
               NWORK = ITAUP + N
*
*              Bidiagonalize R in A
*              (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
*              (RWorkspace: need N)
*
               CALL ZGEBRD( N, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                      IERR )
               NRWORK = IE + N
*
```

```
*              Perform bidiagonal SVD, compute singular values only
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAN)
*
               CALL DBDSDC( 'U', 'N', N, S, RWORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, RWORK( NRWORK ), IWORK, INFO )
*
            ELSE IF( WNTQO ) THEN
*
*              Path 2 (M much larger than N, JOBZ='O')
*              N left singular vectors to be overwritten on A and
*              N right singular vectors to be computed in VT
*
               IU = 1
*
*              WORK(IU) is N by N
*
               LDWRKU = N
               IR = IU + LDWRKU*N
               IF( LWORK.GE.M*N+N*N+3*N ) THEN
*
*                 WORK(IR) is M by N
*
                  LDWRKR = M
               ELSE
                  LDWRKR = ( LWORK-N*N-3*N ) / N
               END IF
               ITAU = IR + LDWRKR*N
               NWORK = ITAU + N
*
*              Compute A=Q*R
*              (CWorkspace: need N*N+2*N, prefer M*N+N+N*NB)
*              (RWorkspace: 0)
*
               CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy R to WORK( IR ), zeroing out below it
*
               CALL ZLACPY( 'U', N, N, A, LDA, WORK( IR ), LDWRKR )
               CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO, WORK( IR+1 ),
     $                      LDWRKR )
*
*              Generate Q in A
*              (CWorkspace: need 2*N, prefer N+N*NB)
*              (RWorkspace: 0)
*
               CALL ZUNGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
               IE = 1
```

```
               ITAUQ = ITAU
               ITAUP = ITAUQ + N
               NWORK = ITAUP + N
*
*              Bidiagonalize R in WORK(IR)
*              (CWorkspace: need N*N+3*N, prefer M*N+2*N+2*N*NB)
*              (RWorkspace: need N)
*
               CALL ZGEBRD( N, N, WORK( IR ), LDWRKR, S, RWORK( IE ),
     $                      WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of R in WORK(IRU) and computing right singular vectors
*              of R in WORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               IRU = IE + N
               IRVT = IRU + N*N
               NRWORK = IRVT + N*N
               CALL DBDSDC( 'U', 'I', N, S, RWORK( IE ), RWORK( IRU ),
     $                      N, RWORK( IRVT ), N, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Copy real matrix RWORK(IRU) to complex matrix WORK(IU)
*              Overwrite WORK(IU) by the left singular vectors of R
*              (CWorkspace: need 2*N*N+3*N, prefer M*N+N*N+2*N+N*NB)
*              (RWorkspace: 0)
*
               CALL ZLACP2( 'F', N, N, RWORK( IRU ), N, WORK( IU ),
     $                      LDWRKU )
               CALL ZUNMBR( 'Q', 'L', 'N', N, N, N, WORK( IR ), LDWRKR,
     $                      WORK( ITAUQ ), WORK( IU ), LDWRKU,
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Copy real matrix RWORK(IRVT) to complex matrix VT
*              Overwrite VT by the right singular vectors of R
*              (CWorkspace: need N*N+3*N, prefer M*N+2*N+N*NB)
*              (RWorkspace: 0)
*
               CALL ZLACP2( 'F', N, N, RWORK( IRVT ), N, VT, LDVT )
               CALL ZUNMBR( 'P', 'R', 'C', N, N, N, WORK( IR ), LDWRKR,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Multiply Q in A by left singular vectors of R in
*              WORK(IU), storing result in WORK(IR) and copying to A
*              (CWorkspace: need 2*N*N, prefer N*N+M*N)
*              (RWorkspace: 0)
```

```
*
               DO 10 I = 1, M, LDWRKR
                  CHUNK = MIN( M-I+1, LDWRKR )
                  CALL ZGEMM( 'N', 'N', CHUNK, N, N, CONE, A( I, 1 ),
     $                        LDA, WORK( IU ), LDWRKU, CZERO,
     $                        WORK( IR ), LDWRKR )
                  CALL ZLACPY( 'F', CHUNK, N, WORK( IR ), LDWRKR,
     $                         A( I, 1 ), LDA )
   10          CONTINUE
*
            ELSE IF( WNTQS ) THEN
*
               Path 3 (M much larger than N, JOBZ='S')
*              N left singular vectors to be computed in U and
*              N right singular vectors to be computed in VT
*
               IR = 1
*
*              WORK(IR) is N by N
*
               LDWRKR = N
               ITAU = IR + LDWRKR*N
               NWORK = ITAU + N
*
*              Compute A=Q*R
*              (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*              (RWorkspace: 0)
*
               CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy R to WORK(IR), zeroing out below it
*
               CALL ZLACPY( 'U', N, N, A, LDA, WORK( IR ), LDWRKR )
               CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO, WORK( IR+1 ),
     $                      LDWRKR )
*
*              Generate Q in A
*              (CWorkspace: need 2*N, prefer N+N*NB)
*              (RWorkspace: 0)
*
               CALL ZUNGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
               IE = 1
               ITAUQ = ITAU
               ITAUP = ITAUQ + N
               NWORK = ITAUP + N
*
*              Bidiagonalize R in WORK(IR)
*              (CWorkspace: need N*N+3*N, prefer N*N+2*N+2*N*NB)
```

```
*               (RWorkspace: need N)
*
                CALL ZGEBRD( N, N, WORK( IR ), LDWRKR, S, RWORK( IE ),
     $                       WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                       LWORK-NWORK+1, IERR )
*
*               Perform bidiagonal SVD, computing left singular vectors
*               of bidiagonal matrix in RWORK(IRU) and computing right
*               singular vectors of bidiagonal matrix in RWORK(IRVT)
*               (CWorkspace: need 0)
*               (RWorkspace: need BDSPAC)
*
                IRU = IE + N
                IRVT = IRU + N*N
                NRWORK = IRVT + N*N
                CALL DBDSDC( 'U', 'I', N, S, RWORK( IE ), RWORK( IRU ),
     $                       N, RWORK( IRVT ), N, DUM, IDUM,
     $                       RWORK( NRWORK ), IWORK, INFO )
*
*               Copy real matrix RWORK(IRU) to complex matrix U
*               Overwrite U by left singular vectors of R
*               (CWorkspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*               (RWorkspace: 0)
*
                CALL ZLACP2( 'F', N, N, RWORK( IRU ), N, U, LDU )
                CALL ZUNMBR( 'Q', 'L', 'N', N, N, N, WORK( IR ), LDWRKR,
     $                       WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                       LWORK-NWORK+1, IERR )
*
*               Copy real matrix RWORK(IRVT) to complex matrix VT
*               Overwrite VT by right singular vectors of R
*               (CWorkspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*               (RWorkspace: 0)
*
                CALL ZLACP2( 'F', N, N, RWORK( IRVT ), N, VT, LDVT )
                CALL ZUNMBR( 'P', 'R', 'C', N, N, N, WORK( IR ), LDWRKR,
     $                       WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                       LWORK-NWORK+1, IERR )
*
*               Multiply Q in A by left singular vectors of R in
*               WORK(IR), storing result in U
*               (CWorkspace: need N*N)
*               (RWorkspace: 0)
*
                CALL ZLACPY( 'F', N, N, U, LDU, WORK( IR ), LDWRKR )
                CALL ZGEMM( 'N', 'N', M, N, N, CONE, A, LDA, WORK( IR ),
     $                      LDWRKR, CZERO, U, LDU )
*
            ELSE IF( WNTQA ) THEN
*
```

```
*             Path 4 (M much larger than N, JOBZ='A')
*             M left singular vectors to be computed in U and
*             N right singular vectors to be computed in VT
*
              IU = 1
*
*             WORK(IU) is N by N
*
              LDWRKU = N
              ITAU = IU + LDWRKU*N
              NWORK = ITAU + N
*
*             Compute A=Q*R, copying result to U
*             (CWorkspace: need 2*N, prefer N+N*NB)
*             (RWorkspace: 0)
*
              CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
              CALL ZLACPY( 'L', M, N, A, LDA, U, LDU )
*
*             Generate Q in U
*             (CWorkspace: need N+M, prefer N+M*NB)
*             (RWorkspace: 0)
*
              CALL ZUNGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*             Produce R in A, zeroing out below it
*
              CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO, A( 2, 1 ),
     $                     LDA )
              IE = 1
              ITAUQ = ITAU
              ITAUP = ITAUQ + N
              NWORK = ITAUP + N
*
*             Bidiagonalize R in A
*             (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
*             (RWorkspace: need N)
*
              CALL ZGEBRD( N, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                     WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                     IERR )
              IRU = IE + N
              IRVT = IRU + N*N
              NRWORK = IRVT + N*N
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in RWORK(IRU) and computing right
*             singular vectors of bidiagonal matrix in RWORK(IRVT)
```

```
*             (CWorkspace: need 0)
*             (RWorkspace: need BDSPAC)
*
              CALL DBDSDC( 'U', 'I', N, S, RWORK( IE ), RWORK( IRU ),
     $                     N, RWORK( IRVT ), N, DUM, IDUM,
     $                     RWORK( NRWORK ), IWORK, INFO )
*
*             Copy real matrix RWORK(IRU) to complex matrix WORK(IU)
*             Overwrite WORK(IU) by left singular vectors of R
*             (CWorkspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*             (RWorkspace: 0)
*
              CALL ZLACP2( 'F', N, N, RWORK( IRU ), N, WORK( IU ),
     $                     LDWRKU )
              CALL ZUNMBR( 'Q', 'L', 'N', N, N, N, A, LDA,
     $                     WORK( ITAUQ ), WORK( IU ), LDWRKU,
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*             Copy real matrix RWORK(IRVT) to complex matrix VT
*             Overwrite VT by right singular vectors of R
*             (CWorkspace: need 3*N, prefer 2*N+N*NB)
*             (RWorkspace: 0)
*
              CALL ZLACP2( 'F', N, N, RWORK( IRVT ), N, VT, LDVT )
              CALL ZUNMBR( 'P', 'R', 'C', N, N, N, A, LDA,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*             Multiply Q in U by left singular vectors of R in
*             WORK(IU), storing result in A
*             (CWorkspace: need N*N)
*             (RWorkspace: 0)
*
              CALL ZGEMM( 'N', 'N', M, N, N, CONE, U, LDU, WORK( IU ),
     $                    LDWRKU, CZERO, A, LDA )
*
*             Copy left singular vectors of A from A to U
*
              CALL ZLACPY( 'F', M, N, A, LDA, U, LDU )
*
           END IF
*
        ELSE IF( M.GE.MNTHR2 ) THEN
*
           MNTHR2 <= M < MNTHR1
*
           Path 5 (M much larger than N, but not as much as MNTHR1)
           Reduce to bidiagonal form without QR decomposition, use
           ZUNGBR and matrix multiplication to compute singular vectors
*
```

```
            IE = 1
            NRWORK = IE + N
            ITAUQ = 1
            ITAUP = ITAUQ + N
            NWORK = ITAUP + N
*
*           Bidiagonalize A
*           (CWorkspace: need 2*N+M, prefer 2*N+(M+N)*NB)
*           (RWorkspace: need N)
*
            CALL ZGEBRD( M, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                   WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                   IERR )
            IF( WNTQN ) THEN
*
*              Compute singular values only
*              (Cworkspace: 0)
*              (Rworkspace: need BDSPAN)
*
               CALL DBDSDC( 'U', 'N', N, S, RWORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, RWORK( NRWORK ), IWORK, INFO )
            ELSE IF( WNTQO ) THEN
               IU = NWORK
               IRU = NRWORK
               IRVT = IRU + N*N
               NRWORK = IRVT + N*N
*
*              Copy A to VT, generate P**H
*              (Cworkspace: need 2*N, prefer N+N*NB)
*              (Rworkspace: 0)
*
               CALL ZLACPY( 'U', N, N, A, LDA, VT, LDVT )
               CALL ZUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Generate Q in A
*              (CWorkspace: need 2*N, prefer N+N*NB)
*              (RWorkspace: 0)
*
               CALL ZUNGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
               IF( LWORK.GE.M*N+3*N ) THEN
*
*                 WORK( IU ) is M by N
*
                  LDWRKU = M
               ELSE
*
*                 WORK(IU) is LDWRKU by N
```

```
*
                 LDWRKU = ( LWORK-3*N ) / N
              END IF
              NWORK = IU + LDWRKU*N
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in RWORK(IRU) and computing right
*             singular vectors of bidiagonal matrix in RWORK(IRVT)
*             (CWorkspace: need 0)
*             (RWorkspace: need BDSPAC)
*
              CALL DBDSDC( 'U', 'I', N, S, RWORK( IE ), RWORK( IRU ),
     $                     N, RWORK( IRVT ), N, DUM, IDUM,
     $                     RWORK( NRWORK ), IWORK, INFO )
*
*             Multiply real matrix RWORK(IRVT) by P**H in VT,
*             storing the result in WORK(IU), copying to VT
*             (Cworkspace: need 0)
*             (Rworkspace: need 3*N*N)
*
              CALL ZLARCM( N, N, RWORK( IRVT ), N, VT, LDVT,
     $                     WORK( IU ), LDWRKU, RWORK( NRWORK ) )
              CALL ZLACPY( 'F', N, N, WORK( IU ), LDWRKU, VT, LDVT )
*
*             Multiply Q in A by real matrix RWORK(IRU), storing the
*             result in WORK(IU), copying to A
*             (CWorkspace: need N*N, prefer M*N)
*             (Rworkspace: need 3*N*N, prefer N*N+2*M*N)
*
              NRWORK = IRVT
              DO 20 I = 1, M, LDWRKU
                 CHUNK = MIN( M-I+1, LDWRKU )
                 CALL ZLACRM( CHUNK, N, A( I, 1 ), LDA, RWORK( IRU ),
     $                        N, WORK( IU ), LDWRKU, RWORK( NRWORK ) )
                 CALL ZLACPY( 'F', CHUNK, N, WORK( IU ), LDWRKU,
     $                        A( I, 1 ), LDA )
   20         CONTINUE
*
           ELSE IF( WNTQS ) THEN
*
              Copy A to VT, generate P**H
*             (Cworkspace: need 2*N, prefer N+N*NB)
*             (Rworkspace: 0)
*
              CALL ZLACPY( 'U', N, N, A, LDA, VT, LDVT )
              CALL ZUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*             Copy A to U, generate Q
*             (Cworkspace: need 2*N, prefer N+N*NB)
```

```
*              (Rworkspace: 0)
*
               CALL ZLACPY( 'L', M, N, A, LDA, U, LDU )
               CALL ZUNGBR( 'Q', M, N, N, U, LDU, WORK( ITAUQ ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               IRU = NRWORK
               IRVT = IRU + N*N
               NRWORK = IRVT + N*N
               CALL DBDSDC( 'U', 'I', N, S, RWORK( IE ), RWORK( IRU ),
     $                      N, RWORK( IRVT ), N, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Multiply real matrix RWORK(IRVT) by P**H in VT,
*              storing the result in A, copying to VT
*              (Cworkspace: need 0)
*              (Rworkspace: need 3*N*N)
*
               CALL ZLARCM( N, N, RWORK( IRVT ), N, VT, LDVT, A, LDA,
     $                      RWORK( NRWORK ) )
               CALL ZLACPY( 'F', N, N, A, LDA, VT, LDVT )
*
*              Multiply Q in U by real matrix RWORK(IRU), storing the
*              result in A, copying to U
*              (CWorkspace: need 0)
*              (Rworkspace: need N*N+2*M*N)
*
               NRWORK = IRVT
               CALL ZLACRM( M, N, U, LDU, RWORK( IRU ), N, A, LDA,
     $                      RWORK( NRWORK ) )
               CALL ZLACPY( 'F', M, N, A, LDA, U, LDU )
            ELSE
*
*              Copy A to VT, generate P**H
*              (Cworkspace: need 2*N, prefer N+N*NB)
*              (Rworkspace: 0)
*
               CALL ZLACPY( 'U', N, N, A, LDA, VT, LDVT )
               CALL ZUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Copy A to U, generate Q
*              (Cworkspace: need 2*N, prefer N+N*NB)
*              (Rworkspace: 0)
```

```
*
               CALL ZLACPY( 'L', M, N, A, LDA, U, LDU )
               CALL ZUNGBR( 'Q', M, M, N, U, LDU, WORK( ITAUQ ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               IRU = NRWORK
               IRVT = IRU + N*N
               NRWORK = IRVT + N*N
               CALL DBDSDC( 'U', 'I', N, S, RWORK( IE ), RWORK( IRU ),
     $                      N, RWORK( IRVT ), N, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Multiply real matrix RWORK(IRVT) by P**H in VT,
*              storing the result in A, copying to VT
*              (Cworkspace: need 0)
*              (Rworkspace: need 3*N*N)
*
               CALL ZLARCM( N, N, RWORK( IRVT ), N, VT, LDVT, A, LDA,
     $                      RWORK( NRWORK ) )
               CALL ZLACPY( 'F', N, N, A, LDA, VT, LDVT )
*
*              Multiply Q in U by real matrix RWORK(IRU), storing the
*              result in A, copying to U
*              (CWorkspace: 0)
*              (Rworkspace: need 3*N*N)
*
               NRWORK = IRVT
               CALL ZLACRM( M, N, U, LDU, RWORK( IRU ), N, A, LDA,
     $                      RWORK( NRWORK ) )
               CALL ZLACPY( 'F', M, N, A, LDA, U, LDU )
            END IF
*
         ELSE
*
*           M .LT. MNTHR2
*
*           Path 6 (M at least N, but not much larger)
*           Reduce to bidiagonal form without QR decomposition
*           Use ZUNMBR to compute singular vectors
*
            IE = 1
            NRWORK = IE + N
            ITAUQ = 1
            ITAUP = ITAUQ + N
```

```
            NWORK = ITAUP + N
*
*           Bidiagonalize A
*           (CWorkspace: need 2*N+M, prefer 2*N+(M+N)*NB)
*           (RWorkspace: need N)
*
            CALL ZGEBRD( M, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                   WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                   IERR )
            IF( WNTQN ) THEN
*
*              Compute singular values only
*              (Cworkspace: 0)
*              (Rworkspace: need BDSPAN)
*
               CALL DBDSDC( 'U', 'N', N, S, RWORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, RWORK( NRWORK ), IWORK, INFO )
            ELSE IF( WNTQO ) THEN
               IU = NWORK
               IRU = NRWORK
               IRVT = IRU + N*N
               NRWORK = IRVT + N*N
               IF( LWORK.GE.M*N+3*N ) THEN
*
*                 WORK( IU ) is M by N
*
                  LDWRKU = M
               ELSE
*
*                 WORK( IU ) is LDWRKU by N
*
                  LDWRKU = ( LWORK-3*N ) / N
               END IF
               NWORK = IU + LDWRKU*N
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               CALL DBDSDC( 'U', 'I', N, S, RWORK( IE ), RWORK( IRU ),
     $                      N, RWORK( IRVT ), N, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Copy real matrix RWORK(IRVT) to complex matrix VT
*              Overwrite VT by right singular vectors of A
*              (Cworkspace: need 2*N, prefer N+N*NB)
*              (Rworkspace: need 0)
*
```

```
               CALL ZLACP2( 'F', N, N, RWORK( IRVT ), N, VT, LDVT )
               CALL ZUNMBR( 'P', 'R', 'C', N, N, N, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
               IF( LWORK.GE.M*N+3*N ) THEN
*
*              Copy real matrix RWORK(IRU) to complex matrix WORK(IU)
*              Overwrite WORK(IU) by left singular vectors of A, copying
*              to A
*              (Cworkspace: need M*N+2*N, prefer M*N+N+N*NB)
*              (Rworkspace: need 0)
*
                  CALL ZLASET( 'F', M, N, CZERO, CZERO, WORK( IU ),
     $                         LDWRKU )
                  CALL ZLACP2( 'F', N, N, RWORK( IRU ), N, WORK( IU ),
     $                         LDWRKU )
                  CALL ZUNMBR( 'Q', 'L', 'N', M, N, N, A, LDA,
     $                         WORK( ITAUQ ), WORK( IU ), LDWRKU,
     $                         WORK( NWORK ), LWORK-NWORK+1, IERR )
                  CALL ZLACPY( 'F', M, N, WORK( IU ), LDWRKU, A, LDA )
               ELSE
*
*                 Generate Q in A
*                 (Cworkspace: need 2*N, prefer N+N*NB)
*                 (Rworkspace: need 0)
*
                  CALL ZUNGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
     $                         WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                 Multiply Q in A by real matrix RWORK(IRU), storing the
*                 result in WORK(IU), copying to A
*                 (CWorkspace: need N*N, prefer M*N)
*                 (Rworkspace: need 3*N*N, prefer N*N+2*M*N)
*
                  NRWORK = IRVT
                  DO 30 I = 1, M, LDWRKU
                     CHUNK = MIN( M-I+1, LDWRKU )
                     CALL ZLACRM( CHUNK, N, A( I, 1 ), LDA,
     $                            RWORK( IRU ), N, WORK( IU ), LDWRKU,
     $                            RWORK( NRWORK ) )
                     CALL ZLACPY( 'F', CHUNK, N, WORK( IU ), LDWRKU,
     $                            A( I, 1 ), LDA )
   30             CONTINUE
               END IF
*
            ELSE IF( WNTQS ) THEN
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
```

```
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               IRU = NRWORK
               IRVT = IRU + N*N
               NRWORK = IRVT + N*N
               CALL DBDSDC( 'U', 'I', N, S, RWORK( IE ), RWORK( IRU ),
     $                      N, RWORK( IRVT ), N, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Copy real matrix RWORK(IRU) to complex matrix U
*              Overwrite U by left singular vectors of A
*              (CWorkspace: need 3*N, prefer 2*N+N*NB)
*              (RWorkspace: 0)
*
               CALL ZLASET( 'F', M, N, CZERO, CZERO, U, LDU )
               CALL ZLACP2( 'F', N, N, RWORK( IRU ), N, U, LDU )
               CALL ZUNMBR( 'Q', 'L', 'N', M, N, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy real matrix RWORK(IRVT) to complex matrix VT
*              Overwrite VT by right singular vectors of A
*              (CWorkspace: need 3*N, prefer 2*N+N*NB)
*              (RWorkspace: 0)
*
               CALL ZLACP2( 'F', N, N, RWORK( IRVT ), N, VT, LDVT )
               CALL ZUNMBR( 'P', 'R', 'C', N, N, N, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
            ELSE
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               IRU = NRWORK
               IRVT = IRU + N*N
               NRWORK = IRVT + N*N
               CALL DBDSDC( 'U', 'I', N, S, RWORK( IE ), RWORK( IRU ),
     $                      N, RWORK( IRVT ), N, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Set the right corner of U to identity matrix
*
               CALL ZLASET( 'F', M, M, CZERO, CZERO, U, LDU )
               IF( M.GT.N ) THEN
```

```
                 CALL ZLASET( 'F', M-N, M-N, CZERO, CONE,
     $                        U( N+1, N+1 ), LDU )
              END IF
*
*             Copy real matrix RWORK(IRU) to complex matrix U
*             Overwrite U by left singular vectors of A
*             (CWorkspace: need 2*N+M, prefer 2*N+M*NB)
*             (RWorkspace: 0)
*
              CALL ZLACP2( 'F', N, N, RWORK( IRU ), N, U, LDU )
              CALL ZUNMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*             Copy real matrix RWORK(IRVT) to complex matrix VT
*             Overwrite VT by right singular vectors of A
*             (CWorkspace: need 3*N, prefer 2*N+N*NB)
*             (RWorkspace: 0)
*
              CALL ZLACP2( 'F', N, N, RWORK( IRVT ), N, VT, LDVT )
              CALL ZUNMBR( 'P', 'R', 'C', N, N, N, A, LDA,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
           END IF
*
        END IF
*
      ELSE
*
        A has more columns than rows. If A has sufficiently more
*       columns than rows, first reduce using the LQ decomposition (if
*       sufficient workspace available)
*
        IF( N.GE.MNTHR1 ) THEN
*
           IF( WNTQN ) THEN
*
              Path 1t (N much larger than M, JOBZ='N')
*             No singular vectors to be computed
*
              ITAU = 1
              NWORK = ITAU + M
*
*             Compute A=L*Q
*             (CWorkspace: need 2*M, prefer M+M*NB)
*             (RWorkspace: 0)
*
              CALL ZGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
```

```
*              Zero out above L
*
               CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO, A( 1, 2 ),
     $                      LDA )
               IE = 1
               ITAUQ = 1
               ITAUP = ITAUQ + M
               NWORK = ITAUP + M
*
*              Bidiagonalize L in A
*              (CWorkspace: need 3*M, prefer 2*M+2*M*NB)
*              (RWorkspace: need M)
*
               CALL ZGEBRD( M, M, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                      IERR )
               NRWORK = IE + M
*
*              Perform bidiagonal SVD, compute singular values only
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAN)
*
               CALL DBDSDC( 'U', 'N', M, S, RWORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, RWORK( NRWORK ), IWORK, INFO )
*
            ELSE IF( WNTQO ) THEN
*
*              Path 2t (N much larger than M, JOBZ='O')
*              M right singular vectors to be overwritten on A and
*              M left singular vectors to be computed in U
*
               IVT = 1
               LDWKVT = M
*
*              WORK(IVT) is M by M
*
               IL = IVT + LDWKVT*M
               IF( LWORK.GE.M*N+M*M+3*M ) THEN
*
*                 WORK(IL) M by N
*
                  LDWRKL = M
                  CHUNK = N
               ELSE
*
*                 WORK(IL) is M by CHUNK
*
                  LDWRKL = M
                  CHUNK = ( LWORK-M*M-3*M ) / M
               END IF
```

```
               ITAU = IL + LDWRKL*CHUNK
               NWORK = ITAU + M
*
*              Compute A=L*Q
*              (CWorkspace: need 2*M, prefer M+M*NB)
*              (RWorkspace: 0)
*
               CALL ZGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy L to WORK(IL), zeroing about above it
*
               CALL ZLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWRKL )
               CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                      WORK( IL+LDWRKL ), LDWRKL )
*
*              Generate Q in A
*              (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*              (RWorkspace: 0)
*
               CALL ZUNGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
               IE = 1
               ITAUQ = ITAU
               ITAUP = ITAUQ + M
               NWORK = ITAUP + M
*
*              Bidiagonalize L in WORK(IL)
*              (CWorkspace: need M*M+3*M, prefer M*M+2*M+2*M*NB)
*              (RWorkspace: need M)
*
               CALL ZGEBRD( M, M, WORK( IL ), LDWRKL, S, RWORK( IE ),
     $                      WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               IRU = IE + M
               IRVT = IRU + M*M
               NRWORK = IRVT + M*M
               CALL DBDSDC( 'U', 'I', M, S, RWORK( IE ), RWORK( IRU ),
     $                      M, RWORK( IRVT ), M, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Copy real matrix RWORK(IRU) to complex matrix WORK(IU)
*              Overwrite WORK(IU) by the left singular vectors of L
```

```
*              (CWorkspace: need N*N+3*N, prefer M*N+2*N+N*NB)
*              (RWorkspace: 0)
*
               CALL ZLACP2( 'F', M, M, RWORK( IRU ), M, U, LDU )
               CALL ZUNMBR( 'Q', 'L', 'N', M, M, M, WORK( IL ), LDWRKL,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy real matrix RWORK(IRVT) to complex matrix WORK(IVT)
*              Overwrite WORK(IVT) by the right singular vectors of L
*              (CWorkspace: need N*N+3*N, prefer M*N+2*N+N*NB)
*              (RWorkspace: 0)
*
               CALL ZLACP2( 'F', M, M, RWORK( IRVT ), M, WORK( IVT ),
     $                      LDWKVT )
               CALL ZUNMBR( 'P', 'R', 'C', M, M, M, WORK( IL ), LDWRKL,
     $                      WORK( ITAUP ), WORK( IVT ), LDWKVT,
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Multiply right singular vectors of L in WORK(IL) by Q
*              in A, storing result in WORK(IL) and copying to A
*              (CWorkspace: need 2*M*M, prefer M*M+M*N))
*              (RWorkspace: 0)
*
               DO 40 I = 1, N, CHUNK
                  BLK = MIN( N-I+1, CHUNK )
                  CALL ZGEMM( 'N', 'N', M, BLK, M, CONE, WORK( IVT ), M,
     $                        A( 1, I ), LDA, CZERO, WORK( IL ),
     $                        LDWRKL )
                  CALL ZLACPY( 'F', M, BLK, WORK( IL ), LDWRKL,
     $                         A( 1, I ), LDA )
   40          CONTINUE
*
            ELSE IF( WNTQS ) THEN
*
              Path 3t (N much larger than M, JOBZ='S')
*             M right singular vectors to be computed in VT and
*             M left singular vectors to be computed in U
*
               IL = 1
*
*              WORK(IL) is M by M
*
               LDWRKL = M
               ITAU = IL + LDWRKL*M
               NWORK = ITAU + M
*
*              Compute A=L*Q
*              (CWorkspace: need 2*M, prefer M+M*NB)
*              (RWorkspace: 0)
```

```
*
               CALL ZGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy L to WORK(IL), zeroing out above it
*
               CALL ZLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWRKL )
               CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                      WORK( IL+LDWRKL ), LDWRKL )
*
*              Generate Q in A
*              (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*              (RWorkspace: 0)
*
               CALL ZUNGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
               IE = 1
               ITAUQ = ITAU
               ITAUP = ITAUQ + M
               NWORK = ITAUP + M
*
*              Bidiagonalize L in WORK(IL)
*              (CWorkspace: need M*M+3*M, prefer M*M+2*M+2*M*NB)
*              (RWorkspace: need M)
*
               CALL ZGEBRD( M, M, WORK( IL ), LDWRKL, S, RWORK( IE ),
     $                      WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               IRU = IE + M
               IRVT = IRU + M*M
               NRWORK = IRVT + M*M
               CALL DBDSDC( 'U', 'I', M, S, RWORK( IE ), RWORK( IRU ),
     $                      M, RWORK( IRVT ), M, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Copy real matrix RWORK(IRU) to complex matrix U
*              Overwrite U by left singular vectors of L
*              (CWorkspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*              (RWorkspace: 0)
*
               CALL ZLACP2( 'F', M, M, RWORK( IRU ), M, U, LDU )
               CALL ZUNMBR( 'Q', 'L', 'N', M, M, M, WORK( IL ), LDWRKL,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
```

```
     $                     LWORK-NWORK+1, IERR )
*
*             Copy real matrix RWORK(IRVT) to complex matrix VT
*             Overwrite VT by left singular vectors of L
*             (CWorkspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*             (RWorkspace: 0)
*
              CALL ZLACP2( 'F', M, M, RWORK( IRVT ), M, VT, LDVT )
              CALL ZUNMBR( 'P', 'R', 'C', M, M, M, WORK( IL ), LDWRKL,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*             Copy VT to WORK(IL), multiply right singular vectors of L
*             in WORK(IL) by Q in A, storing result in VT
*             (CWorkspace: need M*M)
*             (RWorkspace: 0)
*
              CALL ZLACPY( 'F', M, M, VT, LDVT, WORK( IL ), LDWRKL )
              CALL ZGEMM( 'N', 'N', M, N, M, CONE, WORK( IL ), LDWRKL,
     $                    A, LDA, CZERO, VT, LDVT )
*
           ELSE IF( WNTQA ) THEN
*
*             Path 9t (N much larger than M, JOBZ='A')
*             N right singular vectors to be computed in VT and
*             M left singular vectors to be computed in U
*
              IVT = 1
*
*             WORK(IVT) is M by M
*
              LDWKVT = M
              ITAU = IVT + LDWKVT*M
              NWORK = ITAU + M
*
*             Compute A=L*Q, copying result to VT
*             (CWorkspace: need 2*M, prefer M+M*NB)
*             (RWorkspace: 0)
*
              CALL ZGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
              CALL ZLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*             Generate Q in VT
*             (CWorkspace: need M+N, prefer M+N*NB)
*             (RWorkspace: 0)
*
              CALL ZUNGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
*
```

```
*               Produce L in A, zeroing out above it
*
                CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO, A( 1, 2 ),
     $                       LDA )
                IE = 1
                ITAUQ = ITAU
                ITAUP = ITAUQ + M
                NWORK = ITAUP + M
*
*               Bidiagonalize L in A
*               (CWorkspace: need M*M+3*M, prefer M*M+2*M+2*M*NB)
*               (RWorkspace: need M)
*
                CALL ZGEBRD( M, M, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                       WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                       IERR )
*
*               Perform bidiagonal SVD, computing left singular vectors
*               of bidiagonal matrix in RWORK(IRU) and computing right
*               singular vectors of bidiagonal matrix in RWORK(IRVT)
*               (CWorkspace: need 0)
*               (RWorkspace: need BDSPAC)
*
                IRU = IE + M
                IRVT = IRU + M*M
                NRWORK = IRVT + M*M
                CALL DBDSDC( 'U', 'I', M, S, RWORK( IE ), RWORK( IRU ),
     $                       M, RWORK( IRVT ), M, DUM, IDUM,
     $                       RWORK( NRWORK ), IWORK, INFO )
*
*               Copy real matrix RWORK(IRU) to complex matrix U
*               Overwrite U by left singular vectors of L
*               (CWorkspace: need 3*M, prefer 2*M+M*NB)
*               (RWorkspace: 0)
*
                CALL ZLACP2( 'F', M, M, RWORK( IRU ), M, U, LDU )
                CALL ZUNMBR( 'Q', 'L', 'N', M, M, M, A, LDA,
     $                       WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                       LWORK-NWORK+1, IERR )
*
*               Copy real matrix RWORK(IRVT) to complex matrix WORK(IVT)
*               Overwrite WORK(IVT) by right singular vectors of L
*               (CWorkspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*               (RWorkspace: 0)
*
                CALL ZLACP2( 'F', M, M, RWORK( IRVT ), M, WORK( IVT ),
     $                       LDWKVT )
                CALL ZUNMBR( 'P', 'R', 'C', M, M, M, A, LDA,
     $                       WORK( ITAUP ), WORK( IVT ), LDWKVT,
     $                       WORK( NWORK ), LWORK-NWORK+1, IERR )
```

```
*
*             Multiply right singular vectors of L in WORK(IVT) by
*             Q in VT, storing result in A
*             (CWorkspace: need M*M)
*             (RWorkspace: 0)
*
              CALL ZGEMM( 'N', 'N', M, N, M, CONE, WORK( IVT ), LDWKVT,
     $                    VT, LDVT, CZERO, A, LDA )
*
*             Copy right singular vectors of A from A to VT
*
              CALL ZLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
           END IF
*
        ELSE IF( N.GE.MNTHR2 ) THEN
*
*          MNTHR2 <= N < MNTHR1
*
*          Path 5t (N much larger than M, but not as much as MNTHR1)
*          Reduce to bidiagonal form without QR decomposition, use
*          ZUNGBR and matrix multiplication to compute singular vectors
*
*
           IE = 1
           NRWORK = IE + M
           ITAUQ = 1
           ITAUP = ITAUQ + M
           NWORK = ITAUP + M
*
*          Bidiagonalize A
*          (CWorkspace: need 2*M+N, prefer 2*M+(M+N)*NB)
*          (RWorkspace: M)
*
           CALL ZGEBRD( M, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                  WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                  IERR )
*
           IF( WNTQN ) THEN
*
*             Compute singular values only
*             (Cworkspace: 0)
*             (Rworkspace: need BDSPAN)
*
              CALL DBDSDC( 'L', 'N', M, S, RWORK( IE ), DUM, 1, DUM, 1,
     $                     DUM, IDUM, RWORK( NRWORK ), IWORK, INFO )
           ELSE IF( WNTQO ) THEN
              IRVT = NRWORK
              IRU = IRVT + M*M
              NRWORK = IRU + M*M
```

```
               IVT = NWORK
*
*              Copy A to U, generate Q
*              (Cworkspace: need 2*M, prefer M+M*NB)
*              (Rworkspace: 0)
*
               CALL ZLACPY( 'L', M, M, A, LDA, U, LDU )
               CALL ZUNGBR( 'Q', M, M, N, U, LDU, WORK( ITAUQ ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Generate P**H in A
*              (Cworkspace: need 2*M, prefer M+M*NB)
*              (Rworkspace: 0)
*
               CALL ZUNGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
               LDWKVT = M
               IF( LWORK.GE.M*N+3*M ) THEN
*
*                 WORK( IVT ) is M by N
*
                  NWORK = IVT + LDWKVT*N
                  CHUNK = N
               ELSE
*
*                 WORK( IVT ) is M by CHUNK
*
                  CHUNK = ( LWORK-3*M ) / M
                  NWORK = IVT + LDWKVT*CHUNK
               END IF
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               CALL DBDSDC( 'L', 'I', M, S, RWORK( IE ), RWORK( IRU ),
     $                      M, RWORK( IRVT ), M, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Multiply Q in U by real matrix RWORK(IRVT)
*              storing the result in WORK(IVT), copying to U
*              (Cworkspace: need 0)
*              (Rworkspace: need 2*M*M)
*
               CALL ZLACRM( M, M, U, LDU, RWORK( IRU ), M, WORK( IVT ),
     $                      LDWKVT, RWORK( NRWORK ) )
               CALL ZLACPY( 'F', M, M, WORK( IVT ), LDWKVT, U, LDU )
```

```
*
*              Multiply RWORK(IRVT) by P**H in A, storing the
*              result in WORK(IVT), copying to A
*              (CWorkspace: need M*M, prefer M*N)
*              (Rworkspace: need 2*M*M, prefer 2*M*N)
*
               NRWORK = IRU
               DO 50 I = 1, N, CHUNK
                  BLK = MIN( N-I+1, CHUNK )
                  CALL ZLARCM( M, BLK, RWORK( IRVT ), M, A( 1, I ), LDA,
     $                         WORK( IVT ), LDWKVT, RWORK( NRWORK ) )
                  CALL ZLACPY( 'F', M, BLK, WORK( IVT ), LDWKVT,
     $                         A( 1, I ), LDA )
   50          CONTINUE
            ELSE IF( WNTQS ) THEN
*
*              Copy A to U, generate Q
*              (Cworkspace: need 2*M, prefer M+M*NB)
*              (Rworkspace: 0)
*
               CALL ZLACPY( 'L', M, M, A, LDA, U, LDU )
               CALL ZUNGBR( 'Q', M, M, N, U, LDU, WORK( ITAUQ ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Copy A to VT, generate P**H
*              (Cworkspace: need 2*M, prefer M+M*NB)
*              (Rworkspace: 0)
*
               CALL ZLACPY( 'U', M, N, A, LDA, VT, LDVT )
               CALL ZUNGBR( 'P', M, N, M, VT, LDVT, WORK( ITAUP ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               IRVT = NRWORK
               IRU = IRVT + M*M
               NRWORK = IRU + M*M
               CALL DBDSDC( 'L', 'I', M, S, RWORK( IE ), RWORK( IRU ),
     $                      M, RWORK( IRVT ), M, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Multiply Q in U by real matrix RWORK(IRU), storing the
*              result in A, copying to U
*              (CWorkspace: need 0)
*              (Rworkspace: need 3*M*M)
*
```

```
               CALL ZLACRM( M, M, U, LDU, RWORK( IRU ), M, A, LDA,
     $                      RWORK( NRWORK ) )
               CALL ZLACPY( 'F', M, M, A, LDA, U, LDU )
*
*              Multiply real matrix RWORK(IRVT) by P**H in VT,
*              storing the result in A, copying to VT
*              (Cworkspace: need 0)
*              (Rworkspace: need M*M+2*M*N)
*
               NRWORK = IRU
               CALL ZLARCM( M, N, RWORK( IRVT ), M, VT, LDVT, A, LDA,
     $                      RWORK( NRWORK ) )
               CALL ZLACPY( 'F', M, N, A, LDA, VT, LDVT )
            ELSE
*
*              Copy A to U, generate Q
*              (Cworkspace: need 2*M, prefer M+M*NB)
*              (Rworkspace: 0)
*
               CALL ZLACPY( 'L', M, M, A, LDA, U, LDU )
               CALL ZUNGBR( 'Q', M, M, N, U, LDU, WORK( ITAUQ ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Copy A to VT, generate P**H
*              (Cworkspace: need 2*M, prefer M+M*NB)
*              (Rworkspace: 0)
*
               CALL ZLACPY( 'U', M, N, A, LDA, VT, LDVT )
               CALL ZUNGBR( 'P', N, N, M, VT, LDVT, WORK( ITAUP ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               IRVT = NRWORK
               IRU = IRVT + M*M
               NRWORK = IRU + M*M
               CALL DBDSDC( 'L', 'I', M, S, RWORK( IE ), RWORK( IRU ),
     $                      M, RWORK( IRVT ), M, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Multiply Q in U by real matrix RWORK(IRU), storing the
*              result in A, copying to U
*              (CWorkspace: need 0)
*              (Rworkspace: need 3*M*M)
*
               CALL ZLACRM( M, M, U, LDU, RWORK( IRU ), M, A, LDA,
```

```
     $                     RWORK( NRWORK ) )
               CALL ZLACPY( 'F', M, M, A, LDA, U, LDU )
*
*              Multiply real matrix RWORK(IRVT) by P**H in VT,
*              storing the result in A, copying to VT
*              (Cworkspace: need 0)
*              (Rworkspace: need M*M+2*M*N)
*
               CALL ZLARCM( M, N, RWORK( IRVT ), M, VT, LDVT, A, LDA,
     $                     RWORK( NRWORK ) )
               CALL ZLACPY( 'F', M, N, A, LDA, VT, LDVT )
            END IF
*
         ELSE
*
*           N .LT. MNTHR2
*
*           Path 6t (N greater than M, but not much larger)
*           Reduce to bidiagonal form without LQ decomposition
*           Use ZUNMBR to compute singular vectors
*
            IE = 1
            NRWORK = IE + M
            ITAUQ = 1
            ITAUP = ITAUQ + M
            NWORK = ITAUP + M
*
*           Bidiagonalize A
*           (CWorkspace: need 2*M+N, prefer 2*M+(M+N)*NB)
*           (RWorkspace: M)
*
            CALL ZGEBRD( M, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                   WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                   IERR )
            IF( WNTQN ) THEN
*
               Compute singular values only
               (Cworkspace: 0)
               (Rworkspace: need BDSPAN)
*
               CALL DBDSDC( 'L', 'N', M, S, RWORK( IE ), DUM, 1, DUM, 1,
     $                     DUM, IDUM, RWORK( NRWORK ), IWORK, INFO )
            ELSE IF( WNTQO ) THEN
               LDWKVT = M
               IVT = NWORK
               IF( LWORK.GE.M*N+3*M ) THEN
*
                  WORK( IVT ) is M by N
*
                  CALL ZLASET( 'F', M, N, CZERO, CZERO, WORK( IVT ),
```

```
     $                        LDWKVT )
                 NWORK = IVT + LDWKVT*N
              ELSE
*
*                WORK( IVT ) is M by CHUNK
*
                 CHUNK = ( LWORK-3*M ) / M
                 NWORK = IVT + LDWKVT*CHUNK
              END IF
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in RWORK(IRU) and computing right
*             singular vectors of bidiagonal matrix in RWORK(IRVT)
*             (CWorkspace: need 0)
*             (RWorkspace: need BDSPAC)
*
              IRVT = NRWORK
              IRU = IRVT + M*M
              NRWORK = IRU + M*M
              CALL DBDSDC( 'L', 'I', M, S, RWORK( IE ), RWORK( IRU ),
     $                     M, RWORK( IRVT ), M, DUM, IDUM,
     $                     RWORK( NRWORK ), IWORK, INFO )
*
*             Copy real matrix RWORK(IRU) to complex matrix U
*             Overwrite U by left singular vectors of A
*             (Cworkspace: need 2*M, prefer M+M*NB)
*             (Rworkspace: need 0)
*
              CALL ZLACP2( 'F', M, M, RWORK( IRU ), M, U, LDU )
              CALL ZUNMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
              IF( LWORK.GE.M*N+3*M ) THEN
*
*             Copy real matrix RWORK(IRVT) to complex matrix WORK(IVT)
*             Overwrite WORK(IVT) by right singular vectors of A,
*             copying to A
*             (Cworkspace: need M*N+2*M, prefer M*N+M+M*NB)
*             (Rworkspace: need 0)
*
                 CALL ZLACP2( 'F', M, M, RWORK( IRVT ), M, WORK( IVT ),
     $                        LDWKVT )
                 CALL ZUNMBR( 'P', 'R', 'C', M, N, M, A, LDA,
     $                        WORK( ITAUP ), WORK( IVT ), LDWKVT,
     $                        WORK( NWORK ), LWORK-NWORK+1, IERR )
                 CALL ZLACPY( 'F', M, N, WORK( IVT ), LDWKVT, A, LDA )
              ELSE
*
*                Generate P**H in A
```

```
*                 (Cworkspace: need 2*M, prefer M+M*NB)
*                 (Rworkspace: need 0)
*
                  CALL ZUNGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                         WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                 Multiply Q in A by real matrix RWORK(IRU), storing the
*                 result in WORK(IU), copying to A
*                 (CWorkspace: need M*M, prefer M*N)
*                 (Rworkspace: need 3*M*M, prefer M*M+2*M*N)
*
                  NRWORK = IRU
                  DO 60 I = 1, N, CHUNK
                     BLK = MIN( N-I+1, CHUNK )
                     CALL ZLARCM( M, BLK, RWORK( IRVT ), M, A( 1, I ),
     $                            LDA, WORK( IVT ), LDWKVT,
     $                            RWORK( NRWORK ) )
                     CALL ZLACPY( 'F', M, BLK, WORK( IVT ), LDWKVT,
     $                            A( 1, I ), LDA )
   60             CONTINUE
               END IF
            ELSE IF( WNTQS ) THEN
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               IRVT = NRWORK
               IRU = IRVT + M*M
               NRWORK = IRU + M*M
               CALL DBDSDC( 'L', 'I', M, S, RWORK( IE ), RWORK( IRU ),
     $                      M, RWORK( IRVT ), M, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Copy real matrix RWORK(IRU) to complex matrix U
*              Overwrite U by left singular vectors of A
*              (CWorkspace: need 3*M, prefer 2*M+M*NB)
*              (RWorkspace: M*M)
*
               CALL ZLACP2( 'F', M, M, RWORK( IRU ), M, U, LDU )
               CALL ZUNMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy real matrix RWORK(IRVT) to complex matrix VT
*              Overwrite VT by right singular vectors of A
*              (CWorkspace: need 3*M, prefer 2*M+M*NB)
*              (RWorkspace: M*M)
```

```
*
               CALL ZLASET( 'F', M, N, CZERO, CZERO, VT, LDVT )
               CALL ZLACP2( 'F', M, M, RWORK( IRVT ), M, VT, LDVT )
               CALL ZUNMBR( 'P', 'R', 'C', M, N, M, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
            ELSE
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in RWORK(IRU) and computing right
*              singular vectors of bidiagonal matrix in RWORK(IRVT)
*              (CWorkspace: need 0)
*              (RWorkspace: need BDSPAC)
*
               IRVT = NRWORK
               IRU = IRVT + M*M
               NRWORK = IRU + M*M
*
               CALL DBDSDC( 'L', 'I', M, S, RWORK( IE ), RWORK( IRU ),
     $                      M, RWORK( IRVT ), M, DUM, IDUM,
     $                      RWORK( NRWORK ), IWORK, INFO )
*
*              Copy real matrix RWORK(IRU) to complex matrix U
*              Overwrite U by left singular vectors of A
*              (CWorkspace: need 3*M, prefer 2*M+M*NB)
*              (RWorkspace: M*M)
*
               CALL ZLACP2( 'F', M, M, RWORK( IRU ), M, U, LDU )
               CALL ZUNMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Set all of VT to identity matrix
*
               CALL ZLASET( 'F', N, N, CZERO, CONE, VT, LDVT )
*
*              Copy real matrix RWORK(IRVT) to complex matrix VT
*              Overwrite VT by right singular vectors of A
*              (CWorkspace: need 2*M+N, prefer 2*M+N*NB)
*              (RWorkspace: M*M)
*
               CALL ZLACP2( 'F', M, M, RWORK( IRVT ), M, VT, LDVT )
               CALL ZUNMBR( 'P', 'R', 'C', N, N, M, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
            END IF
*
         END IF
*
      END IF
```

```
*
*     Undo scaling if necessary
*
      IF( ISCL.EQ.1 ) THEN
         IF( ANRM.GT.BIGNUM )
     $      CALL DLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( INFO.NE.0 .AND. ANRM.GT.BIGNUM )
     $      CALL DLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN-1, 1,
     $                   RWORK( IE ), MINMN, IERR )
         IF( ANRM.LT.SMLNUM )
     $      CALL DLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( INFO.NE.0 .AND. ANRM.LT.SMLNUM )
     $      CALL DLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN-1, 1,
     $                   RWORK( IE ), MINMN, IERR )
      END IF
*
*     Return optimal workspace in WORK(1)
*
      WORK( 1 ) = MAXWRK
*
      RETURN
*
*     End of ZGESDD
*
      END
```

———-

31.2.4 lisp code

— **zgesdd.lisp** —

```
(let* ((lquerv (- 1))
       (czero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (cone (f2cl-lib:cmplx 1.0d0 0.0d0))
       (zero 0.0d0)
       (one 1.0d0))
  (declare (type (f2cl-lib:integer4) lquerv)
           (type (f2cl-lib:complex16) czero)
           (type (f2cl-lib:complex16) cone)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable lquerv czero cone zero one))
  (defun zgesdd (jobz m n a lda s u ldu vt ldvt work lwork rwork iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (array double-float (*)) rwork s)
```

```
          (type (array f2cl-lib:complex16 (*)) work vt u a)
          (type (f2cl-lib:integer4) info lwork ldvt ldu lda n m)
          (type (string *) jobz))
  (f2cl-lib:with-multi-array-data
      ((jobz character jobz-%data% jobz-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (u f2cl-lib:complex16 u-%data% u-%offset%)
       (vt f2cl-lib:complex16 vt-%data% vt-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%)
       (s double-float s-%data% s-%offset%)
       (rwork double-float rwork-%data% rwork-%offset%)
       (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
    (prog ((dum (make-array 1 :element-type 'double-float))
          (idum (make-array 1 :element-type 'f2cl-lib:integer4))
          (anrm 0.0d0) (bignum 0.0d0) (eps 0.0d0) (smlnum 0.0d0) (blk 0)
          (chunk 0) (i 0) (ie 0) (ierr 0) (il 0) (ir 0) (iru 0) (irvt 0)
          (iscl 0) (itau 0) (itaup 0) (itauq 0) (iu 0) (ivt 0) (ldwkvt 0)
          (ldwrkl 0) (ldwrkr 0) (ldwrku 0) (maxwrk 0) (minmn 0) (minwrk 0)
          (mnthr1 0) (mnthr2 0) (nrwork 0) (nwork 0) (wrkbl 0) (wntqa nil)
          (wntqas nil) (wntqn nil) (wntqo nil) (wntqs nil))
      (declare (type (array double-float (1)) dum)
               (type (array f2cl-lib:integer4 (1)) idum)
               (type (double-float) anrm bignum eps smlnum)
               (type (f2cl-lib:integer4) blk chunk i ie ierr il ir iru irvt
                iscl itau itaup itauq iu ivt ldwkvt ldwrkl ldwrkr ldwrku
                maxwrk minmn minwrk mnthr1 mnthr2 nrwork nwork wrkbl)
               (type f2cl-lib:logical wntqa wntqas wntqn wntqo wntqs))
      (setf info 0)
      (setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
      (setf mnthr1 (f2cl-lib:int (/ (* minmn 17.0d0) 9.0d0)))
      (setf mnthr2 (f2cl-lib:int (/ (* minmn 5.0d0) 3.0d0)))
      (setf wntqa
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame jobz "A")
                (declare (ignore var-1))
                (when var-0 (setf jobz var-0))
                ret-val))
      (setf wntqs
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame jobz "S")
                (declare (ignore var-1))
                (when var-0 (setf jobz var-0))
                ret-val))
      (setf wntqas (or wntqa wntqs))
      (setf wntqo
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame jobz "O")
                (declare (ignore var-1))
                (when var-0 (setf jobz var-0))
                ret-val))
```

```
(setf wntqn
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobz "N")
          (declare (ignore var-1))
          (when var-0 (setf jobz var-0))
          ret-val))
(setf minwrk 1)
(setf maxwrk 1)
(cond ((not (or wntqa wntqs wntqo wntqn)) (setf info -1))
      ((< m 0) (setf info -2)) ((< n 0) (setf info -3))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -5))
      ((or (< ldu 1) (and wntqas (< ldu m))
           (and wntqo (< m n) (< ldu m)))
       (setf info -8))
      ((or (< ldvt 1) (and wntqa (< ldvt n)) (and wntqs (< ldvt minmn))
           (and wntqo (>= m n) (< ldvt n)))
       (setf info -10)))
(cond
 ((and (= info 0) (> m 0) (> n 0))
  (cond
   ((>= m n)
    (cond
     ((>= m mnthr1)
      (cond
       (wntqn
        (setf maxwrk
                (f2cl-lib:int-add n
                                  (f2cl-lib:int-mul n
                                                    (multiple-value-bind
                                                        (ret-val var-0
                                                         var-1 var-2
                                                         var-3 var-4
                                                         var-5 var-6)
                                                        (ilaenv 1
                                                         "ZGEQRF" " " m
                                                         n -1 -1)
                                                      (declare
                                                       (ignore var-0
                                                        var-1 var-2
                                                        var-5 var-6))
                                                      (setf m var-3)
                                                      (setf n var-4)
                                                      ret-val))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                            (f2cl-lib:int-mul 2 n
```

```
                                                                    (multiple-value-bind
                                                                        (ret-val
                                                                         var-0
                                                                         var-1
                                                                         var-2
                                                                         var-3
                                                                         var-4
                                                                         var-5
                                                                         var-6)
                                                                        (ilaenv
                                                                         1
                                                                         "ZGEBRD"
                                                                         " "
                                                                         n n
                                                                         -1
                                                                         -1)
                                                                      (declare
                                                                       (ignore
                                                                        var-0
                                                                        var-1
                                                                        var-2
                                                                        var-5
                                                                        var-6))
                                                                      (setf n
                                                                              var-3)
                                                                      (setf n
                                                                              var-4)
                                                                      ret-val))))))
                 (setf minwrk (f2cl-lib:int-mul 3 n)))
                (wntqo
                 (setf wrkbl
                         (f2cl-lib:int-add n
                                           (f2cl-lib:int-mul n
                                                             (multiple-value-bind
                                                                 (ret-val var-0
                                                                  var-1 var-2
                                                                  var-3 var-4
                                                                  var-5 var-6)
                                                                 (ilaenv 1
                                                                  "ZGEQRF" " " m
                                                                  n -1 -1)
                                                               (declare
                                                                (ignore var-0
                                                                 var-1 var-2
                                                                 var-5 var-6))
                                                               (setf m var-3)
                                                               (setf n var-4)
                                                               ret-val))))
                 (setf wrkbl
                         (max (the f2cl-lib:integer4 wrkbl)
```

```
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add n
                                           (f2cl-lib:int-mul n
                                                             (multiple-value-bind
                                                                 (ret-val
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-3
                                                                  var-4
                                                                  var-5
                                                                  var-6)
                                                                 (ilaenv
                                                                  1
                                                                  "ZUNGQR"
                                                                  " "
                                                                  m n
                                                                  n
                                                                  -1)
                                                               (declare
                                                                (ignore
                                                                 var-0
                                                                 var-1
                                                                 var-2
                                                                 var-6))
                                                               (setf m
                                                                       var-3)
                                                               (setf n
                                                                       var-4)
                                                               (setf n
                                                                       var-5)
                                                               ret-val))))))
            (setf wrkbl
                    (max (the f2cl-lib:integer4 wrkbl)
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                (f2cl-lib:int-mul 2 n
                                                                  (multiple-value-bind
                                                                      (ret-val
                                                                       var-0
                                                                       var-1
                                                                       var-2
                                                                       var-3
                                                                       var-4
                                                                       var-5
                                                                       var-6)
                                                                      (ilaenv
                                                                       1
                                                                       "ZGEBRD"
                                                                       " "
```

```
                                                                    n n
                                                                    -1
                                                                    -1)
                                                                (declare
                                                                 (ignore
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-5
                                                                  var-6))
                                                                (setf n
                                                                        var-3)
                                                                (setf n
                                                                        var-4)
                                                                ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "ZUNMBR"
                                                           "QLN"
                                                           n n
                                                           n
                                                           -1)
                                                                (declare
                                                                 (ignore
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-6))
                                                                (setf n
                                                                        var-3)
                                                                (setf n
                                                                        var-4)
                                                                (setf n
                                                                        var-5)
                                                                ret-val))))))
(setf wrkbl
```

```
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                            (f2cl-lib:int-mul n
                                                              (multiple-value-bind
                                                                  (ret-val
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-3
                                                                   var-4
                                                                   var-5
                                                                   var-6)
                                                                  (ilaenv
                                                                   1
                                                                   "ZUNMBR"
                                                                   "PRC"
                                                                   n n
                                                                   n
                                                                   -1)
                                                                 (declare
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-6))
                                                                 (setf n
                                                                         var-3)
                                                                 (setf n
                                                                         var-4)
                                                                 (setf n
                                                                         var-5)
                                                                 ret-val))))))
         (setf maxwrk
                 (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                                   (f2cl-lib:int-mul n n) wrkbl))
         (setf minwrk
                 (f2cl-lib:int-add (f2cl-lib:int-mul 2 n n)
                                   (f2cl-lib:int-mul 3 n))))
        (wntqs
         (setf wrkbl
                 (f2cl-lib:int-add n
                                   (f2cl-lib:int-mul n
                                                     (multiple-value-bind
                                                         (ret-val var-0
                                                          var-1 var-2
                                                          var-3 var-4
                                                          var-5 var-6)
                                                         (ilaenv 1
                                                          "ZGEQRF" " " m
```

```
                                                    n -1 -1)
                                                 (declare
                                                  (ignore var-0
                                                   var-1 var-2
                                                   var-5 var-6))
                                                 (setf m var-3)
                                                 (setf n var-4)
                                                 ret-val))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add n
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "ZUNGQR"
                                                           " "
                                                           m n
                                                           n
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (setf m
                                                                var-3)
                                                        (setf n
                                                                var-4)
                                                        (setf n
                                                                var-5)
                                                        ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul 2 n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
```

```
                                                                    var-1
                                                                    var-2
                                                                    var-3
                                                                    var-4
                                                                    var-5
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "ZGEBRD"
                                                                    " "
                                                                    n n
                                                                    -1
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-5
                                                                    var-6))
                                                                  (setf n
                                                                          var-3)
                                                                  (setf n
                                                                          var-4)
                                                                  ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "ZUNMBR"
                                                           "QLN"
                                                           n n
                                                           n
                                                           -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
```

```
                                                                  var-2
                                                                  var-6))
                                                                (setf n
                                                                        var-3)
                                                                (setf n
                                                                        var-4)
                                                                (setf n
                                                                        var-5)
                                                                ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                    (f2cl-lib:int-mul n
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "ZUNMBR"
                                                                           "PRC"
                                                                           n n
                                                                           n
                                                                           -1)
                                                                        (declare
                                                                         (ignore
                                                                          var-0
                                                                          var-1
                                                                          var-2
                                                                          var-6))
                                                                        (setf n
                                                                                var-3)
                                                                        (setf n
                                                                                var-4)
                                                                        (setf n
                                                                                var-5)
                                                                        ret-val))))))
                (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
                (setf minwrk
                        (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                                          (f2cl-lib:int-mul 3 n))))
               (wntqa
                (setf wrkbl
                        (f2cl-lib:int-add n
```

```
                                        (f2cl-lib:int-mul n
                                                          (multiple-value-bind
                                                              (ret-val var-0
                                                               var-1 var-2
                                                               var-3 var-4
                                                               var-5 var-6)
                                                              (ilaenv 1
                                                               "ZGEQRF" " " m
                                                               n -1 -1)
                                                            (declare
                                                             (ignore var-0
                                                              var-1 var-2
                                                              var-5 var-6))
                                                            (setf m var-3)
                                                            (setf n var-4)
                                                            ret-val))))
                 (setf wrkbl
                         (max (the f2cl-lib:integer4 wrkbl)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add n
                                                     (f2cl-lib:int-mul m
                                                                       (multiple-value-bind
                                                                           (ret-val
                                                                            var-0
                                                                            var-1
                                                                            var-2
                                                                            var-3
                                                                            var-4
                                                                            var-5
                                                                            var-6)
                                                                           (ilaenv
                                                                            1
                                                                            "ZUNGQR"
                                                                            " "
                                                                            m m
                                                                            n
                                                                            -1)
                                                                         (declare
                                                                          (ignore
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-6))
                                                                         (setf m
                                                                                 var-3)
                                                                         (setf m
                                                                                 var-4)
                                                                         (setf n
                                                                                 var-5)
                                                                         ret-val))))))
```

```
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul 2 n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "ZGEBRD"
                                                           " "
                                                           n n
                                                           -1
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-5
                                                          var-6))
                                                        (setf n
                                                                var-3)
                                                        (setf n
                                                                var-4)
                                                        ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "ZUNMBR"
```

```
                                                                    "QLN"
                                                                    n n
                                                                    n
                                                                    -1)
                                                                (declare
                                                                 (ignore
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-6))
                                                                (setf n
                                                                        var-3)
                                                                (setf n
                                                                        var-4)
                                                                (setf n
                                                                        var-5)
                                                                ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                    (f2cl-lib:int-mul n
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "ZUNMBR"
                                                                           "PRC"
                                                                           n n
                                                                           n
                                                                           -1)
                                                                      (declare
                                                                       (ignore
                                                                        var-0
                                                                        var-1
                                                                        var-2
                                                                        var-6))
                                                                      (setf n
                                                                              var-3)
                                                                      (setf n
                                                                              var-4)
                                                                      (setf n
                                                                              var-5)
```

```
                                                                ret-val))))))
          (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
          (setf minwrk
                  (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                                    (f2cl-lib:int-mul 2 n) m)))))
        ((>= m mnthr2)
         (setf maxwrk
                 (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-add m n)
                                    (multiple-value-bind
                                        (ret-val var-0 var-1 var-2 var-3
                                         var-4 var-5 var-6)
                                        (ilaenv 1 "ZGEBRD" " " m n -1 -1)
                                      (declare
                                       (ignore var-0 var-1 var-2 var-5
                                        var-6))
                                      (setf m var-3)
                                      (setf n var-4)
                                      ret-val))))
         (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m))
         (cond
          (wntqo
           (setf maxwrk
                   (max (the f2cl-lib:integer4 maxwrk)
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                               (f2cl-lib:int-mul n
                                                                 (multiple-value-bind
                                                                     (ret-val
                                                                      var-0
                                                                      var-1
                                                                      var-2
                                                                      var-3
                                                                      var-4
                                                                      var-5
                                                                      var-6)
                                                                     (ilaenv
                                                                      1
                                                                      "ZUNGBR"
                                                                      "P"
                                                                      n n
                                                                      n
                                                                      -1)
                                                                   (declare
                                                                    (ignore
                                                                     var-0
                                                                     var-1
                                                                     var-2
                                                                     var-6))
```

```
                                                                (setf n
                                                                        var-3)
                                                                (setf n
                                                                        var-4)
                                                                (setf n
                                                                        var-5)
                                                                ret-val))))))
                (setf maxwrk
                        (max (the f2cl-lib:integer4 maxwrk)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                    (f2cl-lib:int-mul n
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "ZUNGBR"
                                                                           "Q"
                                                                           m n
                                                                           n
                                                                           -1)
                                                                (declare
                                                                 (ignore
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-6))
                                                                (setf m
                                                                        var-3)
                                                                (setf n
                                                                        var-4)
                                                                (setf n
                                                                        var-5)
                                                                ret-val))))))
                (setf maxwrk (f2cl-lib:int-add maxwrk (f2cl-lib:int-mul m n)))
                (setf minwrk (f2cl-lib:int-add minwrk (f2cl-lib:int-mul n n))))
               (wntqs
                (setf maxwrk
                        (max (the f2cl-lib:integer4 maxwrk)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                    (f2cl-lib:int-mul n
                                                                      (multiple-value-bind
```

```
                                                                    (ret-val
                                                                     var-0
                                                                     var-1
                                                                     var-2
                                                                     var-3
                                                                     var-4
                                                                     var-5
                                                                     var-6)
                                                                    (ilaenv
                                                                     1
                                                                     "ZUNGBR"
                                                                     "P"
                                                                     n n
                                                                     n
                                                                     -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (setf n
                                                                          var-3)
                                                                  (setf n
                                                                          var-4)
                                                                  (setf n
                                                                          var-5)
                                                                  ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                                    (ret-val
                                                                     var-0
                                                                     var-1
                                                                     var-2
                                                                     var-3
                                                                     var-4
                                                                     var-5
                                                                     var-6)
                                                                    (ilaenv
                                                                     1
                                                                     "ZUNGBR"
                                                                     "Q"
                                                                     m n
                                                                     n
                                                                     -1)
                                                                  (declare
```

```
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-6))
                                                                 (setf m
                                                                         var-3)
                                                                 (setf n
                                                                         var-4)
                                                                 (setf n
                                                                         var-5)
                                                                 ret-val)))))))
                                                        (wntqa
                                                         (setf maxwrk
                                                                 (max (the f2cl-lib:integer4 maxwrk)
                                                                      (the f2cl-lib:integer4
                                                                           (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                                                             (f2cl-lib:int-mul n
                                                                                                               (multiple-value-bind
                                                                                                                   (ret-val
                                                                                                                    var-0
                                                                                                                    var-1
                                                                                                                    var-2
                                                                                                                    var-3
                                                                                                                    var-4
                                                                                                                    var-5
                                                                                                                    var-6)
                                                                                                                   (ilaenv
                                                                                                                    1
                                                                                                                    "ZUNGBR"
                                                                                                                    "P"
                                                                                                                    n n
                                                                                                                    n
                                                                                                                    -1)
                                                                                                                 (declare
                                                                                                                  (ignore
                                                                                                                   var-0
                                                                                                                   var-1
                                                                                                                   var-2
                                                                                                                   var-6))
                                                                                                                 (setf n
                                                                                                                         var-3)
                                                                                                                 (setf n
                                                                                                                         var-4)
                                                                                                                 (setf n
                                                                                                                         var-5)
                                                                                                                 ret-val))))))
                                                         (setf maxwrk
                                                                 (max (the f2cl-lib:integer4 maxwrk)
                                                                      (the f2cl-lib:integer4
```

```
                                (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                  (f2cl-lib:int-mul m
                                                                    (multiple-value-bind
                                                                        (ret-val
                                                                         var-0
                                                                         var-1
                                                                         var-2
                                                                         var-3
                                                                         var-4
                                                                         var-5
                                                                         var-6)
                                                                        (ilaenv
                                                                         1
                                                                         "ZUNGBR"
                                                                         "Q"
                                                                         m m
                                                                         n
                                                                         -1)
                                                                      (declare
                                                                       (ignore
                                                                        var-0
                                                                        var-1
                                                                        var-2
                                                                        var-6))
                                                                      (setf m
                                                                              var-3)
                                                                      (setf m
                                                                              var-4)
                                                                      (setf n
                                                                              var-5)
                                                                      ret-val)))))))))
                        (t
                         (setf maxwrk
                                 (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                   (f2cl-lib:int-mul
                                                    (f2cl-lib:int-add m n)
                                                    (multiple-value-bind
                                                        (ret-val var-0 var-1 var-2 var-3
                                                         var-4 var-5 var-6)
                                                        (ilaenv 1 "ZGEBRD" " " m n -1 -1)
                                                      (declare
                                                       (ignore var-0 var-1 var-2 var-5
                                                        var-6))
                                                      (setf m var-3)
                                                      (setf n var-4)
                                                      ret-val))))
                         (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m))
                         (cond
                          (wntqo
                           (setf maxwrk
```

```
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                            (f2cl-lib:int-mul n
                                                              (multiple-value-bind
                                                                  (ret-val
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-3
                                                                   var-4
                                                                   var-5
                                                                   var-6)
                                                                  (ilaenv
                                                                   1
                                                                   "ZUNMBR"
                                                                   "PRC"
                                                                   n n
                                                                   n
                                                                   -1)
                                                                 (declare
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-6))
                                                                 (setf n
                                                                         var-3)
                                                                 (setf n
                                                                         var-4)
                                                                 (setf n
                                                                         var-5)
                                                                 ret-val))))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                            (f2cl-lib:int-mul n
                                                              (multiple-value-bind
                                                                  (ret-val
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-3
                                                                   var-4
                                                                   var-5
                                                                   var-6)
                                                                  (ilaenv
                                                                   1
                                                                   "ZUNMBR"
```

```
                                                                  "QLN"
                                                                  m n
                                                                  n
                                                                  -1)
                                                                (declare
                                                                 (ignore
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-6))
                                                                (setf m
                                                                        var-3)
                                                                (setf n
                                                                        var-4)
                                                                (setf n
                                                                        var-5)
                                                                ret-val))))))
                 (setf maxwrk (f2cl-lib:int-add maxwrk (f2cl-lib:int-mul m n)))
                 (setf minwrk (f2cl-lib:int-add minwrk (f2cl-lib:int-mul n n))))
                (wntqs
                 (setf maxwrk
                         (max (the f2cl-lib:integer4 maxwrk)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                     (f2cl-lib:int-mul n
                                                                       (multiple-value-bind
                                                                           (ret-val
                                                                            var-0
                                                                            var-1
                                                                            var-2
                                                                            var-3
                                                                            var-4
                                                                            var-5
                                                                            var-6)
                                                                           (ilaenv
                                                                            1
                                                                            "ZUNMBR"
                                                                            "PRC"
                                                                            n n
                                                                            n
                                                                            -1)
                                                                         (declare
                                                                          (ignore
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-6))
                                                                         (setf n
                                                                                 var-3)
                                                                         (setf n
```

```
                                                                        var-4)
                                                                 (setf n
                                                                        var-5)
                                                                 ret-val))))))
                (setf maxwrk
                        (max (the f2cl-lib:integer4 maxwrk)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                    (f2cl-lib:int-mul n
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "ZUNMBR"
                                                                           "QLN"
                                                                           m n
                                                                           n
                                                                           -1)
                                                                 (declare
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-6))
                                                                 (setf m
                                                                        var-3)
                                                                 (setf n
                                                                        var-4)
                                                                 (setf n
                                                                        var-5)
                                                                 ret-val)))))))
               (wntqa
                (setf maxwrk
                        (max (the f2cl-lib:integer4 maxwrk)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                                    (f2cl-lib:int-mul n
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
```

```
                                                                var-4
                                                                var-5
                                                                var-6)
                                                               (ilaenv
                                                                1
                                                                "ZUNGBR"
                                                                "PRC"
                                                                n n
                                                                n
                                                                -1)
                                                              (declare
                                                               (ignore
                                                                var-0
                                                                var-1
                                                                var-2
                                                                var-6))
                                                              (setf n
                                                                      var-3)
                                                              (setf n
                                                                      var-4)
                                                              (setf n
                                                                      var-5)
                                                              ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "ZUNGBR"
                                                           "QLN"
                                                           m m
                                                           n
                                                           -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-6))
```

```
                                                                    (setf m
                                                                            var-3)
                                                                    (setf m
                                                                            var-4)
                                                                    (setf n
                                                                            var-5)
                                                                   ret-val)))))))))))
                (t
                 (cond
                  ((>= n mnthr1)
                   (cond
                    (wntqn
                     (setf maxwrk
                             (f2cl-lib:int-add m
                                               (f2cl-lib:int-mul m
                                                                 (multiple-value-bind
                                                                     (ret-val var-0
                                                                      var-1 var-2
                                                                      var-3 var-4
                                                                      var-5 var-6)
                                                                     (ilaenv 1
                                                                      "ZGELQF" " " m
                                                                      n -1 -1)
                                                                   (declare
                                                                    (ignore var-0
                                                                     var-1 var-2
                                                                     var-5 var-6))
                                                                   (setf m var-3)
                                                                   (setf n var-4)
                                                                   ret-val))))
                     (setf maxwrk
                             (max (the f2cl-lib:integer4 maxwrk)
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                         (f2cl-lib:int-mul 2 m
                                                                           (multiple-value-bind
                                                                               (ret-val
                                                                                var-0
                                                                                var-1
                                                                                var-2
                                                                                var-3
                                                                                var-4
                                                                                var-5
                                                                                var-6)
                                                                               (ilaenv
                                                                                1
                                                                                "ZGEBRD"
                                                                                " "
                                                                                m m
                                                                                -1
```

```
                                                                  -1)
                                                                (declare
                                                                 (ignore
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-5
                                                                  var-6))
                                                                (setf m
                                                                        var-3)
                                                                (setf m
                                                                        var-4)
                                                                ret-val))))))
 (setf minwrk (f2cl-lib:int-mul 3 m)))
(wntqo
 (setf wrkbl
         (f2cl-lib:int-add m
                           (f2cl-lib:int-mul m
                                             (multiple-value-bind
                                                 (ret-val var-0
                                                  var-1 var-2
                                                  var-3 var-4
                                                  var-5 var-6)
                                                 (ilaenv 1
                                                  "ZGELQF" " " m
                                                  n -1 -1)
                                               (declare
                                                (ignore var-0
                                                 var-1 var-2
                                                 var-5 var-6))
                                               (setf m var-3)
                                               (setf n var-4)
                                               ret-val))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add m
                                     (f2cl-lib:int-mul m
                                                       (multiple-value-bind
                                                           (ret-val
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-3
                                                            var-4
                                                            var-5
                                                            var-6)
                                                           (ilaenv
                                                            1
                                                            "ZUNGLQ"
```

```
                                                                    " "
                                                                    m n
                                                                    m
                                                                    -1)
                                                                 (declare
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-6))
                                                                 (setf m
                                                                         var-3)
                                                                 (setf n
                                                                         var-4)
                                                                 (setf m
                                                                         var-5)
                                                                 ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                    (f2cl-lib:int-mul 2 m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "ZGEBRD"
                                                                           " "
                                                                           m m
                                                                           -1
                                                                           -1)
                                                                 (declare
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-5
                                                                   var-6))
                                                                 (setf m
                                                                         var-3)
                                                                 (setf m
                                                                         var-4)
                                                                 ret-val))))))
```

```
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "ZUNMBR"
                                                           "PRC"
                                                           m m
                                                           m
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (setf m
                                                                var-3)
                                                        (setf m
                                                                var-4)
                                                        (setf m
                                                                var-5)
                                                        ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
```

```
                                                                "ZUNMBR"
                                                                "QLN"
                                                                m m
                                                                m
                                                                -1)
                                                            (declare
                                                             (ignore
                                                              var-0
                                                              var-1
                                                              var-2
                                                              var-6))
                                                            (setf m
                                                                    var-3)
                                                            (setf m
                                                                    var-4)
                                                            (setf m
                                                                    var-5)
                                                            ret-val))))))
                (setf maxwrk
                        (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                                          (f2cl-lib:int-mul m m) wrkbl))
                (setf minwrk
                        (f2cl-lib:int-add (f2cl-lib:int-mul 2 m m)
                                          (f2cl-lib:int-mul 3 m))))
               (wntqs
                (setf wrkbl
                        (f2cl-lib:int-add m
                                          (f2cl-lib:int-mul m
                                                            (multiple-value-bind
                                                                (ret-val var-0
                                                                 var-1 var-2
                                                                 var-3 var-4
                                                                 var-5 var-6)
                                                                (ilaenv 1
                                                                 "ZGELQF" " " m
                                                                 n -1 -1)
                                                              (declare
                                                               (ignore var-0
                                                                var-1 var-2
                                                                var-5 var-6))
                                                              (setf m var-3)
                                                              (setf n var-4)
                                                              ret-val))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add m
                                                    (f2cl-lib:int-mul m
                                                                      (multiple-value-bind
                                                                          (ret-val
```

```
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-3
                                                                    var-4
                                                                    var-5
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "ZUNGLQ"
                                                                    " "
                                                                    m n
                                                                    m
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (setf m
                                                                          var-3)
                                                                  (setf n
                                                                          var-4)
                                                                  (setf m
                                                                          var-5)
                                                                  ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    (f2cl-lib:int-mul 2 m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "ZGEBRD"
                                                           " "
                                                           m m
                                                           -1
                                                           -1)
                                                        (declare
                                                         (ignore
```

```
                                                                var-0
                                                                var-1
                                                                var-2
                                                                var-5
                                                                var-6))
                                                              (setf m
                                                                      var-3)
                                                              (setf m
                                                                      var-4)
                                                              ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "ZUNMBR"
                                                           "PRC"
                                                           m m
                                                           m
                                                           -1)
                                                              (declare
                                                               (ignore
                                                                var-0
                                                                var-1
                                                                var-2
                                                                var-6))
                                                              (setf m
                                                                      var-3)
                                                              (setf m
                                                                      var-4)
                                                              (setf m
                                                                      var-5)
                                                              ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
```

```
                                                                  (ret-val
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-3
                                                                   var-4
                                                                   var-5
                                                                   var-6)
                                                                  (ilaenv
                                                                   1
                                                                   "ZUNMBR"
                                                                   "QLN"
                                                                   m m
                                                                   m
                                                                   -1)
                                                                 (declare
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-6))
                                                                 (setf m
                                                                         var-3)
                                                                 (setf m
                                                                         var-4)
                                                                 (setf m
                                                                         var-5)
                                                                 ret-val))))))
                 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
                 (setf minwrk
                         (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                           (f2cl-lib:int-mul 3 m))))
                (wntqa
                 (setf wrkbl
                         (f2cl-lib:int-add m
                                           (f2cl-lib:int-mul m
                                                             (multiple-value-bind
                                                                 (ret-val var-0
                                                                  var-1 var-2
                                                                  var-3 var-4
                                                                  var-5 var-6)
                                                                 (ilaenv 1
                                                                  "ZGELQF" " " m
                                                                  n -1 -1)
                                                               (declare
                                                                (ignore var-0
                                                                 var-1 var-2
                                                                 var-5 var-6))
                                                               (setf m var-3)
                                                               (setf n var-4)
```

```
                                                    ret-val))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add m
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "ZUNGLQ"
                                                           " "
                                                           n n
                                                           m
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (setf n
                                                                var-3)
                                                        (setf n
                                                                var-4)
                                                        (setf m
                                                                var-5)
                                                        ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    (f2cl-lib:int-mul 2 m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
```

```
                                                                    1
                                                                    "ZGEBRD"
                                                                    " "
                                                                    m m
                                                                    -1
                                                                    -1)
                                                                 (declare
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-5
                                                                   var-6))
                                                                 (setf m
                                                                         var-3)
                                                                 (setf m
                                                                         var-4)
                                                                 ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "ZUNMBR"
                                                           "PRC"
                                                           m m
                                                           m
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (setf m
                                                                var-3)
                                                        (setf m
                                                                var-4)
                                                        (setf m
```

```
                                                                    var-5)
                                                              ret-val))))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                        (f2cl-lib:int-mul m
                                                          (multiple-value-bind
                                                              (ret-val
                                                               var-0
                                                               var-1
                                                               var-2
                                                               var-3
                                                               var-4
                                                               var-5
                                                               var-6)
                                                              (ilaenv
                                                               1
                                                               "ZUNMBR"
                                                               "QLN"
                                                               m m
                                                               m
                                                               -1)
                                                            (declare
                                                             (ignore
                                                              var-0
                                                              var-1
                                                              var-2
                                                              var-6))
                                                            (setf m
                                                                    var-3)
                                                            (setf m
                                                                    var-4)
                                                            (setf m
                                                                    var-5)
                                                            ret-val))))))
    (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
    (setf minwrk
            (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                              (f2cl-lib:int-mul 2 m) n)))))
((>= n mnthr2)
 (setf maxwrk
         (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-add m n)
                            (multiple-value-bind
                                (ret-val var-0 var-1 var-2 var-3
                                 var-4 var-5 var-6)
                                (ilaenv 1 "ZGEBRD" " " m n -1 -1)
                              (declare
```

```
                                 (ignore var-0 var-1 var-2 var-5
                                  var-6))
                                (setf m var-3)
                                (setf n var-4)
                                ret-val))))
(setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n))
(cond
 (wntqo
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                      (f2cl-lib:int-mul m
                                                        (multiple-value-bind
                                                            (ret-val
                                                             var-0
                                                             var-1
                                                             var-2
                                                             var-3
                                                             var-4
                                                             var-5
                                                             var-6)
                                                            (ilaenv
                                                             1
                                                             "ZUNGBR"
                                                             "P"
                                                             m n
                                                             m
                                                             -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-6))
                                                          (setf m
                                                                  var-3)
                                                          (setf n
                                                                  var-4)
                                                          (setf m
                                                                  var-5)
                                                          ret-val))))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                      (f2cl-lib:int-mul m
                                                        (multiple-value-bind
                                                            (ret-val
                                                             var-0
```

```
                                                                var-1
                                                                var-2
                                                                var-3
                                                                var-4
                                                                var-5
                                                                var-6)
                                                               (ilaenv
                                                                1
                                                                "ZUNGBR"
                                                                "Q"
                                                                m m
                                                                n
                                                                -1)
                                                              (declare
                                                               (ignore
                                                                var-0
                                                                var-1
                                                                var-2
                                                                var-6))
                                                              (setf m
                                                                      var-3)
                                                              (setf m
                                                                      var-4)
                                                              (setf n
                                                                      var-5)
                                                              ret-val))))))
 (setf maxwrk (f2cl-lib:int-add maxwrk (f2cl-lib:int-mul m n)))
 (setf minwrk (f2cl-lib:int-add minwrk (f2cl-lib:int-mul m m))))
(wntqs
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                     (f2cl-lib:int-mul m
                                                       (multiple-value-bind
                                                           (ret-val
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-3
                                                            var-4
                                                            var-5
                                                            var-6)
                                                           (ilaenv
                                                            1
                                                            "ZUNGBR"
                                                            "P"
                                                            m n
                                                            m
                                                            -1)
```

```
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (setf m
                                                                var-3)
                                                        (setf n
                                                                var-4)
                                                        (setf m
                                                                var-5)
                                                        ret-val))))))
                 (setf maxwrk
                         (max (the f2cl-lib:integer4 maxwrk)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                     (f2cl-lib:int-mul m
                                                                       (multiple-value-bind
                                                                           (ret-val
                                                                            var-0
                                                                            var-1
                                                                            var-2
                                                                            var-3
                                                                            var-4
                                                                            var-5
                                                                            var-6)
                                                                           (ilaenv
                                                                            1
                                                                            "ZUNGBR"
                                                                            "Q"
                                                                            m m
                                                                            n
                                                                            -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (setf m
                                                                var-3)
                                                        (setf m
                                                                var-4)
                                                        (setf n
                                                                var-5)
                                                        ret-val)))))))
                (wntqa
                 (setf maxwrk
                         (max (the f2cl-lib:integer4 maxwrk)
```

```
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                           (f2cl-lib:int-mul n
                                                             (multiple-value-bind
                                                                 (ret-val
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-3
                                                                  var-4
                                                                  var-5
                                                                  var-6)
                                                                 (ilaenv
                                                                  1
                                                                  "ZUNGBR"
                                                                  "P"
                                                                  n n
                                                                  m
                                                                  -1)
                                                               (declare
                                                                (ignore
                                                                 var-0
                                                                 var-1
                                                                 var-2
                                                                 var-6))
                                                               (setf n
                                                                       var-3)
                                                               (setf n
                                                                       var-4)
                                                               (setf m
                                                                       var-5)
                                                               ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "ZUNGBR"
                                                           "Q"
```

```
                                                                    m m
                                                                    n
                                                                    -1)
                                                                 (declare
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-6))
                                                                 (setf m
                                                                         var-3)
                                                                 (setf m
                                                                         var-4)
                                                                 (setf n
                                                                         var-5)
                                                                 ret-val)))))))))
                 (t
                  (setf maxwrk
                          (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                            (f2cl-lib:int-mul
                                             (f2cl-lib:int-add m n)
                                             (multiple-value-bind
                                                 (ret-val var-0 var-1 var-2 var-3
                                                  var-4 var-5 var-6)
                                                 (ilaenv 1 "ZGEBRD" " " m n -1 -1)
                                               (declare
                                                (ignore var-0 var-1 var-2 var-5
                                                 var-6))
                                               (setf m var-3)
                                               (setf n var-4)
                                               ret-val))))
                  (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n))
                  (cond
                   (wntqo
                    (setf maxwrk
                            (max (the f2cl-lib:integer4 maxwrk)
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                        (f2cl-lib:int-mul m
                                                                          (multiple-value-bind
                                                                              (ret-val
                                                                               var-0
                                                                               var-1
                                                                               var-2
                                                                               var-3
                                                                               var-4
                                                                               var-5
                                                                               var-6)
                                                                              (ilaenv
                                                                               1
```

```
                                                                    "ZUNMBR"
                                                                    "PRC"
                                                                    m n
                                                                    m
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (setf m
                                                                          var-3)
                                                                  (setf n
                                                                          var-4)
                                                                  (setf m
                                                                          var-5)
                                                                  ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "ZUNMBR"
                                                           "QLN"
                                                           m m
                                                           n
                                                           -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-6))
                                                          (setf m
                                                                  var-3)
                                                          (setf m
                                                                  var-4)
                                                          (setf n
```

```
                                                                var-5)
                                                          ret-val))))))
 (setf maxwrk (f2cl-lib:int-add maxwrk (f2cl-lib:int-mul m n)))
 (setf minwrk (f2cl-lib:int-add minwrk (f2cl-lib:int-mul m m))))
(wntqs
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                     (f2cl-lib:int-mul m
                                                       (multiple-value-bind
                                                           (ret-val
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-3
                                                            var-4
                                                            var-5
                                                            var-6)
                                                           (ilaenv
                                                            1
                                                            "ZUNGBR"
                                                            "PRC"
                                                            m n
                                                            m
                                                            -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-6))
                                                          (setf m
                                                                  var-3)
                                                          (setf n
                                                                  var-4)
                                                          (setf m
                                                                  var-5)
                                                          ret-val))))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                     (f2cl-lib:int-mul m
                                                       (multiple-value-bind
                                                           (ret-val
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-3
```

```
                                                                    var-4
                                                                    var-5
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "ZUNGBR"
                                                                    "QLN"
                                                                    m m
                                                                    n
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (setf m
                                                                          var-3)
                                                                  (setf m
                                                                          var-4)
                                                                  (setf n
                                                                          var-5)
                                                                  ret-val)))))))
                                                                 (wntqa
                                                                  (setf maxwrk
                                                                          (max (the f2cl-lib:integer4 maxwrk)
                                                                               (the f2cl-lib:integer4
                                                                                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                                                                      (f2cl-lib:int-mul n
                                                                                                                        (multiple-value-bind
                                                                                                                            (ret-val
                                                                                                                             var-0
                                                                                                                             var-1
                                                                                                                             var-2
                                                                                                                             var-3
                                                                                                                             var-4
                                                                                                                             var-5
                                                                                                                             var-6)
                                                                                                                            (ilaenv
                                                                                                                             1
                                                                                                                             "ZUNGBR"
                                                                                                                             "PRC"
                                                                                                                             n n
                                                                                                                             m
                                                                                                                             -1)
                                                                                                                           (declare
                                                                                                                            (ignore
                                                                                                                             var-0
                                                                                                                             var-1
                                                                                                                             var-2
```

```
                                                                    var-6))
                                                                  (setf n
                                                                          var-3)
                                                                  (setf n
                                                                          var-4)
                                                                  (setf m
                                                                          var-5)
                                                                  ret-val))))))
                (setf maxwrk
                        (max (the f2cl-lib:integer4 maxwrk)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                                    (f2cl-lib:int-mul m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "ZUNGBR"
                                                                           "QLN"
                                                                           m m
                                                                           n
                                                                           -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (setf m
                                                                          var-3)
                                                                  (setf m
                                                                          var-4)
                                                                  (setf n
                                                                          var-5)
                                                                  ret-val)))))))))))
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4 minwrk)))))
        (cond
         ((= info 0)
          (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                  (coerce maxwrk 'f2cl-lib:complex16))
          (if (and (< lwork minwrk) (/= lwork lquerv))
```

```
      (setf info -13))))
(cond
 ((/= info 0) (xerbla "ZGESDD" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= lwork lquerv)
    (go end_label))
(cond ((or (= m 0) (= n 0)) (go end_label)))
(setf eps (dlamch "P"))
(setf smlnum (/ (f2cl-lib:fsqrt (dlamch "S")) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "M" m n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iscl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf iscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 anrm smlnum m n a lda ierr)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf ierr var-9))))
 ((> anrm bignum) (setf iscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 anrm bignum m n a lda ierr)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf ierr var-9)))))
(cond
 ((>= m n)
  (cond
   ((>= m mnthr1)
    (cond
     (wntqn (setf itau 1) (setf nwork (f2cl-lib:int-add itau n))
      (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgeqrf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (2 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(setf ie 1) (setf itauq 1)
(setf itaup (f2cl-lib:int-add itauq n))
(setf nwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgebrd n n a lda s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(setf nrwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dbdsdc "U" "N" n s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
```

```
      dum 1 dum 1 dum idum
      (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                            ((1 *)) rwork-%offset%)
      iwork info)
    (declare
     (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12))
    (when var-2 (setf n var-2))
    (when var-13 (setf info var-13))))
  (wntqo (setf iu 1) (setf ldwrku n)
   (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                           (f2cl-lib:int-mul n n)
                           (f2cl-lib:int-mul 3 n)))
    (setf ldwrkr m))
    (t
    (setf ldwrkr
            (the f2cl-lib:integer4
                 (truncate (- lwork (* n n) (* 3 n)) n)))))
   (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
   (setf nwork (f2cl-lib:int-add itau n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (zgeqrf m n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (zlacpy "U" n n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                              ((1 *)) work-%offset%)
        ldwrkr)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldwrkr var-6)))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (zlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
        czero czero
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
```

```
                          ((+ ir 1)) ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zungqr m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf nwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(setf iru (f2cl-lib:int-add ie n))
(setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul n n)))
(setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul n n)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
```

```
    (dbdsdc "U" "I" n s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     n
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     n dum idum
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf n var-2))
  (when var-6 (setf n var-6))
  (when var-8 (setf n var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacp2 "F" n n
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "Q" "L" "N" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
```

```
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldwrkr var-7))
  (when var-10 (setf ldwrku var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacp2 "F" n n
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     n vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "P" "R" "C" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldwrkr var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i ldwrkr))
              ((> i m) nil)
  (tagbody
    (setf chunk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
             (the f2cl-lib:integer4 ldwrkr)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (zgemm "N" "N" chunk n n cone
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i 1)
                               ((1 lda) (1 *)) a-%offset%)
```

```
                lda
                (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                      (iu) ((1 *)) work-%offset%)
                ldwrku czero
                (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                      (ir) ((1 *)) work-%offset%)
                ldwrkr)
              (declare (ignore var-0 var-1 var-6 var-8 var-11))
              (when var-2 (setf chunk var-2))
              (when var-3 (setf n var-3))
              (when var-4 (setf n var-4))
              (when var-5 (setf cone var-5))
              (when var-7 (setf lda var-7))
              (when var-9 (setf ldwrku var-9))
              (when var-10 (setf czero var-10))
              (when var-12 (setf ldwrkr var-12)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (zlacpy "F" chunk n
                 (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                       (ir) ((1 *)) work-%offset%)
                 ldwrkr
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i 1)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare (ignore var-0 var-3 var-5))
              (when var-1 (setf chunk var-1))
              (when var-2 (setf n var-2))
              (when var-4 (setf ldwrkr var-4))
              (when var-6 (setf lda var-6)))
          label10)))
      (wntqs (setf ir 1) (setf ldwrkr n)
       (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
       (setf nwork (f2cl-lib:int-add itau n))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
           (zgeqrf m n a lda
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                                  ((1 *)) work-%offset%)
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                                  ((1 *)) work-%offset%)
            (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
         (declare (ignore var-2 var-4 var-5 var-6))
         (setf m var-0)
         (setf n var-1)
         (setf lda var-3)
         (setf ierr var-7))
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
           (zlacpy "U" n n a lda
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
```

```
                          ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                          ((+ ir 1)) ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zungqr m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                          ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf nwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                          ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                          rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                          ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
```

```
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(setf iru (f2cl-lib:int-add ie n))
(setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul n n)))
(setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul n n)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dbdsdc "U" "I" n s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     n
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     n dum idum
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf n var-2))
  (when var-6 (setf n var-6))
  (when var-8 (setf n var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacp2 "F" n n
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     n u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "Q" "L" "N" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     u ldu
```

```
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldwrkr var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacp2 "F" n n
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     n vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "P" "R" "C" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldwrkr var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "F" n n u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
```

```
    (when var-4 (setf ldu var-4))
    (when var-6 (setf ldwrkr var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (zgemm "N" "N" m n n cone a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                             ((1 *)) work-%offset%)
       ldwrkr czero u ldu)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldwrkr var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf ldu var-12))))
 (wntqa (setf iu 1) (setf ldwrku n)
  (setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
  (setf nwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zungqr m m n u ldu
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
```

```
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (2 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf nwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgebrd n n a lda s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(setf iru (f2cl-lib:int-add ie n))
(setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul n n)))
(setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul n n)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dbdsdc "U" "I" n s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     n
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
```

```
      n dum idum
      (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                            ((1 *)) rwork-%offset%)
      iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf n var-2))
  (when var-6 (setf n var-6))
  (when var-8 (setf n var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacp2 "F" n n
      (f2cl-lib:array-slice rwork-%data% double-float (iru)
                            ((1 *)) rwork-%offset%)
      n
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iu)
                            ((1 *)) work-%offset%)
      ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "Q" "L" "N" n n n a lda
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iu)
                            ((1 *)) work-%offset%)
      ldwrku
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldwrku var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacp2 "F" n n
      (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                            ((1 *)) rwork-%offset%)
      n vt ldvt)
  (declare (ignore var-0 var-3 var-5))
```

```
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf n var-4))
        (when var-6 (setf ldvt var-6)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13)
          (zunmbr "P" "R" "C" n n n a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                                 ((1 *)) work-%offset%)
           vt ldvt
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
        (declare
         (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
        (when var-3 (setf n var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf n var-5))
        (when var-7 (setf lda var-7))
        (when var-10 (setf ldvt var-10))
        (when var-13 (setf ierr var-13)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (zgemm "N" "N" m n n cone u ldu
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iu)
                                 ((1 *)) work-%offset%)
           ldwrku czero a lda)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf n var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf cone var-5))
        (when var-7 (setf ldu var-7))
        (when var-9 (setf ldwrku var-9))
        (when var-10 (setf czero var-10))
        (when var-12 (setf lda var-12)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlacpy "F" m n a lda u ldu)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldu var-6))))))
    ((>= m mnthr2) (setf ie 1) (setf nrwork (f2cl-lib:int-add ie n))
     (setf itauq 1) (setf itaup (f2cl-lib:int-add itauq n))
     (setf nwork (f2cl-lib:int-add itaup n))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
      var-10)
     (zgebrd m n a lda s
      (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                            rwork-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-10))
 (cond
  (wntqn
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13)
       (dbdsdc "U" "N" n s
        (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                              rwork-%offset%)
        dum 1 dum 1 dum idum
        (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                              ((1 *)) rwork-%offset%)
        iwork info)
     (declare
      (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12))
     (when var-2 (setf n var-2))
     (when var-13 (setf info var-13))))
  (wntqo (setf iu nwork) (setf iru nrwork)
   (setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul n n)))
   (setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul n n)))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (zlacpy "U" n n a lda vt ldvt)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf n var-1))
     (when var-2 (setf n var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf ldvt var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (zungbr "P" n n n vt ldvt
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                              ((1 *)) work-%offset%)
```

```
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (zungbr "Q" m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf ierr var-9)))
(cond
 ((>= lwork
      (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                        (f2cl-lib:int-mul 3 n)))
  (setf ldwrku m))
 (t
  (setf ldwrku
          (the f2cl-lib:integer4
               (truncate (- lwork (* 3 n)) n)))))
(setf nwork (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dbdsdc "U" "I" n s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     n
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     n dum idum
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf n var-2))
```

```
  (when var-6 (setf n var-6))
  (when var-8 (setf n var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zlarcm n n
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     n vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                           ((1 *)) rwork-%offset%))
  (declare (ignore var-2 var-4 var-6 var-8))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-7 (setf ldwrku var-7)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "F" n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iu)
                           ((1 *)) work-%offset%)
     ldwrku vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldvt var-6)))
(setf nrwork irvt)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i ldwrku))
              ((> i m) nil)
  (tagbody
    (setf chunk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
             (the f2cl-lib:integer4 ldwrku)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (zlacrm chunk n
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice rwork-%data% double-float (iru)
                               ((1 *)) rwork-%offset%)
         n
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (iu) ((1 *)) work-%offset%)
```

```
            ldwrku
            (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                                  ((1 *)) rwork-%offset%))
        (declare (ignore var-2 var-4 var-6 var-8))
        (when var-0 (setf chunk var-0))
        (when var-1 (setf n var-1))
        (when var-3 (setf lda var-3))
        (when var-5 (setf n var-5))
        (when var-7 (setf ldwrku var-7)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlacpy "F" chunk n
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                  (iu) ((1 *)) work-%offset%)
            ldwrku
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i 1)
                                  ((1 lda) (1 *)) a-%offset%)
            lda)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf chunk var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf ldwrku var-4))
        (when var-6 (setf lda var-6)))
    label20)))
(wntqs
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (zlacpy "U" n n a lda vt ldvt)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldvt var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
     (zungbr "P" n n n vt ldvt
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
   (declare (ignore var-0 var-4 var-6 var-7 var-8))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-3 (setf n var-3))
   (when var-5 (setf ldvt var-5))
   (when var-9 (setf ierr var-9)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (zlacpy "L" m n a lda u ldu)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf m var-1))
```

```
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (zungbr "Q" m n n u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf iru nrwork)
(setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul n n)))
(setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul n n)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dbdsdc "U" "I" n s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     n
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     n dum idum
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf n var-2))
  (when var-6 (setf n var-6))
  (when var-8 (setf n var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zlarcm n n
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     n vt ldvt a lda
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                           ((1 *)) rwork-%offset%))
```

```
    (declare (ignore var-2 var-4 var-6 var-8))
    (when var-0 (setf n var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf n var-3))
    (when var-5 (setf ldvt var-5))
    (when var-7 (setf lda var-7)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "F" n n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (setf nrwork irvt)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zlacrm m n u ldu
       (f2cl-lib:array-slice rwork-%data% double-float (iru)
                             ((1 *)) rwork-%offset%)
       n a lda
       (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                             ((1 *)) rwork-%offset%))
    (declare (ignore var-2 var-4 var-6 var-8))
    (when var-0 (setf m var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf ldu var-3))
    (when var-5 (setf n var-5))
    (when var-7 (setf lda var-7)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "F" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6))))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "U" n n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zungbr "P" n n n vt ldvt
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                             ((1 *)) work-%offset%)
```

```
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (zungbr "Q" m m n u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf iru nrwork)
(setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul n n)))
(setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul n n)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dbdsdc "U" "I" n s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     n
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     n dum idum
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf n var-2))
```

```
        (when var-6 (setf n var-6))
        (when var-8 (setf n var-8))
        (when var-13 (setf info var-13)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (zlarcm n n
           (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                                 ((1 *)) rwork-%offset%)
           n vt ldvt a lda
           (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                                 ((1 *)) rwork-%offset%))
        (declare (ignore var-2 var-4 var-6 var-8))
        (when var-0 (setf n var-0))
        (when var-1 (setf n var-1))
        (when var-3 (setf n var-3))
        (when var-5 (setf ldvt var-5))
        (when var-7 (setf lda var-7)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlacpy "F" n n a lda vt ldvt)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldvt var-6)))
      (setf nrwork irvt)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (zlacrm m n u ldu
           (f2cl-lib:array-slice rwork-%data% double-float (iru)
                                 ((1 *)) rwork-%offset%)
           n a lda
           (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                                 ((1 *)) rwork-%offset%))
        (declare (ignore var-2 var-4 var-6 var-8))
        (when var-0 (setf m var-0))
        (when var-1 (setf n var-1))
        (when var-3 (setf ldu var-3))
        (when var-5 (setf n var-5))
        (when var-7 (setf lda var-7)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlacpy "F" m n a lda u ldu)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldu var-6))))))
    (t (setf ie 1) (setf nrwork (f2cl-lib:int-add ie n)) (setf itauq 1)
     (setf itaup (f2cl-lib:int-add itauq n))
     (setf nwork (f2cl-lib:int-add itaup n))
     (multiple-value-bind
```

```
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (zgebrd m n a lda s
       (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                             rwork-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-10))
  (cond
   (wntqn
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (dbdsdc "U" "N" n s
         (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                               rwork-%offset%)
         dum 1 dum 1 dum idum
         (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                               ((1 *)) rwork-%offset%)
         iwork info)
      (declare
       (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12))
      (when var-2 (setf n var-2))
      (when var-13 (setf info var-13))))
   (wntqo (setf iu nwork) (setf iru nrwork)
    (setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul n n)))
    (setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul n n)))
    (cond
     ((>= lwork
          (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                            (f2cl-lib:int-mul 3 n)))
      (setf ldwrku m))
     (t
      (setf ldwrku
              (the f2cl-lib:integer4
                   (truncate (- lwork (* 3 n)) n)))))
    (setf nwork (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
```

```
    (dbdsdc "U" "I" n s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     n
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     n dum idum
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf n var-2))
  (when var-6 (setf n var-6))
  (when var-8 (setf n var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacp2 "F" n n
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     n vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "P" "R" "C" n n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(cond
 ((>= lwork
      (f2cl-lib:int-add (f2cl-lib:int-mul m n)
```

```
                      (f2cl-lib:int-mul 3 n)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "F" m n czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacp2 "F" n n
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (zunmbr "Q" "L" "N" m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (nwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldwrku var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "F" m n
```

```
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iu)
                            ((1 *)) work-%offset%)
      ldwrku a lda)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldwrku var-4))
    (when var-6 (setf lda var-6))))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9)
       (zungbr "Q" m n n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itauq) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (nwork) ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))
   (setf nrwork irvt)
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i ldwrku))
                 ((> i m) nil)
    (tagbody
      (setf chunk
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
               (the f2cl-lib:integer4 ldwrku)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (zlacrm chunk n
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                 (i 1) ((1 lda) (1 *))
                                 a-%offset%)
           lda
           (f2cl-lib:array-slice rwork-%data% double-float (iru)
                                 ((1 *)) rwork-%offset%)
           n
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                 (iu) ((1 *)) work-%offset%)
           ldwrku
           (f2cl-lib:array-slice rwork-%data% double-float
                                 (nrwork) ((1 *))
                                 rwork-%offset%))
        (declare (ignore var-2 var-4 var-6 var-8))
```

```
            (when var-0 (setf chunk var-0))
            (when var-1 (setf n var-1))
            (when var-3 (setf lda var-3))
            (when var-5 (setf n var-5))
            (when var-7 (setf ldwrku var-7)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (zlacpy "F" chunk n
               (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                     (iu) ((1 *)) work-%offset%)
               ldwrku
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                     (i 1) ((1 lda) (1 *))
                                     a-%offset%)
               lda)
            (declare (ignore var-0 var-3 var-5))
            (when var-1 (setf chunk var-1))
            (when var-2 (setf n var-2))
            (when var-4 (setf ldwrku var-4))
            (when var-6 (setf lda var-6)))
        label30)))))
  (wntqs (setf iru nrwork)
   (setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul n n)))
   (setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul n n)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13)
       (dbdsdc "U" "I" n s
        (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                              rwork-%offset%)
        (f2cl-lib:array-slice rwork-%data% double-float (iru)
                              ((1 *)) rwork-%offset%)
        n
        (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                              ((1 *)) rwork-%offset%)
        n dum idum
        (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                              ((1 *)) rwork-%offset%)
        iwork info)
     (declare
      (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
       var-11 var-12))
     (when var-2 (setf n var-2))
     (when var-6 (setf n var-6))
     (when var-8 (setf n var-8))
     (when var-13 (setf info var-13)))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (zlaset "F" m n czero czero u ldu)
     (declare (ignore var-0 var-5))
     (when var-1 (setf m var-1))
```

```
  (when var-2 (setf n var-2))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacp2 "F" n n
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     n u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "Q" "L" "N" m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacp2 "F" n n
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     n vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "P" "R" "C" n n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
```

```
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf n var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldvt var-10))
    (when var-13 (setf ierr var-13))))
  (t (setf iru nrwork)
   (setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul n n)))
   (setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul n n)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13)
       (dbdsdc "U" "I" n s
        (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                              rwork-%offset%)
        (f2cl-lib:array-slice rwork-%data% double-float (iru)
                              ((1 *)) rwork-%offset%)
        n
        (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                              ((1 *)) rwork-%offset%)
        n dum idum
        (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                              ((1 *)) rwork-%offset%)
        iwork info)
    (declare
     (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
      var-11 var-12))
    (when var-2 (setf n var-2))
    (when var-6 (setf n var-6))
    (when var-8 (setf n var-8))
    (when var-13 (setf info var-13)))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (zlaset "F" m m czero czero u ldu)
    (declare (ignore var-0 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldu var-6)))
   (cond
    ((> m n)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (zlaset "F" (f2cl-lib:int-sub m n) (f2cl-lib:int-sub m n)
          czero cone
          (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
```

```
                                ((+ n 1) (f2cl-lib:int-add n 1))
                                ((1 ldu) (1 *)) u-%offset%)
        ldu)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf czero var-3))
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldu var-6)))))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacp2 "F" n n
       (f2cl-lib:array-slice rwork-%data% double-float (iru)
                             ((1 *)) rwork-%offset%)
       n u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf n var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (zunmbr "Q" "L" "N" m m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                             ((1 *)) work-%offset%)
       u ldu
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf n var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldu var-10))
    (when var-13 (setf ierr var-13)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacp2 "F" n n
       (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                             ((1 *)) rwork-%offset%)
       n vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf n var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (zunmbr "P" "R" "C" n n n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
```

```
                                  ((1 *)) work-%offset%)
            vt ldvt
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                                  ((1 *)) work-%offset%)
            (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
          (declare
           (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
          (when var-3 (setf n var-3))
          (when var-4 (setf n var-4))
          (when var-5 (setf n var-5))
          (when var-7 (setf lda var-7))
          (when var-10 (setf ldvt var-10))
          (when var-13 (setf ierr var-13))))))))
(t
 (cond
  ((>= n mnthr1)
   (cond
    (wntqn (setf itau 1) (setf nwork (f2cl-lib:int-add itau m))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (zgelqf m n a lda
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
       (declare (ignore var-2 var-4 var-5 var-6))
       (setf m var-0)
       (setf n var-1)
       (setf lda var-3)
       (setf ierr var-7))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
          czero czero
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 2)
                                ((1 lda) (1 *)) a-%offset%)
          lda)
       (declare (ignore var-0 var-1 var-2 var-5))
       (when var-3 (setf czero var-3))
       (when var-4 (setf czero var-4))
       (when var-6 (setf lda var-6)))
     (setf ie 1) (setf itauq 1)
     (setf itaup (f2cl-lib:int-add itauq m))
     (setf nwork (f2cl-lib:int-add itaup m))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (zgebrd m m a lda s
          (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                                rwork-%offset%)
```

```
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
    (setf m var-0)
    (setf m var-1)
    (setf lda var-3)
    (setf ierr var-10))
  (setf nrwork (f2cl-lib:int-add ie m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dbdsdc "U" "N" m s
       (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                             rwork-%offset%)
       dum 1 dum 1 dum idum
       (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                             ((1 *)) rwork-%offset%)
       iwork info)
    (declare
     (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12))
    (when var-2 (setf m var-2))
    (when var-13 (setf info var-13))))
 (wntqo (setf ivt 1) (setf ldwkvt m)
  (setf il (f2cl-lib:int-add ivt (f2cl-lib:int-mul ldwkvt m)))
  (cond
   ((>= lwork
        (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                          (f2cl-lib:int-mul m m)
                          (f2cl-lib:int-mul 3 m)))
    (setf ldwrkl m) (setf chunk n))
   (t (setf ldwrkl m)
    (setf chunk
            (the f2cl-lib:integer4
                 (truncate (- lwork (* m m) (* 3 m)) m)))))
  (setf itau (f2cl-lib:int-add il (f2cl-lib:int-mul ldwrkl chunk)))
  (setf nwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgelqf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
```

```
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                           ((1 *)) work-%offset%)
     ldwrkl)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkl var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ il ldwrkl)) ((1 *)) work-%offset%)
     ldwrkl)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrkl var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zunglq m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf nwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                           ((1 *)) work-%offset%)
     ldwrkl s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
```

```
                          rwork-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                          ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                          ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                          ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkl var-3)
  (setf ierr var-10))
(setf iru (f2cl-lib:int-add ie m))
(setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul m m)))
(setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul m m)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dbdsdc "U" "I" m s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                          rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                          ((1 *)) rwork-%offset%)
     m
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                          ((1 *)) rwork-%offset%)
     m dum idum
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                          ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf m var-2))
  (when var-6 (setf m var-6))
  (when var-8 (setf m var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                          ((1 *)) rwork-%offset%)
     m u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
     var-10 var-11 var-12 var-13)
    (zunmbr "Q" "L" "N" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                           ((1 *)) work-%offset%)
     ldwrkl
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwrkl var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ivt)
                           ((1 *)) work-%offset%)
     ldwkvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldwkvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "P" "R" "C" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                           ((1 *)) work-%offset%)
     ldwrkl
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ivt)
                           ((1 *)) work-%offset%)
     ldwkvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
```

```
    (when var-4 (setf m var-4))
    (when var-5 (setf m var-5))
    (when var-7 (setf ldwrkl var-7))
    (when var-10 (setf ldwkvt var-10))
    (when var-13 (setf ierr var-13)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
                ((> i n) nil)
    (tagbody
      (setf blk
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
               (the f2cl-lib:integer4 chunk)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (zgemm "N" "N" m blk m cone
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                 (ivt) ((1 *)) work-%offset%)
           m
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda czero
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                 (il) ((1 *)) work-%offset%)
           ldwrkl)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf blk var-3))
        (when var-4 (setf m var-4))
        (when var-5 (setf cone var-5))
        (when var-7 (setf m var-7))
        (when var-9 (setf lda var-9))
        (when var-10 (setf czero var-10))
        (when var-12 (setf ldwrkl var-12)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlacpy "F" m blk
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                 (il) ((1 *)) work-%offset%)
           ldwrkl
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf blk var-2))
        (when var-4 (setf ldwrkl var-4))
        (when var-6 (setf lda var-6)))
     label40)))
```

```
(wntqs (setf il 1) (setf ldwrkl m)
 (setf itau (f2cl-lib:int-add il (f2cl-lib:int-mul ldwrkl m)))
 (setf nwork (f2cl-lib:int-add itau m))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (zgelqf m n a lda
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-7))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (zlacpy "L" m m a lda
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                            ((1 *)) work-%offset%)
      ldwrkl)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf m var-1))
   (when var-2 (setf m var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldwrkl var-6)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
      czero czero
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            ((+ il ldwrkl)) ((1 *)) work-%offset%)
      ldwrkl)
   (declare (ignore var-0 var-1 var-2 var-5))
   (when var-3 (setf czero var-3))
   (when var-4 (setf czero var-4))
   (when var-6 (setf ldwrkl var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (zunglq m n m a lda
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
   (declare (ignore var-3 var-5 var-6 var-7))
   (when var-0 (setf m var-0))
   (when var-1 (setf n var-1))
   (when var-2 (setf m var-2))
   (when var-4 (setf lda var-4))
   (when var-8 (setf ierr var-8)))
```

```
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf nwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                           ((1 *)) work-%offset%)
     ldwrkl s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkl var-3)
  (setf ierr var-10))
(setf iru (f2cl-lib:int-add ie m))
(setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul m m)))
(setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul m m)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dbdsdc "U" "I" m s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     m
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     m dum idum
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf m var-2))
  (when var-6 (setf m var-6))
  (when var-8 (setf m var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
    (zlacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     m u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "Q" "L" "N" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                           ((1 *)) work-%offset%)
     ldwrkl
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwrkl var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     m vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "P" "R" "C" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                           ((1 *)) work-%offset%)
     ldwrkl
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
```

```
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf m var-5))
    (when var-7 (setf ldwrkl var-7))
    (when var-10 (setf ldvt var-10))
    (when var-13 (setf ierr var-13)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "F" m m vt ldvt
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                            ((1 *)) work-%offset%)
      ldwrkl)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf ldvt var-4))
    (when var-6 (setf ldwrkl var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (zgemm "N" "N" m n m cone
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (il)
                            ((1 *)) work-%offset%)
      ldwrkl a lda czero vt ldvt)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf ldwrkl var-7))
    (when var-9 (setf lda var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf ldvt var-12))))
 (wntqa (setf ivt 1) (setf ldwkvt m)
  (setf itau (f2cl-lib:int-add ivt (f2cl-lib:int-mul ldwkvt m)))
  (setf nwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgelqf m n a lda
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
```

```
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "U" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zunglq n n m vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf nwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgebrd m m a lda s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
```

```
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(setf iru (f2cl-lib:int-add ie m))
(setf irvt (f2cl-lib:int-add iru (f2cl-lib:int-mul m m)))
(setf nrwork (f2cl-lib:int-add irvt (f2cl-lib:int-mul m m)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dbdsdc "U" "I" m s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     m
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     m dum idum
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf m var-2))
  (when var-6 (setf m var-6))
  (when var-8 (setf m var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     m u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "Q" "L" "N" m m m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
```

```
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ivt)
                           ((1 *)) work-%offset%)
     ldwkvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldwkvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "P" "R" "C" m m m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ivt)
                           ((1 *)) work-%offset%)
     ldwkvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldwkvt var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (zgemm "N" "N" m n m cone
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ivt)
                           ((1 *)) work-%offset%)
     ldwkvt vt ldvt czero a lda)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
```

```
        (when var-3 (setf n var-3))
        (when var-4 (setf m var-4))
        (when var-5 (setf cone var-5))
        (when var-7 (setf ldwkvt var-7))
        (when var-9 (setf ldvt var-9))
        (when var-10 (setf czero var-10))
        (when var-12 (setf lda var-12)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlacpy "F" m n a lda vt ldvt)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldvt var-6))))))
    ((>= n mnthr2) (setf ie 1) (setf nrwork (f2cl-lib:int-add ie m))
     (setf itauq 1) (setf itaup (f2cl-lib:int-add itauq m))
     (setf nwork (f2cl-lib:int-add itaup m))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (zgebrd m n a lda s
          (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                                rwork-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
       (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
       (setf m var-0)
       (setf n var-1)
       (setf lda var-3)
       (setf ierr var-10))
     (cond
      (wntqn
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10 var-11 var-12 var-13)
           (dbdsdc "L" "N" m s
            (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                                  rwork-%offset%)
            dum 1 dum 1 dum idum
            (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                                  ((1 *)) rwork-%offset%)
            iwork info)
         (declare
          (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12))
```

```
    (when var-2 (setf m var-2))
    (when var-13 (setf info var-13))))
  (wntqo (setf irvt nrwork)
   (setf iru (f2cl-lib:int-add irvt (f2cl-lib:int-mul m m)))
   (setf nrwork (f2cl-lib:int-add iru (f2cl-lib:int-mul m m)))
   (setf ivt nwork)
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (zlacpy "L" m m a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (zungbr "Q" m m n u ldu
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf ldu var-5))
    (when var-9 (setf ierr var-9)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (zungbr "P" m n m a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))
   (setf ldwkvt m)
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                           (f2cl-lib:int-mul 3 m)))
     (setf nwork (f2cl-lib:int-add ivt (f2cl-lib:int-mul ldwkvt n)))
     (setf chunk n))
    (t
     (setf chunk
```

```
          (the f2cl-lib:integer4 (truncate (- lwork (* 3 m)) m)))
  (setf nwork
          (f2cl-lib:int-add ivt
                            (f2cl-lib:int-mul ldwkvt chunk)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dbdsdc "L" "I" m s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     m
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     m dum idum
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf m var-2))
  (when var-6 (setf m var-6))
  (when var-8 (setf m var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zlacrm m m u ldu
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ivt)
                           ((1 *)) work-%offset%)
     ldwkvt
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                           ((1 *)) rwork-%offset%))
  (declare (ignore var-2 var-4 var-6 var-8))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-3 (setf ldu var-3))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwkvt var-7)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "F" m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ivt)
                           ((1 *)) work-%offset%)
     ldwkvt u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
```

```
    (when var-2 (setf m var-2))
    (when var-4 (setf ldwkvt var-4))
    (when var-6 (setf ldu var-6)))
  (setf nrwork iru)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
                ((> i n) nil)
    (tagbody
      (setf blk
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
               (the f2cl-lib:integer4 chunk)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (zlarcm m blk
           (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                                 ((1 *)) rwork-%offset%)
           m
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                 (ivt) ((1 *)) work-%offset%)
           ldwkvt
           (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                                 ((1 *)) rwork-%offset%))
        (declare (ignore var-2 var-4 var-6 var-8))
        (when var-0 (setf m var-0))
        (when var-1 (setf blk var-1))
        (when var-3 (setf m var-3))
        (when var-5 (setf lda var-5))
        (when var-7 (setf ldwkvt var-7)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlacpy "F" m blk
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                 (ivt) ((1 *)) work-%offset%)
           ldwkvt
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf blk var-2))
        (when var-4 (setf ldwkvt var-4))
        (when var-6 (setf lda var-6)))
    label50)))
(wntqs
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (zlacpy "L" m m a lda u ldu)
```

```
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (zungbr "Q" m m n u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "U" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (zungbr "P" m n m vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf irvt nrwork)
(setf iru (f2cl-lib:int-add irvt (f2cl-lib:int-mul m m)))
(setf nrwork (f2cl-lib:int-add iru (f2cl-lib:int-mul m m)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dbdsdc "L" "I" m s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
```

```
                              ((1 *)) rwork-%offset%)
      m
      (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                              ((1 *)) rwork-%offset%)
      m dum idum
      (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                              ((1 *)) rwork-%offset%)
      iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf m var-2))
  (when var-6 (setf m var-6))
  (when var-8 (setf m var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zlacrm m m u ldu
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                              ((1 *)) rwork-%offset%)
     m a lda
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                              ((1 *)) rwork-%offset%))
  (declare (ignore var-2 var-4 var-6 var-8))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-3 (setf ldu var-3))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "F" m m a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(setf nrwork iru)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zlarcm m n
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                              ((1 *)) rwork-%offset%)
     m vt ldvt a lda
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                              ((1 *)) rwork-%offset%))
  (declare (ignore var-2 var-4 var-6 var-8))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldvt var-5))
```

```
    (when var-7 (setf lda var-7)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "F" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6))))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "L" m m a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zungbr "Q" m m n u ldu
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf ldu var-5))
    (when var-9 (setf ierr var-9)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zungbr "P" n n m vt ldvt
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf ldvt var-5))
```

```
  (when var-9 (setf ierr var-9)))
(setf irvt nrwork)
(setf iru (f2cl-lib:int-add irvt (f2cl-lib:int-mul m m)))
(setf nrwork (f2cl-lib:int-add iru (f2cl-lib:int-mul m m)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dbdsdc "L" "I" m s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     m
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     m dum idum
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf m var-2))
  (when var-6 (setf m var-6))
  (when var-8 (setf m var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zlacrm m m u ldu
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     m a lda
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                           ((1 *)) rwork-%offset%))
  (declare (ignore var-2 var-4 var-6 var-8))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-3 (setf ldu var-3))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "F" m m a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zlarcm m n
```

```
          (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                                ((1 *)) rwork-%offset%)
          m vt ldvt a lda
          (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                                ((1 *)) rwork-%offset%))
      (declare (ignore var-2 var-4 var-6 var-8))
      (when var-0 (setf m var-0))
      (when var-1 (setf n var-1))
      (when var-3 (setf m var-3))
      (when var-5 (setf ldvt var-5))
      (when var-7 (setf lda var-7)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlacpy "F" m n a lda vt ldvt)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-6 (setf ldvt var-6))))))
(t (setf ie 1) (setf nrwork (f2cl-lib:int-add ie m)) (setf itauq 1)
 (setf itaup (f2cl-lib:int-add itauq m))
 (setf nwork (f2cl-lib:int-add itaup m))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (zgebrd m n a lda s
      (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                            rwork-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-10))
 (cond
  (wntqn
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13)
       (dbdsdc "L" "N" m s
        (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                              rwork-%offset%)
        dum 1 dum 1 dum idum
        (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                              ((1 *)) rwork-%offset%)
```

```
      iwork info)
    (declare
     (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12))
    (when var-2 (setf m var-2))
    (when var-13 (setf info var-13))))
  (wntqo (setf ldwkvt m) (setf ivt nwork)
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                           (f2cl-lib:int-mul 3 m)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (zlaset "F" m n czero czero
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ivt)
                                ((1 *)) work-%offset%)
          ldwkvt)
       (declare (ignore var-0 var-5))
       (when var-1 (setf m var-1))
       (when var-2 (setf n var-2))
       (when var-3 (setf czero var-3))
       (when var-4 (setf czero var-4))
       (when var-6 (setf ldwkvt var-6)))
     (setf nwork
             (f2cl-lib:int-add ivt (f2cl-lib:int-mul ldwkvt n))))
    (t
     (setf chunk
             (the f2cl-lib:integer4 (truncate (- lwork (* 3 m)) m)))
     (setf nwork
             (f2cl-lib:int-add ivt
                               (f2cl-lib:int-mul ldwkvt chunk)))))
   (setf irvt nrwork)
   (setf iru (f2cl-lib:int-add irvt (f2cl-lib:int-mul m m)))
   (setf nrwork (f2cl-lib:int-add iru (f2cl-lib:int-mul m m)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13)
       (dbdsdc "L" "I" m s
        (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                              rwork-%offset%)
        (f2cl-lib:array-slice rwork-%data% double-float (iru)
                              ((1 *)) rwork-%offset%)
        m
        (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                              ((1 *)) rwork-%offset%)
        m dum idum
        (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                              ((1 *)) rwork-%offset%)
        iwork info)
     (declare
```

```
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf m var-2))
  (when var-6 (setf m var-6))
  (when var-8 (setf m var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     m u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "Q" "L" "N" m m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(cond
 ((>= lwork
      (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                        (f2cl-lib:int-mul 3 m)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacp2 "F" m m
       (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                             ((1 *)) rwork-%offset%)
       m
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ivt)
                             ((1 *)) work-%offset%)
       ldwkvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
```

```
    (when var-4 (setf m var-4))
    (when var-6 (setf ldwkvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13)
      (zunmbr "P" "R" "C" m n m a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (itaup) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ivt)
                             ((1 *)) work-%offset%)
       ldwkvt
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (nwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf m var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldwkvt var-10))
    (when var-13 (setf ierr var-13)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "F" m n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ivt)
                             ((1 *)) work-%offset%)
       ldwkvt a lda)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldwkvt var-4))
    (when var-6 (setf lda var-6))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (zungbr "P" m n m a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (itaup) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (nwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))
  (setf nrwork iru)
```

```
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
                    ((> i n) nil)
        (tagbody
          (setf blk
                  (min
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
                   (the f2cl-lib:integer4 chunk)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (zlarcm m blk
               (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                                     ((1 *)) rwork-%offset%)
               m
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                     (1 i) ((1 lda) (1 *))
                                     a-%offset%)
               lda
               (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                     (ivt) ((1 *)) work-%offset%)
               ldwkvt
               (f2cl-lib:array-slice rwork-%data% double-float
                                     (nrwork) ((1 *))
                                     rwork-%offset%))
            (declare (ignore var-2 var-4 var-6 var-8))
            (when var-0 (setf m var-0))
            (when var-1 (setf blk var-1))
            (when var-3 (setf m var-3))
            (when var-5 (setf lda var-5))
            (when var-7 (setf ldwkvt var-7)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (zlacpy "F" m blk
               (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                     (ivt) ((1 *)) work-%offset%)
               ldwkvt
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                     (1 i) ((1 lda) (1 *))
                                     a-%offset%)
               lda)
            (declare (ignore var-0 var-3 var-5))
            (when var-1 (setf m var-1))
            (when var-2 (setf blk var-2))
            (when var-4 (setf ldwkvt var-4))
            (when var-6 (setf lda var-6)))
       label60)))))
    (wntqs (setf irvt nrwork)
     (setf iru (f2cl-lib:int-add irvt (f2cl-lib:int-mul m m)))
     (setf nrwork (f2cl-lib:int-add iru (f2cl-lib:int-mul m m)))
     (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dbdsdc "L" "I" m s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     m
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
     m dum idum
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf m var-2))
  (when var-6 (setf m var-6))
  (when var-8 (setf m var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     m u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "Q" "L" "N" m m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
      (zlaset "F" m n czero czero vt ldvt)
    (declare (ignore var-0 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacp2 "F" m m
       (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                             ((1 *)) rwork-%offset%)
       m vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf m var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (zunmbr "P" "R" "C" m n m a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                             ((1 *)) work-%offset%)
       vt ldvt
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf m var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldvt var-10))
    (when var-13 (setf ierr var-13))))
 (t (setf irvt nrwork)
  (setf iru (f2cl-lib:int-add irvt (f2cl-lib:int-mul m m)))
  (setf nrwork (f2cl-lib:int-add iru (f2cl-lib:int-mul m m)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dbdsdc "L" "I" m s
       (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                             rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% double-float (iru)
                             ((1 *)) rwork-%offset%)
       m
       (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                             ((1 *)) rwork-%offset%)
       m dum idum
```

```
     (f2cl-lib:array-slice rwork-%data% double-float (nrwork)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (when var-2 (setf m var-2))
  (when var-6 (setf m var-6))
  (when var-8 (setf m var-8))
  (when var-13 (setf info var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% double-float (iru)
                           ((1 *)) rwork-%offset%)
     m u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "Q" "L" "N" m m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "F" n n czero cone vt ldvt)
  (declare (ignore var-0 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf czero var-3))
  (when var-4 (setf cone var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacp2 "F" m m
     (f2cl-lib:array-slice rwork-%data% double-float (irvt)
                           ((1 *)) rwork-%offset%)
```

```
            m vt ldvt)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf m var-2))
          (when var-4 (setf m var-4))
          (when var-6 (setf ldvt var-6)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11 var-12 var-13)
            (zunmbr "P" "R" "C" n n m a lda
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                                   ((1 *)) work-%offset%)
             vt ldvt
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (nwork)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
          (declare
           (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
          (when var-3 (setf n var-3))
          (when var-4 (setf n var-4))
          (when var-5 (setf m var-5))
          (when var-7 (setf lda var-7))
          (when var-10 (setf ldvt var-10))
          (when var-13 (setf ierr var-13)))))))))
(cond
 ((= iscl 1)
  (if (> anrm bignum)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (dlascl "G" 0 0 bignum anrm minmn 1 s minmn ierr)
        (declare (ignore var-0 var-1 var-2 var-6 var-7))
        (when var-3 (setf bignum var-3))
        (when var-4 (setf anrm var-4))
        (when var-5 (setf minmn var-5))
        (when var-8 (setf minmn var-8))
        (when var-9 (setf ierr var-9))))
  (if (and (/= info 0) (> anrm bignum))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (dlascl "G" 0 0 bignum anrm (f2cl-lib:int-sub minmn 1) 1
           (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                                 rwork-%offset%)
           minmn ierr)
        (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
        (when var-3 (setf bignum var-3))
        (when var-4 (setf anrm var-4))
        (when var-8 (setf minmn var-8))
        (when var-9 (setf ierr var-9))))
  (if (< anrm smlnum)
      (multiple-value-bind
```

```
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (dlascl "G" 0 0 smlnum anrm minmn 1 s minmn ierr)
                (declare (ignore var-0 var-1 var-2 var-6 var-7))
                (when var-3 (setf smlnum var-3))
                (when var-4 (setf anrm var-4))
                (when var-5 (setf minmn var-5))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf ierr var-9))))
          (if (and (/= info 0) (< anrm smlnum))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (dlascl "G" 0 0 smlnum anrm (f2cl-lib:int-sub minmn 1) 1
                   (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                                         rwork-%offset%)
                   minmn ierr)
                (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
                (when var-3 (setf smlnum var-3))
                (when var-4 (setf anrm var-4))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf ierr var-9))))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce maxwrk 'f2cl-lib:complex16))
        (go end_label)
       end_label
        (return
         (values jobz m n nil lda nil nil ldu nil ldvt nil nil nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgesdd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
```

```
  (array double-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::jobz
  fortran-to-lisp::m
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil
  fortran-to-lisp::ldu nil
  fortran-to-lisp::ldvt nil nil nil
  nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zgelqf
  fortran-to-lisp::zgebrd
  fortran-to-lisp::zgeqrf
  fortran-to-lisp::xerbla
  fortran-to-lisp::ilaenv))))
```

———-

31.3 zgesvd

31.3.1 Axiom unit tests

— **zgesvd.input** —

```
)set break resume
)sys rm -f zgesvd.output
)spool zgesvd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

31.3.2 Axiom help page

— **zgesvd.help** —

```
====================================================================
zgesvd examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgesvd (JOBU, JOBVT, M, N, A, LDA, S, U, LDU, VT, LDVT,
           WORK, LWORK, RWORK, INFO)
            ZGESVD computes the singular value decomposition (SVD) for GE
           matrices

Function/Subroutine Documentation
   subroutine zgesvd (character JOBU, character JOBVT, integer M, integer N,
       complex*16, dimension( lda, * ) A, integer LDA, double precision,
       dimension( * ) S, complex*16, dimension( ldu, * ) U, integer LDU,
       complex*16, dimension( ldvt, * ) VT, integer LDVT, complex*16,
       dimension( * ) WORK, integer LWORK, double precision, dimension( * )
       RWORK, integer INFO)
        ZGESVD computes the singular value decomposition (SVD) for GE matrices

Purpose:
   ZGESVD computes the singular value decomposition (SVD) of a complex
   M-by-N matrix A, optionally computing the left and/or right singular
   vectors. The SVD is written

   A = U * SIGMA * conjugate-transpose(V)

   where SIGMA is an M-by-N matrix which is zero except for its
   min(m,n) diagonal elements, U is an M-by-M unitary matrix, and
   V is an N-by-N unitary matrix.  The diagonal elements of SIGMA
   are the singular values of A; they are real and non-negative, and
   are returned in descending order.  The first min(m,n) columns of
   U and V are the left and right singular vectors of A.

   Note that the routine returns V**H, not V.

Parameters:

 JOBU
   JOBU is CHARACTER*1
   Specifies options for computing all or part of the matrix U:
   = 'A':  all M columns of U are returned in array U:
   = 'S':  the first min(m,n) columns of U (the left singular
           vectors) are returned in the array U;
   = 'O':  the first min(m,n) columns of U (the left singular
           vectors) are overwritten on the array A;
```

```
  = 'N':  no columns of U (no left singular vectors) are computed.

JOBVT
  JOBVT is CHARACTER*1
  Specifies options for computing all or part of the matrix V**H:
  = 'A':  all N rows of V**H are returned in the array VT;
  = 'S':  the first min(m,n) rows of V**H (the right singular
          vectors) are returned in the array VT;
  = 'O':  the first min(m,n) rows of V**H (the right singular
          vectors) are overwritten on the array A;
  = 'N':  no rows of V**H (no right singular vectors) are computed.

  JOBVT and JOBU cannot both be 'O'.

M
  M is INTEGER
  The number of rows of the input matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the input matrix A.  N >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit,
  if JOBU = 'O',  A is overwritten with the first min(m,n)
       columns of U (the left singular vectors, stored columnwise);
  if JOBVT = 'O', A is overwritten with the first min(m,n)
       rows of V**H (the right singular vectors, stored rowwise);
  if JOBU .ne. 'O' and JOBVT .ne. 'O', the contents of A
       are destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

S
  S is DOUBLE PRECISION array, dimension (min(M,N))
  The singular values of A, sorted so that S(i) >= S(i+1).

U
  U is COMPLEX*16 array, dimension (LDU,UCOL)
  (LDU,M) if JOBU = 'A' or (LDU,min(M,N)) if JOBU = 'S'.
  If JOBU = 'A', U contains the M-by-M unitary matrix U;
  if JOBU = 'S', U contains the first min(m,n) columns of U
  (the left singular vectors, stored columnwise);
  if JOBU = 'N' or 'O', U is not referenced.

LDU
```

```
  LDU is INTEGER
  The leading dimension of the array U.  LDU >= 1; if
  JOBU = 'S' or 'A', LDU >= M.

VT
  VT is COMPLEX*16 array, dimension (LDVT,N)
  If JOBVT = 'A', VT contains the N-by-N unitary matrix V**H;
  if JOBVT = 'S', VT contains the first min(m,n) rows of
  V**H (the right singular vectors, stored rowwise);
  if JOBVT = 'N' or 'O', VT is not referenced.

LDVT
  LDVT is INTEGER
  The leading dimension of the array VT.  LDVT >= 1; if
  JOBVT = 'A', LDVT >= N; if JOBVT = 'S', LDVT >= min(M,N).

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  LWORK >=  MAX(1,2*MIN(M,N)+MAX(M,N)).
  For good performance, LWORK should generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (5*min(M,N))
  On exit, if INFO > 0, RWORK(1:MIN(M,N)-1) contains the
  unconverged superdiagonal elements of an upper bidiagonal
  matrix B whose diagonal is in S (not necessarily sorted).
  B satisfies A = U * B * VT, so it has the same singular
  values as A, and singular vectors related by U and VT.

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if ZBDSQR did not converge, INFO specifies how many
        superdiagonals of an intermediate bidiagonal form B
        did not converge to zero. See the description of RWORK
        above for details.
```

———-

31.3.3 fortran code

— zgesvd.f —

```
*  =====================================================================
      SUBROUTINE ZGESVD( JOBU, JOBVT, M, N, A, LDA, S, U, LDU,
     $                   VT, LDVT, WORK, LWORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          JOBU, JOBVT
      INTEGER            INFO, LDA, LDU, LDVT, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   RWORK( * ), S( * )
      COMPLEX*16         A( LDA, * ), U( LDU, * ), VT( LDVT, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0D0, 0.0D0 ),
     $                   CONE = ( 1.0D0, 0.0D0 ) )
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, WNTUA, WNTUAS, WNTUN, WNTUO, WNTUS,
     $                   WNTVA, WNTVAS, WNTVN, WNTVO, WNTVS
      INTEGER            BLK, CHUNK, I, IE, IERR, IR, IRWORK, ISCL,
     $                   ITAU, ITAUP, ITAUQ, IU, IWORK, LDWRKR, LDWRKU,
     $                   MAXWRK, MINMN, MINWRK, MNTHR, NCU, NCVT, NRU,
     $                   NRVT, WRKBL
      INTEGER            LWORK_ZGEQRF, LWORK_ZUNGQR_N, LWORK_ZUNGQR_M,
     $                   LWORK_ZGEBRD, LWORK_ZUNGBR_P, LWORK_ZUNGBR_Q,
     $                   LWORK_ZGELQF, LWORK_ZUNGLQ_N, LWORK_ZUNGLQ_M
      DOUBLE PRECISION   ANRM, BIGNUM, EPS, SMLNUM
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   DUM( 1 )
      COMPLEX*16         CDUM( 1 )
*     ..
*     .. External Subroutines ..
```

```
      EXTERNAL           DLASCL, XERBLA, ZBDSQR, ZGEBRD, ZGELQF, ZGEMM,
     $                   ZGEQRF, ZLACPY, ZLASCL, ZLASET, ZUNGBR, ZUNGLQ,
     $                   ZUNGQR, ZUNMBR
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, ZLANGE
      EXTERNAL           LSAME, ILAENV, DLAMCH, ZLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      MINMN = MIN( M, N )
      WNTUA = LSAME( JOBU, 'A' )
      WNTUS = LSAME( JOBU, 'S' )
      WNTUAS = WNTUA .OR. WNTUS
      WNTUO = LSAME( JOBU, 'O' )
      WNTUN = LSAME( JOBU, 'N' )
      WNTVA = LSAME( JOBVT, 'A' )
      WNTVS = LSAME( JOBVT, 'S' )
      WNTVAS = WNTVA .OR. WNTVS
      WNTVO = LSAME( JOBVT, 'O' )
      WNTVN = LSAME( JOBVT, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
*
      IF( .NOT.( WNTUA .OR. WNTUS .OR. WNTUO .OR. WNTUN ) ) THEN
         INFO = -1
      ELSE IF( .NOT.( WNTVA .OR. WNTVS .OR. WNTVO .OR. WNTVN ) .OR.
     $         ( WNTVO .AND. WNTUO ) ) THEN
         INFO = -2
      ELSE IF( M.LT.0 ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -6
      ELSE IF( LDU.LT.1 .OR. ( WNTUAS .AND. LDU.LT.M ) ) THEN
         INFO = -9
      ELSE IF( LDVT.LT.1 .OR. ( WNTVA .AND. LDVT.LT.N ) .OR.
     $         ( WNTVS .AND. LDVT.LT.MINMN ) ) THEN
         INFO = -11
      END IF
*
*     Compute workspace
```

```
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       CWorkspace refers to complex workspace, and RWorkspace to
*       real workspace. NB refers to the optimal block size for the
*       immediately following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = 1
         MAXWRK = 1
         IF( M.GE.N .AND. MINMN.GT.0 ) THEN
*
*           Space needed for ZBDSQR is BDSPAC = 5*N
*
            MNTHR = ILAENV( 6, 'ZGESVD', JOBU // JOBVT, M, N, 0, 0 )
*           Compute space needed for ZGEQRF
            CALL ZGEQRF( M, N, A, LDA, CDUM(1), CDUM(1), -1, IERR )
            LWORK_ZGEQRF=CDUM(1)
*           Compute space needed for ZUNGQR
            CALL ZUNGQR( M, N, N, A, LDA, CDUM(1), CDUM(1), -1, IERR )
            LWORK_ZUNGQR_N=CDUM(1)
            CALL ZUNGQR( M, M, N, A, LDA, CDUM(1), CDUM(1), -1, IERR )
            LWORK_ZUNGQR_M=CDUM(1)
*           Compute space needed for ZGEBRD
            CALL ZGEBRD( N, N, A, LDA, S, DUM(1), CDUM(1),
     $                   CDUM(1), CDUM(1), -1, IERR )
            LWORK_ZGEBRD=CDUM(1)
*           Compute space needed for ZUNGBR
            CALL ZUNGBR( 'P', N, N, N, A, LDA, CDUM(1),
     $                   CDUM(1), -1, IERR )
            LWORK_ZUNGBR_P=CDUM(1)
            CALL ZUNGBR( 'Q', N, N, N, A, LDA, CDUM(1),
     $                   CDUM(1), -1, IERR )
            LWORK_ZUNGBR_Q=CDUM(1)
*
            IF( M.GE.MNTHR ) THEN
               IF( WNTUN ) THEN
*
*                 Path 1 (M much larger than N, JOBU='N')
*
                  MAXWRK = N + LWORK_ZGEQRF
                  MAXWRK = MAX( MAXWRK, 2*N+LWORK_ZGEBRD )
                  IF( WNTVO .OR. WNTVAS )
     $               MAXWRK = MAX( MAXWRK, 2*N+LWORK_ZUNGBR_P )
                  MINWRK = 3*N
               ELSE IF( WNTUO .AND. WNTVN ) THEN
*
*                 Path 2 (M much larger than N, JOBU='O', JOBVT='N')
*
                  WRKBL = N + LWORK_ZGEQRF
```

```
                 WRKBL = MAX( WRKBL, N+LWORK_ZUNGQR_N )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_ZGEBRD )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_ZUNGBR_Q )
                 MAXWRK = MAX( N*N+WRKBL, N*N+M*N )
                 MINWRK = 2*N + M
              ELSE IF( WNTUO .AND. WNTVAS ) THEN
*
*                Path 3 (M much larger than N, JOBU='O', JOBVT='S' or
*                'A')
*
                 WRKBL = N + LWORK_ZGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_ZUNGQR_N )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_ZGEBRD )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_ZUNGBR_Q )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_ZUNGBR_P )
                 MAXWRK = MAX( N*N+WRKBL, N*N+M*N )
                 MINWRK = 2*N + M
              ELSE IF( WNTUS .AND. WNTVN ) THEN
*
*                Path 4 (M much larger than N, JOBU='S', JOBVT='N')
*
                 WRKBL = N + LWORK_ZGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_ZUNGQR_N )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_ZGEBRD )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_ZUNGBR_Q )
                 MAXWRK = N*N + WRKBL
                 MINWRK = 2*N + M
              ELSE IF( WNTUS .AND. WNTVO ) THEN
*
*                Path 5 (M much larger than N, JOBU='S', JOBVT='O')
*
                 WRKBL = N + LWORK_ZGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_ZUNGQR_N )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_ZGEBRD )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_ZUNGBR_Q )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_ZUNGBR_P )
                 MAXWRK = 2*N*N + WRKBL
                 MINWRK = 2*N + M
              ELSE IF( WNTUS .AND. WNTVAS ) THEN
*
*                Path 6 (M much larger than N, JOBU='S', JOBVT='S' or
*                'A')
*
                 WRKBL = N + LWORK_ZGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_ZUNGQR_N )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_ZGEBRD )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_ZUNGBR_Q )
                 WRKBL = MAX( WRKBL, 2*N+LWORK_ZUNGBR_P )
                 MAXWRK = N*N + WRKBL
                 MINWRK = 2*N + M
```

```
               ELSE IF( WNTUA .AND. WNTVN ) THEN
*
*                 Path 7 (M much larger than N, JOBU='A', JOBVT='N')
*
                  WRKBL = N + LWORK_ZGEQRF
                  WRKBL = MAX( WRKBL, N+LWORK_ZUNGQR_M )
                  WRKBL = MAX( WRKBL, 2*N+LWORK_ZGEBRD )
                  WRKBL = MAX( WRKBL, 2*N+LWORK_ZUNGBR_Q )
                  MAXWRK = N*N + WRKBL
                  MINWRK = 2*N + M
               ELSE IF( WNTUA .AND. WNTVO ) THEN
*
*                 Path 8 (M much larger than N, JOBU='A', JOBVT='O')
*
                  WRKBL = N + LWORK_ZGEQRF
                  WRKBL = MAX( WRKBL, N+LWORK_ZUNGQR_M )
                  WRKBL = MAX( WRKBL, 2*N+LWORK_ZGEBRD )
                  WRKBL = MAX( WRKBL, 2*N+LWORK_ZUNGBR_Q )
                  WRKBL = MAX( WRKBL, 2*N+LWORK_ZUNGBR_P )
                  MAXWRK = 2*N*N + WRKBL
                  MINWRK = 2*N + M
               ELSE IF( WNTUA .AND. WNTVAS ) THEN
*
*                 Path 9 (M much larger than N, JOBU='A', JOBVT='S' or
*                 'A')
*
                  WRKBL = N + LWORK_ZGEQRF
                  WRKBL = MAX( WRKBL, N+LWORK_ZUNGQR_M )
                  WRKBL = MAX( WRKBL, 2*N+LWORK_ZGEBRD )
                  WRKBL = MAX( WRKBL, 2*N+LWORK_ZUNGBR_Q )
                  WRKBL = MAX( WRKBL, 2*N+LWORK_ZUNGBR_P )
                  MAXWRK = N*N + WRKBL
                  MINWRK = 2*N + M
               END IF
            ELSE
*
*              Path 10 (M at least N, but not much larger)
*
               CALL ZGEBRD( M, N, A, LDA, S, DUM(1), CDUM(1),
     $                    CDUM(1), CDUM(1), -1, IERR )
               LWORK_ZGEBRD=CDUM(1)
               MAXWRK = 2*N + LWORK_ZGEBRD
               IF( WNTUS .OR. WNTUO ) THEN
                  CALL ZUNGBR( 'Q', M, N, N, A, LDA, CDUM(1),
     $                    CDUM(1), -1, IERR )
                  LWORK_ZUNGBR_Q=CDUM(1)
                  MAXWRK = MAX( MAXWRK, 2*N+LWORK_ZUNGBR_Q )
               END IF
               IF( WNTUA ) THEN
                  CALL ZUNGBR( 'Q', M, M, N, A, LDA, CDUM(1),
```

```
     $                  CDUM(1), -1, IERR )
                   LWORK_ZUNGBR_Q=CDUM(1)
                   MAXWRK = MAX( MAXWRK, 2*N+LWORK_ZUNGBR_Q )
                END IF
                IF( .NOT.WNTVN ) THEN
                   MAXWRK = MAX( MAXWRK, 2*N+LWORK_ZUNGBR_P )
                MINWRK = 2*N + M
                END IF
             END IF
          ELSE IF( MINMN.GT.0 ) THEN
*
*            Space needed for ZBDSQR is BDSPAC = 5*M
*
             MNTHR = ILAENV( 6, 'ZGESVD', JOBU // JOBVT, M, N, 0, 0 )
*            Compute space needed for ZGELQF
             CALL ZGELQF( M, N, A, LDA, CDUM(1), CDUM(1), -1, IERR )
             LWORK_ZGELQF=CDUM(1)
*            Compute space needed for ZUNGLQ
             CALL ZUNGLQ( N, N, M, CDUM(1), N, CDUM(1), CDUM(1), -1,
     $                    IERR )
             LWORK_ZUNGLQ_N=CDUM(1)
             CALL ZUNGLQ( M, N, M, A, LDA, CDUM(1), CDUM(1), -1, IERR )
             LWORK_ZUNGLQ_M=CDUM(1)
*            Compute space needed for ZGEBRD
             CALL ZGEBRD( M, M, A, LDA, S, DUM(1), CDUM(1),
     $                    CDUM(1), CDUM(1), -1, IERR )
             LWORK_ZGEBRD=CDUM(1)
*             Compute space needed for ZUNGBR P
             CALL ZUNGBR( 'P', M, M, M, A, N, CDUM(1),
     $                    CDUM(1), -1, IERR )
             LWORK_ZUNGBR_P=CDUM(1)
*            Compute space needed for ZUNGBR Q
             CALL ZUNGBR( 'Q', M, M, M, A, N, CDUM(1),
     $                    CDUM(1), -1, IERR )
             LWORK_ZUNGBR_Q=CDUM(1)
             IF( N.GE.MNTHR ) THEN
                IF( WNTVN ) THEN
*
*                  Path 1t(N much larger than M, JOBVT='N')
*
                   MAXWRK = M + LWORK_ZGELQF
                   MAXWRK = MAX( MAXWRK, 2*M+LWORK_ZGEBRD )
                   IF( WNTUO .OR. WNTUAS )
     $                MAXWRK = MAX( MAXWRK, 2*M+LWORK_ZUNGBR_Q )
                   MINWRK = 3*M
                ELSE IF( WNTVO .AND. WNTUN ) THEN
*
*                  Path 2t(N much larger than M, JOBU='N', JOBVT='O')
*
                   WRKBL = M + LWORK_ZGELQF
```

```
                 WRKBL = MAX( WRKBL, M+LWORK_ZUNGLQ_M )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_ZGEBRD )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_ZUNGBR_P )
                 MAXWRK = MAX( M*M+WRKBL, M*M+M*N )
                 MINWRK = 2*M + N
              ELSE IF( WNTVO .AND. WNTUAS ) THEN
*
*                Path 3t(N much larger than M, JOBU='S' or 'A',
*                JOBVT='O')
*
                 WRKBL = M + LWORK_ZGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_ZUNGLQ_M )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_ZGEBRD )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_ZUNGBR_P )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_ZUNGBR_Q )
                 MAXWRK = MAX( M*M+WRKBL, M*M+M*N )
                 MINWRK = 2*M + N
              ELSE IF( WNTVS .AND. WNTUN ) THEN
*
*                Path 4t(N much larger than M, JOBU='N', JOBVT='S')
*
                 WRKBL = M + LWORK_ZGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_ZUNGLQ_M )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_ZGEBRD )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_ZUNGBR_P )
                 MAXWRK = M*M + WRKBL
                 MINWRK = 2*M + N
              ELSE IF( WNTVS .AND. WNTUO ) THEN
*
*                Path 5t(N much larger than M, JOBU='O', JOBVT='S')
*
                 WRKBL = M + LWORK_ZGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_ZUNGLQ_M )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_ZGEBRD )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_ZUNGBR_P )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_ZUNGBR_Q )
                 MAXWRK = 2*M*M + WRKBL
                 MINWRK = 2*M + N
              ELSE IF( WNTVS .AND. WNTUAS ) THEN
*
*                Path 6t(N much larger than M, JOBU='S' or 'A',
*                JOBVT='S')
*
                 WRKBL = M + LWORK_ZGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_ZUNGLQ_M )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_ZGEBRD )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_ZUNGBR_P )
                 WRKBL = MAX( WRKBL, 2*M+LWORK_ZUNGBR_Q )
                 MAXWRK = M*M + WRKBL
                 MINWRK = 2*M + N
```

```
               ELSE IF( WNTVA .AND. WNTUN ) THEN
*
*                 Path 7t(N much larger than M, JOBU='N', JOBVT='A')
*
                  WRKBL = M + LWORK_ZGELQF
                  WRKBL = MAX( WRKBL, M+LWORK_ZUNGLQ_N )
                  WRKBL = MAX( WRKBL, 2*M+LWORK_ZGEBRD )
                  WRKBL = MAX( WRKBL, 2*M+LWORK_ZUNGBR_P )
                  MAXWRK = M*M + WRKBL
                  MINWRK = 2*M + N
               ELSE IF( WNTVA .AND. WNTUO ) THEN
*
*                 Path 8t(N much larger than M, JOBU='O', JOBVT='A')
*
                  WRKBL = M + LWORK_ZGELQF
                  WRKBL = MAX( WRKBL, M+LWORK_ZUNGLQ_N )
                  WRKBL = MAX( WRKBL, 2*M+LWORK_ZGEBRD )
                  WRKBL = MAX( WRKBL, 2*M+LWORK_ZUNGBR_P )
                  WRKBL = MAX( WRKBL, 2*M+LWORK_ZUNGBR_Q )
                  MAXWRK = 2*M*M + WRKBL
                  MINWRK = 2*M + N
               ELSE IF( WNTVA .AND. WNTUAS ) THEN
*
*                 Path 9t(N much larger than M, JOBU='S' or 'A',
*                 JOBVT='A')
*
                  WRKBL = M + LWORK_ZGELQF
                  WRKBL = MAX( WRKBL, M+LWORK_ZUNGLQ_N )
                  WRKBL = MAX( WRKBL, 2*M+LWORK_ZGEBRD )
                  WRKBL = MAX( WRKBL, 2*M+LWORK_ZUNGBR_P )
                  WRKBL = MAX( WRKBL, 2*M+LWORK_ZUNGBR_Q )
                  MAXWRK = M*M + WRKBL
                  MINWRK = 2*M + N
               END IF
            ELSE
*
*              Path 10t(N greater than M, but not much larger)
*
               CALL ZGEBRD( M, N, A, LDA, S, DUM(1), CDUM(1),
     $                    CDUM(1), CDUM(1), -1, IERR )
               LWORK_ZGEBRD=CDUM(1)
               MAXWRK = 2*M + LWORK_ZGEBRD
               IF( WNTVS .OR. WNTVO ) THEN
*                Compute space needed for ZUNGBR P
                 CALL ZUNGBR( 'P', M, N, M, A, N, CDUM(1),
     $                    CDUM(1), -1, IERR )
                 LWORK_ZUNGBR_P=CDUM(1)
                 MAXWRK = MAX( MAXWRK, 2*M+LWORK_ZUNGBR_P )
               END IF
               IF( WNTVA ) THEN
```

```
                CALL ZUNGBR( 'P', N,  N, M, A, N, CDUM(1),
     $                  CDUM(1), -1, IERR )
                LWORK_ZUNGBR_P=CDUM(1)
                MAXWRK = MAX( MAXWRK, 2*M+LWORK_ZUNGBR_P )
             END IF
             IF( .NOT.WNTUN ) THEN
                MAXWRK = MAX( MAXWRK, 2*M+LWORK_ZUNGBR_Q )
             MINWRK = 2*M + N
             END IF
          END IF
       END IF
       MAXWRK = MAX( MAXWRK, MINWRK )
       WORK( 1 ) = MAXWRK
*
       IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
          INFO = -13
       END IF
    END IF
*
    IF( INFO.NE.0 ) THEN
       CALL XERBLA( 'ZGESVD', -INFO )
       RETURN
    ELSE IF( LQUERY ) THEN
       RETURN
    END IF
*
*   Quick return if possible
*
    IF( M.EQ.0 .OR. N.EQ.0 ) THEN
       RETURN
    END IF
*
*   Get machine constants
*
    EPS = DLAMCH( 'P' )
    SMLNUM = SQRT( DLAMCH( 'S' ) ) / EPS
    BIGNUM = ONE / SMLNUM
*
*   Scale A if max element outside range [SMLNUM,BIGNUM]
*
    ANRM = ZLANGE( 'M', M, N, A, LDA, DUM )
    ISCL = 0
    IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
       ISCL = 1
       CALL ZLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, IERR )
    ELSE IF( ANRM.GT.BIGNUM ) THEN
       ISCL = 1
       CALL ZLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, IERR )
    END IF
*
```

```
      IF( M.GE.N ) THEN
*
*        A has at least as many rows as columns. If A has sufficiently
*        more rows than columns, first reduce using the QR
*        decomposition (if sufficient workspace available)
*
         IF( M.GE.MNTHR ) THEN
*
            IF( WNTUN ) THEN
*
*              Path 1 (M much larger than N, JOBU='N')
*              No left singular vectors to be computed
*
               ITAU = 1
               IWORK = ITAU + N
*
*              Compute A=Q*R
*              (CWorkspace: need 2*N, prefer N+N*NB)
*              (RWorkspace: need 0)
*
               CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
     $                    LWORK-IWORK+1, IERR )
*
*              Zero out below R
*
               CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO, A( 2, 1 ),
     $                    LDA )
               IE = 1
               ITAUQ = 1
               ITAUP = ITAUQ + N
               IWORK = ITAUP + N
*
*              Bidiagonalize R in A
*              (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
*              (RWorkspace: need N)
*
               CALL ZGEBRD( N, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                    WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                    IERR )
               NCVT = 0
               IF( WNTVO .OR. WNTVAS ) THEN
*
*                If right singular vectors desired, generate P'.
*                (CWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*                (RWorkspace: 0)
*
                 CALL ZUNGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
                 NCVT = N
               END IF
```

```
               IRWORK = IE + N
*
*              Perform bidiagonal QR iteration, computing right
*              singular vectors of A in A if desired
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAC)
*
               CALL ZBDSQR( 'U', N, NCVT, 0, 0, S, RWORK( IE ), A, LDA,
     $                      CDUM, 1, CDUM, 1, RWORK( IRWORK ), INFO )
*
*              If right singular vectors desired in VT, copy them there
*
               IF( WNTVAS )
     $            CALL ZLACPY( 'F', N, N, A, LDA, VT, LDVT )
*
            ELSE IF( WNTUO .AND. WNTVN ) THEN
*
*              Path 2 (M much larger than N, JOBU='O', JOBVT='N')
*              N left singular vectors to be overwritten on A and
*              no right singular vectors to be computed
*
               IF( LWORK.GE.N*N+3*N ) THEN
*
*                Sufficient workspace for a fast algorithm
*
                 IR = 1
                 IF( LWORK.GE.MAX( WRKBL, LDA*N )+LDA*N ) THEN
*
*                   WORK(IU) is LDA by N, WORK(IR) is LDA by N
*
                    LDWRKU = LDA
                    LDWRKR = LDA
                 ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N )+N*N ) THEN
*
*                   WORK(IU) is LDA by N, WORK(IR) is N by N
*
                    LDWRKU = LDA
                    LDWRKR = N
                 ELSE
*
*                   WORK(IU) is LDWRKU by N, WORK(IR) is N by N
*
                    LDWRKU = ( LWORK-N*N ) / N
                    LDWRKR = N
                 END IF
                 ITAU = IR + LDWRKR*N
                 IWORK = ITAU + N
*
*                Compute A=Q*R
*                (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
```

```
*                 (RWorkspace: 0)
*
                  CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Copy R to WORK(IR) and zero out below it
*
                  CALL ZLACPY( 'U', N, N, A, LDA, WORK( IR ), LDWRKR )
                  CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                         WORK( IR+1 ), LDWRKR )
*
*                 Generate Q in A
*                 (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*                 (RWorkspace: 0)
*
                  CALL ZUNGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IE = 1
                  ITAUQ = ITAU
                  ITAUP = ITAUQ + N
                  IWORK = ITAUP + N
*
*                 Bidiagonalize R in WORK(IR)
*                 (CWorkspace: need N*N+3*N, prefer N*N+2*N+2*N*NB)
*                 (RWorkspace: need N)
*
                  CALL ZGEBRD( N, N, WORK( IR ), LDWRKR, S, RWORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Generate left vectors bidiagonalizing R
*                 (CWorkspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*                 (RWorkspace: need 0)
*
                  CALL ZUNGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                         WORK( ITAUQ ), WORK( IWORK ),
     $                         LWORK-IWORK+1, IERR )
                  IRWORK = IE + N
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of R in WORK(IR)
*                 (CWorkspace: need N*N)
*                 (RWorkspace: need BDSPAC)
*
                  CALL ZBDSQR( 'U', N, 0, N, 0, S, RWORK( IE ), CDUM, 1,
     $                         WORK( IR ), LDWRKR, CDUM, 1,
     $                         RWORK( IRWORK ), INFO )
                  IU = ITAUQ
*
*                 Multiply Q in A by left singular vectors of R in
```

```
*                 WORK(IR), storing result in WORK(IU) and copying to A
*                 (CWorkspace: need N*N+N, prefer N*N+M*N)
*                 (RWorkspace: 0)
*
                  DO 10 I = 1, M, LDWRKU
                     CHUNK = MIN( M-I+1, LDWRKU )
                     CALL ZGEMM( 'N', 'N', CHUNK, N, N, CONE, A( I, 1 ),
     $                           LDA, WORK( IR ), LDWRKR, CZERO,
     $                           WORK( IU ), LDWRKU )
                     CALL ZLACPY( 'F', CHUNK, N, WORK( IU ), LDWRKU,
     $                            A( I, 1 ), LDA )
   10             CONTINUE
*
               ELSE
*
*                 Insufficient workspace for a fast algorithm
*
                  IE = 1
                  ITAUQ = 1
                  ITAUP = ITAUQ + N
                  IWORK = ITAUP + N
*
*                 Bidiagonalize A
*                 (CWorkspace: need 2*N+M, prefer 2*N+(M+N)*NB)
*                 (RWorkspace: N)
*
                  CALL ZGEBRD( M, N, A, LDA, S, RWORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Generate left vectors bidiagonalizing A
*                 (CWorkspace: need 3*N, prefer 2*N+N*NB)
*                 (RWorkspace: 0)
*
                  CALL ZUNGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IRWORK = IE + N
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of A in A
*                 (CWorkspace: need 0)
*                 (RWorkspace: need BDSPAC)
*
                  CALL ZBDSQR( 'U', N, 0, M, 0, S, RWORK( IE ), CDUM, 1,
     $                         A, LDA, CDUM, 1, RWORK( IRWORK ), INFO )
*
               END IF
*
            ELSE IF( WNTUO .AND. WNTVAS ) THEN
*
```

```
*              Path 3 (M much larger than N, JOBU='O', JOBVT='S' or 'A')
*              N left singular vectors to be overwritten on A and
*              N right singular vectors to be computed in VT
*
               IF( LWORK.GE.N*N+3*N ) THEN
*
*                 Sufficient workspace for a fast algorithm
*
                  IR = 1
                  IF( LWORK.GE.MAX( WRKBL, LDA*N )+LDA*N ) THEN
*
*                    WORK(IU) is LDA by N and WORK(IR) is LDA by N
*
                     LDWRKU = LDA
                     LDWRKR = LDA
                  ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N )+N*N ) THEN
*
*                    WORK(IU) is LDA by N and WORK(IR) is N by N
*
                     LDWRKU = LDA
                     LDWRKR = N
                  ELSE
*
*                    WORK(IU) is LDWRKU by N and WORK(IR) is N by N
*
                     LDWRKU = ( LWORK-N*N ) / N
                     LDWRKR = N
                  END IF
                  ITAU = IR + LDWRKR*N
                  IWORK = ITAU + N
*
*                 Compute A=Q*R
*                 (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*                 (RWorkspace: 0)
*
                  CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Copy R to VT, zeroing out below it
*
                  CALL ZLACPY( 'U', N, N, A, LDA, VT, LDVT )
                  IF( N.GT.1 )
     $               CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                            VT( 2, 1 ), LDVT )
*
*                 Generate Q in A
*                 (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*                 (RWorkspace: 0)
*
                  CALL ZUNGQR( M, N, N, A, LDA, WORK( ITAU ),
```

```
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IE = 1
                  ITAUQ = ITAU
                  ITAUP = ITAUQ + N
                  IWORK = ITAUP + N
*
*                 Bidiagonalize R in VT, copying result to WORK(IR)
*                 (CWorkspace: need N*N+3*N, prefer N*N+2*N+2*N*NB)
*                 (RWorkspace: need N)
*
                  CALL ZGEBRD( N, N, VT, LDVT, S, RWORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                  CALL ZLACPY( 'L', N, N, VT, LDVT, WORK( IR ), LDWRKR )
*
*                 Generate left vectors bidiagonalizing R in WORK(IR)
*                 (CWorkspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*                 (RWorkspace: 0)
*
                  CALL ZUNGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                        WORK( ITAUQ ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                 Generate right vectors bidiagonalizing R in VT
*                 (CWorkspace: need N*N+3*N-1, prefer N*N+2*N+(N-1)*NB)
*                 (RWorkspace: 0)
*
                  CALL ZUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IRWORK = IE + N
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of R in WORK(IR) and computing right
*                 singular vectors of R in VT
*                 (CWorkspace: need N*N)
*                 (RWorkspace: need BDSPAC)
*
                  CALL ZBDSQR( 'U', N, N, N, 0, S, RWORK( IE ), VT,
     $                        LDVT, WORK( IR ), LDWRKR, CDUM, 1,
     $                        RWORK( IRWORK ), INFO )
                  IU = ITAUQ
*
*                 Multiply Q in A by left singular vectors of R in
*                 WORK(IR), storing result in WORK(IU) and copying to A
*                 (CWorkspace: need N*N+N, prefer N*N+M*N)
*                 (RWorkspace: 0)
*
                  DO 20 I = 1, M, LDWRKU
                     CHUNK = MIN( M-I+1, LDWRKU )
                     CALL ZGEMM( 'N', 'N', CHUNK, N, N, CONE, A( I, 1 ),
```

```
     $                          LDA, WORK( IR ), LDWRKR, CZERO,
     $                          WORK( IU ), LDWRKU )
                   CALL ZLACPY( 'F', CHUNK, N, WORK( IU ), LDWRKU,
     $                           A( I, 1 ), LDA )
   20            CONTINUE
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + N
*
*                Compute A=Q*R
*                (CWorkspace: need 2*N, prefer N+N*NB)
*                (RWorkspace: 0)
*
                 CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy R to VT, zeroing out below it
*
                 CALL ZLACPY( 'U', N, N, A, LDA, VT, LDVT )
                 IF( N.GT.1 )
     $              CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                           VT( 2, 1 ), LDVT )
*
*                Generate Q in A
*                (CWorkspace: need 2*N, prefer N+N*NB)
*                (RWorkspace: 0)
*
                 CALL ZUNGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = 1
                 ITAUQ = ITAU
                 ITAUP = ITAUQ + N
                 IWORK = ITAUP + N
*
*                Bidiagonalize R in VT
*                (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
*                (RWorkspace: N)
*
                 CALL ZGEBRD( N, N, VT, LDVT, S, RWORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Multiply Q in A by left vectors bidiagonalizing R
*                (CWorkspace: need 2*N+M, prefer 2*N+M*NB)
*                (RWorkspace: 0)
*
```

```
                 CALL ZUNMBR( 'Q', 'R', 'N', M, N, N, VT, LDVT,
     $                        WORK( ITAUQ ), A, LDA, WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate right vectors bidiagonalizing R in VT
*                (CWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*                (RWorkspace: 0)
*
                 CALL ZUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IRWORK = IE + N
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of A in A and computing right
*                singular vectors of A in VT
*                (CWorkspace: 0)
*                (RWorkspace: need BDSPAC)
*
                 CALL ZBDSQR( 'U', N, N, M, 0, S, RWORK( IE ), VT,
     $                        LDVT, A, LDA, CDUM, 1, RWORK( IRWORK ),
     $                        INFO )
*
              END IF
*
           ELSE IF( WNTUS ) THEN
*
              IF( WNTVN ) THEN
*
*                Path 4 (M much larger than N, JOBU='S', JOBVT='N')
*                N left singular vectors to be computed in U and
*                no right singular vectors to be computed
*
                 IF( LWORK.GE.N*N+3*N ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IR = 1
                    IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                      WORK(IR) is LDA by N
*
                       LDWRKR = LDA
                    ELSE
*
*                      WORK(IR) is N by N
*
                       LDWRKR = N
                    END IF
                    ITAU = IR + LDWRKR*N
                    IWORK = ITAU + N
```

```
*
*                    Compute A=Q*R
*                    (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*                    (RWorkspace: 0)
*
                     CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy R to WORK(IR), zeroing out below it
*
                     CALL ZLACPY( 'U', N, N, A, LDA, WORK( IR ),
     $                            LDWRKR )
                     CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                            WORK( IR+1 ), LDWRKR )
*
*                    Generate Q in A
*                    (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = 1
                     ITAUQ = ITAU
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Bidiagonalize R in WORK(IR)
*                    (CWorkspace: need N*N+3*N, prefer N*N+2*N+2*N*NB)
*                    (RWorkspace: need N)
*
                     CALL ZGEBRD( N, N, WORK( IR ), LDWRKR, S,
     $                            RWORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate left vectors bidiagonalizing R in WORK(IR)
*                    (CWorkspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                            WORK( ITAUQ ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IRWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IR)
*                    (CWorkspace: need N*N)
*                    (RWorkspace: need BDSPAC)
*
                     CALL ZBDSQR( 'U', N, 0, N, 0, S, RWORK( IE ), CDUM,
```

```
     $                          1, WORK( IR ), LDWRKR, CDUM, 1,
     $                          RWORK( IRWORK ), INFO )
*
*                    Multiply Q in A by left singular vectors of R in
*                    WORK(IR), storing result in U
*                    (CWorkspace: need N*N)
*                    (RWorkspace: 0)
*
                     CALL ZGEMM( 'N', 'N', M, N, N, CONE, A, LDA,
     $                           WORK( IR ), LDWRKR, CZERO, U, LDU )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (CWorkspace: need 2*N, prefer N+N*NB)
*                    (RWorkspace: 0)
*
                     CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL ZLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                    Generate Q in U
*                    (CWorkspace: need 2*N, prefer N+N*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGQR( M, N, N, U, LDU, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = 1
                     ITAUQ = ITAU
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Zero out below R in A
*
                     CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                            A( 2, 1 ), LDA )
*
*                    Bidiagonalize R in A
*                    (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
*                    (RWorkspace: need N)
*
                     CALL ZGEBRD( N, N, A, LDA, S, RWORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
```

```
*                    Multiply Q in U by left vectors bidiagonalizing R
*                    (CWorkspace: need 2*N+M, prefer 2*N+M*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                            WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IRWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U
*                    (CWorkspace: 0)
*                    (RWorkspace: need BDSPAC)
*
                     CALL ZBDSQR( 'U', N, 0, M, 0, S, RWORK( IE ), CDUM,
     $                            1, U, LDU, CDUM, 1, RWORK( IRWORK ),
     $                            INFO )
*
                  END IF
*
               ELSE IF( WNTVO ) THEN
*
                  Path 5 (M much larger than N, JOBU='S', JOBVT='O')
*                 N left singular vectors to be computed in U and
*                 N right singular vectors to be overwritten on A
*
                  IF( LWORK.GE.2*N*N+3*N ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+2*LDA*N ) THEN
*
*                       WORK(IU) is LDA by N and WORK(IR) is LDA by N
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*N
                        LDWRKR = LDA
                     ELSE IF( LWORK.GE.WRKBL+( LDA+N )*N ) THEN
*
*                       WORK(IU) is LDA by N and WORK(IR) is N by N
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*N
                        LDWRKR = N
                     ELSE
*
*                       WORK(IU) is N by N and WORK(IR) is N by N
*
                        LDWRKU = N
```

```
                      IR = IU + LDWRKU*N
                      LDWRKR = N
                   END IF
                   ITAU = IR + LDWRKR*N
                   IWORK = ITAU + N
*
*                  Compute A=Q*R
*                  (CWorkspace: need 2*N*N+2*N, prefer 2*N*N+N+N*NB)
*                  (RWorkspace: 0)
*
                   CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                  Copy R to WORK(IU), zeroing out below it
*
                   CALL ZLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                           LDWRKU )
                   CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                           WORK( IU+1 ), LDWRKU )
*
*                  Generate Q in A
*                  (CWorkspace: need 2*N*N+2*N, prefer 2*N*N+N+N*NB)
*                  (RWorkspace: 0)
*
                   CALL ZUNGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                   IE = 1
                   ITAUQ = ITAU
                   ITAUP = ITAUQ + N
                   IWORK = ITAUP + N
*
*                  Bidiagonalize R in WORK(IU), copying result to
*                  WORK(IR)
*                  (CWorkspace: need   2*N*N+3*N,
*                               prefer 2*N*N+2*N+2*N*NB)
*                  (RWorkspace: need   N)
*
                   CALL ZGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                           RWORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                   CALL ZLACPY( 'U', N, N, WORK( IU ), LDWRKU,
     $                           WORK( IR ), LDWRKR )
*
*                  Generate left bidiagonalizing vectors in WORK(IU)
*                  (CWorkspace: need 2*N*N+3*N, prefer 2*N*N+2*N+N*NB)
*                  (RWorkspace: 0)
*
                   CALL ZUNGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                           WORK( ITAUQ ), WORK( IWORK ),
```

```
     $                           LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in WORK(IR)
*                    (CWorkspace: need   2*N*N+3*N-1,
*                                 prefer 2*N*N+2*N+(N-1)*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGBR( 'P', N, N, N, WORK( IR ), LDWRKR,
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                     IRWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IU) and computing
*                    right singular vectors of R in WORK(IR)
*                    (CWorkspace: need 2*N*N)
*                    (RWorkspace: need BDSPAC)
*
                     CALL ZBDSQR( 'U', N, N, N, 0, S, RWORK( IE ),
     $                           WORK( IR ), LDWRKR, WORK( IU ),
     $                           LDWRKU, CDUM, 1, RWORK( IRWORK ),
     $                           INFO )
*
*                    Multiply Q in A by left singular vectors of R in
*                    WORK(IU), storing result in U
*                    (CWorkspace: need N*N)
*                    (RWorkspace: 0)
*
                     CALL ZGEMM( 'N', 'N', M, N, N, CONE, A, LDA,
     $                          WORK( IU ), LDWRKU, CZERO, U, LDU )
*
*                    Copy right singular vectors of R to A
*                    (CWorkspace: need N*N)
*                    (RWorkspace: 0)
*
                     CALL ZLACPY( 'F', N, N, WORK( IR ), LDWRKR, A,
     $                           LDA )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (CWorkspace: need 2*N, prefer N+N*NB)
*                    (RWorkspace: 0)
*
                     CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ),
```

```
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                      CALL ZLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                     Generate Q in U
*                     (CWorkspace: need 2*N, prefer N+N*NB)
*                     (RWorkspace: 0)
*
                      CALL ZUNGQR( M, N, N, U, LDU, WORK( ITAU ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                      IE = 1
                      ITAUQ = ITAU
                      ITAUP = ITAUQ + N
                      IWORK = ITAUP + N
*
*                     Zero out below R in A
*
                      CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                          A( 2, 1 ), LDA )
*
*                     Bidiagonalize R in A
*                     (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
*                     (RWorkspace: need N)
*
                      CALL ZGEBRD( N, N, A, LDA, S, RWORK( IE ),
     $                          WORK( ITAUQ ), WORK( ITAUP ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                     Multiply Q in U by left vectors bidiagonalizing R
*                     (CWorkspace: need 2*N+M, prefer 2*N+M*NB)
*                     (RWorkspace: 0)
*
                      CALL ZUNMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                          WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                          LWORK-IWORK+1, IERR )
*
*                     Generate right vectors bidiagonalizing R in A
*                     (CWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*                     (RWorkspace: 0)
*
                      CALL ZUNGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                      IRWORK = IE + N
*
*                     Perform bidiagonal QR iteration, computing left
*                     singular vectors of A in U and computing right
*                     singular vectors of A in A
*                     (CWorkspace: 0)
*                     (RWorkspace: need BDSPAC)
*
                      CALL ZBDSQR( 'U', N, N, M, 0, S, RWORK( IE ), A,
```

```
     $                          LDA, U, LDU, CDUM, 1, RWORK( IRWORK ),
     $                          INFO )
*
                 END IF
*
              ELSE IF( WNTVAS ) THEN
*
                 Path 6 (M much larger than N, JOBU='S', JOBVT='S'
*                        or 'A')
                 N left singular vectors to be computed in U and
                 N right singular vectors to be computed in VT
*
                 IF( LWORK.GE.N*N+3*N ) THEN
*
                    Sufficient workspace for a fast algorithm
*
                    IU = 1
                    IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
                       WORK(IU) is LDA by N
*
                       LDWRKU = LDA
                    ELSE
*
                       WORK(IU) is N by N
*
                       LDWRKU = N
                    END IF
                    ITAU = IU + LDWRKU*N
                    IWORK = ITAU + N
*
                    Compute A=Q*R
*                   (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*                   (RWorkspace: 0)
*
                    CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy R to WORK(IU), zeroing out below it
*
                    CALL ZLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                           LDWRKU )
                    CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                           WORK( IU+1 ), LDWRKU )
*
*                   Generate Q in A
*                   (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*                   (RWorkspace: 0)
*
                    CALL ZUNGQR( M, N, N, A, LDA, WORK( ITAU ),
```

```
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                      IE = 1
                      ITAUQ = ITAU
                      ITAUP = ITAUQ + N
                      IWORK = ITAUP + N
*
*                     Bidiagonalize R in WORK(IU), copying result to VT
*                     (CWorkspace: need N*N+3*N, prefer N*N+2*N+2*N*NB)
*                     (RWorkspace: need N)
*
                      CALL ZGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                             RWORK( IE ), WORK( ITAUQ ),
     $                             WORK( ITAUP ), WORK( IWORK ),
     $                             LWORK-IWORK+1, IERR )
                      CALL ZLACPY( 'U', N, N, WORK( IU ), LDWRKU, VT,
     $                             LDVT )
*
*                     Generate left bidiagonalizing vectors in WORK(IU)
*                     (CWorkspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*                     (RWorkspace: 0)
*
                      CALL ZUNGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                             WORK( ITAUQ ), WORK( IWORK ),
     $                             LWORK-IWORK+1, IERR )
*
*                     Generate right bidiagonalizing vectors in VT
*                     (CWorkspace: need   N*N+3*N-1,
*                                  prefer N*N+2*N+(N-1)*NB)
*                     (RWorkspace: 0)
*
                      CALL ZUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                             WORK( IWORK ), LWORK-IWORK+1, IERR )
                      IRWORK = IE + N
*
*                     Perform bidiagonal QR iteration, computing left
*                     singular vectors of R in WORK(IU) and computing
*                     right singular vectors of R in VT
*                     (CWorkspace: need N*N)
*                     (RWorkspace: need BDSPAC)
*
                      CALL ZBDSQR( 'U', N, N, N, 0, S, RWORK( IE ), VT,
     $                             LDVT, WORK( IU ), LDWRKU, CDUM, 1,
     $                             RWORK( IRWORK ), INFO )
*
*                     Multiply Q in A by left singular vectors of R in
*                     WORK(IU), storing result in U
*                     (CWorkspace: need N*N)
*                     (RWorkspace: 0)
*
                      CALL ZGEMM( 'N', 'N', M, N, N, CONE, A, LDA,
```

```
     $                          WORK( IU ), LDWRKU, CZERO, U, LDU )
*
                 ELSE
*
*                   Insufficient workspace for a fast algorithm
*
                    ITAU = 1
                    IWORK = ITAU + N
*
*                   Compute A=Q*R, copying result to U
*                   (CWorkspace: need 2*N, prefer N+N*NB)
*                   (RWorkspace: 0)
*
                    CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    CALL ZLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                   Generate Q in U
*                   (CWorkspace: need 2*N, prefer N+N*NB)
*                   (RWorkspace: 0)
*
                    CALL ZUNGQR( M, N, N, U, LDU, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy R to VT, zeroing out below it
*
                    CALL ZLACPY( 'U', N, N, A, LDA, VT, LDVT )
                    IF( N.GT.1 )
     $                 CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                              VT( 2, 1 ), LDVT )
                    IE = 1
                    ITAUQ = ITAU
                    ITAUP = ITAUQ + N
                    IWORK = ITAUP + N
*
*                   Bidiagonalize R in VT
*                   (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
*                   (RWorkspace: need N)
*
                    CALL ZGEBRD( N, N, VT, LDVT, S, RWORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Multiply Q in U by left bidiagonalizing vectors
*                   in VT
*                   (CWorkspace: need 2*N+M, prefer 2*N+M*NB)
*                   (RWorkspace: 0)
*
                    CALL ZUNMBR( 'Q', 'R', 'N', M, N, N, VT, LDVT,
     $                           WORK( ITAUQ ), U, LDU, WORK( IWORK ),
```

```
     $                           LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in VT
*                    (CWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IRWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in VT
*                    (CWorkspace: 0)
*                    (RWorkspace: need BDSPAC)
*
                     CALL ZBDSQR( 'U', N, N, M, 0, S, RWORK( IE ), VT,
     $                           LDVT, U, LDU, CDUM, 1,
     $                           RWORK( IRWORK ), INFO )
*
                  END IF
*
               END IF
*
            ELSE IF( WNTUA ) THEN
*
               IF( WNTVN ) THEN
*
*                 Path 7 (M much larger than N, JOBU='A', JOBVT='N')
*                 M left singular vectors to be computed in U and
*                 no right singular vectors to be computed
*
                  IF( LWORK.GE.N*N+MAX( N+M, 3*N ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IR = 1
                     IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                       WORK(IR) is LDA by N
*
                        LDWRKR = LDA
                     ELSE
*
*                       WORK(IR) is N by N
*
                        LDWRKR = N
                     END IF
                     ITAU = IR + LDWRKR*N
                     IWORK = ITAU + N
```

```
*
*                    Compute A=Q*R, copying result to U
*                    (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*                    (RWorkspace: 0)
*
                     CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL ZLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                    Copy R to WORK(IR), zeroing out below it
*
                     CALL ZLACPY( 'U', N, N, A, LDA, WORK( IR ),
     $                            LDWRKR )
                     CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                            WORK( IR+1 ), LDWRKR )
*
*                    Generate Q in U
*                    (CWorkspace: need N*N+N+M, prefer N*N+N+M*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = 1
                     ITAUQ = ITAU
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Bidiagonalize R in WORK(IR)
*                    (CWorkspace: need N*N+3*N, prefer N*N+2*N+2*N*NB)
*                    (RWorkspace: need N)
*
                     CALL ZGEBRD( N, N, WORK( IR ), LDWRKR, S,
     $                            RWORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors in WORK(IR)
*                    (CWorkspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                            WORK( ITAUQ ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IRWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IR)
*                    (CWorkspace: need N*N)
*                    (RWorkspace: need BDSPAC)
*
```

```
                 CALL ZBDSQR( 'U', N, 0, N, 0, S, RWORK( IE ), CDUM,
     $                        1, WORK( IR ), LDWRKR, CDUM, 1,
     $                        RWORK( IRWORK ), INFO )
*
*                Multiply Q in U by left singular vectors of R in
*                WORK(IR), storing result in A
*                (CWorkspace: need N*N)
*                (RWorkspace: 0)
*
                 CALL ZGEMM( 'N', 'N', M, N, N, CONE, U, LDU,
     $                       WORK( IR ), LDWRKR, CZERO, A, LDA )
*
*                Copy left singular vectors of A from A to U
*
                 CALL ZLACPY( 'F', M, N, A, LDA, U, LDU )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + N
*
*                Compute A=Q*R, copying result to U
*                (CWorkspace: need 2*N, prefer N+N*NB)
*                (RWorkspace: 0)
*
                 CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL ZLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                Generate Q in U
*                (CWorkspace: need N+M, prefer N+M*NB)
*                (RWorkspace: 0)
*
                 CALL ZUNGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = 1
                 ITAUQ = ITAU
                 ITAUP = ITAUQ + N
                 IWORK = ITAUP + N
*
*                Zero out below R in A
*
                 CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                        A( 2, 1 ), LDA )
*
*                Bidiagonalize R in A
*                (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
*                (RWorkspace: need N)
```

```
*
                    CALL ZGEBRD( N, N, A, LDA, S, RWORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Multiply Q in U by left bidiagonalizing vectors
*                   in A
*                   (CWorkspace: need 2*N+M, prefer 2*N+M*NB)
*                   (RWorkspace: 0)
*
                    CALL ZUNMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                           WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    IRWORK = IE + N
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of A in U
*                   (CWorkspace: 0)
*                   (RWorkspace: need BDSPAC)
*
                    CALL ZBDSQR( 'U', N, 0, M, 0, S, RWORK( IE ), CDUM,
     $                           1, U, LDU, CDUM, 1, RWORK( IRWORK ),
     $                           INFO )
*
                 END IF
*
              ELSE IF( WNTVO ) THEN
*
                 Path 8 (M much larger than N, JOBU='A', JOBVT='O')
*                M left singular vectors to be computed in U and
*                N right singular vectors to be overwritten on A
*
                 IF( LWORK.GE.2*N*N+MAX( N+M, 3*N ) ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IU = 1
                    IF( LWORK.GE.WRKBL+2*LDA*N ) THEN
*
*                      WORK(IU) is LDA by N and WORK(IR) is LDA by N
*
                       LDWRKU = LDA
                       IR = IU + LDWRKU*N
                       LDWRKR = LDA
                    ELSE IF( LWORK.GE.WRKBL+( LDA+N )*N ) THEN
*
*                      WORK(IU) is LDA by N and WORK(IR) is N by N
*
                       LDWRKU = LDA
                       IR = IU + LDWRKU*N
```

```
                      LDWRKR = N
                   ELSE
*
*                     WORK(IU) is N by N and WORK(IR) is N by N
*
                      LDWRKU = N
                      IR = IU + LDWRKU*N
                      LDWRKR = N
                   END IF
                   ITAU = IR + LDWRKR*N
                   IWORK = ITAU + N
*
*                  Compute A=Q*R, copying result to U
*                  (CWorkspace: need 2*N*N+2*N, prefer 2*N*N+N+N*NB)
*                  (RWorkspace: 0)
*
                   CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                   CALL ZLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                  Generate Q in U
*                  (CWorkspace: need 2*N*N+N+M, prefer 2*N*N+N+M*NB)
*                  (RWorkspace: 0)
*
                   CALL ZUNGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                  Copy R to WORK(IU), zeroing out below it
*
                   CALL ZLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                           LDWRKU )
                   CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                           WORK( IU+1 ), LDWRKU )
                   IE = 1
                   ITAUQ = ITAU
                   ITAUP = ITAUQ + N
                   IWORK = ITAUP + N
*
*                  Bidiagonalize R in WORK(IU), copying result to
*                  WORK(IR)
*                  (CWorkspace: need   2*N*N+3*N,
*                               prefer 2*N*N+2*N+2*N*NB)
*                  (RWorkspace: need   N)
*
                   CALL ZGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                           RWORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                   CALL ZLACPY( 'U', N, N, WORK( IU ), LDWRKU,
     $                           WORK( IR ), LDWRKR )
```

```
*
*                    Generate left bidiagonalizing vectors in WORK(IU)
*                    (CWorkspace: need 2*N*N+3*N, prefer 2*N*N+2*N+N*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                            WORK( ITAUQ ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in WORK(IR)
*                    (CWorkspace: need   2*N*N+3*N-1,
*                                 prefer 2*N*N+2*N+(N-1)*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGBR( 'P', N, N, N, WORK( IR ), LDWRKR,
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IRWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IU) and computing
*                    right singular vectors of R in WORK(IR)
*                    (CWorkspace: need 2*N*N)
*                    (RWorkspace: need BDSPAC)
*
                     CALL ZBDSQR( 'U', N, N, N, 0, S, RWORK( IE ),
     $                            WORK( IR ), LDWRKR, WORK( IU ),
     $                            LDWRKU, CDUM, 1, RWORK( IRWORK ),
     $                            INFO )
*
*                    Multiply Q in U by left singular vectors of R in
*                    WORK(IU), storing result in A
*                    (CWorkspace: need N*N)
*                    (RWorkspace: 0)
*
                     CALL ZGEMM( 'N', 'N', M, N, N, CONE, U, LDU,
     $                           WORK( IU ), LDWRKU, CZERO, A, LDA )
*
*                    Copy left singular vectors of A from A to U
*
                     CALL ZLACPY( 'F', M, N, A, LDA, U, LDU )
*
*                    Copy right singular vectors of R from WORK(IR) to A
*
                     CALL ZLACPY( 'F', N, N, WORK( IR ), LDWRKR, A,
     $                            LDA )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
```

```
*
                    ITAU = 1
                    IWORK = ITAU + N
*
*                   Compute A=Q*R, copying result to U
*                   (CWorkspace: need 2*N, prefer N+N*NB)
*                   (RWorkspace: 0)
*
                    CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    CALL ZLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                   Generate Q in U
*                   (CWorkspace: need N+M, prefer N+M*NB)
*                   (RWorkspace: 0)
*
                    CALL ZUNGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = 1
                    ITAUQ = ITAU
                    ITAUP = ITAUQ + N
                    IWORK = ITAUP + N
*
*                   Zero out below R in A
*
                    CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                           A( 2, 1 ), LDA )
*
*                   Bidiagonalize R in A
*                   (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
*                   (RWorkspace: need N)
*
                    CALL ZGEBRD( N, N, A, LDA, S, RWORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Multiply Q in U by left bidiagonalizing vectors
*                   in A
*                   (CWorkspace: need 2*N+M, prefer 2*N+M*NB)
*                   (RWorkspace: 0)
*
                    CALL ZUNMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                           WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate right bidiagonalizing vectors in A
*                   (CWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*                   (RWorkspace: 0)
*
                    CALL ZUNGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
```

```
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IRWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in A
*                    (CWorkspace: 0)
*                    (RWorkspace: need BDSPAC)
*
                     CALL ZBDSQR( 'U', N, N, M, 0, S, RWORK( IE ), A,
     $                           LDA, U, LDU, CDUM, 1, RWORK( IRWORK ),
     $                           INFO )
*
                  END IF
*
               ELSE IF( WNTVAS ) THEN
*
                  Path 9 (M much larger than N, JOBU='A', JOBVT='S'
*                         or 'A')
*                 M left singular vectors to be computed in U and
*                 N right singular vectors to be computed in VT
*
                  IF( LWORK.GE.N*N+MAX( N+M, 3*N ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                       WORK(IU) is LDA by N
*
                        LDWRKU = LDA
                     ELSE
*
*                       WORK(IU) is N by N
*
                        LDWRKU = N
                     END IF
                     ITAU = IU + LDWRKU*N
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (CWorkspace: need N*N+2*N, prefer N*N+N+N*NB)
*                    (RWorkspace: 0)
*
                     CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL ZLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                    Generate Q in U
```

```
*                    (CWorkspace: need N*N+N+M, prefer N*N+N+M*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy R to WORK(IU), zeroing out below it
*
                     CALL ZLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                            LDWRKU )
                     CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                            WORK( IU+1 ), LDWRKU )
                     IE = 1
                     ITAUQ = ITAU
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Bidiagonalize R in WORK(IU), copying result to VT
*                    (CWorkspace: need N*N+3*N, prefer N*N+2*N+2*N*NB)
*                    (RWorkspace: need N)
*
                     CALL ZGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                            RWORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     CALL ZLACPY( 'U', N, N, WORK( IU ), LDWRKU, VT,
     $                            LDVT )
*
*                    Generate left bidiagonalizing vectors in WORK(IU)
*                    (CWorkspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                            WORK( ITAUQ ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in VT
*                    (CWorkspace: need   N*N+3*N-1,
*                                 prefer N*N+2*N+(N-1)*NB)
*                    (RWorkspace: need   0)
*
                     CALL ZUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IRWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IU) and computing
*                    right singular vectors of R in VT
*                    (CWorkspace: need N*N)
*                    (RWorkspace: need BDSPAC)
```

```
*
                 CALL ZBDSQR( 'U', N, N, N, 0, S, RWORK( IE ), VT,
     $                        LDVT, WORK( IU ), LDWRKU, CDUM, 1,
     $                        RWORK( IRWORK ), INFO )
*
*                Multiply Q in U by left singular vectors of R in
*                WORK(IU), storing result in A
*                (CWorkspace: need N*N)
*                (RWorkspace: 0)
*
                 CALL ZGEMM( 'N', 'N', M, N, N, CONE, U, LDU,
     $                       WORK( IU ), LDWRKU, CZERO, A, LDA )
*
*                Copy left singular vectors of A from A to U
*
                 CALL ZLACPY( 'F', M, N, A, LDA, U, LDU )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + N
*
*                Compute A=Q*R, copying result to U
*                (CWorkspace: need 2*N, prefer N+N*NB)
*                (RWorkspace: 0)
*
                 CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL ZLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                Generate Q in U
*                (CWorkspace: need N+M, prefer N+M*NB)
*                (RWorkspace: 0)
*
                 CALL ZUNGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy R from A to VT, zeroing out below it
*
                 CALL ZLACPY( 'U', N, N, A, LDA, VT, LDVT )
                 IF( N.GT.1 )
     $              CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                           VT( 2, 1 ), LDVT )
                 IE = 1
                 ITAUQ = ITAU
                 ITAUP = ITAUQ + N
                 IWORK = ITAUP + N
*
```

```
*                    Bidiagonalize R in VT
*                    (CWorkspace: need 3*N, prefer 2*N+2*N*NB)
*                    (RWorkspace: need N)
*
                     CALL ZGEBRD( N, N, VT, LDVT, S, RWORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply Q in U by left bidiagonalizing vectors
*                    in VT
*                    (CWorkspace: need 2*N+M, prefer 2*N+M*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNMBR( 'Q', 'R', 'N', M, N, N, VT, LDVT,
     $                            WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in VT
*                    (CWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IRWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in VT
*                    (CWorkspace: 0)
*                    (RWorkspace: need BDSPAC)
*
                     CALL ZBDSQR( 'U', N, N, M, 0, S, RWORK( IE ), VT,
     $                            LDVT, U, LDU, CDUM, 1,
     $                            RWORK( IRWORK ), INFO )
*
                  END IF
*
               END IF
*
            END IF
*
         ELSE
*
            M .LT. MNTHR
*
            Path 10 (M at least N, but not much larger)
            Reduce to bidiagonal form without QR decomposition
*
            IE = 1
            ITAUQ = 1
```

```
            ITAUP = ITAUQ + N
            IWORK = ITAUP + N
*
*           Bidiagonalize A
*           (CWorkspace: need 2*N+M, prefer 2*N+(M+N)*NB)
*           (RWorkspace: need N)
*
            CALL ZGEBRD( M, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                   WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                   IERR )
            IF( WNTUAS ) THEN
*
*              If left singular vectors desired in U, copy result to U
*              and generate left bidiagonalizing vectors in U
*              (CWorkspace: need 2*N+NCU, prefer 2*N+NCU*NB)
*              (RWorkspace: 0)
*
               CALL ZLACPY( 'L', M, N, A, LDA, U, LDU )
               IF( WNTUS )
     $            NCU = N
               IF( WNTUA )
     $            NCU = M
               CALL ZUNGBR( 'Q', M, NCU, N, U, LDU, WORK( ITAUQ ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IF( WNTVAS ) THEN
*
*              If right singular vectors desired in VT, copy result to
*              VT and generate right bidiagonalizing vectors in VT
*              (CWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*              (RWorkspace: 0)
*
               CALL ZLACPY( 'U', N, N, A, LDA, VT, LDVT )
               CALL ZUNGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IF( WNTUO ) THEN
*
*              If left singular vectors desired in A, generate left
*              bidiagonalizing vectors in A
*              (CWorkspace: need 3*N, prefer 2*N+N*NB)
*              (RWorkspace: 0)
*
               CALL ZUNGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IF( WNTVO ) THEN
*
*              If right singular vectors desired in A, generate right
*              bidiagonalizing vectors in A
```

```
*              (CWorkspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*              (RWorkspace: 0)
*
               CALL ZUNGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IRWORK = IE + N
            IF( WNTUAS .OR. WNTUO )
     $         NRU = M
            IF( WNTUN )
     $         NRU = 0
            IF( WNTVAS .OR. WNTVO )
     $         NCVT = N
            IF( WNTVN )
     $         NCVT = 0
            IF( ( .NOT.WNTUO ) .AND. ( .NOT.WNTVO ) ) THEN
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in U and computing right singular
*              vectors in VT
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAC)
*
               CALL ZBDSQR( 'U', N, NCVT, NRU, 0, S, RWORK( IE ), VT,
     $                      LDVT, U, LDU, CDUM, 1, RWORK( IRWORK ),
     $                      INFO )
            ELSE IF( ( .NOT.WNTUO ) .AND. WNTVO ) THEN
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in U and computing right singular
*              vectors in A
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAC)
*
               CALL ZBDSQR( 'U', N, NCVT, NRU, 0, S, RWORK( IE ), A,
     $                      LDA, U, LDU, CDUM, 1, RWORK( IRWORK ),
     $                      INFO )
            ELSE
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in A and computing right singular
*              vectors in VT
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAC)
*
               CALL ZBDSQR( 'U', N, NCVT, NRU, 0, S, RWORK( IE ), VT,
     $                      LDVT, A, LDA, CDUM, 1, RWORK( IRWORK ),
     $                      INFO )
            END IF
*
```

```
         END IF
*
      ELSE
*
*        A has more columns than rows. If A has sufficiently more
*        columns than rows, first reduce using the LQ decomposition (if
*        sufficient workspace available)
*
         IF( N.GE.MNTHR ) THEN
*
            IF( WNTVN ) THEN
*
*              Path 1t(N much larger than M, JOBVT='N')
*              No right singular vectors to be computed
*
               ITAU = 1
               IWORK = ITAU + M
*
*              Compute A=L*Q
*              (CWorkspace: need 2*M, prefer M+M*NB)
*              (RWorkspace: 0)
*
               CALL ZGELQF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
     $                      LWORK-IWORK+1, IERR )
*
*              Zero out above L
*
               CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO, A( 1, 2 ),
     $                      LDA )
               IE = 1
               ITAUQ = 1
               ITAUP = ITAUQ + M
               IWORK = ITAUP + M
*
*              Bidiagonalize L in A
*              (CWorkspace: need 3*M, prefer 2*M+2*M*NB)
*              (RWorkspace: need M)
*
               CALL ZGEBRD( M, M, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                      IERR )
               IF( WNTUO .OR. WNTUAS ) THEN
*
*                If left singular vectors desired, generate Q
*                (CWorkspace: need 3*M, prefer 2*M+M*NB)
*                (RWorkspace: 0)
*
                 CALL ZUNGBR( 'Q', M, M, M, A, LDA, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
               END IF
```

```
               IRWORK = IE + M
               NRU = 0
               IF( WNTUO .OR. WNTUAS )
     $            NRU = M
*
*              Perform bidiagonal QR iteration, computing left singular
*              vectors of A in A if desired
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAC)
*
               CALL ZBDSQR( 'U', M, 0, NRU, 0, S, RWORK( IE ), CDUM, 1,
     $                      A, LDA, CDUM, 1, RWORK( IRWORK ), INFO )
*
*              If left singular vectors desired in U, copy them there
*
               IF( WNTUAS )
     $            CALL ZLACPY( 'F', M, M, A, LDA, U, LDU )
*
            ELSE IF( WNTVO .AND. WNTUN ) THEN
*
*              Path 2t(N much larger than M, JOBU='N', JOBVT='O')
*              M right singular vectors to be overwritten on A and
*              no left singular vectors to be computed
*
               IF( LWORK.GE.M*M+3*M ) THEN
*
*                Sufficient workspace for a fast algorithm
*
                 IR = 1
                 IF( LWORK.GE.MAX( WRKBL, LDA*N )+LDA*M ) THEN
*
*                  WORK(IU) is LDA by N and WORK(IR) is LDA by M
*
                   LDWRKU = LDA
                   CHUNK = N
                   LDWRKR = LDA
                 ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N )+M*M ) THEN
*
*                  WORK(IU) is LDA by N and WORK(IR) is M by M
*
                   LDWRKU = LDA
                   CHUNK = N
                   LDWRKR = M
                 ELSE
*
*                  WORK(IU) is M by CHUNK and WORK(IR) is M by M
*
                   LDWRKU = M
                   CHUNK = ( LWORK-M*M ) / M
                   LDWRKR = M
```

```
                END IF
                ITAU = IR + LDWRKR*M
                IWORK = ITAU + M
*
*               Compute A=L*Q
*               (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*               (RWorkspace: 0)
*
                CALL ZGELQF( M, N, A, LDA, WORK( ITAU ),
     $                       WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*               Copy L to WORK(IR) and zero out above it
*
                CALL ZLACPY( 'L', M, M, A, LDA, WORK( IR ), LDWRKR )
                CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                       WORK( IR+LDWRKR ), LDWRKR )
*
*               Generate Q in A
*               (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*               (RWorkspace: 0)
*
                CALL ZUNGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                       WORK( IWORK ), LWORK-IWORK+1, IERR )
                IE = 1
                ITAUQ = ITAU
                ITAUP = ITAUQ + M
                IWORK = ITAUP + M
*
*               Bidiagonalize L in WORK(IR)
*               (CWorkspace: need M*M+3*M, prefer M*M+2*M+2*M*NB)
*               (RWorkspace: need M)
*
                CALL ZGEBRD( M, M, WORK( IR ), LDWRKR, S, RWORK( IE ),
     $                       WORK( ITAUQ ), WORK( ITAUP ),
     $                       WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*               Generate right vectors bidiagonalizing L
*               (CWorkspace: need M*M+3*M-1, prefer M*M+2*M+(M-1)*NB)
*               (RWorkspace: 0)
*
                CALL ZUNGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                       WORK( ITAUP ), WORK( IWORK ),
     $                       LWORK-IWORK+1, IERR )
                IRWORK = IE + M
*
*               Perform bidiagonal QR iteration, computing right
*               singular vectors of L in WORK(IR)
*               (CWorkspace: need M*M)
*               (RWorkspace: need BDSPAC)
*
```

```
                 CALL ZBDSQR( 'U', M, M, 0, 0, S, RWORK( IE ),
     $                        WORK( IR ), LDWRKR, CDUM, 1, CDUM, 1,
     $                        RWORK( IRWORK ), INFO )
                 IU = ITAUQ
*
*                Multiply right singular vectors of L in WORK(IR) by Q
*                in A, storing result in WORK(IU) and copying to A
*                (CWorkspace: need M*M+M, prefer M*M+M*N)
*                (RWorkspace: 0)
*
                 DO 30 I = 1, N, CHUNK
                    BLK = MIN( N-I+1, CHUNK )
                    CALL ZGEMM( 'N', 'N', M, BLK, M, CONE, WORK( IR ),
     $                          LDWRKR, A( 1, I ), LDA, CZERO,
     $                          WORK( IU ), LDWRKU )
                    CALL ZLACPY( 'F', M, BLK, WORK( IU ), LDWRKU,
     $                           A( 1, I ), LDA )
   30            CONTINUE
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 IE = 1
                 ITAUQ = 1
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Bidiagonalize A
*                (CWorkspace: need 2*M+N, prefer 2*M+(M+N)*NB)
*                (RWorkspace: need M)
*
                 CALL ZGEBRD( M, N, A, LDA, S, RWORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Generate right vectors bidiagonalizing A
*                (CWorkspace: need 3*M, prefer 2*M+M*NB)
*                (RWorkspace: 0)
*
                 CALL ZUNGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IRWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing right
*                singular vectors of A in A
*                (CWorkspace: 0)
*                (RWorkspace: need BDSPAC)
*
                 CALL ZBDSQR( 'L', M, N, 0, 0, S, RWORK( IE ), A, LDA,
```

```
     $                        CDUM, 1, CDUM, 1, RWORK( IRWORK ), INFO )
*
               END IF
*
            ELSE IF( WNTVO .AND. WNTUAS ) THEN
*
               Path 3t(N much larger than M, JOBU='S' or 'A', JOBVT='O')
               M right singular vectors to be overwritten on A and
               M left singular vectors to be computed in U
*
               IF( LWORK.GE.M*M+3*M ) THEN
*
                  Sufficient workspace for a fast algorithm
*
                  IR = 1
                  IF( LWORK.GE.MAX( WRKBL, LDA*N )+LDA*M ) THEN
*
                     WORK(IU) is LDA by N and WORK(IR) is LDA by M
*
                     LDWRKU = LDA
                     CHUNK = N
                     LDWRKR = LDA
                  ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N )+M*M ) THEN
*
                     WORK(IU) is LDA by N and WORK(IR) is M by M
*
                     LDWRKU = LDA
                     CHUNK = N
                     LDWRKR = M
                  ELSE
*
                     WORK(IU) is M by CHUNK and WORK(IR) is M by M
*
                     LDWRKU = M
                     CHUNK = ( LWORK-M*M ) / M
                     LDWRKR = M
                  END IF
                  ITAU = IR + LDWRKR*M
                  IWORK = ITAU + M
*
                  Compute A=L*Q
*                 (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*                 (RWorkspace: 0)
*
                  CALL ZGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Copy L to U, zeroing about above it
*
                  CALL ZLACPY( 'L', M, M, A, LDA, U, LDU )
```

```
                CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO, U( 1, 2 ),
     $                       LDU )
*
*               Generate Q in A
*               (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*               (RWorkspace: 0)
*
                CALL ZUNGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                       WORK( IWORK ), LWORK-IWORK+1, IERR )
                IE = 1
                ITAUQ = ITAU
                ITAUP = ITAUQ + M
                IWORK = ITAUP + M
*
*               Bidiagonalize L in U, copying result to WORK(IR)
*               (CWorkspace: need M*M+3*M, prefer M*M+2*M+2*M*NB)
*               (RWorkspace: need M)
*
                CALL ZGEBRD( M, M, U, LDU, S, RWORK( IE ),
     $                       WORK( ITAUQ ), WORK( ITAUP ),
     $                       WORK( IWORK ), LWORK-IWORK+1, IERR )
                CALL ZLACPY( 'U', M, M, U, LDU, WORK( IR ), LDWRKR )
*
*               Generate right vectors bidiagonalizing L in WORK(IR)
*               (CWorkspace: need M*M+3*M-1, prefer M*M+2*M+(M-1)*NB)
*               (RWorkspace: 0)
*
                CALL ZUNGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                       WORK( ITAUP ), WORK( IWORK ),
     $                       LWORK-IWORK+1, IERR )
*
*               Generate left vectors bidiagonalizing L in U
*               (CWorkspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*               (RWorkspace: 0)
*
                CALL ZUNGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                       WORK( IWORK ), LWORK-IWORK+1, IERR )
                IRWORK = IE + M
*
*               Perform bidiagonal QR iteration, computing left
*               singular vectors of L in U, and computing right
*               singular vectors of L in WORK(IR)
*               (CWorkspace: need M*M)
*               (RWorkspace: need BDSPAC)
*
                CALL ZBDSQR( 'U', M, M, M, 0, S, RWORK( IE ),
     $                       WORK( IR ), LDWRKR, U, LDU, CDUM, 1,
     $                       RWORK( IRWORK ), INFO )
                IU = ITAUQ
*
```

```
*                 Multiply right singular vectors of L in WORK(IR) by Q
*                 in A, storing result in WORK(IU) and copying to A
*                 (CWorkspace: need M*M+M, prefer M*M+M*N))
*                 (RWorkspace: 0)
*
                  DO 40 I = 1, N, CHUNK
                     BLK = MIN( N-I+1, CHUNK )
                     CALL ZGEMM( 'N', 'N', M, BLK, M, CONE, WORK( IR ),
     $                           LDWRKR, A( 1, I ), LDA, CZERO,
     $                           WORK( IU ), LDWRKU )
                     CALL ZLACPY( 'F', M, BLK, WORK( IU ), LDWRKU,
     $                            A( 1, I ), LDA )
   40             CONTINUE
*
               ELSE
*
*                 Insufficient workspace for a fast algorithm
*
                  ITAU = 1
                  IWORK = ITAU + M
*
*                 Compute A=L*Q
*                 (CWorkspace: need 2*M, prefer M+M*NB)
*                 (RWorkspace: 0)
*
                  CALL ZGELQF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Copy L to U, zeroing out above it
*
                  CALL ZLACPY( 'L', M, M, A, LDA, U, LDU )
                  CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO, U( 1, 2 ),
     $                         LDU )
*
*                 Generate Q in A
*                 (CWorkspace: need 2*M, prefer M+M*NB)
*                 (RWorkspace: 0)
*
                  CALL ZUNGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IE = 1
                  ITAUQ = ITAU
                  ITAUP = ITAUQ + M
                  IWORK = ITAUP + M
*
*                 Bidiagonalize L in U
*                 (CWorkspace: need 3*M, prefer 2*M+2*M*NB)
*                 (RWorkspace: need M)
*
                  CALL ZGEBRD( M, M, U, LDU, S, RWORK( IE ),
```

```
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Multiply right vectors bidiagonalizing L by Q in A
*                (CWorkspace: need 2*M+N, prefer 2*M+N*NB)
*                (RWorkspace: 0)
*
                 CALL ZUNMBR( 'P', 'L', 'C', M, N, M, U, LDU,
     $                        WORK( ITAUP ), A, LDA, WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate left vectors bidiagonalizing L in U
*                (CWorkspace: need 3*M, prefer 2*M+M*NB)
*                (RWorkspace: 0)
*
                 CALL ZUNGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IRWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of A in U and computing right
*                singular vectors of A in A
*                (CWorkspace: 0)
*                (RWorkspace: need BDSPAC)
*
                 CALL ZBDSQR( 'U', M, N, M, 0, S, RWORK( IE ), A, LDA,
     $                        U, LDU, CDUM, 1, RWORK( IRWORK ), INFO )
*
              END IF
*
           ELSE IF( WNTVS ) THEN
*
              IF( WNTUN ) THEN
*
*                Path 4t(N much larger than M, JOBU='N', JOBVT='S')
*                M right singular vectors to be computed in VT and
*                no left singular vectors to be computed
*
                 IF( LWORK.GE.M*M+3*M ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IR = 1
                    IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                      WORK(IR) is LDA by M
*
                       LDWRKR = LDA
                    ELSE
*
```

```
*                       WORK(IR) is M by M
*
                        LDWRKR = M
                     END IF
                     ITAU = IR + LDWRKR*M
                     IWORK = ITAU + M
*
*                    Compute A=L*Q
*                    (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL ZGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to WORK(IR), zeroing out above it
*
                     CALL ZLACPY( 'L', M, M, A, LDA, WORK( IR ),
     $                            LDWRKR )
                     CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                            WORK( IR+LDWRKR ), LDWRKR )
*
*                    Generate Q in A
*                    (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = 1
                     ITAUQ = ITAU
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Bidiagonalize L in WORK(IR)
*                    (CWorkspace: need M*M+3*M, prefer M*M+2*M+2*M*NB)
*                    (RWorkspace: need M)
*
                     CALL ZGEBRD( M, M, WORK( IR ), LDWRKR, S,
     $                            RWORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right vectors bidiagonalizing L in
*                    WORK(IR)
*                    (CWorkspace: need M*M+3*M, prefer M*M+2*M+(M-1)*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IRWORK = IE + M
```

```
*
*                    Perform bidiagonal QR iteration, computing right
*                    singular vectors of L in WORK(IR)
*                    (CWorkspace: need M*M)
*                    (RWorkspace: need BDSPAC)
*
                     CALL ZBDSQR( 'U', M, M, 0, 0, S, RWORK( IE ),
     $                            WORK( IR ), LDWRKR, CDUM, 1, CDUM, 1,
     $                            RWORK( IRWORK ), INFO )
*
*                    Multiply right singular vectors of L in WORK(IR) by
*                    Q in A, storing result in VT
*                    (CWorkspace: need M*M)
*                    (RWorkspace: 0)
*
                     CALL ZGEMM( 'N', 'N', M, N, M, CONE, WORK( IR ),
     $                           LDWRKR, A, LDA, CZERO, VT, LDVT )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + M
*
*                    Compute A=L*Q
*                    (CWorkspace: need 2*M, prefer M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL ZGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy result to VT
*
                     CALL ZLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (CWorkspace: need 2*M, prefer M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGLQ( M, N, M, VT, LDVT, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = 1
                     ITAUQ = ITAU
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Zero out above L in A
*
                     CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO,
```

```
     $                           A( 1, 2 ), LDA )
*
*                    Bidiagonalize L in A
*                    (CWorkspace: need 3*M, prefer 2*M+2*M*NB)
*                    (RWorkspace: need M)
*
                     CALL ZGEBRD( M, M, A, LDA, S, RWORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply right vectors bidiagonalizing L by Q in VT
*                    (CWorkspace: need 2*M+N, prefer 2*M+N*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNMBR( 'P', 'L', 'C', M, N, M, A, LDA,
     $                           WORK( ITAUP ), VT, LDVT,
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IRWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing right
*                    singular vectors of A in VT
*                    (CWorkspace: 0)
*                    (RWorkspace: need BDSPAC)
*
                     CALL ZBDSQR( 'U', M, N, 0, 0, S, RWORK( IE ), VT,
     $                           LDVT, CDUM, 1, CDUM, 1,
     $                           RWORK( IRWORK ), INFO )
*
                  END IF
*
               ELSE IF( WNTUO ) THEN
*
                  Path 5t(N much larger than M, JOBU='O', JOBVT='S')
*                 M right singular vectors to be computed in VT and
*                 M left singular vectors to be overwritten on A
*
                  IF( LWORK.GE.2*M*M+3*M ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+2*LDA*M ) THEN
*
*                       WORK(IU) is LDA by M and WORK(IR) is LDA by M
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*M
                        LDWRKR = LDA
                     ELSE IF( LWORK.GE.WRKBL+( LDA+M )*M ) THEN
*
```

```
*                       WORK(IU) is LDA by M and WORK(IR) is M by M
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*M
                        LDWRKR = M
                     ELSE
*
*                       WORK(IU) is M by M and WORK(IR) is M by M
*
                        LDWRKU = M
                        IR = IU + LDWRKU*M
                        LDWRKR = M
                     END IF
                     ITAU = IR + LDWRKR*M
                     IWORK = ITAU + M
*
*                    Compute A=L*Q
*                    (CWorkspace: need 2*M*M+2*M, prefer 2*M*M+M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL ZGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to WORK(IU), zeroing out below it
*
                     CALL ZLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                            LDWRKU )
                     CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                            WORK( IU+LDWRKU ), LDWRKU )
*
*                    Generate Q in A
*                    (CWorkspace: need 2*M*M+2*M, prefer 2*M*M+M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = 1
                     ITAUQ = ITAU
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Bidiagonalize L in WORK(IU), copying result to
*                    WORK(IR)
*                    (CWorkspace: need   2*M*M+3*M,
*                                 prefer 2*M*M+2*M+2*M*NB)
*                    (RWorkspace: need   M)
*
                     CALL ZGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                            RWORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
```

```
     $                           LWORK-IWORK+1, IERR )
                     CALL ZLACPY( 'L', M, M, WORK( IU ), LDWRKU,
     $                           WORK( IR ), LDWRKR )
*
*                    Generate right bidiagonalizing vectors in WORK(IU)
*                    (CWorkspace: need   2*M*M+3*M-1,
*                                 prefer 2*M*M+2*M+(M-1)*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors in WORK(IR)
*                    (CWorkspace: need 2*M*M+3*M, prefer 2*M*M+2*M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGBR( 'Q', M, M, M, WORK( IR ), LDWRKR,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                     IRWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of L in WORK(IR) and computing
*                    right singular vectors of L in WORK(IU)
*                    (CWorkspace: need 2*M*M)
*                    (RWorkspace: need BDSPAC)
*
                     CALL ZBDSQR( 'U', M, M, M, 0, S, RWORK( IE ),
     $                           WORK( IU ), LDWRKU, WORK( IR ),
     $                           LDWRKR, CDUM, 1, RWORK( IRWORK ),
     $                           INFO )
*
*                    Multiply right singular vectors of L in WORK(IU) by
*                    Q in A, storing result in VT
*                    (CWorkspace: need M*M)
*                    (RWorkspace: 0)
*
                     CALL ZGEMM( 'N', 'N', M, N, M, CONE, WORK( IU ),
     $                          LDWRKU, A, LDA, CZERO, VT, LDVT )
*
*                    Copy left singular vectors of L to A
*                    (CWorkspace: need M*M)
*                    (RWorkspace: 0)
*
                     CALL ZLACPY( 'F', M, M, WORK( IR ), LDWRKR, A,
     $                           LDA )
*
                 ELSE
*
```

```
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (CWorkspace: need 2*M, prefer M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL ZGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL ZLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (CWorkspace: need 2*M, prefer M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGLQ( M, N, M, VT, LDVT, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = 1
                     ITAUQ = ITAU
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Zero out above L in A
*
                     CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                            A( 1, 2 ), LDA )
*
*                    Bidiagonalize L in A
*                    (CWorkspace: need 3*M, prefer 2*M+2*M*NB)
*                    (RWorkspace: need M)
*
                     CALL ZGEBRD( M, M, A, LDA, S, RWORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply right vectors bidiagonalizing L by Q in VT
*                    (CWorkspace: need 2*M+N, prefer 2*M+N*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNMBR( 'P', 'L', 'C', M, N, M, A, LDA,
     $                            WORK( ITAUP ), VT, LDVT,
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors of L in A
*                    (CWorkspace: need 3*M, prefer 2*M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGBR( 'Q', M, M, M, A, LDA, WORK( ITAUQ ),
```

```
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IRWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in A and computing right
*                    singular vectors of A in VT
*                    (CWorkspace: 0)
*                    (RWorkspace: need BDSPAC)
*
                     CALL ZBDSQR( 'U', M, N, M, 0, S, RWORK( IE ), VT,
     $                          LDVT, A, LDA, CDUM, 1,
     $                          RWORK( IRWORK ), INFO )
*
                  END IF
*
               ELSE IF( WNTUAS ) THEN
*
                  Path 6t(N much larger than M, JOBU='S' or 'A',
*                         JOBVT='S')
*                 M right singular vectors to be computed in VT and
*                 M left singular vectors to be computed in U
*
                  IF( LWORK.GE.M*M+3*M ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                       WORK(IU) is LDA by N
*
                        LDWRKU = LDA
                     ELSE
*
*                       WORK(IU) is LDA by M
*
                        LDWRKU = M
                     END IF
                     ITAU = IU + LDWRKU*M
                     IWORK = ITAU + M
*
*                    Compute A=L*Q
*                    (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL ZGELQF( M, N, A, LDA, WORK( ITAU ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to WORK(IU), zeroing out above it
*
```

```
                    CALL ZLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                           LDWRKU )
                    CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                           WORK( IU+LDWRKU ), LDWRKU )
*
*                   Generate Q in A
*                   (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*                   (RWorkspace: 0)
*
                    CALL ZUNGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = 1
                    ITAUQ = ITAU
                    ITAUP = ITAUQ + M
                    IWORK = ITAUP + M
*
*                   Bidiagonalize L in WORK(IU), copying result to U
*                   (CWorkspace: need M*M+3*M, prefer M*M+2*M+2*M*NB)
*                   (RWorkspace: need M)
*
                    CALL ZGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                           RWORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    CALL ZLACPY( 'L', M, M, WORK( IU ), LDWRKU, U,
     $                           LDU )
*
*                   Generate right bidiagonalizing vectors in WORK(IU)
*                   (CWorkspace: need   M*M+3*M-1,
*                                prefer M*M+2*M+(M-1)*NB)
*                   (RWorkspace: 0)
*
                    CALL ZUNGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate left bidiagonalizing vectors in U
*                   (CWorkspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*                   (RWorkspace: 0)
*
                    CALL ZUNGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IRWORK = IE + M
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of L in U and computing right
*                   singular vectors of L in WORK(IU)
*                   (CWorkspace: need M*M)
*                   (RWorkspace: need BDSPAC)
*
```

```
                 CALL ZBDSQR( 'U', M, M, M, 0, S, RWORK( IE ),
     $                        WORK( IU ), LDWRKU, U, LDU, CDUM, 1,
     $                        RWORK( IRWORK ), INFO )
*
*                Multiply right singular vectors of L in WORK(IU) by
*                Q in A, storing result in VT
*                (CWorkspace: need M*M)
*                (RWorkspace: 0)
*
                 CALL ZGEMM( 'N', 'N', M, N, M, CONE, WORK( IU ),
     $                       LDWRKU, A, LDA, CZERO, VT, LDVT )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + M
*
*                Compute A=L*Q, copying result to VT
*                (CWorkspace: need 2*M, prefer M+M*NB)
*                (RWorkspace: 0)
*
                 CALL ZGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL ZLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                Generate Q in VT
*                (CWorkspace: need 2*M, prefer M+M*NB)
*                (RWorkspace: 0)
*
                 CALL ZUNGLQ( M, N, M, VT, LDVT, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy L to U, zeroing out above it
*
                 CALL ZLACPY( 'L', M, M, A, LDA, U, LDU )
                 CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                        U( 1, 2 ), LDU )
                 IE = 1
                 ITAUQ = ITAU
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Bidiagonalize L in U
*                (CWorkspace: need 3*M, prefer 2*M+2*M*NB)
*                (RWorkspace: need M)
*
                 CALL ZGEBRD( M, M, U, LDU, S, RWORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
```

```
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply right bidiagonalizing vectors in U by Q
*                    in VT
*                    (CWorkspace: need 2*M+N, prefer 2*M+N*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNMBR( 'P', 'L', 'C', M, N, M, U, LDU,
     $                          WORK( ITAUP ), VT, LDVT,
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors in U
*                    (CWorkspace: need 3*M, prefer 2*M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IRWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in VT
*                    (CWorkspace: 0)
*                    (RWorkspace: need BDSPAC)
*
                     CALL ZBDSQR( 'U', M, N, M, 0, S, RWORK( IE ), VT,
     $                          LDVT, U, LDU, CDUM, 1,
     $                          RWORK( IRWORK ), INFO )
*
                  END IF
*
               END IF
*
            ELSE IF( WNTVA ) THEN
*
               IF( WNTUN ) THEN
*
*                 Path 7t(N much larger than M, JOBU='N', JOBVT='A')
*                 N right singular vectors to be computed in VT and
*                 no left singular vectors to be computed
*
                  IF( LWORK.GE.M*M+MAX( N+M, 3*M ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IR = 1
                     IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                       WORK(IR) is LDA by M
*
```

```
                      LDWRKR = LDA
                   ELSE
*
*                     WORK(IR) is M by M
*
                      LDWRKR = M
                   END IF
                   ITAU = IR + LDWRKR*M
                   IWORK = ITAU + M
*
*                  Compute A=L*Q, copying result to VT
*                  (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*                  (RWorkspace: 0)
*
                   CALL ZGELQF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                   CALL ZLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                  Copy L to WORK(IR), zeroing out above it
*
                   CALL ZLACPY( 'L', M, M, A, LDA, WORK( IR ),
     $                           LDWRKR )
                   CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                           WORK( IR+LDWRKR ), LDWRKR )
*
*                  Generate Q in VT
*                  (CWorkspace: need M*M+M+N, prefer M*M+M+N*NB)
*                  (RWorkspace: 0)
*
                   CALL ZUNGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                   IE = 1
                   ITAUQ = ITAU
                   ITAUP = ITAUQ + M
                   IWORK = ITAUP + M
*
*                  Bidiagonalize L in WORK(IR)
*                  (CWorkspace: need M*M+3*M, prefer M*M+2*M+2*M*NB)
*                  (RWorkspace: need M)
*
                   CALL ZGEBRD( M, M, WORK( IR ), LDWRKR, S,
     $                           RWORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                  Generate right bidiagonalizing vectors in WORK(IR)
*                  (CWorkspace: need   M*M+3*M-1,
*                               prefer M*M+2*M+(M-1)*NB)
*                  (RWorkspace: 0)
*
```

```
                 CALL ZUNGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                        WORK( ITAUP ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
                 IRWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing right
*                singular vectors of L in WORK(IR)
*                (CWorkspace: need M*M)
*                (RWorkspace: need BDSPAC)
*
                 CALL ZBDSQR( 'U', M, M, 0, 0, S, RWORK( IE ),
     $                        WORK( IR ), LDWRKR, CDUM, 1, CDUM, 1,
     $                        RWORK( IRWORK ), INFO )
*
*                Multiply right singular vectors of L in WORK(IR) by
*                Q in VT, storing result in A
*                (CWorkspace: need M*M)
*                (RWorkspace: 0)
*
                 CALL ZGEMM( 'N', 'N', M, N, M, CONE, WORK( IR ),
     $                       LDWRKR, VT, LDVT, CZERO, A, LDA )
*
*                Copy right singular vectors of A from A to VT
*
                 CALL ZLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + M
*
*                Compute A=L*Q, copying result to VT
*                (CWorkspace: need 2*M, prefer M+M*NB)
*                (RWorkspace: 0)
*
                 CALL ZGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL ZLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                Generate Q in VT
*                (CWorkspace: need M+N, prefer M+N*NB)
*                (RWorkspace: 0)
*
                 CALL ZUNGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = 1
                 ITAUQ = ITAU
                 ITAUP = ITAUQ + M
```

```
                    IWORK = ITAUP + M
*
*                   Zero out above L in A
*
                    CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                           A( 1, 2 ), LDA )
*
*                   Bidiagonalize L in A
*                   (CWorkspace: need 3*M, prefer 2*M+2*M*NB)
*                   (RWorkspace: need M)
*
                    CALL ZGEBRD( M, M, A, LDA, S, RWORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Multiply right bidiagonalizing vectors in A by Q
*                   in VT
*                   (CWorkspace: need 2*M+N, prefer 2*M+N*NB)
*                   (RWorkspace: 0)
*
                    CALL ZUNMBR( 'P', 'L', 'C', M, N, M, A, LDA,
     $                           WORK( ITAUP ), VT, LDVT,
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IRWORK = IE + M
*
*                   Perform bidiagonal QR iteration, computing right
*                   singular vectors of A in VT
*                   (CWorkspace: 0)
*                   (RWorkspace: need BDSPAC)
*
                    CALL ZBDSQR( 'U', M, N, 0, 0, S, RWORK( IE ), VT,
     $                           LDVT, CDUM, 1, CDUM, 1,
     $                           RWORK( IRWORK ), INFO )
*
                 END IF
*
              ELSE IF( WNTUO ) THEN
*
                 Path 8t(N much larger than M, JOBU='O', JOBVT='A')
*                N right singular vectors to be computed in VT and
*                M left singular vectors to be overwritten on A
*
                 IF( LWORK.GE.2*M*M+MAX( N+M, 3*M ) ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IU = 1
                    IF( LWORK.GE.WRKBL+2*LDA*M ) THEN
*
*                      WORK(IU) is LDA by M and WORK(IR) is LDA by M
```

```
*
                      LDWRKU = LDA
                      IR = IU + LDWRKU*M
                      LDWRKR = LDA
                   ELSE IF( LWORK.GE.WRKBL+( LDA+M )*M ) THEN
*
*                     WORK(IU) is LDA by M and WORK(IR) is M by M
*
                      LDWRKU = LDA
                      IR = IU + LDWRKU*M
                      LDWRKR = M
                   ELSE
*
*                     WORK(IU) is M by M and WORK(IR) is M by M
*
                      LDWRKU = M
                      IR = IU + LDWRKU*M
                      LDWRKR = M
                   END IF
                   ITAU = IR + LDWRKR*M
                   IWORK = ITAU + M
*
*                  Compute A=L*Q, copying result to VT
*                  (CWorkspace: need 2*M*M+2*M, prefer 2*M*M+M+M*NB)
*                  (RWorkspace: 0)
*
                   CALL ZGELQF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                   CALL ZLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                  Generate Q in VT
*                  (CWorkspace: need 2*M*M+M+N, prefer 2*M*M+M+N*NB)
*                  (RWorkspace: 0)
*
                   CALL ZUNGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                  Copy L to WORK(IU), zeroing out above it
*
                   CALL ZLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                           LDWRKU )
                   CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                           WORK( IU+LDWRKU ), LDWRKU )
                   IE = 1
                   ITAUQ = ITAU
                   ITAUP = ITAUQ + M
                   IWORK = ITAUP + M
*
*                  Bidiagonalize L in WORK(IU), copying result to
*                  WORK(IR)
```

```
*                    (CWorkspace: need   2*M*M+3*M,
*                                 prefer 2*M*M+2*M+2*M*NB)
*                    (RWorkspace: need   M)
*
                     CALL ZGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                            RWORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     CALL ZLACPY( 'L', M, M, WORK( IU ), LDWRKU,
     $                            WORK( IR ), LDWRKR )
*
*                    Generate right bidiagonalizing vectors in WORK(IU)
*                    (CWorkspace: need   2*M*M+3*M-1,
*                                 prefer 2*M*M+2*M+(M-1)*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors in WORK(IR)
*                    (CWorkspace: need 2*M*M+3*M, prefer 2*M*M+2*M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGBR( 'Q', M, M, M, WORK( IR ), LDWRKR,
     $                            WORK( ITAUQ ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IRWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of L in WORK(IR) and computing
*                    right singular vectors of L in WORK(IU)
*                    (CWorkspace: need 2*M*M)
*                    (RWorkspace: need BDSPAC)
*
                     CALL ZBDSQR( 'U', M, M, M, 0, S, RWORK( IE ),
     $                            WORK( IU ), LDWRKU, WORK( IR ),
     $                            LDWRKR, CDUM, 1, RWORK( IRWORK ),
     $                            INFO )
*
*                    Multiply right singular vectors of L in WORK(IU) by
*                    Q in VT, storing result in A
*                    (CWorkspace: need M*M)
*                    (RWorkspace: 0)
*
                     CALL ZGEMM( 'N', 'N', M, N, M, CONE, WORK( IU ),
     $                           LDWRKU, VT, LDVT, CZERO, A, LDA )
*
*                    Copy right singular vectors of A from A to VT
*
```

```
                 CALL ZLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
*                Copy left singular vectors of A from WORK(IR) to A
*
                 CALL ZLACPY( 'F', M, M, WORK( IR ), LDWRKR, A,
     $                       LDA )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + M
*
*                Compute A=L*Q, copying result to VT
*                (CWorkspace: need 2*M, prefer M+M*NB)
*                (RWorkspace: 0)
*
                 CALL ZGELQF( M, N, A, LDA, WORK( ITAU ),
     $                       WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL ZLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                Generate Q in VT
*                (CWorkspace: need M+N, prefer M+N*NB)
*                (RWorkspace: 0)
*
                 CALL ZUNGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                       WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = 1
                 ITAUQ = ITAU
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Zero out above L in A
*
                 CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                       A( 1, 2 ), LDA )
*
*                Bidiagonalize L in A
*                (CWorkspace: need 3*M, prefer 2*M+2*M*NB)
*                (RWorkspace: need M)
*
                 CALL ZGEBRD( M, M, A, LDA, S, RWORK( IE ),
     $                       WORK( ITAUQ ), WORK( ITAUP ),
     $                       WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Multiply right bidiagonalizing vectors in A by Q
*                in VT
*                (CWorkspace: need 2*M+N, prefer 2*M+N*NB)
*                (RWorkspace: 0)
```

```
*
                    CALL ZUNMBR( 'P', 'L', 'C', M, N, M, A, LDA,
     $                           WORK( ITAUP ), VT, LDVT,
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Generate left bidiagonalizing vectors in A
*                   (CWorkspace: need 3*M, prefer 2*M+M*NB)
*                   (RWorkspace: 0)
*
                    CALL ZUNGBR( 'Q', M, M, M, A, LDA, WORK( ITAUQ ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IRWORK = IE + M
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of A in A and computing right
*                   singular vectors of A in VT
*                   (CWorkspace: 0)
*                   (RWorkspace: need BDSPAC)
*
                    CALL ZBDSQR( 'U', M, N, M, 0, S, RWORK( IE ), VT,
     $                           LDVT, A, LDA, CDUM, 1,
     $                           RWORK( IRWORK ), INFO )
*
                 END IF
*
              ELSE IF( WNTUAS ) THEN
*
                 Path 9t(N much larger than M, JOBU='S' or 'A',
*                        JOBVT='A')
                 N right singular vectors to be computed in VT and
                 M left singular vectors to be computed in U
*
                 IF( LWORK.GE.M*M+MAX( N+M, 3*M ) ) THEN
*
                    Sufficient workspace for a fast algorithm
*
                    IU = 1
                    IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
                       WORK(IU) is LDA by M
*
                       LDWRKU = LDA
                    ELSE
*
                       WORK(IU) is M by M
*
                       LDWRKU = M
                    END IF
                    ITAU = IU + LDWRKU*M
                    IWORK = ITAU + M
```

```
*
*                    Compute A=L*Q, copying result to VT
*                    (CWorkspace: need M*M+2*M, prefer M*M+M+M*NB)
*                    (RWorkspace: 0)
*
                     CALL ZGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL ZLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (CWorkspace: need M*M+M+N, prefer M*M+M+N*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to WORK(IU), zeroing out above it
*
                     CALL ZLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                            LDWRKU )
                     CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                            WORK( IU+LDWRKU ), LDWRKU )
                     IE = 1
                     ITAUQ = ITAU
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Bidiagonalize L in WORK(IU), copying result to U
*                    (CWorkspace: need M*M+3*M, prefer M*M+2*M+2*M*NB)
*                    (RWorkspace: need M)
*
                     CALL ZGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                            RWORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     CALL ZLACPY( 'L', M, M, WORK( IU ), LDWRKU, U,
     $                            LDU )
*
*                    Generate right bidiagonalizing vectors in WORK(IU)
*                    (CWorkspace: need M*M+3*M, prefer M*M+2*M+(M-1)*NB)
*                    (RWorkspace: 0)
*
                     CALL ZUNGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors in U
*                    (CWorkspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*                    (RWorkspace: 0)
*
```

```
                 CALL ZUNGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IRWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of L in U and computing right
*                singular vectors of L in WORK(IU)
*                (CWorkspace: need M*M)
*                (RWorkspace: need BDSPAC)
*
                 CALL ZBDSQR( 'U', M, M, M, 0, S, RWORK( IE ),
     $                        WORK( IU ), LDWRKU, U, LDU, CDUM, 1,
     $                        RWORK( IRWORK ), INFO )
*
*                Multiply right singular vectors of L in WORK(IU) by
*                Q in VT, storing result in A
*                (CWorkspace: need M*M)
*                (RWorkspace: 0)
*
                 CALL ZGEMM( 'N', 'N', M, N, M, CONE, WORK( IU ),
     $                       LDWRKU, VT, LDVT, CZERO, A, LDA )
*
*                Copy right singular vectors of A from A to VT
*
                 CALL ZLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + M
*
*                Compute A=L*Q, copying result to VT
*                (CWorkspace: need 2*M, prefer M+M*NB)
*                (RWorkspace: 0)
*
                 CALL ZGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL ZLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                Generate Q in VT
*                (CWorkspace: need M+N, prefer M+N*NB)
*                (RWorkspace: 0)
*
                 CALL ZUNGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy L to U, zeroing out above it
*
```

```
                    CALL ZLACPY( 'L', M, M, A, LDA, U, LDU )
                    CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                           U( 1, 2 ), LDU )
                    IE = 1
                    ITAUQ = ITAU
                    ITAUP = ITAUQ + M
                    IWORK = ITAUP + M
*
*                   Bidiagonalize L in U
*                   (CWorkspace: need 3*M, prefer 2*M+2*M*NB)
*                   (RWorkspace: need M)
*
                    CALL ZGEBRD( M, M, U, LDU, S, RWORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Multiply right bidiagonalizing vectors in U by Q
*                   in VT
*                   (CWorkspace: need 2*M+N, prefer 2*M+N*NB)
*                   (RWorkspace: 0)
*
                    CALL ZUNMBR( 'P', 'L', 'C', M, N, M, U, LDU,
     $                           WORK( ITAUP ), VT, LDVT,
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Generate left bidiagonalizing vectors in U
*                   (CWorkspace: need 3*M, prefer 2*M+M*NB)
*                   (RWorkspace: 0)
*
                    CALL ZUNGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IRWORK = IE + M
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of A in U and computing right
*                   singular vectors of A in VT
*                   (CWorkspace: 0)
*                   (RWorkspace: need BDSPAC)
*
                    CALL ZBDSQR( 'U', M, N, M, 0, S, RWORK( IE ), VT,
     $                           LDVT, U, LDU, CDUM, 1,
     $                           RWORK( IRWORK ), INFO )
*
                 END IF
*
              END IF
*
           END IF
*
        ELSE
```

```
*
*          N .LT. MNTHR
*
*          Path 10t(N greater than M, but not much larger)
*          Reduce to bidiagonal form without LQ decomposition
*
           IE = 1
           ITAUQ = 1
           ITAUP = ITAUQ + M
           IWORK = ITAUP + M
*
*          Bidiagonalize A
*          (CWorkspace: need 2*M+N, prefer 2*M+(M+N)*NB)
*          (RWorkspace: M)
*
           CALL ZGEBRD( M, N, A, LDA, S, RWORK( IE ), WORK( ITAUQ ),
     $                  WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                  IERR )
           IF( WNTUAS ) THEN
*
*             If left singular vectors desired in U, copy result to U
*             and generate left bidiagonalizing vectors in U
*             (CWorkspace: need 3*M-1, prefer 2*M+(M-1)*NB)
*             (RWorkspace: 0)
*
              CALL ZLACPY( 'L', M, M, A, LDA, U, LDU )
              CALL ZUNGBR( 'Q', M, M, N, U, LDU, WORK( ITAUQ ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
           IF( WNTVAS ) THEN
*
*             If right singular vectors desired in VT, copy result to
*             VT and generate right bidiagonalizing vectors in VT
*             (CWorkspace: need 2*M+NRVT, prefer 2*M+NRVT*NB)
*             (RWorkspace: 0)
*
              CALL ZLACPY( 'U', M, N, A, LDA, VT, LDVT )
              IF( WNTVA )
     $           NRVT = N
              IF( WNTVS )
     $           NRVT = M
              CALL ZUNGBR( 'P', NRVT, N, M, VT, LDVT, WORK( ITAUP ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
           IF( WNTUO ) THEN
*
*             If left singular vectors desired in A, generate left
*             bidiagonalizing vectors in A
*             (CWorkspace: need 3*M-1, prefer 2*M+(M-1)*NB)
*             (RWorkspace: 0)
```

```
*
               CALL ZUNGBR( 'Q', M, M, N, A, LDA, WORK( ITAUQ ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IF( WNTVO ) THEN
*
*              If right singular vectors desired in A, generate right
*              bidiagonalizing vectors in A
*              (CWorkspace: need 3*M, prefer 2*M+M*NB)
*              (RWorkspace: 0)
*
               CALL ZUNGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IRWORK = IE + M
            IF( WNTUAS .OR. WNTUO )
     $         NRU = M
            IF( WNTUN )
     $         NRU = 0
            IF( WNTVAS .OR. WNTVO )
     $         NCVT = N
            IF( WNTVN )
     $         NCVT = 0
            IF( ( .NOT.WNTUO ) .AND. ( .NOT.WNTVO ) ) THEN
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in U and computing right singular
*              vectors in VT
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAC)
*
               CALL ZBDSQR( 'L', M, NCVT, NRU, 0, S, RWORK( IE ), VT,
     $                      LDVT, U, LDU, CDUM, 1, RWORK( IRWORK ),
     $                      INFO )
            ELSE IF( ( .NOT.WNTUO ) .AND. WNTVO ) THEN
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in U and computing right singular
*              vectors in A
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAC)
*
               CALL ZBDSQR( 'L', M, NCVT, NRU, 0, S, RWORK( IE ), A,
     $                      LDA, U, LDU, CDUM, 1, RWORK( IRWORK ),
     $                      INFO )
            ELSE
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in A and computing right singular
*              vectors in VT
```

```
*              (CWorkspace: 0)
*              (RWorkspace: need BDSPAC)
*
               CALL ZBDSQR( 'L', M, NCVT, NRU, 0, S, RWORK( IE ), VT,
     $                      LDVT, A, LDA, CDUM, 1, RWORK( IRWORK ),
     $                      INFO )
            END IF
*
         END IF
*
      END IF
*
*     Undo scaling if necessary
*
      IF( ISCL.EQ.1 ) THEN
         IF( ANRM.GT.BIGNUM )
     $      CALL DLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( INFO.NE.0 .AND. ANRM.GT.BIGNUM )
     $      CALL DLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN-1, 1,
     $                   RWORK( IE ), MINMN, IERR )
         IF( ANRM.LT.SMLNUM )
     $      CALL DLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( INFO.NE.0 .AND. ANRM.LT.SMLNUM )
     $      CALL DLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN-1, 1,
     $                   RWORK( IE ), MINMN, IERR )
      END IF
*
*     Return optimal workspace in WORK(1)
*
      WORK( 1 ) = MAXWRK
*
      RETURN
*
*     End of ZGESVD
*
      END
```

———-

31.3.4 lisp code

— **zgesvd.lisp** —

```
(let* ((czero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (cone (f2cl-lib:cmplx 1.0d0 0.0d0))
       (zero 0.0d0)
```

```
      (one 1.0d0))
  (declare (type (f2cl-lib:complex16) czero)
           (type (f2cl-lib:complex16) cone)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable czero cone zero one))
  (defun zgesvd (jobu jobvt m n a lda s u ldu vt ldvt work lwork rwork info)
    (declare (type (array double-float (*)) rwork s)
             (type (array f2cl-lib:complex16 (*)) work vt u a)
             (type (f2cl-lib:integer4) info lwork ldvt ldu lda n m)
             (type (string *) jobvt jobu))
    (f2cl-lib:with-multi-array-data
        ((jobu character jobu-%data% jobu-%offset%)
         (jobvt character jobvt-%data% jobvt-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (u f2cl-lib:complex16 u-%data% u-%offset%)
         (vt f2cl-lib:complex16 vt-%data% vt-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (s double-float s-%data% s-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (prog ((cdum (make-array 1 :element-type 'f2cl-lib:complex16))
             (dum (make-array 1 :element-type 'double-float)) (anrm 0.0d0)
             (bignum 0.0d0) (eps 0.0d0) (smlnum 0.0d0) (lwork_zgeqrf 0)
             (lwork_zungqr_n 0) (lwork_zungqr_m 0) (lwork_zgebrd 0)
             (lwork_zungbr_p 0) (lwork_zungbr_q 0) (lwork_zgelqf 0)
             (lwork_zunglq_n 0) (lwork_zunglq_m 0) (blk 0) (chunk 0) (i 0)
             (ie 0) (ierr 0) (ir 0) (irwork 0) (iscl 0) (itau 0) (itaup 0)
             (itauq 0) (iu 0) (iwork 0) (ldwrkr 0) (ldwrku 0) (maxwrk 0)
             (minmn 0) (minwrk 0) (mnthr 0) (ncu 0) (ncvt 0) (nru 0) (nrvt 0)
             (wrkbl 0) (lquery nil) (wntua nil) (wntuas nil) (wntun nil)
             (wntuo nil) (wntus nil) (wntva nil) (wntvas nil) (wntvn nil)
             (wntvo nil) (wntvs nil))
        (declare (type (array f2cl-lib:complex16 (1)) cdum)
                 (type (array double-float (1)) dum)
                 (type (double-float) anrm bignum eps smlnum)
                 (type (f2cl-lib:integer4) lwork_zgeqrf lwork_zungqr_n
                  lwork_zungqr_m lwork_zgebrd lwork_zungbr_p lwork_zungbr_q
                  lwork_zgelqf lwork_zunglq_n lwork_zunglq_m blk chunk i ie
                  ierr ir irwork iscl itau itaup itauq iu iwork ldwrkr ldwrku
                  maxwrk minmn minwrk mnthr ncu ncvt nru nrvt wrkbl)
                 (type f2cl-lib:logical lquery wntua wntuas wntun wntuo wntus
                  wntva wntvas wntvn wntvo wntvs))
        (setf info 0)
        (setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (setf wntua
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame jobu "A")
                  (declare (ignore var-1))
                  (when var-0 (setf jobu var-0))
                  ret-val))
```

```
(setf wntus
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobu "S")
          (declare (ignore var-1))
          (when var-0 (setf jobu var-0))
          ret-val))
(setf wntuas (or wntua wntus))
(setf wntuo
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobu "O")
          (declare (ignore var-1))
          (when var-0 (setf jobu var-0))
          ret-val))
(setf wntun
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobu "N")
          (declare (ignore var-1))
          (when var-0 (setf jobu var-0))
          ret-val))
(setf wntva
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvt "A")
          (declare (ignore var-1))
          (when var-0 (setf jobvt var-0))
          ret-val))
(setf wntvs
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvt "S")
          (declare (ignore var-1))
          (when var-0 (setf jobvt var-0))
          ret-val))
(setf wntvas (or wntva wntvs))
(setf wntvo
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvt "O")
          (declare (ignore var-1))
          (when var-0 (setf jobvt var-0))
          ret-val))
(setf wntvn
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvt "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvt var-0))
          ret-val))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((not (or wntua wntus wntuo wntun)) (setf info -1))
      ((or (not (or wntva wntvs wntvo wntvn)) (and wntvo wntuo))
       (setf info -2))
      ((< m 0) (setf info -3)) ((< n 0) (setf info -4))
      ((< lda
```

```
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
        (setf info -6))
       ((or (< ldu 1) (and wntuas (< ldu m))) (setf info -9))
       ((or (< ldvt 1) (and wntva (< ldvt n))
            (and wntvs (< ldvt minmn)))
        (setf info -11)))
(cond
 ((= info 0) (setf minwrk 1) (setf maxwrk 1)
  (cond
   ((and (>= m n) (> minmn 0))
    (setf mnthr
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 6 "ZGESVD" (f2cl-lib:f2cl-// jobu jobvt) m n 0
                 0)
              (declare (ignore var-0 var-1 var-2 var-5 var-6))
              (setf m var-3)
              (setf n var-4)
              ret-val))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (zgeqrf m n a lda
         (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
         (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1))) -1
         ierr)
      (declare (ignore var-2 var-4 var-5 var-6))
      (setf m var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf ierr var-7))
    (setf lwork_zgeqrf (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (zungqr m n n a lda
         (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
         (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1))) -1
         ierr)
      (declare (ignore var-3 var-5 var-6 var-7))
      (when var-0 (setf m var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-8 (setf ierr var-8)))
    (setf lwork_zungqr_n
            (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (zungqr m m n a lda
         (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
         (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1))) -1
```

```
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf lwork_zungqr_m
        (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgebrd n n a lda s
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1))) -1
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(setf lwork_zgebrd (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (zungbr "P" n n n a lda
     (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1))) -1
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf ierr var-9)))
(setf lwork_zungbr_p
        (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (zungbr "Q" n n n a lda
     (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1))) -1
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf ierr var-9)))
```

```
(setf lwork_zungbr_q
        (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
(cond
 ((>= m mnthr)
  (cond
   (wntun (setf maxwrk (f2cl-lib:int-add n lwork_zgeqrf))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        lwork_zgebrd))))
    (if (or wntvo wntvas)
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                            lwork_zungbr_p)))))
    (setf minwrk (f2cl-lib:int-mul 3 n)))
   ((and wntuo wntvn)
    (setf wrkbl (f2cl-lib:int-add n lwork_zgeqrf))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add n lwork_zungqr_n))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        lwork_zgebrd))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        lwork_zungbr_q))))
    (setf maxwrk
            (max
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                                    (f2cl-lib:int-mul m n)))))
    (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))
   ((and wntuo wntvas)
    (setf wrkbl (f2cl-lib:int-add n lwork_zgeqrf))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add n lwork_zungqr_n))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
```

```
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                       lwork_zgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_zungbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_zungbr_p))))
 (setf maxwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                                 (f2cl-lib:int-mul m n)))))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))
((and wntus wntvn)
 (setf wrkbl (f2cl-lib:int-add n lwork_zgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_zungqr_n))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_zgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_zungbr_q))))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))
((and wntus wntvo)
 (setf wrkbl (f2cl-lib:int-add n lwork_zgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_zungqr_n))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_zgebrd))))
```

```
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_zungbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_zungbr_p))))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n n) wrkbl))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))
((and wntus wntvas)
 (setf wrkbl (f2cl-lib:int-add n lwork_zgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_zungqr_n))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_zgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_zungbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_zungbr_p))))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))
((and wntua wntvn)
 (setf wrkbl (f2cl-lib:int-add n lwork_zgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_zungqr_m))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     lwork_zgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
```

```
                                        lwork_zungbr_q))))
    (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
    (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))
   ((and wntua wntvo)
    (setf wrkbl (f2cl-lib:int-add n lwork_zgeqrf))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add n lwork_zungqr_m))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        lwork_zgebrd))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        lwork_zungbr_q))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        lwork_zungbr_p))))
    (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n n) wrkbl))
    (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))
   ((and wntua wntvas)
    (setf wrkbl (f2cl-lib:int-add n lwork_zgeqrf))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add n lwork_zungqr_m))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        lwork_zgebrd))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        lwork_zungbr_q))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        lwork_zungbr_p))))
    (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
    (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))))
  (t
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgebrd m n a lda s
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
     -1 ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(setf lwork_zgebrd
        (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
(setf maxwrk
        (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) lwork_zgebrd))
(cond
 ((or wntus wntuo)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (zungbr "Q" m n n a lda
       (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
       (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
       -1 ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))
  (setf lwork_zungbr_q
          (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                      lwork_zungbr_q))))))
(cond
 (wntua
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (zungbr "Q" m m n a lda
       (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
       (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
       -1 ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
```

```
        (when var-1 (setf m var-1))
        (when var-2 (setf m var-2))
        (when var-3 (setf n var-3))
        (when var-5 (setf lda var-5))
        (when var-9 (setf ierr var-9)))
      (setf lwork_zungbr_q
              (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
      (setf maxwrk
              (max (the f2cl-lib:integer4 maxwrk)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                          lwork_zungbr_q))))))
    (cond
     ((not wntvn)
      (setf maxwrk
              (max (the f2cl-lib:integer4 maxwrk)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                          lwork_zungbr_p))))
      (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))))))
((> minmn 0)
 (setf mnthr
         (multiple-value-bind
             (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
             (ilaenv 6 "ZGESVD" (f2cl-lib:f2cl-// jobu jobvt) m n 0
              0)
           (declare (ignore var-0 var-1 var-2 var-5 var-6))
           (setf m var-3)
           (setf n var-4)
           ret-val))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (zgelqf m n a lda
      (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
      (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1))) -1
      ierr)
   (declare (ignore var-2 var-4 var-5 var-6))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-7))
 (setf lwork_zgelqf (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (zunglq n n m
      (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1))) n
      (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
      (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1))) -1
      ierr)
   (declare (ignore var-3 var-5 var-6 var-7))
```

```
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf n var-4))
  (when var-8 (setf ierr var-8)))
(setf lwork_zunglq_n
        (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zunglq m n m a lda
     (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1))) -1
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf lwork_zunglq_m
        (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgebrd m m a lda s
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1))) -1
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(setf lwork_zgebrd (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (zungbr "P" m m m a n
     (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1))) -1
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf n var-5))
  (when var-9 (setf ierr var-9)))
(setf lwork_zungbr_p
        (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (zungbr "Q" m m m a n
     (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
     (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1))) -1
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf n var-5))
  (when var-9 (setf ierr var-9)))
(setf lwork_zungbr_q
        (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
(cond
 ((>= n mnthr)
  (cond
   (wntvn (setf maxwrk (f2cl-lib:int-add m lwork_zgelqf))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                        lwork_zgebrd))))
    (if (or wntuo wntuas)
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                            lwork_zungbr_q)))))
    (setf minwrk (f2cl-lib:int-mul 3 m)))
   ((and wntvo wntun)
    (setf wrkbl (f2cl-lib:int-add m lwork_zgelqf))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add m lwork_zunglq_m))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                        lwork_zgebrd))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                        lwork_zungbr_p))))
    (setf maxwrk
            (max
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
```

```
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                 (f2cl-lib:int-mul m n)))))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))
((and wntvo wntuas)
 (setf wrkbl (f2cl-lib:int-add m lwork_zgelqf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add m lwork_zunglq_m))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                     lwork_zgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                     lwork_zungbr_p))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                     lwork_zungbr_q))))
 (setf maxwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                 (f2cl-lib:int-mul m n)))))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))
((and wntvs wntun)
 (setf wrkbl (f2cl-lib:int-add m lwork_zgelqf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add m lwork_zunglq_m))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                     lwork_zgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                     lwork_zungbr_p))))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
```

```
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))
((and wntvs wntuo)
 (setf wrkbl (f2cl-lib:int-add m lwork_zgelqf))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add m lwork_zunglq_m))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    lwork_zgebrd))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    lwork_zungbr_p))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    lwork_zungbr_q))))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m m) wrkbl))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))
((and wntvs wntuas)
 (setf wrkbl (f2cl-lib:int-add m lwork_zgelqf))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add m lwork_zunglq_m))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    lwork_zgebrd))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    lwork_zungbr_p))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    lwork_zungbr_q))))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))
((and wntva wntun)
 (setf wrkbl (f2cl-lib:int-add m lwork_zgelqf))
 (setf wrkbl
```

```
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add m lwork_zunglq_n))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    lwork_zgebrd))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    lwork_zungbr_p))))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))
((and wntva wntuo)
 (setf wrkbl (f2cl-lib:int-add m lwork_zgelqf))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add m lwork_zunglq_n))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    lwork_zgebrd))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    lwork_zungbr_p))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    lwork_zungbr_q))))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m m) wrkbl))
 (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))
((and wntva wntuas)
 (setf wrkbl (f2cl-lib:int-add m lwork_zgelqf))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add m lwork_zunglq_n))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                    lwork_zgebrd))))
 (setf wrkbl
```

```
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                            lwork_zungbr_p))))
        (setf wrkbl
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                            lwork_zungbr_q))))
        (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
        (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n)))))
      (t
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10)
           (zgebrd m n a lda s
            (f2cl-lib:array-slice dum double-float (1) ((1 1)))
            (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
            (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
            (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
            -1 ierr)
        (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
        (setf m var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf ierr var-10))
       (setf lwork_zgebrd
               (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
       (setf maxwrk
               (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) lwork_zgebrd))
       (cond
        ((or wntvs wntvo)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9)
             (zungbr "P" m n m a n
              (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
              (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
              -1 ierr)
           (declare (ignore var-0 var-4 var-6 var-7 var-8))
           (when var-1 (setf m var-1))
           (when var-2 (setf n var-2))
           (when var-3 (setf m var-3))
           (when var-5 (setf n var-5))
           (when var-9 (setf ierr var-9)))
         (setf lwork_zungbr_p
                 (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
         (setf maxwrk
                 (max (the f2cl-lib:integer4 maxwrk)
                      (the f2cl-lib:integer4
```

```
                          (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                            lwork_zungbr_p))))))
        (cond
         (wntva
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9)
              (zungbr "P" n n m a n
               (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
               (f2cl-lib:array-slice cdum f2cl-lib:complex16 (1) ((1 1)))
               -1 ierr)
            (declare (ignore var-0 var-4 var-6 var-7 var-8))
            (when var-1 (setf n var-1))
            (when var-2 (setf n var-2))
            (when var-3 (setf m var-3))
            (when var-5 (setf n var-5))
            (when var-9 (setf ierr var-9)))
          (setf lwork_zungbr_p
                  (f2cl-lib:int (f2cl-lib:fref cdum (1) ((1 1)))))
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                              lwork_zungbr_p))))))
        (cond
         ((not wntun)
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add (f2cl-lib:int-mul 2 m)
                                              lwork_zungbr_q))))
          (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n))))))))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4 minwrk)))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce maxwrk 'f2cl-lib:complex16))
    (cond ((and (< lwork minwrk) (not lquery)) (setf info -13)))))
  (cond
   ((/= info 0) (xerbla "ZGESVD" (f2cl-lib:int-sub info)) (go end_label))
   (lquery (go end_label)))
  (cond ((or (= m 0) (= n 0)) (go end_label)))
  (setf eps (dlamch "P"))
  (setf smlnum (/ (f2cl-lib:fsqrt (dlamch "S")) eps))
  (setf bignum (/ one smlnum))
  (setf anrm
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
              (zlange "M" m n a lda dum)
            (declare (ignore var-0 var-3 var-5))
```

```
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iscl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf iscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 anrm smlnum m n a lda ierr)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf ierr var-9))))
 ((> anrm bignum) (setf iscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 anrm bignum m n a lda ierr)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf ierr var-9)))))
(cond
 ((>= m n)
  (cond
   ((>= m mnthr)
    (cond
     (wntun (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (zgeqrf m n a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
        (declare (ignore var-2 var-4 var-5 var-6))
        (setf m var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf ierr var-7))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
           czero czero
```

```
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (2 1)
                            ((1 lda) (1 *)) a-%offset%)
      lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(setf ie 1) (setf itauq 1)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgebrd n n a lda s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(setf ncvt 0)
(cond
 ((or wntvo wntvas)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (zungbr "P" n n n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (itaup) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))
  (setf ncvt n)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
        var-10 var-11 var-12 var-13 var-14)
       (zbdsqr "U" n ncvt 0 0 s
        (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                              rwork-%offset%)
        a lda cdum 1 cdum 1
        (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                              ((1 *)) rwork-%offset%)
        info)
      (declare
       (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
        var-11 var-12 var-13))
      (setf n var-1)
      (setf ncvt var-2)
      (setf lda var-8)
      (setf info var-14))
    (if wntvas
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zlacpy "F" n n a lda vt ldvt)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          (when var-6 (setf ldvt var-6)))))
  ((and wntuo wntvn)
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                           (f2cl-lib:int-mul 3 n)))
      (setf ir 1)
      (cond
       ((>= lwork
            (f2cl-lib:int-add
             (max (the f2cl-lib:integer4 wrkbl)
                  (the f2cl-lib:integer4 (f2cl-lib:int-mul lda n)))
             (f2cl-lib:int-mul lda n)))
        (setf ldwrku lda) (setf ldwrkr lda))
       ((>= lwork
            (f2cl-lib:int-add
             (max (the f2cl-lib:integer4 wrkbl)
                  (the f2cl-lib:integer4 (f2cl-lib:int-mul lda n)))
             (f2cl-lib:int-mul n n)))
        (setf ldwrku lda) (setf ldwrkr n))
       (t
        (setf ldwrku
                (the f2cl-lib:integer4
                     (truncate (- lwork (* n n)) n)))
        (setf ldwrkr n)))
      (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
      (setf iwork (f2cl-lib:int-add itau n))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgeqrf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ ir 1)) ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zungqr m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
```

```
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "Q" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" n 0 n 0 s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     cdum 1
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
```

```
                              ((1 *)) work-%offset%)
      ldwrkr cdum 1
      (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                              ((1 *)) rwork-%offset%)
      info)
  (declare
   (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
    var-11 var-12 var-13))
  (setf n var-1)
  (setf n var-3)
  (setf ldwrkr var-10)
  (setf info var-14))
(setf iu itauq)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i ldwrku))
              ((> i m) nil)
  (tagbody
    (setf chunk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
             (the f2cl-lib:integer4 ldwrku)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (zgemm "N" "N" chunk n n cone
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               (i 1) ((1 lda) (1 *))
                               a-%offset%)
         lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (ir) ((1 *)) work-%offset%)
         ldwrkr czero
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (iu) ((1 *)) work-%offset%)
         ldwrku)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf chunk var-2))
      (when var-3 (setf n var-3))
      (when var-4 (setf n var-4))
      (when var-5 (setf cone var-5))
      (when var-7 (setf lda var-7))
      (when var-9 (setf ldwrkr var-9))
      (when var-10 (setf czero var-10))
      (when var-12 (setf ldwrku var-12)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlacpy "F" chunk n
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (iu) ((1 *)) work-%offset%)
         ldwrku
```

```
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                  (i 1) ((1 lda) (1 *))
                                  a-%offset%)
            lda)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf chunk var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf ldwrku var-4))
        (when var-6 (setf lda var-6)))
    label10)))
(t (setf ie 1) (setf itauq 1)
 (setf itaup (f2cl-lib:int-add itauq n))
 (setf iwork (f2cl-lib:int-add itaup n))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10)
     (zgebrd m n a lda s
      (f2cl-lib:array-slice rwork-%data% double-float (ie)
                            ((1 *)) rwork-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (itauq) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (itaup) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-10))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9)
     (zungbr "Q" m n n a lda
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (itauq) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
   (declare (ignore var-0 var-4 var-6 var-7 var-8))
   (when var-1 (setf m var-1))
   (when var-2 (setf n var-2))
   (when var-3 (setf n var-3))
   (when var-5 (setf lda var-5))
   (when var-9 (setf ierr var-9)))
 (setf irwork (f2cl-lib:int-add ie n))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12 var-13 var-14)
```

```
        (zbdsqr "U" n 0 m 0 s
         (f2cl-lib:array-slice rwork-%data% double-float (ie)
                               ((1 *)) rwork-%offset%)
         cdum 1 a lda cdum 1
         (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                               ((1 *)) rwork-%offset%)
         info)
      (declare
       (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
        var-11 var-12 var-13))
      (setf n var-1)
      (setf m var-3)
      (setf lda var-10)
      (setf info var-14)))))
((and wntuo wntvas)
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                         (f2cl-lib:int-mul 3 n)))
   (setf ir 1)
   (cond
    ((>= lwork
         (f2cl-lib:int-add
          (max (the f2cl-lib:integer4 wrkbl)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul lda n)))
          (f2cl-lib:int-mul lda n)))
     (setf ldwrku lda) (setf ldwrkr lda))
    ((>= lwork
         (f2cl-lib:int-add
          (max (the f2cl-lib:integer4 wrkbl)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul lda n)))
          (f2cl-lib:int-mul n n)))
     (setf ldwrku lda) (setf ldwrkr n))
    (t
     (setf ldwrku
             (the f2cl-lib:integer4
                  (truncate (- lwork (* n n)) n)))
     (setf ldwrkr n)))
   (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
   (setf iwork (f2cl-lib:int-add itau n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (zgeqrf m n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itau) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6))
     (setf m var-0)
```

```
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "U" n n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(if (> n 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlaset "L" (f2cl-lib:int-sub n 1)
         (f2cl-lib:int-sub n 1) czero czero
         (f2cl-lib:array-slice vt-%data% f2cl-lib:complex16
                               (2 1) ((1 ldvt) (1 *))
                               vt-%offset%)
         ldvt)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldvt var-6))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zungqr m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd n n vt ldvt s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
```

```
                              (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldvt var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" n n vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                              ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldvt var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "Q" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                              ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "P" n n n vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
```

```
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" n n n 0 s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr cdum 1
     (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf n var-1)
  (setf n var-2)
  (setf n var-3)
  (setf ldvt var-8)
  (setf ldwrkr var-10)
  (setf info var-14))
(setf iu itauq)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i ldwrku))
              ((> i m) nil)
  (tagbody
    (setf chunk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
             (the f2cl-lib:integer4 ldwrku)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (zgemm "N" "N" chunk n n cone
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               (i 1) ((1 lda) (1 *))
                               a-%offset%)
         lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (ir) ((1 *)) work-%offset%)
         ldwrkr czero
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (iu) ((1 *)) work-%offset%)
         ldwrku)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
```

```
        (when var-2 (setf chunk var-2))
        (when var-3 (setf n var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf cone var-5))
        (when var-7 (setf lda var-7))
        (when var-9 (setf ldwrkr var-9))
        (when var-10 (setf czero var-10))
        (when var-12 (setf ldwrku var-12)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlacpy "F" chunk n
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                 (iu) ((1 *)) work-%offset%)
           ldwrku
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                 (i 1) ((1 lda) (1 *))
                                 a-%offset%)
           lda)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf chunk var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf ldwrku var-4))
        (when var-6 (setf lda var-6)))
    label20)))
(t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (zgeqrf m n a lda
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (itau) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-7))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (zlacpy "U" n n a lda vt ldvt)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldvt var-6)))
 (if (> n 1)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (zlaset "L" (f2cl-lib:int-sub n 1)
```

```
            (f2cl-lib:int-sub n 1) czero czero
            (f2cl-lib:array-slice vt-%data% f2cl-lib:complex16
                                  (2 1) ((1 ldvt) (1 *))
                                  vt-%offset%)
            ldvt)
        (declare (ignore var-0 var-1 var-2 var-5))
        (when var-3 (setf czero var-3))
        (when var-4 (setf czero var-4))
        (when var-6 (setf ldvt var-6))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zungqr m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd n n vt ldvt s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldvt var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (zunmbr "Q" "R" "N" m n n vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
```

```
          a lda
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                (iwork) ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
        (declare
         (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
        (when var-3 (setf m var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf n var-5))
        (when var-7 (setf ldvt var-7))
        (when var-10 (setf lda var-10))
        (when var-13 (setf ierr var-13)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9)
          (zungbr "P" n n n vt ldvt
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                 (itaup) ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                 (iwork) ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
        (declare (ignore var-0 var-4 var-6 var-7 var-8))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf n var-3))
        (when var-5 (setf ldvt var-5))
        (when var-9 (setf ierr var-9)))
      (setf irwork (f2cl-lib:int-add ie n))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12 var-13 var-14)
          (zbdsqr "U" n n m 0 s
           (f2cl-lib:array-slice rwork-%data% double-float (ie)
                                 ((1 *)) rwork-%offset%)
           vt ldvt a lda cdum 1
           (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                                 ((1 *)) rwork-%offset%)
           info)
        (declare
         (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
          var-13))
        (setf n var-1)
        (setf n var-2)
        (setf m var-3)
        (setf ldvt var-8)
        (setf lda var-10)
        (setf info var-14)))))
  (wntus
   (cond
    (wntvn
```

```
(cond
 ((>= lwork
      (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                        (f2cl-lib:int-mul 3 n)))
  (setf ir 1)
  (cond
   ((>= lwork
        (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda n)))
    (setf ldwrkr lda))
   (t (setf ldwrkr n)))
  (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
  (setf iwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "U" n n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (ir) ((1 *)) work-%offset%)
       ldwrkr)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldwrkr var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
       czero czero
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             ((+ ir 1)) ((1 *)) work-%offset%)
       ldwrkr)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf czero var-3))
    (when var-4 (setf czero var-4))
    (when var-6 (setf ldwrkr var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
```

```
    (zungqr m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (ir) ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "Q" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (ir) ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
```

```
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf ldwrkr var-5))
    (when var-9 (setf ierr var-9)))
  (setf irwork (f2cl-lib:int-add ie n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13 var-14)
      (zbdsqr "U" n 0 n 0 s
       (f2cl-lib:array-slice rwork-%data% double-float (ie)
                             ((1 *)) rwork-%offset%)
       cdum 1
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (ir) ((1 *)) work-%offset%)
       ldwrkr cdum 1
       (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                             ((1 *)) rwork-%offset%)
       info)
    (declare
     (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
      var-11 var-12 var-13))
    (setf n var-1)
    (setf n var-3)
    (setf ldwrkr var-10)
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "N" "N" m n n cone a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (ir) ((1 *)) work-%offset%)
       ldwrkr czero u ldu)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldwrkr var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf ldu var-12))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
```

```
                              (itau) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zungqr m n n u ldu
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itau) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
      czero czero
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (2 1)
                              ((1 lda) (1 *)) a-%offset%)
      lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10)
```

```
    (zgebrd n n a lda s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (zunmbr "Q" "R" "N" m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" n 0 m 0 s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     cdum 1 u ldu cdum 1
     (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
```

```
        var-11 var-12 var-13))
      (setf n var-1)
      (setf m var-3)
      (setf ldu var-10)
      (setf info var-14)))))
(wntvo
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul 2 n n)
                         (f2cl-lib:int-mul 3 n)))
   (setf iu 1)
   (cond
    ((>= lwork
         (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 lda n)))
     (setf ldwrku lda)
     (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
     (setf ldwrkr lda))
    ((>= lwork
         (f2cl-lib:int-add wrkbl
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-add lda n) n)))
     (setf ldwrku lda)
     (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
     (setf ldwrkr n))
    (t (setf ldwrku n)
     (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
     (setf ldwrkr n)))
   (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
   (setf iwork (f2cl-lib:int-add itau n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (zgeqrf m n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itau) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
     (declare (ignore var-2 var-4 var-5 var-6))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-7))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (zlacpy "U" n n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iu) ((1 *)) work-%offset%)
        ldwrku)
     (declare (ignore var-0 var-3 var-5))
```

```
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ iu 1)) ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zungqr m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
```

```
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "U" n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (ir) ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "Q" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "P" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (ir) ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
```

```
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" n n n 0 s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (ir) ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku cdum 1
     (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf n var-1)
  (setf n var-2)
  (setf n var-3)
  (setf ldwrkr var-8)
  (setf ldwrku var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (zgemm "N" "N" m n n cone a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku czero u ldu)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf cone var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldwrku var-9))
  (when var-10 (setf czero var-10))
  (when var-12 (setf ldu var-12)))
(multiple-value-bind
```

```
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "F" n n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (ir) ((1 *)) work-%offset%)
       ldwrkr a lda)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldwrkr var-4))
    (when var-6 (setf lda var-6))))
  (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
   (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
   (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
   (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zungqr m n n u ldu
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf m var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldu var-4))
    (when var-8 (setf ierr var-8)))
   (setf ie 1) (setf itauq itau)
   (setf itaup (f2cl-lib:int-add itauq n))
```

```
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (2 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd n n a lda s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (zunmbr "Q" "R" "N" m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
```

```
      (when var-10 (setf ldu var-10))
      (when var-13 (setf ierr var-13)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9)
        (zungbr "P" n n n a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (itaup) ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (iwork) ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
         ierr)
      (declare (ignore var-0 var-4 var-6 var-7 var-8))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf n var-3))
      (when var-5 (setf lda var-5))
      (when var-9 (setf ierr var-9)))
    (setf irwork (f2cl-lib:int-add ie n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14)
        (zbdsqr "U" n n m 0 s
         (f2cl-lib:array-slice rwork-%data% double-float (ie)
                               ((1 *)) rwork-%offset%)
         a lda u ldu cdum 1
         (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                               ((1 *)) rwork-%offset%)
         info)
      (declare
       (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
        var-13))
      (setf n var-1)
      (setf n var-2)
      (setf m var-3)
      (setf lda var-8)
      (setf ldu var-10)
      (setf info var-14)))))
(wntvas
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                         (f2cl-lib:int-mul 3 n)))
    (setf iu 1)
    (cond
     ((>= lwork
          (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda n)))
      (setf ldwrku lda))
     (t (setf ldwrku n)))
    (setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
```

```
(setf iwork (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgeqrf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ iu 1)) ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zungqr m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
```

```
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "U" n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "Q" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
```

```
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "P" n n n vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" n n n 0 s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku cdum 1
     (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf n var-1)
  (setf n var-2)
  (setf n var-3)
  (setf ldvt var-8)
  (setf ldwrku var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
```

```
      (zgemm "N" "N" m n n cone a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (iu) ((1 *)) work-%offset%)
       ldwrku czero u ldu)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldwrku var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf ldu var-12))))
  (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
   (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
   (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
   (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zungqr m n n u ldu
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf m var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldu var-4))
```

```
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "U" n n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(if (> n 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlaset "L" (f2cl-lib:int-sub n 1)
         (f2cl-lib:int-sub n 1) czero czero
         (f2cl-lib:array-slice vt-%data% f2cl-lib:complex16
                               (2 1) ((1 ldvt) (1 *))
                               vt-%offset%)
         ldvt)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldvt var-6))))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd n n vt ldvt s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldvt var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (zunmbr "Q" "R" "N" m n n vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
```

```
                              (itauq) ((1 *)) work-%offset%)
      u ldu
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldvt var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "P" n n n vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" n n m 0 s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                              ((1 *)) rwork-%offset%)
     vt ldvt u ldu cdum 1
     (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                              ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf n var-1)
  (setf n var-2)
  (setf m var-3)
  (setf ldvt var-8)
  (setf ldu var-10)
```

```
        (setf info var-14)))))))
(wntua
 (cond
  (wntvn
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add n m))
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-mul 3 n)))))
     (setf ir 1)
     (cond
      ((>= lwork
           (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda n)))
       (setf ldwrkr lda))
      (t (setf ldwrkr n)))
     (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
     (setf iwork (f2cl-lib:int-add itau n))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (zgeqrf m n a lda
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                (itau) ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                (iwork) ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
          ierr)
       (declare (ignore var-2 var-4 var-5 var-6))
       (setf m var-0)
       (setf n var-1)
       (setf lda var-3)
       (setf ierr var-7))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (zlacpy "L" m n a lda u ldu)
       (declare (ignore var-0 var-3 var-5))
       (when var-1 (setf m var-1))
       (when var-2 (setf n var-2))
       (when var-4 (setf lda var-4))
       (when var-6 (setf ldu var-6)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (zlacpy "U" n n a lda
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                (ir) ((1 *)) work-%offset%)
          ldwrkr)
       (declare (ignore var-0 var-3 var-5))
       (when var-1 (setf n var-1))
```

```
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ ir 1)) ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zungqr m m n u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (ir) ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
```

```
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "Q" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (ir) ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" n 0 n 0 s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     cdum 1
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (ir) ((1 *)) work-%offset%)
     ldwrkr cdum 1
     (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
    var-11 var-12 var-13))
  (setf n var-1)
  (setf n var-3)
  (setf ldwrkr var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (zgemm "N" "N" m n n cone u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
```

```
                              (ir) ((1 *)) work-%offset%)
      ldwrkr czero a lda)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf ldu var-7))
    (when var-9 (setf ldwrkr var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf lda var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "F" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zungqr m m n u ldu
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
```

```
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (2 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd n n a lda s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (zunmbr "Q" "R" "N" m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
```

```
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
          ierr)
        (declare
         (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
          var-12))
        (when var-3 (setf m var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf n var-5))
        (when var-7 (setf lda var-7))
        (when var-10 (setf ldu var-10))
        (when var-13 (setf ierr var-13)))
      (setf irwork (f2cl-lib:int-add ie n))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12 var-13 var-14)
          (zbdsqr "U" n 0 m 0 s
           (f2cl-lib:array-slice rwork-%data% double-float (ie)
                                 ((1 *)) rwork-%offset%)
           cdum 1 u ldu cdum 1
           (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                                 ((1 *)) rwork-%offset%)
           info)
        (declare
         (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
          var-11 var-12 var-13))
        (setf n var-1)
        (setf m var-3)
        (setf ldu var-10)
        (setf info var-14)))))
  (wntvo
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul 2 n n)
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add n m))
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-mul 3 n)))))
      (setf iu 1)
      (cond
       ((>= lwork
            (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 lda n)))
        (setf ldwrku lda)
        (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
        (setf ldwrkr lda))
       ((>= lwork
            (f2cl-lib:int-add wrkbl
                              (f2cl-lib:int-mul
                               (f2cl-lib:int-add lda n) n)))
        (setf ldwrku lda)
```

```
  (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
  (setf ldwrkr n))
 (t (setf ldwrku n)
  (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
  (setf ldwrkr n)))
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
(setf iwork (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgeqrf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zungqr m m n u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku)
```

```
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ iu 1)) ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrku var-6)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "U" n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (ir) ((1 *)) work-%offset%)
```

```
      ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "Q" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "P" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (ir) ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" n n n 0 s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
```

```
                          ((1 *)) rwork-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                          (ir) ((1 *)) work-%offset%)
      ldwrkr
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                          (iu) ((1 *)) work-%offset%)
      ldwrku cdum 1
      (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                          ((1 *)) rwork-%offset%)
      info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf n var-1)
  (setf n var-2)
  (setf n var-3)
  (setf ldwrkr var-8)
  (setf ldwrku var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (zgemm "N" "N" m n n cone u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                          (iu) ((1 *)) work-%offset%)
     ldwrku czero a lda)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf cone var-5))
  (when var-7 (setf ldu var-7))
  (when var-9 (setf ldwrku var-9))
  (when var-10 (setf czero var-10))
  (when var-12 (setf lda var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "F" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "F" n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                          (ir) ((1 *)) work-%offset%)
     ldwrkr a lda)
  (declare (ignore var-0 var-3 var-5))
```

```
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldwrkr var-4))
    (when var-6 (setf lda var-6))))
  (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (zgeqrf m n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itau) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (zlacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (zungqr m m n u ldu
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itau) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf m var-0))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldu var-4))
    (when var-8 (setf ierr var-8)))
   (setf ie 1) (setf itauq itau)
   (setf itaup (f2cl-lib:int-add itauq n))
   (setf iwork (f2cl-lib:int-add itaup n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (zlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
        czero czero
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (2 1)
```

```
                          ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd n n a lda s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                          ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                          (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                          (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                          (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (zunmbr "Q" "R" "N" m n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                          (itauq) ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                          (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "P" n n n a lda
```

```
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                  (itaup) ((1 *)) work-%offset%)
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                  (iwork) ((1 *)) work-%offset%)
            (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
            ierr)
          (declare (ignore var-0 var-4 var-6 var-7 var-8))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-3 (setf n var-3))
          (when var-5 (setf lda var-5))
          (when var-9 (setf ierr var-9)))
        (setf irwork (f2cl-lib:int-add ie n))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12 var-13 var-14)
            (zbdsqr "U" n n m 0 s
             (f2cl-lib:array-slice rwork-%data% double-float (ie)
                                   ((1 *)) rwork-%offset%)
             a lda u ldu cdum 1
             (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                                   ((1 *)) rwork-%offset%)
             info)
          (declare
           (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
            var-13))
          (setf n var-1)
          (setf n var-2)
          (setf m var-3)
          (setf lda var-8)
          (setf ldu var-10)
          (setf info var-14)))))
      (wntvas
       (cond
        ((>= lwork
             (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                               (max
                                (the f2cl-lib:integer4
                                     (f2cl-lib:int-add n m))
                                (the f2cl-lib:integer4
                                     (f2cl-lib:int-mul 3 n)))))
        (setf iu 1)
        (cond
         ((>= lwork
              (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda n)))
          (setf ldwrku lda))
         (t (setf ldwrku n)))
        (setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
        (setf iwork (f2cl-lib:int-add itau n))
        (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgeqrf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zungqr m m n u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
```

```
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ iu 1)) ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrku var-6)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "U" n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "Q" n n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
```

```
                              (iu) ((1 *)) work-%offset%)
      ldwrku
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itauq) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "P" n n n vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" n n n 0 s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                              ((1 *)) rwork-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iu) ((1 *)) work-%offset%)
     ldwrku cdum 1
     (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                              ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf n var-1)
  (setf n var-2)
  (setf n var-3)
```

```
    (setf ldvt var-8)
    (setf ldwrku var-10)
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "N" "N" m n n cone u ldu
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (iu) ((1 *)) work-%offset%)
       ldwrku czero a lda)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf ldu var-7))
    (when var-9 (setf ldwrku var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf lda var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "F" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zungqr m m n u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "U" n n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(if (> n 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlaset "L" (f2cl-lib:int-sub n 1)
         (f2cl-lib:int-sub n 1) czero czero
         (f2cl-lib:array-slice vt-%data% f2cl-lib:complex16
                               (2 1) ((1 ldvt) (1 *))
                               vt-%offset%)
         ldvt)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldvt var-6))))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd n n vt ldvt s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
```

```
                              (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldvt var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (zunmbr "Q" "R" "N" m n n vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itauq) ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldvt var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "P" n n n vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" n n m 0 s
```

```
                (f2cl-lib:array-slice rwork-%data% double-float (ie)
                                      ((1 *)) rwork-%offset%)
                vt ldvt u ldu cdum 1
                (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                                      ((1 *)) rwork-%offset%)
                info)
              (declare
               (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
                var-13))
              (setf n var-1)
              (setf n var-2)
              (setf m var-3)
              (setf ldvt var-8)
              (setf ldu var-10)
              (setf info var-14)))))))))
    (t (setf ie 1) (setf itauq 1)
     (setf itaup (f2cl-lib:int-add itauq n))
     (setf iwork (f2cl-lib:int-add itaup n))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (zgebrd m n a lda s
          (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                                rwork-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
       (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
       (setf m var-0)
       (setf n var-1)
       (setf lda var-3)
       (setf ierr var-10))
     (cond
      (wntuas
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
           (zlacpy "L" m n a lda u ldu)
         (declare (ignore var-0 var-3 var-5))
         (when var-1 (setf m var-1))
         (when var-2 (setf n var-2))
         (when var-4 (setf lda var-4))
         (when var-6 (setf ldu var-6)))
       (if wntus
           (setf ncu n))
       (if wntua
           (setf ncu m))
       (multiple-value-bind
```

```
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zungbr "Q" m ncu n u ldu
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf ncu var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf ldu var-5))
    (when var-9 (setf ierr var-9)))))
(cond
 (wntvas
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "U" n n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zungbr "P" n n n vt ldvt
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf ldvt var-5))
    (when var-9 (setf ierr var-9)))))
(cond
 (wntuo
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zungbr "Q" m n n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
```

```
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))))
(cond
 (wntvo
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zungbr "P" n n n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))))
(setf irwork (f2cl-lib:int-add ie n))
(if (or wntuas wntuo)
    (setf nru m))
(if wntun
    (setf nru 0))
(if (or wntvas wntvo)
    (setf ncvt n))
(if wntvn
    (setf ncvt 0))
(cond
 ((and (not wntuo) (not wntvo))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (zbdsqr "U" n ncvt nru 0 s
       (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                             rwork-%offset%)
       vt ldvt u ldu cdum 1
       (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                             ((1 *)) rwork-%offset%)
       info)
    (declare
     (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
      var-13))
    (setf n var-1)
    (setf ncvt var-2)
    (setf nru var-3)
    (setf ldvt var-8)
    (setf ldu var-10)
    (setf info var-14)))
 ((and (not wntuo) wntvo)
  (multiple-value-bind
```

```
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11 var-12 var-13 var-14)
            (zbdsqr "U" n ncvt nru 0 s
             (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                                   rwork-%offset%)
             a lda u ldu cdum 1
             (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                                   ((1 *)) rwork-%offset%)
             info)
          (declare
           (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
            var-13))
          (setf n var-1)
          (setf ncvt var-2)
          (setf nru var-3)
          (setf lda var-8)
          (setf ldu var-10)
          (setf info var-14)))
      (t
       (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11 var-12 var-13 var-14)
            (zbdsqr "U" n ncvt nru 0 s
             (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                                   rwork-%offset%)
             vt ldvt a lda cdum 1
             (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                                   ((1 *)) rwork-%offset%)
             info)
          (declare
           (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
            var-13))
          (setf n var-1)
          (setf ncvt var-2)
          (setf nru var-3)
          (setf ldvt var-8)
          (setf lda var-10)
          (setf info var-14)))))))
(t
 (cond
  ((>= n mnthr)
   (cond
    (wntvn (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
     (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (zgelqf m n a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                                 ((1 *)) work-%offset%)
```

```
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(setf ie 1) (setf itauq 1)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgebrd m m a lda s
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(cond
 ((or wntuo wntuas)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (zungbr "Q" m m m a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (itauq) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
```

```
        (when var-2 (setf m var-2))
        (when var-3 (setf m var-3))
        (when var-5 (setf lda var-5))
        (when var-9 (setf ierr var-9)))))
    (setf irwork (f2cl-lib:int-add ie m)) (setf nru 0)
    (if (or wntuo wntuas)
        (setf nru m))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14)
        (zbdsqr "U" m 0 nru 0 s
         (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                               rwork-%offset%)
         cdum 1 a lda cdum 1
         (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                               ((1 *)) rwork-%offset%)
         info)
      (declare
       (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9 var-11
        var-12 var-13))
      (setf m var-1)
      (setf nru var-3)
      (setf lda var-10)
      (setf info var-14))
    (if wntuas
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zlacpy "F" m m a lda u ldu)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf m var-2))
          (when var-4 (setf lda var-4))
          (when var-6 (setf ldu var-6)))))
  ((and wntvo wntun)
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                           (f2cl-lib:int-mul 3 m)))
      (setf ir 1)
      (cond
       ((>= lwork
            (f2cl-lib:int-add
             (max (the f2cl-lib:integer4 wrkbl)
                  (the f2cl-lib:integer4 (f2cl-lib:int-mul lda n)))
             (f2cl-lib:int-mul lda m)))
        (setf ldwrku lda) (setf chunk n) (setf ldwrkr lda))
       ((>= lwork
            (f2cl-lib:int-add
             (max (the f2cl-lib:integer4 wrkbl)
                  (the f2cl-lib:integer4 (f2cl-lib:int-mul lda n)))
```

```
       (f2cl-lib:int-mul m m)))
  (setf ldwrku lda) (setf chunk n) (setf ldwrkr m))
 (t (setf ldwrku m)
  (setf chunk
          (the f2cl-lib:integer4
               (truncate (- lwork (* m m)) m)))
  (setf ldwrkr m)))
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
(setf iwork (f2cl-lib:int-add itau m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgelqf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ ir ldwrkr)) ((1 *))
                           work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zunglq m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
```

```
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "P" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
```

```
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" m m 0 0 s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr cdum 1 cdum 1
     (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
    var-11 var-12 var-13))
  (setf m var-1)
  (setf m var-2)
  (setf ldwrkr var-8)
  (setf info var-14))
(setf iu itauq)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
              ((> i n) nil)
  (tagbody
    (setf blk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
             (the f2cl-lib:integer4 chunk)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (zgemm "N" "N" m blk m cone
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (ir) ((1 *)) work-%offset%)
         ldwrkr
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               (1 i) ((1 lda) (1 *))
                               a-%offset%)
         lda czero
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (iu) ((1 *)) work-%offset%)
         ldwrku)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf blk var-3))
      (when var-4 (setf m var-4))
      (when var-5 (setf cone var-5))
      (when var-7 (setf ldwrkr var-7))
```

```
          (when var-9 (setf lda var-9))
          (when var-10 (setf czero var-10))
          (when var-12 (setf ldwrku var-12)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zlacpy "F" m blk
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                   (iu) ((1 *)) work-%offset%)
             ldwrku
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   (1 i) ((1 lda) (1 *))
                                   a-%offset%)
             lda)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf blk var-2))
          (when var-4 (setf ldwrku var-4))
          (when var-6 (setf lda var-6)))
      label30)))
  (t (setf ie 1) (setf itauq 1)
   (setf itaup (f2cl-lib:int-add itauq m))
   (setf iwork (f2cl-lib:int-add itaup m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (zgebrd m n a lda s
        (f2cl-lib:array-slice rwork-%data% double-float (ie)
                              ((1 *)) rwork-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itauq) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itaup) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-10))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9)
       (zungbr "P" m n m a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itaup) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
     (declare (ignore var-0 var-4 var-6 var-7 var-8))
```

```
      (when var-1 (setf m var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf m var-3))
      (when var-5 (setf lda var-5))
      (when var-9 (setf ierr var-9)))
    (setf irwork (f2cl-lib:int-add ie m))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14)
        (zbdsqr "L" m n 0 0 s
         (f2cl-lib:array-slice rwork-%data% double-float (ie)
                               ((1 *)) rwork-%offset%)
         a lda cdum 1 cdum 1
         (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                               ((1 *)) rwork-%offset%)
         info)
      (declare
       (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
        var-11 var-12 var-13))
      (setf m var-1)
      (setf n var-2)
      (setf lda var-8)
      (setf info var-14)))))
((and wntvo wntuas)
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                         (f2cl-lib:int-mul 3 m)))
   (setf ir 1)
   (cond
    ((>= lwork
         (f2cl-lib:int-add
          (max (the f2cl-lib:integer4 wrkbl)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul lda n)))
          (f2cl-lib:int-mul lda m)))
     (setf ldwrku lda) (setf chunk n) (setf ldwrkr lda))
    ((>= lwork
         (f2cl-lib:int-add
          (max (the f2cl-lib:integer4 wrkbl)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul lda n)))
          (f2cl-lib:int-mul m m)))
     (setf ldwrku lda) (setf chunk n) (setf ldwrkr m))
    (t (setf ldwrku m)
     (setf chunk
             (the f2cl-lib:integer4
                  (truncate (- lwork (* m m)) m)))
     (setf ldwrkr m)))
   (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
   (setf iwork (f2cl-lib:int-add itau m))
   (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgelqf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" m m a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice u-%data% f2cl-lib:complex16 (1 2)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zunglq m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
     var-9 var-10)
    (zgebrd m m u ldu s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldu var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "U" m m u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldu var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "P" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
```

```
                          (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                          (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" m m m 0 s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                          ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ir)
                          ((1 *)) work-%offset%)
     ldwrkr u ldu cdum 1
     (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                          ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf m var-1)
  (setf m var-2)
  (setf m var-3)
  (setf ldwrkr var-8)
  (setf ldu var-10)
  (setf info var-14))
(setf iu itauq)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
              ((> i n) nil)
  (tagbody
    (setf blk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
             (the f2cl-lib:integer4 chunk)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (zgemm "N" "N" m blk m cone
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (ir) ((1 *)) work-%offset%)
         ldwrkr
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               (1 i) ((1 lda) (1 *))
```

```
                                a-%offset%)
            lda czero
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                (iu) ((1 *)) work-%offset%)
            ldwrku)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf blk var-3))
        (when var-4 (setf m var-4))
        (when var-5 (setf cone var-5))
        (when var-7 (setf ldwrkr var-7))
        (when var-9 (setf lda var-9))
        (when var-10 (setf czero var-10))
        (when var-12 (setf ldwrku var-12)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlacpy "F" m blk
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                (iu) ((1 *)) work-%offset%)
           ldwrku
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                (1 i) ((1 lda) (1 *))
                                a-%offset%)
           lda)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf blk var-2))
        (when var-4 (setf ldwrku var-4))
        (when var-6 (setf lda var-6)))
    label40)))
(t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (zgelqf m n a lda
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (itau) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-7))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (zlacpy "L" m m a lda u ldu)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf m var-1))
   (when var-2 (setf m var-2))
```

```
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice u-%data% f2cl-lib:complex16 (1 2)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zunglq m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd m m u ldu s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldu var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
     var-9 var-10 var-11 var-12 var-13)
    (zunmbr "P" "L" "C" m n m u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldu var-7))
  (when var-10 (setf lda var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" m n m 0 s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     a lda u ldu cdum 1
     (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf m var-1)
  (setf n var-2)
  (setf m var-3)
  (setf lda var-8)
  (setf ldu var-10)
```

```
      (setf info var-14)))))
 (wntvs
  (cond
   (wntun
    (cond
     ((>= lwork
          (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                            (f2cl-lib:int-mul 3 m)))
      (setf ir 1)
      (cond
       ((>= lwork
            (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda m)))
        (setf ldwrkr lda))
       (t (setf ldwrkr m)))
      (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
      (setf iwork (f2cl-lib:int-add itau m))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (zgelqf m n a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                 (itau) ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                 (iwork) ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
           ierr)
        (declare (ignore var-2 var-4 var-5 var-6))
        (setf m var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf ierr var-7))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlacpy "L" m m a lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                 (ir) ((1 *)) work-%offset%)
           ldwrkr)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf m var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldwrkr var-6)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
           czero czero
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                 ((+ ir ldwrkr)) ((1 *))
                                 work-%offset%)
           ldwrkr)
        (declare (ignore var-0 var-1 var-2 var-5))
```

```
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zunglq m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (ir) ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "P" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (ir) ((1 *)) work-%offset%)
```

```
      ldwrkr
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (itaup) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" m m 0 0 s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (ir) ((1 *)) work-%offset%)
     ldwrkr cdum 1 cdum 1
     (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
    var-11 var-12 var-13))
  (setf m var-1)
  (setf m var-2)
  (setf ldwrkr var-8)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (zgemm "N" "N" m n m cone
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (ir) ((1 *)) work-%offset%)
     ldwrkr a lda czero vt ldvt)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf cone var-5))
  (when var-7 (setf ldwrkr var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf czero var-10))
  (when var-12 (setf ldvt var-12))))
(t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgelqf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "U" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zunglq m n m vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
```

```
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd m m a lda s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (zunmbr "P" "L" "C" m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" m n 0 0 s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     vt ldvt cdum 1 cdum 1
     (f2cl-lib:array-slice rwork-%data% double-float (irwork)
```

```
                                ((1 *)) rwork-%offset%)
          info)
        (declare
         (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
          var-11 var-12 var-13))
        (setf m var-1)
        (setf n var-2)
        (setf ldvt var-8)
        (setf info var-14)))))
(wntuo
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul 2 m m)
                         (f2cl-lib:int-mul 3 m)))
   (setf iu 1)
   (cond
    ((>= lwork
         (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 lda m)))
     (setf ldwrku lda)
     (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
     (setf ldwrkr lda))
    ((>= lwork
         (f2cl-lib:int-add wrkbl
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-add lda m) m)))
     (setf ldwrku lda)
     (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
     (setf ldwrkr m))
    (t (setf ldwrku m)
     (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
     (setf ldwrkr m)))
   (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
   (setf iwork (f2cl-lib:int-add itau m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (zgelqf m n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itau) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
     (declare (ignore var-2 var-4 var-5 var-6))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-7))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (zlacpy "L" m m a lda
```

```
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ iu ldwrku)) ((1 *))
                           work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zunglq m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
```

```
                              (itaup) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iu) ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (ir) ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "P" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iu) ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "Q" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (ir) ((1 *)) work-%offset%)
```

```
      ldwrkr
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (itauq) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" m m m 0 s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (ir) ((1 *)) work-%offset%)
     ldwrkr cdum 1
     (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf m var-1)
  (setf m var-2)
  (setf m var-3)
  (setf ldwrku var-8)
  (setf ldwrkr var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (zgemm "N" "N" m n m cone
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku a lda czero vt ldvt)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf cone var-5))
```

```
    (when var-7 (setf ldwrku var-7))
    (when var-9 (setf lda var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf ldvt var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "F" m m
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (ir) ((1 *)) work-%offset%)
       ldwrkr a lda)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf ldwrkr var-4))
    (when var-6 (setf lda var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgelqf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zunglq m n m vt ldvt
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf m var-0))
    (when var-1 (setf n var-1))
```

```
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd m m a lda s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (zunmbr "P" "L" "C" m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
```

```
        var-12))
      (when var-3 (setf m var-3))
      (when var-4 (setf n var-4))
      (when var-5 (setf m var-5))
      (when var-7 (setf lda var-7))
      (when var-10 (setf ldvt var-10))
      (when var-13 (setf ierr var-13)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9)
        (zungbr "Q" m m m a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (itauq) ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (iwork) ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
         ierr)
      (declare (ignore var-0 var-4 var-6 var-7 var-8))
      (when var-1 (setf m var-1))
      (when var-2 (setf m var-2))
      (when var-3 (setf m var-3))
      (when var-5 (setf lda var-5))
      (when var-9 (setf ierr var-9)))
    (setf irwork (f2cl-lib:int-add ie m))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14)
        (zbdsqr "U" m n m 0 s
         (f2cl-lib:array-slice rwork-%data% double-float (ie)
                               ((1 *)) rwork-%offset%)
         vt ldvt a lda cdum 1
         (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                               ((1 *)) rwork-%offset%)
         info)
      (declare
       (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
        var-13))
      (setf m var-1)
      (setf n var-2)
      (setf m var-3)
      (setf ldvt var-8)
      (setf lda var-10)
      (setf info var-14)))))
  (wntuas
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                           (f2cl-lib:int-mul 3 m)))
    (setf iu 1)
    (cond
```

```
 ((>= lwork
      (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda m)))
  (setf ldwrku lda))
 (t (setf ldwrku m)))
(setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
(setf iwork (f2cl-lib:int-add itau m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgelqf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ iu ldwrku)) ((1 *))
                           work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zunglq m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
```

```
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "P" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
```

```
                              (iu) ((1 *)) work-%offset%)
      ldwrku
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itaup) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" m m m 0 s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                              ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iu) ((1 *)) work-%offset%)
     ldwrku u ldu cdum 1
     (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                              ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf m var-1)
  (setf m var-2)
  (setf m var-3)
  (setf ldwrku var-8)
```

```
    (setf ldu var-10)
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "N" "N" m n m cone
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (iu) ((1 *)) work-%offset%)
       ldwrku a lda czero vt ldvt)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf ldwrku var-7))
    (when var-9 (setf lda var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf ldvt var-12))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgelqf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zunglq m n m vt ldvt
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
```

```
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" m m a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice u-%data% f2cl-lib:complex16 (1 2)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldu var-6)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd m m u ldu s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldu var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
      var-9 var-10 var-11 var-12 var-13)
     (zunmbr "P" "L" "C" m n m u ldu
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (itaup) ((1 *)) work-%offset%)
      vt ldvt
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
   (declare
    (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
     var-12))
   (when var-3 (setf m var-3))
   (when var-4 (setf n var-4))
   (when var-5 (setf m var-5))
   (when var-7 (setf ldu var-7))
   (when var-10 (setf ldvt var-10))
   (when var-13 (setf ierr var-13)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9)
     (zungbr "Q" m m m u ldu
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (itauq) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
   (declare (ignore var-0 var-4 var-6 var-7 var-8))
   (when var-1 (setf m var-1))
   (when var-2 (setf m var-2))
   (when var-3 (setf m var-3))
   (when var-5 (setf ldu var-5))
   (when var-9 (setf ierr var-9)))
 (setf irwork (f2cl-lib:int-add ie m))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12 var-13 var-14)
     (zbdsqr "U" m n m 0 s
      (f2cl-lib:array-slice rwork-%data% double-float (ie)
                            ((1 *)) rwork-%offset%)
      vt ldvt u ldu cdum 1
      (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                            ((1 *)) rwork-%offset%)
      info)
   (declare
    (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
     var-13))
   (setf m var-1)
   (setf n var-2)
```

```
        (setf m var-3)
        (setf ldvt var-8)
        (setf ldu var-10)
        (setf info var-14)))))))
(wntva
 (cond
  (wntun
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add n m))
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-mul 3 m)))))
     (setf ir 1)
     (cond
      ((>= lwork
           (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda m)))
       (setf ldwrkr lda))
      (t (setf ldwrkr m)))
     (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
     (setf iwork (f2cl-lib:int-add itau m))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (zgelqf m n a lda
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                (itau) ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                (iwork) ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
          ierr)
       (declare (ignore var-2 var-4 var-5 var-6))
       (setf m var-0)
       (setf n var-1)
       (setf lda var-3)
       (setf ierr var-7))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (zlacpy "U" m n a lda vt ldvt)
       (declare (ignore var-0 var-3 var-5))
       (when var-1 (setf m var-1))
       (when var-2 (setf n var-2))
       (when var-4 (setf lda var-4))
       (when var-6 (setf ldvt var-6)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (zlacpy "L" m m a lda
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                (ir) ((1 *)) work-%offset%)
```

```
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ ir ldwrkr)) ((1 *))
                           work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zunglq n n m vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (ir) ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
```

```
                          (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "P" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                          (ir) ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                          (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                          (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" m m 0 0 s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                          ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                          (ir) ((1 *)) work-%offset%)
     ldwrkr cdum 1 cdum 1
     (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                          ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
    var-11 var-12 var-13))
  (setf m var-1)
  (setf m var-2)
  (setf ldwrkr var-8)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
      var-9 var-10 var-11 var-12)
     (zgemm "N" "N" m n m cone
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (ir) ((1 *)) work-%offset%)
      ldwrkr vt ldvt czero a lda)
   (declare (ignore var-0 var-1 var-6 var-8 var-11))
   (when var-2 (setf m var-2))
   (when var-3 (setf n var-3))
   (when var-4 (setf m var-4))
   (when var-5 (setf cone var-5))
   (when var-7 (setf ldwrkr var-7))
   (when var-9 (setf ldvt var-9))
   (when var-10 (setf czero var-10))
   (when var-12 (setf lda var-12)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (zlacpy "F" m n a lda vt ldvt)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf m var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldvt var-6))))
(t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (zgelqf m n a lda
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (itau) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
   (declare (ignore var-2 var-4 var-5 var-6))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-7))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (zlacpy "U" m n a lda vt ldvt)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf m var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldvt var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (zunglq n n m vt ldvt
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (itau) ((1 *)) work-%offset%)
```

```
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd m m a lda s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (zunmbr "P" "L" "C" m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
```

```
          vt ldvt
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                (iwork) ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
          ierr)
        (declare
         (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
          var-12))
        (when var-3 (setf m var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf m var-5))
        (when var-7 (setf lda var-7))
        (when var-10 (setf ldvt var-10))
        (when var-13 (setf ierr var-13)))
      (setf irwork (f2cl-lib:int-add ie m))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12 var-13 var-14)
          (zbdsqr "U" m n 0 0 s
           (f2cl-lib:array-slice rwork-%data% double-float (ie)
                                 ((1 *)) rwork-%offset%)
           vt ldvt cdum 1 cdum 1
           (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                                 ((1 *)) rwork-%offset%)
           info)
        (declare
         (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
          var-11 var-12 var-13))
        (setf m var-1)
        (setf n var-2)
        (setf ldvt var-8)
        (setf info var-14)))))
  (wntuo
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul 2 m m)
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add n m))
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-mul 3 m)))))
      (setf iu 1)
      (cond
       ((>= lwork
            (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 lda m)))
        (setf ldwrku lda)
        (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
        (setf ldwrkr lda))
       ((>= lwork
            (f2cl-lib:int-add wrkbl
```

```
                        (f2cl-lib:int-mul
                         (f2cl-lib:int-add lda m) m)))
  (setf ldwrku lda)
  (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
  (setf ldwrkr m))
 (t (setf ldwrku m)
  (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
  (setf ldwrkr m)))
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
(setf iwork (f2cl-lib:int-add itau m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgelqf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "U" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zunglq n n m vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" m m a lda
```

```
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (iu) ((1 *)) work-%offset%)
      ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ iu ldwrku)) ((1 *))
                           work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrku var-6)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
```

```
                              (iu) ((1 *)) work-%offset%)
      ldwrku
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (ir) ((1 *)) work-%offset%)
      ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "P" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iu) ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "Q" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (ir) ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" m m m 0 s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (ir) ((1 *)) work-%offset%)
     ldwrkr cdum 1
     (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf m var-1)
  (setf m var-2)
  (setf m var-3)
  (setf ldwrku var-8)
  (setf ldwrkr var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (zgemm "N" "N" m n m cone
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku vt ldvt czero a lda)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf cone var-5))
  (when var-7 (setf ldwrku var-7))
  (when var-9 (setf ldvt var-9))
  (when var-10 (setf czero var-10))
  (when var-12 (setf lda var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "F" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "F" m m
```

```
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            (ir) ((1 *)) work-%offset%)
      ldwrkr a lda)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf ldwrkr var-4))
    (when var-6 (setf lda var-6))))
  (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (zgelqf m n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itau) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (zlacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (zunglq n n m vt ldvt
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (itau) ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                              (iwork) ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf n var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf ldvt var-4))
    (when var-8 (setf ierr var-8)))
   (setf ie 1) (setf itauq itau)
   (setf itaup (f2cl-lib:int-add itauq m))
   (setf iwork (f2cl-lib:int-add itaup m))
   (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd m m a lda s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (zunmbr "P" "L" "C" m n m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
```

```
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9)
        (zungbr "Q" m m m a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (itauq) ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (iwork) ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
         ierr)
      (declare (ignore var-0 var-4 var-6 var-7 var-8))
      (when var-1 (setf m var-1))
      (when var-2 (setf m var-2))
      (when var-3 (setf m var-3))
      (when var-5 (setf lda var-5))
      (when var-9 (setf ierr var-9)))
    (setf irwork (f2cl-lib:int-add ie m))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14)
        (zbdsqr "U" m n m 0 s
         (f2cl-lib:array-slice rwork-%data% double-float (ie)
                               ((1 *)) rwork-%offset%)
         vt ldvt a lda cdum 1
         (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                               ((1 *)) rwork-%offset%)
         info)
      (declare
       (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
        var-13))
      (setf m var-1)
      (setf n var-2)
      (setf m var-3)
      (setf ldvt var-8)
      (setf lda var-10)
      (setf info var-14)))))
(wntuas
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add n m))
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-mul 3 m)))))
    (setf iu 1)
    (cond
     ((>= lwork
          (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda m)))
      (setf ldwrku lda))
```

```
 (t (setf ldwrku m)))
(setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
(setf iwork (f2cl-lib:int-add itau m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgelqf m n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "U" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zunglq n n m vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
```

```
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ iu ldwrku)) ((1 *))
                           work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldwrku var-6)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldu var-6)))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "P" m m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itaup) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (zbdsqr "U" m m m 0 s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iu) ((1 *)) work-%offset%)
     ldwrku u ldu cdum 1
     (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                           ((1 *)) rwork-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
```

```
      var-13))
    (setf m var-1)
    (setf m var-2)
    (setf m var-3)
    (setf ldwrku var-8)
    (setf ldu var-10)
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "N" "N" m n m cone
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (iu) ((1 *)) work-%offset%)
       ldwrku vt ldvt czero a lda)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf cone var-5))
    (when var-7 (setf ldwrku var-7))
    (when var-9 (setf ldvt var-9))
    (when var-10 (setf czero var-10))
    (when var-12 (setf lda var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "F" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgelqf m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (itau) ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             (iwork) ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
```

```
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zunglq n n m vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itau) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" m m a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     czero czero
     (f2cl-lib:array-slice u-%data% f2cl-lib:complex16 (1 2)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldu var-6)))
(setf ie 1) (setf itauq itau)
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgebrd m m u ldu s
     (f2cl-lib:array-slice rwork-%data% double-float (ie)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
```

```
                                (itaup) ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                (iwork) ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldu var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (zunmbr "P" "L" "C" m n m u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                (itaup) ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldu var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zungbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                (itauq) ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                (iwork) ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf irwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
                var-9 var-10 var-11 var-12 var-13 var-14)
               (zbdsqr "U" m n m 0 s
                (f2cl-lib:array-slice rwork-%data% double-float (ie)
                                      ((1 *)) rwork-%offset%)
                vt ldvt u ldu cdum 1
                (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                                      ((1 *)) rwork-%offset%)
                info)
              (declare
               (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
                var-13))
              (setf m var-1)
              (setf n var-2)
              (setf m var-3)
              (setf ldvt var-8)
              (setf ldu var-10)
              (setf info var-14)))))))))
(t (setf ie 1) (setf itauq 1)
 (setf itaup (f2cl-lib:int-add itauq m))
 (setf iwork (f2cl-lib:int-add itaup m))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (zgebrd m n a lda s
      (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                            rwork-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-10))
 (cond
  (wntuas
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (zlacpy "L" m m a lda u ldu)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf m var-1))
     (when var-2 (setf m var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf ldu var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (zungbr "Q" m m n u ldu
```

```
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf ldu var-5))
    (when var-9 (setf ierr var-9)))))
(cond
 (wntvas
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (if wntva
      (setf nrvt n))
  (if wntvs
      (setf nrvt m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zungbr "P" nrvt n m vt ldvt
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf nrvt var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf ldvt var-5))
    (when var-9 (setf ierr var-9)))))
(cond
 (wntuo
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zungbr "Q" m m n a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
```

```
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))))
(cond
 (wntvo
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zungbr "P" m n m a lda
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))))
(setf irwork (f2cl-lib:int-add ie m))
(if (or wntuas wntuo)
    (setf nru m))
(if wntun
    (setf nru 0))
(if (or wntvas wntvo)
    (setf ncvt n))
(if wntvn
    (setf ncvt 0))
(cond
 ((and (not wntuo) (not wntvo))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (zbdsqr "L" m ncvt nru 0 s
       (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                             rwork-%offset%)
       vt ldvt u ldu cdum 1
       (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                             ((1 *)) rwork-%offset%)
       info)
    (declare
     (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
      var-13))
    (setf m var-1)
    (setf ncvt var-2)
    (setf nru var-3)
    (setf ldvt var-8)
    (setf ldu var-10)
    (setf info var-14)))
```

```
        ((and (not wntuo) wntvo)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
              var-10 var-11 var-12 var-13 var-14)
             (zbdsqr "L" m ncvt nru 0 s
              (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                                    rwork-%offset%)
              a lda u ldu cdum 1
              (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                                    ((1 *)) rwork-%offset%)
              info)
           (declare
            (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
             var-13))
           (setf m var-1)
           (setf ncvt var-2)
           (setf nru var-3)
           (setf lda var-8)
           (setf ldu var-10)
           (setf info var-14)))
        (t
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
              var-10 var-11 var-12 var-13 var-14)
             (zbdsqr "L" m ncvt nru 0 s
              (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                                    rwork-%offset%)
              vt ldvt a lda cdum 1
              (f2cl-lib:array-slice rwork-%data% double-float (irwork)
                                    ((1 *)) rwork-%offset%)
              info)
           (declare
            (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
             var-13))
           (setf m var-1)
           (setf ncvt var-2)
           (setf nru var-3)
           (setf ldvt var-8)
           (setf lda var-10)
           (setf info var-14))))))))
    (cond
     ((= iscl 1)
      (if (> anrm bignum)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (dlascl "G" 0 0 bignum anrm minmn 1 s minmn ierr)
           (declare (ignore var-0 var-1 var-2 var-6 var-7))
           (when var-3 (setf bignum var-3))
           (when var-4 (setf anrm var-4))
           (when var-5 (setf minmn var-5))
```

```
                (when var-8 (setf minmn var-8))
                (when var-9 (setf ierr var-9))))
          (if (and (/= info 0) (> anrm bignum))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (dlascl "G" 0 0 bignum anrm (f2cl-lib:int-sub minmn 1) 1
                   (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                                         rwork-%offset%)
                   minmn ierr)
                (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
                (when var-3 (setf bignum var-3))
                (when var-4 (setf anrm var-4))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf ierr var-9))))
          (if (< anrm smlnum)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (dlascl "G" 0 0 smlnum anrm minmn 1 s minmn ierr)
                (declare (ignore var-0 var-1 var-2 var-6 var-7))
                (when var-3 (setf smlnum var-3))
                (when var-4 (setf anrm var-4))
                (when var-5 (setf minmn var-5))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf ierr var-9))))
          (if (and (/= info 0) (< anrm smlnum))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (dlascl "G" 0 0 smlnum anrm (f2cl-lib:int-sub minmn 1) 1
                   (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                                         rwork-%offset%)
                   minmn ierr)
                (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
                (when var-3 (setf smlnum var-3))
                (when var-4 (setf anrm var-4))
                (when var-8 (setf minmn var-8))
                (when var-9 (setf ierr var-9))))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce maxwrk 'f2cl-lib:complex16))
        (go end_label)
       end_label
        (return
         (values jobu jobvt m n nil lda nil nil ldu nil ldvt nil nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgesvd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
```

```
'((string) (string)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::jobu
  fortran-to-lisp::jobvt
  fortran-to-lisp::m
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil
  fortran-to-lisp::ldu nil
  fortran-to-lisp::ldvt nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zbdsqr
  fortran-to-lisp::xerbla
  fortran-to-lisp::zgelqf
  fortran-to-lisp::zgebrd
  fortran-to-lisp::zgeqrf
  fortran-to-lisp::ilaenv))))
```

———-

31.4 zgesvdx

31.4.1 Axiom unit tests

— zgesvdx.input —

```
)set break resume
)sys rm -f zgesvdx.output
```

```
)spool zgesvdx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

31.4.2 Axiom help page

— **zgesvdx.help** —

```
==================================================================
zgesvdx examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgesvdx (JOBU, JOBVT, RANGE, M, N, A, LDA, VL, VU, IL, IU,
           NS, S, U, LDU, VT, LDVT, WORK, LWORK, RWORK, IWORK, INFO)
            ZGESVDX computes the singular value decomposition (SVD) for GE
           matrices

Function/Subroutine Documentation
   subroutine zgesvdx (character JOBU, character JOBVT, character RANGE,
       integer M, integer N, complex*16, dimension( lda, * ) A, integer LDA,
       double precision VL, double precision VU, integer IL, integer IU,
       integer NS, double precision, dimension( * ) S, complex*16, dimension(
       ldu, * ) U, integer LDU, complex*16, dimension( ldvt, * ) VT, integer
       LDVT, complex*16, dimension( * ) WORK, integer LWORK, double precision,
       dimension( * ) RWORK, integer, dimension( * ) IWORK, integer INFO)
        ZGESVDX computes the singular value decomposition (SVD) for GE
       matrices

Parameters:

 JOBU
   JOBU is CHARACTER*1
   Specifies options for computing all or part of the matrix U:
   = 'V':  the first min(m,n) columns of U (the left singular
           vectors) or as specified by RANGE are returned in the array U;
   = 'N':  no columns of U (no left singular vectors) are computed.
```

```
JOBVT
  JOBVT is CHARACTER*1
  Specifies options for computing all or part of the matrix
  V**T:
  = 'V':  the first min(m,n) rows of V**T (the right singular
          vectors) or as specified by RANGE are returned in the array VT;
  = 'N':  no rows of V**T (no right singular vectors) are computed.

RANGE
  RANGE is CHARACTER*1
  = 'A': all singular values will be found.
  = 'V': all singular values in the half-open interval (VL,VU] will be found.
  = 'I': the IL-th through IU-th singular values will be found.

M
  M is INTEGER
  The number of rows of the input matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the input matrix A.  N >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit, the contents of A are destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

VL
  VL is DOUBLE PRECISION
  VL >=0.

VU
  VU is DOUBLE PRECISION
  If RANGE='V', the lower and upper bounds of the interval to
  be searched for singular values. VU > VL.
  Not referenced if RANGE = 'A' or 'I'.

IL
  IL is INTEGER

IU
  IU is INTEGER
  If RANGE='I', the indices (in ascending order) of the
  smallest and largest singular values to be returned.
  1 <= IL <= IU <= min(M,N), if min(M,N) > 0.
```

```
  Not referenced if RANGE = 'A' or 'V'.

NS
  NS is INTEGER
  The total number of singular values found,
  0 <= NS <= min(M,N).
  If RANGE = 'A', NS = min(M,N); if RANGE = 'I', NS = IU-IL+1.

S
  S is DOUBLE PRECISION array, dimension (min(M,N))
  The singular values of A, sorted so that S(i) >= S(i+1).

U
  U is COMPLEX*16 array, dimension (LDU,UCOL)
  If JOBU = 'V', U contains columns of U (the left singular
  vectors, stored columnwise) as specified by RANGE; if
  JOBU = 'N', U is not referenced.
  Note: The user must ensure that UCOL >= NS; if RANGE = 'V',
  the exact value of NS is not known ILQFin advance and an upper
  bound must be used.

LDU
  LDU is INTEGER
  The leading dimension of the array U.  LDU >= 1; if
  JOBU = 'V', LDU >= M.

VT
  VT is COMPLEX*16 array, dimension (LDVT,N)
  If JOBVT = 'V', VT contains the rows of V**T (the right singular
  vectors, stored rowwise) as specified by RANGE; if JOBVT = 'N',
  VT is not referenced.
  Note: The user must ensure that LDVT >= NS; if RANGE = 'V',
  the exact value of NS is not known in advance and an upper
  bound must be used.

LDVT
  LDVT is INTEGER
  The leading dimension of the array VT.  LDVT >= 1; if
  JOBVT = 'V', LDVT >= NS (see above).

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK;

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  LWORK >= MAX(1,MIN(M,N)*(MIN(M,N)+4)) for the paths (see
  comments inside the code):
  - PATH 1  (M much larger than N)
```

```
  - PATH 1t (N much larger than M)
  LWORK >= MAX(1,MIN(M,N)*2+MAX(M,N)) for the other paths.
  For good performance, LWORK should generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (MAX(1,LRWORK))
  LRWORK >= MIN(M,N)*(MIN(M,N)*2+15*MIN(M,N)).

IWORK
  IWORK is INTEGER array, dimension (12*MIN(M,N))
  If INFO = 0, the first NS elements of IWORK are zero. If INFO > 0,
  then IWORK contains the indices of the eigenvectors that failed
  to converge in DBDSVDX/DSTEVX.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, then i eigenvectors failed to converge
        in DBDSVDX/DSTEVX.
        if INFO = N*2 + 1, an internal error occurred in DBDSVDX
```

———-

31.4.3 fortran code

— **zgesvdx.f** —

```
*  ===================================================================
      SUBROUTINE ZGESVDX( JOBU, JOBVT, RANGE, M, N, A, LDA, VL, VU,
     $                    IL, IU, NS, S, U, LDU, VT, LDVT, WORK,
     $                    LWORK, RWORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBU, JOBVT, RANGE
      INTEGER            IL, INFO, IU, LDA, LDU, LDVT, LWORK, M, N, NS
      DOUBLE PRECISION   VL, VU
```

```
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   S( * ), RWORK( * )
      COMPLEX*16         A( LDA, * ), U( LDU, * ), VT( LDVT, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0D0, 0.0D0 ),
     $                   CONE = ( 1.0D0, 0.0D0 ) )
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      CHARACTER          JOBZ, RNGTGK
      LOGICAL            ALLS, INDS, LQUERY, VALS, WANTU, WANTVT
      INTEGER            I, ID, IE, IERR, ILQF, ILTGK, IQRF, ISCL,
     $                   ITAU, ITAUP, ITAUQ, ITEMP, ITGKZ, IUTGK,
     $                   J, K, MAXWRK, MINMN, MINWRK, MNTHR
      DOUBLE PRECISION   ABSTOL, ANRM, BIGNUM, EPS, SMLNUM
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZGEBRD, ZGELQF, ZGEQRF, ZLASCL, ZLASET,
     $                   DLASCL, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, ZLANGE
      EXTERNAL           LSAME, ILAENV, DLAMCH, ZLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      NS = 0
      INFO = 0
      ABSTOL = 2*DLAMCH('S')
      LQUERY = ( LWORK.EQ.-1 )
      MINMN = MIN( M, N )
```

```
      WANTU = LSAME( JOBU, 'V' )
      WANTVT = LSAME( JOBVT, 'V' )
      IF( WANTU .OR. WANTVT ) THEN
         JOBZ = 'V'
      ELSE
         JOBZ = 'N'
      END IF
      ALLS = LSAME( RANGE, 'A' )
      VALS = LSAME( RANGE, 'V' )
      INDS = LSAME( RANGE, 'I' )
*
      INFO = 0
      IF( .NOT.LSAME( JOBU, 'V' ) .AND.
     $    .NOT.LSAME( JOBU, 'N' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.LSAME( JOBVT, 'V' ) .AND.
     $         .NOT.LSAME( JOBVT, 'N' ) ) THEN
         INFO = -2
      ELSE IF( .NOT.( ALLS .OR. VALS .OR. INDS ) ) THEN
         INFO = -3
      ELSE IF( M.LT.0 ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( M.GT.LDA ) THEN
         INFO = -7
      ELSE IF( MINMN.GT.0 ) THEN
         IF( VALS ) THEN
            IF( VL.LT.ZERO ) THEN
               INFO = -8
            ELSE IF( VU.LE.VL ) THEN
               INFO = -9
            END IF
         ELSE IF( INDS ) THEN
            IF( IL.LT.1 .OR. IL.GT.MAX( 1, MINMN ) ) THEN
               INFO = -10
            ELSE IF( IU.LT.MIN( MINMN, IL ) .OR. IU.GT.MINMN ) THEN
               INFO = -11
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
            IF( WANTU .AND. LDU.LT.M ) THEN
               INFO = -15
            ELSE IF( WANTVT .AND. LDVT.LT.MINMN ) THEN
               INFO = -16
            END IF
         END IF
      END IF
*
```

```
*     Compute workspace
*     (Note: Comments in the code beginning "Workspace:" describe the
*     minimal amount of workspace needed at that point in the code,
*     as well as the preferred amount for good performance.
*     NB refers to the optimal block size for the immediately
*     following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = 1
         MAXWRK = 1
         IF( MINMN.GT.0 ) THEN
            IF( M.GE.N ) THEN
               MNTHR = ILAENV( 6, 'ZGESVD', JOBU // JOBVT, M, N, 0, 0 )
               IF( M.GE.MNTHR ) THEN
*
*                Path 1 (M much larger than N)
*
                 MAXWRK = N + N*
     $                   ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
                 MAXWRK = MAX( MAXWRK, N*N + N + 2*N*
     $                   ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                 MINWRK = N*(N+4)
               ELSE
*
*                Path 2 (M at least N, but not much larger)
*
                 MAXWRK = 2*N + ( M+N )*
     $                   ILAENV( 1, 'ZGEBRD', ' ', M, N, -1, -1 )
                 MINWRK = 2*N + M
               END IF
            ELSE
               MNTHR = ILAENV( 6, 'ZGESVD', JOBU // JOBVT, M, N, 0, 0 )
               IF( N.GE.MNTHR ) THEN
*
*                Path 1t (N much larger than M)
*
                 MAXWRK = M + M*
     $                   ILAENV( 1, 'ZGELQF', ' ', M, N, -1, -1 )
                 MAXWRK = MAX( MAXWRK, M*M + M + 2*M*
     $                   ILAENV( 1, 'ZGEBRD', ' ', M, M, -1, -1 ) )
                 MINWRK = M*(M+4)
               ELSE
*
*                Path 2t (N greater than M, but not much larger)
*
                 MAXWRK = M*(M*2+19) + ( M+N )*
     $                   ILAENV( 1, 'ZGEBRD', ' ', M, N, -1, -1 )
                 MINWRK = 2*M + N
               END IF
            END IF
```

```
         END IF
         MAXWRK = MAX( MAXWRK, MINWRK )
         WORK( 1 ) = DCMPLX( DBLE( MAXWRK ), ZERO )
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -19
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGESVDX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         RETURN
      END IF
*
*     Set singular values indices accord to RANGE='A'.
*
      ALLS = LSAME( RANGE, 'A' )
      INDS = LSAME( RANGE, 'I' )
      IF( ALLS ) THEN
         RNGTGK = 'I'
         ILTGK = 1
         IUTGK = MIN( M, N )
      ELSE IF( INDS ) THEN
         RNGTGK = 'I'
         ILTGK = IL
         IUTGK = IU
      ELSE
         RNGTGK = 'V'
         ILTGK = 0
         IUTGK = 0
      END IF
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = SQRT( DLAMCH( 'S' ) ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = ZLANGE( 'M', M, N, A, LDA, DUM )
      ISCL = 0
```

```
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ISCL = 1
         CALL ZLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, INFO )
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ISCL = 1
         CALL ZLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, INFO )
      END IF
*
      IF( M.GE.N ) THEN
*
*        A has at least as many rows as columns. If A has sufficiently
*        more rows than columns, first reduce A using the QR
*        decomposition.
*
         IF( M.GE.MNTHR ) THEN
*
*           Path 1 (M much larger than N):
*           A = Q * R = Q * ( QB * B * PB**T )
*                     = Q * ( QB * ( UB * S * VB**T ) * PB**T )
*           U = Q * QB * UB; V**T = VB**T * PB**T
*
*           Compute A=Q*R
*           (Workspace: need 2*N, prefer N+N*NB)
*
            ITAU = 1
            ITEMP = ITAU + N
            CALL ZGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( ITEMP ),
     $                   LWORK-ITEMP+1, INFO )
*
*           Copy R into WORK and bidiagonalize it:
*           (Workspace: need N*N+3*N, prefer N*N+N+2*N*NB)
*
            IQRF = ITEMP
            ITAUQ = ITEMP + N*N
            ITAUP = ITAUQ + N
            ITEMP = ITAUP + N
            ID = 1
            IE = ID + N
            ITGKZ = IE + N
            CALL ZLACPY( 'U', N, N, A, LDA, WORK( IQRF ), N )
            CALL ZLASET( 'L', N-1, N-1, CZERO, CZERO,
     $                   WORK( IQRF+1 ), N )
            CALL ZGEBRD( N, N, WORK( IQRF ), N, RWORK( ID ),
     $                   RWORK( IE ), WORK( ITAUQ ), WORK( ITAUP ),
     $                   WORK( ITEMP ), LWORK-ITEMP+1, INFO )
            ITEMP = ITGKZ + N*(N*2+1)
*
*           Solve eigenvalue problem TGK*Z=Z*S.
*           (Workspace: need 2*N*N+14*N)
*
```

```
            CALL DBDSVDX( 'U', JOBZ, RNGTGK, N, RWORK( ID ),
     $                    RWORK( IE ), VL, VU, ILTGK, IUTGK, NS, S,
     $                    RWORK( ITGKZ ), N*2, RWORK( ITEMP ),
     $                    IWORK, INFO)
*
*           If needed, compute left singular vectors.
*
            IF( WANTU ) THEN
               K = ITGKZ
               DO I = 1, NS
                  DO J = 1, N
                     U( J, I ) = DCMPLX( RWORK( K ), ZERO )
                     K = K + 1
                  END DO
                  K = K + N
               END DO
               CALL ZLASET( 'A', M-N, N, CZERO, CZERO, U( N+1,1 ), LDU )
*
*              Call ZUNMBR to compute QB*UB.
*              (Workspace in WORK( ITEMP ): need N, prefer N*NB)
*
               CALL ZUNMBR( 'Q', 'L', 'N', N, NS, N, WORK( IQRF ), N,
     $                      WORK( ITAUQ ), U, LDU, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
*
*              Call ZUNMQR to compute Q*(QB*UB).
*              (Workspace in WORK( ITEMP ): need N, prefer N*NB)
*
               CALL ZUNMQR( 'L', 'N', M, NS, N, A, LDA,
     $                      WORK( ITAU ), U, LDU, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
            END IF
*
*           If needed, compute right singular vectors.
*
            IF( WANTVT) THEN
               K = ITGKZ + N
               DO I = 1, NS
                  DO J = 1, N
                     VT( I, J ) = DCMPLX( RWORK( K ), ZERO )
                     K = K + 1
                  END DO
                  K = K + N
               END DO
*
*              Call ZUNMBR to compute VB**T * PB**T
*              (Workspace in WORK( ITEMP ): need N, prefer N*NB)
*
               CALL ZUNMBR( 'P', 'R', 'C', NS, N, N, WORK( IQRF ), N,
     $                      WORK( ITAUP ), VT, LDVT, WORK( ITEMP ),
```

```
     $                     LWORK-ITEMP+1, INFO )
            END IF
         ELSE
*
*           Path 2 (M at least N, but not much larger)
*           Reduce A to bidiagonal form without QR decomposition
*           A = QB * B * PB**T = QB * ( UB * S * VB**T ) * PB**T
*           U = QB * UB; V**T = VB**T * PB**T
*
*           Bidiagonalize A
*           (Workspace: need 2*N+M, prefer 2*N+(M+N)*NB)
*
            ITAUQ = 1
            ITAUP = ITAUQ + N
            ITEMP = ITAUP + N
            ID = 1
            IE = ID + N
            ITGKZ = IE + N
            CALL ZGEBRD( M, N, A, LDA, RWORK( ID ), RWORK( IE ),
     $                   WORK( ITAUQ ), WORK( ITAUP ), WORK( ITEMP ),
     $                   LWORK-ITEMP+1, INFO )
            ITEMP = ITGKZ + N*(N*2+1)
*
*           Solve eigenvalue problem TGK*Z=Z*S.
*           (Workspace: need 2*N*N+14*N)
*
            CALL DBDSVDX( 'U', JOBZ, RNGTGK, N, RWORK( ID ),
     $                    RWORK( IE ), VL, VU, ILTGK, IUTGK, NS, S,
     $                    RWORK( ITGKZ ), N*2, RWORK( ITEMP ),
     $                    IWORK, INFO)
*
*           If needed, compute left singular vectors.
*
            IF( WANTU ) THEN
               K = ITGKZ
               DO I = 1, NS
                  DO J = 1, N
                     U( J, I ) = DCMPLX( RWORK( K ), ZERO )
                     K = K + 1
                  END DO
                  K = K + N
               END DO
               CALL ZLASET( 'A', M-N, N, CZERO, CZERO, U( N+1,1 ), LDU )
*
*              Call ZUNMBR to compute QB*UB.
*              (Workspace in WORK( ITEMP ): need N, prefer N*NB)
*
               CALL ZUNMBR( 'Q', 'L', 'N', M, NS, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, IERR )
```

```
            END IF
*
*           If needed, compute right singular vectors.
*
            IF( WANTVT) THEN
               K = ITGKZ + N
               DO I = 1, NS
                  DO J = 1, N
                     VT( I, J ) = DCMPLX( RWORK( K ), ZERO )
                     K = K + 1
                  END DO
                  K = K + N
               END DO
*
*              Call ZUNMBR to compute VB**T * PB**T
*              (Workspace in WORK( ITEMP ): need N, prefer N*NB)
*
               CALL ZUNMBR( 'P', 'R', 'C', NS, N, N, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, IERR )
            END IF
         END IF
      ELSE
*
*        A has more columns than rows. If A has sufficiently more
*        columns than rows, first reduce A using the LQ decomposition.
*
         IF( N.GE.MNTHR ) THEN
*
*           Path 1t (N much larger than M):
*           A = L * Q = ( QB * B * PB**T ) * Q
*                     = ( QB * ( UB * S * VB**T ) * PB**T ) * Q
*           U = QB * UB ; V**T = VB**T * PB**T * Q
*
*           Compute A=L*Q
*           (Workspace: need 2*M, prefer M+M*NB)
*
            ITAU = 1
            ITEMP = ITAU + M
            CALL ZGELQF( M, N, A, LDA, WORK( ITAU ), WORK( ITEMP ),
     $                   LWORK-ITEMP+1, INFO )

*           Copy L into WORK and bidiagonalize it:
*           (Workspace in WORK( ITEMP ): need M*M+3*M, prefer M*M+M+2*M*NB)
*
            ILQF = ITEMP
            ITAUQ = ILQF + M*M
            ITAUP = ITAUQ + M
            ITEMP = ITAUP + M
            ID = 1
```

```
            IE = ID + M
            ITGKZ = IE + M
            CALL ZLACPY( 'L', M, M, A, LDA, WORK( ILQF ), M )
            CALL ZLASET( 'U', M-1, M-1, CZERO, CZERO,
     $                   WORK( ILQF+M ), M )
            CALL ZGEBRD( M, M, WORK( ILQF ), M, RWORK( ID ),
     $                   RWORK( IE ), WORK( ITAUQ ), WORK( ITAUP ),
     $                   WORK( ITEMP ), LWORK-ITEMP+1, INFO )
            ITEMP = ITGKZ + M*(M*2+1)
*
*           Solve eigenvalue problem TGK*Z=Z*S.
*           (Workspace: need 2*M*M+14*M)
*
            CALL DBDSVDX( 'U', JOBZ, RNGTGK, M, RWORK( ID ),
     $                    RWORK( IE ), VL, VU, ILTGK, IUTGK, NS, S,
     $                    RWORK( ITGKZ ), M*2, RWORK( ITEMP ),
     $                    IWORK, INFO)
*
*           If needed, compute left singular vectors.
*
            IF( WANTU ) THEN
               K = ITGKZ
               DO I = 1, NS
                  DO J = 1, M
                     U( J, I ) = DCMPLX( RWORK( K ), ZERO )
                     K = K + 1
                  END DO
                  K = K + M
               END DO
*
*              Call ZUNMBR to compute QB*UB.
*              (Workspace in WORK( ITEMP ): need M, prefer M*NB)
*
               CALL ZUNMBR( 'Q', 'L', 'N', M, NS, M, WORK( ILQF ), M,
     $                      WORK( ITAUQ ), U, LDU, WORK( ITEMP ),
     $                      LWORK-ITEMP+1, INFO )
            END IF
*
*           If needed, compute right singular vectors.
*
            IF( WANTVT) THEN
               K = ITGKZ + M
               DO I = 1, NS
                  DO J = 1, M
                     VT( I, J ) = DCMPLX( RWORK( K ), ZERO )
                     K = K + 1
                  END DO
                  K = K + M
               END DO
               CALL ZLASET( 'A', M, N-M, CZERO, CZERO,
```

```
     $                     VT( 1,M+1 ), LDVT )
*
*             Call ZUNMBR to compute (VB**T)*(PB**T)
*             (Workspace in WORK( ITEMP ): need M, prefer M*NB)
*
              CALL ZUNMBR( 'P', 'R', 'C', NS, M, M, WORK( ILQF ), M,
     $                     WORK( ITAUP ), VT, LDVT, WORK( ITEMP ),
     $                     LWORK-ITEMP+1, INFO )
*
*             Call ZUNMLQ to compute ((VB**T)*(PB**T))*Q.
*             (Workspace in WORK( ITEMP ): need M, prefer M*NB)
*
              CALL ZUNMLQ( 'R', 'N', NS, N, M, A, LDA,
     $                     WORK( ITAU ), VT, LDVT, WORK( ITEMP ),
     $                     LWORK-ITEMP+1, INFO )
           END IF
        ELSE
*
*          Path 2t (N greater than M, but not much larger)
*          Reduce to bidiagonal form without LQ decomposition
*          A = QB * B * PB**T = QB * ( UB * S * VB**T ) * PB**T
*          U = QB * UB; V**T = VB**T * PB**T
*
*          Bidiagonalize A
*          (Workspace: need 2*M+N, prefer 2*M+(M+N)*NB)
*
           ITAUQ = 1
           ITAUP = ITAUQ + M
           ITEMP = ITAUP + M
           ID = 1
           IE = ID + M
           ITGKZ = IE + M
           CALL ZGEBRD( M, N, A, LDA, RWORK( ID ), RWORK( IE ),
     $                  WORK( ITAUQ ), WORK( ITAUP ), WORK( ITEMP ),
     $                  LWORK-ITEMP+1, INFO )
           ITEMP = ITGKZ + M*(M*2+1)
*
*          Solve eigenvalue problem TGK*Z=Z*S.
*          (Workspace: need 2*M*M+14*M)
*
           CALL DBDSVDX( 'L', JOBZ, RNGTGK, M, RWORK( ID ),
     $                   RWORK( IE ), VL, VU, ILTGK, IUTGK, NS, S,
     $                   RWORK( ITGKZ ), M*2, RWORK( ITEMP ),
     $                   IWORK, INFO)
*
*          If needed, compute left singular vectors.
*
           IF( WANTU ) THEN
              K = ITGKZ
              DO I = 1, NS
```

```
                 DO J = 1, M
                    U( J, I ) = DCMPLX( RWORK( K ), ZERO )
                    K = K + 1
                 END DO
                 K = K + M
              END DO
*
*             Call ZUNMBR to compute QB*UB.
*             (Workspace in WORK( ITEMP ): need M, prefer M*NB)
*
              CALL ZUNMBR( 'Q', 'L', 'N', M, NS, N, A, LDA,
     $                     WORK( ITAUQ ), U, LDU, WORK( ITEMP ),
     $                     LWORK-ITEMP+1, INFO )
           END IF
*
*          If needed, compute right singular vectors.
*
           IF( WANTVT) THEN
              K = ITGKZ + M
              DO I = 1, NS
                 DO J = 1, M
                    VT( I, J ) = DCMPLX( RWORK( K ), ZERO )
                    K = K + 1
                 END DO
                 K = K + M
              END DO
              CALL ZLASET( 'A', M, N-M, CZERO, CZERO,
     $                     VT( 1,M+1 ), LDVT )
*
*             Call ZUNMBR to compute VB**T * PB**T
*             (Workspace in WORK( ITEMP ): need M, prefer M*NB)
*
              CALL ZUNMBR( 'P', 'R', 'C', NS, N, M, A, LDA,
     $                     WORK( ITAUP ), VT, LDVT, WORK( ITEMP ),
     $                     LWORK-ITEMP+1, INFO )
           END IF
        END IF
     END IF
*
*    Undo scaling if necessary
*
     IF( ISCL.EQ.1 ) THEN
        IF( ANRM.GT.BIGNUM )
     $      CALL DLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1,
     $                   S, MINMN, INFO )
        IF( ANRM.LT.SMLNUM )
     $      CALL DLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1,
     $                   S, MINMN, INFO )
     END IF
*
```

```
*     Return optimal workspace in WORK(1)
*
      WORK( 1 ) = DCMPLX( DBLE( MAXWRK ), ZERO )
*
      RETURN
*
*     End of ZGESVDX
*
      END
```

———-

31.4.4 lisp code

— **zgesvdx.lisp** —

```
(let* ((czero (f2cl-lib:cmplx 0.0d0 0.0d0)) (zero 0.0d0) (one 1.0d0))
  (declare (type (f2cl-lib:complex16) czero)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable czero zero one))
  (defun zgesvdx
         (jobu jobvt range m n a lda vl vu il iu ns s u ldu vt ldvt work lwork
          rwork iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (array double-float (*)) rwork s)
             (type (double-float) vu vl)
             (type (array f2cl-lib:complex16 (*)) work vt u a)
             (type (f2cl-lib:integer4) info lwork ldvt ldu ns iu il lda n m)
             (type (string *) range jobvt jobu))
    (f2cl-lib:with-multi-array-data
        ((jobu character jobu-%data% jobu-%offset%)
         (jobvt character jobvt-%data% jobvt-%offset%)
         (range character range-%data% range-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (u f2cl-lib:complex16 u-%data% u-%offset%)
         (vt f2cl-lib:complex16 vt-%data% vt-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (s double-float s-%data% s-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((dum (make-array 1 :element-type 'double-float)) (abstol 0.0d0)
             (anrm 0.0d0) (bignum 0.0d0) (eps 0.0d0) (smlnum 0.0d0) (i 0)
             (id 0) (ie 0) (ierr 0) (ilqf 0) (iltgk 0) (iqrf 0) (iscl 0)
             (itau 0) (itaup 0) (itauq 0) (itemp 0) (itgkz 0) (iutgk 0) (j 0)
             (k 0) (maxwrk 0) (minmn 0) (minwrk 0) (mnthr 0) (alls nil)
             (inds nil) (lquery nil) (vals nil) (wantu nil) (wantvt nil)
             (jobz
```

```
      (make-array '(1) :element-type 'character :initial-element #\ ))
     (rngtgk
      (make-array '(1) :element-type 'character :initial-element #\ )))
(declare (type (array double-float (1)) dum)
         (type (double-float) abstol anrm bignum eps smlnum)
         (type (f2cl-lib:integer4) i id ie ierr ilqf iltgk iqrf iscl
          itau itaup itauq itemp itgkz iutgk j k maxwrk minmn minwrk
          mnthr)
         (type f2cl-lib:logical alls inds lquery vals wantu wantvt)
         (type (string 1) jobz rngtgk))
(setf ns 0)
(setf info 0)
(setf abstol (* 2 (dlamch "S")))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(setf wantu
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobu "V")
          (declare (ignore var-1))
          (when var-0 (setf jobu var-0))
          ret-val))
(setf wantvt
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvt "V")
          (declare (ignore var-1))
          (when var-0 (setf jobvt var-0))
          ret-val))
(cond
 ((or wantu wantvt) (f2cl-lib:f2cl-set-string jobz "V" (string 1)))
 (t (f2cl-lib:f2cl-set-string jobz "N" (string 1))))
(setf alls
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "A")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf vals
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "V")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf inds
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "I")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf info 0)
(cond
```

```
((and
  (not
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame jobu "V")
     (declare (ignore var-1))
     (when var-0 (setf jobu var-0))
     ret-val))
  (not
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame jobu "N")
     (declare (ignore var-1))
     (when var-0 (setf jobu var-0))
     ret-val)))
 (setf info -1))
((and
  (not
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame jobvt "V")
     (declare (ignore var-1))
     (when var-0 (setf jobvt var-0))
     ret-val))
  (not
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame jobvt "N")
     (declare (ignore var-1))
     (when var-0 (setf jobvt var-0))
     ret-val)))
 (setf info -2))
((not (or alls vals inds)) (setf info -3)) ((< m 0) (setf info -4))
((< n 0) (setf info -5)) ((> m lda) (setf info -7))
((> minmn 0)
 (cond
  (vals
   (cond ((< vl zero) (setf info -8)) ((<= vu vl) (setf info -9))))
  (inds
   (cond
    ((or (< il 1)
         (> il
            (max (the f2cl-lib:integer4 1)
                 (the f2cl-lib:integer4 minmn))))
     (setf info -10))
    ((or
      (< iu
         (min (the f2cl-lib:integer4 minmn)
              (the f2cl-lib:integer4 il)))
      (> iu minmn))
     (setf info -11)))))
 (cond
  ((= info 0)
   (cond ((and wantu (< ldu m)) (setf info -15))
```

```
          ((and wantvt (< ldvt minmn)) (setf info -16)))))))
(cond
 ((= info 0) (setf minwrk 1) (setf maxwrk 1)
  (cond
   ((> minmn 0)
    (cond
     ((>= m n)
      (setf mnthr
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 6 "ZGESVD" (f2cl-lib:f2cl-// jobu jobvt) m n
                   0 0)
                (declare (ignore var-0 var-1 var-2 var-5 var-6))
                (setf m var-3)
                (setf n var-4)
                ret-val))
      (cond
       ((>= m mnthr)
        (setf maxwrk
                (f2cl-lib:int-add n
                                  (f2cl-lib:int-mul n
                                                    (multiple-value-bind
                                                        (ret-val var-0
                                                         var-1 var-2
                                                         var-3 var-4
                                                         var-5 var-6)
                                                        (ilaenv 1
                                                         "DGEQRF" " " m
                                                         n -1 -1)
                                                      (declare
                                                       (ignore var-0
                                                        var-1 var-2
                                                        var-5 var-6))
                                                      (setf m var-3)
                                                      (setf n var-4)
                                                      ret-val))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul n n) n
                                            (f2cl-lib:int-mul 2 n
                                                              (multiple-value-bind
                                                                  (ret-val
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-3
                                                                   var-4
                                                                   var-5
                                                                   var-6)
```

```
                                                                  (ilaenv
                                                                   1
                                                                   "DGEBRD"
                                                                   " "
                                                                   n n
                                                                   -1
                                                                   -1)
                                                                 (declare
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-5
                                                                   var-6))
                                                                 (setf n
                                                                         var-3)
                                                                 (setf n
                                                                         var-4)
                                                                 ret-val))))))
    (setf minwrk (f2cl-lib:int-mul n (f2cl-lib:int-add n 4))))
   (t
    (setf maxwrk
            (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                              (f2cl-lib:int-mul
                               (f2cl-lib:int-add m n)
                               (multiple-value-bind
                                   (ret-val var-0 var-1 var-2 var-3
                                    var-4 var-5 var-6)
                                   (ilaenv 1 "ZGEBRD" " " m n -1
                                    -1)
                                 (declare
                                  (ignore var-0 var-1 var-2 var-5
                                   var-6))
                                 (setf m var-3)
                                 (setf n var-4)
                                 ret-val))))
    (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) m)))))
  (t
   (setf mnthr
           (multiple-value-bind
               (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
               (ilaenv 6 "ZGESVD" (f2cl-lib:f2cl-// jobu jobvt) m n
                0 0)
             (declare (ignore var-0 var-1 var-2 var-5 var-6))
             (setf m var-3)
             (setf n var-4)
             ret-val))
   (cond
    ((>= n mnthr)
     (setf maxwrk
```

```
                (f2cl-lib:int-add m
                                  (f2cl-lib:int-mul m
                                                    (multiple-value-bind
                                                        (ret-val var-0
                                                         var-1 var-2
                                                         var-3 var-4
                                                         var-5 var-6)
                                                        (ilaenv 1
                                                         "ZGELQF" " " m
                                                         n -1 -1)
                                                      (declare
                                                       (ignore var-0
                                                        var-1 var-2
                                                        var-5 var-6))
                                                      (setf m var-3)
                                                      (setf n var-4)
                                                      ret-val))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul m m) m
                                            (f2cl-lib:int-mul 2 m
                                                              (multiple-value-bind
                                                                  (ret-val
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-3
                                                                   var-4
                                                                   var-5
                                                                   var-6)
                                                                  (ilaenv
                                                                   1
                                                                   "ZGEBRD"
                                                                   " "
                                                                   m m
                                                                   -1
                                                                   -1)
                                                                (declare
                                                                 (ignore
                                                                  var-0
                                                                  var-1
                                                                  var-2
                                                                  var-5
                                                                  var-6))
                                                                (setf m
                                                                        var-3)
                                                                (setf m
                                                                        var-4)
                                                                ret-val))))))
```

```
        (setf minwrk (f2cl-lib:int-mul m (f2cl-lib:int-add m 4))))
       (t
        (setf maxwrk
                (f2cl-lib:int-add
                 (f2cl-lib:int-mul m
                                   (f2cl-lib:int-add
                                    (f2cl-lib:int-mul m 2) 19))
                 (f2cl-lib:int-mul (f2cl-lib:int-add m n)
                                   (multiple-value-bind
                                       (ret-val var-0 var-1 var-2 var-3
                                        var-4 var-5 var-6)
                                       (ilaenv 1 "ZGEBRD" " " m n -1
                                        -1)
                                     (declare
                                      (ignore var-0 var-1 var-2 var-5
                                       var-6))
                                     (setf m var-3)
                                     (setf n var-4)
                                     ret-val))))
        (setf minwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m) n))))))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4 minwrk)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:dcmplx (f2cl-lib:dble maxwrk) zero))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -19)))))
(cond
 ((/= info 0) (xerbla "ZGESVDX" (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(cond ((or (= m 0) (= n 0)) (go end_label)))
(setf alls
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "A")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf inds
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "I")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(cond
 (alls (f2cl-lib:f2cl-set-string rngtgk "I" (string 1)) (setf iltgk 1)
  (setf iutgk
          (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n))))
 (inds (f2cl-lib:f2cl-set-string rngtgk "I" (string 1)) (setf iltgk il)
  (setf iutgk iu))
 (t (f2cl-lib:f2cl-set-string rngtgk "V" (string 1)) (setf iltgk 0)
```

```
  (setf iutgk 0)))
(setf eps (dlamch "P"))
(setf smlnum (/ (f2cl-lib:fsqrt (dlamch "S")) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "M" m n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iscl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf iscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 anrm smlnum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9))))
 ((> anrm bignum) (setf iscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 anrm bignum m n a lda info)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf info var-9)))))
(cond
 ((>= m n)
  (cond
   ((>= m mnthr) (setf itau 1) (setf itemp (f2cl-lib:int-add itau n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (zgeqrf m n a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itemp)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
      (declare (ignore var-2 var-4 var-5 var-6))
```

```
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-7))
(setf iqrf itemp)
(setf itauq (f2cl-lib:int-add itemp (f2cl-lib:int-mul n n)))
(setf itaup (f2cl-lib:int-add itauq n))
(setf itemp (f2cl-lib:int-add itaup n)) (setf id 1)
(setf ie (f2cl-lib:int-add id n))
(setf itgkz (f2cl-lib:int-add ie n))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iqrf)
                           ((1 *)) work-%offset%)
     n)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf n var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1) czero
     czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ iqrf 1)) ((1 *)) work-%offset%)
     n)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf n var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgebrd n n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iqrf)
                           ((1 *)) work-%offset%)
     n
     (f2cl-lib:array-slice rwork-%data% double-float (id) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itemp)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
```

```
  (setf n var-1)
  (setf n var-3)
  (setf info var-10))
(setf itemp
        (f2cl-lib:int-add itgkz
                          (f2cl-lib:int-mul n
                                            (f2cl-lib:int-add
                                             (f2cl-lib:int-mul n 2)
                                             1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (dbdsvdx "U" jobz rngtgk n
     (f2cl-lib:array-slice rwork-%data% double-float (id) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     vl vu iltgk iutgk ns s
     (f2cl-lib:array-slice rwork-%data% double-float (itgkz)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:int-mul n 2)
     (f2cl-lib:array-slice rwork-%data% double-float (itemp)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-4 var-5 var-11 var-12 var-13 var-14 var-15))
  (when var-1 (setf jobz var-1))
  (when var-2 (setf rngtgk var-2))
  (when var-3 (setf n var-3))
  (when var-6 (setf vl var-6))
  (when var-7 (setf vu var-7))
  (when var-8 (setf iltgk var-8))
  (when var-9 (setf iutgk var-9))
  (when var-10 (setf ns var-10))
  (when var-16 (setf info var-16)))
(cond
 (wantu (setf k itgkz)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf (f2cl-lib:fref u-%data% (j i) ((1 ldu) (1 *))
                               u-%offset%)
                  (f2cl-lib:dcmplx
                   (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                  rwork-%offset%)
                   zero))
          (setf k (f2cl-lib:int-add k 1))
```

```
        label100001))
      (setf k (f2cl-lib:int-add k n))
    label100000))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "A" (f2cl-lib:int-sub m n) n czero czero
     (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                           ((+ n 1) 1) ((1 ldu) (1 *))
                           u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-5))
  (when var-2 (setf n var-2))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (zunmbr "Q" "L" "N" n ns n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iqrf)
                           ((1 *)) work-%offset%)
     n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itemp)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf ns var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf n var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf info var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (zunmqr "L" "N" m ns n a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itemp)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf ns var-3))
  (when var-4 (setf n var-4))
  (when var-6 (setf lda var-6))
```

```
        (when var-9 (setf ldu var-9))
        (when var-12 (setf info var-12)))))
    (cond
     (wantvt (setf k (f2cl-lib:int-add itgkz n))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i ns) nil)
        (tagbody
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf (f2cl-lib:fref vt-%data% (i j) ((1 ldvt) (1 *))
                                   vt-%offset%)
                    (f2cl-lib:dcmplx
                     (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                    rwork-%offset%)
                     zero))
              (setf k (f2cl-lib:int-add k 1))
             label100003))
          (setf k (f2cl-lib:int-add k n))
         label100002))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13)
          (zunmbr "P" "R" "C" ns n n
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iqrf)
                                 ((1 *)) work-%offset%)
           n
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                                 ((1 *)) work-%offset%)
           vt ldvt
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itemp)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
        (declare
         (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
        (when var-3 (setf ns var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf n var-5))
        (when var-7 (setf n var-7))
        (when var-10 (setf ldvt var-10))
        (when var-13 (setf info var-13))))))
    (t (setf itauq 1) (setf itaup (f2cl-lib:int-add itauq n))
     (setf itemp (f2cl-lib:int-add itaup n)) (setf id 1)
     (setf ie (f2cl-lib:int-add id n))
     (setf itgkz (f2cl-lib:int-add ie n))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (zgebrd m n a lda
          (f2cl-lib:array-slice rwork-%data% double-float (id) ((1 *))
```

```
                              rwork-%offset%)
      (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                              rwork-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                              ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                              ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itemp)
                              ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-10))
(setf itemp
        (f2cl-lib:int-add itgkz
                            (f2cl-lib:int-mul n
                                                (f2cl-lib:int-add
                                                 (f2cl-lib:int-mul n 2)
                                                 1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (dbdsvdx "U" jobz rngtgk n
     (f2cl-lib:array-slice rwork-%data% double-float (id) ((1 *))
                              rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                              rwork-%offset%)
     vl vu iltgk iutgk ns s
     (f2cl-lib:array-slice rwork-%data% double-float (itgkz)
                              ((1 *)) rwork-%offset%)
     (f2cl-lib:int-mul n 2)
     (f2cl-lib:array-slice rwork-%data% double-float (itemp)
                              ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-4 var-5 var-11 var-12 var-13 var-14 var-15))
  (when var-1 (setf jobz var-1))
  (when var-2 (setf rngtgk var-2))
  (when var-3 (setf n var-3))
  (when var-6 (setf vl var-6))
  (when var-7 (setf vu var-7))
  (when var-8 (setf iltgk var-8))
  (when var-9 (setf iutgk var-9))
  (when var-10 (setf ns var-10))
  (when var-16 (setf info var-16)))
(cond
 (wantu (setf k itgkz)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                ((> i ns) nil)
      (tagbody
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf (f2cl-lib:fref u-%data% (j i) ((1 ldu) (1 *))
                                 u-%offset%)
                  (f2cl-lib:dcmplx
                   (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                  rwork-%offset%)
                   zero))
            (setf k (f2cl-lib:int-add k 1))
           label100005))
        (setf k (f2cl-lib:int-add k n))
       label100004))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlaset "A" (f2cl-lib:int-sub m n) n czero czero
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                               ((+ n 1) 1) ((1 ldu) (1 *))
                               u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-5))
      (when var-2 (setf n var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldu var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (zunmbr "Q" "L" "N" m ns n a lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                               ((1 *)) work-%offset%)
         u ldu
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itemp)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) ierr)
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
      (when var-3 (setf m var-3))
      (when var-4 (setf ns var-4))
      (when var-5 (setf n var-5))
      (when var-7 (setf lda var-7))
      (when var-10 (setf ldu var-10))
      (when var-13 (setf ierr var-13)))))
  (cond
   (wantvt (setf k (f2cl-lib:int-add itgkz n))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i ns) nil)
      (tagbody
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                        ((> j n) nil)
                (tagbody
                  (setf (f2cl-lib:fref vt-%data% (i j) ((1 ldvt) (1 *))
                                       vt-%offset%)
                        (f2cl-lib:dcmplx
                         (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                        rwork-%offset%)
                         zero))
                  (setf k (f2cl-lib:int-add k 1))
                 label100007))
              (setf k (f2cl-lib:int-add k n))
             label100006))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10 var-11 var-12 var-13)
              (zunmbr "P" "R" "C" ns n n a lda
               (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                                     ((1 *)) work-%offset%)
               vt ldvt
               (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itemp)
                                     ((1 *)) work-%offset%)
               (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) ierr)
            (declare
             (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
            (when var-3 (setf ns var-3))
            (when var-4 (setf n var-4))
            (when var-5 (setf n var-5))
            (when var-7 (setf lda var-7))
            (when var-10 (setf ldvt var-10))
            (when var-13 (setf ierr var-13))))))))
(t
 (cond
  ((>= n mnthr) (setf itau 1) (setf itemp (f2cl-lib:int-add itau m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (zgelqf m n a lda
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itemp)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
     (declare (ignore var-2 var-4 var-5 var-6))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf info var-7))
   (setf ilqf itemp)
   (setf itauq (f2cl-lib:int-add ilqf (f2cl-lib:int-mul m m)))
   (setf itaup (f2cl-lib:int-add itauq m))
   (setf itemp (f2cl-lib:int-add itaup m)) (setf id 1)
```

```
(setf ie (f2cl-lib:int-add id m))
(setf itgkz (f2cl-lib:int-add ie m))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ilqf)
                           ((1 *)) work-%offset%)
     m)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf m var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1) czero
     czero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ ilqf m)) ((1 *)) work-%offset%)
     m)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf czero var-3))
  (when var-4 (setf czero var-4))
  (when var-6 (setf m var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgebrd m m
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ilqf)
                           ((1 *)) work-%offset%)
     m
     (f2cl-lib:array-slice rwork-%data% double-float (id) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itemp)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf m var-3)
  (setf info var-10))
(setf itemp
        (f2cl-lib:int-add itgkz
                          (f2cl-lib:int-mul m
                                            (f2cl-lib:int-add
                                             (f2cl-lib:int-mul m 2)
```

```
                                                1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (dbdsvdx "U" jobz rngtgk m
     (f2cl-lib:array-slice rwork-%data% double-float (id) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     vl vu iltgk iutgk ns s
     (f2cl-lib:array-slice rwork-%data% double-float (itgkz)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:int-mul m 2)
     (f2cl-lib:array-slice rwork-%data% double-float (itemp)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-4 var-5 var-11 var-12 var-13 var-14 var-15))
  (when var-1 (setf jobz var-1))
  (when var-2 (setf rngtgk var-2))
  (when var-3 (setf m var-3))
  (when var-6 (setf vl var-6))
  (when var-7 (setf vu var-7))
  (when var-8 (setf iltgk var-8))
  (when var-9 (setf iutgk var-9))
  (when var-10 (setf ns var-10))
  (when var-16 (setf info var-16)))
(cond
 (wantu (setf k itgkz)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j m) nil)
        (tagbody
          (setf (f2cl-lib:fref u-%data% (j i) ((1 ldu) (1 *))
                                u-%offset%)
                  (f2cl-lib:dcmplx
                   (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                  rwork-%offset%)
                   zero))
          (setf k (f2cl-lib:int-add k 1))
         label100009))
      (setf k (f2cl-lib:int-add k m))
     label100008))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (zunmbr "Q" "L" "N" m ns m
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ilqf)
```

```
                                ((1 *)) work-%offset%)
          m
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                                ((1 *)) work-%offset%)
          u ldu
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itemp)
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
      (when var-3 (setf m var-3))
      (when var-4 (setf ns var-4))
      (when var-5 (setf m var-5))
      (when var-7 (setf m var-7))
      (when var-10 (setf ldu var-10))
      (when var-13 (setf info var-13)))))
  (cond
   (wantvt (setf k (f2cl-lib:int-add itgkz m))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i ns) nil)
      (tagbody
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j m) nil)
          (tagbody
            (setf (f2cl-lib:fref vt-%data% (i j) ((1 ldvt) (1 *))
                                 vt-%offset%)
                  (f2cl-lib:dcmplx
                   (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                  rwork-%offset%)
                   zero))
            (setf k (f2cl-lib:int-add k 1))
           label100011))
        (setf k (f2cl-lib:int-add k m))
       label100010))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlaset "A" m (f2cl-lib:int-sub n m) czero czero
         (f2cl-lib:array-slice vt-%data% f2cl-lib:complex16
                               (1 (f2cl-lib:int-add m 1))
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt)
      (declare (ignore var-0 var-2 var-5))
      (when var-1 (setf m var-1))
      (when var-3 (setf czero var-3))
      (when var-4 (setf czero var-4))
      (when var-6 (setf ldvt var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13)
        (zunmbr "P" "R" "C" ns m m
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ilqf)
```

```
                                ((1 *)) work-%offset%)
                m
                (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                                ((1 *)) work-%offset%)
                vt ldvt
                (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itemp)
                                ((1 *)) work-%offset%)
                (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
            (declare
             (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
            (when var-3 (setf ns var-3))
            (when var-4 (setf m var-4))
            (when var-5 (setf m var-5))
            (when var-7 (setf m var-7))
            (when var-10 (setf ldvt var-10))
            (when var-13 (setf info var-13)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10 var-11 var-12)
              (zunmlq "R" "N" ns n m a lda
               (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                                ((1 *)) work-%offset%)
               vt ldvt
               (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itemp)
                                ((1 *)) work-%offset%)
               (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
            (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
            (when var-2 (setf ns var-2))
            (when var-3 (setf n var-3))
            (when var-4 (setf m var-4))
            (when var-6 (setf lda var-6))
            (when var-9 (setf ldvt var-9))
            (when var-12 (setf info var-12))))))
      (t (setf itauq 1) (setf itaup (f2cl-lib:int-add itauq m))
       (setf itemp (f2cl-lib:int-add itaup m)) (setf id 1)
       (setf ie (f2cl-lib:int-add id m))
       (setf itgkz (f2cl-lib:int-add ie m))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10)
           (zgebrd m n a lda
            (f2cl-lib:array-slice rwork-%data% double-float (id) ((1 *))
                                  rwork-%offset%)
            (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                                  rwork-%offset%)
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                                  ((1 *)) work-%offset%)
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                                  ((1 *)) work-%offset%)
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itemp)
```

```
                          ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf info var-10))
(setf itemp
        (f2cl-lib:int-add itgkz
                          (f2cl-lib:int-mul m
                                            (f2cl-lib:int-add
                                             (f2cl-lib:int-mul m 2)
                                             1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (dbdsvdx "L" jobz rngtgk m
     (f2cl-lib:array-slice rwork-%data% double-float (id) ((1 *))
                           rwork-%offset%)
     (f2cl-lib:array-slice rwork-%data% double-float (ie) ((1 *))
                           rwork-%offset%)
     vl vu iltgk iutgk ns s
     (f2cl-lib:array-slice rwork-%data% double-float (itgkz)
                           ((1 *)) rwork-%offset%)
     (f2cl-lib:int-mul m 2)
     (f2cl-lib:array-slice rwork-%data% double-float (itemp)
                           ((1 *)) rwork-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-4 var-5 var-11 var-12 var-13 var-14 var-15))
  (when var-1 (setf jobz var-1))
  (when var-2 (setf rngtgk var-2))
  (when var-3 (setf m var-3))
  (when var-6 (setf vl var-6))
  (when var-7 (setf vu var-7))
  (when var-8 (setf iltgk var-8))
  (when var-9 (setf iutgk var-9))
  (when var-10 (setf ns var-10))
  (when var-16 (setf info var-16)))
(cond
 (wantu (setf k itgkz)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j m) nil)
        (tagbody
          (setf (f2cl-lib:fref u-%data% (j i) ((1 ldu) (1 *))
                               u-%offset%)
                (f2cl-lib:dcmplx
```

```
                        (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                       rwork-%offset%)
                        zero))
                (setf k (f2cl-lib:int-add k 1))
              label100013))
            (setf k (f2cl-lib:int-add k m))
          label100012))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11 var-12 var-13)
            (zunmbr "Q" "L" "N" m ns n a lda
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itauq)
                                   ((1 *)) work-%offset%)
             u ldu
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itemp)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
          (declare
           (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
          (when var-3 (setf m var-3))
          (when var-4 (setf ns var-4))
          (when var-5 (setf n var-5))
          (when var-7 (setf lda var-7))
          (when var-10 (setf ldu var-10))
          (when var-13 (setf info var-13)))))
      (cond
       (wantvt (setf k (f2cl-lib:int-add itgkz m))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i ns) nil)
          (tagbody
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j m) nil)
              (tagbody
                (setf (f2cl-lib:fref vt-%data% (i j) ((1 ldvt) (1 *))
                                     vt-%offset%)
                      (f2cl-lib:dcmplx
                       (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                      rwork-%offset%)
                       zero))
                (setf k (f2cl-lib:int-add k 1))
              label100015))
            (setf k (f2cl-lib:int-add k m))
          label100014))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zlaset "A" m (f2cl-lib:int-sub n m) czero czero
             (f2cl-lib:array-slice vt-%data% f2cl-lib:complex16
                                   (1 (f2cl-lib:int-add m 1))
                                   ((1 ldvt) (1 *)) vt-%offset%)
             ldvt)
          (declare (ignore var-0 var-2 var-5))
```

```
            (when var-1 (setf m var-1))
            (when var-3 (setf czero var-3))
            (when var-4 (setf czero var-4))
            (when var-6 (setf ldvt var-6)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10 var-11 var-12 var-13)
              (zunmbr "P" "R" "C" ns n m a lda
               (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itaup)
                                     ((1 *)) work-%offset%)
               vt ldvt
               (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itemp)
                                     ((1 *)) work-%offset%)
               (f2cl-lib:int-add (f2cl-lib:int-sub lwork itemp) 1) info)
            (declare
             (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
            (when var-3 (setf ns var-3))
            (when var-4 (setf n var-4))
            (when var-5 (setf m var-5))
            (when var-7 (setf lda var-7))
            (when var-10 (setf ldvt var-10))
            (when var-13 (setf info var-13)))))))))
  (cond
   ((= iscl 1)
    (if (> anrm bignum)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
            (dlascl "G" 0 0 bignum anrm minmn 1 s minmn info)
          (declare (ignore var-0 var-1 var-2 var-6 var-7))
          (when var-3 (setf bignum var-3))
          (when var-4 (setf anrm var-4))
          (when var-5 (setf minmn var-5))
          (when var-8 (setf minmn var-8))
          (when var-9 (setf info var-9))))
    (if (< anrm smlnum)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
            (dlascl "G" 0 0 smlnum anrm minmn 1 s minmn info)
          (declare (ignore var-0 var-1 var-2 var-6 var-7))
          (when var-3 (setf smlnum var-3))
          (when var-4 (setf anrm var-4))
          (when var-5 (setf minmn var-5))
          (when var-8 (setf minmn var-8))
          (when var-9 (setf info var-9))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:dcmplx (f2cl-lib:dble maxwrk) zero))
  (go end_label)
 end_label
  (return
   (values jobu jobvt range m n nil lda vl vu nil nil ns nil nil ldu nil
```

```
                 ldvt nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgesvdx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::jobu
                                              fortran-to-lisp::jobvt
                                              fortran-to-lisp::range
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda
                                              fortran-to-lisp::vl
                                              fortran-to-lisp::vu nil nil
                                              fortran-to-lisp::ns nil nil
                                              fortran-to-lisp::ldu nil
                                              fortran-to-lisp::ldvt nil nil nil
                                              nil fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::zgelqf
                                              fortran-to-lisp::zgebrd
                                              fortran-to-lisp::zgeqrf
                                              fortran-to-lisp::xerbla
```

```
                                        fortran-to-lisp::ilaenv))))
```

Chapter 32

LAPACK - General Matrices, Comp. Routines, Double

32.1 dgebak

32.1.1 Axiom unit tests

— dgebak.input —

```
)set break resume
)sys rm -f dgebak.output
)spool dgebak.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.1.2 Axiom help page

— dgebak.help —

```
==========================================================================
dgebak examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgebak (JOB, SIDE, N, ILO, IHI, SCALE, M, V, LDV, INFO)
           DGEBAK

Function/Subroutine Documentation
   subroutine dgebak (character JOB, character SIDE, integer N, integer ILO,
       integer IHI, double precision, dimension( * ) SCALE, integer M, double
       precision, dimension( ldv, * ) V, integer LDV, integer INFO)
       DGEBAK

Purpose:
   DGEBAK forms the right or left eigenvectors of a real general matrix
   by backward transformation on the computed eigenvectors of the
   balanced matrix output by DGEBAL.

Parameters:

 JOB
   JOB is CHARACTER*1
   Specifies the type of backward transformation required:
   = 'N', do nothing, return immediately;
   = 'P', do backward transformation for permutation only;
   = 'S', do backward transformation for scaling only;
   = 'B', do backward transformations for both permutation and scaling.
   JOB must be the same as the argument JOB supplied to DGEBAL.

 SIDE
   SIDE is CHARACTER*1
   = 'R':  V contains right eigenvectors;
   = 'L':  V contains left eigenvectors.

 N
   N is INTEGER
   The number of rows of the matrix V.  N >= 0.

 ILO
   ILO is INTEGER

 IHI
   IHI is INTEGER
   The integers ILO and IHI determined by DGEBAL.
   1 <= ILO <= IHI <= N, if N > 0; ILO=1 and IHI=0, if N=0.

 SCALE
```

```
  SCALE is DOUBLE PRECISION array, dimension (N)
  Details of the permutation and scaling factors, as returned by DGEBAL.

M
  M is INTEGER
  The number of columns of the matrix V.  M >= 0.

V
  V is DOUBLE PRECISION array, dimension (LDV,M)
  On entry, the matrix of right or left eigenvectors to be
  transformed, as returned by DHSEIN or DTREVC.
  On exit, V is overwritten by the transformed eigenvectors.

LDV
  LDV is INTEGER
  The leading dimension of the array V. LDV >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

32.1.3 fortran code

— dgebak.f —

```
*  =====================================================================
      SUBROUTINE DGEBAK( JOB, SIDE, N, ILO, IHI, SCALE, M, V, LDV,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOB, SIDE
      INTEGER            IHI, ILO, INFO, LDV, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   SCALE( * ), V( LDV, * )
*     ..
*
*  =====================================================================
*
```

```
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LEFTV, RIGHTV
      INTEGER            I, II, K
      DOUBLE PRECISION   S
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DSCAL, DSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Decode and Test the input parameters
*
      RIGHTV = LSAME( SIDE, 'R' )
      LEFTV = LSAME( SIDE, 'L' )
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.RIGHTV .AND. .NOT.LEFTV ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -5
      ELSE IF( M.LT.0 ) THEN
         INFO = -7
      ELSE IF( LDV.LT.MAX( 1, N ) ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEBAK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
```

```
      IF( N.EQ.0 )
     $   RETURN
      IF( M.EQ.0 )
     $   RETURN
      IF( LSAME( JOB, 'N' ) )
     $   RETURN
*
      IF( ILO.EQ.IHI )
     $   GO TO 30
*
*     Backward balance
*
      IF( LSAME( JOB, 'S' ) .OR. LSAME( JOB, 'B' ) ) THEN
*
         IF( RIGHTV ) THEN
            DO 10 I = ILO, IHI
               S = SCALE( I )
               CALL DSCAL( M, S, V( I, 1 ), LDV )
   10       CONTINUE
         END IF
*
         IF( LEFTV ) THEN
            DO 20 I = ILO, IHI
               S = ONE / SCALE( I )
               CALL DSCAL( M, S, V( I, 1 ), LDV )
   20       CONTINUE
         END IF
*
      END IF
*
*     Backward permutation
*
*     For  I = ILO-1 step -1 until 1,
*              IHI+1 step 1 until N do --
*
   30 CONTINUE
      IF( LSAME( JOB, 'P' ) .OR. LSAME( JOB, 'B' ) ) THEN
         IF( RIGHTV ) THEN
            DO 40 II = 1, N
               I = II
               IF( I.GE.ILO .AND. I.LE.IHI )
     $            GO TO 40
               IF( I.LT.ILO )
     $            I = ILO - II
               K = SCALE( I )
               IF( K.EQ.I )
     $            GO TO 40
               CALL DSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   40       CONTINUE
         END IF
```

```
*
         IF( LEFTV ) THEN
            DO 50 II = 1, N
               I = II
               IF( I.GE.ILO .AND. I.LE.IHI )
     $            GO TO 50
               IF( I.LT.ILO )
     $            I = ILO - II
               K = SCALE( I )
               IF( K.EQ.I )
     $            GO TO 50
               CALL DSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   50       CONTINUE
         END IF
      END IF
*
      RETURN
*
*     End of DGEBAK
*
      END
```

———-

32.1.4 lisp code

— **dgebak.lisp** —

———-

32.2 dgebal

32.2.1 Axiom unit tests

— **dgebal.input** —

```
)set break resume
)sys rm -f dgebal.output
)spool dgebal.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

32.2.2 Axiom help page

— **dgebal.help** —

```
=====================================================================
dgebal examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgebal (JOB, N, A, LDA, ILO, IHI, SCALE, INFO)
           DGEBAL

Function/Subroutine Documentation
   subroutine dgebal (character JOB, integer N, double precision, dimension(
       lda, * ) A, integer LDA, integer ILO, integer IHI, double precision,
       dimension( * ) SCALE, integer INFO)
       DGEBAL

Purpose:
   DGEBAL balances a general real matrix A.  This involves, first,
   permuting A by a similarity transformation to isolate eigenvalues
   in the first 1 to ILO-1 and last IHI+1 to N elements on the
   diagonal; and second, applying a diagonal similarity transformation
   to rows and columns ILO to IHI to make the rows and columns as
   close in norm as possible.  Both steps are optional.

   Balancing may reduce the 1-norm of the matrix, and improve the
   accuracy of the computed eigenvalues and/or eigenvectors.

Parameters:

 JOB
   JOB is CHARACTER*1
   Specifies the operations to be performed on A:
   = 'N':  none:  simply set ILO = 1, IHI = N, SCALE(I) = 1.0 for i = 1,...,N;
   = 'P':  permute only;
   = 'S':  scale only;
```

```
  = 'B':  both permute and scale.

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

A
  A is DOUBLE array, dimension (LDA,N)
  On entry, the input matrix A.
  On exit,  A is overwritten by the balanced matrix.
  If JOB = 'N', A is not referenced.
  See Further Details.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER
  ILO and IHI are set to integers such that on exit
  A(i,j) = 0 if i > j and j = 1,...,ILO-1 or I = IHI+1,...,N.
  If JOB = 'N' or 'S', ILO = 1 and IHI = N.

SCALE
  SCALE is DOUBLE array, dimension (N)
  Details of the permutations and scaling factors applied to
  A.  If P(j) is the index of the row and column interchanged
  with row and column j and D(j) is the scaling factor
  applied to row and column j, then
  SCALE(j) = P(j)    for j = 1,...,ILO-1
           = D(j)    for j = ILO,...,IHI
           = P(j)    for j = IHI+1,...,N.
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.

Further Details:
  The permutations consist of row and column interchanges which put
  the matrix in the form

                        ( T1   X   Y  )
               P A P = (  0   B   Z  )
                        (  0   0   T2 )
```

```
where T1 and T2 are upper triangular matrices whose eigenvalues lie
along the diagonal.  The column indices ILO and IHI mark the starting
and ending columns of the submatrix B. Balancing consists of applying
a diagonal similarity transformation inv(D) * B * D to make the
1-norms of each row of B and its corresponding column nearly equal.
The output matrix is

             ( T1     X*D          Y    )
             (  0  inv(D)*B*D  inv(D)*Z ).
             (  0      0           T2   )

Information about the permutations P and the diagonal matrix D is
returned in the vector SCALE.
```

32.2.3 fortran code

— dgebal.f —

```
*  ===================================================================
      SUBROUTINE DGEBAL( JOB, N, A, LDA, ILO, IHI, SCALE, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOB
      INTEGER            IHI, ILO, INFO, LDA, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), SCALE( * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   SCLFAC
      PARAMETER          ( SCLFAC = 2.0D+0 )
      DOUBLE PRECISION   FACTOR
      PARAMETER          ( FACTOR = 0.95D+0 )
*     ..
```

```
*     .. Local Scalars ..
      LOGICAL            NOCONV
      INTEGER            I, ICA, IEXC, IRA, J, K, L, M
      DOUBLE PRECISION   C, CA, F, G, R, RA, S, SFMAX1, SFMAX2, SFMIN1,
     $                   SFMIN2
*     ..
*     .. External Functions ..
      LOGICAL            DISNAN, LSAME
      INTEGER            IDAMAX
      DOUBLE PRECISION   DLAMCH, DNRM2
      EXTERNAL           DISNAN, LSAME, IDAMAX, DLAMCH, DNRM2
*     ..
*     .. External Subroutines ..
      EXTERNAL           DSCAL, DSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     Test the input parameters
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEBAL', -INFO )
         RETURN
      END IF
*
      K = 1
      L = N
*
      IF( N.EQ.0 )
     $   GO TO 210
*
      IF( LSAME( JOB, 'N' ) ) THEN
         DO 10 I = 1, N
            SCALE( I ) = ONE
   10    CONTINUE
         GO TO 210
      END IF
*
      IF( LSAME( JOB, 'S' ) )
     $   GO TO 120
*
```

```
*     Permutation to isolate eigenvalues if possible
*
      GO TO 50
*
*     Row and column exchange.
*
   20 CONTINUE
      SCALE( M ) = J
      IF( J.EQ.M )
     $   GO TO 30
*
      CALL DSWAP( L, A( 1, J ), 1, A( 1, M ), 1 )
      CALL DSWAP( N-K+1, A( J, K ), LDA, A( M, K ), LDA )
*
   30 CONTINUE
      GO TO ( 40, 80 )IEXC
*
*     Search for rows isolating an eigenvalue and push them down.
*
   40 CONTINUE
      IF( L.EQ.1 )
     $   GO TO 210
      L = L - 1
*
   50 CONTINUE
      DO 70 J = L, 1, -1
*
         DO 60 I = 1, L
            IF( I.EQ.J )
     $         GO TO 60
            IF( A( J, I ).NE.ZERO )
     $         GO TO 70
   60    CONTINUE
*
         M = L
         IEXC = 1
         GO TO 20
   70 CONTINUE
*
      GO TO 90
*
*     Search for columns isolating an eigenvalue and push them left.
*
   80 CONTINUE
      K = K + 1
*
   90 CONTINUE
      DO 110 J = K, L
*
         DO 100 I = K, L
```

```
            IF( I.EQ.J )
     $         GO TO 100
            IF( A( I, J ).NE.ZERO )
     $         GO TO 110
  100    CONTINUE
*
         M = K
         IEXC = 2
         GO TO 20
  110 CONTINUE
*
  120 CONTINUE
      DO 130 I = K, L
         SCALE( I ) = ONE
  130 CONTINUE
*
      IF( LSAME( JOB, 'P' ) )
     $   GO TO 210
*
*     Balance the submatrix in rows K to L.
*
*     Iterative loop for norm reduction
*
      SFMIN1 = DLAMCH( 'S' ) / DLAMCH( 'P' )
      SFMAX1 = ONE / SFMIN1
      SFMIN2 = SFMIN1*SCLFAC
      SFMAX2 = ONE / SFMIN2
*
  140 CONTINUE
      NOCONV = .FALSE.
*
      DO 200 I = K, L
*
         C = DNRM2( L-K+1, A( K, I ), 1 )
         R = DNRM2( L-K+1, A( I, K ), LDA )
         ICA = IDAMAX( L, A( 1, I ), 1 )
         CA = ABS( A( ICA, I ) )
         IRA = IDAMAX( N-K+1, A( I, K ), LDA )
         RA = ABS( A( I, IRA+K-1 ) )
*
*        Guard against zero C or R due to underflow.
*
         IF( C.EQ.ZERO .OR. R.EQ.ZERO )
     $      GO TO 200
         G = R / SCLFAC
         F = ONE
         S = C + R
  160    CONTINUE
         IF( C.GE.G .OR. MAX( F, C, CA ).GE.SFMAX2 .OR.
     $       MIN( R, G, RA ).LE.SFMIN2 )GO TO 170
```

```
            IF( DISNAN( C+F+CA+R+G+RA ) ) THEN
*
*           Exit if NaN to avoid infinite loop
*
            INFO = -3
            CALL XERBLA( 'DGEBAL', -INFO )
            RETURN
         END IF
         F = F*SCLFAC
         C = C*SCLFAC
         CA = CA*SCLFAC
         R = R / SCLFAC
         G = G / SCLFAC
         RA = RA / SCLFAC
         GO TO 160
*
  170    CONTINUE
         G = C / SCLFAC
  180    CONTINUE
         IF( G.LT.R .OR. MAX( R, RA ).GE.SFMAX2 .OR.
     $       MIN( F, C, G, CA ).LE.SFMIN2 )GO TO 190
         F = F / SCLFAC
         C = C / SCLFAC
         G = G / SCLFAC
         CA = CA / SCLFAC
         R = R*SCLFAC
         RA = RA*SCLFAC
         GO TO 180
*
*        Now balance.
*
  190    CONTINUE
         IF( ( C+R ).GE.FACTOR*S )
     $      GO TO 200
         IF( F.LT.ONE .AND. SCALE( I ).LT.ONE ) THEN
            IF( F*SCALE( I ).LE.SFMIN1 )
     $         GO TO 200
         END IF
         IF( F.GT.ONE .AND. SCALE( I ).GT.ONE ) THEN
            IF( SCALE( I ).GE.SFMAX1 / F )
     $         GO TO 200
         END IF
         G = ONE / F
         SCALE( I ) = SCALE( I )*F
         NOCONV = .TRUE.
*
         CALL DSCAL( N-K+1, G, A( I, K ), LDA )
         CALL DSCAL( L, F, A( 1, I ), 1 )
*
  200 CONTINUE
```

```
*
      IF( NOCONV )
     $   GO TO 140
*
  210 CONTINUE
      ILO = K
      IHI = L
*
      RETURN
*
*     End of DGEBAL
*
      END
```

———-

32.2.4 lisp code

— **dgebal.lisp** —

———-

32.3 dgebd2

32.3.1 Axiom unit tests

— **dgebd2.input** —

```
)set break resume
)sys rm -f dgebd2.output
)spool dgebd2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.3.2 Axiom help page

— dgebd2.help —

```
==================================================================
dgebd2 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgebd2 (M, N, A, LDA, D, E, TAUQ, TAUP, WORK, INFO)
           DGEBD2 reduces a general matrix to bidiagonal form using an
           unblocked algorithm.

Function/Subroutine Documentation
   subroutine dgebd2 (integer M, integer N, double precision, dimension( lda,
       * ) A, integer LDA, double precision, dimension( * ) D, double
       precision, dimension( * ) E, double precision, dimension( * ) TAUQ,
       double precision, dimension( * ) TAUP, double precision, dimension( * )
       WORK, integer INFO)
       DGEBD2 reduces a general matrix to bidiagonal form using an unblocked
       algorithm.

Purpose:
   DGEBD2 reduces a real general m by n matrix A to upper or lower
   bidiagonal form B by an orthogonal transformation: Q**T * A * P = B.

   If m >= n, B is upper bidiagonal; if m < n, B is lower bidiagonal.

Parameters:

 M
   M is INTEGER
   The number of rows in the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns in the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the m by n general matrix to be reduced.
   On exit,
   if m >= n, the diagonal and the first superdiagonal are
     overwritten with the upper bidiagonal matrix B; the
```

```
    elements below the diagonal, with the array TAUQ, represent
    the orthogonal matrix Q as a product of elementary
    reflectors, and the elements above the first superdiagonal,
    with the array TAUP, represent the orthogonal matrix P as
    a product of elementary reflectors;
  if m < n, the diagonal and the first subdiagonal are
    overwritten with the lower bidiagonal matrix B; the
    elements below the first subdiagonal, with the array TAUQ,
    represent the orthogonal matrix Q as a product of
    elementary reflectors, and the elements above the diagonal,
    with the array TAUP, represent the orthogonal matrix P as
    a product of elementary reflectors.
  See Further Details.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

D
  D is DOUBLE PRECISION array, dimension (min(M,N))
  The diagonal elements of the bidiagonal matrix B: D(i) = A(i,i).

E
  E is DOUBLE PRECISION array, dimension (min(M,N)-1)
  The off-diagonal elements of the bidiagonal matrix B:
  if m >= n, E(i) = A(i,i+1) for i = 1,2,...,n-1;
  if m < n, E(i) = A(i+1,i) for i = 1,2,...,m-1.

TAUQ
  TAUQ is DOUBLE PRECISION array dimension (min(M,N))
  The scalar factors of the elementary reflectors which
  represent the orthogonal matrix Q. See Further Details.

TAUP
  TAUP is DOUBLE PRECISION array, dimension (min(M,N))
  The scalar factors of the elementary reflectors which
  represent the orthogonal matrix P. See Further Details.

WORK
  WORK is DOUBLE PRECISION array, dimension (max(M,N))

INFO
  INFO is INTEGER
  = 0: successful exit.
  < 0: if INFO = -i, the i-th argument had an illegal value.

Further Details:
  The matrices Q and P are represented as products of elementary
  reflectors:
```

```
If m >= n,

            Q = H(1) H(2) . . . H(n)  and  P = G(1) G(2) . . . G(n-1)

Each H(i) and G(i) has the form:

            H(i) = I - tauq * v * v**T  and G(i) = I - taup * u * u**T

where tauq and taup are real scalars, and v and u are real vectors;
v(1:i-1) = 0, v(i) = 1, and v(i+1:m) is stored on exit in A(i+1:m,i);
u(1:i) = 0, u(i+1) = 1, and u(i+2:n) is stored on exit in A(i,i+2:n);
tauq is stored in TAUQ(i) and taup in TAUP(i).

If m < n,

            Q = H(1) H(2) . . . H(m-1)  and  P = G(1) G(2) . . . G(m)

Each H(i) and G(i) has the form:

            H(i) = I - tauq * v * v**T  and G(i) = I - taup * u * u**T

where tauq and taup are real scalars, and v and u are real vectors;
v(1:i) = 0, v(i+1) = 1, and v(i+2:m) is stored on exit in A(i+2:m,i);
u(1:i-1) = 0, u(i) = 1, and u(i+1:n) is stored on exit in A(i,i+1:n);
tauq is stored in TAUQ(i) and taup in TAUP(i).

The contents of A on exit are illustrated by the following examples:

          m = 6 and n = 5 (m > n):          m = 5 and n = 6 (m < n):

            (  d   e   u1  u1  u1 )           (  d   u1  u1  u1  u1  u1 )
            (  v1  d   e   u2  u2 )           (  e   d   u2  u2  u2  u2 )
            (  v1  v2  d   e   u3 )           (  v1  e   d   u3  u3  u3 )
            (  v1  v2  v3  d   e  )           (  v1  v2  e   d   u4  u4 )
            (  v1  v2  v3  v4  d  )           (  v1  v2  v3  e   d   u5 )
            (  v1  v2  v3  v4  v5 )

where d and e denote diagonal and off-diagonal elements of B, vi
denotes an element of the vector defining H(i), and ui an element of
the vector defining G(i).
```

———-

32.3.3 fortran code

— dgebd2.f —

```
*  =====================================================================
      SUBROUTINE DGEBD2( M, N, A, LDA, D, E, TAUQ, TAUP, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), D( * ), E( * ), TAUP( * ),
     $                   TAUQ( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARF, DLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.LT.0 ) THEN
         CALL XERBLA( 'DGEBD2', -INFO )
         RETURN
      END IF
*
      IF( M.GE.N ) THEN
*
```

```
*        Reduce to upper bidiagonal form
*
         DO 10 I = 1, N
*
*           Generate elementary reflector H(i) to annihilate A(i+1:m,i)
*
            CALL DLARFG( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
     $                   TAUQ( I ) )
            D( I ) = A( I, I )
            A( I, I ) = ONE
*
*           Apply H(i) to A(i:m,i+1:n) from the left
*
            IF( I.LT.N )
     $         CALL DLARF( 'Left', M-I+1, N-I, A( I, I ), 1, TAUQ( I ),
     $                     A( I, I+1 ), LDA, WORK )
            A( I, I ) = D( I )
*
            IF( I.LT.N ) THEN
*
*              Generate elementary reflector G(i) to annihilate
*              A(i,i+2:n)
*
               CALL DLARFG( N-I, A( I, I+1 ), A( I, MIN( I+2, N ) ),
     $                      LDA, TAUP( I ) )
               E( I ) = A( I, I+1 )
               A( I, I+1 ) = ONE
*
*              Apply G(i) to A(i+1:m,i+1:n) from the right
*
               CALL DLARF( 'Right', M-I, N-I, A( I, I+1 ), LDA,
     $                     TAUP( I ), A( I+1, I+1 ), LDA, WORK )
               A( I, I+1 ) = E( I )
            ELSE
               TAUP( I ) = ZERO
            END IF
   10    CONTINUE
      ELSE
*
*        Reduce to lower bidiagonal form
*
         DO 20 I = 1, M
*
*           Generate elementary reflector G(i) to annihilate A(i,i+1:n)
*
            CALL DLARFG( N-I+1, A( I, I ), A( I, MIN( I+1, N ) ), LDA,
     $                   TAUP( I ) )
            D( I ) = A( I, I )
            A( I, I ) = ONE
*
```

```
*           Apply G(i) to A(i+1:m,i:n) from the right
*
            IF( I.LT.M )
     $         CALL DLARF( 'Right', M-I, N-I+1, A( I, I ), LDA,
     $                     TAUP( I ), A( I+1, I ), LDA, WORK )
            A( I, I ) = D( I )
*
            IF( I.LT.M ) THEN
*
*              Generate elementary reflector H(i) to annihilate
*              A(i+2:m,i)
*
               CALL DLARFG( M-I, A( I+1, I ), A( MIN( I+2, M ), I ), 1,
     $                      TAUQ( I ) )
               E( I ) = A( I+1, I )
               A( I+1, I ) = ONE
*
*              Apply H(i) to A(i+1:m,i+1:n) from the left
*
               CALL DLARF( 'Left', M-I, N-I, A( I+1, I ), 1, TAUQ( I ),
     $                     A( I+1, I+1 ), LDA, WORK )
               A( I+1, I ) = E( I )
            ELSE
               TAUQ( I ) = ZERO
            END IF
   20    CONTINUE
      END IF
      RETURN
*
*     End of DGEBD2
*
      END
```

———-

32.3.4 lisp code

— dgebd2.lisp —

———-

32.4 dgebrd

32.4.1 Axiom unit tests

— **dgebrd.input** —

```
)set break resume
)sys rm -f dgebrd.output
)spool dgebrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.4.2 Axiom help page

— **dgebrd.help** —

```
======================================================================
dgebrd examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgebrd (M, N, A, LDA, D, E, TAUQ, TAUP, WORK, LWORK, INFO)
           DGEBRD

Function/Subroutine Documentation
   subroutine dgebrd (integer M, integer N, double precision, dimension( lda,
       * ) A, integer LDA, double precision, dimension( * ) D, double
       precision, dimension( * ) E, double precision, dimension( * ) TAUQ,
       double precision, dimension( * ) TAUP, double precision, dimension( * )
       WORK, integer LWORK, integer INFO)
       DGEBRD

Purpose:
   DGEBRD reduces a general real M-by-N matrix A to upper or lower
   bidiagonal form B by an orthogonal transformation: Q**T * A * P = B.
```

```
   If m >= n, B is upper bidiagonal; if m < n, B is lower bidiagonal.

Parameters:

 M
   M is INTEGER
   The number of rows in the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns in the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the M-by-N general matrix to be reduced.
   On exit,
   if m >= n, the diagonal and the first superdiagonal are
     overwritten with the upper bidiagonal matrix B; the
     elements below the diagonal, with the array TAUQ, represent
     the orthogonal matrix Q as a product of elementary
     reflectors, and the elements above the first superdiagonal,
     with the array TAUP, represent the orthogonal matrix P as
     a product of elementary reflectors;
   if m < n, the diagonal and the first subdiagonal are
     overwritten with the lower bidiagonal matrix B; the
     elements below the first subdiagonal, with the array TAUQ,
     represent the orthogonal matrix Q as a product of
     elementary reflectors, and the elements above the diagonal,
     with the array TAUP, represent the orthogonal matrix P as
     a product of elementary reflectors.
   See Further Details.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 D
   D is DOUBLE PRECISION array, dimension (min(M,N))
   The diagonal elements of the bidiagonal matrix B: D(i) = A(i,i).

 E
   E is DOUBLE PRECISION array, dimension (min(M,N)-1)
   The off-diagonal elements of the bidiagonal matrix B:
   if m >= n, E(i) = A(i,i+1) for i = 1,2,...,n-1;
   if m < n, E(i) = A(i+1,i) for i = 1,2,...,m-1.

 TAUQ
   TAUQ is DOUBLE PRECISION array dimension (min(M,N))
   The scalar factors of the elementary reflectors which
```

```
  represent the orthogonal matrix Q. See Further Details.

 TAUP
   TAUP is DOUBLE PRECISION array, dimension (min(M,N))
   The scalar factors of the elementary reflectors which
   represent the orthogonal matrix P. See Further Details.

 WORK
   WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The length of the array WORK.  LWORK >= max(1,M,N).
   For optimum performance LWORK >= (M+N)*NB, where NB
   is the optimal blocksize.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value.

Further Details:
   The matrices Q and P are represented as products of elementary
   reflectors:

   If m >= n,

                Q = H(1) H(2) . . . H(n)  and  P = G(1) G(2) . . . G(n-1)

   Each H(i) and G(i) has the form:

                H(i) = I - tauq * v * v**T  and G(i) = I - taup * u * u**T

   where tauq and taup are real scalars, and v and u are real vectors;
   v(1:i-1) = 0, v(i) = 1, and v(i+1:m) is stored on exit in A(i+1:m,i);
   u(1:i) = 0, u(i+1) = 1, and u(i+2:n) is stored on exit in A(i,i+2:n);
   tauq is stored in TAUQ(i) and taup in TAUP(i).

   If m < n,

                Q = H(1) H(2) . . . H(m-1)  and  P = G(1) G(2) . . . G(m)

   Each H(i) and G(i) has the form:
```

```
              H(i) = I - tauq * v * v**T  and G(i) = I - taup * u * u**T

   where tauq and taup are real scalars, and v and u are real vectors;
   v(1:i) = 0, v(i+1) = 1, and v(i+2:m) is stored on exit in A(i+2:m,i);
   u(1:i-1) = 0, u(i) = 1, and u(i+1:n) is stored on exit in A(i,i+1:n);
   tauq is stored in TAUQ(i) and taup in TAUP(i).

   The contents of A on exit are illustrated by the following examples:

            m = 6 and n = 5 (m > n):          m = 5 and n = 6 (m < n):

              (  d   e   u1  u1  u1 )           (  d   u1  u1  u1  u1  u1 )
              (  v1  d   e   u2  u2 )           (  e   d   u2  u2  u2  u2 )
              (  v1  v2  d   e   u3 )           (  v1  e   d   u3  u3  u3 )
              (  v1  v2  v3  d   e  )           (  v1  v2  e   d   u4  u4 )
              (  v1  v2  v3  v4  d  )           (  v1  v2  v3  e   d   u5 )
              (  v1  v2  v3  v4  v5 )

   where d and e denote diagonal and off-diagonal elements of B, vi
   denotes an element of the vector defining H(i), and ui an element of
   the vector defining G(i).
```

32.4.3 fortran code

— dgebrd.f —

```
*  =====================================================================
      SUBROUTINE DGEBRD( M, N, A, LDA, D, E, TAUQ, TAUP, WORK, LWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), D( * ), E( * ), TAUP( * ),
     $                   TAUQ( * ), WORK( * )
*     ..
*
*  =====================================================================
*
```

```
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IINFO, J, LDWRKX, LDWRKY, LWKOPT, MINMN, NB,
     $                   NBMIN, NX
      DOUBLE PRECISION   WS
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEBD2, DGEMM, DLABRD, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      NB = MAX( 1, ILAENV( 1, 'DGEBRD', ' ', M, N, -1, -1 ) )
      LWKOPT = ( M+N )*NB
      WORK( 1 ) = DBLE( LWKOPT )
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, M, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -10
      END IF
      IF( INFO.LT.0 ) THEN
         CALL XERBLA( 'DGEBRD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      MINMN = MIN( M, N )
      IF( MINMN.EQ.0 ) THEN
         WORK( 1 ) = 1
```

```
         RETURN
      END IF
*
      WS = MAX( M, N )
      LDWRKX = M
      LDWRKY = N
*
      IF( NB.GT.1 .AND. NB.LT.MINMN ) THEN
*
*        Set the crossover point NX.
*
         NX = MAX( NB, ILAENV( 3, 'DGEBRD', ' ', M, N, -1, -1 ) )
*
*        Determine when to switch from blocked to unblocked code.
*
         IF( NX.LT.MINMN ) THEN
            WS = ( M+N )*NB
            IF( LWORK.LT.WS ) THEN
*
*              Not enough work space for the optimal NB, consider using
*              a smaller block size.
*
               NBMIN = ILAENV( 2, 'DGEBRD', ' ', M, N, -1, -1 )
               IF( LWORK.GE.( M+N )*NBMIN ) THEN
                  NB = LWORK / ( M+N )
               ELSE
                  NB = 1
                  NX = MINMN
               END IF
            END IF
         END IF
      ELSE
         NX = MINMN
      END IF
*
      DO 30 I = 1, MINMN - NX, NB
*
*        Reduce rows and columns i:i+nb-1 to bidiagonal form and return
*        the matrices X and Y which are needed to update the unreduced
*        part of the matrix
*
         CALL DLABRD( M-I+1, N-I+1, NB, A( I, I ), LDA, D( I ), E( I ),
     $                TAUQ( I ), TAUP( I ), WORK, LDWRKX,
     $                WORK( LDWRKX*NB+1 ), LDWRKY )
*
*        Update the trailing submatrix A(i+nb:m,i+nb:n), using an update
*        of the form  A := A - V*Y**T - X*U**T
*
         CALL DGEMM( 'No transpose', 'Transpose', M-I-NB+1, N-I-NB+1,
     $               NB, -ONE, A( I+NB, I ), LDA,
```

```
     $               WORK( LDWRKX*NB+NB+1 ), LDWRKY, ONE,
     $               A( I+NB, I+NB ), LDA )
         CALL DGEMM( 'No transpose', 'No transpose', M-I-NB+1, N-I-NB+1,
     $               NB, -ONE, WORK( NB+1 ), LDWRKX, A( I, I+NB ), LDA,
     $               ONE, A( I+NB, I+NB ), LDA )
*
*        Copy diagonal and off-diagonal elements of B back into A
*
         IF( M.GE.N ) THEN
            DO 10 J = I, I + NB - 1
               A( J, J ) = D( J )
               A( J, J+1 ) = E( J )
   10       CONTINUE
         ELSE
            DO 20 J = I, I + NB - 1
               A( J, J ) = D( J )
               A( J+1, J ) = E( J )
   20       CONTINUE
         END IF
   30 CONTINUE
*
*     Use unblocked code to reduce the remainder of the matrix
*
      CALL DGEBD2( M-I+1, N-I+1, A( I, I ), LDA, D( I ), E( I ),
     $             TAUQ( I ), TAUP( I ), WORK, IINFO )
      WORK( 1 ) = WS
      RETURN
*
*     End of DGEBRD
*
      END
```

———-

32.4.4 lisp code

— dgebrd.lisp —

———-

32.5 dgecon

32.5.1 Axiom unit tests

— **dgecon.input** —

```
)set break resume
)sys rm -f dgecon.output
)spool dgecon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.5.2 Axiom help page

— **dgecon.help** —

```
======================================================================
dgecon examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgecon (NORM, N, A, LDA, ANORM, RCOND, WORK, IWORK, INFO)
           DGECON

Function/Subroutine Documentation
   subroutine dgecon (character NORM, integer N, double precision, dimension(
       lda, * ) A, integer LDA, double precision ANORM, double precision
       RCOND, double precision, dimension( * ) WORK, integer, dimension( * )
       IWORK, integer INFO)
       DGECON

Purpose:
   DGECON estimates the reciprocal of the condition number of a general
   real matrix A, in either the 1-norm or the infinity-norm, using
   the LU factorization computed by DGETRF.
```

```
  An estimate is obtained for norm(inv(A)), and the reciprocal of the
  condition number is computed as
     RCOND = 1 / ( norm(A) * norm(inv(A)) ).

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies whether the 1-norm condition number or the
   infinity-norm condition number is required:
   = '1' or 'O':  1-norm;
   = 'I':         Infinity-norm.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   The factors L and U from the factorization A = P*L*U
   as computed by DGETRF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 ANORM
   ANORM is DOUBLE PRECISION
   If NORM = '1' or 'O', the 1-norm of the original matrix A.
   If NORM = 'I', the infinity-norm of the original matrix A.

 RCOND
   RCOND is DOUBLE PRECISION
   The reciprocal of the condition number of the matrix A,
   computed as RCOND = 1/(norm(A) * norm(inv(A))).

 WORK
   WORK is DOUBLE PRECISION array, dimension (4*N)

 IWORK
   IWORK is INTEGER array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

32.5.3 fortran code

— dgecon.f —

```
*  =====================================================================
      SUBROUTINE DGECON( NORM, N, A, LDA, ANORM, RCOND, WORK, IWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            INFO, LDA, N
      DOUBLE PRECISION   ANORM, RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ONENRM
      CHARACTER          NORMIN
      INTEGER            IX, KASE, KASE1
      DOUBLE PRECISION   AINVNM, SCALE, SL, SMLNUM, SU
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, IDAMAX, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLACN2, DLATRS, DRSCL, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
```

```
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      ONENRM = NORM.EQ.'1' .OR. LSAME( NORM, 'O' )
      IF( .NOT.ONENRM .AND. .NOT.LSAME( NORM, 'I' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -5
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGECON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
      SMLNUM = DLAMCH( 'Safe minimum' )
*
*     Estimate the norm of inv(A).
*
      AINVNM = ZERO
      NORMIN = 'N'
      IF( ONENRM ) THEN
         KASE1 = 1
      ELSE
         KASE1 = 2
      END IF
      KASE = 0
   10 CONTINUE
      CALL DLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.KASE1 ) THEN
*
*           Multiply by inv(L).
*
```

```
            CALL DLATRS( 'Lower', 'No transpose', 'Unit', NORMIN, N, A,
     $                   LDA, WORK, SL, WORK( 2*N+1 ), INFO )
*
*           Multiply by inv(U).
*
            CALL DLATRS( 'Upper', 'No transpose', 'Non-unit', NORMIN, N,
     $                   A, LDA, WORK, SU, WORK( 3*N+1 ), INFO )
         ELSE
*
*           Multiply by inv(U**T).
*
            CALL DLATRS( 'Upper', 'Transpose', 'Non-unit', NORMIN, N, A,
     $                   LDA, WORK, SU, WORK( 3*N+1 ), INFO )
*
*           Multiply by inv(L**T).
*
            CALL DLATRS( 'Lower', 'Transpose', 'Unit', NORMIN, N, A,
     $                   LDA, WORK, SL, WORK( 2*N+1 ), INFO )
         END IF
*
*        Divide X by 1/(SL*SU) if doing so will not cause overflow.
*
         SCALE = SL*SU
         NORMIN = 'Y'
         IF( SCALE.NE.ONE ) THEN
            IX = IDAMAX( N, WORK, 1 )
            IF( SCALE.LT.ABS( WORK( IX ) )*SMLNUM .OR. SCALE.EQ.ZERO )
     $         GO TO 20
            CALL DRSCL( N, SCALE, WORK, 1 )
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
   20 CONTINUE
      RETURN
*
*     End of DGECON
*
      END
```

———-

32.5.4 lisp code

— dgecon.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dgecon (norm n a lda anorm rcond work iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (double-float) rcond anorm)
             (type (array double-float (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (a double-float a-%data% a-%offset%)
         (work double-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (ainvnm 0.0d0) (scale 0.0d0) (sl 0.0d0) (smlnum 0.0d0) (su 0.0d0)
             (ix 0) (kase 0) (kase1 0)
             (normin
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (onenrm nil))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (double-float) ainvnm scale sl smlnum su)
                 (type (f2cl-lib:integer4) ix kase kase1)
                 (type (string 1) normin)
                 (type f2cl-lib:logical onenrm))
        (setf info 0)
        (setf onenrm
                (or (f2cl-lib:fstring-= norm "1")
                    (multiple-value-bind (ret-val var-0 var-1)
                        (lsame norm "O")
                      (declare (ignore var-1))
                      (when var-0 (setf norm var-0))
                      ret-val)))
        (cond
         ((and (not onenrm)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame norm "I")
                  (declare (ignore var-1))
                  (when var-0 (setf norm var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
```

```
    (setf info -4))
   ((< anorm zero) (setf info -5)))
  (cond
   ((/= info 0) (xerbla "DGECON" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf rcond zero)
  (cond ((= n 0) (setf rcond one) (go end_label))
        ((= anorm zero) (go end_label)))
  (setf smlnum (dlamch "Safe minimum"))
  (setf ainvnm zero)
  (f2cl-lib:f2cl-set-string normin "N" (string 1))
  (cond (onenrm (setf kase1 1)) (t (setf kase1 2)))
  (setf kase 0)
 label10
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacn2 n
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1)) ((1 *))
                             work-%offset%)
       work iwork ainvnm kase isave)
    (declare (ignore var-1 var-2 var-3 var-6))
    (when var-0 (setf n var-0))
    (when var-4 (setf ainvnm var-4))
    (when var-5 (setf kase var-5)))
  (cond
   ((/= kase 0)
    (cond
     ((= kase kase1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (dlatrs "Lower" "No transpose" "Unit" normin n a lda work sl
           (f2cl-lib:array-slice work-%data% double-float
                                 ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                                 work-%offset%)
           info)
        (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
        (when var-3 (setf normin var-3))
        (when var-4 (setf n var-4))
        (when var-6 (setf lda var-6))
        (when var-8 (setf sl var-8))
        (when var-10 (setf info var-10)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (dlatrs "Upper" "No transpose" "Non-unit" normin n a lda work
           su
           (f2cl-lib:array-slice work-%data% double-float
                                 ((+ (f2cl-lib:int-mul 3 n) 1)) ((1 *))
                                 work-%offset%)
           info)
```

```
      (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
      (when var-3 (setf normin var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf su var-8))
      (when var-10 (setf info var-10))))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (dlatrs "Upper" "Transpose" "Non-unit" normin n a lda work su
          (f2cl-lib:array-slice work-%data% double-float
                                ((+ (f2cl-lib:int-mul 3 n) 1)) ((1 *))
                                work-%offset%)
          info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
      (when var-3 (setf normin var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf su var-8))
      (when var-10 (setf info var-10)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (dlatrs "Lower" "Transpose" "Unit" normin n a lda work sl
          (f2cl-lib:array-slice work-%data% double-float
                                ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                                work-%offset%)
          info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
      (when var-3 (setf normin var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf sl var-8))
      (when var-10 (setf info var-10)))))
  (setf scale (* sl su))
  (f2cl-lib:f2cl-set-string normin "Y" (string 1))
  (cond
   ((/= scale one)
    (setf ix
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (idamax n work 1)
              (declare (ignore var-1 var-2))
              (when var-0 (setf n var-0))
              ret-val))
    (if (or
         (< scale
            (*
             (abs
              (f2cl-lib:fref work-%data% (ix) ((1 *)) work-%offset%))
```

```
                    smlnum))
                 (= scale zero))
                (go label20))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (drscl n scale work 1)
              (declare (ignore var-2 var-3))
              (when var-0 (setf n var-0))
              (when var-1 (setf scale var-1)))))
          (go label10)))
        (if (/= ainvnm zero)
            (setf rcond (/ (/ one ainvnm) anorm)))
       label20
        (go end_label)
       end_label
        (return (values norm n nil lda nil rcond nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgecon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::norm
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::rcond nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

32.6 dgeequ

32.6.1 Axiom unit tests

— dgeequ.input —

```
)set break resume
)sys rm -f dgeequ.output
)spool dgeequ.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.6.2 Axiom help page

— dgeequ.help —

```
======================================================================
dgeequ examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgeequ (M, N, A, LDA, R, C, ROWCND, COLCND, AMAX, INFO)
           DGEEQU

Function/Subroutine Documentation
   subroutine dgeequ (integer M, integer N, double precision, dimension( lda,
       * ) A, integer LDA, double precision, dimension( * ) R, double
       precision, dimension( * ) C, double precision ROWCND, double precision
       COLCND, double precision AMAX, integer INFO)
       DGEEQU

Purpose:
   DGEEQU computes row and column scalings intended to equilibrate an
   M-by-N matrix A and reduce its condition number.  R returns the row
   scale factors and C the column scale factors, chosen to try to make
   the largest element in each row and column of the matrix B with
   elements B(i,j)=R(i)*A(i,j)*C(j) have absolute value 1.

   R(i) and C(j) are restricted to be between SMLNUM = smallest safe
   number and BIGNUM = largest safe number.  Use of these scaling
   factors is not guaranteed to reduce the condition number of A but
```

```
   works well in practice.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   The M-by-N matrix whose equilibration factors are to be computed.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 R
   R is DOUBLE PRECISION array, dimension (M)
   If INFO = 0 or INFO > M, R contains the row scale factors for A.

 C
   C is DOUBLE PRECISION array, dimension (N)
   If INFO = 0,  C contains the column scale factors for A.

 ROWCND
   ROWCND is DOUBLE PRECISION
   If INFO = 0 or INFO > M, ROWCND contains the ratio of the
   smallest R(i) to the largest R(i).  If ROWCND >= 0.1 and
   AMAX is neither too large nor too small, it is not worth scaling by R.

 COLCND
   COLCND is DOUBLE PRECISION
   If INFO = 0, COLCND contains the ratio of the smallest
   C(i) to the largest C(i).  If COLCND >= 0.1, it is not
   worth scaling by C.

 AMAX
   AMAX is DOUBLE PRECISION
   Absolute value of largest matrix element.  If AMAX is very
   close to overflow or very close to underflow, the matrix should be scaled.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i,  and i is
```

```
     <= M:  the i-th row of A is exactly zero
     >  M:  the (i-M)-th column of A is exactly zero
```

———-

32.6.3 fortran code

— **dgeequ.f** —

```
*  =====================================================================
      SUBROUTINE DGEEQU( M, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
      DOUBLE PRECISION   AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), C( * ), R( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   BIGNUM, RCMAX, RCMIN, SMLNUM
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
```

```
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEEQU', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         ROWCND = ONE
         COLCND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Get machine constants.
*
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
*
*     Compute row scale factors.
*
      DO 10 I = 1, M
         R( I ) = ZERO
   10 CONTINUE
*
*     Find the maximum element in each row.
*
      DO 30 J = 1, N
         DO 20 I = 1, M
            R( I ) = MAX( R( I ), ABS( A( I, J ) ) )
   20    CONTINUE
   30 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 40 I = 1, M
         RCMAX = MAX( RCMAX, R( I ) )
```

```
         RCMIN = MIN( RCMIN, R( I ) )
   40 CONTINUE
      AMAX = RCMAX
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 50 I = 1, M
            IF( R( I ).EQ.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   50    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 60 I = 1, M
            R( I ) = ONE / MIN( MAX( R( I ), SMLNUM ), BIGNUM )
   60    CONTINUE
*
*        Compute ROWCND = min(R(I)) / max(R(I))
*
         ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
*     Compute column scale factors
*
      DO 70 J = 1, N
         C( J ) = ZERO
   70 CONTINUE
*
*     Find the maximum element in each column,
*     assuming the row scaling computed above.
*
      DO 90 J = 1, N
         DO 80 I = 1, M
            C( J ) = MAX( C( J ), ABS( A( I, J ) )*R( I ) )
   80    CONTINUE
   90 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 100 J = 1, N
         RCMIN = MIN( RCMIN, C( J ) )
         RCMAX = MAX( RCMAX, C( J ) )
  100 CONTINUE
```

```
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 110 J = 1, N
            IF( C( J ).EQ.ZERO ) THEN
               INFO = M + J
               RETURN
            END IF
  110    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 120 J = 1, N
            C( J ) = ONE / MIN( MAX( C( J ), SMLNUM ), BIGNUM )
  120    CONTINUE
*
*        Compute COLCND = min(C(J)) / max(C(J))
*
         COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
      RETURN
*
*     End of DGEEQU
*
      END
```

32.6.4 lisp code

— dgeequ.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dgeequ (m n a lda r c rowcnd colcnd amax info)
    (declare (type (double-float) amax colcnd rowcnd)
             (type (array double-float (*)) c r a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (r double-float r-%data% r-%offset%)
         (c double-float c-%data% c-%offset%))
```

```
(prog ((bignum 0.0d0) (rcmax 0.0d0) (rcmin 0.0d0) (smlnum 0.0d0) (i 0)
       (j 0))
  (declare (type (double-float) bignum rcmax rcmin smlnum)
           (type (f2cl-lib:integer4) i j))
  (setf info 0)
  (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
        ((< lda
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -4)))
  (cond
   ((/= info 0) (xerbla "DGEEQU" (f2cl-lib:int-sub info))
    (go end_label)))
  (cond
   ((or (= m 0) (= n 0)) (setf rowcnd one) (setf colcnd one)
    (setf amax zero) (go end_label)))
  (setf smlnum (dlamch "S"))
  (setf bignum (/ one smlnum))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%) zero)
     label10))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                  (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                       (abs
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))))
         label20))
     label30))
  (setf rcmin bignum)
  (setf rcmax zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf rcmax
              (max rcmax
                   (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
      (setf rcmin
              (min rcmin
                   (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
     label40))
  (setf amax rcmax)
  (cond
   ((= rcmin zero)
```

```
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref r (i) ((1 *))) zero) (setf info i)
        (go end_label)))
     label50)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
              (/ one
                 (min
                  (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                       smlnum)
                  bignum)))
     label60))
  (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) zero)
   label70))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                     (*
                      (abs
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%))
                      (f2cl-lib:fref r-%data% (i) ((1 *))
                                     r-%offset%))))
       label80))
   label90))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf rcmin
            (min rcmin
                 (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
    (setf rcmax
            (max rcmax
```

```
                        (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
           label100))
         (cond
          ((= rcmin zero)
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (cond
                ((= (f2cl-lib:fref c (j) ((1 *))) zero)
                 (setf info (f2cl-lib:int-add m j)) (go end_label)))
              label110)))
          (t
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                       (/ one
                          (min
                           (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                                smlnum)
                           bignum)))
              label120))
           (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
         (go end_label)
        end_label
         (return (values nil nil nil nil nil nil rowcnd colcnd amax info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgeequ
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (double-float) (double-float)
                                              (double-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil
                                              fortran-to-lisp::rowcnd
                                              fortran-to-lisp::colcnd
                                              fortran-to-lisp::amax
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

32.7 dgeequb

32.7.1 Axiom unit tests

— **dgeequb.input** —

```
)set break resume
)sys rm -f dgeequb.output
)spool dgeequb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.7.2 Axiom help page

— **dgeequb.help** —

```
======================================================================
dgeequb examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgeequb (M, N, A, LDA, R, C, ROWCND, COLCND, AMAX, INFO)
           DGEEQUB

Function/Subroutine Documentation
   subroutine dgeequb (integer M, integer N, double precision, dimension( lda,
       * ) A, integer LDA, double precision, dimension( * ) R, double
       precision, dimension( * ) C, double precision ROWCND, double precision
       COLCND, double precision AMAX, integer INFO)
```

```
      DGEEQUB

Purpose:
   DGEEQUB computes row and column scalings intended to equilibrate an
   M-by-N matrix A and reduce its condition number.  R returns the row
   scale factors and C the column scale factors, chosen to try to make
   the largest element in each row and column of the matrix B with
   elements B(i,j)=R(i)*A(i,j)*C(j) have an absolute value of at most
   the radix.

   R(i) and C(j) are restricted to be a power of the radix between
   SMLNUM = smallest safe number and BIGNUM = largest safe number.  Use
   of these scaling factors is not guaranteed to reduce the condition
   number of A but works well in practice.

   This routine differs from DGEEQU by restricting the scaling factors
   to a power of the radix.  Baring over- and underflow, scaling by
   these factors introduces no additional rounding errors.  However, the
   scaled entries' magnitured are no longer approximately 1 but lie
   between sqrt(radix) and 1/sqrt(radix).

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   The M-by-N matrix whose equilibration factors are to be computed.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 R
   R is DOUBLE PRECISION array, dimension (M)
   If INFO = 0 or INFO > M, R contains the row scale factors for A.

 C
   C is DOUBLE PRECISION array, dimension (N)
   If INFO = 0,  C contains the column scale factors for A.

 ROWCND
   ROWCND is DOUBLE PRECISION
   If INFO = 0 or INFO > M, ROWCND contains the ratio of the
```

```
  smallest R(i) to the largest R(i).  If ROWCND >= 0.1 and
  AMAX is neither too large nor too small, it is not worth scaling by R.

  COLCND
  COLCND is DOUBLE PRECISION
  If INFO = 0, COLCND contains the ratio of the smallest
  C(i) to the largest C(i).  If COLCND >= 0.1, it is not worth scaling by C.

 AMAX
  AMAX is DOUBLE PRECISION
  Absolute value of largest matrix element.  If AMAX is very
  close to overflow or very close to underflow, the matrix should be scaled.

 INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i,  and i is
    <= M:  the i-th row of A is exactly zero
    >  M:  the (i-M)-th column of A is exactly zero
```

32.7.3 fortran code

— dgeequb.f —

```
*  ====================================================================
      SUBROUTINE DGEEQUB( M, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $                    INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
      DOUBLE PRECISION   AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), C( * ), R( * )
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
```

```
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   BIGNUM, RCMAX, RCMIN, SMLNUM, RADIX, LOGRDX
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, LOG
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEEQUB', -INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         ROWCND = ONE
         COLCND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Get machine constants.  Assume SMLNUM is a power of the radix.
*
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      RADIX = DLAMCH( 'B' )
      LOGRDX = LOG( RADIX )
*
```

```
*     Compute row scale factors.
*
      DO 10 I = 1, M
         R( I ) = ZERO
   10 CONTINUE
*
*     Find the maximum element in each row.
*
      DO 30 J = 1, N
         DO 20 I = 1, M
            R( I ) = MAX( R( I ), ABS( A( I, J ) ) )
   20    CONTINUE
   30 CONTINUE
      DO I = 1, M
         IF( R( I ).GT.ZERO ) THEN
            R( I ) = RADIX**INT( LOG( R( I ) ) / LOGRDX )
         END IF
      END DO
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 40 I = 1, M
         RCMAX = MAX( RCMAX, R( I ) )
         RCMIN = MIN( RCMIN, R( I ) )
   40 CONTINUE
      AMAX = RCMAX
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 50 I = 1, M
            IF( R( I ).EQ.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   50    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 60 I = 1, M
            R( I ) = ONE / MIN( MAX( R( I ), SMLNUM ), BIGNUM )
   60    CONTINUE
*
*        Compute ROWCND = min(R(I)) / max(R(I)).
*
         ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
```

```
      END IF
*
*     Compute column scale factors
*
      DO 70 J = 1, N
         C( J ) = ZERO
   70 CONTINUE
*
*     Find the maximum element in each column,
*     assuming the row scaling computed above.
*
      DO 90 J = 1, N
         DO 80 I = 1, M
            C( J ) = MAX( C( J ), ABS( A( I, J ) )*R( I ) )
   80    CONTINUE
         IF( C( J ).GT.ZERO ) THEN
            C( J ) = RADIX**INT( LOG( C( J ) ) / LOGRDX )
         END IF
   90 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 100 J = 1, N
         RCMIN = MIN( RCMIN, C( J ) )
         RCMAX = MAX( RCMAX, C( J ) )
  100 CONTINUE
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 110 J = 1, N
            IF( C( J ).EQ.ZERO ) THEN
               INFO = M + J
               RETURN
            END IF
  110    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 120 J = 1, N
            C( J ) = ONE / MIN( MAX( C( J ), SMLNUM ), BIGNUM )
  120    CONTINUE
*
*        Compute COLCND = min(C(J)) / max(C(J)).
*
         COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
```

```
      END IF
*
      RETURN
*
*     End of DGEEQUB
*
      END
```

32.7.4 lisp code

— dgeequb.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dgeequb (m n a lda r c rowcnd colcnd amax info)
    (declare (type (double-float) amax colcnd rowcnd)
             (type (array double-float (*)) c r a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (r double-float r-%data% r-%offset%)
         (c double-float c-%data% c-%offset%))
      (prog ((bignum 0.0d0) (rcmax 0.0d0) (rcmin 0.0d0) (smlnum 0.0d0)
             (radix 0.0d0) (logrdx 0.0d0) (i 0) (j 0))
        (declare (type (double-float) bignum rcmax rcmin smlnum radix logrdx)
                 (type (f2cl-lib:integer4) i j))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "DGEEQUB" (f2cl-lib:int-sub info))
          (go end_label)))
        (cond
         ((or (= m 0) (= n 0)) (setf rowcnd one) (setf colcnd one)
          (setf amax zero) (go end_label)))
        (setf smlnum (dlamch "S"))
        (setf bignum (/ one smlnum))
        (setf radix (dlamch "B"))
        (setf logrdx (f2cl-lib:flog radix))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
```

```
    (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%) zero)
   label10))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                     (abs
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))))
       label20))
   label30))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref r (i) ((1 *))) zero)
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
              (expt radix
                    (f2cl-lib:int
                     (/
                      (f2cl-lib:flog
                       (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%))
                      logrdx))))))
   label100000))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf rcmax
            (max rcmax
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
    (setf rcmin
            (min rcmin
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
   label40))
(setf amax rcmax)
(cond
 ((= rcmin zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref r (i) ((1 *))) zero) (setf info i)
        (go end_label)))
     label50)))
```

```
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
              (/ one
                 (min
                  (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                       smlnum)
                  bignum)))
     label60))
  (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) zero)
   label70))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                     (*
                      (abs
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%))
                      (f2cl-lib:fref r-%data% (i) ((1 *))
                                     r-%offset%))))
       label80))
    (cond
     ((> (f2cl-lib:fref c (j) ((1 *))) zero)
      (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
              (expt radix
                    (f2cl-lib:int
                     (/
                      (f2cl-lib:flog
                       (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
                      logrdx))))))
   label90))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf rcmin
            (min rcmin
                 (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
```

```
            (setf rcmax
                    (max rcmax
                         (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
           label100))
         (cond
          ((= rcmin zero)
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (cond
                ((= (f2cl-lib:fref c (j) ((1 *))) zero)
                 (setf info (f2cl-lib:int-add m j)) (go end_label)))
              label110)))
          (t
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                       (/ one
                          (min
                           (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                                smlnum)
                           bignum)))
              label120))
           (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
         (go end_label)
        end_label
         (return (values nil nil nil nil nil nil rowcnd colcnd amax info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgeequb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (double-float) (double-float)
                                              (double-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil
                                              fortran-to-lisp::rowcnd
                                              fortran-to-lisp::colcnd
                                              fortran-to-lisp::amax
                                              fortran-to-lisp::info)
```

```
                                        :calls 'nil)))
```

———-

32.8 dgehd2

32.8.1 Axiom unit tests

— **dgehd2.input** —

```
)set break resume
)sys rm -f dgehd2.output
)spool dgehd2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.8.2 Axiom help page

— **dgehd2.help** —

```
======================================================================
dgehd2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgehd2 (N, ILO, IHI, A, LDA, TAU, WORK, INFO)
           DGEHD2 reduces a general square matrix to upper Hessenberg form
           using an unblocked algorithm.

Function/Subroutine Documentation
   subroutine dgehd2 (integer N, integer ILO, integer IHI, double precision,
```

```
      dimension( lda, * ) A, integer LDA, double precision, dimension( * )
      TAU, double precision, dimension( * ) WORK, integer INFO)
      DGEHD2 reduces a general square matrix to upper Hessenberg form using
      an unblocked algorithm.

Purpose:
   DGEHD2 reduces a real general matrix A to upper Hessenberg form H by
   an orthogonal similarity transformation:  Q**T * A * Q = H .

 Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 ILO
   ILO is INTEGER

 IHI
   IHI is INTEGER

   It is assumed that A is already upper triangular in rows
   and columns 1:ILO-1 and IHI+1:N. ILO and IHI are normally
   set by a previous call to DGEBAL; otherwise they should be
   set to 1 and N respectively. See Further Details.
   1 <= ILO <= IHI <= max(1,N).

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the n by n general matrix to be reduced.
   On exit, the upper triangle and the first subdiagonal of A
   are overwritten with the upper Hessenberg matrix H, and the
   elements below the first subdiagonal, with the array TAU,
   represent the orthogonal matrix Q as a product of elementary
   reflectors. See Further Details.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 TAU
   TAU is DOUBLE PRECISION array, dimension (N-1)
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is DOUBLE PRECISION array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit.
```

```
   < 0:  if INFO = -i, the i-th argument had an illegal value.

Further Details:
   The matrix Q is represented as a product of (ihi-ilo) elementary
   reflectors

                Q = H(ilo) H(ilo+1) . . . H(ihi-1).

   Each H(i) has the form

                H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(1:i) = 0, v(i+1) = 1 and v(ihi+1:n) = 0; v(i+2:ihi) is stored on
   exit in A(i+2:ihi,i), and tau in TAU(i).

   The contents of A are illustrated by the following example, with
   n = 7, ilo = 2 and ihi = 6:

             on entry,                        on exit,

             ( a   a   a   a   a   a   a )    (  a   a   h   h   h   h   a )
             (     a   a   a   a   a   a )    (      a   h   h   h   h   a )
             (     a   a   a   a   a   a )    (      h   h   h   h   h   h )
             (     a   a   a   a   a   a )    (      v2  h   h   h   h   h )
             (     a   a   a   a   a   a )    (      v2  v3  h   h   h   h )
             (     a   a   a   a   a   a )    (      v2  v3  v4  h   h   h )
             (                       a )    (                          a )

   where a denotes an element of the original matrix A, h denotes a
   modified element of the upper Hessenberg matrix H, and vi denotes an
   element of the vector defining H(i).
```

32.8.3 fortran code

— dgehd2.f —

```
*  =====================================================================
      SUBROUTINE DGEHD2( N, ILO, IHI, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
```

```
*     .. Scalar Arguments ..
      INTEGER            IHI, ILO, INFO, LDA, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      DOUBLE PRECISION   AII
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARF, DLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -2
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEHD2', -INFO )
         RETURN
      END IF
*
      DO 10 I = ILO, IHI - 1
*
*        Compute elementary reflector H(i) to annihilate A(i+2:ihi,i)
*
         CALL DLARFG( IHI-I, A( I+1, I ), A( MIN( I+2, N ), I ), 1,
     $                TAU( I ) )
         AII = A( I+1, I )
         A( I+1, I ) = ONE
```

```
*
*       Apply H(i) to A(1:ihi,i+1:ihi) from the right
*
        CALL DLARF( 'Right', IHI, IHI-I, A( I+1, I ), 1, TAU( I ),
     $               A( 1, I+1 ), LDA, WORK )
*
*       Apply H(i) to A(i+1:ihi,i+1:n) from the left
*
        CALL DLARF( 'Left', IHI-I, N-I, A( I+1, I ), 1, TAU( I ),
     $               A( I+1, I+1 ), LDA, WORK )
*
        A( I+1, I ) = AII
   10 CONTINUE
*
      RETURN
*
*     End of DGEHD2
*
      END
```

———-

32.8.4 lisp code

— **dgehd2.lisp** —

———-

32.9 dgehrd

32.9.1 Axiom unit tests

— **dgehrd.input** —

```
)set break resume
)sys rm -f dgehrd.output
)spool dgehrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.9.2 Axiom help page

— dgehrd.help —

```
=====================================================================
dgehrd examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgehrd (N, ILO, IHI, A, LDA, TAU, WORK, LWORK, INFO)
           DGEHRD

Function/Subroutine Documentation
   subroutine dgehrd (integer N, integer ILO, integer IHI, double precision,
       dimension( lda, * ) A, integer LDA, double precision, dimension( * )
       TAU, double precision, dimension( * ) WORK, integer LWORK, integer
       INFO)
       DGEHRD

Purpose:
   DGEHRD reduces a real general matrix A to upper Hessenberg form H by
   an orthogonal similarity transformation:  Q**T * A * Q = H .

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 ILO
   ILO is INTEGER

 IHI
   IHI is INTEGER

   It is assumed that A is already upper triangular in rows
   and columns 1:ILO-1 and IHI+1:N. ILO and IHI are normally
   set by a previous call to DGEBAL; otherwise they should be
   set to 1 and N respectively. See Further Details.
   1 <= ILO <= IHI <= N, if N > 0; ILO=1 and IHI=0, if N=0.
```

```
 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the N-by-N general matrix to be reduced.
   On exit, the upper triangle and the first subdiagonal of A
   are overwritten with the upper Hessenberg matrix H, and the
   elements below the first subdiagonal, with the array TAU,
   represent the orthogonal matrix Q as a product of elementary
   reflectors. See Further Details.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 TAU
   TAU is DOUBLE PRECISION array, dimension (N-1)
   The scalar factors of the elementary reflectors (see Further
   Details). Elements 1:ILO-1 and IHI:N-1 of TAU are set to zero.

 WORK
   WORK is DOUBLE PRECISION array, dimension (LWORK)
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The length of the array WORK.  LWORK >= max(1,N).
   For good performance, LWORK should generally be larger.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value.

Further Details:
   The matrix Q is represented as a product of (ihi-ilo) elementary
   reflectors

                Q = H(ilo) H(ilo+1) . . . H(ihi-1).

   Each H(i) has the form

                H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(1:i) = 0, v(i+1) = 1 and v(ihi+1:n) = 0; v(i+2:ihi) is stored on
   exit in A(i+2:ihi,i), and tau in TAU(i).
```

```
The contents of A are illustrated by the following example, with
n = 7, ilo = 2 and ihi = 6:

          on entry,                        on exit,

          ( a   a   a   a   a   a   a )    (  a   a   h   h   h   h   a )
          (     a   a   a   a   a   a )    (      a   h   h   h   h   a )
          (     a   a   a   a   a   a )    (      h   h   h   h   h   h )
          (     a   a   a   a   a   a )    (      v2  h   h   h   h   h )
          (     a   a   a   a   a   a )    (      v2  v3  h   h   h   h )
          (     a   a   a   a   a   a )    (      v2  v3  v4  h   h   h )
          (                         a )    (                          a )

where a denotes an element of the original matrix A, h denotes a
modified element of the upper Hessenberg matrix H, and vi denotes an
element of the vector defining H(i).

This file is a slight modification of LAPACK-3.0's DGEHRD
subroutine incorporating improvements proposed by Quintana-Orti and
Van de Geijn (2006). (See DLAHR2.)
```

32.9.3 fortran code

— dgehrd.f —

```
*  =====================================================================
      SUBROUTINE DGEHRD( N, ILO, IHI, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            IHI, ILO, INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION  A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            NBMAX, LDT, TSIZE
```

```
      PARAMETER          ( NBMAX = 64, LDT = NBMAX+1,
     $                     TSIZE = LDT*NBMAX )
      DOUBLE PRECISION  ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0,
     $                     ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWT, J, LDWORK, LWKOPT, NB,
     $                   NBMIN, NH, NX
      DOUBLE PRECISION  EI
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DGEHD2, DGEMM, DLAHR2, DLARFB, DTRMM,
     $                   XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -2
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -8
      END IF
*
      IF( INFO.EQ.0 ) THEN
*
*        Compute the workspace requirements
*
         NB = MIN( NBMAX, ILAENV( 1, 'DGEHRD', ' ', N, ILO, IHI, -1 ) )
         LWKOPT = N*NB + TSIZE
         WORK( 1 ) = LWKOPT
      END IF
*
```

```
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEHRD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Set elements 1:ILO-1 and IHI:N-1 of TAU to zero
*
      DO 10 I = 1, ILO - 1
         TAU( I ) = ZERO
   10 CONTINUE
      DO 20 I = MAX( 1, IHI ), N - 1
         TAU( I ) = ZERO
   20 CONTINUE
*
*     Quick return if possible
*
      NH = IHI - ILO + 1
      IF( NH.LE.1 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
*     Determine the block size
*
      NB = MIN( NBMAX, ILAENV( 1, 'DGEHRD', ' ', N, ILO, IHI, -1 ) )
      NBMIN = 2
      IF( NB.GT.1 .AND. NB.LT.NH ) THEN
*
*        Determine when to cross over from blocked to unblocked code
*        (last block is always handled by unblocked code)
*
         NX = MAX( NB, ILAENV( 3, 'DGEHRD', ' ', N, ILO, IHI, -1 ) )
         IF( NX.LT.NH ) THEN
*
*           Determine if workspace is large enough for blocked code
*
            IF( LWORK.LT.N*NB+TSIZE ) THEN
*
*              Not enough workspace to use optimal NB:  determine the
*              minimum value of NB, and reduce NB or force use of
*              unblocked code
*
               NBMIN = MAX( 2, ILAENV( 2, 'DGEHRD', ' ', N, ILO, IHI,
     $                 -1 ) )
               IF( LWORK.GE.(N*NBMIN + TSIZE) ) THEN
                  NB = (LWORK-TSIZE) / N
               ELSE
                  NB = 1
```

```
            END IF
         END IF
      END IF
   END IF
   LDWORK = N
*
   IF( NB.LT.NBMIN .OR. NB.GE.NH ) THEN
*
*     Use unblocked code below
*
      I = ILO
*
   ELSE
*
*     Use blocked code
*
      IWT = 1 + N*NB
      DO 40 I = ILO, IHI - 1 - NX, NB
         IB = MIN( NB, IHI-I )
*
*        Reduce columns i:i+ib-1 to Hessenberg form, returning the
*        matrices V and T of the block reflector H = I - V*T*V**T
*        which performs the reduction, and also the matrix Y = A*V*T
*
         CALL DLAHR2( IHI, I, IB, A( 1, I ), LDA, TAU( I ),
     $                WORK( IWT ), LDT, WORK, LDWORK )
*
*        Apply the block reflector H to A(1:ihi,i+ib:ihi) from the
*        right, computing  A := A - Y * V**T. V(i+ib,ib-1) must be set
*        to 1
*
         EI = A( I+IB, I+IB-1 )
         A( I+IB, I+IB-1 ) = ONE
         CALL DGEMM( 'No transpose', 'Transpose',
     $               IHI, IHI-I-IB+1,
     $               IB, -ONE, WORK, LDWORK, A( I+IB, I ), LDA, ONE,
     $               A( 1, I+IB ), LDA )
         A( I+IB, I+IB-1 ) = EI
*
*        Apply the block reflector H to A(1:i,i+1:i+ib-1) from the
*        right
*
         CALL DTRMM( 'Right', 'Lower', 'Transpose',
     $               'Unit', I, IB-1,
     $               ONE, A( I+1, I ), LDA, WORK, LDWORK )
         DO 30 J = 0, IB-2
            CALL DAXPY( I, -ONE, WORK( LDWORK*J+1 ), 1,
     $                  A( 1, I+J+1 ), 1 )
   30    CONTINUE
*
```

```
*           Apply the block reflector H to A(i+1:ihi,i+ib:n) from the
*           left
*
            CALL DLARFB( 'Left', 'Transpose', 'Forward',
     $                   'Columnwise',
     $                   IHI-I, N-I-IB+1, IB, A( I+1, I ), LDA,
     $                   WORK( IWT ), LDT, A( I+1, I+IB ), LDA,
     $                   WORK, LDWORK )
   40    CONTINUE
      END IF
*
*     Use unblocked code to reduce the rest of the matrix
*
      CALL DGEHD2( N, I, IHI, A, LDA, TAU, WORK, IINFO )
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of DGEHRD
*
      END
```

———-

32.9.4 lisp code

— dgehrd.lisp —

———-

32.10 dgelq2

32.10.1 Axiom unit tests

— dgelq2.input —

```
)set break resume
)sys rm -f dgelq2.output
)spool dgelq2.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

32.10.2 Axiom help page

— **dgelq2.help** —

```
==================================================================
dgelq2 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgelq2 (M, N, A, LDA, TAU, WORK, INFO)
           DGELQ2 computes the LQ factorization of a general rectangular
           matrix using an unblocked algorithm.

Function/Subroutine Documentation
   subroutine dgelq2 (integer M, integer N, double precision, dimension( lda,
       * ) A, integer LDA, double precision, dimension( * ) TAU, double
       precision, dimension( * ) WORK, integer INFO)
       DGELQ2 computes the LQ factorization of a general rectangular matrix
       using an unblocked algorithm.

Purpose:
   DGELQ2 computes an LQ factorization of a real m by n matrix A:
   A = L * Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the m by n matrix A.
```

```
   On exit, the elements on and below the diagonal of the array
   contain the m by min(m,n) lower trapezoidal matrix L (L is
   lower triangular if m <= n); the elements above the diagonal,
   with the array TAU, represent the orthogonal matrix Q as a
   product of elementary reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is DOUBLE PRECISION array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is DOUBLE PRECISION array, dimension (M)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

Further Details:
   The matrix Q is represented as a product of elementary reflectors

               Q = H(k) . . . H(2) H(1), where k = min(m,n).

   Each H(i) has the form

               H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(1:i-1) = 0 and v(i) = 1; v(i+1:n) is stored on exit in A(i,i+1:n),
   and tau in TAU(i).
```

———-

32.10.3 fortran code

— dgelq2.f —

```
*  =====================================================================
      SUBROUTINE DGELQ2( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
```

```
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      DOUBLE PRECISION   AII
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARF, DLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGELQ2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = 1, K
*
*        Generate elementary reflector H(i) to annihilate A(i,i+1:n)
*
         CALL DLARFG( N-I+1, A( I, I ), A( I, MIN( I+1, N ) ), LDA,
     $                TAU( I ) )
```

```
         IF( I.LT.M ) THEN
*
*           Apply H(i) to A(i+1:m,i:n) from the right
*
            AII = A( I, I )
            A( I, I ) = ONE
            CALL DLARF( 'Right', M-I, N-I+1, A( I, I ), LDA, TAU( I ),
     $                  A( I+1, I ), LDA, WORK )
            A( I, I ) = AII
         END IF
   10 CONTINUE
      RETURN
*
*     End of DGELQ2
*
      END
```

———-

32.10.4 lisp code

— **dgelq2.lisp** —

———-

32.11 dgelqf

32.11.1 Axiom unit tests

— **dgelqf.input** —

```
)set break resume
)sys rm -f dgelqf.output
)spool dgelqf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.11.2 Axiom help page

— dgelqf.help —

```
====================================================================
dgelqf examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgelqf (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           DGELQF

Function/Subroutine Documentation
   subroutine dgelqf (integer M, integer N, double precision, dimension( lda,
       * ) A, integer LDA, double precision, dimension( * ) TAU, double
       precision, dimension( * ) WORK, integer LWORK, integer INFO)
       DGELQF

Purpose:
   DGELQF computes an LQ factorization of a real M-by-N matrix A: A = L * Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, the elements on and below the diagonal of the array
   contain the m-by-min(m,n) lower trapezoidal matrix L (L is
   lower triangular if m <= n); the elements above the diagonal,
   with the array TAU, represent the orthogonal matrix Q as a
   product of elementary reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).
```

```
 TAU
   TAU is DOUBLE PRECISION array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.  LWORK >= max(1,M).
   For optimum performance LWORK >= M*NB, where NB is the
   optimal blocksize.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details:
   The matrix Q is represented as a product of elementary reflectors

               Q = H(k) . . . H(2) H(1), where k = min(m,n).

   Each H(i) has the form

               H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(1:i-1) = 0 and v(i) = 1; v(i+1:n) is stored on exit in A(i,i+1:n),
   and tau in TAU(i).
```

32.11.3 fortran code

— dgelqf.f —

```
*  =====================================================================
      SUBROUTINE DGELQF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
```

```
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, LDWORK, LWKOPT, NB,
     $                   NBMIN, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGELQ2, DLARFB, DLARFT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NB = ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
      LWKOPT = M*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, M ) .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGELQF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
```

```
         RETURN
      END IF
*
*     Quick return if possible
*
      K = MIN( M, N )
      IF( K.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      NX = 0
      IWS = M
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'DGELQF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = M
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'DGELQF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially
*
         DO 10 I = 1, K - NX, NB
            IB = MIN( K-I+1, NB )
*
*           Compute the LQ factorization of the current block
*           A(i:i+ib-1,i:n)
*
            CALL DGELQ2( IB, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                   IINFO )
            IF( I+IB.LE.M ) THEN
```

```
*
*             Form the triangular factor of the block reflector
*             H = H(i) H(i+1) . . . H(i+ib-1)
*
              CALL DLARFT( 'Forward', 'Rowwise', N-I+1, IB, A( I, I ),
     $                     LDA, TAU( I ), WORK, LDWORK )
*
*             Apply H to A(i+ib:m,i:n) from the right
*
              CALL DLARFB( 'Right', 'No transpose', 'Forward',
     $                     'Rowwise', M-I-IB+1, N-I+1, IB, A( I, I ),
     $                     LDA, WORK, LDWORK, A( I+IB, I ), LDA,
     $                     WORK( IB+1 ), LDWORK )
           END IF
   10    CONTINUE
      ELSE
         I = 1
      END IF
*
*     Use unblocked code to factor the last or only block.
*
      IF( I.LE.K )
     $   CALL DGELQ2( M-I+1, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                IINFO )
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of DGELQF
*
      END
```

———-

32.11.4 lisp code

— **dgelqf.lisp** —

———-

32.12 dgemqrt

32.12.1 Axiom unit tests

— dgemqrt.input —

```
)set break resume
)sys rm -f dgemqrt.output
)spool dgemqrt.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.12.2 Axiom help page

— dgemqrt.help —

```
======================================================================
dgemqrt examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgemqrt (SIDE, TRANS, M, N, K, NB, V, LDV, T, LDT, C, LDC,
           WORK, INFO)
           DGEMQRT

Function/Subroutine Documentation
   subroutine dgemqrt (character SIDE, character TRANS, integer M, integer N,
       integer K, integer NB, double precision, dimension( ldv, * ) V, integer
       LDV, double precision, dimension( ldt, * ) T, integer LDT, double
       precision, dimension( ldc, * ) C, integer LDC, double precision,
       dimension( * ) WORK, integer INFO)
       DGEMQRT

Purpose:
   DGEMQRT overwrites the general real M-by-N matrix C with
```

```
                          SIDE = 'L'     SIDE = 'R'
          TRANS = 'N':      Q C            C Q
          TRANS = 'T':   Q**T C            C Q**T

  where Q is a real orthogonal matrix defined as the product of K
  elementary reflectors:

                Q = H(1) H(2) . . . H(K) = I - V T V**T

  generated using the compact WY representation as returned by DGEQRT.

  Q is of order M if SIDE = 'L' and of order N  if SIDE = 'R'.

Parameters:

 SIDE
  SIDE is CHARACTER*1
  = 'L': apply Q or Q**T from the Left;
  = 'R': apply Q or Q**T from the Right.

 TRANS
  TRANS is CHARACTER*1
  = 'N':  No transpose, apply Q;
  = 'C':  Transpose, apply Q**T.

 M
  M is INTEGER
  The number of rows of the matrix C. M >= 0.

 N
  N is INTEGER
  The number of columns of the matrix C. N >= 0.

 K
  K is INTEGER
  The number of elementary reflectors whose product defines the matrix Q.
  If SIDE = 'L', M >= K >= 0;
  if SIDE = 'R', N >= K >= 0.

 NB
  NB is INTEGER
  The block size used for the storage of T.  K >= NB >= 1.
  This must be the same value of NB used to generate T in CGEQRT.

 V
  V is DOUBLE PRECISION array, dimension (LDV,K)
  The i-th column must contain the vector which defines the
  elementary reflector H(i), for i = 1,2,...,k, as returned by
  CGEQRT in the first K columns of its array argument A.
```

```
LDV
  LDV is INTEGER
  The leading dimension of the array V.
  If SIDE = 'L', LDA >= max(1,M);
  if SIDE = 'R', LDA >= max(1,N).

T
  T is DOUBLE PRECISION array, dimension (LDT,K)
  The upper triangular factors of the block reflectors
  as returned by CGEQRT, stored as a NB-by-N matrix.

LDT
  LDT is INTEGER
  The leading dimension of the array T.  LDT >= NB.

C
  C is DOUBLE PRECISION array, dimension (LDC,N)
  On entry, the M-by-N matrix C.
  On exit, C is overwritten by Q C, Q**T C, C Q**T or C Q.

LDC
  LDC is INTEGER
  The leading dimension of the array C. LDC >= max(1,M).

WORK
  WORK is DOUBLE PRECISION array. The dimension of
  WORK is N*NB if SIDE = 'L', or  M*NB if SIDE = 'R'.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

32.12.3 fortran code

— dgemqrt.f —

```
*  =====================================================================
      SUBROUTINE DGEMQRT( SIDE, TRANS, M, N, K, NB, V, LDV, T, LDT,
     $                    C, LDC, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
```

```
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER SIDE, TRANS
      INTEGER   INFO, K, LDV, LDC, M, N, NB, LDT
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION V( LDV, * ), C( LDC, * ), T( LDT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     ..
*     .. Local Scalars ..
      LOGICAL            LEFT, RIGHT, TRAN, NOTRAN
      INTEGER            I, IB, LDWORK, KF, Q
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, DLARFB
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     .. Test the input arguments ..
*
      INFO   = 0
      LEFT   = LSAME( SIDE,  'L' )
      RIGHT  = LSAME( SIDE,  'R' )
      TRAN   = LSAME( TRANS, 'T' )
      NOTRAN = LSAME( TRANS, 'N' )
*
      IF( LEFT ) THEN
         LDWORK = MAX( 1, N )
         Q = M
      ELSE IF ( RIGHT ) THEN
         LDWORK = MAX( 1, M )
         Q = N
      END IF
      IF( .NOT.LEFT .AND. .NOT.RIGHT ) THEN
         INFO = -1
      ELSE IF( .NOT.TRAN .AND. .NOT.NOTRAN ) THEN
         INFO = -2
      ELSE IF( M.LT.0 ) THEN
         INFO = -3
```

```
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( K.LT.0 .OR. K.GT.Q ) THEN
         INFO = -5
      ELSE IF( NB.LT.1 .OR. (NB.GT.K .AND. K.GT.0)) THEN
         INFO = -6
      ELSE IF( LDV.LT.MAX( 1, Q ) ) THEN
         INFO = -8
      ELSE IF( LDT.LT.NB ) THEN
         INFO = -10
      ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
         INFO = -12
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEMQRT', -INFO )
         RETURN
      END IF
*
*     .. Quick return if possible ..
*
      IF( M.EQ.0 .OR. N.EQ.0 .OR. K.EQ.0 ) RETURN
*
      IF( LEFT .AND. TRAN ) THEN
*
         DO I = 1, K, NB
            IB = MIN( NB, K-I+1 )
            CALL DLARFB( 'L', 'T', 'F', 'C', M-I+1, N, IB,
     $                   V( I, I ), LDV, T( 1, I ), LDT,
     $                   C( I, 1 ), LDC, WORK, LDWORK )
         END DO
*
      ELSE IF( RIGHT .AND. NOTRAN ) THEN
*
         DO I = 1, K, NB
            IB = MIN( NB, K-I+1 )
            CALL DLARFB( 'R', 'N', 'F', 'C', M, N-I+1, IB,
     $                   V( I, I ), LDV, T( 1, I ), LDT,
     $                   C( 1, I ), LDC, WORK, LDWORK )
         END DO
*
      ELSE IF( LEFT .AND. NOTRAN ) THEN
*
         KF = ((K-1)/NB)*NB+1
         DO I = KF, 1, -NB
            IB = MIN( NB, K-I+1 )
            CALL DLARFB( 'L', 'N', 'F', 'C', M-I+1, N, IB,
     $                   V( I, I ), LDV, T( 1, I ), LDT,
     $                   C( I, 1 ), LDC, WORK, LDWORK )
         END DO
```

```
*
      ELSE IF( RIGHT .AND. TRAN ) THEN
*
         KF = ((K-1)/NB)*NB+1
         DO I = KF, 1, -NB
            IB = MIN( NB, K-I+1 )
            CALL DLARFB( 'R', 'T', 'F', 'C', M, N-I+1, IB,
     $                   V( I, I ), LDV, T( 1, I ), LDT,
     $                   C( 1, I ), LDC, WORK, LDWORK )
         END DO
*
      END IF
*
      RETURN
*
*     End of DGEMQRT
*
      END
```

32.12.4 lisp code

— dgemqrt.lisp —

```
(defun dgemqrt (side trans m n k nb v ldv t$ ldt c ldc work info)
  (declare (type (array double-float (*)) work c t$ v)
           (type (f2cl-lib:integer4) info ldc ldt ldv nb k n m)
           (type (string *) trans side))
  (f2cl-lib:with-multi-array-data
      ((side character side-%data% side-%offset%)
       (trans character trans-%data% trans-%offset%)
       (v double-float v-%data% v-%offset%)
       (t$ double-float t$-%data% t$-%offset%)
       (c double-float c-%data% c-%offset%)
       (work double-float work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (ldwork 0) (kf 0) (q 0) (left nil) (right nil)
           (tran nil) (notran nil))
      (declare (type f2cl-lib:logical notran tran right left)
               (type (f2cl-lib:integer4) q kf ldwork ib i))
      (setf info 0)
      (setf left
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame side "L")
                (declare (ignore var-1))
                (when var-0 (setf side var-0))
                ret-val))
      (setf right
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "R")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val))
(setf tran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 (left
  (setf ldwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf q m))
 (right
  (setf ldwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf q n)))
(cond ((and (not left) (not right)) (setf info -1))
      ((and (not tran) (not notran)) (setf info -2))
      ((< m 0) (setf info -3)) ((< n 0) (setf info -4))
      ((or (< k 0) (> k q)) (setf info -5))
      ((or (< nb 1) (and (> nb k) (> k 0))) (setf info -6))
      ((< ldv (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 q)))
       (setf info -8))
      ((< ldt nb) (setf info -10))
      ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -12)))
(cond
 ((/= info 0) (xerbla "DGEMQRT" (f2cl-lib:int-sub info)) (go end_label)))
(if (or (= m 0) (= n 0) (= k 0))
    (go end_label))
(cond
 ((and left tran)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                ((> i k) nil)
    (tagbody
      (setf ib
              (min (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13 var-14)
```

```
            (dlarfb "L" "T" "F" "C"
             (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) n ib
             (f2cl-lib:array-slice v-%data% double-float (i i)
                                   ((1 ldv) (1 *)) v-%offset%)
             ldv
             (f2cl-lib:array-slice t$-%data% double-float (1 i)
                                   ((1 ldt) (1 *)) t$-%offset%)
             ldt
             (f2cl-lib:array-slice c-%data% double-float (i 1)
                                   ((1 ldc) (1 *)) c-%offset%)
             ldc work ldwork)
           (declare
            (ignore var-0 var-1 var-2 var-3 var-4 var-7 var-9 var-11
             var-13))
           (when var-5 (setf n var-5))
           (when var-6 (setf ib var-6))
           (when var-8 (setf ldv var-8))
           (when var-10 (setf ldt var-10))
           (when var-12 (setf ldc var-12))
           (when var-14 (setf ldwork var-14)))
       label100000)))
    ((and right notran)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                   ((> i k) nil)
       (tagbody
         (setf ib
                 (min (the f2cl-lib:integer4 nb)
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
              var-10 var-11 var-12 var-13 var-14)
             (dlarfb "R" "N" "F" "C" m
              (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) ib
              (f2cl-lib:array-slice v-%data% double-float (i i)
                                    ((1 ldv) (1 *)) v-%offset%)
              ldv
              (f2cl-lib:array-slice t$-%data% double-float (1 i)
                                    ((1 ldt) (1 *)) t$-%offset%)
              ldt
              (f2cl-lib:array-slice c-%data% double-float (1 i)
                                    ((1 ldc) (1 *)) c-%offset%)
              ldc work ldwork)
           (declare
            (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9 var-11
             var-13))
           (when var-4 (setf m var-4))
           (when var-6 (setf ib var-6))
           (when var-8 (setf ldv var-8))
           (when var-10 (setf ldt var-10))
```

```
        (when var-12 (setf ldc var-12))
        (when var-14 (setf ldwork var-14)))
    label100001)))
((and left notran)
 (setf kf (+ (* (the f2cl-lib:integer4 (truncate (- k 1) nb)) nb) 1))
 (f2cl-lib:fdo (i kf (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
               ((> i 1) nil)
   (tagbody
     (setf ib
             (min (the f2cl-lib:integer4 nb)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12 var-13 var-14)
         (dlarfb "L" "N" "F" "C"
          (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) n ib
          (f2cl-lib:array-slice v-%data% double-float (i i)
                                ((1 ldv) (1 *)) v-%offset%)
          ldv
          (f2cl-lib:array-slice t$-%data% double-float (1 i)
                                ((1 ldt) (1 *)) t$-%offset%)
          ldt
          (f2cl-lib:array-slice c-%data% double-float (i 1)
                                ((1 ldc) (1 *)) c-%offset%)
          ldc work ldwork)
       (declare
        (ignore var-0 var-1 var-2 var-3 var-4 var-7 var-9 var-11
         var-13))
       (when var-5 (setf n var-5))
       (when var-6 (setf ib var-6))
       (when var-8 (setf ldv var-8))
       (when var-10 (setf ldt var-10))
       (when var-12 (setf ldc var-12))
       (when var-14 (setf ldwork var-14)))
    label100002)))
((and right tran)
 (setf kf (+ (* (the f2cl-lib:integer4 (truncate (- k 1) nb)) nb) 1))
 (f2cl-lib:fdo (i kf (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
               ((> i 1) nil)
   (tagbody
     (setf ib
             (min (the f2cl-lib:integer4 nb)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12 var-13 var-14)
         (dlarfb "R" "T" "F" "C" m
          (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) ib
```

```
                (f2cl-lib:array-slice v-%data% double-float (i i)
                                      ((1 ldv) (1 *)) v-%offset%)
                ldv
                (f2cl-lib:array-slice t$-%data% double-float (1 i)
                                      ((1 ldt) (1 *)) t$-%offset%)
                ldt
                (f2cl-lib:array-slice c-%data% double-float (1 i)
                                      ((1 ldc) (1 *)) c-%offset%)
                ldc work ldwork)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9 var-11
                var-13))
              (when var-4 (setf m var-4))
              (when var-6 (setf ib var-6))
              (when var-8 (setf ldv var-8))
              (when var-10 (setf ldt var-10))
              (when var-12 (setf ldc var-12))
              (when var-14 (setf ldwork var-14)))
           label100003))))
      (go end_label)
     end_label
      (return
       (values side trans m n nil nil nil ldv nil ldt nil ldc nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgemqrt
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::side
                                              fortran-to-lisp::trans
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::ldv nil
                                              fortran-to-lisp::ldt nil
```

```
                                        fortran-to-lisp::ldc nil
                                        fortran-to-lisp::info)
                                      :calls 'nil)))
```

———-

32.13 dgeql2

32.13.1 Axiom unit tests

— **dgeql2.input** —

```
)set break resume
)sys rm -f dgeql2.output
)spool dgeql2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.13.2 Axiom help page

— **dgeql2.help** —

```
======================================================================
dgeql2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgeql2 (M, N, A, LDA, TAU, WORK, INFO)
           DGEQL2 computes the QL factorization of a general rectangular
           matrix using an unblocked algorithm.
```

```
Function/Subroutine Documentation
   subroutine dgeql2 (integer M, integer N, double precision, dimension( lda,
       * ) A, integer LDA, double precision, dimension( * ) TAU, double
       precision, dimension( * ) WORK, integer INFO)
       DGEQL2 computes the QL factorization of a general rectangular matrix
       using an unblocked algorithm.

Purpose:
   DGEQL2 computes a QL factorization of a real m by n matrix A: A = Q * L.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the m by n matrix A.
   On exit, if m >= n, the lower triangle of the subarray
   A(m-n+1:m,1:n) contains the n by n lower triangular matrix L;
   if m <= n, the elements on and below the (n-m)-th
   superdiagonal contain the m by n lower trapezoidal matrix L;
   the remaining elements, with the array TAU, represent the
   orthogonal matrix Q as a product of elementary reflectors
   (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is DOUBLE PRECISION array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is DOUBLE PRECISION array, dimension (N)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

Further Details:
   The matrix Q is represented as a product of elementary reflectors
```

```
              Q = H(k) . . . H(2) H(1), where k = min(m,n).

   Each H(i) has the form

              H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(m-k+i+1:m) = 0 and v(m-k+i) = 1; v(1:m-k+i-1) is stored on exit in
   A(1:m-k+i-1,n-k+i), and tau in TAU(i).
```

32.13.3 fortran code

— dgeql2.f —

```
*  =====================================================================
      SUBROUTINE DGEQL2( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      DOUBLE PRECISION   AII
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARF, DLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
```

```
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEQL2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = K, 1, -1
*
*        Generate elementary reflector H(i) to annihilate
*        A(1:m-k+i-1,n-k+i)
*
         CALL DLARFG( M-K+I, A( M-K+I, N-K+I ), A( 1, N-K+I ), 1,
     $                TAU( I ) )
*
*        Apply H(i) to A(1:m-k+i,1:n-k+i-1) from the left
*
         AII = A( M-K+I, N-K+I )
         A( M-K+I, N-K+I ) = ONE
         CALL DLARF( 'Left', M-K+I, N-K+I-1, A( 1, N-K+I ), 1, TAU( I ),
     $               A, LDA, WORK )
         A( M-K+I, N-K+I ) = AII
   10 CONTINUE
      RETURN
*
*     End of DGEQL2
*
      END
```

———-

32.13.4 lisp code

— dgeql2.lisp —

```
(let* ((one 1.0d0))
```

```
(declare (type (double-float 1.0d0 1.0d0) one)
         (ignorable one))
(defun dgeql2 (m n a lda tau work info)
  (declare (type (array double-float (*)) work tau a)
           (type (f2cl-lib:integer4) info lda n m))
  (f2cl-lib:with-multi-array-data
      ((a double-float a-%data% a-%offset%)
       (tau double-float tau-%data% tau-%offset%)
       (work double-float work-%data% work-%offset%))
    (prog ((aii 0.0d0) (i 0) (k 0))
      (declare (type (double-float) aii)
               (type (f2cl-lib:integer4) i k))
      (setf info 0)
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4)))
      (cond
       ((/= info 0) (xerbla "DGEQL2" (f2cl-lib:int-sub info))
        (go end_label)))
      (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
      (f2cl-lib:fdo (i k (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                    ((> i 1) nil)
        (tagbody
          (dlarfg (f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
           (f2cl-lib:array-slice a-%data% double-float
                                 ((+ m (f2cl-lib:int-sub k) i)
                                  (f2cl-lib:int-add (f2cl-lib:int-sub n k)
                                                    i))
                                 ((1 lda) (1 *)) a-%offset%)
           (f2cl-lib:array-slice a-%data% double-float
                                 (1
                                  (f2cl-lib:int-add (f2cl-lib:int-sub n k)
                                                    i))
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                 tau-%offset%))
          (setf aii
                  (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                  (f2cl-lib:int-add (f2cl-lib:int-sub n k)
                                                    i))
                                 ((1 lda) (1 *)) a-%offset%))
          (setf (f2cl-lib:fref a-%data%
                               ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                (f2cl-lib:int-add (f2cl-lib:int-sub n k) i))
                               ((1 lda) (1 *)) a-%offset%)
                  one)
          (multiple-value-bind
```

```
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (dlarf "Left" (f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                 (f2cl-lib:int-sub (f2cl-lib:int-add (f2cl-lib:int-sub n k) i)
                                   1)
                 (f2cl-lib:array-slice a-%data% double-float
                                       (1
                                        (f2cl-lib:int-add
                                         (f2cl-lib:int-sub n k) i))
                                       ((1 lda) (1 *)) a-%offset%)
                 1
                 (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                       tau-%offset%)
                 a lda work)
               (declare
                (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
               (when var-7 (setf lda var-7)))
             (setf (f2cl-lib:fref a-%data%
                                  ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                   (f2cl-lib:int-add (f2cl-lib:int-sub n k) i))
                                  ((1 lda) (1 *)) a-%offset%)
                   aii)
           label10))
        (go end_label)
       end_label
        (return (values nil nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgeql2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::lda
                                              nil nil fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

32.14 dgeqlf

32.14.1 Axiom unit tests

— **dgeqlf.input** —

```
)set break resume
)sys rm -f dgeqlf.output
)spool dgeqlf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.14.2 Axiom help page

— **dgeqlf.help** —

```
======================================================================
dgeqlf examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgeqlf (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           DGEQLF

Function/Subroutine Documentation
   subroutine dgeqlf (integer M, integer N, double precision, dimension( lda,
       * ) A, integer LDA, double precision, dimension( * ) TAU, double
       precision, dimension( * ) WORK, integer LWORK, integer INFO)
       DGEQLF

Purpose:
   DGEQLF computes a QL factorization of a real M-by-N matrix A: A = Q * L.

Parameters:
```

```
 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit,
   if m >= n, the lower triangle of the subarray
   A(m-n+1:m,1:n) contains the N-by-N lower triangular matrix L;
   if m <= n, the elements on and below the (n-m)-th
   superdiagonal contain the M-by-N lower trapezoidal matrix L;
   the remaining elements, with the array TAU, represent the
   orthogonal matrix Q as a product of elementary reflectors
   (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is DOUBLE PRECISION array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.  LWORK >= max(1,N).
   For optimum performance LWORK >= N*NB, where NB is the
   optimal blocksize.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details:
   The matrix Q is represented as a product of elementary reflectors
```

```
              Q = H(k) . . . H(2) H(1), where k = min(m,n).

   Each H(i) has the form

              H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(m-k+i+1:m) = 0 and v(m-k+i) = 1; v(1:m-k+i-1) is stored on exit in
   A(1:m-k+i-1,n-k+i), and tau in TAU(i).
```

32.14.3 fortran code

— dgeqlf.f —

```
*  =====================================================================
      SUBROUTINE DGEQLF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, KI, KK, LDWORK, LWKOPT,
     $                   MU, NB, NBMIN, NU, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEQL2, DLARFB, DLARFT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
```

```
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
*
      IF( INFO.EQ.0 ) THEN
         K = MIN( M, N )
         IF( K.EQ.0 ) THEN
            LWKOPT = 1
         ELSE
            NB = ILAENV( 1, 'DGEQLF', ' ', M, N, -1, -1 )
            LWKOPT = N*NB
         END IF
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
            INFO = -7
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEQLF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( K.EQ.0 ) THEN
         RETURN
      END IF
*
      NBMIN = 2
      NX = 1
      IWS = N
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
```

```
*
         NX = MAX( 0, ILAENV( 3, 'DGEQLF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = N
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'DGEQLF', ' ', M, N, -1,
     $                  -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially.
*        The last kk columns are handled by the block method.
*
         KI = ( ( K-NX-1 ) / NB )*NB
         KK = MIN( K, KI+NB )
*
         DO 10 I = K - KK + KI + 1, K - KK + 1, -NB
            IB = MIN( K-I+1, NB )
*
*           Compute the QL factorization of the current block
*           A(1:m-k+i+ib-1,n-k+i:n-k+i+ib-1)
*
            CALL DGEQL2( M-K+I+IB-1, IB, A( 1, N-K+I ), LDA, TAU( I ),
     $                   WORK, IINFO )
            IF( N-K+I.GT.1 ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i+ib-1) . . . H(i+1) H(i)
*
               CALL DLARFT( 'Backward', 'Columnwise', M-K+I+IB-1, IB,
     $                      A( 1, N-K+I ), LDA, TAU( I ), WORK, LDWORK )
*
*              Apply H**T to A(1:m-k+i+ib-1,1:n-k+i-1) from the left
*
               CALL DLARFB( 'Left', 'Transpose', 'Backward',
     $                      'Columnwise', M-K+I+IB-1, N-K+I-1, IB,
     $                      A( 1, N-K+I ), LDA, WORK, LDWORK, A, LDA,
     $                      WORK( IB+1 ), LDWORK )
```

```
            END IF
   10    CONTINUE
         MU = M - K + I + NB - 1
         NU = N - K + I + NB - 1
      ELSE
         MU = M
         NU = N
      END IF
*
*     Use unblocked code to factor the last or only block
*
      IF( MU.GT.0 .AND. NU.GT.0 )
     $   CALL DGEQL2( MU, NU, A, LDA, TAU, WORK, IINFO )
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of DGEQLF
*
      END
```

32.14.4 lisp code

— dgeqlf.lisp —

```
(defun dgeqlf (m n a lda tau work lwork info)
  (declare (type (array double-float (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a double-float a-%data% a-%offset%)
       (tau double-float tau-%data% tau-%offset%)
       (work double-float work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ki 0) (kk 0) (ldwork 0)
           (lwkopt 0) (mu 0) (nb 0) (nbmin 0) (nu 0) (nx 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) nx nu nbmin nb mu lwkopt ldwork kk ki
                k iws iinfo ib i))
      (setf info 0)
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4)))
      (cond
       ((= info 0)
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (cond ((= k 0) (setf lwkopt 1))
```

```
        (t
         (setf nb
                 (multiple-value-bind
                     (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                     (ilaenv 1 "DGEQLF" " " m n -1 -1)
                   (declare (ignore var-0 var-1 var-2 var-5 var-6))
                   (when var-3 (setf m var-3))
                   (when var-4 (setf n var-4))
                   ret-val))
         (setf lwkopt (f2cl-lib:int-mul n nb))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
  (cond
   ((and
     (< lwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
     (not lquery))
    (setf info -7)))))
(cond
 ((/= info 0) (xerbla "DGEQLF" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond ((= k 0) (go end_label)))
(setf nbmin 2)
(setf nx 1)
(setf iws n)
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "DGEQLF" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (when var-3 (setf m var-3))
                      (when var-4 (setf n var-4))
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork n) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
                            (ilaenv 2 "DGEQLF" " " m n -1 -1)
                          (declare
```

```
                              (ignore var-0 var-1 var-2 var-5 var-6))
                             (when var-3 (setf m var-3))
                             (when var-4 (setf n var-4))
                             ret-val)))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (setf ki (* (the f2cl-lib:integer4 (truncate (- k nx 1) nb)) nb))
  (setf kk
          (min (the f2cl-lib:integer4 k)
               (the f2cl-lib:integer4 (f2cl-lib:int-add ki nb))))
  (f2cl-lib:fdo (i (f2cl-lib:int-add k (f2cl-lib:int-sub kk) ki 1)
                 (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
                ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub kk) 1)) nil)
    (tagbody
      (setf ib
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
               (the f2cl-lib:integer4 nb)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dgeql2
           (f2cl-lib:int-sub
            (f2cl-lib:int-add (f2cl-lib:int-sub m k) i ib) 1)
           ib
           (f2cl-lib:array-slice a-%data% double-float
                                 (1
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-sub n k) i))
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                 tau-%offset%)
           work iinfo)
        (declare (ignore var-0 var-1 var-2 var-4 var-5))
        (setf lda var-3)
        (setf iinfo var-6))
      (cond
       ((> (f2cl-lib:int-add n (f2cl-lib:int-sub k) i) 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (dlarft "Backward" "Columnwise"
             (f2cl-lib:int-sub
              (f2cl-lib:int-add (f2cl-lib:int-sub m k) i ib) 1)
             ib
             (f2cl-lib:array-slice a-%data% double-float
                                   (1
                                    (f2cl-lib:int-add
                                     (f2cl-lib:int-sub n k) i))
                                   ((1 lda) (1 *)) a-%offset%)
             lda
```

```
                (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                      tau-%offset%)
                work ldwork)
            (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
            (when var-3 (setf ib var-3))
            (when var-5 (setf lda var-5))
            (when var-8 (setf ldwork var-8)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10 var-11 var-12 var-13 var-14)
              (dlarfb "Left" "Transpose" "Backward" "Columnwise"
               (f2cl-lib:int-sub
                (f2cl-lib:int-add (f2cl-lib:int-sub m k) i ib) 1)
               (f2cl-lib:int-sub
                (f2cl-lib:int-add (f2cl-lib:int-sub n k) i) 1)
               ib
               (f2cl-lib:array-slice a-%data% double-float
                                     (1
                                      (f2cl-lib:int-add
                                       (f2cl-lib:int-sub n k) i))
                                     ((1 lda) (1 *)) a-%offset%)
               lda work ldwork a lda
               (f2cl-lib:array-slice work-%data% double-float ((+ ib 1))
                                     ((1 *)) work-%offset%)
               ldwork)
            (declare
             (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
              var-13))
            (when var-6 (setf ib var-6))
            (when var-8 (setf lda var-8))
            (when var-10 (setf ldwork var-10))
            (when var-12 (setf lda var-12))
            (when var-14 (setf ldwork var-14)))))
      label10))
  (setf mu
          (f2cl-lib:int-sub
           (f2cl-lib:int-add (f2cl-lib:int-sub m k) i nb) 1))
  (setf nu
          (f2cl-lib:int-sub
           (f2cl-lib:int-add (f2cl-lib:int-sub n k) i nb) 1)))
 (t (setf mu m) (setf nu n)))
(if (and (> mu 0) (> nu 0))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dgeql2 mu nu a lda tau work iinfo)
      (declare (ignore var-0 var-1 var-2 var-4 var-5))
      (setf lda var-3)
      (setf iinfo var-6)))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce (the f2cl-lib:integer4 iws) 'double-float))
(go end_label)
```

```
     end_label
      (return (values m n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgeqlf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::dgeql2))))
```

———-

32.15 dgeqp3

32.15.1 Axiom unit tests

— **dgeqp3.input** —

```
)set break resume
)sys rm -f dgeqp3.output
)spool dgeqp3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.15.2 Axiom help page

— dgeqp3.help —

```
====================================================================
dgeqp3 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgeqp3 (M, N, A, LDA, JPVT, TAU, WORK, LWORK, INFO)
           DGEQP3

Function/Subroutine Documentation
   subroutine dgeqp3 (integer M, integer N, double precision, dimension( lda,
       * ) A, integer LDA, integer, dimension( * ) JPVT, double precision,
       dimension( * ) TAU, double precision, dimension( * ) WORK, integer
       LWORK, integer INFO)
       DGEQP3

Purpose:
   DGEQP3 computes a QR factorization with column pivoting of a
   matrix A:  A*P = Q*R  using Level 3 BLAS.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A. M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, the upper triangle of the array contains the
   min(M,N)-by-N upper trapezoidal matrix R; the elements below
   the diagonal, together with the array TAU, represent the
   orthogonal matrix Q as a product of min(M,N) elementary reflectors.

 LDA
   LDA is INTEGER
   The leading dimension of the array A. LDA >= max(1,M).
```

```
 JPVT
   JPVT is INTEGER array, dimension (N)
   On entry, if JPVT(J).ne.0, the J-th column of A is permuted
   to the front of A*P (a leading column); if JPVT(J)=0,
   the J-th column of A is a free column.
   On exit, if JPVT(J)=K, then the J-th column of A*P was the
   the K-th column of A.

 TAU
   TAU is DOUBLE PRECISION array, dimension (min(M,N))
   The scalar factors of the elementary reflectors.

 WORK
   WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
   On exit, if INFO=0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK. LWORK >= 3*N+1.
   For optimal performance LWORK >= 2*N+( N+1 )*NB, where NB
   is the optimal blocksize.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0: successful exit.
   < 0: if INFO = -i, the i-th argument had an illegal value.

Further Details:
   The matrix Q is represented as a product of elementary reflectors

                Q = H(1) H(2) . . . H(k), where k = min(m,n).

   Each H(i) has the form

                H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real/complex vector
   with v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in
   A(i+1:m,i), and tau in TAU(i).
```

———-

32.15.3 fortran code

— dgeqp3.f —

```
*  =====================================================================
      SUBROUTINE DGEQP3( M, N, A, LDA, JPVT, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            JPVT( * )
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            INB, INBMIN, IXOVER
      PARAMETER          ( INB = 1, INBMIN = 2, IXOVER = 3 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            FJB, IWS, J, JB, LWKOPT, MINMN, MINWS, NA, NB,
     $                   NBMIN, NFXD, NX, SM, SMINMN, SN, TOPBMN
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEQRF, DLAQP2, DLAQPS, DORMQR, DSWAP, XERBLA
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      DOUBLE PRECISION   DNRM2
      EXTERNAL           ILAENV, DNRM2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          INT, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test input arguments
*  ====================
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
```

```
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
*
      IF( INFO.EQ.0 ) THEN
         MINMN = MIN( M, N )
         IF( MINMN.EQ.0 ) THEN
            IWS = 1
            LWKOPT = 1
         ELSE
            IWS = 3*N + 1
            NB = ILAENV( INB, 'DGEQRF', ' ', M, N, -1, -1 )
            LWKOPT = 2*N + ( N + 1 )*NB
         END IF
         WORK( 1 ) = LWKOPT
*
         IF( ( LWORK.LT.IWS ) .AND. .NOT.LQUERY ) THEN
            INFO = -8
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEQP3', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Move initial columns up front.
*
      NFXD = 1
      DO 10 J = 1, N
         IF( JPVT( J ).NE.0 ) THEN
            IF( J.NE.NFXD ) THEN
               CALL DSWAP( M, A( 1, J ), 1, A( 1, NFXD ), 1 )
               JPVT( J ) = JPVT( NFXD )
               JPVT( NFXD ) = J
            ELSE
               JPVT( J ) = J
            END IF
            NFXD = NFXD + 1
         ELSE
            JPVT( J ) = J
         END IF
   10 CONTINUE
      NFXD = NFXD - 1
```

```
*
*     Factorize fixed columns
*  =======================
*
*     Compute the QR factorization of fixed columns and update
*     remaining columns.
*
      IF( NFXD.GT.0 ) THEN
         NA = MIN( M, NFXD )
*CC      CALL DGEQR2( M, NA, A, LDA, TAU, WORK, INFO )
         CALL DGEQRF( M, NA, A, LDA, TAU, WORK, LWORK, INFO )
         IWS = MAX( IWS, INT( WORK( 1 ) ) )
         IF( NA.LT.N ) THEN
*CC         CALL DORM2R( 'Left', 'Transpose', M, N-NA, NA, A, LDA,
*CC  $                   TAU, A( 1, NA+1 ), LDA, WORK, INFO )
            CALL DORMQR( 'Left', 'Transpose', M, N-NA, NA, A, LDA, TAU,
     $                   A( 1, NA+1 ), LDA, WORK, LWORK, INFO )
            IWS = MAX( IWS, INT( WORK( 1 ) ) )
         END IF
      END IF
*
*     Factorize free columns
*  ======================
*
      IF( NFXD.LT.MINMN ) THEN
*
         SM = M - NFXD
         SN = N - NFXD
         SMINMN = MINMN - NFXD
*
*        Determine the block size.
*
         NB = ILAENV( INB, 'DGEQRF', ' ', SM, SN, -1, -1 )
         NBMIN = 2
         NX = 0
*
         IF( ( NB.GT.1 ) .AND. ( NB.LT.SMINMN ) ) THEN
*
*           Determine when to cross over from blocked to unblocked code.
*
            NX = MAX( 0, ILAENV( IXOVER, 'DGEQRF', ' ', SM, SN, -1,
     $           -1 ) )
*
*
            IF( NX.LT.SMINMN ) THEN
*
*              Determine if workspace is large enough for blocked code.
*
               MINWS = 2*SN + ( SN+1 )*NB
               IWS = MAX( IWS, MINWS )
```

```
               IF( LWORK.LT.MINWS ) THEN
*
*                 Not enough workspace to use optimal NB: Reduce NB and
*                 determine the minimum value of NB.
*
                  NB = ( LWORK-2*SN ) / ( SN+1 )
                  NBMIN = MAX( 2, ILAENV( INBMIN, 'DGEQRF', ' ', SM, SN,
     $                    -1, -1 ) )
*
*
               END IF
            END IF
         END IF
*
*        Initialize partial column norms. The first N elements of work
*        store the exact column norms.
*
         DO 20 J = NFXD + 1, N
            WORK( J ) = DNRM2( SM, A( NFXD+1, J ), 1 )
            WORK( N+J ) = WORK( J )
   20    CONTINUE
*
         IF( ( NB.GE.NBMIN ) .AND. ( NB.LT.SMINMN ) .AND.
     $       ( NX.LT.SMINMN ) ) THEN
*
*          Use blocked code initially.
*
           J = NFXD + 1
*
*          Compute factorization: while loop.
*
*
           TOPBMN = MINMN - NX
   30      CONTINUE
           IF( J.LE.TOPBMN ) THEN
              JB = MIN( NB, TOPBMN-J+1 )
*
*             Factorize JB columns among columns J:N.
*
              CALL DLAQPS( M, N-J+1, J-1, JB, FJB, A( 1, J ), LDA,
     $                     JPVT( J ), TAU( J ), WORK( J ), WORK( N+J ),
     $                     WORK( 2*N+1 ), WORK( 2*N+JB+1 ), N-J+1 )
*
              J = J + FJB
              GO TO 30
           END IF
         ELSE
           J = NFXD + 1
         END IF
*
```

```
*        Use unblocked code to factor the last or only block.
*
*
         IF( J.LE.MINMN )
     $      CALL DLAQP2( M, N-J+1, J-1, A( 1, J ), LDA, JPVT( J ),
     $                   TAU( J ), WORK( J ), WORK( N+J ),
     $                   WORK( 2*N+1 ) )
*
      END IF
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of DGEQP3
*
      END
```

32.15.4 lisp code

— dgeqp3.lisp —

```
(let* ((inb 1) (inbmin 2) (ixover 3))
  (declare (type (f2cl-lib:integer4 1 1) inb)
           (type (f2cl-lib:integer4 2 2) inbmin)
           (type (f2cl-lib:integer4 3 3) ixover)
           (ignorable inb inbmin ixover))
  (defun dgeqp3 (m n a lda jpvt tau work lwork info)
    (declare (type (array f2cl-lib:integer4 (*)) jpvt)
             (type (array double-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lwork lda n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (work double-float work-%data% work-%offset%)
         (jpvt f2cl-lib:integer4 jpvt-%data% jpvt-%offset%))
      (prog ((fjb 0) (iws 0) (j 0) (jb 0) (lwkopt 0) (minmn 0) (minws 0) (na 0)
             (nb 0) (nbmin 0) (nfxd 0) (nx 0) (sm 0) (sminmn 0) (sn 0)
             (topbmn 0) (lquery nil))
        (declare
         (type (f2cl-lib:integer4) fjb iws j jb lwkopt minmn minws na nb nbmin
          nfxd nx sm sminmn sn topbmn)
         (type f2cl-lib:logical lquery))
        (setf info 0)
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
```

```
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
      (setf info -4)))
(cond
 ((= info 0)
  (setf minmn
          (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
  (cond ((= minmn 0) (setf iws 1) (setf lwkopt 1))
        (t (setf iws (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) 1))
         (setf nb
                 (multiple-value-bind
                     (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                      var-6)
                     (ilaenv inb "DGEQRF" " " m n -1 -1)
                   (declare (ignore var-1 var-2 var-5 var-6))
                   (when var-0 (setf inb var-0))
                   (when var-3 (setf m var-3))
                   (when var-4 (setf n var-4))
                   ret-val))
         (setf lwkopt
                 (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-add n 1) nb)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
  (cond ((and (< lwork iws) (not lquery)) (setf info -8)))))
(cond
 ((/= info 0) (xerbla "DGEQP3" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(setf nfxd 1)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (cond
     ((/= (f2cl-lib:fref jpvt (j) ((1 *))) 0)
      (cond
       ((/= j nfxd)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dswap m
             (f2cl-lib:array-slice a-%data% double-float (1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice a-%data% double-float (1 nfxd)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf m var-0)))
        (setf (f2cl-lib:fref jpvt-%data% (j) ((1 *)) jpvt-%offset%)
                (f2cl-lib:fref jpvt-%data% (nfxd) ((1 *))
                               jpvt-%offset%))
        (setf (f2cl-lib:fref jpvt-%data% (nfxd) ((1 *)) jpvt-%offset%)
```

```
                j))
        (t
         (setf (f2cl-lib:fref jpvt-%data% (j) ((1 *)) jpvt-%offset%)
                j)))
       (setf nfxd (f2cl-lib:int-add nfxd 1)))
      (t
       (setf (f2cl-lib:fref jpvt-%data% (j) ((1 *)) jpvt-%offset%) j)))
   label10))
(setf nfxd (f2cl-lib:int-sub nfxd 1))
(cond
 ((> nfxd 0)
  (setf na
          (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 nfxd)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dgeqrf m na a lda tau work lwork info)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf na var-1)
    (setf lda var-3)
    (setf info var-7))
  (setf iws
          (max (the f2cl-lib:integer4 iws)
               (the f2cl-lib:integer4
                    (f2cl-lib:int
                     (f2cl-lib:fref work-%data% (1) ((1 *))
                                    work-%offset%)))))
  (cond
   ((< na n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (dormqr "Left" "Transpose" m (f2cl-lib:int-sub n na) na a lda
         tau
         (f2cl-lib:array-slice a-%data% double-float
                               (1 (f2cl-lib:int-add na 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda work lwork info)
      (declare (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10))
      (when var-2 (setf m var-2))
      (when var-4 (setf na var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf lda var-9))
      (when var-11 (setf lwork var-11))
      (when var-12 (setf info var-12)))
    (setf iws
            (max (the f2cl-lib:integer4 iws)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int
                       (f2cl-lib:fref work-%data% (1) ((1 *))
```

```
                                        work-%offset%))))))))))
        (cond
         ((< nfxd minmn) (setf sm (f2cl-lib:int-sub m nfxd))
          (setf sn (f2cl-lib:int-sub n nfxd))
          (setf sminmn (f2cl-lib:int-sub minmn nfxd))
          (setf nb
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                      (ilaenv inb "DGEQRF" " " sm sn -1 -1)
                    (declare (ignore var-1 var-2 var-5 var-6))
                    (when var-0 (setf inb var-0))
                    (when var-3 (setf sm var-3))
                    (when var-4 (setf sn var-4))
                    ret-val))
          (setf nbmin 2) (setf nx 0)
          (cond
           ((and (> nb 1) (< nb sminmn))
            (setf nx
                    (max (the f2cl-lib:integer4 0)
                         (the f2cl-lib:integer4
                              (multiple-value-bind
                                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                                   var-6)
                                  (ilaenv ixover "DGEQRF" " " sm sn -1 -1)
                                (declare (ignore var-1 var-2 var-5 var-6))
                                (when var-0 (setf ixover var-0))
                                (when var-3 (setf sm var-3))
                                (when var-4 (setf sn var-4))
                                ret-val))))
            (cond
             ((< nx sminmn)
              (setf minws
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 sn)
                                        (f2cl-lib:int-mul
                                         (f2cl-lib:int-add sn 1) nb)))
              (setf iws
                      (max (the f2cl-lib:integer4 iws)
                           (the f2cl-lib:integer4 minws)))
              (cond
               ((< lwork minws)
                (setf nb
                        (the f2cl-lib:integer4
                             (truncate (- lwork (* 2 sn)) (+ sn 1))))
                (setf nbmin
                        (max (the f2cl-lib:integer4 2)
                             (the f2cl-lib:integer4
                                  (multiple-value-bind
                                      (ret-val var-0 var-1 var-2 var-3 var-4
                                       var-5 var-6)
                                      (ilaenv inbmin "DGEQRF" " " sm sn -1 -1)
```

```
                            (declare (ignore var-1 var-2 var-5 var-6))
                            (when var-0 (setf inbmin var-0))
                            (when var-3 (setf sm var-3))
                            (when var-4 (setf sn var-4))
                            ret-val)))))))))
(f2cl-lib:fdo (j (f2cl-lib:int-add nfxd 1) (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (dnrm2 sm
                 (f2cl-lib:array-slice a-%data% double-float
                                       ((+ nfxd 1) j) ((1 lda) (1 *))
                                       a-%offset%)
                 1)
              (declare (ignore var-1 var-2))
              (when var-0 (setf sm var-0))
              ret-val))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j)) ((1 *))
                         work-%offset%)
            (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%))
   label20))
(cond
 ((and (>= nb nbmin) (< nb sminmn) (< nx sminmn))
  (tagbody
    (setf j (f2cl-lib:int-add nfxd 1))
    (setf topbmn (f2cl-lib:int-sub minmn nx))
   label30
    (cond
     ((<= j topbmn)
      (setf jb
              (min (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub topbmn j)
                                          1))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12 var-13)
          (dlaqps m (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1)
           (f2cl-lib:int-sub j 1) jb fjb
           (f2cl-lib:array-slice a-%data% double-float (1 j)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice jpvt-%data% f2cl-lib:integer4 (j)
                                 ((1 *)) jpvt-%offset%)
           (f2cl-lib:array-slice tau-%data% double-float (j) ((1 *))
                                 tau-%offset%)
           (f2cl-lib:array-slice work-%data% double-float (j) ((1 *))
                                 work-%offset%)
           (f2cl-lib:array-slice work-%data% double-float ((+ n j))
```

```
                                        ((1 *)) work-%offset%)
                    (f2cl-lib:array-slice work-%data% double-float
                                        ((+ (f2cl-lib:int-mul 2 n) 1))
                                        ((1 *)) work-%offset%)
                    (f2cl-lib:array-slice work-%data% double-float
                                        ((+ (f2cl-lib:int-mul 2 n) jb 1))
                                        ((1 *)) work-%offset%)
                    (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1))
                 (declare
                  (ignore var-1 var-2 var-5 var-7 var-8 var-9 var-10 var-11
                   var-12 var-13))
                 (when var-0 (setf m var-0))
                 (when var-3 (setf jb var-3))
                 (when var-4 (setf fjb var-4))
                 (when var-6 (setf lda var-6)))
               (setf j (f2cl-lib:int-add j fjb)) (go label30)))))
          (t (setf j (f2cl-lib:int-add nfxd 1))))
         (if (<= j minmn)
             (multiple-value-bind
                 (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                 (dlaqp2 m (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1)
                  (f2cl-lib:int-sub j 1)
                  (f2cl-lib:array-slice a-%data% double-float (1 j)
                                        ((1 lda) (1 *)) a-%offset%)
                  lda
                  (f2cl-lib:array-slice jpvt-%data% f2cl-lib:integer4 (j)
                                        ((1 *)) jpvt-%offset%)
                  (f2cl-lib:array-slice tau-%data% double-float (j) ((1 *))
                                        tau-%offset%)
                  (f2cl-lib:array-slice work-%data% double-float (j) ((1 *))
                                        work-%offset%)
                  (f2cl-lib:array-slice work-%data% double-float ((+ n j))
                                        ((1 *)) work-%offset%)
                  (f2cl-lib:array-slice work-%data% double-float
                                        ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                                        work-%offset%))
               (declare
                (ignore var-1 var-2 var-3 var-5 var-6 var-7 var-8 var-9))
               (when var-0 (setf m var-0))
               (when var-4 (setf lda var-4))))))
       (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
               (coerce (the f2cl-lib:integer4 iws) 'double-float))
       (go end_label)
      end_label
       (return (values m n nil lda nil nil nil lwork info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgeqp3
```

```
        fortran-to-lisp::*f2cl-function-info*)
    (fortran-to-lisp::make-f2cl-finfo :arg-types
                                      '((fortran-to-lisp::integer4)
                                        (fortran-to-lisp::integer4)
                                        (array double-float (*))
                                        (fortran-to-lisp::integer4)
                                        (array fortran-to-lisp::integer4
                                         (*))
                                        (array double-float (*))
                                        (array double-float (*))
                                        (fortran-to-lisp::integer4)
                                        (fortran-to-lisp::integer4))
                                      :return-values
                                      '(fortran-to-lisp::m
                                        fortran-to-lisp::n nil
                                        fortran-to-lisp::lda nil nil nil
                                        fortran-to-lisp::lwork
                                        fortran-to-lisp::info)
                                      :calls '(fortran-to-lisp::dgeqrf))))
```

———-

32.16 dgeqpf - DEPRECATED use dgeqp3

32.17 dgeqr2

32.17.1 Axiom unit tests

— **dgeqr2.input** —

```
)set break resume
)sys rm -f dgeqr2.output
)spool dgeqr2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.17.2 Axiom help page

— dgeqr2.help —

```
=====================================================================
dgeqr2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgeqr2 (M, N, A, LDA, TAU, WORK, INFO)
           DGEQR2 computes the QR factorization of a general rectangular
           matrix using an unblocked algorithm.

Function/Subroutine Documentation
   subroutine dgeqr2 (integer M, integer N, double precision, dimension( lda,
       * ) A, integer LDA, double precision, dimension( * ) TAU, double
       precision, dimension( * ) WORK, integer INFO)
       DGEQR2 computes the QR factorization of a general rectangular matrix
       using an unblocked algorithm.

Purpose:
   DGEQR2 computes a QR factorization of a real m by n matrix A: A = Q * R.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the m by n matrix A.
   On exit, the elements on and above the diagonal of the array
   contain the min(m,n) by n upper trapezoidal matrix R (R is
   upper triangular if m >= n); the elements below the diagonal,
   with the array TAU, represent the orthogonal matrix Q as a
   product of elementary reflectors (see Further Details).

 LDA
   LDA is INTEGER
```

```
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is DOUBLE PRECISION array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is DOUBLE PRECISION array, dimension (N)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

Further Details:
   The matrix Q is represented as a product of elementary reflectors

               Q = H(1) H(2) . . . H(k), where k = min(m,n).

   Each H(i) has the form

               H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in A(i+1:m,i),
   and tau in TAU(i).
```

———-

32.17.3 fortran code

From Elmroth and Gustavson [Elmr00]:

DGEQR2(m,n,A,τ)
do $j = 1, n$
  compute Householder transform $Q(j) = I - \tau uu^T$ such that
  $Q(j)^T A(j\colon m, j) = (x, 0)^T$ via a call to DLARFG
  if (j .LT. n) **then**
    apply$Q(j)^T$ to $A(j\colon m, j+1\colon n)$ from the left by calling DLARF
  endif
enddo

— **dgeqr2.f** —

```
*  =====================================================================
      SUBROUTINE DGEQR2( M, N, A, LDA, TAU, WORK, INFO )
*
```

```
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      DOUBLE PRECISION   AII
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARF, DLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEQR2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = 1, K
*
*        Generate elementary reflector H(i) to annihilate A(i+1:m,i)
```

```
*
         CALL DLARFG( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
     $                TAU( I ) )
         IF( I.LT.N ) THEN
*
*           Apply H(i) to A(i:m,i+1:n) from the left
*
            AII = A( I, I )
            A( I, I ) = ONE
            CALL DLARF( 'Left', M-I+1, N-I, A( I, I ), 1, TAU( I ),
     $                  A( I, I+1 ), LDA, WORK )
            A( I, I ) = AII
         END IF
   10 CONTINUE
      RETURN
*
*     End of DGEQR2
*
      END
```

———-

32.17.4 lisp code

— **dgeqr2.lisp** —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dgeqr2p (m n a lda tau work info)
    (declare (type (array double-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((aii 0.0d0) (i 0) (k 0))
        (declare (type (double-float) aii)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "DGEQR2P" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
```

```
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i k) nil)
          (tagbody
            (dlarfgp (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:array-slice a-%data% double-float (i i) ((1 lda) (1 *))
                                   a-%offset%)
             (f2cl-lib:array-slice a-%data% double-float
                                   ((min (f2cl-lib:int-add i 1) m) i)
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                   tau-%offset%))
            (cond
             ((< i n)
              (setf aii
                      (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                      one)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                  (dlarf "Left" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
                   (f2cl-lib:int-sub n i)
                   (f2cl-lib:array-slice a-%data% double-float (i i)
                                         ((1 lda) (1 *)) a-%offset%)
                   1
                   (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                         tau-%offset%)
                   (f2cl-lib:array-slice a-%data% double-float
                                         (i (f2cl-lib:int-add i 1))
                                         ((1 lda) (1 *)) a-%offset%)
                   lda work)
                (declare
                 (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
                (when var-7 (setf lda var-7)))
              (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                      aii)))
           label10))
        (go end_label)
       end_label
        (return (values nil nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgeqr2p
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
```

```
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil fortran-to-lisp::lda
  nil nil fortran-to-lisp::info)
:calls 'nil)))
```

———-

32.18 dgeqr2p

32.18.1 Axiom unit tests

— **dgeqr2p.input** —

```
)set break resume
)sys rm -f dgeqr2p.output
)spool dgeqr2p.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.18.2 Axiom help page [Demm08]

— **dgeqr2p.help** —

```
==========================================================================
dgeqr2p examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine dgeqr2p (M, N, A, LDA, TAU, WORK, INFO)
           DGEQR2P computes the QR factorization of a general rectangular
           matrix with non-negative diagonal elements using an unblocked
           algorithm.

Function/Subroutine Documentation
   subroutine dgeqr2p (integer M, integer N, double precision, dimension( lda,
       * ) A, integer LDA, double precision, dimension( * ) TAU, double
       precision, dimension( * ) WORK, integer INFO)
       DGEQR2P computes the QR factorization of a general rectangular matrix
       with non-negative diagonal elements using an unblocked algorithm.

Purpose:
   DGEQR2 computes a QR factorization of a real m by n matrix A:
   A = Q * R. The diagonal entries of R are nonnegative.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the m by n matrix A.
   On exit, the elements on and above the diagonal of the array
   contain the min(m,n) by n upper trapezoidal matrix R (R is
   upper triangular if m >= n). The diagonal entries of R are
   nonnegative; the elements below the diagonal,
   with the array TAU, represent the orthogonal matrix Q as a
   product of elementary reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is DOUBLE PRECISION array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is DOUBLE PRECISION array, dimension (N)

 INFO
```

```
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

Further Details:
   The matrix Q is represented as a product of elementary reflectors

               Q = H(1) H(2) . . . H(k), where k = min(m,n).

   Each H(i) has the form

               H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in A(i+1:m,i),
   and tau in TAU(i).
```

32.18.3 fortran code

— dgeqr2p.f —

```
*  =====================================================================
      SUBROUTINE DGEQR2P( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      DOUBLE PRECISION   AII
```

```
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARF, DLARFGP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEQR2P', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = 1, K
*
*        Generate elementary reflector H(i) to annihilate A(i+1:m,i)
*
         CALL DLARFGP( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
     $                TAU( I ) )
         IF( I.LT.N ) THEN
*
*           Apply H(i) to A(i:m,i+1:n) from the left
*
            AII = A( I, I )
            A( I, I ) = ONE
            CALL DLARF( 'Left', M-I+1, N-I, A( I, I ), 1, TAU( I ),
     $                  A( I, I+1 ), LDA, WORK )
            A( I, I ) = AII
         END IF
   10 CONTINUE
      RETURN
*
*     End of DGEQR2P
*
      END
```

———-

32.18.4 lisp code

— dgeqr2p.lisp —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dgeqr2p (m n a lda tau work info)
    (declare (type (array double-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((aii 0.0d0) (i 0) (k 0))
        (declare (type (double-float) aii)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "DGEQR2P" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i k) nil)
          (tagbody
            (dlarfgp (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:array-slice a-%data% double-float (i i) ((1 lda) (1 *))
                                   a-%offset%)
             (f2cl-lib:array-slice a-%data% double-float
                                   ((min (f2cl-lib:int-add i 1) m) i)
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                   tau-%offset%))
            (cond
             ((< i n)
              (setf aii
                      (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                      one)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                  (dlarf "Left" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
                   (f2cl-lib:int-sub n i)
```

```
                  (f2cl-lib:array-slice a-%data% double-float (i i)
                                        ((1 lda) (1 *)) a-%offset%)
                  1
                  (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                        tau-%offset%)
                  (f2cl-lib:array-slice a-%data% double-float
                                        (i (f2cl-lib:int-add i 1))
                                        ((1 lda) (1 *)) a-%offset%)
                  lda work)
                (declare
                 (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
                (when var-7 (setf lda var-7)))
              (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                    aii)))
           label10))
        (go end_label)
       end_label
        (return (values nil nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgeqr2p
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::lda
                                              nil nil fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

32.19 dgeqrf

32.19.1 Axiom unit tests

— dgeqrf.input —

```
)set break resume
)sys rm -f dgeqrf.output
)spool dgeqrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.19.2 Axiom help page

— **dgeqrf.help** —

```
==========================================================================
dgeqrf examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgeqrf (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           DGEQRF

Function/Subroutine Documentation
   subroutine dgeqrf (integer M, integer N, double precision, dimension( lda,
       * ) A, integer LDA, double precision, dimension( * ) TAU, double
       precision, dimension( * ) WORK, integer LWORK, integer INFO)
       DGEQRF

Purpose:
   DGEQRF computes a QR factorization of a real M-by-N matrix A: A = Q * R.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.
```

```
 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, the elements on and above the diagonal of the array
   contain the min(M,N)-by-N upper trapezoidal matrix R (R is
   upper triangular if m >= n); the elements below the diagonal,
   with the array TAU, represent the orthogonal matrix Q as a
   product of min(m,n) elementary reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is DOUBLE PRECISION array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.  LWORK >= max(1,N).
   For optimum performance LWORK >= N*NB, where NB is
   the optimal blocksize.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details:
   The matrix Q is represented as a product of elementary reflectors

                Q = H(1) H(2) . . . H(k), where k = min(m,n).

   Each H(i) has the form

                H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in A(i+1:m,i),
   and tau in TAU(i).
```

———-

32.19.3 fortran code

From Elmroth and Gustavson [Elmr00]:

DGEQRF(m,n,A,τ,$work$)
do $j = 1, n, nb$!nb is the block size
  $jb = \min(j - j + 1, nb)$
  call DGEQR2$[m - j + 1, jb, A(j, j), \tau(j)]$
  if $(j + jb$ .LE. $n)$ **then**
    compute $T(1\colon jb, 1\colon jn)$ in $work$ via a call to DLARFT
    compute $(I - YT^TY^T)A(j\colon m, j + jb\colon n)$ using $work$ and T
    via a call to DLARFB
  endif
enddo

— **dgeqrf.f** —

```
*  =====================================================================
      SUBROUTINE DGEQRF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, LDWORK, LWKOPT, NB,
     $                   NBMIN, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEQR2, DLARFB, DLARFT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
```

```
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NB = ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
      LWKOPT = N*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEQRF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      K = MIN( M, N )
      IF( K.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      NX = 0
      IWS = N
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'DGEQRF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = N
            IWS = LDWORK*NB
```

```
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'DGEQRF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially
*
         DO 10 I = 1, K - NX, NB
            IB = MIN( K-I+1, NB )
*
*           Compute the QR factorization of the current block
*           A(i:m,i:i+ib-1)
*
            CALL DGEQR2( M-I+1, IB, A( I, I ), LDA, TAU( I ), WORK,
     $                   IINFO )
            IF( I+IB.LE.N ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i) H(i+1) . . . H(i+ib-1)
*
               CALL DLARFT( 'Forward', 'Columnwise', M-I+1, IB,
     $                      A( I, I ), LDA, TAU( I ), WORK, LDWORK )
*
*              Apply H**T to A(i:m,i+ib:n) from the left
*
               CALL DLARFB( 'Left', 'Transpose', 'Forward',
     $                      'Columnwise', M-I+1, N-I-IB+1, IB,
     $                      A( I, I ), LDA, WORK, LDWORK, A( I, I+IB ),
     $                      LDA, WORK( IB+1 ), LDWORK )
            END IF
   10    CONTINUE
      ELSE
         I = 1
      END IF
*
      Use unblocked code to factor the last or only block.
*
      IF( I.LE.K )
     $   CALL DGEQR2( M-I+1, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                IINFO )
*
```

```
      WORK( 1 ) = IWS
      RETURN
*
*     End of DGEQRF
*
      END
```

———-

32.19.4 lisp code

— **dgeqrf.lisp** —

```
(defun dgeqrfp (m n a lda tau work lwork info)
  (declare (type (array double-float (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a double-float a-%data% a-%offset%)
       (tau double-float tau-%data% tau-%offset%)
       (work double-float work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ldwork 0) (lwkopt 0) (nb 0)
           (nbmin 0) (nx 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) nx nbmin nb lwkopt ldwork k iws iinfo
                ib i))
      (setf info 0)
      (setf nb
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 1 "DGEQRF" " " m n -1 -1)
                (declare (ignore var-0 var-1 var-2 var-5 var-6))
                (when var-3 (setf m var-3))
                (when var-4 (setf n var-4))
                ret-val))
      (setf lwkopt (f2cl-lib:int-mul n nb))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4))
            ((and
              (< lwork
                 (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
              (not lquery))
             (setf info -7)))
      (cond
       ((/= info 0) (xerbla "DGEQRFP" (f2cl-lib:int-sub info)) (go end_label))
```

```
 (lquery (go end_label)))
(setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(cond
 ((= k 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'double-float))
  (go end_label)))
(setf nbmin 2)
(setf nx 0)
(setf iws n)
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "DGEQRF" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (when var-3 (setf m var-3))
                      (when var-4 (setf n var-4))
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork n) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
                            (ilaenv 2 "DGEQRF" " " m n -1 -1)
                          (declare
                           (ignore var-0 var-1 var-2 var-5 var-6))
                          (when var-3 (setf m var-3))
                          (when var-4 (setf n var-4))
                          ret-val)))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub nx))) nil)
    (tagbody
      (setf ib
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
               (the f2cl-lib:integer4 nb)))
```

```
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dgeqr2p (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
     (f2cl-lib:array-slice a-%data% double-float (i i)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                           tau-%offset%)
     work iinfo)
  (declare (ignore var-0 var-1 var-2 var-4 var-5))
  (setf lda var-3)
  (setf iinfo var-6))
(cond
 ((<= (f2cl-lib:int-add i ib) n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dlarft "Forward" "Columnwise"
       (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
       (f2cl-lib:array-slice a-%data% double-float (i i)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                             tau-%offset%)
       work ldwork)
    (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
    (when var-3 (setf ib var-3))
    (when var-5 (setf lda var-5))
    (when var-8 (setf ldwork var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (dlarfb "Left" "Transpose" "Forward" "Columnwise"
       (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
       (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1) ib
       (f2cl-lib:array-slice a-%data% double-float (i i)
                             ((1 lda) (1 *)) a-%offset%)
       lda work ldwork
       (f2cl-lib:array-slice a-%data% double-float
                             (i (f2cl-lib:int-add i ib))
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice work-%data% double-float ((+ ib 1))
                             ((1 *)) work-%offset%)
       ldwork)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
      var-13))
    (when var-6 (setf ib var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldwork var-10))
    (when var-12 (setf lda var-12))
```

```
                (when var-14 (setf ldwork var-14)))))
            label10)))
        (t (setf i 1)))
      (if (<= i k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (dgeqr2p (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
               (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
               (f2cl-lib:array-slice a-%data% double-float (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                     tau-%offset%)
               work iinfo)
            (declare (ignore var-0 var-1 var-2 var-4 var-5))
            (setf lda var-3)
            (setf iinfo var-6)))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 iws) 'double-float))
      (go end_label)
     end_label
      (return (values m n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgeqrfp
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::dgeqr2p))))
```

———-

32.20 dgeqrfp

32.20.1 Axiom unit tests

— **dgeqrfp.input** —

```
)set break resume
)sys rm -f dgeqrfp.output
)spool dgeqrfp.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.20.2 Axiom help page [Demm08]

— **dgeqrfp.help** —

```
======================================================================
dgeqrfp examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgeqrfp (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           DGEQRFP

Function/Subroutine Documentation
   subroutine dgeqrfp (integer M, integer N, double precision, dimension( lda,
       * ) A, integer LDA, double precision, dimension( * ) TAU, double
       precision, dimension( * ) WORK, integer LWORK, integer INFO)
       DGEQRFP

Purpose:
   DGEQRFP computes a QR factorization of a real M-by-N matrix A:
   A = Q * R. The diagonal entries of R are nonnegative.

Parameters:
```

```
 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, the elements on and above the diagonal of the array
   contain the min(M,N)-by-N upper trapezoidal matrix R (R is
   upper triangular if m >= n). The diagonal entries of R
   are nonnegative; the elements below the diagonal,
   with the array TAU, represent the orthogonal matrix Q as a
   product of min(m,n) elementary reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is DOUBLE PRECISION array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.  LWORK >= max(1,N).
   For optimum performance LWORK >= N*NB, where NB is
   the optimal blocksize.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details:
   The matrix Q is represented as a product of elementary reflectors
```

```
              Q = H(1) H(2) . . . H(k), where k = min(m,n).

   Each H(i) has the form

              H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in A(i+1:m,i),
   and tau in TAU(i).
```

———-

32.20.3 fortran code

— **dgeqrfp.f** —

```
*  =====================================================================
      SUBROUTINE DGEQRFP( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, LDWORK, LWKOPT, NB,
     $                   NBMIN, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEQR2P, DLARFB, DLARFT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
```

```
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NB = ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
      LWKOPT = N*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEQRFP', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      K = MIN( M, N )
      IF( K.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      NX = 0
      IWS = N
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'DGEQRF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = N
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
```

```
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'DGEQRF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially
*
         DO 10 I = 1, K - NX, NB
            IB = MIN( K-I+1, NB )
*
*           Compute the QR factorization of the current block
*           A(i:m,i:i+ib-1)
*
            CALL DGEQR2P( M-I+1, IB, A( I, I ), LDA, TAU( I ), WORK,
     $                    IINFO )
            IF( I+IB.LE.N ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i) H(i+1) . . . H(i+ib-1)
*
               CALL DLARFT( 'Forward', 'Columnwise', M-I+1, IB,
     $                      A( I, I ), LDA, TAU( I ), WORK, LDWORK )
*
*              Apply H**T to A(i:m,i+ib:n) from the left
*
               CALL DLARFB( 'Left', 'Transpose', 'Forward',
     $                      'Columnwise', M-I+1, N-I-IB+1, IB,
     $                      A( I, I ), LDA, WORK, LDWORK, A( I, I+IB ),
     $                      LDA, WORK( IB+1 ), LDWORK )
            END IF
   10    CONTINUE
      ELSE
         I = 1
      END IF
*
*     Use unblocked code to factor the last or only block.
*
      IF( I.LE.K )
     $   CALL DGEQR2P( M-I+1, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                 IINFO )
*
      WORK( 1 ) = IWS
      RETURN
```

```
*
*     End of DGEQRFP
*
      END
```

———-

32.20.4 lisp code

— **dgeqrfp.lisp** —

```
(defun dgeqrfp (m n a lda tau work lwork info)
  (declare (type (array double-float (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a double-float a-%data% a-%offset%)
       (tau double-float tau-%data% tau-%offset%)
       (work double-float work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ldwork 0) (lwkopt 0) (nb 0)
           (nbmin 0) (nx 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) nx nbmin nb lwkopt ldwork k iws iinfo
                ib i))
      (setf info 0)
      (setf nb
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 1 "DGEQRF" " " m n -1 -1)
                (declare (ignore var-0 var-1 var-2 var-5 var-6))
                (when var-3 (setf m var-3))
                (when var-4 (setf n var-4))
                ret-val))
      (setf lwkopt (f2cl-lib:int-mul n nb))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4))
            ((and
              (< lwork
                 (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
              (not lquery))
             (setf info -7)))
      (cond
       ((/= info 0) (xerbla "DGEQRFP" (f2cl-lib:int-sub info)) (go end_label))
       (lquery (go end_label)))
      (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
```

```
(cond
 ((= k 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'double-float))
  (go end_label)))
(setf nbmin 2)
(setf nx 0)
(setf iws n)
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "DGEQRF" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (when var-3 (setf m var-3))
                      (when var-4 (setf n var-4))
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork n) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
                            (ilaenv 2 "DGEQRF" " " m n -1 -1)
                          (declare
                           (ignore var-0 var-1 var-2 var-5 var-6))
                          (when var-3 (setf m var-3))
                          (when var-4 (setf n var-4))
                          ret-val))))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub nx))) nil)
    (tagbody
      (setf ib
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
               (the f2cl-lib:integer4 nb)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dgeqr2p (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
```

```
        (f2cl-lib:array-slice a-%data% double-float (i i)
                              ((1 lda) (1 *)) a-%offset%)
        lda
        (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                              tau-%offset%)
        work iinfo)
    (declare (ignore var-0 var-1 var-2 var-4 var-5))
    (setf lda var-3)
    (setf iinfo var-6))
  (cond
   ((<= (f2cl-lib:int-add i ib) n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dlarft "Forward" "Columnwise"
         (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
         (f2cl-lib:array-slice a-%data% double-float (i i)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                               tau-%offset%)
         work ldwork)
      (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
      (when var-3 (setf ib var-3))
      (when var-5 (setf lda var-5))
      (when var-8 (setf ldwork var-8)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14)
        (dlarfb "Left" "Transpose" "Forward" "Columnwise"
         (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
         (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1) ib
         (f2cl-lib:array-slice a-%data% double-float (i i)
                               ((1 lda) (1 *)) a-%offset%)
         lda work ldwork
         (f2cl-lib:array-slice a-%data% double-float
                               (i (f2cl-lib:int-add i ib))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice work-%data% double-float ((+ ib 1))
                               ((1 *)) work-%offset%)
         ldwork)
      (declare
       (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
        var-13))
      (when var-6 (setf ib var-6))
      (when var-8 (setf lda var-8))
      (when var-10 (setf ldwork var-10))
      (when var-12 (setf lda var-12))
      (when var-14 (setf ldwork var-14)))))
label10)))
```

```
       (t (setf i 1)))
      (if (<= i k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (dgeqr2p (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
               (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
               (f2cl-lib:array-slice a-%data% double-float (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                     tau-%offset%)
               work iinfo)
            (declare (ignore var-0 var-1 var-2 var-4 var-5))
            (setf lda var-3)
            (setf iinfo var-6)))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 iws) 'double-float))
      (go end_label)
     end_label
      (return (values m n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgeqrfp
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::dgeqr2p))))
```

———-

32.21 dgeqrt

32.21.1 Axiom unit tests

— dgeqrt.input —

```
)set break resume
)sys rm -f dgeqrt.output
)spool dgeqrt.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.21.2 Axiom help page

— dgeqrt.help —

```
======================================================================
dgeqrt examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgeqrt (M, N, NB, A, LDA, T, LDT, WORK, INFO)
           DGEQRT

Function/Subroutine Documentation
   subroutine dgeqrt (integer M, integer N, integer NB, double precision,
       dimension( lda, * ) A, integer LDA, double precision, dimension( ldt, *
       ) T, integer LDT, double precision, dimension( * ) WORK, integer INFO)
       DGEQRT

Purpose:
   DGEQRT computes a blocked QR factorization of a real M-by-N matrix A
   using the compact WY representation of Q.

Parameters:
```

```
 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 NB
   NB is INTEGER
   The block size to be used in the blocked QR.  MIN(M,N) >= NB >= 1.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, the elements on and above the diagonal of the array
   contain the min(M,N)-by-N upper trapezoidal matrix R (R is
   upper triangular if M >= N); the elements below the diagonal
   are the columns of V.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 T
   T is DOUBLE PRECISION array, dimension (LDT,MIN(M,N))
   The upper triangular block reflectors stored in compact form
   as a sequence of upper triangular blocks.  See below for further details.

 LDT
   LDT is INTEGER
   The leading dimension of the array T.  LDT >= NB.

 WORK
   WORK is DOUBLE PRECISION array, dimension (NB*N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details:
   The matrix V stores the elementary reflectors H(i) in the i-th column
   below the diagonal. For example, if M=5 and N=3, the matrix V is

                         V = (  1       )
                             ( v1  1    )
                             ( v1 v2  1 )
                             ( v1 v2 v3 )
```

```
                              ( v1 v2 v3 )

   where the vi's represent the vectors which define H(i), which are returned
   in the matrix A.  The 1's along the diagonal of V are not stored in A.

   Let K=MIN(M,N).  The number of blocks is B = ceiling(K/NB), where each
   block is of order NB except for the last block, which is of order
   IB = K - (B-1)*NB.  For each of the B blocks, a upper triangular block
   reflector factor is computed: T1, T2, ..., TB.  The NB-by-NB (and IB-by-IB
   for the last block) T's are stored in the NB-by-N matrix T as

                         T = (T1 T2 ... TB).
```

32.21.3 fortran code

— dgeqrt.f —

```
*  =====================================================================
      SUBROUTINE DGEQRT( M, N, NB, A, LDA, T, LDT, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      INTEGER INFO, LDA, LDT, M, N, NB
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A( LDA, * ), T( LDT, * ), WORK( * )
*     ..
*
* =====================================================================
*
*     ..
*     .. Local Scalars ..
      INTEGER    I, IB, IINFO, K
      LOGICAL    USE_RECURSIVE_QR
      PARAMETER( USE_RECURSIVE_QR=.TRUE. )
*     ..
*     .. External Subroutines ..
      EXTERNAL   DGEQRT2, DGEQRT3, DLARFB, XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
```

```
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NB.LT.1 .OR. ( NB.GT.MIN(M,N) .AND. MIN(M,N).GT.0 ) )THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDT.LT.NB ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEQRT', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      K = MIN( M, N )
      IF( K.EQ.0 ) RETURN
*
*     Blocked loop of length K
*
      DO I = 1, K,  NB
         IB = MIN( K-I+1, NB )
*
*     Compute the QR factorization of the current block A(I:M,I:I+IB-1)
*
         IF( USE_RECURSIVE_QR ) THEN
            CALL DGEQRT3( M-I+1, IB, A(I,I), LDA, T(1,I), LDT, IINFO )
         ELSE
            CALL DGEQRT2( M-I+1, IB, A(I,I), LDA, T(1,I), LDT, IINFO )
         END IF
         IF( I+IB.LE.N ) THEN
*
*     Update by applying H**T to A(I:M,I+IB:N) from the left
*
            CALL DLARFB( 'L', 'T', 'F', 'C', M-I+1, N-I-IB+1, IB,
     $                   A( I, I ), LDA, T( 1, I ), LDT,
     $                   A( I, I+IB ), LDA, WORK , N-I-IB+1 )
         END IF
      END DO
      RETURN
*
*     End of DGEQRT
*
      END
```

———-

32.21.4 lisp code

— dgeqrt.lisp —

```
(let* ((use_recursive_qr f2cl-lib:%true%))
  (declare (type f2cl-lib:logical use_recursive_qr)
           (ignorable use_recursive_qr))
  (defun dgeqrt (m n nb a lda t$ ldt work info)
    (declare (type (array double-float (*)) work t$ a)
             (type (f2cl-lib:integer4) info ldt lda nb n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (t$ double-float t$-%data% t$-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((i 0) (ib 0) (iinfo 0) (k 0))
        (declare (type (f2cl-lib:integer4) i ib iinfo k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((or (< nb 1)
                   (and
                    (> nb
                       (min (the f2cl-lib:integer4 m)
                            (the f2cl-lib:integer4 n)))
                    (>
                     (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n))
                     0)))
               (setf info -3))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -5))
              ((< ldt nb) (setf info -7)))
        (cond
         ((/= info 0) (xerbla "DGEQRT" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (if (= k 0)
            (go end_label))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                      ((> i k) nil)
          (tagbody
            (setf ib
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
                     (the f2cl-lib:integer4 nb)))
            (cond
```

```
 (use_recursive_qr
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dgeqrt3 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
       (f2cl-lib:array-slice a-%data% double-float (i i)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice t$-%data% double-float (1 i)
                             ((1 ldt) (1 *)) t$-%offset%)
       ldt iinfo)
    (declare (ignore var-0 var-2 var-4))
    (when var-1 (setf ib var-1))
    (when var-3 (setf lda var-3))
    (when var-5 (setf ldt var-5))
    (when var-6 (setf iinfo var-6))))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dgeqrt2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
       (f2cl-lib:array-slice a-%data% double-float (i i)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice t$-%data% double-float (1 i)
                             ((1 ldt) (1 *)) t$-%offset%)
       ldt iinfo)
    (declare (ignore var-0 var-1 var-2 var-4))
    (setf lda var-3)
    (setf ldt var-5)
    (setf iinfo var-6))))
(cond
 ((<= (f2cl-lib:int-add i ib) n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (dlarfb "L" "T" "F" "C"
       (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
       (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1) ib
       (f2cl-lib:array-slice a-%data% double-float (i i)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice t$-%data% double-float (1 i)
                             ((1 ldt) (1 *)) t$-%offset%)
       ldt
       (f2cl-lib:array-slice a-%data% double-float
                             (i (f2cl-lib:int-add i ib))
                             ((1 lda) (1 *)) a-%offset%)
       lda work (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1))
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
      var-13 var-14))
    (when var-6 (setf ib var-6))
    (when var-8 (setf lda var-8))
```

```
                (when var-10 (setf ldt var-10))
                (when var-12 (setf lda var-12)))))
           label100000))
        (go end_label)
       end_label
        (return (values nil nil nil nil lda nil ldt nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgeqrt
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldt nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::dgeqrt2))))
```

———-

32.22 dgeqrt2

32.22.1 Axiom unit tests

— dgeqrt2.input —

```
)set break resume
)sys rm -f dgeqrt2.output
)spool dgeqrt2.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

32.22.2 Axiom help page

— **dgeqrt2.help** —

```
=====================================================================
dgeqrt2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgeqrt2 (M, N, A, LDA, T, LDT, INFO)
           DGEQRT2 computes a QR factorization of a general real or complex
           matrix using the compact WY representation of Q.

Function/Subroutine Documentation
   subroutine dgeqrt2 (integer M, integer N, double precision, dimension( lda,
       * ) A, integer LDA, double precision, dimension( ldt, * ) T, integer
       LDT, integer INFO)
       DGEQRT2 computes a QR factorization of a general real or complex matrix
       using the compact WY representation of Q.

Purpose:
   DGEQRT2 computes a QR factorization of a real M-by-N matrix A,
   using the compact WY representation of Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= N.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the real M-by-N matrix A.  On exit, the elements on and
   above the diagonal contain the N-by-N upper triangular matrix R; the
```

```
   elements below the diagonal are the columns of V.  See below for
   further details.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 T
   T is DOUBLE PRECISION array, dimension (LDT,N)
   The N-by-N upper triangular factor of the block reflector.
   The elements on and above the diagonal contain the block
   reflector T; the elements below the diagonal are not used.
   See below for further details.

 LDT
   LDT is INTEGER
   The leading dimension of the array T.  LDT >= max(1,N).

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

Further Details:
   The matrix V stores the elementary reflectors H(i) in the i-th column
   below the diagonal. For example, if M=5 and N=3, the matrix V is

                        V = (  1       )
                            ( v1  1    )
                            ( v1 v2  1 )
                            ( v1 v2 v3 )
                            ( v1 v2 v3 )

   where the vi's represent the vectors which define H(i), which are returned
   in the matrix A.  The 1's along the diagonal of V are not stored in A.  The
   block reflector H is then given by

                        H = I - V * T * V**T

   where V**T is the transpose of V.
```

———-

32.22.3 fortran code

— dgeqrt2.f —

```
*  =====================================================================
      SUBROUTINE DGEQRT2( M, N, A, LDA, T, LDT, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER   INFO, LDA, LDT, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), T( LDT, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION  ONE, ZERO
      PARAMETER( ONE = 1.0D+00, ZERO = 0.0D+00 )
*     ..
*     .. Local Scalars ..
      INTEGER   I, K
      DOUBLE PRECISION   AII, ALPHA
*     ..
*     .. External Subroutines ..
      EXTERNAL  DLARFG, DGEMV, DGER, DTRMV, XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LDT.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEQRT2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO I = 1, K
```

```
*
*        Generate elem. refl. H(i) to annihilate A(i+1:m,i), tau(I) -> T(I,1)
*
         CALL DLARFG( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
     $                T( I, 1 ) )
         IF( I.LT.N ) THEN
*
*           Apply H(i) to A(I:M,I+1:N) from the left
*
            AII = A( I, I )
            A( I, I ) = ONE
*
*           W(1:N-I) := A(I:M,I+1:N)^H * A(I:M,I) [W = T(:,N)]
*
            CALL DGEMV( 'T',M-I+1, N-I, ONE, A( I, I+1 ), LDA,
     $                  A( I, I ), 1, ZERO, T( 1, N ), 1 )
*
*           A(I:M,I+1:N) = A(I:m,I+1:N) + alpha*A(I:M,I)*W(1:N-1)^H
*
            ALPHA = -(T( I, 1 ))
            CALL DGER( M-I+1, N-I, ALPHA, A( I, I ), 1,
     $           T( 1, N ), 1, A( I, I+1 ), LDA )
            A( I, I ) = AII
         END IF
      END DO
*
      DO I = 2, N
         AII = A( I, I )
         A( I, I ) = ONE
*
*        T(1:I-1,I) := alpha * A(I:M,1:I-1)**T * A(I:M,I)
*
         ALPHA = -T( I, 1 )
         CALL DGEMV( 'T', M-I+1, I-1, ALPHA, A( I, 1 ), LDA,
     $               A( I, I ), 1, ZERO, T( 1, I ), 1 )
         A( I, I ) = AII
*
*        T(1:I-1,I) := T(1:I-1,1:I-1) * T(1:I-1,I)
*
         CALL DTRMV( 'U', 'N', 'N', I-1, T, LDT, T( 1, I ), 1 )
*
*           T(I,I) = tau(I)
*
            T( I, I ) = T( I, 1 )
            T( I, 1) = ZERO
      END DO

*
*     End of DGEQRT2
*
```

```
      END
```

———-

32.22.4 lisp code

— **dgeqrt2.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dgeqrt2 (m n a lda t$ ldt info)
    (declare (type (array double-float (*)) t$ a)
             (type (f2cl-lib:integer4) info ldt lda n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (t$ double-float t$-%data% t$-%offset%))
      (prog ((aii 0.0d0) (alpha 0.0d0) (i 0) (k 0))
        (declare (type (double-float) aii alpha)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4))
              ((< ldt
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -6)))
        (cond
         ((/= info 0) (xerbla "DGEQRT2" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i k) nil)
          (tagbody
            (dlarfg (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:array-slice a-%data% double-float (i i) ((1 lda) (1 *))
                                   a-%offset%)
             (f2cl-lib:array-slice a-%data% double-float
                                   ((min (f2cl-lib:int-add i 1) m) i)
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice t$-%data% double-float (i 1) ((1 ldt) (1 *))
                                   t$-%offset%))
            (cond
             ((< i n)
              (setf aii
```

```
                (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                               a-%offset%))
        (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                one)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10)
            (dgemv "T" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:int-sub n i) one
             (f2cl-lib:array-slice a-%data% double-float
                                   (i (f2cl-lib:int-add i 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice a-%data% double-float (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             1 zero
             (f2cl-lib:array-slice t$-%data% double-float (1 n)
                                   ((1 ldt) (1 *)) t$-%offset%)
             1)
          (declare
           (ignore var-0 var-1 var-2 var-4 var-6 var-7 var-9 var-10))
          (when var-3 (setf one var-3))
          (when var-5 (setf lda var-5))
          (when var-8 (setf zero var-8)))
        (setf alpha
                (-
                 (f2cl-lib:fref t$-%data% (i 1) ((1 ldt) (1 *))
                                t$-%offset%)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (dger (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:int-sub n i) alpha
             (f2cl-lib:array-slice a-%data% double-float (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice t$-%data% double-float (1 n)
                                   ((1 ldt) (1 *)) t$-%offset%)
             1
             (f2cl-lib:array-slice a-%data% double-float
                                   (i (f2cl-lib:int-add i 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7))
          (when var-2 (setf alpha var-2))
          (when var-8 (setf lda var-8)))
        (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                aii)))
    label100000))
  (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                ((> i n) nil)
```

```
          (tagbody
            (setf aii
                    (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%))
            (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                    one)
            (setf alpha
                    (-
                     (f2cl-lib:fref t$-%data% (i 1) ((1 ldt) (1 *))
                                    t$-%offset%)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                 var-10)
                (dgemv "T" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
                 (f2cl-lib:int-sub i 1) alpha
                 (f2cl-lib:array-slice a-%data% double-float (i 1)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice a-%data% double-float (i i)
                                       ((1 lda) (1 *)) a-%offset%)
                 1 zero
                 (f2cl-lib:array-slice t$-%data% double-float (1 i)
                                       ((1 ldt) (1 *)) t$-%offset%)
                 1)
              (declare
               (ignore var-0 var-1 var-2 var-4 var-6 var-7 var-9 var-10))
              (when var-3 (setf alpha var-3))
              (when var-5 (setf lda var-5))
              (when var-8 (setf zero var-8)))
            (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                    aii)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
                (dtrmv "U" "N" "N" (f2cl-lib:int-sub i 1) t$ ldt
                 (f2cl-lib:array-slice t$-%data% double-float (1 i)
                                       ((1 ldt) (1 *)) t$-%offset%)
                 1)
              (declare (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7))
              (when var-5 (setf ldt var-5)))
            (setf (f2cl-lib:fref t$-%data% (i i) ((1 ldt) (1 *)) t$-%offset%)
                    (f2cl-lib:fref t$-%data% (i 1) ((1 ldt) (1 *))
                                   t$-%offset%))
            (setf (f2cl-lib:fref t$-%data% (i 1) ((1 ldt) (1 *)) t$-%offset%)
                    zero)
           label100001))
        end_label
         (return (values nil nil nil lda nil ldt info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
```

```
(setf (gethash 'fortran-to-lisp::dgeqrt2
               fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (array double-float (*))
                                            (fortran-to-lisp::integer4)
                                            (array double-float (*))
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4))
                                          :return-values
                                          '(nil nil nil fortran-to-lisp::lda
                                            nil fortran-to-lisp::ldt
                                            fortran-to-lisp::info)
                                          :calls 'nil)))
```

———-

32.23 dgeqrt3

32.23.1 Axiom unit tests

— **dgeqrt3.input** —

```
)set break resume
)sys rm -f dgeqrt3.output
)spool dgeqrt3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.23.2 Axiom help page [Elmr00]

— **dgeqrt3.help** —

```
==========================================================================
dgeqrt3 examples
```

```
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       recursive subroutine dgeqrt3 (M, N, A, LDA, T, LDT, INFO)
           DGEQRT3 recursively computes a QR factorization of a general real
           or complex matrix using the compact WY representation of Q.

Function/Subroutine Documentation
   recursive subroutine dgeqrt3 (integer M, integer N, double precision,
       dimension( lda, * ) A, integer LDA, double precision, dimension( ldt, *
       ) T, integer LDT, integer INFO)
       DGEQRT3 recursively computes a QR factorization of a general real or
       complex matrix using the compact WY representation of Q.

Purpose:
   DGEQRT3 recursively computes a QR factorization of a real M-by-N
   matrix A, using the compact WY representation of Q.

       Parameters:
           M

                     M is INTEGER
                     The number of rows of the matrix A.  M >= N.

           N

                     N is INTEGER
                     The number of columns of the matrix A.  N >= 0.

           A

                     A is DOUBLE PRECISION array, dimension (LDA,N)
                     On entry, the real M-by-N matrix A.  On exit, the elements on and
                     above the diagonal contain the N-by-N upper triangular matrix R; the
                     elements below the diagonal are the columns of V.  See below for
                     further details.

           LDA

                     LDA is INTEGER
                     The leading dimension of the array A.  LDA >= max(1,M).

           T

                     T is DOUBLE PRECISION array, dimension (LDT,N)
```

```
                The N-by-N upper triangular factor of the block reflector.
                The elements on and above the diagonal contain the block
                reflector T; the elements below the diagonal are not used.
                See below for further details.

        LDT

                LDT is INTEGER
                The leading dimension of the array T.  LDT >= max(1,N).

        INFO

                INFO is INTEGER
                = 0: successful exit
                < 0: if INFO = -i, the i-th argument had an illegal value

    Author:
        Univ. of Tennessee

        Univ. of California Berkeley

        Univ. of Colorado Denver

        NAG Ltd.

    Date:
        September 2012

    Further Details:


          The matrix V stores the elementary reflectors H(i) in the i-th column
          below the diagonal. For example, if M=5 and N=3, the matrix V is

                       V = (  1       )
                           ( v1  1    )
                           ( v1 v2  1 )
                           ( v1 v2 v3 )
                           ( v1 v2 v3 )

          where the vi's represent the vectors which define H(i), which are returned
          in the matrix A.  The 1's along the diagonal of V are not stored in A.  The
          block reflector H is then given by

                       H = I - V * T * V**T

          where V**T is the transpose of V.

          For details of the algorithm, see Elmroth and Gustavson (cited above).
```

Author
Generated automatically by Doxygen for LAPACK from the source code.

Version 3.6.0 Mon Nov 16 2015 dgeqrt3.f(3)

———-

32.23.3 fortran code

— **dgeqrt3.f** —

```
*  ===================================================================
      RECURSIVE SUBROUTINE DGEQRT3( M, N, A, LDA, T, LDT, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER   INFO, LDA, M, N, LDT
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), T( LDT, * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER ( ONE = 1.0D+00 )
*     ..
*     .. Local Scalars ..
      INTEGER   I, I1, J, J1, N1, N2, IINFO
*     ..
*     .. External Subroutines ..
      EXTERNAL  DLARFG, DTRMM, DGEMM, XERBLA
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      IF( N .LT. 0 ) THEN
         INFO = -2
      ELSE IF( M .LT. N ) THEN
         INFO = -1
```

```
      ELSE IF( LDA .LT. MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LDT .LT. MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEQRT3', -INFO )
         RETURN
      END IF
*
      IF( N.EQ.1 ) THEN
*
*        Compute Householder transform when N=1
*
         CALL DLARFG( M, A, A( MIN( 2, M ), 1 ), 1, T )
*
      ELSE
*
*        Otherwise, split A into blocks...
*
         N1 = N/2
         N2 = N-N1
         J1 = MIN( N1+1, N )
         I1 = MIN( N+1, M )
*
*        Compute A(1:M,1:N1) <- (Y1,R1,T1), where Q1 = I - Y1 T1 Y1^H
*
         CALL DGEQRT3( M, N1, A, LDA, T, LDT, IINFO )
*
*        Compute A(1:M,J1:N) = Q1^H A(1:M,J1:N) [workspace: T(1:N1,J1:N)]
*
         DO J=1,N2
            DO I=1,N1
               T( I, J+N1 ) = A( I, J+N1 )
            END DO
         END DO
         CALL DTRMM( 'L', 'L', 'T', 'U', N1, N2, ONE,
     &               A, LDA, T( 1, J1 ), LDT )
*
         CALL DGEMM( 'T', 'N', N1, N2, M-N1, ONE, A( J1, 1 ), LDA,
     &               A( J1, J1 ), LDA, ONE, T( 1, J1 ), LDT)
*
         CALL DTRMM( 'L', 'U', 'T', 'N', N1, N2, ONE,
     &               T, LDT, T( 1, J1 ), LDT )
*
         CALL DGEMM( 'N', 'N', M-N1, N2, N1, -ONE, A( J1, 1 ), LDA,
     &               T( 1, J1 ), LDT, ONE, A( J1, J1 ), LDA )
*
         CALL DTRMM( 'L', 'L', 'N', 'U', N1, N2, ONE,
     &               A, LDA, T( 1, J1 ), LDT )
```

```
*
         DO J=1,N2
            DO I=1,N1
               A( I, J+N1 ) = A( I, J+N1 ) - T( I, J+N1 )
            END DO
         END DO
*
*        Compute A(J1:M,J1:N) <- (Y2,R2,T2) where Q2 = I - Y2 T2 Y2^H
*
         CALL DGEQRT3( M-N1, N2, A( J1, J1 ), LDA,
     &                T( J1, J1 ), LDT, IINFO )
*
*        Compute T3 = T(1:N1,J1:N) = -T1 Y1^H Y2 T2
*
         DO I=1,N1
            DO J=1,N2
               T( I, J+N1 ) = (A( J+N1, I ))
            END DO
         END DO
*
         CALL DTRMM( 'R', 'L', 'N', 'U', N1, N2, ONE,
     &               A( J1, J1 ), LDA, T( 1, J1 ), LDT )
*
         CALL DGEMM( 'T', 'N', N1, N2, M-N, ONE, A( I1, 1 ), LDA,
     &               A( I1, J1 ), LDA, ONE, T( 1, J1 ), LDT )
*
         CALL DTRMM( 'L', 'U', 'N', 'N', N1, N2, -ONE, T, LDT,
     &               T( 1, J1 ), LDT )
*
         CALL DTRMM( 'R', 'U', 'N', 'N', N1, N2, ONE,
     &               T( J1, J1 ), LDT, T( 1, J1 ), LDT )
*
*        Y = (Y1,Y2); R = [ R1  A(1:N1,J1:N) ];  T = [T1 T3]
*                         [  0        R2     ]       [ 0 T2]
*
      END IF
*
      RETURN
*
*     End of DGEQRT3
*
      END
```

———-

32.23.4 lisp code

— dgeqrt3.lisp —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun *main* ()
    (prog ((i 0) (i1 0) (j 0) (j1 0) (n1 0) (n2 0) (iinfo 0) (m 0) (info 0)
           (ldt 0)
           (t$
            (make-array (the fixnum (reduce #'* (list ldt *))) :element-type
                        'double-float))
           (lda 0)
           (a
            (make-array (the fixnum (reduce #'* (list lda *))) :element-type
                        'double-float))
           (n 0) (subroutine 0.0))
      (declare (type (single-float) subroutine)
               (type (array double-float (*)) a t$)
               (type (f2cl-lib:integer4) n lda ldt info i i1 j j1 n1 n2 iinfo
                m))
      '"****NOT TRANSLATED: (RECURSIVE SUBROUTINE DGEQRT3 (M |,| N |,| A |,| LDA |,| T |,| LDT |,| INFO))"
      (setf info 0)
      (cond ((< n 0) (setf info -2)) ((< m n) (setf info -1))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4))
            ((< ldt (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -6)))
      (cond
       ((/= info 0) (xerbla "DGEQRT3" (f2cl-lib:int-sub info)) (go end_label)))
      (cond
       ((= n 1)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dlarfg m a
             (f2cl-lib:array-slice a double-float ((min 2 m) 1)
                                   ((1 lda) (1 *)))
             1 t$)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf m var-0))))
       (t (setf n1 (the f2cl-lib:integer4 (truncate n 2)))
        (setf n2 (f2cl-lib:int-sub n n1))
        (setf j1
                (min (the f2cl-lib:integer4 (f2cl-lib:int-add n1 1))
                     (the f2cl-lib:integer4 n)))
        (setf i1
                (min (the f2cl-lib:integer4 (f2cl-lib:int-add n 1))
                     (the f2cl-lib:integer4 m)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dgeqrt3 m n1 a lda t$ ldt iinfo)
          (declare (ignore var-2 var-4))
          (when var-0 (setf m var-0))
          (when var-1 (setf n1 var-1))
          (when var-3 (setf lda var-3))
```

```
  (when var-5 (setf ldt var-5))
  (when var-6 (setf iinfo var-6)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n2) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n1) nil)
      (tagbody
        (setf (f2cl-lib:fref t$ (i (f2cl-lib:int-add j n1))
                              ((1 ldt) (1 *)))
                (f2cl-lib:fref a (i (f2cl-lib:int-add j n1))
                               ((1 lda) (1 *))))
       label100001))
   label100000))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "L" "L" "T" "U" n1 n2 one a lda
     (f2cl-lib:array-slice t$ double-float (1 j1) ((1 ldt) (1 *))) ldt)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf n2 var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf ldt var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (dgemm "T" "N" n1 n2 (f2cl-lib:int-sub m n1) one
     (f2cl-lib:array-slice a double-float (j1 1) ((1 lda) (1 *))) lda
     (f2cl-lib:array-slice a double-float (j1 j1) ((1 lda) (1 *))) lda
     one (f2cl-lib:array-slice t$ double-float (1 j1) ((1 ldt) (1 *)))
     ldt)
  (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
  (when var-2 (setf n1 var-2))
  (when var-3 (setf n2 var-3))
  (when var-5 (setf one var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf ldt var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "L" "U" "T" "N" n1 n2 one t$ ldt
     (f2cl-lib:array-slice t$ double-float (1 j1) ((1 ldt) (1 *))) ldt)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf n2 var-5))
  (when var-6 (setf one var-6))
```

```
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldt var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (dgemm "N" "N" (f2cl-lib:int-sub m n1) n2 n1 (- one)
     (f2cl-lib:array-slice a double-float (j1 1) ((1 lda) (1 *))) lda
     (f2cl-lib:array-slice t$ double-float (1 j1) ((1 ldt) (1 *))) ldt
     one (f2cl-lib:array-slice a double-float (j1 j1) ((1 lda) (1 *)))
     lda)
  (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
  (when var-3 (setf n2 var-3))
  (when var-4 (setf n1 var-4))
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldt var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf lda var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "L" "L" "N" "U" n1 n2 one a lda
     (f2cl-lib:array-slice t$ double-float (1 j1) ((1 ldt) (1 *))) ldt)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf n2 var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf ldt var-10)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n2) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n1) nil)
      (tagbody
        (setf (f2cl-lib:fref a (i (f2cl-lib:int-add j n1))
                              ((1 lda) (1 *)))
                (-
                 (f2cl-lib:fref a (i (f2cl-lib:int-add j n1))
                                  ((1 lda) (1 *)))
                 (f2cl-lib:fref t$ (i (f2cl-lib:int-add j n1))
                                  ((1 ldt) (1 *)))))
       label100003))
   label100002))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dgeqrt3 (f2cl-lib:int-sub m n1) n2
     (f2cl-lib:array-slice a double-float (j1 j1) ((1 lda) (1 *))) lda
     (f2cl-lib:array-slice t$ double-float (j1 j1) ((1 ldt) (1 *))) ldt
     iinfo)
  (declare (ignore var-0 var-2 var-4))
  (when var-1 (setf n2 var-1))
```

```
  (when var-3 (setf lda var-3))
  (when var-5 (setf ldt var-5))
  (when var-6 (setf iinfo var-6)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n1) nil)
  (tagbody
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n2) nil)
      (tagbody
        (setf (f2cl-lib:fref t$ (i (f2cl-lib:int-add j n1))
                              ((1 ldt) (1 *)))
                (f2cl-lib:fref a ((f2cl-lib:int-add j n1) i)
                                 ((1 lda) (1 *))))
       label100005))
   label100004))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "R" "L" "N" "U" n1 n2 one
     (f2cl-lib:array-slice a double-float (j1 j1) ((1 lda) (1 *))) lda
     (f2cl-lib:array-slice t$ double-float (1 j1) ((1 ldt) (1 *))) ldt)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf n2 var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf ldt var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (dgemm "T" "N" n1 n2 (f2cl-lib:int-sub m n) one
     (f2cl-lib:array-slice a double-float (i1 1) ((1 lda) (1 *))) lda
     (f2cl-lib:array-slice a double-float (i1 j1) ((1 lda) (1 *))) lda
     one (f2cl-lib:array-slice t$ double-float (1 j1) ((1 ldt) (1 *)))
     ldt)
  (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
  (when var-2 (setf n1 var-2))
  (when var-3 (setf n2 var-3))
  (when var-5 (setf one var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf ldt var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "L" "U" "N" "N" n1 n2 (- one) t$ ldt
     (f2cl-lib:array-slice t$ double-float (1 j1) ((1 ldt) (1 *))) ldt)
  (declare (ignore var-0 var-1 var-2 var-3 var-6 var-7 var-9))
  (when var-4 (setf n1 var-4))
```

```
          (when var-5 (setf n2 var-5))
          (when var-8 (setf ldt var-8))
          (when var-10 (setf ldt var-10)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10)
            (dtrmm "R" "U" "N" "N" n1 n2 one
             (f2cl-lib:array-slice t$ double-float (j1 j1) ((1 ldt) (1 *))) ldt
             (f2cl-lib:array-slice t$ double-float (1 j1) ((1 ldt) (1 *))) ldt)
          (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
          (when var-4 (setf n1 var-4))
          (when var-5 (setf n2 var-5))
          (when var-6 (setf one var-6))
          (when var-8 (setf ldt var-8))
          (when var-10 (setf ldt var-10)))))
      (go end_label)
     end_label
      (return nil))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::*main*
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types 'nil :return-values 'nil
                                            :calls 'nil)))
```

———-

32.24 dgerfs

32.24.1 Axiom unit tests

— dgerfs.input —

```
)set break resume
)sys rm -f dgerfs.output
)spool dgerfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.24.2 Axiom help page

— dgerfs.help —

```
==================================================================
dgerfs examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgerfs (TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB, X,
           LDX, FERR, BERR, WORK, IWORK, INFO)
           DGERFS

Function/Subroutine Documentation
   subroutine dgerfs (character TRANS, integer N, integer NRHS, double
       precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV,
       double precision, dimension( ldb, * ) B, integer LDB, double precision,
       dimension( ldx, * ) X, integer LDX, double precision, dimension( * )
       FERR, double precision, dimension( * ) BERR, double precision,
       dimension( * ) WORK, integer, dimension( * ) IWORK, integer INFO)
       DGERFS

Purpose:
   DGERFS improves the computed solution to a system of linear
   equations and provides error bounds and backward error estimates for
   the solution.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose = Transpose)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.
```

```
NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  The original N-by-N matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is DOUBLE PRECISION array, dimension (LDAF,N)
  The factors L and U from the factorization A = P*L*U
  as computed by DGETRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from DGETRF; for 1<=i<=N, row i of the
  matrix was interchanged with row IPIV(i).

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by DGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

FERR
  FERR is DOUBLE PRECISION array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
```

```
   element in (X(j) - XTRUE) divided by the magnitude of the
   largest element in X(j).  The estimate is as reliable as
   the estimate for RCOND, and is almost always a slight
   overestimate of the true error.

 BERR
   BERR is DOUBLE PRECISION array, dimension (NRHS)
   The componentwise relative backward error of each solution
   vector X(j) (i.e., the smallest relative change in
   any element of A or B that makes X(j) an exact solution).

 WORK
   WORK is DOUBLE PRECISION array, dimension (3*N)

 IWORK
   IWORK is INTEGER array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

 ITMAX is the maximum number of steps of iterative refinement.
```

———-

32.24.3 fortran code

— **dgerfs.f** —

```
*  =====================================================================
      SUBROUTINE DGERFS( TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB,
     $                   X, LDX, FERR, BERR, WORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
```

```
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   BERR( * ), FERR( * ), WORK( * ), X( LDX, * )
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D+0 )
      DOUBLE PRECISION   THREE
      PARAMETER          ( THREE = 3.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      CHARACTER          TRANST
      INTEGER            COUNT, I, J, K, KASE, NZ
      DOUBLE PRECISION   EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN, XK
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DCOPY, DGEMV, DGETRS, DLACN2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
```

```
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -10
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -12
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGERFS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
      IF( NOTRAN ) THEN
         TRANST = 'T'
      ELSE
         TRANST = 'N'
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = N + 1
      EPS = DLAMCH( 'Epsilon' )
      SAFMIN = DLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 140 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - op(A) * X,
```

```
*        where op(A) = A, A**T, or A**H, depending on TRANS.
*
         CALL DCOPY( N, B( 1, J ), 1, WORK( N+1 ), 1 )
         CALL DGEMV( TRANS, N, N, -ONE, A, LDA, X( 1, J ), 1, ONE,
     $               WORK( N+1 ), 1 )
*
*        Compute componentwise relative backward error from formula
*
*        max(i) ( abs(R(i)) / ( abs(op(A))*abs(X) + abs(B) )(i) )
*
*        where abs(Z) is the componentwise absolute value of the matrix
*        or vector Z.  If the i-th component of the denominator is less
*        than SAFE2, then SAFE1 is added to the i-th components of the
*        numerator and denominator before dividing.
*
         DO 30 I = 1, N
            WORK( I ) = ABS( B( I, J ) )
   30    CONTINUE
*
*        Compute abs(op(A))*abs(X) + abs(B).
*
         IF( NOTRAN ) THEN
            DO 50 K = 1, N
               XK = ABS( X( K, J ) )
               DO 40 I = 1, N
                  WORK( I ) = WORK( I ) + ABS( A( I, K ) )*XK
   40          CONTINUE
   50       CONTINUE
         ELSE
            DO 70 K = 1, N
               S = ZERO
               DO 60 I = 1, N
                  S = S + ABS( A( I, K ) )*ABS( X( I, J ) )
   60          CONTINUE
               WORK( K ) = WORK( K ) + S
   70       CONTINUE
         END IF
         S = ZERO
         DO 80 I = 1, N
            IF( WORK( I ).GT.SAFE2 ) THEN
               S = MAX( S, ABS( WORK( N+I ) ) / WORK( I ) )
            ELSE
               S = MAX( S, ( ABS( WORK( N+I ) )+SAFE1 ) /
     $             ( WORK( I )+SAFE1 ) )
            END IF
   80    CONTINUE
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
*           1) The residual BERR(J) is larger than machine epsilon, and
```

```
*          2) BERR(J) decreased by at least a factor of 2 during the
*             last iteration, and
*          3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
*
*          Update solution and try again.
*
           CALL DGETRS( TRANS, N, 1, AF, LDAF, IPIV, WORK( N+1 ), N,
     $                  INFO )
           CALL DAXPY( N, ONE, WORK( N+1 ), 1, X( 1, J ), 1 )
           LSTRES = BERR( J )
           COUNT = COUNT + 1
           GO TO 20
         END IF
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(op(A)))*
*           ( abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
*          inv(op(A)) is the inverse of op(A)
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(op(A))*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(op(A))*abs(X) + abs(B) is less than SAFE2.
*
*        Use DLACN2 to estimate the infinity-norm of the matrix
*           inv(op(A)) * diag(W),
*        where W = abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) )))
*
         DO 90 I = 1, N
           IF( WORK( I ).GT.SAFE2 ) THEN
             WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I )
           ELSE
             WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I ) + SAFE1
           END IF
   90    CONTINUE
*
         KASE = 0
  100    CONTINUE
         CALL DLACN2( N, WORK( 2*N+1 ), WORK( N+1 ), IWORK, FERR( J ),
```

```
     $                KASE, ISAVE )
         IF( KASE.NE.0 ) THEN
            IF( KASE.EQ.1 ) THEN
*
*              Multiply by diag(W)*inv(op(A)**T).
*
               CALL DGETRS( TRANST, N, 1, AF, LDAF, IPIV, WORK( N+1 ),
     $                      N, INFO )
               DO 110 I = 1, N
                  WORK( N+I ) = WORK( I )*WORK( N+I )
  110          CONTINUE
            ELSE
*
*              Multiply by inv(op(A))*diag(W).
*
               DO 120 I = 1, N
                  WORK( N+I ) = WORK( I )*WORK( N+I )
  120          CONTINUE
               CALL DGETRS( TRANS, N, 1, AF, LDAF, IPIV, WORK( N+1 ), N,
     $                      INFO )
            END IF
            GO TO 100
         END IF
*
*        Normalize error.
*
         LSTRES = ZERO
         DO 130 I = 1, N
            LSTRES = MAX( LSTRES, ABS( X( I, J ) ) )
  130    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  140 CONTINUE
*
      RETURN
*
*     End of DGERFS
*
      END
```

———-

32.24.4 lisp code

— dgerfs.lisp —

```
(let* ((itmax 5) (zero 0.0d0) (one 1.0d0) (two 2.0d0) (three 3.0d0))
```

```
(declare (type (f2cl-lib:integer4 5 5) itmax)
         (type (double-float 0.0d0 0.0d0) zero)
         (type (double-float 1.0d0 1.0d0) one)
         (type (double-float 2.0d0 2.0d0) two)
         (type (double-float 3.0d0 3.0d0) three)
         (ignorable itmax zero one two three))
(defun dgerfs
       (trans n nrhs a lda af ldaf ipiv b ldb$ x ldx ferr berr work iwork
        info)
  (declare (type (array f2cl-lib:integer4 (*)) iwork ipiv)
           (type (array double-float (*)) work berr ferr x b af a)
           (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (a double-float a-%data% a-%offset%)
       (af double-float af-%data% af-%offset%)
       (b double-float b-%data% b-%offset%)
       (x double-float x-%data% x-%offset%)
       (ferr double-float ferr-%data% ferr-%offset%)
       (berr double-float berr-%data% berr-%offset%)
       (work double-float work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
    (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
           (eps 0.0d0) (lstres 0.0d0) (s 0.0d0) (safe1 0.0d0) (safe2 0.0d0)
           (safmin 0.0d0) (xk 0.0d0) (count$ 0) (i 0) (j 0) (k 0) (kase 0)
           (nz 0)
           (transt
            (make-array '(1) :element-type 'character :initial-element #\ ))
           (notran nil))
      (declare (type (array f2cl-lib:integer4 (3)) isave)
               (type (double-float) eps lstres s safe1 safe2 safmin xk)
               (type (f2cl-lib:integer4) count$ i j k kase nz)
               (type (string 1) transt)
               (type f2cl-lib:logical notran))
      (setf info 0)
      (setf notran
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "N")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val))
      (cond
       ((and (not notran)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "T")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
```

```
          ret-val))
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame trans "C")
           (declare (ignore var-1))
           (when var-0 (setf trans var-0))
           ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -10))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -12)))
(cond
 ((/= info 0) (xerbla "DGERFS" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%) zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) zero)
     label10))
  (go end_label)))
(cond (notran (f2cl-lib:f2cl-set-string transt "T" (string 1)))
      (t (f2cl-lib:f2cl-set-string transt "N" (string 1))))
(setf nz (f2cl-lib:int-add n 1))
(setf eps (dlamch "Epsilon"))
(setf safmin (dlamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy n
         (f2cl-lib:array-slice b-%data% double-float (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1
         (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                               ((1 *)) work-%offset%)
         1)
```

```
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgemv trans n n (- one) a lda
     (f2cl-lib:array-slice x-%data% double-float (1 j)
                           ((1 ldx) (1 *)) x-%offset%)
     1 one
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                           ((1 *)) work-%offset%)
     1)
  (declare (ignore var-3 var-4 var-6 var-7 var-9 var-10))
  (when var-0 (setf trans var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-5 (setf lda var-5))
  (when var-8 (setf one var-8)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (abs
             (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                            b-%offset%)))
   label30))
(cond
 (notran
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (setf xk
              (abs
               (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                              x-%offset%)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                  (+
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                   (*
                    (abs
                     (f2cl-lib:fref a-%data% (i k) ((1 lda) (1 *))
                                    a-%offset%))
                    xk)))
         label40))
     label50)))
```

```
            (t
             (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                           ((> k n) nil)
               (tagbody
                 (setf s zero)
                 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                               ((> i n) nil)
                   (tagbody
                     (setf s
                             (+ s
                                (*
                                 (abs
                                  (f2cl-lib:fref a-%data% (i k)
                                                 ((1 lda) (1 *)) a-%offset%))
                                 (abs
                                  (f2cl-lib:fref x-%data% (i j)
                                                 ((1 ldx) (1 *))
                                                 x-%offset%)))))
                     label60))
                 (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                         (+
                          (f2cl-lib:fref work-%data% (k) ((1 *))
                                         work-%offset%)
                          s))
                label70))))
           (setf s zero)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (cond
                ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
                 (setf s
                         (max s
                              (/
                               (abs
                                (f2cl-lib:fref work-%data%
                                               ((f2cl-lib:int-add n i))
                                               ((1 *)) work-%offset%))
                               (f2cl-lib:fref work-%data% (i) ((1 *))
                                              work-%offset%)))))
                (t
                 (setf s
                         (max s
                              (/
                               (+
                                (abs
                                 (f2cl-lib:fref work-%data%
                                                ((f2cl-lib:int-add n i))
                                                ((1 *)) work-%offset%))
                                safe1)
```

```
                    (+
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)
                     safe1))))))
   label80))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dgetrs trans n 1 af ldaf ipiv
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 *)) work-%offset%)
       n info)
    (declare (ignore var-2 var-3 var-5 var-6))
    (when var-0 (setf trans var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf ldaf var-4))
    (when var-7 (setf n var-7))
    (when var-8 (setf info var-8)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (daxpy n one
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 *)) work-%offset%)
       1
       (f2cl-lib:array-slice x-%data% double-float (1 j)
                             ((1 ldx) (1 *)) x-%offset%)
       1)
    (declare (ignore var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))
    (when var-1 (setf one var-1)))
  (setf lstres
          (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
  (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (+
               (abs
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%))
               (* nz eps
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)))))
     (t
```

```
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (+
                   (abs
                    (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                   ((1 *)) work-%offset%))
                   (* nz eps
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%))
                   safe1))))
      label90))
   (setf kase 0)
  label100
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlacn2 n
        (f2cl-lib:array-slice work-%data% double-float
                              ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                              ((1 *)) work-%offset%)
        iwork
        (f2cl-lib:array-slice ferr-%data% double-float (j) ((1 *))
                              ferr-%offset%)
        kase isave)
     (declare (ignore var-1 var-2 var-3 var-4 var-6))
     (when var-0 (setf n var-0))
     (when var-5 (setf kase var-5)))
   (cond
    ((/= kase 0)
     (cond
      ((= kase 1)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
           (dgetrs transt n 1 af ldaf ipiv
            (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                                  ((1 *)) work-%offset%)
            n info)
         (declare (ignore var-2 var-3 var-5 var-6))
         (when var-0 (setf transt var-0))
         (when var-1 (setf n var-1))
         (when var-4 (setf ldaf var-4))
         (when var-7 (setf n var-7))
         (when var-8 (setf info var-8)))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                ((1 *)) work-%offset%)
                   (*
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)
```

```
                        (f2cl-lib:fref work-%data%
                                       ((f2cl-lib:int-add n i)) ((1 *))
                                       work-%offset%)))
                 label110)))
             (t
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i n) nil)
                (tagbody
                  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                       ((1 *)) work-%offset%)
                        (*
                         (f2cl-lib:fref work-%data% (i) ((1 *))
                                        work-%offset%)
                         (f2cl-lib:fref work-%data%
                                        ((f2cl-lib:int-add n i)) ((1 *))
                                        work-%offset%)))
                 label120))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                  (dgetrs trans n 1 af ldaf ipiv
                   (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                                         ((1 *)) work-%offset%)
                   n info)
                (declare (ignore var-2 var-3 var-5 var-6))
                (when var-0 (setf trans var-0))
                (when var-1 (setf n var-1))
                (when var-4 (setf ldaf var-4))
                (when var-7 (setf n var-7))
                (when var-8 (setf info var-8)))))
            (go label100)))
          (setf lstres zero)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf lstres
                      (max lstres
                           (abs
                            (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                           x-%offset%))))
             label130))
          (if (/= lstres zero)
              (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                      (/
                       (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                       lstres)))
         label140))
       (go end_label)
      end_label
       (return
        (values trans n nil nil lda nil ldaf nil nil nil nil nil nil nil nil
```

```
                nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgerfs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::n nil nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldaf nil nil nil
                                              nil nil nil nil nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

32.25 dgerfsx

32.25.1 Axiom unit tests

— dgerfsx.input —

```
)set break resume
```

```
)sys rm -f dgerfsx.output
)spool dgerfsx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

32.25.2 Axiom help page [Demm05]

— dgerfsx.help —

```
==========================================================================
dgerfsx examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgerfsx (TRANS, EQUED, N, NRHS, A, LDA, AF, LDAF, IPIV, R,
           C, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS, ERR_BNDS_NORM,
           ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, IWORK, INFO)
           DGERFSX

Function/Subroutine Documentation
   subroutine dgerfsx (character TRANS, character EQUED, integer N, integer
       NRHS, double precision, dimension( lda, * ) A, integer LDA, double
       precision, dimension( ldaf, * ) AF, integer LDAF, integer, dimension( *
       ) IPIV, double precision, dimension( * ) R, double precision,
       dimension( * ) C, double precision, dimension( ldb, * ) B, integer LDB,
       double precision, dimension( ldx , * ) X, integer LDX, double precision
       RCOND, double precision, dimension( * ) BERR, integer N_ERR_BNDS,
       double precision, dimension( nrhs, * ) ERR_BNDS_NORM, double precision,
       dimension( nrhs, * ) ERR_BNDS_COMP, integer NPARAMS, double precision,
       dimension( * ) PARAMS, double precision, dimension( * ) WORK, integer,
       dimension( * ) IWORK, integer INFO)
       DGERFSX

Purpose:
   DGERFSX improves the computed solution to a system of linear
   equations and provides error bounds and backward error estimates
   for the solution.  In addition to normwise error bound, the code
```

```
  provides maximum componentwise error bound if possible.  See
  comments for ERR_BNDS_NORM and ERR_BNDS_COMP for details of the
  error bounds.

  The original system of linear equations may have been equilibrated
  before calling this routine, as described by arguments EQUED, R
  and C below. In this case, the solution and error bounds returned
  are for the original unequilibrated system.

  Some optional parameters are bundled in the PARAMS array.  These
  settings determine how refinement is performed, but often the
  defaults are acceptable.  If the defaults are acceptable, users
  can pass NPARAMS = 0 which prevents the source code from accessing
  the PARAMS argument.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose = Transpose)

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done to A
   before calling this routine. This is needed to compute
   the solution and error bounds correctly.
   = 'N':  No equilibration
   = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
   = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
   = 'B':  Both row and column equilibration, i.e., A has been
           replaced by diag(R) * A * diag(C).
           The right hand side B has been changed accordingly.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   The original N-by-N matrix A.

 LDA
```

```
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is DOUBLE PRECISION array, dimension (LDAF,N)
  The factors L and U from the factorization A = P*L*U
  as computed by DGETRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from DGETRF; for 1<=i<=N, row i of the
  matrix was interchanged with row IPIV(i).

R
  R is DOUBLE PRECISION array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.
  If R is accessed, each element of R should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

C
  C is DOUBLE PRECISION array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.
  If C is accessed, each element of C should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
```

```
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by DGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
            max_j (abs(XTRUE(j,i) - X(j,i)))
             ------------------------------
                    max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
```

```
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
          reciprocal condition number is less than the threshold
          sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
          almost certainly within a factor of 10 of the true error
          so long as the next entry is greater than the threshold
          sqrt(n) * dlamch('Epsilon'). This error bound should only
          be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
           reciprocal condition number.  Compared with the threshold
           sqrt(n) * dlamch('Epsilon') to determine if the error
           estimate is "guaranteed". These reciprocal condition
           numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
           appropriately scaled matrix Z.
           Let Z = S*A, where S scales each row by a power of the
           radix so all absolute row sums of Z are approximately 1.

See Lapack Working Note 165 for further details and extra cautions.

ERR_BNDS_COMP
  ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                    abs(XTRUE(j,i) - X(j,i))
             max_j ----------------------
                          abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
          reciprocal condition number is less than the threshold
          sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
```

```
          almost certainly within a factor of 10 of the true error
          so long as the next entry is greater than the threshold
          sqrt(n) * dlamch('Epsilon'). This error bound should only
          be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
           reciprocal condition number.  Compared with the threshold
           sqrt(n) * dlamch('Epsilon') to determine if the error
           estimate is "guaranteed". These reciprocal condition
           numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
           appropriately scaled matrix Z.
           Let Z = S*(A*diag(x)), where x is the solution for the
           current right-hand side and S scales each row of
           A*diag(x) by a power of the radix so all absolute row
           sums of Z are approximately 1.

  See Lapack Working Note 165 for further details and extra cautions.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is DOUBLE PRECISION array, dimension (NPARAMS)
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0D+0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
          (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
```

```
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is DOUBLE PRECISION array, dimension (4*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is guaranteed.
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
     has been completed, but the factor U is exactly singular, so
     the solution and error bounds could not be computed. RCOND = 0
     is returned.
  = N+J: The solution corresponding to the Jth right-hand side is
     not guaranteed. The solutions corresponding to other right-
     hand sides K with K > J may not be guaranteed as well, but
     only the first such right-hand side is reported. If a small
     componentwise error is not requested (PARAMS(3) = 0.0) then
     the Jth right-hand side is the first with a normwise error
     bound that is not guaranteed (the smallest J such
     that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
     the Jth right-hand side is the first with either a normwise or
     componentwise error bound that is not guaranteed (the smallest
     J such that either ERR_BNDS_NORM(J,1) = 0.0 or
     ERR_BNDS_COMP(J,1) = 0.0). See the definition of
     ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
     about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

———-

32.25.3 fortran code

— dgerfsx.f —

```
*  =====================================================================
      SUBROUTINE DGERFSX( TRANS, EQUED, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $                    R, C, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS,
     $                    ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $                    WORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
```

```
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS, EQUED
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX , * ), WORK( * )
      DOUBLE PRECISION   R( * ), C( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   ITREF_DEFAULT, ITHRESH_DEFAULT
      DOUBLE PRECISION   COMPONENTWISE_DEFAULT, RTHRESH_DEFAULT
      DOUBLE PRECISION   DZTHRESH_DEFAULT
      PARAMETER          ( ITREF_DEFAULT = 1.0D+0 )
      PARAMETER          ( ITHRESH_DEFAULT = 10.0D+0 )
      PARAMETER          ( COMPONENTWISE_DEFAULT = 1.0D+0 )
      PARAMETER          ( RTHRESH_DEFAULT = 0.5D+0 )
      PARAMETER          ( DZTHRESH_DEFAULT = 0.25D+0 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. Local Scalars ..
      CHARACTER(1)       NORM
      LOGICAL            ROWEQU, COLEQU, NOTRAN
      INTEGER            J, TRANS_TYPE, PREC_TYPE, REF_TYPE
      INTEGER            N_NORMS
      DOUBLE PRECISION   ANORM, RCOND_TMP
      DOUBLE PRECISION   ILLRCOND_THRESH, ERR_LBND, CWISE_WRONG
      LOGICAL            IGNORE_CWISE
      INTEGER            ITHRESH
```

```
      DOUBLE PRECISION   RTHRESH, UNSTABLE_THRESH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, DGECON, DLA_GERFSX_EXTENDED
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, BLAS_FPINFO_X, ILATRANS, ILAPREC
      EXTERNAL           DLAMCH, DLANGE, DLA_GERCOND
      DOUBLE PRECISION   DLAMCH, DLANGE, DLA_GERCOND
      LOGICAL            LSAME
      INTEGER            BLAS_FPINFO_X
      INTEGER            ILATRANS, ILAPREC
*     ..
*     .. Executable Statements ..
*
*     Check the input parameters.
*
      INFO = 0
      TRANS_TYPE = ILATRANS( TRANS )
      REF_TYPE = INT( ITREF_DEFAULT )
      IF ( NPARAMS .GE. LA_LINRX_ITREF_I ) THEN
         IF ( PARAMS( LA_LINRX_ITREF_I ) .LT. 0.0D+0 ) THEN
            PARAMS( LA_LINRX_ITREF_I ) = ITREF_DEFAULT
         ELSE
            REF_TYPE = PARAMS( LA_LINRX_ITREF_I )
         END IF
      END IF
*
*     Set default parameters.
*
      ILLRCOND_THRESH = DBLE( N ) * DLAMCH( 'Epsilon' )
      ITHRESH = INT( ITHRESH_DEFAULT )
      RTHRESH = RTHRESH_DEFAULT
      UNSTABLE_THRESH = DZTHRESH_DEFAULT
      IGNORE_CWISE = COMPONENTWISE_DEFAULT .EQ. 0.0D+0
*
      IF ( NPARAMS.GE.LA_LINRX_ITHRESH_I ) THEN
         IF ( PARAMS( LA_LINRX_ITHRESH_I ).LT.0.0D+0 ) THEN
            PARAMS( LA_LINRX_ITHRESH_I ) = ITHRESH
         ELSE
            ITHRESH = INT( PARAMS( LA_LINRX_ITHRESH_I ) )
         END IF
      END IF
      IF ( NPARAMS.GE.LA_LINRX_CWISE_I ) THEN
         IF ( PARAMS( LA_LINRX_CWISE_I ).LT.0.0D+0 ) THEN
            IF ( IGNORE_CWISE ) THEN
               PARAMS( LA_LINRX_CWISE_I ) = 0.0D+0
```

```
            ELSE
               PARAMS( LA_LINRX_CWISE_I ) = 1.0D+0
            END IF
         ELSE
            IGNORE_CWISE = PARAMS( LA_LINRX_CWISE_I ) .EQ. 0.0D+0
         END IF
      END IF
      IF ( REF_TYPE .EQ. 0 .OR. N_ERR_BNDS .EQ. 0 ) THEN
         N_NORMS = 0
      ELSE IF ( IGNORE_CWISE ) THEN
         N_NORMS = 1
      ELSE
         N_NORMS = 2
      END IF
*
      NOTRAN = LSAME( TRANS, 'N' )
      ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
      COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
*
*     Test input parameters.
*
      IF( TRANS_TYPE.EQ.-1 ) THEN
        INFO = -1
      ELSE IF( .NOT.ROWEQU .AND. .NOT.COLEQU .AND.
     $         .NOT.LSAME( EQUED, 'N' ) ) THEN
        INFO = -2
      ELSE IF( N.LT.0 ) THEN
        INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
        INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
        INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
        INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
        INFO = -13
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
        INFO = -15
      END IF
      IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'DGERFSX', -INFO )
        RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         RCOND = 1.0D+0
         DO J = 1, NRHS
            BERR( J ) = 0.0D+0
```

```
            IF ( N_ERR_BNDS .GE. 1 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
            IF ( N_ERR_BNDS .GE. 2 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I) = 0.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 0.0D+0
            END IF
            IF ( N_ERR_BNDS .GE. 3 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 1.0D+0
            END IF
         END DO
         RETURN
      END IF
*
*     Default to failure.
*
      RCOND = 0.0D+0
      DO J = 1, NRHS
         BERR( J ) = 1.0D+0
         IF ( N_ERR_BNDS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
         END IF
         IF ( N_ERR_BNDS .GE. 2 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
         END IF
         IF ( N_ERR_BNDS .GE. 3 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 0.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 0.0D+0
         END IF
      END DO
*
*     Compute the norm of A and the reciprocal of the condition
*     number of A.
*
      IF( NOTRAN ) THEN
         NORM = 'I'
      ELSE
         NORM = '1'
      END IF
      ANORM = DLANGE( NORM, N, N, A, LDA, WORK )
      CALL DGECON( NORM, N, AF, LDAF, ANORM, RCOND, WORK, IWORK, INFO )
*
*     Perform refinement on each right-hand side
*
      IF ( REF_TYPE .NE. 0 ) THEN
```

```
         PREC_TYPE = ILAPREC( 'E' )

         IF ( NOTRAN ) THEN
            CALL DLA_GERFSX_EXTENDED( PREC_TYPE, TRANS_TYPE,  N,
     $           NRHS, A, LDA, AF, LDAF, IPIV, COLEQU, C, B,
     $           LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM,
     $           ERR_BNDS_COMP, WORK(N+1), WORK(1), WORK(2*N+1),
     $           WORK(1), RCOND, ITHRESH, RTHRESH, UNSTABLE_THRESH,
     $           IGNORE_CWISE, INFO )
         ELSE
            CALL DLA_GERFSX_EXTENDED( PREC_TYPE, TRANS_TYPE,  N,
     $           NRHS, A, LDA, AF, LDAF, IPIV, ROWEQU, R, B,
     $           LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM,
     $           ERR_BNDS_COMP, WORK(N+1), WORK(1), WORK(2*N+1),
     $           WORK(1), RCOND, ITHRESH, RTHRESH, UNSTABLE_THRESH,
     $           IGNORE_CWISE, INFO )
         END IF
      END IF

      ERR_LBND = MAX( 10.0D+0, SQRT( DBLE( N ) ) ) * DLAMCH( 'Epsilon' )
      IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 1 ) THEN
*
*     Compute scaled normwise condition number cond(A*C).
*
         IF ( COLEQU .AND. NOTRAN ) THEN
            RCOND_TMP = DLA_GERCOND( TRANS, N, A, LDA, AF, LDAF, IPIV,
     $           -1, C, INFO, WORK, IWORK )
         ELSE IF ( ROWEQU .AND. .NOT. NOTRAN ) THEN
            RCOND_TMP = DLA_GERCOND( TRANS, N, A, LDA, AF, LDAF, IPIV,
     $           -1, R, INFO, WORK, IWORK )
         ELSE
            RCOND_TMP = DLA_GERCOND( TRANS, N, A, LDA, AF, LDAF, IPIV,
     $           0, R, INFO, WORK, IWORK )
         END IF
         DO J = 1, NRHS
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .GT. 1.0D+0 )
     $           ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 0.0D+0
               IF ( INFO .LE. N ) INFO = N + J
            ELSE IF ( ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .LT. ERR_LBND )
     $     THEN
```

```
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF
         END DO
      END IF

      IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 2 ) THEN
*
*     Compute componentwise condition number cond(A*diag(Y(:,J))) for
*     each right-hand side using the current solution as an estimate of
*     the true solution.  If the componentwise error estimate is too
*     large, then the solution is a lousy estimate of truth and the
*     estimated RCOND may be too optimistic.  To avoid misleading users,
*     the inverse condition number is set to 0.0 when the estimated
*     cwise error is at least CWISE_WRONG.
*
         CWISE_WRONG = SQRT( DLAMCH( 'Epsilon' ) )
         DO J = 1, NRHS
            IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .LT. CWISE_WRONG )
     $           THEN
               RCOND_TMP = DLA_GERCOND( TRANS, N, A, LDA, AF, LDAF,
     $              IPIV, 1, X(1,J), INFO, WORK, IWORK )
            ELSE
               RCOND_TMP = 0.0D+0
            END IF
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .GT. 1.0D+0 )
     $           ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 0.0D+0
               IF ( PARAMS( LA_LINRX_CWISE_I ) .EQ. 1.0D+0
     $              .AND. INFO.LT.N + J ) INFO = N + J
            ELSE IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I )
     $              .LT. ERR_LBND ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
```

```
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF
         END DO
      END IF
*
      RETURN
*
*     End of DGERFSX
*
      END
```

———-

32.25.4 lisp code

— **dgerfsx.lisp** —

———-

32.26 dgerq2

32.26.1 Axiom unit tests

— **dgerq2.input** —

```
)set break resume
)sys rm -f dgerq2.output
)spool dgerq2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.26.2 Axiom help page

— dgerq2.help —

```
=====================================================================
dgerq2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgerq2 (M, N, A, LDA, TAU, WORK, INFO)
           DGERQ2 computes the RQ factorization of a general rectangular
           matrix using an unblocked algorithm.

Function/Subroutine Documentation
   subroutine dgerq2 (integer M, integer N, double precision, dimension( lda,
       * ) A, integer LDA, double precision, dimension( * ) TAU, double
       precision, dimension( * ) WORK, integer INFO)
       DGERQ2 computes the RQ factorization of a general rectangular matrix
       using an unblocked algorithm.

Purpose:
   DGERQ2 computes an RQ factorization of a real m by n matrix A: A = R * Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the m by n matrix A.
   On exit, if m <= n, the upper triangle of the subarray
   A(1:m,n-m+1:n) contains the m by m upper triangular matrix R;
   if m >= n, the elements on and above the (m-n)-th subdiagonal
   contain the m by n upper trapezoidal matrix R; the remaining
   elements, with the array TAU, represent the orthogonal matrix
   Q as a product of elementary reflectors (see Further Details).

 LDA
```

```
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is DOUBLE PRECISION array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is DOUBLE PRECISION array, dimension (M)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

Further Details:
   The matrix Q is represented as a product of elementary reflectors

               Q = H(1) H(2) . . . H(k), where k = min(m,n).

   Each H(i) has the form

               H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(n-k+i+1:n) = 0 and v(n-k+i) = 1; v(1:n-k+i-1) is stored on exit in
   A(m-k+i,1:n-k+i-1), and tau in TAU(i).
```

———-

32.26.3 fortran code

— dgerq2.f —

```
*  =====================================================================
      SUBROUTINE DGERQ2( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      DOUBLE PRECISION   AII
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARF, DLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGERQ2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = K, 1, -1
*
*        Generate elementary reflector H(i) to annihilate
*        A(m-k+i,1:n-k+i-1)
*
         CALL DLARFG( N-K+I, A( M-K+I, N-K+I ), A( M-K+I, 1 ), LDA,
     $                TAU( I ) )
*
*        Apply H(i) to A(1:m-k+i-1,1:n-k+i) from the right
*
         AII = A( M-K+I, N-K+I )
         A( M-K+I, N-K+I ) = ONE
         CALL DLARF( 'Right', M-K+I-1, N-K+I, A( M-K+I, 1 ), LDA,
```

```
     $               TAU( I ), A, LDA, WORK )
         A( M-K+I, N-K+I ) = AII
   10 CONTINUE
      RETURN
*
*     End of DGERQ2
*
      END
```

32.26.4 lisp code

— **dgerq2.lisp** —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dgerq2 (m n a lda tau work info)
    (declare (type (array double-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((aii 0.0d0) (i 0) (k 0))
        (declare (type (double-float) aii)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "DGERQ2" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (f2cl-lib:fdo (i k (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (dlarfg (f2cl-lib:int-add (f2cl-lib:int-sub n k) i)
                 (f2cl-lib:array-slice a-%data% double-float
                                       ((+ m (f2cl-lib:int-sub k) i)
                                        (f2cl-lib:int-add
                                         (f2cl-lib:int-sub n k) i))
                                       ((1 lda) (1 *)) a-%offset%)
                 (f2cl-lib:array-slice a-%data% double-float
```

```
                                    ((+ m (f2cl-lib:int-sub k) i) 1)
                                    ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                    tau-%offset%))
               (declare (ignore var-0 var-1 var-2 var-4))
               (when var-3 (setf lda var-3)))
             (setf aii
                     (f2cl-lib:fref a-%data%
                                   ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                    (f2cl-lib:int-add (f2cl-lib:int-sub n k)
                                                      i))
                                   ((1 lda) (1 *)) a-%offset%))
             (setf (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                  (f2cl-lib:int-add (f2cl-lib:int-sub n k) i))
                                 ((1 lda) (1 *)) a-%offset%)
                     one)
             (multiple-value-bind
                 (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                 (dlarf "Right"
                  (f2cl-lib:int-sub (f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                   1)
                  (f2cl-lib:int-add (f2cl-lib:int-sub n k) i)
                  (f2cl-lib:array-slice a-%data% double-float
                                    ((+ m (f2cl-lib:int-sub k) i) 1)
                                    ((1 lda) (1 *)) a-%offset%)
                  lda
                  (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                    tau-%offset%)
                  a lda work)
               (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8))
               (when var-4 (setf lda var-4))
               (when var-7 (setf lda var-7)))
             (setf (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                  (f2cl-lib:int-add (f2cl-lib:int-sub n k) i))
                                 ((1 lda) (1 *)) a-%offset%)
                     aii)
           label10))
        (go end_label)
       end_label
        (return (values nil nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgerq2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
```

```
'((fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil fortran-to-lisp::lda
  nil nil fortran-to-lisp::info)
:calls 'nil)))
```

———-

32.27 dgerqf

32.27.1 Axiom unit tests

— dgerqf.input —

```
)set break resume
)sys rm -f dgerqf.output
)spool dgerqf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.27.2 Axiom help page

— dgerqf.help —

```
======================================================================
dgerqf examples
======================================================================

======================================================================
Man Page Details
```

```
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgerqf (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           DGERQF

Function/Subroutine Documentation
   subroutine dgerqf (integer M, integer N, double precision, dimension( lda,
       * ) A, integer LDA, double precision, dimension( * ) TAU, double
       precision, dimension( * ) WORK, integer LWORK, integer INFO)
       DGERQF

Purpose:
   DGERQF computes an RQ factorization of a real M-by-N matrix A: A = R * Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit,
   if m <= n, the upper triangle of the subarray
   A(1:m,n-m+1:n) contains the M-by-M upper triangular matrix R;
   if m >= n, the elements on and above the (m-n)-th subdiagonal
   contain the M-by-N upper trapezoidal matrix R;
   the remaining elements, with the array TAU, represent the
   orthogonal matrix Q as a product of min(m,n) elementary
   reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is DOUBLE PRECISION array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.
```

```
 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.  LWORK >= max(1,M).
   For optimum performance LWORK >= M*NB, where NB is
   the optimal blocksize.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details:
   The matrix Q is represented as a product of elementary reflectors

               Q = H(1) H(2) . . . H(k), where k = min(m,n).

   Each H(i) has the form

               H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(n-k+i+1:n) = 0 and v(n-k+i) = 1; v(1:n-k+i-1) is stored on exit in
   A(m-k+i,1:n-k+i-1), and tau in TAU(i).
```

32.27.3 fortran code

— dgerqf.f —

```
*  =====================================================================
      SUBROUTINE DGERQF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
```

```
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, KI, KK, LDWORK, LWKOPT,
     $                   MU, NB, NBMIN, NU, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGERQ2, DLARFB, DLARFT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
*
      IF( INFO.EQ.0 ) THEN
         K = MIN( M, N )
         IF( K.EQ.0 ) THEN
            LWKOPT = 1
         ELSE
            NB = ILAENV( 1, 'DGERQF', ' ', M, N, -1, -1 )
            LWKOPT = M*NB
         END IF
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.MAX( 1, M ) .AND. .NOT.LQUERY ) THEN
            INFO = -7
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
```

```
         CALL XERBLA( 'DGERQF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( K.EQ.0 ) THEN
         RETURN
      END IF
*
      NBMIN = 2
      NX = 1
      IWS = M
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'DGERQF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = M
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'DGERQF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially.
*        The last kk rows are handled by the block method.
*
         KI = ( ( K-NX-1 ) / NB )*NB
         KK = MIN( K, KI+NB )
*
         DO 10 I = K - KK + KI + 1, K - KK + 1, -NB
            IB = MIN( K-I+1, NB )
*
*           Compute the RQ factorization of the current block
```

```
*          A(m-k+i:m-k+i+ib-1,1:n-k+i+ib-1)
*
           CALL DGERQ2( IB, N-K+I+IB-1, A( M-K+I, 1 ), LDA, TAU( I ),
     $                  WORK, IINFO )
           IF( M-K+I.GT.1 ) THEN
*
*             Form the triangular factor of the block reflector
*             H = H(i+ib-1) . . . H(i+1) H(i)
*
              CALL DLARFT( 'Backward', 'Rowwise', N-K+I+IB-1, IB,
     $                     A( M-K+I, 1 ), LDA, TAU( I ), WORK, LDWORK )
*
*             Apply H to A(1:m-k+i-1,1:n-k+i+ib-1) from the right
*
              CALL DLARFB( 'Right', 'No transpose', 'Backward',
     $                     'Rowwise', M-K+I-1, N-K+I+IB-1, IB,
     $                     A( M-K+I, 1 ), LDA, WORK, LDWORK, A, LDA,
     $                     WORK( IB+1 ), LDWORK )
           END IF
   10    CONTINUE
         MU = M - K + I + NB - 1
         NU = N - K + I + NB - 1
      ELSE
         MU = M
         NU = N
      END IF
*
*     Use unblocked code to factor the last or only block
*
      IF( MU.GT.0 .AND. NU.GT.0 )
     $   CALL DGERQ2( MU, NU, A, LDA, TAU, WORK, IINFO )
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of DGERQF
*
      END
```

———-

32.27.4 lisp code

— **dgerqf.lisp** —

```
(defun dgerqf (m n a lda tau work lwork info)
  (declare (type (array double-float (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
```

```
(f2cl-lib:with-multi-array-data
    ((a double-float a-%data% a-%offset%)
     (tau double-float tau-%data% tau-%offset%)
     (work double-float work-%data% work-%offset%))
  (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ki 0) (kk 0) (ldwork 0)
         (lwkopt 0) (mu 0) (nb 0) (nbmin 0) (nu 0) (nx 0) (lquery nil))
    (declare (type f2cl-lib:logical lquery)
             (type (f2cl-lib:integer4) nx nu nbmin nb mu lwkopt ldwork kk ki
              k iws iinfo ib i))
    (setf info 0)
    (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
    (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
          ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
           (setf info -4)))
    (cond
     ((= info 0)
      (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
      (cond ((= k 0) (setf lwkopt 1))
            (t
             (setf nb
                     (multiple-value-bind
                         (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                         (ilaenv 1 "DGERQF" " " m n -1 -1)
                       (declare (ignore var-0 var-1 var-2 var-5 var-6))
                       (when var-3 (setf m var-3))
                       (when var-4 (setf n var-4))
                       ret-val))
             (setf lwkopt (f2cl-lib:int-mul m nb))))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
      (cond
       ((and
         (< lwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (not lquery))
        (setf info -7)))))
    (cond
     ((/= info 0) (xerbla "DGERQF" (f2cl-lib:int-sub info)) (go end_label))
     (lquery (go end_label)))
    (cond ((= k 0) (go end_label)))
    (setf nbmin 2)
    (setf nx 1)
    (setf iws m)
    (cond
     ((and (> nb 1) (< nb k))
      (setf nx
              (max (the f2cl-lib:integer4 0)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
```

```
                          (ilaenv 3 "DGERQF" " " m n -1 -1)
                        (declare (ignore var-0 var-1 var-2 var-5 var-6))
                        (when var-3 (setf m var-3))
                        (when var-4 (setf n var-4))
                        ret-val))))
  (cond
   ((< nx k) (setf ldwork m) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
                            (ilaenv 2 "DGERQF" " " m n -1 -1)
                          (declare
                           (ignore var-0 var-1 var-2 var-5 var-6))
                          (when var-3 (setf m var-3))
                          (when var-4 (setf n var-4))
                          ret-val)))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (setf ki (* (the f2cl-lib:integer4 (truncate (- k nx 1) nb)) nb))
  (setf kk
          (min (the f2cl-lib:integer4 k)
               (the f2cl-lib:integer4 (f2cl-lib:int-add ki nb))))
  (f2cl-lib:fdo (i (f2cl-lib:int-add k (f2cl-lib:int-sub kk) ki 1)
                 (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
                ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub kk) 1)) nil)
    (tagbody
      (setf ib
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
               (the f2cl-lib:integer4 nb)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dgerq2 ib
           (f2cl-lib:int-sub
            (f2cl-lib:int-add (f2cl-lib:int-sub n k) i ib) 1)
           (f2cl-lib:array-slice a-%data% double-float
                                 ((+ m (f2cl-lib:int-sub k) i) 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                 tau-%offset%)
           work iinfo)
        (declare (ignore var-0 var-1 var-2 var-4 var-5))
        (setf lda var-3)
```

```
      (setf iinfo var-6))
    (cond
     ((> (f2cl-lib:int-add m (f2cl-lib:int-sub k) i) 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dlarft "Backward" "Rowwise"
           (f2cl-lib:int-sub
            (f2cl-lib:int-add (f2cl-lib:int-sub n k) i ib) 1)
           ib
           (f2cl-lib:array-slice a-%data% double-float
                                 ((+ m (f2cl-lib:int-sub k) i) 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                 tau-%offset%)
           work ldwork)
        (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
        (when var-3 (setf ib var-3))
        (when var-5 (setf lda var-5))
        (when var-8 (setf ldwork var-8)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13 var-14)
          (dlarfb "Right" "No transpose" "Backward" "Rowwise"
           (f2cl-lib:int-sub
            (f2cl-lib:int-add (f2cl-lib:int-sub m k) i) 1)
           (f2cl-lib:int-sub
            (f2cl-lib:int-add (f2cl-lib:int-sub n k) i ib) 1)
           ib
           (f2cl-lib:array-slice a-%data% double-float
                                 ((+ m (f2cl-lib:int-sub k) i) 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda work ldwork a lda
           (f2cl-lib:array-slice work-%data% double-float ((+ ib 1))
                                 ((1 *)) work-%offset%)
           ldwork)
        (declare
         (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
          var-13))
        (when var-6 (setf ib var-6))
        (when var-8 (setf lda var-8))
        (when var-10 (setf ldwork var-10))
        (when var-12 (setf lda var-12))
        (when var-14 (setf ldwork var-14)))))
   label10))
(setf mu
        (f2cl-lib:int-sub
         (f2cl-lib:int-add (f2cl-lib:int-sub m k) i nb) 1))
(setf nu
        (f2cl-lib:int-sub
```

```
                (f2cl-lib:int-add (f2cl-lib:int-sub n k) i nb) 1)))
       (t (setf mu m) (setf nu n)))
      (if (and (> mu 0) (> nu 0))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (dgerq2 mu nu a lda tau work iinfo)
            (declare (ignore var-0 var-1 var-2 var-4 var-5))
            (setf lda var-3)
            (setf iinfo var-6)))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 iws) 'double-float))
      (go end_label)
     end_label
      (return (values m n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgerqf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::dgerq2))))
```

———-

32.28 dgesvj

32.28.1 Axiom unit tests

— dgesvj.input —

```
)set break resume
)sys rm -f dgesvj.output
```

```
)spool dgesvj.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.28.2 Axiom help page

— **dgesvj.help** —

```
==================================================================
dgesvj examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgesvj (JOBA, JOBU, JOBV, M, N, A, LDA, SVA, MV, V, LDV,
           WORK, LWORK, INFO)
           DGESVJ

Function/Subroutine Documentation
   subroutine dgesvj (character*1 JOBA, character*1 JOBU, character*1 JOBV,
       integer M, integer N, double precision, dimension( lda, * ) A, integer
       LDA, double precision, dimension( n ) SVA, integer MV, double
       precision, dimension( ldv, * ) V, integer LDV, double precision,
       dimension( lwork ) WORK, integer LWORK, integer INFO)
       DGESVJ

Purpose:
   DGESVJ computes the singular value decomposition (SVD) of a real
   M-by-N matrix A, where M >= N. The SVD of A is written as
                                     [++]   [xx]   [x0]   [xx]
             A = U * SIGMA * V^t,  [++] = [xx] * [ox] * [xx]
                                     [++]   [xx]
   where SIGMA is an N-by-N diagonal matrix, U is an M-by-N orthonormal
   matrix, and V is an N-by-N orthogonal matrix. The diagonal elements
   of SIGMA are the singular values of A. The columns of U and V are the
   left and the right singular vectors of A, respectively.
   DGESVJ can sometimes compute tiny singular values and their singular
   vectors much more accurately than other SVD routines,
```

```
   see below under Further Details.

Parameters:

 JOBA
   JOBA is CHARACTER* 1
   Specifies the structure of A.
   = 'L': The input matrix A is lower triangular;
   = 'U': The input matrix A is upper triangular;
   = 'G': The input matrix A is general M-by-N matrix, M >= N.

 JOBU
   JOBU is CHARACTER*1
   Specifies whether to compute the left singular vectors
   (columns of U):
   = 'U': The left singular vectors corresponding to the nonzero
      singular values are computed and returned in the leading
      columns of A. See more details in the description of A.
      The default numerical orthogonality threshold is set to
      approximately TOL=CTOL*EPS, CTOL=DSQRT(M), EPS=DLAMCH('E').
   = 'C': Analogous to JOBU='U', except that user can control the
      level of numerical orthogonality of the computed left
      singular vectors. TOL can be set to TOL = CTOL*EPS, where
      CTOL is given on input in the array WORK.
      No CTOL smaller than ONE is allowed. CTOL greater
      than 1 / EPS is meaningless. The option 'C'
      can be used if M*EPS is satisfactory orthogonality
      of the computed left singular vectors, so CTOL=M could
      save few sweeps of Jacobi rotations.
      See the descriptions of A and WORK(1).
   = 'N': The matrix U is not computed. However, see the description of A.

 JOBV
   JOBV is CHARACTER*1
   Specifies whether to compute the right singular vectors, that
   is, the matrix V:
   = 'V' : the matrix V is computed and returned in the array V
   = 'A' : the Jacobi rotations are applied to the MV-by-N
      array V. In other words, the right singular vector
      matrix V is not computed explicitly, instead it is
      applied to an MV-by-N matrix initially stored in the first MV rows of V.
   = 'N' : the matrix V is not computed and the array V is not referenced

 M
   M is INTEGER
   The number of rows of the input matrix A. 1/DLAMCH('E') > M >= 0.

 N
   N is INTEGER
   The number of columns of the input matrix A.
```

```
  M >= N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit :
  If JOBU .EQ. 'U' .OR. JOBU .EQ. 'C' :
    If INFO .EQ. 0 :
       RANKA orthonormal columns of U are returned in the
       leading RANKA columns of the array A. Here RANKA <= N
       is the number of computed singular values of A that are
       above the underflow threshold DLAMCH('S'). The singular
       vectors corresponding to underflowed or zero singular
       values are not computed. The value of RANKA is returned
       in the array WORK as RANKA=NINT(WORK(2)). Also see the
       descriptions of SVA and WORK. The computed columns of U
       are mutually numerically orthogonal up to approximately
       TOL=DSQRT(M)*EPS (default); or TOL=CTOL*EPS (JOBU.EQ.'C'),
       see the description of JOBU.
    If INFO .GT. 0 :
       the procedure DGESVJ did not converge in the given number
       of iterations (sweeps). In that case, the computed
       columns of U may not be orthogonal up to TOL. The output
       U (stored in A), SIGMA (given by the computed singular
       values in SVA(1:N)) and V is still a decomposition of the
       input matrix A in the sense that the residual
       ||A-SCALE*U*SIGMA*V^T||_2 / ||A||_2 is small.

  If JOBU .EQ. 'N' :
    If INFO .EQ. 0 :
       Note that the left singular vectors are 'for free' in the
       one-sided Jacobi SVD algorithm. However, if only the
       singular values are needed, the level of numerical
       orthogonality of U is not an issue and iterations are
       stopped when the columns of the iterated matrix are
       numerically orthogonal up to approximately M*EPS. Thus,
       on exit, A contains the columns of U scaled with the
       corresponding singular values.
    If INFO .GT. 0 :
       the procedure DGESVJ did not converge in the given number
       of iterations (sweeps).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

SVA
  SVA is DOUBLE PRECISION array, dimension (N)
  On exit :
  If INFO .EQ. 0 :
```

```
     depending on the value SCALE = WORK(1), we have:
     If SCALE .EQ. ONE :
        SVA(1:N) contains the computed singular values of A.
        During the computation SVA contains the Euclidean column
        norms of the iterated matrices in the array A.
     If SCALE .NE. ONE :
        The singular values of A are SCALE*SVA(1:N), and this
        factored representation is due to the fact that some of the
        singular values of A might underflow or overflow.
  If INFO .GT. 0 :
     the procedure DGESVJ did not converge in the given number of
     iterations (sweeps) and SCALE*SVA(1:N) may not be accurate.

MV
  MV is INTEGER
  If JOBV .EQ. 'A', then the product of Jacobi rotations in DGESVJ
  is applied to the first MV rows of V. See the description of JOBV.

V
  V is DOUBLE PRECISION array, dimension (LDV,N)
  If JOBV = 'V', then V contains on exit the N-by-N matrix of
     the right singular vectors;
  If JOBV = 'A', then V contains the product of the computed right
     singular vector matrix and the initial matrix in
     the array V.
  If JOBV = 'N', then V is not referenced.

LDV
  LDV is INTEGER
  The leading dimension of the array V, LDV .GE. 1.
  If JOBV .EQ. 'V', then LDV .GE. max(1,N).
  If JOBV .EQ. 'A', then LDV .GE. max(1,MV) .

WORK
  WORK is DOUBLE PRECISION array, dimension max(4,M+N).
  On entry :
  If JOBU .EQ. 'C' :
  WORK(1) = CTOL, where CTOL defines the threshold for convergence.
     The process stops if all columns of A are mutually
     orthogonal up to CTOL*EPS, EPS=DLAMCH('E').
     It is required that CTOL >= ONE, i.e. it is not
     allowed to force the routine to obtain orthogonality below EPS.
  On exit :
     WORK(1) = SCALE is the scaling factor such that SCALE*SVA(1:N)
       are the computed singular values of A.
       (See description of SVA().)
     WORK(2) = NINT(WORK(2)) is the number of the computed nonzero
       singular values.
     WORK(3) = NINT(WORK(3)) is the number of the computed singular
       values that are larger than the underflow threshold.
```

```
      WORK(4) = NINT(WORK(4)) is the number of sweeps of Jacobi
        rotations needed for numerical convergence.
      WORK(5) = max_{i.NE.j} |COS(A(:,i),A(:,j))| in the last sweep.
        This is useful information in cases when DGESVJ did
        not converge, as it can be used to estimate whether
        the output is stil useful and for post festum analysis.
      WORK(6) = the largest absolute value over all sines of the
        Jacobi rotation angles in the last sweep. It can be
        useful for a post festum analysis.

 LWORK
   LWORK is INTEGER
   length of WORK, WORK >= MAX(6,M+N)

 INFO
   INFO is INTEGER
   = 0 : successful exit.
   < 0 : if INFO = -i, then the i-th argument had an illegal value
   > 0 : DGESVJ did not converge in the maximal allowed number (30)
         of sweeps. The output may still be useful. See the
         description of WORK.

Further Details:
   The orthogonal N-by-N matrix V is obtained as a product of Jacobi plane
   rotations. The rotations are implemented as fast scaled rotations of
   Anda and Park [1]. In the case of underflow of the Jacobi angle, a
   modified Jacobi transformation of Drmac [4] is used. Pivot strategy uses
   column interchanges of de Rijk [2]. The relative accuracy of the computed
   singular values and the accuracy of the computed singular vectors (in
   angle metric) is as guaranteed by the theory of Demmel and Veselic [3].
   The condition number that determines the accuracy in the full rank case
   is essentially min_{D=diag} kappa(A*D), where kappa(.) is the
   spectral condition number. The best performance of this Jacobi SVD
   procedure is achieved if used in an  accelerated version of Drmac and
   Veselic [5,6], and it is the kernel routine in the SIGMA library [7].
   Some tunning parameters (marked with [TP]) are available for the
   implementer.
   The computational range for the nonzero singular values is the  machine
   number interval ( UNDERFLOW , OVERFLOW ). In extreme cases, even
   denormalized singular values can be computed with the corresponding
   gradual loss of accurate digits.
```

———-

32.28.3 fortran code

— dgesvj.f —

```
*  =====================================================================
      SUBROUTINE DGESVJ( JOBA, JOBU, JOBV, M, N, A, LDA, SVA, MV, V,
     $                   LDV, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDV, LWORK, M, MV, N
      CHARACTER*1        JOBA, JOBU, JOBV
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), SVA( N ), V( LDV, * ),
     $                   WORK( LWORK )
*     ..
*
*  =====================================================================
*
*     .. Local Parameters ..
      DOUBLE PRECISION   ZERO, HALF, ONE
      PARAMETER          ( ZERO = 0.0D0, HALF = 0.5D0, ONE = 1.0D0)
      INTEGER            NSWEEP
      PARAMETER          ( NSWEEP = 30 )
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION   AAPP, AAPP0, AAPQ, AAQQ, APOAQ, AQOAP, BIG,
     $                   BIGTHETA, CS, CTOL, EPSLN, LARGE, MXAAPQ,
     $                   MXSINJ, ROOTBIG, ROOTEPS, ROOTSFMIN, ROOTTOL,
     $                   SKL, SFMIN, SMALL, SN, T, TEMP1, THETA,
     $                   THSIGN, TOL
      INTEGER            BLSKIP, EMPTSW, i, ibr, IERR, igl, IJBLSK, ir1,
     $                   ISWROT, jbc, jgl, KBL, LKAHEAD, MVL, N2, N34,
     $                   N4, NBL, NOTROT, p, PSKIPPED, q, ROWSKIP,
     $                   SWBAND
      LOGICAL            APPLV, GOSCALE, LOWER, LSVEC, NOSCALE, ROTOK,
     $                   RSVEC, UCTOL, UPPER
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   FASTR( 5 )
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DABS, MAX, MIN, DBLE, DSIGN, DSQRT
*     ..
*     .. External Functions ..
*     ..
*     from BLAS
      DOUBLE PRECISION   DDOT, DNRM2
      EXTERNAL           DDOT, DNRM2
```

```
      INTEGER            IDAMAX
      EXTERNAL           IDAMAX
*     from LAPACK
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
*     ..
*     from BLAS
      EXTERNAL           DAXPY, DCOPY, DROTM, DSCAL, DSWAP
*     from LAPACK
      EXTERNAL           DLASCL, DLASET, DLASSQ, XERBLA
*
      EXTERNAL           DGSVJ0, DGSVJ1
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      LSVEC = LSAME( JOBU, 'U' )
      UCTOL = LSAME( JOBU, 'C' )
      RSVEC = LSAME( JOBV, 'V' )
      APPLV = LSAME( JOBV, 'A' )
      UPPER = LSAME( JOBA, 'U' )
      LOWER = LSAME( JOBA, 'L' )
*
      IF( .NOT.( UPPER .OR. LOWER .OR. LSAME( JOBA, 'G' ) ) ) THEN
         INFO = -1
      ELSE IF( .NOT.( LSVEC .OR. UCTOL .OR. LSAME( JOBU, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( .NOT.( RSVEC .OR. APPLV .OR. LSAME( JOBV, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( M.LT.0 ) THEN
         INFO = -4
      ELSE IF( ( N.LT.0 ) .OR. ( N.GT.M ) ) THEN
         INFO = -5
      ELSE IF( LDA.LT.M ) THEN
         INFO = -7
      ELSE IF( MV.LT.0 ) THEN
         INFO = -9
      ELSE IF( ( RSVEC .AND. ( LDV.LT.N ) ) .OR.
     $         ( APPLV .AND. ( LDV.LT.MV ) ) ) THEN
         INFO = -11
      ELSE IF( UCTOL .AND. ( WORK( 1 ).LE.ONE ) ) THEN
         INFO = -12
      ELSE IF( LWORK.LT.MAX( M+N, 6 ) ) THEN
         INFO = -13
      ELSE
```

```
         INFO = 0
      END IF
*
*     #:(
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGESVJ', -INFO )
         RETURN
      END IF
*
* #:) Quick return for void matrix
*
      IF( ( M.EQ.0 ) .OR. ( N.EQ.0 ) )RETURN
*
*     Set numerical parameters
*     The stopping criterion for Jacobi rotations is
*
*     max_{i<>j}|A(:,i)^T * A(:,j)|/(||A(:,i)||*||A(:,j)||) < CTOL*EPS
*
*     where EPS is the round-off and CTOL is defined as follows:
*
      IF( UCTOL ) THEN
*        ... user controlled
         CTOL = WORK( 1 )
      ELSE
*        ... default
         IF( LSVEC .OR. RSVEC .OR. APPLV ) THEN
            CTOL = DSQRT( DBLE( M ) )
         ELSE
            CTOL = DBLE( M )
         END IF
      END IF
*     ... and the machine dependent parameters are
*[!]  (Make sure that DLAMCH() works properly on the target machine.)
*
      EPSLN = DLAMCH( 'Epsilon' )
      ROOTEPS = DSQRT( EPSLN )
      SFMIN = DLAMCH( 'SafeMinimum' )
      ROOTSFMIN = DSQRT( SFMIN )
      SMALL = SFMIN / EPSLN
      BIG = DLAMCH( 'Overflow' )
*     BIG         = ONE    / SFMIN
      ROOTBIG = ONE / ROOTSFMIN
      LARGE = BIG / DSQRT( DBLE( M*N ) )
      BIGTHETA = ONE / ROOTEPS
*
      TOL = CTOL*EPSLN
      ROOTTOL = DSQRT( TOL )
*
      IF( DBLE( M )*EPSLN.GE.ONE ) THEN
         INFO = -4
```

```
         CALL XERBLA( 'DGESVJ', -INFO )
         RETURN
      END IF
*
*     Initialize the right singular vector matrix.
*
      IF( RSVEC ) THEN
         MVL = N
         CALL DLASET( 'A', MVL, N, ZERO, ONE, V, LDV )
      ELSE IF( APPLV ) THEN
         MVL = MV
      END IF
      RSVEC = RSVEC .OR. APPLV
*
*     Initialize SVA( 1:N ) = ( ||A e_i||_2, i = 1:N )
*(!)  If necessary, scale A to protect the largest singular value
*     from overflow. It is possible that saving the largest singular
*     value destroys the information about the small ones.
*     This initial scaling is almost minimal in the sense that the
*     goal is to make sure that no column norm overflows, and that
*     DSQRT(N)*max_i SVA(i) does not overflow. If INFinite entries
*     in A are detected, the procedure returns with INFO=-6.
*
      SKL= ONE / DSQRT( DBLE( M )*DBLE( N ) )
      NOSCALE = .TRUE.
      GOSCALE = .TRUE.
*
      IF( LOWER ) THEN
*        the input matrix is M-by-N lower triangular (trapezoidal)
         DO 1874 p = 1, N
            AAPP = ZERO
            AAQQ = ONE
            CALL DLASSQ( M-p+1, A( p, p ), 1, AAPP, AAQQ )
            IF( AAPP.GT.BIG ) THEN
               INFO = -6
               CALL XERBLA( 'DGESVJ', -INFO )
               RETURN
            END IF
            AAQQ = DSQRT( AAQQ )
            IF( ( AAPP.LT.( BIG / AAQQ ) ) .AND. NOSCALE ) THEN
               SVA( p ) = AAPP*AAQQ
            ELSE
               NOSCALE = .FALSE.
               SVA( p ) = AAPP*( AAQQ*SKL)
               IF( GOSCALE ) THEN
                  GOSCALE = .FALSE.
                  DO 1873 q = 1, p - 1
                     SVA( q ) = SVA( q )*SKL
 1873             CONTINUE
               END IF
```

```
            END IF
 1874    CONTINUE
      ELSE IF( UPPER ) THEN
*        the input matrix is M-by-N upper triangular (trapezoidal)
         DO 2874 p = 1, N
            AAPP = ZERO
            AAQQ = ONE
            CALL DLASSQ( p, A( 1, p ), 1, AAPP, AAQQ )
            IF( AAPP.GT.BIG ) THEN
               INFO = -6
               CALL XERBLA( 'DGESVJ', -INFO )
               RETURN
            END IF
            AAQQ = DSQRT( AAQQ )
            IF( ( AAPP.LT.( BIG / AAQQ ) ) .AND. NOSCALE ) THEN
               SVA( p ) = AAPP*AAQQ
            ELSE
               NOSCALE = .FALSE.
               SVA( p ) = AAPP*( AAQQ*SKL)
               IF( GOSCALE ) THEN
                  GOSCALE = .FALSE.
                  DO 2873 q = 1, p - 1
                     SVA( q ) = SVA( q )*SKL
 2873             CONTINUE
               END IF
            END IF
 2874    CONTINUE
      ELSE
*        the input matrix is M-by-N general dense
         DO 3874 p = 1, N
            AAPP = ZERO
            AAQQ = ONE
            CALL DLASSQ( M, A( 1, p ), 1, AAPP, AAQQ )
            IF( AAPP.GT.BIG ) THEN
               INFO = -6
               CALL XERBLA( 'DGESVJ', -INFO )
               RETURN
            END IF
            AAQQ = DSQRT( AAQQ )
            IF( ( AAPP.LT.( BIG / AAQQ ) ) .AND. NOSCALE ) THEN
               SVA( p ) = AAPP*AAQQ
            ELSE
               NOSCALE = .FALSE.
               SVA( p ) = AAPP*( AAQQ*SKL)
               IF( GOSCALE ) THEN
                  GOSCALE = .FALSE.
                  DO 3873 q = 1, p - 1
                     SVA( q ) = SVA( q )*SKL
 3873             CONTINUE
               END IF
```

```
            END IF
 3874    CONTINUE
      END IF
*
      IF( NOSCALE )SKL= ONE
*
*     Move the smaller part of the spectrum from the underflow threshold
*(!)  Start by determining the position of the nonzero entries of the
*     array SVA() relative to ( SFMIN, BIG ).
*
      AAPP = ZERO
      AAQQ = BIG
      DO 4781 p = 1, N
         IF( SVA( p ).NE.ZERO )AAQQ = MIN( AAQQ, SVA( p ) )
         AAPP = MAX( AAPP, SVA( p ) )
 4781 CONTINUE
*
* #:) Quick return for zero matrix
*
      IF( AAPP.EQ.ZERO ) THEN
         IF( LSVEC )CALL DLASET( 'G', M, N, ZERO, ONE, A, LDA )
         WORK( 1 ) = ONE
         WORK( 2 ) = ZERO
         WORK( 3 ) = ZERO
         WORK( 4 ) = ZERO
         WORK( 5 ) = ZERO
         WORK( 6 ) = ZERO
         RETURN
      END IF
*
* #:) Quick return for one-column matrix
*
      IF( N.EQ.1 ) THEN
         IF( LSVEC )CALL DLASCL( 'G', 0, 0, SVA( 1 ), SKL, M, 1,
     $                           A( 1, 1 ), LDA, IERR )
         WORK( 1 ) = ONE / SKL
         IF( SVA( 1 ).GE.SFMIN ) THEN
            WORK( 2 ) = ONE
         ELSE
            WORK( 2 ) = ZERO
         END IF
         WORK( 3 ) = ZERO
         WORK( 4 ) = ZERO
         WORK( 5 ) = ZERO
         WORK( 6 ) = ZERO
         RETURN
      END IF
*
*     Protect small singular values from underflow, and try to
*     avoid underflows/overflows in computing Jacobi rotations.
```

```
*
      SN = DSQRT( SFMIN / EPSLN )
      TEMP1 = DSQRT( BIG / DBLE( N ) )
      IF( ( AAPP.LE.SN ) .OR. ( AAQQ.GE.TEMP1 ) .OR.
     $    ( ( SN.LE.AAQQ ) .AND. ( AAPP.LE.TEMP1 ) ) ) THEN
         TEMP1 = MIN( BIG, TEMP1 / AAPP )
*         AAQQ  = AAQQ*TEMP1
*         AAPP  = AAPP*TEMP1
      ELSE IF( ( AAQQ.LE.SN ) .AND. ( AAPP.LE.TEMP1 ) ) THEN
         TEMP1 = MIN( SN / AAQQ, BIG / ( AAPP*DSQRT( DBLE( N ) ) ) )
*         AAQQ  = AAQQ*TEMP1
*         AAPP  = AAPP*TEMP1
      ELSE IF( ( AAQQ.GE.SN ) .AND. ( AAPP.GE.TEMP1 ) ) THEN
         TEMP1 = MAX( SN / AAQQ, TEMP1 / AAPP )
*         AAQQ  = AAQQ*TEMP1
*         AAPP  = AAPP*TEMP1
      ELSE IF( ( AAQQ.LE.SN ) .AND. ( AAPP.GE.TEMP1 ) ) THEN
         TEMP1 = MIN( SN / AAQQ, BIG / ( DSQRT( DBLE( N ) )*AAPP ) )
*         AAQQ  = AAQQ*TEMP1
*         AAPP  = AAPP*TEMP1
      ELSE
         TEMP1 = ONE
      END IF
*
*     Scale, if necessary
*
      IF( TEMP1.NE.ONE ) THEN
         CALL DLASCL( 'G', 0, 0, ONE, TEMP1, N, 1, SVA, N, IERR )
      END IF
      SKL= TEMP1*SKL
      IF( SKL.NE.ONE ) THEN
         CALL DLASCL( JOBA, 0, 0, ONE, SKL, M, N, A, LDA, IERR )
         SKL= ONE / SKL
      END IF
*
*     Row-cyclic Jacobi SVD algorithm with column pivoting
*
      EMPTSW = ( N*( N-1 ) ) / 2
      NOTROT = 0
      FASTR( 1 ) = ZERO
*
*     A is represented in factored form A = A * diag(WORK), where diag(WORK)
*     is initialized to identity. WORK is updated during fast scaled
*     rotations.
*
      DO 1868 q = 1, N
         WORK( q ) = ONE
 1868 CONTINUE
*
*
```

```
      SWBAND = 3
*[TP] SWBAND is a tuning parameter [TP]. It is meaningful and effective
*     if DGESVJ is used as a computational routine in the preconditioned
*     Jacobi SVD algorithm DGESVJ. For sweeps i=1:SWBAND the procedure
*     works on pivots inside a band-like region around the diagonal.
*     The boundaries are determined dynamically, based on the number of
*     pivots above a threshold.
*
      KBL = MIN( 8, N )
*[TP] KBL is a tuning parameter that defines the tile size in the
*     tiling of the p-q loops of pivot pairs. In general, an optimal
*     value of KBL depends on the matrix dimensions and on the
*     parameters of the computer's memory.
*
      NBL = N / KBL
      IF( ( NBL*KBL ).NE.N )NBL = NBL + 1
*
      BLSKIP = KBL**2
*[TP] BLKSKIP is a tuning parameter that depends on SWBAND and KBL.
*
      ROWSKIP = MIN( 5, KBL )
*[TP] ROWSKIP is a tuning parameter.
*
      LKAHEAD = 1
*[TP] LKAHEAD is a tuning parameter.
*
*     Quasi block transformations, using the lower (upper) triangular
*     structure of the input matrix. The quasi-block-cycling usually
*     invokes cubic convergence. Big part of this cycle is done inside
*     canonical subspaces of dimensions less than M.
*
      IF( ( LOWER .OR. UPPER ) .AND. ( N.GT.MAX( 64, 4*KBL ) ) ) THEN
*[TP] The number of partition levels and the actual partition are
*     tuning parameters.
         N4 = N / 4
         N2 = N / 2
         N34 = 3*N4
         IF( APPLV ) THEN
            q = 0
         ELSE
            q = 1
         END IF
*
         IF( LOWER ) THEN
*
*     This works very well on lower triangular matrices, in particular
*     in the framework of the preconditioned Jacobi SVD (xGEJSV).
*     The idea is simple:
*     [+ 0 0 0]   Note that Jacobi transformations of [0 0]
*     [+ + 0 0]                                       [0 0]
```

```
*     [+ + x 0]   actually work on [x 0]              [x 0]
*     [+ + x x]                    [x x].             [x x]
*
            CALL DGSVJ0( JOBV, M-N34, N-N34, A( N34+1, N34+1 ), LDA,
     $                   WORK( N34+1 ), SVA( N34+1 ), MVL,
     $                   V( N34*q+1, N34+1 ), LDV, EPSLN, SFMIN, TOL,
     $                   2, WORK( N+1 ), LWORK-N, IERR )
*
            CALL DGSVJ0( JOBV, M-N2, N34-N2, A( N2+1, N2+1 ), LDA,
     $                   WORK( N2+1 ), SVA( N2+1 ), MVL,
     $                   V( N2*q+1, N2+1 ), LDV, EPSLN, SFMIN, TOL, 2,
     $                   WORK( N+1 ), LWORK-N, IERR )
*
            CALL DGSVJ1( JOBV, M-N2, N-N2, N4, A( N2+1, N2+1 ), LDA,
     $                   WORK( N2+1 ), SVA( N2+1 ), MVL,
     $                   V( N2*q+1, N2+1 ), LDV, EPSLN, SFMIN, TOL, 1,
     $                   WORK( N+1 ), LWORK-N, IERR )
*
            CALL DGSVJ0( JOBV, M-N4, N2-N4, A( N4+1, N4+1 ), LDA,
     $                   WORK( N4+1 ), SVA( N4+1 ), MVL,
     $                   V( N4*q+1, N4+1 ), LDV, EPSLN, SFMIN, TOL, 1,
     $                   WORK( N+1 ), LWORK-N, IERR )
*
            CALL DGSVJ0( JOBV, M, N4, A, LDA, WORK, SVA, MVL, V, LDV,
     $                   EPSLN, SFMIN, TOL, 1, WORK( N+1 ), LWORK-N,
     $                   IERR )
*
            CALL DGSVJ1( JOBV, M, N2, N4, A, LDA, WORK, SVA, MVL, V,
     $                   LDV, EPSLN, SFMIN, TOL, 1, WORK( N+1 ),
     $                   LWORK-N, IERR )
*
*
         ELSE IF( UPPER ) THEN
*
*
            CALL DGSVJ0( JOBV, N4, N4, A, LDA, WORK, SVA, MVL, V, LDV,
     $                   EPSLN, SFMIN, TOL, 2, WORK( N+1 ), LWORK-N,
     $                   IERR )
*
            CALL DGSVJ0( JOBV, N2, N4, A( 1, N4+1 ), LDA, WORK( N4+1 ),
     $                   SVA( N4+1 ), MVL, V( N4*q+1, N4+1 ), LDV,
     $                   EPSLN, SFMIN, TOL, 1, WORK( N+1 ), LWORK-N,
     $                   IERR )
*
            CALL DGSVJ1( JOBV, N2, N2, N4, A, LDA, WORK, SVA, MVL, V,
     $                   LDV, EPSLN, SFMIN, TOL, 1, WORK( N+1 ),
     $                   LWORK-N, IERR )
*
            CALL DGSVJ0( JOBV, N2+N4, N4, A( 1, N2+1 ), LDA,
     $                   WORK( N2+1 ), SVA( N2+1 ), MVL,
```

```
     $                  V( N2*q+1, N2+1 ), LDV, EPSLN, SFMIN, TOL, 1,
     $                  WORK( N+1 ), LWORK-N, IERR )

         END IF
*
      END IF
*
*     .. Row-cyclic pivot strategy with de Rijk's pivoting ..
*
      DO 1993 i = 1, NSWEEP
*
*     .. go go go ...
*
         MXAAPQ = ZERO
         MXSINJ = ZERO
         ISWROT = 0
*
         NOTROT = 0
         PSKIPPED = 0
*
*     Each sweep is unrolled using KBL-by-KBL tiles over the pivot pairs
*     1 <= p < q <= N. This is the first step toward a blocked implementation
*     of the rotations. New implementation, based on block transformations,
*     is under development.
*
         DO 2000 ibr = 1, NBL
*
            igl = ( ibr-1 )*KBL + 1
*
            DO 1002 ir1 = 0, MIN( LKAHEAD, NBL-ibr )
*
               igl = igl + ir1*KBL
*
               DO 2001 p = igl, MIN( igl+KBL-1, N-1 )
*
*     .. de Rijk's pivoting
*
                  q = IDAMAX( N-p+1, SVA( p ), 1 ) + p - 1
                  IF( p.NE.q ) THEN
                     CALL DSWAP( M, A( 1, p ), 1, A( 1, q ), 1 )
                     IF( RSVEC )CALL DSWAP( MVL, V( 1, p ), 1,
     $                                      V( 1, q ), 1 )
                     TEMP1 = SVA( p )
                     SVA( p ) = SVA( q )
                     SVA( q ) = TEMP1
                     TEMP1 = WORK( p )
                     WORK( p ) = WORK( q )
                     WORK( q ) = TEMP1
                  END IF
*
```

```
                 IF( ir1.EQ.0 ) THEN
*
*        Column norms are periodically updated by explicit
*        norm computation.
*        Caveat:
*        Unfortunately, some BLAS implementations compute DNRM2(M,A(1,p),1)
*        as DSQRT(DDOT(M,A(1,p),1,A(1,p),1)), which may cause the result to
*        overflow for ||A(:,p)||_2 > DSQRT(overflow_threshold), and to
*        underflow for ||A(:,p)||_2 < DSQRT(underflow_threshold).
*        Hence, DNRM2 cannot be trusted, not even in the case when
*        the true norm is far from the under(over)flow boundaries.
*        If properly implemented DNRM2 is available, the IF-THEN-ELSE
*        below should read "AAPP = DNRM2( M, A(1,p), 1 ) * WORK(p)".
*
                    IF( ( SVA( p ).LT.ROOTBIG ) .AND.
     $                  ( SVA( p ).GT.ROOTSFMIN ) ) THEN
                       SVA( p ) = DNRM2( M, A( 1, p ), 1 )*WORK( p )
                    ELSE
                       TEMP1 = ZERO
                       AAPP = ONE
                       CALL DLASSQ( M, A( 1, p ), 1, TEMP1, AAPP )
                       SVA( p ) = TEMP1*DSQRT( AAPP )*WORK( p )
                    END IF
                    AAPP = SVA( p )
                 ELSE
                    AAPP = SVA( p )
                 END IF
*
                 IF( AAPP.GT.ZERO ) THEN
*
                    PSKIPPED = 0
*
                    DO 2002 q = p + 1, MIN( igl+KBL-1, N )
*
                       AAQQ = SVA( q )
*
                       IF( AAQQ.GT.ZERO ) THEN
*
                          AAPP0 = AAPP
                          IF( AAQQ.GE.ONE ) THEN
                             ROTOK = ( SMALL*AAPP ).LE.AAQQ
                             IF( AAPP.LT.( BIG / AAQQ ) ) THEN
                                AAPQ = ( DDOT( M, A( 1, p ), 1, A( 1,
     $                                 q ), 1 )*WORK( p )*WORK( q ) /
     $                                 AAQQ ) / AAPP
                             ELSE
                                CALL DCOPY( M, A( 1, p ), 1,
     $                                      WORK( N+1 ), 1 )
                                CALL DLASCL( 'G', 0, 0, AAPP,
     $                                       WORK( p ), M, 1,
```

```
     $                                     WORK( N+1 ), LDA, IERR )
                              AAPQ = DDOT( M, WORK( N+1 ), 1,
     $                               A( 1, q ), 1 )*WORK( q ) / AAQQ
                           END IF
                        ELSE
                           ROTOK = AAPP.LE.( AAQQ / SMALL )
                           IF( AAPP.GT.( SMALL / AAQQ ) ) THEN
                              AAPQ = ( DDOT( M, A( 1, p ), 1, A( 1,
     $                               q ), 1 )*WORK( p )*WORK( q ) /
     $                               AAQQ ) / AAPP
                           ELSE
                              CALL DCOPY( M, A( 1, q ), 1,
     $                                    WORK( N+1 ), 1 )
                              CALL DLASCL( 'G', 0, 0, AAQQ,
     $                                     WORK( q ), M, 1,
     $                                     WORK( N+1 ), LDA, IERR )
                              AAPQ = DDOT( M, WORK( N+1 ), 1,
     $                               A( 1, p ), 1 )*WORK( p ) / AAPP
                           END IF
                        END IF
*
                        MXAAPQ = MAX( MXAAPQ, DABS( AAPQ ) )
*
*       TO rotate or NOT to rotate, THAT is the question ...
*
                        IF( DABS( AAPQ ).GT.TOL ) THEN
*
*          .. rotate
*[RTD]      ROTATED = ROTATED + ONE
*
                           IF( ir1.EQ.0 ) THEN
                              NOTROT = 0
                              PSKIPPED = 0
                              ISWROT = ISWROT + 1
                           END IF
*
                           IF( ROTOK ) THEN
*
                              AQOAP = AAQQ / AAPP
                              APOAQ = AAPP / AAQQ
                              THETA = -HALF*DABS(AQOAP-APOAQ)/AAPQ
*
                              IF( DABS( THETA ).GT.BIGTHETA ) THEN
*
                                 T = HALF / THETA
                                 FASTR( 3 ) = T*WORK( p ) / WORK( q )
                                 FASTR( 4 ) = -T*WORK( q ) /
     $                                        WORK( p )
                                 CALL DROTM( M, A( 1, p ), 1,
     $                                       A( 1, q ), 1, FASTR )
```

```
                                 IF( RSVEC )CALL DROTM( MVL,
     $                                          V( 1, p ), 1,
     $                                          V( 1, q ), 1,
     $                                          FASTR )
                                 SVA( q ) = AAQQ*DSQRT( MAX( ZERO,
     $                                     ONE+T*APOAQ*AAPQ ) )
                                 AAPP = AAPP*DSQRT( MAX( ZERO,
     $                                 ONE-T*AQOAP*AAPQ ) )
                                 MXSINJ = MAX( MXSINJ, DABS( T ) )
*
                              ELSE
*
*                 .. choose correct signum for THETA and rotate
*
                                 THSIGN = -DSIGN( ONE, AAPQ )
                                 T = ONE / ( THETA+THSIGN*
     $                               DSQRT( ONE+THETA*THETA ) )
                                 CS = DSQRT( ONE / ( ONE+T*T ) )
                                 SN = T*CS
*
                                 MXSINJ = MAX( MXSINJ, DABS( SN ) )
                                 SVA( q ) = AAQQ*DSQRT( MAX( ZERO,
     $                                     ONE+T*APOAQ*AAPQ ) )
                                 AAPP = AAPP*DSQRT( MAX( ZERO,
     $                                 ONE-T*AQOAP*AAPQ ) )
*
                                 APOAQ = WORK( p ) / WORK( q )
                                 AQOAP = WORK( q ) / WORK( p )
                                 IF( WORK( p ).GE.ONE ) THEN
                                    IF( WORK( q ).GE.ONE ) THEN
                                       FASTR( 3 ) = T*APOAQ
                                       FASTR( 4 ) = -T*AQOAP
                                       WORK( p ) = WORK( p )*CS
                                       WORK( q ) = WORK( q )*CS
                                       CALL DROTM( M, A( 1, p ), 1,
     $                                             A( 1, q ), 1,
     $                                             FASTR )
                                       IF( RSVEC )CALL DROTM( MVL,
     $                                     V( 1, p ), 1, V( 1, q ),
     $                                     1, FASTR )
                                    ELSE
                                       CALL DAXPY( M, -T*AQOAP,
     $                                             A( 1, q ), 1,
     $                                             A( 1, p ), 1 )
                                       CALL DAXPY( M, CS*SN*APOAQ,
     $                                             A( 1, p ), 1,
     $                                             A( 1, q ), 1 )
                                       WORK( p ) = WORK( p )*CS
                                       WORK( q ) = WORK( q ) / CS
                                       IF( RSVEC ) THEN
```

```
                                    CALL DAXPY( MVL, -T*AQOAP,
     $                                          V( 1, q ), 1,
     $                                          V( 1, p ), 1 )
                                    CALL DAXPY( MVL,
     $                                          CS*SN*APOAQ,
     $                                          V( 1, p ), 1,
     $                                          V( 1, q ), 1 )
                                 END IF
                              END IF
                           ELSE
                              IF( WORK( q ).GE.ONE ) THEN
                                 CALL DAXPY( M, T*APOAQ,
     $                                       A( 1, p ), 1,
     $                                       A( 1, q ), 1 )
                                 CALL DAXPY( M, -CS*SN*AQOAP,
     $                                       A( 1, q ), 1,
     $                                       A( 1, p ), 1 )
                                 WORK( p ) = WORK( p ) / CS
                                 WORK( q ) = WORK( q )*CS
                                 IF( RSVEC ) THEN
                                    CALL DAXPY( MVL, T*APOAQ,
     $                                          V( 1, p ), 1,
     $                                          V( 1, q ), 1 )
                                    CALL DAXPY( MVL,
     $                                          -CS*SN*AQOAP,
     $                                          V( 1, q ), 1,
     $                                          V( 1, p ), 1 )
                                 END IF
                              ELSE
                                 IF( WORK( p ).GE.WORK( q ) )
     $                               THEN
                                    CALL DAXPY( M, -T*AQOAP,
     $                                          A( 1, q ), 1,
     $                                          A( 1, p ), 1 )
                                    CALL DAXPY( M, CS*SN*APOAQ,
     $                                          A( 1, p ), 1,
     $                                          A( 1, q ), 1 )
                                    WORK( p ) = WORK( p )*CS
                                    WORK( q ) = WORK( q ) / CS
                                    IF( RSVEC ) THEN
                                       CALL DAXPY( MVL,
     $                                          -T*AQOAP,
     $                                          V( 1, q ), 1,
     $                                          V( 1, p ), 1 )
                                       CALL DAXPY( MVL,
     $                                          CS*SN*APOAQ,
     $                                          V( 1, p ), 1,
     $                                          V( 1, q ), 1 )
                                    END IF
                                 ELSE
```

```
                                        CALL DAXPY( M, T*APOAQ,
     $                                              A( 1, p ), 1,
     $                                              A( 1, q ), 1 )
                                        CALL DAXPY( M,
     $                                              -CS*SN*AQOAP,
     $                                              A( 1, q ), 1,
     $                                              A( 1, p ), 1 )
                                        WORK( p ) = WORK( p ) / CS
                                        WORK( q ) = WORK( q )*CS
                                        IF( RSVEC ) THEN
                                           CALL DAXPY( MVL,
     $                                           T*APOAQ, V( 1, p ),
     $                                           1, V( 1, q ), 1 )
                                           CALL DAXPY( MVL,
     $                                           -CS*SN*AQOAP,
     $                                           V( 1, q ), 1,
     $                                           V( 1, p ), 1 )
                                        END IF
                                     END IF
                                  END IF
                               END IF
                            END IF
*
                         ELSE
*             .. have to use modified Gram-Schmidt like transformation
                            CALL DCOPY( M, A( 1, p ), 1,
     $                                  WORK( N+1 ), 1 )
                            CALL DLASCL( 'G', 0, 0, AAPP, ONE, M,
     $                                   1, WORK( N+1 ), LDA,
     $                                   IERR )
                            CALL DLASCL( 'G', 0, 0, AAQQ, ONE, M,
     $                                   1, A( 1, q ), LDA, IERR )
                            TEMP1 = -AAPQ*WORK( p ) / WORK( q )
                            CALL DAXPY( M, TEMP1, WORK( N+1 ), 1,
     $                                  A( 1, q ), 1 )
                            CALL DLASCL( 'G', 0, 0, ONE, AAQQ, M,
     $                                   1, A( 1, q ), LDA, IERR )
                            SVA( q ) = AAQQ*DSQRT( MAX( ZERO,
     $                                 ONE-AAPQ*AAPQ ) )
                            MXSINJ = MAX( MXSINJ, SFMIN )
                         END IF
*          END IF ROTOK THEN ... ELSE
*
*          In the case of cancellation in updating SVA(q), SVA(p)
*          recompute SVA(q), SVA(p).
*
                         IF( ( SVA( q ) / AAQQ )**2.LE.ROOTEPS )
     $                       THEN
                            IF( ( AAQQ.LT.ROOTBIG ) .AND.
     $                          ( AAQQ.GT.ROOTSFMIN ) ) THEN
```

```
                                 SVA( q ) = DNRM2( M, A( 1, q ), 1 )*
     $                                     WORK( q )
                              ELSE
                                 T = ZERO
                                 AAQQ = ONE
                                 CALL DLASSQ( M, A( 1, q ), 1, T,
     $                                       AAQQ )
                                 SVA( q ) = T*DSQRT( AAQQ )*WORK( q )
                              END IF
                           END IF
                           IF( ( AAPP / AAPP0 ).LE.ROOTEPS ) THEN
                              IF( ( AAPP.LT.ROOTBIG ) .AND.
     $                            ( AAPP.GT.ROOTSFMIN ) ) THEN
                                 AAPP = DNRM2( M, A( 1, p ), 1 )*
     $                                  WORK( p )
                              ELSE
                                 T = ZERO
                                 AAPP = ONE
                                 CALL DLASSQ( M, A( 1, p ), 1, T,
     $                                       AAPP )
                                 AAPP = T*DSQRT( AAPP )*WORK( p )
                              END IF
                              SVA( p ) = AAPP
                           END IF
*
                        ELSE
*        A(:,p) and A(:,q) already numerically orthogonal
                           IF( ir1.EQ.0 )NOTROT = NOTROT + 1
*[RTD]      SKIPPED  = SKIPPED  + 1
                           PSKIPPED = PSKIPPED + 1
                        END IF
                     ELSE
*        A(:,q) is zero column
                        IF( ir1.EQ.0 )NOTROT = NOTROT + 1
                        PSKIPPED = PSKIPPED + 1
                     END IF
*
                     IF( ( i.LE.SWBAND ) .AND.
     $                   ( PSKIPPED.GT.ROWSKIP ) ) THEN
                        IF( ir1.EQ.0 )AAPP = -AAPP
                        NOTROT = 0
                        GO TO 2103
                     END IF
*
 2002             CONTINUE
*     END q-LOOP
*
 2103             CONTINUE
*     bailed out of q-loop
*
```

```
                    SVA( p ) = AAPP
*
                 ELSE
                    SVA( p ) = AAPP
                    IF( ( ir1.EQ.0 ) .AND. ( AAPP.EQ.ZERO ) )
     $                  NOTROT = NOTROT + MIN( igl+KBL-1, N ) - p
                 END IF
*
 2001          CONTINUE
*     end of the p-loop
*     end of doing the block ( ibr, ibr )
 1002       CONTINUE
*     end of ir1-loop
*
* ... go to the off diagonal blocks
*
            igl = ( ibr-1 )*KBL + 1
*
            DO 2010 jbc = ibr + 1, NBL
*
               jgl = ( jbc-1 )*KBL + 1
*
*        doing the block at ( ibr, jbc )
*
               IJBLSK = 0
               DO 2100 p = igl, MIN( igl+KBL-1, N )
*
                  AAPP = SVA( p )
                  IF( AAPP.GT.ZERO ) THEN
*
                     PSKIPPED = 0
*
                     DO 2200 q = jgl, MIN( jgl+KBL-1, N )
*
                        AAQQ = SVA( q )
                        IF( AAQQ.GT.ZERO ) THEN
                           AAPP0 = AAPP
*
*     .. M x 2 Jacobi SVD ..
*
*        Safe Gram matrix computation
*
                           IF( AAQQ.GE.ONE ) THEN
                              IF( AAPP.GE.AAQQ ) THEN
                                 ROTOK = ( SMALL*AAPP ).LE.AAQQ
                              ELSE
                                 ROTOK = ( SMALL*AAQQ ).LE.AAPP
                              END IF
                              IF( AAPP.LT.( BIG / AAQQ ) ) THEN
                                 AAPQ = ( DDOT( M, A( 1, p ), 1, A( 1,
```

```
     $                                q ), 1 )*WORK( p )*WORK( q ) /
     $                                AAQQ ) / AAPP
                              ELSE
                                 CALL DCOPY( M, A( 1, p ), 1,
     $                                       WORK( N+1 ), 1 )
                                 CALL DLASCL( 'G', 0, 0, AAPP,
     $                                        WORK( p ), M, 1,
     $                                        WORK( N+1 ), LDA, IERR )
                                 AAPQ = DDOT( M, WORK( N+1 ), 1,
     $                                A( 1, q ), 1 )*WORK( q ) / AAQQ
                              END IF
                           ELSE
                              IF( AAPP.GE.AAQQ ) THEN
                                 ROTOK = AAPP.LE.( AAQQ / SMALL )
                              ELSE
                                 ROTOK = AAQQ.LE.( AAPP / SMALL )
                              END IF
                              IF( AAPP.GT.( SMALL / AAQQ ) ) THEN
                                 AAPQ = ( DDOT( M, A( 1, p ), 1, A( 1,
     $                                q ), 1 )*WORK( p )*WORK( q ) /
     $                                AAQQ ) / AAPP
                              ELSE
                                 CALL DCOPY( M, A( 1, q ), 1,
     $                                       WORK( N+1 ), 1 )
                                 CALL DLASCL( 'G', 0, 0, AAQQ,
     $                                        WORK( q ), M, 1,
     $                                        WORK( N+1 ), LDA, IERR )
                                 AAPQ = DDOT( M, WORK( N+1 ), 1,
     $                                A( 1, p ), 1 )*WORK( p ) / AAPP
                              END IF
                           END IF
*
                           MXAAPQ = MAX( MXAAPQ, DABS( AAPQ ) )
*
*        TO rotate or NOT to rotate, THAT is the question ...
*
                           IF( DABS( AAPQ ).GT.TOL ) THEN
                              NOTROT = 0
*[RTD]      ROTATED  = ROTATED + 1
                              PSKIPPED = 0
                              ISWROT = ISWROT + 1
*
                              IF( ROTOK ) THEN
*
                                 AQOAP = AAQQ / AAPP
                                 APOAQ = AAPP / AAQQ
                                 THETA = -HALF*DABS(AQOAP-APOAQ)/AAPQ
                                 IF( AAQQ.GT.AAPPO )THETA = -THETA
*
                                 IF( DABS( THETA ).GT.BIGTHETA ) THEN
```

```
                                 T = HALF / THETA
                                 FASTR( 3 ) = T*WORK( p ) / WORK( q )
                                 FASTR( 4 ) = -T*WORK( q ) /
     $                                         WORK( p )
                                 CALL DROTM( M, A( 1, p ), 1,
     $                                       A( 1, q ), 1, FASTR )
                                 IF( RSVEC )CALL DROTM( MVL,
     $                                           V( 1, p ), 1,
     $                                           V( 1, q ), 1,
     $                                           FASTR )
                                 SVA( q ) = AAQQ*DSQRT( MAX( ZERO,
     $                                     ONE+T*APOAQ*AAPQ ) )
                                 AAPP = AAPP*DSQRT( MAX( ZERO,
     $                                  ONE-T*AQOAP*AAPQ ) )
                                 MXSINJ = MAX( MXSINJ, DABS( T ) )
                              ELSE
*
*                 .. choose correct signum for THETA and rotate
*
                                 THSIGN = -DSIGN( ONE, AAPQ )
                                 IF( AAQQ.GT.AAPPO )THSIGN = -THSIGN
                                 T = ONE / ( THETA+THSIGN*
     $                               DSQRT( ONE+THETA*THETA ) )
                                 CS = DSQRT( ONE / ( ONE+T*T ) )
                                 SN = T*CS
                                 MXSINJ = MAX( MXSINJ, DABS( SN ) )
                                 SVA( q ) = AAQQ*DSQRT( MAX( ZERO,
     $                                     ONE+T*APOAQ*AAPQ ) )
                                 AAPP = AAPP*DSQRT( MAX( ZERO,
     $                                  ONE-T*AQOAP*AAPQ ) )
*
                                 APOAQ = WORK( p ) / WORK( q )
                                 AQOAP = WORK( q ) / WORK( p )
                                 IF( WORK( p ).GE.ONE ) THEN
*
                                    IF( WORK( q ).GE.ONE ) THEN
                                       FASTR( 3 ) = T*APOAQ
                                       FASTR( 4 ) = -T*AQOAP
                                       WORK( p ) = WORK( p )*CS
                                       WORK( q ) = WORK( q )*CS
                                       CALL DROTM( M, A( 1, p ), 1,
     $                                             A( 1, q ), 1,
     $                                             FASTR )
                                       IF( RSVEC )CALL DROTM( MVL,
     $                                     V( 1, p ), 1, V( 1, q ),
     $                                     1, FASTR )
                                    ELSE
                                       CALL DAXPY( M, -T*AQOAP,
     $                                             A( 1, q ), 1,
     $                                             A( 1, p ), 1 )
```

```
                                    CALL DAXPY( M, CS*SN*APOAQ,
     $                                          A( 1, p ), 1,
     $                                          A( 1, q ), 1 )
                                    IF( RSVEC ) THEN
                                       CALL DAXPY( MVL, -T*AQOAP,
     $                                             V( 1, q ), 1,
     $                                             V( 1, p ), 1 )
                                       CALL DAXPY( MVL,
     $                                             CS*SN*APOAQ,
     $                                             V( 1, p ), 1,
     $                                             V( 1, q ), 1 )
                                    END IF
                                    WORK( p ) = WORK( p )*CS
                                    WORK( q ) = WORK( q ) / CS
                                 END IF
                              ELSE
                                 IF( WORK( q ).GE.ONE ) THEN
                                    CALL DAXPY( M, T*APOAQ,
     $                                          A( 1, p ), 1,
     $                                          A( 1, q ), 1 )
                                    CALL DAXPY( M, -CS*SN*AQOAP,
     $                                          A( 1, q ), 1,
     $                                          A( 1, p ), 1 )
                                    IF( RSVEC ) THEN
                                       CALL DAXPY( MVL, T*APOAQ,
     $                                             V( 1, p ), 1,
     $                                             V( 1, q ), 1 )
                                       CALL DAXPY( MVL,
     $                                             -CS*SN*AQOAP,
     $                                             V( 1, q ), 1,
     $                                             V( 1, p ), 1 )
                                    END IF
                                    WORK( p ) = WORK( p ) / CS
                                    WORK( q ) = WORK( q )*CS
                                 ELSE
                                    IF( WORK( p ).GE.WORK( q ) )
     $                               THEN
                                       CALL DAXPY( M, -T*AQOAP,
     $                                             A( 1, q ), 1,
     $                                             A( 1, p ), 1 )
                                       CALL DAXPY( M, CS*SN*APOAQ,
     $                                             A( 1, p ), 1,
     $                                             A( 1, q ), 1 )
                                       WORK( p ) = WORK( p )*CS
                                       WORK( q ) = WORK( q ) / CS
                                       IF( RSVEC ) THEN
                                          CALL DAXPY( MVL,
     $                                          -T*AQOAP,
     $                                          V( 1, q ), 1,
     $                                          V( 1, p ), 1 )
```

```
                                          CALL DAXPY( MVL,
     $                                         CS*SN*APOAQ,
     $                                         V( 1, p ), 1,
     $                                         V( 1, q ), 1 )
                                       END IF
                                    ELSE
                                       CALL DAXPY( M, T*APOAQ,
     $                                            A( 1, p ), 1,
     $                                            A( 1, q ), 1 )
                                       CALL DAXPY( M,
     $                                            -CS*SN*AQOAP,
     $                                            A( 1, q ), 1,
     $                                            A( 1, p ), 1 )
                                       WORK( p ) = WORK( p ) / CS
                                       WORK( q ) = WORK( q )*CS
                                       IF( RSVEC ) THEN
                                          CALL DAXPY( MVL,
     $                                         T*APOAQ, V( 1, p ),
     $                                         1, V( 1, q ), 1 )
                                          CALL DAXPY( MVL,
     $                                         -CS*SN*AQOAP,
     $                                         V( 1, q ), 1,
     $                                         V( 1, p ), 1 )
                                       END IF
                                    END IF
                                 END IF
                              END IF
                           END IF
*
                        ELSE
                           IF( AAPP.GT.AAQQ ) THEN
                              CALL DCOPY( M, A( 1, p ), 1,
     $                                    WORK( N+1 ), 1 )
                              CALL DLASCL( 'G', 0, 0, AAPP, ONE,
     $                                     M, 1, WORK( N+1 ), LDA,
     $                                     IERR )
                              CALL DLASCL( 'G', 0, 0, AAQQ, ONE,
     $                                     M, 1, A( 1, q ), LDA,
     $                                     IERR )
                              TEMP1 = -AAPQ*WORK( p ) / WORK( q )
                              CALL DAXPY( M, TEMP1, WORK( N+1 ),
     $                                    1, A( 1, q ), 1 )
                              CALL DLASCL( 'G', 0, 0, ONE, AAQQ,
     $                                     M, 1, A( 1, q ), LDA,
     $                                     IERR )
                              SVA( q ) = AAQQ*DSQRT( MAX( ZERO,
     $                                   ONE-AAPQ*AAPQ ) )
                              MXSINJ = MAX( MXSINJ, SFMIN )
                           ELSE
                              CALL DCOPY( M, A( 1, q ), 1,
```

```
     $                                        WORK( N+1 ), 1 )
                                 CALL DLASCL( 'G', 0, 0, AAQQ, ONE,
     $                                        M, 1, WORK( N+1 ), LDA,
     $                                        IERR )
                                 CALL DLASCL( 'G', 0, 0, AAPP, ONE,
     $                                        M, 1, A( 1, p ), LDA,
     $                                        IERR )
                                 TEMP1 = -AAPQ*WORK( q ) / WORK( p )
                                 CALL DAXPY( M, TEMP1, WORK( N+1 ),
     $                                       1, A( 1, p ), 1 )
                                 CALL DLASCL( 'G', 0, 0, ONE, AAPP,
     $                                        M, 1, A( 1, p ), LDA,
     $                                        IERR )
                                 SVA( p ) = AAPP*DSQRT( MAX( ZERO,
     $                                      ONE-AAPQ*AAPQ ) )
                                 MXSINJ = MAX( MXSINJ, SFMIN )
                              END IF
                           END IF
*          END IF ROTOK THEN ... ELSE
*
*          In the case of cancellation in updating SVA(q)
*          .. recompute SVA(q)
                           IF( ( SVA( q ) / AAQQ )**2.LE.ROOTEPS )
     $                         THEN
                              IF( ( AAQQ.LT.ROOTBIG ) .AND.
     $                            ( AAQQ.GT.ROOTSFMIN ) ) THEN
                                 SVA( q ) = DNRM2( M, A( 1, q ), 1 )*
     $                                      WORK( q )
                              ELSE
                                 T = ZERO
                                 AAQQ = ONE
                                 CALL DLASSQ( M, A( 1, q ), 1, T,
     $                                        AAQQ )
                                 SVA( q ) = T*DSQRT( AAQQ )*WORK( q )
                              END IF
                           END IF
                           IF( ( AAPP / AAPP0 )**2.LE.ROOTEPS ) THEN
                              IF( ( AAPP.LT.ROOTBIG ) .AND.
     $                            ( AAPP.GT.ROOTSFMIN ) ) THEN
                                 AAPP = DNRM2( M, A( 1, p ), 1 )*
     $                                 WORK( p )
                              ELSE
                                 T = ZERO
                                 AAPP = ONE
                                 CALL DLASSQ( M, A( 1, p ), 1, T,
     $                                        AAPP )
                                 AAPP = T*DSQRT( AAPP )*WORK( p )
                              END IF
                              SVA( p ) = AAPP
                           END IF
```

```
*              end of OK rotation
                          ELSE
                             NOTROT = NOTROT + 1
*[RTD]      SKIPPED  = SKIPPED  + 1
                             PSKIPPED = PSKIPPED + 1
                             IJBLSK = IJBLSK + 1
                          END IF
                       ELSE
                          NOTROT = NOTROT + 1
                          PSKIPPED = PSKIPPED + 1
                          IJBLSK = IJBLSK + 1
                       END IF
*
                       IF( ( i.LE.SWBAND ) .AND. ( IJBLSK.GE.BLSKIP ) )
     $                     THEN
                          SVA( p ) = AAPP
                          NOTROT = 0
                          GO TO 2011
                       END IF
                       IF( ( i.LE.SWBAND ) .AND.
     $                     ( PSKIPPED.GT.ROWSKIP ) ) THEN
                          AAPP = -AAPP
                          NOTROT = 0
                          GO TO 2203
                       END IF
*
 2200                CONTINUE
*        end of the q-loop
 2203                CONTINUE
*
                     SVA( p ) = AAPP
*
                  ELSE
*
                     IF( AAPP.EQ.ZERO )NOTROT = NOTROT +
     $                   MIN( jgl+KBL-1, N ) - jgl + 1
                     IF( AAPP.LT.ZERO )NOTROT = 0
*
                  END IF
*
 2100          CONTINUE
*     end of the p-loop
 2010       CONTINUE
*     end of the jbc-loop
 2011       CONTINUE
*2011 bailed out of the jbc-loop
            DO 2012 p = igl, MIN( igl+KBL-1, N )
               SVA( p ) = DABS( SVA( p ) )
 2012       CONTINUE
***
```

```
 2000    CONTINUE
*2000 :: end of the ibr-loop
*
*     .. update SVA(N)
         IF( ( SVA( N ).LT.ROOTBIG ) .AND. ( SVA( N ).GT.ROOTSFMIN ) )
     $       THEN
            SVA( N ) = DNRM2( M, A( 1, N ), 1 )*WORK( N )
         ELSE
            T = ZERO
            AAPP = ONE
            CALL DLASSQ( M, A( 1, N ), 1, T, AAPP )
            SVA( N ) = T*DSQRT( AAPP )*WORK( N )
         END IF
*
*     Additional steering devices
*
         IF( ( i.LT.SWBAND ) .AND. ( ( MXAAPQ.LE.ROOTTOL ) .OR.
     $       ( ISWROT.LE.N ) ) )SWBAND = i
*
         IF( ( i.GT.SWBAND+1 ) .AND. ( MXAAPQ.LT.DSQRT( DBLE( N ) )*
     $       TOL ) .AND. ( DBLE( N )*MXAAPQ*MXSINJ.LT.TOL ) ) THEN
            GO TO 1994
         END IF
*
         IF( NOTROT.GE.EMPTSW )GO TO 1994
*
 1993 CONTINUE
*     end i=1:NSWEEP loop
*
* #:( Reaching this point means that the procedure has not converged.
      INFO = NSWEEP - 1
      GO TO 1995
*
 1994 CONTINUE
* #:) Reaching this point means numerical convergence after the i-th
*     sweep.
*
      INFO = 0
* #:) INFO = 0 confirms successful iterations.
 1995 CONTINUE
*
*     Sort the singular values and find how many are above
*     the underflow threshold.
*
      N2 = 0
      N4 = 0
      DO 5991 p = 1, N - 1
         q = IDAMAX( N-p+1, SVA( p ), 1 ) + p - 1
         IF( p.NE.q ) THEN
            TEMP1 = SVA( p )
```

```
            SVA( p ) = SVA( q )
            SVA( q ) = TEMP1
            TEMP1 = WORK( p )
            WORK( p ) = WORK( q )
            WORK( q ) = TEMP1
            CALL DSWAP( M, A( 1, p ), 1, A( 1, q ), 1 )
            IF( RSVEC )CALL DSWAP( MVL, V( 1, p ), 1, V( 1, q ), 1 )
         END IF
         IF( SVA( p ).NE.ZERO ) THEN
            N4 = N4 + 1
            IF( SVA( p )*SKL.GT.SFMIN )N2 = N2 + 1
         END IF
 5991 CONTINUE
      IF( SVA( N ).NE.ZERO ) THEN
         N4 = N4 + 1
         IF( SVA( N )*SKL.GT.SFMIN )N2 = N2 + 1
      END IF
*
*     Normalize the left singular vectors.
*
      IF( LSVEC .OR. UCTOL ) THEN
         DO 1998 p = 1, N2
            CALL DSCAL( M, WORK( p ) / SVA( p ), A( 1, p ), 1 )
 1998    CONTINUE
      END IF
*
*     Scale the product of Jacobi rotations (assemble the fast rotations).
*
      IF( RSVEC ) THEN
         IF( APPLV ) THEN
            DO 2398 p = 1, N
               CALL DSCAL( MVL, WORK( p ), V( 1, p ), 1 )
 2398       CONTINUE
         ELSE
            DO 2399 p = 1, N
               TEMP1 = ONE / DNRM2( MVL, V( 1, p ), 1 )
               CALL DSCAL( MVL, TEMP1, V( 1, p ), 1 )
 2399       CONTINUE
         END IF
      END IF
*
*     Undo scaling, if necessary (and possible).
      IF( ( ( SKL.GT.ONE ) .AND. ( SVA( 1 ).LT.( BIG / SKL) ) )
     $    .OR. ( ( SKL.LT.ONE ) .AND. ( SVA( MAX( N2, 1 ) ) .GT.
     $    ( SFMIN / SKL) ) ) ) THEN
         DO 2400 p = 1, N
            SVA( P ) = SKL*SVA( P )
 2400    CONTINUE
         SKL= ONE
      END IF
```

```
*
      WORK( 1 ) = SKL
*     The singular values of A are SKL*SVA(1:N). If SKL.NE.ONE
*     then some of the singular values may overflow or underflow and
*     the spectrum is given in this factored representation.
*
      WORK( 2 ) = DBLE( N4 )
*     N4 is the number of computed nonzero singular values of A.
*
      WORK( 3 ) = DBLE( N2 )
*     N2 is the number of singular values of A greater than SFMIN.
*     If N2<N, SVA(N2:N) contains ZEROS and/or denormalized numbers
*     that may carry some information.
*
      WORK( 4 ) = DBLE( i )
*     i is the index of the last sweep before declaring convergence.
*
      WORK( 5 ) = MXAAPQ
*     MXAAPQ is the largest absolute value of scaled pivots in the
*     last sweep
*
      WORK( 6 ) = MXSINJ
*     MXSINJ is the largest absolute value of the sines of Jacobi angles
*     in the last sweep
*
      RETURN
*     ..
*     .. END OF DGESVJ
*     ..
      END
```

———-

32.28.4 lisp code

— dgesvj.lisp —

```
(let* ((zero 0.0d0) (half 0.5d0) (one 1.0d0) (nsweep 30))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 0.5d0 0.5d0) half)
           (type (double-float 1.0d0 1.0d0) one)
           (type (f2cl-lib:integer4 30 30) nsweep)
           (ignorable zero half one nsweep))
  (defun dgesvj (joba jobu jobv m n a lda sva mv v ldv work lwork info)
    (declare (type (array double-float (*)) work v sva a)
             (type (f2cl-lib:integer4) info lwork ldv mv lda n m)
             (type (string *) jobv jobu joba))
    (f2cl-lib:with-multi-array-data
```

```
  ((joba character joba-%data% joba-%offset%)
   (jobu character jobu-%data% jobu-%offset%)
   (jobv character jobv-%data% jobv-%offset%)
   (a double-float a-%data% a-%offset%)
   (sva double-float sva-%data% sva-%offset%)
   (v double-float v-%data% v-%offset%)
   (work double-float work-%data% work-%offset%))
 (prog ((fastr (make-array 5 :element-type 'double-float)) (applv nil)
        (goscale nil) (lower nil) (lsvec nil) (noscale nil) (rotok nil)
        (rsvec nil) (uctol nil) (upper nil) (blskip 0) (emptsw 0) (i 0)
        (ibr 0) (ierr 0) (igl 0) (ijblsk 0) (ir1 0) (iswrot 0) (jbc 0)
        (jgl 0) (kbl 0) (lkahead 0) (mvl 0) (n2 0) (n34 0) (n4 0) (nbl 0)
        (notrot 0) (p 0) (pskipped 0) (q 0) (rowskip 0) (swband 0)
        (aapp 0.0d0) (aapp0 0.0d0) (aapq 0.0d0) (aaqq 0.0d0) (apoaq 0.0d0)
        (aqoap 0.0d0) (big 0.0d0) (bigtheta 0.0d0) (cs 0.0d0) (ctol 0.0d0)
        (epsln 0.0d0) (large 0.0d0) (mxaapq 0.0d0) (mxsinj 0.0d0)
        (rootbig 0.0d0) (rooteps 0.0d0) (rootsfmin 0.0d0) (roottol 0.0d0)
        (skl 0.0d0) (sfmin 0.0d0) (small 0.0d0) (sn 0.0d0) (temp1 0.0d0)
        (theta 0.0d0) (thsign 0.0d0) (tol 0.0d0) (t$ 0.0d0))
  (declare (type (array double-float (5)) fastr)
           (type f2cl-lib:logical applv goscale lower lsvec noscale rotok
            rsvec uctol upper)
           (type (f2cl-lib:integer4) blskip emptsw i ibr ierr igl ijblsk
            ir1 iswrot jbc jgl kbl lkahead mvl n2 n34 n4 nbl notrot p
            pskipped q rowskip swband)
           (type (double-float) t$ aapp aapp0 aapq aaqq apoaq aqoap big
            bigtheta cs ctol epsln large mxaapq mxsinj rootbig rooteps
            rootsfmin roottol skl sfmin small sn temp1 theta thsign tol))
  (setf lsvec
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame jobu "U")
            (declare (ignore var-1))
            (when var-0 (setf jobu var-0))
            ret-val))
  (setf uctol
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame jobu "C")
            (declare (ignore var-1))
            (when var-0 (setf jobu var-0))
            ret-val))
  (setf rsvec
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame jobv "V")
            (declare (ignore var-1))
            (when var-0 (setf jobv var-0))
            ret-val))
  (setf applv
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame jobv "A")
            (declare (ignore var-1))
```

```
          (when var-0 (setf jobv var-0))
          ret-val))
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame joba "U")
          (declare (ignore var-1))
          (when var-0 (setf joba var-0))
          ret-val))
(setf lower
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame joba "L")
          (declare (ignore var-1))
          (when var-0 (setf joba var-0))
          ret-val))
(cond
 ((not
   (or upper lower
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame joba "G")
         (declare (ignore var-1))
         (when var-0 (setf joba var-0))
         ret-val)))
  (setf info -1))
 ((not
   (or lsvec uctol
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobu "N")
         (declare (ignore var-1))
         (when var-0 (setf jobu var-0))
         ret-val)))
  (setf info -2))
 ((not
   (or rsvec applv
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobv "N")
         (declare (ignore var-1))
         (when var-0 (setf jobv var-0))
         ret-val)))
  (setf info -3))
 ((< m 0) (setf info -4)) ((or (< n 0) (> n m)) (setf info -5))
 ((< lda m) (setf info -7)) ((< mv 0) (setf info -9))
 ((or (and rsvec (< ldv n)) (and applv (< ldv mv))) (setf info -11))
 ((and uctol (<= (f2cl-lib:fref work (1) ((1 lwork))) one))
  (setf info -12))
 ((< lwork
     (max (the f2cl-lib:integer4 (f2cl-lib:int-add m n))
          (the f2cl-lib:integer4 6)))
  (setf info -13))
 (t (setf info 0)))
(cond
```

```
 ((/= info 0) (xerbla "DGESVJ" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= m 0) (= n 0))
    (go end_label))
(cond
 (uctol
  (setf ctol
          (f2cl-lib:fref work-%data% (1) ((1 lwork)) work-%offset%)))
 (t
  (cond
   ((or lsvec rsvec applv)
    (setf ctol (f2cl-lib:dsqrt (f2cl-lib:dble m))))
   (t (setf ctol (f2cl-lib:dble m))))))
(setf epsln (dlamch "Epsilon"))
(setf rooteps (f2cl-lib:dsqrt epsln))
(setf sfmin (dlamch "SafeMinimum"))
(setf rootsfmin (f2cl-lib:dsqrt sfmin))
(setf small (/ sfmin epsln))
(setf big (dlamch "Overflow"))
(setf rootbig (/ one rootsfmin))
(setf large
        (/ big
           (f2cl-lib:dsqrt (f2cl-lib:dble (f2cl-lib:int-mul m n)))))
(setf bigtheta (/ one rooteps))
(setf tol (* ctol epsln))
(setf roottol (f2cl-lib:dsqrt tol))
(cond
 ((>= (* (f2cl-lib:dble m) epsln) one) (setf info -4)
  (xerbla "DGESVJ" (f2cl-lib:int-sub info)) (go end_label)))
(cond
 (rsvec (setf mvl n)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" mvl n zero one v ldv)
    (declare (ignore var-0 var-5))
    (when var-1 (setf mvl var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldv var-6))))
 (applv (setf mvl mv)))
(setf rsvec (or rsvec applv))
(setf skl
        (/ one
           (f2cl-lib:dsqrt (* (f2cl-lib:dble m) (f2cl-lib:dble n)))))
(setf noscale f2cl-lib:%true%)
(setf goscale f2cl-lib:%true%)
(cond
 (lower
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
```

```
    (tagbody
      (setf aapp zero)
      (setf aaqq one)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dlassq (f2cl-lib:int-add (f2cl-lib:int-sub m p) 1)
           (f2cl-lib:array-slice a-%data% double-float (p p)
                                 ((1 lda) (1 *)) a-%offset%)
           1 aapp aaqq)
        (declare (ignore var-0 var-1 var-2))
        (when var-3 (setf aapp var-3))
        (when var-4 (setf aaqq var-4)))
      (cond
       ((> aapp big) (setf info -6)
        (xerbla "DGESVJ" (f2cl-lib:int-sub info)) (go end_label)))
      (setf aaqq (f2cl-lib:dsqrt aaqq))
      (cond
       ((and (< aapp (f2cl-lib:f2cl/ big aaqq)) noscale)
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                (* aapp aaqq)))
       (t (setf noscale f2cl-lib:%false%)
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                (* aapp (* aaqq skl)))
        (cond
         (goscale (setf goscale f2cl-lib:%false%)
          (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                        ((> q
                            (f2cl-lib:int-add p (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%)
                      (*
                       (f2cl-lib:fref sva-%data% (q) ((1 n))
                                      sva-%offset%)
                       skl))
             label1873))))))
    label1874)))
(upper
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p n) nil)
   (tagbody
     (setf aapp zero)
     (setf aaqq one)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (dlassq p
          (f2cl-lib:array-slice a-%data% double-float (1 p)
                                ((1 lda) (1 *)) a-%offset%)
          1 aapp aaqq)
       (declare (ignore var-1 var-2))
       (when var-0 (setf p var-0))
       (when var-3 (setf aapp var-3))
```

```
          (when var-4 (setf aaqq var-4)))
        (cond
         ((> aapp big) (setf info -6)
          (xerbla "DGESVJ" (f2cl-lib:int-sub info)) (go end_label)))
        (setf aaqq (f2cl-lib:dsqrt aaqq))
        (cond
         ((and (< aapp (f2cl-lib:f2cl/ big aaqq)) noscale)
          (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                  (* aapp aaqq)))
         (t (setf noscale f2cl-lib:%false%)
          (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                  (* aapp (* aaqq skl)))
          (cond
           (goscale (setf goscale f2cl-lib:%false%)
            (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                          ((> q
                              (f2cl-lib:int-add p (f2cl-lib:int-sub 1)))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%)
                        (*
                         (f2cl-lib:fref sva-%data% (q) ((1 n))
                                        sva-%offset%)
                         skl))
               label2873))))))
    label2874)))
(t
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p n) nil)
   (tagbody
     (setf aapp zero)
     (setf aaqq one)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (dlassq m
          (f2cl-lib:array-slice a-%data% double-float (1 p)
                                ((1 lda) (1 *)) a-%offset%)
          1 aapp aaqq)
       (declare (ignore var-1 var-2))
       (when var-0 (setf m var-0))
       (when var-3 (setf aapp var-3))
       (when var-4 (setf aaqq var-4)))
     (cond
      ((> aapp big) (setf info -6)
       (xerbla "DGESVJ" (f2cl-lib:int-sub info)) (go end_label)))
     (setf aaqq (f2cl-lib:dsqrt aaqq))
     (cond
      ((and (< aapp (f2cl-lib:f2cl/ big aaqq)) noscale)
       (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
               (* aapp aaqq)))
      (t (setf noscale f2cl-lib:%false%)
```

```
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                (* aapp (* aaqq skl)))
        (cond
         (goscale (setf goscale f2cl-lib:%false%)
          (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                        ((> q
                            (f2cl-lib:int-add p (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%)
                      (*
                       (f2cl-lib:fref sva-%data% (q) ((1 n))
                                      sva-%offset%)
                       skl))
             label3873))))))
     label3874))))
(if noscale
    (setf skl one))
(setf aapp zero)
(setf aaqq big)
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (if (/= (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%) zero)
        (setf aaqq
                (min aaqq
                     (f2cl-lib:fref sva-%data% (p) ((1 n))
                                    sva-%offset%))))
    (setf aapp
            (max aapp
                 (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)))
   label4781))
(cond
 ((= aapp zero)
  (if lsvec
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlaset "G" m n zero one a lda)
        (declare (ignore var-0 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf zero var-3))
        (when var-4 (setf one var-4))
        (when var-6 (setf lda var-6))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 lwork)) work-%offset%) one)
  (setf (f2cl-lib:fref work-%data% (2) ((1 lwork)) work-%offset%) zero)
  (setf (f2cl-lib:fref work-%data% (3) ((1 lwork)) work-%offset%) zero)
  (setf (f2cl-lib:fref work-%data% (4) ((1 lwork)) work-%offset%) zero)
  (setf (f2cl-lib:fref work-%data% (5) ((1 lwork)) work-%offset%) zero)
  (setf (f2cl-lib:fref work-%data% (6) ((1 lwork)) work-%offset%) zero)
  (go end_label)))
```

```
(cond
 ((= n 1)
  (if lsvec
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (dlascl "G" 0 0
           (f2cl-lib:array-slice sva-%data% double-float (1) ((1 n))
                                 sva-%offset%)
           skl m 1
           (f2cl-lib:array-slice a-%data% double-float (1 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda ierr)
        (declare (ignore var-0 var-1 var-2 var-3 var-6 var-7))
        (when var-4 (setf skl var-4))
        (when var-5 (setf m var-5))
        (when var-8 (setf lda var-8))
        (when var-9 (setf ierr var-9))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 lwork)) work-%offset%)
          (/ one skl))
  (cond
   ((>= (f2cl-lib:fref sva (1) ((1 n))) sfmin)
    (setf (f2cl-lib:fref work-%data% (2) ((1 lwork)) work-%offset%)
            one))
   (t
    (setf (f2cl-lib:fref work-%data% (2) ((1 lwork)) work-%offset%)
            zero)))
  (setf (f2cl-lib:fref work-%data% (3) ((1 lwork)) work-%offset%) zero)
  (setf (f2cl-lib:fref work-%data% (4) ((1 lwork)) work-%offset%) zero)
  (setf (f2cl-lib:fref work-%data% (5) ((1 lwork)) work-%offset%) zero)
  (setf (f2cl-lib:fref work-%data% (6) ((1 lwork)) work-%offset%) zero)
  (go end_label)))
(setf sn (f2cl-lib:dsqrt (/ sfmin epsln)))
(setf temp1 (f2cl-lib:dsqrt (/ big (f2cl-lib:dble n))))
(cond
 ((or (<= aapp sn) (>= aaqq temp1) (and (<= sn aaqq) (<= aapp temp1)))
  (setf temp1 (min big (/ temp1 aapp))))
 ((and (<= aaqq sn) (<= aapp temp1))
  (setf temp1
          (min (/ sn aaqq)
               (/ big (* aapp (f2cl-lib:dsqrt (f2cl-lib:dble n)))))))
 ((and (>= aaqq sn) (>= aapp temp1))
  (setf temp1 (max (/ sn aaqq) (/ temp1 aapp))))
 ((and (<= aaqq sn) (>= aapp temp1))
  (setf temp1
          (min (/ sn aaqq)
               (/ big (* (f2cl-lib:dsqrt (f2cl-lib:dble n)) aapp)))))
 (t (setf temp1 one)))
(cond
 ((/= temp1 one)
  (multiple-value-bind
```

```
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 one temp1 n 1 sva n ierr)
    (declare (ignore var-0 var-1 var-2 var-6 var-7))
    (when var-3 (setf one var-3))
    (when var-4 (setf temp1 var-4))
    (when var-5 (setf n var-5))
    (when var-8 (setf n var-8))
    (when var-9 (setf ierr var-9)))))
(setf skl (* temp1 skl))
(cond
 ((/= skl one)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl joba 0 0 one skl m n a lda ierr)
    (declare (ignore var-1 var-2 var-7))
    (when var-0 (setf joba var-0))
    (when var-3 (setf one var-3))
    (when var-4 (setf skl var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf ierr var-9)))
  (setf skl (/ one skl))))
(setf emptsw (the f2cl-lib:integer4 (truncate (* n (- n 1)) 2)))
(setf notrot 0)
(setf (f2cl-lib:fref fastr (1) ((1 5))) zero)
(f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
              ((> q n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (q) ((1 lwork)) work-%offset%)
            one)
   label1868))
(setf swband 3)
(setf kbl (min (the f2cl-lib:integer4 8) (the f2cl-lib:integer4 n)))
(setf nbl (the f2cl-lib:integer4 (truncate n kbl)))
(if (/= (f2cl-lib:int-mul nbl kbl) n)
    (setf nbl (f2cl-lib:int-add nbl 1)))
(setf blskip (expt kbl 2))
(setf rowskip
        (min (the f2cl-lib:integer4 5) (the f2cl-lib:integer4 kbl)))
(setf lkahead 1)
(cond
 ((and (or lower upper)
       (> n
          (max (the f2cl-lib:integer4 64)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul 4 kbl)))))
  (setf n4 (the f2cl-lib:integer4 (truncate n 4)))
  (setf n2 (the f2cl-lib:integer4 (truncate n 2)))
  (setf n34 (f2cl-lib:int-mul 3 n4))
  (cond (applv (setf q 0)) (t (setf q 1)))
```

```
(cond
 (lower
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16)
      (dgsvj0 jobv (f2cl-lib:int-sub m n34) (f2cl-lib:int-sub n n34)
       (f2cl-lib:array-slice a-%data% double-float
                             ((+ n34 1) (f2cl-lib:int-add n34 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice work-%data% double-float ((+ n34 1))
                             ((1 lwork)) work-%offset%)
       (f2cl-lib:array-slice sva-%data% double-float ((+ n34 1))
                             ((1 n)) sva-%offset%)
       mvl
       (f2cl-lib:array-slice v-%data% double-float
                             ((+ (f2cl-lib:int-mul n34 q) 1)
                              (f2cl-lib:int-add n34 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv epsln sfmin tol 2
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 lwork)) work-%offset%)
       (f2cl-lib:int-sub lwork n) ierr)
    (declare
     (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-13 var-14
      var-15))
    (when var-0 (setf jobv var-0))
    (when var-4 (setf lda var-4))
    (when var-7 (setf mvl var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf epsln var-10))
    (when var-11 (setf sfmin var-11))
    (when var-12 (setf tol var-12))
    (when var-16 (setf ierr var-16)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16)
      (dgsvj0 jobv (f2cl-lib:int-sub m n2) (f2cl-lib:int-sub n34 n2)
       (f2cl-lib:array-slice a-%data% double-float
                             ((+ n2 1) (f2cl-lib:int-add n2 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice work-%data% double-float ((+ n2 1))
                             ((1 lwork)) work-%offset%)
       (f2cl-lib:array-slice sva-%data% double-float ((+ n2 1))
                             ((1 n)) sva-%offset%)
       mvl
       (f2cl-lib:array-slice v-%data% double-float
                             ((+ (f2cl-lib:int-mul n2 q) 1)
                              (f2cl-lib:int-add n2 1))
```

```
                          ((1 ldv) (1 *)) v-%offset%)
      ldv epsln sfmin tol 2
      (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                          ((1 lwork)) work-%offset%)
      (f2cl-lib:int-sub lwork n) ierr)
  (declare
   (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-13 var-14
    var-15))
  (when var-0 (setf jobv var-0))
  (when var-4 (setf lda var-4))
  (when var-7 (setf mvl var-7))
  (when var-9 (setf ldv var-9))
  (when var-10 (setf epsln var-10))
  (when var-11 (setf sfmin var-11))
  (when var-12 (setf tol var-12))
  (when var-16 (setf ierr var-16)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17)
    (dgsvj1 jobv (f2cl-lib:int-sub m n2) (f2cl-lib:int-sub n n2) n4
     (f2cl-lib:array-slice a-%data% double-float
                          ((+ n2 1) (f2cl-lib:int-add n2 1))
                          ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% double-float ((+ n2 1))
                          ((1 lwork)) work-%offset%)
     (f2cl-lib:array-slice sva-%data% double-float ((+ n2 1))
                          ((1 n)) sva-%offset%)
     mvl
     (f2cl-lib:array-slice v-%data% double-float
                          ((+ (f2cl-lib:int-mul n2 q) 1)
                           (f2cl-lib:int-add n2 1))
                          ((1 ldv) (1 *)) v-%offset%)
     ldv epsln sfmin tol 1
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                          ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare
   (ignore var-1 var-2 var-4 var-6 var-7 var-9 var-14 var-15
    var-16))
  (when var-0 (setf jobv var-0))
  (when var-3 (setf n4 var-3))
  (when var-5 (setf lda var-5))
  (when var-8 (setf mvl var-8))
  (when var-10 (setf ldv var-10))
  (when var-11 (setf epsln var-11))
  (when var-12 (setf sfmin var-12))
  (when var-13 (setf tol var-13))
  (when var-17 (setf ierr var-17)))
(multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (dgsvj0 jobv (f2cl-lib:int-sub m n4) (f2cl-lib:int-sub n2 n4)
     (f2cl-lib:array-slice a-%data% double-float
                           ((+ n4 1) (f2cl-lib:int-add n4 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% double-float ((+ n4 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:array-slice sva-%data% double-float ((+ n4 1))
                           ((1 n)) sva-%offset%)
     mvl
     (f2cl-lib:array-slice v-%data% double-float
                           ((+ (f2cl-lib:int-mul n4 q) 1)
                            (f2cl-lib:int-add n4 1))
                           ((1 ldv) (1 *)) v-%offset%)
     ldv epsln sfmin tol 1
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare
   (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-13 var-14
    var-15))
  (when var-0 (setf jobv var-0))
  (when var-4 (setf lda var-4))
  (when var-7 (setf mvl var-7))
  (when var-9 (setf ldv var-9))
  (when var-10 (setf epsln var-10))
  (when var-11 (setf sfmin var-11))
  (when var-12 (setf tol var-12))
  (when var-16 (setf ierr var-16)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (dgsvj0 jobv m n4 a lda work sva mvl v ldv epsln sfmin tol 1
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-3 var-5 var-6 var-8 var-13 var-14 var-15))
  (when var-0 (setf jobv var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n4 var-2))
  (when var-4 (setf lda var-4))
  (when var-7 (setf mvl var-7))
  (when var-9 (setf ldv var-9))
  (when var-10 (setf epsln var-10))
  (when var-11 (setf sfmin var-11))
  (when var-12 (setf tol var-12))
  (when var-16 (setf ierr var-16)))
(multiple-value-bind
```

```
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17)
      (dgsvj1 jobv m n2 n4 a lda work sva mvl v ldv epsln sfmin tol 1
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 lwork)) work-%offset%)
       (f2cl-lib:int-sub lwork n) ierr)
    (declare (ignore var-4 var-6 var-7 var-9 var-14 var-15 var-16))
    (when var-0 (setf jobv var-0))
    (when var-1 (setf m var-1))
    (when var-2 (setf n2 var-2))
    (when var-3 (setf n4 var-3))
    (when var-5 (setf lda var-5))
    (when var-8 (setf mvl var-8))
    (when var-10 (setf ldv var-10))
    (when var-11 (setf epsln var-11))
    (when var-12 (setf sfmin var-12))
    (when var-13 (setf tol var-13))
    (when var-17 (setf ierr var-17))))
  (upper
   (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16)
      (dgsvj0 jobv n4 n4 a lda work sva mvl v ldv epsln sfmin tol 2
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 lwork)) work-%offset%)
       (f2cl-lib:int-sub lwork n) ierr)
    (declare (ignore var-3 var-5 var-6 var-8 var-13 var-14 var-15))
    (when var-0 (setf jobv var-0))
    (when var-1 (setf n4 var-1))
    (when var-2 (setf n4 var-2))
    (when var-4 (setf lda var-4))
    (when var-7 (setf mvl var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf epsln var-10))
    (when var-11 (setf sfmin var-11))
    (when var-12 (setf tol var-12))
    (when var-16 (setf ierr var-16)))
   (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16)
      (dgsvj0 jobv n2 n4
       (f2cl-lib:array-slice a-%data% double-float
                             (1 (f2cl-lib:int-add n4 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice work-%data% double-float ((+ n4 1))
                             ((1 lwork)) work-%offset%)
       (f2cl-lib:array-slice sva-%data% double-float ((+ n4 1))
                             ((1 n)) sva-%offset%)
       mvl
```

```
      (f2cl-lib:array-slice v-%data% double-float
                            ((+ (f2cl-lib:int-mul n4 q) 1)
                             (f2cl-lib:int-add n4 1))
                            ((1 ldv) (1 *)) v-%offset%)
      ldv epsln sfmin tol 1
      (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                            ((1 lwork)) work-%offset%)
      (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-3 var-5 var-6 var-8 var-13 var-14 var-15))
  (when var-0 (setf jobv var-0))
  (when var-1 (setf n2 var-1))
  (when var-2 (setf n4 var-2))
  (when var-4 (setf lda var-4))
  (when var-7 (setf mvl var-7))
  (when var-9 (setf ldv var-9))
  (when var-10 (setf epsln var-10))
  (when var-11 (setf sfmin var-11))
  (when var-12 (setf tol var-12))
  (when var-16 (setf ierr var-16)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17)
    (dgsvj1 jobv n2 n2 n4 a lda work sva mvl v ldv epsln sfmin tol
     1
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-4 var-6 var-7 var-9 var-14 var-15 var-16))
  (when var-0 (setf jobv var-0))
  (when var-1 (setf n2 var-1))
  (when var-2 (setf n2 var-2))
  (when var-3 (setf n4 var-3))
  (when var-5 (setf lda var-5))
  (when var-8 (setf mvl var-8))
  (when var-10 (setf ldv var-10))
  (when var-11 (setf epsln var-11))
  (when var-12 (setf sfmin var-12))
  (when var-13 (setf tol var-13))
  (when var-17 (setf ierr var-17)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (dgsvj0 jobv (f2cl-lib:int-add n2 n4) n4
     (f2cl-lib:array-slice a-%data% double-float
                           (1 (f2cl-lib:int-add n2 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% double-float ((+ n2 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:array-slice sva-%data% double-float ((+ n2 1))
```

```
                                ((1 n)) sva-%offset%)
            mvl
            (f2cl-lib:array-slice v-%data% double-float
                                ((+ (f2cl-lib:int-mul n2 q) 1)
                                 (f2cl-lib:int-add n2 1))
                                ((1 ldv) (1 *)) v-%offset%)
            ldv epsln sfmin tol 1
            (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                                ((1 lwork)) work-%offset%)
            (f2cl-lib:int-sub lwork n) ierr)
        (declare
         (ignore var-1 var-3 var-5 var-6 var-8 var-13 var-14 var-15))
        (when var-0 (setf jobv var-0))
        (when var-2 (setf n4 var-2))
        (when var-4 (setf lda var-4))
        (when var-7 (setf mvl var-7))
        (when var-9 (setf ldv var-9))
        (when var-10 (setf epsln var-10))
        (when var-11 (setf sfmin var-11))
        (when var-12 (setf tol var-12))
        (when var-16 (setf ierr var-16)))))))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i nsweep) nil)
    (tagbody
      (setf mxaapq zero)
      (setf mxsinj zero)
      (setf iswrot 0)
      (setf notrot 0)
      (setf pskipped 0)
      (f2cl-lib:fdo (ibr 1 (f2cl-lib:int-add ibr 1))
                    ((> ibr nbl) nil)
        (tagbody
          (setf igl
                  (f2cl-lib:int-add
                   (f2cl-lib:int-mul (f2cl-lib:int-sub ibr 1) kbl) 1))
          (f2cl-lib:fdo (ir1 0 (f2cl-lib:int-add ir1 1))
                        ((> ir1
                            (min (the f2cl-lib:integer4 lkahead)
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add nbl
                                                        (f2cl-lib:int-sub
                                                         ibr)))))
                         nil)
            (tagbody
              (setf igl
                      (f2cl-lib:int-add igl (f2cl-lib:int-mul ir1 kbl)))
              (f2cl-lib:fdo (p igl (f2cl-lib:int-add p 1))
                            ((> p
                                (min
                                 (the f2cl-lib:integer4
```

```
                        (f2cl-lib:int-add igl kbl
                                          (f2cl-lib:int-sub
                                           1)))
                    (the f2cl-lib:integer4
                        (f2cl-lib:int-add n
                                          (f2cl-lib:int-sub
                                           1)))))
              nil)
(tagbody
  (setf q
          (f2cl-lib:int-sub
           (f2cl-lib:int-add
            (idamax
             (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
             (f2cl-lib:array-slice sva-%data%
                                   double-float (p)
                                   ((1 n)) sva-%offset%)
             1)
            p)
           1))
  (cond
   ((/= p q)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dswap m
         (f2cl-lib:array-slice a-%data% double-float
                               (1 p) ((1 lda) (1 *))
                               a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% double-float
                               (1 q) ((1 lda) (1 *))
                               a-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf m var-0)))
    (if rsvec
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4)
            (dswap mvl
             (f2cl-lib:array-slice v-%data% double-float
                                   (1 p) ((1 ldv) (1 *))
                                   v-%offset%)
             1
             (f2cl-lib:array-slice v-%data% double-float
                                   (1 q) ((1 ldv) (1 *))
                                   v-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf mvl var-0))))
    (setf temp1
            (f2cl-lib:fref sva-%data% (p) ((1 n))
```

```
                          sva-%offset%))
  (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                       sva-%offset%)
        (f2cl-lib:fref sva-%data% (q) ((1 n))
                          sva-%offset%))
  (setf (f2cl-lib:fref sva-%data% (q) ((1 n))
                       sva-%offset%)
        temp1)
  (setf temp1
        (f2cl-lib:fref work-%data% (p) ((1 lwork))
                          work-%offset%))
  (setf (f2cl-lib:fref work-%data% (p) ((1 lwork))
                       work-%offset%)
        (f2cl-lib:fref work-%data% (q) ((1 lwork))
                          work-%offset%))
  (setf (f2cl-lib:fref work-%data% (q) ((1 lwork))
                       work-%offset%)
        temp1)))
(cond
 ((= ir1 0)
  (cond
   ((and (< (f2cl-lib:fref sva (p) ((1 n))) rootbig)
         (> (f2cl-lib:fref sva (p) ((1 n))) rootsfmin))
    (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                          sva-%offset%)
          (*
           (multiple-value-bind
               (ret-val var-0 var-1 var-2)
               (dnrm2 m
                (f2cl-lib:array-slice a-%data%
                                      double-float
                                      (1 p)
                                      ((1 lda) (1 *))
                                      a-%offset%)
                1)
             (declare (ignore var-1 var-2))
             (when var-0 (setf m var-0))
             ret-val)
           (f2cl-lib:fref work-%data% (p) ((1 lwork))
                          work-%offset%))))
   (t (setf temp1 zero) (setf aapp one)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4)
        (dlassq m
         (f2cl-lib:array-slice a-%data% double-float
                               (1 p) ((1 lda) (1 *))
                               a-%offset%)
         1 temp1 aapp)
      (declare (ignore var-1 var-2))
      (when var-0 (setf m var-0))
```

```
          (when var-3 (setf temp1 var-3))
          (when var-4 (setf aapp var-4)))
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                             sva-%offset%)
                (* temp1 (f2cl-lib:dsqrt aapp)
                   (f2cl-lib:fref work-%data% (p)
                                  ((1 lwork))
                                  work-%offset%)))))
      (setf aapp
              (f2cl-lib:fref sva-%data% (p) ((1 n))
                             sva-%offset%)))
     (t
      (setf aapp
              (f2cl-lib:fref sva-%data% (p) ((1 n))
                             sva-%offset%))))
    (cond
     ((> aapp zero)
      (tagbody
        (setf pskipped 0)
        (f2cl-lib:fdo (q (f2cl-lib:int-add p 1)
                         (f2cl-lib:int-add q 1))
                        ((> q
                            (min
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add igl kbl
                                                    (f2cl-lib:int-sub
                                                     1)))
                             (the f2cl-lib:integer4 n)))
                         nil)
          (tagbody
            (setf aaqq
                    (f2cl-lib:fref sva-%data% (q) ((1 n))
                                   sva-%offset%))
            (cond
             ((> aaqq zero) (setf aapp0 aapp)
              (cond
               ((>= aaqq one)
                (setf rotok (<= (* small aapp) aaqq))
                (cond
                 ((< aapp (f2cl-lib:f2cl/ big aaqq))
                  (setf aapq
                          (/
                           (/
                            (*
                             (multiple-value-bind
                                 (ret-val var-0 var-1 var-2
                                  var-3 var-4)
                                 (ddot m
                                  (f2cl-lib:array-slice
                                   a-%data% double-float
```

```
                    (1 p) ((1 lda) (1 *))
                    a-%offset%)
                   1
                   (f2cl-lib:array-slice
                    a-%data% double-float
                    (1 q) ((1 lda) (1 *))
                    a-%offset%)
                   1)
                (declare
                 (ignore var-1 var-2 var-3
                  var-4))
                (when var-0 (setf m var-0))
                ret-val)
              (f2cl-lib:fref work-%data% (p)
                             ((1 lwork))
                             work-%offset%)
              (f2cl-lib:fref work-%data% (q)
                             ((1 lwork))
                             work-%offset%))
           aaqq)
          aapp)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (dcopy m
      (f2cl-lib:array-slice a-%data%
                            double-float
                            (1 p)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice work-%data%
                            double-float
                            ((+ n 1))
                            ((1 lwork))
                            work-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5
      var-6 var-7 var-8 var-9)
     (dlascl "G" 0 0 aapp
      (f2cl-lib:array-slice work-%data%
                            double-float
                            (p)
                            ((1 lwork))
                            work-%offset%)
```

```
            m 1
            (f2cl-lib:array-slice work-%data%
                                  double-float
                                  ((+ n 1))
                                  ((1 lwork))
                                  work-%offset%)
            lda ierr)
          (declare
           (ignore var-0 var-1 var-2 var-4 var-6
            var-7))
          (when var-3 (setf aapp var-3))
          (when var-5 (setf m var-5))
          (when var-8 (setf lda var-8))
          (when var-9 (setf ierr var-9)))
        (setf aapq
                (/
                 (*
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2
                       var-3 var-4)
                      (ddot m
                       (f2cl-lib:array-slice
                        work-%data% double-float
                        ((+ n 1)) ((1 lwork))
                        work-%offset%)
                       1
                       (f2cl-lib:array-slice
                        a-%data% double-float
                        (1 q) ((1 lda) (1 *))
                        a-%offset%)
                       1)
                    (declare
                     (ignore var-1 var-2 var-3
                      var-4))
                    (when var-0 (setf m var-0))
                    ret-val)
                  (f2cl-lib:fref work-%data% (q)
                                 ((1 lwork))
                                 work-%offset%))
                 aaqq)))))
    (t (setf rotok (<= aapp (/ aaqq small)))
     (cond
      ((> aapp (f2cl-lib:f2cl/ small aaqq))
       (setf aapq
               (/
                (/
                 (*
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2
                       var-3 var-4)
```

```
                    (ddot m
                     (f2cl-lib:array-slice
                      a-%data% double-float
                      (1 p) ((1 lda) (1 *))
                      a-%offset%)
                     1
                     (f2cl-lib:array-slice
                      a-%data% double-float
                      (1 q) ((1 lda) (1 *))
                      a-%offset%)
                     1)
                   (declare
                    (ignore var-1 var-2 var-3
                     var-4))
                   (when var-0 (setf m var-0))
                   ret-val)
                 (f2cl-lib:fref work-%data% (p)
                                ((1 lwork))
                                work-%offset%)
                 (f2cl-lib:fref work-%data% (q)
                                ((1 lwork))
                                work-%offset%))
               aaqq)
              aapp)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (dcopy m
      (f2cl-lib:array-slice a-%data%
                            double-float
                            (1 q)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice work-%data%
                            double-float
                            ((+ n 1))
                            ((1 lwork))
                            work-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5
      var-6 var-7 var-8 var-9)
     (dlascl "G" 0 0 aaqq
      (f2cl-lib:array-slice work-%data%
                            double-float
```

```
                                    (q)
                                    ((1 lwork))
                                    work-%offset%)
                m 1
                (f2cl-lib:array-slice work-%data%
                                    double-float
                                    ((+ n 1))
                                    ((1 lwork))
                                    work-%offset%)
                lda ierr)
              (declare
               (ignore var-0 var-1 var-2 var-4 var-6
                var-7))
              (when var-3 (setf aaqq var-3))
              (when var-5 (setf m var-5))
              (when var-8 (setf lda var-8))
              (when var-9 (setf ierr var-9)))
            (setf aapq
                    (/
                     (*
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2
                           var-3 var-4)
                          (ddot m
                           (f2cl-lib:array-slice
                            work-%data% double-float
                            ((+ n 1)) ((1 lwork))
                            work-%offset%)
                           1
                           (f2cl-lib:array-slice
                            a-%data% double-float
                            (1 p) ((1 lda) (1 *))
                            a-%offset%)
                           1)
                        (declare
                         (ignore var-1 var-2 var-3
                          var-4))
                        (when var-0 (setf m var-0))
                        ret-val)
                      (f2cl-lib:fref work-%data% (p)
                                     ((1 lwork))
                                     work-%offset%))
                     aapp))))))
        (setf mxaapq
                (max mxaapq (f2cl-lib:dabs aapq)))
        (cond
         ((> (f2cl-lib:dabs aapq) tol)
          (cond
           ((= ir1 0) (setf notrot 0)
            (setf pskipped 0)
```

```
  (setf iswrot
          (f2cl-lib:int-add iswrot 1))))
(cond
 (rotok (setf aqoap (/ aaqq aapp))
  (setf apoaq (/ aapp aaqq))
  (setf theta
          (/
           (* (- half)
              (f2cl-lib:dabs
               (- aqoap apoaq)))
           aapq))
  (cond
   ((> (f2cl-lib:dabs theta) bigtheta)
    (setf t$ (/ half theta))
    (setf (f2cl-lib:fref fastr (3) ((1 5)))
            (/
             (* t$
                (f2cl-lib:fref work-%data%
                               (p)
                               ((1 lwork))
                               work-%offset%))
             (f2cl-lib:fref work-%data% (q)
                            ((1 lwork))
                            work-%offset%)))
    (setf (f2cl-lib:fref fastr (4) ((1 5)))
            (/
             (* (- t$)
                (f2cl-lib:fref work-%data%
                               (q)
                               ((1 lwork))
                               work-%offset%))
             (f2cl-lib:fref work-%data% (p)
                            ((1 lwork))
                            work-%offset%)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4
         var-5)
        (drotm m
         (f2cl-lib:array-slice a-%data%
                               double-float
                               (1 p)
                               ((1 lda)
                                (1 *))
                               a-%offset%)
         1
         (f2cl-lib:array-slice a-%data%
                               double-float
                               (1 q)
                               ((1 lda)
                                (1 *))
```

```
                                a-%offset%)
      1 fastr)
   (declare
    (ignore var-1 var-2 var-3 var-4
     var-5))
   (when var-0 (setf m var-0)))
 (if rsvec
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4
          var-5)
         (drotm mvl
          (f2cl-lib:array-slice v-%data%
                                double-float
                                (1 p)
                                ((1 ldv)
                                 (1 *))
                                v-%offset%)
          1
          (f2cl-lib:array-slice v-%data%
                                double-float
                                (1 q)
                                ((1 ldv)
                                 (1 *))
                                v-%offset%)
          1 fastr)
       (declare
        (ignore var-1 var-2 var-3 var-4
         var-5))
       (when var-0 (setf mvl var-0))))
 (setf (f2cl-lib:fref sva-%data% (q)
                      ((1 n))
                      sva-%offset%)
         (* aaqq
            (f2cl-lib:dsqrt
             (max zero
                  (+ one
                     (* t$ apoaq
                        aapq))))))
 (setf aapp
         (* aapp
            (f2cl-lib:dsqrt
             (max zero
                  (+ one
                     (* (- t$) aqoap
                        aapq))))))
 (setf mxsinj
         (max mxsinj
              (f2cl-lib:dabs t$))))
(t
 (setf thsign
```

```
        (coerce
         (- (f2cl-lib:dsign one aapq))
         'double-float))
(setf t$
        (/ one
           (+ theta
              (* thsign
                 (f2cl-lib:dsqrt
                  (+ one
                     (* theta
                        theta)))))))
(setf cs
        (f2cl-lib:dsqrt
         (/ one (+ one (* t$ t$)))))
(setf sn (* t$ cs))
(setf mxsinj
        (max mxsinj
             (f2cl-lib:dabs sn)))
(setf (f2cl-lib:fref sva-%data% (q)
                     ((1 n))
                     sva-%offset%)
        (* aaqq
           (f2cl-lib:dsqrt
            (max zero
                 (+ one
                    (* t$ apoaq
                       aapq))))))
(setf aapp
        (* aapp
           (f2cl-lib:dsqrt
            (max zero
                 (+ one
                    (* (- t$) aqoap
                       aapq))))))
(setf apoaq
        (/
         (f2cl-lib:fref work-%data% (p)
                        ((1 lwork))
                        work-%offset%)
         (f2cl-lib:fref work-%data% (q)
                        ((1 lwork))
                        work-%offset%)))
(setf aqoap
        (/
         (f2cl-lib:fref work-%data% (q)
                        ((1 lwork))
                        work-%offset%)
         (f2cl-lib:fref work-%data% (p)
                        ((1 lwork))
                        work-%offset%)))
```

```
(cond
 ((>=
   (f2cl-lib:fref work (p) ((1 lwork)))
   one)
  (cond
   ((>=
     (f2cl-lib:fref work (q)
                    ((1 lwork)))
     one)
    (setf (f2cl-lib:fref fastr (3)
                         ((1 5)))
          (* t$ apoaq))
    (setf (f2cl-lib:fref fastr (4)
                         ((1 5)))
          (* (- t$) aqoap))
    (setf (f2cl-lib:fref work-%data%
                         (p)
                         ((1 lwork))
                         work-%offset%)
          (*
           (f2cl-lib:fref work-%data%
                          (p)
                          ((1 lwork))
                          work-%offset%)
           cs))
    (setf (f2cl-lib:fref work-%data%
                         (q)
                         ((1 lwork))
                         work-%offset%)
          (*
           (f2cl-lib:fref work-%data%
                          (q)
                          ((1 lwork))
                          work-%offset%)
           cs))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4
         var-5)
        (drotm m
         (f2cl-lib:array-slice a-%data%
                               double-float
                               (1 p)
                               ((1 lda)
                                (1 *))
                               a-%offset%)
         1
         (f2cl-lib:array-slice a-%data%
                               double-float
                               (1 q)
                               ((1 lda)
```

```
                                (1 *))
                               a-%offset%)
      1 fastr)
    (declare
     (ignore var-1 var-2 var-3 var-4
      var-5))
    (when var-0 (setf m var-0)))
  (if rsvec
      (multiple-value-bind
          (var-0 var-1 var-2 var-3
           var-4 var-5)
          (drotm mvl
           (f2cl-lib:array-slice
            v-%data% double-float
            (1 p) ((1 ldv) (1 *))
            v-%offset%)
           1
           (f2cl-lib:array-slice
            v-%data% double-float
            (1 q) ((1 ldv) (1 *))
            v-%offset%)
           1 fastr)
        (declare
         (ignore var-1 var-2 var-3
          var-4 var-5))
        (when var-0
          (setf mvl var-0)))))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4
      var-5)
     (daxpy m (* (- t$) aqoap)
      (f2cl-lib:array-slice a-%data%
                            double-float
                            (1 q)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice a-%data%
                            double-float
                            (1 p)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1)
    (declare
     (ignore var-1 var-2 var-3 var-4
      var-5))
    (when var-0 (setf m var-0)))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5)
    (daxpy m (* cs sn apoaq)
     (f2cl-lib:array-slice a-%data%
                           double-float
                           (1 p)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1
     (f2cl-lib:array-slice a-%data%
                           double-float
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3 var-4
    var-5))
  (when var-0 (setf m var-0)))
(setf (f2cl-lib:fref work-%data%
                     (p)
                     ((1 lwork))
                     work-%offset%)
        (*
         (f2cl-lib:fref work-%data%
                        (p)
                        ((1 lwork))
                        work-%offset%)
         cs))
(setf (f2cl-lib:fref work-%data%
                     (q)
                     ((1 lwork))
                     work-%offset%)
        (/
         (f2cl-lib:fref work-%data%
                        (q)
                        ((1 lwork))
                        work-%offset%)
         cs))
(cond
 (rsvec
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
       var-4 var-5)
      (daxpy mvl (* (- t$) aqoap)
       (f2cl-lib:array-slice
        v-%data% double-float (1 q)
```

```
                ((1 ldv) (1 *)) v-%offset%)
               1
               (f2cl-lib:array-slice
                v-%data% double-float (1 p)
                ((1 ldv) (1 *)) v-%offset%)
               1)
            (declare
             (ignore var-1 var-2 var-3
              var-4 var-5))
            (when var-0 (setf mvl var-0)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3
               var-4 var-5)
              (daxpy mvl (* cs sn apoaq)
               (f2cl-lib:array-slice
                v-%data% double-float (1 p)
                ((1 ldv) (1 *)) v-%offset%)
               1
               (f2cl-lib:array-slice
                v-%data% double-float (1 q)
                ((1 ldv) (1 *)) v-%offset%)
               1)
            (declare
             (ignore var-1 var-2 var-3
              var-4 var-5))
            (when var-0
              (setf mvl var-0))))))))
(t
 (cond
  ((>=
    (f2cl-lib:fref work (q)
                   ((1 lwork)))
    one)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4
        var-5)
       (daxpy m (* t$ apoaq)
        (f2cl-lib:array-slice a-%data%
                              double-float
                              (1 p)
                              ((1 lda)
                               (1 *))
                              a-%offset%)
        1
        (f2cl-lib:array-slice a-%data%
                              double-float
                              (1 q)
                              ((1 lda)
                               (1 *))
                              a-%offset%)
```

```
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4
     var-5))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4
      var-5)
     (daxpy m (* (- cs) sn aqoap)
      (f2cl-lib:array-slice a-%data%
                            double-float
                            (1 q)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice a-%data%
                            double-float
                            (1 p)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4
     var-5))
   (when var-0 (setf m var-0)))
 (setf (f2cl-lib:fref work-%data%
                      (p)
                      ((1 lwork))
                      work-%offset%)
         (/
          (f2cl-lib:fref work-%data%
                         (p)
                         ((1 lwork))
                         work-%offset%)
          cs))
 (setf (f2cl-lib:fref work-%data%
                      (q)
                      ((1 lwork))
                      work-%offset%)
         (*
          (f2cl-lib:fref work-%data%
                         (q)
                         ((1 lwork))
                         work-%offset%)
          cs))
 (cond
  (rsvec
   (multiple-value-bind
```

```
          (var-0 var-1 var-2 var-3
           var-4 var-5)
          (daxpy mvl (* t$ apoaq)
           (f2cl-lib:array-slice
            v-%data% double-float (1 p)
            ((1 ldv) (1 *)) v-%offset%)
           1
           (f2cl-lib:array-slice
            v-%data% double-float (1 q)
            ((1 ldv) (1 *)) v-%offset%)
           1)
        (declare
         (ignore var-1 var-2 var-3
          var-4 var-5))
        (when var-0 (setf mvl var-0)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3
           var-4 var-5)
          (daxpy mvl
           (* (- cs) sn aqoap)
           (f2cl-lib:array-slice
            v-%data% double-float (1 q)
            ((1 ldv) (1 *)) v-%offset%)
           1
           (f2cl-lib:array-slice
            v-%data% double-float (1 p)
            ((1 ldv) (1 *)) v-%offset%)
           1)
        (declare
         (ignore var-1 var-2 var-3
          var-4 var-5))
        (when var-0
          (setf mvl var-0))))))
  (t
   (cond
    ((>=
      (f2cl-lib:fref work (p)
                     ((1 lwork)))
      (f2cl-lib:fref work (q)
                     ((1 lwork))))
     (multiple-value-bind
          (var-0 var-1 var-2 var-3
           var-4 var-5)
          (daxpy m (* (- t$) aqoap)
           (f2cl-lib:array-slice
            a-%data% double-float (1 q)
            ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice
            a-%data% double-float (1 p)
```

```
      ((1 lda) (1 *)) a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3
    var-4 var-5))
  (when var-0 (setf m var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3
     var-4 var-5)
    (daxpy m (* cs sn apoaq)
     (f2cl-lib:array-slice
      a-%data% double-float (1 p)
      ((1 lda) (1 *)) a-%offset%)
     1
     (f2cl-lib:array-slice
      a-%data% double-float (1 q)
      ((1 lda) (1 *)) a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3
    var-4 var-5))
  (when var-0 (setf m var-0)))
(setf (f2cl-lib:fref work-%data%
                     (p)
                     ((1 lwork))
                     work-%offset%)
        (*
         (f2cl-lib:fref
          work-%data% (p)
          ((1 lwork))
          work-%offset%)
         cs))
(setf (f2cl-lib:fref work-%data%
                     (q)
                     ((1 lwork))
                     work-%offset%)
        (/
         (f2cl-lib:fref
          work-%data% (q)
          ((1 lwork))
          work-%offset%)
         cs))
(cond
 (rsvec
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
       var-4 var-5)
      (daxpy mvl (* (- t$) aqoap)
       (f2cl-lib:array-slice
        v-%data% double-float
```

```
            (1 q) ((1 ldv) (1 *))
            v-%offset%)
           1
           (f2cl-lib:array-slice
            v-%data% double-float
            (1 p) ((1 ldv) (1 *))
            v-%offset%)
           1)
       (declare
        (ignore var-1 var-2 var-3
         var-4 var-5))
       (when var-0
         (setf mvl var-0)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3
          var-4 var-5)
         (daxpy mvl (* cs sn apoaq)
          (f2cl-lib:array-slice
           v-%data% double-float
           (1 p) ((1 ldv) (1 *))
           v-%offset%)
          1
          (f2cl-lib:array-slice
           v-%data% double-float
           (1 q) ((1 ldv) (1 *))
           v-%offset%)
          1)
       (declare
        (ignore var-1 var-2 var-3
         var-4 var-5))
       (when var-0
         (setf mvl var-0))))))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3
        var-4 var-5)
       (daxpy m (* t$ apoaq)
        (f2cl-lib:array-slice
         a-%data% double-float (1 p)
         ((1 lda) (1 *)) a-%offset%)
        1
        (f2cl-lib:array-slice
         a-%data% double-float (1 q)
         ((1 lda) (1 *)) a-%offset%)
        1)
     (declare
      (ignore var-1 var-2 var-3
       var-4 var-5))
     (when var-0 (setf m var-0)))
   (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3
     var-4 var-5)
    (daxpy m (* (- cs) sn aqoap)
     (f2cl-lib:array-slice
      a-%data% double-float (1 q)
      ((1 lda) (1 *)) a-%offset%)
     1
     (f2cl-lib:array-slice
      a-%data% double-float (1 p)
      ((1 lda) (1 *)) a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3
    var-4 var-5))
  (when var-0 (setf m var-0)))
(setf (f2cl-lib:fref work-%data%
                     (p)
                     ((1 lwork))
                     work-%offset%)
        (/
         (f2cl-lib:fref
          work-%data% (p)
          ((1 lwork))
          work-%offset%)
         cs))
(setf (f2cl-lib:fref work-%data%
                     (q)
                     ((1 lwork))
                     work-%offset%)
        (*
         (f2cl-lib:fref
          work-%data% (q)
          ((1 lwork))
          work-%offset%)
         cs))
(cond
 (rsvec
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
       var-4 var-5)
      (daxpy mvl (* t$ apoaq)
       (f2cl-lib:array-slice
        v-%data% double-float
        (1 p) ((1 ldv) (1 *))
        v-%offset%)
       1
       (f2cl-lib:array-slice
        v-%data% double-float
        (1 q) ((1 ldv) (1 *))
        v-%offset%)
```

```
                    1)
                (declare
                 (ignore var-1 var-2 var-3
                  var-4 var-5))
                (when var-0
                  (setf mvl var-0)))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3
                   var-4 var-5)
                  (daxpy mvl
                   (* (- cs) sn aqoap)
                   (f2cl-lib:array-slice
                    v-%data% double-float
                    (1 q) ((1 ldv) (1 *))
                    v-%offset%)
                   1
                   (f2cl-lib:array-slice
                    v-%data% double-float
                    (1 p) ((1 ldv) (1 *))
                    v-%offset%)
                   1)
                (declare
                 (ignore var-1 var-2 var-3
                  var-4 var-5))
                (when var-0
                  (setf mvl
                          var-0)))))))))))))))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (dcopy m
      (f2cl-lib:array-slice a-%data%
                            double-float
                            (1 p)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice work-%data%
                            double-float
                            ((+ n 1))
                            ((1 lwork))
                            work-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5
      var-6 var-7 var-8 var-9)
```

```
    (dlascl "G" 0 0 aapp one m 1
     (f2cl-lib:array-slice work-%data%
                           double-float
                           ((+ n 1))
                           ((1 lwork))
                           work-%offset%)
     lda ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6
    var-7))
  (when var-3 (setf aapp var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5
     var-6 var-7 var-8 var-9)
    (dlascl "G" 0 0 aaqq one m 1
     (f2cl-lib:array-slice a-%data%
                           double-float
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     lda ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6
    var-7))
  (when var-3 (setf aaqq var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(setf temp1
        (/
         (* (- aapq)
            (f2cl-lib:fref work-%data%
                           (p)
                           ((1 lwork))
                           work-%offset%))
         (f2cl-lib:fref work-%data% (q)
                        ((1 lwork))
                        work-%offset%)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5)
    (daxpy m temp1
     (f2cl-lib:array-slice work-%data%
                           double-float
                           ((+ n 1))
```

```
                                ((1 lwork))
                                work-%offset%)
        1
        (f2cl-lib:array-slice a-%data%
                                double-float
                                (1 q)
                                ((1 lda)
                                 (1 *))
                                a-%offset%)
        1)
      (declare
       (ignore var-2 var-3 var-4 var-5))
      (when var-0 (setf m var-0))
      (when var-1 (setf temp1 var-1)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5
         var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 one aaqq m 1
         (f2cl-lib:array-slice a-%data%
                                double-float
                                (1 q)
                                ((1 lda)
                                 (1 *))
                                a-%offset%)
        lda ierr)
      (declare
       (ignore var-0 var-1 var-2 var-6
        var-7))
      (when var-3 (setf one var-3))
      (when var-4 (setf aaqq var-4))
      (when var-5 (setf m var-5))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9)))
    (setf (f2cl-lib:fref sva-%data% (q)
                         ((1 n))
                         sva-%offset%)
            (* aaqq
               (f2cl-lib:dsqrt
                (max zero
                     (- one
                        (* aapq aapq))))))
    (setf mxsinj (max mxsinj sfmin))))
  (cond
   ((<=
     (expt
      (f2cl-lib:f2cl/
       (f2cl-lib:fref sva (q) ((1 n))) aaqq)
      2)
     rooteps)
    (cond
```

```
                    ((and (< aaqq rootbig)
                          (> aaqq rootsfmin))
                     (setf (f2cl-lib:fref sva-%data% (q)
                                          ((1 n))
                                          sva-%offset%)
                             (*
                              (multiple-value-bind
                                  (ret-val var-0 var-1
                                   var-2)
                                  (dnrm2 m
                                   (f2cl-lib:array-slice
                                    a-%data% double-float
                                    (1 q) ((1 lda) (1 *))
                                    a-%offset%)
                                   1)
                                (declare
                                 (ignore var-1 var-2))
                                (when var-0 (setf m var-0))
                                ret-val)
                              (f2cl-lib:fref work-%data% (q)
                                             ((1 lwork))
                                             work-%offset%))))
                    (t (setf t$ zero) (setf aaqq one)
                     (multiple-value-bind
                         (var-0 var-1 var-2 var-3 var-4)
                         (dlassq m
                          (f2cl-lib:array-slice a-%data%
                                                double-float
                                                (1 q)
                                                ((1 lda)
                                                 (1 *))
                                                a-%offset%)
                          1 t$ aaqq)
                       (declare (ignore var-1 var-2))
                       (when var-0 (setf m var-0))
                       (when var-3 (setf t$ var-3))
                       (when var-4 (setf aaqq var-4)))
                     (setf (f2cl-lib:fref sva-%data% (q)
                                          ((1 n))
                                          sva-%offset%)
                             (* t$ (f2cl-lib:dsqrt aaqq)
                                (f2cl-lib:fref work-%data%
                                               (q)
                                               ((1 lwork))
                                               work-%offset%)))))))
                (cond
                 (((<= (f2cl-lib:f2cl/ aapp aapp0) rooteps)
                  (cond
                   ((and (< aapp rootbig)
                         (> aapp rootsfmin))
```

```
        (setf aapp
                (*
                 (multiple-value-bind
                     (ret-val var-0 var-1
                      var-2)
                     (dnrm2 m
                      (f2cl-lib:array-slice
                       a-%data% double-float
                       (1 p) ((1 lda) (1 *))
                       a-%offset%)
                      1)
                    (declare
                     (ignore var-1 var-2))
                    (when var-0 (setf m var-0))
                    ret-val)
                 (f2cl-lib:fref work-%data% (p)
                                ((1 lwork))
                                work-%offset%))))
       (t (setf t$ zero) (setf aapp one)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4)
            (dlassq m
             (f2cl-lib:array-slice a-%data%
                                   double-float
                                   (1 p)
                                   ((1 lda)
                                    (1 *))
                                   a-%offset%)
             1 t$ aapp)
          (declare (ignore var-1 var-2))
          (when var-0 (setf m var-0))
          (when var-3 (setf t$ var-3))
          (when var-4 (setf aapp var-4)))
        (setf aapp
                (* t$ (f2cl-lib:dsqrt aapp)
                   (f2cl-lib:fref work-%data%
                                  (p)
                                  ((1 lwork))
                                  work-%offset%)))))
      (setf (f2cl-lib:fref sva-%data% (p)
                           ((1 n))
                           sva-%offset%)
            aapp))))
  (t
   (if (= ir1 0)
       (setf notrot
               (f2cl-lib:int-add notrot 1)))
   (setf pskipped
           (f2cl-lib:int-add pskipped 1)))))
(t
```

```
                    (if (= ir1 0)
                        (setf notrot
                                (f2cl-lib:int-add notrot 1)))
                    (setf pskipped
                            (f2cl-lib:int-add pskipped 1))))
                  (cond
                   ((and (<= i swband) (> pskipped rowskip))
                    (if (= ir1 0)
                        (setf aapp (- aapp)))
                    (setf notrot 0) (go label2103)))
                 label2002))
             label2103
              (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                                   sva-%offset%)
                      aapp)))
            (t
             (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                                  sva-%offset%)
                    aapp)
             (if (and (= ir1 0) (= aapp zero))
                 (setf notrot
                         (f2cl-lib:int-sub
                          (f2cl-lib:int-add notrot
                                            (min
                                             (the
                                              f2cl-lib:integer4
                                              (f2cl-lib:int-sub
                                               (f2cl-lib:int-add
                                                igl kbl)
                                               1))
                                             (the
                                              f2cl-lib:integer4
                                              n)))
                          p)))))
        label2001))
    label1002))
(setf igl
        (f2cl-lib:int-add
         (f2cl-lib:int-mul (f2cl-lib:int-sub ibr 1) kbl) 1))
(f2cl-lib:fdo (jbc (f2cl-lib:int-add ibr 1)
               (f2cl-lib:int-add jbc 1))
              ((> jbc nbl) nil)
  (tagbody
    (setf jgl
            (f2cl-lib:int-add
             (f2cl-lib:int-mul (f2cl-lib:int-sub jbc 1) kbl)
             1))
    (setf ijblsk 0)
    (f2cl-lib:fdo (p igl (f2cl-lib:int-add p 1))
                  ((> p
```

```
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add igl kbl
                                            (f2cl-lib:int-sub
                                             1)))
                     (the f2cl-lib:integer4 n)))
                nil)
(tagbody
  (setf aapp
          (f2cl-lib:fref sva-%data% (p) ((1 n))
                         sva-%offset%))
  (cond
   ((> aapp zero)
    (tagbody
      (setf pskipped 0)
      (f2cl-lib:fdo (q jgl (f2cl-lib:int-add q 1))
                    ((> q
                        (min
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add jgl kbl
                                                (f2cl-lib:int-sub
                                                 1)))
                         (the f2cl-lib:integer4 n)))
                     nil)
        (tagbody
          (setf aaqq
                  (f2cl-lib:fref sva-%data% (q) ((1 n))
                                 sva-%offset%))
          (cond
           ((> aaqq zero) (setf aapp0 aapp)
            (cond
             ((>= aaqq one)
              (cond
               ((>= aapp aaqq)
                (setf rotok (<= (* small aapp) aaqq)))
               (t (setf rotok (<= (* small aaqq) aapp))))
              (cond
               ((< aapp (f2cl-lib:f2cl/ big aaqq))
                (setf aapq
                        (/
                         (/
                          (*
                           (multiple-value-bind
                               (ret-val var-0 var-1 var-2
                                var-3 var-4)
                               (ddot m
                                (f2cl-lib:array-slice
                                 a-%data% double-float
                                 (1 p) ((1 lda) (1 *))
                                 a-%offset%)
```

```
                    1
                    (f2cl-lib:array-slice
                     a-%data% double-float
                     (1 q) ((1 lda) (1 *))
                     a-%offset%)
                    1)
                  (declare
                   (ignore var-1 var-2 var-3
                    var-4))
                  (when var-0 (setf m var-0))
                  ret-val)
                (f2cl-lib:fref work-%data% (p)
                               ((1 lwork))
                               work-%offset%)
                (f2cl-lib:fref work-%data% (q)
                               ((1 lwork))
                               work-%offset%))
             aaqq)
            aapp)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (dcopy m
      (f2cl-lib:array-slice a-%data%
                            double-float
                            (1 p)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice work-%data%
                            double-float
                            ((+ n 1))
                            ((1 lwork))
                            work-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5
      var-6 var-7 var-8 var-9)
     (dlascl "G" 0 0 aapp
      (f2cl-lib:array-slice work-%data%
                            double-float
                            (p)
                            ((1 lwork))
                            work-%offset%)
      m 1
      (f2cl-lib:array-slice work-%data%
```

```
                                  double-float
                                  ((+ n 1))
                                  ((1 lwork))
                                  work-%offset%)
          lda ierr)
        (declare
         (ignore var-0 var-1 var-2 var-4 var-6
          var-7))
        (when var-3 (setf aapp var-3))
        (when var-5 (setf m var-5))
        (when var-8 (setf lda var-8))
        (when var-9 (setf ierr var-9)))
      (setf aapq
              (/
               (*
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2
                     var-3 var-4)
                    (ddot m
                     (f2cl-lib:array-slice
                      work-%data% double-float
                      ((+ n 1)) ((1 lwork))
                      work-%offset%)
                     1
                     (f2cl-lib:array-slice
                      a-%data% double-float
                      (1 q) ((1 lda) (1 *))
                      a-%offset%)
                     1)
                  (declare
                   (ignore var-1 var-2 var-3
                    var-4))
                  (when var-0 (setf m var-0))
                  ret-val)
                (f2cl-lib:fref work-%data% (q)
                               ((1 lwork))
                               work-%offset%))
              aaqq)))))
  (t
   (cond
    ((>= aapp aaqq)
     (setf rotok (<= aapp (/ aaqq small))))
    (t (setf rotok (<= aaqq (/ aapp small)))))
   (cond
    ((> aapp (f2cl-lib:f2cl/ small aaqq))
     (setf aapq
             (/
              (/
               (*
                (multiple-value-bind
```

```
                    (ret-val var-0 var-1 var-2
                     var-3 var-4)
                    (ddot m
                     (f2cl-lib:array-slice
                      a-%data% double-float
                      (1 p) ((1 lda) (1 *))
                      a-%offset%)
                     1
                     (f2cl-lib:array-slice
                      a-%data% double-float
                      (1 q) ((1 lda) (1 *))
                      a-%offset%)
                     1)
                  (declare
                   (ignore var-1 var-2 var-3
                    var-4))
                  (when var-0 (setf m var-0))
                  ret-val)
                (f2cl-lib:fref work-%data% (p)
                               ((1 lwork))
                               work-%offset%)
                (f2cl-lib:fref work-%data% (q)
                               ((1 lwork))
                               work-%offset%))
            aaqq)
           aapp)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (dcopy m
      (f2cl-lib:array-slice a-%data%
                            double-float
                            (1 q)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice work-%data%
                            double-float
                            ((+ n 1))
                            ((1 lwork))
                            work-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5
      var-6 var-7 var-8 var-9)
     (dlascl "G" 0 0 aaqq
```

```
            (f2cl-lib:array-slice work-%data%
                                  double-float
                                  (q)
                                  ((1 lwork))
                                  work-%offset%)
            m 1
            (f2cl-lib:array-slice work-%data%
                                  double-float
                                  ((+ n 1))
                                  ((1 lwork))
                                  work-%offset%)
            lda ierr)
          (declare
           (ignore var-0 var-1 var-2 var-4 var-6
            var-7))
          (when var-3 (setf aaqq var-3))
          (when var-5 (setf m var-5))
          (when var-8 (setf lda var-8))
          (when var-9 (setf ierr var-9)))
        (setf aapq
                (/
                 (*
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2
                       var-3 var-4)
                      (ddot m
                       (f2cl-lib:array-slice
                        work-%data% double-float
                        ((+ n 1)) ((1 lwork))
                        work-%offset%)
                       1
                       (f2cl-lib:array-slice
                        a-%data% double-float
                        (1 p) ((1 lda) (1 *))
                        a-%offset%)
                       1)
                    (declare
                     (ignore var-1 var-2 var-3
                      var-4))
                    (when var-0 (setf m var-0))
                    ret-val)
                  (f2cl-lib:fref work-%data% (p)
                                 ((1 lwork))
                                 work-%offset%))
                aapp))))))
    (setf mxaapq
            (max mxaapq (f2cl-lib:dabs aapq)))
    (cond
     ((> (f2cl-lib:dabs aapq) tol)
      (setf notrot 0) (setf pskipped 0)
```

```
(setf iswrot (f2cl-lib:int-add iswrot 1))
(cond
 (rotok (setf aqoap (/ aaqq aapp))
  (setf apoaq (/ aapp aaqq))
  (setf theta
          (/
           (* (- half)
              (f2cl-lib:dabs
               (- aqoap apoaq)))
           aapq))
  (if (> aaqq aapp0)
      (setf theta (- theta)))
  (cond
   ((> (f2cl-lib:dabs theta) bigtheta)
    (setf t$ (/ half theta))
    (setf (f2cl-lib:fref fastr (3) ((1 5)))
            (/
             (* t$
                (f2cl-lib:fref work-%data%
                               (p)
                               ((1 lwork))
                               work-%offset%))
             (f2cl-lib:fref work-%data% (q)
                            ((1 lwork))
                            work-%offset%)))
    (setf (f2cl-lib:fref fastr (4) ((1 5)))
            (/
             (* (- t$)
                (f2cl-lib:fref work-%data%
                               (q)
                               ((1 lwork))
                               work-%offset%))
             (f2cl-lib:fref work-%data% (p)
                            ((1 lwork))
                            work-%offset%)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4
         var-5)
        (drotm m
         (f2cl-lib:array-slice a-%data%
                               double-float
                               (1 p)
                               ((1 lda)
                                (1 *))
                               a-%offset%)
         1
         (f2cl-lib:array-slice a-%data%
                               double-float
                               (1 q)
                               ((1 lda)
```

```
                                (1 *))
                               a-%offset%)
      1 fastr)
   (declare
    (ignore var-1 var-2 var-3 var-4
     var-5))
   (when var-0 (setf m var-0)))
 (if rsvec
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4
          var-5)
         (drotm mvl
          (f2cl-lib:array-slice v-%data%
                                double-float
                                (1 p)
                                ((1 ldv)
                                 (1 *))
                                v-%offset%)
          1
          (f2cl-lib:array-slice v-%data%
                                double-float
                                (1 q)
                                ((1 ldv)
                                 (1 *))
                                v-%offset%)
          1 fastr)
       (declare
        (ignore var-1 var-2 var-3 var-4
         var-5))
       (when var-0 (setf mvl var-0))))
 (setf (f2cl-lib:fref sva-%data% (q)
                      ((1 n))
                      sva-%offset%)
         (* aaqq
            (f2cl-lib:dsqrt
             (max zero
                  (+ one
                     (* t$ apoaq
                        aapq))))))
 (setf aapp
         (* aapp
            (f2cl-lib:dsqrt
             (max zero
                  (+ one
                     (* (- t$) aqoap
                        aapq))))))
 (setf mxsinj
         (max mxsinj
              (f2cl-lib:dabs t))))
(t
```

```
(setf thsign
        (coerce
         (- (f2cl-lib:dsign one aapq))
         'double-float))
(if (> aaqq aapp0)
    (setf thsign (- thsign)))
(setf t$
        (/ one
           (+ theta
              (* thsign
                 (f2cl-lib:dsqrt
                  (+ one
                     (* theta
                        theta)))))))
(setf cs
        (f2cl-lib:dsqrt
         (/ one (+ one (* t$ t$)))))
(setf sn (* t$ cs))
(setf mxsinj
        (max mxsinj
             (f2cl-lib:dabs sn)))
(setf (f2cl-lib:fref sva-%data% (q)
                     ((1 n))
                     sva-%offset%)
        (* aaqq
           (f2cl-lib:dsqrt
            (max zero
                 (+ one
                    (* t$ apoaq
                       aapq))))))
(setf aapp
        (* aapp
           (f2cl-lib:dsqrt
            (max zero
                 (+ one
                    (* (- t$) aqoap
                       aapq))))))
(setf apoaq
        (/
         (f2cl-lib:fref work-%data% (p)
                        ((1 lwork))
                        work-%offset%)
         (f2cl-lib:fref work-%data% (q)
                        ((1 lwork))
                        work-%offset%)))
(setf aqoap
        (/
         (f2cl-lib:fref work-%data% (q)
                        ((1 lwork))
                        work-%offset%)
```

```
        (f2cl-lib:fref work-%data% (p)
                       ((1 lwork))
                       work-%offset%)))
(cond
 ((>=
   (f2cl-lib:fref work (p) ((1 lwork)))
   one)
  (cond
   ((>=
     (f2cl-lib:fref work (q)
                    ((1 lwork)))
     one)
    (setf (f2cl-lib:fref fastr (3)
                         ((1 5)))
          (* t$ apoaq))
    (setf (f2cl-lib:fref fastr (4)
                         ((1 5)))
          (* (- t$) aqoap))
    (setf (f2cl-lib:fref work-%data%
                         (p)
                         ((1 lwork))
                         work-%offset%)
          (*
           (f2cl-lib:fref work-%data%
                          (p)
                          ((1 lwork))
                          work-%offset%)
           cs))
    (setf (f2cl-lib:fref work-%data%
                         (q)
                         ((1 lwork))
                         work-%offset%)
          (*
           (f2cl-lib:fref work-%data%
                          (q)
                          ((1 lwork))
                          work-%offset%)
           cs))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4
         var-5)
        (drotm m
         (f2cl-lib:array-slice a-%data%
                               double-float
                               (1 p)
                               ((1 lda)
                                (1 *))
                               a-%offset%)
         1
         (f2cl-lib:array-slice a-%data%
```

```
                                double-float
                                (1 q)
                                ((1 lda)
                                 (1 *))
                                a-%offset%)
      1 fastr)
    (declare
     (ignore var-1 var-2 var-3 var-4
      var-5))
    (when var-0 (setf m var-0)))
  (if rsvec
      (multiple-value-bind
          (var-0 var-1 var-2 var-3
           var-4 var-5)
          (drotm mvl
           (f2cl-lib:array-slice
            v-%data% double-float
            (1 p) ((1 ldv) (1 *))
            v-%offset%)
           1
           (f2cl-lib:array-slice
            v-%data% double-float
            (1 q) ((1 ldv) (1 *))
            v-%offset%)
           1 fastr)
        (declare
         (ignore var-1 var-2 var-3
          var-4 var-5))
        (when var-0
          (setf mvl var-0)))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4
       var-5)
      (daxpy m (* (- t$) aqoap)
       (f2cl-lib:array-slice a-%data%
                             double-float
                             (1 q)
                             ((1 lda)
                              (1 *))
                             a-%offset%)
       1
       (f2cl-lib:array-slice a-%data%
                             double-float
                             (1 p)
                             ((1 lda)
                              (1 *))
                             a-%offset%)
       1)
    (declare
```

```
   (ignore var-1 var-2 var-3 var-4
    var-5))
  (when var-0 (setf m var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5)
    (daxpy m (* cs sn apoaq)
     (f2cl-lib:array-slice a-%data%
                           double-float
                           (1 p)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1
     (f2cl-lib:array-slice a-%data%
                           double-float
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3 var-4
    var-5))
  (when var-0 (setf m var-0)))
(cond
 (rsvec
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
       var-4 var-5)
      (daxpy mvl (* (- t$) aqoap)
       (f2cl-lib:array-slice
        v-%data% double-float (1 q)
        ((1 ldv) (1 *)) v-%offset%)
       1
       (f2cl-lib:array-slice
        v-%data% double-float (1 p)
        ((1 ldv) (1 *)) v-%offset%)
       1)
    (declare
     (ignore var-1 var-2 var-3
      var-4 var-5))
    (when var-0 (setf mvl var-0)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
       var-4 var-5)
      (daxpy mvl (* cs sn apoaq)
       (f2cl-lib:array-slice
        v-%data% double-float (1 p)
        ((1 ldv) (1 *)) v-%offset%)
```

```
                1
                (f2cl-lib:array-slice
                 v-%data% double-float (1 q)
                 ((1 ldv) (1 *)) v-%offset%)
                1)
            (declare
             (ignore var-1 var-2 var-3
              var-4 var-5))
            (when var-0
              (setf mvl var-0)))))
      (setf (f2cl-lib:fref work-%data%
                           (p)
                           ((1 lwork))
                           work-%offset%)
              (*
               (f2cl-lib:fref work-%data%
                              (p)
                              ((1 lwork))
                              work-%offset%)
               cs))
      (setf (f2cl-lib:fref work-%data%
                           (q)
                           ((1 lwork))
                           work-%offset%)
              (/
               (f2cl-lib:fref work-%data%
                              (q)
                              ((1 lwork))
                              work-%offset%)
               cs)))))
  (t
   (cond
    ((>=
      (f2cl-lib:fref work (q)
                     ((1 lwork)))
      one)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4
          var-5)
         (daxpy m (* t$ apoaq)
          (f2cl-lib:array-slice a-%data%
                                double-float
                                (1 p)
                                ((1 lda)
                                 (1 *))
                                a-%offset%)
          1
          (f2cl-lib:array-slice a-%data%
                                double-float
                                (1 q)
```

```
                                ((1 lda)
                                 (1 *))
                                a-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4
     var-5))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4
      var-5)
     (daxpy m (* (- cs) sn aqoap)
      (f2cl-lib:array-slice a-%data%
                            double-float
                            (1 q)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice a-%data%
                            double-float
                            (1 p)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4
     var-5))
   (when var-0 (setf m var-0)))
 (cond
  (rsvec
   (multiple-value-bind
       (var-0 var-1 var-2 var-3
        var-4 var-5)
       (daxpy mvl (* t$ apoaq)
        (f2cl-lib:array-slice
         v-%data% double-float (1 p)
         ((1 ldv) (1 *)) v-%offset%)
        1
        (f2cl-lib:array-slice
         v-%data% double-float (1 q)
         ((1 ldv) (1 *)) v-%offset%)
        1)
     (declare
      (ignore var-1 var-2 var-3
       var-4 var-5))
     (when var-0 (setf mvl var-0)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3
```

```
        var-4 var-5)
       (daxpy mvl
        (* (- cs) sn aqoap)
        (f2cl-lib:array-slice
         v-%data% double-float (1 q)
         ((1 ldv) (1 *)) v-%offset%)
        1
        (f2cl-lib:array-slice
         v-%data% double-float (1 p)
         ((1 ldv) (1 *)) v-%offset%)
        1)
     (declare
      (ignore var-1 var-2 var-3
       var-4 var-5))
     (when var-0
       (setf mvl var-0)))))
 (setf (f2cl-lib:fref work-%data%
                      (p)
                      ((1 lwork))
                      work-%offset%)
        (/
         (f2cl-lib:fref work-%data%
                        (p)
                        ((1 lwork))
                        work-%offset%)
         cs))
 (setf (f2cl-lib:fref work-%data%
                      (q)
                      ((1 lwork))
                      work-%offset%)
        (*
         (f2cl-lib:fref work-%data%
                        (q)
                        ((1 lwork))
                        work-%offset%)
         cs)))
(t
 (cond
  ((>=
    (f2cl-lib:fref work (p)
                   ((1 lwork)))
    (f2cl-lib:fref work (q)
                   ((1 lwork))))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3
        var-4 var-5)
       (daxpy m (* (- t$) aqoap)
        (f2cl-lib:array-slice
         a-%data% double-float (1 q)
         ((1 lda) (1 *)) a-%offset%)
```

```
      1
      (f2cl-lib:array-slice
       a-%data% double-float (1 p)
       ((1 lda) (1 *)) a-%offset%)
      1)
  (declare
   (ignore var-1 var-2 var-3
    var-4 var-5))
  (when var-0 (setf m var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3
     var-4 var-5)
    (daxpy m (* cs sn apoaq)
     (f2cl-lib:array-slice
      a-%data% double-float (1 p)
      ((1 lda) (1 *)) a-%offset%)
     1
     (f2cl-lib:array-slice
      a-%data% double-float (1 q)
      ((1 lda) (1 *)) a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3
    var-4 var-5))
  (when var-0 (setf m var-0)))
(setf (f2cl-lib:fref work-%data%
                     (p)
                     ((1 lwork))
                     work-%offset%)
        (*
         (f2cl-lib:fref
          work-%data% (p)
          ((1 lwork))
          work-%offset%)
         cs))
(setf (f2cl-lib:fref work-%data%
                     (q)
                     ((1 lwork))
                     work-%offset%)
        (/
         (f2cl-lib:fref
          work-%data% (q)
          ((1 lwork))
          work-%offset%)
         cs))
(cond
 (rsvec
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
       var-4 var-5)
```

```
                (daxpy mvl (* (- t$) aqoap)
                 (f2cl-lib:array-slice
                  v-%data% double-float
                  (1 q) ((1 ldv) (1 *))
                  v-%offset%)
                 1
                 (f2cl-lib:array-slice
                  v-%data% double-float
                  (1 p) ((1 ldv) (1 *))
                  v-%offset%)
                 1)
               (declare
                (ignore var-1 var-2 var-3
                 var-4 var-5))
               (when var-0
                 (setf mvl var-0)))
             (multiple-value-bind
                 (var-0 var-1 var-2 var-3
                  var-4 var-5)
                 (daxpy mvl (* cs sn apoaq)
                  (f2cl-lib:array-slice
                   v-%data% double-float
                   (1 p) ((1 ldv) (1 *))
                   v-%offset%)
                  1
                  (f2cl-lib:array-slice
                   v-%data% double-float
                   (1 q) ((1 ldv) (1 *))
                   v-%offset%)
                  1)
               (declare
                (ignore var-1 var-2 var-3
                 var-4 var-5))
               (when var-0
                 (setf mvl var-0))))))
           (t
            (multiple-value-bind
                (var-0 var-1 var-2 var-3
                 var-4 var-5)
                (daxpy m (* t$ apoaq)
                 (f2cl-lib:array-slice
                  a-%data% double-float (1 p)
                  ((1 lda) (1 *)) a-%offset%)
                 1
                 (f2cl-lib:array-slice
                  a-%data% double-float (1 q)
                  ((1 lda) (1 *)) a-%offset%)
                 1)
             (declare
              (ignore var-1 var-2 var-3
```

```
    var-4 var-5))
  (when var-0 (setf m var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3
     var-4 var-5)
    (daxpy m (* (- cs) sn aqoap)
     (f2cl-lib:array-slice
      a-%data% double-float (1 q)
      ((1 lda) (1 *)) a-%offset%)
     1
     (f2cl-lib:array-slice
      a-%data% double-float (1 p)
      ((1 lda) (1 *)) a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3
    var-4 var-5))
  (when var-0 (setf m var-0)))
(setf (f2cl-lib:fref work-%data%
                     (p)
                     ((1 lwork))
                     work-%offset%)
        (/
         (f2cl-lib:fref
          work-%data% (p)
          ((1 lwork))
          work-%offset%)
         cs))
(setf (f2cl-lib:fref work-%data%
                     (q)
                     ((1 lwork))
                     work-%offset%)
        (*
         (f2cl-lib:fref
          work-%data% (q)
          ((1 lwork))
          work-%offset%)
         cs))
(cond
 (rsvec
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
       var-4 var-5)
      (daxpy mvl (* t$ apoaq)
       (f2cl-lib:array-slice
        v-%data% double-float
        (1 p) ((1 ldv) (1 *))
        v-%offset%)
       1
       (f2cl-lib:array-slice
```

```
                    v-%data% double-float
                    (1 q) ((1 ldv) (1 *))
                    v-%offset%)
                   1)
                (declare
                 (ignore var-1 var-2 var-3
                  var-4 var-5))
                (when var-0
                  (setf mvl var-0)))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3
                   var-4 var-5)
                  (daxpy mvl
                   (* (- cs) sn aqoap)
                   (f2cl-lib:array-slice
                    v-%data% double-float
                    (1 q) ((1 ldv) (1 *))
                    v-%offset%)
                   1
                   (f2cl-lib:array-slice
                    v-%data% double-float
                    (1 p) ((1 ldv) (1 *))
                    v-%offset%)
                   1)
                (declare
                 (ignore var-1 var-2 var-3
                  var-4 var-5))
                (when var-0
                  (setf mvl
                          var-0)))))))))))))))
(t
 (cond
  ((> aapp aaqq)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4)
       (dcopy m
        (f2cl-lib:array-slice a-%data%
                              double-float
                              (1 p)
                              ((1 lda)
                               (1 *))
                              a-%offset%)
        1
        (f2cl-lib:array-slice work-%data%
                              double-float
                              ((+ n 1))
                              ((1 lwork))
                              work-%offset%)
        1)
     (declare
```

```
   (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf m var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5 var-6 var-7 var-8 var-9)
    (dlascl "G" 0 0 aapp one m 1
     (f2cl-lib:array-slice work-%data%
                           double-float
                           ((+ n 1))
                           ((1 lwork))
                           work-%offset%)
     lda ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6
    var-7))
  (when var-3 (setf aapp var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5 var-6 var-7 var-8 var-9)
    (dlascl "G" 0 0 aaqq one m 1
     (f2cl-lib:array-slice a-%data%
                           double-float
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     lda ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6
    var-7))
  (when var-3 (setf aaqq var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(setf temp1
        (/
         (* (- aapq)
            (f2cl-lib:fref work-%data%
                           (p)
                           ((1 lwork))
                           work-%offset%))
         (f2cl-lib:fref work-%data% (q)
                        ((1 lwork))
                        work-%offset%)))
(multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4
     var-5)
    (daxpy m temp1
     (f2cl-lib:array-slice work-%data%
                           double-float
                           ((+ n 1))
                           ((1 lwork))
                           work-%offset%)
     1
     (f2cl-lib:array-slice a-%data%
                           double-float
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1)
  (declare
   (ignore var-2 var-3 var-4 var-5))
  (when var-0 (setf m var-0))
  (when var-1 (setf temp1 var-1)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5 var-6 var-7 var-8 var-9)
    (dlascl "G" 0 0 one aaqq m 1
     (f2cl-lib:array-slice a-%data%
                           double-float
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     lda ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6
    var-7))
  (when var-3 (setf one var-3))
  (when var-4 (setf aaqq var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(setf (f2cl-lib:fref sva-%data% (q)
                     ((1 n))
                     sva-%offset%)
        (* aaqq
           (f2cl-lib:dsqrt
            (max zero
                 (- one
                    (* aapq aapq))))))
(setf mxsinj (max mxsinj sfmin)))
(t
 (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4)
    (dcopy m
     (f2cl-lib:array-slice a-%data%
                           double-float
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1
     (f2cl-lib:array-slice work-%data%
                           double-float
                           ((+ n 1))
                           ((1 lwork))
                           work-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf m var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5 var-6 var-7 var-8 var-9)
    (dlascl "G" 0 0 aaqq one m 1
     (f2cl-lib:array-slice work-%data%
                           double-float
                           ((+ n 1))
                           ((1 lwork))
                           work-%offset%)
     lda ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6
    var-7))
  (when var-3 (setf aaqq var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5 var-6 var-7 var-8 var-9)
    (dlascl "G" 0 0 aapp one m 1
     (f2cl-lib:array-slice a-%data%
                           double-float
                           (1 p)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     lda ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6
    var-7))
```

```
  (when var-3 (setf aapp var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(setf temp1
        (/
         (* (- aapq)
            (f2cl-lib:fref work-%data%
                           (q)
                           ((1 lwork))
                           work-%offset%))
         (f2cl-lib:fref work-%data% (p)
                        ((1 lwork))
                        work-%offset%)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5)
    (daxpy m temp1
     (f2cl-lib:array-slice work-%data%
                           double-float
                           ((+ n 1))
                           ((1 lwork))
                           work-%offset%)
     1
     (f2cl-lib:array-slice a-%data%
                           double-float
                           (1 p)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1)
  (declare
   (ignore var-2 var-3 var-4 var-5))
  (when var-0 (setf m var-0))
  (when var-1 (setf temp1 var-1)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5 var-6 var-7 var-8 var-9)
    (dlascl "G" 0 0 one aapp m 1
     (f2cl-lib:array-slice a-%data%
                           double-float
                           (1 p)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     lda ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6
    var-7))
```

```
      (when var-3 (setf one var-3))
      (when var-4 (setf aapp var-4))
      (when var-5 (setf m var-5))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9)))
    (setf (f2cl-lib:fref sva-%data% (p)
                         ((1 n))
                         sva-%offset%)
            (* aapp
               (f2cl-lib:dsqrt
                (max zero
                     (- one
                        (* aapq aapq))))))
    (setf mxsinj (max mxsinj sfmin))))))
(cond
 ((<=
   (expt
    (f2cl-lib:f2cl/
     (f2cl-lib:fref sva (q) ((1 n))) aaqq)
    2)
   rooteps)
  (cond
   ((and (< aaqq rootbig)
         (> aaqq rootsfmin))
    (setf (f2cl-lib:fref sva-%data% (q)
                         ((1 n))
                         sva-%offset%)
            (*
             (multiple-value-bind
                 (ret-val var-0 var-1
                  var-2)
                 (dnrm2 m
                  (f2cl-lib:array-slice
                   a-%data% double-float
                   (1 q) ((1 lda) (1 *))
                   a-%offset%)
                  1)
               (declare
                (ignore var-1 var-2))
               (when var-0 (setf m var-0))
               ret-val)
             (f2cl-lib:fref work-%data% (q)
                            ((1 lwork))
                            work-%offset%))))
   (t (setf t$ zero) (setf aaqq one)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4)
        (dlassq m
         (f2cl-lib:array-slice a-%data%
                               double-float
```

```
                                  (1 q)
                                  ((1 lda)
                                   (1 *))
                                  a-%offset%)
          1 t$ aaqq)
        (declare (ignore var-1 var-2))
        (when var-0 (setf m var-0))
        (when var-3 (setf t$ var-3))
        (when var-4 (setf aaqq var-4)))
      (setf (f2cl-lib:fref sva-%data% (q)
                           ((1 n))
                           sva-%offset%)
              (* t$ (f2cl-lib:dsqrt aaqq)
                 (f2cl-lib:fref work-%data%
                                (q)
                                ((1 lwork))
                                work-%offset%)))))))
(cond
 ((<= (expt (f2cl-lib:f2cl/ aapp aapp0) 2)
      rooteps)
  (cond
   ((and (< aapp rootbig)
         (> aapp rootsfmin))
    (setf aapp
            (*
             (multiple-value-bind
                 (ret-val var-0 var-1
                  var-2)
                 (dnrm2 m
                  (f2cl-lib:array-slice
                   a-%data% double-float
                   (1 p) ((1 lda) (1 *))
                   a-%offset%)
                  1)
                (declare
                 (ignore var-1 var-2))
                (when var-0 (setf m var-0))
                ret-val)
             (f2cl-lib:fref work-%data% (p)
                            ((1 lwork))
                            work-%offset%))))
   (t (setf t$ zero) (setf aapp one)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4)
        (dlassq m
         (f2cl-lib:array-slice a-%data%
                               double-float
                               (1 p)
                               ((1 lda)
                                (1 *))
```

```
                                                a-%offset%)
                          1 t$ aapp)
                       (declare (ignore var-1 var-2))
                       (when var-0 (setf m var-0))
                       (when var-3 (setf t$ var-3))
                       (when var-4 (setf aapp var-4)))
                     (setf aapp
                             (* t$ (f2cl-lib:dsqrt aapp)
                                (f2cl-lib:fref work-%data%
                                               (p)
                                               ((1 lwork))
                                               work-%offset%)))))
                  (setf (f2cl-lib:fref sva-%data% (p)
                                       ((1 n))
                                       sva-%offset%)
                            aapp))))
              (t (setf notrot (f2cl-lib:int-add notrot 1))
               (setf pskipped
                       (f2cl-lib:int-add pskipped 1))
               (setf ijblsk
                       (f2cl-lib:int-add ijblsk 1)))))
            (t (setf notrot (f2cl-lib:int-add notrot 1))
             (setf pskipped (f2cl-lib:int-add pskipped 1))
             (setf ijblsk (f2cl-lib:int-add ijblsk 1))))
           (cond
            ((and (<= i swband) (>= ijblsk blskip))
             (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                                  sva-%offset%)
                     aapp)
             (setf notrot 0) (go label2011)))
           (cond
            ((and (<= i swband) (> pskipped rowskip))
             (setf aapp (- aapp)) (setf notrot 0)
             (go label2203)))
          label2200))
      label2203
       (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                            sva-%offset%)
               aapp)))
    (t
     (if (= aapp zero)
         (setf notrot
                 (f2cl-lib:int-add
                  (f2cl-lib:int-sub
                   (f2cl-lib:int-add notrot
                                     (min
                                      (the
                                       f2cl-lib:integer4
                                       (f2cl-lib:int-sub
                                        (f2cl-lib:int-add
```

```
                                                        jgl kbl)
                                                       1))
                                                     (the
                                                      f2cl-lib:integer4
                                                      n)))
                                 jgl)
                                1)))
                  (if (< aapp zero)
                      (setf notrot 0))))
              label2100))
          label2010))
      label2011
       (f2cl-lib:fdo (p igl (f2cl-lib:int-add p 1))
                     ((> p
                         (min
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add igl kbl
                                                 (f2cl-lib:int-sub
                                                  1)))
                          (the f2cl-lib:integer4 n)))
                      nil)
         (tagbody
           (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                   (f2cl-lib:dabs
                    (f2cl-lib:fref sva-%data% (p) ((1 n))
                                   sva-%offset%)))
          label2012))
      label2000))
  (cond
   ((and (< (f2cl-lib:fref sva (n) ((1 n))) rootbig)
         (> (f2cl-lib:fref sva (n) ((1 n))) rootsfmin))
    (setf (f2cl-lib:fref sva-%data% (n) ((1 n)) sva-%offset%)
            (*
             (multiple-value-bind (ret-val var-0 var-1 var-2)
                 (dnrm2 m
                  (f2cl-lib:array-slice a-%data% double-float (1 n)
                                        ((1 lda) (1 *)) a-%offset%)
                  1)
               (declare (ignore var-1 var-2))
               (when var-0 (setf m var-0))
               ret-val)
             (f2cl-lib:fref work-%data% (n) ((1 lwork))
                            work-%offset%))))
   (t (setf t$ zero) (setf aapp one)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dlassq m
         (f2cl-lib:array-slice a-%data% double-float (1 n)
                               ((1 lda) (1 *)) a-%offset%)
         1 t$ aapp)
      (declare (ignore var-1 var-2))
```

```
          (when var-0 (setf m var-0))
          (when var-3 (setf t$ var-3))
          (when var-4 (setf aapp var-4)))
        (setf (f2cl-lib:fref sva-%data% (n) ((1 n)) sva-%offset%)
                (* t$ (f2cl-lib:dsqrt aapp)
                   (f2cl-lib:fref work-%data% (n) ((1 lwork))
                                  work-%offset%)))))
      (if (and (< i swband) (or (<= mxaapq roottol) (<= iswrot n)))
          (setf swband i))
      (cond
       ((and (> i (f2cl-lib:int-add swband 1))
             (< mxaapq (* (f2cl-lib:dsqrt (f2cl-lib:dble n)) tol))
             (< (* (f2cl-lib:dble n) mxaapq mxsinj) tol))
        (go label1994)))
      (if (>= notrot emptsw)
          (go label1994))
    label1993))
 (setf info (f2cl-lib:int-sub nsweep 1))
 (go label1995)
label1994
 (setf info 0)
label1995
 (setf n2 0)
 (setf n4 0)
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
   (tagbody
     (setf q
             (f2cl-lib:int-sub
              (f2cl-lib:int-add
               (idamax (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
                (f2cl-lib:array-slice sva-%data% double-float (p)
                                      ((1 n)) sva-%offset%)
                1)
               p)
              1))
     (cond
      ((/= p q)
       (setf temp1 (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%))
       (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
               (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%))
       (setf (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%) temp1)
       (setf temp1
               (f2cl-lib:fref work-%data% (p) ((1 lwork))
                              work-%offset%))
       (setf (f2cl-lib:fref work-%data% (p) ((1 lwork)) work-%offset%)
               (f2cl-lib:fref work-%data% (q) ((1 lwork))
                              work-%offset%))
       (setf (f2cl-lib:fref work-%data% (q) ((1 lwork)) work-%offset%)
               temp1)
```

```
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dswap m
               (f2cl-lib:array-slice a-%data% double-float (1 p)
                                     ((1 lda) (1 *)) a-%offset%)
               1
               (f2cl-lib:array-slice a-%data% double-float (1 q)
                                     ((1 lda) (1 *)) a-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf m var-0)))
          (if rsvec
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (dswap mvl
                   (f2cl-lib:array-slice v-%data% double-float (1 p)
                                         ((1 ldv) (1 *)) v-%offset%)
                   1
                   (f2cl-lib:array-slice v-%data% double-float (1 q)
                                         ((1 ldv) (1 *)) v-%offset%)
                   1)
                (declare (ignore var-1 var-2 var-3 var-4))
                (when var-0 (setf mvl var-0))))))
        (cond
         ((/= (f2cl-lib:fref sva (p) ((1 n))) zero)
          (setf n4 (f2cl-lib:int-add n4 1))
          (if (>
               (* (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%) skl)
               sfmin)
              (setf n2 (f2cl-lib:int-add n2 1)))))
      label5991))
    (cond
     ((/= (f2cl-lib:fref sva (n) ((1 n))) zero)
      (setf n4 (f2cl-lib:int-add n4 1))
      (if (> (* (f2cl-lib:fref sva-%data% (n) ((1 n)) sva-%offset%) skl)
             sfmin)
          (setf n2 (f2cl-lib:int-add n2 1)))))
    (cond
     ((or lsvec uctol)
      (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                    ((> p n2) nil)
        (tagbody
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (dscal m
               (/ (f2cl-lib:fref work-%data% (p) ((1 lwork)) work-%offset%)
                  (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%))
               (f2cl-lib:array-slice a-%data% double-float (1 p)
                                     ((1 lda) (1 *)) a-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3))
            (when var-0 (setf m var-0)))
          label1998))))
```

```
(cond
 (rsvec
  (cond
   (applv
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal mvl
             (f2cl-lib:array-slice work-%data% double-float (p)
                                   ((1 lwork)) work-%offset%)
             (f2cl-lib:array-slice v-%data% double-float (1 p)
                                   ((1 ldv) (1 *)) v-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3))
          (when var-0 (setf mvl var-0)))
       label2398)))
   (t
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (setf temp1
                (/ one
                   (multiple-value-bind (ret-val var-0 var-1 var-2)
                       (dnrm2 mvl
                        (f2cl-lib:array-slice v-%data% double-float
                                              (1 p) ((1 ldv) (1 *))
                                              v-%offset%)
                        1)
                     (declare (ignore var-1 var-2))
                     (when var-0 (setf mvl var-0))
                     ret-val)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal mvl temp1
             (f2cl-lib:array-slice v-%data% double-float (1 p)
                                   ((1 ldv) (1 *)) v-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf mvl var-0))
          (when var-1 (setf temp1 var-1)))
       label2399))))))
(cond
 ((or
   (and (> skl one)
        (< (f2cl-lib:fref sva (1) ((1 n))) (f2cl-lib:f2cl/ big skl)))
   (and (< skl one)
        (>
         (f2cl-lib:fref sva
                        ((max (the f2cl-lib:integer4 n2)
                              (the f2cl-lib:integer4 1)))
```

```
                              ((1 n)))
                 (f2cl-lib:f2cl/ sfmin skl))))
          (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                        ((> p n) nil)
            (tagbody
              (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                      (* skl
                         (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)))
             label2400))
          (setf skl one)))
        (setf (f2cl-lib:fref work-%data% (1) ((1 lwork)) work-%offset%) skl)
        (setf (f2cl-lib:fref work-%data% (2) ((1 lwork)) work-%offset%)
                (f2cl-lib:dble n4))
        (setf (f2cl-lib:fref work-%data% (3) ((1 lwork)) work-%offset%)
                (f2cl-lib:dble n2))
        (setf (f2cl-lib:fref work-%data% (4) ((1 lwork)) work-%offset%)
                (f2cl-lib:dble i))
        (setf (f2cl-lib:fref work-%data% (5) ((1 lwork)) work-%offset%) mxaapq)
        (setf (f2cl-lib:fref work-%data% (6) ((1 lwork)) work-%offset%) mxsinj)
        (go end_label)
       end_label
        (return
         (values joba jobu jobv m n nil lda nil nil nil ldv nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgesvj
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::joba
                                              fortran-to-lisp::jobu
                                              fortran-to-lisp::jobv
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::ldv nil nil
```

```
                                        fortran-to-lisp::info)
                                      :calls 'nil)))
```

———-

32.29 dgetf2

32.29.1 Axiom unit tests

— **dgetf2.input** —

```
)set break resume
)sys rm -f dgetf2.output
)spool dgetf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.29.2 Axiom help page

— **dgetf2.help** —

```
======================================================================
dgetf2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgetf2 (M, N, A, LDA, IPIV, INFO)
           DGETF2 computes the LU factorization of a general m-by-n matrix
           using partial pivoting with row interchanges (unblocked algorithm).

Function/Subroutine Documentation
```

```
   subroutine dgetf2 (integer M, integer N, double precision, dimension( lda,
       * ) A, integer LDA, integer, dimension( * ) IPIV, integer INFO)
       DGETF2 computes the LU factorization of a general m-by-n matrix using
       partial pivoting with row interchanges (unblocked algorithm).

Purpose:
   DGETF2 computes an LU factorization of a general m-by-n matrix A
   using partial pivoting with row interchanges.

   The factorization has the form
      A = P * L * U
   where P is a permutation matrix, L is lower triangular with unit
   diagonal elements (lower trapezoidal if m > n), and U is upper
   triangular (upper trapezoidal if m < n).

   This is the right-looking Level 2 BLAS version of the algorithm.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the m by n matrix to be factored.
   On exit, the factors L and U from the factorization
   A = P*L*U; the unit diagonal elements of L are not stored.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 IPIV
   IPIV is INTEGER array, dimension (min(M,N))
   The pivot indices; for 1 <= i <= min(M,N), row i of the
   matrix was interchanged with row IPIV(i).

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -k, the k-th argument had an illegal value
   > 0: if INFO = k, U(k,k) is exactly zero. The factorization
      has been completed, but the factor U is exactly
      singular, and division by zero will occur if it is used
      to solve a system of equations.
```

———-

32.29.3 fortran code

— **dgetf2.f** —

```
*  =====================================================================
      SUBROUTINE DGETF2( M, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION   SFMIN
      INTEGER            I, J, JP
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      INTEGER            IDAMAX
      EXTERNAL           DLAMCH, IDAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGER, DSCAL, DSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
```

```
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGETF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Compute machine safe minimum
*
      SFMIN = DLAMCH('S')
*
      DO 10 J = 1, MIN( M, N )
*
*        Find pivot and test for singularity.
*
         JP = J - 1 + IDAMAX( M-J+1, A( J, J ), 1 )
         IPIV( J ) = JP
         IF( A( JP, J ).NE.ZERO ) THEN
*
*           Apply the interchange to columns 1:N.
*
            IF( JP.NE.J )
     $         CALL DSWAP( N, A( J, 1 ), LDA, A( JP, 1 ), LDA )
*
*           Compute elements J+1:M of J-th column.
*
            IF( J.LT.M ) THEN
               IF( ABS(A( J, J )) .GE. SFMIN ) THEN
                  CALL DSCAL( M-J, ONE / A( J, J ), A( J+1, J ), 1 )
               ELSE
                 DO 20 I = 1, M-J
                    A( J+I, J ) = A( J+I, J ) / A( J, J )
   20            CONTINUE
               END IF
            END IF
*
         ELSE IF( INFO.EQ.0 ) THEN
*
```

```
            INFO = J
         END IF
*
         IF( J.LT.MIN( M, N ) ) THEN
*
*           Update trailing submatrix.
*
            CALL DGER( M-J, N-J, -ONE, A( J+1, J ), 1, A( J, J+1 ), LDA,
     $                 A( J+1, J+1 ), LDA )
         END IF
   10 CONTINUE
      RETURN
*
*     End of DGETF2
*
      END
```

———-

32.29.4 lisp code

— dgetf2.lisp —

———-

32.30 dgetrf

32.30.1 Axiom unit tests

— dgetrf.input —

```
)set break resume
)sys rm -f dgetrf.output
)spool dgetrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.30.2 Axiom help page

— dgetrf.help —

```
=====================================================================
dgetrf examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgetrf (M, N, A, LDA, IPIV, INFO)
           DGETRF

Function/Subroutine Documentation
   subroutine dgetrf (integer M, integer N, double precision, dimension( lda,
       * ) A, integer LDA, integer, dimension( * ) IPIV, integer INFO)
       DGETRF

Purpose:
   DGETRF computes an LU factorization of a general M-by-N matrix A
   using partial pivoting with row interchanges.

   The factorization has the form
      A = P * L * U
   where P is a permutation matrix, L is lower triangular with unit
   diagonal elements (lower trapezoidal if m > n), and U is upper
   triangular (upper trapezoidal if m < n).

   This is the right-looking Level 3 BLAS version of the algorithm.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the M-by-N matrix to be factored.
   On exit, the factors L and U from the factorization
   A = P*L*U; the unit diagonal elements of L are not stored.
```

```
 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 IPIV
   IPIV is INTEGER array, dimension (min(M,N))
   The pivot indices; for 1 <= i <= min(M,N), row i of the
   matrix was interchanged with row IPIV(i).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, U(i,i) is exactly zero. The factorization
      has been completed, but the factor U is exactly
      singular, and division by zero will occur if it is used
      to solve a system of equations.
```

32.30.3 fortran code

— dgetrf.f —

```
*  =====================================================================
      SUBROUTINE DGETRF( M, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
```

```
*     .. Local Scalars ..
      INTEGER            I, IINFO, J, JB, NB
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEMM, DGETRF2, DLASWP, DTRSM, XERBLA
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGETRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Determine the block size for this environment.
*
      NB = ILAENV( 1, 'DGETRF', ' ', M, N, -1, -1 )
      IF( NB.LE.1 .OR. NB.GE.MIN( M, N ) ) THEN
*
*        Use unblocked code.
*
         CALL DGETRF2( M, N, A, LDA, IPIV, INFO )
      ELSE
*
*        Use blocked code.
*
         DO 20 J = 1, MIN( M, N ), NB
            JB = MIN( MIN( M, N )-J+1, NB )
*
```

```
*          Factor diagonal and subdiagonal blocks and test for exact
*          singularity.
*
           CALL DGETRF2( M-J+1, JB, A( J, J ), LDA, IPIV( J ), IINFO )
*
*          Adjust INFO and the pivot indices.
*
           IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $        INFO = IINFO + J - 1
           DO 10 I = J, MIN( M, J+JB-1 )
              IPIV( I ) = J - 1 + IPIV( I )
   10      CONTINUE
*
*          Apply interchanges to columns 1:J-1.
*
           CALL DLASWP( J-1, A, LDA, J, J+JB-1, IPIV, 1 )
*
           IF( J+JB.LE.N ) THEN
*
*             Apply interchanges to columns J+JB:N.
*
              CALL DLASWP( N-J-JB+1, A( 1, J+JB ), LDA, J, J+JB-1,
     $                     IPIV, 1 )
*
*             Compute block row of U.
*
              CALL DTRSM( 'Left', 'Lower', 'No transpose', 'Unit', JB,
     $                    N-J-JB+1, ONE, A( J, J ), LDA, A( J, J+JB ),
     $                    LDA )
              IF( J+JB.LE.M ) THEN
*
*                Update trailing submatrix.
*
                 CALL DGEMM( 'No transpose', 'No transpose', M-J-JB+1,
     $                       N-J-JB+1, JB, -ONE, A( J+JB, J ), LDA,
     $                       A( J, J+JB ), LDA, ONE, A( J+JB, J+JB ),
     $                       LDA )
              END IF
           END IF
   20    CONTINUE
      END IF
      RETURN
*
*     End of DGETRF
*
      END
```

———-

32.30.4 lisp code

— dgetrf.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun *main* ()
    (prog ((i 0) (iinfo 0) (n1 0) (n2 0) (sfmin 0.0d0) (temp 0.0d0) (m 0)
           (info 0) (ipiv (make-array * :element-type 'f2cl-lib:integer4))
           (lda 0)
           (a
            (make-array (the fixnum (reduce #'* (list lda *))) :element-type
                        'double-float))
           (n 0) (subroutine 0.0))
      (declare (type (single-float) subroutine)
               (type (array double-float (*)) a)
               (type (array f2cl-lib:integer4 (*)) ipiv)
               (type (double-float) sfmin temp)
               (type (f2cl-lib:integer4) n lda info i iinfo n1 n2 m))
      '"****NOT TRANSLATED: (RECURSIVE SUBROUTINE DGETRF2 (M |,| N |,| A |,| LDA |,| IPIV |,| IN
      (setf info 0)
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4)))
      (cond
       ((/= info 0) (xerbla "DGETRF2" (f2cl-lib:int-sub info)) (go end_label)))
      (if (or (= m 0) (= n 0))
          (go end_label))
      (cond
       ((= m 1) (setf (f2cl-lib:fref ipiv (1) ((1 *))) 1)
        (if (= (f2cl-lib:fref a (1 1) ((1 lda) (1 *))) zero)
            (setf info 1)))
       ((= n 1) (setf sfmin (dlamch "S"))
        (setf i
                (multiple-value-bind (ret-val var-0 var-1 var-2)
                    (idamax m
                     (f2cl-lib:array-slice a double-float (1 1)
                                           ((1 lda) (1 *)))
                     1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf m var-0))
                  ret-val))
        (setf (f2cl-lib:fref ipiv (1) ((1 *))) i)
        (cond
         ((/= (f2cl-lib:fref a (i 1) ((1 lda) (1 *))) zero)
          (cond
           ((/= i 1) (setf temp (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))
```

```
      (setf (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))
              (f2cl-lib:fref a (i 1) ((1 lda) (1 *))))
      (setf (f2cl-lib:fref a (i 1) ((1 lda) (1 *))) temp)))
    (cond
     ((>= (abs (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))) sfmin)
      (dscal (f2cl-lib:int-sub m 1)
       (/ one (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))
       (f2cl-lib:array-slice a double-float (2 1) ((1 lda) (1 *))) 1))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
        (tagbody
          (setf (f2cl-lib:fref a ((f2cl-lib:int-add 1 i) 1)
                                 ((1 lda) (1 *)))
                (/
                 (f2cl-lib:fref a ((f2cl-lib:int-add 1 i) 1)
                                  ((1 lda) (1 *)))
                 (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))))
        label10)))))
  (t (setf info 1))))
(t (setf n1 (the f2cl-lib:integer4 (truncate (min m n) 2)))
 (setf n2 (f2cl-lib:int-sub n n1))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
     (dgetrf2 m n1 a lda ipiv iinfo)
   (declare (ignore var-2 var-4))
   (when var-0 (setf m var-0))
   (when var-1 (setf n1 var-1))
   (when var-3 (setf lda var-3))
   (when var-5 (setf iinfo var-5)))
 (if (and (= info 0) (> iinfo 0))
     (setf info iinfo))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlaswp n2
      (f2cl-lib:array-slice a double-float (1 (f2cl-lib:int-add n1 1))
                            ((1 lda) (1 *)))
      lda 1 n1 ipiv 1)
   (declare (ignore var-1 var-3 var-5 var-6))
   (when var-0 (setf n2 var-0))
   (when var-2 (setf lda var-2))
   (when var-4 (setf n1 var-4)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (dtrsm "L" "L" "N" "U" n1 n2 one a lda
      (f2cl-lib:array-slice a double-float (1 (f2cl-lib:int-add n1 1))
                            ((1 lda) (1 *)))
      lda)
   (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
   (when var-4 (setf n1 var-4))
   (when var-5 (setf n2 var-5))
```

```
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf lda var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (dgemm "N" "N" (f2cl-lib:int-sub m n1) n2 n1 (- one)
     (f2cl-lib:array-slice a double-float ((+ n1 1) 1) ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice a double-float (1 (f2cl-lib:int-add n1 1))
                           ((1 lda) (1 *)))
     lda one
     (f2cl-lib:array-slice a double-float
                           ((+ n1 1) (f2cl-lib:int-add n1 1))
                           ((1 lda) (1 *)))
     lda)
  (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
  (when var-3 (setf n2 var-3))
  (when var-4 (setf n1 var-4))
  (when var-7 (setf lda var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf lda var-12)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (dgetrf2 (f2cl-lib:int-sub m n1) n2
     (f2cl-lib:array-slice a double-float
                           ((+ n1 1) (f2cl-lib:int-add n1 1))
                           ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice ipiv f2cl-lib:integer4 ((+ n1 1)) ((1 *)))
     iinfo)
  (declare (ignore var-0 var-2 var-4))
  (when var-1 (setf n2 var-1))
  (when var-3 (setf lda var-3))
  (when var-5 (setf iinfo var-5)))
(if (and (= info 0) (> iinfo 0))
    (setf info (f2cl-lib:int-add iinfo n1)))
(f2cl-lib:fdo (i (f2cl-lib:int-add n1 1) (f2cl-lib:int-add i 1))
              ((> i
                  (min (the f2cl-lib:integer4 m)
                       (the f2cl-lib:integer4 n)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref ipiv (i) ((1 *)))
            (f2cl-lib:int-add (f2cl-lib:fref ipiv (i) ((1 *))) n1))
   label20))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaswp n1
     (f2cl-lib:array-slice a double-float (1 1) ((1 lda) (1 *))) lda
     (f2cl-lib:int-add n1 1)
```

```
            (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) ipiv 1)
          (declare (ignore var-1 var-3 var-4 var-5 var-6))
          (when var-0 (setf n1 var-0))
          (when var-2 (setf lda var-2)))))
      (go end_label)
     end_label
      (return nil))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::*main*
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types 'nil :return-values 'nil
                                            :calls 'nil)))
```

———-

32.31 dgetrf2

32.31.1 Axiom unit tests

— **dgetrf2.input** —

```
)set break resume
)sys rm -f dgetrf2.output
)spool dgetrf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.31.2 Axiom help page

— **dgetrf2.help** —

```
======================================================================
dgetrf2 examples
```

```
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       recursive subroutine dgetrf2 (M, N, A, LDA, IPIV, INFO)
           DGETRF2

Function/Subroutine Documentation
   recursive subroutine dgetrf2 (integer M, integer N, double precision,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       integer INFO)
       DGETRF2

Purpose:
   DGETRF2 computes an LU factorization of a general M-by-N matrix A
   using partial pivoting with row interchanges.

   The factorization has the form
      A = P * L * U
   where P is a permutation matrix, L is lower triangular with unit
   diagonal elements (lower trapezoidal if m > n), and U is upper
   triangular (upper trapezoidal if m < n).

   This is the recursive version of the algorithm. It divides
   the matrix into four submatrices:

           [  A11 | A12  ]  where A11 is n1 by n1 and A22 is n2 by n2
       A = [ -----|----- ]  with n1 = min(m,n)
           [  A21 | A22  ]       n2 = n-n1

                                           [ A11 ]
   The subroutine calls itself to factor [ --- ],
                                           [ A12 ]
                   [ A12 ]
   do the swaps on [ --- ], solve A12, update A22,
                   [ A22 ]

   then calls itself to factor A22 and do the swaps on A21.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
```

```
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the M-by-N matrix to be factored.
  On exit, the factors L and U from the factorization
  A = P*L*U; the unit diagonal elements of L are not stored.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

IPIV
  IPIV is INTEGER array, dimension (min(M,N))
  The pivot indices; for 1 <= i <= min(M,N), row i of the
  matrix was interchanged with row IPIV(i).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, U(i,i) is exactly zero. The factorization
     has been completed, but the factor U is exactly
     singular, and division by zero will occur if it is used
     to solve a system of equations.
```

32.31.3 fortran code

— dgetrf2.f —

```
*  =====================================================================
      RECURSIVE SUBROUTINE DGETRF2( M, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION   SFMIN, TEMP
      INTEGER            I, IINFO, N1, N2
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      INTEGER            IDAMAX
      EXTERNAL           DLAMCH, IDAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEMM, DSCAL, DLASWP, DTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGETRF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN

      IF ( M.EQ.1 ) THEN
*
*        Use unblocked code for one row case
*        Just need to handle IPIV and INFO
*
```

```
         IPIV( 1 ) = 1
         IF ( A(1,1).EQ.ZERO )
     $      INFO = 1
*
      ELSE IF( N.EQ.1 ) THEN
*
*        Use unblocked code for one column case
*
*
*        Compute machine safe minimum
*
         SFMIN = DLAMCH('S')
*
*        Find pivot and test for singularity
*
         I = IDAMAX( M, A( 1, 1 ), 1 )
         IPIV( 1 ) = I
         IF( A( I, 1 ).NE.ZERO ) THEN
*
*           Apply the interchange
*
            IF( I.NE.1 ) THEN
               TEMP = A( 1, 1 )
               A( 1, 1 ) = A( I, 1 )
               A( I, 1 ) = TEMP
            END IF
*
*           Compute elements 2:M of the column
*
            IF( ABS(A( 1, 1 )) .GE. SFMIN ) THEN
               CALL DSCAL( M-1, ONE / A( 1, 1 ), A( 2, 1 ), 1 )
            ELSE
               DO 10 I = 1, M-1
                  A( 1+I, 1 ) = A( 1+I, 1 ) / A( 1, 1 )
   10          CONTINUE
            END IF
*
         ELSE
            INFO = 1
         END IF
*
      ELSE
*
*        Use recursive code
*
         N1 = MIN( M, N ) / 2
         N2 = N-N1
*
*               [ A11 ]
*        Factor [ --- ]
```

```
*               [ A21 ]
*
         CALL DGETRF2( M, N1, A, LDA, IPIV, IINFO )

         IF ( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO
*
*                             [ A12 ]
*        Apply interchanges to [ --- ]
*                             [ A22 ]
*
         CALL DLASWP( N2, A( 1, N1+1 ), LDA, 1, N1, IPIV, 1 )
*
*        Solve A12
*
         CALL DTRSM( 'L', 'L', 'N', 'U', N1, N2, ONE, A, LDA,
     $               A( 1, N1+1 ), LDA )
*
*        Update A22
*
         CALL DGEMM( 'N', 'N', M-N1, N2, N1, -ONE, A( N1+1, 1 ), LDA,
     $               A( 1, N1+1 ), LDA, ONE, A( N1+1, N1+1 ), LDA )
*
*        Factor A22
*
         CALL DGETRF2( M-N1, N2, A( N1+1, N1+1 ), LDA, IPIV( N1+1 ),
     $                 IINFO )
*
*        Adjust INFO and the pivot indices
*
         IF ( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO + N1
         DO 20 I = N1+1, MIN( M, N )
            IPIV( I ) = IPIV( I ) + N1
   20    CONTINUE
*
*        Apply interchanges to A21
*
         CALL DLASWP( N1, A( 1, 1 ), LDA, N1+1, MIN( M, N), IPIV, 1 )
*
      END IF
      RETURN
*
*     End of DGETRF2
*
      END
```

———-

32.31.4 lisp code

— dgetrf2.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun *main* ()
    (prog ((i 0) (iinfo 0) (n1 0) (n2 0) (sfmin 0.0d0) (temp 0.0d0) (m 0)
           (info 0) (ipiv (make-array * :element-type 'f2cl-lib:integer4))
           (lda 0)
           (a
            (make-array (the fixnum (reduce #'* (list lda *))) :element-type
                        'double-float))
           (n 0) (subroutine 0.0))
      (declare (type (single-float) subroutine)
               (type (array double-float (*)) a)
               (type (array f2cl-lib:integer4 (*)) ipiv)
               (type (double-float) sfmin temp)
               (type (f2cl-lib:integer4) n lda info i iinfo n1 n2 m))
      '"****NOT TRANSLATED: (RECURSIVE SUBROUTINE DGETRF2 (M |,| N |,| A |,| LDA |,| IPIV |,| INFO))"
      (setf info 0)
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4)))
      (cond
       ((/= info 0) (xerbla "DGETRF2" (f2cl-lib:int-sub info)) (go end_label)))
      (if (or (= m 0) (= n 0))
          (go end_label))
      (cond
       ((= m 1) (setf (f2cl-lib:fref ipiv (1) ((1 *))) 1)
        (if (= (f2cl-lib:fref a (1 1) ((1 lda) (1 *))) zero)
            (setf info 1)))
       ((= n 1) (setf sfmin (dlamch "S"))
        (setf i
                (multiple-value-bind (ret-val var-0 var-1 var-2)
                    (idamax m
                     (f2cl-lib:array-slice a double-float (1 1)
                                           ((1 lda) (1 *)))
                     1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf m var-0))
                  ret-val))
        (setf (f2cl-lib:fref ipiv (1) ((1 *))) i)
        (cond
         ((/= (f2cl-lib:fref a (i 1) ((1 lda) (1 *))) zero)
          (cond
           ((/= i 1) (setf temp (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))
```

```
      (setf (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))
              (f2cl-lib:fref a (i 1) ((1 lda) (1 *))))
      (setf (f2cl-lib:fref a (i 1) ((1 lda) (1 *))) temp)))
    (cond
     ((>= (abs (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))) sfmin)
      (dscal (f2cl-lib:int-sub m 1)
       (/ one (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))
       (f2cl-lib:array-slice a double-float (2 1) ((1 lda) (1 *))) 1))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
        (tagbody
          (setf (f2cl-lib:fref a ((f2cl-lib:int-add 1 i) 1)
                                 ((1 lda) (1 *)))
                (/
                 (f2cl-lib:fref a ((f2cl-lib:int-add 1 i) 1)
                                  ((1 lda) (1 *)))
                 (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))))
         label10)))))
   (t (setf info 1))))
  (t (setf n1 (the f2cl-lib:integer4 (truncate (min m n) 2)))
   (setf n2 (f2cl-lib:int-sub n n1))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
       (dgetrf2 m n1 a lda ipiv iinfo)
     (declare (ignore var-2 var-4))
     (when var-0 (setf m var-0))
     (when var-1 (setf n1 var-1))
     (when var-3 (setf lda var-3))
     (when var-5 (setf iinfo var-5)))
   (if (and (= info 0) (> iinfo 0))
       (setf info iinfo))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlaswp n2
        (f2cl-lib:array-slice a double-float (1 (f2cl-lib:int-add n1 1))
                              ((1 lda) (1 *)))
        lda 1 n1 ipiv 1)
     (declare (ignore var-1 var-3 var-5 var-6))
     (when var-0 (setf n2 var-0))
     (when var-2 (setf lda var-2))
     (when var-4 (setf n1 var-4)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (dtrsm "L" "L" "N" "U" n1 n2 one a lda
        (f2cl-lib:array-slice a double-float (1 (f2cl-lib:int-add n1 1))
                              ((1 lda) (1 *)))
        lda)
     (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
     (when var-4 (setf n1 var-4))
     (when var-5 (setf n2 var-5))
```

```
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf lda var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (dgemm "N" "N" (f2cl-lib:int-sub m n1) n2 n1 (- one)
     (f2cl-lib:array-slice a double-float ((+ n1 1) 1) ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice a double-float (1 (f2cl-lib:int-add n1 1))
                           ((1 lda) (1 *)))
     lda one
     (f2cl-lib:array-slice a double-float
                           ((+ n1 1) (f2cl-lib:int-add n1 1))
                           ((1 lda) (1 *)))
     lda)
  (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
  (when var-3 (setf n2 var-3))
  (when var-4 (setf n1 var-4))
  (when var-7 (setf lda var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf lda var-12)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (dgetrf2 (f2cl-lib:int-sub m n1) n2
     (f2cl-lib:array-slice a double-float
                           ((+ n1 1) (f2cl-lib:int-add n1 1))
                           ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice ipiv f2cl-lib:integer4 ((+ n1 1)) ((1 *)))
     iinfo)
  (declare (ignore var-0 var-2 var-4))
  (when var-1 (setf n2 var-1))
  (when var-3 (setf lda var-3))
  (when var-5 (setf iinfo var-5)))
(if (and (= info 0) (> iinfo 0))
    (setf info (f2cl-lib:int-add iinfo n1)))
(f2cl-lib:fdo (i (f2cl-lib:int-add n1 1) (f2cl-lib:int-add i 1))
              ((> i
                  (min (the f2cl-lib:integer4 m)
                       (the f2cl-lib:integer4 n)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref ipiv (i) ((1 *)))
            (f2cl-lib:int-add (f2cl-lib:fref ipiv (i) ((1 *))) n1))
   label20))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaswp n1
     (f2cl-lib:array-slice a double-float (1 1) ((1 lda) (1 *))) lda
     (f2cl-lib:int-add n1 1)
```

```
            (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) ipiv 1)
          (declare (ignore var-1 var-3 var-4 var-5 var-6))
          (when var-0 (setf n1 var-0))
          (when var-2 (setf lda var-2)))))
      (go end_label)
     end_label
      (return nil))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::*main*
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types 'nil :return-values 'nil
                                            :calls 'nil)))
```

32.32 dgetri

32.32.1 Axiom unit tests

— dgetri.input —

```
)set break resume
)sys rm -f dgetri.output
)spool dgetri.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

32.32.2 Axiom help page

— dgetri.help —

```
======================================================================
dgetri examples
```

```
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgetri (N, A, LDA, IPIV, WORK, LWORK, INFO)
           DGETRI

Function/Subroutine Documentation
   subroutine dgetri (integer N, double precision, dimension( lda, * ) A,
       integer LDA, integer, dimension( * ) IPIV, double precision, dimension(
       * ) WORK, integer LWORK, integer INFO)
       DGETRI

Purpose:
   DGETRI computes the inverse of a matrix using the LU factorization
   computed by DGETRF.

   This method inverts U and then computes inv(A) by solving the system
   inv(A)*L = inv(U) for inv(A).

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the factors L and U from the factorization
   A = P*L*U as computed by DGETRF.
   On exit, if INFO = 0, the inverse of the original matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   The pivot indices from DGETRF; for 1<=i<=N, row i of the
   matrix was interchanged with row IPIV(i).

 WORK
   WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
   On exit, if INFO=0, then WORK(1) returns the optimal LWORK.

 LWORK
```

```
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,N).
  For optimal performance LWORK >= N*NB, where NB is
  the optimal blocksize returned by ILAENV.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, U(i,i) is exactly zero; the matrix is
     singular and its inverse could not be computed.
```

32.32.3 fortran code

— dgetri.f —

```
*  =====================================================================
      SUBROUTINE DGETRI( N, A, LDA, IPIV, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
```

```
      INTEGER            I, IWS, J, JB, JJ, JP, LDWORK, LWKOPT, NB,
     $                   NBMIN, NN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEMM, DGEMV, DSWAP, DTRSM, DTRTRI, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NB = ILAENV( 1, 'DGETRI', ' ', N, -1, -1, -1 )
      LWKOPT = N*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -3
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGETRI', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Form inv(U).  If INFO > 0 from DTRTRI, then U is singular,
*     and the inverse is not computed.
*
      CALL DTRTRI( 'Upper', 'Non-unit', N, A, LDA, INFO )
      IF( INFO.GT.0 )
     $   RETURN
*
      NBMIN = 2
      LDWORK = N
```

```
      IF( NB.GT.1 .AND. NB.LT.N ) THEN
         IWS = MAX( LDWORK*NB, 1 )
         IF( LWORK.LT.IWS ) THEN
            NB = LWORK / LDWORK
            NBMIN = MAX( 2, ILAENV( 2, 'DGETRI', ' ', N, -1, -1, -1 ) )
         END IF
      ELSE
         IWS = N
      END IF
*
*     Solve the equation inv(A)*L = inv(U) for inv(A).
*
      IF( NB.LT.NBMIN .OR. NB.GE.N ) THEN
*
*        Use unblocked code.
*
         DO 20 J = N, 1, -1
*
*           Copy current column of L to WORK and replace with zeros.
*
            DO 10 I = J + 1, N
               WORK( I ) = A( I, J )
               A( I, J ) = ZERO
   10       CONTINUE
*
*           Compute current column of inv(A).
*
            IF( J.LT.N )
     $         CALL DGEMV( 'No transpose', N, N-J, -ONE, A( 1, J+1 ),
     $                     LDA, WORK( J+1 ), 1, ONE, A( 1, J ), 1 )
   20    CONTINUE
      ELSE
*
*        Use blocked code.
*
         NN = ( ( N-1 ) / NB )*NB + 1
         DO 50 J = NN, 1, -NB
            JB = MIN( NB, N-J+1 )
*
*           Copy current block column of L to WORK and replace with
*           zeros.
*
            DO 40 JJ = J, J + JB - 1
               DO 30 I = JJ + 1, N
                  WORK( I+( JJ-J )*LDWORK ) = A( I, JJ )
                  A( I, JJ ) = ZERO
   30          CONTINUE
   40       CONTINUE
*
*           Compute current block column of inv(A).
```

```
*
            IF( J+JB.LE.N )
     $         CALL DGEMM( 'No transpose', 'No transpose', N, JB,
     $                     N-J-JB+1, -ONE, A( 1, J+JB ), LDA,
     $                     WORK( J+JB ), LDWORK, ONE, A( 1, J ), LDA )
            CALL DTRSM( 'Right', 'Lower', 'No transpose', 'Unit', N, JB,
     $                  ONE, WORK( J ), LDWORK, A( 1, J ), LDA )
   50    CONTINUE
      END IF
*
*     Apply column interchanges.
*
      DO 60 J = N - 1, 1, -1
         JP = IPIV( J )
         IF( JP.NE.J )
     $      CALL DSWAP( N, A( 1, J ), 1, A( 1, JP ), 1 )
   60 CONTINUE
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of DGETRI
*
      END
```

———-

32.32.4 lisp code

— dgetri.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dgetri (n a lda ipiv work lwork info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array double-float (*)) work a)
             (type (f2cl-lib:integer4) info lwork lda n))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (work double-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((i 0) (iws 0) (j 0) (jb 0) (jj 0) (jp 0) (ldwork 0) (lwkopt 0)
             (nb 0) (nbmin 0) (nn 0) (lquery nil))
        (declare
         (type (f2cl-lib:integer4) i iws j jb jj jp ldwork lwkopt nb nbmin nn)
         (type f2cl-lib:logical lquery))
```

```
(setf info 0)
(setf nb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 1 "DGETRI" " " n -1 -1 -1)
          (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6))
          (when var-3 (setf n var-3))
          ret-val))
(setf lwkopt (f2cl-lib:int-mul n nb))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((< n 0) (setf info -1))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -3))
      ((and
        (< lwork
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (not lquery))
       (setf info -6)))
(cond
 ((/= info 0) (xerbla "DGETRI" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (dtrtri "Upper" "Non-unit" n a lda info)
  (declare (ignore var-0 var-1 var-3))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-5 (setf info var-5)))
(if (> info 0)
    (go end_label))
(setf nbmin 2)
(setf ldwork n)
(cond
 ((and (> nb 1) (< nb n))
  (setf iws
          (max (the f2cl-lib:integer4 (f2cl-lib:int-mul ldwork nb))
               (the f2cl-lib:integer4 1)))
  (cond
   ((< lwork iws)
    (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
    (setf nbmin
            (max (the f2cl-lib:integer4 2)
                 (the f2cl-lib:integer4
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
```

```
                          (ilaenv 2 "DGETRI" " " n -1 -1 -1)
                        (declare
                         (ignore var-0 var-1 var-2 var-4 var-5 var-6))
                        (when var-3 (setf n var-3))
                        ret-val)))))))
 (t (setf iws n)))
(cond
 ((or (< nb nbmin) (>= nb n))
  (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                ((> j 1) nil)
    (tagbody
      (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%))
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%)
                  zero)
         label10))
      (if (< j n)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10)
              (dgemv "No transpose" n (f2cl-lib:int-sub n j) (- one)
               (f2cl-lib:array-slice a-%data% double-float
                                     (1 (f2cl-lib:int-add j 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice work-%data% double-float ((+ j 1))
                                     ((1 *)) work-%offset%)
               1 one
               (f2cl-lib:array-slice a-%data% double-float (1 j)
                                     ((1 lda) (1 *)) a-%offset%)
               1)
            (declare
             (ignore var-0 var-2 var-3 var-4 var-6 var-7 var-9 var-10))
            (when var-1 (setf n var-1))
            (when var-5 (setf lda var-5))
            (when var-8 (setf one var-8))))
     label20)))
 (t
  (setf nn (+ (* (the f2cl-lib:integer4 (truncate (- n 1) nb)) nb) 1))
  (f2cl-lib:fdo (j nn (f2cl-lib:int-add j (f2cl-lib:int-sub nb)))
                ((> j 1) nil)
    (tagbody
      (setf jb
              (min (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
```

```
                    (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1))))
(f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
              ((> jj
                  (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (f2cl-lib:fdo (i (f2cl-lib:int-add jj 1)
                   (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add i
                                                (f2cl-lib:int-mul
                                                 (f2cl-lib:int-sub
                                                  jj j)
                                                 ldwork)))
                             ((1 *)) work-%offset%)
              (f2cl-lib:fref a-%data% (i jj) ((1 lda) (1 *))
                             a-%offset%))
        (setf (f2cl-lib:fref a-%data% (i jj) ((1 lda) (1 *))
                             a-%offset%)
              zero)
       label30))
   label40))
(if (<= (f2cl-lib:int-add j jb) n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (dgemm "No transpose" "No transpose" n jb
         (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) (- one)
         (f2cl-lib:array-slice a-%data% double-float
                               (1 (f2cl-lib:int-add j jb))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice work-%data% double-float
                               ((+ j jb)) ((1 *)) work-%offset%)
         ldwork one
         (f2cl-lib:array-slice a-%data% double-float (1 j)
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare
       (ignore var-0 var-1 var-4 var-5 var-6 var-8 var-11))
      (when var-2 (setf n var-2))
      (when var-3 (setf jb var-3))
      (when var-7 (setf lda var-7))
      (when var-9 (setf ldwork var-9))
      (when var-10 (setf one var-10))
      (when var-12 (setf lda var-12))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
                  var-10)
                 (dtrsm "Right" "Lower" "No transpose" "Unit" n jb one
                  (f2cl-lib:array-slice work-%data% double-float (j) ((1 *))
                                        work-%offset%)
                  ldwork
                  (f2cl-lib:array-slice a-%data% double-float (1 j)
                                        ((1 lda) (1 *)) a-%offset%)
                  lda)
                (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
                (when var-4 (setf n var-4))
                (when var-5 (setf jb var-5))
                (when var-6 (setf one var-6))
                (when var-8 (setf ldwork var-8))
                (when var-10 (setf lda var-10)))
             label50))))
        (f2cl-lib:fdo (j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                       (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                      ((> j 1) nil)
          (tagbody
            (setf jp (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
            (if (/= jp j)
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (dswap n
                     (f2cl-lib:array-slice a-%data% double-float (1 j)
                                           ((1 lda) (1 *)) a-%offset%)
                     1
                     (f2cl-lib:array-slice a-%data% double-float (1 jp)
                                           ((1 lda) (1 *)) a-%offset%)
                     1)
                  (declare (ignore var-1 var-2 var-3 var-4))
                  (when var-0 (setf n var-0))))
           label60))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 iws) 'double-float))
        (go end_label)
       end_label
        (return (values n nil lda nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgetri
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
```

```
                                                  (fortran-to-lisp::integer4)
                                                  (fortran-to-lisp::integer4))
                                                :return-values
                                                '(fortran-to-lisp::n nil
                                                  fortran-to-lisp::lda nil nil nil
                                                  fortran-to-lisp::info)
                                                :calls 'nil)))
```

———-

32.33 dgetrs

32.33.1 Axiom unit tests

— **dgetrs.input** —

```
)set break resume
)sys rm -f dgetrs.output
)spool dgetrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.33.2 Axiom help page

— **dgetrs.help** —

```
======================================================================
dgetrs examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
```

```
      subroutine dgetrs (TRANS, N, NRHS, A, LDA, IPIV, B, LDB, INFO)
          DGETRS

Function/Subroutine Documentation
   subroutine dgetrs (character TRANS, integer N, integer NRHS, double
       precision, dimension( lda, * ) A, integer LDA, integer, dimension( * )
       IPIV, double precision, dimension( ldb, * ) B, integer LDB, integer
       INFO)
       DGETRS

Purpose:
   DGETRS solves a system of linear equations
      A * X = B  or  A**T * X = B
   with a general N-by-N matrix A using the LU factorization computed
   by DGETRF.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B  (No transpose)
   = 'T':  A**T* X = B  (Transpose)
   = 'C':  A**T* X = B  (Conjugate transpose = Transpose)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   The factors L and U from the factorization A = P*L*U
   as computed by DGETRF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   The pivot indices from DGETRF; for 1<=i<=N, row i of the
   matrix was interchanged with row IPIV(i).

 B
   B is DOUBLE PRECISION array, dimension (LDB,NRHS)
```

```
  On entry, the right hand side matrix B.
  On exit, the solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

32.33.3 fortran code

— **dgetrs.f** —

```
*  =====================================================================
      SUBROUTINE DGETRS( TRANS, N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
```

```
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASWP, DTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGETRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( NOTRAN ) THEN
*
*        Solve A * X = B.
*
*        Apply row interchanges to the right hand sides.
*
         CALL DLASWP( NRHS, B, LDB, 1, N, IPIV, 1 )
*
*        Solve L*X = B, overwriting B with X.
*
         CALL DTRSM( 'Left', 'Lower', 'No transpose', 'Unit', N, NRHS,
     $               ONE, A, LDA, B, LDB )
*
*        Solve U*X = B, overwriting B with X.
*
```

```
         CALL DTRSM( 'Left', 'Upper', 'No transpose', 'Non-unit', N,
     $               NRHS, ONE, A, LDA, B, LDB )
      ELSE
*
*        Solve A**T * X = B.
*
*        Solve U**T *X = B, overwriting B with X.
*
         CALL DTRSM( 'Left', 'Upper', 'Transpose', 'Non-unit', N, NRHS,
     $               ONE, A, LDA, B, LDB )
*
*        Solve L**T *X = B, overwriting B with X.
*
         CALL DTRSM( 'Left', 'Lower', 'Transpose', 'Unit', N, NRHS, ONE,
     $               A, LDA, B, LDB )
*
*        Apply row interchanges to the solution vectors.
*
         CALL DLASWP( NRHS, B, LDB, 1, N, IPIV, -1 )
      END IF
*
      RETURN
*
*     End of DGETRS
*
      END
```

———-

32.33.4 lisp code

— dgetrs.lisp —

———-

32.34 dhgeqz

32.34.1 Axiom unit tests

— dhgeqz.input —

```
)set break resume
```

```
)sys rm -f dhgeqz.output
)spool dhgeqz.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

32.34.2 Axiom help page [Mole71]

— dhgeqz.help —

```
==================================================================
dhgeqz examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dhgeqz (JOB, COMPQ, COMPZ, N, ILO, IHI, H, LDH, T, LDT,
           ALPHAR, ALPHAI, BETA, Q, LDQ, Z, LDZ, WORK, LWORK, INFO)
           DHGEQZ

Function/Subroutine Documentation
   subroutine dhgeqz (character JOB, character COMPQ, character COMPZ, integer
       N, integer ILO, integer IHI, double precision, dimension( ldh, * ) H,
       integer LDH, double precision, dimension( ldt, * ) T, integer LDT,
       double precision, dimension( * ) ALPHAR, double precision, dimension( *
       ) ALPHAI, double precision, dimension( * ) BETA, double precision,
       dimension( ldq, * ) Q, integer LDQ, double precision, dimension( ldz, *
       ) Z, integer LDZ, double precision, dimension( * ) WORK, integer LWORK,
       integer INFO)
       DHGEQZ

Purpose:
   DHGEQZ computes the eigenvalues of a real matrix pair (H,T),
   where H is an upper Hessenberg matrix and T is upper triangular,
   using the double-shift QZ method.
   Matrix pairs of this type are produced by the reduction to
   generalized upper Hessenberg form of a real matrix pair (A,B):

   A = Q1*H*Z1**T,  B = Q1*T*Z1**T,
```

```
   as computed by DGGHRD.

   If JOB='S', then the Hessenberg-triangular pair (H,T) is
   also reduced to generalized Schur form,

   H = Q*S*Z**T,  T = Q*P*Z**T,

   where Q and Z are orthogonal matrices, P is an upper triangular
   matrix, and S is a quasi-triangular matrix with 1-by-1 and 2-by-2
   diagonal blocks.

   The 1-by-1 blocks correspond to real eigenvalues of the matrix pair
   (H,T) and the 2-by-2 blocks correspond to complex conjugate pairs of
   eigenvalues.

   Additionally, the 2-by-2 upper triangular diagonal blocks of P
   corresponding to 2-by-2 blocks of S are reduced to positive diagonal
   form, i.e., if S(j+1,j) is non-zero, then P(j+1,j) = P(j,j+1) = 0,
   P(j,j) > 0, and P(j+1,j+1) > 0.

   Optionally, the orthogonal matrix Q from the generalized Schur
   factorization may be postmultiplied into an input matrix Q1, and the
   orthogonal matrix Z may be postmultiplied into an input matrix Z1.
   If Q1 and Z1 are the orthogonal matrices from DGGHRD that reduced
   the matrix pair (A,B) to generalized upper Hessenberg form, then the
   output matrices Q1*Q and Z1*Z are the orthogonal factors from the
   generalized Schur factorization of (A,B):

   A = (Q1*Q)*S*(Z1*Z)**T,  B = (Q1*Q)*P*(Z1*Z)**T.

   To avoid overflow, eigenvalues of the matrix pair (H,T) (equivalently,
   of (A,B)) are computed as a pair of values (alpha,beta), where alpha is
   complex and beta real.
   If beta is nonzero, lambda = alpha / beta is an eigenvalue of the
   generalized nonsymmetric eigenvalue problem (GNEP)
        A*x = lambda*B*x
   and if alpha is nonzero, mu = beta / alpha is an eigenvalue of the
   alternate form of the GNEP
        mu*A*y = B*y.
   Real eigenvalues can be read directly from the generalized Schur
   form:
        alpha = S(i,i), beta = P(i,i).

Parameters:

 JOB
   JOB is CHARACTER*1
   = 'E': Compute eigenvalues only;
   = 'S': Compute eigenvalues and the Schur form.
```

```
COMPQ
  COMPQ is CHARACTER*1
  = 'N': Left Schur vectors (Q) are not computed;
  = 'I': Q is initialized to the unit matrix and the matrix Q
     of left Schur vectors of (H,T) is returned;
  = 'V': Q must contain an orthogonal matrix Q1 on entry and
     the product Q1*Q is returned.

COMPZ
  COMPZ is CHARACTER*1
  = 'N': Right Schur vectors (Z) are not computed;
  = 'I': Z is initialized to the unit matrix and the matrix Z
         of right Schur vectors of (H,T) is returned;
  = 'V': Z must contain an orthogonal matrix Z1 on entry and
         the product Z1*Z is returned.

N
  N is INTEGER
  The order of the matrices H, T, Q, and Z.  N >= 0.

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER
  ILO and IHI mark the rows and columns of H which are in
  Hessenberg form.  It is assumed that A is already upper
  triangular in rows and columns 1:ILO-1 and IHI+1:N.
  If N > 0, 1 <= ILO <= IHI <= N; if N = 0, ILO=1 and IHI=0.

H
  H is DOUBLE PRECISION array, dimension (LDH, N)
  On entry, the N-by-N upper Hessenberg matrix H.
  On exit, if JOB = 'S', H contains the upper quasi-triangular
  matrix S from the generalized Schur factorization.
  If JOB = 'E', the diagonal blocks of H match those of S, but
  the rest of H is unspecified.

LDH
  LDH is INTEGER
  The leading dimension of the array H.  LDH >= max( 1, N ).

T
  T is DOUBLE PRECISION array, dimension (LDT, N)
  On entry, the N-by-N upper triangular matrix T.
  On exit, if JOB = 'S', T contains the upper triangular
  matrix P from the generalized Schur factorization;
  2-by-2 diagonal blocks of P corresponding to 2-by-2 blocks of S
  are reduced to positive diagonal form, i.e., if H(j+1,j) is
```

```
  non-zero, then T(j+1,j) = T(j,j+1) = 0, T(j,j) > 0, and
  T(j+1,j+1) > 0.
  If JOB = 'E', the diagonal blocks of T match those of P, but
  the rest of T is unspecified.

LDT
  LDT is INTEGER
  The leading dimension of the array T.  LDT >= max( 1, N ).

ALPHAR
  ALPHAR is DOUBLE PRECISION array, dimension (N)
  The real parts of each scalar alpha defining an eigenvalue of GNEP.

ALPHAI
  ALPHAI is DOUBLE PRECISION array, dimension (N)
  The imaginary parts of each scalar alpha defining an
  eigenvalue of GNEP.
  If ALPHAI(j) is zero, then the j-th eigenvalue is real; if
  positive, then the j-th and (j+1)-st eigenvalues are a
  complex conjugate pair, with ALPHAI(j+1) = -ALPHAI(j).

BETA
  BETA is DOUBLE PRECISION array, dimension (N)
  The scalars beta that define the eigenvalues of GNEP.
  Together, the quantities alpha = (ALPHAR(j),ALPHAI(j)) and
  beta = BETA(j) represent the j-th eigenvalue of the matrix
  pair (A,B), in one of the forms lambda = alpha/beta or
  mu = beta/alpha.  Since either lambda or mu may overflow,
  they should not, in general, be computed.

Q
  Q is DOUBLE PRECISION array, dimension (LDQ, N)
  On entry, if COMPZ = 'V', the orthogonal matrix Q1 used in
  the reduction of (A,B) to generalized Hessenberg form.
  On exit, if COMPZ = 'I', the orthogonal matrix of left Schur
  vectors of (H,T), and if COMPZ = 'V', the orthogonal matrix
  of left Schur vectors of (A,B).
  Not referenced if COMPZ = 'N'.

LDQ
  LDQ is INTEGER
  The leading dimension of the array Q.  LDQ >= 1.
  If COMPQ='V' or 'I', then LDQ >= N.

Z
  Z is DOUBLE PRECISION array, dimension (LDZ, N)
  On entry, if COMPZ = 'V', the orthogonal matrix Z1 used in
  the reduction of (A,B) to generalized Hessenberg form.
  On exit, if COMPZ = 'I', the orthogonal matrix of
  right Schur vectors of (H,T), and if COMPZ = 'V', the
```

```
  orthogonal matrix of right Schur vectors of (A,B).
  Not referenced if COMPZ = 'N'.

 LDZ
  LDZ is INTEGER
  The leading dimension of the array Z.  LDZ >= 1.
  If COMPZ='V' or 'I', then LDZ >= N.

 WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO >= 0, WORK(1) returns the optimal LWORK.

 LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,N).

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

 INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  = 1,...,N: the QZ iteration did not converge.  (H,T) is not
             in Schur form, but ALPHAR(i), ALPHAI(i), and
             BETA(i), i=INFO+1,...,N should be correct.
  = N+1,...,2*N: the shift calculation failed.  (H,T) is not
             in Schur form, but ALPHAR(i), ALPHAI(i), and
             BETA(i), i=INFO-N+1,...,N should be correct.

Further Details:

 Iteration counters:

 JITER  -- counts iterations.
 IITER  -- counts iterations run since ILAST was last
           changed.  This is therefore reset only when a 1-by-1 or
           2-by-2 block deflates off the bottom.
```

———-

32.34.3 fortran code

— dhgeqz.f —

```
*  =====================================================================
      SUBROUTINE DHGEQZ( JOB, COMPQ, COMPZ, N, ILO, IHI, H, LDH, T, LDT,
     $                   ALPHAR, ALPHAI, BETA, Q, LDQ, Z, LDZ, WORK,
     $                   LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER          COMPQ, COMPZ, JOB
      INTEGER            IHI, ILO, INFO, LDH, LDQ, LDT, LDZ, LWORK, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   ALPHAI( * ), ALPHAR( * ), BETA( * ),
     $                   H( LDH, * ), Q( LDQ, * ), T( LDT, * ),
     $                   WORK( * ), Z( LDZ, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
*    $                     SAFETY = 1.0E+0 )
      DOUBLE PRECISION   HALF, ZERO, ONE, SAFETY
      PARAMETER          ( HALF = 0.5D+0, ZERO = 0.0D+0, ONE = 1.0D+0,
     $                   SAFETY = 1.0D+2 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ILAZR2, ILAZRO, ILPIVT, ILQ, ILSCHR, ILZ,
     $                   LQUERY
      INTEGER            ICOMPQ, ICOMPZ, IFIRST, IFRSTM, IITER, ILAST,
     $                   ILASTM, IN, ISCHUR, ISTART, J, JC, JCH, JITER,
     $                   JR, MAXIT
      DOUBLE PRECISION   A11, A12, A1I, A1R, A21, A22, A2I, A2R, AD11,
     $                   AD11L, AD12, AD12L, AD21, AD21L, AD22, AD22L,
     $                   AD32L, AN, ANORM, ASCALE, ATOL, B11, B1A, B1I,
     $                   B1R, B22, B2A, B2I, B2R, BN, BNORM, BSCALE,
     $                   BTOL, C, C11I, C11R, C12, C21, C22I, C22R, CL,
     $                   CQ, CR, CZ, ESHIFT, S, S1, S1INV, S2, SAFMAX,
     $                   SAFMIN, SCALE, SL, SQI, SQR, SR, SZI, SZR, T1,
     $                   TAU, TEMP, TEMP2, TEMPI, TEMPR, U1, U12, U12L,
     $                   U2, ULP, VS, W11, W12, W21, W22, WABS, WI, WR,
     $                   WR2
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   V( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
```

```
      DOUBLE PRECISION   DLAMCH, DLANHS, DLAPY2, DLAPY3
      EXTERNAL           LSAME, DLAMCH, DLANHS, DLAPY2, DLAPY3
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLAG2, DLARFG, DLARTG, DLASET, DLASV2, DROT,
     $                   XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode JOB, COMPQ, COMPZ
*
      IF( LSAME( JOB, 'E' ) ) THEN
         ILSCHR = .FALSE.
         ISCHUR = 1
      ELSE IF( LSAME( JOB, 'S' ) ) THEN
         ILSCHR = .TRUE.
         ISCHUR = 2
      ELSE
         ISCHUR = 0
      END IF
*
      IF( LSAME( COMPQ, 'N' ) ) THEN
         ILQ = .FALSE.
         ICOMPQ = 1
      ELSE IF( LSAME( COMPQ, 'V' ) ) THEN
         ILQ = .TRUE.
         ICOMPQ = 2
      ELSE IF( LSAME( COMPQ, 'I' ) ) THEN
         ILQ = .TRUE.
         ICOMPQ = 3
      ELSE
         ICOMPQ = 0
      END IF
*
      IF( LSAME( COMPZ, 'N' ) ) THEN
         ILZ = .FALSE.
         ICOMPZ = 1
      ELSE IF( LSAME( COMPZ, 'V' ) ) THEN
         ILZ = .TRUE.
         ICOMPZ = 2
      ELSE IF( LSAME( COMPZ, 'I' ) ) THEN
         ILZ = .TRUE.
         ICOMPZ = 3
      ELSE
         ICOMPZ = 0
      END IF
*
```

```
*     Check Argument Values
*
      INFO = 0
      WORK( 1 ) = MAX( 1, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( ISCHUR.EQ.0 ) THEN
         INFO = -1
      ELSE IF( ICOMPQ.EQ.0 ) THEN
         INFO = -2
      ELSE IF( ICOMPZ.EQ.0 ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( ILO.LT.1 ) THEN
         INFO = -5
      ELSE IF( IHI.GT.N .OR. IHI.LT.ILO-1 ) THEN
         INFO = -6
      ELSE IF( LDH.LT.N ) THEN
         INFO = -8
      ELSE IF( LDT.LT.N ) THEN
         INFO = -10
      ELSE IF( LDQ.LT.1 .OR. ( ILQ .AND. LDQ.LT.N ) ) THEN
         INFO = -15
      ELSE IF( LDZ.LT.1 .OR. ( ILZ .AND. LDZ.LT.N ) ) THEN
         INFO = -17
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -19
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DHGEQZ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.LE.0 ) THEN
         WORK( 1 ) = DBLE( 1 )
         RETURN
      END IF
*
*     Initialize Q and Z
*
      IF( ICOMPQ.EQ.3 )
     $   CALL DLASET( 'Full', N, N, ZERO, ONE, Q, LDQ )
      IF( ICOMPZ.EQ.3 )
     $   CALL DLASET( 'Full', N, N, ZERO, ONE, Z, LDZ )
*
*     Machine Constants
```

```
*
      IN = IHI + 1 - ILO
      SAFMIN = DLAMCH( 'S' )
      SAFMAX = ONE / SAFMIN
      ULP = DLAMCH( 'E' )*DLAMCH( 'B' )
      ANORM = DLANHS( 'F', IN, H( ILO, ILO ), LDH, WORK )
      BNORM = DLANHS( 'F', IN, T( ILO, ILO ), LDT, WORK )
      ATOL = MAX( SAFMIN, ULP*ANORM )
      BTOL = MAX( SAFMIN, ULP*BNORM )
      ASCALE = ONE / MAX( SAFMIN, ANORM )
      BSCALE = ONE / MAX( SAFMIN, BNORM )
*
*     Set Eigenvalues IHI+1:N
*
      DO 30 J = IHI + 1, N
         IF( T( J, J ).LT.ZERO ) THEN
            IF( ILSCHR ) THEN
               DO 10 JR = 1, J
                  H( JR, J ) = -H( JR, J )
                  T( JR, J ) = -T( JR, J )
   10          CONTINUE
            ELSE
               H( J, J ) = -H( J, J )
               T( J, J ) = -T( J, J )
            END IF
            IF( ILZ ) THEN
               DO 20 JR = 1, N
                  Z( JR, J ) = -Z( JR, J )
   20          CONTINUE
            END IF
         END IF
         ALPHAR( J ) = H( J, J )
         ALPHAI( J ) = ZERO
         BETA( J ) = T( J, J )
   30 CONTINUE
*
*     If IHI < ILO, skip QZ steps
*
      IF( IHI.LT.ILO )
     $   GO TO 380
*
*     MAIN QZ ITERATION LOOP
*
*     Initialize dynamic indices
*
*     Eigenvalues ILAST+1:N have been found.
*        Column operations modify rows IFRSTM:whatever.
*        Row operations modify columns whatever:ILASTM.
*
*     If only eigenvalues are being computed, then
```

```
*        IFRSTM is the row of the last splitting row above row ILAST;
*        this is always at least ILO.
*     IITER counts iterations since the last eigenvalue was found,
*        to tell when to use an extraordinary shift.
*     MAXIT is the maximum number of QZ sweeps allowed.
*
      ILAST = IHI
      IF( ILSCHR ) THEN
         IFRSTM = 1
         ILASTM = N
      ELSE
         IFRSTM = ILO
         ILASTM = IHI
      END IF
      IITER = 0
      ESHIFT = ZERO
      MAXIT = 30*( IHI-ILO+1 )
*
      DO 360 JITER = 1, MAXIT
*
*        Split the matrix if possible.
*
*        Two tests:
*           1: H(j,j-1)=0  or  j=ILO
*           2: T(j,j)=0
*
         IF( ILAST.EQ.ILO ) THEN
*
*           Special case: j=ILAST
*
            GO TO 80
         ELSE
            IF( ABS( H( ILAST, ILAST-1 ) ).LE.ATOL ) THEN
               H( ILAST, ILAST-1 ) = ZERO
               GO TO 80
            END IF
         END IF
*
         IF( ABS( T( ILAST, ILAST ) ).LE.BTOL ) THEN
            T( ILAST, ILAST ) = ZERO
            GO TO 70
         END IF
*
*        General case: j<ILAST
*
         DO 60 J = ILAST - 1, ILO, -1
*
*           Test 1: for H(j,j-1)=0 or j=ILO
*
            IF( J.EQ.ILO ) THEN
```

```
               ILAZRO = .TRUE.
            ELSE
               IF( ABS( H( J, J-1 ) ).LE.ATOL ) THEN
                  H( J, J-1 ) = ZERO
                  ILAZRO = .TRUE.
               ELSE
                  ILAZRO = .FALSE.
               END IF
            END IF
*
*           Test 2: for T(j,j)=0
*
            IF( ABS( T( J, J ) ).LT.BTOL ) THEN
               T( J, J ) = ZERO
*
*              Test 1a: Check for 2 consecutive small subdiagonals in A
*
               ILAZR2 = .FALSE.
               IF( .NOT.ILAZRO ) THEN
                  TEMP = ABS( H( J, J-1 ) )
                  TEMP2 = ABS( H( J, J ) )
                  TEMPR = MAX( TEMP, TEMP2 )
                  IF( TEMPR.LT.ONE .AND. TEMPR.NE.ZERO ) THEN
                     TEMP = TEMP / TEMPR
                     TEMP2 = TEMP2 / TEMPR
                  END IF
                  IF( TEMP*( ASCALE*ABS( H( J+1, J ) ) ).LE.TEMP2*
     $                ( ASCALE*ATOL ) )ILAZR2 = .TRUE.
               END IF
*
*              If both tests pass (1 & 2), i.e., the leading diagonal
*              element of B in the block is zero, split a 1x1 block off
*              at the top. (I.e., at the J-th row/column) The leading
*              diagonal element of the remainder can also be zero, so
*              this may have to be done repeatedly.
*
               IF( ILAZRO .OR. ILAZR2 ) THEN
                  DO 40 JCH = J, ILAST - 1
                     TEMP = H( JCH, JCH )
                     CALL DLARTG( TEMP, H( JCH+1, JCH ), C, S,
     $                            H( JCH, JCH ) )
                     H( JCH+1, JCH ) = ZERO
                     CALL DROT( ILASTM-JCH, H( JCH, JCH+1 ), LDH,
     $                          H( JCH+1, JCH+1 ), LDH, C, S )
                     CALL DROT( ILASTM-JCH, T( JCH, JCH+1 ), LDT,
     $                          T( JCH+1, JCH+1 ), LDT, C, S )
                     IF( ILQ )
     $                  CALL DROT( N, Q( 1, JCH ), 1, Q( 1, JCH+1 ), 1,
     $                             C, S )
                     IF( ILAZR2 )
```

```
     $                 H( JCH, JCH-1 ) = H( JCH, JCH-1 )*C
                    ILAZR2 = .FALSE.
                    IF( ABS( T( JCH+1, JCH+1 ) ).GE.BTOL ) THEN
                       IF( JCH+1.GE.ILAST ) THEN
                          GO TO 80
                       ELSE
                          IFIRST = JCH + 1
                          GO TO 110
                       END IF
                    END IF
                    T( JCH+1, JCH+1 ) = ZERO
   40            CONTINUE
                 GO TO 70
              ELSE
*
*                Only test 2 passed -- chase the zero to T(ILAST,ILAST)
*                Then process as in the case T(ILAST,ILAST)=0
*
                 DO 50 JCH = J, ILAST - 1
                    TEMP = T( JCH, JCH+1 )
                    CALL DLARTG( TEMP, T( JCH+1, JCH+1 ), C, S,
     $                           T( JCH, JCH+1 ) )
                    T( JCH+1, JCH+1 ) = ZERO
                    IF( JCH.LT.ILASTM-1 )
     $                 CALL DROT( ILASTM-JCH-1, T( JCH, JCH+2 ), LDT,
     $                            T( JCH+1, JCH+2 ), LDT, C, S )
                    CALL DROT( ILASTM-JCH+2, H( JCH, JCH-1 ), LDH,
     $                         H( JCH+1, JCH-1 ), LDH, C, S )
                    IF( ILQ )
     $                 CALL DROT( N, Q( 1, JCH ), 1, Q( 1, JCH+1 ), 1,
     $                            C, S )
                    TEMP = H( JCH+1, JCH )
                    CALL DLARTG( TEMP, H( JCH+1, JCH-1 ), C, S,
     $                           H( JCH+1, JCH ) )
                    H( JCH+1, JCH-1 ) = ZERO
                    CALL DROT( JCH+1-IFRSTM, H( IFRSTM, JCH ), 1,
     $                         H( IFRSTM, JCH-1 ), 1, C, S )
                    CALL DROT( JCH-IFRSTM, T( IFRSTM, JCH ), 1,
     $                         T( IFRSTM, JCH-1 ), 1, C, S )
                    IF( ILZ )
     $                 CALL DROT( N, Z( 1, JCH ), 1, Z( 1, JCH-1 ), 1,
     $                            C, S )
   50            CONTINUE
                 GO TO 70
              END IF
           ELSE IF( ILAZRO ) THEN
*
*             Only test 1 passed -- work on J:ILAST
*
              IFIRST = J
```

```
               GO TO 110
            END IF
*
*           Neither test passed -- try next J
*
   60    CONTINUE
*
*        (Drop-through is "impossible")
*
         INFO = N + 1
         GO TO 420
*
*        T(ILAST,ILAST)=0 -- clear H(ILAST,ILAST-1) to split off a
*        1x1 block.
*
   70    CONTINUE
         TEMP = H( ILAST, ILAST )
         CALL DLARTG( TEMP, H( ILAST, ILAST-1 ), C, S,
     $                H( ILAST, ILAST ) )
         H( ILAST, ILAST-1 ) = ZERO
         CALL DROT( ILAST-IFRSTM, H( IFRSTM, ILAST ), 1,
     $              H( IFRSTM, ILAST-1 ), 1, C, S )
         CALL DROT( ILAST-IFRSTM, T( IFRSTM, ILAST ), 1,
     $              T( IFRSTM, ILAST-1 ), 1, C, S )
         IF( ILZ )
     $      CALL DROT( N, Z( 1, ILAST ), 1, Z( 1, ILAST-1 ), 1, C, S )
*
*        H(ILAST,ILAST-1)=0 -- Standardize B, set ALPHAR, ALPHAI,
*                              and BETA
*
   80    CONTINUE
         IF( T( ILAST, ILAST ).LT.ZERO ) THEN
            IF( ILSCHR ) THEN
               DO 90 J = IFRSTM, ILAST
                  H( J, ILAST ) = -H( J, ILAST )
                  T( J, ILAST ) = -T( J, ILAST )
   90          CONTINUE
            ELSE
               H( ILAST, ILAST ) = -H( ILAST, ILAST )
               T( ILAST, ILAST ) = -T( ILAST, ILAST )
            END IF
            IF( ILZ ) THEN
               DO 100 J = 1, N
                  Z( J, ILAST ) = -Z( J, ILAST )
  100          CONTINUE
            END IF
         END IF
         ALPHAR( ILAST ) = H( ILAST, ILAST )
         ALPHAI( ILAST ) = ZERO
         BETA( ILAST ) = T( ILAST, ILAST )
```

```
*
*        Go to next block -- exit if finished.
*
         ILAST = ILAST - 1
         IF( ILAST.LT.ILO )
     $      GO TO 380
*
*        Reset counters
*
         IITER = 0
         ESHIFT = ZERO
         IF( .NOT.ILSCHR ) THEN
            ILASTM = ILAST
            IF( IFRSTM.GT.ILAST )
     $         IFRSTM = ILO
         END IF
         GO TO 350
*
*        QZ step
*
*        This iteration only involves rows/columns IFIRST:ILAST. We
*        assume IFIRST < ILAST, and that the diagonal of B is non-zero.
*
  110    CONTINUE
         IITER = IITER + 1
         IF( .NOT.ILSCHR ) THEN
            IFRSTM = IFIRST
         END IF
*
*        Compute single shifts.
*
*        At this point, IFIRST < ILAST, and the diagonal elements of
*        T(IFIRST:ILAST,IFIRST,ILAST) are larger than BTOL (in
*        magnitude)
*
         IF( ( IITER / 10 )*10.EQ.IITER ) THEN
*
*           Exceptional shift.  Chosen for no particularly good reason.
*           (Single shift only.)
*
            IF( ( DBLE( MAXIT )*SAFMIN )*ABS( H( ILAST, ILAST-1 ) ).LT.
     $          ABS( T( ILAST-1, ILAST-1 ) ) ) THEN
               ESHIFT = H( ILAST, ILAST-1 ) /
     $                  T( ILAST-1, ILAST-1 )
            ELSE
               ESHIFT = ESHIFT + ONE / ( SAFMIN*DBLE( MAXIT ) )
            END IF
            S1 = ONE
            WR = ESHIFT
*
```

```
         ELSE
*
*           Shifts based on the generalized eigenvalues of the
*           bottom-right 2x2 block of A and B. The first eigenvalue
*           returned by DLAG2 is the Wilkinson shift (AEP p.512),
*
            CALL DLAG2( H( ILAST-1, ILAST-1 ), LDH,
     $                  T( ILAST-1, ILAST-1 ), LDT, SAFMIN*SAFETY, S1,
     $                  S2, WR, WR2, WI )
*
            IF ( ABS( (WR/S1)*T( ILAST, ILAST ) - H( ILAST, ILAST ) )
     $         .GT. ABS( (WR2/S2)*T( ILAST, ILAST )
     $         - H( ILAST, ILAST ) ) ) THEN
               TEMP = WR
               WR = WR2
               WR2 = TEMP
               TEMP = S1
               S1 = S2
               S2 = TEMP
            END IF
            TEMP = MAX( S1, SAFMIN*MAX( ONE, ABS( WR ), ABS( WI ) ) )
            IF( WI.NE.ZERO )
     $         GO TO 200
         END IF
*
*        Fiddle with shift to avoid overflow
*
         TEMP = MIN( ASCALE, ONE )*( HALF*SAFMAX )
         IF( S1.GT.TEMP ) THEN
            SCALE = TEMP / S1
         ELSE
            SCALE = ONE
         END IF
*
         TEMP = MIN( BSCALE, ONE )*( HALF*SAFMAX )
         IF( ABS( WR ).GT.TEMP )
     $      SCALE = MIN( SCALE, TEMP / ABS( WR ) )
         S1 = SCALE*S1
         WR = SCALE*WR
*
*        Now check for two consecutive small subdiagonals.
*
         DO 120 J = ILAST - 1, IFIRST + 1, -1
            ISTART = J
            TEMP = ABS( S1*H( J, J-1 ) )
            TEMP2 = ABS( S1*H( J, J )-WR*T( J, J ) )
            TEMPR = MAX( TEMP, TEMP2 )
            IF( TEMPR.LT.ONE .AND. TEMPR.NE.ZERO ) THEN
               TEMP = TEMP / TEMPR
               TEMP2 = TEMP2 / TEMPR
```

```
            END IF
            IF( ABS( ( ASCALE*H( J+1, J ) )*TEMP ).LE.( ASCALE*ATOL )*
     $          TEMP2 )GO TO 130
  120    CONTINUE
*
         ISTART = IFIRST
  130    CONTINUE
*
*        Do an implicit single-shift QZ sweep.
*
*        Initial Q
*
         TEMP = S1*H( ISTART, ISTART ) - WR*T( ISTART, ISTART )
         TEMP2 = S1*H( ISTART+1, ISTART )
         CALL DLARTG( TEMP, TEMP2, C, S, TEMPR )
*
*        Sweep
*
         DO 190 J = ISTART, ILAST - 1
            IF( J.GT.ISTART ) THEN
               TEMP = H( J, J-1 )
               CALL DLARTG( TEMP, H( J+1, J-1 ), C, S, H( J, J-1 ) )
               H( J+1, J-1 ) = ZERO
            END IF
*
            DO 140 JC = J, ILASTM
               TEMP = C*H( J, JC ) + S*H( J+1, JC )
               H( J+1, JC ) = -S*H( J, JC ) + C*H( J+1, JC )
               H( J, JC ) = TEMP
               TEMP2 = C*T( J, JC ) + S*T( J+1, JC )
               T( J+1, JC ) = -S*T( J, JC ) + C*T( J+1, JC )
               T( J, JC ) = TEMP2
  140       CONTINUE
            IF( ILQ ) THEN
               DO 150 JR = 1, N
                  TEMP = C*Q( JR, J ) + S*Q( JR, J+1 )
                  Q( JR, J+1 ) = -S*Q( JR, J ) + C*Q( JR, J+1 )
                  Q( JR, J ) = TEMP
  150          CONTINUE
            END IF
*
            TEMP = T( J+1, J+1 )
            CALL DLARTG( TEMP, T( J+1, J ), C, S, T( J+1, J+1 ) )
            T( J+1, J ) = ZERO
*
            DO 160 JR = IFRSTM, MIN( J+2, ILAST )
               TEMP = C*H( JR, J+1 ) + S*H( JR, J )
               H( JR, J ) = -S*H( JR, J+1 ) + C*H( JR, J )
               H( JR, J+1 ) = TEMP
  160       CONTINUE
```

```
            DO 170 JR = IFRSTM, J
               TEMP = C*T( JR, J+1 ) + S*T( JR, J )
               T( JR, J ) = -S*T( JR, J+1 ) + C*T( JR, J )
               T( JR, J+1 ) = TEMP
  170       CONTINUE
            IF( ILZ ) THEN
               DO 180 JR = 1, N
                  TEMP = C*Z( JR, J+1 ) + S*Z( JR, J )
                  Z( JR, J ) = -S*Z( JR, J+1 ) + C*Z( JR, J )
                  Z( JR, J+1 ) = TEMP
  180          CONTINUE
            END IF
  190    CONTINUE
*
         GO TO 350
*
*        Use Francis double-shift
*
*        Note: the Francis double-shift should work with real shifts,
*              but only if the block is at least 3x3.
*              This code may break if this point is reached with
*              a 2x2 block with real eigenvalues.
*
  200    CONTINUE
         IF( IFIRST+1.EQ.ILAST ) THEN
*
*           Special case -- 2x2 block with complex eigenvectors
*
*           Step 1: Standardize, that is, rotate so that
*
*                       ( B11  0  )
*                   B = (         )  with B11 non-negative.
*                       (  0  B22 )
*
            CALL DLASV2( T( ILAST-1, ILAST-1 ), T( ILAST-1, ILAST ),
     $                   T( ILAST, ILAST ), B22, B11, SR, CR, SL, CL )
*
            IF( B11.LT.ZERO ) THEN
               CR = -CR
               SR = -SR
               B11 = -B11
               B22 = -B22
            END IF
*
            CALL DROT( ILASTM+1-IFIRST, H( ILAST-1, ILAST-1 ), LDH,
     $                 H( ILAST, ILAST-1 ), LDH, CL, SL )
            CALL DROT( ILAST+1-IFRSTM, H( IFRSTM, ILAST-1 ), 1,
     $                 H( IFRSTM, ILAST ), 1, CR, SR )
*
            IF( ILAST.LT.ILASTM )
```

```
     $         CALL DROT( ILASTM-ILAST, T( ILAST-1, ILAST+1 ), LDT,
     $                    T( ILAST, ILAST+1 ), LDT, CL, SL )
           IF( IFRSTM.LT.ILAST-1 )
     $         CALL DROT( IFIRST-IFRSTM, T( IFRSTM, ILAST-1 ), 1,
     $                    T( IFRSTM, ILAST ), 1, CR, SR )
*
           IF( ILQ )
     $         CALL DROT( N, Q( 1, ILAST-1 ), 1, Q( 1, ILAST ), 1, CL,
     $                    SL )
           IF( ILZ )
     $         CALL DROT( N, Z( 1, ILAST-1 ), 1, Z( 1, ILAST ), 1, CR,
     $                    SR )
*
           T( ILAST-1, ILAST-1 ) = B11
           T( ILAST-1, ILAST ) = ZERO
           T( ILAST, ILAST-1 ) = ZERO
           T( ILAST, ILAST ) = B22
*
*          If B22 is negative, negate column ILAST
*
           IF( B22.LT.ZERO ) THEN
              DO 210 J = IFRSTM, ILAST
                 H( J, ILAST ) = -H( J, ILAST )
                 T( J, ILAST ) = -T( J, ILAST )
  210         CONTINUE
*
              IF( ILZ ) THEN
                 DO 220 J = 1, N
                    Z( J, ILAST ) = -Z( J, ILAST )
  220            CONTINUE
              END IF
              B22 = -B22
           END IF
*
*          Step 2: Compute ALPHAR, ALPHAI, and BETA (see refs.)
*
*          Recompute shift
*
           CALL DLAG2( H( ILAST-1, ILAST-1 ), LDH,
     $                 T( ILAST-1, ILAST-1 ), LDT, SAFMIN*SAFETY, S1,
     $                 TEMP, WR, TEMP2, WI )
*
*          If standardization has perturbed the shift onto real line,
*          do another (real single-shift) QR step.
*
           IF( WI.EQ.ZERO )
     $         GO TO 350
           S1INV = ONE / S1
*
*          Do EISPACK (QZVAL) computation of alpha and beta
```

```
*
            A11 = H( ILAST-1, ILAST-1 )
            A21 = H( ILAST, ILAST-1 )
            A12 = H( ILAST-1, ILAST )
            A22 = H( ILAST, ILAST )
*
*           Compute complex Givens rotation on right
*           (Assume some element of C = (sA - wB) > unfl )
*                            __
*           (sA - wB) ( CZ   -SZ )
*                     ( SZ    CZ )
*
            C11R = S1*A11 - WR*B11
            C11I = -WI*B11
            C12 = S1*A12
            C21 = S1*A21
            C22R = S1*A22 - WR*B22
            C22I = -WI*B22
*
            IF( ABS( C11R )+ABS( C11I )+ABS( C12 ).GT.ABS( C21 )+
     $          ABS( C22R )+ABS( C22I ) ) THEN
               T1 = DLAPY3( C12, C11R, C11I )
               CZ = C12 / T1
               SZR = -C11R / T1
               SZI = -C11I / T1
            ELSE
               CZ = DLAPY2( C22R, C22I )
               IF( CZ.LE.SAFMIN ) THEN
                  CZ = ZERO
                  SZR = ONE
                  SZI = ZERO
               ELSE
                  TEMPR = C22R / CZ
                  TEMPI = C22I / CZ
                  T1 = DLAPY2( CZ, C21 )
                  CZ = CZ / T1
                  SZR = -C21*TEMPR / T1
                  SZI = C21*TEMPI / T1
               END IF
            END IF
*
*           Compute Givens rotation on left
*
*           (  CQ   SQ )
*           (  __      )  A or B
*           ( -SQ   CQ )
*
            AN = ABS( A11 ) + ABS( A12 ) + ABS( A21 ) + ABS( A22 )
            BN = ABS( B11 ) + ABS( B22 )
            WABS = ABS( WR ) + ABS( WI )
```

```
            IF( S1*AN.GT.WABS*BN ) THEN
               CQ = CZ*B11
               SQR = SZR*B22
               SQI = -SZI*B22
            ELSE
               A1R = CZ*A11 + SZR*A12
               A1I = SZI*A12
               A2R = CZ*A21 + SZR*A22
               A2I = SZI*A22
               CQ = DLAPY2( A1R, A1I )
               IF( CQ.LE.SAFMIN ) THEN
                  CQ = ZERO
                  SQR = ONE
                  SQI = ZERO
               ELSE
                  TEMPR = A1R / CQ
                  TEMPI = A1I / CQ
                  SQR = TEMPR*A2R + TEMPI*A2I
                  SQI = TEMPI*A2R - TEMPR*A2I
               END IF
            END IF
            T1 = DLAPY3( CQ, SQR, SQI )
            CQ = CQ / T1
            SQR = SQR / T1
            SQI = SQI / T1
*
*           Compute diagonal elements of QBZ
*
            TEMPR = SQR*SZR - SQI*SZI
            TEMPI = SQR*SZI + SQI*SZR
            B1R = CQ*CZ*B11 + TEMPR*B22
            B1I = TEMPI*B22
            B1A = DLAPY2( B1R, B1I )
            B2R = CQ*CZ*B22 + TEMPR*B11
            B2I = -TEMPI*B11
            B2A = DLAPY2( B2R, B2I )
*
*           Normalize so beta > 0, and Im( alpha1 ) > 0
*
            BETA( ILAST-1 ) = B1A
            BETA( ILAST ) = B2A
            ALPHAR( ILAST-1 ) = ( WR*B1A )*S1INV
            ALPHAI( ILAST-1 ) = ( WI*B1A )*S1INV
            ALPHAR( ILAST ) = ( WR*B2A )*S1INV
            ALPHAI( ILAST ) = -( WI*B2A )*S1INV
*
*           Step 3: Go to next block -- exit if finished.
*
            ILAST = IFIRST - 1
            IF( ILAST.LT.ILO )
```

```
     $         GO TO 380
*
*          Reset counters
*
           IITER = 0
           ESHIFT = ZERO
           IF( .NOT.ILSCHR ) THEN
              ILASTM = ILAST
              IF( IFRSTM.GT.ILAST )
     $           IFRSTM = ILO
           END IF
           GO TO 350
        ELSE
*
*          Usual case: 3x3 or larger block, using Francis implicit
*                      double-shift
*
*                                     2
*          Eigenvalue equation is  w  - c w + d = 0,
*
*                                         -1 2        -1
*          so compute 1st column of  (A B  )  - c A B   + d
*          using the formula in QZIT (from EISPACK)
*
*          We assume that the block is at least 3x3
*
           AD11 = ( ASCALE*H( ILAST-1, ILAST-1 ) ) /
     $            ( BSCALE*T( ILAST-1, ILAST-1 ) )
           AD21 = ( ASCALE*H( ILAST, ILAST-1 ) ) /
     $            ( BSCALE*T( ILAST-1, ILAST-1 ) )
           AD12 = ( ASCALE*H( ILAST-1, ILAST ) ) /
     $            ( BSCALE*T( ILAST, ILAST ) )
           AD22 = ( ASCALE*H( ILAST, ILAST ) ) /
     $            ( BSCALE*T( ILAST, ILAST ) )
           U12 = T( ILAST-1, ILAST ) / T( ILAST, ILAST )
           AD11L = ( ASCALE*H( IFIRST, IFIRST ) ) /
     $             ( BSCALE*T( IFIRST, IFIRST ) )
           AD21L = ( ASCALE*H( IFIRST+1, IFIRST ) ) /
     $             ( BSCALE*T( IFIRST, IFIRST ) )
           AD12L = ( ASCALE*H( IFIRST, IFIRST+1 ) ) /
     $             ( BSCALE*T( IFIRST+1, IFIRST+1 ) )
           AD22L = ( ASCALE*H( IFIRST+1, IFIRST+1 ) ) /
     $             ( BSCALE*T( IFIRST+1, IFIRST+1 ) )
           AD32L = ( ASCALE*H( IFIRST+2, IFIRST+1 ) ) /
     $             ( BSCALE*T( IFIRST+1, IFIRST+1 ) )
           U12L = T( IFIRST, IFIRST+1 ) / T( IFIRST+1, IFIRST+1 )
*
           V( 1 ) = ( AD11-AD11L )*( AD22-AD11L ) - AD12*AD21 +
     $              AD21*U12*AD11L + ( AD12L-AD11L*U12L )*AD21L
           V( 2 ) = ( ( AD22L-AD11L )-AD21L*U12L-( AD11-AD11L )-
```

```
     $               ( AD22-AD11L )+AD21*U12 )*AD21L
            V( 3 ) = AD32L*AD21L
*
            ISTART = IFIRST
*
            CALL DLARFG( 3, V( 1 ), V( 2 ), 1, TAU )
            V( 1 ) = ONE
*
*           Sweep
*
            DO 290 J = ISTART, ILAST - 2
*
*              All but last elements: use 3x3 Householder transforms.
*
*              Zero (j-1)st column of A
*
               IF( J.GT.ISTART ) THEN
                  V( 1 ) = H( J, J-1 )
                  V( 2 ) = H( J+1, J-1 )
                  V( 3 ) = H( J+2, J-1 )
*
                  CALL DLARFG( 3, H( J, J-1 ), V( 2 ), 1, TAU )
                  V( 1 ) = ONE
                  H( J+1, J-1 ) = ZERO
                  H( J+2, J-1 ) = ZERO
               END IF
*
               DO 230 JC = J, ILASTM
                  TEMP = TAU*( H( J, JC )+V( 2 )*H( J+1, JC )+V( 3 )*
     $                   H( J+2, JC ) )
                  H( J, JC ) = H( J, JC ) - TEMP
                  H( J+1, JC ) = H( J+1, JC ) - TEMP*V( 2 )
                  H( J+2, JC ) = H( J+2, JC ) - TEMP*V( 3 )
                  TEMP2 = TAU*( T( J, JC )+V( 2 )*T( J+1, JC )+V( 3 )*
     $                    T( J+2, JC ) )
                  T( J, JC ) = T( J, JC ) - TEMP2
                  T( J+1, JC ) = T( J+1, JC ) - TEMP2*V( 2 )
                  T( J+2, JC ) = T( J+2, JC ) - TEMP2*V( 3 )
  230          CONTINUE
               IF( ILQ ) THEN
                  DO 240 JR = 1, N
                     TEMP = TAU*( Q( JR, J )+V( 2 )*Q( JR, J+1 )+V( 3 )*
     $                       Q( JR, J+2 ) )
                     Q( JR, J ) = Q( JR, J ) - TEMP
                     Q( JR, J+1 ) = Q( JR, J+1 ) - TEMP*V( 2 )
                     Q( JR, J+2 ) = Q( JR, J+2 ) - TEMP*V( 3 )
  240             CONTINUE
               END IF
*
*              Zero j-th column of B (see DLAGBC for details)
```

```
*
*              Swap rows to pivot
*
               ILPIVT = .FALSE.
               TEMP = MAX( ABS( T( J+1, J+1 ) ), ABS( T( J+1, J+2 ) ) )
               TEMP2 = MAX( ABS( T( J+2, J+1 ) ), ABS( T( J+2, J+2 ) ) )
               IF( MAX( TEMP, TEMP2 ).LT.SAFMIN ) THEN
                  SCALE = ZERO
                  U1 = ONE
                  U2 = ZERO
                  GO TO 250
               ELSE IF( TEMP.GE.TEMP2 ) THEN
                  W11 = T( J+1, J+1 )
                  W21 = T( J+2, J+1 )
                  W12 = T( J+1, J+2 )
                  W22 = T( J+2, J+2 )
                  U1 = T( J+1, J )
                  U2 = T( J+2, J )
               ELSE
                  W21 = T( J+1, J+1 )
                  W11 = T( J+2, J+1 )
                  W22 = T( J+1, J+2 )
                  W12 = T( J+2, J+2 )
                  U2 = T( J+1, J )
                  U1 = T( J+2, J )
               END IF
*
*              Swap columns if nec.
*
               IF( ABS( W12 ).GT.ABS( W11 ) ) THEN
                  ILPIVT = .TRUE.
                  TEMP = W12
                  TEMP2 = W22
                  W12 = W11
                  W22 = W21
                  W11 = TEMP
                  W21 = TEMP2
               END IF
*
*              LU-factor
*
               TEMP = W21 / W11
               U2 = U2 - TEMP*U1
               W22 = W22 - TEMP*W12
               W21 = ZERO
*
*              Compute SCALE
*
               SCALE = ONE
               IF( ABS( W22 ).LT.SAFMIN ) THEN
```

```
                 SCALE = ZERO
                 U2 = ONE
                 U1 = -W12 / W11
                 GO TO 250
              END IF
              IF( ABS( W22 ).LT.ABS( U2 ) )
     $           SCALE = ABS( W22 / U2 )
              IF( ABS( W11 ).LT.ABS( U1 ) )
     $           SCALE = MIN( SCALE, ABS( W11 / U1 ) )
*
*             Solve
*
              U2 = ( SCALE*U2 ) / W22
              U1 = ( SCALE*U1-W12*U2 ) / W11
*
  250         CONTINUE
              IF( ILPIVT ) THEN
                 TEMP = U2
                 U2 = U1
                 U1 = TEMP
              END IF
*
*             Compute Householder Vector
*
              T1 = SQRT( SCALE**2+U1**2+U2**2 )
              TAU = ONE + SCALE / T1
              VS = -ONE / ( SCALE+T1 )
              V( 1 ) = ONE
              V( 2 ) = VS*U1
              V( 3 ) = VS*U2
*
*             Apply transformations from the right.
*
              DO 260 JR = IFRSTM, MIN( J+3, ILAST )
                 TEMP = TAU*( H( JR, J )+V( 2 )*H( JR, J+1 )+V( 3 )*
     $                  H( JR, J+2 ) )
                 H( JR, J ) = H( JR, J ) - TEMP
                 H( JR, J+1 ) = H( JR, J+1 ) - TEMP*V( 2 )
                 H( JR, J+2 ) = H( JR, J+2 ) - TEMP*V( 3 )
  260         CONTINUE
              DO 270 JR = IFRSTM, J + 2
                 TEMP = TAU*( T( JR, J )+V( 2 )*T( JR, J+1 )+V( 3 )*
     $                  T( JR, J+2 ) )
                 T( JR, J ) = T( JR, J ) - TEMP
                 T( JR, J+1 ) = T( JR, J+1 ) - TEMP*V( 2 )
                 T( JR, J+2 ) = T( JR, J+2 ) - TEMP*V( 3 )
  270         CONTINUE
              IF( ILZ ) THEN
                 DO 280 JR = 1, N
                    TEMP = TAU*( Z( JR, J )+V( 2 )*Z( JR, J+1 )+V( 3 )*
```

```
     $                     Z( JR, J+2 ) )
                     Z( JR, J ) = Z( JR, J ) - TEMP
                     Z( JR, J+1 ) = Z( JR, J+1 ) - TEMP*V( 2 )
                     Z( JR, J+2 ) = Z( JR, J+2 ) - TEMP*V( 3 )
  280             CONTINUE
               END IF
               T( J+1, J ) = ZERO
               T( J+2, J ) = ZERO
  290       CONTINUE
*
*           Last elements: Use Givens rotations
*
*           Rotations from the left
*
            J = ILAST - 1
            TEMP = H( J, J-1 )
            CALL DLARTG( TEMP, H( J+1, J-1 ), C, S, H( J, J-1 ) )
            H( J+1, J-1 ) = ZERO
*
            DO 300 JC = J, ILASTM
               TEMP = C*H( J, JC ) + S*H( J+1, JC )
               H( J+1, JC ) = -S*H( J, JC ) + C*H( J+1, JC )
               H( J, JC ) = TEMP
               TEMP2 = C*T( J, JC ) + S*T( J+1, JC )
               T( J+1, JC ) = -S*T( J, JC ) + C*T( J+1, JC )
               T( J, JC ) = TEMP2
  300       CONTINUE
            IF( ILQ ) THEN
               DO 310 JR = 1, N
                  TEMP = C*Q( JR, J ) + S*Q( JR, J+1 )
                  Q( JR, J+1 ) = -S*Q( JR, J ) + C*Q( JR, J+1 )
                  Q( JR, J ) = TEMP
  310          CONTINUE
            END IF
*
*           Rotations from the right.
*
            TEMP = T( J+1, J+1 )
            CALL DLARTG( TEMP, T( J+1, J ), C, S, T( J+1, J+1 ) )
            T( J+1, J ) = ZERO
*
            DO 320 JR = IFRSTM, ILAST
               TEMP = C*H( JR, J+1 ) + S*H( JR, J )
               H( JR, J ) = -S*H( JR, J+1 ) + C*H( JR, J )
               H( JR, J+1 ) = TEMP
  320       CONTINUE
            DO 330 JR = IFRSTM, ILAST - 1
               TEMP = C*T( JR, J+1 ) + S*T( JR, J )
               T( JR, J ) = -S*T( JR, J+1 ) + C*T( JR, J )
               T( JR, J+1 ) = TEMP
```

```
  330       CONTINUE
            IF( ILZ ) THEN
               DO 340 JR = 1, N
                  TEMP = C*Z( JR, J+1 ) + S*Z( JR, J )
                  Z( JR, J ) = -S*Z( JR, J+1 ) + C*Z( JR, J )
                  Z( JR, J+1 ) = TEMP
  340          CONTINUE
            END IF
*
*           End of Double-Shift code
*
         END IF
*
         GO TO 350
*
*        End of iteration loop
*
  350    CONTINUE
  360 CONTINUE
*
*     Drop-through = non-convergence
*
      INFO = ILAST
      GO TO 420
*
*     Successful completion of all QZ steps
*
  380 CONTINUE
*
*     Set Eigenvalues 1:ILO-1
*
      DO 410 J = 1, ILO - 1
         IF( T( J, J ).LT.ZERO ) THEN
            IF( ILSCHR ) THEN
               DO 390 JR = 1, J
                  H( JR, J ) = -H( JR, J )
                  T( JR, J ) = -T( JR, J )
  390          CONTINUE
            ELSE
               H( J, J ) = -H( J, J )
               T( J, J ) = -T( J, J )
            END IF
            IF( ILZ ) THEN
               DO 400 JR = 1, N
                  Z( JR, J ) = -Z( JR, J )
  400          CONTINUE
            END IF
         END IF
         ALPHAR( J ) = H( J, J )
         ALPHAI( J ) = ZERO
```

```
         BETA( J ) = T( J, J )
  410 CONTINUE
*
*     Normal Termination
*
      INFO = 0
*
*     Exit (other than argument error) -- return optimal workspace size
*
  420 CONTINUE
      WORK( 1 ) = DBLE( N )
      RETURN
*
*     End of DHGEQZ
*
      END
```

32.34.4 lisp code

— dhgeqz.lisp —

```
(let* ((half 0.5d0) (zero 0.0d0) (one 1.0d0) (safety 100.0d0))
  (declare (type (double-float 0.5d0 0.5d0) half)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 100.0d0 100.0d0) safety)
           (ignorable half zero one safety))
  (defun dhgeqz
         (job compq compz n ilo ihi h ldh t$ ldt alphar alphai beta q ldq z ldz
          work lwork info)
    (declare (type (array double-float (*)) work z q beta alphai alphar t$ h)
             (type (f2cl-lib:integer4) info lwork ldz ldq ldt ldh ihi ilo n)
             (type (string *) compz compq job))
    (f2cl-lib:with-multi-array-data
        ((job character job-%data% job-%offset%)
         (compq character compq-%data% compq-%offset%)
         (compz character compz-%data% compz-%offset%)
         (h double-float h-%data% h-%offset%)
         (t$ double-float t$-%data% t$-%offset%)
         (alphar double-float alphar-%data% alphar-%offset%)
         (alphai double-float alphai-%data% alphai-%offset%)
         (beta double-float beta-%data% beta-%offset%)
         (q double-float q-%data% q-%offset%)
         (z double-float z-%data% z-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((v (make-array 3 :element-type 'double-float)) (a11 0.0d0)
```

```
     (a12 0.0d0) (a1i 0.0d0) (a1r 0.0d0) (a21 0.0d0) (a22 0.0d0)
     (a2i 0.0d0) (a2r 0.0d0) (ad11 0.0d0) (ad11l 0.0d0) (ad12 0.0d0)
     (ad12l 0.0d0) (ad21 0.0d0) (ad21l 0.0d0) (ad22 0.0d0)
     (ad22l 0.0d0) (ad32l 0.0d0) (an 0.0d0) (anorm 0.0d0)
     (ascale 0.0d0) (atol 0.0d0) (b11 0.0d0) (b1a 0.0d0) (b1i 0.0d0)
     (b1r 0.0d0) (b22 0.0d0) (b2a 0.0d0) (b2i 0.0d0) (b2r 0.0d0)
     (bn 0.0d0) (bnorm 0.0d0) (bscale 0.0d0) (btol 0.0d0) (c 0.0d0)
     (c11i 0.0d0) (c11r 0.0d0) (c12 0.0d0) (c21 0.0d0) (c22i 0.0d0)
     (c22r 0.0d0) (cl 0.0d0) (cq 0.0d0) (cr 0.0d0) (cz 0.0d0)
     (eshift 0.0d0) (s 0.0d0) (s1 0.0d0) (s1inv 0.0d0) (s2 0.0d0)
     (safmax 0.0d0) (safmin 0.0d0) (scale 0.0d0) (sl 0.0d0) (sqi 0.0d0)
     (sqr 0.0d0) (sr 0.0d0) (szi 0.0d0) (szr 0.0d0) (t1 0.0d0)
     (tau 0.0d0) (temp 0.0d0) (temp2 0.0d0) (tempi 0.0d0) (tempr 0.0d0)
     (u1 0.0d0) (u12 0.0d0) (u12l 0.0d0) (u2 0.0d0) (ulp 0.0d0)
     (vs 0.0d0) (w11 0.0d0) (w12 0.0d0) (w21 0.0d0) (w22 0.0d0)
     (wabs 0.0d0) (wi 0.0d0) (wr 0.0d0) (wr2 0.0d0) (icompq 0)
     (icompz 0) (ifirst 0) (ifrstm 0) (iiter 0) (ilast 0) (ilastm 0)
     (in 0) (ischur 0) (istart 0) (j 0) (jc 0) (jch 0) (jiter 0) (jr 0)
     (maxit 0) (ilazr2 nil) (ilazro nil) (ilpivt nil) (ilq nil)
     (ilschr nil) (ilz nil) (lquery nil))
(declare (type (array double-float (3)) v)
         (type (double-float) a11 a12 a1i a1r a21 a22 a2i a2r ad11
          ad11l ad12 ad12l ad21 ad21l ad22 ad22l ad32l an anorm ascale
          atol b11 b1a b1i b1r b22 b2a b2i b2r bn bnorm bscale btol c
          c11i c11r c12 c21 c22i c22r cl cq cr cz eshift s s1 s1inv s2
          safmax safmin scale sl sqi sqr sr szi szr t1 tau temp temp2
          tempi tempr u1 u12 u12l u2 ulp vs w11 w12 w21 w22 wabs wi wr
          wr2)
         (type (f2cl-lib:integer4) icompq icompz ifirst ifrstm iiter
          ilast ilastm in ischur istart j jc jch jiter jr maxit)
         (type f2cl-lib:logical ilazr2 ilazro ilpivt ilq ilschr ilz
          lquery))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame job "E")
    (declare (ignore var-1))
    (when var-0 (setf job var-0))
    ret-val)
  (setf ilschr f2cl-lib:%false%) (setf ischur 1))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame job "S")
    (declare (ignore var-1))
    (when var-0 (setf job var-0))
    ret-val)
  (setf ilschr f2cl-lib:%true%) (setf ischur 2))
 (t (setf ischur 0)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compq "N")
    (declare (ignore var-1))
```

```
    (when var-0 (setf compq var-0))
    ret-val)
  (setf ilq f2cl-lib:%false%) (setf icompq 1))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compq "V")
    (declare (ignore var-1))
    (when var-0 (setf compq var-0))
    ret-val)
  (setf ilq f2cl-lib:%true%) (setf icompq 2))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compq "I")
    (declare (ignore var-1))
    (when var-0 (setf compq var-0))
    ret-val)
  (setf ilq f2cl-lib:%true%) (setf icompq 3))
 (t (setf icompq 0)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "N")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf ilz f2cl-lib:%false%) (setf icompz 1))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "V")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf ilz f2cl-lib:%true%) (setf icompz 2))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "I")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf ilz f2cl-lib:%true%) (setf icompz 3))
 (t (setf icompz 0)))
(setf info 0)
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce
         (the f2cl-lib:integer4
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 n)))
         'double-float))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((= ischur 0) (setf info -1)) ((= icompq 0) (setf info -2))
      ((= icompz 0) (setf info -3)) ((< n 0) (setf info -4))
      ((< ilo 1) (setf info -5))
      ((or (> ihi n)
           (< ihi (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))))
       (setf info -6))
```

```
      ((< ldh n) (setf info -8)) ((< ldt n) (setf info -10))
      ((or (< ldq 1) (and ilq (< ldq n))) (setf info -15))
      ((or (< ldz 1) (and ilz (< ldz n))) (setf info -17))
      ((and
        (< lwork
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (not lquery))
       (setf info -19)))
(cond
 ((/= info 0) (xerbla "DHGEQZ" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond
 ((<= n 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:dble 1))
  (go end_label)))
(if (= icompq 3)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "Full" n n zero one q ldq)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldq var-6))))
(if (= icompz 3)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "Full" n n zero one z ldz)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldz var-6))))
(setf in (f2cl-lib:int-sub (f2cl-lib:int-add ihi 1) ilo))
(setf safmin (dlamch "S"))
(setf safmax (/ one safmin))
(setf ulp (* (dlamch "E") (dlamch "B")))
(setf anorm
        (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
            (dlanhs "F" in
             (f2cl-lib:array-slice h-%data% double-float (ilo ilo)
                                   ((1 ldh) (1 *)) h-%offset%)
             ldh work)
          (declare (ignore var-0 var-2 var-4))
          (when var-1 (setf in var-1))
          (when var-3 (setf ldh var-3))
          ret-val))
(setf bnorm
        (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
```

```
              (dlanhs "F" in
               (f2cl-lib:array-slice t$-%data% double-float (ilo ilo)
                                     ((1 ldt) (1 *)) t$-%offset%)
               ldt work)
            (declare (ignore var-0 var-2 var-4))
            (when var-1 (setf in var-1))
            (when var-3 (setf ldt var-3))
            ret-val))
(setf atol (max safmin (* ulp anorm)))
(setf btol (max safmin (* ulp bnorm)))
(setf ascale (/ one (max safmin anorm)))
(setf bscale (/ one (max safmin bnorm)))
(f2cl-lib:fdo (j (f2cl-lib:int-add ihi 1) (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (cond
     ((< (f2cl-lib:fref t$ (j j) ((1 ldt) (1 *))) zero)
      (cond
       (ilschr
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr j) nil)
          (tagbody
            (setf (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                                 h-%offset%)
                  (-
                   (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                                  h-%offset%)))
            (setf (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                                 t$-%offset%)
                  (-
                   (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                                  t$-%offset%)))
           label10)))
       (t
        (setf (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *)) h-%offset%)
              (-
               (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *))
                              h-%offset%)))
        (setf (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                             t$-%offset%)
              (-
               (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                              t$-%offset%)))))
      (cond
       (ilz
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr n) nil)
          (tagbody
            (setf (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                 z-%offset%)
```

```
                    (-
                     (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                    z-%offset%)))
            label20))))))
    (setf (f2cl-lib:fref alphar-%data% (j) ((1 *)) alphar-%offset%)
            (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *)) h-%offset%))
    (setf (f2cl-lib:fref alphai-%data% (j) ((1 *)) alphai-%offset%)
            zero)
    (setf (f2cl-lib:fref beta-%data% (j) ((1 *)) beta-%offset%)
            (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                           t$-%offset%))
   label30))
(if (< ihi ilo)
    (go label380))
(setf ilast ihi)
(cond (ilschr (setf ifrstm 1) (setf ilastm n))
      (t (setf ifrstm ilo) (setf ilastm ihi)))
(setf iiter 0)
(setf eshift zero)
(setf maxit
        (f2cl-lib:int-mul 30
                          (f2cl-lib:int-add (f2cl-lib:int-sub ihi ilo)
                                            1)))
(f2cl-lib:fdo (jiter 1 (f2cl-lib:int-add jiter 1))
              ((> jiter maxit) nil)
  (tagbody
    (cond ((= ilast ilo) (go label80))
          (t
           (cond
            ((<=
              (abs
               (f2cl-lib:fref h
                              (ilast
                               (f2cl-lib:int-add ilast
                                                 (f2cl-lib:int-sub 1)))
                              ((1 ldh) (1 *))))
              atol)
             (setf (f2cl-lib:fref h-%data%
                                  (ilast (f2cl-lib:int-sub ilast 1))
                                  ((1 ldh) (1 *)) h-%offset%)
                   zero)
             (go label80)))))
    (cond
     ((<= (abs (f2cl-lib:fref t$ (ilast ilast) ((1 ldt) (1 *)))) btol)
      (setf (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                           t$-%offset%)
            zero)
      (go label70)))
    (f2cl-lib:fdo (j (f2cl-lib:int-add ilast (f2cl-lib:int-sub 1))
                  (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
```

```
            ((> j ilo) nil)
(tagbody
  (cond ((= j ilo) (setf ilazro f2cl-lib:%true%))
        (t
         (cond
          ((<=
            (abs
             (f2cl-lib:fref h
                            (j
                             (f2cl-lib:int-add j
                                               (f2cl-lib:int-sub
                                                1)))
                            ((1 ldh) (1 *))))
            atol)
           (setf (f2cl-lib:fref h-%data%
                                (j (f2cl-lib:int-sub j 1))
                                ((1 ldh) (1 *)) h-%offset%)
                 zero)
           (setf ilazro f2cl-lib:%true%))
          (t (setf ilazro f2cl-lib:%false%)))))
  (cond
   ((< (abs (f2cl-lib:fref t$ (j j) ((1 ldt) (1 *)))) btol)
    (setf (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                         t$-%offset%)
          zero)
    (setf ilazr2 f2cl-lib:%false%)
    (cond
     ((not ilazro)
      (setf temp
            (abs
             (f2cl-lib:fref h-%data% (j (f2cl-lib:int-sub j 1))
                            ((1 ldh) (1 *)) h-%offset%)))
      (setf temp2
            (abs
             (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *))
                            h-%offset%)))
      (setf tempr (max temp temp2))
      (cond
       ((and (< tempr one) (/= tempr zero))
        (setf temp (/ temp tempr)) (setf temp2 (/ temp2 tempr))))
      (if (<=
           (* temp
              (* ascale
                 (abs
                  (f2cl-lib:fref h-%data%
                                 ((f2cl-lib:int-add j 1) j)
                                 ((1 ldh) (1 *)) h-%offset%))))
           (* temp2 (* ascale atol)))
          (setf ilazr2 f2cl-lib:%true%))))
    (cond
```

```
((or ilazro ilazr2)
 (f2cl-lib:fdo (jch j (f2cl-lib:int-add jch 1))
               ((> jch
                   (f2cl-lib:int-add ilast
                                     (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (setf temp
             (f2cl-lib:fref h-%data% (jch jch)
                            ((1 ldh) (1 *)) h-%offset%))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (dlartg temp
          (f2cl-lib:array-slice h-%data% double-float
                                ((+ jch 1) jch)
                                ((1 ldh) (1 *)) h-%offset%)
          c s
          (f2cl-lib:array-slice h-%data% double-float
                                (jch jch) ((1 ldh) (1 *))
                                h-%offset%))
       (declare (ignore var-1 var-4))
       (when var-0 (setf temp var-0))
       (when var-2 (setf c var-2))
       (when var-3 (setf s var-3)))
     (setf (f2cl-lib:fref h-%data%
                          ((f2cl-lib:int-add jch 1) jch)
                          ((1 ldh) (1 *)) h-%offset%)
             zero)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (drot (f2cl-lib:int-sub ilastm jch)
          (f2cl-lib:array-slice h-%data% double-float
                                (jch
                                 (f2cl-lib:int-add jch 1))
                                ((1 ldh) (1 *)) h-%offset%)
          ldh
          (f2cl-lib:array-slice h-%data% double-float
                                ((+ jch 1)
                                 (f2cl-lib:int-add jch 1))
                                ((1 ldh) (1 *)) h-%offset%)
          ldh c s)
       (declare (ignore var-0 var-1 var-3))
       (when var-2 (setf ldh var-2))
       (when var-4 (setf ldh var-4))
       (when var-5 (setf c var-5))
       (when var-6 (setf s var-6)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (drot (f2cl-lib:int-sub ilastm jch)
          (f2cl-lib:array-slice t$-%data% double-float
                                (jch
```

```
                              (f2cl-lib:int-add jch 1))
                             ((1 ldt) (1 *)) t$-%offset%)
     ldt
     (f2cl-lib:array-slice t$-%data% double-float
                             ((+ jch 1)
                              (f2cl-lib:int-add jch 1))
                             ((1 ldt) (1 *)) t$-%offset%)
     ldt c s)
  (declare (ignore var-0 var-1 var-3))
  (when var-2 (setf ldt var-2))
  (when var-4 (setf ldt var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(if ilq
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (drot n
         (f2cl-lib:array-slice q-%data% double-float
                               (1 jch) ((1 ldq) (1 *))
                               q-%offset%)
         1
         (f2cl-lib:array-slice q-%data% double-float
                               (1
                                (f2cl-lib:int-add jch
                                                  1))
                               ((1 ldq) (1 *))
                               q-%offset%)
         1 c s)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0))
      (when var-5 (setf c var-5))
      (when var-6 (setf s var-6))))
(if ilazr2
    (setf (f2cl-lib:fref h-%data%
                         (jch (f2cl-lib:int-sub jch 1))
                         ((1 ldh) (1 *)) h-%offset%)
            (*
             (f2cl-lib:fref h-%data%
                            (jch
                             (f2cl-lib:int-sub jch 1))
                            ((1 ldh) (1 *)) h-%offset%)
             c)))
(setf ilazr2 f2cl-lib:%false%)
(cond
 ((>=
   (abs
    (f2cl-lib:fref t$
                   ((f2cl-lib:int-add jch 1)
                    (f2cl-lib:int-add jch 1))
                   ((1 ldt) (1 *))))
```

```
        btol)
       (cond
        ((>= (f2cl-lib:int-add jch 1) ilast) (go label80))
        (t (setf ifirst (f2cl-lib:int-add jch 1))
         (go label110)))))
     (setf (f2cl-lib:fref t$-%data%
                          ((f2cl-lib:int-add jch 1)
                           (f2cl-lib:int-add jch 1))
                          ((1 ldt) (1 *)) t$-%offset%)
           zero)
   label40))
 (go label70))
(t
 (f2cl-lib:fdo (jch j (f2cl-lib:int-add jch 1))
               ((> jch
                   (f2cl-lib:int-add ilast
                                     (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (setf temp
           (f2cl-lib:fref t$-%data%
                          (jch (f2cl-lib:int-add jch 1))
                          ((1 ldt) (1 *)) t$-%offset%))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (dlartg temp
          (f2cl-lib:array-slice t$-%data% double-float
                                ((+ jch 1)
                                 (f2cl-lib:int-add jch 1))
                                ((1 ldt) (1 *)) t$-%offset%)
          c s
          (f2cl-lib:array-slice t$-%data% double-float
                                (jch
                                 (f2cl-lib:int-add jch 1))
                                ((1 ldt) (1 *))
                                t$-%offset%))
       (declare (ignore var-1 var-4))
       (when var-0 (setf temp var-0))
       (when var-2 (setf c var-2))
       (when var-3 (setf s var-3)))
     (setf (f2cl-lib:fref t$-%data%
                          ((f2cl-lib:int-add jch 1)
                           (f2cl-lib:int-add jch 1))
                          ((1 ldt) (1 *)) t$-%offset%)
           zero)
     (if (< jch (f2cl-lib:int-sub ilastm 1))
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
             (drot (f2cl-lib:int-sub ilastm jch 1)
              (f2cl-lib:array-slice t$-%data% double-float
                                    (jch
```

```
                                  (f2cl-lib:int-add jch
                                                    2))
                                 ((1 ldt) (1 *))
                                 t$-%offset%)
          ldt
          (f2cl-lib:array-slice t$-%data% double-float
                                 ((+ jch 1)
                                  (f2cl-lib:int-add jch
                                                    2))
                                 ((1 ldt) (1 *))
                                 t$-%offset%)
          ldt c s)
      (declare (ignore var-0 var-1 var-3))
      (when var-2 (setf ldt var-2))
      (when var-4 (setf ldt var-4))
      (when var-5 (setf c var-5))
      (when var-6 (setf s var-6))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (drot
     (f2cl-lib:int-add (f2cl-lib:int-sub ilastm jch) 2)
     (f2cl-lib:array-slice h-%data% double-float
                           (jch
                            (f2cl-lib:int-sub jch 1))
                           ((1 ldh) (1 *)) h-%offset%)
     ldh
     (f2cl-lib:array-slice h-%data% double-float
                           ((+ jch 1)
                            (f2cl-lib:int-sub jch 1))
                           ((1 ldh) (1 *)) h-%offset%)
     ldh c s)
  (declare (ignore var-0 var-1 var-3))
  (when var-2 (setf ldh var-2))
  (when var-4 (setf ldh var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(if ilq
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (drot n
         (f2cl-lib:array-slice q-%data% double-float
                                (1 jch) ((1 ldq) (1 *))
                                q-%offset%)
         1
         (f2cl-lib:array-slice q-%data% double-float
                                (1
                                 (f2cl-lib:int-add jch
                                                   1))
                                ((1 ldq) (1 *))
                                q-%offset%)
```

```
          1 c s)
       (declare (ignore var-1 var-2 var-3 var-4))
       (when var-0 (setf n var-0))
       (when var-5 (setf c var-5))
       (when var-6 (setf s var-6))))
(setf temp
        (f2cl-lib:fref h-%data%
                       ((f2cl-lib:int-add jch 1) jch)
                       ((1 ldh) (1 *)) h-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dlartg temp
     (f2cl-lib:array-slice h-%data% double-float
                           ((+ jch 1)
                            (f2cl-lib:int-sub jch 1))
                           ((1 ldh) (1 *)) h-%offset%)
     c s
     (f2cl-lib:array-slice h-%data% double-float
                           ((+ jch 1) jch)
                           ((1 ldh) (1 *)) h-%offset%))
  (declare (ignore var-1 var-4))
  (when var-0 (setf temp var-0))
  (when var-2 (setf c var-2))
  (when var-3 (setf s var-3)))
(setf (f2cl-lib:fref h-%data%
                     ((f2cl-lib:int-add jch 1)
                      (f2cl-lib:int-sub jch 1))
                     ((1 ldh) (1 *)) h-%offset%)
        zero)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (drot
     (f2cl-lib:int-sub (f2cl-lib:int-add jch 1) ifrstm)
     (f2cl-lib:array-slice h-%data% double-float
                           (ifrstm jch) ((1 ldh) (1 *))
                           h-%offset%)
     1
     (f2cl-lib:array-slice h-%data% double-float
                           (ifrstm
                            (f2cl-lib:int-sub jch 1))
                           ((1 ldh) (1 *)) h-%offset%)
     1 c s)
  (declare (ignore var-0 var-1 var-2 var-3 var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (drot (f2cl-lib:int-sub jch ifrstm)
     (f2cl-lib:array-slice t$-%data% double-float
                           (ifrstm jch) ((1 ldt) (1 *))
                           t$-%offset%)
```

```
                    1
                    (f2cl-lib:array-slice t$-%data% double-float
                                          (ifrstm
                                           (f2cl-lib:int-sub jch 1))
                                          ((1 ldt) (1 *)) t$-%offset%)
                    1 c s)
                  (declare (ignore var-0 var-1 var-2 var-3 var-4))
                  (when var-5 (setf c var-5))
                  (when var-6 (setf s var-6)))
                (if ilz
                    (multiple-value-bind
                        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                        (drot n
                         (f2cl-lib:array-slice z-%data% double-float
                                               (1 jch) ((1 ldz) (1 *))
                                               z-%offset%)
                         1
                         (f2cl-lib:array-slice z-%data% double-float
                                               (1
                                                (f2cl-lib:int-sub jch
                                                                  1))
                                               ((1 ldz) (1 *))
                                               z-%offset%)
                         1 c s)
                      (declare (ignore var-1 var-2 var-3 var-4))
                      (when var-0 (setf n var-0))
                      (when var-5 (setf c var-5))
                      (when var-6 (setf s var-6))))
            label50))
          (go label70))))
      (ilazro (setf ifirst j) (go label110)))
    label60))
 (setf info (f2cl-lib:int-add n 1))
 (go label420)
label70
 (setf temp
         (f2cl-lib:fref h-%data% (ilast ilast) ((1 ldh) (1 *))
                        h-%offset%))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (dlartg temp
      (f2cl-lib:array-slice h-%data% double-float
                            (ilast (f2cl-lib:int-sub ilast 1))
                            ((1 ldh) (1 *)) h-%offset%)
      c s
      (f2cl-lib:array-slice h-%data% double-float (ilast ilast)
                            ((1 ldh) (1 *)) h-%offset%))
   (declare (ignore var-1 var-4))
   (when var-0 (setf temp var-0))
   (when var-2 (setf c var-2))
   (when var-3 (setf s var-3)))
```

```
 (setf (f2cl-lib:fref h-%data% (ilast (f2cl-lib:int-sub ilast 1))
                      ((1 ldh) (1 *)) h-%offset%)
        zero)
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (drot (f2cl-lib:int-sub ilast ifrstm)
      (f2cl-lib:array-slice h-%data% double-float (ifrstm ilast)
                            ((1 ldh) (1 *)) h-%offset%)
      1
      (f2cl-lib:array-slice h-%data% double-float
                            (ifrstm (f2cl-lib:int-sub ilast 1))
                            ((1 ldh) (1 *)) h-%offset%)
      1 c s)
   (declare (ignore var-0 var-1 var-2 var-3 var-4))
   (when var-5 (setf c var-5))
   (when var-6 (setf s var-6)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (drot (f2cl-lib:int-sub ilast ifrstm)
      (f2cl-lib:array-slice t$-%data% double-float (ifrstm ilast)
                            ((1 ldt) (1 *)) t$-%offset%)
      1
      (f2cl-lib:array-slice t$-%data% double-float
                            (ifrstm (f2cl-lib:int-sub ilast 1))
                            ((1 ldt) (1 *)) t$-%offset%)
      1 c s)
   (declare (ignore var-0 var-1 var-2 var-3 var-4))
   (when var-5 (setf c var-5))
   (when var-6 (setf s var-6)))
 (if ilz
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (drot n
          (f2cl-lib:array-slice z-%data% double-float (1 ilast)
                                ((1 ldz) (1 *)) z-%offset%)
          1
          (f2cl-lib:array-slice z-%data% double-float
                                (1 (f2cl-lib:int-sub ilast 1))
                                ((1 ldz) (1 *)) z-%offset%)
          1 c s)
       (declare (ignore var-1 var-2 var-3 var-4))
       (when var-0 (setf n var-0))
       (when var-5 (setf c var-5))
       (when var-6 (setf s var-6))))
label80
 (cond
  ((< (f2cl-lib:fref t$ (ilast ilast) ((1 ldt) (1 *))) zero)
   (cond
    (ilschr
     (f2cl-lib:fdo (j ifrstm (f2cl-lib:int-add j 1))
                   ((> j ilast) nil)
       (tagbody
```

```
          (setf (f2cl-lib:fref h-%data% (j ilast) ((1 ldh) (1 *))
                               h-%offset%)
                  (-
                   (f2cl-lib:fref h-%data% (j ilast) ((1 ldh) (1 *))
                                  h-%offset%)))
          (setf (f2cl-lib:fref t$-%data% (j ilast) ((1 ldt) (1 *))
                               t$-%offset%)
                  (-
                   (f2cl-lib:fref t$-%data% (j ilast) ((1 ldt) (1 *))
                                  t$-%offset%)))
        label90)))
    (t
     (setf (f2cl-lib:fref h-%data% (ilast ilast) ((1 ldh) (1 *))
                           h-%offset%)
             (-
              (f2cl-lib:fref h-%data% (ilast ilast) ((1 ldh) (1 *))
                             h-%offset%)))
     (setf (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                           t$-%offset%)
             (-
              (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                             t$-%offset%)))))
  (cond
   (ilz
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref z-%data% (j ilast) ((1 ldz) (1 *))
                               z-%offset%)
                  (-
                   (f2cl-lib:fref z-%data% (j ilast) ((1 ldz) (1 *))
                                  z-%offset%)))
        label100))))))
(setf (f2cl-lib:fref alphar-%data% (ilast) ((1 *)) alphar-%offset%)
        (f2cl-lib:fref h-%data% (ilast ilast) ((1 ldh) (1 *))
                       h-%offset%))
(setf (f2cl-lib:fref alphai-%data% (ilast) ((1 *)) alphai-%offset%)
        zero)
(setf (f2cl-lib:fref beta-%data% (ilast) ((1 *)) beta-%offset%)
        (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                       t$-%offset%))
(setf ilast (f2cl-lib:int-sub ilast 1))
(if (< ilast ilo)
    (go label380))
(setf iiter 0)
(setf eshift zero)
(cond
 ((not ilschr) (setf ilastm ilast)
  (if (> ifrstm ilast)
      (setf ifrstm ilo))))
```

```
 (go label350)
label110
 (setf iiter (f2cl-lib:int-add iiter 1))
 (cond ((not ilschr) (setf ifrstm ifirst)))
 (cond
  ((= (f2cl-lib:int-mul (f2cl-lib:f2cl/ iiter 10) 10) iiter)
   (cond
    ((<
      (* (f2cl-lib:dble maxit) safmin
         (abs
          (f2cl-lib:fref h
                         (ilast
                          (f2cl-lib:int-add ilast
                                            (f2cl-lib:int-sub 1)))
                         ((1 ldh) (1 *)))))
      (abs
       (f2cl-lib:fref t$
                      ((f2cl-lib:int-add ilast (f2cl-lib:int-sub 1))
                       (f2cl-lib:int-add ilast
                                         (f2cl-lib:int-sub 1)))
                      ((1 ldt) (1 *)))))
     (setf eshift
             (/
              (f2cl-lib:fref h-%data%
                             (ilast (f2cl-lib:int-sub ilast 1))
                             ((1 ldh) (1 *)) h-%offset%)
              (f2cl-lib:fref t$-%data%
                             ((f2cl-lib:int-sub ilast 1)
                              (f2cl-lib:int-sub ilast 1))
                             ((1 ldt) (1 *)) t$-%offset%))))
    (t
     (setf eshift
             (+ eshift (/ one (* safmin (f2cl-lib:dble maxit)))))))
   (setf s1 one) (setf wr eshift))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (dlag2
        (f2cl-lib:array-slice h-%data% double-float
                              ((+ ilast (f2cl-lib:int-sub 1))
                               (f2cl-lib:int-sub ilast 1))
                              ((1 ldh) (1 *)) h-%offset%)
        ldh
        (f2cl-lib:array-slice t$-%data% double-float
                              ((+ ilast (f2cl-lib:int-sub 1))
                               (f2cl-lib:int-sub ilast 1))
                              ((1 ldt) (1 *)) t$-%offset%)
        ldt (* safmin safety) s1 s2 wr wr2 wi)
     (declare (ignore var-0 var-2 var-4))
     (when var-1 (setf ldh var-1))
```

```
      (when var-3 (setf ldt var-3))
      (when var-5 (setf s1 var-5))
      (when var-6 (setf s2 var-6))
      (when var-7 (setf wr var-7))
      (when var-8 (setf wr2 var-8))
      (when var-9 (setf wi var-9)))
    (cond
     ((>
       (abs
        (+
         (* (f2cl-lib:f2cl/ wr s1)
            (f2cl-lib:fref t$ (ilast ilast) ((1 ldt) (1 *))))
         (- (f2cl-lib:fref h (ilast ilast) ((1 ldh) (1 *))))))
       (abs
        (+
         (* (f2cl-lib:f2cl/ wr2 s2)
            (f2cl-lib:fref t$ (ilast ilast) ((1 ldt) (1 *))))
         (- (f2cl-lib:fref h (ilast ilast) ((1 ldh) (1 *)))))))
      (setf temp wr) (setf wr wr2) (setf wr2 temp) (setf temp s1)
      (setf s1 s2) (setf s2 temp)))
    (setf temp (max s1 (* safmin (max one (abs wr) (abs wi)))))
    (if (/= wi zero)
        (go label200))))
  (setf temp (* (min ascale one) (* half safmax)))
  (cond ((> s1 temp) (setf scale (/ temp s1))) (t (setf scale one)))
  (setf temp (* (min bscale one) (* half safmax)))
  (if (> (abs wr) temp)
      (setf scale (min scale (/ temp (abs wr)))))
  (setf s1 (* scale s1))
  (setf wr (* scale wr))
  (f2cl-lib:fdo (j (f2cl-lib:int-add ilast (f2cl-lib:int-sub 1))
                 (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                ((> j (f2cl-lib:int-add ifirst 1)) nil)
    (tagbody
      (setf istart j)
      (setf temp
              (abs
               (* s1
                  (f2cl-lib:fref h-%data% (j (f2cl-lib:int-sub j 1))
                                 ((1 ldh) (1 *)) h-%offset%))))
      (setf temp2
              (abs
               (-
                (* s1
                   (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *))
                                  h-%offset%))
                (* wr
                   (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                                  t$-%offset%)))))
      (setf tempr (max temp temp2))
```

```
        (cond
         ((and (< tempr one) (/= tempr zero))
          (setf temp (/ temp tempr)) (setf temp2 (/ temp2 tempr))))
        (if (<=
             (abs
              (* ascale
                 (f2cl-lib:fref h-%data% ((f2cl-lib:int-add j 1) j)
                                ((1 ldh) (1 *)) h-%offset%)
                 temp))
             (* ascale atol temp2))
            (go label130))
       label120))
    (setf istart ifirst)
   label130
    (setf temp
            (-
             (* s1
                (f2cl-lib:fref h-%data% (istart istart) ((1 ldh) (1 *))
                               h-%offset%))
             (* wr
                (f2cl-lib:fref t$-%data% (istart istart)
                               ((1 ldt) (1 *)) t$-%offset%))))
    (setf temp2
            (* s1
               (f2cl-lib:fref h-%data%
                              ((f2cl-lib:int-add istart 1) istart)
                              ((1 ldh) (1 *)) h-%offset%)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dlartg temp temp2 c s tempr)
      (declare (ignore))
      (when var-0 (setf temp var-0))
      (when var-1 (setf temp2 var-1))
      (when var-2 (setf c var-2))
      (when var-3 (setf s var-3))
      (when var-4 (setf tempr var-4)))
    (f2cl-lib:fdo (j istart (f2cl-lib:int-add j 1))
                  ((> j (f2cl-lib:int-add ilast (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (cond
         ((> j istart)
          (setf temp
                  (f2cl-lib:fref h-%data% (j (f2cl-lib:int-sub j 1))
                                 ((1 ldh) (1 *)) h-%offset%))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dlartg temp
               (f2cl-lib:array-slice h-%data% double-float
                                     ((+ j 1) (f2cl-lib:int-sub j 1))
                                     ((1 ldh) (1 *)) h-%offset%)
               c s
```

```
        (f2cl-lib:array-slice h-%data% double-float
                              (j (f2cl-lib:int-sub j 1))
                              ((1 ldh) (1 *)) h-%offset%))
      (declare (ignore var-1 var-4))
      (when var-0 (setf temp var-0))
      (when var-2 (setf c var-2))
      (when var-3 (setf s var-3)))
    (setf (f2cl-lib:fref h-%data%
                         ((f2cl-lib:int-add j 1)
                          (f2cl-lib:int-sub j 1))
                         ((1 ldh) (1 *)) h-%offset%)
          zero)))
  (f2cl-lib:fdo (jc j (f2cl-lib:int-add jc 1))
                ((> jc ilastm) nil)
    (tagbody
      (setf temp
              (+
               (* c
                  (f2cl-lib:fref h-%data% (j jc) ((1 ldh) (1 *))
                                 h-%offset%))
               (* s
                  (f2cl-lib:fref h-%data%
                                 ((f2cl-lib:int-add j 1) jc)
                                 ((1 ldh) (1 *)) h-%offset%))))
      (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add j 1) jc)
                           ((1 ldh) (1 *)) h-%offset%)
              (+
               (* (- s)
                  (f2cl-lib:fref h-%data% (j jc) ((1 ldh) (1 *))
                                 h-%offset%))
               (* c
                  (f2cl-lib:fref h-%data%
                                 ((f2cl-lib:int-add j 1) jc)
                                 ((1 ldh) (1 *)) h-%offset%))))
      (setf (f2cl-lib:fref h-%data% (j jc) ((1 ldh) (1 *))
                           h-%offset%)
              temp)
      (setf temp2
              (+
               (* c
                  (f2cl-lib:fref t$-%data% (j jc) ((1 ldt) (1 *))
                                 t$-%offset%))
               (* s
                  (f2cl-lib:fref t$-%data%
                                 ((f2cl-lib:int-add j 1) jc)
                                 ((1 ldt) (1 *)) t$-%offset%))))
      (setf (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 1) jc)
                           ((1 ldt) (1 *)) t$-%offset%)
              (+
               (* (- s)
```

```
                  (f2cl-lib:fref t$-%data% (j jc) ((1 ldt) (1 *))
                                 t$-%offset%))
               (* c
                  (f2cl-lib:fref t$-%data%
                                 ((f2cl-lib:int-add j 1) jc)
                                 ((1 ldt) (1 *)) t$-%offset%))))
    (setf (f2cl-lib:fref t$-%data% (j jc) ((1 ldt) (1 *))
                         t$-%offset%)
            temp2)
   label140))
(cond
 (ilq
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
      (setf temp
              (+
               (* c
                  (f2cl-lib:fref q-%data% (jr j)
                                 ((1 ldq) (1 *)) q-%offset%))
               (* s
                  (f2cl-lib:fref q-%data%
                                 (jr (f2cl-lib:int-add j 1))
                                 ((1 ldq) (1 *)) q-%offset%))))
      (setf (f2cl-lib:fref q-%data% (jr (f2cl-lib:int-add j 1))
                           ((1 ldq) (1 *)) q-%offset%)
              (+
               (* (- s)
                  (f2cl-lib:fref q-%data% (jr j)
                                 ((1 ldq) (1 *)) q-%offset%))
               (* c
                  (f2cl-lib:fref q-%data%
                                 (jr (f2cl-lib:int-add j 1))
                                 ((1 ldq) (1 *)) q-%offset%))))
      (setf (f2cl-lib:fref q-%data% (jr j) ((1 ldq) (1 *))
                           q-%offset%)
              temp)
     label150))))
(setf temp
        (f2cl-lib:fref t$-%data%
                       ((f2cl-lib:int-add j 1)
                        (f2cl-lib:int-add j 1))
                       ((1 ldt) (1 *)) t$-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dlartg temp
     (f2cl-lib:array-slice t$-%data% double-float ((+ j 1) j)
                           ((1 ldt) (1 *)) t$-%offset%)
     c s
     (f2cl-lib:array-slice t$-%data% double-float
                           ((+ j 1) (f2cl-lib:int-add j 1))
```

```
                            ((1 ldt) (1 *)) t$-%offset%))
  (declare (ignore var-1 var-4))
  (when var-0 (setf temp var-0))
  (when var-2 (setf c var-2))
  (when var-3 (setf s var-3)))
(setf (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 1) j)
                      ((1 ldt) (1 *)) t$-%offset%)
        zero)
(f2cl-lib:fdo (jr ifrstm (f2cl-lib:int-add jr 1))
              ((> jr
                  (min
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add j 2))
                   (the f2cl-lib:integer4 ilast)))
               nil)
  (tagbody
    (setf temp
            (+
             (* c
                (f2cl-lib:fref h-%data%
                               (jr (f2cl-lib:int-add j 1))
                               ((1 ldh) (1 *)) h-%offset%))
             (* s
                (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                               h-%offset%))))
    (setf (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                         h-%offset%)
            (+
             (* (- s)
                (f2cl-lib:fref h-%data%
                               (jr (f2cl-lib:int-add j 1))
                               ((1 ldh) (1 *)) h-%offset%))
             (* c
                (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                               h-%offset%))))
    (setf (f2cl-lib:fref h-%data% (jr (f2cl-lib:int-add j 1))
                         ((1 ldh) (1 *)) h-%offset%)
            temp)
   label160))
(f2cl-lib:fdo (jr ifrstm (f2cl-lib:int-add jr 1))
              ((> jr j) nil)
  (tagbody
    (setf temp
            (+
             (* c
                (f2cl-lib:fref t$-%data%
                               (jr (f2cl-lib:int-add j 1))
                               ((1 ldt) (1 *)) t$-%offset%))
             (* s
                (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
```

```
                                    t$-%offset%))))
            (setf (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                                 t$-%offset%)
                    (+
                     (* (- s)
                        (f2cl-lib:fref t$-%data%
                                       (jr (f2cl-lib:int-add j 1))
                                       ((1 ldt) (1 *)) t$-%offset%))
                     (* c
                        (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                                       t$-%offset%))))
            (setf (f2cl-lib:fref t$-%data% (jr (f2cl-lib:int-add j 1))
                                 ((1 ldt) (1 *)) t$-%offset%)
                    temp)
           label170))
         (cond
          (ilz
           (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                         ((> jr n) nil)
            (tagbody
              (setf temp
                      (+
                       (* c
                          (f2cl-lib:fref z-%data%
                                         (jr (f2cl-lib:int-add j 1))
                                         ((1 ldz) (1 *)) z-%offset%))
                       (* s
                          (f2cl-lib:fref z-%data% (jr j)
                                         ((1 ldz) (1 *)) z-%offset%))))
              (setf (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                   z-%offset%)
                      (+
                       (* (- s)
                          (f2cl-lib:fref z-%data%
                                         (jr (f2cl-lib:int-add j 1))
                                         ((1 ldz) (1 *)) z-%offset%))
                       (* c
                          (f2cl-lib:fref z-%data% (jr j)
                                         ((1 ldz) (1 *)) z-%offset%))))
              (setf (f2cl-lib:fref z-%data% (jr (f2cl-lib:int-add j 1))
                                   ((1 ldz) (1 *)) z-%offset%)
                      temp)
            label180))))
       label190))
     (go label350)
    label200
     (cond
      ((= (f2cl-lib:int-add ifirst 1) ilast)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
```

```
    (dlasv2
     (f2cl-lib:array-slice t$-%data% double-float
                           ((+ ilast (f2cl-lib:int-sub 1))
                            (f2cl-lib:int-sub ilast 1))
                           ((1 ldt) (1 *)) t$-%offset%)
     (f2cl-lib:array-slice t$-%data% double-float
                           ((+ ilast (f2cl-lib:int-sub 1)) ilast)
                           ((1 ldt) (1 *)) t$-%offset%)
     (f2cl-lib:array-slice t$-%data% double-float (ilast ilast)
                           ((1 ldt) (1 *)) t$-%offset%)
     b22 b11 sr cr sl cl)
  (declare (ignore var-0 var-1 var-2))
  (when var-3 (setf b22 var-3))
  (when var-4 (setf b11 var-4))
  (when var-5 (setf sr var-5))
  (when var-6 (setf cr var-6))
  (when var-7 (setf sl var-7))
  (when var-8 (setf cl var-8)))
(cond
 ((< b11 zero) (setf cr (- cr)) (setf sr (- sr))
  (setf b11 (- b11)) (setf b22 (- b22))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (drot (f2cl-lib:int-sub (f2cl-lib:int-add ilastm 1) ifirst)
     (f2cl-lib:array-slice h-%data% double-float
                           ((+ ilast (f2cl-lib:int-sub 1))
                            (f2cl-lib:int-sub ilast 1))
                           ((1 ldh) (1 *)) h-%offset%)
     ldh
     (f2cl-lib:array-slice h-%data% double-float
                           (ilast (f2cl-lib:int-sub ilast 1))
                           ((1 ldh) (1 *)) h-%offset%)
     ldh cl sl)
  (declare (ignore var-0 var-1 var-3))
  (when var-2 (setf ldh var-2))
  (when var-4 (setf ldh var-4))
  (when var-5 (setf cl var-5))
  (when var-6 (setf sl var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (drot (f2cl-lib:int-sub (f2cl-lib:int-add ilast 1) ifrstm)
     (f2cl-lib:array-slice h-%data% double-float
                           (ifrstm (f2cl-lib:int-sub ilast 1))
                           ((1 ldh) (1 *)) h-%offset%)
     1
     (f2cl-lib:array-slice h-%data% double-float (ifrstm ilast)
                           ((1 ldh) (1 *)) h-%offset%)
     1 cr sr)
  (declare (ignore var-0 var-1 var-2 var-3 var-4))
  (when var-5 (setf cr var-5))
  (when var-6 (setf sr var-6)))
(if (< ilast ilastm)
```

```
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (drot (f2cl-lib:int-sub ilastm ilast)
         (f2cl-lib:array-slice t$-%data% double-float
                               ((+ ilast (f2cl-lib:int-sub 1))
                                (f2cl-lib:int-add ilast 1))
                               ((1 ldt) (1 *)) t$-%offset%)
         ldt
         (f2cl-lib:array-slice t$-%data% double-float
                               (ilast (f2cl-lib:int-add ilast 1))
                               ((1 ldt) (1 *)) t$-%offset%)
         ldt cl sl)
      (declare (ignore var-0 var-1 var-3))
      (when var-2 (setf ldt var-2))
      (when var-4 (setf ldt var-4))
      (when var-5 (setf cl var-5))
      (when var-6 (setf sl var-6))))
(if (< ifrstm (f2cl-lib:int-sub ilast 1))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (drot (f2cl-lib:int-sub ifirst ifrstm)
         (f2cl-lib:array-slice t$-%data% double-float
                               (ifrstm
                                (f2cl-lib:int-sub ilast 1))
                               ((1 ldt) (1 *)) t$-%offset%)
         1
         (f2cl-lib:array-slice t$-%data% double-float
                               (ifrstm ilast) ((1 ldt) (1 *))
                               t$-%offset%)
         1 cr sr)
      (declare (ignore var-0 var-1 var-2 var-3 var-4))
      (when var-5 (setf cr var-5))
      (when var-6 (setf sr var-6))))
(if ilq
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (drot n
         (f2cl-lib:array-slice q-%data% double-float
                               (1 (f2cl-lib:int-sub ilast 1))
                               ((1 ldq) (1 *)) q-%offset%)
         1
         (f2cl-lib:array-slice q-%data% double-float (1 ilast)
                               ((1 ldq) (1 *)) q-%offset%)
         1 cl sl)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0))
      (when var-5 (setf cl var-5))
      (when var-6 (setf sl var-6))))
(if ilz
    (multiple-value-bind
```

```
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (drot n
           (f2cl-lib:array-slice z-%data% double-float
                                 (1 (f2cl-lib:int-sub ilast 1))
                                 ((1 ldz) (1 *)) z-%offset%)
           1
           (f2cl-lib:array-slice z-%data% double-float (1 ilast)
                                 ((1 ldz) (1 *)) z-%offset%)
           1 cr sr)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf n var-0))
        (when var-5 (setf cr var-5))
        (when var-6 (setf sr var-6))))
  (setf (f2cl-lib:fref t$-%data%
                       ((f2cl-lib:int-sub ilast 1)
                        (f2cl-lib:int-sub ilast 1))
                       ((1 ldt) (1 *)) t$-%offset%)
          b11)
  (setf (f2cl-lib:fref t$-%data% ((f2cl-lib:int-sub ilast 1) ilast)
                       ((1 ldt) (1 *)) t$-%offset%)
          zero)
  (setf (f2cl-lib:fref t$-%data% (ilast (f2cl-lib:int-sub ilast 1))
                       ((1 ldt) (1 *)) t$-%offset%)
          zero)
  (setf (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                       t$-%offset%)
          b22)
  (cond
   ((< b22 zero)
    (f2cl-lib:fdo (j ifrstm (f2cl-lib:int-add j 1))
                  ((> j ilast) nil)
      (tagbody
        (setf (f2cl-lib:fref h-%data% (j ilast) ((1 ldh) (1 *))
                             h-%offset%)
              (-
               (f2cl-lib:fref h-%data% (j ilast) ((1 ldh) (1 *))
                              h-%offset%)))
        (setf (f2cl-lib:fref t$-%data% (j ilast) ((1 ldt) (1 *))
                             t$-%offset%)
              (-
               (f2cl-lib:fref t$-%data% (j ilast) ((1 ldt) (1 *))
                              t$-%offset%)))
      label210))
    (cond
     (ilz
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf (f2cl-lib:fref z-%data% (j ilast) ((1 ldz) (1 *))
                               z-%offset%)
```

```
                 (-
                  (f2cl-lib:fref z-%data% (j ilast)
                                 ((1 ldz) (1 *)) z-%offset%)))
       label220))))
  (setf b22 (- b22))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dlag2
     (f2cl-lib:array-slice h-%data% double-float
                           ((+ ilast (f2cl-lib:int-sub 1))
                            (f2cl-lib:int-sub ilast 1))
                           ((1 ldh) (1 *)) h-%offset%)
     ldh
     (f2cl-lib:array-slice t$-%data% double-float
                           ((+ ilast (f2cl-lib:int-sub 1))
                            (f2cl-lib:int-sub ilast 1))
                           ((1 ldt) (1 *)) t$-%offset%)
     ldt (* safmin safety) s1 temp wr temp2 wi)
  (declare (ignore var-0 var-2 var-4))
  (when var-1 (setf ldh var-1))
  (when var-3 (setf ldt var-3))
  (when var-5 (setf s1 var-5))
  (when var-6 (setf temp var-6))
  (when var-7 (setf wr var-7))
  (when var-8 (setf temp2 var-8))
  (when var-9 (setf wi var-9)))
(if (= wi zero)
    (go label350))
(setf s1inv (/ one s1))
(setf a11
        (f2cl-lib:fref h-%data%
                       ((f2cl-lib:int-sub ilast 1)
                        (f2cl-lib:int-sub ilast 1))
                       ((1 ldh) (1 *)) h-%offset%))
(setf a21
        (f2cl-lib:fref h-%data%
                       (ilast (f2cl-lib:int-sub ilast 1))
                       ((1 ldh) (1 *)) h-%offset%))
(setf a12
        (f2cl-lib:fref h-%data%
                       ((f2cl-lib:int-sub ilast 1) ilast)
                       ((1 ldh) (1 *)) h-%offset%))
(setf a22
        (f2cl-lib:fref h-%data% (ilast ilast) ((1 ldh) (1 *))
                       h-%offset%))
(setf c11r (- (* s1 a11) (* wr b11))) (setf c11i (* (- wi) b11))
(setf c12 (* s1 a12)) (setf c21 (* s1 a21))
(setf c22r (- (* s1 a22) (* wr b22))) (setf c22i (* (- wi) b22))
(cond
 ((> (+ (abs c11r) (abs c11i) (abs c12))
```

```
      (+ (abs c21) (abs c22r) (abs c22i)))
  (setf t1
          (multiple-value-bind (ret-val var-0 var-1 var-2)
              (dlapy3 c12 c11r c11i)
            (declare (ignore))
            (when var-0 (setf c12 var-0))
            (when var-1 (setf c11r var-1))
            (when var-2 (setf c11i var-2))
            ret-val))
  (setf cz (/ c12 t1)) (setf szr (/ (- c11r) t1))
  (setf szi (/ (- c11i) t1)))
 (t
  (setf cz
          (multiple-value-bind (ret-val var-0 var-1)
              (dlapy2 c22r c22i)
            (declare (ignore))
            (when var-0 (setf c22r var-0))
            (when var-1 (setf c22i var-1))
            ret-val))
  (cond
   ((<= cz safmin) (setf cz zero) (setf szr one) (setf szi zero))
   (t (setf tempr (/ c22r cz)) (setf tempi (/ c22i cz))
    (setf t1
            (multiple-value-bind (ret-val var-0 var-1)
                (dlapy2 cz c21)
              (declare (ignore))
              (when var-0 (setf cz var-0))
              (when var-1 (setf c21 var-1))
              ret-val))
    (setf cz (/ cz t1)) (setf szr (/ (* (- c21) tempr) t1))
    (setf szi (/ (* c21 tempi) t1))))))
(setf an (+ (abs a11) (abs a12) (abs a21) (abs a22)))
(setf bn (+ (abs b11) (abs b22)))
(setf wabs (+ (abs wr) (abs wi)))
(cond
 ((> (* s1 an) (* wabs bn)) (setf cq (* cz b11))
  (setf sqr (* szr b22)) (setf sqi (* (- szi) b22)))
 (t (setf a1r (+ (* cz a11) (* szr a12))) (setf a1i (* szi a12))
  (setf a2r (+ (* cz a21) (* szr a22))) (setf a2i (* szi a22))
  (setf cq
          (multiple-value-bind (ret-val var-0 var-1)
              (dlapy2 a1r a1i)
            (declare (ignore))
            (when var-0 (setf a1r var-0))
            (when var-1 (setf a1i var-1))
            ret-val))
  (cond
   ((<= cq safmin) (setf cq zero) (setf sqr one) (setf sqi zero))
   (t (setf tempr (/ a1r cq)) (setf tempi (/ a1i cq))
    (setf sqr (+ (* tempr a2r) (* tempi a2i)))
```

```
    (setf sqi (- (* tempi a2r) (* tempr a2i)))))))
(setf t1
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (dlapy3 cq sqr sqi)
          (declare (ignore))
          (when var-0 (setf cq var-0))
          (when var-1 (setf sqr var-1))
          (when var-2 (setf sqi var-2))
          ret-val))
(setf cq (/ cq t1)) (setf sqr (/ sqr t1)) (setf sqi (/ sqi t1))
(setf tempr (- (* sqr szr) (* sqi szi)))
(setf tempi (+ (* sqr szi) (* sqi szr)))
(setf b1r (+ (* cq cz b11) (* tempr b22)))
(setf b1i (* tempi b22))
(setf b1a
        (multiple-value-bind (ret-val var-0 var-1)
            (dlapy2 b1r b1i)
          (declare (ignore))
          (when var-0 (setf b1r var-0))
          (when var-1 (setf b1i var-1))
          ret-val))
(setf b2r (+ (* cq cz b22) (* tempr b11)))
(setf b2i (* (- tempi) b11))
(setf b2a
        (multiple-value-bind (ret-val var-0 var-1)
            (dlapy2 b2r b2i)
          (declare (ignore))
          (when var-0 (setf b2r var-0))
          (when var-1 (setf b2i var-1))
          ret-val))
(setf (f2cl-lib:fref beta-%data% ((f2cl-lib:int-sub ilast 1))
                     ((1 *)) beta-%offset%)
        b1a)
(setf (f2cl-lib:fref beta-%data% (ilast) ((1 *)) beta-%offset%)
        b2a)
(setf (f2cl-lib:fref alphar-%data% ((f2cl-lib:int-sub ilast 1))
                     ((1 *)) alphar-%offset%)
        (* wr b1a s1inv))
(setf (f2cl-lib:fref alphai-%data% ((f2cl-lib:int-sub ilast 1))
                     ((1 *)) alphai-%offset%)
        (* wi b1a s1inv))
(setf (f2cl-lib:fref alphar-%data% (ilast) ((1 *))
                     alphar-%offset%)
        (* wr b2a s1inv))
(setf (f2cl-lib:fref alphai-%data% (ilast) ((1 *))
                     alphai-%offset%)
        (* (- (* wi b2a)) s1inv))
(setf ilast (f2cl-lib:int-sub ifirst 1))
(if (< ilast ilo)
    (go label380))
```

```
 (setf iiter 0) (setf eshift zero)
 (cond
  ((not ilschr) (setf ilastm ilast)
   (if (> ifrstm ilast)
       (setf ifrstm ilo))))
 (go label350))
(t
 (setf ad11
         (/
          (* ascale
             (f2cl-lib:fref h-%data%
                            ((f2cl-lib:int-sub ilast 1)
                             (f2cl-lib:int-sub ilast 1))
                            ((1 ldh) (1 *)) h-%offset%))
          (* bscale
             (f2cl-lib:fref t$-%data%
                            ((f2cl-lib:int-sub ilast 1)
                             (f2cl-lib:int-sub ilast 1))
                            ((1 ldt) (1 *)) t$-%offset%))))
 (setf ad21
         (/
          (* ascale
             (f2cl-lib:fref h-%data%
                            (ilast (f2cl-lib:int-sub ilast 1))
                            ((1 ldh) (1 *)) h-%offset%))
          (* bscale
             (f2cl-lib:fref t$-%data%
                            ((f2cl-lib:int-sub ilast 1)
                             (f2cl-lib:int-sub ilast 1))
                            ((1 ldt) (1 *)) t$-%offset%))))
 (setf ad12
         (/
          (* ascale
             (f2cl-lib:fref h-%data%
                            ((f2cl-lib:int-sub ilast 1) ilast)
                            ((1 ldh) (1 *)) h-%offset%))
          (* bscale
             (f2cl-lib:fref t$-%data% (ilast ilast)
                            ((1 ldt) (1 *)) t$-%offset%))))
 (setf ad22
         (/
          (* ascale
             (f2cl-lib:fref h-%data% (ilast ilast) ((1 ldh) (1 *))
                            h-%offset%))
          (* bscale
             (f2cl-lib:fref t$-%data% (ilast ilast)
                            ((1 ldt) (1 *)) t$-%offset%))))
 (setf u12
         (/
          (f2cl-lib:fref t$-%data%
```

```
                          ((f2cl-lib:int-sub ilast 1) ilast)
                          ((1 ldt) (1 *)) t$-%offset%)
          (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                          t$-%offset%)))
(setf ad11l
        (/
         (* ascale
            (f2cl-lib:fref h-%data% (ifirst ifirst)
                           ((1 ldh) (1 *)) h-%offset%))
         (* bscale
            (f2cl-lib:fref t$-%data% (ifirst ifirst)
                           ((1 ldt) (1 *)) t$-%offset%))))
(setf ad21l
        (/
         (* ascale
            (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-add ifirst 1) ifirst)
                           ((1 ldh) (1 *)) h-%offset%))
         (* bscale
            (f2cl-lib:fref t$-%data% (ifirst ifirst)
                           ((1 ldt) (1 *)) t$-%offset%))))
(setf ad12l
        (/
         (* ascale
            (f2cl-lib:fref h-%data%
                           (ifirst (f2cl-lib:int-add ifirst 1))
                           ((1 ldh) (1 *)) h-%offset%))
         (* bscale
            (f2cl-lib:fref t$-%data%
                           ((f2cl-lib:int-add ifirst 1)
                            (f2cl-lib:int-add ifirst 1))
                           ((1 ldt) (1 *)) t$-%offset%))))
(setf ad22l
        (/
         (* ascale
            (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-add ifirst 1)
                            (f2cl-lib:int-add ifirst 1))
                           ((1 ldh) (1 *)) h-%offset%))
         (* bscale
            (f2cl-lib:fref t$-%data%
                           ((f2cl-lib:int-add ifirst 1)
                            (f2cl-lib:int-add ifirst 1))
                           ((1 ldt) (1 *)) t$-%offset%))))
(setf ad32l
        (/
         (* ascale
            (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-add ifirst 2)
                            (f2cl-lib:int-add ifirst 1))
```

```
                              ((1 ldh) (1 *)) h-%offset%))
          (* bscale
             (f2cl-lib:fref t$-%data%
                              ((f2cl-lib:int-add ifirst 1)
                               (f2cl-lib:int-add ifirst 1))
                              ((1 ldt) (1 *)) t$-%offset%))))
(setf u12l
        (/
         (f2cl-lib:fref t$-%data%
                        (ifirst (f2cl-lib:int-add ifirst 1))
                        ((1 ldt) (1 *)) t$-%offset%)
         (f2cl-lib:fref t$-%data%
                        ((f2cl-lib:int-add ifirst 1)
                         (f2cl-lib:int-add ifirst 1))
                        ((1 ldt) (1 *)) t$-%offset%)))
(setf (f2cl-lib:fref v (1) ((1 3)))
        (+ (- (* (- ad11 ad11l) (- ad22 ad11l)) (* ad12 ad21))
           (* ad21 u12 ad11l)
           (* (- ad12l (* ad11l u12l)) ad21l)))
(setf (f2cl-lib:fref v (2) ((1 3)))
        (*
         (+
          (- ad22l ad11l (* ad21l u12l) (- ad11 ad11l)
             (- ad22 ad11l))
          (* ad21 u12))
         ad21l))
(setf (f2cl-lib:fref v (3) ((1 3))) (* ad32l ad21l))
(setf istart ifirst)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dlarfg 3 (f2cl-lib:array-slice v double-float (1) ((1 3)))
     (f2cl-lib:array-slice v double-float (2) ((1 3))) 1 tau)
  (declare (ignore var-0 var-1 var-2 var-3))
  (when var-4 (setf tau var-4)))
(setf (f2cl-lib:fref v (1) ((1 3))) one)
(f2cl-lib:fdo (j istart (f2cl-lib:int-add j 1))
              ((> j
                  (f2cl-lib:int-add ilast (f2cl-lib:int-sub 2)))
               nil)
  (tagbody
    (cond
     ((> j istart)
      (setf (f2cl-lib:fref v (1) ((1 3)))
              (f2cl-lib:fref h-%data% (j (f2cl-lib:int-sub j 1))
                              ((1 ldh) (1 *)) h-%offset%))
      (setf (f2cl-lib:fref v (2) ((1 3)))
              (f2cl-lib:fref h-%data%
                              ((f2cl-lib:int-add j 1)
                               (f2cl-lib:int-sub j 1))
                              ((1 ldh) (1 *)) h-%offset%))
      (setf (f2cl-lib:fref v (3) ((1 3)))
```

```
            (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-add j 2)
                            (f2cl-lib:int-sub j 1))
                           ((1 ldh) (1 *)) h-%offset%))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dlarfg 3
         (f2cl-lib:array-slice h-%data% double-float
                               (j (f2cl-lib:int-sub j 1))
                               ((1 ldh) (1 *)) h-%offset%)
         (f2cl-lib:array-slice v double-float (2) ((1 3))) 1
         tau)
      (declare (ignore var-0 var-1 var-2 var-3))
      (when var-4 (setf tau var-4)))
    (setf (f2cl-lib:fref v (1) ((1 3))) one)
    (setf (f2cl-lib:fref h-%data%
                         ((f2cl-lib:int-add j 1)
                          (f2cl-lib:int-sub j 1))
                         ((1 ldh) (1 *)) h-%offset%)
            zero)
    (setf (f2cl-lib:fref h-%data%
                         ((f2cl-lib:int-add j 2)
                          (f2cl-lib:int-sub j 1))
                         ((1 ldh) (1 *)) h-%offset%)
            zero)))
  (f2cl-lib:fdo (jc j (f2cl-lib:int-add jc 1))
                ((> jc ilastm) nil)
    (tagbody
      (setf temp
              (* tau
                 (+
                  (f2cl-lib:fref h-%data% (j jc)
                                 ((1 ldh) (1 *)) h-%offset%)
                  (* (f2cl-lib:fref v (2) ((1 3)))
                     (f2cl-lib:fref h-%data%
                                    ((f2cl-lib:int-add j 1) jc)
                                    ((1 ldh) (1 *))
                                    h-%offset%))
                  (* (f2cl-lib:fref v (3) ((1 3)))
                     (f2cl-lib:fref h-%data%
                                    ((f2cl-lib:int-add j 2) jc)
                                    ((1 ldh) (1 *))
                                    h-%offset%)))))
      (setf (f2cl-lib:fref h-%data% (j jc) ((1 ldh) (1 *))
                           h-%offset%)
              (-
               (f2cl-lib:fref h-%data% (j jc) ((1 ldh) (1 *))
                              h-%offset%)
               temp))
      (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add j 1) jc)
                           ((1 ldh) (1 *)) h-%offset%)
```

```
        (-
         (f2cl-lib:fref h-%data%
                        ((f2cl-lib:int-add j 1) jc)
                        ((1 ldh) (1 *)) h-%offset%)
         (* temp (f2cl-lib:fref v (2) ((1 3))))))
 (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add j 2) jc)
                      ((1 ldh) (1 *)) h-%offset%)
        (-
         (f2cl-lib:fref h-%data%
                        ((f2cl-lib:int-add j 2) jc)
                        ((1 ldh) (1 *)) h-%offset%)
         (* temp (f2cl-lib:fref v (3) ((1 3))))))
 (setf temp2
        (* tau
           (+
            (f2cl-lib:fref t$-%data% (j jc)
                           ((1 ldt) (1 *)) t$-%offset%)
            (* (f2cl-lib:fref v (2) ((1 3)))
               (f2cl-lib:fref t$-%data%
                              ((f2cl-lib:int-add j 1) jc)
                              ((1 ldt) (1 *))
                              t$-%offset%))
            (* (f2cl-lib:fref v (3) ((1 3)))
               (f2cl-lib:fref t$-%data%
                              ((f2cl-lib:int-add j 2) jc)
                              ((1 ldt) (1 *))
                              t$-%offset%)))))
 (setf (f2cl-lib:fref t$-%data% (j jc) ((1 ldt) (1 *))
                      t$-%offset%)
        (-
         (f2cl-lib:fref t$-%data% (j jc) ((1 ldt) (1 *))
                        t$-%offset%)
         temp2))
 (setf (f2cl-lib:fref t$-%data%
                      ((f2cl-lib:int-add j 1) jc)
                      ((1 ldt) (1 *)) t$-%offset%)
        (-
         (f2cl-lib:fref t$-%data%
                        ((f2cl-lib:int-add j 1) jc)
                        ((1 ldt) (1 *)) t$-%offset%)
         (* temp2 (f2cl-lib:fref v (2) ((1 3))))))
 (setf (f2cl-lib:fref t$-%data%
                      ((f2cl-lib:int-add j 2) jc)
                      ((1 ldt) (1 *)) t$-%offset%)
        (-
         (f2cl-lib:fref t$-%data%
                        ((f2cl-lib:int-add j 2) jc)
                        ((1 ldt) (1 *)) t$-%offset%)
         (* temp2 (f2cl-lib:fref v (3) ((1 3))))))
label230))
```

```
(cond
 (ilq
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
      (setf temp
              (* tau
                 (+
                  (f2cl-lib:fref q-%data% (jr j)
                                 ((1 ldq) (1 *)) q-%offset%)
                  (* (f2cl-lib:fref v (2) ((1 3)))
                     (f2cl-lib:fref q-%data%
                                    (jr
                                     (f2cl-lib:int-add j 1))
                                    ((1 ldq) (1 *))
                                    q-%offset%))
                  (* (f2cl-lib:fref v (3) ((1 3)))
                     (f2cl-lib:fref q-%data%
                                    (jr
                                     (f2cl-lib:int-add j 2))
                                    ((1 ldq) (1 *))
                                    q-%offset%)))))
      (setf (f2cl-lib:fref q-%data% (jr j) ((1 ldq) (1 *))
                           q-%offset%)
              (-
               (f2cl-lib:fref q-%data% (jr j) ((1 ldq) (1 *))
                              q-%offset%)
               temp))
      (setf (f2cl-lib:fref q-%data%
                           (jr (f2cl-lib:int-add j 1))
                           ((1 ldq) (1 *)) q-%offset%)
              (-
               (f2cl-lib:fref q-%data%
                              (jr (f2cl-lib:int-add j 1))
                              ((1 ldq) (1 *)) q-%offset%)
               (* temp (f2cl-lib:fref v (2) ((1 3))))))
      (setf (f2cl-lib:fref q-%data%
                           (jr (f2cl-lib:int-add j 2))
                           ((1 ldq) (1 *)) q-%offset%)
              (-
               (f2cl-lib:fref q-%data%
                              (jr (f2cl-lib:int-add j 2))
                              ((1 ldq) (1 *)) q-%offset%)
               (* temp (f2cl-lib:fref v (3) ((1 3))))))
     label240))))
(setf ilpivt f2cl-lib:%false%)
(setf temp
        (max
         (abs
          (f2cl-lib:fref t$-%data%
```

```
                            ((f2cl-lib:int-add j 1)
                             (f2cl-lib:int-add j 1))
                            ((1 ldt) (1 *)) t$-%offset%))
                 (abs
                  (f2cl-lib:fref t$-%data%
                            ((f2cl-lib:int-add j 1)
                             (f2cl-lib:int-add j 2))
                            ((1 ldt) (1 *)) t$-%offset%))))
        (setf temp2
                (max
                 (abs
                  (f2cl-lib:fref t$-%data%
                            ((f2cl-lib:int-add j 2)
                             (f2cl-lib:int-add j 1))
                            ((1 ldt) (1 *)) t$-%offset%))
                 (abs
                  (f2cl-lib:fref t$-%data%
                            ((f2cl-lib:int-add j 2)
                             (f2cl-lib:int-add j 2))
                            ((1 ldt) (1 *)) t$-%offset%))))
        (cond
         ((< (max temp temp2) safmin) (setf scale zero) (setf u1 one)
          (setf u2 zero) (go label250))
         ((>= temp temp2)
          (setf w11
                  (f2cl-lib:fref t$-%data%
                            ((f2cl-lib:int-add j 1)
                             (f2cl-lib:int-add j 1))
                            ((1 ldt) (1 *)) t$-%offset%))
          (setf w21
                  (f2cl-lib:fref t$-%data%
                            ((f2cl-lib:int-add j 2)
                             (f2cl-lib:int-add j 1))
                            ((1 ldt) (1 *)) t$-%offset%))
          (setf w12
                  (f2cl-lib:fref t$-%data%
                            ((f2cl-lib:int-add j 1)
                             (f2cl-lib:int-add j 2))
                            ((1 ldt) (1 *)) t$-%offset%))
          (setf w22
                  (f2cl-lib:fref t$-%data%
                            ((f2cl-lib:int-add j 2)
                             (f2cl-lib:int-add j 2))
                            ((1 ldt) (1 *)) t$-%offset%))
          (setf u1
                  (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 1) j)
                            ((1 ldt) (1 *)) t$-%offset%))
          (setf u2
                  (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 2) j)
                            ((1 ldt) (1 *)) t$-%offset%)))
```

```
  (t
   (setf w21
           (f2cl-lib:fref t$-%data%
                          ((f2cl-lib:int-add j 1)
                           (f2cl-lib:int-add j 1))
                          ((1 ldt) (1 *)) t$-%offset%))
   (setf w11
           (f2cl-lib:fref t$-%data%
                          ((f2cl-lib:int-add j 2)
                           (f2cl-lib:int-add j 1))
                          ((1 ldt) (1 *)) t$-%offset%))
   (setf w22
           (f2cl-lib:fref t$-%data%
                          ((f2cl-lib:int-add j 1)
                           (f2cl-lib:int-add j 2))
                          ((1 ldt) (1 *)) t$-%offset%))
   (setf w12
           (f2cl-lib:fref t$-%data%
                          ((f2cl-lib:int-add j 2)
                           (f2cl-lib:int-add j 2))
                          ((1 ldt) (1 *)) t$-%offset%))
   (setf u2
           (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 1) j)
                          ((1 ldt) (1 *)) t$-%offset%))
   (setf u1
           (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 2) j)
                          ((1 ldt) (1 *)) t$-%offset%))))
 (cond
  ((> (abs w12) (abs w11)) (setf ilpivt f2cl-lib:%true%)
   (setf temp w12) (setf temp2 w22) (setf w12 w11)
   (setf w22 w21) (setf w11 temp) (setf w21 temp2)))
 (setf temp (/ w21 w11))
 (setf u2 (- u2 (* temp u1)))
 (setf w22 (- w22 (* temp w12)))
 (setf w21 zero)
 (setf scale one)
 (cond
  ((< (abs w22) safmin) (setf scale zero) (setf u2 one)
   (setf u1 (/ (- w12) w11)) (go label250)))
 (if (< (abs w22) (abs u2))
     (setf scale (abs (/ w22 u2))))
 (if (< (abs w11) (abs u1))
     (setf scale (min scale (abs (/ w11 u1)))))
 (setf u2 (/ (* scale u2) w22))
 (setf u1 (/ (- (* scale u1) (* w12 u2)) w11))
label250
 (cond (ilpivt (setf temp u2) (setf u2 u1) (setf u1 temp)))
 (setf t1
         (f2cl-lib:fsqrt
          (+ (expt scale 2) (expt u1 2) (expt u2 2))))
```

```
(setf tau (+ one (/ scale t1)))
(setf vs (/ (- one) (+ scale t1)))
(setf (f2cl-lib:fref v (1) ((1 3))) one)
(setf (f2cl-lib:fref v (2) ((1 3))) (* vs u1))
(setf (f2cl-lib:fref v (3) ((1 3))) (* vs u2))
(f2cl-lib:fdo (jr ifrstm (f2cl-lib:int-add jr 1))
              ((> jr
                  (min
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add j 3))
                   (the f2cl-lib:integer4 ilast)))
               nil)
  (tagbody
    (setf temp
            (* tau
               (+
                (f2cl-lib:fref h-%data% (jr j)
                               ((1 ldh) (1 *)) h-%offset%)
                (* (f2cl-lib:fref v (2) ((1 3)))
                   (f2cl-lib:fref h-%data%
                                  (jr (f2cl-lib:int-add j 1))
                                  ((1 ldh) (1 *))
                                  h-%offset%))
                (* (f2cl-lib:fref v (3) ((1 3)))
                   (f2cl-lib:fref h-%data%
                                  (jr (f2cl-lib:int-add j 2))
                                  ((1 ldh) (1 *))
                                  h-%offset%)))))
    (setf (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                         h-%offset%)
            (-
             (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                            h-%offset%)
             temp))
    (setf (f2cl-lib:fref h-%data% (jr (f2cl-lib:int-add j 1))
                         ((1 ldh) (1 *)) h-%offset%)
            (-
             (f2cl-lib:fref h-%data%
                            (jr (f2cl-lib:int-add j 1))
                            ((1 ldh) (1 *)) h-%offset%)
             (* temp (f2cl-lib:fref v (2) ((1 3))))))
    (setf (f2cl-lib:fref h-%data% (jr (f2cl-lib:int-add j 2))
                         ((1 ldh) (1 *)) h-%offset%)
            (-
             (f2cl-lib:fref h-%data%
                            (jr (f2cl-lib:int-add j 2))
                            ((1 ldh) (1 *)) h-%offset%)
             (* temp (f2cl-lib:fref v (3) ((1 3))))))
   label260))
(f2cl-lib:fdo (jr ifrstm (f2cl-lib:int-add jr 1))
```

```
                ((> jr (f2cl-lib:int-add j 2)) nil)
      (tagbody
        (setf temp
                (* tau
                   (+
                    (f2cl-lib:fref t$-%data% (jr j)
                                   ((1 ldt) (1 *)) t$-%offset%)
                    (* (f2cl-lib:fref v (2) ((1 3)))
                       (f2cl-lib:fref t$-%data%
                                      (jr (f2cl-lib:int-add j 1))
                                      ((1 ldt) (1 *))
                                      t$-%offset%))
                    (* (f2cl-lib:fref v (3) ((1 3)))
                       (f2cl-lib:fref t$-%data%
                                      (jr (f2cl-lib:int-add j 2))
                                      ((1 ldt) (1 *))
                                      t$-%offset%)))))
        (setf (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                             t$-%offset%)
                (-
                 (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                                t$-%offset%)
                 temp))
        (setf (f2cl-lib:fref t$-%data%
                             (jr (f2cl-lib:int-add j 1))
                             ((1 ldt) (1 *)) t$-%offset%)
                (-
                 (f2cl-lib:fref t$-%data%
                                (jr (f2cl-lib:int-add j 1))
                                ((1 ldt) (1 *)) t$-%offset%)
                 (* temp (f2cl-lib:fref v (2) ((1 3))))))
        (setf (f2cl-lib:fref t$-%data%
                             (jr (f2cl-lib:int-add j 2))
                             ((1 ldt) (1 *)) t$-%offset%)
                (-
                 (f2cl-lib:fref t$-%data%
                                (jr (f2cl-lib:int-add j 2))
                                ((1 ldt) (1 *)) t$-%offset%)
                 (* temp (f2cl-lib:fref v (3) ((1 3))))))
       label270))
    (cond
     (ilz
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf temp
                  (* tau
                     (+
                      (f2cl-lib:fref z-%data% (jr j)
                                     ((1 ldz) (1 *)) z-%offset%)
```

```
                        (* (f2cl-lib:fref v (2) ((1 3)))
                           (f2cl-lib:fref z-%data%
                                          (jr
                                           (f2cl-lib:int-add j 1))
                                          ((1 ldz) (1 *))
                                          z-%offset%))
                        (* (f2cl-lib:fref v (3) ((1 3)))
                           (f2cl-lib:fref z-%data%
                                          (jr
                                           (f2cl-lib:int-add j 2))
                                          ((1 ldz) (1 *))
                                          z-%offset%)))))
          (setf (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                z-%offset%)
                  (-
                   (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                  z-%offset%)
                   temp))
          (setf (f2cl-lib:fref z-%data%
                                (jr (f2cl-lib:int-add j 1))
                                ((1 ldz) (1 *)) z-%offset%)
                  (-
                   (f2cl-lib:fref z-%data%
                                  (jr (f2cl-lib:int-add j 1))
                                  ((1 ldz) (1 *)) z-%offset%)
                   (* temp (f2cl-lib:fref v (2) ((1 3))))))
          (setf (f2cl-lib:fref z-%data%
                                (jr (f2cl-lib:int-add j 2))
                                ((1 ldz) (1 *)) z-%offset%)
                  (-
                   (f2cl-lib:fref z-%data%
                                  (jr (f2cl-lib:int-add j 2))
                                  ((1 ldz) (1 *)) z-%offset%)
                   (* temp (f2cl-lib:fref v (3) ((1 3))))))
        label280))))
    (setf (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 1) j)
                         ((1 ldt) (1 *)) t$-%offset%)
          zero)
    (setf (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 2) j)
                         ((1 ldt) (1 *)) t$-%offset%)
          zero)
   label290))
(setf j (f2cl-lib:int-sub ilast 1))
(setf temp
        (f2cl-lib:fref h-%data% (j (f2cl-lib:int-sub j 1))
                       ((1 ldh) (1 *)) h-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dlartg temp
     (f2cl-lib:array-slice h-%data% double-float
                           ((+ j 1) (f2cl-lib:int-sub j 1))
```

```
                              ((1 ldh) (1 *)) h-%offset%)
     c s
     (f2cl-lib:array-slice h-%data% double-float
                              (j (f2cl-lib:int-sub j 1))
                              ((1 ldh) (1 *)) h-%offset%))
  (declare (ignore var-1 var-4))
  (when var-0 (setf temp var-0))
  (when var-2 (setf c var-2))
  (when var-3 (setf s var-3)))
(setf (f2cl-lib:fref h-%data%
                     ((f2cl-lib:int-add j 1)
                      (f2cl-lib:int-sub j 1))
                     ((1 ldh) (1 *)) h-%offset%)
        zero)
(f2cl-lib:fdo (jc j (f2cl-lib:int-add jc 1))
              ((> jc ilastm) nil)
  (tagbody
    (setf temp
            (+
             (* c
                (f2cl-lib:fref h-%data% (j jc) ((1 ldh) (1 *))
                               h-%offset%))
             (* s
                (f2cl-lib:fref h-%data%
                               ((f2cl-lib:int-add j 1) jc)
                               ((1 ldh) (1 *)) h-%offset%))))
    (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add j 1) jc)
                         ((1 ldh) (1 *)) h-%offset%)
            (+
             (* (- s)
                (f2cl-lib:fref h-%data% (j jc) ((1 ldh) (1 *))
                               h-%offset%))
             (* c
                (f2cl-lib:fref h-%data%
                               ((f2cl-lib:int-add j 1) jc)
                               ((1 ldh) (1 *)) h-%offset%))))
    (setf (f2cl-lib:fref h-%data% (j jc) ((1 ldh) (1 *))
                         h-%offset%)
            temp)
    (setf temp2
            (+
             (* c
                (f2cl-lib:fref t$-%data% (j jc) ((1 ldt) (1 *))
                               t$-%offset%))
             (* s
                (f2cl-lib:fref t$-%data%
                               ((f2cl-lib:int-add j 1) jc)
                               ((1 ldt) (1 *)) t$-%offset%))))
    (setf (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 1) jc)
                         ((1 ldt) (1 *)) t$-%offset%)
```

```
            (+
             (* (- s)
                (f2cl-lib:fref t$-%data% (j jc) ((1 ldt) (1 *))
                               t$-%offset%))
             (* c
                (f2cl-lib:fref t$-%data%
                               ((f2cl-lib:int-add j 1) jc)
                               ((1 ldt) (1 *)) t$-%offset%))))
    (setf (f2cl-lib:fref t$-%data% (j jc) ((1 ldt) (1 *))
                         t$-%offset%)
            temp2)
   label300))
(cond
 (ilq
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
      (setf temp
              (+
               (* c
                  (f2cl-lib:fref q-%data% (jr j) ((1 ldq) (1 *))
                                 q-%offset%))
               (* s
                  (f2cl-lib:fref q-%data%
                                 (jr (f2cl-lib:int-add j 1))
                                 ((1 ldq) (1 *)) q-%offset%))))
      (setf (f2cl-lib:fref q-%data% (jr (f2cl-lib:int-add j 1))
                           ((1 ldq) (1 *)) q-%offset%)
              (+
               (* (- s)
                  (f2cl-lib:fref q-%data% (jr j) ((1 ldq) (1 *))
                                 q-%offset%))
               (* c
                  (f2cl-lib:fref q-%data%
                                 (jr (f2cl-lib:int-add j 1))
                                 ((1 ldq) (1 *)) q-%offset%))))
      (setf (f2cl-lib:fref q-%data% (jr j) ((1 ldq) (1 *))
                           q-%offset%)
              temp)
     label310))))
(setf temp
        (f2cl-lib:fref t$-%data%
                       ((f2cl-lib:int-add j 1)
                        (f2cl-lib:int-add j 1))
                       ((1 ldt) (1 *)) t$-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dlartg temp
     (f2cl-lib:array-slice t$-%data% double-float ((+ j 1) j)
                           ((1 ldt) (1 *)) t$-%offset%)
     c s
```

```
     (f2cl-lib:array-slice t$-%data% double-float
                           ((+ j 1) (f2cl-lib:int-add j 1))
                           ((1 ldt) (1 *)) t$-%offset%))
  (declare (ignore var-1 var-4))
  (when var-0 (setf temp var-0))
  (when var-2 (setf c var-2))
  (when var-3 (setf s var-3)))
(setf (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 1) j)
                     ((1 ldt) (1 *)) t$-%offset%)
        zero)
(f2cl-lib:fdo (jr ifrstm (f2cl-lib:int-add jr 1))
              ((> jr ilast) nil)
  (tagbody
    (setf temp
            (+
             (* c
                (f2cl-lib:fref h-%data%
                               (jr (f2cl-lib:int-add j 1))
                               ((1 ldh) (1 *)) h-%offset%))
             (* s
                (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                               h-%offset%))))
    (setf (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                         h-%offset%)
            (+
             (* (- s)
                (f2cl-lib:fref h-%data%
                               (jr (f2cl-lib:int-add j 1))
                               ((1 ldh) (1 *)) h-%offset%))
             (* c
                (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                               h-%offset%))))
    (setf (f2cl-lib:fref h-%data% (jr (f2cl-lib:int-add j 1))
                         ((1 ldh) (1 *)) h-%offset%)
            temp)
   label320))
(f2cl-lib:fdo (jr ifrstm (f2cl-lib:int-add jr 1))
              ((> jr
                  (f2cl-lib:int-add ilast (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf temp
            (+
             (* c
                (f2cl-lib:fref t$-%data%
                               (jr (f2cl-lib:int-add j 1))
                               ((1 ldt) (1 *)) t$-%offset%))
             (* s
                (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                               t$-%offset%))))
```

```
            (setf (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                                 t$-%offset%)
                  (+
                   (* (- s)
                      (f2cl-lib:fref t$-%data%
                                     (jr (f2cl-lib:int-add j 1))
                                     ((1 ldt) (1 *)) t$-%offset%))
                   (* c
                      (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                                     t$-%offset%))))
            (setf (f2cl-lib:fref t$-%data% (jr (f2cl-lib:int-add j 1))
                                 ((1 ldt) (1 *)) t$-%offset%)
                  temp)
          label330))
        (cond
         (ilz
          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                        ((> jr n) nil)
            (tagbody
              (setf temp
                      (+
                       (* c
                          (f2cl-lib:fref z-%data%
                                         (jr (f2cl-lib:int-add j 1))
                                         ((1 ldz) (1 *)) z-%offset%))
                       (* s
                          (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                         z-%offset%))))
              (setf (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                   z-%offset%)
                      (+
                       (* (- s)
                          (f2cl-lib:fref z-%data%
                                         (jr (f2cl-lib:int-add j 1))
                                         ((1 ldz) (1 *)) z-%offset%))
                       (* c
                          (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                         z-%offset%))))
              (setf (f2cl-lib:fref z-%data% (jr (f2cl-lib:int-add j 1))
                                   ((1 ldz) (1 *)) z-%offset%)
                      temp)
           label340))))))
     (go label350)
    label350
    label360))
 (setf info ilast)
 (go label420)
label380
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))) nil)
```

```
(tagbody
  (cond
   ((< (f2cl-lib:fref t$ (j j) ((1 ldt) (1 *))) zero)
    (cond
     (ilschr
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr j) nil)
        (tagbody
          (setf (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                                h-%offset%)
                  (-
                   (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                                  h-%offset%)))
          (setf (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                                t$-%offset%)
                  (-
                   (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                                  t$-%offset%)))
         label390)))
     (t
      (setf (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *)) h-%offset%)
              (-
               (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *))
                              h-%offset%)))
      (setf (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                            t$-%offset%)
              (-
               (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                              t$-%offset%)))))
    (cond
     (ilz
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                z-%offset%)
                  (-
                   (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                  z-%offset%)))
         label400))))))
  (setf (f2cl-lib:fref alphar-%data% (j) ((1 *)) alphar-%offset%)
          (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *)) h-%offset%))
  (setf (f2cl-lib:fref alphai-%data% (j) ((1 *)) alphai-%offset%)
          zero)
  (setf (f2cl-lib:fref beta-%data% (j) ((1 *)) beta-%offset%)
          (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                         t$-%offset%))
 label410))
(setf info 0)
label420
```

```
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (f2cl-lib:dble n))
        (go end_label)
       end_label
        (return
         (values job compq compz n nil nil nil ldh nil ldt nil nil nil nil ldq
                 nil ldz nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dhgeqz
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::job
                                              fortran-to-lisp::compq
                                              fortran-to-lisp::compz
                                              fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::ldh nil
                                              fortran-to-lisp::ldt nil nil nil
                                              nil fortran-to-lisp::ldq nil
                                              fortran-to-lisp::ldz nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

32.35 dla_geamv

32.35.1 Axiom unit tests

— dla_geamv.input —

```
)set break resume
)sys rm -f dlageamv.output
)spool dlageamv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.35.2 Axiom help page

— dla_geamv.help —

```
=====================================================================
dla_geamv examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dla_geamv (TRANS, M, N, ALPHA, A, LDA, X, INCX, BETA, Y,
           INCY)
           DLA_GEAMV computes a matrix-vector product using a general matrix
           to calculate error bounds.

Function/Subroutine Documentation
   subroutine dla_geamv (integer TRANS, integer M, integer N, double precision
       ALPHA, double precision, dimension( lda, * ) A, integer LDA, double
       precision, dimension( * ) X, integer INCX, double precision BETA,
       double precision, dimension( * ) Y, integer INCY)
       DLA_GEAMV computes a matrix-vector product using a general matrix to
       calculate error bounds.

Purpose:
```

```
  DLA_GEAMV  performs one of the matrix-vector operations

            y := alpha*abs(A)*abs(x) + beta*abs(y),
       or   y := alpha*abs(A)**T*abs(x) + beta*abs(y),

  where alpha and beta are scalars, x and y are vectors and A is an
  m by n matrix.

  This function is primarily used in calculating error bounds.
  To protect against underflow during evaluation, components in
  the resulting vector are perturbed away from zero by (N+1)
  times the underflow threshold.  To prevent unnecessarily large
  errors for block-structure embedded in general matrices,
  "symbolically" zero components are not perturbed.  A zero
  entry is considered "symbolic" if all multiplications involved
  in computing that entry have at least one zero multiplicand.

Parameters:

 TRANS
  TRANS is INTEGER
  On entry, TRANS specifies the operation to be performed as
  follows:

  BLAS_NO_TRANS      y := alpha*abs(A)*abs(x) + beta*abs(y)
  BLAS_TRANS         y := alpha*abs(A**T)*abs(x) + beta*abs(y)
  BLAS_CONJ_TRANS    y := alpha*abs(A**T)*abs(x) + beta*abs(y)

  Unchanged on exit.

 M
  M is INTEGER
  On entry, M specifies the number of rows of the matrix A.
  M must be at least zero.
  Unchanged on exit.

 N
  N is INTEGER
  On entry, N specifies the number of columns of the matrix A.
  N must be at least zero.
  Unchanged on exit.

 ALPHA
  ALPHA is DOUBLE PRECISION
  On entry, ALPHA specifies the scalar alpha.
  Unchanged on exit.

 A
  A is DOUBLE PRECISION array of DIMENSION ( LDA, n )
  Before entry, the leading m by n part of the array A must
```

```
  contain the matrix of coefficients.
  Unchanged on exit.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least
  max( 1, m ).
  Unchanged on exit.

X
  X is DOUBLE PRECISION array, dimension
  ( 1 + ( n - 1 )*abs( INCX ) ) when TRANS = 'N' or 'n'
  and at least
  ( 1 + ( m - 1 )*abs( INCX ) ) otherwise.
  Before entry, the incremented array X must contain the
  vector x.
  Unchanged on exit.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of
  X. INCX must not be zero.
  Unchanged on exit.

BETA
  BETA is DOUBLE PRECISION
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.
  Unchanged on exit.

Y
  Y is DOUBLE PRECISION
  Array of DIMENSION at least
     ( 1 + ( m - 1 )*abs( INCY ) ) when TRANS = 'N' or 'n'
  and at least
     ( 1 + ( n - 1 )*abs( INCY ) ) otherwise.
  Before entry with BETA non-zero, the incremented array Y
  must contain the vector y. On exit, Y is overwritten by the
  updated vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of
  Y. INCY must not be zero.
  Unchanged on exit.
```

32.35.3 fortran code

— dla_geamv.f —

```
*  =====================================================================
      SUBROUTINE DLA_GEAMV ( TRANS, M, N, ALPHA, A, LDA, X, INCX, BETA,
     $                       Y, INCY )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   ALPHA, BETA
      INTEGER            INCX, INCY, LDA, M, N, TRANS
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), X( * ), Y( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            SYMB_ZERO
      DOUBLE PRECISION   TEMP, SAFE1
      INTEGER            I, INFO, IY, J, JX, KX, KY, LENX, LENY
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, DLAMCH
      DOUBLE PRECISION   DLAMCH
*     ..
*     .. External Functions ..
      EXTERNAL           ILATRANS
      INTEGER            ILATRANS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, ABS, SIGN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF     ( .NOT.( ( TRANS.EQ.ILATRANS( 'N' ) )
```

```
     $           .OR. ( TRANS.EQ.ILATRANS( 'T' ) )
     $           .OR. ( TRANS.EQ.ILATRANS( 'C' )) ) ) THEN
         INFO = 1
      ELSE IF( M.LT.0 )THEN
         INFO = 2
      ELSE IF( N.LT.0 )THEN
         INFO = 3
      ELSE IF( LDA.LT.MAX( 1, M ) )THEN
         INFO = 6
      ELSE IF( INCX.EQ.0 )THEN
         INFO = 8
      ELSE IF( INCY.EQ.0 )THEN
         INFO = 11
      END IF
      IF( INFO.NE.0 )THEN
         CALL XERBLA( 'DLA_GEAMV ', INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( ( M.EQ.0 ).OR.( N.EQ.0 ).OR.
     $    ( ( ALPHA.EQ.ZERO ).AND.( BETA.EQ.ONE ) ) )
     $   RETURN
*
*     Set  LENX  and  LENY, the lengths of the vectors x and y, and set
*     up the start points in  X  and  Y.
*
      IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
         LENX = N
         LENY = M
      ELSE
         LENX = M
         LENY = N
      END IF
      IF( INCX.GT.0 )THEN
         KX = 1
      ELSE
         KX = 1 - ( LENX - 1 )*INCX
      END IF
      IF( INCY.GT.0 )THEN
         KY = 1
      ELSE
         KY = 1 - ( LENY - 1 )*INCY
      END IF
*
*     Set SAFE1 essentially to be the underflow threshold times the
*     number of additions in each row.
*
      SAFE1 = DLAMCH( 'Safe minimum' )
```

```
      SAFE1 = (N+1)*SAFE1
*
*     Form  y := alpha*abs(A)*abs(x) + beta*abs(y).
*
*     The O(M*N) SYMB_ZERO tests could be replaced by O(N) queries to
*     the inexact flag.  Still doesn't help change the iteration order
*     to per-column.
*
      IY = KY
      IF ( INCX.EQ.1 ) THEN
         IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
            DO I = 1, LENY
               IF ( BETA .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
                  Y( IY ) = 0.0D+0
               ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
               ELSE
                  SYMB_ZERO = .FALSE.
                  Y( IY ) = BETA * ABS( Y( IY ) )
               END IF
               IF ( ALPHA .NE. ZERO ) THEN
                  DO J = 1, LENX
                     TEMP = ABS( A( I, J ) )
                     SYMB_ZERO = SYMB_ZERO .AND.
     $                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                     Y( IY ) = Y( IY ) + ALPHA*ABS( X( J ) )*TEMP
                  END DO
               END IF

               IF ( .NOT.SYMB_ZERO )
     $              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

               IY = IY + INCY
            END DO
         ELSE
            DO I = 1, LENY
               IF ( BETA .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
                  Y( IY ) = 0.0D+0
               ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
               ELSE
                  SYMB_ZERO = .FALSE.
                  Y( IY ) = BETA * ABS( Y( IY ) )
               END IF
               IF ( ALPHA .NE. ZERO ) THEN
                  DO J = 1, LENX
                     TEMP = ABS( A( J, I ) )
```

```
                SYMB_ZERO = SYMB_ZERO .AND.
$                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                Y( IY ) = Y( IY ) + ALPHA*ABS( X( J ) )*TEMP
             END DO
          END IF

          IF ( .NOT.SYMB_ZERO )
$              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

          IY = IY + INCY
       END DO
    END IF
 ELSE
    IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
       DO I = 1, LENY
          IF ( BETA .EQ. ZERO ) THEN
             SYMB_ZERO = .TRUE.
             Y( IY ) = 0.0D+0
          ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
             SYMB_ZERO = .TRUE.
          ELSE
             SYMB_ZERO = .FALSE.
             Y( IY ) = BETA * ABS( Y( IY ) )
          END IF
          IF ( ALPHA .NE. ZERO ) THEN
             JX = KX
             DO J = 1, LENX
                TEMP = ABS( A( I, J ) )
                SYMB_ZERO = SYMB_ZERO .AND.
$                    ( X( JX ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                Y( IY ) = Y( IY ) + ALPHA*ABS( X( JX ) )*TEMP
                JX = JX + INCX
             END DO
          END IF

          IF (.NOT.SYMB_ZERO)
$              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

          IY = IY + INCY
       END DO
    ELSE
       DO I = 1, LENY
          IF ( BETA .EQ. ZERO ) THEN
             SYMB_ZERO = .TRUE.
             Y( IY ) = 0.0D+0
          ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
             SYMB_ZERO = .TRUE.
          ELSE
```

```
                 SYMB_ZERO = .FALSE.
                 Y( IY ) = BETA * ABS( Y( IY ) )
              END IF
              IF ( ALPHA .NE. ZERO ) THEN
                 JX = KX
                 DO J = 1, LENX
                    TEMP = ABS( A( J, I ) )
                    SYMB_ZERO = SYMB_ZERO .AND.
     $                   ( X( JX ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                    Y( IY ) = Y( IY ) + ALPHA*ABS( X( JX ) )*TEMP
                    JX = JX + INCX
                 END DO
              END IF

              IF (.NOT.SYMB_ZERO)
     $              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

              IY = IY + INCY
           END DO
        END IF

      END IF
*
      RETURN
*
*     End of DLA_GEAMV
*
      END
```

32.35.4 lisp code

— dla_geamv.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dla_geamv (trans m n alpha a lda x incx beta y incy)
    (declare (type (array double-float (*)) y x a)
             (type (double-float) beta alpha)
             (type (f2cl-lib:integer4) incy incx lda n m trans))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (x double-float x-%data% x-%offset%)
         (y double-float y-%data% y-%offset%))
```

```
(prog ((i 0) (info 0) (iy 0) (j 0) (jx 0) (kx 0) (ky 0) (lenx 0) (leny 0)
       (temp 0.0d0) (safe1 0.0d0) (symb_zero nil))
  (declare (type (f2cl-lib:integer4) i info iy j jx kx ky lenx leny)
           (type (double-float) temp safe1)
           (type f2cl-lib:logical symb_zero))
  (setf info 0)
  (cond
   ((not
     (or (= trans (ilatrans "N")) (= trans (ilatrans "T"))
         (= trans (ilatrans "C"))))
    (setf info 1))
   ((< m 0) (setf info 2)) ((< n 0) (setf info 3))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
    (setf info 6))
   ((= incx 0) (setf info 8)) ((= incy 0) (setf info 11)))
  (cond
   ((/= info 0)
    (multiple-value-bind (var-0 var-1)
        (xerbla "DLA_GEAMV " info)
      (declare (ignore var-0))
      (when var-1 (setf info var-1)))
    (go end_label)))
  (if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
      (go end_label))
  (cond ((= trans (ilatrans "N")) (setf lenx n) (setf leny m))
        (t (setf lenx m) (setf leny n)))
  (cond ((> incx 0) (setf kx 1))
        (t
         (setf kx
                 (f2cl-lib:int-sub 1
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-sub lenx 1) incx)))))
  (cond ((> incy 0) (setf ky 1))
        (t
         (setf ky
                 (f2cl-lib:int-sub 1
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-sub leny 1) incy)))))
  (setf safe1 (dlamch "Safe minimum"))
  (setf safe1 (* (f2cl-lib:int-add n 1) safe1))
  (setf iy ky)
  (cond
   ((= incx 1)
    (cond
     ((= trans (ilatrans "N"))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i leny) nil)
        (tagbody
          (cond
           ((= beta zero) (setf symb_zero f2cl-lib:%true%)
```

```
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  0.0d0))
         ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
          (setf symb_zero f2cl-lib:%true%))
         (t (setf symb_zero f2cl-lib:%false%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (* beta
                     (abs
                      (f2cl-lib:fref y-%data% (iy) ((1 *))
                                     y-%offset%))))))
       (cond
        ((/= alpha zero)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j lenx) nil)
           (tagbody
             (setf temp
                     (abs
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%)))
             (setf symb_zero
                     (and symb_zero
                          (or
                           (=
                            (f2cl-lib:fref x-%data% (j) ((1 *))
                                           x-%offset%)
                            zero)
                           (= temp zero))))
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (+
                      (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (* alpha
                         (abs
                          (f2cl-lib:fref x-%data% (j) ((1 *))
                                         x-%offset%))
                         temp)))
             label100001))))
       (if (not symb_zero)
           (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                   (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (f2cl-lib:sign safe1
                                     (f2cl-lib:fref y-%data% (iy)
                                                    ((1 *))
                                                    y-%offset%)))))
       (setf iy (f2cl-lib:int-add iy incy))
      label100000)))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i leny) nil)
     (tagbody
       (cond
```

```
              ((= beta zero) (setf symb_zero f2cl-lib:%true%)
               (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       0.0d0))
              ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
               (setf symb_zero f2cl-lib:%true%))
              (t (setf symb_zero f2cl-lib:%false%)
               (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (* beta
                          (abs
                           (f2cl-lib:fref y-%data% (iy) ((1 *))
                                          y-%offset%))))))
             (cond
              ((/= alpha zero)
               (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                             ((> j lenx) nil)
                 (tagbody
                   (setf temp
                           (abs
                            (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                           a-%offset%)))
                   (setf symb_zero
                           (and symb_zero
                                (or
                                 (=
                                  (f2cl-lib:fref x-%data% (j) ((1 *))
                                                 x-%offset%)
                                  zero)
                                 (= temp zero))))
                   (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                           (+
                            (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                            (* alpha
                               (abs
                                (f2cl-lib:fref x-%data% (j) ((1 *))
                                               x-%offset%))
                               temp)))
                 label100003))))
             (if (not symb_zero)
                 (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                            (f2cl-lib:sign safe1
                                           (f2cl-lib:fref y-%data% (iy)
                                                          ((1 *))
                                                          y-%offset%)))))
             (setf iy (f2cl-lib:int-add iy incy))
            label100002)))))
        (t
         (cond
          ((= trans (ilatrans "N"))
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
            ((> i leny) nil)
(tagbody
  (cond
   ((= beta zero) (setf symb_zero f2cl-lib:%true%)
    (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
            0.0d0))
   ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
    (setf symb_zero f2cl-lib:%true%))
   (t (setf symb_zero f2cl-lib:%false%)
    (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
            (* beta
               (abs
                (f2cl-lib:fref y-%data% (iy) ((1 *))
                               y-%offset%))))))
  (cond
   ((/= alpha zero) (setf jx kx)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j lenx) nil)
      (tagbody
        (setf temp
                (abs
                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)))
        (setf symb_zero
                (and symb_zero
                     (or
                      (=
                       (f2cl-lib:fref x-%data% (jx) ((1 *))
                                      x-%offset%)
                       zero)
                      (= temp zero))))
        (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                (+
                 (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (* alpha
                    (abs
                     (f2cl-lib:fref x-%data% (jx) ((1 *))
                                    x-%offset%))
                    temp)))
        (setf jx (f2cl-lib:int-add jx incx))
      label100005))))
  (if (not symb_zero)
      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
              (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (f2cl-lib:sign safe1
                                (f2cl-lib:fref y-%data% (iy)
                                               ((1 *))
                                               y-%offset%)))))
  (setf iy (f2cl-lib:int-add iy incy))
 label100004)))
```

```
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i leny) nil)
   (tagbody
     (cond
      ((= beta zero) (setf symb_zero f2cl-lib:%true%)
       (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
               0.0d0))
      ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
       (setf symb_zero f2cl-lib:%true%))
      (t (setf symb_zero f2cl-lib:%false%)
       (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
               (* beta
                  (abs
                   (f2cl-lib:fref y-%data% (iy) ((1 *))
                                  y-%offset%))))))
     (cond
      ((/= alpha zero) (setf jx kx)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j lenx) nil)
         (tagbody
           (setf temp
                   (abs
                    (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                   a-%offset%)))
           (setf symb_zero
                   (and symb_zero
                        (or
                         (=
                          (f2cl-lib:fref x-%data% (jx) ((1 *))
                                         x-%offset%)
                          zero)
                         (= temp zero))))
           (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                   (+
                    (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (* alpha
                       (abs
                        (f2cl-lib:fref x-%data% (jx) ((1 *))
                                       x-%offset%))
                       temp)))
           (setf jx (f2cl-lib:int-add jx incx))
          label100007))))
     (if (not symb_zero)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (f2cl-lib:sign safe1
                                   (f2cl-lib:fref y-%data% (iy)
                                                  ((1 *))
                                                  y-%offset%)))))
```

```
                (setf iy (f2cl-lib:int-add iy incy))
               label100006))))))
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dla_geamv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (double-float)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
                                              nil nil nil)
                                            :calls 'nil)))
```

———-

32.36 dla_gercond

32.36.1 Axiom unit tests

— **dlagercond.input** —

```
)set break resume
)sys rm -f dlagercond.output
)spool dlagercond.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.36.2 Axiom help page

— dla_gercond.help —

```
==================================================================
dla_gercond examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function dla_gercond (TRANS, N, A, LDA, AF, LDAF,
           IPIV, CMODE, C, INFO, WORK, IWORK)
           DLA_GERCOND estimates the Skeel condition number for a general
           matrix.

Function/Subroutine Documentation
   double precision function dla_gercond (character TRANS, integer N, double
       precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV,
       integer CMODE, double precision, dimension( * ) C, integer INFO, double
       precision, dimension( * ) WORK, integer, dimension( * ) IWORK)
       DLA_GERCOND estimates the Skeel condition number for a general matrix.

Purpose:
   DLA_GERCOND estimates the Skeel condition number of op(A) * op2(C)
   where op2 is determined by CMODE as follows
   CMODE =  1    op2(C) = C
   CMODE =  0    op2(C) = I
   CMODE = -1    op2(C) = inv(C)
   The Skeel condition number cond(A) = norminf( |inv(A)||A| )
   is computed by computing scaling factors R such that
   diag(R)*A*op2(C) is row equilibrated and computing the standard
   infinity-norm condition number.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate Transpose = Transpose)
```

```
N
  N is INTEGER
  The number of linear equations, i.e., the order of the
  matrix A.  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the N-by-N matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is DOUBLE PRECISION array, dimension (LDAF,N)
  The factors L and U from the factorization
  A = P*L*U as computed by DGETRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from the factorization A = P*L*U
  as computed by DGETRF; row i of the matrix was interchanged
  with row IPIV(i).

CMODE
  CMODE is INTEGER
  Determines op2(C) in the formula op(A) * op2(C) as follows:
  CMODE =  1    op2(C) = C
  CMODE =  0    op2(C) = I
  CMODE = -1    op2(C) = inv(C)

C
  C is DOUBLE PRECISION array, dimension (N)
  The vector C in the formula op(A) * op2(C).

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  i > 0:  The ith argument is invalid.

WORK
  WORK is DOUBLE PRECISION array, dimension (3*N).
  Workspace.

IWORK
```

```
IWORK is INTEGER array, dimension (N).
Workspace.
```

———-

32.36.3 fortran code

— **dla_gercond.f** —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION DLA_GERCOND ( TRANS, N, A, LDA, AF,
     $                                        LDAF, IPIV, CMODE, C,
     $                                        INFO, WORK, IWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            N, LDA, LDAF, INFO, CMODE
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * ), WORK( * ),
     $                   C( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            NOTRANS
      INTEGER            KASE, I, J
      DOUBLE PRECISION   AINVNM, TMP
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLACN2, DGETRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
      DLA_GERCOND = 0.0D+0
*
      INFO = 0
      NOTRANS = LSAME( TRANS, 'N' )
      IF ( .NOT. NOTRANS .AND. .NOT. LSAME(TRANS, 'T')
     $     .AND. .NOT. LSAME(TRANS, 'C') ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLA_GERCOND', -INFO )
         RETURN
      END IF
      IF( N.EQ.0 ) THEN
         DLA_GERCOND = 1.0D+0
         RETURN
      END IF
*
*     Compute the equilibration matrix R such that
*     inv(R)*A*C has unit 1-norm.
*
      IF (NOTRANS) THEN
         DO I = 1, N
            TMP = 0.0D+0
            IF ( CMODE .EQ. 1 ) THEN
               DO J = 1, N
                  TMP = TMP + ABS( A( I, J ) * C( J ) )
               END DO
            ELSE IF ( CMODE .EQ. 0 ) THEN
               DO J = 1, N
                  TMP = TMP + ABS( A( I, J ) )
               END DO
            ELSE
               DO J = 1, N
                  TMP = TMP + ABS( A( I, J ) / C( J ) )
               END DO
            END IF
            WORK( 2*N+I ) = TMP
         END DO
      ELSE
         DO I = 1, N
```

```
            TMP = 0.0D+0
            IF ( CMODE .EQ. 1 ) THEN
               DO J = 1, N
                  TMP = TMP + ABS( A( J, I ) * C( J ) )
               END DO
            ELSE IF ( CMODE .EQ. 0 ) THEN
               DO J = 1, N
                  TMP = TMP + ABS( A( J, I ) )
               END DO
            ELSE
               DO J = 1, N
                  TMP = TMP + ABS( A( J, I ) / C( J ) )
               END DO
            END IF
            WORK( 2*N+I ) = TMP
         END DO
      END IF
*
*     Estimate the norm of inv(op(A)).
*
      AINVNM = 0.0D+0

      KASE = 0
   10 CONTINUE
      CALL DLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*
*           Multiply by R.
*
            DO I = 1, N
               WORK(I) = WORK(I) * WORK(2*N+I)
            END DO

            IF (NOTRANS) THEN
               CALL DGETRS( 'No transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ELSE
               CALL DGETRS( 'Transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            END IF
*
*           Multiply by inv(C).
*
            IF ( CMODE .EQ. 1 ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) / C( I )
               END DO
            ELSE IF ( CMODE .EQ. -1 ) THEN
               DO I = 1, N
```

```
                 WORK( I ) = WORK( I ) * C( I )
              END DO
           END IF
        ELSE
*
*          Multiply by inv(C**T).
*
           IF ( CMODE .EQ. 1 ) THEN
              DO I = 1, N
                 WORK( I ) = WORK( I ) / C( I )
              END DO
           ELSE IF ( CMODE .EQ. -1 ) THEN
              DO I = 1, N
                 WORK( I ) = WORK( I ) * C( I )
              END DO
           END IF

           IF (NOTRANS) THEN
              CALL DGETRS( 'Transpose', N, 1, AF, LDAF, IPIV,
     $           WORK, N, INFO )
           ELSE
              CALL DGETRS( 'No transpose', N, 1, AF, LDAF, IPIV,
     $           WORK, N, INFO )
           END IF
*
*          Multiply by R.
*
           DO I = 1, N
              WORK( I ) = WORK( I ) * WORK( 2*N+I )
           END DO
        END IF
        GO TO 10
     END IF
*
*    Compute the estimate of the reciprocal condition number.
*
     IF( AINVNM .NE. 0.0D+0 )
     $   DLA_GERCOND = ( 1.0D+0 / AINVNM )
*
     RETURN
*
     END
```

———-

32.36.4 lisp code

— dla_gercond.lisp —

```
(defun dla_gercond (trans n a lda af ldaf ipiv cmode c info work iwork)
  (declare (type (array f2cl-lib:integer4 (*)) iwork ipiv)
           (type (array double-float (*)) work c af a)
           (type (f2cl-lib:integer4) info cmode ldaf lda n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (a double-float a-%data% a-%offset%)
       (af double-float af-%data% af-%offset%)
       (c double-float c-%data% c-%offset%)
       (work double-float work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
    (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
           (ainvnm 0.0d0) (tmp 0.0d0) (kase 0) (i 0) (j 0) (notrans nil)
           (dla_gercond 0.0d0))
      (declare (type f2cl-lib:logical notrans)
               (type (f2cl-lib:integer4) j i kase)
               (type (double-float) dla_gercond tmp ainvnm)
               (type (array f2cl-lib:integer4 (3)) isave))
      (setf dla_gercond 0.0d0)
      (setf info 0)
      (setf notrans
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "N")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val))
      (cond
       ((and (not notrans)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "T")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val))
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "C")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -4))
       ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -6)))
      (cond
       ((/= info 0) (xerbla "DLA_GERCOND" (f2cl-lib:int-sub info))
```

```
  (go end_label)))
(cond ((= n 0) (setf dla_gercond 1.0d0) (go end_label)))
(cond
 (notrans
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf tmp 0.0d0)
      (cond
       ((= cmode 1)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (abs
                        (*
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)
                         (f2cl-lib:fref c-%data% (j) ((1 *))
                                        c-%offset%)))))
           label100001)))
       ((= cmode 0)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (abs
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))))
           label100002)))
       (t
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (abs
                        (/
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)
                         (f2cl-lib:fref c-%data% (j) ((1 *))
                                        c-%offset%)))))
           label100003))))
     (setf (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) i))
                          ((1 *)) work-%offset%)
             tmp)
    label100000)))
```

```
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf tmp 0.0d0)
       (cond
        ((= cmode 1)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (abs
                         (*
                          (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref c-%data% (j) ((1 *))
                                         c-%offset%)))))
            label100005)))
        ((= cmode 0)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (abs
                         (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                        a-%offset%))))
            label100006)))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (abs
                         (/
                          (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref c-%data% (j) ((1 *))
                                         c-%offset%)))))
            label100007))))
       (setf (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) i))
                            ((1 *)) work-%offset%)
             tmp)
      label100004))))
 (setf ainvnm 0.0d0)
 (setf kase 0)
label10
```

```
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacn2 n
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1)) ((1 *))
                           work-%offset%)
     work iwork ainvnm kase isave)
  (declare (ignore var-1 var-2 var-3 var-6))
  (setf n var-0)
  (setf ainvnm var-4)
  (setf kase var-5))
(cond
 ((/= kase 0)
  (cond
   ((= kase 2)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add
                                    (f2cl-lib:int-mul 2 n) i))
                                  ((1 *)) work-%offset%)))
       label100008))
    (cond
     (notrans
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dgetrs "No transpose" n 1 af ldaf ipiv work n info)
        (declare (ignore var-0 var-2 var-3 var-5 var-6))
        (setf n var-1)
        (setf ldaf var-4)
        (setf n var-7)
        (setf info var-8)))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dgetrs "Transpose" n 1 af ldaf ipiv work n info)
        (declare (ignore var-0 var-2 var-3 var-5 var-6))
        (setf n var-1)
        (setf ldaf var-4)
        (setf n var-7)
        (setf info var-8))))
    (cond
     ((= cmode 1)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (/
                   (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
```

```
                    (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
        label100009)))
    ((= cmode (f2cl-lib:int-sub 1))
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (*
                    (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                    (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
        label100010)))))
  (t
   (cond
    ((= cmode 1)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (/
                    (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                    (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
        label100011)))
    ((= cmode (f2cl-lib:int-sub 1))
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (*
                    (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                    (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
        label100012))))
   (cond
    (notrans
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (dgetrs "Transpose" n 1 af ldaf ipiv work n info)
       (declare (ignore var-0 var-2 var-3 var-5 var-6))
       (setf n var-1)
       (setf ldaf var-4)
       (setf n var-7)
       (setf info var-8)))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (dgetrs "No transpose" n 1 af ldaf ipiv work n info)
       (declare (ignore var-0 var-2 var-3 var-5 var-6))
       (setf n var-1)
       (setf ldaf var-4)
       (setf n var-7)
       (setf info var-8))))
```

```
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                      (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                         (f2cl-lib:fref work-%data%
                                        ((f2cl-lib:int-add
                                          (f2cl-lib:int-mul 2 n) i))
                                        ((1 *)) work-%offset%)))
             label100013))))
        (go label10)))
      (if (/= ainvnm 0.0d0)
          (setf dla_gercond (/ 1.0d0 ainvnm)))
      (go end_label)
     end_label
      (return
       (values dla_gercond trans n nil nil nil ldaf nil nil nil info nil
               nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dla_gercond
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*)))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::ldaf nil nil nil
                                              fortran-to-lisp::info nil nil)
                                            :calls
                                            '(fortran-to-lisp::dgetrs
                                              fortran-to-lisp::dlacn2))))
```

32.37 dla_gerfsx_extended

32.37.1 Axiom unit tests

— dlagerfsxextended.input —

```
)set break resume
)sys rm -f dlagerfsxextended.output
)spool dlagerfsxextended.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

32.37.2 Axiom help page [Demm05]

— dla_gerfsx_extended.help —

```
======================================================================
dla_gerfsx_extended examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dla_gerfsx_extended (PREC_TYPE, TRANS_TYPE, N, NRHS, A, LDA,
           AF, LDAF, IPIV, COLEQU, C, B, LDB, Y, LDY, BERR_OUT, N_NORMS,
           ERRS_N, ERRS_C, RES, AYB, DY, Y_TAIL, RCOND, ITHRESH, RTHRESH,
           DZ_UB, IGNORE_CWISE, INFO)
           DLA_GERFSX_EXTENDED improves the computed solution to a system of
           linear equations for general matrices by performing extra-precise
           iterative refinement and provides error bounds and backward error
           estimates for the solution.

Function/Subroutine Documentation
```

```
   subroutine dla_gerfsx_extended (integer PREC_TYPE, integer TRANS_TYPE,
       integer N, integer NRHS, double precision, dimension( lda, * ) A,
       integer LDA, double precision, dimension( ldaf, * ) AF, integer LDAF,
       integer, dimension( * ) IPIV, logical COLEQU, double precision,
       dimension( * ) C, double precision, dimension( ldb, * ) B, integer LDB,
       double precision, dimension( ldy, * ) Y, integer LDY, double precision,
       dimension( * ) BERR_OUT, integer N_NORMS, double precision, dimension(
       nrhs, * ) ERRS_N, double precision, dimension( nrhs, * ) ERRS_C, double
       precision, dimension( * ) RES, double precision, dimension( * ) AYB,
       double precision, dimension( * ) DY, double precision, dimension( * )
       Y_TAIL, double precision RCOND, integer ITHRESH, double precision
       RTHRESH, double precision DZ_UB, logical IGNORE_CWISE, integer INFO)
       DLA_GERFSX_EXTENDED improves the computed solution to a system of
       linear equations for general matrices by performing extra-precise
       iterative refinement and provides error bounds and backward error
       estimates for the solution.

Purpose:
   DLA_GERFSX_EXTENDED improves the computed solution to a system of
   linear equations by performing extra-precise iterative refinement
   and provides error bounds and backward error estimates for the solution.
   This subroutine is called by DGERFSX to perform iterative refinement.
   In addition to normwise error bound, the code provides maximum
   componentwise error bound if possible. See comments for ERRS_N
   and ERRS_C for details of the error bounds. Note that this
   subroutine is only resonsible for setting the second fields of
   ERRS_N and ERRS_C.

Parameters:

 PREC_TYPE
   PREC_TYPE is INTEGER
   Specifies the intermediate precision to be used in refinement.
   The value is defined by ILAPREC(P) where P is a CHARACTER and
   P    = 'S':  Single
        = 'D':  Double
        = 'I':  Indigenous
        = 'X', 'E':  Extra

 TRANS_TYPE
   TRANS_TYPE is INTEGER
   Specifies the transposition operation on A.
   The value is defined by ILATRANS(T) where T is a CHARACTER and
   T    = 'N':  No transpose
        = 'T':  Transpose
        = 'C':  Conjugate transpose

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.
```

```
NRHS
  NRHS is INTEGER
  The number of right-hand-sides, i.e., the number of columns of the
  matrix B.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the N-by-N matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is DOUBLE PRECISION array, dimension (LDAF,N)
  The factors L and U from the factorization
  A = P*L*U as computed by DGETRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from the factorization A = P*L*U
  as computed by DGETRF; row i of the matrix was interchanged
  with row IPIV(i).

COLEQU
  COLEQU is LOGICAL
  If .TRUE. then column equilibration was done to A before calling
  this routine. This is needed to compute the solution and error
  bounds correctly.

C
  C is DOUBLE PRECISION array, dimension (N)
  The column scale factors for A. If COLEQU = .FALSE., C
  is not accessed. If C is input, each element of C should be a power
  of the radix to ensure a reliable solution and error estimates.
  Scaling by powers of the radix does not cause rounding errors unless
  the result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  The right-hand-side matrix B.

LDB
```

```
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

Y
  Y is DOUBLE PRECISION array, dimension (LDY,NRHS)
  On entry, the solution matrix X, as computed by DGETRS.
  On exit, the improved solution matrix Y.

LDY
  LDY is INTEGER
  The leading dimension of the array Y.  LDY >= max(1,N).

BERR_OUT
  BERR_OUT is DOUBLE PRECISION array, dimension (NRHS)
  On exit, BERR_OUT(j) contains the componentwise relative backward
  error for right-hand-side j from the formula
  max(i) ( abs(RES(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
  where abs(Z) is the componentwise absolute value of the matrix
  or vector Z. This is computed by DLA_LIN_BERR.

N_NORMS
  N_NORMS is INTEGER
  Determines which error bounds to return (see ERRS_N and ERRS_C).
  If N_NORMS >= 1 return normwise error bounds.
  If N_NORMS >= 2 return componentwise error bounds.

ERRS_N
  ERRS_N is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
           max_j (abs(XTRUE(j,i) - X(j,i)))
            ------------------------------
                    max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERRS_N(i,:) corresponds to the ith
  right-hand side.

  The second index in ERRS_N(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').
```

```
  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.
  See Lapack Working Note 165 for further details and extra cautions.

ERRS_C
  ERRS_C is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                 abs(XTRUE(j,i) - X(j,i))
           max_j ----------------------
                       abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERRS_C is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERRS_C(i,:) corresponds to the ith
  right-hand side.

  The second index in ERRS_C(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
```

```
      be trusted if the previous boolean is true.

   err = 3  Reciprocal condition number: Estimated componentwise
      reciprocal condition number.  Compared with the threshold
      sqrt(n) * slamch('Epsilon') to determine if the error
      estimate is "guaranteed". These reciprocal condition
      numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
      appropriately scaled matrix Z.
      Let Z = S*(A*diag(x)), where x is the solution for the
      current right-hand side and S scales each row of
      A*diag(x) by a power of the radix so all absolute row
      sums of Z are approximately 1.

   This subroutine is only responsible for setting the second field above.
   See Lapack Working Note 165 for further details and extra cautions.

RES
   RES is DOUBLE PRECISION array, dimension (N)
   Workspace to hold the intermediate residual.

AYB
   AYB is DOUBLE PRECISION array, dimension (N)
   Workspace. This can be the same workspace passed for Y_TAIL.

DY
   DY is DOUBLE PRECISION array, dimension (N)
   Workspace to hold the intermediate solution.

Y_TAIL
   Y_TAIL is DOUBLE PRECISION array, dimension (N)
   Workspace to hold the trailing bits of the intermediate solution.

RCOND
   RCOND is DOUBLE PRECISION
   Reciprocal scaled condition number.  This is an estimate of the
   reciprocal Skeel condition number of the matrix A after
   equilibration (if done).  If this is less than the machine
   precision (in particular, if it is zero), the matrix is singular
   to working precision.  Note that the error may still be small even
   if this number is very small and the matrix appears ill-conditioned.

ITHRESH
   ITHRESH is INTEGER
   The maximum number of residual computations allowed for
   refinement. The default is 10. For 'aggressive' set to 100 to
   permit convergence using approximate factorizations or
   factorizations other than LU. If the factorization uses a
   technique other than Gaussian elimination, the guarantees in
   ERRS_N and ERRS_C may no longer be trustworthy.
```

```
 RTHRESH
   RTHRESH is DOUBLE PRECISION
   Determines when to stop refinement if the error estimate stops
   decreasing. Refinement will stop when the next solution no longer
   satisfies norm(dx_{i+1}) < RTHRESH * norm(dx_i) where norm(Z) is
   the infinity norm of Z. RTHRESH satisfies 0 < RTHRESH <= 1. The
   default value is 0.5. For 'aggressive' set to 0.9 to permit
   convergence on extremely ill-conditioned matrices. See LAWN 165
   for more details.

 DZ_UB
   DZ_UB is DOUBLE PRECISION
   Determines when to start considering componentwise convergence.
   Componentwise convergence is only considered after each component
   of the solution Y is stable, which we definte as the relative
   change in each component being less than DZ_UB. The default value
   is 0.25, requiring the first bit to be stable. See LAWN 165 for
   more details.

 IGNORE_CWISE
   IGNORE_CWISE is LOGICAL
   If .TRUE. then ignore componentwise convergence. Default value
   is .FALSE..

 INFO
   INFO is INTEGER
   = 0:  Successful exit.
   < 0:  if INFO = -i, the ith argument to DGETRS had an illegal value
```

32.37.3 fortran code

— dla_gerfsx_extended.f —

```
*  =====================================================================
      SUBROUTINE DLA_GERFSX_EXTENDED( PREC_TYPE, TRANS_TYPE, N, NRHS, A,
     $                                LDA, AF, LDAF, IPIV, COLEQU, C, B,
     $                                LDB, Y, LDY, BERR_OUT, N_NORMS,
     $                                ERRS_N, ERRS_C, RES, AYB, DY,
     $                                Y_TAIL, RCOND, ITHRESH, RTHRESH,
     $                                DZ_UB, IGNORE_CWISE, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
```

```
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDAF, LDB, LDY, N, NRHS, PREC_TYPE,
     $                   TRANS_TYPE, N_NORMS, ITHRESH
      LOGICAL            COLEQU, IGNORE_CWISE
      DOUBLE PRECISION   RTHRESH, DZ_UB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   Y( LDY, * ), RES( * ), DY( * ), Y_TAIL( * )
      DOUBLE PRECISION   C( * ), AYB( * ), RCOND, BERR_OUT( * ),
     $                   ERRS_N( NRHS, * ), ERRS_C( NRHS, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      CHARACTER          TRANS
      INTEGER            CNT, I, J, X_STATE, Z_STATE, Y_PREC_STATE
      DOUBLE PRECISION   YK, DYK, YMIN, NORMY, NORMX, NORMDX, DXRAT,
     $                   DZRAT, PREVNORMDX, PREV_DZ_Z, DXRATMAX,
     $                   DZRATMAX, DX_X, DZ_Z, FINAL_DX_X, FINAL_DZ_Z,
     $                   EPS, HUGEVAL, INCR_THRESH
      LOGICAL            INCR_PREC
*     ..
*     .. Parameters ..
      INTEGER            UNSTABLE_STATE, WORKING_STATE, CONV_STATE,
     $                   NOPROG_STATE, BASE_RESIDUAL, EXTRA_RESIDUAL,
     $                   EXTRA_Y
      PARAMETER          ( UNSTABLE_STATE = 0, WORKING_STATE = 1,
     $                   CONV_STATE = 2, NOPROG_STATE = 3 )
      PARAMETER          ( BASE_RESIDUAL = 0, EXTRA_RESIDUAL = 1,
     $                   EXTRA_Y = 2 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
```

```
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DCOPY, DGETRS, DGEMV, BLAS_DGEMV_X,
     $                   BLAS_DGEMV2_X, DLA_GEAMV, DLA_WWADDW, DLAMCH,
     $                   CHLA_TRANSTYPE, DLA_LIN_BERR
      DOUBLE PRECISION   DLAMCH
      CHARACTER          CHLA_TRANSTYPE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      IF ( INFO.NE.0 ) RETURN
      TRANS = CHLA_TRANSTYPE(TRANS_TYPE)
      EPS = DLAMCH( 'Epsilon' )
      HUGEVAL = DLAMCH( 'Overflow' )
*     Force HUGEVAL to Inf
      HUGEVAL = HUGEVAL * HUGEVAL
*     Using HUGEVAL may lead to spurious underflows.
      INCR_THRESH = DBLE( N ) * EPS
*
      DO J = 1, NRHS
         Y_PREC_STATE = EXTRA_RESIDUAL
         IF ( Y_PREC_STATE .EQ. EXTRA_Y ) THEN
            DO I = 1, N
               Y_TAIL( I ) = 0.0D+0
            END DO
         END IF

         DXRAT = 0.0D+0
         DXRATMAX = 0.0D+0
         DZRAT = 0.0D+0
         DZRATMAX = 0.0D+0
         FINAL_DX_X = HUGEVAL
         FINAL_DZ_Z = HUGEVAL
         PREVNORMDX = HUGEVAL
         PREV_DZ_Z = HUGEVAL
         DZ_Z = HUGEVAL
         DX_X = HUGEVAL

         X_STATE = WORKING_STATE
         Z_STATE = UNSTABLE_STATE
         INCR_PREC = .FALSE.

         DO CNT = 1, ITHRESH
*
*         Compute residual RES = B_s - op(A_s) * Y,
*             op(A) = A, A**T, or A**H depending on TRANS (and type).
```

```
*
            CALL DCOPY( N, B( 1, J ), 1, RES, 1 )
            IF ( Y_PREC_STATE .EQ. BASE_RESIDUAL ) THEN
               CALL DGEMV( TRANS, N, N, -1.0D+0, A, LDA, Y( 1, J ), 1,
     $              1.0D+0, RES, 1 )
            ELSE IF ( Y_PREC_STATE .EQ. EXTRA_RESIDUAL ) THEN
               CALL BLAS_DGEMV_X( TRANS_TYPE, N, N, -1.0D+0, A, LDA,
     $              Y( 1, J ), 1, 1.0D+0, RES, 1, PREC_TYPE )
            ELSE
               CALL BLAS_DGEMV2_X( TRANS_TYPE, N, N, -1.0D+0, A, LDA,
     $              Y( 1, J ), Y_TAIL, 1, 1.0D+0, RES, 1, PREC_TYPE )
            END IF

!        XXX: RES is no longer needed.
            CALL DCOPY( N, RES, 1, DY, 1 )
            CALL DGETRS( TRANS, N, 1, AF, LDAF, IPIV, DY, N, INFO )
*
*         Calculate relative changes DX_X, DZ_Z and ratios DXRAT, DZRAT.
*
            NORMX = 0.0D+0
            NORMY = 0.0D+0
            NORMDX = 0.0D+0
            DZ_Z = 0.0D+0
            YMIN = HUGEVAL
*
            DO I = 1, N
               YK = ABS( Y( I, J ) )
               DYK = ABS( DY( I ) )

               IF ( YK .NE. 0.0D+0 ) THEN
                  DZ_Z = MAX( DZ_Z, DYK / YK )
               ELSE IF ( DYK .NE. 0.0D+0 ) THEN
                  DZ_Z = HUGEVAL
               END IF

               YMIN = MIN( YMIN, YK )

               NORMY = MAX( NORMY, YK )

               IF ( COLEQU ) THEN
                  NORMX = MAX( NORMX, YK * C( I ) )
                  NORMDX = MAX( NORMDX, DYK * C( I ) )
               ELSE
                  NORMX = NORMY
                  NORMDX = MAX( NORMDX, DYK )
               END IF
            END DO

            IF ( NORMX .NE. 0.0D+0 ) THEN
               DX_X = NORMDX / NORMX
```

```
            ELSE IF ( NORMDX .EQ. 0.0D+0 ) THEN
               DX_X = 0.0D+0
            ELSE
               DX_X = HUGEVAL
            END IF

            DXRAT = NORMDX / PREVNORMDX
            DZRAT = DZ_Z / PREV_DZ_Z
*
*         Check termination criteria
*
            IF (.NOT.IGNORE_CWISE
     $           .AND. YMIN*RCOND .LT. INCR_THRESH*NORMY
     $           .AND. Y_PREC_STATE .LT. EXTRA_Y)
     $           INCR_PREC = .TRUE.

            IF ( X_STATE .EQ. NOPROG_STATE .AND. DXRAT .LE. RTHRESH )
     $           X_STATE = WORKING_STATE
            IF ( X_STATE .EQ. WORKING_STATE ) THEN
               IF ( DX_X .LE. EPS ) THEN
                  X_STATE = CONV_STATE
               ELSE IF ( DXRAT .GT. RTHRESH ) THEN
                  IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                     INCR_PREC = .TRUE.
                  ELSE
                     X_STATE = NOPROG_STATE
                  END IF
               ELSE
                  IF ( DXRAT .GT. DXRATMAX ) DXRATMAX = DXRAT
               END IF
               IF ( X_STATE .GT. WORKING_STATE ) FINAL_DX_X = DX_X
            END IF

            IF ( Z_STATE .EQ. UNSTABLE_STATE .AND. DZ_Z .LE. DZ_UB )
     $           Z_STATE = WORKING_STATE
            IF ( Z_STATE .EQ. NOPROG_STATE .AND. DZRAT .LE. RTHRESH )
     $           Z_STATE = WORKING_STATE
            IF ( Z_STATE .EQ. WORKING_STATE ) THEN
               IF ( DZ_Z .LE. EPS ) THEN
                  Z_STATE = CONV_STATE
               ELSE IF ( DZ_Z .GT. DZ_UB ) THEN
                  Z_STATE = UNSTABLE_STATE
                  DZRATMAX = 0.0D+0
                  FINAL_DZ_Z = HUGEVAL
               ELSE IF ( DZRAT .GT. RTHRESH ) THEN
                  IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                     INCR_PREC = .TRUE.
                  ELSE
                     Z_STATE = NOPROG_STATE
                  END IF
```

```
               ELSE
                  IF ( DZRAT .GT. DZRATMAX ) DZRATMAX = DZRAT
               END IF
               IF ( Z_STATE .GT. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
            END IF
*
*           Exit if both normwise and componentwise stopped working,
*           but if componentwise is unstable, let it go at least two
*           iterations.
*
            IF ( X_STATE.NE.WORKING_STATE ) THEN
               IF ( IGNORE_CWISE) GOTO 666
               IF ( Z_STATE.EQ.NOPROG_STATE .OR. Z_STATE.EQ.CONV_STATE )
     $              GOTO 666
               IF ( Z_STATE.EQ.UNSTABLE_STATE .AND. CNT.GT.1 ) GOTO 666
            END IF

            IF ( INCR_PREC ) THEN
               INCR_PREC = .FALSE.
               Y_PREC_STATE = Y_PREC_STATE + 1
               DO I = 1, N
                  Y_TAIL( I ) = 0.0D+0
               END DO
            END IF

            PREVNORMDX = NORMDX
            PREV_DZ_Z = DZ_Z
*
*           Update soluton.
*
            IF ( Y_PREC_STATE .LT. EXTRA_Y ) THEN
               CALL DAXPY( N, 1.0D+0, DY, 1, Y( 1, J ), 1 )
            ELSE
               CALL DLA_WWADDW( N, Y( 1, J ), Y_TAIL, DY )
            END IF

         END DO
*        Target of "IF (Z_STOP .AND. X_STOP)".  Sun's f77 won't EXIT.
 666     CONTINUE
*
*     Set final_* when cnt hits ithresh.
*
         IF ( X_STATE .EQ. WORKING_STATE ) FINAL_DX_X = DX_X
         IF ( Z_STATE .EQ. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
*
*     Compute error bounds
*
         IF (N_NORMS .GE. 1) THEN
            ERRS_N( J, LA_LINRX_ERR_I ) = FINAL_DX_X / (1 - DXRATMAX)
         END IF
```

```
         IF ( N_NORMS .GE. 2 ) THEN
            ERRS_C( J, LA_LINRX_ERR_I ) = FINAL_DZ_Z / (1 - DZRATMAX)
         END IF
*
*     Compute componentwise relative backward error from formula
*         max(i) ( abs(R(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
*     where abs(Z) is the componentwise absolute value of the matrix
*     or vector Z.
*
*         Compute residual RES = B_s - op(A_s) * Y,
*             op(A) = A, A**T, or A**H depending on TRANS (and type).
*
         CALL DCOPY( N, B( 1, J ), 1, RES, 1 )
         CALL DGEMV( TRANS, N, N, -1.0D+0, A, LDA, Y(1,J), 1, 1.0D+0,
     $     RES, 1 )

         DO I = 1, N
            AYB( I ) = ABS( B( I, J ) )
         END DO
*
*     Compute abs(op(A_s))*abs(Y) + abs(B_s).
*
         CALL DLA_GEAMV ( TRANS_TYPE, N, N, 1.0D+0,
     $        A, LDA, Y(1, J), 1, 1.0D+0, AYB, 1 )

         CALL DLA_LIN_BERR ( N, N, 1, RES, AYB, BERR_OUT( J ) )
*
*     End of loop for each RHS.
*
      END DO
*
      RETURN
      END
```

———-

32.37.4 lisp code

— dla_gerfsx_extended.lisp —

```
(let* ((unstable_state 0)
       (working_state 1)
       (conv_state 2)
       (noprog_state 3)
       (base_residual 0)
       (extra_residual 1)
       (extra_y 2)
       (la_linrx_err_i 2))
```

```
(declare (type (f2cl-lib:integer4 0 0) unstable_state)
         (type (f2cl-lib:integer4 1 1) working_state)
         (type (f2cl-lib:integer4 2 2) conv_state)
         (type (f2cl-lib:integer4 3 3) noprog_state)
         (type (f2cl-lib:integer4 0 0) base_residual)
         (type (f2cl-lib:integer4 1 1) extra_residual)
         (type (f2cl-lib:integer4 2 2) extra_y)
         (type (f2cl-lib:integer4 2 2) la_linrx_err_i)
         (ignorable unstable_state working_state conv_state noprog_state
          base_residual extra_residual extra_y la_linrx_err_i))
(defun dla_gerfsx_extended
       (prec_type trans_type n nrhs a lda af ldaf ipiv colequ c b ldb$ y ldy
        berr_out n_norms errs_n errs_c res ayb dy y_tail rcond ithresh
        rthresh dz_ub ignore_cwise info)
  (declare (type (double-float) dz_ub rthresh rcond)
           (type f2cl-lib:logical ignore_cwise colequ)
           (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array double-float (*)) y_tail dy ayb res errs_c errs_n
            berr_out y b c af a)
           (type (f2cl-lib:integer4) info ithresh n_norms ldy ldb$ ldaf lda
            nrhs n trans_type prec_type))
  (f2cl-lib:with-multi-array-data
      ((a double-float a-%data% a-%offset%)
       (af double-float af-%data% af-%offset%)
       (c double-float c-%data% c-%offset%)
       (b double-float b-%data% b-%offset%)
       (y double-float y-%data% y-%offset%)
       (berr_out double-float berr_out-%data% berr_out-%offset%)
       (errs_n double-float errs_n-%data% errs_n-%offset%)
       (errs_c double-float errs_c-%data% errs_c-%offset%)
       (res double-float res-%data% res-%offset%)
       (ayb double-float ayb-%data% ayb-%offset%)
       (dy double-float dy-%data% dy-%offset%)
       (y_tail double-float y_tail-%data% y_tail-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((incr_prec nil) (yk 0.0d0) (dyk 0.0d0) (ymin 0.0d0) (normy 0.0d0)
           (normx 0.0d0) (normdx 0.0d0) (dxrat 0.0d0) (dzrat 0.0d0)
           (prevnormdx 0.0d0) (prev_dz_z 0.0d0) (dxratmax 0.0d0)
           (dzratmax 0.0d0) (dx_x 0.0d0) (dz_z 0.0d0) (final_dx_x 0.0d0)
           (final_dz_z 0.0d0) (eps 0.0d0) (hugeval 0.0d0) (incr_thresh 0.0d0)
           (cnt 0) (i 0) (j 0) (x_state 0) (z_state 0) (y_prec_state 0)
           (trans
            (make-array '(1) :element-type 'character :initial-element #\ )))
      (declare (type f2cl-lib:logical incr_prec)
               (type (double-float) yk dyk ymin normy normx normdx dxrat
                dzrat prevnormdx prev_dz_z dxratmax dzratmax dx_x dz_z
                final_dx_x final_dz_z eps hugeval incr_thresh)
               (type (f2cl-lib:integer4) cnt i j x_state z_state
                y_prec_state)
               (type (string 1) trans))
```

```
(if (/= info 0)
    (go end_label))
(f2cl-lib:f2cl-set-string trans
                          (f2cl-lib:fref-string #'chla_transtype
                                                (trans_type))
                          (string 1))
(setf eps (dlamch "Epsilon"))
(setf hugeval (dlamch "Overflow"))
(setf hugeval (* hugeval hugeval))
(setf incr_thresh (* (f2cl-lib:dble n) eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf y_prec_state extra_residual)
    (cond
     ((= y_prec_state extra_y)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                               y_tail-%offset%)
                0.0d0)
         label100001))))
    (setf dxrat 0.0d0)
    (setf dxratmax 0.0d0)
    (setf dzrat 0.0d0)
    (setf dzratmax 0.0d0)
    (setf final_dx_x hugeval)
    (setf final_dz_z hugeval)
    (setf prevnormdx hugeval)
    (setf prev_dz_z hugeval)
    (setf dz_z hugeval)
    (setf dx_x hugeval)
    (setf x_state working_state)
    (setf z_state unstable_state)
    (setf incr_prec f2cl-lib:%false%)
    (f2cl-lib:fdo (cnt 1 (f2cl-lib:int-add cnt 1))
                  ((> cnt ithresh) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dcopy n
             (f2cl-lib:array-slice b-%data% double-float (1 j)
                                   ((1 ldb$) (1 *)) b-%offset%)
             1 res 1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf n var-0)))
        (cond
         ((= y_prec_state base_residual)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
        var-9 var-10)
       (dgemv trans n n -1.0d0 a lda
        (f2cl-lib:array-slice y-%data% double-float (1 j)
                              ((1 ldy) (1 *)) y-%offset%)
        1 1.0d0 res 1)
     (declare
      (ignore var-3 var-4 var-6 var-7 var-8 var-9 var-10))
     (when var-0 (setf trans var-0))
     (when var-1 (setf n var-1))
     (when var-2 (setf n var-2))
     (when var-5 (setf lda var-5))))
  ((= y_prec_state extra_residual)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10 var-11)
       (blas_dgemv_x trans_type n n -1.0d0 a lda
        (f2cl-lib:array-slice y-%data% double-float (1 j)
                              ((1 ldy) (1 *)) y-%offset%)
        1 1.0d0 res 1 prec_type)
     (declare
      (ignore var-3 var-4 var-6 var-7 var-8 var-9 var-10))
     (when var-0 (setf trans_type var-0))
     (when var-1 (setf n var-1))
     (when var-2 (setf n var-2))
     (when var-5 (setf lda var-5))
     (when var-11 (setf prec_type var-11))))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10 var-11 var-12)
       (blas_dgemv2_x trans_type n n -1.0d0 a lda
        (f2cl-lib:array-slice y-%data% double-float (1 j)
                              ((1 ldy) (1 *)) y-%offset%)
        y_tail 1 1.0d0 res 1 prec_type)
     (declare
      (ignore var-3 var-4 var-6 var-7 var-8 var-9 var-10
       var-11))
     (when var-0 (setf trans_type var-0))
     (when var-1 (setf n var-1))
     (when var-2 (setf n var-2))
     (when var-5 (setf lda var-5))
     (when var-12 (setf prec_type var-12)))))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (dcopy n res 1 dy 1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf n var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (dgetrs trans n 1 af ldaf ipiv dy n info)
   (declare (ignore var-2 var-3 var-5 var-6))
```

```
  (setf trans var-0)
  (setf n var-1)
  (setf ldaf var-4)
  (setf n var-7)
  (setf info var-8))
(setf normx 0.0d0)
(setf normy 0.0d0)
(setf normdx 0.0d0)
(setf dz_z 0.0d0)
(setf ymin hugeval)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf yk
            (abs
             (f2cl-lib:fref y-%data% (i j) ((1 ldy) (1 *))
                            y-%offset%)))
    (setf dyk
            (abs
             (f2cl-lib:fref dy-%data% (i) ((1 *))
                            dy-%offset%)))
    (cond ((/= yk 0.0d0) (setf dz_z (max dz_z (/ dyk yk))))
          ((/= dyk 0.0d0) (setf dz_z hugeval)))
    (setf ymin (min ymin yk))
    (setf normy (max normy yk))
    (cond
     (colequ
      (setf normx
              (max normx
                   (* yk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%))))
      (setf normdx
              (max normdx
                   (* dyk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%)))))
     (t (setf normx normy) (setf normdx (max normdx dyk))))
   label100003))
(cond ((/= normx 0.0d0) (setf dx_x (/ normdx normx)))
      ((= normdx 0.0d0) (setf dx_x 0.0d0))
      (t (setf dx_x hugeval)))
(setf dxrat (/ normdx prevnormdx))
(setf dzrat (/ dz_z prev_dz_z))
(if (and (not ignore_cwise)
         (< (* ymin rcond) (* incr_thresh normy))
         (< y_prec_state extra_y))
    (setf incr_prec f2cl-lib:%true%))
(if (and (= x_state noprog_state) (<= dxrat rthresh))
    (setf x_state working_state))
```

```
(cond
 ((= x_state working_state)
  (cond ((<= dx_x eps) (setf x_state conv_state))
        ((> dxrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf x_state noprog_state))))
        (t
         (if (> dxrat dxratmax)
             (setf dxratmax dxrat))))
  (if (> x_state working_state)
      (setf final_dx_x dx_x))))
(if (and (= z_state unstable_state) (<= dz_z dz_ub))
    (setf z_state working_state))
(if (and (= z_state noprog_state) (<= dzrat rthresh))
    (setf z_state working_state))
(cond
 ((= z_state working_state)
  (cond ((<= dz_z eps) (setf z_state conv_state))
        ((> dz_z dz_ub) (setf z_state unstable_state)
         (setf dzratmax 0.0d0) (setf final_dz_z hugeval))
        ((> dzrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf z_state noprog_state))))
        (t
         (if (> dzrat dzratmax)
             (setf dzratmax dzrat))))
  (if (> z_state working_state)
      (setf final_dz_z dz_z))))
(cond
 ((/= x_state working_state)
  (if ignore_cwise
      (go label666))
  (if (or (= z_state noprog_state) (= z_state conv_state))
      (go label666))
  (if (and (= z_state unstable_state) (> cnt 1))
      (go label666))))
(cond
 (incr_prec (setf incr_prec f2cl-lib:%false%)
  (setf y_prec_state (f2cl-lib:int-add y_prec_state 1))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                           y_tail-%offset%)
              0.0d0)
     label100004))))
```

```
        (setf prevnormdx normdx)
        (setf prev_dz_z dz_z)
        (cond
         ((< y_prec_state extra_y)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
              (daxpy n 1.0d0 dy 1
               (f2cl-lib:array-slice y-%data% double-float (1 j)
                                     ((1 ldy) (1 *)) y-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3 var-4 var-5))
            (when var-0 (setf n var-0))))
         (t
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (dla_wwaddw n
               (f2cl-lib:array-slice y-%data% double-float (1 j)
                                     ((1 ldy) (1 *)) y-%offset%)
               y_tail dy)
            (declare (ignore var-1 var-2 var-3))
            (when var-0 (setf n var-0)))))
      label100002))
  label666
   (if (= x_state working_state)
       (setf final_dx_x dx_x))
   (if (= z_state working_state)
       (setf final_dz_z dz_z))
   (cond
    ((>= n_norms 1)
     (setf (f2cl-lib:fref errs_n-%data% (j la_linrx_err_i)
                          ((1 nrhs) (1 *)) errs_n-%offset%)
           (/ final_dx_x (- 1 dxratmax)))))
   (cond
    ((>= n_norms 2)
     (setf (f2cl-lib:fref errs_c-%data% (j la_linrx_err_i)
                          ((1 nrhs) (1 *)) errs_c-%offset%)
           (/ final_dz_z (- 1 dzratmax)))))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
       (dcopy n
        (f2cl-lib:array-slice b-%data% double-float (1 j)
                              ((1 ldb$) (1 *)) b-%offset%)
        1 res 1)
     (declare (ignore var-1 var-2 var-3 var-4))
     (when var-0 (setf n var-0)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (dgemv trans n n -1.0d0 a lda
        (f2cl-lib:array-slice y-%data% double-float (1 j)
                              ((1 ldy) (1 *)) y-%offset%)
        1 1.0d0 res 1)
     (declare (ignore var-3 var-4 var-6 var-7 var-8 var-9 var-10))
```

```
              (when var-0 (setf trans var-0))
              (when var-1 (setf n var-1))
              (when var-2 (setf n var-2))
              (when var-5 (setf lda var-5)))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref ayb-%data% (i) ((1 *)) ayb-%offset%)
                        (abs
                         (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                        b-%offset%)))
               label100005))
            (dla_geamv trans_type n n 1.0d0 a lda
             (f2cl-lib:array-slice y-%data% double-float (1 j) ((1 ldy) (1 *))
                                   y-%offset%)
             1 1.0d0 ayb 1)
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                (dla_lin_berr n n 1 res ayb
                 (f2cl-lib:array-slice berr_out-%data% double-float (j) ((1 *))
                                       berr_out-%offset%))
              (declare (ignore var-2 var-3 var-4 var-5))
              (when var-0 (setf n var-0))
              (when var-1 (setf n var-1)))
          label100000))
        (go end_label)
       end_label
        (return
         (values prec_type trans_type n nil nil lda nil ldaf nil nil nil nil
                 nil nil nil nil nil nil nil nil nil nil nil nil nil nil nil
                 nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dla_gerfsx_extended
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                             fortran-to-lisp::logical
                                              (array double-float (*))
                                              (array double-float (*))
```

```
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (double-float)
  (fortran-to-lisp::integer4)
  (double-float) (double-float)
  fortran-to-lisp::logical
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::prec_type
  fortran-to-lisp::trans_type
  fortran-to-lisp::n nil nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldaf nil nil nil
  nil nil nil nil nil nil nil nil
  nil nil nil nil nil nil nil nil
  nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::dla_geamv
  fortran-to-lisp::dgetrs))))
```

———-

32.38 dla_gerpvgrw

32.38.1 Axiom unit tests

— dlagerpvgrw.input —

```
)set break resume
)sys rm -f dlagerpvgrw.output
)spool dlagerpvgrw.output
)set message test on
)set message auto off
)clear all

)spool
```

```
)lisp (bye)
```

———-

32.38.2 Axiom help page

— **dla_gerpvgrw.help** —

```
==========================================================================
dla_gerpvgrw examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function dla_gerpvgrw (N, NCOLS, A, LDA, AF, LDAF)
           DLA_GERPVGRW

Function/Subroutine Documentation
   double precision function dla_gerpvgrw (integer N, integer NCOLS, double
       precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldaf, * ) AF, integer LDAF)
       DLA_GERPVGRW

Purpose:
   DLA_GERPVGRW computes the reciprocal pivot growth factor
   norm(A)/norm(U). The "max absolute element" norm is used. If this is
   much less than 1, the stability of the LU factorization of the
   (equilibrated) matrix A could be poor. This also means that the
   solution X, estimated condition numbers, and error bounds could be
   unreliable.

Parameters:

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NCOLS
   NCOLS is INTEGER
   The number of columns of the matrix A. NCOLS >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the N-by-N matrix A.
```

```
LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is DOUBLE PRECISION array, dimension (LDAF,N)
  The factors L and U from the factorization
  A = P*L*U as computed by DGETRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).
```

———-

32.38.3 fortran code

— **dla_gerpvgrw.f** —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION DLA_GERPVGRW( N, NCOLS, A, LDA, AF,
     $         LDAF )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            N, NCOLS, LDA, LDAF
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   AMAX, UMAX, RPVGRW
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
```

```
      RPVGRW = 1.0D+0

      DO J = 1, NCOLS
         AMAX = 0.0D+0
         UMAX = 0.0D+0
         DO I = 1, N
            AMAX = MAX( ABS( A( I, J ) ), AMAX )
         END DO
         DO I = 1, J
            UMAX = MAX( ABS( AF( I, J ) ), UMAX )
         END DO
         IF ( UMAX /= 0.0D+0 ) THEN
            RPVGRW = MIN( AMAX / UMAX, RPVGRW )
         END IF
      END DO
      DLA_GERPVGRW = RPVGRW
      END
```

32.38.4 lisp code

— dla_gerpvgrw.lisp —

```
(defun dla_gerpvgrw (n ncols a lda af ldaf)
  (declare (type (array double-float (*)) af a)
           (type (f2cl-lib:integer4) ldaf lda ncols n))
  (f2cl-lib:with-multi-array-data
      ((a double-float a-%data% a-%offset%)
       (af double-float af-%data% af-%offset%))
    (prog ((amax 0.0d0) (umax 0.0d0) (rpvgrw 0.0d0) (i 0) (j 0)
           (dla_gerpvgrw 0.0d0))
      (declare (type (f2cl-lib:integer4) j i)
               (type (double-float) dla_gerpvgrw rpvgrw umax amax))
      (setf rpvgrw 1.0d0)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j ncols) nil)
        (tagbody
          (setf amax 0.0d0)
          (setf umax 0.0d0)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf amax
                      (max
                       (abs
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))
```

```
                    amax))
            label100001))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf umax
                      (max
                       (abs
                        (f2cl-lib:fref af-%data% (i j) ((1 ldaf) (1 *))
                                       af-%offset%))
                       umax))
            label100002))
          (cond
           ((f2cl-lib:f2cl/ umax (= 0.0d0))
            (setf rpvgrw (min (/ amax umax) rpvgrw))))
        label100000))
      (setf dla_gerpvgrw rpvgrw)
     end_label
      (return (values dla_gerpvgrw nil nil nil nil nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dla_gerpvgrw
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil) :calls
                                            'nil)))
```

———-

32.39 dtgevc

32.39.1 Axiom unit tests

— dtgevc.input —

```
)set break resume
```

```
)sys rm -f dtgevc.output
)spool dtgevc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

32.39.2 Axiom help page

— dtgevc.help —

```
=====================================================================
dtgevc examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dtgevc (SIDE, HOWMNY, SELECT, N, S, LDS, P, LDP, VL, LDVL,
           VR, LDVR, MM, M, WORK, INFO)
           DTGEVC

Function/Subroutine Documentation
   subroutine dtgevc (character SIDE, character HOWMNY, logical, dimension( *
       ) SELECT, integer N, double precision, dimension( lds, * ) S, integer
       LDS, double precision, dimension( ldp, * ) P, integer LDP, double
       precision, dimension( ldvl, * ) VL, integer LDVL, double precision,
       dimension( ldvr, * ) VR, integer LDVR, integer MM, integer M, double
       precision, dimension( * ) WORK, integer INFO)
       DTGEVC

Purpose:
   DTGEVC computes some or all of the right and/or left eigenvectors of
   a pair of real matrices (S,P), where S is a quasi-triangular matrix
   and P is upper triangular.  Matrix pairs of this type are produced by
   the generalized Schur factorization of a matrix pair (A,B):

   A = Q*S*Z**T,  B = Q*P*Z**T

   as computed by DGGHRD + DHGEQZ.
```

```
   The right eigenvector x and the left eigenvector y of (S,P)
   corresponding to an eigenvalue w are defined by:

   S*x = w*P*x,  (y**H)*S = w*(y**H)*P,

   where y**H denotes the conjugate tranpose of y.
   The eigenvalues are not input to this routine, but are computed
   directly from the diagonal blocks of S and P.

   This routine returns the matrices X and/or Y of right and left
   eigenvectors of (S,P), or the products Z*X and/or Q*Y,
   where Z and Q are input matrices.
   If Q and Z are the orthogonal factors from the generalized Schur
   factorization of a matrix pair (A,B), then Z*X and Q*Y
   are the matrices of right and left eigenvectors of (A,B).

Parameters:

 SIDE
   SIDE is CHARACTER*1
   = 'R': compute right eigenvectors only;
   = 'L': compute left eigenvectors only;
   = 'B': compute both right and left eigenvectors.

 HOWMNY
   HOWMNY is CHARACTER*1
   = 'A': compute all right and/or left eigenvectors;
   = 'B': compute all right and/or left eigenvectors,
      backtransformed by the matrices in VR and/or VL;
   = 'S': compute selected right and/or left eigenvectors,
      specified by the logical array SELECT.

 SELECT
   SELECT is LOGICAL array, dimension (N)
   If HOWMNY='S', SELECT specifies the eigenvectors to be
   computed.  If w(j) is a real eigenvalue, the corresponding
   real eigenvector is computed if SELECT(j) is .TRUE..
   If w(j) and w(j+1) are the real and imaginary parts of a
   complex eigenvalue, the corresponding complex eigenvector
   is computed if either SELECT(j) or SELECT(j+1) is .TRUE.,
   and on exit SELECT(j) is set to .TRUE. and SELECT(j+1) is
   set to .FALSE..
   Not referenced if HOWMNY = 'A' or 'B'.

 N
   N is INTEGER
   The order of the matrices S and P.  N >= 0.

 S
   S is DOUBLE PRECISION array, dimension (LDS,N)
```

```
  The upper quasi-triangular matrix S from a generalized Schur
  factorization, as computed by DHGEQZ.

LDS
  LDS is INTEGER
  The leading dimension of array S.  LDS >= max(1,N).

P
  P is DOUBLE PRECISION array, dimension (LDP,N)
  The upper triangular matrix P from a generalized Schur
  factorization, as computed by DHGEQZ.
  2-by-2 diagonal blocks of P corresponding to 2-by-2 blocks
  of S must be in positive diagonal form.

LDP
  LDP is INTEGER
  The leading dimension of array P.  LDP >= max(1,N).

VL
  VL is DOUBLE PRECISION array, dimension (LDVL,MM)
  On entry, if SIDE = 'L' or 'B' and HOWMNY = 'B', VL must
  contain an N-by-N matrix Q (usually the orthogonal matrix Q
  of left Schur vectors returned by DHGEQZ).
  On exit, if SIDE = 'L' or 'B', VL contains:
  if HOWMNY = 'A', the matrix Y of left eigenvectors of (S,P);
  if HOWMNY = 'B', the matrix Q*Y;
  if HOWMNY = 'S', the left eigenvectors of (S,P) specified by
                   SELECT, stored consecutively in the columns of
                   VL, in the same order as their eigenvalues.

  A complex eigenvector corresponding to a complex eigenvalue
  is stored in two consecutive columns, the first holding the
  real part, and the second the imaginary part.

  Not referenced if SIDE = 'R'.

LDVL
  LDVL is INTEGER
  The leading dimension of array VL.  LDVL >= 1, and if
  SIDE = 'L' or 'B', LDVL >= N.

VR
  VR is DOUBLE PRECISION array, dimension (LDVR,MM)
  On entry, if SIDE = 'R' or 'B' and HOWMNY = 'B', VR must
  contain an N-by-N matrix Z (usually the orthogonal matrix Z
  of right Schur vectors returned by DHGEQZ).

  On exit, if SIDE = 'R' or 'B', VR contains:
  if HOWMNY = 'A', the matrix X of right eigenvectors of (S,P);
  if HOWMNY = 'B' or 'b', the matrix Z*X;
```

```
   if HOWMNY = 'S' or 's', the right eigenvectors of (S,P)
                specified by SELECT, stored consecutively in the
                columns of VR, in the same order as their eigenvalues.

   A complex eigenvector corresponding to a complex eigenvalue
   is stored in two consecutive columns, the first holding the
   real part and the second the imaginary part.

   Not referenced if SIDE = 'L'.

 LDVR
   LDVR is INTEGER
   The leading dimension of the array VR.  LDVR >= 1, and if
   SIDE = 'R' or 'B', LDVR >= N.

 MM
   MM is INTEGER
   The number of columns in the arrays VL and/or VR. MM >= M.

 M
   M is INTEGER
   The number of columns in the arrays VL and/or VR actually
   used to store the eigenvectors.  If HOWMNY = 'A' or 'B', M
   is set to N.  Each selected real eigenvector occupies one
   column and each selected complex eigenvector occupies two columns.

 WORK
   WORK is DOUBLE PRECISION array, dimension (6*N)

 INFO
   INFO is INTEGER
   = 0:  successful exit.
   < 0:  if INFO = -i, the i-th argument had an illegal value.
   > 0:  the 2-by-2 block (INFO:INFO+1) does not have a complex eigenvalue.

Further Details:

   Allocation of workspace:
   ---------- -- ---------

   WORK( j ) = 1-norm of j-th column of A, above the diagonal
   WORK( N+j ) = 1-norm of j-th column of B, above the diagonal
   WORK( 2*N+1:3*N ) = real part of eigenvector
   WORK( 3*N+1:4*N ) = imaginary part of eigenvector
   WORK( 4*N+1:5*N ) = real part of back-transformed eigenvector
   WORK( 5*N+1:6*N ) = imaginary part of back-transformed eigenvector

   Rowwise vs. columnwise solution methods:
   ------- --  ---------- -------- -------
```

```
Finding a generalized eigenvector consists basically of solving the
singular triangular system

(A - w B) x = 0     (for right) or:   (A - w B)**H y = 0  (for left)

Consider finding the i-th right eigenvector (assume all eigenvalues
are real). The equation to be solved is:
         n                   i
    0 = sum  C(j,k) v(k)  = sum  C(j,k) v(k)     for j = i,. . .,1
        k=j                 k=j

where  C = (A - w B)  (The components v(i+1:n) are 0.)

The "rowwise" method is:

(1)  v(i) := 1
     for j = i-1,. . .,1:
                                  i
     (2) compute  s = - sum C(j,k) v(k)   and
                       k=j+1

 (3) v(j) := s / C(j,j)

Step 2 is sometimes called the "dot product" step, since it is an
inner product between the j-th row and the portion of the eigenvector
that has been computed so far.

The "columnwise" method consists basically in doing the sums
for all the rows in parallel.  As each v(j) is computed, the
contribution of v(j) times the j-th column of C is added to the
partial sums.  Since FORTRAN arrays are stored columnwise, this has
the advantage that at each step, the elements of C that are accessed
are adjacent to one another, whereas with the rowwise method, the
elements accessed at a step are spaced LDS (and LDP) words apart.

When finding left eigenvectors, the matrix in question is the
transpose of the one in storage, so the rowwise method then
actually accesses columns of A and B at each step, and so is the
preferred method.
```

32.39.3 fortran code

— dtgevc.f —

```
*  ====================================================================
```

```
      SUBROUTINE DTGEVC( SIDE, HOWMNY, SELECT, N, S, LDS, P, LDP, VL,
     $                   LDVL, VR, LDVR, MM, M, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          HOWMNY, SIDE
      INTEGER            INFO, LDP, LDS, LDVL, LDVR, M, MM, N
*     ..
*     .. Array Arguments ..
      LOGICAL            SELECT( * )
      DOUBLE PRECISION   P( LDP, * ), S( LDS, * ), VL( LDVL, * ),
     $                   VR( LDVR, * ), WORK( * )
*     ..
*
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE, SAFETY
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0,
     $                   SAFETY = 1.0D+2 )
*     ..
*     .. Local Scalars ..
      LOGICAL            COMPL, COMPR, IL2BY2, ILABAD, ILALL, ILBACK,
     $                   ILBBAD, ILCOMP, ILCPLX, LSA, LSB
      INTEGER            I, IBEG, IEIG, IEND, IHWMNY, IINFO, IM, ISIDE,
     $                   J, JA, JC, JE, JR, JW, NA, NW
      DOUBLE PRECISION   ACOEF, ACOEFA, ANORM, ASCALE, BCOEFA, BCOEFI,
     $                   BCOEFR, BIG, BIGNUM, BNORM, BSCALE, CIM2A,
     $                   CIM2B, CIMAGA, CIMAGB, CRE2A, CRE2B, CREALA,
     $                   CREALB, DMIN, SAFMIN, SALFAR, SBETA, SCALE,
     $                   SMALL, TEMP, TEMP2, TEMP2I, TEMP2R, ULP, XMAX,
     $                   XSCALE
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   BDIAG( 2 ), SUM( 2, 2 ), SUMS( 2, 2 ),
     $                   SUMP( 2, 2 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEMV, DLABAD, DLACPY, DLAG2, DLALN2, XERBLA
*     ..
```

```
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Decode and Test the input parameters
*
      IF( LSAME( HOWMNY, 'A' ) ) THEN
         IHWMNY = 1
         ILALL = .TRUE.
         ILBACK = .FALSE.
      ELSE IF( LSAME( HOWMNY, 'S' ) ) THEN
         IHWMNY = 2
         ILALL = .FALSE.
         ILBACK = .FALSE.
      ELSE IF( LSAME( HOWMNY, 'B' ) ) THEN
         IHWMNY = 3
         ILALL = .TRUE.
         ILBACK = .TRUE.
      ELSE
         IHWMNY = -1
         ILALL = .TRUE.
      END IF
*
      IF( LSAME( SIDE, 'R' ) ) THEN
         ISIDE = 1
         COMPL = .FALSE.
         COMPR = .TRUE.
      ELSE IF( LSAME( SIDE, 'L' ) ) THEN
         ISIDE = 2
         COMPL = .TRUE.
         COMPR = .FALSE.
      ELSE IF( LSAME( SIDE, 'B' ) ) THEN
         ISIDE = 3
         COMPL = .TRUE.
         COMPR = .TRUE.
      ELSE
         ISIDE = -1
      END IF
*
      INFO = 0
      IF( ISIDE.LT.0 ) THEN
         INFO = -1
      ELSE IF( IHWMNY.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDS.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDP.LT.MAX( 1, N ) ) THEN
```

```
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DTGEVC', -INFO )
         RETURN
      END IF
*
*     Count the number of eigenvectors to be computed
*
      IF( .NOT.ILALL ) THEN
         IM = 0
         ILCPLX = .FALSE.
         DO 10 J = 1, N
            IF( ILCPLX ) THEN
               ILCPLX = .FALSE.
               GO TO 10
            END IF
            IF( J.LT.N ) THEN
               IF( S( J+1, J ).NE.ZERO )
     $            ILCPLX = .TRUE.
            END IF
            IF( ILCPLX ) THEN
               IF( SELECT( J ) .OR. SELECT( J+1 ) )
     $            IM = IM + 2
            ELSE
               IF( SELECT( J ) )
     $            IM = IM + 1
            END IF
   10    CONTINUE
      ELSE
         IM = N
      END IF
*
*     Check 2-by-2 diagonal blocks of A, B
*
      ILABAD = .FALSE.
      ILBBAD = .FALSE.
      DO 20 J = 1, N - 1
         IF( S( J+1, J ).NE.ZERO ) THEN
            IF( P( J, J ).EQ.ZERO .OR. P( J+1, J+1 ).EQ.ZERO .OR.
     $          P( J, J+1 ).NE.ZERO )ILBBAD = .TRUE.
            IF( J.LT.N-1 ) THEN
               IF( S( J+2, J+1 ).NE.ZERO )
     $            ILABAD = .TRUE.
            END IF
         END IF
   20 CONTINUE
*
      IF( ILABAD ) THEN
         INFO = -5
```

```
      ELSE IF( ILBBAD ) THEN
         INFO = -7
      ELSE IF( COMPL .AND. LDVL.LT.N .OR. LDVL.LT.1 ) THEN
         INFO = -10
      ELSE IF( COMPR .AND. LDVR.LT.N .OR. LDVR.LT.1 ) THEN
         INFO = -12
      ELSE IF( MM.LT.IM ) THEN
         INFO = -13
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DTGEVC', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      M = IM
      IF( N.EQ.0 )
     $   RETURN
*
*     Machine Constants
*
      SAFMIN = DLAMCH( 'Safe minimum' )
      BIG = ONE / SAFMIN
      CALL DLABAD( SAFMIN, BIG )
      ULP = DLAMCH( 'Epsilon' )*DLAMCH( 'Base' )
      SMALL = SAFMIN*N / ULP
      BIG = ONE / SMALL
      BIGNUM = ONE / ( SAFMIN*N )
*
*     Compute the 1-norm of each column of the strictly upper triangular
*     part (i.e., excluding all elements belonging to the diagonal
*     blocks) of A and B to check for possible overflow in the
*     triangular solver.
*
      ANORM = ABS( S( 1, 1 ) )
      IF( N.GT.1 )
     $   ANORM = ANORM + ABS( S( 2, 1 ) )
      BNORM = ABS( P( 1, 1 ) )
      WORK( 1 ) = ZERO
      WORK( N+1 ) = ZERO
*
      DO 50 J = 2, N
         TEMP = ZERO
         TEMP2 = ZERO
         IF( S( J, J-1 ).EQ.ZERO ) THEN
            IEND = J - 1
         ELSE
            IEND = J - 2
         END IF
```

```
         DO 30 I = 1, IEND
            TEMP = TEMP + ABS( S( I, J ) )
            TEMP2 = TEMP2 + ABS( P( I, J ) )
   30    CONTINUE
         WORK( J ) = TEMP
         WORK( N+J ) = TEMP2
         DO 40 I = IEND + 1, MIN( J+1, N )
            TEMP = TEMP + ABS( S( I, J ) )
            TEMP2 = TEMP2 + ABS( P( I, J ) )
   40    CONTINUE
         ANORM = MAX( ANORM, TEMP )
         BNORM = MAX( BNORM, TEMP2 )
   50 CONTINUE
*
      ASCALE = ONE / MAX( ANORM, SAFMIN )
      BSCALE = ONE / MAX( BNORM, SAFMIN )
*
*     Left eigenvectors
*
      IF( COMPL ) THEN
         IEIG = 0
*
*        Main loop over eigenvalues
*
         ILCPLX = .FALSE.
         DO 220 JE = 1, N
*
*           Skip this iteration if (a) HOWMNY='S' and SELECT=.FALSE., or
*           (b) this would be the second of a complex pair.
*           Check for complex eigenvalue, so as to be sure of which
*           entry(-ies) of SELECT to look at.
*
            IF( ILCPLX ) THEN
               ILCPLX = .FALSE.
               GO TO 220
            END IF
            NW = 1
            IF( JE.LT.N ) THEN
               IF( S( JE+1, JE ).NE.ZERO ) THEN
                  ILCPLX = .TRUE.
                  NW = 2
               END IF
            END IF
            IF( ILALL ) THEN
               ILCOMP = .TRUE.
            ELSE IF( ILCPLX ) THEN
               ILCOMP = SELECT( JE ) .OR. SELECT( JE+1 )
            ELSE
               ILCOMP = SELECT( JE )
            END IF
```

```
            IF( .NOT.ILCOMP )
     $         GO TO 220
*
*           Decide if (a) singular pencil, (b) real eigenvalue, or
*           (c) complex eigenvalue.
*
            IF( .NOT.ILCPLX ) THEN
               IF( ABS( S( JE, JE ) ).LE.SAFMIN .AND.
     $             ABS( P( JE, JE ) ).LE.SAFMIN ) THEN
*
*                 Singular matrix pencil -- return unit eigenvector
*
                  IEIG = IEIG + 1
                  DO 60 JR = 1, N
                     VL( JR, IEIG ) = ZERO
   60             CONTINUE
                  VL( IEIG, IEIG ) = ONE
                  GO TO 220
               END IF
            END IF
*
*           Clear vector
*
            DO 70 JR = 1, NW*N
               WORK( 2*N+JR ) = ZERO
   70       CONTINUE
*                                                   T
*           Compute coefficients in  ( a A - b B )  y = 0
*              a  is  ACOEF
*              b  is  BCOEFR + i*BCOEFI
*
            IF( .NOT.ILCPLX ) THEN
*
*              Real eigenvalue
*
               TEMP = ONE / MAX( ABS( S( JE, JE ) )*ASCALE,
     $                ABS( P( JE, JE ) )*BSCALE, SAFMIN )
               SALFAR = ( TEMP*S( JE, JE ) )*ASCALE
               SBETA = ( TEMP*P( JE, JE ) )*BSCALE
               ACOEF = SBETA*ASCALE
               BCOEFR = SALFAR*BSCALE
               BCOEFI = ZERO
*
*              Scale to avoid underflow
*
               SCALE = ONE
               LSA = ABS( SBETA ).GE.SAFMIN .AND. ABS( ACOEF ).LT.SMALL
               LSB = ABS( SALFAR ).GE.SAFMIN .AND. ABS( BCOEFR ).LT.
     $               SMALL
               IF( LSA )
```

```
     $            SCALE = ( SMALL / ABS( SBETA ) )*MIN( ANORM, BIG )
               IF( LSB )
     $            SCALE = MAX( SCALE, ( SMALL / ABS( SALFAR ) )*
     $                    MIN( BNORM, BIG ) )
               IF( LSA .OR. LSB ) THEN
                  SCALE = MIN( SCALE, ONE /
     $                    ( SAFMIN*MAX( ONE, ABS( ACOEF ),
     $                    ABS( BCOEFR ) ) ) )
                  IF( LSA ) THEN
                     ACOEF = ASCALE*( SCALE*SBETA )
                  ELSE
                     ACOEF = SCALE*ACOEF
                  END IF
                  IF( LSB ) THEN
                     BCOEFR = BSCALE*( SCALE*SALFAR )
                  ELSE
                     BCOEFR = SCALE*BCOEFR
                  END IF
               END IF
               ACOEFA = ABS( ACOEF )
               BCOEFA = ABS( BCOEFR )
*
*              First component is 1
*
               WORK( 2*N+JE ) = ONE
               XMAX = ONE
            ELSE
*
*              Complex eigenvalue
*
               CALL DLAG2( S( JE, JE ), LDS, P( JE, JE ), LDP,
     $                     SAFMIN*SAFETY, ACOEF, TEMP, BCOEFR, TEMP2,
     $                     BCOEFI )
               BCOEFI = -BCOEFI
               IF( BCOEFI.EQ.ZERO ) THEN
                  INFO = JE
                  RETURN
               END IF
*
*              Scale to avoid over/underflow
*
               ACOEFA = ABS( ACOEF )
               BCOEFA = ABS( BCOEFR ) + ABS( BCOEFI )
               SCALE = ONE
               IF( ACOEFA*ULP.LT.SAFMIN .AND. ACOEFA.GE.SAFMIN )
     $            SCALE = ( SAFMIN / ULP ) / ACOEFA
               IF( BCOEFA*ULP.LT.SAFMIN .AND. BCOEFA.GE.SAFMIN )
     $            SCALE = MAX( SCALE, ( SAFMIN / ULP ) / BCOEFA )
               IF( SAFMIN*ACOEFA.GT.ASCALE )
     $            SCALE = ASCALE / ( SAFMIN*ACOEFA )
```

```
               IF( SAFMIN*BCOEFA.GT.BSCALE )
     $            SCALE = MIN( SCALE, BSCALE / ( SAFMIN*BCOEFA ) )
               IF( SCALE.NE.ONE ) THEN
                  ACOEF = SCALE*ACOEF
                  ACOEFA = ABS( ACOEF )
                  BCOEFR = SCALE*BCOEFR
                  BCOEFI = SCALE*BCOEFI
                  BCOEFA = ABS( BCOEFR ) + ABS( BCOEFI )
               END IF
*
*              Compute first two components of eigenvector
*
               TEMP = ACOEF*S( JE+1, JE )
               TEMP2R = ACOEF*S( JE, JE ) - BCOEFR*P( JE, JE )
               TEMP2I = -BCOEFI*P( JE, JE )
               IF( ABS( TEMP ).GT.ABS( TEMP2R )+ABS( TEMP2I ) ) THEN
                  WORK( 2*N+JE ) = ONE
                  WORK( 3*N+JE ) = ZERO
                  WORK( 2*N+JE+1 ) = -TEMP2R / TEMP
                  WORK( 3*N+JE+1 ) = -TEMP2I / TEMP
               ELSE
                  WORK( 2*N+JE+1 ) = ONE
                  WORK( 3*N+JE+1 ) = ZERO
                  TEMP = ACOEF*S( JE, JE+1 )
                  WORK( 2*N+JE ) = ( BCOEFR*P( JE+1, JE+1 )-ACOEF*
     $                             S( JE+1, JE+1 ) ) / TEMP
                  WORK( 3*N+JE ) = BCOEFI*P( JE+1, JE+1 ) / TEMP
               END IF
               XMAX = MAX( ABS( WORK( 2*N+JE ) )+ABS( WORK( 3*N+JE ) ),
     $                ABS( WORK( 2*N+JE+1 ) )+ABS( WORK( 3*N+JE+1 ) ) )
            END IF
*
            DMIN = MAX( ULP*ACOEFA*ANORM, ULP*BCOEFA*BNORM, SAFMIN )
*
*                                          T
*           Triangular solve of  (a A - b B)  y = 0
*
*                                  T
*           (rowwise in  (a A - b B) , or columnwise in (a A - b B) )
*
            IL2BY2 = .FALSE.
*
            DO 160 J = JE + NW, N
               IF( IL2BY2 ) THEN
                  IL2BY2 = .FALSE.
                  GO TO 160
               END IF
*
               NA = 1
               BDIAG( 1 ) = P( J, J )
```

```
               IF( J.LT.N ) THEN
                  IF( S( J+1, J ).NE.ZERO ) THEN
                     IL2BY2 = .TRUE.
                     BDIAG( 2 ) = P( J+1, J+1 )
                     NA = 2
                  END IF
               END IF
*
*              Check whether scaling is necessary for dot products
*
               XSCALE = ONE / MAX( ONE, XMAX )
               TEMP = MAX( WORK( J ), WORK( N+J ),
     $                ACOEFA*WORK( J )+BCOEFA*WORK( N+J ) )
               IF( IL2BY2 )
     $            TEMP = MAX( TEMP, WORK( J+1 ), WORK( N+J+1 ),
     $                   ACOEFA*WORK( J+1 )+BCOEFA*WORK( N+J+1 ) )
               IF( TEMP.GT.BIGNUM*XSCALE ) THEN
                  DO 90 JW = 0, NW - 1
                     DO 80 JR = JE, J - 1
                        WORK( ( JW+2 )*N+JR ) = XSCALE*
     $                     WORK( ( JW+2 )*N+JR )
   80                CONTINUE
   90             CONTINUE
                  XMAX = XMAX*XSCALE
               END IF
*
*              Compute dot products
*
*                    j-1
*              SUM = sum  conjg( a*S(k,j) - b*P(k,j) )*x(k)
*                    k=je
*
*              To reduce the op count, this is done as
*
*              _        j-1                   _        j-1
*              a*conjg( sum  S(k,j)*x(k) ) - b*conjg( sum  P(k,j)*x(k) )
*                       k=je                           k=je
*
*              which may cause underflow problems if A or B are close
*              to underflow.  (E.g., less than SMALL.)
*
*
               DO 120 JW = 1, NW
                  DO 110 JA = 1, NA
                     SUMS( JA, JW ) = ZERO
                     SUMP( JA, JW ) = ZERO
*
                     DO 100 JR = JE, J - 1
                        SUMS( JA, JW ) = SUMS( JA, JW ) +
     $                                   S( JR, J+JA-1 )*
```

```
     $                                 WORK( ( JW+1 )*N+JR )
                     SUMP( JA, JW ) = SUMP( JA, JW ) +
     $                                 P( JR, J+JA-1 )*
     $                                 WORK( ( JW+1 )*N+JR )
  100               CONTINUE
  110            CONTINUE
  120         CONTINUE
*
              DO 130 JA = 1, NA
                 IF( ILCPLX ) THEN
                    SUM( JA, 1 ) = -ACOEF*SUMS( JA, 1 ) +
     $                             BCOEFR*SUMP( JA, 1 ) -
     $                             BCOEFI*SUMP( JA, 2 )
                    SUM( JA, 2 ) = -ACOEF*SUMS( JA, 2 ) +
     $                             BCOEFR*SUMP( JA, 2 ) +
     $                             BCOEFI*SUMP( JA, 1 )
                 ELSE
                    SUM( JA, 1 ) = -ACOEF*SUMS( JA, 1 ) +
     $                             BCOEFR*SUMP( JA, 1 )
                 END IF
  130         CONTINUE
*
*                                T
*             Solve  ( a A - b B )  y = SUM(,)
*             with scaling and perturbation of the denominator
*
              CALL DLALN2( .TRUE., NA, NW, DMIN, ACOEF, S( J, J ), LDS,
     $                     BDIAG( 1 ), BDIAG( 2 ), SUM, 2, BCOEFR,
     $                     BCOEFI, WORK( 2*N+J ), N, SCALE, TEMP,
     $                     IINFO )
              IF( SCALE.LT.ONE ) THEN
                 DO 150 JW = 0, NW - 1
                    DO 140 JR = JE, J - 1
                       WORK( ( JW+2 )*N+JR ) = SCALE*
     $                    WORK( ( JW+2 )*N+JR )
  140               CONTINUE
  150            CONTINUE
                 XMAX = SCALE*XMAX
              END IF
              XMAX = MAX( XMAX, TEMP )
  160       CONTINUE
*
*           Copy eigenvector to VL, back transforming if
*           HOWMNY='B'.
*
            IEIG = IEIG + 1
            IF( ILBACK ) THEN
               DO 170 JW = 0, NW - 1
                  CALL DGEMV( 'N', N, N+1-JE, ONE, VL( 1, JE ), LDVL,
     $                        WORK( ( JW+2 )*N+JE ), 1, ZERO,
```

```
     $                       WORK( ( JW+4 )*N+1 ), 1 )
  170          CONTINUE
               CALL DLACPY( ' ', N, NW, WORK( 4*N+1 ), N, VL( 1, JE ),
     $                     LDVL )
               IBEG = 1
            ELSE
               CALL DLACPY( ' ', N, NW, WORK( 2*N+1 ), N, VL( 1, IEIG ),
     $                     LDVL )
               IBEG = JE
            END IF
*
*           Scale eigenvector
*
            XMAX = ZERO
            IF( ILCPLX ) THEN
               DO 180 J = IBEG, N
                  XMAX = MAX( XMAX, ABS( VL( J, IEIG ) )+
     $                   ABS( VL( J, IEIG+1 ) ) )
  180          CONTINUE
            ELSE
               DO 190 J = IBEG, N
                  XMAX = MAX( XMAX, ABS( VL( J, IEIG ) ) )
  190          CONTINUE
            END IF
*
            IF( XMAX.GT.SAFMIN ) THEN
               XSCALE = ONE / XMAX
*
               DO 210 JW = 0, NW - 1
                  DO 200 JR = IBEG, N
                     VL( JR, IEIG+JW ) = XSCALE*VL( JR, IEIG+JW )
  200             CONTINUE
  210          CONTINUE
            END IF
            IEIG = IEIG + NW - 1
*
  220    CONTINUE
      END IF
*
*     Right eigenvectors
*
      IF( COMPR ) THEN
         IEIG = IM + 1
*
*        Main loop over eigenvalues
*
         ILCPLX = .FALSE.
         DO 500 JE = N, 1, -1
*
*           Skip this iteration if (a) HOWMNY='S' and SELECT=.FALSE., or
```

```
*           (b) this would be the second of a complex pair.
*           Check for complex eigenvalue, so as to be sure of which
*           entry(-ies) of SELECT to look at -- if complex, SELECT(JE)
*           or SELECT(JE-1).
*           If this is a complex pair, the 2-by-2 diagonal block
*           corresponding to the eigenvalue is in rows/columns JE-1:JE
*
            IF( ILCPLX ) THEN
               ILCPLX = .FALSE.
               GO TO 500
            END IF
            NW = 1
            IF( JE.GT.1 ) THEN
               IF( S( JE, JE-1 ).NE.ZERO ) THEN
                  ILCPLX = .TRUE.
                  NW = 2
               END IF
            END IF
            IF( ILALL ) THEN
               ILCOMP = .TRUE.
            ELSE IF( ILCPLX ) THEN
               ILCOMP = SELECT( JE ) .OR. SELECT( JE-1 )
            ELSE
               ILCOMP = SELECT( JE )
            END IF
            IF( .NOT.ILCOMP )
     $         GO TO 500
*
*           Decide if (a) singular pencil, (b) real eigenvalue, or
*           (c) complex eigenvalue.
*
            IF( .NOT.ILCPLX ) THEN
               IF( ABS( S( JE, JE ) ).LE.SAFMIN .AND.
     $             ABS( P( JE, JE ) ).LE.SAFMIN ) THEN
*
*                 Singular matrix pencil -- unit eigenvector
*
                  IEIG = IEIG - 1
                  DO 230 JR = 1, N
                     VR( JR, IEIG ) = ZERO
  230             CONTINUE
                  VR( IEIG, IEIG ) = ONE
                  GO TO 500
               END IF
            END IF
*
*           Clear vector
*
            DO 250 JW = 0, NW - 1
               DO 240 JR = 1, N
```

```
                 WORK( ( JW+2 )*N+JR ) = ZERO
  240         CONTINUE
  250      CONTINUE
*
*          Compute coefficients in  ( a A - b B ) x = 0
*             a  is  ACOEF
*             b  is  BCOEFR + i*BCOEFI
*
           IF( .NOT.ILCPLX ) THEN
*
*             Real eigenvalue
*
              TEMP = ONE / MAX( ABS( S( JE, JE ) )*ASCALE,
     $               ABS( P( JE, JE ) )*BSCALE, SAFMIN )
              SALFAR = ( TEMP*S( JE, JE ) )*ASCALE
              SBETA = ( TEMP*P( JE, JE ) )*BSCALE
              ACOEF = SBETA*ASCALE
              BCOEFR = SALFAR*BSCALE
              BCOEFI = ZERO
*
*             Scale to avoid underflow
*
              SCALE = ONE
              LSA = ABS( SBETA ).GE.SAFMIN .AND. ABS( ACOEF ).LT.SMALL
              LSB = ABS( SALFAR ).GE.SAFMIN .AND. ABS( BCOEFR ).LT.
     $              SMALL
              IF( LSA )
     $           SCALE = ( SMALL / ABS( SBETA ) )*MIN( ANORM, BIG )
              IF( LSB )
     $           SCALE = MAX( SCALE, ( SMALL / ABS( SALFAR ) )*
     $                   MIN( BNORM, BIG ) )
              IF( LSA .OR. LSB ) THEN
                 SCALE = MIN( SCALE, ONE /
     $                   ( SAFMIN*MAX( ONE, ABS( ACOEF ),
     $                   ABS( BCOEFR ) ) ) )
                 IF( LSA ) THEN
                    ACOEF = ASCALE*( SCALE*SBETA )
                 ELSE
                    ACOEF = SCALE*ACOEF
                 END IF
                 IF( LSB ) THEN
                    BCOEFR = BSCALE*( SCALE*SALFAR )
                 ELSE
                    BCOEFR = SCALE*BCOEFR
                 END IF
              END IF
              ACOEFA = ABS( ACOEF )
              BCOEFA = ABS( BCOEFR )
*
*             First component is 1
```

```
*
               WORK( 2*N+JE ) = ONE
               XMAX = ONE
*
*              Compute contribution from column JE of A and B to sum
*              (See "Further Details", above.)
*
               DO 260 JR = 1, JE - 1
                  WORK( 2*N+JR ) = BCOEFR*P( JR, JE ) -
     $                             ACOEF*S( JR, JE )
  260          CONTINUE
            ELSE
*
*              Complex eigenvalue
*
               CALL DLAG2( S( JE-1, JE-1 ), LDS, P( JE-1, JE-1 ), LDP,
     $                     SAFMIN*SAFETY, ACOEF, TEMP, BCOEFR, TEMP2,
     $                     BCOEFI )
               IF( BCOEFI.EQ.ZERO ) THEN
                  INFO = JE - 1
                  RETURN
               END IF
*
*              Scale to avoid over/underflow
*
               ACOEFA = ABS( ACOEF )
               BCOEFA = ABS( BCOEFR ) + ABS( BCOEFI )
               SCALE = ONE
               IF( ACOEFA*ULP.LT.SAFMIN .AND. ACOEFA.GE.SAFMIN )
     $            SCALE = ( SAFMIN / ULP ) / ACOEFA
               IF( BCOEFA*ULP.LT.SAFMIN .AND. BCOEFA.GE.SAFMIN )
     $            SCALE = MAX( SCALE, ( SAFMIN / ULP ) / BCOEFA )
               IF( SAFMIN*ACOEFA.GT.ASCALE )
     $            SCALE = ASCALE / ( SAFMIN*ACOEFA )
               IF( SAFMIN*BCOEFA.GT.BSCALE )
     $            SCALE = MIN( SCALE, BSCALE / ( SAFMIN*BCOEFA ) )
               IF( SCALE.NE.ONE ) THEN
                  ACOEF = SCALE*ACOEF
                  ACOEFA = ABS( ACOEF )
                  BCOEFR = SCALE*BCOEFR
                  BCOEFI = SCALE*BCOEFI
                  BCOEFA = ABS( BCOEFR ) + ABS( BCOEFI )
               END IF
*
*              Compute first two components of eigenvector
*              and contribution to sums
*
               TEMP = ACOEF*S( JE, JE-1 )
               TEMP2R = ACOEF*S( JE, JE ) - BCOEFR*P( JE, JE )
               TEMP2I = -BCOEFI*P( JE, JE )
```

```
               IF( ABS( TEMP ).GE.ABS( TEMP2R )+ABS( TEMP2I ) ) THEN
                  WORK( 2*N+JE ) = ONE
                  WORK( 3*N+JE ) = ZERO
                  WORK( 2*N+JE-1 ) = -TEMP2R / TEMP
                  WORK( 3*N+JE-1 ) = -TEMP2I / TEMP
               ELSE
                  WORK( 2*N+JE-1 ) = ONE
                  WORK( 3*N+JE-1 ) = ZERO
                  TEMP = ACOEF*S( JE-1, JE )
                  WORK( 2*N+JE ) = ( BCOEFR*P( JE-1, JE-1 )-ACOEF*
     $                             S( JE-1, JE-1 ) ) / TEMP
                  WORK( 3*N+JE ) = BCOEFI*P( JE-1, JE-1 ) / TEMP
               END IF
*
               XMAX = MAX( ABS( WORK( 2*N+JE ) )+ABS( WORK( 3*N+JE ) ),
     $                ABS( WORK( 2*N+JE-1 ) )+ABS( WORK( 3*N+JE-1 ) ) )
*
*              Compute contribution from columns JE and JE-1
*              of A and B to the sums.
*
               CREALA = ACOEF*WORK( 2*N+JE-1 )
               CIMAGA = ACOEF*WORK( 3*N+JE-1 )
               CREALB = BCOEFR*WORK( 2*N+JE-1 ) -
     $                  BCOEFI*WORK( 3*N+JE-1 )
               CIMAGB = BCOEFI*WORK( 2*N+JE-1 ) +
     $                  BCOEFR*WORK( 3*N+JE-1 )
               CRE2A = ACOEF*WORK( 2*N+JE )
               CIM2A = ACOEF*WORK( 3*N+JE )
               CRE2B = BCOEFR*WORK( 2*N+JE ) - BCOEFI*WORK( 3*N+JE )
               CIM2B = BCOEFI*WORK( 2*N+JE ) + BCOEFR*WORK( 3*N+JE )
               DO 270 JR = 1, JE - 2
                  WORK( 2*N+JR ) = -CREALA*S( JR, JE-1 ) +
     $                             CREALB*P( JR, JE-1 ) -
     $                             CRE2A*S( JR, JE ) + CRE2B*P( JR, JE )
                  WORK( 3*N+JR ) = -CIMAGA*S( JR, JE-1 ) +
     $                             CIMAGB*P( JR, JE-1 ) -
     $                             CIM2A*S( JR, JE ) + CIM2B*P( JR, JE )
  270          CONTINUE
            END IF
*
            DMIN = MAX( ULP*ACOEFA*ANORM, ULP*BCOEFA*BNORM, SAFMIN )
*
*           Columnwise triangular solve of  (a A - b B)  x = 0
*
            IL2BY2 = .FALSE.
            DO 370 J = JE - NW, 1, -1
*
*              If a 2-by-2 block, is in position j-1:j, wait until
*              next iteration to process it (when it will be j:j+1)
*
```

```
               IF( .NOT.IL2BY2 .AND. J.GT.1 ) THEN
                  IF( S( J, J-1 ).NE.ZERO ) THEN
                     IL2BY2 = .TRUE.
                     GO TO 370
                  END IF
               END IF
               BDIAG( 1 ) = P( J, J )
               IF( IL2BY2 ) THEN
                  NA = 2
                  BDIAG( 2 ) = P( J+1, J+1 )
               ELSE
                  NA = 1
               END IF
*
*              Compute x(j) (and x(j+1), if 2-by-2 block)
*
               CALL DLALN2( .FALSE., NA, NW, DMIN, ACOEF, S( J, J ),
     $                    LDS, BDIAG( 1 ), BDIAG( 2 ), WORK( 2*N+J ),
     $                    N, BCOEFR, BCOEFI, SUM, 2, SCALE, TEMP,
     $                    IINFO )
               IF( SCALE.LT.ONE ) THEN
*
                  DO 290 JW = 0, NW - 1
                     DO 280 JR = 1, JE
                        WORK( ( JW+2 )*N+JR ) = SCALE*
     $                     WORK( ( JW+2 )*N+JR )
  280                CONTINUE
  290             CONTINUE
               END IF
               XMAX = MAX( SCALE*XMAX, TEMP )
*
               DO 310 JW = 1, NW
                  DO 300 JA = 1, NA
                     WORK( ( JW+1 )*N+J+JA-1 ) = SUM( JA, JW )
  300             CONTINUE
  310          CONTINUE
*
*              w = w + x(j)*(a S(*,j) - b P(*,j) ) with scaling
*
               IF( J.GT.1 ) THEN
*
*                Check whether scaling is necessary for sum.
*
                  XSCALE = ONE / MAX( ONE, XMAX )
                  TEMP = ACOEFA*WORK( J ) + BCOEFA*WORK( N+J )
                  IF( IL2BY2 )
     $               TEMP = MAX( TEMP, ACOEFA*WORK( J+1 )+BCOEFA*
     $                      WORK( N+J+1 ) )
                  TEMP = MAX( TEMP, ACOEFA, BCOEFA )
                  IF( TEMP.GT.BIGNUM*XSCALE ) THEN
```

```
*
                    DO 330 JW = 0, NW - 1
                       DO 320 JR = 1, JE
                          WORK( ( JW+2 )*N+JR ) = XSCALE*
     $                       WORK( ( JW+2 )*N+JR )
  320                 CONTINUE
  330                 CONTINUE
                      XMAX = XMAX*XSCALE
                   END IF
*
*                  Compute the contributions of the off-diagonals of
*                  column j (and j+1, if 2-by-2 block) of A and B to the
*                  sums.
*
*
                   DO 360 JA = 1, NA
                      IF( ILCPLX ) THEN
                         CREALA = ACOEF*WORK( 2*N+J+JA-1 )
                         CIMAGA = ACOEF*WORK( 3*N+J+JA-1 )
                         CREALB = BCOEFR*WORK( 2*N+J+JA-1 ) -
     $                            BCOEFI*WORK( 3*N+J+JA-1 )
                         CIMAGB = BCOEFI*WORK( 2*N+J+JA-1 ) +
     $                            BCOEFR*WORK( 3*N+J+JA-1 )
                         DO 340 JR = 1, J - 1
                            WORK( 2*N+JR ) = WORK( 2*N+JR ) -
     $                                       CREALA*S( JR, J+JA-1 ) +
     $                                       CREALB*P( JR, J+JA-1 )
                            WORK( 3*N+JR ) = WORK( 3*N+JR ) -
     $                                       CIMAGA*S( JR, J+JA-1 ) +
     $                                       CIMAGB*P( JR, J+JA-1 )
  340                    CONTINUE
                      ELSE
                         CREALA = ACOEF*WORK( 2*N+J+JA-1 )
                         CREALB = BCOEFR*WORK( 2*N+J+JA-1 )
                         DO 350 JR = 1, J - 1
                            WORK( 2*N+JR ) = WORK( 2*N+JR ) -
     $                                       CREALA*S( JR, J+JA-1 ) +
     $                                       CREALB*P( JR, J+JA-1 )
  350                    CONTINUE
                      END IF
  360              CONTINUE
                END IF
*
                IL2BY2 = .FALSE.
  370        CONTINUE
*
*            Copy eigenvector to VR, back transforming if
*            HOWMNY='B'.
*
             IEIG = IEIG - NW
```

```
            IF( ILBACK ) THEN
*
               DO 410 JW = 0, NW - 1
                  DO 380 JR = 1, N
                     WORK( ( JW+4 )*N+JR ) = WORK( ( JW+2 )*N+1 )*
     $                                       VR( JR, 1 )
  380             CONTINUE
*
*                 A series of compiler directives to defeat
*                 vectorization for the next loop
*
*
                  DO 400 JC = 2, JE
                     DO 390 JR = 1, N
                        WORK( ( JW+4 )*N+JR ) = WORK( ( JW+4 )*N+JR ) +
     $                     WORK( ( JW+2 )*N+JC )*VR( JR, JC )
  390                CONTINUE
  400             CONTINUE
  410          CONTINUE
*
               DO 430 JW = 0, NW - 1
                  DO 420 JR = 1, N
                     VR( JR, IEIG+JW ) = WORK( ( JW+4 )*N+JR )
  420             CONTINUE
  430          CONTINUE
*
               IEND = N
            ELSE
               DO 450 JW = 0, NW - 1
                  DO 440 JR = 1, N
                     VR( JR, IEIG+JW ) = WORK( ( JW+2 )*N+JR )
  440             CONTINUE
  450          CONTINUE
*
               IEND = JE
            END IF
*
*           Scale eigenvector
*
            XMAX = ZERO
            IF( ILCPLX ) THEN
               DO 460 J = 1, IEND
                  XMAX = MAX( XMAX, ABS( VR( J, IEIG ) )+
     $                   ABS( VR( J, IEIG+1 ) ) )
  460          CONTINUE
            ELSE
               DO 470 J = 1, IEND
                  XMAX = MAX( XMAX, ABS( VR( J, IEIG ) ) )
  470          CONTINUE
            END IF
```

```
*
            IF( XMAX.GT.SAFMIN ) THEN
               XSCALE = ONE / XMAX
               DO 490 JW = 0, NW - 1
                  DO 480 JR = 1, IEND
                     VR( JR, IEIG+JW ) = XSCALE*VR( JR, IEIG+JW )
  480             CONTINUE
  490          CONTINUE
            END IF
  500    CONTINUE
      END IF
*
      RETURN
*
*     End of DTGEVC
*
      END
```

———-

32.39.4 lisp code

— dtgevc.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0) (safety 100.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 100.0d0 100.0d0) safety)
           (ignorable zero one safety))
  (defun dtgevc
         (side howmny select n s lds p ldp vl ldvl vr ldvr mm m work info)
    (declare (type (array double-float (*)) work vr vl p s)
             (type (f2cl-lib:integer4) info m mm ldvr ldvl ldp lds n)
             (type (array f2cl-lib:logical (*)) select)
             (type (string *) howmny side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (howmny character howmny-%data% howmny-%offset%)
         (select f2cl-lib:logical select-%data% select-%offset%)
         (s double-float s-%data% s-%offset%)
         (p double-float p-%data% p-%offset%)
         (vl double-float vl-%data% vl-%offset%)
         (vr double-float vr-%data% vr-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((bdiag (make-array 2 :element-type 'double-float))
             (sum (make-array 4 :element-type 'double-float))
             (sums (make-array 4 :element-type 'double-float))
             (sump (make-array 4 :element-type 'double-float)) (acoef 0.0d0)
```

```
     (acoefa 0.0d0) (anorm 0.0d0) (ascale 0.0d0) (bcoefa 0.0d0)
     (bcoefi 0.0d0) (bcoefr 0.0d0) (big 0.0d0) (bignum 0.0d0)
     (bnorm 0.0d0) (bscale 0.0d0) (cim2a 0.0d0) (cim2b 0.0d0)
     (cimaga 0.0d0) (cimagb 0.0d0) (cre2a 0.0d0) (cre2b 0.0d0)
     (creala 0.0d0) (crealb 0.0d0) (dmin 0.0d0) (safmin 0.0d0)
     (salfar 0.0d0) (sbeta 0.0d0) (scale 0.0d0) (small 0.0d0)
     (temp 0.0d0) (temp2 0.0d0) (temp2i 0.0d0) (temp2r 0.0d0)
     (ulp 0.0d0) (xmax 0.0d0) (xscale 0.0d0) (i 0) (ibeg 0) (ieig 0)
     (iend 0) (ihwmny 0) (iinfo 0) (im 0) (iside 0) (j 0) (ja 0) (jc 0)
     (je 0) (jr 0) (jw 0) (na 0) (nw 0) (compl nil) (compr nil)
     (il2by2 nil) (ilabad nil) (ilall nil) (ilback nil) (ilbbad nil)
     (ilcomp nil) (ilcplx nil) (lsa nil) (lsb nil))
(declare (type (array double-float (2)) bdiag)
         (type (array double-float (4)) sum sums sump)
         (type (double-float) acoef acoefa anorm ascale bcoefa bcoefi
          bcoefr big bignum bnorm bscale cim2a cim2b cimaga cimagb
          cre2a cre2b creala crealb dmin safmin salfar sbeta scale
          small temp temp2 temp2i temp2r ulp xmax xscale)
         (type (f2cl-lib:integer4) i ibeg ieig iend ihwmny iinfo im
          iside j ja jc je jr jw na nw)
         (type f2cl-lib:logical compl compr il2by2 ilabad ilall ilback
          ilbbad ilcomp ilcplx lsa lsb))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame howmny "A")
    (declare (ignore var-1))
    (when var-0 (setf howmny var-0))
    ret-val)
  (setf ihwmny 1) (setf ilall f2cl-lib:%true%)
  (setf ilback f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame howmny "S")
    (declare (ignore var-1))
    (when var-0 (setf howmny var-0))
    ret-val)
  (setf ihwmny 2) (setf ilall f2cl-lib:%false%)
  (setf ilback f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame howmny "B")
    (declare (ignore var-1))
    (when var-0 (setf howmny var-0))
    ret-val)
  (setf ihwmny 3) (setf ilall f2cl-lib:%true%)
  (setf ilback f2cl-lib:%true%))
 (t (setf ihwmny -1) (setf ilall f2cl-lib:%true%)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame side "R")
    (declare (ignore var-1))
    (when var-0 (setf side var-0))
```

```
    ret-val)
  (setf iside 1) (setf compl f2cl-lib:%false%)
  (setf compr f2cl-lib:%true%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame side "L")
    (declare (ignore var-1))
    (when var-0 (setf side var-0))
    ret-val)
  (setf iside 2) (setf compl f2cl-lib:%true%)
  (setf compr f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame side "B")
    (declare (ignore var-1))
    (when var-0 (setf side var-0))
    ret-val)
  (setf iside 3) (setf compl f2cl-lib:%true%)
  (setf compr f2cl-lib:%true%))
 (t (setf iside -1)))
(setf info 0)
(cond ((< iside 0) (setf info -1)) ((< ihwmny 0) (setf info -2))
      ((< n 0) (setf info -4))
      ((< lds
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -6))
      ((< ldp
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -8)))
(cond
 ((/= info 0) (xerbla "DTGEVC" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((not ilall) (setf im 0) (setf ilcplx f2cl-lib:%false%)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (cond (ilcplx (setf ilcplx f2cl-lib:%false%) (go label10)))
      (cond
       ((< j n)
        (if (/=
             (f2cl-lib:fref s-%data% ((f2cl-lib:int-add j 1) j)
                            ((1 lds) (1 *)) s-%offset%)
             zero)
            (setf ilcplx f2cl-lib:%true%))))
      (cond
       (ilcplx
        (if (or
             (f2cl-lib:fref select-%data% (j) ((1 *)) select-%offset%)
             (f2cl-lib:fref select-%data% ((f2cl-lib:int-add j 1))
                            ((1 *)) select-%offset%))
            (setf im (f2cl-lib:int-add im 2))))
```

```
        (t
         (if (f2cl-lib:fref select-%data% (j) ((1 *)) select-%offset%)
             (setf im (f2cl-lib:int-add im 1)))))
     label10)))
 (t (setf im n)))
(setf ilabad f2cl-lib:%false%)
(setf ilbbad f2cl-lib:%false%)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (cond
     ((/= (f2cl-lib:fref s ((f2cl-lib:int-add j 1) j) ((1 lds) (1 *)))
          zero)
      (if (or
           (= (f2cl-lib:fref p-%data% (j j) ((1 ldp) (1 *)) p-%offset%)
              zero)
           (=
            (f2cl-lib:fref p-%data%
                           ((f2cl-lib:int-add j 1)
                            (f2cl-lib:int-add j 1))
                           ((1 ldp) (1 *)) p-%offset%)
            zero)
           (/=
            (f2cl-lib:fref p-%data% (j (f2cl-lib:int-add j 1))
                           ((1 ldp) (1 *)) p-%offset%)
            zero))
          (setf ilbbad f2cl-lib:%true%))
      (cond
       ((< j (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
        (if (/=
             (f2cl-lib:fref s-%data%
                            ((f2cl-lib:int-add j 2)
                             (f2cl-lib:int-add j 1))
                            ((1 lds) (1 *)) s-%offset%)
             zero)
            (setf ilabad f2cl-lib:%true%))))))
   label20))
(cond (ilabad (setf info -5)) (ilbbad (setf info -7))
      ((or (and compl (< ldvl n)) (< ldvl 1)) (setf info -10))
      ((or (and compr (< ldvr n)) (< ldvr 1)) (setf info -12))
      ((< mm im) (setf info -13)))
(cond
 ((/= info 0) (xerbla "DTGEVC" (f2cl-lib:int-sub info))
  (go end_label)))
(setf m im)
(if (= n 0)
    (go end_label))
(setf safmin (dlamch "Safe minimum"))
(setf big (/ one safmin))
(multiple-value-bind (var-0 var-1)
```

```
    (dlabad safmin big)
  (declare (ignore))
  (when var-0 (setf safmin var-0))
  (when var-1 (setf big var-1)))
(setf ulp (* (dlamch "Epsilon") (dlamch "Base")))
(setf small (/ (* safmin n) ulp))
(setf big (/ one small))
(setf bignum (/ one (* safmin n)))
(setf anorm
        (abs
         (f2cl-lib:fref s-%data% (1 1) ((1 lds) (1 *)) s-%offset%)))
(if (> n 1)
    (setf anorm
            (+ anorm
               (abs
                (f2cl-lib:fref s-%data% (2 1) ((1 lds) (1 *))
                               s-%offset%)))))
(setf bnorm
        (abs
         (f2cl-lib:fref p-%data% (1 1) ((1 ldp) (1 *)) p-%offset%)))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) zero)
(setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 1)) ((1 *))
                     work-%offset%)
        zero)
(f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf temp zero)
    (setf temp2 zero)
    (cond
     ((=
       (f2cl-lib:fref s (j (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((1 lds) (1 *)))
       zero)
      (setf iend (f2cl-lib:int-sub j 1)))
     (t (setf iend (f2cl-lib:int-sub j 2))))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i iend) nil)
      (tagbody
        (setf temp
                (+ temp
                   (abs
                    (f2cl-lib:fref s-%data% (i j) ((1 lds) (1 *))
                                   s-%offset%))))
        (setf temp2
                (+ temp2
                   (abs
                    (f2cl-lib:fref p-%data% (i j) ((1 ldp) (1 *))
                                   p-%offset%))))
       label30))
```

```
    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%) temp)
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j)) ((1 *))
                         work-%offset%)
            temp2)
    (f2cl-lib:fdo (i (f2cl-lib:int-add iend 1) (f2cl-lib:int-add i 1))
                  ((> i
                      (min
                       (the f2cl-lib:integer4 (f2cl-lib:int-add j 1))
                       (the f2cl-lib:integer4 n)))
                   nil)
      (tagbody
        (setf temp
                (+ temp
                   (abs
                    (f2cl-lib:fref s-%data% (i j) ((1 lds) (1 *))
                                   s-%offset%))))
        (setf temp2
                (+ temp2
                   (abs
                    (f2cl-lib:fref p-%data% (i j) ((1 ldp) (1 *))
                                   p-%offset%))))
       label40))
    (setf anorm (max anorm temp))
    (setf bnorm (max bnorm temp2))
   label50))
(setf ascale (/ one (max anorm safmin)))
(setf bscale (/ one (max bnorm safmin)))
(cond
 (compl (setf ieig 0) (setf ilcplx f2cl-lib:%false%)
  (f2cl-lib:fdo (je 1 (f2cl-lib:int-add je 1))
                ((> je n) nil)
    (tagbody
      (cond (ilcplx (setf ilcplx f2cl-lib:%false%) (go label220)))
      (setf nw 1)
      (cond
       ((< je n)
        (cond
         ((/=
           (f2cl-lib:fref s ((f2cl-lib:int-add je 1) je)
                          ((1 lds) (1 *)))
           zero)
          (setf ilcplx f2cl-lib:%true%) (setf nw 2)))))
      (cond (ilall (setf ilcomp f2cl-lib:%true%))
            (ilcplx
             (setf ilcomp
                     (or
                      (f2cl-lib:fref select-%data% (je) ((1 *))
                                     select-%offset%)
                      (f2cl-lib:fref select-%data%
                                     ((f2cl-lib:int-add je 1)) ((1 *))
```

```
                                 select-%offset%))))
      (t
       (setf ilcomp
               (f2cl-lib:fref select-%data% (je) ((1 *))
                              select-%offset%))))
(if (not ilcomp)
    (go label220))
(cond
 ((not ilcplx)
  (cond
   ((and
     (<= (abs (f2cl-lib:fref s (je je) ((1 lds) (1 *)))) safmin)
     (<= (abs (f2cl-lib:fref p (je je) ((1 ldp) (1 *)))) safmin))
    (setf ieig (f2cl-lib:int-add ieig 1))
    (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                  ((> jr n) nil)
      (tagbody
        (setf (f2cl-lib:fref vl-%data% (jr ieig) ((1 ldvl) (1 *))
                             vl-%offset%)
              zero)
       label60))
    (setf (f2cl-lib:fref vl-%data% (ieig ieig) ((1 ldvl) (1 *))
                         vl-%offset%)
          one)
    (go label220)))))
(f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
              ((> jr (f2cl-lib:int-mul nw n)) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add
                           (f2cl-lib:int-mul 2 n) jr))
                         ((1 *)) work-%offset%)
          zero)
   label70))
(cond
 ((not ilcplx)
  (setf temp
          (/ one
             (max
              (*
               (abs
                (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                               s-%offset%))
               ascale)
              (*
               (abs
                (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
                               p-%offset%))
               bscale)
              safmin)))
```

```
(setf salfar
        (* temp
           (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                          s-%offset%)
           ascale))
(setf sbeta
        (* temp
           (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
                          p-%offset%)
           bscale))
(setf acoef (* sbeta ascale)) (setf bcoefr (* salfar bscale))
(setf bcoefi zero) (setf scale one)
(setf lsa (and (>= (abs sbeta) safmin) (< (abs acoef) small)))
(setf lsb
        (and (>= (abs salfar) safmin) (< (abs bcoefr) small)))
(if lsa
    (setf scale (* (/ small (abs sbeta)) (min anorm big))))
(if lsb
    (setf scale
            (max scale
                 (* (/ small (abs salfar)) (min bnorm big)))))
(cond
 ((or lsa lsb)
  (setf scale
          (min scale
               (/ one
                  (* safmin
                     (max one (abs acoef) (abs bcoefr))))))
  (cond (lsa (setf acoef (* ascale (* scale sbeta))))
        (t (setf acoef (* scale acoef))))
  (cond (lsb (setf bcoefr (* bscale (* scale salfar))))
        (t (setf bcoefr (* scale bcoefr))))))
(setf acoefa (abs acoef)) (setf bcoefa (abs bcoefr))
(setf (f2cl-lib:fref work-%data%
                     ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        je))
                     ((1 *)) work-%offset%)
        one)
(setf xmax one))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9)
     (dlag2
      (f2cl-lib:array-slice s-%data% double-float (je je)
                            ((1 lds) (1 *)) s-%offset%)
      lds
      (f2cl-lib:array-slice p-%data% double-float (je je)
                            ((1 ldp) (1 *)) p-%offset%)
      ldp (* safmin safety) acoef temp bcoefr temp2 bcoefi)
```

```
  (declare (ignore var-0 var-2 var-4))
  (when var-1 (setf lds var-1))
  (when var-3 (setf ldp var-3))
  (when var-5 (setf acoef var-5))
  (when var-6 (setf temp var-6))
  (when var-7 (setf bcoefr var-7))
  (when var-8 (setf temp2 var-8))
  (when var-9 (setf bcoefi var-9)))
(setf bcoefi (- bcoefi))
(cond ((= bcoefi zero) (setf info je) (go end_label)))
(setf acoefa (abs acoef))
(setf bcoefa (+ (abs bcoefr) (abs bcoefi))) (setf scale one)
(if (and (< (* acoefa ulp) safmin) (>= acoefa safmin))
    (setf scale (/ (/ safmin ulp) acoefa)))
(if (and (< (* bcoefa ulp) safmin) (>= bcoefa safmin))
    (setf scale (max scale (/ (/ safmin ulp) bcoefa))))
(if (> (* safmin acoefa) ascale)
    (setf scale (/ ascale (* safmin acoefa))))
(if (> (* safmin bcoefa) bscale)
    (setf scale (min scale (/ bscale (* safmin bcoefa)))))
(cond
 ((/= scale one) (setf acoef (* scale acoef))
  (setf acoefa (abs acoef)) (setf bcoefr (* scale bcoefr))
  (setf bcoefi (* scale bcoefi))
  (setf bcoefa (+ (abs bcoefr) (abs bcoefi)))))
(setf temp
        (* acoef
           (f2cl-lib:fref s-%data% ((f2cl-lib:int-add je 1) je)
                          ((1 lds) (1 *)) s-%offset%)))
(setf temp2r
        (-
         (* acoef
            (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                           s-%offset%))
         (* bcoefr
            (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
                           p-%offset%))))
(setf temp2i
        (* (- bcoefi)
           (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
                          p-%offset%)))
(cond
 ((> (abs temp) (+ (abs temp2r) (abs temp2i)))
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 2 n) je))
                       ((1 *)) work-%offset%)
          one)
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
```

```
                        (f2cl-lib:int-mul 3 n) je))
                      ((1 *)) work-%offset%)
        zero)
 (setf (f2cl-lib:fref work-%data%
                      ((f2cl-lib:int-add
                        (f2cl-lib:int-mul 2 n) je 1))
                      ((1 *)) work-%offset%)
        (/ (- temp2r) temp))
 (setf (f2cl-lib:fref work-%data%
                      ((f2cl-lib:int-add
                        (f2cl-lib:int-mul 3 n) je 1))
                      ((1 *)) work-%offset%)
        (/ (- temp2i) temp)))
(t
 (setf (f2cl-lib:fref work-%data%
                      ((f2cl-lib:int-add
                        (f2cl-lib:int-mul 2 n) je 1))
                      ((1 *)) work-%offset%)
        one)
 (setf (f2cl-lib:fref work-%data%
                      ((f2cl-lib:int-add
                        (f2cl-lib:int-mul 3 n) je 1))
                      ((1 *)) work-%offset%)
        zero)
 (setf temp
        (* acoef
           (f2cl-lib:fref s-%data%
                          (je (f2cl-lib:int-add je 1))
                          ((1 lds) (1 *)) s-%offset%)))
 (setf (f2cl-lib:fref work-%data%
                      ((f2cl-lib:int-add
                        (f2cl-lib:int-mul 2 n) je))
                      ((1 *)) work-%offset%)
        (/
         (-
          (* bcoefr
             (f2cl-lib:fref p-%data%
                            ((f2cl-lib:int-add je 1)
                             (f2cl-lib:int-add je 1))
                            ((1 ldp) (1 *)) p-%offset%))
          (* acoef
             (f2cl-lib:fref s-%data%
                            ((f2cl-lib:int-add je 1)
                             (f2cl-lib:int-add je 1))
                            ((1 lds) (1 *)) s-%offset%)))
         temp))
 (setf (f2cl-lib:fref work-%data%
                      ((f2cl-lib:int-add
                        (f2cl-lib:int-mul 3 n) je))
                      ((1 *)) work-%offset%)
```

```
                (/
                 (* bcoefi
                    (f2cl-lib:fref p-%data%
                                   ((f2cl-lib:int-add je 1)
                                    (f2cl-lib:int-add je 1))
                                   ((1 ldp) (1 *)) p-%offset%))
                 temp))))
  (setf xmax
          (max
           (+
            (abs
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add
                              (f2cl-lib:int-mul 2 n) je))
                            ((1 *)) work-%offset%))
            (abs
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add
                              (f2cl-lib:int-mul 3 n) je))
                            ((1 *)) work-%offset%)))
           (+
            (abs
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add
                              (f2cl-lib:int-mul 2 n) je 1))
                            ((1 *)) work-%offset%))
            (abs
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add
                              (f2cl-lib:int-mul 3 n) je 1))
                            ((1 *)) work-%offset%)))))))
(setf dmin
        (max (* ulp acoefa anorm) (* ulp bcoefa bnorm) safmin))
(setf il2by2 f2cl-lib:%false%)
(f2cl-lib:fdo (j (f2cl-lib:int-add je nw) (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (cond (il2by2 (setf il2by2 f2cl-lib:%false%) (go label160)))
    (setf na 1)
    (setf (f2cl-lib:fref bdiag (1) ((1 2)))
            (f2cl-lib:fref p-%data% (j j) ((1 ldp) (1 *))
                           p-%offset%))
    (cond
     ((< j n)
      (cond
       ((/=
         (f2cl-lib:fref s ((f2cl-lib:int-add j 1) j)
                        ((1 lds) (1 *)))
         zero)
        (setf il2by2 f2cl-lib:%true%)
```

```
    (setf (f2cl-lib:fref bdiag (2) ((1 2)))
            (f2cl-lib:fref p-%data%
                           ((f2cl-lib:int-add j 1)
                            (f2cl-lib:int-add j 1))
                           ((1 ldp) (1 *)) p-%offset%))
    (setf na 2)))))
(setf xscale (/ one (max one xmax)))
(setf temp
        (max
         (f2cl-lib:fref work-%data% (j) ((1 *))
                        work-%offset%)
         (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j))
                        ((1 *)) work-%offset%)
         (+
          (* acoefa
             (f2cl-lib:fref work-%data% (j) ((1 *))
                            work-%offset%))
          (* bcoefa
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add n j)) ((1 *))
                            work-%offset%)))))
(if il2by2
    (setf temp
            (max temp
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add j 1))
                                ((1 *)) work-%offset%)
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add n j 1))
                                ((1 *)) work-%offset%)
                 (+
                  (* acoefa
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add j 1))
                                    ((1 *)) work-%offset%))
                  (* bcoefa
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add n j
                                                       1))
                                    ((1 *))
                                    work-%offset%))))))
(cond
 ((> temp (* bignum xscale))
  (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                ((> jw
                    (f2cl-lib:int-add nw
                                      (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (f2cl-lib:fdo (jr je (f2cl-lib:int-add jr 1))
```

```
                          ((> jr
                              (f2cl-lib:int-add j
                                                (f2cl-lib:int-sub
                                                 1)))
                           nil)
                 (tagbody
                   (setf (f2cl-lib:fref work-%data%
                                        ((f2cl-lib:int-add
                                          (f2cl-lib:int-mul
                                           (f2cl-lib:int-add jw 2) n)
                                          jr))
                                        ((1 *)) work-%offset%)
                         (* xscale
                            (f2cl-lib:fref work-%data%
                                           ((f2cl-lib:int-add
                                             (f2cl-lib:int-mul
                                              (f2cl-lib:int-add jw
                                                                2)
                                              n)
                                             jr))
                                           ((1 *)) work-%offset%)))
                  label80))
              label90))
          (setf xmax (* xmax xscale))))
        (f2cl-lib:fdo (jw 1 (f2cl-lib:int-add jw 1))
                      ((> jw nw) nil)
          (tagbody
            (f2cl-lib:fdo (ja 1 (f2cl-lib:int-add ja 1))
                          ((> ja na) nil)
              (tagbody
                (setf (f2cl-lib:fref sums (ja jw) ((1 2) (1 2)))
                      zero)
                (setf (f2cl-lib:fref sump (ja jw) ((1 2) (1 2)))
                      zero)
                (f2cl-lib:fdo (jr je (f2cl-lib:int-add jr 1))
                              ((> jr
                                  (f2cl-lib:int-add j
                                                    (f2cl-lib:int-sub
                                                     1)))
                               nil)
                  (tagbody
                    (setf (f2cl-lib:fref sums (ja jw) ((1 2) (1 2)))
                          (+
                           (f2cl-lib:fref sums (ja jw)
                                          ((1 2) (1 2)))
                           (*
                            (f2cl-lib:fref s-%data%
                                           (jr
                                            (f2cl-lib:int-sub
                                             (f2cl-lib:int-add j
```

```
                                                                ja)
                                                  1))
                                                ((1 lds) (1 *))
                                                s-%offset%)
                                  (f2cl-lib:fref work-%data%
                                                ((f2cl-lib:int-add
                                                  (f2cl-lib:int-mul
                                                   (f2cl-lib:int-add jw
                                                                     1)
                                                   n)
                                                  jr))
                                                ((1 *))
                                                work-%offset%))))
                        (setf (f2cl-lib:fref sump (ja jw) ((1 2) (1 2)))
                                (+
                                 (f2cl-lib:fref sump (ja jw)
                                                ((1 2) (1 2)))
                                 (*
                                  (f2cl-lib:fref p-%data%
                                                (jr
                                                 (f2cl-lib:int-sub
                                                  (f2cl-lib:int-add j
                                                                ja)
                                                  1))
                                                ((1 ldp) (1 *))
                                                p-%offset%)
                                  (f2cl-lib:fref work-%data%
                                                ((f2cl-lib:int-add
                                                  (f2cl-lib:int-mul
                                                   (f2cl-lib:int-add jw
                                                                     1)
                                                   n)
                                                  jr))
                                                ((1 *))
                                                work-%offset%))))
                    label100))
                label110))
            label120))
        (f2cl-lib:fdo (ja 1 (f2cl-lib:int-add ja 1))
                      ((> ja na) nil)
          (tagbody
            (cond
             (ilcplx
              (setf (f2cl-lib:fref sum (ja 1) ((1 2) (1 2)))
                    (-
                     (+
                      (* (- acoef)
                         (f2cl-lib:fref sums (ja 1) ((1 2) (1 2))))
                      (* bcoefr
                         (f2cl-lib:fref sump (ja 1)
```

```
                                        ((1 2) (1 2)))))
                    (* bcoefi
                       (f2cl-lib:fref sump (ja 2)
                                      ((1 2) (1 2))))))
         (setf (f2cl-lib:fref sum (ja 2) ((1 2) (1 2)))
                 (+
                    (* (- acoef)
                       (f2cl-lib:fref sums (ja 2) ((1 2) (1 2))))
                    (* bcoefr
                       (f2cl-lib:fref sump (ja 2) ((1 2) (1 2))))
                    (* bcoefi
                       (f2cl-lib:fref sump (ja 1)
                                      ((1 2) (1 2)))))))
        (t
         (setf (f2cl-lib:fref sum (ja 1) ((1 2) (1 2)))
                 (+
                    (* (- acoef)
                       (f2cl-lib:fref sums (ja 1) ((1 2) (1 2))))
                    (* bcoefr
                       (f2cl-lib:fref sump (ja 1)
                                      ((1 2) (1 2))))))))
      label130))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
         var-17)
        (dlaln2 f2cl-lib:%true% na nw dmin acoef
         (f2cl-lib:array-slice s-%data% double-float (j j)
                               ((1 lds) (1 *)) s-%offset%)
         lds
         (f2cl-lib:array-slice bdiag double-float (1) ((1 2)))
         (f2cl-lib:array-slice bdiag double-float (2) ((1 2)))
         sum 2 bcoefr bcoefi
         (f2cl-lib:array-slice work-%data% double-float
                               ((+ (f2cl-lib:int-mul 2 n) j))
                               ((1 *)) work-%offset%)
         n scale temp iinfo)
      (declare
       (ignore var-0 var-5 var-7 var-8 var-9 var-10 var-13))
      (when var-1 (setf na var-1))
      (when var-2 (setf nw var-2))
      (when var-3 (setf dmin var-3))
      (when var-4 (setf acoef var-4))
      (when var-6 (setf lds var-6))
      (when var-11 (setf bcoefr var-11))
      (when var-12 (setf bcoefi var-12))
      (when var-14 (setf n var-14))
      (when var-15 (setf scale var-15))
      (when var-16 (setf temp var-16))
      (when var-17 (setf iinfo var-17)))
```

```
      (cond
       ((< scale one)
        (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                      ((> jw
                          (f2cl-lib:int-add nw
                                            (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (f2cl-lib:fdo (jr je (f2cl-lib:int-add jr 1))
                          ((> jr
                              (f2cl-lib:int-add j
                                                (f2cl-lib:int-sub
                                                 1)))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-add
                                       (f2cl-lib:int-mul
                                        (f2cl-lib:int-add jw 2) n)
                                       jr))
                                     ((1 *)) work-%offset%)
                      (* scale
                         (f2cl-lib:fref work-%data%
                                        ((f2cl-lib:int-add
                                          (f2cl-lib:int-mul
                                           (f2cl-lib:int-add jw
                                                             2)
                                           n)
                                          jr))
                                        ((1 *)) work-%offset%)))
              label140))
          label150))
        (setf xmax (* scale xmax))))
      (setf xmax (max xmax temp))
     label160))
  (setf ieig (f2cl-lib:int-add ieig 1))
  (cond
   (ilback
    (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                  ((> jw
                      (f2cl-lib:int-add nw (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10)
            (dgemv "N" n
             (f2cl-lib:int-sub (f2cl-lib:int-add n 1) je) one
             (f2cl-lib:array-slice vl-%data% double-float (1 je)
                                   ((1 ldvl) (1 *)) vl-%offset%)
```

```
            ldvl
            (f2cl-lib:array-slice work-%data% double-float
                                  ((+
                                    (f2cl-lib:int-mul
                                     (f2cl-lib:int-add jw 2) n)
                                    je))
                                  ((1 *)) work-%offset%)
            1 zero
            (f2cl-lib:array-slice work-%data% double-float
                                  ((+
                                    (f2cl-lib:int-mul
                                     (f2cl-lib:int-add jw 4) n)
                                    1))
                                  ((1 *)) work-%offset%)
            1)
          (declare
           (ignore var-0 var-2 var-4 var-6 var-7 var-9 var-10))
          (when var-1 (setf n var-1))
          (when var-3 (setf one var-3))
          (when var-5 (setf ldvl var-5))
          (when var-8 (setf zero var-8)))
       label170))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (dlacpy " " n nw
          (f2cl-lib:array-slice work-%data% double-float
                                ((+ (f2cl-lib:int-mul 4 n) 1))
                                ((1 *)) work-%offset%)
          n
          (f2cl-lib:array-slice vl-%data% double-float (1 je)
                                ((1 ldvl) (1 *)) vl-%offset%)
          ldvl)
       (declare (ignore var-0 var-3 var-5))
       (when var-1 (setf n var-1))
       (when var-2 (setf nw var-2))
       (when var-4 (setf n var-4))
       (when var-6 (setf ldvl var-6)))
     (setf ibeg 1))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (dlacpy " " n nw
          (f2cl-lib:array-slice work-%data% double-float
                                ((+ (f2cl-lib:int-mul 2 n) 1))
                                ((1 *)) work-%offset%)
          n
          (f2cl-lib:array-slice vl-%data% double-float (1 ieig)
                                ((1 ldvl) (1 *)) vl-%offset%)
          ldvl)
       (declare (ignore var-0 var-3 var-5))
```

```
      (when var-1 (setf n var-1))
      (when var-2 (setf nw var-2))
      (when var-4 (setf n var-4))
      (when var-6 (setf ldvl var-6)))
    (setf ibeg je)))
  (setf xmax zero)
  (cond
   (ilcplx
    (f2cl-lib:fdo (j ibeg (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf xmax
                (max xmax
                     (+
                      (abs
                       (f2cl-lib:fref vl-%data% (j ieig)
                                      ((1 ldvl) (1 *))
                                      vl-%offset%))
                      (abs
                       (f2cl-lib:fref vl-%data%
                                      (j (f2cl-lib:int-add ieig 1))
                                      ((1 ldvl) (1 *))
                                      vl-%offset%)))))
       label180)))
   (t
    (f2cl-lib:fdo (j ibeg (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf xmax
                (max xmax
                     (abs
                      (f2cl-lib:fref vl-%data% (j ieig)
                                     ((1 ldvl) (1 *))
                                     vl-%offset%))))
       label190))))
  (cond
   ((> xmax safmin) (setf xscale (/ one xmax))
    (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                  ((> jw
                      (f2cl-lib:int-add nw (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (f2cl-lib:fdo (jr ibeg (f2cl-lib:int-add jr 1))
                      ((> jr n) nil)
          (tagbody
            (setf (f2cl-lib:fref vl-%data%
                                 (jr (f2cl-lib:int-add ieig jw))
                                 ((1 ldvl) (1 *)) vl-%offset%)
                  (* xscale
                     (f2cl-lib:fref vl-%data%
```

```
                                                (jr
                                                 (f2cl-lib:int-add ieig jw))
                                                ((1 ldvl) (1 *))
                                                vl-%offset%)))
                    label200))
                 label210))))
          (setf ieig (f2cl-lib:int-sub (f2cl-lib:int-add ieig nw) 1))
         label220))))
    (cond
     (compr (setf ieig (f2cl-lib:int-add im 1))
      (setf ilcplx f2cl-lib:%false%)
      (f2cl-lib:fdo (je n (f2cl-lib:int-add je (f2cl-lib:int-sub 1)))
                    ((> je 1) nil)
        (tagbody
          (cond (ilcplx (setf ilcplx f2cl-lib:%false%) (go label500)))
          (setf nw 1)
          (cond
           ((> je 1)
            (cond
             ((/=
               (f2cl-lib:fref s
                              (je
                               (f2cl-lib:int-add je (f2cl-lib:int-sub 1)))
                              ((1 lds) (1 *)))
               zero)
              (setf ilcplx f2cl-lib:%true%) (setf nw 2)))))
          (cond (ilall (setf ilcomp f2cl-lib:%true%))
                (ilcplx
                 (setf ilcomp
                         (or
                          (f2cl-lib:fref select-%data% (je) ((1 *))
                                         select-%offset%)
                          (f2cl-lib:fref select-%data%
                                         ((f2cl-lib:int-sub je 1)) ((1 *))
                                         select-%offset%))))
                (t
                 (setf ilcomp
                         (f2cl-lib:fref select-%data% (je) ((1 *))
                                        select-%offset%))))
          (if (not ilcomp)
              (go label500))
          (cond
           ((not ilcplx)
            (cond
             ((and
               (<= (abs (f2cl-lib:fref s (je je) ((1 lds) (1 *)))) safmin)
               (<= (abs (f2cl-lib:fref p (je je) ((1 ldp) (1 *)))) safmin))
              (setf ieig (f2cl-lib:int-sub ieig 1))
              (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                            ((> jr n) nil)
```

```
      (tagbody
        (setf (f2cl-lib:fref vr-%data% (jr ieig) ((1 ldvr) (1 *))
                             vr-%offset%)
              zero)
       label230))
    (setf (f2cl-lib:fref vr-%data% (ieig ieig) ((1 ldvr) (1 *))
                         vr-%offset%)
          one)
    (go label500)))))
(f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
              ((> jw (f2cl-lib:int-add nw (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                  ((> jr n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-mul
                                (f2cl-lib:int-add jw 2) n)
                               jr))
                             ((1 *)) work-%offset%)
              zero)
       label240))
   label250))
(cond
 ((not ilcplx)
  (setf temp
        (/ one
           (max
            (*
             (abs
              (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                             s-%offset%))
             ascale)
            (*
             (abs
              (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
                             p-%offset%))
             bscale)
            safmin)))
  (setf salfar
        (* temp
           (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                          s-%offset%)
           ascale))
  (setf sbeta
        (* temp
           (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
                          p-%offset%)
```

```
           bscale))
 (setf acoef (* sbeta ascale)) (setf bcoefr (* salfar bscale))
 (setf bcoefi zero) (setf scale one)
 (setf lsa (and (>= (abs sbeta) safmin) (< (abs acoef) small)))
 (setf lsb
        (and (>= (abs salfar) safmin) (< (abs bcoefr) small)))
 (if lsa
     (setf scale (* (/ small (abs sbeta)) (min anorm big))))
 (if lsb
     (setf scale
             (max scale
                  (* (/ small (abs salfar)) (min bnorm big)))))
 (cond
  ((or lsa lsb)
   (setf scale
           (min scale
                (/ one
                   (* safmin
                      (max one (abs acoef) (abs bcoefr))))))
   (cond (lsa (setf acoef (* ascale (* scale sbeta))))
         (t (setf acoef (* scale acoef))))
   (cond (lsb (setf bcoefr (* bscale (* scale salfar))))
         (t (setf bcoefr (* scale bcoefr))))))
 (setf acoefa (abs acoef)) (setf bcoefa (abs bcoefr))
 (setf (f2cl-lib:fref work-%data%
                      ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                         je))
                      ((1 *)) work-%offset%)
        one)
 (setf xmax one)
 (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
               ((> jr
                   (f2cl-lib:int-add je (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add
                            (f2cl-lib:int-mul 2 n) jr))
                          ((1 *)) work-%offset%)
             (-
              (* bcoefr
                 (f2cl-lib:fref p-%data% (jr je) ((1 ldp) (1 *))
                                p-%offset%))
              (* acoef
                 (f2cl-lib:fref s-%data% (jr je) ((1 lds) (1 *))
                                s-%offset%))))
    label260)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
     var-9)
    (dlag2
     (f2cl-lib:array-slice s-%data% double-float
                           ((+ je (f2cl-lib:int-sub 1))
                            (f2cl-lib:int-sub je 1))
                           ((1 lds) (1 *)) s-%offset%)
     lds
     (f2cl-lib:array-slice p-%data% double-float
                           ((+ je (f2cl-lib:int-sub 1))
                            (f2cl-lib:int-sub je 1))
                           ((1 ldp) (1 *)) p-%offset%)
     ldp (* safmin safety) acoef temp bcoefr temp2 bcoefi)
  (declare (ignore var-0 var-2 var-4))
  (when var-1 (setf lds var-1))
  (when var-3 (setf ldp var-3))
  (when var-5 (setf acoef var-5))
  (when var-6 (setf temp var-6))
  (when var-7 (setf bcoefr var-7))
  (when var-8 (setf temp2 var-8))
  (when var-9 (setf bcoefi var-9)))
(cond
 ((= bcoefi zero) (setf info (f2cl-lib:int-sub je 1))
  (go end_label)))
(setf acoefa (abs acoef))
(setf bcoefa (+ (abs bcoefr) (abs bcoefi))) (setf scale one)
(if (and (< (* acoefa ulp) safmin) (>= acoefa safmin))
    (setf scale (/ (/ safmin ulp) acoefa)))
(if (and (< (* bcoefa ulp) safmin) (>= bcoefa safmin))
    (setf scale (max scale (/ (/ safmin ulp) bcoefa))))
(if (> (* safmin acoefa) ascale)
    (setf scale (/ ascale (* safmin acoefa))))
(if (> (* safmin bcoefa) bscale)
    (setf scale (min scale (/ bscale (* safmin bcoefa)))))
(cond
 ((/= scale one) (setf acoef (* scale acoef))
  (setf acoefa (abs acoef)) (setf bcoefr (* scale bcoefr))
  (setf bcoefi (* scale bcoefi))
  (setf bcoefa (+ (abs bcoefr) (abs bcoefi)))))
(setf temp
        (* acoef
           (f2cl-lib:fref s-%data% (je (f2cl-lib:int-sub je 1))
                          ((1 lds) (1 *)) s-%offset%)))
(setf temp2r
        (-
         (* acoef
            (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                           s-%offset%))
         (* bcoefr
            (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
                           p-%offset%))))
```

```
(setf temp2i
        (* (- bcoefi)
           (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
                          p-%offset%)))
(cond
 ((>= (abs temp) (+ (abs temp2r) (abs temp2i)))
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 2 n) je))
                       ((1 *)) work-%offset%)
         one)
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 3 n) je))
                       ((1 *)) work-%offset%)
         zero)
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-sub
                         (f2cl-lib:int-add
                          (f2cl-lib:int-mul 2 n) je)
                         1))
                       ((1 *)) work-%offset%)
         (/ (- temp2r) temp))
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-sub
                         (f2cl-lib:int-add
                          (f2cl-lib:int-mul 3 n) je)
                         1))
                       ((1 *)) work-%offset%)
         (/ (- temp2i) temp)))
 (t
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-sub
                         (f2cl-lib:int-add
                          (f2cl-lib:int-mul 2 n) je)
                         1))
                       ((1 *)) work-%offset%)
         one)
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-sub
                         (f2cl-lib:int-add
                          (f2cl-lib:int-mul 3 n) je)
                         1))
                       ((1 *)) work-%offset%)
         zero)
  (setf temp
         (* acoef
            (f2cl-lib:fref s-%data%
                           ((f2cl-lib:int-sub je 1) je)
                           ((1 lds) (1 *)) s-%offset%)))
```

```
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 2 n) je))
                       ((1 *)) work-%offset%)
          (/
           (-
            (* bcoefr
               (f2cl-lib:fref p-%data%
                              ((f2cl-lib:int-sub je 1)
                               (f2cl-lib:int-sub je 1))
                              ((1 ldp) (1 *)) p-%offset%))
            (* acoef
               (f2cl-lib:fref s-%data%
                              ((f2cl-lib:int-sub je 1)
                               (f2cl-lib:int-sub je 1))
                              ((1 lds) (1 *)) s-%offset%)))
           temp))
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 3 n) je))
                       ((1 *)) work-%offset%)
          (/
           (* bcoefi
              (f2cl-lib:fref p-%data%
                             ((f2cl-lib:int-sub je 1)
                              (f2cl-lib:int-sub je 1))
                             ((1 ldp) (1 *)) p-%offset%))
           temp))))
(setf xmax
        (max
         (+
          (abs
           (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add
                            (f2cl-lib:int-mul 2 n) je))
                          ((1 *)) work-%offset%))
          (abs
           (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add
                            (f2cl-lib:int-mul 3 n) je))
                          ((1 *)) work-%offset%)))
         (+
          (abs
           (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add
                             (f2cl-lib:int-mul 2 n) je)
                            1))
                          ((1 *)) work-%offset%))
          (abs
```

```
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add
                              (f2cl-lib:int-mul 3 n) je)
                             1))
                           ((1 *)) work-%offset%)))))
(setf creala
        (* acoef
           (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add
                              (f2cl-lib:int-mul 2 n) je)
                             1))
                           ((1 *)) work-%offset%)))
(setf cimaga
        (* acoef
           (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add
                              (f2cl-lib:int-mul 3 n) je)
                             1))
                           ((1 *)) work-%offset%)))
(setf crealb
        (-
         (* bcoefr
            (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add
                               (f2cl-lib:int-mul 2 n) je)
                              1))
                            ((1 *)) work-%offset%))
         (* bcoefi
            (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add
                               (f2cl-lib:int-mul 3 n) je)
                              1))
                            ((1 *)) work-%offset%))))
(setf cimagb
        (+
         (* bcoefi
            (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add
                               (f2cl-lib:int-mul 2 n) je)
                              1))
                            ((1 *)) work-%offset%))
         (* bcoefr
            (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-sub
```

```
                                (f2cl-lib:int-add
                                 (f2cl-lib:int-mul 3 n) je)
                                1))
                              ((1 *)) work-%offset%))))
(setf cre2a
        (* acoef
           (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add
                            (f2cl-lib:int-mul 2 n) je))
                          ((1 *)) work-%offset%)))
(setf cim2a
        (* acoef
           (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add
                            (f2cl-lib:int-mul 3 n) je))
                          ((1 *)) work-%offset%)))
(setf cre2b
        (-
         (* bcoefr
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add
                             (f2cl-lib:int-mul 2 n) je))
                           ((1 *)) work-%offset%))
         (* bcoefi
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add
                             (f2cl-lib:int-mul 3 n) je))
                           ((1 *)) work-%offset%))))
(setf cim2b
        (+
         (* bcoefi
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add
                             (f2cl-lib:int-mul 2 n) je))
                           ((1 *)) work-%offset%))
         (* bcoefr
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add
                             (f2cl-lib:int-mul 3 n) je))
                           ((1 *)) work-%offset%))))
(f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
              ((> jr
                  (f2cl-lib:int-add je (f2cl-lib:int-sub 2)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add
                           (f2cl-lib:int-mul 2 n) jr))
                         ((1 *)) work-%offset%)
            (+
```

```
                    (-
                     (+
                      (* (- creala)
                         (f2cl-lib:fref s-%data%
                                        (jr (f2cl-lib:int-sub je 1))
                                        ((1 lds) (1 *)) s-%offset%))
                      (* crealb
                         (f2cl-lib:fref p-%data%
                                        (jr (f2cl-lib:int-sub je 1))
                                        ((1 ldp) (1 *)) p-%offset%)))
                     (* cre2a
                        (f2cl-lib:fref s-%data% (jr je)
                                       ((1 lds) (1 *)) s-%offset%)))
                    (* cre2b
                       (f2cl-lib:fref p-%data% (jr je) ((1 ldp) (1 *))
                                      p-%offset%))))
          (setf (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add
                                 (f2cl-lib:int-mul 3 n) jr))
                               ((1 *)) work-%offset%)
                  (+
                   (-
                    (+
                     (* (- cimaga)
                        (f2cl-lib:fref s-%data%
                                       (jr (f2cl-lib:int-sub je 1))
                                       ((1 lds) (1 *)) s-%offset%))
                     (* cimagb
                        (f2cl-lib:fref p-%data%
                                       (jr (f2cl-lib:int-sub je 1))
                                       ((1 ldp) (1 *)) p-%offset%)))
                    (* cim2a
                       (f2cl-lib:fref s-%data% (jr je)
                                      ((1 lds) (1 *)) s-%offset%)))
                   (* cim2b
                      (f2cl-lib:fref p-%data% (jr je) ((1 ldp) (1 *))
                                     p-%offset%))))
         label270))))
    (setf dmin
            (max (* ulp acoefa anorm) (* ulp bcoefa bnorm) safmin))
    (setf il2by2 f2cl-lib:%false%)
    (f2cl-lib:fdo (j (f2cl-lib:int-add je (f2cl-lib:int-sub nw))
                   (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                  ((> j 1) nil)
      (tagbody
        (cond
         ((and (not il2by2) (> j 1))
          (cond
           ((/=
             (f2cl-lib:fref s
```

```
                    (j
                     (f2cl-lib:int-add j
                                       (f2cl-lib:int-sub 1)))
                    ((1 lds) (1 *)))
     zero)
    (setf il2by2 f2cl-lib:%true%) (go label370)))))
(setf (f2cl-lib:fref bdiag (1) ((1 2)))
        (f2cl-lib:fref p-%data% (j j) ((1 ldp) (1 *))
                       p-%offset%))
(cond
 (il2by2 (setf na 2)
  (setf (f2cl-lib:fref bdiag (2) ((1 2)))
          (f2cl-lib:fref p-%data%
                         ((f2cl-lib:int-add j 1)
                          (f2cl-lib:int-add j 1))
                         ((1 ldp) (1 *)) p-%offset%)))
 (t (setf na 1)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
    (dlaln2 f2cl-lib:%false% na nw dmin acoef
     (f2cl-lib:array-slice s-%data% double-float (j j)
                           ((1 lds) (1 *)) s-%offset%)
     lds
     (f2cl-lib:array-slice bdiag double-float (1) ((1 2)))
     (f2cl-lib:array-slice bdiag double-float (2) ((1 2)))
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul 2 n) j))
                           ((1 *)) work-%offset%)
     n bcoefr bcoefi sum 2 scale temp iinfo)
  (declare
   (ignore var-0 var-5 var-7 var-8 var-9 var-13 var-14))
  (when var-1 (setf na var-1))
  (when var-2 (setf nw var-2))
  (when var-3 (setf dmin var-3))
  (when var-4 (setf acoef var-4))
  (when var-6 (setf lds var-6))
  (when var-10 (setf n var-10))
  (when var-11 (setf bcoefr var-11))
  (when var-12 (setf bcoefi var-12))
  (when var-15 (setf scale var-15))
  (when var-16 (setf temp var-16))
  (when var-17 (setf iinfo var-17)))
(cond
 ((< scale one)
  (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                ((> jw
                    (f2cl-lib:int-add nw
                                      (f2cl-lib:int-sub 1)))
```

```
                 nil)
      (tagbody
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr je) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-add jw 2) n)
                                   jr))
                                 ((1 *)) work-%offset%)
                  (* scale
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add
                                      (f2cl-lib:int-mul
                                       (f2cl-lib:int-add jw
                                                         2)
                                       n)
                                      jr))
                                    ((1 *)) work-%offset%)))
          label280))
      label290))))
(setf xmax (max (* scale xmax) temp))
(f2cl-lib:fdo (jw 1 (f2cl-lib:int-add jw 1))
              ((> jw nw) nil)
  (tagbody
    (f2cl-lib:fdo (ja 1 (f2cl-lib:int-add ja 1))
                  ((> ja na) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-sub
                               (f2cl-lib:int-add
                                (f2cl-lib:int-mul
                                 (f2cl-lib:int-add jw 1) n)
                                j ja)
                               1))
                             ((1 *)) work-%offset%)
                (f2cl-lib:fref sum (ja jw) ((1 2) (1 2))))
       label300))
   label310))
(cond
 ((> j 1) (setf xscale (/ one (max one xmax)))
  (setf temp
          (+
           (* acoefa
              (f2cl-lib:fref work-%data% (j) ((1 *))
                             work-%offset%))
           (* bcoefa
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add n j)) ((1 *))
```

```
                            work-%offset%))))
(if il2by2
    (setf temp
            (max temp
                 (+
                  (* acoefa
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add j
                                                       1))
                                    ((1 *)) work-%offset%))
                  (* bcoefa
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add n j
                                                       1))
                                    ((1 *))
                                    work-%offset%))))))
(setf temp (max temp acoefa bcoefa))
(cond
 ((> temp (* bignum xscale))
  (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                ((> jw
                    (f2cl-lib:int-add nw
                                      (f2cl-lib:int-sub
                                       1)))
                 nil)
    (tagbody
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr je) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-add jw 2)
                                  n)
                                 jr))
                               ((1 *)) work-%offset%)
                  (* xscale
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add
                                      (f2cl-lib:int-mul
                                       (f2cl-lib:int-add jw
                                                         2)
                                       n)
                                      jr))
                                    ((1 *))
                                    work-%offset%)))
         label320))
     label330))
  (setf xmax (* xmax xscale))))
(f2cl-lib:fdo (ja 1 (f2cl-lib:int-add ja 1))
```

```
            ((> ja na) nil)
(tagbody
  (cond
   (ilcplx
    (setf creala
            (* acoef
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add
                                 (f2cl-lib:int-mul 2 n) j
                                 ja)
                                1))
                              ((1 *)) work-%offset%)))
    (setf cimaga
            (* acoef
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add
                                 (f2cl-lib:int-mul 3 n) j
                                 ja)
                                1))
                              ((1 *)) work-%offset%)))
    (setf crealb
            (-
             (* bcoefr
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add
                                  (f2cl-lib:int-mul 2 n)
                                  j ja)
                                 1))
                               ((1 *)) work-%offset%))
             (* bcoefi
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add
                                  (f2cl-lib:int-mul 3 n)
                                  j ja)
                                 1))
                               ((1 *)) work-%offset%))))
    (setf cimagb
            (+
             (* bcoefi
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add
                                  (f2cl-lib:int-mul 2 n)
                                  j ja)
                                 1))
                               ((1 *)) work-%offset%))
```

```
                (* bcoefr
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-sub
                                    (f2cl-lib:int-add
                                     (f2cl-lib:int-mul 3 n)
                                     j ja)
                                    1))
                                  ((1 *)) work-%offset%))))
(f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
              ((> jr
                  (f2cl-lib:int-add j
                                    (f2cl-lib:int-sub
                                     1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add
                           (f2cl-lib:int-mul 2 n)
                           jr))
                         ((1 *)) work-%offset%)
          (+
           (-
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add
                             (f2cl-lib:int-mul 2 n)
                             jr))
                           ((1 *)) work-%offset%)
            (* creala
               (f2cl-lib:fref s-%data%
                              (jr
                               (f2cl-lib:int-sub
                                (f2cl-lib:int-add j
                                                  ja)
                                1))
                              ((1 lds) (1 *))
                              s-%offset%)))
           (* crealb
              (f2cl-lib:fref p-%data%
                             (jr
                              (f2cl-lib:int-sub
                               (f2cl-lib:int-add j
                                                 ja)
                               1))
                             ((1 ldp) (1 *))
                             p-%offset%))))
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add
                           (f2cl-lib:int-mul 3 n)
                           jr))
                         ((1 *)) work-%offset%)
```

```
                    (+
                     (-
                      (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-add
                                       (f2cl-lib:int-mul 3 n)
                                       jr))
                                     ((1 *)) work-%offset%)
                      (* cimaga
                         (f2cl-lib:fref s-%data%
                                        (jr
                                         (f2cl-lib:int-sub
                                          (f2cl-lib:int-add j
                                                            ja)
                                          1))
                                        ((1 lds) (1 *))
                                        s-%offset%)))
                     (* cimagb
                        (f2cl-lib:fref p-%data%
                                       (jr
                                        (f2cl-lib:int-sub
                                         (f2cl-lib:int-add j
                                                           ja)
                                         1))
                                       ((1 ldp) (1 *))
                                       p-%offset%))))
                label340)))
            (t
             (setf creala
                     (* acoef
                        (f2cl-lib:fref work-%data%
                                       ((f2cl-lib:int-sub
                                         (f2cl-lib:int-add
                                          (f2cl-lib:int-mul 2 n) j
                                          ja)
                                         1))
                                       ((1 *)) work-%offset%)))
             (setf crealb
                     (* bcoefr
                        (f2cl-lib:fref work-%data%
                                       ((f2cl-lib:int-sub
                                         (f2cl-lib:int-add
                                          (f2cl-lib:int-mul 2 n) j
                                          ja)
                                         1))
                                       ((1 *)) work-%offset%)))
             (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                           ((> jr
                               (f2cl-lib:int-add j
                                                 (f2cl-lib:int-sub
                                                  1)))
```

```
                                nil)
                        (tagbody
                          (setf (f2cl-lib:fref work-%data%
                                               ((f2cl-lib:int-add
                                                 (f2cl-lib:int-mul 2 n)
                                                 jr))
                                               ((1 *)) work-%offset%)
                                (+
                                 (-
                                  (f2cl-lib:fref work-%data%
                                                 ((f2cl-lib:int-add
                                                   (f2cl-lib:int-mul 2 n)
                                                   jr))
                                                 ((1 *)) work-%offset%)
                                  (* creala
                                     (f2cl-lib:fref s-%data%
                                                    (jr
                                                     (f2cl-lib:int-sub
                                                      (f2cl-lib:int-add j
                                                                        ja)
                                                      1))
                                                    ((1 lds) (1 *))
                                                    s-%offset%)))
                                 (* crealb
                                    (f2cl-lib:fref p-%data%
                                                   (jr
                                                    (f2cl-lib:int-sub
                                                     (f2cl-lib:int-add j
                                                                       ja)
                                                     1))
                                                   ((1 ldp) (1 *))
                                                   p-%offset%))))
                        label350))))
                 label360))))
            (setf il2by2 f2cl-lib:%false%)
           label370))
        (setf ieig (f2cl-lib:int-sub ieig nw))
        (cond
         (ilback
          (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                        ((> jw
                            (f2cl-lib:int-add nw (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                            ((> jr n) nil)
                (tagbody
                  (setf (f2cl-lib:fref work-%data%
                                       ((f2cl-lib:int-add
                                         (f2cl-lib:int-mul
```

```
                                        (f2cl-lib:int-add jw 4) n)
                                       jr))
                                    ((1 *)) work-%offset%)
                          (*
                           (f2cl-lib:fref work-%data%
                                        ((f2cl-lib:int-add
                                          (f2cl-lib:int-mul
                                           (f2cl-lib:int-add jw 2) n)
                                          1))
                                        ((1 *)) work-%offset%)
                           (f2cl-lib:fref vr-%data% (jr 1)
                                        ((1 ldvr) (1 *)) vr-%offset%)))
                 label380))
             (f2cl-lib:fdo (jc 2 (f2cl-lib:int-add jc 1))
                           ((> jc je) nil)
               (tagbody
                 (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                               ((> jr n) nil)
                   (tagbody
                     (setf (f2cl-lib:fref work-%data%
                                          ((f2cl-lib:int-add
                                            (f2cl-lib:int-mul
                                             (f2cl-lib:int-add jw 4) n)
                                            jr))
                                          ((1 *)) work-%offset%)
                           (+
                            (f2cl-lib:fref work-%data%
                                           ((f2cl-lib:int-add
                                             (f2cl-lib:int-mul
                                              (f2cl-lib:int-add jw 4)
                                              n)
                                             jr))
                                           ((1 *)) work-%offset%)
                            (*
                             (f2cl-lib:fref work-%data%
                                            ((f2cl-lib:int-add
                                              (f2cl-lib:int-mul
                                               (f2cl-lib:int-add jw 2)
                                               n)
                                              jc))
                                            ((1 *)) work-%offset%)
                             (f2cl-lib:fref vr-%data% (jr jc)
                                            ((1 ldvr) (1 *))
                                            vr-%offset%))))
                   label390))
                 label400))
           label410))
       (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                     ((> jw
                         (f2cl-lib:int-add nw (f2cl-lib:int-sub 1)))
```

```
                 nil)
      (tagbody
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr n) nil)
          (tagbody
            (setf (f2cl-lib:fref vr-%data%
                                 (jr (f2cl-lib:int-add ieig jw))
                                 ((1 ldvr) (1 *)) vr-%offset%)
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-add jw 4) n)
                                   jr))
                                 ((1 *)) work-%offset%))
          label420))
      label430))
    (setf iend n))
   (t
    (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                  ((> jw
                      (f2cl-lib:int-add nw (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr n) nil)
          (tagbody
            (setf (f2cl-lib:fref vr-%data%
                                 (jr (f2cl-lib:int-add ieig jw))
                                 ((1 ldvr) (1 *)) vr-%offset%)
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-add jw 2) n)
                                   jr))
                                 ((1 *)) work-%offset%))
          label440))
      label450))
    (setf iend je)))
  (setf xmax zero)
  (cond
   (ilcplx
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j iend) nil)
      (tagbody
        (setf xmax
              (max xmax
                   (+
                    (abs
                     (f2cl-lib:fref vr-%data% (j ieig)
                                    ((1 ldvr) (1 *))
```

```
                                                vr-%offset%))
                                   (abs
                                    (f2cl-lib:fref vr-%data%
                                                   (j (f2cl-lib:int-add ieig 1))
                                                   ((1 ldvr) (1 *))
                                                   vr-%offset%)))))
                        label460)))
                     (t
                      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                                    ((> j iend) nil)
                        (tagbody
                          (setf xmax
                                  (max xmax
                                       (abs
                                        (f2cl-lib:fref vr-%data% (j ieig)
                                                       ((1 ldvr) (1 *))
                                                       vr-%offset%))))
                        label470))))
                    (cond
                     ((> xmax safmin) (setf xscale (/ one xmax))
                      (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                                    ((> jw
                                        (f2cl-lib:int-add nw (f2cl-lib:int-sub 1)))
                                     nil)
                        (tagbody
                          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                                        ((> jr iend) nil)
                            (tagbody
                              (setf (f2cl-lib:fref vr-%data%
                                                   (jr (f2cl-lib:int-add ieig jw))
                                                   ((1 ldvr) (1 *)) vr-%offset%)
                                    (* xscale
                                       (f2cl-lib:fref vr-%data%
                                                      (jr
                                                       (f2cl-lib:int-add ieig jw))
                                                      ((1 ldvr) (1 *))
                                                      vr-%offset%)))
                              label480))
                        label490))))
                  label500))))
          (go end_label)
        end_label
          (return
           (values side howmny nil n nil lds nil ldp nil ldvl nil nil nil m nil
                   info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dtgevc
```

```
        fortran-to-lisp::*f2cl-function-info*)
    (fortran-to-lisp::make-f2cl-finfo :arg-types
                                      '((string) (string)
                                        (array fortran-to-lisp::logical
                                         (*))
                                        (fortran-to-lisp::integer4)
                                        (array double-float (*))
                                        (fortran-to-lisp::integer4)
                                        (array double-float (*))
                                        (fortran-to-lisp::integer4)
                                        (array double-float (*))
                                        (fortran-to-lisp::integer4)
                                        (array double-float (*))
                                        (fortran-to-lisp::integer4)
                                        (fortran-to-lisp::integer4)
                                        (fortran-to-lisp::integer4)
                                        (array double-float (*))
                                        (fortran-to-lisp::integer4))
                                      :return-values
                                      '(fortran-to-lisp::side
                                        fortran-to-lisp::howmny nil
                                        fortran-to-lisp::n nil
                                        fortran-to-lisp::lds nil
                                        fortran-to-lisp::ldp nil
                                        fortran-to-lisp::ldvl nil nil nil
                                        fortran-to-lisp::m nil
                                        fortran-to-lisp::info)
                                      :calls 'nil)))
```

———-

32.40 dtgexc

32.40.1 Axiom unit tests

— dtgexc.input —

```
)set break resume
)sys rm -f dtgexc.output
)spool dtgexc.output
)set message test on
)set message auto off
)clear all

)spool
```

```
)lisp (bye)
```

———-

32.40.2 Axiom help page

— **dtgexc.help** —

```
==================================================================
dtgexc examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dtgexc (WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z, LDZ,
           IFST, ILST, WORK, LWORK, INFO)
           DTGEXC

Function/Subroutine Documentation
   subroutine dtgexc (logical WANTQ, logical WANTZ, integer N, double
       precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldb, * ) B, integer LDB, double precision, dimension( ldq, *
       ) Q, integer LDQ, double precision, dimension( ldz, * ) Z, integer LDZ,
       integer IFST, integer ILST, double precision, dimension( * ) WORK,
       integer LWORK, integer INFO)
       DTGEXC

Purpose:
   DTGEXC reorders the generalized real Schur decomposition of a real
   matrix pair (A,B) using an orthogonal equivalence transformation

     (A, B) = Q * (A, B) * Z**T,

   so that the diagonal block of (A, B) with row index IFST is moved
   to row ILST.

   (A, B) must be in generalized real Schur canonical form (as returned
   by DGGES), i.e. A is block upper triangular with 1-by-1 and 2-by-2
   diagonal blocks. B is upper triangular.

   Optionally, the matrices Q and Z of generalized Schur vectors are
   updated.

     Q(in) * A(in) * Z(in)**T = Q(out) * A(out) * Z(out)**T
```

```
     Q(in) * B(in) * Z(in)**T = Q(out) * B(out) * Z(out)**T

Parameters:

 WANTQ
   WANTQ is LOGICAL
   .TRUE. : update the left transformation matrix Q;
   .FALSE.: do not update Q.

 WANTZ
   WANTZ is LOGICAL
   .TRUE. : update the right transformation matrix Z;
   .FALSE.: do not update Z.

 N
   N is INTEGER
   The order of the matrices A and B. N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the matrix A in generalized real Schur canonical form.
   On exit, the updated matrix A, again in generalized
   real Schur canonical form.

 LDA
   LDA is INTEGER
   The leading dimension of the array A. LDA >= max(1,N).

 B
   B is DOUBLE PRECISION array, dimension (LDB,N)
   On entry, the matrix B in generalized real Schur canonical form (A,B).
   On exit, the updated matrix B, again in generalized
   real Schur canonical form (A,B).

 LDB
   LDB is INTEGER
   The leading dimension of the array B. LDB >= max(1,N).

 Q
   Q is DOUBLE PRECISION array, dimension (LDQ,N)
   On entry, if WANTQ = .TRUE., the orthogonal matrix Q.
   On exit, the updated matrix Q.
   If WANTQ = .FALSE., Q is not referenced.

 LDQ
   LDQ is INTEGER
   The leading dimension of the array Q. LDQ >= 1.
   If WANTQ = .TRUE., LDQ >= N.

 Z
```

```
  Z is DOUBLE PRECISION array, dimension (LDZ,N)
  On entry, if WANTZ = .TRUE., the orthogonal matrix Z.
  On exit, the updated matrix Z.
  If WANTZ = .FALSE., Z is not referenced.

LDZ
  LDZ is INTEGER
  The leading dimension of the array Z. LDZ >= 1.
  If WANTZ = .TRUE., LDZ >= N.

IFST
  IFST is INTEGER

ILST
  ILST is INTEGER
  Specify the reordering of the diagonal blocks of (A, B).
  The block with row index IFST is moved to row ILST, by a
  sequence of swapping between adjacent blocks.
  On exit, if IFST pointed on entry to the second row of
  a 2-by-2 block, it is changed to point to the first row;
  ILST always points to the first row of the block in its
  final position (which may differ from its input value by
  +1 or -1). 1 <= IFST, ILST <= N.

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  LWORK >= 1 when N <= 1, otherwise LWORK >= 4*N + 16.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  =0:  successful exit.
  <0:  if INFO = -i, the i-th argument had an illegal value.
  =1:  The transformed matrix pair (A, B) would be too far
       from generalized Schur form; the problem is ill-
       conditioned. (A, B) may have been partially reordered,
       and ILST points to the first row of the current
       position of the block being moved.
```

32.40.3 fortran code

— dtgexc.f —

```
*  =====================================================================
      SUBROUTINE DTGEXC( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                   LDZ, IFST, ILST, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      LOGICAL            WANTQ, WANTZ
      INTEGER            IFST, ILST, INFO, LDA, LDB, LDQ, LDZ, LWORK, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * ), Q( LDQ, * ),
     $                   WORK( * ), Z( LDZ, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            HERE, LWMIN, NBF, NBL, NBNEXT
*     ..
*     .. External Subroutines ..
      EXTERNAL           DTGEX2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Decode and test input arguments.
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
```

```
         INFO = -7
      ELSE IF( LDQ.LT.1 .OR. WANTQ .AND. ( LDQ.LT.MAX( 1, N ) ) ) THEN
         INFO = -9
      ELSE IF( LDZ.LT.1 .OR. WANTZ .AND. ( LDZ.LT.MAX( 1, N ) ) ) THEN
         INFO = -11
      ELSE IF( IFST.LT.1 .OR. IFST.GT.N ) THEN
         INFO = -12
      ELSE IF( ILST.LT.1 .OR. ILST.GT.N ) THEN
         INFO = -13
      END IF
*
      IF( INFO.EQ.0 ) THEN
         IF( N.LE.1 ) THEN
            LWMIN = 1
         ELSE
            LWMIN = 4*N + 16
         END IF
         WORK(1) = LWMIN
*
         IF (LWORK.LT.LWMIN .AND. .NOT.LQUERY) THEN
            INFO = -15
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DTGEXC', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.LE.1 )
     $   RETURN
*
*     Determine the first row of the specified block and find out
*     if it is 1-by-1 or 2-by-2.
*
      IF( IFST.GT.1 ) THEN
         IF( A( IFST, IFST-1 ).NE.ZERO )
     $      IFST = IFST - 1
      END IF
      NBF = 1
      IF( IFST.LT.N ) THEN
         IF( A( IFST+1, IFST ).NE.ZERO )
     $      NBF = 2
      END IF
*
*     Determine the first row of the final block
```

```
*     and find out if it is 1-by-1 or 2-by-2.
*
      IF( ILST.GT.1 ) THEN
         IF( A( ILST, ILST-1 ).NE.ZERO )
     $      ILST = ILST - 1
      END IF
      NBL = 1
      IF( ILST.LT.N ) THEN
         IF( A( ILST+1, ILST ).NE.ZERO )
     $      NBL = 2
      END IF
      IF( IFST.EQ.ILST )
     $   RETURN
*
      IF( IFST.LT.ILST ) THEN
*
*        Update ILST.
*
         IF( NBF.EQ.2 .AND. NBL.EQ.1 )
     $      ILST = ILST - 1
         IF( NBF.EQ.1 .AND. NBL.EQ.2 )
     $      ILST = ILST + 1
*
         HERE = IFST
*
   10    CONTINUE
*
*        Swap with next one below.
*
         IF( NBF.EQ.1 .OR. NBF.EQ.2 ) THEN
*
*           Current block either 1-by-1 or 2-by-2.
*
            NBNEXT = 1
            IF( HERE+NBF+1.LE.N ) THEN
               IF( A( HERE+NBF+1, HERE+NBF ).NE.ZERO )
     $            NBNEXT = 2
            END IF
            CALL DTGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                   LDZ, HERE, NBF, NBNEXT, WORK, LWORK, INFO )
            IF( INFO.NE.0 ) THEN
               ILST = HERE
               RETURN
            END IF
            HERE = HERE + NBNEXT
*
*           Test if 2-by-2 block breaks into two 1-by-1 blocks.
*
            IF( NBF.EQ.2 ) THEN
               IF( A( HERE+1, HERE ).EQ.ZERO )
```

```
     $            NBF = 3
            END IF
*
         ELSE
*
*           Current block consists of two 1-by-1 blocks, each of which
*           must be swapped individually.
*
            NBNEXT = 1
            IF( HERE+3.LE.N ) THEN
               IF( A( HERE+3, HERE+2 ).NE.ZERO )
     $            NBNEXT = 2
            END IF
            CALL DTGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                   LDZ, HERE+1, 1, NBNEXT, WORK, LWORK, INFO )
            IF( INFO.NE.0 ) THEN
               ILST = HERE
               RETURN
            END IF
            IF( NBNEXT.EQ.1 ) THEN
*
*              Swap two 1-by-1 blocks.
*
               CALL DTGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                      LDZ, HERE, 1, 1, WORK, LWORK, INFO )
               IF( INFO.NE.0 ) THEN
                  ILST = HERE
                  RETURN
               END IF
               HERE = HERE + 1
*
            ELSE
*
*              Recompute NBNEXT in case of 2-by-2 split.
*
               IF( A( HERE+2, HERE+1 ).EQ.ZERO )
     $            NBNEXT = 1
               IF( NBNEXT.EQ.2 ) THEN
*
*                 2-by-2 block did not split.
*
                  CALL DTGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ,
     $                         Z, LDZ, HERE, 1, NBNEXT, WORK, LWORK,
     $                         INFO )
                  IF( INFO.NE.0 ) THEN
                     ILST = HERE
                     RETURN
                  END IF
                  HERE = HERE + 2
               ELSE
```

```
*
*                2-by-2 block did split.
*
                 CALL DTGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ,
     $                        Z, LDZ, HERE, 1, 1, WORK, LWORK, INFO )
                 IF( INFO.NE.0 ) THEN
                    ILST = HERE
                    RETURN
                 END IF
                 HERE = HERE + 1
                 CALL DTGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ,
     $                        Z, LDZ, HERE, 1, 1, WORK, LWORK, INFO )
                 IF( INFO.NE.0 ) THEN
                    ILST = HERE
                    RETURN
                 END IF
                 HERE = HERE + 1
              END IF
*
           END IF
        END IF
        IF( HERE.LT.ILST )
     $      GO TO 10
      ELSE
        HERE = IFST
*
   20   CONTINUE
*
*       Swap with next one below.
*
        IF( NBF.EQ.1 .OR. NBF.EQ.2 ) THEN
*
*          Current block either 1-by-1 or 2-by-2.
*
           NBNEXT = 1
           IF( HERE.GE.3 ) THEN
              IF( A( HERE-1, HERE-2 ).NE.ZERO )
     $           NBNEXT = 2
           END IF
           CALL DTGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                  LDZ, HERE-NBNEXT, NBNEXT, NBF, WORK, LWORK,
     $                  INFO )
           IF( INFO.NE.0 ) THEN
              ILST = HERE
              RETURN
           END IF
           HERE = HERE - NBNEXT
*
*          Test if 2-by-2 block breaks into two 1-by-1 blocks.
*
```

```
            IF( NBF.EQ.2 ) THEN
               IF( A( HERE+1, HERE ).EQ.ZERO )
     $            NBF = 3
            END IF
*
         ELSE
*
*           Current block consists of two 1-by-1 blocks, each of which
*           must be swapped individually.
*
            NBNEXT = 1
            IF( HERE.GE.3 ) THEN
               IF( A( HERE-1, HERE-2 ).NE.ZERO )
     $            NBNEXT = 2
            END IF
            CALL DTGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                   LDZ, HERE-NBNEXT, NBNEXT, 1, WORK, LWORK,
     $                   INFO )
            IF( INFO.NE.0 ) THEN
               ILST = HERE
               RETURN
            END IF
            IF( NBNEXT.EQ.1 ) THEN
*
*              Swap two 1-by-1 blocks.
*
               CALL DTGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                      LDZ, HERE, NBNEXT, 1, WORK, LWORK, INFO )
               IF( INFO.NE.0 ) THEN
                  ILST = HERE
                  RETURN
               END IF
               HERE = HERE - 1
            ELSE
*
*             Recompute NBNEXT in case of 2-by-2 split.
*
               IF( A( HERE, HERE-1 ).EQ.ZERO )
     $            NBNEXT = 1
               IF( NBNEXT.EQ.2 ) THEN
*
*                 2-by-2 block did not split.
*
                  CALL DTGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ,
     $                         Z, LDZ, HERE-1, 2, 1, WORK, LWORK, INFO )
                  IF( INFO.NE.0 ) THEN
                     ILST = HERE
                     RETURN
                  END IF
                  HERE = HERE - 2
```

```
               ELSE
*
*                 2-by-2 block did split.
*
                  CALL DTGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ,
     $                         Z, LDZ, HERE, 1, 1, WORK, LWORK, INFO )
                  IF( INFO.NE.0 ) THEN
                     ILST = HERE
                     RETURN
                  END IF
                  HERE = HERE - 1
                  CALL DTGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ,
     $                         Z, LDZ, HERE, 1, 1, WORK, LWORK, INFO )
                  IF( INFO.NE.0 ) THEN
                     ILST = HERE
                     RETURN
                  END IF
                  HERE = HERE - 1
               END IF
            END IF
         END IF
         IF( HERE.GT.ILST )
     $      GO TO 20
      END IF
      ILST = HERE
      WORK( 1 ) = LWMIN
      RETURN
*
*     End of DTGEXC
*
      END
```

———-

32.40.4 lisp code

— dtgexc.lisp —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun dtgexc
         (wantq wantz n a lda b ldb$ q ldq z ldz ifst ilst work lwork info)
    (declare (type (array double-float (*)) work z q b a)
             (type (f2cl-lib:integer4) info lwork ilst ifst ldz ldq ldb$ lda n)
             (type f2cl-lib:logical wantz wantq))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
```

```
    (b double-float b-%data% b-%offset%)
    (q double-float q-%data% q-%offset%)
    (z double-float z-%data% z-%offset%)
    (work double-float work-%data% work-%offset%))
  (prog ((here 0) (lwmin 0) (nbf 0) (nbl 0) (nbnext 0) (lquery nil))
    (declare (type (f2cl-lib:integer4) here lwmin nbf nbl nbnext)
             (type f2cl-lib:logical lquery))
    (setf info 0)
    (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
    (cond ((< n 0) (setf info -3))
          ((< lda
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
           (setf info -5))
          ((< ldb$
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
           (setf info -7))
          ((or (< ldq 1)
               (and wantq
                    (< ldq
                       (max (the f2cl-lib:integer4 1)
                            (the f2cl-lib:integer4 n)))))
           (setf info -9))
          ((or (< ldz 1)
               (and wantz
                    (< ldz
                       (max (the f2cl-lib:integer4 1)
                            (the f2cl-lib:integer4 n)))))
           (setf info -11))
          ((or (< ifst 1) (> ifst n)) (setf info -12))
          ((or (< ilst 1) (> ilst n)) (setf info -13)))
    (cond
     ((= info 0)
      (cond ((<= n 1) (setf lwmin 1))
            (t (setf lwmin (f2cl-lib:int-add (f2cl-lib:int-mul 4 n) 16))))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 lwmin) 'double-float))
      (cond ((and (< lwork lwmin) (not lquery)) (setf info -15)))))
    (cond
     ((/= info 0) (xerbla "DTGEXC" (f2cl-lib:int-sub info)) (go end_label))
     (lquery (go end_label)))
    (if (<= n 1)
        (go end_label))
    (cond
     ((> ifst 1)
      (if (/=
           (f2cl-lib:fref a-%data% (ifst (f2cl-lib:int-sub ifst 1))
                          ((1 lda) (1 *)) a-%offset%)
           zero)
          (setf ifst (f2cl-lib:int-sub ifst 1)))))
    (setf nbf 1)
```

```
(cond
 ((< ifst n)
  (if (/=
       (f2cl-lib:fref a-%data% ((f2cl-lib:int-add ifst 1) ifst)
                      ((1 lda) (1 *)) a-%offset%)
       zero)
      (setf nbf 2))))
(cond
 ((> ilst 1)
  (if (/=
       (f2cl-lib:fref a-%data% (ilst (f2cl-lib:int-sub ilst 1))
                      ((1 lda) (1 *)) a-%offset%)
       zero)
      (setf ilst (f2cl-lib:int-sub ilst 1)))))
(setf nbl 1)
(cond
 ((< ilst n)
  (if (/=
       (f2cl-lib:fref a-%data% ((f2cl-lib:int-add ilst 1) ilst)
                      ((1 lda) (1 *)) a-%offset%)
       zero)
      (setf nbl 2))))
(if (= ifst ilst)
    (go end_label))
(cond
 ((< ifst ilst)
  (tagbody
    (if (and (= nbf 2) (= nbl 1))
        (setf ilst (f2cl-lib:int-sub ilst 1)))
    (if (and (= nbf 1) (= nbl 2))
        (setf ilst (f2cl-lib:int-add ilst 1)))
    (setf here ifst)
   label10
    (cond
     ((or (= nbf 1) (= nbf 2)) (setf nbnext 1)
      (cond
       ((<= (f2cl-lib:int-add here nbf 1) n)
        (if (/=
             (f2cl-lib:fref a-%data%
                            ((f2cl-lib:int-add here nbf 1)
                             (f2cl-lib:int-add here nbf))
                            ((1 lda) (1 *)) a-%offset%)
             zero)
            (setf nbnext 2))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13 var-14 var-15 var-16)
          (dtgex2 wantq wantz n a lda b ldb$ q ldq z ldz here nbf
           nbnext work lwork info)
        (declare
```

```
     (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-14
      var-15))
    (setf n var-2)
    (setf lda var-4)
    (setf ldb$ var-6)
    (setf ldq var-8)
    (setf ldz var-10)
    (setf nbf var-12)
    (setf nbnext var-13)
    (setf info var-16))
  (cond ((/= info 0) (setf ilst here) (go end_label)))
  (setf here (f2cl-lib:int-add here nbnext))
  (cond
   ((= nbf 2)
    (if (=
         (f2cl-lib:fref a-%data% ((f2cl-lib:int-add here 1) here)
                        ((1 lda) (1 *)) a-%offset%)
         zero)
        (setf nbf 3)))))
 (t (setf nbnext 1)
  (cond
   ((<= (f2cl-lib:int-add here 3) n)
    (if (/=
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-add here 3)
                         (f2cl-lib:int-add here 2))
                        ((1 lda) (1 *)) a-%offset%)
         zero)
        (setf nbnext 2))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16)
      (dtgex2 wantq wantz n a lda b ldb$ q ldq z ldz
       (f2cl-lib:int-add here 1) 1 nbnext work lwork info)
    (declare
     (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-12
      var-14 var-15))
    (setf n var-2)
    (setf lda var-4)
    (setf ldb$ var-6)
    (setf ldq var-8)
    (setf ldz var-10)
    (setf nbnext var-13)
    (setf info var-16))
  (cond ((/= info 0) (setf ilst here) (go end_label)))
  (cond
   ((= nbnext 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16)
```

```
      (dtgex2 wantq wantz n a lda b ldb$ q ldq z ldz here 1 1
       work lwork info)
    (declare
     (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-12
      var-13 var-14 var-15))
    (setf n var-2)
    (setf lda var-4)
    (setf ldb$ var-6)
    (setf ldq var-8)
    (setf ldz var-10)
    (setf info var-16))
  (cond ((/= info 0) (setf ilst here) (go end_label)))
  (setf here (f2cl-lib:int-add here 1)))
 (t
  (if (=
       (f2cl-lib:fref a-%data%
                      ((f2cl-lib:int-add here 2)
                       (f2cl-lib:int-add here 1))
                      ((1 lda) (1 *)) a-%offset%)
       zero)
      (setf nbnext 1))
  (cond
   ((= nbnext 2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16)
        (dtgex2 wantq wantz n a lda b ldb$ q ldq z ldz here 1
         nbnext work lwork info)
      (declare
       (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-12
        var-14 var-15))
      (setf n var-2)
      (setf lda var-4)
      (setf ldb$ var-6)
      (setf ldq var-8)
      (setf ldz var-10)
      (setf nbnext var-13)
      (setf info var-16))
    (cond ((/= info 0) (setf ilst here) (go end_label)))
    (setf here (f2cl-lib:int-add here 2)))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16)
        (dtgex2 wantq wantz n a lda b ldb$ q ldq z ldz here 1 1
         work lwork info)
      (declare
       (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-12
        var-13 var-14 var-15))
      (setf n var-2)
```

```
                (setf lda var-4)
                (setf ldb$ var-6)
                (setf ldq var-8)
                (setf ldz var-10)
                (setf info var-16))
              (cond ((/= info 0) (setf ilst here) (go end_label)))
              (setf here (f2cl-lib:int-add here 1))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                   var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16)
                  (dtgex2 wantq wantz n a lda b ldb$ q ldq z ldz here 1 1
                   work lwork info)
                (declare
                 (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-12
                  var-13 var-14 var-15))
                (setf n var-2)
                (setf lda var-4)
                (setf ldb$ var-6)
                (setf ldq var-8)
                (setf ldz var-10)
                (setf info var-16))
              (cond ((/= info 0) (setf ilst here) (go end_label)))
              (setf here (f2cl-lib:int-add here 1))))))))
    (if (< here ilst)
        (go label10))))
(t
 (tagbody
   (setf here ifst)
  label20
   (cond
    ((or (= nbf 1) (= nbf 2)) (setf nbnext 1)
     (cond
      ((>= here 3)
       (if (/=
            (f2cl-lib:fref a-%data%
                           ((f2cl-lib:int-sub here 1)
                            (f2cl-lib:int-sub here 2))
                           ((1 lda) (1 *)) a-%offset%)
            zero)
           (setf nbnext 2))))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12 var-13 var-14 var-15 var-16)
         (dtgex2 wantq wantz n a lda b ldb$ q ldq z ldz
          (f2cl-lib:int-sub here nbnext) nbnext nbf work lwork info)
       (declare
        (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-14
         var-15))
       (setf n var-2)
       (setf lda var-4)
```

```
    (setf ldb$ var-6)
    (setf ldq var-8)
    (setf ldz var-10)
    (setf nbnext var-12)
    (setf nbf var-13)
    (setf info var-16))
  (cond ((/= info 0) (setf ilst here) (go end_label)))
  (setf here (f2cl-lib:int-sub here nbnext))
  (cond
   ((= nbf 2)
    (if (=
         (f2cl-lib:fref a-%data% ((f2cl-lib:int-add here 1) here)
                        ((1 lda) (1 *)) a-%offset%)
         zero)
        (setf nbf 3)))))
 (t (setf nbnext 1)
  (cond
   ((>= here 3)
    (if (/=
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-sub here 1)
                         (f2cl-lib:int-sub here 2))
                        ((1 lda) (1 *)) a-%offset%)
         zero)
        (setf nbnext 2))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16)
      (dtgex2 wantq wantz n a lda b ldb$ q ldq z ldz
       (f2cl-lib:int-sub here nbnext) nbnext 1 work lwork info)
    (declare
     (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-13
      var-14 var-15))
    (setf n var-2)
    (setf lda var-4)
    (setf ldb$ var-6)
    (setf ldq var-8)
    (setf ldz var-10)
    (setf nbnext var-12)
    (setf info var-16))
  (cond ((/= info 0) (setf ilst here) (go end_label)))
  (cond
   ((= nbnext 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16)
        (dtgex2 wantq wantz n a lda b ldb$ q ldq z ldz here nbnext
         1 work lwork info)
      (declare
       (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-13
```

```
      var-14 var-15))
    (setf n var-2)
    (setf lda var-4)
    (setf ldb$ var-6)
    (setf ldq var-8)
    (setf ldz var-10)
    (setf nbnext var-12)
    (setf info var-16))
  (cond ((/= info 0) (setf ilst here) (go end_label)))
  (setf here (f2cl-lib:int-sub here 1)))
 (t
  (if (=
       (f2cl-lib:fref a-%data% (here (f2cl-lib:int-sub here 1))
                      ((1 lda) (1 *)) a-%offset%)
       zero)
      (setf nbnext 1))
  (cond
   ((= nbnext 2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16)
        (dtgex2 wantq wantz n a lda b ldb$ q ldq z ldz
         (f2cl-lib:int-sub here 1) 2 1 work lwork info)
      (declare
       (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-12
        var-13 var-14 var-15))
      (setf n var-2)
      (setf lda var-4)
      (setf ldb$ var-6)
      (setf ldq var-8)
      (setf ldz var-10)
      (setf info var-16))
    (cond ((/= info 0) (setf ilst here) (go end_label)))
    (setf here (f2cl-lib:int-sub here 2)))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16)
        (dtgex2 wantq wantz n a lda b ldb$ q ldq z ldz here 1 1
         work lwork info)
      (declare
       (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-12
        var-13 var-14 var-15))
      (setf n var-2)
      (setf lda var-4)
      (setf ldb$ var-6)
      (setf ldq var-8)
      (setf ldz var-10)
      (setf info var-16))
    (cond ((/= info 0) (setf ilst here) (go end_label)))
```

```
                  (setf here (f2cl-lib:int-sub here 1))
                  (multiple-value-bind
                      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                       var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16)
                      (dtgex2 wantq wantz n a lda b ldb$ q ldq z ldz here 1 1
                       work lwork info)
                    (declare
                     (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-12
                      var-13 var-14 var-15))
                    (setf n var-2)
                    (setf lda var-4)
                    (setf ldb$ var-6)
                    (setf ldq var-8)
                    (setf ldz var-10)
                    (setf info var-16))
                  (cond ((/= info 0) (setf ilst here) (go end_label)))
                  (setf here (f2cl-lib:int-sub here 1))))))))
              (if (> here ilst)
                  (go label20)))))
        (setf ilst here)
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwmin) 'double-float))
        (go end_label)
       end_label
        (return
         (values nil nil n nil lda nil ldb$ nil ldq nil ldz ifst ilst nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dtgexc
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '(fortran-to-lisp::logical
                                              fortran-to-lisp::logical
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
```

```
:return-values
'(nil nil fortran-to-lisp::n nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldq nil
  fortran-to-lisp::ldz
  fortran-to-lisp::ifst
  fortran-to-lisp::ilst nil nil
  fortran-to-lisp::info)
:calls '(fortran-to-lisp::dtgex2))))
```

———-

32.41 zgesvj

32.41.1 Axiom unit tests

— zgesvj.input —

```
)set break resume
)sys rm -f zgesvj.output
)spool zgesvj.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

32.41.2 Axiom help page

— zgesvj.help —

```
==========================================================================
zgesvj examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine zgesvj (JOBA, JOBU, JOBV, M, N, A, LDA, SVA, MV, V, LDV,
           CWORK, LWORK, RWORK, LRWORK, INFO)
           ZGESVJ

Function/Subroutine Documentation
   subroutine zgesvj (character*1 JOBA, character*1 JOBU, character*1 JOBV,
       integer M, integer N, complex*16, dimension( lda, * ) A, integer LDA,
       double precision, dimension( n ) SVA, integer MV, complex*16,
       dimension( ldv, * ) V, integer LDV, complex*16, dimension( lwork )
       CWORK, integer LWORK, double precision, dimension( lrwork ) RWORK,
       integer LRWORK, integer INFO)
       ZGESVJ

Purpose:
   ZGESVJ computes the singular value decomposition (SVD) of a complex
   M-by-N matrix A, where M >= N. The SVD of A is written as
                                [++]   [xx]   [x0]   [xx]
           A = U * SIGMA * V^*,  [++] = [xx] * [ox] * [xx]
                                [++]   [xx]
   where SIGMA is an N-by-N diagonal matrix, U is an M-by-N orthonormal
   matrix, and V is an N-by-N unitary matrix. The diagonal elements
   of SIGMA are the singular values of A. The columns of U and V are the
   left and the right singular vectors of A, respectively.

Parameters:

 JOBA
   JOBA is CHARACTER* 1
   Specifies the structure of A.
   = 'L': The input matrix A is lower triangular;
   = 'U': The input matrix A is upper triangular;
   = 'G': The input matrix A is general M-by-N matrix, M >= N.

 JOBU
   JOBU is CHARACTER*1
   Specifies whether to compute the left singular vectors
   (columns of U):
   = 'U': The left singular vectors corresponding to the nonzero
          singular values are computed and returned in the leading
          columns of A. See more details in the description of A.
          The default numerical orthogonality threshold is set to
          approximately TOL=CTOL*EPS, CTOL=DSQRT(M), EPS=DLAMCH('E').
   = 'C': Analogous to JOBU='U', except that user can control the
          level of numerical orthogonality of the computed left
          singular vectors. TOL can be set to TOL = CTOL*EPS, where
          CTOL is given on input in the array WORK.
          No CTOL smaller than ONE is allowed. CTOL greater
```

```
         than 1 / EPS is meaningless. The option 'C'
         can be used if M*EPS is satisfactory orthogonality
         of the computed left singular vectors, so CTOL=M could
         save few sweeps of Jacobi rotations.
         See the descriptions of A and WORK(1).
  = 'N': The matrix U is not computed. However, see the
         description of A.

JOBV
  JOBV is CHARACTER*1
  Specifies whether to compute the right singular vectors, that
  is, the matrix V:
  = 'V' : the matrix V is computed and returned in the array V
  = 'A' : the Jacobi rotations are applied to the MV-by-N
          array V. In other words, the right singular vector
          matrix V is not computed explicitly, instead it is
          applied to an MV-by-N matrix initially stored in the
          first MV rows of V.
  = 'N' : the matrix V is not computed and the array V is not referenced

M
  M is INTEGER
  The number of rows of the input matrix A. 1/DLAMCH('E') > M >= 0.

N
  N is INTEGER
  The number of columns of the input matrix A.
  M >= N >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit,
  If JOBU .EQ. 'U' .OR. JOBU .EQ. 'C':
    If INFO .EQ. 0 :
      RANKA orthonormal columns of U are returned in the
      leading RANKA columns of the array A. Here RANKA <= N
      is the number of computed singular values of A that are
      above the underflow threshold DLAMCH('S'). The singular
      vectors corresponding to underflowed or zero singular
      values are not computed. The value of RANKA is returned
      in the array RWORK as RANKA=NINT(RWORK(2)). Also see the
      descriptions of SVA and RWORK. The computed columns of U
      are mutually numerically orthogonal up to approximately
      TOL=SQRT(M)*EPS (default); or TOL=CTOL*EPS (JOBU.EQ.'C'),
      see the description of JOBU.
    If INFO .GT. 0,
      the procedure ZGESVJ did not converge in the given number
      of iterations (sweeps). In that case, the computed
      columns of U may not be orthogonal up to TOL. The output
```

```
      U (stored in A), SIGMA (given by the computed singular
      values in SVA(1:N)) and V is still a decomposition of the
      input matrix A in the sense that the residual
      || A - SCALE * U * SIGMA * V^* ||_2 / ||A||_2 is small.
  If JOBU .EQ. 'N':
    If INFO .EQ. 0 :
      Note that the left singular vectors are 'for free' in the
      one-sided Jacobi SVD algorithm. However, if only the
      singular values are needed, the level of numerical
      orthogonality of U is not an issue and iterations are
      stopped when the columns of the iterated matrix are
      numerically orthogonal up to approximately M*EPS. Thus,
      on exit, A contains the columns of U scaled with the
      corresponding singular values.
    If INFO .GT. 0 :
      the procedure ZGESVJ did not converge in the given number
      of iterations (sweeps).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

SVA
  SVA is DOUBLE PRECISION array, dimension (N)
  On exit,
  If INFO .EQ. 0 :
    depending on the value SCALE = RWORK(1), we have:
      If SCALE .EQ. ONE:
        SVA(1:N) contains the computed singular values of A.
        During the computation SVA contains the Euclidean column
        norms of the iterated matrices in the array A.
      If SCALE .NE. ONE:
        The singular values of A are SCALE*SVA(1:N), and this
        factored representation is due to the fact that some of the
        singular values of A might underflow or overflow.

  If INFO .GT. 0 :
    the procedure ZGESVJ did not converge in the given number of
    iterations (sweeps) and SCALE*SVA(1:N) may not be accurate.

MV
  MV is INTEGER
  If JOBV .EQ. 'A', then the product of Jacobi rotations in ZGESVJ
  is applied to the first MV rows of V. See the description of JOBV.

V
  V is COMPLEX*16 array, dimension (LDV,N)
  If JOBV = 'V', then V contains on exit the N-by-N matrix of
     the right singular vectors;
  If JOBV = 'A', then V contains the product of the computed right
```

```
     singular vector matrix and the initial matrix in the array V.
  If JOBV = 'N', then V is not referenced.

LDV
  LDV is INTEGER
  The leading dimension of the array V, LDV .GE. 1.
  If JOBV .EQ. 'V', then LDV .GE. max(1,N).
  If JOBV .EQ. 'A', then LDV .GE. max(1,MV) .

CWORK
  CWORK is COMPLEX*16 array, dimension M+N.
  Used as work space.

LWORK
  LWORK is INTEGER.
  Length of CWORK, LWORK >= M+N.

RWORK
  RWORK is DOUBLE PRECISION array, dimension max(6,M+N).
  On entry,
  If JOBU .EQ. 'C' :
    RWORK(1) = CTOL, where CTOL defines the threshold for convergence.
      The process stops if all columns of A are mutually
      orthogonal up to CTOL*EPS, EPS=DLAMCH('E').
      It is required that CTOL >= ONE, i.e. it is not
      allowed to force the routine to obtain orthogonality below EPSILON.
  On exit,
    RWORK(1) = SCALE is the scaling factor such that SCALE*SVA(1:N)
      are the computed singular values of A. (See description of SVA().)
    RWORK(2) = NINT(RWORK(2)) is the number of the computed nonzero
      singular values.
    RWORK(3) = NINT(RWORK(3)) is the number of the computed singular
      values that are larger than the underflow threshold.
    RWORK(4) = NINT(RWORK(4)) is the number of sweeps of Jacobi
      rotations needed for numerical convergence.
    RWORK(5) = max_{i.NE.j} |COS(A(:,i),A(:,j))| in the last sweep.
      This is useful information in cases when ZGESVJ did
      not converge, as it can be used to estimate whether
      the output is stil useful and for post festum analysis.
    RWORK(6) = the largest absolute value over all sines of the
      Jacobi rotation angles in the last sweep. It can be
      useful for a post festum analysis.

LRWORK
  LRWORK is INTEGER
  Length of RWORK, LRWORK >= MAX(6,N).

INFO
  INFO is INTEGER
  = 0 : successful exit.
```

```
   < 0 : if INFO = -i, then the i-th argument had an illegal value
   > 0 : ZGESVJ did not converge in the maximal allowed number
         (NSWEEP=30) of sweeps. The output may still be useful.
         See the description of RWORK.

Further Details:

   The orthogonal N-by-N matrix V is obtained as a product of Jacobi plane
   rotations. In the case of underflow of the tangent of the Jacobi angle, a
   modified Jacobi transformation of Drmac [3] is used. Pivot strategy uses
   column interchanges of de Rijk [1]. The relative accuracy of the computed
   singular values and the accuracy of the computed singular vectors (in
   angle metric) is as guaranteed by the theory of Demmel and Veselic [2].
   The condition number that determines the accuracy in the full rank case
   is essentially min_{D=diag} kappa(A*D), where kappa(.) is the
   spectral condition number. The best performance of this Jacobi SVD
   procedure is achieved if used in an  accelerated version of Drmac and
   Veselic [4,5], and it is the kernel routine in the SIGMA library [6].
   Some tunning parameters (marked with [TP]) are available for the
   implementer.
   The computational range for the nonzero singular values is the  machine
   number interval ( UNDERFLOW , OVERFLOW ). In extreme cases, even
   denormalized singular values can be computed with the corresponding
   gradual loss of accurate digits.
```

———-

32.41.3 fortran code

— **zgesvj.f** —

```
*  ===================================================================
      SUBROUTINE ZGESVJ( JOBA, JOBU, JOBV, M, N, A, LDA, SVA, MV, V,
     $                   LDV, CWORK, LWORK, RWORK, LRWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
      IMPLICIT NONE
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDV, LWORK, LRWORK, M, MV, N
      CHARACTER*1        JOBA, JOBU, JOBV
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ),  V( LDV, * ), CWORK( LWORK )
```

```
      DOUBLE PRECISION   RWORK( LRWORK ), SVA( N )
*     ..
*
*  ====================================================================
*
*     .. Local Parameters ..
      DOUBLE PRECISION   ZERO,         HALF,         ONE
      PARAMETER  ( ZERO = 0.0D0, HALF = 0.5D0, ONE = 1.0D0)
      COMPLEX*16      CZERO,                  CONE
      PARAMETER  ( CZERO = (0.0D0, 0.0D0), CONE = (1.0D0, 0.0D0) )
      INTEGER      NSWEEP
      PARAMETER  ( NSWEEP = 30 )
*     ..
*     .. Local Scalars ..
      COMPLEX*16 AAPQ, OMPQ
      DOUBLE PRECISION    AAPP, AAPP0, AAPQ1, AAQQ, APOAQ, AQOAP, BIG,
     $        BIGTHETA, CS, CTOL, EPSLN, LARGE, MXAAPQ,
     $        MXSINJ, ROOTBIG, ROOTEPS, ROOTSFMIN, ROOTTOL,
     $        SKL, SFMIN, SMALL, SN, T, TEMP1, THETA, THSIGN, TOL
      INTEGER BLSKIP, EMPTSW, i, ibr, IERR, igl, IJBLSK, ir1,
     $        ISWROT, jbc, jgl, KBL, LKAHEAD, MVL, N2, N34,
     $        N4, NBL, NOTROT, p, PSKIPPED, q, ROWSKIP, SWBAND
      LOGICAL APPLV, GOSCALE, LOWER, LSVEC, NOSCALE, ROTOK,
     $        RSVEC, UCTOL, UPPER
*     ..
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC ABS, DMAX1, DMIN1, DCONJG, DFLOAT, MIN0, MAX0,
     $          DSIGN, DSQRT
*     ..
*     .. External Functions ..
*     ..
*     from BLAS
      DOUBLE PRECISION   DZNRM2
      COMPLEX*16         ZDOTC
      EXTERNAL           ZDOTC, DZNRM2
      INTEGER            IDAMAX
      EXTERNAL           IDAMAX
*     from LAPACK
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
*     ..
*     from BLAS
      EXTERNAL           ZCOPY, ZROT, ZDSCAL, ZSWAP
*     from LAPACK
      EXTERNAL           ZLASCL, ZLASET, ZLASSQ, XERBLA
```

```
      EXTERNAL           ZGSVJ0, ZGSVJ1
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      LSVEC = LSAME( JOBU, 'U' )
      UCTOL = LSAME( JOBU, 'C' )
      RSVEC = LSAME( JOBV, 'V' )
      APPLV = LSAME( JOBV, 'A' )
      UPPER = LSAME( JOBA, 'U' )
      LOWER = LSAME( JOBA, 'L' )
*
      IF( .NOT.( UPPER .OR. LOWER .OR. LSAME( JOBA, 'G' ) ) ) THEN
         INFO = -1
      ELSE IF( .NOT.( LSVEC .OR. UCTOL .OR. LSAME( JOBU, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( .NOT.( RSVEC .OR. APPLV .OR. LSAME( JOBV, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( M.LT.0 ) THEN
         INFO = -4
      ELSE IF( ( N.LT.0 ) .OR. ( N.GT.M ) ) THEN
         INFO = -5
      ELSE IF( LDA.LT.M ) THEN
         INFO = -7
      ELSE IF( MV.LT.0 ) THEN
         INFO = -9
      ELSE IF( ( RSVEC .AND. ( LDV.LT.N ) ) .OR.
     $          ( APPLV .AND. ( LDV.LT.MV ) ) ) THEN
         INFO = -11
      ELSE IF( UCTOL .AND. ( RWORK( 1 ).LE.ONE ) ) THEN
         INFO = -12
      ELSE IF( LWORK.LT.( M+N ) ) THEN
         INFO = -13
      ELSE IF( LRWORK.LT.MAX0( N, 6 ) ) THEN
         INFO = -15
      ELSE
         INFO = 0
      END IF
*
*     #:(
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGESVJ', -INFO )
         RETURN
      END IF
*
* #:) Quick return for void matrix
*
      IF( ( M.EQ.0 ) .OR. ( N.EQ.0 ) )RETURN
*
```

```
*     Set numerical parameters
*     The stopping criterion for Jacobi rotations is
*
*     max_{i<>j}|A(:,i)^* * A(:,j)| / (||A(:,i)||*||A(:,j)||) < CTOL*EPS
*
*     where EPS is the round-off and CTOL is defined as follows:
*
      IF( UCTOL ) THEN
*        ... user controlled
         CTOL = RWORK( 1 )
      ELSE
*        ... default
         IF( LSVEC .OR. RSVEC .OR. APPLV ) THEN
            CTOL = DSQRT( DFLOAT( M ) )
         ELSE
            CTOL = DFLOAT( M )
         END IF
      END IF
*     ... and the machine dependent parameters are
*[!]  (Make sure that DLAMCH() works properly on the target machine.)
*
      EPSLN = DLAMCH( 'Epsilon' )
      ROOTEPS = DSQRT( EPSLN )
      SFMIN = DLAMCH( 'SafeMinimum' )
      ROOTSFMIN = DSQRT( SFMIN )
      SMALL = SFMIN / EPSLN
      BIG = DLAMCH( 'Overflow' )
*     BIG         = ONE    / SFMIN
      ROOTBIG = ONE / ROOTSFMIN
      LARGE = BIG / DSQRT( DFLOAT( M*N ) )
      BIGTHETA = ONE / ROOTEPS
*
      TOL = CTOL*EPSLN
      ROOTTOL = DSQRT( TOL )
*
      IF( DFLOAT( M )*EPSLN.GE.ONE ) THEN
         INFO = -4
         CALL XERBLA( 'ZGESVJ', -INFO )
         RETURN
      END IF
*
*     Initialize the right singular vector matrix.
*
      IF( RSVEC ) THEN
         MVL = N
         CALL ZLASET( 'A', MVL, N, CZERO, CONE, V, LDV )
      ELSE IF( APPLV ) THEN
         MVL = MV
      END IF
      RSVEC = RSVEC .OR. APPLV
```

```
*
*     Initialize SVA( 1:N ) = ( ||A e_i||_2, i = 1:N )
*(!)  If necessary, scale A to protect the largest singular value
*     from overflow. It is possible that saving the largest singular
*     value destroys the information about the small ones.
*     This initial scaling is almost minimal in the sense that the
*     goal is to make sure that no column norm overflows, and that
*     SQRT(N)*max_i SVA(i) does not overflow. If INFinite entries
*     in A are detected, the procedure returns with INFO=-6.
*
      SKL = ONE / DSQRT( DFLOAT( M )*DFLOAT( N ) )
      NOSCALE = .TRUE.
      GOSCALE = .TRUE.
*
      IF( LOWER ) THEN
*        the input matrix is M-by-N lower triangular (trapezoidal)
         DO 1874 p = 1, N
            AAPP = ZERO
            AAQQ = ONE
            CALL ZLASSQ( M-p+1, A( p, p ), 1, AAPP, AAQQ )
            IF( AAPP.GT.BIG ) THEN
               INFO = -6
               CALL XERBLA( 'ZGESVJ', -INFO )
               RETURN
            END IF
            AAQQ = DSQRT( AAQQ )
            IF( ( AAPP.LT.( BIG / AAQQ ) ) .AND. NOSCALE ) THEN
               SVA( p ) = AAPP*AAQQ
            ELSE
               NOSCALE = .FALSE.
               SVA( p ) = AAPP*( AAQQ*SKL )
               IF( GOSCALE ) THEN
                  GOSCALE = .FALSE.
                  DO 1873 q = 1, p - 1
                     SVA( q ) = SVA( q )*SKL
 1873             CONTINUE
               END IF
            END IF
 1874    CONTINUE
      ELSE IF( UPPER ) THEN
*        the input matrix is M-by-N upper triangular (trapezoidal)
         DO 2874 p = 1, N
            AAPP = ZERO
            AAQQ = ONE
            CALL ZLASSQ( p, A( 1, p ), 1, AAPP, AAQQ )
            IF( AAPP.GT.BIG ) THEN
               INFO = -6
               CALL XERBLA( 'ZGESVJ', -INFO )
               RETURN
            END IF
```

```
            AAQQ = DSQRT( AAQQ )
            IF( ( AAPP.LT.( BIG / AAQQ ) ) .AND. NOSCALE ) THEN
               SVA( p ) = AAPP*AAQQ
            ELSE
               NOSCALE = .FALSE.
               SVA( p ) = AAPP*( AAQQ*SKL )
               IF( GOSCALE ) THEN
                  GOSCALE = .FALSE.
                  DO 2873 q = 1, p - 1
                     SVA( q ) = SVA( q )*SKL
 2873             CONTINUE
               END IF
            END IF
 2874    CONTINUE
      ELSE
*        the input matrix is M-by-N general dense
         DO 3874 p = 1, N
            AAPP = ZERO
            AAQQ = ONE
            CALL ZLASSQ( M, A( 1, p ), 1, AAPP, AAQQ )
            IF( AAPP.GT.BIG ) THEN
               INFO = -6
               CALL XERBLA( 'ZGESVJ', -INFO )
               RETURN
            END IF
            AAQQ = DSQRT( AAQQ )
            IF( ( AAPP.LT.( BIG / AAQQ ) ) .AND. NOSCALE ) THEN
               SVA( p ) = AAPP*AAQQ
            ELSE
               NOSCALE = .FALSE.
               SVA( p ) = AAPP*( AAQQ*SKL )
               IF( GOSCALE ) THEN
                  GOSCALE = .FALSE.
                  DO 3873 q = 1, p - 1
                     SVA( q ) = SVA( q )*SKL
 3873             CONTINUE
               END IF
            END IF
 3874    CONTINUE
      END IF
*
      IF( NOSCALE )SKL = ONE
*
*     Move the smaller part of the spectrum from the underflow threshold
*(!)  Start by determining the position of the nonzero entries of the
*     array SVA() relative to ( SFMIN, BIG ).
*
      AAPP = ZERO
      AAQQ = BIG
      DO 4781 p = 1, N
```

```
         IF( SVA( p ).NE.ZERO )AAQQ = DMIN1( AAQQ, SVA( p ) )
         AAPP = DMAX1( AAPP, SVA( p ) )
 4781 CONTINUE
*
* #:) Quick return for zero matrix
*
      IF( AAPP.EQ.ZERO ) THEN
         IF( LSVEC )CALL ZLASET( 'G', M, N, CZERO, CONE, A, LDA )
         RWORK( 1 ) = ONE
         RWORK( 2 ) = ZERO
         RWORK( 3 ) = ZERO
         RWORK( 4 ) = ZERO
         RWORK( 5 ) = ZERO
         RWORK( 6 ) = ZERO
         RETURN
      END IF
*
* #:) Quick return for one-column matrix
*
      IF( N.EQ.1 ) THEN
         IF( LSVEC )CALL ZLASCL( 'G', 0, 0, SVA( 1 ), SKL, M, 1,
     $                           A( 1, 1 ), LDA, IERR )
         RWORK( 1 ) = ONE / SKL
         IF( SVA( 1 ).GE.SFMIN ) THEN
            RWORK( 2 ) = ONE
         ELSE
            RWORK( 2 ) = ZERO
         END IF
         RWORK( 3 ) = ZERO
         RWORK( 4 ) = ZERO
         RWORK( 5 ) = ZERO
         RWORK( 6 ) = ZERO
         RETURN
      END IF
*
*     Protect small singular values from underflow, and try to
*     avoid underflows/overflows in computing Jacobi rotations.
*
      SN = DSQRT( SFMIN / EPSLN )
      TEMP1 = DSQRT( BIG / DFLOAT( N ) )
      IF( ( AAPP.LE.SN ) .OR. ( AAQQ.GE.TEMP1 ) .OR.
     $    ( ( SN.LE.AAQQ ) .AND. ( AAPP.LE.TEMP1 ) ) ) THEN
         TEMP1 = DMIN1( BIG, TEMP1 / AAPP )
*         AAQQ  = AAQQ*TEMP1
*         AAPP  = AAPP*TEMP1
      ELSE IF( ( AAQQ.LE.SN ) .AND. ( AAPP.LE.TEMP1 ) ) THEN
         TEMP1 = DMIN1( SN / AAQQ, BIG / (AAPP*DSQRT( DFLOAT(N)) ) )
*         AAQQ  = AAQQ*TEMP1
*         AAPP  = AAPP*TEMP1
      ELSE IF( ( AAQQ.GE.SN ) .AND. ( AAPP.GE.TEMP1 ) ) THEN
```

```
         TEMP1 = DMAX1( SN / AAQQ, TEMP1 / AAPP )
*         AAQQ  = AAQQ*TEMP1
*         AAPP  = AAPP*TEMP1
      ELSE IF( ( AAQQ.LE.SN ) .AND. ( AAPP.GE.TEMP1 ) ) THEN
         TEMP1 = DMIN1( SN / AAQQ, BIG / ( DSQRT( DFLOAT( N ) )*AAPP ) )
*         AAQQ  = AAQQ*TEMP1
*         AAPP  = AAPP*TEMP1
      ELSE
         TEMP1 = ONE
      END IF
*
*     Scale, if necessary
*
      IF( TEMP1.NE.ONE ) THEN
         CALL ZLASCL( 'G', 0, 0, ONE, TEMP1, N, 1, SVA, N, IERR )
      END IF
      SKL = TEMP1*SKL
      IF( SKL.NE.ONE ) THEN
         CALL ZLASCL( JOBA, 0, 0, ONE, SKL, M, N, A, LDA, IERR )
         SKL = ONE / SKL
      END IF
*
*     Row-cyclic Jacobi SVD algorithm with column pivoting
*
      EMPTSW = ( N*( N-1 ) ) / 2
      NOTROT = 0

      DO 1868 q = 1, N
         CWORK( q ) = CONE
 1868 CONTINUE
*
*
*
      SWBAND = 3
*[TP] SWBAND is a tuning parameter [TP]. It is meaningful and effective
*     if ZGESVJ is used as a computational routine in the preconditioned
*     Jacobi SVD algorithm ZGEJSV. For sweeps i=1:SWBAND the procedure
*     works on pivots inside a band-like region around the diagonal.
*     The boundaries are determined dynamically, based on the number of
*     pivots above a threshold.
*
      KBL = MIN0( 8, N )
*[TP] KBL is a tuning parameter that defines the tile size in the
*     tiling of the p-q loops of pivot pairs. In general, an optimal
*     value of KBL depends on the matrix dimensions and on the
*     parameters of the computer's memory.
*
      NBL = N / KBL
      IF( ( NBL*KBL ).NE.N )NBL = NBL + 1
*
```

```
      BLSKIP = KBL**2
*[TP] BLKSKIP is a tuning parameter that depends on SWBAND and KBL.
*
      ROWSKIP = MIN0( 5, KBL )
*[TP] ROWSKIP is a tuning parameter.
*
      LKAHEAD = 1
*[TP] LKAHEAD is a tuning parameter.
*
*     Quasi block transformations, using the lower (upper) triangular
*     structure of the input matrix. The quasi-block-cycling usually
*     invokes cubic convergence. Big part of this cycle is done inside
*     canonical subspaces of dimensions less than M.
*
      IF( ( LOWER .OR. UPPER ) .AND. ( N.GT.MAX0( 64, 4*KBL ) ) ) THEN
*[TP] The number of partition levels and the actual partition are
*     tuning parameters.
         N4 = N / 4
         N2 = N / 2
         N34 = 3*N4
         IF( APPLV ) THEN
            q = 0
         ELSE
            q = 1
         END IF
*
         IF( LOWER ) THEN
*
*     This works very well on lower triangular matrices, in particular
*     in the framework of the preconditioned Jacobi SVD (xGEJSV).
*     The idea is simple:
*     [+ 0 0 0]   Note that Jacobi transformations of [0 0]
*     [+ + 0 0]                                       [0 0]
*     [+ + x 0]   actually work on [x 0]              [x 0]
*     [+ + x x]                    [x x].             [x x]
*
            CALL ZGSVJ0( JOBV, M-N34, N-N34, A( N34+1, N34+1 ), LDA,
     $                   CWORK( N34+1 ), SVA( N34+1 ), MVL,
     $                   V( N34*q+1, N34+1 ), LDV, EPSLN, SFMIN, TOL,
     $                   2, CWORK( N+1 ), LWORK-N, IERR )

            CALL ZGSVJ0( JOBV, M-N2, N34-N2, A( N2+1, N2+1 ), LDA,
     $                   CWORK( N2+1 ), SVA( N2+1 ), MVL,
     $                   V( N2*q+1, N2+1 ), LDV, EPSLN, SFMIN, TOL, 2,
     $                   CWORK( N+1 ), LWORK-N, IERR )

            CALL ZGSVJ1( JOBV, M-N2, N-N2, N4, A( N2+1, N2+1 ), LDA,
     $                   CWORK( N2+1 ), SVA( N2+1 ), MVL,
     $                   V( N2*q+1, N2+1 ), LDV, EPSLN, SFMIN, TOL, 1,
     $                   CWORK( N+1 ), LWORK-N, IERR )
```

```
            CALL ZGSVJ0( JOBV, M-N4, N2-N4, A( N4+1, N4+1 ), LDA,
     $                   CWORK( N4+1 ), SVA( N4+1 ), MVL,
     $                   V( N4*q+1, N4+1 ), LDV, EPSLN, SFMIN, TOL, 1,
     $                   CWORK( N+1 ), LWORK-N, IERR )
*
            CALL ZGSVJ0( JOBV, M, N4, A, LDA, CWORK, SVA, MVL, V, LDV,
     $                   EPSLN, SFMIN, TOL, 1, CWORK( N+1 ), LWORK-N,
     $                   IERR )
*
            CALL ZGSVJ1( JOBV, M, N2, N4, A, LDA, CWORK, SVA, MVL, V,
     $                   LDV, EPSLN, SFMIN, TOL, 1, CWORK( N+1 ),
     $                   LWORK-N, IERR )
*
*
         ELSE IF( UPPER ) THEN
*
*
            CALL ZGSVJ0( JOBV, N4, N4, A, LDA, CWORK, SVA, MVL, V, LDV,
     $                   EPSLN, SFMIN, TOL, 2, CWORK( N+1 ), LWORK-N,
     $                   IERR )
*
            CALL ZGSVJ0( JOBV, N2, N4, A( 1, N4+1 ), LDA, CWORK( N4+1 ),
     $                   SVA( N4+1 ), MVL, V( N4*q+1, N4+1 ), LDV,
     $                   EPSLN, SFMIN, TOL, 1, CWORK( N+1 ), LWORK-N,
     $                   IERR )
*
            CALL ZGSVJ1( JOBV, N2, N2, N4, A, LDA, CWORK, SVA, MVL, V,
     $                   LDV, EPSLN, SFMIN, TOL, 1, CWORK( N+1 ),
     $                   LWORK-N, IERR )
*
            CALL ZGSVJ0( JOBV, N2+N4, N4, A( 1, N2+1 ), LDA,
     $                   CWORK( N2+1 ), SVA( N2+1 ), MVL,
     $                   V( N2*q+1, N2+1 ), LDV, EPSLN, SFMIN, TOL, 1,
     $                   CWORK( N+1 ), LWORK-N, IERR )

         END IF
*
      END IF
*
*     .. Row-cyclic pivot strategy with de Rijk's pivoting ..
*
      DO 1993 i = 1, NSWEEP
*
*     .. go go go ...
*
         MXAAPQ = ZERO
         MXSINJ = ZERO
         ISWROT = 0
*
```

```
         NOTROT = 0
         PSKIPPED = 0
*
*     Each sweep is unrolled using KBL-by-KBL tiles over the pivot pairs
*     1 <= p < q <= N. This is the first step toward a blocked implementation
*     of the rotations. New implementation, based on block transformations,
*     is under development.
*
         DO 2000 ibr = 1, NBL
*
            igl = ( ibr-1 )*KBL + 1
*
            DO 1002 ir1 = 0, MIN0( LKAHEAD, NBL-ibr )
*
               igl = igl + ir1*KBL
*
               DO 2001 p = igl, MIN0( igl+KBL-1, N-1 )
*
*     .. de Rijk's pivoting
*
                  q = IDAMAX( N-p+1, SVA( p ), 1 ) + p - 1
                  IF( p.NE.q ) THEN
                     CALL ZSWAP( M, A( 1, p ), 1, A( 1, q ), 1 )
                     IF( RSVEC )CALL ZSWAP( MVL, V( 1, p ), 1,
     $                                       V( 1, q ), 1 )
                     TEMP1 = SVA( p )
                     SVA( p ) = SVA( q )
                     SVA( q ) = TEMP1
                     AAPQ = CWORK(p)
                     CWORK(p) = CWORK(q)
                     CWORK(q) = AAPQ
                  END IF
*
                  IF( ir1.EQ.0 ) THEN
*
         Column norms are periodically updated by explicit
         norm computation.
*[!]     Caveat:
*        Unfortunately, some BLAS implementations compute DZNRM2(M,A(1,p),1)
*        as SQRT(S=CDOTC(M,A(1,p),1,A(1,p),1)), which may cause the result to
*        overflow for ||A(:,p)||_2 > SQRT(overflow_threshold), and to
*        underflow for ||A(:,p)||_2 < SQRT(underflow_threshold).
*        Hence, DZNRM2 cannot be trusted, not even in the case when
*        the true norm is far from the under(over)flow boundaries.
*        If properly implemented SCNRM2 is available, the IF-THEN-ELSE-END IF
*        below should be replaced with "AAPP = DZNRM2( M, A(1,p), 1 )".
*
                     IF( ( SVA( p ).LT.ROOTBIG ) .AND.
     $                    ( SVA( p ).GT.ROOTSFMIN ) ) THEN
                        SVA( p ) = DZNRM2( M, A( 1, p ), 1 )
```

```
                  ELSE
                     TEMP1 = ZERO
                     AAPP = ONE
                     CALL ZLASSQ( M, A( 1, p ), 1, TEMP1, AAPP )
                     SVA( p ) = TEMP1*DSQRT( AAPP )
                  END IF
                  AAPP = SVA( p )
               ELSE
                  AAPP = SVA( p )
               END IF
*
               IF( AAPP.GT.ZERO ) THEN
*
                  PSKIPPED = 0
*
                  DO 2002 q = p + 1, MIN0( igl+KBL-1, N )
*
                     AAQQ = SVA( q )
*
                     IF( AAQQ.GT.ZERO ) THEN
*
                        AAPP0 = AAPP
                        IF( AAQQ.GE.ONE ) THEN
                           ROTOK = ( SMALL*AAPP ).LE.AAQQ
                           IF( AAPP.LT.( BIG / AAQQ ) ) THEN
                              AAPQ = ( ZDOTC( M, A( 1, p ), 1,
     $                                 A( 1, q ), 1 ) / AAQQ ) / AAPP
                           ELSE
                              CALL ZCOPY( M, A( 1, p ), 1,
     $                                    CWORK(N+1), 1 )
                              CALL ZLASCL( 'G', 0, 0, AAPP, ONE,
     $                             M, 1, CWORK(N+1), LDA, IERR )
                              AAPQ = ZDOTC( M, CWORK(N+1), 1,
     $                                 A( 1, q ), 1 ) / AAQQ
                           END IF
                        ELSE
                           ROTOK = AAPP.LE.( AAQQ / SMALL )
                           IF( AAPP.GT.( SMALL / AAQQ ) ) THEN
                              AAPQ = ( ZDOTC( M, A( 1, p ), 1,
     $                                  A( 1, q ), 1 ) / AAQQ ) / AAPP
                           ELSE
                              CALL ZCOPY( M, A( 1, q ), 1,
     $                                    CWORK(N+1), 1 )
                              CALL ZLASCL( 'G', 0, 0, AAQQ,
     $                                     ONE, M, 1,
     $                                     CWORK(N+1), LDA, IERR )
                              AAPQ = ZDOTC( M, A(1, p ), 1,
     $                                 CWORK(N+1), 1 ) / AAPP
                           END IF
                        END IF
```

```
*
                          OMPQ = AAPQ / ABS(AAPQ)
*                          AAPQ = AAPQ * DCONJG( CWORK(p) ) * CWORK(q)
                          AAPQ1  = -ABS(AAPQ)
                          MXAAPQ = DMAX1( MXAAPQ, -AAPQ1 )
*
*        TO rotate or NOT to rotate, THAT is the question ...
*
                          IF( ABS( AAPQ1 ).GT.TOL ) THEN
*
*           .. rotate
*[RTD]      ROTATED = ROTATED + ONE
*
                             IF( ir1.EQ.0 ) THEN
                                NOTROT = 0
                                PSKIPPED = 0
                                ISWROT = ISWROT + 1
                             END IF
*
                             IF( ROTOK ) THEN
*
                                AQOAP = AAQQ / AAPP
                                APOAQ = AAPP / AAQQ
                                THETA = -HALF*ABS( AQOAP-APOAQ )/AAPQ1
*
                                IF( ABS( THETA ).GT.BIGTHETA ) THEN
*
                                   T  = HALF / THETA
                                   CS = ONE

                                   CALL ZROT( M, A(1,p), 1, A(1,q), 1,
     $                                        CS, DCONJG(OMPQ)*T )
                                   IF ( RSVEC ) THEN
                                       CALL ZROT( MVL, V(1,p), 1,
     $                                 V(1,q), 1, CS, DCONJG(OMPQ)*T )
                                   END IF

                                   SVA( q ) = AAQQ*DSQRT( DMAX1( ZERO,
     $                                        ONE+T*APOAQ*AAPQ1 ) )
                                   AAPP = AAPP*DSQRT( DMAX1( ZERO,
     $                                        ONE-T*AQOAP*AAPQ1 ) )
                                   MXSINJ = DMAX1( MXSINJ, ABS( T ) )
*
                                ELSE
*
*                 .. choose correct signum for THETA and rotate
*
                                   THSIGN = -DSIGN( ONE, AAPQ1 )
                                   T = ONE / ( THETA+THSIGN*
     $                                  DSQRT( ONE+THETA*THETA ) )
```

```
                                 CS = DSQRT( ONE / ( ONE+T*T ) )
                                 SN = T*CS
*
                                 MXSINJ = DMAX1( MXSINJ, ABS( SN ) )
                                 SVA( q ) = AAQQ*DSQRT( DMAX1( ZERO,
     $                                      ONE+T*APOAQ*AAPQ1 ) )
                                 AAPP = AAPP*DSQRT( DMAX1( ZERO,
     $                                 ONE-T*AQOAP*AAPQ1 ) )
*
                                 CALL ZROT( M, A(1,p), 1, A(1,q), 1,
     $                                      CS, DCONJG(OMPQ)*SN )
                                 IF ( RSVEC ) THEN
                                     CALL ZROT( MVL, V(1,p), 1,
     $                               V(1,q), 1, CS, DCONJG(OMPQ)*SN )
                                 END IF
                              END IF
                              CWORK(p) = -CWORK(q) * OMPQ
*
                              ELSE
*             .. have to use modified Gram-Schmidt like transformation
                              CALL ZCOPY( M, A( 1, p ), 1,
     $                                    CWORK(N+1), 1 )
                              CALL ZLASCL( 'G', 0, 0, AAPP, ONE, M,
     $                                     1, CWORK(N+1), LDA,
     $                                     IERR )
                              CALL ZLASCL( 'G', 0, 0, AAQQ, ONE, M,
     $                                     1, A( 1, q ), LDA, IERR )
                              CALL ZAXPY( M, -AAPQ, CWORK(N+1), 1,
     $                                    A( 1, q ), 1 )
                              CALL ZLASCL( 'G', 0, 0, ONE, AAQQ, M,
     $                                     1, A( 1, q ), LDA, IERR )
                              SVA( q ) = AAQQ*DSQRT( DMAX1( ZERO,
     $                                   ONE-AAPQ1*AAPQ1 ) )
                              MXSINJ = DMAX1( MXSINJ, SFMIN )
                           END IF
*          END IF ROTOK THEN ... ELSE
*
*          In the case of cancellation in updating SVA(q), SVA(p)
*          recompute SVA(q), SVA(p).
*
                           IF( ( SVA( q ) / AAQQ )**2.LE.ROOTEPS )
     $                         THEN
                              IF( ( AAQQ.LT.ROOTBIG ) .AND.
     $                            ( AAQQ.GT.ROOTSFMIN ) ) THEN
                                 SVA( q ) = DZNRM2( M, A( 1, q ), 1 )
                              ELSE
                                 T = ZERO
                                 AAQQ = ONE
                                 CALL ZLASSQ( M, A( 1, q ), 1, T,
     $                                        AAQQ )
```

```
                                 SVA( q ) = T*DSQRT( AAQQ )
                              END IF
                           END IF
                           IF( ( AAPP / AAPP0 ).LE.ROOTEPS ) THEN
                              IF( ( AAPP.LT.ROOTBIG ) .AND.
     $                            ( AAPP.GT.ROOTSFMIN ) ) THEN
                                 AAPP = DZNRM2( M, A( 1, p ), 1 )
                              ELSE
                                 T = ZERO
                                 AAPP = ONE
                                 CALL ZLASSQ( M, A( 1, p ), 1, T,
     $                                       AAPP )
                                 AAPP = T*DSQRT( AAPP )
                              END IF
                              SVA( p ) = AAPP
                           END IF
*
                        ELSE
*                          A(:,p) and A(:,q) already numerically orthogonal
                           IF( ir1.EQ.0 )NOTROT = NOTROT + 1
*[RTD]      SKIPPED  = SKIPPED + 1
                           PSKIPPED = PSKIPPED + 1
                        END IF
                     ELSE
*                       A(:,q) is zero column
                        IF( ir1.EQ.0 )NOTROT = NOTROT + 1
                        PSKIPPED = PSKIPPED + 1
                     END IF
*
                     IF( ( i.LE.SWBAND ) .AND.
     $                   ( PSKIPPED.GT.ROWSKIP ) ) THEN
                        IF( ir1.EQ.0 )AAPP = -AAPP
                        NOTROT = 0
                        GO TO 2103
                     END IF
*
 2002                CONTINUE
*     END q-LOOP
*
 2103                CONTINUE
*     bailed out of q-loop
*
                     SVA( p ) = AAPP
*
                  ELSE
                     SVA( p ) = AAPP
                     IF( ( ir1.EQ.0 ) .AND. ( AAPP.EQ.ZERO ) )
     $                   NOTROT = NOTROT + MIN0( igl+KBL-1, N ) - p
                  END IF
*
```

```
 2001          CONTINUE
*     end of the p-loop
*     end of doing the block ( ibr, ibr )
 1002       CONTINUE
*     end of ir1-loop
*
* ... go to the off diagonal blocks
*
            igl = ( ibr-1 )*KBL + 1
*
            DO 2010 jbc = ibr + 1, NBL
*
               jgl = ( jbc-1 )*KBL + 1
*
        doing the block at ( ibr, jbc )
*
               IJBLSK = 0
               DO 2100 p = igl, MIN0( igl+KBL-1, N )
*
                  AAPP = SVA( p )
                  IF( AAPP.GT.ZERO ) THEN
*
                     PSKIPPED = 0
*
                     DO 2200 q = jgl, MIN0( jgl+KBL-1, N )
*
                        AAQQ = SVA( q )
                        IF( AAQQ.GT.ZERO ) THEN
                           AAPP0 = AAPP
*
*     .. M x 2 Jacobi SVD ..
*
        Safe Gram matrix computation
*
                           IF( AAQQ.GE.ONE ) THEN
                              IF( AAPP.GE.AAQQ ) THEN
                                 ROTOK = ( SMALL*AAPP ).LE.AAQQ
                              ELSE
                                 ROTOK = ( SMALL*AAQQ ).LE.AAPP
                              END IF
                              IF( AAPP.LT.( BIG / AAQQ ) ) THEN
                                 AAPQ = ( ZDOTC( M, A( 1, p ), 1,
     $                                  A( 1, q ), 1 ) / AAQQ ) / AAPP
                              ELSE
                                 CALL ZCOPY( M, A( 1, p ), 1,
     $                                       CWORK(N+1), 1 )
                                 CALL ZLASCL( 'G', 0, 0, AAPP,
     $                                        ONE, M, 1,
     $                                        CWORK(N+1), LDA, IERR )
                                 AAPQ = ZDOTC( M, CWORK(N+1), 1,
```

```
     $                                A( 1, q ), 1 ) / AAQQ
                            END IF
                         ELSE
                            IF( AAPP.GE.AAQQ ) THEN
                               ROTOK = AAPP.LE.( AAQQ / SMALL )
                            ELSE
                               ROTOK = AAQQ.LE.( AAPP / SMALL )
                            END IF
                            IF( AAPP.GT.( SMALL / AAQQ ) ) THEN
                               AAPQ = ( ZDOTC( M, A( 1, p ), 1,
     $                                  A( 1, q ), 1 ) / AAQQ ) / AAPP
                            ELSE
                               CALL ZCOPY( M, A( 1, q ), 1,
     $                                     CWORK(N+1), 1 )
                               CALL ZLASCL( 'G', 0, 0, AAQQ,
     $                                      ONE, M, 1,
     $                                      CWORK(N+1), LDA, IERR )
                               AAPQ = ZDOTC( M, A( 1, p ), 1,
     $                                CWORK(N+1),  1 ) / AAPP
                            END IF
                         END IF
*
                         OMPQ = AAPQ / ABS(AAPQ)
*                         AAPQ = AAPQ * DCONJG(CWORK(p))*CWORK(q)
                         AAPQ1  = -ABS(AAPQ)
                         MXAAPQ = DMAX1( MXAAPQ, -AAPQ1 )
*
*        TO rotate or NOT to rotate, THAT is the question ...
*
                         IF( ABS( AAPQ1 ).GT.TOL ) THEN
                            NOTROT = 0
*[RTD]      ROTATED  = ROTATED + 1
                            PSKIPPED = 0
                            ISWROT = ISWROT + 1
*
                            IF( ROTOK ) THEN
*
                               AQOAP = AAQQ / AAPP
                               APOAQ = AAPP / AAQQ
                               THETA = -HALF*ABS( AQOAP-APOAQ )/ AAPQ1
                               IF( AAQQ.GT.AAPPO )THETA = -THETA
*
                               IF( ABS( THETA ).GT.BIGTHETA ) THEN
                                  T  = HALF / THETA
                                  CS = ONE
                                  CALL ZROT( M, A(1,p), 1, A(1,q), 1,
     $                                       CS, DCONJG(OMPQ)*T )
                                  IF( RSVEC ) THEN
                                      CALL ZROT( MVL, V(1,p), 1,
     $                                V(1,q), 1, CS, DCONJG(OMPQ)*T )
```

```
                                 END IF
                                 SVA( q ) = AAQQ*DSQRT( DMAX1( ZERO,
     $                                     ONE+T*APOAQ*AAPQ1 ) )
                                 AAPP = AAPP*DSQRT( DMAX1( ZERO,
     $                                ONE-T*AQOAP*AAPQ1 ) )
                                 MXSINJ = DMAX1( MXSINJ, ABS( T ) )
                              ELSE
*
*                 .. choose correct signum for THETA and rotate
*
                                 THSIGN = -DSIGN( ONE, AAPQ1 )
                                 IF( AAQQ.GT.AAPP0 )THSIGN = -THSIGN
                                 T = ONE / ( THETA+THSIGN*
     $                               DSQRT( ONE+THETA*THETA ) )
                                 CS = DSQRT( ONE / ( ONE+T*T ) )
                                 SN = T*CS
                                 MXSINJ = DMAX1( MXSINJ, ABS( SN ) )
                                 SVA( q ) = AAQQ*DSQRT( DMAX1( ZERO,
     $                                     ONE+T*APOAQ*AAPQ1 ) )
                                 AAPP = AAPP*DSQRT( DMAX1( ZERO,
     $                                     ONE-T*AQOAP*AAPQ1 ) )
*
                                 CALL ZROT( M, A(1,p), 1, A(1,q), 1,
     $                                      CS, DCONJG(OMPQ)*SN )
                                 IF( RSVEC ) THEN
                                     CALL ZROT( MVL, V(1,p), 1,
     $                               V(1,q), 1, CS, DCONJG(OMPQ)*SN )
                                 END IF
                              END IF
                              CWORK(p) = -CWORK(q) * OMPQ
*
                           ELSE
*              .. have to use modified Gram-Schmidt like transformation
                            IF( AAPP.GT.AAQQ ) THEN
                                 CALL ZCOPY( M, A( 1, p ), 1,
     $                                      CWORK(N+1), 1 )
                                 CALL ZLASCL( 'G', 0, 0, AAPP, ONE,
     $                                       M, 1, CWORK(N+1),LDA,
     $                                       IERR )
                                 CALL ZLASCL( 'G', 0, 0, AAQQ, ONE,
     $                                       M, 1, A( 1, q ), LDA,
     $                                       IERR )
                                 CALL ZAXPY( M, -AAPQ, CWORK(N+1),
     $                                     1, A( 1, q ), 1 )
                                 CALL ZLASCL( 'G', 0, 0, ONE, AAQQ,
     $                                       M, 1, A( 1, q ), LDA,
     $                                       IERR )
                                 SVA( q ) = AAQQ*DSQRT( DMAX1( ZERO,
     $                                     ONE-AAPQ1*AAPQ1 ) )
                                 MXSINJ = DMAX1( MXSINJ, SFMIN )
```

```
                              ELSE
                                  CALL ZCOPY( M, A( 1, q ), 1,
     $                                         CWORK(N+1), 1 )
                                   CALL ZLASCL( 'G', 0, 0, AAQQ, ONE,
     $                                          M, 1, CWORK(N+1),LDA,
     $                                          IERR )
                                   CALL ZLASCL( 'G', 0, 0, AAPP, ONE,
     $                                          M, 1, A( 1, p ), LDA,
     $                                          IERR )
                                   CALL ZAXPY( M, -DCONJG(AAPQ),
     $                                  CWORK(N+1), 1, A( 1, p ), 1 )
                                   CALL ZLASCL( 'G', 0, 0, ONE, AAPP,
     $                                          M, 1, A( 1, p ), LDA,
     $                                          IERR )
                                   SVA( p ) = AAPP*DSQRT( DMAX1( ZERO,
     $                                       ONE-AAPQ1*AAPQ1 ) )
                                   MXSINJ = DMAX1( MXSINJ, SFMIN )
                              END IF
                             END IF
*          END IF ROTOK THEN ... ELSE
*
*          In the case of cancellation in updating SVA(q), SVA(p)
*          .. recompute SVA(q), SVA(p)
                             IF( ( SVA( q ) / AAQQ )**2.LE.ROOTEPS )
     $                           THEN
                                IF( ( AAQQ.LT.ROOTBIG ) .AND.
     $                              ( AAQQ.GT.ROOTSFMIN ) ) THEN
                                   SVA( q ) = DZNRM2( M, A( 1, q ), 1)
                                 ELSE
                                   T = ZERO
                                   AAQQ = ONE
                                   CALL ZLASSQ( M, A( 1, q ), 1, T,
     $                                          AAQQ )
                                   SVA( q ) = T*DSQRT( AAQQ )
                                END IF
                             END IF
                             IF( ( AAPP / AAPP0 )**2.LE.ROOTEPS ) THEN
                                IF( ( AAPP.LT.ROOTBIG ) .AND.
     $                              ( AAPP.GT.ROOTSFMIN ) ) THEN
                                   AAPP = DZNRM2( M, A( 1, p ), 1 )
                                ELSE
                                   T = ZERO
                                   AAPP = ONE
                                   CALL ZLASSQ( M, A( 1, p ), 1, T,
     $                                          AAPP )
                                   AAPP = T*DSQRT( AAPP )
                                END IF
                                SVA( p ) = AAPP
                             END IF
*             end of OK rotation
```

```
                          ELSE
                             NOTROT = NOTROT + 1
*[RTD]      SKIPPED  = SKIPPED  + 1
                             PSKIPPED = PSKIPPED + 1
                             IJBLSK = IJBLSK + 1
                          END IF
                       ELSE
                          NOTROT = NOTROT + 1
                          PSKIPPED = PSKIPPED + 1
                          IJBLSK = IJBLSK + 1
                       END IF
*
                       IF( ( i.LE.SWBAND ) .AND. ( IJBLSK.GE.BLSKIP ) )
     $                     THEN
                          SVA( p ) = AAPP
                          NOTROT = 0
                          GO TO 2011
                       END IF
                       IF( ( i.LE.SWBAND ) .AND.
     $                     ( PSKIPPED.GT.ROWSKIP ) ) THEN
                          AAPP = -AAPP
                          NOTROT = 0
                          GO TO 2203
                       END IF
*
 2200                CONTINUE
*        end of the q-loop
 2203                CONTINUE
*
                     SVA( p ) = AAPP
*
                  ELSE
*
                     IF( AAPP.EQ.ZERO )NOTROT = NOTROT +
     $                   MIN0( jgl+KBL-1, N ) - jgl + 1
                     IF( AAPP.LT.ZERO )NOTROT = 0
*
                  END IF
*
 2100          CONTINUE
*     end of the p-loop
 2010       CONTINUE
*     end of the jbc-loop
 2011       CONTINUE
*2011 bailed out of the jbc-loop
            DO 2012 p = igl, MIN0( igl+KBL-1, N )
               SVA( p ) = ABS( SVA( p ) )
 2012       CONTINUE
***
 2000    CONTINUE
```

```
*2000 :: end of the ibr-loop
*
*     .. update SVA(N)
         IF( ( SVA( N ).LT.ROOTBIG ) .AND. ( SVA( N ).GT.ROOTSFMIN ) )
     $       THEN
            SVA( N ) = DZNRM2( M, A( 1, N ), 1 )
         ELSE
            T = ZERO
            AAPP = ONE
            CALL ZLASSQ( M, A( 1, N ), 1, T, AAPP )
            SVA( N ) = T*DSQRT( AAPP )
         END IF
*
*     Additional steering devices
*
         IF( ( i.LT.SWBAND ) .AND. ( ( MXAAPQ.LE.ROOTTOL ) .OR.
     $       ( ISWROT.LE.N ) ) )SWBAND = i
*
         IF( ( i.GT.SWBAND+1 ) .AND. ( MXAAPQ.LT.DSQRT( DFLOAT( N ) )*
     $       TOL ) .AND. ( DFLOAT( N )*MXAAPQ*MXSINJ.LT.TOL ) ) THEN
            GO TO 1994
         END IF
*
         IF( NOTROT.GE.EMPTSW )GO TO 1994
*
 1993 CONTINUE
*     end i=1:NSWEEP loop
*
* #:( Reaching this point means that the procedure has not converged.
      INFO = NSWEEP - 1
      GO TO 1995
*
 1994 CONTINUE
* #:) Reaching this point means numerical convergence after the i-th
*     sweep.
*
      INFO = 0
* #:) INFO = 0 confirms successful iterations.
 1995 CONTINUE
*
*     Sort the singular values and find how many are above
*     the underflow threshold.
*
      N2 = 0
      N4 = 0
      DO 5991 p = 1, N - 1
         q = IDAMAX( N-p+1, SVA( p ), 1 ) + p - 1
         IF( p.NE.q ) THEN
            TEMP1 = SVA( p )
            SVA( p ) = SVA( q )
```

```
            SVA( q ) = TEMP1
            CALL ZSWAP( M, A( 1, p ), 1, A( 1, q ), 1 )
            IF( RSVEC )CALL ZSWAP( MVL, V( 1, p ), 1, V( 1, q ), 1 )
         END IF
         IF( SVA( p ).NE.ZERO ) THEN
            N4 = N4 + 1
            IF( SVA( p )*SKL.GT.SFMIN )N2 = N2 + 1
         END IF
 5991 CONTINUE
      IF( SVA( N ).NE.ZERO ) THEN
         N4 = N4 + 1
         IF( SVA( N )*SKL.GT.SFMIN )N2 = N2 + 1
      END IF
*
*     Normalize the left singular vectors.
*
      IF( LSVEC .OR. UCTOL ) THEN
         DO 1998 p = 1, N2
            CALL ZDSCAL( M, ONE / SVA( p ), A( 1, p ), 1 )
 1998    CONTINUE
      END IF
*
*     Scale the product of Jacobi rotations.
*
      IF( RSVEC ) THEN
            DO 2399 p = 1, N
               TEMP1 = ONE / DZNRM2( MVL, V( 1, p ), 1 )
               CALL ZDSCAL( MVL, TEMP1, V( 1, p ), 1 )
 2399       CONTINUE
      END IF
*
*     Undo scaling, if necessary (and possible).
      IF( ( ( SKL.GT.ONE ) .AND. ( SVA( 1 ).LT.( BIG / SKL ) ) )
     $    .OR. ( ( SKL.LT.ONE ) .AND. ( SVA( MAX( N2, 1 ) ) .GT.
     $    ( SFMIN / SKL ) ) ) ) THEN
         DO 2400 p = 1, N
            SVA( P ) = SKL*SVA( P )
 2400    CONTINUE
         SKL = ONE
      END IF
*
      RWORK( 1 ) = SKL
*     The singular values of A are SKL*SVA(1:N). If SKL.NE.ONE
*     then some of the singular values may overflow or underflow and
*     the spectrum is given in this factored representation.
*
      RWORK( 2 ) = DFLOAT( N4 )
*     N4 is the number of computed nonzero singular values of A.
*
      RWORK( 3 ) = DFLOAT( N2 )
```

```
*     N2 is the number of singular values of A greater than SFMIN.
*     If N2<N, SVA(N2:N) contains ZEROS and/or denormalized numbers
*     that may carry some information.
*
      RWORK( 4 ) = DFLOAT( i )
*     i is the index of the last sweep before declaring convergence.
*
      RWORK( 5 ) = MXAAPQ
*     MXAAPQ is the largest absolute value of scaled pivots in the
*     last sweep
*
      RWORK( 6 ) = MXSINJ
*     MXSINJ is the largest absolute value of the sines of Jacobi angles
*     in the last sweep
*
      RETURN
*     ..
*     .. END OF ZGESVJ
*     ..
      END
```

32.41.4 lisp code

— zgesvj.lisp —

```
(let* ((zero 0.0d0)
       (half 0.5d0)
       (one 1.0d0)
       (czero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (cone (f2cl-lib:cmplx 1.0d0 0.0d0))
       (nsweep 30))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 0.5d0 0.5d0) half)
           (type (double-float 1.0d0 1.0d0) one)
           (type (f2cl-lib:complex16) czero)
           (type (f2cl-lib:complex16) cone)
           (type (f2cl-lib:integer4 30 30) nsweep)
           (ignorable zero half one czero cone nsweep))
  (defun zgesvj
         (joba jobu jobv m n a lda sva mv v ldv cwork lwork rwork lrwork info)
    (declare (type (array double-float (*)) rwork sva)
             (type (array f2cl-lib:complex16 (*)) cwork v a)
             (type (f2cl-lib:integer4) info lrwork lwork ldv mv lda n m)
             (type (string *) jobv jobu joba))
    (f2cl-lib:with-multi-array-data
        ((joba character joba-%data% joba-%offset%)
```

```
    (jobu character jobu-%data% jobu-%offset%)
    (jobv character jobv-%data% jobv-%offset%)
    (a f2cl-lib:complex16 a-%data% a-%offset%)
    (v f2cl-lib:complex16 v-%data% v-%offset%)
    (cwork f2cl-lib:complex16 cwork-%data% cwork-%offset%)
    (sva double-float sva-%data% sva-%offset%)
    (rwork double-float rwork-%data% rwork-%offset%))
  (prog ((applv nil) (goscale nil) (lower nil) (lsvec nil) (noscale nil)
         (rotok nil) (rsvec nil) (uctol nil) (upper nil) (blskip 0)
         (emptsw 0) (i 0) (ibr 0) (ierr 0) (igl 0) (ijblsk 0) (ir1 0)
         (iswrot 0) (jbc 0) (jgl 0) (kbl 0) (lkahead 0) (mvl 0) (n2 0)
         (n34 0) (n4 0) (nbl 0) (notrot 0) (p 0) (pskipped 0) (q 0)
         (rowskip 0) (swband 0) (aapp 0.0d0) (aapp0 0.0d0) (aapq1 0.0d0)
         (aaqq 0.0d0) (apoaq 0.0d0) (aqoap 0.0d0) (big 0.0d0)
         (bigtheta 0.0d0) (cs 0.0d0) (ctol 0.0d0) (epsln 0.0d0)
         (large 0.0d0) (mxaapq 0.0d0) (mxsinj 0.0d0) (rootbig 0.0d0)
         (rooteps 0.0d0) (rootsfmin 0.0d0) (roottol 0.0d0) (skl 0.0d0)
         (sfmin 0.0d0) (small 0.0d0) (sn 0.0d0) (temp1 0.0d0) (theta 0.0d0)
         (thsign 0.0d0) (tol 0.0d0) (aapq #C(0.0d0 0.0d0))
         (ompq #C(0.0d0 0.0d0)) (t$ 0.0d0))
    (declare
     (type f2cl-lib:logical applv goscale lower lsvec noscale rotok rsvec
      uctol upper)
     (type (f2cl-lib:integer4) blskip emptsw i ibr ierr igl ijblsk ir1
      iswrot jbc jgl kbl lkahead mvl n2 n34 n4 nbl notrot p pskipped q
      rowskip swband)
     (type (double-float) t$ aapp aapp0 aapq1 aaqq apoaq aqoap big bigtheta
      cs ctol epsln large mxaapq mxsinj rootbig rooteps rootsfmin roottol
      skl sfmin small sn temp1 theta thsign tol)
     (type (f2cl-lib:complex16) aapq ompq))
    (setf lsvec
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobu "U")
              (declare (ignore var-1))
              (when var-0 (setf jobu var-0))
              ret-val))
    (setf uctol
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobu "C")
              (declare (ignore var-1))
              (when var-0 (setf jobu var-0))
              ret-val))
    (setf rsvec
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobv "V")
              (declare (ignore var-1))
              (when var-0 (setf jobv var-0))
              ret-val))
    (setf applv
            (multiple-value-bind (ret-val var-0 var-1)
```

```
            (lsame jobv "A")
          (declare (ignore var-1))
          (when var-0 (setf jobv var-0))
          ret-val))
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame joba "U")
          (declare (ignore var-1))
          (when var-0 (setf joba var-0))
          ret-val))
(setf lower
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame joba "L")
          (declare (ignore var-1))
          (when var-0 (setf joba var-0))
          ret-val))
(cond
 ((not
   (or upper lower
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame joba "G")
         (declare (ignore var-1))
         (when var-0 (setf joba var-0))
         ret-val)))
  (setf info -1))
 ((not
   (or lsvec uctol
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobu "N")
         (declare (ignore var-1))
         (when var-0 (setf jobu var-0))
         ret-val)))
  (setf info -2))
 ((not
   (or rsvec applv
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobv "N")
         (declare (ignore var-1))
         (when var-0 (setf jobv var-0))
         ret-val)))
  (setf info -3))
 ((< m 0) (setf info -4)) ((or (< n 0) (> n m)) (setf info -5))
 ((< lda m) (setf info -7)) ((< mv 0) (setf info -9))
 ((or (and rsvec (< ldv n)) (and applv (< ldv mv))) (setf info -11))
 ((and uctol (<= (f2cl-lib:fref rwork (1) ((1 lrwork))) one))
  (setf info -12))
 ((< lwork (f2cl-lib:int-add m n)) (setf info -13))
 ((< lrwork (f2cl-lib:max0 n 6)) (setf info -15)) (t (setf info 0)))
(cond
 ((/= info 0) (xerbla "ZGESVJ" (f2cl-lib:int-sub info))
```

```
  (go end_label)))
(if (or (= m 0) (= n 0))
    (go end_label))
(cond
 (uctol
  (setf ctol
          (f2cl-lib:fref rwork-%data% (1) ((1 lrwork))
                         rwork-%offset%)))
 (t
  (cond
   ((or lsvec rsvec applv)
    (setf ctol (f2cl-lib:dsqrt (f2cl-lib:dfloat m))))
   (t (setf ctol (coerce (f2cl-lib:dfloat m) 'double-float))))))
(setf epsln (dlamch "Epsilon"))
(setf rooteps (f2cl-lib:dsqrt epsln))
(setf sfmin (dlamch "SafeMinimum"))
(setf rootsfmin (f2cl-lib:dsqrt sfmin))
(setf small (/ sfmin epsln))
(setf big (dlamch "Overflow"))
(setf rootbig (/ one rootsfmin))
(setf large
        (/ big
           (f2cl-lib:dsqrt (f2cl-lib:dfloat (f2cl-lib:int-mul m n)))))
(setf bigtheta (/ one rooteps))
(setf tol (* ctol epsln))
(setf roottol (f2cl-lib:dsqrt tol))
(cond
 ((>= (* (f2cl-lib:dfloat m) epsln) one) (setf info -4)
  (xerbla "ZGESVJ" (f2cl-lib:int-sub info)) (go end_label)))
(cond
 (rsvec (setf mvl n)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaset "A" mvl n czero cone v ldv)
    (declare (ignore var-0 var-5))
    (when var-1 (setf mvl var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf cone var-4))
    (when var-6 (setf ldv var-6))))
 (applv (setf mvl mv)))
(setf rsvec (or rsvec applv))
(setf skl
        (/ one
           (f2cl-lib:dsqrt
            (* (f2cl-lib:dfloat m) (f2cl-lib:dfloat n)))))
(setf noscale f2cl-lib:%true%)
(setf goscale f2cl-lib:%true%)
(cond
 (lower
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
```

```
                ((> p n) nil)
    (tagbody
      (setf aapp zero)
      (setf aaqq one)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (zlassq (f2cl-lib:int-add (f2cl-lib:int-sub m p) 1)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (p p)
                                 ((1 lda) (1 *)) a-%offset%)
           1 aapp aaqq)
        (declare (ignore var-0 var-1 var-2))
        (when var-3 (setf aapp var-3))
        (when var-4 (setf aaqq var-4)))
      (cond
       ((> aapp big) (setf info -6)
        (xerbla "ZGESVJ" (f2cl-lib:int-sub info)) (go end_label)))
      (setf aaqq (f2cl-lib:dsqrt aaqq))
      (cond
       ((and (< aapp (f2cl-lib:f2cl/ big aaqq)) noscale)
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                (* aapp aaqq)))
       (t (setf noscale f2cl-lib:%false%)
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                (* aapp (* aaqq skl)))
        (cond
         (goscale (setf goscale f2cl-lib:%false%)
          (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                        ((> q
                            (f2cl-lib:int-add p (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%)
                      (*
                       (f2cl-lib:fref sva-%data% (q) ((1 n))
                                      sva-%offset%)
                       skl))
             label1873))))))
   label1874)))
(upper
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p n) nil)
   (tagbody
     (setf aapp zero)
     (setf aaqq one)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (zlassq p
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 p)
                                ((1 lda) (1 *)) a-%offset%)
          1 aapp aaqq)
       (declare (ignore var-1 var-2))
       (when var-0 (setf p var-0))
```

```
          (when var-3 (setf aapp var-3))
          (when var-4 (setf aaqq var-4)))
        (cond
         ((> aapp big) (setf info -6)
          (xerbla "ZGESVJ" (f2cl-lib:int-sub info)) (go end_label)))
        (setf aaqq (f2cl-lib:dsqrt aaqq))
        (cond
         ((and (< aapp (f2cl-lib:f2cl/ big aaqq)) noscale)
          (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                  (* aapp aaqq)))
         (t (setf noscale f2cl-lib:%false%)
          (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                  (* aapp (* aaqq skl)))
          (cond
           (goscale (setf goscale f2cl-lib:%false%)
            (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                          ((> q
                              (f2cl-lib:int-add p (f2cl-lib:int-sub 1)))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%)
                        (*
                         (f2cl-lib:fref sva-%data% (q) ((1 n))
                                        sva-%offset%)
                         skl))
              label2873))))))
    label2874)))
(t
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p n) nil)
   (tagbody
     (setf aapp zero)
     (setf aaqq one)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (zlassq m
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 p)
                                ((1 lda) (1 *)) a-%offset%)
          1 aapp aaqq)
       (declare (ignore var-1 var-2))
       (when var-0 (setf m var-0))
       (when var-3 (setf aapp var-3))
       (when var-4 (setf aaqq var-4)))
     (cond
      ((> aapp big) (setf info -6)
       (xerbla "ZGESVJ" (f2cl-lib:int-sub info)) (go end_label)))
     (setf aaqq (f2cl-lib:dsqrt aaqq))
     (cond
      ((and (< aapp (f2cl-lib:f2cl/ big aaqq)) noscale)
       (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
               (* aapp aaqq)))
```

```
        (t (setf noscale f2cl-lib:%false%)
         (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                 (* aapp (* aaqq skl)))
         (cond
          (goscale (setf goscale f2cl-lib:%false%)
           (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                         ((> q
                             (f2cl-lib:int-add p (f2cl-lib:int-sub 1)))
                          nil)
             (tagbody
               (setf (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%)
                       (*
                        (f2cl-lib:fref sva-%data% (q) ((1 n))
                                       sva-%offset%)
                        skl))
              label3873))))))
     label3874))))
(if noscale
    (setf skl one))
(setf aapp zero)
(setf aaqq big)
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (if (/= (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%) zero)
        (setf aaqq
                (f2cl-lib:dmin1 aaqq
                                (f2cl-lib:fref sva-%data% (p) ((1 n))
                                               sva-%offset%))))
    (setf aapp
            (f2cl-lib:dmax1 aapp
                            (f2cl-lib:fref sva-%data% (p) ((1 n))
                                           sva-%offset%)))
   label4781))
(cond
 ((= aapp zero)
  (if lsvec
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlaset "G" m n czero cone a lda)
        (declare (ignore var-0 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf czero var-3))
        (when var-4 (setf cone var-4))
        (when var-6 (setf lda var-6))))
  (setf (f2cl-lib:fref rwork-%data% (1) ((1 lrwork)) rwork-%offset%)
          one)
  (setf (f2cl-lib:fref rwork-%data% (2) ((1 lrwork)) rwork-%offset%)
          zero)
  (setf (f2cl-lib:fref rwork-%data% (3) ((1 lrwork)) rwork-%offset%)
```

```
            zero)
    (setf (f2cl-lib:fref rwork-%data% (4) ((1 lrwork)) rwork-%offset%)
            zero)
    (setf (f2cl-lib:fref rwork-%data% (5) ((1 lrwork)) rwork-%offset%)
            zero)
    (setf (f2cl-lib:fref rwork-%data% (6) ((1 lrwork)) rwork-%offset%)
            zero)
    (go end_label)))
  (cond
   ((= n 1)
    (if lsvec
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
            (zlascl "G" 0 0
             (f2cl-lib:array-slice sva-%data% double-float (1) ((1 n))
                                   sva-%offset%)
             skl m 1
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 1)
                                   ((1 lda) (1 *)) a-%offset%)
             lda ierr)
          (declare (ignore var-0 var-1 var-2 var-3 var-6 var-7))
          (when var-4 (setf skl var-4))
          (when var-5 (setf m var-5))
          (when var-8 (setf lda var-8))
          (when var-9 (setf ierr var-9))))
    (setf (f2cl-lib:fref rwork-%data% (1) ((1 lrwork)) rwork-%offset%)
            (/ one skl))
    (cond
     ((>= (f2cl-lib:fref sva (1) ((1 n))) sfmin)
      (setf (f2cl-lib:fref rwork-%data% (2) ((1 lrwork)) rwork-%offset%)
              one))
     (t
      (setf (f2cl-lib:fref rwork-%data% (2) ((1 lrwork)) rwork-%offset%)
              zero)))
    (setf (f2cl-lib:fref rwork-%data% (3) ((1 lrwork)) rwork-%offset%)
            zero)
    (setf (f2cl-lib:fref rwork-%data% (4) ((1 lrwork)) rwork-%offset%)
            zero)
    (setf (f2cl-lib:fref rwork-%data% (5) ((1 lrwork)) rwork-%offset%)
            zero)
    (setf (f2cl-lib:fref rwork-%data% (6) ((1 lrwork)) rwork-%offset%)
            zero)
    (go end_label)))
  (setf sn (f2cl-lib:dsqrt (/ sfmin epsln)))
  (setf temp1 (f2cl-lib:dsqrt (/ big (f2cl-lib:dfloat n))))
  (cond
   ((or (<= aapp sn) (>= aaqq temp1) (and (<= sn aaqq) (<= aapp temp1)))
    (setf temp1 (f2cl-lib:dmin1 big (/ temp1 aapp))))
   ((and (<= aaqq sn) (<= aapp temp1))
    (setf temp1
```

```
          (f2cl-lib:dmin1 (/ sn aaqq)
                          (/ big
                             (* aapp
                                (f2cl-lib:dsqrt
                                 (f2cl-lib:dfloat n)))))))
 ((and (>= aaqq sn) (>= aapp temp1))
  (setf temp1 (f2cl-lib:dmax1 (/ sn aaqq) (/ temp1 aapp))))
 ((and (<= aaqq sn) (>= aapp temp1))
  (setf temp1
          (f2cl-lib:dmin1 (/ sn aaqq)
                          (/ big
                             (* (f2cl-lib:dsqrt (f2cl-lib:dfloat n))
                                aapp)))))
 (t (setf temp1 one)))
(cond
 ((/= temp1 one)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl "G" 0 0 one temp1 n 1 sva n ierr)
    (declare (ignore var-0 var-1 var-2 var-6 var-7))
    (when var-3 (setf one var-3))
    (when var-4 (setf temp1 var-4))
    (when var-5 (setf n var-5))
    (when var-8 (setf n var-8))
    (when var-9 (setf ierr var-9)))))
(setf skl (* temp1 skl))
(cond
 ((/= skl one)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zlascl joba 0 0 one skl m n a lda ierr)
    (declare (ignore var-1 var-2 var-7))
    (when var-0 (setf joba var-0))
    (when var-3 (setf one var-3))
    (when var-4 (setf skl var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf ierr var-9)))
  (setf skl (/ one skl))))
(setf emptsw (the f2cl-lib:integer4 (truncate (* n (- n 1)) 2)))
(setf notrot 0)
(f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
              ((> q n) nil)
  (tagbody
    (setf (f2cl-lib:fref cwork-%data% (q) ((1 lwork)) cwork-%offset%)
            cone)
   label1868))
(setf swband 3)
(setf kbl (f2cl-lib:min0 8 n))
```

```
(setf nbl (the f2cl-lib:integer4 (truncate n kbl)))
(if (/= (f2cl-lib:int-mul nbl kbl) n)
    (setf nbl (f2cl-lib:int-add nbl 1)))
(setf blskip (expt kbl 2))
(setf rowskip (f2cl-lib:min0 5 kbl))
(setf lkahead 1)
(cond
 ((and (or lower upper)
       (> n (f2cl-lib:max0 64 (f2cl-lib:int-mul 4 kbl))))
  (setf n4 (the f2cl-lib:integer4 (truncate n 4)))
  (setf n2 (the f2cl-lib:integer4 (truncate n 2)))
  (setf n34 (f2cl-lib:int-mul 3 n4))
  (cond (applv (setf q 0)) (t (setf q 1)))
  (cond
   (lower
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14 var-15 var-16)
        (zgsvj0 jobv (f2cl-lib:int-sub m n34) (f2cl-lib:int-sub n n34)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               ((+ n34 1) (f2cl-lib:int-add n34 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                               ((+ n34 1)) ((1 lwork)) cwork-%offset%)
         (f2cl-lib:array-slice sva-%data% double-float ((+ n34 1))
                               ((1 n)) sva-%offset%)
         mvl
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                               ((+ (f2cl-lib:int-mul n34 q) 1)
                                (f2cl-lib:int-add n34 1))
                               ((1 ldv) (1 *)) v-%offset%)
         ldv epsln sfmin tol 2
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                               ((+ n 1)) ((1 lwork)) cwork-%offset%)
         (f2cl-lib:int-sub lwork n) ierr)
      (declare
       (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-13 var-14
        var-15))
      (when var-0 (setf jobv var-0))
      (when var-4 (setf lda var-4))
      (when var-7 (setf mvl var-7))
      (when var-9 (setf ldv var-9))
      (when var-10 (setf epsln var-10))
      (when var-11 (setf sfmin var-11))
      (when var-12 (setf tol var-12))
      (when var-16 (setf ierr var-16)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14 var-15 var-16)
```

```
        (zgsvj0 jobv (f2cl-lib:int-sub m n2) (f2cl-lib:int-sub n34 n2)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               ((+ n2 1) (f2cl-lib:int-add n2 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                               ((+ n2 1)) ((1 lwork)) cwork-%offset%)
         (f2cl-lib:array-slice sva-%data% double-float ((+ n2 1))
                               ((1 n)) sva-%offset%)
         mvl
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                               ((+ (f2cl-lib:int-mul n2 q) 1)
                                (f2cl-lib:int-add n2 1))
                               ((1 ldv) (1 *)) v-%offset%)
         ldv epsln sfmin tol 2
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                               ((+ n 1)) ((1 lwork)) cwork-%offset%)
         (f2cl-lib:int-sub lwork n) ierr)
      (declare
       (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-13 var-14
        var-15))
      (when var-0 (setf jobv var-0))
      (when var-4 (setf lda var-4))
      (when var-7 (setf mvl var-7))
      (when var-9 (setf ldv var-9))
      (when var-10 (setf epsln var-10))
      (when var-11 (setf sfmin var-11))
      (when var-12 (setf tol var-12))
      (when var-16 (setf ierr var-16)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17)
        (zgsvj1 jobv (f2cl-lib:int-sub m n2) (f2cl-lib:int-sub n n2) n4
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               ((+ n2 1) (f2cl-lib:int-add n2 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                               ((+ n2 1)) ((1 lwork)) cwork-%offset%)
         (f2cl-lib:array-slice sva-%data% double-float ((+ n2 1))
                               ((1 n)) sva-%offset%)
         mvl
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                               ((+ (f2cl-lib:int-mul n2 q) 1)
                                (f2cl-lib:int-add n2 1))
                               ((1 ldv) (1 *)) v-%offset%)
         ldv epsln sfmin tol 1
         (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                               ((+ n 1)) ((1 lwork)) cwork-%offset%)
         (f2cl-lib:int-sub lwork n) ierr)
```

```
  (declare
   (ignore var-1 var-2 var-4 var-6 var-7 var-9 var-14 var-15
    var-16))
  (when var-0 (setf jobv var-0))
  (when var-3 (setf n4 var-3))
  (when var-5 (setf lda var-5))
  (when var-8 (setf mvl var-8))
  (when var-10 (setf ldv var-10))
  (when var-11 (setf epsln var-11))
  (when var-12 (setf sfmin var-12))
  (when var-13 (setf tol var-13))
  (when var-17 (setf ierr var-17)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (zgsvj0 jobv (f2cl-lib:int-sub m n4) (f2cl-lib:int-sub n2 n4)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                           ((+ n4 1) (f2cl-lib:int-add n4 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ n4 1)) ((1 lwork)) cwork-%offset%)
     (f2cl-lib:array-slice sva-%data% double-float ((+ n4 1))
                           ((1 n)) sva-%offset%)
     mvl
     (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                           ((+ (f2cl-lib:int-mul n4 q) 1)
                            (f2cl-lib:int-add n4 1))
                           ((1 ldv) (1 *)) v-%offset%)
     ldv epsln sfmin tol 1
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ n 1)) ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare
   (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-13 var-14
    var-15))
  (when var-0 (setf jobv var-0))
  (when var-4 (setf lda var-4))
  (when var-7 (setf mvl var-7))
  (when var-9 (setf ldv var-9))
  (when var-10 (setf epsln var-10))
  (when var-11 (setf sfmin var-11))
  (when var-12 (setf tol var-12))
  (when var-16 (setf ierr var-16)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (zgsvj0 jobv m n4 a lda cwork sva mvl v ldv epsln sfmin tol 1
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ n 1)) ((1 lwork)) cwork-%offset%)
```

```
        (f2cl-lib:int-sub lwork n) ierr)
    (declare (ignore var-3 var-5 var-6 var-8 var-13 var-14 var-15))
    (when var-0 (setf jobv var-0))
    (when var-1 (setf m var-1))
    (when var-2 (setf n4 var-2))
    (when var-4 (setf lda var-4))
    (when var-7 (setf mvl var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf epsln var-10))
    (when var-11 (setf sfmin var-11))
    (when var-12 (setf tol var-12))
    (when var-16 (setf ierr var-16)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17)
      (zgsvj1 jobv m n2 n4 a lda cwork sva mvl v ldv epsln sfmin tol
       1
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ n 1)) ((1 lwork)) cwork-%offset%)
       (f2cl-lib:int-sub lwork n) ierr)
    (declare (ignore var-4 var-6 var-7 var-9 var-14 var-15 var-16))
    (when var-0 (setf jobv var-0))
    (when var-1 (setf m var-1))
    (when var-2 (setf n2 var-2))
    (when var-3 (setf n4 var-3))
    (when var-5 (setf lda var-5))
    (when var-8 (setf mvl var-8))
    (when var-10 (setf ldv var-10))
    (when var-11 (setf epsln var-11))
    (when var-12 (setf sfmin var-12))
    (when var-13 (setf tol var-13))
    (when var-17 (setf ierr var-17))))
 (upper
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16)
      (zgsvj0 jobv n4 n4 a lda cwork sva mvl v ldv epsln sfmin tol 2
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                             ((+ n 1)) ((1 lwork)) cwork-%offset%)
       (f2cl-lib:int-sub lwork n) ierr)
    (declare (ignore var-3 var-5 var-6 var-8 var-13 var-14 var-15))
    (when var-0 (setf jobv var-0))
    (when var-1 (setf n4 var-1))
    (when var-2 (setf n4 var-2))
    (when var-4 (setf lda var-4))
    (when var-7 (setf mvl var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf epsln var-10))
    (when var-11 (setf sfmin var-11))
    (when var-12 (setf tol var-12))
```

```
  (when var-16 (setf ierr var-16)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (zgsvj0 jobv n2 n4
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                           (1 (f2cl-lib:int-add n4 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ n4 1)) ((1 lwork)) cwork-%offset%)
     (f2cl-lib:array-slice sva-%data% double-float ((+ n4 1))
                           ((1 n)) sva-%offset%)
     mvl
     (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                           ((+ (f2cl-lib:int-mul n4 q) 1)
                            (f2cl-lib:int-add n4 1))
                           ((1 ldv) (1 *)) v-%offset%)
     ldv epsln sfmin tol 1
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ n 1)) ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-3 var-5 var-6 var-8 var-13 var-14 var-15))
  (when var-0 (setf jobv var-0))
  (when var-1 (setf n2 var-1))
  (when var-2 (setf n4 var-2))
  (when var-4 (setf lda var-4))
  (when var-7 (setf mvl var-7))
  (when var-9 (setf ldv var-9))
  (when var-10 (setf epsln var-10))
  (when var-11 (setf sfmin var-11))
  (when var-12 (setf tol var-12))
  (when var-16 (setf ierr var-16)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17)
    (zgsvj1 jobv n2 n2 n4 a lda cwork sva mvl v ldv epsln sfmin tol
     1
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                           ((+ n 1)) ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-4 var-6 var-7 var-9 var-14 var-15 var-16))
  (when var-0 (setf jobv var-0))
  (when var-1 (setf n2 var-1))
  (when var-2 (setf n2 var-2))
  (when var-3 (setf n4 var-3))
  (when var-5 (setf lda var-5))
  (when var-8 (setf mvl var-8))
  (when var-10 (setf ldv var-10))
  (when var-11 (setf epsln var-11))
```

```
        (when var-12 (setf sfmin var-12))
        (when var-13 (setf tol var-13))
        (when var-17 (setf ierr var-17)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13 var-14 var-15 var-16)
          (zgsvj0 jobv (f2cl-lib:int-add n2 n4) n4
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                 (1 (f2cl-lib:int-add n2 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                                 ((+ n2 1)) ((1 lwork)) cwork-%offset%)
           (f2cl-lib:array-slice sva-%data% double-float ((+ n2 1))
                                 ((1 n)) sva-%offset%)
           mvl
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                                 ((+ (f2cl-lib:int-mul n2 q) 1)
                                  (f2cl-lib:int-add n2 1))
                                 ((1 ldv) (1 *)) v-%offset%)
           ldv epsln sfmin tol 1
           (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex16
                                 ((+ n 1)) ((1 lwork)) cwork-%offset%)
           (f2cl-lib:int-sub lwork n) ierr)
        (declare
         (ignore var-1 var-3 var-5 var-6 var-8 var-13 var-14 var-15))
        (when var-0 (setf jobv var-0))
        (when var-2 (setf n4 var-2))
        (when var-4 (setf lda var-4))
        (when var-7 (setf mvl var-7))
        (when var-9 (setf ldv var-9))
        (when var-10 (setf epsln var-10))
        (when var-11 (setf sfmin var-11))
        (when var-12 (setf tol var-12))
        (when var-16 (setf ierr var-16)))))))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i nsweep) nil)
    (tagbody
      (setf mxaapq zero)
      (setf mxsinj zero)
      (setf iswrot 0)
      (setf notrot 0)
      (setf pskipped 0)
      (f2cl-lib:fdo (ibr 1 (f2cl-lib:int-add ibr 1))
                    ((> ibr nbl) nil)
        (tagbody
          (setf igl
                  (f2cl-lib:int-add
                   (f2cl-lib:int-mul (f2cl-lib:int-sub ibr 1) kbl) 1))
          (f2cl-lib:fdo (ir1 0 (f2cl-lib:int-add ir1 1))
```

```
                ((> ir1
                    (f2cl-lib:min0 lkahead
                                   (f2cl-lib:int-add nbl
                                                     (f2cl-lib:int-sub
                                                      ibr))))
                 nil)
        (tagbody
          (setf igl
                  (f2cl-lib:int-add igl (f2cl-lib:int-mul ir1 kbl)))
          (f2cl-lib:fdo (p igl (f2cl-lib:int-add p 1))
                        ((> p
                            (f2cl-lib:min0
                             (f2cl-lib:int-add igl kbl
                                               (f2cl-lib:int-sub 1))
                             (f2cl-lib:int-add n
                                               (f2cl-lib:int-sub
                                                1))))
                         nil)
            (tagbody
              (setf q
                      (f2cl-lib:int-sub
                       (f2cl-lib:int-add
                        (idamax
                         (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
                         (f2cl-lib:array-slice sva-%data%
                                               double-float (p)
                                               ((1 n)) sva-%offset%)
                         1)
                        p)
                       1))
              (cond
               ((/= p q)
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (zswap m
                     (f2cl-lib:array-slice a-%data%
                                           f2cl-lib:complex16 (1 p)
                                           ((1 lda) (1 *))
                                           a-%offset%)
                     1
                     (f2cl-lib:array-slice a-%data%
                                           f2cl-lib:complex16 (1 q)
                                           ((1 lda) (1 *))
                                           a-%offset%)
                     1)
                  (declare (ignore var-1 var-2 var-3 var-4))
                  (when var-0 (setf m var-0)))
                (if rsvec
                    (multiple-value-bind
                        (var-0 var-1 var-2 var-3 var-4)
                        (zswap mvl
```

```
                (f2cl-lib:array-slice v-%data%
                                      f2cl-lib:complex16
                                      (1 p) ((1 ldv) (1 *))
                                      v-%offset%)
                1
                (f2cl-lib:array-slice v-%data%
                                      f2cl-lib:complex16
                                      (1 q) ((1 ldv) (1 *))
                                      v-%offset%)
                1)
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf mvl var-0))))
        (setf temp1
                (f2cl-lib:fref sva-%data% (p) ((1 n))
                               sva-%offset%))
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                             sva-%offset%)
                (f2cl-lib:fref sva-%data% (q) ((1 n))
                               sva-%offset%))
        (setf (f2cl-lib:fref sva-%data% (q) ((1 n))
                             sva-%offset%)
                temp1)
        (setf aapq
                (f2cl-lib:fref cwork-%data% (p) ((1 lwork))
                               cwork-%offset%))
        (setf (f2cl-lib:fref cwork-%data% (p) ((1 lwork))
                             cwork-%offset%)
                (f2cl-lib:fref cwork-%data% (q) ((1 lwork))
                               cwork-%offset%))
        (setf (f2cl-lib:fref cwork-%data% (q) ((1 lwork))
                             cwork-%offset%)
                aapq)))
      (cond
       ((= ir1 0)
        (cond
         ((and (< (f2cl-lib:fref sva (p) ((1 n))) rootbig)
               (> (f2cl-lib:fref sva (p) ((1 n))) rootsfmin))
          (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                               sva-%offset%)
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2)
                    (dznrm2 m
                     (f2cl-lib:array-slice a-%data%
                                           f2cl-lib:complex16
                                           (1 p)
                                           ((1 lda) (1 *))
                                           a-%offset%)
                     1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf m var-0))
```

```
                ret-val)))
      (t (setf temp1 zero) (setf aapp one)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4)
           (zlassq m
            (f2cl-lib:array-slice a-%data%
                                  f2cl-lib:complex16 (1 p)
                                  ((1 lda) (1 *))
                                  a-%offset%)
            1 temp1 aapp)
         (declare (ignore var-1 var-2))
         (when var-0 (setf m var-0))
         (when var-3 (setf temp1 var-3))
         (when var-4 (setf aapp var-4)))
       (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                            sva-%offset%)
             (* temp1 (f2cl-lib:dsqrt aapp)))))
     (setf aapp
             (f2cl-lib:fref sva-%data% (p) ((1 n))
                            sva-%offset%)))
    (t
     (setf aapp
             (f2cl-lib:fref sva-%data% (p) ((1 n))
                            sva-%offset%))))
   (cond
    ((> aapp zero)
     (tagbody
       (setf pskipped 0)
       (f2cl-lib:fdo (q (f2cl-lib:int-add p 1)
                        (f2cl-lib:int-add q 1))
                     ((> q
                         (f2cl-lib:min0
                          (f2cl-lib:int-add igl kbl
                                            (f2cl-lib:int-sub
                                             1))
                          n))
                      nil)
         (tagbody
           (setf aaqq
                   (f2cl-lib:fref sva-%data% (q) ((1 n))
                                  sva-%offset%))
           (cond
            ((> aaqq zero) (setf aapp0 aapp)
             (cond
              ((>= aaqq one)
               (setf rotok (<= (* small aapp) aaqq))
               (cond
                ((< aapp (f2cl-lib:f2cl/ big aaqq))
                 (setf aapq
                         (/
```

```
                (/
                 (multiple-value-bind
                     (ret-val var-0 var-1 var-2
                      var-3 var-4)
                     (zdotc m
                      (f2cl-lib:array-slice
                       a-%data%
                       f2cl-lib:complex16 (1 p)
                       ((1 lda) (1 *))
                       a-%offset%)
                      1
                      (f2cl-lib:array-slice
                       a-%data%
                       f2cl-lib:complex16 (1 q)
                       ((1 lda) (1 *))
                       a-%offset%)
                      1)
                   (declare
                    (ignore var-1 var-2 var-3
                     var-4))
                   (when var-0 (setf m var-0))
                   ret-val)
                 aaqq)
                aapp)))
          (t
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4)
               (zcopy m
                (f2cl-lib:array-slice a-%data%
                                      f2cl-lib:complex16
                                      (1 p)
                                      ((1 lda)
                                       (1 *))
                                      a-%offset%)
                1
                (f2cl-lib:array-slice cwork-%data%
                                      f2cl-lib:complex16
                                      ((+ n 1))
                                      ((1 lwork))
                                      cwork-%offset%)
                1)
             (declare
              (ignore var-1 var-2 var-3 var-4))
             (when var-0 (setf m var-0)))
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5
                var-6 var-7 var-8 var-9)
               (zlascl "G" 0 0 aapp one m 1
                (f2cl-lib:array-slice cwork-%data%
                                      f2cl-lib:complex16
```

```
                                        ((+ n 1))
                                        ((1 lwork))
                                        cwork-%offset%)
                    lda ierr)
                  (declare
                   (ignore var-0 var-1 var-2 var-6
                    var-7))
                  (when var-3 (setf aapp var-3))
                  (when var-4 (setf one var-4))
                  (when var-5 (setf m var-5))
                  (when var-8 (setf lda var-8))
                  (when var-9 (setf ierr var-9)))
                (setf aapq
                        (/
                         (multiple-value-bind
                             (ret-val var-0 var-1 var-2
                              var-3 var-4)
                             (zdotc m
                              (f2cl-lib:array-slice
                               cwork-%data%
                               f2cl-lib:complex16
                               ((+ n 1)) ((1 lwork))
                               cwork-%offset%)
                              1
                              (f2cl-lib:array-slice
                               a-%data%
                               f2cl-lib:complex16 (1 q)
                               ((1 lda) (1 *))
                               a-%offset%)
                              1)
                           (declare
                            (ignore var-1 var-2 var-3
                             var-4))
                           (when var-0 (setf m var-0))
                           ret-val)
                         aaqq)))))
            (t (setf rotok (<= aapp (/ aaqq small)))
             (cond
              ((> aapp (f2cl-lib:f2cl/ small aaqq))
               (setf aapq
                       (/
                        (/
                         (multiple-value-bind
                             (ret-val var-0 var-1 var-2
                              var-3 var-4)
                             (zdotc m
                              (f2cl-lib:array-slice
                               a-%data%
                               f2cl-lib:complex16 (1 p)
                               ((1 lda) (1 *))
```

```
                 a-%offset%)
                1
                (f2cl-lib:array-slice
                 a-%data%
                 f2cl-lib:complex16 (1 q)
                 ((1 lda) (1 *))
                 a-%offset%)
                1)
             (declare
              (ignore var-1 var-2 var-3
               var-4))
             (when var-0 (setf m var-0))
             ret-val)
           aaqq)
          aapp)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (zcopy m
      (f2cl-lib:array-slice a-%data%
                            f2cl-lib:complex16
                            (1 q)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex16
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5
      var-6 var-7 var-8 var-9)
     (zlascl "G" 0 0 aaqq one m 1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex16
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
      lda ierr)
   (declare
    (ignore var-0 var-1 var-2 var-6
     var-7))
   (when var-3 (setf aaqq var-3))
   (when var-4 (setf one var-4))
```

```
        (when var-5 (setf m var-5))
        (when var-8 (setf lda var-8))
        (when var-9 (setf ierr var-9)))
      (setf aapq
              (/
               (multiple-value-bind
                   (ret-val var-0 var-1 var-2
                    var-3 var-4)
                   (zdotc m
                    (f2cl-lib:array-slice
                     a-%data%
                     f2cl-lib:complex16 (1 p)
                     ((1 lda) (1 *))
                     a-%offset%)
                    1
                    (f2cl-lib:array-slice
                     cwork-%data%
                     f2cl-lib:complex16
                     ((+ n 1)) ((1 lwork))
                     cwork-%offset%)
                    1)
                 (declare
                  (ignore var-1 var-2 var-3
                   var-4))
                 (when var-0 (setf m var-0))
                 ret-val)
               aapp))))))
(setf ompq (/ aapq (abs aapq)))
(setf aapq1 (- (abs aapq)))
(setf mxaapq
        (f2cl-lib:dmax1 mxaapq (- aapq1)))
(cond
 ((> (abs aapq1) tol)
  (cond
   ((= ir1 0) (setf notrot 0)
    (setf pskipped 0)
    (setf iswrot
            (f2cl-lib:int-add iswrot 1))))
  (cond
   (rotok (setf aqoap (/ aaqq aapp))
    (setf apoaq (/ aapp aaqq))
    (setf theta
            (/
             (* (- half)
                (abs (- aqoap apoaq)))
             aapq1))
    (cond
     ((> (abs theta) bigtheta)
      (setf t$ (/ half theta)) (setf cs one)
      (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4
     var-5 var-6)
    (zrot m
     (f2cl-lib:array-slice a-%data%
                           f2cl-lib:complex16
                           (1 p)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1
     (f2cl-lib:array-slice a-%data%
                           f2cl-lib:complex16
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1 cs
     (* (f2cl-lib:dconjg ompq) t$))
  (declare
   (ignore var-1 var-2 var-3 var-4
    var-6))
  (when var-0 (setf m var-0))
  (when var-5 (setf cs var-5)))
(cond
 (rsvec
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4
       var-5 var-6)
      (zrot mvl
       (f2cl-lib:array-slice v-%data%
                             f2cl-lib:complex16
                             (1 p)
                             ((1 ldv)
                              (1 *))
                             v-%offset%)
       1
       (f2cl-lib:array-slice v-%data%
                             f2cl-lib:complex16
                             (1 q)
                             ((1 ldv)
                              (1 *))
                             v-%offset%)
       1 cs
       (* (f2cl-lib:dconjg ompq) t$))
    (declare
     (ignore var-1 var-2 var-3 var-4
      var-6))
    (when var-0 (setf mvl var-0))
    (when var-5 (setf cs var-5)))))
(setf (f2cl-lib:fref sva-%data% (q)
```

```
                        ((1 n))
                        sva-%offset%)
        (* aaqq
           (f2cl-lib:dsqrt
            (f2cl-lib:dmax1 zero
                            (+ one
                               (* t$
                                  apoaq
                                  aapq1))))))
 (setf aapp
        (* aapp
           (f2cl-lib:dsqrt
            (f2cl-lib:dmax1 zero
                            (+ one
                               (*
                                (- t$)
                                aqoap
                                aapq1))))))
 (setf mxsinj
        (f2cl-lib:dmax1 mxsinj
                        (abs t$))))
(t
 (setf thsign
        (coerce
         (- (f2cl-lib:dsign one aapq1))
         'double-float))
 (setf t$
        (/ one
           (+ theta
              (* thsign
                 (f2cl-lib:dsqrt
                  (+ one
                     (* theta
                        theta)))))))
 (setf cs
        (f2cl-lib:dsqrt
         (/ one (+ one (* t$ t$)))))
 (setf sn (* t$ cs))
 (setf mxsinj
        (f2cl-lib:dmax1 mxsinj
                        (abs sn)))
 (setf (f2cl-lib:fref sva-%data% (q)
                        ((1 n))
                        sva-%offset%)
        (* aaqq
           (f2cl-lib:dsqrt
            (f2cl-lib:dmax1 zero
                            (+ one
                               (* t$
                                  apoaq
```

```
                                    aapq1))))))
(setf aapp
        (* aapp
           (f2cl-lib:dsqrt
            (f2cl-lib:dmax1 zero
                            (+ one
                               (*
                                (- t$)
                                aqoap
                                aapq1))))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5 var-6)
    (zrot m
     (f2cl-lib:array-slice a-%data%
                           f2cl-lib:complex16
                           (1 p)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1
     (f2cl-lib:array-slice a-%data%
                           f2cl-lib:complex16
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1 cs
     (* (f2cl-lib:dconjg ompq) sn))
  (declare
   (ignore var-1 var-2 var-3 var-4
    var-6))
  (when var-0 (setf m var-0))
  (when var-5 (setf cs var-5)))
(cond
 (rsvec
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4
       var-5 var-6)
      (zrot mvl
       (f2cl-lib:array-slice v-%data%
                             f2cl-lib:complex16
                             (1 p)
                             ((1 ldv)
                              (1 *))
                             v-%offset%)
       1
       (f2cl-lib:array-slice v-%data%
                             f2cl-lib:complex16
                             (1 q)
```

```
                                    ((1 ldv)
                                     (1 *))
                                    v-%offset%)
            1 cs
            (* (f2cl-lib:dconjg ompq) sn))
          (declare
           (ignore var-1 var-2 var-3 var-4
            var-6))
          (when var-0 (setf mvl var-0))
          (when var-5 (setf cs var-5)))))))
 (setf (f2cl-lib:fref cwork-%data% (p)
                      ((1 lwork))
                      cwork-%offset%)
          (*
           (-
            (f2cl-lib:fref cwork-%data% (q)
                           ((1 lwork))
                           cwork-%offset%))
            ompq)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (zcopy m
      (f2cl-lib:array-slice a-%data%
                            f2cl-lib:complex16
                            (1 p)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex16
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5
      var-6 var-7 var-8 var-9)
     (zlascl "G" 0 0 aapp one m 1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex16
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
      lda ierr)
   (declare
```

```
   (ignore var-0 var-1 var-2 var-6
    var-7))
  (when var-3 (setf aapp var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5
     var-6 var-7 var-8 var-9)
    (zlascl "G" 0 0 aaqq one m 1
     (f2cl-lib:array-slice a-%data%
                           f2cl-lib:complex16
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     lda ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6
    var-7))
  (when var-3 (setf aaqq var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5)
    (zaxpy m (- aapq)
     (f2cl-lib:array-slice cwork-%data%
                           f2cl-lib:complex16
                           ((+ n 1))
                           ((1 lwork))
                           cwork-%offset%)
     1
     (f2cl-lib:array-slice a-%data%
                           f2cl-lib:complex16
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3 var-4
    var-5))
  (when var-0 (setf m var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5
     var-6 var-7 var-8 var-9)
    (zlascl "G" 0 0 one aaqq m 1
```

```
        (f2cl-lib:array-slice a-%data%
                              f2cl-lib:complex16
                              (1 q)
                              ((1 lda)
                               (1 *))
                              a-%offset%)
        lda ierr)
      (declare
       (ignore var-0 var-1 var-2 var-6
        var-7))
      (when var-3 (setf one var-3))
      (when var-4 (setf aaqq var-4))
      (when var-5 (setf m var-5))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9)))
    (setf (f2cl-lib:fref sva-%data% (q)
                         ((1 n))
                         sva-%offset%)
          (* aaqq
             (f2cl-lib:dsqrt
              (f2cl-lib:dmax1 zero
                              (- one
                                 (* aapq1
                                    aapq1))))))
    (setf mxsinj
          (f2cl-lib:dmax1 mxsinj sfmin))))
  (cond
   ((<=
     (expt
      (f2cl-lib:f2cl/
       (f2cl-lib:fref sva (q) ((1 n))) aaqq)
      2)
     rooteps)
    (cond
     ((and (< aaqq rootbig)
           (> aaqq rootsfmin))
      (setf (f2cl-lib:fref sva-%data% (q)
                           ((1 n))
                           sva-%offset%)
            (multiple-value-bind
                (ret-val var-0 var-1 var-2)
                (dznrm2 m
                 (f2cl-lib:array-slice
                  a-%data%
                  f2cl-lib:complex16 (1 q)
                  ((1 lda) (1 *))
                  a-%offset%)
                 1)
               (declare
                (ignore var-1 var-2))
```

```
                (when var-0 (setf m var-0))
                ret-val)))
    (t (setf t$ zero) (setf aaqq one)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4)
         (zlassq m
          (f2cl-lib:array-slice a-%data%
                                f2cl-lib:complex16
                                (1 q)
                                ((1 lda)
                                 (1 *))
                                a-%offset%)
          1 t$ aaqq)
       (declare (ignore var-1 var-2))
       (when var-0 (setf m var-0))
       (when var-3 (setf t$ var-3))
       (when var-4 (setf aaqq var-4)))
     (setf (f2cl-lib:fref sva-%data% (q)
                          ((1 n))
                          sva-%offset%)
             (* t$
                (f2cl-lib:dsqrt aaqq)))))))
(cond
 ((<= (f2cl-lib:f2cl/ aapp aapp0) rooteps)
  (cond
   ((and (< aapp rootbig)
         (> aapp rootsfmin))
    (setf aapp
            (multiple-value-bind
                (ret-val var-0 var-1 var-2)
                (dznrm2 m
                 (f2cl-lib:array-slice
                  a-%data%
                  f2cl-lib:complex16 (1 p)
                  ((1 lda) (1 *))
                  a-%offset%)
                 1)
               (declare
                (ignore var-1 var-2))
               (when var-0 (setf m var-0))
               ret-val)))
   (t (setf t$ zero) (setf aapp one)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4)
        (zlassq m
         (f2cl-lib:array-slice a-%data%
                               f2cl-lib:complex16
                               (1 p)
                               ((1 lda)
                                (1 *))
```

```
                                                a-%offset%)
                          1 t$ aapp)
                       (declare (ignore var-1 var-2))
                       (when var-0 (setf m var-0))
                       (when var-3 (setf t$ var-3))
                       (when var-4 (setf aapp var-4)))
                     (setf aapp
                             (* t$ (f2cl-lib:dsqrt aapp)))))
                   (setf (f2cl-lib:fref sva-%data% (p)
                                        ((1 n))
                                        sva-%offset%)
                         aapp))))
               (t
                (if (= ir1 0)
                    (setf notrot
                            (f2cl-lib:int-add notrot 1)))
                (setf pskipped
                        (f2cl-lib:int-add pskipped 1)))))
             (t
              (if (= ir1 0)
                  (setf notrot
                          (f2cl-lib:int-add notrot 1)))
              (setf pskipped
                      (f2cl-lib:int-add pskipped 1))))
            (cond
             ((and (<= i swband) (> pskipped rowskip))
              (if (= ir1 0)
                  (setf aapp (- aapp)))
              (setf notrot 0) (go label2103)))
           label2002))
      label2103
       (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                            sva-%offset%)
             aapp)))
    (t
     (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                          sva-%offset%)
           aapp)
     (if (and (= ir1 0) (= aapp zero))
         (setf notrot
                 (f2cl-lib:int-sub
                  (f2cl-lib:int-add notrot
                                    (f2cl-lib:min0
                                     (f2cl-lib:int-sub
                                      (f2cl-lib:int-add
                                       igl kbl)
                                      1)
                                     n))
                  p)))))
label2001))
```

```
   label1002))
(setf igl
        (f2cl-lib:int-add
         (f2cl-lib:int-mul (f2cl-lib:int-sub ibr 1) kbl) 1))
(f2cl-lib:fdo (jbc (f2cl-lib:int-add ibr 1)
               (f2cl-lib:int-add jbc 1))
              ((> jbc nbl) nil)
  (tagbody
    (setf jgl
            (f2cl-lib:int-add
             (f2cl-lib:int-mul (f2cl-lib:int-sub jbc 1) kbl)
             1))
    (setf ijblsk 0)
    (f2cl-lib:fdo (p igl (f2cl-lib:int-add p 1))
                  ((> p
                      (f2cl-lib:min0
                       (f2cl-lib:int-add igl kbl
                                         (f2cl-lib:int-sub 1))
                       n))
                   nil)
      (tagbody
        (setf aapp
                (f2cl-lib:fref sva-%data% (p) ((1 n))
                               sva-%offset%))
        (cond
         ((> aapp zero)
          (tagbody
            (setf pskipped 0)
            (f2cl-lib:fdo (q jgl (f2cl-lib:int-add q 1))
                          ((> q
                              (f2cl-lib:min0
                               (f2cl-lib:int-add jgl kbl
                                                 (f2cl-lib:int-sub
                                                  1))
                               n))
                           nil)
              (tagbody
                (setf aaqq
                        (f2cl-lib:fref sva-%data% (q) ((1 n))
                                       sva-%offset%))
                (cond
                 ((> aaqq zero) (setf aapp0 aapp)
                  (cond
                   ((>= aaqq one)
                    (cond
                     ((>= aapp aaqq)
                      (setf rotok (<= (* small aapp) aaqq)))
                     (t (setf rotok (<= (* small aaqq) aapp))))
                    (cond
                     ((< aapp (f2cl-lib:f2cl/ big aaqq))
```

```
                    (setf aapq
                            (/
                             (/
                              (multiple-value-bind
                                  (ret-val var-0 var-1 var-2
                                   var-3 var-4)
                                  (zdotc m
                                   (f2cl-lib:array-slice
                                    a-%data%
                                    f2cl-lib:complex16 (1 p)
                                    ((1 lda) (1 *))
                                    a-%offset%)
                                   1
                                   (f2cl-lib:array-slice
                                    a-%data%
                                    f2cl-lib:complex16 (1 q)
                                    ((1 lda) (1 *))
                                    a-%offset%)
                                   1)
                                (declare
                                 (ignore var-1 var-2 var-3
                                  var-4))
                                (when var-0 (setf m var-0))
                                ret-val)
                              aaqq)
                             aapp)))
                   (t
                    (multiple-value-bind
                        (var-0 var-1 var-2 var-3 var-4)
                        (zcopy m
                         (f2cl-lib:array-slice a-%data%
                                               f2cl-lib:complex16
                                               (1 p)
                                               ((1 lda)
                                                (1 *))
                                               a-%offset%)
                         1
                         (f2cl-lib:array-slice cwork-%data%
                                               f2cl-lib:complex16
                                               ((+ n 1))
                                               ((1 lwork))
                                               cwork-%offset%)
                         1)
                      (declare
                       (ignore var-1 var-2 var-3 var-4))
                      (when var-0 (setf m var-0)))
                    (multiple-value-bind
                        (var-0 var-1 var-2 var-3 var-4 var-5
                         var-6 var-7 var-8 var-9)
                        (zlascl "G" 0 0 aapp one m 1
```

```
          (f2cl-lib:array-slice cwork-%data%
                                f2cl-lib:complex16
                                ((+ n 1))
                                ((1 lwork))
                                cwork-%offset%)
          lda ierr)
        (declare
         (ignore var-0 var-1 var-2 var-6
          var-7))
        (when var-3 (setf aapp var-3))
        (when var-4 (setf one var-4))
        (when var-5 (setf m var-5))
        (when var-8 (setf lda var-8))
        (when var-9 (setf ierr var-9)))
      (setf aapq
              (/
               (multiple-value-bind
                   (ret-val var-0 var-1 var-2
                    var-3 var-4)
                   (zdotc m
                    (f2cl-lib:array-slice
                     cwork-%data%
                     f2cl-lib:complex16
                     ((+ n 1)) ((1 lwork))
                     cwork-%offset%)
                    1
                    (f2cl-lib:array-slice
                     a-%data%
                     f2cl-lib:complex16 (1 q)
                     ((1 lda) (1 *))
                     a-%offset%)
                    1)
                 (declare
                  (ignore var-1 var-2 var-3
                   var-4))
                 (when var-0 (setf m var-0))
                 ret-val)
               aaqq)))))
    (t
     (cond
      ((>= aapp aaqq)
       (setf rotok (<= aapp (/ aaqq small))))
      (t (setf rotok (<= aaqq (/ aapp small)))))
     (cond
      ((> aapp (f2cl-lib:f2cl/ small aaqq))
       (setf aapq
               (/
                (/
                 (multiple-value-bind
                     (ret-val var-0 var-1 var-2
```

```
                var-3 var-4)
               (zdotc m
                (f2cl-lib:array-slice
                 a-%data%
                 f2cl-lib:complex16 (1 p)
                 ((1 lda) (1 *))
                 a-%offset%)
                1
                (f2cl-lib:array-slice
                 a-%data%
                 f2cl-lib:complex16 (1 q)
                 ((1 lda) (1 *))
                 a-%offset%)
                1)
              (declare
               (ignore var-1 var-2 var-3
                var-4))
              (when var-0 (setf m var-0))
              ret-val)
            aaqq)
           aapp)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (zcopy m
      (f2cl-lib:array-slice a-%data%
                            f2cl-lib:complex16
                            (1 q)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex16
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5
      var-6 var-7 var-8 var-9)
     (zlascl "G" 0 0 aaqq one m 1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex16
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
```

```
          lda ierr)
        (declare
         (ignore var-0 var-1 var-2 var-6
          var-7))
        (when var-3 (setf aaqq var-3))
        (when var-4 (setf one var-4))
        (when var-5 (setf m var-5))
        (when var-8 (setf lda var-8))
        (when var-9 (setf ierr var-9)))
      (setf aapq
              (/
               (multiple-value-bind
                   (ret-val var-0 var-1 var-2
                    var-3 var-4)
                   (zdotc m
                    (f2cl-lib:array-slice
                     a-%data%
                     f2cl-lib:complex16 (1 p)
                     ((1 lda) (1 *))
                     a-%offset%)
                    1
                    (f2cl-lib:array-slice
                     cwork-%data%
                     f2cl-lib:complex16
                     ((+ n 1)) ((1 lwork))
                     cwork-%offset%)
                    1)
                 (declare
                  (ignore var-1 var-2 var-3
                   var-4))
                 (when var-0 (setf m var-0))
                 ret-val)
               aapp))))))
(setf ompq (/ aapq (abs aapq)))
(setf aapq1 (- (abs aapq)))
(setf mxaapq
        (f2cl-lib:dmax1 mxaapq (- aapq1)))
(cond
 ((> (abs aapq1) tol) (setf notrot 0)
  (setf pskipped 0)
  (setf iswrot (f2cl-lib:int-add iswrot 1))
  (cond
   (rotok (setf aqoap (/ aaqq aapp))
    (setf apoaq (/ aapp aaqq))
    (setf theta
            (/
             (* (- half)
                (abs (- aqoap apoaq)))
             aapq1))
    (if (> aaqq aapp0)
```

```
    (setf theta (- theta)))
(cond
 ((> (abs theta) bigtheta)
  (setf t$ (/ half theta)) (setf cs one)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4
       var-5 var-6)
      (zrot m
       (f2cl-lib:array-slice a-%data%
                             f2cl-lib:complex16
                             (1 p)
                             ((1 lda)
                              (1 *))
                             a-%offset%)
       1
       (f2cl-lib:array-slice a-%data%
                             f2cl-lib:complex16
                             (1 q)
                             ((1 lda)
                              (1 *))
                             a-%offset%)
       1 cs
       (* (f2cl-lib:dconjg ompq) t$))
    (declare
     (ignore var-1 var-2 var-3 var-4
      var-6))
    (when var-0 (setf m var-0))
    (when var-5 (setf cs var-5)))
  (cond
   (rsvec
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4
         var-5 var-6)
        (zrot mvl
         (f2cl-lib:array-slice v-%data%
                               f2cl-lib:complex16
                               (1 p)
                               ((1 ldv)
                                (1 *))
                               v-%offset%)
         1
         (f2cl-lib:array-slice v-%data%
                               f2cl-lib:complex16
                               (1 q)
                               ((1 ldv)
                                (1 *))
                               v-%offset%)
         1 cs
         (* (f2cl-lib:dconjg ompq) t$))
      (declare
```

```
     (ignore var-1 var-2 var-3 var-4
      var-6))
    (when var-0 (setf mvl var-0))
    (when var-5 (setf cs var-5)))))
 (setf (f2cl-lib:fref sva-%data% (q)
                      ((1 n))
                      sva-%offset%)
        (* aaqq
           (f2cl-lib:dsqrt
            (f2cl-lib:dmax1 zero
                            (+ one
                               (* t$
                                  apoaq
                                  aapq1))))))
 (setf aapp
        (* aapp
           (f2cl-lib:dsqrt
            (f2cl-lib:dmax1 zero
                            (+ one
                               (*
                                (- t$)
                                aqoap
                                aapq1))))))
 (setf mxsinj
        (f2cl-lib:dmax1 mxsinj
                        (abs t$))))
(t
 (setf thsign
        (coerce
         (- (f2cl-lib:dsign one aapq1))
         'double-float))
 (if (> aaqq aapp0)
     (setf thsign (- thsign)))
 (setf t$
        (/ one
           (+ theta
              (* thsign
                 (f2cl-lib:dsqrt
                  (+ one
                     (* theta
                        theta)))))))
 (setf cs
        (f2cl-lib:dsqrt
         (/ one (+ one (* t$ t$)))))
 (setf sn (* t$ cs))
 (setf mxsinj
        (f2cl-lib:dmax1 mxsinj
                        (abs sn)))
 (setf (f2cl-lib:fref sva-%data% (q)
                      ((1 n))
```

```
                        sva-%offset%)
        (* aaqq
           (f2cl-lib:dsqrt
            (f2cl-lib:dmax1 zero
                            (+ one
                               (* t$
                                  apoaq
                                  aapq1))))))
(setf aapp
        (* aapp
           (f2cl-lib:dsqrt
            (f2cl-lib:dmax1 zero
                            (+ one
                               (*
                                (- t$)
                                aqoap
                                aapq1))))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5 var-6)
    (zrot m
     (f2cl-lib:array-slice a-%data%
                           f2cl-lib:complex16
                           (1 p)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1
     (f2cl-lib:array-slice a-%data%
                           f2cl-lib:complex16
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1 cs
     (* (f2cl-lib:dconjg ompq) sn))
  (declare
   (ignore var-1 var-2 var-3 var-4
    var-6))
  (when var-0 (setf m var-0))
  (when var-5 (setf cs var-5)))
(cond
 (rsvec
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4
       var-5 var-6)
      (zrot mvl
       (f2cl-lib:array-slice v-%data%
                             f2cl-lib:complex16
                             (1 p)
```

```
                                  ((1 ldv)
                                   (1 *))
                                  v-%offset%)
                1
                (f2cl-lib:array-slice v-%data%
                                  f2cl-lib:complex16
                                  (1 q)
                                  ((1 ldv)
                                   (1 *))
                                  v-%offset%)
                1 cs
                (* (f2cl-lib:dconjg ompq) sn))
            (declare
             (ignore var-1 var-2 var-3 var-4
              var-6))
            (when var-0 (setf mvl var-0))
            (when var-5 (setf cs var-5)))))))
     (setf (f2cl-lib:fref cwork-%data% (p)
                          ((1 lwork))
                          cwork-%offset%)
              (*
               (-
                (f2cl-lib:fref cwork-%data% (q)
                               ((1 lwork))
                               cwork-%offset%))
               ompq)))
    (t
     (cond
      ((> aapp aaqq)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4)
           (zcopy m
            (f2cl-lib:array-slice a-%data%
                                  f2cl-lib:complex16
                                  (1 p)
                                  ((1 lda)
                                   (1 *))
                                  a-%offset%)
            1
            (f2cl-lib:array-slice cwork-%data%
                                  f2cl-lib:complex16
                                  ((+ n 1))
                                  ((1 lwork))
                                  cwork-%offset%)
            1)
         (declare
          (ignore var-1 var-2 var-3 var-4))
         (when var-0 (setf m var-0)))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4
```

```
      var-5 var-6 var-7 var-8 var-9)
     (zlascl "G" 0 0 aapp one m 1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex16
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
      lda ierr)
   (declare
    (ignore var-0 var-1 var-2 var-6
     var-7))
   (when var-3 (setf aapp var-3))
   (when var-4 (setf one var-4))
   (when var-5 (setf m var-5))
   (when var-8 (setf lda var-8))
   (when var-9 (setf ierr var-9)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4
      var-5 var-6 var-7 var-8 var-9)
     (zlascl "G" 0 0 aaqq one m 1
      (f2cl-lib:array-slice a-%data%
                            f2cl-lib:complex16
                            (1 q)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      lda ierr)
   (declare
    (ignore var-0 var-1 var-2 var-6
     var-7))
   (when var-3 (setf aaqq var-3))
   (when var-4 (setf one var-4))
   (when var-5 (setf m var-5))
   (when var-8 (setf lda var-8))
   (when var-9 (setf ierr var-9)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4
      var-5)
     (zaxpy m (- aapq)
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex16
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
      1
      (f2cl-lib:array-slice a-%data%
                            f2cl-lib:complex16
                            (1 q)
                            ((1 lda)
                             (1 *))
```

```
                              a-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4
     var-5))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4
      var-5 var-6 var-7 var-8 var-9)
     (zlascl "G" 0 0 one aaqq m 1
      (f2cl-lib:array-slice a-%data%
                            f2cl-lib:complex16
                            (1 q)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      lda ierr)
   (declare
    (ignore var-0 var-1 var-2 var-6
     var-7))
   (when var-3 (setf one var-3))
   (when var-4 (setf aaqq var-4))
   (when var-5 (setf m var-5))
   (when var-8 (setf lda var-8))
   (when var-9 (setf ierr var-9)))
 (setf (f2cl-lib:fref sva-%data% (q)
                      ((1 n))
                      sva-%offset%)
         (* aaqq
            (f2cl-lib:dsqrt
             (f2cl-lib:dmax1 zero
                              (- one
                                 (* aapq1
                                    aapq1))))))
 (setf mxsinj
         (f2cl-lib:dmax1 mxsinj sfmin)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (zcopy m
      (f2cl-lib:array-slice a-%data%
                            f2cl-lib:complex16
                            (1 q)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex16
                            ((+ n 1))
```

```
                                ((1 lwork))
                                cwork-%offset%)
      1)
  (declare
   (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf m var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5 var-6 var-7 var-8 var-9)
    (zlascl "G" 0 0 aaqq one m 1
     (f2cl-lib:array-slice cwork-%data%
                           f2cl-lib:complex16
                           ((+ n 1))
                           ((1 lwork))
                           cwork-%offset%)
     lda ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6
    var-7))
  (when var-3 (setf aaqq var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5 var-6 var-7 var-8 var-9)
    (zlascl "G" 0 0 aapp one m 1
     (f2cl-lib:array-slice a-%data%
                           f2cl-lib:complex16
                           (1 p)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     lda ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6
    var-7))
  (when var-3 (setf aapp var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5)
    (zaxpy m (- (f2cl-lib:dconjg aapq))
     (f2cl-lib:array-slice cwork-%data%
                           f2cl-lib:complex16
                           ((+ n 1))
```

```
                                  ((1 lwork))
                                  cwork-%offset%)
                1
                (f2cl-lib:array-slice a-%data%
                                  f2cl-lib:complex16
                                  (1 p)
                                  ((1 lda)
                                   (1 *))
                                  a-%offset%)
                1)
            (declare
             (ignore var-1 var-2 var-3 var-4
              var-5))
            (when var-0 (setf m var-0)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4
             var-5 var-6 var-7 var-8 var-9)
            (zlascl "G" 0 0 one aapp m 1
             (f2cl-lib:array-slice a-%data%
                                  f2cl-lib:complex16
                                  (1 p)
                                  ((1 lda)
                                   (1 *))
                                  a-%offset%)
             lda ierr)
            (declare
             (ignore var-0 var-1 var-2 var-6
              var-7))
            (when var-3 (setf one var-3))
            (when var-4 (setf aapp var-4))
            (when var-5 (setf m var-5))
            (when var-8 (setf lda var-8))
            (when var-9 (setf ierr var-9)))
        (setf (f2cl-lib:fref sva-%data% (p)
                             ((1 n))
                             sva-%offset%)
                (* aapp
                   (f2cl-lib:dsqrt
                    (f2cl-lib:dmax1 zero
                                    (- one
                                       (* aapq1
                                          aapq1))))))
        (setf mxsinj
                (f2cl-lib:dmax1 mxsinj
                                sfmin))))))
    (cond
     ((<=
       (expt
        (f2cl-lib:f2cl/
         (f2cl-lib:fref sva (q) ((1 n))) aaqq)
```

```
    2)
   rooteps)
  (cond
   ((and (< aaqq rootbig)
         (> aaqq rootsfmin))
    (setf (f2cl-lib:fref sva-%data% (q)
                         ((1 n))
                         sva-%offset%)
          (multiple-value-bind
              (ret-val var-0 var-1 var-2)
              (dznrm2 m
               (f2cl-lib:array-slice
                a-%data%
                f2cl-lib:complex16 (1 q)
                ((1 lda) (1 *))
                a-%offset%)
               1)
             (declare
              (ignore var-1 var-2))
             (when var-0 (setf m var-0))
             ret-val)))
   (t (setf t$ zero) (setf aaqq one)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4)
        (zlassq m
         (f2cl-lib:array-slice a-%data%
                               f2cl-lib:complex16
                               (1 q)
                               ((1 lda)
                                (1 *))
                               a-%offset%)
         1 t$ aaqq)
      (declare (ignore var-1 var-2))
      (when var-0 (setf m var-0))
      (when var-3 (setf t$ var-3))
      (when var-4 (setf aaqq var-4)))
    (setf (f2cl-lib:fref sva-%data% (q)
                         ((1 n))
                         sva-%offset%)
          (* t$
             (f2cl-lib:dsqrt aaqq)))))))
(cond
 ((<= (expt (f2cl-lib:f2cl/ aapp aapp0) 2)
      rooteps)
  (cond
   ((and (< aapp rootbig)
         (> aapp rootsfmin))
    (setf aapp
          (multiple-value-bind
              (ret-val var-0 var-1 var-2)
```

```
                        (dznrm2 m
                         (f2cl-lib:array-slice
                          a-%data%
                          f2cl-lib:complex16 (1 p)
                          ((1 lda) (1 *))
                          a-%offset%)
                         1)
                       (declare
                        (ignore var-1 var-2))
                       (when var-0 (setf m var-0))
                       ret-val)))
        (t (setf t$ zero) (setf aapp one)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4)
             (zlassq m
              (f2cl-lib:array-slice a-%data%
                                    f2cl-lib:complex16
                                    (1 p)
                                    ((1 lda)
                                     (1 *))
                                    a-%offset%)
              1 t$ aapp)
           (declare (ignore var-1 var-2))
           (when var-0 (setf m var-0))
           (when var-3 (setf t$ var-3))
           (when var-4 (setf aapp var-4)))
         (setf aapp
                 (* t$ (f2cl-lib:dsqrt aapp)))))
       (setf (f2cl-lib:fref sva-%data% (p)
                            ((1 n))
                            sva-%offset%)
              aapp))))
    (t (setf notrot (f2cl-lib:int-add notrot 1))
     (setf pskipped
             (f2cl-lib:int-add pskipped 1))
     (setf ijblsk
             (f2cl-lib:int-add ijblsk 1)))))
  (t (setf notrot (f2cl-lib:int-add notrot 1))
   (setf pskipped (f2cl-lib:int-add pskipped 1))
   (setf ijblsk (f2cl-lib:int-add ijblsk 1))))
 (cond
  ((and (<= i swband) (>= ijblsk blskip))
   (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                        sva-%offset%)
           aapp)
   (setf notrot 0) (go label2011)))
 (cond
  ((and (<= i swband) (> pskipped rowskip))
   (setf aapp (- aapp)) (setf notrot 0)
   (go label2203)))
```

```
                    label2200))
                 label2203
                  (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                                       sva-%offset%)
                        aapp)))
               (t
                (if (= aapp zero)
                    (setf notrot
                            (f2cl-lib:int-add
                             (f2cl-lib:int-sub
                              (f2cl-lib:int-add notrot
                                                (f2cl-lib:min0
                                                 (f2cl-lib:int-sub
                                                  (f2cl-lib:int-add
                                                   jgl kbl)
                                                  1)
                                                 n))
                              jgl)
                             1)))
                (if (< aapp zero)
                    (setf notrot 0))))
            label2100))
        label2010))
    label2011
     (f2cl-lib:fdo (p igl (f2cl-lib:int-add p 1))
                   ((> p
                       (f2cl-lib:min0
                        (f2cl-lib:int-add igl kbl
                                          (f2cl-lib:int-sub 1))
                        n))
                    nil)
       (tagbody
         (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                 (abs
                  (f2cl-lib:fref sva-%data% (p) ((1 n))
                                 sva-%offset%)))
        label2012))
    label2000))
  (cond
   ((and (< (f2cl-lib:fref sva (n) ((1 n))) rootbig)
         (> (f2cl-lib:fref sva (n) ((1 n))) rootsfmin))
    (setf (f2cl-lib:fref sva-%data% (n) ((1 n)) sva-%offset%)
          (multiple-value-bind (ret-val var-0 var-1 var-2)
              (dznrm2 m
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                     (1 n) ((1 lda) (1 *))
                                     a-%offset%)
               1)
            (declare (ignore var-1 var-2))
            (when var-0 (setf m var-0))
```

```
                ret-val)))
       (t (setf t$ zero) (setf aapp one)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (zlassq m
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 n)
                                   ((1 lda) (1 *)) a-%offset%)
             1 t$ aapp)
          (declare (ignore var-1 var-2))
          (when var-0 (setf m var-0))
          (when var-3 (setf t$ var-3))
          (when var-4 (setf aapp var-4)))
        (setf (f2cl-lib:fref sva-%data% (n) ((1 n)) sva-%offset%)
                (* t$ (f2cl-lib:dsqrt aapp)))))
      (if (and (< i swband) (or (<= mxaapq roottol) (<= iswrot n)))
          (setf swband i))
      (cond
       ((and (> i (f2cl-lib:int-add swband 1))
             (< mxaapq (* (f2cl-lib:dsqrt (f2cl-lib:dfloat n)) tol))
             (< (* (f2cl-lib:dfloat n) mxaapq mxsinj) tol))
        (go label1994)))
      (if (>= notrot emptsw)
          (go label1994))
     label1993))
  (setf info (f2cl-lib:int-sub nsweep 1))
  (go label1995)
 label1994
  (setf info 0)
 label1995
  (setf n2 0)
  (setf n4 0)
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (setf q
              (f2cl-lib:int-sub
               (f2cl-lib:int-add
                (idamax (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
                 (f2cl-lib:array-slice sva-%data% double-float (p)
                                       ((1 n)) sva-%offset%)
                 1)
                p)
               1))
      (cond
       ((/= p q)
        (setf temp1 (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%))
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%))
        (setf (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%) temp1)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (zswap m
```

```
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 p)
                                      ((1 lda) (1 *)) a-%offset%)
                1
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 q)
                                      ((1 lda) (1 *)) a-%offset%)
                1)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf m var-0)))
        (if rsvec
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (zswap mvl
                 (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (1 p)
                                       ((1 ldv) (1 *)) v-%offset%)
                 1
                 (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (1 q)
                                       ((1 ldv) (1 *)) v-%offset%)
                 1)
               (declare (ignore var-1 var-2 var-3 var-4))
               (when var-0 (setf mvl var-0))))))
      (cond
       ((/= (f2cl-lib:fref sva (p) ((1 n))) zero)
        (setf n4 (f2cl-lib:int-add n4 1))
        (if (>
             (* (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%) skl)
             sfmin)
            (setf n2 (f2cl-lib:int-add n2 1)))))
    label5991))
  (cond
   ((/= (f2cl-lib:fref sva (n) ((1 n))) zero)
    (setf n4 (f2cl-lib:int-add n4 1))
    (if (> (* (f2cl-lib:fref sva-%data% (n) ((1 n)) sva-%offset%) skl)
           sfmin)
        (setf n2 (f2cl-lib:int-add n2 1)))))
  (cond
   ((or lsvec uctol)
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n2) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (zdscal m
             (/ one (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%))
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 p)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3))
          (when var-0 (setf m var-0)))
      label1998))))
  (cond
   (rsvec
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
```

```
                ((> p n) nil)
    (tagbody
      (setf temp1
              (/ one
                 (multiple-value-bind (ret-val var-0 var-1 var-2)
                     (dznrm2 mvl
                      (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                                            (1 p) ((1 ldv) (1 *))
                                            v-%offset%)
                      1)
                    (declare (ignore var-1 var-2))
                    (when var-0 (setf mvl var-0))
                    ret-val)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (zdscal mvl temp1
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (1 p)
                                 ((1 ldv) (1 *)) v-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf mvl var-0))
        (when var-1 (setf temp1 var-1)))
     label2399))))
(cond
 ((or
   (and (> skl one)
        (< (f2cl-lib:fref sva (1) ((1 n))) (f2cl-lib:f2cl/ big skl)))
   (and (< skl one)
        (>
         (f2cl-lib:fref sva
                        ((max (the f2cl-lib:integer4 n2)
                              (the f2cl-lib:integer4 1)))
                        ((1 n)))
         (f2cl-lib:f2cl/ sfmin skl))))
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
              (* skl
                 (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)))
     label2400))
  (setf skl one)))
(setf (f2cl-lib:fref rwork-%data% (1) ((1 lrwork)) rwork-%offset%) skl)
(setf (f2cl-lib:fref rwork-%data% (2) ((1 lrwork)) rwork-%offset%)
        (coerce (f2cl-lib:dfloat n4) 'double-float))
(setf (f2cl-lib:fref rwork-%data% (3) ((1 lrwork)) rwork-%offset%)
        (coerce (f2cl-lib:dfloat n2) 'double-float))
(setf (f2cl-lib:fref rwork-%data% (4) ((1 lrwork)) rwork-%offset%)
        (coerce (f2cl-lib:dfloat i) 'double-float))
(setf (f2cl-lib:fref rwork-%data% (5) ((1 lrwork)) rwork-%offset%)
        mxaapq)
```

```
        (setf (f2cl-lib:fref rwork-%data% (6) ((1 lrwork)) rwork-%offset%)
                mxsinj)
        (go end_label)
       end_label
        (return
         (values joba jobu jobv m n nil lda nil nil nil ldv nil nil nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgesvj
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::joba
                                              fortran-to-lisp::jobu
                                              fortran-to-lisp::jobv
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::ldv nil nil nil
                                              nil fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

Chapter 33

LAPACK - General Matrices, Comp. Routines, Real

33.1 sgebak

33.1.1 Axiom unit tests

— sgebak.input —

```
)set break resume
)sys rm -f sgebak.output
)spool sgebak.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.1.2 Axiom help page

— sgebak.help —

```
==========================================================================
sgebak examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgebak (JOB, SIDE, N, ILO, IHI, SCALE, M, V, LDV, INFO)
           SGEBAK

Function/Subroutine Documentation
   subroutine sgebak (character JOB, character SIDE, integer N, integer ILO,
       integer IHI, real, dimension( * ) SCALE, integer M, real, dimension(
       ldv, * ) V, integer LDV, integer INFO)
       SGEBAK

Purpose:
   SGEBAK forms the right or left eigenvectors of a real general matrix
   by backward transformation on the computed eigenvectors of the
   balanced matrix output by SGEBAL.

Parameters:

 JOB
   JOB is CHARACTER*1
   Specifies the type of backward transformation required:
   = 'N', do nothing, return immediately;
   = 'P', do backward transformation for permutation only;
   = 'S', do backward transformation for scaling only;
   = 'B', do backward transformations for both permutation and scaling.
   JOB must be the same as the argument JOB supplied to SGEBAL.

 SIDE
   SIDE is CHARACTER*1
   = 'R':  V contains right eigenvectors;
   = 'L':  V contains left eigenvectors.

 N
   N is INTEGER
   The number of rows of the matrix V.  N >= 0.

 ILO
   ILO is INTEGER

 IHI
   IHI is INTEGER
   The integers ILO and IHI determined by SGEBAL.
   1 <= ILO <= IHI <= N, if N > 0; ILO=1 and IHI=0, if N=0.

 SCALE
```

```
  SCALE is REAL array, dimension (N)
  Details of the permutation and scaling factors, as returned by SGEBAL.

M
  M is INTEGER
  The number of columns of the matrix V.  M >= 0.

V
  V is REAL array, dimension (LDV,M)
  On entry, the matrix of right or left eigenvectors to be
  transformed, as returned by SHSEIN or STREVC.
  On exit, V is overwritten by the transformed eigenvectors.

LDV
  LDV is INTEGER
  The leading dimension of the array V. LDV >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

33.1.3 fortran code

— sgebak.f —

```
*  =====================================================================
      SUBROUTINE SGEBAK( JOB, SIDE, N, ILO, IHI, SCALE, M, V, LDV,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOB, SIDE
      INTEGER            IHI, ILO, INFO, LDV, M, N
*     ..
*     .. Array Arguments ..
      REAL               V( LDV, * ), SCALE( * )
*     ..
*
*  =====================================================================
*
```

```
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LEFTV, RIGHTV
      INTEGER            I, II, K
      REAL               S
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SSCAL, SSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Decode and Test the input parameters
*
      RIGHTV = LSAME( SIDE, 'R' )
      LEFTV = LSAME( SIDE, 'L' )
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.RIGHTV .AND. .NOT.LEFTV ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -5
      ELSE IF( M.LT.0 ) THEN
         INFO = -7
      ELSE IF( LDV.LT.MAX( 1, N ) ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEBAK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
```

```
      IF( N.EQ.0 )
     $   RETURN
      IF( M.EQ.0 )
     $   RETURN
      IF( LSAME( JOB, 'N' ) )
     $   RETURN
*
      IF( ILO.EQ.IHI )
     $   GO TO 30
*
*     Backward balance
*
      IF( LSAME( JOB, 'S' ) .OR. LSAME( JOB, 'B' ) ) THEN
*
         IF( RIGHTV ) THEN
            DO 10 I = ILO, IHI
               S = SCALE( I )
               CALL SSCAL( M, S, V( I, 1 ), LDV )
   10       CONTINUE
         END IF
*
         IF( LEFTV ) THEN
            DO 20 I = ILO, IHI
               S = ONE / SCALE( I )
               CALL SSCAL( M, S, V( I, 1 ), LDV )
   20       CONTINUE
         END IF
*
      END IF
*
*     Backward permutation
*
*     For  I = ILO-1 step -1 until 1,
*              IHI+1 step 1 until N do --
*
   30 CONTINUE
      IF( LSAME( JOB, 'P' ) .OR. LSAME( JOB, 'B' ) ) THEN
         IF( RIGHTV ) THEN
            DO 40 II = 1, N
               I = II
               IF( I.GE.ILO .AND. I.LE.IHI )
     $            GO TO 40
               IF( I.LT.ILO )
     $            I = ILO - II
               K = SCALE( I )
               IF( K.EQ.I )
     $            GO TO 40
               CALL SSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   40       CONTINUE
         END IF
```

```
*
         IF( LEFTV ) THEN
            DO 50 II = 1, N
               I = II
               IF( I.GE.ILO .AND. I.LE.IHI )
     $            GO TO 50
               IF( I.LT.ILO )
     $            I = ILO - II
               K = SCALE( I )
               IF( K.EQ.I )
     $            GO TO 50
               CALL SSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   50       CONTINUE
         END IF
      END IF
*
      RETURN
*
*     End of SGEBAK
*
      END
```

———-

33.1.4 lisp code

— **sgebak.lisp** —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun sgebak (job side n ilo ihi scale m v ldv info)
    (declare (type (array single-float (*)) v scale)
             (type (f2cl-lib:integer4) info ldv m ihi ilo n)
             (type (string *) side job))
    (f2cl-lib:with-multi-array-data
        ((job character job-%data% job-%offset%)
         (side character side-%data% side-%offset%)
         (scale single-float scale-%data% scale-%offset%)
         (v single-float v-%data% v-%offset%))
      (prog ((s 0.0) (i 0) (ii 0) (k 0) (leftv nil) (rightv nil))
        (declare (type (single-float) s)
                 (type (f2cl-lib:integer4) i ii k)
                 (type f2cl-lib:logical leftv rightv))
        (setf rightv
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame side "R")
                  (declare (ignore var-1))
```

```
          (when var-0 (setf side var-0))
          ret-val))
(setf leftv
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "L")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val))
(setf info 0)
(cond
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "N")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "P")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "S")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "B")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)))
  (setf info -1))
 ((and (not rightv) (not leftv)) (setf info -2))
 ((< n 0) (setf info -3))
 ((or (< ilo 1)
      (> ilo
         (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n))))
  (setf info -4))
 ((or
   (< ihi (min (the f2cl-lib:integer4 ilo) (the f2cl-lib:integer4 n)))
   (> ihi n))
  (setf info -5))
 ((< m 0) (setf info -7))
 ((< ldv (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9)))
(cond
```

```
 ((/= info 0) (xerbla "SGEBAK" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(if (= m 0)
    (go end_label))
(if (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "N")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)
    (go end_label))
(if (= ilo ihi)
    (go label30))
(cond
 ((or
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame job "S")
     (declare (ignore var-1))
     (when var-0 (setf job var-0))
     ret-val)
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame job "B")
     (declare (ignore var-1))
     (when var-0 (setf job var-0))
     ret-val))
  (cond
   (rightv
    (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                  ((> i ihi) nil)
      (tagbody
        (setf s
                (f2cl-lib:fref scale-%data% (i) ((1 *))
                               scale-%offset%))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal m s
             (f2cl-lib:array-slice v-%data% single-float (i 1)
                                   ((1 ldv) (1 *)) v-%offset%)
             ldv)
          (declare (ignore var-2))
          (when var-0 (setf m var-0))
          (when var-1 (setf s var-1))
          (when var-3 (setf ldv var-3)))
       label10))))
  (cond
   (leftv
    (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                  ((> i ihi) nil)
      (tagbody
        (setf s
```

```
                    (/ one
                       (f2cl-lib:fref scale-%data% (i) ((1 *))
                                      scale-%offset%)))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (sscal m s
                 (f2cl-lib:array-slice v-%data% single-float (i 1)
                                       ((1 ldv) (1 *)) v-%offset%)
                 ldv)
              (declare (ignore var-2))
              (when var-0 (setf m var-0))
              (when var-1 (setf s var-1))
              (when var-3 (setf ldv var-3)))
          label20))))))
  label30
   (cond
    ((or
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame job "P")
        (declare (ignore var-1))
        (when var-0 (setf job var-0))
        ret-val)
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame job "B")
        (declare (ignore var-1))
        (when var-0 (setf job var-0))
        ret-val))
     (cond
      (rightv
       (f2cl-lib:fdo (ii 1 (f2cl-lib:int-add ii 1))
                     ((> ii n) nil)
         (tagbody
           (setf i ii)
           (if (and (>= i ilo) (<= i ihi))
               (go label40))
           (if (< i ilo)
               (setf i (f2cl-lib:int-sub ilo ii)))
           (setf k
                   (f2cl-lib:int
                    (f2cl-lib:fref scale-%data% (i) ((1 *))
                                   scale-%offset%)))
           (if (= k i)
               (go label40))
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
               (sswap m
                (f2cl-lib:array-slice v-%data% single-float (i 1)
                                      ((1 ldv) (1 *)) v-%offset%)
                ldv
                (f2cl-lib:array-slice v-%data% single-float (k 1)
                                      ((1 ldv) (1 *)) v-%offset%)
                ldv)
```

```
                (declare (ignore var-1 var-3))
                (when var-0 (setf m var-0))
                (when var-2 (setf ldv var-2))
                (when var-4 (setf ldv var-4)))
            label40))))
        (cond
         (leftv
          (f2cl-lib:fdo (ii 1 (f2cl-lib:int-add ii 1))
                        ((> ii n) nil)
            (tagbody
              (setf i ii)
              (if (and (>= i ilo) (<= i ihi))
                  (go label50))
              (if (< i ilo)
                  (setf i (f2cl-lib:int-sub ilo ii)))
              (setf k
                      (f2cl-lib:int
                       (f2cl-lib:fref scale-%data% (i) ((1 *))
                                      scale-%offset%)))
              (if (= k i)
                  (go label50))
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (sswap m
                   (f2cl-lib:array-slice v-%data% single-float (i 1)
                                         ((1 ldv) (1 *)) v-%offset%)
                   ldv
                   (f2cl-lib:array-slice v-%data% single-float (k 1)
                                         ((1 ldv) (1 *)) v-%offset%)
                   ldv)
                (declare (ignore var-1 var-3))
                (when var-0 (setf m var-0))
                (when var-2 (setf ldv var-2))
                (when var-4 (setf ldv var-4)))
            label50))))))
      (go end_label)
     end_label
      (return (values job side nil nil nil nil m nil ldv info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgebak
                 fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((string) (string)
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (array single-float (*))
                                            (fortran-to-lisp::integer4)
```

```
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::job
  fortran-to-lisp::side nil nil nil
  nil fortran-to-lisp::m nil
  fortran-to-lisp::ldv
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

33.2 sgebal

33.2.1 Axiom unit tests

— **sgebal.input** —

```
)set break resume
)sys rm -f sgebal.output
)spool sgebal.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.2.2 Axiom help page

— **sgebal.help** —

```
=====================================================================
sgebal examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine sgebal (JOB, N, A, LDA, ILO, IHI, SCALE, INFO)
           SGEBAL

Function/Subroutine Documentation
   subroutine sgebal (character JOB, integer N, real, dimension( lda, * ) A,
       integer LDA, integer ILO, integer IHI, real, dimension( * ) SCALE,
       integer INFO)
       SGEBAL

Purpose:
   SGEBAL balances a general real matrix A.  This involves, first,
   permuting A by a similarity transformation to isolate eigenvalues
   in the first 1 to ILO-1 and last IHI+1 to N elements on the
   diagonal; and second, applying a diagonal similarity transformation
   to rows and columns ILO to IHI to make the rows and columns as
   close in norm as possible.  Both steps are optional.

   Balancing may reduce the 1-norm of the matrix, and improve the
   accuracy of the computed eigenvalues and/or eigenvectors.

Parameters:

 JOB
   JOB is CHARACTER*1
   Specifies the operations to be performed on A:
   = 'N':  none:  simply set ILO = 1, IHI = N, SCALE(I) = 1.0
      for i = 1,...,N;
   = 'P':  permute only;
   = 'S':  scale only;
   = 'B':  both permute and scale.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the input matrix A.
   On exit,  A is overwritten by the balanced matrix.
   If JOB = 'N', A is not referenced.
   See Further Details.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 ILO
```

```
  ILO is INTEGER

 IHI
  IHI is INTEGER
  ILO and IHI are set to integers such that on exit
  A(i,j) = 0 if i > j and j = 1,...,ILO-1 or I = IHI+1,...,N.
  If JOB = 'N' or 'S', ILO = 1 and IHI = N.

 SCALE
  SCALE is REAL array, dimension (N)
  Details of the permutations and scaling factors applied to
  A.  If P(j) is the index of the row and column interchanged
  with row and column j and D(j) is the scaling factor
  applied to row and column j, then
  SCALE(j) = P(j)    for j = 1,...,ILO-1
           = D(j)    for j = ILO,...,IHI
           = P(j)    for j = IHI+1,...,N.
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

 INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.

Further Details:

  The permutations consist of row and column interchanges which put
  the matrix in the form

                         ( T1   X   Y  )
               P A P = (  0   B   Z  )
                         (  0   0   T2 )

  where T1 and T2 are upper triangular matrices whose eigenvalues lie
  along the diagonal.  The column indices ILO and IHI mark the starting
  and ending columns of the submatrix B. Balancing consists of applying
  a diagonal similarity transformation inv(D) * B * D to make the
  1-norms of each row of B and its corresponding column nearly equal.
  The output matrix is

               ( T1     X*D          Y    )
               (  0  inv(D)*B*D  inv(D)*Z ).
               (  0      0           T2   )

  Information about the permutations P and the diagonal matrix D is
  returned in the vector SCALE.

  This subroutine is based on the EISPACK routine BALANC.
```

———-

33.2.3 fortran code

— sgebal.f —

```
*  =====================================================================
      SUBROUTINE SGEBAL( JOB, N, A, LDA, ILO, IHI, SCALE, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOB
      INTEGER            IHI, ILO, INFO, LDA, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), SCALE( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               SCLFAC
      PARAMETER          ( SCLFAC = 2.0E+0 )
      REAL               FACTOR
      PARAMETER          ( FACTOR = 0.95E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOCONV
      INTEGER            I, ICA, IEXC, IRA, J, K, L, M
      REAL               C, CA, F, G, R, RA, S, SFMAX1, SFMAX2, SFMIN1,
     $                   SFMIN2
*     ..
*     .. External Functions ..
      LOGICAL            SISNAN, LSAME
      INTEGER            ISAMAX
      REAL               SLAMCH, SNRM2
      EXTERNAL           SISNAN, LSAME, ISAMAX, SLAMCH, SNRM2
*     ..
*     .. External Subroutines ..
      EXTERNAL           SSCAL, SSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          ABS, MAX, MIN
*
*     Test the input parameters
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEBAL', -INFO )
         RETURN
      END IF
*
      K = 1
      L = N
*
      IF( N.EQ.0 )
     $   GO TO 210
*
      IF( LSAME( JOB, 'N' ) ) THEN
         DO 10 I = 1, N
            SCALE( I ) = ONE
   10    CONTINUE
         GO TO 210
      END IF
*
      IF( LSAME( JOB, 'S' ) )
     $   GO TO 120
*
*     Permutation to isolate eigenvalues if possible
*
      GO TO 50
*
*     Row and column exchange.
*
   20 CONTINUE
      SCALE( M ) = J
      IF( J.EQ.M )
     $   GO TO 30
*
      CALL SSWAP( L, A( 1, J ), 1, A( 1, M ), 1 )
      CALL SSWAP( N-K+1, A( J, K ), LDA, A( M, K ), LDA )
*
   30 CONTINUE
      GO TO ( 40, 80 )IEXC
```

```
*
*     Search for rows isolating an eigenvalue and push them down.
*
   40 CONTINUE
      IF( L.EQ.1 )
     $   GO TO 210
      L = L - 1
*
   50 CONTINUE
      DO 70 J = L, 1, -1
*
         DO 60 I = 1, L
            IF( I.EQ.J )
     $         GO TO 60
            IF( A( J, I ).NE.ZERO )
     $         GO TO 70
   60    CONTINUE
*
         M = L
         IEXC = 1
         GO TO 20
   70 CONTINUE
*
      GO TO 90
*
*     Search for columns isolating an eigenvalue and push them left.
*
   80 CONTINUE
      K = K + 1
*
   90 CONTINUE
      DO 110 J = K, L
*
         DO 100 I = K, L
            IF( I.EQ.J )
     $         GO TO 100
            IF( A( I, J ).NE.ZERO )
     $         GO TO 110
  100    CONTINUE
*
         M = K
         IEXC = 2
         GO TO 20
  110 CONTINUE
*
  120 CONTINUE
      DO 130 I = K, L
         SCALE( I ) = ONE
  130 CONTINUE
*
```

```
      IF( LSAME( JOB, 'P' ) )
     $   GO TO 210
*
*     Balance the submatrix in rows K to L.
*
*     Iterative loop for norm reduction
*
      SFMIN1 = SLAMCH( 'S' ) / SLAMCH( 'P' )
      SFMAX1 = ONE / SFMIN1
      SFMIN2 = SFMIN1*SCLFAC
      SFMAX2 = ONE / SFMIN2
  140 CONTINUE
      NOCONV = .FALSE.
*
      DO 200 I = K, L
*
         C = SNRM2( L-K+1, A( K, I ), 1 )
         R = SNRM2( L-K+1, A( I, K ), LDA )
         ICA = ISAMAX( L, A( 1, I ), 1 )
         CA = ABS( A( ICA, I ) )
         IRA = ISAMAX( N-K+1, A( I, K ), LDA )
         RA = ABS( A( I, IRA+K-1 ) )
*
*        Guard against zero C or R due to underflow.
*
         IF( C.EQ.ZERO .OR. R.EQ.ZERO )
     $      GO TO 200
         G = R / SCLFAC
         F = ONE
         S = C + R
  160    CONTINUE
         IF( C.GE.G .OR. MAX( F, C, CA ).GE.SFMAX2 .OR.
     $       MIN( R, G, RA ).LE.SFMIN2 )GO TO 170
         F = F*SCLFAC
         C = C*SCLFAC
         CA = CA*SCLFAC
         R = R / SCLFAC
         G = G / SCLFAC
         RA = RA / SCLFAC
         GO TO 160
*
  170    CONTINUE
         G = C / SCLFAC
  180    CONTINUE
         IF( G.LT.R .OR. MAX( R, RA ).GE.SFMAX2 .OR.
     $       MIN( F, C, G, CA ).LE.SFMIN2 )GO TO 190
            IF( SISNAN( C+F+CA+R+G+RA ) ) THEN
*
*           Exit if NaN to avoid infinite loop
*
```

```
            INFO = -3
            CALL XERBLA( 'SGEBAL', -INFO )
            RETURN
         END IF
         F = F / SCLFAC
         C = C / SCLFAC
         G = G / SCLFAC
         CA = CA / SCLFAC
         R = R*SCLFAC
         RA = RA*SCLFAC
         GO TO 180
*
*        Now balance.
*
  190    CONTINUE
         IF( ( C+R ).GE.FACTOR*S )
     $      GO TO 200
         IF( F.LT.ONE .AND. SCALE( I ).LT.ONE ) THEN
            IF( F*SCALE( I ).LE.SFMIN1 )
     $         GO TO 200
         END IF
         IF( F.GT.ONE .AND. SCALE( I ).GT.ONE ) THEN
            IF( SCALE( I ).GE.SFMAX1 / F )
     $         GO TO 200
         END IF
         G = ONE / F
         SCALE( I ) = SCALE( I )*F
         NOCONV = .TRUE.
*
         CALL SSCAL( N-K+1, G, A( I, K ), LDA )
         CALL SSCAL( L, F, A( 1, I ), 1 )
*
  200 CONTINUE
*
      IF( NOCONV )
     $   GO TO 140
*
  210 CONTINUE
      ILO = K
      IHI = L
*
      RETURN
*
*     End of SGEBAL
*
      END
```

———-

33.2.4 lisp code

— sgebal.lisp —

```
(let* ((zero 0.0) (one 1.0) (sclfac 2.0) (factor 0.95))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) sclfac)
           (type (single-float 0.95 0.95) factor)
           (ignorable zero one sclfac factor))
  (defun sgebal (job n a lda ilo ihi scale info)
    (declare (type (array single-float (*)) scale a)
             (type (f2cl-lib:integer4) info ihi ilo lda n)
             (type (string *) job))
    (f2cl-lib:with-multi-array-data
        ((job character job-%data% job-%offset%)
         (a single-float a-%data% a-%offset%)
         (scale single-float scale-%data% scale-%offset%))
      (prog ((c 0.0) (ca 0.0) (f 0.0) (g 0.0) (r 0.0) (ra 0.0) (s 0.0)
             (sfmax1 0.0) (sfmax2 0.0) (sfmin1 0.0) (sfmin2 0.0) (i 0) (ica 0)
             (iexc 0) (ira 0) (j 0) (k 0) (l 0) (m 0) (noconv nil))
        (declare
         (type (single-float) c ca f g r ra s sfmax1 sfmax2 sfmin1 sfmin2)
         (type (f2cl-lib:integer4) i ica iexc ira j k l m)
         (type f2cl-lib:logical noconv))
        (setf info 0)
        (cond
         ((and
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame job "N")
              (declare (ignore var-1))
              (when var-0 (setf job var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame job "P")
              (declare (ignore var-1))
              (when var-0 (setf job var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame job "S")
              (declare (ignore var-1))
              (when var-0 (setf job var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame job "B")
```

```
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4)))
(cond
 ((/= info 0) (xerbla "SGEBAL" (f2cl-lib:int-sub info))
  (go end_label)))
(setf k 1)
(setf l n)
(if (= n 0)
    (go label210))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame job "N")
    (declare (ignore var-1))
    (when var-0 (setf job var-0))
    ret-val)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%)
              one)
     label10))
  (go label210)))
(if (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "S")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)
    (go label120))
(go label50)
label20
(setf (f2cl-lib:fref scale-%data% (m) ((1 *)) scale-%offset%)
        (coerce (the f2cl-lib:integer4 j) 'single-float))
(if (= j m)
    (go label30))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (sswap l
     (f2cl-lib:array-slice a-%data% single-float (1 j) ((1 lda) (1 *))
                           a-%offset%)
     1
     (f2cl-lib:array-slice a-%data% single-float (1 m) ((1 lda) (1 *))
                           a-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf l var-0)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
      (sswap (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
       (f2cl-lib:array-slice a-%data% single-float (j k) ((1 lda) (1 *))
                             a-%offset%)
       lda
       (f2cl-lib:array-slice a-%data% single-float (m k) ((1 lda) (1 *))
                             a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-3))
    (when var-2 (setf lda var-2))
    (when var-4 (setf lda var-4)))
 label30
  (f2cl-lib:computed-goto (label40 label80) iexc)
 label40
  (if (= l 1)
      (go label210))
  (setf l (f2cl-lib:int-sub l 1))
 label50
  (f2cl-lib:fdo (j l (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                ((> j 1) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i l) nil)
        (tagbody
          (if (= i j)
              (go label60))
          (if (/=
               (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *)) a-%offset%)
               zero)
              (go label70))
         label60))
      (setf m l)
      (setf iexc 1)
      (go label20)
     label70))
  (go label90)
 label80
  (setf k (f2cl-lib:int-add k 1))
 label90
  (f2cl-lib:fdo (j k (f2cl-lib:int-add j 1))
                ((> j l) nil)
    (tagbody
      (f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
                    ((> i l) nil)
        (tagbody
          (if (= i j)
              (go label100))
          (if (/=
               (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
               zero)
              (go label110))
```

```
        label100))
      (setf m k)
      (setf iexc 2)
      (go label20)
     label110))
label120
 (f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
               ((> i l) nil)
   (tagbody
     (setf (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%) one)
    label130))
 (if (multiple-value-bind (ret-val var-0 var-1)
         (lsame job "P")
       (declare (ignore var-1))
       (when var-0 (setf job var-0))
       ret-val)
     (go label210))
 (setf sfmin1 (/ (slamch "S") (slamch "P")))
 (setf sfmax1 (/ one sfmin1))
 (setf sfmin2 (* sfmin1 sclfac))
 (setf sfmax2 (/ one sfmin2))
label140
 (setf noconv f2cl-lib:%false%)
 (f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
               ((> i l) nil)
   (tagbody
     (setf c
             (snrm2 (f2cl-lib:int-add (f2cl-lib:int-sub l k) 1)
              (f2cl-lib:array-slice a-%data% single-float (k i)
                                    ((1 lda) (1 *)) a-%offset%)
              1))
     (setf r
             (multiple-value-bind (ret-val var-0 var-1 var-2)
                 (snrm2 (f2cl-lib:int-add (f2cl-lib:int-sub l k) 1)
                  (f2cl-lib:array-slice a-%data% single-float (i k)
                                        ((1 lda) (1 *)) a-%offset%)
                  lda)
               (declare (ignore var-0 var-1))
               (when var-2 (setf lda var-2))
               ret-val))
     (setf ica
             (multiple-value-bind (ret-val var-0 var-1 var-2)
                 (isamax l
                  (f2cl-lib:array-slice a-%data% single-float (1 i)
                                        ((1 lda) (1 *)) a-%offset%)
                  1)
               (declare (ignore var-1 var-2))
               (when var-0 (setf l var-0))
               ret-val))
     (setf ca
```

```
        (abs
         (f2cl-lib:fref a-%data% (ica i) ((1 lda) (1 *))
                        a-%offset%)))
 (setf ira
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (isamax (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
             (f2cl-lib:array-slice a-%data% single-float (i k)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-1))
          (when var-2 (setf lda var-2))
          ret-val))
 (setf ra
        (abs
         (f2cl-lib:fref a-%data%
                        (i
                         (f2cl-lib:int-sub (f2cl-lib:int-add ira k)
                                           1))
                        ((1 lda) (1 *)) a-%offset%)))
 (if (or (= c zero) (= r zero))
     (go label200))
 (setf g (/ r sclfac))
 (setf f one)
 (setf s (+ c r))
label160
 (if (or (>= c g) (>= (max f c ca) sfmax2) (<= (min r g ra) sfmin2))
     (go label170))
 (setf f (* f sclfac))
 (setf c (* c sclfac))
 (setf ca (* ca sclfac))
 (setf r (/ r sclfac))
 (setf g (/ g sclfac))
 (setf ra (/ ra sclfac))
 (go label160)
label170
 (setf g (/ c sclfac))
label180
 (if (or (< g r) (>= (max r ra) sfmax2) (<= (min f c g ca) sfmin2))
     (go label190))
 (cond
  ((sisnan (+ c f ca r g ra)) (setf info -3)
   (xerbla "SGEBAL" (f2cl-lib:int-sub info)) (go end_label)))
 (setf f (/ f sclfac))
 (setf c (/ c sclfac))
 (setf g (/ g sclfac))
 (setf ca (/ ca sclfac))
 (setf r (* r sclfac))
 (setf ra (* ra sclfac))
 (go label180)
label190
```

```
            (if (>= (+ c r) (* factor s))
                (go label200))
            (cond
             ((and (< f one) (< (f2cl-lib:fref scale (i) ((1 *))) one))
              (if (<=
                   (* f
                      (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%))
                   sfmin1)
                  (go label200))))
            (cond
             ((and (> f one) (> (f2cl-lib:fref scale (i) ((1 *))) one))
              (if (>= (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%)
                      (/ sfmax1 f))
                  (go label200))))
            (setf g (/ one f))
            (setf (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%)
                    (* (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%)
                       f))
            (setf noconv f2cl-lib:%true%)
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (sscal (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1) g
                 (f2cl-lib:array-slice a-%data% single-float (i k)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare (ignore var-0 var-2))
              (when var-1 (setf g var-1))
              (when var-3 (setf lda var-3)))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (sscal l f
                 (f2cl-lib:array-slice a-%data% single-float (1 i)
                                       ((1 lda) (1 *)) a-%offset%)
                 1)
              (declare (ignore var-2 var-3))
              (when var-0 (setf l var-0))
              (when var-1 (setf f var-1)))
           label200))
         (if noconv
             (go label140))
        label210
         (setf ilo k)
         (setf ihi l)
         (go end_label)
        end_label
         (return (values job nil nil lda ilo ihi nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgebal
                 fortran-to-lisp::*f2cl-function-info*)
```

```
(fortran-to-lisp::make-f2cl-finfo :arg-types
                                  '((string)
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (fortran-to-lisp::integer4)
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4))
                                  :return-values
                                  '(fortran-to-lisp::job nil nil
                                    fortran-to-lisp::lda
                                    fortran-to-lisp::ilo
                                    fortran-to-lisp::ihi nil
                                    fortran-to-lisp::info)
                                  :calls 'nil)))
```

———-

33.3 sgebd2

33.3.1 Axiom unit tests

— **sgebd2.input** —

```
)set break resume
)sys rm -f sgebd2.output
)spool sgebd2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.3.2 Axiom help page

— **sgebd2.help** —

```
==================================================================
```

```
sgebd2 examples
===================================================================

===================================================================
Man Page Details
===================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgebd2 (M, N, A, LDA, D, E, TAUQ, TAUP, WORK, INFO)
           SGEBD2 reduces a general matrix to bidiagonal form using an
           unblocked algorithm.

Function/Subroutine Documentation
   subroutine sgebd2 (integer M, integer N, real, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) D, real, dimension( * ) E, real,
       dimension( * ) TAUQ, real, dimension( * ) TAUP, real, dimension( * )
       WORK, integer INFO)
       SGEBD2 reduces a general matrix to bidiagonal form using an unblocked
       algorithm.

Purpose:
   SGEBD2 reduces a real general m by n matrix A to upper or lower
   bidiagonal form B by an orthogonal transformation: Q**T * A * P = B.

   If m >= n, B is upper bidiagonal; if m < n, B is lower bidiagonal.

Parameters:

 M
   M is INTEGER
   The number of rows in the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns in the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the m by n general matrix to be reduced.
   On exit,
   if m >= n, the diagonal and the first superdiagonal are
     overwritten with the upper bidiagonal matrix B; the
     elements below the diagonal, with the array TAUQ, represent
     the orthogonal matrix Q as a product of elementary
     reflectors, and the elements above the first superdiagonal,
     with the array TAUP, represent the orthogonal matrix P as
     a product of elementary reflectors;
   if m < n, the diagonal and the first subdiagonal are
     overwritten with the lower bidiagonal matrix B; the
```

```
     elements below the first subdiagonal, with the array TAUQ,
     represent the orthogonal matrix Q as a product of
     elementary reflectors, and the elements above the diagonal,
     with the array TAUP, represent the orthogonal matrix P as
     a product of elementary reflectors.
     See Further Details.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 D
   D is REAL array, dimension (min(M,N))
   The diagonal elements of the bidiagonal matrix B: D(i) = A(i,i).

 E
   E is REAL array, dimension (min(M,N)-1)
   The off-diagonal elements of the bidiagonal matrix B:
   if m >= n, E(i) = A(i,i+1) for i = 1,2,...,n-1;
   if m < n, E(i) = A(i+1,i) for i = 1,2,...,m-1.

 TAUQ
   TAUQ is REAL array dimension (min(M,N))
   The scalar factors of the elementary reflectors which
   represent the orthogonal matrix Q. See Further Details.

 TAUP
   TAUP is REAL array, dimension (min(M,N))
   The scalar factors of the elementary reflectors which
   represent the orthogonal matrix P. See Further Details.

 WORK
   WORK is REAL array, dimension (max(M,N))

 INFO
   INFO is INTEGER
   = 0: successful exit.
   < 0: if INFO = -i, the i-th argument had an illegal value.

Further Details:
   The matrices Q and P are represented as products of elementary
   reflectors:

   If m >= n,

       Q = H(1) H(2) . . . H(n)  and  P = G(1) G(2) . . . G(n-1)

   Each H(i) and G(i) has the form:

       H(i) = I - tauq * v * v**T  and G(i) = I - taup * u * u**T
```

```
where tauq and taup are real scalars, and v and u are real vectors;
v(1:i-1) = 0, v(i) = 1, and v(i+1:m) is stored on exit in A(i+1:m,i);
u(1:i) = 0, u(i+1) = 1, and u(i+2:n) is stored on exit in A(i,i+2:n);
tauq is stored in TAUQ(i) and taup in TAUP(i).

If m < n,

     Q = H(1) H(2) . . . H(m-1)  and  P = G(1) G(2) . . . G(m)

Each H(i) and G(i) has the form:

     H(i) = I - tauq * v * v**T  and G(i) = I - taup * u * u**T

where tauq and taup are real scalars, and v and u are real vectors;
v(1:i) = 0, v(i+1) = 1, and v(i+2:m) is stored on exit in A(i+2:m,i);
u(1:i-1) = 0, u(i) = 1, and u(i+1:n) is stored on exit in A(i,i+1:n);
tauq is stored in TAUQ(i) and taup in TAUP(i).

The contents of A on exit are illustrated by the following examples:

          m = 6 and n = 5 (m > n):          m = 5 and n = 6 (m < n):

            (  d   e   u1  u1  u1 )           (  d   u1  u1  u1  u1  u1 )
            (  v1  d   e   u2  u2 )           (  e   d   u2  u2  u2  u2 )
            (  v1  v2  d   e   u3 )           (  v1  e   d   u3  u3  u3 )
            (  v1  v2  v3  d   e  )           (  v1  v2  e   d   u4  u4 )
            (  v1  v2  v3  v4  d  )           (  v1  v2  v3  e   d   u5 )
            (  v1  v2  v3  v4  v5 )

where d and e denote diagonal and off-diagonal elements of B, vi
denotes an element of the vector defining H(i), and ui an element of
the vector defining G(i).
```

———-

33.3.3 fortran code

— sgebd2.f —

```
*  =====================================================================
      SUBROUTINE SGEBD2( M, N, A, LDA, D, E, TAUQ, TAUP, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
```

```
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), D( * ), E( * ), TAUP( * ),
     $                   TAUQ( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLARF, SLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.LT.0 ) THEN
         CALL XERBLA( 'SGEBD2', -INFO )
         RETURN
      END IF
*
      IF( M.GE.N ) THEN
*
*        Reduce to upper bidiagonal form
*
         DO 10 I = 1, N
*
*           Generate elementary reflector H(i) to annihilate A(i+1:m,i)
*
            CALL SLARFG( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
```

```
     $                  TAUQ( I ) )
            D( I ) = A( I, I )
            A( I, I ) = ONE
*
*           Apply H(i) to A(i:m,i+1:n) from the left
*
            IF( I.LT.N )
     $         CALL SLARF( 'Left', M-I+1, N-I, A( I, I ), 1, TAUQ( I ),
     $                     A( I, I+1 ), LDA, WORK )
            A( I, I ) = D( I )
*
            IF( I.LT.N ) THEN
*
*              Generate elementary reflector G(i) to annihilate
*              A(i,i+2:n)
*
               CALL SLARFG( N-I, A( I, I+1 ), A( I, MIN( I+2, N ) ),
     $                      LDA, TAUP( I ) )
               E( I ) = A( I, I+1 )
               A( I, I+1 ) = ONE
*
*              Apply G(i) to A(i+1:m,i+1:n) from the right
*
               CALL SLARF( 'Right', M-I, N-I, A( I, I+1 ), LDA,
     $                     TAUP( I ), A( I+1, I+1 ), LDA, WORK )
               A( I, I+1 ) = E( I )
            ELSE
               TAUP( I ) = ZERO
            END IF
   10    CONTINUE
      ELSE
*
*        Reduce to lower bidiagonal form
*
         DO 20 I = 1, M
*
*           Generate elementary reflector G(i) to annihilate A(i,i+1:n)
*
            CALL SLARFG( N-I+1, A( I, I ), A( I, MIN( I+1, N ) ), LDA,
     $                   TAUP( I ) )
            D( I ) = A( I, I )
            A( I, I ) = ONE
*
*           Apply G(i) to A(i+1:m,i:n) from the right
*
            IF( I.LT.M )
     $         CALL SLARF( 'Right', M-I, N-I+1, A( I, I ), LDA,
     $                     TAUP( I ), A( I+1, I ), LDA, WORK )
            A( I, I ) = D( I )
*
```

```
            IF( I.LT.M ) THEN
*
*              Generate elementary reflector H(i) to annihilate
*              A(i+2:m,i)
*
               CALL SLARFG( M-I, A( I+1, I ), A( MIN( I+2, M ), I ), 1,
     $                      TAUQ( I ) )
               E( I ) = A( I+1, I )
               A( I+1, I ) = ONE
*
*              Apply H(i) to A(i+1:m,i+1:n) from the left
*
               CALL SLARF( 'Left', M-I, N-I, A( I+1, I ), 1, TAUQ( I ),
     $                     A( I+1, I+1 ), LDA, WORK )
               A( I+1, I ) = E( I )
            ELSE
               TAUQ( I ) = ZERO
            END IF
   20    CONTINUE
      END IF
      RETURN
*
*     End of SGEBD2
*
      END
```

———-

33.3.4 lisp code

— sgebd2.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sgebd2 (m n a lda d e tauq taup work info)
    (declare (type (array single-float (*)) work taup tauq e d a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%)
         (tauq single-float tauq-%data% tauq-%offset%)
         (taup single-float taup-%data% taup-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((i 0))
        (declare (type (f2cl-lib:integer4) i))
```

```
(setf info 0)
(cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -4)))
(cond
 ((< info 0) (xerbla "SGEBD2" (f2cl-lib:int-sub info)) (go end_label)))
(cond
 ((>= m n)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (slarfg (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
       (f2cl-lib:array-slice a-%data% single-float (i i)
                             ((1 lda) (1 *)) a-%offset%)
       (f2cl-lib:array-slice a-%data% single-float
                             ((min (f2cl-lib:int-add i 1) m) i)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice tauq-%data% single-float (i) ((1 *))
                             tauq-%offset%))
      (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
              (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                             a-%offset%))
      (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
              one)
      (if (< i n)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (slarf "Left" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
               (f2cl-lib:int-sub n i)
               (f2cl-lib:array-slice a-%data% single-float (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               1
               (f2cl-lib:array-slice tauq-%data% single-float (i)
                                     ((1 *)) tauq-%offset%)
               (f2cl-lib:array-slice a-%data% single-float
                                     (i (f2cl-lib:int-add i 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda work)
            (declare
             (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
            (when var-7 (setf lda var-7))))
      (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
      (cond
       ((< i n)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (slarfg (f2cl-lib:int-sub n i)
             (f2cl-lib:array-slice a-%data% single-float
```

```
                                    (i (f2cl-lib:int-add i 1))
                                    ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:array-slice a-%data% single-float
                                    (i
                                     (min
                                      (the f2cl-lib:integer4
                                           (f2cl-lib:int-add i 2))
                                      (the f2cl-lib:integer4 n)))
                                    ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice taup-%data% single-float (i) ((1 *))
                                    taup-%offset%))
              (declare (ignore var-0 var-1 var-2 var-4))
              (when var-3 (setf lda var-3)))
            (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                    (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
                                   ((1 lda) (1 *)) a-%offset%))
            (setf (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
                                 ((1 lda) (1 *)) a-%offset%)
                    one)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (slarf "Right" (f2cl-lib:int-sub m i)
                 (f2cl-lib:int-sub n i)
                 (f2cl-lib:array-slice a-%data% single-float
                                    (i (f2cl-lib:int-add i 1))
                                    ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice taup-%data% single-float (i) ((1 *))
                                    taup-%offset%)
                 (f2cl-lib:array-slice a-%data% single-float
                                    ((+ i 1) (f2cl-lib:int-add i 1))
                                    ((1 lda) (1 *)) a-%offset%)
                 lda work)
              (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8))
              (when var-4 (setf lda var-4))
              (when var-7 (setf lda var-7)))
            (setf (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
                                 ((1 lda) (1 *)) a-%offset%)
                    (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
          (t
            (setf (f2cl-lib:fref taup-%data% (i) ((1 *)) taup-%offset%)
                    zero)))
        label10)))
    (t
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i m) nil)
       (tagbody
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (slarfg (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
```

```
     (f2cl-lib:array-slice a-%data% single-float (i i)
                           ((1 lda) (1 *)) a-%offset%)
     (f2cl-lib:array-slice a-%data% single-float
                           (i
                            (min
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add i 1))
                             (the f2cl-lib:integer4 n)))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice taup-%data% single-float (i) ((1 *))
                           taup-%offset%))
  (declare (ignore var-0 var-1 var-2 var-4))
  (when var-3 (setf lda var-3)))
(setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
        (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                       a-%offset%))
(setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
        one)
(if (< i m)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (slarf "Right" (f2cl-lib:int-sub m i)
         (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
         (f2cl-lib:array-slice a-%data% single-float (i i)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice taup-%data% single-float (i)
                               ((1 *)) taup-%offset%)
         (f2cl-lib:array-slice a-%data% single-float ((+ i 1) i)
                               ((1 lda) (1 *)) a-%offset%)
         lda work)
      (declare
       (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8))
      (when var-4 (setf lda var-4))
      (when var-7 (setf lda var-7))))
(setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
        (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
(cond
 ((< i m)
  (slarfg (f2cl-lib:int-sub m i)
   (f2cl-lib:array-slice a-%data% single-float ((+ i 1) i)
                         ((1 lda) (1 *)) a-%offset%)
   (f2cl-lib:array-slice a-%data% single-float
                         ((min (f2cl-lib:int-add i 2) m) i)
                         ((1 lda) (1 *)) a-%offset%)
   1
   (f2cl-lib:array-slice tauq-%data% single-float (i) ((1 *))
                         tauq-%offset%))
  (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
```

```
                        (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                                       ((1 lda) (1 *)) a-%offset%))
                (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                                     ((1 lda) (1 *)) a-%offset%)
                        one)
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                    (slarf "Left" (f2cl-lib:int-sub m i) (f2cl-lib:int-sub n i)
                     (f2cl-lib:array-slice a-%data% single-float ((+ i 1) i)
                                           ((1 lda) (1 *)) a-%offset%)
                     1
                     (f2cl-lib:array-slice tauq-%data% single-float (i) ((1 *))
                                           tauq-%offset%)
                     (f2cl-lib:array-slice a-%data% single-float
                                           ((+ i 1) (f2cl-lib:int-add i 1))
                                           ((1 lda) (1 *)) a-%offset%)
                     lda work)
                  (declare
                   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
                  (when var-7 (setf lda var-7)))
                (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                                     ((1 lda) (1 *)) a-%offset%)
                        (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
               (t
                (setf (f2cl-lib:fref tauq-%data% (i) ((1 *)) tauq-%offset%)
                        zero)))
             label20))))
        (go end_label)
       end_label
        (return (values nil nil nil lda nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgebd2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::lda
                                              nil nil nil nil nil
```

```
                                        fortran-to-lisp::info)
                                      :calls 'nil)))
```

33.4 sgebrd

33.4.1 Axiom unit tests

— sgebrd.input —

```
)set break resume
)sys rm -f sgebrd.output
)spool sgebrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

33.4.2 Axiom help page

— sgebrd.help —

```
======================================================================
sgebrd examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgebrd (M, N, A, LDA, D, E, TAUQ, TAUP, WORK, LWORK, INFO)
           SGEBRD

Function/Subroutine Documentation
   subroutine sgebrd (integer M, integer N, real, dimension( lda, * ) A,
```

```
      integer LDA, real, dimension( * ) D, real, dimension( * ) E, real,
      dimension( * ) TAUQ, real, dimension( * ) TAUP, real, dimension( * )
      WORK, integer LWORK, integer INFO)
      SGEBRD

Purpose:
   SGEBRD reduces a general real M-by-N matrix A to upper or lower
   bidiagonal form B by an orthogonal transformation: Q**T * A * P = B.

   If m >= n, B is upper bidiagonal; if m < n, B is lower bidiagonal.

Parameters:

 M
   M is INTEGER
   The number of rows in the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns in the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the M-by-N general matrix to be reduced.
   On exit,
   if m >= n, the diagonal and the first superdiagonal are
      overwritten with the upper bidiagonal matrix B; the
      elements below the diagonal, with the array TAUQ, represent
      the orthogonal matrix Q as a product of elementary
      reflectors, and the elements above the first superdiagonal,
      with the array TAUP, represent the orthogonal matrix P as
      a product of elementary reflectors;
   if m < n, the diagonal and the first subdiagonal are
      overwritten with the lower bidiagonal matrix B; the
      elements below the first subdiagonal, with the array TAUQ,
      represent the orthogonal matrix Q as a product of
      elementary reflectors, and the elements above the diagonal,
      with the array TAUP, represent the orthogonal matrix P as
      a product of elementary reflectors.
   See Further Details.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 D
   D is REAL array, dimension (min(M,N))
   The diagonal elements of the bidiagonal matrix B: D(i) = A(i,i).

 E
```

```
  E is REAL array, dimension (min(M,N)-1)
  The off-diagonal elements of the bidiagonal matrix B:
  if m >= n, E(i) = A(i,i+1) for i = 1,2,...,n-1;
  if m < n, E(i) = A(i+1,i) for i = 1,2,...,m-1.

TAUQ
  TAUQ is REAL array dimension (min(M,N))
  The scalar factors of the elementary reflectors which
  represent the orthogonal matrix Q. See Further Details.

TAUP
  TAUP is REAL array, dimension (min(M,N))
  The scalar factors of the elementary reflectors which
  represent the orthogonal matrix P. See Further Details.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of the array WORK.  LWORK >= max(1,M,N).
  For optimum performance LWORK >= (M+N)*NB, where NB
  is the optimal blocksize.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.

Further Details:

  The matrices Q and P are represented as products of elementary
  reflectors:

  If m >= n,

     Q = H(1) H(2) . . . H(n)  and  P = G(1) G(2) . . . G(n-1)

  Each H(i) and G(i) has the form:

     H(i) = I - tauq * v * v**T  and G(i) = I - taup * u * u**T

  where tauq and taup are real scalars, and v and u are real vectors;
  v(1:i-1) = 0, v(i) = 1, and v(i+1:m) is stored on exit in A(i+1:m,i);
```

```
u(1:i) = 0, u(i+1) = 1, and u(i+2:n) is stored on exit in A(i,i+2:n);
tauq is stored in TAUQ(i) and taup in TAUP(i).

If m < n,

    Q = H(1) H(2) . . . H(m-1)  and  P = G(1) G(2) . . . G(m)

Each H(i) and G(i) has the form:

    H(i) = I - tauq * v * v**T  and G(i) = I - taup * u * u**T

where tauq and taup are real scalars, and v and u are real vectors;
v(1:i) = 0, v(i+1) = 1, and v(i+2:m) is stored on exit in A(i+2:m,i);
u(1:i-1) = 0, u(i) = 1, and u(i+1:n) is stored on exit in A(i,i+1:n);
tauq is stored in TAUQ(i) and taup in TAUP(i).

The contents of A on exit are illustrated by the following examples:

          m = 6 and n = 5 (m > n):          m = 5 and n = 6 (m < n):

            (  d   e   u1  u1  u1 )           (  d   u1  u1  u1  u1  u1 )
            (  v1  d   e   u2  u2 )           (  e   d   u2  u2  u2  u2 )
            (  v1  v2  d   e   u3 )           (  v1  e   d   u3  u3  u3 )
            (  v1  v2  v3  d   e  )           (  v1  v2  e   d   u4  u4 )
            (  v1  v2  v3  v4  d  )           (  v1  v2  v3  e   d   u5 )
            (  v1  v2  v3  v4  v5 )

where d and e denote diagonal and off-diagonal elements of B, vi
denotes an element of the vector defining H(i), and ui an element of
the vector defining G(i).
```

33.4.3 fortran code

— sgebrd.f —

```
*  ===================================================================
      SUBROUTINE SGEBRD( M, N, A, LDA, D, E, TAUQ, TAUP, WORK, LWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
```

```
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), D( * ), E( * ), TAUP( * ),
     $                   TAUQ( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IINFO, J, LDWRKX, LDWRKY, LWKOPT, MINMN, NB,
     $                   NBMIN, NX
      REAL               WS
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEBD2, SGEMM, SLABRD, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, REAL
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      NB = MAX( 1, ILAENV( 1, 'SGEBRD', ' ', M, N, -1, -1 ) )
      LWKOPT = ( M+N )*NB
      WORK( 1 ) = REAL( LWKOPT )
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, M, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -10
      END IF
      IF( INFO.LT.0 ) THEN
         CALL XERBLA( 'SGEBRD', -INFO )
         RETURN
```

```
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      MINMN = MIN( M, N )
      IF( MINMN.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      WS = MAX( M, N )
      LDWRKX = M
      LDWRKY = N
*
      IF( NB.GT.1 .AND. NB.LT.MINMN ) THEN
*
*        Set the crossover point NX.
*
         NX = MAX( NB, ILAENV( 3, 'SGEBRD', ' ', M, N, -1, -1 ) )
*
*        Determine when to switch from blocked to unblocked code.
*
         IF( NX.LT.MINMN ) THEN
            WS = ( M+N )*NB
            IF( LWORK.LT.WS ) THEN
*
*              Not enough work space for the optimal NB, consider using
*              a smaller block size.
*
               NBMIN = ILAENV( 2, 'SGEBRD', ' ', M, N, -1, -1 )
               IF( LWORK.GE.( M+N )*NBMIN ) THEN
                  NB = LWORK / ( M+N )
               ELSE
                  NB = 1
                  NX = MINMN
               END IF
            END IF
         END IF
      ELSE
         NX = MINMN
      END IF
*
      DO 30 I = 1, MINMN - NX, NB
*
*        Reduce rows and columns i:i+nb-1 to bidiagonal form and return
*        the matrices X and Y which are needed to update the unreduced
*        part of the matrix
*
```

```
         CALL SLABRD( M-I+1, N-I+1, NB, A( I, I ), LDA, D( I ), E( I ),
     $                TAUQ( I ), TAUP( I ), WORK, LDWRKX,
     $                WORK( LDWRKX*NB+1 ), LDWRKY )
*
*        Update the trailing submatrix A(i+nb:m,i+nb:n), using an update
*        of the form  A := A - V*Y**T - X*U**T
*
         CALL SGEMM( 'No transpose', 'Transpose', M-I-NB+1, N-I-NB+1,
     $               NB, -ONE, A( I+NB, I ), LDA,
     $               WORK( LDWRKX*NB+NB+1 ), LDWRKY, ONE,
     $               A( I+NB, I+NB ), LDA )
         CALL SGEMM( 'No transpose', 'No transpose', M-I-NB+1, N-I-NB+1,
     $               NB, -ONE, WORK( NB+1 ), LDWRKX, A( I, I+NB ), LDA,
     $               ONE, A( I+NB, I+NB ), LDA )
*
*        Copy diagonal and off-diagonal elements of B back into A
*
         IF( M.GE.N ) THEN
            DO 10 J = I, I + NB - 1
               A( J, J ) = D( J )
               A( J, J+1 ) = E( J )
   10       CONTINUE
         ELSE
            DO 20 J = I, I + NB - 1
               A( J, J ) = D( J )
               A( J+1, J ) = E( J )
   20       CONTINUE
         END IF
   30 CONTINUE
*
*     Use unblocked code to reduce the remainder of the matrix
*
      CALL SGEBD2( M-I+1, N-I+1, A( I, I ), LDA, D( I ), E( I ),
     $             TAUQ( I ), TAUP( I ), WORK, IINFO )
      WORK( 1 ) = WS
      RETURN
*
*     End of SGEBRD
*
      END
```

———-

33.4.4 lisp code

— sgebrd.lisp —

```
(let* ((one 1.0))
```

```
(declare (type (single-float 1.0 1.0) one)
         (ignorable one))
(defun sgebrd (m n a lda d e tauq taup work lwork info)
  (declare (type (array single-float (*)) work taup tauq e d a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a single-float a-%data% a-%offset%)
       (d single-float d-%data% d-%offset%)
       (e single-float e-%data% e-%offset%)
       (tauq single-float tauq-%data% tauq-%offset%)
       (taup single-float taup-%data% taup-%offset%)
       (work single-float work-%data% work-%offset%))
    (prog ((ws 0.0) (i 0) (iinfo 0) (j 0) (ldwrkx 0) (ldwrky 0) (lwkopt 0)
           (minmn 0) (nb 0) (nbmin 0) (nx 0) (lquery nil))
      (declare (type (single-float) ws)
               (type (f2cl-lib:integer4) i iinfo j ldwrkx ldwrky lwkopt minmn
                nb nbmin nx)
               (type f2cl-lib:logical lquery))
      (setf info 0)
      (setf nb
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
                            (ilaenv 1 "SGEBRD" " " m n -1 -1)
                          (declare (ignore var-0 var-1 var-2 var-5 var-6))
                          (setf m var-3)
                          (setf n var-4)
                          ret-val))))
      (setf lwkopt (f2cl-lib:int-mul (f2cl-lib:int-add m n) nb))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (f2cl-lib:freal lwkopt))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4))
            ((and
              (< lwork
                 (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)
                      (the f2cl-lib:integer4 n)))
              (not lquery))
             (setf info -10)))
      (cond
       ((< info 0) (xerbla "SGEBRD" (f2cl-lib:int-sub info)) (go end_label))
       (lquery (go end_label)))
      (setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
      (cond
       ((= minmn 0)
```

```
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'single-float))
  (go end_label)))
(setf ws
        (coerce
         (the f2cl-lib:integer4
              (max (the f2cl-lib:integer4 m)
                   (the f2cl-lib:integer4 n)))
         'single-float))
(setf ldwrkx m)
(setf ldwrky n)
(cond
 ((and (> nb 1) (< nb minmn))
  (setf nx
          (max (the f2cl-lib:integer4 nb)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "SGEBRD" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (setf m var-3)
                      (setf n var-4)
                      ret-val))))
  (cond
   ((< nx minmn)
    (setf ws
            (coerce
             (the f2cl-lib:integer4
                  (f2cl-lib:int-mul (f2cl-lib:int-add m n) nb))
             'single-figure))
    (cond
     ((< lwork ws)
      (setf nbmin
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 2 "SGEBRD" " " m n -1 -1)
                (declare (ignore var-0 var-1 var-2 var-5 var-6))
                (setf m var-3)
                (setf n var-4)
                ret-val))
      (cond
       ((>= lwork (f2cl-lib:int-mul (f2cl-lib:int-add m n) nbmin))
        (setf nb (the f2cl-lib:integer4 (truncate lwork (+ m n)))))
       (t (setf nb 1) (setf nx minmn))))))))
 (t (setf nx minmn)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
              ((> i (f2cl-lib:int-add minmn (f2cl-lib:int-sub nx))) nil)
  (tagbody
    (multiple-value-bind
```

```
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (slabrd (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
       (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) nb
       (f2cl-lib:array-slice a-%data% single-float (i i)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice d-%data% single-float (i) ((1 *))
                             d-%offset%)
       (f2cl-lib:array-slice e-%data% single-float (i) ((1 *))
                             e-%offset%)
       (f2cl-lib:array-slice tauq-%data% single-float (i) ((1 *))
                             tauq-%offset%)
       (f2cl-lib:array-slice taup-%data% single-float (i) ((1 *))
                             taup-%offset%)
       work ldwrkx
       (f2cl-lib:array-slice work-%data% single-float
                             ((+ (f2cl-lib:int-mul ldwrkx nb) 1))
                             ((1 *)) work-%offset%)
       ldwrky)
    (declare
     (ignore var-0 var-1 var-3 var-5 var-6 var-7 var-8 var-9 var-11))
    (when var-2 (setf nb var-2))
    (when var-4 (setf lda var-4))
    (when var-10 (setf ldwrkx var-10))
    (when var-12 (setf ldwrky var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sgemm "No transpose" "Transpose"
       (f2cl-lib:int-add (f2cl-lib:int-sub m i nb) 1)
       (f2cl-lib:int-add (f2cl-lib:int-sub n i nb) 1) nb (- one)
       (f2cl-lib:array-slice a-%data% single-float ((+ i nb) i)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice work-%data% single-float
                             ((+ (f2cl-lib:int-mul ldwrkx nb) nb 1))
                             ((1 *)) work-%offset%)
       ldwrky one
       (f2cl-lib:array-slice a-%data% single-float
                             ((+ i nb) (f2cl-lib:int-add i nb))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8 var-11))
    (when var-4 (setf nb var-4))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldwrky var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf lda var-12)))
```

```
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12)
     (sgemm "No transpose" "No transpose"
      (f2cl-lib:int-add (f2cl-lib:int-sub m i nb) 1)
      (f2cl-lib:int-add (f2cl-lib:int-sub n i nb) 1) nb (- one)
      (f2cl-lib:array-slice work-%data% single-float ((+ nb 1))
                            ((1 *)) work-%offset%)
      ldwrkx
      (f2cl-lib:array-slice a-%data% single-float
                            (i (f2cl-lib:int-add i nb))
                            ((1 lda) (1 *)) a-%offset%)
      lda one
      (f2cl-lib:array-slice a-%data% single-float
                            ((+ i nb) (f2cl-lib:int-add i nb))
                            ((1 lda) (1 *)) a-%offset%)
      lda)
   (declare
    (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8 var-11))
   (when var-4 (setf nb var-4))
   (when var-7 (setf ldwrkx var-7))
   (when var-9 (setf lda var-9))
   (when var-10 (setf one var-10))
   (when var-12 (setf lda var-12)))
 (cond
  ((>= m n)
   (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                 ((> j (f2cl-lib:int-add i nb (f2cl-lib:int-sub 1)))
                  nil)
     (tagbody
       (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                            a-%offset%)
               (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%))
       (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-add j 1))
                            ((1 lda) (1 *)) a-%offset%)
               (f2cl-lib:fref e-%data% (j) ((1 *)) e-%offset%))
      label10)))
  (t
   (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                 ((> j (f2cl-lib:int-add i nb (f2cl-lib:int-sub 1)))
                  nil)
     (tagbody
       (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                            a-%offset%)
               (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%))
       (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add j 1) j)
                            ((1 lda) (1 *)) a-%offset%)
               (f2cl-lib:fref e-%data% (j) ((1 *)) e-%offset%))
      label20))))
label30))
```

```
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
            (sgebd2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
             (f2cl-lib:array-slice a-%data% single-float (i i) ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice d-%data% single-float (i) ((1 *))
                                   d-%offset%)
             (f2cl-lib:array-slice e-%data% single-float (i) ((1 *))
                                   e-%offset%)
             (f2cl-lib:array-slice tauq-%data% single-float (i) ((1 *))
                                   tauq-%offset%)
             (f2cl-lib:array-slice taup-%data% single-float (i) ((1 *))
                                   taup-%offset%)
             work iinfo)
          (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-7 var-8))
          (setf lda var-3)
          (setf iinfo var-9))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) ws)
        (go end_label)
       end_label
        (return (values m n nil lda nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgebrd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::sgebd2
                                              fortran-to-lisp::ilaenv))))
```

———-

33.5 sgecon

33.5.1 Axiom unit tests

— **sgecon.input** —

```
)set break resume
)sys rm -f sgecon.output
)spool sgecon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.5.2 Axiom help page

— **sgecon.help** —

```
======================================================================
sgecon examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgecon (NORM, N, A, LDA, ANORM, RCOND, WORK, IWORK, INFO)
           SGECON

Function/Subroutine Documentation
   subroutine sgecon (character NORM, integer N, real, dimension( lda, * ) A,
       integer LDA, real ANORM, real RCOND, real, dimension( * ) WORK,
       integer, dimension( * ) IWORK, integer INFO)
```

```
      SGECON

Purpose:
   SGECON estimates the reciprocal of the condition number of a general
   real matrix A, in either the 1-norm or the infinity-norm, using
   the LU factorization computed by SGETRF.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as
      RCOND = 1 / ( norm(A) * norm(inv(A)) ).

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies whether the 1-norm condition number or the
   infinity-norm condition number is required:
   = '1' or 'O':  1-norm;
   = 'I':         Infinity-norm.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   The factors L and U from the factorization A = P*L*U
   as computed by SGETRF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 ANORM
   ANORM is REAL
   If NORM = '1' or 'O', the 1-norm of the original matrix A.
   If NORM = 'I', the infinity-norm of the original matrix A.

 RCOND
   RCOND is REAL
   The reciprocal of the condition number of the matrix A,
   computed as RCOND = 1/(norm(A) * norm(inv(A))).

 WORK
   WORK is REAL array, dimension (4*N)

 IWORK
   IWORK is INTEGER array, dimension (N)

 INFO
```

```
INFO is INTEGER
= 0:  successful exit
< 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

33.5.3 fortran code

— **sgecon.f** —

```
*  ====================================================================
      SUBROUTINE SGECON( NORM, N, A, LDA, ANORM, RCOND, WORK, IWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            INFO, LDA, N
      REAL               ANORM, RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      REAL               A( LDA, * ), WORK( * )
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ONENRM
      CHARACTER          NORMIN
      INTEGER            IX, KASE, KASE1
      REAL               AINVNM, SCALE, SL, SMLNUM, SU
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ISAMAX
```

```
      REAL               SLAMCH
      EXTERNAL           LSAME, ISAMAX, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLACN2, SLATRS, SRSCL, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      ONENRM = NORM.EQ.'1' .OR. LSAME( NORM, 'O' )
      IF( .NOT.ONENRM .AND. .NOT.LSAME( NORM, 'I' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -5
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGECON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
      SMLNUM = SLAMCH( 'Safe minimum' )
*
*     Estimate the norm of inv(A).
*
      AINVNM = ZERO
      NORMIN = 'N'
      IF( ONENRM ) THEN
         KASE1 = 1
      ELSE
         KASE1 = 2
      END IF
```

```
      KASE = 0
   10 CONTINUE
      CALL SLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.KASE1 ) THEN
*
*           Multiply by inv(L).
*
            CALL SLATRS( 'Lower', 'No transpose', 'Unit', NORMIN, N, A,
     $                   LDA, WORK, SL, WORK( 2*N+1 ), INFO )
*
*           Multiply by inv(U).
*
            CALL SLATRS( 'Upper', 'No transpose', 'Non-unit', NORMIN, N,
     $                   A, LDA, WORK, SU, WORK( 3*N+1 ), INFO )
         ELSE
*
*           Multiply by inv(U**T).
*
            CALL SLATRS( 'Upper', 'Transpose', 'Non-unit', NORMIN, N, A,
     $                   LDA, WORK, SU, WORK( 3*N+1 ), INFO )
*
*           Multiply by inv(L**T).
*
            CALL SLATRS( 'Lower', 'Transpose', 'Unit', NORMIN, N, A,
     $                   LDA, WORK, SL, WORK( 2*N+1 ), INFO )
         END IF
*
*        Divide X by 1/(SL*SU) if doing so will not cause overflow.
*
         SCALE = SL*SU
         NORMIN = 'Y'
         IF( SCALE.NE.ONE ) THEN
            IX = ISAMAX( N, WORK, 1 )
            IF( SCALE.LT.ABS( WORK( IX ) )*SMLNUM .OR. SCALE.EQ.ZERO )
     $         GO TO 20
            CALL SRSCL( N, SCALE, WORK, 1 )
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
   20 CONTINUE
      RETURN
*
*     End of SGECON
```

```
*
      END
```

———-

33.5.4 lisp code

— **sgecon.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun sgecon (norm n a lda anorm rcond work iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (single-float) rcond anorm)
             (type (array single-float (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (a single-float a-%data% a-%offset%)
         (work single-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (ainvnm 0.0) (scale 0.0) (sl 0.0) (smlnum 0.0) (su 0.0) (ix 0)
             (kase 0) (kase1 0)
             (normin
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (onenrm nil))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (single-float) ainvnm scale sl smlnum su)
                 (type (f2cl-lib:integer4) ix kase kase1)
                 (type (string 1) normin)
                 (type f2cl-lib:logical onenrm))
        (setf info 0)
        (setf onenrm
                (or (f2cl-lib:fstring-= norm "1")
                    (multiple-value-bind (ret-val var-0 var-1)
                        (lsame norm "O")
                      (declare (ignore var-1))
                      (when var-0 (setf norm var-0))
                      ret-val)))
        (cond
         ((and (not onenrm)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame norm "I")
```

```
          (declare (ignore var-1))
          (when var-0 (setf norm var-0))
          ret-val)))
   (setf info -1))
  ((< n 0) (setf info -2))
  ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
   (setf info -4))
  ((< anorm zero) (setf info -5)))
 (cond
  ((/= info 0) (xerbla "SGECON" (f2cl-lib:int-sub info))
   (go end_label)))
 (setf rcond zero)
 (cond ((= n 0) (setf rcond one) (go end_label))
       ((= anorm zero) (go end_label)))
 (setf smlnum (slamch "Safe minimum"))
 (setf ainvnm zero)
 (f2cl-lib:f2cl-set-string normin "N" (string 1))
 (cond (onenrm (setf kase1 1)) (t (setf kase1 2)))
 (setf kase 0)
label10
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slacn2 n
      (f2cl-lib:array-slice work-%data% single-float ((+ n 1)) ((1 *))
                            work-%offset%)
      work iwork ainvnm kase isave)
   (declare (ignore var-1 var-2 var-3 var-6))
   (when var-0 (setf n var-0))
   (when var-4 (setf ainvnm var-4))
   (when var-5 (setf kase var-5)))
 (cond
  ((/= kase 0)
   (cond
    ((= kase kase1)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (slatrs "Lower" "No transpose" "Unit" normin n a lda work sl
          (f2cl-lib:array-slice work-%data% single-float
                                ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                                work-%offset%)
          info)
       (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
       (when var-3 (setf normin var-3))
       (when var-4 (setf n var-4))
       (when var-6 (setf lda var-6))
       (when var-8 (setf sl var-8))
       (when var-10 (setf info var-10)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
```

```
        (slatrs "Upper" "No transpose" "Non-unit" normin n a lda work
         su
         (f2cl-lib:array-slice work-%data% single-float
                               ((+ (f2cl-lib:int-mul 3 n) 1)) ((1 *))
                               work-%offset%)
         info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
      (when var-3 (setf normin var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf su var-8))
      (when var-10 (setf info var-10))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (slatrs "Upper" "Transpose" "Non-unit" normin n a lda work su
         (f2cl-lib:array-slice work-%data% single-float
                               ((+ (f2cl-lib:int-mul 3 n) 1)) ((1 *))
                               work-%offset%)
         info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
      (when var-3 (setf normin var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf su var-8))
      (when var-10 (setf info var-10)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (slatrs "Lower" "Transpose" "Unit" normin n a lda work sl
         (f2cl-lib:array-slice work-%data% single-float
                               ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                               work-%offset%)
         info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
      (when var-3 (setf normin var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf sl var-8))
      (when var-10 (setf info var-10)))))
  (setf scale (* sl su))
  (f2cl-lib:f2cl-set-string normin "Y" (string 1))
  (cond
   ((/= scale one)
    (setf ix
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (isamax n work 1)
              (declare (ignore var-1 var-2))
              (when var-0 (setf n var-0))
```

```
                    ret-val))
            (if (or
                 (< scale
                    (*
                     (abs
                      (f2cl-lib:fref work-%data% (ix) ((1 *)) work-%offset%))
                     smlnum))
                 (= scale zero))
                (go label20))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (srscl n scale work 1)
              (declare (ignore var-2 var-3))
              (when var-0 (setf n var-0))
              (when var-1 (setf scale var-1)))))
          (go label10)))
        (if (/= ainvnm zero)
            (setf rcond (/ (/ one ainvnm) anorm)))
      label20
        (go end_label)
      end_label
        (return (values norm n nil lda nil rcond nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgecon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (array single-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::norm
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::rcond nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

33.6 sgeequ

33.6.1 Axiom unit tests

— **sgeequ.input** —

```
)set break resume
)sys rm -f sgeequ.output
)spool sgeequ.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.6.2 Axiom help page

— **sgeequ.help** —

```
======================================================================
sgeequ examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgeequ (M, N, A, LDA, R, C, ROWCND, COLCND, AMAX, INFO)
           SGEEQU

Function/Subroutine Documentation
   subroutine sgeequ (integer M, integer N, real, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) R, real, dimension( * ) C, real
       ROWCND, real COLCND, real AMAX, integer INFO)
       SGEEQU

Purpose:
   SGEEQU computes row and column scalings intended to equilibrate an
   M-by-N matrix A and reduce its condition number.  R returns the row
   scale factors and C the column scale factors, chosen to try to make
   the largest element in each row and column of the matrix B with
```

```
  elements B(i,j)=R(i)*A(i,j)*C(j) have absolute value 1.

  R(i) and C(j) are restricted to be between SMLNUM = smallest safe
  number and BIGNUM = largest safe number.  Use of these scaling
  factors is not guaranteed to reduce the condition number of A but
  works well in practice.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   The M-by-N matrix whose equilibration factors are to be computed.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 R
   R is REAL array, dimension (M)
   If INFO = 0 or INFO > M, R contains the row scale factors for A.

 C
   C is REAL array, dimension (N)
   If INFO = 0,  C contains the column scale factors for A.

 ROWCND
   ROWCND is REAL
   If INFO = 0 or INFO > M, ROWCND contains the ratio of the
   smallest R(i) to the largest R(i).  If ROWCND >= 0.1 and
   AMAX is neither too large nor too small, it is not worth scaling by R.

 COLCND
   COLCND is REAL
   If INFO = 0, COLCND contains the ratio of the smallest
   C(i) to the largest C(i).  If COLCND >= 0.1, it is not worth scaling by C.

 AMAX
   AMAX is REAL
   Absolute value of largest matrix element.  If AMAX is very
   close to overflow or very close to underflow, the matrix should be scaled.

 INFO
```

```
INFO is INTEGER
= 0:  successful exit
< 0:  if INFO = -i, the i-th argument had an illegal value
> 0:  if INFO = i,  and i is
  <= M:  the i-th row of A is exactly zero
  >  M:  the (i-M)-th column of A is exactly zero
```

33.6.3 fortran code

— sgeequ.f —

```
*  =====================================================================
      SUBROUTINE SGEEQU( M, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
      REAL               AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), C( * ), R( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               BIGNUM, RCMAX, RCMIN, SMLNUM
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
```

```
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEEQU', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         ROWCND = ONE
         COLCND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Get machine constants.
*
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
*
*     Compute row scale factors.
*
      DO 10 I = 1, M
         R( I ) = ZERO
   10 CONTINUE
*
*     Find the maximum element in each row.
*
      DO 30 J = 1, N
         DO 20 I = 1, M
            R( I ) = MAX( R( I ), ABS( A( I, J ) ) )
   20    CONTINUE
   30 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
```

```
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 40 I = 1, M
         RCMAX = MAX( RCMAX, R( I ) )
         RCMIN = MIN( RCMIN, R( I ) )
   40 CONTINUE
      AMAX = RCMAX
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 50 I = 1, M
            IF( R( I ).EQ.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   50    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 60 I = 1, M
            R( I ) = ONE / MIN( MAX( R( I ), SMLNUM ), BIGNUM )
   60    CONTINUE
*
*        Compute ROWCND = min(R(I)) / max(R(I))
*
         ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
*     Compute column scale factors
*
      DO 70 J = 1, N
         C( J ) = ZERO
   70 CONTINUE
*
*     Find the maximum element in each column,
*     assuming the row scaling computed above.
*
      DO 90 J = 1, N
         DO 80 I = 1, M
            C( J ) = MAX( C( J ), ABS( A( I, J ) )*R( I ) )
   80    CONTINUE
   90 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
```

```
      DO 100 J = 1, N
         RCMIN = MIN( RCMIN, C( J ) )
         RCMAX = MAX( RCMAX, C( J ) )
  100 CONTINUE
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 110 J = 1, N
            IF( C( J ).EQ.ZERO ) THEN
               INFO = M + J
               RETURN
            END IF
  110    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 120 J = 1, N
            C( J ) = ONE / MIN( MAX( C( J ), SMLNUM ), BIGNUM )
  120    CONTINUE
*
*        Compute COLCND = min(C(J)) / max(C(J))
*
         COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
      RETURN
*
*     End of SGEEQU
*
      END
```

———-

33.6.4 lisp code

— **sgeequ.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun sgeequ (m n a lda r c rowcnd colcnd amax info)
    (declare (type (single-float) amax colcnd rowcnd)
             (type (array single-float (*)) c r a)
             (type (f2cl-lib:integer4) info lda n m))
```

```
(f2cl-lib:with-multi-array-data
    ((a single-float a-%data% a-%offset%)
     (r single-float r-%data% r-%offset%)
     (c single-float c-%data% c-%offset%))
  (prog ((bignum 0.0) (rcmax 0.0) (rcmin 0.0) (smlnum 0.0) (i 0) (j 0))
    (declare (type (single-float) bignum rcmax rcmin smlnum)
             (type (f2cl-lib:integer4) i j))
    (setf info 0)
    (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
          ((< lda
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
           (setf info -4)))
    (cond
     ((/= info 0) (xerbla "SGEEQU" (f2cl-lib:int-sub info))
      (go end_label)))
    (cond
     ((or (= m 0) (= n 0)) (setf rowcnd one) (setf colcnd one)
      (setf amax zero) (go end_label)))
    (setf smlnum (slamch "S"))
    (setf bignum (/ one smlnum))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%) zero)
       label10))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                    (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                         (abs
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%))))
           label20))
       label30))
    (setf rcmin bignum)
    (setf rcmax zero)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf rcmax
                (max rcmax
                     (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
        (setf rcmin
                (min rcmin
                     (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
       label40))
```

```
(setf amax rcmax)
(cond
 ((= rcmin zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref r (i) ((1 *))) zero) (setf info i)
        (go end_label)))
     label50)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
              (/ one
                 (min
                  (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                       smlnum)
                  bignum)))
     label60))
  (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) zero)
   label70))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                     (*
                      (abs
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%))
                      (f2cl-lib:fref r-%data% (i) ((1 *))
                                     r-%offset%))))
       label80))
   label90))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf rcmin
            (min rcmin
```

```
                        (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
            (setf rcmax
                    (max rcmax
                         (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
           label100))
         (cond
          ((= rcmin zero)
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
            (tagbody
              (cond
               ((= (f2cl-lib:fref c (j) ((1 *))) zero)
                (setf info (f2cl-lib:int-add m j)) (go end_label)))
             label110)))
          (t
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                      (/ one
                         (min
                          (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                               smlnum)
                          bignum)))
             label120))
           (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
         (go end_label)
        end_label
         (return (values nil nil nil nil nil nil rowcnd colcnd amax info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgeequ
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (single-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil
                                              fortran-to-lisp::rowcnd
                                              fortran-to-lisp::colcnd
                                              fortran-to-lisp::amax
```

```
                                        fortran-to-lisp::info)
                                      :calls 'nil)))
```

———-

33.7 sgeequb

33.7.1 Axiom unit tests

— **sgeequb.input** —

```
)set break resume
)sys rm -f sgeequb.output
)spool sgeequb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.7.2 Axiom help page

— **sgeequb.help** —

```
======================================================================
sgeequb examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgeequb (M, N, A, LDA, R, C, ROWCND, COLCND, AMAX, INFO)
           SGEEQUB

Function/Subroutine Documentation
   subroutine sgeequb (integer M, integer N, real, dimension( lda, * ) A,
```

```
      integer LDA, real, dimension( * ) R, real, dimension( * ) C, real
      ROWCND, real COLCND, real AMAX, integer INFO)
      SGEEQUB

Purpose:
   SGEEQUB computes row and column scalings intended to equilibrate an
   M-by-N matrix A and reduce its condition number.  R returns the row
   scale factors and C the column scale factors, chosen to try to make
   the largest element in each row and column of the matrix B with
   elements B(i,j)=R(i)*A(i,j)*C(j) have an absolute value of at most
   the radix.

   R(i) and C(j) are restricted to be a power of the radix between
   SMLNUM = smallest safe number and BIGNUM = largest safe number.  Use
   of these scaling factors is not guaranteed to reduce the condition
   number of A but works well in practice.

   This routine differs from SGEEQU by restricting the scaling factors
   to a power of the radix.  Baring over- and underflow, scaling by
   these factors introduces no additional rounding errors.  However, the
   scaled entries' magnitured are no longer approximately 1 but lie
   between sqrt(radix) and 1/sqrt(radix).

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   The M-by-N matrix whose equilibration factors are to be computed.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 R
   R is REAL array, dimension (M)
   If INFO = 0 or INFO > M, R contains the row scale factors for A.

 C
   C is REAL array, dimension (N)
   If INFO = 0,  C contains the column scale factors for A.

 ROWCND
```

```
  ROWCND is REAL
  If INFO = 0 or INFO > M, ROWCND contains the ratio of the
  smallest R(i) to the largest R(i).  If ROWCND >= 0.1 and
  AMAX is neither too large nor too small, it is not worth scaling by R.

COLCND
  COLCND is REAL
  If INFO = 0, COLCND contains the ratio of the smallest
  C(i) to the largest C(i).  If COLCND >= 0.1, it is not worth scaling by C.

AMAX
  AMAX is REAL
  Absolute value of largest matrix element.  If AMAX is very
  close to overflow or very close to underflow, the matrix should be scaled.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i,  and i is
    <= M:  the i-th row of A is exactly zero
    >  M:  the (i-M)-th column of A is exactly zero
```

———-

33.7.3 fortran code

— **sgeequb.f** —

```
*  ====================================================================
      SUBROUTINE SGEEQUB( M, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $                    INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
      REAL               AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), C( * ), R( * )
*     ..
*
*  ====================================================================
```

```
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               BIGNUM, RCMAX, RCMIN, SMLNUM, RADIX, LOGRDX
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, LOG
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEEQUB', -INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         ROWCND = ONE
         COLCND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Get machine constants.  Assume SMLNUM is a power of the radix.
*
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      RADIX = SLAMCH( 'B' )
```

```
      LOGRDX = LOG( RADIX )
*
*     Compute row scale factors.
*
      DO 10 I = 1, M
         R( I ) = ZERO
   10 CONTINUE
*
*     Find the maximum element in each row.
*
      DO 30 J = 1, N
         DO 20 I = 1, M
            R( I ) = MAX( R( I ), ABS( A( I, J ) ) )
   20    CONTINUE
   30 CONTINUE
      DO I = 1, M
         IF( R( I ).GT.ZERO ) THEN
            R( I ) = RADIX**INT( LOG( R( I ) ) / LOGRDX )
         END IF
      END DO
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 40 I = 1, M
         RCMAX = MAX( RCMAX, R( I ) )
         RCMIN = MIN( RCMIN, R( I ) )
   40 CONTINUE
      AMAX = RCMAX
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 50 I = 1, M
            IF( R( I ).EQ.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   50    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 60 I = 1, M
            R( I ) = ONE / MIN( MAX( R( I ), SMLNUM ), BIGNUM )
   60    CONTINUE
*
*        Compute ROWCND = min(R(I)) / max(R(I)).
```

```
*
         ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
*     Compute column scale factors
*
      DO 70 J = 1, N
         C( J ) = ZERO
   70 CONTINUE
*
*     Find the maximum element in each column,
*     assuming the row scaling computed above.
*
      DO 90 J = 1, N
         DO 80 I = 1, M
            C( J ) = MAX( C( J ), ABS( A( I, J ) )*R( I ) )
   80    CONTINUE
         IF( C( J ).GT.ZERO ) THEN
            C( J ) = RADIX**INT( LOG( C( J ) ) / LOGRDX )
         END IF
   90 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 100 J = 1, N
         RCMIN = MIN( RCMIN, C( J ) )
         RCMAX = MAX( RCMAX, C( J ) )
  100 CONTINUE
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 110 J = 1, N
            IF( C( J ).EQ.ZERO ) THEN
               INFO = M + J
               RETURN
            END IF
  110    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 120 J = 1, N
            C( J ) = ONE / MIN( MAX( C( J ), SMLNUM ), BIGNUM )
  120    CONTINUE
*
*        Compute COLCND = min(C(J)) / max(C(J)).
```

```
*
         COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
      RETURN
*
*     End of SGEEQUB
*
      END
```

33.7.4 lisp code

— sgeequb.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun sgeequb (m n a lda r c rowcnd colcnd amax info)
    (declare (type (single-float) amax colcnd rowcnd)
             (type (array single-float (*)) c r a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (r single-float r-%data% r-%offset%)
         (c single-float c-%data% c-%offset%))
      (prog ((bignum 0.0) (rcmax 0.0) (rcmin 0.0) (smlnum 0.0) (radix 0.0)
             (logrdx 0.0) (i 0) (j 0))
        (declare (type (single-float) bignum rcmax rcmin smlnum radix logrdx)
                 (type (f2cl-lib:integer4) i j))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "SGEEQUB" (f2cl-lib:int-sub info))
          (go end_label)))
        (cond
         ((or (= m 0) (= n 0)) (setf rowcnd one) (setf colcnd one)
          (setf amax zero) (go end_label)))
        (setf smlnum (slamch "S"))
        (setf bignum (/ one smlnum))
        (setf radix (slamch "B"))
        (setf logrdx (f2cl-lib:flog radix))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
              ((> i m) nil)
  (tagbody
    (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%) zero)
   label10))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                     (abs
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))))
       label20))
   label30))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref r (i) ((1 *))) zero)
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
              (expt radix
                    (f2cl-lib:int
                     (/
                      (f2cl-lib:flog
                       (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%))
                      logrdx))))))
   label100000))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf rcmax
            (max rcmax
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
    (setf rcmin
            (min rcmin
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
   label40))
(setf amax rcmax)
(cond
 ((= rcmin zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref r (i) ((1 *))) zero) (setf info i)
```

```
        (go end_label)))
     label50)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
              (/ one
                 (min
                  (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                       smlnum)
                  bignum)))
     label60))
  (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) zero)
   label70))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                     (*
                      (abs
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%))
                      (f2cl-lib:fref r-%data% (i) ((1 *))
                                     r-%offset%))))
       label80))
    (cond
     ((> (f2cl-lib:fref c (j) ((1 *))) zero)
      (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
              (expt radix
                    (f2cl-lib:int
                     (/
                      (f2cl-lib:flog
                       (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
                      logrdx))))))
   label90))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf rcmin
```

```
                    (min rcmin
                         (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
            (setf rcmax
                    (max rcmax
                         (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
           label100))
         (cond
          ((= rcmin zero)
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
            (tagbody
              (cond
               ((= (f2cl-lib:fref c (j) ((1 *))) zero)
                (setf info (f2cl-lib:int-add m j)) (go end_label)))
             label110)))
          (t
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                      (/ one
                         (min
                          (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                               smlnum)
                          bignum)))
             label120))
           (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
         (go end_label)
        end_label
         (return (values nil nil nil nil nil nil rowcnd colcnd amax info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgeequb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (single-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil
                                              fortran-to-lisp::rowcnd
                                              fortran-to-lisp::colcnd
```

```
                                          fortran-to-lisp::amax
                                          fortran-to-lisp::info)
                                        :calls 'nil)))
```

33.8 sgehd2

33.8.1 Axiom unit tests

— sgehd2.input —

```
)set break resume
)sys rm -f sgehd2.output
)spool sgehd2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

33.8.2 Axiom help page

— sgehd2.help —

```
=====================================================================
sgehd2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgehd2 (N, ILO, IHI, A, LDA, TAU, WORK, INFO)
           SGEHD2 reduces a general square matrix to upper Hessenberg form
           using an unblocked algorithm.
```

```
Function/Subroutine Documentation
   subroutine sgehd2 (integer N, integer ILO, integer IHI, real, dimension(
       lda, * ) A, integer LDA, real, dimension( * ) TAU, real, dimension( * )
       WORK, integer INFO)
       SGEHD2 reduces a general square matrix to upper Hessenberg form using
       an unblocked algorithm.

Purpose:
   SGEHD2 reduces a real general matrix A to upper Hessenberg form H by
   an orthogonal similarity transformation:  Q**T * A * Q = H .

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 ILO
   ILO is INTEGER

 IHI
   IHI is INTEGER

   It is assumed that A is already upper triangular in rows
   and columns 1:ILO-1 and IHI+1:N. ILO and IHI are normally
   set by a previous call to SGEBAL; otherwise they should be
   set to 1 and N respectively. See Further Details.
   1 <= ILO <= IHI <= max(1,N).

 A
   A is REAL array, dimension (LDA,N)
   On entry, the n by n general matrix to be reduced.
   On exit, the upper triangle and the first subdiagonal of A
   are overwritten with the upper Hessenberg matrix H, and the
   elements below the first subdiagonal, with the array TAU,
   represent the orthogonal matrix Q as a product of elementary
   reflectors. See Further Details.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 TAU
   TAU is REAL array, dimension (N-1)
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is REAL array, dimension (N)

 INFO
```

```
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.

 Further Details:

  The matrix Q is represented as a product of (ihi-ilo) elementary
  reflectors

     Q = H(ilo) H(ilo+1) . . . H(ihi-1).

  Each H(i) has the form

     H(i) = I - tau * v * v**T

  where tau is a real scalar, and v is a real vector with
  v(1:i) = 0, v(i+1) = 1 and v(ihi+1:n) = 0; v(i+2:ihi) is stored on
  exit in A(i+2:ihi,i), and tau in TAU(i).

  The contents of A are illustrated by the following example, with
  n = 7, ilo = 2 and ihi = 6:

            on entry,                        on exit,

            ( a   a   a   a   a   a   a )    (  a   a   h   h   h   h   a )
            (     a   a   a   a   a   a )    (      a   h   h   h   h   a )
            (     a   a   a   a   a   a )    (      h   h   h   h   h   h )
            (     a   a   a   a   a   a )    (      v2  h   h   h   h   h )
            (     a   a   a   a   a   a )    (      v2  v3  h   h   h   h )
            (     a   a   a   a   a   a )    (      v2  v3  v4  h   h   h )
            (                       a )    (                          a )

  where a denotes an element of the original matrix A, h denotes a
  modified element of the upper Hessenberg matrix H, and vi denotes an
  element of the vector defining H(i).
```

———-

33.8.3 fortran code

— **sgehd2.f** —

```
*  =====================================================================
      SUBROUTINE SGEHD2( N, ILO, IHI, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
```

```
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            IHI, ILO, INFO, LDA, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      REAL               AII
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLARF, SLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -2
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEHD2', -INFO )
         RETURN
      END IF
*
      DO 10 I = ILO, IHI - 1
*
*        Compute elementary reflector H(i) to annihilate A(i+2:ihi,i)
*
         CALL SLARFG( IHI-I, A( I+1, I ), A( MIN( I+2, N ), I ), 1,
```

```
     $                TAU( I ) )
         AII = A( I+1, I )
         A( I+1, I ) = ONE
*
*        Apply H(i) to A(1:ihi,i+1:ihi) from the right
*
         CALL SLARF( 'Right', IHI, IHI-I, A( I+1, I ), 1, TAU( I ),
     $               A( 1, I+1 ), LDA, WORK )
*
*        Apply H(i) to A(i+1:ihi,i+1:n) from the left
*
         CALL SLARF( 'Left', IHI-I, N-I, A( I+1, I ), 1, TAU( I ),
     $               A( I+1, I+1 ), LDA, WORK )
*
         A( I+1, I ) = AII
   10 CONTINUE
*
      RETURN
*
*     End of SGEHD2
*
      END
```

———-

33.8.4 lisp code

— **sgehd2.lisp** —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun sgehd2 (n ilo ihi a lda tau work info)
    (declare (type (array single-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lda ihi ilo n))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (tau single-float tau-%data% tau-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((aii 0.0) (i 0))
        (declare (type (single-float) aii)
                 (type (f2cl-lib:integer4) i))
        (setf info 0)
        (cond ((< n 0) (setf info -1))
              ((or (< ilo 1)
                   (> ilo
                      (max (the f2cl-lib:integer4 1)
                           (the f2cl-lib:integer4 n))))
```

```
          (setf info -2))
         ((or
           (< ihi
              (min (the f2cl-lib:integer4 ilo) (the f2cl-lib:integer4 n)))
           (> ihi n))
          (setf info -3))
         ((< lda
             (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -5)))
  (cond
   ((/= info 0) (xerbla "SGEHD2" (f2cl-lib:int-sub info))
    (go end_label)))
  (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                ((> i (f2cl-lib:int-add ihi (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (slarfg (f2cl-lib:int-sub ihi i)
       (f2cl-lib:array-slice a-%data% single-float ((+ i 1) i)
                             ((1 lda) (1 *)) a-%offset%)
       (f2cl-lib:array-slice a-%data% single-float
                             ((min (f2cl-lib:int-add i 2) n) i)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                             tau-%offset%))
      (setf aii
              (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                             ((1 lda) (1 *)) a-%offset%))
      (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                           ((1 lda) (1 *)) a-%offset%)
              one)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (slarf "Right" ihi (f2cl-lib:int-sub ihi i)
           (f2cl-lib:array-slice a-%data% single-float ((+ i 1) i)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                 tau-%offset%)
           (f2cl-lib:array-slice a-%data% single-float
                                 (1 (f2cl-lib:int-add i 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda work)
        (declare (ignore var-0 var-2 var-3 var-4 var-5 var-6 var-8))
        (when var-1 (setf ihi var-1))
        (when var-7 (setf lda var-7)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (slarf "Left" (f2cl-lib:int-sub ihi i) (f2cl-lib:int-sub n i)
           (f2cl-lib:array-slice a-%data% single-float ((+ i 1) i)
                                 ((1 lda) (1 *)) a-%offset%)
```

```
                1
                (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                      tau-%offset%)
                (f2cl-lib:array-slice a-%data% single-float
                                      ((+ i 1) (f2cl-lib:int-add i 1))
                                      ((1 lda) (1 *)) a-%offset%)
                lda work)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
              (when var-7 (setf lda var-7)))
            (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                                 ((1 lda) (1 *)) a-%offset%)
                  aii)
          label10))
        (go end_label)
       end_label
        (return (values nil nil ihi nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgehd2
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(nil nil fortran-to-lisp::ihi nil
                                             fortran-to-lisp::lda nil nil
                                             fortran-to-lisp::info)
                                           :calls 'nil)))
```

———-

33.9 sgehrd

33.9.1 Axiom unit tests

— sgehrd.input —

```
)set break resume
)sys rm -f sgehrd.output
)spool sgehrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.9.2 Axiom help page

— sgehrd.help —

```
=====================================================================
sgehrd examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgehrd (N, ILO, IHI, A, LDA, TAU, WORK, LWORK, INFO)
           SGEHRD

Function/Subroutine Documentation
   subroutine sgehrd (integer N, integer ILO, integer IHI, real, dimension(
       lda, * ) A, integer LDA, real, dimension( * ) TAU, real, dimension( * )
       WORK, integer LWORK, integer INFO)
       SGEHRD

Purpose:
   SGEHRD reduces a real general matrix A to upper Hessenberg form H by
   an orthogonal similarity transformation:  Q**T * A * Q = H .

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 ILO
   ILO is INTEGER
```

```
 IHI
   IHI is INTEGER

   It is assumed that A is already upper triangular in rows
   and columns 1:ILO-1 and IHI+1:N. ILO and IHI are normally
   set by a previous call to SGEBAL; otherwise they should be
   set to 1 and N respectively. See Further Details.
   1 <= ILO <= IHI <= N, if N > 0; ILO=1 and IHI=0, if N=0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the N-by-N general matrix to be reduced.
   On exit, the upper triangle and the first subdiagonal of A
   are overwritten with the upper Hessenberg matrix H, and the
   elements below the first subdiagonal, with the array TAU,
   represent the orthogonal matrix Q as a product of elementary
   reflectors. See Further Details.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 TAU
   TAU is REAL array, dimension (N-1)
   The scalar factors of the elementary reflectors (see Further
   Details). Elements 1:ILO-1 and IHI:N-1 of TAU are set to
   zero.

 WORK
   WORK is REAL array, dimension (LWORK)
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The length of the array WORK.  LWORK >= max(1,N).
   For good performance, LWORK should generally be larger.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value.

Further Details:

   The matrix Q is represented as a product of (ihi-ilo) elementary
```

```
reflectors

   Q = H(ilo) H(ilo+1) . . . H(ihi-1).

Each H(i) has the form

   H(i) = I - tau * v * v**T

where tau is a real scalar, and v is a real vector with
v(1:i) = 0, v(i+1) = 1 and v(ihi+1:n) = 0; v(i+2:ihi) is stored on
exit in A(i+2:ihi,i), and tau in TAU(i).

The contents of A are illustrated by the following example, with
n = 7, ilo = 2 and ihi = 6:

          on entry,                        on exit,

          ( a   a   a   a   a   a   a )    (  a   a   h   h   h   h   a )
          (     a   a   a   a   a   a )    (      a   h   h   h   h   a )
          (     a   a   a   a   a   a )    (      h   h   h   h   h   h )
          (     a   a   a   a   a   a )    (      v2  h   h   h   h   h )
          (     a   a   a   a   a   a )    (      v2  v3  h   h   h   h )
          (     a   a   a   a   a   a )    (      v2  v3  v4  h   h   h )
          (                       a )    (                          a )

where a denotes an element of the original matrix A, h denotes a
modified element of the upper Hessenberg matrix H, and vi denotes an
element of the vector defining H(i).

This file is a slight modification of LAPACK-3.0's DGEHRD
subroutine incorporating improvements proposed by Quintana-Orti and
Van de Geijn (2006). (See DLAHR2.)
```

———-

33.9.3 fortran code

— sgehrd.f —

```
*  =====================================================================
      SUBROUTINE SGEHRD( N, ILO, IHI, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
```

```
*     .. Scalar Arguments ..
      INTEGER            IHI, ILO, INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      REAL              A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            NBMAX, LDT, TSIZE
      PARAMETER          ( NBMAX = 64, LDT = NBMAX+1,
     $                     TSIZE = LDT*NBMAX )
      REAL              ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0,
     $                     ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWT, J, LDWORK, LWKOPT, NB,
     $                   NBMIN, NH, NX
      REAL              EI
*     ..
*     .. External Subroutines ..
      EXTERNAL           SAXPY, SGEHD2, SGEMM, SLAHR2, SLARFB, STRMM,
     $                   XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -2
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -8
```

```
      END IF
*
      IF( INFO.EQ.0 ) THEN
*
*       Compute the workspace requirements
*
         NB = MIN( NBMAX, ILAENV( 1, 'SGEHRD', ' ', N, ILO, IHI, -1 ) )
         LWKOPT = N*NB + TSIZE
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEHRD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Set elements 1:ILO-1 and IHI:N-1 of TAU to zero
*
      DO 10 I = 1, ILO - 1
         TAU( I ) = ZERO
   10 CONTINUE
      DO 20 I = MAX( 1, IHI ), N - 1
         TAU( I ) = ZERO
   20 CONTINUE
*
*     Quick return if possible
*
      NH = IHI - ILO + 1
      IF( NH.LE.1 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
*     Determine the block size
*
      NB = MIN( NBMAX, ILAENV( 1, 'SGEHRD', ' ', N, ILO, IHI, -1 ) )
      NBMIN = 2
      IF( NB.GT.1 .AND. NB.LT.NH ) THEN
*
*        Determine when to cross over from blocked to unblocked code
*        (last block is always handled by unblocked code)
*
         NX = MAX( NB, ILAENV( 3, 'SGEHRD', ' ', N, ILO, IHI, -1 ) )
         IF( NX.LT.NH ) THEN
*
*           Determine if workspace is large enough for blocked code
*
            IF( LWORK.LT.N*NB+TSIZE ) THEN
```

```
*
*             Not enough workspace to use optimal NB:  determine the
*             minimum value of NB, and reduce NB or force use of
*             unblocked code
*
              NBMIN = MAX( 2, ILAENV( 2, 'SGEHRD', ' ', N, ILO, IHI,
     $                -1 ) )
              IF( LWORK.GE.(N*NBMIN + TSIZE) ) THEN
                 NB = (LWORK-TSIZE) / N
              ELSE
                 NB = 1
              END IF
           END IF
        END IF
     END IF
     LDWORK = N
*
     IF( NB.LT.NBMIN .OR. NB.GE.NH ) THEN
*
*       Use unblocked code below
*
        I = ILO
*
     ELSE
*
*       Use blocked code
*
        IWT = 1 + N*NB
        DO 40 I = ILO, IHI - 1 - NX, NB
           IB = MIN( NB, IHI-I )
*
*          Reduce columns i:i+ib-1 to Hessenberg form, returning the
*          matrices V and T of the block reflector H = I - V*T*V**T
*          which performs the reduction, and also the matrix Y = A*V*T
*
           CALL SLAHR2( IHI, I, IB, A( 1, I ), LDA, TAU( I ),
     $                  WORK( IWT ), LDT, WORK, LDWORK )
*
*          Apply the block reflector H to A(1:ihi,i+ib:ihi) from the
*          right, computing  A := A - Y * V**T. V(i+ib,ib-1) must be set
*          to 1
*
           EI = A( I+IB, I+IB-1 )
           A( I+IB, I+IB-1 ) = ONE
           CALL SGEMM( 'No transpose', 'Transpose',
     $                 IHI, IHI-I-IB+1,
     $                 IB, -ONE, WORK, LDWORK, A( I+IB, I ), LDA, ONE,
     $                 A( 1, I+IB ), LDA )
           A( I+IB, I+IB-1 ) = EI
*
```

```
*           Apply the block reflector H to A(1:i,i+1:i+ib-1) from the
*           right
*
            CALL STRMM( 'Right', 'Lower', 'Transpose',
     $                  'Unit', I, IB-1,
     $                  ONE, A( I+1, I ), LDA, WORK, LDWORK )
            DO 30 J = 0, IB-2
               CALL SAXPY( I, -ONE, WORK( LDWORK*J+1 ), 1,
     $                     A( 1, I+J+1 ), 1 )
   30       CONTINUE
*
*           Apply the block reflector H to A(i+1:ihi,i+ib:n) from the
*           left
*
            CALL SLARFB( 'Left', 'Transpose', 'Forward',
     $                   'Columnwise',
     $                   IHI-I, N-I-IB+1, IB, A( I+1, I ), LDA,
     $                   WORK( IWT ), LDT, A( I+1, I+IB ), LDA,
     $                   WORK, LDWORK )
   40    CONTINUE
      END IF
*
*     Use unblocked code to reduce the rest of the matrix
*
      CALL SGEHD2( N, I, IHI, A, LDA, TAU, WORK, IINFO )
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of SGEHRD
*
      END
```

———-

33.9.4 lisp code

— sgehrd.lisp —

```
(let* ((nbmax 64) (ldt (+ nbmax 1)) (tsize (* ldt nbmax)) (zero 0.0) (one 1.0))
  (declare (type (f2cl-lib:integer4 64 64) nbmax)
           (type (f2cl-lib:integer4) ldt)
           (type (f2cl-lib:integer4) tsize)
           (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable nbmax ldt tsize zero one))
  (defun sgehrd (n ilo ihi a lda tau work lwork info)
    (declare (type (array single-float (*)) work tau a)
```

```
          (type (f2cl-lib:integer4) info lwork lda ihi ilo n))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (tau single-float tau-%data% tau-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((ei 0.0) (i 0) (ib 0) (iinfo 0) (iwt 0) (j 0) (ldwork 0)
             (lwkopt 0) (nb 0) (nbmin 0) (nh 0) (nx 0) (lquery nil))
        (declare (type (single-float) ei)
                 (type (f2cl-lib:integer4) i ib iinfo iwt j ldwork lwkopt nb
                  nbmin nh nx)
                 (type f2cl-lib:logical lquery))
        (setf info 0)
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond ((< n 0) (setf info -1))
              ((or (< ilo 1)
                   (> ilo
                      (max (the f2cl-lib:integer4 1)
                           (the f2cl-lib:integer4 n))))
               (setf info -2))
              ((or
                (< ihi
                   (min (the f2cl-lib:integer4 ilo) (the f2cl-lib:integer4 n)))
                (> ihi n))
               (setf info -3))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -5))
              ((and
                (< lwork
                   (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
                (not lquery))
               (setf info -8)))
        (cond
         ((= info 0)
          (setf nb
                  (min (the f2cl-lib:integer4 nbmax)
                       (the f2cl-lib:integer4
                            (multiple-value-bind
                                (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                                 var-6)
                                (ilaenv 1 "SGEHRD" " " n ilo ihi -1)
                              (declare (ignore var-0 var-1 var-2 var-6))
                              (setf n var-3)
                              (setf ilo var-4)
                              (setf ihi var-5)
                              ret-val))))
          (setf lwkopt (f2cl-lib:int-add (f2cl-lib:int-mul n nb) tsize))
          (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                  (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))))
        (cond
```

```
 ((/= info 0) (xerbla "SGEHRD" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (setf (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%) zero)
   label10))
(f2cl-lib:fdo (i
               (max (the f2cl-lib:integer4 1)
                    (the f2cl-lib:integer4 ihi))
               (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (setf (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%) zero)
   label20))
(setf nh (f2cl-lib:int-add (f2cl-lib:int-sub ihi ilo) 1))
(cond
 ((<= nh 1)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'single-float))
  (go end_label)))
(setf nb
        (min (the f2cl-lib:integer4 nbmax)
             (the f2cl-lib:integer4
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                       var-6)
                      (ilaenv 1 "SGEHRD" " " n ilo ihi -1)
                    (declare (ignore var-0 var-1 var-2 var-6))
                    (setf n var-3)
                    (setf ilo var-4)
                    (setf ihi var-5)
                    ret-val))))
(setf nbmin 2)
(cond
 ((and (> nb 1) (< nb nh))
  (setf nx
          (max (the f2cl-lib:integer4 nb)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "SGEHRD" " " n ilo ihi -1)
                      (declare (ignore var-0 var-1 var-2 var-6))
                      (setf n var-3)
                      (setf ilo var-4)
                      (setf ihi var-5)
                      ret-val))))
  (cond
   ((< nx nh)
```

```
    (cond
     ((< lwork (f2cl-lib:int-add (f2cl-lib:int-mul n nb) tsize))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4
                             var-5 var-6)
                            (ilaenv 2 "SGEHRD" " " n ilo ihi -1)
                          (declare (ignore var-0 var-1 var-2 var-6))
                          (setf n var-3)
                          (setf ilo var-4)
                          (setf ihi var-5)
                          ret-val))))
      (cond
       ((>= lwork (f2cl-lib:int-add (f2cl-lib:int-mul n nbmin) tsize))
        (setf nb (the f2cl-lib:integer4 (truncate (- lwork tsize) n))))
       (t (setf nb 1)))))))))
(setf ldwork n)
(cond ((or (< nb nbmin) (>= nb nh)) (setf i ilo))
      (t (setf iwt (f2cl-lib:int-add 1 (f2cl-lib:int-mul n nb)))
       (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i nb))
                     ((> i
                         (f2cl-lib:int-add ihi (f2cl-lib:int-sub 1)
                                           (f2cl-lib:int-sub nx)))
                      nil)
         (tagbody
           (setf ib
                   (min (the f2cl-lib:integer4 nb)
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-sub ihi i))))
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                var-9)
               (slahr2 ihi i ib
                (f2cl-lib:array-slice a-%data% single-float (1 i)
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice tau-%data% single-float (i)
                                      ((1 *)) tau-%offset%)
                (f2cl-lib:array-slice work-%data% single-float (iwt)
                                      ((1 *)) work-%offset%)
                ldt work ldwork)
             (declare (ignore var-3 var-5 var-6 var-8))
             (when var-0 (setf ihi var-0))
             (when var-1 (setf i var-1))
             (when var-2 (setf ib var-2))
             (when var-4 (setf lda var-4))
             (when var-7 (setf ldt var-7))
             (when var-9 (setf ldwork var-9)))
```

```
(setf ei
        (f2cl-lib:fref a-%data%
                       ((f2cl-lib:int-add i ib)
                        (f2cl-lib:int-sub
                         (f2cl-lib:int-add i ib) 1))
                       ((1 lda) (1 *)) a-%offset%))
(setf (f2cl-lib:fref a-%data%
                     ((f2cl-lib:int-add i ib)
                      (f2cl-lib:int-sub
                       (f2cl-lib:int-add i ib) 1))
                     ((1 lda) (1 *)) a-%offset%)
        one)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (sgemm "No transpose" "Transpose" ihi
     (f2cl-lib:int-add (f2cl-lib:int-sub ihi i ib) 1) ib
     (- one) work ldwork
     (f2cl-lib:array-slice a-%data% single-float
                           ((+ i ib) i) ((1 lda) (1 *))
                           a-%offset%)
     lda one
     (f2cl-lib:array-slice a-%data% single-float
                           (1 (f2cl-lib:int-add i ib))
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare
   (ignore var-0 var-1 var-3 var-5 var-6 var-8 var-11))
  (when var-2 (setf ihi var-2))
  (when var-4 (setf ib var-4))
  (when var-7 (setf ldwork var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf lda var-12)))
(setf (f2cl-lib:fref a-%data%
                     ((f2cl-lib:int-add i ib)
                      (f2cl-lib:int-sub
                       (f2cl-lib:int-add i ib) 1))
                     ((1 lda) (1 *)) a-%offset%)
        ei)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (strmm "Right" "Lower" "Transpose" "Unit" i
     (f2cl-lib:int-sub ib 1) one
     (f2cl-lib:array-slice a-%data% single-float ((+ i 1) i)
                           ((1 lda) (1 *)) a-%offset%)
     lda work ldwork)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9))
```

```
  (when var-4 (setf i var-4))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf ldwork var-10)))
(f2cl-lib:fdo (j 0 (f2cl-lib:int-add j 1))
              ((> j
                  (f2cl-lib:int-add ib
                                    (f2cl-lib:int-sub 2)))
               nil)
  (tagbody
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5)
        (saxpy i (- one)
         (f2cl-lib:array-slice work-%data% single-float
                               ((+
                                 (f2cl-lib:int-mul ldwork j)
                                 1))
                               ((1 *)) work-%offset%)
         1
         (f2cl-lib:array-slice a-%data% single-float
                               (1 (f2cl-lib:int-add i j 1))
                               ((1 lda) (1 *)) a-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4 var-5))
      (when var-0 (setf i var-0)))
   label30))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (slarfb "Left" "Transpose" "Forward" "Columnwise"
     (f2cl-lib:int-sub ihi i)
     (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1) ib
     (f2cl-lib:array-slice a-%data% single-float ((+ i 1) i)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% single-float (iwt)
                           ((1 *)) work-%offset%)
     ldt
     (f2cl-lib:array-slice a-%data% single-float
                           ((+ i 1) (f2cl-lib:int-add i ib))
                           ((1 lda) (1 *)) a-%offset%)
     lda work ldwork)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9
    var-11 var-13))
  (when var-6 (setf ib var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf ldt var-10))
  (when var-12 (setf lda var-12))
  (when var-14 (setf ldwork var-14)))
```

```
                 label40))))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (sgehd2 n i ihi a lda tau work iinfo)
          (declare (ignore var-0 var-1 var-3 var-5 var-6))
          (setf ihi var-2)
          (setf lda var-4)
          (setf iinfo var-7))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
        (go end_label)
       end_label
        (return (values n ilo ihi nil lda nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgehrd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n
                                              fortran-to-lisp::ilo
                                              fortran-to-lisp::ihi nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::sgehd2
                                              fortran-to-lisp::ilaenv))))
```

———-

33.10 sgelq2

33.10.1 Axiom unit tests

— sgelq2.input —

```
)set break resume
)sys rm -f sgelq2.output
)spool sgelq2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

33.10.2 Axiom help page

— sgelq2.help —

```
======================================================================
sgelq2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgelq2 (M, N, A, LDA, TAU, WORK, INFO)
           SGELQ2 computes the LQ factorization of a general rectangular
           matrix using an unblocked algorithm.

Function/Subroutine Documentation
   subroutine sgelq2 (integer M, integer N, real, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) TAU, real, dimension( * ) WORK,
       integer INFO)
       SGELQ2 computes the LQ factorization of a general rectangular matrix
       using an unblocked algorithm.

Purpose:
   SGELQ2 computes an LQ factorization of a real m by n matrix A: A = L * Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
```

```
   The number of columns of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the m by n matrix A.
   On exit, the elements on and below the diagonal of the array
   contain the m by min(m,n) lower trapezoidal matrix L (L is
   lower triangular if m <= n); the elements above the diagonal,
   with the array TAU, represent the orthogonal matrix Q as a
   product of elementary reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is REAL array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is REAL array, dimension (M)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

Further Details:

   The matrix Q is represented as a product of elementary reflectors

      Q = H(k) . . . H(2) H(1), where k = min(m,n).

   Each H(i) has the form

      H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(1:i-1) = 0 and v(i) = 1; v(i+1:n) is stored on exit in A(i,i+1:n),
   and tau in TAU(i).
```

———-

33.10.3 fortran code

— sgelq2.f —

```
*  =====================================================================
      SUBROUTINE SGELQ2( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      REAL               AII
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLARF, SLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGELQ2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
```

```
      DO 10 I = 1, K
*
*        Generate elementary reflector H(i) to annihilate A(i,i+1:n)
*
         CALL SLARFG( N-I+1, A( I, I ), A( I, MIN( I+1, N ) ), LDA,
     $                TAU( I ) )
         IF( I.LT.M ) THEN
*
*           Apply H(i) to A(i+1:m,i:n) from the right
*
            AII = A( I, I )
            A( I, I ) = ONE
            CALL SLARF( 'Right', M-I, N-I+1, A( I, I ), LDA, TAU( I ),
     $                  A( I+1, I ), LDA, WORK )
            A( I, I ) = AII
         END IF
   10 CONTINUE
      RETURN
*
*     End of SGELQ2
*
      END
```

———-

33.10.4 lisp code

— sgelq2.lisp —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun sgelq2 (m n a lda tau work info)
    (declare (type (array single-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (tau single-float tau-%data% tau-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((aii 0.0) (i 0) (k 0))
        (declare (type (single-float) aii)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
```

```
 ((/= info 0) (xerbla "SGELQ2" (f2cl-lib:int-sub info))
  (go end_label)))
(setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i k) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (slarfg (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
         (f2cl-lib:array-slice a-%data% single-float (i i)
                               ((1 lda) (1 *)) a-%offset%)
         (f2cl-lib:array-slice a-%data% single-float
                               (i
                                (min
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add i 1))
                                 (the f2cl-lib:integer4 n)))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                               tau-%offset%))
      (declare (ignore var-0 var-1 var-2 var-4))
      (when var-3 (setf lda var-3)))
    (cond
     ((< i m)
      (setf aii
              (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                             a-%offset%))
      (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
              one)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (slarf "Right" (f2cl-lib:int-sub m i)
           (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
           (f2cl-lib:array-slice a-%data% single-float (i i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                 tau-%offset%)
           (f2cl-lib:array-slice a-%data% single-float ((+ i 1) i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda work)
        (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8))
        (when var-4 (setf lda var-4))
        (when var-7 (setf lda var-7)))
      (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
              aii)))
   label10))
 (go end_label)
end_label
 (return (values nil nil nil lda nil nil info))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgelq2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::lda
                                              nil nil fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

33.11 sgelqf

33.11.1 Axiom unit tests

— **sgelqf.input** —

```
)set break resume
)sys rm -f sgelqf.output
)spool sgelqf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.11.2 Axiom help page

— **sgelqf.help** —

```
====================================================================
sgelqf examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgelqf (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           SGELQF

Function/Subroutine Documentation
   subroutine sgelqf (integer M, integer N, real, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) TAU, real, dimension( * ) WORK,
       integer LWORK, integer INFO)
       SGELQF

Purpose:
   SGELQF computes an LQ factorization of a real M-by-N matrix A: A = L * Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, the elements on and below the diagonal of the array
   contain the m-by-min(m,n) lower trapezoidal matrix L (L is
   lower triangular if m <= n); the elements above the diagonal,
   with the array TAU, represent the orthogonal matrix Q as a
   product of elementary reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is REAL array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
```

```
   WORK is REAL array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.  LWORK >= max(1,M).
   For optimum performance LWORK >= M*NB, where NB is the
   optimal blocksize.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details:

   The matrix Q is represented as a product of elementary reflectors

      Q = H(k) . . . H(2) H(1), where k = min(m,n).

   Each H(i) has the form

      H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(1:i-1) = 0 and v(i) = 1; v(i+1:n) is stored on exit in A(i,i+1:n),
   and tau in TAU(i).
```

———-

33.11.3 fortran code

— sgelqf.f —

```
*  =====================================================================
      SUBROUTINE SGELQF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
```

```
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, LDWORK, LWKOPT, NB,
     $                   NBMIN, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGELQ2, SLARFB, SLARFT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NB = ILAENV( 1, 'SGELQF', ' ', M, N, -1, -1 )
      LWKOPT = M*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, M ) .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGELQF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
```

```
*
      K = MIN( M, N )
      IF( K.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      NX = 0
      IWS = M
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'SGELQF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = M
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'SGELQF', ' ', M, N, -1,
     $                  -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially
*
         DO 10 I = 1, K - NX, NB
            IB = MIN( K-I+1, NB )
*
*           Compute the LQ factorization of the current block
*           A(i:i+ib-1,i:n)
*
            CALL SGELQ2( IB, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                   IINFO )
            IF( I+IB.LE.M ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i) H(i+1) . . . H(i+ib-1)
*
```

```
              CALL SLARFT( 'Forward', 'Rowwise', N-I+1, IB, A( I, I ),
     $                     LDA, TAU( I ), WORK, LDWORK )
*
*             Apply H to A(i+ib:m,i:n) from the right
*
              CALL SLARFB( 'Right', 'No transpose', 'Forward',
     $                     'Rowwise', M-I-IB+1, N-I+1, IB, A( I, I ),
     $                     LDA, WORK, LDWORK, A( I+IB, I ), LDA,
     $                     WORK( IB+1 ), LDWORK )
           END IF
   10    CONTINUE
      ELSE
         I = 1
      END IF
*
*     Use unblocked code to factor the last or only block.
*
      IF( I.LE.K )
     $   CALL SGELQ2( M-I+1, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                IINFO )
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of SGELQF
*
      END
```

———-

33.11.4 lisp code

— sgelqf.lisp —

```
(defun sgelqf (m n a lda tau work lwork info)
  (declare (type (array single-float (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a single-float a-%data% a-%offset%)
       (tau single-float tau-%data% tau-%offset%)
       (work single-float work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ldwork 0) (lwkopt 0) (nb 0)
           (nbmin 0) (nx 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) nx nbmin nb lwkopt ldwork k iws iinfo
                ib i))
      (setf info 0)
      (setf nb
```

```
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 1 "SGELQF" " " m n -1 -1)
          (declare (ignore var-0 var-1 var-2 var-5 var-6))
          (setf m var-3)
          (setf n var-4)
          ret-val))
(setf lwkopt (f2cl-lib:int-mul m nb))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
      ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -4))
      ((and
        (< lwork
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
        (not lquery))
       (setf info -7)))
(cond
 ((/= info 0) (xerbla "SGELQF" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(cond
 ((= k 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'single-float))
  (go end_label)))
(setf nbmin 2)
(setf nx 0)
(setf iws m)
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "SGELQF" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (setf m var-3)
                      (setf n var-4)
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork m) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
```

```
                (max (the f2cl-lib:integer4 2)
                     (the f2cl-lib:integer4
                          (multiple-value-bind
                              (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                               var-6)
                              (ilaenv 2 "SGELQF" " " m n -1 -1)
                            (declare
                             (ignore var-0 var-1 var-2 var-5 var-6))
                            (setf m var-3)
                            (setf n var-4)
                            ret-val)))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub nx))) nil)
    (tagbody
      (setf ib
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
               (the f2cl-lib:integer4 nb)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (sgelq2 ib (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
           (f2cl-lib:array-slice a-%data% single-float (i i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                 tau-%offset%)
           work iinfo)
        (declare (ignore var-0 var-1 var-2 var-4 var-5))
        (setf lda var-3)
        (setf iinfo var-6))
      (cond
       ((<= (f2cl-lib:int-add i ib) m)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (slarft "Forward" "Rowwise"
             (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) ib
             (f2cl-lib:array-slice a-%data% single-float (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                   tau-%offset%)
             work ldwork)
          (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
          (when var-3 (setf ib var-3))
          (when var-5 (setf lda var-5))
          (when var-8 (setf ldwork var-8)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
                  var-10 var-11 var-12 var-13 var-14)
                 (slarfb "Right" "No transpose" "Forward" "Rowwise"
                  (f2cl-lib:int-add (f2cl-lib:int-sub m i ib) 1)
                  (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) ib
                  (f2cl-lib:array-slice a-%data% single-float (i i)
                                        ((1 lda) (1 *)) a-%offset%)
                  lda work ldwork
                  (f2cl-lib:array-slice a-%data% single-float ((+ i ib) i)
                                        ((1 lda) (1 *)) a-%offset%)
                  lda
                  (f2cl-lib:array-slice work-%data% single-float ((+ ib 1))
                                        ((1 *)) work-%offset%)
                  ldwork)
                (declare
                 (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
                  var-13))
                (when var-6 (setf ib var-6))
                (when var-8 (setf lda var-8))
                (when var-10 (setf ldwork var-10))
                (when var-12 (setf lda var-12))
                (when var-14 (setf ldwork var-14)))))
           label10)))
       (t (setf i 1)))
      (if (<= i k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (sgelq2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
               (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
               (f2cl-lib:array-slice a-%data% single-float (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                     tau-%offset%)
               work iinfo)
            (declare (ignore var-0 var-1 var-2 var-4 var-5))
            (setf lda var-3)
            (setf iinfo var-6)))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 iws) 'single-float))
      (go end_label)
     end_label
      (return (values m n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgelqf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
```

```
                                        (array single-float (*))
                                        (fortran-to-lisp::integer4)
                                        (array single-float (*))
                                        (array single-float (*))
                                        (fortran-to-lisp::integer4)
                                        (fortran-to-lisp::integer4))
                                      :return-values
                                      '(fortran-to-lisp::m
                                        fortran-to-lisp::n nil
                                        fortran-to-lisp::lda nil nil nil
                                        fortran-to-lisp::info)
                                      :calls
                                      '(fortran-to-lisp::sgelq2
                                        fortran-to-lisp::ilaenv))))
```

———-

33.12 sgemqrt

33.12.1 Axiom unit tests

— **sgemqrt.input** —

```
)set break resume
)sys rm -f sgemqrt.output
)spool sgemqrt.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.12.2 Axiom help page

— **sgemqrt.help** —

```
==================================================================
sgemqrt examples
==================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgemqrt (SIDE, TRANS, M, N, K, NB, V, LDV, T, LDT, C, LDC,
           WORK, INFO)
           SGEMQRT

Function/Subroutine Documentation
   subroutine sgemqrt (character SIDE, character TRANS, integer M, integer N,
       integer K, integer NB, real, dimension( ldv, * ) V, integer LDV, real,
       dimension( ldt, * ) T, integer LDT, real, dimension( ldc, * ) C,
       integer LDC, real, dimension( * ) WORK, integer INFO)
       SGEMQRT

Purpose:
   SGEMQRT overwrites the general real M-by-N matrix C with

                              SIDE = 'L'     SIDE = 'R'
            TRANS = 'N':      Q C            C Q
            TRANS = 'T':   Q**T C            C Q**T

   where Q is a real orthogonal matrix defined as the product of K
   elementary reflectors:

                  Q = H(1) H(2) . . . H(K) = I - V T V**T

   generated using the compact WY representation as returned by SGEQRT.

   Q is of order M if SIDE = 'L' and of order N  if SIDE = 'R'.

Parameters:

 SIDE
   SIDE is CHARACTER*1
   = 'L': apply Q or Q**T from the Left;
   = 'R': apply Q or Q**T from the Right.

 TRANS
   TRANS is CHARACTER*1
   = 'N':  No transpose, apply Q;
   = 'T':  Transpose, apply Q**T.

 M
   M is INTEGER
   The number of rows of the matrix C. M >= 0.
```

```
N
  N is INTEGER
  The number of columns of the matrix C. N >= 0.

K
  K is INTEGER
  The number of elementary reflectors whose product defines the matrix Q.
  If SIDE = 'L', M >= K >= 0;
                   if SIDE = 'R', N >= K >= 0.

NB
  NB is INTEGER
  The block size used for the storage of T.  K >= NB >= 1.
  This must be the same value of NB used to generate T in CGEQRT.

V
  V is REAL array, dimension (LDV,K)
  The i-th column must contain the vector which defines the
  elementary reflector H(i), for i = 1,2,...,k, as returned by
  CGEQRT in the first K columns of its array argument A.

LDV
  LDV is INTEGER
  The leading dimension of the array V.
  If SIDE = 'L', LDA >= max(1,M);
  if SIDE = 'R', LDA >= max(1,N).

T
  T is REAL array, dimension (LDT,K)
  The upper triangular factors of the block reflectors
  as returned by CGEQRT, stored as a NB-by-N matrix.

LDT
  LDT is INTEGER
  The leading dimension of the array T.  LDT >= NB.

C
  C is REAL array, dimension (LDC,N)
  On entry, the M-by-N matrix C.
  On exit, C is overwritten by Q C, Q**T C, C Q**T or C Q.

LDC
  LDC is INTEGER
  The leading dimension of the array C. LDC >= max(1,M).

WORK
  WORK is REAL array. The dimension of WORK is
  N*NB if SIDE = 'L', or  M*NB if SIDE = 'R'.

INFO
```

```
INFO is INTEGER
= 0:  successful exit
< 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

33.12.3 fortran code

— sgemqrt.f —

```
*  =====================================================================
      SUBROUTINE SGEMQRT( SIDE, TRANS, M, N, K, NB, V, LDV, T, LDT,
     $                    C, LDC, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER SIDE, TRANS
      INTEGER   INFO, K, LDV, LDC, M, N, NB, LDT
*     ..
*     .. Array Arguments ..
      REAL   V( LDV, * ), C( LDC, * ), T( LDT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     ..
*     .. Local Scalars ..
      LOGICAL            LEFT, RIGHT, TRAN, NOTRAN
      INTEGER            I, IB, LDWORK, KF, Q
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, SLARFB
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     .. Test the input arguments ..
```

```
*
      INFO   = 0
      LEFT   = LSAME( SIDE,  'L' )
      RIGHT  = LSAME( SIDE,  'R' )
      TRAN   = LSAME( TRANS, 'T' )
      NOTRAN = LSAME( TRANS, 'N' )
*
      IF( LEFT ) THEN
         LDWORK = MAX( 1, N )
         Q = M
      ELSE IF ( RIGHT ) THEN
         LDWORK = MAX( 1, M )
         Q = N
      END IF
      IF( .NOT.LEFT .AND. .NOT.RIGHT ) THEN
         INFO = -1
      ELSE IF( .NOT.TRAN .AND. .NOT.NOTRAN ) THEN
         INFO = -2
      ELSE IF( M.LT.0 ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( K.LT.0 .OR. K.GT.Q ) THEN
         INFO = -5
      ELSE IF( NB.LT.1 .OR. (NB.GT.K .AND. K.GT.0)) THEN
         INFO = -6
      ELSE IF( LDV.LT.MAX( 1, Q ) ) THEN
         INFO = -8
      ELSE IF( LDT.LT.NB ) THEN
         INFO = -10
      ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
         INFO = -12
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEMQRT', -INFO )
         RETURN
      END IF
*
*     .. Quick return if possible ..
*
      IF( M.EQ.0 .OR. N.EQ.0 .OR. K.EQ.0 ) RETURN
*
      IF( LEFT .AND. TRAN ) THEN
*
         DO I = 1, K, NB
            IB = MIN( NB, K-I+1 )
            CALL SLARFB( 'L', 'T', 'F', 'C', M-I+1, N, IB,
     $                   V( I, I ), LDV, T( 1, I ), LDT,
     $                   C( I, 1 ), LDC, WORK, LDWORK )
```

```
         END DO
*
      ELSE IF( RIGHT .AND. NOTRAN ) THEN
*
         DO I = 1, K, NB
            IB = MIN( NB, K-I+1 )
            CALL SLARFB( 'R', 'N', 'F', 'C', M, N-I+1, IB,
     $                   V( I, I ), LDV, T( 1, I ), LDT,
     $                   C( 1, I ), LDC, WORK, LDWORK )
         END DO
*
      ELSE IF( LEFT .AND. NOTRAN ) THEN
*
         KF = ((K-1)/NB)*NB+1
         DO I = KF, 1, -NB
            IB = MIN( NB, K-I+1 )
            CALL SLARFB( 'L', 'N', 'F', 'C', M-I+1, N, IB,
     $                   V( I, I ), LDV, T( 1, I ), LDT,
     $                   C( I, 1 ), LDC, WORK, LDWORK )
         END DO
*
      ELSE IF( RIGHT .AND. TRAN ) THEN
*
         KF = ((K-1)/NB)*NB+1
         DO I = KF, 1, -NB
            IB = MIN( NB, K-I+1 )
            CALL SLARFB( 'R', 'T', 'F', 'C', M, N-I+1, IB,
     $                   V( I, I ), LDV, T( 1, I ), LDT,
     $                   C( 1, I ), LDC, WORK, LDWORK )
         END DO
*
      END IF
*
      RETURN
*
*     End of SGEMQRT
*
      END
```

———-

33.12.4 lisp code

— sgemqrt.lisp —

```
(defun sgemqrt (side trans m n k nb v ldv t$ ldt c ldc work info)
  (declare (type (array single-float (*)) work c t$ v)
           (type (f2cl-lib:integer4) info ldc ldt ldv nb k n m)
```

```
          (type (string *) trans side))
  (f2cl-lib:with-multi-array-data
      ((side character side-%data% side-%offset%)
       (trans character trans-%data% trans-%offset%)
       (v single-float v-%data% v-%offset%)
       (t$ single-float t$-%data% t$-%offset%)
       (c single-float c-%data% c-%offset%)
       (work single-float work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (ldwork 0) (kf 0) (q 0) (left nil) (right nil)
           (tran nil) (notran nil))
      (declare (type f2cl-lib:logical notran tran right left)
               (type (f2cl-lib:integer4) q kf ldwork ib i))
      (setf info 0)
      (setf left
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame side "L")
                (declare (ignore var-1))
                (when var-0 (setf side var-0))
                ret-val))
      (setf right
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame side "R")
                (declare (ignore var-1))
                (when var-0 (setf side var-0))
                ret-val))
      (setf tran
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "T")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val))
      (setf notran
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "N")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val))
      (cond
       (left
        (setf ldwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf q m))
       (right
        (setf ldwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
        (setf q n)))
      (cond ((and (not left) (not right)) (setf info -1))
            ((and (not tran) (not notran)) (setf info -2))
            ((< m 0) (setf info -3)) ((< n 0) (setf info -4))
            ((or (< k 0) (> k q)) (setf info -5))
            ((or (< nb 1) (and (> nb k) (> k 0))) (setf info -6))
            ((< ldv (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 q)))
```

```
        (setf info -8))
       ((< ldt nb) (setf info -10))
       ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
        (setf info -12)))
(cond
 ((/= info 0) (xerbla "SGEMQRT" (f2cl-lib:int-sub info)) (go end_label)))
(if (or (= m 0) (= n 0) (= k 0))
    (go end_label))
(cond
 ((and left tran)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                ((> i k) nil)
    (tagbody
      (setf ib
              (min (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13 var-14)
          (slarfb "L" "T" "F" "C"
           (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) n ib
           (f2cl-lib:array-slice v-%data% single-float (i i)
                                 ((1 ldv) (1 *)) v-%offset%)
           ldv
           (f2cl-lib:array-slice t$-%data% single-float (1 i)
                                 ((1 ldt) (1 *)) t$-%offset%)
           ldt
           (f2cl-lib:array-slice c-%data% single-float (i 1)
                                 ((1 ldc) (1 *)) c-%offset%)
           ldc work ldwork)
        (declare
         (ignore var-0 var-1 var-2 var-3 var-4 var-7 var-9 var-11
          var-13))
        (when var-5 (setf n var-5))
        (when var-6 (setf ib var-6))
        (when var-8 (setf ldv var-8))
        (when var-10 (setf ldt var-10))
        (when var-12 (setf ldc var-12))
        (when var-14 (setf ldwork var-14)))
    label100000)))
 ((and right notran)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                ((> i k) nil)
    (tagbody
      (setf ib
              (min (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
      (multiple-value-bind
```

```
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11 var-12 var-13 var-14)
            (slarfb "R" "N" "F" "C" m
             (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) ib
             (f2cl-lib:array-slice v-%data% single-float (i i)
                                   ((1 ldv) (1 *)) v-%offset%)
             ldv
             (f2cl-lib:array-slice t$-%data% single-float (1 i)
                                   ((1 ldt) (1 *)) t$-%offset%)
             ldt
             (f2cl-lib:array-slice c-%data% single-float (1 i)
                                   ((1 ldc) (1 *)) c-%offset%)
             ldc work ldwork)
          (declare
           (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9 var-11
            var-13))
          (when var-4 (setf m var-4))
          (when var-6 (setf ib var-6))
          (when var-8 (setf ldv var-8))
          (when var-10 (setf ldt var-10))
          (when var-12 (setf ldc var-12))
          (when var-14 (setf ldwork var-14)))
      label100001)))
  ((and left notran)
   (setf kf (+ (* (the f2cl-lib:integer4 (truncate (- k 1) nb)) nb) 1))
   (f2cl-lib:fdo (i kf (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
                 ((> i 1) nil)
     (tagbody
       (setf ib
               (min (the f2cl-lib:integer4 nb)
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10 var-11 var-12 var-13 var-14)
           (slarfb "L" "N" "F" "C"
            (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) n ib
            (f2cl-lib:array-slice v-%data% single-float (i i)
                                  ((1 ldv) (1 *)) v-%offset%)
            ldv
            (f2cl-lib:array-slice t$-%data% single-float (1 i)
                                  ((1 ldt) (1 *)) t$-%offset%)
            ldt
            (f2cl-lib:array-slice c-%data% single-float (i 1)
                                  ((1 ldc) (1 *)) c-%offset%)
            ldc work ldwork)
          (declare
           (ignore var-0 var-1 var-2 var-3 var-4 var-7 var-9 var-11
            var-13))
          (when var-5 (setf n var-5))
```

```
              (when var-6 (setf ib var-6))
              (when var-8 (setf ldv var-8))
              (when var-10 (setf ldt var-10))
              (when var-12 (setf ldc var-12))
              (when var-14 (setf ldwork var-14)))
           label100002)))
       ((and right tran)
        (setf kf (+ (* (the f2cl-lib:integer4 (truncate (- k 1) nb)) nb) 1))
        (f2cl-lib:fdo (i kf (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
                      ((> i 1) nil)
          (tagbody
            (setf ib
                    (min (the f2cl-lib:integer4 nb)
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                 var-10 var-11 var-12 var-13 var-14)
                (slarfb "R" "T" "F" "C" m
                 (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) ib
                 (f2cl-lib:array-slice v-%data% single-float (i i)
                                       ((1 ldv) (1 *)) v-%offset%)
                 ldv
                 (f2cl-lib:array-slice t$-%data% single-float (1 i)
                                       ((1 ldt) (1 *)) t$-%offset%)
                 ldt
                 (f2cl-lib:array-slice c-%data% single-float (1 i)
                                       ((1 ldc) (1 *)) c-%offset%)
                 ldc work ldwork)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9 var-11
                var-13))
              (when var-4 (setf m var-4))
              (when var-6 (setf ib var-6))
              (when var-8 (setf ldv var-8))
              (when var-10 (setf ldt var-10))
              (when var-12 (setf ldc var-12))
              (when var-14 (setf ldwork var-14)))
           label100003))))
      (go end_label)
     end_label
      (return
       (values side trans m n nil nil nil ldv nil ldt nil ldc nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgemqrt
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
```

```
'((string) (string)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::side
  fortran-to-lisp::trans
  fortran-to-lisp::m
  fortran-to-lisp::n nil nil nil
  fortran-to-lisp::ldv nil
  fortran-to-lisp::ldt nil
  fortran-to-lisp::ldc nil
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

33.13 sgeql2

33.13.1 Axiom unit tests

— sgeql2.input —

```
)set break resume
)sys rm -f sgeql2.output
)spool sgeql2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.13.2 Axiom help page

— sgeql2.help —

```
==================================================================
sgeql2 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgeql2 (M, N, A, LDA, TAU, WORK, INFO)
           SGEQL2 computes the QL factorization of a general rectangular
           matrix using an unblocked algorithm.

Function/Subroutine Documentation
   subroutine sgeql2 (integer M, integer N, real, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) TAU, real, dimension( * ) WORK,
       integer INFO)
       SGEQL2 computes the QL factorization of a general rectangular matrix
       using an unblocked algorithm.

Purpose:
   SGEQL2 computes a QL factorization of a real m by n matrix A: A = Q * L.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the m by n matrix A.
   On exit, if m >= n, the lower triangle of the subarray
   A(m-n+1:m,1:n) contains the n by n lower triangular matrix L;
   if m <= n, the elements on and below the (n-m)-th
   superdiagonal contain the m by n lower trapezoidal matrix L;
   the remaining elements, with the array TAU, represent the
   orthogonal matrix Q as a product of elementary reflectors
   (see Further Details).
```

```
 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is REAL array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is REAL array, dimension (N)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

Further Details:

   The matrix Q is represented as a product of elementary reflectors

      Q = H(k) . . . H(2) H(1), where k = min(m,n).

   Each H(i) has the form

      H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(m-k+i+1:m) = 0 and v(m-k+i) = 1; v(1:m-k+i-1) is stored on exit in
   A(1:m-k+i-1,n-k+i), and tau in TAU(i).
```

———-

33.13.3 fortran code

— **sgeql2.f** —

```
*  =====================================================================
      SUBROUTINE SGEQL2( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
```

```
*     .. Array Arguments ..
      REAL               A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      REAL               AII
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLARF, SLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEQL2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = K, 1, -1
*
*        Generate elementary reflector H(i) to annihilate
*        A(1:m-k+i-1,n-k+i)
*
         CALL SLARFG( M-K+I, A( M-K+I, N-K+I ), A( 1, N-K+I ), 1,
     $                TAU( I ) )
*
*        Apply H(i) to A(1:m-k+i,1:n-k+i-1) from the left
*
         AII = A( M-K+I, N-K+I )
```

```
         A( M-K+I, N-K+I ) = ONE
         CALL SLARF( 'Left', M-K+I, N-K+I-1, A( 1, N-K+I ), 1, TAU( I ),
     $               A, LDA, WORK )
         A( M-K+I, N-K+I ) = AII
   10 CONTINUE
      RETURN
*
*     End of SGEQL2
*
      END
```

———-

33.13.4 lisp code

— **sgeql2.lisp** —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun sgeql2 (m n a lda tau work info)
    (declare (type (array single-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (tau single-float tau-%data% tau-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((aii 0.0) (i 0) (k 0))
        (declare (type (single-float) aii)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "SGEQL2" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (f2cl-lib:fdo (i k (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (slarfg (f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
             (f2cl-lib:array-slice a-%data% single-float
                                   ((+ m (f2cl-lib:int-sub k) i)
                                    (f2cl-lib:int-add (f2cl-lib:int-sub n k)
                                                      i))
                                   ((1 lda) (1 *)) a-%offset%)
```

```
             (f2cl-lib:array-slice a-%data% single-float
                                   (1
                                    (f2cl-lib:int-add (f2cl-lib:int-sub n k)
                                                      i))
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                   tau-%offset%))
            (setf aii
                    (f2cl-lib:fref a-%data%
                                   ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                    (f2cl-lib:int-add (f2cl-lib:int-sub n k)
                                                      i))
                                   ((1 lda) (1 *)) a-%offset%))
            (setf (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                  (f2cl-lib:int-add (f2cl-lib:int-sub n k) i))
                                 ((1 lda) (1 *)) a-%offset%)
                    one)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (slarf "Left" (f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                 (f2cl-lib:int-sub (f2cl-lib:int-add (f2cl-lib:int-sub n k) i)
                                   1)
                 (f2cl-lib:array-slice a-%data% single-float
                                       (1
                                        (f2cl-lib:int-add
                                         (f2cl-lib:int-sub n k) i))
                                       ((1 lda) (1 *)) a-%offset%)
                 1
                 (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                       tau-%offset%)
                 a lda work)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
              (when var-7 (setf lda var-7)))
            (setf (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                  (f2cl-lib:int-add (f2cl-lib:int-sub n k) i))
                                 ((1 lda) (1 *)) a-%offset%)
                    aii)
           label10))
         (go end_label)
        end_label
         (return (values nil nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgeql2
```

```
        fortran-to-lisp::*f2cl-function-info*)
  (fortran-to-lisp::make-f2cl-finfo :arg-types
                                    '((fortran-to-lisp::integer4)
                                      (fortran-to-lisp::integer4)
                                      (array single-float (*))
                                      (fortran-to-lisp::integer4)
                                      (array single-float (*))
                                      (array single-float (*))
                                      (fortran-to-lisp::integer4))
                                    :return-values
                                    '(nil nil nil fortran-to-lisp::lda
                                      nil nil fortran-to-lisp::info)
                                    :calls 'nil)))
```

———-

33.14 sgeqlf

33.14.1 Axiom unit tests

— **sgeqlf.input** —

```
)set break resume
)sys rm -f sgeqlf.output
)spool sgeqlf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.14.2 Axiom help page

— **sgeqlf.help** —

```
======================================================================
sgeqlf examples
======================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgeqlf (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           SGEQLF

Function/Subroutine Documentation
   subroutine sgeqlf (integer M, integer N, real, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) TAU, real, dimension( * ) WORK,
       integer LWORK, integer INFO)
       SGEQLF

Purpose:
   SGEQLF computes a QL factorization of a real M-by-N matrix A: A = Q * L.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit,
   if m >= n, the lower triangle of the subarray
   A(m-n+1:m,1:n) contains the N-by-N lower triangular matrix L;
   if m <= n, the elements on and below the (n-m)-th
   superdiagonal contain the M-by-N lower trapezoidal matrix L;
   the remaining elements, with the array TAU, represent the
   orthogonal matrix Q as a product of elementary reflectors
   (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is REAL array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is REAL array, dimension (MAX(1,LWORK))
```

```
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.  LWORK >= max(1,N).
   For optimum performance LWORK >= N*NB, where NB is the
   optimal blocksize.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details:

   The matrix Q is represented as a product of elementary reflectors

      Q = H(k) . . . H(2) H(1), where k = min(m,n).

   Each H(i) has the form

      H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(m-k+i+1:m) = 0 and v(m-k+i) = 1; v(1:m-k+i-1) is stored on exit in
   A(1:m-k+i-1,n-k+i), and tau in TAU(i).
```

33.14.3 fortran code

— sgeqlf.f —

```
*  ===================================================================
      SUBROUTINE SGEQLF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
```

```
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, KI, KK, LDWORK, LWKOPT,
     $                   MU, NB, NBMIN, NU, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEQL2, SLARFB, SLARFT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
*
      IF( INFO.EQ.0 ) THEN
         K = MIN( M, N )
         IF( K.EQ.0 ) THEN
            LWKOPT = 1
         ELSE
            NB = ILAENV( 1, 'SGEQLF', ' ', M, N, -1, -1 )
            LWKOPT = N*NB
         END IF
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
            INFO = -7
         END IF
```

```
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEQLF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( K.EQ.0 ) THEN
         RETURN
      END IF
*
      NBMIN = 2
      NX = 1
      IWS = N
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'SGEQLF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = N
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'SGEQLF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially.
*        The last kk columns are handled by the block method.
*
         KI = ( ( K-NX-1 ) / NB )*NB
         KK = MIN( K, KI+NB )
*
         DO 10 I = K - KK + KI + 1, K - KK + 1, -NB
```

```
            IB = MIN( K-I+1, NB )
*
*           Compute the QL factorization of the current block
*           A(1:m-k+i+ib-1,n-k+i:n-k+i+ib-1)
*
            CALL SGEQL2( M-K+I+IB-1, IB, A( 1, N-K+I ), LDA, TAU( I ),
     $                   WORK, IINFO )
            IF( N-K+I.GT.1 ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i+ib-1) . . . H(i+1) H(i)
*
               CALL SLARFT( 'Backward', 'Columnwise', M-K+I+IB-1, IB,
     $                      A( 1, N-K+I ), LDA, TAU( I ), WORK, LDWORK )
*
*              Apply H**T to A(1:m-k+i+ib-1,1:n-k+i-1) from the left
*
               CALL SLARFB( 'Left', 'Transpose', 'Backward',
     $                      'Columnwise', M-K+I+IB-1, N-K+I-1, IB,
     $                      A( 1, N-K+I ), LDA, WORK, LDWORK, A, LDA,
     $                      WORK( IB+1 ), LDWORK )
            END IF
   10    CONTINUE
         MU = M - K + I + NB - 1
         NU = N - K + I + NB - 1
      ELSE
         MU = M
         NU = N
      END IF
*
*     Use unblocked code to factor the last or only block
*
      IF( MU.GT.0 .AND. NU.GT.0 )
     $   CALL SGEQL2( MU, NU, A, LDA, TAU, WORK, IINFO )
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of SGEQLF
*
      END
```

———-

33.14.4 lisp code

— sgeqlf.lisp —

```
(defun sgeqlf (m n a lda tau work lwork info)
  (declare (type (array single-float (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a single-float a-%data% a-%offset%)
       (tau single-float tau-%data% tau-%offset%)
       (work single-float work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ki 0) (kk 0) (ldwork 0)
           (lwkopt 0) (mu 0) (nb 0) (nbmin 0) (nu 0) (nx 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) nx nu nbmin nb mu lwkopt ldwork kk ki
                k iws iinfo ib i))
      (setf info 0)
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4)))
      (cond
       ((= info 0)
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (cond ((= k 0) (setf lwkopt 1))
              (t
               (setf nb
                       (multiple-value-bind
                           (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                           (ilaenv 1 "SGEQLF" " " m n -1 -1)
                         (declare (ignore var-0 var-1 var-2 var-5 var-6))
                         (setf m var-3)
                         (setf n var-4)
                         ret-val))
               (setf lwkopt (f2cl-lib:int-mul n nb))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
        (cond
         ((and
           (< lwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
           (not lquery))
          (setf info -7)))))
      (cond
       ((/= info 0) (xerbla "SGEQLF" (f2cl-lib:int-sub info)) (go end_label))
       (lquery (go end_label)))
      (cond ((= k 0) (go end_label)))
      (setf nbmin 2)
      (setf nx 1)
      (setf iws n)
      (cond
       ((and (> nb 1) (< nb k))
        (setf nx
                (max (the f2cl-lib:integer4 0)
                     (the f2cl-lib:integer4
```

```
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "SGEQLF" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (setf m var-3)
                      (setf n var-4)
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork n) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
                            (ilaenv 2 "SGEQLF" " " m n -1 -1)
                          (declare
                           (ignore var-0 var-1 var-2 var-5 var-6))
                          (setf m var-3)
                          (setf n var-4)
                          ret-val)))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (setf ki (* (the f2cl-lib:integer4 (truncate (- k nx 1) nb)) nb))
  (setf kk
          (min (the f2cl-lib:integer4 k)
               (the f2cl-lib:integer4 (f2cl-lib:int-add ki nb))))
  (f2cl-lib:fdo (i (f2cl-lib:int-add k (f2cl-lib:int-sub kk) ki 1)
                 (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
                ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub kk) 1)) nil)
    (tagbody
      (setf ib
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
               (the f2cl-lib:integer4 nb)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (sgeql2
           (f2cl-lib:int-sub
            (f2cl-lib:int-add (f2cl-lib:int-sub m k) i ib) 1)
           ib
           (f2cl-lib:array-slice a-%data% single-float
                                 (1
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-sub n k) i))
                                 ((1 lda) (1 *)) a-%offset%)
```

```
      lda
      (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                            tau-%offset%)
      work iinfo)
  (declare (ignore var-0 var-1 var-2 var-4 var-5))
  (setf lda var-3)
  (setf iinfo var-6))
(cond
 ((> (f2cl-lib:int-add n (f2cl-lib:int-sub k) i) 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (slarft "Backward" "Columnwise"
       (f2cl-lib:int-sub
        (f2cl-lib:int-add (f2cl-lib:int-sub m k) i ib) 1)
       ib
       (f2cl-lib:array-slice a-%data% single-float
                             (1
                              (f2cl-lib:int-add
                               (f2cl-lib:int-sub n k) i))
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                             tau-%offset%)
       work ldwork)
    (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
    (when var-3 (setf ib var-3))
    (when var-5 (setf lda var-5))
    (when var-8 (setf ldwork var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (slarfb "Left" "Transpose" "Backward" "Columnwise"
       (f2cl-lib:int-sub
        (f2cl-lib:int-add (f2cl-lib:int-sub m k) i ib) 1)
       (f2cl-lib:int-sub
        (f2cl-lib:int-add (f2cl-lib:int-sub n k) i) 1)
       ib
       (f2cl-lib:array-slice a-%data% single-float
                             (1
                              (f2cl-lib:int-add
                               (f2cl-lib:int-sub n k) i))
                             ((1 lda) (1 *)) a-%offset%)
       lda work ldwork a lda
       (f2cl-lib:array-slice work-%data% single-float ((+ ib 1))
                             ((1 *)) work-%offset%)
       ldwork)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
      var-13))
    (when var-6 (setf ib var-6))
```

```
                (when var-8 (setf lda var-8))
                (when var-10 (setf ldwork var-10))
                (when var-12 (setf lda var-12))
                (when var-14 (setf ldwork var-14)))))
           label10))
        (setf mu
                (f2cl-lib:int-sub
                 (f2cl-lib:int-add (f2cl-lib:int-sub m k) i nb) 1))
        (setf nu
                (f2cl-lib:int-sub
                 (f2cl-lib:int-add (f2cl-lib:int-sub n k) i nb) 1)))
       (t (setf mu m) (setf nu n)))
      (if (and (> mu 0) (> nu 0))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (sgeql2 mu nu a lda tau work iinfo)
            (declare (ignore var-0 var-1 var-2 var-4 var-5))
            (setf lda var-3)
            (setf iinfo var-6)))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 iws) 'single-float))
      (go end_label)
     end_label
      (return (values m n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgeqlf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::sgeql2
                                              fortran-to-lisp::ilaenv))))
```

33.15 sgeqp3

33.15.1 Axiom unit tests

— sgeqp3.input —

```
)set break resume
)sys rm -f sgeqp3.output
)spool sgeqp3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.15.2 Axiom help page

— sgeqp3.help —

```
======================================================================
sgeqp3 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgeqp3 (M, N, A, LDA, JPVT, TAU, WORK, LWORK, INFO)
           SGEQP3

Function/Subroutine Documentation
   subroutine sgeqp3 (integer M, integer N, real, dimension( lda, * ) A,
       integer LDA, integer, dimension( * ) JPVT, real, dimension( * ) TAU,
       real, dimension( * ) WORK, integer LWORK, integer INFO)
       SGEQP3

Purpose:
   SGEQP3 computes a QR factorization with column pivoting of a
   matrix A:  A*P = Q*R  using Level 3 BLAS.

Parameters:
```

```
M
  M is INTEGER
  The number of rows of the matrix A. M >= 0.

N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

A
  A is REAL array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit, the upper triangle of the array contains the
  min(M,N)-by-N upper trapezoidal matrix R; the elements below
  the diagonal, together with the array TAU, represent the
  orthogonal matrix Q as a product of min(M,N) elementary reflectors.

LDA
  LDA is INTEGER
  The leading dimension of the array A. LDA >= max(1,M).

JPVT
  JPVT is INTEGER array, dimension (N)
  On entry, if JPVT(J).ne.0, the J-th column of A is permuted
  to the front of A*P (a leading column); if JPVT(J)=0,
  the J-th column of A is a free column.
  On exit, if JPVT(J)=K, then the J-th column of A*P was the
  the K-th column of A.

TAU
  TAU is REAL array, dimension (min(M,N))
  The scalar factors of the elementary reflectors.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO=0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK >= 3*N+1.
  For optimal performance LWORK >= 2*N+( N+1 )*NB, where NB
  is the optimal blocksize.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
```

```
   = 0: successful exit.
   < 0: if INFO = -i, the i-th argument had an illegal value.

Further Details:

   The matrix Q is represented as a product of elementary reflectors

      Q = H(1) H(2) . . . H(k), where k = min(m,n).

   Each H(i) has the form

      H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real/complex vector
   with v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in
   A(i+1:m,i), and tau in TAU(i).
```

33.15.3 fortran code

— sgeqp3.f —

```
*  =====================================================================
      SUBROUTINE SGEQP3( M, N, A, LDA, JPVT, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            JPVT( * )
      REAL               A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            INB, INBMIN, IXOVER
      PARAMETER          ( INB = 1, INBMIN = 2, IXOVER = 3 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
```

```
      INTEGER            FJB, IWS, J, JB, LWKOPT, MINMN, MINWS, NA, NB,
     $                   NBMIN, NFXD, NX, SM, SMINMN, SN, TOPBMN
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEQRF, SLAQP2, SLAQPS, SORMQR, SSWAP, XERBLA
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      REAL               SNRM2
      EXTERNAL           ILAENV, SNRM2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          INT, MAX, MIN
*     Test input arguments
*  ====================
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
*
      IF( INFO.EQ.0 ) THEN
         MINMN = MIN( M, N )
         IF( MINMN.EQ.0 ) THEN
            IWS = 1
            LWKOPT = 1
         ELSE
            IWS = 3*N + 1
            NB = ILAENV( INB, 'SGEQRF', ' ', M, N, -1, -1 )
            LWKOPT = 2*N + ( N + 1 )*NB
         END IF
         WORK( 1 ) = LWKOPT
*
         IF( ( LWORK.LT.IWS ) .AND. .NOT.LQUERY ) THEN
            INFO = -8
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEQP3', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
```

```
*     Move initial columns up front.
*
      NFXD = 1
      DO 10 J = 1, N
         IF( JPVT( J ).NE.0 ) THEN
            IF( J.NE.NFXD ) THEN
               CALL SSWAP( M, A( 1, J ), 1, A( 1, NFXD ), 1 )
               JPVT( J ) = JPVT( NFXD )
               JPVT( NFXD ) = J
            ELSE
               JPVT( J ) = J
            END IF
            NFXD = NFXD + 1
         ELSE
            JPVT( J ) = J
         END IF
   10 CONTINUE
      NFXD = NFXD - 1
*
*     Factorize fixed columns
*  =======================
*
*     Compute the QR factorization of fixed columns and update
*     remaining columns.
*
      IF( NFXD.GT.0 ) THEN
         NA = MIN( M, NFXD )
*CC      CALL SGEQR2( M, NA, A, LDA, TAU, WORK, INFO )
         CALL SGEQRF( M, NA, A, LDA, TAU, WORK, LWORK, INFO )
         IWS = MAX( IWS, INT( WORK( 1 ) ) )
         IF( NA.LT.N ) THEN
*CC         CALL SORM2R( 'Left', 'Transpose', M, N-NA, NA, A, LDA,
*CC  $                   TAU, A( 1, NA+1 ), LDA, WORK, INFO )
            CALL SORMQR( 'Left', 'Transpose', M, N-NA, NA, A, LDA, TAU,
     $                   A( 1, NA+1 ), LDA, WORK, LWORK, INFO )
            IWS = MAX( IWS, INT( WORK( 1 ) ) )
         END IF
      END IF
*
*     Factorize free columns
*  ======================
*
      IF( NFXD.LT.MINMN ) THEN
*
         SM = M - NFXD
         SN = N - NFXD
         SMINMN = MINMN - NFXD
*
*        Determine the block size.
*
```

```
         NB = ILAENV( INB, 'SGEQRF', ' ', SM, SN, -1, -1 )
         NBMIN = 2
         NX = 0
*
         IF( ( NB.GT.1 ) .AND. ( NB.LT.SMINMN ) ) THEN
*
*           Determine when to cross over from blocked to unblocked code.
*
            NX = MAX( 0, ILAENV( IXOVER, 'SGEQRF', ' ', SM, SN, -1,
     $           -1 ) )
*
*
            IF( NX.LT.SMINMN ) THEN
*
*              Determine if workspace is large enough for blocked code.
*
               MINWS = 2*SN + ( SN+1 )*NB
               IWS = MAX( IWS, MINWS )
               IF( LWORK.LT.MINWS ) THEN
*
*                Not enough workspace to use optimal NB: Reduce NB and
*                determine the minimum value of NB.
*
                 NB = ( LWORK-2*SN ) / ( SN+1 )
                 NBMIN = MAX( 2, ILAENV( INBMIN, 'SGEQRF', ' ', SM, SN,
     $                   -1, -1 ) )
*
*
               END IF
            END IF
         END IF
*
*        Initialize partial column norms. The first N elements of work
*        store the exact column norms.
*
         DO 20 J = NFXD + 1, N
            WORK( J ) = SNRM2( SM, A( NFXD+1, J ), 1 )
            WORK( N+J ) = WORK( J )
   20    CONTINUE
*
         IF( ( NB.GE.NBMIN ) .AND. ( NB.LT.SMINMN ) .AND.
     $       ( NX.LT.SMINMN ) ) THEN
*
*           Use blocked code initially.
*
            J = NFXD + 1
*
*           Compute factorization: while loop.
*
*
```

```
            TOPBMN = MINMN - NX
   30       CONTINUE
            IF( J.LE.TOPBMN ) THEN
               JB = MIN( NB, TOPBMN-J+1 )
*
*              Factorize JB columns among columns J:N.
*
               CALL SLAQPS( M, N-J+1, J-1, JB, FJB, A( 1, J ), LDA,
     $                      JPVT( J ), TAU( J ), WORK( J ), WORK( N+J ),
     $                      WORK( 2*N+1 ), WORK( 2*N+JB+1 ), N-J+1 )
*
               J = J + FJB
               GO TO 30
            END IF
         ELSE
            J = NFXD + 1
         END IF
*
*        Use unblocked code to factor the last or only block.
*
*
         IF( J.LE.MINMN )
     $      CALL SLAQP2( M, N-J+1, J-1, A( 1, J ), LDA, JPVT( J ),
     $                   TAU( J ), WORK( J ), WORK( N+J ),
     $                   WORK( 2*N+1 ) )
*
      END IF
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of SGEQP3
*
      END
```

———-

33.15.4 lisp code

— **sgeqp3.lisp** —

```
(let* ((inb 1) (inbmin 2) (ixover 3))
  (declare (type (f2cl-lib:integer4 1 1) inb)
           (type (f2cl-lib:integer4 2 2) inbmin)
           (type (f2cl-lib:integer4 3 3) ixover)
           (ignorable inb inbmin ixover))
  (defun sgeqp3 (m n a lda jpvt tau work lwork info)
    (declare (type (array f2cl-lib:integer4 (*)) jpvt)
```

```
          (type (array single-float (*)) work tau a)
          (type (f2cl-lib:integer4) info lwork lda n m))
(f2cl-lib:with-multi-array-data
    ((a single-float a-%data% a-%offset%)
     (tau single-float tau-%data% tau-%offset%)
     (work single-float work-%data% work-%offset%)
     (jpvt f2cl-lib:integer4 jpvt-%data% jpvt-%offset%))
  (prog ((fjb 0) (iws 0) (j 0) (jb 0) (lwkopt 0) (minmn 0) (minws 0) (na 0)
         (nb 0) (nbmin 0) (nfxd 0) (nx 0) (sm 0) (sminmn 0) (sn 0)
         (topbmn 0) (lquery nil))
    (declare
     (type (f2cl-lib:integer4) fjb iws j jb lwkopt minmn minws na nb nbmin
      nfxd nx sm sminmn sn topbmn)
     (type f2cl-lib:logical lquery))
    (setf info 0)
    (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
    (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
          ((< lda
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
           (setf info -4)))
    (cond
     ((= info 0)
      (setf minmn
              (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
      (cond ((= minmn 0) (setf iws 1) (setf lwkopt 1))
            (t (setf iws (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) 1))
             (setf nb
                     (multiple-value-bind
                         (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                          var-6)
                         (ilaenv inb "SGEQRF" " " m n -1 -1)
                       (declare (ignore var-1 var-2 var-5 var-6))
                       (setf inb var-0)
                       (setf m var-3)
                       (setf n var-4)
                       ret-val))
             (setf lwkopt
                     (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                       (f2cl-lib:int-mul
                                        (f2cl-lib:int-add n 1) nb)))))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
      (cond ((and (< lwork iws) (not lquery)) (setf info -8)))))
    (cond
     ((/= info 0) (xerbla "SGEQP3" (f2cl-lib:int-sub info)) (go end_label))
     (lquery (go end_label)))
    (setf nfxd 1)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
```

```
    (cond
     ((/= (f2cl-lib:fref jpvt (j) ((1 *))) 0)
      (cond
       ((/= j nfxd)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (sswap m
             (f2cl-lib:array-slice a-%data% single-float (1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice a-%data% single-float (1 nfxd)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf m var-0)))
        (setf (f2cl-lib:fref jpvt-%data% (j) ((1 *)) jpvt-%offset%)
                (f2cl-lib:fref jpvt-%data% (nfxd) ((1 *))
                               jpvt-%offset%))
        (setf (f2cl-lib:fref jpvt-%data% (nfxd) ((1 *)) jpvt-%offset%)
                j))
       (t
        (setf (f2cl-lib:fref jpvt-%data% (j) ((1 *)) jpvt-%offset%)
                j)))
      (setf nfxd (f2cl-lib:int-add nfxd 1)))
     (t
      (setf (f2cl-lib:fref jpvt-%data% (j) ((1 *)) jpvt-%offset%) j)))
   label10))
(setf nfxd (f2cl-lib:int-sub nfxd 1))
(cond
 ((> nfxd 0)
  (setf na
          (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 nfxd)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (sgeqrf m na a lda tau work lwork info)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf na var-1)
    (setf lda var-3)
    (setf info var-7))
  (setf iws
          (max (the f2cl-lib:integer4 iws)
               (the f2cl-lib:integer4
                    (f2cl-lib:int
                     (f2cl-lib:fref work-%data% (1) ((1 *))
                                    work-%offset%)))))
  (cond
   ((< na n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
```

```
          (sormqr "Left" "Transpose" m (f2cl-lib:int-sub n na) na a lda
           tau
           (f2cl-lib:array-slice a-%data% single-float
                                 (1 (f2cl-lib:int-add na 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda work lwork info)
        (declare (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10))
        (when var-2 (setf m var-2))
        (when var-4 (setf na var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf lda var-9))
        (when var-11 (setf lwork var-11))
        (when var-12 (setf info var-12)))
      (setf iws
              (max (the f2cl-lib:integer4 iws)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int
                         (f2cl-lib:fref work-%data% (1) ((1 *))
                                        work-%offset%)))))))))
(cond
 ((< nfxd minmn) (setf sm (f2cl-lib:int-sub m nfxd))
  (setf sn (f2cl-lib:int-sub n nfxd))
  (setf sminmn (f2cl-lib:int-sub minmn nfxd))
  (setf nb
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (ilaenv inb "SGEQRF" " " sm sn -1 -1)
            (declare (ignore var-1 var-2 var-5 var-6))
            (setf inb var-0)
            (setf sm var-3)
            (setf sn var-4)
            ret-val))
  (setf nbmin 2) (setf nx 0)
  (cond
   ((and (> nb 1) (< nb sminmn))
    (setf nx
            (max (the f2cl-lib:integer4 0)
                 (the f2cl-lib:integer4
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
                          (ilaenv ixover "SGEQRF" " " sm sn -1 -1)
                        (declare (ignore var-1 var-2 var-5 var-6))
                        (setf ixover var-0)
                        (setf sm var-3)
                        (setf sn var-4)
                        ret-val))))
    (cond
     ((< nx sminmn)
      (setf minws
```

```
                (f2cl-lib:int-add (f2cl-lib:int-mul 2 sn)
                                  (f2cl-lib:int-mul
                                   (f2cl-lib:int-add sn 1) nb)))
        (setf iws
                (max (the f2cl-lib:integer4 iws)
                     (the f2cl-lib:integer4 minws)))
        (cond
         ((< lwork minws)
          (setf nb
                  (the f2cl-lib:integer4
                       (truncate (- lwork (* 2 sn)) (+ sn 1))))
          (setf nbmin
                  (max (the f2cl-lib:integer4 2)
                       (the f2cl-lib:integer4
                            (multiple-value-bind
                                (ret-val var-0 var-1 var-2 var-3 var-4
                                 var-5 var-6)
                                (ilaenv inbmin "SGEQRF" " " sm sn -1 -1)
                              (declare (ignore var-1 var-2 var-5 var-6))
                              (setf inbmin var-0)
                              (setf sm var-3)
                              (setf sn var-4)
                              ret-val)))))))))
(f2cl-lib:fdo (j (f2cl-lib:int-add nfxd 1) (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (snrm2 sm
                 (f2cl-lib:array-slice a-%data% single-float
                                       ((+ nfxd 1) j) ((1 lda) (1 *))
                                       a-%offset%)
                 1)
              (declare (ignore var-1 var-2))
              (when var-0 (setf sm var-0))
              ret-val))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j)) ((1 *))
                         work-%offset%)
            (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%))
   label20))
(cond
 ((and (>= nb nbmin) (< nb sminmn) (< nx sminmn))
  (tagbody
    (setf j (f2cl-lib:int-add nfxd 1))
    (setf topbmn (f2cl-lib:int-sub minmn nx))
   label30
    (cond
     ((<= j topbmn)
      (setf jb
              (min (the f2cl-lib:integer4 nb)
```

```
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add (f2cl-lib:int-sub topbmn j)
                                           1))))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12 var-13)
            (slaqps m (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1)
             (f2cl-lib:int-sub j 1) jb fjb
             (f2cl-lib:array-slice a-%data% single-float (1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice jpvt-%data% f2cl-lib:integer4 (j)
                                   ((1 *)) jpvt-%offset%)
             (f2cl-lib:array-slice tau-%data% single-float (j) ((1 *))
                                   tau-%offset%)
             (f2cl-lib:array-slice work-%data% single-float (j) ((1 *))
                                   work-%offset%)
             (f2cl-lib:array-slice work-%data% single-float ((+ n j))
                                   ((1 *)) work-%offset%)
             (f2cl-lib:array-slice work-%data% single-float
                                   ((+ (f2cl-lib:int-mul 2 n) 1))
                                   ((1 *)) work-%offset%)
             (f2cl-lib:array-slice work-%data% single-float
                                   ((+ (f2cl-lib:int-mul 2 n) jb 1))
                                   ((1 *)) work-%offset%)
             (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1))
          (declare
           (ignore var-1 var-2 var-5 var-7 var-8 var-9 var-10 var-11
            var-12 var-13))
          (when var-0 (setf m var-0))
          (when var-3 (setf jb var-3))
          (when var-4 (setf fjb var-4))
          (when var-6 (setf lda var-6)))
        (setf j (f2cl-lib:int-add j fjb)) (go label30)))))
  (t (setf j (f2cl-lib:int-add nfxd 1))))
 (if (<= j minmn)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (slaqp2 m (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1)
          (f2cl-lib:int-sub j 1)
          (f2cl-lib:array-slice a-%data% single-float (1 j)
                                ((1 lda) (1 *)) a-%offset%)
          lda
          (f2cl-lib:array-slice jpvt-%data% f2cl-lib:integer4 (j)
                                ((1 *)) jpvt-%offset%)
          (f2cl-lib:array-slice tau-%data% single-float (j) ((1 *))
                                tau-%offset%)
          (f2cl-lib:array-slice work-%data% single-float (j) ((1 *))
                                work-%offset%)
          (f2cl-lib:array-slice work-%data% single-float ((+ n j))
```

```
                                        ((1 *)) work-%offset%)
                   (f2cl-lib:array-slice work-%data% single-float
                                        ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                                        work-%offset%))
                 (declare
                  (ignore var-1 var-2 var-3 var-5 var-6 var-7 var-8 var-9))
                 (when var-0 (setf m var-0))
                 (when var-4 (setf lda var-4))))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                 (coerce (the f2cl-lib:integer4 iws) 'single-float))
        (go end_label)
       end_label
        (return (values m n nil lda nil nil nil lwork info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgeqp3
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::lwork
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::sgeqrf
                                              fortran-to-lisp::ilaenv))))
```

33.16 sgeqpf - DEPRECATED use sgeqp3

33.17 sgeqr2

33.17.1 Axiom unit tests

— sgeqr2.input —

```
)set break resume
)sys rm -f sgeqr2.output
)spool sgeqr2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.17.2 Axiom help page

— sgeqr2.help —

```
======================================================================
sgeqr2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgeqr2 (M, N, A, LDA, TAU, WORK, INFO)
           SGEQR2 computes the QR factorization of a general rectangular
           matrix using an unblocked algorithm.

Function/Subroutine Documentation
   subroutine sgeqr2 (integer M, integer N, real, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) TAU, real, dimension( * ) WORK,
       integer INFO)
       SGEQR2 computes the QR factorization of a general rectangular matrix
       using an unblocked algorithm.
```

```
Purpose:
   SGEQR2 computes a QR factorization of a real m by n matrix A: A = Q * R.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the m by n matrix A.
   On exit, the elements on and above the diagonal of the array
   contain the min(m,n) by n upper trapezoidal matrix R (R is
   upper triangular if m >= n); the elements below the diagonal,
   with the array TAU, represent the orthogonal matrix Q as a
   product of elementary reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is REAL array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is REAL array, dimension (N)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

Further Details:

   The matrix Q is represented as a product of elementary reflectors

      Q = H(1) H(2) . . . H(k), where k = min(m,n).

   Each H(i) has the form

      H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in A(i+1:m,i),
```

and tau in TAU(i).

———-

33.17.3 fortran code

— **sgeqr2.f** —

```
*  ===================================================================
      SUBROUTINE SGEQR2( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      REAL               AII
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLARF, SLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
```

```
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEQR2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = 1, K
*
*        Generate elementary reflector H(i) to annihilate A(i+1:m,i)
*
         CALL SLARFG( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
     $                TAU( I ) )
         IF( I.LT.N ) THEN
*
*           Apply H(i) to A(i:m,i+1:n) from the left
*
            AII = A( I, I )
            A( I, I ) = ONE
            CALL SLARF( 'Left', M-I+1, N-I, A( I, I ), 1, TAU( I ),
     $                  A( I, I+1 ), LDA, WORK )
            A( I, I ) = AII
         END IF
   10 CONTINUE
      RETURN
*
*     End of SGEQR2
*
      END
```

———-

33.17.4 lisp code

— **sgeqr2.lisp** —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun sgeqr2 (m n a lda tau work info)
    (declare (type (array single-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
```

```
    (tau single-float tau-%data% tau-%offset%)
    (work single-float work-%data% work-%offset%))
  (prog ((aii 0.0) (i 0) (k 0))
    (declare (type (single-float) aii)
             (type (f2cl-lib:integer4) i k))
    (setf info 0)
    (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
          ((< lda
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
           (setf info -4)))
    (cond
     ((/= info 0) (xerbla "SGEQR2" (f2cl-lib:int-sub info))
      (go end_label)))
    (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i k) nil)
      (tagbody
        (slarfg (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
         (f2cl-lib:array-slice a-%data% single-float (i i) ((1 lda) (1 *))
                               a-%offset%)
         (f2cl-lib:array-slice a-%data% single-float
                               ((min (f2cl-lib:int-add i 1) m) i)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                               tau-%offset%))
        (cond
         ((< i n)
          (setf aii
                  (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                 a-%offset%))
          (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                  one)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (slarf "Left" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
               (f2cl-lib:int-sub n i)
               (f2cl-lib:array-slice a-%data% single-float (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               1
               (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                     tau-%offset%)
               (f2cl-lib:array-slice a-%data% single-float
                                     (i (f2cl-lib:int-add i 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda work)
            (declare
             (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
            (when var-7 (setf lda var-7)))
          (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
```

```
                    aii)))
          label10))
       (go end_label)
      end_label
       (return (values nil nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgeqr2
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(nil nil nil fortran-to-lisp::lda
                                             nil nil fortran-to-lisp::info)
                                           :calls 'nil)))
```

———-

33.18 sgeqr2p

33.18.1 Axiom unit tests

— **sgeqr2p.input** —

```
)set break resume
)sys rm -f sgeqr2p.output
)spool sgeqr2p.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.18.2 Axiom help page [Demm08]

— sgeqr2p.help —

```
==========================================================================
sgeqr2p examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgeqr2p (M, N, A, LDA, TAU, WORK, INFO)
           SGEQR2P computes the QR factorization of a general rectangular
           matrix with non-negative diagonal elements using an unblocked
           algorithm.

Function/Subroutine Documentation
   subroutine sgeqr2p (integer M, integer N, real, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) TAU, real, dimension( * ) WORK,
       integer INFO)
       SGEQR2P computes the QR factorization of a general rectangular matrix
       with non-negative diagonal elements using an unblocked algorithm.

Purpose:
   SGEQR2P computes a QR factorization of a real m by n matrix A:
   A = Q * R. The diagonal entries of R are nonnegative.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the m by n matrix A.
   On exit, the elements on and above the diagonal of the array
   contain the min(m,n) by n upper trapezoidal matrix R (R is
   upper triangular if m >= n). The diagonal entries of R
   are nonnegative; the elements below the diagonal,
   with the array TAU, represent the orthogonal matrix Q as a
   product of elementary reflectors (see Further Details).
```

```
 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is REAL array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is REAL array, dimension (N)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

Further Details:

   The matrix Q is represented as a product of elementary reflectors

      Q = H(1) H(2) . . . H(k), where k = min(m,n).

   Each H(i) has the form

      H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in A(i+1:m,i),
   and tau in TAU(i).
```

———-

33.18.3 fortran code

— **sgeqr2p.f** —

```
*  =====================================================================
      SUBROUTINE SGEQR2P( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
```

```
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      REAL               AII
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLARF, SLARFGP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEQR2P', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = 1, K
*
*        Generate elementary reflector H(i) to annihilate A(i+1:m,i)
*
         CALL SLARFGP( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
     $                TAU( I ) )
         IF( I.LT.N ) THEN
*
*           Apply H(i) to A(i:m,i+1:n) from the left
*
```

```
            AII = A( I, I )
            A( I, I ) = ONE
            CALL SLARF( 'Left', M-I+1, N-I, A( I, I ), 1, TAU( I ),
     $                  A( I, I+1 ), LDA, WORK )
            A( I, I ) = AII
         END IF
   10 CONTINUE
      RETURN
*
*     End of SGEQR2P
*
      END
```

———-

33.18.4 lisp code

— **sgeqr2p.lisp** —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun sgeqr2p (m n a lda tau work info)
    (declare (type (array single-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (tau single-float tau-%data% tau-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((aii 0.0) (i 0) (k 0))
        (declare (type (single-float) aii)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "SGEQR2P" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i k) nil)
          (tagbody
            (slarfgp (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:array-slice a-%data% single-float (i i) ((1 lda) (1 *))
                                   a-%offset%)
             (f2cl-lib:array-slice a-%data% single-float
```

```
                                ((min (f2cl-lib:int-add i 1) m) i)
                                ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                tau-%offset%))
            (cond
             ((< i n)
              (setf aii
                      (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                      one)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                  (slarf "Left" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
                   (f2cl-lib:int-sub n i)
                   (f2cl-lib:array-slice a-%data% single-float (i i)
                                         ((1 lda) (1 *)) a-%offset%)
                   1
                   (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                         tau-%offset%)
                   (f2cl-lib:array-slice a-%data% single-float
                                         (i (f2cl-lib:int-add i 1))
                                         ((1 lda) (1 *)) a-%offset%)
                   lda work)
                (declare
                 (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
                (when var-7 (setf lda var-7)))
              (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                      aii)))
           label10))
         (go end_label)
        end_label
         (return (values nil nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgeqr2p
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::lda
```

```
                                        nil nil fortran-to-lisp::info)
                                      :calls 'nil)))
```

———-

33.19 sgeqrf

33.19.1 Axiom unit tests

— **sgeqrf.input** —

```
)set break resume
)sys rm -f sgeqrf.output
)spool sgeqrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.19.2 Axiom help page

— **sgeqrf.help** —

```
====================================================================
sgeqrf examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgeqrf (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           SGEQRF

Function/Subroutine Documentation
   subroutine sgeqrf (integer M, integer N, real, dimension( lda, * ) A,
```

```
      integer LDA, real, dimension( * ) TAU, real, dimension( * ) WORK,
      integer LWORK, integer INFO)
      SGEQRF

Purpose:
   SGEQRF computes a QR factorization of a real M-by-N matrix A: A = Q * R.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, the elements on and above the diagonal of the array
   contain the min(M,N)-by-N upper trapezoidal matrix R (R is
   upper triangular if m >= n); the elements below the diagonal,
   with the array TAU, represent the orthogonal matrix Q as a
   product of min(m,n) elementary reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is REAL array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is REAL array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.  LWORK >= max(1,N).
   For optimum performance LWORK >= N*NB, where NB is
   the optimal blocksize.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
```

```
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details:

   The matrix Q is represented as a product of elementary reflectors

      Q = H(1) H(2) . . . H(k), where k = min(m,n).

   Each H(i) has the form

      H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in A(i+1:m,i),
   and tau in TAU(i).
```

33.19.3 fortran code

— sgeqrf.f —

```
*  =====================================================================
      SUBROUTINE SGEQRF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, LDWORK, LWKOPT, NB,
     $                   NBMIN, NX
*     ..
*     .. External Subroutines ..
```

```
      EXTERNAL           SGEQR2, SLARFB, SLARFT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NB = ILAENV( 1, 'SGEQRF', ' ', M, N, -1, -1 )
      LWKOPT = N*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEQRF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      K = MIN( M, N )
      IF( K.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      NX = 0
      IWS = N
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'SGEQRF', ' ', M, N, -1, -1 ) )
```

```
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = N
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'SGEQRF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
         Use blocked code initially
*
         DO 10 I = 1, K - NX, NB
            IB = MIN( K-I+1, NB )
*
            Compute the QR factorization of the current block
            A(i:m,i:i+ib-1)
*
            CALL SGEQR2( M-I+1, IB, A( I, I ), LDA, TAU( I ), WORK,
     $                   IINFO )
            IF( I+IB.LE.N ) THEN
*
               Form the triangular factor of the block reflector
               H = H(i) H(i+1) . . . H(i+ib-1)
*
               CALL SLARFT( 'Forward', 'Columnwise', M-I+1, IB,
     $                      A( I, I ), LDA, TAU( I ), WORK, LDWORK )
*
               Apply H**T to A(i:m,i+ib:n) from the left
*
               CALL SLARFB( 'Left', 'Transpose', 'Forward',
     $                      'Columnwise', M-I+1, N-I-IB+1, IB,
     $                      A( I, I ), LDA, WORK, LDWORK, A( I, I+IB ),
     $                      LDA, WORK( IB+1 ), LDWORK )
            END IF
   10    CONTINUE
      ELSE
         I = 1
      END IF
*
```

```
*     Use unblocked code to factor the last or only block.
*
      IF( I.LE.K )
     $   CALL SGEQR2( M-I+1, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                IINFO )
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of SGEQRF
*
      END
```

———-

33.19.4 lisp code

— **sgeqrf.lisp** —

```
(defun sgeqrf (m n a lda tau work lwork info)
  (declare (type (array single-float (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a single-float a-%data% a-%offset%)
       (tau single-float tau-%data% tau-%offset%)
       (work single-float work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ldwork 0) (lwkopt 0) (nb 0)
           (nbmin 0) (nx 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) nx nbmin nb lwkopt ldwork k iws iinfo
                ib i))
      (setf info 0)
      (setf nb
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 1 "SGEQRF" " " m n -1 -1)
                (declare (ignore var-0 var-1 var-2 var-5 var-6))
                (setf m var-3)
                (setf n var-4)
                ret-val))
      (setf lwkopt (f2cl-lib:int-mul n nb))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4))
            ((and
```

```
        (< lwork
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (not lquery))
       (setf info -7)))
(cond
 ((/= info 0) (xerbla "SGEQRF" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(cond
 ((= k 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'single-float))
  (go end_label)))
(setf nbmin 2)
(setf nx 0)
(setf iws n)
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "SGEQRF" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (setf m var-3)
                      (setf n var-4)
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork n) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
                            (ilaenv 2 "SGEQRF" " " m n -1 -1)
                          (declare
                           (ignore var-0 var-1 var-2 var-5 var-6))
                          (setf m var-3)
                          (setf n var-4)
                          ret-val)))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub nx))) nil)
```

```
(tagbody
  (setf ib
          (min
           (the f2cl-lib:integer4
                (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
           (the f2cl-lib:integer4 nb)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (sgeqr2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
       (f2cl-lib:array-slice a-%data% single-float (i i)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                             tau-%offset%)
       work iinfo)
    (declare (ignore var-0 var-1 var-2 var-4 var-5))
    (setf lda var-3)
    (setf iinfo var-6))
  (cond
   ((<= (f2cl-lib:int-add i ib) n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (slarft "Forward" "Columnwise"
         (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
         (f2cl-lib:array-slice a-%data% single-float (i i)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                               tau-%offset%)
         work ldwork)
      (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
      (when var-3 (setf ib var-3))
      (when var-5 (setf lda var-5))
      (when var-8 (setf ldwork var-8)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14)
        (slarfb "Left" "Transpose" "Forward" "Columnwise"
         (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
         (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1) ib
         (f2cl-lib:array-slice a-%data% single-float (i i)
                               ((1 lda) (1 *)) a-%offset%)
         lda work ldwork
         (f2cl-lib:array-slice a-%data% single-float
                               (i (f2cl-lib:int-add i ib))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice work-%data% single-float ((+ ib 1))
                               ((1 *)) work-%offset%)
         ldwork)
      (declare
```

```
                (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
                 var-13))
               (when var-6 (setf ib var-6))
               (when var-8 (setf lda var-8))
               (when var-10 (setf ldwork var-10))
               (when var-12 (setf lda var-12))
               (when var-14 (setf ldwork var-14)))))
           label10)))
       (t (setf i 1)))
      (if (<= i k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (sgeqr2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
               (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
               (f2cl-lib:array-slice a-%data% single-float (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                     tau-%offset%)
               work iinfo)
            (declare (ignore var-0 var-1 var-2 var-4 var-5))
            (setf lda var-3)
            (setf iinfo var-6)))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 iws) 'single-float))
      (go end_label)
     end_label
      (return (values m n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgeqrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::sgeqr2
                                              fortran-to-lisp::ilaenv))))
```

———-

33.20 sgeqrfp

33.20.1 Axiom unit tests

— **sgeqrfp.input** —

```
)set break resume
)sys rm -f sgeqrfp.output
)spool sgeqrfp.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.20.2 Axiom help page [Demm08]

— **sgeqrfp.help** —

```
======================================================================
sgeqrfp examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgeqrfp (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           SGEQRFP

Function/Subroutine Documentation
   subroutine sgeqrfp (integer M, integer N, real, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) TAU, real, dimension( * ) WORK,
       integer LWORK, integer INFO)
```

```
       SGEQRFP

Purpose:
   SGEQRFP computes a QR factorization of a real M-by-N matrix A:
   A = Q * R. The diagonal entries of R are nonnegative.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, the elements on and above the diagonal of the array
   contain the min(M,N)-by-N upper trapezoidal matrix R (R is
   upper triangular if m >= n). The diagonal entries of R
   are nonnegative; the elements below the diagonal,
   with the array TAU, represent the orthogonal matrix Q as a
   product of min(m,n) elementary reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is REAL array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is REAL array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.  LWORK >= max(1,N).
   For optimum performance LWORK >= N*NB, where NB is
   the optimal blocksize.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
```

```
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details:

   The matrix Q is represented as a product of elementary reflectors

      Q = H(1) H(2) . . . H(k), where k = min(m,n).

   Each H(i) has the form

      H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in A(i+1:m,i),
   and tau in TAU(i).
```

33.20.3 fortran code

— sgeqrfp.f —

```
*  =====================================================================
      SUBROUTINE SGEQRFP( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, LDWORK, LWKOPT, NB,
     $                   NBMIN, NX
*     ..
*     .. External Subroutines ..
```

```
      EXTERNAL           SGEQR2P, SLARFB, SLARFT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NB = ILAENV( 1, 'SGEQRF', ' ', M, N, -1, -1 )
      LWKOPT = N*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEQRFP', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      K = MIN( M, N )
      IF( K.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      NX = 0
      IWS = N
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'SGEQRF', ' ', M, N, -1, -1 ) )
```

```
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = N
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'SGEQRF', ' ', M, N, -1,
     $                  -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
         Use blocked code initially
*
         DO 10 I = 1, K - NX, NB
            IB = MIN( K-I+1, NB )
*
            Compute the QR factorization of the current block
            A(i:m,i:i+ib-1)
*
            CALL SGEQR2P( M-I+1, IB, A( I, I ), LDA, TAU( I ), WORK,
     $                    IINFO )
            IF( I+IB.LE.N ) THEN
*
               Form the triangular factor of the block reflector
               H = H(i) H(i+1) . . . H(i+ib-1)
*
               CALL SLARFT( 'Forward', 'Columnwise', M-I+1, IB,
     $                      A( I, I ), LDA, TAU( I ), WORK, LDWORK )
*
               Apply H**T to A(i:m,i+ib:n) from the left
*
               CALL SLARFB( 'Left', 'Transpose', 'Forward',
     $                      'Columnwise', M-I+1, N-I-IB+1, IB,
     $                      A( I, I ), LDA, WORK, LDWORK, A( I, I+IB ),
     $                      LDA, WORK( IB+1 ), LDWORK )
            END IF
   10    CONTINUE
      ELSE
         I = 1
      END IF
*
```

```
*     Use unblocked code to factor the last or only block.
*
      IF( I.LE.K )
     $   CALL SGEQR2P( M-I+1, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                IINFO )
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of SGEQRFP
*
      END
```

———-

33.20.4 lisp code

— **sgeqrfp.lisp** —

```
(defun sgeqrfp (m n a lda tau work lwork info)
  (declare (type (array single-float (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a single-float a-%data% a-%offset%)
       (tau single-float tau-%data% tau-%offset%)
       (work single-float work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ldwork 0) (lwkopt 0) (nb 0)
           (nbmin 0) (nx 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) nx nbmin nb lwkopt ldwork k iws iinfo
                ib i))
      (setf info 0)
      (setf nb
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 1 "SGEQRF" " " m n -1 -1)
                (declare (ignore var-0 var-1 var-2 var-5 var-6))
                (setf m var-3)
                (setf n var-4)
                ret-val))
      (setf lwkopt (f2cl-lib:int-mul n nb))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4))
            ((and
```

```
        (< lwork
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (not lquery))
       (setf info -7)))
(cond
 ((/= info 0) (xerbla "SGEQRFP" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(cond
 ((= k 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'single-float))
  (go end_label)))
(setf nbmin 2)
(setf nx 0)
(setf iws n)
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "SGEQRF" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (setf m var-3)
                      (setf n var-4)
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork n) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
                            (ilaenv 2 "SGEQRF" " " m n -1 -1)
                          (declare
                           (ignore var-0 var-1 var-2 var-5 var-6))
                          (setf m var-3)
                          (setf n var-4)
                          ret-val)))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub nx))) nil)
```

```
(tagbody
  (setf ib
          (min
           (the f2cl-lib:integer4
                (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
           (the f2cl-lib:integer4 nb)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (sgeqr2p (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
       (f2cl-lib:array-slice a-%data% single-float (i i)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                             tau-%offset%)
       work iinfo)
    (declare (ignore var-0 var-1 var-2 var-4 var-5))
    (setf lda var-3)
    (setf iinfo var-6))
  (cond
   ((<= (f2cl-lib:int-add i ib) n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (slarft "Forward" "Columnwise"
         (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
         (f2cl-lib:array-slice a-%data% single-float (i i)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                               tau-%offset%)
         work ldwork)
      (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
      (when var-3 (setf ib var-3))
      (when var-5 (setf lda var-5))
      (when var-8 (setf ldwork var-8)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14)
        (slarfb "Left" "Transpose" "Forward" "Columnwise"
         (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
         (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1) ib
         (f2cl-lib:array-slice a-%data% single-float (i i)
                               ((1 lda) (1 *)) a-%offset%)
         lda work ldwork
         (f2cl-lib:array-slice a-%data% single-float
                               (i (f2cl-lib:int-add i ib))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice work-%data% single-float ((+ ib 1))
                               ((1 *)) work-%offset%)
         ldwork)
      (declare
```

```
                 (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
                  var-13))
                (when var-6 (setf ib var-6))
                (when var-8 (setf lda var-8))
                (when var-10 (setf ldwork var-10))
                (when var-12 (setf lda var-12))
                (when var-14 (setf ldwork var-14)))))
            label10)))
       (t (setf i 1)))
      (if (<= i k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (sgeqr2p (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
               (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
               (f2cl-lib:array-slice a-%data% single-float (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                     tau-%offset%)
               work iinfo)
            (declare (ignore var-0 var-1 var-2 var-4 var-5))
            (setf lda var-3)
            (setf iinfo var-6)))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 iws) 'single-float))
      (go end_label)
     end_label
      (return (values m n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgeqrfp
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::sgeqr2p
                                              fortran-to-lisp::ilaenv))))
```

———-

33.21 sgeqrt

33.21.1 Axiom unit tests

— **sgeqrt.input** —

```
)set break resume
)sys rm -f sgeqrt.output
)spool sgeqrt.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.21.2 Axiom help page

— **sgeqrt.help** —

```
====================================================================
sgeqrt examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgeqrt (M, N, NB, A, LDA, T, LDT, WORK, INFO)
           SGEQRT

Function/Subroutine Documentation
   subroutine sgeqrt (integer M, integer N, integer NB, real, dimension( lda,
       * ) A, integer LDA, real, dimension( ldt, * ) T, integer LDT, real,
       dimension( * ) WORK, integer INFO)
```

```
       SGEQRT

Purpose:
   SGEQRT computes a blocked QR factorization of a real M-by-N matrix A
   using the compact WY representation of Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 NB
   NB is INTEGER
   The block size to be used in the blocked QR.  MIN(M,N) >= NB >= 1.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, the elements on and above the diagonal of the array
   contain the min(M,N)-by-N upper trapezoidal matrix R (R is
   upper triangular if M >= N); the elements below the diagonal
   are the columns of V.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 T
   T is REAL array, dimension (LDT,MIN(M,N))
   The upper triangular block reflectors stored in compact form
   as a sequence of upper triangular blocks.  See below
   for further details.

 LDT
   LDT is INTEGER
   The leading dimension of the array T.  LDT >= NB.

 WORK
   WORK is REAL array, dimension (NB*N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

```
Further Details:

   The matrix V stores the elementary reflectors H(i) in the i-th column
   below the diagonal. For example, if M=5 and N=3, the matrix V is

                        V = (  1       )
                            ( v1  1    )
                            ( v1 v2  1 )
                            ( v1 v2 v3 )
                            ( v1 v2 v3 )

   where the vi's represent the vectors which define H(i), which are returned
   in the matrix A.  The 1's along the diagonal of V are not stored in A.

   Let K=MIN(M,N).  The number of blocks is B = ceiling(K/NB), where each
   block is of order NB except for the last block, which is of order
   IB = K - (B-1)*NB.  For each of the B blocks, a upper triangular block
   reflector factor is computed: T1, T2, ..., TB.  The NB-by-NB (and IB-by-IB
   for the last block) T's are stored in the NB-by-N matrix T as

                        T = (T1 T2 ... TB).
```

33.21.3 fortran code

— sgeqrt.f —

```
*  =====================================================================
      SUBROUTINE SGEQRT( M, N, NB, A, LDA, T, LDT, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      INTEGER INFO, LDA, LDT, M, N, NB
*     ..
*     .. Array Arguments ..
      REAL A( LDA, * ), T( LDT, * ), WORK( * )
*     ..
*
* =====================================================================
*
*     ..
*     .. Local Scalars ..
```

```
      INTEGER    I, IB, IINFO, K
      LOGICAL    USE_RECURSIVE_QR
      PARAMETER( USE_RECURSIVE_QR=.TRUE. )
*     ..
*     .. External Subroutines ..
      EXTERNAL   SGEQRT2, SGEQRT3, SLARFB, XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NB.LT.1 .OR. ( NB.GT.MIN(M,N) .AND. MIN(M,N).GT.0 ) )THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDT.LT.NB ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEQRT', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      K = MIN( M, N )
      IF( K.EQ.0 ) RETURN
*
*     Blocked loop of length K
*
      DO I = 1, K,  NB
         IB = MIN( K-I+1, NB )
*
*     Compute the QR factorization of the current block A(I:M,I:I+IB-1)
*
         IF( USE_RECURSIVE_QR ) THEN
            CALL SGEQRT3( M-I+1, IB, A(I,I), LDA, T(1,I), LDT, IINFO )
         ELSE
            CALL SGEQRT2( M-I+1, IB, A(I,I), LDA, T(1,I), LDT, IINFO )
         END IF
         IF( I+IB.LE.N ) THEN
*
*     Update by applying H**T to A(I:M,I+IB:N) from the left
*
            CALL SLARFB( 'L', 'T', 'F', 'C', M-I+1, N-I-IB+1, IB,
```

```
     $                   A( I, I ), LDA, T( 1, I ), LDT,
     $                   A( I, I+IB ), LDA, WORK , N-I-IB+1 )
         END IF
      END DO
      RETURN
*
*     End of SGEQRT
*
      END
```

———-

33.21.4 lisp code

— **sgeqrt.lisp** —

```
(let* ((use_recursive_qr f2cl-lib:%true%))
  (declare (type f2cl-lib:logical use_recursive_qr)
           (ignorable use_recursive_qr))
  (defun sgeqrt (m n nb a lda t$ ldt work info)
    (declare (type (array single-float (*)) work t$ a)
             (type (f2cl-lib:integer4) info ldt lda nb n m))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (t$ single-float t$-%data% t$-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((i 0) (ib 0) (iinfo 0) (k 0))
        (declare (type (f2cl-lib:integer4) i ib iinfo k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((or (< nb 1)
                   (and
                    (> nb
                       (min (the f2cl-lib:integer4 m)
                            (the f2cl-lib:integer4 n)))
                    (>
                     (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n))
                     0)))
               (setf info -3))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -5))
              ((< ldt nb) (setf info -7)))
        (cond
         ((/= info 0) (xerbla "SGEQRT" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (if (= k 0)
```

```
    (go end_label))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
               ((> i k) nil)
  (tagbody
    (setf ib
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
             (the f2cl-lib:integer4 nb)))
    (cond
     (use_recursive_qr
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (sgeqrt3 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
           (f2cl-lib:array-slice a-%data% single-float (i i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice t$-%data% single-float (1 i)
                                 ((1 ldt) (1 *)) t$-%offset%)
           ldt iinfo)
        (declare (ignore var-0 var-2 var-4))
        (when var-1 (setf ib var-1))
        (when var-3 (setf lda var-3))
        (when var-5 (setf ldt var-5))
        (when var-6 (setf iinfo var-6))))
     (t
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (sgeqrt2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
           (f2cl-lib:array-slice a-%data% single-float (i i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice t$-%data% single-float (1 i)
                                 ((1 ldt) (1 *)) t$-%offset%)
           ldt iinfo)
        (declare (ignore var-0 var-1 var-2 var-4))
        (setf lda var-3)
        (setf ldt var-5)
        (setf iinfo var-6))))
    (cond
     ((<= (f2cl-lib:int-add i ib) n)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13 var-14)
          (slarfb "L" "T" "F" "C"
           (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
           (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1) ib
           (f2cl-lib:array-slice a-%data% single-float (i i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice t$-%data% single-float (1 i)
                                 ((1 ldt) (1 *)) t$-%offset%)
```

```
                  ldt
                  (f2cl-lib:array-slice a-%data% single-float
                                        (i (f2cl-lib:int-add i ib))
                                        ((1 lda) (1 *)) a-%offset%)
                  lda work (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1))
                (declare
                 (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
                  var-13 var-14))
                (when var-6 (setf ib var-6))
                (when var-8 (setf lda var-8))
                (when var-10 (setf ldt var-10))
                (when var-12 (setf lda var-12)))))
           label100000))
        (go end_label)
       end_label
        (return (values nil nil nil nil lda nil ldt nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgeqrt
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldt nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::sgeqrt2))))
```

33.22 sgeqrt2

33.22.1 Axiom unit tests

— **sgeqrt2.input** —

```
)set break resume
)sys rm -f sgeqrt2.output
)spool sgeqrt2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.22.2 Axiom help page

— **sgeqrt2.help** —

```
======================================================================
sgeqrt2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgeqrt2 (M, N, A, LDA, T, LDT, INFO)
           SGEQRT2 computes a QR factorization of a general real or complex
           matrix using the compact WY representation of Q.

Function/Subroutine Documentation
   subroutine sgeqrt2 (integer M, integer N, real, dimension( lda, * ) A,
       integer LDA, real, dimension( ldt, * ) T, integer LDT, integer INFO)
       SGEQRT2 computes a QR factorization of a general real or complex matrix
       using the compact WY representation of Q.

Purpose:
   SGEQRT2 computes a QR factorization of a real M-by-N matrix A,
   using the compact WY representation of Q.
```

```
Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= N.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the real M-by-N matrix A.  On exit, the elements on and
   above the diagonal contain the N-by-N upper triangular matrix R; the
   elements below the diagonal are the columns of V.  See below for
   further details.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 T
   T is REAL array, dimension (LDT,N)
   The N-by-N upper triangular factor of the block reflector.
   The elements on and above the diagonal contain the block
   reflector T; the elements below the diagonal are not used.
   See below for further details.

 LDT
   LDT is INTEGER
   The leading dimension of the array T.  LDT >= max(1,N).

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

Further Details:

   The matrix V stores the elementary reflectors H(i) in the i-th column
   below the diagonal. For example, if M=5 and N=3, the matrix V is

                         V = (  1       )
                             ( v1  1    )
                             ( v1 v2  1 )
                             ( v1 v2 v3 )
                             ( v1 v2 v3 )

   where the vi's represent the vectors which define H(i), which are returned
   in the matrix A.  The 1's along the diagonal of V are not stored in A.  The
```

```
   block reflector H is then given by

                        H = I - V * T * V**T

   where V**T is the transpose of V.
```

33.22.3 fortran code

— sgeqrt2.f —

```
*  =====================================================================
      SUBROUTINE SGEQRT2( M, N, A, LDA, T, LDT, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER   INFO, LDA, LDT, M, N
*     ..
*     .. Array Arguments ..
      REAL   A( LDA, * ), T( LDT, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL  ONE, ZERO
      PARAMETER( ONE = 1.0, ZERO = 0.0 )
*     ..
*     .. Local Scalars ..
      INTEGER   I, K
      REAL   AII, ALPHA
*     ..
*     .. External Subroutines ..
      EXTERNAL  SLARFG, SGEMV, SGER, STRMV, XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
```

```
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LDT.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEQRT2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO I = 1, K
*
*        Generate elem. refl. H(i) to annihilate A(i+1:m,i), tau(I) -> T(I,1)
*
         CALL SLARFG( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
     $                T( I, 1 ) )
         IF( I.LT.N ) THEN
*
*           Apply H(i) to A(I:M,I+1:N) from the left
*
            AII = A( I, I )
            A( I, I ) = ONE
*
*           W(1:N-I) := A(I:M,I+1:N)^H * A(I:M,I) [W = T(:,N)]
*
            CALL SGEMV( 'T',M-I+1, N-I, ONE, A( I, I+1 ), LDA,
     $                  A( I, I ), 1, ZERO, T( 1, N ), 1 )
*
*           A(I:M,I+1:N) = A(I:m,I+1:N) + alpha*A(I:M,I)*W(1:N-1)^H
*
            ALPHA = -(T( I, 1 ))
            CALL SGER( M-I+1, N-I, ALPHA, A( I, I ), 1,
     $           T( 1, N ), 1, A( I, I+1 ), LDA )
            A( I, I ) = AII
         END IF
      END DO
*
      DO I = 2, N
         AII = A( I, I )
         A( I, I ) = ONE
*
*        T(1:I-1,I) := alpha * A(I:M,1:I-1)**T * A(I:M,I)
*
         ALPHA = -T( I, 1 )
         CALL SGEMV( 'T', M-I+1, I-1, ALPHA, A( I, 1 ), LDA,
     $               A( I, I ), 1, ZERO, T( 1, I ), 1 )
```

```
         A( I, I ) = AII
*
*        T(1:I-1,I) := T(1:I-1,1:I-1) * T(1:I-1,I)
*
         CALL STRMV( 'U', 'N', 'N', I-1, T, LDT, T( 1, I ), 1 )
*
*           T(I,I) = tau(I)
*
            T( I, I ) = T( I, 1 )
            T( I, 1) = ZERO
      END DO

*
*     End of SGEQRT2
*
      END
```

———-

33.22.4 lisp code

— sgeqrt2.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun sgeqrt2 (m n a lda t$ ldt info)
    (declare (type (array single-float (*)) t$ a)
             (type (f2cl-lib:integer4) info ldt lda n m))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (t$ single-float t$-%data% t$-%offset%))
      (prog ((aii 0.0) (alpha 0.0) (i 0) (k 0))
        (declare (type (single-float) aii alpha)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4))
              ((< ldt
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -6)))
        (cond
         ((/= info 0) (xerbla "SGEQRT2" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
```

```
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i k) nil)
  (tagbody
    (slarfg (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
     (f2cl-lib:array-slice a-%data% single-float (i i) ((1 lda) (1 *))
                           a-%offset%)
     (f2cl-lib:array-slice a-%data% single-float
                           ((min (f2cl-lib:int-add i 1) m) i)
                           ((1 lda) (1 *)) a-%offset%)
     1
     (f2cl-lib:array-slice t$-%data% single-float (i 1) ((1 ldt) (1 *))
                           t$-%offset%))
    (cond
     ((< i n)
      (setf aii
              (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                             a-%offset%))
      (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
              one)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (sgemv "T" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
           (f2cl-lib:int-sub n i) one
           (f2cl-lib:array-slice a-%data% single-float
                                 (i (f2cl-lib:int-add i 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice a-%data% single-float (i i)
                                 ((1 lda) (1 *)) a-%offset%)
           1 zero
           (f2cl-lib:array-slice t$-%data% single-float (1 n)
                                 ((1 ldt) (1 *)) t$-%offset%)
           1)
        (declare
         (ignore var-0 var-1 var-2 var-4 var-6 var-7 var-9 var-10))
        (when var-3 (setf one var-3))
        (when var-5 (setf lda var-5))
        (when var-8 (setf zero var-8)))
      (setf alpha
              (-
               (f2cl-lib:fref t$-%data% (i 1) ((1 ldt) (1 *))
                              t$-%offset%)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (sger (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
           (f2cl-lib:int-sub n i) alpha
           (f2cl-lib:array-slice a-%data% single-float (i i)
                                 ((1 lda) (1 *)) a-%offset%)
           1
```

```
            (f2cl-lib:array-slice t$-%data% single-float (1 n)
                                  ((1 ldt) (1 *)) t$-%offset%)
            1
            (f2cl-lib:array-slice a-%data% single-float
                                  (i (f2cl-lib:int-add i 1))
                                  ((1 lda) (1 *)) a-%offset%)
            lda)
          (declare (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7))
          (when var-2 (setf alpha var-2))
          (when var-8 (setf lda var-8)))
        (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
              aii)))
    label100000))
  (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf aii
              (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%))
      (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
              one)
      (setf alpha
              (-
               (f2cl-lib:fref t$-%data% (i 1) ((1 ldt) (1 *))
                              t$-%offset%)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (sgemv "T" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
           (f2cl-lib:int-sub i 1) alpha
           (f2cl-lib:array-slice a-%data% single-float (i 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice a-%data% single-float (i i)
                                 ((1 lda) (1 *)) a-%offset%)
           1 zero
           (f2cl-lib:array-slice t$-%data% single-float (1 i)
                                 ((1 ldt) (1 *)) t$-%offset%)
           1)
        (declare
         (ignore var-0 var-1 var-2 var-4 var-6 var-7 var-9 var-10))
        (when var-3 (setf alpha var-3))
        (when var-5 (setf lda var-5))
        (when var-8 (setf zero var-8)))
      (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
              aii)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (strmv "U" "N" "N" (f2cl-lib:int-sub i 1) t$ ldt
           (f2cl-lib:array-slice t$-%data% single-float (1 i)
                                 ((1 ldt) (1 *)) t$-%offset%)
```

```
                1)
              (declare (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7))
              (when var-5 (setf ldt var-5)))
            (setf (f2cl-lib:fref t$-%data% (i i) ((1 ldt) (1 *)) t$-%offset%)
                    (f2cl-lib:fref t$-%data% (i 1) ((1 ldt) (1 *))
                                   t$-%offset%))
            (setf (f2cl-lib:fref t$-%data% (i 1) ((1 ldt) (1 *)) t$-%offset%)
                    zero)
           label100001))
       end_label
        (return (values nil nil nil lda nil ldt info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgeqrt2
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(nil nil nil fortran-to-lisp::lda
                                             nil fortran-to-lisp::ldt
                                             fortran-to-lisp::info)
                                           :calls 'nil)))
```

———-

33.23 sgeqrt3

33.23.1 Axiom unit tests

— **sgeqrt3.input** —

```
)set break resume
)sys rm -f sgeqrt3.output
)spool sgeqrt3.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

33.23.2 Axiom help page [Elmr00]

— **sgeqrt3.help** —

```
==========================================================================
sgeqrt3 examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       recursive subroutine sgeqrt3 (M, N, A, LDA, T, LDT, INFO)
           SGEQRT3 recursively computes a QR factorization of a general real
           or complex matrix using the compact WY representation of Q.

Function/Subroutine Documentation
   recursive subroutine sgeqrt3 (integer M, integer N, real, dimension( lda, *
       ) A, integer LDA, real, dimension( ldt, * ) T, integer LDT, integer
       INFO)
       SGEQRT3 recursively computes a QR factorization of a general real or
       complex matrix using the compact WY representation of Q.

Purpose:
   SGEQRT3 recursively computes a QR factorization of a real M-by-N
   matrix A, using the compact WY representation of Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= N.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the real M-by-N matrix A.  On exit, the elements on and
```

```
  above the diagonal contain the N-by-N upper triangular matrix R; the
  elements below the diagonal are the columns of V.  See below for
  further details.

 LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

 T
  T is REAL array, dimension (LDT,N)
  The N-by-N upper triangular factor of the block reflector.
  The elements on and above the diagonal contain the block
  reflector T; the elements below the diagonal are not used.
  See below for further details.

 LDT
  LDT is INTEGER
  The leading dimension of the array T.  LDT >= max(1,N).

 INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value

Further Details:

  The matrix V stores the elementary reflectors H(i) in the i-th column
  below the diagonal. For example, if M=5 and N=3, the matrix V is

                        V = (  1       )
                            ( v1  1    )
                            ( v1 v2  1 )
                            ( v1 v2 v3 )
                            ( v1 v2 v3 )

  where the vi's represent the vectors which define H(i), which are returned
  in the matrix A.  The 1's along the diagonal of V are not stored in A.  The
  block reflector H is then given by

                        H = I - V * T * V**T

  where V**T is the transpose of V.
```

———-

33.23.3 fortran code

— **sgeqrt3.f** —

```
*  =====================================================================
      RECURSIVE SUBROUTINE SGEQRT3( M, N, A, LDA, T, LDT, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER   INFO, LDA, M, N, LDT
*     ..
*     .. Array Arguments ..
      REAL   A( LDA, * ), T( LDT, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL   ONE
      PARAMETER ( ONE = 1.0 )
*     ..
*     .. Local Scalars ..
      INTEGER   I, I1, J, J1, N1, N2, IINFO
*     ..
*     .. External Subroutines ..
      EXTERNAL  SLARFG, STRMM, SGEMM, XERBLA
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      IF( N .LT. 0 ) THEN
         INFO = -2
      ELSE IF( M .LT. N ) THEN
         INFO = -1
      ELSE IF( LDA .LT. MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LDT .LT. MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGEQRT3', -INFO )
         RETURN
      END IF
*
      IF( N.EQ.1 ) THEN
*
*        Compute Householder transform when N=1
*
```

```
         CALL SLARFG( M, A, A( MIN( 2, M ), 1 ), 1, T )
*
      ELSE
*
*        Otherwise, split A into blocks...
*
         N1 = N/2
         N2 = N-N1
         J1 = MIN( N1+1, N )
         I1 = MIN( N+1, M )
*
*        Compute A(1:M,1:N1) <- (Y1,R1,T1), where Q1 = I - Y1 T1 Y1^H
*
         CALL SGEQRT3( M, N1, A, LDA, T, LDT, IINFO )
*
*        Compute A(1:M,J1:N) = Q1^H A(1:M,J1:N) [workspace: T(1:N1,J1:N)]
*
         DO J=1,N2
            DO I=1,N1
               T( I, J+N1 ) = A( I, J+N1 )
            END DO
         END DO
         CALL STRMM( 'L', 'L', 'T', 'U', N1, N2, ONE,
     &               A, LDA, T( 1, J1 ), LDT )
*
         CALL SGEMM( 'T', 'N', N1, N2, M-N1, ONE, A( J1, 1 ), LDA,
     &               A( J1, J1 ), LDA, ONE, T( 1, J1 ), LDT)
*
         CALL STRMM( 'L', 'U', 'T', 'N', N1, N2, ONE,
     &               T, LDT, T( 1, J1 ), LDT )
*
         CALL SGEMM( 'N', 'N', M-N1, N2, N1, -ONE, A( J1, 1 ), LDA,
     &               T( 1, J1 ), LDT, ONE, A( J1, J1 ), LDA )
*
         CALL STRMM( 'L', 'L', 'N', 'U', N1, N2, ONE,
     &               A, LDA, T( 1, J1 ), LDT )
*
         DO J=1,N2
            DO I=1,N1
               A( I, J+N1 ) = A( I, J+N1 ) - T( I, J+N1 )
            END DO
         END DO
*
*        Compute A(J1:M,J1:N) <- (Y2,R2,T2) where Q2 = I - Y2 T2 Y2^H
*
         CALL SGEQRT3( M-N1, N2, A( J1, J1 ), LDA,
     &                T( J1, J1 ), LDT, IINFO )
*
*        Compute T3 = T(1:N1,J1:N) = -T1 Y1^H Y2 T2
*
```

```
         DO I=1,N1
            DO J=1,N2
               T( I, J+N1 ) = (A( J+N1, I ))
            END DO
         END DO
*
         CALL STRMM( 'R', 'L', 'N', 'U', N1, N2, ONE,
     &               A( J1, J1 ), LDA, T( 1, J1 ), LDT )
*
         CALL SGEMM( 'T', 'N', N1, N2, M-N, ONE, A( I1, 1 ), LDA,
     &               A( I1, J1 ), LDA, ONE, T( 1, J1 ), LDT )
*
         CALL STRMM( 'L', 'U', 'N', 'N', N1, N2, -ONE, T, LDT,
     &               T( 1, J1 ), LDT )
*
         CALL STRMM( 'R', 'U', 'N', 'N', N1, N2, ONE,
     &               T( J1, J1 ), LDT, T( 1, J1 ), LDT )
*
*        Y = (Y1,Y2); R = [ R1  A(1:N1,J1:N) ];  T = [T1 T3]
*                         [  0        R2     ]       [ 0 T2]
*
      END IF
*
      RETURN
*
*     End of SGEQRT3
*
      END
```

———-

33.23.4 lisp code

— **sgeqrt3.lisp** —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun *main* ()
    (prog ((i 0) (i1 0) (j 0) (j1 0) (n1 0) (n2 0) (iinfo 0) (m 0) (info 0)
           (ldt 0)
           (t$
            (make-array (the fixnum (reduce #'* (list ldt *))) :element-type
                        'single-float))
           (lda 0)
           (a
            (make-array (the fixnum (reduce #'* (list lda *))) :element-type
                        'single-float))
```

```
     (n 0) (subroutine 0.0))
(declare (type (single-float) subroutine)
         (type (array single-float (*)) a t$)
         (type (f2cl-lib:integer4) n lda ldt info i i1 j j1 n1 n2 iinfo
          m))
'"****NOT TRANSLATED: (RECURSIVE SUBROUTINE SGEQRT3 (M |,| N |,| A |,| LDA |,| T |,| LDT |
(setf info 0)
(cond ((< n 0) (setf info -2)) ((< m n) (setf info -1))
      ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -4))
      ((< ldt (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -6)))
(cond
 ((/= info 0) (xerbla "SGEQRT3" (f2cl-lib:int-sub info)) (go end_label)))
(cond
 ((= n 1)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slarfg m a
       (f2cl-lib:array-slice a single-float ((min 2 m) 1)
                             ((1 lda) (1 *)))
       1 t$)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf m var-0))))
 (t (setf n1 (the f2cl-lib:integer4 (truncate n 2)))
  (setf n2 (f2cl-lib:int-sub n n1))
  (setf j1
          (min (the f2cl-lib:integer4 (f2cl-lib:int-add n1 1))
               (the f2cl-lib:integer4 n)))
  (setf i1
          (min (the f2cl-lib:integer4 (f2cl-lib:int-add n 1))
               (the f2cl-lib:integer4 m)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (sgeqrt3 m n1 a lda t$ ldt iinfo)
    (declare (ignore var-2 var-4))
    (when var-0 (setf m var-0))
    (when var-1 (setf n1 var-1))
    (when var-3 (setf lda var-3))
    (when var-5 (setf ldt var-5))
    (when var-6 (setf iinfo var-6)))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n2) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n1) nil)
        (tagbody
          (setf (f2cl-lib:fref t$ (i (f2cl-lib:int-add j n1))
                                  ((1 ldt) (1 *)))
                  (f2cl-lib:fref a (i (f2cl-lib:int-add j n1))
                                    ((1 lda) (1 *))))
         label100001))
```

```
   label100000))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (strmm "L" "L" "T" "U" n1 n2 one a lda
     (f2cl-lib:array-slice t$ single-float (1 j1) ((1 ldt) (1 *))) ldt)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf n2 var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf ldt var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (sgemm "T" "N" n1 n2 (f2cl-lib:int-sub m n1) one
     (f2cl-lib:array-slice a single-float (j1 1) ((1 lda) (1 *))) lda
     (f2cl-lib:array-slice a single-float (j1 j1) ((1 lda) (1 *))) lda
     one (f2cl-lib:array-slice t$ single-float (1 j1) ((1 ldt) (1 *)))
     ldt)
  (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
  (when var-2 (setf n1 var-2))
  (when var-3 (setf n2 var-3))
  (when var-5 (setf one var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf ldt var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (strmm "L" "U" "T" "N" n1 n2 one t$ ldt
     (f2cl-lib:array-slice t$ single-float (1 j1) ((1 ldt) (1 *))) ldt)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf n2 var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldt var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (sgemm "N" "N" (f2cl-lib:int-sub m n1) n2 n1 (- one)
     (f2cl-lib:array-slice a single-float (j1 1) ((1 lda) (1 *))) lda
     (f2cl-lib:array-slice t$ single-float (1 j1) ((1 ldt) (1 *))) ldt
     one (f2cl-lib:array-slice a single-float (j1 j1) ((1 lda) (1 *)))
     lda)
  (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
  (when var-3 (setf n2 var-3))
  (when var-4 (setf n1 var-4))
```

```
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldt var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf lda var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (strmm "L" "L" "N" "U" n1 n2 one a lda
     (f2cl-lib:array-slice t$ single-float (1 j1) ((1 ldt) (1 *))) ldt)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf n2 var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf ldt var-10)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n2) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n1) nil)
      (tagbody
        (setf (f2cl-lib:fref a (i (f2cl-lib:int-add j n1))
                              ((1 lda) (1 *)))
                (-
                 (f2cl-lib:fref a (i (f2cl-lib:int-add j n1))
                                  ((1 lda) (1 *)))
                 (f2cl-lib:fref t$ (i (f2cl-lib:int-add j n1))
                                  ((1 ldt) (1 *)))))
       label100003))
   label100002))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (sgeqrt3 (f2cl-lib:int-sub m n1) n2
     (f2cl-lib:array-slice a single-float (j1 j1) ((1 lda) (1 *))) lda
     (f2cl-lib:array-slice t$ single-float (j1 j1) ((1 ldt) (1 *))) ldt
     iinfo)
  (declare (ignore var-0 var-2 var-4))
  (when var-1 (setf n2 var-1))
  (when var-3 (setf lda var-3))
  (when var-5 (setf ldt var-5))
  (when var-6 (setf iinfo var-6)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n1) nil)
  (tagbody
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n2) nil)
      (tagbody
        (setf (f2cl-lib:fref t$ (i (f2cl-lib:int-add j n1))
                              ((1 ldt) (1 *)))
                (f2cl-lib:fref a ((f2cl-lib:int-add j n1) i)
                                ((1 lda) (1 *))))
```

```
        label100005))
    label100004))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (strmm "R" "L" "N" "U" n1 n2 one
     (f2cl-lib:array-slice a single-float (j1 j1) ((1 lda) (1 *))) lda
     (f2cl-lib:array-slice t$ single-float (1 j1) ((1 ldt) (1 *))) ldt)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf n2 var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf ldt var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (sgemm "T" "N" n1 n2 (f2cl-lib:int-sub m n) one
     (f2cl-lib:array-slice a single-float (i1 1) ((1 lda) (1 *))) lda
     (f2cl-lib:array-slice a single-float (i1 j1) ((1 lda) (1 *))) lda
     one (f2cl-lib:array-slice t$ single-float (1 j1) ((1 ldt) (1 *)))
     ldt)
  (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
  (when var-2 (setf n1 var-2))
  (when var-3 (setf n2 var-3))
  (when var-5 (setf one var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf ldt var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (strmm "L" "U" "N" "N" n1 n2 (- one) t$ ldt
     (f2cl-lib:array-slice t$ single-float (1 j1) ((1 ldt) (1 *))) ldt)
  (declare (ignore var-0 var-1 var-2 var-3 var-6 var-7 var-9))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf n2 var-5))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldt var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (strmm "R" "U" "N" "N" n1 n2 one
     (f2cl-lib:array-slice t$ single-float (j1 j1) ((1 ldt) (1 *))) ldt
     (f2cl-lib:array-slice t$ single-float (1 j1) ((1 ldt) (1 *))) ldt)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf n2 var-5))
  (when var-6 (setf one var-6))
```

```
          (when var-8 (setf ldt var-8))
          (when var-10 (setf ldt var-10)))))
      (go end_label)
     end_label
      (return nil))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::*main*
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types 'nil :return-values 'nil
                                            :calls 'nil)))
```

———-

33.24 sgerfs

33.24.1 Axiom unit tests

— **sgerfs.input** —

```
)set break resume
)sys rm -f sgerfs.output
)spool sgerfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.24.2 Axiom help page

— **sgerfs.help** —

```
======================================================================
sgerfs examples
======================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgerfs (TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB, X,
           LDX, FERR, BERR, WORK, IWORK, INFO)
           SGERFS

Function/Subroutine Documentation
   subroutine sgerfs (character TRANS, integer N, integer NRHS, real,
       dimension( lda, * ) A, integer LDA, real, dimension( ldaf, * ) AF,
       integer LDAF, integer, dimension( * ) IPIV, real, dimension( ldb, * )
       B, integer LDB, real, dimension( ldx, * ) X, integer LDX, real,
       dimension( * ) FERR, real, dimension( * ) BERR, real, dimension( * )
       WORK, integer, dimension( * ) IWORK, integer INFO)
       SGERFS

Purpose:
   SGERFS improves the computed solution to a system of linear
   equations and provides error bounds and backward error estimates for
   the solution.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose = Transpose)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 A
   A is REAL array, dimension (LDA,N)
   The original N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).
```

```
AF
  AF is REAL array, dimension (LDAF,N)
  The factors L and U from the factorization A = P*L*U
  as computed by SGETRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from SGETRF; for 1<=i<=N, row i of the
  matrix was interchanged with row IPIV(i).

B
  B is REAL array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is REAL array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by SGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

FERR
  FERR is REAL array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is REAL array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is REAL array, dimension (3*N)
```

```
 IWORK
   IWORK is INTEGER array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

   ITMAX is the maximum number of steps of iterative refinement.
```

———-

33.24.3 fortran code

— **sgerfs.f** —

```
*  =====================================================================
      SUBROUTINE SGERFS( TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB,
     $                   X, LDX, FERR, BERR, WORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      REAL               A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   BERR( * ), FERR( * ), WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
```

```
      REAL               TWO
      PARAMETER          ( TWO = 2.0E+0 )
      REAL               THREE
      PARAMETER          ( THREE = 3.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      CHARACTER          TRANST
      INTEGER            COUNT, I, J, K, KASE, NZ
      REAL               EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN, XK
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SAXPY, SCOPY, SGEMV, SGETRS, SLACN2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH
      EXTERNAL           LSAME, SLAMCH
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -10
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -12
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGERFS', -INFO )
         RETURN
```

```
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
      IF( NOTRAN ) THEN
         TRANST = 'T'
      ELSE
         TRANST = 'N'
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = N + 1
      EPS = SLAMCH( 'Epsilon' )
      SAFMIN = SLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 140 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - op(A) * X,
*        where op(A) = A, A**T, or A**H, depending on TRANS.
*
         CALL SCOPY( N, B( 1, J ), 1, WORK( N+1 ), 1 )
         CALL SGEMV( TRANS, N, N, -ONE, A, LDA, X( 1, J ), 1, ONE,
     $               WORK( N+1 ), 1 )
*
*        Compute componentwise relative backward error from formula
*
*        max(i) ( abs(R(i)) / ( abs(op(A))*abs(X) + abs(B) )(i) )
*
*        where abs(Z) is the componentwise absolute value of the matrix
*        or vector Z.  If the i-th component of the denominator is less
*        than SAFE2, then SAFE1 is added to the i-th components of the
```

```
*        numerator and denominator before dividing.
*
         DO 30 I = 1, N
            WORK( I ) = ABS( B( I, J ) )
   30    CONTINUE
*
*        Compute abs(op(A))*abs(X) + abs(B).
*
         IF( NOTRAN ) THEN
            DO 50 K = 1, N
               XK = ABS( X( K, J ) )
               DO 40 I = 1, N
                  WORK( I ) = WORK( I ) + ABS( A( I, K ) )*XK
   40          CONTINUE
   50       CONTINUE
         ELSE
            DO 70 K = 1, N
               S = ZERO
               DO 60 I = 1, N
                  S = S + ABS( A( I, K ) )*ABS( X( I, J ) )
   60          CONTINUE
               WORK( K ) = WORK( K ) + S
   70       CONTINUE
         END IF
         S = ZERO
         DO 80 I = 1, N
            IF( WORK( I ).GT.SAFE2 ) THEN
               S = MAX( S, ABS( WORK( N+I ) ) / WORK( I ) )
            ELSE
               S = MAX( S, ( ABS( WORK( N+I ) )+SAFE1 ) /
     $             ( WORK( I )+SAFE1 ) )
            END IF
   80    CONTINUE
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
*           1) The residual BERR(J) is larger than machine epsilon, and
*           2) BERR(J) decreased by at least a factor of 2 during the
*              last iteration, and
*           3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
*
*           Update solution and try again.
*
            CALL SGETRS( TRANS, N, 1, AF, LDAF, IPIV, WORK( N+1 ), N,
     $                   INFO )
            CALL SAXPY( N, ONE, WORK( N+1 ), 1, X( 1, J ), 1 )
            LSTRES = BERR( J )
```

```
            COUNT = COUNT + 1
            GO TO 20
         END IF
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(op(A)))*
*           ( abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
*          inv(op(A)) is the inverse of op(A)
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(op(A))*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(op(A))*abs(X) + abs(B) is less than SAFE2.
*
*        Use SLACN2 to estimate the infinity-norm of the matrix
*           inv(op(A)) * diag(W),
*        where W = abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) )))
*
         DO 90 I = 1, N
            IF( WORK( I ).GT.SAFE2 ) THEN
               WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I )
            ELSE
               WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I ) + SAFE1
            END IF
   90    CONTINUE
*
         KASE = 0
  100    CONTINUE
         CALL SLACN2( N, WORK( 2*N+1 ), WORK( N+1 ), IWORK, FERR( J ),
     $               KASE, ISAVE )
         IF( KASE.NE.0 ) THEN
            IF( KASE.EQ.1 ) THEN
*
*              Multiply by diag(W)*inv(op(A)**T).
*
               CALL SGETRS( TRANST, N, 1, AF, LDAF, IPIV, WORK( N+1 ),
     $                      N, INFO )
               DO 110 I = 1, N
                  WORK( N+I ) = WORK( I )*WORK( N+I )
  110          CONTINUE
            ELSE
*
```

```
*             Multiply by inv(op(A))*diag(W).
*
              DO 120 I = 1, N
                 WORK( N+I ) = WORK( I )*WORK( N+I )
  120         CONTINUE
              CALL SGETRS( TRANS, N, 1, AF, LDAF, IPIV, WORK( N+1 ), N,
     $                     INFO )
           END IF
           GO TO 100
        END IF
*
*       Normalize error.
*
        LSTRES = ZERO
        DO 130 I = 1, N
           LSTRES = MAX( LSTRES, ABS( X( I, J ) ) )
  130   CONTINUE
        IF( LSTRES.NE.ZERO )
     $     FERR( J ) = FERR( J ) / LSTRES
*
  140 CONTINUE
*
      RETURN
*
*     End of SGERFS
*
      END
```

———-

33.24.4 lisp code

— **sgerfs.lisp** —

```
(let* ((itmax 5) (zero 0.0) (one 1.0) (two 2.0) (three 3.0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 3.0 3.0) three)
           (ignorable itmax zero one two three))
  (defun sgerfs
         (trans n nrhs a lda af ldaf ipiv b ldb$ x ldx ferr berr work iwork
          info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array single-float (*)) work berr ferr x b af a)
             (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
             (type (string *) trans))
```

```
(f2cl-lib:with-multi-array-data
    ((trans character trans-%data% trans-%offset%)
     (a single-float a-%data% a-%offset%)
     (af single-float af-%data% af-%offset%)
     (b single-float b-%data% b-%offset%)
     (x single-float x-%data% x-%offset%)
     (ferr single-float ferr-%data% ferr-%offset%)
     (berr single-float berr-%data% berr-%offset%)
     (work single-float work-%data% work-%offset%)
     (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
     (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4)) (eps 0.0)
         (lstres 0.0) (s 0.0) (safe1 0.0) (safe2 0.0) (safmin 0.0) (xk 0.0)
         (count$ 0) (i 0) (j 0) (k 0) (kase 0) (nz 0)
         (transt
          (make-array '(1) :element-type 'character :initial-element #\ ))
         (notran nil))
    (declare (type (array f2cl-lib:integer4 (3)) isave)
             (type (single-float) eps lstres s safe1 safe2 safmin xk)
             (type (f2cl-lib:integer4) count$ i j k kase nz)
             (type (string 1) transt)
             (type f2cl-lib:logical notran))
    (setf info 0)
    (setf notran
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "N")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val))
    (cond
     ((and (not notran)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "T")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "C")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val)))
      (setf info -1))
     ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
     ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -5))
     ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -7))
     ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
```

```
  (setf info -10))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -12)))
(cond
 ((/= info 0) (xerbla "SGERFS" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%) zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) zero)
     label10))
  (go end_label)))
(cond (notran (f2cl-lib:f2cl-set-string transt "T" (string 1)))
      (t (f2cl-lib:f2cl-set-string transt "N" (string 1))))
(setf nz (f2cl-lib:int-add n 1))
(setf eps (slamch "Epsilon"))
(setf safmin (slamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy n
         (f2cl-lib:array-slice b-%data% single-float (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1
         (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                               ((1 *)) work-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (sgemv trans n n (- one) a lda
         (f2cl-lib:array-slice x-%data% single-float (1 j)
                               ((1 ldx) (1 *)) x-%offset%)
         1 one
         (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                               ((1 *)) work-%offset%)
         1)
      (declare (ignore var-3 var-4 var-6 var-7 var-9 var-10))
      (when var-0 (setf trans var-0))
```

```
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-5 (setf lda var-5))
  (when var-8 (setf one var-8)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (abs
             (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                            b-%offset%)))
   label30))
(cond
 (notran
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (setf xk
              (abs
               (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                              x-%offset%)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                  (+
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                   (*
                    (abs
                     (f2cl-lib:fref a-%data% (i k) ((1 lda) (1 *))
                                    a-%offset%))
                    xk)))
         label40))
     label50)))
 (t
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (setf s zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf s
                  (+ s
                     (*
                      (abs
                       (f2cl-lib:fref a-%data% (i k)
                                      ((1 lda) (1 *)) a-%offset%))
```

```
                        (abs
                         (f2cl-lib:fref x-%data% (i j)
                                        ((1 ldx) (1 *))
                                        x-%offset%)))))
           label60))
        (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                (+
                 (f2cl-lib:fref work-%data% (k) ((1 *))
                                work-%offset%)
                 s))
      label70))))
  (setf s zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
        (setf s
                (max s
                     (/
                      (abs
                       (f2cl-lib:fref work-%data%
                                      ((f2cl-lib:int-add n i))
                                      ((1 *)) work-%offset%))
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%)))))
       (t
        (setf s
                (max s
                     (/
                      (+
                       (abs
                        (f2cl-lib:fref work-%data%
                                       ((f2cl-lib:int-add n i))
                                       ((1 *)) work-%offset%))
                       safe1)
                      (+
                       (f2cl-lib:fref work-%data% (i) ((1 *))
                                      work-%offset%)
                       safe1))))))
     label80))
  (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
  (cond
   ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
         (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
         (<= count$ itmax))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (sgetrs trans n 1 af ldaf ipiv
         (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
```

```
                                ((1 *)) work-%offset%)
          n info)
        (declare (ignore var-2 var-3 var-5 var-6))
        (when var-0 (setf trans var-0))
        (when var-1 (setf n var-1))
        (when var-4 (setf ldaf var-4))
        (when var-7 (setf n var-7))
        (when var-8 (setf info var-8)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (saxpy n one
           (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                                ((1 *)) work-%offset%)
           1
           (f2cl-lib:array-slice x-%data% single-float (1 j)
                                ((1 ldx) (1 *)) x-%offset%)
           1)
        (declare (ignore var-2 var-3 var-4 var-5))
        (when var-0 (setf n var-0))
        (when var-1 (setf one var-1)))
      (setf lstres
              (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
      (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (cond
         ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (+
                   (abs
                    (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                   ((1 *)) work-%offset%))
                   (* nz eps
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%)))))
         (t
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (+
                   (abs
                    (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                   ((1 *)) work-%offset%))
                   (* nz eps
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%))
                   safe1))))
       label90))
    (setf kase 0)
   label100
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slacn2 n
```

```
      (f2cl-lib:array-slice work-%data% single-float
                            ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                            ((1 *)) work-%offset%)
      iwork
      (f2cl-lib:array-slice ferr-%data% single-float (j) ((1 *))
                            ferr-%offset%)
      kase isave)
    (declare (ignore var-1 var-2 var-3 var-4 var-6))
    (when var-0 (setf n var-0))
    (when var-5 (setf kase var-5)))
  (cond
   ((/= kase 0)
    (cond
     ((= kase 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (sgetrs transt n 1 af ldaf ipiv
           (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                                 ((1 *)) work-%offset%)
           n info)
        (declare (ignore var-2 var-3 var-5 var-6))
        (when var-0 (setf transt var-0))
        (when var-1 (setf n var-1))
        (when var-4 (setf ldaf var-4))
        (when var-7 (setf n var-7))
        (when var-8 (setf info var-8)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%)
                (*
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add n i)) ((1 *))
                                work-%offset%)))
         label110)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%)
                (*
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 (f2cl-lib:fref work-%data%
```

```
                                          ((f2cl-lib:int-add n i)) ((1 *))
                                          work-%offset%)))
                   label120))
                 (multiple-value-bind
                     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                     (sgetrs trans n 1 af ldaf ipiv
                      (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                                            ((1 *)) work-%offset%)
                      n info)
                   (declare (ignore var-2 var-3 var-5 var-6))
                   (when var-0 (setf trans var-0))
                   (when var-1 (setf n var-1))
                   (when var-4 (setf ldaf var-4))
                   (when var-7 (setf n var-7))
                   (when var-8 (setf info var-8)))))
               (go label100)))
             (setf lstres zero)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i n) nil)
               (tagbody
                 (setf lstres
                         (max lstres
                              (abs
                               (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                              x-%offset%))))
                label130))
             (if (/= lstres zero)
                 (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                         (/
                          (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                          lstres)))
            label140))
         (go end_label)
        end_label
         (return
          (values trans n nil nil lda nil ldaf nil nil nil nil nil nil nil nil
                  nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgerfs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
```

```
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::trans
  fortran-to-lisp::n nil nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldaf nil nil nil
  nil nil nil nil nil nil
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

33.25 sgerfsx

33.25.1 Axiom unit tests

— **sgerfsx.input** —

```
)set break resume
)sys rm -f sgerfsx.output
)spool sgerfsx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.25.2 Axiom help page [Demm05]

— sgerfsx.help —

```
==================================================================
sgerfsx examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgerfsx (TRANS, EQUED, N, NRHS, A, LDA, AF, LDAF, IPIV, R,
           C, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS, ERR_BNDS_NORM,
           ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, IWORK, INFO)
           SGERFSX

Function/Subroutine Documentation
   subroutine sgerfsx (character TRANS, character EQUED, integer N, integer
       NRHS, real, dimension( lda, * ) A, integer LDA, real, dimension( ldaf,
       * ) AF, integer LDAF, integer, dimension( * ) IPIV, real, dimension( *
       ) R, real, dimension( * ) C, real, dimension( ldb, * ) B, integer LDB,
       real, dimension( ldx , * ) X, integer LDX, real RCOND, real, dimension(
       * ) BERR, integer N_ERR_BNDS, real, dimension( nrhs, * ) ERR_BNDS_NORM,
       real, dimension( nrhs, * ) ERR_BNDS_COMP, integer NPARAMS, real,
       dimension( * ) PARAMS, real, dimension( * ) WORK, integer, dimension( *
       ) IWORK, integer INFO)
       SGERFSX

Purpose:
   SGERFSX improves the computed solution to a system of linear
   equations and provides error bounds and backward error estimates
   for the solution.  In addition to normwise error bound, the code
   provides maximum componentwise error bound if possible.  See
   comments for ERR_BNDS_NORM and ERR_BNDS_COMP for details of the
   error bounds.

   The original system of linear equations may have been equilibrated
   before calling this routine, as described by arguments EQUED, R
   and C below. In this case, the solution and error bounds returned
   are for the original unequilibrated system.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.
```

```
Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose = Transpose)

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done to A
   before calling this routine. This is needed to compute
   the solution and error bounds correctly.
   = 'N':  No equilibration
   = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
   = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
   = 'B':  Both row and column equilibration, i.e., A has been
           replaced by diag(R) * A * diag(C).
   The right hand side B has been changed accordingly.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 A
   A is REAL array, dimension (LDA,N)
   The original N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is REAL array, dimension (LDAF,N)
   The factors L and U from the factorization A = P*L*U
   as computed by SGETRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
```

```
  The pivot indices from SGETRF; for 1<=i<=N, row i of the
  matrix was interchanged with row IPIV(i).

R
  R is REAL array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.
  If R is accessed, each element of R should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

C
  C is REAL array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.
  If C is accessed, each element of C should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is REAL array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is REAL array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by SGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
```

```
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

BERR
  BERR is REAL array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
           max_j (abs(XTRUE(j,i) - X(j,i)))
            ------------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
    reciprocal condition number is less than the threshold
    sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
    almost certainly within a factor of 10 of the true error
    so long as the next entry is greater than the threshold
    sqrt(n) * slamch('Epsilon'). This error bound should only
    be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
    reciprocal condition number.  Compared with the threshold
    sqrt(n) * slamch('Epsilon') to determine if the error
    estimate is "guaranteed". These reciprocal condition
```

```
    numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
    appropriately scaled matrix Z.
    Let Z = S*A, where S scales each row by a power of the
    radix so all absolute row sums of Z are approximately 1.

ERR_BNDS_COMP
  ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                 abs(XTRUE(j,i) - X(j,i))
            max_j ----------------------
                        abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
    reciprocal condition number is less than the threshold
    sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
    almost certainly within a factor of 10 of the true error
    so long as the next entry is greater than the threshold
    sqrt(n) * slamch('Epsilon'). This error bound should only
    be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
    reciprocal condition number.  Compared with the threshold
    sqrt(n) * slamch('Epsilon') to determine if the error
    estimate is "guaranteed". These reciprocal condition
    numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
    appropriately scaled matrix Z.
    Let Z = S*(A*diag(x)), where x is the solution for the
    current right-hand side and S scales each row of
    A*diag(x) by a power of the radix so all absolute row
    sums of Z are approximately 1.
```

```
NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is REAL array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
          (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be
              trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is REAL array, dimension (4*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is
  guaranteed.
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
  has been completed, but the factor U is exactly singular, so
```

```
   the solution and error bounds could not be computed. RCOND = 0
   is returned.
   = N+J: The solution corresponding to the Jth right-hand side is
      not guaranteed. The solutions corresponding to other right-
      hand sides K with K > J may not be guaranteed as well, but
      only the first such right-hand side is reported. If a small
      componentwise error is not requested (PARAMS(3) = 0.0) then
      the Jth right-hand side is the first with a normwise error
      bound that is not guaranteed (the smallest J such
      that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
      the Jth right-hand side is the first with either a normwise or
      componentwise error bound that is not guaranteed (the smallest
      J such that either ERR_BNDS_NORM(J,1) = 0.0 or
      ERR_BNDS_COMP(J,1) = 0.0). See the definition of
      ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
      about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

———-

33.25.3 fortran code

— sgerfsx.f —

```
*  ====================================================================
      SUBROUTINE SGERFSX( TRANS, EQUED, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $                    R, C, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS,
     $                    ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $                    WORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS, EQUED
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      REAL               A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX , * ), WORK( * )
      REAL               R( * ), C( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               ITREF_DEFAULT, ITHRESH_DEFAULT,
     $                   COMPONENTWISE_DEFAULT
      REAL               RTHRESH_DEFAULT, DZTHRESH_DEFAULT
      PARAMETER          ( ITREF_DEFAULT = 1.0 )
      PARAMETER          ( ITHRESH_DEFAULT = 10.0 )
      PARAMETER          ( COMPONENTWISE_DEFAULT = 1.0 )
      PARAMETER          ( RTHRESH_DEFAULT = 0.5 )
      PARAMETER          ( DZTHRESH_DEFAULT = 0.25 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. Local Scalars ..
      CHARACTER(1)       NORM
      LOGICAL            ROWEQU, COLEQU, NOTRAN
      INTEGER            J, TRANS_TYPE, PREC_TYPE, REF_TYPE
      INTEGER            N_NORMS
      REAL               ANORM, RCOND_TMP
      REAL               ILLRCOND_THRESH, ERR_LBND, CWISE_WRONG
      LOGICAL            IGNORE_CWISE
      INTEGER            ITHRESH
      REAL               RTHRESH, UNSTABLE_THRESH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, SGECON, SLA_GERFSX_EXTENDED
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, BLAS_FPINFO_X, ILATRANS, ILAPREC
      EXTERNAL           SLAMCH, SLANGE, SLA_GERCOND
      REAL               SLAMCH, SLANGE, SLA_GERCOND
      LOGICAL            LSAME
      INTEGER            BLAS_FPINFO_X
      INTEGER            ILATRANS, ILAPREC
*     ..
```

```
*     .. Executable Statements ..
*
*     Check the input parameters.
*
      INFO = 0
      TRANS_TYPE = ILATRANS( TRANS )
      REF_TYPE = INT( ITREF_DEFAULT )
      IF ( NPARAMS .GE. LA_LINRX_ITREF_I ) THEN
         IF ( PARAMS( LA_LINRX_ITREF_I ) .LT. 0.0 ) THEN
            PARAMS( LA_LINRX_ITREF_I ) = ITREF_DEFAULT
         ELSE
            REF_TYPE = PARAMS( LA_LINRX_ITREF_I )
         END IF
      END IF
*
*     Set default parameters.
*
      ILLRCOND_THRESH = REAL( N ) * SLAMCH( 'Epsilon' )
      ITHRESH = INT( ITHRESH_DEFAULT )
      RTHRESH = RTHRESH_DEFAULT
      UNSTABLE_THRESH = DZTHRESH_DEFAULT
      IGNORE_CWISE = COMPONENTWISE_DEFAULT .EQ. 0.0
*
      IF ( NPARAMS.GE.LA_LINRX_ITHRESH_I ) THEN
         IF ( PARAMS( LA_LINRX_ITHRESH_I ).LT.0.0 ) THEN
            PARAMS( LA_LINRX_ITHRESH_I ) = ITHRESH
         ELSE
            ITHRESH = INT( PARAMS( LA_LINRX_ITHRESH_I ) )
         END IF
      END IF
      IF ( NPARAMS.GE.LA_LINRX_CWISE_I ) THEN
         IF ( PARAMS( LA_LINRX_CWISE_I ).LT.0.0 ) THEN
            IF ( IGNORE_CWISE ) THEN
               PARAMS( LA_LINRX_CWISE_I ) = 0.0
            ELSE
               PARAMS( LA_LINRX_CWISE_I ) = 1.0
            END IF
         ELSE
            IGNORE_CWISE = PARAMS( LA_LINRX_CWISE_I ) .EQ. 0.0
         END IF
      END IF
      IF ( REF_TYPE .EQ. 0 .OR. N_ERR_BNDS .EQ. 0 ) THEN
         N_NORMS = 0
      ELSE IF ( IGNORE_CWISE ) THEN
         N_NORMS = 1
      ELSE
         N_NORMS = 2
      END IF
*
      NOTRAN = LSAME( TRANS, 'N' )
```

```
      ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
      COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
*
*     Test input parameters.
*
      IF( TRANS_TYPE.EQ.-1 ) THEN
        INFO = -1
      ELSE IF( .NOT.ROWEQU .AND. .NOT.COLEQU .AND.
     $         .NOT.LSAME( EQUED, 'N' ) ) THEN
        INFO = -2
      ELSE IF( N.LT.0 ) THEN
        INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
        INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
        INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
        INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
        INFO = -13
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
        INFO = -15
      END IF
      IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'SGERFSX', -INFO )
        RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         RCOND = 1.0
         DO J = 1, NRHS
            BERR( J ) = 0.0
            IF ( N_ERR_BNDS .GE. 1 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
            IF ( N_ERR_BNDS .GE. 2 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I) = 0.0
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 0.0
            END IF
            IF ( N_ERR_BNDS .GE. 3 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 1.0
            END IF
         END DO
         RETURN
      END IF
*
```

```
*     Default to failure.
*
      RCOND = 0.0
      DO J = 1, NRHS
         BERR( J ) = 1.0
         IF ( N_ERR_BNDS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
            ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
         END IF
         IF ( N_ERR_BNDS .GE. 2 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
         END IF
         IF ( N_ERR_BNDS .GE. 3 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 0.0
            ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 0.0
         END IF
      END DO
*
*     Compute the norm of A and the reciprocal of the condition
*     number of A.
*
      IF( NOTRAN ) THEN
         NORM = 'I'
      ELSE
         NORM = '1'
      END IF
      ANORM = SLANGE( NORM, N, N, A, LDA, WORK )
      CALL SGECON( NORM, N, AF, LDAF, ANORM, RCOND, WORK, IWORK, INFO )
*
*     Perform refinement on each right-hand side
*
      IF ( REF_TYPE .NE. 0 ) THEN

         PREC_TYPE = ILAPREC( 'D' )

         IF ( NOTRAN ) THEN
            CALL SLA_GERFSX_EXTENDED( PREC_TYPE, TRANS_TYPE,  N,
     $           NRHS, A, LDA, AF, LDAF, IPIV, COLEQU, C, B,
     $           LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM,
     $           ERR_BNDS_COMP, WORK(N+1), WORK(1), WORK(2*N+1),
     $           WORK(1), RCOND, ITHRESH, RTHRESH, UNSTABLE_THRESH,
     $           IGNORE_CWISE, INFO )
         ELSE
            CALL SLA_GERFSX_EXTENDED( PREC_TYPE, TRANS_TYPE,  N,
     $           NRHS, A, LDA, AF, LDAF, IPIV, ROWEQU, R, B,
     $           LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM,
     $           ERR_BNDS_COMP, WORK(N+1), WORK(1), WORK(2*N+1),
     $           WORK(1), RCOND, ITHRESH, RTHRESH, UNSTABLE_THRESH,
     $           IGNORE_CWISE, INFO )
```

```
         END IF
      END IF

      ERR_LBND = MAX( 10.0, SQRT( REAL( N ) ) ) * SLAMCH( 'Epsilon' )
      IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 1 ) THEN
*
*     Compute scaled normwise condition number cond(A*C).
*
         IF ( COLEQU .AND. NOTRAN ) THEN
            RCOND_TMP = SLA_GERCOND( TRANS, N, A, LDA, AF, LDAF, IPIV,
     $           -1, C, INFO, WORK, IWORK )
         ELSE IF ( ROWEQU .AND. .NOT. NOTRAN ) THEN
            RCOND_TMP = SLA_GERCOND( TRANS, N, A, LDA, AF, LDAF, IPIV,
     $           -1, R, INFO, WORK, IWORK )
         ELSE
            RCOND_TMP = SLA_GERCOND( TRANS, N, A, LDA, AF, LDAF, IPIV,
     $           0, R, INFO, WORK, IWORK )
         END IF
         DO J = 1, NRHS
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .GT. 1.0 )
     $           ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 0.0
               IF ( INFO .LE. N ) INFO = N + J
            ELSE IF ( ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .LT. ERR_LBND )
     $      THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF
         END DO
      END IF

      IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 2 ) THEN
*
*     Compute componentwise condition number cond(A*diag(Y(:,J))) for
*     each right-hand side using the current solution as an estimate of
```

```
*     the true solution.  If the componentwise error estimate is too
*     large, then the solution is a lousy estimate of truth and the
*     estimated RCOND may be too optimistic.  To avoid misleading users,
*     the inverse condition number is set to 0.0 when the estimated
*     cwise error is at least CWISE_WRONG.
*
         CWISE_WRONG = SQRT( SLAMCH( 'Epsilon' ) )
         DO J = 1, NRHS
            IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .LT. CWISE_WRONG )
     $           THEN
               RCOND_TMP = SLA_GERCOND( TRANS, N, A, LDA, AF, LDAF,
     $              IPIV, 1, X(1,J), INFO, WORK, IWORK )
            ELSE
               RCOND_TMP = 0.0
            END IF
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .GT. 1.0 )
     $           ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 0.0
               IF ( PARAMS( LA_LINRX_CWISE_I ) .EQ. 1.0
     $              .AND. INFO.LT.N + J ) INFO = N + J
            ELSE IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I )
     $              .LT. ERR_LBND ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF
         END DO
      END IF
*
      RETURN
*
*     End of SGERFSX
*
      END
```

———-

33.25.4 lisp code

— sgerfsx.lisp —

———-

33.26 sgerq2

33.26.1 Axiom unit tests

— sgerq2.input —

```
)set break resume
)sys rm -f sgerq2.output
)spool sgerq2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.26.2 Axiom help page

— sgerq2.help —

```
======================================================================
sgerq2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgerq2 (M, N, A, LDA, TAU, WORK, INFO)
           SGERQ2 computes the RQ factorization of a general rectangular
           matrix using an unblocked algorithm.
```

```
Function/Subroutine Documentation
   subroutine sgerq2 (integer M, integer N, real, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) TAU, real, dimension( * ) WORK,
       integer INFO)
       SGERQ2 computes the RQ factorization of a general rectangular matrix
       using an unblocked algorithm.

Purpose:
   SGERQ2 computes an RQ factorization of a real m by n matrix A: A = R * Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the m by n matrix A.
   On exit, if m <= n, the upper triangle of the subarray
   A(1:m,n-m+1:n) contains the m by m upper triangular matrix R;
   if m >= n, the elements on and above the (m-n)-th subdiagonal
   contain the m by n upper trapezoidal matrix R; the remaining
   elements, with the array TAU, represent the orthogonal matrix
   Q as a product of elementary reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is REAL array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is REAL array, dimension (M)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

 Further Details:

   The matrix Q is represented as a product of elementary reflectors
```

```
      Q = H(1) H(2) . . . H(k), where k = min(m,n).

   Each H(i) has the form

      H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(n-k+i+1:n) = 0 and v(n-k+i) = 1; v(1:n-k+i-1) is stored on exit in
   A(m-k+i,1:n-k+i-1), and tau in TAU(i).
```

33.26.3 fortran code

— sgerq2.f —

```
*  =====================================================================
      SUBROUTINE SGERQ2( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      REAL               AII
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLARF, SLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
```

```
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGERQ2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = K, 1, -1
*
*        Generate elementary reflector H(i) to annihilate
*        A(m-k+i,1:n-k+i-1)
*
         CALL SLARFG( N-K+I, A( M-K+I, N-K+I ), A( M-K+I, 1 ), LDA,
     $                TAU( I ) )
*
*        Apply H(i) to A(1:m-k+i-1,1:n-k+i) from the right
*
         AII = A( M-K+I, N-K+I )
         A( M-K+I, N-K+I ) = ONE
         CALL SLARF( 'Right', M-K+I-1, N-K+I, A( M-K+I, 1 ), LDA,
     $               TAU( I ), A, LDA, WORK )
         A( M-K+I, N-K+I ) = AII
   10 CONTINUE
      RETURN
*
*     End of SGERQ2
*
      END
```

———-

33.26.4 lisp code

— sgerq2.lisp —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun sgerq2 (m n a lda tau work info)
    (declare (type (array single-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (tau single-float tau-%data% tau-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((aii 0.0) (i 0) (k 0))
        (declare (type (single-float) aii)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "SGERQ2" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (f2cl-lib:fdo (i k (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (slarfg (f2cl-lib:int-add (f2cl-lib:int-sub n k) i)
                 (f2cl-lib:array-slice a-%data% single-float
                                       ((+ m (f2cl-lib:int-sub k) i)
                                        (f2cl-lib:int-add
                                         (f2cl-lib:int-sub n k) i))
                                       ((1 lda) (1 *)) a-%offset%)
                 (f2cl-lib:array-slice a-%data% single-float
                                       ((+ m (f2cl-lib:int-sub k) i) 1)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                       tau-%offset%))
              (declare (ignore var-0 var-1 var-2 var-4))
              (when var-3 (setf lda var-3)))
            (setf aii
                    (f2cl-lib:fref a-%data%
                                   ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                    (f2cl-lib:int-add (f2cl-lib:int-sub n k)
                                                      i))
                                   ((1 lda) (1 *)) a-%offset%))
            (setf (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                  (f2cl-lib:int-add (f2cl-lib:int-sub n k) i))
                                 ((1 lda) (1 *)) a-%offset%)
```

```
                one)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (slarf "Right"
                 (f2cl-lib:int-sub (f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                   1)
                 (f2cl-lib:int-add (f2cl-lib:int-sub n k) i)
                 (f2cl-lib:array-slice a-%data% single-float
                                       ((+ m (f2cl-lib:int-sub k) i) 1)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                       tau-%offset%)
                 a lda work)
              (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8))
              (when var-4 (setf lda var-4))
              (when var-7 (setf lda var-7)))
            (setf (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                  (f2cl-lib:int-add (f2cl-lib:int-sub n k) i))
                                 ((1 lda) (1 *)) a-%offset%)
                aii)
          label10))
        (go end_label)
       end_label
        (return (values nil nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgerq2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::lda
                                              nil nil fortran-to-lisp::info)
                                            :calls 'nil)))
```

33.27 sgerqf

33.27.1 Axiom unit tests

— sgerqf.input —

```
)set break resume
)sys rm -f sgerqf.output
)spool sgerqf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.27.2 Axiom help page

— sgerqf.help —

```
======================================================================
sgerqf examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgerqf (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           SGERQF

Function/Subroutine Documentation
   subroutine sgerqf (integer M, integer N, real, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) TAU, real, dimension( * ) WORK,
       integer LWORK, integer INFO)
       SGERQF

Purpose:
   SGERQF computes an RQ factorization of a real M-by-N matrix A: A = R * Q.

Parameters:
```

```
 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit,
   if m <= n, the upper triangle of the subarray
   A(1:m,n-m+1:n) contains the M-by-M upper triangular matrix R;
   if m >= n, the elements on and above the (m-n)-th subdiagonal
   contain the M-by-N upper trapezoidal matrix R;
   the remaining elements, with the array TAU, represent the
   orthogonal matrix Q as a product of min(m,n) elementary
   reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is REAL array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is REAL array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.  LWORK >= max(1,M).
   For optimum performance LWORK >= M*NB, where NB is
   the optimal blocksize.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details:
```

```
The matrix Q is represented as a product of elementary reflectors

   Q = H(1) H(2) . . . H(k), where k = min(m,n).

Each H(i) has the form

   H(i) = I - tau * v * v**T

where tau is a real scalar, and v is a real vector with
v(n-k+i+1:n) = 0 and v(n-k+i) = 1; v(1:n-k+i-1) is stored on exit in
A(m-k+i,1:n-k+i-1), and tau in TAU(i).
```

———-

33.27.3 fortran code

— **sgerqf.f** —

```
*  =====================================================================
      SUBROUTINE SGERQF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, KI, KK, LDWORK, LWKOPT,
     $                   MU, NB, NBMIN, NU, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGERQ2, SLARFB, SLARFT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
```

```
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, M ) .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
*
      IF( INFO.EQ.0 ) THEN
         K = MIN( M, N )
         IF( K.EQ.0 ) THEN
            LWKOPT = 1
         ELSE
            NB = ILAENV( 1, 'SGERQF', ' ', M, N, -1, -1 )
            LWKOPT = M*NB
            WORK( 1 ) = LWKOPT
         END IF
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.MAX( 1, M ) .AND. .NOT.LQUERY ) THEN
            INFO = -7
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGERQF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( K.EQ.0 ) THEN
         RETURN
      END IF
*
      NBMIN = 2
      NX = 1
```

```
      IWS = M
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'SGERQF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = M
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'SGERQF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially.
*        The last kk rows are handled by the block method.
*
         KI = ( ( K-NX-1 ) / NB )*NB
         KK = MIN( K, KI+NB )
*
         DO 10 I = K - KK + KI + 1, K - KK + 1, -NB
            IB = MIN( K-I+1, NB )
*
*           Compute the RQ factorization of the current block
*           A(m-k+i:m-k+i+ib-1,1:n-k+i+ib-1)
*
            CALL SGERQ2( IB, N-K+I+IB-1, A( M-K+I, 1 ), LDA, TAU( I ),
     $                   WORK, IINFO )
            IF( M-K+I.GT.1 ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i+ib-1) . . . H(i+1) H(i)
*
               CALL SLARFT( 'Backward', 'Rowwise', N-K+I+IB-1, IB,
     $                      A( M-K+I, 1 ), LDA, TAU( I ), WORK, LDWORK )
*
*              Apply H to A(1:m-k+i-1,1:n-k+i+ib-1) from the right
*
```

```
               CALL SLARFB( 'Right', 'No transpose', 'Backward',
     $                      'Rowwise', M-K+I-1, N-K+I+IB-1, IB,
     $                      A( M-K+I, 1 ), LDA, WORK, LDWORK, A, LDA,
     $                      WORK( IB+1 ), LDWORK )
            END IF
   10    CONTINUE
         MU = M - K + I + NB - 1
         NU = N - K + I + NB - 1
      ELSE
         MU = M
         NU = N
      END IF
*
*     Use unblocked code to factor the last or only block
*
      IF( MU.GT.0 .AND. NU.GT.0 )
     $   CALL SGERQ2( MU, NU, A, LDA, TAU, WORK, IINFO )
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of SGERQF
*
      END
```

———-

33.27.4 lisp code

— **sgerqf.lisp** —

```
(defun sgerqf (m n a lda tau work lwork info)
  (declare (type (array single-float (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a single-float a-%data% a-%offset%)
       (tau single-float tau-%data% tau-%offset%)
       (work single-float work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ki 0) (kk 0) (ldwork 0)
           (lwkopt 0) (mu 0) (nb 0) (nbmin 0) (nu 0) (nx 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) nx nu nbmin nb mu lwkopt ldwork kk ki
                k iws iinfo ib i))
      (setf info 0)
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4))
```

```
      ((and
        (< lwork
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
        (not lquery))
       (setf info -7)))
(cond
 ((= info 0)
  (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
  (cond ((= k 0) (setf lwkopt 1))
        (t
         (setf nb
                 (multiple-value-bind
                     (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                     (ilaenv 1 "SGERQF" " " m n -1 -1)
                   (declare (ignore var-0 var-1 var-2 var-5 var-6))
                   (setf m var-3)
                   (setf n var-4)
                   ret-val))
         (setf lwkopt (f2cl-lib:int-mul m nb))
         (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                 (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
  (cond
   ((and
     (< lwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
     (not lquery))
    (setf info -7)))))
(cond
 ((/= info 0) (xerbla "SGERQF" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond ((= k 0) (go end_label)))
(setf nbmin 2)
(setf nx 1)
(setf iws m)
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "SGERQF" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (setf m var-3)
                      (setf n var-4)
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork m) (setf iws (f2cl-lib:int-mul ldwork nb))
```

```
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
                            (ilaenv 2 "SGERQF" " " m n -1 -1)
                          (declare
                           (ignore var-0 var-1 var-2 var-5 var-6))
                          (setf m var-3)
                          (setf n var-4)
                          ret-val)))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (setf ki (* (the f2cl-lib:integer4 (truncate (- k nx 1) nb)) nb))
  (setf kk
          (min (the f2cl-lib:integer4 k)
               (the f2cl-lib:integer4 (f2cl-lib:int-add ki nb))))
  (f2cl-lib:fdo (i (f2cl-lib:int-add k (f2cl-lib:int-sub kk) ki 1)
                 (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
                ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub kk) 1)) nil)
    (tagbody
      (setf ib
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
               (the f2cl-lib:integer4 nb)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (sgerq2 ib
           (f2cl-lib:int-sub
            (f2cl-lib:int-add (f2cl-lib:int-sub n k) i ib) 1)
           (f2cl-lib:array-slice a-%data% single-float
                                 ((+ m (f2cl-lib:int-sub k) i) 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                 tau-%offset%)
           work iinfo)
        (declare (ignore var-0 var-1 var-2 var-4 var-5))
        (setf lda var-3)
        (setf iinfo var-6))
      (cond
       ((> (f2cl-lib:int-add m (f2cl-lib:int-sub k) i) 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (slarft "Backward" "Rowwise"
             (f2cl-lib:int-sub
```

```
                 (f2cl-lib:int-add (f2cl-lib:int-sub n k) i ib) 1)
                ib
                (f2cl-lib:array-slice a-%data% single-float
                                      ((+ m (f2cl-lib:int-sub k) i) 1)
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                      tau-%offset%)
                work ldwork)
              (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
              (when var-3 (setf ib var-3))
              (when var-5 (setf lda var-5))
              (when var-8 (setf ldwork var-8)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                 var-10 var-11 var-12 var-13 var-14)
                (slarfb "Right" "No transpose" "Backward" "Rowwise"
                 (f2cl-lib:int-sub
                  (f2cl-lib:int-add (f2cl-lib:int-sub m k) i) 1)
                 (f2cl-lib:int-sub
                  (f2cl-lib:int-add (f2cl-lib:int-sub n k) i ib) 1)
                 ib
                 (f2cl-lib:array-slice a-%data% single-float
                                       ((+ m (f2cl-lib:int-sub k) i) 1)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda work ldwork a lda
                 (f2cl-lib:array-slice work-%data% single-float ((+ ib 1))
                                       ((1 *)) work-%offset%)
                 ldwork)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
                var-13))
              (when var-6 (setf ib var-6))
              (when var-8 (setf lda var-8))
              (when var-10 (setf ldwork var-10))
              (when var-12 (setf lda var-12))
              (when var-14 (setf ldwork var-14)))))
        label10))
      (setf mu
              (f2cl-lib:int-sub
               (f2cl-lib:int-add (f2cl-lib:int-sub m k) i nb) 1))
      (setf nu
              (f2cl-lib:int-sub
               (f2cl-lib:int-add (f2cl-lib:int-sub n k) i nb) 1)))
     (t (setf mu m) (setf nu n)))
    (if (and (> mu 0) (> nu 0))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (sgerq2 mu nu a lda tau work iinfo)
          (declare (ignore var-0 var-1 var-2 var-4 var-5))
          (setf lda var-3)
```

```
            (setf iinfo var-6)))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 iws) 'single-float))
      (go end_label)
     end_label
      (return (values m n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgerqf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::sgerq2
                                              fortran-to-lisp::ilaenv))))
```

———-

33.28 sgesvj

33.28.1 Axiom unit tests

— sgesvj.input —

```
)set break resume
)sys rm -f sgesvj.output
)spool sgesvj.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

33.28.2 Axiom help page

— **sgesvj.help** —

```
==================================================================
sgesvj examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgesvj (JOBA, JOBU, JOBV, M, N, A, LDA, SVA, MV, V, LDV,
           WORK, LWORK, INFO)
           SGESVJ

Function/Subroutine Documentation
   subroutine sgesvj (character*1 JOBA, character*1 JOBU, character*1 JOBV,
       integer M, integer N, real, dimension( lda, * ) A, integer LDA, real,
       dimension( n ) SVA, integer MV, real, dimension( ldv, * ) V, integer
       LDV, real, dimension( lwork ) WORK, integer LWORK, integer INFO)
       SGESVJ

Purpose:
   SGESVJ computes the singular value decomposition (SVD) of a real
   M-by-N matrix A, where M >= N. The SVD of A is written as
                                    [++]   [xx]   [x0]   [xx]
             A = U * SIGMA * V^t,   [++] = [xx] * [ox] * [xx]
                                    [++]   [xx]
   where SIGMA is an N-by-N diagonal matrix, U is an M-by-N orthonormal
   matrix, and V is an N-by-N orthogonal matrix. The diagonal elements
   of SIGMA are the singular values of A. The columns of U and V are the
   left and the right singular vectors of A, respectively.
   SGESVJ can sometimes compute tiny singular values and their singular
   vectors much more accurately than other SVD routines,
   see below under Further Details.

Parameters:

 JOBA
   JOBA is CHARACTER* 1
```

```
  Specifies the structure of A.
  = 'L': The input matrix A is lower triangular;
  = 'U': The input matrix A is upper triangular;
  = 'G': The input matrix A is general M-by-N matrix, M >= N.

JOBU
  JOBU is CHARACTER*1
  Specifies whether to compute the left singular vectors
  (columns of U):
  = 'U': The left singular vectors corresponding to the nonzero
     singular values are computed and returned in the leading
     columns of A. See more details in the description of A.
     The default numerical orthogonality threshold is set to
     approximately TOL=CTOL*EPS, CTOL=SQRT(M), EPS=SLAMCH('E').
  = 'C': Analogous to JOBU='U', except that user can control the
     level of numerical orthogonality of the computed left
     singular vectors. TOL can be set to TOL = CTOL*EPS, where
     CTOL is given on input in the array WORK.
     No CTOL smaller than ONE is allowed. CTOL greater
     than 1 / EPS is meaningless. The option 'C'
     can be used if M*EPS is satisfactory orthogonality
     of the computed left singular vectors, so CTOL=M could
     save few sweeps of Jacobi rotations.
     See the descriptions of A and WORK(1).
  = 'N': The matrix U is not computed. However, see the description of A.

JOBV
  JOBV is CHARACTER*1
  Specifies whether to compute the right singular vectors, that
  is, the matrix V:
  = 'V' : the matrix V is computed and returned in the array V
  = 'A' : the Jacobi rotations are applied to the MV-by-N
     array V. In other words, the right singular vector
     matrix V is not computed explicitly; instead it is
     applied to an MV-by-N matrix initially stored in the first MV rows of V.
  = 'N' : the matrix V is not computed and the array V is not referenced

M
  M is INTEGER
  The number of rows of the input matrix A. 1/SLAMCH('E') > M >= 0.

N
  N is INTEGER
  The number of columns of the input matrix A.
  M >= N >= 0.

A
  A is REAL array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit,
```

```
  If JOBU .EQ. 'U' .OR. JOBU .EQ. 'C':
    If INFO .EQ. 0 :
      RANKA orthonormal columns of U are returned in the
      leading RANKA columns of the array A. Here RANKA <= N
      is the number of computed singular values of A that are
      above the underflow threshold SLAMCH('S'). The singular
      vectors corresponding to underflowed or zero singular
      values are not computed. The value of RANKA is returned
      in the array WORK as RANKA=NINT(WORK(2)). Also see the
      descriptions of SVA and WORK. The computed columns of U
      are mutually numerically orthogonal up to approximately
      TOL=SQRT(M)*EPS (default); or TOL=CTOL*EPS (JOBU.EQ.'C'),
      see the description of JOBU.
    If INFO .GT. 0,
      the procedure SGESVJ did not converge in the given number
      of iterations (sweeps). In that case, the computed
      columns of U may not be orthogonal up to TOL. The output
      U (stored in A), SIGMA (given by the computed singular
      values in SVA(1:N)) and V is still a decomposition of the
      input matrix A in the sense that the residual
      ||A-SCALE*U*SIGMA*V^T||_2 / ||A||_2 is small.
  If JOBU .EQ. 'N':
    If INFO .EQ. 0 :
      Note that the left singular vectors are 'for free' in the
      one-sided Jacobi SVD algorithm. However, if only the
      singular values are needed, the level of numerical
      orthogonality of U is not an issue and iterations are
      stopped when the columns of the iterated matrix are
      numerically orthogonal up to approximately M*EPS. Thus,
      on exit, A contains the columns of U scaled with the
      corresponding singular values.
    If INFO .GT. 0 :
      the procedure SGESVJ did not converge in the given number
      of iterations (sweeps).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

SVA
  SVA is REAL array, dimension (N)
  On exit,
  If INFO .EQ. 0 :
  depending on the value SCALE = WORK(1), we have:
    If SCALE .EQ. ONE:
      SVA(1:N) contains the computed singular values of A.
      During the computation SVA contains the Euclidean column
      norms of the iterated matrices in the array A.
    If SCALE .NE. ONE:
      The singular values of A are SCALE*SVA(1:N), and this
```

```
      factored representation is due to the fact that some of the
      singular values of A might underflow or overflow.

  If INFO .GT. 0 :
    the procedure SGESVJ did not converge in the given number of
    iterations (sweeps) and SCALE*SVA(1:N) may not be accurate.

MV
  MV is INTEGER
  If JOBV .EQ. 'A', then the product of Jacobi rotations in SGESVJ
  is applied to the first MV rows of V. See the description of JOBV.

V
  V is REAL array, dimension (LDV,N)
  If JOBV = 'V', then V contains on exit the N-by-N matrix of
     the right singular vectors;
  If JOBV = 'A', then V contains the product of the computed right
     singular vector matrix and the initial matrix in the array V.
  If JOBV = 'N', then V is not referenced.

LDV
  LDV is INTEGER
  The leading dimension of the array V, LDV .GE. 1.
  If JOBV .EQ. 'V', then LDV .GE. max(1,N).
  If JOBV .EQ. 'A', then LDV .GE. max(1,MV) .

WORK
  WORK is REAL array, dimension max(4,M+N).
  On entry,
  If JOBU .EQ. 'C' :
    WORK(1) = CTOL, where CTOL defines the threshold for convergence.
      The process stops if all columns of A are mutually
      orthogonal up to CTOL*EPS, EPS=SLAMCH('E').
      It is required that CTOL >= ONE, i.e. it is not
      allowed to force the routine to obtain orthogonality
      below EPSILON.
  On exit,
    WORK(1) = SCALE is the scaling factor such that SCALE*SVA(1:N)
      are the computed singular vcalues of A.
      (See description of SVA().)
    WORK(2) = NINT(WORK(2)) is the number of the computed nonzero
      singular values.
    WORK(3) = NINT(WORK(3)) is the number of the computed singular
      values that are larger than the underflow threshold.
    WORK(4) = NINT(WORK(4)) is the number of sweeps of Jacobi
      rotations needed for numerical convergence.
    WORK(5) = max_{i.NE.j} |COS(A(:,i),A(:,j))| in the last sweep.
      This is useful information in cases when SGESVJ did
      not converge, as it can be used to estimate whether
      the output is stil useful and for post festum analysis.
```

```
     WORK(6) = the largest absolute value over all sines of the
       Jacobi rotation angles in the last sweep. It can be
       useful for a post festum analysis.

 LWORK
   LWORK is INTEGER
   length of WORK, WORK >= MAX(6,M+N)

 INFO
   INFO is INTEGER
   = 0 : successful exit.
   < 0 : if INFO = -i, then the i-th argument had an illegal value
   > 0 : SGESVJ did not converge in the maximal allowed number (30)
         of sweeps. The output may still be useful. See the
         description of WORK.

 Further Details:

   The orthogonal N-by-N matrix V is obtained as a product of Jacobi
   plane rotations. The rotations are implemented as fast scaled
   rotations of Anda and Park [1]. In the case of underflow of the
   Jacobi angle, a modified Jacobi transformation of Drmac [4] is
   used. Pivot strategy uses column interchanges of de Rijk [2]. The
   relative accuracy of the computed singular values and the accuracy
   of the computed singular vectors (in angle metric) is as guaranteed
   by the theory of Demmel and Veselic [3]. The condition number that
   determines the accuracy in the full rank case is essentially
   min_{D=diag} kappa(A*D), where kappa(.) is the spectral condition
   number. The best performance of this Jacobi SVD procedure is
   achieved if used in an accelerated version of Drmac and Veselic
   [5,6], and it is the kernel routine in the SIGMA library [7]. Some
   tunning parameters (marked with [TP]) are available for the
   implementer.
   The computational range for the nonzero singular values is the
   machine number interval ( UNDERFLOW , OVERFLOW ). In extreme cases,
   even denormalized singular values can be computed with the
   corresponding gradual loss of accurate digits.
```

———-

33.28.3 fortran code

— sgesvj.f —

```
*  ===================================================================
      SUBROUTINE SGESVJ( JOBA, JOBU, JOBV, M, N, A, LDA, SVA, MV, V,
     $                   LDV, WORK, LWORK, INFO )
```

```
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDV, LWORK, M, MV, N
      CHARACTER*1        JOBA, JOBU, JOBV
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), SVA( N ), V( LDV, * ),
     $                   WORK( LWORK )
*     ..
*
*  =====================================================================
*
*     .. Local Parameters ..
      REAL               ZERO, HALF, ONE
      PARAMETER          ( ZERO = 0.0E0, HALF = 0.5E0, ONE = 1.0E0)
      INTEGER            NSWEEP
      PARAMETER          ( NSWEEP = 30 )
*     ..
*     .. Local Scalars ..
      REAL               AAPP, AAPP0, AAPQ, AAQQ, APOAQ, AQOAP, BIG,
     $                   BIGTHETA, CS, CTOL, EPSLN, LARGE, MXAAPQ,
     $                   MXSINJ, ROOTBIG, ROOTEPS, ROOTSFMIN, ROOTTOL,
     $                   SKL, SFMIN, SMALL, SN, T, TEMP1, THETA,
     $                   THSIGN, TOL
      INTEGER            BLSKIP, EMPTSW, i, ibr, IERR, igl, IJBLSK, ir1,
     $                   ISWROT, jbc, jgl, KBL, LKAHEAD, MVL, N2, N34,
     $                   N4, NBL, NOTROT, p, PSKIPPED, q, ROWSKIP,
     $                   SWBAND
      LOGICAL            APPLV, GOSCALE, LOWER, LSVEC, NOSCALE, ROTOK,
     $                   RSVEC, UCTOL, UPPER
*     ..
*     .. Local Arrays ..
      REAL               FASTR( 5 )
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, FLOAT, SIGN, SQRT
*     ..
*     .. External Functions ..
*     ..
*     from BLAS
      REAL               SDOT, SNRM2
      EXTERNAL           SDOT, SNRM2
      INTEGER            ISAMAX
      EXTERNAL           ISAMAX
*     from LAPACK
```

```
      REAL               SLAMCH
      EXTERNAL           SLAMCH
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
*     ..
*     from BLAS
      EXTERNAL           SAXPY, SCOPY, SROTM, SSCAL, SSWAP
*     from LAPACK
      EXTERNAL           SLASCL, SLASET, SLASSQ, XERBLA
*
      EXTERNAL           SGSVJ0, SGSVJ1
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      LSVEC = LSAME( JOBU, 'U' )
      UCTOL = LSAME( JOBU, 'C' )
      RSVEC = LSAME( JOBV, 'V' )
      APPLV = LSAME( JOBV, 'A' )
      UPPER = LSAME( JOBA, 'U' )
      LOWER = LSAME( JOBA, 'L' )
*
      IF( .NOT.( UPPER .OR. LOWER .OR. LSAME( JOBA, 'G' ) ) ) THEN
         INFO = -1
      ELSE IF( .NOT.( LSVEC .OR. UCTOL .OR. LSAME( JOBU, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( .NOT.( RSVEC .OR. APPLV .OR. LSAME( JOBV, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( M.LT.0 ) THEN
         INFO = -4
      ELSE IF( ( N.LT.0 ) .OR. ( N.GT.M ) ) THEN
         INFO = -5
      ELSE IF( LDA.LT.M ) THEN
         INFO = -7
      ELSE IF( MV.LT.0 ) THEN
         INFO = -9
      ELSE IF( ( RSVEC .AND. ( LDV.LT.N ) ) .OR.
     $         ( APPLV .AND. ( LDV.LT.MV ) ) ) THEN
         INFO = -11
      ELSE IF( UCTOL .AND. ( WORK( 1 ).LE.ONE ) ) THEN
         INFO = -12
      ELSE IF( LWORK.LT.MAX( M+N, 6 ) ) THEN
         INFO = -13
      ELSE
         INFO = 0
      END IF
*
```

```
*     #:(
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGESVJ', -INFO )
         RETURN
      END IF
*
* #:) Quick return for void matrix
*
      IF( ( M.EQ.0 ) .OR. ( N.EQ.0 ) )RETURN
*
*     Set numerical parameters
*     The stopping criterion for Jacobi rotations is
*
*     max_{i<>j}|A(:,i)^T * A(:,j)|/(||A(:,i)||*||A(:,j)||) < CTOL*EPS
*
*     where EPS is the round-off and CTOL is defined as follows:
*
      IF( UCTOL ) THEN
*        ... user controlled
         CTOL = WORK( 1 )
      ELSE
*        ... default
         IF( LSVEC .OR. RSVEC .OR. APPLV ) THEN
            CTOL = SQRT( FLOAT( M ) )
         ELSE
            CTOL = FLOAT( M )
         END IF
      END IF
*     ... and the machine dependent parameters are
*[!]  (Make sure that SLAMCH() works properly on the target machine.)
*
      EPSLN = SLAMCH( 'Epsilon' )
      ROOTEPS = SQRT( EPSLN )
      SFMIN = SLAMCH( 'SafeMinimum' )
      ROOTSFMIN = SQRT( SFMIN )
      SMALL = SFMIN / EPSLN
      BIG = SLAMCH( 'Overflow' )
*     BIG         = ONE    / SFMIN
      ROOTBIG = ONE / ROOTSFMIN
      LARGE = BIG / SQRT( FLOAT( M*N ) )
      BIGTHETA = ONE / ROOTEPS
*
      TOL = CTOL*EPSLN
      ROOTTOL = SQRT( TOL )
*
      IF( FLOAT( M )*EPSLN.GE.ONE ) THEN
         INFO = -4
         CALL XERBLA( 'SGESVJ', -INFO )
         RETURN
      END IF
```

```
*
*     Initialize the right singular vector matrix.
*
      IF( RSVEC ) THEN
         MVL = N
         CALL SLASET( 'A', MVL, N, ZERO, ONE, V, LDV )
      ELSE IF( APPLV ) THEN
         MVL = MV
      END IF
      RSVEC = RSVEC .OR. APPLV
*
*     Initialize SVA( 1:N ) = ( ||A e_i||_2, i = 1:N )
*(!)  If necessary, scale A to protect the largest singular value
*     from overflow. It is possible that saving the largest singular
*     value destroys the information about the small ones.
*     This initial scaling is almost minimal in the sense that the
*     goal is to make sure that no column norm overflows, and that
*     SQRT(N)*max_i SVA(i) does not overflow. If INFinite entries
*     in A are detected, the procedure returns with INFO=-6.
*
      SKL = ONE / SQRT( FLOAT( M )*FLOAT( N ) )
      NOSCALE = .TRUE.
      GOSCALE = .TRUE.
*
      IF( LOWER ) THEN
*        the input matrix is M-by-N lower triangular (trapezoidal)
         DO 1874 p = 1, N
            AAPP = ZERO
            AAQQ = ONE
            CALL SLASSQ( M-p+1, A( p, p ), 1, AAPP, AAQQ )
            IF( AAPP.GT.BIG ) THEN
               INFO = -6
               CALL XERBLA( 'SGESVJ', -INFO )
               RETURN
            END IF
            AAQQ = SQRT( AAQQ )
            IF( ( AAPP.LT.( BIG / AAQQ ) ) .AND. NOSCALE ) THEN
               SVA( p ) = AAPP*AAQQ
            ELSE
               NOSCALE = .FALSE.
               SVA( p ) = AAPP*( AAQQ*SKL )
               IF( GOSCALE ) THEN
                  GOSCALE = .FALSE.
                  DO 1873 q = 1, p - 1
                     SVA( q ) = SVA( q )*SKL
 1873             CONTINUE
               END IF
            END IF
 1874    CONTINUE
      ELSE IF( UPPER ) THEN
```

```
*        the input matrix is M-by-N upper triangular (trapezoidal)
         DO 2874 p = 1, N
            AAPP = ZERO
            AAQQ = ONE
            CALL SLASSQ( p, A( 1, p ), 1, AAPP, AAQQ )
            IF( AAPP.GT.BIG ) THEN
               INFO = -6
               CALL XERBLA( 'SGESVJ', -INFO )
               RETURN
            END IF
            AAQQ = SQRT( AAQQ )
            IF( ( AAPP.LT.( BIG / AAQQ ) ) .AND. NOSCALE ) THEN
               SVA( p ) = AAPP*AAQQ
            ELSE
               NOSCALE = .FALSE.
               SVA( p ) = AAPP*( AAQQ*SKL )
               IF( GOSCALE ) THEN
                  GOSCALE = .FALSE.
                  DO 2873 q = 1, p - 1
                     SVA( q ) = SVA( q )*SKL
 2873             CONTINUE
               END IF
            END IF
 2874    CONTINUE
      ELSE
*        the input matrix is M-by-N general dense
         DO 3874 p = 1, N
            AAPP = ZERO
            AAQQ = ONE
            CALL SLASSQ( M, A( 1, p ), 1, AAPP, AAQQ )
            IF( AAPP.GT.BIG ) THEN
               INFO = -6
               CALL XERBLA( 'SGESVJ', -INFO )
               RETURN
            END IF
            AAQQ = SQRT( AAQQ )
            IF( ( AAPP.LT.( BIG / AAQQ ) ) .AND. NOSCALE ) THEN
               SVA( p ) = AAPP*AAQQ
            ELSE
               NOSCALE = .FALSE.
               SVA( p ) = AAPP*( AAQQ*SKL )
               IF( GOSCALE ) THEN
                  GOSCALE = .FALSE.
                  DO 3873 q = 1, p - 1
                     SVA( q ) = SVA( q )*SKL
 3873             CONTINUE
               END IF
            END IF
 3874    CONTINUE
      END IF
```

```
*
      IF( NOSCALE )SKL = ONE
*
*     Move the smaller part of the spectrum from the underflow threshold
*(!)  Start by determining the position of the nonzero entries of the
*     array SVA() relative to ( SFMIN, BIG ).
*
      AAPP = ZERO
      AAQQ = BIG
      DO 4781 p = 1, N
         IF( SVA( p ).NE.ZERO )AAQQ = MIN( AAQQ, SVA( p ) )
         AAPP = MAX( AAPP, SVA( p ) )
 4781 CONTINUE
*
* #:) Quick return for zero matrix
*
      IF( AAPP.EQ.ZERO ) THEN
         IF( LSVEC )CALL SLASET( 'G', M, N, ZERO, ONE, A, LDA )
         WORK( 1 ) = ONE
         WORK( 2 ) = ZERO
         WORK( 3 ) = ZERO
         WORK( 4 ) = ZERO
         WORK( 5 ) = ZERO
         WORK( 6 ) = ZERO
         RETURN
      END IF
*
* #:) Quick return for one-column matrix
*
      IF( N.EQ.1 ) THEN
         IF( LSVEC )CALL SLASCL( 'G', 0, 0, SVA( 1 ), SKL, M, 1,
     $                           A( 1, 1 ), LDA, IERR )
         WORK( 1 ) = ONE / SKL
         IF( SVA( 1 ).GE.SFMIN ) THEN
            WORK( 2 ) = ONE
         ELSE
            WORK( 2 ) = ZERO
         END IF
         WORK( 3 ) = ZERO
         WORK( 4 ) = ZERO
         WORK( 5 ) = ZERO
         WORK( 6 ) = ZERO
         RETURN
      END IF
*
*     Protect small singular values from underflow, and try to
*     avoid underflows/overflows in computing Jacobi rotations.
*
      SN = SQRT( SFMIN / EPSLN )
      TEMP1 = SQRT( BIG / FLOAT( N ) )
```

```
      IF( ( AAPP.LE.SN ) .OR. ( AAQQ.GE.TEMP1 ) .OR.
     $    ( ( SN.LE.AAQQ ) .AND. ( AAPP.LE.TEMP1 ) ) ) THEN
         TEMP1 = MIN( BIG, TEMP1 / AAPP )
*         AAQQ  = AAQQ*TEMP1
*         AAPP  = AAPP*TEMP1
      ELSE IF( ( AAQQ.LE.SN ) .AND. ( AAPP.LE.TEMP1 ) ) THEN
         TEMP1 = MIN( SN / AAQQ, BIG / ( AAPP*SQRT( FLOAT( N ) ) ) )
*         AAQQ  = AAQQ*TEMP1
*         AAPP  = AAPP*TEMP1
      ELSE IF( ( AAQQ.GE.SN ) .AND. ( AAPP.GE.TEMP1 ) ) THEN
         TEMP1 = MAX( SN / AAQQ, TEMP1 / AAPP )
*         AAQQ  = AAQQ*TEMP1
*         AAPP  = AAPP*TEMP1
      ELSE IF( ( AAQQ.LE.SN ) .AND. ( AAPP.GE.TEMP1 ) ) THEN
         TEMP1 = MIN( SN / AAQQ, BIG / ( SQRT( FLOAT( N ) )*AAPP ) )
*         AAQQ  = AAQQ*TEMP1
*         AAPP  = AAPP*TEMP1
      ELSE
         TEMP1 = ONE
      END IF
*
*     Scale, if necessary
*
      IF( TEMP1.NE.ONE ) THEN
         CALL SLASCL( 'G', 0, 0, ONE, TEMP1, N, 1, SVA, N, IERR )
      END IF
      SKL = TEMP1*SKL
      IF( SKL.NE.ONE ) THEN
         CALL SLASCL( JOBA, 0, 0, ONE, SKL, M, N, A, LDA, IERR )
         SKL = ONE / SKL
      END IF
*
*     Row-cyclic Jacobi SVD algorithm with column pivoting
*
      EMPTSW = ( N*( N-1 ) ) / 2
      NOTROT = 0
      FASTR( 1 ) = ZERO
*
*     A is represented in factored form A = A * diag(WORK), where diag(WORK)
*     is initialized to identity. WORK is updated during fast scaled
*     rotations.
*
      DO 1868 q = 1, N
         WORK( q ) = ONE
 1868 CONTINUE
*
*
      SWBAND = 3
*[TP] SWBAND is a tuning parameter [TP]. It is meaningful and effective
*     if SGESVJ is used as a computational routine in the preconditioned
```

```
*     Jacobi SVD algorithm SGESVJ. For sweeps i=1:SWBAND the procedure
*     works on pivots inside a band-like region around the diagonal.
*     The boundaries are determined dynamically, based on the number of
*     pivots above a threshold.
*
      KBL = MIN( 8, N )
*[TP] KBL is a tuning parameter that defines the tile size in the
*     tiling of the p-q loops of pivot pairs. In general, an optimal
*     value of KBL depends on the matrix dimensions and on the
*     parameters of the computer's memory.
*
      NBL = N / KBL
      IF( ( NBL*KBL ).NE.N )NBL = NBL + 1
*
      BLSKIP = KBL**2
*[TP] BLKSKIP is a tuning parameter that depends on SWBAND and KBL.
*
      ROWSKIP = MIN( 5, KBL )
*[TP] ROWSKIP is a tuning parameter.
*
      LKAHEAD = 1
*[TP] LKAHEAD is a tuning parameter.
*
*     Quasi block transformations, using the lower (upper) triangular
*     structure of the input matrix. The quasi-block-cycling usually
*     invokes cubic convergence. Big part of this cycle is done inside
*     canonical subspaces of dimensions less than M.
*
      IF( ( LOWER .OR. UPPER ) .AND. ( N.GT.MAX( 64, 4*KBL ) ) ) THEN
*[TP] The number of partition levels and the actual partition are
*     tuning parameters.
         N4 = N / 4
         N2 = N / 2
         N34 = 3*N4
         IF( APPLV ) THEN
            q = 0
         ELSE
            q = 1
         END IF
*
         IF( LOWER ) THEN
*
*     This works very well on lower triangular matrices, in particular
*     in the framework of the preconditioned Jacobi SVD (xGEJSV).
*     The idea is simple:
*     [+ 0 0 0]   Note that Jacobi transformations of [0 0]
*     [+ + 0 0]                                       [0 0]
*     [+ + x 0]   actually work on [x 0]              [x 0]
*     [+ + x x]                    [x x].             [x x]
*
```

```
            CALL SGSVJ0( JOBV, M-N34, N-N34, A( N34+1, N34+1 ), LDA,
     $                   WORK( N34+1 ), SVA( N34+1 ), MVL,
     $                   V( N34*q+1, N34+1 ), LDV, EPSLN, SFMIN, TOL,
     $                   2, WORK( N+1 ), LWORK-N, IERR )
*
            CALL SGSVJ0( JOBV, M-N2, N34-N2, A( N2+1, N2+1 ), LDA,
     $                   WORK( N2+1 ), SVA( N2+1 ), MVL,
     $                   V( N2*q+1, N2+1 ), LDV, EPSLN, SFMIN, TOL, 2,
     $                   WORK( N+1 ), LWORK-N, IERR )
*
            CALL SGSVJ1( JOBV, M-N2, N-N2, N4, A( N2+1, N2+1 ), LDA,
     $                   WORK( N2+1 ), SVA( N2+1 ), MVL,
     $                   V( N2*q+1, N2+1 ), LDV, EPSLN, SFMIN, TOL, 1,
     $                   WORK( N+1 ), LWORK-N, IERR )
*
            CALL SGSVJ0( JOBV, M-N4, N2-N4, A( N4+1, N4+1 ), LDA,
     $                   WORK( N4+1 ), SVA( N4+1 ), MVL,
     $                   V( N4*q+1, N4+1 ), LDV, EPSLN, SFMIN, TOL, 1,
     $                   WORK( N+1 ), LWORK-N, IERR )
*
            CALL SGSVJ0( JOBV, M, N4, A, LDA, WORK, SVA, MVL, V, LDV,
     $                   EPSLN, SFMIN, TOL, 1, WORK( N+1 ), LWORK-N,
     $                   IERR )
*
            CALL SGSVJ1( JOBV, M, N2, N4, A, LDA, WORK, SVA, MVL, V,
     $                   LDV, EPSLN, SFMIN, TOL, 1, WORK( N+1 ),
     $                   LWORK-N, IERR )
*
*
         ELSE IF( UPPER ) THEN
*
*
            CALL SGSVJ0( JOBV, N4, N4, A, LDA, WORK, SVA, MVL, V, LDV,
     $                   EPSLN, SFMIN, TOL, 2, WORK( N+1 ), LWORK-N,
     $                   IERR )
*
            CALL SGSVJ0( JOBV, N2, N4, A( 1, N4+1 ), LDA, WORK( N4+1 ),
     $                   SVA( N4+1 ), MVL, V( N4*q+1, N4+1 ), LDV,
     $                   EPSLN, SFMIN, TOL, 1, WORK( N+1 ), LWORK-N,
     $                   IERR )
*
            CALL SGSVJ1( JOBV, N2, N2, N4, A, LDA, WORK, SVA, MVL, V,
     $                   LDV, EPSLN, SFMIN, TOL, 1, WORK( N+1 ),
     $                   LWORK-N, IERR )
*
            CALL SGSVJ0( JOBV, N2+N4, N4, A( 1, N2+1 ), LDA,
     $                   WORK( N2+1 ), SVA( N2+1 ), MVL,
     $                   V( N2*q+1, N2+1 ), LDV, EPSLN, SFMIN, TOL, 1,
     $                   WORK( N+1 ), LWORK-N, IERR )
```

```
         END IF
*
      END IF
*
*     .. Row-cyclic pivot strategy with de Rijk's pivoting ..
*
      DO 1993 i = 1, NSWEEP
*
*     .. go go go ...
*
         MXAAPQ = ZERO
         MXSINJ = ZERO
         ISWROT = 0
*
         NOTROT = 0
         PSKIPPED = 0
*
*     Each sweep is unrolled using KBL-by-KBL tiles over the pivot pairs
*     1 <= p < q <= N. This is the first step toward a blocked implementation
*     of the rotations. New implementation, based on block transformations,
*     is under development.
*
         DO 2000 ibr = 1, NBL
*
            igl = ( ibr-1 )*KBL + 1
*
            DO 1002 ir1 = 0, MIN( LKAHEAD, NBL-ibr )
*
               igl = igl + ir1*KBL
*
               DO 2001 p = igl, MIN( igl+KBL-1, N-1 )
*
*     .. de Rijk's pivoting
*
                  q = ISAMAX( N-p+1, SVA( p ), 1 ) + p - 1
                  IF( p.NE.q ) THEN
                     CALL SSWAP( M, A( 1, p ), 1, A( 1, q ), 1 )
                     IF( RSVEC )CALL SSWAP( MVL, V( 1, p ), 1,
     $                                      V( 1, q ), 1 )
                     TEMP1 = SVA( p )
                     SVA( p ) = SVA( q )
                     SVA( q ) = TEMP1
                     TEMP1 = WORK( p )
                     WORK( p ) = WORK( q )
                     WORK( q ) = TEMP1
                  END IF
*
                  IF( ir1.EQ.0 ) THEN
*
         Column norms are periodically updated by explicit
```

```
*        norm computation.
*        Caveat:
*        Unfortunately, some BLAS implementations compute SNRM2(M,A(1,p),1)
*        as SQRT(SDOT(M,A(1,p),1,A(1,p),1)), which may cause the result to
*        overflow for ||A(:,p)||_2 > SQRT(overflow_threshold), and to
*        underflow for ||A(:,p)||_2 < SQRT(underflow_threshold).
*        Hence, SNRM2 cannot be trusted, not even in the case when
*        the true norm is far from the under(over)flow boundaries.
*        If properly implemented SNRM2 is available, the IF-THEN-ELSE
*        below should read "AAPP = SNRM2( M, A(1,p), 1 ) * WORK(p)".
*
                    IF( ( SVA( p ).LT.ROOTBIG ) .AND.
     $                  ( SVA( p ).GT.ROOTSFMIN ) ) THEN
                       SVA( p ) = SNRM2( M, A( 1, p ), 1 )*WORK( p )
                    ELSE
                       TEMP1 = ZERO
                       AAPP = ONE
                       CALL SLASSQ( M, A( 1, p ), 1, TEMP1, AAPP )
                       SVA( p ) = TEMP1*SQRT( AAPP )*WORK( p )
                    END IF
                    AAPP = SVA( p )
                 ELSE
                    AAPP = SVA( p )
                 END IF
*
                 IF( AAPP.GT.ZERO ) THEN
*
                    PSKIPPED = 0
*
                    DO 2002 q = p + 1, MIN( igl+KBL-1, N )
*
                       AAQQ = SVA( q )
*
                       IF( AAQQ.GT.ZERO ) THEN
*
                          AAPP0 = AAPP
                          IF( AAQQ.GE.ONE ) THEN
                             ROTOK = ( SMALL*AAPP ).LE.AAQQ
                             IF( AAPP.LT.( BIG / AAQQ ) ) THEN
                                AAPQ = ( SDOT( M, A( 1, p ), 1, A( 1,
     $                                 q ), 1 )*WORK( p )*WORK( q ) /
     $                                 AAQQ ) / AAPP
                             ELSE
                                CALL SCOPY( M, A( 1, p ), 1,
     $                                      WORK( N+1 ), 1 )
                                CALL SLASCL( 'G', 0, 0, AAPP,
     $                                       WORK( p ), M, 1,
     $                                       WORK( N+1 ), LDA, IERR )
                                AAPQ = SDOT( M, WORK( N+1 ), 1,
     $                                 A( 1, q ), 1 )*WORK( q ) / AAQQ
```

```
                              END IF
                           ELSE
                              ROTOK = AAPP.LE.( AAQQ / SMALL )
                              IF( AAPP.GT.( SMALL / AAQQ ) ) THEN
                                 AAPQ = ( SDOT( M, A( 1, p ), 1, A( 1,
     $                                  q ), 1 )*WORK( p )*WORK( q ) /
     $                                  AAQQ ) / AAPP
                              ELSE
                                 CALL SCOPY( M, A( 1, q ), 1,
     $                                       WORK( N+1 ), 1 )
                                 CALL SLASCL( 'G', 0, 0, AAQQ,
     $                                        WORK( q ), M, 1,
     $                                        WORK( N+1 ), LDA, IERR )
                                 AAPQ = SDOT( M, WORK( N+1 ), 1,
     $                                  A( 1, p ), 1 )*WORK( p ) / AAPP
                              END IF
                           END IF
*
                           MXAAPQ = MAX( MXAAPQ, ABS( AAPQ ) )
*
*        TO rotate or NOT to rotate, THAT is the question ...
*
                           IF( ABS( AAPQ ).GT.TOL ) THEN
*
*           .. rotate
*[RTD]      ROTATED = ROTATED + ONE
*
                              IF( ir1.EQ.0 ) THEN
                                 NOTROT = 0
                                 PSKIPPED = 0
                                 ISWROT = ISWROT + 1
                              END IF
*
                              IF( ROTOK ) THEN
*
                                 AQOAP = AAQQ / AAPP
                                 APOAQ = AAPP / AAQQ
                                 THETA = -HALF*ABS( AQOAP-APOAQ ) / AAPQ
*
                                 IF( ABS( THETA ).GT.BIGTHETA ) THEN
*
                                    T = HALF / THETA
                                    FASTR( 3 ) = T*WORK( p ) / WORK( q )
                                    FASTR( 4 ) = -T*WORK( q ) /
     $                                           WORK( p )
                                    CALL SROTM( M, A( 1, p ), 1,
     $                                          A( 1, q ), 1, FASTR )
                                    IF( RSVEC )CALL SROTM( MVL,
     $                                                 V( 1, p ), 1,
     $                                                 V( 1, q ), 1,
```

```
     $                                          FASTR )
                                  SVA( q ) = AAQQ*SQRT( MAX( ZERO,
     $                                     ONE+T*APOAQ*AAPQ ) )
                                  AAPP = AAPP*SQRT( MAX( ZERO,
     $                                     ONE-T*AQOAP*AAPQ ) )
                                  MXSINJ = MAX( MXSINJ, ABS( T ) )
*
                                ELSE
*
*                 .. choose correct signum for THETA and rotate
*
                                  THSIGN = -SIGN( ONE, AAPQ )
                                  T = ONE / ( THETA+THSIGN*
     $                                SQRT( ONE+THETA*THETA ) )
                                  CS = SQRT( ONE / ( ONE+T*T ) )
                                  SN = T*CS
*
                                  MXSINJ = MAX( MXSINJ, ABS( SN ) )
                                  SVA( q ) = AAQQ*SQRT( MAX( ZERO,
     $                                     ONE+T*APOAQ*AAPQ ) )
                                  AAPP = AAPP*SQRT( MAX( ZERO,
     $                                   ONE-T*AQOAP*AAPQ ) )
*
                                  APOAQ = WORK( p ) / WORK( q )
                                  AQOAP = WORK( q ) / WORK( p )
                                  IF( WORK( p ).GE.ONE ) THEN
                                     IF( WORK( q ).GE.ONE ) THEN
                                        FASTR( 3 ) = T*APOAQ
                                        FASTR( 4 ) = -T*AQOAP
                                        WORK( p ) = WORK( p )*CS
                                        WORK( q ) = WORK( q )*CS
                                        CALL SROTM( M, A( 1, p ), 1,
     $                                              A( 1, q ), 1,
     $                                              FASTR )
                                        IF( RSVEC )CALL SROTM( MVL,
     $                                      V( 1, p ), 1, V( 1, q ),
     $                                      1, FASTR )
                                     ELSE
                                        CALL SAXPY( M, -T*AQOAP,
     $                                              A( 1, q ), 1,
     $                                              A( 1, p ), 1 )
                                        CALL SAXPY( M, CS*SN*APOAQ,
     $                                              A( 1, p ), 1,
     $                                              A( 1, q ), 1 )
                                        WORK( p ) = WORK( p )*CS
                                        WORK( q ) = WORK( q ) / CS
                                        IF( RSVEC ) THEN
                                           CALL SAXPY( MVL, -T*AQOAP,
     $                                                 V( 1, q ), 1,
     $                                                 V( 1, p ), 1 )
```

```
                                          CALL SAXPY( MVL,
     $                                                CS*SN*APOAQ,
     $                                                V( 1, p ), 1,
     $                                                V( 1, q ), 1 )
                                       END IF
                                    END IF
                                 ELSE
                                    IF( WORK( q ).GE.ONE ) THEN
                                       CALL SAXPY( M, T*APOAQ,
     $                                             A( 1, p ), 1,
     $                                             A( 1, q ), 1 )
                                       CALL SAXPY( M, -CS*SN*AQOAP,
     $                                             A( 1, q ), 1,
     $                                             A( 1, p ), 1 )
                                       WORK( p ) = WORK( p ) / CS
                                       WORK( q ) = WORK( q )*CS
                                       IF( RSVEC ) THEN
                                          CALL SAXPY( MVL, T*APOAQ,
     $                                                V( 1, p ), 1,
     $                                                V( 1, q ), 1 )
                                          CALL SAXPY( MVL,
     $                                                -CS*SN*AQOAP,
     $                                                V( 1, q ), 1,
     $                                                V( 1, p ), 1 )
                                       END IF
                                    ELSE
                                       IF( WORK( p ).GE.WORK( q ) )
     $                                  THEN
                                          CALL SAXPY( M, -T*AQOAP,
     $                                                A( 1, q ), 1,
     $                                                A( 1, p ), 1 )
                                          CALL SAXPY( M, CS*SN*APOAQ,
     $                                                A( 1, p ), 1,
     $                                                A( 1, q ), 1 )
                                          WORK( p ) = WORK( p )*CS
                                          WORK( q ) = WORK( q ) / CS
                                          IF( RSVEC ) THEN
                                             CALL SAXPY( MVL,
     $                                            -T*AQOAP,
     $                                            V( 1, q ), 1,
     $                                            V( 1, p ), 1 )
                                             CALL SAXPY( MVL,
     $                                            CS*SN*APOAQ,
     $                                            V( 1, p ), 1,
     $                                            V( 1, q ), 1 )
                                          END IF
                                       ELSE
                                          CALL SAXPY( M, T*APOAQ,
     $                                                A( 1, p ), 1,
     $                                                A( 1, q ), 1 )
```

```
                                        CALL SAXPY( M,
     $                                              -CS*SN*AQOAP,
     $                                              A( 1, q ), 1,
     $                                              A( 1, p ), 1 )
                                        WORK( p ) = WORK( p ) / CS
                                        WORK( q ) = WORK( q )*CS
                                        IF( RSVEC ) THEN
                                           CALL SAXPY( MVL,
     $                                          T*APOAQ, V( 1, p ),
     $                                          1, V( 1, q ), 1 )
                                           CALL SAXPY( MVL,
     $                                          -CS*SN*AQOAP,
     $                                          V( 1, q ), 1,
     $                                          V( 1, p ), 1 )
                                        END IF
                                     END IF
                                  END IF
                               END IF
                            END IF
*
                         ELSE
*             .. have to use modified Gram-Schmidt like transformation
                            CALL SCOPY( M, A( 1, p ), 1,
     $                                  WORK( N+1 ), 1 )
                            CALL SLASCL( 'G', 0, 0, AAPP, ONE, M,
     $                                   1, WORK( N+1 ), LDA,
     $                                   IERR )
                            CALL SLASCL( 'G', 0, 0, AAQQ, ONE, M,
     $                                   1, A( 1, q ), LDA, IERR )
                            TEMP1 = -AAPQ*WORK( p ) / WORK( q )
                            CALL SAXPY( M, TEMP1, WORK( N+1 ), 1,
     $                                  A( 1, q ), 1 )
                            CALL SLASCL( 'G', 0, 0, ONE, AAQQ, M,
     $                                   1, A( 1, q ), LDA, IERR )
                            SVA( q ) = AAQQ*SQRT( MAX( ZERO,
     $                                 ONE-AAPQ*AAPQ ) )
                            MXSINJ = MAX( MXSINJ, SFMIN )
                         END IF
*          END IF ROTOK THEN ... ELSE
*
*          In the case of cancellation in updating SVA(q), SVA(p)
*          recompute SVA(q), SVA(p).
*
                         IF( ( SVA( q ) / AAQQ )**2.LE.ROOTEPS )
     $                       THEN
                            IF( ( AAQQ.LT.ROOTBIG ) .AND.
     $                          ( AAQQ.GT.ROOTSFMIN ) ) THEN
                               SVA( q ) = SNRM2( M, A( 1, q ), 1 )*
     $                                     WORK( q )
                            ELSE
```

```
                                 T = ZERO
                                 AAQQ = ONE
                                 CALL SLASSQ( M, A( 1, q ), 1, T,
     $                                       AAQQ )
                                 SVA( q ) = T*SQRT( AAQQ )*WORK( q )
                              END IF
                           END IF
                           IF( ( AAPP / AAPP0 ).LE.ROOTEPS ) THEN
                              IF( ( AAPP.LT.ROOTBIG ) .AND.
     $                            ( AAPP.GT.ROOTSFMIN ) ) THEN
                                 AAPP = SNRM2( M, A( 1, p ), 1 )*
     $                                  WORK( p )
                              ELSE
                                 T = ZERO
                                 AAPP = ONE
                                 CALL SLASSQ( M, A( 1, p ), 1, T,
     $                                       AAPP )
                                 AAPP = T*SQRT( AAPP )*WORK( p )
                              END IF
                              SVA( p ) = AAPP
                           END IF
*
                        ELSE
*        A(:,p) and A(:,q) already numerically orthogonal
                           IF( ir1.EQ.0 )NOTROT = NOTROT + 1
*[RTD]      SKIPPED  = SKIPPED  + 1
                           PSKIPPED = PSKIPPED + 1
                        END IF
                     ELSE
*        A(:,q) is zero column
                        IF( ir1.EQ.0 )NOTROT = NOTROT + 1
                        PSKIPPED = PSKIPPED + 1
                     END IF
*
                     IF( ( i.LE.SWBAND ) .AND.
     $                   ( PSKIPPED.GT.ROWSKIP ) ) THEN
                        IF( ir1.EQ.0 )AAPP = -AAPP
                        NOTROT = 0
                        GO TO 2103
                     END IF
*
 2002                CONTINUE
*     END q-LOOP
*
 2103                CONTINUE
*     bailed out of q-loop
*
                     SVA( p ) = AAPP
*
                  ELSE
```

```
                    SVA( p ) = AAPP
                    IF( ( ir1.EQ.0 ) .AND. ( AAPP.EQ.ZERO ) )
     $                  NOTROT = NOTROT + MIN( igl+KBL-1, N ) - p
                 END IF
*
 2001          CONTINUE
*     end of the p-loop
*     end of doing the block ( ibr, ibr )
 1002       CONTINUE
*     end of ir1-loop
*
* ... go to the off diagonal blocks
*
            igl = ( ibr-1 )*KBL + 1
*
            DO 2010 jbc = ibr + 1, NBL
*
               jgl = ( jbc-1 )*KBL + 1
*
        doing the block at ( ibr, jbc )
*
               IJBLSK = 0
               DO 2100 p = igl, MIN( igl+KBL-1, N )
*
                  AAPP = SVA( p )
                  IF( AAPP.GT.ZERO ) THEN
*
                     PSKIPPED = 0
*
                     DO 2200 q = jgl, MIN( jgl+KBL-1, N )
*
                        AAQQ = SVA( q )
                        IF( AAQQ.GT.ZERO ) THEN
                           AAPP0 = AAPP
*
*     .. M x 2 Jacobi SVD ..
*
*        Safe Gram matrix computation
*
                           IF( AAQQ.GE.ONE ) THEN
                              IF( AAPP.GE.AAQQ ) THEN
                                 ROTOK = ( SMALL*AAPP ).LE.AAQQ
                              ELSE
                                 ROTOK = ( SMALL*AAQQ ).LE.AAPP
                              END IF
                              IF( AAPP.LT.( BIG / AAQQ ) ) THEN
                                 AAPQ = ( SDOT( M, A( 1, p ), 1, A( 1,
     $                                  q ), 1 )*WORK( p )*WORK( q ) /
     $                                  AAQQ ) / AAPP
                              ELSE
```

```
                              CALL SCOPY( M, A( 1, p ), 1,
     $                                    WORK( N+1 ), 1 )
                              CALL SLASCL( 'G', 0, 0, AAPP,
     $                                     WORK( p ), M, 1,
     $                                     WORK( N+1 ), LDA, IERR )
                              AAPQ = SDOT( M, WORK( N+1 ), 1,
     $                               A( 1, q ), 1 )*WORK( q ) / AAQQ
                           END IF
                        ELSE
                           IF( AAPP.GE.AAQQ ) THEN
                              ROTOK = AAPP.LE.( AAQQ / SMALL )
                           ELSE
                              ROTOK = AAQQ.LE.( AAPP / SMALL )
                           END IF
                           IF( AAPP.GT.( SMALL / AAQQ ) ) THEN
                              AAPQ = ( SDOT( M, A( 1, p ), 1, A( 1,
     $                               q ), 1 )*WORK( p )*WORK( q ) /
     $                               AAQQ ) / AAPP
                           ELSE
                              CALL SCOPY( M, A( 1, q ), 1,
     $                                    WORK( N+1 ), 1 )
                              CALL SLASCL( 'G', 0, 0, AAQQ,
     $                                     WORK( q ), M, 1,
     $                                     WORK( N+1 ), LDA, IERR )
                              AAPQ = SDOT( M, WORK( N+1 ), 1,
     $                               A( 1, p ), 1 )*WORK( p ) / AAPP
                           END IF
                        END IF
*
                        MXAAPQ = MAX( MXAAPQ, ABS( AAPQ ) )
*
*       TO rotate or NOT to rotate, THAT is the question ...
*
                        IF( ABS( AAPQ ).GT.TOL ) THEN
                           NOTROT = 0
*[RTD]      ROTATED  = ROTATED + 1
                           PSKIPPED = 0
                           ISWROT = ISWROT + 1
*
                           IF( ROTOK ) THEN
*
                              AQOAP = AAQQ / AAPP
                              APOAQ = AAPP / AAQQ
                              THETA = -HALF*ABS( AQOAP-APOAQ ) / AAPQ
                              IF( AAQQ.GT.AAPP0 )THETA = -THETA
*
                              IF( ABS( THETA ).GT.BIGTHETA ) THEN
                                 T = HALF / THETA
                                 FASTR( 3 ) = T*WORK( p ) / WORK( q )
                                 FASTR( 4 ) = -T*WORK( q ) /
```

```
     $                                        WORK( p )
                                   CALL SROTM( M, A( 1, p ), 1,
     $                                       A( 1, q ), 1, FASTR )
                                   IF( RSVEC )CALL SROTM( MVL,
     $                                            V( 1, p ), 1,
     $                                            V( 1, q ), 1,
     $                                            FASTR )
                                   SVA( q ) = AAQQ*SQRT( MAX( ZERO,
     $                                      ONE+T*APOAQ*AAPQ ) )
                                   AAPP = AAPP*SQRT( MAX( ZERO,
     $                                     ONE-T*AQOAP*AAPQ ) )
                                   MXSINJ = MAX( MXSINJ, ABS( T ) )
                                 ELSE
*
*                 .. choose correct signum for THETA and rotate
*
                                   THSIGN = -SIGN( ONE, AAPQ )
                                   IF( AAQQ.GT.AAPPO )THSIGN = -THSIGN
                                   T = ONE / ( THETA+THSIGN*
     $                                 SQRT( ONE+THETA*THETA ) )
                                   CS = SQRT( ONE / ( ONE+T*T ) )
                                   SN = T*CS
                                   MXSINJ = MAX( MXSINJ, ABS( SN ) )
                                   SVA( q ) = AAQQ*SQRT( MAX( ZERO,
     $                                      ONE+T*APOAQ*AAPQ ) )
                                   AAPP = AAPP*SQRT( MAX( ZERO,
     $                                      ONE-T*AQOAP*AAPQ ) )
*
                                   APOAQ = WORK( p ) / WORK( q )
                                   AQOAP = WORK( q ) / WORK( p )
                                   IF( WORK( p ).GE.ONE ) THEN
*
                                      IF( WORK( q ).GE.ONE ) THEN
                                         FASTR( 3 ) = T*APOAQ
                                         FASTR( 4 ) = -T*AQOAP
                                         WORK( p ) = WORK( p )*CS
                                         WORK( q ) = WORK( q )*CS
                                         CALL SROTM( M, A( 1, p ), 1,
     $                                               A( 1, q ), 1,
     $                                               FASTR )
                                         IF( RSVEC )CALL SROTM( MVL,
     $                                           V( 1, p ), 1, V( 1, q ),
     $                                           1, FASTR )
                                      ELSE
                                         CALL SAXPY( M, -T*AQOAP,
     $                                               A( 1, q ), 1,
     $                                               A( 1, p ), 1 )
                                         CALL SAXPY( M, CS*SN*APOAQ,
     $                                               A( 1, p ), 1,
     $                                               A( 1, q ), 1 )
```

```
                                    IF( RSVEC ) THEN
                                       CALL SAXPY( MVL, -T*AQOAP,
     $                                             V( 1, q ), 1,
     $                                             V( 1, p ), 1 )
                                       CALL SAXPY( MVL,
     $                                             CS*SN*APOAQ,
     $                                             V( 1, p ), 1,
     $                                             V( 1, q ), 1 )
                                    END IF
                                    WORK( p ) = WORK( p )*CS
                                    WORK( q ) = WORK( q ) / CS
                                 END IF
                              ELSE
                                 IF( WORK( q ).GE.ONE ) THEN
                                    CALL SAXPY( M, T*APOAQ,
     $                                          A( 1, p ), 1,
     $                                          A( 1, q ), 1 )
                                    CALL SAXPY( M, -CS*SN*AQOAP,
     $                                          A( 1, q ), 1,
     $                                          A( 1, p ), 1 )
                                    IF( RSVEC ) THEN
                                       CALL SAXPY( MVL, T*APOAQ,
     $                                             V( 1, p ), 1,
     $                                             V( 1, q ), 1 )
                                       CALL SAXPY( MVL,
     $                                             -CS*SN*AQOAP,
     $                                             V( 1, q ), 1,
     $                                             V( 1, p ), 1 )
                                    END IF
                                    WORK( p ) = WORK( p ) / CS
                                    WORK( q ) = WORK( q )*CS
                                 ELSE
                                    IF( WORK( p ).GE.WORK( q ) )
     $                               THEN
                                       CALL SAXPY( M, -T*AQOAP,
     $                                             A( 1, q ), 1,
     $                                             A( 1, p ), 1 )
                                       CALL SAXPY( M, CS*SN*APOAQ,
     $                                             A( 1, p ), 1,
     $                                             A( 1, q ), 1 )
                                       WORK( p ) = WORK( p )*CS
                                       WORK( q ) = WORK( q ) / CS
                                       IF( RSVEC ) THEN
                                          CALL SAXPY( MVL,
     $                                          -T*AQOAP,
     $                                          V( 1, q ), 1,
     $                                          V( 1, p ), 1 )
                                          CALL SAXPY( MVL,
     $                                          CS*SN*APOAQ,
     $                                          V( 1, p ), 1,
```

```
     $                                          V( 1, q ), 1 )
                                           END IF
                                        ELSE
                                           CALL SAXPY( M, T*APOAQ,
     $                                                 A( 1, p ), 1,
     $                                                 A( 1, q ), 1 )
                                           CALL SAXPY( M,
     $                                                 -CS*SN*AQOAP,
     $                                                 A( 1, q ), 1,
     $                                                 A( 1, p ), 1 )
                                           WORK( p ) = WORK( p ) / CS
                                           WORK( q ) = WORK( q )*CS
                                           IF( RSVEC ) THEN
                                              CALL SAXPY( MVL,
     $                                             T*APOAQ, V( 1, p ),
     $                                             1, V( 1, q ), 1 )
                                              CALL SAXPY( MVL,
     $                                             -CS*SN*AQOAP,
     $                                             V( 1, q ), 1,
     $                                             V( 1, p ), 1 )
                                           END IF
                                        END IF
                                     END IF
                                  END IF
                               END IF
*
                            ELSE
                               IF( AAPP.GT.AAQQ ) THEN
                                  CALL SCOPY( M, A( 1, p ), 1,
     $                                        WORK( N+1 ), 1 )
                                  CALL SLASCL( 'G', 0, 0, AAPP, ONE,
     $                                         M, 1, WORK( N+1 ), LDA,
     $                                         IERR )
                                  CALL SLASCL( 'G', 0, 0, AAQQ, ONE,
     $                                         M, 1, A( 1, q ), LDA,
     $                                         IERR )
                                  TEMP1 = -AAPQ*WORK( p ) / WORK( q )
                                  CALL SAXPY( M, TEMP1, WORK( N+1 ),
     $                                        1, A( 1, q ), 1 )
                                  CALL SLASCL( 'G', 0, 0, ONE, AAQQ,
     $                                         M, 1, A( 1, q ), LDA,
     $                                         IERR )
                                  SVA( q ) = AAQQ*SQRT( MAX( ZERO,
     $                                       ONE-AAPQ*AAPQ ) )
                                  MXSINJ = MAX( MXSINJ, SFMIN )
                               ELSE
                                  CALL SCOPY( M, A( 1, q ), 1,
     $                                        WORK( N+1 ), 1 )
                                  CALL SLASCL( 'G', 0, 0, AAQQ, ONE,
     $                                         M, 1, WORK( N+1 ), LDA,
```

```
     $                                          IERR )
                                   CALL SLASCL( 'G', 0, 0, AAPP, ONE,
     $                                          M, 1, A( 1, p ), LDA,
     $                                          IERR )
                                   TEMP1 = -AAPQ*WORK( q ) / WORK( p )
                                   CALL SAXPY( M, TEMP1, WORK( N+1 ),
     $                                        1, A( 1, p ), 1 )
                                   CALL SLASCL( 'G', 0, 0, ONE, AAPP,
     $                                          M, 1, A( 1, p ), LDA,
     $                                          IERR )
                                   SVA( p ) = AAPP*SQRT( MAX( ZERO,
     $                                       ONE-AAPQ*AAPQ ) )
                                   MXSINJ = MAX( MXSINJ, SFMIN )
                                END IF
                             END IF
*          END IF ROTOK THEN ... ELSE
*
*          In the case of cancellation in updating SVA(q)
*          .. recompute SVA(q)
                             IF( ( SVA( q ) / AAQQ )**2.LE.ROOTEPS )
     $                           THEN
                                IF( ( AAQQ.LT.ROOTBIG ) .AND.
     $                              ( AAQQ.GT.ROOTSFMIN ) ) THEN
                                   SVA( q ) = SNRM2( M, A( 1, q ), 1 )*
     $                                       WORK( q )
                                ELSE
                                   T = ZERO
                                   AAQQ = ONE
                                   CALL SLASSQ( M, A( 1, q ), 1, T,
     $                                          AAQQ )
                                   SVA( q ) = T*SQRT( AAQQ )*WORK( q )
                                END IF
                             END IF
                             IF( ( AAPP / AAPP0 )**2.LE.ROOTEPS ) THEN
                                IF( ( AAPP.LT.ROOTBIG ) .AND.
     $                              ( AAPP.GT.ROOTSFMIN ) ) THEN
                                   AAPP = SNRM2( M, A( 1, p ), 1 )*
     $                                    WORK( p )
                                ELSE
                                   T = ZERO
                                   AAPP = ONE
                                   CALL SLASSQ( M, A( 1, p ), 1, T,
     $                                          AAPP )
                                   AAPP = T*SQRT( AAPP )*WORK( p )
                                END IF
                                SVA( p ) = AAPP
                             END IF
*             end of OK rotation
                          ELSE
                             NOTROT = NOTROT + 1
```

```
*[RTD]      SKIPPED  = SKIPPED  + 1
                              PSKIPPED = PSKIPPED + 1
                              IJBLSK = IJBLSK + 1
                           END IF
                        ELSE
                           NOTROT = NOTROT + 1
                           PSKIPPED = PSKIPPED + 1
                           IJBLSK = IJBLSK + 1
                        END IF
*
                        IF( ( i.LE.SWBAND ) .AND. ( IJBLSK.GE.BLSKIP ) )
     $                      THEN
                           SVA( p ) = AAPP
                           NOTROT = 0
                           GO TO 2011
                        END IF
                        IF( ( i.LE.SWBAND ) .AND.
     $                      ( PSKIPPED.GT.ROWSKIP ) ) THEN
                           AAPP = -AAPP
                           NOTROT = 0
                           GO TO 2203
                        END IF
*
 2200                CONTINUE
*        end of the q-loop
 2203                CONTINUE
*
                     SVA( p ) = AAPP
*
                  ELSE
*
                     IF( AAPP.EQ.ZERO )NOTROT = NOTROT +
     $                   MIN( jgl+KBL-1, N ) - jgl + 1
                     IF( AAPP.LT.ZERO )NOTROT = 0
*
                  END IF
*
 2100          CONTINUE
*     end of the p-loop
 2010       CONTINUE
*     end of the jbc-loop
 2011       CONTINUE
*2011 bailed out of the jbc-loop
            DO 2012 p = igl, MIN( igl+KBL-1, N )
               SVA( p ) = ABS( SVA( p ) )
 2012       CONTINUE
***
 2000    CONTINUE
*2000 :: end of the ibr-loop
*
```

```
*     .. update SVA(N)
         IF( ( SVA( N ).LT.ROOTBIG ) .AND. ( SVA( N ).GT.ROOTSFMIN ) )
     $       THEN
            SVA( N ) = SNRM2( M, A( 1, N ), 1 )*WORK( N )
         ELSE
            T = ZERO
            AAPP = ONE
            CALL SLASSQ( M, A( 1, N ), 1, T, AAPP )
            SVA( N ) = T*SQRT( AAPP )*WORK( N )
         END IF
*
*     Additional steering devices
*
         IF( ( i.LT.SWBAND ) .AND. ( ( MXAAPQ.LE.ROOTTOL ) .OR.
     $       ( ISWROT.LE.N ) ) )SWBAND = i
*
         IF( ( i.GT.SWBAND+1 ) .AND. ( MXAAPQ.LT.SQRT( FLOAT( N ) )*
     $       TOL ) .AND. ( FLOAT( N )*MXAAPQ*MXSINJ.LT.TOL ) ) THEN
            GO TO 1994
         END IF
*
         IF( NOTROT.GE.EMPTSW )GO TO 1994
*
 1993 CONTINUE
*     end i=1:NSWEEP loop
*
* #:( Reaching this point means that the procedure has not converged.
      INFO = NSWEEP - 1
      GO TO 1995
*
 1994 CONTINUE
* #:) Reaching this point means numerical convergence after the i-th
*     sweep.
*
      INFO = 0
* #:) INFO = 0 confirms successful iterations.
 1995 CONTINUE
*
*     Sort the singular values and find how many are above
*     the underflow threshold.
*
      N2 = 0
      N4 = 0
      DO 5991 p = 1, N - 1
         q = ISAMAX( N-p+1, SVA( p ), 1 ) + p - 1
         IF( p.NE.q ) THEN
            TEMP1 = SVA( p )
            SVA( p ) = SVA( q )
            SVA( q ) = TEMP1
            TEMP1 = WORK( p )
```

```
            WORK( p ) = WORK( q )
            WORK( q ) = TEMP1
            CALL SSWAP( M, A( 1, p ), 1, A( 1, q ), 1 )
            IF( RSVEC )CALL SSWAP( MVL, V( 1, p ), 1, V( 1, q ), 1 )
         END IF
         IF( SVA( p ).NE.ZERO ) THEN
            N4 = N4 + 1
            IF( SVA( p )*SKL.GT.SFMIN )N2 = N2 + 1
         END IF
 5991 CONTINUE
      IF( SVA( N ).NE.ZERO ) THEN
         N4 = N4 + 1
         IF( SVA( N )*SKL.GT.SFMIN )N2 = N2 + 1
      END IF
*
*     Normalize the left singular vectors.
*
      IF( LSVEC .OR. UCTOL ) THEN
         DO 1998 p = 1, N2
            CALL SSCAL( M, WORK( p ) / SVA( p ), A( 1, p ), 1 )
 1998    CONTINUE
      END IF
*
*     Scale the product of Jacobi rotations (assemble the fast rotations).
*
      IF( RSVEC ) THEN
         IF( APPLV ) THEN
            DO 2398 p = 1, N
               CALL SSCAL( MVL, WORK( p ), V( 1, p ), 1 )
 2398       CONTINUE
         ELSE
            DO 2399 p = 1, N
               TEMP1 = ONE / SNRM2( MVL, V( 1, p ), 1 )
               CALL SSCAL( MVL, TEMP1, V( 1, p ), 1 )
 2399       CONTINUE
         END IF
      END IF
*
*     Undo scaling, if necessary (and possible).
      IF( ( ( SKL.GT.ONE ) .AND. ( SVA( 1 ).LT.( BIG / SKL ) ) )
     $    .OR. ( ( SKL.LT.ONE ) .AND. ( SVA( MAX( N2, 1 ) ) .GT.
     $    ( SFMIN / SKL ) ) ) ) THEN
         DO 2400 p = 1, N
            SVA( P ) = SKL*SVA( P )
 2400    CONTINUE
         SKL = ONE
      END IF
*
      WORK( 1 ) = SKL
*     The singular values of A are SKL*SVA(1:N). If SKL.NE.ONE
```

```
*     then some of the singular values may overflow or underflow and
*     the spectrum is given in this factored representation.
*
      WORK( 2 ) = FLOAT( N4 )
*     N4 is the number of computed nonzero singular values of A.
*
      WORK( 3 ) = FLOAT( N2 )
*     N2 is the number of singular values of A greater than SFMIN.
*     If N2<N, SVA(N2:N) contains ZEROS and/or denormalized numbers
*     that may carry some information.
*
      WORK( 4 ) = FLOAT( i )
*     i is the index of the last sweep before declaring convergence.
*
      WORK( 5 ) = MXAAPQ
*     MXAAPQ is the largest absolute value of scaled pivots in the
*     last sweep
*
      WORK( 6 ) = MXSINJ
*     MXSINJ is the largest absolute value of the sines of Jacobi angles
*     in the last sweep
*
      RETURN
*     ..
*     .. END OF SGESVJ
*     ..
      END
```

———-

33.28.4 lisp code

— **sgesvj.lisp** —

```
(let* ((zero 0.0) (half 0.5) (one 1.0) (nsweep 30))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 0.5 0.5) half)
           (type (single-float 1.0 1.0) one)
           (type (f2cl-lib:integer4 30 30) nsweep)
           (ignorable zero half one nsweep))
  (defun sgesvj (joba jobu jobv m n a lda sva mv v ldv work lwork info)
    (declare (type (array single-float (*)) work v sva a)
             (type (f2cl-lib:integer4) info lwork ldv mv lda n m)
             (type (string *) jobv jobu joba))
    (f2cl-lib:with-multi-array-data
        ((joba character joba-%data% joba-%offset%)
         (jobu character jobu-%data% jobu-%offset%)
         (jobv character jobv-%data% jobv-%offset%)
```

```
    (a single-float a-%data% a-%offset%)
    (sva single-float sva-%data% sva-%offset%)
    (v single-float v-%data% v-%offset%)
    (work single-float work-%data% work-%offset%))
  (prog ((fastr (make-array 5 :element-type 'single-float)) (applv nil)
         (goscale nil) (lower nil) (lsvec nil) (noscale nil) (rotok nil)
         (rsvec nil) (uctol nil) (upper nil) (blskip 0) (emptsw 0) (i 0)
         (ibr 0) (ierr 0) (igl 0) (ijblsk 0) (ir1 0) (iswrot 0) (jbc 0)
         (jgl 0) (kbl 0) (lkahead 0) (mvl 0) (n2 0) (n34 0) (n4 0) (nbl 0)
         (notrot 0) (p 0) (pskipped 0) (q 0) (rowskip 0) (swband 0)
         (aapp 0.0) (aapp0 0.0) (aapq 0.0) (aaqq 0.0) (apoaq 0.0)
         (aqoap 0.0) (big 0.0) (bigtheta 0.0) (cs 0.0) (ctol 0.0)
         (epsln 0.0) (large 0.0) (mxaapq 0.0) (mxsinj 0.0) (rootbig 0.0)
         (rooteps 0.0) (rootsfmin 0.0) (roottol 0.0) (skl 0.0) (sfmin 0.0)
         (small 0.0) (sn 0.0) (temp1 0.0) (theta 0.0) (thsign 0.0)
         (tol 0.0) (t$ 0.0))
    (declare (type (array single-float (5)) fastr)
             (type f2cl-lib:logical applv goscale lower lsvec noscale rotok
              rsvec uctol upper)
             (type (f2cl-lib:integer4) blskip emptsw i ibr ierr igl ijblsk
              ir1 iswrot jbc jgl kbl lkahead mvl n2 n34 n4 nbl notrot p
              pskipped q rowskip swband)
             (type (single-float) t$ aapp aapp0 aapq aaqq apoaq aqoap big
              bigtheta cs ctol epsln large mxaapq mxsinj rootbig rooteps
              rootsfmin roottol skl sfmin small sn temp1 theta thsign tol))
    (setf lsvec
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobu "U")
              (declare (ignore var-1))
              (when var-0 (setf jobu var-0))
              ret-val))
    (setf uctol
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobu "C")
              (declare (ignore var-1))
              (when var-0 (setf jobu var-0))
              ret-val))
    (setf rsvec
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobv "V")
              (declare (ignore var-1))
              (when var-0 (setf jobv var-0))
              ret-val))
    (setf applv
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobv "A")
              (declare (ignore var-1))
              (when var-0 (setf jobv var-0))
              ret-val))
    (setf upper
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame joba "U")
          (declare (ignore var-1))
          (when var-0 (setf joba var-0))
          ret-val))
(setf lower
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame joba "L")
          (declare (ignore var-1))
          (when var-0 (setf joba var-0))
          ret-val))
(cond
 ((not
   (or upper lower
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame joba "G")
         (declare (ignore var-1))
         (when var-0 (setf joba var-0))
         ret-val)))
  (setf info -1))
 ((not
   (or lsvec uctol
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobu "N")
         (declare (ignore var-1))
         (when var-0 (setf jobu var-0))
         ret-val)))
  (setf info -2))
 ((not
   (or rsvec applv
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobv "N")
         (declare (ignore var-1))
         (when var-0 (setf jobv var-0))
         ret-val)))
  (setf info -3))
 ((< m 0) (setf info -4)) ((or (< n 0) (> n m)) (setf info -5))
 ((< lda m) (setf info -7)) ((< mv 0) (setf info -9))
 ((or (and rsvec (< ldv n)) (and applv (< ldv mv))) (setf info -11))
 ((and uctol (<= (f2cl-lib:fref work (1) ((1 lwork))) one))
  (setf info -12))
 ((< lwork
     (max (the f2cl-lib:integer4 (f2cl-lib:int-add m n))
          (the f2cl-lib:integer4 6)))
  (setf info -13))
 (t (setf info 0)))
(cond
 ((/= info 0) (xerbla "SGESVJ" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= m 0) (= n 0))
```

```
    (go end_label))
(cond
 (uctol
  (setf ctol
          (f2cl-lib:fref work-%data% (1) ((1 lwork)) work-%offset%)))
 (t
  (cond
   ((or lsvec rsvec applv)
    (setf ctol (f2cl-lib:fsqrt (f2cl-lib:ffloat m))))
   (t (setf ctol (f2cl-lib:ffloat m))))))
(setf epsln (slamch "Epsilon"))
(setf rooteps (f2cl-lib:fsqrt epsln))
(setf sfmin (slamch "SafeMinimum"))
(setf rootsfmin (f2cl-lib:fsqrt sfmin))
(setf small (/ sfmin epsln))
(setf big (slamch "Overflow"))
(setf rootbig (/ one rootsfmin))
(setf large
        (/ big
           (f2cl-lib:fsqrt (f2cl-lib:ffloat (f2cl-lib:int-mul m n)))))
(setf bigtheta (/ one rooteps))
(setf tol (* ctol epsln))
(setf roottol (f2cl-lib:fsqrt tol))
(cond
 ((>= (* (f2cl-lib:ffloat m) epsln) one) (setf info -4)
  (xerbla "SGESVJ" (f2cl-lib:int-sub info)) (go end_label)))
(cond
 (rsvec (setf mvl n)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "A" mvl n zero one v ldv)
    (declare (ignore var-0 var-5))
    (when var-1 (setf mvl var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldv var-6))))
 (applv (setf mvl mv)))
(setf rsvec (or rsvec applv))
(setf skl
        (/ one
           (f2cl-lib:fsqrt
            (* (f2cl-lib:ffloat m) (f2cl-lib:ffloat n)))))
(setf noscale f2cl-lib:%true%)
(setf goscale f2cl-lib:%true%)
(cond
 (lower
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (setf aapp zero)
```

```
      (setf aaqq one)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (slassq (f2cl-lib:int-add (f2cl-lib:int-sub m p) 1)
           (f2cl-lib:array-slice a-%data% single-float (p p)
                                 ((1 lda) (1 *)) a-%offset%)
           1 aapp aaqq)
        (declare (ignore var-0 var-1 var-2))
        (when var-3 (setf aapp var-3))
        (when var-4 (setf aaqq var-4)))
      (cond
       ((> aapp big) (setf info -6)
        (xerbla "SGESVJ" (f2cl-lib:int-sub info)) (go end_label)))
      (setf aaqq (f2cl-lib:fsqrt aaqq))
      (cond
       ((and (< aapp (f2cl-lib:f2cl/ big aaqq)) noscale)
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                (* aapp aaqq)))
       (t (setf noscale f2cl-lib:%false%)
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                (* aapp (* aaqq skl)))
        (cond
         (goscale (setf goscale f2cl-lib:%false%)
          (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                        ((> q
                            (f2cl-lib:int-add p (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%)
                      (*
                       (f2cl-lib:fref sva-%data% (q) ((1 n))
                                      sva-%offset%)
                       skl))
             label1873))))))
    label1874)))
(upper
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p n) nil)
   (tagbody
     (setf aapp zero)
     (setf aaqq one)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (slassq p
          (f2cl-lib:array-slice a-%data% single-float (1 p)
                                ((1 lda) (1 *)) a-%offset%)
          1 aapp aaqq)
       (declare (ignore var-1 var-2))
       (when var-0 (setf p var-0))
       (when var-3 (setf aapp var-3))
       (when var-4 (setf aaqq var-4)))
     (cond
```

```
        ((> aapp big) (setf info -6)
         (xerbla "SGESVJ" (f2cl-lib:int-sub info)) (go end_label)))
       (setf aaqq (f2cl-lib:fsqrt aaqq))
       (cond
        ((and (< aapp (f2cl-lib:f2cl/ big aaqq)) noscale)
         (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                 (* aapp aaqq)))
        (t (setf noscale f2cl-lib:%false%)
         (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                 (* aapp (* aaqq skl)))
         (cond
          (goscale (setf goscale f2cl-lib:%false%)
           (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                         ((> q
                             (f2cl-lib:int-add p (f2cl-lib:int-sub 1)))
                          nil)
             (tagbody
               (setf (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%)
                       (*
                        (f2cl-lib:fref sva-%data% (q) ((1 n))
                                       sva-%offset%)
                        skl))
              label2873))))))
    label2874)))
(t
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p n) nil)
   (tagbody
     (setf aapp zero)
     (setf aaqq one)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (slassq m
          (f2cl-lib:array-slice a-%data% single-float (1 p)
                                ((1 lda) (1 *)) a-%offset%)
          1 aapp aaqq)
       (declare (ignore var-1 var-2))
       (when var-0 (setf m var-0))
       (when var-3 (setf aapp var-3))
       (when var-4 (setf aaqq var-4)))
     (cond
      ((> aapp big) (setf info -6)
       (xerbla "SGESVJ" (f2cl-lib:int-sub info)) (go end_label)))
     (setf aaqq (f2cl-lib:fsqrt aaqq))
     (cond
      ((and (< aapp (f2cl-lib:f2cl/ big aaqq)) noscale)
       (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
               (* aapp aaqq)))
      (t (setf noscale f2cl-lib:%false%)
       (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
               (* aapp (* aaqq skl)))
```

```
        (cond
         (goscale (setf goscale f2cl-lib:%false%)
          (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                        ((> q
                            (f2cl-lib:int-add p (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%)
                      (*
                       (f2cl-lib:fref sva-%data% (q) ((1 n))
                                      sva-%offset%)
                       skl))
             label3873))))))
     label3874))))
(if noscale
    (setf skl one))
(setf aapp zero)
(setf aaqq big)
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (if (/= (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%) zero)
        (setf aaqq
                (min aaqq
                     (f2cl-lib:fref sva-%data% (p) ((1 n))
                                    sva-%offset%))))
    (setf aapp
            (max aapp
                 (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)))
   label4781))
(cond
 ((= aapp zero)
  (if lsvec
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (slaset "G" m n zero one a lda)
        (declare (ignore var-0 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf zero var-3))
        (when var-4 (setf one var-4))
        (when var-6 (setf lda var-6))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 lwork)) work-%offset%) one)
  (setf (f2cl-lib:fref work-%data% (2) ((1 lwork)) work-%offset%) zero)
  (setf (f2cl-lib:fref work-%data% (3) ((1 lwork)) work-%offset%) zero)
  (setf (f2cl-lib:fref work-%data% (4) ((1 lwork)) work-%offset%) zero)
  (setf (f2cl-lib:fref work-%data% (5) ((1 lwork)) work-%offset%) zero)
  (setf (f2cl-lib:fref work-%data% (6) ((1 lwork)) work-%offset%) zero)
  (go end_label)))
(cond
 ((= n 1)
```

```
  (if lsvec
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (slascl "G" 0 0
           (f2cl-lib:array-slice sva-%data% single-float (1) ((1 n))
                                 sva-%offset%)
           skl m 1
           (f2cl-lib:array-slice a-%data% single-float (1 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda ierr)
        (declare (ignore var-0 var-1 var-2 var-3 var-6 var-7))
        (when var-4 (setf skl var-4))
        (when var-5 (setf m var-5))
        (when var-8 (setf lda var-8))
        (when var-9 (setf ierr var-9))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 lwork)) work-%offset%)
          (/ one skl))
  (cond
   ((>= (f2cl-lib:fref sva (1) ((1 n))) sfmin)
    (setf (f2cl-lib:fref work-%data% (2) ((1 lwork)) work-%offset%)
            one))
   (t
    (setf (f2cl-lib:fref work-%data% (2) ((1 lwork)) work-%offset%)
            zero)))
  (setf (f2cl-lib:fref work-%data% (3) ((1 lwork)) work-%offset%) zero)
  (setf (f2cl-lib:fref work-%data% (4) ((1 lwork)) work-%offset%) zero)
  (setf (f2cl-lib:fref work-%data% (5) ((1 lwork)) work-%offset%) zero)
  (setf (f2cl-lib:fref work-%data% (6) ((1 lwork)) work-%offset%) zero)
  (go end_label)))
(setf sn (f2cl-lib:fsqrt (/ sfmin epsln)))
(setf temp1 (f2cl-lib:fsqrt (/ big (f2cl-lib:ffloat n))))
(cond
 ((or (<= aapp sn) (>= aaqq temp1) (and (<= sn aaqq) (<= aapp temp1)))
  (setf temp1 (min big (/ temp1 aapp))))
 ((and (<= aaqq sn) (<= aapp temp1))
  (setf temp1
          (min (/ sn aaqq)
               (/ big (* aapp (f2cl-lib:fsqrt (f2cl-lib:ffloat n)))))))
 ((and (>= aaqq sn) (>= aapp temp1))
  (setf temp1 (max (/ sn aaqq) (/ temp1 aapp))))
 ((and (<= aaqq sn) (>= aapp temp1))
  (setf temp1
          (min (/ sn aaqq)
               (/ big (* (f2cl-lib:fsqrt (f2cl-lib:ffloat n)) aapp)))))
 (t (setf temp1 one)))
(cond
 ((/= temp1 one)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 one temp1 n 1 sva n ierr)
```

```
    (declare (ignore var-0 var-1 var-2 var-6 var-7))
    (when var-3 (setf one var-3))
    (when var-4 (setf temp1 var-4))
    (when var-5 (setf n var-5))
    (when var-8 (setf n var-8))
    (when var-9 (setf ierr var-9)))))
(setf skl (* temp1 skl))
(cond
 ((/= skl one)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl joba 0 0 one skl m n a lda ierr)
    (declare (ignore var-1 var-2 var-7))
    (when var-0 (setf joba var-0))
    (when var-3 (setf one var-3))
    (when var-4 (setf skl var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf ierr var-9)))
  (setf skl (/ one skl))))
(setf emptsw (the f2cl-lib:integer4 (truncate (* n (- n 1)) 2)))
(setf notrot 0)
(setf (f2cl-lib:fref fastr (1) ((1 5))) zero)
(f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
              ((> q n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (q) ((1 lwork)) work-%offset%)
            one)
   label1868))
(setf swband 3)
(setf kbl (min (the f2cl-lib:integer4 8) (the f2cl-lib:integer4 n)))
(setf nbl (the f2cl-lib:integer4 (truncate n kbl)))
(if (/= (f2cl-lib:int-mul nbl kbl) n)
    (setf nbl (f2cl-lib:int-add nbl 1)))
(setf blskip (expt kbl 2))
(setf rowskip
        (min (the f2cl-lib:integer4 5) (the f2cl-lib:integer4 kbl)))
(setf lkahead 1)
(cond
 ((and (or lower upper)
       (> n
          (max (the f2cl-lib:integer4 64)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul 4 kbl)))))
  (setf n4 (the f2cl-lib:integer4 (truncate n 4)))
  (setf n2 (the f2cl-lib:integer4 (truncate n 2)))
  (setf n34 (f2cl-lib:int-mul 3 n4))
  (cond (applv (setf q 0)) (t (setf q 1)))
  (cond
   (lower
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (sgsvj0 jobv (f2cl-lib:int-sub m n34) (f2cl-lib:int-sub n n34)
     (f2cl-lib:array-slice a-%data% single-float
                           ((+ n34 1) (f2cl-lib:int-add n34 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% single-float ((+ n34 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:array-slice sva-%data% single-float ((+ n34 1))
                           ((1 n)) sva-%offset%)
     mvl
     (f2cl-lib:array-slice v-%data% single-float
                           ((+ (f2cl-lib:int-mul n34 q) 1)
                            (f2cl-lib:int-add n34 1))
                           ((1 ldv) (1 *)) v-%offset%)
     ldv epsln sfmin tol 2
     (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare
   (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-13 var-14
    var-15))
  (when var-0 (setf jobv var-0))
  (when var-4 (setf lda var-4))
  (when var-7 (setf mvl var-7))
  (when var-9 (setf ldv var-9))
  (when var-10 (setf epsln var-10))
  (when var-11 (setf sfmin var-11))
  (when var-12 (setf tol var-12))
  (when var-16 (setf ierr var-16)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (sgsvj0 jobv (f2cl-lib:int-sub m n2) (f2cl-lib:int-sub n34 n2)
     (f2cl-lib:array-slice a-%data% single-float
                           ((+ n2 1) (f2cl-lib:int-add n2 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% single-float ((+ n2 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:array-slice sva-%data% single-float ((+ n2 1))
                           ((1 n)) sva-%offset%)
     mvl
     (f2cl-lib:array-slice v-%data% single-float
                           ((+ (f2cl-lib:int-mul n2 q) 1)
                            (f2cl-lib:int-add n2 1))
                           ((1 ldv) (1 *)) v-%offset%)
     ldv epsln sfmin tol 2
```

```
      (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                            ((1 lwork)) work-%offset%)
      (f2cl-lib:int-sub lwork n) ierr)
  (declare
   (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-13 var-14
    var-15))
  (when var-0 (setf jobv var-0))
  (when var-4 (setf lda var-4))
  (when var-7 (setf mvl var-7))
  (when var-9 (setf ldv var-9))
  (when var-10 (setf epsln var-10))
  (when var-11 (setf sfmin var-11))
  (when var-12 (setf tol var-12))
  (when var-16 (setf ierr var-16)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17)
    (sgsvj1 jobv (f2cl-lib:int-sub m n2) (f2cl-lib:int-sub n n2) n4
     (f2cl-lib:array-slice a-%data% single-float
                           ((+ n2 1) (f2cl-lib:int-add n2 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% single-float ((+ n2 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:array-slice sva-%data% single-float ((+ n2 1))
                           ((1 n)) sva-%offset%)
     mvl
     (f2cl-lib:array-slice v-%data% single-float
                           ((+ (f2cl-lib:int-mul n2 q) 1)
                            (f2cl-lib:int-add n2 1))
                           ((1 ldv) (1 *)) v-%offset%)
     ldv epsln sfmin tol 1
     (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare
   (ignore var-1 var-2 var-4 var-6 var-7 var-9 var-14 var-15
    var-16))
  (when var-0 (setf jobv var-0))
  (when var-3 (setf n4 var-3))
  (when var-5 (setf lda var-5))
  (when var-8 (setf mvl var-8))
  (when var-10 (setf ldv var-10))
  (when var-11 (setf epsln var-11))
  (when var-12 (setf sfmin var-12))
  (when var-13 (setf tol var-13))
  (when var-17 (setf ierr var-17)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
```

```
    (sgsvj0 jobv (f2cl-lib:int-sub m n4) (f2cl-lib:int-sub n2 n4)
     (f2cl-lib:array-slice a-%data% single-float
                           ((+ n4 1) (f2cl-lib:int-add n4 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% single-float ((+ n4 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:array-slice sva-%data% single-float ((+ n4 1))
                           ((1 n)) sva-%offset%)
     mvl
     (f2cl-lib:array-slice v-%data% single-float
                           ((+ (f2cl-lib:int-mul n4 q) 1)
                            (f2cl-lib:int-add n4 1))
                           ((1 ldv) (1 *)) v-%offset%)
     ldv epsln sfmin tol 1
     (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare
   (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-13 var-14
    var-15))
  (when var-0 (setf jobv var-0))
  (when var-4 (setf lda var-4))
  (when var-7 (setf mvl var-7))
  (when var-9 (setf ldv var-9))
  (when var-10 (setf epsln var-10))
  (when var-11 (setf sfmin var-11))
  (when var-12 (setf tol var-12))
  (when var-16 (setf ierr var-16)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (sgsvj0 jobv m n4 a lda work sva mvl v ldv epsln sfmin tol 1
     (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                           ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-3 var-5 var-6 var-8 var-13 var-14 var-15))
  (when var-0 (setf jobv var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n4 var-2))
  (when var-4 (setf lda var-4))
  (when var-7 (setf mvl var-7))
  (when var-9 (setf ldv var-9))
  (when var-10 (setf epsln var-10))
  (when var-11 (setf sfmin var-11))
  (when var-12 (setf tol var-12))
  (when var-16 (setf ierr var-16)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17)
```

```
      (sgsvj1 jobv m n2 n4 a lda work sva mvl v ldv epsln sfmin tol 1
       (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                             ((1 lwork)) work-%offset%)
       (f2cl-lib:int-sub lwork n) ierr)
    (declare (ignore var-4 var-6 var-7 var-9 var-14 var-15 var-16))
    (when var-0 (setf jobv var-0))
    (when var-1 (setf m var-1))
    (when var-2 (setf n2 var-2))
    (when var-3 (setf n4 var-3))
    (when var-5 (setf lda var-5))
    (when var-8 (setf mvl var-8))
    (when var-10 (setf ldv var-10))
    (when var-11 (setf epsln var-11))
    (when var-12 (setf sfmin var-12))
    (when var-13 (setf tol var-13))
    (when var-17 (setf ierr var-17))))
  (upper
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13 var-14 var-15 var-16)
       (sgsvj0 jobv n4 n4 a lda work sva mvl v ldv epsln sfmin tol 2
        (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                              ((1 lwork)) work-%offset%)
        (f2cl-lib:int-sub lwork n) ierr)
    (declare (ignore var-3 var-5 var-6 var-8 var-13 var-14 var-15))
    (when var-0 (setf jobv var-0))
    (when var-1 (setf n4 var-1))
    (when var-2 (setf n4 var-2))
    (when var-4 (setf lda var-4))
    (when var-7 (setf mvl var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf epsln var-10))
    (when var-11 (setf sfmin var-11))
    (when var-12 (setf tol var-12))
    (when var-16 (setf ierr var-16)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13 var-14 var-15 var-16)
       (sgsvj0 jobv n2 n4
        (f2cl-lib:array-slice a-%data% single-float
                              (1 (f2cl-lib:int-add n4 1))
                              ((1 lda) (1 *)) a-%offset%)
        lda
        (f2cl-lib:array-slice work-%data% single-float ((+ n4 1))
                              ((1 lwork)) work-%offset%)
        (f2cl-lib:array-slice sva-%data% single-float ((+ n4 1))
                              ((1 n)) sva-%offset%)
        mvl
        (f2cl-lib:array-slice v-%data% single-float
                              ((+ (f2cl-lib:int-mul n4 q) 1)
```

```
                              (f2cl-lib:int-add n4 1))
                             ((1 ldv) (1 *)) v-%offset%)
      ldv epsln sfmin tol 1
      (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                             ((1 lwork)) work-%offset%)
      (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-3 var-5 var-6 var-8 var-13 var-14 var-15))
  (when var-0 (setf jobv var-0))
  (when var-1 (setf n2 var-1))
  (when var-2 (setf n4 var-2))
  (when var-4 (setf lda var-4))
  (when var-7 (setf mvl var-7))
  (when var-9 (setf ldv var-9))
  (when var-10 (setf epsln var-10))
  (when var-11 (setf sfmin var-11))
  (when var-12 (setf tol var-12))
  (when var-16 (setf ierr var-16)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17)
    (sgsvj1 jobv n2 n2 n4 a lda work sva mvl v ldv epsln sfmin tol
     1
     (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                             ((1 lwork)) work-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-4 var-6 var-7 var-9 var-14 var-15 var-16))
  (when var-0 (setf jobv var-0))
  (when var-1 (setf n2 var-1))
  (when var-2 (setf n2 var-2))
  (when var-3 (setf n4 var-3))
  (when var-5 (setf lda var-5))
  (when var-8 (setf mvl var-8))
  (when var-10 (setf ldv var-10))
  (when var-11 (setf epsln var-11))
  (when var-12 (setf sfmin var-12))
  (when var-13 (setf tol var-13))
  (when var-17 (setf ierr var-17)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (sgsvj0 jobv (f2cl-lib:int-add n2 n4) n4
     (f2cl-lib:array-slice a-%data% single-float
                             (1 (f2cl-lib:int-add n2 1))
                             ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% single-float ((+ n2 1))
                             ((1 lwork)) work-%offset%)
     (f2cl-lib:array-slice sva-%data% single-float ((+ n2 1))
                             ((1 n)) sva-%offset%)
     mvl
```

```
          (f2cl-lib:array-slice v-%data% single-float
                                ((+ (f2cl-lib:int-mul n2 q) 1)
                                 (f2cl-lib:int-add n2 1))
                                ((1 ldv) (1 *)) v-%offset%)
          ldv epsln sfmin tol 1
          (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                                ((1 lwork)) work-%offset%)
          (f2cl-lib:int-sub lwork n) ierr)
        (declare
         (ignore var-1 var-3 var-5 var-6 var-8 var-13 var-14 var-15))
        (when var-0 (setf jobv var-0))
        (when var-2 (setf n4 var-2))
        (when var-4 (setf lda var-4))
        (when var-7 (setf mvl var-7))
        (when var-9 (setf ldv var-9))
        (when var-10 (setf epsln var-10))
        (when var-11 (setf sfmin var-11))
        (when var-12 (setf tol var-12))
        (when var-16 (setf ierr var-16)))))))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i nsweep) nil)
    (tagbody
      (setf mxaapq zero)
      (setf mxsinj zero)
      (setf iswrot 0)
      (setf notrot 0)
      (setf pskipped 0)
      (f2cl-lib:fdo (ibr 1 (f2cl-lib:int-add ibr 1))
                    ((> ibr nbl) nil)
        (tagbody
          (setf igl
                  (f2cl-lib:int-add
                   (f2cl-lib:int-mul (f2cl-lib:int-sub ibr 1) kbl) 1))
          (f2cl-lib:fdo (ir1 0 (f2cl-lib:int-add ir1 1))
                        ((> ir1
                            (min (the f2cl-lib:integer4 lkahead)
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add nbl
                                                        (f2cl-lib:int-sub
                                                         ibr)))))
                         nil)
            (tagbody
              (setf igl
                      (f2cl-lib:int-add igl (f2cl-lib:int-mul ir1 kbl)))
              (f2cl-lib:fdo (p igl (f2cl-lib:int-add p 1))
                            ((> p
                                (min
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add igl kbl
                                                        (f2cl-lib:int-sub
```

```
                                                 1)))
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add n
                                               (f2cl-lib:int-sub
                                                1)))))
                  nil)
      (tagbody
        (setf q
                (f2cl-lib:int-sub
                 (f2cl-lib:int-add
                  (isamax
                   (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
                   (f2cl-lib:array-slice sva-%data%
                                         single-float (p)
                                         ((1 n)) sva-%offset%)
                   1)
                  p)
                 1))
        (cond
         ((/= p q)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (sswap m
               (f2cl-lib:array-slice a-%data% single-float
                                     (1 p) ((1 lda) (1 *))
                                     a-%offset%)
               1
               (f2cl-lib:array-slice a-%data% single-float
                                     (1 q) ((1 lda) (1 *))
                                     a-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf m var-0)))
          (if rsvec
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4)
                  (sswap mvl
                   (f2cl-lib:array-slice v-%data% single-float
                                         (1 p) ((1 ldv) (1 *))
                                         v-%offset%)
                   1
                   (f2cl-lib:array-slice v-%data% single-float
                                         (1 q) ((1 ldv) (1 *))
                                         v-%offset%)
                   1)
                (declare (ignore var-1 var-2 var-3 var-4))
                (when var-0 (setf mvl var-0))))
          (setf temp1
                  (f2cl-lib:fref sva-%data% (p) ((1 n))
                                 sva-%offset%))
          (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
```

```
                        sva-%offset%)
                (f2cl-lib:fref sva-%data% (q) ((1 n))
                                 sva-%offset%))
        (setf (f2cl-lib:fref sva-%data% (q) ((1 n))
                        sva-%offset%)
                temp1)
        (setf temp1
                (f2cl-lib:fref work-%data% (p) ((1 lwork))
                                 work-%offset%))
        (setf (f2cl-lib:fref work-%data% (p) ((1 lwork))
                        work-%offset%)
                (f2cl-lib:fref work-%data% (q) ((1 lwork))
                                 work-%offset%))
        (setf (f2cl-lib:fref work-%data% (q) ((1 lwork))
                        work-%offset%)
                temp1)))
      (cond
       ((= ir1 0)
        (cond
         ((and (< (f2cl-lib:fref sva (p) ((1 n))) rootbig)
               (> (f2cl-lib:fref sva (p) ((1 n))) rootsfmin))
          (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                                 sva-%offset%)
                  (*
                   (multiple-value-bind
                       (ret-val var-0 var-1 var-2)
                       (snrm2 m
                        (f2cl-lib:array-slice a-%data%
                                              single-float
                                              (1 p)
                                              ((1 lda) (1 *))
                                              a-%offset%)
                        1)
                     (declare (ignore var-1 var-2))
                     (when var-0 (setf m var-0))
                     ret-val)
                   (f2cl-lib:fref work-%data% (p) ((1 lwork))
                                  work-%offset%))))
         (t (setf temp1 zero) (setf aapp one)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4)
              (slassq m
               (f2cl-lib:array-slice a-%data% single-float
                                     (1 p) ((1 lda) (1 *))
                                     a-%offset%)
               1 temp1 aapp)
            (declare (ignore var-1 var-2))
            (when var-0 (setf m var-0))
            (when var-3 (setf temp1 var-3))
            (when var-4 (setf aapp var-4)))
```

```
    (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                         sva-%offset%)
            (* temp1 (f2cl-lib:fsqrt aapp)
               (f2cl-lib:fref work-%data% (p)
                              ((1 lwork))
                              work-%offset%)))))
  (setf aapp
          (f2cl-lib:fref sva-%data% (p) ((1 n))
                         sva-%offset%)))
 (t
  (setf aapp
          (f2cl-lib:fref sva-%data% (p) ((1 n))
                         sva-%offset%))))
(cond
 ((> aapp zero)
  (tagbody
    (setf pskipped 0)
    (f2cl-lib:fdo (q (f2cl-lib:int-add p 1)
                   (f2cl-lib:int-add q 1))
                  ((> q
                      (min
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add igl kbl
                                              (f2cl-lib:int-sub
                                               1)))
                       (the f2cl-lib:integer4 n)))
                   nil)
      (tagbody
        (setf aaqq
                (f2cl-lib:fref sva-%data% (q) ((1 n))
                               sva-%offset%))
        (cond
         ((> aaqq zero) (setf aapp0 aapp)
          (cond
           ((>= aaqq one)
            (setf rotok (<= (* small aapp) aaqq))
            (cond
             ((< aapp (f2cl-lib:f2cl/ big aaqq))
              (setf aapq
                      (/
                       (/
                        (*
                         (multiple-value-bind
                             (ret-val var-0 var-1 var-2
                              var-3 var-4)
                             (sdot m
                              (f2cl-lib:array-slice
                               a-%data% single-float
                               (1 p) ((1 lda) (1 *))
                               a-%offset%)
```

```
                    1
                    (f2cl-lib:array-slice
                     a-%data% single-float
                     (1 q) ((1 lda) (1 *))
                     a-%offset%)
                    1)
                  (declare
                   (ignore var-1 var-2 var-3
                    var-4))
                  (when var-0 (setf m var-0))
                  ret-val)
                (f2cl-lib:fref work-%data% (p)
                               ((1 lwork))
                               work-%offset%)
                (f2cl-lib:fref work-%data% (q)
                               ((1 lwork))
                               work-%offset%))
            aaqq)
           aapp)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (scopy m
      (f2cl-lib:array-slice a-%data%
                            single-float
                            (1 p)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice work-%data%
                            single-float
                            ((+ n 1))
                            ((1 lwork))
                            work-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5
      var-6 var-7 var-8 var-9)
     (slascl "G" 0 0 aapp
      (f2cl-lib:array-slice work-%data%
                            single-float
                            (p)
                            ((1 lwork))
                            work-%offset%)
      m 1
      (f2cl-lib:array-slice work-%data%
```

```
                                    single-float
                                    ((+ n 1))
                                    ((1 lwork))
                                    work-%offset%)
                lda ierr)
              (declare
               (ignore var-0 var-1 var-2 var-4 var-6
                var-7))
              (when var-3 (setf aapp var-3))
              (when var-5 (setf m var-5))
              (when var-8 (setf lda var-8))
              (when var-9 (setf ierr var-9)))
            (setf aapq
                    (/
                     (*
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2
                           var-3 var-4)
                          (sdot m
                           (f2cl-lib:array-slice
                            work-%data% single-float
                            ((+ n 1)) ((1 lwork))
                            work-%offset%)
                           1
                           (f2cl-lib:array-slice
                            a-%data% single-float
                            (1 q) ((1 lda) (1 *))
                            a-%offset%)
                           1)
                        (declare
                         (ignore var-1 var-2 var-3
                          var-4))
                        (when var-0 (setf m var-0))
                        ret-val)
                      (f2cl-lib:fref work-%data% (q)
                                     ((1 lwork))
                                     work-%offset%))
                    aaqq)))))
        (t (setf rotok (<= aapp (/ aaqq small)))
         (cond
          ((> aapp (f2cl-lib:f2cl/ small aaqq))
           (setf aapq
                   (/
                    (/
                     (*
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2
                           var-3 var-4)
                          (sdot m
                           (f2cl-lib:array-slice
```

```
                    a-%data% single-float
                    (1 p) ((1 lda) (1 *))
                    a-%offset%)
                   1
                   (f2cl-lib:array-slice
                    a-%data% single-float
                    (1 q) ((1 lda) (1 *))
                    a-%offset%)
                   1)
                (declare
                 (ignore var-1 var-2 var-3
                  var-4))
                (when var-0 (setf m var-0))
                ret-val)
              (f2cl-lib:fref work-%data% (p)
                             ((1 lwork))
                             work-%offset%)
              (f2cl-lib:fref work-%data% (q)
                             ((1 lwork))
                             work-%offset%))
            aaqq)
           aapp)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (scopy m
      (f2cl-lib:array-slice a-%data%
                            single-float
                            (1 q)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice work-%data%
                            single-float
                            ((+ n 1))
                            ((1 lwork))
                            work-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5
      var-6 var-7 var-8 var-9)
     (slascl "G" 0 0 aaqq
      (f2cl-lib:array-slice work-%data%
                            single-float
                            (q)
                            ((1 lwork))
```

```
                                  work-%offset%)
            m 1
            (f2cl-lib:array-slice work-%data%
                                  single-float
                                  ((+ n 1))
                                  ((1 lwork))
                                  work-%offset%)
            lda ierr)
        (declare
         (ignore var-0 var-1 var-2 var-4 var-6
          var-7))
        (when var-3 (setf aaqq var-3))
        (when var-5 (setf m var-5))
        (when var-8 (setf lda var-8))
        (when var-9 (setf ierr var-9)))
      (setf aapq
              (/
               (*
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2
                     var-3 var-4)
                    (sdot m
                     (f2cl-lib:array-slice
                      work-%data% single-float
                      ((+ n 1)) ((1 lwork))
                      work-%offset%)
                     1
                     (f2cl-lib:array-slice
                      a-%data% single-float
                      (1 p) ((1 lda) (1 *))
                      a-%offset%)
                     1)
                  (declare
                   (ignore var-1 var-2 var-3
                    var-4))
                  (when var-0 (setf m var-0))
                  ret-val)
                (f2cl-lib:fref work-%data% (p)
                               ((1 lwork))
                               work-%offset%))
               aapp))))))
(setf mxaapq (max mxaapq (abs aapq)))
(cond
 ((> (abs aapq) tol)
  (cond
   ((= ir1 0) (setf notrot 0)
    (setf pskipped 0)
    (setf iswrot
            (f2cl-lib:int-add iswrot 1))))
  (cond
```

```
(rotok (setf aqoap (/ aaqq aapp))
 (setf apoaq (/ aapp aaqq))
 (setf theta
         (/
          (* (- half)
             (abs (- aqoap apoaq)))
          aapq))
 (cond
  ((> (abs theta) bigtheta)
   (setf t$ (/ half theta))
   (setf (f2cl-lib:fref fastr (3) ((1 5)))
           (/
            (* t$
               (f2cl-lib:fref work-%data%
                              (p)
                              ((1 lwork))
                              work-%offset%))
            (f2cl-lib:fref work-%data% (q)
                           ((1 lwork))
                           work-%offset%)))
   (setf (f2cl-lib:fref fastr (4) ((1 5)))
           (/
            (* (- t$)
               (f2cl-lib:fref work-%data%
                              (q)
                              ((1 lwork))
                              work-%offset%))
            (f2cl-lib:fref work-%data% (p)
                           ((1 lwork))
                           work-%offset%)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4
        var-5)
       (srotm m
        (f2cl-lib:array-slice a-%data%
                              single-float
                              (1 p)
                              ((1 lda)
                               (1 *))
                              a-%offset%)
        1
        (f2cl-lib:array-slice a-%data%
                              single-float
                              (1 q)
                              ((1 lda)
                               (1 *))
                              a-%offset%)
        1 fastr)
     (declare
      (ignore var-1 var-2 var-3 var-4
```

```
    var-5))
  (when var-0 (setf m var-0)))
 (if rsvec
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4
          var-5)
         (srotm mvl
          (f2cl-lib:array-slice v-%data%
                                single-float
                                (1 p)
                                ((1 ldv)
                                 (1 *))
                                v-%offset%)
          1
          (f2cl-lib:array-slice v-%data%
                                single-float
                                (1 q)
                                ((1 ldv)
                                 (1 *))
                                v-%offset%)
          1 fastr)
       (declare
        (ignore var-1 var-2 var-3 var-4
         var-5))
       (when var-0 (setf mvl var-0))))
 (setf (f2cl-lib:fref sva-%data% (q)
                      ((1 n))
                      sva-%offset%)
         (* aaqq
            (f2cl-lib:fsqrt
             (max zero
                  (+ one
                     (* t$ apoaq
                        aapq))))))
 (setf aapp
         (* aapp
            (f2cl-lib:fsqrt
             (max zero
                  (+ one
                     (* (- t$) aqoap
                        aapq))))))
 (setf mxsinj (max mxsinj (abs t))))
(t
 (setf thsign
         (- (f2cl-lib:sign one aapq)))
 (setf t$
         (/ one
            (+ theta
               (* thsign
                  (f2cl-lib:fsqrt
```

```
                    (+ one
                       (* theta
                          theta)))))))
(setf cs
        (f2cl-lib:fsqrt
         (/ one (+ one (* t$ t$)))))
(setf sn (* t$ cs))
(setf mxsinj (max mxsinj (abs sn)))
(setf (f2cl-lib:fref sva-%data% (q)
                      ((1 n))
                      sva-%offset%)
        (* aaqq
           (f2cl-lib:fsqrt
            (max zero
                 (+ one
                    (* t$ apoaq
                       aapq))))))
(setf aapp
        (* aapp
           (f2cl-lib:fsqrt
            (max zero
                 (+ one
                    (* (- t$) aqoap
                       aapq))))))
(setf apoaq
        (/
         (f2cl-lib:fref work-%data% (p)
                        ((1 lwork))
                        work-%offset%)
         (f2cl-lib:fref work-%data% (q)
                        ((1 lwork))
                        work-%offset%)))
(setf aqoap
        (/
         (f2cl-lib:fref work-%data% (q)
                        ((1 lwork))
                        work-%offset%)
         (f2cl-lib:fref work-%data% (p)
                        ((1 lwork))
                        work-%offset%)))
(cond
 ((>=
   (f2cl-lib:fref work (p) ((1 lwork)))
   one)
  (cond
   ((>=
     (f2cl-lib:fref work (q)
                    ((1 lwork)))
     one)
    (setf (f2cl-lib:fref fastr (3)
```

```
                      ((1 5)))
        (* t$ apoaq))
(setf (f2cl-lib:fref fastr (4)
                      ((1 5)))
        (* (- t$) aqoap))
(setf (f2cl-lib:fref work-%data%
                      (p)
                      ((1 lwork))
                      work-%offset%)
        (*
         (f2cl-lib:fref work-%data%
                        (p)
                        ((1 lwork))
                        work-%offset%)
         cs))
(setf (f2cl-lib:fref work-%data%
                      (q)
                      ((1 lwork))
                      work-%offset%)
        (*
         (f2cl-lib:fref work-%data%
                        (q)
                        ((1 lwork))
                        work-%offset%)
         cs))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5)
    (srotm m
     (f2cl-lib:array-slice a-%data%
                           single-float
                           (1 p)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1
     (f2cl-lib:array-slice a-%data%
                           single-float
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1 fastr)
  (declare
   (ignore var-1 var-2 var-3 var-4
    var-5))
  (when var-0 (setf m var-0)))
(if rsvec
    (multiple-value-bind
        (var-0 var-1 var-2 var-3
```

```
                var-4 var-5)
               (srotm mvl
                (f2cl-lib:array-slice
                 v-%data% single-float
                 (1 p) ((1 ldv) (1 *))
                 v-%offset%)
                1
                (f2cl-lib:array-slice
                 v-%data% single-float
                 (1 q) ((1 ldv) (1 *))
                 v-%offset%)
                1 fastr)
              (declare
               (ignore var-1 var-2 var-3
                var-4 var-5))
              (when var-0
                (setf mvl var-0)))))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4
      var-5)
     (saxpy m (* (- t$) aqoap)
      (f2cl-lib:array-slice a-%data%
                            single-float
                            (1 q)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice a-%data%
                            single-float
                            (1 p)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4
     var-5))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4
      var-5)
     (saxpy m (* cs sn apoaq)
      (f2cl-lib:array-slice a-%data%
                            single-float
                            (1 p)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
```

```
     1
     (f2cl-lib:array-slice a-%data%
                           single-float
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3 var-4
    var-5))
  (when var-0 (setf m var-0)))
(setf (f2cl-lib:fref work-%data%
                     (p)
                     ((1 lwork))
                     work-%offset%)
        (*
         (f2cl-lib:fref work-%data%
                        (p)
                        ((1 lwork))
                        work-%offset%)
         cs))
(setf (f2cl-lib:fref work-%data%
                     (q)
                     ((1 lwork))
                     work-%offset%)
        (/
         (f2cl-lib:fref work-%data%
                        (q)
                        ((1 lwork))
                        work-%offset%)
         cs))
(cond
 (rsvec
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
       var-4 var-5)
      (saxpy mvl (* (- t$) aqoap)
       (f2cl-lib:array-slice
        v-%data% single-float (1 q)
        ((1 ldv) (1 *)) v-%offset%)
       1
       (f2cl-lib:array-slice
        v-%data% single-float (1 p)
        ((1 ldv) (1 *)) v-%offset%)
       1)
    (declare
     (ignore var-1 var-2 var-3
      var-4 var-5))
    (when var-0 (setf mvl var-0)))
```

```
      (multiple-value-bind
          (var-0 var-1 var-2 var-3
           var-4 var-5)
          (saxpy mvl (* cs sn apoaq)
           (f2cl-lib:array-slice
            v-%data% single-float (1 p)
            ((1 ldv) (1 *)) v-%offset%)
           1
           (f2cl-lib:array-slice
            v-%data% single-float (1 q)
            ((1 ldv) (1 *)) v-%offset%)
           1)
        (declare
         (ignore var-1 var-2 var-3
          var-4 var-5))
        (when var-0
          (setf mvl var-0))))))))
(t
 (cond
  ((>=
    (f2cl-lib:fref work (q)
                   ((1 lwork)))
    one)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4
        var-5)
       (saxpy m (* t$ apoaq)
        (f2cl-lib:array-slice a-%data%
                              single-float
                              (1 p)
                              ((1 lda)
                               (1 *))
                              a-%offset%)
        1
        (f2cl-lib:array-slice a-%data%
                              single-float
                              (1 q)
                              ((1 lda)
                               (1 *))
                              a-%offset%)
        1)
     (declare
      (ignore var-1 var-2 var-3 var-4
       var-5))
     (when var-0 (setf m var-0)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4
        var-5)
       (saxpy m (* (- cs) sn aqoap)
        (f2cl-lib:array-slice a-%data%
```

```
                                 single-float
                                 (1 q)
                                 ((1 lda)
                                  (1 *))
                                 a-%offset%)
     1
     (f2cl-lib:array-slice a-%data%
                                 single-float
                                 (1 p)
                                 ((1 lda)
                                  (1 *))
                                 a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3 var-4
    var-5))
  (when var-0 (setf m var-0)))
(setf (f2cl-lib:fref work-%data%
                     (p)
                     ((1 lwork))
                     work-%offset%)
        (/
         (f2cl-lib:fref work-%data%
                        (p)
                        ((1 lwork))
                        work-%offset%)
         cs))
(setf (f2cl-lib:fref work-%data%
                     (q)
                     ((1 lwork))
                     work-%offset%)
        (*
         (f2cl-lib:fref work-%data%
                        (q)
                        ((1 lwork))
                        work-%offset%)
         cs))
(cond
 (rsvec
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
       var-4 var-5)
      (saxpy mvl (* t$ apoaq)
       (f2cl-lib:array-slice
        v-%data% single-float (1 p)
        ((1 ldv) (1 *)) v-%offset%)
       1
       (f2cl-lib:array-slice
        v-%data% single-float (1 q)
        ((1 ldv) (1 *)) v-%offset%)
```

```
            1)
          (declare
           (ignore var-1 var-2 var-3
            var-4 var-5))
          (when var-0 (setf mvl var-0)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3
             var-4 var-5)
            (saxpy mvl
             (* (- cs) sn aqoap)
             (f2cl-lib:array-slice
              v-%data% single-float (1 q)
              ((1 ldv) (1 *)) v-%offset%)
             1
             (f2cl-lib:array-slice
              v-%data% single-float (1 p)
              ((1 ldv) (1 *)) v-%offset%)
             1)
          (declare
           (ignore var-1 var-2 var-3
            var-4 var-5))
          (when var-0
            (setf mvl var-0))))))
    (t
     (cond
      ((>=
        (f2cl-lib:fref work (p)
                       ((1 lwork)))
        (f2cl-lib:fref work (q)
                       ((1 lwork))))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3
             var-4 var-5)
            (saxpy m (* (- t$) aqoap)
             (f2cl-lib:array-slice
              a-%data% single-float (1 q)
              ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice
              a-%data% single-float (1 p)
              ((1 lda) (1 *)) a-%offset%)
             1)
          (declare
           (ignore var-1 var-2 var-3
            var-4 var-5))
          (when var-0 (setf m var-0)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3
             var-4 var-5)
            (saxpy m (* cs sn apoaq)
```

```
     (f2cl-lib:array-slice
      a-%data% single-float (1 p)
      ((1 lda) (1 *)) a-%offset%)
     1
     (f2cl-lib:array-slice
      a-%data% single-float (1 q)
      ((1 lda) (1 *)) a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3
    var-4 var-5))
  (when var-0 (setf m var-0)))
(setf (f2cl-lib:fref work-%data%
                     (p)
                     ((1 lwork))
                     work-%offset%)
        (*
         (f2cl-lib:fref
          work-%data% (p)
          ((1 lwork))
          work-%offset%)
         cs))
(setf (f2cl-lib:fref work-%data%
                     (q)
                     ((1 lwork))
                     work-%offset%)
        (/
         (f2cl-lib:fref
          work-%data% (q)
          ((1 lwork))
          work-%offset%)
         cs))
(cond
 (rsvec
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
       var-4 var-5)
      (saxpy mvl (* (- t$) aqoap)
       (f2cl-lib:array-slice
        v-%data% single-float
        (1 q) ((1 ldv) (1 *))
        v-%offset%)
       1
       (f2cl-lib:array-slice
        v-%data% single-float
        (1 p) ((1 ldv) (1 *))
        v-%offset%)
       1)
    (declare
     (ignore var-1 var-2 var-3
```

```
        var-4 var-5))
      (when var-0
        (setf mvl var-0)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3
         var-4 var-5)
        (saxpy mvl (* cs sn apoaq)
         (f2cl-lib:array-slice
          v-%data% single-float
          (1 p) ((1 ldv) (1 *))
          v-%offset%)
         1
         (f2cl-lib:array-slice
          v-%data% single-float
          (1 q) ((1 ldv) (1 *))
          v-%offset%)
         1)
      (declare
       (ignore var-1 var-2 var-3
        var-4 var-5))
      (when var-0
        (setf mvl var-0))))))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3
      var-4 var-5)
     (saxpy m (* t$ apoaq)
      (f2cl-lib:array-slice
       a-%data% single-float (1 p)
       ((1 lda) (1 *)) a-%offset%)
      1
      (f2cl-lib:array-slice
       a-%data% single-float (1 q)
       ((1 lda) (1 *)) a-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3
     var-4 var-5))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3
      var-4 var-5)
     (saxpy m (* (- cs) sn aqoap)
      (f2cl-lib:array-slice
       a-%data% single-float (1 q)
       ((1 lda) (1 *)) a-%offset%)
      1
      (f2cl-lib:array-slice
       a-%data% single-float (1 p)
       ((1 lda) (1 *)) a-%offset%)
```

```
     1)
  (declare
   (ignore var-1 var-2 var-3
    var-4 var-5))
  (when var-0 (setf m var-0)))
(setf (f2cl-lib:fref work-%data%
                     (p)
                     ((1 lwork))
                     work-%offset%)
        (/
         (f2cl-lib:fref
          work-%data% (p)
          ((1 lwork))
          work-%offset%)
         cs))
(setf (f2cl-lib:fref work-%data%
                     (q)
                     ((1 lwork))
                     work-%offset%)
        (*
         (f2cl-lib:fref
          work-%data% (q)
          ((1 lwork))
          work-%offset%)
         cs))
(cond
 (rsvec
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
       var-4 var-5)
      (saxpy mvl (* t$ apoaq)
       (f2cl-lib:array-slice
        v-%data% single-float
        (1 p) ((1 ldv) (1 *))
        v-%offset%)
       1
       (f2cl-lib:array-slice
        v-%data% single-float
        (1 q) ((1 ldv) (1 *))
        v-%offset%)
       1)
    (declare
     (ignore var-1 var-2 var-3
      var-4 var-5))
    (when var-0
      (setf mvl var-0)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
       var-4 var-5)
      (saxpy mvl
```

```
                    (* (- cs) sn aqoap)
                    (f2cl-lib:array-slice
                     v-%data% single-float
                     (1 q) ((1 ldv) (1 *))
                     v-%offset%)
                    1
                    (f2cl-lib:array-slice
                     v-%data% single-float
                     (1 p) ((1 ldv) (1 *))
                     v-%offset%)
                    1)
                (declare
                 (ignore var-1 var-2 var-3
                  var-4 var-5))
                (when var-0
                  (setf mvl
                          var-0)))))))))))))))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (scopy m
      (f2cl-lib:array-slice a-%data%
                            single-float
                            (1 p)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice work-%data%
                            single-float
                            ((+ n 1))
                            ((1 lwork))
                            work-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5
      var-6 var-7 var-8 var-9)
     (slascl "G" 0 0 aapp one m 1
      (f2cl-lib:array-slice work-%data%
                            single-float
                            ((+ n 1))
                            ((1 lwork))
                            work-%offset%)
      lda ierr)
   (declare
    (ignore var-0 var-1 var-2 var-6
     var-7))
```

```
  (when var-3 (setf aapp var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5
     var-6 var-7 var-8 var-9)
    (slascl "G" 0 0 aaqq one m 1
     (f2cl-lib:array-slice a-%data%
                           single-float
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     lda ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6
    var-7))
  (when var-3 (setf aaqq var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(setf temp1
        (/
         (* (- aapq)
            (f2cl-lib:fref work-%data%
                           (p)
                           ((1 lwork))
                           work-%offset%))
         (f2cl-lib:fref work-%data% (q)
                        ((1 lwork))
                        work-%offset%)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5)
    (saxpy m temp1
     (f2cl-lib:array-slice work-%data%
                           single-float
                           ((+ n 1))
                           ((1 lwork))
                           work-%offset%)
     1
     (f2cl-lib:array-slice a-%data%
                           single-float
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1)
```

```
      (declare
       (ignore var-2 var-3 var-4 var-5))
      (when var-0 (setf m var-0))
      (when var-1 (setf temp1 var-1)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5
         var-6 var-7 var-8 var-9)
        (slascl "G" 0 0 one aaqq m 1
         (f2cl-lib:array-slice a-%data%
                               single-float
                               (1 q)
                               ((1 lda)
                                (1 *))
                               a-%offset%)
         lda ierr)
      (declare
       (ignore var-0 var-1 var-2 var-6
        var-7))
      (when var-3 (setf one var-3))
      (when var-4 (setf aaqq var-4))
      (when var-5 (setf m var-5))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9)))
    (setf (f2cl-lib:fref sva-%data% (q)
                         ((1 n))
                         sva-%offset%)
          (* aaqq
             (f2cl-lib:fsqrt
              (max zero
                   (- one
                      (* aapq aapq))))))
    (setf mxsinj (max mxsinj sfmin))))
  (cond
   ((<=
     (expt
      (f2cl-lib:f2cl/
       (f2cl-lib:fref sva (q) ((1 n))) aaqq)
      2)
     rooteps)
    (cond
     ((and (< aaqq rootbig)
           (> aaqq rootsfmin))
      (setf (f2cl-lib:fref sva-%data% (q)
                           ((1 n))
                           sva-%offset%)
            (*
             (multiple-value-bind
                 (ret-val var-0 var-1
                  var-2)
                 (snrm2 m
```

```
                    (f2cl-lib:array-slice
                     a-%data% single-float
                     (1 q) ((1 lda) (1 *))
                     a-%offset%)
                    1)
                  (declare
                   (ignore var-1 var-2))
                  (when var-0 (setf m var-0))
                  ret-val)
                (f2cl-lib:fref work-%data% (q)
                               ((1 lwork))
                               work-%offset%))))
   (t (setf t$ zero) (setf aaqq one)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4)
        (slassq m
         (f2cl-lib:array-slice a-%data%
                               single-float
                               (1 q)
                               ((1 lda)
                                (1 *))
                               a-%offset%)
         1 t$ aaqq)
      (declare (ignore var-1 var-2))
      (when var-0 (setf m var-0))
      (when var-3 (setf t$ var-3))
      (when var-4 (setf aaqq var-4)))
    (setf (f2cl-lib:fref sva-%data% (q)
                         ((1 n))
                         sva-%offset%)
            (* t$ (f2cl-lib:fsqrt aaqq)
               (f2cl-lib:fref work-%data%
                              (q)
                              ((1 lwork))
                              work-%offset%)))))))
(cond
 ((<= (f2cl-lib:f2cl/ aapp aapp0) rooteps)
  (cond
   ((and (< aapp rootbig)
         (> aapp rootsfmin))
    (setf aapp
            (*
             (multiple-value-bind
                 (ret-val var-0 var-1
                  var-2)
                 (snrm2 m
                  (f2cl-lib:array-slice
                   a-%data% single-float
                   (1 p) ((1 lda) (1 *))
                   a-%offset%)
```

```
                          1)
                        (declare
                         (ignore var-1 var-2))
                        (when var-0 (setf m var-0))
                        ret-val)
                      (f2cl-lib:fref work-%data% (p)
                                     ((1 lwork))
                                     work-%offset%))))
          (t (setf t$ zero) (setf aapp one)
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4)
               (slassq m
                (f2cl-lib:array-slice a-%data%
                                      single-float
                                      (1 p)
                                      ((1 lda)
                                       (1 *))
                                      a-%offset%)
                1 t$ aapp)
             (declare (ignore var-1 var-2))
             (when var-0 (setf m var-0))
             (when var-3 (setf t$ var-3))
             (when var-4 (setf aapp var-4)))
           (setf aapp
                   (* t$ (f2cl-lib:fsqrt aapp)
                      (f2cl-lib:fref work-%data%
                                     (p)
                                     ((1 lwork))
                                     work-%offset%)))))
        (setf (f2cl-lib:fref sva-%data% (p)
                             ((1 n))
                             sva-%offset%)
              aapp))))
    (t
     (if (= ir1 0)
         (setf notrot
                 (f2cl-lib:int-add notrot 1)))
     (setf pskipped
             (f2cl-lib:int-add pskipped 1)))))
  (t
   (if (= ir1 0)
       (setf notrot
               (f2cl-lib:int-add notrot 1)))
   (setf pskipped
           (f2cl-lib:int-add pskipped 1))))
 (cond
  ((and (<= i swband) (> pskipped rowskip))
   (if (= ir1 0)
       (setf aapp (- aapp)))
   (setf notrot 0) (go label2103)))
```

```
                 label2002))
             label2103
              (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                                   sva-%offset%)
                    aapp)))
          (t
           (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                                sva-%offset%)
                 aapp)
           (if (and (= ir1 0) (= aapp zero))
               (setf notrot
                       (f2cl-lib:int-sub
                        (f2cl-lib:int-add notrot
                                          (min
                                           (the
                                            f2cl-lib:integer4
                                            (f2cl-lib:int-sub
                                             (f2cl-lib:int-add
                                              igl kbl)
                                             1))
                                           (the
                                            f2cl-lib:integer4
                                            n)))
                        p)))))
       label2001))
   label1002))
(setf igl
        (f2cl-lib:int-add
         (f2cl-lib:int-mul (f2cl-lib:int-sub ibr 1) kbl) 1))
(f2cl-lib:fdo (jbc (f2cl-lib:int-add ibr 1)
               (f2cl-lib:int-add jbc 1))
              ((> jbc nbl) nil)
  (tagbody
    (setf jgl
            (f2cl-lib:int-add
             (f2cl-lib:int-mul (f2cl-lib:int-sub jbc 1) kbl)
             1))
    (setf ijblsk 0)
    (f2cl-lib:fdo (p igl (f2cl-lib:int-add p 1))
                  ((> p
                      (min
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add igl kbl
                                              (f2cl-lib:int-sub
                                               1)))
                       (the f2cl-lib:integer4 n)))
                   nil)
      (tagbody
        (setf aapp
                (f2cl-lib:fref sva-%data% (p) ((1 n))
```

```
                        sva-%offset%))
(cond
 ((> aapp zero)
  (tagbody
    (setf pskipped 0)
    (f2cl-lib:fdo (q jgl (f2cl-lib:int-add q 1))
                  ((> q
                      (min
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add jgl kbl
                                              (f2cl-lib:int-sub
                                               1)))
                       (the f2cl-lib:integer4 n)))
                   nil)
      (tagbody
        (setf aaqq
                (f2cl-lib:fref sva-%data% (q) ((1 n))
                               sva-%offset%))
        (cond
         ((> aaqq zero) (setf aapp0 aapp)
          (cond
           ((>= aaqq one)
            (cond
             ((>= aapp aaqq)
              (setf rotok (<= (* small aapp) aaqq)))
             (t (setf rotok (<= (* small aaqq) aapp))))
            (cond
             ((< aapp (f2cl-lib:f2cl/ big aaqq))
              (setf aapq
                      (/
                       (/
                        (*
                         (multiple-value-bind
                             (ret-val var-0 var-1 var-2
                              var-3 var-4)
                             (sdot m
                              (f2cl-lib:array-slice
                               a-%data% single-float
                               (1 p) ((1 lda) (1 *))
                               a-%offset%)
                              1
                              (f2cl-lib:array-slice
                               a-%data% single-float
                               (1 q) ((1 lda) (1 *))
                               a-%offset%)
                              1)
                           (declare
                            (ignore var-1 var-2 var-3
                             var-4))
                           (when var-0 (setf m var-0))
```

```
                ret-val)
              (f2cl-lib:fref work-%data% (p)
                             ((1 lwork))
                             work-%offset%)
              (f2cl-lib:fref work-%data% (q)
                             ((1 lwork))
                             work-%offset%))
            aaqq)
           aapp)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (scopy m
      (f2cl-lib:array-slice a-%data%
                            single-float
                            (1 p)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice work-%data%
                            single-float
                            ((+ n 1))
                            ((1 lwork))
                            work-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5
      var-6 var-7 var-8 var-9)
     (slascl "G" 0 0 aapp
      (f2cl-lib:array-slice work-%data%
                            single-float
                            (p)
                            ((1 lwork))
                            work-%offset%)
      m 1
      (f2cl-lib:array-slice work-%data%
                            single-float
                            ((+ n 1))
                            ((1 lwork))
                            work-%offset%)
      lda ierr)
   (declare
    (ignore var-0 var-1 var-2 var-4 var-6
     var-7))
   (when var-3 (setf aapp var-3))
   (when var-5 (setf m var-5))
```

```
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9)))
    (setf aapq
            (/
             (*
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2
                   var-3 var-4)
                  (sdot m
                   (f2cl-lib:array-slice
                    work-%data% single-float
                    ((+ n 1)) ((1 lwork))
                    work-%offset%)
                   1
                   (f2cl-lib:array-slice
                    a-%data% single-float
                    (1 q) ((1 lda) (1 *))
                    a-%offset%)
                   1)
                (declare
                 (ignore var-1 var-2 var-3
                  var-4))
                (when var-0 (setf m var-0))
                ret-val)
              (f2cl-lib:fref work-%data% (q)
                             ((1 lwork))
                             work-%offset%))
            aaqq)))))
(t
 (cond
  ((>= aapp aaqq)
   (setf rotok (<= aapp (/ aaqq small))))
  (t (setf rotok (<= aaqq (/ aapp small)))))
 (cond
  ((> aapp (f2cl-lib:f2cl/ small aaqq))
   (setf aapq
           (/
            (/
             (*
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2
                   var-3 var-4)
                  (sdot m
                   (f2cl-lib:array-slice
                    a-%data% single-float
                    (1 p) ((1 lda) (1 *))
                    a-%offset%)
                   1
                   (f2cl-lib:array-slice
                    a-%data% single-float
```

```
                    (1 q) ((1 lda) (1 *))
                    a-%offset%)
                   1)
                (declare
                 (ignore var-1 var-2 var-3
                  var-4))
                (when var-0 (setf m var-0))
                ret-val)
              (f2cl-lib:fref work-%data% (p)
                             ((1 lwork))
                             work-%offset%)
              (f2cl-lib:fref work-%data% (q)
                             ((1 lwork))
                             work-%offset%))
            aaqq)
           aapp)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (scopy m
      (f2cl-lib:array-slice a-%data%
                            single-float
                            (1 q)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice work-%data%
                            single-float
                            ((+ n 1))
                            ((1 lwork))
                            work-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5
      var-6 var-7 var-8 var-9)
     (slascl "G" 0 0 aaqq
      (f2cl-lib:array-slice work-%data%
                            single-float
                            (q)
                            ((1 lwork))
                            work-%offset%)
      m 1
      (f2cl-lib:array-slice work-%data%
                            single-float
                            ((+ n 1))
                            ((1 lwork))
```

```
                                work-%offset%)
          lda ierr)
        (declare
         (ignore var-0 var-1 var-2 var-4 var-6
          var-7))
        (when var-3 (setf aaqq var-3))
        (when var-5 (setf m var-5))
        (when var-8 (setf lda var-8))
        (when var-9 (setf ierr var-9)))
      (setf aapq
              (/
               (*
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2
                     var-3 var-4)
                    (sdot m
                     (f2cl-lib:array-slice
                      work-%data% single-float
                      ((+ n 1)) ((1 lwork))
                      work-%offset%)
                     1
                     (f2cl-lib:array-slice
                      a-%data% single-float
                      (1 p) ((1 lda) (1 *))
                      a-%offset%)
                     1)
                  (declare
                   (ignore var-1 var-2 var-3
                    var-4))
                  (when var-0 (setf m var-0))
                  ret-val)
                (f2cl-lib:fref work-%data% (p)
                               ((1 lwork))
                               work-%offset%))
               aapp))))))
(setf mxaapq (max mxaapq (abs aapq)))
(cond
 ((> (abs aapq) tol) (setf notrot 0)
  (setf pskipped 0)
  (setf iswrot (f2cl-lib:int-add iswrot 1))
  (cond
   (rotok (setf aqoap (/ aaqq aapp))
    (setf apoaq (/ aapp aaqq))
    (setf theta
            (/
             (* (- half)
                (abs (- aqoap apoaq)))
             aapq))
    (if (> aaqq aapp0)
        (setf theta (- theta)))
```

```
(cond
 ((> (abs theta) bigtheta)
  (setf t$ (/ half theta))
  (setf (f2cl-lib:fref fastr (3) ((1 5)))
          (/
           (* t$
              (f2cl-lib:fref work-%data%
                             (p)
                             ((1 lwork))
                             work-%offset%))
           (f2cl-lib:fref work-%data% (q)
                          ((1 lwork))
                          work-%offset%)))
  (setf (f2cl-lib:fref fastr (4) ((1 5)))
          (/
           (* (- t$)
              (f2cl-lib:fref work-%data%
                             (q)
                             ((1 lwork))
                             work-%offset%))
           (f2cl-lib:fref work-%data% (p)
                          ((1 lwork))
                          work-%offset%)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4
       var-5)
      (srotm m
       (f2cl-lib:array-slice a-%data%
                             single-float
                             (1 p)
                             ((1 lda)
                              (1 *))
                             a-%offset%)
       1
       (f2cl-lib:array-slice a-%data%
                             single-float
                             (1 q)
                             ((1 lda)
                              (1 *))
                             a-%offset%)
       1 fastr)
    (declare
     (ignore var-1 var-2 var-3 var-4
      var-5))
    (when var-0 (setf m var-0)))
  (if rsvec
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4
           var-5)
          (srotm mvl
```

```
          (f2cl-lib:array-slice v-%data%
                                single-float
                                (1 p)
                                ((1 ldv)
                                 (1 *))
                                v-%offset%)
          1
          (f2cl-lib:array-slice v-%data%
                                single-float
                                (1 q)
                                ((1 ldv)
                                 (1 *))
                                v-%offset%)
          1 fastr)
       (declare
        (ignore var-1 var-2 var-3 var-4
         var-5))
       (when var-0 (setf mvl var-0))))
 (setf (f2cl-lib:fref sva-%data% (q)
                      ((1 n))
                      sva-%offset%)
         (* aaqq
            (f2cl-lib:fsqrt
             (max zero
                  (+ one
                     (* t$ apoaq
                        aapq))))))
 (setf aapp
         (* aapp
            (f2cl-lib:fsqrt
             (max zero
                  (+ one
                     (* (- t$) aqoap
                        aapq))))))
 (setf mxsinj (max mxsinj (abs t$))))
(t
 (setf thsign
         (- (f2cl-lib:sign one aapq)))
 (if (> aaqq aapp0)
     (setf thsign (- thsign)))
 (setf t$
         (/ one
            (+ theta
               (* thsign
                  (f2cl-lib:fsqrt
                   (+ one
                      (* theta
                         theta)))))))
 (setf cs
         (f2cl-lib:fsqrt
```

```
         (/ one (+ one (* t$ t$)))))
(setf sn (* t$ cs))
(setf mxsinj (max mxsinj (abs sn)))
(setf (f2cl-lib:fref sva-%data% (q)
                     ((1 n))
                     sva-%offset%)
        (* aaqq
           (f2cl-lib:fsqrt
            (max zero
                 (+ one
                    (* t$ apoaq
                       aapq))))))
(setf aapp
        (* aapp
           (f2cl-lib:fsqrt
            (max zero
                 (+ one
                    (* (- t$) aqoap
                       aapq))))))
(setf apoaq
        (/
         (f2cl-lib:fref work-%data% (p)
                        ((1 lwork))
                        work-%offset%)
         (f2cl-lib:fref work-%data% (q)
                        ((1 lwork))
                        work-%offset%)))
(setf aqoap
        (/
         (f2cl-lib:fref work-%data% (q)
                        ((1 lwork))
                        work-%offset%)
         (f2cl-lib:fref work-%data% (p)
                        ((1 lwork))
                        work-%offset%)))
(cond
 ((>=
   (f2cl-lib:fref work (p) ((1 lwork)))
   one)
  (cond
   ((>=
     (f2cl-lib:fref work (q)
                    ((1 lwork)))
     one)
    (setf (f2cl-lib:fref fastr (3)
                         ((1 5)))
            (* t$ apoaq))
    (setf (f2cl-lib:fref fastr (4)
                         ((1 5)))
            (* (- t$) aqoap))
```

```
(setf (f2cl-lib:fref work-%data%
                     (p)
                     ((1 lwork))
                     work-%offset%)
        (*
         (f2cl-lib:fref work-%data%
                        (p)
                        ((1 lwork))
                        work-%offset%)
         cs))
(setf (f2cl-lib:fref work-%data%
                     (q)
                     ((1 lwork))
                     work-%offset%)
        (*
         (f2cl-lib:fref work-%data%
                        (q)
                        ((1 lwork))
                        work-%offset%)
         cs))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5)
    (srotm m
     (f2cl-lib:array-slice a-%data%
                           single-float
                           (1 p)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1
     (f2cl-lib:array-slice a-%data%
                           single-float
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1 fastr)
  (declare
   (ignore var-1 var-2 var-3 var-4
    var-5))
  (when var-0 (setf m var-0)))
(if rsvec
    (multiple-value-bind
        (var-0 var-1 var-2 var-3
         var-4 var-5)
        (srotm mvl
         (f2cl-lib:array-slice
          v-%data% single-float
          (1 p) ((1 ldv) (1 *))
```

```
            v-%offset%)
           1
           (f2cl-lib:array-slice
            v-%data% single-float
            (1 q) ((1 ldv) (1 *))
            v-%offset%)
           1 fastr)
        (declare
         (ignore var-1 var-2 var-3
          var-4 var-5))
        (when var-0
          (setf mvl var-0)))))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4
      var-5)
     (saxpy m (* (- t$) aqoap)
      (f2cl-lib:array-slice a-%data%
                            single-float
                            (1 q)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice a-%data%
                            single-float
                            (1 p)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4
     var-5))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4
      var-5)
     (saxpy m (* cs sn apoaq)
      (f2cl-lib:array-slice a-%data%
                            single-float
                            (1 p)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice a-%data%
                            single-float
                            (1 q)
                            ((1 lda)
```

```
                                (1 *))
                               a-%offset%)
      1)
  (declare
   (ignore var-1 var-2 var-3 var-4
    var-5))
  (when var-0 (setf m var-0)))
(cond
 (rsvec
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
       var-4 var-5)
      (saxpy mvl (* (- t$) aqoap)
       (f2cl-lib:array-slice
        v-%data% single-float (1 q)
        ((1 ldv) (1 *)) v-%offset%)
       1
       (f2cl-lib:array-slice
        v-%data% single-float (1 p)
        ((1 ldv) (1 *)) v-%offset%)
       1)
    (declare
     (ignore var-1 var-2 var-3
      var-4 var-5))
    (when var-0 (setf mvl var-0)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
       var-4 var-5)
      (saxpy mvl (* cs sn apoaq)
       (f2cl-lib:array-slice
        v-%data% single-float (1 p)
        ((1 ldv) (1 *)) v-%offset%)
       1
       (f2cl-lib:array-slice
        v-%data% single-float (1 q)
        ((1 ldv) (1 *)) v-%offset%)
       1)
    (declare
     (ignore var-1 var-2 var-3
      var-4 var-5))
    (when var-0
      (setf mvl var-0)))))
(setf (f2cl-lib:fref work-%data%
                     (p)
                     ((1 lwork))
                     work-%offset%)
        (*
         (f2cl-lib:fref work-%data%
                        (p)
                        ((1 lwork))
```

```
                                work-%offset%)
                 cs))
        (setf (f2cl-lib:fref work-%data%
                             (q)
                             ((1 lwork))
                             work-%offset%)
                (/
                 (f2cl-lib:fref work-%data%
                                (q)
                                ((1 lwork))
                                work-%offset%)
                 cs)))))
    (t
     (cond
      ((>=
        (f2cl-lib:fref work (q)
                       ((1 lwork)))
        one)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4
            var-5)
           (saxpy m (* t$ apoaq)
            (f2cl-lib:array-slice a-%data%
                                  single-float
                                  (1 p)
                                  ((1 lda)
                                   (1 *))
                                  a-%offset%)
            1
            (f2cl-lib:array-slice a-%data%
                                  single-float
                                  (1 q)
                                  ((1 lda)
                                   (1 *))
                                  a-%offset%)
            1)
         (declare
          (ignore var-1 var-2 var-3 var-4
           var-5))
         (when var-0 (setf m var-0)))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4
            var-5)
           (saxpy m (* (- cs) sn aqoap)
            (f2cl-lib:array-slice a-%data%
                                  single-float
                                  (1 q)
                                  ((1 lda)
                                   (1 *))
                                  a-%offset%)
```

```
      1
      (f2cl-lib:array-slice a-%data%
                            single-float
                            (1 p)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1)
  (declare
   (ignore var-1 var-2 var-3 var-4
    var-5))
  (when var-0 (setf m var-0)))
(cond
 (rsvec
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
       var-4 var-5)
      (saxpy mvl (* t$ apoaq)
       (f2cl-lib:array-slice
        v-%data% single-float (1 p)
        ((1 ldv) (1 *)) v-%offset%)
       1
       (f2cl-lib:array-slice
        v-%data% single-float (1 q)
        ((1 ldv) (1 *)) v-%offset%)
       1)
    (declare
     (ignore var-1 var-2 var-3
      var-4 var-5))
    (when var-0 (setf mvl var-0)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
       var-4 var-5)
      (saxpy mvl
       (* (- cs) sn aqoap)
       (f2cl-lib:array-slice
        v-%data% single-float (1 q)
        ((1 ldv) (1 *)) v-%offset%)
       1
       (f2cl-lib:array-slice
        v-%data% single-float (1 p)
        ((1 ldv) (1 *)) v-%offset%)
       1)
    (declare
     (ignore var-1 var-2 var-3
      var-4 var-5))
    (when var-0
      (setf mvl var-0)))))
(setf (f2cl-lib:fref work-%data%
                     (p)
```

```
                        ((1 lwork))
                        work-%offset%)
          (/
           (f2cl-lib:fref work-%data%
                          (p)
                          ((1 lwork))
                          work-%offset%)
           cs))
 (setf (f2cl-lib:fref work-%data%
                      (q)
                      ((1 lwork))
                      work-%offset%)
          (*
           (f2cl-lib:fref work-%data%
                          (q)
                          ((1 lwork))
                          work-%offset%)
           cs)))
(t
 (cond
  ((>=
    (f2cl-lib:fref work (p)
                   ((1 lwork)))
    (f2cl-lib:fref work (q)
                   ((1 lwork))))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3
        var-4 var-5)
       (saxpy m (* (- t$) aqoap)
        (f2cl-lib:array-slice
         a-%data% single-float (1 q)
         ((1 lda) (1 *)) a-%offset%)
        1
        (f2cl-lib:array-slice
         a-%data% single-float (1 p)
         ((1 lda) (1 *)) a-%offset%)
        1)
     (declare
      (ignore var-1 var-2 var-3
       var-4 var-5))
     (when var-0 (setf m var-0)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3
        var-4 var-5)
       (saxpy m (* cs sn apoaq)
        (f2cl-lib:array-slice
         a-%data% single-float (1 p)
         ((1 lda) (1 *)) a-%offset%)
        1
        (f2cl-lib:array-slice
```

```
      a-%data% single-float (1 q)
      ((1 lda) (1 *)) a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3
    var-4 var-5))
  (when var-0 (setf m var-0)))
(setf (f2cl-lib:fref work-%data%
                     (p)
                     ((1 lwork))
                     work-%offset%)
        (*
         (f2cl-lib:fref
          work-%data% (p)
          ((1 lwork))
          work-%offset%)
         cs))
(setf (f2cl-lib:fref work-%data%
                     (q)
                     ((1 lwork))
                     work-%offset%)
        (/
         (f2cl-lib:fref
          work-%data% (q)
          ((1 lwork))
          work-%offset%)
         cs))
(cond
 (rsvec
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
       var-4 var-5)
      (saxpy mvl (* (- t$) aqoap)
       (f2cl-lib:array-slice
        v-%data% single-float
        (1 q) ((1 ldv) (1 *))
        v-%offset%)
       1
       (f2cl-lib:array-slice
        v-%data% single-float
        (1 p) ((1 ldv) (1 *))
        v-%offset%)
       1)
    (declare
     (ignore var-1 var-2 var-3
      var-4 var-5))
    (when var-0
      (setf mvl var-0)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
```

```
        var-4 var-5)
       (saxpy mvl (* cs sn apoaq)
        (f2cl-lib:array-slice
         v-%data% single-float
         (1 p) ((1 ldv) (1 *))
         v-%offset%)
        1
        (f2cl-lib:array-slice
         v-%data% single-float
         (1 q) ((1 ldv) (1 *))
         v-%offset%)
        1)
     (declare
      (ignore var-1 var-2 var-3
       var-4 var-5))
     (when var-0
       (setf mvl var-0))))))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3
      var-4 var-5)
     (saxpy m (* t$ apoaq)
      (f2cl-lib:array-slice
       a-%data% single-float (1 p)
       ((1 lda) (1 *)) a-%offset%)
      1
      (f2cl-lib:array-slice
       a-%data% single-float (1 q)
       ((1 lda) (1 *)) a-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3
     var-4 var-5))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3
      var-4 var-5)
     (saxpy m (* (- cs) sn aqoap)
      (f2cl-lib:array-slice
       a-%data% single-float (1 q)
       ((1 lda) (1 *)) a-%offset%)
      1
      (f2cl-lib:array-slice
       a-%data% single-float (1 p)
       ((1 lda) (1 *)) a-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3
     var-4 var-5))
   (when var-0 (setf m var-0)))
```

```
(setf (f2cl-lib:fref work-%data%
                     (p)
                     ((1 lwork))
                     work-%offset%)
        (/
         (f2cl-lib:fref
          work-%data% (p)
          ((1 lwork))
          work-%offset%)
         cs))
(setf (f2cl-lib:fref work-%data%
                     (q)
                     ((1 lwork))
                     work-%offset%)
        (*
         (f2cl-lib:fref
          work-%data% (q)
          ((1 lwork))
          work-%offset%)
         cs))
(cond
 (rsvec
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
       var-4 var-5)
      (saxpy mvl (* t$ apoaq)
       (f2cl-lib:array-slice
        v-%data% single-float
        (1 p) ((1 ldv) (1 *))
        v-%offset%)
       1
       (f2cl-lib:array-slice
        v-%data% single-float
        (1 q) ((1 ldv) (1 *))
        v-%offset%)
       1)
    (declare
     (ignore var-1 var-2 var-3
      var-4 var-5))
    (when var-0
      (setf mvl var-0)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3
       var-4 var-5)
      (saxpy mvl
       (* (- cs) sn aqoap)
       (f2cl-lib:array-slice
        v-%data% single-float
        (1 q) ((1 ldv) (1 *))
        v-%offset%)
```

```
                1
                (f2cl-lib:array-slice
                 v-%data% single-float
                 (1 p) ((1 ldv) (1 *))
                 v-%offset%)
                1)
            (declare
             (ignore var-1 var-2 var-3
              var-4 var-5))
            (when var-0
              (setf mvl
                      var-0)))))))))))))))
(t
 (cond
  ((> aapp aaqq)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4)
       (scopy m
        (f2cl-lib:array-slice a-%data%
                              single-float
                              (1 p)
                              ((1 lda)
                               (1 *))
                              a-%offset%)
        1
        (f2cl-lib:array-slice work-%data%
                              single-float
                              ((+ n 1))
                              ((1 lwork))
                              work-%offset%)
        1)
     (declare
      (ignore var-1 var-2 var-3 var-4))
     (when var-0 (setf m var-0)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4
        var-5 var-6 var-7 var-8 var-9)
       (slascl "G" 0 0 aapp one m 1
        (f2cl-lib:array-slice work-%data%
                              single-float
                              ((+ n 1))
                              ((1 lwork))
                              work-%offset%)
        lda ierr)
     (declare
      (ignore var-0 var-1 var-2 var-6
       var-7))
     (when var-3 (setf aapp var-3))
     (when var-4 (setf one var-4))
     (when var-5 (setf m var-5))
```

```
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5 var-6 var-7 var-8 var-9)
    (slascl "G" 0 0 aaqq one m 1
     (f2cl-lib:array-slice a-%data%
                           single-float
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     lda ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6
    var-7))
  (when var-3 (setf aaqq var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(setf temp1
        (/
         (* (- aapq)
            (f2cl-lib:fref work-%data%
                           (p)
                           ((1 lwork))
                           work-%offset%))
         (f2cl-lib:fref work-%data% (q)
                        ((1 lwork))
                        work-%offset%)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5)
    (saxpy m temp1
     (f2cl-lib:array-slice work-%data%
                           single-float
                           ((+ n 1))
                           ((1 lwork))
                           work-%offset%)
     1
     (f2cl-lib:array-slice a-%data%
                           single-float
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1)
  (declare
   (ignore var-2 var-3 var-4 var-5))
```

```
  (when var-0 (setf m var-0))
  (when var-1 (setf temp1 var-1)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4
      var-5 var-6 var-7 var-8 var-9)
     (slascl "G" 0 0 one aaqq m 1
      (f2cl-lib:array-slice a-%data%
                            single-float
                            (1 q)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      lda ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6
    var-7))
  (when var-3 (setf one var-3))
  (when var-4 (setf aaqq var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
 (setf (f2cl-lib:fref sva-%data% (q)
                      ((1 n))
                      sva-%offset%)
        (* aaqq
           (f2cl-lib:fsqrt
            (max zero
                 (- one
                    (* aapq aapq))))))
 (setf mxsinj (max mxsinj sfmin)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (scopy m
      (f2cl-lib:array-slice a-%data%
                            single-float
                            (1 q)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice work-%data%
                            single-float
                            ((+ n 1))
                            ((1 lwork))
                            work-%offset%)
      1)
  (declare
   (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf m var-0)))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5 var-6 var-7 var-8 var-9)
    (slascl "G" 0 0 aaqq one m 1
     (f2cl-lib:array-slice work-%data%
                           single-float
                           ((+ n 1))
                           ((1 lwork))
                           work-%offset%)
     lda ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6
    var-7))
  (when var-3 (setf aaqq var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5 var-6 var-7 var-8 var-9)
    (slascl "G" 0 0 aapp one m 1
     (f2cl-lib:array-slice a-%data%
                           single-float
                           (1 p)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     lda ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6
    var-7))
  (when var-3 (setf aapp var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(setf temp1
        (/
         (* (- aapq)
            (f2cl-lib:fref work-%data%
                           (q)
                           ((1 lwork))
                           work-%offset%))
         (f2cl-lib:fref work-%data% (p)
                        ((1 lwork))
                        work-%offset%)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5)
```

```
                (saxpy m temp1
                 (f2cl-lib:array-slice work-%data%
                                       single-float
                                       ((+ n 1))
                                       ((1 lwork))
                                       work-%offset%)
                 1
                 (f2cl-lib:array-slice a-%data%
                                       single-float
                                       (1 p)
                                       ((1 lda)
                                        (1 *))
                                       a-%offset%)
                 1)
              (declare
               (ignore var-2 var-3 var-4 var-5))
              (when var-0 (setf m var-0))
              (when var-1 (setf temp1 var-1)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4
                 var-5 var-6 var-7 var-8 var-9)
                (slascl "G" 0 0 one aapp m 1
                 (f2cl-lib:array-slice a-%data%
                                       single-float
                                       (1 p)
                                       ((1 lda)
                                        (1 *))
                                       a-%offset%)
                 lda ierr)
              (declare
               (ignore var-0 var-1 var-2 var-6
                var-7))
              (when var-3 (setf one var-3))
              (when var-4 (setf aapp var-4))
              (when var-5 (setf m var-5))
              (when var-8 (setf lda var-8))
              (when var-9 (setf ierr var-9)))
            (setf (f2cl-lib:fref sva-%data% (p)
                                 ((1 n))
                                 sva-%offset%)
                  (* aapp
                     (f2cl-lib:fsqrt
                      (max zero
                           (- one
                              (* aapq aapq))))))
            (setf mxsinj (max mxsinj sfmin))))))
        (cond
         ((<=
           (expt
            (f2cl-lib:f2cl/
```

```
                    (f2cl-lib:fref sva (q) ((1 n))) aaqq)
                   2)
                  rooteps)
                 (cond
                  ((and (< aaqq rootbig)
                        (> aaqq rootsfmin))
                   (setf (f2cl-lib:fref sva-%data% (q)
                                        ((1 n))
                                        sva-%offset%)
                         (*
                          (multiple-value-bind
                              (ret-val var-0 var-1
                               var-2)
                              (snrm2 m
                               (f2cl-lib:array-slice
                                a-%data% single-float
                                (1 q) ((1 lda) (1 *))
                                a-%offset%)
                               1)
                            (declare
                             (ignore var-1 var-2))
                            (when var-0 (setf m var-0))
                            ret-val)
                          (f2cl-lib:fref work-%data% (q)
                                         ((1 lwork))
                                         work-%offset%))))
                  (t (setf t$ zero) (setf aaqq one)
                   (multiple-value-bind
                       (var-0 var-1 var-2 var-3 var-4)
                       (slassq m
                        (f2cl-lib:array-slice a-%data%
                                              single-float
                                              (1 q)
                                              ((1 lda)
                                               (1 *))
                                              a-%offset%)
                        1 t$ aaqq)
                     (declare (ignore var-1 var-2))
                     (when var-0 (setf m var-0))
                     (when var-3 (setf t$ var-3))
                     (when var-4 (setf aaqq var-4)))
                   (setf (f2cl-lib:fref sva-%data% (q)
                                        ((1 n))
                                        sva-%offset%)
                         (* t$ (f2cl-lib:fsqrt aaqq)
                            (f2cl-lib:fref work-%data%
                                           (q)
                                           ((1 lwork))
                                           work-%offset%)))))))
(cond
```

```
((<= (expt (f2cl-lib:f2cl/ aapp aapp0) 2)
     rooteps)
 (cond
  ((and (< aapp rootbig)
        (> aapp rootsfmin))
   (setf aapp
           (*
            (multiple-value-bind
                (ret-val var-0 var-1
                 var-2)
                (snrm2 m
                 (f2cl-lib:array-slice
                  a-%data% single-float
                  (1 p) ((1 lda) (1 *))
                  a-%offset%)
                 1)
               (declare
                (ignore var-1 var-2))
               (when var-0 (setf m var-0))
               ret-val)
            (f2cl-lib:fref work-%data% (p)
                           ((1 lwork))
                           work-%offset%))))
  (t (setf t$ zero) (setf aapp one)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4)
       (slassq m
        (f2cl-lib:array-slice a-%data%
                              single-float
                              (1 p)
                              ((1 lda)
                               (1 *))
                              a-%offset%)
        1 t$ aapp)
     (declare (ignore var-1 var-2))
     (when var-0 (setf m var-0))
     (when var-3 (setf t$ var-3))
     (when var-4 (setf aapp var-4)))
   (setf aapp
           (* t$ (f2cl-lib:fsqrt aapp)
              (f2cl-lib:fref work-%data%
                             (p)
                             ((1 lwork))
                             work-%offset%)))))
  (setf (f2cl-lib:fref sva-%data% (p)
                       ((1 n))
                       sva-%offset%)
        aapp))))
(t (setf notrot (f2cl-lib:int-add notrot 1))
 (setf pskipped
```

```
                                  (f2cl-lib:int-add pskipped 1))
                            (setf ijblsk
                                  (f2cl-lib:int-add ijblsk 1)))))
                         (t (setf notrot (f2cl-lib:int-add notrot 1))
                          (setf pskipped (f2cl-lib:int-add pskipped 1))
                          (setf ijblsk (f2cl-lib:int-add ijblsk 1))))
                        (cond
                         ((and (<= i swband) (>= ijblsk blskip))
                          (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                                               sva-%offset%)
                                aapp)
                          (setf notrot 0) (go label2011)))
                        (cond
                         ((and (<= i swband) (> pskipped rowskip))
                          (setf aapp (- aapp)) (setf notrot 0)
                          (go label2203)))
                       label2200))
                  label2203
                   (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                                        sva-%offset%)
                         aapp)))
                (t
                 (if (= aapp zero)
                     (setf notrot
                             (f2cl-lib:int-add
                              (f2cl-lib:int-sub
                               (f2cl-lib:int-add notrot
                                                 (min
                                                  (the
                                                   f2cl-lib:integer4
                                                   (f2cl-lib:int-sub
                                                    (f2cl-lib:int-add
                                                     jgl kbl)
                                                    1))
                                                  (the
                                                   f2cl-lib:integer4
                                                   n)))
                               jgl)
                              1)))
                 (if (< aapp zero)
                     (setf notrot 0))))
            label2100))
        label2010))
    label2011
     (f2cl-lib:fdo (p igl (f2cl-lib:int-add p 1))
                   ((> p
                       (min
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add igl kbl
                                               (f2cl-lib:int-sub
```

```
                                                    1)))
                                (the f2cl-lib:integer4 n)))
                            nil)
                 (tagbody
                   (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                           (abs
                            (f2cl-lib:fref sva-%data% (p) ((1 n))
                                           sva-%offset%)))
                  label2012))
              label2000))
            (cond
             ((and (< (f2cl-lib:fref sva (n) ((1 n))) rootbig)
                   (> (f2cl-lib:fref sva (n) ((1 n))) rootsfmin))
              (setf (f2cl-lib:fref sva-%data% (n) ((1 n)) sva-%offset%)
                      (*
                       (multiple-value-bind (ret-val var-0 var-1 var-2)
                           (snrm2 m
                            (f2cl-lib:array-slice a-%data% single-float (1 n)
                                                  ((1 lda) (1 *)) a-%offset%)
                            1)
                         (declare (ignore var-1 var-2))
                         (when var-0 (setf m var-0))
                         ret-val)
                       (f2cl-lib:fref work-%data% (n) ((1 lwork))
                                      work-%offset%))))
             (t (setf t$ zero) (setf aapp one)
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (slassq m
                   (f2cl-lib:array-slice a-%data% single-float (1 n)
                                         ((1 lda) (1 *)) a-%offset%)
                   1 t$ aapp)
                (declare (ignore var-1 var-2))
                (when var-0 (setf m var-0))
                (when var-3 (setf t$ var-3))
                (when var-4 (setf aapp var-4)))
              (setf (f2cl-lib:fref sva-%data% (n) ((1 n)) sva-%offset%)
                      (* t$ (f2cl-lib:fsqrt aapp)
                         (f2cl-lib:fref work-%data% (n) ((1 lwork))
                                        work-%offset%)))))
            (if (and (< i swband) (or (<= mxaapq roottol) (<= iswrot n)))
                (setf swband i))
            (cond
             ((and (> i (f2cl-lib:int-add swband 1))
                   (< mxaapq (* (f2cl-lib:fsqrt (f2cl-lib:ffloat n)) tol))
                   (< (* (f2cl-lib:ffloat n) mxaapq mxsinj) tol))
              (go label1994)))
            (if (>= notrot emptsw)
                (go label1994))
           label1993))
        (setf info (f2cl-lib:int-sub nsweep 1))
```

```
 (go label1995)
label1994
 (setf info 0)
label1995
 (setf n2 0)
 (setf n4 0)
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
   (tagbody
     (setf q
             (f2cl-lib:int-sub
              (f2cl-lib:int-add
               (isamax (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
                (f2cl-lib:array-slice sva-%data% single-float (p)
                                      ((1 n)) sva-%offset%)
                1)
               p)
              1))
     (cond
      ((/= p q)
       (setf temp1 (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%))
       (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
               (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%))
       (setf (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%) temp1)
       (setf temp1
               (f2cl-lib:fref work-%data% (p) ((1 lwork))
                              work-%offset%))
       (setf (f2cl-lib:fref work-%data% (p) ((1 lwork)) work-%offset%)
               (f2cl-lib:fref work-%data% (q) ((1 lwork))
                              work-%offset%))
       (setf (f2cl-lib:fref work-%data% (q) ((1 lwork)) work-%offset%)
               temp1)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (sswap m
            (f2cl-lib:array-slice a-%data% single-float (1 p)
                                  ((1 lda) (1 *)) a-%offset%)
            1
            (f2cl-lib:array-slice a-%data% single-float (1 q)
                                  ((1 lda) (1 *)) a-%offset%)
            1)
         (declare (ignore var-1 var-2 var-3 var-4))
         (when var-0 (setf m var-0)))
       (if rsvec
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
               (sswap mvl
                (f2cl-lib:array-slice v-%data% single-float (1 p)
                                      ((1 ldv) (1 *)) v-%offset%)
                1
                (f2cl-lib:array-slice v-%data% single-float (1 q)
                                      ((1 ldv) (1 *)) v-%offset%)
```

```
                1)
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf mvl var-0))))))
     (cond
      ((/= (f2cl-lib:fref sva (p) ((1 n))) zero)
       (setf n4 (f2cl-lib:int-add n4 1))
       (if (>
            (* (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%) skl)
            sfmin)
           (setf n2 (f2cl-lib:int-add n2 1)))))
   label5991))
 (cond
  ((/= (f2cl-lib:fref sva (n) ((1 n))) zero)
   (setf n4 (f2cl-lib:int-add n4 1))
   (if (> (* (f2cl-lib:fref sva-%data% (n) ((1 n)) sva-%offset%) skl)
          sfmin)
       (setf n2 (f2cl-lib:int-add n2 1)))))
 (cond
  ((or lsvec uctol)
   (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                 ((> p n2) nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2 var-3)
           (sscal m
            (/ (f2cl-lib:fref work-%data% (p) ((1 lwork)) work-%offset%)
               (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%))
            (f2cl-lib:array-slice a-%data% single-float (1 p)
                                  ((1 lda) (1 *)) a-%offset%)
            1)
         (declare (ignore var-1 var-2 var-3))
         (when var-0 (setf m var-0)))
     label1998))))
 (cond
  (rsvec
   (cond
    (applv
     (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                   ((> p n) nil)
       (tagbody
         (multiple-value-bind (var-0 var-1 var-2 var-3)
             (sscal mvl
              (f2cl-lib:array-slice work-%data% single-float (p)
                                    ((1 lwork)) work-%offset%)
              (f2cl-lib:array-slice v-%data% single-float (1 p)
                                    ((1 ldv) (1 *)) v-%offset%)
              1)
           (declare (ignore var-1 var-2 var-3))
           (when var-0 (setf mvl var-0)))
       label2398)))
   (t
```

```
    (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                  ((> p n) nil)
      (tagbody
        (setf temp1
                (/ one
                   (multiple-value-bind (ret-val var-0 var-1 var-2)
                       (snrm2 mvl
                        (f2cl-lib:array-slice v-%data% single-float
                                              (1 p) ((1 ldv) (1 *))
                                              v-%offset%)
                        1)
                      (declare (ignore var-1 var-2))
                      (when var-0 (setf mvl var-0))
                      ret-val)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal mvl temp1
             (f2cl-lib:array-slice v-%data% single-float (1 p)
                                   ((1 ldv) (1 *)) v-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf mvl var-0))
          (when var-1 (setf temp1 var-1)))
       label2399))))))
(cond
 ((or
   (and (> skl one)
        (< (f2cl-lib:fref sva (1) ((1 n))) (f2cl-lib:f2cl/ big skl)))
   (and (< skl one)
        (>
         (f2cl-lib:fref sva
                        ((max (the f2cl-lib:integer4 n2)
                              (the f2cl-lib:integer4 1)))
                        ((1 n)))
         (f2cl-lib:f2cl/ sfmin skl))))
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
              (* skl
                 (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)))
     label2400))
  (setf skl one)))
(setf (f2cl-lib:fref work-%data% (1) ((1 lwork)) work-%offset%) skl)
(setf (f2cl-lib:fref work-%data% (2) ((1 lwork)) work-%offset%)
        (f2cl-lib:ffloat n4))
(setf (f2cl-lib:fref work-%data% (3) ((1 lwork)) work-%offset%)
        (f2cl-lib:ffloat n2))
(setf (f2cl-lib:fref work-%data% (4) ((1 lwork)) work-%offset%)
        (f2cl-lib:ffloat i))
(setf (f2cl-lib:fref work-%data% (5) ((1 lwork)) work-%offset%) mxaapq)
```

```
        (setf (f2cl-lib:fref work-%data% (6) ((1 lwork)) work-%offset%) mxsinj)
        (go end_label)
       end_label
        (return
         (values joba jobu jobv m n nil lda nil nil nil ldv nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgesvj
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::joba
                                              fortran-to-lisp::jobu
                                              fortran-to-lisp::jobv
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::ldv nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

33.29 sgetf2

33.29.1 Axiom unit tests

— sgetf2.input —

```
)set break resume
)sys rm -f sgetf2.output
```

```
)spool sgetf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

33.29.2 Axiom help page

— sgetf2.help —

```
=====================================================================
sgetf2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgetf2 (M, N, A, LDA, IPIV, INFO)
           SGETF2 computes the LU factorization of a general m-by-n matrix
           using partial pivoting with row interchanges (unblocked algorithm).

Function/Subroutine Documentation
   subroutine sgetf2 (integer M, integer N, real, dimension( lda, * ) A,
       integer LDA, integer, dimension( * ) IPIV, integer INFO)
       SGETF2 computes the LU factorization of a general m-by-n matrix using
       partial pivoting with row interchanges (unblocked algorithm).

Purpose:
   SGETF2 computes an LU factorization of a general m-by-n matrix A
   using partial pivoting with row interchanges.

   The factorization has the form  A = P * L * U
   where P is a permutation matrix, L is lower triangular with unit
   diagonal elements (lower trapezoidal if m > n), and U is upper
   triangular (upper trapezoidal if m < n).

   This is the right-looking Level 2 BLAS version of the algorithm.

Parameters:

 M
```

```
  M is INTEGER
  The number of rows of the matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

A
  A is REAL array, dimension (LDA,N)
  On entry, the m by n matrix to be factored.
  On exit, the factors L and U from the factorization
  A = P*L*U; the unit diagonal elements of L are not stored.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

IPIV
  IPIV is INTEGER array, dimension (min(M,N))
  The pivot indices; for 1 <= i <= min(M,N), row i of the
  matrix was interchanged with row IPIV(i).

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -k, the k-th argument had an illegal value
  > 0: if INFO = k, U(k,k) is exactly zero. The factorization
     has been completed, but the factor U is exactly
     singular, and division by zero will occur if it is used
     to solve a system of equations.
```

33.29.3 fortran code

— sgetf2.f —

```
*  =====================================================================
      SUBROUTINE SGETF2( M, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
```

```
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      REAL               SFMIN
      INTEGER            I, J, JP
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      INTEGER            ISAMAX
      EXTERNAL           SLAMCH, ISAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGER, SSCAL, SSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGETF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Compute machine safe minimum
```

```
*
      SFMIN = SLAMCH('S')
*
      DO 10 J = 1, MIN( M, N )
*
*        Find pivot and test for singularity.
*
         JP = J - 1 + ISAMAX( M-J+1, A( J, J ), 1 )
         IPIV( J ) = JP
         IF( A( JP, J ).NE.ZERO ) THEN
*
*           Apply the interchange to columns 1:N.
*
            IF( JP.NE.J )
     $         CALL SSWAP( N, A( J, 1 ), LDA, A( JP, 1 ), LDA )
*
*           Compute elements J+1:M of J-th column.
*
            IF( J.LT.M ) THEN
               IF( ABS(A( J, J )) .GE. SFMIN ) THEN
                  CALL SSCAL( M-J, ONE / A( J, J ), A( J+1, J ), 1 )
               ELSE
                 DO 20 I = 1, M-J
                    A( J+I, J ) = A( J+I, J ) / A( J, J )
   20            CONTINUE
               END IF
            END IF
*
         ELSE IF( INFO.EQ.0 ) THEN
*
            INFO = J
         END IF
*
         IF( J.LT.MIN( M, N ) ) THEN
*
*           Update trailing submatrix.
*
            CALL SGER( M-J, N-J, -ONE, A( J+1, J ), 1, A( J, J+1 ), LDA,
     $                 A( J+1, J+1 ), LDA )
         END IF
   10 CONTINUE
      RETURN
*
*     End of SGETF2
*
      END
```

———-

33.29.4 lisp code

— sgetf2.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun sgetf2 (m n a lda ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array single-float (*)) a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((i 0) (j 0) (jp 0) (sfmin 0.0))
        (declare (type (f2cl-lib:integer4) i j jp)
                 (type (single-float) sfmin))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "SGETF2" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (or (= m 0) (= n 0))
            (go end_label))
        (setf sfmin (slamch "S"))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j
                          (min (the f2cl-lib:integer4 m)
                               (the f2cl-lib:integer4 n)))
                       nil)
          (tagbody
            (setf jp
                    (f2cl-lib:int-add (f2cl-lib:int-sub j 1)
                                      (isamax
                                       (f2cl-lib:int-add (f2cl-lib:int-sub m j)
                                                         1)
                                       (f2cl-lib:array-slice a-%data%
                                                             single-float (j j)
                                                             ((1 lda) (1 *))
                                                             a-%offset%)
                                       1)))
            (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%) jp)
            (cond
             ((/= (f2cl-lib:fref a (jp j) ((1 lda) (1 *))) zero)
              (if (/= jp j)
```

```
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (sswap n
             (f2cl-lib:array-slice a-%data% single-float (j 1)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice a-%data% single-float (jp 1)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-1 var-3))
          (when var-0 (setf n var-0))
          (when var-2 (setf lda var-2))
          (when var-4 (setf lda var-4))))
  (cond
   ((< j m)
    (cond
     ((>= (abs (f2cl-lib:fref a (j j) ((1 lda) (1 *)))) sfmin)
      (sscal (f2cl-lib:int-sub m j)
       (/ one
          (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                         a-%offset%))
       (f2cl-lib:array-slice a-%data% single-float ((+ j 1) j)
                             ((1 lda) (1 *)) a-%offset%)
       1))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (f2cl-lib:int-add m (f2cl-lib:int-sub j)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add j i) j)
                                ((1 lda) (1 *)) a-%offset%)
                (/
                 (f2cl-lib:fref a-%data%
                                ((f2cl-lib:int-add j i) j)
                                ((1 lda) (1 *)) a-%offset%)
                 (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                a-%offset%)))
          label20)))))))
 ((= info 0) (setf info j)))
(cond
 ((< j (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (sger (f2cl-lib:int-sub m j) (f2cl-lib:int-sub n j) (- one)
       (f2cl-lib:array-slice a-%data% single-float ((+ j 1) j)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% single-float
                             (j (f2cl-lib:int-add j 1))
                             ((1 lda) (1 *)) a-%offset%)
```

```
                  lda
                  (f2cl-lib:array-slice a-%data% single-float
                                        ((+ j 1) (f2cl-lib:int-add j 1))
                                        ((1 lda) (1 *)) a-%offset%)
                  lda)
                (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7))
                (when var-6 (setf lda var-6))
                (when var-8 (setf lda var-8)))))
           label10))
        (go end_label)
       end_label
        (return (values nil n nil lda nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgetf2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

33.30 sgetrf

33.30.1 Axiom unit tests

— **sgetrf.input** —

```
)set break resume
)sys rm -f sgetrf.output
)spool sgetrf.output
)set message test on
)set message auto off
```

```
)clear all

)spool
)lisp (bye)
```

33.30.2 Axiom help page

— sgetrf.help —

```
=====================================================================
sgetrf examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgetrf (M, N, A, LDA, IPIV, INFO)
           SGETRF

Function/Subroutine Documentation
   subroutine sgetrf (integer M, integer N, real, dimension( lda, * ) A,
       integer LDA, integer, dimension( * ) IPIV, integer INFO)
       SGETRF

Purpose:
   SGETRF computes an LU factorization of a general M-by-N matrix A
   using partial pivoting with row interchanges.

   The factorization has the form A = P * L * U
   where P is a permutation matrix, L is lower triangular with unit
   diagonal elements (lower trapezoidal if m > n), and U is upper
   triangular (upper trapezoidal if m < n).

   This is the right-looking Level 3 BLAS version of the algorithm.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
```

```
  The number of columns of the matrix A.  N >= 0.

A
  A is REAL array, dimension (LDA,N)
  On entry, the M-by-N matrix to be factored.
  On exit, the factors L and U from the factorization
  A = P*L*U; the unit diagonal elements of L are not stored.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

IPIV
  IPIV is INTEGER array, dimension (min(M,N))
  The pivot indices; for 1 <= i <= min(M,N), row i of the
  matrix was interchanged with row IPIV(i).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, U(i,i) is exactly zero. The factorization
     has been completed, but the factor U is exactly
     singular, and division by zero will occur if it is used
     to solve a system of equations.
```

33.30.3 fortran code

— sgetrf.f —

```
*  =====================================================================
      SUBROUTINE SGETRF( M, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * )
*     ..
```

```
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IINFO, J, JB, NB
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEMM, SGETRF2, SLASWP, STRSM, XERBLA
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGETRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Determine the block size for this environment.
*
      NB = ILAENV( 1, 'SGETRF', ' ', M, N, -1, -1 )
      IF( NB.LE.1 .OR. NB.GE.MIN( M, N ) ) THEN
*
*        Use unblocked code.
*
         CALL SGETRF2( M, N, A, LDA, IPIV, INFO )
```

```
      ELSE
*
*        Use blocked code.
*
         DO 20 J = 1, MIN( M, N ), NB
            JB = MIN( MIN( M, N )-J+1, NB )
*
*           Factor diagonal and subdiagonal blocks and test for exact
*           singularity.
*
            CALL SGETRF2( M-J+1, JB, A( J, J ), LDA, IPIV( J ), IINFO )
*
*           Adjust INFO and the pivot indices.
*
            IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $         INFO = IINFO + J - 1
            DO 10 I = J, MIN( M, J+JB-1 )
               IPIV( I ) = J - 1 + IPIV( I )
   10       CONTINUE
*
*           Apply interchanges to columns 1:J-1.
*
            CALL SLASWP( J-1, A, LDA, J, J+JB-1, IPIV, 1 )
*
            IF( J+JB.LE.N ) THEN
*
*              Apply interchanges to columns J+JB:N.
*
               CALL SLASWP( N-J-JB+1, A( 1, J+JB ), LDA, J, J+JB-1,
     $                      IPIV, 1 )
*
*              Compute block row of U.
*
               CALL STRSM( 'Left', 'Lower', 'No transpose', 'Unit', JB,
     $                     N-J-JB+1, ONE, A( J, J ), LDA, A( J, J+JB ),
     $                     LDA )
               IF( J+JB.LE.M ) THEN
*
*                 Update trailing submatrix.
*
                  CALL SGEMM( 'No transpose', 'No transpose', M-J-JB+1,
     $                        N-J-JB+1, JB, -ONE, A( J+JB, J ), LDA,
     $                        A( J, J+JB ), LDA, ONE, A( J+JB, J+JB ),
     $                        LDA )
               END IF
            END IF
   20    CONTINUE
      END IF
      RETURN
*
```

```
*     End of SGETRF
*
      END
```

———-

33.30.4 lisp code

— **sgetrf.lisp** —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun sgetrf (m n a lda ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array single-float (*)) a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((i 0) (iinfo 0) (j 0) (jb 0) (nb 0))
        (declare (type (f2cl-lib:integer4) i iinfo j jb nb))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "SGETRF" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (or (= m 0) (= n 0))
            (go end_label))
        (setf nb
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (ilaenv 1 "SGETRF" " " m n -1 -1)
                  (declare (ignore var-0 var-1 var-2 var-5 var-6))
                  (setf m var-3)
                  (setf n var-4)
                  ret-val))
        (cond
         ((or (<= nb 1)
              (>= nb
                  (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n))))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
              (sgetrf2 m n a lda ipiv info)
            (declare (ignore var-2 var-4))
            (when var-0 (setf m var-0))
```

```
    (when var-1 (setf n var-1))
    (when var-3 (setf lda var-3))
    (when var-5 (setf info var-5))))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                 ((> j
                     (min (the f2cl-lib:integer4 m)
                          (the f2cl-lib:integer4 n)))
                  nil)
    (tagbody
      (setf jb
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add
                     (f2cl-lib:int-sub
                      (min (the f2cl-lib:integer4 m)
                           (the f2cl-lib:integer4 n))
                      j)
                     1))
               (the f2cl-lib:integer4 nb)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (sgetrf2 (f2cl-lib:int-add (f2cl-lib:int-sub m j) 1) jb
           (f2cl-lib:array-slice a-%data% single-float (j j)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (j)
                                 ((1 *)) ipiv-%offset%)
           iinfo)
        (declare (ignore var-0 var-2 var-4))
        (when var-1 (setf jb var-1))
        (when var-3 (setf lda var-3))
        (when var-5 (setf iinfo var-5)))
      (if (and (= info 0) (> iinfo 0))
          (setf info (f2cl-lib:int-sub (f2cl-lib:int-add iinfo j) 1)))
      (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                    ((> i
                        (min (the f2cl-lib:integer4 m)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add j jb
                                                    (f2cl-lib:int-sub
                                                     1)))))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
                  (f2cl-lib:int-add (f2cl-lib:int-sub j 1)
                                    (f2cl-lib:fref ipiv-%data% (i)
                                                   ((1 *))
                                                   ipiv-%offset%)))
         label10))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
      (slaswp (f2cl-lib:int-sub j 1) a lda j
       (f2cl-lib:int-sub (f2cl-lib:int-add j jb) 1) ipiv 1)
    (declare (ignore var-0 var-1 var-4 var-5 var-6))
    (when var-2 (setf lda var-2))
    (when var-3 (setf j var-3)))
  (cond
   ((<= (f2cl-lib:int-add j jb) n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaswp (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1)
         (f2cl-lib:array-slice a-%data% single-float
                               (1 (f2cl-lib:int-add j jb))
                               ((1 lda) (1 *)) a-%offset%)
         lda j (f2cl-lib:int-sub (f2cl-lib:int-add j jb) 1) ipiv 1)
      (declare (ignore var-0 var-1 var-4 var-5 var-6))
      (when var-2 (setf lda var-2))
      (when var-3 (setf j var-3)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10)
        (strsm "Left" "Lower" "No transpose" "Unit" jb
         (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) one
         (f2cl-lib:array-slice a-%data% single-float (j j)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice a-%data% single-float
                               (j (f2cl-lib:int-add j jb))
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9))
      (when var-4 (setf jb var-4))
      (when var-6 (setf one var-6))
      (when var-8 (setf lda var-8))
      (when var-10 (setf lda var-10)))
    (cond
     ((<= (f2cl-lib:int-add j jb) m)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (sgemm "No transpose" "No transpose"
           (f2cl-lib:int-add (f2cl-lib:int-sub m j jb) 1)
           (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) jb
           (- one)
           (f2cl-lib:array-slice a-%data% single-float ((+ j jb) j)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice a-%data% single-float
                                 (j (f2cl-lib:int-add j jb))
                                 ((1 lda) (1 *)) a-%offset%)
           lda one
```

```
                      (f2cl-lib:array-slice a-%data% single-float
                                            ((+ j jb) (f2cl-lib:int-add j jb))
                                            ((1 lda) (1 *)) a-%offset%)
                      lda)
                    (declare
                     (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8 var-11))
                    (when var-4 (setf jb var-4))
                    (when var-7 (setf lda var-7))
                    (when var-9 (setf lda var-9))
                    (when var-10 (setf one var-10))
                    (when var-12 (setf lda var-12)))))))
             label20))))
        (go end_label)
       end_label
        (return (values m n nil lda nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgetrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::ilaenv))))
```

———-

33.31 sgetrf2

33.31.1 Axiom unit tests

— sgetrf2.input —

```
)set break resume
```

```
)sys rm -f sgetrf2.output
)spool sgetrf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.31.2 Axiom help page

— **sgetrf2.help** —

```
======================================================================
sgetrf2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       recursive subroutine sgetrf2 (M, N, A, LDA, IPIV, INFO)
           SGETRF2

Function/Subroutine Documentation
   recursive subroutine sgetrf2 (integer M, integer N, real, dimension( lda, *
       ) A, integer LDA, integer, dimension( * ) IPIV, integer INFO)
       SGETRF2

Purpose:
   SGETRF2 computes an LU factorization of a general M-by-N matrix A
   using partial pivoting with row interchanges.

   The factorization has the form
   A = P * L * U
   where P is a permutation matrix, L is lower triangular with unit
   diagonal elements (lower trapezoidal if m > n), and U is upper
   triangular (upper trapezoidal if m < n).

   This is the recursive version of the algorithm. It divides
   the matrix into four submatrices:

           [  A11 | A12  ]  where A11 is n1 by n1 and A22 is n2 by n2
      A = [ --&mdash;|--&mdash; ]  with n1 = min(m,n)
```

```
                                        [ A11 ]
  The subroutine calls itself to factor [ --- ],
                                        [ A12 ]
                  [ A12 ]
  do the swaps on [ --- ], solve A12, update A22,
                  [ A22 ]

  then calls itself to factor A22 and do the swaps on A21.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the M-by-N matrix to be factored.
   On exit, the factors L and U from the factorization
   A = P*L*U; the unit diagonal elements of L are not stored.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 IPIV
   IPIV is INTEGER array, dimension (min(M,N))
   The pivot indices; for 1 <= i <= min(M,N), row i of the
   matrix was interchanged with row IPIV(i).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, U(i,i) is exactly zero. The factorization
      has been completed, but the factor U is exactly
      singular, and division by zero will occur if it is used
      to solve a system of equations.
```

33.31.3 fortran code

— **sgetrf2.f** —

```
*  =====================================================================
      RECURSIVE SUBROUTINE SGETRF2( M, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      REAL               SFMIN, TEMP
      INTEGER            I, IINFO, n1, n2
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      INTEGER            ISAMAX
      EXTERNAL           SLAMCH, ISAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEMM, SSCAL, SLASWP, STRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
```

```
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGETRF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN

      IF ( M.EQ.1 ) THEN
*
*        Use unblocked code for one row case
*        Just need to handle IPIV and INFO
*
         IPIV( 1 ) = 1
         IF ( A(1,1).EQ.ZERO )
     $      INFO = 1
*
      ELSE IF( N.EQ.1 ) THEN
*
*        Use unblocked code for one column case
*
*
*        Compute machine safe minimum
*
         SFMIN = SLAMCH('S')
*
*        Find pivot and test for singularity
*
         I = ISAMAX( M, A( 1, 1 ), 1 )
         IPIV( 1 ) = I
         IF( A( I, 1 ).NE.ZERO ) THEN
*
*           Apply the interchange
*
            IF( I.NE.1 ) THEN
               TEMP = A( 1, 1 )
               A( 1, 1 ) = A( I, 1 )
               A( I, 1 ) = TEMP
            END IF
*
*           Compute elements 2:M of the column
*
            IF( ABS(A( 1, 1 )) .GE. SFMIN ) THEN
               CALL SSCAL( M-1, ONE / A( 1, 1 ), A( 2, 1 ), 1 )
            ELSE
               DO 10 I = 1, M-1
                  A( 1+I, 1 ) = A( 1+I, 1 ) / A( 1, 1 )
```

```
   10          CONTINUE
            END IF
*
         ELSE
            INFO = 1
         END IF
*
      ELSE
*
*        Use recursive code
*
         N1 = MIN( M, N ) / 2
         N2 = N-N1
*
*               [ A11 ]
*        Factor [ --- ]
*               [ A21 ]
*
         CALL SGETRF2( m, n1, A, lda, ipiv, iinfo )

         IF ( info.EQ.0 .AND. iinfo.GT.0 )
     $      info = iinfo
*
*                              [ A12 ]
*        Apply interchanges to [ --- ]
*                              [ A22 ]
*
         CALL SLASWP( N2, A( 1, N1+1 ), LDA, 1, N1, IPIV, 1 )
*
*        Solve A12
*
         CALL STRSM( 'L', 'L', 'N', 'U', N1, N2, ONE, A, LDA,
     $               A( 1, N1+1 ), LDA )
*
*        Update A22
*
         CALL SGEMM( 'N', 'N', M-N1, N2, N1, -ONE, A( N1+1, 1 ), LDA,
     $               A( 1, N1+1 ), LDA, ONE, A( N1+1, N1+1 ), LDA )
*
*        Factor A22
*
         CALL SGETRF2( M-N1, N2, A( N1+1, N1+1 ), LDA, IPIV( N1+1 ),
     $                 IINFO )
*
*        Adjust INFO and the pivot indices
*
         IF ( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO + N1
         DO 20 I = N1+1, MIN( M, N )
            IPIV( I ) = IPIV( I ) + N1
```

```
   20    CONTINUE
*
*        Apply interchanges to A21
*
         CALL SLASWP( N1, A( 1, 1 ), LDA, N1+1, MIN( M, N), IPIV, 1 )
*
      END IF
      RETURN
*
*     End of SGETRF2
*
      END
```

———-

33.31.4 lisp code

— **sgetrf2.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun *main* ()
    (prog ((i 0) (iinfo 0) (n1 0) (n2 0) (sfmin 0.0) (temp 0.0) (m 0) (info 0)
           (ipiv (make-array * :element-type 'f2cl-lib:integer4)) (lda 0)
           (a
            (make-array (the fixnum (reduce #'* (list lda *))) :element-type
                        'single-float))
           (n 0) (subroutine 0.0))
      (declare (type (array single-float (*)) a)
               (type (array f2cl-lib:integer4 (*)) ipiv)
               (type (single-float) subroutine sfmin temp)
               (type (f2cl-lib:integer4) n lda info i iinfo n1 n2 m))
      '"****NOT TRANSLATED: (RECURSIVE SUBROUTINE SGETRF2 (M |,| N |,| A |,| LDA |,| IPIV |,| IN
      (setf info 0)
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4)))
      (cond
       ((/= info 0) (xerbla "SGETRF2" (f2cl-lib:int-sub info)) (go end_label)))
      (if (or (= m 0) (= n 0))
          (go end_label))
      (cond
       ((= m 1) (setf (f2cl-lib:fref ipiv (1) ((1 *))) 1)
        (if (= (f2cl-lib:fref a (1 1) ((1 lda) (1 *))) zero)
            (setf info 1)))
       ((= n 1) (setf sfmin (slamch "S"))
```

```
(setf i
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (isamax m
             (f2cl-lib:array-slice a single-float (1 1)
                                   ((1 lda) (1 *)))
             1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf m var-0))
          ret-val))
(setf (f2cl-lib:fref ipiv (1) ((1 *))) i)
(cond
 ((/= (f2cl-lib:fref a (i 1) ((1 lda) (1 *))) zero)
  (cond
   ((/= i 1) (setf temp (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))
    (setf (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))
            (f2cl-lib:fref a (i 1) ((1 lda) (1 *))))
    (setf (f2cl-lib:fref a (i 1) ((1 lda) (1 *))) temp)))
  (cond
   ((>= (abs (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))) sfmin)
    (sscal (f2cl-lib:int-sub m 1)
     (/ one (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))
     (f2cl-lib:array-slice a single-float (2 1) ((1 lda) (1 *))) 1))
   (t
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (setf (f2cl-lib:fref a ((f2cl-lib:int-add 1 i) 1)
                             ((1 lda) (1 *)))
                (/
                 (f2cl-lib:fref a ((f2cl-lib:int-add 1 i) 1)
                                ((1 lda) (1 *)))
                 (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))))
       label10)))))
 (t (setf info 1))))
(t (setf n1 (the f2cl-lib:integer4 (truncate (min m n) 2)))
 (setf n2 (f2cl-lib:int-sub n n1))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
     (sgetrf2 m n1 a lda ipiv iinfo)
   (declare (ignore var-2 var-4))
   (when var-0 (setf m var-0))
   (when var-1 (setf n1 var-1))
   (when var-3 (setf lda var-3))
   (when var-5 (setf iinfo var-5)))
 (if (and (= info 0) (> iinfo 0))
     (setf info iinfo))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slaswp n2
      (f2cl-lib:array-slice a single-float (1 (f2cl-lib:int-add n1 1))
                            ((1 lda) (1 *)))
      lda 1 n1 ipiv 1)
```

```
  (declare (ignore var-1 var-3 var-5 var-6))
  (when var-0 (setf n2 var-0))
  (when var-2 (setf lda var-2))
  (when var-4 (setf n1 var-4)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (strsm "L" "L" "N" "U" n1 n2 one a lda
     (f2cl-lib:array-slice a single-float (1 (f2cl-lib:int-add n1 1))
                           ((1 lda) (1 *)))
     lda)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf n2 var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf lda var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (sgemm "N" "N" (f2cl-lib:int-sub m n1) n2 n1 (- one)
     (f2cl-lib:array-slice a single-float ((+ n1 1) 1) ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice a single-float (1 (f2cl-lib:int-add n1 1))
                           ((1 lda) (1 *)))
     lda one
     (f2cl-lib:array-slice a single-float
                           ((+ n1 1) (f2cl-lib:int-add n1 1))
                           ((1 lda) (1 *)))
     lda)
  (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
  (when var-3 (setf n2 var-3))
  (when var-4 (setf n1 var-4))
  (when var-7 (setf lda var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf lda var-12)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (sgetrf2 (f2cl-lib:int-sub m n1) n2
     (f2cl-lib:array-slice a single-float
                           ((+ n1 1) (f2cl-lib:int-add n1 1))
                           ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice ipiv f2cl-lib:integer4 ((+ n1 1)) ((1 *)))
     iinfo)
  (declare (ignore var-0 var-2 var-4))
  (when var-1 (setf n2 var-1))
  (when var-3 (setf lda var-3))
  (when var-5 (setf iinfo var-5)))
(if (and (= info 0) (> iinfo 0))
```

```
            (setf info (f2cl-lib:int-add iinfo n1)))
        (f2cl-lib:fdo (i (f2cl-lib:int-add n1 1) (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the f2cl-lib:integer4 m)
                               (the f2cl-lib:integer4 n)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref ipiv (i) ((1 *)))
                    (f2cl-lib:int-add (f2cl-lib:fref ipiv (i) ((1 *))) n1))
           label20))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (slaswp n1
             (f2cl-lib:array-slice a single-float (1 1) ((1 lda) (1 *))) lda
             (f2cl-lib:int-add n1 1)
             (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) ipiv 1)
          (declare (ignore var-1 var-3 var-4 var-5 var-6))
          (when var-0 (setf n1 var-0))
          (when var-2 (setf lda var-2)))))
      (go end_label)
     end_label
      (return nil))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::*main*
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types 'nil :return-values 'nil
                                            :calls 'nil)))
```

———-

33.32 sgetri

33.32.1 Axiom unit tests

— **sgetri.input** —

```
)set break resume
)sys rm -f sgetri.output
)spool sgetri.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

33.32.2 Axiom help page

— **sgetri.help** —

```
==========================================================================
sgetri examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgetri (N, A, LDA, IPIV, WORK, LWORK, INFO)
           SGETRI

Function/Subroutine Documentation
   subroutine sgetri (integer N, real, dimension( lda, * ) A, integer LDA,
       integer, dimension( * ) IPIV, real, dimension( * ) WORK, integer LWORK,
       integer INFO)
       SGETRI

Purpose:
   SGETRI computes the inverse of a matrix using the LU factorization
   computed by SGETRF.

   This method inverts U and then computes inv(A) by solving the system
   inv(A)*L = inv(U) for inv(A).

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the factors L and U from the factorization
   A = P*L*U as computed by SGETRF.
   On exit, if INFO = 0, the inverse of the original matrix A.

 LDA
```

```
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from SGETRF; for 1<=i<=N, row i of the
  matrix was interchanged with row IPIV(i).

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO=0, then WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,N).
  For optimal performance LWORK >= N*NB, where NB is
  the optimal blocksize returned by ILAENV.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, U(i,i) is exactly zero; the matrix is
        singular and its inverse could not be computed.
```

———-

33.32.3 fortran code

— **sgetri.f** —

```
*  =====================================================================
      SUBROUTINE SGETRI( N, A, LDA, IPIV, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, N
*     ..
```

```
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IWS, J, JB, JJ, JP, LDWORK, LWKOPT, NB,
     $                   NBMIN, NN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEMM, SGEMV, SSWAP, STRSM, STRTRI, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NB = ILAENV( 1, 'SGETRI', ' ', N, -1, -1, -1 )
      LWKOPT = N*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -3
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGETRI', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
```

```
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Form inv(U).  If INFO > 0 from STRTRI, then U is singular,
*     and the inverse is not computed.
*
      CALL STRTRI( 'Upper', 'Non-unit', N, A, LDA, INFO )
      IF( INFO.GT.0 )
     $   RETURN
*
      NBMIN = 2
      LDWORK = N
      IF( NB.GT.1 .AND. NB.LT.N ) THEN
         IWS = MAX( LDWORK*NB, 1 )
         IF( LWORK.LT.IWS ) THEN
            NB = LWORK / LDWORK
            NBMIN = MAX( 2, ILAENV( 2, 'SGETRI', ' ', N, -1, -1, -1 ) )
         END IF
      ELSE
         IWS = N
      END IF
*
*     Solve the equation inv(A)*L = inv(U) for inv(A).
*
      IF( NB.LT.NBMIN .OR. NB.GE.N ) THEN
*
*        Use unblocked code.
*
         DO 20 J = N, 1, -1
*
*           Copy current column of L to WORK and replace with zeros.
*
            DO 10 I = J + 1, N
               WORK( I ) = A( I, J )
               A( I, J ) = ZERO
   10       CONTINUE
*
*           Compute current column of inv(A).
*
            IF( J.LT.N )
     $         CALL SGEMV( 'No transpose', N, N-J, -ONE, A( 1, J+1 ),
     $                     LDA, WORK( J+1 ), 1, ONE, A( 1, J ), 1 )
   20    CONTINUE
      ELSE
*
*        Use blocked code.
*
         NN = ( ( N-1 ) / NB )*NB + 1
         DO 50 J = NN, 1, -NB
```

```
            JB = MIN( NB, N-J+1 )
*
*           Copy current block column of L to WORK and replace with
*           zeros.
*
            DO 40 JJ = J, J + JB - 1
               DO 30 I = JJ + 1, N
                  WORK( I+( JJ-J )*LDWORK ) = A( I, JJ )
                  A( I, JJ ) = ZERO
   30          CONTINUE
   40       CONTINUE
*
*           Compute current block column of inv(A).
*
            IF( J+JB.LE.N )
     $         CALL SGEMM( 'No transpose', 'No transpose', N, JB,
     $                     N-J-JB+1, -ONE, A( 1, J+JB ), LDA,
     $                     WORK( J+JB ), LDWORK, ONE, A( 1, J ), LDA )
            CALL STRSM( 'Right', 'Lower', 'No transpose', 'Unit', N, JB,
     $                  ONE, WORK( J ), LDWORK, A( 1, J ), LDA )
   50    CONTINUE
      END IF
*
*     Apply column interchanges.
*
      DO 60 J = N - 1, 1, -1
         JP = IPIV( J )
         IF( JP.NE.J )
     $      CALL SSWAP( N, A( 1, J ), 1, A( 1, JP ), 1 )
   60 CONTINUE
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of SGETRI
*
      END
```

———-

33.32.4 lisp code

— **sgetri.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
```

```
(defun sgetri (n a lda ipiv work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array single-float (*)) work a)
           (type (f2cl-lib:integer4) info lwork lda n))
  (f2cl-lib:with-multi-array-data
      ((a single-float a-%data% a-%offset%)
       (work single-float work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((i 0) (iws 0) (j 0) (jb 0) (jj 0) (jp 0) (ldwork 0) (lwkopt 0)
           (nb 0) (nbmin 0) (nn 0) (lquery nil))
      (declare
       (type (f2cl-lib:integer4) i iws j jb jj jp ldwork lwkopt nb nbmin nn)
       (type f2cl-lib:logical lquery))
      (setf info 0)
      (setf nb
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 1 "SGETRI" " " n -1 -1 -1)
                (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6))
                (setf n var-3)
                ret-val))
      (setf lwkopt (f2cl-lib:int-mul n nb))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< n 0) (setf info -1))
            ((< lda
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -3))
            ((and
              (< lwork
                 (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
              (not lquery))
             (setf info -6)))
      (cond
       ((/= info 0) (xerbla "SGETRI" (f2cl-lib:int-sub info)) (go end_label))
       (lquery (go end_label)))
      (if (= n 0)
          (go end_label))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (strtri "Upper" "Non-unit" n a lda info)
        (declare (ignore var-0 var-1 var-3))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-5 (setf info var-5)))
      (if (> info 0)
          (go end_label))
      (setf nbmin 2)
      (setf ldwork n)
      (cond
```

```
 ((and (> nb 1) (< nb n))
  (setf iws
          (max (the f2cl-lib:integer4 (f2cl-lib:int-mul ldwork nb))
               (the f2cl-lib:integer4 1)))
  (cond
   ((< lwork iws)
    (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
    (setf nbmin
            (max (the f2cl-lib:integer4 2)
                 (the f2cl-lib:integer4
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
                          (ilaenv 2 "SGETRI" " " n -1 -1 -1)
                        (declare
                         (ignore var-0 var-1 var-2 var-4 var-5 var-6))
                        (setf n var-3)
                        ret-val)))))))
 (t (setf iws n)))
(cond
 ((or (< nb nbmin) (>= nb n))
  (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                ((> j 1) nil)
    (tagbody
      (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%))
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%)
                  zero)
         label10))
      (if (< j n)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10)
              (sgemv "No transpose" n (f2cl-lib:int-sub n j) (- one)
               (f2cl-lib:array-slice a-%data% single-float
                                     (1 (f2cl-lib:int-add j 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice work-%data% single-float ((+ j 1))
                                     ((1 *)) work-%offset%)
               1 one
               (f2cl-lib:array-slice a-%data% single-float (1 j)
                                     ((1 lda) (1 *)) a-%offset%)
               1)
            (declare
```

```
            (ignore var-0 var-2 var-3 var-4 var-6 var-7 var-9 var-10))
           (when var-1 (setf n var-1))
           (when var-5 (setf lda var-5))
           (when var-8 (setf one var-8))))
    label20)))
(t
 (setf nn (+ (* (the f2cl-lib:integer4 (truncate (- n 1) nb)) nb) 1))
 (f2cl-lib:fdo (j nn (f2cl-lib:int-add j (f2cl-lib:int-sub nb)))
               ((> j 1) nil)
   (tagbody
     (setf jb
             (min (the f2cl-lib:integer4 nb)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1))))
     (f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
                   ((> jj
                       (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
                    nil)
       (tagbody
         (f2cl-lib:fdo (i (f2cl-lib:int-add jj 1)
                          (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add i
                                                     (f2cl-lib:int-mul
                                                      (f2cl-lib:int-sub
                                                       jj j)
                                                      ldwork)))
                                  ((1 *)) work-%offset%)
                     (f2cl-lib:fref a-%data% (i jj) ((1 lda) (1 *))
                                    a-%offset%))
             (setf (f2cl-lib:fref a-%data% (i jj) ((1 lda) (1 *))
                                  a-%offset%)
                     zero)
            label30))
        label40))
     (if (<= (f2cl-lib:int-add j jb) n)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9 var-10 var-11 var-12)
             (sgemm "No transpose" "No transpose" n jb
              (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) (- one)
              (f2cl-lib:array-slice a-%data% single-float
                                    (1 (f2cl-lib:int-add j jb))
                                    ((1 lda) (1 *)) a-%offset%)
              lda
              (f2cl-lib:array-slice work-%data% single-float
                                    ((+ j jb)) ((1 *)) work-%offset%)
              ldwork one
```

```
                  (f2cl-lib:array-slice a-%data% single-float (1 j)
                                        ((1 lda) (1 *)) a-%offset%)
                  lda)
                (declare
                 (ignore var-0 var-1 var-4 var-5 var-6 var-8 var-11))
                (when var-2 (setf n var-2))
                (when var-3 (setf jb var-3))
                (when var-7 (setf lda var-7))
                (when var-9 (setf ldwork var-9))
                (when var-10 (setf one var-10))
                (when var-12 (setf lda var-12))))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10)
            (strsm "Right" "Lower" "No transpose" "Unit" n jb one
             (f2cl-lib:array-slice work-%data% single-float (j) ((1 *))
                                   work-%offset%)
             ldwork
             (f2cl-lib:array-slice a-%data% single-float (1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
          (when var-4 (setf n var-4))
          (when var-5 (setf jb var-5))
          (when var-6 (setf one var-6))
          (when var-8 (setf ldwork var-8))
          (when var-10 (setf lda var-10)))
      label50))))
 (f2cl-lib:fdo (j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (setf jp (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
     (if (/= jp j)
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (sswap n
              (f2cl-lib:array-slice a-%data% single-float (1 j)
                                    ((1 lda) (1 *)) a-%offset%)
              1
              (f2cl-lib:array-slice a-%data% single-float (1 jp)
                                    ((1 lda) (1 *)) a-%offset%)
              1)
           (declare (ignore var-1 var-2 var-3 var-4))
           (when var-0 (setf n var-0))))
    label60))
 (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce (the f2cl-lib:integer4 iws) 'single-float))
 (go end_label)
end_label
 (return (values n nil lda nil nil nil info))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgetri
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::ilaenv))))
```

———-

33.33 sgetrs

33.33.1 Axiom unit tests

— **sgetrs.input** —

```
)set break resume
)sys rm -f sgetrs.output
)spool sgetrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.33.2 Axiom help page

— sgetrs.help —

```
=====================================================================
sgetrs examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgetrs (TRANS, N, NRHS, A, LDA, IPIV, B, LDB, INFO)
           SGETRS

Function/Subroutine Documentation
   subroutine sgetrs (character TRANS, integer N, integer NRHS, real,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV, real,
       dimension( ldb, * ) B, integer LDB, integer INFO)
       SGETRS

Purpose:
   SGETRS solves a system of linear equations
      A * X = B  or  A**T * X = B
   with a general N-by-N matrix A using the LU factorization computed
   by SGETRF.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B  (No transpose)
   = 'T':  A**T* X = B  (Transpose)
   = 'C':  A**T* X = B  (Conjugate transpose = Transpose)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is REAL array, dimension (LDA,N)
   The factors L and U from the factorization A = P*L*U
   as computed by SGETRF.
```

```
LDA

  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from SGETRF; for 1<=i<=N, row i of the
  matrix was interchanged with row IPIV(i).

B
  B is REAL array, dimension (LDB,NRHS)
  On entry, the right hand side matrix B.
  On exit, the solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

33.33.3 fortran code

— **sgetrs.f** —

```
*  =====================================================================
      SUBROUTINE SGETRS( TRANS, N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), B( LDB, * )
*     ..
*
```

```
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLASWP, STRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGETRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( NOTRAN ) THEN
*
*        Solve A * X = B.
```

```
*
*        Apply row interchanges to the right hand sides.
*
         CALL SLASWP( NRHS, B, LDB, 1, N, IPIV, 1 )
*
*        Solve L*X = B, overwriting B with X.
*
         CALL STRSM( 'Left', 'Lower', 'No transpose', 'Unit', N, NRHS,
     $               ONE, A, LDA, B, LDB )
*
*        Solve U*X = B, overwriting B with X.
*
         CALL STRSM( 'Left', 'Upper', 'No transpose', 'Non-unit', N,
     $               NRHS, ONE, A, LDA, B, LDB )
      ELSE
*
*        Solve A**T * X = B.
*
*        Solve U**T *X = B, overwriting B with X.
*
         CALL STRSM( 'Left', 'Upper', 'Transpose', 'Non-unit', N, NRHS,
     $               ONE, A, LDA, B, LDB )
*
*        Solve L**T *X = B, overwriting B with X.
*
         CALL STRSM( 'Left', 'Lower', 'Transpose', 'Unit', N, NRHS, ONE,
     $               A, LDA, B, LDB )
*
*        Apply row interchanges to the solution vectors.
*
         CALL SLASWP( NRHS, B, LDB, 1, N, IPIV, -1 )
      END IF
*
      RETURN
*
*     End of SGETRS
*
      END
```

———-

33.33.4 lisp code

— **sgetrs.lisp** —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
```

```
(defun sgetrs (trans n nrhs a lda ipiv b ldb$ info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array single-float (*)) b a)
           (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (a single-float a-%data% a-%offset%)
       (b single-float b-%data% b-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((notran nil))
      (declare (type f2cl-lib:logical notran))
      (setf info 0)
      (setf notran
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "N")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val))
      (cond
       ((and (not notran)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "T")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val))
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "C")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -5))
       ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -8)))
      (cond
       ((/= info 0) (xerbla "SGETRS" (f2cl-lib:int-sub info))
        (go end_label)))
      (if (or (= n 0) (= nrhs 0))
          (go end_label))
      (cond
       (notran
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (slaswp nrhs b ldb$ 1 n ipiv 1)
          (declare (ignore var-1 var-3 var-5 var-6))
          (when var-0 (setf nrhs var-0))
```

```
    (when var-2 (setf ldb$ var-2))
    (when var-4 (setf n var-4)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (strsm "Left" "Lower" "No transpose" "Unit" n nrhs one a lda b
       ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (strsm "Left" "Upper" "No transpose" "Non-unit" n nrhs one a lda
       b ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (strsm "Left" "Upper" "Transpose" "Non-unit" n nrhs one a lda b
       ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (strsm "Left" "Lower" "Transpose" "Unit" n nrhs one a lda b ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaswp nrhs b ldb$ 1 n ipiv -1)
    (declare (ignore var-1 var-3 var-5 var-6))
    (when var-0 (setf nrhs var-0))
```

```
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf n var-4)))))
        (go end_label)
       end_label
        (return (values trans n nrhs nil lda nil nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgetrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

33.34 shgeqz

33.34.1 Axiom unit tests

— **shgeqz.input** —

```
)set break resume
)sys rm -f shgeqz.output
)spool shgeqz.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

33.34.2 Axiom help page [Mole71]

— **shgeqz.help** —

```
==========================================================================
shgeqz examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine shgeqz (JOB, COMPQ, COMPZ, N, ILO, IHI, H, LDH, T, LDT,
           ALPHAR, ALPHAI, BETA, Q, LDQ, Z, LDZ, WORK, LWORK, INFO)
           SHGEQZ

Function/Subroutine Documentation
   subroutine shgeqz (character JOB, character COMPQ, character COMPZ, integer
       N, integer ILO, integer IHI, real, dimension( ldh, * ) H, integer LDH,
       real, dimension( ldt, * ) T, integer LDT, real, dimension( * ) ALPHAR,
       real, dimension( * ) ALPHAI, real, dimension( * ) BETA, real,
       dimension( ldq, * ) Q, integer LDQ, real, dimension( ldz, * ) Z,
       integer LDZ, real, dimension( * ) WORK, integer LWORK, integer INFO)
       SHGEQZ

Purpose:
   SHGEQZ computes the eigenvalues of a real matrix pair (H,T),
   where H is an upper Hessenberg matrix and T is upper triangular,
   using the double-shift QZ method.
   Matrix pairs of this type are produced by the reduction to
   generalized upper Hessenberg form of a real matrix pair (A,B):

      A = Q1*H*Z1**T,  B = Q1*T*Z1**T,

   as computed by SGGHRD.

   If JOB='S', then the Hessenberg-triangular pair (H,T) is
   also reduced to generalized Schur form,

      H = Q*S*Z**T,  T = Q*P*Z**T,
```

```
   where Q and Z are orthogonal matrices, P is an upper triangular
   matrix, and S is a quasi-triangular matrix with 1-by-1 and 2-by-2
   diagonal blocks.

   The 1-by-1 blocks correspond to real eigenvalues of the matrix pair
   (H,T) and the 2-by-2 blocks correspond to complex conjugate pairs of
   eigenvalues.

   Additionally, the 2-by-2 upper triangular diagonal blocks of P
   corresponding to 2-by-2 blocks of S are reduced to positive diagonal
   form, i.e., if S(j+1,j) is non-zero, then P(j+1,j) = P(j,j+1) = 0,
   P(j,j) > 0, and P(j+1,j+1) > 0.

   Optionally, the orthogonal matrix Q from the generalized Schur
   factorization may be postmultiplied into an input matrix Q1, and the
   orthogonal matrix Z may be postmultiplied into an input matrix Z1.
   If Q1 and Z1 are the orthogonal matrices from SGGHRD that reduced
   the matrix pair (A,B) to generalized upper Hessenberg form, then the
   output matrices Q1*Q and Z1*Z are the orthogonal factors from the
   generalized Schur factorization of (A,B):

      A = (Q1*Q)*S*(Z1*Z)**T,  B = (Q1*Q)*P*(Z1*Z)**T.

   To avoid overflow, eigenvalues of the matrix pair (H,T) (equivalently,
   of (A,B)) are computed as a pair of values (alpha,beta), where alpha is
   complex and beta real.
   If beta is nonzero, lambda = alpha / beta is an eigenvalue of the
   generalized nonsymmetric eigenvalue problem (GNEP)
      A*x = lambda*B*x
   and if alpha is nonzero, mu = beta / alpha is an eigenvalue of the
   alternate form of the GNEP
      mu*A*y = B*y.
   Real eigenvalues can be read directly from the generalized Schur
   form:
      alpha = S(i,i), beta = P(i,i).

Parameters:

 JOB
   JOB is CHARACTER*1
   = 'E': Compute eigenvalues only;
   = 'S': Compute eigenvalues and the Schur form.

 COMPQ
   COMPQ is CHARACTER*1
   = 'N': Left Schur vectors (Q) are not computed;
   = 'I': Q is initialized to the unit matrix and the matrix Q
      of left Schur vectors of (H,T) is returned;
   = 'V': Q must contain an orthogonal matrix Q1 on entry and
```

```
     the product Q1*Q is returned.

COMPZ
  COMPZ is CHARACTER*1
  = 'N': Right Schur vectors (Z) are not computed;
  = 'I': Z is initialized to the unit matrix and the matrix Z
     of right Schur vectors of (H,T) is returned;
  = 'V': Z must contain an orthogonal matrix Z1 on entry and
     the product Z1*Z is returned.

N
  N is INTEGER
  The order of the matrices H, T, Q, and Z.  N >= 0.

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER
  ILO and IHI mark the rows and columns of H which are in
  Hessenberg form.  It is assumed that A is already upper
  triangular in rows and columns 1:ILO-1 and IHI+1:N.
  If N > 0, 1 <= ILO <= IHI <= N; if N = 0, ILO=1 and IHI=0.

H
  H is REAL array, dimension (LDH, N)
  On entry, the N-by-N upper Hessenberg matrix H.
  On exit, if JOB = 'S', H contains the upper quasi-triangular
  matrix S from the generalized Schur factorization.
  If JOB = 'E', the diagonal blocks of H match those of S, but
  the rest of H is unspecified.

LDH
  LDH is INTEGER
  The leading dimension of the array H.  LDH >= max( 1, N ).

T
  T is REAL array, dimension (LDT, N)
  On entry, the N-by-N upper triangular matrix T.
  On exit, if JOB = 'S', T contains the upper triangular
  matrix P from the generalized Schur factorization;
  2-by-2 diagonal blocks of P corresponding to 2-by-2 blocks of S
  are reduced to positive diagonal form, i.e., if H(j+1,j) is
  non-zero, then T(j+1,j) = T(j,j+1) = 0, T(j,j) > 0, and
  T(j+1,j+1) > 0.
  If JOB = 'E', the diagonal blocks of T match those of P, but
  the rest of T is unspecified.

LDT
  LDT is INTEGER
```

```
  The leading dimension of the array T.  LDT >= max( 1, N ).

ALPHAR
  ALPHAR is REAL array, dimension (N)
  The real parts of each scalar alpha defining an eigenvalue of GNEP.

ALPHAI
  ALPHAI is REAL array, dimension (N)
  The imaginary parts of each scalar alpha defining an
  eigenvalue of GNEP.
  If ALPHAI(j) is zero, then the j-th eigenvalue is real; if
  positive, then the j-th and (j+1)-st eigenvalues are a
  complex conjugate pair, with ALPHAI(j+1) = -ALPHAI(j).

BETA
  BETA is REAL array, dimension (N)
  The scalars beta that define the eigenvalues of GNEP.
  Together, the quantities alpha = (ALPHAR(j),ALPHAI(j)) and
  beta = BETA(j) represent the j-th eigenvalue of the matrix
  pair (A,B), in one of the forms lambda = alpha/beta or
  mu = beta/alpha.  Since either lambda or mu may overflow,
  they should not, in general, be computed.

Q
  Q is REAL array, dimension (LDQ, N)
  On entry, if COMPZ = 'V', the orthogonal matrix Q1 used in
  the reduction of (A,B) to generalized Hessenberg form.
  On exit, if COMPZ = 'I', the orthogonal matrix of left Schur
  vectors of (H,T), and if COMPZ = 'V', the orthogonal matrix
  of left Schur vectors of (A,B).
  Not referenced if COMPZ = 'N'.

LDQ
  LDQ is INTEGER
  The leading dimension of the array Q.  LDQ >= 1.
  If COMPQ='V' or 'I', then LDQ >= N.

Z
  Z is REAL array, dimension (LDZ, N)
  On entry, if COMPZ = 'V', the orthogonal matrix Z1 used in
  the reduction of (A,B) to generalized Hessenberg form.
  On exit, if COMPZ = 'I', the orthogonal matrix of
  right Schur vectors of (H,T), and if COMPZ = 'V', the
  orthogonal matrix of right Schur vectors of (A,B).
  Not referenced if COMPZ = 'N'.

LDZ
  LDZ is INTEGER
  The leading dimension of the array Z.  LDZ >= 1.
  If COMPZ='V' or 'I', then LDZ >= N.
```

```
 WORK
   WORK is REAL array, dimension (MAX(1,LWORK))
   On exit, if INFO >= 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.  LWORK >= max(1,N).

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value
   = 1,...,N: the QZ iteration did not converge.  (H,T) is not
      in Schur form, but ALPHAR(i), ALPHAI(i), and
      BETA(i), i=INFO+1,...,N should be correct.
   = N+1,...,2*N: the shift calculation failed.  (H,T) is not
      in Schur form, but ALPHAR(i), ALPHAI(i), and
      BETA(i), i=INFO-N+1,...,N should be correct.

Further Details:

 Iteration counters:

 JITER  -- counts iterations.
 IITER  -- counts iterations run since ILAST was last
           changed.  This is therefore reset only when a 1-by-1 or
           2-by-2 block deflates off the bottom.
```

———-

33.34.3 fortran code

— shgeqz.f —

```
*  ====================================================================
      SUBROUTINE SHGEQZ( JOB, COMPQ, COMPZ, N, ILO, IHI, H, LDH, T, LDT,
     $                   ALPHAR, ALPHAI, BETA, Q, LDQ, Z, LDZ, WORK,
     $                   LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
```

```
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER          COMPQ, COMPZ, JOB
      INTEGER            IHI, ILO, INFO, LDH, LDQ, LDT, LDZ, LWORK, N
*     ..
*     .. Array Arguments ..
      REAL               ALPHAI( * ), ALPHAR( * ), BETA( * ),
     $                   H( LDH, * ), Q( LDQ, * ), T( LDT, * ),
     $                   WORK( * ), Z( LDZ, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
*    $                     SAFETY = 1.0E+0 )
      REAL               HALF, ZERO, ONE, SAFETY
      PARAMETER          ( HALF = 0.5E+0, ZERO = 0.0E+0, ONE = 1.0E+0,
     $                   SAFETY = 1.0E+2 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ILAZR2, ILAZRO, ILPIVT, ILQ, ILSCHR, ILZ,
     $                   LQUERY
      INTEGER            ICOMPQ, ICOMPZ, IFIRST, IFRSTM, IITER, ILAST,
     $                   ILASTM, IN, ISCHUR, ISTART, J, JC, JCH, JITER,
     $                   JR, MAXIT
      REAL               A11, A12, A1I, A1R, A21, A22, A2I, A2R, AD11,
     $                   AD11L, AD12, AD12L, AD21, AD21L, AD22, AD22L,
     $                   AD32L, AN, ANORM, ASCALE, ATOL, B11, B1A, B1I,
     $                   B1R, B22, B2A, B2I, B2R, BN, BNORM, BSCALE,
     $                   BTOL, C, C11I, C11R, C12, C21, C22I, C22R, CL,
     $                   CQ, CR, CZ, ESHIFT, S, S1, S1INV, S2, SAFMAX,
     $                   SAFMIN, SCALE, SL, SQI, SQR, SR, SZI, SZR, T1,
     $                   TAU, TEMP, TEMP2, TEMPI, TEMPR, U1, U12, U12L,
     $                   U2, ULP, VS, W11, W12, W21, W22, WABS, WI, WR,
     $                   WR2
*     ..
*     .. Local Arrays ..
      REAL               V( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH, SLANHS, SLAPY2, SLAPY3
      EXTERNAL           LSAME, SLAMCH, SLANHS, SLAPY2, SLAPY3
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLAG2, SLARFG, SLARTG, SLASET, SLASV2, SROT,
     $                   XERBLA
*     ..
```

```
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, REAL, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode JOB, COMPQ, COMPZ
*
      IF( LSAME( JOB, 'E' ) ) THEN
         ILSCHR = .FALSE.
         ISCHUR = 1
      ELSE IF( LSAME( JOB, 'S' ) ) THEN
         ILSCHR = .TRUE.
         ISCHUR = 2
      ELSE
         ISCHUR = 0
      END IF
*
      IF( LSAME( COMPQ, 'N' ) ) THEN
         ILQ = .FALSE.
         ICOMPQ = 1
      ELSE IF( LSAME( COMPQ, 'V' ) ) THEN
         ILQ = .TRUE.
         ICOMPQ = 2
      ELSE IF( LSAME( COMPQ, 'I' ) ) THEN
         ILQ = .TRUE.
         ICOMPQ = 3
      ELSE
         ICOMPQ = 0
      END IF
*
      IF( LSAME( COMPZ, 'N' ) ) THEN
         ILZ = .FALSE.
         ICOMPZ = 1
      ELSE IF( LSAME( COMPZ, 'V' ) ) THEN
         ILZ = .TRUE.
         ICOMPZ = 2
      ELSE IF( LSAME( COMPZ, 'I' ) ) THEN
         ILZ = .TRUE.
         ICOMPZ = 3
      ELSE
         ICOMPZ = 0
      END IF
*
*     Check Argument Values
*
      INFO = 0
      WORK( 1 ) = MAX( 1, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( ISCHUR.EQ.0 ) THEN
         INFO = -1
```

```
      ELSE IF( ICOMPQ.EQ.0 ) THEN
         INFO = -2
      ELSE IF( ICOMPZ.EQ.0 ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( ILO.LT.1 ) THEN
         INFO = -5
      ELSE IF( IHI.GT.N .OR. IHI.LT.ILO-1 ) THEN
         INFO = -6
      ELSE IF( LDH.LT.N ) THEN
         INFO = -8
      ELSE IF( LDT.LT.N ) THEN
         INFO = -10
      ELSE IF( LDQ.LT.1 .OR. ( ILQ .AND. LDQ.LT.N ) ) THEN
         INFO = -15
      ELSE IF( LDZ.LT.1 .OR. ( ILZ .AND. LDZ.LT.N ) ) THEN
         INFO = -17
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -19
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SHGEQZ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.LE.0 ) THEN
         WORK( 1 ) = REAL( 1 )
         RETURN
      END IF
*
*     Initialize Q and Z
*
      IF( ICOMPQ.EQ.3 )
     $   CALL SLASET( 'Full', N, N, ZERO, ONE, Q, LDQ )
      IF( ICOMPZ.EQ.3 )
     $   CALL SLASET( 'Full', N, N, ZERO, ONE, Z, LDZ )
*
*     Machine Constants
*
      IN = IHI + 1 - ILO
      SAFMIN = SLAMCH( 'S' )
      SAFMAX = ONE / SAFMIN
      ULP = SLAMCH( 'E' )*SLAMCH( 'B' )
      ANORM = SLANHS( 'F', IN, H( ILO, ILO ), LDH, WORK )
      BNORM = SLANHS( 'F', IN, T( ILO, ILO ), LDT, WORK )
```

```
      ATOL = MAX( SAFMIN, ULP*ANORM )
      BTOL = MAX( SAFMIN, ULP*BNORM )
      ASCALE = ONE / MAX( SAFMIN, ANORM )
      BSCALE = ONE / MAX( SAFMIN, BNORM )
*
*     Set Eigenvalues IHI+1:N
*
      DO 30 J = IHI + 1, N
         IF( T( J, J ).LT.ZERO ) THEN
            IF( ILSCHR ) THEN
               DO 10 JR = 1, J
                  H( JR, J ) = -H( JR, J )
                  T( JR, J ) = -T( JR, J )
   10          CONTINUE
            ELSE
               H( J, J ) = -H( J, J )
               T( J, J ) = -T( J, J )
            END IF
            IF( ILZ ) THEN
               DO 20 JR = 1, N
                  Z( JR, J ) = -Z( JR, J )
   20          CONTINUE
            END IF
         END IF
         ALPHAR( J ) = H( J, J )
         ALPHAI( J ) = ZERO
         BETA( J ) = T( J, J )
   30 CONTINUE
*
*     If IHI < ILO, skip QZ steps
*
      IF( IHI.LT.ILO )
     $   GO TO 380
*
*     MAIN QZ ITERATION LOOP
*
*     Initialize dynamic indices
*
*     Eigenvalues ILAST+1:N have been found.
*        Column operations modify rows IFRSTM:whatever.
*        Row operations modify columns whatever:ILASTM.
*
*     If only eigenvalues are being computed, then
*        IFRSTM is the row of the last splitting row above row ILAST;
*        this is always at least ILO.
*     IITER counts iterations since the last eigenvalue was found,
*        to tell when to use an extraordinary shift.
*     MAXIT is the maximum number of QZ sweeps allowed.
*
      ILAST = IHI
```

```
      IF( ILSCHR ) THEN
         IFRSTM = 1
         ILASTM = N
      ELSE
         IFRSTM = ILO
         ILASTM = IHI
      END IF
      IITER = 0
      ESHIFT = ZERO
      MAXIT = 30*( IHI-ILO+1 )
*
      DO 360 JITER = 1, MAXIT
*
*        Split the matrix if possible.
*
*        Two tests:
*           1: H(j,j-1)=0  or  j=ILO
*           2: T(j,j)=0
*
         IF( ILAST.EQ.ILO ) THEN
*
*           Special case: j=ILAST
*
            GO TO 80
         ELSE
            IF( ABS( H( ILAST, ILAST-1 ) ).LE.ATOL ) THEN
               H( ILAST, ILAST-1 ) = ZERO
               GO TO 80
            END IF
         END IF
*
         IF( ABS( T( ILAST, ILAST ) ).LE.BTOL ) THEN
            T( ILAST, ILAST ) = ZERO
            GO TO 70
         END IF
*
*        General case: j<ILAST
*
         DO 60 J = ILAST - 1, ILO, -1
*
*           Test 1: for H(j,j-1)=0 or j=ILO
*
            IF( J.EQ.ILO ) THEN
               ILAZRO = .TRUE.
            ELSE
               IF( ABS( H( J, J-1 ) ).LE.ATOL ) THEN
                  H( J, J-1 ) = ZERO
                  ILAZRO = .TRUE.
               ELSE
                  ILAZRO = .FALSE.
```

```
            END IF
         END IF
*
*        Test 2: for T(j,j)=0
*
         IF( ABS( T( J, J ) ).LT.BTOL ) THEN
            T( J, J ) = ZERO
*
*           Test 1a: Check for 2 consecutive small subdiagonals in A
*
            ILAZR2 = .FALSE.
            IF( .NOT.ILAZRO ) THEN
               TEMP = ABS( H( J, J-1 ) )
               TEMP2 = ABS( H( J, J ) )
               TEMPR = MAX( TEMP, TEMP2 )
               IF( TEMPR.LT.ONE .AND. TEMPR.NE.ZERO ) THEN
                  TEMP = TEMP / TEMPR
                  TEMP2 = TEMP2 / TEMPR
               END IF
               IF( TEMP*( ASCALE*ABS( H( J+1, J ) ) ).LE.TEMP2*
     $             ( ASCALE*ATOL ) )ILAZR2 = .TRUE.
            END IF
*
*           If both tests pass (1 & 2), i.e., the leading diagonal
*           element of B in the block is zero, split a 1x1 block off
*           at the top. (I.e., at the J-th row/column) The leading
*           diagonal element of the remainder can also be zero, so
*           this may have to be done repeatedly.
*
            IF( ILAZRO .OR. ILAZR2 ) THEN
               DO 40 JCH = J, ILAST - 1
                  TEMP = H( JCH, JCH )
                  CALL SLARTG( TEMP, H( JCH+1, JCH ), C, S,
     $                         H( JCH, JCH ) )
                  H( JCH+1, JCH ) = ZERO
                  CALL SROT( ILASTM-JCH, H( JCH, JCH+1 ), LDH,
     $                       H( JCH+1, JCH+1 ), LDH, C, S )
                  CALL SROT( ILASTM-JCH, T( JCH, JCH+1 ), LDT,
     $                       T( JCH+1, JCH+1 ), LDT, C, S )
                  IF( ILQ )
     $               CALL SROT( N, Q( 1, JCH ), 1, Q( 1, JCH+1 ), 1,
     $                          C, S )
                  IF( ILAZR2 )
     $               H( JCH, JCH-1 ) = H( JCH, JCH-1 )*C
                  ILAZR2 = .FALSE.
                  IF( ABS( T( JCH+1, JCH+1 ) ).GE.BTOL ) THEN
                     IF( JCH+1.GE.ILAST ) THEN
                        GO TO 80
                     ELSE
                        IFIRST = JCH + 1
```

```
                        GO TO 110
                     END IF
                  END IF
                  T( JCH+1, JCH+1 ) = ZERO
   40          CONTINUE
               GO TO 70
            ELSE
*
*              Only test 2 passed -- chase the zero to T(ILAST,ILAST)
*              Then process as in the case T(ILAST,ILAST)=0
*
               DO 50 JCH = J, ILAST - 1
                  TEMP = T( JCH, JCH+1 )
                  CALL SLARTG( TEMP, T( JCH+1, JCH+1 ), C, S,
     $                         T( JCH, JCH+1 ) )
                  T( JCH+1, JCH+1 ) = ZERO
                  IF( JCH.LT.ILASTM-1 )
     $               CALL SROT( ILASTM-JCH-1, T( JCH, JCH+2 ), LDT,
     $                          T( JCH+1, JCH+2 ), LDT, C, S )
                  CALL SROT( ILASTM-JCH+2, H( JCH, JCH-1 ), LDH,
     $                       H( JCH+1, JCH-1 ), LDH, C, S )
                  IF( ILQ )
     $               CALL SROT( N, Q( 1, JCH ), 1, Q( 1, JCH+1 ), 1,
     $                          C, S )
                  TEMP = H( JCH+1, JCH )
                  CALL SLARTG( TEMP, H( JCH+1, JCH-1 ), C, S,
     $                         H( JCH+1, JCH ) )
                  H( JCH+1, JCH-1 ) = ZERO
                  CALL SROT( JCH+1-IFRSTM, H( IFRSTM, JCH ), 1,
     $                       H( IFRSTM, JCH-1 ), 1, C, S )
                  CALL SROT( JCH-IFRSTM, T( IFRSTM, JCH ), 1,
     $                       T( IFRSTM, JCH-1 ), 1, C, S )
                  IF( ILZ )
     $               CALL SROT( N, Z( 1, JCH ), 1, Z( 1, JCH-1 ), 1,
     $                          C, S )
   50          CONTINUE
               GO TO 70
            END IF
         ELSE IF( ILAZRO ) THEN
*
*           Only test 1 passed -- work on J:ILAST
*
            IFIRST = J
            GO TO 110
         END IF
*
*        Neither test passed -- try next J
*
   60    CONTINUE
*
```

```
*        (Drop-through is "impossible")
*
         INFO = N + 1
         GO TO 420
*
*        T(ILAST,ILAST)=0 -- clear H(ILAST,ILAST-1) to split off a
*        1x1 block.
*
   70    CONTINUE
         TEMP = H( ILAST, ILAST )
         CALL SLARTG( TEMP, H( ILAST, ILAST-1 ), C, S,
     $               H( ILAST, ILAST ) )
         H( ILAST, ILAST-1 ) = ZERO
         CALL SROT( ILAST-IFRSTM, H( IFRSTM, ILAST ), 1,
     $             H( IFRSTM, ILAST-1 ), 1, C, S )
         CALL SROT( ILAST-IFRSTM, T( IFRSTM, ILAST ), 1,
     $             T( IFRSTM, ILAST-1 ), 1, C, S )
         IF( ILZ )
     $      CALL SROT( N, Z( 1, ILAST ), 1, Z( 1, ILAST-1 ), 1, C, S )
*
*        H(ILAST,ILAST-1)=0 -- Standardize B, set ALPHAR, ALPHAI,
*                              and BETA
*
   80    CONTINUE
         IF( T( ILAST, ILAST ).LT.ZERO ) THEN
            IF( ILSCHR ) THEN
               DO 90 J = IFRSTM, ILAST
                  H( J, ILAST ) = -H( J, ILAST )
                  T( J, ILAST ) = -T( J, ILAST )
   90          CONTINUE
            ELSE
               H( ILAST, ILAST ) = -H( ILAST, ILAST )
               T( ILAST, ILAST ) = -T( ILAST, ILAST )
            END IF
            IF( ILZ ) THEN
               DO 100 J = 1, N
                  Z( J, ILAST ) = -Z( J, ILAST )
  100          CONTINUE
            END IF
         END IF
         ALPHAR( ILAST ) = H( ILAST, ILAST )
         ALPHAI( ILAST ) = ZERO
         BETA( ILAST ) = T( ILAST, ILAST )
*
*        Go to next block -- exit if finished.
*
         ILAST = ILAST - 1
         IF( ILAST.LT.ILO )
     $      GO TO 380
*
```

```
*        Reset counters
*
         IITER = 0
         ESHIFT = ZERO
         IF( .NOT.ILSCHR ) THEN
            ILASTM = ILAST
            IF( IFRSTM.GT.ILAST )
     $         IFRSTM = ILO
         END IF
         GO TO 350
*
*        QZ step
*
*        This iteration only involves rows/columns IFIRST:ILAST. We
*        assume IFIRST < ILAST, and that the diagonal of B is non-zero.
*
  110    CONTINUE
         IITER = IITER + 1
         IF( .NOT.ILSCHR ) THEN
            IFRSTM = IFIRST
         END IF
*
*        Compute single shifts.
*
*        At this point, IFIRST < ILAST, and the diagonal elements of
*        T(IFIRST:ILAST,IFIRST,ILAST) are larger than BTOL (in
*        magnitude)
*
         IF( ( IITER / 10 )*10.EQ.IITER ) THEN
*
*           Exceptional shift.  Chosen for no particularly good reason.
*           (Single shift only.)
*
            IF( ( REAL( MAXIT )*SAFMIN )*ABS( H( ILAST, ILAST-1 ) ).LT.
     $          ABS( T( ILAST-1, ILAST-1 ) ) ) THEN
               ESHIFT = H( ILAST, ILAST-1 ) /
     $                  T( ILAST-1, ILAST-1 )
            ELSE
               ESHIFT = ESHIFT + ONE / ( SAFMIN*REAL( MAXIT ) )
            END IF
            S1 = ONE
            WR = ESHIFT
*
         ELSE
*
*           Shifts based on the generalized eigenvalues of the
*           bottom-right 2x2 block of A and B. The first eigenvalue
*           returned by SLAG2 is the Wilkinson shift (AEP p.512),
*
            CALL SLAG2( H( ILAST-1, ILAST-1 ), LDH,
```

```
     $                 T( ILAST-1, ILAST-1 ), LDT, SAFMIN*SAFETY, S1,
     $                 S2, WR, WR2, WI )
*
            IF ( ABS( (WR/S1)*T( ILAST, ILAST ) - H( ILAST, ILAST ) )
     $         .GT. ABS( (WR2/S2)*T( ILAST, ILAST )
     $         - H( ILAST, ILAST ) ) ) THEN
               TEMP = WR
               WR = WR2
               WR2 = TEMP
               TEMP = S1
               S1 = S2
               S2 = TEMP
            END IF
            TEMP = MAX( S1, SAFMIN*MAX( ONE, ABS( WR ), ABS( WI ) ) )
            IF( WI.NE.ZERO )
     $         GO TO 200
         END IF
*
*        Fiddle with shift to avoid overflow
*
         TEMP = MIN( ASCALE, ONE )*( HALF*SAFMAX )
         IF( S1.GT.TEMP ) THEN
            SCALE = TEMP / S1
         ELSE
            SCALE = ONE
         END IF
*
         TEMP = MIN( BSCALE, ONE )*( HALF*SAFMAX )
         IF( ABS( WR ).GT.TEMP )
     $      SCALE = MIN( SCALE, TEMP / ABS( WR ) )
         S1 = SCALE*S1
         WR = SCALE*WR
*
*        Now check for two consecutive small subdiagonals.
*
         DO 120 J = ILAST - 1, IFIRST + 1, -1
            ISTART = J
            TEMP = ABS( S1*H( J, J-1 ) )
            TEMP2 = ABS( S1*H( J, J )-WR*T( J, J ) )
            TEMPR = MAX( TEMP, TEMP2 )
            IF( TEMPR.LT.ONE .AND. TEMPR.NE.ZERO ) THEN
               TEMP = TEMP / TEMPR
               TEMP2 = TEMP2 / TEMPR
            END IF
            IF( ABS( ( ASCALE*H( J+1, J ) )*TEMP ).LE.( ASCALE*ATOL )*
     $          TEMP2 )GO TO 130
  120    CONTINUE
*
         ISTART = IFIRST
  130    CONTINUE
```

```
*
*        Do an implicit single-shift QZ sweep.
*
*        Initial Q
*
         TEMP = S1*H( ISTART, ISTART ) - WR*T( ISTART, ISTART )
         TEMP2 = S1*H( ISTART+1, ISTART )
         CALL SLARTG( TEMP, TEMP2, C, S, TEMPR )
*
*        Sweep
*
         DO 190 J = ISTART, ILAST - 1
            IF( J.GT.ISTART ) THEN
               TEMP = H( J, J-1 )
               CALL SLARTG( TEMP, H( J+1, J-1 ), C, S, H( J, J-1 ) )
               H( J+1, J-1 ) = ZERO
            END IF
*
            DO 140 JC = J, ILASTM
               TEMP = C*H( J, JC ) + S*H( J+1, JC )
               H( J+1, JC ) = -S*H( J, JC ) + C*H( J+1, JC )
               H( J, JC ) = TEMP
               TEMP2 = C*T( J, JC ) + S*T( J+1, JC )
               T( J+1, JC ) = -S*T( J, JC ) + C*T( J+1, JC )
               T( J, JC ) = TEMP2
  140       CONTINUE
            IF( ILQ ) THEN
               DO 150 JR = 1, N
                  TEMP = C*Q( JR, J ) + S*Q( JR, J+1 )
                  Q( JR, J+1 ) = -S*Q( JR, J ) + C*Q( JR, J+1 )
                  Q( JR, J ) = TEMP
  150          CONTINUE
            END IF
*
            TEMP = T( J+1, J+1 )
            CALL SLARTG( TEMP, T( J+1, J ), C, S, T( J+1, J+1 ) )
            T( J+1, J ) = ZERO
*
            DO 160 JR = IFRSTM, MIN( J+2, ILAST )
               TEMP = C*H( JR, J+1 ) + S*H( JR, J )
               H( JR, J ) = -S*H( JR, J+1 ) + C*H( JR, J )
               H( JR, J+1 ) = TEMP
  160       CONTINUE
            DO 170 JR = IFRSTM, J
               TEMP = C*T( JR, J+1 ) + S*T( JR, J )
               T( JR, J ) = -S*T( JR, J+1 ) + C*T( JR, J )
               T( JR, J+1 ) = TEMP
  170       CONTINUE
            IF( ILZ ) THEN
               DO 180 JR = 1, N
```

```
                 TEMP = C*Z( JR, J+1 ) + S*Z( JR, J )
                 Z( JR, J ) = -S*Z( JR, J+1 ) + C*Z( JR, J )
                 Z( JR, J+1 ) = TEMP
  180         CONTINUE
           END IF
  190    CONTINUE
*
         GO TO 350
*
*        Use Francis double-shift
*
*        Note: the Francis double-shift should work with real shifts,
*              but only if the block is at least 3x3.
*              This code may break if this point is reached with
*              a 2x2 block with real eigenvalues.
*
  200    CONTINUE
         IF( IFIRST+1.EQ.ILAST ) THEN
*
*          Special case -- 2x2 block with complex eigenvectors
*
*          Step 1: Standardize, that is, rotate so that
*
*                      ( B11  0  )
*                  B = (         )  with B11 non-negative.
*                      (  0  B22 )
*
           CALL SLASV2( T( ILAST-1, ILAST-1 ), T( ILAST-1, ILAST ),
     $                  T( ILAST, ILAST ), B22, B11, SR, CR, SL, CL )
*
           IF( B11.LT.ZERO ) THEN
              CR = -CR
              SR = -SR
              B11 = -B11
              B22 = -B22
           END IF
*
           CALL SROT( ILASTM+1-IFIRST, H( ILAST-1, ILAST-1 ), LDH,
     $                H( ILAST, ILAST-1 ), LDH, CL, SL )
           CALL SROT( ILAST+1-IFRSTM, H( IFRSTM, ILAST-1 ), 1,
     $                H( IFRSTM, ILAST ), 1, CR, SR )
*
           IF( ILAST.LT.ILASTM )
     $        CALL SROT( ILASTM-ILAST, T( ILAST-1, ILAST+1 ), LDT,
     $                   T( ILAST, ILAST+1 ), LDT, CL, SL )
           IF( IFRSTM.LT.ILAST-1 )
     $        CALL SROT( IFIRST-IFRSTM, T( IFRSTM, ILAST-1 ), 1,
     $                   T( IFRSTM, ILAST ), 1, CR, SR )
*
           IF( ILQ )
```

```
     $         CALL SROT( N, Q( 1, ILAST-1 ), 1, Q( 1, ILAST ), 1, CL,
     $                    SL )
           IF( ILZ )
     $         CALL SROT( N, Z( 1, ILAST-1 ), 1, Z( 1, ILAST ), 1, CR,
     $                    SR )
*
           T( ILAST-1, ILAST-1 ) = B11
           T( ILAST-1, ILAST ) = ZERO
           T( ILAST, ILAST-1 ) = ZERO
           T( ILAST, ILAST ) = B22
*
*          If B22 is negative, negate column ILAST
*
           IF( B22.LT.ZERO ) THEN
              DO 210 J = IFRSTM, ILAST
                 H( J, ILAST ) = -H( J, ILAST )
                 T( J, ILAST ) = -T( J, ILAST )
  210         CONTINUE
*
              IF( ILZ ) THEN
                 DO 220 J = 1, N
                    Z( J, ILAST ) = -Z( J, ILAST )
  220            CONTINUE
              END IF
              B22 = -B22
           END IF
*
*          Step 2: Compute ALPHAR, ALPHAI, and BETA (see refs.)
*
*          Recompute shift
*
           CALL SLAG2( H( ILAST-1, ILAST-1 ), LDH,
     $                 T( ILAST-1, ILAST-1 ), LDT, SAFMIN*SAFETY, S1,
     $                 TEMP, WR, TEMP2, WI )
*
*          If standardization has perturbed the shift onto real line,
*          do another (real single-shift) QR step.
*
           IF( WI.EQ.ZERO )
     $         GO TO 350
           S1INV = ONE / S1
*
*          Do EISPACK (QZVAL) computation of alpha and beta
*
           A11 = H( ILAST-1, ILAST-1 )
           A21 = H( ILAST, ILAST-1 )
           A12 = H( ILAST-1, ILAST )
           A22 = H( ILAST, ILAST )
*
*          Compute complex Givens rotation on right
```

```
*          (Assume some element of C = (sA - wB) > unfl )
*                                --
*          (sA - wB) ( CZ   -SZ )
*                    ( SZ    CZ )
*
           C11R = S1*A11 - WR*B11
           C11I = -WI*B11
           C12 = S1*A12
           C21 = S1*A21
           C22R = S1*A22 - WR*B22
           C22I = -WI*B22
*
           IF( ABS( C11R )+ABS( C11I )+ABS( C12 ).GT.ABS( C21 )+
     $         ABS( C22R )+ABS( C22I ) ) THEN
              T1 = SLAPY3( C12, C11R, C11I )
              CZ = C12 / T1
              SZR = -C11R / T1
              SZI = -C11I / T1
           ELSE
              CZ = SLAPY2( C22R, C22I )
              IF( CZ.LE.SAFMIN ) THEN
                 CZ = ZERO
                 SZR = ONE
                 SZI = ZERO
              ELSE
                 TEMPR = C22R / CZ
                 TEMPI = C22I / CZ
                 T1 = SLAPY2( CZ, C21 )
                 CZ = CZ / T1
                 SZR = -C21*TEMPR / T1
                 SZI = C21*TEMPI / T1
              END IF
           END IF
*
*          Compute Givens rotation on left
*
*          (  CQ   SQ )
*          (  __      )  A or B
*          ( -SQ   CQ )
*
           AN = ABS( A11 ) + ABS( A12 ) + ABS( A21 ) + ABS( A22 )
           BN = ABS( B11 ) + ABS( B22 )
           WABS = ABS( WR ) + ABS( WI )
           IF( S1*AN.GT.WABS*BN ) THEN
              CQ = CZ*B11
              SQR = SZR*B22
              SQI = -SZI*B22
           ELSE
              A1R = CZ*A11 + SZR*A12
              A1I = SZI*A12
```

```
               A2R = CZ*A21 + SZR*A22
               A2I = SZI*A22
               CQ = SLAPY2( A1R, A1I )
               IF( CQ.LE.SAFMIN ) THEN
                  CQ = ZERO
                  SQR = ONE
                  SQI = ZERO
               ELSE
                  TEMPR = A1R / CQ
                  TEMPI = A1I / CQ
                  SQR = TEMPR*A2R + TEMPI*A2I
                  SQI = TEMPI*A2R - TEMPR*A2I
               END IF
            END IF
            T1 = SLAPY3( CQ, SQR, SQI )
            CQ = CQ / T1
            SQR = SQR / T1
            SQI = SQI / T1
*
*           Compute diagonal elements of QBZ
*
            TEMPR = SQR*SZR - SQI*SZI
            TEMPI = SQR*SZI + SQI*SZR
            B1R = CQ*CZ*B11 + TEMPR*B22
            B1I = TEMPI*B22
            B1A = SLAPY2( B1R, B1I )
            B2R = CQ*CZ*B22 + TEMPR*B11
            B2I = -TEMPI*B11
            B2A = SLAPY2( B2R, B2I )
*
*           Normalize so beta > 0, and Im( alpha1 ) > 0
*
            BETA( ILAST-1 ) = B1A
            BETA( ILAST ) = B2A
            ALPHAR( ILAST-1 ) = ( WR*B1A )*S1INV
            ALPHAI( ILAST-1 ) = ( WI*B1A )*S1INV
            ALPHAR( ILAST ) = ( WR*B2A )*S1INV
            ALPHAI( ILAST ) = -( WI*B2A )*S1INV
*
*           Step 3: Go to next block -- exit if finished.
*
            ILAST = IFIRST - 1
            IF( ILAST.LT.ILO )
     $         GO TO 380
*
*           Reset counters
*
            IITER = 0
            ESHIFT = ZERO
            IF( .NOT.ILSCHR ) THEN
```

```
               ILASTM = ILAST
               IF( IFRSTM.GT.ILAST )
     $            IFRSTM = ILO
            END IF
            GO TO 350
         ELSE
*
*           Usual case: 3x3 or larger block, using Francis implicit
*                       double-shift
*
*                                     2
*           Eigenvalue equation is  w  - c w + d = 0,
*
*                                         -1 2        -1
*           so compute 1st column of  (A B  )  - c A B   + d
*           using the formula in QZIT (from EISPACK)
*
*           We assume that the block is at least 3x3
*
            AD11 = ( ASCALE*H( ILAST-1, ILAST-1 ) ) /
     $             ( BSCALE*T( ILAST-1, ILAST-1 ) )
            AD21 = ( ASCALE*H( ILAST, ILAST-1 ) ) /
     $             ( BSCALE*T( ILAST-1, ILAST-1 ) )
            AD12 = ( ASCALE*H( ILAST-1, ILAST ) ) /
     $             ( BSCALE*T( ILAST, ILAST ) )
            AD22 = ( ASCALE*H( ILAST, ILAST ) ) /
     $             ( BSCALE*T( ILAST, ILAST ) )
            U12 = T( ILAST-1, ILAST ) / T( ILAST, ILAST )
            AD11L = ( ASCALE*H( IFIRST, IFIRST ) ) /
     $              ( BSCALE*T( IFIRST, IFIRST ) )
            AD21L = ( ASCALE*H( IFIRST+1, IFIRST ) ) /
     $              ( BSCALE*T( IFIRST, IFIRST ) )
            AD12L = ( ASCALE*H( IFIRST, IFIRST+1 ) ) /
     $              ( BSCALE*T( IFIRST+1, IFIRST+1 ) )
            AD22L = ( ASCALE*H( IFIRST+1, IFIRST+1 ) ) /
     $              ( BSCALE*T( IFIRST+1, IFIRST+1 ) )
            AD32L = ( ASCALE*H( IFIRST+2, IFIRST+1 ) ) /
     $              ( BSCALE*T( IFIRST+1, IFIRST+1 ) )
            U12L = T( IFIRST, IFIRST+1 ) / T( IFIRST+1, IFIRST+1 )
*
            V( 1 ) = ( AD11-AD11L )*( AD22-AD11L ) - AD12*AD21 +
     $               AD21*U12*AD11L + ( AD12L-AD11L*U12L )*AD21L
            V( 2 ) = ( ( AD22L-AD11L )-AD21L*U12L-( AD11-AD11L )-
     $               ( AD22-AD11L )+AD21*U12 )*AD21L
            V( 3 ) = AD32L*AD21L
*
            ISTART = IFIRST
*
            CALL SLARFG( 3, V( 1 ), V( 2 ), 1, TAU )
            V( 1 ) = ONE
```

```
*
*          Sweep
*
           DO 290 J = ISTART, ILAST - 2
*
*             All but last elements: use 3x3 Householder transforms.
*
*             Zero (j-1)st column of A
*
              IF( J.GT.ISTART ) THEN
                 V( 1 ) = H( J, J-1 )
                 V( 2 ) = H( J+1, J-1 )
                 V( 3 ) = H( J+2, J-1 )
*
                 CALL SLARFG( 3, H( J, J-1 ), V( 2 ), 1, TAU )
                 V( 1 ) = ONE
                 H( J+1, J-1 ) = ZERO
                 H( J+2, J-1 ) = ZERO
              END IF
*
              DO 230 JC = J, ILASTM
                 TEMP = TAU*( H( J, JC )+V( 2 )*H( J+1, JC )+V( 3 )*
     $                  H( J+2, JC ) )
                 H( J, JC ) = H( J, JC ) - TEMP
                 H( J+1, JC ) = H( J+1, JC ) - TEMP*V( 2 )
                 H( J+2, JC ) = H( J+2, JC ) - TEMP*V( 3 )
                 TEMP2 = TAU*( T( J, JC )+V( 2 )*T( J+1, JC )+V( 3 )*
     $                   T( J+2, JC ) )
                 T( J, JC ) = T( J, JC ) - TEMP2
                 T( J+1, JC ) = T( J+1, JC ) - TEMP2*V( 2 )
                 T( J+2, JC ) = T( J+2, JC ) - TEMP2*V( 3 )
  230         CONTINUE
              IF( ILQ ) THEN
                 DO 240 JR = 1, N
                    TEMP = TAU*( Q( JR, J )+V( 2 )*Q( JR, J+1 )+V( 3 )*
     $                      Q( JR, J+2 ) )
                    Q( JR, J ) = Q( JR, J ) - TEMP
                    Q( JR, J+1 ) = Q( JR, J+1 ) - TEMP*V( 2 )
                    Q( JR, J+2 ) = Q( JR, J+2 ) - TEMP*V( 3 )
  240            CONTINUE
              END IF
*
*             Zero j-th column of B (see SLAGBC for details)
*
*             Swap rows to pivot
*
              ILPIVT = .FALSE.
              TEMP = MAX( ABS( T( J+1, J+1 ) ), ABS( T( J+1, J+2 ) ) )
              TEMP2 = MAX( ABS( T( J+2, J+1 ) ), ABS( T( J+2, J+2 ) ) )
              IF( MAX( TEMP, TEMP2 ).LT.SAFMIN ) THEN
```

```
                 SCALE = ZERO
                 U1 = ONE
                 U2 = ZERO
                 GO TO 250
              ELSE IF( TEMP.GE.TEMP2 ) THEN
                 W11 = T( J+1, J+1 )
                 W21 = T( J+2, J+1 )
                 W12 = T( J+1, J+2 )
                 W22 = T( J+2, J+2 )
                 U1 = T( J+1, J )
                 U2 = T( J+2, J )
              ELSE
                 W21 = T( J+1, J+1 )
                 W11 = T( J+2, J+1 )
                 W22 = T( J+1, J+2 )
                 W12 = T( J+2, J+2 )
                 U2 = T( J+1, J )
                 U1 = T( J+2, J )
              END IF
*
*             Swap columns if nec.
*
              IF( ABS( W12 ).GT.ABS( W11 ) ) THEN
                 ILPIVT = .TRUE.
                 TEMP = W12
                 TEMP2 = W22
                 W12 = W11
                 W22 = W21
                 W11 = TEMP
                 W21 = TEMP2
              END IF
*
*             LU-factor
*
              TEMP = W21 / W11
              U2 = U2 - TEMP*U1
              W22 = W22 - TEMP*W12
              W21 = ZERO
*
*             Compute SCALE
*
              SCALE = ONE
              IF( ABS( W22 ).LT.SAFMIN ) THEN
                 SCALE = ZERO
                 U2 = ONE
                 U1 = -W12 / W11
                 GO TO 250
              END IF
              IF( ABS( W22 ).LT.ABS( U2 ) )
     $           SCALE = ABS( W22 / U2 )
```

```
               IF( ABS( W11 ).LT.ABS( U1 ) )
     $            SCALE = MIN( SCALE, ABS( W11 / U1 ) )
*
*              Solve
*
               U2 = ( SCALE*U2 ) / W22
               U1 = ( SCALE*U1-W12*U2 ) / W11
*
  250          CONTINUE
               IF( ILPIVT ) THEN
                  TEMP = U2
                  U2 = U1
                  U1 = TEMP
               END IF
*
*              Compute Householder Vector
*
               T1 = SQRT( SCALE**2+U1**2+U2**2 )
               TAU = ONE + SCALE / T1
               VS = -ONE / ( SCALE+T1 )
               V( 1 ) = ONE
               V( 2 ) = VS*U1
               V( 3 ) = VS*U2
*
*              Apply transformations from the right.
*
               DO 260 JR = IFRSTM, MIN( J+3, ILAST )
                  TEMP = TAU*( H( JR, J )+V( 2 )*H( JR, J+1 )+V( 3 )*
     $                   H( JR, J+2 ) )
                  H( JR, J ) = H( JR, J ) - TEMP
                  H( JR, J+1 ) = H( JR, J+1 ) - TEMP*V( 2 )
                  H( JR, J+2 ) = H( JR, J+2 ) - TEMP*V( 3 )
  260          CONTINUE
               DO 270 JR = IFRSTM, J + 2
                  TEMP = TAU*( T( JR, J )+V( 2 )*T( JR, J+1 )+V( 3 )*
     $                   T( JR, J+2 ) )
                  T( JR, J ) = T( JR, J ) - TEMP
                  T( JR, J+1 ) = T( JR, J+1 ) - TEMP*V( 2 )
                  T( JR, J+2 ) = T( JR, J+2 ) - TEMP*V( 3 )
  270          CONTINUE
               IF( ILZ ) THEN
                  DO 280 JR = 1, N
                     TEMP = TAU*( Z( JR, J )+V( 2 )*Z( JR, J+1 )+V( 3 )*
     $                      Z( JR, J+2 ) )
                     Z( JR, J ) = Z( JR, J ) - TEMP
                     Z( JR, J+1 ) = Z( JR, J+1 ) - TEMP*V( 2 )
                     Z( JR, J+2 ) = Z( JR, J+2 ) - TEMP*V( 3 )
  280             CONTINUE
               END IF
               T( J+1, J ) = ZERO
```

```
               T( J+2, J ) = ZERO
  290       CONTINUE
*
*           Last elements: Use Givens rotations
*
*           Rotations from the left
*
            J = ILAST - 1
            TEMP = H( J, J-1 )
            CALL SLARTG( TEMP, H( J+1, J-1 ), C, S, H( J, J-1 ) )
            H( J+1, J-1 ) = ZERO
*
            DO 300 JC = J, ILASTM
               TEMP = C*H( J, JC ) + S*H( J+1, JC )
               H( J+1, JC ) = -S*H( J, JC ) + C*H( J+1, JC )
               H( J, JC ) = TEMP
               TEMP2 = C*T( J, JC ) + S*T( J+1, JC )
               T( J+1, JC ) = -S*T( J, JC ) + C*T( J+1, JC )
               T( J, JC ) = TEMP2
  300       CONTINUE
            IF( ILQ ) THEN
               DO 310 JR = 1, N
                  TEMP = C*Q( JR, J ) + S*Q( JR, J+1 )
                  Q( JR, J+1 ) = -S*Q( JR, J ) + C*Q( JR, J+1 )
                  Q( JR, J ) = TEMP
  310          CONTINUE
            END IF
*
*           Rotations from the right.
*
            TEMP = T( J+1, J+1 )
            CALL SLARTG( TEMP, T( J+1, J ), C, S, T( J+1, J+1 ) )
            T( J+1, J ) = ZERO
*
            DO 320 JR = IFRSTM, ILAST
               TEMP = C*H( JR, J+1 ) + S*H( JR, J )
               H( JR, J ) = -S*H( JR, J+1 ) + C*H( JR, J )
               H( JR, J+1 ) = TEMP
  320       CONTINUE
            DO 330 JR = IFRSTM, ILAST - 1
               TEMP = C*T( JR, J+1 ) + S*T( JR, J )
               T( JR, J ) = -S*T( JR, J+1 ) + C*T( JR, J )
               T( JR, J+1 ) = TEMP
  330       CONTINUE
            IF( ILZ ) THEN
               DO 340 JR = 1, N
                  TEMP = C*Z( JR, J+1 ) + S*Z( JR, J )
                  Z( JR, J ) = -S*Z( JR, J+1 ) + C*Z( JR, J )
                  Z( JR, J+1 ) = TEMP
  340          CONTINUE
```

```
            END IF
*
*           End of Double-Shift code
*
         END IF
*
         GO TO 350
*
*        End of iteration loop
*
  350    CONTINUE
  360 CONTINUE
*
*     Drop-through = non-convergence
*
      INFO = ILAST
      GO TO 420
*
*     Successful completion of all QZ steps
*
  380 CONTINUE
*
*     Set Eigenvalues 1:ILO-1
*
      DO 410 J = 1, ILO - 1
         IF( T( J, J ).LT.ZERO ) THEN
            IF( ILSCHR ) THEN
               DO 390 JR = 1, J
                  H( JR, J ) = -H( JR, J )
                  T( JR, J ) = -T( JR, J )
  390          CONTINUE
            ELSE
               H( J, J ) = -H( J, J )
               T( J, J ) = -T( J, J )
            END IF
            IF( ILZ ) THEN
               DO 400 JR = 1, N
                  Z( JR, J ) = -Z( JR, J )
  400          CONTINUE
            END IF
         END IF
         ALPHAR( J ) = H( J, J )
         ALPHAI( J ) = ZERO
         BETA( J ) = T( J, J )
  410 CONTINUE
*
*     Normal Termination
*
      INFO = 0
*
```

```
*     Exit (other than argument error) -- return optimal workspace size
*
  420 CONTINUE
      WORK( 1 ) = REAL( N )
      RETURN
*
*     End of SHGEQZ
*
      END
```

———-

33.34.4 lisp code

— **shgeqz.lisp** —

```
(let* ((half 0.5) (zero 0.0) (one 1.0) (safety 100.0))
  (declare (type (single-float 0.5 0.5) half)
           (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 100.0 100.0) safety)
           (ignorable half zero one safety))
  (defun shgeqz
         (job compq compz n ilo ihi h ldh t$ ldt alphar alphai beta q ldq z ldz
          work lwork info)
    (declare (type (array single-float (*)) work z q beta alphai alphar t$ h)
             (type (f2cl-lib:integer4) info lwork ldz ldq ldt ldh ihi ilo n)
             (type (string *) compz compq job))
    (f2cl-lib:with-multi-array-data
        ((job character job-%data% job-%offset%)
         (compq character compq-%data% compq-%offset%)
         (compz character compz-%data% compz-%offset%)
         (h single-float h-%data% h-%offset%)
         (t$ single-float t$-%data% t$-%offset%)
         (alphar single-float alphar-%data% alphar-%offset%)
         (alphai single-float alphai-%data% alphai-%offset%)
         (beta single-float beta-%data% beta-%offset%)
         (q single-float q-%data% q-%offset%)
         (z single-float z-%data% z-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((v (make-array 3 :element-type 'single-float)) (a11 0.0) (a12 0.0)
             (a1i 0.0) (a1r 0.0) (a21 0.0) (a22 0.0) (a2i 0.0) (a2r 0.0)
             (ad11 0.0) (ad11l 0.0) (ad12 0.0) (ad12l 0.0) (ad21 0.0)
             (ad21l 0.0) (ad22 0.0) (ad22l 0.0) (ad32l 0.0) (an 0.0)
             (anorm 0.0) (ascale 0.0) (atol 0.0) (b11 0.0) (b1a 0.0) (b1i 0.0)
             (b1r 0.0) (b22 0.0) (b2a 0.0) (b2i 0.0) (b2r 0.0) (bn 0.0)
             (bnorm 0.0) (bscale 0.0) (btol 0.0) (c 0.0) (c11i 0.0) (c11r 0.0)
             (c12 0.0) (c21 0.0) (c22i 0.0) (c22r 0.0) (cl 0.0) (cq 0.0)
```

```
     (cr 0.0) (cz 0.0) (eshift 0.0) (s 0.0) (s1 0.0) (s1inv 0.0)
     (s2 0.0) (safmax 0.0) (safmin 0.0) (scale 0.0) (sl 0.0) (sqi 0.0)
     (sqr 0.0) (sr 0.0) (szi 0.0) (szr 0.0) (t1 0.0) (tau 0.0)
     (temp 0.0) (temp2 0.0) (tempi 0.0) (tempr 0.0) (u1 0.0) (u12 0.0)
     (u12l 0.0) (u2 0.0) (ulp 0.0) (vs 0.0) (w11 0.0) (w12 0.0)
     (w21 0.0) (w22 0.0) (wabs 0.0) (wi 0.0) (wr 0.0) (wr2 0.0)
     (icompq 0) (icompz 0) (ifirst 0) (ifrstm 0) (iiter 0) (ilast 0)
     (ilastm 0) (in 0) (ischur 0) (istart 0) (j 0) (jc 0) (jch 0)
     (jiter 0) (jr 0) (maxit 0) (ilazr2 nil) (ilazro nil) (ilpivt nil)
     (ilq nil) (ilschr nil) (ilz nil) (lquery nil))
(declare (type (array single-float (3)) v)
         (type (single-float) a11 a12 a1i a1r a21 a22 a2i a2r ad11
          ad11l ad12 ad12l ad21 ad21l ad22 ad22l ad32l an anorm ascale
          atol b11 b1a b1i b1r b22 b2a b2i b2r bn bnorm bscale btol c
          c11i c11r c12 c21 c22i c22r cl cq cr cz eshift s s1 s1inv s2
          safmax safmin scale sl sqi sqr sr szi szr t1 tau temp temp2
          tempi tempr u1 u12 u12l u2 ulp vs w11 w12 w21 w22 wabs wi wr
          wr2)
         (type (f2cl-lib:integer4) icompq icompz ifirst ifrstm iiter
          ilast ilastm in ischur istart j jc jch jiter jr maxit)
         (type f2cl-lib:logical ilazr2 ilazro ilpivt ilq ilschr ilz
          lquery))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame job "E")
    (declare (ignore var-1))
    (when var-0 (setf job var-0))
    ret-val)
  (setf ilschr f2cl-lib:%false%) (setf ischur 1))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame job "S")
    (declare (ignore var-1))
    (when var-0 (setf job var-0))
    ret-val)
  (setf ilschr f2cl-lib:%true%) (setf ischur 2))
 (t (setf ischur 0)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compq "N")
    (declare (ignore var-1))
    (when var-0 (setf compq var-0))
    ret-val)
  (setf ilq f2cl-lib:%false%) (setf icompq 1))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compq "V")
    (declare (ignore var-1))
    (when var-0 (setf compq var-0))
    ret-val)
  (setf ilq f2cl-lib:%true%) (setf icompq 2))
 ((multiple-value-bind (ret-val var-0 var-1)
```

```
      (lsame compq "I")
    (declare (ignore var-1))
    (when var-0 (setf compq var-0))
    ret-val)
  (setf ilq f2cl-lib:%true%) (setf icompq 3))
 (t (setf icompq 0)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "N")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf ilz f2cl-lib:%false%) (setf icompz 1))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "V")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf ilz f2cl-lib:%true%) (setf icompz 2))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "I")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf ilz f2cl-lib:%true%) (setf icompz 3))
 (t (setf icompz 0)))
(setf info 0)
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce
         (the f2cl-lib:integer4
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 n)))
         'single-float))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((= ischur 0) (setf info -1)) ((= icompq 0) (setf info -2))
      ((= icompz 0) (setf info -3)) ((< n 0) (setf info -4))
      ((< ilo 1) (setf info -5))
      ((or (> ihi n)
           (< ihi (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))))
       (setf info -6))
      ((< ldh n) (setf info -8)) ((< ldt n) (setf info -10))
      ((or (< ldq 1) (and ilq (< ldq n))) (setf info -15))
      ((or (< ldz 1) (and ilz (< ldz n))) (setf info -17))
      ((and
        (< lwork
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (not lquery))
       (setf info -19)))
(cond
 ((/= info 0) (xerbla "SHGEQZ" (f2cl-lib:int-sub info)) (go end_label))
```

```
 (lquery (go end_label)))
(cond
 ((<= n 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:freal 1))
  (go end_label)))
(if (= icompq 3)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "Full" n n zero one q ldq)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldq var-6))))
(if (= icompz 3)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "Full" n n zero one z ldz)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldz var-6))))
(setf in (f2cl-lib:int-sub (f2cl-lib:int-add ihi 1) ilo))
(setf safmin (slamch "S"))
(setf safmax (/ one safmin))
(setf ulp (* (slamch "E") (slamch "B")))
(setf anorm
        (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
            (slanhs "F" in
             (f2cl-lib:array-slice h-%data% single-float (ilo ilo)
                                   ((1 ldh) (1 *)) h-%offset%)
             ldh work)
          (declare (ignore var-0 var-2 var-4))
          (when var-1 (setf in var-1))
          (when var-3 (setf ldh var-3))
          ret-val))
(setf bnorm
        (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
            (slanhs "F" in
             (f2cl-lib:array-slice t$-%data% single-float (ilo ilo)
                                   ((1 ldt) (1 *)) t$-%offset%)
             ldt work)
          (declare (ignore var-0 var-2 var-4))
          (when var-1 (setf in var-1))
          (when var-3 (setf ldt var-3))
          ret-val))
(setf atol (max safmin (* ulp anorm)))
(setf btol (max safmin (* ulp bnorm)))
```

```
(setf ascale (/ one (max safmin anorm)))
(setf bscale (/ one (max safmin bnorm)))
(f2cl-lib:fdo (j (f2cl-lib:int-add ihi 1) (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (cond
     ((< (f2cl-lib:fref t$ (j j) ((1 ldt) (1 *))) zero)
      (cond
       (ilschr
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr j) nil)
          (tagbody
            (setf (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                                 h-%offset%)
                  (-
                   (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                                  h-%offset%)))
            (setf (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                                 t$-%offset%)
                  (-
                   (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                                  t$-%offset%)))
           label10)))
       (t
        (setf (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *)) h-%offset%)
              (-
               (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *))
                              h-%offset%)))
        (setf (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                             t$-%offset%)
              (-
               (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                              t$-%offset%)))))
      (cond
       (ilz
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr n) nil)
          (tagbody
            (setf (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                 z-%offset%)
                  (-
                   (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                  z-%offset%)))
           label20))))))
    (setf (f2cl-lib:fref alphar-%data% (j) ((1 *)) alphar-%offset%)
          (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *)) h-%offset%))
    (setf (f2cl-lib:fref alphai-%data% (j) ((1 *)) alphai-%offset%)
          zero)
    (setf (f2cl-lib:fref beta-%data% (j) ((1 *)) beta-%offset%)
          (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
```

```
                            t$-%offset%))
   label30))
(if (< ihi ilo)
    (go label380))
(setf ilast ihi)
(cond (ilschr (setf ifrstm 1) (setf ilastm n))
      (t (setf ifrstm ilo) (setf ilastm ihi)))
(setf iiter 0)
(setf eshift zero)
(setf maxit
        (f2cl-lib:int-mul 30
                           (f2cl-lib:int-add (f2cl-lib:int-sub ihi ilo)
                                             1)))
(f2cl-lib:fdo (jiter 1 (f2cl-lib:int-add jiter 1))
              ((> jiter maxit) nil)
  (tagbody
    (cond ((= ilast ilo) (go label80))
          (t
           (cond
            ((<=
              (abs
               (f2cl-lib:fref h
                              (ilast
                               (f2cl-lib:int-add ilast
                                                 (f2cl-lib:int-sub 1)))
                              ((1 ldh) (1 *))))
              atol)
             (setf (f2cl-lib:fref h-%data%
                                  (ilast (f2cl-lib:int-sub ilast 1))
                                  ((1 ldh) (1 *)) h-%offset%)
                   zero)
             (go label80)))))
    (cond
     ((<= (abs (f2cl-lib:fref t$ (ilast ilast) ((1 ldt) (1 *)))) btol)
      (setf (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                           t$-%offset%)
            zero)
      (go label70)))
    (f2cl-lib:fdo (j (f2cl-lib:int-add ilast (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                  ((> j ilo) nil)
      (tagbody
        (cond ((= j ilo) (setf ilazro f2cl-lib:%true%))
              (t
               (cond
                ((<=
                  (abs
                   (f2cl-lib:fref h
                                  (j
                                   (f2cl-lib:int-add j
```

```
                                                    (f2cl-lib:int-sub
                                                     1)))
                                  ((1 ldh) (1 *))))
                 atol)
                (setf (f2cl-lib:fref h-%data%
                                     (j (f2cl-lib:int-sub j 1))
                                     ((1 ldh) (1 *)) h-%offset%)
                      zero)
                (setf ilazro f2cl-lib:%true%))
               (t (setf ilazro f2cl-lib:%false%)))))
(cond
 ((< (abs (f2cl-lib:fref t$ (j j) ((1 ldt) (1 *)))) btol)
  (setf (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                       t$-%offset%)
        zero)
  (setf ilazr2 f2cl-lib:%false%)
  (cond
   ((not ilazro)
    (setf temp
          (abs
           (f2cl-lib:fref h-%data% (j (f2cl-lib:int-sub j 1))
                          ((1 ldh) (1 *)) h-%offset%)))
    (setf temp2
          (abs
           (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *))
                          h-%offset%)))
    (setf tempr (max temp temp2))
    (cond
     ((and (< tempr one) (/= tempr zero))
      (setf temp (/ temp tempr)) (setf temp2 (/ temp2 tempr))))
    (if (<=
         (* temp
            (* ascale
               (abs
                (f2cl-lib:fref h-%data%
                               ((f2cl-lib:int-add j 1) j)
                               ((1 ldh) (1 *)) h-%offset%))))
         (* temp2 (* ascale atol)))
        (setf ilazr2 f2cl-lib:%true%))))
  (cond
   ((or ilazro ilazr2)
    (f2cl-lib:fdo (jch j (f2cl-lib:int-add jch 1))
                  ((> jch
                      (f2cl-lib:int-add ilast
                                        (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf temp
                (f2cl-lib:fref h-%data% (jch jch)
                               ((1 ldh) (1 *)) h-%offset%))
```

```
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (slartg temp
     (f2cl-lib:array-slice h-%data% single-float
                           ((+ jch 1) jch)
                           ((1 ldh) (1 *)) h-%offset%)
     c s
     (f2cl-lib:array-slice h-%data% single-float
                           (jch jch) ((1 ldh) (1 *))
                           h-%offset%))
  (declare (ignore var-1 var-4))
  (when var-0 (setf temp var-0))
  (when var-2 (setf c var-2))
  (when var-3 (setf s var-3)))
(setf (f2cl-lib:fref h-%data%
                     ((f2cl-lib:int-add jch 1) jch)
                     ((1 ldh) (1 *)) h-%offset%)
        zero)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (srot (f2cl-lib:int-sub ilastm jch)
     (f2cl-lib:array-slice h-%data% single-float
                           (jch
                            (f2cl-lib:int-add jch 1))
                           ((1 ldh) (1 *)) h-%offset%)
     ldh
     (f2cl-lib:array-slice h-%data% single-float
                           ((+ jch 1)
                            (f2cl-lib:int-add jch 1))
                           ((1 ldh) (1 *)) h-%offset%)
     ldh c s)
  (declare (ignore var-0 var-1 var-3))
  (when var-2 (setf ldh var-2))
  (when var-4 (setf ldh var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (srot (f2cl-lib:int-sub ilastm jch)
     (f2cl-lib:array-slice t$-%data% single-float
                           (jch
                            (f2cl-lib:int-add jch 1))
                           ((1 ldt) (1 *)) t$-%offset%)
     ldt
     (f2cl-lib:array-slice t$-%data% single-float
                           ((+ jch 1)
                            (f2cl-lib:int-add jch 1))
                           ((1 ldt) (1 *)) t$-%offset%)
     ldt c s)
  (declare (ignore var-0 var-1 var-3))
  (when var-2 (setf ldt var-2))
```

```
  (when var-4 (setf ldt var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(if ilq
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (srot n
         (f2cl-lib:array-slice q-%data% single-float
                               (1 jch) ((1 ldq) (1 *))
                               q-%offset%)
         1
         (f2cl-lib:array-slice q-%data% single-float
                               (1
                                (f2cl-lib:int-add jch
                                                  1))
                               ((1 ldq) (1 *))
                               q-%offset%)
         1 c s)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0))
      (when var-5 (setf c var-5))
      (when var-6 (setf s var-6))))
(if ilazr2
    (setf (f2cl-lib:fref h-%data%
                         (jch (f2cl-lib:int-sub jch 1))
                         ((1 ldh) (1 *)) h-%offset%)
            (*
             (f2cl-lib:fref h-%data%
                            (jch
                             (f2cl-lib:int-sub jch 1))
                            ((1 ldh) (1 *)) h-%offset%)
             c)))
(setf ilazr2 f2cl-lib:%false%)
(cond
 ((>=
   (abs
    (f2cl-lib:fref t$
                   ((f2cl-lib:int-add jch 1)
                    (f2cl-lib:int-add jch 1))
                   ((1 ldt) (1 *))))
   btol)
  (cond
   ((>= (f2cl-lib:int-add jch 1) ilast) (go label80))
   (t (setf ifirst (f2cl-lib:int-add jch 1))
    (go label110)))))
(setf (f2cl-lib:fref t$-%data%
                     ((f2cl-lib:int-add jch 1)
                      (f2cl-lib:int-add jch 1))
                     ((1 ldt) (1 *)) t$-%offset%)
        zero)
```

```
   label40))
 (go label70))
(t
 (f2cl-lib:fdo (jch j (f2cl-lib:int-add jch 1))
               ((> jch
                   (f2cl-lib:int-add ilast
                                     (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (setf temp
             (f2cl-lib:fref t$-%data%
                            (jch (f2cl-lib:int-add jch 1))
                            ((1 ldt) (1 *)) t$-%offset%))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (slartg temp
          (f2cl-lib:array-slice t$-%data% single-float
                                ((+ jch 1)
                                 (f2cl-lib:int-add jch 1))
                                ((1 ldt) (1 *)) t$-%offset%)
          c s
          (f2cl-lib:array-slice t$-%data% single-float
                                (jch
                                 (f2cl-lib:int-add jch 1))
                                ((1 ldt) (1 *))
                                t$-%offset%))
       (declare (ignore var-1 var-4))
       (when var-0 (setf temp var-0))
       (when var-2 (setf c var-2))
       (when var-3 (setf s var-3)))
     (setf (f2cl-lib:fref t$-%data%
                          ((f2cl-lib:int-add jch 1)
                           (f2cl-lib:int-add jch 1))
                          ((1 ldt) (1 *)) t$-%offset%)
             zero)
     (if (< jch (f2cl-lib:int-sub ilastm 1))
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
             (srot (f2cl-lib:int-sub ilastm jch 1)
              (f2cl-lib:array-slice t$-%data% single-float
                                    (jch
                                     (f2cl-lib:int-add jch
                                                       2))
                                    ((1 ldt) (1 *))
                                    t$-%offset%)
              ldt
              (f2cl-lib:array-slice t$-%data% single-float
                                    ((+ jch 1)
                                     (f2cl-lib:int-add jch
                                                       2))
                                    ((1 ldt) (1 *))
```

```
                                t$-%offset%)
          ldt c s)
      (declare (ignore var-0 var-1 var-3))
      (when var-2 (setf ldt var-2))
      (when var-4 (setf ldt var-4))
      (when var-5 (setf c var-5))
      (when var-6 (setf s var-6))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (srot
     (f2cl-lib:int-add (f2cl-lib:int-sub ilastm jch) 2)
     (f2cl-lib:array-slice h-%data% single-float
                           (jch
                            (f2cl-lib:int-sub jch 1))
                           ((1 ldh) (1 *)) h-%offset%)
     ldh
     (f2cl-lib:array-slice h-%data% single-float
                           ((+ jch 1)
                            (f2cl-lib:int-sub jch 1))
                           ((1 ldh) (1 *)) h-%offset%)
     ldh c s)
  (declare (ignore var-0 var-1 var-3))
  (when var-2 (setf ldh var-2))
  (when var-4 (setf ldh var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(if ilq
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (srot n
         (f2cl-lib:array-slice q-%data% single-float
                               (1 jch) ((1 ldq) (1 *))
                               q-%offset%)
         1
         (f2cl-lib:array-slice q-%data% single-float
                               (1
                                (f2cl-lib:int-add jch
                                                  1))
                               ((1 ldq) (1 *))
                               q-%offset%)
         1 c s)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0))
      (when var-5 (setf c var-5))
      (when var-6 (setf s var-6))))
(setf temp
        (f2cl-lib:fref h-%data%
                       ((f2cl-lib:int-add jch 1) jch)
                       ((1 ldh) (1 *)) h-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
    (slartg temp
     (f2cl-lib:array-slice h-%data% single-float
                           ((+ jch 1)
                            (f2cl-lib:int-sub jch 1))
                           ((1 ldh) (1 *)) h-%offset%)
     c s
     (f2cl-lib:array-slice h-%data% single-float
                           ((+ jch 1) jch)
                           ((1 ldh) (1 *)) h-%offset%))
  (declare (ignore var-1 var-4))
  (when var-0 (setf temp var-0))
  (when var-2 (setf c var-2))
  (when var-3 (setf s var-3)))
(setf (f2cl-lib:fref h-%data%
                     ((f2cl-lib:int-add jch 1)
                      (f2cl-lib:int-sub jch 1))
                     ((1 ldh) (1 *)) h-%offset%)
        zero)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (srot
     (f2cl-lib:int-sub (f2cl-lib:int-add jch 1) ifrstm)
     (f2cl-lib:array-slice h-%data% single-float
                           (ifrstm jch) ((1 ldh) (1 *))
                           h-%offset%)
     1
     (f2cl-lib:array-slice h-%data% single-float
                           (ifrstm
                            (f2cl-lib:int-sub jch 1))
                           ((1 ldh) (1 *)) h-%offset%)
     1 c s)
  (declare (ignore var-0 var-1 var-2 var-3 var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (srot (f2cl-lib:int-sub jch ifrstm)
     (f2cl-lib:array-slice t$-%data% single-float
                           (ifrstm jch) ((1 ldt) (1 *))
                           t$-%offset%)
     1
     (f2cl-lib:array-slice t$-%data% single-float
                           (ifrstm
                            (f2cl-lib:int-sub jch 1))
                           ((1 ldt) (1 *)) t$-%offset%)
     1 c s)
  (declare (ignore var-0 var-1 var-2 var-3 var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(if ilz
```

```
                    (multiple-value-bind
                        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                        (srot n
                         (f2cl-lib:array-slice z-%data% single-float
                                               (1 jch) ((1 ldz) (1 *))
                                               z-%offset%)
                         1
                         (f2cl-lib:array-slice z-%data% single-float
                                               (1
                                                (f2cl-lib:int-sub jch
                                                                  1))
                                               ((1 ldz) (1 *))
                                               z-%offset%)
                         1 c s)
                      (declare (ignore var-1 var-2 var-3 var-4))
                      (when var-0 (setf n var-0))
                      (when var-5 (setf c var-5))
                      (when var-6 (setf s var-6))))
                 label50))
            (go label70))))
        (ilazro (setf ifirst j) (go label110)))
      label60))
  (setf info (f2cl-lib:int-add n 1))
  (go label420)
 label70
  (setf temp
          (f2cl-lib:fref h-%data% (ilast ilast) ((1 ldh) (1 *))
                         h-%offset%))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartg temp
       (f2cl-lib:array-slice h-%data% single-float
                             (ilast (f2cl-lib:int-sub ilast 1))
                             ((1 ldh) (1 *)) h-%offset%)
       c s
       (f2cl-lib:array-slice h-%data% single-float (ilast ilast)
                             ((1 ldh) (1 *)) h-%offset%))
    (declare (ignore var-1 var-4))
    (when var-0 (setf temp var-0))
    (when var-2 (setf c var-2))
    (when var-3 (setf s var-3)))
  (setf (f2cl-lib:fref h-%data% (ilast (f2cl-lib:int-sub ilast 1))
                       ((1 ldh) (1 *)) h-%offset%)
          zero)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (srot (f2cl-lib:int-sub ilast ifrstm)
       (f2cl-lib:array-slice h-%data% single-float (ifrstm ilast)
                             ((1 ldh) (1 *)) h-%offset%)
       1
       (f2cl-lib:array-slice h-%data% single-float
                             (ifrstm (f2cl-lib:int-sub ilast 1))
```

```
                              ((1 ldh) (1 *)) h-%offset%)
        1 c s)
      (declare (ignore var-0 var-1 var-2 var-3 var-4))
      (when var-5 (setf c var-5))
      (when var-6 (setf s var-6)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (srot (f2cl-lib:int-sub ilast ifrstm)
         (f2cl-lib:array-slice t$-%data% single-float (ifrstm ilast)
                              ((1 ldt) (1 *)) t$-%offset%)
         1
         (f2cl-lib:array-slice t$-%data% single-float
                              (ifrstm (f2cl-lib:int-sub ilast 1))
                              ((1 ldt) (1 *)) t$-%offset%)
         1 c s)
      (declare (ignore var-0 var-1 var-2 var-3 var-4))
      (when var-5 (setf c var-5))
      (when var-6 (setf s var-6)))
    (if ilz
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (srot n
             (f2cl-lib:array-slice z-%data% single-float (1 ilast)
                                   ((1 ldz) (1 *)) z-%offset%)
             1
             (f2cl-lib:array-slice z-%data% single-float
                                   (1 (f2cl-lib:int-sub ilast 1))
                                   ((1 ldz) (1 *)) z-%offset%)
             1 c s)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf n var-0))
          (when var-5 (setf c var-5))
          (when var-6 (setf s var-6))))
  label80
   (cond
    ((< (f2cl-lib:fref t$ (ilast ilast) ((1 ldt) (1 *))) zero)
     (cond
      (ilschr
       (f2cl-lib:fdo (j ifrstm (f2cl-lib:int-add j 1))
                     ((> j ilast) nil)
         (tagbody
           (setf (f2cl-lib:fref h-%data% (j ilast) ((1 ldh) (1 *))
                                h-%offset%)
                 (-
                  (f2cl-lib:fref h-%data% (j ilast) ((1 ldh) (1 *))
                                 h-%offset%)))
           (setf (f2cl-lib:fref t$-%data% (j ilast) ((1 ldt) (1 *))
                                t$-%offset%)
                 (-
                  (f2cl-lib:fref t$-%data% (j ilast) ((1 ldt) (1 *))
                                 t$-%offset%)))
```

```
          label90)))
      (t
       (setf (f2cl-lib:fref h-%data% (ilast ilast) ((1 ldh) (1 *))
                            h-%offset%)
             (-
              (f2cl-lib:fref h-%data% (ilast ilast) ((1 ldh) (1 *))
                             h-%offset%)))
       (setf (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                            t$-%offset%)
             (-
              (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                             t$-%offset%)))))
    (cond
     (ilz
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf (f2cl-lib:fref z-%data% (j ilast) ((1 ldz) (1 *))
                               z-%offset%)
                (-
                 (f2cl-lib:fref z-%data% (j ilast) ((1 ldz) (1 *))
                                z-%offset%)))
        label100))))))
  (setf (f2cl-lib:fref alphar-%data% (ilast) ((1 *)) alphar-%offset%)
        (f2cl-lib:fref h-%data% (ilast ilast) ((1 ldh) (1 *))
                       h-%offset%))
  (setf (f2cl-lib:fref alphai-%data% (ilast) ((1 *)) alphai-%offset%)
        zero)
  (setf (f2cl-lib:fref beta-%data% (ilast) ((1 *)) beta-%offset%)
        (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                       t$-%offset%))
  (setf ilast (f2cl-lib:int-sub ilast 1))
  (if (< ilast ilo)
      (go label380))
  (setf iiter 0)
  (setf eshift zero)
  (cond
   ((not ilschr) (setf ilastm ilast)
    (if (> ifrstm ilast)
        (setf ifrstm ilo))))
  (go label350)
 label110
  (setf iiter (f2cl-lib:int-add iiter 1))
  (cond ((not ilschr) (setf ifrstm ifirst)))
  (cond
   ((= (f2cl-lib:int-mul (f2cl-lib:f2cl/ iiter 10) 10) iiter)
    (cond
     ((<
       (* (f2cl-lib:freal maxit) safmin
          (abs
```

```
        (f2cl-lib:fref h
                       (ilast
                        (f2cl-lib:int-add ilast
                                          (f2cl-lib:int-sub 1)))
                       ((1 ldh) (1 *)))))
    (abs
     (f2cl-lib:fref t$
                    ((f2cl-lib:int-add ilast (f2cl-lib:int-sub 1))
                     (f2cl-lib:int-add ilast
                                       (f2cl-lib:int-sub 1)))
                    ((1 ldt) (1 *)))))
   (setf eshift
           (/
            (f2cl-lib:fref h-%data%
                           (ilast (f2cl-lib:int-sub ilast 1))
                           ((1 ldh) (1 *)) h-%offset%)
            (f2cl-lib:fref t$-%data%
                           ((f2cl-lib:int-sub ilast 1)
                            (f2cl-lib:int-sub ilast 1))
                           ((1 ldt) (1 *)) t$-%offset%))))
  (t
   (setf eshift
           (+ eshift (/ one (* safmin (f2cl-lib:freal maxit)))))))
 (setf s1 one) (setf wr eshift))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
     (slag2
      (f2cl-lib:array-slice h-%data% single-float
                            ((+ ilast (f2cl-lib:int-sub 1))
                             (f2cl-lib:int-sub ilast 1))
                            ((1 ldh) (1 *)) h-%offset%)
      ldh
      (f2cl-lib:array-slice t$-%data% single-float
                            ((+ ilast (f2cl-lib:int-sub 1))
                             (f2cl-lib:int-sub ilast 1))
                            ((1 ldt) (1 *)) t$-%offset%)
      ldt (* safmin safety) s1 s2 wr wr2 wi)
   (declare (ignore var-0 var-2 var-4))
   (when var-1 (setf ldh var-1))
   (when var-3 (setf ldt var-3))
   (when var-5 (setf s1 var-5))
   (when var-6 (setf s2 var-6))
   (when var-7 (setf wr var-7))
   (when var-8 (setf wr2 var-8))
   (when var-9 (setf wi var-9)))
 (cond
  ((>
    (abs
     (+
```

```
        (* (f2cl-lib:f2cl/ wr s1)
           (f2cl-lib:fref t$ (ilast ilast) ((1 ldt) (1 *))))
        (- (f2cl-lib:fref h (ilast ilast) ((1 ldh) (1 *))))))
      (abs
       (+
        (* (f2cl-lib:f2cl/ wr2 s2)
           (f2cl-lib:fref t$ (ilast ilast) ((1 ldt) (1 *))))
        (- (f2cl-lib:fref h (ilast ilast) ((1 ldh) (1 *)))))))
    (setf temp wr) (setf wr wr2) (setf wr2 temp) (setf temp s1)
    (setf s1 s2) (setf s2 temp)))
  (setf temp (max s1 (* safmin (max one (abs wr) (abs wi)))))
  (if (/= wi zero)
      (go label200))))
(setf temp (* (min ascale one) (* half safmax)))
(cond ((> s1 temp) (setf scale (/ temp s1))) (t (setf scale one)))
(setf temp (* (min bscale one) (* half safmax)))
(if (> (abs wr) temp)
    (setf scale (min scale (/ temp (abs wr)))))
(setf s1 (* scale s1))
(setf wr (* scale wr))
(f2cl-lib:fdo (j (f2cl-lib:int-add ilast (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j (f2cl-lib:int-add ifirst 1)) nil)
  (tagbody
    (setf istart j)
    (setf temp
            (abs
             (* s1
                (f2cl-lib:fref h-%data% (j (f2cl-lib:int-sub j 1))
                               ((1 ldh) (1 *)) h-%offset%))))
    (setf temp2
            (abs
             (-
              (* s1
                 (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *))
                                h-%offset%))
              (* wr
                 (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                                t$-%offset%)))))
    (setf tempr (max temp temp2))
    (cond
     ((and (< tempr one) (/= tempr zero))
      (setf temp (/ temp tempr)) (setf temp2 (/ temp2 tempr))))
    (if (<=
         (abs
          (* ascale
             (f2cl-lib:fref h-%data% ((f2cl-lib:int-add j 1) j)
                            ((1 ldh) (1 *)) h-%offset%)
             temp))
         (* ascale atol temp2))
```

```
          (go label130))
      label120))
   (setf istart ifirst)
  label130
   (setf temp
          (-
           (* s1
              (f2cl-lib:fref h-%data% (istart istart) ((1 ldh) (1 *))
                             h-%offset%))
           (* wr
              (f2cl-lib:fref t$-%data% (istart istart)
                             ((1 ldt) (1 *)) t$-%offset%))))
   (setf temp2
          (* s1
             (f2cl-lib:fref h-%data%
                            ((f2cl-lib:int-add istart 1) istart)
                            ((1 ldh) (1 *)) h-%offset%)))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
       (slartg temp temp2 c s tempr)
     (declare (ignore))
     (when var-0 (setf temp var-0))
     (when var-1 (setf temp2 var-1))
     (when var-2 (setf c var-2))
     (when var-3 (setf s var-3))
     (when var-4 (setf tempr var-4)))
   (f2cl-lib:fdo (j istart (f2cl-lib:int-add j 1))
                 ((> j (f2cl-lib:int-add ilast (f2cl-lib:int-sub 1)))
                  nil)
     (tagbody
       (cond
        ((> j istart)
         (setf temp
                 (f2cl-lib:fref h-%data% (j (f2cl-lib:int-sub j 1))
                                ((1 ldh) (1 *)) h-%offset%))
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (slartg temp
              (f2cl-lib:array-slice h-%data% single-float
                                    ((+ j 1) (f2cl-lib:int-sub j 1))
                                    ((1 ldh) (1 *)) h-%offset%)
              c s
              (f2cl-lib:array-slice h-%data% single-float
                                    (j (f2cl-lib:int-sub j 1))
                                    ((1 ldh) (1 *)) h-%offset%))
           (declare (ignore var-1 var-4))
           (when var-0 (setf temp var-0))
           (when var-2 (setf c var-2))
           (when var-3 (setf s var-3)))
         (setf (f2cl-lib:fref h-%data%
                              ((f2cl-lib:int-add j 1)
                               (f2cl-lib:int-sub j 1))
```

```
                        ((1 ldh) (1 *)) h-%offset%)
          zero)))
(f2cl-lib:fdo (jc j (f2cl-lib:int-add jc 1))
              ((> jc ilastm) nil)
  (tagbody
    (setf temp
            (+
             (* c
                (f2cl-lib:fref h-%data% (j jc) ((1 ldh) (1 *))
                               h-%offset%))
             (* s
                (f2cl-lib:fref h-%data%
                               ((f2cl-lib:int-add j 1) jc)
                               ((1 ldh) (1 *)) h-%offset%))))
    (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add j 1) jc)
                         ((1 ldh) (1 *)) h-%offset%)
            (+
             (* (- s)
                (f2cl-lib:fref h-%data% (j jc) ((1 ldh) (1 *))
                               h-%offset%))
             (* c
                (f2cl-lib:fref h-%data%
                               ((f2cl-lib:int-add j 1) jc)
                               ((1 ldh) (1 *)) h-%offset%))))
    (setf (f2cl-lib:fref h-%data% (j jc) ((1 ldh) (1 *))
                         h-%offset%)
            temp)
    (setf temp2
            (+
             (* c
                (f2cl-lib:fref t$-%data% (j jc) ((1 ldt) (1 *))
                               t$-%offset%))
             (* s
                (f2cl-lib:fref t$-%data%
                               ((f2cl-lib:int-add j 1) jc)
                               ((1 ldt) (1 *)) t$-%offset%))))
    (setf (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 1) jc)
                         ((1 ldt) (1 *)) t$-%offset%)
            (+
             (* (- s)
                (f2cl-lib:fref t$-%data% (j jc) ((1 ldt) (1 *))
                               t$-%offset%))
             (* c
                (f2cl-lib:fref t$-%data%
                               ((f2cl-lib:int-add j 1) jc)
                               ((1 ldt) (1 *)) t$-%offset%))))
    (setf (f2cl-lib:fref t$-%data% (j jc) ((1 ldt) (1 *))
                         t$-%offset%)
            temp2)
  label140))
```

```
(cond
 (ilq
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
      (setf temp
              (+
               (* c
                  (f2cl-lib:fref q-%data% (jr j)
                                 ((1 ldq) (1 *)) q-%offset%))
               (* s
                  (f2cl-lib:fref q-%data%
                                 (jr (f2cl-lib:int-add j 1))
                                 ((1 ldq) (1 *)) q-%offset%))))
      (setf (f2cl-lib:fref q-%data% (jr (f2cl-lib:int-add j 1))
                           ((1 ldq) (1 *)) q-%offset%)
              (+
               (* (- s)
                  (f2cl-lib:fref q-%data% (jr j)
                                 ((1 ldq) (1 *)) q-%offset%))
               (* c
                  (f2cl-lib:fref q-%data%
                                 (jr (f2cl-lib:int-add j 1))
                                 ((1 ldq) (1 *)) q-%offset%))))
      (setf (f2cl-lib:fref q-%data% (jr j) ((1 ldq) (1 *))
                           q-%offset%)
              temp)
     label150))))
(setf temp
        (f2cl-lib:fref t$-%data%
                       ((f2cl-lib:int-add j 1)
                        (f2cl-lib:int-add j 1))
                       ((1 ldt) (1 *)) t$-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (slartg temp
     (f2cl-lib:array-slice t$-%data% single-float ((+ j 1) j)
                           ((1 ldt) (1 *)) t$-%offset%)
     c s
     (f2cl-lib:array-slice t$-%data% single-float
                           ((+ j 1) (f2cl-lib:int-add j 1))
                           ((1 ldt) (1 *)) t$-%offset%))
  (declare (ignore var-1 var-4))
  (when var-0 (setf temp var-0))
  (when var-2 (setf c var-2))
  (when var-3 (setf s var-3)))
(setf (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 1) j)
                     ((1 ldt) (1 *)) t$-%offset%)
        zero)
(f2cl-lib:fdo (jr ifrstm (f2cl-lib:int-add jr 1))
              ((> jr
```

```
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add j 2))
                     (the f2cl-lib:integer4 ilast)))
                 nil)
      (tagbody
        (setf temp
                (+
                 (* c
                    (f2cl-lib:fref h-%data%
                                   (jr (f2cl-lib:int-add j 1))
                                   ((1 ldh) (1 *)) h-%offset%))
                 (* s
                    (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                                   h-%offset%))))
        (setf (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                             h-%offset%)
                (+
                 (* (- s)
                    (f2cl-lib:fref h-%data%
                                   (jr (f2cl-lib:int-add j 1))
                                   ((1 ldh) (1 *)) h-%offset%))
                 (* c
                    (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                                   h-%offset%))))
        (setf (f2cl-lib:fref h-%data% (jr (f2cl-lib:int-add j 1))
                             ((1 ldh) (1 *)) h-%offset%)
                temp)
       label160))
    (f2cl-lib:fdo (jr ifrstm (f2cl-lib:int-add jr 1))
                  ((> jr j) nil)
      (tagbody
        (setf temp
                (+
                 (* c
                    (f2cl-lib:fref t$-%data%
                                   (jr (f2cl-lib:int-add j 1))
                                   ((1 ldt) (1 *)) t$-%offset%))
                 (* s
                    (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                                   t$-%offset%))))
        (setf (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                             t$-%offset%)
                (+
                 (* (- s)
                    (f2cl-lib:fref t$-%data%
                                   (jr (f2cl-lib:int-add j 1))
                                   ((1 ldt) (1 *)) t$-%offset%))
                 (* c
                    (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
```

```
                                    t$-%offset%))))
            (setf (f2cl-lib:fref t$-%data% (jr (f2cl-lib:int-add j 1))
                                  ((1 ldt) (1 *)) t$-%offset%)
                  temp)
          label170))
        (cond
         (ilz
          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                        ((> jr n) nil)
            (tagbody
              (setf temp
                      (+
                       (* c
                          (f2cl-lib:fref z-%data%
                                         (jr (f2cl-lib:int-add j 1))
                                         ((1 ldz) (1 *)) z-%offset%))
                       (* s
                          (f2cl-lib:fref z-%data% (jr j)
                                         ((1 ldz) (1 *)) z-%offset%))))
              (setf (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                    z-%offset%)
                      (+
                       (* (- s)
                          (f2cl-lib:fref z-%data%
                                         (jr (f2cl-lib:int-add j 1))
                                         ((1 ldz) (1 *)) z-%offset%))
                       (* c
                          (f2cl-lib:fref z-%data% (jr j)
                                         ((1 ldz) (1 *)) z-%offset%))))
              (setf (f2cl-lib:fref z-%data% (jr (f2cl-lib:int-add j 1))
                                    ((1 ldz) (1 *)) z-%offset%)
                      temp)
            label180))))
      label190))
    (go label350)
  label200
    (cond
     ((= (f2cl-lib:int-add ifirst 1) ilast)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (slasv2
           (f2cl-lib:array-slice t$-%data% single-float
                                 ((+ ilast (f2cl-lib:int-sub 1))
                                  (f2cl-lib:int-sub ilast 1))
                                 ((1 ldt) (1 *)) t$-%offset%)
           (f2cl-lib:array-slice t$-%data% single-float
                                 ((+ ilast (f2cl-lib:int-sub 1)) ilast)
                                 ((1 ldt) (1 *)) t$-%offset%)
           (f2cl-lib:array-slice t$-%data% single-float (ilast ilast)
                                 ((1 ldt) (1 *)) t$-%offset%)
```

```
      b22 b11 sr cr sl cl)
  (declare (ignore var-0 var-1 var-2))
  (when var-3 (setf b22 var-3))
  (when var-4 (setf b11 var-4))
  (when var-5 (setf sr var-5))
  (when var-6 (setf cr var-6))
  (when var-7 (setf sl var-7))
  (when var-8 (setf cl var-8)))
(cond
 ((< b11 zero) (setf cr (- cr)) (setf sr (- sr))
  (setf b11 (- b11)) (setf b22 (- b22))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (srot (f2cl-lib:int-sub (f2cl-lib:int-add ilastm 1) ifirst)
     (f2cl-lib:array-slice h-%data% single-float
                           ((+ ilast (f2cl-lib:int-sub 1))
                            (f2cl-lib:int-sub ilast 1))
                           ((1 ldh) (1 *)) h-%offset%)
     ldh
     (f2cl-lib:array-slice h-%data% single-float
                           (ilast (f2cl-lib:int-sub ilast 1))
                           ((1 ldh) (1 *)) h-%offset%)
     ldh cl sl)
  (declare (ignore var-0 var-1 var-3))
  (when var-2 (setf ldh var-2))
  (when var-4 (setf ldh var-4))
  (when var-5 (setf cl var-5))
  (when var-6 (setf sl var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (srot (f2cl-lib:int-sub (f2cl-lib:int-add ilast 1) ifrstm)
     (f2cl-lib:array-slice h-%data% single-float
                           (ifrstm (f2cl-lib:int-sub ilast 1))
                           ((1 ldh) (1 *)) h-%offset%)
     1
     (f2cl-lib:array-slice h-%data% single-float (ifrstm ilast)
                           ((1 ldh) (1 *)) h-%offset%)
     1 cr sr)
  (declare (ignore var-0 var-1 var-2 var-3 var-4))
  (when var-5 (setf cr var-5))
  (when var-6 (setf sr var-6)))
(if (< ilast ilastm)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (srot (f2cl-lib:int-sub ilastm ilast)
         (f2cl-lib:array-slice t$-%data% single-float
                               ((+ ilast (f2cl-lib:int-sub 1))
                                (f2cl-lib:int-add ilast 1))
                               ((1 ldt) (1 *)) t$-%offset%)
         ldt
         (f2cl-lib:array-slice t$-%data% single-float
                               (ilast (f2cl-lib:int-add ilast 1))
```

```
                                ((1 ldt) (1 *)) t$-%offset%)
          ldt cl sl)
      (declare (ignore var-0 var-1 var-3))
      (when var-2 (setf ldt var-2))
      (when var-4 (setf ldt var-4))
      (when var-5 (setf cl var-5))
      (when var-6 (setf sl var-6))))
(if (< ifrstm (f2cl-lib:int-sub ilast 1))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (srot (f2cl-lib:int-sub ifirst ifrstm)
         (f2cl-lib:array-slice t$-%data% single-float
                                (ifrstm
                                 (f2cl-lib:int-sub ilast 1))
                                ((1 ldt) (1 *)) t$-%offset%)
         1
         (f2cl-lib:array-slice t$-%data% single-float
                                (ifrstm ilast) ((1 ldt) (1 *))
                                t$-%offset%)
         1 cr sr)
      (declare (ignore var-0 var-1 var-2 var-3 var-4))
      (when var-5 (setf cr var-5))
      (when var-6 (setf sr var-6))))
(if ilq
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (srot n
         (f2cl-lib:array-slice q-%data% single-float
                                (1 (f2cl-lib:int-sub ilast 1))
                                ((1 ldq) (1 *)) q-%offset%)
         1
         (f2cl-lib:array-slice q-%data% single-float (1 ilast)
                                ((1 ldq) (1 *)) q-%offset%)
         1 cl sl)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0))
      (when var-5 (setf cl var-5))
      (when var-6 (setf sl var-6))))
(if ilz
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (srot n
         (f2cl-lib:array-slice z-%data% single-float
                                (1 (f2cl-lib:int-sub ilast 1))
                                ((1 ldz) (1 *)) z-%offset%)
         1
         (f2cl-lib:array-slice z-%data% single-float (1 ilast)
                                ((1 ldz) (1 *)) z-%offset%)
         1 cr sr)
      (declare (ignore var-1 var-2 var-3 var-4))
```

```
      (when var-0 (setf n var-0))
      (when var-5 (setf cr var-5))
      (when var-6 (setf sr var-6))))
(setf (f2cl-lib:fref t$-%data%
                     ((f2cl-lib:int-sub ilast 1)
                      (f2cl-lib:int-sub ilast 1))
                     ((1 ldt) (1 *)) t$-%offset%)
        b11)
(setf (f2cl-lib:fref t$-%data% ((f2cl-lib:int-sub ilast 1) ilast)
                     ((1 ldt) (1 *)) t$-%offset%)
        zero)
(setf (f2cl-lib:fref t$-%data% (ilast (f2cl-lib:int-sub ilast 1))
                     ((1 ldt) (1 *)) t$-%offset%)
        zero)
(setf (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                     t$-%offset%)
        b22)
(cond
 ((< b22 zero)
  (f2cl-lib:fdo (j ifrstm (f2cl-lib:int-add j 1))
                ((> j ilast) nil)
    (tagbody
      (setf (f2cl-lib:fref h-%data% (j ilast) ((1 ldh) (1 *))
                           h-%offset%)
              (-
               (f2cl-lib:fref h-%data% (j ilast) ((1 ldh) (1 *))
                              h-%offset%)))
      (setf (f2cl-lib:fref t$-%data% (j ilast) ((1 ldt) (1 *))
                           t$-%offset%)
              (-
               (f2cl-lib:fref t$-%data% (j ilast) ((1 ldt) (1 *))
                              t$-%offset%)))
     label210))
  (cond
   (ilz
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref z-%data% (j ilast) ((1 ldz) (1 *))
                             z-%offset%)
                (-
                 (f2cl-lib:fref z-%data% (j ilast)
                                ((1 ldz) (1 *)) z-%offset%)))
       label220))))
  (setf b22 (- b22))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (slag2
     (f2cl-lib:array-slice h-%data% single-float
                           ((+ ilast (f2cl-lib:int-sub 1))
```

```
                              (f2cl-lib:int-sub ilast 1))
                             ((1 ldh) (1 *)) h-%offset%)
      ldh
      (f2cl-lib:array-slice t$-%data% single-float
                             ((+ ilast (f2cl-lib:int-sub 1))
                              (f2cl-lib:int-sub ilast 1))
                             ((1 ldt) (1 *)) t$-%offset%)
      ldt (* safmin safety) s1 temp wr temp2 wi)
  (declare (ignore var-0 var-2 var-4))
  (when var-1 (setf ldh var-1))
  (when var-3 (setf ldt var-3))
  (when var-5 (setf s1 var-5))
  (when var-6 (setf temp var-6))
  (when var-7 (setf wr var-7))
  (when var-8 (setf temp2 var-8))
  (when var-9 (setf wi var-9)))
(if (= wi zero)
    (go label350))
(setf s1inv (/ one s1))
(setf a11
        (f2cl-lib:fref h-%data%
                       ((f2cl-lib:int-sub ilast 1)
                        (f2cl-lib:int-sub ilast 1))
                       ((1 ldh) (1 *)) h-%offset%))
(setf a21
        (f2cl-lib:fref h-%data%
                       (ilast (f2cl-lib:int-sub ilast 1))
                       ((1 ldh) (1 *)) h-%offset%))
(setf a12
        (f2cl-lib:fref h-%data%
                       ((f2cl-lib:int-sub ilast 1) ilast)
                       ((1 ldh) (1 *)) h-%offset%))
(setf a22
        (f2cl-lib:fref h-%data% (ilast ilast) ((1 ldh) (1 *))
                       h-%offset%))
(setf c11r (- (* s1 a11) (* wr b11))) (setf c11i (* (- wi) b11))
(setf c12 (* s1 a12)) (setf c21 (* s1 a21))
(setf c22r (- (* s1 a22) (* wr b22))) (setf c22i (* (- wi) b22))
(cond
 ((> (+ (abs c11r) (abs c11i) (abs c12))
     (+ (abs c21) (abs c22r) (abs c22i)))
  (setf t1
          (multiple-value-bind (ret-val var-0 var-1 var-2)
              (slapy3 c12 c11r c11i)
            (declare (ignore))
            (when var-0 (setf c12 var-0))
            (when var-1 (setf c11r var-1))
            (when var-2 (setf c11i var-2))
            ret-val))
  (setf cz (/ c12 t1)) (setf szr (/ (- c11r) t1))
```

```
  (setf szi (/ (- c11i) t1)))
 (t
  (setf cz
          (multiple-value-bind (ret-val var-0 var-1)
              (slapy2 c22r c22i)
            (declare (ignore))
            (when var-0 (setf c22r var-0))
            (when var-1 (setf c22i var-1))
            ret-val))
  (cond
   ((<= cz safmin) (setf cz zero) (setf szr one) (setf szi zero))
   (t (setf tempr (/ c22r cz)) (setf tempi (/ c22i cz))
    (setf t1
            (multiple-value-bind (ret-val var-0 var-1)
                (slapy2 cz c21)
              (declare (ignore))
              (when var-0 (setf cz var-0))
              (when var-1 (setf c21 var-1))
              ret-val))
    (setf cz (/ cz t1)) (setf szr (/ (* (- c21) tempr) t1))
    (setf szi (/ (* c21 tempi) t1))))))
(setf an (+ (abs a11) (abs a12) (abs a21) (abs a22)))
(setf bn (+ (abs b11) (abs b22)))
(setf wabs (+ (abs wr) (abs wi)))
(cond
 ((> (* s1 an) (* wabs bn)) (setf cq (* cz b11))
  (setf sqr (* szr b22)) (setf sqi (* (- szi) b22)))
 (t (setf a1r (+ (* cz a11) (* szr a12))) (setf a1i (* szi a12))
  (setf a2r (+ (* cz a21) (* szr a22))) (setf a2i (* szi a22))
  (setf cq
          (multiple-value-bind (ret-val var-0 var-1)
              (slapy2 a1r a1i)
            (declare (ignore))
            (when var-0 (setf a1r var-0))
            (when var-1 (setf a1i var-1))
            ret-val))
  (cond
   ((<= cq safmin) (setf cq zero) (setf sqr one) (setf sqi zero))
   (t (setf tempr (/ a1r cq)) (setf tempi (/ a1i cq))
    (setf sqr (+ (* tempr a2r) (* tempi a2i)))
    (setf sqi (- (* tempi a2r) (* tempr a2i)))))))
(setf t1
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (slapy3 cq sqr sqi)
          (declare (ignore))
          (when var-0 (setf cq var-0))
          (when var-1 (setf sqr var-1))
          (when var-2 (setf sqi var-2))
          ret-val))
(setf cq (/ cq t1)) (setf sqr (/ sqr t1)) (setf sqi (/ sqi t1))
```

```
(setf tempr (- (* sqr szr) (* sqi szi)))
(setf tempi (+ (* sqr szi) (* sqi szr)))
(setf b1r (+ (* cq cz b11) (* tempr b22)))
(setf b1i (* tempi b22))
(setf b1a
        (multiple-value-bind (ret-val var-0 var-1)
            (slapy2 b1r b1i)
          (declare (ignore))
          (when var-0 (setf b1r var-0))
          (when var-1 (setf b1i var-1))
          ret-val))
(setf b2r (+ (* cq cz b22) (* tempr b11)))
(setf b2i (* (- tempi) b11))
(setf b2a
        (multiple-value-bind (ret-val var-0 var-1)
            (slapy2 b2r b2i)
          (declare (ignore))
          (when var-0 (setf b2r var-0))
          (when var-1 (setf b2i var-1))
          ret-val))
(setf (f2cl-lib:fref beta-%data% ((f2cl-lib:int-sub ilast 1))
                     ((1 *)) beta-%offset%)
        b1a)
(setf (f2cl-lib:fref beta-%data% (ilast) ((1 *)) beta-%offset%)
        b2a)
(setf (f2cl-lib:fref alphar-%data% ((f2cl-lib:int-sub ilast 1))
                     ((1 *)) alphar-%offset%)
        (* wr b1a s1inv))
(setf (f2cl-lib:fref alphai-%data% ((f2cl-lib:int-sub ilast 1))
                     ((1 *)) alphai-%offset%)
        (* wi b1a s1inv))
(setf (f2cl-lib:fref alphar-%data% (ilast) ((1 *))
                     alphar-%offset%)
        (* wr b2a s1inv))
(setf (f2cl-lib:fref alphai-%data% (ilast) ((1 *))
                     alphai-%offset%)
        (* (- (* wi b2a)) s1inv))
(setf ilast (f2cl-lib:int-sub ifirst 1))
(if (< ilast ilo)
    (go label380))
(setf iiter 0) (setf eshift zero)
(cond
 ((not ilschr) (setf ilastm ilast)
  (if (> ifrstm ilast)
      (setf ifrstm ilo))))
(go label350))
(t
(setf ad11
        (/
         (* ascale
```

```
            (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-sub ilast 1)
                            (f2cl-lib:int-sub ilast 1))
                           ((1 ldh) (1 *)) h-%offset%))
         (* bscale
            (f2cl-lib:fref t$-%data%
                           ((f2cl-lib:int-sub ilast 1)
                            (f2cl-lib:int-sub ilast 1))
                           ((1 ldt) (1 *)) t$-%offset%))))
(setf ad21
        (/
         (* ascale
            (f2cl-lib:fref h-%data%
                           (ilast (f2cl-lib:int-sub ilast 1))
                           ((1 ldh) (1 *)) h-%offset%))
         (* bscale
            (f2cl-lib:fref t$-%data%
                           ((f2cl-lib:int-sub ilast 1)
                            (f2cl-lib:int-sub ilast 1))
                           ((1 ldt) (1 *)) t$-%offset%))))
(setf ad12
        (/
         (* ascale
            (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-sub ilast 1) ilast)
                           ((1 ldh) (1 *)) h-%offset%))
         (* bscale
            (f2cl-lib:fref t$-%data% (ilast ilast)
                           ((1 ldt) (1 *)) t$-%offset%))))
(setf ad22
        (/
         (* ascale
            (f2cl-lib:fref h-%data% (ilast ilast) ((1 ldh) (1 *))
                           h-%offset%))
         (* bscale
            (f2cl-lib:fref t$-%data% (ilast ilast)
                           ((1 ldt) (1 *)) t$-%offset%))))
(setf u12
        (/
         (f2cl-lib:fref t$-%data%
                        ((f2cl-lib:int-sub ilast 1) ilast)
                        ((1 ldt) (1 *)) t$-%offset%)
         (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                        t$-%offset%)))
(setf ad11l
        (/
         (* ascale
            (f2cl-lib:fref h-%data% (ifirst ifirst)
                           ((1 ldh) (1 *)) h-%offset%))
         (* bscale
```

```
            (f2cl-lib:fref t$-%data% (ifirst ifirst)
                           ((1 ldt) (1 *)) t$-%offset%))))
(setf ad21l
        (/
         (* ascale
            (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-add ifirst 1) ifirst)
                           ((1 ldh) (1 *)) h-%offset%))
         (* bscale
            (f2cl-lib:fref t$-%data% (ifirst ifirst)
                           ((1 ldt) (1 *)) t$-%offset%))))
(setf ad12l
        (/
         (* ascale
            (f2cl-lib:fref h-%data%
                           (ifirst (f2cl-lib:int-add ifirst 1))
                           ((1 ldh) (1 *)) h-%offset%))
         (* bscale
            (f2cl-lib:fref t$-%data%
                           ((f2cl-lib:int-add ifirst 1)
                            (f2cl-lib:int-add ifirst 1))
                           ((1 ldt) (1 *)) t$-%offset%))))
(setf ad22l
        (/
         (* ascale
            (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-add ifirst 1)
                            (f2cl-lib:int-add ifirst 1))
                           ((1 ldh) (1 *)) h-%offset%))
         (* bscale
            (f2cl-lib:fref t$-%data%
                           ((f2cl-lib:int-add ifirst 1)
                            (f2cl-lib:int-add ifirst 1))
                           ((1 ldt) (1 *)) t$-%offset%))))
(setf ad32l
        (/
         (* ascale
            (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-add ifirst 2)
                            (f2cl-lib:int-add ifirst 1))
                           ((1 ldh) (1 *)) h-%offset%))
         (* bscale
            (f2cl-lib:fref t$-%data%
                           ((f2cl-lib:int-add ifirst 1)
                            (f2cl-lib:int-add ifirst 1))
                           ((1 ldt) (1 *)) t$-%offset%))))
(setf u12l
        (/
         (f2cl-lib:fref t$-%data%
                        (ifirst (f2cl-lib:int-add ifirst 1))
```

```
                          ((1 ldt) (1 *)) t$-%offset%)
          (f2cl-lib:fref t$-%data%
                          ((f2cl-lib:int-add ifirst 1)
                           (f2cl-lib:int-add ifirst 1))
                          ((1 ldt) (1 *)) t$-%offset%)))
(setf (f2cl-lib:fref v (1) ((1 3)))
        (+ (- (* (- ad11 ad11l) (- ad22 ad11l)) (* ad12 ad21))
           (* ad21 u12 ad11l)
           (* (- ad12l (* ad11l u12l)) ad21l)))
(setf (f2cl-lib:fref v (2) ((1 3)))
        (*
         (+
          (- ad22l ad11l (* ad21l u12l) (- ad11 ad11l)
             (- ad22 ad11l))
          (* ad21 u12))
         ad21l))
(setf (f2cl-lib:fref v (3) ((1 3))) (* ad32l ad21l))
(setf istart ifirst)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (slarfg 3 (f2cl-lib:array-slice v single-float (1) ((1 3)))
     (f2cl-lib:array-slice v single-float (2) ((1 3))) 1 tau)
  (declare (ignore var-0 var-1 var-2 var-3))
  (when var-4 (setf tau var-4)))
(setf (f2cl-lib:fref v (1) ((1 3))) one)
(f2cl-lib:fdo (j istart (f2cl-lib:int-add j 1))
              ((> j
                  (f2cl-lib:int-add ilast (f2cl-lib:int-sub 2)))
               nil)
  (tagbody
    (cond
     ((> j istart)
      (setf (f2cl-lib:fref v (1) ((1 3)))
              (f2cl-lib:fref h-%data% (j (f2cl-lib:int-sub j 1))
                              ((1 ldh) (1 *)) h-%offset%))
      (setf (f2cl-lib:fref v (2) ((1 3)))
              (f2cl-lib:fref h-%data%
                              ((f2cl-lib:int-add j 1)
                               (f2cl-lib:int-sub j 1))
                              ((1 ldh) (1 *)) h-%offset%))
      (setf (f2cl-lib:fref v (3) ((1 3)))
              (f2cl-lib:fref h-%data%
                              ((f2cl-lib:int-add j 2)
                               (f2cl-lib:int-sub j 1))
                              ((1 ldh) (1 *)) h-%offset%))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (slarfg 3
           (f2cl-lib:array-slice h-%data% single-float
                                 (j (f2cl-lib:int-sub j 1))
                                 ((1 ldh) (1 *)) h-%offset%)
           (f2cl-lib:array-slice v single-float (2) ((1 3))) 1
```

```
        tau)
      (declare (ignore var-0 var-1 var-2 var-3))
      (when var-4 (setf tau var-4)))
    (setf (f2cl-lib:fref v (1) ((1 3))) one)
    (setf (f2cl-lib:fref h-%data%
                         ((f2cl-lib:int-add j 1)
                          (f2cl-lib:int-sub j 1))
                         ((1 ldh) (1 *)) h-%offset%)
          zero)
    (setf (f2cl-lib:fref h-%data%
                         ((f2cl-lib:int-add j 2)
                          (f2cl-lib:int-sub j 1))
                         ((1 ldh) (1 *)) h-%offset%)
          zero)))
  (f2cl-lib:fdo (jc j (f2cl-lib:int-add jc 1))
                ((> jc ilastm) nil)
    (tagbody
      (setf temp
              (* tau
                 (+
                  (f2cl-lib:fref h-%data% (j jc)
                                 ((1 ldh) (1 *)) h-%offset%)
                  (* (f2cl-lib:fref v (2) ((1 3)))
                     (f2cl-lib:fref h-%data%
                                    ((f2cl-lib:int-add j 1) jc)
                                    ((1 ldh) (1 *))
                                    h-%offset%))
                  (* (f2cl-lib:fref v (3) ((1 3)))
                     (f2cl-lib:fref h-%data%
                                    ((f2cl-lib:int-add j 2) jc)
                                    ((1 ldh) (1 *))
                                    h-%offset%)))))
      (setf (f2cl-lib:fref h-%data% (j jc) ((1 ldh) (1 *))
                           h-%offset%)
              (-
               (f2cl-lib:fref h-%data% (j jc) ((1 ldh) (1 *))
                              h-%offset%)
               temp))
      (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add j 1) jc)
                           ((1 ldh) (1 *)) h-%offset%)
              (-
               (f2cl-lib:fref h-%data%
                              ((f2cl-lib:int-add j 1) jc)
                              ((1 ldh) (1 *)) h-%offset%)
               (* temp (f2cl-lib:fref v (2) ((1 3))))))
      (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add j 2) jc)
                           ((1 ldh) (1 *)) h-%offset%)
              (-
               (f2cl-lib:fref h-%data%
                              ((f2cl-lib:int-add j 2) jc)
```

```
                                ((1 ldh) (1 *)) h-%offset%)
                 (* temp (f2cl-lib:fref v (3) ((1 3))))))
        (setf temp2
                (* tau
                   (+
                    (f2cl-lib:fref t$-%data% (j jc)
                                   ((1 ldt) (1 *)) t$-%offset%)
                    (* (f2cl-lib:fref v (2) ((1 3)))
                       (f2cl-lib:fref t$-%data%
                                      ((f2cl-lib:int-add j 1) jc)
                                      ((1 ldt) (1 *))
                                      t$-%offset%))
                    (* (f2cl-lib:fref v (3) ((1 3)))
                       (f2cl-lib:fref t$-%data%
                                      ((f2cl-lib:int-add j 2) jc)
                                      ((1 ldt) (1 *))
                                      t$-%offset%)))))
        (setf (f2cl-lib:fref t$-%data% (j jc) ((1 ldt) (1 *))
                             t$-%offset%)
                (-
                 (f2cl-lib:fref t$-%data% (j jc) ((1 ldt) (1 *))
                                t$-%offset%)
                 temp2))
        (setf (f2cl-lib:fref t$-%data%
                             ((f2cl-lib:int-add j 1) jc)
                             ((1 ldt) (1 *)) t$-%offset%)
                (-
                 (f2cl-lib:fref t$-%data%
                                ((f2cl-lib:int-add j 1) jc)
                                ((1 ldt) (1 *)) t$-%offset%)
                 (* temp2 (f2cl-lib:fref v (2) ((1 3))))))
        (setf (f2cl-lib:fref t$-%data%
                             ((f2cl-lib:int-add j 2) jc)
                             ((1 ldt) (1 *)) t$-%offset%)
                (-
                 (f2cl-lib:fref t$-%data%
                                ((f2cl-lib:int-add j 2) jc)
                                ((1 ldt) (1 *)) t$-%offset%)
                 (* temp2 (f2cl-lib:fref v (3) ((1 3))))))
      label230))
    (cond
     (ilq
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf temp
                  (* tau
                     (+
                      (f2cl-lib:fref q-%data% (jr j)
                                     ((1 ldq) (1 *)) q-%offset%)
```

```
                    (* (f2cl-lib:fref v (2) ((1 3)))
                       (f2cl-lib:fref q-%data%
                                      (jr
                                       (f2cl-lib:int-add j 1))
                                      ((1 ldq) (1 *))
                                      q-%offset%))
                    (* (f2cl-lib:fref v (3) ((1 3)))
                       (f2cl-lib:fref q-%data%
                                      (jr
                                       (f2cl-lib:int-add j 2))
                                      ((1 ldq) (1 *))
                                      q-%offset%)))))
      (setf (f2cl-lib:fref q-%data% (jr j) ((1 ldq) (1 *))
                           q-%offset%)
              (-
               (f2cl-lib:fref q-%data% (jr j) ((1 ldq) (1 *))
                              q-%offset%)
               temp))
      (setf (f2cl-lib:fref q-%data%
                           (jr (f2cl-lib:int-add j 1))
                           ((1 ldq) (1 *)) q-%offset%)
              (-
               (f2cl-lib:fref q-%data%
                              (jr (f2cl-lib:int-add j 1))
                              ((1 ldq) (1 *)) q-%offset%)
               (* temp (f2cl-lib:fref v (2) ((1 3))))))
      (setf (f2cl-lib:fref q-%data%
                           (jr (f2cl-lib:int-add j 2))
                           ((1 ldq) (1 *)) q-%offset%)
              (-
               (f2cl-lib:fref q-%data%
                              (jr (f2cl-lib:int-add j 2))
                              ((1 ldq) (1 *)) q-%offset%)
               (* temp (f2cl-lib:fref v (3) ((1 3))))))
     label240))))
(setf ilpivt f2cl-lib:%false%)
(setf temp
        (max
         (abs
          (f2cl-lib:fref t$-%data%
                         ((f2cl-lib:int-add j 1)
                          (f2cl-lib:int-add j 1))
                         ((1 ldt) (1 *)) t$-%offset%))
         (abs
          (f2cl-lib:fref t$-%data%
                         ((f2cl-lib:int-add j 1)
                          (f2cl-lib:int-add j 2))
                         ((1 ldt) (1 *)) t$-%offset%))))
(setf temp2
        (max
```

```
            (abs
             (f2cl-lib:fref t$-%data%
                            ((f2cl-lib:int-add j 2)
                             (f2cl-lib:int-add j 1))
                            ((1 ldt) (1 *)) t$-%offset%))
            (abs
             (f2cl-lib:fref t$-%data%
                            ((f2cl-lib:int-add j 2)
                             (f2cl-lib:int-add j 2))
                            ((1 ldt) (1 *)) t$-%offset%))))
    (cond
     ((< (max temp temp2) safmin) (setf scale zero) (setf u1 one)
      (setf u2 zero) (go label250))
     ((>= temp temp2)
      (setf w11
              (f2cl-lib:fref t$-%data%
                             ((f2cl-lib:int-add j 1)
                              (f2cl-lib:int-add j 1))
                             ((1 ldt) (1 *)) t$-%offset%))
      (setf w21
              (f2cl-lib:fref t$-%data%
                             ((f2cl-lib:int-add j 2)
                              (f2cl-lib:int-add j 1))
                             ((1 ldt) (1 *)) t$-%offset%))
      (setf w12
              (f2cl-lib:fref t$-%data%
                             ((f2cl-lib:int-add j 1)
                              (f2cl-lib:int-add j 2))
                             ((1 ldt) (1 *)) t$-%offset%))
      (setf w22
              (f2cl-lib:fref t$-%data%
                             ((f2cl-lib:int-add j 2)
                              (f2cl-lib:int-add j 2))
                             ((1 ldt) (1 *)) t$-%offset%))
      (setf u1
              (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 1) j)
                             ((1 ldt) (1 *)) t$-%offset%))
      (setf u2
              (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 2) j)
                             ((1 ldt) (1 *)) t$-%offset%)))
     (t
      (setf w21
              (f2cl-lib:fref t$-%data%
                             ((f2cl-lib:int-add j 1)
                              (f2cl-lib:int-add j 1))
                             ((1 ldt) (1 *)) t$-%offset%))
      (setf w11
              (f2cl-lib:fref t$-%data%
                             ((f2cl-lib:int-add j 2)
                              (f2cl-lib:int-add j 1))
```

```
                          ((1 ldt) (1 *)) t$-%offset%))
    (setf w22
            (f2cl-lib:fref t$-%data%
                          ((f2cl-lib:int-add j 1)
                           (f2cl-lib:int-add j 2))
                          ((1 ldt) (1 *)) t$-%offset%))
    (setf w12
            (f2cl-lib:fref t$-%data%
                          ((f2cl-lib:int-add j 2)
                           (f2cl-lib:int-add j 2))
                          ((1 ldt) (1 *)) t$-%offset%))
    (setf u2
            (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 1) j)
                          ((1 ldt) (1 *)) t$-%offset%))
    (setf u1
            (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 2) j)
                          ((1 ldt) (1 *)) t$-%offset%))))
  (cond
   ((> (abs w12) (abs w11)) (setf ilpivt f2cl-lib:%true%)
    (setf temp w12) (setf temp2 w22) (setf w12 w11)
    (setf w22 w21) (setf w11 temp) (setf w21 temp2)))
  (setf temp (/ w21 w11))
  (setf u2 (- u2 (* temp u1)))
  (setf w22 (- w22 (* temp w12)))
  (setf w21 zero)
  (setf scale one)
  (cond
   ((< (abs w22) safmin) (setf scale zero) (setf u2 one)
    (setf u1 (/ (- w12) w11)) (go label250)))
  (if (< (abs w22) (abs u2))
      (setf scale (abs (/ w22 u2))))
  (if (< (abs w11) (abs u1))
      (setf scale (min scale (abs (/ w11 u1)))))
  (setf u2 (/ (* scale u2) w22))
  (setf u1 (/ (- (* scale u1) (* w12 u2)) w11))
 label250
  (cond (ilpivt (setf temp u2) (setf u2 u1) (setf u1 temp)))
  (setf t1
          (f2cl-lib:fsqrt
           (+ (expt scale 2) (expt u1 2) (expt u2 2))))
  (setf tau (+ one (/ scale t1)))
  (setf vs (/ (- one) (+ scale t1)))
  (setf (f2cl-lib:fref v (1) ((1 3))) one)
  (setf (f2cl-lib:fref v (2) ((1 3))) (* vs u1))
  (setf (f2cl-lib:fref v (3) ((1 3))) (* vs u2))
  (f2cl-lib:fdo (jr ifrstm (f2cl-lib:int-add jr 1))
                ((> jr
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add j 3))
```

```
                    (the f2cl-lib:integer4 ilast)))
                nil)
  (tagbody
    (setf temp
            (* tau
               (+
                (f2cl-lib:fref h-%data% (jr j)
                               ((1 ldh) (1 *)) h-%offset%)
                (* (f2cl-lib:fref v (2) ((1 3)))
                   (f2cl-lib:fref h-%data%
                                  (jr (f2cl-lib:int-add j 1))
                                  ((1 ldh) (1 *))
                                  h-%offset%))
                (* (f2cl-lib:fref v (3) ((1 3)))
                   (f2cl-lib:fref h-%data%
                                  (jr (f2cl-lib:int-add j 2))
                                  ((1 ldh) (1 *))
                                  h-%offset%)))))
    (setf (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                         h-%offset%)
            (-
             (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                            h-%offset%)
             temp))
    (setf (f2cl-lib:fref h-%data% (jr (f2cl-lib:int-add j 1))
                         ((1 ldh) (1 *)) h-%offset%)
            (-
             (f2cl-lib:fref h-%data%
                            (jr (f2cl-lib:int-add j 1))
                            ((1 ldh) (1 *)) h-%offset%)
             (* temp (f2cl-lib:fref v (2) ((1 3))))))
    (setf (f2cl-lib:fref h-%data% (jr (f2cl-lib:int-add j 2))
                         ((1 ldh) (1 *)) h-%offset%)
            (-
             (f2cl-lib:fref h-%data%
                            (jr (f2cl-lib:int-add j 2))
                            ((1 ldh) (1 *)) h-%offset%)
             (* temp (f2cl-lib:fref v (3) ((1 3))))))
   label260))
(f2cl-lib:fdo (jr ifrstm (f2cl-lib:int-add jr 1))
              ((> jr (f2cl-lib:int-add j 2)) nil)
  (tagbody
    (setf temp
            (* tau
               (+
                (f2cl-lib:fref t$-%data% (jr j)
                               ((1 ldt) (1 *)) t$-%offset%)
                (* (f2cl-lib:fref v (2) ((1 3)))
                   (f2cl-lib:fref t$-%data%
                                  (jr (f2cl-lib:int-add j 1))
```

```
                                        ((1 ldt) (1 *))
                                        t$-%offset%))
                              (* (f2cl-lib:fref v (3) ((1 3)))
                                 (f2cl-lib:fref t$-%data%
                                        (jr (f2cl-lib:int-add j 2))
                                        ((1 ldt) (1 *))
                                        t$-%offset%)))))
                (setf (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                                     t$-%offset%)
                      (-
                       (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                                      t$-%offset%)
                       temp))
                (setf (f2cl-lib:fref t$-%data%
                                     (jr (f2cl-lib:int-add j 1))
                                     ((1 ldt) (1 *)) t$-%offset%)
                      (-
                       (f2cl-lib:fref t$-%data%
                                      (jr (f2cl-lib:int-add j 1))
                                      ((1 ldt) (1 *)) t$-%offset%)
                       (* temp (f2cl-lib:fref v (2) ((1 3))))))
                (setf (f2cl-lib:fref t$-%data%
                                     (jr (f2cl-lib:int-add j 2))
                                     ((1 ldt) (1 *)) t$-%offset%)
                      (-
                       (f2cl-lib:fref t$-%data%
                                      (jr (f2cl-lib:int-add j 2))
                                      ((1 ldt) (1 *)) t$-%offset%)
                       (* temp (f2cl-lib:fref v (3) ((1 3))))))
              label270))
            (cond
             (ilz
              (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                            ((> jr n) nil)
                (tagbody
                  (setf temp
                          (* tau
                             (+
                              (f2cl-lib:fref z-%data% (jr j)
                                             ((1 ldz) (1 *)) z-%offset%)
                              (* (f2cl-lib:fref v (2) ((1 3)))
                                 (f2cl-lib:fref z-%data%
                                                (jr
                                                 (f2cl-lib:int-add j 1))
                                                ((1 ldz) (1 *))
                                                z-%offset%))
                              (* (f2cl-lib:fref v (3) ((1 3)))
                                 (f2cl-lib:fref z-%data%
                                                (jr
                                                 (f2cl-lib:int-add j 2))
```

```
                                        ((1 ldz) (1 *))
                                        z-%offset%)))))
            (setf (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                 z-%offset%)
                  (-
                   (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                  z-%offset%)
                   temp))
            (setf (f2cl-lib:fref z-%data%
                                 (jr (f2cl-lib:int-add j 1))
                                 ((1 ldz) (1 *)) z-%offset%)
                  (-
                   (f2cl-lib:fref z-%data%
                                  (jr (f2cl-lib:int-add j 1))
                                  ((1 ldz) (1 *)) z-%offset%)
                   (* temp (f2cl-lib:fref v (2) ((1 3))))))
            (setf (f2cl-lib:fref z-%data%
                                 (jr (f2cl-lib:int-add j 2))
                                 ((1 ldz) (1 *)) z-%offset%)
                  (-
                   (f2cl-lib:fref z-%data%
                                  (jr (f2cl-lib:int-add j 2))
                                  ((1 ldz) (1 *)) z-%offset%)
                   (* temp (f2cl-lib:fref v (3) ((1 3))))))
          label280))))
    (setf (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 1) j)
                          ((1 ldt) (1 *)) t$-%offset%)
            zero)
    (setf (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 2) j)
                          ((1 ldt) (1 *)) t$-%offset%)
            zero)
   label290))
(setf j (f2cl-lib:int-sub ilast 1))
(setf temp
        (f2cl-lib:fref h-%data% (j (f2cl-lib:int-sub j 1))
                       ((1 ldh) (1 *)) h-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (slartg temp
     (f2cl-lib:array-slice h-%data% single-float
                           ((+ j 1) (f2cl-lib:int-sub j 1))
                           ((1 ldh) (1 *)) h-%offset%)
     c s
     (f2cl-lib:array-slice h-%data% single-float
                           (j (f2cl-lib:int-sub j 1))
                           ((1 ldh) (1 *)) h-%offset%))
  (declare (ignore var-1 var-4))
  (when var-0 (setf temp var-0))
  (when var-2 (setf c var-2))
  (when var-3 (setf s var-3)))
(setf (f2cl-lib:fref h-%data%
```

```
                        ((f2cl-lib:int-add j 1)
                         (f2cl-lib:int-sub j 1))
                        ((1 ldh) (1 *)) h-%offset%)
        zero)
(f2cl-lib:fdo (jc j (f2cl-lib:int-add jc 1))
              ((> jc ilastm) nil)
  (tagbody
    (setf temp
            (+
             (* c
                (f2cl-lib:fref h-%data% (j jc) ((1 ldh) (1 *))
                               h-%offset%))
             (* s
                (f2cl-lib:fref h-%data%
                               ((f2cl-lib:int-add j 1) jc)
                               ((1 ldh) (1 *)) h-%offset%))))
    (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add j 1) jc)
                         ((1 ldh) (1 *)) h-%offset%)
            (+
             (* (- s)
                (f2cl-lib:fref h-%data% (j jc) ((1 ldh) (1 *))
                               h-%offset%))
             (* c
                (f2cl-lib:fref h-%data%
                               ((f2cl-lib:int-add j 1) jc)
                               ((1 ldh) (1 *)) h-%offset%))))
    (setf (f2cl-lib:fref h-%data% (j jc) ((1 ldh) (1 *))
                         h-%offset%)
            temp)
    (setf temp2
            (+
             (* c
                (f2cl-lib:fref t$-%data% (j jc) ((1 ldt) (1 *))
                               t$-%offset%))
             (* s
                (f2cl-lib:fref t$-%data%
                               ((f2cl-lib:int-add j 1) jc)
                               ((1 ldt) (1 *)) t$-%offset%))))
    (setf (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 1) jc)
                         ((1 ldt) (1 *)) t$-%offset%)
            (+
             (* (- s)
                (f2cl-lib:fref t$-%data% (j jc) ((1 ldt) (1 *))
                               t$-%offset%))
             (* c
                (f2cl-lib:fref t$-%data%
                               ((f2cl-lib:int-add j 1) jc)
                               ((1 ldt) (1 *)) t$-%offset%))))
    (setf (f2cl-lib:fref t$-%data% (j jc) ((1 ldt) (1 *))
                         t$-%offset%)
```

```
            temp2)
   label300))
(cond
 (ilq
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
      (setf temp
              (+
               (* c
                  (f2cl-lib:fref q-%data% (jr j) ((1 ldq) (1 *))
                                 q-%offset%))
               (* s
                  (f2cl-lib:fref q-%data%
                                 (jr (f2cl-lib:int-add j 1))
                                 ((1 ldq) (1 *)) q-%offset%))))
      (setf (f2cl-lib:fref q-%data% (jr (f2cl-lib:int-add j 1))
                           ((1 ldq) (1 *)) q-%offset%)
              (+
               (* (- s)
                  (f2cl-lib:fref q-%data% (jr j) ((1 ldq) (1 *))
                                 q-%offset%))
               (* c
                  (f2cl-lib:fref q-%data%
                                 (jr (f2cl-lib:int-add j 1))
                                 ((1 ldq) (1 *)) q-%offset%))))
      (setf (f2cl-lib:fref q-%data% (jr j) ((1 ldq) (1 *))
                           q-%offset%)
              temp)
     label310))))
(setf temp
        (f2cl-lib:fref t$-%data%
                       ((f2cl-lib:int-add j 1)
                        (f2cl-lib:int-add j 1))
                       ((1 ldt) (1 *)) t$-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (slartg temp
     (f2cl-lib:array-slice t$-%data% single-float ((+ j 1) j)
                           ((1 ldt) (1 *)) t$-%offset%)
     c s
     (f2cl-lib:array-slice t$-%data% single-float
                           ((+ j 1) (f2cl-lib:int-add j 1))
                           ((1 ldt) (1 *)) t$-%offset%))
  (declare (ignore var-1 var-4))
  (when var-0 (setf temp var-0))
  (when var-2 (setf c var-2))
  (when var-3 (setf s var-3)))
(setf (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 1) j)
                     ((1 ldt) (1 *)) t$-%offset%)
        zero)
```

```
(f2cl-lib:fdo (jr ifrstm (f2cl-lib:int-add jr 1))
              ((> jr ilast) nil)
  (tagbody
    (setf temp
            (+
             (* c
                (f2cl-lib:fref h-%data%
                               (jr (f2cl-lib:int-add j 1))
                               ((1 ldh) (1 *)) h-%offset%))
             (* s
                (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                               h-%offset%))))
    (setf (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                         h-%offset%)
            (+
             (* (- s)
                (f2cl-lib:fref h-%data%
                               (jr (f2cl-lib:int-add j 1))
                               ((1 ldh) (1 *)) h-%offset%))
             (* c
                (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                               h-%offset%))))
    (setf (f2cl-lib:fref h-%data% (jr (f2cl-lib:int-add j 1))
                         ((1 ldh) (1 *)) h-%offset%)
            temp)
   label320))
(f2cl-lib:fdo (jr ifrstm (f2cl-lib:int-add jr 1))
              ((> jr
                  (f2cl-lib:int-add ilast (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf temp
            (+
             (* c
                (f2cl-lib:fref t$-%data%
                               (jr (f2cl-lib:int-add j 1))
                               ((1 ldt) (1 *)) t$-%offset%))
             (* s
                (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                               t$-%offset%))))
    (setf (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                         t$-%offset%)
            (+
             (* (- s)
                (f2cl-lib:fref t$-%data%
                               (jr (f2cl-lib:int-add j 1))
                               ((1 ldt) (1 *)) t$-%offset%))
             (* c
                (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                               t$-%offset%))))
```

```
            (setf (f2cl-lib:fref t$-%data% (jr (f2cl-lib:int-add j 1))
                                 ((1 ldt) (1 *)) t$-%offset%)
                  temp)
          label330))
        (cond
         (ilz
          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                        ((> jr n) nil)
            (tagbody
              (setf temp
                      (+
                       (* c
                          (f2cl-lib:fref z-%data%
                                         (jr (f2cl-lib:int-add j 1))
                                         ((1 ldz) (1 *)) z-%offset%))
                       (* s
                          (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                         z-%offset%))))
              (setf (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                   z-%offset%)
                      (+
                       (* (- s)
                          (f2cl-lib:fref z-%data%
                                         (jr (f2cl-lib:int-add j 1))
                                         ((1 ldz) (1 *)) z-%offset%))
                       (* c
                          (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                         z-%offset%))))
              (setf (f2cl-lib:fref z-%data% (jr (f2cl-lib:int-add j 1))
                                   ((1 ldz) (1 *)) z-%offset%)
                      temp)
            label340))))))
      (go label350)
     label350
     label360))
  (setf info ilast)
  (go label420)
 label380
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (cond
       ((< (f2cl-lib:fref t$ (j j) ((1 ldt) (1 *))) zero)
        (cond
         (ilschr
          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                        ((> jr j) nil)
            (tagbody
              (setf (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                                   h-%offset%)
```

```
                          (-
                           (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                                          h-%offset%)))
                   (setf (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                                        t$-%offset%)
                          (-
                           (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                                          t$-%offset%)))
                  label390)))
                (t
                 (setf (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *)) h-%offset%)
                        (-
                         (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *))
                                        h-%offset%)))
                 (setf (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                                      t$-%offset%)
                        (-
                         (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                                        t$-%offset%)))))
               (cond
                (ilz
                 (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                               ((> jr n) nil)
                   (tagbody
                     (setf (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                          z-%offset%)
                          (-
                           (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                          z-%offset%)))
                  label400))))))
             (setf (f2cl-lib:fref alphar-%data% (j) ((1 *)) alphar-%offset%)
                     (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *)) h-%offset%))
             (setf (f2cl-lib:fref alphai-%data% (j) ((1 *)) alphai-%offset%)
                     zero)
             (setf (f2cl-lib:fref beta-%data% (j) ((1 *)) beta-%offset%)
                     (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                                    t$-%offset%))
           label410))
         (setf info 0)
        label420
         (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                 (f2cl-lib:freal n))
         (go end_label)
        end_label
         (return
          (values job compq compz n nil nil nil ldh nil ldt nil nil nil nil ldq
                  nil ldz nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
```

```
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::shgeqz
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::job
                                              fortran-to-lisp::compq
                                              fortran-to-lisp::compz
                                              fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::ldh nil
                                              fortran-to-lisp::ldt nil nil nil
                                              nil fortran-to-lisp::ldq nil
                                              fortran-to-lisp::ldz nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

33.35 sla_geamv

33.35.1 Axiom unit tests

— slageamv.input —

```
)set break resume
)sys rm -f slageamv.output
)spool slageamv.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.35.2 Axiom help page

— **sla_geamv.help** —

```
==================================================================
sla_geamv examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sla_geamv (TRANS, M, N, ALPHA, A, LDA, X, INCX, BETA, Y,
           INCY)
           SLA_GEAMV computes a matrix-vector product using a general matrix
           to calculate error bounds.

Function/Subroutine Documentation
   subroutine sla_geamv (integer TRANS, integer M, integer N, real ALPHA,
       real, dimension( lda, * ) A, integer LDA, real, dimension( * ) X,
       integer INCX, real BETA, real, dimension( * ) Y, integer INCY)
       SLA_GEAMV computes a matrix-vector product using a general matrix to
       calculate error bounds.

Purpose:
   SLA_GEAMV  performs one of the matrix-vector operations

           y := alpha*abs(A)*abs(x) + beta*abs(y),
      or   y := alpha*abs(A)**T*abs(x) + beta*abs(y),

   where alpha and beta are scalars, x and y are vectors and A is an
   m by n matrix.

   This function is primarily used in calculating error bounds.
   To protect against underflow during evaluation, components in
   the resulting vector are perturbed away from zero by (N+1)
   times the underflow threshold.  To prevent unnecessarily large
```

```
   errors for block-structure embedded in general matrices,
   "symbolically" zero components are not perturbed.  A zero
   entry is considered "symbolic" if all multiplications involved
   in computing that entry have at least one zero multiplicand.

Parameters:

 TRANS
   TRANS is INTEGER
   On entry, TRANS specifies the operation to be performed as
   follows:

   BLAS_NO_TRANS      y := alpha*abs(A)*abs(x) + beta*abs(y)
   BLAS_TRANS         y := alpha*abs(A**T)*abs(x) + beta*abs(y)
   BLAS_CONJ_TRANS    y := alpha*abs(A**T)*abs(x) + beta*abs(y)

   Unchanged on exit.

 M
   M is INTEGER
   On entry, M specifies the number of rows of the matrix A.
   M must be at least zero.
   Unchanged on exit.

 N
   N is INTEGER
   On entry, N specifies the number of columns of the matrix A.
   N must be at least zero.
   Unchanged on exit.

 ALPHA
   ALPHA is REAL
   On entry, ALPHA specifies the scalar alpha.
   Unchanged on exit.

 A
   A is REAL array of DIMENSION ( LDA, n )
   Before entry, the leading m by n part of the array A must
   contain the matrix of coefficients.
   Unchanged on exit.

 LDA
   LDA is INTEGER
   On entry, LDA specifies the first dimension of A as declared
   in the calling (sub) program. LDA must be at least
   max( 1, m ).
   Unchanged on exit.

 X
   X is REAL array, dimension
```

```
  ( 1 + ( n - 1 )*abs( INCX ) ) when TRANS = 'N' or 'n' and at least
  ( 1 + ( m - 1 )*abs( INCX ) ) otherwise.
  Before entry, the incremented array X must contain the vector x.
  Unchanged on exit.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of
  X. INCX must not be zero.
  Unchanged on exit.

BETA
  BETA is REAL
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.
  Unchanged on exit.

Y
  Y is REAL
  Array of DIMENSION at least
  ( 1 + ( m - 1 )*abs( INCY ) ) when TRANS = 'N' or 'n'
  and at least
  ( 1 + ( n - 1 )*abs( INCY ) ) otherwise.
  Before entry with BETA non-zero, the incremented array Y
  must contain the vector y. On exit, Y is overwritten by the
  updated vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of
  Y. INCY must not be zero.
  Unchanged on exit.
```

———-

33.35.3 fortran code

— sla_geamv.f —

```
*  =====================================================================
      SUBROUTINE SLA_GEAMV ( TRANS, M, N, ALPHA, A, LDA, X, INCX, BETA,
     $                       Y, INCY )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
```

```
*
*     .. Scalar Arguments ..
      REAL               ALPHA, BETA
      INTEGER            INCX, INCY, LDA, M, N, TRANS
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), X( * ), Y( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            SYMB_ZERO
      REAL               TEMP, SAFE1
      INTEGER            I, INFO, IY, J, JX, KX, KY, LENX, LENY
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, SLAMCH
      REAL               SLAMCH
*     ..
*     .. External Functions ..
      EXTERNAL           ILATRANS
      INTEGER            ILATRANS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, ABS, SIGN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF     ( .NOT.( ( TRANS.EQ.ILATRANS( 'N' ) )
     $           .OR. ( TRANS.EQ.ILATRANS( 'T' ) )
     $           .OR. ( TRANS.EQ.ILATRANS( 'C' )) ) ) THEN
         INFO = 1
      ELSE IF( M.LT.0 )THEN
         INFO = 2
      ELSE IF( N.LT.0 )THEN
         INFO = 3
      ELSE IF( LDA.LT.MAX( 1, M ) )THEN
         INFO = 6
      ELSE IF( INCX.EQ.0 )THEN
         INFO = 8
      ELSE IF( INCY.EQ.0 )THEN
         INFO = 11
```

```
      END IF
      IF( INFO.NE.0 )THEN
         CALL XERBLA( 'SLA_GEAMV ', INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( ( M.EQ.0 ).OR.( N.EQ.0 ).OR.
     $    ( ( ALPHA.EQ.ZERO ).AND.( BETA.EQ.ONE ) ) )
     $   RETURN
*
*     Set  LENX  and  LENY, the lengths of the vectors x and y, and set
*     up the start points in  X  and  Y.
*
      IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
         LENX = N
         LENY = M
      ELSE
         LENX = M
         LENY = N
      END IF
      IF( INCX.GT.0 )THEN
         KX = 1
      ELSE
         KX = 1 - ( LENX - 1 )*INCX
      END IF
      IF( INCY.GT.0 )THEN
         KY = 1
      ELSE
         KY = 1 - ( LENY - 1 )*INCY
      END IF
*
*     Set SAFE1 essentially to be the underflow threshold times the
*     number of additions in each row.
*
      SAFE1 = SLAMCH( 'Safe minimum' )
      SAFE1 = (N+1)*SAFE1
*
*     Form  y := alpha*abs(A)*abs(x) + beta*abs(y).
*
*     The O(M*N) SYMB_ZERO tests could be replaced by O(N) queries to
*     the inexact flag.  Still doesn't help change the iteration order
*     to per-column.
*
      IY = KY
      IF ( INCX.EQ.1 ) THEN
         IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
            DO I = 1, LENY
               IF ( BETA .EQ. ZERO ) THEN
```

```
            SYMB_ZERO = .TRUE.
            Y( IY ) = 0.0
         ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
            SYMB_ZERO = .TRUE.
         ELSE
            SYMB_ZERO = .FALSE.
            Y( IY ) = BETA * ABS( Y( IY ) )
         END IF
         IF ( ALPHA .NE. ZERO ) THEN
            DO J = 1, LENX
               TEMP = ABS( A( I, J ) )
               SYMB_ZERO = SYMB_ZERO .AND.
$                   ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

               Y( IY ) = Y( IY ) + ALPHA*ABS( X( J ) )*TEMP
            END DO
         END IF

         IF ( .NOT.SYMB_ZERO )
$              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

         IY = IY + INCY
      END DO
   ELSE
      DO I = 1, LENY
         IF ( BETA .EQ. ZERO ) THEN
            SYMB_ZERO = .TRUE.
            Y( IY ) = 0.0
         ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
            SYMB_ZERO = .TRUE.
         ELSE
            SYMB_ZERO = .FALSE.
            Y( IY ) = BETA * ABS( Y( IY ) )
         END IF
         IF ( ALPHA .NE. ZERO ) THEN
            DO J = 1, LENX
               TEMP = ABS( A( J, I ) )
               SYMB_ZERO = SYMB_ZERO .AND.
$                   ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

               Y( IY ) = Y( IY ) + ALPHA*ABS( X( J ) )*TEMP
            END DO
         END IF

         IF ( .NOT.SYMB_ZERO )
$              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

         IY = IY + INCY
      END DO
   END IF
```

```
 ELSE
    IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
       DO I = 1, LENY
          IF ( BETA .EQ. ZERO ) THEN
             SYMB_ZERO = .TRUE.
             Y( IY ) = 0.0
          ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
             SYMB_ZERO = .TRUE.
          ELSE
             SYMB_ZERO = .FALSE.
             Y( IY ) = BETA * ABS( Y( IY ) )
          END IF
          IF ( ALPHA .NE. ZERO ) THEN
             JX = KX
             DO J = 1, LENX
                TEMP = ABS( A( I, J ) )
                SYMB_ZERO = SYMB_ZERO .AND.
$                    ( X( JX ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                Y( IY ) = Y( IY ) + ALPHA*ABS( X( JX ) )*TEMP
                JX = JX + INCX
             END DO
          END IF

          IF (.NOT.SYMB_ZERO)
$              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

          IY = IY + INCY
       END DO
    ELSE
       DO I = 1, LENY
          IF ( BETA .EQ. ZERO ) THEN
             SYMB_ZERO = .TRUE.
             Y( IY ) = 0.0
          ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
             SYMB_ZERO = .TRUE.
          ELSE
             SYMB_ZERO = .FALSE.
             Y( IY ) = BETA * ABS( Y( IY ) )
          END IF
          IF ( ALPHA .NE. ZERO ) THEN
             JX = KX
             DO J = 1, LENX
                TEMP = ABS( A( J, I ) )
                SYMB_ZERO = SYMB_ZERO .AND.
$                    ( X( JX ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                Y( IY ) = Y( IY ) + ALPHA*ABS( X( JX ) )*TEMP
                JX = JX + INCX
             END DO
```

```
               END IF

               IF (.NOT.SYMB_ZERO)
     $              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

               IY = IY + INCY
            END DO
         END IF

      END IF
*
      RETURN
*
*     End of SLA_GEAMV
*
      END
```

33.35.4 lisp code

— sla_geamv.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun sla_geamv (trans m n alpha a lda x incx beta y incy)
    (declare (type (array single-float (*)) y x a)
             (type (single-float) beta alpha)
             (type (f2cl-lib:integer4) incy incx lda n m trans))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (x single-float x-%data% x-%offset%)
         (y single-float y-%data% y-%offset%))
      (prog ((i 0) (info 0) (iy 0) (j 0) (jx 0) (kx 0) (ky 0) (lenx 0) (leny 0)
             (temp 0.0) (safe1 0.0) (symb_zero nil))
        (declare (type (f2cl-lib:integer4) i info iy j jx kx ky lenx leny)
                 (type (single-float) temp safe1)
                 (type f2cl-lib:logical symb_zero))
        (setf info 0)
        (cond
         ((not
           (or (= trans (ilatrans "N")) (= trans (ilatrans "T"))
               (= trans (ilatrans "C"))))
          (setf info 1))
         ((< m 0) (setf info 2)) ((< n 0) (setf info 3))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
```

```
  (setf info 6))
 ((= incx 0) (setf info 8)) ((= incy 0) (setf info 11)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "SLA_GEAMV " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
    (go end_label))
(cond ((= trans (ilatrans "N")) (setf lenx n) (setf leny m))
      (t (setf lenx m) (setf leny n)))
(cond ((> incx 0) (setf kx 1))
      (t
       (setf kx
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub lenx 1) incx)))))
(cond ((> incy 0) (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub leny 1) incy)))))
(setf safe1 (slamch "Safe minimum"))
(setf safe1 (* (f2cl-lib:int-add n 1) safe1))
(setf iy ky)
(cond
 ((= incx 1)
  (cond
   ((= trans (ilatrans "N"))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i leny) nil)
      (tagbody
        (cond
         ((= beta zero) (setf symb_zero f2cl-lib:%true%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) 0.0))
         ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
          (setf symb_zero f2cl-lib:%true%))
         (t (setf symb_zero f2cl-lib:%false%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (* beta
                     (abs
                      (f2cl-lib:fref y-%data% (iy) ((1 *))
                                     y-%offset%))))))
        (cond
         ((/= alpha zero)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j lenx) nil)
```

```
            (tagbody
              (setf temp
                      (abs
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)))
              (setf symb_zero
                      (and symb_zero
                           (or
                            (=
                             (f2cl-lib:fref x-%data% (j) ((1 *))
                                            x-%offset%)
                             zero)
                            (= temp zero))))
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (+
                       (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (* alpha
                          (abs
                           (f2cl-lib:fref x-%data% (j) ((1 *))
                                          x-%offset%))
                          temp)))
             label100001))))
      (if (not symb_zero)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (f2cl-lib:sign safe1
                                    (f2cl-lib:fref y-%data% (iy)
                                                   ((1 *))
                                                   y-%offset%)))))
      (setf iy (f2cl-lib:int-add iy incy))
    label100000)))
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i leny) nil)
   (tagbody
     (cond
      ((= beta zero) (setf symb_zero f2cl-lib:%true%)
       (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) 0.0))
      ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
       (setf symb_zero f2cl-lib:%true%))
      (t (setf symb_zero f2cl-lib:%false%)
       (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
               (* beta
                  (abs
                   (f2cl-lib:fref y-%data% (iy) ((1 *))
                                  y-%offset%))))))
     (cond
      ((/= alpha zero)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j lenx) nil)
```

```
            (tagbody
              (setf temp
                      (abs
                       (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                      a-%offset%)))
              (setf symb_zero
                      (and symb_zero
                           (or
                            (=
                             (f2cl-lib:fref x-%data% (j) ((1 *))
                                            x-%offset%)
                             zero)
                            (= temp zero))))
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (+
                       (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (* alpha
                          (abs
                           (f2cl-lib:fref x-%data% (j) ((1 *))
                                          x-%offset%))
                          temp)))
             label100003))))
        (if (not symb_zero)
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (f2cl-lib:sign safe1
                                      (f2cl-lib:fref y-%data% (iy)
                                                     ((1 *))
                                                     y-%offset%)))))
        (setf iy (f2cl-lib:int-add iy incy))
      label100002)))))
(t
 (cond
  ((= trans (ilatrans "N"))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i leny) nil)
     (tagbody
       (cond
        ((= beta zero) (setf symb_zero f2cl-lib:%true%)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) 0.0))
        ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
         (setf symb_zero f2cl-lib:%true%))
        (t (setf symb_zero f2cl-lib:%false%)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (* beta
                    (abs
                     (f2cl-lib:fref y-%data% (iy) ((1 *))
                                    y-%offset%))))))
       (cond
        ((/= alpha zero) (setf jx kx)
```

```
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j lenx) nil)
            (tagbody
              (setf temp
                      (abs
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)))
              (setf symb_zero
                      (and symb_zero
                           (or
                            (=
                             (f2cl-lib:fref x-%data% (jx) ((1 *))
                                            x-%offset%)
                             zero)
                            (= temp zero))))
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (+
                       (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (* alpha
                          (abs
                           (f2cl-lib:fref x-%data% (jx) ((1 *))
                                          x-%offset%))
                          temp)))
              (setf jx (f2cl-lib:int-add jx incx))
             label100005))))
      (if (not symb_zero)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (f2cl-lib:sign safe1
                                    (f2cl-lib:fref y-%data% (iy)
                                                   ((1 *))
                                                   y-%offset%)))))
      (setf iy (f2cl-lib:int-add iy incy))
     label100004)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i leny) nil)
    (tagbody
      (cond
       ((= beta zero) (setf symb_zero f2cl-lib:%true%)
        (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) 0.0))
       ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
        (setf symb_zero f2cl-lib:%true%))
       (t (setf symb_zero f2cl-lib:%false%)
        (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                (* beta
                   (abs
                    (f2cl-lib:fref y-%data% (iy) ((1 *))
                                   y-%offset%))))))
      (cond
```

```
                 ((/= alpha zero) (setf jx kx)
                  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                                ((> j lenx) nil)
                    (tagbody
                      (setf temp
                              (abs
                               (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                              a-%offset%)))
                      (setf symb_zero
                              (and symb_zero
                                   (or
                                    (=
                                     (f2cl-lib:fref x-%data% (jx) ((1 *))
                                                    x-%offset%)
                                     zero)
                                    (= temp zero))))
                      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                              (+
                               (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                               (* alpha
                                  (abs
                                   (f2cl-lib:fref x-%data% (jx) ((1 *))
                                                  x-%offset%))
                                  temp)))
                      (setf jx (f2cl-lib:int-add jx incx))
                     label100007))))
                (if (not symb_zero)
                    (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                            (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                               (f2cl-lib:sign safe1
                                              (f2cl-lib:fref y-%data% (iy)
                                                             ((1 *))
                                                             y-%offset%)))))
                (setf iy (f2cl-lib:int-add iy incy))
               label100006))))))
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sla_geamv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (single-float)
                                              (array single-float (*))
```

```
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (single-float)
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil nil nil nil nil nil
  nil nil nil)
:calls
'(fortran-to-lisp::ilatrans))))
```

———-

33.36 sla_gercond

33.36.1 Axiom unit tests

— **slagercond.input** —

```
)set break resume
)sys rm -f slagercond.output
)spool slagercond.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.36.2 Axiom help page

— **sla_gercond.help** —

```
==================================================================
sla_gercond examples
==================================================================

==================================================================
Man Page Details
```

```
====================================================================

SYNOPSIS
   Functions/Subroutines
       real function sla_gercond (TRANS, N, A, LDA, AF, LDAF, IPIV, CMODE, C,
           INFO, WORK, IWORK)
           SLA_GERCOND estimates the Skeel condition number for a general
           matrix.

Function/Subroutine Documentation
   real function sla_gercond (character TRANS, integer N, real, dimension(
       lda, * ) A, integer LDA, real, dimension( ldaf, * ) AF, integer LDAF,
       integer, dimension( * ) IPIV, integer CMODE, real, dimension( * ) C,
       integer INFO, real, dimension( * ) WORK, integer, dimension( * ) IWORK)
       SLA_GERCOND estimates the Skeel condition number for a general matrix.

Purpose:
   SLA_GERCOND estimates the Skeel condition number of op(A) * op2(C)
   where op2 is determined by CMODE as follows
   CMODE =  1    op2(C) = C
   CMODE =  0    op2(C) = I
   CMODE = -1    op2(C) = inv(C)
   The Skeel condition number cond(A) = norminf( |inv(A)||A| )
   is computed by computing scaling factors R such that
   diag(R)*A*op2(C) is row equilibrated and computing the standard
   infinity-norm condition number.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate Transpose = Transpose)

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
```

```
  AF is REAL array, dimension (LDAF,N)
  The factors L and U from the factorization
  A = P*L*U as computed by SGETRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from the factorization A = P*L*U
  as computed by SGETRF; row i of the matrix was interchanged
  with row IPIV(i).

CMODE
  CMODE is INTEGER
  Determines op2(C) in the formula op(A) * op2(C) as follows:
  CMODE =  1    op2(C) = C
  CMODE =  0    op2(C) = I
  CMODE = -1    op2(C) = inv(C)

C
  C is REAL array, dimension (N)
  The vector C in the formula op(A) * op2(C).

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  i > 0:  The ith argument is invalid.

WORK
  WORK is REAL array, dimension (3*N).
  Workspace.

IWORK
  IWORK is INTEGER array, dimension (N).
  Workspace.2
```

———-

33.36.3 fortran code

— sla_gercond.f —

```
*  =====================================================================
      REAL FUNCTION SLA_GERCOND ( TRANS, N, A, LDA, AF, LDAF, IPIV,
     $                            CMODE, C, INFO, WORK, IWORK )
```

```
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            N, LDA, LDAF, INFO, CMODE
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      REAL               A( LDA, * ), AF( LDAF, * ), WORK( * ),
     $                   C( * )
*    ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            NOTRANS
      INTEGER            KASE, I, J
      REAL               AINVNM, TMP
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLACN2, SGETRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
      SLA_GERCOND = 0.0
*
      INFO = 0
      NOTRANS = LSAME( TRANS, 'N' )
      IF ( .NOT. NOTRANS .AND. .NOT. LSAME(TRANS, 'T')
     $     .AND. .NOT. LSAME(TRANS, 'C') ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
```

```
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SLA_GERCOND', -INFO )
         RETURN
      END IF
      IF( N.EQ.0 ) THEN
         SLA_GERCOND = 1.0
         RETURN
      END IF
*
*     Compute the equilibration matrix R such that
*     inv(R)*A*C has unit 1-norm.
*
      IF (NOTRANS) THEN
         DO I = 1, N
            TMP = 0.0
            IF ( CMODE .EQ. 1 ) THEN
               DO J = 1, N
                  TMP = TMP + ABS( A( I, J ) * C( J ) )
               END DO
            ELSE IF ( CMODE .EQ. 0 ) THEN
               DO J = 1, N
                  TMP = TMP + ABS( A( I, J ) )
               END DO
            ELSE
               DO J = 1, N
                  TMP = TMP + ABS( A( I, J ) / C( J ) )
               END DO
            END IF
            WORK( 2*N+I ) = TMP
         END DO
      ELSE
         DO I = 1, N
            TMP = 0.0
            IF ( CMODE .EQ. 1 ) THEN
               DO J = 1, N
                  TMP = TMP + ABS( A( J, I ) * C( J ) )
               END DO
            ELSE IF ( CMODE .EQ. 0 ) THEN
               DO J = 1, N
                  TMP = TMP + ABS( A( J, I ) )
               END DO
            ELSE
               DO J = 1, N
                  TMP = TMP + ABS( A( J, I ) / C( J ) )
               END DO
            END IF
            WORK( 2*N+I ) = TMP
         END DO
```

```
      END IF
*
*     Estimate the norm of inv(op(A)).
*
      AINVNM = 0.0

      KASE = 0
   10 CONTINUE
      CALL SLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*
*           Multiply by R.
*
            DO I = 1, N
               WORK(I) = WORK(I) * WORK(2*N+I)
            END DO

            IF (NOTRANS) THEN
               CALL SGETRS( 'No transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ELSE
               CALL SGETRS( 'Transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            END IF
*
*           Multiply by inv(C).
*
            IF ( CMODE .EQ. 1 ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) / C( I )
               END DO
            ELSE IF ( CMODE .EQ. -1 ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF
         ELSE
*
*           Multiply by inv(C**T).
*
            IF ( CMODE .EQ. 1 ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) / C( I )
               END DO
            ELSE IF ( CMODE .EQ. -1 ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF
```

```
            IF (NOTRANS) THEN
               CALL SGETRS( 'Transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ELSE
               CALL SGETRS( 'No transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            END IF
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * WORK( 2*N+I )
            END DO
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM .NE. 0.0 )
     $   SLA_GERCOND = ( 1.0 / AINVNM )
*
      RETURN
*
      END
```

———-

33.36.4 lisp code

— **sla_gercond.lisp** —

```
(defun sla_gercond (trans n a lda af ldaf ipiv cmode c info work iwork)
  (declare (type (array f2cl-lib:integer4 (*)) iwork ipiv)
           (type (array single-float (*)) work c af a)
           (type (f2cl-lib:integer4) info cmode ldaf lda n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (a single-float a-%data% a-%offset%)
       (af single-float af-%data% af-%offset%)
       (c single-float c-%data% c-%offset%)
       (work single-float work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
    (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4)) (ainvnm 0.0)
           (tmp 0.0) (kase 0) (i 0) (j 0) (notrans nil) (sla_gercond 0.0))
```

```
(declare (type f2cl-lib:logical notrans)
         (type (f2cl-lib:integer4) j i kase)
         (type (single-float) sla_gercond tmp ainvnm)
         (type (array f2cl-lib:integer4 (3)) isave))
(setf sla_gercond 0.0)
(setf info 0)
(setf notrans
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((and (not notrans)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6)))
(cond
 ((/= info 0) (xerbla "SLA_GERCOND" (f2cl-lib:int-sub info))
  (go end_label)))
(cond ((= n 0) (setf sla_gercond 1.0) (go end_label)))
(cond
 (notrans
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf tmp 0.0)
      (cond
       ((= cmode 1)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (abs
```

```
                        (*
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)
                         (f2cl-lib:fref c-%data% (j) ((1 *))
                                        c-%offset%)))))
           label100001)))
       ((= cmode 0)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (abs
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))))
           label100002)))
       (t
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (abs
                        (/
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)
                         (f2cl-lib:fref c-%data% (j) ((1 *))
                                        c-%offset%)))))
           label100003))))
     (setf (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) i))
                          ((1 *)) work-%offset%)
             tmp)
    label100000)))
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf tmp 0.0)
     (cond
      ((= cmode 1)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (setf tmp
                   (+ tmp
                      (abs
                       (*
                        (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                       a-%offset%)
```

```
                              (f2cl-lib:fref c-%data% (j) ((1 *))
                                             c-%offset%)))))
            label100005)))
         ((= cmode 0)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf tmp
                      (+ tmp
                         (abs
                          (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                         a-%offset%))))
            label100006)))
         (t
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf tmp
                      (+ tmp
                         (abs
                          (/
                           (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                          a-%offset%)
                           (f2cl-lib:fref c-%data% (j) ((1 *))
                                          c-%offset%)))))
            label100007))))
       (setf (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) i))
                            ((1 *)) work-%offset%)
             tmp)
      label100004))))
 (setf ainvnm 0.0)
 (setf kase 0)
label10
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slacn2 n
      (f2cl-lib:array-slice work-%data% single-float ((+ n 1)) ((1 *))
                            work-%offset%)
      work iwork ainvnm kase isave)
   (declare (ignore var-1 var-2 var-3 var-6))
   (setf n var-0)
   (setf ainvnm var-4)
   (setf kase var-5))
 (cond
  ((/= kase 0)
   (cond
    ((= kase 2)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
```

```
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add
                                  (f2cl-lib:int-mul 2 n) i))
                                ((1 *)) work-%offset%)))
     label100008))
   (cond
    (notrans
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (sgetrs "No transpose" n 1 af ldaf ipiv work n info)
       (declare (ignore var-0 var-2 var-3 var-5 var-6))
       (setf n var-1)
       (setf ldaf var-4)
       (setf n var-7)
       (setf info var-8)))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (sgetrs "Transpose" n 1 af ldaf ipiv work n info)
       (declare (ignore var-0 var-2 var-3 var-5 var-6))
       (setf n var-1)
       (setf ldaf var-4)
       (setf n var-7)
       (setf info var-8))))
   (cond
    ((= cmode 1)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (/
                  (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
       label100009)))
    ((= cmode (f2cl-lib:int-sub 1))
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (*
                  (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
       label100010)))))
  (t
   (cond
    ((= cmode 1)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
```

```
            (tagbody
              (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                      (/
                       (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                       (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
             label100011)))
         ((= cmode (f2cl-lib:int-sub 1))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                      (*
                       (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                       (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
             label100012))))
        (cond
         (notrans
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (sgetrs "Transpose" n 1 af ldaf ipiv work n info)
            (declare (ignore var-0 var-2 var-3 var-5 var-6))
            (setf n var-1)
            (setf ldaf var-4)
            (setf n var-7)
            (setf info var-8)))
         (t
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (sgetrs "No transpose" n 1 af ldaf ipiv work n info)
            (declare (ignore var-0 var-2 var-3 var-5 var-6))
            (setf n var-1)
            (setf ldaf var-4)
            (setf n var-7)
            (setf info var-8))))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                    (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                       (f2cl-lib:fref work-%data%
                                      ((f2cl-lib:int-add
                                        (f2cl-lib:int-mul 2 n) i))
                                      ((1 *)) work-%offset%)))
           label100013))))
      (go label10)))
    (if (/= ainvnm 0.0)
        (setf sla_gercond (/ 1.0 ainvnm)))
    (go end_label)
   end_label
    (return
```

```
      (values sla_gercond trans n nil nil nil ldaf nil nil nil info nil
              nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sla_gercond
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*)))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::ldaf nil nil nil
                                              fortran-to-lisp::info nil nil)
                                            :calls
                                            '(fortran-to-lisp::sgetrs
                                              fortran-to-lisp::slacn2))))
```

———-

33.37 sla_gerfsx_extended

33.37.1 Axiom unit tests

— slagerfsxextended.input —

```
)set break resume
)sys rm -f slagerfsxextended.output
)spool slagerfsxextended.output
)set message test on
)set message auto off
```

```
)clear all

)spool
)lisp (bye)
```

———-

33.37.2 Axiom help page [Demm05]

— **sla_gerfsx_extended.help** —

```
==================================================================
sla_gerfsx_extended examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sla_gerfsx_extended (PREC_TYPE, TRANS_TYPE, N, NRHS, A, LDA,
           AF, LDAF, IPIV, COLEQU, C, B, LDB, Y, LDY, BERR_OUT, N_NORMS,
           ERRS_N, ERRS_C, RES, AYB, DY, Y_TAIL, RCOND, ITHRESH, RTHRESH,
           DZ_UB, IGNORE_CWISE, INFO)
           SLA_GERFSX_EXTENDED improves the computed solution to a system of
           linear equations for general matrices by performing extra-precise
           iterative refinement and provides error bounds and backward error
           estimates for the solution.

Function/Subroutine Documentation
   subroutine sla_gerfsx_extended (integer PREC_TYPE, integer TRANS_TYPE,
       integer N, integer NRHS, real, dimension( lda, * ) A, integer LDA,
       real, dimension( ldaf, * ) AF, integer LDAF, integer, dimension( * )
       IPIV, logical COLEQU, real, dimension( * ) C, real, dimension( ldb, * )
       B, integer LDB, real, dimension( ldy, * ) Y, integer LDY, real,
       dimension( * ) BERR_OUT, integer N_NORMS, real, dimension( nrhs, * )
       ERRS_N, real, dimension( nrhs, * ) ERRS_C, real, dimension( * ) RES,
       real, dimension( * ) AYB, real, dimension( * ) DY, real, dimension( * )
       Y_TAIL, real RCOND, integer ITHRESH, real RTHRESH, real DZ_UB, logical
       IGNORE_CWISE, integer INFO)
       SLA_GERFSX_EXTENDED improves the computed solution to a system of
       linear equations for general matrices by performing extra-precise
       iterative refinement and provides error bounds and backward error
       estimates for the solution.

Purpose:
   SLA_GERFSX_EXTENDED improves the computed solution to a system of
```

```
  linear equations by performing extra-precise iterative refinement
  and provides error bounds and backward error estimates for the solution.
  This subroutine is called by SGERFSX to perform iterative refinement.
  In addition to normwise error bound, the code provides maximum
  componentwise error bound if possible. See comments for ERRS_N
  and ERRS_C for details of the error bounds. Note that this
  subroutine is only resonsible for setting the second fields of
  ERRS_N and ERRS_C.

Parameters:

 PREC_TYPE
   PREC_TYPE is INTEGER
   Specifies the intermediate precision to be used in refinement.
   The value is defined by ILAPREC(P) where P is a CHARACTER and
   P    = 'S':  Single
        = 'D':  Double
        = 'I':  Indigenous
        = 'X', 'E':  Extra

 TRANS_TYPE
   TRANS_TYPE is INTEGER
   Specifies the transposition operation on A.
   The value is defined by ILATRANS(T) where T is a CHARACTER and
   T    = 'N':  No transpose
        = 'T':  Transpose
        = 'C':  Conjugate transpose

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right-hand-sides, i.e., the number of columns of the
   matrix B.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is REAL array, dimension (LDAF,N)
   The factors L and U from the factorization
   A = P*L*U as computed by SGETRF.
```

```
LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from the factorization A = P*L*U
  as computed by SGETRF; row i of the matrix was interchanged
  with row IPIV(i).

COLEQU
  COLEQU is LOGICAL
  If .TRUE. then column equilibration was done to A before calling
  this routine. This is needed to compute the solution and error
  bounds correctly.

C
  C is REAL array, dimension (N)
  The column scale factors for A. If COLEQU = .FALSE., C
  is not accessed. If C is input, each element of C should be a power
  of the radix to ensure a reliable solution and error estimates.
  Scaling by powers of the radix does not cause rounding errors unless
  the result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is REAL array, dimension (LDB,NRHS)
  The right-hand-side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

Y
  Y is REAL array, dimension (LDY,NRHS)
  On entry, the solution matrix X, as computed by SGETRS.
  On exit, the improved solution matrix Y.

LDY
  LDY is INTEGER
  The leading dimension of the array Y.  LDY >= max(1,N).

BERR_OUT
  BERR_OUT is REAL array, dimension (NRHS)
  On exit, BERR_OUT(j) contains the componentwise relative backward
  error for right-hand-side j from the formula
     max(i) ( abs(RES(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
  where abs(Z) is the componentwise absolute value of the matrix
  or vector Z. This is computed by SLA_LIN_BERR.
```

```
N_NORMS
  N_NORMS is INTEGER
  Determines which error bounds to return (see ERRS_N
  and ERRS_C).
  If N_NORMS >= 1 return normwise error bounds.
  If N_NORMS >= 2 return componentwise error bounds.

ERRS_N
  ERRS_N is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
            max_j (abs(XTRUE(j,i) - X(j,i)))
             ------------------------------
                     max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERRS_N(i,:) corresponds to the ith
  right-hand side.

  The second index in ERRS_N(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

ERRS_C
```

```
  ERRS_C is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                 abs(XTRUE(j,i) - X(j,i))
           max_j ----------------------
                       abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERRS_C is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERRS_C(i,:) corresponds to the ith
  right-hand side.

  The second index in ERRS_C(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

This subroutine is only responsible for setting the second field above.

RES
  RES is REAL array, dimension (N)
  Workspace to hold the intermediate residual.
```

```
AYB
  AYB is REAL array, dimension (N)
  Workspace. This can be the same workspace passed for Y_TAIL.

DY
  DY is REAL array, dimension (N)
  Workspace to hold the intermediate solution.

Y_TAIL
  Y_TAIL is REAL array, dimension (N)
  Workspace to hold the trailing bits of the intermediate solution.

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

ITHRESH
  ITHRESH is INTEGER
  The maximum number of residual computations allowed for
  refinement. The default is 10. For 'aggressive' set to 100 to
  permit convergence using approximate factorizations or
  factorizations other than LU. If the factorization uses a
  technique other than Gaussian elimination, the guarantees in
  ERRS_N and ERRS_C may no longer be trustworthy.

RTHRESH
  RTHRESH is REAL
  Determines when to stop refinement if the error estimate stops
  decreasing. Refinement will stop when the next solution no longer
  satisfies norm(dx_{i+1}) < RTHRESH * norm(dx_i) where norm(Z) is
  the infinity norm of Z. RTHRESH satisfies 0 < RTHRESH <= 1. The
  default value is 0.5. For 'aggressive' set to 0.9 to permit
  convergence on extremely ill-conditioned matrices. See LAWN 165
  for more details.

DZ_UB
  DZ_UB is REAL
  Determines when to start considering componentwise convergence.
  Componentwise convergence is only considered after each component
  of the solution Y is stable, which we definte as the relative
  change in each component being less than DZ_UB. The default value
  is 0.25, requiring the first bit to be stable. See LAWN 165 for
  more details.

IGNORE_CWISE
```

```
  IGNORE_CWISE is LOGICAL
  If .TRUE. then ignore componentwise convergence. Default value
  is .FALSE..

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  < 0:  if INFO = -i, the ith argument to SGETRS had an illegal value
```

———-

33.37.3 fortran code

— **sla_gerfsx_extended.f** —

```
*  =====================================================================
      SUBROUTINE SLA_GERFSX_EXTENDED( PREC_TYPE, TRANS_TYPE, N, NRHS, A,
     $                                LDA, AF, LDAF, IPIV, COLEQU, C, B,
     $                                LDB, Y, LDY, BERR_OUT, N_NORMS,
     $                                ERRS_N, ERRS_C, RES,
     $                                AYB, DY, Y_TAIL, RCOND, ITHRESH,
     $                                RTHRESH, DZ_UB, IGNORE_CWISE,
     $                                INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDAF, LDB, LDY, N, NRHS, PREC_TYPE,
     $                   TRANS_TYPE, N_NORMS, ITHRESH
      LOGICAL            COLEQU, IGNORE_CWISE
      REAL               RTHRESH, DZ_UB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   Y( LDY, * ), RES( * ), DY( * ), Y_TAIL( * )
      REAL               C( * ), AYB( * ), RCOND, BERR_OUT( * ),
     $                   ERRS_N( NRHS, * ),
     $                   ERRS_C( NRHS, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
```

```
      CHARACTER          TRANS
      INTEGER            CNT, I, J, X_STATE, Z_STATE, Y_PREC_STATE
      REAL               YK, DYK, YMIN, NORMY, NORMX, NORMDX, DXRAT,
     $                   DZRAT, PREVNORMDX, PREV_DZ_Z, DXRATMAX,
     $                   DZRATMAX, DX_X, DZ_Z, FINAL_DX_X, FINAL_DZ_Z,
     $                   EPS, HUGEVAL, INCR_THRESH
      LOGICAL            INCR_PREC
*     ..
*     .. Parameters ..
      INTEGER            UNSTABLE_STATE, WORKING_STATE, CONV_STATE,
     $                   NOPROG_STATE, BASE_RESIDUAL, EXTRA_RESIDUAL,
     $                   EXTRA_Y
      PARAMETER          ( UNSTABLE_STATE = 0, WORKING_STATE = 1,
     $                   CONV_STATE = 2, NOPROG_STATE = 3 )
      PARAMETER          ( BASE_RESIDUAL = 0, EXTRA_RESIDUAL = 1,
     $                   EXTRA_Y = 2 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SAXPY, SCOPY, SGETRS, SGEMV, BLAS_SGEMV_X,
     $                   BLAS_SGEMV2_X, SLA_GEAMV, SLA_WWADDW, SLAMCH,
     $                   CHLA_TRANSTYPE, SLA_LIN_BERR
      REAL               SLAMCH
      CHARACTER          CHLA_TRANSTYPE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      IF ( INFO.NE.0 ) RETURN
      TRANS = CHLA_TRANSTYPE(TRANS_TYPE)
      EPS = SLAMCH( 'Epsilon' )
      HUGEVAL = SLAMCH( 'Overflow' )
```

```
*     Force HUGEVAL to Inf
      HUGEVAL = HUGEVAL * HUGEVAL
*     Using HUGEVAL may lead to spurious underflows.
      INCR_THRESH = REAL( N ) * EPS
*
      DO J = 1, NRHS
         Y_PREC_STATE = EXTRA_RESIDUAL
         IF ( Y_PREC_STATE .EQ. EXTRA_Y ) THEN
            DO I = 1, N
               Y_TAIL( I ) = 0.0
            END DO
         END IF

         DXRAT = 0.0
         DXRATMAX = 0.0
         DZRAT = 0.0
         DZRATMAX = 0.0
         FINAL_DX_X = HUGEVAL
         FINAL_DZ_Z = HUGEVAL
         PREVNORMDX = HUGEVAL
         PREV_DZ_Z = HUGEVAL
         DZ_Z = HUGEVAL
         DX_X = HUGEVAL

         X_STATE = WORKING_STATE
         Z_STATE = UNSTABLE_STATE
         INCR_PREC = .FALSE.

         DO CNT = 1, ITHRESH
*
*         Compute residual RES = B_s - op(A_s) * Y,
*             op(A) = A, A**T, or A**H depending on TRANS (and type).
*
            CALL SCOPY( N, B( 1, J ), 1, RES, 1 )
            IF ( Y_PREC_STATE .EQ. BASE_RESIDUAL ) THEN
               CALL SGEMV( TRANS, N, N, -1.0, A, LDA, Y( 1, J ), 1,
     $              1.0, RES, 1 )
            ELSE IF ( Y_PREC_STATE .EQ. EXTRA_RESIDUAL ) THEN
               CALL BLAS_SGEMV_X( TRANS_TYPE, N, N, -1.0, A, LDA,
     $              Y( 1, J ), 1, 1.0, RES, 1, PREC_TYPE )
            ELSE
               CALL BLAS_SGEMV2_X( TRANS_TYPE, N, N, -1.0, A, LDA,
     $              Y( 1, J ), Y_TAIL, 1, 1.0, RES, 1, PREC_TYPE )
            END IF

!        XXX: RES is no longer needed.
            CALL SCOPY( N, RES, 1, DY, 1 )
            CALL SGETRS( TRANS, N, 1, AF, LDAF, IPIV, DY, N, INFO )
*
*         Calculate relative changes DX_X, DZ_Z and ratios DXRAT, DZRAT.
```

```
*
            NORMX = 0.0
            NORMY = 0.0
            NORMDX = 0.0
            DZ_Z = 0.0
            YMIN = HUGEVAL
*
            DO I = 1, N
               YK = ABS( Y( I, J ) )
               DYK = ABS( DY( I ) )

               IF ( YK .NE. 0.0 ) THEN
                  DZ_Z = MAX( DZ_Z, DYK / YK )
               ELSE IF ( DYK .NE. 0.0 ) THEN
                  DZ_Z = HUGEVAL
               END IF

               YMIN = MIN( YMIN, YK )

               NORMY = MAX( NORMY, YK )

               IF ( COLEQU ) THEN
                  NORMX = MAX( NORMX, YK * C( I ) )
                  NORMDX = MAX( NORMDX, DYK * C( I ) )
               ELSE
                  NORMX = NORMY
                  NORMDX = MAX( NORMDX, DYK )
               END IF
            END DO

            IF ( NORMX .NE. 0.0 ) THEN
               DX_X = NORMDX / NORMX
            ELSE IF ( NORMDX .EQ. 0.0 ) THEN
               DX_X = 0.0
            ELSE
               DX_X = HUGEVAL
            END IF

            DXRAT = NORMDX / PREVNORMDX
            DZRAT = DZ_Z / PREV_DZ_Z
*
*         Check termination criteria
*
            IF (.NOT.IGNORE_CWISE
     $           .AND. YMIN*RCOND .LT. INCR_THRESH*NORMY
     $           .AND. Y_PREC_STATE .LT. EXTRA_Y)
     $           INCR_PREC = .TRUE.

            IF ( X_STATE .EQ. NOPROG_STATE .AND. DXRAT .LE. RTHRESH )
     $           X_STATE = WORKING_STATE
```

```
      IF ( X_STATE .EQ. WORKING_STATE ) THEN
         IF ( DX_X .LE. EPS ) THEN
            X_STATE = CONV_STATE
         ELSE IF ( DXRAT .GT. RTHRESH ) THEN
            IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
               INCR_PREC = .TRUE.
            ELSE
               X_STATE = NOPROG_STATE
            END IF
         ELSE
            IF ( DXRAT .GT. DXRATMAX ) DXRATMAX = DXRAT
         END IF
         IF ( X_STATE .GT. WORKING_STATE ) FINAL_DX_X = DX_X
      END IF

      IF ( Z_STATE .EQ. UNSTABLE_STATE .AND. DZ_Z .LE. DZ_UB )
     $      Z_STATE = WORKING_STATE
      IF ( Z_STATE .EQ. NOPROG_STATE .AND. DZRAT .LE. RTHRESH )
     $      Z_STATE = WORKING_STATE
      IF ( Z_STATE .EQ. WORKING_STATE ) THEN
         IF ( DZ_Z .LE. EPS ) THEN
            Z_STATE = CONV_STATE
         ELSE IF ( DZ_Z .GT. DZ_UB ) THEN
            Z_STATE = UNSTABLE_STATE
            DZRATMAX = 0.0
            FINAL_DZ_Z = HUGEVAL
         ELSE IF ( DZRAT .GT. RTHRESH ) THEN
            IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
               INCR_PREC = .TRUE.
            ELSE
               Z_STATE = NOPROG_STATE
            END IF
         ELSE
            IF ( DZRAT .GT. DZRATMAX ) DZRATMAX = DZRAT
         END IF
         IF ( Z_STATE .GT. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
      END IF
*
*     Exit if both normwise and componentwise stopped working,
*     but if componentwise is unstable, let it go at least two
*     iterations.
*
      IF ( X_STATE.NE.WORKING_STATE ) THEN
         IF ( IGNORE_CWISE) GOTO 666
         IF ( Z_STATE.EQ.NOPROG_STATE .OR. Z_STATE.EQ.CONV_STATE )
     $        GOTO 666
         IF ( Z_STATE.EQ.UNSTABLE_STATE .AND. CNT.GT.1 ) GOTO 666
      END IF

      IF ( INCR_PREC ) THEN
```

```
               INCR_PREC = .FALSE.
               Y_PREC_STATE = Y_PREC_STATE + 1
               DO I = 1, N
                  Y_TAIL( I ) = 0.0
               END DO
            END IF

            PREVNORMDX = NORMDX
            PREV_DZ_Z = DZ_Z
*
*           Update soluton.
*
            IF ( Y_PREC_STATE .LT. EXTRA_Y ) THEN
               CALL SAXPY( N, 1.0, DY, 1, Y( 1, J ), 1 )
            ELSE
               CALL SLA_WWADDW( N, Y( 1, J ), Y_TAIL, DY )
            END IF

         END DO
*        Target of "IF (Z_STOP .AND. X_STOP)".  Sun's f77 won't EXIT.
 666     CONTINUE
*
*     Set final_* when cnt hits ithresh.
*
         IF ( X_STATE .EQ. WORKING_STATE ) FINAL_DX_X = DX_X
         IF ( Z_STATE .EQ. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
*
*     Compute error bounds
*
         IF (N_NORMS .GE. 1) THEN
            ERRS_N( J, LA_LINRX_ERR_I ) =
     $           FINAL_DX_X / (1 - DXRATMAX)
         END IF
         IF ( N_NORMS .GE. 2 ) THEN
            ERRS_C( J, LA_LINRX_ERR_I ) =
     $           FINAL_DZ_Z / (1 - DZRATMAX)
         END IF
*
*     Compute componentwise relative backward error from formula
*         max(i) ( abs(R(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
*     where abs(Z) is the componentwise absolute value of the matrix
*     or vector Z.
*
*         Compute residual RES = B_s - op(A_s) * Y,
*             op(A) = A, A**T, or A**H depending on TRANS (and type).
*
         CALL SCOPY( N, B( 1, J ), 1, RES, 1 )
         CALL SGEMV( TRANS, N, N, -1.0, A, LDA, Y(1,J), 1, 1.0, RES, 1 )

         DO I = 1, N
```

```
            AYB( I ) = ABS( B( I, J ) )
         END DO
*
*     Compute abs(op(A_s))*abs(Y) + abs(B_s).
*
         CALL SLA_GEAMV ( TRANS_TYPE, N, N, 1.0,
     $        A, LDA, Y(1, J), 1, 1.0, AYB, 1 )

         CALL SLA_LIN_BERR ( N, N, 1, RES, AYB, BERR_OUT( J ) )
*
*     End of loop for each RHS.
*
      END DO
*
      RETURN
      END
```

———-

33.37.4 lisp code

— sla_gerfsx_extended.lisp —

```
(let* ((unstable_state 0)
       (working_state 1)
       (conv_state 2)
       (noprog_state 3)
       (base_residual 0)
       (extra_residual 1)
       (extra_y 2)
       (la_linrx_err_i 2))
  (declare (type (f2cl-lib:integer4 0 0) unstable_state)
           (type (f2cl-lib:integer4 1 1) working_state)
           (type (f2cl-lib:integer4 2 2) conv_state)
           (type (f2cl-lib:integer4 3 3) noprog_state)
           (type (f2cl-lib:integer4 0 0) base_residual)
           (type (f2cl-lib:integer4 1 1) extra_residual)
           (type (f2cl-lib:integer4 2 2) extra_y)
           (type (f2cl-lib:integer4 2 2) la_linrx_err_i)
           (ignorable unstable_state working_state conv_state noprog_state
            base_residual extra_residual extra_y la_linrx_err_i))
  (defun sla_gerfsx_extended
         (prec_type trans_type n nrhs a lda af ldaf ipiv colequ c b ldb$ y ldy
          berr_out n_norms errs_n errs_c res ayb dy y_tail rcond ithresh
          rthresh dz_ub ignore_cwise info)
    (declare (type (single-float) dz_ub rthresh rcond)
             (type f2cl-lib:logical ignore_cwise colequ)
             (type (array f2cl-lib:integer4 (*)) ipiv)
```

```
          (type (array single-float (*)) y_tail dy ayb res errs_c errs_n
           berr_out y b c af a)
          (type (f2cl-lib:integer4) info ithresh n_norms ldy ldb$ ldaf lda
           nrhs n trans_type prec_type))
  (f2cl-lib:with-multi-array-data
      ((a single-float a-%data% a-%offset%)
       (af single-float af-%data% af-%offset%)
       (c single-float c-%data% c-%offset%)
       (b single-float b-%data% b-%offset%)
       (y single-float y-%data% y-%offset%)
       (berr_out single-float berr_out-%data% berr_out-%offset%)
       (errs_n single-float errs_n-%data% errs_n-%offset%)
       (errs_c single-float errs_c-%data% errs_c-%offset%)
       (res single-float res-%data% res-%offset%)
       (ayb single-float ayb-%data% ayb-%offset%)
       (dy single-float dy-%data% dy-%offset%)
       (y_tail single-float y_tail-%data% y_tail-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((incr_prec nil) (yk 0.0) (dyk 0.0) (ymin 0.0) (normy 0.0)
           (normx 0.0) (normdx 0.0) (dxrat 0.0) (dzrat 0.0) (prevnormdx 0.0)
           (prev_dz_z 0.0) (dxratmax 0.0) (dzratmax 0.0) (dx_x 0.0)
           (dz_z 0.0) (final_dx_x 0.0) (final_dz_z 0.0) (eps 0.0)
           (hugeval 0.0) (incr_thresh 0.0) (cnt 0) (i 0) (j 0) (x_state 0)
           (z_state 0) (y_prec_state 0)
           (trans
            (make-array '(1) :element-type 'character :initial-element #\ )))
      (declare (type f2cl-lib:logical incr_prec)
               (type (single-float) yk dyk ymin normy normx normdx dxrat
                dzrat prevnormdx prev_dz_z dxratmax dzratmax dx_x dz_z
                final_dx_x final_dz_z eps hugeval incr_thresh)
               (type (f2cl-lib:integer4) cnt i j x_state z_state
                y_prec_state)
               (type (string 1) trans))
      (if (/= info 0)
          (go end_label))
      (f2cl-lib:f2cl-set-string trans
                                (f2cl-lib:fref-string #'chla_transtype
                                                      (trans_type))
                                (string 1))
      (setf eps (slamch "Epsilon"))
      (setf hugeval (slamch "Overflow"))
      (setf hugeval (* hugeval hugeval))
      (setf incr_thresh (* (f2cl-lib:freal n) eps))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nrhs) nil)
        (tagbody
          (setf y_prec_state extra_residual)
          (cond
           ((= y_prec_state extra_y)
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                           y_tail-%offset%)
              0.0)
     label100001))))
(setf dxrat 0.0)
(setf dxratmax 0.0)
(setf dzrat 0.0)
(setf dzratmax 0.0)
(setf final_dx_x hugeval)
(setf final_dz_z hugeval)
(setf prevnormdx hugeval)
(setf prev_dz_z hugeval)
(setf dz_z hugeval)
(setf dx_x hugeval)
(setf x_state working_state)
(setf z_state unstable_state)
(setf incr_prec f2cl-lib:%false%)
(f2cl-lib:fdo (cnt 1 (f2cl-lib:int-add cnt 1))
              ((> cnt ithresh) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy n
         (f2cl-lib:array-slice b-%data% single-float (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1 res 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (cond
     ((= y_prec_state base_residual)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (sgemv trans n n -1.0 a lda
           (f2cl-lib:array-slice y-%data% single-float (1 j)
                                 ((1 ldy) (1 *)) y-%offset%)
           1 1.0 res 1)
        (declare
         (ignore var-3 var-4 var-6 var-7 var-8 var-9 var-10))
        (when var-0 (setf trans var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-5 (setf lda var-5))))
     ((= y_prec_state extra_residual)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11)
          (blas_sgemv_x trans_type n n -1.0 a lda
           (f2cl-lib:array-slice y-%data% single-float (1 j)
```

```
                                ((1 ldy) (1 *)) y-%offset%)
        1 1.0 res 1 prec_type)
    (declare
     (ignore var-3 var-4 var-6 var-7 var-8 var-9 var-10))
    (when var-0 (setf trans_type var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-5 (setf lda var-5))
    (when var-11 (setf prec_type var-11))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (blas_sgemv2_x trans_type n n -1.0 a lda
       (f2cl-lib:array-slice y-%data% single-float (1 j)
                                ((1 ldy) (1 *)) y-%offset%)
       y_tail 1 1.0 res 1 prec_type)
    (declare
     (ignore var-3 var-4 var-6 var-7 var-8 var-9 var-10
      var-11))
    (when var-0 (setf trans_type var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-5 (setf lda var-5))
    (when var-12 (setf prec_type var-12)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (scopy n res 1 dy 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sgetrs trans n 1 af ldaf ipiv dy n info)
  (declare (ignore var-2 var-3 var-5 var-6))
  (setf trans var-0)
  (setf n var-1)
  (setf ldaf var-4)
  (setf n var-7)
  (setf info var-8))
(setf normx 0.0)
(setf normy 0.0)
(setf normdx 0.0)
(setf dz_z 0.0)
(setf ymin hugeval)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf yk
            (abs
             (f2cl-lib:fref y-%data% (i j) ((1 ldy) (1 *))
                            y-%offset%)))
```

```
    (setf dyk
            (abs
             (f2cl-lib:fref dy-%data% (i) ((1 *))
                            dy-%offset%)))
    (cond ((/= yk 0.0) (setf dz_z (max dz_z (/ dyk yk))))
          ((/= dyk 0.0) (setf dz_z hugeval)))
    (setf ymin (min ymin yk))
    (setf normy (max normy yk))
    (cond
     (colequ
      (setf normx
              (max normx
                   (* yk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%))))
      (setf normdx
              (max normdx
                   (* dyk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%)))))
     (t (setf normx normy) (setf normdx (max normdx dyk))))
   label100003))
(cond ((/= normx 0.0) (setf dx_x (/ normdx normx)))
      ((= normdx 0.0) (setf dx_x 0.0)) (t (setf dx_x hugeval)))
(setf dxrat (/ normdx prevnormdx))
(setf dzrat (/ dz_z prev_dz_z))
(if (and (not ignore_cwise)
         (< (* ymin rcond) (* incr_thresh normy))
         (< y_prec_state extra_y))
    (setf incr_prec f2cl-lib:%true%))
(if (and (= x_state noprog_state) (<= dxrat rthresh))
    (setf x_state working_state))
(cond
 ((= x_state working_state)
  (cond ((<= dx_x eps) (setf x_state conv_state))
        ((> dxrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf x_state noprog_state))))
        (t
         (if (> dxrat dxratmax)
             (setf dxratmax dxrat))))
  (if (> x_state working_state)
      (setf final_dx_x dx_x))))
(if (and (= z_state unstable_state) (<= dz_z dz_ub))
    (setf z_state working_state))
(if (and (= z_state noprog_state) (<= dzrat rthresh))
    (setf z_state working_state))
(cond
```

```
 ((= z_state working_state)
  (cond ((<= dz_z eps) (setf z_state conv_state))
        ((> dz_z dz_ub) (setf z_state unstable_state)
         (setf dzratmax 0.0) (setf final_dz_z hugeval))
        ((> dzrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf z_state noprog_state))))
        (t
         (if (> dzrat dzratmax)
             (setf dzratmax dzrat))))
  (if (> z_state working_state)
      (setf final_dz_z dz_z))))
(cond
 ((/= x_state working_state)
  (if ignore_cwise
      (go label666))
  (if (or (= z_state noprog_state) (= z_state conv_state))
      (go label666))
  (if (and (= z_state unstable_state) (> cnt 1))
      (go label666))))
(cond
 (incr_prec (setf incr_prec f2cl-lib:%false%)
  (setf y_prec_state (f2cl-lib:int-add y_prec_state 1))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                           y_tail-%offset%)
              0.0)
     label100004))))
(setf prevnormdx normdx)
(setf prev_dz_z dz_z)
(cond
 ((< y_prec_state extra_y)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (saxpy n 1.0 dy 1
       (f2cl-lib:array-slice y-%data% single-float (1 j)
                             ((1 ldy) (1 *)) y-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (sla_wwaddw n
       (f2cl-lib:array-slice y-%data% single-float (1 j)
                             ((1 ldy) (1 *)) y-%offset%)
       y_tail dy)
    (declare (ignore var-1 var-2 var-3))
```

```
          (when var-0 (setf n var-0)))))
     label100002))
  label666
   (if (= x_state working_state)
       (setf final_dx_x dx_x))
   (if (= z_state working_state)
       (setf final_dz_z dz_z))
   (cond
    ((>= n_norms 1)
     (setf (f2cl-lib:fref errs_n-%data% (j la_linrx_err_i)
                          ((1 nrhs) (1 *)) errs_n-%offset%)
           (/ final_dx_x (- 1 dxratmax)))))
   (cond
    ((>= n_norms 2)
     (setf (f2cl-lib:fref errs_c-%data% (j la_linrx_err_i)
                          ((1 nrhs) (1 *)) errs_c-%offset%)
           (/ final_dz_z (- 1 dzratmax)))))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
       (scopy n
        (f2cl-lib:array-slice b-%data% single-float (1 j)
                              ((1 ldb$) (1 *)) b-%offset%)
        1 res 1)
     (declare (ignore var-1 var-2 var-3 var-4))
     (when var-0 (setf n var-0)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (sgemv trans n n -1.0 a lda
        (f2cl-lib:array-slice y-%data% single-float (1 j)
                              ((1 ldy) (1 *)) y-%offset%)
        1 1.0 res 1)
     (declare (ignore var-3 var-4 var-6 var-7 var-8 var-9 var-10))
     (when var-0 (setf trans var-0))
     (when var-1 (setf n var-1))
     (when var-2 (setf n var-2))
     (when var-5 (setf lda var-5)))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf (f2cl-lib:fref ayb-%data% (i) ((1 *)) ayb-%offset%)
               (abs
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)))
     label100005))
   (sla_geamv trans_type n n 1.0 a lda
    (f2cl-lib:array-slice y-%data% single-float (1 j) ((1 ldy) (1 *))
                          y-%offset%)
    1 1.0 ayb 1)
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
       (sla_lin_berr n n 1 res ayb
```

```
                (f2cl-lib:array-slice berr_out-%data% single-float (j) ((1 *))
                                      berr_out-%offset%))
            (declare (ignore var-2 var-3 var-4 var-5))
            (when var-0 (setf n var-0))
            (when var-1 (setf n var-1)))
          label100000))
       (go end_label)
      end_label
       (return
        (values prec_type trans_type n nil nil lda nil ldaf nil nil nil nil
                nil nil nil nil nil nil nil nil nil nil nil nil nil nil nil
                nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sla_gerfsx_extended
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::integer4
                                              (*))
                                             fortran-to-lisp::logical
                                             (array single-float (*))
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (array single-float (*))
                                             (array single-float (*))
                                             (array single-float (*))
                                             (array single-float (*))
                                             (array single-float (*))
                                             (single-float)
                                             (fortran-to-lisp::integer4)
                                             (single-float) (single-float)
                                             fortran-to-lisp::logical
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(fortran-to-lisp::prec_type
```

```
                                        fortran-to-lisp::trans_type
                                        fortran-to-lisp::n nil nil
                                        fortran-to-lisp::lda nil
                                        fortran-to-lisp::ldaf nil nil nil
                                        nil nil nil nil nil nil nil nil
                                        nil nil nil nil nil nil nil nil
                                        nil fortran-to-lisp::info)
                                      :calls
                                      '(fortran-to-lisp::sla_geamv
                                        fortran-to-lisp::sgetrs))))
```

———-

33.38 sla_gerpvgrw

33.38.1 Axiom unit tests

— slagerpvgrw.input —

```
)set break resume
)sys rm -f slagerpvgrw.output
)spool slagerpvgrw.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.38.2 Axiom help page

— sla_gerpvgrw.help —

```
==================================================================
sla_gerpvgrw examples
==================================================================

==================================================================
Man Page Details
==================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       real function sla_gerpvgrw (N, NCOLS, A, LDA, AF, LDAF)
           SLA_GERPVGRW

Function/Subroutine Documentation
   real function sla_gerpvgrw (integer N, integer NCOLS, real, dimension( lda,
       * ) A, integer LDA, real, dimension( ldaf, * ) AF, integer LDAF)
       SLA_GERPVGRW

Purpose:
   SLA_GERPVGRW computes the reciprocal pivot growth factor
   norm(A)/norm(U). The "max absolute element" norm is used. If this is
   much less than 1, the stability of the LU factorization of the
   (equilibrated) matrix A could be poor. This also means that the
   solution X, estimated condition numbers, and error bounds could be
   unreliable.

Parameters:

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 NCOLS
   NCOLS is INTEGER
   The number of columns of the matrix A. NCOLS >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is REAL array, dimension (LDAF,N)
   The factors L and U from the factorization
   A = P*L*U as computed by SGETRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).
```

33.38.3 fortran code

— sla_gerpvgrw.f —

```
*  =====================================================================
      REAL FUNCTION SLA_GERPVGRW( N, NCOLS, A, LDA, AF, LDAF )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            N, NCOLS, LDA, LDAF
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), AF( LDAF, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               AMAX, UMAX, RPVGRW
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      RPVGRW = 1.0

      DO J = 1, NCOLS
         AMAX = 0.0
         UMAX = 0.0
         DO I = 1, N
            AMAX = MAX( ABS( A( I, J ) ), AMAX )
         END DO
         DO I = 1, J
            UMAX = MAX( ABS( AF( I, J ) ), UMAX )
         END DO
         IF ( UMAX /= 0.0 ) THEN
            RPVGRW = MIN( AMAX / UMAX, RPVGRW )
         END IF
      END DO
      SLA_GERPVGRW = RPVGRW
      END
```

———-

33.38.4 lisp code

— sla_gerpvgrw.lisp —

```
(defun sla_gerpvgrw (n ncols a lda af ldaf)
  (declare (type (array single-float (*)) af a)
           (type (f2cl-lib:integer4) ldaf lda ncols n))
  (f2cl-lib:with-multi-array-data
      ((a single-float a-%data% a-%offset%)
       (af single-float af-%data% af-%offset%))
    (prog ((amax 0.0) (umax 0.0) (rpvgrw 0.0) (i 0) (j 0) (sla_gerpvgrw 0.0))
      (declare (type (f2cl-lib:integer4) j i)
               (type (single-float) sla_gerpvgrw rpvgrw umax amax))
      (setf rpvgrw 1.0)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j ncols) nil)
        (tagbody
          (setf amax 0.0)
          (setf umax 0.0)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf amax
                      (max
                       (abs
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))
                       amax))
             label100001))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf umax
                      (max
                       (abs
                        (f2cl-lib:fref af-%data% (i j) ((1 ldaf) (1 *))
                                       af-%offset%))
                       umax))
             label100002))
          (cond
           ((f2cl-lib:f2cl/ umax (= 0.0))
            (setf rpvgrw (min (/ amax umax) rpvgrw))))
         label100000))
      (setf sla_gerpvgrw rpvgrw)
     end_label
      (return (values sla_gerpvgrw nil nil nil nil nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
```

```
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sla_gerpvgrw
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil) :calls
                                            'nil)))
```

———-

33.39 stgevc

33.39.1 Axiom unit tests

— **stgevc.input** —

```
)set break resume
)sys rm -f stgevc.output
)spool stgevc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.39.2 Axiom help page

— **stgevc.help** —

```
======================================================================
stgevc examples
======================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine stgevc (SIDE, HOWMNY, SELECT, N, S, LDS, P, LDP, VL, LDVL,
           VR, LDVR, MM, M, WORK, INFO)
           STGEVC

Function/Subroutine Documentation
   subroutine stgevc (character SIDE, character HOWMNY, logical, dimension( *
       ) SELECT, integer N, real, dimension( lds, * ) S, integer LDS, real,
       dimension( ldp, * ) P, integer LDP, real, dimension( ldvl, * ) VL,
       integer LDVL, real, dimension( ldvr, * ) VR, integer LDVR, integer MM,
       integer M, real, dimension( * ) WORK, integer INFO)
       STGEVC

Purpose:
   STGEVC computes some or all of the right and/or left eigenvectors of
   a pair of real matrices (S,P), where S is a quasi-triangular matrix
   and P is upper triangular.  Matrix pairs of this type are produced by
   the generalized Schur factorization of a matrix pair (A,B):

      A = Q*S*Z**T,  B = Q*P*Z**T

   as computed by SGGHRD + SHGEQZ.

   The right eigenvector x and the left eigenvector y of (S,P)
   corresponding to an eigenvalue w are defined by:

      S*x = w*P*x,  (y**H)*S = w*(y**H)*P,

   where y**H denotes the conjugate tranpose of y.
   The eigenvalues are not input to this routine, but are computed
   directly from the diagonal blocks of S and P.

   This routine returns the matrices X and/or Y of right and left
   eigenvectors of (S,P), or the products Z*X and/or Q*Y,
   where Z and Q are input matrices.
   If Q and Z are the orthogonal factors from the generalized Schur
   factorization of a matrix pair (A,B), then Z*X and Q*Y
   are the matrices of right and left eigenvectors of (A,B).

Parameters:

 SIDE
   SIDE is CHARACTER*1
   = 'R': compute right eigenvectors only;
   = 'L': compute left eigenvectors only;
```

```
  = 'B': compute both right and left eigenvectors.

HOWMNY
  HOWMNY is CHARACTER*1
  = 'A': compute all right and/or left eigenvectors;
  = 'B': compute all right and/or left eigenvectors,
         backtransformed by the matrices in VR and/or VL;
  = 'S': compute selected right and/or left eigenvectors,
         specified by the logical array SELECT.

SELECT
  SELECT is LOGICAL array, dimension (N)
  If HOWMNY='S', SELECT specifies the eigenvectors to be
  computed.  If w(j) is a real eigenvalue, the corresponding
  real eigenvector is computed if SELECT(j) is .TRUE..
  If w(j) and w(j+1) are the real and imaginary parts of a
  complex eigenvalue, the corresponding complex eigenvector
  is computed if either SELECT(j) or SELECT(j+1) is .TRUE.,
  and on exit SELECT(j) is set to .TRUE. and SELECT(j+1) is
  set to .FALSE..
  Not referenced if HOWMNY = 'A' or 'B'.

N
  N is INTEGER
  The order of the matrices S and P.  N >= 0.

S
  S is REAL array, dimension (LDS,N)
  The upper quasi-triangular matrix S from a generalized Schur
  factorization, as computed by SHGEQZ.

LDS
  LDS is INTEGER
  The leading dimension of array S.  LDS >= max(1,N).

P
  P is REAL array, dimension (LDP,N)
  The upper triangular matrix P from a generalized Schur
  factorization, as computed by SHGEQZ.
  2-by-2 diagonal blocks of P corresponding to 2-by-2 blocks
  of S must be in positive diagonal form.

LDP
  LDP is INTEGER
  The leading dimension of array P.  LDP >= max(1,N).

VL
  VL is REAL array, dimension (LDVL,MM)
  On entry, if SIDE = 'L' or 'B' and HOWMNY = 'B', VL must
  contain an N-by-N matrix Q (usually the orthogonal matrix Q
```

```
  of left Schur vectors returned by SHGEQZ).
  On exit, if SIDE = 'L' or 'B', VL contains:
  if HOWMNY = 'A', the matrix Y of left eigenvectors of (S,P);
  if HOWMNY = 'B', the matrix Q*Y;
  if HOWMNY = 'S', the left eigenvectors of (S,P) specified by
       SELECT, stored consecutively in the columns of
       VL, in the same order as their eigenvalues.

  A complex eigenvector corresponding to a complex eigenvalue
  is stored in two consecutive columns, the first holding the
  real part, and the second the imaginary part.

  Not referenced if SIDE = 'R'.

LDVL
  LDVL is INTEGER
  The leading dimension of array VL.  LDVL >= 1, and if
  SIDE = 'L' or 'B', LDVL >= N.

VR
  VR is REAL array, dimension (LDVR,MM)
  On entry, if SIDE = 'R' or 'B' and HOWMNY = 'B', VR must
  contain an N-by-N matrix Z (usually the orthogonal matrix Z
  of right Schur vectors returned by SHGEQZ).

  On exit, if SIDE = 'R' or 'B', VR contains:
  if HOWMNY = 'A', the matrix X of right eigenvectors of (S,P);
  if HOWMNY = 'B' or 'b', the matrix Z*X;
  if HOWMNY = 'S' or 's', the right eigenvectors of (S,P)
              specified by SELECT, stored consecutively in the
              columns of VR, in the same order as their eigenvalues.

  A complex eigenvector corresponding to a complex eigenvalue
  is stored in two consecutive columns, the first holding the
  real part and the second the imaginary part.

  Not referenced if SIDE = 'L'.

LDVR
  LDVR is INTEGER
  The leading dimension of the array VR.  LDVR >= 1, and if
  SIDE = 'R' or 'B', LDVR >= N.

MM
  MM is INTEGER
  The number of columns in the arrays VL and/or VR. MM >= M.

M
  M is INTEGER
  The number of columns in the arrays VL and/or VR actually
```

```
   used to store the eigenvectors.  If HOWMNY = 'A' or 'B', M
   is set to N.  Each selected real eigenvector occupies one
   column and each selected complex eigenvector occupies two columns.

 WORK
   WORK is REAL array, dimension (6*N)

 INFO
   INFO is INTEGER
   = 0:  successful exit.
   < 0:  if INFO = -i, the i-th argument had an illegal value.
   > 0:  the 2-by-2 block (INFO:INFO+1) does not have a complex eigenvalue.

Further Details:

   Allocation of workspace:
   ---------- -- ---------

   WORK( j ) = 1-norm of j-th column of A, above the diagonal
   WORK( N+j ) = 1-norm of j-th column of B, above the diagonal
   WORK( 2*N+1:3*N ) = real part of eigenvector
   WORK( 3*N+1:4*N ) = imaginary part of eigenvector
   WORK( 4*N+1:5*N ) = real part of back-transformed eigenvector
   WORK( 5*N+1:6*N ) = imaginary part of back-transformed eigenvector

   Rowwise vs. columnwise solution methods:
   ------- --  ---------- -------- -------

   Finding a generalized eigenvector consists basically of solving the
   singular triangular system

      (A - w B) x = 0     (for right) or:   (A - w B)**H y = 0  (for left)

   Consider finding the i-th right eigenvector (assume all eigenvalues
   are real). The equation to be solved is:

                n                   i
        0 = sum  C(j,k) v(k)  = sum  C(j,k) v(k)     for j = i,. . .,1
               k=j                 k=j

   where  C = (A - w B)  (The components v(i+1:n) are 0.)

   The "rowwise" method is:

   (1)  v(i) := 1
   for j = i-1,. . .,1:
                              i
        (2) compute  s = - sum C(j,k) v(k)   and
                            k=j+1
```

```
      (3) v(j) := s / C(j,j)

   Step 2 is sometimes called the "dot product" step, since it is an
   inner product between the j-th row and the portion of the eigenvector
   that has been computed so far.

   The "columnwise" method consists basically in doing the sums
   for all the rows in parallel.  As each v(j) is computed, the
   contribution of v(j) times the j-th column of C is added to the
   partial sums.  Since FORTRAN arrays are stored columnwise, this has
   the advantage that at each step, the elements of C that are accessed
   are adjacent to one another, whereas with the rowwise method, the
   elements accessed at a step are spaced LDS (and LDP) words apart.

   When finding left eigenvectors, the matrix in question is the
   transpose of the one in storage, so the rowwise method then
   actually accesses columns of A and B at each step, and so is the
   preferred method.
```

33.39.3 fortran code

— stgevc.f —

```
*  =====================================================================
      SUBROUTINE STGEVC( SIDE, HOWMNY, SELECT, N, S, LDS, P, LDP, VL,
     $                   LDVL, VR, LDVR, MM, M, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          HOWMNY, SIDE
      INTEGER            INFO, LDP, LDS, LDVL, LDVR, M, MM, N
*     ..
*     .. Array Arguments ..
      LOGICAL            SELECT( * )
      REAL               P( LDP, * ), S( LDS, * ), VL( LDVL, * ),
     $                   VR( LDVR, * ), WORK( * )
*     ..
*
*
*  =====================================================================
*
```

```
*     .. Parameters ..
      REAL               ZERO, ONE, SAFETY
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0,
     $                   SAFETY = 1.0E+2 )
*     ..
*     .. Local Scalars ..
      LOGICAL            COMPL, COMPR, IL2BY2, ILABAD, ILALL, ILBACK,
     $                   ILBBAD, ILCOMP, ILCPLX, LSA, LSB
      INTEGER            I, IBEG, IEIG, IEND, IHWMNY, IINFO, IM, ISIDE,
     $                   J, JA, JC, JE, JR, JW, NA, NW
      REAL               ACOEF, ACOEFA, ANORM, ASCALE, BCOEFA, BCOEFI,
     $                   BCOEFR, BIG, BIGNUM, BNORM, BSCALE, CIM2A,
     $                   CIM2B, CIMAGA, CIMAGB, CRE2A, CRE2B, CREALA,
     $                   CREALB, DMIN, SAFMIN, SALFAR, SBETA, SCALE,
     $                   SMALL, TEMP, TEMP2, TEMP2I, TEMP2R, ULP, XMAX,
     $                   XSCALE
*     ..
*     .. Local Arrays ..
      REAL               BDIAG( 2 ), SUM( 2, 2 ), SUMS( 2, 2 ),
     $                   SUMP( 2, 2 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH
      EXTERNAL           LSAME, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEMV, SLABAD, SLACPY, SLAG2, SLALN2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Decode and Test the input parameters
*
      IF( LSAME( HOWMNY, 'A' ) ) THEN
         IHWMNY = 1
         ILALL = .TRUE.
         ILBACK = .FALSE.
      ELSE IF( LSAME( HOWMNY, 'S' ) ) THEN
         IHWMNY = 2
         ILALL = .FALSE.
         ILBACK = .FALSE.
      ELSE IF( LSAME( HOWMNY, 'B' ) ) THEN
         IHWMNY = 3
         ILALL = .TRUE.
         ILBACK = .TRUE.
      ELSE
         IHWMNY = -1
```

```
         ILALL = .TRUE.
      END IF
*
      IF( LSAME( SIDE, 'R' ) ) THEN
         ISIDE = 1
         COMPL = .FALSE.
         COMPR = .TRUE.
      ELSE IF( LSAME( SIDE, 'L' ) ) THEN
         ISIDE = 2
         COMPL = .TRUE.
         COMPR = .FALSE.
      ELSE IF( LSAME( SIDE, 'B' ) ) THEN
         ISIDE = 3
         COMPL = .TRUE.
         COMPR = .TRUE.
      ELSE
         ISIDE = -1
      END IF
*
      INFO = 0
      IF( ISIDE.LT.0 ) THEN
         INFO = -1
      ELSE IF( IHWMNY.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDS.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDP.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'STGEVC', -INFO )
         RETURN
      END IF
*
*     Count the number of eigenvectors to be computed
*
      IF( .NOT.ILALL ) THEN
         IM = 0
         ILCPLX = .FALSE.
         DO 10 J = 1, N
            IF( ILCPLX ) THEN
               ILCPLX = .FALSE.
               GO TO 10
            END IF
            IF( J.LT.N ) THEN
               IF( S( J+1, J ).NE.ZERO )
     $            ILCPLX = .TRUE.
            END IF
```

```
            IF( ILCPLX ) THEN
               IF( SELECT( J ) .OR. SELECT( J+1 ) )
     $            IM = IM + 2
            ELSE
               IF( SELECT( J ) )
     $            IM = IM + 1
            END IF
   10    CONTINUE
      ELSE
         IM = N
      END IF
*
*     Check 2-by-2 diagonal blocks of A, B
*
      ILABAD = .FALSE.
      ILBBAD = .FALSE.
      DO 20 J = 1, N - 1
         IF( S( J+1, J ).NE.ZERO ) THEN
            IF( P( J, J ).EQ.ZERO .OR. P( J+1, J+1 ).EQ.ZERO .OR.
     $          P( J, J+1 ).NE.ZERO )ILBBAD = .TRUE.
            IF( J.LT.N-1 ) THEN
               IF( S( J+2, J+1 ).NE.ZERO )
     $            ILABAD = .TRUE.
            END IF
         END IF
   20 CONTINUE
*
      IF( ILABAD ) THEN
         INFO = -5
      ELSE IF( ILBBAD ) THEN
         INFO = -7
      ELSE IF( COMPL .AND. LDVL.LT.N .OR. LDVL.LT.1 ) THEN
         INFO = -10
      ELSE IF( COMPR .AND. LDVR.LT.N .OR. LDVR.LT.1 ) THEN
         INFO = -12
      ELSE IF( MM.LT.IM ) THEN
         INFO = -13
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'STGEVC', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      M = IM
      IF( N.EQ.0 )
     $   RETURN
*
*     Machine Constants
```

```
*
      SAFMIN = SLAMCH( 'Safe minimum' )
      BIG = ONE / SAFMIN
      CALL SLABAD( SAFMIN, BIG )
      ULP = SLAMCH( 'Epsilon' )*SLAMCH( 'Base' )
      SMALL = SAFMIN*N / ULP
      BIG = ONE / SMALL
      BIGNUM = ONE / ( SAFMIN*N )
*
*     Compute the 1-norm of each column of the strictly upper triangular
*     part (i.e., excluding all elements belonging to the diagonal
*     blocks) of A and B to check for possible overflow in the
*     triangular solver.
*
      ANORM = ABS( S( 1, 1 ) )
      IF( N.GT.1 )
     $   ANORM = ANORM + ABS( S( 2, 1 ) )
      BNORM = ABS( P( 1, 1 ) )
      WORK( 1 ) = ZERO
      WORK( N+1 ) = ZERO
*
      DO 50 J = 2, N
         TEMP = ZERO
         TEMP2 = ZERO
         IF( S( J, J-1 ).EQ.ZERO ) THEN
            IEND = J - 1
         ELSE
            IEND = J - 2
         END IF
         DO 30 I = 1, IEND
            TEMP = TEMP + ABS( S( I, J ) )
            TEMP2 = TEMP2 + ABS( P( I, J ) )
   30    CONTINUE
         WORK( J ) = TEMP
         WORK( N+J ) = TEMP2
         DO 40 I = IEND + 1, MIN( J+1, N )
            TEMP = TEMP + ABS( S( I, J ) )
            TEMP2 = TEMP2 + ABS( P( I, J ) )
   40    CONTINUE
         ANORM = MAX( ANORM, TEMP )
         BNORM = MAX( BNORM, TEMP2 )
   50 CONTINUE
*
      ASCALE = ONE / MAX( ANORM, SAFMIN )
      BSCALE = ONE / MAX( BNORM, SAFMIN )
*
*     Left eigenvectors
*
      IF( COMPL ) THEN
         IEIG = 0
```

```
*
*        Main loop over eigenvalues
*
         ILCPLX = .FALSE.
         DO 220 JE = 1, N
*
*           Skip this iteration if (a) HOWMNY='S' and SELECT=.FALSE., or
*           (b) this would be the second of a complex pair.
*           Check for complex eigenvalue, so as to be sure of which
*           entry(-ies) of SELECT to look at.
*
            IF( ILCPLX ) THEN
               ILCPLX = .FALSE.
               GO TO 220
            END IF
            NW = 1
            IF( JE.LT.N ) THEN
               IF( S( JE+1, JE ).NE.ZERO ) THEN
                  ILCPLX = .TRUE.
                  NW = 2
               END IF
            END IF
            IF( ILALL ) THEN
               ILCOMP = .TRUE.
            ELSE IF( ILCPLX ) THEN
               ILCOMP = SELECT( JE ) .OR. SELECT( JE+1 )
            ELSE
               ILCOMP = SELECT( JE )
            END IF
            IF( .NOT.ILCOMP )
     $         GO TO 220
*
*           Decide if (a) singular pencil, (b) real eigenvalue, or
*           (c) complex eigenvalue.
*
            IF( .NOT.ILCPLX ) THEN
               IF( ABS( S( JE, JE ) ).LE.SAFMIN .AND.
     $             ABS( P( JE, JE ) ).LE.SAFMIN ) THEN
*
*                 Singular matrix pencil -- return unit eigenvector
*
                  IEIG = IEIG + 1
                  DO 60 JR = 1, N
                     VL( JR, IEIG ) = ZERO
   60             CONTINUE
                  VL( IEIG, IEIG ) = ONE
                  GO TO 220
               END IF
            END IF
*
```

```
*          Clear vector
*
           DO 70 JR = 1, NW*N
              WORK( 2*N+JR ) = ZERO
   70      CONTINUE
*                                                T
*          Compute coefficients in  ( a A - b B )  y = 0
*             a  is  ACOEF
*             b  is  BCOEFR + i*BCOEFI
*
           IF( .NOT.ILCPLX ) THEN
*
*             Real eigenvalue
*
              TEMP = ONE / MAX( ABS( S( JE, JE ) )*ASCALE,
     $                ABS( P( JE, JE ) )*BSCALE, SAFMIN )
              SALFAR = ( TEMP*S( JE, JE ) )*ASCALE
              SBETA = ( TEMP*P( JE, JE ) )*BSCALE
              ACOEF = SBETA*ASCALE
              BCOEFR = SALFAR*BSCALE
              BCOEFI = ZERO
*
*             Scale to avoid underflow
*
              SCALE = ONE
              LSA = ABS( SBETA ).GE.SAFMIN .AND. ABS( ACOEF ).LT.SMALL
              LSB = ABS( SALFAR ).GE.SAFMIN .AND. ABS( BCOEFR ).LT.
     $              SMALL
              IF( LSA )
     $           SCALE = ( SMALL / ABS( SBETA ) )*MIN( ANORM, BIG )
              IF( LSB )
     $           SCALE = MAX( SCALE, ( SMALL / ABS( SALFAR ) )*
     $                   MIN( BNORM, BIG ) )
              IF( LSA .OR. LSB ) THEN
                 SCALE = MIN( SCALE, ONE /
     $                   ( SAFMIN*MAX( ONE, ABS( ACOEF ),
     $                   ABS( BCOEFR ) ) ) )
                 IF( LSA ) THEN
                    ACOEF = ASCALE*( SCALE*SBETA )
                 ELSE
                    ACOEF = SCALE*ACOEF
                 END IF
                 IF( LSB ) THEN
                    BCOEFR = BSCALE*( SCALE*SALFAR )
                 ELSE
                    BCOEFR = SCALE*BCOEFR
                 END IF
              END IF
              ACOEFA = ABS( ACOEF )
              BCOEFA = ABS( BCOEFR )
```

```
*
*              First component is 1
*
               WORK( 2*N+JE ) = ONE
               XMAX = ONE
            ELSE
*
*              Complex eigenvalue
*
               CALL SLAG2( S( JE, JE ), LDS, P( JE, JE ), LDP,
     $                     SAFMIN*SAFETY, ACOEF, TEMP, BCOEFR, TEMP2,
     $                     BCOEFI )
               BCOEFI = -BCOEFI
               IF( BCOEFI.EQ.ZERO ) THEN
                  INFO = JE
                  RETURN
               END IF
*
*              Scale to avoid over/underflow
*
               ACOEFA = ABS( ACOEF )
               BCOEFA = ABS( BCOEFR ) + ABS( BCOEFI )
               SCALE = ONE
               IF( ACOEFA*ULP.LT.SAFMIN .AND. ACOEFA.GE.SAFMIN )
     $            SCALE = ( SAFMIN / ULP ) / ACOEFA
               IF( BCOEFA*ULP.LT.SAFMIN .AND. BCOEFA.GE.SAFMIN )
     $            SCALE = MAX( SCALE, ( SAFMIN / ULP ) / BCOEFA )
               IF( SAFMIN*ACOEFA.GT.ASCALE )
     $            SCALE = ASCALE / ( SAFMIN*ACOEFA )
               IF( SAFMIN*BCOEFA.GT.BSCALE )
     $            SCALE = MIN( SCALE, BSCALE / ( SAFMIN*BCOEFA ) )
               IF( SCALE.NE.ONE ) THEN
                  ACOEF = SCALE*ACOEF
                  ACOEFA = ABS( ACOEF )
                  BCOEFR = SCALE*BCOEFR
                  BCOEFI = SCALE*BCOEFI
                  BCOEFA = ABS( BCOEFR ) + ABS( BCOEFI )
               END IF
*
*              Compute first two components of eigenvector
*
               TEMP = ACOEF*S( JE+1, JE )
               TEMP2R = ACOEF*S( JE, JE ) - BCOEFR*P( JE, JE )
               TEMP2I = -BCOEFI*P( JE, JE )
               IF( ABS( TEMP ).GT.ABS( TEMP2R )+ABS( TEMP2I ) ) THEN
                  WORK( 2*N+JE ) = ONE
                  WORK( 3*N+JE ) = ZERO
                  WORK( 2*N+JE+1 ) = -TEMP2R / TEMP
                  WORK( 3*N+JE+1 ) = -TEMP2I / TEMP
               ELSE
```

```
                 WORK( 2*N+JE+1 ) = ONE
                 WORK( 3*N+JE+1 ) = ZERO
                 TEMP = ACOEF*S( JE, JE+1 )
                 WORK( 2*N+JE ) = ( BCOEFR*P( JE+1, JE+1 )-ACOEF*
     $                            S( JE+1, JE+1 ) ) / TEMP
                 WORK( 3*N+JE ) = BCOEFI*P( JE+1, JE+1 ) / TEMP
              END IF
              XMAX = MAX( ABS( WORK( 2*N+JE ) )+ABS( WORK( 3*N+JE ) ),
     $                ABS( WORK( 2*N+JE+1 ) )+ABS( WORK( 3*N+JE+1 ) ) )
           END IF
*
           DMIN = MAX( ULP*ACOEFA*ANORM, ULP*BCOEFA*BNORM, SAFMIN )
*
*                                          T
*          Triangular solve of  (a A - b B)  y = 0
*
*                                  T
*          (rowwise in  (a A - b B) , or columnwise in (a A - b B) )
*
           IL2BY2 = .FALSE.
*
           DO 160 J = JE + NW, N
              IF( IL2BY2 ) THEN
                 IL2BY2 = .FALSE.
                 GO TO 160
              END IF
*
              NA = 1
              BDIAG( 1 ) = P( J, J )
              IF( J.LT.N ) THEN
                 IF( S( J+1, J ).NE.ZERO ) THEN
                    IL2BY2 = .TRUE.
                    BDIAG( 2 ) = P( J+1, J+1 )
                    NA = 2
                 END IF
              END IF
*
*             Check whether scaling is necessary for dot products
*
              XSCALE = ONE / MAX( ONE, XMAX )
              TEMP = MAX( WORK( J ), WORK( N+J ),
     $                ACOEFA*WORK( J )+BCOEFA*WORK( N+J ) )
              IF( IL2BY2 )
     $           TEMP = MAX( TEMP, WORK( J+1 ), WORK( N+J+1 ),
     $                  ACOEFA*WORK( J+1 )+BCOEFA*WORK( N+J+1 ) )
              IF( TEMP.GT.BIGNUM*XSCALE ) THEN
                 DO 90 JW = 0, NW - 1
                    DO 80 JR = JE, J - 1
                       WORK( ( JW+2 )*N+JR ) = XSCALE*
     $                    WORK( ( JW+2 )*N+JR )
```

```
   80                CONTINUE
   90             CONTINUE
                  XMAX = XMAX*XSCALE
               END IF
*
*              Compute dot products
*
*                    j-1
*              SUM = sum  conjg( a*S(k,j) - b*P(k,j) )*x(k)
*                    k=je
*
*              To reduce the op count, this is done as
*
*              _        j-1                   _        j-1
*              a*conjg( sum  S(k,j)*x(k) ) - b*conjg( sum  P(k,j)*x(k) )
*                       k=je                           k=je
*
*              which may cause underflow problems if A or B are close
*              to underflow.  (E.g., less than SMALL.)
*
*
               DO 120 JW = 1, NW
                  DO 110 JA = 1, NA
                     SUMS( JA, JW ) = ZERO
                     SUMP( JA, JW ) = ZERO
*
                     DO 100 JR = JE, J - 1
                        SUMS( JA, JW ) = SUMS( JA, JW ) +
     $                                   S( JR, J+JA-1 )*
     $                                   WORK( ( JW+1 )*N+JR )
                        SUMP( JA, JW ) = SUMP( JA, JW ) +
     $                                   P( JR, J+JA-1 )*
     $                                   WORK( ( JW+1 )*N+JR )
  100                CONTINUE
  110             CONTINUE
  120          CONTINUE
*
               DO 130 JA = 1, NA
                  IF( ILCPLX ) THEN
                     SUM( JA, 1 ) = -ACOEF*SUMS( JA, 1 ) +
     $                               BCOEFR*SUMP( JA, 1 ) -
     $                               BCOEFI*SUMP( JA, 2 )
                     SUM( JA, 2 ) = -ACOEF*SUMS( JA, 2 ) +
     $                               BCOEFR*SUMP( JA, 2 ) +
     $                               BCOEFI*SUMP( JA, 1 )
                  ELSE
                     SUM( JA, 1 ) = -ACOEF*SUMS( JA, 1 ) +
     $                               BCOEFR*SUMP( JA, 1 )
                  END IF
  130          CONTINUE
```

```
*
*                                T
*             Solve  ( a A - b B )  y = SUM(,)
*             with scaling and perturbation of the denominator
*
              CALL SLALN2( .TRUE., NA, NW, DMIN, ACOEF, S( J, J ), LDS,
     $                     BDIAG( 1 ), BDIAG( 2 ), SUM, 2, BCOEFR,
     $                     BCOEFI, WORK( 2*N+J ), N, SCALE, TEMP,
     $                     IINFO )
              IF( SCALE.LT.ONE ) THEN
                 DO 150 JW = 0, NW - 1
                    DO 140 JR = JE, J - 1
                       WORK( ( JW+2 )*N+JR ) = SCALE*
     $                    WORK( ( JW+2 )*N+JR )
  140               CONTINUE
  150            CONTINUE
                 XMAX = SCALE*XMAX
              END IF
              XMAX = MAX( XMAX, TEMP )
  160       CONTINUE
*
*           Copy eigenvector to VL, back transforming if
*           HOWMNY='B'.
*
            IEIG = IEIG + 1
            IF( ILBACK ) THEN
              DO 170 JW = 0, NW - 1
                 CALL SGEMV( 'N', N, N+1-JE, ONE, VL( 1, JE ), LDVL,
     $                       WORK( ( JW+2 )*N+JE ), 1, ZERO,
     $                       WORK( ( JW+4 )*N+1 ), 1 )
  170         CONTINUE
              CALL SLACPY( ' ', N, NW, WORK( 4*N+1 ), N, VL( 1, JE ),
     $                     LDVL )
              IBEG = 1
            ELSE
              CALL SLACPY( ' ', N, NW, WORK( 2*N+1 ), N, VL( 1, IEIG ),
     $                     LDVL )
              IBEG = JE
            END IF
*
*           Scale eigenvector
*
            XMAX = ZERO
            IF( ILCPLX ) THEN
              DO 180 J = IBEG, N
                 XMAX = MAX( XMAX, ABS( VL( J, IEIG ) )+
     $                  ABS( VL( J, IEIG+1 ) ) )
  180         CONTINUE
            ELSE
              DO 190 J = IBEG, N
```

```
                 XMAX = MAX( XMAX, ABS( VL( J, IEIG ) ) )
  190         CONTINUE
           END IF
*
           IF( XMAX.GT.SAFMIN ) THEN
              XSCALE = ONE / XMAX
*
              DO 210 JW = 0, NW - 1
                 DO 200 JR = IBEG, N
                    VL( JR, IEIG+JW ) = XSCALE*VL( JR, IEIG+JW )
  200            CONTINUE
  210         CONTINUE
           END IF
           IEIG = IEIG + NW - 1
*
  220    CONTINUE
      END IF
*
*     Right eigenvectors
*
      IF( COMPR ) THEN
         IEIG = IM + 1
*
*        Main loop over eigenvalues
*
         ILCPLX = .FALSE.
         DO 500 JE = N, 1, -1
*
*           Skip this iteration if (a) HOWMNY='S' and SELECT=.FALSE., or
*           (b) this would be the second of a complex pair.
*           Check for complex eigenvalue, so as to be sure of which
*           entry(-ies) of SELECT to look at -- if complex, SELECT(JE)
*           or SELECT(JE-1).
*           If this is a complex pair, the 2-by-2 diagonal block
*           corresponding to the eigenvalue is in rows/columns JE-1:JE
*
           IF( ILCPLX ) THEN
              ILCPLX = .FALSE.
              GO TO 500
           END IF
           NW = 1
           IF( JE.GT.1 ) THEN
              IF( S( JE, JE-1 ).NE.ZERO ) THEN
                 ILCPLX = .TRUE.
                 NW = 2
              END IF
           END IF
           IF( ILALL ) THEN
              ILCOMP = .TRUE.
           ELSE IF( ILCPLX ) THEN
```

```
               ILCOMP = SELECT( JE ) .OR. SELECT( JE-1 )
            ELSE
               ILCOMP = SELECT( JE )
            END IF
            IF( .NOT.ILCOMP )
     $         GO TO 500
*
*           Decide if (a) singular pencil, (b) real eigenvalue, or
*           (c) complex eigenvalue.
*
            IF( .NOT.ILCPLX ) THEN
               IF( ABS( S( JE, JE ) ).LE.SAFMIN .AND.
     $             ABS( P( JE, JE ) ).LE.SAFMIN ) THEN
*
*                 Singular matrix pencil -- unit eigenvector
*
                  IEIG = IEIG - 1
                  DO 230 JR = 1, N
                     VR( JR, IEIG ) = ZERO
  230             CONTINUE
                  VR( IEIG, IEIG ) = ONE
                  GO TO 500
               END IF
            END IF
*
*           Clear vector
*
            DO 250 JW = 0, NW - 1
               DO 240 JR = 1, N
                  WORK( ( JW+2 )*N+JR ) = ZERO
  240          CONTINUE
  250       CONTINUE
*
*           Compute coefficients in  ( a A - b B ) x = 0
*              a  is  ACOEF
*              b  is  BCOEFR + i*BCOEFI
*
            IF( .NOT.ILCPLX ) THEN
*
*              Real eigenvalue
*
               TEMP = ONE / MAX( ABS( S( JE, JE ) )*ASCALE,
     $                ABS( P( JE, JE ) )*BSCALE, SAFMIN )
               SALFAR = ( TEMP*S( JE, JE ) )*ASCALE
               SBETA = ( TEMP*P( JE, JE ) )*BSCALE
               ACOEF = SBETA*ASCALE
               BCOEFR = SALFAR*BSCALE
               BCOEFI = ZERO
*
*              Scale to avoid underflow
```

```
*
              SCALE = ONE
              LSA = ABS( SBETA ).GE.SAFMIN .AND. ABS( ACOEF ).LT.SMALL
              LSB = ABS( SALFAR ).GE.SAFMIN .AND. ABS( BCOEFR ).LT.
     $              SMALL
              IF( LSA )
     $           SCALE = ( SMALL / ABS( SBETA ) )*MIN( ANORM, BIG )
              IF( LSB )
     $           SCALE = MAX( SCALE, ( SMALL / ABS( SALFAR ) )*
     $                   MIN( BNORM, BIG ) )
              IF( LSA .OR. LSB ) THEN
                 SCALE = MIN( SCALE, ONE /
     $                   ( SAFMIN*MAX( ONE, ABS( ACOEF ),
     $                   ABS( BCOEFR ) ) ) )
                 IF( LSA ) THEN
                    ACOEF = ASCALE*( SCALE*SBETA )
                 ELSE
                    ACOEF = SCALE*ACOEF
                 END IF
                 IF( LSB ) THEN
                    BCOEFR = BSCALE*( SCALE*SALFAR )
                 ELSE
                    BCOEFR = SCALE*BCOEFR
                 END IF
              END IF
              ACOEFA = ABS( ACOEF )
              BCOEFA = ABS( BCOEFR )
*
*             First component is 1
*
              WORK( 2*N+JE ) = ONE
              XMAX = ONE
*
*             Compute contribution from column JE of A and B to sum
*             (See "Further Details", above.)
*
              DO 260 JR = 1, JE - 1
                 WORK( 2*N+JR ) = BCOEFR*P( JR, JE ) -
     $                            ACOEF*S( JR, JE )
  260         CONTINUE
           ELSE
*
*             Complex eigenvalue
*
              CALL SLAG2( S( JE-1, JE-1 ), LDS, P( JE-1, JE-1 ), LDP,
     $                    SAFMIN*SAFETY, ACOEF, TEMP, BCOEFR, TEMP2,
     $                    BCOEFI )
              IF( BCOEFI.EQ.ZERO ) THEN
                 INFO = JE - 1
                 RETURN
```

```
               END IF
*
*              Scale to avoid over/underflow
*
               ACOEFA = ABS( ACOEF )
               BCOEFA = ABS( BCOEFR ) + ABS( BCOEFI )
               SCALE = ONE
               IF( ACOEFA*ULP.LT.SAFMIN .AND. ACOEFA.GE.SAFMIN )
     $            SCALE = ( SAFMIN / ULP ) / ACOEFA
               IF( BCOEFA*ULP.LT.SAFMIN .AND. BCOEFA.GE.SAFMIN )
     $            SCALE = MAX( SCALE, ( SAFMIN / ULP ) / BCOEFA )
               IF( SAFMIN*ACOEFA.GT.ASCALE )
     $            SCALE = ASCALE / ( SAFMIN*ACOEFA )
               IF( SAFMIN*BCOEFA.GT.BSCALE )
     $            SCALE = MIN( SCALE, BSCALE / ( SAFMIN*BCOEFA ) )
               IF( SCALE.NE.ONE ) THEN
                  ACOEF = SCALE*ACOEF
                  ACOEFA = ABS( ACOEF )
                  BCOEFR = SCALE*BCOEFR
                  BCOEFI = SCALE*BCOEFI
                  BCOEFA = ABS( BCOEFR ) + ABS( BCOEFI )
               END IF
*
*              Compute first two components of eigenvector
*              and contribution to sums
*
               TEMP = ACOEF*S( JE, JE-1 )
               TEMP2R = ACOEF*S( JE, JE ) - BCOEFR*P( JE, JE )
               TEMP2I = -BCOEFI*P( JE, JE )
               IF( ABS( TEMP ).GE.ABS( TEMP2R )+ABS( TEMP2I ) ) THEN
                  WORK( 2*N+JE ) = ONE
                  WORK( 3*N+JE ) = ZERO
                  WORK( 2*N+JE-1 ) = -TEMP2R / TEMP
                  WORK( 3*N+JE-1 ) = -TEMP2I / TEMP
               ELSE
                  WORK( 2*N+JE-1 ) = ONE
                  WORK( 3*N+JE-1 ) = ZERO
                  TEMP = ACOEF*S( JE-1, JE )
                  WORK( 2*N+JE ) = ( BCOEFR*P( JE-1, JE-1 )-ACOEF*
     $                             S( JE-1, JE-1 ) ) / TEMP
                  WORK( 3*N+JE ) = BCOEFI*P( JE-1, JE-1 ) / TEMP
               END IF
*
               XMAX = MAX( ABS( WORK( 2*N+JE ) )+ABS( WORK( 3*N+JE ) ),
     $                ABS( WORK( 2*N+JE-1 ) )+ABS( WORK( 3*N+JE-1 ) ) )
*
*              Compute contribution from columns JE and JE-1
*              of A and B to the sums.
*
               CREALA = ACOEF*WORK( 2*N+JE-1 )
```

```
               CIMAGA = ACOEF*WORK( 3*N+JE-1 )
               CREALB = BCOEFR*WORK( 2*N+JE-1 ) -
     $                  BCOEFI*WORK( 3*N+JE-1 )
               CIMAGB = BCOEFI*WORK( 2*N+JE-1 ) +
     $                  BCOEFR*WORK( 3*N+JE-1 )
               CRE2A = ACOEF*WORK( 2*N+JE )
               CIM2A = ACOEF*WORK( 3*N+JE )
               CRE2B = BCOEFR*WORK( 2*N+JE ) - BCOEFI*WORK( 3*N+JE )
               CIM2B = BCOEFI*WORK( 2*N+JE ) + BCOEFR*WORK( 3*N+JE )
               DO 270 JR = 1, JE - 2
                  WORK( 2*N+JR ) = -CREALA*S( JR, JE-1 ) +
     $                             CREALB*P( JR, JE-1 ) -
     $                             CRE2A*S( JR, JE ) + CRE2B*P( JR, JE )
                  WORK( 3*N+JR ) = -CIMAGA*S( JR, JE-1 ) +
     $                             CIMAGB*P( JR, JE-1 ) -
     $                             CIM2A*S( JR, JE ) + CIM2B*P( JR, JE )
  270          CONTINUE
            END IF
*
            DMIN = MAX( ULP*ACOEFA*ANORM, ULP*BCOEFA*BNORM, SAFMIN )
*
*           Columnwise triangular solve of  (a A - b B)  x = 0
*
            IL2BY2 = .FALSE.
            DO 370 J = JE - NW, 1, -1
*
*              If a 2-by-2 block, is in position j-1:j, wait until
*              next iteration to process it (when it will be j:j+1)
*
               IF( .NOT.IL2BY2 .AND. J.GT.1 ) THEN
                  IF( S( J, J-1 ).NE.ZERO ) THEN
                     IL2BY2 = .TRUE.
                     GO TO 370
                  END IF
               END IF
               BDIAG( 1 ) = P( J, J )
               IF( IL2BY2 ) THEN
                  NA = 2
                  BDIAG( 2 ) = P( J+1, J+1 )
               ELSE
                  NA = 1
               END IF
*
*              Compute x(j) (and x(j+1), if 2-by-2 block)
*
               CALL SLALN2( .FALSE., NA, NW, DMIN, ACOEF, S( J, J ),
     $                    LDS, BDIAG( 1 ), BDIAG( 2 ), WORK( 2*N+J ),
     $                    N, BCOEFR, BCOEFI, SUM, 2, SCALE, TEMP,
     $                    IINFO )
               IF( SCALE.LT.ONE ) THEN
```

```
*
                 DO 290 JW = 0, NW - 1
                    DO 280 JR = 1, JE
                       WORK( ( JW+2 )*N+JR ) = SCALE*
     $                    WORK( ( JW+2 )*N+JR )
  280               CONTINUE
  290            CONTINUE
              END IF
              XMAX = MAX( SCALE*XMAX, TEMP )
*
              DO 310 JW = 1, NW
                 DO 300 JA = 1, NA
                    WORK( ( JW+1 )*N+J+JA-1 ) = SUM( JA, JW )
  300            CONTINUE
  310         CONTINUE
*
*             w = w + x(j)*(a S(*,j) - b P(*,j) ) with scaling
*
              IF( J.GT.1 ) THEN
*
*                Check whether scaling is necessary for sum.
*
                 XSCALE = ONE / MAX( ONE, XMAX )
                 TEMP = ACOEFA*WORK( J ) + BCOEFA*WORK( N+J )
                 IF( IL2BY2 )
     $              TEMP = MAX( TEMP, ACOEFA*WORK( J+1 )+BCOEFA*
     $                      WORK( N+J+1 ) )
                 TEMP = MAX( TEMP, ACOEFA, BCOEFA )
                 IF( TEMP.GT.BIGNUM*XSCALE ) THEN
*
                    DO 330 JW = 0, NW - 1
                       DO 320 JR = 1, JE
                          WORK( ( JW+2 )*N+JR ) = XSCALE*
     $                       WORK( ( JW+2 )*N+JR )
  320                 CONTINUE
  330                CONTINUE
                    XMAX = XMAX*XSCALE
                 END IF
*
*                Compute the contributions of the off-diagonals of
*                column j (and j+1, if 2-by-2 block) of A and B to the
*                sums.
*
*
                 DO 360 JA = 1, NA
                    IF( ILCPLX ) THEN
                       CREALA = ACOEF*WORK( 2*N+J+JA-1 )
                       CIMAGA = ACOEF*WORK( 3*N+J+JA-1 )
                       CREALB = BCOEFR*WORK( 2*N+J+JA-1 ) -
     $                          BCOEFI*WORK( 3*N+J+JA-1 )
```

```
                      CIMAGB = BCOEFI*WORK( 2*N+J+JA-1 ) +
     $                         BCOEFR*WORK( 3*N+J+JA-1 )
                      DO 340 JR = 1, J - 1
                         WORK( 2*N+JR ) = WORK( 2*N+JR ) -
     $                                    CREALA*S( JR, J+JA-1 ) +
     $                                    CREALB*P( JR, J+JA-1 )
                         WORK( 3*N+JR ) = WORK( 3*N+JR ) -
     $                                    CIMAGA*S( JR, J+JA-1 ) +
     $                                    CIMAGB*P( JR, J+JA-1 )
  340                 CONTINUE
                   ELSE
                      CREALA = ACOEF*WORK( 2*N+J+JA-1 )
                      CREALB = BCOEFR*WORK( 2*N+J+JA-1 )
                      DO 350 JR = 1, J - 1
                         WORK( 2*N+JR ) = WORK( 2*N+JR ) -
     $                                    CREALA*S( JR, J+JA-1 ) +
     $                                    CREALB*P( JR, J+JA-1 )
  350                 CONTINUE
                   END IF
  360            CONTINUE
              END IF
*
              IL2BY2 = .FALSE.
  370       CONTINUE
*
*           Copy eigenvector to VR, back transforming if
*           HOWMNY='B'.
*
            IEIG = IEIG - NW
            IF( ILBACK ) THEN
*
              DO 410 JW = 0, NW - 1
                 DO 380 JR = 1, N
                    WORK( ( JW+4 )*N+JR ) = WORK( ( JW+2 )*N+1 )*
     $                                      VR( JR, 1 )
  380            CONTINUE
*
*                A series of compiler directives to defeat
*                vectorization for the next loop
*
*
                 DO 400 JC = 2, JE
                    DO 390 JR = 1, N
                       WORK( ( JW+4 )*N+JR ) = WORK( ( JW+4 )*N+JR ) +
     $                    WORK( ( JW+2 )*N+JC )*VR( JR, JC )
  390               CONTINUE
  400            CONTINUE
  410         CONTINUE
*
              DO 430 JW = 0, NW - 1
```

```
                 DO 420 JR = 1, N
                    VR( JR, IEIG+JW ) = WORK( ( JW+4 )*N+JR )
  420            CONTINUE
  430         CONTINUE
*
              IEND = N
           ELSE
              DO 450 JW = 0, NW - 1
                 DO 440 JR = 1, N
                    VR( JR, IEIG+JW ) = WORK( ( JW+2 )*N+JR )
  440            CONTINUE
  450         CONTINUE
*
              IEND = JE
           END IF
*
*          Scale eigenvector
*
           XMAX = ZERO
           IF( ILCPLX ) THEN
              DO 460 J = 1, IEND
                 XMAX = MAX( XMAX, ABS( VR( J, IEIG ) )+
     $                  ABS( VR( J, IEIG+1 ) ) )
  460         CONTINUE
           ELSE
              DO 470 J = 1, IEND
                 XMAX = MAX( XMAX, ABS( VR( J, IEIG ) ) )
  470         CONTINUE
           END IF
*
           IF( XMAX.GT.SAFMIN ) THEN
              XSCALE = ONE / XMAX
              DO 490 JW = 0, NW - 1
                 DO 480 JR = 1, IEND
                    VR( JR, IEIG+JW ) = XSCALE*VR( JR, IEIG+JW )
  480            CONTINUE
  490         CONTINUE
           END IF
  500    CONTINUE
      END IF
*
      RETURN
*
*     End of STGEVC
*
      END
```

———-

33.39.4 lisp code

— stgevc.lisp —

```
(let* ((zero 0.0) (one 1.0) (safety 100.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 100.0 100.0) safety)
           (ignorable zero one safety))
  (defun stgevc
         (side howmny select n s lds p ldp vl ldvl vr ldvr mm m work info)
    (declare (type (array single-float (*)) work vr vl p s)
             (type (f2cl-lib:integer4) info m mm ldvr ldvl ldp lds n)
             (type (array f2cl-lib:logical (*)) select)
             (type (string *) howmny side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (howmny character howmny-%data% howmny-%offset%)
         (select f2cl-lib:logical select-%data% select-%offset%)
         (s single-float s-%data% s-%offset%)
         (p single-float p-%data% p-%offset%)
         (vl single-float vl-%data% vl-%offset%)
         (vr single-float vr-%data% vr-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((bdiag (make-array 2 :element-type 'single-float))
             (sum (make-array 4 :element-type 'single-float))
             (sums (make-array 4 :element-type 'single-float))
             (sump (make-array 4 :element-type 'single-float)) (acoef 0.0)
             (acoefa 0.0) (anorm 0.0) (ascale 0.0) (bcoefa 0.0) (bcoefi 0.0)
             (bcoefr 0.0) (big 0.0) (bignum 0.0) (bnorm 0.0) (bscale 0.0)
             (cim2a 0.0) (cim2b 0.0) (cimaga 0.0) (cimagb 0.0) (cre2a 0.0)
             (cre2b 0.0) (creala 0.0) (crealb 0.0) (dmin 0.0) (safmin 0.0)
             (salfar 0.0) (sbeta 0.0) (scale 0.0) (small 0.0) (temp 0.0)
             (temp2 0.0) (temp2i 0.0) (temp2r 0.0) (ulp 0.0) (xmax 0.0)
             (xscale 0.0) (i 0) (ibeg 0) (ieig 0) (iend 0) (ihwmny 0) (iinfo 0)
             (im 0) (iside 0) (j 0) (ja 0) (jc 0) (je 0) (jr 0) (jw 0) (na 0)
             (nw 0) (compl nil) (compr nil) (il2by2 nil) (ilabad nil)
             (ilall nil) (ilback nil) (ilbbad nil) (ilcomp nil) (ilcplx nil)
             (lsa nil) (lsb nil))
        (declare (type (array single-float (2)) bdiag)
                 (type (array single-float (4)) sum sums sump)
                 (type (single-float) acoef acoefa anorm ascale bcoefa bcoefi
                  bcoefr big bignum bnorm bscale cim2a cim2b cimaga cimagb
                  cre2a cre2b creala crealb dmin safmin salfar sbeta scale
                  small temp temp2 temp2i temp2r ulp xmax xscale)
                 (type (f2cl-lib:integer4) i ibeg ieig iend ihwmny iinfo im
                  iside j ja jc je jr jw na nw)
                 (type f2cl-lib:logical compl compr il2by2 ilabad ilall ilback
                  ilbbad ilcomp ilcplx lsa lsb))
```

```
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame howmny "A")
    (declare (ignore var-1))
    (when var-0 (setf howmny var-0))
    ret-val)
  (setf ihwmny 1) (setf ilall f2cl-lib:%true%)
  (setf ilback f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame howmny "S")
    (declare (ignore var-1))
    (when var-0 (setf howmny var-0))
    ret-val)
  (setf ihwmny 2) (setf ilall f2cl-lib:%false%)
  (setf ilback f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame howmny "B")
    (declare (ignore var-1))
    (when var-0 (setf howmny var-0))
    ret-val)
  (setf ihwmny 3) (setf ilall f2cl-lib:%true%)
  (setf ilback f2cl-lib:%true%))
 (t (setf ihwmny -1) (setf ilall f2cl-lib:%true%)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame side "R")
    (declare (ignore var-1))
    (when var-0 (setf side var-0))
    ret-val)
  (setf iside 1) (setf compl f2cl-lib:%false%)
  (setf compr f2cl-lib:%true%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame side "L")
    (declare (ignore var-1))
    (when var-0 (setf side var-0))
    ret-val)
  (setf iside 2) (setf compl f2cl-lib:%true%)
  (setf compr f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame side "B")
    (declare (ignore var-1))
    (when var-0 (setf side var-0))
    ret-val)
  (setf iside 3) (setf compl f2cl-lib:%true%)
  (setf compr f2cl-lib:%true%))
 (t (setf iside -1)))
(setf info 0)
(cond ((< iside 0) (setf info -1)) ((< ihwmny 0) (setf info -2))
      ((< n 0) (setf info -4))
      ((< lds
```

```
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -6))
      ((< ldp
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -8)))
(cond
 ((/= info 0) (xerbla "STGEVC" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((not ilall) (setf im 0) (setf ilcplx f2cl-lib:%false%)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (cond (ilcplx (setf ilcplx f2cl-lib:%false%) (go label10)))
      (cond
       ((< j n)
        (if (/=
             (f2cl-lib:fref s-%data% ((f2cl-lib:int-add j 1) j)
                            ((1 lds) (1 *)) s-%offset%)
             zero)
            (setf ilcplx f2cl-lib:%true%))))
      (cond
       (ilcplx
        (if (or
             (f2cl-lib:fref select-%data% (j) ((1 *)) select-%offset%)
             (f2cl-lib:fref select-%data% ((f2cl-lib:int-add j 1))
                            ((1 *)) select-%offset%))
            (setf im (f2cl-lib:int-add im 2))))
       (t
        (if (f2cl-lib:fref select-%data% (j) ((1 *)) select-%offset%)
            (setf im (f2cl-lib:int-add im 1)))))
     label10)))
 (t (setf im n)))
(setf ilabad f2cl-lib:%false%)
(setf ilbbad f2cl-lib:%false%)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (cond
     ((/= (f2cl-lib:fref s ((f2cl-lib:int-add j 1) j) ((1 lds) (1 *)))
          zero)
      (if (or
           (= (f2cl-lib:fref p-%data% (j j) ((1 ldp) (1 *)) p-%offset%)
              zero)
           (=
            (f2cl-lib:fref p-%data%
                           ((f2cl-lib:int-add j 1)
                            (f2cl-lib:int-add j 1))
                           ((1 ldp) (1 *)) p-%offset%)
            zero)
```

```
            (/=
             (f2cl-lib:fref p-%data% (j (f2cl-lib:int-add j 1))
                            ((1 ldp) (1 *)) p-%offset%)
             zero))
           (setf ilbbad f2cl-lib:%true%))
       (cond
        ((< j (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
         (if (/=
              (f2cl-lib:fref s-%data%
                             ((f2cl-lib:int-add j 2)
                              (f2cl-lib:int-add j 1))
                             ((1 lds) (1 *)) s-%offset%)
              zero)
             (setf ilabad f2cl-lib:%true%))))))
   label20))
(cond (ilabad (setf info -5)) (ilbbad (setf info -7))
      ((or (and compl (< ldvl n)) (< ldvl 1)) (setf info -10))
      ((or (and compr (< ldvr n)) (< ldvr 1)) (setf info -12))
      ((< mm im) (setf info -13)))
(cond
 ((/= info 0) (xerbla "STGEVC" (f2cl-lib:int-sub info))
  (go end_label)))
(setf m im)
(if (= n 0)
    (go end_label))
(setf safmin (slamch "Safe minimum"))
(setf big (/ one safmin))
(multiple-value-bind (var-0 var-1)
    (slabad safmin big)
  (declare (ignore))
  (setf safmin var-0)
  (setf big var-1))
(setf ulp (* (slamch "Epsilon") (slamch "Base")))
(setf small (/ (* safmin n) ulp))
(setf big (/ one small))
(setf bignum (/ one (* safmin n)))
(setf anorm
        (abs
         (f2cl-lib:fref s-%data% (1 1) ((1 lds) (1 *)) s-%offset%)))
(if (> n 1)
    (setf anorm
            (+ anorm
               (abs
                (f2cl-lib:fref s-%data% (2 1) ((1 lds) (1 *))
                               s-%offset%)))))
(setf bnorm
        (abs
         (f2cl-lib:fref p-%data% (1 1) ((1 ldp) (1 *)) p-%offset%)))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) zero)
(setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 1)) ((1 *))
```

```
                    work-%offset%)
        zero)
(f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf temp zero)
    (setf temp2 zero)
    (cond
     ((=
       (f2cl-lib:fref s (j (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((1 lds) (1 *)))
       zero)
      (setf iend (f2cl-lib:int-sub j 1)))
     (t (setf iend (f2cl-lib:int-sub j 2))))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i iend) nil)
      (tagbody
        (setf temp
                (+ temp
                   (abs
                    (f2cl-lib:fref s-%data% (i j) ((1 lds) (1 *))
                                   s-%offset%))))
        (setf temp2
                (+ temp2
                   (abs
                    (f2cl-lib:fref p-%data% (i j) ((1 ldp) (1 *))
                                   p-%offset%))))
       label30))
    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%) temp)
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j)) ((1 *))
                         work-%offset%)
          temp2)
    (f2cl-lib:fdo (i (f2cl-lib:int-add iend 1) (f2cl-lib:int-add i 1))
                  ((> i
                      (min
                       (the f2cl-lib:integer4 (f2cl-lib:int-add j 1))
                       (the f2cl-lib:integer4 n)))
                   nil)
      (tagbody
        (setf temp
                (+ temp
                   (abs
                    (f2cl-lib:fref s-%data% (i j) ((1 lds) (1 *))
                                   s-%offset%))))
        (setf temp2
                (+ temp2
                   (abs
                    (f2cl-lib:fref p-%data% (i j) ((1 ldp) (1 *))
                                   p-%offset%))))
       label40))
```

```
    (setf anorm (max anorm temp))
    (setf bnorm (max bnorm temp2))
   label50))
(setf ascale (/ one (max anorm safmin)))
(setf bscale (/ one (max bnorm safmin)))
(cond
 (compl (setf ieig 0) (setf ilcplx f2cl-lib:%false%)
  (f2cl-lib:fdo (je 1 (f2cl-lib:int-add je 1))
                ((> je n) nil)
    (tagbody
      (cond (ilcplx (setf ilcplx f2cl-lib:%false%) (go label220)))
      (setf nw 1)
      (cond
       ((< je n)
        (cond
         ((/=
           (f2cl-lib:fref s ((f2cl-lib:int-add je 1) je)
                          ((1 lds) (1 *)))
           zero)
          (setf ilcplx f2cl-lib:%true%) (setf nw 2)))))
      (cond (ilall (setf ilcomp f2cl-lib:%true%))
            (ilcplx
             (setf ilcomp
                     (or
                      (f2cl-lib:fref select-%data% (je) ((1 *))
                                     select-%offset%)
                      (f2cl-lib:fref select-%data%
                                     ((f2cl-lib:int-add je 1)) ((1 *))
                                     select-%offset%))))
            (t
             (setf ilcomp
                     (f2cl-lib:fref select-%data% (je) ((1 *))
                                    select-%offset%))))
      (if (not ilcomp)
          (go label220))
      (cond
       ((not ilcplx)
        (cond
         ((and
           (<= (abs (f2cl-lib:fref s (je je) ((1 lds) (1 *)))) safmin)
           (<= (abs (f2cl-lib:fref p (je je) ((1 ldp) (1 *)))) safmin))
          (setf ieig (f2cl-lib:int-add ieig 1))
          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                        ((> jr n) nil)
            (tagbody
              (setf (f2cl-lib:fref vl-%data% (jr ieig) ((1 ldvl) (1 *))
                                   vl-%offset%)
                    zero)
             label60))
          (setf (f2cl-lib:fref vl-%data% (ieig ieig) ((1 ldvl) (1 *))
```

```
                          vl-%offset%)
            one)
    (go label220)))))
(f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
              ((> jr (f2cl-lib:int-mul nw n)) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add
                           (f2cl-lib:int-mul 2 n) jr))
                         ((1 *)) work-%offset%)
            zero)
   label70))
(cond
 ((not ilcplx)
  (setf temp
          (/ one
             (max
              (*
               (abs
                (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                               s-%offset%))
               ascale)
              (*
               (abs
                (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
                               p-%offset%))
               bscale)
              safmin)))
  (setf salfar
          (* temp
             (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                            s-%offset%)
             ascale))
  (setf sbeta
          (* temp
             (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
                            p-%offset%)
             bscale))
  (setf acoef (* sbeta ascale)) (setf bcoefr (* salfar bscale))
  (setf bcoefi zero) (setf scale one)
  (setf lsa (and (>= (abs sbeta) safmin) (< (abs acoef) small)))
  (setf lsb
          (and (>= (abs salfar) safmin) (< (abs bcoefr) small)))
  (if lsa
      (setf scale (* (/ small (abs sbeta)) (min anorm big))))
  (if lsb
      (setf scale
              (max scale
                   (* (/ small (abs salfar)) (min bnorm big)))))
  (cond
```

```
  ((or lsa lsb)
   (setf scale
           (min scale
                (/ one
                   (* safmin
                      (max one (abs acoef) (abs bcoefr))))))
   (cond (lsa (setf acoef (* ascale (* scale sbeta))))
         (t (setf acoef (* scale acoef))))
   (cond (lsb (setf bcoefr (* bscale (* scale salfar))))
         (t (setf bcoefr (* scale bcoefr))))))
 (setf acoefa (abs acoef)) (setf bcoefa (abs bcoefr))
 (setf (f2cl-lib:fref work-%data%
                      ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                         je))
                      ((1 *)) work-%offset%)
         one)
 (setf xmax one))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9)
     (slag2
      (f2cl-lib:array-slice s-%data% single-float (je je)
                            ((1 lds) (1 *)) s-%offset%)
      lds
      (f2cl-lib:array-slice p-%data% single-float (je je)
                            ((1 ldp) (1 *)) p-%offset%)
      ldp (* safmin safety) acoef temp bcoefr temp2 bcoefi)
   (declare (ignore var-0 var-1 var-2 var-3 var-4))
   (setf acoef var-5)
   (setf temp var-6)
   (setf bcoefr var-7)
   (setf temp2 var-8)
   (setf bcoefi var-9))
 (setf bcoefi (- bcoefi))
 (cond ((= bcoefi zero) (setf info je) (go end_label)))
 (setf acoefa (abs acoef))
 (setf bcoefa (+ (abs bcoefr) (abs bcoefi))) (setf scale one)
 (if (and (< (* acoefa ulp) safmin) (>= acoefa safmin))
     (setf scale (/ (/ safmin ulp) acoefa)))
 (if (and (< (* bcoefa ulp) safmin) (>= bcoefa safmin))
     (setf scale (max scale (/ (/ safmin ulp) bcoefa))))
 (if (> (* safmin acoefa) ascale)
     (setf scale (/ ascale (* safmin acoefa))))
 (if (> (* safmin bcoefa) bscale)
     (setf scale (min scale (/ bscale (* safmin bcoefa)))))
 (cond
  ((/= scale one) (setf acoef (* scale acoef))
   (setf acoefa (abs acoef)) (setf bcoefr (* scale bcoefr))
   (setf bcoefi (* scale bcoefi))
```

```
  (setf bcoefa (+ (abs bcoefr) (abs bcoefi)))))
(setf temp
        (* acoef
           (f2cl-lib:fref s-%data% ((f2cl-lib:int-add je 1) je)
                          ((1 lds) (1 *)) s-%offset%)))
(setf temp2r
        (-
         (* acoef
            (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                           s-%offset%))
         (* bcoefr
            (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
                           p-%offset%))))
(setf temp2i
        (* (- bcoefi)
           (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
                          p-%offset%)))
(cond
 ((> (abs temp) (+ (abs temp2r) (abs temp2i)))
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 2 n) je))
                       ((1 *)) work-%offset%)
         one)
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 3 n) je))
                       ((1 *)) work-%offset%)
         zero)
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 2 n) je 1))
                       ((1 *)) work-%offset%)
         (/ (- temp2r) temp))
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 3 n) je 1))
                       ((1 *)) work-%offset%)
         (/ (- temp2i) temp)))
 (t
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 2 n) je 1))
                       ((1 *)) work-%offset%)
         one)
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 3 n) je 1))
                       ((1 *)) work-%offset%)
         zero)
```

```
  (setf temp
          (* acoef
             (f2cl-lib:fref s-%data%
                            (je (f2cl-lib:int-add je 1))
                            ((1 lds) (1 *)) s-%offset%)))
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 2 n) je))
                       ((1 *)) work-%offset%)
          (/
           (-
            (* bcoefr
               (f2cl-lib:fref p-%data%
                              ((f2cl-lib:int-add je 1)
                               (f2cl-lib:int-add je 1))
                              ((1 ldp) (1 *)) p-%offset%))
            (* acoef
               (f2cl-lib:fref s-%data%
                              ((f2cl-lib:int-add je 1)
                               (f2cl-lib:int-add je 1))
                              ((1 lds) (1 *)) s-%offset%)))
           temp))
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 3 n) je))
                       ((1 *)) work-%offset%)
          (/
           (* bcoefi
              (f2cl-lib:fref p-%data%
                             ((f2cl-lib:int-add je 1)
                              (f2cl-lib:int-add je 1))
                             ((1 ldp) (1 *)) p-%offset%))
           temp))))
(setf xmax
        (max
         (+
          (abs
           (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add
                            (f2cl-lib:int-mul 2 n) je))
                          ((1 *)) work-%offset%))
          (abs
           (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add
                            (f2cl-lib:int-mul 3 n) je))
                          ((1 *)) work-%offset%)))
         (+
          (abs
           (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add
```

```
                                  (f2cl-lib:int-mul 2 n) je 1))
                                ((1 *)) work-%offset%))
                (abs
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add
                                  (f2cl-lib:int-mul 3 n) je 1))
                                ((1 *)) work-%offset%)))))))
(setf dmin
        (max (* ulp acoefa anorm) (* ulp bcoefa bnorm) safmin))
(setf il2by2 f2cl-lib:%false%)
(f2cl-lib:fdo (j (f2cl-lib:int-add je nw) (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (cond (il2by2 (setf il2by2 f2cl-lib:%false%) (go label160)))
    (setf na 1)
    (setf (f2cl-lib:fref bdiag (1) ((1 2)))
            (f2cl-lib:fref p-%data% (j j) ((1 ldp) (1 *))
                           p-%offset%))
    (cond
     ((< j n)
      (cond
       ((/=
         (f2cl-lib:fref s ((f2cl-lib:int-add j 1) j)
                        ((1 lds) (1 *)))
         zero)
        (setf il2by2 f2cl-lib:%true%)
        (setf (f2cl-lib:fref bdiag (2) ((1 2)))
                (f2cl-lib:fref p-%data%
                               ((f2cl-lib:int-add j 1)
                                (f2cl-lib:int-add j 1))
                               ((1 ldp) (1 *)) p-%offset%))
        (setf na 2)))))
    (setf xscale (/ one (max one xmax)))
    (setf temp
            (max
             (f2cl-lib:fref work-%data% (j) ((1 *))
                            work-%offset%)
             (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j))
                            ((1 *)) work-%offset%)
             (+
              (* acoefa
                 (f2cl-lib:fref work-%data% (j) ((1 *))
                                work-%offset%))
              (* bcoefa
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add n j)) ((1 *))
                                work-%offset%)))))
    (if il2by2
        (setf temp
                (max temp
```

```
                    (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-add j 1))
                                   ((1 *)) work-%offset%)
                    (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-add n j 1))
                                   ((1 *)) work-%offset%)
                    (+
                     (* acoefa
                        (f2cl-lib:fref work-%data%
                                       ((f2cl-lib:int-add j 1))
                                       ((1 *)) work-%offset%))
                     (* bcoefa
                        (f2cl-lib:fref work-%data%
                                       ((f2cl-lib:int-add n j
                                                          1))
                                       ((1 *))
                                       work-%offset%))))))
(cond
 ((> temp (* bignum xscale))
  (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                ((> jw
                    (f2cl-lib:int-add nw
                                      (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (f2cl-lib:fdo (jr je (f2cl-lib:int-add jr 1))
                    ((> jr
                        (f2cl-lib:int-add j
                                          (f2cl-lib:int-sub
                                           1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-add jw 2) n)
                                 jr))
                               ((1 *)) work-%offset%)
                     (* xscale
                        (f2cl-lib:fref work-%data%
                                       ((f2cl-lib:int-add
                                         (f2cl-lib:int-mul
                                          (f2cl-lib:int-add jw
                                                            2)
                                          n)
                                         jr))
                                       ((1 *)) work-%offset%)))
         label80))
     label90))
  (setf xmax (* xmax xscale))))
```

```
(f2cl-lib:fdo (jw 1 (f2cl-lib:int-add jw 1))
              ((> jw nw) nil)
  (tagbody
    (f2cl-lib:fdo (ja 1 (f2cl-lib:int-add ja 1))
                  ((> ja na) nil)
      (tagbody
        (setf (f2cl-lib:fref sums (ja jw) ((1 2) (1 2)))
                zero)
        (setf (f2cl-lib:fref sump (ja jw) ((1 2) (1 2)))
                zero)
        (f2cl-lib:fdo (jr je (f2cl-lib:int-add jr 1))
                      ((> jr
                          (f2cl-lib:int-add j
                                            (f2cl-lib:int-sub
                                             1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref sums (ja jw) ((1 2) (1 2)))
                    (+
                     (f2cl-lib:fref sums (ja jw)
                                    ((1 2) (1 2)))
                     (*
                      (f2cl-lib:fref s-%data%
                                     (jr
                                      (f2cl-lib:int-sub
                                       (f2cl-lib:int-add j
                                                         ja)
                                       1))
                                     ((1 lds) (1 *))
                                     s-%offset%)
                      (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-add
                                       (f2cl-lib:int-mul
                                        (f2cl-lib:int-add jw
                                                          1)
                                        n)
                                       jr))
                                     ((1 *))
                                     work-%offset%))))
            (setf (f2cl-lib:fref sump (ja jw) ((1 2) (1 2)))
                    (+
                     (f2cl-lib:fref sump (ja jw)
                                    ((1 2) (1 2)))
                     (*
                      (f2cl-lib:fref p-%data%
                                     (jr
                                      (f2cl-lib:int-sub
                                       (f2cl-lib:int-add j
                                                         ja)
                                       1))
```

```
                                        ((1 ldp) (1 *))
                                        p-%offset%)
                          (f2cl-lib:fref work-%data%
                                        ((f2cl-lib:int-add
                                          (f2cl-lib:int-mul
                                           (f2cl-lib:int-add jw
                                                             1)
                                           n)
                                          jr))
                                        ((1 *))
                                        work-%offset%))))
            label100))
        label110))
   label120))
(f2cl-lib:fdo (ja 1 (f2cl-lib:int-add ja 1))
              ((> ja na) nil)
  (tagbody
    (cond
     (ilcplx
      (setf (f2cl-lib:fref sum (ja 1) ((1 2) (1 2)))
              (-
               (+
                (* (- acoef)
                   (f2cl-lib:fref sums (ja 1) ((1 2) (1 2))))
                (* bcoefr
                   (f2cl-lib:fref sump (ja 1)
                                  ((1 2) (1 2)))))
               (* bcoefi
                  (f2cl-lib:fref sump (ja 2)
                                 ((1 2) (1 2))))))
      (setf (f2cl-lib:fref sum (ja 2) ((1 2) (1 2)))
              (+
               (* (- acoef)
                  (f2cl-lib:fref sums (ja 2) ((1 2) (1 2))))
               (* bcoefr
                  (f2cl-lib:fref sump (ja 2) ((1 2) (1 2))))
               (* bcoefi
                  (f2cl-lib:fref sump (ja 1)
                                 ((1 2) (1 2)))))))
     (t
      (setf (f2cl-lib:fref sum (ja 1) ((1 2) (1 2)))
              (+
               (* (- acoef)
                  (f2cl-lib:fref sums (ja 1) ((1 2) (1 2))))
               (* bcoefr
                  (f2cl-lib:fref sump (ja 1)
                                 ((1 2) (1 2))))))))
   label130))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
      var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
      var-17)
     (slaln2 f2cl-lib:%true% na nw dmin acoef
      (f2cl-lib:array-slice s-%data% single-float (j j)
                            ((1 lds) (1 *)) s-%offset%)
      lds (f2cl-lib:fref bdiag (1) ((1 2)))
      (f2cl-lib:fref bdiag (2) ((1 2))) sum 2 bcoefr bcoefi
      (f2cl-lib:array-slice work-%data% single-float
                            ((+ (f2cl-lib:int-mul 2 n) j))
                            ((1 *)) work-%offset%)
      n scale temp iinfo)
   (declare
    (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
     var-8 var-9 var-10 var-11 var-12 var-13 var-14))
   (setf scale var-15)
   (setf temp var-16)
   (setf iinfo var-17))
 (cond
  ((< scale one)
   (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                 ((> jw
                     (f2cl-lib:int-add nw
                                       (f2cl-lib:int-sub 1)))
                  nil)
     (tagbody
       (f2cl-lib:fdo (jr je (f2cl-lib:int-add jr 1))
                     ((> jr
                         (f2cl-lib:int-add j
                                           (f2cl-lib:int-sub
                                            1)))
                      nil)
         (tagbody
           (setf (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add
                                  (f2cl-lib:int-mul
                                   (f2cl-lib:int-add jw 2) n)
                                  jr))
                                ((1 *)) work-%offset%)
                 (* scale
                    (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-add
                                     (f2cl-lib:int-mul
                                      (f2cl-lib:int-add jw
                                                        2)
                                      n)
                                     jr))
                                   ((1 *)) work-%offset%)))
          label140))
      label150))
   (setf xmax (* scale xmax))))
```

```
    (setf xmax (max xmax temp))
   label160))
(setf ieig (f2cl-lib:int-add ieig 1))
(cond
 (ilback
  (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                ((> jw
                    (f2cl-lib:int-add nw (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (sgemv "N" n
           (f2cl-lib:int-sub (f2cl-lib:int-add n 1) je) one
           (f2cl-lib:array-slice vl-%data% single-float (1 je)
                                 ((1 ldvl) (1 *)) vl-%offset%)
           ldvl
           (f2cl-lib:array-slice work-%data% single-float
                                 ((+
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-add jw 2) n)
                                   je))
                                 ((1 *)) work-%offset%)
           1 zero
           (f2cl-lib:array-slice work-%data% single-float
                                 ((+
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-add jw 4) n)
                                   1))
                                 ((1 *)) work-%offset%)
           1)
        (declare
         (ignore var-0 var-2 var-4 var-6 var-7 var-9 var-10))
        (when var-1 (setf n var-1))
        (when var-3 (setf one var-3))
        (when var-5 (setf ldvl var-5))
        (when var-8 (setf zero var-8)))
     label170))
  (slacpy " " n nw
   (f2cl-lib:array-slice work-%data% single-float
                         ((+ (f2cl-lib:int-mul 4 n) 1)) ((1 *))
                         work-%offset%)
   n
   (f2cl-lib:array-slice vl-%data% single-float (1 je)
                         ((1 ldvl) (1 *)) vl-%offset%)
   ldvl)
  (setf ibeg 1))
 (t
  (slacpy " " n nw
```

```
    (f2cl-lib:array-slice work-%data% single-float
                          ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                          work-%offset%)
    n
    (f2cl-lib:array-slice vl-%data% single-float (1 ieig)
                          ((1 ldvl) (1 *)) vl-%offset%)
    ldvl)
   (setf ibeg je)))
 (setf xmax zero)
 (cond
  (ilcplx
   (f2cl-lib:fdo (j ibeg (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf xmax
               (max xmax
                    (+
                     (abs
                      (f2cl-lib:fref vl-%data% (j ieig)
                                     ((1 ldvl) (1 *))
                                     vl-%offset%))
                     (abs
                      (f2cl-lib:fref vl-%data%
                                     (j (f2cl-lib:int-add ieig 1))
                                     ((1 ldvl) (1 *))
                                     vl-%offset%)))))
     label180)))
  (t
   (f2cl-lib:fdo (j ibeg (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf xmax
               (max xmax
                    (abs
                     (f2cl-lib:fref vl-%data% (j ieig)
                                    ((1 ldvl) (1 *))
                                    vl-%offset%))))
     label190))))
 (cond
  ((> xmax safmin) (setf xscale (/ one xmax))
   (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                 ((> jw
                     (f2cl-lib:int-add nw (f2cl-lib:int-sub 1)))
                  nil)
     (tagbody
       (f2cl-lib:fdo (jr ibeg (f2cl-lib:int-add jr 1))
                     ((> jr n) nil)
         (tagbody
           (setf (f2cl-lib:fref vl-%data%
                                (jr (f2cl-lib:int-add ieig jw))
```

```
                                        ((1 ldvl) (1 *)) vl-%offset%)
                          (* xscale
                             (f2cl-lib:fref vl-%data%
                                            (jr
                                             (f2cl-lib:int-add ieig jw))
                                            ((1 ldvl) (1 *))
                                            vl-%offset%)))
                 label200))
             label210))))
       (setf ieig (f2cl-lib:int-sub (f2cl-lib:int-add ieig nw) 1))
      label220))))
 (cond
  (compr (setf ieig (f2cl-lib:int-add im 1))
   (setf ilcplx f2cl-lib:%false%)
   (f2cl-lib:fdo (je n (f2cl-lib:int-add je (f2cl-lib:int-sub 1)))
                 ((> je 1) nil)
     (tagbody
       (cond (ilcplx (setf ilcplx f2cl-lib:%false%) (go label500)))
       (setf nw 1)
       (cond
        ((> je 1)
         (cond
          ((/=
            (f2cl-lib:fref s
                           (je
                            (f2cl-lib:int-add je (f2cl-lib:int-sub 1)))
                           ((1 lds) (1 *)))
            zero)
           (setf ilcplx f2cl-lib:%true%) (setf nw 2)))))
       (cond (ilall (setf ilcomp f2cl-lib:%true%))
             (ilcplx
              (setf ilcomp
                      (or
                       (f2cl-lib:fref select-%data% (je) ((1 *))
                                      select-%offset%)
                       (f2cl-lib:fref select-%data%
                                      ((f2cl-lib:int-sub je 1)) ((1 *))
                                      select-%offset%))))
             (t
              (setf ilcomp
                      (f2cl-lib:fref select-%data% (je) ((1 *))
                                     select-%offset%))))
       (if (not ilcomp)
           (go label500))
       (cond
        ((not ilcplx)
         (cond
          ((and
            (<= (abs (f2cl-lib:fref s (je je) ((1 lds) (1 *)))) safmin)
            (<= (abs (f2cl-lib:fref p (je je) ((1 ldp) (1 *)))) safmin))
```

```
    (setf ieig (f2cl-lib:int-sub ieig 1))
    (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                  ((> jr n) nil)
      (tagbody
        (setf (f2cl-lib:fref vr-%data% (jr ieig) ((1 ldvr) (1 *))
                             vr-%offset%)
              zero)
       label230))
    (setf (f2cl-lib:fref vr-%data% (ieig ieig) ((1 ldvr) (1 *))
                         vr-%offset%)
          one)
    (go label500)))))
(f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
              ((> jw (f2cl-lib:int-add nw (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                  ((> jr n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-mul
                                (f2cl-lib:int-add jw 2) n)
                               jr))
                             ((1 *)) work-%offset%)
              zero)
       label240))
   label250))
(cond
 ((not ilcplx)
  (setf temp
          (/ one
             (max
              (*
               (abs
                (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                               s-%offset%))
               ascale)
              (*
               (abs
                (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
                               p-%offset%))
               bscale)
              safmin)))
  (setf salfar
          (* temp
             (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                            s-%offset%)
             ascale))
  (setf sbeta
```

```
        (* temp
           (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
                          p-%offset%)
           bscale))
(setf acoef (* sbeta ascale)) (setf bcoefr (* salfar bscale))
(setf bcoefi zero) (setf scale one)
(setf lsa (and (>= (abs sbeta) safmin) (< (abs acoef) small)))
(setf lsb
        (and (>= (abs salfar) safmin) (< (abs bcoefr) small)))
(if lsa
    (setf scale (* (/ small (abs sbeta)) (min anorm big))))
(if lsb
    (setf scale
            (max scale
                 (* (/ small (abs salfar)) (min bnorm big)))))
(cond
 ((or lsa lsb)
  (setf scale
          (min scale
               (/ one
                  (* safmin
                     (max one (abs acoef) (abs bcoefr))))))
  (cond (lsa (setf acoef (* ascale (* scale sbeta))))
        (t (setf acoef (* scale acoef))))
  (cond (lsb (setf bcoefr (* bscale (* scale salfar))))
        (t (setf bcoefr (* scale bcoefr))))))
(setf acoefa (abs acoef)) (setf bcoefa (abs bcoefr))
(setf (f2cl-lib:fref work-%data%
                     ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        je))
                     ((1 *)) work-%offset%)
        one)
(setf xmax one)
(f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
              ((> jr
                  (f2cl-lib:int-add je (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add
                           (f2cl-lib:int-mul 2 n) jr))
                         ((1 *)) work-%offset%)
           (-
            (* bcoefr
               (f2cl-lib:fref p-%data% (jr je) ((1 ldp) (1 *))
                              p-%offset%))
            (* acoef
               (f2cl-lib:fref s-%data% (jr je) ((1 lds) (1 *))
                              s-%offset%))))
   label260)))
```

```
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9)
     (slag2
      (f2cl-lib:array-slice s-%data% single-float
                            ((+ je (f2cl-lib:int-sub 1))
                             (f2cl-lib:int-sub je 1))
                            ((1 lds) (1 *)) s-%offset%)
      lds
      (f2cl-lib:array-slice p-%data% single-float
                            ((+ je (f2cl-lib:int-sub 1))
                             (f2cl-lib:int-sub je 1))
                            ((1 ldp) (1 *)) p-%offset%)
      ldp (* safmin safety) acoef temp bcoefr temp2 bcoefi)
   (declare (ignore var-0 var-1 var-2 var-3 var-4))
   (setf acoef var-5)
   (setf temp var-6)
   (setf bcoefr var-7)
   (setf temp2 var-8)
   (setf bcoefi var-9))
 (cond
  ((= bcoefi zero) (setf info (f2cl-lib:int-sub je 1))
   (go end_label)))
 (setf acoefa (abs acoef))
 (setf bcoefa (+ (abs bcoefr) (abs bcoefi))) (setf scale one)
 (if (and (< (* acoefa ulp) safmin) (>= acoefa safmin))
     (setf scale (/ (/ safmin ulp) acoefa)))
 (if (and (< (* bcoefa ulp) safmin) (>= bcoefa safmin))
     (setf scale (max scale (/ (/ safmin ulp) bcoefa))))
 (if (> (* safmin acoefa) ascale)
     (setf scale (/ ascale (* safmin acoefa))))
 (if (> (* safmin bcoefa) bscale)
     (setf scale (min scale (/ bscale (* safmin bcoefa)))))
 (cond
  ((/= scale one) (setf acoef (* scale acoef))
   (setf acoefa (abs acoef)) (setf bcoefr (* scale bcoefr))
   (setf bcoefi (* scale bcoefi))
   (setf bcoefa (+ (abs bcoefr) (abs bcoefi)))))
 (setf temp
         (* acoef
            (f2cl-lib:fref s-%data% (je (f2cl-lib:int-sub je 1))
                           ((1 lds) (1 *)) s-%offset%)))
 (setf temp2r
         (-
          (* acoef
             (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                            s-%offset%))
          (* bcoefr
             (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
```

```
                            p-%offset%))))
(setf temp2i
        (* (- bcoefi)
           (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
                          p-%offset%)))
(cond
 ((>= (abs temp) (+ (abs temp2r) (abs temp2i)))
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 2 n) je))
                       ((1 *)) work-%offset%)
         one)
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 3 n) je))
                       ((1 *)) work-%offset%)
         zero)
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-sub
                         (f2cl-lib:int-add
                          (f2cl-lib:int-mul 2 n) je)
                         1))
                       ((1 *)) work-%offset%)
         (/ (- temp2r) temp))
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-sub
                         (f2cl-lib:int-add
                          (f2cl-lib:int-mul 3 n) je)
                         1))
                       ((1 *)) work-%offset%)
         (/ (- temp2i) temp)))
 (t
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-sub
                         (f2cl-lib:int-add
                          (f2cl-lib:int-mul 2 n) je)
                         1))
                       ((1 *)) work-%offset%)
         one)
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-sub
                         (f2cl-lib:int-add
                          (f2cl-lib:int-mul 3 n) je)
                         1))
                       ((1 *)) work-%offset%)
         zero)
  (setf temp
         (* acoef
            (f2cl-lib:fref s-%data%
                           ((f2cl-lib:int-sub je 1) je)
```

```
                                ((1 lds) (1 *)) s-%offset%)))
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 2 n) je))
                       ((1 *)) work-%offset%)
        (/
         (-
          (* bcoefr
             (f2cl-lib:fref p-%data%
                            ((f2cl-lib:int-sub je 1)
                             (f2cl-lib:int-sub je 1))
                            ((1 ldp) (1 *)) p-%offset%))
          (* acoef
             (f2cl-lib:fref s-%data%
                            ((f2cl-lib:int-sub je 1)
                             (f2cl-lib:int-sub je 1))
                            ((1 lds) (1 *)) s-%offset%)))
         temp))
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-mul 3 n) je))
                       ((1 *)) work-%offset%)
        (/
         (* bcoefi
            (f2cl-lib:fref p-%data%
                           ((f2cl-lib:int-sub je 1)
                            (f2cl-lib:int-sub je 1))
                           ((1 ldp) (1 *)) p-%offset%))
         temp))))
(setf xmax
      (max
       (+
        (abs
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add
                          (f2cl-lib:int-mul 2 n) je))
                        ((1 *)) work-%offset%))
        (abs
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add
                          (f2cl-lib:int-mul 3 n) je))
                        ((1 *)) work-%offset%)))
       (+
        (abs
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add
                           (f2cl-lib:int-mul 2 n) je)
                          1))
                        ((1 *)) work-%offset%))
```

```
          (abs
           (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add
                             (f2cl-lib:int-mul 3 n) je)
                            1))
                          ((1 *)) work-%offset%)))))
(setf creala
        (* acoef
           (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add
                             (f2cl-lib:int-mul 2 n) je)
                            1))
                          ((1 *)) work-%offset%)))
(setf cimaga
        (* acoef
           (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add
                             (f2cl-lib:int-mul 3 n) je)
                            1))
                          ((1 *)) work-%offset%)))
(setf crealb
        (-
         (* bcoefr
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add
                              (f2cl-lib:int-mul 2 n) je)
                             1))
                           ((1 *)) work-%offset%))
         (* bcoefi
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add
                              (f2cl-lib:int-mul 3 n) je)
                             1))
                           ((1 *)) work-%offset%))))
(setf cimagb
        (+
         (* bcoefi
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add
                              (f2cl-lib:int-mul 2 n) je)
                             1))
                           ((1 *)) work-%offset%))
         (* bcoefr
            (f2cl-lib:fref work-%data%
```

```
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add
                                 (f2cl-lib:int-mul 3 n) je)
                                1))
                              ((1 *)) work-%offset%))))
(setf cre2a
        (* acoef
           (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add
                            (f2cl-lib:int-mul 2 n) je))
                          ((1 *)) work-%offset%)))
(setf cim2a
        (* acoef
           (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add
                            (f2cl-lib:int-mul 3 n) je))
                          ((1 *)) work-%offset%)))
(setf cre2b
        (-
         (* bcoefr
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add
                             (f2cl-lib:int-mul 2 n) je))
                           ((1 *)) work-%offset%))
         (* bcoefi
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add
                             (f2cl-lib:int-mul 3 n) je))
                           ((1 *)) work-%offset%))))
(setf cim2b
        (+
         (* bcoefi
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add
                             (f2cl-lib:int-mul 2 n) je))
                           ((1 *)) work-%offset%))
         (* bcoefr
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add
                             (f2cl-lib:int-mul 3 n) je))
                           ((1 *)) work-%offset%))))
(f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
              ((> jr
                  (f2cl-lib:int-add je (f2cl-lib:int-sub 2)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add
                           (f2cl-lib:int-mul 2 n) jr))
                         ((1 *)) work-%offset%)
```

```
                (+
                 (-
                  (+
                   (* (- creala)
                      (f2cl-lib:fref s-%data%
                                     (jr (f2cl-lib:int-sub je 1))
                                     ((1 lds) (1 *)) s-%offset%))
                   (* crealb
                      (f2cl-lib:fref p-%data%
                                     (jr (f2cl-lib:int-sub je 1))
                                     ((1 ldp) (1 *)) p-%offset%)))
                  (* cre2a
                     (f2cl-lib:fref s-%data% (jr je)
                                    ((1 lds) (1 *)) s-%offset%)))
                 (* cre2b
                    (f2cl-lib:fref p-%data% (jr je) ((1 ldp) (1 *))
                                   p-%offset%))))
      (setf (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add
                             (f2cl-lib:int-mul 3 n) jr))
                           ((1 *)) work-%offset%)
                (+
                 (-
                  (+
                   (* (- cimaga)
                      (f2cl-lib:fref s-%data%
                                     (jr (f2cl-lib:int-sub je 1))
                                     ((1 lds) (1 *)) s-%offset%))
                   (* cimagb
                      (f2cl-lib:fref p-%data%
                                     (jr (f2cl-lib:int-sub je 1))
                                     ((1 ldp) (1 *)) p-%offset%)))
                  (* cim2a
                     (f2cl-lib:fref s-%data% (jr je)
                                    ((1 lds) (1 *)) s-%offset%)))
                 (* cim2b
                    (f2cl-lib:fref p-%data% (jr je) ((1 ldp) (1 *))
                                   p-%offset%))))
     label270))))
(setf dmin
        (max (* ulp acoefa anorm) (* ulp bcoefa bnorm) safmin))
(setf il2by2 f2cl-lib:%false%)
(f2cl-lib:fdo (j (f2cl-lib:int-add je (f2cl-lib:int-sub nw))
               (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j 1) nil)
  (tagbody
    (cond
     ((and (not il2by2) (> j 1))
      (cond
       ((/=
```

```
     (f2cl-lib:fref s
                    (j
                     (f2cl-lib:int-add j
                                       (f2cl-lib:int-sub 1)))
                    ((1 lds) (1 *)))
     zero)
    (setf il2by2 f2cl-lib:%true%) (go label370)))))
(setf (f2cl-lib:fref bdiag (1) ((1 2)))
        (f2cl-lib:fref p-%data% (j j) ((1 ldp) (1 *))
                       p-%offset%))
(cond
 (il2by2 (setf na 2)
  (setf (f2cl-lib:fref bdiag (2) ((1 2)))
          (f2cl-lib:fref p-%data%
                         ((f2cl-lib:int-add j 1)
                          (f2cl-lib:int-add j 1))
                         ((1 ldp) (1 *)) p-%offset%)))
 (t (setf na 1)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
    (slaln2 f2cl-lib:%false% na nw dmin acoef
     (f2cl-lib:array-slice s-%data% single-float (j j)
                           ((1 lds) (1 *)) s-%offset%)
     lds (f2cl-lib:fref bdiag (1) ((1 2)))
     (f2cl-lib:fref bdiag (2) ((1 2)))
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul 2 n) j))
                           ((1 *)) work-%offset%)
     n bcoefr bcoefi sum 2 scale temp iinfo)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
    var-8 var-9 var-10 var-11 var-12 var-13 var-14))
  (setf scale var-15)
  (setf temp var-16)
  (setf iinfo var-17))
(cond
 ((< scale one)
  (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                ((> jw
                    (f2cl-lib:int-add nw
                                      (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr je) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add
```

```
                                    (f2cl-lib:int-mul
                                     (f2cl-lib:int-add jw 2) n)
                                    jr))
                                  ((1 *)) work-%offset%)
                     (* scale
                        (f2cl-lib:fref work-%data%
                                       ((f2cl-lib:int-add
                                         (f2cl-lib:int-mul
                                          (f2cl-lib:int-add jw
                                                            2)
                                          n)
                                         jr))
                                       ((1 *)) work-%offset%)))
         label280))
     label290))))
(setf xmax (max (* scale xmax) temp))
(f2cl-lib:fdo (jw 1 (f2cl-lib:int-add jw 1))
              ((> jw nw) nil)
  (tagbody
    (f2cl-lib:fdo (ja 1 (f2cl-lib:int-add ja 1))
                  ((> ja na) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-sub
                               (f2cl-lib:int-add
                                (f2cl-lib:int-mul
                                 (f2cl-lib:int-add jw 1) n)
                                j ja)
                               1))
                             ((1 *)) work-%offset%)
              (f2cl-lib:fref sum (ja jw) ((1 2) (1 2))))
       label300))
   label310))
(cond
 ((> j 1) (setf xscale (/ one (max one xmax)))
  (setf temp
          (+
           (* acoefa
              (f2cl-lib:fref work-%data% (j) ((1 *))
                             work-%offset%))
           (* bcoefa
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add n j)) ((1 *))
                             work-%offset%))))
  (if il2by2
      (setf temp
              (max temp
                   (+
                    (* acoefa
                       (f2cl-lib:fref work-%data%
```

```
                                        ((f2cl-lib:int-add j
                                                           1))
                                        ((1 *)) work-%offset%))
                     (* bcoefa
                        (f2cl-lib:fref work-%data%
                                        ((f2cl-lib:int-add n j
                                                           1))
                                        ((1 *))
                                        work-%offset%))))))
(setf temp (max temp acoefa bcoefa))
(cond
 ((> temp (* bignum xscale))
  (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                ((> jw
                    (f2cl-lib:int-add nw
                                      (f2cl-lib:int-sub
                                       1)))
                 nil)
    (tagbody
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr je) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-add jw 2)
                                  n)
                                 jr))
                               ((1 *)) work-%offset%)
                     (* xscale
                        (f2cl-lib:fref work-%data%
                                        ((f2cl-lib:int-add
                                          (f2cl-lib:int-mul
                                           (f2cl-lib:int-add jw
                                                             2)
                                           n)
                                          jr))
                                        ((1 *))
                                        work-%offset%)))
         label320))
     label330))
  (setf xmax (* xmax xscale))))
(f2cl-lib:fdo (ja 1 (f2cl-lib:int-add ja 1))
              ((> ja na) nil)
  (tagbody
    (cond
     (ilcplx
      (setf creala
              (* acoef
                 (f2cl-lib:fref work-%data%
```

```
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add
                                 (f2cl-lib:int-mul 2 n) j
                                 ja)
                                1))
                              ((1 *)) work-%offset%)))
(setf cimaga
        (* acoef
           (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add
                                 (f2cl-lib:int-mul 3 n) j
                                 ja)
                                1))
                              ((1 *)) work-%offset%)))
(setf crealb
        (-
         (* bcoefr
            (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add
                                 (f2cl-lib:int-mul 2 n)
                                 j ja)
                                1))
                              ((1 *)) work-%offset%))
         (* bcoefi
            (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add
                                 (f2cl-lib:int-mul 3 n)
                                 j ja)
                                1))
                              ((1 *)) work-%offset%))))
(setf cimagb
        (+
         (* bcoefi
            (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add
                                 (f2cl-lib:int-mul 2 n)
                                 j ja)
                                1))
                              ((1 *)) work-%offset%))
         (* bcoefr
            (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add
                                 (f2cl-lib:int-mul 3 n)
                                 j ja)
                                1))
```

```
                                ((1 *)) work-%offset%))))
(f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
              ((> jr
                  (f2cl-lib:int-add j
                                    (f2cl-lib:int-sub
                                     1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add
                           (f2cl-lib:int-mul 2 n)
                           jr))
                         ((1 *)) work-%offset%)
          (+
           (-
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add
                             (f2cl-lib:int-mul 2 n)
                             jr))
                           ((1 *)) work-%offset%)
            (* creala
               (f2cl-lib:fref s-%data%
                              (jr
                               (f2cl-lib:int-sub
                                (f2cl-lib:int-add j
                                                  ja)
                                1))
                              ((1 lds) (1 *))
                              s-%offset%)))
           (* crealb
              (f2cl-lib:fref p-%data%
                             (jr
                              (f2cl-lib:int-sub
                               (f2cl-lib:int-add j
                                                 ja)
                               1))
                             ((1 ldp) (1 *))
                             p-%offset%))))
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add
                           (f2cl-lib:int-mul 3 n)
                           jr))
                         ((1 *)) work-%offset%)
          (+
           (-
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add
                             (f2cl-lib:int-mul 3 n)
                             jr))
                           ((1 *)) work-%offset%)
```

```
                    (* cimaga
                       (f2cl-lib:fref s-%data%
                                      (jr
                                       (f2cl-lib:int-sub
                                        (f2cl-lib:int-add j
                                                          ja)
                                        1))
                                      ((1 lds) (1 *))
                                      s-%offset%)))
                   (* cimagb
                      (f2cl-lib:fref p-%data%
                                     (jr
                                      (f2cl-lib:int-sub
                                       (f2cl-lib:int-add j
                                                         ja)
                                       1))
                                     ((1 ldp) (1 *))
                                     p-%offset%))))
    label340)))
(t
 (setf creala
         (* acoef
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add
                              (f2cl-lib:int-mul 2 n) j
                              ja)
                             1))
                           ((1 *)) work-%offset%)))
 (setf crealb
         (* bcoefr
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add
                              (f2cl-lib:int-mul 2 n) j
                              ja)
                             1))
                           ((1 *)) work-%offset%)))
 (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
               ((> jr
                   (f2cl-lib:int-add j
                                     (f2cl-lib:int-sub
                                      1)))
                nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add
                            (f2cl-lib:int-mul 2 n)
                            jr))
                          ((1 *)) work-%offset%)
```

```
                                    (+
                                     (-
                                      (f2cl-lib:fref work-%data%
                                                     ((f2cl-lib:int-add
                                                       (f2cl-lib:int-mul 2 n)
                                                       jr))
                                                     ((1 *)) work-%offset%)
                                      (* creala
                                         (f2cl-lib:fref s-%data%
                                                        (jr
                                                         (f2cl-lib:int-sub
                                                          (f2cl-lib:int-add j
                                                                            ja)
                                                          1))
                                                        ((1 lds) (1 *))
                                                        s-%offset%)))
                                     (* crealb
                                        (f2cl-lib:fref p-%data%
                                                       (jr
                                                        (f2cl-lib:int-sub
                                                         (f2cl-lib:int-add j
                                                                           ja)
                                                         1))
                                                       ((1 ldp) (1 *))
                                                       p-%offset%))))
                              label350))))
                        label360))))
                   (setf il2by2 f2cl-lib:%false%)
                  label370))
                (setf ieig (f2cl-lib:int-sub ieig nw))
                (cond
                 (ilback
                  (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                                ((> jw
                                    (f2cl-lib:int-add nw (f2cl-lib:int-sub 1)))
                                 nil)
                    (tagbody
                      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                                    ((> jr n) nil)
                        (tagbody
                          (setf (f2cl-lib:fref work-%data%
                                               ((f2cl-lib:int-add
                                                 (f2cl-lib:int-mul
                                                  (f2cl-lib:int-add jw 4) n)
                                                 jr))
                                               ((1 *)) work-%offset%)
                                (*
                                 (f2cl-lib:fref work-%data%
                                                ((f2cl-lib:int-add
                                                  (f2cl-lib:int-mul
```

```
                                        (f2cl-lib:int-add jw 2) n)
                                       1))
                                    ((1 *)) work-%offset%)
                        (f2cl-lib:fref vr-%data% (jr 1)
                                    ((1 ldvr) (1 *)) vr-%offset%)))
          label380))
       (f2cl-lib:fdo (jc 2 (f2cl-lib:int-add jc 1))
                     ((> jc je) nil)
         (tagbody
           (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                         ((> jr n) nil)
             (tagbody
               (setf (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add
                                      (f2cl-lib:int-mul
                                       (f2cl-lib:int-add jw 4) n)
                                      jr))
                                    ((1 *)) work-%offset%)
                     (+
                      (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-add
                                       (f2cl-lib:int-mul
                                        (f2cl-lib:int-add jw 4)
                                        n)
                                       jr))
                                     ((1 *)) work-%offset%)
                      (*
                       (f2cl-lib:fref work-%data%
                                      ((f2cl-lib:int-add
                                        (f2cl-lib:int-mul
                                         (f2cl-lib:int-add jw 2)
                                         n)
                                        jc))
                                      ((1 *)) work-%offset%)
                       (f2cl-lib:fref vr-%data% (jr jc)
                                      ((1 ldvr) (1 *))
                                      vr-%offset%))))
             label390))
         label400))
     label410))
  (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                ((> jw
                    (f2cl-lib:int-add nw (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf (f2cl-lib:fref vr-%data%
                               (jr (f2cl-lib:int-add ieig jw))
```

```
                                    ((1 ldvr) (1 *)) vr-%offset%)
                        (f2cl-lib:fref work-%data%
                                       ((f2cl-lib:int-add
                                         (f2cl-lib:int-mul
                                          (f2cl-lib:int-add jw 4) n)
                                         jr))
                                       ((1 *)) work-%offset%))
                label420))
            label430))
          (setf iend n))
         (t
          (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                        ((> jw
                            (f2cl-lib:int-add nw (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                            ((> jr n) nil)
                (tagbody
                  (setf (f2cl-lib:fref vr-%data%
                                       (jr (f2cl-lib:int-add ieig jw))
                                       ((1 ldvr) (1 *)) vr-%offset%)
                        (f2cl-lib:fref work-%data%
                                       ((f2cl-lib:int-add
                                         (f2cl-lib:int-mul
                                          (f2cl-lib:int-add jw 2) n)
                                         jr))
                                       ((1 *)) work-%offset%))
                label440))
            label450))
          (setf iend je)))
        (setf xmax zero)
        (cond
         (ilcplx
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j iend) nil)
            (tagbody
              (setf xmax
                      (max xmax
                           (+
                            (abs
                             (f2cl-lib:fref vr-%data% (j ieig)
                                            ((1 ldvr) (1 *))
                                            vr-%offset%))
                            (abs
                             (f2cl-lib:fref vr-%data%
                                            (j (f2cl-lib:int-add ieig 1))
                                            ((1 ldvr) (1 *))
                                            vr-%offset%)))))
            label460)))
```

```
               (t
                (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                              ((> j iend) nil)
                  (tagbody
                    (setf xmax
                            (max xmax
                                 (abs
                                  (f2cl-lib:fref vr-%data% (j ieig)
                                                 ((1 ldvr) (1 *))
                                                 vr-%offset%))))
                     label470))))
              (cond
               ((> xmax safmin) (setf xscale (/ one xmax))
                (f2cl-lib:fdo (jw 0 (f2cl-lib:int-add jw 1))
                              ((> jw
                                  (f2cl-lib:int-add nw (f2cl-lib:int-sub 1)))
                               nil)
                  (tagbody
                    (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                                  ((> jr iend) nil)
                      (tagbody
                        (setf (f2cl-lib:fref vr-%data%
                                             (jr (f2cl-lib:int-add ieig jw))
                                             ((1 ldvr) (1 *)) vr-%offset%)
                              (* xscale
                                 (f2cl-lib:fref vr-%data%
                                                (jr
                                                 (f2cl-lib:int-add ieig jw))
                                                ((1 ldvr) (1 *))
                                                vr-%offset%)))
                        label480))
                     label490))))
            label500))))
        (go end_label)
       end_label
        (return
         (values side howmny nil n nil nil nil nil nil ldvl nil nil nil m nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::stgevc
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (array fortran-to-lisp::logical
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
```

```
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::side
  fortran-to-lisp::howmny nil
  fortran-to-lisp::n nil nil nil
  nil nil fortran-to-lisp::ldvl nil
  nil nil fortran-to-lisp::m nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::slacpy
  fortran-to-lisp::slaln2
  fortran-to-lisp::slag2
  fortran-to-lisp::slabad))))
```

———-

33.40 stgexc

33.40.1 Axiom unit tests

— stgexc.input —

```
)set break resume
)sys rm -f stgexc.output
)spool stgexc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

33.40.2 Axiom help page

— stgexc.help —

```
====================================================================
stgexc examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine stgexc (WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z, LDZ,
           IFST, ILST, WORK, LWORK, INFO)
           STGEXC

Function/Subroutine Documentation
   subroutine stgexc (logical WANTQ, logical WANTZ, integer N, real,
       dimension( lda, * ) A, integer LDA, real, dimension( ldb, * ) B,
       integer LDB, real, dimension( ldq, * ) Q, integer LDQ, real, dimension(
       ldz, * ) Z, integer LDZ, integer IFST, integer ILST, real, dimension( *
       ) WORK, integer LWORK, integer INFO)
       STGEXC

Purpose:
   STGEXC reorders the generalized real Schur decomposition of a real
   matrix pair (A,B) using an orthogonal equivalence transformation

                   (A, B) = Q * (A, B) * Z**T,

   so that the diagonal block of (A, B) with row index IFST is moved
   to row ILST.

   (A, B) must be in generalized real Schur canonical form (as returned
   by SGGES), i.e. A is block upper triangular with 1-by-1 and 2-by-2
   diagonal blocks. B is upper triangular.

   Optionally, the matrices Q and Z of generalized Schur vectors are
   updated.

            Q(in) * A(in) * Z(in)**T = Q(out) * A(out) * Z(out)**T
            Q(in) * B(in) * Z(in)**T = Q(out) * B(out) * Z(out)**T

Parameters:

 WANTQ
   WANTQ is LOGICAL
```

```
  .TRUE. : update the left transformation matrix Q;
  .FALSE.: do not update Q.

WANTZ
  WANTZ is LOGICAL
  .TRUE. : update the right transformation matrix Z;
  .FALSE.: do not update Z.

N
  N is INTEGER
  The order of the matrices A and B. N >= 0.

A
  A is REAL array, dimension (LDA,N)
  On entry, the matrix A in generalized real Schur canonical form.
  On exit, the updated matrix A, again in generalized
  real Schur canonical form.

LDA
  LDA is INTEGER
  The leading dimension of the array A. LDA >= max(1,N).

B
  B is REAL array, dimension (LDB,N)
  On entry, the matrix B in generalized real Schur canonical form (A,B).
  On exit, the updated matrix B, again in generalized
  real Schur canonical form (A,B).

LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= max(1,N).

Q
  Q is REAL array, dimension (LDZ,N)
  On entry, if WANTQ = .TRUE., the orthogonal matrix Q.
  On exit, the updated matrix Q.
  If WANTQ = .FALSE., Q is not referenced.

LDQ
  LDQ is INTEGER
  The leading dimension of the array Q. LDQ >= 1.
  If WANTQ = .TRUE., LDQ >= N.

Z
  Z is REAL array, dimension (LDZ,N)
  On entry, if WANTZ = .TRUE., the orthogonal matrix Z.
  On exit, the updated matrix Z.
  If WANTZ = .FALSE., Z is not referenced.

LDZ
```

```
  LDZ is INTEGER
  The leading dimension of the array Z. LDZ >= 1.
  If WANTZ = .TRUE., LDZ >= N.

IFST
  IFST is INTEGER

ILST
  ILST is INTEGER
  Specify the reordering of the diagonal blocks of (A, B).
  The block with row index IFST is moved to row ILST, by a
  sequence of swapping between adjacent blocks.
  On exit, if IFST pointed on entry to the second row of
  a 2-by-2 block, it is changed to point to the first row;
  ILST always points to the first row of the block in its
  final position (which may differ from its input value by
  +1 or -1). 1 <= IFST, ILST <= N.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  LWORK >= 1 when N <= 1, otherwise LWORK >= 4*N + 16.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  =0:  successful exit.
  <0:  if INFO = -i, the i-th argument had an illegal value.
  =1:  The transformed matrix pair (A, B) would be too far
       from generalized Schur form; the problem is ill-
       conditioned. (A, B) may have been partially reordered,
       and ILST points to the first row of the current
       position of the block being moved.
```

———-

33.40.3 fortran code

— stgexc.f —

```
*  =====================================================================
      SUBROUTINE STGEXC( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                   LDZ, IFST, ILST, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      LOGICAL            WANTQ, WANTZ
      INTEGER            IFST, ILST, INFO, LDA, LDB, LDQ, LDZ, LWORK, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), B( LDB, * ), Q( LDQ, * ),
     $                   WORK( * ), Z( LDZ, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            HERE, LWMIN, NBF, NBL, NBNEXT
*     ..
*     .. External Subroutines ..
      EXTERNAL           STGEX2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Decode and test input arguments.
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDQ.LT.1 .OR. WANTQ .AND. ( LDQ.LT.MAX( 1, N ) ) ) THEN
         INFO = -9
      ELSE IF( LDZ.LT.1 .OR. WANTZ .AND. ( LDZ.LT.MAX( 1, N ) ) ) THEN
         INFO = -11
```

```
      ELSE IF( IFST.LT.1 .OR. IFST.GT.N ) THEN
         INFO = -12
      ELSE IF( ILST.LT.1 .OR. ILST.GT.N ) THEN
         INFO = -13
      END IF
*
      IF( INFO.EQ.0 ) THEN
         IF( N.LE.1 ) THEN
            LWMIN = 1
         ELSE
            LWMIN = 4*N + 16
         END IF
         WORK(1) = LWMIN
*
         IF (LWORK.LT.LWMIN .AND. .NOT.LQUERY) THEN
            INFO = -15
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'STGEXC', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.LE.1 )
     $   RETURN
*
*     Determine the first row of the specified block and find out
*     if it is 1-by-1 or 2-by-2.
*
      IF( IFST.GT.1 ) THEN
         IF( A( IFST, IFST-1 ).NE.ZERO )
     $      IFST = IFST - 1
      END IF
      NBF = 1
      IF( IFST.LT.N ) THEN
         IF( A( IFST+1, IFST ).NE.ZERO )
     $      NBF = 2
      END IF
*
*     Determine the first row of the final block
*     and find out if it is 1-by-1 or 2-by-2.
*
      IF( ILST.GT.1 ) THEN
         IF( A( ILST, ILST-1 ).NE.ZERO )
     $      ILST = ILST - 1
```

```
      END IF
      NBL = 1
      IF( ILST.LT.N ) THEN
         IF( A( ILST+1, ILST ).NE.ZERO )
     $      NBL = 2
      END IF
      IF( IFST.EQ.ILST )
     $   RETURN
*
      IF( IFST.LT.ILST ) THEN
*
*        Update ILST.
*
         IF( NBF.EQ.2 .AND. NBL.EQ.1 )
     $      ILST = ILST - 1
         IF( NBF.EQ.1 .AND. NBL.EQ.2 )
     $      ILST = ILST + 1
*
         HERE = IFST
*
   10    CONTINUE
*
*        Swap with next one below.
*
         IF( NBF.EQ.1 .OR. NBF.EQ.2 ) THEN
*
*           Current block either 1-by-1 or 2-by-2.
*
            NBNEXT = 1
            IF( HERE+NBF+1.LE.N ) THEN
               IF( A( HERE+NBF+1, HERE+NBF ).NE.ZERO )
     $            NBNEXT = 2
            END IF
            CALL STGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                   LDZ, HERE, NBF, NBNEXT, WORK, LWORK, INFO )
            IF( INFO.NE.0 ) THEN
               ILST = HERE
               RETURN
            END IF
            HERE = HERE + NBNEXT
*
*           Test if 2-by-2 block breaks into two 1-by-1 blocks.
*
            IF( NBF.EQ.2 ) THEN
               IF( A( HERE+1, HERE ).EQ.ZERO )
     $            NBF = 3
            END IF
*
         ELSE
*
```

```
*          Current block consists of two 1-by-1 blocks, each of which
*          must be swapped individually.
*
           NBNEXT = 1
           IF( HERE+3.LE.N ) THEN
              IF( A( HERE+3, HERE+2 ).NE.ZERO )
     $           NBNEXT = 2
           END IF
           CALL STGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                  LDZ, HERE+1, 1, NBNEXT, WORK, LWORK, INFO )
           IF( INFO.NE.0 ) THEN
              ILST = HERE
              RETURN
           END IF
           IF( NBNEXT.EQ.1 ) THEN
*
*             Swap two 1-by-1 blocks.
*
              CALL STGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                     LDZ, HERE, 1, 1, WORK, LWORK, INFO )
              IF( INFO.NE.0 ) THEN
                 ILST = HERE
                 RETURN
              END IF
              HERE = HERE + 1
*
           ELSE
*
*             Recompute NBNEXT in case of 2-by-2 split.
*
              IF( A( HERE+2, HERE+1 ).EQ.ZERO )
     $           NBNEXT = 1
              IF( NBNEXT.EQ.2 ) THEN
*
*                2-by-2 block did not split.
*
                 CALL STGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ,
     $                        Z, LDZ, HERE, 1, NBNEXT, WORK, LWORK,
     $                        INFO )
                 IF( INFO.NE.0 ) THEN
                    ILST = HERE
                    RETURN
                 END IF
                 HERE = HERE + 2
              ELSE
*
*                2-by-2 block did split.
*
                 CALL STGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ,
     $                        Z, LDZ, HERE, 1, 1, WORK, LWORK, INFO )
```

```
                 IF( INFO.NE.0 ) THEN
                    ILST = HERE
                    RETURN
                 END IF
                 HERE = HERE + 1
                 CALL STGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ,
     $                        Z, LDZ, HERE, 1, 1, WORK, LWORK, INFO )
                 IF( INFO.NE.0 ) THEN
                    ILST = HERE
                    RETURN
                 END IF
                 HERE = HERE + 1
              END IF
*
           END IF
        END IF
        IF( HERE.LT.ILST )
     $     GO TO 10
     ELSE
        HERE = IFST
*
   20   CONTINUE
*
*       Swap with next one below.
*
        IF( NBF.EQ.1 .OR. NBF.EQ.2 ) THEN
*
*          Current block either 1-by-1 or 2-by-2.
*
           NBNEXT = 1
           IF( HERE.GE.3 ) THEN
              IF( A( HERE-1, HERE-2 ).NE.ZERO )
     $           NBNEXT = 2
           END IF
           CALL STGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                  LDZ, HERE-NBNEXT, NBNEXT, NBF, WORK, LWORK,
     $                  INFO )
           IF( INFO.NE.0 ) THEN
              ILST = HERE
              RETURN
           END IF
           HERE = HERE - NBNEXT
*
*          Test if 2-by-2 block breaks into two 1-by-1 blocks.
*
           IF( NBF.EQ.2 ) THEN
              IF( A( HERE+1, HERE ).EQ.ZERO )
     $           NBF = 3
           END IF
*
```

```
         ELSE
*
*           Current block consists of two 1-by-1 blocks, each of which
*           must be swapped individually.
*
            NBNEXT = 1
            IF( HERE.GE.3 ) THEN
               IF( A( HERE-1, HERE-2 ).NE.ZERO )
     $            NBNEXT = 2
            END IF
            CALL STGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                   LDZ, HERE-NBNEXT, NBNEXT, 1, WORK, LWORK,
     $                   INFO )
            IF( INFO.NE.0 ) THEN
               ILST = HERE
               RETURN
            END IF
            IF( NBNEXT.EQ.1 ) THEN
*
*              Swap two 1-by-1 blocks.
*
               CALL STGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                      LDZ, HERE, NBNEXT, 1, WORK, LWORK, INFO )
               IF( INFO.NE.0 ) THEN
                  ILST = HERE
                  RETURN
               END IF
               HERE = HERE - 1
            ELSE
*
*             Recompute NBNEXT in case of 2-by-2 split.
*
               IF( A( HERE, HERE-1 ).EQ.ZERO )
     $            NBNEXT = 1
               IF( NBNEXT.EQ.2 ) THEN
*
*                 2-by-2 block did not split.
*
                  CALL STGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ,
     $                         Z, LDZ, HERE-1, 2, 1, WORK, LWORK, INFO )
                  IF( INFO.NE.0 ) THEN
                     ILST = HERE
                     RETURN
                  END IF
                  HERE = HERE - 2
               ELSE
*
*                 2-by-2 block did split.
*
                  CALL STGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ,
```

```
     $                        Z, LDZ, HERE, 1, 1, WORK, LWORK, INFO )
                 IF( INFO.NE.0 ) THEN
                    ILST = HERE
                    RETURN
                 END IF
                 HERE = HERE - 1
                 CALL STGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ,
     $                        Z, LDZ, HERE, 1, 1, WORK, LWORK, INFO )
                 IF( INFO.NE.0 ) THEN
                    ILST = HERE
                    RETURN
                 END IF
                 HERE = HERE - 1
              END IF
           END IF
        END IF
        IF( HERE.GT.ILST )
     $      GO TO 20
      END IF
      ILST = HERE
      WORK( 1 ) = LWMIN
      RETURN
*
*     End of STGEXC
*
      END
```

———-

33.40.4 lisp code

— **stgexc.lisp** —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun stgexc
         (wantq wantz n a lda b ldb$ q ldq z ldz ifst ilst work lwork info)
    (declare (type (array single-float (*)) work z q b a)
             (type (f2cl-lib:integer4) info lwork ilst ifst ldz ldq ldb$ lda n)
             (type f2cl-lib:logical wantz wantq))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%)
         (q single-float q-%data% q-%offset%)
         (z single-float z-%data% z-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((here 0) (lwmin 0) (nbf 0) (nbl 0) (nbnext 0) (lquery nil))
```

```
(declare (type (f2cl-lib:integer4) here lwmin nbf nbl nbnext)
         (type f2cl-lib:logical lquery))
(setf info 0)
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((< n 0) (setf info -3))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -5))
      ((< ldb$
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -7))
      ((or (< ldq 1)
           (and wantq
                (< ldq
                   (max (the f2cl-lib:integer4 1)
                        (the f2cl-lib:integer4 n)))))
       (setf info -9))
      ((or (< ldz 1)
           (and wantz
                (< ldz
                   (max (the f2cl-lib:integer4 1)
                        (the f2cl-lib:integer4 n)))))
       (setf info -11))
      ((or (< ifst 1) (> ifst n)) (setf info -12))
      ((or (< ilst 1) (> ilst n)) (setf info -13)))
(cond
 ((= info 0)
  (cond ((<= n 1) (setf lwmin 1))
        (t (setf lwmin (f2cl-lib:int-add (f2cl-lib:int-mul 4 n) 16))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwmin) 'single-float))
  (cond ((and (< lwork lwmin) (not lquery)) (setf info -15)))))
(cond
 ((/= info 0) (xerbla "STGEXC" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (<= n 1)
    (go end_label))
(cond
 ((> ifst 1)
  (if (/=
       (f2cl-lib:fref a-%data% (ifst (f2cl-lib:int-sub ifst 1))
                      ((1 lda) (1 *)) a-%offset%)
       zero)
      (setf ifst (f2cl-lib:int-sub ifst 1)))))
(setf nbf 1)
(cond
 ((< ifst n)
  (if (/=
       (f2cl-lib:fref a-%data% ((f2cl-lib:int-add ifst 1) ifst)
                      ((1 lda) (1 *)) a-%offset%)
```

```
       zero)
      (setf nbf 2))))
(cond
 ((> ilst 1)
  (if (/=
       (f2cl-lib:fref a-%data% (ilst (f2cl-lib:int-sub ilst 1))
                      ((1 lda) (1 *)) a-%offset%)
       zero)
      (setf ilst (f2cl-lib:int-sub ilst 1)))))
(setf nbl 1)
(cond
 ((< ilst n)
  (if (/=
       (f2cl-lib:fref a-%data% ((f2cl-lib:int-add ilst 1) ilst)
                      ((1 lda) (1 *)) a-%offset%)
       zero)
      (setf nbl 2))))
(if (= ifst ilst)
    (go end_label))
(cond
 ((< ifst ilst)
  (tagbody
    (if (and (= nbf 2) (= nbl 1))
        (setf ilst (f2cl-lib:int-sub ilst 1)))
    (if (and (= nbf 1) (= nbl 2))
        (setf ilst (f2cl-lib:int-add ilst 1)))
    (setf here ifst)
   label10
    (cond
     ((or (= nbf 1) (= nbf 2)) (setf nbnext 1)
      (cond
       ((<= (f2cl-lib:int-add here nbf 1) n)
        (if (/=
             (f2cl-lib:fref a-%data%
                            ((f2cl-lib:int-add here nbf 1)
                             (f2cl-lib:int-add here nbf))
                            ((1 lda) (1 *)) a-%offset%)
             zero)
            (setf nbnext 2))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13 var-14 var-15 var-16)
          (stgex2 wantq wantz n a lda b ldb$ q ldq z ldz here nbf
           nbnext work lwork info)
        (declare
         (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-14
          var-15))
        (setf n var-2)
        (setf lda var-4)
        (setf ldb$ var-6)
```

```
    (setf ldq var-8)
    (setf ldz var-10)
    (setf nbf var-12)
    (setf nbnext var-13)
    (setf info var-16))
  (cond ((/= info 0) (setf ilst here) (go end_label)))
  (setf here (f2cl-lib:int-add here nbnext))
  (cond
   ((= nbf 2)
    (if (=
         (f2cl-lib:fref a-%data% ((f2cl-lib:int-add here 1) here)
                        ((1 lda) (1 *)) a-%offset%)
         zero)
        (setf nbf 3)))))
 (t (setf nbnext 1)
  (cond
   ((<= (f2cl-lib:int-add here 3) n)
    (if (/=
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-add here 3)
                         (f2cl-lib:int-add here 2))
                        ((1 lda) (1 *)) a-%offset%)
         zero)
        (setf nbnext 2))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16)
      (stgex2 wantq wantz n a lda b ldb$ q ldq z ldz
       (f2cl-lib:int-add here 1) 1 nbnext work lwork info)
    (declare
     (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-12
      var-14 var-15))
    (setf n var-2)
    (setf lda var-4)
    (setf ldb$ var-6)
    (setf ldq var-8)
    (setf ldz var-10)
    (setf nbnext var-13)
    (setf info var-16))
  (cond ((/= info 0) (setf ilst here) (go end_label)))
  (cond
   ((= nbnext 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16)
        (stgex2 wantq wantz n a lda b ldb$ q ldq z ldz here 1 1
         work lwork info)
      (declare
       (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-12
        var-13 var-14 var-15))
```

```
    (setf n var-2)
    (setf lda var-4)
    (setf ldb$ var-6)
    (setf ldq var-8)
    (setf ldz var-10)
    (setf info var-16))
  (cond ((/= info 0) (setf ilst here) (go end_label)))
  (setf here (f2cl-lib:int-add here 1)))
 (t
  (if (=
       (f2cl-lib:fref a-%data%
                      ((f2cl-lib:int-add here 2)
                       (f2cl-lib:int-add here 1))
                      ((1 lda) (1 *)) a-%offset%)
       zero)
      (setf nbnext 1))
  (cond
   ((= nbnext 2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16)
        (stgex2 wantq wantz n a lda b ldb$ q ldq z ldz here 1
         nbnext work lwork info)
      (declare
       (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-12
        var-14 var-15))
      (setf n var-2)
      (setf lda var-4)
      (setf ldb$ var-6)
      (setf ldq var-8)
      (setf ldz var-10)
      (setf nbnext var-13)
      (setf info var-16))
    (cond ((/= info 0) (setf ilst here) (go end_label)))
    (setf here (f2cl-lib:int-add here 2)))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16)
        (stgex2 wantq wantz n a lda b ldb$ q ldq z ldz here 1 1
         work lwork info)
      (declare
       (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-12
        var-13 var-14 var-15))
      (setf n var-2)
      (setf lda var-4)
      (setf ldb$ var-6)
      (setf ldq var-8)
      (setf ldz var-10)
      (setf info var-16))
```

```
            (cond ((/= info 0) (setf ilst here) (go end_label)))
            (setf here (f2cl-lib:int-add here 1))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16)
                (stgex2 wantq wantz n a lda b ldb$ q ldq z ldz here 1 1
                 work lwork info)
              (declare
               (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-12
                var-13 var-14 var-15))
              (setf n var-2)
              (setf lda var-4)
              (setf ldb$ var-6)
              (setf ldq var-8)
              (setf ldz var-10)
              (setf info var-16))
            (cond ((/= info 0) (setf ilst here) (go end_label)))
            (setf here (f2cl-lib:int-add here 1))))))))
    (if (< here ilst)
        (go label10))))
(t
 (tagbody
   (setf here ifst)
  label20
   (cond
    ((or (= nbf 1) (= nbf 2)) (setf nbnext 1)
     (cond
      ((>= here 3)
       (if (/=
            (f2cl-lib:fref a-%data%
                           ((f2cl-lib:int-sub here 1)
                            (f2cl-lib:int-sub here 2))
                           ((1 lda) (1 *)) a-%offset%)
            zero)
           (setf nbnext 2))))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12 var-13 var-14 var-15 var-16)
         (stgex2 wantq wantz n a lda b ldb$ q ldq z ldz
          (f2cl-lib:int-sub here nbnext) nbnext nbf work lwork info)
       (declare
        (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-14
         var-15))
       (setf n var-2)
       (setf lda var-4)
       (setf ldb$ var-6)
       (setf ldq var-8)
       (setf ldz var-10)
       (setf nbnext var-12)
       (setf nbf var-13)
```

```
    (setf info var-16))
  (cond ((/= info 0) (setf ilst here) (go end_label)))
  (setf here (f2cl-lib:int-sub here nbnext))
  (cond
   ((= nbf 2)
    (if (=
         (f2cl-lib:fref a-%data% ((f2cl-lib:int-add here 1) here)
                        ((1 lda) (1 *)) a-%offset%)
         zero)
        (setf nbf 3)))))
 (t (setf nbnext 1)
  (cond
   ((>= here 3)
    (if (/=
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-sub here 1)
                         (f2cl-lib:int-sub here 2))
                        ((1 lda) (1 *)) a-%offset%)
         zero)
        (setf nbnext 2))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16)
      (stgex2 wantq wantz n a lda b ldb$ q ldq z ldz
       (f2cl-lib:int-sub here nbnext) nbnext 1 work lwork info)
    (declare
     (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-13
      var-14 var-15))
    (setf n var-2)
    (setf lda var-4)
    (setf ldb$ var-6)
    (setf ldq var-8)
    (setf ldz var-10)
    (setf nbnext var-12)
    (setf info var-16))
  (cond ((/= info 0) (setf ilst here) (go end_label)))
  (cond
   ((= nbnext 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16)
        (stgex2 wantq wantz n a lda b ldb$ q ldq z ldz here nbnext
         1 work lwork info)
      (declare
       (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-13
        var-14 var-15))
      (setf n var-2)
      (setf lda var-4)
      (setf ldb$ var-6)
      (setf ldq var-8)
```

```
    (setf ldz var-10)
    (setf nbnext var-12)
    (setf info var-16))
  (cond ((/= info 0) (setf ilst here) (go end_label)))
  (setf here (f2cl-lib:int-sub here 1)))
 (t
  (if (=
       (f2cl-lib:fref a-%data% (here (f2cl-lib:int-sub here 1))
                      ((1 lda) (1 *)) a-%offset%)
       zero)
      (setf nbnext 1))
  (cond
   ((= nbnext 2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16)
        (stgex2 wantq wantz n a lda b ldb$ q ldq z ldz
         (f2cl-lib:int-sub here 1) 2 1 work lwork info)
      (declare
       (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-12
        var-13 var-14 var-15))
      (setf n var-2)
      (setf lda var-4)
      (setf ldb$ var-6)
      (setf ldq var-8)
      (setf ldz var-10)
      (setf info var-16))
    (cond ((/= info 0) (setf ilst here) (go end_label)))
    (setf here (f2cl-lib:int-sub here 2)))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16)
        (stgex2 wantq wantz n a lda b ldb$ q ldq z ldz here 1 1
         work lwork info)
      (declare
       (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-12
        var-13 var-14 var-15))
      (setf n var-2)
      (setf lda var-4)
      (setf ldb$ var-6)
      (setf ldq var-8)
      (setf ldz var-10)
      (setf info var-16))
    (cond ((/= info 0) (setf ilst here) (go end_label)))
    (setf here (f2cl-lib:int-sub here 1))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16)
        (stgex2 wantq wantz n a lda b ldb$ q ldq z ldz here 1 1
```

```
                      work lwork info)
                    (declare
                     (ignore var-0 var-1 var-3 var-5 var-7 var-9 var-11 var-12
                      var-13 var-14 var-15))
                    (setf n var-2)
                    (setf lda var-4)
                    (setf ldb$ var-6)
                    (setf ldq var-8)
                    (setf ldz var-10)
                    (setf info var-16))
                  (cond ((/= info 0) (setf ilst here) (go end_label)))
                  (setf here (f2cl-lib:int-sub here 1)))))))))
              (if (> here ilst)
                  (go label20)))))
        (setf ilst here)
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwmin) 'single-float))
        (go end_label)
       end_label
        (return
         (values nil nil n nil lda nil ldb$ nil ldq nil ldz ifst ilst nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::stgexc
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '(fortran-to-lisp::logical
                                              fortran-to-lisp::logical
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$ nil
                                              fortran-to-lisp::ldq nil
```

```
  fortran-to-lisp::ldz
  fortran-to-lisp::ifst
  fortran-to-lisp::ilst nil nil
  fortran-to-lisp::info)
:calls '(fortran-to-lisp::stgex2))))
```

Chapter 34

LAPACK - General Matrices, Comp. Routines, Complex

34.1 cgebak

34.1.1 Axiom unit tests

— **cgebak.input** —

```
)set break resume
)sys rm -f cgebak.output
)spool cgebak.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.1.2 Axiom help page

— **cgebak.help** —

```
==========================================================================
cgebak examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgebak (JOB, SIDE, N, ILO, IHI, SCALE, M, V, LDV, INFO)
           CGEBAK

Function/Subroutine Documentation
   subroutine cgebak (character JOB, character SIDE, integer N, integer ILO,
       integer IHI, real, dimension( * ) SCALE, integer M, complex, dimension(
       ldv, * ) V, integer LDV, integer INFO)
       CGEBAK

Purpose:
   CGEBAK forms the right or left eigenvectors of a complex general
   matrix by backward transformation on the computed eigenvectors of the
   balanced matrix output by CGEBAL.

Parameters:

   JOB
   JOB is CHARACTER*1
   Specifies the type of backward transformation required:
   = 'N', do nothing, return immediately;
   = 'P', do backward transformation for permutation only;
   = 'S', do backward transformation for scaling only;
   = 'B', do backward transformations for both permutation and scaling.
   JOB must be the same as the argument JOB supplied to CGEBAL.

 SIDE
   SIDE is CHARACTER*1
   = 'R':  V contains right eigenvectors;
   = 'L':  V contains left eigenvectors.

 N
   N is INTEGER
   The number of rows of the matrix V.  N >= 0.

 ILO
   ILO is INTEGER

 IHI
   IHI is INTEGER
   The integers ILO and IHI determined by CGEBAL.
   1 <= ILO <= IHI <= N, if N > 0; ILO=1 and IHI=0, if N=0.

 SCALE
```

```
  SCALE is REAL array, dimension (N)
  Details of the permutation and scaling factors, as returned by CGEBAL.

M
  M is INTEGER
  The number of columns of the matrix V.  M >= 0.

V
  V is COMPLEX array, dimension (LDV,M)
  On entry, the matrix of right or left eigenvectors to be
  transformed, as returned by CHSEIN or CTREVC.
  On exit, V is overwritten by the transformed eigenvectors.

LDV
  LDV is INTEGER
  The leading dimension of the array V. LDV >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

34.1.3 fortran code

— cgebak.f —

```
*  =====================================================================
      SUBROUTINE CGEBAK( JOB, SIDE, N, ILO, IHI, SCALE, M, V, LDV,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOB, SIDE
      INTEGER            IHI, ILO, INFO, LDV, M, N
*     ..
*     .. Array Arguments ..
      REAL               SCALE( * )
      COMPLEX            V( LDV, * )
*     ..
*
*  =====================================================================
```

```
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LEFTV, RIGHTV
      INTEGER            I, II, K
      REAL               S
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CSSCAL, CSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Decode and Test the input parameters
*
      RIGHTV = LSAME( SIDE, 'R' )
      LEFTV = LSAME( SIDE, 'L' )
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.RIGHTV .AND. .NOT.LEFTV ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -5
      ELSE IF( M.LT.0 ) THEN
         INFO = -7
      ELSE IF( LDV.LT.MAX( 1, N ) ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEBAK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
```

```
*
      IF( N.EQ.0 )
     $   RETURN
      IF( M.EQ.0 )
     $   RETURN
      IF( LSAME( JOB, 'N' ) )
     $   RETURN
*
      IF( ILO.EQ.IHI )
     $   GO TO 30
*
*     Backward balance
*
      IF( LSAME( JOB, 'S' ) .OR. LSAME( JOB, 'B' ) ) THEN
*
         IF( RIGHTV ) THEN
            DO 10 I = ILO, IHI
               S = SCALE( I )
               CALL CSSCAL( M, S, V( I, 1 ), LDV )
   10       CONTINUE
         END IF
*
         IF( LEFTV ) THEN
            DO 20 I = ILO, IHI
               S = ONE / SCALE( I )
               CALL CSSCAL( M, S, V( I, 1 ), LDV )
   20       CONTINUE
         END IF
*
      END IF
*
*     Backward permutation
*
*     For  I = ILO-1 step -1 until 1,
*              IHI+1 step 1 until N do --
*
   30 CONTINUE
      IF( LSAME( JOB, 'P' ) .OR. LSAME( JOB, 'B' ) ) THEN
         IF( RIGHTV ) THEN
            DO 40 II = 1, N
               I = II
               IF( I.GE.ILO .AND. I.LE.IHI )
     $            GO TO 40
               IF( I.LT.ILO )
     $            I = ILO - II
               K = SCALE( I )
               IF( K.EQ.I )
     $            GO TO 40
               CALL CSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   40       CONTINUE
```

```
         END IF
*
         IF( LEFTV ) THEN
            DO 50 II = 1, N
               I = II
               IF( I.GE.ILO .AND. I.LE.IHI )
     $            GO TO 50
               IF( I.LT.ILO )
     $            I = ILO - II
               K = SCALE( I )
               IF( K.EQ.I )
     $            GO TO 50
               CALL CSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   50       CONTINUE
         END IF
      END IF
*
      RETURN
*
*     End of CGEBAK
*
      END
```

———-

34.1.4 lisp code

— **cgebak.lisp** —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun cgebak (job side n ilo ihi scale m v ldv info)
    (declare (type (array f2cl-lib:complex8 (*)) v)
             (type (array single-float (*)) scale)
             (type (f2cl-lib:integer4) info ldv m ihi ilo n)
             (type (string *) side job))
    (f2cl-lib:with-multi-array-data
        ((job character job-%data% job-%offset%)
         (side character side-%data% side-%offset%)
         (scale single-float scale-%data% scale-%offset%)
         (v f2cl-lib:complex8 v-%data% v-%offset%))
      (prog ((s 0.0) (i 0) (ii 0) (k 0) (leftv nil) (rightv nil))
        (declare (type (single-float) s)
                 (type (f2cl-lib:integer4) i ii k)
                 (type f2cl-lib:logical leftv rightv))
        (setf rightv
                (multiple-value-bind (ret-val var-0 var-1)
```

```
            (lsame side "R")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val))
(setf leftv
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "L")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val))
(setf info 0)
(cond
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "N")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "P")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "S")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "B")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)))
  (setf info -1))
 ((and (not rightv) (not leftv)) (setf info -2))
 ((< n 0) (setf info -3))
 ((or (< ilo 1)
      (> ilo
         (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n))))
  (setf info -4))
 ((or
   (< ihi (min (the f2cl-lib:integer4 ilo) (the f2cl-lib:integer4 n)))
   (> ihi n))
  (setf info -5))
 ((< m 0) (setf info -7))
 ((< ldv (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
```

```
  (setf info -9)))
(cond
 ((/= info 0) (xerbla "CGEBAK" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(if (= m 0)
    (go end_label))
(if (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "N")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)
    (go end_label))
(if (= ilo ihi)
    (go label30))
(cond
 ((or
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame job "S")
     (declare (ignore var-1))
     (when var-0 (setf job var-0))
     ret-val)
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame job "B")
     (declare (ignore var-1))
     (when var-0 (setf job var-0))
     ret-val))
  (cond
   (rightv
    (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                  ((> i ihi) nil)
      (tagbody
        (setf s
                (f2cl-lib:fref scale-%data% (i) ((1 *))
                               scale-%offset%))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (csscal m s
             (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (i 1)
                                   ((1 ldv) (1 *)) v-%offset%)
             ldv)
          (declare (ignore var-2))
          (when var-0 (setf m var-0))
          (when var-1 (setf s var-1))
          (when var-3 (setf ldv var-3)))
       label10))))
  (cond
   (leftv
    (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                  ((> i ihi) nil)
```

```
        (tagbody
          (setf s
                  (/ one
                     (f2cl-lib:fref scale-%data% (i) ((1 *))
                                    scale-%offset%)))
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (csscal m s
               (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (i 1)
                                     ((1 ldv) (1 *)) v-%offset%)
               ldv)
            (declare (ignore var-2))
            (when var-0 (setf m var-0))
            (when var-1 (setf s var-1))
            (when var-3 (setf ldv var-3)))
         label20))))))
label30
 (cond
  ((or
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "P")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "B")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (cond
    (rightv
     (f2cl-lib:fdo (ii 1 (f2cl-lib:int-add ii 1))
                   ((> ii n) nil)
       (tagbody
         (setf i ii)
         (if (and (>= i ilo) (<= i ihi))
             (go label40))
         (if (< i ilo)
             (setf i (f2cl-lib:int-sub ilo ii)))
         (setf k
                 (f2cl-lib:int
                  (f2cl-lib:fref scale-%data% (i) ((1 *))
                                 scale-%offset%)))
         (if (= k i)
             (go label40))
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (cswap m
              (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (i 1)
                                    ((1 ldv) (1 *)) v-%offset%)
              ldv
              (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (k 1)
```

```
                                        ((1 ldv) (1 *)) v-%offset%)
                    ldv)
                  (declare (ignore var-1 var-3))
                  (when var-0 (setf m var-0))
                  (when var-2 (setf ldv var-2))
                  (when var-4 (setf ldv var-4)))
              label40))))
          (cond
           (leftv
            (f2cl-lib:fdo (ii 1 (f2cl-lib:int-add ii 1))
                          ((> ii n) nil)
              (tagbody
                (setf i ii)
                (if (and (>= i ilo) (<= i ihi))
                    (go label50))
                (if (< i ilo)
                    (setf i (f2cl-lib:int-sub ilo ii)))
                (setf k
                        (f2cl-lib:int
                         (f2cl-lib:fref scale-%data% (i) ((1 *))
                                        scale-%offset%)))
                (if (= k i)
                    (go label50))
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (cswap m
                     (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (i 1)
                                           ((1 ldv) (1 *)) v-%offset%)
                     ldv
                     (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (k 1)
                                           ((1 ldv) (1 *)) v-%offset%)
                     ldv)
                  (declare (ignore var-1 var-3))
                  (when var-0 (setf m var-0))
                  (when var-2 (setf ldv var-2))
                  (when var-4 (setf ldv var-4)))
              label50))))))
        (go end_label)
       end_label
        (return (values job side nil nil nil nil m nil ldv info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgebak
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
```

```
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::job
  fortran-to-lisp::side nil nil nil
  nil fortran-to-lisp::m nil
  fortran-to-lisp::ldv
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

34.2 cgebal

34.2.1 Axiom unit tests

— **cgebal.input** —

```
)set break resume
)sys rm -f cgebal.output
)spool cgebal.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.2.2 Axiom help page

— **cgebal.help** —

```
==================================================================
cgebal examples
==================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgebal (JOB, N, A, LDA, ILO, IHI, SCALE, INFO)
           CGEBAL

Function/Subroutine Documentation
   subroutine cgebal (character JOB, integer N, complex, dimension( lda, * )
       A, integer LDA, integer ILO, integer IHI, real, dimension( * ) SCALE,
       integer INFO)
       CGEBAL

Purpose:
   CGEBAL balances a general complex matrix A.  This involves, first,
   permuting A by a similarity transformation to isolate eigenvalues
   in the first 1 to ILO-1 and last IHI+1 to N elements on the
   diagonal; and second, applying a diagonal similarity transformation
   to rows and columns ILO to IHI to make the rows and columns as
   close in norm as possible.  Both steps are optional.

   Balancing may reduce the 1-norm of the matrix, and improve the
   accuracy of the computed eigenvalues and/or eigenvectors.

Parameters:

 JOB
   JOB is CHARACTER*1
   Specifies the operations to be performed on A:
   = 'N':  none:  simply set ILO = 1, IHI = N, SCALE(I) = 1.0
     for i = 1,...,N;
   = 'P':  permute only;
   = 'S':  scale only;
   = 'B':  both permute and scale.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the input matrix A.
   On exit,  A is overwritten by the balanced matrix.
   If JOB = 'N', A is not referenced.
   See Further Details.

 LDA
   LDA is INTEGER
```

```
  The leading dimension of the array A.  LDA >= max(1,N).

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER
  ILO and IHI are set to integers such that on exit
  A(i,j) = 0 if i > j and j = 1,...,ILO-1 or I = IHI+1,...,N.
  If JOB = 'N' or 'S', ILO = 1 and IHI = N.

SCALE
  SCALE is REAL array, dimension (N)
  Details of the permutations and scaling factors applied to
  A.  If P(j) is the index of the row and column interchanged
  with row and column j and D(j) is the scaling factor
  applied to row and column j, then
  SCALE(j) = P(j)    for j = 1,...,ILO-1
           = D(j)    for j = ILO,...,IHI
           = P(j)    for j = IHI+1,...,N.
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.

  The permutations consist of row and column interchanges which put
  the matrix in the form

                         ( T1   X   Y  )
                P A P = (  0   B   Z  )
                         (  0   0   T2 )

  where T1 and T2 are upper triangular matrices whose eigenvalues lie
  along the diagonal.  The column indices ILO and IHI mark the starting
  and ending columns of the submatrix B. Balancing consists of applying
  a diagonal similarity transformation inv(D) * B * D to make the
  1-norms of each row of B and its corresponding column nearly equal.

  The output matrix is

                ( T1     X*D          Y    )
                (  0  inv(D)*B*D  inv(D)*Z ).
                (  0      0           T2   )

  Information about the permutations P and the diagonal matrix D is
  returned in the vector SCALE.
```

```
This subroutine is based on the EISPACK routine CBAL.
```

———-

34.2.3 fortran code

— **cgebal.f** —

```
*  =====================================================================
      SUBROUTINE CGEBAL( JOB, N, A, LDA, ILO, IHI, SCALE, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOB
      INTEGER            IHI, ILO, INFO, LDA, N
*     ..
*     .. Array Arguments ..
      REAL               SCALE( * )
      COMPLEX            A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               SCLFAC
      PARAMETER          ( SCLFAC = 2.0E+0 )
      REAL               FACTOR
      PARAMETER          ( FACTOR = 0.95E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOCONV
      INTEGER            I, ICA, IEXC, IRA, J, K, L, M
      REAL               C, CA, F, G, R, RA, S, SFMAX1, SFMAX2, SFMIN1,
     $                   SFMIN2
      COMPLEX            CDUM
*     ..
*     .. External Functions ..
      LOGICAL            SISNAN, LSAME
      INTEGER            ICAMAX
      REAL               SLAMCH, SCNRM2
      EXTERNAL           SISNAN, LSAME, ICAMAX, SLAMCH, SCNRM2
```

```
*     ..
*     .. External Subroutines ..
      EXTERNAL           CSSCAL, CSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, MAX, MIN, REAL
*
*     Test the input parameters
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEBAL', -INFO )
         RETURN
      END IF
*
      K = 1
      L = N
*
      IF( N.EQ.0 )
     $   GO TO 210
*
      IF( LSAME( JOB, 'N' ) ) THEN
         DO 10 I = 1, N
            SCALE( I ) = ONE
   10    CONTINUE
         GO TO 210
      END IF
*
      IF( LSAME( JOB, 'S' ) )
     $   GO TO 120
*
*     Permutation to isolate eigenvalues if possible
*
      GO TO 50
*
*     Row and column exchange.
*
   20 CONTINUE
      SCALE( M ) = J
      IF( J.EQ.M )
     $   GO TO 30
*
```

```
      CALL CSWAP( L, A( 1, J ), 1, A( 1, M ), 1 )
      CALL CSWAP( N-K+1, A( J, K ), LDA, A( M, K ), LDA )
*
   30 CONTINUE
      GO TO ( 40, 80 )IEXC
*
*     Search for rows isolating an eigenvalue and push them down.
*
   40 CONTINUE
      IF( L.EQ.1 )
     $   GO TO 210
      L = L - 1
*
   50 CONTINUE
      DO 70 J = L, 1, -1
*
         DO 60 I = 1, L
            IF( I.EQ.J )
     $         GO TO 60
            IF( REAL( A( J, I ) ).NE.ZERO .OR. AIMAG( A( J, I ) ).NE.
     $          ZERO )GO TO 70
   60    CONTINUE
*
         M = L
         IEXC = 1
         GO TO 20
   70 CONTINUE
*
      GO TO 90
*
*     Search for columns isolating an eigenvalue and push them left.
*
   80 CONTINUE
      K = K + 1
*
   90 CONTINUE
      DO 110 J = K, L
*
         DO 100 I = K, L
            IF( I.EQ.J )
     $         GO TO 100
            IF( REAL( A( I, J ) ).NE.ZERO .OR. AIMAG( A( I, J ) ).NE.
     $          ZERO )GO TO 110
  100    CONTINUE
*
         M = K
         IEXC = 2
         GO TO 20
  110 CONTINUE
*
```

```
  120 CONTINUE
      DO 130 I = K, L
         SCALE( I ) = ONE
  130 CONTINUE
*
      IF( LSAME( JOB, 'P' ) )
     $   GO TO 210
*
*     Balance the submatrix in rows K to L.
*
*     Iterative loop for norm reduction
*
      SFMIN1 = SLAMCH( 'S' ) / SLAMCH( 'P' )
      SFMAX1 = ONE / SFMIN1
      SFMIN2 = SFMIN1*SCLFAC
      SFMAX2 = ONE / SFMIN2
  140 CONTINUE
      NOCONV = .FALSE.
*
      DO 200 I = K, L
*
         C = SCNRM2( L-K+1, A( K, I ), 1 )
         R = SCNRM2( L-K+1, A( I , K ), LDA )
         ICA = ICAMAX( L, A( 1, I ), 1 )
         CA = ABS( A( ICA, I ) )
         IRA = ICAMAX( N-K+1, A( I, K ), LDA )
         RA = ABS( A( I, IRA+K-1 ) )
*
*        Guard against zero C or R due to underflow.
*
         IF( C.EQ.ZERO .OR. R.EQ.ZERO )
     $      GO TO 200
         G = R / SCLFAC
         F = ONE
         S = C + R
  160    CONTINUE
         IF( C.GE.G .OR. MAX( F, C, CA ).GE.SFMAX2 .OR.
     $       MIN( R, G, RA ).LE.SFMIN2 )GO TO 170
            IF( SISNAN( C+F+CA+R+G+RA ) ) THEN
*
*           Exit if NaN to avoid infinite loop
*
            INFO = -3
            CALL XERBLA( 'CGEBAL', -INFO )
            RETURN
         END IF
         F = F*SCLFAC
         C = C*SCLFAC
         CA = CA*SCLFAC
         R = R / SCLFAC
```

```
         G = G / SCLFAC
         RA = RA / SCLFAC
         GO TO 160
*
  170    CONTINUE
         G = C / SCLFAC
  180    CONTINUE
         IF( G.LT.R .OR. MAX( R, RA ).GE.SFMAX2 .OR.
     $       MIN( F, C, G, CA ).LE.SFMIN2 )GO TO 190
         F = F / SCLFAC
         C = C / SCLFAC
         G = G / SCLFAC
         CA = CA / SCLFAC
         R = R*SCLFAC
         RA = RA*SCLFAC
         GO TO 180
*
*        Now balance.
*
  190    CONTINUE
         IF( ( C+R ).GE.FACTOR*S )
     $      GO TO 200
         IF( F.LT.ONE .AND. SCALE( I ).LT.ONE ) THEN
            IF( F*SCALE( I ).LE.SFMIN1 )
     $         GO TO 200
         END IF
         IF( F.GT.ONE .AND. SCALE( I ).GT.ONE ) THEN
            IF( SCALE( I ).GE.SFMAX1 / F )
     $         GO TO 200
         END IF
         G = ONE / F
         SCALE( I ) = SCALE( I )*F
         NOCONV = .TRUE.
*
         CALL CSSCAL( N-K+1, G, A( I, K ), LDA )
         CALL CSSCAL( L, F, A( 1, I ), 1 )
*
  200 CONTINUE
*
      IF( NOCONV )
     $   GO TO 140
*
  210 CONTINUE
      ILO = K
      IHI = L
*
      RETURN
*
*     End of CGEBAL
*
```

```
      END
```

———-

34.2.4 lisp code

— **cgebal.lisp** —

```
(let* ((zero 0.0) (one 1.0) (sclfac 2.0) (factor 0.95))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) sclfac)
           (type (single-float 0.95 0.95) factor)
           (ignorable zero one sclfac factor))
  (defun cgebal (job n a lda ilo ihi scale info)
    (declare (type (array single-float (*)) scale)
             (type (array f2cl-lib:complex8 (*)) a)
             (type (f2cl-lib:integer4) info ihi ilo lda n)
             (type (string *) job))
    (f2cl-lib:with-multi-array-data
        ((job character job-%data% job-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (scale single-float scale-%data% scale-%offset%))
      (prog ((cdum #C(0.0 0.0)) (c 0.0) (ca 0.0) (f 0.0) (g 0.0) (r 0.0)
             (ra 0.0) (s 0.0) (sfmax1 0.0) (sfmax2 0.0) (sfmin1 0.0)
             (sfmin2 0.0) (i 0) (ica 0) (iexc 0) (ira 0) (j 0) (k 0) (l 0)
             (m 0) (noconv nil))
        (declare (type (f2cl-lib:complex8) cdum)
                 (type (single-float) c ca f g r ra s sfmax1 sfmax2 sfmin1
                  sfmin2)
                 (type (f2cl-lib:integer4) i ica iexc ira j k l m)
                 (type f2cl-lib:logical noconv))
        (setf info 0)
        (cond
         ((and
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame job "N")
              (declare (ignore var-1))
              (when var-0 (setf job var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame job "P")
              (declare (ignore var-1))
              (when var-0 (setf job var-0))
              ret-val))
           (not
```

```
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame job "S")
        (declare (ignore var-1))
        (when var-0 (setf job var-0))
        ret-val))
     (not
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame job "B")
        (declare (ignore var-1))
        (when var-0 (setf job var-0))
        ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4)))
  (cond
   ((/= info 0) (xerbla "CGEBAL" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf k 1)
  (setf l n)
  (if (= n 0)
      (go label210))
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame job "N")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%)
                one)
       label10))
    (go label210)))
  (if (multiple-value-bind (ret-val var-0 var-1)
          (lsame job "S")
        (declare (ignore var-1))
        (when var-0 (setf job var-0))
        ret-val)
      (go label120))
  (go label50)
 label20
  (setf (f2cl-lib:fref scale-%data% (m) ((1 *)) scale-%offset%)
          (coerce (the f2cl-lib:integer4 j) 'single-float))
  (if (= j m)
      (go label30))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (cswap l
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 j)
```

```
                              ((1 lda) (1 *)) a-%offset%)
      1
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 m)
                              ((1 lda) (1 *)) a-%offset%)
      1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf l var-0)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (cswap (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (j k)
                              ((1 lda) (1 *)) a-%offset%)
      lda
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (m k)
                              ((1 lda) (1 *)) a-%offset%)
      lda)
   (declare (ignore var-0 var-1 var-3))
   (when var-2 (setf lda var-2))
   (when var-4 (setf lda var-4)))
label30
 (f2cl-lib:computed-goto (label40 label80) iexc)
label40
 (if (= l 1)
     (go label210))
 (setf l (f2cl-lib:int-sub l 1))
label50
 (f2cl-lib:fdo (j l (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i l) nil)
       (tagbody
         (if (= i j)
             (go label60))
         (if (or
              (/=
               (f2cl-lib:freal
                (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                               a-%offset%))
               zero)
              (/=
               (f2cl-lib:aimag
                (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                               a-%offset%))
               zero))
             (go label70))
        label60))
     (setf m l)
     (setf iexc 1)
     (go label20)
    label70))
```

```
 (go label90)
label80
 (setf k (f2cl-lib:int-add k 1))
label90
 (f2cl-lib:fdo (j k (f2cl-lib:int-add j 1))
               ((> j l) nil)
   (tagbody
     (f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
                   ((> i l) nil)
       (tagbody
         (if (= i j)
             (go label100))
         (if (or
              (/=
               (f2cl-lib:freal
                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%))
               zero)
              (/=
               (f2cl-lib:aimag
                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%))
               zero))
             (go label110))
        label100))
     (setf m k)
     (setf iexc 2)
     (go label20)
    label110))
label120
 (f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
               ((> i l) nil)
   (tagbody
     (setf (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%) one)
    label130))
 (if (multiple-value-bind (ret-val var-0 var-1)
         (lsame job "P")
       (declare (ignore var-1))
       (when var-0 (setf job var-0))
       ret-val)
     (go label210))
 (setf sfmin1 (/ (slamch "S") (slamch "P")))
 (setf sfmax1 (/ one sfmin1))
 (setf sfmin2 (* sfmin1 sclfac))
 (setf sfmax2 (/ one sfmin2))
label140
 (setf noconv f2cl-lib:%false%)
 (f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
               ((> i l) nil)
   (tagbody
```

```
 (setf c
         (scnrm2 (f2cl-lib:int-add (f2cl-lib:int-sub l k) 1)
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (k i)
                                ((1 lda) (1 *)) a-%offset%)
          1))
 (setf r
         (multiple-value-bind (ret-val var-0 var-1 var-2)
             (scnrm2 (f2cl-lib:int-add (f2cl-lib:int-sub l k) 1)
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i k)
                                    ((1 lda) (1 *)) a-%offset%)
              lda)
           (declare (ignore var-0 var-1))
           (when var-2 (setf lda var-2))
           ret-val))
 (setf ica
         (multiple-value-bind (ret-val var-0 var-1 var-2)
             (icamax l
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 i)
                                    ((1 lda) (1 *)) a-%offset%)
              1)
           (declare (ignore var-1 var-2))
           (when var-0 (setf l var-0))
           ret-val))
 (setf ca
         (abs
          (f2cl-lib:fref a-%data% (ica i) ((1 lda) (1 *))
                         a-%offset%)))
 (setf ira
         (multiple-value-bind (ret-val var-0 var-1 var-2)
             (icamax (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i k)
                                    ((1 lda) (1 *)) a-%offset%)
              lda)
           (declare (ignore var-0 var-1))
           (when var-2 (setf lda var-2))
           ret-val))
 (setf ra
         (abs
          (f2cl-lib:fref a-%data%
                         (i
                          (f2cl-lib:int-sub (f2cl-lib:int-add ira k)
                                            1))
                         ((1 lda) (1 *)) a-%offset%)))
 (if (or (= c zero) (= r zero))
     (go label200))
 (setf g (/ r sclfac))
 (setf f one)
 (setf s (+ c r))
label160
 (if (or (>= c g) (>= (max f c ca) sfmax2) (<= (min r g ra) sfmin2))
```

```
      (go label170))
  (cond
   ((sisnan (+ c f ca r g ra)) (setf info -3)
    (xerbla "CGEBAL" (f2cl-lib:int-sub info)) (go end_label)))
  (setf f (* f sclfac))
  (setf c (* c sclfac))
  (setf ca (* ca sclfac))
  (setf r (/ r sclfac))
  (setf g (/ g sclfac))
  (setf ra (/ ra sclfac))
  (go label160)
 label170
  (setf g (/ c sclfac))
 label180
  (if (or (< g r) (>= (max r ra) sfmax2) (<= (min f c g ca) sfmin2))
      (go label190))
  (setf f (/ f sclfac))
  (setf c (/ c sclfac))
  (setf g (/ g sclfac))
  (setf ca (/ ca sclfac))
  (setf r (* r sclfac))
  (setf ra (* ra sclfac))
  (go label180)
 label190
  (if (>= (+ c r) (* factor s))
      (go label200))
  (cond
   ((and (< f one) (< (f2cl-lib:fref scale (i) ((1 *))) one))
    (if (<=
         (* f
            (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%))
         sfmin1)
        (go label200))))
  (cond
   ((and (> f one) (> (f2cl-lib:fref scale (i) ((1 *))) one))
    (if (>= (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%)
            (/ sfmax1 f))
        (go label200))))
  (setf g (/ one f))
  (setf (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%)
          (* (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%)
             f))
  (setf noconv f2cl-lib:%true%)
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (csscal (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1) g
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i k)
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-2))
    (when var-1 (setf g var-1))
```

```
              (when var-3 (setf lda var-3)))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (csscal l f
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 i)
                                       ((1 lda) (1 *)) a-%offset%)
                 1)
              (declare (ignore var-2 var-3))
              (when var-0 (setf l var-0))
              (when var-1 (setf f var-1)))
           label200))
        (if noconv
            (go label140))
       label210
        (setf ilo k)
        (setf ihi l)
        (go end_label)
       end_label
        (return (values job nil nil lda ilo ihi nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgebal
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::job nil nil
                                              fortran-to-lisp::lda
                                              fortran-to-lisp::ilo
                                              fortran-to-lisp::ihi nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

34.3 cgebd2

34.3.1 Axiom unit tests

— cgebd2.input —

```
)set break resume
)sys rm -f cgebd2.output
)spool cgebd2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.3.2 Axiom help page

— cgebd2.help —

```
======================================================================
cgebd2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgebd2 (M, N, A, LDA, D, E, TAUQ, TAUP, WORK, INFO)
           CGEBD2 reduces a general matrix to bidiagonal form using an
           unblocked algorithm.

Function/Subroutine Documentation
   subroutine cgebd2 (integer M, integer N, complex, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) D, real, dimension( * ) E, complex,
       dimension( * ) TAUQ, complex, dimension( * ) TAUP, complex, dimension(
       * ) WORK, integer INFO)
       CGEBD2 reduces a general matrix to bidiagonal form using an unblocked
       algorithm.

Purpose:
   CGEBD2 reduces a complex general m by n matrix A to upper or lower
```

```
  real bidiagonal form B by a unitary transformation: Q**H * A * P = B.

  If m >= n, B is upper bidiagonal; if m < n, B is lower bidiagonal.

Parameters:

 M
   M is INTEGER
   The number of rows in the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns in the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the m by n general matrix to be reduced.
   On exit,
   if m >= n, the diagonal and the first superdiagonal are
     overwritten with the upper bidiagonal matrix B; the
     elements below the diagonal, with the array TAUQ, represent
     the unitary matrix Q as a product of elementary
     reflectors, and the elements above the first superdiagonal,
     with the array TAUP, represent the unitary matrix P as
     a product of elementary reflectors;
   if m < n, the diagonal and the first subdiagonal are
     overwritten with the lower bidiagonal matrix B; the
     elements below the first subdiagonal, with the array TAUQ,
     represent the unitary matrix Q as a product of
     elementary reflectors, and the elements above the diagonal,
     with the array TAUP, represent the unitary matrix P as
     a product of elementary reflectors.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 D
   D is REAL array, dimension (min(M,N))
   The diagonal elements of the bidiagonal matrix B:
   D(i) = A(i,i).

 E
   E is REAL array, dimension (min(M,N)-1)
   The off-diagonal elements of the bidiagonal matrix B:
   if m >= n, E(i) = A(i,i+1) for i = 1,2,...,n-1;
   if m < n, E(i) = A(i+1,i) for i = 1,2,...,m-1.

 TAUQ
   TAUQ is COMPLEX array dimension (min(M,N))
```

```
  The scalar factors of the elementary reflectors which
  represent the unitary matrix Q. See Further Details.

TAUP
  TAUP is COMPLEX array, dimension (min(M,N))
  The scalar factors of the elementary reflectors which
  represent the unitary matrix P. See Further Details.

WORK
  WORK is COMPLEX array, dimension (max(M,N))

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value.

  The matrices Q and P are represented as products of elementary
  reflectors:

  If m >= n,

     Q = H(1) H(2) . . . H(n)  and  P = G(1) G(2) . . . G(n-1)

  Each H(i) and G(i) has the form:

     H(i) = I - tauq * v * v**H  and G(i) = I - taup * u * u**H

  where tauq and taup are complex scalars, and v and u are complex
  vectors; v(1:i-1) = 0, v(i) = 1, and v(i+1:m) is stored on exit in
  A(i+1:m,i); u(1:i) = 0, u(i+1) = 1, and u(i+2:n) is stored on exit in
  A(i,i+2:n); tauq is stored in TAUQ(i) and taup in TAUP(i).

  If m < n,

     Q = H(1) H(2) . . . H(m-1)  and  P = G(1) G(2) . . . G(m)

  Each H(i) and G(i) has the form:

     H(i) = I - tauq * v * v**H  and G(i) = I - taup * u * u**H

  where tauq and taup are complex scalars, v and u are complex vectors;
  v(1:i) = 0, v(i+1) = 1, and v(i+2:m) is stored on exit in A(i+2:m,i);
  u(1:i-1) = 0, u(i) = 1, and u(i+1:n) is stored on exit in A(i,i+1:n);
  tauq is stored in TAUQ(i) and taup in TAUP(i).

  The contents of A on exit are illustrated by the following examples:

    m = 6 and n = 5 (m > n):          m = 5 and n = 6 (m < n):

    (  d   e   u1  u1  u1 )           (  d   u1  u1  u1  u1  u1 )
```

```
  (  v1  d   e   u2  u2 )          (  e   d   u2  u2  u2  u2 )
  (  v1  v2  d   e   u3 )          (  v1  e   d   u3  u3  u3 )
  (  v1  v2  v3  d   e  )          (  v1  v2  e   d   u4  u4 )
  (  v1  v2  v3  v4  d  )          (  v1  v2  v3  e   d   u5 )
  (  v1  v2  v3  v4  v5 )
```

where d and e denote diagonal and off-diagonal elements of B, vi denotes an element of the vector defining H(i), and ui an element of the vector defining G(i).

———-

34.3.3 fortran code

— cgebd2.f —

```
*  =====================================================================
      SUBROUTINE CGEBD2( M, N, A, LDA, D, E, TAUQ, TAUP, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      REAL               D( * ), E( * )
      COMPLEX            A( LDA, * ), TAUP( * ), TAUQ( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ZERO, ONE
      PARAMETER          ( ZERO = ( 0.0E+0, 0.0E+0 ),
     $                   ONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      COMPLEX            ALPHA
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLACGV, CLARF, CLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          CONJG, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.LT.0 ) THEN
         CALL XERBLA( 'CGEBD2', -INFO )
         RETURN
      END IF
*
      IF( M.GE.N ) THEN
*
*        Reduce to upper bidiagonal form
*
         DO 10 I = 1, N
*
*           Generate elementary reflector H(i) to annihilate A(i+1:m,i)
*
            ALPHA = A( I, I )
            CALL CLARFG( M-I+1, ALPHA, A( MIN( I+1, M ), I ), 1,
     $                   TAUQ( I ) )
            D( I ) = ALPHA
            A( I, I ) = ONE
*
*           Apply H(i)**H to A(i:m,i+1:n) from the left
*
            IF( I.LT.N )
     $         CALL CLARF( 'Left', M-I+1, N-I, A( I, I ), 1,
     $                     CONJG( TAUQ( I ) ), A( I, I+1 ), LDA, WORK )
            A( I, I ) = D( I )
*
            IF( I.LT.N ) THEN
*
*              Generate elementary reflector G(i) to annihilate
*              A(i,i+2:n)
*
               CALL CLACGV( N-I, A( I, I+1 ), LDA )
               ALPHA = A( I, I+1 )
               CALL CLARFG( N-I, ALPHA, A( I, MIN( I+2, N ) ),
     $                      LDA, TAUP( I ) )
               E( I ) = ALPHA
```

```
               A( I, I+1 ) = ONE
*
*              Apply G(i) to A(i+1:m,i+1:n) from the right
*
               CALL CLARF( 'Right', M-I, N-I, A( I, I+1 ), LDA,
     $                     TAUP( I ), A( I+1, I+1 ), LDA, WORK )
               CALL CLACGV( N-I, A( I, I+1 ), LDA )
               A( I, I+1 ) = E( I )
            ELSE
               TAUP( I ) = ZERO
            END IF
   10    CONTINUE
      ELSE
*
*        Reduce to lower bidiagonal form
*
         DO 20 I = 1, M
*
*           Generate elementary reflector G(i) to annihilate A(i,i+1:n)
*
            CALL CLACGV( N-I+1, A( I, I ), LDA )
            ALPHA = A( I, I )
            CALL CLARFG( N-I+1, ALPHA, A( I, MIN( I+1, N ) ), LDA,
     $                   TAUP( I ) )
            D( I ) = ALPHA
            A( I, I ) = ONE
*
*           Apply G(i) to A(i+1:m,i:n) from the right
*
            IF( I.LT.M )
     $         CALL CLARF( 'Right', M-I, N-I+1, A( I, I ), LDA,
     $                     TAUP( I ), A( I+1, I ), LDA, WORK )
            CALL CLACGV( N-I+1, A( I, I ), LDA )
            A( I, I ) = D( I )
*
            IF( I.LT.M ) THEN
*
               Generate elementary reflector H(i) to annihilate
               A(i+2:m,i)
*
               ALPHA = A( I+1, I )
               CALL CLARFG( M-I, ALPHA, A( MIN( I+2, M ), I ), 1,
     $                      TAUQ( I ) )
               E( I ) = ALPHA
               A( I+1, I ) = ONE
*
               Apply H(i)**H to A(i+1:m,i+1:n) from the left
*
               CALL CLARF( 'Left', M-I, N-I, A( I+1, I ), 1,
     $                     CONJG( TAUQ( I ) ), A( I+1, I+1 ), LDA,
```

```
     $                     WORK )
               A( I+1, I ) = E( I )
            ELSE
               TAUQ( I ) = ZERO
            END IF
   20    CONTINUE
      END IF
      RETURN
*
*     End of CGEBD2
*
      END
```

———-

34.3.4 lisp code

— cgebd2.lisp —

```
(let* ((zero (f2cl-lib:cmplx 0.0 0.0)) (one (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:complex8) zero)
           (type (f2cl-lib:complex8) one)
           (ignorable zero one))
  (defun cgebd2 (m n a lda d e tauq taup work info)
    (declare (type (array single-float (*)) e d)
             (type (array f2cl-lib:complex8 (*)) work taup tauq a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (tauq f2cl-lib:complex8 tauq-%data% tauq-%offset%)
         (taup f2cl-lib:complex8 taup-%data% taup-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%))
      (prog ((alpha #C(0.0 0.0)) (i 0))
        (declare (type (f2cl-lib:complex8) alpha)
                 (type (f2cl-lib:integer4) i))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((< info 0) (xerbla "CGEBD2" (f2cl-lib:int-sub info)) (go end_label)))
        (cond
         ((>= m n)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
```

```
(tagbody
  (setf alpha
          (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                         a-%offset%))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (clarfg (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) alpha
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((min (f2cl-lib:int-add i 1) m) i)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice tauq-%data% f2cl-lib:complex8 (i)
                             ((1 *)) tauq-%offset%))
    (declare (ignore var-0 var-2 var-3 var-4))
    (when var-1 (setf alpha var-1)))
  (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
          (coerce (realpart alpha) 'single-float))
  (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
          one)
  (if (< i n)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (clarf "Left" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
           (f2cl-lib:int-sub n i)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:conjg
            (f2cl-lib:fref tauq-%data% (i) ((1 *)) tauq-%offset%))
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 (i (f2cl-lib:int-add i 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda work)
        (declare
         (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
        (when var-7 (setf lda var-7))))
  (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
          (coerce (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                  'f2cl-lib:complex8))
  (cond
   ((< i n)
    (multiple-value-bind (var-0 var-1 var-2)
        (clacgv (f2cl-lib:int-sub n i)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               (i (f2cl-lib:int-add i 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-0 var-1))
      (when var-2 (setf lda var-2)))
    (setf alpha
            (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
```

```
                        ((1 lda) (1 *)) a-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (clarfg (f2cl-lib:int-sub n i) alpha
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                           (i
                            (min
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add i 2))
                             (the f2cl-lib:integer4 n)))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice taup-%data% f2cl-lib:complex8 (i)
                           ((1 *)) taup-%offset%))
  (declare (ignore var-0 var-2 var-4))
  (when var-1 (setf alpha var-1))
  (when var-3 (setf lda var-3)))
(setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
        (coerce (realpart alpha) 'single-float))
(setf (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
                      ((1 lda) (1 *)) a-%offset%)
        one)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (clarf "Right" (f2cl-lib:int-sub m i)
     (f2cl-lib:int-sub n i)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                           (i (f2cl-lib:int-add i 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice taup-%data% f2cl-lib:complex8 (i)
                           ((1 *)) taup-%offset%)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                           ((+ i 1) (f2cl-lib:int-add i 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda work)
  (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8))
  (when var-4 (setf lda var-4))
  (when var-7 (setf lda var-7)))
(multiple-value-bind (var-0 var-1 var-2)
    (clacgv (f2cl-lib:int-sub n i)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                           (i (f2cl-lib:int-add i 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1))
  (when var-2 (setf lda var-2)))
(setf (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
                      ((1 lda) (1 *)) a-%offset%)
        (coerce (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                'f2cl-lib:complex8)))
```

```
        (t
         (setf (f2cl-lib:fref taup-%data% (i) ((1 *)) taup-%offset%)
                 zero)))
     label10)))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i m) nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2)
           (clacgv (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                  ((1 lda) (1 *)) a-%offset%)
            lda)
         (declare (ignore var-0 var-1))
         (when var-2 (setf lda var-2)))
       (setf alpha
               (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                              a-%offset%))
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (clarfg (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) alpha
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                  (i
                                   (min
                                    (the f2cl-lib:integer4
                                         (f2cl-lib:int-add i 1))
                                    (the f2cl-lib:integer4 n)))
                                  ((1 lda) (1 *)) a-%offset%)
            lda
            (f2cl-lib:array-slice taup-%data% f2cl-lib:complex8 (i)
                                  ((1 *)) taup-%offset%))
         (declare (ignore var-0 var-2 var-4))
         (when var-1 (setf alpha var-1))
         (when var-3 (setf lda var-3)))
       (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
               (coerce (realpart alpha) 'single-float))
       (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
               one)
       (if (< i m)
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
               (clarf "Right" (f2cl-lib:int-sub m i)
                (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice taup-%data% f2cl-lib:complex8 (i)
                                      ((1 *)) taup-%offset%)
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                      ((+ i 1) i) ((1 lda) (1 *))
                                      a-%offset%)
```

```
          lda work)
       (declare
        (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8))
       (when var-4 (setf lda var-4))
       (when var-7 (setf lda var-7))))
(multiple-value-bind (var-0 var-1 var-2)
    (clacgv (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1))
  (when var-2 (setf lda var-2)))
(setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
        (coerce (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                'f2cl-lib:complex8))
(cond
 ((< i m)
  (setf alpha
          (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                         ((1 lda) (1 *)) a-%offset%))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (clarfg (f2cl-lib:int-sub m i) alpha
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((min (f2cl-lib:int-add i 2) m) i)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice tauq-%data% f2cl-lib:complex8 (i)
                             ((1 *)) tauq-%offset%))
    (declare (ignore var-0 var-2 var-3 var-4))
    (when var-1 (setf alpha var-1)))
  (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
          (coerce (realpart alpha) 'single-float))
  (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                        ((1 lda) (1 *)) a-%offset%)
          one)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (clarf "Left" (f2cl-lib:int-sub m i) (f2cl-lib:int-sub n i)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ i 1) i) ((1 lda) (1 *))
                             a-%offset%)
       1
       (f2cl-lib:conjg
        (f2cl-lib:fref tauq-%data% (i) ((1 *)) tauq-%offset%))
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ i 1) (f2cl-lib:int-add i 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda work)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
```

```
                  (when var-7 (setf lda var-7)))
                 (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                                       ((1 lda) (1 *)) a-%offset%)
                         (coerce (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                                 'f2cl-lib:complex8)))
                (t
                 (setf (f2cl-lib:fref tauq-%data% (i) ((1 *)) tauq-%offset%)
                         zero)))
             label20))))
        (go end_label)
       end_label
        (return (values nil nil nil lda nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgebd2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::lda
                                              nil nil nil nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

34.4 cgebrd

34.4.1 Axiom unit tests

— cgebrd.input —

```
)set break resume
)sys rm -f cgebrd.output
)spool cgebrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.4.2 Axiom help page

— cgebrd.help —

```
====================================================================
cgebrd examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgebrd (M, N, A, LDA, D, E, TAUQ, TAUP, WORK, LWORK, INFO)
           CGEBRD

Function/Subroutine Documentation
   subroutine cgebrd (integer M, integer N, complex, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) D, real, dimension( * ) E, complex,
       dimension( * ) TAUQ, complex, dimension( * ) TAUP, complex, dimension(
       * ) WORK, integer LWORK, integer INFO)
       CGEBRD

Purpose:
   CGEBRD reduces a general complex M-by-N matrix A to upper or lower
   bidiagonal form B by a unitary transformation: Q**H * A * P = B.

   If m >= n, B is upper bidiagonal; if m < n, B is lower bidiagonal.

Parameters:

 M
   M is INTEGER
```

```
  The number of rows in the matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns in the matrix A.  N >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the M-by-N general matrix to be reduced.
  On exit,
  if m >= n, the diagonal and the first superdiagonal are
    overwritten with the upper bidiagonal matrix B; the
    elements below the diagonal, with the array TAUQ, represent
    the unitary matrix Q as a product of elementary
    reflectors, and the elements above the first superdiagonal,
    with the array TAUP, represent the unitary matrix P as
    a product of elementary reflectors;
  if m < n, the diagonal and the first subdiagonal are
    overwritten with the lower bidiagonal matrix B; the
    elements below the first subdiagonal, with the array TAUQ,
    represent the unitary matrix Q as a product of
    elementary reflectors, and the elements above the diagonal,
    with the array TAUP, represent the unitary matrix P as
    a product of elementary reflectors.


LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

D
  D is REAL array, dimension (min(M,N))
  The diagonal elements of the bidiagonal matrix B:
  D(i) = A(i,i).

E
  E is REAL array, dimension (min(M,N)-1)
  The off-diagonal elements of the bidiagonal matrix B:
  if m >= n, E(i) = A(i,i+1) for i = 1,2,...,n-1;
  if m < n, E(i) = A(i+1,i) for i = 1,2,...,m-1.

TAUQ
  TAUQ is COMPLEX array dimension (min(M,N))
  The scalar factors of the elementary reflectors which
  represent the unitary matrix Q. See Further Details.

TAUP
  TAUP is COMPLEX array, dimension (min(M,N))
  The scalar factors of the elementary reflectors which
  represent the unitary matrix P. See Further Details.
```

```
WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of the array WORK.  LWORK >= max(1,M,N).
  For optimum performance LWORK >= (M+N)*NB, where NB
  is the optimal blocksize.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.

  The matrices Q and P are represented as products of elementary
  reflectors:

  If m >= n,

     Q = H(1) H(2) . . . H(n)  and  P = G(1) G(2) . . . G(n-1)

  Each H(i) and G(i) has the form:

     H(i) = I - tauq * v * v**H  and G(i) = I - taup * u * u**H

  where tauq and taup are complex scalars, and v and u are complex
  vectors; v(1:i-1) = 0, v(i) = 1, and v(i+1:m) is stored on exit in
  A(i+1:m,i); u(1:i) = 0, u(i+1) = 1, and u(i+2:n) is stored on exit in
  A(i,i+2:n); tauq is stored in TAUQ(i) and taup in TAUP(i).

  If m < n,

     Q = H(1) H(2) . . . H(m-1)  and  P = G(1) G(2) . . . G(m)

  Each H(i) and G(i) has the form:

     H(i) = I - tauq * v * v**H  and G(i) = I - taup * u * u**H

  where tauq and taup are complex scalars, and v and u are complex
  vectors; v(1:i) = 0, v(i+1) = 1, and v(i+2:m) is stored on exit in
  A(i+2:m,i); u(1:i-1) = 0, u(i) = 1, and u(i+1:n) is stored on exit in
  A(i,i+1:n); tauq is stored in TAUQ(i) and taup in TAUP(i).
```

```
The contents of A on exit are illustrated by the following examples:

     m = 6 and n = 5 (m > n):          m = 5 and n = 6 (m < n):

       (  d   e   u1  u1  u1 )           (  d   u1  u1  u1  u1  u1 )
       (  v1  d   e   u2  u2 )           (  e   d   u2  u2  u2  u2 )
       (  v1  v2  d   e   u3 )           (  v1  e   d   u3  u3  u3 )
       (  v1  v2  v3  d   e  )           (  v1  v2  e   d   u4  u4 )
       (  v1  v2  v3  v4  d  )           (  v1  v2  v3  e   d   u5 )
       (  v1  v2  v3  v4  v5 )

where d and e denote diagonal and off-diagonal elements of B, vi
denotes an element of the vector defining H(i), and ui an element of
the vector defining G(i).
```

34.4.3 fortran code

— cgebrd.f —

```
*  =====================================================================
      SUBROUTINE CGEBRD( M, N, A, LDA, D, E, TAUQ, TAUP, WORK, LWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      REAL               D( * ), E( * )
      COMPLEX            A( LDA, * ), TAUP( * ), TAUQ( * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ONE
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
```

```
      INTEGER            I, IINFO, J, LDWRKX, LDWRKY, LWKOPT, MINMN, NB,
     $                   NBMIN, NX
      REAL               WS
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEBD2, CGEMM, CLABRD, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, REAL
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      NB = MAX( 1, ILAENV( 1, 'CGEBRD', ' ', M, N, -1, -1 ) )
      LWKOPT = ( M+N )*NB
      WORK( 1 ) = REAL( LWKOPT )
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, M, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -10
      END IF
      IF( INFO.LT.0 ) THEN
         CALL XERBLA( 'CGEBRD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      MINMN = MIN( M, N )
      IF( MINMN.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      WS = MAX( M, N )
      LDWRKX = M
      LDWRKY = N
```

```
*
      IF( NB.GT.1 .AND. NB.LT.MINMN ) THEN
*
*        Set the crossover point NX.
*
         NX = MAX( NB, ILAENV( 3, 'CGEBRD', ' ', M, N, -1, -1 ) )
*
*        Determine when to switch from blocked to unblocked code.
*
         IF( NX.LT.MINMN ) THEN
            WS = ( M+N )*NB
            IF( LWORK.LT.WS ) THEN
*
*              Not enough work space for the optimal NB, consider using
*              a smaller block size.
*
               NBMIN = ILAENV( 2, 'CGEBRD', ' ', M, N, -1, -1 )
               IF( LWORK.GE.( M+N )*NBMIN ) THEN
                  NB = LWORK / ( M+N )
               ELSE
                  NB = 1
                  NX = MINMN
               END IF
            END IF
         END IF
      ELSE
         NX = MINMN
      END IF
*
      DO 30 I = 1, MINMN - NX, NB
*
*        Reduce rows and columns i:i+ib-1 to bidiagonal form and return
*        the matrices X and Y which are needed to update the unreduced
*        part of the matrix
*
         CALL CLABRD( M-I+1, N-I+1, NB, A( I, I ), LDA, D( I ), E( I ),
     $                TAUQ( I ), TAUP( I ), WORK, LDWRKX,
     $                WORK( LDWRKX*NB+1 ), LDWRKY )
*
*        Update the trailing submatrix A(i+ib:m,i+ib:n), using
*        an update of the form  A := A - V*Y**H - X*U**H
*
         CALL CGEMM( 'No transpose', 'Conjugate transpose', M-I-NB+1,
     $               N-I-NB+1, NB, -ONE, A( I+NB, I ), LDA,
     $               WORK( LDWRKX*NB+NB+1 ), LDWRKY, ONE,
     $               A( I+NB, I+NB ), LDA )
         CALL CGEMM( 'No transpose', 'No transpose', M-I-NB+1, N-I-NB+1,
     $               NB, -ONE, WORK( NB+1 ), LDWRKX, A( I, I+NB ), LDA,
     $               ONE, A( I+NB, I+NB ), LDA )
*
```

```
*        Copy diagonal and off-diagonal elements of B back into A
*
         IF( M.GE.N ) THEN
            DO 10 J = I, I + NB - 1
               A( J, J ) = D( J )
               A( J, J+1 ) = E( J )
   10       CONTINUE
         ELSE
            DO 20 J = I, I + NB - 1
               A( J, J ) = D( J )
               A( J+1, J ) = E( J )
   20       CONTINUE
         END IF
   30 CONTINUE
*
*     Use unblocked code to reduce the remainder of the matrix
*
      CALL CGEBD2( M-I+1, N-I+1, A( I, I ), LDA, D( I ), E( I ),
     $             TAUQ( I ), TAUP( I ), WORK, IINFO )
      WORK( 1 ) = WS
      RETURN
*
*     End of CGEBRD
*
      END
```

———-

34.4.4 lisp code

— **cgebrd.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (ignorable one))
  (defun cgebrd (m n a lda d e tauq taup work lwork info)
    (declare (type (array single-float (*)) e d)
             (type (array f2cl-lib:complex8 (*)) work taup tauq a)
             (type (f2cl-lib:integer4) info lwork lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (tauq f2cl-lib:complex8 tauq-%data% tauq-%offset%)
         (taup f2cl-lib:complex8 taup-%data% taup-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%))
      (prog ((ws 0.0) (i 0) (iinfo 0) (j 0) (ldwrkx 0) (ldwrky 0) (lwkopt 0)
             (minmn 0) (nb 0) (nbmin 0) (nx 0) (lquery nil))
```

```
(declare (type (single-float) ws)
         (type (f2cl-lib:integer4) i iinfo j ldwrkx ldwrky lwkopt minmn
          nb nbmin nx)
         (type f2cl-lib:logical lquery))
(setf info 0)
(setf nb
        (max (the f2cl-lib:integer4 1)
             (the f2cl-lib:integer4
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                       var-6)
                      (ilaenv 1 "CGEBRD" " " m n -1 -1)
                    (declare (ignore var-0 var-1 var-2 var-5 var-6))
                    (when var-3 (setf m var-3))
                    (when var-4 (setf n var-4))
                    ret-val))))
(setf lwkopt (f2cl-lib:int-mul (f2cl-lib:int-add m n) nb))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce (f2cl-lib:freal lwkopt) 'f2cl-lib:complex8))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -4))
      ((and
        (< lwork
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)
                (the f2cl-lib:integer4 n)))
        (not lquery))
       (setf info -10)))
(cond
 ((< info 0) (xerbla "CGEBRD" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(cond
 ((= minmn 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce 1 'f2cl-lib:complex8))
  (go end_label)))
(setf ws
        (coerce
         (the f2cl-lib:integer4
              (max (the f2cl-lib:integer4 m)
                   (the f2cl-lib:integer4 n)))
         'single-float))
(setf ldwrkx m)
(setf ldwrky n)
(cond
 ((and (> nb 1) (< nb minmn))
  (setf nx
```

```
              (max (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
                            (ilaenv 3 "CGEBRD" " " m n -1 -1)
                          (declare (ignore var-0 var-1 var-2 var-5 var-6))
                          (when var-3 (setf m var-3))
                          (when var-4 (setf n var-4))
                          ret-val))))
  (cond
   ((< nx minmn)
    (setf ws
            (coerce
             (the f2cl-lib:integer4
                  (f2cl-lib:int-mul (f2cl-lib:int-add m n) nb))
             'single-float))
    (cond
     ((< lwork ws)
      (setf nbmin
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 2 "CGEBRD" " " m n -1 -1)
                (declare (ignore var-0 var-1 var-2 var-5 var-6))
                (when var-3 (setf m var-3))
                (when var-4 (setf n var-4))
                ret-val))
      (cond
       ((>= lwork (f2cl-lib:int-mul (f2cl-lib:int-add m n) nbmin))
        (setf nb (the f2cl-lib:integer4 (truncate lwork (+ m n)))))
       (t (setf nb 1) (setf nx minmn))))))))
 (t (setf nx minmn)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
              ((> i (f2cl-lib:int-add minmn (f2cl-lib:int-sub nx))) nil)
  (tagbody
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (clabrd (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
         (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) nb
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice d-%data% single-float (i) ((1 *))
                               d-%offset%)
         (f2cl-lib:array-slice e-%data% single-float (i) ((1 *))
                               e-%offset%)
         (f2cl-lib:array-slice tauq-%data% f2cl-lib:complex8 (i)
                               ((1 *)) tauq-%offset%)
         (f2cl-lib:array-slice taup-%data% f2cl-lib:complex8 (i)
```

```
                          ((1 *)) taup-%offset%)
      work ldwrkx
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                          ((+ (f2cl-lib:int-mul ldwrkx nb) 1))
                          ((1 *)) work-%offset%)
      ldwrky)
   (declare
    (ignore var-0 var-1 var-3 var-5 var-6 var-7 var-8 var-9 var-11))
   (when var-2 (setf nb var-2))
   (when var-4 (setf lda var-4))
   (when var-10 (setf ldwrkx var-10))
   (when var-12 (setf ldwrky var-12)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12)
     (cgemm "No transpose" "Conjugate transpose"
      (f2cl-lib:int-add (f2cl-lib:int-sub m i nb) 1)
      (f2cl-lib:int-add (f2cl-lib:int-sub n i nb) 1) nb (- one)
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 ((+ i nb) i)
                          ((1 lda) (1 *)) a-%offset%)
      lda
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                          ((+ (f2cl-lib:int-mul ldwrkx nb) nb 1))
                          ((1 *)) work-%offset%)
      ldwrky one
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                          ((+ i nb) (f2cl-lib:int-add i nb))
                          ((1 lda) (1 *)) a-%offset%)
      lda)
   (declare
    (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8 var-11))
   (when var-4 (setf nb var-4))
   (when var-7 (setf lda var-7))
   (when var-9 (setf ldwrky var-9))
   (when var-10 (setf one var-10))
   (when var-12 (setf lda var-12)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12)
     (cgemm "No transpose" "No transpose"
      (f2cl-lib:int-add (f2cl-lib:int-sub m i nb) 1)
      (f2cl-lib:int-add (f2cl-lib:int-sub n i nb) 1) nb (- one)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ nb 1))
                          ((1 *)) work-%offset%)
      ldwrkx
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                          (i (f2cl-lib:int-add i nb))
                          ((1 lda) (1 *)) a-%offset%)
      lda one
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
```

```
                                  ((+ i nb) (f2cl-lib:int-add i nb))
                                  ((1 lda) (1 *)) a-%offset%)
              lda)
            (declare
             (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8 var-11))
            (when var-4 (setf nb var-4))
            (when var-7 (setf ldwrkx var-7))
            (when var-9 (setf lda var-9))
            (when var-10 (setf one var-10))
            (when var-12 (setf lda var-12)))
          (cond
           ((>= m n)
            (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                          ((> j (f2cl-lib:int-add i nb (f2cl-lib:int-sub 1)))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                      a-%offset%)
                      (coerce
                       (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                       'f2cl-lib:complex8))
                (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-add j 1))
                                      ((1 lda) (1 *)) a-%offset%)
                      (coerce
                       (f2cl-lib:fref e-%data% (j) ((1 *)) e-%offset%)
                       'f2cl-lib:complex8))
              label10)))
           (t
            (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                          ((> j (f2cl-lib:int-add i nb (f2cl-lib:int-sub 1)))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                      a-%offset%)
                      (coerce
                       (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                       'f2cl-lib:complex8))
                (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add j 1) j)
                                      ((1 lda) (1 *)) a-%offset%)
                      (coerce
                       (f2cl-lib:fref e-%data% (j) ((1 *)) e-%offset%)
                       'f2cl-lib:complex8))
              label20))))
        label30))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (cgebd2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
           (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                 ((1 lda) (1 *)) a-%offset%)
```

```
            lda
            (f2cl-lib:array-slice d-%data% single-float (i) ((1 *))
                                  d-%offset%)
            (f2cl-lib:array-slice e-%data% single-float (i) ((1 *))
                                  e-%offset%)
            (f2cl-lib:array-slice tauq-%data% f2cl-lib:complex8 (i) ((1 *))
                                  tauq-%offset%)
            (f2cl-lib:array-slice taup-%data% f2cl-lib:complex8 (i) ((1 *))
                                  taup-%offset%)
            work iinfo)
          (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-7 var-8))
          (setf lda var-3)
          (setf iinfo var-9))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce ws 'f2cl-lib:complex8))
        (go end_label)
       end_label
        (return (values m n nil lda nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgebrd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              nil nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::cgebd2))))
```

———-

34.5 cgecon

34.5.1 Axiom unit tests

— **cgecon.input** —

```
)set break resume
)sys rm -f cgecon.output
)spool cgecon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.5.2 Axiom help page

— **cgecon.help** —

```
====================================================================
cgecon examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgecon (NORM, N, A, LDA, ANORM, RCOND, WORK, RWORK, INFO)
           CGECON

Function/Subroutine Documentation
   subroutine cgecon (character NORM, integer N, complex, dimension( lda, * )
       A, integer LDA, real ANORM, real RCOND, complex, dimension( * ) WORK,
       real, dimension( * ) RWORK, integer INFO)
       CGECON

Purpose:
```

```
   CGECON estimates the reciprocal of the condition number of a general
   complex matrix A, in either the 1-norm or the infinity-norm, using
   the LU factorization computed by CGETRF.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as
       RCOND = 1 / ( norm(A) * norm(inv(A)) ).

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies whether the 1-norm condition number or the
   infinity-norm condition number is required:
   = '1' or 'O':  1-norm;
   = 'I':         Infinity-norm.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   The factors L and U from the factorization A = P*L*U
   as computed by CGETRF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 ANORM
   ANORM is REAL
   If NORM = '1' or 'O', the 1-norm of the original matrix A.
   If NORM = 'I', the infinity-norm of the original matrix A.

 RCOND
   RCOND is REAL
   The reciprocal of the condition number of the matrix A,
   computed as RCOND = 1/(norm(A) * norm(inv(A))).

 WORK
   WORK is COMPLEX array, dimension (2*N)

 RWORK
   RWORK is REAL array, dimension (2*N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

34.5.3 fortran code

— **cgecon.f** —

```
*  =====================================================================
      SUBROUTINE CGECON( NORM, N, A, LDA, ANORM, RCOND, WORK, RWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            INFO, LDA, N
      REAL               ANORM, RCOND
*     ..
*     .. Array Arguments ..
      REAL               RWORK( * )
      COMPLEX            A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ONENRM
      CHARACTER          NORMIN
      INTEGER            IX, KASE, KASE1
      REAL               AINVNM, SCALE, SL, SMLNUM, SU
      COMPLEX            ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ICAMAX
      REAL               SLAMCH
      EXTERNAL           LSAME, ICAMAX, SLAMCH
```

```
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLACN2, CLATRS, CSRSCL, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, MAX, REAL
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      ONENRM = NORM.EQ.'1' .OR. LSAME( NORM, 'O' )
      IF( .NOT.ONENRM .AND. .NOT.LSAME( NORM, 'I' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -5
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGECON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
      SMLNUM = SLAMCH( 'Safe minimum' )
*
*     Estimate the norm of inv(A).
*
      AINVNM = ZERO
      NORMIN = 'N'
      IF( ONENRM ) THEN
```

```
         KASE1 = 1
      ELSE
         KASE1 = 2
      END IF
      KASE = 0
   10 CONTINUE
      CALL CLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.KASE1 ) THEN
*
*           Multiply by inv(L).
*
            CALL CLATRS( 'Lower', 'No transpose', 'Unit', NORMIN, N, A,
     $                   LDA, WORK, SL, RWORK, INFO )
*
*           Multiply by inv(U).
*
            CALL CLATRS( 'Upper', 'No transpose', 'Non-unit', NORMIN, N,
     $                   A, LDA, WORK, SU, RWORK( N+1 ), INFO )
         ELSE
*
*           Multiply by inv(U**H).
*
            CALL CLATRS( 'Upper', 'Conjugate transpose', 'Non-unit',
     $                   NORMIN, N, A, LDA, WORK, SU, RWORK( N+1 ),
     $                   INFO )
*
*           Multiply by inv(L**H).
*
            CALL CLATRS( 'Lower', 'Conjugate transpose', 'Unit', NORMIN,
     $                   N, A, LDA, WORK, SL, RWORK, INFO )
         END IF
*
*        Divide X by 1/(SL*SU) if doing so will not cause overflow.
*
         SCALE = SL*SU
         NORMIN = 'Y'
         IF( SCALE.NE.ONE ) THEN
            IX = ICAMAX( N, WORK, 1 )
            IF( SCALE.LT.CABS1( WORK( IX ) )*SMLNUM .OR. SCALE.EQ.ZERO )
     $         GO TO 20
            CALL CSRSCL( N, SCALE, WORK, 1 )
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
```

```
*
   20 CONTINUE
      RETURN
*
*     End of CGECON
*
      END
```

34.5.4 lisp code

— **cgecon.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun cgecon (norm n a lda anorm rcond work rwork info)
    (declare (type (array single-float (*)) rwork)
             (type (single-float) rcond anorm)
             (type (array f2cl-lib:complex8 (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
          cabs1))
        (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
               (zdum #C(0.0 0.0)) (ainvnm 0.0) (scale 0.0) (sl 0.0)
               (smlnum 0.0) (su 0.0) (ix 0) (kase 0) (kase1 0)
               (normin
                (make-array '(1) :element-type 'character :initial-element
                            #\ ))
               (onenrm nil))
          (declare (type (array f2cl-lib:integer4 (3)) isave)
                   (type (f2cl-lib:complex8) zdum)
                   (type (single-float) ainvnm scale sl smlnum su)
                   (type (f2cl-lib:integer4) ix kase kase1)
                   (type (string 1) normin)
                   (type f2cl-lib:logical onenrm))
          (setf info 0)
```

```
 (setf onenrm
         (or (f2cl-lib:fstring-= norm "1")
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame norm "O")
               (declare (ignore var-1))
               (when var-0 (setf norm var-0))
               ret-val)))
 (cond
  ((and (not onenrm)
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame norm "I")
           (declare (ignore var-1))
           (when var-0 (setf norm var-0))
           ret-val)))
   (setf info -1))
  ((< n 0) (setf info -2))
  ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
   (setf info -4))
  ((< anorm zero) (setf info -5)))
 (cond
  ((/= info 0) (xerbla "CGECON" (f2cl-lib:int-sub info))
   (go end_label)))
 (setf rcond zero)
 (cond ((= n 0) (setf rcond one) (go end_label))
       ((= anorm zero) (go end_label)))
 (setf smlnum (slamch "Safe minimum"))
 (setf ainvnm zero)
 (f2cl-lib:f2cl-set-string normin "N" (string 1))
 (cond (onenrm (setf kase1 1)) (t (setf kase1 2)))
 (setf kase 0)
label10
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
     (clacn2 n
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ n 1))
                            ((1 *)) work-%offset%)
      work ainvnm kase isave)
   (declare (ignore var-1 var-2 var-5))
   (when var-0 (setf n var-0))
   (when var-3 (setf ainvnm var-3))
   (when var-4 (setf kase var-4)))
 (cond
  ((/= kase 0)
   (cond
    ((= kase kase1)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (clatrs "Lower" "No transpose" "Unit" normin n a lda work sl
          rwork info)
```

```
      (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
      (when var-3 (setf normin var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf sl var-8))
      (when var-10 (setf info var-10)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (clatrs "Upper" "No transpose" "Non-unit" normin n a lda work
         su
         (f2cl-lib:array-slice rwork-%data% single-float ((+ n 1))
                               ((1 *)) rwork-%offset%)
         info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
      (when var-3 (setf normin var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf su var-8))
      (when var-10 (setf info var-10))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (clatrs "Upper" "Conjugate transpose" "Non-unit" normin n a
         lda work su
         (f2cl-lib:array-slice rwork-%data% single-float ((+ n 1))
                               ((1 *)) rwork-%offset%)
         info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
      (when var-3 (setf normin var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf su var-8))
      (when var-10 (setf info var-10)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (clatrs "Lower" "Conjugate transpose" "Unit" normin n a lda
         work sl rwork info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
      (when var-3 (setf normin var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf sl var-8))
      (when var-10 (setf info var-10)))))
  (setf scale (* sl su))
  (f2cl-lib:f2cl-set-string normin "Y" (string 1))
  (cond
   ((/= scale one)
```

```
              (setf ix
                      (multiple-value-bind (ret-val var-0 var-1 var-2)
                          (icamax n work 1)
                        (declare (ignore var-1 var-2))
                        (when var-0 (setf n var-0))
                        ret-val))
              (if (or
                   (< scale
                      (*
                       (cabs1
                        (f2cl-lib:fref work-%data% (ix) ((1 *)) work-%offset%))
                       smlnum))
                   (= scale zero))
                  (go label20))
              (multiple-value-bind (var-0 var-1 var-2 var-3)
                  (csrscl n scale work 1)
                (declare (ignore var-2 var-3))
                (when var-0 (setf n var-0))
                (when var-1 (setf scale var-1)))))
            (go label10)))
          (if (/= ainvnm zero)
              (setf rcond (/ (/ one ainvnm) anorm)))
         label20
          (go end_label)
         end_label
          (return (values norm n nil lda nil rcond nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgecon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::norm
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::rcond nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

34.6 cgeequ

34.6.1 Axiom unit tests

— **cgeequ.input** —

```
)set break resume
)sys rm -f cgeequ.output
)spool cgeequ.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.6.2 Axiom help page

— **cgeequ.help** —

```
======================================================================
cgeequ examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgeequ (M, N, A, LDA, R, C, ROWCND, COLCND, AMAX, INFO)
           CGEEQU

Function/Subroutine Documentation
   subroutine cgeequ (integer M, integer N, complex, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) R, real, dimension( * ) C, real
       ROWCND, real COLCND, real AMAX, integer INFO)
```

```
       CGEEQU

Purpose:
   CGEEQU computes row and column scalings intended to equilibrate an
   M-by-N matrix A and reduce its condition number.  R returns the row
   scale factors and C the column scale factors, chosen to try to make
   the largest element in each row and column of the matrix B with
   elements B(i,j)=R(i)*A(i,j)*C(j) have absolute value 1.

   R(i) and C(j) are restricted to be between SMLNUM = smallest safe
   number and BIGNUM = largest safe number.  Use of these scaling
   factors is not guaranteed to reduce the condition number of A but
   works well in practice.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   The M-by-N matrix whose equilibration factors are to be computed.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 R
   R is REAL array, dimension (M)
   If INFO = 0 or INFO > M, R contains the row scale factors for A.

 C
   C is REAL array, dimension (N)
   If INFO = 0,  C contains the column scale factors for A.

 ROWCND
   ROWCND is REAL
   If INFO = 0 or INFO > M, ROWCND contains the ratio of the
   smallest R(i) to the largest R(i).  If ROWCND >= 0.1 and
   AMAX is neither too large nor too small, it is not worth
   scaling by R.

 COLCND
   COLCND is REAL
   If INFO = 0, COLCND contains the ratio of the smallest
```

```
  C(i) to the largest C(i).  If COLCND >= 0.1, it is not
  worth scaling by C.

AMAX
  AMAX is REAL
  Absolute value of largest matrix element.  If AMAX is very
  close to overflow or very close to underflow, the matrix
  should be scaled.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i,  and i is
        <= M:  the i-th row of A is exactly zero
        >  M:  the (i-M)-th column of A is exactly zero
```

34.6.3 fortran code

— cgeequ.f —

```
*  =====================================================================
      SUBROUTINE CGEEQU( M, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
      REAL               AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      REAL               C( * ), R( * )
      COMPLEX            A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
```

```
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               BIGNUM, RCMAX, RCMIN, SMLNUM
      COMPLEX            ZDUM
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, MAX, MIN, REAL
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEEQU', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         ROWCND = ONE
         COLCND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Get machine constants.
*
      SMLNUM = SLAMCH( 'S' )
```

```
      BIGNUM = ONE / SMLNUM
*
*     Compute row scale factors.
*
      DO 10 I = 1, M
         R( I ) = ZERO
   10 CONTINUE
*
*     Find the maximum element in each row.
*
      DO 30 J = 1, N
         DO 20 I = 1, M
            R( I ) = MAX( R( I ), CABS1( A( I, J ) ) )
   20    CONTINUE
   30 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 40 I = 1, M
         RCMAX = MAX( RCMAX, R( I ) )
         RCMIN = MIN( RCMIN, R( I ) )
   40 CONTINUE
      AMAX = RCMAX
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 50 I = 1, M
            IF( R( I ).EQ.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   50    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 60 I = 1, M
            R( I ) = ONE / MIN( MAX( R( I ), SMLNUM ), BIGNUM )
   60    CONTINUE
*
*        Compute ROWCND = min(R(I)) / max(R(I))
*
         ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
*     Compute column scale factors
```

```
*
      DO 70 J = 1, N
         C( J ) = ZERO
   70 CONTINUE
*
*     Find the maximum element in each column,
*     assuming the row scaling computed above.
*
      DO 90 J = 1, N
         DO 80 I = 1, M
            C( J ) = MAX( C( J ), CABS1( A( I, J ) )*R( I ) )
   80    CONTINUE
   90 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 100 J = 1, N
         RCMIN = MIN( RCMIN, C( J ) )
         RCMAX = MAX( RCMAX, C( J ) )
  100 CONTINUE
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 110 J = 1, N
            IF( C( J ).EQ.ZERO ) THEN
               INFO = M + J
               RETURN
            END IF
  110    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 120 J = 1, N
            C( J ) = ONE / MIN( MAX( C( J ), SMLNUM ), BIGNUM )
  120    CONTINUE
*
*        Compute COLCND = min(C(J)) / max(C(J))
*
         COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
      RETURN
*
*     End of CGEEQU
*
```

```
      END
```

34.6.4 lisp code

— **cgeequ.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun cgeequ (m n a lda r c rowcnd colcnd amax info)
    (declare (type (single-float) amax colcnd rowcnd)
             (type (array single-float (*)) c r)
             (type (array f2cl-lib:complex8 (*)) a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (r single-float r-%data% r-%offset%)
         (c single-float c-%data% c-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
          cabs1))
        (prog ((zdum #C(0.0 0.0)) (bignum 0.0) (rcmax 0.0) (rcmin 0.0)
               (smlnum 0.0) (i 0) (j 0))
          (declare (type (f2cl-lib:complex8) zdum)
                   (type (single-float) bignum rcmax rcmin smlnum)
                   (type (f2cl-lib:integer4) i j))
          (setf info 0)
          (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
                ((< lda
                    (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
                 (setf info -4)))
          (cond
           ((/= info 0) (xerbla "CGEEQU" (f2cl-lib:int-sub info))
            (go end_label)))
          (cond
           ((or (= m 0) (= n 0)) (setf rowcnd one) (setf colcnd one)
            (setf amax zero) (go end_label)))
          (setf smlnum (slamch "S"))
          (setf bignum (/ one smlnum))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%) zero)
```

```
   label10))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                     (cabs1
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))))
       label20))
   label30))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf rcmax
            (max rcmax
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
    (setf rcmin
            (min rcmin
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
   label40))
(setf amax rcmax)
(cond
 ((= rcmin zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref r (i) ((1 *))) zero) (setf info i)
        (go end_label)))
     label50)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
              (/ one
                 (min
                  (max
                   (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                   smlnum)
                  bignum)))
     label60))
  (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) zero)
   label70))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                     (*
                      (cabs1
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%))
                      (f2cl-lib:fref r-%data% (i) ((1 *))
                                     r-%offset%))))
       label80))
   label90))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf rcmin
            (min rcmin
                 (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
    (setf rcmax
            (max rcmax
                 (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
   label100))
(cond
 ((= rcmin zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref c (j) ((1 *))) zero)
        (setf info (f2cl-lib:int-add m j)) (go end_label)))
     label110)))
 (t
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
              (/ one
                 (min
                  (max
                   (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
```

```
                              smlnum)
                             bignum)))
                 label120))
               (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
             (go end_label)
           end_label
             (return (values nil nil nil nil nil nil rowcnd colcnd amax info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgeequ
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (single-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil
                                              fortran-to-lisp::rowcnd
                                              fortran-to-lisp::colcnd
                                              fortran-to-lisp::amax
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

34.7 cgeequb

34.7.1 Axiom unit tests

— **cgeequb.input** —

```
)set break resume
)sys rm -f cgeequb.output
)spool cgeequb.output
)set message test on
)set message auto off
```

```
)clear all

)spool
)lisp (bye)
```

———-

34.7.2 Axiom help page

— **cgeequb.help** —

```
=====================================================================
cgeequb examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgeequb (M, N, A, LDA, R, C, ROWCND, COLCND, AMAX, INFO)
           CGEEQUB

Function/Subroutine Documentation
   subroutine cgeequb (integer M, integer N, complex, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) R, real, dimension( * ) C, real
       ROWCND, real COLCND, real AMAX, integer INFO)
       CGEEQUB

Purpose:
   CGEEQUB computes row and column scalings intended to equilibrate an
   M-by-N matrix A and reduce its condition number.  R returns the row
   scale factors and C the column scale factors, chosen to try to make
   the largest element in each row and column of the matrix B with
   elements B(i,j)=R(i)*A(i,j)*C(j) have an absolute value of at most
   the radix.

   R(i) and C(j) are restricted to be a power of the radix between
   SMLNUM = smallest safe number and BIGNUM = largest safe number.  Use
   of these scaling factors is not guaranteed to reduce the condition
   number of A but works well in practice.

   This routine differs from CGEEQU by restricting the scaling factors
   to a power of the radix.  Baring over- and underflow, scaling by
   these factors introduces no additional rounding errors.  However, the
   scaled entries' magnitured are no longer approximately 1 but lie
   between sqrt(radix) and 1/sqrt(radix).
```

```
Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   The M-by-N matrix whose equilibration factors are to be computed.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 R
   R is REAL array, dimension (M)
   If INFO = 0 or INFO > M, R contains the row scale factors for A.

 C
   C is REAL array, dimension (N)
   If INFO = 0,  C contains the column scale factors for A.

 ROWCND
   ROWCND is REAL
   If INFO = 0 or INFO > M, ROWCND contains the ratio of the
   smallest R(i) to the largest R(i).  If ROWCND >= 0.1 and
   AMAX is neither too large nor too small, it is not worth
   scaling by R.

 COLCND
   COLCND is REAL
   If INFO = 0, COLCND contains the ratio of the smallest
   C(i) to the largest C(i).  If COLCND >= 0.1, it is not
   worth scaling by C.

 AMAX
   AMAX is REAL
   Absolute value of largest matrix element.  If AMAX is very
   close to overflow or very close to underflow, the matrix
   should be scaled.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

```
> 0:  if INFO = i,  and i is
      <= M:  the i-th row of A is exactly zero
      >  M:  the (i-M)-th column of A is exactly zero
```

———-

34.7.3 fortran code

— **cgeequb.f** —

```
*  =====================================================================
      SUBROUTINE CGEEQUB( M, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $                    INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
      REAL               AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      REAL               C( * ), R( * )
      COMPLEX            A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               BIGNUM, RCMAX, RCMIN, SMLNUM, RADIX, LOGRDX
      COMPLEX            ZDUM
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          ABS, MAX, MIN, LOG, REAL, AIMAG
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEEQUB', -INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         ROWCND = ONE
         COLCND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Get machine constants.  Assume SMLNUM is a power of the radix.
*
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      RADIX = SLAMCH( 'B' )
      LOGRDX = LOG( RADIX )
*
*     Compute row scale factors.
*
      DO 10 I = 1, M
         R( I ) = ZERO
   10 CONTINUE
*
*     Find the maximum element in each row.
*
      DO 30 J = 1, N
```

```
         DO 20 I = 1, M
            R( I ) = MAX( R( I ), CABS1( A( I, J ) ) )
   20    CONTINUE
   30 CONTINUE
      DO I = 1, M
         IF( R( I ).GT.ZERO ) THEN
            R( I ) = RADIX**INT( LOG(R( I ) ) / LOGRDX )
         END IF
      END DO
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 40 I = 1, M
         RCMAX = MAX( RCMAX, R( I ) )
         RCMIN = MIN( RCMIN, R( I ) )
   40 CONTINUE
      AMAX = RCMAX
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 50 I = 1, M
            IF( R( I ).EQ.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   50    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 60 I = 1, M
            R( I ) = ONE / MIN( MAX( R( I ), SMLNUM ), BIGNUM )
   60    CONTINUE
*
*        Compute ROWCND = min(R(I)) / max(R(I)).
*
         ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
*     Compute column scale factors.
*
      DO 70 J = 1, N
         C( J ) = ZERO
   70 CONTINUE
*
*     Find the maximum element in each column,
```

```
*     assuming the row scaling computed above.
*
      DO 90 J = 1, N
         DO 80 I = 1, M
            C( J ) = MAX( C( J ), CABS1( A( I, J ) )*R( I ) )
   80    CONTINUE
         IF( C( J ).GT.ZERO ) THEN
            C( J ) = RADIX**INT( LOG( C( J ) ) / LOGRDX )
         END IF
   90 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 100 J = 1, N
         RCMIN = MIN( RCMIN, C( J ) )
         RCMAX = MAX( RCMAX, C( J ) )
  100 CONTINUE
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 110 J = 1, N
            IF( C( J ).EQ.ZERO ) THEN
               INFO = M + J
               RETURN
            END IF
  110    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 120 J = 1, N
            C( J ) = ONE / MIN( MAX( C( J ), SMLNUM ), BIGNUM )
  120    CONTINUE
*
*        Compute COLCND = min(C(J)) / max(C(J)).
*
         COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
      RETURN
*
*     End of CGEEQUB
*
      END
```

———-

34.7.4 lisp code

— cgeequb.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun cgeequb (m n a lda r c rowcnd colcnd amax info)
    (declare (type (single-float) amax colcnd rowcnd)
             (type (array single-float (*)) c r)
             (type (array f2cl-lib:complex8 (*)) a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (r single-float r-%data% r-%offset%)
         (c single-float c-%data% c-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
          cabs1))
        (prog ((zdum #C(0.0 0.0)) (bignum 0.0) (rcmax 0.0) (rcmin 0.0)
               (smlnum 0.0) (radix 0.0) (logrdx 0.0) (i 0) (j 0))
          (declare (type (f2cl-lib:complex8) zdum)
                   (type (single-float) bignum rcmax rcmin smlnum radix logrdx)
                   (type (f2cl-lib:integer4) i j))
          (setf info 0)
          (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
                ((< lda
                    (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
                 (setf info -4)))
          (cond
           ((/= info 0) (xerbla "CGEEQUB" (f2cl-lib:int-sub info))
            (go end_label)))
          (cond
           ((or (= m 0) (= n 0)) (setf rowcnd one) (setf colcnd one)
            (setf amax zero) (go end_label)))
          (setf smlnum (slamch "S"))
          (setf bignum (/ one smlnum))
          (setf radix (slamch "B"))
          (setf logrdx (f2cl-lib:flog radix))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%) zero)
             label10))
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
```

```
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                     (cabs1
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))))
       label20))
   label30))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref r (i) ((1 *))) zero)
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
              (expt radix
                    (f2cl-lib:int
                     (/
                      (f2cl-lib:flog
                       (f2cl-lib:fref r-%data% (i) ((1 *))
                                      r-%offset%))
                      logrdx))))))
   label100000))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf rcmax
            (max rcmax
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
    (setf rcmin
            (min rcmin
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
   label40))
(setf amax rcmax)
(cond
 ((= rcmin zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref r (i) ((1 *))) zero) (setf info i)
        (go end_label)))
     label50)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
```

```
      (tagbody
        (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                (/ one
                   (min
                    (max
                     (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                     smlnum)
                    bignum)))
       label60))
    (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) zero)
     label70))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                  (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                       (*
                        (cabs1
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%))
                        (f2cl-lib:fref r-%data% (i) ((1 *))
                                       r-%offset%))))
         label80))
      (cond
       ((> (f2cl-lib:fref c (j) ((1 *))) zero)
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                (expt radix
                      (f2cl-lib:int
                       (/
                        (f2cl-lib:flog
                         (f2cl-lib:fref c-%data% (j) ((1 *))
                                        c-%offset%))
                        logrdx))))))
     label90))
  (setf rcmin bignum)
  (setf rcmax zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf rcmin
              (min rcmin
                   (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
      (setf rcmax
```

```
                    (max rcmax
                         (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
            label100))
          (cond
           ((= rcmin zero)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (cond
                 ((= (f2cl-lib:fref c (j) ((1 *))) zero)
                  (setf info (f2cl-lib:int-add m j)) (go end_label)))
               label110)))
           (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                        (/ one
                           (min
                            (max
                             (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                             smlnum)
                            bignum)))
               label120))
            (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
          (go end_label)
         end_label
          (return (values nil nil nil nil nil nil rowcnd colcnd amax info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgeequb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (single-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil
                                              fortran-to-lisp::rowcnd
                                              fortran-to-lisp::colcnd
                                              fortran-to-lisp::amax
```

```
                                        fortran-to-lisp::info)
                                      :calls 'nil)))
```

———-

34.8 cgehd2

34.8.1 Axiom unit tests

— **cgehd2.input** —

```
)set break resume
)sys rm -f cgehd2.output
)spool cgehd2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.8.2 Axiom help page

— **cgehd2.help** —

```
======================================================================
cgehd2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgehd2 (N, ILO, IHI, A, LDA, TAU, WORK, INFO)
           CGEHD2 reduces a general square matrix to upper Hessenberg form
           using an unblocked algorithm.

Function/Subroutine Documentation
```

```
   subroutine cgehd2 (integer N, integer ILO, integer IHI, complex, dimension(
       lda, * ) A, integer LDA, complex, dimension( * ) TAU, complex,
       dimension( * ) WORK, integer INFO)
       CGEHD2 reduces a general square matrix to upper Hessenberg form using
       an unblocked algorithm.

Purpose:
   CGEHD2 reduces a complex general matrix A to upper Hessenberg form H
   by a unitary similarity transformation:  Q**H * A * Q = H .

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 ILO
   ILO is INTEGER

 IHI
   IHI is INTEGER

   It is assumed that A is already upper triangular in rows
   and columns 1:ILO-1 and IHI+1:N. ILO and IHI are normally
   set by a previous call to CGEBAL; otherwise they should be
   set to 1 and N respectively. See Further Details.
   1 <= ILO <= IHI <= max(1,N).

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the n by n general matrix to be reduced.
   On exit, the upper triangle and the first subdiagonal of A
   are overwritten with the upper Hessenberg matrix H, and the
   elements below the first subdiagonal, with the array TAU,
   represent the unitary matrix Q as a product of elementary
   reflectors. See Further Details.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 TAU
   TAU is COMPLEX array, dimension (N-1)
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is COMPLEX array, dimension (N)

 INFO
   INFO is INTEGER
```

```
= 0:  successful exit
< 0:  if INFO = -i, the i-th argument had an illegal value.


The matrix Q is represented as a product of (ihi-ilo) elementary
reflectors

      Q = H(ilo) H(ilo+1) . . . H(ihi-1).

Each H(i) has the form

      H(i) = I - tau * v * v**H

where tau is a complex scalar, and v is a complex vector with
v(1:i) = 0, v(i+1) = 1 and v(ihi+1:n) = 0; v(i+2:ihi) is stored on
exit in A(i+2:ihi,i), and tau in TAU(i).

The contents of A are illustrated by the following example, with
n = 7, ilo = 2 and ihi = 6:

on entry,                        on exit,

( a   a   a   a   a   a   a )    (  a   a   h   h   h   h   a )
(     a   a   a   a   a   a )    (      a   h   h   h   h   a )
(     a   a   a   a   a   a )    (      h   h   h   h   h   h )
(     a   a   a   a   a   a )    (      v2  h   h   h   h   h )
(     a   a   a   a   a   a )    (      v2  v3  h   h   h   h )
(     a   a   a   a   a   a )    (      v2  v3  v4  h   h   h )
(                         a )    (                          a )

where a denotes an element of the original matrix A, h denotes a
modified element of the upper Hessenberg matrix H, and vi denotes an
element of the vector defining H(i).
```

———-

34.8.3 fortran code

— cgehd2.f —

```
*  =====================================================================
      SUBROUTINE CGEHD2( N, ILO, IHI, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
```

```
*
*     .. Scalar Arguments ..
      INTEGER            IHI, ILO, INFO, LDA, N
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ONE
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      COMPLEX            ALPHA
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLARF, CLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          CONJG, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -2
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEHD2', -INFO )
         RETURN
      END IF
*
      DO 10 I = ILO, IHI - 1
*
*        Compute elementary reflector H(i) to annihilate A(i+2:ihi,i)
*
         ALPHA = A( I+1, I )
         CALL CLARFG( IHI-I, ALPHA, A( MIN( I+2, N ), I ), 1, TAU( I ) )
         A( I+1, I ) = ONE
```

```
*
*        Apply H(i) to A(1:ihi,i+1:ihi) from the right
*
         CALL CLARF( 'Right', IHI, IHI-I, A( I+1, I ), 1, TAU( I ),
     $               A( 1, I+1 ), LDA, WORK )
*
*        Apply H(i)**H to A(i+1:ihi,i+1:n) from the left
*
         CALL CLARF( 'Left', IHI-I, N-I, A( I+1, I ), 1,
     $               CONJG( TAU( I ) ), A( I+1, I+1 ), LDA, WORK )
*
         A( I+1, I ) = ALPHA
   10 CONTINUE
*
      RETURN
*
*     End of CGEHD2
*
      END
```

34.8.4 lisp code

— cgehd2.lisp —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (ignorable one))
  (defun cgehd2 (n ilo ihi a lda tau work info)
    (declare (type (array f2cl-lib:complex8 (*)) work tau a)
             (type (f2cl-lib:integer4) info lda ihi ilo n))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (tau f2cl-lib:complex8 tau-%data% tau-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%))
      (prog ((alpha #C(0.0 0.0)) (i 0))
        (declare (type (f2cl-lib:complex8) alpha)
                 (type (f2cl-lib:integer4) i))
        (setf info 0)
        (cond ((< n 0) (setf info -1))
              ((or (< ilo 1)
                   (> ilo
                      (max (the f2cl-lib:integer4 1)
                           (the f2cl-lib:integer4 n))))
               (setf info -2))
              ((or
                (< ihi
```

```
                (min (the f2cl-lib:integer4 ilo) (the f2cl-lib:integer4 n)))
            (> ihi n))
           (setf info -3))
          ((< lda
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
           (setf info -5)))
(cond
 ((/= info 0) (xerbla "CGEHD2" (f2cl-lib:int-sub info))
  (go end_label)))
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add ihi (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (setf alpha
            (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                           ((1 lda) (1 *)) a-%offset%))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (clarfg (f2cl-lib:int-sub ihi i) alpha
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               ((min (f2cl-lib:int-add i 2) n) i)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i) ((1 *))
                               tau-%offset%))
      (declare (ignore var-0 var-2 var-3 var-4))
      (when var-1 (setf alpha var-1)))
    (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                         ((1 lda) (1 *)) a-%offset%)
            one)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clarf "Right" ihi (f2cl-lib:int-sub ihi i)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 ((+ i 1) i)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i) ((1 *))
                               tau-%offset%)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               (1 (f2cl-lib:int-add i 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda work)
      (declare (ignore var-0 var-2 var-3 var-4 var-5 var-6 var-8))
      (when var-1 (setf ihi var-1))
      (when var-7 (setf lda var-7)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clarf "Left" (f2cl-lib:int-sub ihi i) (f2cl-lib:int-sub n i)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 ((+ i 1) i)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:conjg
```

```
                 (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                      ((+ i 1) (f2cl-lib:int-add i 1))
                                      ((1 lda) (1 *)) a-%offset%)
                lda work)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
              (when var-7 (setf lda var-7)))
            (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                                  ((1 lda) (1 *)) a-%offset%)
                  alpha)
          label10))
        (go end_label)
       end_label
        (return (values nil nil ihi nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgehd2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil fortran-to-lisp::ihi nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

34.9 cgehrd

34.9.1 Axiom unit tests

— cgehrd.input —

```
)set break resume
)sys rm -f cgehrd.output
)spool cgehrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.9.2 Axiom help page

— cgehrd.help —

```
======================================================================
cgehrd examples
======================================================================

======================================================================
Man Page Details
======================================================================
   This file is a slight modification of LAPACK-3.0's DGEHRD
   subroutine incorporating improvements proposed by Quintana-Orti and
   Van de Geijn (2006). (See DLAHR2.)

SYNOPSIS
   Functions/Subroutines
       subroutine cgehrd (N, ILO, IHI, A, LDA, TAU, WORK, LWORK, INFO)
           CGEHRD

Function/Subroutine Documentation
   subroutine cgehrd (integer N, integer ILO, integer IHI, complex, dimension(
       lda, * ) A, integer LDA, complex, dimension( * ) TAU, complex,
       dimension( * ) WORK, integer LWORK, integer INFO)
       CGEHRD

Purpose:
   CGEHRD reduces a complex general matrix A to upper Hessenberg form H by
   an unitary similarity transformation:  Q**H * A * Q = H .

Parameters:

 N
   N is INTEGER
```

```
  The order of the matrix A.  N >= 0.

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER

  It is assumed that A is already upper triangular in rows
  and columns 1:ILO-1 and IHI+1:N. ILO and IHI are normally
  set by a previous call to CGEBAL; otherwise they should be
  set to 1 and N respectively. See Further Details.
  1 <= ILO <= IHI <= N, if N > 0; ILO=1 and IHI=0, if N=0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the N-by-N general matrix to be reduced.
  On exit, the upper triangle and the first subdiagonal of A
  are overwritten with the upper Hessenberg matrix H, and the
  elements below the first subdiagonal, with the array TAU,
  represent the unitary matrix Q as a product of elementary
  reflectors. See Further Details.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

TAU
  TAU is COMPLEX array, dimension (N-1)
  The scalar factors of the elementary reflectors (see Further
  Details). Elements 1:ILO-1 and IHI:N-1 of TAU are set to zero.

WORK
  WORK is COMPLEX array, dimension (LWORK)
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of the array WORK.  LWORK >= max(1,N).
  For good performance, LWORK should generally be larger.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

```
The matrix Q is represented as a product of (ihi-ilo) elementary
reflectors

   Q = H(ilo) H(ilo+1) . . . H(ihi-1).

Each H(i) has the form

   H(i) = I - tau * v * v**H

where tau is a complex scalar, and v is a complex vector with
v(1:i) = 0, v(i+1) = 1 and v(ihi+1:n) = 0; v(i+2:ihi) is stored on
exit in A(i+2:ihi,i), and tau in TAU(i).

The contents of A are illustrated by the following example, with
n = 7, ilo = 2 and ihi = 6:

on entry,                        on exit,

( a   a   a   a   a   a   a )    (  a   a   h   h   h   h   a )
(     a   a   a   a   a   a )    (      a   h   h   h   h   a )
(     a   a   a   a   a   a )    (      h   h   h   h   h   h )
(     a   a   a   a   a   a )    (      v2  h   h   h   h   h )
(     a   a   a   a   a   a )    (      v2  v3  h   h   h   h )
(     a   a   a   a   a   a )    (      v2  v3  v4  h   h   h )
(                         a )    (                          a )

where a denotes an element of the original matrix A, h denotes a
modified element of the upper Hessenberg matrix H, and vi denotes an
element of the vector defining H(i).

This file is a slight modification of LAPACK-3.0's DGEHRD
subroutine incorporating improvements proposed by Quintana-Orti and
Van de Geijn (2006). (See DLAHR2.)
```

———-

34.9.3 fortran code

— cgehrd.f —

```
*  =====================================================================
      SUBROUTINE CGEHRD( N, ILO, IHI, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
```

```
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            IHI, ILO, INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            NBMAX, LDT, TSIZE
      PARAMETER          ( NBMAX = 64, LDT = NBMAX+1,
     $                     TSIZE = LDT*NBMAX )
      COMPLEX            ZERO, ONE
      PARAMETER          ( ZERO = ( 0.0E+0, 0.0E+0 ),
     $                     ONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWT, J, LDWORK, LWKOPT, NB,
     $                   NBMIN, NH, NX
      COMPLEX            EI
*     ..
*     .. External Subroutines ..
      EXTERNAL           CAXPY, CGEHD2, CGEMM, CLAHR2, CLARFB, CTRMM,
     $                   XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -2
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
```

```
         INFO = -5
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -8
      END IF
*
      IF( INFO.EQ.0 ) THEN
*
*        Compute the workspace requirements
*
         NB = MIN( NBMAX, ILAENV( 1, 'CGEHRD', ' ', N, ILO, IHI, -1 ) )
         LWKOPT = N*NB + TSIZE
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEHRD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Set elements 1:ILO-1 and IHI:N-1 of TAU to zero
*
      DO 10 I = 1, ILO - 1
         TAU( I ) = ZERO
   10 CONTINUE
      DO 20 I = MAX( 1, IHI ), N - 1
         TAU( I ) = ZERO
   20 CONTINUE
*
*     Quick return if possible
*
      NH = IHI - ILO + 1
      IF( NH.LE.1 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
*     Determine the block size
*
      NB = MIN( NBMAX, ILAENV( 1, 'CGEHRD', ' ', N, ILO, IHI, -1 ) )
      NBMIN = 2
      IF( NB.GT.1 .AND. NB.LT.NH ) THEN
*
*        Determine when to cross over from blocked to unblocked code
*        (last block is always handled by unblocked code)
*
         NX = MAX( NB, ILAENV( 3, 'CGEHRD', ' ', N, ILO, IHI, -1 ) )
         IF( NX.LT.NH ) THEN
*
```

```
*          Determine if workspace is large enough for blocked code
*
           IF( LWORK.LT.N*NB+TSIZE ) THEN
*
*             Not enough workspace to use optimal NB:  determine the
*             minimum value of NB, and reduce NB or force use of
*             unblocked code
*
              NBMIN = MAX( 2, ILAENV( 2, 'CGEHRD', ' ', N, ILO, IHI,
     $                -1 ) )
              IF( LWORK.GE.(N*NBMIN+TSIZE) ) THEN
                 NB = (LWORK-TSIZE) / N
              ELSE
                 NB = 1
              END IF
           END IF
        END IF
     END IF
     LDWORK = N
*
     IF( NB.LT.NBMIN .OR. NB.GE.NH ) THEN
*
*       Use unblocked code below
*
        I = ILO
*
     ELSE
*
*       Use blocked code
*
        IWT = 1 + N*NB
        DO 40 I = ILO, IHI - 1 - NX, NB
           IB = MIN( NB, IHI-I )
*
*          Reduce columns i:i+ib-1 to Hessenberg form, returning the
*          matrices V and T of the block reflector H = I - V*T*V**H
*          which performs the reduction, and also the matrix Y = A*V*T
*
           CALL CLAHR2( IHI, I, IB, A( 1, I ), LDA, TAU( I ),
     $                  WORK( IWT ), LDT, WORK, LDWORK )
*
*          Apply the block reflector H to A(1:ihi,i+ib:ihi) from the
*          right, computing  A := A - Y * V**H. V(i+ib,ib-1) must be set
*          to 1
*
           EI = A( I+IB, I+IB-1 )
           A( I+IB, I+IB-1 ) = ONE
           CALL CGEMM( 'No transpose', 'Conjugate transpose',
     $                 IHI, IHI-I-IB+1,
     $                 IB, -ONE, WORK, LDWORK, A( I+IB, I ), LDA, ONE,
```

```
     $                  A( 1, I+IB ), LDA )
            A( I+IB, I+IB-1 ) = EI
*
*           Apply the block reflector H to A(1:i,i+1:i+ib-1) from the
*           right
*
            CALL CTRMM( 'Right', 'Lower', 'Conjugate transpose',
     $                  'Unit', I, IB-1,
     $                  ONE, A( I+1, I ), LDA, WORK, LDWORK )
            DO 30 J = 0, IB-2
               CALL CAXPY( I, -ONE, WORK( LDWORK*J+1 ), 1,
     $                     A( 1, I+J+1 ), 1 )
   30       CONTINUE
*
*           Apply the block reflector H to A(i+1:ihi,i+ib:n) from the
*           left
*
            CALL CLARFB( 'Left', 'Conjugate transpose', 'Forward',
     $                   'Columnwise',
     $                   IHI-I, N-I-IB+1, IB, A( I+1, I ), LDA,
     $                   WORK( IWT ), LDT, A( I+1, I+IB ), LDA,
     $                   WORK, LDWORK )
   40    CONTINUE
      END IF
*
*     Use unblocked code to reduce the rest of the matrix
*
      CALL CGEHD2( N, I, IHI, A, LDA, TAU, WORK, IINFO )
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of CGEHRD
*
      END
```

———-

34.9.4 lisp code

— **cgehrd.lisp** —

```
(let* ((nbmax 64)
       (ldt (+ nbmax 1))
       (tsize (* ldt nbmax))
       (zero (f2cl-lib:cmplx 0.0 0.0))
       (one (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:integer4 64 64) nbmax)
```

```
          (type (f2cl-lib:integer4) ldt)
          (type (f2cl-lib:integer4) tsize)
          (type (f2cl-lib:complex8) zero)
          (type (f2cl-lib:complex8) one)
          (ignorable nbmax ldt tsize zero one))
(defun cgehrd (n ilo ihi a lda tau work lwork info)
  (declare (type (array f2cl-lib:complex8 (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda ihi ilo n))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex8 a-%data% a-%offset%)
       (tau f2cl-lib:complex8 tau-%data% tau-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%))
    (prog ((ei #C(0.0 0.0)) (i 0) (ib 0) (iinfo 0) (iwt 0) (j 0) (ldwork 0)
           (lwkopt 0) (nb 0) (nbmin 0) (nh 0) (nx 0) (lquery nil))
      (declare (type (f2cl-lib:complex8) ei)
               (type (f2cl-lib:integer4) i ib iinfo iwt j ldwork lwkopt nb
                nbmin nh nx)
               (type f2cl-lib:logical lquery))
      (setf info 0)
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< n 0) (setf info -1))
            ((or (< ilo 1)
                 (> ilo
                    (max (the f2cl-lib:integer4 1)
                         (the f2cl-lib:integer4 n))))
             (setf info -2))
            ((or
              (< ihi
                 (min (the f2cl-lib:integer4 ilo) (the f2cl-lib:integer4 n)))
              (> ihi n))
             (setf info -3))
            ((< lda
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -5))
            ((and
              (< lwork
                 (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
              (not lquery))
             (setf info -8)))
      (cond
       ((= info 0)
        (setf nb
                (min (the f2cl-lib:integer4 nbmax)
                     (the f2cl-lib:integer4
                          (multiple-value-bind
                              (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                               var-6)
                              (ilaenv 1 "CGEHRD" " " n ilo ihi -1)
                            (declare (ignore var-0 var-1 var-2 var-6))
                            (when var-3 (setf n var-3))
```

```
                        (when var-4 (setf ilo var-4))
                        (when var-5 (setf ihi var-5))
                        ret-val))))
  (setf lwkopt (f2cl-lib:int-add (f2cl-lib:int-mul n nb) tsize))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwkopt 'f2cl-lib:complex8))))
(cond
 ((/= info 0) (xerbla "CGEHRD" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (setf (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%) zero)
   label10))
(f2cl-lib:fdo (i
               (max (the f2cl-lib:integer4 1)
                    (the f2cl-lib:integer4 ihi))
               (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (setf (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%) zero)
   label20))
(setf nh (f2cl-lib:int-add (f2cl-lib:int-sub ihi ilo) 1))
(cond
 ((<= nh 1)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce 1 'f2cl-lib:complex8))
  (go end_label)))
(setf nb
        (min (the f2cl-lib:integer4 nbmax)
             (the f2cl-lib:integer4
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                       var-6)
                      (ilaenv 1 "CGEHRD" " " n ilo ihi -1)
                    (declare (ignore var-0 var-1 var-2 var-6))
                    (when var-3 (setf n var-3))
                    (when var-4 (setf ilo var-4))
                    (when var-5 (setf ihi var-5))
                    ret-val))))
(setf nbmin 2)
(cond
 ((and (> nb 1) (< nb nh))
  (setf nx
          (max (the f2cl-lib:integer4 nb)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "CGEHRD" " " n ilo ihi -1)
```

```
                        (declare (ignore var-0 var-1 var-2 var-6))
                        (when var-3 (setf n var-3))
                        (when var-4 (setf ilo var-4))
                        (when var-5 (setf ihi var-5))
                        ret-val))))
  (cond
   ((< nx nh)
    (cond
     ((< lwork (f2cl-lib:int-add (f2cl-lib:int-mul n nb) tsize))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4
                             var-5 var-6)
                            (ilaenv 2 "CGEHRD" " " n ilo ihi -1)
                          (declare (ignore var-0 var-1 var-2 var-6))
                          (when var-3 (setf n var-3))
                          (when var-4 (setf ilo var-4))
                          (when var-5 (setf ihi var-5))
                          ret-val))))
      (cond
       ((>= lwork (f2cl-lib:int-add (f2cl-lib:int-mul n nbmin) tsize))
        (setf nb (the f2cl-lib:integer4 (truncate (- lwork tsize) n))))
       (t (setf nb 1)))))))))
(setf ldwork n)
(cond ((or (< nb nbmin) (>= nb nh)) (setf i ilo))
      (t (setf iwt (f2cl-lib:int-add 1 (f2cl-lib:int-mul n nb)))
       (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i nb))
                     ((> i
                         (f2cl-lib:int-add ihi (f2cl-lib:int-sub 1)
                                           (f2cl-lib:int-sub nx)))
                      nil)
         (tagbody
           (setf ib
                   (min (the f2cl-lib:integer4 nb)
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-sub ihi i))))
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                var-9)
               (clahr2 ihi i ib
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 i)
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i)
                                      ((1 *)) tau-%offset%)
                (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                      (iwt) ((1 *)) work-%offset%)
                ldt work ldwork)
```

```
  (declare (ignore var-3 var-5 var-6 var-8))
  (when var-0 (setf ihi var-0))
  (when var-1 (setf i var-1))
  (when var-2 (setf ib var-2))
  (when var-4 (setf lda var-4))
  (when var-7 (setf ldt var-7))
  (when var-9 (setf ldwork var-9)))
(setf ei
        (f2cl-lib:fref a-%data%
                       ((f2cl-lib:int-add i ib)
                        (f2cl-lib:int-sub
                         (f2cl-lib:int-add i ib) 1))
                       ((1 lda) (1 *)) a-%offset%))
(setf (f2cl-lib:fref a-%data%
                     ((f2cl-lib:int-add i ib)
                      (f2cl-lib:int-sub
                       (f2cl-lib:int-add i ib) 1))
                     ((1 lda) (1 *)) a-%offset%)
        one)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (cgemm "No transpose" "Conjugate transpose" ihi
     (f2cl-lib:int-add (f2cl-lib:int-sub ihi i ib) 1) ib
     (- one) work ldwork
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                           ((+ i ib) i) ((1 lda) (1 *))
                           a-%offset%)
     lda one
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                           (1 (f2cl-lib:int-add i ib))
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare
   (ignore var-0 var-1 var-3 var-5 var-6 var-8 var-11))
  (when var-2 (setf ihi var-2))
  (when var-4 (setf ib var-4))
  (when var-7 (setf ldwork var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf lda var-12)))
(setf (f2cl-lib:fref a-%data%
                     ((f2cl-lib:int-add i ib)
                      (f2cl-lib:int-sub
                       (f2cl-lib:int-add i ib) 1))
                     ((1 lda) (1 *)) a-%offset%)
        ei)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
```

```
    (ctrmm "Right" "Lower" "Conjugate transpose" "Unit" i
     (f2cl-lib:int-sub ib 1) one
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                           ((+ i 1) i) ((1 lda) (1 *))
                           a-%offset%)
     lda work ldwork)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9))
  (when var-4 (setf i var-4))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf ldwork var-10)))
(f2cl-lib:fdo (j 0 (f2cl-lib:int-add j 1))
              ((> j
                  (f2cl-lib:int-add ib
                                    (f2cl-lib:int-sub 2)))
               nil)
  (tagbody
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5)
        (caxpy i (- one)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                               ((+
                                 (f2cl-lib:int-mul ldwork j)
                                 1))
                               ((1 *)) work-%offset%)
         1
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               (1 (f2cl-lib:int-add i j 1))
                               ((1 lda) (1 *)) a-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4 var-5))
      (when var-0 (setf i var-0)))
   label30))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (clarfb "Left" "Conjugate transpose" "Forward"
     "Columnwise" (f2cl-lib:int-sub ihi i)
     (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1) ib
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                           ((+ i 1) i) ((1 lda) (1 *))
                           a-%offset%)
     lda
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           (iwt) ((1 *)) work-%offset%)
     ldt
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                           ((+ i 1) (f2cl-lib:int-add i ib))
                           ((1 lda) (1 *)) a-%offset%)
```

```
                      lda work ldwork)
                    (declare
                     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9
                      var-11 var-13))
                    (when var-6 (setf ib var-6))
                    (when var-8 (setf lda var-8))
                    (when var-10 (setf ldt var-10))
                    (when var-12 (setf lda var-12))
                    (when var-14 (setf ldwork var-14)))
                 label40))))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (cgehd2 n i ihi a lda tau work iinfo)
          (declare (ignore var-0 var-1 var-3 var-5 var-6))
          (setf ihi var-2)
          (setf lda var-4)
          (setf iinfo var-7))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce lwkopt 'f2cl-lib:complex8))
        (go end_label)
       end_label
        (return (values n ilo ihi nil lda nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgehrd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n
                                              fortran-to-lisp::ilo
                                              fortran-to-lisp::ihi nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::cgehd2))))
```

———-

34.10 cgelq2

34.10.1 Axiom unit tests

— **cgelq2.input** —

```
)set break resume
)sys rm -f cgelq2.output
)spool cgelq2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.10.2 Axiom help page

— **cgelq2.help** —

```
======================================================================
cgelq2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgelq2 (M, N, A, LDA, TAU, WORK, INFO)
           CGELQ2 computes the LQ factorization of a general rectangular
           matrix using an unblocked algorithm.

Function/Subroutine Documentation
   subroutine cgelq2 (integer M, integer N, complex, dimension( lda, * ) A,
       integer LDA, complex, dimension( * ) TAU, complex, dimension( * ) WORK,
       integer INFO)
       CGELQ2 computes the LQ factorization of a general rectangular matrix
       using an unblocked algorithm.
```

```
Purpose:
   CGELQ2 computes an LQ factorization of a complex m by n matrix A:
   A = L * Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the m by n matrix A.
   On exit, the elements on and below the diagonal of the array
   contain the m by min(m,n) lower trapezoidal matrix L (L is
   lower triangular if m <= n); the elements above the diagonal,
   with the array TAU, represent the unitary matrix Q as a
   product of elementary reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is COMPLEX array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is COMPLEX array, dimension (M)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

   The matrix Q is represented as a product of elementary reflectors

      Q = H(k)**H . . . H(2)**H H(1)**H, where k = min(m,n).

   Each H(i) has the form

      H(i) = I - tau * v * v**H

   where tau is a complex scalar, and v is a complex vector with
   v(1:i-1) = 0 and v(i) = 1; conjg(v(i+1:n)) is stored on exit in
```

A(i,i+1:n), and tau in TAU(i).

———-

34.10.3 fortran code

— **cgelq2.f** —

```
*  ===================================================================
      SUBROUTINE CGELQ2( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      COMPLEX            ONE
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      COMPLEX            ALPHA
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLACGV, CLARF, CLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
```

```
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGELQ2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = 1, K
*
*        Generate elementary reflector H(i) to annihilate A(i,i+1:n)
*
         CALL CLACGV( N-I+1, A( I, I ), LDA )
         ALPHA = A( I, I )
         CALL CLARFG( N-I+1, ALPHA, A( I, MIN( I+1, N ) ), LDA,
     $                TAU( I ) )
         IF( I.LT.M ) THEN
*
*           Apply H(i) to A(i+1:m,i:n) from the right
*
            A( I, I ) = ONE
            CALL CLARF( 'Right', M-I, N-I+1, A( I, I ), LDA, TAU( I ),
     $                  A( I+1, I ), LDA, WORK )
         END IF
         A( I, I ) = ALPHA
         CALL CLACGV( N-I+1, A( I, I ), LDA )
   10 CONTINUE
      RETURN
*
*     End of CGELQ2
*
      END
```

———-

34.10.4 lisp code

— **cgelq2.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (ignorable one))
  (defun cgelq2 (m n a lda tau work info)
    (declare (type (array f2cl-lib:complex8 (*)) work tau a)
             (type (f2cl-lib:integer4) info lda n m))
```

```
(f2cl-lib:with-multi-array-data
    ((a f2cl-lib:complex8 a-%data% a-%offset%)
     (tau f2cl-lib:complex8 tau-%data% tau-%offset%)
     (work f2cl-lib:complex8 work-%data% work-%offset%))
  (prog ((alpha #C(0.0 0.0)) (i 0) (k 0))
    (declare (type (f2cl-lib:complex8) alpha)
             (type (f2cl-lib:integer4) i k))
    (setf info 0)
    (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
          ((< lda
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
           (setf info -4)))
    (cond
     ((/= info 0) (xerbla "CGELQ2" (f2cl-lib:int-sub info))
      (go end_label)))
    (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i k) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2)
            (clacgv (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-1))
          (when var-2 (setf lda var-2)))
        (setf alpha
                (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (clarfg (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) alpha
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   (i
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add i 1))
                                     (the f2cl-lib:integer4 n)))
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i) ((1 *))
                                   tau-%offset%))
          (declare (ignore var-0 var-2 var-4))
          (when var-1 (setf alpha var-1))
          (when var-3 (setf lda var-3)))
        (cond
         ((< i m)
          (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                one)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (clarf "Right" (f2cl-lib:int-sub m i)
```

```
                  (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
                  (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                        ((1 lda) (1 *)) a-%offset%)
                  lda
                  (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i)
                                        ((1 *)) tau-%offset%)
                  (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 ((+ i 1) i)
                                        ((1 lda) (1 *)) a-%offset%)
                  lda work)
                (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8))
                (when var-4 (setf lda var-4))
                (when var-7 (setf lda var-7)))))
            (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                    alpha)
            (multiple-value-bind (var-0 var-1 var-2)
                (clacgv (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare (ignore var-0 var-1))
              (when var-2 (setf lda var-2)))
          label10))
       (go end_label)
      end_label
       (return (values nil nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgelq2
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(nil nil nil fortran-to-lisp::lda
                                             nil nil fortran-to-lisp::info)
                                           :calls 'nil)))
```

———-

34.11 cgelqf

34.11.1 Axiom unit tests

— **cgelqf.input** —

```
)set break resume
)sys rm -f cgelqf.output
)spool cgelqf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.11.2 Axiom help page

— **cgelqf.help** —

```
======================================================================
cgelqf examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgelqf (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           CGELQF

Function/Subroutine Documentation
   subroutine cgelqf (integer M, integer N, complex, dimension( lda, * ) A,
       integer LDA, complex, dimension( * ) TAU, complex, dimension( * ) WORK,
       integer LWORK, integer INFO)
       CGELQF

Purpose:
   CGELQF computes an LQ factorization of a complex M-by-N matrix A:
   A = L * Q.

Parameters:
```

```
M
  M is INTEGER
  The number of rows of the matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit, the elements on and below the diagonal of the array
  contain the m-by-min(m,n) lower trapezoidal matrix L (L is
  lower triangular if m <= n); the elements above the diagonal,
  with the array TAU, represent the unitary matrix Q as a
  product of elementary reflectors (see Further Details).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

TAU
  TAU is COMPLEX array, dimension (min(M,N))
  The scalar factors of the elementary reflectors (see Further Details).

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,M).
  For optimum performance LWORK >= M*NB, where NB is the
  optimal blocksize.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value

  The matrix Q is represented as a product of elementary reflectors

     Q = H(k)**H . . . H(2)**H H(1)**H, where k = min(m,n).
```

```
Each H(i) has the form

   H(i) = I - tau * v * v**H

where tau is a complex scalar, and v is a complex vector with
v(1:i-1) = 0 and v(i) = 1; conjg(v(i+1:n)) is stored on exit in
A(i,i+1:n), and tau in TAU(i).
```

34.11.3 fortran code

— cgelqf.f —

```
*  ===================================================================
      SUBROUTINE CGELQF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  ===================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, LDWORK, LWKOPT, NB,
     $                   NBMIN, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGELQ2, CLARFB, CLARFT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
```

```
*
*     Test the input arguments
*
      INFO = 0
      NB = ILAENV( 1, 'CGELQF', ' ', M, N, -1, -1 )
      LWKOPT = M*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, M ) .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGELQF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      K = MIN( M, N )
      IF( K.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      NX = 0
      IWS = M
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'CGELQF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = M
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
```

```
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'CGELQF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially
*
         DO 10 I = 1, K - NX, NB
            IB = MIN( K-I+1, NB )
*
*           Compute the LQ factorization of the current block
*           A(i:i+ib-1,i:n)
*
            CALL CGELQ2( IB, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                   IINFO )
            IF( I+IB.LE.M ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i) H(i+1) . . . H(i+ib-1)
*
               CALL CLARFT( 'Forward', 'Rowwise', N-I+1, IB, A( I, I ),
     $                      LDA, TAU( I ), WORK, LDWORK )
*
*              Apply H to A(i+ib:m,i:n) from the right
*
               CALL CLARFB( 'Right', 'No transpose', 'Forward',
     $                      'Rowwise', M-I-IB+1, N-I+1, IB, A( I, I ),
     $                      LDA, WORK, LDWORK, A( I+IB, I ), LDA,
     $                      WORK( IB+1 ), LDWORK )
            END IF
   10    CONTINUE
      ELSE
         I = 1
      END IF
*
*     Use unblocked code to factor the last or only block.
*
      IF( I.LE.K )
     $   CALL CGELQ2( M-I+1, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                IINFO )
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of CGELQF
```

```
*
      END
```

———-

34.11.4 lisp code

— **cgelqf.lisp** —

```
(defun cgelqf (m n a lda tau work lwork info)
  (declare (type (array f2cl-lib:complex8 (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex8 a-%data% a-%offset%)
       (tau f2cl-lib:complex8 tau-%data% tau-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ldwork 0) (lwkopt 0) (nb 0)
           (nbmin 0) (nx 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) nx nbmin nb lwkopt ldwork k iws iinfo
                ib i))
      (setf info 0)
      (setf nb
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 1 "CGELQF" " " m n -1 -1)
                (declare (ignore var-0 var-1 var-2 var-5 var-6))
                (when var-3 (setf m var-3))
                (when var-4 (setf n var-4))
                ret-val))
      (setf lwkopt (f2cl-lib:int-mul m nb))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce lwkopt 'f2cl-lib:complex8))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4))
            ((and
              (< lwork
                 (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
              (not lquery))
             (setf info -7)))
      (cond
       ((/= info 0) (xerbla "CGELQF" (f2cl-lib:int-sub info)) (go end_label))
       (lquery (go end_label)))
      (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
      (cond
       ((= k 0)
```

```
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce 1 'f2cl-lib:complex8))
  (go end_label)))
(setf nbmin 2)
(setf nx 0)
(setf iws m)
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "CGELQF" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (when var-3 (setf m var-3))
                      (when var-4 (setf n var-4))
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork m) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
                            (ilaenv 2 "CGELQF" " " m n -1 -1)
                          (declare
                           (ignore var-0 var-1 var-2 var-5 var-6))
                          (when var-3 (setf m var-3))
                          (when var-4 (setf n var-4))
                          ret-val)))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub nx))) nil)
    (tagbody
      (setf ib
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
               (the f2cl-lib:integer4 nb)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (cgelq2 ib (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                 ((1 lda) (1 *)) a-%offset%)
```

```
                lda
                (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i) ((1 *))
                                      tau-%offset%)
                work iinfo)
            (declare (ignore var-0 var-1 var-2 var-4 var-5))
            (setf lda var-3)
            (setf iinfo var-6))
          (cond
           ((<= (f2cl-lib:int-add i ib) m)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (clarft "Forward" "Rowwise"
                 (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) ib
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i)
                                       ((1 *)) tau-%offset%)
                 work ldwork)
              (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
              (when var-3 (setf ib var-3))
              (when var-5 (setf lda var-5))
              (when var-8 (setf ldwork var-8)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                 var-10 var-11 var-12 var-13 var-14)
                (clarfb "Right" "No transpose" "Forward" "Rowwise"
                 (f2cl-lib:int-add (f2cl-lib:int-sub m i ib) 1)
                 (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) ib
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda work ldwork
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                       ((+ i ib) i) ((1 lda) (1 *))
                                       a-%offset%)
                 lda
                 (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                       ((+ ib 1)) ((1 *)) work-%offset%)
                 ldwork)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
                var-13))
              (when var-6 (setf ib var-6))
              (when var-8 (setf lda var-8))
              (when var-10 (setf ldwork var-10))
              (when var-12 (setf lda var-12))
              (when var-14 (setf ldwork var-14)))))
      label10)))
  (t (setf i 1)))
 (if (<= i k)
```

```
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (cgelq2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
               (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i) ((1 *))
                                     tau-%offset%)
               work iinfo)
            (declare (ignore var-0 var-1 var-2 var-4 var-5))
            (setf lda var-3)
            (setf iinfo var-6)))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce iws 'f2cl-lib:complex8))
      (go end_label)
     end_label
      (return (values m n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgelqf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::cgelq2))))
```

———-

34.12 cgemqrt

34.12.1 Axiom unit tests

— cgemqrt.input —

```
)set break resume
)sys rm -f cgemqrt.output
)spool cgemqrt.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.12.2 Axiom help page

— cgemqrt.help —

```
======================================================================
cgemqrt examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgemqrt (SIDE, TRANS, M, N, K, NB, V, LDV, T, LDT, C, LDC,
           WORK, INFO)
           CGEMQRT

Function/Subroutine Documentation
   subroutine cgemqrt (character SIDE, character TRANS, integer M, integer N,
       integer K, integer NB, complex, dimension( ldv, * ) V, integer LDV,
       complex, dimension( ldt, * ) T, integer LDT, complex, dimension( ldc, *
       ) C, integer LDC, complex, dimension( * ) WORK, integer INFO)
       CGEMQRT

Purpose:
   CGEMQRT overwrites the general complex M-by-N matrix C with
```

```
                    SIDE = 'L'     SIDE = 'R'
    TRANS = 'N':      Q C            C Q
    TRANS = 'C':    Q**H C            C Q**H

  where Q is a complex orthogonal matrix defined as the product of K
  elementary reflectors:

     Q = H(1) H(2) . . . H(K) = I - V T V**H

  generated using the compact WY representation as returned by CGEQRT.

  Q is of order M if SIDE = 'L' and of order N  if SIDE = 'R'.

Parameters:

 SIDE
   SIDE is CHARACTER*1
   = 'L': apply Q or Q**H from the Left;
   = 'R': apply Q or Q**H from the Right.

 TRANS
   TRANS is CHARACTER*1
   = 'N':  No transpose, apply Q;
   = 'C':  Transpose, apply Q**H.

 M
   M is INTEGER
   The number of rows of the matrix C. M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix C. N >= 0.

 K
   K is INTEGER
   The number of elementary reflectors whose product defines
   the matrix Q.
   If SIDE = 'L', M >= K >= 0;
   if SIDE = 'R', N >= K >= 0.

 NB
   NB is INTEGER
   The block size used for the storage of T.  K >= NB >= 1.
   This must be the same value of NB used to generate T in CGEQRT.

 V
   V is COMPLEX array, dimension (LDV,K)
   The i-th column must contain the vector which defines the
   elementary reflector H(i), for i = 1,2,...,k, as returned by
   CGEQRT in the first K columns of its array argument A.
```

```
LDV
  LDV is INTEGER
  The leading dimension of the array V.
  If SIDE = 'L', LDA >= max(1,M);
  if SIDE = 'R', LDA >= max(1,N).

T
  T is COMPLEX array, dimension (LDT,K)
  The upper triangular factors of the block reflectors
  as returned by CGEQRT, stored as a NB-by-N matrix.

LDT
  LDT is INTEGER
  The leading dimension of the array T.  LDT >= NB.

C
  C is COMPLEX array, dimension (LDC,N)
  On entry, the M-by-N matrix C.
  On exit, C is overwritten by Q C, Q**H C, C Q**H or C Q.

LDC
  LDC is INTEGER
  The leading dimension of the array C. LDC >= max(1,M).

WORK
  WORK is COMPLEX array. The dimension of WORK is
  N*NB if SIDE = 'L', or  M*NB if SIDE = 'R'.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

34.12.3 fortran code

— cgemqrt.f —

```
*  =====================================================================
      SUBROUTINE CGEMQRT( SIDE, TRANS, M, N, K, NB, V, LDV, T, LDT,
     $                    C, LDC, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
```

```
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER SIDE, TRANS
      INTEGER   INFO, K, LDV, LDC, M, N, NB, LDT
*     ..
*     .. Array Arguments ..
      COMPLEX   V( LDV, * ), C( LDC, * ), T( LDT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     ..
*     .. Local Scalars ..
      LOGICAL            LEFT, RIGHT, TRAN, NOTRAN
      INTEGER            I, IB, LDWORK, KF, Q
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, CLARFB
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     .. Test the input arguments ..
*
      INFO   = 0
      LEFT   = LSAME( SIDE,  'L' )
      RIGHT  = LSAME( SIDE,  'R' )
      TRAN   = LSAME( TRANS, 'C' )
      NOTRAN = LSAME( TRANS, 'N' )
*
      IF( LEFT ) THEN
         LDWORK = MAX( 1, N )
         Q = M
      ELSE IF ( RIGHT ) THEN
         LDWORK = MAX( 1, M )
         Q = N
      END IF
      IF( .NOT.LEFT .AND. .NOT.RIGHT ) THEN
         INFO = -1
      ELSE IF( .NOT.TRAN .AND. .NOT.NOTRAN ) THEN
         INFO = -2
      ELSE IF( M.LT.0 ) THEN
```

```
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( K.LT.0 .OR. K.GT.Q ) THEN
         INFO = -5
      ELSE IF( NB.LT.1 .OR. (NB.GT.K .AND. K.GT.0)) THEN
         INFO = -6
      ELSE IF( LDV.LT.MAX( 1, Q ) ) THEN
         INFO = -8
      ELSE IF( LDT.LT.NB ) THEN
         INFO = -10
      ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
         INFO = -12
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEMQRT', -INFO )
         RETURN
      END IF
*
*     .. Quick return if possible ..
*
      IF( M.EQ.0 .OR. N.EQ.0 .OR. K.EQ.0 ) RETURN
*
      IF( LEFT .AND. TRAN ) THEN
*
         DO I = 1, K, NB
            IB = MIN( NB, K-I+1 )
            CALL CLARFB( 'L', 'C', 'F', 'C', M-I+1, N, IB,
     $                   V( I, I ), LDV, T( 1, I ), LDT,
     $                   C( I, 1 ), LDC, WORK, LDWORK )
         END DO
*
      ELSE IF( RIGHT .AND. NOTRAN ) THEN
*
         DO I = 1, K, NB
            IB = MIN( NB, K-I+1 )
            CALL CLARFB( 'R', 'N', 'F', 'C', M, N-I+1, IB,
     $                   V( I, I ), LDV, T( 1, I ), LDT,
     $                   C( 1, I ), LDC, WORK, LDWORK )
         END DO
*
      ELSE IF( LEFT .AND. NOTRAN ) THEN
*
         KF = ((K-1)/NB)*NB+1
         DO I = KF, 1, -NB
            IB = MIN( NB, K-I+1 )
            CALL CLARFB( 'L', 'N', 'F', 'C', M-I+1, N, IB,
     $                   V( I, I ), LDV, T( 1, I ), LDT,
     $                   C( I, 1 ), LDC, WORK, LDWORK )
```

```
         END DO
*
      ELSE IF( RIGHT .AND. TRAN ) THEN
*
         KF = ((K-1)/NB)*NB+1
         DO I = KF, 1, -NB
            IB = MIN( NB, K-I+1 )
            CALL CLARFB( 'R', 'C', 'F', 'C', M, N-I+1, IB,
     $                   V( I, I ), LDV, T( 1, I ), LDT,
     $                   C( 1, I ), LDC, WORK, LDWORK )
         END DO
*
      END IF
*
      RETURN
*
*     End of CGEMQRT
*
      END
```

———-

34.12.4 lisp code

— **cgemqrt.lisp** —

```
(defun cgemqrt (side trans m n k nb v ldv t$ ldt c ldc work info)
  (declare (type (array f2cl-lib:complex8 (*)) work c t$ v)
           (type (f2cl-lib:integer4) info ldc ldt ldv nb k n m)
           (type (string *) trans side))
  (f2cl-lib:with-multi-array-data
      ((side character side-%data% side-%offset%)
       (trans character trans-%data% trans-%offset%)
       (v f2cl-lib:complex8 v-%data% v-%offset%)
       (t$ f2cl-lib:complex8 t$-%data% t$-%offset%)
       (c f2cl-lib:complex8 c-%data% c-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (ldwork 0) (kf 0) (q 0) (left nil) (right nil)
           (tran nil) (notran nil))
      (declare (type f2cl-lib:logical notran tran right left)
               (type (f2cl-lib:integer4) q kf ldwork ib i))
      (setf info 0)
      (setf left
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame side "L")
                (declare (ignore var-1))
                (when var-0 (setf side var-0))
                ret-val))
```

```
(setf right
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "R")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val))
(setf tran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 (left
  (setf ldwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf q m))
 (right
  (setf ldwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf q n)))
(cond ((and (not left) (not right)) (setf info -1))
      ((and (not tran) (not notran)) (setf info -2))
      ((< m 0) (setf info -3)) ((< n 0) (setf info -4))
      ((or (< k 0) (> k q)) (setf info -5))
      ((or (< nb 1) (and (> nb k) (> k 0))) (setf info -6))
      ((< ldv (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 q)))
       (setf info -8))
      ((< ldt nb) (setf info -10))
      ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -12)))
(cond
 ((/= info 0) (xerbla "CGEMQRT" (f2cl-lib:int-sub info)) (go end_label)))
(if (or (= m 0) (= n 0) (= k 0))
    (go end_label))
(cond
 ((and left tran)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                ((> i k) nil)
    (tagbody
      (setf ib
              (min (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
          var-10 var-11 var-12 var-13 var-14)
         (clarfb "L" "C" "F" "C"
          (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) n ib
          (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (i i)
                                ((1 ldv) (1 *)) v-%offset%)
          ldv
          (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8 (1 i)
                                ((1 ldt) (1 *)) t$-%offset%)
          ldt
          (f2cl-lib:array-slice c-%data% f2cl-lib:complex8 (i 1)
                                ((1 ldc) (1 *)) c-%offset%)
          ldc work ldwork)
        (declare
         (ignore var-0 var-1 var-2 var-3 var-4 var-7 var-9 var-11
          var-13))
        (when var-5 (setf n var-5))
        (when var-6 (setf ib var-6))
        (when var-8 (setf ldv var-8))
        (when var-10 (setf ldt var-10))
        (when var-12 (setf ldc var-12))
        (when var-14 (setf ldwork var-14)))
    label100000)))
((and right notran)
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
               ((> i k) nil)
   (tagbody
     (setf ib
             (min (the f2cl-lib:integer4 nb)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12 var-13 var-14)
         (clarfb "R" "N" "F" "C" m
          (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) ib
          (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (i i)
                                ((1 ldv) (1 *)) v-%offset%)
          ldv
          (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8 (1 i)
                                ((1 ldt) (1 *)) t$-%offset%)
          ldt
          (f2cl-lib:array-slice c-%data% f2cl-lib:complex8 (1 i)
                                ((1 ldc) (1 *)) c-%offset%)
          ldc work ldwork)
        (declare
         (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9 var-11
          var-13))
        (when var-4 (setf m var-4))
        (when var-6 (setf ib var-6))
        (when var-8 (setf ldv var-8))
```

```
          (when var-10 (setf ldt var-10))
          (when var-12 (setf ldc var-12))
          (when var-14 (setf ldwork var-14)))
      label100001)))
  ((and left notran)
   (setf kf (+ (* (the f2cl-lib:integer4 (truncate (- k 1) nb)) nb) 1))
   (f2cl-lib:fdo (i kf (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
                 ((> i 1) nil)
     (tagbody
       (setf ib
               (min (the f2cl-lib:integer4 nb)
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10 var-11 var-12 var-13 var-14)
           (clarfb "L" "N" "F" "C"
            (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) n ib
            (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (i i)
                                  ((1 ldv) (1 *)) v-%offset%)
            ldv
            (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8 (1 i)
                                  ((1 ldt) (1 *)) t$-%offset%)
            ldt
            (f2cl-lib:array-slice c-%data% f2cl-lib:complex8 (i 1)
                                  ((1 ldc) (1 *)) c-%offset%)
            ldc work ldwork)
          (declare
           (ignore var-0 var-1 var-2 var-3 var-4 var-7 var-9 var-11
            var-13))
          (when var-5 (setf n var-5))
          (when var-6 (setf ib var-6))
          (when var-8 (setf ldv var-8))
          (when var-10 (setf ldt var-10))
          (when var-12 (setf ldc var-12))
          (when var-14 (setf ldwork var-14)))
      label100002)))
  ((and right tran)
   (setf kf (+ (* (the f2cl-lib:integer4 (truncate (- k 1) nb)) nb) 1))
   (f2cl-lib:fdo (i kf (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
                 ((> i 1) nil)
     (tagbody
       (setf ib
               (min (the f2cl-lib:integer4 nb)
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10 var-11 var-12 var-13 var-14)
           (clarfb "R" "C" "F" "C" m
```

```
                (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) ib
                (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (i i)
                                      ((1 ldv) (1 *)) v-%offset%)
                ldv
                (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8 (1 i)
                                      ((1 ldt) (1 *)) t$-%offset%)
                ldt
                (f2cl-lib:array-slice c-%data% f2cl-lib:complex8 (1 i)
                                      ((1 ldc) (1 *)) c-%offset%)
                ldc work ldwork)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9 var-11
                var-13))
              (when var-4 (setf m var-4))
              (when var-6 (setf ib var-6))
              (when var-8 (setf ldv var-8))
              (when var-10 (setf ldt var-10))
              (when var-12 (setf ldc var-12))
              (when var-14 (setf ldwork var-14)))
          label100003))))
      (go end_label)
     end_label
      (return
       (values side trans m n nil nil nil ldv nil ldt nil ldc nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgemqrt
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string) (string)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(fortran-to-lisp::side
```

```
                                        fortran-to-lisp::trans
                                        fortran-to-lisp::m
                                        fortran-to-lisp::n nil nil nil
                                        fortran-to-lisp::ldv nil
                                        fortran-to-lisp::ldt nil
                                        fortran-to-lisp::ldc nil
                                        fortran-to-lisp::info)
                                      :calls 'nil)))
```

———-

34.13 cgeql2

34.13.1 Axiom unit tests

— **cgeql2.input** —

```
)set break resume
)sys rm -f cgeql2.output
)spool cgeql2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.13.2 Axiom help page

— **cgeql2.help** —

```
====================================================================
cgeql2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
```

```
   Functions/Subroutines
       subroutine cgeql2 (M, N, A, LDA, TAU, WORK, INFO)
           CGEQL2 computes the QL factorization of a general rectangular
           matrix using an unblocked algorithm.

Function/Subroutine Documentation
   subroutine cgeql2 (integer M, integer N, complex, dimension( lda, * ) A,
       integer LDA, complex, dimension( * ) TAU, complex, dimension( * ) WORK,
       integer INFO)
       CGEQL2 computes the QL factorization of a general rectangular matrix
       using an unblocked algorithm.

Purpose:
   CGEQL2 computes a QL factorization of a complex m by n matrix A:
   A = Q * L.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the m by n matrix A.
   On exit, if m >= n, the lower triangle of the subarray
   A(m-n+1:m,1:n) contains the n by n lower triangular matrix L;
   if m <= n, the elements on and below the (n-m)-th
   superdiagonal contain the m by n lower trapezoidal matrix L;
   the remaining elements, with the array TAU, represent the
   unitary matrix Q as a product of elementary reflectors
   (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is COMPLEX array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is COMPLEX array, dimension (N)

 INFO
   INFO is INTEGER
```

```
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

 The matrix Q is represented as a product of elementary reflectors

    Q = H(k) . . . H(2) H(1), where k = min(m,n).

 Each H(i) has the form

    H(i) = I - tau * v * v**H

   where tau is a complex scalar, and v is a complex vector with
   v(m-k+i+1:m) = 0 and v(m-k+i) = 1; v(1:m-k+i-1) is stored on exit in
   A(1:m-k+i-1,n-k+i), and tau in TAU(i).
```

———-

34.13.3 fortran code

— cgeql2.f —

```
*  =====================================================================
      SUBROUTINE CGEQL2( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ONE
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      COMPLEX            ALPHA
*     ..
*     .. External Subroutines ..
```

```
      EXTERNAL           CLARF, CLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          CONJG, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEQL2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = K, 1, -1
*
*        Generate elementary reflector H(i) to annihilate
*        A(1:m-k+i-1,n-k+i)
*
         ALPHA = A( M-K+I, N-K+I )
         CALL CLARFG( M-K+I, ALPHA, A( 1, N-K+I ), 1, TAU( I ) )
*
*        Apply H(i)**H to A(1:m-k+i,1:n-k+i-1) from the left
*
         A( M-K+I, N-K+I ) = ONE
         CALL CLARF( 'Left', M-K+I, N-K+I-1, A( 1, N-K+I ), 1,
     $               CONJG( TAU( I ) ), A, LDA, WORK )
         A( M-K+I, N-K+I ) = ALPHA
   10 CONTINUE
      RETURN
*
*     End of CGEQL2
*
      END
```

34.13.4 lisp code

— cgeql2.lisp —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (ignorable one))
  (defun cgeql2 (m n a lda tau work info)
    (declare (type (array f2cl-lib:complex8 (*)) work tau a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (tau f2cl-lib:complex8 tau-%data% tau-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%))
      (prog ((alpha #C(0.0 0.0)) (i 0) (k 0))
        (declare (type (f2cl-lib:complex8) alpha)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "CGEQL2" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (f2cl-lib:fdo (i k (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (setf alpha
                    (f2cl-lib:fref a-%data%
                                   ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                    (f2cl-lib:int-add (f2cl-lib:int-sub n k)
                                                      i))
                                   ((1 lda) (1 *)) a-%offset%))
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (clarfg (f2cl-lib:int-add (f2cl-lib:int-sub m k) i) alpha
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                       (1
                                        (f2cl-lib:int-add
                                         (f2cl-lib:int-sub n k) i))
                                       ((1 lda) (1 *)) a-%offset%)
                 1
                 (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i) ((1 *))
                                       tau-%offset%))
              (declare (ignore var-0 var-2 var-3 var-4))
              (when var-1 (setf alpha var-1)))
            (setf (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
```

```
                                 (f2cl-lib:int-add (f2cl-lib:int-sub n k) i))
                                ((1 lda) (1 *)) a-%offset%)
                      one)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (clarf "Left" (f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                 (f2cl-lib:int-sub (f2cl-lib:int-add (f2cl-lib:int-sub n k) i)
                                   1)
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                       (1
                                        (f2cl-lib:int-add
                                         (f2cl-lib:int-sub n k) i))
                                       ((1 lda) (1 *)) a-%offset%)
                 1
                 (f2cl-lib:conjg
                  (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
                 a lda work)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
              (when var-7 (setf lda var-7)))
            (setf (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                  (f2cl-lib:int-add (f2cl-lib:int-sub n k) i))
                                 ((1 lda) (1 *)) a-%offset%)
                      alpha)
          label10))
        (go end_label)
       end_label
        (return (values nil nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgeql2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::lda
                                              nil nil fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

34.14 cgeqlf

34.14.1 Axiom unit tests

— **cgeqlf.input** —

```
)set break resume
)sys rm -f cgeqlf.output
)spool cgeqlf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.14.2 Axiom help page

— **cgeqlf.help** —

```
====================================================================
cgeqlf examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgeqlf (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           CGEQLF

Function/Subroutine Documentation
   subroutine cgeqlf (integer M, integer N, complex, dimension( lda, * ) A,
       integer LDA, complex, dimension( * ) TAU, complex, dimension( * ) WORK,
       integer LWORK, integer INFO)
```

```
       CGEQLF

Purpose:
   CGEQLF computes a QL factorization of a complex M-by-N matrix A:
   A = Q * L.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit,
   if m >= n, the lower triangle of the subarray
   A(m-n+1:m,1:n) contains the N-by-N lower triangular matrix L;
   if m <= n, the elements on and below the (n-m)-th
   superdiagonal contain the M-by-N lower trapezoidal matrix L;
   the remaining elements, with the array TAU, represent the
   unitary matrix Q as a product of elementary reflectors
   (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is COMPLEX array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is COMPLEX array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.  LWORK >= max(1,N).
   For optimum performance LWORK >= N*NB, where NB is
   the optimal blocksize.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.
```

```
INFO
INFO is INTEGER
= 0:  successful exit
< 0:  if INFO = -i, the i-th argument had an illegal value

The matrix Q is represented as a product of elementary reflectors

    Q = H(k) . . . H(2) H(1), where k = min(m,n).

Each H(i) has the form

    H(i) = I - tau * v * v**H

where tau is a complex scalar, and v is a complex vector with
v(m-k+i+1:m) = 0 and v(m-k+i) = 1; v(1:m-k+i-1) is stored on exit in
A(1:m-k+i-1,n-k+i), and tau in TAU(i).
```

34.14.3 fortran code

— cgeqlf.f —

```
*  ===================================================================
      SUBROUTINE CGEQLF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  ===================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, KI, KK, LDWORK, LWKOPT,
     $                   MU, NB, NBMIN, NU, NX
*     ..
*     .. External Subroutines ..
```

```
      EXTERNAL           CGEQL2, CLARFB, CLARFT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
*
      IF( INFO.EQ.0 ) THEN
         K = MIN( M, N )
         IF( K.EQ.0 ) THEN
            LWKOPT = 1
         ELSE
            NB = ILAENV( 1, 'CGEQLF', ' ', M, N, -1, -1 )
            LWKOPT = N*NB
         END IF
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
            INFO = -7
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEQLF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( K.EQ.0 ) THEN
         RETURN
      END IF
```

```
*
      NBMIN = 2
      NX = 1
      IWS = N
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'CGEQLF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = N
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'CGEQLF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially.
*        The last kk columns are handled by the block method.
*
         KI = ( ( K-NX-1 ) / NB )*NB
         KK = MIN( K, KI+NB )
*
         DO 10 I = K - KK + KI + 1, K - KK + 1, -NB
            IB = MIN( K-I+1, NB )
*
*           Compute the QL factorization of the current block
*           A(1:m-k+i+ib-1,n-k+i:n-k+i+ib-1)
*
            CALL CGEQL2( M-K+I+IB-1, IB, A( 1, N-K+I ), LDA, TAU( I ),
     $                   WORK, IINFO )
            IF( N-K+I.GT.1 ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i+ib-1) . . . H(i+1) H(i)
*
               CALL CLARFT( 'Backward', 'Columnwise', M-K+I+IB-1, IB,
     $                      A( 1, N-K+I ), LDA, TAU( I ), WORK, LDWORK )
```

```
*
*             Apply H**H to A(1:m-k+i+ib-1,1:n-k+i-1) from the left
*
              CALL CLARFB( 'Left', 'Conjugate transpose', 'Backward',
     $                     'Columnwise', M-K+I+IB-1, N-K+I-1, IB,
     $                     A( 1, N-K+I ), LDA, WORK, LDWORK, A, LDA,
     $                     WORK( IB+1 ), LDWORK )
           END IF
   10    CONTINUE
         MU = M - K + I + NB - 1
         NU = N - K + I + NB - 1
      ELSE
         MU = M
         NU = N
      END IF
*
*     Use unblocked code to factor the last or only block
*
      IF( MU.GT.0 .AND. NU.GT.0 )
     $   CALL CGEQL2( MU, NU, A, LDA, TAU, WORK, IINFO )
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of CGEQLF
*
      END
```

———-

34.14.4 lisp code

— **cgeqlf.lisp** —

```
(defun cgeqlf (m n a lda tau work lwork info)
  (declare (type (array f2cl-lib:complex8 (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex8 a-%data% a-%offset%)
       (tau f2cl-lib:complex8 tau-%data% tau-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ki 0) (kk 0) (ldwork 0)
           (lwkopt 0) (mu 0) (nb 0) (nbmin 0) (nu 0) (nx 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) nx nu nbmin nb mu lwkopt ldwork kk ki
                k iws iinfo ib i))
      (setf info 0)
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
```

```
(cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
      ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -4)))
(cond
 ((= info 0)
  (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
  (cond ((= k 0) (setf lwkopt 1))
        (t
         (setf nb
                 (multiple-value-bind
                     (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                     (ilaenv 1 "CGEQLF" " " m n -1 -1)
                   (declare (ignore var-0 var-1 var-2 var-5 var-6))
                   (when var-3 (setf m var-3))
                   (when var-4 (setf n var-4))
                   ret-val))
         (setf lwkopt (f2cl-lib:int-mul n nb))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwkopt 'f2cl-lib:complex8))
  (cond
   ((and
     (< lwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
     (not lquery))
    (setf info -7)))))
(cond
 ((/= info 0) (xerbla "CGEQLF" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond ((= k 0) (go end_label)))
(setf nbmin 2)
(setf nx 1)
(setf iws n)
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "CGEQLF" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (when var-3 (setf m var-3))
                      (when var-4 (setf n var-4))
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork n) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
```

```
                (max (the f2cl-lib:integer4 2)
                     (the f2cl-lib:integer4
                          (multiple-value-bind
                              (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                               var-6)
                              (ilaenv 2 "CGEQLF" " " m n -1 -1)
                            (declare
                             (ignore var-0 var-1 var-2 var-5 var-6))
                            (when var-3 (setf m var-3))
                            (when var-4 (setf n var-4))
                            ret-val)))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (setf ki (* (the f2cl-lib:integer4 (truncate (- k nx 1) nb)) nb))
  (setf kk
          (min (the f2cl-lib:integer4 k)
               (the f2cl-lib:integer4 (f2cl-lib:int-add ki nb))))
  (f2cl-lib:fdo (i (f2cl-lib:int-add k (f2cl-lib:int-sub kk) ki 1)
                 (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
                ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub kk) 1)) nil)
    (tagbody
      (setf ib
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
               (the f2cl-lib:integer4 nb)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (cgeql2
           (f2cl-lib:int-sub
            (f2cl-lib:int-add (f2cl-lib:int-sub m k) i ib) 1)
           ib
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 (1
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-sub n k) i))
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i) ((1 *))
                                 tau-%offset%)
           work iinfo)
        (declare (ignore var-0 var-1 var-2 var-4 var-5))
        (setf lda var-3)
        (setf iinfo var-6))
      (cond
       ((> (f2cl-lib:int-add n (f2cl-lib:int-sub k) i) 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (clarft "Backward" "Columnwise"
             (f2cl-lib:int-sub
              (f2cl-lib:int-add (f2cl-lib:int-sub m k) i ib) 1)
```

```
                ib
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                      (1
                                       (f2cl-lib:int-add
                                        (f2cl-lib:int-sub n k) i))
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i)
                                      ((1 *)) tau-%offset%)
                work ldwork)
              (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
              (when var-3 (setf ib var-3))
              (when var-5 (setf lda var-5))
              (when var-8 (setf ldwork var-8)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                 var-10 var-11 var-12 var-13 var-14)
                (clarfb "Left" "Conjugate transpose" "Backward" "Columnwise"
                 (f2cl-lib:int-sub
                  (f2cl-lib:int-add (f2cl-lib:int-sub m k) i ib) 1)
                 (f2cl-lib:int-sub
                  (f2cl-lib:int-add (f2cl-lib:int-sub n k) i) 1)
                 ib
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                       (1
                                        (f2cl-lib:int-add
                                         (f2cl-lib:int-sub n k) i))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda work ldwork a lda
                 (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                       ((+ ib 1)) ((1 *)) work-%offset%)
                 ldwork)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
                var-13))
              (when var-6 (setf ib var-6))
              (when var-8 (setf lda var-8))
              (when var-10 (setf ldwork var-10))
              (when var-12 (setf lda var-12))
              (when var-14 (setf ldwork var-14)))))
        label10))
      (setf mu
              (f2cl-lib:int-sub
               (f2cl-lib:int-add (f2cl-lib:int-sub m k) i nb) 1))
      (setf nu
              (f2cl-lib:int-sub
               (f2cl-lib:int-add (f2cl-lib:int-sub n k) i nb) 1)))
     (t (setf mu m) (setf nu n)))
    (if (and (> mu 0) (> nu 0))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
              (cgeql2 mu nu a lda tau work iinfo)
            (declare (ignore var-0 var-1 var-2 var-4 var-5))
            (setf lda var-3)
            (setf iinfo var-6)))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce iws 'f2cl-lib:complex8))
      (go end_label)
     end_label
      (return (values m n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgeqlf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::cgeql2))))
```

———-

34.15 cgeqp3

34.15.1 Axiom unit tests

— cgeqp3.input —

```
)set break resume
)sys rm -f cgeqp3.output
)spool cgeqp3.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.15.2 Axiom help page

— cgeqp3.help —

```
=====================================================================
cgeqp3 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgeqp3 (M, N, A, LDA, JPVT, TAU, WORK, LWORK, RWORK, INFO)
           CGEQP3

Function/Subroutine Documentation
   subroutine cgeqp3 (integer M, integer N, complex, dimension( lda, * ) A,
       integer LDA, integer, dimension( * ) JPVT, complex, dimension( * ) TAU,
       complex, dimension( * ) WORK, integer LWORK, real, dimension( * )
       RWORK, integer INFO)
       CGEQP3

Purpose:
   CGEQP3 computes a QR factorization with column pivoting of a
   matrix A:  A*P = Q*R  using Level 3 BLAS.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A. M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
```

```
  A is COMPLEX array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit, the upper triangle of the array contains the
  min(M,N)-by-N upper trapezoidal matrix R; the elements below
  the diagonal, together with the array TAU, represent the
  unitary matrix Q as a product of min(M,N) elementary reflectors.

LDA
  LDA is INTEGER
  The leading dimension of the array A. LDA >= max(1,M).

JPVT
  JPVT is INTEGER array, dimension (N)
  On entry, if JPVT(J).ne.0, the J-th column of A is permuted
  to the front of A*P (a leading column); if JPVT(J)=0,
  the J-th column of A is a free column.
  On exit, if JPVT(J)=K, then the J-th column of A*P was the
  the K-th column of A.

TAU
  TAU is COMPLEX array, dimension (min(M,N))
  The scalar factors of the elementary reflectors.

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO=0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK >= N+1.
  For optimal performance LWORK >= ( N+1 )*NB, where NB
  is the optimal blocksize.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is REAL array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0: successful exit.
  < 0: if INFO = -i, the i-th argument had an illegal value.

  The matrix Q is represented as a product of elementary reflectors

     Q = H(1) H(2) . . . H(k), where k = min(m,n).
```

```
Each H(i) has the form

   H(i) = I - tau * v * v**H

where tau is a complex scalar, and v is a real/complex vector
with v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in
A(i+1:m,i), and tau in TAU(i).
```

34.15.3 fortran code

— cgeqp3.f —

```
*  =====================================================================
      SUBROUTINE CGEQP3( M, N, A, LDA, JPVT, TAU, WORK, LWORK, RWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            JPVT( * )
      REAL               RWORK( * )
      COMPLEX            A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            INB, INBMIN, IXOVER
      PARAMETER          ( INB = 1, INBMIN = 2, IXOVER = 3 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            FJB, IWS, J, JB, LWKOPT, MINMN, MINWS, NA, NB,
     $                   NBMIN, NFXD, NX, SM, SMINMN, SN, TOPBMN
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEQRF, CLAQP2, CLAQPS, CSWAP, CUNMQR, XERBLA
*     ..
*     .. External Functions ..
```

```
      INTEGER            ILAENV
      REAL               SCNRM2
      EXTERNAL           ILAENV, SCNRM2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          INT, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test input arguments
*  ====================
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
*
      IF( INFO.EQ.0 ) THEN
         MINMN = MIN( M, N )
         IF( MINMN.EQ.0 ) THEN
            IWS = 1
            LWKOPT = 1
         ELSE
            IWS = N + 1
            NB = ILAENV( INB, 'CGEQRF', ' ', M, N, -1, -1 )
            LWKOPT = ( N + 1 )*NB
         END IF
         WORK( 1 ) = CMPLX( LWKOPT )
*
         IF( ( LWORK.LT.IWS ) .AND. .NOT.LQUERY ) THEN
            INFO = -8
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEQP3', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Move initial columns up front.
*
      NFXD = 1
      DO 10 J = 1, N
```

```
         IF( JPVT( J ).NE.0 ) THEN
            IF( J.NE.NFXD ) THEN
               CALL CSWAP( M, A( 1, J ), 1, A( 1, NFXD ), 1 )
               JPVT( J ) = JPVT( NFXD )
               JPVT( NFXD ) = J
            ELSE
               JPVT( J ) = J
            END IF
            NFXD = NFXD + 1
         ELSE
            JPVT( J ) = J
         END IF
   10 CONTINUE
      NFXD = NFXD - 1
*
*     Factorize fixed columns
*  =======================
*
*     Compute the QR factorization of fixed columns and update
*     remaining columns.
*
      IF( NFXD.GT.0 ) THEN
         NA = MIN( M, NFXD )
*CC      CALL CGEQR2( M, NA, A, LDA, TAU, WORK, INFO )
         CALL CGEQRF( M, NA, A, LDA, TAU, WORK, LWORK, INFO )
         IWS = MAX( IWS, INT( WORK( 1 ) ) )
         IF( NA.LT.N ) THEN
*CC         CALL CUNM2R( 'Left', 'Conjugate Transpose', M, N-NA,
*CC  $                   NA, A, LDA, TAU, A( 1, NA+1 ), LDA, WORK,
*CC  $                   INFO )
            CALL CUNMQR( 'Left', 'Conjugate Transpose', M, N-NA, NA, A,
     $                   LDA, TAU, A( 1, NA+1 ), LDA, WORK, LWORK,
     $                   INFO )
            IWS = MAX( IWS, INT( WORK( 1 ) ) )
         END IF
      END IF
*
*     Factorize free columns
*  ======================
*
      IF( NFXD.LT.MINMN ) THEN
*
         SM = M - NFXD
         SN = N - NFXD
         SMINMN = MINMN - NFXD
*
*        Determine the block size.
*
         NB = ILAENV( INB, 'CGEQRF', ' ', SM, SN, -1, -1 )
         NBMIN = 2
```

```
         NX = 0
*
         IF( ( NB.GT.1 ) .AND. ( NB.LT.SMINMN ) ) THEN
*
*           Determine when to cross over from blocked to unblocked code.
*
            NX = MAX( 0, ILAENV( IXOVER, 'CGEQRF', ' ', SM, SN, -1,
     $           -1 ) )
*
*
            IF( NX.LT.SMINMN ) THEN
*
               Determine if workspace is large enough for blocked code.
*
               MINWS = ( SN+1 )*NB
               IWS = MAX( IWS, MINWS )
               IF( LWORK.LT.MINWS ) THEN
*
                  Not enough workspace to use optimal NB: Reduce NB and
*                 determine the minimum value of NB.
*
                  NB = LWORK / ( SN+1 )
                  NBMIN = MAX( 2, ILAENV( INBMIN, 'CGEQRF', ' ', SM, SN,
     $                    -1, -1 ) )
*
*
               END IF
            END IF
         END IF
*
*        Initialize partial column norms. The first N elements of work
*        store the exact column norms.
*
         DO 20 J = NFXD + 1, N
            RWORK( J ) = SCNRM2( SM, A( NFXD+1, J ), 1 )
            RWORK( N+J ) = RWORK( J )
   20    CONTINUE
*
         IF( ( NB.GE.NBMIN ) .AND. ( NB.LT.SMINMN ) .AND.
     $       ( NX.LT.SMINMN ) ) THEN
*
*           Use blocked code initially.
*
            J = NFXD + 1
*
*           Compute factorization: while loop.
*
*
            TOPBMN = MINMN - NX
   30       CONTINUE
```

```
            IF( J.LE.TOPBMN ) THEN
               JB = MIN( NB, TOPBMN-J+1 )
*
*              Factorize JB columns among columns J:N.
*
               CALL CLAQPS( M, N-J+1, J-1, JB, FJB, A( 1, J ), LDA,
     $                      JPVT( J ), TAU( J ), RWORK( J ),
     $                      RWORK( N+J ), WORK( 1 ), WORK( JB+1 ),
     $                      N-J+1 )
*
               J = J + FJB
               GO TO 30
            END IF
         ELSE
            J = NFXD + 1
         END IF
*
*        Use unblocked code to factor the last or only block.
*
*
         IF( J.LE.MINMN )
     $      CALL CLAQP2( M, N-J+1, J-1, A( 1, J ), LDA, JPVT( J ),
     $                   TAU( J ), RWORK( J ), RWORK( N+J ), WORK( 1 ) )
*
      END IF
*
      WORK( 1 ) = CMPLX( LWKOPT )
      RETURN
*
*     End of CGEQP3
*
      END
```

———-

34.15.4 lisp code

— **cgeqp3.lisp** —

```
(let* ((inb 1) (inbmin 2) (ixover 3))
  (declare (type (f2cl-lib:integer4 1 1) inb)
           (type (f2cl-lib:integer4 2 2) inbmin)
           (type (f2cl-lib:integer4 3 3) ixover)
           (ignorable inb inbmin ixover))
  (defun cgeqp3 (m n a lda jpvt tau work lwork rwork info)
    (declare (type (array single-float (*)) rwork)
             (type (array f2cl-lib:integer4 (*)) jpvt)
             (type (array f2cl-lib:complex8 (*)) work tau a)
```

```
          (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex8 a-%data% a-%offset%)
       (tau f2cl-lib:complex8 tau-%data% tau-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%)
       (jpvt f2cl-lib:integer4 jpvt-%data% jpvt-%offset%)
       (rwork single-float rwork-%data% rwork-%offset%))
    (prog ((fjb 0) (iws 0) (j 0) (jb 0) (lwkopt 0) (minmn 0) (minws 0) (na 0)
           (nb 0) (nbmin 0) (nfxd 0) (nx 0) (sm 0) (sminmn 0) (sn 0)
           (topbmn 0) (lquery nil))
      (declare
       (type (f2cl-lib:integer4) fjb iws j jb lwkopt minmn minws na nb nbmin
        nfxd nx sm sminmn sn topbmn)
       (type f2cl-lib:logical lquery))
      (setf info 0)
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4)))
      (cond
       ((= info 0)
        (setf minmn
                (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (cond ((= minmn 0) (setf iws 1) (setf lwkopt 1))
              (t (setf iws (f2cl-lib:int-add n 1))
               (setf nb
                       (multiple-value-bind
                           (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                            var-6)
                           (ilaenv inb "CGEQRF" " " m n -1 -1)
                         (declare (ignore var-1 var-2 var-5 var-6))
                         (when var-0 (setf inb var-0))
                         (when var-3 (setf m var-3))
                         (when var-4 (setf n var-4))
                         ret-val))
               (setf lwkopt (f2cl-lib:int-mul (f2cl-lib:int-add n 1) nb))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (f2cl-lib:cmplx lwkopt))
        (cond ((and (< lwork iws) (not lquery)) (setf info -8)))))
      (cond
       ((/= info 0) (xerbla "CGEQP3" (f2cl-lib:int-sub info)) (go end_label))
       (lquery (go end_label)))
      (setf nfxd 1)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (cond
           ((/= (f2cl-lib:fref jpvt (j) ((1 *))) 0)
            (cond
```

```
        ((/= j nfxd)
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (cswap m
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 j)
                                    ((1 lda) (1 *)) a-%offset%)
              1
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 nfxd)
                                    ((1 lda) (1 *)) a-%offset%)
              1)
           (declare (ignore var-1 var-2 var-3 var-4))
           (when var-0 (setf m var-0)))
         (setf (f2cl-lib:fref jpvt-%data% (j) ((1 *)) jpvt-%offset%)
                 (f2cl-lib:fref jpvt-%data% (nfxd) ((1 *))
                                jpvt-%offset%))
         (setf (f2cl-lib:fref jpvt-%data% (nfxd) ((1 *)) jpvt-%offset%)
                 j))
        (t
         (setf (f2cl-lib:fref jpvt-%data% (j) ((1 *)) jpvt-%offset%)
                 j)))
       (setf nfxd (f2cl-lib:int-add nfxd 1)))
      (t
       (setf (f2cl-lib:fref jpvt-%data% (j) ((1 *)) jpvt-%offset%) j)))
   label10))
(setf nfxd (f2cl-lib:int-sub nfxd 1))
(cond
 ((> nfxd 0)
  (setf na
          (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 nfxd)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (cgeqrf m na a lda tau work lwork info)
    (declare (ignore var-2 var-4 var-5))
    (when var-0 (setf m var-0))
    (when var-1 (setf na var-1))
    (when var-3 (setf lda var-3))
    (when var-6 (setf lwork var-6))
    (when var-7 (setf info var-7)))
  (setf iws
          (max (the f2cl-lib:integer4 iws)
               (the f2cl-lib:integer4
                    (f2cl-lib:int
                     (f2cl-lib:fref work-%data% (1) ((1 *))
                                    work-%offset%)))))
  (cond
   ((< na n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (cunmqr "Left" "Conjugate Transpose" m (f2cl-lib:int-sub n na)
         na a lda tau
```

```
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                (1 (f2cl-lib:int-add na 1))
                                ((1 lda) (1 *)) a-%offset%)
          lda work lwork info)
        (declare (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10))
        (when var-2 (setf m var-2))
        (when var-4 (setf na var-4))
        (when var-6 (setf lda var-6))
        (when var-9 (setf lda var-9))
        (when var-11 (setf lwork var-11))
        (when var-12 (setf info var-12)))
      (setf iws
              (max (the f2cl-lib:integer4 iws)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int
                         (f2cl-lib:fref work-%data% (1) ((1 *))
                                        work-%offset%)))))))))
(cond
 ((< nfxd minmn) (setf sm (f2cl-lib:int-sub m nfxd))
  (setf sn (f2cl-lib:int-sub n nfxd))
  (setf sminmn (f2cl-lib:int-sub minmn nfxd))
  (setf nb
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (ilaenv inb "CGEQRF" " " sm sn -1 -1)
            (declare (ignore var-1 var-2 var-5 var-6))
            (when var-0 (setf inb var-0))
            (when var-3 (setf sm var-3))
            (when var-4 (setf sn var-4))
            ret-val))
  (setf nbmin 2) (setf nx 0)
  (cond
   ((and (> nb 1) (< nb sminmn))
    (setf nx
            (max (the f2cl-lib:integer4 0)
                 (the f2cl-lib:integer4
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
                          (ilaenv ixover "CGEQRF" " " sm sn -1 -1)
                        (declare (ignore var-1 var-2 var-5 var-6))
                        (when var-0 (setf ixover var-0))
                        (when var-3 (setf sm var-3))
                        (when var-4 (setf sn var-4))
                        ret-val))))
    (cond
     ((< nx sminmn)
      (setf minws (f2cl-lib:int-mul (f2cl-lib:int-add sn 1) nb))
      (setf iws
              (max (the f2cl-lib:integer4 iws)
```

```
                    (the f2cl-lib:integer4 minws)))
      (cond
       ((< lwork minws)
        (setf nb (the f2cl-lib:integer4 (truncate lwork (+ sn 1))))
        (setf nbmin
                (max (the f2cl-lib:integer4 2)
                     (the f2cl-lib:integer4
                          (multiple-value-bind
                              (ret-val var-0 var-1 var-2 var-3 var-4
                               var-5 var-6)
                              (ilaenv inbmin "CGEQRF" " " sm sn -1 -1)
                            (declare (ignore var-1 var-2 var-5 var-6))
                            (when var-0 (setf inbmin var-0))
                            (when var-3 (setf sm var-3))
                            (when var-4 (setf sn var-4))
                            ret-val)))))))))
(f2cl-lib:fdo (j (f2cl-lib:int-add nfxd 1) (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (j) ((1 *)) rwork-%offset%)
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (scnrm2 sm
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                       ((+ nfxd 1) j) ((1 lda) (1 *))
                                       a-%offset%)
                 1)
                (declare (ignore var-1 var-2))
                (when var-0 (setf sm var-0))
                ret-val))
    (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add n j))
                         ((1 *)) rwork-%offset%)
            (f2cl-lib:fref rwork-%data% (j) ((1 *)) rwork-%offset%))
   label20))
(cond
 ((and (>= nb nbmin) (< nb sminmn) (< nx sminmn))
  (tagbody
    (setf j (f2cl-lib:int-add nfxd 1))
    (setf topbmn (f2cl-lib:int-sub minmn nx))
   label30
    (cond
     ((<= j topbmn)
      (setf jb
              (min (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub topbmn j)
                                          1))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12 var-13)
          (claqps m (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1)
```

```
                (f2cl-lib:int-sub j 1) jb fjb
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 j)
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice jpvt-%data% f2cl-lib:integer4 (j)
                                      ((1 *)) jpvt-%offset%)
                (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (j)
                                      ((1 *)) tau-%offset%)
                (f2cl-lib:array-slice rwork-%data% single-float (j)
                                      ((1 *)) rwork-%offset%)
                (f2cl-lib:array-slice rwork-%data% single-float ((+ n j))
                                      ((1 *)) rwork-%offset%)
                (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (1)
                                      ((1 *)) work-%offset%)
                (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                      ((+ jb 1)) ((1 *)) work-%offset%)
                (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1))
              (declare
               (ignore var-1 var-2 var-5 var-7 var-8 var-9 var-10 var-11
                var-12 var-13))
              (when var-0 (setf m var-0))
              (when var-3 (setf jb var-3))
              (when var-4 (setf fjb var-4))
              (when var-6 (setf lda var-6)))
            (setf j (f2cl-lib:int-add j fjb)) (go label30)))))
     (t (setf j (f2cl-lib:int-add nfxd 1))))
    (if (<= j minmn)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
            (claqp2 m (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1)
             (f2cl-lib:int-sub j 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice jpvt-%data% f2cl-lib:integer4 (j)
                                   ((1 *)) jpvt-%offset%)
             (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (j)
                                   ((1 *)) tau-%offset%)
             (f2cl-lib:array-slice rwork-%data% single-float (j) ((1 *))
                                   rwork-%offset%)
             (f2cl-lib:array-slice rwork-%data% single-float ((+ n j))
                                   ((1 *)) rwork-%offset%)
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (1)
                                   ((1 *)) work-%offset%))
          (declare
           (ignore var-1 var-2 var-3 var-5 var-6 var-7 var-8 var-9))
          (when var-0 (setf m var-0))
          (when var-4 (setf lda var-4))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:cmplx lwkopt))
```

```
        (go end_label)
       end_label
        (return (values m n nil lda nil nil nil lwork nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgeqp3
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::lwork nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

34.16 cgeqpf - DEPRECATED use cgeqp3

34.17 cgeqr2

34.17.1 Axiom unit tests

— **cgeqr2.input** —

```
)set break resume
)sys rm -f cgeqr2.output
```

```
)spool cgeqr2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.17.2 Axiom help page

— **cgeqr2.help** —

```
==================================================================
cgeqr2 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgeqr2 (M, N, A, LDA, TAU, WORK, INFO)
           CGEQR2 computes the QR factorization of a general rectangular
           matrix using an unblocked algorithm.

Function/Subroutine Documentation
   subroutine cgeqr2 (integer M, integer N, complex, dimension( lda, * ) A,
       integer LDA, complex, dimension( * ) TAU, complex, dimension( * ) WORK,
       integer INFO)
       CGEQR2 computes the QR factorization of a general rectangular matrix
       using an unblocked algorithm.

Purpose:
   CGEQR2 computes a QR factorization of a complex m by n matrix A:
   A = Q * R.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.
```

```
A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the m by n matrix A.
  On exit, the elements on and above the diagonal of the array
  contain the min(m,n) by n upper trapezoidal matrix R (R is
  upper triangular if m >= n); the elements below the diagonal,
  with the array TAU, represent the unitary matrix Q as a
  product of elementary reflectors (see Further Details).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

TAU
  TAU is COMPLEX array, dimension (min(M,N))
  The scalar factors of the elementary reflectors (see Further Details).

WORK
  WORK is COMPLEX array, dimension (N)

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value

The matrix Q is represented as a product of elementary reflectors

   Q = H(1) H(2) . . . H(k), where k = min(m,n).

Each H(i) has the form

   H(i) = I - tau * v * v**H

  where tau is a complex scalar, and v is a complex vector with
  v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in A(i+1:m,i),
  and tau in TAU(i).
```

———-

34.17.3 fortran code

— cgeqr2.f —

```
*  ===================================================================
      SUBROUTINE CGEQR2( M, N, A, LDA, TAU, WORK, INFO )
*
```

```
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ONE
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      COMPLEX            ALPHA
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLARF, CLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          CONJG, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEQR2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = 1, K
*
*        Generate elementary reflector H(i) to annihilate A(i+1:m,i)
```

```
*
         CALL CLARFG( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
     $               TAU( I ) )
         IF( I.LT.N ) THEN
*
*           Apply H(i)**H to A(i:m,i+1:n) from the left
*
            ALPHA = A( I, I )
            A( I, I ) = ONE
            CALL CLARF( 'Left', M-I+1, N-I, A( I, I ), 1,
     $                  CONJG( TAU( I ) ), A( I, I+1 ), LDA, WORK )
            A( I, I ) = ALPHA
         END IF
   10 CONTINUE
      RETURN
*
*     End of CGEQR2
*
      END
```

———-

34.17.4 lisp code

— **cgeqr2.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (ignorable one))
  (defun cgeqr2 (m n a lda tau work info)
    (declare (type (array f2cl-lib:complex8 (*)) work tau a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (tau f2cl-lib:complex8 tau-%data% tau-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%))
      (prog ((alpha #C(0.0 0.0)) (i 0) (k 0))
        (declare (type (f2cl-lib:complex8) alpha)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "CGEQR2" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
```

```
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i k) nil)
          (tagbody
            (clarfg (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   ((min (f2cl-lib:int-add i 1) m) i)
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i) ((1 *))
                                   tau-%offset%))
            (cond
             ((< i n)
              (setf alpha
                      (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                      one)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                  (clarf "Left" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
                   (f2cl-lib:int-sub n i)
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                         ((1 lda) (1 *)) a-%offset%)
                   1
                   (f2cl-lib:conjg
                    (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                         (i (f2cl-lib:int-add i 1))
                                         ((1 lda) (1 *)) a-%offset%)
                   lda work)
                 (declare
                  (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
                 (when var-7 (setf lda var-7)))
              (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                      alpha)))
           label10))
        (go end_label)
       end_label
        (return (values nil nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgeqr2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
```

```
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil fortran-to-lisp::lda
  nil nil fortran-to-lisp::info)
:calls 'nil)))
```

———-

34.18 cgeqr2p

34.18.1 Axiom unit tests

— **cgeqr2p.input** —

```
)set break resume
)sys rm -f cgeqr2p.output
)spool cgeqr2p.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.18.2 Axiom help page [Demm08]

— **cgeqr2p.help** —

```
==========================================================================
cgeqr2p examples
==========================================================================

==========================================================================
```

```
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgeqr2p (M, N, A, LDA, TAU, WORK, INFO)
           CGEQR2P computes the QR factorization of a general rectangular
           matrix with non-negative diagonal elements using an unblocked
           algorithm.

Function/Subroutine Documentation
   subroutine cgeqr2p (integer M, integer N, complex, dimension( lda, * ) A,
       integer LDA, complex, dimension( * ) TAU, complex, dimension( * ) WORK,
       integer INFO)
       CGEQR2P computes the QR factorization of a general rectangular matrix
       with non-negative diagonal elements using an unblocked algorithm.

Purpose:
   CGEQR2P computes a QR factorization of a complex m by n matrix A:
   A = Q * R. The diagonal entries of R are real and nonnegative.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the m by n matrix A.
   On exit, the elements on and above the diagonal of the array
   contain the min(m,n) by n upper trapezoidal matrix R (R is
   upper triangular if m >= n). The diagonal entries of R are
   real and nonnegative; the elements below the diagonal,
   with the array TAU, represent the unitary matrix Q as a
   product of elementary reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is COMPLEX array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
```

```
  WORK is COMPLEX array, dimension (N)

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value

  The matrix Q is represented as a product of elementary reflectors

      Q = H(1) H(2) . . . H(k), where k = min(m,n).

  Each H(i) has the form

     H(i) = I - tau * v * v**H

  where tau is a complex scalar, and v is a complex vector with
  v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in A(i+1:m,i),
  and tau in TAU(i).
```

34.18.3 fortran code

— cgeqr2p.f —

```
*  =====================================================================
      SUBROUTINE CGEQR2P( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ONE
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
```

```
      INTEGER            I, K
      COMPLEX            ALPHA
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLARF, CLARFGP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          CONJG, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEQR2P', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = 1, K
*
*        Generate elementary reflector H(i) to annihilate A(i+1:m,i)
*
         CALL CLARFGP( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
     $                TAU( I ) )
         IF( I.LT.N ) THEN
*
*           Apply H(i)**H to A(i:m,i+1:n) from the left
*
            ALPHA = A( I, I )
            A( I, I ) = ONE
            CALL CLARF( 'Left', M-I+1, N-I, A( I, I ), 1,
     $                  CONJG( TAU( I ) ), A( I, I+1 ), LDA, WORK )
            A( I, I ) = ALPHA
         END IF
   10 CONTINUE
      RETURN
*
*     End of CGEQR2P
*
      END
```

———-

34.18.4 lisp code

— **cgeqr2p.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (ignorable one))
  (defun cgeqr2p (m n a lda tau work info)
    (declare (type (array f2cl-lib:complex8 (*)) work tau a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (tau f2cl-lib:complex8 tau-%data% tau-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%))
      (prog ((alpha #C(0.0 0.0)) (i 0) (k 0))
        (declare (type (f2cl-lib:complex8) alpha)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "CGEQR2P" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i k) nil)
          (tagbody
            (clarfgp (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   ((min (f2cl-lib:int-add i 1) m) i)
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i) ((1 *))
                                   tau-%offset%))
            (cond
             ((< i n)
              (setf alpha
                      (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                      one)
              (multiple-value-bind
```

```
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                  (clarf "Left" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
                   (f2cl-lib:int-sub n i)
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                         ((1 lda) (1 *)) a-%offset%)
                   1
                   (f2cl-lib:conjg
                    (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                         (i (f2cl-lib:int-add i 1))
                                         ((1 lda) (1 *)) a-%offset%)
                   lda work)
                 (declare
                  (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
                 (when var-7 (setf lda var-7)))
               (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                     alpha)))
           label10))
        (go end_label)
       end_label
        (return (values nil nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgeqr2p
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::lda
                                              nil nil fortran-to-lisp::info)
                                            :calls 'nil)))
```

34.19 cgeqrf

34.19.1 Axiom unit tests

— cgeqrf.input —

```
)set break resume
)sys rm -f cgeqrf.output
)spool cgeqrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.19.2 Axiom help page

— cgeqrf.help —

```
======================================================================
cgeqrf examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgeqrf (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           CGEQRF

Function/Subroutine Documentation
   subroutine cgeqrf (integer M, integer N, complex, dimension( lda, * ) A,
       integer LDA, complex, dimension( * ) TAU, complex, dimension( * ) WORK,
       integer LWORK, integer INFO)
       CGEQRF

Purpose:
   CGEQRF computes a QR factorization of a complex M-by-N matrix A:
   A = Q * R.

Parameters:
```

```
M
  M is INTEGER
  The number of rows of the matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit, the elements on and above the diagonal of the array
  contain the min(M,N)-by-N upper trapezoidal matrix R (R is
  upper triangular if m >= n); the elements below the diagonal,
  with the array TAU, represent the unitary matrix Q as a
  product of min(m,n) elementary reflectors (see Further Details).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

TAU
  TAU is COMPLEX array, dimension (min(M,N))
  The scalar factors of the elementary reflectors (see Further Details).

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,N).
  For optimum performance LWORK >= N*NB, where NB is
  the optimal blocksize.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value

  The matrix Q is represented as a product of elementary reflectors

     Q = H(1) H(2) . . . H(k), where k = min(m,n).
```

```
Each H(i) has the form

   H(i) = I - tau * v * v**H

where tau is a complex scalar, and v is a complex vector with
v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in A(i+1:m,i),
and tau in TAU(i).
```

34.19.3 fortran code

— cgeqrf.f —

```
*  =====================================================================
      SUBROUTINE CGEQRF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, LDWORK, LWKOPT, NB,
     $                   NBMIN, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEQR2, CLARFB, CLARFT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
```

```
*
*     Test the input arguments
*
      INFO = 0
      NB = ILAENV( 1, 'CGEQRF', ' ', M, N, -1, -1 )
      LWKOPT = N*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEQRF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      K = MIN( M, N )
      IF( K.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      NX = 0
      IWS = N
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'CGEQRF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = N
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
```

```
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'CGEQRF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially
*
         DO 10 I = 1, K - NX, NB
            IB = MIN( K-I+1, NB )
*
            Compute the QR factorization of the current block
            A(i:m,i:i+ib-1)
*
            CALL CGEQR2( M-I+1, IB, A( I, I ), LDA, TAU( I ), WORK,
     $                   IINFO )
            IF( I+IB.LE.N ) THEN
*
               Form the triangular factor of the block reflector
               H = H(i) H(i+1) . . . H(i+ib-1)
*
               CALL CLARFT( 'Forward', 'Columnwise', M-I+1, IB,
     $                      A( I, I ), LDA, TAU( I ), WORK, LDWORK )
*
               Apply H**H to A(i:m,i+ib:n) from the left
*
               CALL CLARFB( 'Left', 'Conjugate transpose', 'Forward',
     $                      'Columnwise', M-I+1, N-I-IB+1, IB,
     $                      A( I, I ), LDA, WORK, LDWORK, A( I, I+IB ),
     $                      LDA, WORK( IB+1 ), LDWORK )
            END IF
   10    CONTINUE
      ELSE
         I = 1
      END IF
*
      Use unblocked code to factor the last or only block.
*
      IF( I.LE.K )
     $   CALL CGEQR2( M-I+1, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                IINFO )
*
      WORK( 1 ) = IWS
      RETURN
*
      End of CGEQRF
```

```
*
      END
```

———-

34.19.4 lisp code

— **cgeqrf.lisp** —

```
(defun cgeqrf (m n a lda tau work lwork info)
  (declare (type (array f2cl-lib:complex8 (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex8 a-%data% a-%offset%)
       (tau f2cl-lib:complex8 tau-%data% tau-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ldwork 0) (lwkopt 0) (nb 0)
           (nbmin 0) (nx 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) nx nbmin nb lwkopt ldwork k iws iinfo
                ib i))
      (setf info 0)
      (setf nb
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 1 "CGEQRF" " " m n -1 -1)
                (declare (ignore var-0 var-1 var-2 var-5 var-6))
                (when var-3 (setf m var-3))
                (when var-4 (setf n var-4))
                ret-val))
      (setf lwkopt (f2cl-lib:int-mul n nb))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce lwkopt 'f2cl-lib:complex8))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4))
            ((and
              (< lwork
                 (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
              (not lquery))
             (setf info -7)))
      (cond
       ((/= info 0) (xerbla "CGEQRF" (f2cl-lib:int-sub info)) (go end_label))
       (lquery (go end_label)))
      (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
      (cond
       ((= k 0)
```

```
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce 1 'f2cl-lib:complex8))
  (go end_label)))
(setf nbmin 2)
(setf nx 0)
(setf iws n)
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "CGEQRF" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (when var-3 (setf m var-3))
                      (when var-4 (setf n var-4))
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork n) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
                            (ilaenv 2 "CGEQRF" " " m n -1 -1)
                          (declare
                           (ignore var-0 var-1 var-2 var-5 var-6))
                          (when var-3 (setf m var-3))
                          (when var-4 (setf n var-4))
                          ret-val))))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub nx))) nil)
    (tagbody
      (setf ib
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
               (the f2cl-lib:integer4 nb)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (cgeqr2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                 ((1 lda) (1 *)) a-%offset%)
```

```
            lda
            (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i) ((1 *))
                                  tau-%offset%)
            work iinfo)
          (declare (ignore var-0 var-1 var-2 var-4 var-5))
          (setf lda var-3)
          (setf iinfo var-6))
        (cond
         ((<= (f2cl-lib:int-add i ib) n)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (clarft "Forward" "Columnwise"
               (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i)
                                     ((1 *)) tau-%offset%)
               work ldwork)
            (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
            (when var-3 (setf ib var-3))
            (when var-5 (setf lda var-5))
            (when var-8 (setf ldwork var-8)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10 var-11 var-12 var-13 var-14)
              (clarfb "Left" "Conjugate transpose" "Forward" "Columnwise"
               (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
               (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1) ib
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               lda work ldwork
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                     (i (f2cl-lib:int-add i ib))
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                     ((+ ib 1)) ((1 *)) work-%offset%)
               ldwork)
            (declare
             (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
              var-13))
            (when var-6 (setf ib var-6))
            (when var-8 (setf lda var-8))
            (when var-10 (setf ldwork var-10))
            (when var-12 (setf lda var-12))
            (when var-14 (setf ldwork var-14)))))
      label10)))
  (t (setf i 1)))
 (if (<= i k)
```

```
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (cgeqr2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
               (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i) ((1 *))
                                     tau-%offset%)
               work iinfo)
            (declare (ignore var-0 var-1 var-2 var-4 var-5))
            (setf lda var-3)
            (setf iinfo var-6)))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce iws 'f2cl-lib:complex8))
      (go end_label)
     end_label
      (return (values m n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgeqrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::cgeqr2))))
```

———-

34.20 cgeqrfp

34.20.1 Axiom unit tests

— cgeqrfp.input —

```
)set break resume
)sys rm -f cgeqrfp.output
)spool cgeqrfp.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.20.2 Axiom help page [Demm08]

— cgeqrfp.help —

```
======================================================================
cgeqrfp examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgeqrfp (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           CGEQRFP

Function/Subroutine Documentation
   subroutine cgeqrfp (integer M, integer N, complex, dimension( lda, * ) A,
       integer LDA, complex, dimension( * ) TAU, complex, dimension( * ) WORK,
       integer LWORK, integer INFO)
       CGEQRFP

Purpose:
   CGEQRFP computes a QR factorization of a complex M-by-N matrix A:
   A = Q * R. The diagonal entries of R are real and nonnegative.

Parameters:
```

```
M
  M is INTEGER
  The number of rows of the matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit, the elements on and above the diagonal of the array
  contain the min(M,N)-by-N upper trapezoidal matrix R (R is
  upper triangular if m >= n). The diagonal entries of R
  are real and nonnegative; the elements below the diagonal,
  with the array TAU, represent the unitary matrix Q as a
  product of min(m,n) elementary reflectors (see Further Details).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

TAU
  TAU is COMPLEX array, dimension (min(M,N))
  The scalar factors of the elementary reflectors (see Further Details).

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,N).
  For optimum performance LWORK >= N*NB, where NB is
  the optimal blocksize.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value

  The matrix Q is represented as a product of elementary reflectors

     Q = H(1) H(2) . . . H(k), where k = min(m,n).
```

```
Each H(i) has the form

   H(i) = I - tau * v * v**H

where tau is a complex scalar, and v is a complex vector with
v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in A(i+1:m,i),
and tau in TAU(i).
```

———-

34.20.3 fortran code

— **cgeqrfp.f** —

```
*  ===================================================================
      SUBROUTINE CGEQRFP( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  ===================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, LDWORK, LWKOPT, NB,
     $                   NBMIN, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEQR2P, CLARFB, CLARFT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
```

```
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NB = ILAENV( 1, 'CGEQRF', ' ', M, N, -1, -1 )
      LWKOPT = N*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEQRFP', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      K = MIN( M, N )
      IF( K.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      NX = 0
      IWS = N
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'CGEQRF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = N
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
```

```
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'CGEQRF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially
*
         DO 10 I = 1, K - NX, NB
            IB = MIN( K-I+1, NB )
*
*           Compute the QR factorization of the current block
*           A(i:m,i:i+ib-1)
*
            CALL CGEQR2P( M-I+1, IB, A( I, I ), LDA, TAU( I ), WORK,
     $                    IINFO )
            IF( I+IB.LE.N ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i) H(i+1) . . . H(i+ib-1)
*
               CALL CLARFT( 'Forward', 'Columnwise', M-I+1, IB,
     $                      A( I, I ), LDA, TAU( I ), WORK, LDWORK )
*
*              Apply H**H to A(i:m,i+ib:n) from the left
*
               CALL CLARFB( 'Left', 'Conjugate transpose', 'Forward',
     $                      'Columnwise', M-I+1, N-I-IB+1, IB,
     $                      A( I, I ), LDA, WORK, LDWORK, A( I, I+IB ),
     $                      LDA, WORK( IB+1 ), LDWORK )
            END IF
   10    CONTINUE
      ELSE
         I = 1
      END IF
*
*     Use unblocked code to factor the last or only block.
*
      IF( I.LE.K )
     $   CALL CGEQR2P( M-I+1, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                IINFO )
*
      WORK( 1 ) = IWS
      RETURN
```

```
*
*     End of CGEQRFP
*
      END
```

———-

34.20.4 lisp code

— **cgeqrfp.lisp** —

```
(defun cgeqrfp (m n a lda tau work lwork info)
  (declare (type (array f2cl-lib:complex8 (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex8 a-%data% a-%offset%)
       (tau f2cl-lib:complex8 tau-%data% tau-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ldwork 0) (lwkopt 0) (nb 0)
           (nbmin 0) (nx 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) nx nbmin nb lwkopt ldwork k iws iinfo
                ib i))
      (setf info 0)
      (setf nb
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 1 "CGEQRF" " " m n -1 -1)
                (declare (ignore var-0 var-1 var-2 var-5 var-6))
                (when var-3 (setf m var-3))
                (when var-4 (setf n var-4))
                ret-val))
      (setf lwkopt (f2cl-lib:int-mul n nb))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce lwkopt 'f2cl-lib:complex8))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4))
            ((and
              (< lwork
                 (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
              (not lquery))
             (setf info -7)))
      (cond
       ((/= info 0) (xerbla "CGEQRFP" (f2cl-lib:int-sub info)) (go end_label))
       (lquery (go end_label)))
      (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
```

```
(cond
 ((= k 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce 1 'f2cl-lib:complex8))
  (go end_label)))
(setf nbmin 2)
(setf nx 0)
(setf iws n)
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "CGEQRF" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (when var-3 (setf m var-3))
                      (when var-4 (setf n var-4))
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork n) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
                            (ilaenv 2 "CGEQRF" " " m n -1 -1)
                          (declare
                           (ignore var-0 var-1 var-2 var-5 var-6))
                          (when var-3 (setf m var-3))
                          (when var-4 (setf n var-4))
                          ret-val)))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub nx))) nil)
    (tagbody
      (setf ib
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
               (the f2cl-lib:integer4 nb)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (cgeqr2p (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
```

```
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                            ((1 lda) (1 *)) a-%offset%)
      lda
      (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i) ((1 *))
                            tau-%offset%)
      work iinfo)
    (declare (ignore var-0 var-1 var-2 var-4 var-5))
    (setf lda var-3)
    (setf iinfo var-6))
  (cond
   ((<= (f2cl-lib:int-add i ib) n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clarft "Forward" "Columnwise"
         (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i)
                               ((1 *)) tau-%offset%)
         work ldwork)
      (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
      (when var-3 (setf ib var-3))
      (when var-5 (setf lda var-5))
      (when var-8 (setf ldwork var-8)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14)
        (clarfb "Left" "Conjugate transpose" "Forward" "Columnwise"
         (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
         (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1) ib
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                               ((1 lda) (1 *)) a-%offset%)
         lda work ldwork
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               (i (f2cl-lib:int-add i ib))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                               ((+ ib 1)) ((1 *)) work-%offset%)
         ldwork)
      (declare
       (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
        var-13))
      (when var-6 (setf ib var-6))
      (when var-8 (setf lda var-8))
      (when var-10 (setf ldwork var-10))
      (when var-12 (setf lda var-12))
      (when var-14 (setf ldwork var-14)))))
label10)))
```

```
       (t (setf i 1)))
      (if (<= i k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (cgeqr2p (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
               (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i) ((1 *))
                                     tau-%offset%)
               work iinfo)
            (declare (ignore var-0 var-1 var-2 var-4 var-5))
            (setf lda var-3)
            (setf iinfo var-6)))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce iws 'f2cl-lib:complex8))
      (go end_label)
     end_label
      (return (values m n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgeqrfp
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::cgeqr2p))))
```

34.21 cgeqrt

34.21.1 Axiom unit tests

— cgeqrt.input —

```
)set break resume
)sys rm -f cgeqrt.output
)spool cgeqrt.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.21.2 Axiom help page

— cgeqrt.help —

```
======================================================================
cgeqrt examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgeqrt (M, N, NB, A, LDA, T, LDT, WORK, INFO)
           CGEQRT

Function/Subroutine Documentation
   subroutine cgeqrt (integer M, integer N, integer NB, complex, dimension(
       lda, * ) A, integer LDA, complex, dimension( ldt, * ) T, integer LDT,
       complex, dimension( * ) WORK, integer INFO)
       CGEQRT

Purpose:
   CGEQRT computes a blocked QR factorization of a complex M-by-N matrix A
   using the compact WY representation of Q.

Parameters:
```

```
M
  M is INTEGER
  The number of rows of the matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

NB
  NB is INTEGER
  The block size to be used in the blocked QR.  MIN(M,N) >= NB >= 1.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit, the elements on and above the diagonal of the array
  contain the min(M,N)-by-N upper trapezoidal matrix R (R is
  upper triangular if M >= N); the elements below the diagonal
  are the columns of V.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

T
  T is COMPLEX array, dimension (LDT,MIN(M,N))
  The upper triangular block reflectors stored in compact form
  as a sequence of upper triangular blocks.  See below
  for further details.

LDT
  LDT is INTEGER
  The leading dimension of the array T.  LDT >= NB.

WORK
  WORK is COMPLEX array, dimension (NB*N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value

  The matrix V stores the elementary reflectors H(i) in the i-th column
  below the diagonal. For example, if M=5 and N=3, the matrix V is

          V = (  1       )
              ( v1  1    )
              ( v1 v2  1 )
              ( v1 v2 v3 )
```

```
              ( v1 v2 v3 )

   where the vi's represent the vectors which define H(i), which are returned
   in the matrix A.  The 1's along the diagonal of V are not stored in A.

   Let K=MIN(M,N).  The number of blocks is B = ceiling(K/NB), where each
   block is of order NB except for the last block, which is of order
   IB = K - (B-1)*NB.  For each of the B blocks, a upper triangular block
   reflector factor is computed: T1, T2, ..., TB.  The NB-by-NB (and IB-by-IB
   for the last block) T's are stored in the NB-by-N matrix T as

      T = (T1 T2 ... TB).
```

———-

34.21.3 fortran code

— **cgeqrt.f** —

```
*  =====================================================================
      SUBROUTINE CGEQRT( M, N, NB, A, LDA, T, LDT, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      INTEGER INFO, LDA, LDT, M, N, NB
*     ..
*     .. Array Arguments ..
      COMPLEX A( LDA, * ), T( LDT, * ), WORK( * )
*     ..
*
* =====================================================================
*
*     ..
*     .. Local Scalars ..
      INTEGER    I, IB, IINFO, K
      LOGICAL    USE_RECURSIVE_QR
      PARAMETER( USE_RECURSIVE_QR=.TRUE. )
*     ..
*     .. External Subroutines ..
      EXTERNAL   CGEQRT2, CGEQRT3, CLARFB, XERBLA
*     ..
*     .. Executable Statements ..
```

```
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NB.LT.1 .OR. ( NB.GT.MIN(M,N) .AND. MIN(M,N).GT.0 ) )THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDT.LT.NB ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEQRT', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      K = MIN( M, N )
      IF( K.EQ.0 ) RETURN
*
*     Blocked loop of length K
*
      DO I = 1, K,  NB
         IB = MIN( K-I+1, NB )
*
*     Compute the QR factorization of the current block A(I:M,I:I+IB-1)
*
         IF( USE_RECURSIVE_QR ) THEN
            CALL CGEQRT3( M-I+1, IB, A(I,I), LDA, T(1,I), LDT, IINFO )
         ELSE
            CALL CGEQRT2( M-I+1, IB, A(I,I), LDA, T(1,I), LDT, IINFO )
         END IF
         IF( I+IB.LE.N ) THEN
*
*     Update by applying H**H to A(I:M,I+IB:N) from the left
*
            CALL CLARFB( 'L', 'C', 'F', 'C', M-I+1, N-I-IB+1, IB,
     $                   A( I, I ), LDA, T( 1, I ), LDT,
     $                   A( I, I+IB ), LDA, WORK , N-I-IB+1 )
         END IF
      END DO
      RETURN
*
*     End of CGEQRT
*
```

```
      END
```

———-

34.21.4 lisp code

— **cgeqrt.lisp** —

```
(let* ((use_recursive_qr f2cl-lib:%true%))
  (declare (type f2cl-lib:logical use_recursive_qr)
           (ignorable use_recursive_qr))
  (defun cgeqrt (m n nb a lda t$ ldt work info)
    (declare (type (array f2cl-lib:complex8 (*)) work t$ a)
             (type (f2cl-lib:integer4) info ldt lda nb n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (t$ f2cl-lib:complex8 t$-%data% t$-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%))
      (prog ((i 0) (ib 0) (iinfo 0) (k 0))
        (declare (type (f2cl-lib:integer4) i ib iinfo k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((or (< nb 1)
                   (and
                    (> nb
                       (min (the f2cl-lib:integer4 m)
                            (the f2cl-lib:integer4 n)))
                    (>
                     (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n))
                     0)))
               (setf info -3))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -5))
              ((< ldt nb) (setf info -7)))
        (cond
         ((/= info 0) (xerbla "CGEQRT" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (if (= k 0)
            (go end_label))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                      ((> i k) nil)
          (tagbody
            (setf ib
                  (min
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
```

```
          (the f2cl-lib:integer4 nb)))
(cond
 (use_recursive_qr
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (cgeqrt3 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8 (1 i)
                             ((1 ldt) (1 *)) t$-%offset%)
       ldt iinfo)
    (declare (ignore var-0 var-2 var-4))
    (when var-1 (setf ib var-1))
    (when var-3 (setf lda var-3))
    (when var-5 (setf ldt var-5))
    (when var-6 (setf iinfo var-6))))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (cgeqrt2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8 (1 i)
                             ((1 ldt) (1 *)) t$-%offset%)
       ldt iinfo)
    (declare (ignore var-0 var-1 var-2 var-4))
    (setf lda var-3)
    (setf ldt var-5)
    (setf iinfo var-6))))
(cond
 ((<= (f2cl-lib:int-add i ib) n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (clarfb "L" "C" "F" "C"
       (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
       (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1) ib
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8 (1 i)
                             ((1 ldt) (1 *)) t$-%offset%)
       ldt
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             (i (f2cl-lib:int-add i ib))
                             ((1 lda) (1 *)) a-%offset%)
       lda work (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1))
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
      var-13 var-14))
```

```
                (when var-6 (setf ib var-6))
                (when var-8 (setf lda var-8))
                (when var-10 (setf ldt var-10))
                (when var-12 (setf lda var-12)))))
           label100000))
        (go end_label)
       end_label
        (return (values nil nil nil nil lda nil ldt nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgeqrt
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldt nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::cgeqrt2))))
```

———-

34.22 cgeqrt2

34.22.1 Axiom unit tests

— cgeqrt2.input —

```
)set break resume
)sys rm -f cgeqrt2.output
```

```
)spool cgeqrt2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.22.2 Axiom help page

— **cgeqrt2.help** —

```
==========================================================================
cgeqrt2 examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgeqrt2 (M, N, A, LDA, T, LDT, INFO)
           CGEQRT2 computes a QR factorization of a general real or complex
           matrix using the compact WY representation of Q.

Function/Subroutine Documentation
   subroutine cgeqrt2 (integer M, integer N, complex, dimension( lda, * ) A,
       integer LDA, complex, dimension( ldt, * ) T, integer LDT, integer INFO)
       CGEQRT2 computes a QR factorization of a general real or complex matrix
       using the compact WY representation of Q.

Purpose:
   CGEQRT2 computes a QR factorization of a complex M-by-N matrix A,
   using the compact WY representation of Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= N.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.
```

```
A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the complex M-by-N matrix A.  On exit, the elements on and
  above the diagonal contain the N-by-N upper triangular matrix R; the
  elements below the diagonal are the columns of V.  See below for
  further details.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

T
  T is COMPLEX array, dimension (LDT,N)
  The N-by-N upper triangular factor of the block reflector.
  The elements on and above the diagonal contain the block
  reflector T; the elements below the diagonal are not used.
  See below for further details.

LDT
  LDT is INTEGER
  The leading dimension of the array T.  LDT >= max(1,N).

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value

  The matrix V stores the elementary reflectors H(i) in the i-th column
  below the diagonal. For example, if M=5 and N=3, the matrix V is

          V = (  1       )
              ( v1  1    )
              ( v1 v2  1 )
              ( v1 v2 v3 )
              ( v1 v2 v3 )

  where the vi's represent the vectors which define H(i), which are returned
  in the matrix A.  The 1's along the diagonal of V are not stored in A.  The
  block reflector H is then given by

     H = I - V * T * V**H

  where V**H is the conjugate transpose of V.
```

34.22.3 fortran code

— cgeqrt2.f —

```
*  =====================================================================
      SUBROUTINE CGEQRT2( M, N, A, LDA, T, LDT, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER   INFO, LDA, LDT, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX   A( LDA, * ), T( LDT, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX  ONE, ZERO
      PARAMETER( ONE = (1.0,0.0), ZERO = (0.0,0.0) )
*     ..
*     .. Local Scalars ..
      INTEGER   I, K
      COMPLEX   AII, ALPHA
*     ..
*     .. External Subroutines ..
      EXTERNAL  CLARFG, CGEMV, CGERC, CTRMV, XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LDT.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEQRT2', -INFO )
         RETURN
```

```
      END IF
*
      K = MIN( M, N )
*
      DO I = 1, K
*
*        Generate elem. refl. H(i) to annihilate A(i+1:m,i), tau(I) -> T(I,1)
*
         CALL CLARFG( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
     $                T( I, 1 ) )
         IF( I.LT.N ) THEN
*
*           Apply H(i) to A(I:M,I+1:N) from the left
*
            AII = A( I, I )
            A( I, I ) = ONE
*
*           W(1:N-I) := A(I:M,I+1:N)**H * A(I:M,I) [W = T(:,N)]
*
            CALL CGEMV( 'C',M-I+1, N-I, ONE, A( I, I+1 ), LDA,
     $                  A( I, I ), 1, ZERO, T( 1, N ), 1 )
*
*           A(I:M,I+1:N) = A(I:m,I+1:N) + alpha*A(I:M,I)*W(1:N-1)**H
*
            ALPHA = -CONJG(T( I, 1 ))
            CALL CGERC( M-I+1, N-I, ALPHA, A( I, I ), 1,
     $           T( 1, N ), 1, A( I, I+1 ), LDA )
            A( I, I ) = AII
         END IF
      END DO
*
      DO I = 2, N
         AII = A( I, I )
         A( I, I ) = ONE
*
*        T(1:I-1,I) := alpha * A(I:M,1:I-1)**H * A(I:M,I)
*
         ALPHA = -T( I, 1 )
         CALL CGEMV( 'C', M-I+1, I-1, ALPHA, A( I, 1 ), LDA,
     $               A( I, I ), 1, ZERO, T( 1, I ), 1 )
         A( I, I ) = AII
*
*        T(1:I-1,I) := T(1:I-1,1:I-1) * T(1:I-1,I)
*
         CALL CTRMV( 'U', 'N', 'N', I-1, T, LDT, T( 1, I ), 1 )
*
*           T(I,I) = tau(I)
*
            T( I, I ) = T( I, 1 )
            T( I, 1) = ZERO
```

```
      END DO

*
*     End of CGEQRT2
*
      END
```

———-

34.22.4 lisp code

— **cgeqrt2.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)) (zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (ignorable one zero))
  (defun cgeqrt2 (m n a lda t$ ldt info)
    (declare (type (array f2cl-lib:complex8 (*)) t$ a)
             (type (f2cl-lib:integer4) info ldt lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (t$ f2cl-lib:complex8 t$-%data% t$-%offset%))
      (prog ((aii #C(0.0 0.0)) (alpha #C(0.0 0.0)) (i 0) (k 0))
        (declare (type (f2cl-lib:complex8) aii alpha)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4))
              ((< ldt
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -6)))
        (cond
         ((/= info 0) (xerbla "CGEQRT2" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i k) nil)
          (tagbody
            (clarfg (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   ((min (f2cl-lib:int-add i 1) m) i)
                                   ((1 lda) (1 *)) a-%offset%)
             1
```

```
 (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8 (i 1)
                       ((1 ldt) (1 *)) t$-%offset%))
(cond
 ((< i n)
  (setf aii
          (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
          one)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (cgemv "C" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
       (f2cl-lib:int-sub n i) one
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             (i (f2cl-lib:int-add i 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                             ((1 lda) (1 *)) a-%offset%)
       1 zero
       (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8 (1 n)
                             ((1 ldt) (1 *)) t$-%offset%)
       1)
    (declare
     (ignore var-0 var-1 var-2 var-4 var-6 var-7 var-9 var-10))
    (when var-3 (setf one var-3))
    (when var-5 (setf lda var-5))
    (when var-8 (setf zero var-8)))
  (setf alpha
          (-
           (f2cl-lib:conjg
            (f2cl-lib:fref t$-%data% (i 1) ((1 ldt) (1 *))
                           t$-%offset%))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (cgerc (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
       (f2cl-lib:int-sub n i) alpha
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8 (1 n)
                             ((1 ldt) (1 *)) t$-%offset%)
       1
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             (i (f2cl-lib:int-add i 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7))
    (when var-2 (setf alpha var-2))
```

```
        (when var-8 (setf lda var-8)))
      (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
            aii)))
   label100000))
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf aii
            (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
            one)
    (setf alpha
            (-
             (f2cl-lib:fref t$-%data% (i 1) ((1 ldt) (1 *))
                            t$-%offset%)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (cgemv "C" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
         (f2cl-lib:int-sub i 1) alpha
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i i)
                               ((1 lda) (1 *)) a-%offset%)
         1 zero
         (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8 (1 i)
                               ((1 ldt) (1 *)) t$-%offset%)
         1)
      (declare
       (ignore var-0 var-1 var-2 var-4 var-6 var-7 var-9 var-10))
      (when var-3 (setf alpha var-3))
      (when var-5 (setf lda var-5))
      (when var-8 (setf zero var-8)))
    (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
            aii)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (ctrmv "U" "N" "N" (f2cl-lib:int-sub i 1) t$ ldt
         (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8 (1 i)
                               ((1 ldt) (1 *)) t$-%offset%)
         1)
      (declare (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7))
      (when var-5 (setf ldt var-5)))
    (setf (f2cl-lib:fref t$-%data% (i i) ((1 ldt) (1 *)) t$-%offset%)
            (f2cl-lib:fref t$-%data% (i 1) ((1 ldt) (1 *))
                           t$-%offset%))
    (setf (f2cl-lib:fref t$-%data% (i 1) ((1 ldt) (1 *)) t$-%offset%)
            zero)
   label100001))
```

```
       end_label
        (return (values nil nil nil lda nil ldt info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgeqrt2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::lda
                                              nil fortran-to-lisp::ldt
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

34.23 cgeqrt3

34.23.1 Axiom unit tests

— **cgeqrt3.input** —

```
)set break resume
)sys rm -f cgeqrt3.output
)spool cgeqrt3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.23.2 Axiom help page [Elmr00]

— cgeqrt3.help —

```
==================================================================
cgeqrt3 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       recursive subroutine cgeqrt3 (M, N, A, LDA, T, LDT, INFO)
           CGEQRT3 recursively computes a QR factorization of a general real
           or complex matrix using the compact WY representation of Q.

Function/Subroutine Documentation
   recursive subroutine cgeqrt3 (integer M, integer N, complex, dimension(
       lda, * ) A, integer LDA, complex, dimension( ldt, * ) T, integer LDT,
       integer INFO)
       CGEQRT3 recursively computes a QR factorization of a general real or
       complex matrix using the compact WY representation of Q.

Purpose:
   CGEQRT3 recursively computes a QR factorization of a complex M-by-N
   matrix A, using the compact WY representation of Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= N.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the complex M-by-N matrix A.  On exit, the elements on and
   above the diagonal contain the N-by-N upper triangular matrix R; the
   elements below the diagonal are the columns of V.  See below for
   further details.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).
```

```
T
  T is COMPLEX array, dimension (LDT,N)
  The N-by-N upper triangular factor of the block reflector.
  The elements on and above the diagonal contain the block
  reflector T; the elements below the diagonal are not used.
  See below for further details.

LDT
  LDT is INTEGER
  The leading dimension of the array T.  LDT >= max(1,N).

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value

  The matrix V stores the elementary reflectors H(i) in the i-th column
  below the diagonal. For example, if M=5 and N=3, the matrix V is

          V = (  1       )
              ( v1  1    )
              ( v1 v2  1 )
              ( v1 v2 v3 )
              ( v1 v2 v3 )

  where the vi's represent the vectors which define H(i), which are returned
  in the matrix A.  The 1's along the diagonal of V are not stored in A.  The
  block reflector H is then given by

     H = I - V * T * V**H

  where V**H is the conjugate transpose of V.
```

———-

34.23.3 fortran code

— cgeqrt3.f —

```
*  =====================================================================
      RECURSIVE SUBROUTINE CGEQRT3( M, N, A, LDA, T, LDT, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
```

```
*
*     .. Scalar Arguments ..
      INTEGER   INFO, LDA, M, N, LDT
*     ..
*     .. Array Arguments ..
      COMPLEX   A( LDA, * ), T( LDT, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX   ONE
      PARAMETER ( ONE = (1.0,0.0) )
*     ..
*     .. Local Scalars ..
      INTEGER   I, I1, J, J1, N1, N2, IINFO
*     ..
*     .. External Subroutines ..
      EXTERNAL  CLARFG, CTRMM, CGEMM, XERBLA
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      IF( N .LT. 0 ) THEN
         INFO = -2
      ELSE IF( M .LT. N ) THEN
         INFO = -1
      ELSE IF( LDA .LT. MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LDT .LT. MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGEQRT3', -INFO )
         RETURN
      END IF
*
      IF( N.EQ.1 ) THEN
*
*        Compute Householder transform when N=1
*
         CALL CLARFG( M, A, A( MIN( 2, M ), 1 ), 1, T )
*
      ELSE
*
*        Otherwise, split A into blocks...
*
         N1 = N/2
         N2 = N-N1
         J1 = MIN( N1+1, N )
```

```
         I1 = MIN( N+1, M )
*
*        Compute A(1:M,1:N1) <- (Y1,R1,T1), where Q1 = I - Y1 T1 Y1**H
*
         CALL CGEQRT3( M, N1, A, LDA, T, LDT, IINFO )
*
*        Compute A(1:M,J1:N) = Q1**H A(1:M,J1:N) [workspace: T(1:N1,J1:N)]
*
         DO J=1,N2
            DO I=1,N1
               T( I, J+N1 ) = A( I, J+N1 )
            END DO
         END DO
         CALL CTRMM( 'L', 'L', 'C', 'U', N1, N2, ONE,
     &               A, LDA, T( 1, J1 ), LDT )
*
         CALL CGEMM( 'C', 'N', N1, N2, M-N1, ONE, A( J1, 1 ), LDA,
     &               A( J1, J1 ), LDA, ONE, T( 1, J1 ), LDT)
*
         CALL CTRMM( 'L', 'U', 'C', 'N', N1, N2, ONE,
     &               T, LDT, T( 1, J1 ), LDT )
*
         CALL CGEMM( 'N', 'N', M-N1, N2, N1, -ONE, A( J1, 1 ), LDA,
     &               T( 1, J1 ), LDT, ONE, A( J1, J1 ), LDA )
*
         CALL CTRMM( 'L', 'L', 'N', 'U', N1, N2, ONE,
     &               A, LDA, T( 1, J1 ), LDT )
*
         DO J=1,N2
            DO I=1,N1
               A( I, J+N1 ) = A( I, J+N1 ) - T( I, J+N1 )
            END DO
         END DO
*
*        Compute A(J1:M,J1:N) <- (Y2,R2,T2) where Q2 = I - Y2 T2 Y2**H
*
         CALL CGEQRT3( M-N1, N2, A( J1, J1 ), LDA,
     &                T( J1, J1 ), LDT, IINFO )
*
*        Compute T3 = T(1:N1,J1:N) = -T1 Y1**H Y2 T2
*
         DO I=1,N1
            DO J=1,N2
               T( I, J+N1 ) = CONJG(A( J+N1, I ))
            END DO
         END DO
*
         CALL CTRMM( 'R', 'L', 'N', 'U', N1, N2, ONE,
     &               A( J1, J1 ), LDA, T( 1, J1 ), LDT )
*
```

```
         CALL CGEMM( 'C', 'N', N1, N2, M-N, ONE, A( I1, 1 ), LDA,
     &               A( I1, J1 ), LDA, ONE, T( 1, J1 ), LDT )
*
         CALL CTRMM( 'L', 'U', 'N', 'N', N1, N2, -ONE, T, LDT,
     &               T( 1, J1 ), LDT )
*
         CALL CTRMM( 'R', 'U', 'N', 'N', N1, N2, ONE,
     &               T( J1, J1 ), LDT, T( 1, J1 ), LDT )
*
*        Y = (Y1,Y2); R = [ R1  A(1:N1,J1:N) ];  T = [T1 T3]
*                         [  0        R2     ]       [ 0 T2]
*
      END IF
*
      RETURN
*
*     End of CGEQRT3
*
      END
```

———-

34.23.4 lisp code

— **cgeqrt3.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (ignorable one))
  (defun *main* ()
    (prog ((i 0) (i1 0) (j 0) (j1 0) (n1 0) (n2 0) (iinfo 0) (m 0) (info 0)
           (ldt 0)
           (t$
            (make-array (the fixnum (reduce #'* (list ldt *))) :element-type
                        'f2cl-lib:complex8))
           (lda 0)
           (a
            (make-array (the fixnum (reduce #'* (list lda *))) :element-type
                        'f2cl-lib:complex8))
           (n 0) (subroutine 0.0))
      (declare (type (single-float) subroutine)
               (type (array f2cl-lib:complex8 (*)) a t$)
               (type (f2cl-lib:integer4) n lda ldt info i i1 j j1 n1 n2 iinfo
                m))
      '"****NOT TRANSLATED: (RECURSIVE SUBROUTINE CGEQRT3 (M |,| N |,| A |,| LDA |,| T |,| LDT |,| INFO))"
      (setf info 0)
      (cond ((< n 0) (setf info -2)) ((< m n) (setf info -1))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
```

```
        (setf info -4))
       ((< ldt (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -6)))
(cond
 ((/= info 0) (xerbla "CGEQRT3" (f2cl-lib:int-sub info)) (go end_label)))
(cond
 ((= n 1)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (clarfg m a
       (f2cl-lib:array-slice a f2cl-lib:complex8 ((min 2 m) 1)
                             ((1 lda) (1 *)))
       1 t$)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf m var-0))))
 (t (setf n1 (the f2cl-lib:integer4 (truncate n 2)))
  (setf n2 (f2cl-lib:int-sub n n1))
  (setf j1
          (min (the f2cl-lib:integer4 (f2cl-lib:int-add n1 1))
               (the f2cl-lib:integer4 n)))
  (setf i1
          (min (the f2cl-lib:integer4 (f2cl-lib:int-add n 1))
               (the f2cl-lib:integer4 m)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (cgeqrt3 m n1 a lda t$ ldt iinfo)
    (declare (ignore var-2 var-4))
    (when var-0 (setf m var-0))
    (when var-1 (setf n1 var-1))
    (when var-3 (setf lda var-3))
    (when var-5 (setf ldt var-5))
    (when var-6 (setf iinfo var-6)))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n2) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n1) nil)
        (tagbody
          (setf (f2cl-lib:fref t$ (i (f2cl-lib:int-add j n1))
                                  ((1 ldt) (1 *)))
                (f2cl-lib:fref a (i (f2cl-lib:int-add j n1))
                                 ((1 lda) (1 *))))
        label100001))
    label100000))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ctrmm "L" "L" "C" "U" n1 n2 one a lda
       (f2cl-lib:array-slice t$ f2cl-lib:complex8 (1 j1) ((1 ldt) (1 *)))
       ldt)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n1 var-4))
```

```
  (when var-5 (setf n2 var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf ldt var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (cgemm "C" "N" n1 n2 (f2cl-lib:int-sub m n1) one
     (f2cl-lib:array-slice a f2cl-lib:complex8 (j1 1) ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice a f2cl-lib:complex8 (j1 j1) ((1 lda) (1 *)))
     lda one
     (f2cl-lib:array-slice t$ f2cl-lib:complex8 (1 j1) ((1 ldt) (1 *)))
     ldt)
  (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
  (when var-2 (setf n1 var-2))
  (when var-3 (setf n2 var-3))
  (when var-5 (setf one var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf ldt var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ctrmm "L" "U" "C" "N" n1 n2 one t$ ldt
     (f2cl-lib:array-slice t$ f2cl-lib:complex8 (1 j1) ((1 ldt) (1 *)))
     ldt)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf n2 var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldt var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (cgemm "N" "N" (f2cl-lib:int-sub m n1) n2 n1 (- one)
     (f2cl-lib:array-slice a f2cl-lib:complex8 (j1 1) ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice t$ f2cl-lib:complex8 (1 j1) ((1 ldt) (1 *)))
     ldt one
     (f2cl-lib:array-slice a f2cl-lib:complex8 (j1 j1) ((1 lda) (1 *)))
     lda)
  (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
  (when var-3 (setf n2 var-3))
  (when var-4 (setf n1 var-4))
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldt var-9))
  (when var-10 (setf one var-10))
```

```
  (when var-12 (setf lda var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ctrmm "L" "L" "N" "U" n1 n2 one a lda
     (f2cl-lib:array-slice t$ f2cl-lib:complex8 (1 j1) ((1 ldt) (1 *)))
     ldt)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf n2 var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf ldt var-10)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n2) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n1) nil)
      (tagbody
        (setf (f2cl-lib:fref a (i (f2cl-lib:int-add j n1))
                              ((1 lda) (1 *)))
                (-
                 (f2cl-lib:fref a (i (f2cl-lib:int-add j n1))
                                  ((1 lda) (1 *)))
                 (f2cl-lib:fref t$ (i (f2cl-lib:int-add j n1))
                                  ((1 ldt) (1 *)))))
       label100003))
   label100002))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (cgeqrt3 (f2cl-lib:int-sub m n1) n2
     (f2cl-lib:array-slice a f2cl-lib:complex8 (j1 j1) ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice t$ f2cl-lib:complex8 (j1 j1)
                           ((1 ldt) (1 *)))
     ldt iinfo)
  (declare (ignore var-0 var-2 var-4))
  (when var-1 (setf n2 var-1))
  (when var-3 (setf lda var-3))
  (when var-5 (setf ldt var-5))
  (when var-6 (setf iinfo var-6)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n1) nil)
  (tagbody
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n2) nil)
      (tagbody
        (setf (f2cl-lib:fref t$ (i (f2cl-lib:int-add j n1))
                              ((1 ldt) (1 *)))
                (f2cl-lib:conjg
                 (f2cl-lib:fref a ((f2cl-lib:int-add j n1) i)
```

```
                                ((1 lda) (1 *)))))
        label100005))
    label100004))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ctrmm "R" "L" "N" "U" n1 n2 one
     (f2cl-lib:array-slice a f2cl-lib:complex8 (j1 j1) ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice t$ f2cl-lib:complex8 (1 j1) ((1 ldt) (1 *)))
     ldt)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf n2 var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf ldt var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (cgemm "C" "N" n1 n2 (f2cl-lib:int-sub m n) one
     (f2cl-lib:array-slice a f2cl-lib:complex8 (i1 1) ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice a f2cl-lib:complex8 (i1 j1) ((1 lda) (1 *)))
     lda one
     (f2cl-lib:array-slice t$ f2cl-lib:complex8 (1 j1) ((1 ldt) (1 *)))
     ldt)
  (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
  (when var-2 (setf n1 var-2))
  (when var-3 (setf n2 var-3))
  (when var-5 (setf one var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf ldt var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ctrmm "L" "U" "N" "N" n1 n2 (- one) t$ ldt
     (f2cl-lib:array-slice t$ f2cl-lib:complex8 (1 j1) ((1 ldt) (1 *)))
     ldt)
  (declare (ignore var-0 var-1 var-2 var-3 var-6 var-7 var-9))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf n2 var-5))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldt var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ctrmm "R" "U" "N" "N" n1 n2 one
```

```
            (f2cl-lib:array-slice t$ f2cl-lib:complex8 (j1 j1)
                                   ((1 ldt) (1 *)))
            ldt
            (f2cl-lib:array-slice t$ f2cl-lib:complex8 (1 j1) ((1 ldt) (1 *)))
            ldt)
          (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
          (when var-4 (setf n1 var-4))
          (when var-5 (setf n2 var-5))
          (when var-6 (setf one var-6))
          (when var-8 (setf ldt var-8))
          (when var-10 (setf ldt var-10)))))
      (go end_label)
     end_label
      (return nil))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::*main*
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types 'nil :return-values 'nil
                                            :calls 'nil)))
```

———-

34.24 cgerfs

34.24.1 Axiom unit tests

— **cgerfs.input** —

```
)set break resume
)sys rm -f cgerfs.output
)spool cgerfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.24.2 Axiom help page

— cgerfs.help —

```
=====================================================================
cgerfs examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgerfs (TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB, X,
           LDX, FERR, BERR, WORK, RWORK, INFO)
           CGERFS

Function/Subroutine Documentation
   subroutine cgerfs (character TRANS, integer N, integer NRHS, complex,
       dimension( lda, * ) A, integer LDA, complex, dimension( ldaf, * ) AF,
       integer LDAF, integer, dimension( * ) IPIV, complex, dimension( ldb, *
       ) B, integer LDB, complex, dimension( ldx, * ) X, integer LDX, real,
       dimension( * ) FERR, real, dimension( * ) BERR, complex, dimension( * )
       WORK, real, dimension( * ) RWORK, integer INFO)
       CGERFS

Purpose:
   CGERFS improves the computed solution to a system of linear
   equations and provides error bounds and backward error estimates for
   the solution.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.
```

```
A
  A is COMPLEX array, dimension (LDA,N)
  The original N-by-N matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX array, dimension (LDAF,N)
  The factors L and U from the factorization A = P*L*U
  as computed by CGETRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from CGETRF; for 1<=i<=N, row i of the
  matrix was interchanged with row IPIV(i).

B
  B is COMPLEX array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by CGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

FERR
  FERR is REAL array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.
```

```
 BERR
   BERR is REAL array, dimension (NRHS)
   The componentwise relative backward error of each solution
   vector X(j) (i.e., the smallest relative change in
   any element of A or B that makes X(j) an exact solution).

 WORK
   WORK is COMPLEX array, dimension (2*N)

 RWORK
   RWORK is REAL array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

   ITMAX is the maximum number of steps of iterative refinement.
```

34.24.3 fortran code

— cgerfs.f —

```
*  =====================================================================
      SUBROUTINE CGERFS( TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB,
     $                   X, LDX, FERR, BERR, WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               BERR( * ), FERR( * ), RWORK( * )
      COMPLEX            A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
```

```
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
      COMPLEX            ONE
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ) )
      REAL               TWO
      PARAMETER          ( TWO = 2.0E+0 )
      REAL               THREE
      PARAMETER          ( THREE = 3.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      CHARACTER          TRANSN, TRANST
      INTEGER            COUNT, I, J, K, KASE, NZ
      REAL               EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN, XK
      COMPLEX            ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH
      EXTERNAL           LSAME, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           CAXPY, CCOPY, CGEMV, CGETRS, CLACN2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, MAX, REAL
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
```

```
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -10
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -12
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGERFS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
      IF( NOTRAN ) THEN
         TRANSN = 'N'
         TRANST = 'C'
      ELSE
         TRANSN = 'C'
         TRANST = 'N'
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = N + 1
      EPS = SLAMCH( 'Epsilon' )
      SAFMIN = SLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 140 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
```

```
*
*       Loop until stopping criterion is satisfied.
*
*       Compute residual R = B - op(A) * X,
*       where op(A) = A, A**T, or A**H, depending on TRANS.
*
        CALL CCOPY( N, B( 1, J ), 1, WORK, 1 )
        CALL CGEMV( TRANS, N, N, -ONE, A, LDA, X( 1, J ), 1, ONE, WORK,
     $              1 )
*
*       Compute componentwise relative backward error from formula
*
*       max(i) ( abs(R(i)) / ( abs(op(A))*abs(X) + abs(B) )(i) )
*
*       where abs(Z) is the componentwise absolute value of the matrix
*       or vector Z.  If the i-th component of the denominator is less
*       than SAFE2, then SAFE1 is added to the i-th components of the
*       numerator and denominator before dividing.
*
        DO 30 I = 1, N
           RWORK( I ) = CABS1( B( I, J ) )
   30   CONTINUE
*
*       Compute abs(op(A))*abs(X) + abs(B).
*
        IF( NOTRAN ) THEN
           DO 50 K = 1, N
              XK = CABS1( X( K, J ) )
              DO 40 I = 1, N
                 RWORK( I ) = RWORK( I ) + CABS1( A( I, K ) )*XK
   40         CONTINUE
   50      CONTINUE
        ELSE
           DO 70 K = 1, N
              S = ZERO
              DO 60 I = 1, N
                 S = S + CABS1( A( I, K ) )*CABS1( X( I, J ) )
   60         CONTINUE
              RWORK( K ) = RWORK( K ) + S
   70      CONTINUE
        END IF
        S = ZERO
        DO 80 I = 1, N
           IF( RWORK( I ).GT.SAFE2 ) THEN
              S = MAX( S, CABS1( WORK( I ) ) / RWORK( I ) )
           ELSE
              S = MAX( S, ( CABS1( WORK( I ) )+SAFE1 ) /
     $            ( RWORK( I )+SAFE1 ) )
           END IF
   80   CONTINUE
```

```
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
*           1) The residual BERR(J) is larger than machine epsilon, and
*           2) BERR(J) decreased by at least a factor of 2 during the
*              last iteration, and
*           3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
*
*           Update solution and try again.
*
            CALL CGETRS( TRANS, N, 1, AF, LDAF, IPIV, WORK, N, INFO )
            CALL CAXPY( N, ONE, WORK, 1, X( 1, J ), 1 )
            LSTRES = BERR( J )
            COUNT = COUNT + 1
            GO TO 20
         END IF
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(op(A)))*
*           ( abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
*          inv(op(A)) is the inverse of op(A)
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(op(A))*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(op(A))*abs(X) + abs(B) is less than SAFE2.
*
*        Use CLACN2 to estimate the infinity-norm of the matrix
*           inv(op(A)) * diag(W),
*        where W = abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) )))
*
         DO 90 I = 1, N
            IF( RWORK( I ).GT.SAFE2 ) THEN
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I )
            ELSE
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I ) +
     $                      SAFE1
            END IF
   90    CONTINUE
```

```
*
         KASE = 0
  100    CONTINUE
         CALL CLACN2( N, WORK( N+1 ), WORK, FERR( J ), KASE, ISAVE )
         IF( KASE.NE.0 ) THEN
            IF( KASE.EQ.1 ) THEN
*
*              Multiply by diag(W)*inv(op(A)**H).
*
               CALL CGETRS( TRANST, N, 1, AF, LDAF, IPIV, WORK, N,
     $                      INFO )
               DO 110 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
  110          CONTINUE
            ELSE
*
*              Multiply by inv(op(A))*diag(W).
*
               DO 120 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
  120          CONTINUE
               CALL CGETRS( TRANSN, N, 1, AF, LDAF, IPIV, WORK, N,
     $                      INFO )
            END IF
            GO TO 100
         END IF
*
*        Normalize error.
*
         LSTRES = ZERO
         DO 130 I = 1, N
            LSTRES = MAX( LSTRES, CABS1( X( I, J ) ) )
  130    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  140 CONTINUE
*
      RETURN
*
*     End of CGERFS
*
      END
```

———-

34.24.4 lisp code

— cgerfs.lisp —

```
(let* ((itmax 5)
       (zero 0.0)
       (one (f2cl-lib:cmplx 1.0 0.0))
       (two 2.0)
       (three 3.0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (single-float 0.0 0.0) zero)
           (type (f2cl-lib:complex8) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 3.0 3.0) three)
           (ignorable itmax zero one two three))
  (defun cgerfs
         (trans n nrhs a lda af ldaf ipiv b ldb$ x ldx ferr berr work rwork
          info)
    (declare (type (array single-float (*)) rwork berr ferr)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) work x b af a)
             (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (af f2cl-lib:complex8 af-%data% af-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (ferr single-float ferr-%data% ferr-%offset%)
         (berr single-float berr-%data% berr-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
          cabs1))
        (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
               (zdum #C(0.0 0.0)) (eps 0.0) (lstres 0.0) (s 0.0) (safe1 0.0)
               (safe2 0.0) (safmin 0.0) (xk 0.0) (count$ 0) (i 0) (j 0) (k 0)
               (kase 0) (nz 0)
               (transn
                (make-array '(1) :element-type 'character :initial-element
                            #\ ))
               (transt
                (make-array '(1) :element-type 'character :initial-element
                            #\ ))
               (notran nil))
          (declare (type (array f2cl-lib:integer4 (3)) isave)
                   (type (f2cl-lib:complex8) zdum)
```

```
         (type (single-float) eps lstres s safe1 safe2 safmin xk)
         (type (f2cl-lib:integer4) count$ i j k kase nz)
         (type (string 1) transn transt)
         (type f2cl-lib:logical notran))
(setf info 0)
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -10))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -12)))
(cond
 ((/= info 0) (xerbla "CGERFS" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
              zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%)
              zero)
     label10))
  (go end_label)))
(cond
```

```
 (notran (f2cl-lib:f2cl-set-string transn "N" (string 1))
  (f2cl-lib:f2cl-set-string transt "C" (string 1)))
 (t (f2cl-lib:f2cl-set-string transn "C" (string 1))
  (f2cl-lib:f2cl-set-string transt "N" (string 1))))
(setf nz (f2cl-lib:int-add n 1))
(setf eps (slamch "Epsilon"))
(setf safmin (slamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (ccopy n
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1 work 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (cgemv trans n n (- one) a lda
         (f2cl-lib:array-slice x-%data% f2cl-lib:complex8 (1 j)
                               ((1 ldx) (1 *)) x-%offset%)
         1 one work 1)
      (declare (ignore var-3 var-4 var-6 var-7 var-9 var-10))
      (when var-0 (setf trans var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-5 (setf lda var-5))
      (when var-8 (setf one var-8)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                (cabs1
                 (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)))
       label30))
    (cond
     (notran
      (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                    ((> k n) nil)
        (tagbody
          (setf xk
                  (cabs1
```

```
                (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                               x-%offset%)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                               rwork-%offset%)
                (+
                 (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                rwork-%offset%)
                 (*
                  (cabs1
                   (f2cl-lib:fref a-%data% (i k)
                                  ((1 lda) (1 *)) a-%offset%))
                  xk)))
          label40))
    label50)))
 (t
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (setf s zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf s
                (+ s
                   (*
                    (cabs1
                     (f2cl-lib:fref a-%data% (i k)
                                    ((1 lda) (1 *))
                                    a-%offset%))
                    (cabs1
                     (f2cl-lib:fref x-%data% (i j)
                                    ((1 ldx) (1 *))
                                    x-%offset%)))))
          label60))
      (setf (f2cl-lib:fref rwork-%data% (k) ((1 *))
                           rwork-%offset%)
            (+
             (f2cl-lib:fref rwork-%data% (k) ((1 *))
                            rwork-%offset%)
             s))
    label70))))
(setf s zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
```

```
      (setf s
              (max s
                   (/
                    (cabs1
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%))
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%)))))
     (t
      (setf s
              (max s
                   (/
                    (+
                     (cabs1
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%))
                     safe1)
                    (+
                     (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                    rwork-%offset%)
                     safe1))))))
   label80))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (cgetrs trans n 1 af ldaf ipiv work n info)
    (declare (ignore var-2 var-3 var-5 var-6))
    (when var-0 (setf trans var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf ldaf var-4))
    (when var-7 (setf n var-7))
    (when var-8 (setf info var-8)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (caxpy n one work 1
       (f2cl-lib:array-slice x-%data% f2cl-lib:complex8 (1 j)
                             ((1 ldx) (1 *)) x-%offset%)
       1)
    (declare (ignore var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))
    (when var-1 (setf one var-1)))
  (setf lstres
          (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
  (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
```

```
      (cond
       ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                             rwork-%offset%)
              (+
               (cabs1
                (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%))
               (* nz eps
                  (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                 rwork-%offset%)))))
       (t
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                             rwork-%offset%)
              (+
               (cabs1
                (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%))
               (* nz eps
                  (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                 rwork-%offset%))
               safe1))))
    label90))
  (setf kase 0)
 label100
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (clacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             ((+ n 1)) ((1 *)) work-%offset%)
       work
       (f2cl-lib:array-slice ferr-%data% single-float (j) ((1 *))
                             ferr-%offset%)
       kase isave)
    (declare (ignore var-1 var-2 var-3 var-5))
    (when var-0 (setf n var-0))
    (when var-4 (setf kase var-4)))
  (cond
   ((/= kase 0)
    (cond
     ((= kase 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (cgetrs transt n 1 af ldaf ipiv work n info)
        (declare (ignore var-2 var-3 var-5 var-6))
        (when var-0 (setf transt var-0))
        (when var-1 (setf n var-1))
        (when var-4 (setf ldaf var-4))
        (when var-7 (setf n var-7))
        (when var-8 (setf info var-8)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                    ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)
                    (*
                     (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                    rwork-%offset%)
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)))
             label110)))
          (t
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)
                     (*
                      (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                     rwork-%offset%)
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%)))
             label120))
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
               (cgetrs transn n 1 af ldaf ipiv work n info)
             (declare (ignore var-2 var-3 var-5 var-6))
             (when var-0 (setf transn var-0))
             (when var-1 (setf n var-1))
             (when var-4 (setf ldaf var-4))
             (when var-7 (setf n var-7))
             (when var-8 (setf info var-8)))))
        (go label100)))
      (setf lstres zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf lstres
                  (max lstres
                       (cabs1
                        (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                       x-%offset%))))
         label130))
      (if (/= lstres zero)
          (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                  (/
                   (f2cl-lib:fref ferr-%data% (j) ((1 *))
                                  ferr-%offset%)
                   lstres)))
    label140))
  (go end_label)
```

```
        end_label
         (return
          (values trans n nil nil lda nil ldaf nil nil nil nil nil nil nil nil
                  nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgerfs
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::integer4
                                              (*))
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (array single-float (*))
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(fortran-to-lisp::trans
                                             fortran-to-lisp::n nil nil
                                             fortran-to-lisp::lda nil
                                             fortran-to-lisp::ldaf nil nil nil
                                             nil nil nil nil nil nil
                                             fortran-to-lisp::info)
                                           :calls 'nil)))
```

———-

34.25 cgerfsx

34.25.1 Axiom unit tests

— cgerfsx.input —

```
)set break resume
)sys rm -f cgerfsx.output
)spool cgerfsx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.25.2 Axiom help page [Demm05]

— cgerfsx.help —

```
====================================================================
cgerfsx examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgerfsx (TRANS, EQUED, N, NRHS, A, LDA, AF, LDAF, IPIV, R,
           C, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS, ERR_BNDS_NORM,
           ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, RWORK, INFO)
           CGERFSX

Function/Subroutine Documentation
   subroutine cgerfsx (character TRANS, character EQUED, integer N, integer
       NRHS, complex, dimension( lda, * ) A, integer LDA, complex, dimension(
       ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV, real,
       dimension( * ) R, real, dimension( * ) C, complex, dimension( ldb, * )
       B, integer LDB, complex, dimension( ldx , * ) X, integer LDX, real
       RCOND, real, dimension( * ) BERR, integer N_ERR_BNDS, real, dimension(
       nrhs, * ) ERR_BNDS_NORM, real, dimension( nrhs, * ) ERR_BNDS_COMP,
       integer NPARAMS, real, dimension( * ) PARAMS, complex, dimension( * )
```

```
      WORK, real, dimension( * ) RWORK, integer INFO)
      CGERFSX

Purpose:
   CGERFSX improves the computed solution to a system of linear
   equations and provides error bounds and backward error estimates
   for the solution.  In addition to normwise error bound, the code
   provides maximum componentwise error bound if possible.  See
   comments for ERR_BNDS_NORM and ERR_BNDS_COMP for details of the
   error bounds.

   The original system of linear equations may have been equilibrated
   before calling this routine, as described by arguments EQUED, R
   and C below. In this case, the solution and error bounds returned
   are for the original unequilibrated system.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose = Transpose)

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done to A
   before calling this routine. This is needed to compute
   the solution and error bounds correctly.
   = 'N':  No equilibration
   = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
   = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
   = 'B':  Both row and column equilibration, i.e., A has been
           replaced by diag(R) * A * diag(C).
           The right hand side B has been changed accordingly.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
```

```
  of the matrices B and X.  NRHS >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  The original N-by-N matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX array, dimension (LDAF,N)
  The factors L and U from the factorization A = P*L*U
  as computed by CGETRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from CGETRF; for 1<=i<=N, row i of the
  matrix was interchanged with row IPIV(i).

R
  R is REAL array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.
  If R is accessed, each element of R should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

C
  C is REAL array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.
  If C is accessed, each element of C should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is COMPLEX array, dimension (LDB,NRHS)
```

```
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by CGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

BERR
  BERR is REAL array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
        max_j (abs(XTRUE(j,i) - X(j,i)))
       ------------------------------
             max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information returned.
```

```
  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
          reciprocal condition number is less than the threshold
          sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
          almost certainly within a factor of 10 of the true error
          so long as the next entry is greater than the threshold
          sqrt(n) * slamch('Epsilon'). This error bound should only
          be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
          reciprocal condition number.  Compared with the threshold
          sqrt(n) * slamch('Epsilon') to determine if the error
          estimate is "guaranteed". These reciprocal condition
          numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
          appropriately scaled matrix Z.
          Let Z = S*A, where S scales each row by a power of the
          radix so all absolute row sums of Z are approximately 1.


ERR_BNDS_COMP
  ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
              abs(XTRUE(j,i) - X(j,i))
      max_j ----------------------
                   abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
```

```
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
          reciprocal condition number is less than the threshold
          sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
          almost certainly within a factor of 10 of the true error
          so long as the next entry is greater than the threshold
          sqrt(n) * slamch('Epsilon'). This error bound should only
          be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
          reciprocal condition number.  Compared with the threshold
          sqrt(n) * slamch('Epsilon') to determine if the error
          estimate is "guaranteed". These reciprocal condition
          numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
          appropriately scaled matrix Z.
          Let Z = S*(A*diag(x)), where x is the solution for the
          current right-hand side and S scales each row of
          A*diag(x) by a power of the radix so all absolute row
          sums of Z are approximately 1.


NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is REAL array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.

  Default: 1.0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
          (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.

  Default: 10
    Aggressive: Set to 100 to permit convergence using approximate
                factorizations or factorizations other than LU. If
```

```
                the factorization uses a technique other than
                Gaussian elimination, the guarantees in
                err_bnds_norm and err_bnds_comp may no longer be
                trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
    will attempt to find a solution with small componentwise
    relative error in the double-precision algorithm.  Positive
    is true, 0.0 is false.

  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is COMPLEX array, dimension (2*N)

RWORK
  RWORK is REAL array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is
        guaranteed.
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
      has been completed, but the factor U is exactly singular, so
      the solution and error bounds could not be computed. RCOND = 0
      is returned.
  = N+J: The solution corresponding to the Jth right-hand side is
      not guaranteed. The solutions corresponding to other right-
      hand sides K with K > J may not be guaranteed as well, but
      only the first such right-hand side is reported. If a small
      componentwise error is not requested (PARAMS(3) = 0.0) then
      the Jth right-hand side is the first with a normwise error
      bound that is not guaranteed (the smallest J such
      that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
      the Jth right-hand side is the first with either a normwise or
      componentwise error bound that is not guaranteed (the smallest
      J such that either ERR_BNDS_NORM(J,1) = 0.0 or
      ERR_BNDS_COMP(J,1) = 0.0). See the definition of
      ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
      about all of the right-hand sides check ERR_BNDS_NORM or
      ERR_BNDS_COMP.
```

34.25.3 fortran code

— cgerfsx.f —

```
*  ==================================================================
      SUBROUTINE CGERFSX( TRANS, EQUED, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $                    R, C, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS,
     $                    ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $                    WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS, EQUED
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX , * ), WORK( * )
      REAL               R( * ), C( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * ), RWORK( * )
*     ..
*
*  ================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               ITREF_DEFAULT, ITHRESH_DEFAULT,
     $                   COMPONENTWISE_DEFAULT
      REAL               RTHRESH_DEFAULT, DZTHRESH_DEFAULT
      PARAMETER          ( ITREF_DEFAULT = 1.0 )
      PARAMETER          ( ITHRESH_DEFAULT = 10.0 )
      PARAMETER          ( COMPONENTWISE_DEFAULT = 1.0 )
      PARAMETER          ( RTHRESH_DEFAULT = 0.5 )
      PARAMETER          ( DZTHRESH_DEFAULT = 0.25 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
```

```
*     ..
*     .. Local Scalars ..
      CHARACTER(1)       NORM
      LOGICAL            ROWEQU, COLEQU, NOTRAN
      INTEGER            J, TRANS_TYPE, PREC_TYPE, REF_TYPE
      INTEGER            N_NORMS
      REAL               ANORM, RCOND_TMP
      REAL               ILLRCOND_THRESH, ERR_LBND, CWISE_WRONG
      LOGICAL            IGNORE_CWISE
      INTEGER            ITHRESH
      REAL               RTHRESH, UNSTABLE_THRESH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, CGECON, CLA_GERFSX_EXTENDED
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT, TRANSFER
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, BLAS_FPINFO_X, ILATRANS, ILAPREC
      EXTERNAL           SLAMCH, CLANGE, CLA_GERCOND_X, CLA_GERCOND_C
      REAL               SLAMCH, CLANGE, CLA_GERCOND_X, CLA_GERCOND_C
      LOGICAL            LSAME
      INTEGER            BLAS_FPINFO_X
      INTEGER            ILATRANS, ILAPREC
*     ..
*     .. Executable Statements ..
*
*     Check the input parameters.
*
      INFO = 0
      TRANS_TYPE = ILATRANS( TRANS )
      REF_TYPE = INT( ITREF_DEFAULT )
      IF ( NPARAMS .GE. LA_LINRX_ITREF_I ) THEN
         IF ( PARAMS( LA_LINRX_ITREF_I ) .LT. 0.0 ) THEN
            PARAMS( LA_LINRX_ITREF_I ) = ITREF_DEFAULT
         ELSE
            REF_TYPE = PARAMS( LA_LINRX_ITREF_I )
         END IF
      END IF
*
*     Set default parameters.
*
      ILLRCOND_THRESH = REAL( N ) * SLAMCH( 'Epsilon' )
      ITHRESH = INT( ITHRESH_DEFAULT )
      RTHRESH = RTHRESH_DEFAULT
      UNSTABLE_THRESH = DZTHRESH_DEFAULT
      IGNORE_CWISE = COMPONENTWISE_DEFAULT .EQ. 0.0
*
      IF ( NPARAMS.GE.LA_LINRX_ITHRESH_I ) THEN
```

```
         IF ( PARAMS( LA_LINRX_ITHRESH_I ).LT.0.0 ) THEN
            PARAMS(LA_LINRX_ITHRESH_I) = ITHRESH
         ELSE
            ITHRESH = INT( PARAMS( LA_LINRX_ITHRESH_I ) )
         END IF
      END IF
      IF ( NPARAMS.GE.LA_LINRX_CWISE_I ) THEN
         IF ( PARAMS( LA_LINRX_CWISE_I ).LT.0.0 ) THEN
            IF ( IGNORE_CWISE ) THEN
               PARAMS( LA_LINRX_CWISE_I ) = 0.0
            ELSE
               PARAMS( LA_LINRX_CWISE_I ) = 1.0
            END IF
         ELSE
            IGNORE_CWISE = PARAMS( LA_LINRX_CWISE_I ) .EQ. 0.0
         END IF
      END IF
      IF ( REF_TYPE .EQ. 0 .OR. N_ERR_BNDS .EQ. 0 ) THEN
         N_NORMS = 0
      ELSE IF ( IGNORE_CWISE ) THEN
         N_NORMS = 1
      ELSE
         N_NORMS = 2
      END IF
*
      NOTRAN = LSAME( TRANS, 'N' )
      ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
      COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
*
*     Test input parameters.
*
      IF( TRANS_TYPE.EQ.-1 ) THEN
        INFO = -1
      ELSE IF( .NOT.ROWEQU .AND. .NOT.COLEQU .AND.
     $         .NOT.LSAME( EQUED, 'N' ) ) THEN
        INFO = -2
      ELSE IF( N.LT.0 ) THEN
        INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
        INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
        INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
        INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
        INFO = -13
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
        INFO = -15
      END IF
      IF( INFO.NE.0 ) THEN
```

```
        CALL XERBLA( 'CGERFSX', -INFO )
        RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         RCOND = 1.0
         DO J = 1, NRHS
            BERR( J ) = 0.0
            IF ( N_ERR_BNDS .GE. 1 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) =  1.0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
            IF ( N_ERR_BNDS .GE. 2 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 0.0
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 0.0
            END IF
            IF ( N_ERR_BNDS .GE. 3 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 1.0
            END IF
         END DO
         RETURN
      END IF
*
*     Default to failure.
*
      RCOND = 0.0
      DO J = 1, NRHS
         BERR( J ) = 1.0
         IF ( N_ERR_BNDS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
            ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
         END IF
         IF ( N_ERR_BNDS .GE. 2 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
         END IF
         IF ( N_ERR_BNDS .GE. 3 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 0.0
            ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 0.0
         END IF
      END DO
*
*     Compute the norm of A and the reciprocal of the condition
*     number of A.
*
      IF( NOTRAN ) THEN
         NORM = 'I'
```

```
      ELSE
         NORM = '1'
      END IF
      ANORM = CLANGE( NORM, N, N, A, LDA, RWORK )
      CALL CGECON( NORM, N, AF, LDAF, ANORM, RCOND, WORK, RWORK, INFO )
*
*     Perform refinement on each right-hand side
*
      IF ( REF_TYPE .NE. 0 ) THEN

         PREC_TYPE = ILAPREC( 'D' )

         IF ( NOTRAN ) THEN
            CALL CLA_GERFSX_EXTENDED( PREC_TYPE, TRANS_TYPE,  N,
     $           NRHS, A, LDA, AF, LDAF, IPIV, COLEQU, C, B,
     $           LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM,
     $           ERR_BNDS_COMP, WORK, RWORK, WORK(N+1),
     $           TRANSFER (RWORK(1:2*N), (/ (ZERO, ZERO) /), N),
     $           RCOND, ITHRESH, RTHRESH, UNSTABLE_THRESH, IGNORE_CWISE,
     $           INFO )
         ELSE
            CALL CLA_GERFSX_EXTENDED( PREC_TYPE, TRANS_TYPE,  N,
     $           NRHS, A, LDA, AF, LDAF, IPIV, ROWEQU, R, B,
     $           LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM,
     $           ERR_BNDS_COMP, WORK, RWORK, WORK(N+1),
     $           TRANSFER (RWORK(1:2*N), (/ (ZERO, ZERO) /), N),
     $           RCOND, ITHRESH, RTHRESH, UNSTABLE_THRESH, IGNORE_CWISE,
     $           INFO )
         END IF
      END IF

      ERR_LBND = MAX( 10.0, SQRT( REAL( N ) ) ) * SLAMCH( 'Epsilon' )
      IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 1 ) THEN
*
*     Compute scaled normwise condition number cond(A*C).
*
         IF ( COLEQU .AND. NOTRAN ) THEN
            RCOND_TMP = CLA_GERCOND_C( TRANS, N, A, LDA, AF, LDAF, IPIV,
     $           C, .TRUE., INFO, WORK, RWORK )
         ELSE IF ( ROWEQU .AND. .NOT. NOTRAN ) THEN
            RCOND_TMP = CLA_GERCOND_C( TRANS, N, A, LDA, AF, LDAF, IPIV,
     $           R, .TRUE., INFO, WORK, RWORK )
         ELSE
            RCOND_TMP = CLA_GERCOND_C( TRANS, N, A, LDA, AF, LDAF, IPIV,
     $           C, .FALSE., INFO, WORK, RWORK )
         END IF
         DO J = 1, NRHS
*
*     Cap the error at 1.0.
*
```

```
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .GT. 1.0 )
     $           ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 0.0
               IF ( INFO .LE. N ) INFO = N + J
            ELSE IF (ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .LT. ERR_LBND)
     $              THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF
         END DO
      END IF

      IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 2 ) THEN
*
*     Compute componentwise condition number cond(A*diag(Y(:,J))) for
*     each right-hand side using the current solution as an estimate of
*     the true solution.  If the componentwise error estimate is too
*     large, then the solution is a lousy estimate of truth and the
*     estimated RCOND may be too optimistic.  To avoid misleading users,
*     the inverse condition number is set to 0.0 when the estimated
*     cwise error is at least CWISE_WRONG.
*
         CWISE_WRONG = SQRT( SLAMCH( 'Epsilon' ) )
         DO J = 1, NRHS
            IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .LT. CWISE_WRONG )
     $     THEN
               RCOND_TMP = CLA_GERCOND_X( TRANS, N, A, LDA, AF, LDAF,
     $              IPIV, X(1,J), INFO, WORK, RWORK )
            ELSE
               RCOND_TMP = 0.0
            END IF
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .GT. 1.0 )
     $           ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
*
```

```
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 0.0
               IF ( PARAMS( LA_LINRX_CWISE_I ) .EQ. 1.0
     $              .AND. INFO.LT.N + J ) INFO = N + J
            ELSE IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I )
     $              .LT. ERR_LBND ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF

         END DO
      END IF
*
      RETURN
*
*     End of CGERFSX
*
      END
```

———-

34.25.4 lisp code

— **cgerfsx.lisp** —

———-

34.26 cgerq2

34.26.1 Axiom unit tests

— **cgerq2.input** —

```
)set break resume
```

```
)sys rm -f cgerq2.output
)spool cgerq2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.26.2 Axiom help page

— **cgerq2.help** —

```
==========================================================================
cgerq2 examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgerq2 (M, N, A, LDA, TAU, WORK, INFO)
           CGERQ2 computes the RQ factorization of a general rectangular
           matrix using an unblocked algorithm.

Function/Subroutine Documentation
   subroutine cgerq2 (integer M, integer N, complex, dimension( lda, * ) A,
       integer LDA, complex, dimension( * ) TAU, complex, dimension( * ) WORK,
       integer INFO)
       CGERQ2 computes the RQ factorization of a general rectangular matrix
       using an unblocked algorithm.

Purpose:
   CGERQ2 computes an RQ factorization of a complex m by n matrix A:
   A = R * Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
```

```
  The number of columns of the matrix A.  N >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the m by n matrix A.
  On exit, if m <= n, the upper triangle of the subarray
  A(1:m,n-m+1:n) contains the m by m upper triangular matrix R;
  if m >= n, the elements on and above the (m-n)-th subdiagonal
  contain the m by n upper trapezoidal matrix R; the remaining
  elements, with the array TAU, represent the unitary matrix
  Q as a product of elementary reflectors (see Further Details).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

TAU
  TAU is COMPLEX array, dimension (min(M,N))
  The scalar factors of the elementary reflectors (see Further Details).

WORK
  WORK is COMPLEX array, dimension (M)

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value

  The matrix Q is represented as a product of elementary reflectors

     Q = H(1)**H H(2)**H . . . H(k)**H, where k = min(m,n).

  Each H(i) has the form

     H(i) = I - tau * v * v**H

  where tau is a complex scalar, and v is a complex vector with
  v(n-k+i+1:n) = 0 and v(n-k+i) = 1; conjg(v(1:n-k+i-1)) is stored on
  exit in A(m-k+i,1:n-k+i-1), and tau in TAU(i).
```

———-

34.26.3 fortran code

— cgerq2.f —

```
*  =====================================================================
```

```
      SUBROUTINE CGERQ2( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ONE
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      COMPLEX            ALPHA
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLACGV, CLARF, CLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGERQ2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = K, 1, -1
```

```
*
*        Generate elementary reflector H(i) to annihilate
*        A(m-k+i,1:n-k+i-1)
*
         CALL CLACGV( N-K+I, A( M-K+I, 1 ), LDA )
         ALPHA = A( M-K+I, N-K+I )
         CALL CLARFG( N-K+I, ALPHA, A( M-K+I, 1 ), LDA,
     $                TAU( I ) )
*
*        Apply H(i) to A(1:m-k+i-1,1:n-k+i) from the right
*
         A( M-K+I, N-K+I ) = ONE
         CALL CLARF( 'Right', M-K+I-1, N-K+I, A( M-K+I, 1 ), LDA,
     $               TAU( I ), A, LDA, WORK )
         A( M-K+I, N-K+I ) = ALPHA
         CALL CLACGV( N-K+I-1, A( M-K+I, 1 ), LDA )
   10 CONTINUE
      RETURN
*
*     End of CGERQ2
*
      END
```

———-

34.26.4 lisp code

— cgerq2.lisp —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (ignorable one))
  (defun cgerq2 (m n a lda tau work info)
    (declare (type (array f2cl-lib:complex8 (*)) work tau a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (tau f2cl-lib:complex8 tau-%data% tau-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%))
      (prog ((alpha #C(0.0 0.0)) (i 0) (k 0))
        (declare (type (f2cl-lib:complex8) alpha)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
```

```
 ((/= info 0) (xerbla "CGERQ2" (f2cl-lib:int-sub info))
  (go end_label)))
(setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(f2cl-lib:fdo (i k (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2)
        (clacgv (f2cl-lib:int-add (f2cl-lib:int-sub n k) i)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               ((+ m (f2cl-lib:int-sub k) i) 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-0 var-1))
      (when var-2 (setf lda var-2)))
    (setf alpha
            (f2cl-lib:fref a-%data%
                           ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                            (f2cl-lib:int-add (f2cl-lib:int-sub n k)
                                              i))
                           ((1 lda) (1 *)) a-%offset%))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (clarfg (f2cl-lib:int-add (f2cl-lib:int-sub n k) i) alpha
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               ((+ m (f2cl-lib:int-sub k) i) 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i) ((1 *))
                               tau-%offset%))
      (declare (ignore var-0 var-2 var-4))
      (when var-1 (setf alpha var-1))
      (when var-3 (setf lda var-3)))
    (setf (f2cl-lib:fref a-%data%
                         ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                          (f2cl-lib:int-add (f2cl-lib:int-sub n k) i))
                         ((1 lda) (1 *)) a-%offset%)
            one)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clarf "Right"
         (f2cl-lib:int-sub (f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                           1)
         (f2cl-lib:int-add (f2cl-lib:int-sub n k) i)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               ((+ m (f2cl-lib:int-sub k) i) 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i) ((1 *))
                               tau-%offset%)
         a lda work)
      (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8))
```

```
              (when var-4 (setf lda var-4))
              (when var-7 (setf lda var-7)))
            (setf (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                  (f2cl-lib:int-add (f2cl-lib:int-sub n k) i))
                                 ((1 lda) (1 *)) a-%offset%)
                    alpha)
            (multiple-value-bind (var-0 var-1 var-2)
                (clacgv
                 (f2cl-lib:int-sub (f2cl-lib:int-add (f2cl-lib:int-sub n k) i)
                                   1)
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                       ((+ m (f2cl-lib:int-sub k) i) 1)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare (ignore var-0 var-1))
              (when var-2 (setf lda var-2)))
           label10))
        (go end_label)
       end_label
        (return (values nil nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgerq2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::lda
                                              nil nil fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

34.27 cgerqf

34.27.1 Axiom unit tests

— **cgerqf.input** —

```
)set break resume
)sys rm -f cgerqf.output
)spool cgerqf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.27.2 Axiom help page

— **cgerqf.help** —

```
======================================================================
cgerqf examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgerqf (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           CGERQF

Function/Subroutine Documentation
   subroutine cgerqf (integer M, integer N, complex, dimension( lda, * ) A,
       integer LDA, complex, dimension( * ) TAU, complex, dimension( * ) WORK,
       integer LWORK, integer INFO)
       CGERQF

Purpose:
   CGERQF computes an RQ factorization of a complex M-by-N matrix A:
   A = R * Q.

Parameters:
```

```
M
  M is INTEGER
  The number of rows of the matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit,
  if m <= n, the upper triangle of the subarray
  A(1:m,n-m+1:n) contains the M-by-M upper triangular matrix R;
  if m >= n, the elements on and above the (m-n)-th subdiagonal
  contain the M-by-N upper trapezoidal matrix R;
  the remaining elements, with the array TAU, represent the
  unitary matrix Q as a product of min(m,n) elementary
  reflectors (see Further Details).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

TAU
  TAU is COMPLEX array, dimension (min(M,N))
  The scalar factors of the elementary reflectors (see Further Details).

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,M).
  For optimum performance LWORK >= M*NB, where NB is
  the optimal blocksize.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value

  The matrix Q is represented as a product of elementary reflectors
```

```
   Q = H(1)**H H(2)**H . . . H(k)**H, where k = min(m,n).

Each H(i) has the form

   H(i) = I - tau * v * v**H

where tau is a complex scalar, and v is a complex vector with
v(n-k+i+1:n) = 0 and v(n-k+i) = 1; conjg(v(1:n-k+i-1)) is stored on
exit in A(m-k+i,1:n-k+i-1), and tau in TAU(i).
```

34.27.3 fortran code

— cgerqf.f —

```
*  =====================================================================
      SUBROUTINE CGERQF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, KI, KK, LDWORK, LWKOPT,
     $                   MU, NB, NBMIN, NU, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGERQ2, CLARFB, CLARFT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
```

```
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
*
      IF( INFO.EQ.0 ) THEN
         K = MIN( M, N )
         IF( K.EQ.0 ) THEN
            LWKOPT = 1
         ELSE
            NB = ILAENV( 1, 'CGERQF', ' ', M, N, -1, -1 )
            LWKOPT = M*NB
         END IF
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.MAX( 1, M ) .AND. .NOT.LQUERY ) THEN
            INFO = -7
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGERQF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( K.EQ.0 ) THEN
         RETURN
      END IF
*
      NBMIN = 2
      NX = 1
      IWS = M
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
```

```
*
         NX = MAX( 0, ILAENV( 3, 'CGERQF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = M
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'CGERQF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially.
*        The last kk rows are handled by the block method.
*
         KI = ( ( K-NX-1 ) / NB )*NB
         KK = MIN( K, KI+NB )
*
         DO 10 I = K - KK + KI + 1, K - KK + 1, -NB
            IB = MIN( K-I+1, NB )
*
*           Compute the RQ factorization of the current block
*           A(m-k+i:m-k+i+ib-1,1:n-k+i+ib-1)
*
            CALL CGERQ2( IB, N-K+I+IB-1, A( M-K+I, 1 ), LDA, TAU( I ),
     $                   WORK, IINFO )
            IF( M-K+I.GT.1 ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i+ib-1) . . . H(i+1) H(i)
*
               CALL CLARFT( 'Backward', 'Rowwise', N-K+I+IB-1, IB,
     $                      A( M-K+I, 1 ), LDA, TAU( I ), WORK, LDWORK )
*
*              Apply H to A(1:m-k+i-1,1:n-k+i+ib-1) from the right
*
               CALL CLARFB( 'Right', 'No transpose', 'Backward',
     $                      'Rowwise', M-K+I-1, N-K+I+IB-1, IB,
     $                      A( M-K+I, 1 ), LDA, WORK, LDWORK, A, LDA,
     $                      WORK( IB+1 ), LDWORK )
```

```
            END IF
   10    CONTINUE
         MU = M - K + I + NB - 1
         NU = N - K + I + NB - 1
      ELSE
         MU = M
         NU = N
      END IF
*
*     Use unblocked code to factor the last or only block
*
      IF( MU.GT.0 .AND. NU.GT.0 )
     $   CALL CGERQ2( MU, NU, A, LDA, TAU, WORK, IINFO )
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of CGERQF
*
      END
```

34.27.4 lisp code

— cgerqf.lisp —

```
(defun cgerqf (m n a lda tau work lwork info)
  (declare (type (array f2cl-lib:complex8 (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex8 a-%data% a-%offset%)
       (tau f2cl-lib:complex8 tau-%data% tau-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ki 0) (kk 0) (ldwork 0)
           (lwkopt 0) (mu 0) (nb 0) (nbmin 0) (nu 0) (nx 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) nx nu nbmin nb mu lwkopt ldwork kk ki
                k iws iinfo ib i))
      (setf info 0)
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4)))
      (cond
       ((= info 0)
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (cond ((= k 0) (setf lwkopt 1))
```

```
        (t
         (setf nb
                 (multiple-value-bind
                     (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                     (ilaenv 1 "CGERQF" " " m n -1 -1)
                   (declare (ignore var-0 var-1 var-2 var-5 var-6))
                   (when var-3 (setf m var-3))
                   (when var-4 (setf n var-4))
                   ret-val))
         (setf lwkopt (f2cl-lib:int-mul m nb))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwkopt 'f2cl-lib:complex8))
  (cond
   ((and
     (< lwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
     (not lquery))
    (setf info -7)))))
(cond
 ((/= info 0) (xerbla "CGERQF" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond ((= k 0) (go end_label)))
(setf nbmin 2)
(setf nx 1)
(setf iws m)
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "CGERQF" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (when var-3 (setf m var-3))
                      (when var-4 (setf n var-4))
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork m) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
                            (ilaenv 2 "CGERQF" " " m n -1 -1)
                          (declare
```

```
                              (ignore var-0 var-1 var-2 var-5 var-6))
                             (when var-3 (setf m var-3))
                             (when var-4 (setf n var-4))
                             ret-val)))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (setf ki (* (the f2cl-lib:integer4 (truncate (- k nx 1) nb)) nb))
  (setf kk
          (min (the f2cl-lib:integer4 k)
               (the f2cl-lib:integer4 (f2cl-lib:int-add ki nb))))
  (f2cl-lib:fdo (i (f2cl-lib:int-add k (f2cl-lib:int-sub kk) ki 1)
                 (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
                ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub kk) 1)) nil)
    (tagbody
      (setf ib
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
               (the f2cl-lib:integer4 nb)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (cgerq2 ib
           (f2cl-lib:int-sub
            (f2cl-lib:int-add (f2cl-lib:int-sub n k) i ib) 1)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 ((+ m (f2cl-lib:int-sub k) i) 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i) ((1 *))
                                 tau-%offset%)
           work iinfo)
        (declare (ignore var-0 var-1 var-2 var-4 var-5))
        (setf lda var-3)
        (setf iinfo var-6))
      (cond
       ((> (f2cl-lib:int-add m (f2cl-lib:int-sub k) i) 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (clarft "Backward" "Rowwise"
             (f2cl-lib:int-sub
              (f2cl-lib:int-add (f2cl-lib:int-sub n k) i ib) 1)
             ib
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   ((+ m (f2cl-lib:int-sub k) i) 1)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice tau-%data% f2cl-lib:complex8 (i)
                                   ((1 *)) tau-%offset%)
             work ldwork)
          (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
          (when var-3 (setf ib var-3))
```

```
                (when var-5 (setf lda var-5))
                (when var-8 (setf ldwork var-8)))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                   var-10 var-11 var-12 var-13 var-14)
                  (clarfb "Right" "No transpose" "Backward" "Rowwise"
                   (f2cl-lib:int-sub
                    (f2cl-lib:int-add (f2cl-lib:int-sub m k) i) 1)
                   (f2cl-lib:int-sub
                    (f2cl-lib:int-add (f2cl-lib:int-sub n k) i ib) 1)
                   ib
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                         ((+ m (f2cl-lib:int-sub k) i) 1)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda work ldwork a lda
                   (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                         ((+ ib 1)) ((1 *)) work-%offset%)
                   ldwork)
                (declare
                 (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
                  var-13))
                (when var-6 (setf ib var-6))
                (when var-8 (setf lda var-8))
                (when var-10 (setf ldwork var-10))
                (when var-12 (setf lda var-12))
                (when var-14 (setf ldwork var-14)))))
          label10))
        (setf mu
                (f2cl-lib:int-sub
                 (f2cl-lib:int-add (f2cl-lib:int-sub m k) i nb) 1))
        (setf nu
                (f2cl-lib:int-sub
                 (f2cl-lib:int-add (f2cl-lib:int-sub n k) i nb) 1)))
       (t (setf mu m) (setf nu n)))
      (if (and (> mu 0) (> nu 0))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (cgerq2 mu nu a lda tau work iinfo)
            (declare (ignore var-0 var-1 var-2 var-4 var-5))
            (setf lda var-3)
            (setf iinfo var-6)))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce iws 'f2cl-lib:complex8))
      (go end_label)
     end_label
      (return (values m n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgerqf
```

```
        fortran-to-lisp::*f2cl-function-info*)
    (fortran-to-lisp::make-f2cl-finfo :arg-types
                                      '((fortran-to-lisp::integer4)
                                        (fortran-to-lisp::integer4)
                                        (array fortran-to-lisp::complex8
                                         (*))
                                        (fortran-to-lisp::integer4)
                                        (array fortran-to-lisp::complex8
                                         (*))
                                        (array fortran-to-lisp::complex8
                                         (*))
                                        (fortran-to-lisp::integer4)
                                        (fortran-to-lisp::integer4))
                                      :return-values
                                      '(fortran-to-lisp::m
                                        fortran-to-lisp::n nil
                                        fortran-to-lisp::lda nil nil nil
                                        fortran-to-lisp::info)
                                      :calls '(fortran-to-lisp::cgerq2))))
```

———-

34.28 cgesvj

34.28.1 Axiom unit tests

— cgesvj.input —

```
)set break resume
)sys rm -f cgesvj.output
)spool cgesvj.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.28.2 Axiom help page

— cgesvj.help —

```
====================================================================
cgesvj examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgesvj (JOBA, JOBU, JOBV, M, N, A, LDA, SVA, MV, V, LDV,
           CWORK, LWORK, RWORK, LRWORK, INFO)
           CGESVJ

Function/Subroutine Documentation
   subroutine cgesvj (character*1 JOBA, character*1 JOBU, character*1 JOBV,
       integer M, integer N, complex, dimension( lda, * ) A, integer LDA,
       real, dimension( n ) SVA, integer MV, complex, dimension( ldv, * ) V,
       integer LDV, complex, dimension( lwork ) CWORK, integer LWORK, real,
       dimension( lrwork ) RWORK, integer LRWORK, integer INFO)
       CGESVJ

Purpose:

Parameters:

   JOBA
   JOBA is CHARACTER* 1
   Specifies the structure of A.
   = 'L': The input matrix A is lower triangular;
   = 'U': The input matrix A is upper triangular;
   = 'G': The input matrix A is general M-by-N matrix, M >= N.

 JOBU
   JOBU is CHARACTER*1
   Specifies whether to compute the left singular vectors
   (columns of U):
   = 'U': The left singular vectors corresponding to the nonzero
      singular values are computed and returned in the leading
      columns of A. See more details in the description of A.
      The default numerical orthogonality threshold is set to
      approximately TOL=CTOL*EPS, CTOL=SQRT(M), EPS=SLAMCH('E').
   = 'C': Analogous to JOBU='U', except that user can control the
      level of numerical orthogonality of the computed left
      singular vectors. TOL can be set to TOL = CTOL*EPS, where
      CTOL is given on input in the array WORK.
      No CTOL smaller than ONE is allowed. CTOL greater
      than 1 / EPS is meaningless. The option 'C'
      can be used if M*EPS is satisfactory orthogonality
      of the computed left singular vectors, so CTOL=M could
```

```
     save few sweeps of Jacobi rotations.
     See the descriptions of A and WORK(1).
  = 'N': The matrix U is not computed. However, see the description of A.

JOBV
  JOBV is CHARACTER*1
  Specifies whether to compute the right singular vectors, that
  is, the matrix V:
  = 'V' : the matrix V is computed and returned in the array V
  = 'A' : the Jacobi rotations are applied to the MV-by-N
    array V. In other words, the right singular vector
    matrix V is not computed explicitly; instead it is
    applied to an MV-by-N matrix initially stored in the
    first MV rows of V.
  = 'N' : the matrix V is not computed and the array V is not referenced

M
  M is INTEGER
  The number of rows of the input matrix A. 1/SLAMCH('E') > M >= 0.

N
  N is INTEGER
  The number of columns of the input matrix A.
  M >= N >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the M-by-N matrix A.
  On exit,
  If JOBU .EQ. 'U' .OR. JOBU .EQ. 'C':
    If INFO .EQ. 0 :
      RANKA orthonormal columns of U are returned in the
      leading RANKA columns of the array A. Here RANKA <= N
      is the number of computed singular values of A that are
      above the underflow threshold SLAMCH('S'). The singular
      vectors corresponding to underflowed or zero singular
      values are not computed. The value of RANKA is returned
      in the array RWORK as RANKA=NINT(RWORK(2)). Also see the
      descriptions of SVA and RWORK. The computed columns of U
      are mutually numerically orthogonal up to approximately
      TOL=SQRT(M)*EPS (default); or TOL=CTOL*EPS (JOBU.EQ.'C'),
      see the description of JOBU.
    If INFO .GT. 0,
     the procedure CGESVJ did not converge in the given number
     of iterations (sweeps). In that case, the computed
     columns of U may not be orthogonal up to TOL. The output
     U (stored in A), SIGMA (given by the computed singular
     values in SVA(1:N)) and V is still a decomposition of the
     input matrix A in the sense that the residual
        || A - SCALE * U * SIGMA * V^* ||_2 / ||A||_2 is small.
```

```
  If JOBU .EQ. 'N':
    If INFO .EQ. 0 :
      Note that the left singular vectors are 'for free' in the
      one-sided Jacobi SVD algorithm. However, if only the
      singular values are needed, the level of numerical
      orthogonality of U is not an issue and iterations are
      stopped when the columns of the iterated matrix are
      numerically orthogonal up to approximately M*EPS. Thus,
      on exit, A contains the columns of U scaled with the
      corresponding singular values.
    If INFO .GT. 0 :
      the procedure CGESVJ did not converge in the given number
      of iterations (sweeps).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

  SVA
  SVA is REAL array, dimension (N)
  On exit,
  If INFO .EQ. 0 :
     depending on the value SCALE = RWORK(1), we have:
     If SCALE .EQ. ONE:
       SVA(1:N) contains the computed singular values of A.
       During the computation SVA contains the Euclidean column
       norms of the iterated matrices in the array A.
     If SCALE .NE. ONE:
       The singular values of A are SCALE*SVA(1:N), and this
       factored representation is due to the fact that some of the
       singular values of A might underflow or overflow.
  If INFO .GT. 0 :
    the procedure CGESVJ did not converge in the given number of
    iterations (sweeps) and SCALE*SVA(1:N) may not be accurate.

MV
  MV is INTEGER
  If JOBV .EQ. 'A', then the product of Jacobi rotations in CGESVJ
  is applied to the first MV rows of V. See the description of JOBV.

V
  V is COMPLEX array, dimension (LDV,N)
  If JOBV = 'V', then V contains on exit the N-by-N matrix of
       the right singular vectors;
  If JOBV = 'A', then V contains the product of the computed right
       singular vector matrix and the initial matrix in the array V.
  If JOBV = 'N', then V is not referenced.

LDV
  LDV is INTEGER
```

```
  The leading dimension of the array V, LDV .GE. 1.
  If JOBV .EQ. 'V', then LDV .GE. max(1,N).
  If JOBV .EQ. 'A', then LDV .GE. max(1,MV) .

CWORK
  CWORK is COMPLEX array, dimension M+N. Used as work space. LWORK

LWORK is INTEGER
  Length of CWORK, LWORK >= M+N.

RWORK
  RWORK is REAL array, dimension max(6,M+N).
  On entry,
  If JOBU .EQ. 'C' :
  RWORK(1) = CTOL, where CTOL defines the threshold for convergence.
  The process stops if all columns of A are mutually
  orthogonal up to CTOL*EPS, EPS=SLAMCH('E').
  It is required that CTOL >= ONE, i.e. it is not
  allowed to force the routine to obtain orthogonality
  below EPSILON.
  On exit,
  RWORK(1) = SCALE is the scaling factor such that SCALE*SVA(1:N)
     are the computed singular values of A.
     (See description of SVA().)
  RWORK(2) = NINT(RWORK(2)) is the number of the computed nonzero
     singular values.
  RWORK(3) = NINT(RWORK(3)) is the number of the computed singular
     values that are larger than the underflow threshold.
  RWORK(4) = NINT(RWORK(4)) is the number of sweeps of Jacobi
     rotations needed for numerical convergence.
  RWORK(5) = max_{i.NE.j} |COS(A(:,i),A(:,j))| in the last sweep.
     This is useful information in cases when CGESVJ did
     not converge, as it can be used to estimate whether
     the output is stil useful and for post festum analysis.
  RWORK(6) = the largest absolute value over all sines of the
     Jacobi rotation angles in the last sweep. It can be
     useful for a post festum analysis.

LRWORK
  LRWORK is INTEGER Length of RWORK, LRWORK >= MAX(6,N).

INFO
  INFO is INTEGER
  = 0 : successful exit.
  < 0 : if INFO = -i, then the i-th argument had an illegal value
  > 0 : CGESVJ did not converge in the maximal allowed number
        (NSWEEP=30) of sweeps. The output may still be useful.
        See the description of RWORK.

Further Details:
```

```
The orthogonal N-by-N matrix V is obtained as a product of Jacobi
plane rotations. In the case of underflow of the tangent of the
Jacobi angle, a modified Jacobi transformation of Drmac [3] is
used. Pivot strategy uses column interchanges of de Rijk [1]. The
relative accuracy of the computed singular values and the accuracy
of the computed singular vectors (in angle metric) is as guaranteed
by the theory of Demmel and Veselic [2]. The condition number that
determines the accuracy in the full rank case is essentially
min_{D=diag} kappa(A*D), where kappa(.) is the spectral condition
number. The best performance of this Jacobi SVD procedure is
achieved if used in an accelerated version of Drmac and Veselic
[4,5], and it is the kernel routine in the SIGMA library [6]. Some
tunning parameters (marked with [TP]) are available for the
implementer. The computational range for the nonzero singular
values is the machine number interval ( UNDERFLOW , OVERFLOW ). In
extreme cases, even denormalized singular values can be computed
with the corresponding gradual loss of accurate digits.
```

34.28.3 fortran code

— cgesvj.f —

```
*  ====================================================================
      SUBROUTINE CGESVJ( JOBA, JOBU, JOBV, M, N, A, LDA, SVA, MV, V,
     $                   LDV, CWORK, LWORK, RWORK, LRWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
      IMPLICIT NONE
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDV, LWORK, LRWORK, M, MV, N
      CHARACTER*1        JOBA, JOBU, JOBV
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ),  V( LDV, * ), CWORK( LWORK )
      REAL               RWORK( LRWORK ), SVA( N )
*     ..
*
*  ====================================================================
*
*     .. Local Parameters ..
```

```
      REAL         ZERO,         HALF,         ONE
      PARAMETER  ( ZERO = 0.0E0, HALF = 0.5E0, ONE = 1.0E0)
      COMPLEX      CZERO,                  CONE
      PARAMETER  ( CZERO = (0.0E0, 0.0E0), CONE = (1.0E0, 0.0E0) )
      INTEGER      NSWEEP
      PARAMETER  ( NSWEEP = 30 )
*     ..
*     .. Local Scalars ..
      COMPLEX AAPQ, OMPQ
      REAL    AAPP, AAPP0, AAPQ1, AAQQ, APOAQ, AQOAP, BIG,
     $        BIGTHETA, CS, CTOL, EPSLN, LARGE, MXAAPQ,
     $        MXSINJ, ROOTBIG, ROOTEPS, ROOTSFMIN, ROOTTOL,
     $        SKL, SFMIN, SMALL, SN, T, TEMP1, THETA, THSIGN, TOL
      INTEGER BLSKIP, EMPTSW, i, ibr, IERR, igl, IJBLSK, ir1,
     $        ISWROT, jbc, jgl, KBL, LKAHEAD, MVL, N2, N34,
     $        N4, NBL, NOTROT, p, PSKIPPED, q, ROWSKIP, SWBAND
      LOGICAL APPLV, GOSCALE, LOWER, LSVEC, NOSCALE, ROTOK,
     $        RSVEC, UCTOL, UPPER
*     ..
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC ABS, AMAX1, AMIN1, CONJG, FLOAT, MIN0, MAX0,
     $          SIGN, SQRT
*     ..
*     .. External Functions ..
*     ..
*     from BLAS
      REAL                      SCNRM2
      COMPLEX            CDOTC
      EXTERNAL           CDOTC, SCNRM2
      INTEGER            ISAMAX
      EXTERNAL           ISAMAX
*     from LAPACK
      REAL               SLAMCH
      EXTERNAL           SLAMCH
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
*     ..
*     from BLAS
      EXTERNAL           CCOPY, CROT, CSSCAL, CSWAP
*     from LAPACK
      EXTERNAL           CLASCL, CLASET, CLASSQ, XERBLA
      EXTERNAL           CGSVJ0, CGSVJ1
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
```

```
      LSVEC = LSAME( JOBU, 'U' )
      UCTOL = LSAME( JOBU, 'C' )
      RSVEC = LSAME( JOBV, 'V' )
      APPLV = LSAME( JOBV, 'A' )
      UPPER = LSAME( JOBA, 'U' )
      LOWER = LSAME( JOBA, 'L' )
*
      IF( .NOT.( UPPER .OR. LOWER .OR. LSAME( JOBA, 'G' ) ) ) THEN
         INFO = -1
      ELSE IF( .NOT.( LSVEC .OR. UCTOL .OR. LSAME( JOBU, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( .NOT.( RSVEC .OR. APPLV .OR. LSAME( JOBV, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( M.LT.0 ) THEN
         INFO = -4
      ELSE IF( ( N.LT.0 ) .OR. ( N.GT.M ) ) THEN
         INFO = -5
      ELSE IF( LDA.LT.M ) THEN
         INFO = -7
      ELSE IF( MV.LT.0 ) THEN
         INFO = -9
      ELSE IF( ( RSVEC .AND. ( LDV.LT.N ) ) .OR.
     $          ( APPLV .AND. ( LDV.LT.MV ) ) ) THEN
         INFO = -11
      ELSE IF( UCTOL .AND. ( RWORK( 1 ).LE.ONE ) ) THEN
         INFO = -12
      ELSE IF( LWORK.LT.( M+N ) ) THEN
         INFO = -13
      ELSE IF( LRWORK.LT.MAX0( N, 6 ) ) THEN
         INFO = -15
      ELSE
         INFO = 0
      END IF
*
*     #:(
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGESVJ', -INFO )
         RETURN
      END IF
*
* #:) Quick return for void matrix
*
      IF( ( M.EQ.0 ) .OR. ( N.EQ.0 ) )RETURN
*
*     Set numerical parameters
*     The stopping criterion for Jacobi rotations is
*
*     max_{i<>j}|A(:,i)^* * A(:,j)| / (||A(:,i)||*||A(:,j)||) < CTOL*EPS
*
*     where EPS is the round-off and CTOL is defined as follows:
```

```
*
      IF( UCTOL ) THEN
*        ... user controlled
         CTOL = RWORK( 1 )
      ELSE
*        ... default
         IF( LSVEC .OR. RSVEC .OR. APPLV ) THEN
            CTOL = SQRT( FLOAT( M ) )
         ELSE
            CTOL = FLOAT( M )
         END IF
      END IF
*     ... and the machine dependent parameters are
*[!]  (Make sure that SLAMCH() works properly on the target machine.)
*
      EPSLN = SLAMCH( 'Epsilon' )
      ROOTEPS = SQRT( EPSLN )
      SFMIN = SLAMCH( 'SafeMinimum' )
      ROOTSFMIN = SQRT( SFMIN )
      SMALL = SFMIN / EPSLN
      BIG = SLAMCH( 'Overflow' )
*     BIG         = ONE    / SFMIN
      ROOTBIG = ONE / ROOTSFMIN
      LARGE = BIG / SQRT( FLOAT( M*N ) )
      BIGTHETA = ONE / ROOTEPS
*
      TOL = CTOL*EPSLN
      ROOTTOL = SQRT( TOL )
*
      IF( FLOAT( M )*EPSLN.GE.ONE ) THEN
         INFO = -4
         CALL XERBLA( 'CGESVJ', -INFO )
         RETURN
      END IF
*
*     Initialize the right singular vector matrix.
*
      IF( RSVEC ) THEN
         MVL = N
         CALL CLASET( 'A', MVL, N, CZERO, CONE, V, LDV )
      ELSE IF( APPLV ) THEN
         MVL = MV
      END IF
      RSVEC = RSVEC .OR. APPLV
*
*     Initialize SVA( 1:N ) = ( ||A e_i||_2, i = 1:N )
*(!)  If necessary, scale A to protect the largest singular value
*     from overflow. It is possible that saving the largest singular
*     value destroys the information about the small ones.
*     This initial scaling is almost minimal in the sense that the
```

```
*     goal is to make sure that no column norm overflows, and that
*     SQRT(N)*max_i SVA(i) does not overflow. If INFinite entries
*     in A are detected, the procedure returns with INFO=-6.
*
      SKL = ONE / SQRT( FLOAT( M )*FLOAT( N ) )
      NOSCALE = .TRUE.
      GOSCALE = .TRUE.
*
      IF( LOWER ) THEN
*        the input matrix is M-by-N lower triangular (trapezoidal)
         DO 1874 p = 1, N
            AAPP = ZERO
            AAQQ = ONE
            CALL CLASSQ( M-p+1, A( p, p ), 1, AAPP, AAQQ )
            IF( AAPP.GT.BIG ) THEN
               INFO = -6
               CALL XERBLA( 'CGESVJ', -INFO )
               RETURN
            END IF
            AAQQ = SQRT( AAQQ )
            IF( ( AAPP.LT.( BIG / AAQQ ) ) .AND. NOSCALE ) THEN
               SVA( p ) = AAPP*AAQQ
            ELSE
               NOSCALE = .FALSE.
               SVA( p ) = AAPP*( AAQQ*SKL )
               IF( GOSCALE ) THEN
                  GOSCALE = .FALSE.
                  DO 1873 q = 1, p - 1
                     SVA( q ) = SVA( q )*SKL
 1873             CONTINUE
               END IF
            END IF
 1874    CONTINUE
      ELSE IF( UPPER ) THEN
*        the input matrix is M-by-N upper triangular (trapezoidal)
         DO 2874 p = 1, N
            AAPP = ZERO
            AAQQ = ONE
            CALL CLASSQ( p, A( 1, p ), 1, AAPP, AAQQ )
            IF( AAPP.GT.BIG ) THEN
               INFO = -6
               CALL XERBLA( 'CGESVJ', -INFO )
               RETURN
            END IF
            AAQQ = SQRT( AAQQ )
            IF( ( AAPP.LT.( BIG / AAQQ ) ) .AND. NOSCALE ) THEN
               SVA( p ) = AAPP*AAQQ
            ELSE
               NOSCALE = .FALSE.
               SVA( p ) = AAPP*( AAQQ*SKL )
```

```
               IF( GOSCALE ) THEN
                  GOSCALE = .FALSE.
                  DO 2873 q = 1, p - 1
                     SVA( q ) = SVA( q )*SKL
 2873             CONTINUE
               END IF
            END IF
 2874    CONTINUE
      ELSE
*        the input matrix is M-by-N general dense
         DO 3874 p = 1, N
            AAPP = ZERO
            AAQQ = ONE
            CALL CLASSQ( M, A( 1, p ), 1, AAPP, AAQQ )
            IF( AAPP.GT.BIG ) THEN
               INFO = -6
               CALL XERBLA( 'CGESVJ', -INFO )
               RETURN
            END IF
            AAQQ = SQRT( AAQQ )
            IF( ( AAPP.LT.( BIG / AAQQ ) ) .AND. NOSCALE ) THEN
               SVA( p ) = AAPP*AAQQ
            ELSE
               NOSCALE = .FALSE.
               SVA( p ) = AAPP*( AAQQ*SKL )
               IF( GOSCALE ) THEN
                  GOSCALE = .FALSE.
                  DO 3873 q = 1, p - 1
                     SVA( q ) = SVA( q )*SKL
 3873             CONTINUE
               END IF
            END IF
 3874    CONTINUE
      END IF
*
      IF( NOSCALE )SKL = ONE
*
*     Move the smaller part of the spectrum from the underflow threshold
*(!)  Start by determining the position of the nonzero entries of the
*     array SVA() relative to ( SFMIN, BIG ).
*
      AAPP = ZERO
      AAQQ = BIG
      DO 4781 p = 1, N
         IF( SVA( p ).NE.ZERO )AAQQ = AMIN1( AAQQ, SVA( p ) )
         AAPP = AMAX1( AAPP, SVA( p ) )
 4781 CONTINUE
*
* #:) Quick return for zero matrix
*
```

```
      IF( AAPP.EQ.ZERO ) THEN
         IF( LSVEC )CALL CLASET( 'G', M, N, CZERO, CONE, A, LDA )
         RWORK( 1 ) = ONE
         RWORK( 2 ) = ZERO
         RWORK( 3 ) = ZERO
         RWORK( 4 ) = ZERO
         RWORK( 5 ) = ZERO
         RWORK( 6 ) = ZERO
         RETURN
      END IF
*
* #:) Quick return for one-column matrix
*
      IF( N.EQ.1 ) THEN
         IF( LSVEC )CALL CLASCL( 'G', 0, 0, SVA( 1 ), SKL, M, 1,
     $                           A( 1, 1 ), LDA, IERR )
         RWORK( 1 ) = ONE / SKL
         IF( SVA( 1 ).GE.SFMIN ) THEN
            RWORK( 2 ) = ONE
         ELSE
            RWORK( 2 ) = ZERO
         END IF
         RWORK( 3 ) = ZERO
         RWORK( 4 ) = ZERO
         RWORK( 5 ) = ZERO
         RWORK( 6 ) = ZERO
         RETURN
      END IF
*
*     Protect small singular values from underflow, and try to
*     avoid underflows/overflows in computing Jacobi rotations.
*
      SN = SQRT( SFMIN / EPSLN )
      TEMP1 = SQRT( BIG / FLOAT( N ) )
      IF( ( AAPP.LE.SN ) .OR. ( AAQQ.GE.TEMP1 ) .OR.
     $    ( ( SN.LE.AAQQ ) .AND. ( AAPP.LE.TEMP1 ) ) ) THEN
         TEMP1 = AMIN1( BIG, TEMP1 / AAPP )
*         AAQQ  = AAQQ*TEMP1
*         AAPP  = AAPP*TEMP1
      ELSE IF( ( AAQQ.LE.SN ) .AND. ( AAPP.LE.TEMP1 ) ) THEN
         TEMP1 = AMIN1( SN / AAQQ, BIG / ( AAPP*SQRT( FLOAT( N ) ) ) )
*         AAQQ  = AAQQ*TEMP1
*         AAPP  = AAPP*TEMP1
      ELSE IF( ( AAQQ.GE.SN ) .AND. ( AAPP.GE.TEMP1 ) ) THEN
         TEMP1 = AMAX1( SN / AAQQ, TEMP1 / AAPP )
*         AAQQ  = AAQQ*TEMP1
*         AAPP  = AAPP*TEMP1
      ELSE IF( ( AAQQ.LE.SN ) .AND. ( AAPP.GE.TEMP1 ) ) THEN
         TEMP1 = AMIN1( SN / AAQQ, BIG / ( SQRT( FLOAT( N ) )*AAPP ) )
*         AAQQ  = AAQQ*TEMP1
```

```
*         AAPP  = AAPP*TEMP1
      ELSE
         TEMP1 = ONE
      END IF
*
*     Scale, if necessary
*
      IF( TEMP1.NE.ONE ) THEN
         CALL SLASCL( 'G', 0, 0, ONE, TEMP1, N, 1, SVA, N, IERR )
      END IF
      SKL = TEMP1*SKL
      IF( SKL.NE.ONE ) THEN
         CALL CLASCL( JOBA, 0, 0, ONE, SKL, M, N, A, LDA, IERR )
         SKL = ONE / SKL
      END IF
*
*     Row-cyclic Jacobi SVD algorithm with column pivoting
*
      EMPTSW = ( N*( N-1 ) ) / 2
      NOTROT = 0

      DO 1868 q = 1, N
         CWORK( q ) = CONE
 1868 CONTINUE
*
*
*
      SWBAND = 3
*[TP] SWBAND is a tuning parameter [TP]. It is meaningful and effective
*     if CGESVJ is used as a computational routine in the preconditioned
*     Jacobi SVD algorithm CGEJSV. For sweeps i=1:SWBAND the procedure
*     works on pivots inside a band-like region around the diagonal.
*     The boundaries are determined dynamically, based on the number of
*     pivots above a threshold.
*
      KBL = MIN0( 8, N )
*[TP] KBL is a tuning parameter that defines the tile size in the
*     tiling of the p-q loops of pivot pairs. In general, an optimal
*     value of KBL depends on the matrix dimensions and on the
*     parameters of the computer's memory.
*
      NBL = N / KBL
      IF( ( NBL*KBL ).NE.N )NBL = NBL + 1
*
      BLSKIP = KBL**2
*[TP] BLKSKIP is a tuning parameter that depends on SWBAND and KBL.
*
      ROWSKIP = MIN0( 5, KBL )
*[TP] ROWSKIP is a tuning parameter.
*
```

```
      LKAHEAD = 1
*[TP] LKAHEAD is a tuning parameter.
*
*     Quasi block transformations, using the lower (upper) triangular
*     structure of the input matrix. The quasi-block-cycling usually
*     invokes cubic convergence. Big part of this cycle is done inside
*     canonical subspaces of dimensions less than M.
*
      IF( ( LOWER .OR. UPPER ) .AND. ( N.GT.MAX0( 64, 4*KBL ) ) ) THEN
*[TP] The number of partition levels and the actual partition are
*     tuning parameters.
         N4 = N / 4
         N2 = N / 2
         N34 = 3*N4
         IF( APPLV ) THEN
            q = 0
         ELSE
            q = 1
         END IF
*
         IF( LOWER ) THEN
*
*     This works very well on lower triangular matrices, in particular
*     in the framework of the preconditioned Jacobi SVD (xGEJSV).
*     The idea is simple:
*     [+ 0 0 0]   Note that Jacobi transformations of [0 0]
*     [+ + 0 0]                                       [0 0]
*     [+ + x 0]   actually work on [x 0]              [x 0]
*     [+ + x x]                    [x x].             [x x]
*
            CALL CGSVJ0( JOBV, M-N34, N-N34, A( N34+1, N34+1 ), LDA,
     $                   CWORK( N34+1 ), SVA( N34+1 ), MVL,
     $                   V( N34*q+1, N34+1 ), LDV, EPSLN, SFMIN, TOL,
     $                   2, CWORK( N+1 ), LWORK-N, IERR )

            CALL CGSVJ0( JOBV, M-N2, N34-N2, A( N2+1, N2+1 ), LDA,
     $                   CWORK( N2+1 ), SVA( N2+1 ), MVL,
     $                   V( N2*q+1, N2+1 ), LDV, EPSLN, SFMIN, TOL, 2,
     $                   CWORK( N+1 ), LWORK-N, IERR )

            CALL CGSVJ1( JOBV, M-N2, N-N2, N4, A( N2+1, N2+1 ), LDA,
     $                   CWORK( N2+1 ), SVA( N2+1 ), MVL,
     $                   V( N2*q+1, N2+1 ), LDV, EPSLN, SFMIN, TOL, 1,
     $                   CWORK( N+1 ), LWORK-N, IERR )
*
            CALL CGSVJ0( JOBV, M-N4, N2-N4, A( N4+1, N4+1 ), LDA,
     $                   CWORK( N4+1 ), SVA( N4+1 ), MVL,
     $                   V( N4*q+1, N4+1 ), LDV, EPSLN, SFMIN, TOL, 1,
     $                   CWORK( N+1 ), LWORK-N, IERR )
*
```

```
            CALL CGSVJ0( JOBV, M, N4, A, LDA, CWORK, SVA, MVL, V, LDV,
     $                   EPSLN, SFMIN, TOL, 1, CWORK( N+1 ), LWORK-N,
     $                   IERR )
*
            CALL CGSVJ1( JOBV, M, N2, N4, A, LDA, CWORK, SVA, MVL, V,
     $                   LDV, EPSLN, SFMIN, TOL, 1, CWORK( N+1 ),
     $                   LWORK-N, IERR )
*
*
         ELSE IF( UPPER ) THEN
*
*
            CALL CGSVJ0( JOBV, N4, N4, A, LDA, CWORK, SVA, MVL, V, LDV,
     $                   EPSLN, SFMIN, TOL, 2, CWORK( N+1 ), LWORK-N,
     $                   IERR )
*
            CALL CGSVJ0( JOBV, N2, N4, A( 1, N4+1 ), LDA, CWORK( N4+1 ),
     $                   SVA( N4+1 ), MVL, V( N4*q+1, N4+1 ), LDV,
     $                   EPSLN, SFMIN, TOL, 1, CWORK( N+1 ), LWORK-N,
     $                   IERR )
*
            CALL CGSVJ1( JOBV, N2, N2, N4, A, LDA, CWORK, SVA, MVL, V,
     $                   LDV, EPSLN, SFMIN, TOL, 1, CWORK( N+1 ),
     $                   LWORK-N, IERR )
*
            CALL CGSVJ0( JOBV, N2+N4, N4, A( 1, N2+1 ), LDA,
     $                   CWORK( N2+1 ), SVA( N2+1 ), MVL,
     $                   V( N2*q+1, N2+1 ), LDV, EPSLN, SFMIN, TOL, 1,
     $                   CWORK( N+1 ), LWORK-N, IERR )

         END IF
*
      END IF
*
*     .. Row-cyclic pivot strategy with de Rijk's pivoting ..
*
      DO 1993 i = 1, NSWEEP
*
*     .. go go go ...
*
         MXAAPQ = ZERO
         MXSINJ = ZERO
         ISWROT = 0
*
         NOTROT = 0
         PSKIPPED = 0
*
*     Each sweep is unrolled using KBL-by-KBL tiles over the pivot pairs
*     1 <= p < q <= N. This is the first step toward a blocked implementation
*     of the rotations. New implementation, based on block transformations,
```

```
*     is under development.
*
         DO 2000 ibr = 1, NBL
*
            igl = ( ibr-1 )*KBL + 1
*
            DO 1002 ir1 = 0, MIN0( LKAHEAD, NBL-ibr )
*
               igl = igl + ir1*KBL
*
               DO 2001 p = igl, MIN0( igl+KBL-1, N-1 )
*
*     .. de Rijk's pivoting
*
                  q = ISAMAX( N-p+1, SVA( p ), 1 ) + p - 1
                  IF( p.NE.q ) THEN
                     CALL CSWAP( M, A( 1, p ), 1, A( 1, q ), 1 )
                     IF( RSVEC )CALL CSWAP( MVL, V( 1, p ), 1,
     $                                       V( 1, q ), 1 )
                     TEMP1 = SVA( p )
                     SVA( p ) = SVA( q )
                     SVA( q ) = TEMP1
                     AAPQ = CWORK(p)
                     CWORK(p) = CWORK(q)
                     CWORK(q) = AAPQ
                  END IF
*
                  IF( ir1.EQ.0 ) THEN
*
*        Column norms are periodically updated by explicit
*        norm computation.
*[!]     Caveat:
*        Unfortunately, some BLAS implementations compute SCNRM2(M,A(1,p),1)
*        as SQRT(S=CDOTC(M,A(1,p),1,A(1,p),1)), which may cause the result to
*        overflow for ||A(:,p)||_2 > SQRT(overflow_threshold), and to
*        underflow for ||A(:,p)||_2 < SQRT(underflow_threshold).
*        Hence, SCNRM2 cannot be trusted, not even in the case when
*        the true norm is far from the under(over)flow boundaries.
*        If properly implemented SCNRM2 is available, the IF-THEN-ELSE-END IF
*        below should be replaced with "AAPP = SCNRM2( M, A(1,p), 1 )".
*
                     IF( ( SVA( p ).LT.ROOTBIG ) .AND.
     $                    ( SVA( p ).GT.ROOTSFMIN ) ) THEN
                        SVA( p ) = SCNRM2( M, A( 1, p ), 1 )
                     ELSE
                        TEMP1 = ZERO
                        AAPP = ONE
                        CALL CLASSQ( M, A( 1, p ), 1, TEMP1, AAPP )
                        SVA( p ) = TEMP1*SQRT( AAPP )
                     END IF
```

```
                 AAPP = SVA( p )
              ELSE
                 AAPP = SVA( p )
              END IF
*
              IF( AAPP.GT.ZERO ) THEN
*
                 PSKIPPED = 0
*
                 DO 2002 q = p + 1, MIN0( igl+KBL-1, N )
*
                    AAQQ = SVA( q )
*
                    IF( AAQQ.GT.ZERO ) THEN
*
                       AAPP0 = AAPP
                       IF( AAQQ.GE.ONE ) THEN
                          ROTOK = ( SMALL*AAPP ).LE.AAQQ
                          IF( AAPP.LT.( BIG / AAQQ ) ) THEN
                             AAPQ = ( CDOTC( M, A( 1, p ), 1,
     $                               A( 1, q ), 1 ) / AAQQ ) / AAPP
                          ELSE
                             CALL CCOPY( M, A( 1, p ), 1,
     $                                   CWORK(N+1), 1 )
                             CALL CLASCL( 'G', 0, 0, AAPP, ONE,
     $                            M, 1, CWORK(N+1), LDA, IERR )
                             AAPQ = CDOTC( M, CWORK(N+1), 1,
     $                               A( 1, q ), 1 ) / AAQQ
                          END IF
                       ELSE
                          ROTOK = AAPP.LE.( AAQQ / SMALL )
                          IF( AAPP.GT.( SMALL / AAQQ ) ) THEN
                             AAPQ = ( CDOTC( M, A( 1, p ), 1,
     $                                A( 1, q ), 1 ) / AAQQ ) / AAPP
                          ELSE
                             CALL CCOPY( M, A( 1, q ), 1,
     $                                   CWORK(N+1), 1 )
                             CALL CLASCL( 'G', 0, 0, AAQQ,
     $                                    ONE, M, 1,
     $                                    CWORK(N+1), LDA, IERR )
                             AAPQ = CDOTC( M, A(1, p ), 1,
     $                               CWORK(N+1), 1 ) / AAPP
                          END IF
                       END IF
*
                       OMPQ = AAPQ / ABS(AAPQ)
*                       AAPQ = AAPQ * CONJG( CWORK(p) ) * CWORK(q)
                       AAPQ1  = -ABS(AAPQ)
                       MXAAPQ = AMAX1( MXAAPQ, -AAPQ1 )
*
```

```
*        TO rotate or NOT to rotate, THAT is the question ...
*
                          IF( ABS( AAPQ1 ).GT.TOL ) THEN
*
*           .. rotate
*[RTD]      ROTATED = ROTATED + ONE
*
                             IF( ir1.EQ.0 ) THEN
                                NOTROT = 0
                                PSKIPPED = 0
                                ISWROT = ISWROT + 1
                             END IF
*
                             IF( ROTOK ) THEN
*
                                AQOAP = AAQQ / AAPP
                                APOAQ = AAPP / AAQQ
                                THETA = -HALF*ABS( AQOAP-APOAQ )/AAPQ1
*
                                IF( ABS( THETA ).GT.BIGTHETA ) THEN
*
                                   T  = HALF / THETA
                                   CS = ONE

                                   CALL CROT( M, A(1,p), 1, A(1,q), 1,
     $                                        CS, CONJG(OMPQ)*T )
                                   IF ( RSVEC ) THEN
                                       CALL CROT( MVL, V(1,p), 1,
     $                                 V(1,q), 1, CS, CONJG(OMPQ)*T )
                                   END IF

                                   SVA( q ) = AAQQ*SQRT( AMAX1( ZERO,
     $                                        ONE+T*APOAQ*AAPQ1 ) )
                                   AAPP = AAPP*SQRT( AMAX1( ZERO,
     $                                        ONE-T*AQOAP*AAPQ1 ) )
                                   MXSINJ = AMAX1( MXSINJ, ABS( T ) )
*
                                ELSE
*
*                 .. choose correct signum for THETA and rotate
*
                                   THSIGN = -SIGN( ONE, AAPQ1 )
                                   T = ONE / ( THETA+THSIGN*
     $                                  SQRT( ONE+THETA*THETA ) )
                                   CS = SQRT( ONE / ( ONE+T*T ) )
                                   SN = T*CS
*
                                   MXSINJ = AMAX1( MXSINJ, ABS( SN ) )
                                   SVA( q ) = AAQQ*SQRT( AMAX1( ZERO,
     $                                        ONE+T*APOAQ*AAPQ1 ) )
```

```
                                 AAPP = AAPP*SQRT( AMAX1( ZERO,
     $                                   ONE-T*AQOAP*AAPQ1 ) )
*
                                 CALL CROT( M, A(1,p), 1, A(1,q), 1,
     $                                      CS, CONJG(OMPQ)*SN )
                                 IF ( RSVEC ) THEN
                                     CALL CROT( MVL, V(1,p), 1,
     $                               V(1,q), 1, CS, CONJG(OMPQ)*SN )
                                 END IF
                              END IF
                              CWORK(p) = -CWORK(q) * OMPQ
*
                              ELSE
*             .. have to use modified Gram-Schmidt like transformation
                              CALL CCOPY( M, A( 1, p ), 1,
     $                                    CWORK(N+1), 1 )
                              CALL CLASCL( 'G', 0, 0, AAPP, ONE, M,
     $                                     1, CWORK(N+1), LDA,
     $                                     IERR )
                              CALL CLASCL( 'G', 0, 0, AAQQ, ONE, M,
     $                                     1, A( 1, q ), LDA, IERR )
                              CALL CAXPY( M, -AAPQ, CWORK(N+1), 1,
     $                                    A( 1, q ), 1 )
                              CALL CLASCL( 'G', 0, 0, ONE, AAQQ, M,
     $                                     1, A( 1, q ), LDA, IERR )
                              SVA( q ) = AAQQ*SQRT( AMAX1( ZERO,
     $                                   ONE-AAPQ1*AAPQ1 ) )
                              MXSINJ = AMAX1( MXSINJ, SFMIN )
                           END IF
*          END IF ROTOK THEN ... ELSE
*
*          In the case of cancellation in updating SVA(q), SVA(p)
*          recompute SVA(q), SVA(p).
*
                           IF( ( SVA( q ) / AAQQ )**2.LE.ROOTEPS )
     $                         THEN
                              IF( ( AAQQ.LT.ROOTBIG ) .AND.
     $                            ( AAQQ.GT.ROOTSFMIN ) ) THEN
                                 SVA( q ) = SCNRM2( M, A( 1, q ), 1 )
                              ELSE
                                 T = ZERO
                                 AAQQ = ONE
                                 CALL CLASSQ( M, A( 1, q ), 1, T,
     $                                        AAQQ )
                                 SVA( q ) = T*SQRT( AAQQ )
                              END IF
                           END IF
                           IF( ( AAPP / AAPP0 ).LE.ROOTEPS ) THEN
                              IF( ( AAPP.LT.ROOTBIG ) .AND.
     $                            ( AAPP.GT.ROOTSFMIN ) ) THEN
```

```
                                 AAPP = SCNRM2( M, A( 1, p ), 1 )
                              ELSE
                                 T = ZERO
                                 AAPP = ONE
                                 CALL CLASSQ( M, A( 1, p ), 1, T,
     $                                        AAPP )
                                 AAPP = T*SQRT( AAPP )
                              END IF
                              SVA( p ) = AAPP
                           END IF
*
                        ELSE
*                          A(:,p) and A(:,q) already numerically orthogonal
                           IF( ir1.EQ.0 )NOTROT = NOTROT + 1
*[RTD]      SKIPPED  = SKIPPED + 1
                           PSKIPPED = PSKIPPED + 1
                        END IF
                     ELSE
*                       A(:,q) is zero column
                        IF( ir1.EQ.0 )NOTROT = NOTROT + 1
                        PSKIPPED = PSKIPPED + 1
                     END IF
*
                     IF( ( i.LE.SWBAND ) .AND.
     $                    ( PSKIPPED.GT.ROWSKIP ) ) THEN
                        IF( ir1.EQ.0 )AAPP = -AAPP
                        NOTROT = 0
                        GO TO 2103
                     END IF
*
 2002                CONTINUE
*     END q-LOOP
*
 2103                CONTINUE
*     bailed out of q-loop
*
                     SVA( p ) = AAPP
*
                  ELSE
                     SVA( p ) = AAPP
                     IF( ( ir1.EQ.0 ) .AND. ( AAPP.EQ.ZERO ) )
     $                   NOTROT = NOTROT + MIN0( igl+KBL-1, N ) - p
                  END IF
*
 2001          CONTINUE
*     end of the p-loop
*     end of doing the block ( ibr, ibr )
 1002       CONTINUE
*     end of ir1-loop
*
```

```
* ... go to the off diagonal blocks
*
            igl = ( ibr-1 )*KBL + 1
*
            DO 2010 jbc = ibr + 1, NBL
*
               jgl = ( jbc-1 )*KBL + 1
*
*        doing the block at ( ibr, jbc )
*
               IJBLSK = 0
               DO 2100 p = igl, MIN0( igl+KBL-1, N )
*
                  AAPP = SVA( p )
                  IF( AAPP.GT.ZERO ) THEN
*
                     PSKIPPED = 0
*
                     DO 2200 q = jgl, MIN0( jgl+KBL-1, N )
*
                        AAQQ = SVA( q )
                        IF( AAQQ.GT.ZERO ) THEN
                           AAPP0 = AAPP
*
*     .. M x 2 Jacobi SVD ..
*
*        Safe Gram matrix computation
*
                           IF( AAQQ.GE.ONE ) THEN
                              IF( AAPP.GE.AAQQ ) THEN
                                 ROTOK = ( SMALL*AAPP ).LE.AAQQ
                              ELSE
                                 ROTOK = ( SMALL*AAQQ ).LE.AAPP
                              END IF
                              IF( AAPP.LT.( BIG / AAQQ ) ) THEN
                                 AAPQ = ( CDOTC( M, A( 1, p ), 1,
     $                                   A( 1, q ), 1 ) / AAQQ ) / AAPP
                              ELSE
                                 CALL CCOPY( M, A( 1, p ), 1,
     $                                       CWORK(N+1), 1 )
                                 CALL CLASCL( 'G', 0, 0, AAPP,
     $                                        ONE, M, 1,
     $                                        CWORK(N+1), LDA, IERR )
                                 AAPQ = CDOTC( M, CWORK(N+1), 1,
     $                                   A( 1, q ), 1 ) / AAQQ
                              END IF
                           ELSE
                              IF( AAPP.GE.AAQQ ) THEN
                                 ROTOK = AAPP.LE.( AAQQ / SMALL )
                              ELSE
```

```
                              ROTOK = AAQQ.LE.( AAPP / SMALL )
                           END IF
                           IF( AAPP.GT.( SMALL / AAQQ ) ) THEN
                              AAPQ = ( CDOTC( M, A( 1, p ), 1,
     $                                 A( 1, q ), 1 ) / AAQQ ) / AAPP
                           ELSE
                              CALL CCOPY( M, A( 1, q ), 1,
     $                                    CWORK(N+1), 1 )
                              CALL CLASCL( 'G', 0, 0, AAQQ,
     $                                     ONE, M, 1,
     $                                     CWORK(N+1), LDA, IERR )
                              AAPQ = CDOTC( M, A( 1, p ), 1,
     $                               CWORK(N+1),  1 ) / AAPP
                           END IF
                        END IF
*
                        OMPQ = AAPQ / ABS(AAPQ)
*                        AAPQ = AAPQ * CONJG(CWORK(p))*CWORK(q)
                        AAPQ1  = -ABS(AAPQ)
                        MXAAPQ = AMAX1( MXAAPQ, -AAPQ1 )
*
*       TO rotate or NOT to rotate, THAT is the question ...
*
                        IF( ABS( AAPQ1 ).GT.TOL ) THEN
                           NOTROT = 0
*[RTD]      ROTATED  = ROTATED + 1
                           PSKIPPED = 0
                           ISWROT = ISWROT + 1
*
                           IF( ROTOK ) THEN
*
                              AQOAP = AAQQ / AAPP
                              APOAQ = AAPP / AAQQ
                              THETA = -HALF*ABS( AQOAP-APOAQ )/ AAPQ1
                              IF( AAQQ.GT.AAPP0 )THETA = -THETA
*
                              IF( ABS( THETA ).GT.BIGTHETA ) THEN
                                 T  = HALF / THETA
                                 CS = ONE
                                 CALL CROT( M, A(1,p), 1, A(1,q), 1,
     $                                      CS, CONJG(OMPQ)*T )
                                 IF( RSVEC ) THEN
                                     CALL CROT( MVL, V(1,p), 1,
     $                               V(1,q), 1, CS, CONJG(OMPQ)*T )
                                 END IF
                                 SVA( q ) = AAQQ*SQRT( AMAX1( ZERO,
     $                                     ONE+T*APOAQ*AAPQ1 ) )
                                 AAPP = AAPP*SQRT( AMAX1( ZERO,
     $                                  ONE-T*AQOAP*AAPQ1 ) )
                                 MXSINJ = AMAX1( MXSINJ, ABS( T ) )
```

```
                                ELSE
*
*                 .. choose correct signum for THETA and rotate
*
                                   THSIGN = -SIGN( ONE, AAPQ1 )
                                   IF( AAQQ.GT.AAPP0 )THSIGN = -THSIGN
                                   T = ONE / ( THETA+THSIGN*
     $                                 SQRT( ONE+THETA*THETA ) )
                                   CS = SQRT( ONE / ( ONE+T*T ) )
                                   SN = T*CS
                                   MXSINJ = AMAX1( MXSINJ, ABS( SN ) )
                                   SVA( q ) = AAQQ*SQRT( AMAX1( ZERO,
     $                                       ONE+T*APOAQ*AAPQ1 ) )
                                   AAPP = AAPP*SQRT( AMAX1( ZERO,
     $                                       ONE-T*AQOAP*AAPQ1 ) )
*
                                   CALL CROT( M, A(1,p), 1, A(1,q), 1,
     $                                        CS, CONJG(OMPQ)*SN )
                                   IF( RSVEC ) THEN
                                       CALL CROT( MVL, V(1,p), 1,
     $                                 V(1,q), 1, CS, CONJG(OMPQ)*SN )
                                   END IF
                                END IF
                                CWORK(p) = -CWORK(q) * OMPQ
*
                              ELSE
*               .. have to use modified Gram-Schmidt like transformation
                               IF( AAPP.GT.AAQQ ) THEN
                                   CALL CCOPY( M, A( 1, p ), 1,
     $                                         CWORK(N+1), 1 )
                                   CALL CLASCL( 'G', 0, 0, AAPP, ONE,
     $                                          M, 1, CWORK(N+1),LDA,
     $                                          IERR )
                                   CALL CLASCL( 'G', 0, 0, AAQQ, ONE,
     $                                          M, 1, A( 1, q ), LDA,
     $                                          IERR )
                                   CALL CAXPY( M, -AAPQ, CWORK(N+1),
     $                                         1, A( 1, q ), 1 )
                                   CALL CLASCL( 'G', 0, 0, ONE, AAQQ,
     $                                          M, 1, A( 1, q ), LDA,
     $                                          IERR )
                                   SVA( q ) = AAQQ*SQRT( AMAX1( ZERO,
     $                                       ONE-AAPQ1*AAPQ1 ) )
                                   MXSINJ = AMAX1( MXSINJ, SFMIN )
                               ELSE
                                  CALL CCOPY( M, A( 1, q ), 1,
     $                                         CWORK(N+1), 1 )
                                   CALL CLASCL( 'G', 0, 0, AAQQ, ONE,
     $                                          M, 1, CWORK(N+1),LDA,
     $                                          IERR )
```

```
                                 CALL CLASCL( 'G', 0, 0, AAPP, ONE,
     $                                     M, 1, A( 1, p ), LDA,
     $                                     IERR )
                                 CALL CAXPY( M, -CONJG(AAPQ),
     $                                CWORK(N+1), 1, A( 1, p ), 1 )
                                 CALL CLASCL( 'G', 0, 0, ONE, AAPP,
     $                                     M, 1, A( 1, p ), LDA,
     $                                     IERR )
                                 SVA( p ) = AAPP*SQRT( AMAX1( ZERO,
     $                                   ONE-AAPQ1*AAPQ1 ) )
                                 MXSINJ = AMAX1( MXSINJ, SFMIN )
                              END IF
                            END IF
*          END IF ROTOK THEN ... ELSE
*
*          In the case of cancellation in updating SVA(q), SVA(p)
*          .. recompute SVA(q), SVA(p)
                            IF( ( SVA( q ) / AAQQ )**2.LE.ROOTEPS )
     $                          THEN
                               IF( ( AAQQ.LT.ROOTBIG ) .AND.
     $                             ( AAQQ.GT.ROOTSFMIN ) ) THEN
                                 SVA( q ) = SCNRM2( M, A( 1, q ), 1)
                                THEN
                                 T = ZERO
                                 AAQQ = ONE
                                 CALL CLASSQ( M, A( 1, q ), 1, T,
     $                                     AAQQ )
                                 SVA( q ) = T*SQRT( AAQQ )
                               END IF
                            END IF
                            IF( ( AAPP / AAPP0 )**2.LE.ROOTEPS ) THEN
                               IF( ( AAPP.LT.ROOTBIG ) .AND.
     $                             ( AAPP.GT.ROOTSFMIN ) ) THEN
                                 AAPP = SCNRM2( M, A( 1, p ), 1 )
                               ELSE
                                 T = ZERO
                                 AAPP = ONE
                                 CALL CLASSQ( M, A( 1, p ), 1, T,
     $                                     AAPP )
                                 AAPP = T*SQRT( AAPP )
                               END IF
                               SVA( p ) = AAPP
                            END IF
*             end of OK rotation
                         ELSE
                            NOTROT = NOTROT + 1
*[RTD]      SKIPPED  = SKIPPED  + 1
                            PSKIPPED = PSKIPPED + 1
                            IJBLSK = IJBLSK + 1
                         END IF
```

```
                        ELSE
                           NOTROT = NOTROT + 1
                           PSKIPPED = PSKIPPED + 1
                           IJBLSK = IJBLSK + 1
                        END IF
*
                        IF( ( i.LE.SWBAND ) .AND. ( IJBLSK.GE.BLSKIP ) )
     $                     THEN
                           SVA( p ) = AAPP
                           NOTROT = 0
                           GO TO 2011
                        END IF
                        IF( ( i.LE.SWBAND ) .AND.
     $                     ( PSKIPPED.GT.ROWSKIP ) ) THEN
                           AAPP = -AAPP
                           NOTROT = 0
                           GO TO 2203
                        END IF
*
 2200                CONTINUE
*        end of the q-loop
 2203                CONTINUE
*
                     SVA( p ) = AAPP
*
                  ELSE
*
                     IF( AAPP.EQ.ZERO )NOTROT = NOTROT +
     $                   MIN0( jgl+KBL-1, N ) - jgl + 1
                     IF( AAPP.LT.ZERO )NOTROT = 0
*
                  END IF
*
 2100          CONTINUE
*     end of the p-loop
 2010       CONTINUE
*     end of the jbc-loop
 2011       CONTINUE
*2011 bailed out of the jbc-loop
            DO 2012 p = igl, MIN0( igl+KBL-1, N )
               SVA( p ) = ABS( SVA( p ) )
 2012       CONTINUE
***
 2000    CONTINUE
*2000 :: end of the ibr-loop
*
*     .. update SVA(N)
         IF( ( SVA( N ).LT.ROOTBIG ) .AND. ( SVA( N ).GT.ROOTSFMIN ) )
     $       THEN
            SVA( N ) = SCNRM2( M, A( 1, N ), 1 )
```

```
         ELSE
            T = ZERO
            AAPP = ONE
            CALL CLASSQ( M, A( 1, N ), 1, T, AAPP )
            SVA( N ) = T*SQRT( AAPP )
         END IF
*
*     Additional steering devices
*
         IF( ( i.LT.SWBAND ) .AND. ( ( MXAAPQ.LE.ROOTTOL ) .OR.
     $       ( ISWROT.LE.N ) ) )SWBAND = i
*
         IF( ( i.GT.SWBAND+1 ) .AND. ( MXAAPQ.LT.SQRT( FLOAT( N ) )*
     $       TOL ) .AND. ( FLOAT( N )*MXAAPQ*MXSINJ.LT.TOL ) ) THEN
            GO TO 1994
         END IF
*
         IF( NOTROT.GE.EMPTSW )GO TO 1994
*
 1993 CONTINUE
*     end i=1:NSWEEP loop
*
* #:( Reaching this point means that the procedure has not converged.
      INFO = NSWEEP - 1
      GO TO 1995
*
 1994 CONTINUE
* #:) Reaching this point means numerical convergence after the i-th
*     sweep.
*
      INFO = 0
* #:) INFO = 0 confirms successful iterations.
 1995 CONTINUE
*
*     Sort the singular values and find how many are above
*     the underflow threshold.
*
      N2 = 0
      N4 = 0
      DO 5991 p = 1, N - 1
         q = ISAMAX( N-p+1, SVA( p ), 1 ) + p - 1
         IF( p.NE.q ) THEN
            TEMP1 = SVA( p )
            SVA( p ) = SVA( q )
            SVA( q ) = TEMP1
            CALL CSWAP( M, A( 1, p ), 1, A( 1, q ), 1 )
            IF( RSVEC )CALL CSWAP( MVL, V( 1, p ), 1, V( 1, q ), 1 )
         END IF
         IF( SVA( p ).NE.ZERO ) THEN
            N4 = N4 + 1
```

```
            IF( SVA( p )*SKL.GT.SFMIN )N2 = N2 + 1
         END IF
 5991 CONTINUE
      IF( SVA( N ).NE.ZERO ) THEN
         N4 = N4 + 1
         IF( SVA( N )*SKL.GT.SFMIN )N2 = N2 + 1
      END IF
*
*     Normalize the left singular vectors.
*
      IF( LSVEC .OR. UCTOL ) THEN
         DO 1998 p = 1, N2
            CALL CSSCAL( M, ONE / SVA( p ), A( 1, p ), 1 )
 1998    CONTINUE
      END IF
*
*     Scale the product of Jacobi rotations.
*
      IF( RSVEC ) THEN
            DO 2399 p = 1, N
               TEMP1 = ONE / SCNRM2( MVL, V( 1, p ), 1 )
               CALL CSSCAL( MVL, TEMP1, V( 1, p ), 1 )
 2399       CONTINUE
      END IF
*
*     Undo scaling, if necessary (and possible).
      IF( ( ( SKL.GT.ONE ) .AND. ( SVA( 1 ).LT.( BIG / SKL ) ) )
     $    .OR. ( ( SKL.LT.ONE ) .AND. ( SVA( MAX( N2, 1 ) ) .GT.
     $    ( SFMIN / SKL ) ) ) ) THEN
         DO 2400 p = 1, N
            SVA( P ) = SKL*SVA( P )
 2400    CONTINUE
         SKL = ONE
      END IF
*
      RWORK( 1 ) = SKL
*     The singular values of A are SKL*SVA(1:N). If SKL.NE.ONE
*     then some of the singular values may overflow or underflow and
*     the spectrum is given in this factored representation.
*
      RWORK( 2 ) = FLOAT( N4 )
*     N4 is the number of computed nonzero singular values of A.
*
      RWORK( 3 ) = FLOAT( N2 )
*     N2 is the number of singular values of A greater than SFMIN.
*     If N2<N, SVA(N2:N) contains ZEROS and/or denormalized numbers
*     that may carry some information.
*
      RWORK( 4 ) = FLOAT( i )
*     i is the index of the last sweep before declaring convergence.
```

```
*
      RWORK( 5 ) = MXAAPQ
*     MXAAPQ is the largest absolute value of scaled pivots in the
*     last sweep
*
      RWORK( 6 ) = MXSINJ
*     MXSINJ is the largest absolute value of the sines of Jacobi angles
*     in the last sweep
*
      RETURN
*     ..
*     .. END OF CGESVJ
*     ..
      END
```

34.28.4 lisp code

— cgesvj.lisp —

```
(let* ((zero 0.0)
       (half 0.5)
       (one 1.0)
       (czero (f2cl-lib:cmplx 0.0 0.0))
       (cone (f2cl-lib:cmplx 1.0 0.0))
       (nsweep 30))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 0.5 0.5) half)
           (type (single-float 1.0 1.0) one)
           (type (f2cl-lib:complex8) czero)
           (type (f2cl-lib:complex8) cone)
           (type (f2cl-lib:integer4 30 30) nsweep)
           (ignorable zero half one czero cone nsweep))
  (defun cgesvj
         (joba jobu jobv m n a lda sva mv v ldv cwork lwork rwork lrwork info)
    (declare (type (array single-float (*)) rwork sva)
             (type (array f2cl-lib:complex8 (*)) cwork v a)
             (type (f2cl-lib:integer4) info lrwork lwork ldv mv lda n m)
             (type (string *) jobv jobu joba))
    (f2cl-lib:with-multi-array-data
        ((joba character joba-%data% joba-%offset%)
         (jobu character jobu-%data% jobu-%offset%)
         (jobv character jobv-%data% jobv-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (v f2cl-lib:complex8 v-%data% v-%offset%)
         (cwork f2cl-lib:complex8 cwork-%data% cwork-%offset%)
         (sva single-float sva-%data% sva-%offset%)
```

```
    (rwork single-float rwork-%data% rwork-%offset%))
  (prog ((applv nil) (goscale nil) (lower nil) (lsvec nil) (noscale nil)
         (rotok nil) (rsvec nil) (uctol nil) (upper nil) (blskip 0)
         (emptsw 0) (i 0) (ibr 0) (ierr 0) (igl 0) (ijblsk 0) (ir1 0)
         (iswrot 0) (jbc 0) (jgl 0) (kbl 0) (lkahead 0) (mvl 0) (n2 0)
         (n34 0) (n4 0) (nbl 0) (notrot 0) (p 0) (pskipped 0) (q 0)
         (rowskip 0) (swband 0) (aapp 0.0) (aapp0 0.0) (aapq1 0.0)
         (aaqq 0.0) (apoaq 0.0) (aqoap 0.0) (big 0.0) (bigtheta 0.0)
         (cs 0.0) (ctol 0.0) (epsln 0.0) (large 0.0) (mxaapq 0.0)
         (mxsinj 0.0) (rootbig 0.0) (rooteps 0.0) (rootsfmin 0.0)
         (roottol 0.0) (skl 0.0) (sfmin 0.0) (small 0.0) (sn 0.0)
         (temp1 0.0) (theta 0.0) (thsign 0.0) (tol 0.0) (aapq #C(0.0 0.0))
         (ompq #C(0.0 0.0)) (t$ 0.0))
    (declare
     (type f2cl-lib:logical applv goscale lower lsvec noscale rotok rsvec
      uctol upper)
     (type (f2cl-lib:integer4) blskip emptsw i ibr ierr igl ijblsk ir1
      iswrot jbc jgl kbl lkahead mvl n2 n34 n4 nbl notrot p pskipped q
      rowskip swband)
     (type (single-float) t$ aapp aapp0 aapq1 aaqq apoaq aqoap big bigtheta
      cs ctol epsln large mxaapq mxsinj rootbig rooteps rootsfmin roottol
      skl sfmin small sn temp1 theta thsign tol)
     (type (f2cl-lib:complex8) aapq ompq))
    (setf lsvec
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobu "U")
              (declare (ignore var-1))
              (when var-0 (setf jobu var-0))
              ret-val))
    (setf uctol
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobu "C")
              (declare (ignore var-1))
              (when var-0 (setf jobu var-0))
              ret-val))
    (setf rsvec
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobv "V")
              (declare (ignore var-1))
              (when var-0 (setf jobv var-0))
              ret-val))
    (setf applv
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobv "A")
              (declare (ignore var-1))
              (when var-0 (setf jobv var-0))
              ret-val))
    (setf upper
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame joba "U")
```

```
          (declare (ignore var-1))
          (when var-0 (setf joba var-0))
          ret-val))
(setf lower
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame joba "L")
          (declare (ignore var-1))
          (when var-0 (setf joba var-0))
          ret-val))
(cond
 ((not
   (or upper lower
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame joba "G")
         (declare (ignore var-1))
         (when var-0 (setf joba var-0))
         ret-val)))
  (setf info -1))
 ((not
   (or lsvec uctol
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobu "N")
         (declare (ignore var-1))
         (when var-0 (setf jobu var-0))
         ret-val)))
  (setf info -2))
 ((not
   (or rsvec applv
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobv "N")
         (declare (ignore var-1))
         (when var-0 (setf jobv var-0))
         ret-val)))
  (setf info -3))
 ((< m 0) (setf info -4)) ((or (< n 0) (> n m)) (setf info -5))
 ((< lda m) (setf info -7)) ((< mv 0) (setf info -9))
 ((or (and rsvec (< ldv n)) (and applv (< ldv mv))) (setf info -11))
 ((and uctol (<= (f2cl-lib:fref rwork (1) ((1 lrwork))) one))
  (setf info -12))
 ((< lwork (f2cl-lib:int-add m n)) (setf info -13))
 ((< lrwork (f2cl-lib:max0 n 6)) (setf info -15)) (t (setf info 0)))
(cond
 ((/= info 0) (xerbla "CGESVJ" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= m 0) (= n 0))
    (go end_label))
(cond
 (uctol
  (setf ctol
          (f2cl-lib:fref rwork-%data% (1) ((1 lrwork))
```

```
                          rwork-%offset%)))
 (t
  (cond
   ((or lsvec rsvec applv)
    (setf ctol (f2cl-lib:fsqrt (f2cl-lib:ffloat m))))
   (t (setf ctol (f2cl-lib:ffloat m))))))
(setf epsln (slamch "Epsilon"))
(setf rooteps (f2cl-lib:fsqrt epsln))
(setf sfmin (slamch "SafeMinimum"))
(setf rootsfmin (f2cl-lib:fsqrt sfmin))
(setf small (/ sfmin epsln))
(setf big (slamch "Overflow"))
(setf rootbig (/ one rootsfmin))
(setf large
        (/ big
           (f2cl-lib:fsqrt (f2cl-lib:ffloat (f2cl-lib:int-mul m n)))))
(setf bigtheta (/ one rooteps))
(setf tol (* ctol epsln))
(setf roottol (f2cl-lib:fsqrt tol))
(cond
 ((>= (* (f2cl-lib:ffloat m) epsln) one) (setf info -4)
  (xerbla "CGESVJ" (f2cl-lib:int-sub info)) (go end_label)))
(cond
 (rsvec (setf mvl n)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claset "A" mvl n czero cone v ldv)
    (declare (ignore var-0 var-5))
    (when var-1 (setf mvl var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf czero var-3))
    (when var-4 (setf cone var-4))
    (when var-6 (setf ldv var-6))))
 (applv (setf mvl mv)))
(setf rsvec (or rsvec applv))
(setf skl
        (/ one
           (f2cl-lib:fsqrt
            (* (f2cl-lib:ffloat m) (f2cl-lib:ffloat n)))))
(setf noscale f2cl-lib:%true%)
(setf goscale f2cl-lib:%true%)
(cond
 (lower
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (setf aapp zero)
      (setf aaqq one)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (classq (f2cl-lib:int-add (f2cl-lib:int-sub m p) 1)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (p p)
```

```
                                ((1 lda) (1 *)) a-%offset%)
            1 aapp aaqq)
          (declare (ignore var-0 var-1 var-2))
          (when var-3 (setf aapp var-3))
          (when var-4 (setf aaqq var-4)))
        (cond
         ((> aapp big) (setf info -6)
          (xerbla "CGESVJ" (f2cl-lib:int-sub info)) (go end_label)))
        (setf aaqq (f2cl-lib:fsqrt aaqq))
        (cond
         ((and (< aapp (f2cl-lib:f2cl/ big aaqq)) noscale)
          (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                  (* aapp aaqq)))
         (t (setf noscale f2cl-lib:%false%)
          (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                  (* aapp (* aaqq skl)))
          (cond
           (goscale (setf goscale f2cl-lib:%false%)
            (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                          ((> q
                              (f2cl-lib:int-add p (f2cl-lib:int-sub 1)))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%)
                        (*
                         (f2cl-lib:fref sva-%data% (q) ((1 n))
                                        sva-%offset%)
                         skl))
               label1873))))))
      label1874)))
  (upper
   (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                 ((> p n) nil)
     (tagbody
       (setf aapp zero)
       (setf aaqq one)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (classq p
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 p)
                                  ((1 lda) (1 *)) a-%offset%)
            1 aapp aaqq)
          (declare (ignore var-1 var-2))
          (when var-0 (setf p var-0))
          (when var-3 (setf aapp var-3))
          (when var-4 (setf aaqq var-4)))
        (cond
         ((> aapp big) (setf info -6)
          (xerbla "CGESVJ" (f2cl-lib:int-sub info)) (go end_label)))
        (setf aaqq (f2cl-lib:fsqrt aaqq))
        (cond
```

```
          ((and (< aapp (f2cl-lib:f2cl/ big aaqq)) noscale)
           (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                   (* aapp aaqq)))
          (t (setf noscale f2cl-lib:%false%)
           (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                   (* aapp (* aaqq skl)))
           (cond
            (goscale (setf goscale f2cl-lib:%false%)
             (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                           ((> q
                               (f2cl-lib:int-add p (f2cl-lib:int-sub 1)))
                            nil)
               (tagbody
                 (setf (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%)
                         (*
                          (f2cl-lib:fref sva-%data% (q) ((1 n))
                                         sva-%offset%)
                          skl))
                label2873))))))
      label2874)))
(t
 (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
               ((> p n) nil)
   (tagbody
     (setf aapp zero)
     (setf aaqq one)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (classq m
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 p)
                                ((1 lda) (1 *)) a-%offset%)
          1 aapp aaqq)
       (declare (ignore var-1 var-2))
       (when var-0 (setf m var-0))
       (when var-3 (setf aapp var-3))
       (when var-4 (setf aaqq var-4)))
     (cond
      ((> aapp big) (setf info -6)
       (xerbla "CGESVJ" (f2cl-lib:int-sub info)) (go end_label)))
     (setf aaqq (f2cl-lib:fsqrt aaqq))
     (cond
      ((and (< aapp (f2cl-lib:f2cl/ big aaqq)) noscale)
       (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
               (* aapp aaqq)))
      (t (setf noscale f2cl-lib:%false%)
       (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
               (* aapp (* aaqq skl)))
       (cond
        (goscale (setf goscale f2cl-lib:%false%)
         (f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
                       ((> q
```

```
                              (f2cl-lib:int-add p (f2cl-lib:int-sub 1)))
                          nil)
            (tagbody
              (setf (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%)
                      (*
                       (f2cl-lib:fref sva-%data% (q) ((1 n))
                                      sva-%offset%)
                       skl))
             label3873))))))
     label3874))))
(if noscale
    (setf skl one))
(setf aapp zero)
(setf aaqq big)
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p n) nil)
  (tagbody
    (if (/= (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%) zero)
        (setf aaqq
                (f2cl-lib:amin1 aaqq
                                (f2cl-lib:fref sva-%data% (p) ((1 n))
                                               sva-%offset%))))
    (setf aapp
            (f2cl-lib:amax1 aapp
                            (f2cl-lib:fref sva-%data% (p) ((1 n))
                                           sva-%offset%)))
   label4781))
(cond
 ((= aapp zero)
  (if lsvec
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (claset "G" m n czero cone a lda)
        (declare (ignore var-0 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf czero var-3))
        (when var-4 (setf cone var-4))
        (when var-6 (setf lda var-6))))
  (setf (f2cl-lib:fref rwork-%data% (1) ((1 lrwork)) rwork-%offset%)
          one)
  (setf (f2cl-lib:fref rwork-%data% (2) ((1 lrwork)) rwork-%offset%)
          zero)
  (setf (f2cl-lib:fref rwork-%data% (3) ((1 lrwork)) rwork-%offset%)
          zero)
  (setf (f2cl-lib:fref rwork-%data% (4) ((1 lrwork)) rwork-%offset%)
          zero)
  (setf (f2cl-lib:fref rwork-%data% (5) ((1 lrwork)) rwork-%offset%)
          zero)
  (setf (f2cl-lib:fref rwork-%data% (6) ((1 lrwork)) rwork-%offset%)
          zero)
```

```
  (go end_label)))
(cond
 ((= n 1)
  (if lsvec
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (clascl "G" 0 0
           (f2cl-lib:array-slice sva-%data% single-float (1) ((1 n))
                                 sva-%offset%)
           skl m 1
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda ierr)
        (declare (ignore var-0 var-1 var-2 var-3 var-6 var-7))
        (when var-4 (setf skl var-4))
        (when var-5 (setf m var-5))
        (when var-8 (setf lda var-8))
        (when var-9 (setf ierr var-9))))
  (setf (f2cl-lib:fref rwork-%data% (1) ((1 lrwork)) rwork-%offset%)
          (/ one skl))
  (cond
   ((>= (f2cl-lib:fref sva (1) ((1 n))) sfmin)
    (setf (f2cl-lib:fref rwork-%data% (2) ((1 lrwork)) rwork-%offset%)
            one))
   (t
    (setf (f2cl-lib:fref rwork-%data% (2) ((1 lrwork)) rwork-%offset%)
            zero)))
  (setf (f2cl-lib:fref rwork-%data% (3) ((1 lrwork)) rwork-%offset%)
          zero)
  (setf (f2cl-lib:fref rwork-%data% (4) ((1 lrwork)) rwork-%offset%)
          zero)
  (setf (f2cl-lib:fref rwork-%data% (5) ((1 lrwork)) rwork-%offset%)
          zero)
  (setf (f2cl-lib:fref rwork-%data% (6) ((1 lrwork)) rwork-%offset%)
          zero)
  (go end_label)))
(setf sn (f2cl-lib:fsqrt (/ sfmin epsln)))
(setf temp1 (f2cl-lib:fsqrt (/ big (f2cl-lib:ffloat n))))
(cond
 ((or (<= aapp sn) (>= aaqq temp1) (and (<= sn aaqq) (<= aapp temp1)))
  (setf temp1 (f2cl-lib:amin1 big (/ temp1 aapp))))
 ((and (<= aaqq sn) (<= aapp temp1))
  (setf temp1
          (f2cl-lib:amin1 (/ sn aaqq)
                          (/ big
                             (* aapp
                                (f2cl-lib:fsqrt
                                 (f2cl-lib:ffloat n)))))))
 ((and (>= aaqq sn) (>= aapp temp1))
  (setf temp1 (f2cl-lib:amax1 (/ sn aaqq) (/ temp1 aapp))))
```

```
 ((and (<= aaqq sn) (>= aapp temp1))
  (setf temp1
          (f2cl-lib:amin1 (/ sn aaqq)
                          (/ big
                             (* (f2cl-lib:fsqrt (f2cl-lib:ffloat n))
                                aapp)))))
 (t (setf temp1 one)))
(cond
 ((/= temp1 one)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (slascl "G" 0 0 one temp1 n 1 sva n ierr)
    (declare (ignore var-0 var-1 var-2 var-6 var-7))
    (when var-3 (setf one var-3))
    (when var-4 (setf temp1 var-4))
    (when var-5 (setf n var-5))
    (when var-8 (setf n var-8))
    (when var-9 (setf ierr var-9)))))
(setf skl (* temp1 skl))
(cond
 ((/= skl one)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (clascl joba 0 0 one skl m n a lda ierr)
    (declare (ignore var-1 var-2 var-7))
    (when var-0 (setf joba var-0))
    (when var-3 (setf one var-3))
    (when var-4 (setf skl var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf ierr var-9)))
  (setf skl (/ one skl))))
(setf emptsw (the f2cl-lib:integer4 (truncate (* n (- n 1)) 2)))
(setf notrot 0)
(f2cl-lib:fdo (q 1 (f2cl-lib:int-add q 1))
              ((> q n) nil)
  (tagbody
    (setf (f2cl-lib:fref cwork-%data% (q) ((1 lwork)) cwork-%offset%)
            cone)
   label1868))
(setf swband 3)
(setf kbl (f2cl-lib:min0 8 n))
(setf nbl (the f2cl-lib:integer4 (truncate n kbl)))
(if (/= (f2cl-lib:int-mul nbl kbl) n)
    (setf nbl (f2cl-lib:int-add nbl 1)))
(setf blskip (expt kbl 2))
(setf rowskip (f2cl-lib:min0 5 kbl))
(setf lkahead 1)
(cond
```

```
((and (or lower upper)
      (> n (f2cl-lib:max0 64 (f2cl-lib:int-mul 4 kbl))))
 (setf n4 (the f2cl-lib:integer4 (truncate n 4)))
 (setf n2 (the f2cl-lib:integer4 (truncate n 2)))
 (setf n34 (f2cl-lib:int-mul 3 n4))
 (cond (applv (setf q 0)) (t (setf q 1)))
 (cond
  (lower
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13 var-14 var-15 var-16)
       (cgsvj0 jobv (f2cl-lib:int-sub m n34) (f2cl-lib:int-sub n n34)
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                              ((+ n34 1) (f2cl-lib:int-add n34 1))
                              ((1 lda) (1 *)) a-%offset%)
        lda
        (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                              ((+ n34 1)) ((1 lwork)) cwork-%offset%)
        (f2cl-lib:array-slice sva-%data% single-float ((+ n34 1))
                              ((1 n)) sva-%offset%)
        mvl
        (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                              ((+ (f2cl-lib:int-mul n34 q) 1)
                               (f2cl-lib:int-add n34 1))
                              ((1 ldv) (1 *)) v-%offset%)
        ldv epsln sfmin tol 2
        (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                              ((1 lwork)) cwork-%offset%)
        (f2cl-lib:int-sub lwork n) ierr)
     (declare
      (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-13 var-14
       var-15))
     (when var-0 (setf jobv var-0))
     (when var-4 (setf lda var-4))
     (when var-7 (setf mvl var-7))
     (when var-9 (setf ldv var-9))
     (when var-10 (setf epsln var-10))
     (when var-11 (setf sfmin var-11))
     (when var-12 (setf tol var-12))
     (when var-16 (setf ierr var-16)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13 var-14 var-15 var-16)
       (cgsvj0 jobv (f2cl-lib:int-sub m n2) (f2cl-lib:int-sub n34 n2)
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                              ((+ n2 1) (f2cl-lib:int-add n2 1))
                              ((1 lda) (1 *)) a-%offset%)
        lda
        (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                              ((+ n2 1)) ((1 lwork)) cwork-%offset%)
```

```
      (f2cl-lib:array-slice sva-%data% single-float ((+ n2 1))
                            ((1 n)) sva-%offset%)
      mvl
      (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                            ((+ (f2cl-lib:int-mul n2 q) 1)
                             (f2cl-lib:int-add n2 1))
                            ((1 ldv) (1 *)) v-%offset%)
      ldv epsln sfmin tol 2
      (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                            ((1 lwork)) cwork-%offset%)
      (f2cl-lib:int-sub lwork n) ierr)
    (declare
     (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-13 var-14
      var-15))
    (when var-0 (setf jobv var-0))
    (when var-4 (setf lda var-4))
    (when var-7 (setf mvl var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf epsln var-10))
    (when var-11 (setf sfmin var-11))
    (when var-12 (setf tol var-12))
    (when var-16 (setf ierr var-16)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17)
      (cgsvj1 jobv (f2cl-lib:int-sub m n2) (f2cl-lib:int-sub n n2) n4
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                            ((+ n2 1) (f2cl-lib:int-add n2 1))
                            ((1 lda) (1 *)) a-%offset%)
      lda
      (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                            ((+ n2 1)) ((1 lwork)) cwork-%offset%)
      (f2cl-lib:array-slice sva-%data% single-float ((+ n2 1))
                            ((1 n)) sva-%offset%)
      mvl
      (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                            ((+ (f2cl-lib:int-mul n2 q) 1)
                             (f2cl-lib:int-add n2 1))
                            ((1 ldv) (1 *)) v-%offset%)
      ldv epsln sfmin tol 1
      (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                            ((1 lwork)) cwork-%offset%)
      (f2cl-lib:int-sub lwork n) ierr)
    (declare
     (ignore var-1 var-2 var-4 var-6 var-7 var-9 var-14 var-15
      var-16))
    (when var-0 (setf jobv var-0))
    (when var-3 (setf n4 var-3))
    (when var-5 (setf lda var-5))
    (when var-8 (setf mvl var-8))
```

```
  (when var-10 (setf ldv var-10))
  (when var-11 (setf epsln var-11))
  (when var-12 (setf sfmin var-12))
  (when var-13 (setf tol var-13))
  (when var-17 (setf ierr var-17)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (cgsvj0 jobv (f2cl-lib:int-sub m n4) (f2cl-lib:int-sub n2 n4)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                           ((+ n4 1) (f2cl-lib:int-add n4 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                           ((+ n4 1)) ((1 lwork)) cwork-%offset%)
     (f2cl-lib:array-slice sva-%data% single-float ((+ n4 1))
                           ((1 n)) sva-%offset%)
     mvl
     (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                           ((+ (f2cl-lib:int-mul n4 q) 1)
                            (f2cl-lib:int-add n4 1))
                           ((1 ldv) (1 *)) v-%offset%)
     ldv epsln sfmin tol 1
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare
   (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-13 var-14
    var-15))
  (when var-0 (setf jobv var-0))
  (when var-4 (setf lda var-4))
  (when var-7 (setf mvl var-7))
  (when var-9 (setf ldv var-9))
  (when var-10 (setf epsln var-10))
  (when var-11 (setf sfmin var-11))
  (when var-12 (setf tol var-12))
  (when var-16 (setf ierr var-16)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (cgsvj0 jobv m n4 a lda cwork sva mvl v ldv epsln sfmin tol 1
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                           ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-3 var-5 var-6 var-8 var-13 var-14 var-15))
  (when var-0 (setf jobv var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n4 var-2))
  (when var-4 (setf lda var-4))
  (when var-7 (setf mvl var-7))
```

```
    (when var-9 (setf ldv var-9))
    (when var-10 (setf epsln var-10))
    (when var-11 (setf sfmin var-11))
    (when var-12 (setf tol var-12))
    (when var-16 (setf ierr var-16)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17)
      (cgsvj1 jobv m n2 n4 a lda cwork sva mvl v ldv epsln sfmin tol
       1
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                             ((1 lwork)) cwork-%offset%)
       (f2cl-lib:int-sub lwork n) ierr)
    (declare (ignore var-4 var-6 var-7 var-9 var-14 var-15 var-16))
    (when var-0 (setf jobv var-0))
    (when var-1 (setf m var-1))
    (when var-2 (setf n2 var-2))
    (when var-3 (setf n4 var-3))
    (when var-5 (setf lda var-5))
    (when var-8 (setf mvl var-8))
    (when var-10 (setf ldv var-10))
    (when var-11 (setf epsln var-11))
    (when var-12 (setf sfmin var-12))
    (when var-13 (setf tol var-13))
    (when var-17 (setf ierr var-17))))
 (upper
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16)
      (cgsvj0 jobv n4 n4 a lda cwork sva mvl v ldv epsln sfmin tol 2
       (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                             ((1 lwork)) cwork-%offset%)
       (f2cl-lib:int-sub lwork n) ierr)
    (declare (ignore var-3 var-5 var-6 var-8 var-13 var-14 var-15))
    (when var-0 (setf jobv var-0))
    (when var-1 (setf n4 var-1))
    (when var-2 (setf n4 var-2))
    (when var-4 (setf lda var-4))
    (when var-7 (setf mvl var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf epsln var-10))
    (when var-11 (setf sfmin var-11))
    (when var-12 (setf tol var-12))
    (when var-16 (setf ierr var-16)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16)
      (cgsvj0 jobv n2 n4
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             (1 (f2cl-lib:int-add n4 1))
```

```
                          ((1 lda) (1 *)) a-%offset%)
      lda
      (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                          ((+ n4 1)) ((1 lwork)) cwork-%offset%)
      (f2cl-lib:array-slice sva-%data% single-float ((+ n4 1))
                          ((1 n)) sva-%offset%)
      mvl
      (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                          ((+ (f2cl-lib:int-mul n4 q) 1)
                           (f2cl-lib:int-add n4 1))
                          ((1 ldv) (1 *)) v-%offset%)
      ldv epsln sfmin tol 1
      (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                          ((1 lwork)) cwork-%offset%)
      (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-3 var-5 var-6 var-8 var-13 var-14 var-15))
  (when var-0 (setf jobv var-0))
  (when var-1 (setf n2 var-1))
  (when var-2 (setf n4 var-2))
  (when var-4 (setf lda var-4))
  (when var-7 (setf mvl var-7))
  (when var-9 (setf ldv var-9))
  (when var-10 (setf epsln var-10))
  (when var-11 (setf sfmin var-11))
  (when var-12 (setf tol var-12))
  (when var-16 (setf ierr var-16)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17)
    (cgsvj1 jobv n2 n2 n4 a lda cwork sva mvl v ldv epsln sfmin tol
     1
     (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                          ((1 lwork)) cwork-%offset%)
     (f2cl-lib:int-sub lwork n) ierr)
  (declare (ignore var-4 var-6 var-7 var-9 var-14 var-15 var-16))
  (when var-0 (setf jobv var-0))
  (when var-1 (setf n2 var-1))
  (when var-2 (setf n2 var-2))
  (when var-3 (setf n4 var-3))
  (when var-5 (setf lda var-5))
  (when var-8 (setf mvl var-8))
  (when var-10 (setf ldv var-10))
  (when var-11 (setf epsln var-11))
  (when var-12 (setf sfmin var-12))
  (when var-13 (setf tol var-13))
  (when var-17 (setf ierr var-17)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (cgsvj0 jobv (f2cl-lib:int-add n2 n4) n4
```

```
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                  (1 (f2cl-lib:int-add n2 1))
                                  ((1 lda) (1 *)) a-%offset%)
            lda
            (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8
                                  ((+ n2 1)) ((1 lwork)) cwork-%offset%)
            (f2cl-lib:array-slice sva-%data% single-float ((+ n2 1))
                                  ((1 n)) sva-%offset%)
            mvl
            (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                                  ((+ (f2cl-lib:int-mul n2 q) 1)
                                   (f2cl-lib:int-add n2 1))
                                  ((1 ldv) (1 *)) v-%offset%)
            ldv epsln sfmin tol 1
            (f2cl-lib:array-slice cwork-%data% f2cl-lib:complex8 ((+ n 1))
                                  ((1 lwork)) cwork-%offset%)
            (f2cl-lib:int-sub lwork n) ierr)
          (declare
           (ignore var-1 var-3 var-5 var-6 var-8 var-13 var-14 var-15))
          (when var-0 (setf jobv var-0))
          (when var-2 (setf n4 var-2))
          (when var-4 (setf lda var-4))
          (when var-7 (setf mvl var-7))
          (when var-9 (setf ldv var-9))
          (when var-10 (setf epsln var-10))
          (when var-11 (setf sfmin var-11))
          (when var-12 (setf tol var-12))
          (when var-16 (setf ierr var-16)))))))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i nsweep) nil)
    (tagbody
      (setf mxaapq zero)
      (setf mxsinj zero)
      (setf iswrot 0)
      (setf notrot 0)
      (setf pskipped 0)
      (f2cl-lib:fdo (ibr 1 (f2cl-lib:int-add ibr 1))
                    ((> ibr nbl) nil)
        (tagbody
          (setf igl
                  (f2cl-lib:int-add
                   (f2cl-lib:int-mul (f2cl-lib:int-sub ibr 1) kbl) 1))
          (f2cl-lib:fdo (ir1 0 (f2cl-lib:int-add ir1 1))
                        ((> ir1
                            (f2cl-lib:min0 lkahead
                                           (f2cl-lib:int-add nbl
                                                             (f2cl-lib:int-sub
                                                              ibr))))
                         nil)
           (tagbody
```

```
(setf igl
        (f2cl-lib:int-add igl (f2cl-lib:int-mul ir1 kbl)))
(f2cl-lib:fdo (p igl (f2cl-lib:int-add p 1))
              ((> p
                  (f2cl-lib:min0
                   (f2cl-lib:int-add igl kbl
                                     (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add n
                                     (f2cl-lib:int-sub
                                      1))))
               nil)
  (tagbody
    (setf q
            (f2cl-lib:int-sub
             (f2cl-lib:int-add
              (isamax
               (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
               (f2cl-lib:array-slice sva-%data%
                                     single-float (p)
                                     ((1 n)) sva-%offset%)
               1)
              p)
             1))
    (cond
     ((/= p q)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (cswap m
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 (1 p) ((1 lda) (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 (1 q) ((1 lda) (1 *))
                                 a-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf m var-0)))
      (if rsvec
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4)
              (cswap mvl
               (f2cl-lib:array-slice v-%data%
                                     f2cl-lib:complex8
                                     (1 p) ((1 ldv) (1 *))
                                     v-%offset%)
               1
               (f2cl-lib:array-slice v-%data%
                                     f2cl-lib:complex8
                                     (1 q) ((1 ldv) (1 *))
                                     v-%offset%)
```

```
            1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf mvl var-0))))
    (setf temp1
            (f2cl-lib:fref sva-%data% (p) ((1 n))
                           sva-%offset%))
    (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                         sva-%offset%)
            (f2cl-lib:fref sva-%data% (q) ((1 n))
                           sva-%offset%))
    (setf (f2cl-lib:fref sva-%data% (q) ((1 n))
                         sva-%offset%)
            temp1)
    (setf aapq
            (f2cl-lib:fref cwork-%data% (p) ((1 lwork))
                           cwork-%offset%))
    (setf (f2cl-lib:fref cwork-%data% (p) ((1 lwork))
                         cwork-%offset%)
            (f2cl-lib:fref cwork-%data% (q) ((1 lwork))
                           cwork-%offset%))
    (setf (f2cl-lib:fref cwork-%data% (q) ((1 lwork))
                         cwork-%offset%)
            aapq)))
  (cond
   ((= ir1 0)
    (cond
     ((and (< (f2cl-lib:fref sva (p) ((1 n))) rootbig)
           (> (f2cl-lib:fref sva (p) ((1 n))) rootsfmin))
      (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                           sva-%offset%)
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2)
                  (scnrm2 m
                   (f2cl-lib:array-slice a-%data%
                                         f2cl-lib:complex8
                                         (1 p)
                                         ((1 lda) (1 *))
                                         a-%offset%)
                   1)
                (declare (ignore var-1 var-2))
                (when var-0 (setf m var-0))
                ret-val)))
     (t (setf temp1 zero) (setf aapp one)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4)
          (classq m
           (f2cl-lib:array-slice a-%data%
                                 f2cl-lib:complex8 (1 p)
                                 ((1 lda) (1 *))
                                 a-%offset%)
```

```
            1 temp1 aapp)
          (declare (ignore var-1 var-2))
          (when var-0 (setf m var-0))
          (when var-3 (setf temp1 var-3))
          (when var-4 (setf aapp var-4)))
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                             sva-%offset%)
              (* temp1 (f2cl-lib:fsqrt aapp)))))
      (setf aapp
              (f2cl-lib:fref sva-%data% (p) ((1 n))
                             sva-%offset%)))
     (t
      (setf aapp
              (f2cl-lib:fref sva-%data% (p) ((1 n))
                             sva-%offset%))))
    (cond
     ((> aapp zero)
      (tagbody
        (setf pskipped 0)
        (f2cl-lib:fdo (q (f2cl-lib:int-add p 1)
                         (f2cl-lib:int-add q 1))
                       ((> q
                           (f2cl-lib:min0
                            (f2cl-lib:int-add igl kbl
                                              (f2cl-lib:int-sub
                                               1))
                            n))
                        nil)
          (tagbody
            (setf aaqq
                    (f2cl-lib:fref sva-%data% (q) ((1 n))
                                   sva-%offset%))
            (cond
             ((> aaqq zero) (setf aapp0 aapp)
              (cond
               ((>= aaqq one)
                (setf rotok (<= (* small aapp) aaqq))
                (cond
                 ((< aapp (f2cl-lib:f2cl/ big aaqq))
                  (setf aapq
                          (/
                           (/
                            (multiple-value-bind
                                (ret-val var-0 var-1 var-2
                                 var-3 var-4)
                                (cdotc m
                                 (f2cl-lib:array-slice
                                  a-%data%
                                  f2cl-lib:complex8 (1 p)
                                  ((1 lda) (1 *))
```

```
                 a-%offset%)
                1
                (f2cl-lib:array-slice
                 a-%data%
                 f2cl-lib:complex8 (1 q)
                 ((1 lda) (1 *))
                 a-%offset%)
                1)
             (declare
              (ignore var-1 var-2 var-3
               var-4))
             (when var-0 (setf m var-0))
             ret-val)
           aaqq)
          aapp)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (ccopy m
      (f2cl-lib:array-slice a-%data%
                            f2cl-lib:complex8
                            (1 p)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex8
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5
      var-6 var-7 var-8 var-9)
     (clascl "G" 0 0 aapp one m 1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex8
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
      lda ierr)
   (declare
    (ignore var-0 var-1 var-2 var-6
     var-7))
   (when var-3 (setf aapp var-3))
   (when var-4 (setf one var-4))
```

```
                (when var-5 (setf m var-5))
                (when var-8 (setf lda var-8))
                (when var-9 (setf ierr var-9)))
              (setf aapq
                      (/
                       (multiple-value-bind
                           (ret-val var-0 var-1 var-2
                            var-3 var-4)
                           (cdotc m
                            (f2cl-lib:array-slice
                             cwork-%data%
                             f2cl-lib:complex8
                             ((+ n 1)) ((1 lwork))
                             cwork-%offset%)
                            1
                            (f2cl-lib:array-slice
                             a-%data% f2cl-lib:complex8
                             (1 q) ((1 lda) (1 *))
                             a-%offset%)
                            1)
                         (declare
                          (ignore var-1 var-2 var-3
                           var-4))
                         (when var-0 (setf m var-0))
                         ret-val)
                       aaqq)))))
          (t (setf rotok (<= aapp (/ aaqq small)))
           (cond
            ((> aapp (f2cl-lib:f2cl/ small aaqq))
             (setf aapq
                     (/
                      (/
                       (multiple-value-bind
                           (ret-val var-0 var-1 var-2
                            var-3 var-4)
                           (cdotc m
                            (f2cl-lib:array-slice
                             a-%data%
                             f2cl-lib:complex8 (1 p)
                             ((1 lda) (1 *))
                             a-%offset%)
                            1
                            (f2cl-lib:array-slice
                             a-%data%
                             f2cl-lib:complex8 (1 q)
                             ((1 lda) (1 *))
                             a-%offset%)
                            1)
                         (declare
                          (ignore var-1 var-2 var-3
```

```
                var-4))
              (when var-0 (setf m var-0))
              ret-val)
            aaqq)
           aapp)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (ccopy m
      (f2cl-lib:array-slice a-%data%
                            f2cl-lib:complex8
                            (1 q)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex8
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5
      var-6 var-7 var-8 var-9)
     (clascl "G" 0 0 aaqq one m 1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex8
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
      lda ierr)
   (declare
    (ignore var-0 var-1 var-2 var-6
     var-7))
   (when var-3 (setf aaqq var-3))
   (when var-4 (setf one var-4))
   (when var-5 (setf m var-5))
   (when var-8 (setf lda var-8))
   (when var-9 (setf ierr var-9)))
 (setf aapq
         (/
          (multiple-value-bind
              (ret-val var-0 var-1 var-2
               var-3 var-4)
              (cdotc m
               (f2cl-lib:array-slice
```

```
                    a-%data% f2cl-lib:complex8
                    (1 p) ((1 lda) (1 *))
                    a-%offset%)
                   1
                   (f2cl-lib:array-slice
                    cwork-%data%
                    f2cl-lib:complex8
                    ((+ n 1)) ((1 lwork))
                    cwork-%offset%)
                   1)
                (declare
                 (ignore var-1 var-2 var-3
                  var-4))
                (when var-0 (setf m var-0))
                ret-val)
              aapp))))))
(setf ompq (/ aapq (abs aapq)))
(setf aapq1 (- (abs aapq)))
(setf mxaapq
        (f2cl-lib:amax1 mxaapq (- aapq1)))
(cond
 ((> (abs aapq1) tol)
  (cond
   ((= ir1 0) (setf notrot 0)
    (setf pskipped 0)
    (setf iswrot
            (f2cl-lib:int-add iswrot 1))))
  (cond
   (rotok (setf aqoap (/ aaqq aapp))
    (setf apoaq (/ aapp aaqq))
    (setf theta
            (/
             (* (- half)
                (abs (- aqoap apoaq)))
             aapq1))
    (cond
     ((> (abs theta) bigtheta)
      (setf t$ (/ half theta)) (setf cs one)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4
           var-5 var-6)
          (crot m
           (f2cl-lib:array-slice a-%data%
                                 f2cl-lib:complex8
                                 (1 p)
                                 ((1 lda)
                                  (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice a-%data%
```

```
                              f2cl-lib:complex8
                              (1 q)
                              ((1 lda)
                               (1 *))
                              a-%offset%)
     1 cs (* (f2cl-lib:conjg ompq) t$))
  (declare
   (ignore var-1 var-2 var-3 var-4
    var-6))
  (when var-0 (setf m var-0))
  (when var-5 (setf cs var-5)))
(cond
 (rsvec
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4
       var-5 var-6)
      (crot mvl
       (f2cl-lib:array-slice v-%data%
                             f2cl-lib:complex8
                             (1 p)
                             ((1 ldv)
                              (1 *))
                             v-%offset%)
       1
       (f2cl-lib:array-slice v-%data%
                             f2cl-lib:complex8
                             (1 q)
                             ((1 ldv)
                              (1 *))
                             v-%offset%)
       1 cs
       (* (f2cl-lib:conjg ompq) t$))
    (declare
     (ignore var-1 var-2 var-3 var-4
      var-6))
    (when var-0 (setf mvl var-0))
    (when var-5 (setf cs var-5)))))
(setf (f2cl-lib:fref sva-%data% (q)
                     ((1 n))
                     sva-%offset%)
        (* aaqq
           (f2cl-lib:fsqrt
            (f2cl-lib:amax1 zero
                            (+ one
                               (* t$
                                  apoaq
                                  aapq1))))))
(setf aapp
        (* aapp
           (f2cl-lib:fsqrt
```

```
                (f2cl-lib:amax1 zero
                                (+ one
                                   (*
                                    (- t$)
                                    aqoap
                                    aapq1))))))
 (setf mxsinj
         (f2cl-lib:amax1 mxsinj
                         (abs t$))))
(t
 (setf thsign
         (- (f2cl-lib:sign one aapq1)))
 (setf t$
         (/ one
            (+ theta
               (* thsign
                  (f2cl-lib:fsqrt
                   (+ one
                      (* theta
                         theta)))))))
 (setf cs
         (f2cl-lib:fsqrt
          (/ one (+ one (* t$ t$)))))
 (setf sn (* t$ cs))
 (setf mxsinj
         (f2cl-lib:amax1 mxsinj
                         (abs sn)))
 (setf (f2cl-lib:fref sva-%data% (q)
                      ((1 n))
                      sva-%offset%)
         (* aaqq
            (f2cl-lib:fsqrt
             (f2cl-lib:amax1 zero
                             (+ one
                                (* t$
                                   apoaq
                                   aapq1))))))
 (setf aapp
         (* aapp
            (f2cl-lib:fsqrt
             (f2cl-lib:amax1 zero
                             (+ one
                                (*
                                 (- t$)
                                 aqoap
                                 aapq1))))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4
      var-5 var-6)
     (crot m
```

```
        (f2cl-lib:array-slice a-%data%
                              f2cl-lib:complex8
                              (1 p)
                              ((1 lda)
                               (1 *))
                              a-%offset%)
        1
        (f2cl-lib:array-slice a-%data%
                              f2cl-lib:complex8
                              (1 q)
                              ((1 lda)
                               (1 *))
                              a-%offset%)
        1 cs (* (f2cl-lib:conjg ompq) sn))
      (declare
       (ignore var-1 var-2 var-3 var-4
        var-6))
      (when var-0 (setf m var-0))
      (when var-5 (setf cs var-5)))
    (cond
     (rsvec
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4
           var-5 var-6)
          (crot mvl
           (f2cl-lib:array-slice v-%data%
                                 f2cl-lib:complex8
                                 (1 p)
                                 ((1 ldv)
                                  (1 *))
                                 v-%offset%)
           1
           (f2cl-lib:array-slice v-%data%
                                 f2cl-lib:complex8
                                 (1 q)
                                 ((1 ldv)
                                  (1 *))
                                 v-%offset%)
           1 cs
           (* (f2cl-lib:conjg ompq) sn))
        (declare
         (ignore var-1 var-2 var-3 var-4
          var-6))
        (when var-0 (setf mvl var-0))
        (when var-5 (setf cs var-5)))))))
  (setf (f2cl-lib:fref cwork-%data% (p)
                       ((1 lwork))
                       cwork-%offset%)
          (*
           (-
```

```
            (f2cl-lib:fref cwork-%data% (q)
                           ((1 lwork))
                           cwork-%offset%))
          ompq)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (ccopy m
      (f2cl-lib:array-slice a-%data%
                            f2cl-lib:complex8
                            (1 p)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex8
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5
      var-6 var-7 var-8 var-9)
     (clascl "G" 0 0 aapp one m 1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex8
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
      lda ierr)
   (declare
    (ignore var-0 var-1 var-2 var-6
     var-7))
   (when var-3 (setf aapp var-3))
   (when var-4 (setf one var-4))
   (when var-5 (setf m var-5))
   (when var-8 (setf lda var-8))
   (when var-9 (setf ierr var-9)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5
      var-6 var-7 var-8 var-9)
     (clascl "G" 0 0 aaqq one m 1
      (f2cl-lib:array-slice a-%data%
                            f2cl-lib:complex8
                            (1 q)
                            ((1 lda)
```

```
                              (1 *))
                             a-%offset%)
      lda ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6
    var-7))
  (when var-3 (setf aaqq var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5)
    (caxpy m (- aapq)
     (f2cl-lib:array-slice cwork-%data%
                           f2cl-lib:complex8
                           ((+ n 1))
                           ((1 lwork))
                           cwork-%offset%)
     1
     (f2cl-lib:array-slice a-%data%
                           f2cl-lib:complex8
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3 var-4
    var-5))
  (when var-0 (setf m var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5
     var-6 var-7 var-8 var-9)
    (clascl "G" 0 0 one aaqq m 1
     (f2cl-lib:array-slice a-%data%
                           f2cl-lib:complex8
                           (1 q)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     lda ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6
    var-7))
  (when var-3 (setf one var-3))
  (when var-4 (setf aaqq var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
```

```
  (setf (f2cl-lib:fref sva-%data% (q)
                       ((1 n))
                       sva-%offset%)
          (* aaqq
             (f2cl-lib:fsqrt
              (f2cl-lib:amax1 zero
                              (- one
                                 (* aapq1
                                    aapq1))))))
  (setf mxsinj
          (f2cl-lib:amax1 mxsinj sfmin))))
(cond
 ((<=
   (expt
    (f2cl-lib:f2cl/
     (f2cl-lib:fref sva (q) ((1 n))) aaqq)
    2)
   rooteps)
  (cond
   ((and (< aaqq rootbig)
         (> aaqq rootsfmin))
    (setf (f2cl-lib:fref sva-%data% (q)
                         ((1 n))
                         sva-%offset%)
            (multiple-value-bind
                (ret-val var-0 var-1 var-2)
                (scnrm2 m
                 (f2cl-lib:array-slice
                  a-%data%
                  f2cl-lib:complex8 (1 q)
                  ((1 lda) (1 *))
                  a-%offset%)
                 1)
               (declare
                (ignore var-1 var-2))
               (when var-0 (setf m var-0))
               ret-val)))
   (t (setf t$ zero) (setf aaqq one)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4)
        (classq m
         (f2cl-lib:array-slice a-%data%
                               f2cl-lib:complex8
                               (1 q)
                               ((1 lda)
                                (1 *))
                               a-%offset%)
         1 t$ aaqq)
      (declare (ignore var-1 var-2))
      (when var-0 (setf m var-0))
```

```
        (when var-3 (setf t$ var-3))
        (when var-4 (setf aaqq var-4)))
      (setf (f2cl-lib:fref sva-%data% (q)
                           ((1 n))
                           sva-%offset%)
            (* t$
               (f2cl-lib:fsqrt aaqq)))))))
 (cond
  ((<= (f2cl-lib:f2cl/ aapp aapp0) rooteps)
   (cond
    ((and (< aapp rootbig)
          (> aapp rootsfmin))
     (setf aapp
             (multiple-value-bind
                 (ret-val var-0 var-1 var-2)
                 (scnrm2 m
                  (f2cl-lib:array-slice
                   a-%data%
                   f2cl-lib:complex8 (1 p)
                   ((1 lda) (1 *))
                   a-%offset%)
                  1)
                (declare
                 (ignore var-1 var-2))
                (when var-0 (setf m var-0))
                ret-val)))
    (t (setf t$ zero) (setf aapp one)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4)
         (classq m
          (f2cl-lib:array-slice a-%data%
                                f2cl-lib:complex8
                                (1 p)
                                ((1 lda)
                                 (1 *))
                                a-%offset%)
          1 t$ aapp)
       (declare (ignore var-1 var-2))
       (when var-0 (setf m var-0))
       (when var-3 (setf t$ var-3))
       (when var-4 (setf aapp var-4)))
     (setf aapp
             (* t$ (f2cl-lib:fsqrt aapp)))))
   (setf (f2cl-lib:fref sva-%data% (p)
                        ((1 n))
                        sva-%offset%)
           aapp))))
(t
 (if (= ir1 0)
     (setf notrot
```

```
                                        (f2cl-lib:int-add notrot 1)))
                          (setf pskipped
                                  (f2cl-lib:int-add pskipped 1)))))
                        (t
                         (if (= ir1 0)
                             (setf notrot
                                     (f2cl-lib:int-add notrot 1)))
                         (setf pskipped
                                 (f2cl-lib:int-add pskipped 1))))
                       (cond
                        ((and (<= i swband) (> pskipped rowskip))
                         (if (= ir1 0)
                             (setf aapp (- aapp)))
                         (setf notrot 0) (go label2103)))
                      label2002))
                 label2103
                  (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                                       sva-%offset%)
                          aapp)))
                (t
                 (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                                      sva-%offset%)
                         aapp)
                 (if (and (= ir1 0) (= aapp zero))
                     (setf notrot
                             (f2cl-lib:int-sub
                              (f2cl-lib:int-add notrot
                                                (f2cl-lib:min0
                                                 (f2cl-lib:int-sub
                                                  (f2cl-lib:int-add
                                                   igl kbl)
                                                  1)
                                                 n))
                              p)))))
            label2001))
        label1002))
     (setf igl
             (f2cl-lib:int-add
              (f2cl-lib:int-mul (f2cl-lib:int-sub ibr 1) kbl) 1))
     (f2cl-lib:fdo (jbc (f2cl-lib:int-add ibr 1)
                    (f2cl-lib:int-add jbc 1))
                   ((> jbc nbl) nil)
       (tagbody
         (setf jgl
                 (f2cl-lib:int-add
                  (f2cl-lib:int-mul (f2cl-lib:int-sub jbc 1) kbl)
                  1))
         (setf ijblsk 0)
         (f2cl-lib:fdo (p igl (f2cl-lib:int-add p 1))
                       ((> p
```

```
                    (f2cl-lib:min0
                     (f2cl-lib:int-add igl kbl
                                       (f2cl-lib:int-sub 1))
                     n))
                 nil)
(tagbody
  (setf aapp
          (f2cl-lib:fref sva-%data% (p) ((1 n))
                         sva-%offset%))
  (cond
   ((> aapp zero)
    (tagbody
      (setf pskipped 0)
      (f2cl-lib:fdo (q jgl (f2cl-lib:int-add q 1))
                    ((> q
                        (f2cl-lib:min0
                         (f2cl-lib:int-add jgl kbl
                                           (f2cl-lib:int-sub
                                            1))
                         n))
                     nil)
        (tagbody
          (setf aaqq
                  (f2cl-lib:fref sva-%data% (q) ((1 n))
                                 sva-%offset%))
          (cond
           ((> aaqq zero) (setf aapp0 aapp)
            (cond
             ((>= aaqq one)
              (cond
               ((>= aapp aaqq)
                (setf rotok (<= (* small aapp) aaqq)))
               (t (setf rotok (<= (* small aaqq) aapp))))
              (cond
               ((< aapp (f2cl-lib:f2cl/ big aaqq))
                (setf aapq
                        (/
                         (/
                          (multiple-value-bind
                              (ret-val var-0 var-1 var-2
                               var-3 var-4)
                              (cdotc m
                               (f2cl-lib:array-slice
                                a-%data%
                                f2cl-lib:complex8 (1 p)
                                ((1 lda) (1 *))
                                a-%offset%)
                               1
                               (f2cl-lib:array-slice
                                a-%data%
```

```
                    f2cl-lib:complex8 (1 q)
                    ((1 lda) (1 *))
                    a-%offset%)
                   1)
                (declare
                 (ignore var-1 var-2 var-3
                  var-4))
                (when var-0 (setf m var-0))
                ret-val)
              aaqq)
             aapp)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (ccopy m
      (f2cl-lib:array-slice a-%data%
                            f2cl-lib:complex8
                            (1 p)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex8
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5
      var-6 var-7 var-8 var-9)
     (clascl "G" 0 0 aapp one m 1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex8
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
      lda ierr)
   (declare
    (ignore var-0 var-1 var-2 var-6
     var-7))
   (when var-3 (setf aapp var-3))
   (when var-4 (setf one var-4))
   (when var-5 (setf m var-5))
   (when var-8 (setf lda var-8))
   (when var-9 (setf ierr var-9)))
 (setf aapq
```

```
                    (/
                     (multiple-value-bind
                         (ret-val var-0 var-1 var-2
                          var-3 var-4)
                         (cdotc m
                          (f2cl-lib:array-slice
                           cwork-%data%
                           f2cl-lib:complex8
                           ((+ n 1)) ((1 lwork))
                           cwork-%offset%)
                          1
                          (f2cl-lib:array-slice
                           a-%data% f2cl-lib:complex8
                           (1 q) ((1 lda) (1 *))
                           a-%offset%)
                          1)
                       (declare
                        (ignore var-1 var-2 var-3
                         var-4))
                       (when var-0 (setf m var-0))
                       ret-val)
                     aaqq)))))
            (t
             (cond
              ((>= aapp aaqq)
               (setf rotok (<= aapp (/ aaqq small))))
              (t (setf rotok (<= aaqq (/ aapp small)))))
             (cond
              ((> aapp (f2cl-lib:f2cl/ small aaqq))
               (setf aapq
                       (/
                        (/
                         (multiple-value-bind
                             (ret-val var-0 var-1 var-2
                              var-3 var-4)
                             (cdotc m
                              (f2cl-lib:array-slice
                               a-%data%
                               f2cl-lib:complex8 (1 p)
                               ((1 lda) (1 *))
                               a-%offset%)
                              1
                              (f2cl-lib:array-slice
                               a-%data%
                               f2cl-lib:complex8 (1 q)
                               ((1 lda) (1 *))
                               a-%offset%)
                              1)
                           (declare
                            (ignore var-1 var-2 var-3
```

```
                var-4))
               (when var-0 (setf m var-0))
               ret-val)
             aaqq)
            aapp)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (ccopy m
      (f2cl-lib:array-slice a-%data%
                            f2cl-lib:complex8
                            (1 q)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex8
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5
      var-6 var-7 var-8 var-9)
     (clascl "G" 0 0 aaqq one m 1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex8
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
      lda ierr)
   (declare
    (ignore var-0 var-1 var-2 var-6
     var-7))
   (when var-3 (setf aaqq var-3))
   (when var-4 (setf one var-4))
   (when var-5 (setf m var-5))
   (when var-8 (setf lda var-8))
   (when var-9 (setf ierr var-9)))
 (setf aapq
         (/
          (multiple-value-bind
              (ret-val var-0 var-1 var-2
               var-3 var-4)
              (cdotc m
               (f2cl-lib:array-slice
```

```
                    a-%data% f2cl-lib:complex8
                    (1 p) ((1 lda) (1 *))
                    a-%offset%)
                   1
                   (f2cl-lib:array-slice
                    cwork-%data%
                    f2cl-lib:complex8
                    ((+ n 1)) ((1 lwork))
                    cwork-%offset%)
                   1)
                (declare
                 (ignore var-1 var-2 var-3
                  var-4))
                (when var-0 (setf m var-0))
                ret-val)
              aapp))))))
(setf ompq (/ aapq (abs aapq)))
(setf aapq1 (- (abs aapq)))
(setf mxaapq
        (f2cl-lib:amax1 mxaapq (- aapq1)))
(cond
 ((> (abs aapq1) tol) (setf notrot 0)
  (setf pskipped 0)
  (setf iswrot (f2cl-lib:int-add iswrot 1))
  (cond
   (rotok (setf aqoap (/ aaqq aapp))
    (setf apoaq (/ aapp aaqq))
    (setf theta
            (/
             (* (- half)
                (abs (- aqoap apoaq)))
             aapq1))
    (if (> aaqq aapp0)
        (setf theta (- theta)))
    (cond
     ((> (abs theta) bigtheta)
      (setf t$ (/ half theta)) (setf cs one)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4
           var-5 var-6)
          (crot m
           (f2cl-lib:array-slice a-%data%
                                 f2cl-lib:complex8
                                 (1 p)
                                 ((1 lda)
                                  (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice a-%data%
                                 f2cl-lib:complex8
```

```
                                (1 q)
                                ((1 lda)
                                 (1 *))
                                a-%offset%)
     1 cs (* (f2cl-lib:conjg ompq) t$))
  (declare
   (ignore var-1 var-2 var-3 var-4
    var-6))
  (when var-0 (setf m var-0))
  (when var-5 (setf cs var-5)))
(cond
 (rsvec
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4
       var-5 var-6)
      (crot mvl
       (f2cl-lib:array-slice v-%data%
                             f2cl-lib:complex8
                             (1 p)
                             ((1 ldv)
                              (1 *))
                             v-%offset%)
       1
       (f2cl-lib:array-slice v-%data%
                             f2cl-lib:complex8
                             (1 q)
                             ((1 ldv)
                              (1 *))
                             v-%offset%)
       1 cs
       (* (f2cl-lib:conjg ompq) t$))
    (declare
     (ignore var-1 var-2 var-3 var-4
      var-6))
    (when var-0 (setf mvl var-0))
    (when var-5 (setf cs var-5)))))
(setf (f2cl-lib:fref sva-%data% (q)
                     ((1 n))
                     sva-%offset%)
        (* aaqq
           (f2cl-lib:fsqrt
            (f2cl-lib:amax1 zero
                            (+ one
                               (* t$
                                  apoaq
                                  aapq1))))))
(setf aapp
        (* aapp
           (f2cl-lib:fsqrt
            (f2cl-lib:amax1 zero
```

```
                                (+ one
                                   (*
                                    (- t$)
                                    aqoap
                                    aapq1))))))
 (setf mxsinj
         (f2cl-lib:amax1 mxsinj
                         (abs t$))))
(t
 (setf thsign
         (- (f2cl-lib:sign one aapq1)))
 (if (> aaqq aapp0)
     (setf thsign (- thsign)))
 (setf t$
         (/ one
            (+ theta
               (* thsign
                  (f2cl-lib:fsqrt
                   (+ one
                      (* theta
                         theta)))))))
 (setf cs
         (f2cl-lib:fsqrt
          (/ one (+ one (* t$ t$)))))
 (setf sn (* t$ cs))
 (setf mxsinj
         (f2cl-lib:amax1 mxsinj
                         (abs sn)))
 (setf (f2cl-lib:fref sva-%data% (q)
                      ((1 n))
                      sva-%offset%)
         (* aaqq
            (f2cl-lib:fsqrt
             (f2cl-lib:amax1 zero
                             (+ one
                                (* t$
                                   apoaq
                                   aapq1))))))
 (setf aapp
         (* aapp
            (f2cl-lib:fsqrt
             (f2cl-lib:amax1 zero
                             (+ one
                                (*
                                 (- t$)
                                 aqoap
                                 aapq1))))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4
      var-5 var-6)
```

```
          (crot m
           (f2cl-lib:array-slice a-%data%
                                 f2cl-lib:complex8
                                 (1 p)
                                 ((1 lda)
                                  (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice a-%data%
                                 f2cl-lib:complex8
                                 (1 q)
                                 ((1 lda)
                                  (1 *))
                                 a-%offset%)
           1 cs (* (f2cl-lib:conjg ompq) sn))
        (declare
         (ignore var-1 var-2 var-3 var-4
          var-6))
        (when var-0 (setf m var-0))
        (when var-5 (setf cs var-5)))
      (cond
       (rsvec
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4
             var-5 var-6)
            (crot mvl
             (f2cl-lib:array-slice v-%data%
                                   f2cl-lib:complex8
                                   (1 p)
                                   ((1 ldv)
                                    (1 *))
                                   v-%offset%)
             1
             (f2cl-lib:array-slice v-%data%
                                   f2cl-lib:complex8
                                   (1 q)
                                   ((1 ldv)
                                    (1 *))
                                   v-%offset%)
             1 cs
             (* (f2cl-lib:conjg ompq) sn))
          (declare
           (ignore var-1 var-2 var-3 var-4
            var-6))
          (when var-0 (setf mvl var-0))
          (when var-5 (setf cs var-5)))))))
    (setf (f2cl-lib:fref cwork-%data% (p)
                         ((1 lwork))
                         cwork-%offset%)
          (*
```

```
          (-
           (f2cl-lib:fref cwork-%data% (q)
                          ((1 lwork))
                          cwork-%offset%))
          ompq)))
(t
 (cond
  ((> aapp aaqq)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4)
       (ccopy m
        (f2cl-lib:array-slice a-%data%
                              f2cl-lib:complex8
                              (1 p)
                              ((1 lda)
                               (1 *))
                              a-%offset%)
        1
        (f2cl-lib:array-slice cwork-%data%
                              f2cl-lib:complex8
                              ((+ n 1))
                              ((1 lwork))
                              cwork-%offset%)
        1)
     (declare
      (ignore var-1 var-2 var-3 var-4))
     (when var-0 (setf m var-0)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4
        var-5 var-6 var-7 var-8 var-9)
       (clascl "G" 0 0 aapp one m 1
        (f2cl-lib:array-slice cwork-%data%
                              f2cl-lib:complex8
                              ((+ n 1))
                              ((1 lwork))
                              cwork-%offset%)
        lda ierr)
     (declare
      (ignore var-0 var-1 var-2 var-6
       var-7))
     (when var-3 (setf aapp var-3))
     (when var-4 (setf one var-4))
     (when var-5 (setf m var-5))
     (when var-8 (setf lda var-8))
     (when var-9 (setf ierr var-9)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4
        var-5 var-6 var-7 var-8 var-9)
       (clascl "G" 0 0 aaqq one m 1
        (f2cl-lib:array-slice a-%data%
```

```
                                f2cl-lib:complex8
                                (1 q)
                                ((1 lda)
                                 (1 *))
                                a-%offset%)
      lda ierr)
   (declare
    (ignore var-0 var-1 var-2 var-6
     var-7))
   (when var-3 (setf aaqq var-3))
   (when var-4 (setf one var-4))
   (when var-5 (setf m var-5))
   (when var-8 (setf lda var-8))
   (when var-9 (setf ierr var-9)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4
      var-5)
     (caxpy m (- aapq)
      (f2cl-lib:array-slice cwork-%data%
                                f2cl-lib:complex8
                                ((+ n 1))
                                ((1 lwork))
                                cwork-%offset%)
      1
      (f2cl-lib:array-slice a-%data%
                                f2cl-lib:complex8
                                (1 q)
                                ((1 lda)
                                 (1 *))
                                a-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4
     var-5))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4
      var-5 var-6 var-7 var-8 var-9)
     (clascl "G" 0 0 one aaqq m 1
      (f2cl-lib:array-slice a-%data%
                                f2cl-lib:complex8
                                (1 q)
                                ((1 lda)
                                 (1 *))
                                a-%offset%)
      lda ierr)
   (declare
    (ignore var-0 var-1 var-2 var-6
     var-7))
   (when var-3 (setf one var-3))
```

```
  (when var-4 (setf aaqq var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
 (setf (f2cl-lib:fref sva-%data% (q)
                      ((1 n))
                      sva-%offset%)
        (* aaqq
           (f2cl-lib:fsqrt
            (f2cl-lib:amax1 zero
                            (- one
                               (* aapq1
                                  aapq1))))))
 (setf mxsinj
        (f2cl-lib:amax1 mxsinj sfmin)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4)
     (ccopy m
      (f2cl-lib:array-slice a-%data%
                            f2cl-lib:complex8
                            (1 q)
                            ((1 lda)
                             (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex8
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf m var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4
      var-5 var-6 var-7 var-8 var-9)
     (clascl "G" 0 0 aaqq one m 1
      (f2cl-lib:array-slice cwork-%data%
                            f2cl-lib:complex8
                            ((+ n 1))
                            ((1 lwork))
                            cwork-%offset%)
      lda ierr)
   (declare
    (ignore var-0 var-1 var-2 var-6
     var-7))
   (when var-3 (setf aaqq var-3))
   (when var-4 (setf one var-4))
```

```
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5 var-6 var-7 var-8 var-9)
    (clascl "G" 0 0 aapp one m 1
     (f2cl-lib:array-slice a-%data%
                           f2cl-lib:complex8
                           (1 p)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     lda ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6
    var-7))
  (when var-3 (setf aapp var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf m var-5))
  (when var-8 (setf lda var-8))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5)
    (caxpy m (- (f2cl-lib:conjg aapq))
     (f2cl-lib:array-slice cwork-%data%
                           f2cl-lib:complex8
                           ((+ n 1))
                           ((1 lwork))
                           cwork-%offset%)
     1
     (f2cl-lib:array-slice a-%data%
                           f2cl-lib:complex8
                           (1 p)
                           ((1 lda)
                            (1 *))
                           a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3 var-4
    var-5))
  (when var-0 (setf m var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4
     var-5 var-6 var-7 var-8 var-9)
    (clascl "G" 0 0 one aapp m 1
     (f2cl-lib:array-slice a-%data%
                           f2cl-lib:complex8
                           (1 p)
```

```
                                ((1 lda)
                                 (1 *))
                                a-%offset%)
          lda ierr)
        (declare
         (ignore var-0 var-1 var-2 var-6
          var-7))
        (when var-3 (setf one var-3))
        (when var-4 (setf aapp var-4))
        (when var-5 (setf m var-5))
        (when var-8 (setf lda var-8))
        (when var-9 (setf ierr var-9)))
      (setf (f2cl-lib:fref sva-%data% (p)
                           ((1 n))
                           sva-%offset%)
              (* aapp
                 (f2cl-lib:fsqrt
                  (f2cl-lib:amax1 zero
                                  (- one
                                     (* aapq1
                                        aapq1))))))
      (setf mxsinj
              (f2cl-lib:amax1 mxsinj
                              sfmin))))))
(cond
 ((<=
   (expt
    (f2cl-lib:f2cl/
     (f2cl-lib:fref sva (q) ((1 n))) aaqq)
    2)
   rooteps)
  (cond
   ((and (< aaqq rootbig)
         (> aaqq rootsfmin))
    (setf (f2cl-lib:fref sva-%data% (q)
                         ((1 n))
                         sva-%offset%)
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2)
                  (scnrm2 m
                   (f2cl-lib:array-slice
                    a-%data%
                    f2cl-lib:complex8 (1 q)
                    ((1 lda) (1 *))
                    a-%offset%)
                   1)
                (declare
                 (ignore var-1 var-2))
                (when var-0 (setf m var-0))
                ret-val)))
```

```
    (t (setf t$ zero) (setf aaqq one)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4)
         (classq m
          (f2cl-lib:array-slice a-%data%
                                f2cl-lib:complex8
                                (1 q)
                                ((1 lda)
                                 (1 *))
                                a-%offset%)
          1 t$ aaqq)
       (declare (ignore var-1 var-2))
       (when var-0 (setf m var-0))
       (when var-3 (setf t$ var-3))
       (when var-4 (setf aaqq var-4)))
     (setf (f2cl-lib:fref sva-%data% (q)
                          ((1 n))
                          sva-%offset%)
             (* t$
                (f2cl-lib:fsqrt aaqq)))))))
(cond
 ((<= (expt (f2cl-lib:f2cl/ aapp aapp0) 2)
      rooteps)
  (cond
   ((and (< aapp rootbig)
         (> aapp rootsfmin))
    (setf aapp
            (multiple-value-bind
                (ret-val var-0 var-1 var-2)
                (scnrm2 m
                 (f2cl-lib:array-slice
                  a-%data%
                  f2cl-lib:complex8 (1 p)
                  ((1 lda) (1 *))
                  a-%offset%)
                 1)
               (declare
                (ignore var-1 var-2))
               (when var-0 (setf m var-0))
               ret-val)))
    (t (setf t$ zero) (setf aapp one)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4)
         (classq m
          (f2cl-lib:array-slice a-%data%
                                f2cl-lib:complex8
                                (1 p)
                                ((1 lda)
                                 (1 *))
                                a-%offset%)
```

```
                        1 t$ aapp)
                     (declare (ignore var-1 var-2))
                     (when var-0 (setf m var-0))
                     (when var-3 (setf t$ var-3))
                     (when var-4 (setf aapp var-4)))
                   (setf aapp
                           (* t$ (f2cl-lib:fsqrt aapp)))))
                 (setf (f2cl-lib:fref sva-%data% (p)
                                      ((1 n))
                                      sva-%offset%)
                        aapp))))
            (t (setf notrot (f2cl-lib:int-add notrot 1))
             (setf pskipped
                     (f2cl-lib:int-add pskipped 1))
             (setf ijblsk
                     (f2cl-lib:int-add ijblsk 1)))))
          (t (setf notrot (f2cl-lib:int-add notrot 1))
           (setf pskipped (f2cl-lib:int-add pskipped 1))
           (setf ijblsk (f2cl-lib:int-add ijblsk 1))))
         (cond
          ((and (<= i swband) (>= ijblsk blskip))
           (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                                sva-%offset%)
                   aapp)
           (setf notrot 0) (go label2011)))
         (cond
          ((and (<= i swband) (> pskipped rowskip))
           (setf aapp (- aapp)) (setf notrot 0)
           (go label2203)))
        label2200))
  label2203
   (setf (f2cl-lib:fref sva-%data% (p) ((1 n))
                        sva-%offset%)
           aapp)))
(t
 (if (= aapp zero)
     (setf notrot
             (f2cl-lib:int-add
              (f2cl-lib:int-sub
               (f2cl-lib:int-add notrot
                                 (f2cl-lib:min0
                                  (f2cl-lib:int-sub
                                   (f2cl-lib:int-add
                                    jgl kbl)
                                   1)
                                  n))
               jgl)
              1)))
 (if (< aapp zero)
     (setf notrot 0))))
```

```
            label2100))
        label2010))
    label2011
     (f2cl-lib:fdo (p igl (f2cl-lib:int-add p 1))
                   ((> p
                       (f2cl-lib:min0
                        (f2cl-lib:int-add igl kbl
                                          (f2cl-lib:int-sub 1))
                        n))
                    nil)
       (tagbody
         (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                 (abs
                  (f2cl-lib:fref sva-%data% (p) ((1 n))
                                 sva-%offset%)))
        label2012))
    label2000))
(cond
 ((and (< (f2cl-lib:fref sva (n) ((1 n))) rootbig)
       (> (f2cl-lib:fref sva (n) ((1 n))) rootsfmin))
  (setf (f2cl-lib:fref sva-%data% (n) ((1 n)) sva-%offset%)
          (multiple-value-bind (ret-val var-0 var-1 var-2)
              (scnrm2 m
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                     (1 n) ((1 lda) (1 *))
                                     a-%offset%)
               1)
            (declare (ignore var-1 var-2))
            (when var-0 (setf m var-0))
            ret-val)))
 (t (setf t$ zero) (setf aapp one)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (classq m
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 n)
                             ((1 lda) (1 *)) a-%offset%)
       1 t$ aapp)
    (declare (ignore var-1 var-2))
    (when var-0 (setf m var-0))
    (when var-3 (setf t$ var-3))
    (when var-4 (setf aapp var-4)))
  (setf (f2cl-lib:fref sva-%data% (n) ((1 n)) sva-%offset%)
          (* t$ (f2cl-lib:fsqrt aapp)))))
(if (and (< i swband) (or (<= mxaapq roottol) (<= iswrot n)))
    (setf swband i))
(cond
 ((and (> i (f2cl-lib:int-add swband 1))
       (< mxaapq (* (f2cl-lib:fsqrt (f2cl-lib:ffloat n)) tol))
       (< (* (f2cl-lib:ffloat n) mxaapq mxsinj) tol))
  (go label1994)))
(if (>= notrot emptsw)
```

```
          (go label1994))
      label1993))
  (setf info (f2cl-lib:int-sub nsweep 1))
  (go label1995)
 label1994
  (setf info 0)
 label1995
  (setf n2 0)
  (setf n4 0)
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (setf q
              (f2cl-lib:int-sub
               (f2cl-lib:int-add
                (isamax (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
                 (f2cl-lib:array-slice sva-%data% single-float (p)
                                       ((1 n)) sva-%offset%)
                 1)
                p)
               1))
      (cond
       ((/= p q)
        (setf temp1 (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%))
        (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%))
        (setf (f2cl-lib:fref sva-%data% (q) ((1 n)) sva-%offset%) temp1)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (cswap m
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 p)
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 q)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf m var-0)))
        (if rsvec
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (cswap mvl
                 (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (1 p)
                                       ((1 ldv) (1 *)) v-%offset%)
                 1
                 (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (1 q)
                                       ((1 ldv) (1 *)) v-%offset%)
                 1)
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf mvl var-0))))))
      (cond
       ((/= (f2cl-lib:fref sva (p) ((1 n))) zero)
```

```
      (setf n4 (f2cl-lib:int-add n4 1))
      (if (>
           (* (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%) skl)
           sfmin)
          (setf n2 (f2cl-lib:int-add n2 1)))))
   label5991))
(cond
 ((/= (f2cl-lib:fref sva (n) ((1 n))) zero)
  (setf n4 (f2cl-lib:int-add n4 1))
  (if (> (* (f2cl-lib:fref sva-%data% (n) ((1 n)) sva-%offset%) skl)
         sfmin)
      (setf n2 (f2cl-lib:int-add n2 1)))))
(cond
 ((or lsvec uctol)
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n2) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (csscal m
           (/ one (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%))
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 p)
                                 ((1 lda) (1 *)) a-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf m var-0)))
     label1998))))
(cond
 (rsvec
  (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                ((> p n) nil)
    (tagbody
      (setf temp1
              (/ one
                 (multiple-value-bind (ret-val var-0 var-1 var-2)
                     (scnrm2 mvl
                      (f2cl-lib:array-slice v-%data% f2cl-lib:complex8
                                            (1 p) ((1 ldv) (1 *))
                                            v-%offset%)
                      1)
                   (declare (ignore var-1 var-2))
                   (when var-0 (setf mvl var-0))
                   ret-val)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (csscal mvl temp1
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (1 p)
                                 ((1 ldv) (1 *)) v-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf mvl var-0))
        (when var-1 (setf temp1 var-1)))
```

```
            label2399))))
        (cond
         ((or
           (and (> skl one)
                (< (f2cl-lib:fref sva (1) ((1 n))) (f2cl-lib:f2cl/ big skl)))
           (and (< skl one)
                (>
                 (f2cl-lib:fref sva
                                ((max (the f2cl-lib:integer4 n2)
                                      (the f2cl-lib:integer4 1)))
                                ((1 n)))
                 (f2cl-lib:f2cl/ sfmin skl))))
          (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                        ((> p n) nil)
            (tagbody
              (setf (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)
                      (* skl
                         (f2cl-lib:fref sva-%data% (p) ((1 n)) sva-%offset%)))
            label2400))
          (setf skl one)))
        (setf (f2cl-lib:fref rwork-%data% (1) ((1 lrwork)) rwork-%offset%) skl)
        (setf (f2cl-lib:fref rwork-%data% (2) ((1 lrwork)) rwork-%offset%)
                (f2cl-lib:ffloat n4))
        (setf (f2cl-lib:fref rwork-%data% (3) ((1 lrwork)) rwork-%offset%)
                (f2cl-lib:ffloat n2))
        (setf (f2cl-lib:fref rwork-%data% (4) ((1 lrwork)) rwork-%offset%)
                (f2cl-lib:ffloat i))
        (setf (f2cl-lib:fref rwork-%data% (5) ((1 lrwork)) rwork-%offset%)
                mxaapq)
        (setf (f2cl-lib:fref rwork-%data% (6) ((1 lrwork)) rwork-%offset%)
                mxsinj)
        (go end_label)
       end_label
        (return
         (values joba jobu jobv m n nil lda nil nil nil ldv nil nil nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgesvj
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
```

```
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::joba
  fortran-to-lisp::jobu
  fortran-to-lisp::jobv
  fortran-to-lisp::m
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil nil
  fortran-to-lisp::ldv nil nil nil
  nil fortran-to-lisp::info)
:calls 'nil)))
```

———-

34.29 cgetf2

34.29.1 Axiom unit tests

— **cgetf2.input** —

```
)set break resume
)sys rm -f cgetf2.output
)spool cgetf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.29.2 Axiom help page

— cgetf2.help —

```
======================================================================
cgetf2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgetf2 (M, N, A, LDA, IPIV, INFO)
           CGETF2 computes the LU factorization of a general m-by-n matrix
           using partial pivoting with row interchanges (unblocked algorithm).

Function/Subroutine Documentation
   subroutine cgetf2 (integer M, integer N, complex, dimension( lda, * ) A,
       integer LDA, integer, dimension( * ) IPIV, integer INFO)
       CGETF2 computes the LU factorization of a general m-by-n matrix using
       partial pivoting with row interchanges (unblocked algorithm).

Purpose:
   CGETF2 computes an LU factorization of a general m-by-n matrix A
   using partial pivoting with row interchanges.

   The factorization has the form
      A = P * L * U
   where P is a permutation matrix, L is lower triangular with unit
   diagonal elements (lower trapezoidal if m > n), and U is upper
   triangular (upper trapezoidal if m < n).

 This is the right-looking Level 2 BLAS version of the algorithm.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the m by n matrix to be factored.
   On exit, the factors L and U from the factorization
   A = P*L*U; the unit diagonal elements of L are not stored.
```

```
 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 IPIV
   IPIV is INTEGER array, dimension (min(M,N))
   The pivot indices; for 1 <= i <= min(M,N), row i of the
   matrix was interchanged with row IPIV(i).

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -k, the k-th argument had an illegal value
   > 0: if INFO = k, U(k,k) is exactly zero. The factorization
        has been completed, but the factor U is exactly
        singular, and division by zero will occur if it is used
        to solve a system of equations.
```

34.29.3 fortran code

— cgetf2.f —

```
*  =====================================================================
      SUBROUTINE CGETF2( M, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ONE, ZERO
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ),
     $                   ZERO = ( 0.0E+0, 0.0E+0 ) )
*     ..
```

```
*     .. Local Scalars ..
      REAL               SFMIN
      INTEGER            I, J, JP
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      INTEGER            ICAMAX
      EXTERNAL           SLAMCH, ICAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGERU, CSCAL, CSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGETF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Compute machine safe minimum
*
      SFMIN = SLAMCH('S')
*
      DO 10 J = 1, MIN( M, N )
*
*        Find pivot and test for singularity.
*
         JP = J - 1 + ICAMAX( M-J+1, A( J, J ), 1 )
         IPIV( J ) = JP
         IF( A( JP, J ).NE.ZERO ) THEN
*
*           Apply the interchange to columns 1:N.
```

```
*
           IF( JP.NE.J )
     $        CALL CSWAP( N, A( J, 1 ), LDA, A( JP, 1 ), LDA )
*
*          Compute elements J+1:M of J-th column.
*
           IF( J.LT.M ) THEN
              IF( ABS(A( J, J )) .GE. SFMIN ) THEN
                 CALL CSCAL( M-J, ONE / A( J, J ), A( J+1, J ), 1 )
              ELSE
                 DO 20 I = 1, M-J
                    A( J+I, J ) = A( J+I, J ) / A( J, J )
   20            CONTINUE
              END IF
           END IF
*
        ELSE IF( INFO.EQ.0 ) THEN
*
           INFO = J
        END IF
*
        IF( J.LT.MIN( M, N ) ) THEN
*
*          Update trailing submatrix.
*
           CALL CGERU( M-J, N-J, -ONE, A( J+1, J ), 1, A( J, J+1 ),
     $                 LDA, A( J+1, J+1 ), LDA )
        END IF
   10 CONTINUE
      RETURN
*
*     End of CGETF2
*
      END
```

———-

34.29.4 lisp code

— cgetf2.lisp —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)) (zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (ignorable one zero))
  (defun cgetf2 (m n a lda ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) a)
```

```
          (type (f2cl-lib:integer4) info lda n m))
(f2cl-lib:with-multi-array-data
    ((a f2cl-lib:complex8 a-%data% a-%offset%)
     (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
  (prog ((i 0) (j 0) (jp 0) (sfmin 0.0))
    (declare (type (f2cl-lib:integer4) i j jp)
             (type (single-float) sfmin))
    (setf info 0)
    (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
          ((< lda
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
           (setf info -4)))
    (cond
     ((/= info 0) (xerbla "CGETF2" (f2cl-lib:int-sub info))
      (go end_label)))
    (if (or (= m 0) (= n 0))
        (go end_label))
    (setf sfmin (slamch "S"))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j
                      (min (the f2cl-lib:integer4 m)
                           (the f2cl-lib:integer4 n)))
                   nil)
      (tagbody
        (setf jp
                (f2cl-lib:int-add (f2cl-lib:int-sub j 1)
                                  (icamax
                                   (f2cl-lib:int-add (f2cl-lib:int-sub m j)
                                                     1)
                                   (f2cl-lib:array-slice a-%data%
                                                         f2cl-lib:complex8
                                                         (j j)
                                                         ((1 lda) (1 *))
                                                         a-%offset%)
                                   1)))
        (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%) jp)
        (cond
         ((/= (f2cl-lib:fref a (jp j) ((1 lda) (1 *))) zero)
          (if (/= jp j)
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (cswap n
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (j 1)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (jp 1)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda)
                (declare (ignore var-1 var-3))
                (when var-0 (setf n var-0))
                (when var-2 (setf lda var-2))
```

```
                (when var-4 (setf lda var-4))))
          (cond
           ((< j m)
            (cond
             ((>= (abs (f2cl-lib:fref a (j j) ((1 lda) (1 *)))) sfmin)
              (cscal (f2cl-lib:int-sub m j)
               (/ one
                  (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                 a-%offset%))
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 ((+ j 1) j)
                                     ((1 lda) (1 *)) a-%offset%)
               1))
             (t
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i
                                (f2cl-lib:int-add m (f2cl-lib:int-sub j)))
                             nil)
                (tagbody
                  (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add j i) j)
                                       ((1 lda) (1 *)) a-%offset%)
                        (/
                         (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-add j i) j)
                                        ((1 lda) (1 *)) a-%offset%)
                         (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                        a-%offset%)))
                  label20)))))))
       ((= info 0) (setf info j)))
      (cond
       ((< j (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (cgeru (f2cl-lib:int-sub m j) (f2cl-lib:int-sub n j) (- one)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 ((+ j 1) j)
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   (j (f2cl-lib:int-add j 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   ((+ j 1) (f2cl-lib:int-add j 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
        (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7))
        (when var-6 (setf lda var-6))
        (when var-8 (setf lda var-8)))))
    label10))
 (go end_label)
end_label
```

```
        (return (values nil n nil lda nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgetf2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

34.30 cgetrf

34.30.1 Axiom unit tests

— **cgetrf.input** —

```
)set break resume
)sys rm -f cgetrf.output
)spool cgetrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.30.2 Axiom help page

— cgetrf.help —

```
=====================================================================
cgetrf examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgetrf (M, N, A, LDA, IPIV, INFO)
           CGETRF

Function/Subroutine Documentation
   subroutine cgetrf (integer M, integer N, complex, dimension( lda, * ) A,
       integer LDA, integer, dimension( * ) IPIV, integer INFO)
       CGETRF

Purpose:
   CGETRF computes an LU factorization of a general M-by-N matrix A
   using partial pivoting with row interchanges.

   The factorization has the form
      A = P * L * U
   where P is a permutation matrix, L is lower triangular with unit
   diagonal elements (lower trapezoidal if m > n), and U is upper
   triangular (upper trapezoidal if m < n).

   This is the right-looking Level 3 BLAS version of the algorithm.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the M-by-N matrix to be factored.
   On exit, the factors L and U from the factorization
   A = P*L*U; the unit diagonal elements of L are not stored.
```

```
 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 IPIV
   IPIV is INTEGER array, dimension (min(M,N))
   The pivot indices; for 1 <= i <= min(M,N), row i of the
   matrix was interchanged with row IPIV(i).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, U(i,i) is exactly zero. The factorization
         has been completed, but the factor U is exactly
         singular, and division by zero will occur if it is used
         to solve a system of equations.
```

34.30.3 fortran code

— cgetrf.f —

```
*  =====================================================================
      SUBROUTINE CGETRF( M, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ONE
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
```

```
*     .. Local Scalars ..
      INTEGER            I, IINFO, J, JB, NB
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEMM, CGETRF2, CLASWP, CTRSM, XERBLA
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGETRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Determine the block size for this environment.
*
      NB = ILAENV( 1, 'CGETRF', ' ', M, N, -1, -1 )
      IF( NB.LE.1 .OR. NB.GE.MIN( M, N ) ) THEN
*
*        Use unblocked code.
*
         CALL CGETRF2( M, N, A, LDA, IPIV, INFO )
      ELSE
*
*        Use blocked code.
*
         DO 20 J = 1, MIN( M, N ), NB
            JB = MIN( MIN( M, N )-J+1, NB )
*
```

```
*          Factor diagonal and subdiagonal blocks and test for exact
*          singularity.
*
           CALL CGETRF2( M-J+1, JB, A( J, J ), LDA, IPIV( J ), IINFO )
*
*          Adjust INFO and the pivot indices.
*
           IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $        INFO = IINFO + J - 1
           DO 10 I = J, MIN( M, J+JB-1 )
              IPIV( I ) = J - 1 + IPIV( I )
   10      CONTINUE
*
*          Apply interchanges to columns 1:J-1.
*
           CALL CLASWP( J-1, A, LDA, J, J+JB-1, IPIV, 1 )
*
           IF( J+JB.LE.N ) THEN
*
*             Apply interchanges to columns J+JB:N.
*
              CALL CLASWP( N-J-JB+1, A( 1, J+JB ), LDA, J, J+JB-1,
     $                     IPIV, 1 )
*
*             Compute block row of U.
*
              CALL CTRSM( 'Left', 'Lower', 'No transpose', 'Unit', JB,
     $                    N-J-JB+1, ONE, A( J, J ), LDA, A( J, J+JB ),
     $                    LDA )
              IF( J+JB.LE.M ) THEN
*
*                Update trailing submatrix.
*
                 CALL CGEMM( 'No transpose', 'No transpose', M-J-JB+1,
     $                       N-J-JB+1, JB, -ONE, A( J+JB, J ), LDA,
     $                       A( J, J+JB ), LDA, ONE, A( J+JB, J+JB ),
     $                       LDA )
              END IF
           END IF
   20    CONTINUE
      END IF
      RETURN
*
*     End of CGETRF
*
      END
```

———-

34.30.4 lisp code

— cgetrf.lisp —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (ignorable one))
  (defun cgetrf (m n a lda ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((i 0) (iinfo 0) (j 0) (jb 0) (nb 0))
        (declare (type (f2cl-lib:integer4) i iinfo j jb nb))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "CGETRF" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (or (= m 0) (= n 0))
            (go end_label))
        (setf nb
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (ilaenv 1 "CGETRF" " " m n -1 -1)
                  (declare (ignore var-0 var-1 var-2 var-5 var-6))
                  (when var-3 (setf m var-3))
                  (when var-4 (setf n var-4))
                  ret-val))
        (cond
         ((or (<= nb 1)
              (>= nb
                  (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n))))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
              (cgetrf2 m n a lda ipiv info)
            (declare (ignore var-2 var-4))
            (when var-0 (setf m var-0))
            (when var-1 (setf n var-1))
            (when var-3 (setf lda var-3))
            (when var-5 (setf info var-5))))
         (t
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                        ((> j
                            (min (the f2cl-lib:integer4 m)
```

```
                    (the f2cl-lib:integer4 n)))
            nil)
(tagbody
  (setf jb
          (min
           (the f2cl-lib:integer4
                (f2cl-lib:int-add
                 (f2cl-lib:int-sub
                  (min (the f2cl-lib:integer4 m)
                       (the f2cl-lib:integer4 n))
                  j)
                 1))
           (the f2cl-lib:integer4 nb)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (cgetrf2 (f2cl-lib:int-add (f2cl-lib:int-sub m j) 1) jb
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (j j)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (j)
                             ((1 *)) ipiv-%offset%)
       iinfo)
    (declare (ignore var-0 var-2 var-4))
    (when var-1 (setf jb var-1))
    (when var-3 (setf lda var-3))
    (when var-5 (setf iinfo var-5)))
  (if (and (= info 0) (> iinfo 0))
      (setf info (f2cl-lib:int-sub (f2cl-lib:int-add iinfo j) 1)))
  (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                ((> i
                    (min (the f2cl-lib:integer4 m)
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add j jb
                                                (f2cl-lib:int-sub
                                                 1)))))
                nil)
    (tagbody
      (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
              (f2cl-lib:int-add (f2cl-lib:int-sub j 1)
                                (f2cl-lib:fref ipiv-%data% (i)
                                               ((1 *))
                                               ipiv-%offset%)))
     label10))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (claswp (f2cl-lib:int-sub j 1) a lda j
       (f2cl-lib:int-sub (f2cl-lib:int-add j jb) 1) ipiv 1)
    (declare (ignore var-0 var-1 var-4 var-5 var-6))
    (when var-2 (setf lda var-2))
    (when var-3 (setf j var-3)))
  (cond
   ((<= (f2cl-lib:int-add j jb) n)
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (claswp (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                           (1 (f2cl-lib:int-add j jb))
                           ((1 lda) (1 *)) a-%offset%)
     lda j (f2cl-lib:int-sub (f2cl-lib:int-add j jb) 1) ipiv 1)
  (declare (ignore var-0 var-1 var-4 var-5 var-6))
  (when var-2 (setf lda var-2))
  (when var-3 (setf j var-3)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (ctrsm "Left" "Lower" "No transpose" "Unit" jb
     (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) one
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (j j)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                           (j (f2cl-lib:int-add j jb))
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9))
  (when var-4 (setf jb var-4))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf lda var-10)))
(cond
 ((<= (f2cl-lib:int-add j jb) m)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (cgemm "No transpose" "No transpose"
       (f2cl-lib:int-add (f2cl-lib:int-sub m j jb) 1)
       (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) jb
       (- one)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ j jb) j) ((1 lda) (1 *))
                             a-%offset%)
       lda
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             (j (f2cl-lib:int-add j jb))
                             ((1 lda) (1 *)) a-%offset%)
       lda one
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ j jb) (f2cl-lib:int-add j jb))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8 var-11))
```

```
                    (when var-4 (setf jb var-4))
                    (when var-7 (setf lda var-7))
                    (when var-9 (setf lda var-9))
                    (when var-10 (setf one var-10))
                    (when var-12 (setf lda var-12)))))))
             label20))))
        (go end_label)
       end_label
        (return (values m n nil lda nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgetrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

34.31 cgetrf2

34.31.1 Axiom unit tests

— cgetrf2.input —

```
)set break resume
)sys rm -f cgetrf2.output
)spool cgetrf2.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

34.31.2 Axiom help page

— **cgetrf2.help** —

```
=====================================================================
cgetrf2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       recursive subroutine cgetrf2 (M, N, A, LDA, IPIV, INFO)
           CGETRF2

Function/Subroutine Documentation
   recursive subroutine cgetrf2 (integer M, integer N, complex, dimension(
       lda, * ) A, integer LDA, integer, dimension( * ) IPIV, integer INFO)
       CGETRF2

Purpose:
   CGETRF2 computes an LU factorization of a general M-by-N matrix A
   using partial pivoting with row interchanges.

   The factorization has the form
      A = P * L * U
   where P is a permutation matrix, L is lower triangular with unit
   diagonal elements (lower trapezoidal if m > n), and U is upper
   triangular (upper trapezoidal if m < n).

   This is the recursive version of the algorithm. It divides
   the matrix into four submatrices:

         [  A11 | A12  ]  where A11 is n1 by n1 and A22 is n2 by n2
     A = [ -----|----- ]  with n1 = min(m,n)
         [  A21 | A22  ]       n2 = n-n1

                                           [ A11 ]
   The subroutine calls itself to factor [ --- ],
                                           [ A12 ]
```

```
                  [ A12 ]
   do the swaps on [ --- ], solve A12, update A22,
                  [ A22 ]

   then calls itself to factor A22 and do the swaps on A21.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the M-by-N matrix to be factored.
   On exit, the factors L and U from the factorization
   A = P*L*U; the unit diagonal elements of L are not stored.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 IPIV
   IPIV is INTEGER array, dimension (min(M,N))
   The pivot indices; for 1 <= i <= min(M,N), row i of the
   matrix was interchanged with row IPIV(i).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, U(i,i) is exactly zero. The factorization
         has been completed, but the factor U is exactly
         singular, and division by zero will occur if it is used
         to solve a system of equations.
```

34.31.3 fortran code

— cgetrf2.f —

```
*  =====================================================================
```

```
      RECURSIVE SUBROUTINE CGETRF2( M, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ONE, ZERO
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ),
     $                     ZERO = ( 0.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      REAL               SFMIN
      COMPLEX            TEMP
      INTEGER            I, IINFO, N1, N2
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      INTEGER            ICAMAX
      EXTERNAL           SLAMCH, ICAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEMM, CSCAL, CLASWP, CTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
```

```
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGETRF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN

      IF ( M.EQ.1 ) THEN
*
*        Use unblocked code for one row case
*        Just need to handle IPIV and INFO
*
         IPIV( 1 ) = 1
         IF ( A(1,1).EQ.ZERO )
     $      INFO = 1
*
      ELSE IF( N.EQ.1 ) THEN
*
*        Use unblocked code for one column case
*
*
*        Compute machine safe minimum
*
         SFMIN = SLAMCH('S')
*
*        Find pivot and test for singularity
*
         I = ICAMAX( M, A( 1, 1 ), 1 )
         IPIV( 1 ) = I
         IF( A( I, 1 ).NE.ZERO ) THEN
*
*           Apply the interchange
*
            IF( I.NE.1 ) THEN
               TEMP = A( 1, 1 )
               A( 1, 1 ) = A( I, 1 )
               A( I, 1 ) = TEMP
            END IF
*
*           Compute elements 2:M of the column
*
            IF( ABS(A( 1, 1 )) .GE. SFMIN ) THEN
               CALL CSCAL( M-1, ONE / A( 1, 1 ), A( 2, 1 ), 1 )
            ELSE
               DO 10 I = 1, M-1
                  A( 1+I, 1 ) = A( 1+I, 1 ) / A( 1, 1 )
   10          CONTINUE
```

```
            END IF
*
         ELSE
            INFO = 1
         END IF
*
      ELSE
*
*        Use recursive code
*
         N1 = MIN( M, N ) / 2
         N2 = N-N1
*
*               [ A11 ]
*        Factor [ --- ]
*               [ A21 ]
*
         CALL CGETRF2( M, N1, A, LDA, IPIV, IINFO )

         IF ( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO
*
*                               [ A12 ]
*        Apply interchanges to [ --- ]
*                               [ A22 ]
*
         CALL CLASWP( N2, A( 1, N1+1 ), LDA, 1, N1, IPIV, 1 )
*
*        Solve A12
*
         CALL CTRSM( 'L', 'L', 'N', 'U', N1, N2, ONE, A, LDA,
     $               A( 1, N1+1 ), LDA )
*
*        Update A22
*
         CALL CGEMM( 'N', 'N', M-N1, N2, N1, -ONE, A( N1+1, 1 ), LDA,
     $               A( 1, N1+1 ), LDA, ONE, A( N1+1, N1+1 ), LDA )
*
*        Factor A22
*
         CALL CGETRF2( M-N1, N2, A( N1+1, N1+1 ), LDA, IPIV( N1+1 ),
     $                 IINFO )
*
*        Adjust INFO and the pivot indices
*
         IF ( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO + N1
         DO 20 I = N1+1, MIN( M, N )
            IPIV( I ) = IPIV( I ) + N1
   20    CONTINUE
```

```
*
*        Apply interchanges to A21
*
         CALL CLASWP( N1, A( 1, 1 ), LDA, N1+1, MIN( M, N), IPIV, 1 )
*
      END IF
      RETURN
*
*     End of CGETRF2
*
      END
```

———-

34.31.4 lisp code

— **cgetrf2.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)) (zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (ignorable one zero))
  (defun *main* ()
    (prog ((i 0) (iinfo 0) (n1 0) (n2 0) (temp #C(0.0 0.0)) (sfmin 0.0) (m 0)
           (info 0) (ipiv (make-array * :element-type 'f2cl-lib:integer4))
           (lda 0)
           (a
            (make-array (the fixnum (reduce #'* (list lda *))) :element-type
                        'f2cl-lib:complex8))
           (n 0) (subroutine 0.0))
      (declare (type (array f2cl-lib:complex8 (*)) a)
               (type (array f2cl-lib:integer4 (*)) ipiv)
               (type (f2cl-lib:complex8) temp)
               (type (single-float) subroutine sfmin)
               (type (f2cl-lib:integer4) n lda info i iinfo n1 n2 m))
      '"****NOT TRANSLATED: (RECURSIVE SUBROUTINE CGETRF2 (M |,| N |,| A |,| LDA |,| IPIV |,| INFO))"
      (setf info 0)
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4)))
      (cond
       ((/= info 0) (xerbla "CGETRF2" (f2cl-lib:int-sub info)) (go end_label)))
      (if (or (= m 0) (= n 0))
          (go end_label))
      (cond
       ((= m 1) (setf (f2cl-lib:fref ipiv (1) ((1 *))) 1)
        (if (= (f2cl-lib:fref a (1 1) ((1 lda) (1 *))) zero)
            (setf info 1)))
```

```
((= n 1) (setf sfmin (slamch "S"))
 (setf i
         (multiple-value-bind (ret-val var-0 var-1 var-2)
             (icamax m
              (f2cl-lib:array-slice a f2cl-lib:complex8 (1 1)
                                    ((1 lda) (1 *)))
              1)
           (declare (ignore var-1 var-2))
           (when var-0 (setf m var-0))
           ret-val))
 (setf (f2cl-lib:fref ipiv (1) ((1 *))) i)
 (cond
  ((/= (f2cl-lib:fref a (i 1) ((1 lda) (1 *))) zero)
   (cond
    ((/= i 1) (setf temp (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))
     (setf (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))
             (f2cl-lib:fref a (i 1) ((1 lda) (1 *))))
     (setf (f2cl-lib:fref a (i 1) ((1 lda) (1 *))) temp)))
   (cond
    ((>= (abs (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))) sfmin)
     (cscal (f2cl-lib:int-sub m 1)
      (/ one (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))
      (f2cl-lib:array-slice a f2cl-lib:complex8 (2 1) ((1 lda) (1 *)))
      1))
    (t
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
       (tagbody
         (setf (f2cl-lib:fref a ((f2cl-lib:int-add 1 i) 1)
                               ((1 lda) (1 *)))
                 (/
                  (f2cl-lib:fref a ((f2cl-lib:int-add 1 i) 1)
                                 ((1 lda) (1 *)))
                  (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))))
        label10)))))
  (t (setf info 1))))
(t (setf n1 (the f2cl-lib:integer4 (truncate (min m n) 2)))
 (setf n2 (f2cl-lib:int-sub n n1))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
     (cgetrf2 m n1 a lda ipiv iinfo)
   (declare (ignore var-2 var-4))
   (when var-0 (setf m var-0))
   (when var-1 (setf n1 var-1))
   (when var-3 (setf lda var-3))
   (when var-5 (setf iinfo var-5)))
 (if (and (= info 0) (> iinfo 0))
     (setf info iinfo))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (claswp n2
      (f2cl-lib:array-slice a f2cl-lib:complex8
```

```
                          (1 (f2cl-lib:int-add n1 1)) ((1 lda) (1 *)))
      lda 1 n1 ipiv 1)
  (declare (ignore var-1 var-3 var-5 var-6))
  (when var-0 (setf n2 var-0))
  (when var-2 (setf lda var-2))
  (when var-4 (setf n1 var-4)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ctrsm "L" "L" "N" "U" n1 n2 one a lda
     (f2cl-lib:array-slice a f2cl-lib:complex8
                          (1 (f2cl-lib:int-add n1 1)) ((1 lda) (1 *)))
     lda)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf n2 var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf lda var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (cgemm "N" "N" (f2cl-lib:int-sub m n1) n2 n1 (- one)
     (f2cl-lib:array-slice a f2cl-lib:complex8 ((+ n1 1) 1)
                          ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice a f2cl-lib:complex8
                          (1 (f2cl-lib:int-add n1 1)) ((1 lda) (1 *)))
     lda one
     (f2cl-lib:array-slice a f2cl-lib:complex8
                          ((+ n1 1) (f2cl-lib:int-add n1 1))
                          ((1 lda) (1 *)))
     lda)
  (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
  (when var-3 (setf n2 var-3))
  (when var-4 (setf n1 var-4))
  (when var-7 (setf lda var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf lda var-12)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (cgetrf2 (f2cl-lib:int-sub m n1) n2
     (f2cl-lib:array-slice a f2cl-lib:complex8
                          ((+ n1 1) (f2cl-lib:int-add n1 1))
                          ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice ipiv f2cl-lib:integer4 ((+ n1 1)) ((1 *)))
     iinfo)
  (declare (ignore var-0 var-2 var-4))
  (when var-1 (setf n2 var-1))
```

```
          (when var-3 (setf lda var-3))
          (when var-5 (setf iinfo var-5)))
        (if (and (= info 0) (> iinfo 0))
            (setf info (f2cl-lib:int-add iinfo n1)))
        (f2cl-lib:fdo (i (f2cl-lib:int-add n1 1) (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the f2cl-lib:integer4 m)
                               (the f2cl-lib:integer4 n)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref ipiv (i) ((1 *)))
                    (f2cl-lib:int-add (f2cl-lib:fref ipiv (i) ((1 *))) n1))
           label20))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (claswp n1
             (f2cl-lib:array-slice a f2cl-lib:complex8 (1 1) ((1 lda) (1 *)))
             lda (f2cl-lib:int-add n1 1)
             (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) ipiv 1)
          (declare (ignore var-1 var-3 var-4 var-5 var-6))
          (when var-0 (setf n1 var-0))
          (when var-2 (setf lda var-2)))))
      (go end_label)
     end_label
      (return nil))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::*main*
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types 'nil :return-values 'nil
                                            :calls 'nil)))
```

———-

34.32 cgetri

34.32.1 Axiom unit tests

— **cgetri.input** —

```
)set break resume
)sys rm -f cgetri.output
)spool cgetri.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.32.2 Axiom help page

— **cgetri.help** —

```
=====================================================================
cgetri examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgetri (N, A, LDA, IPIV, WORK, LWORK, INFO)
           CGETRI

Function/Subroutine Documentation
   subroutine cgetri (integer N, complex, dimension( lda, * ) A, integer LDA,
       integer, dimension( * ) IPIV, complex, dimension( * ) WORK, integer
       LWORK, integer INFO)
       CGETRI

Purpose:
   CGETRI computes the inverse of a matrix using the LU factorization
   computed by CGETRF.

   This method inverts U and then computes inv(A) by solving the system
   inv(A)*L = inv(U) for inv(A).

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the factors L and U from the factorization
   A = P*L*U as computed by CGETRF.
```

```
  On exit, if INFO = 0, the inverse of the original matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from CGETRF; for 1<=i<=N, row i of the
  matrix was interchanged with row IPIV(i).

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO=0, then WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,N).
  For optimal performance LWORK >= N*NB, where NB is
  the optimal blocksize returned by ILAENV.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, U(i,i) is exactly zero; the matrix is
        singular and its inverse could not be computed.
```

———-

34.32.3 fortran code

— cgetri.f —

```
*  =====================================================================
      SUBROUTINE CGETRI( N, A, LDA, IPIV, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
```

```
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ZERO, ONE
      PARAMETER          ( ZERO = ( 0.0E+0, 0.0E+0 ),
     $                   ONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IWS, J, JB, JJ, JP, LDWORK, LWKOPT, NB,
     $                   NBMIN, NN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEMM, CGEMV, CSWAP, CTRSM, CTRTRI, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NB = ILAENV( 1, 'CGETRI', ' ', N, -1, -1, -1 )
      LWKOPT = N*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -3
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGETRI', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
```

```
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Form inv(U).  If INFO > 0 from CTRTRI, then U is singular,
*     and the inverse is not computed.
*
      CALL CTRTRI( 'Upper', 'Non-unit', N, A, LDA, INFO )
      IF( INFO.GT.0 )
     $   RETURN
*
      NBMIN = 2
      LDWORK = N
      IF( NB.GT.1 .AND. NB.LT.N ) THEN
         IWS = MAX( LDWORK*NB, 1 )
         IF( LWORK.LT.IWS ) THEN
            NB = LWORK / LDWORK
            NBMIN = MAX( 2, ILAENV( 2, 'CGETRI', ' ', N, -1, -1, -1 ) )
         END IF
      ELSE
         IWS = N
      END IF
*
*     Solve the equation inv(A)*L = inv(U) for inv(A).
*
      IF( NB.LT.NBMIN .OR. NB.GE.N ) THEN
*
*        Use unblocked code.
*
         DO 20 J = N, 1, -1
*
*           Copy current column of L to WORK and replace with zeros.
*
            DO 10 I = J + 1, N
               WORK( I ) = A( I, J )
               A( I, J ) = ZERO
   10       CONTINUE
*
*           Compute current column of inv(A).
*
            IF( J.LT.N )
     $         CALL CGEMV( 'No transpose', N, N-J, -ONE, A( 1, J+1 ),
     $                     LDA, WORK( J+1 ), 1, ONE, A( 1, J ), 1 )
   20    CONTINUE
      ELSE
*
```

```
*        Use blocked code.
*
         NN = ( ( N-1 ) / NB )*NB + 1
         DO 50 J = NN, 1, -NB
            JB = MIN( NB, N-J+1 )
*
*           Copy current block column of L to WORK and replace with
*           zeros.
*
            DO 40 JJ = J, J + JB - 1
               DO 30 I = JJ + 1, N
                  WORK( I+( JJ-J )*LDWORK ) = A( I, JJ )
                  A( I, JJ ) = ZERO
   30          CONTINUE
   40       CONTINUE
*
*           Compute current block column of inv(A).
*
            IF( J+JB.LE.N )
     $         CALL CGEMM( 'No transpose', 'No transpose', N, JB,
     $                     N-J-JB+1, -ONE, A( 1, J+JB ), LDA,
     $                     WORK( J+JB ), LDWORK, ONE, A( 1, J ), LDA )
            CALL CTRSM( 'Right', 'Lower', 'No transpose', 'Unit', N, JB,
     $                  ONE, WORK( J ), LDWORK, A( 1, J ), LDA )
   50    CONTINUE
      END IF
*
*     Apply column interchanges.
*
      DO 60 J = N - 1, 1, -1
         JP = IPIV( J )
         IF( JP.NE.J )
     $      CALL CSWAP( N, A( 1, J ), 1, A( 1, JP ), 1 )
   60 CONTINUE
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of CGETRI
*
      END
```

———-

34.32.4 lisp code

— cgetri.lisp —

```
(let* ((zero (f2cl-lib:cmplx 0.0 0.0)) (one (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:complex8) zero)
           (type (f2cl-lib:complex8) one)
           (ignorable zero one))
  (defun cgetri (n a lda ipiv work lwork info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) work a)
             (type (f2cl-lib:integer4) info lwork lda n))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((i 0) (iws 0) (j 0) (jb 0) (jj 0) (jp 0) (ldwork 0) (lwkopt 0)
             (nb 0) (nbmin 0) (nn 0) (lquery nil))
        (declare
         (type (f2cl-lib:integer4) i iws j jb jj jp ldwork lwkopt nb nbmin nn)
         (type f2cl-lib:logical lquery))
        (setf info 0)
        (setf nb
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (ilaenv 1 "CGETRI" " " n -1 -1 -1)
                  (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6))
                  (when var-3 (setf n var-3))
                  ret-val))
        (setf lwkopt (f2cl-lib:int-mul n nb))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce lwkopt 'f2cl-lib:complex8))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond ((< n 0) (setf info -1))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -3))
              ((and
                (< lwork
                   (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
                (not lquery))
               (setf info -6)))
        (cond
         ((/= info 0) (xerbla "CGETRI" (f2cl-lib:int-sub info)) (go end_label))
         (lquery (go end_label)))
        (if (= n 0)
            (go end_label))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (ctrtri "Upper" "Non-unit" n a lda info)
          (declare (ignore var-0 var-1 var-3))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          (when var-5 (setf info var-5)))
        (if (> info 0)
```

```
    (go end_label))
(setf nbmin 2)
(setf ldwork n)
(cond
 ((and (> nb 1) (< nb n))
  (setf iws
          (max (the f2cl-lib:integer4 (f2cl-lib:int-mul ldwork nb))
               (the f2cl-lib:integer4 1)))
  (cond
   ((< lwork iws)
    (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
    (setf nbmin
            (max (the f2cl-lib:integer4 2)
                 (the f2cl-lib:integer4
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
                          (ilaenv 2 "CGETRI" " " n -1 -1 -1)
                        (declare
                         (ignore var-0 var-1 var-2 var-4 var-5 var-6))
                        (when var-3 (setf n var-3))
                        ret-val)))))))
 (t (setf iws n)))
(cond
 ((or (< nb nbmin) (>= nb n))
  (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                ((> j 1) nil)
    (tagbody
      (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%))
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%)
                  zero)
         label10))
      (if (< j n)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10)
              (cgemv "No transpose" n (f2cl-lib:int-sub n j) (- one)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                     (1 (f2cl-lib:int-add j 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                     ((+ j 1)) ((1 *)) work-%offset%)
               1 one
```

```
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 j)
                                      ((1 lda) (1 *)) a-%offset%)
                1)
            (declare
             (ignore var-0 var-2 var-3 var-4 var-6 var-7 var-9 var-10))
            (when var-1 (setf n var-1))
            (when var-5 (setf lda var-5))
            (when var-8 (setf one var-8))))
    label20)))
(t
 (setf nn (+ (* (the f2cl-lib:integer4 (truncate (- n 1) nb)) nb) 1))
 (f2cl-lib:fdo (j nn (f2cl-lib:int-add j (f2cl-lib:int-sub nb)))
               ((> j 1) nil)
   (tagbody
     (setf jb
             (min (the f2cl-lib:integer4 nb)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1))))
     (f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
                   ((> jj
                       (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
                    nil)
       (tagbody
         (f2cl-lib:fdo (i (f2cl-lib:int-add jj 1)
                          (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add i
                                                     (f2cl-lib:int-mul
                                                      (f2cl-lib:int-sub
                                                       jj j)
                                                      ldwork)))
                                  ((1 *)) work-%offset%)
                     (f2cl-lib:fref a-%data% (i jj) ((1 lda) (1 *))
                                    a-%offset%))
             (setf (f2cl-lib:fref a-%data% (i jj) ((1 lda) (1 *))
                                  a-%offset%)
                     zero)
            label30))
        label40))
     (if (<= (f2cl-lib:int-add j jb) n)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9 var-10 var-11 var-12)
             (cgemm "No transpose" "No transpose" n jb
              (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) (- one)
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                    (1 (f2cl-lib:int-add j jb))
                                    ((1 lda) (1 *)) a-%offset%)
```

```
                lda
                (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                      ((+ j jb)) ((1 *)) work-%offset%)
                ldwork one
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 j)
                                      ((1 lda) (1 *)) a-%offset%)
                lda)
              (declare
               (ignore var-0 var-1 var-4 var-5 var-6 var-8 var-11))
              (when var-2 (setf n var-2))
              (when var-3 (setf jb var-3))
              (when var-7 (setf lda var-7))
              (when var-9 (setf ldwork var-9))
              (when var-10 (setf one var-10))
              (when var-12 (setf lda var-12))))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10)
            (ctrsm "Right" "Lower" "No transpose" "Unit" n jb one
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 (j)
                                   ((1 *)) work-%offset%)
             ldwork
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
          (when var-4 (setf n var-4))
          (when var-5 (setf jb var-5))
          (when var-6 (setf one var-6))
          (when var-8 (setf ldwork var-8))
          (when var-10 (setf lda var-10)))
      label50))))
(f2cl-lib:fdo (j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j 1) nil)
  (tagbody
    (setf jp (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
    (if (/= jp j)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (cswap n
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 jp)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf n var-0))))
   label60))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
```

```
                (coerce iws 'f2cl-lib:complex8))
        (go end_label)
       end_label
        (return (values n nil lda nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgetri
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

34.33 cgetrs

34.33.1 Axiom unit tests

— cgetrs.input —

```
)set break resume
)sys rm -f cgetrs.output
)spool cgetrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.33.2 Axiom help page

— **cgetrs.help** —

```
=====================================================================
cgetrs examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgetrs (TRANS, N, NRHS, A, LDA, IPIV, B, LDB, INFO)
           CGETRS

Function/Subroutine Documentation
   subroutine cgetrs (character TRANS, integer N, integer NRHS, complex,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       complex, dimension( ldb, * ) B, integer LDB, integer INFO)
       CGETRS

Purpose:
   CGETRS solves a system of linear equations
      A * X = B,  A**T * X = B,  or  A**H * X = B
   with a general N-by-N matrix A using the LU factorization computed
   by CGETRF.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.
```

```
A
  A is COMPLEX array, dimension (LDA,N)
  The factors L and U from the factorization A = P*L*U
  as computed by CGETRF.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from CGETRF; for 1<=i<=N, row i of the
  matrix was interchanged with row IPIV(i).

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the right hand side matrix B.
  On exit, the solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

34.33.3 fortran code

— cgetrs.f —

```
*  =====================================================================
      SUBROUTINE CGETRS( TRANS, N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
```

```
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ONE
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLASWP, CTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGETRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
```

```
     $   RETURN
*
      IF( NOTRAN ) THEN
*
*        Solve A * X = B.
*
*        Apply row interchanges to the right hand sides.
*
         CALL CLASWP( NRHS, B, LDB, 1, N, IPIV, 1 )
*
*        Solve L*X = B, overwriting B with X.
*
         CALL CTRSM( 'Left', 'Lower', 'No transpose', 'Unit', N, NRHS,
     $               ONE, A, LDA, B, LDB )
*
*        Solve U*X = B, overwriting B with X.
*
         CALL CTRSM( 'Left', 'Upper', 'No transpose', 'Non-unit', N,
     $               NRHS, ONE, A, LDA, B, LDB )
      ELSE
*
*        Solve A**T * X = B  or A**H * X = B.
*
*        Solve U**T *X = B or U**H *X = B, overwriting B with X.
*
         CALL CTRSM( 'Left', 'Upper', TRANS, 'Non-unit', N, NRHS, ONE,
     $               A, LDA, B, LDB )
*
*        Solve L**T *X = B, or L**H *X = B overwriting B with X.
*
         CALL CTRSM( 'Left', 'Lower', TRANS, 'Unit', N, NRHS, ONE, A,
     $               LDA, B, LDB )
*
*        Apply row interchanges to the solution vectors.
*
         CALL CLASWP( NRHS, B, LDB, 1, N, IPIV, -1 )
      END IF
*
      RETURN
*
*     End of CGETRS
*
      END
```

———-

34.33.4 lisp code

— cgetrs.lisp —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (ignorable one))
  (defun cgetrs (trans n nrhs a lda ipiv b ldb$ info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) b a)
             (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((notran nil))
        (declare (type f2cl-lib:logical notran))
        (setf info 0)
        (setf notran
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame trans "N")
                  (declare (ignore var-1))
                  (when var-0 (setf trans var-0))
                  ret-val))
        (cond
         ((and (not notran)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame trans "T")
                  (declare (ignore var-1))
                  (when var-0 (setf trans var-0))
                  ret-val))
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame trans "C")
                  (declare (ignore var-1))
                  (when var-0 (setf trans var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -5))
         ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -8)))
        (cond
         ((/= info 0) (xerbla "CGETRS" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (or (= n 0) (= nrhs 0))
            (go end_label))
        (cond
```

```
(notran
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (claswp nrhs b ldb$ 1 n ipiv 1)
   (declare (ignore var-1 var-3 var-5 var-6))
   (when var-0 (setf nrhs var-0))
   (when var-2 (setf ldb$ var-2))
   (when var-4 (setf n var-4)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (ctrsm "Left" "Lower" "No transpose" "Unit" n nrhs one a lda b
      ldb$)
   (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
   (when var-4 (setf n var-4))
   (when var-5 (setf nrhs var-5))
   (when var-6 (setf one var-6))
   (when var-8 (setf lda var-8))
   (when var-10 (setf ldb$ var-10)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (ctrsm "Left" "Upper" "No transpose" "Non-unit" n nrhs one a lda
      b ldb$)
   (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
   (when var-4 (setf n var-4))
   (when var-5 (setf nrhs var-5))
   (when var-6 (setf one var-6))
   (when var-8 (setf lda var-8))
   (when var-10 (setf ldb$ var-10))))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (ctrsm "Left" "Upper" trans "Non-unit" n nrhs one a lda b ldb$)
   (declare (ignore var-0 var-1 var-3 var-7 var-9))
   (when var-2 (setf trans var-2))
   (when var-4 (setf n var-4))
   (when var-5 (setf nrhs var-5))
   (when var-6 (setf one var-6))
   (when var-8 (setf lda var-8))
   (when var-10 (setf ldb$ var-10)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (ctrsm "Left" "Lower" trans "Unit" n nrhs one a lda b ldb$)
   (declare (ignore var-0 var-1 var-3 var-7 var-9))
   (when var-2 (setf trans var-2))
   (when var-4 (setf n var-4))
   (when var-5 (setf nrhs var-5))
   (when var-6 (setf one var-6))
```

```
            (when var-8 (setf lda var-8))
            (when var-10 (setf ldb$ var-10)))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (claswp nrhs b ldb$ 1 n ipiv -1)
            (declare (ignore var-1 var-3 var-5 var-6))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf n var-4)))))
        (go end_label)
       end_label
        (return (values trans n nrhs nil lda nil nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgetrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

34.34 chgeqz

34.34.1 Axiom unit tests

— **chgeqz.input** —

```
)set break resume
)sys rm -f chgeqz.output
)spool chgeqz.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.34.2 Axiom help page [Mole71]

— **chgeqz.help** —

```
======================================================================
chgeqz examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine chgeqz (JOB, COMPQ, COMPZ, N, ILO, IHI, H, LDH, T, LDT,
           ALPHA, BETA, Q, LDQ, Z, LDZ, WORK, LWORK, RWORK, INFO)
           CHGEQZ

Function/Subroutine Documentation
   subroutine chgeqz (character JOB, character COMPQ, character COMPZ, integer
       N, integer ILO, integer IHI, complex, dimension( ldh, * ) H, integer
       LDH, complex, dimension( ldt, * ) T, integer LDT, complex, dimension( *
       ) ALPHA, complex, dimension( * ) BETA, complex, dimension( ldq, * ) Q,
       integer LDQ, complex, dimension( ldz, * ) Z, integer LDZ, complex,
       dimension( * ) WORK, integer LWORK, real, dimension( * ) RWORK, integer
       INFO)
       CHGEQZ

Purpose:
   CHGEQZ computes the eigenvalues of a complex matrix pair (H,T),
   where H is an upper Hessenberg matrix and T is upper triangular,
   using the single-shift QZ method.
   Matrix pairs of this type are produced by the reduction to
   generalized upper Hessenberg form of a complex matrix pair (A,B):
```

```
  A = Q1*H*Z1**H,  B = Q1*T*Z1**H,

  as computed by CGGHRD.

  If JOB='S', then the Hessenberg-triangular pair (H,T) is
  also reduced to generalized Schur form,

  H = Q*S*Z**H,  T = Q*P*Z**H,

  where Q and Z are unitary matrices and S and P are upper triangular.

  Optionally, the unitary matrix Q from the generalized Schur
  factorization may be postmultiplied into an input matrix Q1, and the
  unitary matrix Z may be postmultiplied into an input matrix Z1.
  If Q1 and Z1 are the unitary matrices from CGGHRD that reduced
  the matrix pair (A,B) to generalized Hessenberg form, then the output
  matrices Q1*Q and Z1*Z are the unitary factors from the generalized
  Schur factorization of (A,B):

  A = (Q1*Q)*S*(Z1*Z)**H,  B = (Q1*Q)*P*(Z1*Z)**H.

  To avoid overflow, eigenvalues of the matrix pair (H,T)
  (equivalently, of (A,B)) are computed as a pair of complex values
  (alpha,beta).  If beta is nonzero, lambda = alpha / beta is an
  eigenvalue of the generalized nonsymmetric eigenvalue problem (GNEP)
  A*x = lambda*B*x
  and if alpha is nonzero, mu = beta / alpha is an eigenvalue of the
  alternate form of the GNEP
  mu*A*y = B*y.
  The values of alpha and beta for the i-th eigenvalue can be read
  directly from the generalized Schur form:  alpha = S(i,i),
  beta = P(i,i).

Parameters:

JOB
  JOB is CHARACTER*1
  = 'E': Compute eigenvalues only;
  = 'S': Computer eigenvalues and the Schur form.

COMPQ
  COMPQ is CHARACTER*1
  = 'N': Left Schur vectors (Q) are not computed;
  = 'I': Q is initialized to the unit matrix and the matrix Q
     of left Schur vectors of (H,T) is returned;
  = 'V': Q must contain a unitary matrix Q1 on entry and
     the product Q1*Q is returned.

COMPZ
```

```
  COMPZ is CHARACTER*1
  = 'N': Right Schur vectors (Z) are not computed;
  = 'I': Q is initialized to the unit matrix and the matrix Z
     of right Schur vectors of (H,T) is returned;
  = 'V': Z must contain a unitary matrix Z1 on entry and
     the product Z1*Z is returned.

N
  N is INTEGER
  The order of the matrices H, T, Q, and Z.  N >= 0.

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER
  ILO and IHI mark the rows and columns of H which are in
  Hessenberg form.  It is assumed that A is already upper
  triangular in rows and columns 1:ILO-1 and IHI+1:N.
  If N > 0, 1 <= ILO <= IHI <= N; if N = 0, ILO=1 and IHI=0.

H
  H is COMPLEX array, dimension (LDH, N)
  On entry, the N-by-N upper Hessenberg matrix H.
  On exit, if JOB = 'S', H contains the upper triangular
  matrix S from the generalized Schur factorization.
  If JOB = 'E', the diagonal of H matches that of S, but
  the rest of H is unspecified.

LDH
  LDH is INTEGER
  The leading dimension of the array H.  LDH >= max( 1, N ).

T
  T is COMPLEX array, dimension (LDT, N)
  On entry, the N-by-N upper triangular matrix T.
  On exit, if JOB = 'S', T contains the upper triangular
  matrix P from the generalized Schur factorization.
  If JOB = 'E', the diagonal of T matches that of P, but
  the rest of T is unspecified.

LDT
  LDT is INTEGER
  The leading dimension of the array T.  LDT >= max( 1, N ).

ALPHA
  ALPHA is COMPLEX array, dimension (N)
  The complex scalars alpha that define the eigenvalues of
  GNEP.  ALPHA(i) = S(i,i) in the generalized Schur factorization.
```

```
BETA
  BETA is COMPLEX array, dimension (N)
  The real non-negative scalars beta that define the
  eigenvalues of GNEP.  BETA(i) = P(i,i) in the generalized
  Schur factorization.

  Together, the quantities alpha = ALPHA(j) and beta = BETA(j)
  represent the j-th eigenvalue of the matrix pair (A,B), in
  one of the forms lambda = alpha/beta or mu = beta/alpha.
  Since either lambda or mu may overflow, they should not,
  in general, be computed.

Q
  Q is COMPLEX array, dimension (LDQ, N)
  On entry, if COMPZ = 'V', the unitary matrix Q1 used in the
  reduction of (A,B) to generalized Hessenberg form.
  On exit, if COMPZ = 'I', the unitary matrix of left Schur
  vectors of (H,T), and if COMPZ = 'V', the unitary matrix of
  left Schur vectors of (A,B).
  Not referenced if COMPZ = 'N'.

LDQ
  LDQ is INTEGER
  The leading dimension of the array Q.  LDQ >= 1.
  If COMPQ='V' or 'I', then LDQ >= N.

Z
  Z is COMPLEX array, dimension (LDZ, N)
  On entry, if COMPZ = 'V', the unitary matrix Z1 used in the
  reduction of (A,B) to generalized Hessenberg form.
  On exit, if COMPZ = 'I', the unitary matrix of right Schur
  vectors of (H,T), and if COMPZ = 'V', the unitary matrix of
  right Schur vectors of (A,B).
  Not referenced if COMPZ = 'N'.

LDZ
  LDZ is INTEGER
  The leading dimension of the array Z.  LDZ >= 1.
  If COMPZ='V' or 'I', then LDZ >= N.

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO >= 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,N).

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
```

```
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is REAL array, dimension (N)

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  = 1,...,N: the QZ iteration did not converge.  (H,T) is not
     in Schur form, but ALPHA(i) and BETA(i),
     i=INFO+1,...,N should be correct.
  = N+1,...,2*N: the shift calculation failed.  (H,T) is not
     in Schur form, but ALPHA(i) and BETA(i),
     i=INFO-N+1,...,N should be correct.

  We assume that complex ABS works as long as its value is less than
  overflow.
```

34.34.3 fortran code

— chgeqz.f —

```
*  =====================================================================
      SUBROUTINE CHGEQZ( JOB, COMPQ, COMPZ, N, ILO, IHI, H, LDH, T, LDT,
     $                   ALPHA, BETA, Q, LDQ, Z, LDZ, WORK, LWORK,
     $                   RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          COMPQ, COMPZ, JOB
      INTEGER            IHI, ILO, INFO, LDH, LDQ, LDT, LDZ, LWORK, N
*     ..
*     .. Array Arguments ..
      REAL               RWORK( * )
      COMPLEX            ALPHA( * ), BETA( * ), H( LDH, * ),
     $                   Q( LDQ, * ), T( LDT, * ), WORK( * ),
     $                   Z( LDZ, * )
*     ..
*
```

```
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0E+0, 0.0E+0 ),
     $                   CONE = ( 1.0E+0, 0.0E+0 ) )
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               HALF
      PARAMETER          ( HALF = 0.5E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ILAZR2, ILAZRO, ILQ, ILSCHR, ILZ, LQUERY
      INTEGER            ICOMPQ, ICOMPZ, IFIRST, IFRSTM, IITER, ILAST,
     $                   ILASTM, IN, ISCHUR, ISTART, J, JC, JCH, JITER,
     $                   JR, MAXIT
      REAL               ABSB, ANORM, ASCALE, ATOL, BNORM, BSCALE, BTOL,
     $                   C, SAFMIN, TEMP, TEMP2, TEMPR, ULP
      COMPLEX            ABI22, AD11, AD12, AD21, AD22, CTEMP, CTEMP2,
     $                   CTEMP3, ESHIFT, RTDISC, S, SHIFT, SIGNBC, T1,
     $                   U12, X
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               CLANHS, SLAMCH
      EXTERNAL           LSAME, CLANHS, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLARTG, CLASET, CROT, CSCAL, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, CMPLX, CONJG, MAX, MIN, REAL, SQRT
*     ..
*     .. Statement Functions ..
      REAL               ABS1
*     ..
*     .. Statement Function definitions ..
      ABS1( X ) = ABS( REAL( X ) ) + ABS( AIMAG( X ) )
*     ..
*     .. Executable Statements ..
*
*     Decode JOB, COMPQ, COMPZ
*
      IF( LSAME( JOB, 'E' ) ) THEN
         ILSCHR = .FALSE.
         ISCHUR = 1
      ELSE IF( LSAME( JOB, 'S' ) ) THEN
         ILSCHR = .TRUE.
         ISCHUR = 2
      ELSE
```

```
         ISCHUR = 0
      END IF
*
      IF( LSAME( COMPQ, 'N' ) ) THEN
         ILQ = .FALSE.
         ICOMPQ = 1
      ELSE IF( LSAME( COMPQ, 'V' ) ) THEN
         ILQ = .TRUE.
         ICOMPQ = 2
      ELSE IF( LSAME( COMPQ, 'I' ) ) THEN
         ILQ = .TRUE.
         ICOMPQ = 3
      ELSE
         ICOMPQ = 0
      END IF
*
      IF( LSAME( COMPZ, 'N' ) ) THEN
         ILZ = .FALSE.
         ICOMPZ = 1
      ELSE IF( LSAME( COMPZ, 'V' ) ) THEN
         ILZ = .TRUE.
         ICOMPZ = 2
      ELSE IF( LSAME( COMPZ, 'I' ) ) THEN
         ILZ = .TRUE.
         ICOMPZ = 3
      ELSE
         ICOMPZ = 0
      END IF
*
*     Check Argument Values
*
      INFO = 0
      WORK( 1 ) = MAX( 1, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( ISCHUR.EQ.0 ) THEN
         INFO = -1
      ELSE IF( ICOMPQ.EQ.0 ) THEN
         INFO = -2
      ELSE IF( ICOMPZ.EQ.0 ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( ILO.LT.1 ) THEN
         INFO = -5
      ELSE IF( IHI.GT.N .OR. IHI.LT.ILO-1 ) THEN
         INFO = -6
      ELSE IF( LDH.LT.N ) THEN
         INFO = -8
      ELSE IF( LDT.LT.N ) THEN
         INFO = -10
```

```
      ELSE IF( LDQ.LT.1 .OR. ( ILQ .AND. LDQ.LT.N ) ) THEN
         INFO = -14
      ELSE IF( LDZ.LT.1 .OR. ( ILZ .AND. LDZ.LT.N ) ) THEN
         INFO = -16
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -18
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CHGEQZ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
*     WORK( 1 ) = CMPLX( 1 )
      IF( N.LE.0 ) THEN
         WORK( 1 ) = CMPLX( 1 )
         RETURN
      END IF
*
*     Initialize Q and Z
*
      IF( ICOMPQ.EQ.3 )
     $   CALL CLASET( 'Full', N, N, CZERO, CONE, Q, LDQ )
      IF( ICOMPZ.EQ.3 )
     $   CALL CLASET( 'Full', N, N, CZERO, CONE, Z, LDZ )
*
*     Machine Constants
*
      IN = IHI + 1 - ILO
      SAFMIN = SLAMCH( 'S' )
      ULP = SLAMCH( 'E' )*SLAMCH( 'B' )
      ANORM = CLANHS( 'F', IN, H( ILO, ILO ), LDH, RWORK )
      BNORM = CLANHS( 'F', IN, T( ILO, ILO ), LDT, RWORK )
      ATOL = MAX( SAFMIN, ULP*ANORM )
      BTOL = MAX( SAFMIN, ULP*BNORM )
      ASCALE = ONE / MAX( SAFMIN, ANORM )
      BSCALE = ONE / MAX( SAFMIN, BNORM )
*
*
*     Set Eigenvalues IHI+1:N
*
      DO 10 J = IHI + 1, N
         ABSB = ABS( T( J, J ) )
         IF( ABSB.GT.SAFMIN ) THEN
            SIGNBC = CONJG( T( J, J ) / ABSB )
            T( J, J ) = ABSB
            IF( ILSCHR ) THEN
```

```
               CALL CSCAL( J-1, SIGNBC, T( 1, J ), 1 )
               CALL CSCAL( J, SIGNBC, H( 1, J ), 1 )
            ELSE
               CALL CSCAL( 1, SIGNBC, H( J, J ), 1 )
            END IF
            IF( ILZ )
     $         CALL CSCAL( N, SIGNBC, Z( 1, J ), 1 )
         ELSE
            T( J, J ) = CZERO
         END IF
         ALPHA( J ) = H( J, J )
         BETA( J ) = T( J, J )
   10 CONTINUE
*
*     If IHI < ILO, skip QZ steps
*
      IF( IHI.LT.ILO )
     $   GO TO 190
*
*     MAIN QZ ITERATION LOOP
*
*     Initialize dynamic indices
*
*     Eigenvalues ILAST+1:N have been found.
*        Column operations modify rows IFRSTM:whatever
*        Row operations modify columns whatever:ILASTM
*
*     If only eigenvalues are being computed, then
*        IFRSTM is the row of the last splitting row above row ILAST;
*        this is always at least ILO.
*     IITER counts iterations since the last eigenvalue was found,
*        to tell when to use an extraordinary shift.
*     MAXIT is the maximum number of QZ sweeps allowed.
*
      ILAST = IHI
      IF( ILSCHR ) THEN
         IFRSTM = 1
         ILASTM = N
      ELSE
         IFRSTM = ILO
         ILASTM = IHI
      END IF
      IITER = 0
      ESHIFT = CZERO
      MAXIT = 30*( IHI-ILO+1 )
*
      DO 170 JITER = 1, MAXIT
*
*        Check for too many iterations.
*
```

```
         IF( JITER.GT.MAXIT )
     $      GO TO 180
*
*        Split the matrix if possible.
*
*        Two tests:
*           1: H(j,j-1)=0  or  j=ILO
*           2: T(j,j)=0
*
*        Special case: j=ILAST
*
         IF( ILAST.EQ.ILO ) THEN
            GO TO 60
         ELSE
            IF( ABS1( H( ILAST, ILAST-1 ) ).LE.ATOL ) THEN
               H( ILAST, ILAST-1 ) = CZERO
               GO TO 60
            END IF
         END IF
*
         IF( ABS( T( ILAST, ILAST ) ).LE.BTOL ) THEN
            T( ILAST, ILAST ) = CZERO
            GO TO 50
         END IF
*
*        General case: j<ILAST
*
         DO 40 J = ILAST - 1, ILO, -1
*
*           Test 1: for H(j,j-1)=0 or j=ILO
*
            IF( J.EQ.ILO ) THEN
               ILAZRO = .TRUE.
            ELSE
               IF( ABS1( H( J, J-1 ) ).LE.ATOL ) THEN
                  H( J, J-1 ) = CZERO
                  ILAZRO = .TRUE.
               ELSE
                  ILAZRO = .FALSE.
               END IF
            END IF
*
*           Test 2: for T(j,j)=0
*
            IF( ABS( T( J, J ) ).LT.BTOL ) THEN
               T( J, J ) = CZERO
*
*              Test 1a: Check for 2 consecutive small subdiagonals in A
*
               ILAZR2 = .FALSE.
```

```
               IF( .NOT.ILAZRO ) THEN
                  IF( ABS1( H( J, J-1 ) )*( ASCALE*ABS1( H( J+1,
     $                J ) ) ).LE.ABS1( H( J, J ) )*( ASCALE*ATOL ) )
     $                ILAZR2 = .TRUE.
               END IF
*
*              If both tests pass (1 & 2), i.e., the leading diagonal
*              element of B in the block is zero, split a 1x1 block off
*              at the top. (I.e., at the J-th row/column) The leading
*              diagonal element of the remainder can also be zero, so
*              this may have to be done repeatedly.
*
               IF( ILAZRO .OR. ILAZR2 ) THEN
                  DO 20 JCH = J, ILAST - 1
                     CTEMP = H( JCH, JCH )
                     CALL CLARTG( CTEMP, H( JCH+1, JCH ), C, S,
     $                            H( JCH, JCH ) )
                     H( JCH+1, JCH ) = CZERO
                     CALL CROT( ILASTM-JCH, H( JCH, JCH+1 ), LDH,
     $                          H( JCH+1, JCH+1 ), LDH, C, S )
                     CALL CROT( ILASTM-JCH, T( JCH, JCH+1 ), LDT,
     $                          T( JCH+1, JCH+1 ), LDT, C, S )
                     IF( ILQ )
     $                  CALL CROT( N, Q( 1, JCH ), 1, Q( 1, JCH+1 ), 1,
     $                             C, CONJG( S ) )
                     IF( ILAZR2 )
     $                  H( JCH, JCH-1 ) = H( JCH, JCH-1 )*C
                     ILAZR2 = .FALSE.
                     IF( ABS1( T( JCH+1, JCH+1 ) ).GE.BTOL ) THEN
                        IF( JCH+1.GE.ILAST ) THEN
                           GO TO 60
                        ELSE
                           IFIRST = JCH + 1
                           GO TO 70
                        END IF
                     END IF
                     T( JCH+1, JCH+1 ) = CZERO
   20             CONTINUE
                  GO TO 50
               ELSE
*
*                 Only test 2 passed -- chase the zero to T(ILAST,ILAST)
*                 Then process as in the case T(ILAST,ILAST)=0
*
                  DO 30 JCH = J, ILAST - 1
                     CTEMP = T( JCH, JCH+1 )
                     CALL CLARTG( CTEMP, T( JCH+1, JCH+1 ), C, S,
     $                            T( JCH, JCH+1 ) )
                     T( JCH+1, JCH+1 ) = CZERO
                     IF( JCH.LT.ILASTM-1 )
```

```
     $                 CALL CROT( ILASTM-JCH-1, T( JCH, JCH+2 ), LDT,
     $                            T( JCH+1, JCH+2 ), LDT, C, S )
                    CALL CROT( ILASTM-JCH+2, H( JCH, JCH-1 ), LDH,
     $                         H( JCH+1, JCH-1 ), LDH, C, S )
                    IF( ILQ )
     $                 CALL CROT( N, Q( 1, JCH ), 1, Q( 1, JCH+1 ), 1,
     $                            C, CONJG( S ) )
                    CTEMP = H( JCH+1, JCH )
                    CALL CLARTG( CTEMP, H( JCH+1, JCH-1 ), C, S,
     $                           H( JCH+1, JCH ) )
                    H( JCH+1, JCH-1 ) = CZERO
                    CALL CROT( JCH+1-IFRSTM, H( IFRSTM, JCH ), 1,
     $                         H( IFRSTM, JCH-1 ), 1, C, S )
                    CALL CROT( JCH-IFRSTM, T( IFRSTM, JCH ), 1,
     $                         T( IFRSTM, JCH-1 ), 1, C, S )
                    IF( ILZ )
     $                 CALL CROT( N, Z( 1, JCH ), 1, Z( 1, JCH-1 ), 1,
     $                            C, S )
   30            CONTINUE
                 GO TO 50
              END IF
           ELSE IF( ILAZRO ) THEN
*
*             Only test 1 passed -- work on J:ILAST
*
              IFIRST = J
              GO TO 70
           END IF
*
*          Neither test passed -- try next J
*
   40    CONTINUE
*
*        (Drop-through is "impossible")
*
         INFO = 2*N + 1
         GO TO 210
*
*        T(ILAST,ILAST)=0 -- clear H(ILAST,ILAST-1) to split off a
*        1x1 block.
*
   50    CONTINUE
         CTEMP = H( ILAST, ILAST )
         CALL CLARTG( CTEMP, H( ILAST, ILAST-1 ), C, S,
     $                H( ILAST, ILAST ) )
         H( ILAST, ILAST-1 ) = CZERO
         CALL CROT( ILAST-IFRSTM, H( IFRSTM, ILAST ), 1,
     $              H( IFRSTM, ILAST-1 ), 1, C, S )
         CALL CROT( ILAST-IFRSTM, T( IFRSTM, ILAST ), 1,
     $              T( IFRSTM, ILAST-1 ), 1, C, S )
```

```
         IF( ILZ )
     $      CALL CROT( N, Z( 1, ILAST ), 1, Z( 1, ILAST-1 ), 1, C, S )
*
*        H(ILAST,ILAST-1)=0 -- Standardize B, set ALPHA and BETA
*
   60    CONTINUE
         ABSB = ABS( T( ILAST, ILAST ) )
         IF( ABSB.GT.SAFMIN ) THEN
            SIGNBC = CONJG( T( ILAST, ILAST ) / ABSB )
            T( ILAST, ILAST ) = ABSB
            IF( ILSCHR ) THEN
               CALL CSCAL( ILAST-IFRSTM, SIGNBC, T( IFRSTM, ILAST ), 1 )
               CALL CSCAL( ILAST+1-IFRSTM, SIGNBC, H( IFRSTM, ILAST ),
     $                     1 )
            ELSE
               CALL CSCAL( 1, SIGNBC, H( ILAST, ILAST ), 1 )
            END IF
            IF( ILZ )
     $         CALL CSCAL( N, SIGNBC, Z( 1, ILAST ), 1 )
         ELSE
            T( ILAST, ILAST ) = CZERO
         END IF
         ALPHA( ILAST ) = H( ILAST, ILAST )
         BETA( ILAST ) = T( ILAST, ILAST )
*
*        Go to next block -- exit if finished.
*
         ILAST = ILAST - 1
         IF( ILAST.LT.ILO )
     $      GO TO 190
*
*        Reset counters
*
         IITER = 0
         ESHIFT = CZERO
         IF( .NOT.ILSCHR ) THEN
            ILASTM = ILAST
            IF( IFRSTM.GT.ILAST )
     $         IFRSTM = ILO
         END IF
         GO TO 160
*
*        QZ step
*
*        This iteration only involves rows/columns IFIRST:ILAST.  We
*        assume IFIRST < ILAST, and that the diagonal of B is non-zero.
*
   70    CONTINUE
         IITER = IITER + 1
         IF( .NOT.ILSCHR ) THEN
```

```
            IFRSTM = IFIRST
         END IF
*
*        Compute the Shift.
*
*        At this point, IFIRST < ILAST, and the diagonal elements of
*        T(IFIRST:ILAST,IFIRST,ILAST) are larger than BTOL (in
*        magnitude)
*
         IF( ( IITER / 10 )*10.NE.IITER ) THEN
*
*           The Wilkinson shift (AEP p.512), i.e., the eigenvalue of
*           the bottom-right 2x2 block of A inv(B) which is nearest to
*           the bottom-right element.
*
*           We factor B as U*D, where U has unit diagonals, and
*           compute (A*inv(D))*inv(U).
*
            U12 = ( BSCALE*T( ILAST-1, ILAST ) ) /
     $            ( BSCALE*T( ILAST, ILAST ) )
            AD11 = ( ASCALE*H( ILAST-1, ILAST-1 ) ) /
     $             ( BSCALE*T( ILAST-1, ILAST-1 ) )
            AD21 = ( ASCALE*H( ILAST, ILAST-1 ) ) /
     $             ( BSCALE*T( ILAST-1, ILAST-1 ) )
            AD12 = ( ASCALE*H( ILAST-1, ILAST ) ) /
     $             ( BSCALE*T( ILAST, ILAST ) )
            AD22 = ( ASCALE*H( ILAST, ILAST ) ) /
     $             ( BSCALE*T( ILAST, ILAST ) )
            ABI22 = AD22 - U12*AD21
*
            T1 = HALF*( AD11+ABI22 )
            RTDISC = SQRT( T1**2+AD12*AD21-AD11*AD22 )
            TEMP = REAL( T1-ABI22 )*REAL( RTDISC ) +
     $             AIMAG( T1-ABI22 )*AIMAG( RTDISC )
            IF( TEMP.LE.ZERO ) THEN
               SHIFT = T1 + RTDISC
            ELSE
               SHIFT = T1 - RTDISC
            END IF
         ELSE
*
*           Exceptional shift.  Chosen for no particularly good reason.
*
            ESHIFT = ESHIFT + (ASCALE*H(ILAST,ILAST-1))/
     $                        (BSCALE*T(ILAST-1,ILAST-1))
            SHIFT = ESHIFT
         END IF
*
*        Now check for two consecutive small subdiagonals.
*
```

```
         DO 80 J = ILAST - 1, IFIRST + 1, -1
            ISTART = J
            CTEMP = ASCALE*H( J, J ) - SHIFT*( BSCALE*T( J, J ) )
            TEMP = ABS1( CTEMP )
            TEMP2 = ASCALE*ABS1( H( J+1, J ) )
            TEMPR = MAX( TEMP, TEMP2 )
            IF( TEMPR.LT.ONE .AND. TEMPR.NE.ZERO ) THEN
               TEMP = TEMP / TEMPR
               TEMP2 = TEMP2 / TEMPR
            END IF
            IF( ABS1( H( J, J-1 ) )*TEMP2.LE.TEMP*ATOL )
     $         GO TO 90
   80    CONTINUE
*
         ISTART = IFIRST
         CTEMP = ASCALE*H( IFIRST, IFIRST ) -
     $           SHIFT*( BSCALE*T( IFIRST, IFIRST ) )
   90    CONTINUE
*
*        Do an implicit-shift QZ sweep.
*
*        Initial Q
*
         CTEMP2 = ASCALE*H( ISTART+1, ISTART )
         CALL CLARTG( CTEMP, CTEMP2, C, S, CTEMP3 )
*
*        Sweep
*
         DO 150 J = ISTART, ILAST - 1
            IF( J.GT.ISTART ) THEN
               CTEMP = H( J, J-1 )
               CALL CLARTG( CTEMP, H( J+1, J-1 ), C, S, H( J, J-1 ) )
               H( J+1, J-1 ) = CZERO
            END IF
*
            DO 100 JC = J, ILASTM
               CTEMP = C*H( J, JC ) + S*H( J+1, JC )
               H( J+1, JC ) = -CONJG( S )*H( J, JC ) + C*H( J+1, JC )
               H( J, JC ) = CTEMP
               CTEMP2 = C*T( J, JC ) + S*T( J+1, JC )
               T( J+1, JC ) = -CONJG( S )*T( J, JC ) + C*T( J+1, JC )
               T( J, JC ) = CTEMP2
  100       CONTINUE
            IF( ILQ ) THEN
               DO 110 JR = 1, N
                  CTEMP = C*Q( JR, J ) + CONJG( S )*Q( JR, J+1 )
                  Q( JR, J+1 ) = -S*Q( JR, J ) + C*Q( JR, J+1 )
                  Q( JR, J ) = CTEMP
  110          CONTINUE
            END IF
```

```
*
            CTEMP = T( J+1, J+1 )
            CALL CLARTG( CTEMP, T( J+1, J ), C, S, T( J+1, J+1 ) )
            T( J+1, J ) = CZERO
*
            DO 120 JR = IFRSTM, MIN( J+2, ILAST )
               CTEMP = C*H( JR, J+1 ) + S*H( JR, J )
               H( JR, J ) = -CONJG( S )*H( JR, J+1 ) + C*H( JR, J )
               H( JR, J+1 ) = CTEMP
  120       CONTINUE
            DO 130 JR = IFRSTM, J
               CTEMP = C*T( JR, J+1 ) + S*T( JR, J )
               T( JR, J ) = -CONJG( S )*T( JR, J+1 ) + C*T( JR, J )
               T( JR, J+1 ) = CTEMP
  130       CONTINUE
            IF( ILZ ) THEN
               DO 140 JR = 1, N
                  CTEMP = C*Z( JR, J+1 ) + S*Z( JR, J )
                  Z( JR, J ) = -CONJG( S )*Z( JR, J+1 ) + C*Z( JR, J )
                  Z( JR, J+1 ) = CTEMP
  140          CONTINUE
            END IF
  150    CONTINUE
*
  160    CONTINUE
*
  170 CONTINUE
*
*     Drop-through = non-convergence
*
  180 CONTINUE
      INFO = ILAST
      GO TO 210
*
*     Successful completion of all QZ steps
*
  190 CONTINUE
*
*     Set Eigenvalues 1:ILO-1
*
      DO 200 J = 1, ILO - 1
         ABSB = ABS( T( J, J ) )
         IF( ABSB.GT.SAFMIN ) THEN
            SIGNBC = CONJG( T( J, J ) / ABSB )
            T( J, J ) = ABSB
            IF( ILSCHR ) THEN
               CALL CSCAL( J-1, SIGNBC, T( 1, J ), 1 )
               CALL CSCAL( J, SIGNBC, H( 1, J ), 1 )
            ELSE
               CALL CSCAL( 1, SIGNBC, H( J, J ), 1 )
```

```
            END IF
            IF( ILZ )
     $         CALL CSCAL( N, SIGNBC, Z( 1, J ), 1 )
         ELSE
            T( J, J ) = CZERO
         END IF
         ALPHA( J ) = H( J, J )
         BETA( J ) = T( J, J )
  200 CONTINUE
*
*     Normal Termination
*
      INFO = 0
*
*     Exit (other than argument error) -- return optimal workspace size
*
  210 CONTINUE
      WORK( 1 ) = CMPLX( N )
      RETURN
*
*     End of CHGEQZ
*
      END
```

———-

34.34.4 lisp code

— chgeqz.lisp —

```
(let* ((czero (f2cl-lib:cmplx 0.0 0.0))
       (cone (f2cl-lib:cmplx 1.0 0.0))
       (zero 0.0)
       (one 1.0)
       (half 0.5))
  (declare (type (f2cl-lib:complex8) czero)
           (type (f2cl-lib:complex8) cone)
           (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 0.5 0.5) half)
           (ignorable czero cone zero one half))
  (defun chgeqz
         (job compq compz n ilo ihi h ldh t$ ldt alpha beta q ldq z ldz work
          lwork rwork info)
    (declare (type (array single-float (*)) rwork)
             (type (array f2cl-lib:complex8 (*)) work z q beta alpha t$ h)
             (type (f2cl-lib:integer4) info lwork ldz ldq ldt ldh ihi ilo n)
             (type (string *) compz compq job))
```

```
(f2cl-lib:with-multi-array-data
    ((job character job-%data% job-%offset%)
     (compq character compq-%data% compq-%offset%)
     (compz character compz-%data% compz-%offset%)
     (h f2cl-lib:complex8 h-%data% h-%offset%)
     (t$ f2cl-lib:complex8 t$-%data% t$-%offset%)
     (alpha f2cl-lib:complex8 alpha-%data% alpha-%offset%)
     (beta f2cl-lib:complex8 beta-%data% beta-%offset%)
     (q f2cl-lib:complex8 q-%data% q-%offset%)
     (z f2cl-lib:complex8 z-%data% z-%offset%)
     (work f2cl-lib:complex8 work-%data% work-%offset%)
     (rwork single-float rwork-%data% rwork-%offset%))
  (labels ((abs1 (x)
             (+ (abs (f2cl-lib:freal x)) (abs (f2cl-lib:aimag x)))))
    (declare
     (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
      abs1))
    (prog ((abi22 #C(0.0 0.0)) (ad11 #C(0.0 0.0)) (ad12 #C(0.0 0.0))
           (ad21 #C(0.0 0.0)) (ad22 #C(0.0 0.0)) (ctemp #C(0.0 0.0))
           (ctemp2 #C(0.0 0.0)) (ctemp3 #C(0.0 0.0)) (eshift #C(0.0 0.0))
           (rtdisc #C(0.0 0.0)) (s #C(0.0 0.0)) (shift #C(0.0 0.0))
           (signbc #C(0.0 0.0)) (t1 #C(0.0 0.0)) (u12 #C(0.0 0.0))
           (x #C(0.0 0.0)) (absb 0.0) (anorm 0.0) (ascale 0.0) (atol 0.0)
           (bnorm 0.0) (bscale 0.0) (btol 0.0) (c 0.0) (safmin 0.0)
           (temp 0.0) (temp2 0.0) (tempr 0.0) (ulp 0.0) (icompq 0)
           (icompz 0) (ifirst 0) (ifrstm 0) (iiter 0) (ilast 0) (ilastm 0)
           (in 0) (ischur 0) (istart 0) (j 0) (jc 0) (jch 0) (jiter 0)
           (jr 0) (maxit 0) (ilazr2 nil) (ilazro nil) (ilq nil)
           (ilschr nil) (ilz nil) (lquery nil))
      (declare
       (type (f2cl-lib:complex8) abi22 ad11 ad12 ad21 ad22 ctemp ctemp2
        ctemp3 eshift rtdisc s shift signbc t1 u12 x)
       (type (single-float) absb anorm ascale atol bnorm bscale btol c
        safmin temp temp2 tempr ulp)
       (type (f2cl-lib:integer4) icompq icompz ifirst ifrstm iiter ilast
        ilastm in ischur istart j jc jch jiter jr maxit)
       (type f2cl-lib:logical ilazr2 ilazro ilq ilschr ilz lquery))
      (cond
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame job "E")
          (declare (ignore var-1))
          (when var-0 (setf job var-0))
          ret-val)
        (setf ilschr f2cl-lib:%false%) (setf ischur 1))
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame job "S")
          (declare (ignore var-1))
          (when var-0 (setf job var-0))
          ret-val)
        (setf ilschr f2cl-lib:%true%) (setf ischur 2))
```

```
 (t (setf ischur 0)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compq "N")
    (declare (ignore var-1))
    (when var-0 (setf compq var-0))
    ret-val)
  (setf ilq f2cl-lib:%false%) (setf icompq 1))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compq "V")
    (declare (ignore var-1))
    (when var-0 (setf compq var-0))
    ret-val)
  (setf ilq f2cl-lib:%true%) (setf icompq 2))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compq "I")
    (declare (ignore var-1))
    (when var-0 (setf compq var-0))
    ret-val)
  (setf ilq f2cl-lib:%true%) (setf icompq 3))
 (t (setf icompq 0)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "N")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf ilz f2cl-lib:%false%) (setf icompz 1))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "V")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf ilz f2cl-lib:%true%) (setf icompz 2))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "I")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf ilz f2cl-lib:%true%) (setf icompz 3))
 (t (setf icompz 0)))
(setf info 0)
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce
         (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n))
         'f2cl-lib:complex8))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((= ischur 0) (setf info -1)) ((= icompq 0) (setf info -2))
      ((= icompz 0) (setf info -3)) ((< n 0) (setf info -4))
      ((< ilo 1) (setf info -5))
```

```
      ((or (> ihi n)
           (< ihi (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))))
       (setf info -6))
      ((< ldh n) (setf info -8)) ((< ldt n) (setf info -10))
      ((or (< ldq 1) (and ilq (< ldq n))) (setf info -14))
      ((or (< ldz 1) (and ilz (< ldz n))) (setf info -16))
      ((and
        (< lwork
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (not lquery))
       (setf info -18)))
(cond
 ((/= info 0) (xerbla "CHGEQZ" (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(cond
 ((<= n 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:cmplx 1))
  (go end_label)))
(if (= icompq 3)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "Full" n n czero cone q ldq)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldq var-6))))
(if (= icompz 3)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "Full" n n czero cone z ldz)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldz var-6))))
(setf in (f2cl-lib:int-sub (f2cl-lib:int-add ihi 1) ilo))
(setf safmin (slamch "S"))
(setf ulp (* (slamch "E") (slamch "B")))
(setf anorm
        (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
            (clanhs "F" in
             (f2cl-lib:array-slice h-%data% f2cl-lib:complex8
                                   (ilo ilo) ((1 ldh) (1 *))
                                   h-%offset%)
             ldh rwork)
          (declare (ignore var-0 var-2 var-4))
          (when var-1 (setf in var-1))
```

```
          (when var-3 (setf ldh var-3))
          ret-val))
(setf bnorm
        (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
            (clanhs "F" in
             (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8
                                   (ilo ilo) ((1 ldt) (1 *))
                                   t$-%offset%)
             ldt rwork)
          (declare (ignore var-0 var-2 var-4))
          (when var-1 (setf in var-1))
          (when var-3 (setf ldt var-3))
          ret-val))
(setf atol (max safmin (* ulp anorm)))
(setf btol (max safmin (* ulp bnorm)))
(setf ascale (/ one (max safmin anorm)))
(setf bscale (/ one (max safmin bnorm)))
(f2cl-lib:fdo (j (f2cl-lib:int-add ihi 1) (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf absb
            (abs
             (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                            t$-%offset%)))
    (cond
     ((> absb safmin)
      (setf signbc
              (f2cl-lib:conjg
               (/
                (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                               t$-%offset%)
                absb)))
      (setf (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                           t$-%offset%)
              (coerce absb 'f2cl-lib:complex8))
      (cond
       (ilschr
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (cscal (f2cl-lib:int-sub j 1) signbc
             (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8 (1 j)
                                   ((1 ldt) (1 *)) t$-%offset%)
             1)
          (declare (ignore var-0 var-2 var-3))
          (when var-1 (setf signbc var-1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (cscal j signbc
             (f2cl-lib:array-slice h-%data% f2cl-lib:complex8 (1 j)
                                   ((1 ldh) (1 *)) h-%offset%)
             1)
          (declare (ignore var-2 var-3))
```

```
            (when var-0 (setf j var-0))
            (when var-1 (setf signbc var-1))))
         (t
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (cscal 1 signbc
               (f2cl-lib:array-slice h-%data% f2cl-lib:complex8 (j j)
                                     ((1 ldh) (1 *)) h-%offset%)
               1)
            (declare (ignore var-0 var-2 var-3))
            (when var-1 (setf signbc var-1)))))
       (if ilz
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (cscal n signbc
                 (f2cl-lib:array-slice z-%data% f2cl-lib:complex8 (1 j)
                                       ((1 ldz) (1 *)) z-%offset%)
                 1)
              (declare (ignore var-2 var-3))
              (when var-0 (setf n var-0))
              (when var-1 (setf signbc var-1)))))
      (t
       (setf (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                            t$-%offset%)
             czero)))
    (setf (f2cl-lib:fref alpha-%data% (j) ((1 *)) alpha-%offset%)
            (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *))
                            h-%offset%))
    (setf (f2cl-lib:fref beta-%data% (j) ((1 *)) beta-%offset%)
            (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                            t$-%offset%))
   label10))
(if (< ihi ilo)
    (go label190))
(setf ilast ihi)
(cond (ilschr (setf ifrstm 1) (setf ilastm n))
      (t (setf ifrstm ilo) (setf ilastm ihi)))
(setf iiter 0)
(setf eshift czero)
(setf maxit
        (f2cl-lib:int-mul 30
                          (f2cl-lib:int-add
                           (f2cl-lib:int-sub ihi ilo) 1)))
(f2cl-lib:fdo (jiter 1 (f2cl-lib:int-add jiter 1))
              ((> jiter maxit) nil)
  (tagbody
    (if (> jiter maxit)
        (go label180))
    (cond ((= ilast ilo) (go label60))
          (t
           (cond
            ((<=
```

```
          (abs1
           (f2cl-lib:fref h
                          (ilast
                           (f2cl-lib:int-add ilast
                                             (f2cl-lib:int-sub
                                              1)))
                          ((1 ldh) (1 *))))
          atol)
         (setf (f2cl-lib:fref h-%data%
                              (ilast (f2cl-lib:int-sub ilast 1))
                              ((1 ldh) (1 *)) h-%offset%)
                czero)
         (go label60)))))
(cond
 ((<= (abs (f2cl-lib:fref t$ (ilast ilast) ((1 ldt) (1 *))))
      btol)
  (setf (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                       t$-%offset%)
          czero)
  (go label50)))
(f2cl-lib:fdo (j (f2cl-lib:int-add ilast (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j ilo) nil)
  (tagbody
    (cond ((= j ilo) (setf ilazro f2cl-lib:%true%))
          (t
           (cond
            ((<=
              (abs1
               (f2cl-lib:fref h
                              (j
                               (f2cl-lib:int-add j
                                                 (f2cl-lib:int-sub
                                                  1)))
                              ((1 ldh) (1 *))))
              atol)
             (setf (f2cl-lib:fref h-%data%
                                  (j (f2cl-lib:int-sub j 1))
                                  ((1 ldh) (1 *)) h-%offset%)
                   czero)
             (setf ilazro f2cl-lib:%true%))
            (t (setf ilazro f2cl-lib:%false%)))))
    (cond
     ((< (abs (f2cl-lib:fref t$ (j j) ((1 ldt) (1 *)))) btol)
      (setf (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                           t$-%offset%)
            czero)
      (setf ilazr2 f2cl-lib:%false%)
      (cond
       ((not ilazro)
```

```
  (if (<=
       (*
        (abs1
         (f2cl-lib:fref h-%data% (j (f2cl-lib:int-sub j 1))
                        ((1 ldh) (1 *)) h-%offset%))
        (* ascale
           (abs1
            (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-add j 1) j)
                           ((1 ldh) (1 *)) h-%offset%))))
       (*
        (abs1
         (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *))
                        h-%offset%))
        (* ascale atol)))
      (setf ilazr2 f2cl-lib:%true%))))
(cond
 ((or ilazro ilazr2)
  (f2cl-lib:fdo (jch j (f2cl-lib:int-add jch 1))
                ((> jch
                    (f2cl-lib:int-add ilast
                                      (f2cl-lib:int-sub
                                       1)))
                 nil)
    (tagbody
      (setf ctemp
              (f2cl-lib:fref h-%data% (jch jch)
                             ((1 ldh) (1 *)) h-%offset%))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (clartg ctemp
           (f2cl-lib:array-slice h-%data% f2cl-lib:complex8
                                 ((+ jch 1) jch)
                                 ((1 ldh) (1 *))
                                 h-%offset%)
           c s
           (f2cl-lib:array-slice h-%data% f2cl-lib:complex8
                                 (jch jch) ((1 ldh) (1 *))
                                 h-%offset%))
        (declare (ignore var-1 var-4))
        (when var-0 (setf ctemp var-0))
        (when var-2 (setf c var-2))
        (when var-3 (setf s var-3)))
      (setf (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-add jch 1) jch)
                           ((1 ldh) (1 *)) h-%offset%)
              czero)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (crot (f2cl-lib:int-sub ilastm jch)
           (f2cl-lib:array-slice h-%data% f2cl-lib:complex8
```

```
                                (jch
                                 (f2cl-lib:int-add jch 1))
                                ((1 ldh) (1 *))
                                h-%offset%)
      ldh
      (f2cl-lib:array-slice h-%data% f2cl-lib:complex8
                                ((+ jch 1)
                                 (f2cl-lib:int-add jch 1))
                                ((1 ldh) (1 *))
                                h-%offset%)
      ldh c s)
  (declare (ignore var-0 var-1 var-3))
  (when var-2 (setf ldh var-2))
  (when var-4 (setf ldh var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (crot (f2cl-lib:int-sub ilastm jch)
     (f2cl-lib:array-slice t$-%data%
                                f2cl-lib:complex8
                                (jch
                                 (f2cl-lib:int-add jch 1))
                                ((1 ldt) (1 *))
                                t$-%offset%)
      ldt
      (f2cl-lib:array-slice t$-%data%
                                f2cl-lib:complex8
                                ((+ jch 1)
                                 (f2cl-lib:int-add jch 1))
                                ((1 ldt) (1 *))
                                t$-%offset%)
      ldt c s)
  (declare (ignore var-0 var-1 var-3))
  (when var-2 (setf ldt var-2))
  (when var-4 (setf ldt var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(if ilq
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (crot n
         (f2cl-lib:array-slice q-%data%
                                   f2cl-lib:complex8
                                   (1 jch)
                                   ((1 ldq) (1 *))
                                   q-%offset%)
         1
         (f2cl-lib:array-slice q-%data%
                                   f2cl-lib:complex8
```

```
                                        (1
                                         (f2cl-lib:int-add jch
                                                           1))
                                        ((1 ldq) (1 *))
                                        q-%offset%)
                  1 c (f2cl-lib:conj s))
              (declare
               (ignore var-1 var-2 var-3 var-4 var-6))
              (when var-0 (setf n var-0))
              (when var-5 (setf c var-5))))
        (if ilazr2
            (setf (f2cl-lib:fref h-%data%
                                 (jch
                                  (f2cl-lib:int-sub jch 1))
                                 ((1 ldh) (1 *)) h-%offset%)
                  (*
                   (f2cl-lib:fref h-%data%
                                  (jch
                                   (f2cl-lib:int-sub jch
                                                     1))
                                  ((1 ldh) (1 *))
                                  h-%offset%)
                   c)))
        (setf ilazr2 f2cl-lib:%false%)
        (cond
         ((>=
           (abs1
            (f2cl-lib:fref t$
                           ((f2cl-lib:int-add jch 1)
                            (f2cl-lib:int-add jch 1))
                           ((1 ldt) (1 *))))
           btol)
          (cond
           ((>= (f2cl-lib:int-add jch 1) ilast) (go label60))
           (t (setf ifirst (f2cl-lib:int-add jch 1))
            (go label70)))))
        (setf (f2cl-lib:fref t$-%data%
                             ((f2cl-lib:int-add jch 1)
                              (f2cl-lib:int-add jch 1))
                             ((1 ldt) (1 *)) t$-%offset%)
              czero)
       label20))
    (go label50))
   (t
    (f2cl-lib:fdo (jch j (f2cl-lib:int-add jch 1))
                  ((> jch
                      (f2cl-lib:int-add ilast
                                        (f2cl-lib:int-sub
                                         1)))
                   nil)
```

```
(tagbody
  (setf ctemp
          (f2cl-lib:fref t$-%data%
                         (jch (f2cl-lib:int-add jch 1))
                         ((1 ldt) (1 *)) t$-%offset%))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (clartg ctemp
       (f2cl-lib:array-slice t$-%data%
                             f2cl-lib:complex8
                             ((+ jch 1)
                              (f2cl-lib:int-add jch 1))
                             ((1 ldt) (1 *))
                             t$-%offset%)
       c s
       (f2cl-lib:array-slice t$-%data%
                             f2cl-lib:complex8
                             (jch
                              (f2cl-lib:int-add jch 1))
                             ((1 ldt) (1 *))
                             t$-%offset%))
    (declare (ignore var-1 var-4))
    (when var-0 (setf ctemp var-0))
    (when var-2 (setf c var-2))
    (when var-3 (setf s var-3)))
  (setf (f2cl-lib:fref t$-%data%
                       ((f2cl-lib:int-add jch 1)
                        (f2cl-lib:int-add jch 1))
                       ((1 ldt) (1 *)) t$-%offset%)
          czero)
  (if (< jch (f2cl-lib:int-sub ilastm 1))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (crot (f2cl-lib:int-sub ilastm jch 1)
           (f2cl-lib:array-slice t$-%data%
                                 f2cl-lib:complex8
                                 (jch
                                  (f2cl-lib:int-add jch
                                                    2))
                                 ((1 ldt) (1 *))
                                 t$-%offset%)
           ldt
           (f2cl-lib:array-slice t$-%data%
                                 f2cl-lib:complex8
                                 ((+ jch 1)
                                  (f2cl-lib:int-add jch
                                                    2))
                                 ((1 ldt) (1 *))
                                 t$-%offset%)
           ldt c s)
        (declare (ignore var-0 var-1 var-3))
```

```
      (when var-2 (setf ldt var-2))
      (when var-4 (setf ldt var-4))
      (when var-5 (setf c var-5))
      (when var-6 (setf s var-6))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (crot
     (f2cl-lib:int-add (f2cl-lib:int-sub ilastm jch)
                       2)
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex8
                           (jch
                            (f2cl-lib:int-sub jch 1))
                           ((1 ldh) (1 *))
                           h-%offset%)
     ldh
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex8
                           ((+ jch 1)
                            (f2cl-lib:int-sub jch 1))
                           ((1 ldh) (1 *))
                           h-%offset%)
     ldh c s)
  (declare (ignore var-0 var-1 var-3))
  (when var-2 (setf ldh var-2))
  (when var-4 (setf ldh var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(if ilq
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (crot n
         (f2cl-lib:array-slice q-%data%
                               f2cl-lib:complex8
                               (1 jch)
                               ((1 ldq) (1 *))
                               q-%offset%)
         1
         (f2cl-lib:array-slice q-%data%
                               f2cl-lib:complex8
                               (1
                                (f2cl-lib:int-add jch
                                                  1))
                               ((1 ldq) (1 *))
                               q-%offset%)
         1 c (f2cl-lib:conjg s))
      (declare
       (ignore var-1 var-2 var-3 var-4 var-6))
      (when var-0 (setf n var-0))
      (when var-5 (setf c var-5))))
(setf ctemp
        (f2cl-lib:fref h-%data%
```

```
                        ((f2cl-lib:int-add jch 1) jch)
                        ((1 ldh) (1 *)) h-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (clartg ctemp
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex8
                           ((+ jch 1)
                            (f2cl-lib:int-sub jch 1))
                           ((1 ldh) (1 *))
                           h-%offset%)
     c s
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex8
                           ((+ jch 1) jch)
                           ((1 ldh) (1 *))
                           h-%offset%))
  (declare (ignore var-1 var-4))
  (when var-0 (setf ctemp var-0))
  (when var-2 (setf c var-2))
  (when var-3 (setf s var-3)))
(setf (f2cl-lib:fref h-%data%
                     ((f2cl-lib:int-add jch 1)
                      (f2cl-lib:int-sub jch 1))
                     ((1 ldh) (1 *)) h-%offset%)
        czero)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (crot
     (f2cl-lib:int-sub (f2cl-lib:int-add jch 1)
                       ifrstm)
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex8
                           (ifrstm jch)
                           ((1 ldh) (1 *))
                           h-%offset%)
     1
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex8
                           (ifrstm
                            (f2cl-lib:int-sub jch 1))
                           ((1 ldh) (1 *))
                           h-%offset%)
     1 c s)
  (declare (ignore var-0 var-1 var-2 var-3 var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (crot (f2cl-lib:int-sub jch ifrstm)
     (f2cl-lib:array-slice t$-%data%
                           f2cl-lib:complex8
                           (ifrstm jch)
                           ((1 ldt) (1 *))
                           t$-%offset%)
```

```
                    1
                    (f2cl-lib:array-slice t$-%data%
                                          f2cl-lib:complex8
                                          (ifrstm
                                           (f2cl-lib:int-sub jch 1))
                                          ((1 ldt) (1 *))
                                          t$-%offset%)
                    1 c s)
                (declare (ignore var-0 var-1 var-2 var-3 var-4))
                (when var-5 (setf c var-5))
                (when var-6 (setf s var-6)))
            (if ilz
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (crot n
                     (f2cl-lib:array-slice z-%data%
                                           f2cl-lib:complex8
                                           (1 jch)
                                           ((1 ldz) (1 *))
                                           z-%offset%)
                     1
                     (f2cl-lib:array-slice z-%data%
                                           f2cl-lib:complex8
                                           (1
                                            (f2cl-lib:int-sub jch
                                                              1))
                                           ((1 ldz) (1 *))
                                           z-%offset%)
                     1 c s)
                  (declare (ignore var-1 var-2 var-3 var-4))
                  (when var-0 (setf n var-0))
                  (when var-5 (setf c var-5))
                  (when var-6 (setf s var-6))))
          label30))
        (go label50))))
    (ilazro (setf ifirst j) (go label70)))
  label40))
 (setf info (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) 1))
 (go label210)
label50
 (setf ctemp
        (f2cl-lib:fref h-%data% (ilast ilast) ((1 ldh) (1 *))
                       h-%offset%))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (clartg ctemp
      (f2cl-lib:array-slice h-%data% f2cl-lib:complex8
                            (ilast (f2cl-lib:int-sub ilast 1))
                            ((1 ldh) (1 *)) h-%offset%)
      c s
      (f2cl-lib:array-slice h-%data% f2cl-lib:complex8
```

```
                            (ilast ilast) ((1 ldh) (1 *))
                            h-%offset%))
  (declare (ignore var-1 var-4))
  (when var-0 (setf ctemp var-0))
  (when var-2 (setf c var-2))
  (when var-3 (setf s var-3)))
(setf (f2cl-lib:fref h-%data% (ilast (f2cl-lib:int-sub ilast 1))
                      ((1 ldh) (1 *)) h-%offset%)
        czero)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (crot (f2cl-lib:int-sub ilast ifrstm)
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex8
                            (ifrstm ilast) ((1 ldh) (1 *))
                            h-%offset%)
     1
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex8
                            (ifrstm (f2cl-lib:int-sub ilast 1))
                            ((1 ldh) (1 *)) h-%offset%)
     1 c s)
  (declare (ignore var-0 var-1 var-2 var-3 var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (crot (f2cl-lib:int-sub ilast ifrstm)
     (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8
                            (ifrstm ilast) ((1 ldt) (1 *))
                            t$-%offset%)
     1
     (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8
                            (ifrstm (f2cl-lib:int-sub ilast 1))
                            ((1 ldt) (1 *)) t$-%offset%)
     1 c s)
  (declare (ignore var-0 var-1 var-2 var-3 var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(if ilz
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (crot n
         (f2cl-lib:array-slice z-%data% f2cl-lib:complex8
                                (1 ilast) ((1 ldz) (1 *))
                                z-%offset%)
         1
         (f2cl-lib:array-slice z-%data% f2cl-lib:complex8
                                (1 (f2cl-lib:int-sub ilast 1))
                                ((1 ldz) (1 *)) z-%offset%)
         1 c s)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0))
      (when var-5 (setf c var-5))
```

```
        (when var-6 (setf s var-6))))
label60
 (setf absb
         (abs
          (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                         t$-%offset%)))
 (cond
  ((> absb safmin)
   (setf signbc
           (f2cl-lib:conjg
            (/
             (f2cl-lib:fref t$-%data% (ilast ilast)
                            ((1 ldt) (1 *)) t$-%offset%)
             absb)))
   (setf (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                        t$-%offset%)
           (coerce absb 'f2cl-lib:complex8))
   (cond
    (ilschr
     (multiple-value-bind (var-0 var-1 var-2 var-3)
         (cscal (f2cl-lib:int-sub ilast ifrstm) signbc
          (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8
                                (ifrstm ilast) ((1 ldt) (1 *))
                                t$-%offset%)
          1)
       (declare (ignore var-0 var-2 var-3))
       (when var-1 (setf signbc var-1)))
     (multiple-value-bind (var-0 var-1 var-2 var-3)
         (cscal
          (f2cl-lib:int-sub (f2cl-lib:int-add ilast 1) ifrstm)
          signbc
          (f2cl-lib:array-slice h-%data% f2cl-lib:complex8
                                (ifrstm ilast) ((1 ldh) (1 *))
                                h-%offset%)
          1)
       (declare (ignore var-0 var-2 var-3))
       (when var-1 (setf signbc var-1))))
    (t
     (multiple-value-bind (var-0 var-1 var-2 var-3)
         (cscal 1 signbc
          (f2cl-lib:array-slice h-%data% f2cl-lib:complex8
                                (ilast ilast) ((1 ldh) (1 *))
                                h-%offset%)
          1)
       (declare (ignore var-0 var-2 var-3))
       (when var-1 (setf signbc var-1)))))
   (if ilz
       (multiple-value-bind (var-0 var-1 var-2 var-3)
           (cscal n signbc
            (f2cl-lib:array-slice z-%data% f2cl-lib:complex8
```

```
                                        (1 ilast) ((1 ldz) (1 *))
                                        z-%offset%)
                  1)
                (declare (ignore var-2 var-3))
                (when var-0 (setf n var-0))
                (when var-1 (setf signbc var-1)))))
   (t
    (setf (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                         t$-%offset%)
              czero)))
  (setf (f2cl-lib:fref alpha-%data% (ilast) ((1 *)) alpha-%offset%)
                (f2cl-lib:fref h-%data% (ilast ilast) ((1 ldh) (1 *))
                         h-%offset%))
  (setf (f2cl-lib:fref beta-%data% (ilast) ((1 *)) beta-%offset%)
                (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                         t$-%offset%))
  (setf ilast (f2cl-lib:int-sub ilast 1))
  (if (< ilast ilo)
      (go label190))
  (setf iiter 0)
  (setf eshift czero)
  (cond
   ((not ilschr) (setf ilastm ilast)
    (if (> ifrstm ilast)
        (setf ifrstm ilo))))
  (go label160)
 label70
  (setf iiter (f2cl-lib:int-add iiter 1))
  (cond ((not ilschr) (setf ifrstm ifirst)))
  (cond
   ((/= (f2cl-lib:int-mul (f2cl-lib:f2cl/ iiter 10) 10) iiter)
    (setf u12
            (/
             (* bscale
                (f2cl-lib:fref t$-%data%
                               ((f2cl-lib:int-sub ilast 1) ilast)
                               ((1 ldt) (1 *)) t$-%offset%))
             (* bscale
                (f2cl-lib:fref t$-%data% (ilast ilast)
                               ((1 ldt) (1 *)) t$-%offset%))))
    (setf ad11
            (/
             (* ascale
                (f2cl-lib:fref h-%data%
                               ((f2cl-lib:int-sub ilast 1)
                                (f2cl-lib:int-sub ilast 1))
                               ((1 ldh) (1 *)) h-%offset%))
             (* bscale
                (f2cl-lib:fref t$-%data%
                               ((f2cl-lib:int-sub ilast 1)
```

```
                              (f2cl-lib:int-sub ilast 1))
                             ((1 ldt) (1 *)) t$-%offset%))))
 (setf ad21
        (/
         (* ascale
            (f2cl-lib:fref h-%data%
                            (ilast (f2cl-lib:int-sub ilast 1))
                            ((1 ldh) (1 *)) h-%offset%))
         (* bscale
            (f2cl-lib:fref t$-%data%
                            ((f2cl-lib:int-sub ilast 1)
                             (f2cl-lib:int-sub ilast 1))
                            ((1 ldt) (1 *)) t$-%offset%))))
 (setf ad12
        (/
         (* ascale
            (f2cl-lib:fref h-%data%
                            ((f2cl-lib:int-sub ilast 1) ilast)
                            ((1 ldh) (1 *)) h-%offset%))
         (* bscale
            (f2cl-lib:fref t$-%data% (ilast ilast)
                            ((1 ldt) (1 *)) t$-%offset%))))
 (setf ad22
        (/
         (* ascale
            (f2cl-lib:fref h-%data% (ilast ilast)
                            ((1 ldh) (1 *)) h-%offset%))
         (* bscale
            (f2cl-lib:fref t$-%data% (ilast ilast)
                            ((1 ldt) (1 *)) t$-%offset%))))
 (setf abi22 (- ad22 (* u12 ad21)))
 (setf t1 (* half (+ ad11 abi22)))
 (setf rtdisc
        (f2cl-lib:fsqrt
         (- (+ (expt t1 2) (* ad12 ad21)) (* ad11 ad22))))
 (setf temp
        (+
         (* (f2cl-lib:freal (- t1 abi22))
            (f2cl-lib:freal rtdisc))
         (* (f2cl-lib:aimag (- t1 abi22))
            (f2cl-lib:aimag rtdisc))))
 (cond ((<= temp zero) (setf shift (+ t1 rtdisc)))
       (t (setf shift (- t1 rtdisc)))))
(t
 (setf eshift
        (+ eshift
           (/
            (* ascale
               (f2cl-lib:fref h-%data%
                              (ilast
```

```
                                    (f2cl-lib:int-sub ilast 1))
                                   ((1 ldh) (1 *)) h-%offset%))
                 (* bscale
                    (f2cl-lib:fref t$-%data%
                                   ((f2cl-lib:int-sub ilast 1)
                                    (f2cl-lib:int-sub ilast 1))
                                   ((1 ldt) (1 *)) t$-%offset%)))))
  (setf shift eshift)))
(f2cl-lib:fdo (j (f2cl-lib:int-add ilast (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j (f2cl-lib:int-add ifirst 1)) nil)
  (tagbody
    (setf istart j)
    (setf ctemp
            (-
             (* ascale
                (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *))
                               h-%offset%))
             (* shift
                (* bscale
                   (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                                  t$-%offset%)))))
    (setf temp (abs1 ctemp))
    (setf temp2
            (* ascale
               (abs1
                (f2cl-lib:fref h-%data%
                               ((f2cl-lib:int-add j 1) j)
                               ((1 ldh) (1 *)) h-%offset%))))
    (setf tempr (max temp temp2))
    (cond
     ((and (< tempr one) (/= tempr zero))
      (setf temp (/ temp tempr)) (setf temp2 (/ temp2 tempr))))
    (if (<=
         (*
          (abs1
           (f2cl-lib:fref h-%data% (j (f2cl-lib:int-sub j 1))
                          ((1 ldh) (1 *)) h-%offset%))
          temp2)
         (* temp atol))
        (go label90))
   label80))
(setf istart ifirst)
(setf ctemp
        (-
         (* ascale
            (f2cl-lib:fref h-%data% (ifirst ifirst)
                           ((1 ldh) (1 *)) h-%offset%))
         (* shift
            (* bscale
```

```
                (f2cl-lib:fref t$-%data% (ifirst ifirst)
                               ((1 ldt) (1 *)) t$-%offset%)))))
label90
 (setf ctemp2
         (* ascale
            (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-add istart 1) istart)
                           ((1 ldh) (1 *)) h-%offset%)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (clartg ctemp ctemp2 c s ctemp3)
   (declare (ignore))
   (when var-0 (setf ctemp var-0))
   (when var-1 (setf ctemp2 var-1))
   (when var-2 (setf c var-2))
   (when var-3 (setf s var-3))
   (when var-4 (setf ctemp3 var-4)))
 (f2cl-lib:fdo (j istart (f2cl-lib:int-add j 1))
               ((> j
                   (f2cl-lib:int-add ilast (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (cond
      ((> j istart)
       (setf ctemp
               (f2cl-lib:fref h-%data% (j (f2cl-lib:int-sub j 1))
                              ((1 ldh) (1 *)) h-%offset%))
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (clartg ctemp
            (f2cl-lib:array-slice h-%data% f2cl-lib:complex8
                                  ((+ j 1) (f2cl-lib:int-sub j 1))
                                  ((1 ldh) (1 *)) h-%offset%)
            c s
            (f2cl-lib:array-slice h-%data% f2cl-lib:complex8
                                  (j (f2cl-lib:int-sub j 1))
                                  ((1 ldh) (1 *)) h-%offset%))
         (declare (ignore var-1 var-4))
         (when var-0 (setf ctemp var-0))
         (when var-2 (setf c var-2))
         (when var-3 (setf s var-3)))
       (setf (f2cl-lib:fref h-%data%
                            ((f2cl-lib:int-add j 1)
                             (f2cl-lib:int-sub j 1))
                            ((1 ldh) (1 *)) h-%offset%)
               czero)))
     (f2cl-lib:fdo (jc j (f2cl-lib:int-add jc 1))
                   ((> jc ilastm) nil)
       (tagbody
         (setf ctemp
                 (+
                  (* c
```

```
                    (f2cl-lib:fref h-%data% (j jc)
                                   ((1 ldh) (1 *)) h-%offset%))
                 (* s
                    (f2cl-lib:fref h-%data%
                                   ((f2cl-lib:int-add j 1) jc)
                                   ((1 ldh) (1 *)) h-%offset%))))
        (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add j 1) jc)
                             ((1 ldh) (1 *)) h-%offset%)
                (+
                 (* (- (f2cl-lib:conjg s))
                    (f2cl-lib:fref h-%data% (j jc)
                                   ((1 ldh) (1 *)) h-%offset%))
                 (* c
                    (f2cl-lib:fref h-%data%
                                   ((f2cl-lib:int-add j 1) jc)
                                   ((1 ldh) (1 *)) h-%offset%))))
        (setf (f2cl-lib:fref h-%data% (j jc) ((1 ldh) (1 *))
                             h-%offset%)
                ctemp)
        (setf ctemp2
                (+
                 (* c
                    (f2cl-lib:fref t$-%data% (j jc)
                                   ((1 ldt) (1 *)) t$-%offset%))
                 (* s
                    (f2cl-lib:fref t$-%data%
                                   ((f2cl-lib:int-add j 1) jc)
                                   ((1 ldt) (1 *))
                                   t$-%offset%))))
        (setf (f2cl-lib:fref t$-%data%
                             ((f2cl-lib:int-add j 1) jc)
                             ((1 ldt) (1 *)) t$-%offset%)
                (+
                 (* (- (f2cl-lib:conjg s))
                    (f2cl-lib:fref t$-%data% (j jc)
                                   ((1 ldt) (1 *)) t$-%offset%))
                 (* c
                    (f2cl-lib:fref t$-%data%
                                   ((f2cl-lib:int-add j 1) jc)
                                   ((1 ldt) (1 *))
                                   t$-%offset%))))
        (setf (f2cl-lib:fref t$-%data% (j jc) ((1 ldt) (1 *))
                             t$-%offset%)
                ctemp2)
    label100))
  (cond
   (ilq
    (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                  ((> jr n) nil)
      (tagbody
```

```
      (setf ctemp
              (+
               (* c
                  (f2cl-lib:fref q-%data% (jr j)
                                 ((1 ldq) (1 *)) q-%offset%))
               (* (f2cl-lib:conjg s)
                  (f2cl-lib:fref q-%data%
                                 (jr (f2cl-lib:int-add j 1))
                                 ((1 ldq) (1 *))
                                 q-%offset%))))
      (setf (f2cl-lib:fref q-%data%
                           (jr (f2cl-lib:int-add j 1))
                           ((1 ldq) (1 *)) q-%offset%)
              (+
               (* (- s)
                  (f2cl-lib:fref q-%data% (jr j)
                                 ((1 ldq) (1 *)) q-%offset%))
               (* c
                  (f2cl-lib:fref q-%data%
                                 (jr (f2cl-lib:int-add j 1))
                                 ((1 ldq) (1 *))
                                 q-%offset%))))
      (setf (f2cl-lib:fref q-%data% (jr j) ((1 ldq) (1 *))
                           q-%offset%)
              ctemp)
     label110))))
(setf ctemp
        (f2cl-lib:fref t$-%data%
                       ((f2cl-lib:int-add j 1)
                        (f2cl-lib:int-add j 1))
                       ((1 ldt) (1 *)) t$-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (clartg ctemp
     (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8
                           ((+ j 1) j) ((1 ldt) (1 *))
                           t$-%offset%)
     c s
     (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8
                           ((+ j 1) (f2cl-lib:int-add j 1))
                           ((1 ldt) (1 *)) t$-%offset%))
  (declare (ignore var-1 var-4))
  (when var-0 (setf ctemp var-0))
  (when var-2 (setf c var-2))
  (when var-3 (setf s var-3)))
(setf (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 1) j)
                     ((1 ldt) (1 *)) t$-%offset%)
        czero)
(f2cl-lib:fdo (jr ifrstm (f2cl-lib:int-add jr 1))
              ((> jr
                  (min
```

```
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add j 2))
                    (the f2cl-lib:integer4 ilast)))
                nil)
        (tagbody
          (setf ctemp
                  (+
                   (* c
                      (f2cl-lib:fref h-%data%
                                     (jr (f2cl-lib:int-add j 1))
                                     ((1 ldh) (1 *)) h-%offset%))
                   (* s
                      (f2cl-lib:fref h-%data% (jr j)
                                     ((1 ldh) (1 *)) h-%offset%))))
          (setf (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                               h-%offset%)
                  (+
                   (* (- (f2cl-lib:conjg s))
                      (f2cl-lib:fref h-%data%
                                     (jr (f2cl-lib:int-add j 1))
                                     ((1 ldh) (1 *)) h-%offset%))
                   (* c
                      (f2cl-lib:fref h-%data% (jr j)
                                     ((1 ldh) (1 *)) h-%offset%))))
          (setf (f2cl-lib:fref h-%data% (jr (f2cl-lib:int-add j 1))
                               ((1 ldh) (1 *)) h-%offset%)
                  ctemp)
         label120))
      (f2cl-lib:fdo (jr ifrstm (f2cl-lib:int-add jr 1))
                    ((> jr j) nil)
        (tagbody
          (setf ctemp
                  (+
                   (* c
                      (f2cl-lib:fref t$-%data%
                                     (jr (f2cl-lib:int-add j 1))
                                     ((1 ldt) (1 *)) t$-%offset%))
                   (* s
                      (f2cl-lib:fref t$-%data% (jr j)
                                     ((1 ldt) (1 *))
                                     t$-%offset%))))
          (setf (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                               t$-%offset%)
                  (+
                   (* (- (f2cl-lib:conjg s))
                      (f2cl-lib:fref t$-%data%
                                     (jr (f2cl-lib:int-add j 1))
                                     ((1 ldt) (1 *)) t$-%offset%))
                   (* c
                      (f2cl-lib:fref t$-%data% (jr j)
```

```
                                        ((1 ldt) (1 *))
                                        t$-%offset%))))
                (setf (f2cl-lib:fref t$-%data%
                                     (jr (f2cl-lib:int-add j 1))
                                     ((1 ldt) (1 *)) t$-%offset%)
                      ctemp)
              label130))
            (cond
             (ilz
              (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                            ((> jr n) nil)
                (tagbody
                  (setf ctemp
                        (+
                         (* c
                            (f2cl-lib:fref z-%data%
                                           (jr (f2cl-lib:int-add j 1))
                                           ((1 ldz) (1 *)) z-%offset%))
                         (* s
                            (f2cl-lib:fref z-%data% (jr j)
                                           ((1 ldz) (1 *))
                                           z-%offset%))))
                  (setf (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                       z-%offset%)
                        (+
                         (* (- (f2cl-lib:conjg s))
                            (f2cl-lib:fref z-%data%
                                           (jr (f2cl-lib:int-add j 1))
                                           ((1 ldz) (1 *)) z-%offset%))
                         (* c
                            (f2cl-lib:fref z-%data% (jr j)
                                           ((1 ldz) (1 *))
                                           z-%offset%))))
                  (setf (f2cl-lib:fref z-%data%
                                       (jr (f2cl-lib:int-add j 1))
                                       ((1 ldz) (1 *)) z-%offset%)
                        ctemp)
                label140))))
          label150))
      label160
      label170))
  label180
   (setf info ilast)
   (go label210)
  label190
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))) nil)
     (tagbody
       (setf absb
               (abs
```

```
            (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                           t$-%offset%)))
    (cond
     ((> absb safmin)
      (setf signbc
              (f2cl-lib:conjg
               (/
                (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                               t$-%offset%)
                absb)))
      (setf (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                           t$-%offset%)
              (coerce absb 'f2cl-lib:complex8))
      (cond
       (ilschr
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (cscal (f2cl-lib:int-sub j 1) signbc
             (f2cl-lib:array-slice t$-%data% f2cl-lib:complex8 (1 j)
                                   ((1 ldt) (1 *)) t$-%offset%)
             1)
          (declare (ignore var-0 var-2 var-3))
          (when var-1 (setf signbc var-1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (cscal j signbc
             (f2cl-lib:array-slice h-%data% f2cl-lib:complex8 (1 j)
                                   ((1 ldh) (1 *)) h-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf j var-0))
          (when var-1 (setf signbc var-1))))
       (t
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (cscal 1 signbc
             (f2cl-lib:array-slice h-%data% f2cl-lib:complex8 (j j)
                                   ((1 ldh) (1 *)) h-%offset%)
             1)
          (declare (ignore var-0 var-2 var-3))
          (when var-1 (setf signbc var-1)))))
      (if ilz
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (cscal n signbc
               (f2cl-lib:array-slice z-%data% f2cl-lib:complex8 (1 j)
                                     ((1 ldz) (1 *)) z-%offset%)
               1)
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf signbc var-1)))))
     (t
      (setf (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                           t$-%offset%)
```

```
                        czero)))
              (setf (f2cl-lib:fref alpha-%data% (j) ((1 *)) alpha-%offset%)
                      (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *))
                                     h-%offset%))
              (setf (f2cl-lib:fref beta-%data% (j) ((1 *)) beta-%offset%)
                      (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                                     t$-%offset%))
            label200))
          (setf info 0)
         label210
          (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                  (f2cl-lib:cmplx n))
          (go end_label)
         end_label
          (return
           (values job compq compz n nil nil nil ldh nil ldt nil nil nil ldq
                   nil ldz nil nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::chgeqz
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
```

```
'(fortran-to-lisp::job
  fortran-to-lisp::compq
  fortran-to-lisp::compz
  fortran-to-lisp::n nil nil nil
  fortran-to-lisp::ldh nil
  fortran-to-lisp::ldt nil nil nil
  fortran-to-lisp::ldq nil
  fortran-to-lisp::ldz nil nil nil
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

34.35 cla_geamv

34.35.1 Axiom unit tests

— **clageamv.input** —

```
)set break resume
)sys rm -f clageamv.output
)spool clageamv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.35.2 Axiom help page

— **cla_geamv.help** —

```
==========================================================================
cla_geamv examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine cla_geamv (TRANS, M, N, ALPHA, A, LDA, X, INCX, BETA, Y,
           INCY)
           CLA_GEAMV computes a matrix-vector product using a general matrix
           to calculate error bounds.

Function/Subroutine Documentation
   subroutine cla_geamv (integer TRANS, integer M, integer N, real ALPHA,
       complex, dimension( lda, * ) A, integer LDA, complex, dimension( * ) X,
       integer INCX, real BETA, real, dimension( * ) Y, integer INCY)
       CLA_GEAMV computes a matrix-vector product using a general matrix to
       calculate error bounds.

Purpose:
   CLA_GEAMV  performs one of the matrix-vector operations

               y := alpha*abs(A)*abs(x) + beta*abs(y),
          or   y := alpha*abs(A)**T*abs(x) + beta*abs(y),

   where alpha and beta are scalars, x and y are vectors and A is an
   m by n matrix.

   This function is primarily used in calculating error bounds.
   To protect against underflow during evaluation, components in
   the resulting vector are perturbed away from zero by (N+1)
   times the underflow threshold.  To prevent unnecessarily large
   errors for block-structure embedded in general matrices,
   "symbolically" zero components are not perturbed.  A zero
   entry is considered "symbolic" if all multiplications involved
   in computing that entry have at least one zero multiplicand.

Parameters:

 TRANS
   TRANS is INTEGER
   On entry, TRANS specifies the operation to be performed as
   follows:

   BLAS_NO_TRANS      y := alpha*abs(A)*abs(x) + beta*abs(y)
   BLAS_TRANS         y := alpha*abs(A**T)*abs(x) + beta*abs(y)
   BLAS_CONJ_TRANS    y := alpha*abs(A**T)*abs(x) + beta*abs(y)

   Unchanged on exit.

 M
   M is INTEGER
   On entry, M specifies the number of rows of the matrix A.
   M must be at least zero.
```

```
  Unchanged on exit.

N
  N is INTEGER
  On entry, N specifies the number of columns of the matrix A.
  N must be at least zero.
  Unchanged on exit.

ALPHA
  ALPHA is REAL
  On entry, ALPHA specifies the scalar alpha.
  Unchanged on exit.

A
  A is COMPLEX array, dimension (LDA,n)
  Before entry, the leading m by n part of the array A must
  contain the matrix of coefficients.
  Unchanged on exit.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least
  max( 1, m ).
  Unchanged on exit.

X
  X is COMPLEX array, dimension
  ( 1 + ( n - 1 )*abs( INCX ) ) when TRANS = 'N' or 'n'
  and at least
  ( 1 + ( m - 1 )*abs( INCX ) ) otherwise.
  Before entry, the incremented array X must contain the vector x.
  Unchanged on exit.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of
  X. INCX must not be zero.
  Unchanged on exit.

BETA
  BETA is REAL
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.
  Unchanged on exit.

Y
  Y is REAL array, dimension
  ( 1 + ( m - 1 )*abs( INCY ) ) when TRANS = 'N' or 'n'
  and at least
```

```
  ( 1 + ( n - 1 )*abs( INCY ) ) otherwise.
  Before entry with BETA non-zero, the incremented array Y
  must contain the vector y. On exit, Y is overwritten by the
  updated vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of
  Y. INCY must not be zero.
  Unchanged on exit.
```

34.35.3 fortran code

— cla_geamv.f —

```
*  =====================================================================
      SUBROUTINE CLA_GEAMV ( TRANS, M, N, ALPHA, A, LDA, X, INCX, BETA,
     $                       Y, INCY )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      REAL               ALPHA, BETA
      INTEGER            INCX, INCY, LDA, M, N
      INTEGER            TRANS
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), X( * )
      REAL               Y( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            SYMB_ZERO
      REAL               TEMP, SAFE1
      INTEGER            I, INFO, IY, J, JX, KX, KY, LENX, LENY
      COMPLEX            CDUM
```

```
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, SLAMCH
      REAL               SLAMCH
*     ..
*     .. External Functions ..
      EXTERNAL           ILATRANS
      INTEGER            ILATRANS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, ABS, REAL, AIMAG, SIGN
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( CDUM ) = ABS( REAL( CDUM ) ) + ABS( AIMAG( CDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF     ( .NOT.( ( TRANS.EQ.ILATRANS( 'N' ) )
     $           .OR. ( TRANS.EQ.ILATRANS( 'T' ) )
     $           .OR. ( TRANS.EQ.ILATRANS( 'C' ) ) ) ) THEN
         INFO = 1
      ELSE IF( M.LT.0 )THEN
         INFO = 2
      ELSE IF( N.LT.0 )THEN
         INFO = 3
      ELSE IF( LDA.LT.MAX( 1, M ) )THEN
         INFO = 6
      ELSE IF( INCX.EQ.0 )THEN
         INFO = 8
      ELSE IF( INCY.EQ.0 )THEN
         INFO = 11
      END IF
      IF( INFO.NE.0 )THEN
         CALL XERBLA( 'CLA_GEAMV ', INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( ( M.EQ.0 ).OR.( N.EQ.0 ).OR.
     $    ( ( ALPHA.EQ.ZERO ).AND.( BETA.EQ.ONE ) ) )
     $   RETURN
*
*     Set  LENX  and  LENY, the lengths of the vectors x and y, and set
```

```
*     up the start points in  X  and  Y.
*
      IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
         LENX = N
         LENY = M
      ELSE
         LENX = M
         LENY = N
      END IF
      IF( INCX.GT.0 )THEN
         KX = 1
      ELSE
         KX = 1 - ( LENX - 1 )*INCX
      END IF
      IF( INCY.GT.0 )THEN
         KY = 1
      ELSE
         KY = 1 - ( LENY - 1 )*INCY
      END IF
*
*     Set SAFE1 essentially to be the underflow threshold times the
*     number of additions in each row.
*
      SAFE1 = SLAMCH( 'Safe minimum' )
      SAFE1 = (N+1)*SAFE1
*
*     Form  y := alpha*abs(A)*abs(x) + beta*abs(y).
*
*     The O(M*N) SYMB_ZERO tests could be replaced by O(N) queries to
*     the inexact flag.  Still doesn't help change the iteration order
*     to per-column.
*
      IY = KY
      IF ( INCX.EQ.1 ) THEN
         IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
            DO I = 1, LENY
               IF ( BETA .EQ. 0.0 ) THEN
                  SYMB_ZERO = .TRUE.
                  Y( IY ) = 0.0
               ELSE IF ( Y( IY ) .EQ. 0.0 ) THEN
                  SYMB_ZERO = .TRUE.
               ELSE
                  SYMB_ZERO = .FALSE.
                  Y( IY ) = BETA * ABS( Y( IY ) )
               END IF
               IF ( ALPHA .NE. 0.0 ) THEN
                  DO J = 1, LENX
                     TEMP = CABS1( A( I, J ) )
                     SYMB_ZERO = SYMB_ZERO .AND.
     $                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )
```

```
                 Y( IY ) = Y( IY ) + ALPHA*CABS1( X( J ) )*TEMP
              END DO
           END IF

           IF ( .NOT.SYMB_ZERO ) Y( IY ) =
$               Y( IY ) + SIGN( SAFE1, Y( IY ) )

           IY = IY + INCY
        END DO
     ELSE
        DO I = 1, LENY
           IF ( BETA .EQ. 0.0 ) THEN
              SYMB_ZERO = .TRUE.
              Y( IY ) = 0.0
           ELSE IF ( Y( IY ) .EQ. 0.0 ) THEN
              SYMB_ZERO = .TRUE.
           ELSE
              SYMB_ZERO = .FALSE.
              Y( IY ) = BETA * ABS( Y( IY ) )
           END IF
           IF ( ALPHA .NE. 0.0 ) THEN
              DO J = 1, LENX
                 TEMP = CABS1( A( J, I ) )
                 SYMB_ZERO = SYMB_ZERO .AND.
$                     ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                 Y( IY ) = Y( IY ) + ALPHA*CABS1( X( J ) )*TEMP
              END DO
           END IF

           IF ( .NOT.SYMB_ZERO ) Y( IY ) =
$               Y( IY ) + SIGN( SAFE1, Y( IY ) )

           IY = IY + INCY
        END DO
     END IF
  ELSE
     IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
        DO I = 1, LENY
           IF ( BETA .EQ. 0.0 ) THEN
              SYMB_ZERO = .TRUE.
              Y( IY ) = 0.0
           ELSE IF ( Y( IY ) .EQ. 0.0 ) THEN
              SYMB_ZERO = .TRUE.
           ELSE
              SYMB_ZERO = .FALSE.
              Y( IY ) = BETA * ABS( Y( IY ) )
           END IF
           IF ( ALPHA .NE. 0.0 ) THEN
```

```
                JX = KX
                DO J = 1, LENX
                   TEMP = CABS1( A( I, J ) )
                   SYMB_ZERO = SYMB_ZERO .AND.
     $                  ( X( JX ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                   Y( IY ) = Y( IY ) + ALPHA*CABS1( X( JX ) )*TEMP
                   JX = JX + INCX
                END DO
             END IF

             IF ( .NOT.SYMB_ZERO ) Y( IY ) =
     $            Y( IY ) + SIGN( SAFE1, Y( IY ) )

             IY = IY + INCY
          END DO
       ELSE
          DO I = 1, LENY
             IF ( BETA .EQ. 0.0 ) THEN
                SYMB_ZERO = .TRUE.
                Y( IY ) = 0.0
             ELSE IF ( Y( IY ) .EQ. 0.0 ) THEN
                SYMB_ZERO = .TRUE.
             ELSE
                SYMB_ZERO = .FALSE.
                Y( IY ) = BETA * ABS( Y( IY ) )
             END IF
             IF ( ALPHA .NE. 0.0 ) THEN
                JX = KX
                DO J = 1, LENX
                   TEMP = CABS1( A( J, I ) )
                   SYMB_ZERO = SYMB_ZERO .AND.
     $                  ( X( JX ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                   Y( IY ) = Y( IY ) + ALPHA*CABS1( X( JX ) )*TEMP
                   JX = JX + INCX
                END DO
             END IF

             IF ( .NOT.SYMB_ZERO ) Y( IY ) =
     $            Y( IY ) + SIGN( SAFE1, Y( IY ) )

             IY = IY + INCY
          END DO
       END IF

      END IF
*
      RETURN
*
```

```
*     End of CLA_GEAMV
*
      END
```

———-

34.35.4 lisp code

— **cla_geamv.lisp** —

```
(let* ((one (coerce 1.0 'f2cl-lib:complex8))
       (zero (coerce 0.0 'f2cl-lib:complex8)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (ignorable one zero))
  (defun cla_geamv (trans m n alpha a lda x incx beta y incy)
    (declare (type (array single-float (*)) y)
             (type (array f2cl-lib:complex8 (*)) x a)
             (type (single-float) beta alpha)
             (type (f2cl-lib:integer4) incy incx lda n m trans))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%)
         (y single-float y-%data% y-%offset%))
      (labels ((cabs1 (cdum)
                 (+ (abs (f2cl-lib:freal cdum)) (abs (f2cl-lib:aimag cdum)))))
        (declare
         (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
          cabs1))
        (prog ((cdum #C(0.0 0.0)) (i 0) (info 0) (iy 0) (j 0) (jx 0) (kx 0)
               (ky 0) (lenx 0) (leny 0) (temp 0.0) (safe1 0.0) (symb_zero nil))
          (declare (type (f2cl-lib:complex8) cdum)
                   (type (f2cl-lib:integer4) i info iy j jx kx ky lenx leny)
                   (type (single-float) temp safe1)
                   (type f2cl-lib:logical symb_zero))
          (setf info 0)
          (cond
           ((not
             (or (= trans (ilatrans "N")) (= trans (ilatrans "T"))
                 (= trans (ilatrans "C"))))
            (setf info 1))
           ((< m 0) (setf info 2)) ((< n 0) (setf info 3))
           ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
            (setf info 6))
           ((= incx 0) (setf info 8)) ((= incy 0) (setf info 11)))
          (cond
           ((/= info 0)
            (multiple-value-bind (var-0 var-1)
```

```
      (xerbla "CLA_GEAMV " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
    (go end_label))
(cond ((= trans (ilatrans "N")) (setf lenx n) (setf leny m))
      (t (setf lenx m) (setf leny n)))
(cond ((> incx 0) (setf kx 1))
      (t
       (setf kx
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub lenx 1) incx)))))
(cond ((> incy 0) (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub leny 1) incy)))))
(setf safe1 (slamch "Safe minimum"))
(setf safe1 (* (f2cl-lib:int-add n 1) safe1))
(setf iy ky)
(cond
 ((= incx 1)
  (cond
   ((= trans (ilatrans "N"))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i leny) nil)
      (tagbody
        (cond
         ((= beta 0.0) (setf symb_zero f2cl-lib:%true%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  0.0))
         ((= (f2cl-lib:fref y (iy) ((1 *))) 0.0)
          (setf symb_zero f2cl-lib:%true%))
         (t (setf symb_zero f2cl-lib:%false%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (* beta
                     (abs
                      (f2cl-lib:fref y-%data% (iy) ((1 *))
                                     y-%offset%))))))
        (cond
         ((/= alpha 0.0)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j lenx) nil)
            (tagbody
              (setf temp
                      (cabs1
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
```

```
                                        a-%offset%)))
                (setf symb_zero
                        (and symb_zero
                             (or
                              (=
                               (f2cl-lib:fref x-%data% (j) ((1 *))
                                              x-%offset%)
                               zero)
                              (= temp zero))))
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data% (iy) ((1 *))
                                        y-%offset%)
                         (* alpha
                            (cabs1
                             (f2cl-lib:fref x-%data% (j) ((1 *))
                                            x-%offset%))
                            temp)))
             label100001))))
        (if (not symb_zero)
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+
                     (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (f2cl-lib:sign safe1
                                    (f2cl-lib:fref y-%data% (iy)
                                                   ((1 *))
                                                   y-%offset%)))))
        (setf iy (f2cl-lib:int-add iy incy))
      label100000)))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i leny) nil)
     (tagbody
       (cond
        ((= beta 0.0) (setf symb_zero f2cl-lib:%true%)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 0.0))
        ((= (f2cl-lib:fref y (iy) ((1 *))) 0.0)
         (setf symb_zero f2cl-lib:%true%))
        (t (setf symb_zero f2cl-lib:%false%)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (* beta
                    (abs
                     (f2cl-lib:fref y-%data% (iy) ((1 *))
                                    y-%offset%))))))
       (cond
        ((/= alpha 0.0)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j lenx) nil)
           (tagbody
```

```
                (setf temp
                        (cabs1
                         (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                        a-%offset%)))
                (setf symb_zero
                        (and symb_zero
                             (or
                              (=
                               (f2cl-lib:fref x-%data% (j) ((1 *))
                                              x-%offset%)
                               zero)
                              (= temp zero))))
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data% (iy) ((1 *))
                                        y-%offset%)
                         (* alpha
                            (cabs1
                             (f2cl-lib:fref x-%data% (j) ((1 *))
                                            x-%offset%))
                            temp)))
             label100003))))
        (if (not symb_zero)
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+
                     (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (f2cl-lib:sign safe1
                                    (f2cl-lib:fref y-%data% (iy)
                                                   ((1 *))
                                                   y-%offset%)))))
        (setf iy (f2cl-lib:int-add iy incy))
       label100002)))))
(t
 (cond
  ((= trans (ilatrans "N"))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i leny) nil)
     (tagbody
       (cond
        ((= beta 0.0) (setf symb_zero f2cl-lib:%true%)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 0.0))
        ((= (f2cl-lib:fref y (iy) ((1 *))) 0.0)
         (setf symb_zero f2cl-lib:%true%))
        (t (setf symb_zero f2cl-lib:%false%)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (* beta
                    (abs
                     (f2cl-lib:fref y-%data% (iy) ((1 *))
                                    y-%offset%))))))
```

```
      (cond
       ((/= alpha 0.0) (setf jx kx)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j lenx) nil)
          (tagbody
            (setf temp
                    (cabs1
                     (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%)))
            (setf symb_zero
                    (and symb_zero
                         (or
                          (=
                           (f2cl-lib:fref x-%data% (jx) ((1 *))
                                          x-%offset%)
                           zero)
                          (= temp zero))))
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+
                     (f2cl-lib:fref y-%data% (iy) ((1 *))
                                    y-%offset%)
                     (* alpha
                        (cabs1
                         (f2cl-lib:fref x-%data% (jx) ((1 *))
                                        x-%offset%))
                        temp)))
            (setf jx (f2cl-lib:int-add jx incx))
           label100005))))
      (if (not symb_zero)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (+
                   (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                   (f2cl-lib:sign safe1
                                  (f2cl-lib:fref y-%data% (iy)
                                                 ((1 *))
                                                 y-%offset%)))))
      (setf iy (f2cl-lib:int-add iy incy))
     label100004)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i leny) nil)
    (tagbody
      (cond
       ((= beta 0.0) (setf symb_zero f2cl-lib:%true%)
        (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                0.0))
       ((= (f2cl-lib:fref y (iy) ((1 *))) 0.0)
        (setf symb_zero f2cl-lib:%true%))
       (t (setf symb_zero f2cl-lib:%false%)
        (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
```

```
                    (* beta
                       (abs
                        (f2cl-lib:fref y-%data% (iy) ((1 *))
                                       y-%offset%))))))
            (cond
             ((/= alpha 0.0) (setf jx kx)
              (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                            ((> j lenx) nil)
                (tagbody
                  (setf temp
                          (cabs1
                           (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                          a-%offset%)))
                  (setf symb_zero
                          (and symb_zero
                               (or
                                (=
                                 (f2cl-lib:fref x-%data% (jx) ((1 *))
                                                x-%offset%)
                                 zero)
                                (= temp zero))))
                  (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                          (+
                           (f2cl-lib:fref y-%data% (iy) ((1 *))
                                          y-%offset%)
                           (* alpha
                              (cabs1
                               (f2cl-lib:fref x-%data% (jx) ((1 *))
                                              x-%offset%))
                              temp)))
                  (setf jx (f2cl-lib:int-add jx incx))
                 label100007))))
            (if (not symb_zero)
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (f2cl-lib:sign safe1
                                        (f2cl-lib:fref y-%data% (iy)
                                                       ((1 *))
                                                       y-%offset%)))))
            (setf iy (f2cl-lib:int-add iy incy))
           label100006))))))
     (go end_label)
    end_label
     (return (values nil nil nil nil nil nil nil nil nil nil nil)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cla_geamv
```

```
        fortran-to-lisp::*f2cl-function-info*)
    (fortran-to-lisp::make-f2cl-finfo :arg-types
                                      '((fortran-to-lisp::integer4)
                                        (fortran-to-lisp::integer4)
                                        (fortran-to-lisp::integer4)
                                        (single-float)
                                        (array fortran-to-lisp::complex8
                                         (*))
                                        (fortran-to-lisp::integer4)
                                        (array fortran-to-lisp::complex8
                                         (*))
                                        (fortran-to-lisp::integer4)
                                        (single-float)
                                        (array single-float (*))
                                        (fortran-to-lisp::integer4))
                                      :return-values
                                      '(nil nil nil nil nil nil nil nil
                                        nil nil nil)
                                      :calls 'nil)))
```

———-

34.36 cla_gercond_c

34.36.1 Axiom unit tests

— clagercondc.input —

```
)set break resume
)sys rm -f clagercondc.output
)spool clagercondc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.36.2 Axiom help page

— cla_gercond_c.help —

```
====================================================================
cla_gercond_c examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       real function cla_gercond_c (TRANS, N, A, LDA, AF, LDAF, IPIV, C,
           CAPPLY, INFO, WORK, RWORK)
           CLA_GERCOND_C computes the infinity norm condition number of
           op(A)*inv(diag(c)) for general matrices.

Function/Subroutine Documentation
   real function cla_gercond_c (character TRANS, integer N, complex,
       dimension( lda, * ) A, integer LDA, complex, dimension( ldaf, * ) AF,
       integer LDAF, integer, dimension( * ) IPIV, real, dimension( * ) C,
       logical CAPPLY, integer INFO, complex, dimension( * ) WORK, real,
       dimension( * ) RWORK)
       CLA_GERCOND_C computes the infinity norm condition number of
       op(A)*inv(diag(c)) for general matrices.

Purpose:
   CLA_GERCOND_C computes the infinity norm condition number of
   op(A) * inv(diag(C)) where C is a REAL vector.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate Transpose = Transpose)

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the N-by-N matrix A

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).
```

```
 AF
   AF is COMPLEX array, dimension (LDAF,N)
   The factors L and U from the factorization
   A = P*L*U as computed by CGETRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   The pivot indices from the factorization A = P*L*U
   as computed by CGETRF; row i of the matrix was interchanged
   with row IPIV(i).

 C
   C is REAL array, dimension (N)
   The vector C in the formula op(A) * inv(diag(C)).

 CAPPLY
   CAPPLY is LOGICAL
   If .TRUE. then access the vector C in the formula above.

 INFO
   INFO is INTEGER
   = 0:  Successful exit.
   i > 0:  The ith argument is invalid.

 WORK
   WORK is COMPLEX array, dimension (2*N).
   Workspace.

 RWORK
   RWORK is REAL array, dimension (N).
   Workspace.
```

———-

34.36.3 fortran code

— **cla_gercond_c.f** —

```
*  =====================================================================
      REAL FUNCTION CLA_GERCOND_C( TRANS, N, A, LDA, AF, LDAF, IPIV, C,
     $                             CAPPLY, INFO, WORK, RWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
```

```
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Aguments ..
      CHARACTER          TRANS
      LOGICAL            CAPPLY
      INTEGER            N, LDA, LDAF, INFO
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), AF( LDAF, * ), WORK( * )
      REAL               C( * ), RWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            NOTRANS
      INTEGER            KASE, I, J
      REAL               AINVNM, ANORM, TMP
      COMPLEX            ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLACN2, CGETRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, REAL, AIMAG
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
      CLA_GERCOND_C = 0.0E+0
*
      INFO = 0
      NOTRANS = LSAME( TRANS, 'N' )
      IF ( .NOT. NOTRANS .AND. .NOT. LSAME( TRANS, 'T' ) .AND. .NOT.
     $     LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
```

```
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CLA_GERCOND_C', -INFO )
         RETURN
      END IF
*
*     Compute norm of op(A)*op2(C).
*
      ANORM = 0.0E+0
      IF ( NOTRANS ) THEN
         DO I = 1, N
            TMP = 0.0E+0
            IF ( CAPPLY ) THEN
               DO J = 1, N
                  TMP = TMP + CABS1( A( I, J ) ) / C( J )
               END DO
            ELSE
               DO J = 1, N
                  TMP = TMP + CABS1( A( I, J ) )
               END DO
            END IF
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      ELSE
         DO I = 1, N
            TMP = 0.0E+0
            IF ( CAPPLY ) THEN
               DO J = 1, N
                  TMP = TMP + CABS1( A( J, I ) ) / C( J )
               END DO
            ELSE
               DO J = 1, N
                  TMP = TMP + CABS1( A( J, I ) )
               END DO
            END IF
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 ) THEN
```

```
         CLA_GERCOND_C = 1.0E+0
         RETURN
      ELSE IF( ANORM .EQ. 0.0E+0 ) THEN
         RETURN
      END IF
*
*     Estimate the norm of inv(op(A)).
*
      AINVNM = 0.0E+0
*
      KASE = 0
   10 CONTINUE
      CALL CLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
*
            IF (NOTRANS) THEN
               CALL CGETRS( 'No transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ELSE
               CALL CGETRS( 'Conjugate transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ENDIF
*
*           Multiply by inv(C).
*
            IF ( CAPPLY ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF
         ELSE
*
*           Multiply by inv(C**H).
*
            IF ( CAPPLY ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF
*
            IF ( NOTRANS ) THEN
               CALL CGETRS( 'Conjugate transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
```

```
            ELSE
               CALL CGETRS( 'No transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            END IF
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM .NE. 0.0E+0 )
     $   CLA_GERCOND_C = 1.0E+0 / AINVNM
*
      RETURN
*
      END
```

———-

34.36.4 lisp code

— cla_gercond_c.lisp —

```
(defun cla_gercond_c (trans n a lda af ldaf ipiv c capply info work rwork)
  (declare (type f2cl-lib:logical capply)
           (type (array single-float (*)) rwork c)
           (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex8 (*)) work af a)
           (type (f2cl-lib:integer4) info ldaf lda n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (af f2cl-lib:complex8 af-%data% af-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (c single-float c-%data% c-%offset%)
       (rwork single-float rwork-%data% rwork-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
```

```
    cabs1))
  (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
         (zdum #C(0.0 0.0)) (ainvnm 0.0) (anorm 0.0) (tmp 0.0) (kase 0)
         (i 0) (j 0) (notrans nil) (cla_gercond_c 0.0))
    (declare (type f2cl-lib:logical notrans)
             (type (f2cl-lib:integer4) j i kase)
             (type (single-float) cla_gercond_c tmp anorm ainvnm)
             (type (f2cl-lib:complex8) zdum)
             (type (array f2cl-lib:integer4 (3)) isave))
    (setf cla_gercond_c 0.0)
    (setf info 0)
    (setf notrans
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "N")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val))
    (cond
     ((and (not notrans)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "T")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "C")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val)))
      (setf info -1))
     ((< n 0) (setf info -2))
     ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -4))
     ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -6)))
    (cond
     ((/= info 0) (xerbla "CLA_GERCOND_C" (f2cl-lib:int-sub info))
      (go end_label)))
    (setf anorm 0.0)
    (cond
     (notrans
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf tmp 0.0)
          (cond
           (capply
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                        ((> j n) nil)
                (tagbody
                  (setf tmp
                          (+ tmp
                             (/
                              (cabs1
                               (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                              a-%offset%))
                              (f2cl-lib:fref c-%data% (j) ((1 *))
                                             c-%offset%))))
                 label100001)))
              (t
               (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                             ((> j n) nil)
                (tagbody
                  (setf tmp
                          (+ tmp
                             (cabs1
                              (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                             a-%offset%))))
                 label100002))))
             (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                     tmp)
             (setf anorm (max anorm tmp))
            label100000)))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf tmp 0.0)
             (cond
              (capply
               (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                             ((> j n) nil)
                (tagbody
                  (setf tmp
                          (+ tmp
                             (/
                              (cabs1
                               (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                              a-%offset%))
                              (f2cl-lib:fref c-%data% (j) ((1 *))
                                             c-%offset%))))
                 label100004)))
              (t
               (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                             ((> j n) nil)
                (tagbody
                  (setf tmp
                          (+ tmp
```

```
                        (cabs1
                         (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                        a-%offset%))))
            label100005))))
       (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
               tmp)
       (setf anorm (max anorm tmp))
      label100003))))
 (cond ((= n 0) (setf cla_gercond_c 1.0) (go end_label))
       ((= anorm 0.0) (go end_label)))
 (setf ainvnm 0.0)
 (setf kase 0)
label10
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
     (clacn2 n
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ n 1))
                            ((1 *)) work-%offset%)
      work ainvnm kase isave)
   (declare (ignore var-1 var-2 var-5))
   (setf n var-0)
   (setf ainvnm var-3)
   (setf kase var-4))
 (cond
  ((/= kase 0)
   (cond
    ((= kase 2)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (*
                  (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                 rwork-%offset%)))
        label100006))
     (cond
      (notrans
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
           (cgetrs "No transpose" n 1 af ldaf ipiv work n info)
         (declare (ignore var-0 var-2 var-3 var-5 var-6))
         (setf n var-1)
         (setf ldaf var-4)
         (setf n var-7)
         (setf info var-8)))
      (t
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
           (cgetrs "Conjugate transpose" n 1 af ldaf ipiv work n info)
         (declare (ignore var-0 var-2 var-3 var-5 var-6))
```

```
        (setf n var-1)
        (setf ldaf var-4)
        (setf n var-7)
        (setf info var-8))))
    (cond
     (capply
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (*
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                   (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
         label100007)))))
  (t
   (cond
    (capply
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (*
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)
                  (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
         label100008))))
    (cond
     (notrans
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (cgetrs "Conjugate transpose" n 1 af ldaf ipiv work n info)
        (declare (ignore var-0 var-2 var-3 var-5 var-6))
        (setf n var-1)
        (setf ldaf var-4)
        (setf n var-7)
        (setf info var-8)))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (cgetrs "No transpose" n 1 af ldaf ipiv work n info)
        (declare (ignore var-0 var-2 var-3 var-5 var-6))
        (setf n var-1)
        (setf ldaf var-4)
        (setf n var-7)
        (setf info var-8))))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
```

```
                        (*
                         (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                         (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                        rwork-%offset%)))
               label100009))))
          (go label10)))
        (if (/= ainvnm 0.0)
            (setf cla_gercond_c (/ 1.0 ainvnm)))
        (go end_label)
       end_label
        (return
         (values cla_gercond_c trans n nil nil nil ldaf nil nil nil info nil
                 nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cla_gercond_c
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              fortran-to-lisp::logical
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array single-float (*)))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::ldaf nil nil nil
                                              fortran-to-lisp::info nil nil)
                                            :calls
                                            '(fortran-to-lisp::cgetrs
                                              fortran-to-lisp::clacn2))))
```

34.37 cla_gercond_x

34.37.1 Axiom unit tests

— clagercondx.input —

```
)set break resume
)sys rm -f clagercondx.output
)spool clagercondx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.37.2 Axiom help page

— cla_gercond_x.help —

```
====================================================================
cla_gercond_x examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       real function cla_gercond_x (TRANS, N, A, LDA, AF, LDAF, IPIV, X, INFO,
           WORK, RWORK)
           CLA_GERCOND_X computes the infinity norm condition number of
           op(A)*diag(x) for general matrices.

Function/Subroutine Documentation
   real function cla_gercond_x (character TRANS, integer N, complex,
       dimension( lda, * ) A, integer LDA, complex, dimension( ldaf, * ) AF,
       integer LDAF, integer, dimension( * ) IPIV, complex, dimension( * ) X,
       integer INFO, complex, dimension( * ) WORK, real, dimension( * ) RWORK)
       CLA_GERCOND_X computes the infinity norm condition number of
       op(A)*diag(x) for general matrices.

Purpose:
```

```
   CLA_GERCOND_X computes the infinity norm condition number of
   op(A) * diag(X) where X is a COMPLEX vector.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate Transpose = Transpose)

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is COMPLEX array, dimension (LDAF,N)
   The factors L and U from the factorization
   A = P*L*U as computed by CGETRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   The pivot indices from the factorization A = P*L*U
   as computed by CGETRF; row i of the matrix was interchanged
   with row IPIV(i).

 X
   X is COMPLEX array, dimension (N)
   The vector X in the formula op(A) * diag(X).

 INFO
   INFO is INTEGER
   = 0:  Successful exit.
   i > 0:  The ith argument is invalid.

 WORK
```

```
  WORK is COMPLEX array, dimension (2*N).
  Workspace.

RWORK
  RWORK is REAL array, dimension (N).
  Workspace.
```

34.37.3 fortran code

— cla_gercond_x.f —

```
*  =====================================================================
      REAL FUNCTION CLA_GERCOND_X( TRANS, N, A, LDA, AF, LDAF, IPIV, X,
     $                             INFO, WORK, RWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            N, LDA, LDAF, INFO
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), AF( LDAF, * ), WORK( * ), X( * )
      REAL               RWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            NOTRANS
      INTEGER            KASE
      REAL               AINVNM, ANORM, TMP
      INTEGER            I, J
      COMPLEX            ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
```

```
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLACN2, CGETRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, REAL, AIMAG
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      CLA_GERCOND_X = 0.0E+0
*
      INFO = 0
      NOTRANS = LSAME( TRANS, 'N' )
      IF ( .NOT. NOTRANS .AND. .NOT. LSAME( TRANS, 'T' ) .AND. .NOT.
     $     LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CLA_GERCOND_X', -INFO )
         RETURN
      END IF
*
*     Compute norm of op(A)*op2(C).
*
      ANORM = 0.0
      IF ( NOTRANS ) THEN
         DO I = 1, N
            TMP = 0.0E+0
            DO J = 1, N
               TMP = TMP + CABS1( A( I, J ) * X( J ) )
            END DO
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      ELSE
         DO I = 1, N
            TMP = 0.0E+0
            DO J = 1, N
```

```
               TMP = TMP + CABS1( A( J, I ) * X( J ) )
            END DO
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 ) THEN
         CLA_GERCOND_X = 1.0E+0
         RETURN
      ELSE IF( ANORM .EQ. 0.0E+0 ) THEN
         RETURN
      END IF
*
*     Estimate the norm of inv(op(A)).
*
      AINVNM = 0.0E+0
*
      KASE = 0
   10 CONTINUE
      CALL CLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*           Multiply by R.
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
*
            IF ( NOTRANS ) THEN
               CALL CGETRS( 'No transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ELSE
               CALL CGETRS( 'Conjugate transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ENDIF
*
*           Multiply by inv(X).
*
            DO I = 1, N
               WORK( I ) = WORK( I ) / X( I )
            END DO
         ELSE
*
*           Multiply by inv(X**H).
*
            DO I = 1, N
               WORK( I ) = WORK( I ) / X( I )
            END DO
```

```
*
            IF ( NOTRANS ) THEN
               CALL CGETRS( 'Conjugate transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ELSE
               CALL CGETRS( 'No transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            END IF
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM .NE. 0.0E+0 )
     $   CLA_GERCOND_X = 1.0E+0 / AINVNM
*
      RETURN
*
      END
```

———-

34.37.4 lisp code

— **cla_gercond_x.lisp** —

```
(defun cla_gercond_x (trans n a lda af ldaf ipiv x info work rwork)
  (declare (type (array single-float (*)) rwork)
           (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex8 (*)) work x af a)
           (type (f2cl-lib:integer4) info ldaf lda n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (af f2cl-lib:complex8 af-%data% af-%offset%)
       (x f2cl-lib:complex8 x-%data% x-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (rwork single-float rwork-%data% rwork-%offset%))
    (labels ((cabs1 (zdum)
```

```
        (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
(declare
 (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
  cabs1))
(prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
       (zdum #C(0.0 0.0)) (i 0) (j 0) (ainvnm 0.0) (anorm 0.0) (tmp 0.0)
       (kase 0) (notrans nil) (cla_gercond_x 0.0))
  (declare (type f2cl-lib:logical notrans)
           (type (single-float) cla_gercond_x tmp anorm ainvnm)
           (type (f2cl-lib:integer4) kase j i)
           (type (f2cl-lib:complex8) zdum)
           (type (array f2cl-lib:integer4 (3)) isave))
  (setf cla_gercond_x 0.0)
  (setf info 0)
  (setf notrans
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame trans "N")
            (declare (ignore var-1))
            (when var-0 (setf trans var-0))
            ret-val))
  (cond
   ((and (not notrans)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame trans "T")
            (declare (ignore var-1))
            (when var-0 (setf trans var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame trans "C")
            (declare (ignore var-1))
            (when var-0 (setf trans var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4))
   ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -6)))
  (cond
   ((/= info 0) (xerbla "CLA_GERCOND_X" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf anorm 0.0)
  (cond
   (notrans
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf tmp 0.0)
```

```
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (cabs1
                        (*
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%)))))
           label100001))
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                tmp)
        (setf anorm (max anorm tmp))
       label100000)))
   (t
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf tmp 0.0)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (cabs1
                        (*
                         (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                        a-%offset%)
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%)))))
           label100003))
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                tmp)
        (setf anorm (max anorm tmp))
       label100002))))
  (cond ((= n 0) (setf cla_gercond_x 1.0) (go end_label))
        ((= anorm 0.0) (go end_label)))
  (setf ainvnm 0.0)
  (setf kase 0)
 label10
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (clacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ n 1))
                             ((1 *)) work-%offset%)
       work ainvnm kase isave)
    (declare (ignore var-1 var-2 var-5))
    (setf n var-0)
    (setf ainvnm var-3)
```

```
  (setf kase var-4))
(cond
 ((/= kase 0)
  (cond
   ((= kase 2)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (*
                 (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                rwork-%offset%)))
       label100004))
    (cond
     (notrans
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (cgetrs "No transpose" n 1 af ldaf ipiv work n info)
        (declare (ignore var-0 var-2 var-3 var-5 var-6))
        (setf n var-1)
        (setf ldaf var-4)
        (setf n var-7)
        (setf info var-8)))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (cgetrs "Conjugate transpose" n 1 af ldaf ipiv work n info)
        (declare (ignore var-0 var-2 var-3 var-5 var-6))
        (setf n var-1)
        (setf ldaf var-4)
        (setf n var-7)
        (setf info var-8))))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (/
                 (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)))
       label100005)))
   (t
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (/
                 (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)))
       label100006))
```

```
            (cond
             (notrans
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                  (cgetrs "Conjugate transpose" n 1 af ldaf ipiv work n info)
                (declare (ignore var-0 var-2 var-3 var-5 var-6))
                (setf n var-1)
                (setf ldaf var-4)
                (setf n var-7)
                (setf info var-8)))
             (t
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                  (cgetrs "No transpose" n 1 af ldaf ipiv work n info)
                (declare (ignore var-0 var-2 var-3 var-5 var-6))
                (setf n var-1)
                (setf ldaf var-4)
                (setf n var-7)
                (setf info var-8))))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                        (*
                         (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                         (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                        rwork-%offset%)))
               label100007))))
          (go label10)))
        (if (/= ainvnm 0.0)
            (setf cla_gercond_x (/ 1.0 ainvnm)))
        (go end_label)
       end_label
        (return
         (values cla_gercond_x trans n nil nil nil ldaf nil nil info nil
                 nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cla_gercond_x
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
```

```
                                        (fortran-to-lisp::integer4)
                                        (array fortran-to-lisp::integer4
                                         (*))
                                        (array fortran-to-lisp::complex8
                                         (*))
                                        (fortran-to-lisp::integer4)
                                        (array fortran-to-lisp::complex8
                                         (*))
                                        (array single-float (*)))
                                      :return-values
                                      '(fortran-to-lisp::trans
                                        fortran-to-lisp::n nil nil nil
                                        fortran-to-lisp::ldaf nil nil
                                        fortran-to-lisp::info nil nil)
                                      :calls
                                      '(fortran-to-lisp::cgetrs
                                        fortran-to-lisp::clacn2))))
```

———-

34.38 cla_gerfsx_extended

34.38.1 Axiom unit tests

— **clagerfsxextended.input** —

```
)set break resume
)sys rm -f clagerfsxextended.output
)spool clagerfsxextended.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.38.2 Axiom help page [Demm05]

— **cla_gerfsx_extended.help** —

```
==================================================================
```

```
cla_gerfsx_extended examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cla_gerfsx_extended (PREC_TYPE, TRANS_TYPE, N, NRHS, A, LDA,
           AF, LDAF, IPIV, COLEQU, C, B, LDB, Y, LDY, BERR_OUT, N_NORMS,
           ERRS_N, ERRS_C, RES, AYB, DY, Y_TAIL, RCOND, ITHRESH, RTHRESH,
           DZ_UB, IGNORE_CWISE, INFO)
           CLA_GERFSX_EXTENDED

Function/Subroutine Documentation
   subroutine cla_gerfsx_extended (integer PREC_TYPE, integer TRANS_TYPE,
       integer N, integer NRHS, complex, dimension( lda, * ) A, integer LDA,
       complex, dimension( ldaf, * ) AF, integer LDAF, integer, dimension( * )
       IPIV, logical COLEQU, real, dimension( * ) C, complex, dimension( ldb,
       * ) B, integer LDB, complex, dimension( ldy, * ) Y, integer LDY, real,
       dimension( * ) BERR_OUT, integer N_NORMS, real, dimension( nrhs, * )
       ERRS_N, real, dimension( nrhs, * ) ERRS_C, complex, dimension( * ) RES,
       real, dimension( * ) AYB, complex, dimension( * ) DY, complex,
       dimension( * ) Y_TAIL, real RCOND, integer ITHRESH, real RTHRESH, real
       DZ_UB, logical IGNORE_CWISE, integer INFO)
       CLA_GERFSX_EXTENDED

Purpose:
   CLA_GERFSX_EXTENDED improves the computed solution to a system of
   linear equations by performing extra-precise iterative refinement
   and provides error bounds and backward error estimates for the solution.
   This subroutine is called by CGERFSX to perform iterative refinement.
   In addition to normwise error bound, the code provides maximum
   componentwise error bound if possible. See comments for ERRS_N
   and ERRS_C for details of the error bounds. Note that this
   subroutine is only resonsible for setting the second fields of
   ERRS_N and ERRS_C.

Parameters:

 PREC_TYPE
   PREC_TYPE is INTEGER
   Specifies the intermediate precision to be used in refinement.
   The value is defined by ILAPREC(P) where P is a CHARACTER and
   P    = 'S':  Single
        = 'D':  Double
        = 'I':  Indigenous
        = 'X', 'E':  Extra
```

```
TRANS_TYPE
  TRANS_TYPE is INTEGER
  Specifies the transposition operation on A.
  The value is defined by ILATRANS(T) where T is a CHARACTER and
  T    = 'N':  No transpose
       = 'T':  Transpose
       = 'C':  Conjugate transpose

N
  N is INTEGER
  The number of linear equations, i.e., the order of the
  matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right-hand-sides, i.e., the number of columns of the
  matrix B.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the N-by-N matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX array, dimension (LDAF,N)
  The factors L and U from the factorization
  A = P*L*U as computed by CGETRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from the factorization A = P*L*U
  as computed by CGETRF; row i of the matrix was interchanged
  with row IPIV(i).

COLEQU
  COLEQU is LOGICAL
  If .TRUE. then column equilibration was done to A before calling
  this routine. This is needed to compute the solution and error
  bounds correctly.

C
  C is REAL array, dimension (N)
  The column scale factors for A. If COLEQU = .FALSE., C
```

```
  is not accessed. If C is input, each element of C should be a power
  of the radix to ensure a reliable solution and error estimates.
  Scaling by powers of the radix does not cause rounding errors unless
  the result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  The right-hand-side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

Y
  Y is COMPLEX array, dimension (LDY,NRHS)
  On entry, the solution matrix X, as computed by CGETRS.
  On exit, the improved solution matrix Y.

LDY
  LDY is INTEGER
  The leading dimension of the array Y.  LDY >= max(1,N).

BERR_OUT
  BERR_OUT is REAL array, dimension (NRHS)
  On exit, BERR_OUT(j) contains the componentwise relative backward
  error for right-hand-side j from the formula
  max(i) ( abs(RES(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
  where abs(Z) is the componentwise absolute value of the matrix
  or vector Z. This is computed by CLA_LIN_BERR.

N_NORMS
  N_NORMS is INTEGER
  Determines which error bounds to return (see ERRS_N
  and ERRS_C).
  If N_NORMS >= 1 return normwise error bounds.
  If N_NORMS >= 2 return componentwise error bounds.

ERRS_N
  ERRS_N is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
            max_j (abs(XTRUE(j,i) - X(j,i)))
            ------------------------------
                     max_j abs(X(j,i))
```

```
  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERRS_N(i,:) corresponds to the ith
  right-hand side.

  The second index in ERRS_N(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
          reciprocal condition number is less than the threshold
          sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
          almost certainly within a factor of 10 of the true error
          so long as the next entry is greater than the threshold
          sqrt(n) * slamch('Epsilon'). This error bound should only
          be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
           reciprocal condition number.  Compared with the threshold
           sqrt(n) * slamch('Epsilon') to determine if the error
           estimate is "guaranteed". These reciprocal condition
           numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
           appropriately scaled matrix Z.
           Let Z = S*A, where S scales each row by a power of the
           radix so all absolute row sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field
  above.

ERRS_C
  ERRS_C is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                  abs(XTRUE(j,i) - X(j,i))
            max_j ----------------------
                        abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERRS_C is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.
```

```
  The first index in ERRS_C(i,:) corresponds to the ith
  right-hand side.

  The second index in ERRS_C(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
          reciprocal condition number is less than the threshold
          sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
          almost certainly within a factor of 10 of the true error
          so long as the next entry is greater than the threshold
          sqrt(n) * slamch('Epsilon'). This error bound should only
          be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
           reciprocal condition number.  Compared with the threshold
           sqrt(n) * slamch('Epsilon') to determine if the error
           estimate is "guaranteed". These reciprocal condition
           numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
           appropriately scaled matrix Z.
           Let Z = S*(A*diag(x)), where x is the solution for the
           current right-hand side and S scales each row of
           A*diag(x) by a power of the radix so all absolute row
           sums of Z are approximately 1.

This subroutine is only responsible for setting the second field above.

RES
  RES is COMPLEX array, dimension (N)
  Workspace to hold the intermediate residual.

AYB
  AYB is REAL array, dimension (N)
  Workspace.

DY
  DY is COMPLEX array, dimension (N)
  Workspace to hold the intermediate solution.

Y_TAIL
  Y_TAIL is COMPLEX array, dimension (N)
  Workspace to hold the trailing bits of the intermediate solution.

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
```

```
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

ITHRESH
  ITHRESH is INTEGER
  The maximum number of residual computations allowed for
  refinement. The default is 10. For 'aggressive' set to 100 to
  permit convergence using approximate factorizations or
  factorizations other than LU. If the factorization uses a
  technique other than Gaussian elimination, the guarantees in
  ERRS_N and ERRS_C may no longer be trustworthy.

RTHRESH
  RTHRESH is REAL
  Determines when to stop refinement if the error estimate stops
  decreasing. Refinement will stop when the next solution no longer
  satisfies norm(dx_{i+1}) < RTHRESH * norm(dx_i) where norm(Z) is
  the infinity norm of Z. RTHRESH satisfies 0 < RTHRESH <= 1. The
  default value is 0.5. For 'aggressive' set to 0.9 to permit
  convergence on extremely ill-conditioned matrices. See LAWN 165
  for more details.

DZ_UB
  DZ_UB is REAL
  Determines when to start considering componentwise convergence.
  Componentwise convergence is only considered after each component
  of the solution Y is stable, which we definte as the relative
  change in each component being less than DZ_UB. The default value
  is 0.25, requiring the first bit to be stable. See LAWN 165 for
  more details.

IGNORE_CWISE
  IGNORE_CWISE is LOGICAL
  If .TRUE. then ignore componentwise convergence. Default value
  is .FALSE..

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  < 0:  if INFO = -i, the ith argument to CGETRS had an illegal value
```

———-

34.38.3 fortran code

— cla_gerfsx_extended.f —

```
*  =====================================================================
      SUBROUTINE CLA_GERFSX_EXTENDED( PREC_TYPE, TRANS_TYPE, N, NRHS, A,
     $                                LDA, AF, LDAF, IPIV, COLEQU, C, B,
     $                                LDB, Y, LDY, BERR_OUT, N_NORMS,
     $                                ERRS_N, ERRS_C, RES, AYB, DY,
     $                                Y_TAIL, RCOND, ITHRESH, RTHRESH,
     $                                DZ_UB, IGNORE_CWISE, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDAF, LDB, LDY, N, NRHS, PREC_TYPE,
     $                   TRANS_TYPE, N_NORMS
      LOGICAL            COLEQU, IGNORE_CWISE
      INTEGER            ITHRESH
      REAL               RTHRESH, DZ_UB
*     ..
*     .. Array Arguments
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   Y( LDY, * ), RES( * ), DY( * ), Y_TAIL( * )
      REAL               C( * ), AYB( * ), RCOND, BERR_OUT( * ),
     $                   ERRS_N( NRHS, * ), ERRS_C( NRHS, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      CHARACTER          TRANS
      INTEGER            CNT, I, J,  X_STATE, Z_STATE, Y_PREC_STATE
      REAL               YK, DYK, YMIN, NORMY, NORMX, NORMDX, DXRAT,
     $                   DZRAT, PREVNORMDX, PREV_DZ_Z, DXRATMAX,
     $                   DZRATMAX, DX_X, DZ_Z, FINAL_DX_X, FINAL_DZ_Z,
     $                   EPS, HUGEVAL, INCR_THRESH
      LOGICAL            INCR_PREC
      COMPLEX            ZDUM
*     ..
*     .. Parameters ..
      INTEGER            UNSTABLE_STATE, WORKING_STATE, CONV_STATE,
     $                   NOPROG_STATE, BASE_RESIDUAL, EXTRA_RESIDUAL,
     $                   EXTRA_Y
      PARAMETER          ( UNSTABLE_STATE = 0, WORKING_STATE = 1,
     $                   CONV_STATE = 2,
     $                   NOPROG_STATE = 3 )
      PARAMETER          ( BASE_RESIDUAL = 0, EXTRA_RESIDUAL = 1,
     $                   EXTRA_Y = 2 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
```

```
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           CAXPY, CCOPY, CGETRS, CGEMV, BLAS_CGEMV_X,
     $                   BLAS_CGEMV2_X, CLA_GEAMV, CLA_WWADDW, SLAMCH,
     $                   CHLA_TRANSTYPE, CLA_LIN_BERR
      REAL               SLAMCH
      CHARACTER          CHLA_TRANSTYPE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      IF ( INFO.NE.0 ) RETURN
      TRANS = CHLA_TRANSTYPE(TRANS_TYPE)
      EPS = SLAMCH( 'Epsilon' )
      HUGEVAL = SLAMCH( 'Overflow' )
*     Force HUGEVAL to Inf
      HUGEVAL = HUGEVAL * HUGEVAL
*     Using HUGEVAL may lead to spurious underflows.
      INCR_THRESH = REAL( N ) * EPS
*
      DO J = 1, NRHS
         Y_PREC_STATE = EXTRA_RESIDUAL
         IF ( Y_PREC_STATE .EQ. EXTRA_Y ) THEN
            DO I = 1, N
               Y_TAIL( I ) = 0.0
            END DO
```

```
         END IF

         DXRAT = 0.0
         DXRATMAX = 0.0
         DZRAT = 0.0
         DZRATMAX = 0.0
         FINAL_DX_X = HUGEVAL
         FINAL_DZ_Z = HUGEVAL
         PREVNORMDX = HUGEVAL
         PREV_DZ_Z = HUGEVAL
         DZ_Z = HUGEVAL
         DX_X = HUGEVAL

         X_STATE = WORKING_STATE
         Z_STATE = UNSTABLE_STATE
         INCR_PREC = .FALSE.

         DO CNT = 1, ITHRESH
*
*         Compute residual RES = B_s - op(A_s) * Y,
*             op(A) = A, A**T, or A**H depending on TRANS (and type).
*
           CALL CCOPY( N, B( 1, J ), 1, RES, 1 )
           IF ( Y_PREC_STATE .EQ. BASE_RESIDUAL ) THEN
              CALL CGEMV( TRANS, N, N, (-1.0E+0,0.0E+0), A, LDA,
     $             Y( 1, J ), 1, (1.0E+0,0.0E+0), RES, 1)
           ELSE IF (Y_PREC_STATE .EQ. EXTRA_RESIDUAL) THEN
              CALL BLAS_CGEMV_X( TRANS_TYPE, N, N, (-1.0E+0,0.0E+0), A,
     $             LDA, Y( 1, J ), 1, (1.0E+0,0.0E+0),
     $             RES, 1, PREC_TYPE )
           ELSE
              CALL BLAS_CGEMV2_X( TRANS_TYPE, N, N, (-1.0E+0,0.0E+0),
     $             A, LDA, Y(1, J), Y_TAIL, 1, (1.0E+0,0.0E+0), RES, 1,
     $             PREC_TYPE)
           END IF

!         XXX: RES is no longer needed.
           CALL CCOPY( N, RES, 1, DY, 1 )
           CALL CGETRS( TRANS, N, 1, AF, LDAF, IPIV, DY, N, INFO )
*
*         Calculate relative changes DX_X, DZ_Z and ratios DXRAT, DZRAT.
*
           NORMX = 0.0E+0
           NORMY = 0.0E+0
           NORMDX = 0.0E+0
           DZ_Z = 0.0E+0
           YMIN = HUGEVAL
*
           DO I = 1, N
              YK = CABS1( Y( I, J ) )
```

```
               DYK = CABS1( DY( I ) )

               IF ( YK .NE. 0.0E+0 ) THEN
                  DZ_Z = MAX( DZ_Z, DYK / YK )
               ELSE IF ( DYK .NE. 0.0 ) THEN
                  DZ_Z = HUGEVAL
               END IF

               YMIN = MIN( YMIN, YK )

               NORMY = MAX( NORMY, YK )

               IF ( COLEQU ) THEN
                  NORMX = MAX( NORMX, YK * C( I ) )
                  NORMDX = MAX( NORMDX, DYK * C( I ) )
               ELSE
                  NORMX = NORMY
                  NORMDX = MAX(NORMDX, DYK)
               END IF
            END DO

            IF ( NORMX .NE. 0.0 ) THEN
               DX_X = NORMDX / NORMX
            ELSE IF ( NORMDX .EQ. 0.0 ) THEN
               DX_X = 0.0
            ELSE
               DX_X = HUGEVAL
            END IF

            DXRAT = NORMDX / PREVNORMDX
            DZRAT = DZ_Z / PREV_DZ_Z
*
*         Check termination criteria
*
            IF (.NOT.IGNORE_CWISE
     $           .AND. YMIN*RCOND .LT. INCR_THRESH*NORMY
     $           .AND. Y_PREC_STATE .LT. EXTRA_Y )
     $           INCR_PREC = .TRUE.

            IF ( X_STATE .EQ. NOPROG_STATE .AND. DXRAT .LE. RTHRESH )
     $           X_STATE = WORKING_STATE
            IF ( X_STATE .EQ. WORKING_STATE ) THEN
               IF (DX_X .LE. EPS) THEN
                  X_STATE = CONV_STATE
               ELSE IF ( DXRAT .GT. RTHRESH ) THEN
                  IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                     INCR_PREC = .TRUE.
                  ELSE
                     X_STATE = NOPROG_STATE
                  END IF
```

```
            ELSE
               IF ( DXRAT .GT. DXRATMAX ) DXRATMAX = DXRAT
            END IF
            IF ( X_STATE .GT. WORKING_STATE ) FINAL_DX_X = DX_X
         END IF

         IF ( Z_STATE .EQ. UNSTABLE_STATE .AND. DZ_Z .LE. DZ_UB )
     $        Z_STATE = WORKING_STATE
         IF ( Z_STATE .EQ. NOPROG_STATE .AND. DZRAT .LE. RTHRESH )
     $        Z_STATE = WORKING_STATE
         IF ( Z_STATE .EQ. WORKING_STATE ) THEN
            IF ( DZ_Z .LE. EPS ) THEN
               Z_STATE = CONV_STATE
            ELSE IF ( DZ_Z .GT. DZ_UB ) THEN
               Z_STATE = UNSTABLE_STATE
               DZRATMAX = 0.0
               FINAL_DZ_Z = HUGEVAL
            ELSE IF ( DZRAT .GT. RTHRESH ) THEN
               IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                  INCR_PREC = .TRUE.
               ELSE
                  Z_STATE = NOPROG_STATE
               END IF
            ELSE
               IF ( DZRAT .GT. DZRATMAX ) DZRATMAX = DZRAT
            END IF
            IF ( Z_STATE .GT. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
         END IF
*
*        Exit if both normwise and componentwise stopped working,
*        but if componentwise is unstable, let it go at least two
*        iterations.
*
         IF ( X_STATE.NE.WORKING_STATE ) THEN
            IF ( IGNORE_CWISE ) GOTO 666
            IF ( Z_STATE.EQ.NOPROG_STATE .OR. Z_STATE.EQ.CONV_STATE )
     $         GOTO 666
            IF ( Z_STATE.EQ.UNSTABLE_STATE .AND. CNT.GT.1 ) GOTO 666
         END IF

         IF ( INCR_PREC ) THEN
            INCR_PREC = .FALSE.
            Y_PREC_STATE = Y_PREC_STATE + 1
            DO I = 1, N
               Y_TAIL( I ) = 0.0
            END DO
         END IF

         PREVNORMDX = NORMDX
         PREV_DZ_Z = DZ_Z
```

```
*
*          Update soluton.
*
           IF ( Y_PREC_STATE .LT. EXTRA_Y ) THEN
              CALL CAXPY( N, (1.0E+0,0.0E+0), DY, 1, Y(1,J), 1 )
           ELSE
              CALL CLA_WWADDW( N, Y( 1, J ), Y_TAIL, DY )
           END IF

        END DO
*       Target of "IF (Z_STOP .AND. X_STOP)".  Sun's f77 won't EXIT.
 666    CONTINUE
*
*     Set final_* when cnt hits ithresh
*
        IF ( X_STATE .EQ. WORKING_STATE ) FINAL_DX_X = DX_X
        IF ( Z_STATE .EQ. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
*
*     Compute error bounds
*
        IF (N_NORMS .GE. 1) THEN
           ERRS_N( J, LA_LINRX_ERR_I ) = FINAL_DX_X / (1 - DXRATMAX)

        END IF
        IF ( N_NORMS .GE. 2 ) THEN
           ERRS_C( J, LA_LINRX_ERR_I ) = FINAL_DZ_Z / (1 - DZRATMAX)
        END IF
*
*     Compute componentwise relative backward error from formula
*         max(i) ( abs(R(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
*     where abs(Z) is the componentwise absolute value of the matrix
*     or vector Z.
*
*       Compute residual RES = B_s - op(A_s) * Y,
*           op(A) = A, A**T, or A**H depending on TRANS (and type).
*
        CALL CCOPY( N, B( 1, J ), 1, RES, 1 )
        CALL CGEMV( TRANS, N, N, (-1.0E+0,0.0E+0), A, LDA, Y(1,J), 1,
     $        (1.0E+0,0.0E+0), RES, 1 )

        DO I = 1, N
           AYB( I ) = CABS1( B( I, J ) )
        END DO
*
*     Compute abs(op(A_s))*abs(Y) + abs(B_s).
*
        CALL CLA_GEAMV ( TRANS_TYPE, N, N, 1.0E+0,
     $        A, LDA, Y(1, J), 1, 1.0E+0, AYB, 1 )

        CALL CLA_LIN_BERR ( N, N, 1, RES, AYB, BERR_OUT( J ) )
```

```
*
*     End of loop for each RHS.
*
      END DO
*
      RETURN
      END
```

———-

34.38.4 lisp code

— **cla_gerfsx_extended.lisp** —

```
(let* ((unstable_state 0)
       (working_state 1)
       (conv_state 2)
       (noprog_state 3)
       (base_residual 0)
       (extra_residual 1)
       (extra_y 2)
       (la_linrx_err_i 2))
  (declare (type (f2cl-lib:integer4 0 0) unstable_state)
           (type (f2cl-lib:integer4 1 1) working_state)
           (type (f2cl-lib:integer4 2 2) conv_state)
           (type (f2cl-lib:integer4 3 3) noprog_state)
           (type (f2cl-lib:integer4 0 0) base_residual)
           (type (f2cl-lib:integer4 1 1) extra_residual)
           (type (f2cl-lib:integer4 2 2) extra_y)
           (type (f2cl-lib:integer4 2 2) la_linrx_err_i)
           (ignorable unstable_state working_state conv_state noprog_state
            base_residual extra_residual extra_y la_linrx_err_i))
  (defun cla_gerfsx_extended
         (prec_type trans_type n nrhs a lda af ldaf ipiv colequ c b ldb$ y ldy
          berr_out n_norms errs_n errs_c res ayb dy y_tail rcond ithresh
          rthresh dz_ub ignore_cwise info)
    (declare (type (single-float) dz_ub rthresh rcond)
             (type (array single-float (*)) ayb errs_c errs_n berr_out c)
             (type f2cl-lib:logical ignore_cwise colequ)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) y_tail dy res y b af a)
             (type (f2cl-lib:integer4) info ithresh n_norms ldy ldb$ ldaf lda
              nrhs n trans_type prec_type))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (af f2cl-lib:complex8 af-%data% af-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (y f2cl-lib:complex8 y-%data% y-%offset%)
```

```
    (res f2cl-lib:complex8 res-%data% res-%offset%)
    (dy f2cl-lib:complex8 dy-%data% dy-%offset%)
    (y_tail f2cl-lib:complex8 y_tail-%data% y_tail-%offset%)
    (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
    (c single-float c-%data% c-%offset%)
    (berr_out single-float berr_out-%data% berr_out-%offset%)
    (errs_n single-float errs_n-%data% errs_n-%offset%)
    (errs_c single-float errs_c-%data% errs_c-%offset%)
    (ayb single-float ayb-%data% ayb-%offset%))
  (labels ((cabs1 (zdum)
             (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
    (declare
     (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
      cabs1))
    (prog ((zdum #C(0.0 0.0)) (incr_prec nil) (yk 0.0) (dyk 0.0) (ymin 0.0)
           (normy 0.0) (normx 0.0) (normdx 0.0) (dxrat 0.0) (dzrat 0.0)
           (prevnormdx 0.0) (prev_dz_z 0.0) (dxratmax 0.0) (dzratmax 0.0)
           (dx_x 0.0) (dz_z 0.0) (final_dx_x 0.0) (final_dz_z 0.0)
           (eps 0.0) (hugeval 0.0) (incr_thresh 0.0) (cnt 0) (i 0) (j 0)
           (x_state 0) (z_state 0) (y_prec_state 0)
           (trans
            (make-array '(1) :element-type 'character :initial-element
                        #\ )))
     (declare (type (f2cl-lib:complex8) zdum)
              (type f2cl-lib:logical incr_prec)
              (type (single-float) yk dyk ymin normy normx normdx dxrat
               dzrat prevnormdx prev_dz_z dxratmax dzratmax dx_x dz_z
               final_dx_x final_dz_z eps hugeval incr_thresh)
              (type (f2cl-lib:integer4) cnt i j x_state z_state
               y_prec_state)
              (type (string 1) trans))
     (if (/= info 0)
         (go end_label))
     (f2cl-lib:f2cl-set-string trans
                               (f2cl-lib:fref-string #'chla_transtype
                                                     (trans_type))
                               (string 1))
     (setf eps (slamch "Epsilon"))
     (setf hugeval (slamch "Overflow"))
     (setf hugeval (* hugeval hugeval))
     (setf incr_thresh (* (f2cl-lib:freal n) eps))
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j nrhs) nil)
       (tagbody
         (setf y_prec_state extra_residual)
         (cond
          ((= y_prec_state extra_y)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
```

```
      (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                           y_tail-%offset%)
              (coerce 0.0 'f2cl-lib:complex8))
     label100001))))
(setf dxrat 0.0)
(setf dxratmax 0.0)
(setf dzrat 0.0)
(setf dzratmax 0.0)
(setf final_dx_x hugeval)
(setf final_dz_z hugeval)
(setf prevnormdx hugeval)
(setf prev_dz_z hugeval)
(setf dz_z hugeval)
(setf dx_x hugeval)
(setf x_state working_state)
(setf z_state unstable_state)
(setf incr_prec f2cl-lib:%false%)
(f2cl-lib:fdo (cnt 1 (f2cl-lib:int-add cnt 1))
              ((> cnt ithresh) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (ccopy n
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1 res 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (cond
     ((= y_prec_state base_residual)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (cgemv trans n n (f2cl-lib:cmplx -1.0 0.0) a lda
           (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                                 ((1 ldy) (1 *)) y-%offset%)
           1 (f2cl-lib:cmplx 1.0 0.0) res 1)
        (declare
         (ignore var-3 var-4 var-6 var-7 var-8 var-9 var-10))
        (when var-0 (setf trans var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-5 (setf lda var-5))))
     ((= y_prec_state extra_residual)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11)
          (blas_cgemv_x trans_type n n (f2cl-lib:cmplx -1.0 0.0)
           a lda
           (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                                 ((1 ldy) (1 *)) y-%offset%)
```

```
      1 (f2cl-lib:cmplx 1.0 0.0) res 1 prec_type)
    (declare
     (ignore var-3 var-4 var-6 var-7 var-8 var-9 var-10))
    (when var-0 (setf trans_type var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-5 (setf lda var-5))
    (when var-11 (setf prec_type var-11))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (blas_cgemv2_x trans_type n n (f2cl-lib:cmplx -1.0 0.0)
       a lda
       (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                             ((1 ldy) (1 *)) y-%offset%)
       y_tail 1 (f2cl-lib:cmplx 1.0 0.0) res 1 prec_type)
    (declare
     (ignore var-3 var-4 var-6 var-7 var-8 var-9 var-10
      var-11))
    (when var-0 (setf trans_type var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-5 (setf lda var-5))
    (when var-12 (setf prec_type var-12)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (ccopy n res 1 dy 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cgetrs trans n 1 af ldaf ipiv dy n info)
  (declare (ignore var-2 var-3 var-5 var-6))
  (setf trans var-0)
  (setf n var-1)
  (setf ldaf var-4)
  (setf n var-7)
  (setf info var-8))
(setf normx 0.0)
(setf normy 0.0)
(setf normdx 0.0)
(setf dz_z 0.0)
(setf ymin hugeval)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf yk
            (cabs1
             (f2cl-lib:fref y-%data% (i j) ((1 ldy) (1 *))
                            y-%offset%)))
```

```
    (setf dyk
            (cabs1
             (f2cl-lib:fref dy-%data% (i) ((1 *))
                            dy-%offset%)))
    (cond ((/= yk 0.0) (setf dz_z (max dz_z (/ dyk yk))))
          ((/= dyk 0.0) (setf dz_z hugeval)))
    (setf ymin (min ymin yk))
    (setf normy (max normy yk))
    (cond
     (colequ
      (setf normx
              (max normx
                   (* yk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%))))
      (setf normdx
              (max normdx
                   (* dyk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%)))))
     (t (setf normx normy) (setf normdx (max normdx dyk))))
   label100003))
(cond ((/= normx 0.0) (setf dx_x (/ normdx normx)))
      ((= normdx 0.0) (setf dx_x 0.0))
      (t (setf dx_x hugeval)))
(setf dxrat (/ normdx prevnormdx))
(setf dzrat (/ dz_z prev_dz_z))
(if (and (not ignore_cwise)
         (< (* ymin rcond) (* incr_thresh normy))
         (< y_prec_state extra_y))
    (setf incr_prec f2cl-lib:%true%))
(if (and (= x_state noprog_state) (<= dxrat rthresh))
    (setf x_state working_state))
(cond
 ((= x_state working_state)
  (cond ((<= dx_x eps) (setf x_state conv_state))
        ((> dxrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf x_state noprog_state))))
        (t
         (if (> dxrat dxratmax)
             (setf dxratmax dxrat))))
  (if (> x_state working_state)
      (setf final_dx_x dx_x))))
(if (and (= z_state unstable_state) (<= dz_z dz_ub))
    (setf z_state working_state))
(if (and (= z_state noprog_state) (<= dzrat rthresh))
    (setf z_state working_state))
```

```
(cond
 ((= z_state working_state)
  (cond ((<= dz_z eps) (setf z_state conv_state))
        ((> dz_z dz_ub) (setf z_state unstable_state)
         (setf dzratmax 0.0) (setf final_dz_z hugeval))
        ((> dzrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf z_state noprog_state))))
        (t
         (if (> dzrat dzratmax)
             (setf dzratmax dzrat))))
  (if (> z_state working_state)
      (setf final_dz_z dz_z))))
(cond
 ((/= x_state working_state)
  (if ignore_cwise
      (go label666))
  (if (or (= z_state noprog_state) (= z_state conv_state))
      (go label666))
  (if (and (= z_state unstable_state) (> cnt 1))
      (go label666))))
(cond
 (incr_prec (setf incr_prec f2cl-lib:%false%)
  (setf y_prec_state (f2cl-lib:int-add y_prec_state 1))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                           y_tail-%offset%)
              (coerce 0.0 'f2cl-lib:complex8))
     label100004))))
(setf prevnormdx normdx)
(setf prev_dz_z dz_z)
(cond
 ((< y_prec_state extra_y)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (caxpy n (f2cl-lib:cmplx 1.0 0.0) dy 1
       (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                             ((1 ldy) (1 *)) y-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (cla_wwaddw n
       (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                             ((1 ldy) (1 *)) y-%offset%)
       y_tail dy)
```

```
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf n var-0)))))
   label100002))
label666
 (if (= x_state working_state)
     (setf final_dx_x dx_x))
 (if (= z_state working_state)
     (setf final_dz_z dz_z))
 (cond
  ((>= n_norms 1)
   (setf (f2cl-lib:fref errs_n-%data% (j la_linrx_err_i)
                        ((1 nrhs) (1 *)) errs_n-%offset%)
          (/ final_dx_x (- 1 dxratmax)))))
 (cond
  ((>= n_norms 2)
   (setf (f2cl-lib:fref errs_c-%data% (j la_linrx_err_i)
                        ((1 nrhs) (1 *)) errs_c-%offset%)
          (/ final_dz_z (- 1 dzratmax)))))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (ccopy n
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 j)
                            ((1 ldb$) (1 *)) b-%offset%)
      1 res 1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf n var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (cgemv trans n n (f2cl-lib:cmplx -1.0 0.0) a lda
      (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                            ((1 ldy) (1 *)) y-%offset%)
      1 (f2cl-lib:cmplx 1.0 0.0) res 1)
   (declare (ignore var-3 var-4 var-6 var-7 var-8 var-9 var-10))
   (when var-0 (setf trans var-0))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-5 (setf lda var-5)))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf (f2cl-lib:fref ayb-%data% (i) ((1 *)) ayb-%offset%)
             (cabs1
              (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)))
   label100005))
 (cla_geamv trans_type n n 1.0 a lda
  (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                        ((1 ldy) (1 *)) y-%offset%)
  1 1.0 ayb 1)
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
```

```
                  (cla_lin_berr n n 1 res ayb
                   (f2cl-lib:array-slice berr_out-%data% single-float (j)
                                         ((1 *)) berr_out-%offset%))
                (declare (ignore var-2 var-3 var-4 var-5))
                (when var-0 (setf n var-0))
                (when var-1 (setf n var-1)))
             label100000))
          (go end_label)
         end_label
          (return
           (values prec_type trans_type n nil nil lda nil ldaf nil nil nil nil
                   nil nil nil nil nil nil nil nil nil nil nil nil nil nil
                   nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cla_gerfsx_extended
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              fortran-to-lisp::logical
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
```

```
   (*))
  (single-float)
  (fortran-to-lisp::integer4)
  (single-float) (single-float)
  fortran-to-lisp::logical
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::prec_type
  fortran-to-lisp::trans_type
  fortran-to-lisp::n nil nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldaf nil nil nil
  nil nil nil nil nil nil nil nil
  nil nil nil nil nil nil nil nil
  nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::cla_geamv
  fortran-to-lisp::cgetrs))))
```

———-

34.39 cla_gerpvgrw

34.39.1 Axiom unit tests

— clagerpvgrw.input —

```
)set break resume
)sys rm -f clagerpvgrw.output
)spool clagerpvgrw.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.39.2 Axiom help page

— cla_gerpvgrw.help —

```
====================================================================
cla_gerpvgrw examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       real function cla_gerpvgrw (N, NCOLS, A, LDA, AF, LDAF)
           CLA_GERPVGRW multiplies a square real matrix by a complex matrix.

Function/Subroutine Documentation
   real function cla_gerpvgrw (integer N, integer NCOLS, complex, dimension(
       lda, * ) A, integer LDA, complex, dimension( ldaf, * ) AF, integer
       LDAF)
       CLA_GERPVGRW multiplies a square real matrix by a complex matrix.

Purpose:
   CLA_GERPVGRW computes the reciprocal pivot growth factor
   norm(A)/norm(U). The "max absolute element" norm is used. If this is
   much less than 1, the stability of the LU factorization of the
   (equilibrated) matrix A could be poor. This also means that the
   solution X, estimated condition numbers, and error bounds could be
   unreliable.

Parameters:

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NCOLS
   NCOLS is INTEGER
   The number of columns of the matrix A. NCOLS >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is COMPLEX array, dimension (LDAF,N)
   The factors L and U from the factorization
   A = P*L*U as computed by CGETRF.
```

```
 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).
```

34.39.3 fortran code

— cla_gerpvgrw.f —

```
*  =====================================================================
      REAL FUNCTION CLA_GERPVGRW( N, NCOLS, A, LDA, AF, LDAF )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            N, NCOLS, LDA, LDAF
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), AF( LDAF, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               AMAX, UMAX, RPVGRW
      COMPLEX            ZDUM
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, ABS, REAL, AIMAG
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      RPVGRW = 1.0

      DO J = 1, NCOLS
         AMAX = 0.0
```

```
         UMAX = 0.0
         DO I = 1, N
            AMAX = MAX( CABS1( A( I, J ) ), AMAX )
         END DO
         DO I = 1, J
            UMAX = MAX( CABS1( AF( I, J ) ), UMAX )
         END DO
         IF ( UMAX /= 0.0 ) THEN
            RPVGRW = MIN( AMAX / UMAX, RPVGRW )
         END IF
      END DO
      CLA_GERPVGRW = RPVGRW
      END
```

———-

34.39.4 lisp code

— cla_gerpvgrw.lisp —

```
(defun cla_gerpvgrw (n ncols a lda af ldaf)
  (declare (type (array f2cl-lib:complex8 (*)) af a)
           (type (f2cl-lib:integer4) ldaf lda ncols n))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex8 a-%data% a-%offset%)
       (af f2cl-lib:complex8 af-%data% af-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
        cabs1))
      (prog ((zdum #C(0.0 0.0)) (amax 0.0) (umax 0.0) (rpvgrw 0.0) (i 0) (j 0)
             (cla_gerpvgrw 0.0))
        (declare (type (f2cl-lib:integer4) j i)
                 (type (single-float) cla_gerpvgrw rpvgrw umax amax)
                 (type (f2cl-lib:complex8) zdum))
        (setf rpvgrw 1.0)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j ncols) nil)
          (tagbody
            (setf amax 0.0)
            (setf umax 0.0)
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf amax
                        (max
                         (cabs1
```

```
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))
                      amax))
             label100001))
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
             (tagbody
               (setf umax
                       (max
                        (cabs1
                         (f2cl-lib:fref af-%data% (i j) ((1 ldaf) (1 *))
                                       af-%offset%))
                        umax))
             label100002))
           (cond
            ((f2cl-lib:f2cl/ umax (= 0.0))
             (setf rpvgrw (min (/ amax umax) rpvgrw))))
          label100000))
       (setf cla_gerpvgrw rpvgrw)
      end_label
       (return (values cla_gerpvgrw nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cla_gerpvgrw
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(nil nil nil nil nil nil) :calls
                                           'nil)))
```

———-

34.40 ctgevc

34.40.1 Axiom unit tests

— ctgevc.input —

```
)set break resume
)sys rm -f ctgevc.output
)spool ctgevc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.40.2 Axiom help page

— ctgevc.help —

```
=====================================================================
ctgevc examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ctgevc (SIDE, HOWMNY, SELECT, N, S, LDS, P, LDP, VL, LDVL,
           VR, LDVR, MM, M, WORK, RWORK, INFO)
           CTGEVC

Function/Subroutine Documentation
   subroutine ctgevc (character SIDE, character HOWMNY, logical, dimension( *
       ) SELECT, integer N, complex, dimension( lds, * ) S, integer LDS,
       complex, dimension( ldp, * ) P, integer LDP, complex, dimension( ldvl,
       * ) VL, integer LDVL, complex, dimension( ldvr, * ) VR, integer LDVR,
       integer MM, integer M, complex, dimension( * ) WORK, real, dimension( *
       ) RWORK, integer INFO)
       CTGEVC

Purpose:
```

```
   CTGEVC computes some or all of the right and/or left eigenvectors of
   a pair of complex matrices (S,P), where S and P are upper triangular.
   Matrix pairs of this type are produced by the generalized Schur
   factorization of a complex matrix pair (A,B):

      A = Q*S*Z**H,  B = Q*P*Z**H

   as computed by CGGHRD + CHGEQZ.

   The right eigenvector x and the left eigenvector y of (S,P)
   corresponding to an eigenvalue w are defined by:

      S*x = w*P*x,  (y**H)*S = w*(y**H)*P,

   where y**H denotes the conjugate tranpose of y.
   The eigenvalues are not input to this routine, but are computed
   directly from the diagonal elements of S and P.

   This routine returns the matrices X and/or Y of right and left
   eigenvectors of (S,P), or the products Z*X and/or Q*Y,
   where Z and Q are input matrices.
   If Q and Z are the unitary factors from the generalized Schur
   factorization of a matrix pair (A,B), then Z*X and Q*Y
   are the matrices of right and left eigenvectors of (A,B).

Parameters:

 SIDE
   SIDE is CHARACTER*1
   = 'R': compute right eigenvectors only;
   = 'L': compute left eigenvectors only;
   = 'B': compute both right and left eigenvectors.

 HOWMNY
   HOWMNY is CHARACTER*1
   = 'A': compute all right and/or left eigenvectors;
   = 'B': compute all right and/or left eigenvectors,
          backtransformed by the matrices in VR and/or VL;
   = 'S': compute selected right and/or left eigenvectors,
          specified by the logical array SELECT.

 SELECT
   SELECT is LOGICAL array, dimension (N)
   If HOWMNY='S', SELECT specifies the eigenvectors to be
   computed.  The eigenvector corresponding to the j-th
   eigenvalue is computed if SELECT(j) = .TRUE..
   Not referenced if HOWMNY = 'A' or 'B'.

 N
   N is INTEGER
```

```
  The order of the matrices S and P.  N >= 0.

S
  S is COMPLEX array, dimension (LDS,N)
  The upper triangular matrix S from a generalized Schur
  factorization, as computed by CHGEQZ.

LDS
  LDS is INTEGER
  The leading dimension of array S.  LDS >= max(1,N).

P
  P is COMPLEX array, dimension (LDP,N)
  The upper triangular matrix P from a generalized Schur
  factorization, as computed by CHGEQZ.  P must have real diagonal elements.

LDP
  LDP is INTEGER
  The leading dimension of array P.  LDP >= max(1,N).

VL
  VL is COMPLEX array, dimension (LDVL,MM)
  On entry, if SIDE = 'L' or 'B' and HOWMNY = 'B', VL must
  contain an N-by-N matrix Q (usually the unitary matrix Q
  of left Schur vectors returned by CHGEQZ).
  On exit, if SIDE = 'L' or 'B', VL contains:
  if HOWMNY = 'A', the matrix Y of left eigenvectors of (S,P);
  if HOWMNY = 'B', the matrix Q*Y;
  if HOWMNY = 'S', the left eigenvectors of (S,P) specified by
                   SELECT, stored consecutively in the columns of
                   VL, in the same order as their eigenvalues.
  Not referenced if SIDE = 'R'.

LDVL
  LDVL is INTEGER
  The leading dimension of array VL.  LDVL >= 1, and if
  SIDE = 'L' or 'l' or 'B' or 'b', LDVL >= N.

VR
  VR is COMPLEX array, dimension (LDVR,MM)
  On entry, if SIDE = 'R' or 'B' and HOWMNY = 'B', VR must
  contain an N-by-N matrix Q (usually the unitary matrix Z
  of right Schur vectors returned by CHGEQZ).
  On exit, if SIDE = 'R' or 'B', VR contains:
  if HOWMNY = 'A', the matrix X of right eigenvectors of (S,P);
  if HOWMNY = 'B', the matrix Z*X;
  if HOWMNY = 'S', the right eigenvectors of (S,P) specified by
                   SELECT, stored consecutively in the columns of
                   VR, in the same order as their eigenvalues.
  Not referenced if SIDE = 'L'.
```

```
LDVR
  LDVR is INTEGER
  The leading dimension of the array VR.  LDVR >= 1, and if
  SIDE = 'R' or 'B', LDVR >= N.

MM
  MM is INTEGER
  The number of columns in the arrays VL and/or VR. MM >= M.

M
  M is INTEGER
  The number of columns in the arrays VL and/or VR actually
  used to store the eigenvectors.  If HOWMNY = 'A' or 'B', M
  is set to N.  Each selected eigenvector occupies one column.

WORK
  WORK is COMPLEX array, dimension (2*N)

RWORK
  RWORK is REAL array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

———-

34.40.3 fortran code

— ctgevc.f —

```
*  =====================================================================
      SUBROUTINE CTGEVC( SIDE, HOWMNY, SELECT, N, S, LDS, P, LDP, VL,
     $                   LDVL, VR, LDVR, MM, M, WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          HOWMNY, SIDE
      INTEGER            INFO, LDP, LDS, LDVL, LDVR, M, MM, N
*     ..
*     .. Array Arguments ..
```

```
      LOGICAL            SELECT( * )
      REAL               RWORK( * )
      COMPLEX            P( LDP, * ), S( LDS, * ), VL( LDVL, * ),
     $                   VR( LDVR, * ), WORK( * )
*     ..
*
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      COMPLEX            CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0E+0, 0.0E+0 ),
     $                   CONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            COMPL, COMPR, ILALL, ILBACK, ILBBAD, ILCOMP,
     $                   LSA, LSB
      INTEGER            I, IBEG, IEIG, IEND, IHWMNY, IM, ISIDE, ISRC,
     $                   J, JE, JR
      REAL               ACOEFA, ACOEFF, ANORM, ASCALE, BCOEFA, BIG,
     $                   BIGNUM, BNORM, BSCALE, DMIN, SAFMIN, SBETA,
     $                   SCALE, SMALL, TEMP, ULP, XMAX
      COMPLEX            BCOEFF, CA, CB, D, SALPHA, SUM, SUMA, SUMB, X
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH
      COMPLEX            CLADIV
      EXTERNAL           LSAME, SLAMCH, CLADIV
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEMV, SLABAD, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, CMPLX, CONJG, MAX, MIN, REAL
*     ..
*     .. Statement Functions ..
      REAL               ABS1
*     ..
*     .. Statement Function definitions ..
      ABS1( X ) = ABS( REAL( X ) ) + ABS( AIMAG( X ) )
*     ..
*     .. Executable Statements ..
*
*     Decode and Test the input parameters
*
      IF( LSAME( HOWMNY, 'A' ) ) THEN
         IHWMNY = 1
```

```
         ILALL = .TRUE.
         ILBACK = .FALSE.
      ELSE IF( LSAME( HOWMNY, 'S' ) ) THEN
         IHWMNY = 2
         ILALL = .FALSE.
         ILBACK = .FALSE.
      ELSE IF( LSAME( HOWMNY, 'B' ) ) THEN
         IHWMNY = 3
         ILALL = .TRUE.
         ILBACK = .TRUE.
      ELSE
         IHWMNY = -1
      END IF
*
      IF( LSAME( SIDE, 'R' ) ) THEN
         ISIDE = 1
         COMPL = .FALSE.
         COMPR = .TRUE.
      ELSE IF( LSAME( SIDE, 'L' ) ) THEN
         ISIDE = 2
         COMPL = .TRUE.
         COMPR = .FALSE.
      ELSE IF( LSAME( SIDE, 'B' ) ) THEN
         ISIDE = 3
         COMPL = .TRUE.
         COMPR = .TRUE.
      ELSE
         ISIDE = -1
      END IF
*
      INFO = 0
      IF( ISIDE.LT.0 ) THEN
         INFO = -1
      ELSE IF( IHWMNY.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDS.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDP.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CTGEVC', -INFO )
         RETURN
      END IF
*
*     Count the number of eigenvectors
*
      IF( .NOT.ILALL ) THEN
```

```
         IM = 0
         DO 10 J = 1, N
            IF( SELECT( J ) )
     $         IM = IM + 1
   10    CONTINUE
      ELSE
         IM = N
      END IF
*
*     Check diagonal of B
*
      ILBBAD = .FALSE.
      DO 20 J = 1, N
         IF( AIMAG( P( J, J ) ).NE.ZERO )
     $      ILBBAD = .TRUE.
   20 CONTINUE
*
      IF( ILBBAD ) THEN
         INFO = -7
      ELSE IF( COMPL .AND. LDVL.LT.N .OR. LDVL.LT.1 ) THEN
         INFO = -10
      ELSE IF( COMPR .AND. LDVR.LT.N .OR. LDVR.LT.1 ) THEN
         INFO = -12
      ELSE IF( MM.LT.IM ) THEN
         INFO = -13
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CTGEVC', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      M = IM
      IF( N.EQ.0 )
     $   RETURN
*
*     Machine Constants
*
      SAFMIN = SLAMCH( 'Safe minimum' )
      BIG = ONE / SAFMIN
      CALL SLABAD( SAFMIN, BIG )
      ULP = SLAMCH( 'Epsilon' )*SLAMCH( 'Base' )
      SMALL = SAFMIN*N / ULP
      BIG = ONE / SMALL
      BIGNUM = ONE / ( SAFMIN*N )
*
*     Compute the 1-norm of each column of the strictly upper triangular
*     part of A and B to check for possible overflow in the triangular
*     solver.
```

```
*
      ANORM = ABS1( S( 1, 1 ) )
      BNORM = ABS1( P( 1, 1 ) )
      RWORK( 1 ) = ZERO
      RWORK( N+1 ) = ZERO
      DO 40 J = 2, N
         RWORK( J ) = ZERO
         RWORK( N+J ) = ZERO
         DO 30 I = 1, J - 1
            RWORK( J ) = RWORK( J ) + ABS1( S( I, J ) )
            RWORK( N+J ) = RWORK( N+J ) + ABS1( P( I, J ) )
   30    CONTINUE
         ANORM = MAX( ANORM, RWORK( J )+ABS1( S( J, J ) ) )
         BNORM = MAX( BNORM, RWORK( N+J )+ABS1( P( J, J ) ) )
   40 CONTINUE
*
      ASCALE = ONE / MAX( ANORM, SAFMIN )
      BSCALE = ONE / MAX( BNORM, SAFMIN )
*
*     Left eigenvectors
*
      IF( COMPL ) THEN
         IEIG = 0
*
*        Main loop over eigenvalues
*
         DO 140 JE = 1, N
            IF( ILALL ) THEN
               ILCOMP = .TRUE.
            ELSE
               ILCOMP = SELECT( JE )
            END IF
            IF( ILCOMP ) THEN
               IEIG = IEIG + 1
*
               IF( ABS1( S( JE, JE ) ).LE.SAFMIN .AND.
     $             ABS( REAL( P( JE, JE ) ) ).LE.SAFMIN ) THEN
*
*                Singular matrix pencil -- return unit eigenvector
*
                 DO 50 JR = 1, N
                    VL( JR, IEIG ) = CZERO
   50            CONTINUE
                 VL( IEIG, IEIG ) = CONE
                 GO TO 140
               END IF
*
*              Non-singular eigenvalue:
*              Compute coefficients  a  and  b  in
*                   H
```

```
*                 y  ( a A - b B ) = 0
*
                TEMP = ONE / MAX( ABS1( S( JE, JE ) )*ASCALE,
     $                 ABS( REAL( P( JE, JE ) ) )*BSCALE, SAFMIN )
                SALPHA = ( TEMP*S( JE, JE ) )*ASCALE
                SBETA = ( TEMP*REAL( P( JE, JE ) ) )*BSCALE
                ACOEFF = SBETA*ASCALE
                BCOEFF = SALPHA*BSCALE
*
*               Scale to avoid underflow
*
                LSA = ABS( SBETA ).GE.SAFMIN .AND. ABS( ACOEFF ).LT.SMALL
                LSB = ABS1( SALPHA ).GE.SAFMIN .AND. ABS1( BCOEFF ).LT.
     $                SMALL
*
                SCALE = ONE
                IF( LSA )
     $             SCALE = ( SMALL / ABS( SBETA ) )*MIN( ANORM, BIG )
                IF( LSB )
     $             SCALE = MAX( SCALE, ( SMALL / ABS1( SALPHA ) )*
     $                     MIN( BNORM, BIG ) )
                IF( LSA .OR. LSB ) THEN
                   SCALE = MIN( SCALE, ONE /
     $                     ( SAFMIN*MAX( ONE, ABS( ACOEFF ),
     $                     ABS1( BCOEFF ) ) ) )
                   IF( LSA ) THEN
                      ACOEFF = ASCALE*( SCALE*SBETA )
                   ELSE
                      ACOEFF = SCALE*ACOEFF
                   END IF
                   IF( LSB ) THEN
                      BCOEFF = BSCALE*( SCALE*SALPHA )
                   ELSE
                      BCOEFF = SCALE*BCOEFF
                   END IF
                END IF
*
                ACOEFA = ABS( ACOEFF )
                BCOEFA = ABS1( BCOEFF )
                XMAX = ONE
                DO 60 JR = 1, N
                   WORK( JR ) = CZERO
   60           CONTINUE
                WORK( JE ) = CONE
                DMIN = MAX( ULP*ACOEFA*ANORM, ULP*BCOEFA*BNORM, SAFMIN )
*
*                                                H
*               Triangular solve of  (a A - b B)  y = 0
*
*                                          H
```

```
*              (rowwise in  (a A - b B) , or columnwise in a A - b B)
*
               DO 100 J = JE + 1, N
*
*                 Compute
*                       j-1
*                 SUM = sum  conjg( a*S(k,j) - b*P(k,j) )*x(k)
*                       k=je
*                 (Scale if necessary)
*
                  TEMP = ONE / XMAX
                  IF( ACOEFA*RWORK( J )+BCOEFA*RWORK( N+J ).GT.BIGNUM*
     $                TEMP ) THEN
                     DO 70 JR = JE, J - 1
                        WORK( JR ) = TEMP*WORK( JR )
   70                CONTINUE
                     XMAX = ONE
                  END IF
                  SUMA = CZERO
                  SUMB = CZERO
*
                  DO 80 JR = JE, J - 1
                     SUMA = SUMA + CONJG( S( JR, J ) )*WORK( JR )
                     SUMB = SUMB + CONJG( P( JR, J ) )*WORK( JR )
   80             CONTINUE
                  SUM = ACOEFF*SUMA - CONJG( BCOEFF )*SUMB
*
*                 Form x(j) = - SUM / conjg( a*S(j,j) - b*P(j,j) )
*
*                 with scaling and perturbation of the denominator
*
                  D = CONJG( ACOEFF*S( J, J )-BCOEFF*P( J, J ) )
                  IF( ABS1( D ).LE.DMIN )
     $               D = CMPLX( DMIN )
*
                  IF( ABS1( D ).LT.ONE ) THEN
                     IF( ABS1( SUM ).GE.BIGNUM*ABS1( D ) ) THEN
                        TEMP = ONE / ABS1( SUM )
                        DO 90 JR = JE, J - 1
                           WORK( JR ) = TEMP*WORK( JR )
   90                   CONTINUE
                        XMAX = TEMP*XMAX
                        SUM = TEMP*SUM
                     END IF
                  END IF
                  WORK( J ) = CLADIV( -SUM, D )
                  XMAX = MAX( XMAX, ABS1( WORK( J ) ) )
  100          CONTINUE
*
*              Back transform eigenvector if HOWMNY='B'.
```

```
*
              IF( ILBACK ) THEN
                 CALL CGEMV( 'N', N, N+1-JE, CONE, VL( 1, JE ), LDVL,
     $                       WORK( JE ), 1, CZERO, WORK( N+1 ), 1 )
                 ISRC = 2
                 IBEG = 1
              ELSE
                 ISRC = 1
                 IBEG = JE
              END IF
*
*             Copy and scale eigenvector into column of VL
*
              XMAX = ZERO
              DO 110 JR = IBEG, N
                 XMAX = MAX( XMAX, ABS1( WORK( ( ISRC-1 )*N+JR ) ) )
  110         CONTINUE
*
              IF( XMAX.GT.SAFMIN ) THEN
                 TEMP = ONE / XMAX
                 DO 120 JR = IBEG, N
                    VL( JR, IEIG ) = TEMP*WORK( ( ISRC-1 )*N+JR )
  120            CONTINUE
              ELSE
                 IBEG = N + 1
              END IF
*
              DO 130 JR = 1, IBEG - 1
                 VL( JR, IEIG ) = CZERO
  130         CONTINUE
*
           END IF
  140    CONTINUE
      END IF
*
*     Right eigenvectors
*
      IF( COMPR ) THEN
         IEIG = IM + 1
*
*        Main loop over eigenvalues
*
         DO 250 JE = N, 1, -1
            IF( ILALL ) THEN
               ILCOMP = .TRUE.
            ELSE
               ILCOMP = SELECT( JE )
            END IF
            IF( ILCOMP ) THEN
               IEIG = IEIG - 1
```

```
*
               IF( ABS1( S( JE, JE ) ).LE.SAFMIN .AND.
     $             ABS( REAL( P( JE, JE ) ) ).LE.SAFMIN ) THEN
*
*                 Singular matrix pencil -- return unit eigenvector
*
                  DO 150 JR = 1, N
                     VR( JR, IEIG ) = CZERO
  150             CONTINUE
                  VR( IEIG, IEIG ) = CONE
                  GO TO 250
               END IF
*
*              Non-singular eigenvalue:
*              Compute coefficients  a  and  b  in
*
*              ( a A - b B ) x  = 0
*
               TEMP = ONE / MAX( ABS1( S( JE, JE ) )*ASCALE,
     $                ABS( REAL( P( JE, JE ) ) )*BSCALE, SAFMIN )
               SALPHA = ( TEMP*S( JE, JE ) )*ASCALE
               SBETA = ( TEMP*REAL( P( JE, JE ) ) )*BSCALE
               ACOEFF = SBETA*ASCALE
               BCOEFF = SALPHA*BSCALE
*
*              Scale to avoid underflow
*
               LSA = ABS( SBETA ).GE.SAFMIN .AND. ABS( ACOEFF ).LT.SMALL
               LSB = ABS1( SALPHA ).GE.SAFMIN .AND. ABS1( BCOEFF ).LT.
     $               SMALL
*
               SCALE = ONE
               IF( LSA )
     $            SCALE = ( SMALL / ABS( SBETA ) )*MIN( ANORM, BIG )
               IF( LSB )
     $            SCALE = MAX( SCALE, ( SMALL / ABS1( SALPHA ) )*
     $                    MIN( BNORM, BIG ) )
               IF( LSA .OR. LSB ) THEN
                  SCALE = MIN( SCALE, ONE /
     $                    ( SAFMIN*MAX( ONE, ABS( ACOEFF ),
     $                    ABS1( BCOEFF ) ) ) )
                  IF( LSA ) THEN
                     ACOEFF = ASCALE*( SCALE*SBETA )
                  ELSE
                     ACOEFF = SCALE*ACOEFF
                  END IF
                  IF( LSB ) THEN
                     BCOEFF = BSCALE*( SCALE*SALPHA )
                  ELSE
                     BCOEFF = SCALE*BCOEFF
```

```
                 END IF
              END IF
*
              ACOEFA = ABS( ACOEFF )
              BCOEFA = ABS1( BCOEFF )
              XMAX = ONE
              DO 160 JR = 1, N
                 WORK( JR ) = CZERO
  160         CONTINUE
              WORK( JE ) = CONE
              DMIN = MAX( ULP*ACOEFA*ANORM, ULP*BCOEFA*BNORM, SAFMIN )
*
*             Triangular solve of  (a A - b B) x = 0  (columnwise)
*
*             WORK(1:j-1) contains sums w,
*             WORK(j+1:JE) contains x
*
              DO 170 JR = 1, JE - 1
                 WORK( JR ) = ACOEFF*S( JR, JE ) - BCOEFF*P( JR, JE )
  170         CONTINUE
              WORK( JE ) = CONE
*
              DO 210 J = JE - 1, 1, -1
*
*                Form x(j) := - w(j) / d
*                with scaling and perturbation of the denominator
*
                 D = ACOEFF*S( J, J ) - BCOEFF*P( J, J )
                 IF( ABS1( D ).LE.DMIN )
     $              D = CMPLX( DMIN )
*
                 IF( ABS1( D ).LT.ONE ) THEN
                    IF( ABS1( WORK( J ) ).GE.BIGNUM*ABS1( D ) ) THEN
                       TEMP = ONE / ABS1( WORK( J ) )
                       DO 180 JR = 1, JE
                          WORK( JR ) = TEMP*WORK( JR )
  180                  CONTINUE
                    END IF
                 END IF
*
                 WORK( J ) = CLADIV( -WORK( J ), D )
*
                 IF( J.GT.1 ) THEN
*
*                   w = w + x(j)*(a S(*,j) - b P(*,j) ) with scaling
*
                    IF( ABS1( WORK( J ) ).GT.ONE ) THEN
                       TEMP = ONE / ABS1( WORK( J ) )
                       IF( ACOEFA*RWORK( J )+BCOEFA*RWORK( N+J ).GE.
     $                     BIGNUM*TEMP ) THEN
```

```
                         DO 190 JR = 1, JE
                            WORK( JR ) = TEMP*WORK( JR )
  190                    CONTINUE
                      END IF
                   END IF
*
                   CA = ACOEFF*WORK( J )
                   CB = BCOEFF*WORK( J )
                   DO 200 JR = 1, J - 1
                      WORK( JR ) = WORK( JR ) + CA*S( JR, J ) -
     $                             CB*P( JR, J )
  200              CONTINUE
                END IF
  210         CONTINUE
*
*             Back transform eigenvector if HOWMNY='B'.
*
              IF( ILBACK ) THEN
                CALL CGEMV( 'N', N, JE, CONE, VR, LDVR, WORK, 1,
     $                      CZERO, WORK( N+1 ), 1 )
                ISRC = 2
                IEND = N
              ELSE
                ISRC = 1
                IEND = JE
              END IF
*
*             Copy and scale eigenvector into column of VR
*
              XMAX = ZERO
              DO 220 JR = 1, IEND
                XMAX = MAX( XMAX, ABS1( WORK( ( ISRC-1 )*N+JR ) ) )
  220         CONTINUE
*
              IF( XMAX.GT.SAFMIN ) THEN
                TEMP = ONE / XMAX
                DO 230 JR = 1, IEND
                   VR( JR, IEIG ) = TEMP*WORK( ( ISRC-1 )*N+JR )
  230           CONTINUE
              ELSE
                IEND = 0
              END IF
*
              DO 240 JR = IEND + 1, N
                VR( JR, IEIG ) = CZERO
  240         CONTINUE
*
           END IF
  250    CONTINUE
      END IF
```

```
*
      RETURN
*
*     End of CTGEVC
*
      END
```

———-

34.40.4 lisp code

— **ctgevc.lisp** —

```
(let* ((zero 0.0)
       (one 1.0)
       (czero (f2cl-lib:cmplx 0.0 0.0))
       (cone (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (f2cl-lib:complex8) czero)
           (type (f2cl-lib:complex8) cone)
           (ignorable zero one czero cone))
  (defun ctgevc
         (side howmny select n s lds p ldp vl ldvl vr ldvr mm m work rwork
          info)
    (declare (type (array single-float (*)) rwork)
             (type (array f2cl-lib:complex8 (*)) work vr vl p s)
             (type (f2cl-lib:integer4) info m mm ldvr ldvl ldp lds n)
             (type (array f2cl-lib:logical (*)) select)
             (type (string *) howmny side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (howmny character howmny-%data% howmny-%offset%)
         (select f2cl-lib:logical select-%data% select-%offset%)
         (s f2cl-lib:complex8 s-%data% s-%offset%)
         (p f2cl-lib:complex8 p-%data% p-%offset%)
         (vl f2cl-lib:complex8 vl-%data% vl-%offset%)
         (vr f2cl-lib:complex8 vr-%data% vr-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%))
      (labels ((abs1 (x)
                 (+ (abs (f2cl-lib:freal x)) (abs (f2cl-lib:aimag x)))))
        (declare
         (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
          abs1))
        (prog ((bcoeff #C(0.0 0.0)) (ca #C(0.0 0.0)) (cb #C(0.0 0.0))
               (d #C(0.0 0.0)) (salpha #C(0.0 0.0)) (sum #C(0.0 0.0))
               (suma #C(0.0 0.0)) (sumb #C(0.0 0.0)) (x #C(0.0 0.0))
```

```
     (acoefa 0.0) (acoeff 0.0) (anorm 0.0) (ascale 0.0) (bcoefa 0.0)
     (big 0.0) (bignum 0.0) (bnorm 0.0) (bscale 0.0) (dmin 0.0)
     (safmin 0.0) (sbeta 0.0) (scale 0.0) (small 0.0) (temp 0.0)
     (ulp 0.0) (xmax 0.0) (i 0) (ibeg 0) (ieig 0) (iend 0) (ihwmny 0)
     (im 0) (iside 0) (isrc 0) (j 0) (je 0) (jr 0) (compl nil)
     (compr nil) (ilall nil) (ilback nil) (ilbbad nil) (ilcomp nil)
     (lsa nil) (lsb nil))
(declare
 (type (f2cl-lib:complex8) bcoeff ca cb d salpha sum suma sumb x)
 (type (single-float) acoefa acoeff anorm ascale bcoefa big bignum
  bnorm bscale dmin safmin sbeta scale small temp ulp xmax)
 (type (f2cl-lib:integer4) i ibeg ieig iend ihwmny im iside isrc j je
  jr)
 (type f2cl-lib:logical compl compr ilall ilback ilbbad ilcomp lsa
  lsb))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame howmny "A")
    (declare (ignore var-1))
    (when var-0 (setf howmny var-0))
    ret-val)
  (setf ihwmny 1) (setf ilall f2cl-lib:%true%)
  (setf ilback f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame howmny "S")
    (declare (ignore var-1))
    (when var-0 (setf howmny var-0))
    ret-val)
  (setf ihwmny 2) (setf ilall f2cl-lib:%false%)
  (setf ilback f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame howmny "B")
    (declare (ignore var-1))
    (when var-0 (setf howmny var-0))
    ret-val)
  (setf ihwmny 3) (setf ilall f2cl-lib:%true%)
  (setf ilback f2cl-lib:%true%))
 (t (setf ihwmny -1)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame side "R")
    (declare (ignore var-1))
    (when var-0 (setf side var-0))
    ret-val)
  (setf iside 1) (setf compl f2cl-lib:%false%)
  (setf compr f2cl-lib:%true%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame side "L")
    (declare (ignore var-1))
    (when var-0 (setf side var-0))
```

```
    ret-val)
  (setf iside 2) (setf compl f2cl-lib:%true%)
  (setf compr f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame side "B")
    (declare (ignore var-1))
    (when var-0 (setf side var-0))
    ret-val)
  (setf iside 3) (setf compl f2cl-lib:%true%)
  (setf compr f2cl-lib:%true%))
 (t (setf iside -1)))
(setf info 0)
(cond ((< iside 0) (setf info -1)) ((< ihwmny 0) (setf info -2))
      ((< n 0) (setf info -4))
      ((< lds
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -6))
      ((< ldp
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -8)))
(cond
 ((/= info 0) (xerbla "CTGEVC" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((not ilall) (setf im 0)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (if (f2cl-lib:fref select-%data% (j) ((1 *)) select-%offset%)
          (setf im (f2cl-lib:int-add im 1)))
     label10)))
 (t (setf im n)))
(setf ilbbad f2cl-lib:%false%)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (if (/=
         (f2cl-lib:aimag
          (f2cl-lib:fref p-%data% (j j) ((1 ldp) (1 *)) p-%offset%))
         zero)
        (setf ilbbad f2cl-lib:%true%))
   label20))
(cond (ilbbad (setf info -7))
      ((or (and compl (< ldvl n)) (< ldvl 1)) (setf info -10))
      ((or (and compr (< ldvr n)) (< ldvr 1)) (setf info -12))
      ((< mm im) (setf info -13)))
(cond
 ((/= info 0) (xerbla "CTGEVC" (f2cl-lib:int-sub info))
  (go end_label)))
(setf m im)
```

```
(if (= n 0)
    (go end_label))
(setf safmin (slamch "Safe minimum"))
(setf big (/ one safmin))
(multiple-value-bind (var-0 var-1)
    (slabad safmin big)
  (declare (ignore))
  (when var-0 (setf safmin var-0))
  (when var-1 (setf big var-1)))
(setf ulp (* (slamch "Epsilon") (slamch "Base")))
(setf small (/ (* safmin n) ulp))
(setf big (/ one small))
(setf bignum (/ one (* safmin n)))
(setf anorm
        (abs1
         (f2cl-lib:fref s-%data% (1 1) ((1 lds) (1 *)) s-%offset%)))
(setf bnorm
        (abs1
         (f2cl-lib:fref p-%data% (1 1) ((1 ldp) (1 *)) p-%offset%)))
(setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%) zero)
(setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add n 1)) ((1 *))
                     rwork-%offset%)
        zero)
(f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (j) ((1 *)) rwork-%offset%)
            zero)
    (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add n j))
                         ((1 *)) rwork-%offset%)
            zero)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref rwork-%data% (j) ((1 *)) rwork-%offset%)
                (+
                 (f2cl-lib:fref rwork-%data% (j) ((1 *))
                                rwork-%offset%)
                 (abs1
                  (f2cl-lib:fref s-%data% (i j) ((1 lds) (1 *))
                                 s-%offset%))))
        (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add n j))
                             ((1 *)) rwork-%offset%)
                (+
                 (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add n j))
                                ((1 *)) rwork-%offset%)
                 (abs1
                  (f2cl-lib:fref p-%data% (i j) ((1 ldp) (1 *))
                                 p-%offset%))))
```

```
        label30))
    (setf anorm
            (max anorm
                 (+
                  (f2cl-lib:fref rwork-%data% (j) ((1 *))
                                 rwork-%offset%)
                  (abs1
                   (f2cl-lib:fref s-%data% (j j) ((1 lds) (1 *))
                                  s-%offset%)))))
    (setf bnorm
            (max bnorm
                 (+
                  (f2cl-lib:fref rwork-%data%
                                 ((f2cl-lib:int-add n j)) ((1 *))
                                 rwork-%offset%)
                  (abs1
                   (f2cl-lib:fref p-%data% (j j) ((1 ldp) (1 *))
                                  p-%offset%)))))
  label40))
(setf ascale (/ one (max anorm safmin)))
(setf bscale (/ one (max bnorm safmin)))
(cond
 (compl (setf ieig 0)
  (f2cl-lib:fdo (je 1 (f2cl-lib:int-add je 1))
                ((> je n) nil)
    (tagbody
      (cond (ilall (setf ilcomp f2cl-lib:%true%))
            (t
             (setf ilcomp
                     (f2cl-lib:fref select-%data% (je) ((1 *))
                                    select-%offset%))))
      (cond
       (ilcomp (setf ieig (f2cl-lib:int-add ieig 1))
        (cond
         ((and
           (<= (abs1 (f2cl-lib:fref s (je je) ((1 lds) (1 *))))
               safmin)
           (<=
            (abs
             (f2cl-lib:freal
              (f2cl-lib:fref p (je je) ((1 ldp) (1 *)))))
            safmin))
          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                        ((> jr n) nil)
            (tagbody
              (setf (f2cl-lib:fref vl-%data% (jr ieig)
                                   ((1 ldvl) (1 *)) vl-%offset%)
                    czero)
             label50))
          (setf (f2cl-lib:fref vl-%data% (ieig ieig) ((1 ldvl) (1 *))
```

```
                        vl-%offset%)
          cone)
  (go label140)))
(setf temp
        (/ one
           (max
            (*
             (abs1
              (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                             s-%offset%))
             ascale)
            (*
             (abs
              (f2cl-lib:freal
               (f2cl-lib:fref p-%data% (je je)
                              ((1 ldp) (1 *)) p-%offset%)))
             bscale)
            safmin)))
(setf salpha
        (* temp
           (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                          s-%offset%)
           ascale))
(setf sbeta
        (* temp
           (f2cl-lib:freal
            (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
                           p-%offset%))
           bscale))
(setf acoeff (* sbeta ascale))
(setf bcoeff (* salpha bscale))
(setf lsa
        (and (>= (abs sbeta) safmin) (< (abs acoeff) small)))
(setf lsb
        (and (>= (abs1 salpha) safmin)
             (< (abs1 bcoeff) small)))
(setf scale one)
(if lsa
    (setf scale (* (/ small (abs sbeta)) (min anorm big))))
(if lsb
    (setf scale
            (max scale
                 (* (/ small (abs1 salpha))
                    (min bnorm big)))))
(cond
 ((or lsa lsb)
  (setf scale
          (min scale
               (/ one
                  (* safmin
```

```
                    (max one (abs acoeff) (abs1 bcoeff))))))
  (cond (lsa (setf acoeff (* ascale (* scale sbeta))))
        (t (setf acoeff (* scale acoeff))))
  (cond (lsb (setf bcoeff (* bscale (* scale salpha))))
        (t (setf bcoeff (* scale bcoeff))))))
(setf acoefa (abs acoeff)) (setf bcoefa (abs1 bcoeff))
(setf xmax one)
(f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
              ((> jr n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (jr) ((1 *))
                         work-%offset%)
          czero)
   label60))
(setf (f2cl-lib:fref work-%data% (je) ((1 *)) work-%offset%)
        cone)
(setf dmin
        (max (* ulp acoefa anorm) (* ulp bcoefa bnorm)
             safmin))
(f2cl-lib:fdo (j (f2cl-lib:int-add je 1)
               (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf temp (/ one xmax))
    (cond
     ((>
       (+ (* acoefa (f2cl-lib:fref rwork (j) ((1 *))))
          (* bcoefa
             (f2cl-lib:fref rwork ((f2cl-lib:int-add n j))
                            ((1 *)))))
       (* bignum temp))
      (f2cl-lib:fdo (jr je (f2cl-lib:int-add jr 1))
                    ((> jr
                        (f2cl-lib:int-add j
                                          (f2cl-lib:int-sub
                                           1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (jr) ((1 *))
                               work-%offset%)
                (* temp
                   (f2cl-lib:fref work-%data% (jr) ((1 *))
                                  work-%offset%)))
         label70))
      (setf xmax one)))
    (setf suma czero)
    (setf sumb czero)
    (f2cl-lib:fdo (jr je (f2cl-lib:int-add jr 1))
                  ((> jr
                      (f2cl-lib:int-add j
```

```
                                        (f2cl-lib:int-sub
                                         1)))
                    nil)
        (tagbody
          (setf suma
                  (+ suma
                     (*
                      (f2cl-lib:conjg
                       (f2cl-lib:fref s-%data% (jr j)
                                      ((1 lds) (1 *))
                                      s-%offset%))
                      (f2cl-lib:fref work-%data% (jr) ((1 *))
                                     work-%offset%))))
          (setf sumb
                  (+ sumb
                     (*
                      (f2cl-lib:conjg
                       (f2cl-lib:fref p-%data% (jr j)
                                      ((1 ldp) (1 *))
                                      p-%offset%))
                      (f2cl-lib:fref work-%data% (jr) ((1 *))
                                     work-%offset%))))
         label80))
      (setf sum
              (- (* acoeff suma)
                 (* (f2cl-lib:conjg bcoeff) sumb)))
      (setf d
              (f2cl-lib:conjg
               (-
                (* acoeff
                   (f2cl-lib:fref s-%data% (j j)
                                  ((1 lds) (1 *)) s-%offset%))
                (* bcoeff
                   (f2cl-lib:fref p-%data% (j j)
                                  ((1 ldp) (1 *))
                                  p-%offset%)))))
      (if (<= (abs1 d) dmin)
          (setf d (f2cl-lib:cmplx dmin)))
      (cond
       ((< (abs1 d) one)
        (cond
         ((>= (abs1 sum) (* bignum (abs1 d)))
          (setf temp (/ one (abs1 sum)))
          (f2cl-lib:fdo (jr je (f2cl-lib:int-add jr 1))
                        ((> jr
                            (f2cl-lib:int-add j
                                              (f2cl-lib:int-sub
                                               1)))
                         nil)
            (tagbody
```

```
                (setf (f2cl-lib:fref work-%data% (jr) ((1 *))
                                     work-%offset%)
                        (* temp
                           (f2cl-lib:fref work-%data% (jr)
                                          ((1 *))
                                          work-%offset%)))
                label90))
            (setf xmax (* temp xmax)) (setf sum (* temp sum))))))
        (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                             work-%offset%)
                (cladiv (- sum) d))
        (setf xmax
                (max xmax
                     (abs1
                      (f2cl-lib:fref work-%data% (j) ((1 *))
                                     work-%offset%))))
       label100))
    (cond
     (ilback
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (cgemv "N" n
           (f2cl-lib:int-sub (f2cl-lib:int-add n 1) je) cone
           (f2cl-lib:array-slice vl-%data% f2cl-lib:complex8
                                 (1 je) ((1 ldvl) (1 *))
                                 vl-%offset%)
           ldvl
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                 (je) ((1 *)) work-%offset%)
           1 czero
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                 ((+ n 1)) ((1 *)) work-%offset%)
           1)
        (declare
         (ignore var-0 var-2 var-4 var-6 var-7 var-9 var-10))
        (when var-1 (setf n var-1))
        (when var-3 (setf cone var-3))
        (when var-5 (setf ldvl var-5))
        (when var-8 (setf czero var-8)))
      (setf isrc 2) (setf ibeg 1))
     (t (setf isrc 1) (setf ibeg je)))
    (setf xmax zero)
    (f2cl-lib:fdo (jr ibeg (f2cl-lib:int-add jr 1))
                  ((> jr n) nil)
      (tagbody
        (setf xmax
                (max xmax
                     (abs1
                      (f2cl-lib:fref work-%data%
```

```
                                          ((f2cl-lib:int-add
                                            (f2cl-lib:int-mul
                                             (f2cl-lib:int-sub isrc 1)
                                             n)
                                            jr))
                                          ((1 *)) work-%offset%))))
            label110))
          (cond
           ((> xmax safmin) (setf temp (/ one xmax))
            (f2cl-lib:fdo (jr ibeg (f2cl-lib:int-add jr 1))
                          ((> jr n) nil)
              (tagbody
                (setf (f2cl-lib:fref vl-%data% (jr ieig)
                                     ((1 ldvl) (1 *)) vl-%offset%)
                      (* temp
                         (f2cl-lib:fref work-%data%
                                        ((f2cl-lib:int-add
                                          (f2cl-lib:int-mul
                                           (f2cl-lib:int-sub isrc 1)
                                           n)
                                          jr))
                                        ((1 *)) work-%offset%)))
             label120)))
           (t (setf ibeg (f2cl-lib:int-add n 1))))
          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                        ((> jr
                            (f2cl-lib:int-add ibeg
                                              (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref vl-%data% (jr ieig) ((1 ldvl) (1 *))
                                   vl-%offset%)
                    czero)
            label130))))
      label140))))
(cond
 (compr (setf ieig (f2cl-lib:int-add im 1))
  (f2cl-lib:fdo (je n (f2cl-lib:int-add je (f2cl-lib:int-sub 1)))
                ((> je 1) nil)
    (tagbody
      (cond (ilall (setf ilcomp f2cl-lib:%true%))
            (t
             (setf ilcomp
                     (f2cl-lib:fref select-%data% (je) ((1 *))
                                    select-%offset%))))
      (cond
       (ilcomp (setf ieig (f2cl-lib:int-sub ieig 1))
        (cond
         ((and
           (<= (abs1 (f2cl-lib:fref s (je je) ((1 lds) (1 *))))
```

```
        safmin)
    (<=
     (abs
      (f2cl-lib:freal
       (f2cl-lib:fref p (je je) ((1 ldp) (1 *)))))
     safmin))
   (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                 ((> jr n) nil)
     (tagbody
       (setf (f2cl-lib:fref vr-%data% (jr ieig)
                            ((1 ldvr) (1 *)) vr-%offset%)
             czero)
      label150))
   (setf (f2cl-lib:fref vr-%data% (ieig ieig) ((1 ldvr) (1 *))
                        vr-%offset%)
         cone)
   (go label250)))
 (setf temp
        (/ one
           (max
            (*
             (abs1
              (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                             s-%offset%))
             ascale)
            (*
             (abs
              (f2cl-lib:freal
               (f2cl-lib:fref p-%data% (je je)
                              ((1 ldp) (1 *)) p-%offset%)))
             bscale)
            safmin)))
 (setf salpha
        (* temp
           (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                          s-%offset%)
           ascale))
 (setf sbeta
        (* temp
           (f2cl-lib:freal
            (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
                           p-%offset%))
           bscale))
 (setf acoeff (* sbeta ascale))
 (setf bcoeff (* salpha bscale))
 (setf lsa
        (and (>= (abs sbeta) safmin) (< (abs acoeff) small)))
 (setf lsb
        (and (>= (abs1 salpha) safmin)
             (< (abs1 bcoeff) small)))
```

```
(setf scale one)
(if lsa
    (setf scale (* (/ small (abs sbeta)) (min anorm big))))
(if lsb
    (setf scale
            (max scale
                 (* (/ small (abs1 salpha))
                    (min bnorm big)))))
(cond
 ((or lsa lsb)
  (setf scale
          (min scale
               (/ one
                  (* safmin
                     (max one (abs acoeff) (abs1 bcoeff))))))
  (cond (lsa (setf acoeff (* ascale (* scale sbeta))))
        (t (setf acoeff (* scale acoeff))))
  (cond (lsb (setf bcoeff (* bscale (* scale salpha))))
        (t (setf bcoeff (* scale bcoeff))))))
(setf acoefa (abs acoeff)) (setf bcoefa (abs1 bcoeff))
(setf xmax one)
(f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
              ((> jr n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (jr) ((1 *))
                         work-%offset%)
            czero)
   label160))
(setf (f2cl-lib:fref work-%data% (je) ((1 *)) work-%offset%)
        cone)
(setf dmin
        (max (* ulp acoefa anorm) (* ulp bcoefa bnorm)
             safmin))
(f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
              ((> jr
                  (f2cl-lib:int-add je (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (jr) ((1 *))
                         work-%offset%)
            (-
             (* acoeff
                (f2cl-lib:fref s-%data% (jr je)
                               ((1 lds) (1 *)) s-%offset%))
             (* bcoeff
                (f2cl-lib:fref p-%data% (jr je)
                               ((1 ldp) (1 *)) p-%offset%))))
   label170))
(setf (f2cl-lib:fref work-%data% (je) ((1 *)) work-%offset%)
        cone)
```

```
(f2cl-lib:fdo (j (f2cl-lib:int-add je (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j 1) nil)
  (tagbody
    (setf d
            (-
             (* acoeff
                (f2cl-lib:fref s-%data% (j j) ((1 lds) (1 *))
                               s-%offset%))
             (* bcoeff
                (f2cl-lib:fref p-%data% (j j) ((1 ldp) (1 *))
                               p-%offset%))))
    (if (<= (abs1 d) dmin)
        (setf d (f2cl-lib:cmplx dmin)))
    (cond
     ((< (abs1 d) one)
      (cond
       ((>= (abs1 (f2cl-lib:fref work (j) ((1 *))))
            (* bignum (abs1 d)))
        (setf temp
                (/ one
                   (abs1
                    (f2cl-lib:fref work-%data% (j) ((1 *))
                                   work-%offset%))))
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr je) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (jr) ((1 *))
                                 work-%offset%)
                    (* temp
                       (f2cl-lib:fref work-%data% (jr)
                                      ((1 *))
                                      work-%offset%)))
           label180))))))
    (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                         work-%offset%)
            (cladiv
             (-
              (f2cl-lib:fref work-%data% (j) ((1 *))
                             work-%offset%))
             d))
    (cond
     ((> j 1)
      (cond
       ((> (abs1 (f2cl-lib:fref work (j) ((1 *)))) one)
        (setf temp
                (/ one
                   (abs1
                    (f2cl-lib:fref work-%data% (j) ((1 *))
                                   work-%offset%))))
```

```
                (cond
                 ((>=
                   (+ (* acoefa (f2cl-lib:fref rwork (j) ((1 *))))
                      (* bcoefa
                         (f2cl-lib:fref rwork
                                        ((f2cl-lib:int-add n j))
                                        ((1 *)))))
                   (* bignum temp))
                  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                                ((> jr je) nil)
                    (tagbody
                      (setf (f2cl-lib:fref work-%data% (jr) ((1 *))
                                           work-%offset%)
                              (* temp
                                 (f2cl-lib:fref work-%data% (jr)
                                                ((1 *))
                                                work-%offset%)))
                     label190))))))
            (setf ca
                    (* acoeff
                       (f2cl-lib:fref work-%data% (j) ((1 *))
                                      work-%offset%)))
            (setf cb
                    (* bcoeff
                       (f2cl-lib:fref work-%data% (j) ((1 *))
                                      work-%offset%)))
            (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                          ((> jr
                              (f2cl-lib:int-add j
                                                (f2cl-lib:int-sub
                                                 1)))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref work-%data% (jr) ((1 *))
                                     work-%offset%)
                        (-
                         (+
                          (f2cl-lib:fref work-%data% (jr) ((1 *))
                                         work-%offset%)
                          (* ca
                             (f2cl-lib:fref s-%data% (jr j)
                                            ((1 lds) (1 *))
                                            s-%offset%)))
                         (* cb
                            (f2cl-lib:fref p-%data% (jr j)
                                           ((1 ldp) (1 *))
                                           p-%offset%))))
                label200))))
          label210))
        (cond
```

```
 (ilback
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (cgemv "N" n je cone vr ldvr work 1 czero
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             ((+ n 1)) ((1 *)) work-%offset%)
       1)
    (declare (ignore var-0 var-4 var-6 var-7 var-9 var-10))
    (when var-1 (setf n var-1))
    (when var-2 (setf je var-2))
    (when var-3 (setf cone var-3))
    (when var-5 (setf ldvr var-5))
    (when var-8 (setf czero var-8)))
  (setf isrc 2) (setf iend n))
 (t (setf isrc 1) (setf iend je)))
(setf xmax zero)
(f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
              ((> jr iend) nil)
  (tagbody
    (setf xmax
            (max xmax
                 (abs1
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-sub isrc 1)
                                    n)
                                   jr))
                                 ((1 *)) work-%offset%))))
   label220))
(cond
 ((> xmax safmin) (setf temp (/ one xmax))
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr iend) nil)
    (tagbody
      (setf (f2cl-lib:fref vr-%data% (jr ieig)
                           ((1 ldvr) (1 *)) vr-%offset%)
            (* temp
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add
                                (f2cl-lib:int-mul
                                 (f2cl-lib:int-sub isrc 1)
                                 n)
                                jr))
                              ((1 *)) work-%offset%)))
     label230)))
 (t (setf iend 0)))
(f2cl-lib:fdo (jr (f2cl-lib:int-add iend 1)
               (f2cl-lib:int-add jr 1))
```

```
                              ((> jr n) nil)
                    (tagbody
                      (setf (f2cl-lib:fref vr-%data% (jr ieig) ((1 ldvr) (1 *))
                                           vr-%offset%)
                            czero)
                     label240))))
                label250))))
          (go end_label)
         end_label
          (return
           (values side howmny nil n nil nil nil nil nil ldvl nil ldvr nil m
                   nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ctgevc
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (array fortran-to-lisp::logical
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::side
                                              fortran-to-lisp::howmny nil
                                              fortran-to-lisp::n nil nil nil
                                              nil nil fortran-to-lisp::ldvl nil
                                              fortran-to-lisp::ldvr nil
                                              fortran-to-lisp::m nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::cladiv))))
```

———-

34.41 ctgexc

34.41.1 Axiom unit tests

— **ctgexc.input** —

```
)set break resume
)sys rm -f ctgexc.output
)spool ctgexc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

34.41.2 Axiom help page

— **ctgexc.help** —

```
======================================================================
ctgexc examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ctgexc (WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z, LDZ,
           IFST, ILST, INFO)
           CTGEXC

Function/Subroutine Documentation
   subroutine ctgexc (logical WANTQ, logical WANTZ, integer N, complex,
       dimension( lda, * ) A, integer LDA, complex, dimension( ldb, * ) B,
```

```
      integer LDB, complex, dimension( ldq, * ) Q, integer LDQ, complex,
      dimension( ldz, * ) Z, integer LDZ, integer IFST, integer ILST, integer
      INFO)
      CTGEXC

Purpose:
   CTGEXC reorders the generalized Schur decomposition of a complex
   matrix pair (A,B), using an unitary equivalence transformation
   (A, B) := Q * (A, B) * Z**H, so that the diagonal block of (A, B) with
   row index IFST is moved to row ILST.

   (A, B) must be in generalized Schur canonical form, that is, A and
   B are both upper triangular.

   Optionally, the matrices Q and Z of generalized Schur vectors are
   updated.

   Q(in) * A(in) * Z(in)**H = Q(out) * A(out) * Z(out)**H
   Q(in) * B(in) * Z(in)**H = Q(out) * B(out) * Z(out)**H

Parameters:

 WANTQ
   WANTQ is LOGICAL
   .TRUE. : update the left transformation matrix Q;
   .FALSE.: do not update Q.

 WANTZ
   WANTZ is LOGICAL
   .TRUE. : update the right transformation matrix Z;
   .FALSE.: do not update Z.

 N
   N is INTEGER
   The order of the matrices A and B. N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the upper triangular matrix A in the pair (A, B).
   On exit, the updated matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A. LDA >= max(1,N).

 B
   B is COMPLEX array, dimension (LDB,N)
   On entry, the upper triangular matrix B in the pair (A, B).
   On exit, the updated matrix B.
```

```
LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= max(1,N).

Q
  Q is COMPLEX array, dimension (LDZ,N)
  On entry, if WANTQ = .TRUE., the unitary matrix Q.
  On exit, the updated matrix Q.
  If WANTQ = .FALSE., Q is not referenced.

LDQ
  LDQ is INTEGER
  The leading dimension of the array Q. LDQ >= 1;
  If WANTQ = .TRUE., LDQ >= N.

Z
  Z is COMPLEX array, dimension (LDZ,N)
  On entry, if WANTZ = .TRUE., the unitary matrix Z.
  On exit, the updated matrix Z.
  If WANTZ = .FALSE., Z is not referenced.

LDZ
  LDZ is INTEGER
  The leading dimension of the array Z. LDZ >= 1;
  If WANTZ = .TRUE., LDZ >= N.

IFST
  IFST is INTEGER

ILST
  ILST is INTEGER
  Specify the reordering of the diagonal blocks of (A, B).
  The block with row index IFST is moved to row ILST, by a
  sequence of swapping between adjacent blocks.

INFO
  INFO is INTEGER
  =0:  Successful exit.
  <0:  if INFO = -i, the i-th argument had an illegal value.
  =1:  The transformed matrix pair (A, B) would be too far
       from generalized Schur form; the problem is ill-
       conditioned. (A, B) may have been partially reordered,
       and ILST points to the first row of the current
       position of the block being moved.
```

34.41.3 fortran code

— ctgexc.f —

```
*  =====================================================================
      SUBROUTINE CTGEXC( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                   LDZ, IFST, ILST, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      LOGICAL            WANTQ, WANTZ
      INTEGER            IFST, ILST, INFO, LDA, LDB, LDQ, LDZ, N
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), B( LDB, * ), Q( LDQ, * ),
     $                   Z( LDZ, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            HERE
*     ..
*     .. External Subroutines ..
      EXTERNAL           CTGEX2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Decode and test input arguments.
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDQ.LT.1 .OR. WANTQ .AND. ( LDQ.LT.MAX( 1, N ) ) ) THEN
         INFO = -9
      ELSE IF( LDZ.LT.1 .OR. WANTZ .AND. ( LDZ.LT.MAX( 1, N ) ) ) THEN
         INFO = -11
      ELSE IF( IFST.LT.1 .OR. IFST.GT.N ) THEN
         INFO = -12
```

```
      ELSE IF( ILST.LT.1 .OR. ILST.GT.N ) THEN
         INFO = -13
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CTGEXC', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.LE.1 )
     $   RETURN
      IF( IFST.EQ.ILST )
     $   RETURN
*
      IF( IFST.LT.ILST ) THEN
*
         HERE = IFST
*
   10    CONTINUE
*
*        Swap with next one below
*
         CALL CTGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z, LDZ,
     $                HERE, INFO )
         IF( INFO.NE.0 ) THEN
            ILST = HERE
            RETURN
         END IF
         HERE = HERE + 1
         IF( HERE.LT.ILST )
     $      GO TO 10
         HERE = HERE - 1
      ELSE
         HERE = IFST - 1
*
   20    CONTINUE
*
*        Swap with next one above
*
         CALL CTGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z, LDZ,
     $                HERE, INFO )
         IF( INFO.NE.0 ) THEN
            ILST = HERE
            RETURN
         END IF
         HERE = HERE - 1
         IF( HERE.GE.ILST )
     $      GO TO 20
         HERE = HERE + 1
```

```
      END IF
      ILST = HERE
      RETURN
*
*     End of CTGEXC
*
      END
```

———-

34.41.4 lisp code

— **ctgexc.lisp** —

```
(defun ctgexc (wantq wantz n a lda b ldb$ q ldq z ldz ifst ilst info)
  (declare (type (array f2cl-lib:complex8 (*)) z q b a)
           (type (f2cl-lib:integer4) info ilst ifst ldz ldq ldb$ lda n)
           (type f2cl-lib:logical wantz wantq))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex8 a-%data% a-%offset%)
       (b f2cl-lib:complex8 b-%data% b-%offset%)
       (q f2cl-lib:complex8 q-%data% q-%offset%)
       (z f2cl-lib:complex8 z-%data% z-%offset%))
    (prog ((here 0))
      (declare (type (f2cl-lib:integer4) here))
      (setf info 0)
      (cond ((< n 0) (setf info -3))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -5))
            ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -7))
            ((or (< ldq 1)
                 (and wantq
                      (< ldq
                         (max (the f2cl-lib:integer4 1)
                              (the f2cl-lib:integer4 n)))))
             (setf info -9))
            ((or (< ldz 1)
                 (and wantz
                      (< ldz
                         (max (the f2cl-lib:integer4 1)
                              (the f2cl-lib:integer4 n)))))
             (setf info -11))
            ((or (< ifst 1) (> ifst n)) (setf info -12))
            ((or (< ilst 1) (> ilst n)) (setf info -13)))
      (cond
       ((/= info 0) (xerbla "CTGEXC" (f2cl-lib:int-sub info)) (go end_label)))
      (if (<= n 1)
```

```
          (go end_label))
      (if (= ifst ilst)
          (go end_label))
      (cond
       ((< ifst ilst)
        (tagbody
          (setf here ifst)
         label10
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10 var-11 var-12)
              (ctgex2 wantq wantz n a lda b ldb$ q ldq z ldz here info)
            (declare
             (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9 var-10 var-11))
            (setf info var-12))
          (cond ((/= info 0) (setf ilst here) (go end_label)))
          (setf here (f2cl-lib:int-add here 1))
          (if (< here ilst)
              (go label10))
          (setf here (f2cl-lib:int-sub here 1))))
       (t
        (tagbody
          (setf here (f2cl-lib:int-sub ifst 1))
         label20
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10 var-11 var-12)
              (ctgex2 wantq wantz n a lda b ldb$ q ldq z ldz here info)
            (declare
             (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9 var-10 var-11))
            (setf info var-12))
          (cond ((/= info 0) (setf ilst here) (go end_label)))
          (setf here (f2cl-lib:int-sub here 1))
          (if (>= here ilst)
              (go label20))
          (setf here (f2cl-lib:int-add here 1)))))
      (setf ilst here)
      (go end_label)
     end_label
      (return
       (values nil nil nil nil nil nil nil nil nil nil nil nil ilst info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ctgexc
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
```

```
'(fortran-to-lisp::logical
  fortran-to-lisp::logical
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil nil nil nil nil nil
  nil nil nil nil
  fortran-to-lisp::ilst
  fortran-to-lisp::info)
:calls '(fortran-to-lisp::ctgex2))))
```

———-

Chapter 35

LAPACK - General Matrices, Comp. Routines, Complex16

35.1 zgebak

35.1.1 Axiom unit tests

— zgebak.input —

```
)set break resume
)sys rm -f zgebak.output
)spool zgebak.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.1.2 Axiom help page

— zgebak.help —

```
==========================================================================
zgebak examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgebak (JOB, SIDE, N, ILO, IHI, SCALE, M, V, LDV, INFO)
           ZGEBAK

Function/Subroutine Documentation
   subroutine zgebak (character JOB, character SIDE, integer N, integer ILO,
       integer IHI, double precision, dimension( * ) SCALE, integer M,
       complex*16, dimension( ldv, * ) V, integer LDV, integer INFO)
       ZGEBAK

Purpose:
   ZGEBAK forms the right or left eigenvectors of a complex general
   matrix by backward transformation on the computed eigenvectors of the
   balanced matrix output by ZGEBAL.

Parameters:

 JOB
   JOB is CHARACTER*1
   Specifies the type of backward transformation required:
   = 'N', do nothing, return immediately;
   = 'P', do backward transformation for permutation only;
   = 'S', do backward transformation for scaling only;
   = 'B', do backward transformations for both permutation and scaling.
   JOB must be the same as the argument JOB supplied to ZGEBAL.

 SIDE
   SIDE is CHARACTER*1
   = 'R':  V contains right eigenvectors;
   = 'L':  V contains left eigenvectors.

 N
   N is INTEGER
   The number of rows of the matrix V.  N >= 0.

 ILO
   ILO is INTEGER

 IHI
   IHI is INTEGER
   The integers ILO and IHI determined by ZGEBAL.
   1 <= ILO <= IHI <= N, if N > 0; ILO=1 and IHI=0, if N=0.

 SCALE
```

```
  SCALE is DOUBLE PRECISION array, dimension (N)
  Details of the permutation and scaling factors, as returned by ZGEBAL.

M
  M is INTEGER
  The number of columns of the matrix V.  M >= 0.

V
  V is COMPLEX*16 array, dimension (LDV,M)
  On entry, the matrix of right or left eigenvectors to be
  transformed, as returned by ZHSEIN or ZTREVC.
  On exit, V is overwritten by the transformed eigenvectors.

LDV
  LDV is INTEGER
  The leading dimension of the array V. LDV >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

35.1.3 fortran code

— zgebak.f —

```
*  =====================================================================
      SUBROUTINE ZGEBAK( JOB, SIDE, N, ILO, IHI, SCALE, M, V, LDV,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOB, SIDE
      INTEGER            IHI, ILO, INFO, LDV, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   SCALE( * )
      COMPLEX*16         V( LDV, * )
*     ..
*
*  =====================================================================
```

```
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LEFTV, RIGHTV
      INTEGER            I, II, K
      DOUBLE PRECISION   S
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZDSCAL, ZSWAP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Decode and Test the input parameters
*
      RIGHTV = LSAME( SIDE, 'R' )
      LEFTV = LSAME( SIDE, 'L' )
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.RIGHTV .AND. .NOT.LEFTV ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -5
      ELSE IF( M.LT.0 ) THEN
         INFO = -7
      ELSE IF( LDV.LT.MAX( 1, N ) ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEBAK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
```

```
*
      IF( N.EQ.0 )
     $   RETURN
      IF( M.EQ.0 )
     $   RETURN
      IF( LSAME( JOB, 'N' ) )
     $   RETURN
*
      IF( ILO.EQ.IHI )
     $   GO TO 30
*
*     Backward balance
*
      IF( LSAME( JOB, 'S' ) .OR. LSAME( JOB, 'B' ) ) THEN
*
         IF( RIGHTV ) THEN
            DO 10 I = ILO, IHI
               S = SCALE( I )
               CALL ZDSCAL( M, S, V( I, 1 ), LDV )
   10       CONTINUE
         END IF
*
         IF( LEFTV ) THEN
            DO 20 I = ILO, IHI
               S = ONE / SCALE( I )
               CALL ZDSCAL( M, S, V( I, 1 ), LDV )
   20       CONTINUE
         END IF
*
      END IF
*
*     Backward permutation
*
*     For  I = ILO-1 step -1 until 1,
*              IHI+1 step 1 until N do --
*
   30 CONTINUE
      IF( LSAME( JOB, 'P' ) .OR. LSAME( JOB, 'B' ) ) THEN
         IF( RIGHTV ) THEN
            DO 40 II = 1, N
               I = II
               IF( I.GE.ILO .AND. I.LE.IHI )
     $            GO TO 40
               IF( I.LT.ILO )
     $            I = ILO - II
               K = SCALE( I )
               IF( K.EQ.I )
     $            GO TO 40
               CALL ZSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   40       CONTINUE
```

```
         END IF
*
         IF( LEFTV ) THEN
            DO 50 II = 1, N
               I = II
               IF( I.GE.ILO .AND. I.LE.IHI )
     $            GO TO 50
               IF( I.LT.ILO )
     $            I = ILO - II
               K = SCALE( I )
               IF( K.EQ.I )
     $            GO TO 50
               CALL ZSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   50       CONTINUE
         END IF
      END IF
*
      RETURN
*
*     End of ZGEBAK
*
      END
```

———-

35.1.4 lisp code

— **zgebak.lisp** —

———-

35.2 zgebal

35.2.1 Axiom unit tests

— **zgebal.input** —

```
)set break resume
)sys rm -f zgebal.output
)spool zgebal.output
)set message test on
)set message auto off
```

```
)clear all

)spool
)lisp (bye)
```

———-

35.2.2 Axiom help page

— **zgebal.help** —

```
=====================================================================
zgebal examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgebal (JOB, N, A, LDA, ILO, IHI, SCALE, INFO)
           ZGEBAL

Function/Subroutine Documentation
   subroutine zgebal (character JOB, integer N, complex*16, dimension( lda, *
       ) A, integer LDA, integer ILO, integer IHI, double precision,
       dimension( * ) SCALE, integer INFO)
       ZGEBAL

Purpose:
   ZGEBAL balances a general complex matrix A.  This involves, first,
   permuting A by a similarity transformation to isolate eigenvalues
   in the first 1 to ILO-1 and last IHI+1 to N elements on the
   diagonal; and second, applying a diagonal similarity transformation
   to rows and columns ILO to IHI to make the rows and columns as
   close in norm as possible.  Both steps are optional.

   Balancing may reduce the 1-norm of the matrix, and improve the
   accuracy of the computed eigenvalues and/or eigenvectors.

Parameters:

 JOB
   JOB is CHARACTER*1
   Specifies the operations to be performed on A:
   = 'N':  none:  simply set ILO = 1, IHI = N, SCALE(I) = 1.0
           for i = 1,...,N;
```

```
  = 'P':  permute only;
  = 'S':  scale only;
  = 'B':  both permute and scale.

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the input matrix A.
  On exit,  A is overwritten by the balanced matrix.
  If JOB = 'N', A is not referenced.
  See Further Details.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

ILO

IHI
  ILO and IHI are set to INTEGER such that on exit
  A(i,j) = 0 if i > j and j = 1,...,ILO-1 or I = IHI+1,...,N.
  If JOB = 'N' or 'S', ILO = 1 and IHI = N.

SCALE
  SCALE is DOUBLE PRECISION array, dimension (N)
  Details of the permutations and scaling factors applied to
  A.  If P(j) is the index of the row and column interchanged
  with row and column j and D(j) is the scaling factor
  applied to row and column j, then
  SCALE(j) = P(j)    for j = 1,...,ILO-1
           = D(j)    for j = ILO,...,IHI
           = P(j)    for j = IHI+1,...,N.
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.

Further Details:
  The permutations consist of row and column interchanges which put
  the matrix in the form

                         ( T1   X   Y  )
               P A P = (  0   B   Z  )
                         (  0   0   T2 )
```

```
where T1 and T2 are upper triangular matrices whose eigenvalues lie
along the diagonal.  The column indices ILO and IHI mark the starting
and ending columns of the submatrix B. Balancing consists of applying
a diagonal similarity transformation inv(D) * B * D to make the
1-norms of each row of B and its corresponding column nearly equal.
The output matrix is

             ( T1     X*D          Y    )
             (  0  inv(D)*B*D  inv(D)*Z ).
             (  0      0           T2   )

Information about the permutations P and the diagonal matrix D is
returned in the vector SCALE.

This subroutine is based on the EISPACK routine CBAL.
```

35.2.3 fortran code

— zgebal.f —

```
*  ===================================================================
      SUBROUTINE ZGEBAL( JOB, N, A, LDA, ILO, IHI, SCALE, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOB
      INTEGER            IHI, ILO, INFO, LDA, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   SCALE( * )
      COMPLEX*16         A( LDA, * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   SCLFAC
      PARAMETER          ( SCLFAC = 2.0D+0 )
```

```
      DOUBLE PRECISION   FACTOR
      PARAMETER          ( FACTOR = 0.95D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOCONV
      INTEGER            I, ICA, IEXC, IRA, J, K, L, M
      DOUBLE PRECISION   C, CA, F, G, R, RA, S, SFMAX1, SFMAX2, SFMIN1,
     $                   SFMIN2
      COMPLEX*16         CDUM
*     ..
*     .. External Functions ..
      LOGICAL            DISNAN, LSAME
      INTEGER            IZAMAX
      DOUBLE PRECISION   DLAMCH, DZNRM2
      EXTERNAL           DISNAN, LSAME, IZAMAX, DLAMCH, DZNRM2
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZDSCAL, ZSWAP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, MAX, MIN
*
*     Test the input parameters
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEBAL', -INFO )
         RETURN
      END IF
*
      K = 1
      L = N
*
      IF( N.EQ.0 )
     $   GO TO 210
*
      IF( LSAME( JOB, 'N' ) ) THEN
         DO 10 I = 1, N
            SCALE( I ) = ONE
   10    CONTINUE
         GO TO 210
      END IF
```

```
*
      IF( LSAME( JOB, 'S' ) )
     $   GO TO 120
*
*     Permutation to isolate eigenvalues if possible
*
      GO TO 50
*
*     Row and column exchange.
*
   20 CONTINUE
      SCALE( M ) = J
      IF( J.EQ.M )
     $   GO TO 30
*
      CALL ZSWAP( L, A( 1, J ), 1, A( 1, M ), 1 )
      CALL ZSWAP( N-K+1, A( J, K ), LDA, A( M, K ), LDA )
*
   30 CONTINUE
      GO TO ( 40, 80 )IEXC
*
*     Search for rows isolating an eigenvalue and push them down.
*
   40 CONTINUE
      IF( L.EQ.1 )
     $   GO TO 210
      L = L - 1
*
   50 CONTINUE
      DO 70 J = L, 1, -1
*
         DO 60 I = 1, L
            IF( I.EQ.J )
     $         GO TO 60
            IF( DBLE( A( J, I ) ).NE.ZERO .OR. DIMAG( A( J, I ) ).NE.
     $          ZERO )GO TO 70
   60    CONTINUE
*
         M = L
         IEXC = 1
         GO TO 20
   70 CONTINUE
*
      GO TO 90
*
*     Search for columns isolating an eigenvalue and push them left.
*
   80 CONTINUE
      K = K + 1
*
```

```
   90 CONTINUE
      DO 110 J = K, L
*
         DO 100 I = K, L
            IF( I.EQ.J )
     $         GO TO 100
            IF( DBLE( A( I, J ) ).NE.ZERO .OR. DIMAG( A( I, J ) ).NE.
     $          ZERO )GO TO 110
  100    CONTINUE
*
         M = K
         IEXC = 2
         GO TO 20
  110 CONTINUE
*
  120 CONTINUE
      DO 130 I = K, L
         SCALE( I ) = ONE
  130 CONTINUE
*
      IF( LSAME( JOB, 'P' ) )
     $   GO TO 210
*
*     Balance the submatrix in rows K to L.
*
*     Iterative loop for norm reduction
*
      SFMIN1 = DLAMCH( 'S' ) / DLAMCH( 'P' )
      SFMAX1 = ONE / SFMIN1
      SFMIN2 = SFMIN1*SCLFAC
      SFMAX2 = ONE / SFMIN2
  140 CONTINUE
      NOCONV = .FALSE.
*
      DO 200 I = K, L
*
         C = DZNRM2( L-K+1, A( K, I ), 1 )
         R = DZNRM2( L-K+1, A( I, K ), LDA )
         ICA = IZAMAX( L, A( 1, I ), 1 )
         CA = ABS( A( ICA, I ) )
         IRA = IZAMAX( N-K+1, A( I, K ), LDA )
         RA = ABS( A( I, IRA+K-1 ) )
*
*        Guard against zero C or R due to underflow.
*
         IF( C.EQ.ZERO .OR. R.EQ.ZERO )
     $      GO TO 200
         G = R / SCLFAC
         F = ONE
         S = C + R
```

```
  160    CONTINUE
         IF( C.GE.G .OR. MAX( F, C, CA ).GE.SFMAX2 .OR.
     $       MIN( R, G, RA ).LE.SFMIN2 )GO TO 170
            IF( DISNAN( C+F+CA+R+G+RA ) ) THEN
*
*           Exit if NaN to avoid infinite loop
*
            INFO = -3
            CALL XERBLA( 'ZGEBAL', -INFO )
            RETURN
         END IF
         F = F*SCLFAC
         C = C*SCLFAC
         CA = CA*SCLFAC
         R = R / SCLFAC
         G = G / SCLFAC
         RA = RA / SCLFAC
         GO TO 160
*
  170    CONTINUE
         G = C / SCLFAC
  180    CONTINUE
         IF( G.LT.R .OR. MAX( R, RA ).GE.SFMAX2 .OR.
     $       MIN( F, C, G, CA ).LE.SFMIN2 )GO TO 190
         F = F / SCLFAC
         C = C / SCLFAC
         G = G / SCLFAC
         CA = CA / SCLFAC
         R = R*SCLFAC
         RA = RA*SCLFAC
         GO TO 180
*
*        Now balance.
*
  190    CONTINUE
         IF( ( C+R ).GE.FACTOR*S )
     $      GO TO 200
         IF( F.LT.ONE .AND. SCALE( I ).LT.ONE ) THEN
            IF( F*SCALE( I ).LE.SFMIN1 )
     $         GO TO 200
         END IF
         IF( F.GT.ONE .AND. SCALE( I ).GT.ONE ) THEN
            IF( SCALE( I ).GE.SFMAX1 / F )
     $         GO TO 200
         END IF
         G = ONE / F
         SCALE( I ) = SCALE( I )*F
         NOCONV = .TRUE.
*
         CALL ZDSCAL( N-K+1, G, A( I, K ), LDA )
```

```
         CALL ZDSCAL( L, F, A( 1, I ), 1 )
*
  200 CONTINUE
*
      IF( NOCONV )
     $   GO TO 140
*
  210 CONTINUE
      ILO = K
      IHI = L
*
      RETURN
*
*     End of ZGEBAL
*
      END
```

———-

35.2.4 lisp code

— **zgebal.lisp** —

———-

35.3 zgebd2

35.3.1 Axiom unit tests

— **zgebd2.input** —

```
)set break resume
)sys rm -f zgebd2.output
)spool zgebd2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.3.2 Axiom help page

— zgebd2.help —

```
==================================================================
zgebd2 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgebd2 (M, N, A, LDA, D, E, TAUQ, TAUP, WORK, INFO)
           ZGEBD2 reduces a general matrix to bidiagonal form using an
           unblocked algorithm.

Function/Subroutine Documentation
   subroutine zgebd2 (integer M, integer N, complex*16, dimension( lda, * ) A,
       integer LDA, double precision, dimension( * ) D, double precision,
       dimension( * ) E, complex*16, dimension( * ) TAUQ, complex*16,
       dimension( * ) TAUP, complex*16, dimension( * ) WORK, integer INFO)
       ZGEBD2 reduces a general matrix to bidiagonal form using an unblocked
       algorithm.

Purpose:
   ZGEBD2 reduces a complex general m by n matrix A to upper or lower
   real bidiagonal form B by a unitary transformation: Q**H * A * P = B.

   If m >= n, B is upper bidiagonal; if m < n, B is lower bidiagonal.

Parameters:

 M
   M is INTEGER
   The number of rows in the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns in the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the m by n general matrix to be reduced.
   On exit,
   if m >= n, the diagonal and the first superdiagonal are
      overwritten with the upper bidiagonal matrix B; the
      elements below the diagonal, with the array TAUQ, represent
```

```
      the unitary matrix Q as a product of elementary
      reflectors, and the elements above the first superdiagonal,
      with the array TAUP, represent the unitary matrix P as
      a product of elementary reflectors;
   if m < n, the diagonal and the first subdiagonal are
      overwritten with the lower bidiagonal matrix B; the
      elements below the first subdiagonal, with the array TAUQ,
      represent the unitary matrix Q as a product of
      elementary reflectors, and the elements above the diagonal,
      with the array TAUP, represent the unitary matrix P as
      a product of elementary reflectors.
   See Further Details.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 D
   D is DOUBLE PRECISION array, dimension (min(M,N))
   The diagonal elements of the bidiagonal matrix B: D(i) = A(i,i).

 E
   E is DOUBLE PRECISION array, dimension (min(M,N)-1)
   The off-diagonal elements of the bidiagonal matrix B:
   if m >= n, E(i) = A(i,i+1) for i = 1,2,...,n-1;
   if m < n, E(i) = A(i+1,i) for i = 1,2,...,m-1.

 TAUQ
   TAUQ is COMPLEX*16 array dimension (min(M,N))
   The scalar factors of the elementary reflectors which
   represent the unitary matrix Q. See Further Details.

 TAUP
   TAUP is COMPLEX*16 array, dimension (min(M,N))
   The scalar factors of the elementary reflectors which
   represent the unitary matrix P. See Further Details.

 WORK
   WORK is COMPLEX*16 array, dimension (max(M,N))

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value.

Further Details:

   The matrices Q and P are represented as products of elementary
   reflectors:
```

```
If m >= n,

Q = H(1) H(2) . . . H(n)  and  P = G(1) G(2) . . . G(n-1)

Each H(i) and G(i) has the form:

H(i) = I - tauq * v * v**H  and G(i) = I - taup * u * u**H

where tauq and taup are complex scalars, and v and u are complex
vectors; v(1:i-1) = 0, v(i) = 1, and v(i+1:m) is stored on exit in
A(i+1:m,i); u(1:i) = 0, u(i+1) = 1, and u(i+2:n) is stored on exit in
A(i,i+2:n); tauq is stored in TAUQ(i) and taup in TAUP(i).

If m < n,

Q = H(1) H(2) . . . H(m-1)  and  P = G(1) G(2) . . . G(m)

Each H(i) and G(i) has the form:

H(i) = I - tauq * v * v**H  and G(i) = I - taup * u * u**H

where tauq and taup are complex scalars, v and u are complex vectors;
v(1:i) = 0, v(i+1) = 1, and v(i+2:m) is stored on exit in A(i+2:m,i);
u(1:i-1) = 0, u(i) = 1, and u(i+1:n) is stored on exit in A(i,i+1:n);
tauq is stored in TAUQ(i) and taup in TAUP(i).

The contents of A on exit are illustrated by the following examples:

          m = 6 and n = 5 (m > n):          m = 5 and n = 6 (m < n):

            (  d   e   u1  u1  u1 )           (  d   u1  u1  u1  u1  u1 )
            (  v1  d   e   u2  u2 )           (  e   d   u2  u2  u2  u2 )
            (  v1  v2  d   e   u3 )           (  v1  e   d   u3  u3  u3 )
            (  v1  v2  v3  d   e  )           (  v1  v2  e   d   u4  u4 )
            (  v1  v2  v3  v4  d  )           (  v1  v2  v3  e   d   u5 )
            (  v1  v2  v3  v4  v5 )

where d and e denote diagonal and off-diagonal elements of B, vi
denotes an element of the vector defining H(i), and ui an element of
the vector defining G(i).
```

———-

35.3.3 fortran code

— zgebd2.f —

```
*  =====================================================================
      SUBROUTINE ZGEBD2( M, N, A, LDA, D, E, TAUQ, TAUP, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), E( * )
      COMPLEX*16         A( LDA, * ), TAUP( * ), TAUQ( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ZERO, ONE
      PARAMETER          ( ZERO = ( 0.0D+0, 0.0D+0 ),
     $                   ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      COMPLEX*16         ALPHA
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLACGV, ZLARF, ZLARFG
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DCONJG, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.LT.0 ) THEN
         CALL XERBLA( 'ZGEBD2', -INFO )
         RETURN
      END IF
*
```

```
      IF( M.GE.N ) THEN
*
*        Reduce to upper bidiagonal form
*
         DO 10 I = 1, N
*
*           Generate elementary reflector H(i) to annihilate A(i+1:m,i)
*
            ALPHA = A( I, I )
            CALL ZLARFG( M-I+1, ALPHA, A( MIN( I+1, M ), I ), 1,
     $                   TAUQ( I ) )
            D( I ) = ALPHA
            A( I, I ) = ONE
*
*           Apply H(i)**H to A(i:m,i+1:n) from the left
*
            IF( I.LT.N )
     $         CALL ZLARF( 'Left', M-I+1, N-I, A( I, I ), 1,
     $                     DCONJG( TAUQ( I ) ), A( I, I+1 ), LDA, WORK )
            A( I, I ) = D( I )
*
            IF( I.LT.N ) THEN
*
*              Generate elementary reflector G(i) to annihilate
*              A(i,i+2:n)
*
               CALL ZLACGV( N-I, A( I, I+1 ), LDA )
               ALPHA = A( I, I+1 )
               CALL ZLARFG( N-I, ALPHA, A( I, MIN( I+2, N ) ), LDA,
     $                      TAUP( I ) )
               E( I ) = ALPHA
               A( I, I+1 ) = ONE
*
*              Apply G(i) to A(i+1:m,i+1:n) from the right
*
               CALL ZLARF( 'Right', M-I, N-I, A( I, I+1 ), LDA,
     $                     TAUP( I ), A( I+1, I+1 ), LDA, WORK )
               CALL ZLACGV( N-I, A( I, I+1 ), LDA )
               A( I, I+1 ) = E( I )
            ELSE
               TAUP( I ) = ZERO
            END IF
   10    CONTINUE
      ELSE
*
*        Reduce to lower bidiagonal form
*
         DO 20 I = 1, M
*
*           Generate elementary reflector G(i) to annihilate A(i,i+1:n)
```

```
*
            CALL ZLACGV( N-I+1, A( I, I ), LDA )
            ALPHA = A( I, I )
            CALL ZLARFG( N-I+1, ALPHA, A( I, MIN( I+1, N ) ), LDA,
     $                   TAUP( I ) )
            D( I ) = ALPHA
            A( I, I ) = ONE
*
*           Apply G(i) to A(i+1:m,i:n) from the right
*
            IF( I.LT.M )
     $         CALL ZLARF( 'Right', M-I, N-I+1, A( I, I ), LDA,
     $                     TAUP( I ), A( I+1, I ), LDA, WORK )
            CALL ZLACGV( N-I+1, A( I, I ), LDA )
            A( I, I ) = D( I )
*
            IF( I.LT.M ) THEN
*
*              Generate elementary reflector H(i) to annihilate
*              A(i+2:m,i)
*
               ALPHA = A( I+1, I )
               CALL ZLARFG( M-I, ALPHA, A( MIN( I+2, M ), I ), 1,
     $                      TAUQ( I ) )
               E( I ) = ALPHA
               A( I+1, I ) = ONE
*
*              Apply H(i)**H to A(i+1:m,i+1:n) from the left
*
               CALL ZLARF( 'Left', M-I, N-I, A( I+1, I ), 1,
     $                     DCONJG( TAUQ( I ) ), A( I+1, I+1 ), LDA,
     $                     WORK )
               A( I+1, I ) = E( I )
            ELSE
               TAUQ( I ) = ZERO
            END IF
   20    CONTINUE
      END IF
      RETURN
*
*     End of ZGEBD2
*
      END
```

———-

35.3.4 lisp code

— zgebd2.lisp —

```
(let* ((zero (f2cl-lib:cmplx 0.0d0 0.0d0)) (one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) zero)
           (type (f2cl-lib:complex16) one)
           (ignorable zero one))
  (defun zgebd2 (m n a lda d e tauq taup work info)
    (declare (type (array double-float (*)) e d)
             (type (array f2cl-lib:complex16 (*)) work taup tauq a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (tauq f2cl-lib:complex16 tauq-%data% tauq-%offset%)
         (taup f2cl-lib:complex16 taup-%data% taup-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%))
      (prog ((alpha #C(0.0d0 0.0d0)) (i 0))
        (declare (type (f2cl-lib:complex16) alpha)
                 (type (f2cl-lib:integer4) i))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((< info 0) (xerbla "ZGEBD2" (f2cl-lib:int-sub info)) (go end_label)))
        (cond
         ((>= m n)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf alpha
                      (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                     a-%offset%))
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (zlarfg (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) alpha
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                         ((min (f2cl-lib:int-add i 1) m) i)
                                         ((1 lda) (1 *)) a-%offset%)
                   1
                   (f2cl-lib:array-slice tauq-%data% f2cl-lib:complex16 (i)
                                         ((1 *)) tauq-%offset%))
                (declare (ignore var-0 var-2 var-3 var-4))
                (when var-1 (setf alpha var-1)))
              (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                      (coerce (realpart alpha) 'double-float))
              (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                      one)
              (if (< i n)
```

```
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (zlarf "Left" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
         (f2cl-lib:int-sub n i)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:dconjg
          (f2cl-lib:fref tauq-%data% (i) ((1 *)) tauq-%offset%))
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               (i (f2cl-lib:int-add i 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda work)
      (declare
       (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
      (when var-7 (setf lda var-7))))
(setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
        (coerce (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                'f2cl-lib:complex16))
(cond
 ((< i n)
  (multiple-value-bind (var-0 var-1 var-2)
      (zlacgv (f2cl-lib:int-sub n i)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             (i (f2cl-lib:int-add i 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1))
    (when var-2 (setf lda var-2)))
  (setf alpha
          (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
                         ((1 lda) (1 *)) a-%offset%))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (zlarfg (f2cl-lib:int-sub n i) alpha
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             (i
                              (min
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add i 2))
                               (the f2cl-lib:integer4 n)))
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice taup-%data% f2cl-lib:complex16 (i)
                             ((1 *)) taup-%offset%))
    (declare (ignore var-0 var-2 var-4))
    (when var-1 (setf alpha var-1))
    (when var-3 (setf lda var-3)))
  (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
          (coerce (realpart alpha) 'double-float))
  (setf (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
```

```
                              ((1 lda) (1 *)) a-%offset%)
                  one)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (zlarf "Right" (f2cl-lib:int-sub m i)
               (f2cl-lib:int-sub n i)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                     (i (f2cl-lib:int-add i 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice taup-%data% f2cl-lib:complex16 (i)
                                     ((1 *)) taup-%offset%)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                     ((+ i 1) (f2cl-lib:int-add i 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda work)
            (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8))
            (when var-4 (setf lda var-4))
            (when var-7 (setf lda var-7)))
          (multiple-value-bind (var-0 var-1 var-2)
              (zlacgv (f2cl-lib:int-sub n i)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                     (i (f2cl-lib:int-add i 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda)
            (declare (ignore var-0 var-1))
            (when var-2 (setf lda var-2)))
          (setf (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
                                ((1 lda) (1 *)) a-%offset%)
                  (coerce (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                          'f2cl-lib:complex16)))
         (t
          (setf (f2cl-lib:fref taup-%data% (i) ((1 *)) taup-%offset%)
                  zero)))
      label10)))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i m) nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2)
           (zlacgv (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                  ((1 lda) (1 *)) a-%offset%)
            lda)
         (declare (ignore var-0 var-1))
         (when var-2 (setf lda var-2)))
       (setf alpha
               (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                               a-%offset%))
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
    (zlarfg (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) alpha
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                           (i
                            (min
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add i 1))
                             (the f2cl-lib:integer4 n)))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice taup-%data% f2cl-lib:complex16 (i)
                           ((1 *)) taup-%offset%))
  (declare (ignore var-0 var-2 var-4))
  (when var-1 (setf alpha var-1))
  (when var-3 (setf lda var-3)))
(setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
        (coerce (realpart alpha) 'double-float))
(setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
        one)
(if (< i m)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (zlarf "Right" (f2cl-lib:int-sub m i)
         (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice taup-%data% f2cl-lib:complex16 (i)
                               ((1 *)) taup-%offset%)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               ((+ i 1) i) ((1 lda) (1 *))
                               a-%offset%)
         lda work)
      (declare
       (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8))
      (when var-4 (setf lda var-4))
      (when var-7 (setf lda var-7))))
(multiple-value-bind (var-0 var-1 var-2)
    (zlacgv (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1))
  (when var-2 (setf lda var-2)))
(setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
        (coerce (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                'f2cl-lib:complex16))
(cond
 ((< i m)
  (setf alpha
          (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
```

```
                                ((1 lda) (1 *)) a-%offset%))
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (zlarfg (f2cl-lib:int-sub m i) alpha
                     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                           ((min (f2cl-lib:int-add i 2) m) i)
                                           ((1 lda) (1 *)) a-%offset%)
                     1
                     (f2cl-lib:array-slice tauq-%data% f2cl-lib:complex16 (i)
                                           ((1 *)) tauq-%offset%))
                  (declare (ignore var-0 var-2 var-3 var-4))
                  (when var-1 (setf alpha var-1)))
                (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                        (coerce (realpart alpha) 'double-float))
                (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                                      ((1 lda) (1 *)) a-%offset%)
                        one)
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                    (zlarf "Left" (f2cl-lib:int-sub m i) (f2cl-lib:int-sub n i)
                     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                           ((+ i 1) i) ((1 lda) (1 *))
                                           a-%offset%)
                     1
                     (f2cl-lib:dconjg
                      (f2cl-lib:fref tauq-%data% (i) ((1 *)) tauq-%offset%))
                     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                           ((+ i 1) (f2cl-lib:int-add i 1))
                                           ((1 lda) (1 *)) a-%offset%)
                     lda work)
                  (declare
                   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
                  (when var-7 (setf lda var-7)))
                (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                                      ((1 lda) (1 *)) a-%offset%)
                        (coerce (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                                'f2cl-lib:complex16)))
               (t
                (setf (f2cl-lib:fref tauq-%data% (i) ((1 *)) tauq-%offset%)
                        zero)))
             label20))))
        (go end_label)
       end_label
        (return (values nil nil nil lda nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgebd2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
```

```
'((fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (array fortran-to-lisp::complex16
   (*))
  (array fortran-to-lisp::complex16
   (*))
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil fortran-to-lisp::lda
  nil nil nil nil nil
  fortran-to-lisp::info)
:calls '(fortran-to-lisp::xerbla))))
```

35.4 zgebrd

35.4.1 Axiom unit tests

— zgebrd.input —

```
)set break resume
)sys rm -f zgebrd.output
)spool zgebrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

35.4.2 Axiom help page

— zgebrd.help —

```
====================================================================
zgebrd examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgebrd (M, N, A, LDA, D, E, TAUQ, TAUP, WORK, LWORK, INFO)
           ZGEBRD

Function/Subroutine Documentation
   subroutine zgebrd (integer M, integer N, complex*16, dimension( lda, * ) A,
       integer LDA, double precision, dimension( * ) D, double precision,
       dimension( * ) E, complex*16, dimension( * ) TAUQ, complex*16,
       dimension( * ) TAUP, complex*16, dimension( * ) WORK, integer LWORK,
       integer INFO)
       ZGEBRD

Purpose:

   ZGEBRD reduces a general complex M-by-N matrix A to upper or lower
   bidiagonal form B by a unitary transformation: Q**H * A * P = B.

   If m >= n, B is upper bidiagonal; if m < n, B is lower bidiagonal.

Parameters:

 M
   M is INTEGER
   The number of rows in the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns in the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the M-by-N general matrix to be reduced.
   On exit,
   if m >= n, the diagonal and the first superdiagonal are
      overwritten with the upper bidiagonal matrix B; the
      elements below the diagonal, with the array TAUQ, represent
      the unitary matrix Q as a product of elementary
      reflectors, and the elements above the first superdiagonal,
      with the array TAUP, represent the unitary matrix P as
      a product of elementary reflectors;
   if m < n, the diagonal and the first subdiagonal are
```

```
      overwritten with the lower bidiagonal matrix B; the
      elements below the first subdiagonal, with the array TAUQ,
      represent the unitary matrix Q as a product of
      elementary reflectors, and the elements above the diagonal,
      with the array TAUP, represent the unitary matrix P as
      a product of elementary reflectors.
  See Further Details.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

D
  D is DOUBLE PRECISION array, dimension (min(M,N))
  The diagonal elements of the bidiagonal matrix B: D(i) = A(i,i).

E
  E is DOUBLE PRECISION array, dimension (min(M,N)-1)
  The off-diagonal elements of the bidiagonal matrix B:
  if m >= n, E(i) = A(i,i+1) for i = 1,2,...,n-1;
  if m < n, E(i) = A(i+1,i) for i = 1,2,...,m-1.

TAUQ
  TAUQ is COMPLEX*16 array dimension (min(M,N))
  The scalar factors of the elementary reflectors which
  represent the unitary matrix Q. See Further Details.

TAUP
  TAUP is COMPLEX*16 array, dimension (min(M,N))
  The scalar factors of the elementary reflectors which
  represent the unitary matrix P. See Further Details.

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of the array WORK.  LWORK >= max(1,M,N).
  For optimum performance LWORK >= (M+N)*NB, where NB
  is the optimal blocksize.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit.
```

```
   < 0:  if INFO = -i, the i-th argument had an illegal value.

Further Details:

   The matrices Q and P are represented as products of elementary
   reflectors:

   If m >= n,

   Q = H(1) H(2) . . . H(n)  and  P = G(1) G(2) . . . G(n-1)

   Each H(i) and G(i) has the form:

   H(i) = I - tauq * v * v**H  and G(i) = I - taup * u * u**H

   where tauq and taup are complex scalars, and v and u are complex
   vectors; v(1:i-1) = 0, v(i) = 1, and v(i+1:m) is stored on exit in
   A(i+1:m,i); u(1:i) = 0, u(i+1) = 1, and u(i+2:n) is stored on exit in
   A(i,i+2:n); tauq is stored in TAUQ(i) and taup in TAUP(i).

   If m < n,

   Q = H(1) H(2) . . . H(m-1)  and  P = G(1) G(2) . . . G(m)

   Each H(i) and G(i) has the form:

   H(i) = I - tauq * v * v**H  and G(i) = I - taup * u * u**H

   where tauq and taup are complex scalars, and v and u are complex
   vectors; v(1:i) = 0, v(i+1) = 1, and v(i+2:m) is stored on exit in
   A(i+2:m,i); u(1:i-1) = 0, u(i) = 1, and u(i+1:n) is stored on exit in
   A(i,i+1:n); tauq is stored in TAUQ(i) and taup in TAUP(i).

   The contents of A on exit are illustrated by the following examples:

             m = 6 and n = 5 (m > n):          m = 5 and n = 6 (m < n):

               (  d   e   u1  u1  u1 )           (  d   u1  u1  u1  u1  u1 )
               (  v1  d   e   u2  u2 )           (  e   d   u2  u2  u2  u2 )
               (  v1  v2  d   e   u3 )           (  v1  e   d   u3  u3  u3 )
               (  v1  v2  v3  d   e  )           (  v1  v2  e   d   u4  u4 )
               (  v1  v2  v3  v4  d  )           (  v1  v2  v3  e   d   u5 )
               (  v1  v2  v3  v4  v5 )

   where d and e denote diagonal and off-diagonal elements of B, vi
   denotes an element of the vector defining H(i), and ui an element of
   the vector defining G(i).

          ______-
```

35.4.3 fortran code

— zgebrd.f —

```
*  =====================================================================
      SUBROUTINE ZGEBRD( M, N, A, LDA, D, E, TAUQ, TAUP, WORK, LWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), E( * )
      COMPLEX*16         A( LDA, * ), TAUP( * ), TAUQ( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IINFO, J, LDWRKX, LDWRKY, LWKOPT, MINMN, NB,
     $                   NBMIN, NX
      DOUBLE PRECISION   WS
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGEBD2, ZGEMM, ZLABRD
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      NB = MAX( 1, ILAENV( 1, 'ZGEBRD', ' ', M, N, -1, -1 ) )
```

```
      LWKOPT = ( M+N )*NB
      WORK( 1 ) = DBLE( LWKOPT )
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, M, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -10
      END IF
      IF( INFO.LT.0 ) THEN
         CALL XERBLA( 'ZGEBRD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      MINMN = MIN( M, N )
      IF( MINMN.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      WS = MAX( M, N )
      LDWRKX = M
      LDWRKY = N
*
      IF( NB.GT.1 .AND. NB.LT.MINMN ) THEN
*
*        Set the crossover point NX.
*
         NX = MAX( NB, ILAENV( 3, 'ZGEBRD', ' ', M, N, -1, -1 ) )
*
*        Determine when to switch from blocked to unblocked code.
*
         IF( NX.LT.MINMN ) THEN
            WS = ( M+N )*NB
            IF( LWORK.LT.WS ) THEN
*
*              Not enough work space for the optimal NB, consider using
*              a smaller block size.
*
               NBMIN = ILAENV( 2, 'ZGEBRD', ' ', M, N, -1, -1 )
               IF( LWORK.GE.( M+N )*NBMIN ) THEN
                  NB = LWORK / ( M+N )
               ELSE
```

```
                 NB = 1
                 NX = MINMN
              END IF
           END IF
        END IF
     ELSE
        NX = MINMN
     END IF
*
     DO 30 I = 1, MINMN - NX, NB
*
*       Reduce rows and columns i:i+ib-1 to bidiagonal form and return
*       the matrices X and Y which are needed to update the unreduced
*       part of the matrix
*
        CALL ZLABRD( M-I+1, N-I+1, NB, A( I, I ), LDA, D( I ), E( I ),
     $               TAUQ( I ), TAUP( I ), WORK, LDWRKX,
     $               WORK( LDWRKX*NB+1 ), LDWRKY )
*
*       Update the trailing submatrix A(i+ib:m,i+ib:n), using
*       an update of the form  A := A - V*Y**H - X*U**H
*
        CALL ZGEMM( 'No transpose', 'Conjugate transpose', M-I-NB+1,
     $              N-I-NB+1, NB, -ONE, A( I+NB, I ), LDA,
     $              WORK( LDWRKX*NB+NB+1 ), LDWRKY, ONE,
     $              A( I+NB, I+NB ), LDA )
        CALL ZGEMM( 'No transpose', 'No transpose', M-I-NB+1, N-I-NB+1,
     $              NB, -ONE, WORK( NB+1 ), LDWRKX, A( I, I+NB ), LDA,
     $              ONE, A( I+NB, I+NB ), LDA )
*
*       Copy diagonal and off-diagonal elements of B back into A
*
        IF( M.GE.N ) THEN
           DO 10 J = I, I + NB - 1
              A( J, J ) = D( J )
              A( J, J+1 ) = E( J )
   10      CONTINUE
        ELSE
           DO 20 J = I, I + NB - 1
              A( J, J ) = D( J )
              A( J+1, J ) = E( J )
   20      CONTINUE
        END IF
   30 CONTINUE
*
*    Use unblocked code to reduce the remainder of the matrix
*
     CALL ZGEBD2( M-I+1, N-I+1, A( I, I ), LDA, D( I ), E( I ),
     $            TAUQ( I ), TAUP( I ), WORK, IINFO )
     WORK( 1 ) = WS
```

```
      RETURN
*
*     End of ZGEBRD
*
      END
```

———-

35.4.4 lisp code

— **zgebrd.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (ignorable one))
  (defun zgebrd (m n a lda d e tauq taup work lwork info)
    (declare (type (array double-float (*)) e d)
             (type (array f2cl-lib:complex16 (*)) work taup tauq a)
             (type (f2cl-lib:integer4) info lwork lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (tauq f2cl-lib:complex16 tauq-%data% tauq-%offset%)
         (taup f2cl-lib:complex16 taup-%data% taup-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%))
      (prog ((ws 0.0d0) (i 0) (iinfo 0) (j 0) (ldwrkx 0) (ldwrky 0) (lwkopt 0)
             (minmn 0) (nb 0) (nbmin 0) (nx 0) (lquery nil))
        (declare (type (double-float) ws)
                 (type (f2cl-lib:integer4) i iinfo j ldwrkx ldwrky lwkopt minmn
                  nb nbmin nx)
                 (type f2cl-lib:logical lquery))
        (setf info 0)
        (setf nb
                (max (the f2cl-lib:integer4 1)
                     (the f2cl-lib:integer4
                          (multiple-value-bind
                              (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                               var-6)
                              (ilaenv 1 "ZGEBRD" " " m n -1 -1)
                            (declare (ignore var-0 var-1 var-2 var-5 var-6))
                            (setf m var-3)
                            (setf n var-4)
                            ret-val))))
        (setf lwkopt (f2cl-lib:int-mul (f2cl-lib:int-add m n) nb))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (f2cl-lib:dble lwkopt) 'f2cl-lib:complex16))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
```

```
(cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -4))
      ((and
        (< lwork
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)
                (the f2cl-lib:integer4 n)))
        (not lquery))
       (setf info -10)))
(cond
 ((< info 0) (xerbla "ZGEBRD" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(cond
 ((= minmn 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce 1 'f2cl-lib:complex16))
  (go end_label)))
(setf ws
        (coerce
         (the f2cl-lib:integer4
              (max (the f2cl-lib:integer4 m)
                   (the f2cl-lib:integer4 n)))
         'double-float))
(setf ldwrkx m)
(setf ldwrky n)
(cond
 ((and (> nb 1) (< nb minmn))
  (setf nx
          (max (the f2cl-lib:integer4 nb)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "ZGEBRD" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (setf m var-3)
                      (setf n var-4)
                      ret-val))))
  (cond
   ((< nx minmn)
    (setf ws
            (coerce
             (the f2cl-lib:integer4
                  (f2cl-lib:int-mul (f2cl-lib:int-add m n) nb))
             'double-float))
    (cond
     ((< lwork ws)
      (setf nbmin
```

```
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (ilaenv 2 "ZGEBRD" " " m n -1 -1)
                  (declare (ignore var-0 var-1 var-2 var-5 var-6))
                  (setf m var-3)
                  (setf n var-4)
                  ret-val))
        (cond
         ((>= lwork (f2cl-lib:int-mul (f2cl-lib:int-add m n) nbmin))
          (setf nb (the f2cl-lib:integer4 (truncate lwork (+ m n)))))
         (t (setf nb 1) (setf nx minmn))))))))
  (t (setf nx minmn)))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
               ((> i (f2cl-lib:int-add minmn (f2cl-lib:int-sub nx))) nil)
   (tagbody
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12)
         (zlabrd (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
          (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) nb
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                ((1 lda) (1 *)) a-%offset%)
          lda
          (f2cl-lib:array-slice d-%data% double-float (i) ((1 *))
                                d-%offset%)
          (f2cl-lib:array-slice e-%data% double-float (i) ((1 *))
                                e-%offset%)
          (f2cl-lib:array-slice tauq-%data% f2cl-lib:complex16 (i)
                                ((1 *)) tauq-%offset%)
          (f2cl-lib:array-slice taup-%data% f2cl-lib:complex16 (i)
                                ((1 *)) taup-%offset%)
          work ldwrkx
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                ((+ (f2cl-lib:int-mul ldwrkx nb) 1))
                                ((1 *)) work-%offset%)
          ldwrky)
       (declare
        (ignore var-0 var-1 var-3 var-5 var-6 var-7 var-8 var-9 var-11))
       (when var-2 (setf nb var-2))
       (when var-4 (setf lda var-4))
       (when var-10 (setf ldwrkx var-10))
       (when var-12 (setf ldwrky var-12)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12)
         (zgemm "No transpose" "Conjugate transpose"
          (f2cl-lib:int-add (f2cl-lib:int-sub m i nb) 1)
          (f2cl-lib:int-add (f2cl-lib:int-sub n i nb) 1) nb (- one)
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 ((+ i nb) i)
                                ((1 lda) (1 *)) a-%offset%)
```

```
      lda
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            ((+ (f2cl-lib:int-mul ldwrkx nb) nb 1))
                            ((1 *)) work-%offset%)
      ldwrky one
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                            ((+ i nb) (f2cl-lib:int-add i nb))
                            ((1 lda) (1 *)) a-%offset%)
      lda)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8 var-11))
  (when var-4 (setf nb var-4))
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldwrky var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf lda var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (zgemm "No transpose" "No transpose"
     (f2cl-lib:int-add (f2cl-lib:int-sub m i nb) 1)
     (f2cl-lib:int-add (f2cl-lib:int-sub n i nb) 1) nb (- one)
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            ((+ nb 1)) ((1 *)) work-%offset%)
     ldwrkx
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                            (i (f2cl-lib:int-add i nb))
                            ((1 lda) (1 *)) a-%offset%)
     lda one
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                            ((+ i nb) (f2cl-lib:int-add i nb))
                            ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8 var-11))
  (when var-4 (setf nb var-4))
  (when var-7 (setf ldwrkx var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf lda var-12)))
(cond
 ((>= m n)
  (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                ((> j (f2cl-lib:int-add i nb (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                            a-%offset%)
              (coerce
               (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
```

```
                         'f2cl-lib:complex16))
                 (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-add j 1))
                                       ((1 lda) (1 *)) a-%offset%)
                         (coerce
                          (f2cl-lib:fref e-%data% (j) ((1 *)) e-%offset%)
                          'f2cl-lib:complex16))
                label10)))
            (t
             (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                           ((> j (f2cl-lib:int-add i nb (f2cl-lib:int-sub 1)))
                            nil)
               (tagbody
                 (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                       a-%offset%)
                         (coerce
                          (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                          'f2cl-lib:complex16))
                 (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add j 1) j)
                                       ((1 lda) (1 *)) a-%offset%)
                         (coerce
                          (f2cl-lib:fref e-%data% (j) ((1 *)) e-%offset%)
                          'f2cl-lib:complex16))
                label20))))
          label30))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
            (zgebd2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice d-%data% double-float (i) ((1 *))
                                   d-%offset%)
             (f2cl-lib:array-slice e-%data% double-float (i) ((1 *))
                                   e-%offset%)
             (f2cl-lib:array-slice tauq-%data% f2cl-lib:complex16 (i) ((1 *))
                                   tauq-%offset%)
             (f2cl-lib:array-slice taup-%data% f2cl-lib:complex16 (i) ((1 *))
                                   taup-%offset%)
             work iinfo)
          (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-7 var-8))
          (setf lda var-3)
          (setf iinfo var-9))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce ws 'f2cl-lib:complex16))
        (go end_label)
       end_label
        (return (values m n nil lda nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
```

```
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgebrd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::zgebd2
                                              fortran-to-lisp::xerbla
                                              fortran-to-lisp::ilaenv))))
```

35.5 zgecon

35.5.1 Axiom unit tests

— zgecon.input —

```
)set break resume
)sys rm -f zgecon.output
)spool zgecon.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

35.5.2 Axiom help page

— **zgecon.help** —

```
=====================================================================
zgecon examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgecon (NORM, N, A, LDA, ANORM, RCOND, WORK, RWORK, INFO)
           ZGECON

Function/Subroutine Documentation
   subroutine zgecon (character NORM, integer N, complex*16, dimension( lda, *
       ) A, integer LDA, double precision ANORM, double precision RCOND,
       complex*16, dimension( * ) WORK, double precision, dimension( * )
       RWORK, integer INFO)
       ZGECON

Purpose:
   ZGECON estimates the reciprocal of the condition number of a general
   complex matrix A, in either the 1-norm or the infinity-norm, using
   the LU factorization computed by ZGETRF.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as
      RCOND = 1 / ( norm(A) * norm(inv(A)) ).

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies whether the 1-norm condition number or the
   infinity-norm condition number is required:
   = '1' or 'O':  1-norm;
   = 'I':         Infinity-norm.
```

```
N
  N is INTEGER
  The order of the matrix A.  N >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  The factors L and U from the factorization A = P*L*U
  as computed by ZGETRF.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

ANORM
  ANORM is DOUBLE PRECISION
  If NORM = '1' or 'O', the 1-norm of the original matrix A.
  If NORM = 'I', the infinity-norm of the original matrix A.

RCOND
  RCOND is DOUBLE PRECISION
  The reciprocal of the condition number of the matrix A,
  computed as RCOND = 1/(norm(A) * norm(inv(A))).

WORK
  WORK is COMPLEX*16 array, dimension (2*N)

RWORK
  RWORK is DOUBLE PRECISION array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

35.5.3 fortran code

— zgecon.f —

```
*  =====================================================================
      SUBROUTINE ZGECON( NORM, N, A, LDA, ANORM, RCOND, WORK, RWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
```

```
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            INFO, LDA, N
      DOUBLE PRECISION   ANORM, RCOND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   RWORK( * )
      COMPLEX*16         A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ONENRM
      CHARACTER          NORMIN
      INTEGER            IX, KASE, KASE1
      DOUBLE PRECISION   AINVNM, SCALE, SL, SMLNUM, SU
      COMPLEX*16         ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IZAMAX
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, IZAMAX, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZDRSCL, ZLACN2, ZLATRS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, MAX
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
```

```
      INFO = 0
      ONENRM = NORM.EQ.'1' .OR. LSAME( NORM, 'O' )
      IF( .NOT.ONENRM .AND. .NOT.LSAME( NORM, 'I' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -5
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGECON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
      SMLNUM = DLAMCH( 'Safe minimum' )
*
*     Estimate the norm of inv(A).
*
      AINVNM = ZERO
      NORMIN = 'N'
      IF( ONENRM ) THEN
         KASE1 = 1
      ELSE
         KASE1 = 2
      END IF
      KASE = 0
   10 CONTINUE
      CALL ZLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.KASE1 ) THEN
*
*           Multiply by inv(L).
*
            CALL ZLATRS( 'Lower', 'No transpose', 'Unit', NORMIN, N, A,
     $                   LDA, WORK, SL, RWORK, INFO )
*
*           Multiply by inv(U).
*
```

```
            CALL ZLATRS( 'Upper', 'No transpose', 'Non-unit', NORMIN, N,
     $                   A, LDA, WORK, SU, RWORK( N+1 ), INFO )
         ELSE
*
*           Multiply by inv(U**H).
*
            CALL ZLATRS( 'Upper', 'Conjugate transpose', 'Non-unit',
     $                   NORMIN, N, A, LDA, WORK, SU, RWORK( N+1 ),
     $                   INFO )
*
*           Multiply by inv(L**H).
*
            CALL ZLATRS( 'Lower', 'Conjugate transpose', 'Unit', NORMIN,
     $                   N, A, LDA, WORK, SL, RWORK, INFO )
         END IF
*
*        Divide X by 1/(SL*SU) if doing so will not cause overflow.
*
         SCALE = SL*SU
         NORMIN = 'Y'
         IF( SCALE.NE.ONE ) THEN
            IX = IZAMAX( N, WORK, 1 )
            IF( SCALE.LT.CABS1( WORK( IX ) )*SMLNUM .OR. SCALE.EQ.ZERO )
     $         GO TO 20
            CALL ZDRSCL( N, SCALE, WORK, 1 )
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
   20 CONTINUE
      RETURN
*
*     End of ZGECON
*
      END
```

———-

35.5.4 lisp code

— **zgecon.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0))
```

```
(declare (type (double-float 1.0d0 1.0d0) one)
         (type (double-float 0.0d0 0.0d0) zero)
         (ignorable one zero))
(defun zgecon (norm n a lda anorm rcond work rwork info)
  (declare (type (array double-float (*)) rwork)
           (type (double-float) rcond anorm)
           (type (array f2cl-lib:complex16 (*)) work a)
           (type (f2cl-lib:integer4) info lda n)
           (type (string *) norm))
  (f2cl-lib:with-multi-array-data
      ((norm character norm-%data% norm-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%)
       (rwork double-float rwork-%data% rwork-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
        cabs1))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (zdum #C(0.0d0 0.0d0)) (ainvnm 0.0d0) (scale 0.0d0) (sl 0.0d0)
             (smlnum 0.0d0) (su 0.0d0) (ix 0) (kase 0) (kase1 0)
             (normin
              (make-array '(1) :element-type 'character :initial-element
                          #\ ))
             (onenrm nil))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (f2cl-lib:complex16) zdum)
                 (type (double-float) ainvnm scale sl smlnum su)
                 (type (f2cl-lib:integer4) ix kase kase1)
                 (type (string 1) normin)
                 (type f2cl-lib:logical onenrm))
        (setf info 0)
        (setf onenrm
                (or (f2cl-lib:fstring-= norm "1")
                    (multiple-value-bind (ret-val var-0 var-1)
                        (lsame norm "O")
                      (declare (ignore var-1))
                      (when var-0 (setf norm var-0))
                      ret-val)))
        (cond
         ((and (not onenrm)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame norm "I")
                  (declare (ignore var-1))
                  (when var-0 (setf norm var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2))
```

```
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4))
   ((< anorm zero) (setf info -5)))
  (cond
   ((/= info 0) (xerbla "ZGECON" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf rcond zero)
  (cond ((= n 0) (setf rcond one) (go end_label))
        ((= anorm zero) (go end_label)))
  (setf smlnum (dlamch "Safe minimum"))
  (setf ainvnm zero)
  (f2cl-lib:f2cl-set-string normin "N" (string 1))
  (cond (onenrm (setf kase1 1)) (t (setf kase1 2)))
  (setf kase 0)
 label10
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (zlacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 ((+ n 1))
                             ((1 *)) work-%offset%)
       work ainvnm kase isave)
    (declare (ignore var-1 var-2 var-5))
    (when var-0 (setf n var-0))
    (when var-3 (setf ainvnm var-3))
    (when var-4 (setf kase var-4)))
  (cond
   ((/= kase 0)
    (cond
     ((= kase kase1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (zlatrs "Lower" "No transpose" "Unit" normin n a lda work sl
           rwork info)
        (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
        (when var-3 (setf normin var-3))
        (when var-4 (setf n var-4))
        (when var-6 (setf lda var-6))
        (when var-8 (setf sl var-8))
        (when var-10 (setf info var-10)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (zlatrs "Upper" "No transpose" "Non-unit" normin n a lda work
           su
           (f2cl-lib:array-slice rwork-%data% double-float ((+ n 1))
                                 ((1 *)) rwork-%offset%)
           info)
        (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
        (when var-3 (setf normin var-3))
        (when var-4 (setf n var-4))
```

```
      (when var-6 (setf lda var-6))
      (when var-8 (setf su var-8))
      (when var-10 (setf info var-10))))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (zlatrs "Upper" "Conjugate transpose" "Non-unit" normin n a
          lda work su
          (f2cl-lib:array-slice rwork-%data% double-float ((+ n 1))
                                ((1 *)) rwork-%offset%)
          info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
      (when var-3 (setf normin var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf su var-8))
      (when var-10 (setf info var-10)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (zlatrs "Lower" "Conjugate transpose" "Unit" normin n a lda
          work sl rwork info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
      (when var-3 (setf normin var-3))
      (when var-4 (setf n var-4))
      (when var-6 (setf lda var-6))
      (when var-8 (setf sl var-8))
      (when var-10 (setf info var-10)))))
  (setf scale (* sl su))
  (f2cl-lib:f2cl-set-string normin "Y" (string 1))
  (cond
   ((/= scale one)
    (setf ix
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (izamax n work 1)
              (declare (ignore var-1 var-2))
              (when var-0 (setf n var-0))
              ret-val))
    (if (or
         (< scale
            (*
             (cabs1
              (f2cl-lib:fref work-%data% (ix) ((1 *)) work-%offset%))
             smlnum))
         (= scale zero))
        (go label20))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zdrscl n scale work 1)
      (declare (ignore var-1 var-2 var-3))
```

```
                (setf n var-0))))
            (go label10)))
          (if (/= ainvnm zero)
              (setf rcond (/ (/ one ainvnm) anorm)))
         label20
          (go end_label)
         end_label
          (return (values norm n nil lda nil rcond nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgecon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::norm
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::rcond nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::zdrscl
                                              fortran-to-lisp::xerbla))))
```

———-

35.6 zgeequ

35.6.1 Axiom unit tests

— zgeequ.input —

```
)set break resume
)sys rm -f zgeequ.output
```

```
)spool zgeequ.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

35.6.2 Axiom help page

— zgeequ.help —

```
=====================================================================
zgeequ examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgeequ (M, N, A, LDA, R, C, ROWCND, COLCND, AMAX, INFO)
           ZGEEQU

Function/Subroutine Documentation
   subroutine zgeequ (integer M, integer N, complex*16, dimension( lda, * ) A,
       integer LDA, double precision, dimension( * ) R, double precision,
       dimension( * ) C, double precision ROWCND, double precision COLCND,
       double precision AMAX, integer INFO)
       ZGEEQU

Purpose:
   ZGEEQU computes row and column scalings intended to equilibrate an
   M-by-N matrix A and reduce its condition number.  R returns the row
   scale factors and C the column scale factors, chosen to try to make
   the largest element in each row and column of the matrix B with
   elements B(i,j)=R(i)*A(i,j)*C(j) have absolute value 1.

   R(i) and C(j) are restricted to be between SMLNUM = smallest safe
   number and BIGNUM = largest safe number.  Use of these scaling
   factors is not guaranteed to reduce the condition number of A but
   works well in practice.

Parameters:
```

```
M
  M is INTEGER
  The number of rows of the matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  The M-by-N matrix whose equilibration factors are to be computed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

R
  R is DOUBLE PRECISION array, dimension (M)
  If INFO = 0 or INFO > M, R contains the row scale factors for A.

C
  C is DOUBLE PRECISION array, dimension (N)
  If INFO = 0,  C contains the column scale factors for A.

ROWCND
  ROWCND is DOUBLE PRECISION
  If INFO = 0 or INFO > M, ROWCND contains the ratio of the
  smallest R(i) to the largest R(i).  If ROWCND >= 0.1 and
  AMAX is neither too large nor too small, it is not worth scaling by R.

COLCND
  COLCND is DOUBLE PRECISION
  If INFO = 0, COLCND contains the ratio of the smallest
  C(i) to the largest C(i).  If COLCND >= 0.1, it is not worth scaling by C.

AMAX
  AMAX is DOUBLE PRECISION
  Absolute value of largest matrix element.  If AMAX is very
  close to overflow or very close to underflow, the matrix should be scaled.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i,  and i is
    <= M:  the i-th row of A is exactly zero
    >  M:  the (i-M)-th column of A is exactly zero
```

———-

35.6.3 fortran code

— zgeequ.f —

```
*  =====================================================================
      SUBROUTINE ZGEEQU( M, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
      DOUBLE PRECISION   AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   C( * ), R( * )
      COMPLEX*16         A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   BIGNUM, RCMAX, RCMIN, SMLNUM
      COMPLEX*16         ZDUM
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, MAX, MIN
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
```

```
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEEQU', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         ROWCND = ONE
         COLCND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Get machine constants.
*
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
*
*     Compute row scale factors.
*
      DO 10 I = 1, M
         R( I ) = ZERO
   10 CONTINUE
*
*     Find the maximum element in each row.
*
      DO 30 J = 1, N
         DO 20 I = 1, M
            R( I ) = MAX( R( I ), CABS1( A( I, J ) ) )
   20    CONTINUE
   30 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 40 I = 1, M
```

```
         RCMAX = MAX( RCMAX, R( I ) )
         RCMIN = MIN( RCMIN, R( I ) )
   40 CONTINUE
      AMAX = RCMAX
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 50 I = 1, M
            IF( R( I ).EQ.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   50    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 60 I = 1, M
            R( I ) = ONE / MIN( MAX( R( I ), SMLNUM ), BIGNUM )
   60    CONTINUE
*
*        Compute ROWCND = min(R(I)) / max(R(I))
*
         ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
*     Compute column scale factors
*
      DO 70 J = 1, N
         C( J ) = ZERO
   70 CONTINUE
*
*     Find the maximum element in each column,
*     assuming the row scaling computed above.
*
      DO 90 J = 1, N
         DO 80 I = 1, M
            C( J ) = MAX( C( J ), CABS1( A( I, J ) )*R( I ) )
   80    CONTINUE
   90 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 100 J = 1, N
         RCMIN = MIN( RCMIN, C( J ) )
         RCMAX = MAX( RCMAX, C( J ) )
```

```
  100 CONTINUE
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 110 J = 1, N
            IF( C( J ).EQ.ZERO ) THEN
               INFO = M + J
               RETURN
            END IF
  110    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 120 J = 1, N
            C( J ) = ONE / MIN( MAX( C( J ), SMLNUM ), BIGNUM )
  120    CONTINUE
*
*        Compute COLCND = min(C(J)) / max(C(J))
*
         COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
      RETURN
*
*     End of ZGEEQU
*
      END
```

35.6.4 lisp code

— zgeequ.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun zgeequ (m n a lda r c rowcnd colcnd amax info)
    (declare (type (double-float) amax colcnd rowcnd)
             (type (array double-float (*)) c r)
             (type (array f2cl-lib:complex16 (*)) a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
```

```
    (r double-float r-%data% r-%offset%)
    (c double-float c-%data% c-%offset%))
(labels ((cabs1 (zdum)
           (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
  (declare
   (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
    cabs1))
  (prog ((zdum #C(0.0d0 0.0d0)) (bignum 0.0d0) (rcmax 0.0d0)
         (rcmin 0.0d0) (smlnum 0.0d0) (i 0) (j 0))
    (declare (type (f2cl-lib:complex16) zdum)
             (type (double-float) bignum rcmax rcmin smlnum)
             (type (f2cl-lib:integer4) i j))
    (setf info 0)
    (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
          ((< lda
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
           (setf info -4)))
    (cond
     ((/= info 0) (xerbla "ZGEEQU" (f2cl-lib:int-sub info))
      (go end_label)))
    (cond
     ((or (= m 0) (= n 0)) (setf rowcnd one) (setf colcnd one)
      (setf amax zero) (go end_label)))
    (setf smlnum (dlamch "S"))
    (setf bignum (/ one smlnum))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%) zero)
       label10))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                    (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                         (cabs1
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%))))
           label20))
       label30))
    (setf rcmin bignum)
    (setf rcmax zero)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf rcmax
                (max rcmax
```

```
                    (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
      (setf rcmin
              (min rcmin
                   (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
    label40))
  (setf amax rcmax)
  (cond
   ((= rcmin zero)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (cond
         ((= (f2cl-lib:fref r (i) ((1 *))) zero) (setf info i)
          (go end_label)))
       label50)))
   (t
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                (/ one
                   (min
                    (max
                     (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                     smlnum)
                    bignum)))
       label60))
    (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) zero)
     label70))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                  (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                       (*
                        (cabs1
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%))
                        (f2cl-lib:fref r-%data% (i) ((1 *))
                                       r-%offset%))))
         label80))
     label90))
  (setf rcmin bignum)
```

```
          (setf rcmax zero)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf rcmin
                      (min rcmin
                           (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
              (setf rcmax
                      (max rcmax
                           (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
             label100))
          (cond
           ((= rcmin zero)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (cond
                 ((= (f2cl-lib:fref c (j) ((1 *))) zero)
                  (setf info (f2cl-lib:int-add m j)) (go end_label)))
               label110)))
           (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                        (/ one
                           (min
                            (max
                             (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                             smlnum)
                            bignum)))
               label120))
            (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
          (go end_label)
         end_label
          (return (values nil nil nil nil nil nil rowcnd colcnd amax info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgeequ
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
```

```
  (double-float) (double-float)
  (double-float)
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil nil nil nil
  fortran-to-lisp::rowcnd
  fortran-to-lisp::colcnd
  fortran-to-lisp::amax
  fortran-to-lisp::info)
:calls '(fortran-to-lisp::xerbla))))
```

———-

35.7 zgeequb

35.7.1 Axiom unit tests

— **zgeequb.input** —

```
)set break resume
)sys rm -f zgeequb.output
)spool zgeequb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.7.2 Axiom help page

— **zgeequb.help** —

```
==========================================================================
zgeequb examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine zgehd2 (N, ILO, IHI, A, LDA, TAU, WORK, INFO)
           ZGEHD2 reduces a general square matrix to upper Hessenberg form
           using an unblocked algorithm.

Function/Subroutine Documentation
   subroutine zgehd2 (integer N, integer ILO, integer IHI, complex*16,
       dimension( lda, * ) A, integer LDA, complex*16, dimension( * ) TAU,
       complex*16, dimension( * ) WORK, integer INFO)
       ZGEHD2 reduces a general square matrix to upper Hessenberg form using
       an unblocked algorithm.

Purpose:
   ZGEHD2 reduces a complex general matrix A to upper Hessenberg form H
   by a unitary similarity transformation:  Q**H * A * Q = H .

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 ILO
   ILO is INTEGER

 IHI
   IHI is INTEGER

   It is assumed that A is already upper triangular in rows
   and columns 1:ILO-1 and IHI+1:N. ILO and IHI are normally
   set by a previous call to ZGEBAL; otherwise they should be
   set to 1 and N respectively. See Further Details.
   1 <= ILO <= IHI <= max(1,N).

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the n by n general matrix to be reduced.
   On exit, the upper triangle and the first subdiagonal of A
   are overwritten with the upper Hessenberg matrix H, and the
   elements below the first subdiagonal, with the array TAU,
   represent the unitary matrix Q as a product of elementary
   reflectors. See Further Details.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 TAU
```

```
   TAU is COMPLEX*16 array, dimension (N-1)
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is COMPLEX*16 array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value.

Further Details:

   The matrix Q is represented as a product of (ihi-ilo) elementary
   reflectors

   Q = H(ilo) H(ilo+1) . . . H(ihi-1).

   Each H(i) has the form

   H(i) = I - tau * v * v**H

   where tau is a complex scalar, and v is a complex vector with
   v(1:i) = 0, v(i+1) = 1 and v(ihi+1:n) = 0; v(i+2:ihi) is stored on
   exit in A(i+2:ihi,i), and tau in TAU(i).

   The contents of A are illustrated by the following example, with
   n = 7, ilo = 2 and ihi = 6:

             on entry,                        on exit,

             ( a   a   a   a   a   a   a )    (  a   a   h   h   h   h   a )
             (     a   a   a   a   a   a )    (      a   h   h   h   h   a )
             (     a   a   a   a   a   a )    (      h   h   h   h   h   h )
             (     a   a   a   a   a   a )    (      v2  h   h   h   h   h )
             (     a   a   a   a   a   a )    (      v2  v3  h   h   h   h )
             (     a   a   a   a   a   a )    (      v2  v3  v4  h   h   h )
             (                         a )    (                          a )

   where a denotes an element of the original matrix A, h denotes a
   modified element of the upper Hessenberg matrix H, and vi denotes an
   element of the vector defining H(i).
```

35.7.3 fortran code

— zgeequb.f —

```
*  ====================================================================
      SUBROUTINE ZGEEQUB( M, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $                    INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
      DOUBLE PRECISION   AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   C( * ), R( * )
      COMPLEX*16         A( LDA, * )
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   BIGNUM, RCMAX, RCMIN, SMLNUM, RADIX, LOGRDX
      COMPLEX*16         ZDUM
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, LOG, DBLE, DIMAG
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
```

```
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEEQUB', -INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         ROWCND = ONE
         COLCND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Get machine constants.  Assume SMLNUM is a power of the radix.
*
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      RADIX = DLAMCH( 'B' )
      LOGRDX = LOG( RADIX )
*
*     Compute row scale factors.
*
      DO 10 I = 1, M
         R( I ) = ZERO
   10 CONTINUE
*
*     Find the maximum element in each row.
*
      DO 30 J = 1, N
         DO 20 I = 1, M
            R( I ) = MAX( R( I ), CABS1( A( I, J ) ) )
   20    CONTINUE
   30 CONTINUE
      DO I = 1, M
         IF( R( I ).GT.ZERO ) THEN
            R( I ) = RADIX**INT( LOG(R( I ) ) / LOGRDX )
         END IF
      END DO
*
*     Find the maximum and minimum scale factors.
```

```
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 40 I = 1, M
         RCMAX = MAX( RCMAX, R( I ) )
         RCMIN = MIN( RCMIN, R( I ) )
   40 CONTINUE
      AMAX = RCMAX
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 50 I = 1, M
            IF( R( I ).EQ.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   50    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 60 I = 1, M
            R( I ) = ONE / MIN( MAX( R( I ), SMLNUM ), BIGNUM )
   60    CONTINUE
*
*        Compute ROWCND = min(R(I)) / max(R(I)).
*
         ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
*     Compute column scale factors.
*
      DO 70 J = 1, N
         C( J ) = ZERO
   70 CONTINUE
*
*     Find the maximum element in each column,
*     assuming the row scaling computed above.
*
      DO 90 J = 1, N
         DO 80 I = 1, M
            C( J ) = MAX( C( J ), CABS1( A( I, J ) )*R( I ) )
   80    CONTINUE
         IF( C( J ).GT.ZERO ) THEN
            C( J ) = RADIX**INT( LOG( C( J ) ) / LOGRDX )
         END IF
   90 CONTINUE
*
```

```
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 100 J = 1, N
         RCMIN = MIN( RCMIN, C( J ) )
         RCMAX = MAX( RCMAX, C( J ) )
  100 CONTINUE
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 110 J = 1, N
            IF( C( J ).EQ.ZERO ) THEN
               INFO = M + J
               RETURN
            END IF
  110    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 120 J = 1, N
            C( J ) = ONE / MIN( MAX( C( J ), SMLNUM ), BIGNUM )
  120    CONTINUE
*
*        Compute COLCND = min(C(J)) / max(C(J)).
*
         COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
      RETURN
*
*     End of ZGEEQUB
*
      END
```

———-

35.7.4 lisp code

— zgeequb.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
```

```
(defun zgeequb (m n a lda r c rowcnd colcnd amax info)
  (declare (type (double-float) amax colcnd rowcnd)
           (type (array double-float (*)) c r)
           (type (array f2cl-lib:complex16 (*)) a)
           (type (f2cl-lib:integer4) info lda n m))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex16 a-%data% a-%offset%)
       (r double-float r-%data% r-%offset%)
       (c double-float c-%data% c-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
        cabs1))
      (prog ((zdum #C(0.0d0 0.0d0)) (bignum 0.0d0) (rcmax 0.0d0)
             (rcmin 0.0d0) (smlnum 0.0d0) (radix 0.0d0) (logrdx 0.0d0) (i 0)
             (j 0))
        (declare (type (f2cl-lib:complex16) zdum)
                 (type (double-float) bignum rcmax rcmin smlnum radix logrdx)
                 (type (f2cl-lib:integer4) i j))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "ZGEEQUB" (f2cl-lib:int-sub info))
          (go end_label)))
        (cond
         ((or (= m 0) (= n 0)) (setf rowcnd one) (setf colcnd one)
          (setf amax zero) (go end_label)))
        (setf smlnum (dlamch "S"))
        (setf bignum (/ one smlnum))
        (setf radix (dlamch "B"))
        (setf logrdx (f2cl-lib:flog radix))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%) zero)
           label10))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                        (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                             (cabs1
                              (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
```

```
                                        a-%offset%))))
        label20))
    label30))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref r (i) ((1 *))) zero)
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
              (expt radix
                    (f2cl-lib:int
                     (/
                      (f2cl-lib:flog
                       (f2cl-lib:fref r-%data% (i) ((1 *))
                                      r-%offset%))
                      logrdx))))))
    label100000))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf rcmax
            (max rcmax
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
    (setf rcmin
            (min rcmin
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
    label40))
(setf amax rcmax)
(cond
 ((= rcmin zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref r (i) ((1 *))) zero) (setf info i)
        (go end_label)))
     label50)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
              (/ one
                 (min
                  (max
                   (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                   smlnum)
                  bignum)))
```

```
     label60))
  (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) zero)
   label70))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                     (*
                      (cabs1
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%))
                      (f2cl-lib:fref r-%data% (i) ((1 *))
                                     r-%offset%))))
       label80))
    (cond
     ((> (f2cl-lib:fref c (j) ((1 *))) zero)
      (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
              (expt radix
                    (f2cl-lib:int
                     (/
                      (f2cl-lib:flog
                       (f2cl-lib:fref c-%data% (j) ((1 *))
                                      c-%offset%))
                      logrdx))))))
   label90))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf rcmin
            (min rcmin
                 (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
    (setf rcmax
            (max rcmax
                 (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
   label100))
(cond
 ((= rcmin zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
```

```
                (cond
                 ((= (f2cl-lib:fref c (j) ((1 *))) zero)
                  (setf info (f2cl-lib:int-add m j)) (go end_label)))
               label110)))
           (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                        (/ one
                           (min
                            (max
                             (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                             smlnum)
                            bignum)))
               label120))
            (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
          (go end_label)
         end_label
          (return (values nil nil nil nil nil nil rowcnd colcnd amax info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgeequb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (double-float) (double-float)
                                              (double-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil
                                              fortran-to-lisp::rowcnd
                                              fortran-to-lisp::colcnd
                                              fortran-to-lisp::amax
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

35.8 zgehd2

35.8.1 Axiom unit tests

— **zgehd2.input** —

```
)set break resume
)sys rm -f zgehd2.output
)spool zgehd2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.8.2 Axiom help page

— **zgehd2.help** —

```
======================================================================
zgehd2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgehd2 (N, ILO, IHI, A, LDA, TAU, WORK, INFO)
           ZGEHD2 reduces a general square matrix to upper Hessenberg form
           using an unblocked algorithm.

Function/Subroutine Documentation
   subroutine zgehd2 (integer N, integer ILO, integer IHI, complex*16,
       dimension( lda, * ) A, integer LDA, complex*16, dimension( * ) TAU,
       complex*16, dimension( * ) WORK, integer INFO)
       ZGEHD2 reduces a general square matrix to upper Hessenberg form using
       an unblocked algorithm.

Purpose:
   ZGEHD2 reduces a complex general matrix A to upper Hessenberg form H
   by a unitary similarity transformation:  Q**H * A * Q = H .
```

```
Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 ILO
   ILO is INTEGER

 IHI
   IHI is INTEGER

   It is assumed that A is already upper triangular in rows
   and columns 1:ILO-1 and IHI+1:N. ILO and IHI are normally
   set by a previous call to ZGEBAL; otherwise they should be
   set to 1 and N respectively. See Further Details.
   1 <= ILO <= IHI <= max(1,N).

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the n by n general matrix to be reduced.
   On exit, the upper triangle and the first subdiagonal of A
   are overwritten with the upper Hessenberg matrix H, and the
   elements below the first subdiagonal, with the array TAU,
   represent the unitary matrix Q as a product of elementary
   reflectors. See Further Details.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 TAU
   TAU is COMPLEX*16 array, dimension (N-1)
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is COMPLEX*16 array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value.

Further Details:

   The matrix Q is represented as a product of (ihi-ilo) elementary
   reflectors

   Q = H(ilo) H(ilo+1) . . . H(ihi-1).
```

```
Each H(i) has the form

H(i) = I - tau * v * v**H

where tau is a complex scalar, and v is a complex vector with
v(1:i) = 0, v(i+1) = 1 and v(ihi+1:n) = 0; v(i+2:ihi) is stored on
exit in A(i+2:ihi,i), and tau in TAU(i).

The contents of A are illustrated by the following example, with
n = 7, ilo = 2 and ihi = 6:

          on entry,                        on exit,

          ( a   a   a   a   a   a   a )    (  a   a   h   h   h   h   a )
          (     a   a   a   a   a   a )    (      a   h   h   h   h   a )
          (     a   a   a   a   a   a )    (      h   h   h   h   h   h )
          (     a   a   a   a   a   a )    (      v2  h   h   h   h   h )
          (     a   a   a   a   a   a )    (      v2  v3  h   h   h   h )
          (     a   a   a   a   a   a )    (      v2  v3  v4  h   h   h )
          (                         a )    (                          a )

where a denotes an element of the original matrix A, h denotes a
modified element of the upper Hessenberg matrix H, and vi denotes an
element of the vector defining H(i).
```

———-

35.8.3 fortran code

— **zgehd2.f** —

```
*  =====================================================================
      SUBROUTINE ZGEHD2( N, ILO, IHI, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            IHI, ILO, INFO, LDA, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
```

```
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      COMPLEX*16         ALPHA
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLARF, ZLARFG
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DCONJG, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -2
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEHD2', -INFO )
         RETURN
      END IF
*
      DO 10 I = ILO, IHI - 1
*
*        Compute elementary reflector H(i) to annihilate A(i+2:ihi,i)
*
         ALPHA = A( I+1, I )
         CALL ZLARFG( IHI-I, ALPHA, A( MIN( I+2, N ), I ), 1, TAU( I ) )
         A( I+1, I ) = ONE
*
*        Apply H(i) to A(1:ihi,i+1:ihi) from the right
*
         CALL ZLARF( 'Right', IHI, IHI-I, A( I+1, I ), 1, TAU( I ),
     $               A( 1, I+1 ), LDA, WORK )
*
*        Apply H(i)**H to A(i+1:ihi,i+1:n) from the left
*
```

```
         CALL ZLARF( 'Left', IHI-I, N-I, A( I+1, I ), 1,
     $               DCONJG( TAU( I ) ), A( I+1, I+1 ), LDA, WORK )
*
         A( I+1, I ) = ALPHA
   10 CONTINUE
*
      RETURN
*
*     End of ZGEHD2
*
      END
```

———-

35.8.4 lisp code

— zgehd2.lisp —

———-

35.9 zgehrd

35.9.1 Axiom unit tests

— zgehrd.input —

```
)set break resume
)sys rm -f zgehrd.output
)spool zgehrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.9.2 Axiom help page

— zgehrd.help —

```
====================================================================
zgehrd examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgehrd (N, ILO, IHI, A, LDA, TAU, WORK, LWORK, INFO)
           ZGEHRD

Function/Subroutine Documentation
   subroutine zgehrd (integer N, integer ILO, integer IHI, complex*16,
       dimension( lda, * ) A, integer LDA, complex*16, dimension( * ) TAU,
       complex*16, dimension( * ) WORK, integer LWORK, integer INFO)
       ZGEHRD

Purpose:
   ZGEHRD reduces a complex general matrix A to upper Hessenberg form H by
   an unitary similarity transformation:  Q**H * A * Q = H .

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 ILO
   ILO is INTEGER

 IHI
   IHI is INTEGER

   It is assumed that A is already upper triangular in rows
   and columns 1:ILO-1 and IHI+1:N. ILO and IHI are normally
   set by a previous call to ZGEBAL; otherwise they should be
   set to 1 and N respectively. See Further Details.
   1 <= ILO <= IHI <= N, if N > 0; ILO=1 and IHI=0, if N=0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the N-by-N general matrix to be reduced.
   On exit, the upper triangle and the first subdiagonal of A
   are overwritten with the upper Hessenberg matrix H, and the
   elements below the first subdiagonal, with the array TAU,
   represent the unitary matrix Q as a product of elementary
   reflectors. See Further Details.
```

```
 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 TAU
   TAU is COMPLEX*16 array, dimension (N-1)
   The scalar factors of the elementary reflectors (see Further
   Details). Elements 1:ILO-1 and IHI:N-1 of TAU are set to zero.

 WORK
   WORK is COMPLEX*16 array, dimension (LWORK)
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The length of the array WORK.  LWORK >= max(1,N).
   For good performance, LWORK should generally be larger.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value.

Further Details:

   The matrix Q is represented as a product of (ihi-ilo) elementary
   reflectors

   Q = H(ilo) H(ilo+1) . . . H(ihi-1).

   Each H(i) has the form

   H(i) = I - tau * v * v**H

   where tau is a complex scalar, and v is a complex vector with
   v(1:i) = 0, v(i+1) = 1 and v(ihi+1:n) = 0; v(i+2:ihi) is stored on
   exit in A(i+2:ihi,i), and tau in TAU(i).

   The contents of A are illustrated by the following example, with
   n = 7, ilo = 2 and ihi = 6:

             on entry,                        on exit,

             ( a   a   a   a   a   a   a )    ( a   a   h   h   h   h   a )
             (     a   a   a   a   a   a )    (     a   h   h   h   h   a )
```

```
(     a   a   a   a   a   a )    (      h   h   h   h   h   h )
(     a   a   a   a   a   a )    (      v2  h   h   h   h   h )
(     a   a   a   a   a   a )    (      v2  v3  h   h   h   h )
(     a   a   a   a   a   a )    (      v2  v3  v4  h   h   h )
(                         a )    (                          a )
```

where a denotes an element of the original matrix A, h denotes a modified element of the upper Hessenberg matrix H, and vi denotes an element of the vector defining H(i).

———-

35.9.3 fortran code

— zgehrd.f —

```
*  =====================================================================
      SUBROUTINE ZGEHRD( N, ILO, IHI, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            IHI, ILO, INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16        A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            NBMAX, LDT, TSIZE
      PARAMETER          ( NBMAX = 64, LDT = NBMAX+1,
     $                     TSIZE = LDT*NBMAX )
      COMPLEX*16        ZERO, ONE
      PARAMETER          ( ZERO = ( 0.0D+0, 0.0D+0 ),
     $                     ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWT, J, LDWORK, LWKOPT, NB,
     $                   NBMIN, NH, NX
      COMPLEX*16        EI
*     ..
```

```
*     .. External Subroutines ..
      EXTERNAL           ZAXPY, ZGEHD2, ZGEMM, ZLAHR2, ZLARFB, ZTRMM,
     $                   XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -2
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -8
      END IF
*
      IF( INFO.EQ.0 ) THEN
*
*        Compute the workspace requirements
*
         NB = MIN( NBMAX, ILAENV( 1, 'ZGEHRD', ' ', N, ILO, IHI, -1 ) )
         LWKOPT = N*NB + TSIZE
         WORK( 1 ) = LWKOPT
      ENDIF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEHRD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Set elements 1:ILO-1 and IHI:N-1 of TAU to zero
*
      DO 10 I = 1, ILO - 1
         TAU( I ) = ZERO
   10 CONTINUE
```

```
      DO 20 I = MAX( 1, IHI ), N - 1
         TAU( I ) = ZERO
   20 CONTINUE
*
*     Quick return if possible
*
      NH = IHI - ILO + 1
      IF( NH.LE.1 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
*     Determine the block size
*
      NB = MIN( NBMAX, ILAENV( 1, 'ZGEHRD', ' ', N, ILO, IHI, -1 ) )
      NBMIN = 2
      IF( NB.GT.1 .AND. NB.LT.NH ) THEN
*
*        Determine when to cross over from blocked to unblocked code
*        (last block is always handled by unblocked code)
*
         NX = MAX( NB, ILAENV( 3, 'ZGEHRD', ' ', N, ILO, IHI, -1 ) )
         IF( NX.LT.NH ) THEN
*
*           Determine if workspace is large enough for blocked code
*
            IF( LWORK.LT.N*NB+TSIZE ) THEN
*
*              Not enough workspace to use optimal NB:  determine the
*              minimum value of NB, and reduce NB or force use of
*              unblocked code
*
               NBMIN = MAX( 2, ILAENV( 2, 'ZGEHRD', ' ', N, ILO, IHI,
     $                 -1 ) )
               IF( LWORK.GE.(N*NBMIN + TSIZE) ) THEN
                  NB = (LWORK-TSIZE) / N
               ELSE
                  NB = 1
               END IF
            END IF
         END IF
      END IF
      LDWORK = N
*
      IF( NB.LT.NBMIN .OR. NB.GE.NH ) THEN
*
*        Use unblocked code below
*
         I = ILO
*
```

```
      ELSE
*
*        Use blocked code
*
         IWT = 1 + N*NB
         DO 40 I = ILO, IHI - 1 - NX, NB
            IB = MIN( NB, IHI-I )
*
*           Reduce columns i:i+ib-1 to Hessenberg form, returning the
*           matrices V and T of the block reflector H = I - V*T*V**H
*           which performs the reduction, and also the matrix Y = A*V*T
*
            CALL ZLAHR2( IHI, I, IB, A( 1, I ), LDA, TAU( I ),
     $                   WORK( IWT ), LDT, WORK, LDWORK )
*
*           Apply the block reflector H to A(1:ihi,i+ib:ihi) from the
*           right, computing  A := A - Y * V**H. V(i+ib,ib-1) must be set
*           to 1
*
            EI = A( I+IB, I+IB-1 )
            A( I+IB, I+IB-1 ) = ONE
            CALL ZGEMM( 'No transpose', 'Conjugate transpose',
     $                  IHI, IHI-I-IB+1,
     $                  IB, -ONE, WORK, LDWORK, A( I+IB, I ), LDA, ONE,
     $                  A( 1, I+IB ), LDA )
            A( I+IB, I+IB-1 ) = EI
*
*           Apply the block reflector H to A(1:i,i+1:i+ib-1) from the
*           right
*
            CALL ZTRMM( 'Right', 'Lower', 'Conjugate transpose',
     $                  'Unit', I, IB-1,
     $                  ONE, A( I+1, I ), LDA, WORK, LDWORK )
            DO 30 J = 0, IB-2
               CALL ZAXPY( I, -ONE, WORK( LDWORK*J+1 ), 1,
     $                     A( 1, I+J+1 ), 1 )
   30       CONTINUE
*
*           Apply the block reflector H to A(i+1:ihi,i+ib:n) from the
*           left
*
            CALL ZLARFB( 'Left', 'Conjugate transpose', 'Forward',
     $                   'Columnwise',
     $                   IHI-I, N-I-IB+1, IB, A( I+1, I ), LDA,
     $                   WORK( IWT ), LDT, A( I+1, I+IB ), LDA,
     $                   WORK, LDWORK )
   40    CONTINUE
      END IF
*
*     Use unblocked code to reduce the rest of the matrix
```

```
*
      CALL ZGEHD2( N, I, IHI, A, LDA, TAU, WORK, IINFO )
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of ZGEHRD
*
      END
```

———-

35.9.4 lisp code

— **zgehrd.lisp** —

———-

35.10 zgelq2

35.10.1 Axiom unit tests

— **zgelq2.input** —

```
)set break resume
)sys rm -f zgelq2.output
)spool zgelq2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.10.2 Axiom help page

— **zgelq2.help** —

```
====================================================================
zgelq2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgelq2 (M, N, A, LDA, TAU, WORK, INFO)
           ZGELQ2 computes the LQ factorization of a general rectangular
           matrix using an unblocked algorithm.

Function/Subroutine Documentation
   subroutine zgelq2 (integer M, integer N, complex*16, dimension( lda, * ) A,
       integer LDA, complex*16, dimension( * ) TAU, complex*16, dimension( * )
       WORK, integer INFO)
       ZGELQ2 computes the LQ factorization of a general rectangular matrix
       using an unblocked algorithm.

Purpose:
   ZGELQ2 computes an LQ factorization of a complex m by n matrix A:
   A = L * Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the m by n matrix A.
   On exit, the elements on and below the diagonal of the array
   contain the m by min(m,n) lower trapezoidal matrix L (L is
   lower triangular if m <= n); the elements above the diagonal,
   with the array TAU, represent the unitary matrix Q as a
   product of elementary reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is COMPLEX*16 array, dimension (min(M,N))
```

```
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is COMPLEX*16 array, dimension (M)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

Further Details:

   The matrix Q is represented as a product of elementary reflectors

   Q = H(k)**H . . . H(2)**H H(1)**H, where k = min(m,n).

   Each H(i) has the form

   H(i) = I - tau * v * v**H

   where tau is a complex scalar, and v is a complex vector with
   v(1:i-1) = 0 and v(i) = 1; conjg(v(i+1:n)) is stored on exit in
   A(i,i+1:n), and tau in TAU(i).
```

———-

35.10.3 fortran code

— **zgelq2.f** —

```
*  =====================================================================
      SUBROUTINE ZGELQ2( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
```

```
*     .. Parameters ..
      COMPLEX*16         ONE
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      COMPLEX*16         ALPHA
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLACGV, ZLARF, ZLARFG
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGELQ2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = 1, K
*
*        Generate elementary reflector H(i) to annihilate A(i,i+1:n)
*
         CALL ZLACGV( N-I+1, A( I, I ), LDA )
         ALPHA = A( I, I )
         CALL ZLARFG( N-I+1, ALPHA, A( I, MIN( I+1, N ) ), LDA,
     $                TAU( I ) )
         IF( I.LT.M ) THEN
*
*           Apply H(i) to A(i+1:m,i:n) from the right
*
            A( I, I ) = ONE
            CALL ZLARF( 'Right', M-I, N-I+1, A( I, I ), LDA, TAU( I ),
     $                  A( I+1, I ), LDA, WORK )
         END IF
         A( I, I ) = ALPHA
```

```
         CALL ZLACGV( N-I+1, A( I, I ), LDA )
   10 CONTINUE
      RETURN
*
*     End of ZGELQ2
*
      END
```

———-

35.10.4 lisp code

— **zgelq2.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (ignorable one))
  (defun zgelq2 (m n a lda tau work info)
    (declare (type (array f2cl-lib:complex16 (*)) work tau a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%))
      (prog ((alpha #C(0.0d0 0.0d0)) (i 0) (k 0))
        (declare (type (f2cl-lib:complex16) alpha)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "ZGELQ2" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i k) nil)
          (tagbody
            (multiple-value-bind (var-0 var-1 var-2)
                (zlacgv (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare (ignore var-0 var-1))
              (when var-2 (setf lda var-2)))
            (setf alpha
                    (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%))
```

```
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (zlarfg (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) alpha
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                       (i
                                        (min
                                         (the f2cl-lib:integer4
                                              (f2cl-lib:int-add i 1))
                                         (the f2cl-lib:integer4 n)))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                                       ((1 *)) tau-%offset%))
              (declare (ignore var-0 var-2 var-4))
              (when var-1 (setf alpha var-1))
              (when var-3 (setf lda var-3)))
            (cond
             ((< i m)
              (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                      one)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                  (zlarf "Right" (f2cl-lib:int-sub m i)
                   (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda
                   (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                                         ((1 *)) tau-%offset%)
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                         ((+ i 1) i) ((1 lda) (1 *))
                                         a-%offset%)
                   lda work)
                (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8))
                (when var-4 (setf lda var-4))
                (when var-7 (setf lda var-7)))))
            (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                    alpha)
            (multiple-value-bind (var-0 var-1 var-2)
                (zlacgv (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare (ignore var-0 var-1))
              (when var-2 (setf lda var-2)))
          label10))
        (go end_label)
       end_label
        (return (values nil nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
```

```
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgelq2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::lda
                                              nil nil fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

35.11 zgelqf

35.11.1 Axiom unit tests

— **zgelqf.input** —

```
)set break resume
)sys rm -f zgelqf.output
)spool zgelqf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.11.2 Axiom help page

— **zgelqf.help** —

```
====================================================================
zgelqf examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgelqf (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           ZGELQF

Function/Subroutine Documentation
   subroutine zgelqf (integer M, integer N, complex*16, dimension( lda, * ) A,
       integer LDA, complex*16, dimension( * ) TAU, complex*16, dimension( * )
       WORK, integer LWORK, integer INFO)
       ZGELQF

Purpose:
   ZGELQF computes an LQ factorization of a complex M-by-N matrix A:
   A = L * Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, the elements on and below the diagonal of the array
   contain the m-by-min(m,n) lower trapezoidal matrix L (L is
   lower triangular if m <= n); the elements above the diagonal,
   with the array TAU, represent the unitary matrix Q as a
   product of elementary reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is COMPLEX*16 array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).
```

```
 WORK
   WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.  LWORK >= max(1,M).
   For optimum performance LWORK >= M*NB, where NB is the
   optimal blocksize.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details:

   The matrix Q is represented as a product of elementary reflectors

   Q = H(k)**H . . . H(2)**H H(1)**H, where k = min(m,n).

   Each H(i) has the form

   H(i) = I - tau * v * v**H

   where tau is a complex scalar, and v is a complex vector with
   v(1:i-1) = 0 and v(i) = 1; conjg(v(i+1:n)) is stored on exit in
   A(i,i+1:n), and tau in TAU(i).
```

———-

35.11.3 fortran code

— **zgelqf.f** —

```
*  =====================================================================
      SUBROUTINE ZGELQF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
```

```
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, LDWORK, LWKOPT, NB,
     $                   NBMIN, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGELQ2, ZLARFB, ZLARFT
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NB = ILAENV( 1, 'ZGELQF', ' ', M, N, -1, -1 )
      LWKOPT = M*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, M ) .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGELQF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
```

```
*     Quick return if possible
*
      K = MIN( M, N )
      IF( K.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      NX = 0
      IWS = M
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'ZGELQF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = M
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'ZGELQF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially
*
         DO 10 I = 1, K - NX, NB
            IB = MIN( K-I+1, NB )
*
*           Compute the LQ factorization of the current block
*           A(i:i+ib-1,i:n)
*
            CALL ZGELQ2( IB, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                   IINFO )
            IF( I+IB.LE.M ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i) H(i+1) . . . H(i+ib-1)
```

```
*
              CALL ZLARFT( 'Forward', 'Rowwise', N-I+1, IB, A( I, I ),
     $                     LDA, TAU( I ), WORK, LDWORK )
*
*             Apply H to A(i+ib:m,i:n) from the right
*
              CALL ZLARFB( 'Right', 'No transpose', 'Forward',
     $                     'Rowwise', M-I-IB+1, N-I+1, IB, A( I, I ),
     $                     LDA, WORK, LDWORK, A( I+IB, I ), LDA,
     $                     WORK( IB+1 ), LDWORK )
           END IF
   10    CONTINUE
      ELSE
         I = 1
      END IF
*
*     Use unblocked code to factor the last or only block.
*
      IF( I.LE.K )
     $   CALL ZGELQ2( M-I+1, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                IINFO )
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of ZGELQF
*
      END
```

———-

35.11.4 lisp code

— **zgelqf.lisp** —

```
(defun zgelqf (m n a lda tau work lwork info)
  (declare (type (array f2cl-lib:complex16 (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex16 a-%data% a-%offset%)
       (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ldwork 0) (lwkopt 0) (nb 0)
           (nbmin 0) (nx 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) nx nbmin nb lwkopt ldwork k iws iinfo
                ib i))
      (setf info 0)
```

```
(setf nb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 1 "ZGELQF" " " m n -1 -1)
          (declare (ignore var-0 var-1 var-2 var-5 var-6))
          (setf m var-3)
          (setf n var-4)
          ret-val))
(setf lwkopt (f2cl-lib:int-mul m nb))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce lwkopt 'f2cl-lib:complex16))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
      ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -4))
      ((and
        (< lwork
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
        (not lquery))
       (setf info -7)))
(cond
 ((/= info 0) (xerbla "ZGELQF" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(cond
 ((= k 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce 1 'f2cl-lib:complex16))
  (go end_label)))
(setf nbmin 2)
(setf nx 0)
(setf iws m)
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "ZGELQF" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (setf m var-3)
                      (setf n var-4)
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork m) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
```

```
        (setf nbmin
                (max (the f2cl-lib:integer4 2)
                     (the f2cl-lib:integer4
                          (multiple-value-bind
                              (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                               var-6)
                              (ilaenv 2 "ZGELQF" " " m n -1 -1)
                            (declare
                             (ignore var-0 var-1 var-2 var-5 var-6))
                            (setf m var-3)
                            (setf n var-4)
                            ret-val))))))))))
  (cond
   ((and (>= nb nbmin) (< nb k) (< nx k))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                  ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub nx))) nil)
      (tagbody
        (setf ib
                (min
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
                 (the f2cl-lib:integer4 nb)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zgelq2 ib (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                                   ((1 *)) tau-%offset%)
             work iinfo)
          (declare (ignore var-0 var-1 var-2 var-4 var-5))
          (setf lda var-3)
          (setf iinfo var-6))
        (cond
         ((<= (f2cl-lib:int-add i ib) m)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (zlarft "Forward" "Rowwise"
               (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) ib
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                                     ((1 *)) tau-%offset%)
               work ldwork)
            (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
            (when var-3 (setf ib var-3))
            (when var-5 (setf lda var-5))
            (when var-8 (setf ldwork var-8)))
          (multiple-value-bind
```

```
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                   var-10 var-11 var-12 var-13 var-14)
                  (zlarfb "Right" "No transpose" "Forward" "Rowwise"
                   (f2cl-lib:int-add (f2cl-lib:int-sub m i ib) 1)
                   (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) ib
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda work ldwork
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                         ((+ i ib) i) ((1 lda) (1 *))
                                         a-%offset%)
                   lda
                   (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                         ((+ ib 1)) ((1 *)) work-%offset%)
                   ldwork)
                 (declare
                  (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
                   var-13))
                 (when var-6 (setf ib var-6))
                 (when var-8 (setf lda var-8))
                 (when var-10 (setf ldwork var-10))
                 (when var-12 (setf lda var-12))
                 (when var-14 (setf ldwork var-14)))))
           label10)))
       (t (setf i 1)))
      (if (<= i k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (zgelq2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
               (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i) ((1 *))
                                     tau-%offset%)
               work iinfo)
            (declare (ignore var-0 var-1 var-2 var-4 var-5))
            (setf lda var-3)
            (setf iinfo var-6)))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce iws 'f2cl-lib:complex16))
      (go end_label)
     end_label
      (return (values m n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgelqf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
```

```
'((fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::m
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zgelq2
  fortran-to-lisp::xerbla
  fortran-to-lisp::ilaenv))))
```

———-

35.12 zgemqrt

35.12.1 Axiom unit tests

— zgemqrt.input —

```
)set break resume
)sys rm -f zgemqrt.output
)spool zgemqrt.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.12.2 Axiom help page

— zgemqrt.help —

```
====================================================================
zgemqrt examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgemqrt (SIDE, TRANS, M, N, K, NB, V, LDV, T, LDT, C, LDC,
           WORK, INFO)
           ZGEMQRT

Function/Subroutine Documentation
   subroutine zgemqrt (character SIDE, character TRANS, integer M, integer N,
       integer K, integer NB, complex*16, dimension( ldv, * ) V, integer LDV,
       complex*16, dimension( ldt, * ) T, integer LDT, complex*16, dimension(
       ldc, * ) C, integer LDC, complex*16, dimension( * ) WORK, integer INFO)
       ZGEMQRT

Purpose:
   ZGEMQRT overwrites the general complex M-by-N matrix C with

                             SIDE = 'L'     SIDE = 'R'
             TRANS = 'N':      Q C            C Q
             TRANS = 'C':    Q**H C            C Q**H

   where Q is a complex orthogonal matrix defined as the product of K
   elementary reflectors:

   Q = H(1) H(2) . . . H(K) = I - V T V**H

   generated using the compact WY representation as returned by ZGEQRT.

   Q is of order M if SIDE = 'L' and of order N  if SIDE = 'R'.

Parameters:

 SIDE
   SIDE is CHARACTER*1
   = 'L': apply Q or Q**H from the Left;
   = 'R': apply Q or Q**H from the Right.

 TRANS
   TRANS is CHARACTER*1
   = 'N':  No transpose, apply Q;
   = 'C':  Transpose, apply Q**H.
```

```
M
  M is INTEGER
  The number of rows of the matrix C. M >= 0.

N
  N is INTEGER
  The number of columns of the matrix C. N >= 0.

K
  K is INTEGER
  The number of elementary reflectors whose product defines
  the matrix Q.
  If SIDE = 'L', M >= K >= 0;
  if SIDE = 'R', N >= K >= 0.

NB
  NB is INTEGER
  The block size used for the storage of T.  K >= NB >= 1.
  This must be the same value of NB used to generate T in CGEQRT.

V
  V is COMPLEX*16 array, dimension (LDV,K)
  The i-th column must contain the vector which defines the
  elementary reflector H(i), for i = 1,2,...,k, as returned by
  CGEQRT in the first K columns of its array argument A.

LDV
  LDV is INTEGER
  The leading dimension of the array V.
  If SIDE = 'L', LDA >= max(1,M);
  if SIDE = 'R', LDA >= max(1,N).

T
  T is COMPLEX*16 array, dimension (LDT,K)
  The upper triangular factors of the block reflectors
  as returned by CGEQRT, stored as a NB-by-N matrix.

LDT
  LDT is INTEGER
  The leading dimension of the array T.  LDT >= NB.

C
  C is COMPLEX*16 array, dimension (LDC,N)
  On entry, the M-by-N matrix C.
  On exit, C is overwritten by Q C, Q**H C, C Q**H or C Q.

LDC
  LDC is INTEGER
  The leading dimension of the array C. LDC >= max(1,M).
```

```
 WORK
   WORK is COMPLEX*16 array. The dimension of WORK is
   N*NB if SIDE = 'L', or  M*NB if SIDE = 'R'.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

35.12.3 fortran code

— **zgemqrt.f** —

```
*  =====================================================================
      SUBROUTINE ZGEMQRT( SIDE, TRANS, M, N, K, NB, V, LDV, T, LDT,
     $                    C, LDC, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER SIDE, TRANS
      INTEGER   INFO, K, LDV, LDC, M, N, NB, LDT
*     ..
*     .. Array Arguments ..
      COMPLEX*16   V( LDV, * ), C( LDC, * ), T( LDT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     ..
*     .. Local Scalars ..
      LOGICAL            LEFT, RIGHT, TRAN, NOTRAN
      INTEGER            I, IB, LDWORK, KF, Q
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLARFB
*     ..
```

```
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     .. Test the input arguments ..
*
      INFO   = 0
      LEFT   = LSAME( SIDE,  'L' )
      RIGHT  = LSAME( SIDE,  'R' )
      TRAN   = LSAME( TRANS, 'C' )
      NOTRAN = LSAME( TRANS, 'N' )
*
      IF( LEFT ) THEN
         LDWORK = MAX( 1, N )
         Q = M
      ELSE IF ( RIGHT ) THEN
         LDWORK = MAX( 1, M )
         Q = N
      END IF
      IF( .NOT.LEFT .AND. .NOT.RIGHT ) THEN
         INFO = -1
      ELSE IF( .NOT.TRAN .AND. .NOT.NOTRAN ) THEN
         INFO = -2
      ELSE IF( M.LT.0 ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( K.LT.0 .OR. K.GT.Q ) THEN
         INFO = -5
      ELSE IF( NB.LT.1 .OR. (NB.GT.K .AND. K.GT.0)) THEN
         INFO = -6
      ELSE IF( LDV.LT.MAX( 1, Q ) ) THEN
         INFO = -8
      ELSE IF( LDT.LT.NB ) THEN
         INFO = -10
      ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
         INFO = -12
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEMQRT', -INFO )
         RETURN
      END IF
*
*     .. Quick return if possible ..
*
      IF( M.EQ.0 .OR. N.EQ.0 .OR. K.EQ.0 ) RETURN
*
      IF( LEFT .AND. TRAN ) THEN
```

```
*
         DO I = 1, K, NB
            IB = MIN( NB, K-I+1 )
            CALL ZLARFB( 'L', 'C', 'F', 'C', M-I+1, N, IB,
     $                   V( I, I ), LDV, T( 1, I ), LDT,
     $                   C( I, 1 ), LDC, WORK, LDWORK )
         END DO
*
      ELSE IF( RIGHT .AND. NOTRAN ) THEN
*
         DO I = 1, K, NB
            IB = MIN( NB, K-I+1 )
            CALL ZLARFB( 'R', 'N', 'F', 'C', M, N-I+1, IB,
     $                   V( I, I ), LDV, T( 1, I ), LDT,
     $                   C( 1, I ), LDC, WORK, LDWORK )
         END DO
*
      ELSE IF( LEFT .AND. NOTRAN ) THEN
*
         KF = ((K-1)/NB)*NB+1
         DO I = KF, 1, -NB
            IB = MIN( NB, K-I+1 )
            CALL ZLARFB( 'L', 'N', 'F', 'C', M-I+1, N, IB,
     $                   V( I, I ), LDV, T( 1, I ), LDT,
     $                   C( I, 1 ), LDC, WORK, LDWORK )
         END DO
*
      ELSE IF( RIGHT .AND. TRAN ) THEN
*
         KF = ((K-1)/NB)*NB+1
         DO I = KF, 1, -NB
            IB = MIN( NB, K-I+1 )
            CALL ZLARFB( 'R', 'C', 'F', 'C', M, N-I+1, IB,
     $                   V( I, I ), LDV, T( 1, I ), LDT,
     $                   C( 1, I ), LDC, WORK, LDWORK )
         END DO
*
      END IF
*
      RETURN
*
*     End of ZGEMQRT
*
      END
```

———-

35.12.4 lisp code

— zgemqrt.lisp —

```
(defun zgemqrt (side trans m n k nb v ldv t$ ldt c ldc work info)
  (declare (type (array f2cl-lib:complex16 (*)) work c t$ v)
           (type (f2cl-lib:integer4) info ldc ldt ldv nb k n m)
           (type (string *) trans side))
  (f2cl-lib:with-multi-array-data
      ((side character side-%data% side-%offset%)
       (trans character trans-%data% trans-%offset%)
       (v f2cl-lib:complex16 v-%data% v-%offset%)
       (t$ f2cl-lib:complex16 t$-%data% t$-%offset%)
       (c f2cl-lib:complex16 c-%data% c-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (ldwork 0) (kf 0) (q 0) (left nil) (right nil)
           (tran nil) (notran nil))
      (declare (type f2cl-lib:logical notran tran right left)
               (type (f2cl-lib:integer4) q kf ldwork ib i))
      (setf info 0)
      (setf left
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame side "L")
                (declare (ignore var-1))
                (when var-0 (setf side var-0))
                ret-val))
      (setf right
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame side "R")
                (declare (ignore var-1))
                (when var-0 (setf side var-0))
                ret-val))
      (setf tran
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "C")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val))
      (setf notran
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "N")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val))
      (cond
       (left
        (setf ldwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf q m))
       (right
```

```
  (setf ldwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf q n)))
(cond ((and (not left) (not right)) (setf info -1))
      ((and (not tran) (not notran)) (setf info -2))
      ((< m 0) (setf info -3)) ((< n 0) (setf info -4))
      ((or (< k 0) (> k q)) (setf info -5))
      ((or (< nb 1) (and (> nb k) (> k 0))) (setf info -6))
      ((< ldv (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 q)))
       (setf info -8))
      ((< ldt nb) (setf info -10))
      ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -12)))
(cond
 ((/= info 0) (xerbla "ZGEMQRT" (f2cl-lib:int-sub info)) (go end_label)))
(if (or (= m 0) (= n 0) (= k 0))
    (go end_label))
(cond
 ((and left tran)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                ((> i k) nil)
    (tagbody
      (setf ib
              (min (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13 var-14)
          (zlarfb "L" "C" "F" "C"
           (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) n ib
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (i i)
                                 ((1 ldv) (1 *)) v-%offset%)
           ldv
           (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 i)
                                 ((1 ldt) (1 *)) t$-%offset%)
           ldt
           (f2cl-lib:array-slice c-%data% f2cl-lib:complex16 (i 1)
                                 ((1 ldc) (1 *)) c-%offset%)
           ldc work ldwork)
        (declare
         (ignore var-0 var-1 var-2 var-3 var-4 var-7 var-9 var-11
          var-13))
        (when var-5 (setf n var-5))
        (when var-6 (setf ib var-6))
        (when var-8 (setf ldv var-8))
        (when var-10 (setf ldt var-10))
        (when var-12 (setf ldc var-12))
        (when var-14 (setf ldwork var-14)))
    label100000)))
 ((and right notran)
```

```
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
              ((> i k) nil)
  (tagbody
    (setf ib
            (min (the f2cl-lib:integer4 nb)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14)
        (zlarfb "R" "N" "F" "C" m
         (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) ib
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (i i)
                               ((1 ldv) (1 *)) v-%offset%)
         ldv
         (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 i)
                               ((1 ldt) (1 *)) t$-%offset%)
         ldt
         (f2cl-lib:array-slice c-%data% f2cl-lib:complex16 (1 i)
                               ((1 ldc) (1 *)) c-%offset%)
         ldc work ldwork)
      (declare
       (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9 var-11
        var-13))
      (when var-4 (setf m var-4))
      (when var-6 (setf ib var-6))
      (when var-8 (setf ldv var-8))
      (when var-10 (setf ldt var-10))
      (when var-12 (setf ldc var-12))
      (when var-14 (setf ldwork var-14)))
   label100001)))
((and left notran)
 (setf kf (+ (* (the f2cl-lib:integer4 (truncate (- k 1) nb)) nb) 1))
 (f2cl-lib:fdo (i kf (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
               ((> i 1) nil)
  (tagbody
    (setf ib
            (min (the f2cl-lib:integer4 nb)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14)
        (zlarfb "L" "N" "F" "C"
         (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) n ib
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (i i)
                               ((1 ldv) (1 *)) v-%offset%)
         ldv
         (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 i)
                               ((1 ldt) (1 *)) t$-%offset%)
```

```
            ldt
            (f2cl-lib:array-slice c-%data% f2cl-lib:complex16 (i 1)
                                  ((1 ldc) (1 *)) c-%offset%)
            ldc work ldwork)
          (declare
           (ignore var-0 var-1 var-2 var-3 var-4 var-7 var-9 var-11
            var-13))
          (when var-5 (setf n var-5))
          (when var-6 (setf ib var-6))
          (when var-8 (setf ldv var-8))
          (when var-10 (setf ldt var-10))
          (when var-12 (setf ldc var-12))
          (when var-14 (setf ldwork var-14)))
      label100002)))
  ((and right tran)
   (setf kf (+ (* (the f2cl-lib:integer4 (truncate (- k 1) nb)) nb) 1))
   (f2cl-lib:fdo (i kf (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
                 ((> i 1) nil)
     (tagbody
       (setf ib
               (min (the f2cl-lib:integer4 nb)
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10 var-11 var-12 var-13 var-14)
           (zlarfb "R" "C" "F" "C" m
            (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) ib
            (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (i i)
                                  ((1 ldv) (1 *)) v-%offset%)
            ldv
            (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 i)
                                  ((1 ldt) (1 *)) t$-%offset%)
            ldt
            (f2cl-lib:array-slice c-%data% f2cl-lib:complex16 (1 i)
                                  ((1 ldc) (1 *)) c-%offset%)
            ldc work ldwork)
          (declare
           (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9 var-11
            var-13))
          (when var-4 (setf m var-4))
          (when var-6 (setf ib var-6))
          (when var-8 (setf ldv var-8))
          (when var-10 (setf ldt var-10))
          (when var-12 (setf ldc var-12))
          (when var-14 (setf ldwork var-14)))
      label100003))))
 (go end_label)
end_label
 (return
```

```
      (values side trans m n nil nil nil ldv nil ldt nil ldc nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgemqrt
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::side
                                              fortran-to-lisp::trans
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::ldv nil
                                              fortran-to-lisp::ldt nil
                                              fortran-to-lisp::ldc nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

35.13 zgeql2

35.13.1 Axiom unit tests

— zgeql2.input —

```
)set break resume
```

```
)sys rm -f zgeql2.output
)spool zgeql2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

35.13.2 Axiom help page

— zgeql2.help —

```
=====================================================================
zgeql2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgeql2 (M, N, A, LDA, TAU, WORK, INFO)
           ZGEQL2 computes the QL factorization of a general rectangular
           matrix using an unblocked algorithm.

Function/Subroutine Documentation
   subroutine zgeql2 (integer M, integer N, complex*16, dimension( lda, * ) A,
       integer LDA, complex*16, dimension( * ) TAU, complex*16, dimension( * )
       WORK, integer INFO)
       ZGEQL2 computes the QL factorization of a general rectangular matrix
       using an unblocked algorithm.

Purpose:
   ZGEQL2 computes a QL factorization of a complex m by n matrix A:
   A = Q * L.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
```

```
  The number of columns of the matrix A.  N >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the m by n matrix A.
  On exit, if m >= n, the lower triangle of the subarray
  A(m-n+1:m,1:n) contains the n by n lower triangular matrix L;
  if m <= n, the elements on and below the (n-m)-th
  superdiagonal contain the m by n lower trapezoidal matrix L;
  the remaining elements, with the array TAU, represent the
  unitary matrix Q as a product of elementary reflectors
  (see Further Details).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

TAU
  TAU is COMPLEX*16 array, dimension (min(M,N))
  The scalar factors of the elementary reflectors (see Further Details).

WORK
  WORK is COMPLEX*16 array, dimension (N)

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value

Further Details:

  The matrix Q is represented as a product of elementary reflectors

  Q = H(k) . . . H(2) H(1), where k = min(m,n).

  Each H(i) has the form

  H(i) = I - tau * v * v**H

  where tau is a complex scalar, and v is a complex vector with
  v(m-k+i+1:m) = 0 and v(m-k+i) = 1; v(1:m-k+i-1) is stored on exit in
  A(1:m-k+i-1,n-k+i), and tau in TAU(i).
```

———-

35.13.3 fortran code

— zgeql2.f —

```
*  =====================================================================
      SUBROUTINE ZGEQL2( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      COMPLEX*16         ALPHA
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLARF, ZLARFG
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DCONJG, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEQL2', -INFO )
```

```
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = K, 1, -1
*
*        Generate elementary reflector H(i) to annihilate
*        A(1:m-k+i-1,n-k+i)
*
         ALPHA = A( M-K+I, N-K+I )
         CALL ZLARFG( M-K+I, ALPHA, A( 1, N-K+I ), 1, TAU( I ) )
*
*        Apply H(i)**H to A(1:m-k+i,1:n-k+i-1) from the left
*
         A( M-K+I, N-K+I ) = ONE
         CALL ZLARF( 'Left', M-K+I, N-K+I-1, A( 1, N-K+I ), 1,
     $               DCONJG( TAU( I ) ), A, LDA, WORK )
         A( M-K+I, N-K+I ) = ALPHA
   10 CONTINUE
      RETURN
*
*     End of ZGEQL2
*
      END
```

———-

35.13.4 lisp code

— **zgeql2.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (ignorable one))
  (defun zgeql2 (m n a lda tau work info)
    (declare (type (array f2cl-lib:complex16 (*)) work tau a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%))
      (prog ((alpha #C(0.0d0 0.0d0)) (i 0) (k 0))
        (declare (type (f2cl-lib:complex16) alpha)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
```

```
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -4)))
(cond
 ((/= info 0) (xerbla "ZGEQL2" (f2cl-lib:int-sub info))
  (go end_label)))
(setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(f2cl-lib:fdo (i k (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf alpha
            (f2cl-lib:fref a-%data%
                           ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                            (f2cl-lib:int-add (f2cl-lib:int-sub n k)
                                              i))
                           ((1 lda) (1 *)) a-%offset%))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zlarfg (f2cl-lib:int-add (f2cl-lib:int-sub m k) i) alpha
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               (1
                                (f2cl-lib:int-add
                                 (f2cl-lib:int-sub n k) i))
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                               ((1 *)) tau-%offset%))
      (declare (ignore var-0 var-2 var-3 var-4))
      (when var-1 (setf alpha var-1)))
    (setf (f2cl-lib:fref a-%data%
                         ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                          (f2cl-lib:int-add (f2cl-lib:int-sub n k) i))
                         ((1 lda) (1 *)) a-%offset%)
            one)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (zlarf "Left" (f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
         (f2cl-lib:int-sub (f2cl-lib:int-add (f2cl-lib:int-sub n k) i)
                           1)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               (1
                                (f2cl-lib:int-add
                                 (f2cl-lib:int-sub n k) i))
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:dconjg
          (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
         a lda work)
      (declare
       (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
      (when var-7 (setf lda var-7)))
    (setf (f2cl-lib:fref a-%data%
```

```
                                ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                 (f2cl-lib:int-add (f2cl-lib:int-sub n k) i))
                                ((1 lda) (1 *)) a-%offset%)
                    alpha)
           label10))
        (go end_label)
       end_label
        (return (values nil nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgeql2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::lda
                                              nil nil fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

35.14 zgeqlf

35.14.1 Axiom unit tests

— zgeqlf.input —

```
)set break resume
)sys rm -f zgeqlf.output
)spool zgeqlf.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

35.14.2 Axiom help page

— **zgeqlf.help** —

```
=====================================================================
zgeqlf examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgeqlf (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           ZGEQLF

Function/Subroutine Documentation
   subroutine zgeqlf (integer M, integer N, complex*16, dimension( lda, * ) A,
       integer LDA, complex*16, dimension( * ) TAU, complex*16, dimension( * )
       WORK, integer LWORK, integer INFO)
       ZGEQLF

Purpose:
   ZGEQLF computes a QL factorization of a complex M-by-N matrix A:
   A = Q * L.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit,
   if m >= n, the lower triangle of the subarray
   A(m-n+1:m,1:n) contains the N-by-N lower triangular matrix L;
```

```
   if m <= n, the elements on and below the (n-m)-th
   superdiagonal contain the M-by-N lower trapezoidal matrix L;
   the remaining elements, with the array TAU, represent the
   unitary matrix Q as a product of elementary reflectors
   (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is COMPLEX*16 array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.  LWORK >= max(1,N).
   For optimum performance LWORK >= N*NB, where NB is
   the optimal blocksize.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details:

   The matrix Q is represented as a product of elementary reflectors

   Q = H(k) . . . H(2) H(1), where k = min(m,n).

   Each H(i) has the form

   H(i) = I - tau * v * v**H

   where tau is a complex scalar, and v is a complex vector with
   v(m-k+i+1:m) = 0 and v(m-k+i) = 1; v(1:m-k+i-1) is stored on exit in
   A(1:m-k+i-1,n-k+i), and tau in TAU(i).
```

———-

35.14.3 fortran code

— zgeqlf.f —

```
*  =====================================================================
      SUBROUTINE ZGEQLF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, KI, KK, LDWORK, LWKOPT,
     $                   MU, NB, NBMIN, NU, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGEQL2, ZLARFB, ZLARFT
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
```

```
*
      IF( INFO.EQ.0 ) THEN
         K = MIN( M, N )
         IF( K.EQ.0 ) THEN
            LWKOPT = 1
         ELSE
            NB = ILAENV( 1, 'ZGEQLF', ' ', M, N, -1, -1 )
            LWKOPT = N*NB
         END IF
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
            INFO = -7
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEQLF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( K.EQ.0 ) THEN
         RETURN
      END IF
*
      NBMIN = 2
      NX = 1
      IWS = N
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'ZGEQLF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = N
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'ZGEQLF', ' ', M, N, -1,
```

```
     $                -1 ) )
           END IF
        END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially.
*        The last kk columns are handled by the block method.
*
         KI = ( ( K-NX-1 ) / NB )*NB
         KK = MIN( K, KI+NB )
*
         DO 10 I = K - KK + KI + 1, K - KK + 1, -NB
            IB = MIN( K-I+1, NB )
*
*           Compute the QL factorization of the current block
*           A(1:m-k+i+ib-1,n-k+i:n-k+i+ib-1)
*
            CALL ZGEQL2( M-K+I+IB-1, IB, A( 1, N-K+I ), LDA, TAU( I ),
     $                   WORK, IINFO )
            IF( N-K+I.GT.1 ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i+ib-1) . . . H(i+1) H(i)
*
               CALL ZLARFT( 'Backward', 'Columnwise', M-K+I+IB-1, IB,
     $                      A( 1, N-K+I ), LDA, TAU( I ), WORK, LDWORK )
*
*              Apply H**H to A(1:m-k+i+ib-1,1:n-k+i-1) from the left
*
               CALL ZLARFB( 'Left', 'Conjugate transpose', 'Backward',
     $                      'Columnwise', M-K+I+IB-1, N-K+I-1, IB,
     $                      A( 1, N-K+I ), LDA, WORK, LDWORK, A, LDA,
     $                      WORK( IB+1 ), LDWORK )
            END IF
   10    CONTINUE
         MU = M - K + I + NB - 1
         NU = N - K + I + NB - 1
      ELSE
         MU = M
         NU = N
      END IF
*
*     Use unblocked code to factor the last or only block
*
      IF( MU.GT.0 .AND. NU.GT.0 )
     $   CALL ZGEQL2( MU, NU, A, LDA, TAU, WORK, IINFO )
*
      WORK( 1 ) = IWS
```

```
      RETURN
*
*     End of ZGEQLF
*
      END
```

———-

35.14.4 lisp code

— **zgeqlf.lisp** —

```
(defun zgeqlf (m n a lda tau work lwork info)
  (declare (type (array f2cl-lib:complex16 (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex16 a-%data% a-%offset%)
       (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ki 0) (kk 0) (ldwork 0)
           (lwkopt 0) (mu 0) (nb 0) (nbmin 0) (nu 0) (nx 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) nx nu nbmin nb mu lwkopt ldwork kk ki
                k iws iinfo ib i))
      (setf info 0)
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4)))
      (cond
       ((= info 0)
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (cond ((= k 0) (setf lwkopt 1))
              (t
               (setf nb
                       (multiple-value-bind
                           (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                           (ilaenv 1 "ZGEQLF" " " m n -1 -1)
                         (declare (ignore var-0 var-1 var-2 var-5 var-6))
                         (setf m var-3)
                         (setf n var-4)
                         ret-val))
               (setf lwkopt (f2cl-lib:int-mul n nb))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce lwkopt 'f2cl-lib:complex16))
        (cond
         ((and
           (< lwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
```

```
     (not lquery))
    (setf info -7)))))
(cond
 ((/= info 0) (xerbla "ZGEQLF" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond ((= k 0) (go end_label)))
(setf nbmin 2)
(setf nx 1)
(setf iws n)
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "ZGEQLF" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (setf m var-3)
                      (setf n var-4)
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork n) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
                            (ilaenv 2 "ZGEQLF" " " m n -1 -1)
                          (declare
                           (ignore var-0 var-1 var-2 var-5 var-6))
                          (setf m var-3)
                          (setf n var-4)
                          ret-val)))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (setf ki (* (the f2cl-lib:integer4 (truncate (- k nx 1) nb)) nb))
  (setf kk
          (min (the f2cl-lib:integer4 k)
               (the f2cl-lib:integer4 (f2cl-lib:int-add ki nb))))
  (f2cl-lib:fdo (i (f2cl-lib:int-add k (f2cl-lib:int-sub kk) ki 1)
                 (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
                ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub kk) 1)) nil)
    (tagbody
      (setf ib
```

```
        (min
         (the f2cl-lib:integer4
              (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
         (the f2cl-lib:integer4 nb)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zgeql2
     (f2cl-lib:int-sub
      (f2cl-lib:int-add (f2cl-lib:int-sub m k) i ib) 1)
     ib
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                           (1
                            (f2cl-lib:int-add
                             (f2cl-lib:int-sub n k) i))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                           ((1 *)) tau-%offset%)
     work iinfo)
  (declare (ignore var-0 var-1 var-2 var-4 var-5))
  (setf lda var-3)
  (setf iinfo var-6))
(cond
 ((> (f2cl-lib:int-add n (f2cl-lib:int-sub k) i) 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zlarft "Backward" "Columnwise"
       (f2cl-lib:int-sub
        (f2cl-lib:int-add (f2cl-lib:int-sub m k) i ib) 1)
       ib
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             (1
                              (f2cl-lib:int-add
                               (f2cl-lib:int-sub n k) i))
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                             ((1 *)) tau-%offset%)
       work ldwork)
    (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
    (when var-3 (setf ib var-3))
    (when var-5 (setf lda var-5))
    (when var-8 (setf ldwork var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (zlarfb "Left" "Conjugate transpose" "Backward" "Columnwise"
       (f2cl-lib:int-sub
        (f2cl-lib:int-add (f2cl-lib:int-sub m k) i ib) 1)
       (f2cl-lib:int-sub
        (f2cl-lib:int-add (f2cl-lib:int-sub n k) i) 1)
```

```
                  ib
                  (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                        (1
                                         (f2cl-lib:int-add
                                          (f2cl-lib:int-sub n k) i))
                                        ((1 lda) (1 *)) a-%offset%)
                  lda work ldwork a lda
                  (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                        ((+ ib 1)) ((1 *)) work-%offset%)
                  ldwork)
                (declare
                 (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
                  var-13))
                (when var-6 (setf ib var-6))
                (when var-8 (setf lda var-8))
                (when var-10 (setf ldwork var-10))
                (when var-12 (setf lda var-12))
                (when var-14 (setf ldwork var-14)))))
           label10))
        (setf mu
                (f2cl-lib:int-sub
                 (f2cl-lib:int-add (f2cl-lib:int-sub m k) i nb) 1))
        (setf nu
                (f2cl-lib:int-sub
                 (f2cl-lib:int-add (f2cl-lib:int-sub n k) i nb) 1)))
       (t (setf mu m) (setf nu n)))
      (if (and (> mu 0) (> nu 0))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (zgeql2 mu nu a lda tau work iinfo)
            (declare (ignore var-0 var-1 var-2 var-4 var-5))
            (setf lda var-3)
            (setf iinfo var-6)))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce iws 'f2cl-lib:complex16))
      (go end_label)
     end_label
      (return (values m n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgeqlf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
```

```
    (*))
   (array fortran-to-lisp::complex16
    (*))
   (fortran-to-lisp::integer4)
   (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::m
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zgeql2
  fortran-to-lisp::xerbla
  fortran-to-lisp::ilaenv))))
```

———-

35.15 zgeqp3

35.15.1 Axiom unit tests

— **zgeqp3.input** —

```
)set break resume
)sys rm -f zgeqp3.output
)spool zgeqp3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.15.2 Axiom help page

— **zgeqp3.help** —

```
==========================================================================
zgeqp3 examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgeqp3 (M, N, A, LDA, JPVT, TAU, WORK, LWORK, RWORK, INFO)
           ZGEQP3

Function/Subroutine Documentation
   subroutine zgeqp3 (integer M, integer N, complex*16, dimension( lda, * ) A,
       integer LDA, integer, dimension( * ) JPVT, complex*16, dimension( * )
       TAU, complex*16, dimension( * ) WORK, integer LWORK, double precision,
       dimension( * ) RWORK, integer INFO)
       ZGEQP3

Purpose:
   ZGEQP3 computes a QR factorization with column pivoting of a
   matrix A:  A*P = Q*R  using Level 3 BLAS.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A. M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, the upper triangle of the array contains the
   min(M,N)-by-N upper trapezoidal matrix R; the elements below
   the diagonal, together with the array TAU, represent the
   unitary matrix Q as a product of min(M,N) elementary reflectors.

 LDA
   LDA is INTEGER
   The leading dimension of the array A. LDA >= max(1,M).

 JPVT
   JPVT is INTEGER array, dimension (N)
   On entry, if JPVT(J).ne.0, the J-th column of A is permuted
   to the front of A*P (a leading column); if JPVT(J)=0,
   the J-th column of A is a free column.
   On exit, if JPVT(J)=K, then the J-th column of A*P was the
   the K-th column of A.
```

```
 TAU
   TAU is COMPLEX*16 array, dimension (min(M,N))
   The scalar factors of the elementary reflectors.

 WORK
   WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
   On exit, if INFO=0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK. LWORK >= N+1.
   For optimal performance LWORK >= ( N+1 )*NB, where NB
   is the optimal blocksize.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 RWORK
   RWORK is DOUBLE PRECISION array, dimension (2*N)

 INFO
   INFO is INTEGER
   = 0: successful exit.
   < 0: if INFO = -i, the i-th argument had an illegal value.

Further Details:

   The matrix Q is represented as a product of elementary reflectors

   Q = H(1) H(2) . . . H(k), where k = min(m,n).

   Each H(i) has the form

   H(i) = I - tau * v * v**H

   where tau is a complex scalar, and v is a real/complex vector
   with v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in
   A(i+1:m,i), and tau in TAU(i).
```

———-

35.15.3 fortran code

— zgeqp3.f —

```
*  =====================================================================
      SUBROUTINE ZGEQP3( M, N, A, LDA, JPVT, TAU, WORK, LWORK, RWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            JPVT( * )
      DOUBLE PRECISION   RWORK( * )
      COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            INB, INBMIN, IXOVER
      PARAMETER          ( INB = 1, INBMIN = 2, IXOVER = 3 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            FJB, IWS, J, JB, LWKOPT, MINMN, MINWS, NA, NB,
     $                   NBMIN, NFXD, NX, SM, SMINMN, SN, TOPBMN
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGEQRF, ZLAQP2, ZLAQPS, ZSWAP, ZUNMQR
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      DOUBLE PRECISION   DZNRM2
      EXTERNAL           ILAENV, DZNRM2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          INT, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test input arguments
*  ====================
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
```

```
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
*
      IF( INFO.EQ.0 ) THEN
         MINMN = MIN( M, N )
         IF( MINMN.EQ.0 ) THEN
            IWS = 1
            LWKOPT = 1
         ELSE
            IWS = N + 1
            NB = ILAENV( INB, 'ZGEQRF', ' ', M, N, -1, -1 )
            LWKOPT = ( N + 1 )*NB
         END IF
         WORK( 1 ) = DCMPLX( LWKOPT )
*
         IF( ( LWORK.LT.IWS ) .AND. .NOT.LQUERY ) THEN
            INFO = -8
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEQP3', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Move initial columns up front.
*
      NFXD = 1
      DO 10 J = 1, N
         IF( JPVT( J ).NE.0 ) THEN
            IF( J.NE.NFXD ) THEN
               CALL ZSWAP( M, A( 1, J ), 1, A( 1, NFXD ), 1 )
               JPVT( J ) = JPVT( NFXD )
               JPVT( NFXD ) = J
            ELSE
               JPVT( J ) = J
            END IF
            NFXD = NFXD + 1
         ELSE
            JPVT( J ) = J
         END IF
   10 CONTINUE
      NFXD = NFXD - 1
*
*     Factorize fixed columns
*  =======================
```

```
*
*     Compute the QR factorization of fixed columns and update
*     remaining columns.
*
      IF( NFXD.GT.0 ) THEN
         NA = MIN( M, NFXD )
*CC      CALL ZGEQR2( M, NA, A, LDA, TAU, WORK, INFO )
         CALL ZGEQRF( M, NA, A, LDA, TAU, WORK, LWORK, INFO )
         IWS = MAX( IWS, INT( WORK( 1 ) ) )
         IF( NA.LT.N ) THEN
*CC         CALL ZUNM2R( 'Left', 'Conjugate Transpose', M, N-NA,
*CC  $                   NA, A, LDA, TAU, A( 1, NA+1 ), LDA, WORK,
*CC  $                   INFO )
            CALL ZUNMQR( 'Left', 'Conjugate Transpose', M, N-NA, NA, A,
     $                   LDA, TAU, A( 1, NA+1 ), LDA, WORK, LWORK,
     $                   INFO )
            IWS = MAX( IWS, INT( WORK( 1 ) ) )
         END IF
      END IF
*
*     Factorize free columns
*  =====================
*
      IF( NFXD.LT.MINMN ) THEN
*
         SM = M - NFXD
         SN = N - NFXD
         SMINMN = MINMN - NFXD
*
*        Determine the block size.
*
         NB = ILAENV( INB, 'ZGEQRF', ' ', SM, SN, -1, -1 )
         NBMIN = 2
         NX = 0
*
         IF( ( NB.GT.1 ) .AND. ( NB.LT.SMINMN ) ) THEN
*
*           Determine when to cross over from blocked to unblocked code.
*
            NX = MAX( 0, ILAENV( IXOVER, 'ZGEQRF', ' ', SM, SN, -1,
     $           -1 ) )
*
*
            IF( NX.LT.SMINMN ) THEN
*
*              Determine if workspace is large enough for blocked code.
*
               MINWS = ( SN+1 )*NB
               IWS = MAX( IWS, MINWS )
               IF( LWORK.LT.MINWS ) THEN
```

```
*
*                Not enough workspace to use optimal NB: Reduce NB and
*                determine the minimum value of NB.
*
                 NB = LWORK / ( SN+1 )
                 NBMIN = MAX( 2, ILAENV( INBMIN, 'ZGEQRF', ' ', SM, SN,
     $                   -1, -1 ) )
*
*
              END IF
           END IF
        END IF
*
*       Initialize partial column norms. The first N elements of work
*       store the exact column norms.
*
        DO 20 J = NFXD + 1, N
           RWORK( J ) = DZNRM2( SM, A( NFXD+1, J ), 1 )
           RWORK( N+J ) = RWORK( J )
   20   CONTINUE
*
        IF( ( NB.GE.NBMIN ) .AND. ( NB.LT.SMINMN ) .AND.
     $       ( NX.LT.SMINMN ) ) THEN
*
*          Use blocked code initially.
*
           J = NFXD + 1
*
*          Compute factorization: while loop.
*
*
           TOPBMN = MINMN - NX
   30      CONTINUE
           IF( J.LE.TOPBMN ) THEN
              JB = MIN( NB, TOPBMN-J+1 )
*
*             Factorize JB columns among columns J:N.
*
              CALL ZLAQPS( M, N-J+1, J-1, JB, FJB, A( 1, J ), LDA,
     $                     JPVT( J ), TAU( J ), RWORK( J ),
     $                     RWORK( N+J ), WORK( 1 ), WORK( JB+1 ),
     $                     N-J+1 )
*
              J = J + FJB
              GO TO 30
           END IF
        ELSE
           J = NFXD + 1
        END IF
*
```

```
*        Use unblocked code to factor the last or only block.
*
*
         IF( J.LE.MINMN )
     $      CALL ZLAQP2( M, N-J+1, J-1, A( 1, J ), LDA, JPVT( J ),
     $                   TAU( J ), RWORK( J ), RWORK( N+J ), WORK( 1 ) )
*
      END IF
*
      WORK( 1 ) = DCMPLX( LWKOPT )
      RETURN
*
*     End of ZGEQP3
*
      END
```

35.15.4 lisp code

— zgeqp3.lisp —

```
(let* ((inb 1) (inbmin 2) (ixover 3))
  (declare (type (f2cl-lib:integer4 1 1) inb)
           (type (f2cl-lib:integer4 2 2) inbmin)
           (type (f2cl-lib:integer4 3 3) ixover)
           (ignorable inb inbmin ixover))
  (defun zgeqp3 (m n a lda jpvt tau work lwork rwork info)
    (declare (type (array double-float (*)) rwork)
             (type (array f2cl-lib:integer4 (*)) jpvt)
             (type (array f2cl-lib:complex16 (*)) work tau a)
             (type (f2cl-lib:integer4) info lwork lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (jpvt f2cl-lib:integer4 jpvt-%data% jpvt-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (prog ((fjb 0) (iws 0) (j 0) (jb 0) (lwkopt 0) (minmn 0) (minws 0) (na 0)
             (nb 0) (nbmin 0) (nfxd 0) (nx 0) (sm 0) (sminmn 0) (sn 0)
             (topbmn 0) (lquery nil))
        (declare
         (type (f2cl-lib:integer4) fjb iws j jb lwkopt minmn minws na nb nbmin
          nfxd nx sm sminmn sn topbmn)
         (type f2cl-lib:logical lquery))
        (setf info 0)
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
```

```
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -4)))
(cond
 ((= info 0)
  (setf minmn
          (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
  (cond ((= minmn 0) (setf iws 1) (setf lwkopt 1))
        (t (setf iws (f2cl-lib:int-add n 1))
         (setf nb
                 (multiple-value-bind
                     (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                      var-6)
                     (ilaenv inb "ZGEQRF" " " m n -1 -1)
                   (declare (ignore var-1 var-2 var-5 var-6))
                   (setf inb var-0)
                   (setf m var-3)
                   (setf n var-4)
                   ret-val))
         (setf lwkopt (f2cl-lib:int-mul (f2cl-lib:int-add n 1) nb))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:dcmplx lwkopt))
  (cond ((and (< lwork iws) (not lquery)) (setf info -8)))))
(cond
 ((/= info 0) (xerbla "ZGEQP3" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(setf nfxd 1)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (cond
     ((/= (f2cl-lib:fref jpvt (j) ((1 *))) 0)
      (cond
       ((/= j nfxd)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (zswap m
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 nfxd)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf m var-0)))
        (setf (f2cl-lib:fref jpvt-%data% (j) ((1 *)) jpvt-%offset%)
                (f2cl-lib:fref jpvt-%data% (nfxd) ((1 *))
                               jpvt-%offset%))
        (setf (f2cl-lib:fref jpvt-%data% (nfxd) ((1 *)) jpvt-%offset%)
                j))
       (t
```

```
        (setf (f2cl-lib:fref jpvt-%data% (j) ((1 *)) jpvt-%offset%)
                j)))
      (setf nfxd (f2cl-lib:int-add nfxd 1)))
     (t
      (setf (f2cl-lib:fref jpvt-%data% (j) ((1 *)) jpvt-%offset%) j)))
   label10))
(setf nfxd (f2cl-lib:int-sub nfxd 1))
(cond
 ((> nfxd 0)
  (setf na
          (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 nfxd)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgeqrf m na a lda tau work lwork info)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf na var-1)
    (setf lda var-3)
    (setf info var-7))
  (setf iws
          (max (the f2cl-lib:integer4 iws)
               (the f2cl-lib:integer4
                    (f2cl-lib:int
                     (f2cl-lib:fref work-%data% (1) ((1 *))
                                    work-%offset%)))))
  (cond
   ((< na n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (zunmqr "Left" "Conjugate Transpose" m (f2cl-lib:int-sub n na)
         na a lda tau
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               (1 (f2cl-lib:int-add na 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda work lwork info)
      (declare (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10))
      (when var-2 (setf m var-2))
      (when var-4 (setf na var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf lda var-9))
      (when var-11 (setf lwork var-11))
      (when var-12 (setf info var-12)))
    (setf iws
            (max (the f2cl-lib:integer4 iws)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int
                       (f2cl-lib:fref work-%data% (1) ((1 *))
                                      work-%offset%)))))))))
(cond
```

```
((< nfxd minmn) (setf sm (f2cl-lib:int-sub m nfxd))
 (setf sn (f2cl-lib:int-sub n nfxd))
 (setf sminmn (f2cl-lib:int-sub minmn nfxd))
 (setf nb
         (multiple-value-bind
             (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
             (ilaenv inb "ZGEQRF" " " sm sn -1 -1)
           (declare (ignore var-1 var-2 var-5 var-6))
           (setf inb var-0)
           (setf sm var-3)
           (setf sn var-4)
           ret-val))
 (setf nbmin 2) (setf nx 0)
 (cond
  ((and (> nb 1) (< nb sminmn))
   (setf nx
           (max (the f2cl-lib:integer4 0)
                (the f2cl-lib:integer4
                     (multiple-value-bind
                         (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                          var-6)
                         (ilaenv ixover "ZGEQRF" " " sm sn -1 -1)
                       (declare (ignore var-1 var-2 var-5 var-6))
                       (setf ixover var-0)
                       (setf sm var-3)
                       (setf sn var-4)
                       ret-val))))
   (cond
    ((< nx sminmn)
     (setf minws (f2cl-lib:int-mul (f2cl-lib:int-add sn 1) nb))
     (setf iws
             (max (the f2cl-lib:integer4 iws)
                  (the f2cl-lib:integer4 minws)))
     (cond
      ((< lwork minws)
       (setf nb (the f2cl-lib:integer4 (truncate lwork (+ sn 1))))
       (setf nbmin
               (max (the f2cl-lib:integer4 2)
                    (the f2cl-lib:integer4
                         (multiple-value-bind
                             (ret-val var-0 var-1 var-2 var-3 var-4
                              var-5 var-6)
                             (ilaenv inbmin "ZGEQRF" " " sm sn -1 -1)
                           (declare (ignore var-1 var-2 var-5 var-6))
                           (setf inbmin var-0)
                           (setf sm var-3)
                           (setf sn var-4)
                           ret-val))))))))))
(f2cl-lib:fdo (j (f2cl-lib:int-add nfxd 1) (f2cl-lib:int-add j 1))
              ((> j n) nil)
```

```
    (tagbody
      (setf (f2cl-lib:fref rwork-%data% (j) ((1 *)) rwork-%offset%)
              (multiple-value-bind (ret-val var-0 var-1 var-2)
                  (dznrm2 sm
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                         ((+ nfxd 1) j) ((1 lda) (1 *))
                                         a-%offset%)
                   1)
                (declare (ignore var-1 var-2))
                (when var-0 (setf sm var-0))
                ret-val))
      (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add n j))
                           ((1 *)) rwork-%offset%)
              (f2cl-lib:fref rwork-%data% (j) ((1 *)) rwork-%offset%))
     label20))
  (cond
   ((and (>= nb nbmin) (< nb sminmn) (< nx sminmn))
    (tagbody
      (setf j (f2cl-lib:int-add nfxd 1))
      (setf topbmn (f2cl-lib:int-sub minmn nx))
     label30
      (cond
       ((<= j topbmn)
        (setf jb
                (min (the f2cl-lib:integer4 nb)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-sub topbmn j)
                                            1))))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12 var-13)
            (zlaqps m (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1)
             (f2cl-lib:int-sub j 1) jb fjb
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice jpvt-%data% f2cl-lib:integer4 (j)
                                   ((1 *)) jpvt-%offset%)
             (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (j)
                                   ((1 *)) tau-%offset%)
             (f2cl-lib:array-slice rwork-%data% double-float (j)
                                   ((1 *)) rwork-%offset%)
             (f2cl-lib:array-slice rwork-%data% double-float ((+ n j))
                                   ((1 *)) rwork-%offset%)
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (1)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                   ((+ jb 1)) ((1 *)) work-%offset%)
             (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1))
          (declare
```

```
                  (ignore var-1 var-2 var-5 var-7 var-8 var-9 var-10 var-11
                   var-12 var-13))
                 (when var-0 (setf m var-0))
                 (when var-3 (setf jb var-3))
                 (when var-4 (setf fjb var-4))
                 (when var-6 (setf lda var-6)))
               (setf j (f2cl-lib:int-add j fjb)) (go label30)))))
           (t (setf j (f2cl-lib:int-add nfxd 1))))
          (if (<= j minmn)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (zlaqp2 m (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1)
                   (f2cl-lib:int-sub j 1)
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 j)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda
                   (f2cl-lib:array-slice jpvt-%data% f2cl-lib:integer4 (j)
                                         ((1 *)) jpvt-%offset%)
                   (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (j)
                                         ((1 *)) tau-%offset%)
                   (f2cl-lib:array-slice rwork-%data% double-float (j) ((1 *))
                                         rwork-%offset%)
                   (f2cl-lib:array-slice rwork-%data% double-float ((+ n j))
                                         ((1 *)) rwork-%offset%)
                   (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (1)
                                         ((1 *)) work-%offset%))
               (declare
                (ignore var-1 var-2 var-3 var-5 var-6 var-7 var-8 var-9))
               (when var-0 (setf m var-0))
               (when var-4 (setf lda var-4))))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
               (f2cl-lib:dcmplx lwkopt))
        (go end_label)
       end_label
        (return (values m n nil lda nil nil nil lwork nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgeqp3
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
```

```
   (*))
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::m
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil nil
  fortran-to-lisp::lwork nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zgeqrf
  fortran-to-lisp::xerbla
  fortran-to-lisp::ilaenv))))
```

———-

35.16 zgeqpf - DEPRECATED use zgeqp3

35.17 zgeqr2

35.17.1 Axiom unit tests

— **zgeqr2.input** —

```
)set break resume
)sys rm -f zgeqr2.output
)spool zgeqr2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.17.2 Axiom help page

— **zgeqr2.help** —

```
====================================================================
zgeqr2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgeqr2 (M, N, A, LDA, TAU, WORK, INFO)
           ZGEQR2 computes the QR factorization of a general rectangular
           matrix using an unblocked algorithm.

Function/Subroutine Documentation
   subroutine zgeqr2 (integer M, integer N, complex*16, dimension( lda, * ) A,
       integer LDA, complex*16, dimension( * ) TAU, complex*16, dimension( * )
       WORK, integer INFO)
       ZGEQR2 computes the QR factorization of a general rectangular matrix
       using an unblocked algorithm.

Purpose:
   ZGEQR2 computes a QR factorization of a complex m by n matrix A:
   A = Q * R.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the m by n matrix A.
   On exit, the elements on and above the diagonal of the array
   contain the min(m,n) by n upper trapezoidal matrix R (R is
   upper triangular if m >= n); the elements below the diagonal,
   with the array TAU, represent the unitary matrix Q as a
   product of elementary reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is COMPLEX*16 array, dimension (min(M,N))
```

```
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is COMPLEX*16 array, dimension (N)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

Further Details:

   The matrix Q is represented as a product of elementary reflectors

   Q = H(1) H(2) . . . H(k), where k = min(m,n).

   Each H(i) has the form

   H(i) = I - tau * v * v**H

   where tau is a complex scalar, and v is a complex vector with
   v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in A(i+1:m,i),
   and tau in TAU(i).
```

———-

35.17.3 fortran code

— zgeqr2.f —

```
*  =====================================================================
      SUBROUTINE ZGEQR2( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
```

```
*     .. Parameters ..
      COMPLEX*16         ONE
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      COMPLEX*16         ALPHA
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLARF, ZLARFG
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DCONJG, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEQR2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = 1, K
*
*        Generate elementary reflector H(i) to annihilate A(i+1:m,i)
*
         CALL ZLARFG( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
     $                TAU( I ) )
         IF( I.LT.N ) THEN
*
*           Apply H(i)**H to A(i:m,i+1:n) from the left
*
            ALPHA = A( I, I )
            A( I, I ) = ONE
            CALL ZLARF( 'Left', M-I+1, N-I, A( I, I ), 1,
     $                  DCONJG( TAU( I ) ), A( I, I+1 ), LDA, WORK )
            A( I, I ) = ALPHA
         END IF
   10 CONTINUE
```

```
      RETURN
*
*     End of ZGEQR2
*
      END
```

———-

35.17.4 lisp code

— **zgeqr2.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (ignorable one))
  (defun zgeqr2 (m n a lda tau work info)
    (declare (type (array f2cl-lib:complex16 (*)) work tau a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%))
      (prog ((alpha #C(0.0d0 0.0d0)) (i 0) (k 0))
        (declare (type (f2cl-lib:complex16) alpha)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "ZGEQR2" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i k) nil)
          (tagbody
            (zlarfg (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   ((min (f2cl-lib:int-add i 1) m) i)
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i) ((1 *))
                                   tau-%offset%))
            (cond
             ((< i n)
```

```
              (setf alpha
                      (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                      one)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                  (zlarf "Left" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
                   (f2cl-lib:int-sub n i)
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                         ((1 lda) (1 *)) a-%offset%)
                   1
                   (f2cl-lib:dconjg
                    (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                         (i (f2cl-lib:int-add i 1))
                                         ((1 lda) (1 *)) a-%offset%)
                   lda work)
                (declare
                 (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
                (when var-7 (setf lda var-7)))
              (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                      alpha)))
          label10))
       (go end_label)
      end_label
       (return (values nil nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgeqr2
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(nil nil nil fortran-to-lisp::lda
                                             nil nil fortran-to-lisp::info)
                                           :calls '(fortran-to-lisp::xerbla))))
```

———-

35.18 zgeqr2p

35.18.1 Axiom unit tests

— **zgeqr2p.input** —

```
)set break resume
)sys rm -f zgeqr2p.output
)spool zgeqr2p.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.18.2 Axiom help page [Demm08]

— **zgeqr2p.help** —

```
======================================================================
zgeqr2p examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgeqr2p (M, N, A, LDA, TAU, WORK, INFO)
           ZGEQR2P computes the QR factorization of a general rectangular
           matrix with non-negative diagonal elements using an unblocked
           algorithm.

Function/Subroutine Documentation
   subroutine zgeqr2p (integer M, integer N, complex*16, dimension( lda, * )
       A, integer LDA, complex*16, dimension( * ) TAU, complex*16, dimension(
       * ) WORK, integer INFO)
       ZGEQR2P computes the QR factorization of a general rectangular matrix
```

```
       with non-negative diagonal elements using an unblocked algorithm.

Purpose:
   ZGEQR2P computes a QR factorization of a complex m by n matrix A:
   A = Q * R. The diagonal entries of R are real and nonnegative.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the m by n matrix A.
   On exit, the elements on and above the diagonal of the array
   contain the min(m,n) by n upper trapezoidal matrix R (R is
   upper triangular if m >= n). The diagonal entries of R
   are real and nonnegative; the elements below the diagonal,
   with the array TAU, represent the unitary matrix Q as a
   product of elementary reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is COMPLEX*16 array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is COMPLEX*16 array, dimension (N)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

Further Details:

   The matrix Q is represented as a product of elementary reflectors

   Q = H(1) H(2) . . . H(k), where k = min(m,n).

   Each H(i) has the form
```

```
H(i) = I - tau * v * v**H

where tau is a complex scalar, and v is a complex vector with
v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in A(i+1:m,i),
and tau in TAU(i).
```

———-

35.18.3 fortran code

— **zgeqr2p.f** —

```
*  =====================================================================
      SUBROUTINE ZGEQR2P( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      COMPLEX*16         ALPHA
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLARF, ZLARFGP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DCONJG, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
```

```
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEQR2P', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = 1, K
*
*        Generate elementary reflector H(i) to annihilate A(i+1:m,i)
*
         CALL ZLARFGP( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
     $                TAU( I ) )
         IF( I.LT.N ) THEN
*
*           Apply H(i)**H to A(i:m,i+1:n) from the left
*
            ALPHA = A( I, I )
            A( I, I ) = ONE
            CALL ZLARF( 'Left', M-I+1, N-I, A( I, I ), 1,
     $                  DCONJG( TAU( I ) ), A( I, I+1 ), LDA, WORK )
            A( I, I ) = ALPHA
         END IF
   10 CONTINUE
      RETURN
*
*     End of ZGEQR2P
*
      END
```

———-

35.18.4 lisp code

— zgeqr2p.lisp —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (ignorable one))
  (defun zgeqr2p (m n a lda tau work info)
```

```
(declare (type (array f2cl-lib:complex16 (*)) work tau a)
         (type (f2cl-lib:integer4) info lda n m))
(f2cl-lib:with-multi-array-data
    ((a f2cl-lib:complex16 a-%data% a-%offset%)
     (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
     (work f2cl-lib:complex16 work-%data% work-%offset%))
  (prog ((alpha #C(0.0d0 0.0d0)) (i 0) (k 0))
    (declare (type (f2cl-lib:complex16) alpha)
             (type (f2cl-lib:integer4) i k))
    (setf info 0)
    (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
          ((< lda
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
           (setf info -4)))
    (cond
     ((/= info 0) (xerbla "ZGEQR2P" (f2cl-lib:int-sub info))
      (go end_label)))
    (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i k) nil)
      (tagbody
        (zlarfgp (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                               ((1 lda) (1 *)) a-%offset%)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               ((min (f2cl-lib:int-add i 1) m) i)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i) ((1 *))
                               tau-%offset%))
        (cond
         ((< i n)
          (setf alpha
                  (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                 a-%offset%))
          (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                  one)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (zlarf "Left" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
               (f2cl-lib:int-sub n i)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               1
               (f2cl-lib:dconjg
                (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                     (i (f2cl-lib:int-add i 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda work)
```

```
                (declare
                 (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
                (when var-7 (setf lda var-7)))
              (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                    alpha)))
           label10))
        (go end_label)
       end_label
        (return (values nil nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgeqr2p
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::lda
                                              nil nil fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

35.19 zgeqrf

35.19.1 Axiom unit tests

— zgeqrf.input —

```
)set break resume
)sys rm -f zgeqrf.output
)spool zgeqrf.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

35.19.2 Axiom help page

— **zgeqrf.help** —

```
==========================================================================
zgeqrf examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgeqrf (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           ZGEQRF

Function/Subroutine Documentation
   subroutine zgeqrf (integer M, integer N, complex*16, dimension( lda, * ) A,
       integer LDA, complex*16, dimension( * ) TAU, complex*16, dimension( * )
       WORK, integer LWORK, integer INFO)
       ZGEQRF

Purpose:
   ZGEQRF computes a QR factorization of a complex M-by-N matrix A:
   A = Q * R.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, the elements on and above the diagonal of the array
   contain the min(M,N)-by-N upper trapezoidal matrix R (R is
```

```
  upper triangular if m >= n); the elements below the diagonal,
  with the array TAU, represent the unitary matrix Q as a
  product of min(m,n) elementary reflectors (see Further Details).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

TAU
  TAU is COMPLEX*16 array, dimension (min(M,N))
  The scalar factors of the elementary reflectors (see Further Details).

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,N).
  For optimum performance LWORK >= N*NB, where NB is
  the optimal blocksize.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details:

  The matrix Q is represented as a product of elementary reflectors

  Q = H(1) H(2) . . . H(k), where k = min(m,n).

  Each H(i) has the form

  H(i) = I - tau * v * v**H

  where tau is a complex scalar, and v is a complex vector with
  v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in A(i+1:m,i),
  and tau in TAU(i).
```

———-

35.19.3 fortran code

— zgeqrf.f —

```
*  =====================================================================
      SUBROUTINE ZGEQRF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, LDWORK, LWKOPT, NB,
     $                   NBMIN, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGEQR2, ZLARFB, ZLARFT
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NB = ILAENV( 1, 'ZGEQRF', ' ', M, N, -1, -1 )
      LWKOPT = N*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
```

```
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEQRF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      K = MIN( M, N )
      IF( K.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      NX = 0
      IWS = N
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'ZGEQRF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = N
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'ZGEQRF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially
*
```

```
         DO 10 I = 1, K - NX, NB
            IB = MIN( K-I+1, NB )
*
*           Compute the QR factorization of the current block
*           A(i:m,i:i+ib-1)
*
            CALL ZGEQR2( M-I+1, IB, A( I, I ), LDA, TAU( I ), WORK,
     $                   IINFO )
            IF( I+IB.LE.N ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i) H(i+1) . . . H(i+ib-1)
*
               CALL ZLARFT( 'Forward', 'Columnwise', M-I+1, IB,
     $                      A( I, I ), LDA, TAU( I ), WORK, LDWORK )
*
*              Apply H**H to A(i:m,i+ib:n) from the left
*
               CALL ZLARFB( 'Left', 'Conjugate transpose', 'Forward',
     $                      'Columnwise', M-I+1, N-I-IB+1, IB,
     $                      A( I, I ), LDA, WORK, LDWORK, A( I, I+IB ),
     $                      LDA, WORK( IB+1 ), LDWORK )
            END IF
   10    CONTINUE
      ELSE
         I = 1
      END IF
*
*     Use unblocked code to factor the last or only block.
*
      IF( I.LE.K )
     $   CALL ZGEQR2( M-I+1, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                IINFO )
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of ZGEQRF
*
      END
```

———-

35.19.4 lisp code

— zgeqrf.lisp —

```
(defun zgeqrf (m n a lda tau work lwork info)
```

```
(declare (type (array f2cl-lib:complex16 (*)) work tau a)
         (type (f2cl-lib:integer4) info lwork lda n m))
(f2cl-lib:with-multi-array-data
    ((a f2cl-lib:complex16 a-%data% a-%offset%)
     (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
     (work f2cl-lib:complex16 work-%data% work-%offset%))
  (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ldwork 0) (lwkopt 0) (nb 0)
         (nbmin 0) (nx 0) (lquery nil))
    (declare (type f2cl-lib:logical lquery)
             (type (f2cl-lib:integer4) nx nbmin nb lwkopt ldwork k iws iinfo
              ib i))
    (setf info 0)
    (setf nb
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 1 "ZGEQRF" " " m n -1 -1)
              (declare (ignore var-0 var-1 var-2 var-5 var-6))
              (setf m var-3)
              (setf n var-4)
              ret-val))
    (setf lwkopt (f2cl-lib:int-mul n nb))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce lwkopt 'f2cl-lib:complex16))
    (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
    (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
          ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
           (setf info -4))
          ((and
            (< lwork
               (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
            (not lquery))
           (setf info -7)))
    (cond
     ((/= info 0) (xerbla "ZGEQRF" (f2cl-lib:int-sub info)) (go end_label))
     (lquery (go end_label)))
    (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
    (cond
     ((= k 0)
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce 1 'f2cl-lib:complex16))
      (go end_label)))
    (setf nbmin 2)
    (setf nx 0)
    (setf iws n)
    (cond
     ((and (> nb 1) (< nb k))
      (setf nx
              (max (the f2cl-lib:integer4 0)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
```

```
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
                          (ilaenv 3 "ZGEQRF" " " m n -1 -1)
                        (declare (ignore var-0 var-1 var-2 var-5 var-6))
                        (setf m var-3)
                        (setf n var-4)
                        ret-val))))
  (cond
   ((< nx k) (setf ldwork n) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
                            (ilaenv 2 "ZGEQRF" " " m n -1 -1)
                          (declare
                           (ignore var-0 var-1 var-2 var-5 var-6))
                          (setf m var-3)
                          (setf n var-4)
                          ret-val))))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub nx))) nil)
    (tagbody
      (setf ib
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
               (the f2cl-lib:integer4 nb)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zgeqr2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                                 ((1 *)) tau-%offset%)
           work iinfo)
        (declare (ignore var-0 var-1 var-2 var-4 var-5))
        (setf lda var-3)
        (setf iinfo var-6))
      (cond
       ((<= (f2cl-lib:int-add i ib) n)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (zlarft "Forward" "Columnwise"
```

```
                (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                                      ((1 *)) tau-%offset%)
                work ldwork)
              (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
              (when var-3 (setf ib var-3))
              (when var-5 (setf lda var-5))
              (when var-8 (setf ldwork var-8)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                 var-10 var-11 var-12 var-13 var-14)
                (zlarfb "Left" "Conjugate transpose" "Forward" "Columnwise"
                 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
                 (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1) ib
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda work ldwork
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                       (i (f2cl-lib:int-add i ib))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                       ((+ ib 1)) ((1 *)) work-%offset%)
                 ldwork)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
                var-13))
              (when var-6 (setf ib var-6))
              (when var-8 (setf lda var-8))
              (when var-10 (setf ldwork var-10))
              (when var-12 (setf lda var-12))
              (when var-14 (setf ldwork var-14)))))
          label10)))
      (t (setf i 1)))
    (if (<= i k)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zgeqr2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i) ((1 *))
                                   tau-%offset%)
             work iinfo)
          (declare (ignore var-0 var-1 var-2 var-4 var-5))
          (setf lda var-3)
          (setf iinfo var-6)))
```

```
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce iws 'f2cl-lib:complex16))
      (go end_label)
     end_label
      (return (values m n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgeqrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::zgeqr2
                                              fortran-to-lisp::xerbla
                                              fortran-to-lisp::ilaenv))))
```

———-

35.20 zgeqrfp

35.20.1 Axiom unit tests

— zgeqrfp.input —

```
)set break resume
)sys rm -f zgeqrfp.output
)spool zgeqrfp.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.20.2 Axiom help page [Demm08]

— **zgeqrfp.help** —

```
==================================================================
zgeqrfp examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgeqrfp (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           ZGEQRFP

Function/Subroutine Documentation
   subroutine zgeqrfp (integer M, integer N, complex*16, dimension( lda, * )
       A, integer LDA, complex*16, dimension( * ) TAU, complex*16, dimension(
       * ) WORK, integer LWORK, integer INFO)
       ZGEQRFP

Purpose:
   ZGEQRFP computes a QR factorization of a complex M-by-N matrix A:
   A = Q * R. The diagonal entries of R are real and nonnegative.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
```

```
  On exit, the elements on and above the diagonal of the array
  contain the min(M,N)-by-N upper trapezoidal matrix R (R is
  upper triangular if m >= n). The diagonal entries of R
  are real and nonnegative; The elements below the diagonal,
  with the array TAU, represent the unitary matrix Q as a
  product of min(m,n) elementary reflectors (see Further Details).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

TAU
  TAU is COMPLEX*16 array, dimension (min(M,N))
  The scalar factors of the elementary reflectors (see Further Details).

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,N).
  For optimum performance LWORK >= N*NB, where NB is
  the optimal blocksize.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details:

  The matrix Q is represented as a product of elementary reflectors

  Q = H(1) H(2) . . . H(k), where k = min(m,n).

  Each H(i) has the form

  H(i) = I - tau * v * v**H

  where tau is a complex scalar, and v is a complex vector with
  v(1:i-1) = 0 and v(i) = 1; v(i+1:m) is stored on exit in A(i+1:m,i),
  and tau in TAU(i).
```

———-

35.20.3 fortran code

— **zgeqrfp.f** —

```
*  =====================================================================
      SUBROUTINE ZGEQRFP( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, LDWORK, LWKOPT, NB,
     $                   NBMIN, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGEQR2P, ZLARFB, ZLARFT
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NB = ILAENV( 1, 'ZGEQRF', ' ', M, N, -1, -1 )
      LWKOPT = N*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
```

```
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEQRFP', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      K = MIN( M, N )
      IF( K.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      NX = 0
      IWS = N
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'ZGEQRF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = N
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'ZGEQRF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
```

```
*
*        Use blocked code initially
*
         DO 10 I = 1, K - NX, NB
            IB = MIN( K-I+1, NB )
*
*           Compute the QR factorization of the current block
*           A(i:m,i:i+ib-1)
*
            CALL ZGEQR2P( M-I+1, IB, A( I, I ), LDA, TAU( I ), WORK,
     $                    IINFO )
            IF( I+IB.LE.N ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i) H(i+1) . . . H(i+ib-1)
*
               CALL ZLARFT( 'Forward', 'Columnwise', M-I+1, IB,
     $                      A( I, I ), LDA, TAU( I ), WORK, LDWORK )
*
*              Apply H**H to A(i:m,i+ib:n) from the left
*
               CALL ZLARFB( 'Left', 'Conjugate transpose', 'Forward',
     $                      'Columnwise', M-I+1, N-I-IB+1, IB,
     $                      A( I, I ), LDA, WORK, LDWORK, A( I, I+IB ),
     $                      LDA, WORK( IB+1 ), LDWORK )
            END IF
   10    CONTINUE
      ELSE
         I = 1
      END IF
*
*     Use unblocked code to factor the last or only block.
*
      IF( I.LE.K )
     $   CALL ZGEQR2P( M-I+1, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                 IINFO )
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of ZGEQRFP
*
      END
```

———-

35.20.4 lisp code

— zgeqrfp.lisp —

```
(defun zgeqrfp (m n a lda tau work lwork info)
  (declare (type (array f2cl-lib:complex16 (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex16 a-%data% a-%offset%)
       (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ldwork 0) (lwkopt 0) (nb 0)
           (nbmin 0) (nx 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) nx nbmin nb lwkopt ldwork k iws iinfo
                ib i))
      (setf info 0)
      (setf nb
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 1 "ZGEQRF" " " m n -1 -1)
                (declare (ignore var-0 var-1 var-2 var-5 var-6))
                (setf m var-3)
                (setf n var-4)
                ret-val))
      (setf lwkopt (f2cl-lib:int-mul n nb))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce lwkopt 'f2cl-lib:complex16))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4))
            ((and
              (< lwork
                 (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
              (not lquery))
             (setf info -7)))
      (cond
       ((/= info 0) (xerbla "ZGEQRFP" (f2cl-lib:int-sub info)) (go end_label))
       (lquery (go end_label)))
      (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
      (cond
       ((= k 0)
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce 1 'f2cl-lib:complex16))
        (go end_label)))
      (setf nbmin 2)
      (setf nx 0)
      (setf iws n)
      (cond
       ((and (> nb 1) (< nb k))
        (setf nx
```

```
              (max (the f2cl-lib:integer4 0)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
                            (ilaenv 3 "ZGEQRF" " " m n -1 -1)
                          (declare (ignore var-0 var-1 var-2 var-5 var-6))
                          (setf m var-3)
                          (setf n var-4)
                          ret-val))))
      (cond
       ((< nx k) (setf ldwork n) (setf iws (f2cl-lib:int-mul ldwork nb))
        (cond
         ((< lwork iws)
          (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
          (setf nbmin
                  (max (the f2cl-lib:integer4 2)
                       (the f2cl-lib:integer4
                            (multiple-value-bind
                                (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                                 var-6)
                                (ilaenv 2 "ZGEQRF" " " m n -1 -1)
                              (declare
                               (ignore var-0 var-1 var-2 var-5 var-6))
                              (setf m var-3)
                              (setf n var-4)
                              ret-val))))))))))
    (cond
     ((and (>= nb nbmin) (< nb k) (< nx k))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                    ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub nx))) nil)
        (tagbody
          (setf ib
                  (min
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
                   (the f2cl-lib:integer4 nb)))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (zgeqr2p (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                                     ((1 *)) tau-%offset%)
               work iinfo)
            (declare (ignore var-0 var-1 var-2 var-4 var-5))
            (setf lda var-3)
            (setf iinfo var-6))
          (cond
           ((<= (f2cl-lib:int-add i ib) n)
```

```
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (zlarft "Forward" "Columnwise"
                 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                                       ((1 *)) tau-%offset%)
                 work ldwork)
              (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
              (when var-3 (setf ib var-3))
              (when var-5 (setf lda var-5))
              (when var-8 (setf ldwork var-8)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                 var-10 var-11 var-12 var-13 var-14)
                (zlarfb "Left" "Conjugate transpose" "Forward" "Columnwise"
                 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
                 (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1) ib
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda work ldwork
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                       (i (f2cl-lib:int-add i ib))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                       ((+ ib 1)) ((1 *)) work-%offset%)
                 ldwork)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
                var-13))
              (when var-6 (setf ib var-6))
              (when var-8 (setf lda var-8))
              (when var-10 (setf ldwork var-10))
              (when var-12 (setf lda var-12))
              (when var-14 (setf ldwork var-14)))))
         label10)))
     (t (setf i 1)))
    (if (<= i k)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zgeqr2p (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i) ((1 *))
                                   tau-%offset%)
             work iinfo)
```

```
            (declare (ignore var-0 var-1 var-2 var-4 var-5))
            (setf lda var-3)
            (setf iinfo var-6)))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce iws 'f2cl-lib:complex16))
      (go end_label)
     end_label
      (return (values m n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgeqrfp
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::zgeqr2p
                                              fortran-to-lisp::xerbla
                                              fortran-to-lisp::ilaenv))))
```

———-

35.21 zgeqrt

35.21.1 Axiom unit tests

— zgeqrt.input —

```
)set break resume
```

```
)sys rm -f zgeqrt.output
)spool zgeqrt.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

35.21.2 Axiom help page

— zgeqrt.help —

```
====================================================================
zgeqrt examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgeqrt (M, N, NB, A, LDA, T, LDT, WORK, INFO)
           ZGEQRT

Function/Subroutine Documentation
   subroutine zgeqrt (integer M, integer N, integer NB, complex*16, dimension(
       lda, * ) A, integer LDA, complex*16, dimension( ldt, * ) T, integer
       LDT, complex*16, dimension( * ) WORK, integer INFO)
       ZGEQRT

Purpose:
   ZGEQRT computes a blocked QR factorization of a complex M-by-N matrix A
   using the compact WY representation of Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.
```

```
 NB
   NB is INTEGER
   The block size to be used in the blocked QR.  MIN(M,N) >= NB >= 1.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit, the elements on and above the diagonal of the array
   contain the min(M,N)-by-N upper trapezoidal matrix R (R is
   upper triangular if M >= N); the elements below the diagonal
   are the columns of V.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 T
   T is COMPLEX*16 array, dimension (LDT,MIN(M,N))
   The upper triangular block reflectors stored in compact form
   as a sequence of upper triangular blocks.  See below
   for further details.

 LDT
   LDT is INTEGER
   The leading dimension of the array T.  LDT >= NB.

 WORK
   WORK is COMPLEX*16 array, dimension (NB*N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details:

   The matrix V stores the elementary reflectors H(i) in the i-th column
   below the diagonal. For example, if M=5 and N=3, the matrix V is

                         V = (  1       )
                             ( v1  1    )
                             ( v1 v2  1 )
                             ( v1 v2 v3 )
                             ( v1 v2 v3 )

   where the vi's represent the vectors which define H(i), which are returned
   in the matrix A.  The 1's along the diagonal of V are not stored in A.

   Let K=MIN(M,N).  The number of blocks is B = ceiling(K/NB), where each
   block is of order NB except for the last block, which is of order
```

```
IB = K - (B-1)*NB.  For each of the B blocks, a upper triangular block
reflector factor is computed: T1, T2, ..., TB.  The NB-by-NB (and IB-by-IB
for the last block) T's are stored in the NB-by-N matrix T as

                       T = (T1 T2 ... TB).
```

———-

35.21.3 fortran code

— **zgeqrt.f** —

```
*  ===================================================================
      SUBROUTINE ZGEQRT( M, N, NB, A, LDA, T, LDT, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      INTEGER INFO, LDA, LDT, M, N, NB
*     ..
*     .. Array Arguments ..
      COMPLEX*16 A( LDA, * ), T( LDT, * ), WORK( * )
*     ..
*
* ====================================================================
*
*     ..
*     .. Local Scalars ..
      INTEGER    I, IB, IINFO, K
      LOGICAL    USE_RECURSIVE_QR
      PARAMETER( USE_RECURSIVE_QR=.TRUE. )
*     ..
*     .. External Subroutines ..
      EXTERNAL   ZGEQRT2, ZGEQRT3, ZLARFB, XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
```

```
      ELSE IF( NB.LT.1 .OR. ( NB.GT.MIN(M,N) .AND. MIN(M,N).GT.0 ) )THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDT.LT.NB ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEQRT', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      K = MIN( M, N )
      IF( K.EQ.0 ) RETURN
*
*     Blocked loop of length K
*
      DO I = 1, K,  NB
         IB = MIN( K-I+1, NB )
*
*     Compute the QR factorization of the current block A(I:M,I:I+IB-1)
*
         IF( USE_RECURSIVE_QR ) THEN
            CALL ZGEQRT3( M-I+1, IB, A(I,I), LDA, T(1,I), LDT, IINFO )
         ELSE
            CALL ZGEQRT2( M-I+1, IB, A(I,I), LDA, T(1,I), LDT, IINFO )
         END IF
         IF( I+IB.LE.N ) THEN
*
*     Update by applying H**H to A(I:M,I+IB:N) from the left
*
            CALL ZLARFB( 'L', 'C', 'F', 'C', M-I+1, N-I-IB+1, IB,
     $                   A( I, I ), LDA, T( 1, I ), LDT,
     $                   A( I, I+IB ), LDA, WORK , N-I-IB+1 )
         END IF
      END DO
      RETURN
*
*     End of ZGEQRT
*
      END
```

———-

35.21.4 lisp code

— zgeqrt.lisp —

```
(let* ((use_recursive_qr f2cl-lib:%true%))
  (declare (type f2cl-lib:logical use_recursive_qr)
           (ignorable use_recursive_qr))
  (defun zgeqrt (m n nb a lda t$ ldt work info)
    (declare (type (array f2cl-lib:complex16 (*)) work t$ a)
             (type (f2cl-lib:integer4) info ldt lda nb n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (t$ f2cl-lib:complex16 t$-%data% t$-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%))
      (prog ((i 0) (ib 0) (iinfo 0) (k 0))
        (declare (type (f2cl-lib:integer4) i ib iinfo k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((or (< nb 1)
                   (and
                    (> nb
                       (min (the f2cl-lib:integer4 m)
                            (the f2cl-lib:integer4 n)))
                    (>
                     (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n))
                     0)))
               (setf info -3))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -5))
              ((< ldt nb) (setf info -7)))
        (cond
         ((/= info 0) (xerbla "ZGEQRT" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (if (= k 0)
            (go end_label))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                      ((> i k) nil)
          (tagbody
            (setf ib
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
                     (the f2cl-lib:integer4 nb)))
            (cond
             (use_recursive_qr
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (zgeqrt3 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda
```

```
                (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 i)
                                      ((1 ldt) (1 *)) t$-%offset%)
                ldt iinfo)
            (declare (ignore var-0 var-2 var-4))
            (when var-1 (setf ib var-1))
            (when var-3 (setf lda var-3))
            (when var-5 (setf ldt var-5))
            (when var-6 (setf iinfo var-6))))
         (t
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (zgeqrt2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                      ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 i)
                                      ((1 ldt) (1 *)) t$-%offset%)
               ldt iinfo)
            (declare (ignore var-0 var-1 var-2 var-4))
            (setf lda var-3)
            (setf ldt var-5)
            (setf iinfo var-6))))
        (cond
         ((<= (f2cl-lib:int-add i ib) n)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10 var-11 var-12 var-13 var-14)
              (zlarfb "L" "C" "F" "C"
               (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
               (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1) ib
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                      ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 i)
                                      ((1 ldt) (1 *)) t$-%offset%)
               ldt
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                      (i (f2cl-lib:int-add i ib))
                                      ((1 lda) (1 *)) a-%offset%)
               lda work (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1))
            (declare
             (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
              var-13 var-14))
            (when var-6 (setf ib var-6))
            (when var-8 (setf lda var-8))
            (when var-10 (setf ldt var-10))
            (when var-12 (setf lda var-12)))))
      label100000))
   (go end_label)
  end_label
   (return (values nil nil nil nil lda nil ldt nil info))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgeqrt
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldt nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::zgeqrt2
                                              fortran-to-lisp::xerbla))))
```

———-

35.22 zgeqrt2

35.22.1 Axiom unit tests

— zgeqrt2.input —

```
)set break resume
)sys rm -f zgeqrt2.output
)spool zgeqrt2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.22.2 Axiom help page

— zgeqrt2.help —

```
=====================================================================
zgeqrt2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgeqrt2 (M, N, A, LDA, T, LDT, INFO)
           ZGEQRT2 computes a QR factorization of a general real or complex
           matrix using the compact WY representation of Q.

Function/Subroutine Documentation
   subroutine zgeqrt2 (integer M, integer N, complex*16, dimension( lda, * )
       A, integer LDA, complex*16, dimension( ldt, * ) T, integer LDT, integer
       INFO)
       ZGEQRT2 computes a QR factorization of a general real or complex matrix
       using the compact WY representation of Q.

Purpose:
   ZGEQRT2 computes a QR factorization of a complex M-by-N matrix A,
   using the compact WY representation of Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= N.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the complex M-by-N matrix A.  On exit, the elements on and
   above the diagonal contain the N-by-N upper triangular matrix R; the
   elements below the diagonal are the columns of V.  See below for
   further details.
```

```
 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 T
   T is COMPLEX*16 array, dimension (LDT,N)
   The N-by-N upper triangular factor of the block reflector.
   The elements on and above the diagonal contain the block
   reflector T; the elements below the diagonal are not used.
   See below for further details.

 LDT
   LDT is INTEGER
   The leading dimension of the array T.  LDT >= max(1,N).

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

Further Details:

   The matrix V stores the elementary reflectors H(i) in the i-th column
   below the diagonal. For example, if M=5 and N=3, the matrix V is

                         V = (  1       )
                             ( v1  1    )
                             ( v1 v2  1 )
                             ( v1 v2 v3 )
                             ( v1 v2 v3 )

   where the vi's represent the vectors which define H(i), which are returned
   in the matrix A.  The 1's along the diagonal of V are not stored in A.  The
   block reflector H is then given by

                         H = I - V * T * V**H

   where V**H is the conjugate transpose of V.
```

———-

35.22.3 fortran code

— **zgeqrt2.f** —

```
*  =====================================================================
      SUBROUTINE ZGEQRT2( M, N, A, LDA, T, LDT, INFO )
```

```
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER   INFO, LDA, LDT, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16   A( LDA, * ), T( LDT, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16  ONE, ZERO
      PARAMETER( ONE = (1.0D+00,0.0D+00), ZERO = (0.0D+00,0.0D+00) )
*     ..
*     .. Local Scalars ..
      INTEGER   I, K
      COMPLEX*16   AII, ALPHA
*     ..
*     .. External Subroutines ..
      EXTERNAL  ZLARFG, ZGEMV, ZGERC, ZTRMV, XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LDT.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEQRT2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO I = 1, K
*
*        Generate elem. refl. H(i) to annihilate A(i+1:m,i), tau(I) -> T(I,1)
```

```
*
         CALL ZLARFG( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
     $                T( I, 1 ) )
         IF( I.LT.N ) THEN
*
*           Apply H(i) to A(I:M,I+1:N) from the left
*
            AII = A( I, I )
            A( I, I ) = ONE
*
*           W(1:N-I) := A(I:M,I+1:N)^H * A(I:M,I) [W = T(:,N)]
*
            CALL ZGEMV( 'C',M-I+1, N-I, ONE, A( I, I+1 ), LDA,
     $                  A( I, I ), 1, ZERO, T( 1, N ), 1 )
*
*           A(I:M,I+1:N) = A(I:m,I+1:N) + alpha*A(I:M,I)*W(1:N-1)^H
*
            ALPHA = -CONJG(T( I, 1 ))
            CALL ZGERC( M-I+1, N-I, ALPHA, A( I, I ), 1,
     $           T( 1, N ), 1, A( I, I+1 ), LDA )
            A( I, I ) = AII
         END IF
      END DO
*
      DO I = 2, N
         AII = A( I, I )
         A( I, I ) = ONE
*
*        T(1:I-1,I) := alpha * A(I:M,1:I-1)**H * A(I:M,I)
*
         ALPHA = -T( I, 1 )
         CALL ZGEMV( 'C', M-I+1, I-1, ALPHA, A( I, 1 ), LDA,
     $               A( I, I ), 1, ZERO, T( 1, I ), 1 )
         A( I, I ) = AII
*
*        T(1:I-1,I) := T(1:I-1,1:I-1) * T(1:I-1,I)
*
         CALL ZTRMV( 'U', 'N', 'N', I-1, T, LDT, T( 1, I ), 1 )
*
*           T(I,I) = tau(I)
*
            T( I, I ) = T( I, 1 )
            T( I, 1) = ZERO
      END DO

*
*     End of ZGEQRT2
*
      END
```

———-

35.22.4 lisp code

— **zgeqrt2.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)) (zero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (type (f2cl-lib:complex16) zero)
           (ignorable one zero))
  (defun zgeqrt2 (m n a lda t$ ldt info)
    (declare (type (array f2cl-lib:complex16 (*)) t$ a)
             (type (f2cl-lib:integer4) info ldt lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (t$ f2cl-lib:complex16 t$-%data% t$-%offset%))
      (prog ((aii #C(0.0d0 0.0d0)) (alpha #C(0.0d0 0.0d0)) (i 0) (k 0))
        (declare (type (f2cl-lib:complex16) aii alpha)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4))
              ((< ldt
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -6)))
        (cond
         ((/= info 0) (xerbla "ZGEQRT2" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i k) nil)
          (tagbody
            (zlarfg (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   ((min (f2cl-lib:int-add i 1) m) i)
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (i 1)
                                   ((1 ldt) (1 *)) t$-%offset%))
            (cond
             ((< i n)
              (setf aii
                      (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                     a-%offset%))
```

```
        (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                one)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10)
            (zgemv "C" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:int-sub n i) one
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   (i (f2cl-lib:int-add i 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             1 zero
             (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 n)
                                   ((1 ldt) (1 *)) t$-%offset%)
             1)
          (declare
           (ignore var-0 var-1 var-2 var-4 var-6 var-7 var-9 var-10))
          (when var-3 (setf one var-3))
          (when var-5 (setf lda var-5))
          (when var-8 (setf zero var-8)))
        (setf alpha
                (-
                 (f2cl-lib:conjg
                  (f2cl-lib:fref t$-%data% (i 1) ((1 ldt) (1 *))
                                 t$-%offset%))))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (zgerc (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:int-sub n i) alpha
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 n)
                                   ((1 ldt) (1 *)) t$-%offset%)
             1
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   (i (f2cl-lib:int-add i 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7))
          (when var-2 (setf alpha var-2))
          (when var-8 (setf lda var-8)))
        (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                aii)))
    label100000))
  (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                ((> i n) nil)
   (tagbody
```

```
            (setf aii
                    (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%))
            (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                    one)
            (setf alpha
                    (-
                     (f2cl-lib:fref t$-%data% (i 1) ((1 ldt) (1 *))
                                    t$-%offset%)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                 var-10)
                (zgemv "C" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
                 (f2cl-lib:int-sub i 1) alpha
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i 1)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                       ((1 lda) (1 *)) a-%offset%)
                 1 zero
                 (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 i)
                                       ((1 ldt) (1 *)) t$-%offset%)
                 1)
              (declare
               (ignore var-0 var-1 var-2 var-4 var-6 var-7 var-9 var-10))
              (when var-3 (setf alpha var-3))
              (when var-5 (setf lda var-5))
              (when var-8 (setf zero var-8)))
            (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                    aii)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
                (ztrmv "U" "N" "N" (f2cl-lib:int-sub i 1) t$ ldt
                 (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 i)
                                       ((1 ldt) (1 *)) t$-%offset%)
                 1)
              (declare (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7))
              (when var-5 (setf ldt var-5)))
            (setf (f2cl-lib:fref t$-%data% (i i) ((1 ldt) (1 *)) t$-%offset%)
                    (f2cl-lib:fref t$-%data% (i 1) ((1 ldt) (1 *))
                                   t$-%offset%))
            (setf (f2cl-lib:fref t$-%data% (i 1) ((1 ldt) (1 *)) t$-%offset%)
                    zero)
          label100001))
       end_label
        (return (values nil nil nil lda nil ldt info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgeqrt2
```

```
        fortran-to-lisp::*f2cl-function-info*)
    (fortran-to-lisp::make-f2cl-finfo :arg-types
                                      '((fortran-to-lisp::integer4)
                                        (fortran-to-lisp::integer4)
                                        (array fortran-to-lisp::complex16
                                         (*))
                                        (fortran-to-lisp::integer4)
                                        (array fortran-to-lisp::complex16
                                         (*))
                                        (fortran-to-lisp::integer4)
                                        (fortran-to-lisp::integer4))
                                      :return-values
                                      '(nil nil nil fortran-to-lisp::lda
                                        nil fortran-to-lisp::ldt
                                        fortran-to-lisp::info)
                                      :calls '(fortran-to-lisp::xerbla))))
```

———-

35.23 zgeqrt3

35.23.1 Axiom unit tests

— **zgeqrt3.input** —

```
)set break resume
)sys rm -f zgeqrt3.output
)spool zgeqrt3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.23.2 Axiom help page [Elmr00]

— **zgeqrt3.help** —

```
==================================================================
```

```
zgeqrt3 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       recursive subroutine zgeqrt3 (M, N, A, LDA, T, LDT, INFO)
           ZGEQRT3 recursively computes a QR factorization of a general real
           or complex matrix using the compact WY representation of Q.

Function/Subroutine Documentation
   recursive subroutine zgeqrt3 (integer M, integer N, complex*16, dimension(
       lda, * ) A, integer LDA, complex*16, dimension( ldt, * ) T, integer
       LDT, integer INFO)
       ZGEQRT3 recursively computes a QR factorization of a general real or
       complex matrix using the compact WY representation of Q.

Purpose:
   ZGEQRT3 recursively computes a QR factorization of a complex M-by-N
   matrix A, using the compact WY representation of Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= N.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the complex M-by-N matrix A.  On exit, the elements on
   and above the diagonal contain the N-by-N upper triangular matrix R;
   the elements below the diagonal are the columns of V.  See below for
   further details.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 T
   T is COMPLEX*16 array, dimension (LDT,N)
   The N-by-N upper triangular factor of the block reflector.
   The elements on and above the diagonal contain the block
   reflector T; the elements below the diagonal are not used.
```

```
   See below for further details.

 LDT
   LDT is INTEGER
   The leading dimension of the array T.  LDT >= max(1,N).

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

Further Details:

   The matrix V stores the elementary reflectors H(i) in the i-th column
   below the diagonal. For example, if M=5 and N=3, the matrix V is

                         V = (  1       )
                             ( v1  1    )
                             ( v1 v2  1 )
                             ( v1 v2 v3 )
                             ( v1 v2 v3 )

   where the vi's represent the vectors which define H(i), which are returned
   in the matrix A.  The 1's along the diagonal of V are not stored in A.  The
   block reflector H is then given by

                         H = I - V * T * V**H

   where V**H is the conjugate transpose of V.
```

———-

35.23.3 fortran code

— **zgeqrt3.f** —

```
*  ===================================================================
      RECURSIVE SUBROUTINE ZGEQRT3( M, N, A, LDA, T, LDT, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER   INFO, LDA, M, N, LDT
*     ..
```

```
*     .. Array Arguments ..
      COMPLEX*16   A( LDA, * ), T( LDT, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16   ONE
      PARAMETER ( ONE = (1.0D+00,0.0D+00) )
*     ..
*     .. Local Scalars ..
      INTEGER   I, I1, J, J1, N1, N2, IINFO
*     ..
*     .. External Subroutines ..
      EXTERNAL  ZLARFG, ZTRMM, ZGEMM, XERBLA
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      IF( N .LT. 0 ) THEN
         INFO = -2
      ELSE IF( M .LT. N ) THEN
         INFO = -1
      ELSE IF( LDA .LT. MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LDT .LT. MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEQRT3', -INFO )
         RETURN
      END IF
*
      IF( N.EQ.1 ) THEN
*
*        Compute Householder transform when N=1
*
         CALL ZLARFG( M, A, A( MIN( 2, M ), 1 ), 1, T )
*
      ELSE
*
*        Otherwise, split A into blocks...
*
         N1 = N/2
         N2 = N-N1
         J1 = MIN( N1+1, N )
         I1 = MIN( N+1, M )
*
*        Compute A(1:M,1:N1) <- (Y1,R1,T1), where Q1 = I - Y1 T1 Y1^H
*
```

```
         CALL ZGEQRT3( M, N1, A, LDA, T, LDT, IINFO )
*
*        Compute A(1:M,J1:N) = Q1^H A(1:M,J1:N) [workspace: T(1:N1,J1:N)]
*
         DO J=1,N2
            DO I=1,N1
               T( I, J+N1 ) = A( I, J+N1 )
            END DO
         END DO
         CALL ZTRMM( 'L', 'L', 'C', 'U', N1, N2, ONE,
     &               A, LDA, T( 1, J1 ), LDT )
*
         CALL ZGEMM( 'C', 'N', N1, N2, M-N1, ONE, A( J1, 1 ), LDA,
     &               A( J1, J1 ), LDA, ONE, T( 1, J1 ), LDT)
*
         CALL ZTRMM( 'L', 'U', 'C', 'N', N1, N2, ONE,
     &               T, LDT, T( 1, J1 ), LDT )
*
         CALL ZGEMM( 'N', 'N', M-N1, N2, N1, -ONE, A( J1, 1 ), LDA,
     &               T( 1, J1 ), LDT, ONE, A( J1, J1 ), LDA )
*
         CALL ZTRMM( 'L', 'L', 'N', 'U', N1, N2, ONE,
     &               A, LDA, T( 1, J1 ), LDT )
*
         DO J=1,N2
            DO I=1,N1
               A( I, J+N1 ) = A( I, J+N1 ) - T( I, J+N1 )
            END DO
         END DO
*
*        Compute A(J1:M,J1:N) <- (Y2,R2,T2) where Q2 = I - Y2 T2 Y2^H
*
         CALL ZGEQRT3( M-N1, N2, A( J1, J1 ), LDA,
     &                T( J1, J1 ), LDT, IINFO )
*
*        Compute T3 = T(1:N1,J1:N) = -T1 Y1^H Y2 T2
*
         DO I=1,N1
            DO J=1,N2
               T( I, J+N1 ) = CONJG(A( J+N1, I ))
            END DO
         END DO
*
         CALL ZTRMM( 'R', 'L', 'N', 'U', N1, N2, ONE,
     &               A( J1, J1 ), LDA, T( 1, J1 ), LDT )
*
         CALL ZGEMM( 'C', 'N', N1, N2, M-N, ONE, A( I1, 1 ), LDA,
     &               A( I1, J1 ), LDA, ONE, T( 1, J1 ), LDT )
*
         CALL ZTRMM( 'L', 'U', 'N', 'N', N1, N2, -ONE, T, LDT,
```

```
     &               T( 1, J1 ), LDT )
*
         CALL ZTRMM( 'R', 'U', 'N', 'N', N1, N2, ONE,
     &               T( J1, J1 ), LDT, T( 1, J1 ), LDT )
*
*        Y = (Y1,Y2); R = [ R1  A(1:N1,J1:N) ];  T = [T1 T3]
*                         [  0        R2     ]       [ 0 T2]
*
      END IF
*
      RETURN
*
*     End of ZGEQRT3
*
      END
```

———-

35.23.4 lisp code

— **zgeqrt3.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (ignorable one))
  (defun *main* ()
    (prog ((i 0) (i1 0) (j 0) (j1 0) (n1 0) (n2 0) (iinfo 0) (m 0) (info 0)
           (ldt 0)
           (t$
            (make-array (the fixnum (reduce #'* (list ldt *))) :element-type
                        'f2cl-lib:complex16))
           (lda 0)
           (a
            (make-array (the fixnum (reduce #'* (list lda *))) :element-type
                        'f2cl-lib:complex16))
           (n 0) (subroutine 0.0))
      (declare (type (single-float) subroutine)
               (type (array f2cl-lib:complex16 (*)) a t$)
               (type (f2cl-lib:integer4) n lda ldt info i i1 j j1 n1 n2 iinfo
                m))
      '"****NOT TRANSLATED: (RECURSIVE SUBROUTINE ZGEQRT3 (M |,| N |,| A |,| LDA |,| T |,| LDT |
      (setf info 0)
      (cond ((< n 0) (setf info -2)) ((< m n) (setf info -1))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4))
            ((< ldt (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -6)))
      (cond
```

```
 ((/= info 0) (xerbla "ZGEQRT3" (f2cl-lib:int-sub info)) (go end_label)))
(cond
 ((= n 1)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (zlarfg m a
       (f2cl-lib:array-slice a f2cl-lib:complex16 ((min 2 m) 1)
                             ((1 lda) (1 *)))
       1 t$)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf m var-0))))
 (t (setf n1 (the f2cl-lib:integer4 (truncate n 2)))
  (setf n2 (f2cl-lib:int-sub n n1))
  (setf j1
          (min (the f2cl-lib:integer4 (f2cl-lib:int-add n1 1))
               (the f2cl-lib:integer4 n)))
  (setf i1
          (min (the f2cl-lib:integer4 (f2cl-lib:int-add n 1))
               (the f2cl-lib:integer4 m)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zgeqrt3 m n1 a lda t$ ldt iinfo)
    (declare (ignore var-2 var-4))
    (when var-0 (setf m var-0))
    (when var-1 (setf n1 var-1))
    (when var-3 (setf lda var-3))
    (when var-5 (setf ldt var-5))
    (when var-6 (setf iinfo var-6)))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n2) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n1) nil)
        (tagbody
          (setf (f2cl-lib:fref t$ (i (f2cl-lib:int-add j n1))
                                  ((1 ldt) (1 *)))
                  (f2cl-lib:fref a (i (f2cl-lib:int-add j n1))
                                    ((1 lda) (1 *))))
        label100001))
    label100000))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ztrmm "L" "L" "C" "U" n1 n2 one a lda
       (f2cl-lib:array-slice t$ f2cl-lib:complex16 (1 j1)
                             ((1 ldt) (1 *)))
       ldt)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n1 var-4))
    (when var-5 (setf n2 var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
```

```
  (when var-10 (setf ldt var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (zgemm "C" "N" n1 n2 (f2cl-lib:int-sub m n1) one
     (f2cl-lib:array-slice a f2cl-lib:complex16 (j1 1) ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice a f2cl-lib:complex16 (j1 j1)
                           ((1 lda) (1 *)))
     lda one
     (f2cl-lib:array-slice t$ f2cl-lib:complex16 (1 j1)
                           ((1 ldt) (1 *)))
     ldt)
  (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
  (when var-2 (setf n1 var-2))
  (when var-3 (setf n2 var-3))
  (when var-5 (setf one var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf ldt var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "L" "U" "C" "N" n1 n2 one t$ ldt
     (f2cl-lib:array-slice t$ f2cl-lib:complex16 (1 j1)
                           ((1 ldt) (1 *)))
     ldt)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf n2 var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldt var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (zgemm "N" "N" (f2cl-lib:int-sub m n1) n2 n1 (- one)
     (f2cl-lib:array-slice a f2cl-lib:complex16 (j1 1) ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice t$ f2cl-lib:complex16 (1 j1)
                           ((1 ldt) (1 *)))
     ldt one
     (f2cl-lib:array-slice a f2cl-lib:complex16 (j1 j1)
                           ((1 lda) (1 *)))
     lda)
  (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
  (when var-3 (setf n2 var-3))
  (when var-4 (setf n1 var-4))
  (when var-7 (setf lda var-7))
```

```
  (when var-9 (setf ldt var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf lda var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "L" "L" "N" "U" n1 n2 one a lda
     (f2cl-lib:array-slice t$ f2cl-lib:complex16 (1 j1)
                           ((1 ldt) (1 *)))
     ldt)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf n2 var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf ldt var-10)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n2) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n1) nil)
      (tagbody
        (setf (f2cl-lib:fref a (i (f2cl-lib:int-add j n1))
                              ((1 lda) (1 *)))
                (-
                 (f2cl-lib:fref a (i (f2cl-lib:int-add j n1))
                                  ((1 lda) (1 *)))
                 (f2cl-lib:fref t$ (i (f2cl-lib:int-add j n1))
                                  ((1 ldt) (1 *)))))
       label100003))
   label100002))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zgeqrt3 (f2cl-lib:int-sub m n1) n2
     (f2cl-lib:array-slice a f2cl-lib:complex16 (j1 j1)
                           ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice t$ f2cl-lib:complex16 (j1 j1)
                           ((1 ldt) (1 *)))
     ldt iinfo)
  (declare (ignore var-0 var-2 var-4))
  (when var-1 (setf n2 var-1))
  (when var-3 (setf lda var-3))
  (when var-5 (setf ldt var-5))
  (when var-6 (setf iinfo var-6)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n1) nil)
  (tagbody
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n2) nil)
      (tagbody
```

```
          (setf (f2cl-lib:fref t$ (i (f2cl-lib:int-add j n1))
                                  ((1 ldt) (1 *)))
                (f2cl-lib:conjg
                 (f2cl-lib:fref a ((f2cl-lib:int-add j n1) i)
                                  ((1 lda) (1 *)))))
        label100005))
    label100004))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "R" "L" "N" "U" n1 n2 one
     (f2cl-lib:array-slice a f2cl-lib:complex16 (j1 j1)
                           ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice t$ f2cl-lib:complex16 (1 j1)
                           ((1 ldt) (1 *)))
     ldt)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf n2 var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf ldt var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (zgemm "C" "N" n1 n2 (f2cl-lib:int-sub m n) one
     (f2cl-lib:array-slice a f2cl-lib:complex16 (i1 1) ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice a f2cl-lib:complex16 (i1 j1)
                           ((1 lda) (1 *)))
     lda one
     (f2cl-lib:array-slice t$ f2cl-lib:complex16 (1 j1)
                           ((1 ldt) (1 *)))
     ldt)
  (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
  (when var-2 (setf n1 var-2))
  (when var-3 (setf n2 var-3))
  (when var-5 (setf one var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf ldt var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "L" "U" "N" "N" n1 n2 (- one) t$ ldt
     (f2cl-lib:array-slice t$ f2cl-lib:complex16 (1 j1)
                           ((1 ldt) (1 *)))
     ldt)
```

```
          (declare (ignore var-0 var-1 var-2 var-3 var-6 var-7 var-9))
          (when var-4 (setf n1 var-4))
          (when var-5 (setf n2 var-5))
          (when var-8 (setf ldt var-8))
          (when var-10 (setf ldt var-10)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10)
            (ztrmm "R" "U" "N" "N" n1 n2 one
             (f2cl-lib:array-slice t$ f2cl-lib:complex16 (j1 j1)
                                   ((1 ldt) (1 *)))
             ldt
             (f2cl-lib:array-slice t$ f2cl-lib:complex16 (1 j1)
                                   ((1 ldt) (1 *)))
             ldt)
          (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
          (when var-4 (setf n1 var-4))
          (when var-5 (setf n2 var-5))
          (when var-6 (setf one var-6))
          (when var-8 (setf ldt var-8))
          (when var-10 (setf ldt var-10)))))
      (go end_label)
     end_label
      (return nil))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::*main*
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types 'nil :return-values 'nil
                                            :calls 'nil)))
```

———-

35.24 zgerfs

35.24.1 Axiom unit tests

— **zgerfs.input** —

```
)set break resume
)sys rm -f zgerfs.output
)spool zgerfs.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.24.2 Axiom help page

— **zgerfs.help** —

```
==================================================================
zgerfs examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgerfs (TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB, X,
           LDX, FERR, BERR, WORK, RWORK, INFO)
           ZGERFS

Function/Subroutine Documentation
   subroutine zgerfs (character TRANS, integer N, integer NRHS, complex*16,
       dimension( lda, * ) A, integer LDA, complex*16, dimension( ldaf, * )
       AF, integer LDAF, integer, dimension( * ) IPIV, complex*16, dimension(
       ldb, * ) B, integer LDB, complex*16, dimension( ldx, * ) X, integer
       LDX, double precision, dimension( * ) FERR, double precision,
       dimension( * ) BERR, complex*16, dimension( * ) WORK, double precision,
       dimension( * ) RWORK, integer INFO)
       ZGERFS

Purpose:
   ZGERFS improves the computed solution to a system of linear
   equations and provides error bounds and backward error estimates for
   the solution.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
```

```
  = 'C':  A**H * X = B  (Conjugate transpose)

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  The original N-by-N matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX*16 array, dimension (LDAF,N)
  The factors L and U from the factorization A = P*L*U
  as computed by ZGETRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from ZGETRF; for 1<=i<=N, row i of the
  matrix was interchanged with row IPIV(i).

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by ZGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).
```

```
 FERR
   FERR is DOUBLE PRECISION array, dimension (NRHS)
   The estimated forward error bound for each solution vector
   X(j) (the j-th column of the solution matrix X).
   If XTRUE is the true solution corresponding to X(j), FERR(j)
   is an estimated upper bound for the magnitude of the largest
   element in (X(j) - XTRUE) divided by the magnitude of the
   largest element in X(j).  The estimate is as reliable as
   the estimate for RCOND, and is almost always a slight
   overestimate of the true error.

 BERR
   BERR is DOUBLE PRECISION array, dimension (NRHS)
   The componentwise relative backward error of each solution
   vector X(j) (i.e., the smallest relative change in
   any element of A or B that makes X(j) an exact solution).

 WORK
   WORK is COMPLEX*16 array, dimension (2*N)

 RWORK
   RWORK is DOUBLE PRECISION array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

   ITMAX is the maximum number of steps of iterative refinement.
```

35.24.3 fortran code

— zgerfs.f —

```
*  =====================================================================
      SUBROUTINE ZGERFS( TRANS, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB,
     $                   X, LDX, FERR, BERR, WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
```

```
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   BERR( * ), FERR( * ), RWORK( * )
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
      COMPLEX*16         ONE
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ) )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D+0 )
      DOUBLE PRECISION   THREE
      PARAMETER          ( THREE = 3.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      CHARACTER          TRANSN, TRANST
      INTEGER            COUNT, I, J, K, KASE, NZ
      DOUBLE PRECISION   EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN, XK
      COMPLEX*16         ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZAXPY, ZCOPY, ZGEMV, ZGETRS, ZLACN2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, MAX
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
```

```
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -10
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -12
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGERFS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
      IF( NOTRAN ) THEN
         TRANSN = 'N'
         TRANST = 'C'
      ELSE
         TRANSN = 'C'
         TRANST = 'N'
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = N + 1
```

```
      EPS = DLAMCH( 'Epsilon' )
      SAFMIN = DLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 140 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - op(A) * X,
*        where op(A) = A, A**T, or A**H, depending on TRANS.
*
         CALL ZCOPY( N, B( 1, J ), 1, WORK, 1 )
         CALL ZGEMV( TRANS, N, N, -ONE, A, LDA, X( 1, J ), 1, ONE, WORK,
     $               1 )
*
*        Compute componentwise relative backward error from formula
*
*        max(i) ( abs(R(i)) / ( abs(op(A))*abs(X) + abs(B) )(i) )
*
*        where abs(Z) is the componentwise absolute value of the matrix
*        or vector Z.  If the i-th component of the denominator is less
*        than SAFE2, then SAFE1 is added to the i-th components of the
*        numerator and denominator before dividing.
*
         DO 30 I = 1, N
            RWORK( I ) = CABS1( B( I, J ) )
   30    CONTINUE
*
*        Compute abs(op(A))*abs(X) + abs(B).
*
         IF( NOTRAN ) THEN
            DO 50 K = 1, N
               XK = CABS1( X( K, J ) )
               DO 40 I = 1, N
                  RWORK( I ) = RWORK( I ) + CABS1( A( I, K ) )*XK
   40          CONTINUE
   50       CONTINUE
         ELSE
            DO 70 K = 1, N
               S = ZERO
               DO 60 I = 1, N
                  S = S + CABS1( A( I, K ) )*CABS1( X( I, J ) )
   60          CONTINUE
```

```
               RWORK( K ) = RWORK( K ) + S
   70       CONTINUE
         END IF
         S = ZERO
         DO 80 I = 1, N
            IF( RWORK( I ).GT.SAFE2 ) THEN
               S = MAX( S, CABS1( WORK( I ) ) / RWORK( I ) )
            ELSE
               S = MAX( S, ( CABS1( WORK( I ) )+SAFE1 ) /
     $             ( RWORK( I )+SAFE1 ) )
            END IF
   80    CONTINUE
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
*           1) The residual BERR(J) is larger than machine epsilon, and
*           2) BERR(J) decreased by at least a factor of 2 during the
*              last iteration, and
*           3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
*
*           Update solution and try again.
*
            CALL ZGETRS( TRANS, N, 1, AF, LDAF, IPIV, WORK, N, INFO )
            CALL ZAXPY( N, ONE, WORK, 1, X( 1, J ), 1 )
            LSTRES = BERR( J )
            COUNT = COUNT + 1
            GO TO 20
         END IF
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(op(A)))*
*           ( abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
*          inv(op(A)) is the inverse of op(A)
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(op(A))*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(op(A))*abs(X) + abs(B) is less than SAFE2.
*
```

```
*        Use ZLACN2 to estimate the infinity-norm of the matrix
*           inv(op(A)) * diag(W),
*        where W = abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) )))
*
         DO 90 I = 1, N
            IF( RWORK( I ).GT.SAFE2 ) THEN
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I )
            ELSE
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I ) +
     $                      SAFE1
            END IF
   90    CONTINUE
*
         KASE = 0
  100    CONTINUE
         CALL ZLACN2( N, WORK( N+1 ), WORK, FERR( J ), KASE, ISAVE )
         IF( KASE.NE.0 ) THEN
            IF( KASE.EQ.1 ) THEN
*
*              Multiply by diag(W)*inv(op(A)**H).
*
               CALL ZGETRS( TRANST, N, 1, AF, LDAF, IPIV, WORK, N,
     $                      INFO )
               DO 110 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
  110          CONTINUE
            ELSE
*
*              Multiply by inv(op(A))*diag(W).
*
               DO 120 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
  120          CONTINUE
               CALL ZGETRS( TRANSN, N, 1, AF, LDAF, IPIV, WORK, N,
     $                      INFO )
            END IF
            GO TO 100
         END IF
*
*        Normalize error.
*
         LSTRES = ZERO
         DO 130 I = 1, N
            LSTRES = MAX( LSTRES, CABS1( X( I, J ) ) )
  130    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  140 CONTINUE
*
```

```
      RETURN
*
*     End of ZGERFS
*
      END
```

———-

35.24.4 lisp code

— **zgerfs.lisp** —

```
(let* ((itmax 5)
       (zero 0.0d0)
       (one (f2cl-lib:cmplx 1.0d0 0.0d0))
       (two 2.0d0)
       (three 3.0d0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (f2cl-lib:complex16) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 3.0d0 3.0d0) three)
           (ignorable itmax zero one two three))
  (defun zgerfs
         (trans n nrhs a lda af ldaf ipiv b ldb$ x ldx ferr berr work rwork
          info)
    (declare (type (array double-float (*)) rwork berr ferr)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) work x b af a)
             (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (af f2cl-lib:complex16 af-%data% af-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (x f2cl-lib:complex16 x-%data% x-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (ferr double-float ferr-%data% ferr-%offset%)
         (berr double-float berr-%data% berr-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
          cabs1))
        (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
```

```
     (zdum #C(0.0d0 0.0d0)) (eps 0.0d0) (lstres 0.0d0) (s 0.0d0)
     (safe1 0.0d0) (safe2 0.0d0) (safmin 0.0d0) (xk 0.0d0) (count$ 0)
     (i 0) (j 0) (k 0) (kase 0) (nz 0)
     (transn
      (make-array '(1) :element-type 'character :initial-element
                  #\ ))
     (transt
      (make-array '(1) :element-type 'character :initial-element
                  #\ ))
     (notran nil))
(declare (type (array f2cl-lib:integer4 (3)) isave)
         (type (f2cl-lib:complex16) zdum)
         (type (double-float) eps lstres s safe1 safe2 safmin xk)
         (type (f2cl-lib:integer4) count$ i j k kase nz)
         (type (string 1) transn transt)
         (type f2cl-lib:logical notran))
(setf info 0)
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -10))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -12)))
(cond
 ((/= info 0) (xerbla "ZGERFS" (f2cl-lib:int-sub info))
  (go end_label)))
```

```
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
              zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%)
              zero)
     label10))
  (go end_label)))
(cond
 (notran (f2cl-lib:f2cl-set-string transn "N" (string 1))
  (f2cl-lib:f2cl-set-string transt "C" (string 1)))
 (t (f2cl-lib:f2cl-set-string transn "C" (string 1))
  (f2cl-lib:f2cl-set-string transt "N" (string 1))))
(setf nz (f2cl-lib:int-add n 1))
(setf eps (dlamch "Epsilon"))
(setf safmin (dlamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zcopy n
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1 work 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (zgemv trans n n (- one) a lda
         (f2cl-lib:array-slice x-%data% f2cl-lib:complex16 (1 j)
                               ((1 ldx) (1 *)) x-%offset%)
         1 one work 1)
      (declare (ignore var-3 var-4 var-6 var-7 var-9 var-10))
      (when var-0 (setf trans var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-5 (setf lda var-5))
      (when var-8 (setf one var-8)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
```

```
      (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
              (cabs1
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)))
    label30))
  (cond
   (notran
    (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                  ((> k n) nil)
      (tagbody
        (setf xk
                (cabs1
                 (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                                x-%offset%)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                 rwork-%offset%)
                  (+
                   (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                  rwork-%offset%)
                   (*
                    (cabs1
                     (f2cl-lib:fref a-%data% (i k)
                                    ((1 lda) (1 *)) a-%offset%))
                    xk)))
          label40))
      label50)))
   (t
    (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                  ((> k n) nil)
      (tagbody
        (setf s zero)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf s
                    (+ s
                       (*
                        (cabs1
                         (f2cl-lib:fref a-%data% (i k)
                                        ((1 lda) (1 *))
                                        a-%offset%))
                        (cabs1
                         (f2cl-lib:fref x-%data% (i j)
                                        ((1 ldx) (1 *))
                                        x-%offset%)))))
          label60))
        (setf (f2cl-lib:fref rwork-%data% (k) ((1 *))
```

```
                                rwork-%offset%)
                    (+
                     (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                    rwork-%offset%)
                     s))
         label70))))
    (setf s zero)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (cond
         ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
          (setf s
                  (max s
                       (/
                        (cabs1
                         (f2cl-lib:fref work-%data% (i) ((1 *))
                                        work-%offset%))
                        (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                       rwork-%offset%)))))
         (t
          (setf s
                  (max s
                       (/
                        (+
                         (cabs1
                          (f2cl-lib:fref work-%data% (i) ((1 *))
                                         work-%offset%))
                         safe1)
                        (+
                         (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                        rwork-%offset%)
                         safe1))))))
       label80))
    (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
    (cond
     ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
           (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
           (<= count$ itmax))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (zgetrs trans n 1 af ldaf ipiv work n info)
        (declare (ignore var-2 var-3 var-5 var-6))
        (when var-0 (setf trans var-0))
        (when var-1 (setf n var-1))
        (when var-4 (setf ldaf var-4))
        (when var-7 (setf n var-7))
        (when var-8 (setf info var-8)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (zaxpy n one work 1
```

```
        (f2cl-lib:array-slice x-%data% f2cl-lib:complex16 (1 j)
                              ((1 ldx) (1 *)) x-%offset%)
        1)
      (declare (ignore var-2 var-3 var-4 var-5))
      (when var-0 (setf n var-0))
      (when var-1 (setf one var-1)))
    (setf lstres
            (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
    (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                             rwork-%offset%)
                (+
                 (cabs1
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%))
                 (* nz eps
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%)))))
       (t
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                             rwork-%offset%)
                (+
                 (cabs1
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%))
                 (* nz eps
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%))
                 safe1))))
     label90))
  (setf kase 0)
 label100
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (zlacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             ((+ n 1)) ((1 *)) work-%offset%)
       work
       (f2cl-lib:array-slice ferr-%data% double-float (j) ((1 *))
                             ferr-%offset%)
       kase isave)
    (declare (ignore var-1 var-2 var-3 var-5))
    (when var-0 (setf n var-0))
    (when var-4 (setf kase var-4)))
  (cond
   ((/= kase 0)
```

```
    (cond
     ((= kase 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (zgetrs transt n 1 af ldaf ipiv work n info)
        (declare (ignore var-2 var-3 var-5 var-6))
        (when var-0 (setf transt var-0))
        (when var-1 (setf n var-1))
        (when var-4 (setf ldaf var-4))
        (when var-7 (setf n var-7))
        (when var-8 (setf info var-8)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                  (*
                   (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                  rwork-%offset%)
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)))
          label110)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                  (*
                   (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                  rwork-%offset%)
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)))
          label120))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (zgetrs transn n 1 af ldaf ipiv work n info)
        (declare (ignore var-2 var-3 var-5 var-6))
        (when var-0 (setf transn var-0))
        (when var-1 (setf n var-1))
        (when var-4 (setf ldaf var-4))
        (when var-7 (setf n var-7))
        (when var-8 (setf info var-8)))))
    (go label100)))
  (setf lstres zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf lstres
              (max lstres
```

```
                              (cabs1
                               (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                              x-%offset%))))
                 label130))
               (if (/= lstres zero)
                   (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                         (/
                          (f2cl-lib:fref ferr-%data% (j) ((1 *))
                                         ferr-%offset%)
                          lstres)))
             label140))
           (go end_label)
          end_label
           (return
            (values trans n nil nil lda nil ldaf nil nil nil nil nil nil nil nil
                    nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgerfs
                 fortran-to-lisp::*f2cl-function-info*)
           (fortran-to-lisp::make-f2cl-finfo :arg-types
                                             '((string)
                                               (fortran-to-lisp::integer4)
                                               (fortran-to-lisp::integer4)
                                               (array fortran-to-lisp::complex16
                                                (*))
                                               (fortran-to-lisp::integer4)
                                               (array fortran-to-lisp::complex16
                                                (*))
                                               (fortran-to-lisp::integer4)
                                               (array fortran-to-lisp::integer4
                                                (*))
                                               (array fortran-to-lisp::complex16
                                                (*))
                                               (fortran-to-lisp::integer4)
                                               (array fortran-to-lisp::complex16
                                                (*))
                                               (fortran-to-lisp::integer4)
                                               (array double-float (*))
                                               (array double-float (*))
                                               (array fortran-to-lisp::complex16
                                                (*))
                                               (array double-float (*))
                                               (fortran-to-lisp::integer4))
                                             :return-values
                                             '(fortran-to-lisp::trans
                                               fortran-to-lisp::n nil nil
                                               fortran-to-lisp::lda nil
```

```
                                        fortran-to-lisp::ldaf nil nil nil
                                        nil nil nil nil nil nil
                                        fortran-to-lisp::info)
                                      :calls '(fortran-to-lisp::xerbla))))
```

———-

35.25 zgerfsx

35.25.1 Axiom unit tests

— **zgerfsx.input** —

```
)set break resume
)sys rm -f zgerfsx.output
)spool zgerfsx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.25.2 Axiom help page [Demm05]

— **zgerfsx.help** —

```
====================================================================
zgerfsx examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgerfsx (TRANS, EQUED, N, NRHS, A, LDA, AF, LDAF, IPIV, R,
           C, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS, ERR_BNDS_NORM,
           ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, RWORK, INFO)
```

```
          ZGERFSX

Function/Subroutine Documentation
   subroutine zgerfsx (character TRANS, character EQUED, integer N, integer
       NRHS, complex*16, dimension( lda, * ) A, integer LDA, complex*16,
       dimension( ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV,
       double precision, dimension( * ) R, double precision, dimension( * ) C,
       complex*16, dimension( ldb, * ) B, integer LDB, complex*16, dimension(
       ldx , * ) X, integer LDX, double precision RCOND, double precision,
       dimension( * ) BERR, integer N_ERR_BNDS, double precision, dimension(
       nrhs, * ) ERR_BNDS_NORM, double precision, dimension( nrhs, * )
       ERR_BNDS_COMP, integer NPARAMS, double precision, dimension( * )
       PARAMS, complex*16, dimension( * ) WORK, double precision, dimension( *
       ) RWORK, integer INFO)
       ZGERFSX

Purpose:
   ZGERFSX improves the computed solution to a system of linear
   equations and provides error bounds and backward error estimates
   for the solution.  In addition to normwise error bound, the code
   provides maximum componentwise error bound if possible.  See
   comments for ERR_BNDS_NORM and ERR_BNDS_COMP for details of the
   error bounds.

   The original system of linear equations may have been equilibrated
   before calling this routine, as described by arguments EQUED, R
   and C below. In this case, the solution and error bounds returned
   are for the original unequilibrated system.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose = Transpose)

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done to A
   before calling this routine. This is needed to compute
   the solution and error bounds correctly.
   = 'N':  No equilibration
```

```
  = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
  = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
  = 'B':  Both row and column equilibration, i.e., A has been
          replaced by diag(R) * A * diag(C).
  The right hand side B has been changed accordingly.

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  The original N-by-N matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX*16 array, dimension (LDAF,N)
  The factors L and U from the factorization A = P*L*U
  as computed by ZGETRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from ZGETRF; for 1<=i<=N, row i of the
  matrix was interchanged with row IPIV(i).

R
  R is DOUBLE PRECISION array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.
  If R is accessed, each element of R should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be
  reliable.
```

```
C
  C is DOUBLE PRECISION array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.
  If C is accessed, each element of C should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by ZGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.
```

```
ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
          max_j (abs(XTRUE(j,i) - X(j,i)))
           ------------------------------
                 max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * dlamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * dlamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.

ERR_BNDS_COMP
  ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                 abs(XTRUE(j,i) - X(j,i))
            max_j ----------------------
                        abs(X(j,i))
```

```
  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * dlamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * dlamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is DOUBLE PRECISION array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0D+0
```

```
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
          (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be
              trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is COMPLEX*16 array, dimension (2*N)

RWORK
  RWORK is DOUBLE PRECISION array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is guaranteed.
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
      has been completed, but the factor U is exactly singular, so
      the solution and error bounds could not be computed. RCOND = 0
      is returned.
  = N+J: The solution corresponding to the Jth right-hand side is
         not guaranteed. The solutions corresponding to other right-
         hand sides K with K > J may not be guaranteed as well, but
         only the first such right-hand side is reported. If a small
         componentwise error is not requested (PARAMS(3) = 0.0) then
         the Jth right-hand side is the first with a normwise error
         bound that is not guaranteed (the smallest J such
         that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
         the Jth right-hand side is the first with either a normwise or
         componentwise error bound that is not guaranteed (the smallest
         J such that either ERR_BNDS_NORM(J,1) = 0.0 or
         ERR_BNDS_COMP(J,1) = 0.0). See the definition of
         ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
         about all of the right-hand sides check ERR_BNDS_NORM or
```

ERR_BNDS_COMP.

———-

35.25.3 fortran code

— **zgerfsx.f** —

```
*  ==================================================================
      SUBROUTINE ZGERFSX( TRANS, EQUED, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $                    R, C, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS,
     $                    ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $                    WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS, EQUED
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX , * ), WORK( * )
      DOUBLE PRECISION   R( * ), C( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * ), RWORK( * )
*     ..
*
*  ================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   ITREF_DEFAULT, ITHRESH_DEFAULT
      DOUBLE PRECISION   COMPONENTWISE_DEFAULT, RTHRESH_DEFAULT
      DOUBLE PRECISION   DZTHRESH_DEFAULT
      PARAMETER          ( ITREF_DEFAULT = 1.0D+0 )
      PARAMETER          ( ITHRESH_DEFAULT = 10.0D+0 )
      PARAMETER          ( COMPONENTWISE_DEFAULT = 1.0D+0 )
      PARAMETER          ( RTHRESH_DEFAULT = 0.5D+0 )
      PARAMETER          ( DZTHRESH_DEFAULT = 0.25D+0 )
```

```
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. Local Scalars ..
      CHARACTER(1)       NORM
      LOGICAL            ROWEQU, COLEQU, NOTRAN
      INTEGER            J, TRANS_TYPE, PREC_TYPE, REF_TYPE
      INTEGER            N_NORMS
      DOUBLE PRECISION   ANORM, RCOND_TMP
      DOUBLE PRECISION   ILLRCOND_THRESH, ERR_LBND, CWISE_WRONG
      LOGICAL            IGNORE_CWISE
      INTEGER            ITHRESH
      DOUBLE PRECISION   RTHRESH, UNSTABLE_THRESH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGECON, ZLA_GERFSX_EXTENDED
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT, TRANSFER
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, BLAS_FPINFO_X, ILATRANS, ILAPREC
      EXTERNAL           DLAMCH, ZLANGE, ZLA_GERCOND_X, ZLA_GERCOND_C
      DOUBLE PRECISION   DLAMCH, ZLANGE, ZLA_GERCOND_X, ZLA_GERCOND_C
      LOGICAL            LSAME
      INTEGER            BLAS_FPINFO_X
      INTEGER            ILATRANS, ILAPREC
*     ..
*     .. Executable Statements ..
*
*     Check the input parameters.
*
      INFO = 0
      TRANS_TYPE = ILATRANS( TRANS )
      REF_TYPE = INT( ITREF_DEFAULT )
      IF ( NPARAMS .GE. LA_LINRX_ITREF_I ) THEN
         IF ( PARAMS( LA_LINRX_ITREF_I ) .LT. 0.0D+0 ) THEN
            PARAMS( LA_LINRX_ITREF_I ) = ITREF_DEFAULT
         ELSE
            REF_TYPE = PARAMS( LA_LINRX_ITREF_I )
         END IF
      END IF
*
```

```
*     Set default parameters.
*
      ILLRCOND_THRESH = DBLE( N ) * DLAMCH( 'Epsilon' )
      ITHRESH = INT( ITHRESH_DEFAULT )
      RTHRESH = RTHRESH_DEFAULT
      UNSTABLE_THRESH = DZTHRESH_DEFAULT
      IGNORE_CWISE = COMPONENTWISE_DEFAULT .EQ. 0.0D+0
*
      IF ( NPARAMS.GE.LA_LINRX_ITHRESH_I ) THEN
         IF ( PARAMS( LA_LINRX_ITHRESH_I ).LT.0.0D+0 ) THEN
            PARAMS(LA_LINRX_ITHRESH_I) = ITHRESH
         ELSE
            ITHRESH = INT( PARAMS( LA_LINRX_ITHRESH_I ) )
         END IF
      END IF
      IF ( NPARAMS.GE.LA_LINRX_CWISE_I ) THEN
         IF ( PARAMS( LA_LINRX_CWISE_I ).LT.0.0D+0 ) THEN
            IF ( IGNORE_CWISE ) THEN
               PARAMS( LA_LINRX_CWISE_I ) = 0.0D+0
            ELSE
               PARAMS( LA_LINRX_CWISE_I ) = 1.0D+0
            END IF
         ELSE
            IGNORE_CWISE = PARAMS( LA_LINRX_CWISE_I ) .EQ. 0.0D+0
         END IF
      END IF
      IF ( REF_TYPE .EQ. 0 .OR. N_ERR_BNDS .EQ. 0 ) THEN
         N_NORMS = 0
      ELSE IF ( IGNORE_CWISE ) THEN
         N_NORMS = 1
      ELSE
         N_NORMS = 2
      END IF
*
      NOTRAN = LSAME( TRANS, 'N' )
      ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
      COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
*
*     Test input parameters.
*
      IF( TRANS_TYPE.EQ.-1 ) THEN
        INFO = -1
      ELSE IF( .NOT.ROWEQU .AND. .NOT.COLEQU .AND.
     $         .NOT.LSAME( EQUED, 'N' ) ) THEN
        INFO = -2
      ELSE IF( N.LT.0 ) THEN
        INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
        INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
```

```
        INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
        INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
        INFO = -13
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
        INFO = -15
      END IF
      IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'ZGERFSX', -INFO )
        RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         RCOND = 1.0D+0
         DO J = 1, NRHS
            BERR( J ) = 0.0D+0
            IF ( N_ERR_BNDS .GE. 1 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) =  1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
            IF ( N_ERR_BNDS .GE. 2 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 0.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 0.0D+0
            END IF
            IF ( N_ERR_BNDS .GE. 3 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 1.0D+0
            END IF
         END DO
         RETURN
      END IF
*
*     Default to failure.
*
      RCOND = 0.0D+0
      DO J = 1, NRHS
         BERR( J ) = 1.0D+0
         IF ( N_ERR_BNDS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
         END IF
         IF ( N_ERR_BNDS .GE. 2 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
         END IF
         IF ( N_ERR_BNDS .GE. 3 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 0.0D+0
```

```
            ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 0.0D+0
         END IF
      END DO
*
*     Compute the norm of A and the reciprocal of the condition
*     number of A.
*
      IF( NOTRAN ) THEN
         NORM = 'I'
      ELSE
         NORM = '1'
      END IF
      ANORM = ZLANGE( NORM, N, N, A, LDA, RWORK )
      CALL ZGECON( NORM, N, AF, LDAF, ANORM, RCOND, WORK, RWORK, INFO )
*
*     Perform refinement on each right-hand side
*
      IF ( REF_TYPE .NE. 0 ) THEN

         PREC_TYPE = ILAPREC( 'E' )

         IF ( NOTRAN ) THEN
            CALL ZLA_GERFSX_EXTENDED( PREC_TYPE, TRANS_TYPE,  N,
     $           NRHS, A, LDA, AF, LDAF, IPIV, COLEQU, C, B,
     $           LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM,
     $           ERR_BNDS_COMP, WORK, RWORK, WORK(N+1),
     $           TRANSFER (RWORK(1:2*N), (/ (ZERO, ZERO) /), N),
     $           RCOND, ITHRESH, RTHRESH, UNSTABLE_THRESH, IGNORE_CWISE,
     $           INFO )
         ELSE
            CALL ZLA_GERFSX_EXTENDED( PREC_TYPE, TRANS_TYPE,  N,
     $           NRHS, A, LDA, AF, LDAF, IPIV, ROWEQU, R, B,
     $           LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM,
     $           ERR_BNDS_COMP, WORK, RWORK, WORK(N+1),
     $           TRANSFER (RWORK(1:2*N), (/ (ZERO, ZERO) /), N),
     $           RCOND, ITHRESH, RTHRESH, UNSTABLE_THRESH, IGNORE_CWISE,
     $           INFO )
         END IF
      END IF

      ERR_LBND = MAX( 10.0D+0, SQRT( DBLE( N ) ) ) * DLAMCH( 'Epsilon' )
      IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 1 ) THEN
*
*     Compute scaled normwise condition number cond(A*C).
*
         IF ( COLEQU .AND. NOTRAN ) THEN
            RCOND_TMP = ZLA_GERCOND_C( TRANS, N, A, LDA, AF, LDAF, IPIV,
     $           C, .TRUE., INFO, WORK, RWORK )
         ELSE IF ( ROWEQU .AND. .NOT. NOTRAN ) THEN
            RCOND_TMP = ZLA_GERCOND_C( TRANS, N, A, LDA, AF, LDAF, IPIV,
```

```
     $           R, .TRUE., INFO, WORK, RWORK )
         ELSE
            RCOND_TMP = ZLA_GERCOND_C( TRANS, N, A, LDA, AF, LDAF, IPIV,
     $           C, .FALSE., INFO, WORK, RWORK )
         END IF
         DO J = 1, NRHS
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .GT. 1.0D+0 )
     $           ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 0.0D+0
               IF ( INFO .LE. N ) INFO = N + J
            ELSE IF (ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .LT. ERR_LBND)
     $              THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF
         END DO
      END IF

      IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 2 ) THEN
*
*     Compute componentwise condition number cond(A*diag(Y(:,J))) for
*     each right-hand side using the current solution as an estimate of
*     the true solution.  If the componentwise error estimate is too
*     large, then the solution is a lousy estimate of truth and the
*     estimated RCOND may be too optimistic.  To avoid misleading users,
*     the inverse condition number is set to 0.0 when the estimated
*     cwise error is at least CWISE_WRONG.
*
         CWISE_WRONG = SQRT( DLAMCH( 'Epsilon' ) )
         DO J = 1, NRHS
            IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .LT. CWISE_WRONG )
     $     THEN
               RCOND_TMP = ZLA_GERCOND_X( TRANS, N, A, LDA, AF, LDAF,
     $              IPIV, X(1,J), INFO, WORK, RWORK )
            ELSE
```

```
               RCOND_TMP = 0.0D+0
            END IF
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .GT. 1.0D+0 )
     $           ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 0.0D+0
               IF ( PARAMS( LA_LINRX_CWISE_I ) .EQ. 1.0D+0
     $              .AND. INFO.LT.N + J ) INFO = N + J
            ELSE IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I )
     $              .LT. ERR_LBND ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF

         END DO
      END IF
*
      RETURN
*
*     End of ZGERFSX
*
      END
```

———-

35.25.4 lisp code

— zgerfsx.lisp —

———-

35.26 zgerq2

35.26.1 Axiom unit tests

— zgerq2.input —

```
)set break resume
)sys rm -f zgerq2.output
)spool zgerq2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.26.2 Axiom help page

— zgerq2.help —

```
====================================================================
zgerq2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgerq2 (M, N, A, LDA, TAU, WORK, INFO)
           ZGERQ2 computes the RQ factorization of a general rectangular
           matrix using an unblocked algorithm.

Function/Subroutine Documentation
   subroutine zgerq2 (integer M, integer N, complex*16, dimension( lda, * ) A,
       integer LDA, complex*16, dimension( * ) TAU, complex*16, dimension( * )
       WORK, integer INFO)
       ZGERQ2 computes the RQ factorization of a general rectangular matrix
       using an unblocked algorithm.

Purpose:
   ZGERQ2 computes an RQ factorization of a complex m by n matrix A:
   A = R * Q.
```

```
 Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the m by n matrix A.
   On exit, if m <= n, the upper triangle of the subarray
   A(1:m,n-m+1:n) contains the m by m upper triangular matrix R;
   if m >= n, the elements on and above the (m-n)-th subdiagonal
   contain the m by n upper trapezoidal matrix R; the remaining
   elements, with the array TAU, represent the unitary matrix
   Q as a product of elementary reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is COMPLEX*16 array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is COMPLEX*16 array, dimension (M)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value

Further Details:

   The matrix Q is represented as a product of elementary reflectors

   Q = H(1)**H H(2)**H . . . H(k)**H, where k = min(m,n).

   Each H(i) has the form

   H(i) = I - tau * v * v**H

   where tau is a complex scalar, and v is a complex vector with
   v(n-k+i+1:n) = 0 and v(n-k+i) = 1; conjg(v(1:n-k+i-1)) is stored on
   exit in A(m-k+i,1:n-k+i-1), and tau in TAU(i).
```

———-

35.26.3 fortran code

— **zgerq2.f** —

```
*  =====================================================================
      SUBROUTINE ZGERQ2( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      COMPLEX*16         ALPHA
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLACGV, ZLARF, ZLARFG
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
```

```
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGERQ2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = K, 1, -1
*
*        Generate elementary reflector H(i) to annihilate
*        A(m-k+i,1:n-k+i-1)
*
         CALL ZLACGV( N-K+I, A( M-K+I, 1 ), LDA )
         ALPHA = A( M-K+I, N-K+I )
         CALL ZLARFG( N-K+I, ALPHA, A( M-K+I, 1 ), LDA, TAU( I ) )
*
*        Apply H(i) to A(1:m-k+i-1,1:n-k+i) from the right
*
         A( M-K+I, N-K+I ) = ONE
         CALL ZLARF( 'Right', M-K+I-1, N-K+I, A( M-K+I, 1 ), LDA,
     $               TAU( I ), A, LDA, WORK )
         A( M-K+I, N-K+I ) = ALPHA
         CALL ZLACGV( N-K+I-1, A( M-K+I, 1 ), LDA )
   10 CONTINUE
      RETURN
*
*     End of ZGERQ2
*
      END
```

———-

35.26.4 lisp code

— **zgerq2.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (ignorable one))
  (defun zgerq2 (m n a lda tau work info)
    (declare (type (array f2cl-lib:complex16 (*)) work tau a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
```

```
    (work f2cl-lib:complex16 work-%data% work-%offset%))
  (prog ((alpha #C(0.0d0 0.0d0)) (i 0) (k 0))
    (declare (type (f2cl-lib:complex16) alpha)
             (type (f2cl-lib:integer4) i k))
    (setf info 0)
    (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
          ((< lda
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
           (setf info -4)))
    (cond
     ((/= info 0) (xerbla "ZGERQ2" (f2cl-lib:int-sub info))
      (go end_label)))
    (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
    (f2cl-lib:fdo (i k (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2)
            (zlacgv (f2cl-lib:int-add (f2cl-lib:int-sub n k) i)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   ((+ m (f2cl-lib:int-sub k) i) 1)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-1))
          (when var-2 (setf lda var-2)))
        (setf alpha
                (f2cl-lib:fref a-%data%
                               ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                (f2cl-lib:int-add (f2cl-lib:int-sub n k)
                                                  i))
                               ((1 lda) (1 *)) a-%offset%))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (zlarfg (f2cl-lib:int-add (f2cl-lib:int-sub n k) i) alpha
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   ((+ m (f2cl-lib:int-sub k) i) 1)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                                   ((1 *)) tau-%offset%))
          (declare (ignore var-0 var-2 var-4))
          (when var-1 (setf alpha var-1))
          (when var-3 (setf lda var-3)))
        (setf (f2cl-lib:fref a-%data%
                             ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                              (f2cl-lib:int-add (f2cl-lib:int-sub n k) i))
                             ((1 lda) (1 *)) a-%offset%)
              one)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (zlarf "Right"
             (f2cl-lib:int-sub (f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
```

```
                                  1)
                 (f2cl-lib:int-add (f2cl-lib:int-sub n k) i)
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                       ((+ m (f2cl-lib:int-sub k) i) 1)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                                       ((1 *)) tau-%offset%)
                 a lda work)
               (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8))
               (when var-4 (setf lda var-4))
               (when var-7 (setf lda var-7)))
             (setf (f2cl-lib:fref a-%data%
                                  ((f2cl-lib:int-add (f2cl-lib:int-sub m k) i)
                                   (f2cl-lib:int-add (f2cl-lib:int-sub n k) i))
                                  ((1 lda) (1 *)) a-%offset%)
                     alpha)
             (multiple-value-bind (var-0 var-1 var-2)
                 (zlacgv
                 (f2cl-lib:int-sub (f2cl-lib:int-add (f2cl-lib:int-sub n k) i)
                                   1)
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                       ((+ m (f2cl-lib:int-sub k) i) 1)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
               (declare (ignore var-0 var-1))
               (when var-2 (setf lda var-2)))
           label10))
        (go end_label)
       end_label
        (return (values nil nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgerq2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::lda
```

```
                                        nil nil fortran-to-lisp::info)
                                      :calls '(fortran-to-lisp::xerbla))))
```

———-

35.27 zgerqf

35.27.1 Axiom unit tests

— **zgerqf.input** —

```
)set break resume
)sys rm -f zgerqf.output
)spool zgerqf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.27.2 Axiom help page

— **zgerqf.help** —

```
======================================================================
zgerqf examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgerqf (M, N, A, LDA, TAU, WORK, LWORK, INFO)
           ZGERQF

Function/Subroutine Documentation
   subroutine zgerqf (integer M, integer N, complex*16, dimension( lda, * ) A,
```

```
      integer LDA, complex*16, dimension( * ) TAU, complex*16, dimension( * )
      WORK, integer LWORK, integer INFO)
      ZGERQF

Purpose:
   ZGERQF computes an RQ factorization of a complex M-by-N matrix A:
   A = R * Q.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the M-by-N matrix A.
   On exit,
   if m <= n, the upper triangle of the subarray
   A(1:m,n-m+1:n) contains the M-by-M upper triangular matrix R;
   if m >= n, the elements on and above the (m-n)-th subdiagonal
   contain the M-by-N upper trapezoidal matrix R;
   the remaining elements, with the array TAU, represent the
   unitary matrix Q as a product of min(m,n) elementary
   reflectors (see Further Details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 TAU
   TAU is COMPLEX*16 array, dimension (min(M,N))
   The scalar factors of the elementary reflectors (see Further Details).

 WORK
   WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.  LWORK >= max(1,M).
   For optimum performance LWORK >= M*NB, where NB is
   the optimal blocksize.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
```

```
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details:

   The matrix Q is represented as a product of elementary reflectors

   Q = H(1)**H H(2)**H . . . H(k)**H, where k = min(m,n).

   Each H(i) has the form

   H(i) = I - tau * v * v**H

   where tau is a complex scalar, and v is a complex vector with
   v(n-k+i+1:n) = 0 and v(n-k+i) = 1; conjg(v(1:n-k+i-1)) is stored on
   exit in A(m-k+i,1:n-k+i-1), and tau in TAU(i).
```

———-

35.27.3 fortran code

— **zgerqf.f** —

```
*  =====================================================================
      SUBROUTINE ZGERQF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
```

```
      INTEGER            I, IB, IINFO, IWS, K, KI, KK, LDWORK, LWKOPT,
     $                   MU, NB, NBMIN, NU, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGERQ2, ZLARFB, ZLARFT
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
*
      IF( INFO.EQ.0 ) THEN
         K = MIN( M, N )
         IF( K.EQ.0 ) THEN
            LWKOPT = 1
         ELSE
            NB = ILAENV( 1, 'ZGERQF', ' ', M, N, -1, -1 )
            LWKOPT = M*NB
         END IF
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.MAX( 1, M ) .AND. .NOT.LQUERY ) THEN
            INFO = -7
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGERQF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
```

```
*
      IF( K.EQ.0 ) THEN
         RETURN
      END IF
*
      NBMIN = 2
      NX = 1
      IWS = M
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'ZGERQF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = M
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'ZGERQF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially.
*        The last kk rows are handled by the block method.
*
         KI = ( ( K-NX-1 ) / NB )*NB
         KK = MIN( K, KI+NB )
*
         DO 10 I = K - KK + KI + 1, K - KK + 1, -NB
            IB = MIN( K-I+1, NB )
*
*           Compute the RQ factorization of the current block
*           A(m-k+i:m-k+i+ib-1,1:n-k+i+ib-1)
*
            CALL ZGERQ2( IB, N-K+I+IB-1, A( M-K+I, 1 ), LDA, TAU( I ),
     $                   WORK, IINFO )
            IF( M-K+I.GT.1 ) THEN
*
*              Form the triangular factor of the block reflector
```

```
*              H = H(i+ib-1) . . . H(i+1) H(i)
*
               CALL ZLARFT( 'Backward', 'Rowwise', N-K+I+IB-1, IB,
     $                      A( M-K+I, 1 ), LDA, TAU( I ), WORK, LDWORK )
*
*              Apply H to A(1:m-k+i-1,1:n-k+i+ib-1) from the right
*
               CALL ZLARFB( 'Right', 'No transpose', 'Backward',
     $                      'Rowwise', M-K+I-1, N-K+I+IB-1, IB,
     $                      A( M-K+I, 1 ), LDA, WORK, LDWORK, A, LDA,
     $                      WORK( IB+1 ), LDWORK )
            END IF
   10    CONTINUE
         MU = M - K + I + NB - 1
         NU = N - K + I + NB - 1
      ELSE
         MU = M
         NU = N
      END IF
*
*     Use unblocked code to factor the last or only block
*
      IF( MU.GT.0 .AND. NU.GT.0 )
     $   CALL ZGERQ2( MU, NU, A, LDA, TAU, WORK, IINFO )
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of ZGERQF
*
      END
```

———-

35.27.4 lisp code

— **zgerqf.lisp** —

```
(defun zgerqf (m n a lda tau work lwork info)
  (declare (type (array f2cl-lib:complex16 (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex16 a-%data% a-%offset%)
       (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ki 0) (kk 0) (ldwork 0)
           (lwkopt 0) (mu 0) (nb 0) (nbmin 0) (nu 0) (nx 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
```

```
          (type (f2cl-lib:integer4) nx nu nbmin nb mu lwkopt ldwork kk ki
           k iws iinfo ib i))
(setf info 0)
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
      ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -4)))
(cond
 ((= info 0)
  (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
  (cond ((= k 0) (setf lwkopt 1))
        (t
         (setf nb
                 (multiple-value-bind
                     (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                     (ilaenv 1 "ZGERQF" " " m n -1 -1)
                   (declare (ignore var-0 var-1 var-2 var-5 var-6))
                   (setf m var-3)
                   (setf n var-4)
                   ret-val))
         (setf lwkopt (f2cl-lib:int-mul m nb))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwkopt 'f2cl-lib:complex16))
  (cond
   ((and
     (< lwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
     (not lquery))
    (setf info -7)))))
(cond
 ((/= info 0) (xerbla "ZGERQF" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond ((= k 0) (go end_label)))
(setf nbmin 2)
(setf nx 1)
(setf iws m)
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "ZGERQF" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (setf m var-3)
                      (setf n var-4)
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork m) (setf iws (f2cl-lib:int-mul ldwork nb))
```

```
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
                            (ilaenv 2 "ZGERQF" " " m n -1 -1)
                          (declare
                           (ignore var-0 var-1 var-2 var-5 var-6))
                          (setf m var-3)
                          (setf n var-4)
                          ret-val)))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (setf ki (* (the f2cl-lib:integer4 (truncate (- k nx 1) nb)) nb))
  (setf kk
          (min (the f2cl-lib:integer4 k)
               (the f2cl-lib:integer4 (f2cl-lib:int-add ki nb))))
  (f2cl-lib:fdo (i (f2cl-lib:int-add k (f2cl-lib:int-sub kk) ki 1)
                 (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
                ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub kk) 1)) nil)
    (tagbody
      (setf ib
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
               (the f2cl-lib:integer4 nb)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zgerq2 ib
           (f2cl-lib:int-sub
            (f2cl-lib:int-add (f2cl-lib:int-sub n k) i ib) 1)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                 ((+ m (f2cl-lib:int-sub k) i) 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                                 ((1 *)) tau-%offset%)
           work iinfo)
        (declare (ignore var-0 var-1 var-2 var-4 var-5))
        (setf lda var-3)
        (setf iinfo var-6))
      (cond
       ((> (f2cl-lib:int-add m (f2cl-lib:int-sub k) i) 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (zlarft "Backward" "Rowwise"
             (f2cl-lib:int-sub
```

```
                 (f2cl-lib:int-add (f2cl-lib:int-sub n k) i ib) 1)
                ib
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                      ((+ m (f2cl-lib:int-sub k) i) 1)
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                                      ((1 *)) tau-%offset%)
                work ldwork)
              (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
              (when var-3 (setf ib var-3))
              (when var-5 (setf lda var-5))
              (when var-8 (setf ldwork var-8)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                 var-10 var-11 var-12 var-13 var-14)
                (zlarfb "Right" "No transpose" "Backward" "Rowwise"
                 (f2cl-lib:int-sub
                  (f2cl-lib:int-add (f2cl-lib:int-sub m k) i) 1)
                 (f2cl-lib:int-sub
                  (f2cl-lib:int-add (f2cl-lib:int-sub n k) i ib) 1)
                 ib
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                       ((+ m (f2cl-lib:int-sub k) i) 1)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda work ldwork a lda
                 (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                       ((+ ib 1)) ((1 *)) work-%offset%)
                 ldwork)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
                var-13))
              (when var-6 (setf ib var-6))
              (when var-8 (setf lda var-8))
              (when var-10 (setf ldwork var-10))
              (when var-12 (setf lda var-12))
              (when var-14 (setf ldwork var-14)))))
        label10))
      (setf mu
              (f2cl-lib:int-sub
               (f2cl-lib:int-add (f2cl-lib:int-sub m k) i nb) 1))
      (setf nu
              (f2cl-lib:int-sub
               (f2cl-lib:int-add (f2cl-lib:int-sub n k) i nb) 1)))
     (t (setf mu m) (setf nu n)))
    (if (and (> mu 0) (> nu 0))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zgerq2 mu nu a lda tau work iinfo)
          (declare (ignore var-0 var-1 var-2 var-4 var-5))
          (setf lda var-3)
```

```
            (setf iinfo var-6)))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce iws 'f2cl-lib:complex16))
      (go end_label)
     end_label
      (return (values m n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgerqf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::zgerq2
                                              fortran-to-lisp::xerbla
                                              fortran-to-lisp::ilaenv))))
```

———-

35.28 zgetf2

35.28.1 Axiom unit tests

— **zgetf2.input** —

```
)set break resume
)sys rm -f zgetf2.output
)spool zgetf2.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.28.2 Axiom help page

— **zgetf2.help** —

```
==================================================================
zgetf2 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgetf2 (M, N, A, LDA, IPIV, INFO)
           ZGETF2 computes the LU factorization of a general m-by-n matrix
           using partial pivoting with row interchanges (unblocked algorithm).

Function/Subroutine Documentation
   subroutine zgetf2 (integer M, integer N, complex*16, dimension( lda, * ) A,
       integer LDA, integer, dimension( * ) IPIV, integer INFO)
       ZGETF2 computes the LU factorization of a general m-by-n matrix using
       partial pivoting with row interchanges (unblocked algorithm).

Purpose:
   ZGETF2 computes an LU factorization of a general m-by-n matrix A
   using partial pivoting with row interchanges.

   The factorization has the form A = P * L * U
   where P is a permutation matrix, L is lower triangular with unit
   diagonal elements (lower trapezoidal if m > n), and U is upper
   triangular (upper trapezoidal if m < n).

   This is the right-looking Level 2 BLAS version of the algorithm.

Parameters:

 M
   M is INTEGER
```

```
  The number of rows of the matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the m by n matrix to be factored.
  On exit, the factors L and U from the factorization
  A = P*L*U; the unit diagonal elements of L are not stored.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

IPIV
  IPIV is INTEGER array, dimension (min(M,N))
  The pivot indices; for 1 <= i <= min(M,N), row i of the
  matrix was interchanged with row IPIV(i).

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -k, the k-th argument had an illegal value
  > 0: if INFO = k, U(k,k) is exactly zero. The factorization
       has been completed, but the factor U is exactly
       singular, and division by zero will occur if it is used
       to solve a system of equations.
```

———-

35.28.3 fortran code

— **zgetf2.f** —

```
*  =====================================================================
      SUBROUTINE ZGETF2( M, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
```

```
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE, ZERO
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ),
     $                   ZERO = ( 0.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION   SFMIN
      INTEGER            I, J, JP
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      INTEGER            IZAMAX
      EXTERNAL           DLAMCH, IZAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGERU, ZSCAL, ZSWAP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGETF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Compute machine safe minimum
```

```
*
      SFMIN = DLAMCH('S')
*
      DO 10 J = 1, MIN( M, N )
*
*        Find pivot and test for singularity.
*
         JP = J - 1 + IZAMAX( M-J+1, A( J, J ), 1 )
         IPIV( J ) = JP
         IF( A( JP, J ).NE.ZERO ) THEN
*
*           Apply the interchange to columns 1:N.
*
            IF( JP.NE.J )
     $         CALL ZSWAP( N, A( J, 1 ), LDA, A( JP, 1 ), LDA )
*
*           Compute elements J+1:M of J-th column.
*
            IF( J.LT.M ) THEN
               IF( ABS(A( J, J )) .GE. SFMIN ) THEN
                  CALL ZSCAL( M-J, ONE / A( J, J ), A( J+1, J ), 1 )
               ELSE
                  DO 20 I = 1, M-J
                     A( J+I, J ) = A( J+I, J ) / A( J, J )
   20             CONTINUE
               END IF
            END IF
*
         ELSE IF( INFO.EQ.0 ) THEN
*
            INFO = J
         END IF
*
         IF( J.LT.MIN( M, N ) ) THEN
*
*           Update trailing submatrix.
*
            CALL ZGERU( M-J, N-J, -ONE, A( J+1, J ), 1, A( J, J+1 ),
     $                  LDA, A( J+1, J+1 ), LDA )
         END IF
   10 CONTINUE
      RETURN
*
*     End of ZGETF2
*
      END
```

———-

35.28.4 lisp code

— zgetf2.lisp —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)) (zero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (type (f2cl-lib:complex16) zero)
           (ignorable one zero))
  (defun zgetf2 (m n a lda ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((i 0) (j 0) (jp 0) (sfmin 0.0d0))
        (declare (type (f2cl-lib:integer4) i j jp)
                 (type (double-float) sfmin))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "ZGETF2" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (or (= m 0) (= n 0))
            (go end_label))
        (setf sfmin (dlamch "S"))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j
                          (min (the f2cl-lib:integer4 m)
                               (the f2cl-lib:integer4 n)))
                       nil)
          (tagbody
            (setf jp
                    (f2cl-lib:int-add (f2cl-lib:int-sub j 1)
                                      (izamax
                                       (f2cl-lib:int-add (f2cl-lib:int-sub m j)
                                                         1)
                                       (f2cl-lib:array-slice a-%data%
                                                             f2cl-lib:complex16
                                                             (j j)
                                                             ((1 lda) (1 *))
                                                             a-%offset%)
                                       1)))
            (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%) jp)
            (cond
             ((/= (f2cl-lib:fref a (jp j) ((1 lda) (1 *))) zero)
```

```
  (if (/= jp j)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (zswap n
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (j 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (jp 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-1 var-3))
        (when var-0 (setf n var-0))
        (when var-2 (setf lda var-2))
        (when var-4 (setf lda var-4))))
  (cond
   ((< j m)
    (cond
     ((>= (abs (f2cl-lib:fref a (j j) ((1 lda) (1 *)))) sfmin)
      (zscal (f2cl-lib:int-sub m j)
       (/ one
          (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                         a-%offset%))
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             ((+ j 1) j) ((1 lda) (1 *))
                             a-%offset%)
       1))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (f2cl-lib:int-add m (f2cl-lib:int-sub j)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add j i) j)
                               ((1 lda) (1 *)) a-%offset%)
                (/
                 (f2cl-lib:fref a-%data%
                                ((f2cl-lib:int-add j i) j)
                                ((1 lda) (1 *)) a-%offset%)
                 (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                a-%offset%)))
          label20)))))))
 ((= info 0) (setf info j)))
(cond
 ((< j (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zgeru (f2cl-lib:int-sub m j) (f2cl-lib:int-sub n j) (- one)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             ((+ j 1) j) ((1 lda) (1 *))
                             a-%offset%)
       1
```

```
                  (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                        (j (f2cl-lib:int-add j 1))
                                        ((1 lda) (1 *)) a-%offset%)
                  lda
                  (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                        ((+ j 1) (f2cl-lib:int-add j 1))
                                        ((1 lda) (1 *)) a-%offset%)
                  lda)
                (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7))
                (when var-6 (setf lda var-6))
                (when var-8 (setf lda var-8)))))
           label10))
        (go end_label)
       end_label
        (return (values nil n nil lda nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgetf2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

35.29 zgetrf

35.29.1 Axiom unit tests

— zgetrf.input —

```
)set break resume
```

```
)sys rm -f zgetrf.output
)spool zgetrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

35.29.2 Axiom help page

— zgetrf.help —

```
=====================================================================
zgetrf examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgetrf (M, N, A, LDA, IPIV, INFO)
           ZGETRF

Function/Subroutine Documentation
   subroutine zgetrf (integer M, integer N, complex*16, dimension( lda, * ) A,
       integer LDA, integer, dimension( * ) IPIV, integer INFO)
       ZGETRF

Purpose:
   ZGETRF computes an LU factorization of a general M-by-N matrix A
   using partial pivoting with row interchanges.

   The factorization has the form
   A = P * L * U
   where P is a permutation matrix, L is lower triangular with unit
   diagonal elements (lower trapezoidal if m > n), and U is upper
   triangular (upper trapezoidal if m < n).

   This is the right-looking Level 3 BLAS version of the algorithm.

Parameters:

 M
```

```
  M is INTEGER
  The number of rows of the matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the M-by-N matrix to be factored.
  On exit, the factors L and U from the factorization
  A = P*L*U; the unit diagonal elements of L are not stored.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

IPIV
  IPIV is INTEGER array, dimension (min(M,N))
  The pivot indices; for 1 <= i <= min(M,N), row i of the
  matrix was interchanged with row IPIV(i).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, U(i,i) is exactly zero. The factorization
        has been completed, but the factor U is exactly
        singular, and division by zero will occur if it is used
        to solve a system of equations.
```

35.29.3 fortran code

— zgetrf.f —

```
*  =====================================================================
      SUBROUTINE ZGETRF( M, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
```

```
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IINFO, J, JB, NB
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGEMM, ZGETRF2, ZLASWP, ZTRSM
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGETRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Determine the block size for this environment.
*
      NB = ILAENV( 1, 'ZGETRF', ' ', M, N, -1, -1 )
```

```
      IF( NB.LE.1 .OR. NB.GE.MIN( M, N ) ) THEN
*
*        Use unblocked code.
*
         CALL ZGETRF2( M, N, A, LDA, IPIV, INFO )
      ELSE
*
*        Use blocked code.
*
         DO 20 J = 1, MIN( M, N ), NB
            JB = MIN( MIN( M, N )-J+1, NB )
*
*           Factor diagonal and subdiagonal blocks and test for exact
*           singularity.
*
            CALL ZGETRF2( M-J+1, JB, A( J, J ), LDA, IPIV( J ), IINFO )
*
*           Adjust INFO and the pivot indices.
*
            IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $         INFO = IINFO + J - 1
            DO 10 I = J, MIN( M, J+JB-1 )
               IPIV( I ) = J - 1 + IPIV( I )
   10       CONTINUE
*
*           Apply interchanges to columns 1:J-1.
*
            CALL ZLASWP( J-1, A, LDA, J, J+JB-1, IPIV, 1 )
*
            IF( J+JB.LE.N ) THEN
*
*              Apply interchanges to columns J+JB:N.
*
               CALL ZLASWP( N-J-JB+1, A( 1, J+JB ), LDA, J, J+JB-1,
     $                      IPIV, 1 )
*
*              Compute block row of U.
*
               CALL ZTRSM( 'Left', 'Lower', 'No transpose', 'Unit', JB,
     $                     N-J-JB+1, ONE, A( J, J ), LDA, A( J, J+JB ),
     $                     LDA )
               IF( J+JB.LE.M ) THEN
*
*                 Update trailing submatrix.
*
                  CALL ZGEMM( 'No transpose', 'No transpose', M-J-JB+1,
     $                        N-J-JB+1, JB, -ONE, A( J+JB, J ), LDA,
     $                        A( J, J+JB ), LDA, ONE, A( J+JB, J+JB ),
     $                        LDA )
               END IF
```

```
            END IF
   20    CONTINUE
      END IF
      RETURN
*
*     End of ZGETRF
*
      END
```

35.29.4 lisp code

— **zgetrf.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (ignorable one))
  (defun zgetrf (m n a lda ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((i 0) (iinfo 0) (j 0) (jb 0) (nb 0))
        (declare (type (f2cl-lib:integer4) i iinfo j jb nb))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "ZGETRF" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (or (= m 0) (= n 0))
            (go end_label))
        (setf nb
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (ilaenv 1 "ZGETRF" " " m n -1 -1)
                  (declare (ignore var-0 var-1 var-2 var-5 var-6))
                  (setf m var-3)
                  (setf n var-4)
                  ret-val))
        (cond
         ((or (<= nb 1)
              (>= nb
```

```
        (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n))))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
     (zgetrf2 m n a lda ipiv info)
   (declare (ignore var-2 var-4))
   (when var-0 (setf m var-0))
   (when var-1 (setf n var-1))
   (when var-3 (setf lda var-3))
   (when var-5 (setf info var-5))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
               ((> j
                   (min (the f2cl-lib:integer4 m)
                        (the f2cl-lib:integer4 n)))
                nil)
   (tagbody
     (setf jb
             (min
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add
                    (f2cl-lib:int-sub
                     (min (the f2cl-lib:integer4 m)
                          (the f2cl-lib:integer4 n))
                     j)
                    1))
              (the f2cl-lib:integer4 nb)))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
         (zgetrf2 (f2cl-lib:int-add (f2cl-lib:int-sub m j) 1) jb
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (j j)
                                ((1 lda) (1 *)) a-%offset%)
          lda
          (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (j)
                                ((1 *)) ipiv-%offset%)
          iinfo)
       (declare (ignore var-0 var-2 var-4))
       (when var-1 (setf jb var-1))
       (when var-3 (setf lda var-3))
       (when var-5 (setf iinfo var-5)))
     (if (and (= info 0) (> iinfo 0))
         (setf info (f2cl-lib:int-sub (f2cl-lib:int-add iinfo j) 1)))
     (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                   ((> i
                       (min (the f2cl-lib:integer4 m)
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add j jb
                                                   (f2cl-lib:int-sub
                                                    1)))))
                    nil)
       (tagbody
         (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
               (f2cl-lib:int-add (f2cl-lib:int-sub j 1)
```

```
                                (f2cl-lib:fref ipiv-%data% (i)
                                               ((1 *))
                                               ipiv-%offset%)))
   label10))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaswp (f2cl-lib:int-sub j 1) a lda j
     (f2cl-lib:int-sub (f2cl-lib:int-add j jb) 1) ipiv 1)
  (declare (ignore var-0 var-1 var-4 var-5 var-6))
  (when var-2 (setf lda var-2))
  (when var-3 (setf j var-3)))
(cond
 ((<= (f2cl-lib:int-add j jb) n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaswp (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             (1 (f2cl-lib:int-add j jb))
                             ((1 lda) (1 *)) a-%offset%)
       lda j (f2cl-lib:int-sub (f2cl-lib:int-add j jb) 1) ipiv 1)
    (declare (ignore var-0 var-1 var-4 var-5 var-6))
    (when var-2 (setf lda var-2))
    (when var-3 (setf j var-3)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (ztrsm "Left" "Lower" "No transpose" "Unit" jb
       (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) one
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (j j)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             (j (f2cl-lib:int-add j jb))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9))
    (when var-4 (setf jb var-4))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf lda var-10)))
  (cond
   ((<= (f2cl-lib:int-add j jb) m)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (zgemm "No transpose" "No transpose"
         (f2cl-lib:int-add (f2cl-lib:int-sub m j jb) 1)
         (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) jb
         (- one)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               ((+ j jb) j) ((1 lda) (1 *))
```

```
                                        a-%offset%)
                      lda
                      (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                        (j (f2cl-lib:int-add j jb))
                                        ((1 lda) (1 *)) a-%offset%)
                      lda one
                      (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                        ((+ j jb) (f2cl-lib:int-add j jb))
                                        ((1 lda) (1 *)) a-%offset%)
                      lda)
                    (declare
                     (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8 var-11))
                    (when var-4 (setf jb var-4))
                    (when var-7 (setf lda var-7))
                    (when var-9 (setf lda var-9))
                    (when var-10 (setf one var-10))
                    (when var-12 (setf lda var-12)))))))
             label20))))
        (go end_label)
       end_label
        (return (values m n nil lda nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgetrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::ilaenv
                                              fortran-to-lisp::xerbla))))
```

———-

35.30 zgetrf2

35.30.1 Axiom unit tests

— zgetrf2.input —

```
)set break resume
)sys rm -f zgetrf2.output
)spool zgetrf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.30.2 Axiom help page

— zgetrf2.help —

```
====================================================================
zgetrf2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       recursive subroutine zgetrf2 (M, N, A, LDA, IPIV, INFO)
           ZGETRF2

Function/Subroutine Documentation
   recursive subroutine zgetrf2 (integer M, integer N, complex*16, dimension(
       lda, * ) A, integer LDA, integer, dimension( * ) IPIV, integer INFO)
       ZGETRF2

Purpose:
   ZGETRF2 computes an LU factorization of a general M-by-N matrix A
   using partial pivoting with row interchanges.

   The factorization has the form
   A = P * L * U
```

```
   where P is a permutation matrix, L is lower triangular with unit
   diagonal elements (lower trapezoidal if m > n), and U is upper
   triangular (upper trapezoidal if m < n).

   This is the recursive version of the algorithm. It divides
   the matrix into four submatrices:

               [  A11 | A12  ]  where A11 is n1 by n1 and A22 is n2 by n2
           A = [ -----|----- ]  with n1 = min(m,n)
               [  A21 | A22  ]       n2 = n-n1

                                                [ A11 ]
   The subroutine calls itself to factor [ --- ],
                                                [ A12 ]
                    [ A12 ]
   do the swaps on  [ --- ], solve A12, update A22,
                    [ A22 ]

   then calls itself to factor A22 and do the swaps on A21.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the M-by-N matrix to be factored.
   On exit, the factors L and U from the factorization
   A = P*L*U; the unit diagonal elements of L are not stored.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).

 IPIV
   IPIV is INTEGER array, dimension (min(M,N))
   The pivot indices; for 1 <= i <= min(M,N), row i of the
   matrix was interchanged with row IPIV(i).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, U(i,i) is exactly zero. The factorization
```

```
has been completed, but the factor U is exactly
singular, and division by zero will occur if it is used
to solve a system of equations.
```

———-

35.30.3 fortran code

— **zgetrf2.f** —

```
*  =====================================================================
      RECURSIVE SUBROUTINE ZGETRF2( M, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE, ZERO
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ),
     $                     ZERO = ( 0.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION   SFMIN
      COMPLEX*16         TEMP
      INTEGER            I, IINFO, N1, N2
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      INTEGER            IZAMAX
      EXTERNAL           DLAMCH, IZAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZGEMM, ZSCAL, ZLASWP, ZTRSM, ZERBLA
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGETRF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN

      IF ( M.EQ.1 ) THEN
*
*        Use unblocked code for one row case
*        Just need to handle IPIV and INFO
*
         IPIV( 1 ) = 1
         IF ( A(1,1).EQ.ZERO )
     $      INFO = 1
*
      ELSE IF( N.EQ.1 ) THEN
*
*        Use unblocked code for one column case
*
*
*        Compute machine safe minimum
*
         SFMIN = DLAMCH('S')
*
*        Find pivot and test for singularity
*
         I = IZAMAX( M, A( 1, 1 ), 1 )
         IPIV( 1 ) = I
         IF( A( I, 1 ).NE.ZERO ) THEN
*
*           Apply the interchange
*
```

```
            IF( I.NE.1 ) THEN
               TEMP = A( 1, 1 )
               A( 1, 1 ) = A( I, 1 )
               A( I, 1 ) = TEMP
            END IF
*
*           Compute elements 2:M of the column
*
            IF( ABS(A( 1, 1 )) .GE. SFMIN ) THEN
               CALL ZSCAL( M-1, ONE / A( 1, 1 ), A( 2, 1 ), 1 )
            ELSE
               DO 10 I = 1, M-1
                  A( 1+I, 1 ) = A( 1+I, 1 ) / A( 1, 1 )
   10          CONTINUE
            END IF
*
         ELSE
            INFO = 1
         END IF

      ELSE
*
*        Use recursive code
*
         N1 = MIN( M, N ) / 2
         N2 = N-N1
*
*               [ A11 ]
*        Factor [ --- ]
*               [ A21 ]
*
         CALL ZGETRF2( M, N1, A, LDA, IPIV, IINFO )

         IF ( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO
*
*                                [ A12 ]
*        Apply interchanges to   [ --- ]
*                                [ A22 ]
*
         CALL ZLASWP( N2, A( 1, N1+1 ), LDA, 1, N1, IPIV, 1 )
*
*        Solve A12
*
         CALL ZTRSM( 'L', 'L', 'N', 'U', N1, N2, ONE, A, LDA,
     $               A( 1, N1+1 ), LDA )
*
*        Update A22
*
         CALL ZGEMM( 'N', 'N', M-N1, N2, N1, -ONE, A( N1+1, 1 ), LDA,
```

```
     $               A( 1, N1+1 ), LDA, ONE, A( N1+1, N1+1 ), LDA )
*
*        Factor A22
*
         CALL ZGETRF2( M-N1, N2, A( N1+1, N1+1 ), LDA, IPIV( N1+1 ),
     $                 IINFO )
*
*        Adjust INFO and the pivot indices
*
         IF ( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO + N1
         DO 20 I = N1+1, MIN( M, N )
            IPIV( I ) = IPIV( I ) + N1
   20    CONTINUE
*
*        Apply interchanges to A21
*
         CALL ZLASWP( N1, A( 1, 1 ), LDA, N1+1, MIN( M, N), IPIV, 1 )
*
      END IF
      RETURN
*
*     End of ZGETRF2
*
      END
```

———-

35.30.4 lisp code

— **zgetrf2.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)) (zero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (type (f2cl-lib:complex16) zero)
           (ignorable one zero))
  (defun *main* ()
    (prog ((i 0) (iinfo 0) (n1 0) (n2 0) (temp #C(0.0d0 0.0d0)) (sfmin 0.0d0)
           (m 0) (info 0)
           (ipiv (make-array * :element-type 'f2cl-lib:integer4)) (lda 0)
           (a
            (make-array (the fixnum (reduce #'* (list lda *))) :element-type
                        'f2cl-lib:complex16))
           (n 0) (subroutine 0.0))
      (declare (type (single-float) subroutine)
               (type (array f2cl-lib:complex16 (*)) a)
               (type (array f2cl-lib:integer4 (*)) ipiv)
               (type (f2cl-lib:complex16) temp)
```

```
        (type (double-float) sfmin)
        (type (f2cl-lib:integer4) n lda info i iinfo n1 n2 m))
'"****NOT TRANSLATED: (RECURSIVE SUBROUTINE ZGETRF2 (M |,| N |,| A |,| LDA |,| IPIV |,| INFO))"
(setf info 0)
(cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
      ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -4)))
(cond
 ((/= info 0) (xerbla "ZGETRF2" (f2cl-lib:int-sub info)) (go end_label)))
(if (or (= m 0) (= n 0))
    (go end_label))
(cond
 ((= m 1) (setf (f2cl-lib:fref ipiv (1) ((1 *))) 1)
  (if (= (f2cl-lib:fref a (1 1) ((1 lda) (1 *))) zero)
      (setf info 1)))
 ((= n 1) (setf sfmin (dlamch "S"))
  (setf i
          (multiple-value-bind (ret-val var-0 var-1 var-2)
              (izamax m
               (f2cl-lib:array-slice a f2cl-lib:complex16 (1 1)
                                     ((1 lda) (1 *)))
               1)
            (declare (ignore var-1 var-2))
            (when var-0 (setf m var-0))
            ret-val))
  (setf (f2cl-lib:fref ipiv (1) ((1 *))) i)
  (cond
   ((/= (f2cl-lib:fref a (i 1) ((1 lda) (1 *))) zero)
    (cond
     ((/= i 1) (setf temp (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))
      (setf (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))
              (f2cl-lib:fref a (i 1) ((1 lda) (1 *))))
      (setf (f2cl-lib:fref a (i 1) ((1 lda) (1 *))) temp)))
    (cond
     ((>= (abs (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))) sfmin)
      (zscal (f2cl-lib:int-sub m 1)
       (/ one (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))
       (f2cl-lib:array-slice a f2cl-lib:complex16 (2 1) ((1 lda) (1 *)))
       1))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
        (tagbody
          (setf (f2cl-lib:fref a ((f2cl-lib:int-add 1 i) 1)
                                ((1 lda) (1 *)))
                (/
                 (f2cl-lib:fref a ((f2cl-lib:int-add 1 i) 1)
                                ((1 lda) (1 *)))
                 (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))))
         label10)))))
```

```
  (t (setf info 1))))
 (t (setf n1 (the f2cl-lib:integer4 (truncate (min m n) 2)))
  (setf n2 (f2cl-lib:int-sub n n1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (zgetrf2 m n1 a lda ipiv iinfo)
    (declare (ignore var-2 var-4))
    (when var-0 (setf m var-0))
    (when var-1 (setf n1 var-1))
    (when var-3 (setf lda var-3))
    (when var-5 (setf iinfo var-5)))
  (if (and (= info 0) (> iinfo 0))
      (setf info iinfo))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaswp n2
       (f2cl-lib:array-slice a f2cl-lib:complex16
                             (1 (f2cl-lib:int-add n1 1)) ((1 lda) (1 *)))
       lda 1 n1 ipiv 1)
    (declare (ignore var-1 var-3 var-5 var-6))
    (when var-0 (setf n2 var-0))
    (when var-2 (setf lda var-2))
    (when var-4 (setf n1 var-4)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ztrsm "L" "L" "N" "U" n1 n2 one a lda
       (f2cl-lib:array-slice a f2cl-lib:complex16
                             (1 (f2cl-lib:int-add n1 1)) ((1 lda) (1 *)))
       lda)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n1 var-4))
    (when var-5 (setf n2 var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf lda var-10)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
       var-11 var-12)
      (zgemm "N" "N" (f2cl-lib:int-sub m n1) n2 n1 (- one)
       (f2cl-lib:array-slice a f2cl-lib:complex16 ((+ n1 1) 1)
                             ((1 lda) (1 *)))
       lda
       (f2cl-lib:array-slice a f2cl-lib:complex16
                             (1 (f2cl-lib:int-add n1 1)) ((1 lda) (1 *)))
       lda one
       (f2cl-lib:array-slice a f2cl-lib:complex16
                             ((+ n1 1) (f2cl-lib:int-add n1 1))
                             ((1 lda) (1 *)))
       lda)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
    (when var-3 (setf n2 var-3))
```

```
          (when var-4 (setf n1 var-4))
          (when var-7 (setf lda var-7))
          (when var-9 (setf lda var-9))
          (when var-10 (setf one var-10))
          (when var-12 (setf lda var-12)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (zgetrf2 (f2cl-lib:int-sub m n1) n2
             (f2cl-lib:array-slice a f2cl-lib:complex16
                                   ((+ n1 1) (f2cl-lib:int-add n1 1))
                                   ((1 lda) (1 *)))
             lda
             (f2cl-lib:array-slice ipiv f2cl-lib:integer4 ((+ n1 1)) ((1 *)))
             iinfo)
          (declare (ignore var-0 var-2 var-4))
          (when var-1 (setf n2 var-1))
          (when var-3 (setf lda var-3))
          (when var-5 (setf iinfo var-5)))
        (if (and (= info 0) (> iinfo 0))
            (setf info (f2cl-lib:int-add iinfo n1)))
        (f2cl-lib:fdo (i (f2cl-lib:int-add n1 1) (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the f2cl-lib:integer4 m)
                               (the f2cl-lib:integer4 n)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref ipiv (i) ((1 *)))
                  (f2cl-lib:int-add (f2cl-lib:fref ipiv (i) ((1 *))) n1))
           label20))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zlaswp n1
             (f2cl-lib:array-slice a f2cl-lib:complex16 (1 1) ((1 lda) (1 *)))
             lda (f2cl-lib:int-add n1 1)
             (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) ipiv 1)
          (declare (ignore var-1 var-3 var-4 var-5 var-6))
          (when var-0 (setf n1 var-0))
          (when var-2 (setf lda var-2)))))
      (go end_label)
     end_label
      (return nil))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::*main*
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types 'nil :return-values 'nil
                                            :calls 'nil)))
```

———-

35.31 zgetri

35.31.1 Axiom unit tests

— **zgetri.input** —

```
)set break resume
)sys rm -f zgetri.output
)spool zgetri.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.31.2 Axiom help page

— **zgetri.help** —

```
======================================================================
zgetri examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgetri (N, A, LDA, IPIV, WORK, LWORK, INFO)
           ZGETRI

Function/Subroutine Documentation
   subroutine zgetri (integer N, complex*16, dimension( lda, * ) A, integer
       LDA, integer, dimension( * ) IPIV, complex*16, dimension( * ) WORK,
       integer LWORK, integer INFO)
       ZGETRI

Purpose:
```

```
  ZGETRI computes the inverse of a matrix using the LU factorization
  computed by ZGETRF.

  This method inverts U and then computes inv(A) by solving the system
  inv(A)*L = inv(U) for inv(A).

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the factors L and U from the factorization
   A = P*L*U as computed by ZGETRF.
   On exit, if INFO = 0, the inverse of the original matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   The pivot indices from ZGETRF; for 1<=i<=N, row i of the
   matrix was interchanged with row IPIV(i).

 WORK
   WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
   On exit, if INFO=0, then WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.  LWORK >= max(1,N).
   For optimal performance LWORK >= N*NB, where NB is
   the optimal blocksize returned by ILAENV.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, U(i,i) is exactly zero; the matrix is
         singular and its inverse could not be computed.
```

———-

35.31.3 fortran code

— **zgetri.f** —

```
*  =====================================================================
      SUBROUTINE ZGETRI( N, A, LDA, IPIV, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ZERO, ONE
      PARAMETER          ( ZERO = ( 0.0D+0, 0.0D+0 ),
     $                   ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IWS, J, JB, JJ, JP, LDWORK, LWKOPT, NB,
     $                   NBMIN, NN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGEMM, ZGEMV, ZSWAP, ZTRSM, ZTRTRI
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
```

```
      INFO = 0
      NB = ILAENV( 1, 'ZGETRI', ' ', N, -1, -1, -1 )
      LWKOPT = N*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -3
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGETRI', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Form inv(U).  If INFO > 0 from ZTRTRI, then U is singular,
*     and the inverse is not computed.
*
      CALL ZTRTRI( 'Upper', 'Non-unit', N, A, LDA, INFO )
      IF( INFO.GT.0 )
     $   RETURN
*
      NBMIN = 2
      LDWORK = N
      IF( NB.GT.1 .AND. NB.LT.N ) THEN
         IWS = MAX( LDWORK*NB, 1 )
         IF( LWORK.LT.IWS ) THEN
            NB = LWORK / LDWORK
            NBMIN = MAX( 2, ILAENV( 2, 'ZGETRI', ' ', N, -1, -1, -1 ) )
         END IF
      ELSE
         IWS = N
      END IF
*
*     Solve the equation inv(A)*L = inv(U) for inv(A).
*
      IF( NB.LT.NBMIN .OR. NB.GE.N ) THEN
*
*        Use unblocked code.
*
         DO 20 J = N, 1, -1
```

```
*
*          Copy current column of L to WORK and replace with zeros.
*
           DO 10 I = J + 1, N
              WORK( I ) = A( I, J )
              A( I, J ) = ZERO
   10      CONTINUE
*
*          Compute current column of inv(A).
*
           IF( J.LT.N )
     $        CALL ZGEMV( 'No transpose', N, N-J, -ONE, A( 1, J+1 ),
     $                    LDA, WORK( J+1 ), 1, ONE, A( 1, J ), 1 )
   20    CONTINUE
      ELSE
*
*       Use blocked code.
*
        NN = ( ( N-1 ) / NB )*NB + 1
        DO 50 J = NN, 1, -NB
           JB = MIN( NB, N-J+1 )
*
*          Copy current block column of L to WORK and replace with
*          zeros.
*
           DO 40 JJ = J, J + JB - 1
              DO 30 I = JJ + 1, N
                 WORK( I+( JJ-J )*LDWORK ) = A( I, JJ )
                 A( I, JJ ) = ZERO
   30         CONTINUE
   40      CONTINUE
*
*          Compute current block column of inv(A).
*
           IF( J+JB.LE.N )
     $        CALL ZGEMM( 'No transpose', 'No transpose', N, JB,
     $                    N-J-JB+1, -ONE, A( 1, J+JB ), LDA,
     $                    WORK( J+JB ), LDWORK, ONE, A( 1, J ), LDA )
           CALL ZTRSM( 'Right', 'Lower', 'No transpose', 'Unit', N, JB,
     $                 ONE, WORK( J ), LDWORK, A( 1, J ), LDA )
   50    CONTINUE
      END IF
*
*     Apply column interchanges.
*
      DO 60 J = N - 1, 1, -1
         JP = IPIV( J )
         IF( JP.NE.J )
     $      CALL ZSWAP( N, A( 1, J ), 1, A( 1, JP ), 1 )
   60 CONTINUE
```

```
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of ZGETRI
*
      END
```

———-

35.31.4 lisp code

— **zgetri.lisp** —

```
(let* ((zero (f2cl-lib:cmplx 0.0d0 0.0d0)) (one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) zero)
           (type (f2cl-lib:complex16) one)
           (ignorable zero one))
  (defun zgetri (n a lda ipiv work lwork info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) work a)
             (type (f2cl-lib:integer4) info lwork lda n))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((i 0) (iws 0) (j 0) (jb 0) (jj 0) (jp 0) (ldwork 0) (lwkopt 0)
             (nb 0) (nbmin 0) (nn 0) (lquery nil))
        (declare
         (type (f2cl-lib:integer4) i iws j jb jj jp ldwork lwkopt nb nbmin nn)
         (type f2cl-lib:logical lquery))
        (setf info 0)
        (setf nb
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (ilaenv 1 "ZGETRI" " " n -1 -1 -1)
                  (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6))
                  (setf n var-3)
                  ret-val))
        (setf lwkopt (f2cl-lib:int-mul n nb))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce lwkopt 'f2cl-lib:complex16))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond ((< n 0) (setf info -1))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -3))
              ((and
```

```
        (< lwork
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (not lquery))
       (setf info -6)))
(cond
 ((/= info 0) (xerbla "ZGETRI" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (ztrtri "Upper" "Non-unit" n a lda info)
  (declare (ignore var-0 var-1 var-3))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-5 (setf info var-5)))
(if (> info 0)
    (go end_label))
(setf nbmin 2)
(setf ldwork n)
(cond
 ((and (> nb 1) (< nb n))
  (setf iws
          (max (the f2cl-lib:integer4 (f2cl-lib:int-mul ldwork nb))
               (the f2cl-lib:integer4 1)))
  (cond
   ((< lwork iws)
    (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
    (setf nbmin
            (max (the f2cl-lib:integer4 2)
                 (the f2cl-lib:integer4
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
                          (ilaenv 2 "ZGETRI" " " n -1 -1 -1)
                        (declare
                         (ignore var-0 var-1 var-2 var-4 var-5 var-6))
                        (setf n var-3)
                        ret-val)))))))
 (t (setf iws n)))
(cond
 ((or (< nb nbmin) (>= nb n))
  (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                ((> j 1) nil)
    (tagbody
      (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%))
```

```
            (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%)
                  zero)
          label10))
      (if (< j n)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10)
              (zgemv "No transpose" n (f2cl-lib:int-sub n j) (- one)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                     (1 (f2cl-lib:int-add j 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                     ((+ j 1)) ((1 *)) work-%offset%)
               1 one
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 j)
                                     ((1 lda) (1 *)) a-%offset%)
               1)
            (declare
             (ignore var-0 var-2 var-3 var-4 var-6 var-7 var-9 var-10))
            (when var-1 (setf n var-1))
            (when var-5 (setf lda var-5))
            (when var-8 (setf one var-8))))
    label20)))
(t
 (setf nn (+ (* (the f2cl-lib:integer4 (truncate (- n 1) nb)) nb) 1))
 (f2cl-lib:fdo (j nn (f2cl-lib:int-add j (f2cl-lib:int-sub nb)))
               ((> j 1) nil)
   (tagbody
     (setf jb
             (min (the f2cl-lib:integer4 nb)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1))))
     (f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
                   ((> jj
                       (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
                    nil)
       (tagbody
         (f2cl-lib:fdo (i (f2cl-lib:int-add jj 1)
                          (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add i
                                                     (f2cl-lib:int-mul
                                                      (f2cl-lib:int-sub
                                                       jj j)
                                                      ldwork)))
                                  ((1 *)) work-%offset%)
```

```
                        (f2cl-lib:fref a-%data% (i jj) ((1 lda) (1 *))
                                       a-%offset%))
                    (setf (f2cl-lib:fref a-%data% (i jj) ((1 lda) (1 *))
                                         a-%offset%)
                        zero)
                  label30))
               label40))
            (if (<= (f2cl-lib:int-add j jb) n)
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                     var-9 var-10 var-11 var-12)
                    (zgemm "No transpose" "No transpose" n jb
                     (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) (- one)
                     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                           (1 (f2cl-lib:int-add j jb))
                                           ((1 lda) (1 *)) a-%offset%)
                     lda
                     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                           ((+ j jb)) ((1 *)) work-%offset%)
                     ldwork one
                     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 j)
                                           ((1 lda) (1 *)) a-%offset%)
                     lda)
                  (declare
                   (ignore var-0 var-1 var-4 var-5 var-6 var-8 var-11))
                  (when var-2 (setf n var-2))
                  (when var-3 (setf jb var-3))
                  (when var-7 (setf lda var-7))
                  (when var-9 (setf ldwork var-9))
                  (when var-10 (setf one var-10))
                  (when var-12 (setf lda var-12))))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                 var-10)
                (ztrsm "Right" "Lower" "No transpose" "Unit" n jb one
                 (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (j)
                                       ((1 *)) work-%offset%)
                 ldwork
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 j)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
              (when var-4 (setf n var-4))
              (when var-5 (setf jb var-5))
              (when var-6 (setf one var-6))
              (when var-8 (setf ldwork var-8))
              (when var-10 (setf lda var-10)))
          label50))))
     (f2cl-lib:fdo (j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                    (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
```

```
                    ((> j 1) nil)
          (tagbody
            (setf jp (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
            (if (/= jp j)
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (zswap n
                     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 j)
                                           ((1 lda) (1 *)) a-%offset%)
                     1
                     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 jp)
                                           ((1 lda) (1 *)) a-%offset%)
                     1)
                  (declare (ignore var-1 var-2 var-3 var-4))
                  (when var-0 (setf n var-0))))
           label60))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce iws 'f2cl-lib:complex16))
        (go end_label)
       end_label
        (return (values n nil lda nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgetri
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::xerbla
                                              fortran-to-lisp::ilaenv))))
```

———-

35.32 zgetrs

35.32.1 Axiom unit tests

— zgetrs.input —

```
)set break resume
)sys rm -f zgetrs.output
)spool zgetrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.32.2 Axiom help page

— zgetrs.help —

```
======================================================================
zgetrs examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgetrs (TRANS, N, NRHS, A, LDA, IPIV, B, LDB, INFO)
           ZGETRS

Function/Subroutine Documentation
   subroutine zgetrs (character TRANS, integer N, integer NRHS, complex*16,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       complex*16, dimension( ldb, * ) B, integer LDB, integer INFO)
       ZGETRS

Purpose:
   ZGETRS solves a system of linear equations
      A * X = B,  A**T * X = B,  or  A**H * X = B
   with a general N-by-N matrix A using the LU factorization computed
   by ZGETRF.
```

```
Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   The factors L and U from the factorization A = P*L*U
   as computed by ZGETRF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   The pivot indices from ZGETRF; for 1<=i<=N, row i of the
   matrix was interchanged with row IPIV(i).

 B
   B is COMPLEX*16 array, dimension (LDB,NRHS)
   On entry, the right hand side matrix B.
   On exit, the solution matrix X.

 LDB
   LDB is INTEGER
   The leading dimension of the array B.  LDB >= max(1,N).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

35.32.3 fortran code

— zgetrs.f —

```
*  =====================================================================
      SUBROUTINE ZGETRS( TRANS, N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLASWP, ZTRSM
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
```

```
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGETRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( NOTRAN ) THEN
*
*        Solve A * X = B.
*
*        Apply row interchanges to the right hand sides.
*
         CALL ZLASWP( NRHS, B, LDB, 1, N, IPIV, 1 )
*
*        Solve L*X = B, overwriting B with X.
*
         CALL ZTRSM( 'Left', 'Lower', 'No transpose', 'Unit', N, NRHS,
     $               ONE, A, LDA, B, LDB )
*
*        Solve U*X = B, overwriting B with X.
*
         CALL ZTRSM( 'Left', 'Upper', 'No transpose', 'Non-unit', N,
     $               NRHS, ONE, A, LDA, B, LDB )
      ELSE
*
*        Solve A**T * X = B  or A**H * X = B.
*
*        Solve U**T *X = B or U**H *X = B, overwriting B with X.
*
         CALL ZTRSM( 'Left', 'Upper', TRANS, 'Non-unit', N, NRHS, ONE,
     $               A, LDA, B, LDB )
*
*        Solve L**T *X = B, or L**H *X = B overwriting B with X.
*
         CALL ZTRSM( 'Left', 'Lower', TRANS, 'Unit', N, NRHS, ONE, A,
     $               LDA, B, LDB )
*
```

```
*        Apply row interchanges to the solution vectors.
*
         CALL ZLASWP( NRHS, B, LDB, 1, N, IPIV, -1 )
      END IF
*
      RETURN
*
*     End of ZGETRS
*
      END
```

———-

35.32.4 lisp code

— **zgetrs.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (ignorable one))
  (defun zgetrs (trans n nrhs a lda ipiv b ldb$ info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) b a)
             (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((notran nil))
        (declare (type f2cl-lib:logical notran))
        (setf info 0)
        (setf notran
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame trans "N")
                  (declare (ignore var-1))
                  (when var-0 (setf trans var-0))
                  ret-val))
        (cond
         ((and (not notran)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame trans "T")
                  (declare (ignore var-1))
                  (when var-0 (setf trans var-0))
                  ret-val))
               (not
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8)))
(cond
 ((/= info 0) (xerbla "ZGETRS" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= n 0) (= nrhs 0))
    (go end_label))
(cond
 (notran
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlaswp nrhs b ldb$ 1 n ipiv 1)
    (declare (ignore var-1 var-3 var-5 var-6))
    (when var-0 (setf nrhs var-0))
    (when var-2 (setf ldb$ var-2))
    (when var-4 (setf n var-4)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ztrsm "Left" "Lower" "No transpose" "Unit" n nrhs one a lda b
       ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ztrsm "Left" "Upper" "No transpose" "Non-unit" n nrhs one a lda
       b ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
```

```
              (ztrsm "Left" "Upper" trans "Non-unit" n nrhs one a lda b ldb$)
            (declare (ignore var-0 var-1 var-3 var-7 var-9))
            (when var-2 (setf trans var-2))
            (when var-4 (setf n var-4))
            (when var-5 (setf nrhs var-5))
            (when var-6 (setf one var-6))
            (when var-8 (setf lda var-8))
            (when var-10 (setf ldb$ var-10)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10)
              (ztrsm "Left" "Lower" trans "Unit" n nrhs one a lda b ldb$)
            (declare (ignore var-0 var-1 var-3 var-7 var-9))
            (when var-2 (setf trans var-2))
            (when var-4 (setf n var-4))
            (when var-5 (setf nrhs var-5))
            (when var-6 (setf one var-6))
            (when var-8 (setf lda var-8))
            (when var-10 (setf ldb$ var-10)))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (zlaswp nrhs b ldb$ 1 n ipiv -1)
            (declare (ignore var-1 var-3 var-5 var-6))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf n var-4)))))
        (go end_label)
       end_label
        (return (values trans n nrhs nil lda nil nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgetrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::n
```

```
                                        fortran-to-lisp::nrhs nil
                                        fortran-to-lisp::lda nil nil
                                        fortran-to-lisp::ldb$
                                        fortran-to-lisp::info)
                                      :calls '(fortran-to-lisp::xerbla))))
```

———-

35.33 zhgeqz

35.33.1 Axiom unit tests

— **zhgeqz.input** —

```
)set break resume
)sys rm -f zhgeqz.output
)spool zhgeqz.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.33.2 Axiom help page [Mole71]

— **zhgeqz.help** —

```
=====================================================================
zhgeqz examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zhgeqz (JOB, COMPQ, COMPZ, N, ILO, IHI, H, LDH, T, LDT,
           ALPHA, BETA, Q, LDQ, Z, LDZ, WORK, LWORK, RWORK, INFO)
```

```
          ZHGEQZ

Function/Subroutine Documentation
   subroutine zhgeqz (character JOB, character COMPQ, character COMPZ, integer
       N, integer ILO, integer IHI, complex*16, dimension( ldh, * ) H, integer
       LDH, complex*16, dimension( ldt, * ) T, integer LDT, complex*16,
       dimension( * ) ALPHA, complex*16, dimension( * ) BETA, complex*16,
       dimension( ldq, * ) Q, integer LDQ, complex*16, dimension( ldz, * ) Z,
       integer LDZ, complex*16, dimension( * ) WORK, integer LWORK, double
       precision, dimension( * ) RWORK, integer INFO)
       ZHGEQZ

Purpose:
   ZHGEQZ computes the eigenvalues of a complex matrix pair (H,T),
   where H is an upper Hessenberg matrix and T is upper triangular,
   using the single-shift QZ method.
   Matrix pairs of this type are produced by the reduction to
   generalized upper Hessenberg form of a complex matrix pair (A,B):

   A = Q1*H*Z1**H,  B = Q1*T*Z1**H,

   as computed by ZGGHRD.

   If JOB='S', then the Hessenberg-triangular pair (H,T) is
   also reduced to generalized Schur form,

   H = Q*S*Z**H,  T = Q*P*Z**H,

   where Q and Z are unitary matrices and S and P are upper triangular.

   Optionally, the unitary matrix Q from the generalized Schur
   factorization may be postmultiplied into an input matrix Q1, and the
   unitary matrix Z may be postmultiplied into an input matrix Z1.
   If Q1 and Z1 are the unitary matrices from ZGGHRD that reduced
   the matrix pair (A,B) to generalized Hessenberg form, then the output
   matrices Q1*Q and Z1*Z are the unitary factors from the generalized
   Schur factorization of (A,B):

   A = (Q1*Q)*S*(Z1*Z)**H,  B = (Q1*Q)*P*(Z1*Z)**H.

   To avoid overflow, eigenvalues of the matrix pair (H,T)
   (equivalently, of (A,B)) are computed as a pair of complex values
   (alpha,beta).  If beta is nonzero, lambda = alpha / beta is an
   eigenvalue of the generalized nonsymmetric eigenvalue problem (GNEP)
       A*x = lambda*B*x
   and if alpha is nonzero, mu = beta / alpha is an eigenvalue of the
   alternate form of the GNEP
       mu*A*y = B*y.
   The values of alpha and beta for the i-th eigenvalue can be read
   directly from the generalized Schur form:  alpha = S(i,i),
```

```
   beta = P(i,i).

Parameters:

 JOB
   JOB is CHARACTER*1
   = 'E': Compute eigenvalues only;
   = 'S': Computer eigenvalues and the Schur form.

 COMPQ
   COMPQ is CHARACTER*1
   = 'N': Left Schur vectors (Q) are not computed;
   = 'I': Q is initialized to the unit matrix and the matrix Q
          of left Schur vectors of (H,T) is returned;
   = 'V': Q must contain a unitary matrix Q1 on entry and
          the product Q1*Q is returned.

 COMPZ
   COMPZ is CHARACTER*1
   = 'N': Right Schur vectors (Z) are not computed;
   = 'I': Q is initialized to the unit matrix and the matrix Z
          of right Schur vectors of (H,T) is returned;
   = 'V': Z must contain a unitary matrix Z1 on entry and
          the product Z1*Z is returned.

 N
   N is INTEGER
   The order of the matrices H, T, Q, and Z.  N >= 0.

 ILO
   ILO is INTEGER

 IHI
   IHI is INTEGER
   ILO and IHI mark the rows and columns of H which are in
   Hessenberg form.  It is assumed that A is already upper
   triangular in rows and columns 1:ILO-1 and IHI+1:N.
   If N > 0, 1 <= ILO <= IHI <= N; if N = 0, ILO=1 and IHI=0.

 H
   H is COMPLEX*16 array, dimension (LDH, N)
   On entry, the N-by-N upper Hessenberg matrix H.
   On exit, if JOB = 'S', H contains the upper triangular
   matrix S from the generalized Schur factorization.
   If JOB = 'E', the diagonal of H matches that of S, but
   the rest of H is unspecified.

 LDH
   LDH is INTEGER
   The leading dimension of the array H.  LDH >= max( 1, N ).
```

```
T
  T is COMPLEX*16 array, dimension (LDT, N)
  On entry, the N-by-N upper triangular matrix T.
  On exit, if JOB = 'S', T contains the upper triangular
  matrix P from the generalized Schur factorization.
  If JOB = 'E', the diagonal of T matches that of P, but
  the rest of T is unspecified.

LDT
  LDT is INTEGER
  The leading dimension of the array T.  LDT >= max( 1, N ).

ALPHA
  ALPHA is COMPLEX*16 array, dimension (N)
  The complex scalars alpha that define the eigenvalues of
  GNEP.  ALPHA(i) = S(i,i) in the generalized Schur factorization.

BETA
  BETA is COMPLEX*16 array, dimension (N)
  The real non-negative scalars beta that define the
  eigenvalues of GNEP.  BETA(i) = P(i,i) in the generalized
  Schur factorization.

  Together, the quantities alpha = ALPHA(j) and beta = BETA(j)
  represent the j-th eigenvalue of the matrix pair (A,B), in
  one of the forms lambda = alpha/beta or mu = beta/alpha.
  Since either lambda or mu may overflow, they should not,
  in general, be computed.

Q
  Q is COMPLEX*16 array, dimension (LDQ, N)
  On entry, if COMPZ = 'V', the unitary matrix Q1 used in the
  reduction of (A,B) to generalized Hessenberg form.
  On exit, if COMPZ = 'I', the unitary matrix of left Schur
  vectors of (H,T), and if COMPZ = 'V', the unitary matrix of
  left Schur vectors of (A,B).
  Not referenced if COMPZ = 'N'.

LDQ
  LDQ is INTEGER
  The leading dimension of the array Q.  LDQ >= 1.
  If COMPQ='V' or 'I', then LDQ >= N.

Z
  Z is COMPLEX*16 array, dimension (LDZ, N)
  On entry, if COMPZ = 'V', the unitary matrix Z1 used in the
  reduction of (A,B) to generalized Hessenberg form.
  On exit, if COMPZ = 'I', the unitary matrix of right Schur
  vectors of (H,T), and if COMPZ = 'V', the unitary matrix of
```

```
  right Schur vectors of (A,B).
  Not referenced if COMPZ = 'N'.

LDZ
  LDZ is INTEGER
  The leading dimension of the array Z.  LDZ >= 1.
  If COMPZ='V' or 'I', then LDZ >= N.

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO >= 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,N).

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (N)

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  = 1,...,N: the QZ iteration did not converge.  (H,T) is not
       in Schur form, but ALPHA(i) and BETA(i),
  i=INFO+1,...,N should be correct.
       = N+1,...,2*N: the shift calculation failed.  (H,T) is not
       in Schur form, but ALPHA(i) and BETA(i),
  i=INFO-N+1,...,N should be correct.

  We assume that complex ABS works as long as its value is less than
  overflow.
```

———-

35.33.3 fortran code

— zhgeqz.f —

```
*  =====================================================================
      SUBROUTINE ZHGEQZ( JOB, COMPQ, COMPZ, N, ILO, IHI, H, LDH, T, LDT,
     $                   ALPHA, BETA, Q, LDQ, Z, LDZ, WORK, LWORK,
```

```
     $                   RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          COMPQ, COMPZ, JOB
      INTEGER            IHI, ILO, INFO, LDH, LDQ, LDT, LDZ, LWORK, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   RWORK( * )
      COMPLEX*16         ALPHA( * ), BETA( * ), H( LDH, * ),
     $                   Q( LDQ, * ), T( LDT, * ), WORK( * ),
     $                   Z( LDZ, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0D+0, 0.0D+0 ),
     $                   CONE = ( 1.0D+0, 0.0D+0 ) )
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   HALF
      PARAMETER          ( HALF = 0.5D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ILAZR2, ILAZRO, ILQ, ILSCHR, ILZ, LQUERY
      INTEGER            ICOMPQ, ICOMPZ, IFIRST, IFRSTM, IITER, ILAST,
     $                   ILASTM, IN, ISCHUR, ISTART, J, JC, JCH, JITER,
     $                   JR, MAXIT
      DOUBLE PRECISION   ABSB, ANORM, ASCALE, ATOL, BNORM, BSCALE, BTOL,
     $                   C, SAFMIN, TEMP, TEMP2, TEMPR, ULP
      COMPLEX*16         ABI22, AD11, AD12, AD21, AD22, CTEMP, CTEMP2,
     $                   CTEMP3, ESHIFT, RTDISC, S, SHIFT, SIGNBC, T1,
     $                   U12, X
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, ZLANHS
      EXTERNAL           LSAME, DLAMCH, ZLANHS
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLARTG, ZLASET, ZROT, ZSCAL
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DCMPLX, DCONJG, DIMAG, MAX, MIN,
```

```
     $                   SQRT
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   ABS1
*     ..
*     .. Statement Function definitions ..
      ABS1( X ) = ABS( DBLE( X ) ) + ABS( DIMAG( X ) )
*     ..
*     .. Executable Statements ..
*
*     Decode JOB, COMPQ, COMPZ
*
      IF( LSAME( JOB, 'E' ) ) THEN
         ILSCHR = .FALSE.
         ISCHUR = 1
      ELSE IF( LSAME( JOB, 'S' ) ) THEN
         ILSCHR = .TRUE.
         ISCHUR = 2
      ELSE
         ISCHUR = 0
      END IF
*
      IF( LSAME( COMPQ, 'N' ) ) THEN
         ILQ = .FALSE.
         ICOMPQ = 1
      ELSE IF( LSAME( COMPQ, 'V' ) ) THEN
         ILQ = .TRUE.
         ICOMPQ = 2
      ELSE IF( LSAME( COMPQ, 'I' ) ) THEN
         ILQ = .TRUE.
         ICOMPQ = 3
      ELSE
         ICOMPQ = 0
      END IF
*
      IF( LSAME( COMPZ, 'N' ) ) THEN
         ILZ = .FALSE.
         ICOMPZ = 1
      ELSE IF( LSAME( COMPZ, 'V' ) ) THEN
         ILZ = .TRUE.
         ICOMPZ = 2
      ELSE IF( LSAME( COMPZ, 'I' ) ) THEN
         ILZ = .TRUE.
         ICOMPZ = 3
      ELSE
         ICOMPZ = 0
      END IF
*
*     Check Argument Values
*
```

```
      INFO = 0
      WORK( 1 ) = MAX( 1, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( ISCHUR.EQ.0 ) THEN
         INFO = -1
      ELSE IF( ICOMPQ.EQ.0 ) THEN
         INFO = -2
      ELSE IF( ICOMPZ.EQ.0 ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( ILO.LT.1 ) THEN
         INFO = -5
      ELSE IF( IHI.GT.N .OR. IHI.LT.ILO-1 ) THEN
         INFO = -6
      ELSE IF( LDH.LT.N ) THEN
         INFO = -8
      ELSE IF( LDT.LT.N ) THEN
         INFO = -10
      ELSE IF( LDQ.LT.1 .OR. ( ILQ .AND. LDQ.LT.N ) ) THEN
         INFO = -14
      ELSE IF( LDZ.LT.1 .OR. ( ILZ .AND. LDZ.LT.N ) ) THEN
         INFO = -16
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -18
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZHGEQZ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
*     WORK( 1 ) = CMPLX( 1 )
      IF( N.LE.0 ) THEN
         WORK( 1 ) = DCMPLX( 1 )
         RETURN
      END IF
*
*     Initialize Q and Z
*
      IF( ICOMPQ.EQ.3 )
     $   CALL ZLASET( 'Full', N, N, CZERO, CONE, Q, LDQ )
      IF( ICOMPZ.EQ.3 )
     $   CALL ZLASET( 'Full', N, N, CZERO, CONE, Z, LDZ )
*
*     Machine Constants
*
```

```
      IN = IHI + 1 - ILO
      SAFMIN = DLAMCH( 'S' )
      ULP = DLAMCH( 'E' )*DLAMCH( 'B' )
      ANORM = ZLANHS( 'F', IN, H( ILO, ILO ), LDH, RWORK )
      BNORM = ZLANHS( 'F', IN, T( ILO, ILO ), LDT, RWORK )
      ATOL = MAX( SAFMIN, ULP*ANORM )
      BTOL = MAX( SAFMIN, ULP*BNORM )
      ASCALE = ONE / MAX( SAFMIN, ANORM )
      BSCALE = ONE / MAX( SAFMIN, BNORM )
*
*
*     Set Eigenvalues IHI+1:N
*
      DO 10 J = IHI + 1, N
         ABSB = ABS( T( J, J ) )
         IF( ABSB.GT.SAFMIN ) THEN
            SIGNBC = DCONJG( T( J, J ) / ABSB )
            T( J, J ) = ABSB
            IF( ILSCHR ) THEN
               CALL ZSCAL( J-1, SIGNBC, T( 1, J ), 1 )
               CALL ZSCAL( J, SIGNBC, H( 1, J ), 1 )
            ELSE
               CALL ZSCAL( 1, SIGNBC, H( J, J ), 1 )
            END IF
            IF( ILZ )
     $         CALL ZSCAL( N, SIGNBC, Z( 1, J ), 1 )
         ELSE
            T( J, J ) = CZERO
         END IF
         ALPHA( J ) = H( J, J )
         BETA( J ) = T( J, J )
   10 CONTINUE
*
*     If IHI < ILO, skip QZ steps
*
      IF( IHI.LT.ILO )
     $   GO TO 190
*
*     MAIN QZ ITERATION LOOP
*
*     Initialize dynamic indices
*
*     Eigenvalues ILAST+1:N have been found.
*        Column operations modify rows IFRSTM:whatever
*        Row operations modify columns whatever:ILASTM
*
*     If only eigenvalues are being computed, then
*        IFRSTM is the row of the last splitting row above row ILAST;
*        this is always at least ILO.
*     IITER counts iterations since the last eigenvalue was found,
```

```
*        to tell when to use an extraordinary shift.
*     MAXIT is the maximum number of QZ sweeps allowed.
*
      ILAST = IHI
      IF( ILSCHR ) THEN
         IFRSTM = 1
         ILASTM = N
      ELSE
         IFRSTM = ILO
         ILASTM = IHI
      END IF
      IITER = 0
      ESHIFT = CZERO
      MAXIT = 30*( IHI-ILO+1 )
*
      DO 170 JITER = 1, MAXIT
*
*        Check for too many iterations.
*
         IF( JITER.GT.MAXIT )
     $      GO TO 180
*
*        Split the matrix if possible.
*
*        Two tests:
*           1: H(j,j-1)=0  or  j=ILO
*           2: T(j,j)=0
*
*        Special case: j=ILAST
*
         IF( ILAST.EQ.ILO ) THEN
            GO TO 60
         ELSE
            IF( ABS1( H( ILAST, ILAST-1 ) ).LE.ATOL ) THEN
               H( ILAST, ILAST-1 ) = CZERO
               GO TO 60
            END IF
         END IF
*
         IF( ABS( T( ILAST, ILAST ) ).LE.BTOL ) THEN
            T( ILAST, ILAST ) = CZERO
            GO TO 50
         END IF
*
*        General case: j<ILAST
*
         DO 40 J = ILAST - 1, ILO, -1
*
*           Test 1: for H(j,j-1)=0 or j=ILO
*
```

```
            IF( J.EQ.ILO ) THEN
               ILAZRO = .TRUE.
            ELSE
               IF( ABS1( H( J, J-1 ) ).LE.ATOL ) THEN
                  H( J, J-1 ) = CZERO
                  ILAZRO = .TRUE.
               ELSE
                  ILAZRO = .FALSE.
               END IF
            END IF
*
*           Test 2: for T(j,j)=0
*
            IF( ABS( T( J, J ) ).LT.BTOL ) THEN
               T( J, J ) = CZERO
*
*              Test 1a: Check for 2 consecutive small subdiagonals in A
*
               ILAZR2 = .FALSE.
               IF( .NOT.ILAZRO ) THEN
                  IF( ABS1( H( J, J-1 ) )*( ASCALE*ABS1( H( J+1,
     $                J ) ) ).LE.ABS1( H( J, J ) )*( ASCALE*ATOL ) )
     $                ILAZR2 = .TRUE.
               END IF
*
*              If both tests pass (1 & 2), i.e., the leading diagonal
*              element of B in the block is zero, split a 1x1 block off
*              at the top. (I.e., at the J-th row/column) The leading
*              diagonal element of the remainder can also be zero, so
*              this may have to be done repeatedly.
*
               IF( ILAZRO .OR. ILAZR2 ) THEN
                  DO 20 JCH = J, ILAST - 1
                     CTEMP = H( JCH, JCH )
                     CALL ZLARTG( CTEMP, H( JCH+1, JCH ), C, S,
     $                            H( JCH, JCH ) )
                     H( JCH+1, JCH ) = CZERO
                     CALL ZROT( ILASTM-JCH, H( JCH, JCH+1 ), LDH,
     $                          H( JCH+1, JCH+1 ), LDH, C, S )
                     CALL ZROT( ILASTM-JCH, T( JCH, JCH+1 ), LDT,
     $                          T( JCH+1, JCH+1 ), LDT, C, S )
                     IF( ILQ )
     $                  CALL ZROT( N, Q( 1, JCH ), 1, Q( 1, JCH+1 ), 1,
     $                             C, DCONJG( S ) )
                     IF( ILAZR2 )
     $                  H( JCH, JCH-1 ) = H( JCH, JCH-1 )*C
                     ILAZR2 = .FALSE.
                     IF( ABS1( T( JCH+1, JCH+1 ) ).GE.BTOL ) THEN
                        IF( JCH+1.GE.ILAST ) THEN
                           GO TO 60
```

```
                     ELSE
                        IFIRST = JCH + 1
                        GO TO 70
                     END IF
                  END IF
                  T( JCH+1, JCH+1 ) = CZERO
   20          CONTINUE
               GO TO 50
            ELSE
*
*              Only test 2 passed -- chase the zero to T(ILAST,ILAST)
*              Then process as in the case T(ILAST,ILAST)=0
*
               DO 30 JCH = J, ILAST - 1
                  CTEMP = T( JCH, JCH+1 )
                  CALL ZLARTG( CTEMP, T( JCH+1, JCH+1 ), C, S,
     $                         T( JCH, JCH+1 ) )
                  T( JCH+1, JCH+1 ) = CZERO
                  IF( JCH.LT.ILASTM-1 )
     $               CALL ZROT( ILASTM-JCH-1, T( JCH, JCH+2 ), LDT,
     $                          T( JCH+1, JCH+2 ), LDT, C, S )
                  CALL ZROT( ILASTM-JCH+2, H( JCH, JCH-1 ), LDH,
     $                       H( JCH+1, JCH-1 ), LDH, C, S )
                  IF( ILQ )
     $               CALL ZROT( N, Q( 1, JCH ), 1, Q( 1, JCH+1 ), 1,
     $                          C, DCONJG( S ) )
                  CTEMP = H( JCH+1, JCH )
                  CALL ZLARTG( CTEMP, H( JCH+1, JCH-1 ), C, S,
     $                         H( JCH+1, JCH ) )
                  H( JCH+1, JCH-1 ) = CZERO
                  CALL ZROT( JCH+1-IFRSTM, H( IFRSTM, JCH ), 1,
     $                       H( IFRSTM, JCH-1 ), 1, C, S )
                  CALL ZROT( JCH-IFRSTM, T( IFRSTM, JCH ), 1,
     $                       T( IFRSTM, JCH-1 ), 1, C, S )
                  IF( ILZ )
     $               CALL ZROT( N, Z( 1, JCH ), 1, Z( 1, JCH-1 ), 1,
     $                          C, S )
   30          CONTINUE
               GO TO 50
            END IF
         ELSE IF( ILAZRO ) THEN
*
*           Only test 1 passed -- work on J:ILAST
*
            IFIRST = J
            GO TO 70
         END IF
*
*        Neither test passed -- try next J
*
```

```
   40    CONTINUE
*
*        (Drop-through is "impossible")
*
         INFO = 2*N + 1
         GO TO 210
*
*        T(ILAST,ILAST)=0 -- clear H(ILAST,ILAST-1) to split off a
*        1x1 block.
*
   50    CONTINUE
         CTEMP = H( ILAST, ILAST )
         CALL ZLARTG( CTEMP, H( ILAST, ILAST-1 ), C, S,
     $                H( ILAST, ILAST ) )
         H( ILAST, ILAST-1 ) = CZERO
         CALL ZROT( ILAST-IFRSTM, H( IFRSTM, ILAST ), 1,
     $              H( IFRSTM, ILAST-1 ), 1, C, S )
         CALL ZROT( ILAST-IFRSTM, T( IFRSTM, ILAST ), 1,
     $              T( IFRSTM, ILAST-1 ), 1, C, S )
         IF( ILZ )
     $      CALL ZROT( N, Z( 1, ILAST ), 1, Z( 1, ILAST-1 ), 1, C, S )
*
*        H(ILAST,ILAST-1)=0 -- Standardize B, set ALPHA and BETA
*
   60    CONTINUE
         ABSB = ABS( T( ILAST, ILAST ) )
         IF( ABSB.GT.SAFMIN ) THEN
            SIGNBC = DCONJG( T( ILAST, ILAST ) / ABSB )
            T( ILAST, ILAST ) = ABSB
            IF( ILSCHR ) THEN
               CALL ZSCAL( ILAST-IFRSTM, SIGNBC, T( IFRSTM, ILAST ), 1 )
               CALL ZSCAL( ILAST+1-IFRSTM, SIGNBC, H( IFRSTM, ILAST ),
     $                     1 )
            ELSE
               CALL ZSCAL( 1, SIGNBC, H( ILAST, ILAST ), 1 )
            END IF
            IF( ILZ )
     $         CALL ZSCAL( N, SIGNBC, Z( 1, ILAST ), 1 )
         ELSE
            T( ILAST, ILAST ) = CZERO
         END IF
         ALPHA( ILAST ) = H( ILAST, ILAST )
         BETA( ILAST ) = T( ILAST, ILAST )
*
*        Go to next block -- exit if finished.
*
         ILAST = ILAST - 1
         IF( ILAST.LT.ILO )
     $      GO TO 190
*
```

```
*        Reset counters
*
         IITER = 0
         ESHIFT = CZERO
         IF( .NOT.ILSCHR ) THEN
            ILASTM = ILAST
            IF( IFRSTM.GT.ILAST )
     $         IFRSTM = ILO
         END IF
         GO TO 160
*
*        QZ step
*
*        This iteration only involves rows/columns IFIRST:ILAST.  We
*        assume IFIRST < ILAST, and that the diagonal of B is non-zero.
*
   70    CONTINUE
         IITER = IITER + 1
         IF( .NOT.ILSCHR ) THEN
            IFRSTM = IFIRST
         END IF
*
*        Compute the Shift.
*
*        At this point, IFIRST < ILAST, and the diagonal elements of
*        T(IFIRST:ILAST,IFIRST,ILAST) are larger than BTOL (in
*        magnitude)
*
         IF( ( IITER / 10 )*10.NE.IITER ) THEN
*
*           The Wilkinson shift (AEP p.512), i.e., the eigenvalue of
*           the bottom-right 2x2 block of A inv(B) which is nearest to
*           the bottom-right element.
*
*           We factor B as U*D, where U has unit diagonals, and
*           compute (A*inv(D))*inv(U).
*
            U12 = ( BSCALE*T( ILAST-1, ILAST ) ) /
     $            ( BSCALE*T( ILAST, ILAST ) )
            AD11 = ( ASCALE*H( ILAST-1, ILAST-1 ) ) /
     $             ( BSCALE*T( ILAST-1, ILAST-1 ) )
            AD21 = ( ASCALE*H( ILAST, ILAST-1 ) ) /
     $             ( BSCALE*T( ILAST-1, ILAST-1 ) )
            AD12 = ( ASCALE*H( ILAST-1, ILAST ) ) /
     $             ( BSCALE*T( ILAST, ILAST ) )
            AD22 = ( ASCALE*H( ILAST, ILAST ) ) /
     $             ( BSCALE*T( ILAST, ILAST ) )
            ABI22 = AD22 - U12*AD21
*
            T1 = HALF*( AD11+ABI22 )
```

```
            RTDISC = SQRT( T1**2+AD12*AD21-AD11*AD22 )
            TEMP = DBLE( T1-ABI22 )*DBLE( RTDISC ) +
     $             DIMAG( T1-ABI22 )*DIMAG( RTDISC )
            IF( TEMP.LE.ZERO ) THEN
               SHIFT = T1 + RTDISC
            ELSE
               SHIFT = T1 - RTDISC
            END IF
         ELSE
*
*           Exceptional shift.  Chosen for no particularly good reason.
*
            ESHIFT = ESHIFT + (ASCALE*H(ILAST,ILAST-1))/
     $                        (BSCALE*T(ILAST-1,ILAST-1))
            SHIFT = ESHIFT
         END IF
*
*        Now check for two consecutive small subdiagonals.
*
         DO 80 J = ILAST - 1, IFIRST + 1, -1
            ISTART = J
            CTEMP = ASCALE*H( J, J ) - SHIFT*( BSCALE*T( J, J ) )
            TEMP = ABS1( CTEMP )
            TEMP2 = ASCALE*ABS1( H( J+1, J ) )
            TEMPR = MAX( TEMP, TEMP2 )
            IF( TEMPR.LT.ONE .AND. TEMPR.NE.ZERO ) THEN
               TEMP = TEMP / TEMPR
               TEMP2 = TEMP2 / TEMPR
            END IF
            IF( ABS1( H( J, J-1 ) )*TEMP2.LE.TEMP*ATOL )
     $         GO TO 90
   80    CONTINUE
*
         ISTART = IFIRST
         CTEMP = ASCALE*H( IFIRST, IFIRST ) -
     $           SHIFT*( BSCALE*T( IFIRST, IFIRST ) )
   90    CONTINUE
*
*        Do an implicit-shift QZ sweep.
*
*        Initial Q
*
         CTEMP2 = ASCALE*H( ISTART+1, ISTART )
         CALL ZLARTG( CTEMP, CTEMP2, C, S, CTEMP3 )
*
*        Sweep
*
         DO 150 J = ISTART, ILAST - 1
            IF( J.GT.ISTART ) THEN
               CTEMP = H( J, J-1 )
```

```
               CALL ZLARTG( CTEMP, H( J+1, J-1 ), C, S, H( J, J-1 ) )
               H( J+1, J-1 ) = CZERO
            END IF
*
            DO 100 JC = J, ILASTM
               CTEMP = C*H( J, JC ) + S*H( J+1, JC )
               H( J+1, JC ) = -DCONJG( S )*H( J, JC ) + C*H( J+1, JC )
               H( J, JC ) = CTEMP
               CTEMP2 = C*T( J, JC ) + S*T( J+1, JC )
               T( J+1, JC ) = -DCONJG( S )*T( J, JC ) + C*T( J+1, JC )
               T( J, JC ) = CTEMP2
  100       CONTINUE
            IF( ILQ ) THEN
               DO 110 JR = 1, N
                  CTEMP = C*Q( JR, J ) + DCONJG( S )*Q( JR, J+1 )
                  Q( JR, J+1 ) = -S*Q( JR, J ) + C*Q( JR, J+1 )
                  Q( JR, J ) = CTEMP
  110          CONTINUE
            END IF
*
            CTEMP = T( J+1, J+1 )
            CALL ZLARTG( CTEMP, T( J+1, J ), C, S, T( J+1, J+1 ) )
            T( J+1, J ) = CZERO
*
            DO 120 JR = IFRSTM, MIN( J+2, ILAST )
               CTEMP = C*H( JR, J+1 ) + S*H( JR, J )
               H( JR, J ) = -DCONJG( S )*H( JR, J+1 ) + C*H( JR, J )
               H( JR, J+1 ) = CTEMP
  120       CONTINUE
            DO 130 JR = IFRSTM, J
               CTEMP = C*T( JR, J+1 ) + S*T( JR, J )
               T( JR, J ) = -DCONJG( S )*T( JR, J+1 ) + C*T( JR, J )
               T( JR, J+1 ) = CTEMP
  130       CONTINUE
            IF( ILZ ) THEN
               DO 140 JR = 1, N
                  CTEMP = C*Z( JR, J+1 ) + S*Z( JR, J )
                  Z( JR, J ) = -DCONJG( S )*Z( JR, J+1 ) + C*Z( JR, J )
                  Z( JR, J+1 ) = CTEMP
  140          CONTINUE
            END IF
  150    CONTINUE
*
  160    CONTINUE
*
  170 CONTINUE
*
*     Drop-through = non-convergence
*
  180 CONTINUE
```

```
      INFO = ILAST
      GO TO 210
*
*     Successful completion of all QZ steps
*
  190 CONTINUE
*
*     Set Eigenvalues 1:ILO-1
*
      DO 200 J = 1, ILO - 1
         ABSB = ABS( T( J, J ) )
         IF( ABSB.GT.SAFMIN ) THEN
            SIGNBC = DCONJG( T( J, J ) / ABSB )
            T( J, J ) = ABSB
            IF( ILSCHR ) THEN
               CALL ZSCAL( J-1, SIGNBC, T( 1, J ), 1 )
               CALL ZSCAL( J, SIGNBC, H( 1, J ), 1 )
            ELSE
               CALL ZSCAL( 1, SIGNBC, H( J, J ), 1 )
            END IF
            IF( ILZ )
     $         CALL ZSCAL( N, SIGNBC, Z( 1, J ), 1 )
         ELSE
            T( J, J ) = CZERO
         END IF
         ALPHA( J ) = H( J, J )
         BETA( J ) = T( J, J )
  200 CONTINUE
*
*     Normal Termination
*
      INFO = 0
*
*     Exit (other than argument error) -- return optimal workspace size
*
  210 CONTINUE
      WORK( 1 ) = DCMPLX( N )
      RETURN
*
*     End of ZHGEQZ
*
      END
```

———-

35.33.4 lisp code

— zhgeqz.lisp —

```
(let* ((czero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (cone (f2cl-lib:cmplx 1.0d0 0.0d0))
       (zero 0.0d0)
       (one 1.0d0)
       (half 0.5d0))
  (declare (type (f2cl-lib:complex16) czero)
           (type (f2cl-lib:complex16) cone)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.5d0 0.5d0) half)
           (ignorable czero cone zero one half))
  (defun zhgeqz
         (job compq compz n ilo ihi h ldh t$ ldt alpha beta q ldq z ldz work
          lwork rwork info)
    (declare (type (array double-float (*)) rwork)
             (type (array f2cl-lib:complex16 (*)) work z q beta alpha t$ h)
             (type (f2cl-lib:integer4) info lwork ldz ldq ldt ldh ihi ilo n)
             (type (string *) compz compq job))
    (f2cl-lib:with-multi-array-data
        ((job character job-%data% job-%offset%)
         (compq character compq-%data% compq-%offset%)
         (compz character compz-%data% compz-%offset%)
         (h f2cl-lib:complex16 h-%data% h-%offset%)
         (t$ f2cl-lib:complex16 t$-%data% t$-%offset%)
         (alpha f2cl-lib:complex16 alpha-%data% alpha-%offset%)
         (beta f2cl-lib:complex16 beta-%data% beta-%offset%)
         (q f2cl-lib:complex16 q-%data% q-%offset%)
         (z f2cl-lib:complex16 z-%data% z-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (labels ((abs1 (x)
                 (+ (abs (f2cl-lib:dble x)) (abs (f2cl-lib:dimag x)))))
        (declare
         (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
          abs1))
        (prog ((abi22 #C(0.0d0 0.0d0)) (ad11 #C(0.0d0 0.0d0))
               (ad12 #C(0.0d0 0.0d0)) (ad21 #C(0.0d0 0.0d0))
               (ad22 #C(0.0d0 0.0d0)) (ctemp #C(0.0d0 0.0d0))
               (ctemp2 #C(0.0d0 0.0d0)) (ctemp3 #C(0.0d0 0.0d0))
               (eshift #C(0.0d0 0.0d0)) (rtdisc #C(0.0d0 0.0d0))
               (s #C(0.0d0 0.0d0)) (shift #C(0.0d0 0.0d0))
               (signbc #C(0.0d0 0.0d0)) (t1 #C(0.0d0 0.0d0))
               (u12 #C(0.0d0 0.0d0)) (x #C(0.0d0 0.0d0)) (absb 0.0d0)
               (anorm 0.0d0) (ascale 0.0d0) (atol 0.0d0) (bnorm 0.0d0)
               (bscale 0.0d0) (btol 0.0d0) (c 0.0d0) (safmin 0.0d0)
               (temp 0.0d0) (temp2 0.0d0) (tempr 0.0d0) (ulp 0.0d0) (icompq 0)
               (icompz 0) (ifirst 0) (ifrstm 0) (iiter 0) (ilast 0) (ilastm 0)
               (in 0) (ischur 0) (istart 0) (j 0) (jc 0) (jch 0) (jiter 0)
               (jr 0) (maxit 0) (ilazr2 nil) (ilazro nil) (ilq nil)
               (ilschr nil) (ilz nil) (lquery nil))
```

```
(declare
 (type (f2cl-lib:complex16) abi22 ad11 ad12 ad21 ad22 ctemp ctemp2
  ctemp3 eshift rtdisc s shift signbc t1 u12 x)
 (type (double-float) absb anorm ascale atol bnorm bscale btol c
  safmin temp temp2 tempr ulp)
 (type (f2cl-lib:integer4) icompq icompz ifirst ifrstm iiter ilast
  ilastm in ischur istart j jc jch jiter jr maxit)
 (type f2cl-lib:logical ilazr2 ilazro ilq ilschr ilz lquery))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame job "E")
    (declare (ignore var-1))
    (when var-0 (setf job var-0))
    ret-val)
  (setf ilschr f2cl-lib:%false%) (setf ischur 1))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame job "S")
    (declare (ignore var-1))
    (when var-0 (setf job var-0))
    ret-val)
  (setf ilschr f2cl-lib:%true%) (setf ischur 2))
 (t (setf ischur 0)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compq "N")
    (declare (ignore var-1))
    (when var-0 (setf compq var-0))
    ret-val)
  (setf ilq f2cl-lib:%false%) (setf icompq 1))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compq "V")
    (declare (ignore var-1))
    (when var-0 (setf compq var-0))
    ret-val)
  (setf ilq f2cl-lib:%true%) (setf icompq 2))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compq "I")
    (declare (ignore var-1))
    (when var-0 (setf compq var-0))
    ret-val)
  (setf ilq f2cl-lib:%true%) (setf icompq 3))
 (t (setf icompq 0)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "N")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf ilz f2cl-lib:%false%) (setf icompz 1))
 ((multiple-value-bind (ret-val var-0 var-1)
```

```
      (lsame compz "V")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf ilz f2cl-lib:%true%) (setf icompz 2))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "I")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf ilz f2cl-lib:%true%) (setf icompz 3))
 (t (setf icompz 0)))
(setf info 0)
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce
         (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n))
         'f2cl-lib:complex16))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((= ischur 0) (setf info -1)) ((= icompq 0) (setf info -2))
      ((= icompz 0) (setf info -3)) ((< n 0) (setf info -4))
      ((< ilo 1) (setf info -5))
      ((or (> ihi n)
           (< ihi (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))))
       (setf info -6))
      ((< ldh n) (setf info -8)) ((< ldt n) (setf info -10))
      ((or (< ldq 1) (and ilq (< ldq n))) (setf info -14))
      ((or (< ldz 1) (and ilz (< ldz n))) (setf info -16))
      ((and
        (< lwork
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (not lquery))
       (setf info -18)))
(cond
 ((/= info 0) (xerbla "ZHGEQZ" (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(cond
 ((<= n 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:dcmplx 1))
  (go end_label)))
(if (= icompq 3)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlaset "Full" n n czero cone q ldq)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldq var-6))))
```

```
(if (= icompz 3)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlaset "Full" n n czero cone z ldz)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldz var-6))))
(setf in (f2cl-lib:int-sub (f2cl-lib:int-add ihi 1) ilo))
(setf safmin (dlamch "S"))
(setf ulp (* (dlamch "E") (dlamch "B")))
(setf anorm
        (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
            (zlanhs "F" in
             (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                                   (ilo ilo) ((1 ldh) (1 *))
                                   h-%offset%)
             ldh rwork)
          (declare (ignore var-0 var-2 var-4))
          (when var-1 (setf in var-1))
          (when var-3 (setf ldh var-3))
          ret-val))
(setf bnorm
        (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
            (zlanhs "F" in
             (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16
                                   (ilo ilo) ((1 ldt) (1 *))
                                   t$-%offset%)
             ldt rwork)
          (declare (ignore var-0 var-2 var-4))
          (when var-1 (setf in var-1))
          (when var-3 (setf ldt var-3))
          ret-val))
(setf atol (max safmin (* ulp anorm)))
(setf btol (max safmin (* ulp bnorm)))
(setf ascale (/ one (max safmin anorm)))
(setf bscale (/ one (max safmin bnorm)))
(f2cl-lib:fdo (j (f2cl-lib:int-add ihi 1) (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf absb
            (abs
             (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                            t$-%offset%)))
    (cond
     ((> absb safmin)
      (setf signbc
              (coerce
               (f2cl-lib:dconjg
```

```
            (/
             (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                            t$-%offset%)
             absb))
           'f2cl-lib:complex16))
  (setf (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                       t$-%offset%)
          (coerce absb 'f2cl-lib:complex16))
  (cond
   (ilschr
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zscal (f2cl-lib:int-sub j 1) signbc
         (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 j)
                               ((1 ldt) (1 *)) t$-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3))
      (when var-1 (setf signbc var-1)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zscal j signbc
         (f2cl-lib:array-slice h-%data% f2cl-lib:complex16 (1 j)
                               ((1 ldh) (1 *)) h-%offset%)
         1)
      (declare (ignore var-2 var-3))
      (when var-0 (setf j var-0))
      (when var-1 (setf signbc var-1))))
   (t
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zscal 1 signbc
         (f2cl-lib:array-slice h-%data% f2cl-lib:complex16 (j j)
                               ((1 ldh) (1 *)) h-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3))
      (when var-1 (setf signbc var-1)))))
  (if ilz
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (zscal n signbc
           (f2cl-lib:array-slice z-%data% f2cl-lib:complex16
                                 (1 j) ((1 ldz) (1 *))
                                 z-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf n var-0))
        (when var-1 (setf signbc var-1)))))
 (t
  (setf (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                       t$-%offset%)
          czero)))
(setf (f2cl-lib:fref alpha-%data% (j) ((1 *)) alpha-%offset%)
        (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *))
                       h-%offset%))
```

```
    (setf (f2cl-lib:fref beta-%data% (j) ((1 *)) beta-%offset%)
            (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                           t$-%offset%))
   label10))
(if (< ihi ilo)
    (go label190))
(setf ilast ihi)
(cond (ilschr (setf ifrstm 1) (setf ilastm n))
      (t (setf ifrstm ilo) (setf ilastm ihi)))
(setf iiter 0)
(setf eshift czero)
(setf maxit
        (f2cl-lib:int-mul 30
                          (f2cl-lib:int-add
                           (f2cl-lib:int-sub ihi ilo) 1)))
(f2cl-lib:fdo (jiter 1 (f2cl-lib:int-add jiter 1))
              ((> jiter maxit) nil)
  (tagbody
    (if (> jiter maxit)
        (go label180))
    (cond ((= ilast ilo) (go label60))
          (t
           (cond
            ((<=
              (abs1
               (f2cl-lib:fref h
                              (ilast
                               (f2cl-lib:int-add ilast
                                                 (f2cl-lib:int-sub
                                                  1)))
                              ((1 ldh) (1 *))))
              atol)
             (setf (f2cl-lib:fref h-%data%
                                  (ilast (f2cl-lib:int-sub ilast 1))
                                  ((1 ldh) (1 *)) h-%offset%)
                   czero)
             (go label60)))))
    (cond
     ((<= (abs (f2cl-lib:fref t$ (ilast ilast) ((1 ldt) (1 *))))
          btol)
      (setf (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                           t$-%offset%)
            czero)
      (go label50)))
    (f2cl-lib:fdo (j (f2cl-lib:int-add ilast (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                  ((> j ilo) nil)
      (tagbody
        (cond ((= j ilo) (setf ilazro f2cl-lib:%true%))
              (t
```

```
                (cond
                 ((<=
                   (abs1
                    (f2cl-lib:fref h
                                   (j
                                    (f2cl-lib:int-add j
                                                      (f2cl-lib:int-sub
                                                       1)))
                                   ((1 ldh) (1 *))))
                   atol)
                  (setf (f2cl-lib:fref h-%data%
                                       (j (f2cl-lib:int-sub j 1))
                                       ((1 ldh) (1 *)) h-%offset%)
                        czero)
                  (setf ilazro f2cl-lib:%true%))
                 (t (setf ilazro f2cl-lib:%false%)))))
        (cond
         ((< (abs (f2cl-lib:fref t$ (j j) ((1 ldt) (1 *)))) btol)
          (setf (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                               t$-%offset%)
                czero)
          (setf ilazr2 f2cl-lib:%false%)
          (cond
           ((not ilazro)
            (if (<=
                 (*
                  (abs1
                   (f2cl-lib:fref h-%data% (j (f2cl-lib:int-sub j 1))
                                  ((1 ldh) (1 *)) h-%offset%))
                  (* ascale
                     (abs1
                      (f2cl-lib:fref h-%data%
                                     ((f2cl-lib:int-add j 1) j)
                                     ((1 ldh) (1 *)) h-%offset%))))
                 (*
                  (abs1
                   (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *))
                                  h-%offset%))
                  (* ascale atol)))
                (setf ilazr2 f2cl-lib:%true%))))
          (cond
           ((or ilazro ilazr2)
            (f2cl-lib:fdo (jch j (f2cl-lib:int-add jch 1))
                          ((> jch
                              (f2cl-lib:int-add ilast
                                                (f2cl-lib:int-sub
                                                 1)))
                           nil)
              (tagbody
                (setf ctemp
```

```
        (f2cl-lib:fref h-%data% (jch jch)
                       ((1 ldh) (1 *)) h-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (zlartg ctemp
     (f2cl-lib:array-slice h-%data%
                           f2cl-lib:complex16
                           ((+ jch 1) jch)
                           ((1 ldh) (1 *))
                           h-%offset%)
     c s
     (f2cl-lib:array-slice h-%data%
                           f2cl-lib:complex16
                           (jch jch) ((1 ldh) (1 *))
                           h-%offset%))
  (declare (ignore var-1 var-4))
  (when var-0 (setf ctemp var-0))
  (when var-2 (setf c var-2))
  (when var-3 (setf s var-3)))
(setf (f2cl-lib:fref h-%data%
                     ((f2cl-lib:int-add jch 1) jch)
                     ((1 ldh) (1 *)) h-%offset%)
        czero)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zrot (f2cl-lib:int-sub ilastm jch)
     (f2cl-lib:array-slice h-%data%
                           f2cl-lib:complex16
                           (jch
                            (f2cl-lib:int-add jch 1))
                           ((1 ldh) (1 *))
                           h-%offset%)
     ldh
     (f2cl-lib:array-slice h-%data%
                           f2cl-lib:complex16
                           ((+ jch 1)
                            (f2cl-lib:int-add jch 1))
                           ((1 ldh) (1 *))
                           h-%offset%)
     ldh c s)
  (declare (ignore var-0 var-1 var-3))
  (when var-2 (setf ldh var-2))
  (when var-4 (setf ldh var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zrot (f2cl-lib:int-sub ilastm jch)
     (f2cl-lib:array-slice t$-%data%
                           f2cl-lib:complex16
                           (jch
```

```
                                (f2cl-lib:int-add jch 1))
                               ((1 ldt) (1 *))
                               t$-%offset%)
      ldt
      (f2cl-lib:array-slice t$-%data%
                               f2cl-lib:complex16
                               ((+ jch 1)
                                (f2cl-lib:int-add jch 1))
                               ((1 ldt) (1 *))
                               t$-%offset%)
      ldt c s)
  (declare (ignore var-0 var-1 var-3))
  (when var-2 (setf ldt var-2))
  (when var-4 (setf ldt var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(if ilq
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zrot n
         (f2cl-lib:array-slice q-%data%
                               f2cl-lib:complex16
                               (1 jch)
                               ((1 ldq) (1 *))
                               q-%offset%)
         1
         (f2cl-lib:array-slice q-%data%
                               f2cl-lib:complex16
                               (1
                                (f2cl-lib:int-add jch
                                                  1))
                               ((1 ldq) (1 *))
                               q-%offset%)
         1 c (f2cl-lib:dconjg s))
      (declare
       (ignore var-1 var-2 var-3 var-4 var-6))
      (when var-0 (setf n var-0))
      (when var-5 (setf c var-5))))
(if ilazr2
    (setf (f2cl-lib:fref h-%data%
                         (jch
                          (f2cl-lib:int-sub jch 1))
                         ((1 ldh) (1 *)) h-%offset%)
            (*
             (f2cl-lib:fref h-%data%
                            (jch
                             (f2cl-lib:int-sub jch
                                               1))
                            ((1 ldh) (1 *))
                            h-%offset%)
```

```
                  c)))
        (setf ilazr2 f2cl-lib:%false%)
        (cond
         ((>=
           (abs1
            (f2cl-lib:fref t$
                           ((f2cl-lib:int-add jch 1)
                            (f2cl-lib:int-add jch 1))
                           ((1 ldt) (1 *))))
           btol)
          (cond
           ((>= (f2cl-lib:int-add jch 1) ilast) (go label60))
           (t (setf ifirst (f2cl-lib:int-add jch 1))
            (go label70)))))
        (setf (f2cl-lib:fref t$-%data%
                             ((f2cl-lib:int-add jch 1)
                              (f2cl-lib:int-add jch 1))
                             ((1 ldt) (1 *)) t$-%offset%)
              czero)
      label20))
    (go label50))
   (t
    (f2cl-lib:fdo (jch j (f2cl-lib:int-add jch 1))
                  ((> jch
                      (f2cl-lib:int-add ilast
                                        (f2cl-lib:int-sub
                                         1)))
                   nil)
      (tagbody
        (setf ctemp
              (f2cl-lib:fref t$-%data%
                             (jch (f2cl-lib:int-add jch 1))
                             ((1 ldt) (1 *)) t$-%offset%))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (zlartg ctemp
             (f2cl-lib:array-slice t$-%data%
                                   f2cl-lib:complex16
                                   ((+ jch 1)
                                    (f2cl-lib:int-add jch 1))
                                   ((1 ldt) (1 *))
                                   t$-%offset%)
             c s
             (f2cl-lib:array-slice t$-%data%
                                   f2cl-lib:complex16
                                   (jch
                                    (f2cl-lib:int-add jch 1))
                                   ((1 ldt) (1 *))
                                   t$-%offset%))
          (declare (ignore var-1 var-4))
          (when var-0 (setf ctemp var-0))
```

```
  (when var-2 (setf c var-2))
  (when var-3 (setf s var-3)))
(setf (f2cl-lib:fref t$-%data%
                     ((f2cl-lib:int-add jch 1)
                      (f2cl-lib:int-add jch 1))
                     ((1 ldt) (1 *)) t$-%offset%)
        czero)
(if (< jch (f2cl-lib:int-sub ilastm 1))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zrot (f2cl-lib:int-sub ilastm jch 1)
         (f2cl-lib:array-slice t$-%data%
                               f2cl-lib:complex16
                               (jch
                                (f2cl-lib:int-add jch
                                                  2))
                               ((1 ldt) (1 *))
                               t$-%offset%)
         ldt
         (f2cl-lib:array-slice t$-%data%
                               f2cl-lib:complex16
                               ((+ jch 1)
                                (f2cl-lib:int-add jch
                                                  2))
                               ((1 ldt) (1 *))
                               t$-%offset%)
         ldt c s)
      (declare (ignore var-0 var-1 var-3))
      (when var-2 (setf ldt var-2))
      (when var-4 (setf ldt var-4))
      (when var-5 (setf c var-5))
      (when var-6 (setf s var-6))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zrot
     (f2cl-lib:int-add (f2cl-lib:int-sub ilastm jch)
                       2)
     (f2cl-lib:array-slice h-%data%
                           f2cl-lib:complex16
                           (jch
                            (f2cl-lib:int-sub jch 1))
                           ((1 ldh) (1 *))
                           h-%offset%)
     ldh
     (f2cl-lib:array-slice h-%data%
                           f2cl-lib:complex16
                           ((+ jch 1)
                            (f2cl-lib:int-sub jch 1))
                           ((1 ldh) (1 *))
                           h-%offset%)
```

```
     ldh c s)
  (declare (ignore var-0 var-1 var-3))
  (when var-2 (setf ldh var-2))
  (when var-4 (setf ldh var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(if ilq
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zrot n
         (f2cl-lib:array-slice q-%data%
                               f2cl-lib:complex16
                               (1 jch)
                               ((1 ldq) (1 *))
                               q-%offset%)
         1
         (f2cl-lib:array-slice q-%data%
                               f2cl-lib:complex16
                               (1
                                (f2cl-lib:int-add jch
                                                  1))
                               ((1 ldq) (1 *))
                               q-%offset%)
         1 c (f2cl-lib:dconjg s))
      (declare
       (ignore var-1 var-2 var-3 var-4 var-6))
      (when var-0 (setf n var-0))
      (when var-5 (setf c var-5))))
(setf ctemp
        (f2cl-lib:fref h-%data%
                       ((f2cl-lib:int-add jch 1) jch)
                       ((1 ldh) (1 *)) h-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (zlartg ctemp
     (f2cl-lib:array-slice h-%data%
                           f2cl-lib:complex16
                           ((+ jch 1)
                            (f2cl-lib:int-sub jch 1))
                           ((1 ldh) (1 *))
                           h-%offset%)
     c s
     (f2cl-lib:array-slice h-%data%
                           f2cl-lib:complex16
                           ((+ jch 1) jch)
                           ((1 ldh) (1 *))
                           h-%offset%))
  (declare (ignore var-1 var-4))
  (when var-0 (setf ctemp var-0))
  (when var-2 (setf c var-2))
  (when var-3 (setf s var-3)))
```

```
(setf (f2cl-lib:fref h-%data%
                     ((f2cl-lib:int-add jch 1)
                      (f2cl-lib:int-sub jch 1))
                     ((1 ldh) (1 *)) h-%offset%)
        czero)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zrot
     (f2cl-lib:int-sub (f2cl-lib:int-add jch 1)
                       ifrstm)
     (f2cl-lib:array-slice h-%data%
                           f2cl-lib:complex16
                           (ifrstm jch)
                           ((1 ldh) (1 *))
                           h-%offset%)
     1
     (f2cl-lib:array-slice h-%data%
                           f2cl-lib:complex16
                           (ifrstm
                            (f2cl-lib:int-sub jch 1))
                           ((1 ldh) (1 *))
                           h-%offset%)
     1 c s)
  (declare (ignore var-0 var-1 var-2 var-3 var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zrot (f2cl-lib:int-sub jch ifrstm)
     (f2cl-lib:array-slice t$-%data%
                           f2cl-lib:complex16
                           (ifrstm jch)
                           ((1 ldt) (1 *))
                           t$-%offset%)
     1
     (f2cl-lib:array-slice t$-%data%
                           f2cl-lib:complex16
                           (ifrstm
                            (f2cl-lib:int-sub jch 1))
                           ((1 ldt) (1 *))
                           t$-%offset%)
     1 c s)
  (declare (ignore var-0 var-1 var-2 var-3 var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(if ilz
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zrot n
         (f2cl-lib:array-slice z-%data%
```

```
                                                f2cl-lib:complex16
                                                (1 jch)
                                                ((1 ldz) (1 *))
                                                z-%offset%)
                          1
                          (f2cl-lib:array-slice z-%data%
                                                f2cl-lib:complex16
                                                (1
                                                 (f2cl-lib:int-sub jch
                                                                   1))
                                                ((1 ldz) (1 *))
                                                z-%offset%)
                          1 c s)
                       (declare (ignore var-1 var-2 var-3 var-4))
                       (when var-0 (setf n var-0))
                       (when var-5 (setf c var-5))
                       (when var-6 (setf s var-6))))
                label30))
             (go label50))))
          (ilazro (setf ifirst j) (go label70)))
        label40))
     (setf info (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) 1))
     (go label210)
    label50
     (setf ctemp
             (f2cl-lib:fref h-%data% (ilast ilast) ((1 ldh) (1 *))
                            h-%offset%))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (zlartg ctemp
          (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                                (ilast (f2cl-lib:int-sub ilast 1))
                                ((1 ldh) (1 *)) h-%offset%)
          c s
          (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                                (ilast ilast) ((1 ldh) (1 *))
                                h-%offset%))
       (declare (ignore var-1 var-4))
       (when var-0 (setf ctemp var-0))
       (when var-2 (setf c var-2))
       (when var-3 (setf s var-3)))
     (setf (f2cl-lib:fref h-%data% (ilast (f2cl-lib:int-sub ilast 1))
                          ((1 ldh) (1 *)) h-%offset%)
             czero)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (zrot (f2cl-lib:int-sub ilast ifrstm)
          (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                                (ifrstm ilast) ((1 ldh) (1 *))
                                h-%offset%)
          1
          (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
```

```
                                (ifrstm (f2cl-lib:int-sub ilast 1))
                                ((1 ldh) (1 *)) h-%offset%)
        1 c s)
      (declare (ignore var-0 var-1 var-2 var-3 var-4))
      (when var-5 (setf c var-5))
      (when var-6 (setf s var-6)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zrot (f2cl-lib:int-sub ilast ifrstm)
         (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16
                                (ifrstm ilast) ((1 ldt) (1 *))
                                t$-%offset%)
         1
         (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16
                                (ifrstm (f2cl-lib:int-sub ilast 1))
                                ((1 ldt) (1 *)) t$-%offset%)
         1 c s)
      (declare (ignore var-0 var-1 var-2 var-3 var-4))
      (when var-5 (setf c var-5))
      (when var-6 (setf s var-6)))
    (if ilz
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zrot n
             (f2cl-lib:array-slice z-%data% f2cl-lib:complex16
                                   (1 ilast) ((1 ldz) (1 *))
                                   z-%offset%)
             1
             (f2cl-lib:array-slice z-%data% f2cl-lib:complex16
                                   (1 (f2cl-lib:int-sub ilast 1))
                                   ((1 ldz) (1 *)) z-%offset%)
             1 c s)
           (declare (ignore var-1 var-2 var-3 var-4))
           (when var-0 (setf n var-0))
           (when var-5 (setf c var-5))
           (when var-6 (setf s var-6))))
  label60
    (setf absb
            (abs
             (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                            t$-%offset%)))
    (cond
     ((> absb safmin)
      (setf signbc
              (coerce
               (f2cl-lib:dconjg
                (/
                 (f2cl-lib:fref t$-%data% (ilast ilast)
                                ((1 ldt) (1 *)) t$-%offset%)
                 absb))
               'f2cl-lib:complex16))
```

```
  (setf (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                       t$-%offset%)
          (coerce absb 'f2cl-lib:complex16))
  (cond
   (ilschr
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zscal (f2cl-lib:int-sub ilast ifrstm) signbc
         (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16
                               (ifrstm ilast) ((1 ldt) (1 *))
                               t$-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3))
      (when var-1 (setf signbc var-1)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zscal
         (f2cl-lib:int-sub (f2cl-lib:int-add ilast 1) ifrstm)
         signbc
         (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                               (ifrstm ilast) ((1 ldh) (1 *))
                               h-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3))
      (when var-1 (setf signbc var-1))))
   (t
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zscal 1 signbc
         (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                               (ilast ilast) ((1 ldh) (1 *))
                               h-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3))
      (when var-1 (setf signbc var-1)))))
  (if ilz
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (zscal n signbc
           (f2cl-lib:array-slice z-%data% f2cl-lib:complex16
                                 (1 ilast) ((1 ldz) (1 *))
                                 z-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf n var-0))
        (when var-1 (setf signbc var-1)))))
 (t
  (setf (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                       t$-%offset%)
          czero)))
(setf (f2cl-lib:fref alpha-%data% (ilast) ((1 *)) alpha-%offset%)
        (f2cl-lib:fref h-%data% (ilast ilast) ((1 ldh) (1 *))
                       h-%offset%))
(setf (f2cl-lib:fref beta-%data% (ilast) ((1 *)) beta-%offset%)
```

```
          (f2cl-lib:fref t$-%data% (ilast ilast) ((1 ldt) (1 *))
                         t$-%offset%))
  (setf ilast (f2cl-lib:int-sub ilast 1))
  (if (< ilast ilo)
      (go label190))
  (setf iiter 0)
  (setf eshift czero)
  (cond
   ((not ilschr) (setf ilastm ilast)
    (if (> ifrstm ilast)
        (setf ifrstm ilo))))
  (go label160)
 label70
  (setf iiter (f2cl-lib:int-add iiter 1))
  (cond ((not ilschr) (setf ifrstm ifirst)))
  (cond
   ((/= (f2cl-lib:int-mul (f2cl-lib:f2cl/ iiter 10) 10) iiter)
    (setf u12
            (/
             (* bscale
                (f2cl-lib:fref t$-%data%
                               ((f2cl-lib:int-sub ilast 1) ilast)
                               ((1 ldt) (1 *)) t$-%offset%))
             (* bscale
                (f2cl-lib:fref t$-%data% (ilast ilast)
                               ((1 ldt) (1 *)) t$-%offset%))))
    (setf ad11
            (/
             (* ascale
                (f2cl-lib:fref h-%data%
                               ((f2cl-lib:int-sub ilast 1)
                                (f2cl-lib:int-sub ilast 1))
                               ((1 ldh) (1 *)) h-%offset%))
             (* bscale
                (f2cl-lib:fref t$-%data%
                               ((f2cl-lib:int-sub ilast 1)
                                (f2cl-lib:int-sub ilast 1))
                               ((1 ldt) (1 *)) t$-%offset%))))
    (setf ad21
            (/
             (* ascale
                (f2cl-lib:fref h-%data%
                               (ilast (f2cl-lib:int-sub ilast 1))
                               ((1 ldh) (1 *)) h-%offset%))
             (* bscale
                (f2cl-lib:fref t$-%data%
                               ((f2cl-lib:int-sub ilast 1)
                                (f2cl-lib:int-sub ilast 1))
                               ((1 ldt) (1 *)) t$-%offset%))))
    (setf ad12
```

```
            (/
             (* ascale
                (f2cl-lib:fref h-%data%
                               ((f2cl-lib:int-sub ilast 1) ilast)
                               ((1 ldh) (1 *)) h-%offset%))
             (* bscale
                (f2cl-lib:fref t$-%data% (ilast ilast)
                               ((1 ldt) (1 *)) t$-%offset%))))
    (setf ad22
            (/
             (* ascale
                (f2cl-lib:fref h-%data% (ilast ilast)
                               ((1 ldh) (1 *)) h-%offset%))
             (* bscale
                (f2cl-lib:fref t$-%data% (ilast ilast)
                               ((1 ldt) (1 *)) t$-%offset%))))
    (setf abi22 (- ad22 (* u12 ad21)))
    (setf t1 (* half (+ ad11 abi22)))
    (setf rtdisc
            (f2cl-lib:fsqrt
             (- (+ (expt t1 2) (* ad12 ad21)) (* ad11 ad22))))
    (setf temp
            (+
             (* (f2cl-lib:dble (- t1 abi22))
                (f2cl-lib:dble rtdisc))
             (* (f2cl-lib:dimag (- t1 abi22))
                (f2cl-lib:dimag rtdisc))))
    (cond ((<= temp zero) (setf shift (+ t1 rtdisc)))
          (t (setf shift (- t1 rtdisc)))))
   (t
    (setf eshift
            (+ eshift
               (/
                (* ascale
                   (f2cl-lib:fref h-%data%
                                  (ilast
                                   (f2cl-lib:int-sub ilast 1))
                                  ((1 ldh) (1 *)) h-%offset%))
                (* bscale
                   (f2cl-lib:fref t$-%data%
                                  ((f2cl-lib:int-sub ilast 1)
                                   (f2cl-lib:int-sub ilast 1))
                                  ((1 ldt) (1 *)) t$-%offset%)))))
    (setf shift eshift)))
  (f2cl-lib:fdo (j (f2cl-lib:int-add ilast (f2cl-lib:int-sub 1))
                 (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                ((> j (f2cl-lib:int-add ifirst 1)) nil)
    (tagbody
      (setf istart j)
      (setf ctemp
```

```
                (-
                 (* ascale
                    (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *))
                                   h-%offset%))
                 (* shift
                    (* bscale
                       (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                                      t$-%offset%)))))
        (setf temp (abs1 ctemp))
        (setf temp2
                (* ascale
                   (abs1
                    (f2cl-lib:fref h-%data%
                                   ((f2cl-lib:int-add j 1) j)
                                   ((1 ldh) (1 *)) h-%offset%))))
        (setf tempr (max temp temp2))
        (cond
         ((and (< tempr one) (/= tempr zero))
          (setf temp (/ temp tempr)) (setf temp2 (/ temp2 tempr))))
        (if (<=
             (*
              (abs1
               (f2cl-lib:fref h-%data% (j (f2cl-lib:int-sub j 1))
                              ((1 ldh) (1 *)) h-%offset%))
              temp2)
             (* temp atol))
            (go label90))
       label80))
    (setf istart ifirst)
    (setf ctemp
            (-
             (* ascale
                (f2cl-lib:fref h-%data% (ifirst ifirst)
                               ((1 ldh) (1 *)) h-%offset%))
             (* shift
                (* bscale
                   (f2cl-lib:fref t$-%data% (ifirst ifirst)
                                  ((1 ldt) (1 *)) t$-%offset%)))))
  label90
    (setf ctemp2
            (* ascale
               (f2cl-lib:fref h-%data%
                              ((f2cl-lib:int-add istart 1) istart)
                              ((1 ldh) (1 *)) h-%offset%)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zlartg ctemp ctemp2 c s ctemp3)
      (declare (ignore))
      (when var-0 (setf ctemp var-0))
      (when var-1 (setf ctemp2 var-1))
      (when var-2 (setf c var-2))
```

```
    (when var-3 (setf s var-3))
    (when var-4 (setf ctemp3 var-4)))
  (f2cl-lib:fdo (j istart (f2cl-lib:int-add j 1))
                ((> j
                    (f2cl-lib:int-add ilast (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (cond
       ((> j istart)
        (setf ctemp
                (f2cl-lib:fref h-%data% (j (f2cl-lib:int-sub j 1))
                               ((1 ldh) (1 *)) h-%offset%))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (zlartg ctemp
             (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                                   ((+ j 1) (f2cl-lib:int-sub j 1))
                                   ((1 ldh) (1 *)) h-%offset%)
             c s
             (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                                   (j (f2cl-lib:int-sub j 1))
                                   ((1 ldh) (1 *)) h-%offset%))
          (declare (ignore var-1 var-4))
          (when var-0 (setf ctemp var-0))
          (when var-2 (setf c var-2))
          (when var-3 (setf s var-3)))
        (setf (f2cl-lib:fref h-%data%
                             ((f2cl-lib:int-add j 1)
                              (f2cl-lib:int-sub j 1))
                             ((1 ldh) (1 *)) h-%offset%)
                czero)))
      (f2cl-lib:fdo (jc j (f2cl-lib:int-add jc 1))
                    ((> jc ilastm) nil)
        (tagbody
          (setf ctemp
                  (+
                   (* c
                      (f2cl-lib:fref h-%data% (j jc)
                                     ((1 ldh) (1 *)) h-%offset%))
                   (* s
                      (f2cl-lib:fref h-%data%
                                     ((f2cl-lib:int-add j 1) jc)
                                     ((1 ldh) (1 *)) h-%offset%))))
          (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add j 1) jc)
                               ((1 ldh) (1 *)) h-%offset%)
                  (+
                   (* (- (f2cl-lib:dconjg s))
                      (f2cl-lib:fref h-%data% (j jc)
                                     ((1 ldh) (1 *)) h-%offset%))
                   (* c
                      (f2cl-lib:fref h-%data%
```

```
                                ((f2cl-lib:int-add j 1) jc)
                                ((1 ldh) (1 *)) h-%offset%))))
        (setf (f2cl-lib:fref h-%data% (j jc) ((1 ldh) (1 *))
                             h-%offset%)
              ctemp)
        (setf ctemp2
              (+
               (* c
                  (f2cl-lib:fref t$-%data% (j jc)
                                 ((1 ldt) (1 *)) t$-%offset%))
               (* s
                  (f2cl-lib:fref t$-%data%
                                 ((f2cl-lib:int-add j 1) jc)
                                 ((1 ldt) (1 *))
                                 t$-%offset%))))
        (setf (f2cl-lib:fref t$-%data%
                             ((f2cl-lib:int-add j 1) jc)
                             ((1 ldt) (1 *)) t$-%offset%)
              (+
               (* (- (f2cl-lib:dconjg s))
                  (f2cl-lib:fref t$-%data% (j jc)
                                 ((1 ldt) (1 *)) t$-%offset%))
               (* c
                  (f2cl-lib:fref t$-%data%
                                 ((f2cl-lib:int-add j 1) jc)
                                 ((1 ldt) (1 *))
                                 t$-%offset%))))
        (setf (f2cl-lib:fref t$-%data% (j jc) ((1 ldt) (1 *))
                             t$-%offset%)
              ctemp2)
      label100))
    (cond
     (ilq
      (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                    ((> jr n) nil)
        (tagbody
          (setf ctemp
                (+
                 (* c
                    (f2cl-lib:fref q-%data% (jr j)
                                   ((1 ldq) (1 *)) q-%offset%))
                 (* (f2cl-lib:dconjg s)
                    (f2cl-lib:fref q-%data%
                                   (jr (f2cl-lib:int-add j 1))
                                   ((1 ldq) (1 *))
                                   q-%offset%))))
          (setf (f2cl-lib:fref q-%data%
                               (jr (f2cl-lib:int-add j 1))
                               ((1 ldq) (1 *)) q-%offset%)
                (+
```

```
                (* (- s)
                   (f2cl-lib:fref q-%data% (jr j)
                                  ((1 ldq) (1 *)) q-%offset%))
                (* c
                   (f2cl-lib:fref q-%data%
                                  (jr (f2cl-lib:int-add j 1))
                                  ((1 ldq) (1 *))
                                  q-%offset%))))
      (setf (f2cl-lib:fref q-%data% (jr j) ((1 ldq) (1 *))
                           q-%offset%)
              ctemp)
     label110))))
(setf ctemp
        (f2cl-lib:fref t$-%data%
                       ((f2cl-lib:int-add j 1)
                        (f2cl-lib:int-add j 1))
                       ((1 ldt) (1 *)) t$-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (zlartg ctemp
     (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16
                           ((+ j 1) j) ((1 ldt) (1 *))
                           t$-%offset%)
     c s
     (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16
                           ((+ j 1) (f2cl-lib:int-add j 1))
                           ((1 ldt) (1 *)) t$-%offset%))
  (declare (ignore var-1 var-4))
  (when var-0 (setf ctemp var-0))
  (when var-2 (setf c var-2))
  (when var-3 (setf s var-3)))
(setf (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add j 1) j)
                     ((1 ldt) (1 *)) t$-%offset%)
        czero)
(f2cl-lib:fdo (jr ifrstm (f2cl-lib:int-add jr 1))
              ((> jr
                  (min
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add j 2))
                   (the f2cl-lib:integer4 ilast)))
               nil)
  (tagbody
    (setf ctemp
            (+
             (* c
                (f2cl-lib:fref h-%data%
                               (jr (f2cl-lib:int-add j 1))
                               ((1 ldh) (1 *)) h-%offset%))
             (* s
                (f2cl-lib:fref h-%data% (jr j)
                               ((1 ldh) (1 *)) h-%offset%))))
```

```
    (setf (f2cl-lib:fref h-%data% (jr j) ((1 ldh) (1 *))
                         h-%offset%)
            (+
             (* (- (f2cl-lib:dconjg s))
                (f2cl-lib:fref h-%data%
                               (jr (f2cl-lib:int-add j 1))
                               ((1 ldh) (1 *)) h-%offset%))
             (* c
                (f2cl-lib:fref h-%data% (jr j)
                               ((1 ldh) (1 *)) h-%offset%))))
    (setf (f2cl-lib:fref h-%data% (jr (f2cl-lib:int-add j 1))
                         ((1 ldh) (1 *)) h-%offset%)
            ctemp)
   label120))
(f2cl-lib:fdo (jr ifrstm (f2cl-lib:int-add jr 1))
              ((> jr j) nil)
  (tagbody
    (setf ctemp
            (+
             (* c
                (f2cl-lib:fref t$-%data%
                               (jr (f2cl-lib:int-add j 1))
                               ((1 ldt) (1 *)) t$-%offset%))
             (* s
                (f2cl-lib:fref t$-%data% (jr j)
                               ((1 ldt) (1 *))
                               t$-%offset%))))
    (setf (f2cl-lib:fref t$-%data% (jr j) ((1 ldt) (1 *))
                         t$-%offset%)
            (+
             (* (- (f2cl-lib:dconjg s))
                (f2cl-lib:fref t$-%data%
                               (jr (f2cl-lib:int-add j 1))
                               ((1 ldt) (1 *)) t$-%offset%))
             (* c
                (f2cl-lib:fref t$-%data% (jr j)
                               ((1 ldt) (1 *))
                               t$-%offset%))))
    (setf (f2cl-lib:fref t$-%data%
                         (jr (f2cl-lib:int-add j 1))
                         ((1 ldt) (1 *)) t$-%offset%)
            ctemp)
   label130))
(cond
 (ilz
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr n) nil)
    (tagbody
      (setf ctemp
              (+
```

```
                          (* c
                             (f2cl-lib:fref z-%data%
                                            (jr (f2cl-lib:int-add j 1))
                                            ((1 ldz) (1 *)) z-%offset%))
                          (* s
                             (f2cl-lib:fref z-%data% (jr j)
                                            ((1 ldz) (1 *))
                                            z-%offset%))))
                 (setf (f2cl-lib:fref z-%data% (jr j) ((1 ldz) (1 *))
                                      z-%offset%)
                         (+
                          (* (- (f2cl-lib:dconjg s))
                             (f2cl-lib:fref z-%data%
                                            (jr (f2cl-lib:int-add j 1))
                                            ((1 ldz) (1 *)) z-%offset%))
                          (* c
                             (f2cl-lib:fref z-%data% (jr j)
                                            ((1 ldz) (1 *))
                                            z-%offset%))))
                 (setf (f2cl-lib:fref z-%data%
                                      (jr (f2cl-lib:int-add j 1))
                                      ((1 ldz) (1 *)) z-%offset%)
                         ctemp)
               label140))))
        label150))
    label160
    label170))
label180
 (setf info ilast)
 (go label210)
label190
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))) nil)
   (tagbody
     (setf absb
             (abs
              (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                             t$-%offset%)))
     (cond
      ((> absb safmin)
       (setf signbc
               (coerce
                (f2cl-lib:dconjg
                 (/
                  (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                                 t$-%offset%)
                  absb))
                'f2cl-lib:complex16))
       (setf (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                            t$-%offset%)
```

```
                (coerce absb 'f2cl-lib:complex16))
          (cond
           (ilschr
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (zscal (f2cl-lib:int-sub j 1) signbc
                 (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 j)
                                       ((1 ldt) (1 *)) t$-%offset%)
                 1)
              (declare (ignore var-0 var-2 var-3))
              (when var-1 (setf signbc var-1)))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (zscal j signbc
                 (f2cl-lib:array-slice h-%data% f2cl-lib:complex16 (1 j)
                                       ((1 ldh) (1 *)) h-%offset%)
                 1)
              (declare (ignore var-2 var-3))
              (when var-0 (setf j var-0))
              (when var-1 (setf signbc var-1))))
           (t
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (zscal 1 signbc
                 (f2cl-lib:array-slice h-%data% f2cl-lib:complex16 (j j)
                                       ((1 ldh) (1 *)) h-%offset%)
                 1)
              (declare (ignore var-0 var-2 var-3))
              (when var-1 (setf signbc var-1)))))
          (if ilz
              (multiple-value-bind (var-0 var-1 var-2 var-3)
                  (zscal n signbc
                   (f2cl-lib:array-slice z-%data% f2cl-lib:complex16
                                         (1 j) ((1 ldz) (1 *))
                                         z-%offset%)
                   1)
                (declare (ignore var-2 var-3))
                (when var-0 (setf n var-0))
                (when var-1 (setf signbc var-1)))))
         (t
          (setf (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                               t$-%offset%)
                czero)))
        (setf (f2cl-lib:fref alpha-%data% (j) ((1 *)) alpha-%offset%)
                (f2cl-lib:fref h-%data% (j j) ((1 ldh) (1 *))
                               h-%offset%))
        (setf (f2cl-lib:fref beta-%data% (j) ((1 *)) beta-%offset%)
                (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                               t$-%offset%))
      label200))
   (setf info 0)
  label210
   (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
```

```
                 (f2cl-lib:dcmplx n))
          (go end_label)
         end_label
          (return
           (values job compq compz n nil nil nil ldh nil ldt nil nil nil ldq
                   nil ldz nil nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zhgeqz
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::job
                                              fortran-to-lisp::compq
                                              fortran-to-lisp::compz
                                              fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::ldh nil
                                              fortran-to-lisp::ldt nil nil nil
                                              fortran-to-lisp::ldq nil
                                              fortran-to-lisp::ldz nil nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

35.34 zla_geamv

35.34.1 Axiom unit tests

— **zlageamv.input** —

```
)set break resume
)sys rm -f zlageamv.output
)spool zlageamv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.34.2 Axiom help page

— **zla_geamv.help** —

```
====================================================================
zla_geamv examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zla_geamv (TRANS, M, N, ALPHA, A, LDA, X, INCX, BETA, Y,
           INCY)
           ZLA_GEAMV computes a matrix-vector product using a general matrix
           to calculate error bounds.

Function/Subroutine Documentation
   subroutine zla_geamv (integer TRANS, integer M, integer N, double precision
       ALPHA, complex*16, dimension( lda, * ) A, integer LDA, complex*16,
```

```
      dimension( * ) X, integer INCX, double precision BETA, double
      precision, dimension( * ) Y, integer INCY)
      ZLA_GEAMV computes a matrix-vector product using a general matrix to
      calculate error bounds.

Purpose:
   ZLA_GEAMV  performs one of the matrix-vector operations

           y := alpha*abs(A)*abs(x) + beta*abs(y),
      or   y := alpha*abs(A)**T*abs(x) + beta*abs(y),

   where alpha and beta are scalars, x and y are vectors and A is an
   m by n matrix.

   This function is primarily used in calculating error bounds.
   To protect against underflow during evaluation, components in
   the resulting vector are perturbed away from zero by (N+1)
   times the underflow threshold.  To prevent unnecessarily large
   errors for block-structure embedded in general matrices,
   "symbolically" zero components are not perturbed.  A zero
   entry is considered "symbolic" if all multiplications involved
   in computing that entry have at least one zero multiplicand.

Parameters:

 TRANS
   TRANS is INTEGER
   On entry, TRANS specifies the operation to be performed as
   follows:

   BLAS_NO_TRANS      y := alpha*abs(A)*abs(x) + beta*abs(y)
   BLAS_TRANS         y := alpha*abs(A**T)*abs(x) + beta*abs(y)
   BLAS_CONJ_TRANS    y := alpha*abs(A**T)*abs(x) + beta*abs(y)

   Unchanged on exit.

 M
   M is INTEGER
   On entry, M specifies the number of rows of the matrix A.
   M must be at least zero.
   Unchanged on exit.

 N
   N is INTEGER
   On entry, N specifies the number of columns of the matrix A.
   N must be at least zero.
   Unchanged on exit.

 ALPHA
   ALPHA is DOUBLE PRECISION
```

```
  On entry, ALPHA specifies the scalar alpha.
  Unchanged on exit.

A
  A is COMPLEX*16 array of DIMENSION ( LDA, n )
  Before entry, the leading m by n part of the array A must
  contain the matrix of coefficients.
  Unchanged on exit.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, m ).
  Unchanged on exit.

X
  X is COMPLEX*16 array of DIMENSION at least
     ( 1 + ( n - 1 )*abs( INCX ) ) when TRANS = 'N' or 'n'
  and at least
     ( 1 + ( m - 1 )*abs( INCX ) ) otherwise.
  Before entry, the incremented array X must contain the vector x.
  Unchanged on exit.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of
  X. INCX must not be zero.
  Unchanged on exit.

BETA
  BETA is DOUBLE PRECISION
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.
  Unchanged on exit.

Y
  Y is DOUBLE PRECISION array, dimension
     ( 1 + ( m - 1 )*abs( INCY ) ) when TRANS = 'N' or 'n'
  and at least
     ( 1 + ( n - 1 )*abs( INCY ) ) otherwise.
  Before entry with BETA non-zero, the incremented array Y
  must contain the vector y. On exit, Y is overwritten by the
  updated vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of
  Y. INCY must not be zero.
  Unchanged on exit.
```

———-

35.34.3 fortran code

— zla_geamv.f —

```
*  ===================================================================
      SUBROUTINE ZLA_GEAMV ( TRANS, M, N, ALPHA, A, LDA, X, INCX, BETA,
     $                       Y, INCY )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   ALPHA, BETA
      INTEGER            INCX, INCY, LDA, M, N
      INTEGER            TRANS
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), X( * )
      DOUBLE PRECISION   Y( * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            SYMB_ZERO
      DOUBLE PRECISION   TEMP, SAFE1
      INTEGER            I, INFO, IY, J, JX, KX, KY, LENX, LENY
      COMPLEX*16         CDUM
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, DLAMCH
      DOUBLE PRECISION   DLAMCH
*     ..
*     .. External Functions ..
      EXTERNAL           ILATRANS
      INTEGER            ILATRANS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, ABS, REAL, DIMAG, SIGN
*     ..
```

```
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( CDUM ) = ABS( DBLE( CDUM ) ) + ABS( DIMAG( CDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF     ( .NOT.( ( TRANS.EQ.ILATRANS( 'N' ) )
     $           .OR. ( TRANS.EQ.ILATRANS( 'T' ) )
     $           .OR. ( TRANS.EQ.ILATRANS( 'C' ) ) ) ) THEN
         INFO = 1
      ELSE IF( M.LT.0 )THEN
         INFO = 2
      ELSE IF( N.LT.0 )THEN
         INFO = 3
      ELSE IF( LDA.LT.MAX( 1, M ) )THEN
         INFO = 6
      ELSE IF( INCX.EQ.0 )THEN
         INFO = 8
      ELSE IF( INCY.EQ.0 )THEN
         INFO = 11
      END IF
      IF( INFO.NE.0 )THEN
         CALL XERBLA( 'ZLA_GEAMV ', INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( ( M.EQ.0 ).OR.( N.EQ.0 ).OR.
     $    ( ( ALPHA.EQ.ZERO ).AND.( BETA.EQ.ONE ) ) )
     $   RETURN
*
*     Set  LENX  and  LENY, the lengths of the vectors x and y, and set
*     up the start points in  X  and  Y.
*
      IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
         LENX = N
         LENY = M
      ELSE
         LENX = M
         LENY = N
      END IF
      IF( INCX.GT.0 )THEN
         KX = 1
      ELSE
```

```
         KX = 1 - ( LENX - 1 )*INCX
      END IF
      IF( INCY.GT.0 )THEN
         KY = 1
      ELSE
         KY = 1 - ( LENY - 1 )*INCY
      END IF
*
*     Set SAFE1 essentially to be the underflow threshold times the
*     number of additions in each row.
*
      SAFE1 = DLAMCH( 'Safe minimum' )
      SAFE1 = (N+1)*SAFE1
*
*     Form  y := alpha*abs(A)*abs(x) + beta*abs(y).
*
*     The O(M*N) SYMB_ZERO tests could be replaced by O(N) queries to
*     the inexact flag.  Still doesn't help change the iteration order
*     to per-column.
*
      IY = KY
      IF ( INCX.EQ.1 ) THEN
         IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
            DO I = 1, LENY
               IF ( BETA .EQ. 0.0D+0 ) THEN
                  SYMB_ZERO = .TRUE.
                  Y( IY ) = 0.0D+0
               ELSE IF ( Y( IY ) .EQ. 0.0D+0 ) THEN
                  SYMB_ZERO = .TRUE.
               ELSE
                  SYMB_ZERO = .FALSE.
                  Y( IY ) = BETA * ABS( Y( IY ) )
               END IF
               IF ( ALPHA .NE. 0.0D+0 ) THEN
                  DO J = 1, LENX
                     TEMP = CABS1( A( I, J ) )
                     SYMB_ZERO = SYMB_ZERO .AND.
     $                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                     Y( IY ) = Y( IY ) + ALPHA*CABS1( X( J ) )*TEMP
                  END DO
               END IF

               IF ( .NOT.SYMB_ZERO ) Y( IY ) =
     $              Y( IY ) + SIGN( SAFE1, Y( IY ) )

               IY = IY + INCY
            END DO
         ELSE
            DO I = 1, LENY
```

```
            IF ( BETA .EQ. 0.0D+0 ) THEN
               SYMB_ZERO = .TRUE.
               Y( IY ) = 0.0D+0
            ELSE IF ( Y( IY ) .EQ. 0.0D+0 ) THEN
               SYMB_ZERO = .TRUE.
            ELSE
               SYMB_ZERO = .FALSE.
               Y( IY ) = BETA * ABS( Y( IY ) )
            END IF
            IF ( ALPHA .NE. 0.0D+0 ) THEN
               DO J = 1, LENX
                  TEMP = CABS1( A( J, I ) )
                  SYMB_ZERO = SYMB_ZERO .AND.
$                      ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                  Y( IY ) = Y( IY ) + ALPHA*CABS1( X( J ) )*TEMP
               END DO
            END IF

            IF ( .NOT.SYMB_ZERO ) Y( IY ) =
$                Y( IY ) + SIGN( SAFE1, Y( IY ) )

            IY = IY + INCY
         END DO
      END IF
   ELSE
      IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
         DO I = 1, LENY
            IF ( BETA .EQ. 0.0D+0 ) THEN
               SYMB_ZERO = .TRUE.
               Y( IY ) = 0.0D+0
            ELSE IF ( Y( IY ) .EQ. 0.0D+0 ) THEN
               SYMB_ZERO = .TRUE.
            ELSE
               SYMB_ZERO = .FALSE.
               Y( IY ) = BETA * ABS( Y( IY ) )
            END IF
            IF ( ALPHA .NE. 0.0D+0 ) THEN
               JX = KX
               DO J = 1, LENX
                  TEMP = CABS1( A( I, J ) )
                  SYMB_ZERO = SYMB_ZERO .AND.
$                      ( X( JX ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                  Y( IY ) = Y( IY ) + ALPHA*CABS1( X( JX ) )*TEMP
                  JX = JX + INCX
               END DO
            END IF

            IF ( .NOT.SYMB_ZERO ) Y( IY ) =
```

```
     $              Y( IY ) + SIGN( SAFE1, Y( IY ) )

               IY = IY + INCY
            END DO
         ELSE
            DO I = 1, LENY
               IF ( BETA .EQ. 0.0D+0 ) THEN
                  SYMB_ZERO = .TRUE.
                  Y( IY ) = 0.0D+0
               ELSE IF ( Y( IY ) .EQ. 0.0D+0 ) THEN
                  SYMB_ZERO = .TRUE.
               ELSE
                  SYMB_ZERO = .FALSE.
                  Y( IY ) = BETA * ABS( Y( IY ) )
               END IF
               IF ( ALPHA .NE. 0.0D+0 ) THEN
                  JX = KX
                  DO J = 1, LENX
                     TEMP = CABS1( A( J, I ) )
                     SYMB_ZERO = SYMB_ZERO .AND.
     $                    ( X( JX ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                     Y( IY ) = Y( IY ) + ALPHA*CABS1( X( JX ) )*TEMP
                     JX = JX + INCX
                  END DO
               END IF

               IF ( .NOT.SYMB_ZERO ) Y( IY ) =
     $              Y( IY ) + SIGN( SAFE1, Y( IY ) )

               IY = IY + INCY
            END DO
         END IF

      END IF
*
      RETURN
*
*     End of ZLA_GEAMV
*
      END
```

———-

35.34.4 lisp code

— zla_geamv.lisp —

```
(let* ((one (coerce 1.0d0 'f2cl-lib:complex16))
       (zero (coerce 0.0d0 'f2cl-lib:complex16)))
  (declare (type (f2cl-lib:complex16) one)
           (type (f2cl-lib:complex16) zero)
           (ignorable one zero))
  (defun zla_geamv (trans m n alpha a lda x incx beta y incy)
    (declare (type (array double-float (*)) y)
             (type (array f2cl-lib:complex16 (*)) x a)
             (type (double-float) beta alpha)
             (type (f2cl-lib:integer4) incy incx lda n m trans))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (x f2cl-lib:complex16 x-%data% x-%offset%)
         (y double-float y-%data% y-%offset%))
      (labels ((cabs1 (cdum)
                 (+ (abs (f2cl-lib:dble cdum)) (abs (f2cl-lib:dimag cdum)))))
        (declare
         (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
          cabs1))
        (prog ((cdum #C(0.0d0 0.0d0)) (i 0) (info 0) (iy 0) (j 0) (jx 0) (kx 0)
               (ky 0) (lenx 0) (leny 0) (temp 0.0d0) (safe1 0.0d0)
               (symb_zero nil))
          (declare (type (f2cl-lib:complex16) cdum)
                   (type (f2cl-lib:integer4) i info iy j jx kx ky lenx leny)
                   (type (double-float) temp safe1)
                   (type f2cl-lib:logical symb_zero))
          (setf info 0)
          (cond
           ((not
             (or (= trans (ilatrans "N")) (= trans (ilatrans "T"))
                 (= trans (ilatrans "C"))))
            (setf info 1))
           ((< m 0) (setf info 2)) ((< n 0) (setf info 3))
           ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
            (setf info 6))
           ((= incx 0) (setf info 8)) ((= incy 0) (setf info 11)))
          (cond ((/= info 0) (xerbla "ZLA_GEAMV " info) (go end_label)))
          (if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
              (go end_label))
          (cond ((= trans (ilatrans "N")) (setf lenx n) (setf leny m))
                (t (setf lenx m) (setf leny n)))
          (cond ((> incx 0) (setf kx 1))
                (t
                 (setf kx
                         (f2cl-lib:int-sub 1
                                           (f2cl-lib:int-mul
                                            (f2cl-lib:int-sub lenx 1) incx)))))
          (cond ((> incy 0) (setf ky 1))
                (t
                 (setf ky
```

```
                (f2cl-lib:int-sub 1
                                  (f2cl-lib:int-mul
                                   (f2cl-lib:int-sub leny 1) incy)))))
(setf safe1 (dlamch "Safe minimum"))
(setf safe1 (* (f2cl-lib:int-add n 1) safe1))
(setf iy ky)
(cond
 ((= incx 1)
  (cond
   ((= trans (ilatrans "N"))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i leny) nil)
      (tagbody
        (cond
         ((= beta 0.0d0) (setf symb_zero f2cl-lib:%true%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  0.0d0))
         ((= (f2cl-lib:fref y (iy) ((1 *))) 0.0d0)
          (setf symb_zero f2cl-lib:%true%))
         (t (setf symb_zero f2cl-lib:%false%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (* beta
                     (abs
                      (f2cl-lib:fref y-%data% (iy) ((1 *))
                                     y-%offset%))))))
        (cond
         ((/= alpha 0.0d0)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j lenx) nil)
            (tagbody
              (setf temp
                      (cabs1
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)))
              (setf symb_zero
                      (and symb_zero
                           (or
                            (=
                             (f2cl-lib:fref x-%data% (j) ((1 *))
                                            x-%offset%)
                             zero)
                            (= temp zero))))
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (+
                       (f2cl-lib:fref y-%data% (iy) ((1 *))
                                      y-%offset%)
                       (* alpha
                          (cabs1
                           (f2cl-lib:fref x-%data% (j) ((1 *))
                                          x-%offset%))
```

```
                        temp)))
             label100001))))
        (if (not symb_zero)
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+
                     (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (f2cl-lib:sign safe1
                                    (f2cl-lib:fref y-%data% (iy)
                                                   ((1 *))
                                                   y-%offset%)))))
        (setf iy (f2cl-lib:int-add iy incy))
       label100000)))
   (t
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i leny) nil)
      (tagbody
        (cond
         ((= beta 0.0d0) (setf symb_zero f2cl-lib:%true%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  0.0d0))
         ((= (f2cl-lib:fref y (iy) ((1 *))) 0.0d0)
          (setf symb_zero f2cl-lib:%true%))
         (t (setf symb_zero f2cl-lib:%false%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (* beta
                     (abs
                      (f2cl-lib:fref y-%data% (iy) ((1 *))
                                     y-%offset%))))))
        (cond
         ((/= alpha 0.0d0)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j lenx) nil)
            (tagbody
              (setf temp
                      (cabs1
                       (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                      a-%offset%)))
              (setf symb_zero
                      (and symb_zero
                           (or
                            (=
                             (f2cl-lib:fref x-%data% (j) ((1 *))
                                            x-%offset%)
                             zero)
                            (= temp zero))))
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (+
                       (f2cl-lib:fref y-%data% (iy) ((1 *))
                                      y-%offset%)
                       (* alpha
```

```
                              (cabs1
                               (f2cl-lib:fref x-%data% (j) ((1 *))
                                              x-%offset%))
                              temp)))
                 label100003))))
            (if (not symb_zero)
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (f2cl-lib:sign safe1
                                        (f2cl-lib:fref y-%data% (iy)
                                                       ((1 *))
                                                       y-%offset%)))))
            (setf iy (f2cl-lib:int-add iy incy))
           label100002)))))
      (t
       (cond
        ((= trans (ilatrans "N"))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i leny) nil)
           (tagbody
             (cond
              ((= beta 0.0d0) (setf symb_zero f2cl-lib:%true%)
               (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     0.0d0))
              ((= (f2cl-lib:fref y (iy) ((1 *))) 0.0d0)
               (setf symb_zero f2cl-lib:%true%))
              (t (setf symb_zero f2cl-lib:%false%)
               (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (* beta
                        (abs
                         (f2cl-lib:fref y-%data% (iy) ((1 *))
                                        y-%offset%))))))
             (cond
              ((/= alpha 0.0d0) (setf jx kx)
               (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                             ((> j lenx) nil)
                 (tagbody
                   (setf temp
                           (cabs1
                            (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                           a-%offset%)))
                   (setf symb_zero
                           (and symb_zero
                                (or
                                 (=
                                  (f2cl-lib:fref x-%data% (jx) ((1 *))
                                                 x-%offset%)
                                  zero)
                                 (= temp zero))))
```

```
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data% (iy) ((1 *))
                                        y-%offset%)
                         (* alpha
                            (cabs1
                             (f2cl-lib:fref x-%data% (jx) ((1 *))
                                            x-%offset%))
                            temp)))
                (setf jx (f2cl-lib:int-add jx incx))
               label100005))))
          (if (not symb_zero)
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (+
                       (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (f2cl-lib:sign safe1
                                      (f2cl-lib:fref y-%data% (iy)
                                                     ((1 *))
                                                     y-%offset%)))))
          (setf iy (f2cl-lib:int-add iy incy))
         label100004)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i leny) nil)
        (tagbody
          (cond
           ((= beta 0.0d0) (setf symb_zero f2cl-lib:%true%)
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    0.0d0))
           ((= (f2cl-lib:fref y (iy) ((1 *))) 0.0d0)
            (setf symb_zero f2cl-lib:%true%))
           (t (setf symb_zero f2cl-lib:%false%)
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (* beta
                       (abs
                        (f2cl-lib:fref y-%data% (iy) ((1 *))
                                       y-%offset%))))))
          (cond
           ((/= alpha 0.0d0) (setf jx kx)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j lenx) nil)
              (tagbody
                (setf temp
                        (cabs1
                         (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                        a-%offset%)))
                (setf symb_zero
                        (and symb_zero
                             (or
                              (=
```

```
                                    (f2cl-lib:fref x-%data% (jx) ((1 *))
                                                   x-%offset%)
                                    zero)
                                   (= temp zero))))
                        (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                                (+
                                 (f2cl-lib:fref y-%data% (iy) ((1 *))
                                                y-%offset%)
                                 (* alpha
                                    (cabs1
                                     (f2cl-lib:fref x-%data% (jx) ((1 *))
                                                    x-%offset%))
                                    temp)))
                        (setf jx (f2cl-lib:int-add jx incx))
                       label100007))))
                  (if (not symb_zero)
                      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                              (+
                               (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                               (f2cl-lib:sign safe1
                                              (f2cl-lib:fref y-%data% (iy)
                                                             ((1 *))
                                                             y-%offset%)))))
                  (setf iy (f2cl-lib:int-add iy incy))
                 label100006))))))
          (go end_label)
         end_label
          (return (values nil nil nil nil nil nil nil nil nil nil nil)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zla_geamv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (double-float)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
```

```
                                          nil nil nil)
                                        :calls
                                        '(fortran-to-lisp::xerbla
                                          fortran-to-lisp::ilatrans))))
```

———-

35.35 zla_gercond_c

35.35.1 Axiom unit tests

— **zlagercond.input** —

```
)set break resume
)sys rm -f zlagercond.output
)spool zlagercond.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.35.2 Axiom help page

— **zla_gercond_c.help** —

```
==================================================================
zla_gercond examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function zla_gercond_c (TRANS, N, A, LDA, AF, LDAF,
           IPIV, C, CAPPLY, INFO, WORK, RWORK)
           ZLA_GERCOND_C computes the infinity norm condition number of
```

```
          op(A)*inv(diag(c)) for general matrices.

Function/Subroutine Documentation
   double precision function zla_gercond_c (character TRANS, integer N,
       complex*16, dimension( lda, * ) A, integer LDA, complex*16, dimension(
       ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV, double
       precision, dimension( * ) C, logical CAPPLY, integer INFO, complex*16,
       dimension( * ) WORK, double precision, dimension( * ) RWORK)
       ZLA_GERCOND_C computes the infinity norm condition number of
       op(A)*inv(diag(c)) for general matrices.

Purpose:
   ZLA_GERCOND_C computes the infinity norm condition number of
   op(A) * inv(diag(C)) where C is a DOUBLE PRECISION vector.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate Transpose = Transpose)

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the N-by-N matrix A

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is COMPLEX*16 array, dimension (LDAF,N)
   The factors L and U from the factorization
   A = P*L*U as computed by ZGETRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   The pivot indices from the factorization A = P*L*U
   as computed by ZGETRF; row i of the matrix was interchanged
```

```
  with row IPIV(i).

C
  C is DOUBLE PRECISION array, dimension (N)
  The vector C in the formula op(A) * inv(diag(C)).

CAPPLY
  CAPPLY is LOGICAL
  If .TRUE. then access the vector C in the formula above.

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  i > 0:  The ith argument is invalid.

WORK
  WORK is COMPLEX*16 array, dimension (2*N).
  Workspace.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (N).
  Workspace.
```

———-

35.35.3 fortran code

— zla_gercond_c.f —

```
*  ====================================================================
      DOUBLE PRECISION FUNCTION ZLA_GERCOND_C( TRANS, N, A, LDA, AF,
     $                                         LDAF, IPIV, C, CAPPLY,
     $                                         INFO, WORK, RWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Aguments ..
      CHARACTER          TRANS
      LOGICAL            CAPPLY
      INTEGER            N, LDA, LDAF, INFO
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), WORK( * )
```

```
      DOUBLE PRECISION   C( * ), RWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            NOTRANS
      INTEGER            KASE, I, J
      DOUBLE PRECISION   AINVNM, ANORM, TMP
      COMPLEX*16         ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZLACN2, ZGETRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, REAL, DIMAG
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
      ZLA_GERCOND_C = 0.0D+0
*
      INFO = 0
      NOTRANS = LSAME( TRANS, 'N' )
      IF ( .NOT. NOTRANS .AND. .NOT. LSAME( TRANS, 'T' ) .AND. .NOT.
     $     LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZLA_GERCOND_C', -INFO )
         RETURN
      END IF
*
```

```
*     Compute norm of op(A)*op2(C).
*
      ANORM = 0.0D+0
      IF ( NOTRANS ) THEN
         DO I = 1, N
            TMP = 0.0D+0
            IF ( CAPPLY ) THEN
               DO J = 1, N
                  TMP = TMP + CABS1( A( I, J ) ) / C( J )
               END DO
            ELSE
               DO J = 1, N
                  TMP = TMP + CABS1( A( I, J ) )
               END DO
            END IF
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      ELSE
         DO I = 1, N
            TMP = 0.0D+0
            IF ( CAPPLY ) THEN
               DO J = 1, N
                  TMP = TMP + CABS1( A( J, I ) ) / C( J )
               END DO
            ELSE
               DO J = 1, N
                  TMP = TMP + CABS1( A( J, I ) )
               END DO
            END IF
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 ) THEN
         ZLA_GERCOND_C = 1.0D+0
         RETURN
      ELSE IF( ANORM .EQ. 0.0D+0 ) THEN
         RETURN
      END IF
*
*     Estimate the norm of inv(op(A)).
*
      AINVNM = 0.0D+0
*
      KASE = 0
   10 CONTINUE
```

```
      CALL ZLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
*
            IF (NOTRANS) THEN
               CALL ZGETRS( 'No transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ELSE
               CALL ZGETRS( 'Conjugate transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ENDIF
*
*           Multiply by inv(C).
*
            IF ( CAPPLY ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF
         ELSE
*
*           Multiply by inv(C**H).
*
            IF ( CAPPLY ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF
*
            IF ( NOTRANS ) THEN
               CALL ZGETRS( 'Conjugate transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ELSE
               CALL ZGETRS( 'No transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            END IF
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
         END IF
         GO TO 10
```

```
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM .NE. 0.0D+0 )
     $   ZLA_GERCOND_C = 1.0D+0 / AINVNM
*
      RETURN
*
      END
```

———-

35.35.4 lisp code

— zla_gercond_c.lisp —

```
(defun zla_gercond_c (trans n a lda af ldaf ipiv c capply info work rwork)
  (declare (type f2cl-lib:logical capply)
           (type (array double-float (*)) rwork c)
           (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex16 (*)) work af a)
           (type (f2cl-lib:integer4) info ldaf lda n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (af f2cl-lib:complex16 af-%data% af-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (c double-float c-%data% c-%offset%)
       (rwork double-float rwork-%data% rwork-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
        cabs1))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (zdum #C(0.0d0 0.0d0)) (ainvnm 0.0d0) (anorm 0.0d0) (tmp 0.0d0)
             (kase 0) (i 0) (j 0) (notrans nil) (zla_gercond_c 0.0d0))
        (declare (type f2cl-lib:logical notrans)
                 (type (f2cl-lib:integer4) j i kase)
                 (type (double-float) zla_gercond_c tmp anorm ainvnm)
                 (type (f2cl-lib:complex16) zdum)
                 (type (array f2cl-lib:integer4 (3)) isave))
        (setf zla_gercond_c 0.0d0)
        (setf info 0)
        (setf notrans
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((and (not notrans)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6)))
(cond
 ((/= info 0) (xerbla "ZLA_GERCOND_C" (f2cl-lib:int-sub info))
  (go end_label)))
(setf anorm 0.0d0)
(cond
 (notrans
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf tmp 0.0d0)
      (cond
       (capply
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (/
                        (cabs1
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%))
                        (f2cl-lib:fref c-%data% (j) ((1 *))
                                       c-%offset%))))
           label100001)))
       (t
```

```
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf tmp
                      (+ tmp
                         (cabs1
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%))))
             label100002))))
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                tmp)
        (setf anorm (max anorm tmp))
       label100000)))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf tmp 0.0d0)
       (cond
        (capply
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (/
                         (cabs1
                          (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                          a-%offset%))
                         (f2cl-lib:fref c-%data% (j) ((1 *))
                                         c-%offset%))))
             label100004)))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (cabs1
                         (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                         a-%offset%))))
             label100005))))
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                tmp)
        (setf anorm (max anorm tmp))
       label100003))))
 (cond ((= n 0) (setf zla_gercond_c 1.0d0) (go end_label))
       ((= anorm 0.0d0) (go end_label)))
 (setf ainvnm 0.0d0)
 (setf kase 0)
```

```
label10
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
     (zlacn2 n
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 ((+ n 1))
                            ((1 *)) work-%offset%)
      work ainvnm kase isave)
   (declare (ignore var-1 var-2 var-5))
   (setf n var-0)
   (setf ainvnm var-3)
   (setf kase var-4))
 (cond
  ((/= kase 0)
   (cond
    ((= kase 2)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (*
                  (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                 rwork-%offset%)))
        label100006))
     (cond
      (notrans
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
           (zgetrs "No transpose" n 1 af ldaf ipiv work n info)
         (declare (ignore var-0 var-2 var-3 var-5 var-6))
         (setf n var-1)
         (setf ldaf var-4)
         (setf n var-7)
         (setf info var-8)))
      (t
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
           (zgetrs "Conjugate transpose" n 1 af ldaf ipiv work n info)
         (declare (ignore var-0 var-2 var-3 var-5 var-6))
         (setf n var-1)
         (setf ldaf var-4)
         (setf n var-7)
         (setf info var-8))))
     (cond
      (capply
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                   (*
                    (f2cl-lib:fref work-%data% (i) ((1 *))
```

```
                                    work-%offset%)
                     (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
          label100007)))))
      (t
       (cond
        (capply
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                     (*
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)
                      (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
          label100008))))
       (cond
        (notrans
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
             (zgetrs "Conjugate transpose" n 1 af ldaf ipiv work n info)
           (declare (ignore var-0 var-2 var-3 var-5 var-6))
           (setf n var-1)
           (setf ldaf var-4)
           (setf n var-7)
           (setf info var-8)))
        (t
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
             (zgetrs "No transpose" n 1 af ldaf ipiv work n info)
           (declare (ignore var-0 var-2 var-3 var-5 var-6))
           (setf n var-1)
           (setf ldaf var-4)
           (setf n var-7)
           (setf info var-8))))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                   (*
                    (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%)))
          label100009))))
    (go label10)))
  (if (/= ainvnm 0.0d0)
      (setf zla_gercond_c (/ 1.0d0 ainvnm)))
  (go end_label)
 end_label
  (return
   (values zla_gercond_c trans n nil nil nil ldaf nil nil nil info nil
```

```
                nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zla_gercond_c
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              fortran-to-lisp::logical
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array double-float (*)))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::ldaf nil nil nil
                                              fortran-to-lisp::info nil nil)
                                            :calls
                                            '(fortran-to-lisp::zgetrs
                                              fortran-to-lisp::zlacn2
                                              fortran-to-lisp::xerbla))))
```

———-

35.36 zla_gercond_x

_x

35.36.1 Axiom unit tests

— zlagercondx.input —

```
)set break resume
)sys rm -f zlagercondx.output
)spool zlagercondx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.36.2 Axiom help page

— **zla_gercond_x.help** —

```
==================================================================
zla_gercond examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function zla_gercond_x (TRANS, N, A, LDA, AF, LDAF,
           IPIV, X, INFO, WORK, RWORK)
           ZLA_GERCOND_X computes the infinity norm condition number of
           op(A)*diag(x) for general matrices.

Function/Subroutine Documentation
   double precision function zla_gercond_x (character TRANS, integer N,
       complex*16, dimension( lda, * ) A, integer LDA, complex*16, dimension(
       ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV, complex*16,
       dimension( * ) X, integer INFO, complex*16, dimension( * ) WORK, double
       precision, dimension( * ) RWORK)
       ZLA_GERCOND_X computes the infinity norm condition number of
       op(A)*diag(x) for general matrices.

Purpose:
   ZLA_GERCOND_X computes the infinity norm condition number of
   op(A) * diag(X) where X is a COMPLEX*16 vector.

Parameters:

 TRANS
   TRANS is CHARACTER*1
```

```
  Specifies the form of the system of equations:
  = 'N':  A * X = B     (No transpose)
  = 'T':  A**T * X = B  (Transpose)
  = 'C':  A**H * X = B  (Conjugate Transpose = Transpose)

N
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the N-by-N matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX*16 array, dimension (LDAF,N)
  The factors L and U from the factorization
  A = P*L*U as computed by ZGETRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from the factorization A = P*L*U
  as computed by ZGETRF; row i of the matrix was interchanged
  with row IPIV(i).

X
  X is COMPLEX*16 array, dimension (N)
  The vector X in the formula op(A) * diag(X).

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  i > 0:  The ith argument is invalid.

WORK
  WORK is COMPLEX*16 array, dimension (2*N).
  Workspace.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (N).
  Workspace.
```

———-

35.36.3 fortran code

— **zla_gercond_x.f** —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION ZLA_GERCOND_X( TRANS, N, A, LDA, AF,
     $                                         LDAF, IPIV, X, INFO,
     $                                         WORK, RWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            N, LDA, LDAF, INFO
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), WORK( * ), X( * )
      DOUBLE PRECISION   RWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            NOTRANS
      INTEGER            KASE
      DOUBLE PRECISION   AINVNM, ANORM, TMP
      INTEGER            I, J
      COMPLEX*16         ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZLACN2, ZGETRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, REAL, DIMAG
*     ..
```

```
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      ZLA_GERCOND_X = 0.0D+0
*
      INFO = 0
      NOTRANS = LSAME( TRANS, 'N' )
      IF ( .NOT. NOTRANS .AND. .NOT. LSAME( TRANS, 'T' ) .AND. .NOT.
     $     LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZLA_GERCOND_X', -INFO )
         RETURN
      END IF
*
*     Compute norm of op(A)*op2(C).
*
      ANORM = 0.0D+0
      IF ( NOTRANS ) THEN
         DO I = 1, N
            TMP = 0.0D+0
            DO J = 1, N
               TMP = TMP + CABS1( A( I, J ) * X( J ) )
            END DO
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      ELSE
         DO I = 1, N
            TMP = 0.0D+0
            DO J = 1, N
               TMP = TMP + CABS1( A( J, I ) * X( J ) )
            END DO
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      END IF
*
```

```
*     Quick return if possible.
*
      IF( N.EQ.0 ) THEN
         ZLA_GERCOND_X = 1.0D+0
         RETURN
      ELSE IF( ANORM .EQ. 0.0D+0 ) THEN
         RETURN
      END IF
*
*     Estimate the norm of inv(op(A)).
*
      AINVNM = 0.0D+0
*
      KASE = 0
   10 CONTINUE
      CALL ZLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*           Multiply by R.
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
*
            IF ( NOTRANS ) THEN
               CALL ZGETRS( 'No transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ELSE
               CALL ZGETRS( 'Conjugate transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ENDIF
*
*           Multiply by inv(X).
*
            DO I = 1, N
               WORK( I ) = WORK( I ) / X( I )
            END DO
         ELSE
*
*           Multiply by inv(X**H).
*
            DO I = 1, N
               WORK( I ) = WORK( I ) / X( I )
            END DO
*
            IF ( NOTRANS ) THEN
               CALL ZGETRS( 'Conjugate transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ELSE
               CALL ZGETRS( 'No transpose', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
```

```
            END IF
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM .NE. 0.0D+0 )
     $   ZLA_GERCOND_X = 1.0D+0 / AINVNM
*
      RETURN
*
      END
```

35.36.4 lisp code

— zla_gercond_x.lisp —

```
(defun zla_gercond_x (trans n a lda af ldaf ipiv x info work rwork)
  (declare (type (array double-float (*)) rwork)
           (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex16 (*)) work x af a)
           (type (f2cl-lib:integer4) info ldaf lda n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (af f2cl-lib:complex16 af-%data% af-%offset%)
       (x f2cl-lib:complex16 x-%data% x-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (rwork double-float rwork-%data% rwork-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
        cabs1))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (zdum #C(0.0d0 0.0d0)) (i 0) (j 0) (ainvnm 0.0d0) (anorm 0.0d0)
             (tmp 0.0d0) (kase 0) (notrans nil) (zla_gercond_x 0.0d0))
```

```
(declare (type f2cl-lib:logical notrans)
         (type (double-float) zla_gercond_x tmp anorm ainvnm)
         (type (f2cl-lib:integer4) kase j i)
         (type (f2cl-lib:complex16) zdum)
         (type (array f2cl-lib:integer4 (3)) isave))
(setf zla_gercond_x 0.0d0)
(setf info 0)
(setf notrans
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((and (not notrans)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6)))
(cond
 ((/= info 0) (xerbla "ZLA_GERCOND_X" (f2cl-lib:int-sub info))
  (go end_label)))
(setf anorm 0.0d0)
(cond
 (notrans
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf tmp 0.0d0)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf tmp
                  (+ tmp
                     (cabs1
                      (*
```

```
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%)
                        (f2cl-lib:fref x-%data% (j) ((1 *))
                                       x-%offset%)))))
           label100001))
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                tmp)
        (setf anorm (max anorm tmp))
       label100000)))
   (t
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf tmp 0.0d0)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (cabs1
                        (*
                         (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                        a-%offset%)
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%)))))
           label100003))
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                tmp)
        (setf anorm (max anorm tmp))
       label100002))))
  (cond ((= n 0) (setf zla_gercond_x 1.0d0) (go end_label))
        ((= anorm 0.0d0) (go end_label)))
  (setf ainvnm 0.0d0)
  (setf kase 0)
 label10
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (zlacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 ((+ n 1))
                             ((1 *)) work-%offset%)
       work ainvnm kase isave)
    (declare (ignore var-1 var-2 var-5))
    (setf n var-0)
    (setf ainvnm var-3)
    (setf kase var-4))
  (cond
   ((/= kase 0)
    (cond
     ((= kase 2)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
```

```
    (tagbody
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (*
               (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               (f2cl-lib:fref rwork-%data% (i) ((1 *))
                              rwork-%offset%)))
     label100004))
   (cond
    (notrans
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (zgetrs "No transpose" n 1 af ldaf ipiv work n info)
      (declare (ignore var-0 var-2 var-3 var-5 var-6))
      (setf n var-1)
      (setf ldaf var-4)
      (setf n var-7)
      (setf info var-8)))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (zgetrs "Conjugate transpose" n 1 af ldaf ipiv work n info)
      (declare (ignore var-0 var-2 var-3 var-5 var-6))
      (setf n var-1)
      (setf ldaf var-4)
      (setf n var-7)
      (setf info var-8))))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (/
               (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)))
     label100005)))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (/
               (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)))
     label100006))
   (cond
    (notrans
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (zgetrs "Conjugate transpose" n 1 af ldaf ipiv work n info)
      (declare (ignore var-0 var-2 var-3 var-5 var-6))
      (setf n var-1)
```

```
                (setf ldaf var-4)
                (setf n var-7)
                (setf info var-8)))
              (t
               (multiple-value-bind
                   (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                   (zgetrs "No transpose" n 1 af ldaf ipiv work n info)
                (declare (ignore var-0 var-2 var-3 var-5 var-6))
                (setf n var-1)
                (setf ldaf var-4)
                (setf n var-7)
                (setf info var-8))))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                        (*
                         (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                         (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                        rwork-%offset%)))
               label100007))))
          (go label10)))
        (if (/= ainvnm 0.0d0)
            (setf zla_gercond_x (/ 1.0d0 ainvnm)))
        (go end_label)
       end_label
        (return
         (values zla_gercond_x trans n nil nil nil ldaf nil nil info nil
                 nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zla_gercond_x
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
```

```
    (*))
   (array double-float (*)))
 :return-values
 '(fortran-to-lisp::trans
   fortran-to-lisp::n nil nil nil
   fortran-to-lisp::ldaf nil nil
   fortran-to-lisp::info nil nil)
 :calls
 '(fortran-to-lisp::zgetrs
   fortran-to-lisp::zlacn2
   fortran-to-lisp::xerbla))))
```

———-

35.37 zla_gerfsx_extended

35.37.1 Axiom unit tests

— **zlagerfsxextended.input** —

```
)set break resume
)sys rm -f zlagerfsxextended.output
)spool zlagerfsxextended.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.37.2 Axiom help page [Demm05]

— **zla_gerfsx_extended.help** —

```
==========================================================================
zla_gerfsx_extended examples
==========================================================================

==========================================================================
Man Page Details
```

```
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zla_gerfsx_extended (PREC_TYPE, TRANS_TYPE, N, NRHS, A, LDA,
           AF, LDAF, IPIV, COLEQU, C, B, LDB, Y, LDY, BERR_OUT, N_NORMS,
           ERRS_N, ERRS_C, RES, AYB, DY, Y_TAIL, RCOND, ITHRESH, RTHRESH,
           DZ_UB, IGNORE_CWISE, INFO)
           ZLA_GERFSX_EXTENDED

Function/Subroutine Documentation
   subroutine zla_gerfsx_extended (integer PREC_TYPE, integer TRANS_TYPE,
       integer N, integer NRHS, complex*16, dimension( lda, * ) A, integer
       LDA, complex*16, dimension( ldaf, * ) AF, integer LDAF, integer,
       dimension( * ) IPIV, logical COLEQU, double precision, dimension( * )
       C, complex*16, dimension( ldb, * ) B, integer LDB, complex*16,
       dimension( ldy, * ) Y, integer LDY, double precision, dimension( * )
       BERR_OUT, integer N_NORMS, double precision, dimension( nrhs, * )
       ERRS_N, double precision, dimension( nrhs, * ) ERRS_C, complex*16,
       dimension( * ) RES, double precision, dimension( * ) AYB, complex*16,
       dimension( * ) DY, complex*16, dimension( * ) Y_TAIL, double precision
       RCOND, integer ITHRESH, double precision RTHRESH, double precision
       DZ_UB, logical IGNORE_CWISE, integer INFO)
       ZLA_GERFSX_EXTENDED

Purpose:
   ZLA_GERFSX_EXTENDED improves the computed solution to a system of
   linear equations by performing extra-precise iterative refinement
   and provides error bounds and backward error estimates for the solution.
   This subroutine is called by ZGERFSX to perform iterative refinement.
   In addition to normwise error bound, the code provides maximum
   componentwise error bound if possible. See comments for ERRS_N
   and ERRS_C for details of the error bounds. Note that this
   subroutine is only resonsible for setting the second fields of
   ERRS_N and ERRS_C.

Parameters:

 PREC_TYPE
   PREC_TYPE is INTEGER
   Specifies the intermediate precision to be used in refinement.
   The value is defined by ILAPREC(P) where P is a CHARACTER and
   P    = 'S':  Single
        = 'D':  Double
        = 'I':  Indigenous
        = 'X', 'E':  Extra

 TRANS_TYPE
   TRANS_TYPE is INTEGER
   Specifies the transposition operation on A.
```

```
  The value is defined by ILATRANS(T) where T is a CHARACTER and
  T    = 'N':  No transpose
       = 'T':  Transpose
       = 'C':  Conjugate transpose

N
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right-hand-sides, i.e., the number of columns of the
  matrix B.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the N-by-N matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX*16 array, dimension (LDAF,N)
  The factors L and U from the factorization
  A = P*L*U as computed by ZGETRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from the factorization A = P*L*U
  as computed by ZGETRF; row i of the matrix was interchanged
  with row IPIV(i).

COLEQU
  COLEQU is LOGICAL
  If .TRUE. then column equilibration was done to A before calling
  this routine. This is needed to compute the solution and error
  bounds correctly.

C
  C is DOUBLE PRECISION array, dimension (N)
  The column scale factors for A. If COLEQU = .FALSE., C
  is not accessed. If C is input, each element of C should be a power
  of the radix to ensure a reliable solution and error estimates.
  Scaling by powers of the radix does not cause rounding errors unless
  the result underflows or overflows. Rounding errors during scaling
```

```
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  The right-hand-side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

Y
  Y is COMPLEX*16 array, dimension (LDY,NRHS)
  On entry, the solution matrix X, as computed by ZGETRS.
  On exit, the improved solution matrix Y.

LDY
  LDY is INTEGER
  The leading dimension of the array Y.  LDY >= max(1,N).

BERR_OUT
  BERR_OUT is DOUBLE PRECISION array, dimension (NRHS)
  On exit, BERR_OUT(j) contains the componentwise relative backward
  error for right-hand-side j from the formula
  max(i) ( abs(RES(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
  where abs(Z) is the componentwise absolute value of the matrix
  or vector Z. This is computed by ZLA_LIN_BERR.

N_NORMS
  N_NORMS is INTEGER
  Determines which error bounds to return (see ERRS_N
  and ERRS_C).
  If N_NORMS >= 1 return normwise error bounds.
  If N_NORMS >= 2 return componentwise error bounds.

ERRS_N
  ERRS_N is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
            max_j (abs(XTRUE(j,i) - X(j,i)))
             ------------------------------
                    max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.
```

```
  The first index in ERRS_N(i,:) corresponds to the ith
  right-hand side.

  The second index in ERRS_N(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

ERRS_C
  ERRS_C is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                   abs(XTRUE(j,i) - X(j,i))
             max_j ----------------------
                         abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERRS_C is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERRS_C(i,:) corresponds to the ith
  right-hand side.

  The second index in ERRS_C(:,err) contains the following
  three fields:
```

```
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

RES
  RES is COMPLEX*16 array, dimension (N)
  Workspace to hold the intermediate residual.

AYB
  AYB is DOUBLE PRECISION array, dimension (N)
  Workspace.

DY
  DY is COMPLEX*16 array, dimension (N)
  Workspace to hold the intermediate solution.

Y_TAIL
  Y_TAIL is COMPLEX*16 array, dimension (N)
  Workspace to hold the trailing bits of the intermediate solution.

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

ITHRESH
  ITHRESH is INTEGER
```

```
  The maximum number of residual computations allowed for
  refinement. The default is 10. For 'aggressive' set to 100 to
  permit convergence using approximate factorizations or
  factorizations other than LU. If the factorization uses a
  technique other than Gaussian elimination, the guarantees in
  ERRS_N and ERRS_C may no longer be trustworthy.

RTHRESH
  RTHRESH is DOUBLE PRECISION
  Determines when to stop refinement if the error estimate stops
  decreasing. Refinement will stop when the next solution no longer
  satisfies norm(dx_{i+1}) < RTHRESH * norm(dx_i) where norm(Z) is
  the infinity norm of Z. RTHRESH satisfies 0 < RTHRESH <= 1. The
  default value is 0.5. For 'aggressive' set to 0.9 to permit
  convergence on extremely ill-conditioned matrices. See LAWN 165
  for more details.

DZ_UB
  DZ_UB is DOUBLE PRECISION
  Determines when to start considering componentwise convergence.
  Componentwise convergence is only considered after each component
  of the solution Y is stable, which we definte as the relative
  change in each component being less than DZ_UB. The default value
  is 0.25, requiring the first bit to be stable. See LAWN 165 for
  more details.

IGNORE_CWISE
  IGNORE_CWISE is LOGICAL
  If .TRUE. then ignore componentwise convergence. Default value
  is .FALSE..

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  < 0:  if INFO = -i, the ith argument to ZGETRS had an illegal value
```

———-

35.37.3 fortran code

— zla_gerfsx_extended.f —

```
*  ====================================================================
      SUBROUTINE ZLA_GERFSX_EXTENDED( PREC_TYPE, TRANS_TYPE, N, NRHS, A,
     $                                LDA, AF, LDAF, IPIV, COLEQU, C, B,
     $                                LDB, Y, LDY, BERR_OUT, N_NORMS,
     $                                ERRS_N, ERRS_C, RES, AYB, DY,
```

```
     $                                Y_TAIL, RCOND, ITHRESH, RTHRESH,
     $                                DZ_UB, IGNORE_CWISE, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDAF, LDB, LDY, N, NRHS, PREC_TYPE,
     $                   TRANS_TYPE, N_NORMS
      LOGICAL            COLEQU, IGNORE_CWISE
      INTEGER            ITHRESH
      DOUBLE PRECISION   RTHRESH, DZ_UB
*     ..
*     .. Array Arguments
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   Y( LDY, * ), RES( * ), DY( * ), Y_TAIL( * )
      DOUBLE PRECISION   C( * ), AYB( * ), RCOND, BERR_OUT( * ),
     $                   ERRS_N( NRHS, * ), ERRS_C( NRHS, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      CHARACTER          TRANS
      INTEGER            CNT, I, J,  X_STATE, Z_STATE, Y_PREC_STATE
      DOUBLE PRECISION   YK, DYK, YMIN, NORMY, NORMX, NORMDX, DXRAT,
     $                   DZRAT, PREVNORMDX, PREV_DZ_Z, DXRATMAX,
     $                   DZRATMAX, DX_X, DZ_Z, FINAL_DX_X, FINAL_DZ_Z,
     $                   EPS, HUGEVAL, INCR_THRESH
      LOGICAL            INCR_PREC
      COMPLEX*16         ZDUM
*     ..
*     .. Parameters ..
      INTEGER            UNSTABLE_STATE, WORKING_STATE, CONV_STATE,
     $                   NOPROG_STATE, BASE_RESIDUAL, EXTRA_RESIDUAL,
     $                   EXTRA_Y
      PARAMETER          ( UNSTABLE_STATE = 0, WORKING_STATE = 1,
     $                   CONV_STATE = 2,
     $                   NOPROG_STATE = 3 )
      PARAMETER          ( BASE_RESIDUAL = 0, EXTRA_RESIDUAL = 1,
     $                   EXTRA_Y = 2 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
```

```
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZAXPY, ZCOPY, ZGETRS, ZGEMV, BLAS_ZGEMV_X,
     $                   BLAS_ZGEMV2_X, ZLA_GEAMV, ZLA_WWADDW, DLAMCH,
     $                   CHLA_TRANSTYPE, ZLA_LIN_BERR
      DOUBLE PRECISION   DLAMCH
      CHARACTER          CHLA_TRANSTYPE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      IF ( INFO.NE.0 ) RETURN
      TRANS = CHLA_TRANSTYPE(TRANS_TYPE)
      EPS = DLAMCH( 'Epsilon' )
      HUGEVAL = DLAMCH( 'Overflow' )
*     Force HUGEVAL to Inf
      HUGEVAL = HUGEVAL * HUGEVAL
*     Using HUGEVAL may lead to spurious underflows.
      INCR_THRESH = DBLE( N ) * EPS
*
      DO J = 1, NRHS
         Y_PREC_STATE = EXTRA_RESIDUAL
         IF ( Y_PREC_STATE .EQ. EXTRA_Y ) THEN
            DO I = 1, N
               Y_TAIL( I ) = 0.0D+0
            END DO
         END IF

         DXRAT = 0.0D+0
         DXRATMAX = 0.0D+0
         DZRAT = 0.0D+0
```

```
         DZRATMAX = 0.0D+0
         FINAL_DX_X = HUGEVAL
         FINAL_DZ_Z = HUGEVAL
         PREVNORMDX = HUGEVAL
         PREV_DZ_Z = HUGEVAL
         DZ_Z = HUGEVAL
         DX_X = HUGEVAL

         X_STATE = WORKING_STATE
         Z_STATE = UNSTABLE_STATE
         INCR_PREC = .FALSE.

         DO CNT = 1, ITHRESH
*
*         Compute residual RES = B_s - op(A_s) * Y,
*             op(A) = A, A**T, or A**H depending on TRANS (and type).
*
           CALL ZCOPY( N, B( 1, J ), 1, RES, 1 )
           IF ( Y_PREC_STATE .EQ. BASE_RESIDUAL ) THEN
              CALL ZGEMV( TRANS, N, N, (-1.0D+0,0.0D+0), A, LDA,
     $             Y( 1, J ), 1, (1.0D+0,0.0D+0), RES, 1)
           ELSE IF (Y_PREC_STATE .EQ. EXTRA_RESIDUAL) THEN
              CALL BLAS_ZGEMV_X( TRANS_TYPE, N, N, (-1.0D+0,0.0D+0), A,
     $             LDA, Y( 1, J ), 1, (1.0D+0,0.0D+0),
     $             RES, 1, PREC_TYPE )
           ELSE
              CALL BLAS_ZGEMV2_X( TRANS_TYPE, N, N, (-1.0D+0,0.0D+0),
     $             A, LDA, Y(1, J), Y_TAIL, 1, (1.0D+0,0.0D+0), RES, 1,
     $             PREC_TYPE)
           END IF

!         XXX: RES is no longer needed.
           CALL ZCOPY( N, RES, 1, DY, 1 )
           CALL ZGETRS( TRANS, N, 1, AF, LDAF, IPIV, DY, N, INFO )
*
*         Calculate relative changes DX_X, DZ_Z and ratios DXRAT, DZRAT.
*
           NORMX = 0.0D+0
           NORMY = 0.0D+0
           NORMDX = 0.0D+0
           DZ_Z = 0.0D+0
           YMIN = HUGEVAL
*
           DO I = 1, N
              YK = CABS1( Y( I, J ) )
              DYK = CABS1( DY( I ) )

              IF ( YK .NE. 0.0D+0 ) THEN
                 DZ_Z = MAX( DZ_Z, DYK / YK )
              ELSE IF ( DYK .NE. 0.0D+0 ) THEN
```

```
                 DZ_Z = HUGEVAL
              END IF

              YMIN = MIN( YMIN, YK )

              NORMY = MAX( NORMY, YK )

              IF ( COLEQU ) THEN
                 NORMX = MAX( NORMX, YK * C( I ) )
                 NORMDX = MAX( NORMDX, DYK * C( I ) )
              ELSE
                 NORMX = NORMY
                 NORMDX = MAX(NORMDX, DYK)
              END IF
           END DO

           IF ( NORMX .NE. 0.0D+0 ) THEN
              DX_X = NORMDX / NORMX
           ELSE IF ( NORMDX .EQ. 0.0D+0 ) THEN
              DX_X = 0.0D+0
           ELSE
              DX_X = HUGEVAL
           END IF

           DXRAT = NORMDX / PREVNORMDX
           DZRAT = DZ_Z / PREV_DZ_Z
*
*        Check termination criteria
*
           IF (.NOT.IGNORE_CWISE
     $           .AND. YMIN*RCOND .LT. INCR_THRESH*NORMY
     $           .AND. Y_PREC_STATE .LT. EXTRA_Y )
     $           INCR_PREC = .TRUE.

           IF ( X_STATE .EQ. NOPROG_STATE .AND. DXRAT .LE. RTHRESH )
     $           X_STATE = WORKING_STATE
           IF ( X_STATE .EQ. WORKING_STATE ) THEN
              IF (DX_X .LE. EPS) THEN
                 X_STATE = CONV_STATE
              ELSE IF ( DXRAT .GT. RTHRESH ) THEN
                 IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                    INCR_PREC = .TRUE.
                 ELSE
                    X_STATE = NOPROG_STATE
                 END IF
              ELSE
                 IF ( DXRAT .GT. DXRATMAX ) DXRATMAX = DXRAT
              END IF
              IF ( X_STATE .GT. WORKING_STATE ) FINAL_DX_X = DX_X
           END IF
```

```
            IF ( Z_STATE .EQ. UNSTABLE_STATE .AND. DZ_Z .LE. DZ_UB )
     $           Z_STATE = WORKING_STATE
            IF ( Z_STATE .EQ. NOPROG_STATE .AND. DZRAT .LE. RTHRESH )
     $           Z_STATE = WORKING_STATE
            IF ( Z_STATE .EQ. WORKING_STATE ) THEN
               IF ( DZ_Z .LE. EPS ) THEN
                  Z_STATE = CONV_STATE
               ELSE IF ( DZ_Z .GT. DZ_UB ) THEN
                  Z_STATE = UNSTABLE_STATE
                  DZRATMAX = 0.0D+0
                  FINAL_DZ_Z = HUGEVAL
               ELSE IF ( DZRAT .GT. RTHRESH ) THEN
                  IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                     INCR_PREC = .TRUE.
                  ELSE
                     Z_STATE = NOPROG_STATE
                  END IF
               ELSE
                  IF ( DZRAT .GT. DZRATMAX ) DZRATMAX = DZRAT
               END IF
               IF ( Z_STATE .GT. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
            END IF
*
*           Exit if both normwise and componentwise stopped working,
*           but if componentwise is unstable, let it go at least two
*           iterations.
*
            IF ( X_STATE.NE.WORKING_STATE ) THEN
               IF ( IGNORE_CWISE ) GOTO 666
               IF ( Z_STATE.EQ.NOPROG_STATE .OR. Z_STATE.EQ.CONV_STATE )
     $              GOTO 666
               IF ( Z_STATE.EQ.UNSTABLE_STATE .AND. CNT.GT.1 ) GOTO 666
            END IF

            IF ( INCR_PREC ) THEN
               INCR_PREC = .FALSE.
               Y_PREC_STATE = Y_PREC_STATE + 1
               DO I = 1, N
                  Y_TAIL( I ) = 0.0D+0
               END DO
            END IF

            PREVNORMDX = NORMDX
            PREV_DZ_Z = DZ_Z
*
*           Update soluton.
*
            IF ( Y_PREC_STATE .LT. EXTRA_Y ) THEN
               CALL ZAXPY( N, (1.0D+0,0.0D+0), DY, 1, Y(1,J), 1 )
```

```
            ELSE
               CALL ZLA_WWADDW( N, Y( 1, J ), Y_TAIL, DY )
            END IF

         END DO
*        Target of "IF (Z_STOP .AND. X_STOP)".  Sun's f77 won't EXIT.
 666     CONTINUE
*
*     Set final_* when cnt hits ithresh
*
         IF ( X_STATE .EQ. WORKING_STATE ) FINAL_DX_X = DX_X
         IF ( Z_STATE .EQ. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
*
*     Compute error bounds
*
         IF (N_NORMS .GE. 1) THEN
            ERRS_N( J, LA_LINRX_ERR_I ) = FINAL_DX_X / (1 - DXRATMAX)

         END IF
         IF ( N_NORMS .GE. 2 ) THEN
            ERRS_C( J, LA_LINRX_ERR_I ) = FINAL_DZ_Z / (1 - DZRATMAX)
         END IF
*
*     Compute componentwise relative backward error from formula
*         max(i) ( abs(R(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
*     where abs(Z) is the componentwise absolute value of the matrix
*     or vector Z.
*
*        Compute residual RES = B_s - op(A_s) * Y,
*            op(A) = A, A**T, or A**H depending on TRANS (and type).
*
         CALL ZCOPY( N, B( 1, J ), 1, RES, 1 )
         CALL ZGEMV( TRANS, N, N, (-1.0D+0,0.0D+0), A, LDA, Y(1,J), 1,
     $        (1.0D+0,0.0D+0), RES, 1 )

         DO I = 1, N
            AYB( I ) = CABS1( B( I, J ) )
         END DO
*
*     Compute abs(op(A_s))*abs(Y) + abs(B_s).
*
         CALL ZLA_GEAMV ( TRANS_TYPE, N, N, 1.0D+0,
     $        A, LDA, Y(1, J), 1, 1.0D+0, AYB, 1 )

         CALL ZLA_LIN_BERR ( N, N, 1, RES, AYB, BERR_OUT( J ) )
*
*     End of loop for each RHS.
*
      END DO
*
```

```
      RETURN
      END
```

———-

35.37.4 lisp code

— **zla_gerfsx_extended.lisp** —

```
(let* ((unstable_state 0)
       (working_state 1)
       (conv_state 2)
       (noprog_state 3)
       (base_residual 0)
       (extra_residual 1)
       (extra_y 2)
       (la_linrx_err_i 2))
  (declare (type (f2cl-lib:integer4 0 0) unstable_state)
           (type (f2cl-lib:integer4 1 1) working_state)
           (type (f2cl-lib:integer4 2 2) conv_state)
           (type (f2cl-lib:integer4 3 3) noprog_state)
           (type (f2cl-lib:integer4 0 0) base_residual)
           (type (f2cl-lib:integer4 1 1) extra_residual)
           (type (f2cl-lib:integer4 2 2) extra_y)
           (type (f2cl-lib:integer4 2 2) la_linrx_err_i)
           (ignorable unstable_state working_state conv_state noprog_state
            base_residual extra_residual extra_y la_linrx_err_i))
  (defun zla_gerfsx_extended
         (prec_type trans_type n nrhs a lda af ldaf ipiv colequ c b ldb$ y ldy
          berr_out n_norms errs_n errs_c res ayb dy y_tail rcond ithresh
          rthresh dz_ub ignore_cwise info)
    (declare (type (double-float) dz_ub rthresh rcond)
             (type (array double-float (*)) ayb errs_c errs_n berr_out c)
             (type f2cl-lib:logical ignore_cwise colequ)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) y_tail dy res y b af a)
             (type (f2cl-lib:integer4) info ithresh n_norms ldy ldb$ ldaf lda
              nrhs n trans_type prec_type))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (af f2cl-lib:complex16 af-%data% af-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (y f2cl-lib:complex16 y-%data% y-%offset%)
         (res f2cl-lib:complex16 res-%data% res-%offset%)
         (dy f2cl-lib:complex16 dy-%data% dy-%offset%)
         (y_tail f2cl-lib:complex16 y_tail-%data% y_tail-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (c double-float c-%data% c-%offset%)
```

```
    (berr_out double-float berr_out-%data% berr_out-%offset%)
    (errs_n double-float errs_n-%data% errs_n-%offset%)
    (errs_c double-float errs_c-%data% errs_c-%offset%)
    (ayb double-float ayb-%data% ayb-%offset%))
  (labels ((cabs1 (zdum)
             (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
    (declare
     (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
      cabs1))
    (prog ((zdum #C(0.0d0 0.0d0)) (incr_prec nil) (yk 0.0d0) (dyk 0.0d0)
           (ymin 0.0d0) (normy 0.0d0) (normx 0.0d0) (normdx 0.0d0)
           (dxrat 0.0d0) (dzrat 0.0d0) (prevnormdx 0.0d0) (prev_dz_z 0.0d0)
           (dxratmax 0.0d0) (dzratmax 0.0d0) (dx_x 0.0d0) (dz_z 0.0d0)
           (final_dx_x 0.0d0) (final_dz_z 0.0d0) (eps 0.0d0)
           (hugeval 0.0d0) (incr_thresh 0.0d0) (cnt 0) (i 0) (j 0)
           (x_state 0) (z_state 0) (y_prec_state 0)
           (trans
            (make-array '(1) :element-type 'character :initial-element
                        #\ )))
     (declare (type (f2cl-lib:complex16) zdum)
              (type f2cl-lib:logical incr_prec)
              (type (double-float) yk dyk ymin normy normx normdx dxrat
               dzrat prevnormdx prev_dz_z dxratmax dzratmax dx_x dz_z
               final_dx_x final_dz_z eps hugeval incr_thresh)
              (type (f2cl-lib:integer4) cnt i j x_state z_state
               y_prec_state)
              (type (string 1) trans))
     (if (/= info 0)
         (go end_label))
     (f2cl-lib:f2cl-set-string trans
                               (f2cl-lib:fref-string #'chla_transtype
                                                     (trans_type))
                               (string 1))
     (setf eps (dlamch "Epsilon"))
     (setf hugeval (dlamch "Overflow"))
     (setf hugeval (* hugeval hugeval))
     (setf incr_thresh (* (f2cl-lib:dble n) eps))
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j nrhs) nil)
       (tagbody
         (setf y_prec_state extra_residual)
         (cond
          ((= y_prec_state extra_y)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                                    y_tail-%offset%)
                     (coerce 0.0d0 'f2cl-lib:complex16))
             label100001))))
```

```
(setf dxrat 0.0d0)
(setf dxratmax 0.0d0)
(setf dzrat 0.0d0)
(setf dzratmax 0.0d0)
(setf final_dx_x hugeval)
(setf final_dz_z hugeval)
(setf prevnormdx hugeval)
(setf prev_dz_z hugeval)
(setf dz_z hugeval)
(setf dx_x hugeval)
(setf x_state working_state)
(setf z_state unstable_state)
(setf incr_prec f2cl-lib:%false%)
(f2cl-lib:fdo (cnt 1 (f2cl-lib:int-add cnt 1))
              ((> cnt ithresh) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zcopy n
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1 res 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (cond
     ((= y_prec_state base_residual)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (zgemv trans n n (f2cl-lib:cmplx -1.0d0 0.0d0) a lda
           (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                                 (1 j) ((1 ldy) (1 *))
                                 y-%offset%)
           1 (f2cl-lib:cmplx 1.0d0 0.0d0) res 1)
        (declare
         (ignore var-3 var-4 var-6 var-7 var-8 var-9 var-10))
        (when var-0 (setf trans var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-5 (setf lda var-5))))
     ((= y_prec_state extra_residual)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11)
          (blas_zgemv_x trans_type n n
           (f2cl-lib:cmplx -1.0d0 0.0d0) a lda
           (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                                 (1 j) ((1 ldy) (1 *))
                                 y-%offset%)
           1 (f2cl-lib:cmplx 1.0d0 0.0d0) res 1 prec_type)
        (declare
```

```
     (ignore var-3 var-4 var-6 var-7 var-8 var-9 var-10))
    (when var-0 (setf trans_type var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-5 (setf lda var-5))
    (when var-11 (setf prec_type var-11))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (blas_zgemv2_x trans_type n n
       (f2cl-lib:cmplx -1.0d0 0.0d0) a lda
       (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                             (1 j) ((1 ldy) (1 *))
                             y-%offset%)
       y_tail 1 (f2cl-lib:cmplx 1.0d0 0.0d0) res 1 prec_type)
    (declare
     (ignore var-3 var-4 var-6 var-7 var-8 var-9 var-10
      var-11))
    (when var-0 (setf trans_type var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-5 (setf lda var-5))
    (when var-12 (setf prec_type var-12)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (zcopy n res 1 dy 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zgetrs trans n 1 af ldaf ipiv dy n info)
  (declare (ignore var-2 var-3 var-5 var-6))
  (setf trans var-0)
  (setf n var-1)
  (setf ldaf var-4)
  (setf n var-7)
  (setf info var-8))
(setf normx 0.0d0)
(setf normy 0.0d0)
(setf normdx 0.0d0)
(setf dz_z 0.0d0)
(setf ymin hugeval)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf yk
            (cabs1
             (f2cl-lib:fref y-%data% (i j) ((1 ldy) (1 *))
                            y-%offset%)))
    (setf dyk
```

```
            (cabs1
             (f2cl-lib:fref dy-%data% (i) ((1 *))
                            dy-%offset%)))
    (cond ((/= yk 0.0d0) (setf dz_z (max dz_z (/ dyk yk))))
          ((/= dyk 0.0d0) (setf dz_z hugeval)))
    (setf ymin (min ymin yk))
    (setf normy (max normy yk))
    (cond
     (colequ
      (setf normx
              (max normx
                   (* yk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%))))
      (setf normdx
              (max normdx
                   (* dyk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%)))))
     (t (setf normx normy) (setf normdx (max normdx dyk))))
   label100003))
(cond ((/= normx 0.0d0) (setf dx_x (/ normdx normx)))
      ((= normdx 0.0d0) (setf dx_x 0.0d0))
      (t (setf dx_x hugeval)))
(setf dxrat (/ normdx prevnormdx))
(setf dzrat (/ dz_z prev_dz_z))
(if (and (not ignore_cwise)
         (< (* ymin rcond) (* incr_thresh normy))
         (< y_prec_state extra_y))
    (setf incr_prec f2cl-lib:%true%))
(if (and (= x_state noprog_state) (<= dxrat rthresh))
    (setf x_state working_state))
(cond
 ((= x_state working_state)
  (cond ((<= dx_x eps) (setf x_state conv_state))
        ((> dxrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf x_state noprog_state))))
        (t
         (if (> dxrat dxratmax)
             (setf dxratmax dxrat))))
  (if (> x_state working_state)
      (setf final_dx_x dx_x))))
(if (and (= z_state unstable_state) (<= dz_z dz_ub))
    (setf z_state working_state))
(if (and (= z_state noprog_state) (<= dzrat rthresh))
    (setf z_state working_state))
(cond
```

```
 ((= z_state working_state)
  (cond ((<= dz_z eps) (setf z_state conv_state))
        ((> dz_z dz_ub) (setf z_state unstable_state)
         (setf dzratmax 0.0d0) (setf final_dz_z hugeval))
        ((> dzrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf z_state noprog_state))))
        (t
         (if (> dzrat dzratmax)
             (setf dzratmax dzrat))))
  (if (> z_state working_state)
      (setf final_dz_z dz_z))))
(cond
 ((/= x_state working_state)
  (if ignore_cwise
      (go label666))
  (if (or (= z_state noprog_state) (= z_state conv_state))
      (go label666))
  (if (and (= z_state unstable_state) (> cnt 1))
      (go label666))))
(cond
 (incr_prec (setf incr_prec f2cl-lib:%false%)
  (setf y_prec_state (f2cl-lib:int-add y_prec_state 1))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                           y_tail-%offset%)
              (coerce 0.0d0 'f2cl-lib:complex16))
     label100004))))
(setf prevnormdx normdx)
(setf prev_dz_z dz_z)
(cond
 ((< y_prec_state extra_y)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (zaxpy n (f2cl-lib:cmplx 1.0d0 0.0d0) dy 1
       (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                             (1 j) ((1 ldy) (1 *))
                             y-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (zla_wwaddw n
       (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                             (1 j) ((1 ldy) (1 *))
                             y-%offset%)
```

```
            y_tail dy)
         (declare (ignore var-1 var-2 var-3))
         (when var-0 (setf n var-0)))))
    label100002))
label666
 (if (= x_state working_state)
     (setf final_dx_x dx_x))
 (if (= z_state working_state)
     (setf final_dz_z dz_z))
 (cond
  ((>= n_norms 1)
   (setf (f2cl-lib:fref errs_n-%data% (j la_linrx_err_i)
                        ((1 nrhs) (1 *)) errs_n-%offset%)
          (/ final_dx_x (- 1 dxratmax)))))
 (cond
  ((>= n_norms 2)
   (setf (f2cl-lib:fref errs_c-%data% (j la_linrx_err_i)
                        ((1 nrhs) (1 *)) errs_c-%offset%)
          (/ final_dz_z (- 1 dzratmax)))))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (zcopy n
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 j)
                            ((1 ldb$) (1 *)) b-%offset%)
      1 res 1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf n var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (zgemv trans n n (f2cl-lib:cmplx -1.0d0 0.0d0) a lda
      (f2cl-lib:array-slice y-%data% f2cl-lib:complex16 (1 j)
                            ((1 ldy) (1 *)) y-%offset%)
      1 (f2cl-lib:cmplx 1.0d0 0.0d0) res 1)
   (declare (ignore var-3 var-4 var-6 var-7 var-8 var-9 var-10))
   (when var-0 (setf trans var-0))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-5 (setf lda var-5)))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf (f2cl-lib:fref ayb-%data% (i) ((1 *)) ayb-%offset%)
             (cabs1
              (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)))
    label100005))
 (zla_geamv trans_type n n 1.0d0 a lda
  (f2cl-lib:array-slice y-%data% f2cl-lib:complex16 (1 j)
                        ((1 ldy) (1 *)) y-%offset%)
  1 1.0d0 ayb 1)
```

```
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                  (zla_lin_berr n n 1 res ayb
                   (f2cl-lib:array-slice berr_out-%data% double-float (j)
                                         ((1 *)) berr_out-%offset%))
                (declare (ignore var-2 var-3 var-4 var-5))
                (when var-0 (setf n var-0))
                (when var-1 (setf n var-1)))
            label100000))
          (go end_label)
         end_label
          (return
           (values prec_type trans_type n nil nil lda nil ldaf nil nil nil nil
                   nil nil nil nil nil nil nil nil nil nil nil nil nil nil
                   nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zla_gerfsx_extended
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              fortran-to-lisp::logical
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
```

```
  (array fortran-to-lisp::complex16
   (*))
  (double-float)
  (fortran-to-lisp::integer4)
  (double-float) (double-float)
  fortran-to-lisp::logical
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::prec_type
  fortran-to-lisp::trans_type
  fortran-to-lisp::n nil nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldaf nil nil nil
  nil nil nil nil nil nil nil nil
  nil nil nil nil nil nil nil nil
  nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zla_geamv
  fortran-to-lisp::zgetrs))))
```

———-

35.38 zla_gerpvgrw

35.38.1 Axiom unit tests

— zlagerpvgrw.input —

```
)set break resume
)sys rm -f zlagerpvgrw.output
)spool zlagerpvgrw.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.38.2 Axiom help page

— zla_gerpvgrw.help —

```
====================================================================
zla_gerpvgrw examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function zla_gerpvgrw (N, NCOLS, A, LDA, AF, LDAF)
           ZLA_GERPVGRW multiplies a square real matrix by a complex matrix.

Function/Subroutine Documentation
   double precision function zla_gerpvgrw (integer N, integer NCOLS,
       complex*16, dimension( lda, * ) A, integer LDA, complex*16, dimension(
       ldaf, * ) AF, integer LDAF)
       ZLA_GERPVGRW multiplies a square real matrix by a complex matrix.

Purpose:
   ZLA_GERPVGRW computes the reciprocal pivot growth factor
   norm(A)/norm(U). The "max absolute element" norm is used. If this is
   much less than 1, the stability of the LU factorization of the
   (equilibrated) matrix A could be poor. This also means that the
   solution X, estimated condition numbers, and error bounds could be
   unreliable.

Parameters:

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NCOLS
   NCOLS is INTEGER
   The number of columns of the matrix A. NCOLS >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is DOUBLE PRECISION array, dimension (LDAF,N)
   The factors L and U from the factorization
   A = P*L*U as computed by ZGETRF.
```

```
 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).
```

35.38.3 fortran code

— zla_gerpvgrw.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION ZLA_GERPVGRW( N, NCOLS, A, LDA, AF,
     $         LDAF )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            N, NCOLS, LDA, LDAF
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), AF( LDAF, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   AMAX, UMAX, RPVGRW
      COMPLEX*16         ZDUM
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, ABS, REAL, DIMAG
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      RPVGRW = 1.0D+0

      DO J = 1, NCOLS
```

```
         AMAX = 0.0D+0
         UMAX = 0.0D+0
         DO I = 1, N
            AMAX = MAX( CABS1( A( I, J ) ), AMAX )
         END DO
         DO I = 1, J
            UMAX = MAX( CABS1( AF( I, J ) ), UMAX )
         END DO
         IF ( UMAX /= 0.0D+0 ) THEN
            RPVGRW = MIN( AMAX / UMAX, RPVGRW )
         END IF
      END DO
      ZLA_GERPVGRW = RPVGRW
      END
```

———-

35.38.4 lisp code

— zla_gerpvgrw.lisp —

```
(defun zla_gerpvgrw (n ncols a lda af ldaf)
  (declare (type (array f2cl-lib:complex16 (*)) af a)
           (type (f2cl-lib:integer4) ldaf lda ncols n))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex16 a-%data% a-%offset%)
       (af f2cl-lib:complex16 af-%data% af-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
        cabs1))
      (prog ((zdum #C(0.0d0 0.0d0)) (amax 0.0d0) (umax 0.0d0) (rpvgrw 0.0d0)
             (i 0) (j 0) (zla_gerpvgrw 0.0d0))
        (declare (type (f2cl-lib:integer4) j i)
                 (type (double-float) zla_gerpvgrw rpvgrw umax amax)
                 (type (f2cl-lib:complex16) zdum))
        (setf rpvgrw 1.0d0)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j ncols) nil)
          (tagbody
            (setf amax 0.0d0)
            (setf umax 0.0d0)
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf amax
                        (max
```

```
                        (cabs1
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%))
                        amax))
               label100001))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i j) nil)
              (tagbody
                (setf umax
                        (max
                         (cabs1
                          (f2cl-lib:fref af-%data% (i j) ((1 ldaf) (1 *))
                                         af-%offset%))
                         umax))
               label100002))
            (cond
             ((f2cl-lib:f2cl/ umax (= 0.0d0))
              (setf rpvgrw (min (/ amax umax) rpvgrw))))
           label100000))
        (setf zla_gerpvgrw rpvgrw)
       end_label
        (return (values zla_gerpvgrw nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zla_gerpvgrw
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil) :calls
                                            'nil)))
```

———-

35.39 ztgevc

35.39.1 Axiom unit tests

— ztgevc.input —

```
)set break resume
)sys rm -f ztgevc.output
)spool ztgevc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.39.2 Axiom help page

— ztgevc.help —

```
====================================================================
ztgevc examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ztgevc (SIDE, HOWMNY, SELECT, N, S, LDS, P, LDP, VL, LDVL,
           VR, LDVR, MM, M, WORK, RWORK, INFO)
           ZTGEVC

Function/Subroutine Documentation
   subroutine ztgevc (character SIDE, character HOWMNY, logical, dimension( *
       ) SELECT, integer N, complex*16, dimension( lds, * ) S, integer LDS,
       complex*16, dimension( ldp, * ) P, integer LDP, complex*16, dimension(
       ldvl, * ) VL, integer LDVL, complex*16, dimension( ldvr, * ) VR,
       integer LDVR, integer MM, integer M, complex*16, dimension( * ) WORK,
       double precision, dimension( * ) RWORK, integer INFO)
       ZTGEVC

Purpose:
```

```
   ZTGEVC computes some or all of the right and/or left eigenvectors of
   a pair of complex matrices (S,P), where S and P are upper triangular.
   Matrix pairs of this type are produced by the generalized Schur
   factorization of a complex matrix pair (A,B):

   A = Q*S*Z**H,  B = Q*P*Z**H

   as computed by ZGGHRD + ZHGEQZ.

   The right eigenvector x and the left eigenvector y of (S,P)
   corresponding to an eigenvalue w are defined by:

   S*x = w*P*x,  (y**H)*S = w*(y**H)*P,

   where y**H denotes the conjugate tranpose of y.
   The eigenvalues are not input to this routine, but are computed
   directly from the diagonal elements of S and P.

   This routine returns the matrices X and/or Y of right and left
   eigenvectors of (S,P), or the products Z*X and/or Q*Y,
   where Z and Q are input matrices.
   If Q and Z are the unitary factors from the generalized Schur
   factorization of a matrix pair (A,B), then Z*X and Q*Y
   are the matrices of right and left eigenvectors of (A,B).

Parameters:

 SIDE
   SIDE is CHARACTER*1
   = 'R': compute right eigenvectors only;
   = 'L': compute left eigenvectors only;
   = 'B': compute both right and left eigenvectors.

 HOWMNY
   HOWMNY is CHARACTER*1
   = 'A': compute all right and/or left eigenvectors;
   = 'B': compute all right and/or left eigenvectors,
      backtransformed by the matrices in VR and/or VL;
   = 'S': compute selected right and/or left eigenvectors,
      specified by the logical array SELECT.

 SELECT
   SELECT is LOGICAL array, dimension (N)
   If HOWMNY='S', SELECT specifies the eigenvectors to be
   computed.  The eigenvector corresponding to the j-th
   eigenvalue is computed if SELECT(j) = .TRUE..
   Not referenced if HOWMNY = 'A' or 'B'.

 N
   N is INTEGER
```

```
  The order of the matrices S and P.  N >= 0.

S
  S is COMPLEX*16 array, dimension (LDS,N)
  The upper triangular matrix S from a generalized Schur
  factorization, as computed by ZHGEQZ.

LDS
  LDS is INTEGER
  The leading dimension of array S.  LDS >= max(1,N).

P
  P is COMPLEX*16 array, dimension (LDP,N)
  The upper triangular matrix P from a generalized Schur
  factorization, as computed by ZHGEQZ.  P must have real
  diagonal elements.

LDP
  LDP is INTEGER
  The leading dimension of array P.  LDP >= max(1,N).

VL
  VL is COMPLEX*16 array, dimension (LDVL,MM)
  On entry, if SIDE = 'L' or 'B' and HOWMNY = 'B', VL must
  contain an N-by-N matrix Q (usually the unitary matrix Q
  of left Schur vectors returned by ZHGEQZ).
  On exit, if SIDE = 'L' or 'B', VL contains:
  if HOWMNY = 'A', the matrix Y of left eigenvectors of (S,P);
  if HOWMNY = 'B', the matrix Q*Y;
  if HOWMNY = 'S', the left eigenvectors of (S,P) specified by
      SELECT, stored consecutively in the columns of
      VL, in the same order as their eigenvalues.
  Not referenced if SIDE = 'R'.

LDVL
  LDVL is INTEGER
  The leading dimension of array VL.  LDVL >= 1, and if
  SIDE = 'L' or 'l' or 'B' or 'b', LDVL >= N.

VR
  VR is COMPLEX*16 array, dimension (LDVR,MM)
  On entry, if SIDE = 'R' or 'B' and HOWMNY = 'B', VR must
  contain an N-by-N matrix Q (usually the unitary matrix Z
  of right Schur vectors returned by ZHGEQZ).
  On exit, if SIDE = 'R' or 'B', VR contains:
  if HOWMNY = 'A', the matrix X of right eigenvectors of (S,P);
  if HOWMNY = 'B', the matrix Z*X;
  if HOWMNY = 'S', the right eigenvectors of (S,P) specified by
       SELECT, stored consecutively in the columns of
       VR, in the same order as their eigenvalues.
```

```
  Not referenced if SIDE = 'L'.

LDVR
  LDVR is INTEGER
  The leading dimension of the array VR.  LDVR >= 1, and if
  SIDE = 'R' or 'B', LDVR >= N.

MM
  MM is INTEGER
  The number of columns in the arrays VL and/or VR. MM >= M.

M
  M is INTEGER
  The number of columns in the arrays VL and/or VR actually
  used to store the eigenvectors.  If HOWMNY = 'A' or 'B', M
  is set to N.  Each selected eigenvector occupies one column.

WORK
  WORK is COMPLEX*16 array, dimension (2*N)

RWORK
  RWORK is DOUBLE PRECISION array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

———-

35.39.3 fortran code

— **ztgevc.f** —

```
*  =====================================================================
      SUBROUTINE ZTGEVC( SIDE, HOWMNY, SELECT, N, S, LDS, P, LDP, VL,
     $                   LDVL, VR, LDVR, MM, M, WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          HOWMNY, SIDE
      INTEGER            INFO, LDP, LDS, LDVL, LDVR, M, MM, N
*     ..
```

```
*     .. Array Arguments ..
      LOGICAL            SELECT( * )
      DOUBLE PRECISION   RWORK( * )
      COMPLEX*16         P( LDP, * ), S( LDS, * ), VL( LDVL, * ),
     $                   VR( LDVR, * ), WORK( * )
*     ..
*
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      COMPLEX*16         CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0D+0, 0.0D+0 ),
     $                   CONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            COMPL, COMPR, ILALL, ILBACK, ILBBAD, ILCOMP,
     $                   LSA, LSB
      INTEGER            I, IBEG, IEIG, IEND, IHWMNY, IM, ISIDE, ISRC,
     $                   J, JE, JR
      DOUBLE PRECISION   ACOEFA, ACOEFF, ANORM, ASCALE, BCOEFA, BIG,
     $                   BIGNUM, BNORM, BSCALE, DMIN, SAFMIN, SBETA,
     $                   SCALE, SMALL, TEMP, ULP, XMAX
      COMPLEX*16         BCOEFF, CA, CB, D, SALPHA, SUM, SUMA, SUMB, X
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH
      COMPLEX*16         ZLADIV
      EXTERNAL           LSAME, DLAMCH, ZLADIV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, XERBLA, ZGEMV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DCMPLX, DCONJG, DIMAG, MAX, MIN
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   ABS1
*     ..
*     .. Statement Function definitions ..
      ABS1( X ) = ABS( DBLE( X ) ) + ABS( DIMAG( X ) )
*     ..
*     .. Executable Statements ..
*
*     Decode and Test the input parameters
*
      IF( LSAME( HOWMNY, 'A' ) ) THEN
```

```
         IHWMNY = 1
         ILALL = .TRUE.
         ILBACK = .FALSE.
      ELSE IF( LSAME( HOWMNY, 'S' ) ) THEN
         IHWMNY = 2
         ILALL = .FALSE.
         ILBACK = .FALSE.
      ELSE IF( LSAME( HOWMNY, 'B' ) ) THEN
         IHWMNY = 3
         ILALL = .TRUE.
         ILBACK = .TRUE.
      ELSE
         IHWMNY = -1
      END IF
*
      IF( LSAME( SIDE, 'R' ) ) THEN
         ISIDE = 1
         COMPL = .FALSE.
         COMPR = .TRUE.
      ELSE IF( LSAME( SIDE, 'L' ) ) THEN
         ISIDE = 2
         COMPL = .TRUE.
         COMPR = .FALSE.
      ELSE IF( LSAME( SIDE, 'B' ) ) THEN
         ISIDE = 3
         COMPL = .TRUE.
         COMPR = .TRUE.
      ELSE
         ISIDE = -1
      END IF
*
      INFO = 0
      IF( ISIDE.LT.0 ) THEN
         INFO = -1
      ELSE IF( IHWMNY.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDS.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDP.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZTGEVC', -INFO )
         RETURN
      END IF
*
*     Count the number of eigenvectors
*
```

```
      IF( .NOT.ILALL ) THEN
         IM = 0
         DO 10 J = 1, N
            IF( SELECT( J ) )
     $         IM = IM + 1
   10    CONTINUE
      ELSE
         IM = N
      END IF
*
*     Check diagonal of B
*
      ILBBAD = .FALSE.
      DO 20 J = 1, N
         IF( DIMAG( P( J, J ) ).NE.ZERO )
     $      ILBBAD = .TRUE.
   20 CONTINUE
*
      IF( ILBBAD ) THEN
         INFO = -7
      ELSE IF( COMPL .AND. LDVL.LT.N .OR. LDVL.LT.1 ) THEN
         INFO = -10
      ELSE IF( COMPR .AND. LDVR.LT.N .OR. LDVR.LT.1 ) THEN
         INFO = -12
      ELSE IF( MM.LT.IM ) THEN
         INFO = -13
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZTGEVC', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      M = IM
      IF( N.EQ.0 )
     $   RETURN
*
*     Machine Constants
*
      SAFMIN = DLAMCH( 'Safe minimum' )
      BIG = ONE / SAFMIN
      CALL DLABAD( SAFMIN, BIG )
      ULP = DLAMCH( 'Epsilon' )*DLAMCH( 'Base' )
      SMALL = SAFMIN*N / ULP
      BIG = ONE / SMALL
      BIGNUM = ONE / ( SAFMIN*N )
*
*     Compute the 1-norm of each column of the strictly upper triangular
*     part of A and B to check for possible overflow in the triangular
```

```
*     solver.
*
      ANORM = ABS1( S( 1, 1 ) )
      BNORM = ABS1( P( 1, 1 ) )
      RWORK( 1 ) = ZERO
      RWORK( N+1 ) = ZERO
      DO 40 J = 2, N
         RWORK( J ) = ZERO
         RWORK( N+J ) = ZERO
         DO 30 I = 1, J - 1
            RWORK( J ) = RWORK( J ) + ABS1( S( I, J ) )
            RWORK( N+J ) = RWORK( N+J ) + ABS1( P( I, J ) )
   30    CONTINUE
         ANORM = MAX( ANORM, RWORK( J )+ABS1( S( J, J ) ) )
         BNORM = MAX( BNORM, RWORK( N+J )+ABS1( P( J, J ) ) )
   40 CONTINUE
*
      ASCALE = ONE / MAX( ANORM, SAFMIN )
      BSCALE = ONE / MAX( BNORM, SAFMIN )
*
*     Left eigenvectors
*
      IF( COMPL ) THEN
         IEIG = 0
*
*        Main loop over eigenvalues
*
         DO 140 JE = 1, N
            IF( ILALL ) THEN
               ILCOMP = .TRUE.
            ELSE
               ILCOMP = SELECT( JE )
            END IF
            IF( ILCOMP ) THEN
               IEIG = IEIG + 1
*
               IF( ABS1( S( JE, JE ) ).LE.SAFMIN .AND.
     $             ABS( DBLE( P( JE, JE ) ) ).LE.SAFMIN ) THEN
*
*                Singular matrix pencil -- return unit eigenvector
*
                 DO 50 JR = 1, N
                    VL( JR, IEIG ) = CZERO
   50            CONTINUE
                 VL( IEIG, IEIG ) = CONE
                 GO TO 140
               END IF
*
*              Non-singular eigenvalue:
*              Compute coefficients  a  and  b  in
```

```
*                  H
*                y  ( a A - b B ) = 0
*
              TEMP = ONE / MAX( ABS1( S( JE, JE ) )*ASCALE,
     $                ABS( DBLE( P( JE, JE ) ) )*BSCALE, SAFMIN )
              SALPHA = ( TEMP*S( JE, JE ) )*ASCALE
              SBETA = ( TEMP*DBLE( P( JE, JE ) ) )*BSCALE
              ACOEFF = SBETA*ASCALE
              BCOEFF = SALPHA*BSCALE
*
*             Scale to avoid underflow
*
              LSA = ABS( SBETA ).GE.SAFMIN .AND. ABS( ACOEFF ).LT.SMALL
              LSB = ABS1( SALPHA ).GE.SAFMIN .AND. ABS1( BCOEFF ).LT.
     $               SMALL
*
              SCALE = ONE
              IF( LSA )
     $           SCALE = ( SMALL / ABS( SBETA ) )*MIN( ANORM, BIG )
              IF( LSB )
     $           SCALE = MAX( SCALE, ( SMALL / ABS1( SALPHA ) )*
     $                   MIN( BNORM, BIG ) )
              IF( LSA .OR. LSB ) THEN
                 SCALE = MIN( SCALE, ONE /
     $                   ( SAFMIN*MAX( ONE, ABS( ACOEFF ),
     $                   ABS1( BCOEFF ) ) ) )
                 IF( LSA ) THEN
                    ACOEFF = ASCALE*( SCALE*SBETA )
                 ELSE
                    ACOEFF = SCALE*ACOEFF
                 END IF
                 IF( LSB ) THEN
                    BCOEFF = BSCALE*( SCALE*SALPHA )
                 ELSE
                    BCOEFF = SCALE*BCOEFF
                 END IF
              END IF
*
              ACOEFA = ABS( ACOEFF )
              BCOEFA = ABS1( BCOEFF )
              XMAX = ONE
              DO 60 JR = 1, N
                 WORK( JR ) = CZERO
   60         CONTINUE
              WORK( JE ) = CONE
              DMIN = MAX( ULP*ACOEFA*ANORM, ULP*BCOEFA*BNORM, SAFMIN )
*
*                                          H
*             Triangular solve of  (a A - b B)  y = 0
*
```

```
*                                    H
*              (rowwise in  (a A - b B) , or columnwise in a A - b B)
*
               DO 100 J = JE + 1, N
*
*                 Compute
*                       j-1
*                 SUM = sum  conjg( a*S(k,j) - b*P(k,j) )*x(k)
*                       k=je
*                 (Scale if necessary)
*
                  TEMP = ONE / XMAX
                  IF( ACOEFA*RWORK( J )+BCOEFA*RWORK( N+J ).GT.BIGNUM*
     $                TEMP ) THEN
                     DO 70 JR = JE, J - 1
                        WORK( JR ) = TEMP*WORK( JR )
   70                CONTINUE
                     XMAX = ONE
                  END IF
                  SUMA = CZERO
                  SUMB = CZERO
*
                  DO 80 JR = JE, J - 1
                     SUMA = SUMA + DCONJG( S( JR, J ) )*WORK( JR )
                     SUMB = SUMB + DCONJG( P( JR, J ) )*WORK( JR )
   80             CONTINUE
                  SUM = ACOEFF*SUMA - DCONJG( BCOEFF )*SUMB
*
*                 Form x(j) = - SUM / conjg( a*S(j,j) - b*P(j,j) )
*
*                 with scaling and perturbation of the denominator
*
                  D = DCONJG( ACOEFF*S( J, J )-BCOEFF*P( J, J ) )
                  IF( ABS1( D ).LE.DMIN )
     $               D = DCMPLX( DMIN )
*
                  IF( ABS1( D ).LT.ONE ) THEN
                     IF( ABS1( SUM ).GE.BIGNUM*ABS1( D ) ) THEN
                        TEMP = ONE / ABS1( SUM )
                        DO 90 JR = JE, J - 1
                           WORK( JR ) = TEMP*WORK( JR )
   90                   CONTINUE
                        XMAX = TEMP*XMAX
                        SUM = TEMP*SUM
                     END IF
                  END IF
                  WORK( J ) = ZLADIV( -SUM, D )
                  XMAX = MAX( XMAX, ABS1( WORK( J ) ) )
  100          CONTINUE
*
```

```
*              Back transform eigenvector if HOWMNY='B'.
*
               IF( ILBACK ) THEN
                  CALL ZGEMV( 'N', N, N+1-JE, CONE, VL( 1, JE ), LDVL,
     $                        WORK( JE ), 1, CZERO, WORK( N+1 ), 1 )
                  ISRC = 2
                  IBEG = 1
               ELSE
                  ISRC = 1
                  IBEG = JE
               END IF
*
*              Copy and scale eigenvector into column of VL
*
               XMAX = ZERO
               DO 110 JR = IBEG, N
                  XMAX = MAX( XMAX, ABS1( WORK( ( ISRC-1 )*N+JR ) ) )
  110          CONTINUE
*
               IF( XMAX.GT.SAFMIN ) THEN
                  TEMP = ONE / XMAX
                  DO 120 JR = IBEG, N
                     VL( JR, IEIG ) = TEMP*WORK( ( ISRC-1 )*N+JR )
  120             CONTINUE
               ELSE
                  IBEG = N + 1
               END IF
*
               DO 130 JR = 1, IBEG - 1
                  VL( JR, IEIG ) = CZERO
  130          CONTINUE
*
            END IF
  140    CONTINUE
      END IF
*
*     Right eigenvectors
*
      IF( COMPR ) THEN
         IEIG = IM + 1
*
*        Main loop over eigenvalues
*
         DO 250 JE = N, 1, -1
            IF( ILALL ) THEN
               ILCOMP = .TRUE.
            ELSE
               ILCOMP = SELECT( JE )
            END IF
            IF( ILCOMP ) THEN
```

```
               IEIG = IEIG - 1
*
               IF( ABS1( S( JE, JE ) ).LE.SAFMIN .AND.
     $             ABS( DBLE( P( JE, JE ) ) ).LE.SAFMIN ) THEN
*
*                 Singular matrix pencil -- return unit eigenvector
*
                  DO 150 JR = 1, N
                     VR( JR, IEIG ) = CZERO
  150             CONTINUE
                  VR( IEIG, IEIG ) = CONE
                  GO TO 250
               END IF
*
*              Non-singular eigenvalue:
*              Compute coefficients  a  and  b  in
*
*              ( a A - b B ) x  = 0
*
               TEMP = ONE / MAX( ABS1( S( JE, JE ) )*ASCALE,
     $                ABS( DBLE( P( JE, JE ) ) )*BSCALE, SAFMIN )
               SALPHA = ( TEMP*S( JE, JE ) )*ASCALE
               SBETA = ( TEMP*DBLE( P( JE, JE ) ) )*BSCALE
               ACOEFF = SBETA*ASCALE
               BCOEFF = SALPHA*BSCALE
*
*              Scale to avoid underflow
*
               LSA = ABS( SBETA ).GE.SAFMIN .AND. ABS( ACOEFF ).LT.SMALL
               LSB = ABS1( SALPHA ).GE.SAFMIN .AND. ABS1( BCOEFF ).LT.
     $               SMALL
*
               SCALE = ONE
               IF( LSA )
     $            SCALE = ( SMALL / ABS( SBETA ) )*MIN( ANORM, BIG )
               IF( LSB )
     $            SCALE = MAX( SCALE, ( SMALL / ABS1( SALPHA ) )*
     $                    MIN( BNORM, BIG ) )
               IF( LSA .OR. LSB ) THEN
                  SCALE = MIN( SCALE, ONE /
     $                    ( SAFMIN*MAX( ONE, ABS( ACOEFF ),
     $                    ABS1( BCOEFF ) ) ) )
                  IF( LSA ) THEN
                     ACOEFF = ASCALE*( SCALE*SBETA )
                  ELSE
                     ACOEFF = SCALE*ACOEFF
                  END IF
                  IF( LSB ) THEN
                     BCOEFF = BSCALE*( SCALE*SALPHA )
                  ELSE
```

```
                  BCOEFF = SCALE*BCOEFF
               END IF
            END IF
*
            ACOEFA = ABS( ACOEFF )
            BCOEFA = ABS1( BCOEFF )
            XMAX = ONE
            DO 160 JR = 1, N
               WORK( JR ) = CZERO
  160       CONTINUE
            WORK( JE ) = CONE
            DMIN = MAX( ULP*ACOEFA*ANORM, ULP*BCOEFA*BNORM, SAFMIN )
*
*           Triangular solve of  (a A - b B) x = 0  (columnwise)
*
*           WORK(1:j-1) contains sums w,
*           WORK(j+1:JE) contains x
*
            DO 170 JR = 1, JE - 1
               WORK( JR ) = ACOEFF*S( JR, JE ) - BCOEFF*P( JR, JE )
  170       CONTINUE
            WORK( JE ) = CONE
*
            DO 210 J = JE - 1, 1, -1
*
*              Form x(j) := - w(j) / d
*              with scaling and perturbation of the denominator
*
               D = ACOEFF*S( J, J ) - BCOEFF*P( J, J )
               IF( ABS1( D ).LE.DMIN )
     $            D = DCMPLX( DMIN )
*
               IF( ABS1( D ).LT.ONE ) THEN
                  IF( ABS1( WORK( J ) ).GE.BIGNUM*ABS1( D ) ) THEN
                     TEMP = ONE / ABS1( WORK( J ) )
                     DO 180 JR = 1, JE
                        WORK( JR ) = TEMP*WORK( JR )
  180                CONTINUE
                  END IF
               END IF
*
               WORK( J ) = ZLADIV( -WORK( J ), D )
*
               IF( J.GT.1 ) THEN
*
*                 w = w + x(j)*(a S(*,j) - b P(*,j) ) with scaling
*
                  IF( ABS1( WORK( J ) ).GT.ONE ) THEN
                     TEMP = ONE / ABS1( WORK( J ) )
                     IF( ACOEFA*RWORK( J )+BCOEFA*RWORK( N+J ).GE.
```

```
     $                     BIGNUM*TEMP ) THEN
                          DO 190 JR = 1, JE
                             WORK( JR ) = TEMP*WORK( JR )
  190                     CONTINUE
                       END IF
                    END IF
*
                    CA = ACOEFF*WORK( J )
                    CB = BCOEFF*WORK( J )
                    DO 200 JR = 1, J - 1
                       WORK( JR ) = WORK( JR ) + CA*S( JR, J ) -
     $                             CB*P( JR, J )
  200               CONTINUE
                 END IF
  210          CONTINUE
*
*              Back transform eigenvector if HOWMNY='B'.
*
               IF( ILBACK ) THEN
                  CALL ZGEMV( 'N', N, JE, CONE, VR, LDVR, WORK, 1,
     $                        CZERO, WORK( N+1 ), 1 )
                  ISRC = 2
                  IEND = N
               ELSE
                  ISRC = 1
                  IEND = JE
               END IF
*
*              Copy and scale eigenvector into column of VR
*
               XMAX = ZERO
               DO 220 JR = 1, IEND
                  XMAX = MAX( XMAX, ABS1( WORK( ( ISRC-1 )*N+JR ) ) )
  220          CONTINUE
*
               IF( XMAX.GT.SAFMIN ) THEN
                  TEMP = ONE / XMAX
                  DO 230 JR = 1, IEND
                     VR( JR, IEIG ) = TEMP*WORK( ( ISRC-1 )*N+JR )
  230             CONTINUE
               ELSE
                  IEND = 0
               END IF
*
               DO 240 JR = IEND + 1, N
                  VR( JR, IEIG ) = CZERO
  240          CONTINUE
*
            END IF
  250    CONTINUE
```

```
      END IF
*
      RETURN
*
*     End of ZTGEVC
*
      END
```

———-

35.39.4 lisp code

— **ztgevc.lisp** —

```
(let* ((zero 0.0d0)
       (one 1.0d0)
       (czero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (cone (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (f2cl-lib:complex16) czero)
           (type (f2cl-lib:complex16) cone)
           (ignorable zero one czero cone))
  (defun ztgevc
         (side howmny select n s lds p ldp vl ldvl vr ldvr mm m work rwork
          info)
    (declare (type (array double-float (*)) rwork)
             (type (array f2cl-lib:complex16 (*)) work vr vl p s)
             (type (f2cl-lib:integer4) info m mm ldvr ldvl ldp lds n)
             (type (array f2cl-lib:logical (*)) select)
             (type (string *) howmny side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (howmny character howmny-%data% howmny-%offset%)
         (select f2cl-lib:logical select-%data% select-%offset%)
         (s f2cl-lib:complex16 s-%data% s-%offset%)
         (p f2cl-lib:complex16 p-%data% p-%offset%)
         (vl f2cl-lib:complex16 vl-%data% vl-%offset%)
         (vr f2cl-lib:complex16 vr-%data% vr-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (labels ((abs1 (x)
                 (+ (abs (f2cl-lib:dble x)) (abs (f2cl-lib:dimag x)))))
        (declare
         (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
          abs1))
        (prog ((bcoeff #C(0.0d0 0.0d0)) (ca #C(0.0d0 0.0d0))
               (cb #C(0.0d0 0.0d0)) (d #C(0.0d0 0.0d0))
```

```
     (salpha #C(0.0d0 0.0d0)) (sum #C(0.0d0 0.0d0))
     (suma #C(0.0d0 0.0d0)) (sumb #C(0.0d0 0.0d0))
     (x #C(0.0d0 0.0d0)) (acoefa 0.0d0) (acoeff 0.0d0) (anorm 0.0d0)
     (ascale 0.0d0) (bcoefa 0.0d0) (big 0.0d0) (bignum 0.0d0)
     (bnorm 0.0d0) (bscale 0.0d0) (dmin 0.0d0) (safmin 0.0d0)
     (sbeta 0.0d0) (scale 0.0d0) (small 0.0d0) (temp 0.0d0)
     (ulp 0.0d0) (xmax 0.0d0) (i 0) (ibeg 0) (ieig 0) (iend 0)
     (ihwmny 0) (im 0) (iside 0) (isrc 0) (j 0) (je 0) (jr 0)
     (compl nil) (compr nil) (ilall nil) (ilback nil) (ilbbad nil)
     (ilcomp nil) (lsa nil) (lsb nil))
(declare
 (type (f2cl-lib:complex16) bcoeff ca cb d salpha sum suma sumb x)
 (type (double-float) acoefa acoeff anorm ascale bcoefa big bignum
  bnorm bscale dmin safmin sbeta scale small temp ulp xmax)
 (type (f2cl-lib:integer4) i ibeg ieig iend ihwmny im iside isrc j je
  jr)
 (type f2cl-lib:logical compl compr ilall ilback ilbbad ilcomp lsa
  lsb))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame howmny "A")
    (declare (ignore var-1))
    (when var-0 (setf howmny var-0))
    ret-val)
  (setf ihwmny 1) (setf ilall f2cl-lib:%true%)
  (setf ilback f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame howmny "S")
    (declare (ignore var-1))
    (when var-0 (setf howmny var-0))
    ret-val)
  (setf ihwmny 2) (setf ilall f2cl-lib:%false%)
  (setf ilback f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame howmny "B")
    (declare (ignore var-1))
    (when var-0 (setf howmny var-0))
    ret-val)
  (setf ihwmny 3) (setf ilall f2cl-lib:%true%)
  (setf ilback f2cl-lib:%true%))
 (t (setf ihwmny -1)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame side "R")
    (declare (ignore var-1))
    (when var-0 (setf side var-0))
    ret-val)
  (setf iside 1) (setf compl f2cl-lib:%false%)
  (setf compr f2cl-lib:%true%))
 ((multiple-value-bind (ret-val var-0 var-1)
```

```
      (lsame side "L")
    (declare (ignore var-1))
    (when var-0 (setf side var-0))
    ret-val)
  (setf iside 2) (setf compl f2cl-lib:%true%)
  (setf compr f2cl-lib:%false%))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame side "B")
    (declare (ignore var-1))
    (when var-0 (setf side var-0))
    ret-val)
  (setf iside 3) (setf compl f2cl-lib:%true%)
  (setf compr f2cl-lib:%true%))
 (t (setf iside -1)))
(setf info 0)
(cond ((< iside 0) (setf info -1)) ((< ihwmny 0) (setf info -2))
      ((< n 0) (setf info -4))
      ((< lds
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -6))
      ((< ldp
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -8)))
(cond
 ((/= info 0) (xerbla "ZTGEVC" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((not ilall) (setf im 0)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (if (f2cl-lib:fref select-%data% (j) ((1 *)) select-%offset%)
          (setf im (f2cl-lib:int-add im 1)))
     label10)))
 (t (setf im n)))
(setf ilbbad f2cl-lib:%false%)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (if (/=
         (f2cl-lib:dimag
          (f2cl-lib:fref p-%data% (j j) ((1 ldp) (1 *)) p-%offset%))
         zero)
        (setf ilbbad f2cl-lib:%true%))
   label20))
(cond (ilbbad (setf info -7))
      ((or (and compl (< ldvl n)) (< ldvl 1)) (setf info -10))
      ((or (and compr (< ldvr n)) (< ldvr 1)) (setf info -12))
      ((< mm im) (setf info -13)))
(cond
```

```
 ((/= info 0) (xerbla "ZTGEVC" (f2cl-lib:int-sub info))
  (go end_label)))
(setf m im)
(if (= n 0)
    (go end_label))
(setf safmin (dlamch "Safe minimum"))
(setf big (/ one safmin))
(multiple-value-bind (var-0 var-1)
    (dlabad safmin big)
  (declare (ignore))
  (when var-0 (setf safmin var-0))
  (when var-1 (setf big var-1)))
(setf ulp (* (dlamch "Epsilon") (dlamch "Base")))
(setf small (/ (* safmin n) ulp))
(setf big (/ one small))
(setf bignum (/ one (* safmin n)))
(setf anorm
        (abs1
         (f2cl-lib:fref s-%data% (1 1) ((1 lds) (1 *)) s-%offset%)))
(setf bnorm
        (abs1
         (f2cl-lib:fref p-%data% (1 1) ((1 ldp) (1 *)) p-%offset%)))
(setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%) zero)
(setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add n 1)) ((1 *))
                     rwork-%offset%)
        zero)
(f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (j) ((1 *)) rwork-%offset%)
            zero)
    (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add n j))
                         ((1 *)) rwork-%offset%)
            zero)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref rwork-%data% (j) ((1 *)) rwork-%offset%)
                (+
                 (f2cl-lib:fref rwork-%data% (j) ((1 *))
                                rwork-%offset%)
                 (abs1
                  (f2cl-lib:fref s-%data% (i j) ((1 lds) (1 *))
                                 s-%offset%))))
        (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add n j))
                             ((1 *)) rwork-%offset%)
                (+
                 (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add n j))
                                ((1 *)) rwork-%offset%)
```

```
                    (abs1
                     (f2cl-lib:fref p-%data% (i j) ((1 ldp) (1 *))
                                    p-%offset%))))
        label30))
    (setf anorm
            (max anorm
                 (+
                  (f2cl-lib:fref rwork-%data% (j) ((1 *))
                                 rwork-%offset%)
                  (abs1
                   (f2cl-lib:fref s-%data% (j j) ((1 lds) (1 *))
                                  s-%offset%)))))
    (setf bnorm
            (max bnorm
                 (+
                  (f2cl-lib:fref rwork-%data%
                                 ((f2cl-lib:int-add n j)) ((1 *))
                                 rwork-%offset%)
                  (abs1
                   (f2cl-lib:fref p-%data% (j j) ((1 ldp) (1 *))
                                  p-%offset%)))))
   label40))
(setf ascale (/ one (max anorm safmin)))
(setf bscale (/ one (max bnorm safmin)))
(cond
 (compl (setf ieig 0)
  (f2cl-lib:fdo (je 1 (f2cl-lib:int-add je 1))
                ((> je n) nil)
    (tagbody
      (cond (ilall (setf ilcomp f2cl-lib:%true%))
            (t
             (setf ilcomp
                     (f2cl-lib:fref select-%data% (je) ((1 *))
                                    select-%offset%))))
      (cond
       (ilcomp (setf ieig (f2cl-lib:int-add ieig 1))
        (cond
         ((and
           (<= (abs1 (f2cl-lib:fref s (je je) ((1 lds) (1 *))))
               safmin)
           (<=
            (abs
             (f2cl-lib:dble
              (f2cl-lib:fref p (je je) ((1 ldp) (1 *)))))
            safmin))
          (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                        ((> jr n) nil)
            (tagbody
              (setf (f2cl-lib:fref vl-%data% (jr ieig)
                                   ((1 ldvl) (1 *)) vl-%offset%)
```

```
              czero)
     label50))
  (setf (f2cl-lib:fref vl-%data% (ieig ieig) ((1 ldvl) (1 *))
                       vl-%offset%)
          cone)
  (go label140)))
(setf temp
        (/ one
           (max
            (*
             (abs1
              (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                             s-%offset%))
             ascale)
            (*
             (abs
              (f2cl-lib:dble
               (f2cl-lib:fref p-%data% (je je)
                              ((1 ldp) (1 *)) p-%offset%)))
             bscale)
            safmin)))
(setf salpha
        (* temp
           (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                          s-%offset%)
           ascale))
(setf sbeta
        (* temp
           (f2cl-lib:dble
            (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
                           p-%offset%))
           bscale))
(setf acoeff (* sbeta ascale))
(setf bcoeff (* salpha bscale))
(setf lsa
        (and (>= (abs sbeta) safmin) (< (abs acoeff) small)))
(setf lsb
        (and (>= (abs1 salpha) safmin)
             (< (abs1 bcoeff) small)))
(setf scale one)
(if lsa
    (setf scale (* (/ small (abs sbeta)) (min anorm big))))
(if lsb
    (setf scale
            (max scale
                 (* (/ small (abs1 salpha))
                    (min bnorm big)))))
(cond
 ((or lsa lsb)
  (setf scale
```

```
          (min scale
               (/ one
                  (* safmin
                     (max one (abs acoeff) (abs1 bcoeff))))))
  (cond (lsa (setf acoeff (* ascale (* scale sbeta))))
        (t (setf acoeff (* scale acoeff))))
  (cond (lsb (setf bcoeff (* bscale (* scale salpha))))
        (t (setf bcoeff (* scale bcoeff))))))
(setf acoefa (abs acoeff)) (setf bcoefa (abs1 bcoeff))
(setf xmax one)
(f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
              ((> jr n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (jr) ((1 *))
                         work-%offset%)
            czero)
   label60))
(setf (f2cl-lib:fref work-%data% (je) ((1 *)) work-%offset%)
        cone)
(setf dmin
        (max (* ulp acoefa anorm) (* ulp bcoefa bnorm)
             safmin))
(f2cl-lib:fdo (j (f2cl-lib:int-add je 1)
               (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf temp (/ one xmax))
    (cond
     ((>
       (+ (* acoefa (f2cl-lib:fref rwork (j) ((1 *))))
          (* bcoefa
             (f2cl-lib:fref rwork ((f2cl-lib:int-add n j))
                            ((1 *)))))
       (* bignum temp))
      (f2cl-lib:fdo (jr je (f2cl-lib:int-add jr 1))
                    ((> jr
                        (f2cl-lib:int-add j
                                          (f2cl-lib:int-sub
                                           1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (jr) ((1 *))
                               work-%offset%)
                  (* temp
                     (f2cl-lib:fref work-%data% (jr) ((1 *))
                                    work-%offset%)))
         label70))
      (setf xmax one)))
    (setf suma czero)
    (setf sumb czero)
```

```
(f2cl-lib:fdo (jr je (f2cl-lib:int-add jr 1))
              ((> jr
                  (f2cl-lib:int-add j
                                    (f2cl-lib:int-sub
                                     1)))
               nil)
  (tagbody
    (setf suma
            (+ suma
               (*
                (f2cl-lib:dconjg
                 (f2cl-lib:fref s-%data% (jr j)
                                ((1 lds) (1 *))
                                s-%offset%))
                (f2cl-lib:fref work-%data% (jr) ((1 *))
                               work-%offset%))))
    (setf sumb
            (+ sumb
               (*
                (f2cl-lib:dconjg
                 (f2cl-lib:fref p-%data% (jr j)
                                ((1 ldp) (1 *))
                                p-%offset%))
                (f2cl-lib:fref work-%data% (jr) ((1 *))
                               work-%offset%))))
   label80))
(setf sum
        (- (* acoeff suma)
           (* (f2cl-lib:dconjg bcoeff) sumb)))
(setf d
        (coerce
         (f2cl-lib:dconjg
          (-
           (* acoeff
              (f2cl-lib:fref s-%data% (j j)
                             ((1 lds) (1 *)) s-%offset%))
           (* bcoeff
              (f2cl-lib:fref p-%data% (j j)
                             ((1 ldp) (1 *))
                             p-%offset%))))
         'f2cl-lib:complex16))
(if (<= (abs1 d) dmin)
    (setf d (f2cl-lib:dcmplx dmin)))
(cond
 ((< (abs1 d) one)
  (cond
   ((>= (abs1 sum) (* bignum (abs1 d)))
    (setf temp (/ one (abs1 sum)))
    (f2cl-lib:fdo (jr je (f2cl-lib:int-add jr 1))
                  ((> jr
```

```
                              (f2cl-lib:int-add j
                                                (f2cl-lib:int-sub
                                                 1)))
                          nil)
                (tagbody
                  (setf (f2cl-lib:fref work-%data% (jr) ((1 *))
                                       work-%offset%)
                        (* temp
                           (f2cl-lib:fref work-%data% (jr)
                                          ((1 *))
                                          work-%offset%)))
                 label90))
              (setf xmax (* temp xmax)) (setf sum (* temp sum))))))
        (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                             work-%offset%)
                  (zladiv (- sum) d))
        (setf xmax
                  (max xmax
                       (abs1
                        (f2cl-lib:fref work-%data% (j) ((1 *))
                                       work-%offset%))))
       label100))
    (cond
     (ilback
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (zgemv "N" n
           (f2cl-lib:int-sub (f2cl-lib:int-add n 1) je) cone
           (f2cl-lib:array-slice vl-%data% f2cl-lib:complex16
                                 (1 je) ((1 ldvl) (1 *))
                                 vl-%offset%)
           ldvl
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                 (je) ((1 *)) work-%offset%)
           1 czero
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                 ((+ n 1)) ((1 *)) work-%offset%)
           1)
        (declare
         (ignore var-0 var-2 var-4 var-6 var-7 var-9 var-10))
        (when var-1 (setf n var-1))
        (when var-3 (setf cone var-3))
        (when var-5 (setf ldvl var-5))
        (when var-8 (setf czero var-8)))
      (setf isrc 2) (setf ibeg 1))
     (t (setf isrc 1) (setf ibeg je)))
    (setf xmax zero)
    (f2cl-lib:fdo (jr ibeg (f2cl-lib:int-add jr 1))
                  ((> jr n) nil)
```

```
            (tagbody
              (setf xmax
                      (max xmax
                           (abs1
                            (f2cl-lib:fref work-%data%
                                           ((f2cl-lib:int-add
                                             (f2cl-lib:int-mul
                                              (f2cl-lib:int-sub isrc 1)
                                              n)
                                             jr))
                                           ((1 *)) work-%offset%))))
             label110))
           (cond
            ((> xmax safmin) (setf temp (/ one xmax))
             (f2cl-lib:fdo (jr ibeg (f2cl-lib:int-add jr 1))
                           ((> jr n) nil)
               (tagbody
                 (setf (f2cl-lib:fref vl-%data% (jr ieig)
                                      ((1 ldvl) (1 *)) vl-%offset%)
                       (* temp
                          (f2cl-lib:fref work-%data%
                                         ((f2cl-lib:int-add
                                           (f2cl-lib:int-mul
                                            (f2cl-lib:int-sub isrc 1)
                                            n)
                                           jr))
                                         ((1 *)) work-%offset%)))
              label120)))
            (t (setf ibeg (f2cl-lib:int-add n 1))))
           (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                         ((> jr
                             (f2cl-lib:int-add ibeg
                                               (f2cl-lib:int-sub 1)))
                          nil)
             (tagbody
               (setf (f2cl-lib:fref vl-%data% (jr ieig) ((1 ldvl) (1 *))
                                    vl-%offset%)
                     czero)
             label130))))
      label140))))
(cond
 (compr (setf ieig (f2cl-lib:int-add im 1))
  (f2cl-lib:fdo (je n (f2cl-lib:int-add je (f2cl-lib:int-sub 1)))
                ((> je 1) nil)
    (tagbody
      (cond (ilall (setf ilcomp f2cl-lib:%true%))
            (t
             (setf ilcomp
                     (f2cl-lib:fref select-%data% (je) ((1 *))
                                    select-%offset%))))
```

```
(cond
 (ilcomp (setf ieig (f2cl-lib:int-sub ieig 1))
  (cond
   ((and
     (<= (abs1 (f2cl-lib:fref s (je je) ((1 lds) (1 *))))
         safmin)
     (<=
      (abs
       (f2cl-lib:dble
        (f2cl-lib:fref p (je je) ((1 ldp) (1 *)))))
      safmin))
    (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                  ((> jr n) nil)
      (tagbody
        (setf (f2cl-lib:fref vr-%data% (jr ieig)
                             ((1 ldvr) (1 *)) vr-%offset%)
              czero)
      label150))
    (setf (f2cl-lib:fref vr-%data% (ieig ieig) ((1 ldvr) (1 *))
                         vr-%offset%)
          cone)
    (go label250)))
  (setf temp
        (/ one
           (max
            (*
             (abs1
              (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                             s-%offset%))
             ascale)
            (*
             (abs
              (f2cl-lib:dble
               (f2cl-lib:fref p-%data% (je je)
                              ((1 ldp) (1 *)) p-%offset%)))
             bscale)
            safmin)))
  (setf salpha
        (* temp
           (f2cl-lib:fref s-%data% (je je) ((1 lds) (1 *))
                          s-%offset%)
           ascale))
  (setf sbeta
        (* temp
           (f2cl-lib:dble
            (f2cl-lib:fref p-%data% (je je) ((1 ldp) (1 *))
                           p-%offset%))
           bscale))
  (setf acoeff (* sbeta ascale))
  (setf bcoeff (* salpha bscale))
```

```
(setf lsa
        (and (>= (abs sbeta) safmin) (< (abs acoeff) small)))
(setf lsb
        (and (>= (abs1 salpha) safmin)
             (< (abs1 bcoeff) small)))
(setf scale one)
(if lsa
    (setf scale (* (/ small (abs sbeta)) (min anorm big))))
(if lsb
    (setf scale
            (max scale
                 (* (/ small (abs1 salpha))
                    (min bnorm big)))))
(cond
 ((or lsa lsb)
  (setf scale
          (min scale
               (/ one
                  (* safmin
                     (max one (abs acoeff) (abs1 bcoeff))))))
  (cond (lsa (setf acoeff (* ascale (* scale sbeta))))
        (t (setf acoeff (* scale acoeff))))
  (cond (lsb (setf bcoeff (* bscale (* scale salpha))))
        (t (setf bcoeff (* scale bcoeff))))))
(setf acoefa (abs acoeff)) (setf bcoefa (abs1 bcoeff))
(setf xmax one)
(f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
              ((> jr n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (jr) ((1 *))
                         work-%offset%)
            czero)
   label160))
(setf (f2cl-lib:fref work-%data% (je) ((1 *)) work-%offset%)
        cone)
(setf dmin
        (max (* ulp acoefa anorm) (* ulp bcoefa bnorm)
             safmin))
(f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
              ((> jr
                  (f2cl-lib:int-add je (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (jr) ((1 *))
                         work-%offset%)
            (-
             (* acoeff
                (f2cl-lib:fref s-%data% (jr je)
                               ((1 lds) (1 *)) s-%offset%))
             (* bcoeff
```

```
                (f2cl-lib:fref p-%data% (jr je)
                               ((1 ldp) (1 *)) p-%offset%))))
   label170))
(setf (f2cl-lib:fref work-%data% (je) ((1 *)) work-%offset%)
        cone)
(f2cl-lib:fdo (j (f2cl-lib:int-add je (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j 1) nil)
  (tagbody
    (setf d
            (-
             (* acoeff
                (f2cl-lib:fref s-%data% (j j) ((1 lds) (1 *))
                               s-%offset%))
             (* bcoeff
                (f2cl-lib:fref p-%data% (j j) ((1 ldp) (1 *))
                               p-%offset%))))
    (if (<= (abs1 d) dmin)
        (setf d (f2cl-lib:dcmplx dmin)))
    (cond
     ((< (abs1 d) one)
      (cond
       ((>= (abs1 (f2cl-lib:fref work (j) ((1 *))))
            (* bignum (abs1 d)))
        (setf temp
                (/ one
                   (abs1
                    (f2cl-lib:fref work-%data% (j) ((1 *))
                                   work-%offset%))))
        (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                      ((> jr je) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (jr) ((1 *))
                                 work-%offset%)
                    (* temp
                       (f2cl-lib:fref work-%data% (jr)
                                      ((1 *))
                                      work-%offset%)))
             label180))))))
    (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                         work-%offset%)
            (zladiv
             (-
              (f2cl-lib:fref work-%data% (j) ((1 *))
                             work-%offset%))
             d))
    (cond
     ((> j 1)
      (cond
       ((> (abs1 (f2cl-lib:fref work (j) ((1 *)))) one)
```

```
  (setf temp
          (/ one
             (abs1
              (f2cl-lib:fref work-%data% (j) ((1 *))
                             work-%offset%))))
  (cond
   ((>=
     (+ (* acoefa (f2cl-lib:fref rwork (j) ((1 *))))
        (* bcoefa
           (f2cl-lib:fref rwork
                          ((f2cl-lib:int-add n j))
                          ((1 *)))))
     (* bignum temp))
    (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                  ((> jr je) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (jr) ((1 *))
                             work-%offset%)
                (* temp
                   (f2cl-lib:fref work-%data% (jr)
                                  ((1 *))
                                  work-%offset%)))
       label190))))))
(setf ca
        (* acoeff
           (f2cl-lib:fref work-%data% (j) ((1 *))
                          work-%offset%)))
(setf cb
        (* bcoeff
           (f2cl-lib:fref work-%data% (j) ((1 *))
                          work-%offset%)))
(f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
              ((> jr
                  (f2cl-lib:int-add j
                                    (f2cl-lib:int-sub
                                     1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (jr) ((1 *))
                         work-%offset%)
            (-
             (+
              (f2cl-lib:fref work-%data% (jr) ((1 *))
                             work-%offset%)
              (* ca
                 (f2cl-lib:fref s-%data% (jr j)
                                ((1 lds) (1 *))
                                s-%offset%)))
             (* cb
                (f2cl-lib:fref p-%data% (jr j)
```

```
                                        ((1 ldp) (1 *))
                                        p-%offset%))))
          label200))))
   label210))
(cond
 (ilback
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (zgemv "N" n je cone vr ldvr work 1 czero
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             ((+ n 1)) ((1 *)) work-%offset%)
       1)
    (declare (ignore var-0 var-4 var-6 var-7 var-9 var-10))
    (when var-1 (setf n var-1))
    (when var-2 (setf je var-2))
    (when var-3 (setf cone var-3))
    (when var-5 (setf ldvr var-5))
    (when var-8 (setf czero var-8)))
  (setf isrc 2) (setf iend n))
 (t (setf isrc 1) (setf iend je)))
(setf xmax zero)
(f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
              ((> jr iend) nil)
  (tagbody
    (setf xmax
            (max xmax
                 (abs1
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-sub isrc 1)
                                    n)
                                   jr))
                                 ((1 *)) work-%offset%))))
   label220))
(cond
 ((> xmax safmin) (setf temp (/ one xmax))
  (f2cl-lib:fdo (jr 1 (f2cl-lib:int-add jr 1))
                ((> jr iend) nil)
    (tagbody
      (setf (f2cl-lib:fref vr-%data% (jr ieig)
                           ((1 ldvr) (1 *)) vr-%offset%)
              (* temp
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add
                                  (f2cl-lib:int-mul
                                   (f2cl-lib:int-sub isrc 1)
                                   n)
                                  jr))
```

```
                                                ((1 *)) work-%offset%)))
                      label230)))
                   (t (setf iend 0)))
                  (f2cl-lib:fdo (jr (f2cl-lib:int-add iend 1)
                                 (f2cl-lib:int-add jr 1))
                                ((> jr n) nil)
                    (tagbody
                      (setf (f2cl-lib:fref vr-%data% (jr ieig) ((1 ldvr) (1 *))
                                           vr-%offset%)
                            czero)
                     label240))))
                label250))))
           (go end_label)
          end_label
           (return
            (values side howmny nil n nil nil nil nil nil ldvl nil ldvr nil m
                    nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ztgevc
                 fortran-to-lisp::*f2cl-function-info*)
           (fortran-to-lisp::make-f2cl-finfo :arg-types
                                             '((string) (string)
                                               (array fortran-to-lisp::logical
                                                (*))
                                               (fortran-to-lisp::integer4)
                                               (array fortran-to-lisp::complex16
                                                (*))
                                               (fortran-to-lisp::integer4)
                                               (array fortran-to-lisp::complex16
                                                (*))
                                               (fortran-to-lisp::integer4)
                                               (array fortran-to-lisp::complex16
                                                (*))
                                               (fortran-to-lisp::integer4)
                                               (array fortran-to-lisp::complex16
                                                (*))
                                               (fortran-to-lisp::integer4)
                                               (fortran-to-lisp::integer4)
                                               (fortran-to-lisp::integer4)
                                               (array fortran-to-lisp::complex16
                                                (*))
                                               (array double-float (*))
                                               (fortran-to-lisp::integer4))
                                             :return-values
                                             '(fortran-to-lisp::side
                                               fortran-to-lisp::howmny nil
                                               fortran-to-lisp::n nil nil nil
```

```
                                                  nil nil fortran-to-lisp::ldvl nil
                                                  fortran-to-lisp::ldvr nil
                                                  fortran-to-lisp::m nil nil
                                                  fortran-to-lisp::info)
                                                :calls
                                                '(fortran-to-lisp::zladiv
                                                  fortran-to-lisp::xerbla))))
```

———-

35.40 ztgexc

35.40.1 Axiom unit tests

— **ztgexc.input** —

```
)set break resume
)sys rm -f ztgexc.output
)spool ztgexc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

35.40.2 Axiom help page

— **ztgexc.help** —

```
======================================================================
ztgexc examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
```

```
      subroutine ztgexc (WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z, LDZ,
          IFST, ILST, INFO)
          ZTGEXC

Function/Subroutine Documentation
   subroutine ztgexc (logical WANTQ, logical WANTZ, integer N, complex*16,
       dimension( lda, * ) A, integer LDA, complex*16, dimension( ldb, * ) B,
       integer LDB, complex*16, dimension( ldq, * ) Q, integer LDQ,
       complex*16, dimension( ldz, * ) Z, integer LDZ, integer IFST, integer
       ILST, integer INFO)
       ZTGEXC

Purpose:
   ZTGEXC reorders the generalized Schur decomposition of a complex
   matrix pair (A,B), using an unitary equivalence transformation
   (A, B) := Q * (A, B) * Z**H, so that the diagonal block of (A, B) with
   row index IFST is moved to row ILST.

   (A, B) must be in generalized Schur canonical form, that is, A and
   B are both upper triangular.

   Optionally, the matrices Q and Z of generalized Schur vectors are
   updated.

   Q(in) * A(in) * Z(in)**H = Q(out) * A(out) * Z(out)**H
   Q(in) * B(in) * Z(in)**H = Q(out) * B(out) * Z(out)**H

Parameters:

 WANTQ
   WANTQ is LOGICAL
   .TRUE. : update the left transformation matrix Q;
   .FALSE.: do not update Q.

 WANTZ
   WANTZ is LOGICAL
   .TRUE. : update the right transformation matrix Z;
   .FALSE.: do not update Z.

 N
   N is INTEGER
   The order of the matrices A and B. N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the upper triangular matrix A in the pair (A, B).
   On exit, the updated matrix A.

 LDA
   LDA is INTEGER
```

```
  The leading dimension of the array A. LDA >= max(1,N).

B
  B is COMPLEX*16 array, dimension (LDB,N)
  On entry, the upper triangular matrix B in the pair (A, B).
  On exit, the updated matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= max(1,N).

Q
  Q is COMPLEX*16 array, dimension (LDZ,N)
  On entry, if WANTQ = .TRUE., the unitary matrix Q.
  On exit, the updated matrix Q.
  If WANTQ = .FALSE., Q is not referenced.

LDQ
  LDQ is INTEGER
  The leading dimension of the array Q. LDQ >= 1;
  If WANTQ = .TRUE., LDQ >= N.

Z
  Z is COMPLEX*16 array, dimension (LDZ,N)
  On entry, if WANTZ = .TRUE., the unitary matrix Z.
  On exit, the updated matrix Z.
  If WANTZ = .FALSE., Z is not referenced.

LDZ
  LDZ is INTEGER
  The leading dimension of the array Z. LDZ >= 1;
  If WANTZ = .TRUE., LDZ >= N.

IFST
  IFST is INTEGER

ILST
  ILST is INTEGER
  Specify the reordering of the diagonal blocks of (A, B).
  The block with row index IFST is moved to row ILST, by a
  sequence of swapping between adjacent blocks.

INFO
  INFO is INTEGER
  =0:  Successful exit.
  <0:  if INFO = -i, the i-th argument had an illegal value.
  =1:  The transformed matrix pair (A, B) would be too far
       from generalized Schur form; the problem is ill-
       conditioned. (A, B) may have been partially reordered,
       and ILST points to the first row of the current
```

position of the block being moved.

———-

35.40.3 fortran code

— **ztgexc.f** —

```
*  =====================================================================
      SUBROUTINE ZTGEXC( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                   LDZ, IFST, ILST, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      LOGICAL            WANTQ, WANTZ
      INTEGER            IFST, ILST, INFO, LDA, LDB, LDQ, LDZ, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), B( LDB, * ), Q( LDQ, * ),
     $                   Z( LDZ, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            HERE
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZTGEX2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Decode and test input arguments.
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
```

```
      ELSE IF( LDQ.LT.1 .OR. WANTQ .AND. ( LDQ.LT.MAX( 1, N ) ) ) THEN
         INFO = -9
      ELSE IF( LDZ.LT.1 .OR. WANTZ .AND. ( LDZ.LT.MAX( 1, N ) ) ) THEN
         INFO = -11
      ELSE IF( IFST.LT.1 .OR. IFST.GT.N ) THEN
         INFO = -12
      ELSE IF( ILST.LT.1 .OR. ILST.GT.N ) THEN
         INFO = -13
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZTGEXC', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.LE.1 )
     $   RETURN
      IF( IFST.EQ.ILST )
     $   RETURN
*
      IF( IFST.LT.ILST ) THEN
*
         HERE = IFST
*
   10    CONTINUE
*
*        Swap with next one below
*
         CALL ZTGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z, LDZ,
     $                HERE, INFO )
         IF( INFO.NE.0 ) THEN
            ILST = HERE
            RETURN
         END IF
         HERE = HERE + 1
         IF( HERE.LT.ILST )
     $      GO TO 10
         HERE = HERE - 1
      ELSE
         HERE = IFST - 1
*
   20    CONTINUE
*
*        Swap with next one above
*
         CALL ZTGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z, LDZ,
     $                HERE, INFO )
         IF( INFO.NE.0 ) THEN
            ILST = HERE
```

```
            RETURN
         END IF
         HERE = HERE - 1
         IF( HERE.GE.ILST )
     $      GO TO 20
         HERE = HERE + 1
      END IF
      ILST = HERE
      RETURN
*
*     End of ZTGEXC
*
      END
```

———-

35.40.4 lisp code

— **ztgexc.lisp** —

```
(defun ztgexc (wantq wantz n a lda b ldb$ q ldq z ldz ifst ilst info)
  (declare (type (array f2cl-lib:complex16 (*)) z q b a)
           (type (f2cl-lib:integer4) info ilst ifst ldz ldq ldb$ lda n)
           (type f2cl-lib:logical wantz wantq))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex16 a-%data% a-%offset%)
       (b f2cl-lib:complex16 b-%data% b-%offset%)
       (q f2cl-lib:complex16 q-%data% q-%offset%)
       (z f2cl-lib:complex16 z-%data% z-%offset%))
    (prog ((here 0))
      (declare (type (f2cl-lib:integer4) here))
      (setf info 0)
      (cond ((< n 0) (setf info -3))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -5))
            ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -7))
            ((or (< ldq 1)
                 (and wantq
                      (< ldq
                         (max (the f2cl-lib:integer4 1)
                              (the f2cl-lib:integer4 n)))))
             (setf info -9))
            ((or (< ldz 1)
                 (and wantz
                      (< ldz
                         (max (the f2cl-lib:integer4 1)
                              (the f2cl-lib:integer4 n)))))
```

```
        (setf info -11))
       ((or (< ifst 1) (> ifst n)) (setf info -12))
       ((or (< ilst 1) (> ilst n)) (setf info -13)))
 (cond
  ((/= info 0) (xerbla "ZTGEXC" (f2cl-lib:int-sub info)) (go end_label)))
 (if (<= n 1)
     (go end_label))
 (if (= ifst ilst)
     (go end_label))
 (cond
  ((< ifst ilst)
   (tagbody
     (setf here ifst)
    label10
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12)
         (ztgex2 wantq wantz n a lda b ldb$ q ldq z ldz here info)
       (declare
        (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11))
       (setf info var-12))
     (cond ((/= info 0) (setf ilst here) (go end_label)))
     (setf here (f2cl-lib:int-add here 1))
     (if (< here ilst)
         (go label10))
     (setf here (f2cl-lib:int-sub here 1))))
  (t
   (tagbody
     (setf here (f2cl-lib:int-sub ifst 1))
    label20
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12)
         (ztgex2 wantq wantz n a lda b ldb$ q ldq z ldz here info)
       (declare
        (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11))
       (setf info var-12))
     (cond ((/= info 0) (setf ilst here) (go end_label)))
     (setf here (f2cl-lib:int-sub here 1))
     (if (>= here ilst)
         (go label20))
     (setf here (f2cl-lib:int-add here 1)))))
 (setf ilst here)
 (go end_label)
end_label
 (return
  (values nil nil nil nil nil nil nil nil nil nil nil nil ilst info)))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ztgexc
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '(fortran-to-lisp::logical
                                              fortran-to-lisp::logical
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
                                              nil nil nil nil
                                              fortran-to-lisp::ilst
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::ztgex2
                                              fortran-to-lisp::xerbla))))
```

———-

Chapter 36

LAPACK - General Matrices, Auxilliary Routines, Double

36.1 dgesc2

36.1.1 Axiom unit tests

— **dgesc2.input** —

```
)set break resume
)sys rm -f dgesc2.output
)spool dgesc2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

36.1.2 Axiom help page

— **dgesc2.help** —

```
==========================================================================
dgesc2 examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgesc2 (N, A, LDA, RHS, IPIV, JPIV, SCALE)
           DGESC2 solves a system of linear equations using the LU
           factorization with complete pivoting computed by sgetc2.

Function/Subroutine Documentation
   subroutine dgesc2 (integer N, double precision, dimension( lda, * ) A,
       integer LDA, double precision, dimension( * ) RHS, integer, dimension(
       * ) IPIV, integer, dimension( * ) JPIV, double precision SCALE)
       DGESC2 solves a system of linear equations using the LU factorization
       with complete pivoting computed by sgetc2.

Purpose:
   DGESC2 solves a system of linear equations

   A * X = scale* RHS

   with a general N-by-N matrix A using the LU factorization with
   complete pivoting computed by DGETC2.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the  LU part of the factorization of the n-by-n
   matrix A computed by DGETC2:  A = P * L * U * Q

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1, N).

 RHS
   RHS is DOUBLE PRECISION array, dimension (N).
   On entry, the right hand side vector b.
   On exit, the solution vector X.

 IPIV
   IPIV is INTEGER array, dimension (N).
   The pivot indices; for 1 <= i <= N, row i of the
   matrix has been interchanged with row IPIV(i).
```

```
 JPIV
   JPIV is INTEGER array, dimension (N).
   The pivot indices; for 1 <= j <= N, column j of the
   matrix has been interchanged with column JPIV(j).

 SCALE
   SCALE is DOUBLE PRECISION
   On exit, SCALE contains the scale factor. SCALE is chosen
   0 <= SCALE <= 1 to prevent owerflow in the solution.
```

36.1.3 fortran code

— dgesc2.f —

```
*  =====================================================================
      SUBROUTINE DGESC2( N, A, LDA, RHS, IPIV, JPIV, SCALE )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            LDA, N
      DOUBLE PRECISION   SCALE
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), JPIV( * )
      DOUBLE PRECISION   A( LDA, * ), RHS( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, TWO
      PARAMETER          ( ONE = 1.0D+0, TWO = 2.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   BIGNUM, EPS, SMLNUM, TEMP
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASWP, DSCAL
*     ..
```

```
*     .. External Functions ..
      INTEGER            IDAMAX
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           IDAMAX, DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS
*     ..
*     .. Executable Statements ..
*
*      Set constant to control owerflow
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' ) / EPS
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
*
*     Apply permutations IPIV to RHS
*
      CALL DLASWP( 1, RHS, LDA, 1, N-1, IPIV, 1 )
*
*     Solve for L part
*
      DO 20 I = 1, N - 1
         DO 10 J = I + 1, N
            RHS( J ) = RHS( J ) - A( J, I )*RHS( I )
   10    CONTINUE
   20 CONTINUE
*
*     Solve for U part
*
      SCALE = ONE
*
*     Check for scaling
*
      I = IDAMAX( N, RHS, 1 )
      IF( TWO*SMLNUM*ABS( RHS( I ) ).GT.ABS( A( N, N ) ) ) THEN
         TEMP = ( ONE / TWO ) / ABS( RHS( I ) )
         CALL DSCAL( N, TEMP, RHS( 1 ), 1 )
         SCALE = SCALE*TEMP
      END IF
*
      DO 40 I = N, 1, -1
         TEMP = ONE / A( I, I )
         RHS( I ) = RHS( I )*TEMP
         DO 30 J = I + 1, N
            RHS( I ) = RHS( I ) - RHS( J )*( A( I, J )*TEMP )
   30    CONTINUE
   40 CONTINUE
*
```

```
*     Apply permutations JPIV to the solution (RHS)
*
      CALL DLASWP( 1, RHS, LDA, 1, N-1, JPIV, -1 )
      RETURN
*
*     End of DGESC2
*
      END
```

36.1.4 lisp code

— **dgesc2.lisp** —

```
(let* ((one 1.0d0) (two 2.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (ignorable one two))
  (defun dgesc2 (n a lda rhs ipiv jpiv scale)
    (declare (type (double-float) scale)
             (type (array f2cl-lib:integer4 (*)) jpiv ipiv)
             (type (array double-float (*)) rhs a)
             (type (f2cl-lib:integer4) lda n))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (rhs double-float rhs-%data% rhs-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (jpiv f2cl-lib:integer4 jpiv-%data% jpiv-%offset%))
      (prog ((bignum 0.0d0) (eps 0.0d0) (smlnum 0.0d0) (temp 0.0d0) (i 0)
             (j 0))
        (declare (type (double-float) bignum eps smlnum temp)
                 (type (f2cl-lib:integer4) i j))
        (setf eps (dlamch "P"))
        (setf smlnum (/ (dlamch "S") eps))
        (setf bignum (/ one smlnum))
        (multiple-value-bind (var-0 var-1)
            (dlabad smlnum bignum)
          (declare (ignore))
          (when var-0 (setf smlnum var-0))
          (when var-1 (setf bignum var-1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dlaswp 1 rhs lda 1 (f2cl-lib:int-sub n 1) ipiv 1)
          (declare (ignore var-0 var-1 var-3 var-4 var-5 var-6))
          (when var-2 (setf lda var-2)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
          (tagbody
```

```
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref rhs-%data% (j) ((1 *)) rhs-%offset%)
                (- (f2cl-lib:fref rhs-%data% (j) ((1 *)) rhs-%offset%)
                   (*
                    (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                   a-%offset%)
                    (f2cl-lib:fref rhs-%data% (i) ((1 *))
                                   rhs-%offset%))))
       label10))
   label20))
(setf scale one)
(setf i
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (idamax n rhs 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(cond
 ((> (* two smlnum (abs (f2cl-lib:fref rhs (i) ((1 *)))))
     (abs (f2cl-lib:fref a (n n) ((1 lda) (1 *)))))
  (setf temp
          (/ (/ one two)
             (abs
              (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%))))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dscal n temp
       (f2cl-lib:array-slice rhs-%data% double-float (1) ((1 *))
                             rhs-%offset%)
       1)
    (declare (ignore var-2 var-3))
    (when var-0 (setf n var-0))
    (when var-1 (setf temp var-1)))
  (setf scale (* scale temp))))
(f2cl-lib:fdo (i n (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf temp
            (/ one
               (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                              a-%offset%)))
    (setf (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%)
            (* (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%)
               temp))
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%)
                (- (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%)
```

```
                          (*
                           (f2cl-lib:fref rhs-%data% (j) ((1 *)) rhs-%offset%)
                           (*
                            (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                           a-%offset%)
                            temp))))
               label30))
           label40))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dlaswp 1 rhs lda 1 (f2cl-lib:int-sub n 1) jpiv -1)
          (declare (ignore var-0 var-1 var-3 var-4 var-5 var-6))
          (when var-2 (setf lda var-2)))
        (go end_label)
       end_label
        (return (values n nil lda nil nil nil scale))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgesc2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (double-float))
                                            :return-values
                                            '(fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::scale)
                                            :calls 'nil)))
```

———-

36.2 dgetc2

36.2.1 Axiom unit tests

— dgetc2.input —

```
)set break resume
)sys rm -f dgetc2.output
)spool dgetc2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

36.2.2 Axiom help page

— dgetc2.help —

```
==================================================================
dgetc2 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgetc2 (N, A, LDA, IPIV, JPIV, INFO)
           DGETC2 computes the LU factorization with complete pivoting of the
           general n-by-n matrix.

Function/Subroutine Documentation
   subroutine dgetc2 (integer N, double precision, dimension( lda, * ) A,
       integer LDA, integer, dimension( * ) IPIV, integer, dimension( * )
       JPIV, integer INFO)
       DGETC2 computes the LU factorization with complete pivoting of the
       general n-by-n matrix.

Purpose:
   DGETC2 computes an LU factorization with complete pivoting of the
   n-by-n matrix A. The factorization has the form A = P * L * U * Q,
   where P and Q are permutation matrices, L is lower triangular with
   unit diagonal elements and U is upper triangular.

   This is the Level 2 BLAS algorithm.

Parameters:

 N
```

```
  N is INTEGER
  The order of the matrix A. N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA, N)
  On entry, the n-by-n matrix A to be factored.
  On exit, the factors L and U from the factorization
  A = P*L*U*Q; the unit diagonal elements of L are not stored.
  If U(k, k) appears to be less than SMIN, U(k, k) is given the
  value of SMIN, i.e., giving a nonsingular perturbed system.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension(N).
  The pivot indices; for 1 <= i <= N, row i of the
  matrix has been interchanged with row IPIV(i).

JPIV
  JPIV is INTEGER array, dimension(N).
  The pivot indices; for 1 <= j <= N, column j of the
  matrix has been interchanged with column JPIV(j).

INFO
  INFO is INTEGER
  = 0: successful exit
  > 0: if INFO = k, U(k, k) is likely to produce owerflow if
       we try to solve for x in Ax = b. So U is perturbed to
       avoid the overflow.
```

———-

36.2.3 fortran code

— dgetc2.f —

```
*  ===================================================================
      SUBROUTINE DGETC2( N, A, LDA, IPIV, JPIV, INFO )
*
*  -- LAPACK auxiliary routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
```

```
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), JPIV( * )
      DOUBLE PRECISION   A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IP, IPV, J, JP, JPV
      DOUBLE PRECISION   BIGNUM, EPS, SMIN, SMLNUM, XMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGER, DSWAP
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Set constants to control overflow
*
      INFO = 0
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' ) / EPS
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
*
*     Factorize A using complete pivoting.
*     Set pivots less than SMIN to SMIN.
*
      DO 40 I = 1, N - 1
*
*        Find max element in matrix A
*
         XMAX = ZERO
         DO 20 IP = I, N
            DO 10 JP = I, N
               IF( ABS( A( IP, JP ) ).GE.XMAX ) THEN
                  XMAX = ABS( A( IP, JP ) )
                  IPV = IP
```

```
                JPV = JP
             END IF
   10       CONTINUE
   20    CONTINUE
         IF( I.EQ.1 )
     $      SMIN = MAX( EPS*XMAX, SMLNUM )
*
*        Swap rows
*
         IF( IPV.NE.I )
     $      CALL DSWAP( N, A( IPV, 1 ), LDA, A( I, 1 ), LDA )
         IPIV( I ) = IPV
*
*        Swap columns
*
         IF( JPV.NE.I )
     $      CALL DSWAP( N, A( 1, JPV ), 1, A( 1, I ), 1 )
         JPIV( I ) = JPV
*
*        Check for singularity
*
         IF( ABS( A( I, I ) ).LT.SMIN ) THEN
            INFO = I
            A( I, I ) = SMIN
         END IF
         DO 30 J = I + 1, N
            A( J, I ) = A( J, I ) / A( I, I )
   30    CONTINUE
         CALL DGER( N-I, N-I, -ONE, A( I+1, I ), 1, A( I, I+1 ), LDA,
     $              A( I+1, I+1 ), LDA )
   40 CONTINUE
*
      IF( ABS( A( N, N ) ).LT.SMIN ) THEN
         INFO = N
         A( N, N ) = SMIN
      END IF
*
*     Set last pivots to N
*
      IPIV( N ) = N
      JPIV( N ) = N
*
      RETURN
*
*     End of DGETC2
*
      END
```

———-

36.2.4 lisp code

— dgetc2.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dgetc2 (n a lda ipiv jpiv info)
    (declare (type (array f2cl-lib:integer4 (*)) jpiv ipiv)
             (type (array double-float (*)) a)
             (type (f2cl-lib:integer4) info lda n))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (jpiv f2cl-lib:integer4 jpiv-%data% jpiv-%offset%))
      (prog ((bignum 0.0d0) (eps 0.0d0) (smin 0.0d0) (smlnum 0.0d0)
             (xmax 0.0d0) (i 0) (ip 0) (ipv 0) (j 0) (jp 0) (jpv 0))
        (declare (type (double-float) bignum eps smin smlnum xmax)
                 (type (f2cl-lib:integer4) i ip ipv j jp jpv))
        (setf info 0)
        (setf eps (dlamch "P"))
        (setf smlnum (/ (dlamch "S") eps))
        (setf bignum (/ one smlnum))
        (multiple-value-bind (var-0 var-1)
            (dlabad smlnum bignum)
          (declare (ignore))
          (when var-0 (setf smlnum var-0))
          (when var-1 (setf bignum var-1)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
          (tagbody
            (setf xmax zero)
            (f2cl-lib:fdo (ip i (f2cl-lib:int-add ip 1))
                          ((> ip n) nil)
              (tagbody
                (f2cl-lib:fdo (jp i (f2cl-lib:int-add jp 1))
                              ((> jp n) nil)
                  (tagbody
                    (cond
                     ((>= (abs (f2cl-lib:fref a (ip jp) ((1 lda) (1 *)))) xmax)
                      (setf xmax
                              (abs
                               (f2cl-lib:fref a-%data% (ip jp) ((1 lda) (1 *))
                                              a-%offset%)))
                      (setf ipv ip) (setf jpv jp)))
                   label10))
               label20))
            (if (= i 1)
```

```
    (setf smin (max (* eps xmax) smlnum)))
(if (/= ipv i)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dswap n
         (f2cl-lib:array-slice a-%data% double-float (ipv 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice a-%data% double-float (i 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-1 var-3))
      (when var-0 (setf n var-0))
      (when var-2 (setf lda var-2))
      (when var-4 (setf lda var-4))))
(setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) ipv)
(if (/= jpv i)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dswap n
         (f2cl-lib:array-slice a-%data% double-float (1 jpv)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% double-float (1 i)
                               ((1 lda) (1 *)) a-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0))))
(setf (f2cl-lib:fref jpiv-%data% (i) ((1 *)) jpiv-%offset%) jpv)
(cond
 ((< (abs (f2cl-lib:fref a (i i) ((1 lda) (1 *)))) smin)
  (setf info i)
  (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
          smin)))
(f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *)) a-%offset%)
            (/
             (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                            a-%offset%)
             (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                            a-%offset%)))
   label30))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dger (f2cl-lib:int-sub n i) (f2cl-lib:int-sub n i) (- one)
     (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                           ((1 lda) (1 *)) a-%offset%)
     1
     (f2cl-lib:array-slice a-%data% double-float
                           (i (f2cl-lib:int-add i 1))
```

```
                                        ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice a-%data% double-float
                                        ((+ i 1) (f2cl-lib:int-add i 1))
                                        ((1 lda) (1 *)) a-%offset%)
                 lda)
               (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7))
               (when var-6 (setf lda var-6))
               (when var-8 (setf lda var-8)))
            label40))
         (cond
          ((< (abs (f2cl-lib:fref a (n n) ((1 lda) (1 *)))) smin) (setf info n)
           (setf (f2cl-lib:fref a-%data% (n n) ((1 lda) (1 *)) a-%offset%)
                  smin)))
         (setf (f2cl-lib:fref ipiv-%data% (n) ((1 *)) ipiv-%offset%) n)
         (setf (f2cl-lib:fref jpiv-%data% (n) ((1 *)) jpiv-%offset%) n)
         (go end_label)
        end_label
         (return (values n nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgetc2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

36.3 dlange

36.3.1 Axiom unit tests

— dlange.input —

```
)set break resume
)sys rm -f dlange.output
)spool dlange.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

36.3.2 Axiom help page

— dlange.help —

```
======================================================================
dlange examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function dlange (NORM, M, N, A, LDA, WORK)
           DLANGE returns the value of the 1-norm, Frobenius norm, infinity-
           norm, or the largest absolute value of any element of a general
           rectangular matrix.

Function/Subroutine Documentation
   double precision function dlange (character NORM, integer M, integer N,
       double precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( * ) WORK)
       DLANGE returns the value of the 1-norm, Frobenius norm, infinity-norm,
       or the largest absolute value of any element of a general rectangular
       matrix.

Purpose:
```

```
  DLANGE  returns the value of the one norm,  or the Frobenius norm, or
  the  infinity norm,  or the  element of  largest absolute value  of a
  real matrix A.

  Returns:

          DLANGE

              DLANGE = ( max(abs(A(i,j))), NORM = 'M' or 'm'
                       (
                       ( norm1(A),         NORM = '1', 'O' or 'o'
                       (
                       ( normI(A),         NORM = 'I' or 'i'
                       (
                       ( normF(A),         NORM = 'F', 'f', 'E' or 'e'

  where  norm1  denotes the  one norm of a matrix (maximum column sum),
  normI  denotes the  infinity norm  of a matrix  (maximum row sum) and
  normF  denotes the  Frobenius norm of a matrix (square root of sum of
  squares).  Note that  max(abs(A(i,j)))  is not a consistent matrix norm.

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies the value to be returned in DLANGE as described above.

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.  When M = 0,
   DLANGE is set to zero.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.  When N = 0,
   DLANGE is set to zero.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   The m by n matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(M,1).

 WORK
   WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK)),
   where LWORK >= M when NORM = 'I'; otherwise, WORK is not referenced.
```

———-

36.3.3 fortran code

— dlange.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION DLANGE( NORM, M, N, A, LDA, WORK )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), WORK( * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   SCALE, SUM, VALUE, TEMP
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASSQ
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, DISNAN
      EXTERNAL           LSAME, DISNAN
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      IF( MIN( M, N ).EQ.0 ) THEN
         VALUE = ZERO
      ELSE IF( LSAME( NORM, 'M' ) ) THEN
*
```

```
*        Find max(abs(A(i,j))).
*
         VALUE = ZERO
         DO 20 J = 1, N
            DO 10 I = 1, M
               TEMP = ABS( A( I, J ) )
               IF( VALUE.LT.TEMP .OR. DISNAN( TEMP ) ) VALUE = TEMP
   10       CONTINUE
   20    CONTINUE
      ELSE IF( ( LSAME( NORM, 'O' ) ) .OR. ( NORM.EQ.'1' ) ) THEN
*
*        Find norm1(A).
*
         VALUE = ZERO
         DO 40 J = 1, N
            SUM = ZERO
            DO 30 I = 1, M
               SUM = SUM + ABS( A( I, J ) )
   30       CONTINUE
            IF( VALUE.LT.SUM .OR. DISNAN( SUM ) ) VALUE = SUM
   40    CONTINUE
      ELSE IF( LSAME( NORM, 'I' ) ) THEN
*
*        Find normI(A).
*
         DO 50 I = 1, M
            WORK( I ) = ZERO
   50    CONTINUE
         DO 70 J = 1, N
            DO 60 I = 1, M
               WORK( I ) = WORK( I ) + ABS( A( I, J ) )
   60       CONTINUE
   70    CONTINUE
         VALUE = ZERO
         DO 80 I = 1, M
            TEMP = WORK( I )
            IF( VALUE.LT.TEMP .OR. DISNAN( TEMP ) ) VALUE = TEMP
   80    CONTINUE
      ELSE IF( ( LSAME( NORM, 'F' ) ) .OR. ( LSAME( NORM, 'E' ) ) ) THEN
*
*        Find normF(A).
*
         SCALE = ZERO
         SUM = ONE
         DO 90 J = 1, N
            CALL DLASSQ( M, A( 1, J ), 1, SCALE, SUM )
   90    CONTINUE
         VALUE = SCALE*SQRT( SUM )
      END IF
*
```

```
      DLANGE = VALUE
      RETURN
*
*     End of DLANGE
*
      END
```

———-

36.3.4 lisp code

— **dlange.lisp** —

———-

36.4 dlaqge

36.4.1 Axiom unit tests

— **dlaqge.input** —

```
)set break resume
)sys rm -f dlaqge.output
)spool dlaqge.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

36.4.2 Axiom help page

— **dlaqge.help** —

```
====================================================================
dlaqge examples
====================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlaqge (M, N, A, LDA, R, C, ROWCND, COLCND, AMAX, EQUED)
           DLAQGE scales a general rectangular matrix, using row and column
           scaling factors computed by sgeequ.

Function/Subroutine Documentation
   subroutine dlaqge (integer M, integer N, double precision, dimension( lda,
       * ) A, integer LDA, double precision, dimension( * ) R, double
       precision, dimension( * ) C, double precision ROWCND, double precision
       COLCND, double precision AMAX, character EQUED)
       DLAQGE scales a general rectangular matrix, using row and column
       scaling factors computed by sgeequ.

Purpose:
   DLAQGE equilibrates a general M by N matrix A using the row and
   column scaling factors in the vectors R and C.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the M by N matrix A.
   On exit, the equilibrated matrix.  See EQUED for the form of
   the equilibrated matrix.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(M,1).

 R
   R is DOUBLE PRECISION array, dimension (M)
   The row scale factors for A.

 C
   C is DOUBLE PRECISION array, dimension (N)
   The column scale factors for A.
```

```
 ROWCND
   ROWCND is DOUBLE PRECISION
   Ratio of the smallest R(i) to the largest R(i).

 COLCND
   COLCND is DOUBLE PRECISION
   Ratio of the smallest C(i) to the largest C(i).

 AMAX
   AMAX is DOUBLE PRECISION
   Absolute value of largest matrix entry.

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done.
   = 'N':  No equilibration
   = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
   = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
   = 'B':  Both row and column equilibration, i.e., A has been
           replaced by diag(R) * A * diag(C).

Internal Parameters:
   THRESH is a threshold value used to decide if row or column scaling
   should be done based on the ratio of the row or column scaling
   factors.  If ROWCND < THRESH, row scaling is done, and if
   COLCND < THRESH, column scaling is done.

   LARGE and SMALL are threshold values used to decide if row scaling
   should be done based on the absolute size of the largest matrix
   element.  If AMAX > LARGE or AMAX < SMALL, row scaling is done.
```

———-

36.4.3 fortran code

— dlaqge.f —

```
*  ====================================================================
      SUBROUTINE DLAQGE( M, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $                   EQUED )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
```

```
*     .. Scalar Arguments ..
      CHARACTER          EQUED
      INTEGER            LDA, M, N
      DOUBLE PRECISION   AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), C( * ), R( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, THRESH
      PARAMETER          ( ONE = 1.0D+0, THRESH = 0.1D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   CJ, LARGE, SMALL
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( M.LE.0 .OR. N.LE.0 ) THEN
         EQUED = 'N'
         RETURN
      END IF
*
*     Initialize LARGE and SMALL.
*
      SMALL = DLAMCH( 'Safe minimum' ) / DLAMCH( 'Precision' )
      LARGE = ONE / SMALL
*
      IF( ROWCND.GE.THRESH .AND. AMAX.GE.SMALL .AND. AMAX.LE.LARGE )
     $     THEN
*
*        No row scaling
*
         IF( COLCND.GE.THRESH ) THEN
*
*           No column scaling
*
            EQUED = 'N'
         ELSE
*
*           Column scaling
```

```
*
            DO 20 J = 1, N
               CJ = C( J )
               DO 10 I = 1, M
                  A( I, J ) = CJ*A( I, J )
   10          CONTINUE
   20       CONTINUE
            EQUED = 'C'
         END IF
      ELSE IF( COLCND.GE.THRESH ) THEN
*
*        Row scaling, no column scaling
*
         DO 40 J = 1, N
            DO 30 I = 1, M
               A( I, J ) = R( I )*A( I, J )
   30       CONTINUE
   40    CONTINUE
         EQUED = 'R'
      ELSE
*
*        Row and column scaling
*
         DO 60 J = 1, N
            CJ = C( J )
            DO 50 I = 1, M
               A( I, J ) = CJ*R( I )*A( I, J )
   50       CONTINUE
   60    CONTINUE
         EQUED = 'B'
      END IF
*
      RETURN
*
*     End of DLAQGE
*
      END
```

———-

36.4.4 lisp code

— dlaqge.lisp —

```
(let* ((one 1.0d0) (thresh 0.1d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.1d0 0.1d0) thresh)
           (ignorable one thresh))
```

```
(defun dlaqge (m n a lda r c rowcnd colcnd amax equed)
  (declare (type (string *) equed)
           (type (double-float) amax colcnd rowcnd)
           (type (array double-float (*)) c r a)
           (type (f2cl-lib:integer4) lda n m))
  (f2cl-lib:with-multi-array-data
      ((a double-float a-%data% a-%offset%)
       (r double-float r-%data% r-%offset%)
       (c double-float c-%data% c-%offset%)
       (equed character equed-%data% equed-%offset%))
    (prog ((cj 0.0d0) (large 0.0d0) (small 0.0d0) (i 0) (j 0))
      (declare (type (double-float) cj large small)
               (type (f2cl-lib:integer4) i j))
      (cond
       ((or (<= m 0) (<= n 0))
        (f2cl-lib:f2cl-set-string equed "N" (string 1)) (go end_label)))
      (setf small (/ (dlamch "Safe minimum") (dlamch "Precision")))
      (setf large (/ one small))
      (cond
       ((and (>= rowcnd thresh) (>= amax small) (<= amax large))
        (cond
         ((>= colcnd thresh) (f2cl-lib:f2cl-set-string equed "N" (string 1)))
         (t
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf cj (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i m) nil)
                (tagbody
                  (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)
                        (* cj
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%)))
                label10))
            label20))
          (f2cl-lib:f2cl-set-string equed "C" (string 1)))))
       ((>= colcnd thresh)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                      (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)))
```

```
                label30))
            label40))
          (f2cl-lib:f2cl-set-string equed "R" (string 1)))
         (t
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf cj (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i m) nil)
                (tagbody
                  (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%)
                          (* cj (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                            a-%offset%)))
                label50))
            label60))
          (f2cl-lib:f2cl-set-string equed "B" (string 1))))
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil nil nil nil equed))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlaqge
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (double-float) (double-float)
                                              (double-float) (string))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
                                              nil fortran-to-lisp::equed)
                                            :calls 'nil)))
```

36.5 dtgex2

36.5.1 Axiom unit tests

— dtgex2.input —

```
)set break resume
)sys rm -f dtgex2.output
)spool dtgex2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

36.5.2 Axiom help page

— dtgex2.help —

```
======================================================================
dtgex2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dtgex2 (WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z, LDZ, J1,
           N1, N2, WORK, LWORK, INFO)
           DTGEX2 swaps adjacent diagonal blocks in an upper (quasi)
           triangular matrix pair by an orthogonal equivalence transformation.

Function/Subroutine Documentation
   subroutine dtgex2 (logical WANTQ, logical WANTZ, integer N, double
       precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldb, * ) B, integer LDB, double precision, dimension( ldq, *
       ) Q, integer LDQ, double precision, dimension( ldz, * ) Z, integer LDZ,
       integer J1, integer N1, integer N2, double precision, dimension( * )
       WORK, integer LWORK, integer INFO)
       DTGEX2 swaps adjacent diagonal blocks in an upper (quasi) triangular
       matrix pair by an orthogonal equivalence transformation.
```

```
Purpose:
   DTGEX2 swaps adjacent diagonal blocks (A11, B11) and (A22, B22)
   of size 1-by-1 or 2-by-2 in an upper (quasi) triangular matrix pair
   (A, B) by an orthogonal equivalence transformation.

   (A, B) must be in generalized real Schur canonical form (as returned
   by DGGES), i.e. A is block upper triangular with 1-by-1 and 2-by-2
   diagonal blocks. B is upper triangular.

   Optionally, the matrices Q and Z of generalized Schur vectors are
   updated.

   Q(in) * A(in) * Z(in)**T = Q(out) * A(out) * Z(out)**T
   Q(in) * B(in) * Z(in)**T = Q(out) * B(out) * Z(out)**T

Parameters:

 WANTQ
   WANTQ is LOGICAL
   .TRUE. : update the left transformation matrix Q;
   .FALSE.: do not update Q.

 WANTZ
   WANTZ is LOGICAL
   .TRUE. : update the right transformation matrix Z;
   .FALSE.: do not update Z.

 N
   N is INTEGER
   The order of the matrices A and B. N >= 0.

 A
   A is DOUBLE PRECISION array, dimensions (LDA,N)
   On entry, the matrix A in the pair (A, B).
   On exit, the updated matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A. LDA >= max(1,N).

 B
   B is DOUBLE PRECISION array, dimensions (LDB,N)
   On entry, the matrix B in the pair (A, B).
   On exit, the updated matrix B.

 LDB
   LDB is INTEGER
   The leading dimension of the array B. LDB >= max(1,N).
```

```
Q
  Q is DOUBLE PRECISION array, dimension (LDQ,N)
  On entry, if WANTQ = .TRUE., the orthogonal matrix Q.
  On exit, the updated matrix Q.
  Not referenced if WANTQ = .FALSE..

LDQ
  LDQ is INTEGER
  The leading dimension of the array Q. LDQ >= 1.
  If WANTQ = .TRUE., LDQ >= N.

Z
  Z is DOUBLE PRECISION array, dimension (LDZ,N)
  On entry, if WANTZ =.TRUE., the orthogonal matrix Z.
  On exit, the updated matrix Z.
  Not referenced if WANTZ = .FALSE..

LDZ
  LDZ is INTEGER
  The leading dimension of the array Z. LDZ >= 1.
  If WANTZ = .TRUE., LDZ >= N.

J1
  J1 is INTEGER
  The index to the first block (A11, B11). 1 <= J1 <= N.

N1
  N1 is INTEGER
  The order of the first block (A11, B11). N1 = 0, 1 or 2.

N2
  N2 is INTEGER
  The order of the second block (A22, B22). N2 = 0, 1 or 2.

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK)).

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  LWORK >=  MAX( 1, N*(N2+N1), (N2+N1)*(N2+N1)*2 )

INFO
  INFO is INTEGER
  =0: Successful exit
  >0: If INFO = 1, the transformed matrix (A, B) would be
      too far from generalized Schur form; the blocks are
      not swapped and (A, B) and (Q, Z) are unchanged.
      The problem of swapping is too ill-conditioned.
  <0: If INFO = -16: LWORK is too small. Appropriate value
```

```
      for LWORK is returned in WORK(1).

Further Details:

   In the current code both weak and strong stability tests are
   performed. The user can omit the strong stability test by changing
   the internal logical parameter WANDS to .FALSE..
```

36.5.3 fortran code

— dtgex2.f —

```
*  =====================================================================
      SUBROUTINE DTGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                   LDZ, J1, N1, N2, WORK, LWORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      LOGICAL            WANTQ, WANTZ
      INTEGER            INFO, J1, LDA, LDB, LDQ, LDZ, LWORK, N, N1, N2
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * ), Q( LDQ, * ),
     $                   WORK( * ), Z( LDZ, * )
*     ..
*
*  =====================================================================
*  Replaced various illegal calls to DCOPY by calls to DLASET, or by DO
*  loops. Sven Hammarling, 1/5/02.
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   TWENTY
      PARAMETER          ( TWENTY = 2.0D+01 )
      INTEGER            LDST
      PARAMETER          ( LDST = 4 )
      LOGICAL            WANDS
      PARAMETER          ( WANDS = .TRUE. )
*     ..
*     .. Local Scalars ..
```

```
      LOGICAL            DTRONG, WEAK
      INTEGER            I, IDUM, LINFO, M
      DOUBLE PRECISION   BQRA21, BRQA21, DDUM, DNORM, DSCALE, DSUM, EPS,
     $                   F, G, SA, SB, SCALE, SMLNUM, SS, THRESH, WS
*     ..
*     .. Local Arrays ..
      INTEGER            IWORK( LDST )
      DOUBLE PRECISION   AI( 2 ), AR( 2 ), BE( 2 ), IR( LDST, LDST ),
     $                   IRCOP( LDST, LDST ), LI( LDST, LDST ),
     $                   LICOP( LDST, LDST ), S( LDST, LDST ),
     $                   SCPY( LDST, LDST ), T( LDST, LDST ),
     $                   TAUL( LDST ), TAUR( LDST ), TCPY( LDST, LDST )
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEMM, DGEQR2, DGERQ2, DLACPY, DLAGV2, DLARTG,
     $                   DLASET, DLASSQ, DORG2R, DORGR2, DORM2R, DORMR2,
     $                   DROT, DSCAL, DTGSY2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
*     Quick return if possible
*
      IF( N.LE.1 .OR. N1.LE.0 .OR. N2.LE.0 )
     $   RETURN
      IF( N1.GT.N .OR. ( J1+N1 ).GT.N )
     $   RETURN
      M = N1 + N2
      IF( LWORK.LT.MAX( 1, N*M, M*M*2 ) ) THEN
         INFO = -16
         WORK( 1 ) = MAX( 1, N*M, M*M*2 )
         RETURN
      END IF
*
      WEAK = .FALSE.
      DTRONG = .FALSE.
*
*     Make a local copy of selected block
*
      CALL DLASET( 'Full', LDST, LDST, ZERO, ZERO, LI, LDST )
      CALL DLASET( 'Full', LDST, LDST, ZERO, ZERO, IR, LDST )
      CALL DLACPY( 'Full', M, M, A( J1, J1 ), LDA, S, LDST )
```

```
      CALL DLACPY( 'Full', M, M, B( J1, J1 ), LDB, T, LDST )
*
*     Compute threshold for testing acceptance of swapping.
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' ) / EPS
      DSCALE = ZERO
      DSUM = ONE
      CALL DLACPY( 'Full', M, M, S, LDST, WORK, M )
      CALL DLASSQ( M*M, WORK, 1, DSCALE, DSUM )
      CALL DLACPY( 'Full', M, M, T, LDST, WORK, M )
      CALL DLASSQ( M*M, WORK, 1, DSCALE, DSUM )
      DNORM = DSCALE*SQRT( DSUM )
*
*     THRES has been changed from
*        THRESH = MAX( TEN*EPS*SA, SMLNUM )
*     to
*        THRESH = MAX( TWENTY*EPS*SA, SMLNUM )
*     on 04/01/10.
*     "Bug" reported by Ondra Kamenik, confirmed by Julie Langou, fixed by
*     Jim Demmel and Guillaume Revy. See forum post 1783.
*
      THRESH = MAX( TWENTY*EPS*DNORM, SMLNUM )
*
      IF( M.EQ.2 ) THEN
*
*        CASE 1: Swap 1-by-1 and 1-by-1 blocks.
*
*        Compute orthogonal QL and RQ that swap 1-by-1 and 1-by-1 blocks
*        using Givens rotations and perform the swap tentatively.
*
         F = S( 2, 2 )*T( 1, 1 ) - T( 2, 2 )*S( 1, 1 )
         G = S( 2, 2 )*T( 1, 2 ) - T( 2, 2 )*S( 1, 2 )
         SB = ABS( T( 2, 2 ) )
         SA = ABS( S( 2, 2 ) )
         CALL DLARTG( F, G, IR( 1, 2 ), IR( 1, 1 ), DDUM )
         IR( 2, 1 ) = -IR( 1, 2 )
         IR( 2, 2 ) = IR( 1, 1 )
         CALL DROT( 2, S( 1, 1 ), 1, S( 1, 2 ), 1, IR( 1, 1 ),
     $              IR( 2, 1 ) )
         CALL DROT( 2, T( 1, 1 ), 1, T( 1, 2 ), 1, IR( 1, 1 ),
     $              IR( 2, 1 ) )
         IF( SA.GE.SB ) THEN
            CALL DLARTG( S( 1, 1 ), S( 2, 1 ), LI( 1, 1 ), LI( 2, 1 ),
     $                   DDUM )
         ELSE
            CALL DLARTG( T( 1, 1 ), T( 2, 1 ), LI( 1, 1 ), LI( 2, 1 ),
     $                   DDUM )
         END IF
         CALL DROT( 2, S( 1, 1 ), LDST, S( 2, 1 ), LDST, LI( 1, 1 ),
```

```
     $             LI( 2, 1 ) )
         CALL DROT( 2, T( 1, 1 ), LDST, T( 2, 1 ), LDST, LI( 1, 1 ),
     $             LI( 2, 1 ) )
         LI( 2, 2 ) = LI( 1, 1 )
         LI( 1, 2 ) = -LI( 2, 1 )
*
*        Weak stability test:
*           |S21| + |T21| <= O(EPS * F-norm((S, T)))
*
         WS = ABS( S( 2, 1 ) ) + ABS( T( 2, 1 ) )
         WEAK = WS.LE.THRESH
         IF( .NOT.WEAK )
     $      GO TO 70
*
         IF( WANDS ) THEN
*
*           Strong stability test:
*             F-norm((A-QL**T*S*QR, B-QL**T*T*QR)) <= O(EPS*F-norm((A,B)))
*
            CALL DLACPY( 'Full', M, M, A( J1, J1 ), LDA, WORK( M*M+1 ),
     $                   M )
            CALL DGEMM( 'N', 'N', M, M, M, ONE, LI, LDST, S, LDST, ZERO,
     $                  WORK, M )
            CALL DGEMM( 'N', 'T', M, M, M, -ONE, WORK, M, IR, LDST, ONE,
     $                  WORK( M*M+1 ), M )
            DSCALE = ZERO
            DSUM = ONE
            CALL DLASSQ( M*M, WORK( M*M+1 ), 1, DSCALE, DSUM )
*
            CALL DLACPY( 'Full', M, M, B( J1, J1 ), LDB, WORK( M*M+1 ),
     $                   M )
            CALL DGEMM( 'N', 'N', M, M, M, ONE, LI, LDST, T, LDST, ZERO,
     $                  WORK, M )
            CALL DGEMM( 'N', 'T', M, M, M, -ONE, WORK, M, IR, LDST, ONE,
     $                  WORK( M*M+1 ), M )
            CALL DLASSQ( M*M, WORK( M*M+1 ), 1, DSCALE, DSUM )
            SS = DSCALE*SQRT( DSUM )
            DTRONG = SS.LE.THRESH
            IF( .NOT.DTRONG )
     $         GO TO 70
         END IF
*
         Update (A(J1:J1+M-1, M+J1:N), B(J1:J1+M-1, M+J1:N)) and
*               (A(1:J1-1, J1:J1+M), B(1:J1-1, J1:J1+M)).
*
         CALL DROT( J1+1, A( 1, J1 ), 1, A( 1, J1+1 ), 1, IR( 1, 1 ),
     $             IR( 2, 1 ) )
         CALL DROT( J1+1, B( 1, J1 ), 1, B( 1, J1+1 ), 1, IR( 1, 1 ),
     $             IR( 2, 1 ) )
         CALL DROT( N-J1+1, A( J1, J1 ), LDA, A( J1+1, J1 ), LDA,
```

```
     $             LI( 1, 1 ), LI( 2, 1 ) )
         CALL DROT( N-J1+1, B( J1, J1 ), LDB, B( J1+1, J1 ), LDB,
     $             LI( 1, 1 ), LI( 2, 1 ) )
*
*        Set  N1-by-N2 (2,1) - blocks to ZERO.
*
         A( J1+1, J1 ) = ZERO
         B( J1+1, J1 ) = ZERO
*
*        Accumulate transformations into Q and Z if requested.
*
         IF( WANTZ )
     $      CALL DROT( N, Z( 1, J1 ), 1, Z( 1, J1+1 ), 1, IR( 1, 1 ),
     $                 IR( 2, 1 ) )
         IF( WANTQ )
     $      CALL DROT( N, Q( 1, J1 ), 1, Q( 1, J1+1 ), 1, LI( 1, 1 ),
     $                 LI( 2, 1 ) )
*
*        Exit with INFO = 0 if swap was successfully performed.
*
         RETURN
*
      ELSE
*
*        CASE 2: Swap 1-by-1 and 2-by-2 blocks, or 2-by-2
*                and 2-by-2 blocks.
*
*        Solve the generalized Sylvester equation
*                 S11 * R - L * S22 = SCALE * S12
*                 T11 * R - L * T22 = SCALE * T12
*        for R and L. Solutions in LI and IR.
*
         CALL DLACPY( 'Full', N1, N2, T( 1, N1+1 ), LDST, LI, LDST )
         CALL DLACPY( 'Full', N1, N2, S( 1, N1+1 ), LDST,
     $                IR( N2+1, N1+1 ), LDST )
         CALL DTGSY2( 'N', 0, N1, N2, S, LDST, S( N1+1, N1+1 ), LDST,
     $                IR( N2+1, N1+1 ), LDST, T, LDST, T( N1+1, N1+1 ),
     $                LDST, LI, LDST, SCALE, DSUM, DSCALE, IWORK, IDUM,
     $                LINFO )
*
*        Compute orthogonal matrix QL:
*
*                   QL**T * LI = [ TL ]
*                                [ 0  ]
*        where
*                   LI =  [      -L              ]
*                         [ SCALE * identity(N2) ]
*
         DO 10 I = 1, N2
            CALL DSCAL( N1, -ONE, LI( 1, I ), 1 )
```

```
            LI( N1+I, I ) = SCALE
   10    CONTINUE
         CALL DGEQR2( M, N2, LI, LDST, TAUL, WORK, LINFO )
         IF( LINFO.NE.0 )
     $      GO TO 70
         CALL DORG2R( M, M, N2, LI, LDST, TAUL, WORK, LINFO )
         IF( LINFO.NE.0 )
     $      GO TO 70
*
*        Compute orthogonal matrix RQ:
*
*                    IR * RQ**T =   [ 0  TR],
*
*         where IR = [ SCALE * identity(N1), R ]
*
         DO 20 I = 1, N1
            IR( N2+I, I ) = SCALE
   20    CONTINUE
         CALL DGERQ2( N1, M, IR( N2+1, 1 ), LDST, TAUR, WORK, LINFO )
         IF( LINFO.NE.0 )
     $      GO TO 70
         CALL DORGR2( M, M, N1, IR, LDST, TAUR, WORK, LINFO )
         IF( LINFO.NE.0 )
     $      GO TO 70
*
*        Perform the swapping tentatively:
*
         CALL DGEMM( 'T', 'N', M, M, M, ONE, LI, LDST, S, LDST, ZERO,
     $               WORK, M )
         CALL DGEMM( 'N', 'T', M, M, M, ONE, WORK, M, IR, LDST, ZERO, S,
     $               LDST )
         CALL DGEMM( 'T', 'N', M, M, M, ONE, LI, LDST, T, LDST, ZERO,
     $               WORK, M )
         CALL DGEMM( 'N', 'T', M, M, M, ONE, WORK, M, IR, LDST, ZERO, T,
     $               LDST )
         CALL DLACPY( 'F', M, M, S, LDST, SCPY, LDST )
         CALL DLACPY( 'F', M, M, T, LDST, TCPY, LDST )
         CALL DLACPY( 'F', M, M, IR, LDST, IRCOP, LDST )
         CALL DLACPY( 'F', M, M, LI, LDST, LICOP, LDST )
*
*        Triangularize the B-part by an RQ factorization.
*        Apply transformation (from left) to A-part, giving S.
*
         CALL DGERQ2( M, M, T, LDST, TAUR, WORK, LINFO )
         IF( LINFO.NE.0 )
     $      GO TO 70
         CALL DORMR2( 'R', 'T', M, M, M, T, LDST, TAUR, S, LDST, WORK,
     $                LINFO )
         IF( LINFO.NE.0 )
     $      GO TO 70
```

```
         CALL DORMR2( 'L', 'N', M, M, M, T, LDST, TAUR, IR, LDST, WORK,
     $               LINFO )
         IF( LINFO.NE.0 )
     $      GO TO 70
*
*        Compute F-norm(S21) in BRQA21. (T21 is 0.)
*
         DSCALE = ZERO
         DSUM = ONE
         DO 30 I = 1, N2
            CALL DLASSQ( N1, S( N2+1, I ), 1, DSCALE, DSUM )
   30    CONTINUE
         BRQA21 = DSCALE*SQRT( DSUM )
*
*        Triangularize the B-part by a QR factorization.
*        Apply transformation (from right) to A-part, giving S.
*
         CALL DGEQR2( M, M, TCPY, LDST, TAUL, WORK, LINFO )
         IF( LINFO.NE.0 )
     $      GO TO 70
         CALL DORM2R( 'L', 'T', M, M, M, TCPY, LDST, TAUL, SCPY, LDST,
     $               WORK, INFO )
         CALL DORM2R( 'R', 'N', M, M, M, TCPY, LDST, TAUL, LICOP, LDST,
     $               WORK, INFO )
         IF( LINFO.NE.0 )
     $      GO TO 70
*
*        Compute F-norm(S21) in BQRA21. (T21 is 0.)
*
         DSCALE = ZERO
         DSUM = ONE
         DO 40 I = 1, N2
            CALL DLASSQ( N1, SCPY( N2+1, I ), 1, DSCALE, DSUM )
   40    CONTINUE
         BQRA21 = DSCALE*SQRT( DSUM )
*
*        Decide which method to use.
*          Weak stability test:
*             F-norm(S21) <= O(EPS * F-norm((S, T)))
*
         IF( BQRA21.LE.BRQA21 .AND. BQRA21.LE.THRESH ) THEN
            CALL DLACPY( 'F', M, M, SCPY, LDST, S, LDST )
            CALL DLACPY( 'F', M, M, TCPY, LDST, T, LDST )
            CALL DLACPY( 'F', M, M, IRCOP, LDST, IR, LDST )
            CALL DLACPY( 'F', M, M, LICOP, LDST, LI, LDST )
         ELSE IF( BRQA21.GE.THRESH ) THEN
            GO TO 70
         END IF
*
*        Set lower triangle of B-part to zero
```

```
*
         CALL DLASET( 'Lower', M-1, M-1, ZERO, ZERO, T(2,1), LDST )
*
         IF( WANDS ) THEN
*
*           Strong stability test:
*              F-norm((A-QL*S*QR**T, B-QL*T*QR**T)) <= O(EPS*F-norm((A,B)))
*
            CALL DLACPY( 'Full', M, M, A( J1, J1 ), LDA, WORK( M*M+1 ),
     $                   M )
            CALL DGEMM( 'N', 'N', M, M, M, ONE, LI, LDST, S, LDST, ZERO,
     $                  WORK, M )
            CALL DGEMM( 'N', 'N', M, M, M, -ONE, WORK, M, IR, LDST, ONE,
     $                  WORK( M*M+1 ), M )
            DSCALE = ZERO
            DSUM = ONE
            CALL DLASSQ( M*M, WORK( M*M+1 ), 1, DSCALE, DSUM )
*
            CALL DLACPY( 'Full', M, M, B( J1, J1 ), LDB, WORK( M*M+1 ),
     $                   M )
            CALL DGEMM( 'N', 'N', M, M, M, ONE, LI, LDST, T, LDST, ZERO,
     $                  WORK, M )
            CALL DGEMM( 'N', 'N', M, M, M, -ONE, WORK, M, IR, LDST, ONE,
     $                  WORK( M*M+1 ), M )
            CALL DLASSQ( M*M, WORK( M*M+1 ), 1, DSCALE, DSUM )
            SS = DSCALE*SQRT( DSUM )
            DTRONG = ( SS.LE.THRESH )
            IF( .NOT.DTRONG )
     $         GO TO 70
*
         END IF
*
*        If the swap is accepted ("weakly" and "strongly"), apply the
*        transformations and set N1-by-N2 (2,1)-block to zero.
*
         CALL DLASET( 'Full', N1, N2, ZERO, ZERO, S(N2+1,1), LDST )
*
*        copy back M-by-M diagonal block starting at index J1 of (A, B)
*
         CALL DLACPY( 'F', M, M, S, LDST, A( J1, J1 ), LDA )
         CALL DLACPY( 'F', M, M, T, LDST, B( J1, J1 ), LDB )
         CALL DLASET( 'Full', LDST, LDST, ZERO, ZERO, T, LDST )
*
*        Standardize existing 2-by-2 blocks.
*
         CALL DLASET( 'Full', M, M, ZERO, ZERO, WORK, M )
         WORK( 1 ) = ONE
         T( 1, 1 ) = ONE
         IDUM = LWORK - M*M - 2
         IF( N2.GT.1 ) THEN
```

```
         CALL DLAGV2( A( J1, J1 ), LDA, B( J1, J1 ), LDB, AR, AI, BE,
     $                WORK( 1 ), WORK( 2 ), T( 1, 1 ), T( 2, 1 ) )
         WORK( M+1 ) = -WORK( 2 )
         WORK( M+2 ) = WORK( 1 )
         T( N2, N2 ) = T( 1, 1 )
         T( 1, 2 ) = -T( 2, 1 )
      END IF
      WORK( M*M ) = ONE
      T( M, M ) = ONE
*
      IF( N1.GT.1 ) THEN
         CALL DLAGV2( A( J1+N2, J1+N2 ), LDA, B( J1+N2, J1+N2 ), LDB,
     $                TAUR, TAUL, WORK( M*M+1 ), WORK( N2*M+N2+1 ),
     $                WORK( N2*M+N2+2 ), T( N2+1, N2+1 ),
     $                T( M, M-1 ) )
         WORK( M*M ) = WORK( N2*M+N2+1 )
         WORK( M*M-1 ) = -WORK( N2*M+N2+2 )
         T( M, M ) = T( N2+1, N2+1 )
         T( M-1, M ) = -T( M, M-1 )
      END IF
      CALL DGEMM( 'T', 'N', N2, N1, N2, ONE, WORK, M, A( J1, J1+N2 ),
     $            LDA, ZERO, WORK( M*M+1 ), N2 )
      CALL DLACPY( 'Full', N2, N1, WORK( M*M+1 ), N2, A( J1, J1+N2 ),
     $             LDA )
      CALL DGEMM( 'T', 'N', N2, N1, N2, ONE, WORK, M, B( J1, J1+N2 ),
     $            LDB, ZERO, WORK( M*M+1 ), N2 )
      CALL DLACPY( 'Full', N2, N1, WORK( M*M+1 ), N2, B( J1, J1+N2 ),
     $             LDB )
      CALL DGEMM( 'N', 'N', M, M, M, ONE, LI, LDST, WORK, M, ZERO,
     $            WORK( M*M+1 ), M )
      CALL DLACPY( 'Full', M, M, WORK( M*M+1 ), M, LI, LDST )
      CALL DGEMM( 'N', 'N', N2, N1, N1, ONE, A( J1, J1+N2 ), LDA,
     $            T( N2+1, N2+1 ), LDST, ZERO, WORK, N2 )
      CALL DLACPY( 'Full', N2, N1, WORK, N2, A( J1, J1+N2 ), LDA )
      CALL DGEMM( 'N', 'N', N2, N1, N1, ONE, B( J1, J1+N2 ), LDB,
     $            T( N2+1, N2+1 ), LDST, ZERO, WORK, N2 )
      CALL DLACPY( 'Full', N2, N1, WORK, N2, B( J1, J1+N2 ), LDB )
      CALL DGEMM( 'T', 'N', M, M, M, ONE, IR, LDST, T, LDST, ZERO,
     $            WORK, M )
      CALL DLACPY( 'Full', M, M, WORK, M, IR, LDST )
*
*     Accumulate transformations into Q and Z if requested.
*
      IF( WANTQ ) THEN
         CALL DGEMM( 'N', 'N', N, M, M, ONE, Q( 1, J1 ), LDQ, LI,
     $               LDST, ZERO, WORK, N )
         CALL DLACPY( 'Full', N, M, WORK, N, Q( 1, J1 ), LDQ )
*
      END IF
*
```

```
         IF( WANTZ ) THEN
            CALL DGEMM( 'N', 'N', N, M, M, ONE, Z( 1, J1 ), LDZ, IR,
     $                  LDST, ZERO, WORK, N )
            CALL DLACPY( 'Full', N, M, WORK, N, Z( 1, J1 ), LDZ )
*
         END IF
*
*        Update (A(J1:J1+M-1, M+J1:N), B(J1:J1+M-1, M+J1:N)) and
*                (A(1:J1-1, J1:J1+M), B(1:J1-1, J1:J1+M)).
*
         I = J1 + M
         IF( I.LE.N ) THEN
            CALL DGEMM( 'T', 'N', M, N-I+1, M, ONE, LI, LDST,
     $                  A( J1, I ), LDA, ZERO, WORK, M )
            CALL DLACPY( 'Full', M, N-I+1, WORK, M, A( J1, I ), LDA )
            CALL DGEMM( 'T', 'N', M, N-I+1, M, ONE, LI, LDST,
     $                  B( J1, I ), LDB, ZERO, WORK, M )
            CALL DLACPY( 'Full', M, N-I+1, WORK, M, B( J1, I ), LDB )
         END IF
         I = J1 - 1
         IF( I.GT.0 ) THEN
            CALL DGEMM( 'N', 'N', I, M, M, ONE, A( 1, J1 ), LDA, IR,
     $                  LDST, ZERO, WORK, I )
            CALL DLACPY( 'Full', I, M, WORK, I, A( 1, J1 ), LDA )
            CALL DGEMM( 'N', 'N', I, M, M, ONE, B( 1, J1 ), LDB, IR,
     $                  LDST, ZERO, WORK, I )
            CALL DLACPY( 'Full', I, M, WORK, I, B( 1, J1 ), LDB )
         END IF
*
*        Exit with INFO = 0 if swap was successfully performed.
*
         RETURN
*
      END IF
*
*     Exit with INFO = 1 if swap was rejected.
*
   70 CONTINUE
*
      INFO = 1
      RETURN
*
*     End of DTGEX2
*
      END
```

———-

36.5.4 lisp code

— dtgex2.lisp —

```
(let* ((zero 0.0d0)
       (one 1.0d0)
       (twenty 20.0d0)
       (ldst 4)
       (wands f2cl-lib:%true%))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 20.0d0 20.0d0) twenty)
           (type (f2cl-lib:integer4 4 4) ldst)
           (type f2cl-lib:logical wands)
           (ignorable zero one twenty ldst wands))
  (defun dtgex2
         (wantq wantz n a lda b ldb$ q ldq z ldz j1 n1 n2 work lwork info)
    (declare (type (array double-float (*)) work z q b a)
             (type (f2cl-lib:integer4) info lwork n2 n1 j1 ldz ldq ldb$ lda n)
             (type f2cl-lib:logical wantz wantq))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%)
         (q double-float q-%data% q-%offset%)
         (z double-float z-%data% z-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((ai (make-array 2 :element-type 'double-float))
             (ar (make-array 2 :element-type 'double-float))
             (be (make-array 2 :element-type 'double-float))
             (ir
              (make-array (the fixnum (reduce #'* (list ldst ldst)))
                          :element-type 'double-float))
             (ircop
              (make-array (the fixnum (reduce #'* (list ldst ldst)))
                          :element-type 'double-float))
             (li
              (make-array (the fixnum (reduce #'* (list ldst ldst)))
                          :element-type 'double-float))
             (licop
              (make-array (the fixnum (reduce #'* (list ldst ldst)))
                          :element-type 'double-float))
             (s
              (make-array (the fixnum (reduce #'* (list ldst ldst)))
                          :element-type 'double-float))
             (scpy
              (make-array (the fixnum (reduce #'* (list ldst ldst)))
                          :element-type 'double-float))
             (taul (make-array ldst :element-type 'double-float))
             (taur (make-array ldst :element-type 'double-float))
```

```
     (tcpy
      (make-array (the fixnum (reduce #'* (list ldst ldst)))
                  :element-type 'double-float))
     (iwork (make-array ldst :element-type 'f2cl-lib:integer4))
     (bqra21 0.0d0) (brqa21 0.0d0) (ddum 0.0d0) (dnorm 0.0d0)
     (dscale 0.0d0) (dsum 0.0d0) (eps 0.0d0) (f 0.0d0) (g 0.0d0)
     (sa 0.0d0) (sb 0.0d0) (scale 0.0d0) (smlnum 0.0d0) (ss 0.0d0)
     (thresh 0.0d0) (ws 0.0d0) (i 0) (idum 0) (linfo 0) (m 0)
     (dtrong nil) (weak nil)
     (t$
      (make-array (the fixnum (reduce #'* (list ldst ldst)))
                  :element-type 'double-float)))
(declare (type (array double-float (2)) ai ar be)
         (type (array double-float (*)) t$ ir ircop li licop s scpy
          taul taur tcpy)
         (type (array f2cl-lib:integer4 (*)) iwork)
         (type (double-float) bqra21 brqa21 ddum dnorm dscale dsum eps
          f g sa sb scale smlnum ss thresh ws)
         (type (f2cl-lib:integer4) i idum linfo m)
         (type f2cl-lib:logical dtrong weak))
(setf info 0)
(if (or (<= n 1) (<= n1 0) (<= n2 0))
    (go end_label))
(if (or (> n1 n) (> (f2cl-lib:int-add j1 n1) n))
    (go end_label))
(setf m (f2cl-lib:int-add n1 n2))
(cond
 ((< lwork
     (max (the f2cl-lib:integer4 1)
          (the f2cl-lib:integer4 (f2cl-lib:int-mul n m))
          (the f2cl-lib:integer4 (f2cl-lib:int-mul m m 2))))
  (setf info -16)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce
           (the f2cl-lib:integer4
                (max (the f2cl-lib:integer4 1)
                     (the f2cl-lib:integer4 (f2cl-lib:int-mul n m))
                     (the f2cl-lib:integer4 (f2cl-lib:int-mul m m 2))))
           'double-float))
  (go end_label)))
(setf weak f2cl-lib:%false%)
(setf dtrong f2cl-lib:%false%)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "Full" ldst ldst zero zero li ldst)
  (declare (ignore var-0 var-5))
  (when var-1 (setf ldst var-1))
  (when var-2 (setf ldst var-2))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldst var-6)))
```

```
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "Full" ldst ldst zero zero ir ldst)
  (declare (ignore var-0 var-5))
  (when var-1 (setf ldst var-1))
  (when var-2 (setf ldst var-2))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldst var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "Full" m m
     (f2cl-lib:array-slice a-%data% double-float (j1 j1)
                           ((1 lda) (1 *)) a-%offset%)
     lda s ldst)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldst var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "Full" m m
     (f2cl-lib:array-slice b-%data% double-float (j1 j1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ t$ ldst)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf ldst var-6)))
(setf eps (dlamch "P"))
(setf smlnum (/ (dlamch "S") eps))
(setf dscale zero)
(setf dsum one)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "Full" m m s ldst work m)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldst var-4))
  (when var-6 (setf m var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dlassq (f2cl-lib:int-mul m m) work 1 dscale dsum)
  (declare (ignore var-0 var-1 var-2))
  (when var-3 (setf dscale var-3))
  (when var-4 (setf dsum var-4)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "Full" m m t$ ldst work m)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldst var-4))
```

```
  (when var-6 (setf m var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dlassq (f2cl-lib:int-mul m m) work 1 dscale dsum)
  (declare (ignore var-0 var-1 var-2))
  (when var-3 (setf dscale var-3))
  (when var-4 (setf dsum var-4)))
(setf dnorm (* dscale (f2cl-lib:fsqrt dsum)))
(setf thresh (max (* twenty eps dnorm) smlnum))
(cond
 ((= m 2)
  (setf f
          (-
           (* (f2cl-lib:fref s (2 2) ((1 ldst) (1 ldst)))
              (f2cl-lib:fref t$ (1 1) ((1 ldst) (1 ldst))))
           (* (f2cl-lib:fref t$ (2 2) ((1 ldst) (1 ldst)))
              (f2cl-lib:fref s (1 1) ((1 ldst) (1 ldst))))))
  (setf g
          (-
           (* (f2cl-lib:fref s (2 2) ((1 ldst) (1 ldst)))
              (f2cl-lib:fref t$ (1 2) ((1 ldst) (1 ldst))))
           (* (f2cl-lib:fref t$ (2 2) ((1 ldst) (1 ldst)))
              (f2cl-lib:fref s (1 2) ((1 ldst) (1 ldst))))))
  (setf sb (abs (f2cl-lib:fref t$ (2 2) ((1 ldst) (1 ldst)))))
  (setf sa (abs (f2cl-lib:fref s (2 2) ((1 ldst) (1 ldst)))))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dlartg f g
       (f2cl-lib:array-slice ir double-float (1 2) ((1 ldst) (1 ldst)))
       (f2cl-lib:array-slice ir double-float (1 1) ((1 ldst) (1 ldst)))
       ddum)
    (declare (ignore var-2 var-3))
    (when var-0 (setf f var-0))
    (when var-1 (setf g var-1))
    (when var-4 (setf ddum var-4)))
  (setf (f2cl-lib:fref ir (2 1) ((1 ldst) (1 ldst)))
          (- (f2cl-lib:fref ir (1 2) ((1 ldst) (1 ldst)))))
  (setf (f2cl-lib:fref ir (2 2) ((1 ldst) (1 ldst)))
          (f2cl-lib:fref ir (1 1) ((1 ldst) (1 ldst))))
  (drot 2
   (f2cl-lib:array-slice s double-float (1 1) ((1 ldst) (1 ldst))) 1
   (f2cl-lib:array-slice s double-float (1 2) ((1 ldst) (1 ldst))) 1
   (f2cl-lib:array-slice ir double-float (1 1) ((1 ldst) (1 ldst)))
   (f2cl-lib:array-slice ir double-float (2 1) ((1 ldst) (1 ldst))))
  (drot 2
   (f2cl-lib:array-slice t$ double-float (1 1) ((1 ldst) (1 ldst))) 1
   (f2cl-lib:array-slice t$ double-float (1 2) ((1 ldst) (1 ldst))) 1
   (f2cl-lib:array-slice ir double-float (1 1) ((1 ldst) (1 ldst)))
   (f2cl-lib:array-slice ir double-float (2 1) ((1 ldst) (1 ldst))))
  (cond
   ((>= sa sb)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
        (dlartg
         (f2cl-lib:array-slice s double-float (1 1)
                               ((1 ldst) (1 ldst)))
         (f2cl-lib:array-slice s double-float (2 1)
                               ((1 ldst) (1 ldst)))
         (f2cl-lib:array-slice li double-float (1 1)
                               ((1 ldst) (1 ldst)))
         (f2cl-lib:array-slice li double-float (2 1)
                               ((1 ldst) (1 ldst)))
         ddum)
      (declare (ignore var-0 var-1 var-2 var-3))
      (when var-4 (setf ddum var-4))))
   (t
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dlartg
         (f2cl-lib:array-slice t$ double-float (1 1)
                               ((1 ldst) (1 ldst)))
         (f2cl-lib:array-slice t$ double-float (2 1)
                               ((1 ldst) (1 ldst)))
         (f2cl-lib:array-slice li double-float (1 1)
                               ((1 ldst) (1 ldst)))
         (f2cl-lib:array-slice li double-float (2 1)
                               ((1 ldst) (1 ldst)))
         ddum)
      (declare (ignore var-0 var-1 var-2 var-3))
      (when var-4 (setf ddum var-4)))))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (drot 2
       (f2cl-lib:array-slice s double-float (1 1) ((1 ldst) (1 ldst)))
       ldst
       (f2cl-lib:array-slice s double-float (2 1) ((1 ldst) (1 ldst)))
       ldst
       (f2cl-lib:array-slice li double-float (1 1) ((1 ldst) (1 ldst)))
       (f2cl-lib:array-slice li double-float (2 1)
                             ((1 ldst) (1 ldst))))
    (declare (ignore var-0 var-1 var-3 var-5 var-6))
    (when var-2 (setf ldst var-2))
    (when var-4 (setf ldst var-4)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (drot 2
       (f2cl-lib:array-slice t$ double-float (1 1) ((1 ldst) (1 ldst)))
       ldst
       (f2cl-lib:array-slice t$ double-float (2 1) ((1 ldst) (1 ldst)))
       ldst
       (f2cl-lib:array-slice li double-float (1 1) ((1 ldst) (1 ldst)))
       (f2cl-lib:array-slice li double-float (2 1)
                             ((1 ldst) (1 ldst))))
    (declare (ignore var-0 var-1 var-3 var-5 var-6))
    (when var-2 (setf ldst var-2))
    (when var-4 (setf ldst var-4)))
```

```
(setf (f2cl-lib:fref li (2 2) ((1 ldst) (1 ldst)))
        (f2cl-lib:fref li (1 1) ((1 ldst) (1 ldst))))
(setf (f2cl-lib:fref li (1 2) ((1 ldst) (1 ldst)))
        (- (f2cl-lib:fref li (2 1) ((1 ldst) (1 ldst)))))
(setf ws
        (+ (abs (f2cl-lib:fref s (2 1) ((1 ldst) (1 ldst))))
           (abs (f2cl-lib:fref t$ (2 1) ((1 ldst) (1 ldst))))))
(setf weak (<= ws thresh))
(if (not weak)
    (go label70))
(cond
 (wands
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "Full" m m
       (f2cl-lib:array-slice a-%data% double-float (j1 j1)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                             work-%offset%)
       m)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf m var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dgemm "N" "N" m m m one li ldst s ldst zero work m)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf m var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldst var-7))
    (when var-9 (setf ldst var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf m var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dgemm "N" "T" m m m (- one) work m ir ldst one
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                             work-%offset%)
       m)
    (declare (ignore var-0 var-1 var-5 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf m var-3))
```

```
  (when var-4 (setf m var-4))
  (when var-7 (setf m var-7))
  (when var-9 (setf ldst var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf m var-12)))
(setf dscale zero) (setf dsum one)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dlassq (f2cl-lib:int-mul m m)
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
     1 dscale dsum)
  (declare (ignore var-0 var-1 var-2))
  (when var-3 (setf dscale var-3))
  (when var-4 (setf dsum var-4)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "Full" m m
     (f2cl-lib:array-slice b-%data% double-float (j1 j1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
     m)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf m var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dgemm "N" "N" m m m one li ldst t$ ldst zero work m)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldst var-7))
  (when var-9 (setf ldst var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf m var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dgemm "N" "T" m m m (- one) work m ir ldst one
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
     m)
```

```
    (declare (ignore var-0 var-1 var-5 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf m var-3))
    (when var-4 (setf m var-4))
    (when var-7 (setf m var-7))
    (when var-9 (setf ldst var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf m var-12)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dlassq (f2cl-lib:int-mul m m)
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                             work-%offset%)
       1 dscale dsum)
    (declare (ignore var-0 var-1 var-2))
    (when var-3 (setf dscale var-3))
    (when var-4 (setf dsum var-4)))
  (setf ss (* dscale (f2cl-lib:fsqrt dsum)))
  (setf dtrong (<= ss thresh))
  (if (not dtrong)
      (go label70))))
(drot (f2cl-lib:int-add j1 1)
 (f2cl-lib:array-slice a-%data% double-float (1 j1) ((1 lda) (1 *))
                       a-%offset%)
 1
 (f2cl-lib:array-slice a-%data% double-float
                       (1 (f2cl-lib:int-add j1 1)) ((1 lda) (1 *))
                       a-%offset%)
 1 (f2cl-lib:array-slice ir double-float (1 1) ((1 ldst) (1 ldst)))
 (f2cl-lib:array-slice ir double-float (2 1) ((1 ldst) (1 ldst))))
(drot (f2cl-lib:int-add j1 1)
 (f2cl-lib:array-slice b-%data% double-float (1 j1) ((1 ldb$) (1 *))
                       b-%offset%)
 1
 (f2cl-lib:array-slice b-%data% double-float
                       (1 (f2cl-lib:int-add j1 1)) ((1 ldb$) (1 *))
                       b-%offset%)
 1 (f2cl-lib:array-slice ir double-float (1 1) ((1 ldst) (1 ldst)))
 (f2cl-lib:array-slice ir double-float (2 1) ((1 ldst) (1 ldst))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (drot (f2cl-lib:int-add (f2cl-lib:int-sub n j1) 1)
     (f2cl-lib:array-slice a-%data% double-float (j1 j1)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice a-%data% double-float ((+ j1 1) j1)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice li double-float (1 1) ((1 ldst) (1 ldst)))
     (f2cl-lib:array-slice li double-float (2 1)
                           ((1 ldst) (1 ldst))))
```

```
  (declare (ignore var-0 var-1 var-3 var-5 var-6))
  (when var-2 (setf lda var-2))
  (when var-4 (setf lda var-4)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (drot (f2cl-lib:int-add (f2cl-lib:int-sub n j1) 1)
     (f2cl-lib:array-slice b-%data% double-float (j1 j1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice b-%data% double-float ((+ j1 1) j1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice li double-float (1 1) ((1 ldst) (1 ldst)))
     (f2cl-lib:array-slice li double-float (2 1)
                           ((1 ldst) (1 ldst))))
  (declare (ignore var-0 var-1 var-3 var-5 var-6))
  (when var-2 (setf ldb$ var-2))
  (when var-4 (setf ldb$ var-4)))
(setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add j1 1) j1)
                      ((1 lda) (1 *)) a-%offset%)
        zero)
(setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add j1 1) j1)
                      ((1 ldb$) (1 *)) b-%offset%)
        zero)
(if wantz
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (drot n
         (f2cl-lib:array-slice z-%data% double-float (1 j1)
                               ((1 ldz) (1 *)) z-%offset%)
         1
         (f2cl-lib:array-slice z-%data% double-float
                               (1 (f2cl-lib:int-add j1 1))
                               ((1 ldz) (1 *)) z-%offset%)
         1
         (f2cl-lib:array-slice ir double-float (1 1)
                               ((1 ldst) (1 ldst)))
         (f2cl-lib:array-slice ir double-float (2 1)
                               ((1 ldst) (1 ldst))))
      (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
      (when var-0 (setf n var-0))))
(if wantq
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (drot n
         (f2cl-lib:array-slice q-%data% double-float (1 j1)
                               ((1 ldq) (1 *)) q-%offset%)
         1
         (f2cl-lib:array-slice q-%data% double-float
                               (1 (f2cl-lib:int-add j1 1))
                               ((1 ldq) (1 *)) q-%offset%)
         1
         (f2cl-lib:array-slice li double-float (1 1)
```

```
                                ((1 ldst) (1 ldst)))
            (f2cl-lib:array-slice li double-float (2 1)
                                ((1 ldst) (1 ldst))))
        (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
        (when var-0 (setf n var-0))))
 (go end_label))
(t
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlacpy "Full" n1 n2
      (f2cl-lib:array-slice t$ double-float
                            (1 (f2cl-lib:int-add n1 1))
                            ((1 ldst) (1 ldst)))
      ldst li ldst)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf n1 var-1))
   (when var-2 (setf n2 var-2))
   (when var-4 (setf ldst var-4))
   (when var-6 (setf ldst var-6)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlacpy "Full" n1 n2
      (f2cl-lib:array-slice s double-float (1 (f2cl-lib:int-add n1 1))
                            ((1 ldst) (1 ldst)))
      ldst
      (f2cl-lib:array-slice ir double-float
                            ((+ n2 1) (f2cl-lib:int-add n1 1))
                            ((1 ldst) (1 ldst)))
      ldst)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf n1 var-1))
   (when var-2 (setf n2 var-2))
   (when var-4 (setf ldst var-4))
   (when var-6 (setf ldst var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18
      var-19 var-20 var-21)
     (dtgsy2 "N" 0 n1 n2 s ldst
      (f2cl-lib:array-slice s double-float
                            ((+ n1 1) (f2cl-lib:int-add n1 1))
                            ((1 ldst) (1 ldst)))
      ldst
      (f2cl-lib:array-slice ir double-float
                            ((+ n2 1) (f2cl-lib:int-add n1 1))
                            ((1 ldst) (1 ldst)))
      ldst t$ ldst
      (f2cl-lib:array-slice t$ double-float
                            ((+ n1 1) (f2cl-lib:int-add n1 1))
                            ((1 ldst) (1 ldst)))
      ldst li ldst scale dsum dscale iwork idum linfo)
   (declare
```

```
   (ignore var-0 var-1 var-4 var-6 var-8 var-10 var-12 var-14
    var-19))
  (when var-2 (setf n1 var-2))
  (when var-3 (setf n2 var-3))
  (when var-5 (setf ldst var-5))
  (when var-7 (setf ldst var-7))
  (when var-9 (setf ldst var-9))
  (when var-11 (setf ldst var-11))
  (when var-13 (setf ldst var-13))
  (when var-15 (setf ldst var-15))
  (when var-16 (setf scale var-16))
  (when var-17 (setf dsum var-17))
  (when var-18 (setf dscale var-18))
  (when var-20 (setf idum var-20))
  (when var-21 (setf linfo var-21)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n2) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal n1 (- one)
         (f2cl-lib:array-slice li double-float (1 i)
                               ((1 ldst) (1 ldst)))
         1)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf n1 var-0)))
    (setf (f2cl-lib:fref li ((f2cl-lib:int-add n1 i) i)
                          ((1 ldst) (1 ldst)))
          scale)
   label10))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dgeqr2 m n2 li ldst taul work linfo)
  (declare (ignore var-2 var-4 var-5))
  (when var-0 (setf m var-0))
  (when var-1 (setf n2 var-1))
  (when var-3 (setf ldst var-3))
  (when var-6 (setf linfo var-6)))
(if (/= linfo 0)
    (go label70))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dorg2r m m n2 li ldst taul work linfo)
  (declare (ignore var-3 var-5 var-6))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n2 var-2))
  (when var-4 (setf ldst var-4))
  (when var-7 (setf linfo var-7)))
(if (/= linfo 0)
    (go label70))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
              ((> i n1) nil)
  (tagbody
    (setf (f2cl-lib:fref ir ((f2cl-lib:int-add n2 i) i)
                         ((1 ldst) (1 ldst)))
          scale)
   label20))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dgerq2 n1 m
     (f2cl-lib:array-slice ir double-float ((+ n2 1) 1)
                           ((1 ldst) (1 ldst)))
     ldst taur work linfo)
  (declare (ignore var-2 var-4 var-5))
  (when var-0 (setf n1 var-0))
  (when var-1 (setf m var-1))
  (when var-3 (setf ldst var-3))
  (when var-6 (setf linfo var-6)))
(if (/= linfo 0)
    (go label70))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dorgr2 m m n1 ir ldst taur work linfo)
  (declare (ignore var-3 var-5 var-6))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n1 var-2))
  (when var-4 (setf ldst var-4))
  (when var-7 (setf linfo var-7)))
(if (/= linfo 0)
    (go label70))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dgemm "T" "N" m m m one li ldst s ldst zero work m)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldst var-7))
  (when var-9 (setf ldst var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf m var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dgemm "N" "T" m m m one work m ir ldst zero s ldst)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
```

```
  (when var-5 (setf one var-5))
  (when var-7 (setf m var-7))
  (when var-9 (setf ldst var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf ldst var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dgemm "T" "N" m m m one li ldst t$ ldst zero work m)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldst var-7))
  (when var-9 (setf ldst var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf m var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dgemm "N" "T" m m m one work m ir ldst zero t$ ldst)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf m var-7))
  (when var-9 (setf ldst var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf ldst var-12)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "F" m m s ldst scpy ldst)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldst var-4))
  (when var-6 (setf ldst var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "F" m m t$ ldst tcpy ldst)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldst var-4))
  (when var-6 (setf ldst var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "F" m m ir ldst ircop ldst)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
```

```
  (when var-4 (setf ldst var-4))
  (when var-6 (setf ldst var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "F" m m li ldst licop ldst)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldst var-4))
  (when var-6 (setf ldst var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dgerq2 m m t$ ldst taur work linfo)
  (declare (ignore var-2 var-4 var-5))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-3 (setf ldst var-3))
  (when var-6 (setf linfo var-6)))
(if (/= linfo 0)
    (go label70))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (dormr2 "R" "T" m m m t$ ldst taur s ldst work linfo)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldst var-6))
  (when var-9 (setf ldst var-9))
  (when var-11 (setf linfo var-11)))
(if (/= linfo 0)
    (go label70))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (dormr2 "L" "N" m m m t$ ldst taur ir ldst work linfo)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldst var-6))
  (when var-9 (setf ldst var-9))
  (when var-11 (setf linfo var-11)))
(if (/= linfo 0)
    (go label70))
(setf dscale zero) (setf dsum one)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n2) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dlassq n1
```

```
          (f2cl-lib:array-slice s double-float ((+ n2 1) i)
                                ((1 ldst) (1 ldst)))
          1 dscale dsum)
       (declare (ignore var-1 var-2))
       (when var-0 (setf n1 var-0))
       (when var-3 (setf dscale var-3))
       (when var-4 (setf dsum var-4)))
   label30))
(setf brqa21 (* dscale (f2cl-lib:fsqrt dsum)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dgeqr2 m m tcpy ldst taul work linfo)
  (declare (ignore var-2 var-4 var-5))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-3 (setf ldst var-3))
  (when var-6 (setf linfo var-6)))
(if (/= linfo 0)
    (go label70))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (dorm2r "L" "T" m m m tcpy ldst taul scpy ldst work info)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldst var-6))
  (when var-9 (setf ldst var-9))
  (when var-11 (setf info var-11)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (dorm2r "R" "N" m m m tcpy ldst taul licop ldst work info)
  (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldst var-6))
  (when var-9 (setf ldst var-9))
  (when var-11 (setf info var-11)))
(if (/= linfo 0)
    (go label70))
(setf dscale zero) (setf dsum one)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n2) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dlassq n1
         (f2cl-lib:array-slice scpy double-float ((+ n2 1) i)
                               ((1 ldst) (1 ldst)))
```

```
          1 dscale dsum)
       (declare (ignore var-1 var-2))
       (when var-0 (setf n1 var-0))
       (when var-3 (setf dscale var-3))
       (when var-4 (setf dsum var-4)))
   label40))
(setf bqra21 (* dscale (f2cl-lib:fsqrt dsum)))
(cond
 ((and (<= bqra21 brqa21) (<= bqra21 thresh))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "F" m m scpy ldst s ldst)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf ldst var-4))
    (when var-6 (setf ldst var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "F" m m tcpy ldst t$ ldst)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf ldst var-4))
    (when var-6 (setf ldst var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "F" m m ircop ldst ir ldst)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf ldst var-4))
    (when var-6 (setf ldst var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "F" m m licop ldst li ldst)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf ldst var-4))
    (when var-6 (setf ldst var-6))))
 ((>= brqa21 thresh) (go label70)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "Lower" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice t$ double-float (2 1) ((1 ldst) (1 ldst)))
     ldst)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldst var-6)))
(cond
 (wands
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
      (dlacpy "Full" m m
       (f2cl-lib:array-slice a-%data% double-float (j1 j1)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                             work-%offset%)
       m)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf m var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dgemm "N" "N" m m m one li ldst s ldst zero work m)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf m var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldst var-7))
    (when var-9 (setf ldst var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf m var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dgemm "N" "N" m m m (- one) work m ir ldst one
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                             work-%offset%)
       m)
    (declare (ignore var-0 var-1 var-5 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf m var-3))
    (when var-4 (setf m var-4))
    (when var-7 (setf m var-7))
    (when var-9 (setf ldst var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf m var-12)))
  (setf dscale zero) (setf dsum one)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dlassq (f2cl-lib:int-mul m m)
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                             work-%offset%)
       1 dscale dsum)
    (declare (ignore var-0 var-1 var-2))
```

```
  (when var-3 (setf dscale var-3))
  (when var-4 (setf dsum var-4)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "Full" m m
     (f2cl-lib:array-slice b-%data% double-float (j1 j1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
     m)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf m var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dgemm "N" "N" m m m one li ldst t$ ldst zero work m)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldst var-7))
  (when var-9 (setf ldst var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf m var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dgemm "N" "N" m m m (- one) work m ir ldst one
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
     m)
  (declare (ignore var-0 var-1 var-5 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-7 (setf m var-7))
  (when var-9 (setf ldst var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf m var-12)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dlassq (f2cl-lib:int-mul m m)
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
```

```
        1 dscale dsum)
    (declare (ignore var-0 var-1 var-2))
    (when var-3 (setf dscale var-3))
    (when var-4 (setf dsum var-4)))
  (setf ss (* dscale (f2cl-lib:fsqrt dsum)))
  (setf dtrong (<= ss thresh))
  (if (not dtrong)
      (go label70))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "Full" n1 n2 zero zero
     (f2cl-lib:array-slice s double-float ((+ n2 1) 1)
                           ((1 ldst) (1 ldst)))
     ldst)
  (declare (ignore var-0 var-5))
  (when var-1 (setf n1 var-1))
  (when var-2 (setf n2 var-2))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldst var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "F" m m s ldst
     (f2cl-lib:array-slice a-%data% double-float (j1 j1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldst var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "F" m m t$ ldst
     (f2cl-lib:array-slice b-%data% double-float (j1 j1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldst var-4))
  (when var-6 (setf ldb$ var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "Full" ldst ldst zero zero t$ ldst)
  (declare (ignore var-0 var-5))
  (when var-1 (setf ldst var-1))
  (when var-2 (setf ldst var-2))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldst var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "Full" m m zero zero work m)
  (declare (ignore var-0 var-5))
```

```
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf m var-6)))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) one)
(setf (f2cl-lib:fref t$ (1 1) ((1 ldst) (1 ldst))) one)
(setf idum (f2cl-lib:int-sub lwork (f2cl-lib:int-mul m m) 2))
(cond
 ((> n2 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dlagv2
       (f2cl-lib:array-slice a-%data% double-float (j1 j1)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice b-%data% double-float (j1 j1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$ ar ai be
       (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (2) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice t$ double-float (1 1)
                             ((1 ldst) (1 ldst)))
       (f2cl-lib:array-slice t$ double-float (2 1)
                             ((1 ldst) (1 ldst))))
    (declare
     (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9 var-10))
    (when var-1 (setf lda var-1))
    (when var-3 (setf ldb$ var-3)))
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add m 1)) ((1 *))
                       work-%offset%)
        (- (f2cl-lib:fref work-%data% (2) ((1 *)) work-%offset%)))
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add m 2)) ((1 *))
                       work-%offset%)
        (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%))
  (setf (f2cl-lib:fref t$ (n2 n2) ((1 ldst) (1 ldst)))
        (f2cl-lib:fref t$ (1 1) ((1 ldst) (1 ldst))))
  (setf (f2cl-lib:fref t$ (1 2) ((1 ldst) (1 ldst)))
        (- (f2cl-lib:fref t$ (2 1) ((1 ldst) (1 ldst)))))))
(setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-mul m m)) ((1 *))
                     work-%offset%)
      one)
(setf (f2cl-lib:fref t$ (m m) ((1 ldst) (1 ldst))) one)
(cond
 ((> n1 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
     var-10)
    (dlagv2
     (f2cl-lib:array-slice a-%data% double-float
                           ((+ j1 n2) (f2cl-lib:int-add j1 n2))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% double-float
                           ((+ j1 n2) (f2cl-lib:int-add j1 n2))
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ taur taul
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul n2 m) n2 1))
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul n2 m) n2 2))
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice t$ double-float
                           ((+ n2 1) (f2cl-lib:int-add n2 1))
                           ((1 ldst) (1 ldst)))
     (f2cl-lib:array-slice t$ double-float
                           (m (f2cl-lib:int-sub m 1))
                           ((1 ldst) (1 ldst))))
  (declare
   (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9 var-10))
  (when var-1 (setf lda var-1))
  (when var-3 (setf ldb$ var-3)))
(setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-mul m m)) ((1 *))
                     work-%offset%)
        (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add (f2cl-lib:int-mul n2 m)
                                          n2 1))
                       ((1 *)) work-%offset%))
(setf (f2cl-lib:fref work-%data%
                     ((f2cl-lib:int-sub (f2cl-lib:int-mul m m) 1))
                     ((1 *)) work-%offset%)
        (-
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add (f2cl-lib:int-mul n2 m)
                                           n2 2))
                        ((1 *)) work-%offset%)))
(setf (f2cl-lib:fref t$ (m m) ((1 ldst) (1 ldst)))
        (f2cl-lib:fref t$
                       ((f2cl-lib:int-add n2 1)
                        (f2cl-lib:int-add n2 1))
                       ((1 ldst) (1 ldst))))
(setf (f2cl-lib:fref t$ ((f2cl-lib:int-sub m 1) m)
                     ((1 ldst) (1 ldst)))
```

```
          (-
           (f2cl-lib:fref t$ (m (f2cl-lib:int-sub m 1))
                          ((1 ldst) (1 ldst)))))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dgemm "T" "N" n2 n1 n2 one work m
     (f2cl-lib:array-slice a-%data% double-float
                           (j1 (f2cl-lib:int-add j1 n2))
                           ((1 lda) (1 *)) a-%offset%)
     lda zero
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
     n2)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf n2 var-2))
  (when var-3 (setf n1 var-3))
  (when var-4 (setf n2 var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf m var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf n2 var-12)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "Full" n2 n1
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
     n2
     (f2cl-lib:array-slice a-%data% double-float
                           (j1 (f2cl-lib:int-add j1 n2))
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n2 var-1))
  (when var-2 (setf n1 var-2))
  (when var-4 (setf n2 var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dgemm "T" "N" n2 n1 n2 one work m
     (f2cl-lib:array-slice b-%data% double-float
                           (j1 (f2cl-lib:int-add j1 n2))
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ zero
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
```

```
      n2)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf n2 var-2))
  (when var-3 (setf n1 var-3))
  (when var-4 (setf n2 var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf m var-7))
  (when var-9 (setf ldb$ var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf n2 var-12)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "Full" n2 n1
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
     n2
     (f2cl-lib:array-slice b-%data% double-float
                           (j1 (f2cl-lib:int-add j1 n2))
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n2 var-1))
  (when var-2 (setf n1 var-2))
  (when var-4 (setf n2 var-4))
  (when var-6 (setf ldb$ var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dgemm "N" "N" m m m one li ldst work m zero
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
     m)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldst var-7))
  (when var-9 (setf m var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf m var-12)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "Full" m m
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
     m li ldst)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
```

```
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldst var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dgemm "N" "N" n2 n1 n1 one
     (f2cl-lib:array-slice a-%data% double-float
                           (j1 (f2cl-lib:int-add j1 n2))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice t$ double-float
                           ((+ n2 1) (f2cl-lib:int-add n2 1))
                           ((1 ldst) (1 ldst)))
     ldst zero work n2)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf n2 var-2))
  (when var-3 (setf n1 var-3))
  (when var-4 (setf n1 var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf lda var-7))
  (when var-9 (setf ldst var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf n2 var-12)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "Full" n2 n1 work n2
     (f2cl-lib:array-slice a-%data% double-float
                           (j1 (f2cl-lib:int-add j1 n2))
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n2 var-1))
  (when var-2 (setf n1 var-2))
  (when var-4 (setf n2 var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dgemm "N" "N" n2 n1 n1 one
     (f2cl-lib:array-slice b-%data% double-float
                           (j1 (f2cl-lib:int-add j1 n2))
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice t$ double-float
                           ((+ n2 1) (f2cl-lib:int-add n2 1))
                           ((1 ldst) (1 ldst)))
     ldst zero work n2)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf n2 var-2))
  (when var-3 (setf n1 var-3))
```

```
  (when var-4 (setf n1 var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldb$ var-7))
  (when var-9 (setf ldst var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf n2 var-12)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "Full" n2 n1 work n2
     (f2cl-lib:array-slice b-%data% double-float
                           (j1 (f2cl-lib:int-add j1 n2))
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n2 var-1))
  (when var-2 (setf n1 var-2))
  (when var-4 (setf n2 var-4))
  (when var-6 (setf ldb$ var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dgemm "T" "N" m m m one ir ldst t$ ldst zero work m)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldst var-7))
  (when var-9 (setf ldst var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf m var-12)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "Full" m m work m ir ldst)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf m var-4))
  (when var-6 (setf ldst var-6)))
(cond
 (wantq
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dgemm "N" "N" n m m one
       (f2cl-lib:array-slice q-%data% double-float (1 j1)
                             ((1 ldq) (1 *)) q-%offset%)
       ldq li ldst zero work n)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-4 (setf m var-4))
```

```
    (when var-5 (setf one var-5))
    (when var-7 (setf ldq var-7))
    (when var-9 (setf ldst var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf n var-12)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "Full" n m work n
       (f2cl-lib:array-slice q-%data% double-float (1 j1)
                             ((1 ldq) (1 *)) q-%offset%)
       ldq)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf n var-4))
    (when var-6 (setf ldq var-6)))))
(cond
 (wantz
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dgemm "N" "N" n m m one
       (f2cl-lib:array-slice z-%data% double-float (1 j1)
                             ((1 ldz) (1 *)) z-%offset%)
       ldz ir ldst zero work n)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldz var-7))
    (when var-9 (setf ldst var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf n var-12)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "Full" n m work n
       (f2cl-lib:array-slice z-%data% double-float (1 j1)
                             ((1 ldz) (1 *)) z-%offset%)
       ldz)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf n var-4))
    (when var-6 (setf ldz var-6)))))
(setf i (f2cl-lib:int-add j1 m))
(cond
 ((<= i n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dgemm "T" "N" m (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) m
```

```
        one li ldst
        (f2cl-lib:array-slice a-%data% double-float (j1 i)
                              ((1 lda) (1 *)) a-%offset%)
        lda zero work m)
      (declare (ignore var-0 var-1 var-3 var-6 var-8 var-11))
      (when var-2 (setf m var-2))
      (when var-4 (setf m var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf ldst var-7))
      (when var-9 (setf lda var-9))
      (when var-10 (setf zero var-10))
      (when var-12 (setf m var-12)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "Full" m (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
         work m
         (f2cl-lib:array-slice a-%data% double-float (j1 i)
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-0 var-2 var-3 var-5))
      (when var-1 (setf m var-1))
      (when var-4 (setf m var-4))
      (when var-6 (setf lda var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (dgemm "T" "N" m (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) m
         one li ldst
         (f2cl-lib:array-slice b-%data% double-float (j1 i)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$ zero work m)
      (declare (ignore var-0 var-1 var-3 var-6 var-8 var-11))
      (when var-2 (setf m var-2))
      (when var-4 (setf m var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf ldst var-7))
      (when var-9 (setf ldb$ var-9))
      (when var-10 (setf zero var-10))
      (when var-12 (setf m var-12)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "Full" m (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
         work m
         (f2cl-lib:array-slice b-%data% double-float (j1 i)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-0 var-2 var-3 var-5))
      (when var-1 (setf m var-1))
      (when var-4 (setf m var-4))
      (when var-6 (setf ldb$ var-6)))))
  (setf i (f2cl-lib:int-sub j1 1))
  (cond
```

```
((> i 0)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12)
     (dgemm "N" "N" i m m one
      (f2cl-lib:array-slice a-%data% double-float (1 j1)
                            ((1 lda) (1 *)) a-%offset%)
      lda ir ldst zero work i)
   (declare (ignore var-0 var-1 var-6 var-8 var-11))
   (when var-2 (setf i var-2))
   (when var-3 (setf m var-3))
   (when var-4 (setf m var-4))
   (when var-5 (setf one var-5))
   (when var-7 (setf lda var-7))
   (when var-9 (setf ldst var-9))
   (when var-10 (setf zero var-10))
   (when var-12 (setf i var-12)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlacpy "Full" i m work i
      (f2cl-lib:array-slice a-%data% double-float (1 j1)
                            ((1 lda) (1 *)) a-%offset%)
      lda)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf i var-1))
   (when var-2 (setf m var-2))
   (when var-4 (setf i var-4))
   (when var-6 (setf lda var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12)
     (dgemm "N" "N" i m m one
      (f2cl-lib:array-slice b-%data% double-float (1 j1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$ ir ldst zero work i)
   (declare (ignore var-0 var-1 var-6 var-8 var-11))
   (when var-2 (setf i var-2))
   (when var-3 (setf m var-3))
   (when var-4 (setf m var-4))
   (when var-5 (setf one var-5))
   (when var-7 (setf ldb$ var-7))
   (when var-9 (setf ldst var-9))
   (when var-10 (setf zero var-10))
   (when var-12 (setf i var-12)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlacpy "Full" i m work i
      (f2cl-lib:array-slice b-%data% double-float (1 j1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf i var-1))
```

```
              (when var-2 (setf m var-2))
              (when var-4 (setf i var-4))
              (when var-6 (setf ldb$ var-6)))))
          (go end_label)))
       label70
        (setf info 1)
        (go end_label)
       end_label
        (return
         (values nil nil n nil lda nil ldb$ nil ldq nil ldz nil n1 n2 nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dtgex2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '(fortran-to-lisp::logical
                                              fortran-to-lisp::logical
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$ nil
                                              fortran-to-lisp::ldq nil
                                              fortran-to-lisp::ldz nil
                                              fortran-to-lisp::n1
                                              fortran-to-lisp::n2 nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

Chapter 37

LAPACK - General Matrices, Auxilliary Routines, Real

37.1 sgesc2

37.1.1 Axiom unit tests

— **sgesc2.input** —

```
)set break resume
)sys rm -f sgesc2.output
)spool sgesc2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

37.1.2 Axiom help page

— **sgesc2.help** —

```
==========================================================================
sgesc2 examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgesc2 (N, A, LDA, RHS, IPIV, JPIV, SCALE)
           SGESC2 solves a system of linear equations using the LU
           factorization with complete pivoting computed by sgetc2.

Function/Subroutine Documentation
   subroutine sgesc2 (integer N, real, dimension( lda, * ) A, integer LDA,
       real, dimension( * ) RHS, integer, dimension( * ) IPIV, integer,
       dimension( * ) JPIV, real SCALE)
       SGESC2 solves a system of linear equations using the LU factorization
       with complete pivoting computed by sgetc2.

Purpose:
   SGESC2 solves a system of linear equations

           A * X = scale* RHS

   with a general N-by-N matrix A using the LU factorization with
   complete pivoting computed by SGETC2.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the  LU part of the factorization of the n-by-n
   matrix A computed by SGETC2:  A = P * L * U * Q

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1, N).

 RHS
   RHS is REAL array, dimension (N).
   On entry, the right hand side vector b.
   On exit, the solution vector X.

 IPIV
   IPIV is INTEGER array, dimension (N).
   The pivot indices; for 1 <= i <= N, row i of the
   matrix has been interchanged with row IPIV(i).
```

```
 JPIV
   JPIV is INTEGER array, dimension (N).
   The pivot indices; for 1 <= j <= N, column j of the
   matrix has been interchanged with column JPIV(j).

 SCALE
   SCALE is REAL
   On exit, SCALE contains the scale factor. SCALE is chosen
   0 <= SCALE <= 1 to prevent owerflow in the solution.
```

———-

37.1.3 fortran code

— **sgesc2.f** —

```
*  =====================================================================
      SUBROUTINE SGESC2( N, A, LDA, RHS, IPIV, JPIV, SCALE )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            LDA, N
      REAL               SCALE
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), JPIV( * )
      REAL               A( LDA, * ), RHS( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, TWO
      PARAMETER          ( ONE = 1.0E+0, TWO = 2.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               BIGNUM, EPS, SMLNUM, TEMP
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLABAD, SLASWP, SSCAL
*     ..
```

```
*     .. External Functions ..
      INTEGER            ISAMAX
      REAL               SLAMCH
      EXTERNAL           ISAMAX, SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS
*     ..
*     .. Executable Statements ..
*
*      Set constant to control owerflow
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' ) / EPS
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
*
*     Apply permutations IPIV to RHS
*
      CALL SLASWP( 1, RHS, LDA, 1, N-1, IPIV, 1 )
*
*     Solve for L part
*
      DO 20 I = 1, N - 1
         DO 10 J = I + 1, N
            RHS( J ) = RHS( J ) - A( J, I )*RHS( I )
   10    CONTINUE
   20 CONTINUE
*
*     Solve for U part
*
      SCALE = ONE
*
*     Check for scaling
*
      I = ISAMAX( N, RHS, 1 )
      IF( TWO*SMLNUM*ABS( RHS( I ) ).GT.ABS( A( N, N ) ) ) THEN
         TEMP = ( ONE / TWO ) / ABS( RHS( I ) )
         CALL SSCAL( N, TEMP, RHS( 1 ), 1 )
         SCALE = SCALE*TEMP
      END IF
*
      DO 40 I = N, 1, -1
         TEMP = ONE / A( I, I )
         RHS( I ) = RHS( I )*TEMP
         DO 30 J = I + 1, N
            RHS( I ) = RHS( I ) - RHS( J )*( A( I, J )*TEMP )
   30    CONTINUE
   40 CONTINUE
*
```

```
*     Apply permutations JPIV to the solution (RHS)
*
      CALL SLASWP( 1, RHS, LDA, 1, N-1, JPIV, -1 )
      RETURN
*
*     End of SGESC2
*
      END
```

37.1.4 lisp code

— **sgesc2.lisp** —

```
(let* ((one 1.0) (two 2.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (ignorable one two))
  (defun sgesc2 (n a lda rhs ipiv jpiv scale)
    (declare (type (single-float) scale)
             (type (array f2cl-lib:integer4 (*)) jpiv ipiv)
             (type (array single-float (*)) rhs a)
             (type (f2cl-lib:integer4) lda n))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (rhs single-float rhs-%data% rhs-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (jpiv f2cl-lib:integer4 jpiv-%data% jpiv-%offset%))
      (prog ((bignum 0.0) (eps 0.0) (smlnum 0.0) (temp 0.0) (i 0) (j 0))
        (declare (type (single-float) bignum eps smlnum temp)
                 (type (f2cl-lib:integer4) i j))
        (setf eps (slamch "P"))
        (setf smlnum (/ (slamch "S") eps))
        (setf bignum (/ one smlnum))
        (multiple-value-bind (var-0 var-1)
            (slabad smlnum bignum)
          (declare (ignore))
          (when var-0 (setf smlnum var-0))
          (when var-1 (setf bignum var-1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (slaswp 1 rhs lda 1 (f2cl-lib:int-sub n 1) ipiv 1)
          (declare (ignore var-0 var-1 var-3 var-4 var-5 var-6))
          (when var-2 (setf lda var-2)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
          (tagbody
            (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
```

```
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref rhs-%data% (j) ((1 *)) rhs-%offset%)
                (- (f2cl-lib:fref rhs-%data% (j) ((1 *)) rhs-%offset%)
                   (*
                    (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                   a-%offset%)
                    (f2cl-lib:fref rhs-%data% (i) ((1 *))
                                   rhs-%offset%))))
       label10))
   label20))
(setf scale one)
(setf i
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (isamax n rhs 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(cond
 ((> (* two smlnum (abs (f2cl-lib:fref rhs (i) ((1 *)))))
     (abs (f2cl-lib:fref a (n n) ((1 lda) (1 *)))))
  (setf temp
          (/ (/ one two)
             (abs
              (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%))))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (sscal n temp
       (f2cl-lib:array-slice rhs-%data% single-float (1) ((1 *))
                             rhs-%offset%)
       1)
    (declare (ignore var-2 var-3))
    (when var-0 (setf n var-0))
    (when var-1 (setf temp var-1)))
  (setf scale (* scale temp))))
(f2cl-lib:fdo (i n (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf temp
            (/ one
               (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                              a-%offset%)))
    (setf (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%)
            (* (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%)
               temp))
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%)
                (- (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%)
                   (*
```

```
                            (f2cl-lib:fref rhs-%data% (j) ((1 *)) rhs-%offset%)
                            (*
                             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                            a-%offset%)
                             temp))))
                label30))
            label40))
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
             (slaswp 1 rhs lda 1 (f2cl-lib:int-sub n 1) jpiv -1)
           (declare (ignore var-0 var-1 var-3 var-4 var-5 var-6))
           (when var-2 (setf lda var-2)))
         (go end_label)
        end_label
         (return (values n nil lda nil nil nil scale))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgesc2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (single-float))
                                            :return-values
                                            '(fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::scale)
                                            :calls 'nil)))
```

———-

37.2 sgetc2

37.2.1 Axiom unit tests

— sgetc2.input —

```
)set break resume
```

```
)sys rm -f sgetc2.output
)spool sgetc2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

37.2.2 Axiom help page

— sgetc2.help —

```
==================================================================
sgetc2 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgetc2 (N, A, LDA, IPIV, JPIV, INFO)
           SGETC2 computes the LU factorization with complete pivoting of the
           general n-by-n matrix.

Function/Subroutine Documentation
   subroutine sgetc2 (integer N, real, dimension( lda, * ) A, integer LDA,
       integer, dimension( * ) IPIV, integer, dimension( * ) JPIV, integer
       INFO)
       SGETC2 computes the LU factorization with complete pivoting of the
       general n-by-n matrix.

Purpose:
   SGETC2 computes an LU factorization with complete pivoting of the
   n-by-n matrix A. The factorization has the form A = P * L * U * Q,
   where P and Q are permutation matrices, L is lower triangular with
   unit diagonal elements and U is upper triangular.

   This is the Level 2 BLAS algorithm.

Parameters:

 N
   N is INTEGER
```

```
  The order of the matrix A. N >= 0.

A
  A is REAL array, dimension (LDA, N)
  On entry, the n-by-n matrix A to be factored.
  On exit, the factors L and U from the factorization
  A = P*L*U*Q; the unit diagonal elements of L are not stored.
  If U(k, k) appears to be less than SMIN, U(k, k) is given the
  value of SMIN, i.e., giving a nonsingular perturbed system.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension(N).
  The pivot indices; for 1 <= i <= N, row i of the
  matrix has been interchanged with row IPIV(i).

JPIV
  JPIV is INTEGER array, dimension(N).
  The pivot indices; for 1 <= j <= N, column j of the
  matrix has been interchanged with column JPIV(j).

INFO
  INFO is INTEGER
  = 0: successful exit
  > 0: if INFO = k, U(k, k) is likely to produce owerflow if
       we try to solve for x in Ax = b. So U is perturbed to
       avoid the overflow.
```

37.2.3 fortran code

— sgetc2.f —

```
*  =====================================================================
      SUBROUTINE SGETC2( N, A, LDA, IPIV, JPIV, INFO )
*
*  -- LAPACK auxiliary routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, N
```

```
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), JPIV( * )
      REAL               A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IP, IPV, J, JP, JPV
      REAL               BIGNUM, EPS, SMIN, SMLNUM, XMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGER, SLABAD, SSWAP
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Set constants to control overflow
*
      INFO = 0
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' ) / EPS
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
*
*     Factorize A using complete pivoting.
*     Set pivots less than SMIN to SMIN.
*
      DO 40 I = 1, N - 1
*
*        Find max element in matrix A
*
         XMAX = ZERO
         DO 20 IP = I, N
            DO 10 JP = I, N
               IF( ABS( A( IP, JP ) ).GE.XMAX ) THEN
                  XMAX = ABS( A( IP, JP ) )
                  IPV = IP
                  JPV = JP
```

```
            END IF
   10       CONTINUE
   20    CONTINUE
         IF( I.EQ.1 )
     $      SMIN = MAX( EPS*XMAX, SMLNUM )
*
*        Swap rows
*
         IF( IPV.NE.I )
     $      CALL SSWAP( N, A( IPV, 1 ), LDA, A( I, 1 ), LDA )
         IPIV( I ) = IPV
*
*        Swap columns
*
         IF( JPV.NE.I )
     $      CALL SSWAP( N, A( 1, JPV ), 1, A( 1, I ), 1 )
         JPIV( I ) = JPV
*
*        Check for singularity
*
         IF( ABS( A( I, I ) ).LT.SMIN ) THEN
            INFO = I
            A( I, I ) = SMIN
         END IF
         DO 30 J = I + 1, N
            A( J, I ) = A( J, I ) / A( I, I )
   30    CONTINUE
         CALL SGER( N-I, N-I, -ONE, A( I+1, I ), 1, A( I, I+1 ), LDA,
     $              A( I+1, I+1 ), LDA )
   40 CONTINUE
*
      IF( ABS( A( N, N ) ).LT.SMIN ) THEN
         INFO = N
         A( N, N ) = SMIN
      END IF
*
*     Set last pivots to N
*
      IPIV( N ) = N
      JPIV( N ) = N
*
      RETURN
*
*     End of SGETC2
*
      END
```

———-

37.2.4 lisp code

— sgetc2.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sgetc2 (n a lda ipiv jpiv info)
    (declare (type (array f2cl-lib:integer4 (*)) jpiv ipiv)
             (type (array single-float (*)) a)
             (type (f2cl-lib:integer4) info lda n))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (jpiv f2cl-lib:integer4 jpiv-%data% jpiv-%offset%))
      (prog ((bignum 0.0) (eps 0.0) (smin 0.0) (smlnum 0.0) (xmax 0.0) (i 0)
             (ip 0) (ipv 0) (j 0) (jp 0) (jpv 0))
        (declare (type (single-float) bignum eps smin smlnum xmax)
                 (type (f2cl-lib:integer4) i ip ipv j jp jpv))
        (setf info 0)
        (setf eps (slamch "P"))
        (setf smlnum (/ (slamch "S") eps))
        (setf bignum (/ one smlnum))
        (multiple-value-bind (var-0 var-1)
            (slabad smlnum bignum)
          (declare (ignore))
          (when var-0 (setf smlnum var-0))
          (when var-1 (setf bignum var-1)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
          (tagbody
            (setf xmax zero)
            (f2cl-lib:fdo (ip i (f2cl-lib:int-add ip 1))
                          ((> ip n) nil)
              (tagbody
                (f2cl-lib:fdo (jp i (f2cl-lib:int-add jp 1))
                              ((> jp n) nil)
                  (tagbody
                    (cond
                     ((>= (abs (f2cl-lib:fref a (ip jp) ((1 lda) (1 *)))) xmax)
                      (setf xmax
                              (abs
                               (f2cl-lib:fref a-%data% (ip jp) ((1 lda) (1 *))
                                              a-%offset%)))
                      (setf ipv ip) (setf jpv jp)))
                    label10))
                label20))
            (if (= i 1)
```

```
    (setf smin (max (* eps xmax) smlnum)))
(if (/= ipv i)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (sswap n
         (f2cl-lib:array-slice a-%data% single-float (ipv 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice a-%data% single-float (i 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-1 var-3))
      (when var-0 (setf n var-0))
      (when var-2 (setf lda var-2))
      (when var-4 (setf lda var-4))))
(setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) ipv)
(if (/= jpv i)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (sswap n
         (f2cl-lib:array-slice a-%data% single-float (1 jpv)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% single-float (1 i)
                               ((1 lda) (1 *)) a-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0))))
(setf (f2cl-lib:fref jpiv-%data% (i) ((1 *)) jpiv-%offset%) jpv)
(cond
 ((< (abs (f2cl-lib:fref a (i i) ((1 lda) (1 *)))) smin)
  (setf info i)
  (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
          smin)))
(f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *)) a-%offset%)
            (/
             (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                            a-%offset%)
             (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                            a-%offset%)))
   label30))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (sger (f2cl-lib:int-sub n i) (f2cl-lib:int-sub n i) (- one)
     (f2cl-lib:array-slice a-%data% single-float ((+ i 1) i)
                           ((1 lda) (1 *)) a-%offset%)
     1
     (f2cl-lib:array-slice a-%data% single-float
                           (i (f2cl-lib:int-add i 1))
```

```
                                        ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice a-%data% single-float
                                        ((+ i 1) (f2cl-lib:int-add i 1))
                                        ((1 lda) (1 *)) a-%offset%)
                 lda)
               (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7))
               (when var-6 (setf lda var-6))
               (when var-8 (setf lda var-8)))
            label40))
         (cond
          ((< (abs (f2cl-lib:fref a (n n) ((1 lda) (1 *)))) smin) (setf info n)
           (setf (f2cl-lib:fref a-%data% (n n) ((1 lda) (1 *)) a-%offset%)
                  smin)))
         (setf (f2cl-lib:fref ipiv-%data% (n) ((1 *)) ipiv-%offset%) n)
         (setf (f2cl-lib:fref jpiv-%data% (n) ((1 *)) jpiv-%offset%) n)
         (go end_label)
        end_label
         (return (values n nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgetc2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

37.3 slange

37.3.1 Axiom unit tests

— **slange.input** —

```
)set break resume
)sys rm -f slange.output
)spool slange.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

37.3.2 Axiom help page

— **slange.help** —

```
======================================================================
slange examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       real function slange (NORM, M, N, A, LDA, WORK)
           SLANGE returns the value of the 1-norm, Frobenius norm, infinity-
           norm, or the largest absolute value of any element of a general
           rectangular matrix.

Function/Subroutine Documentation
   real function slange (character NORM, integer M, integer N, real,
       dimension( lda, * ) A, integer LDA, real, dimension( * ) WORK)
       SLANGE returns the value of the 1-norm, Frobenius norm, infinity-norm,
       or the largest absolute value of any element of a general rectangular
       matrix.

Purpose:
   SLANGE  returns the value of the one norm,  or the Frobenius norm, or
```

```
  the  infinity norm,  or the  element of  largest absolute value  of a
  real matrix A.

 Returns:

  SLANGE
       SLANGE = ( max(abs(A(i,j))), NORM = 'M' or 'm'
                (
                ( norm1(A),         NORM = '1', 'O' or 'o'
                (
                ( normI(A),         NORM = 'I' or 'i'
                (
                ( normF(A),         NORM = 'F', 'f', 'E' or 'e'

  where  norm1  denotes the  one norm of a matrix (maximum column sum),
  normI  denotes the  infinity norm  of a matrix  (maximum row sum) and
  normF  denotes the  Frobenius norm of a matrix (square root of sum of
  squares).  Note that  max(abs(A(i,j)))  is not a consistent matrix norm.

Parameters:

 NORM
  NORM is CHARACTER*1
  Specifies the value to be returned in SLANGE as described above.

 M
  M is INTEGER
  The number of rows of the matrix A.  M >= 0.  When M = 0,
  SLANGE is set to zero.

 N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.  When N = 0,
  SLANGE is set to zero.

 A
  A is REAL array, dimension (LDA,N)
  The m by n matrix A.

 LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(M,1).

 WORK
  WORK is REAL array, dimension (MAX(1,LWORK)),
  where LWORK >= M when NORM = 'I'; otherwise, WORK is not referenced.
```

37.3.3 fortran code

— slange.f —

```
*  =====================================================================
      REAL             FUNCTION SLANGE( NORM, M, N, A, LDA, WORK )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            LDA, M, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), WORK( * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               SCALE, SUM, VALUE, TEMP
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLASSQ
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, SISNAN
      EXTERNAL           LSAME, SISNAN
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      IF( MIN( M, N ).EQ.0 ) THEN
         VALUE = ZERO
      ELSE IF( LSAME( NORM, 'M' ) ) THEN
*
*        Find max(abs(A(i,j))).
*
         VALUE = ZERO
```

```
         DO 20 J = 1, N
            DO 10 I = 1, M
               TEMP = ABS( A( I, J ) )
               IF( VALUE.LT.TEMP .OR. SISNAN( TEMP ) ) VALUE = TEMP
   10       CONTINUE
   20    CONTINUE
      ELSE IF( ( LSAME( NORM, 'O' ) ) .OR. ( NORM.EQ.'1' ) ) THEN
*
*        Find norm1(A).
*
         VALUE = ZERO
         DO 40 J = 1, N
            SUM = ZERO
            DO 30 I = 1, M
               SUM = SUM + ABS( A( I, J ) )
   30       CONTINUE
            IF( VALUE.LT.SUM .OR. SISNAN( SUM ) ) VALUE = SUM
   40    CONTINUE
      ELSE IF( LSAME( NORM, 'I' ) ) THEN
*
*        Find normI(A).
*
         DO 50 I = 1, M
            WORK( I ) = ZERO
   50    CONTINUE
         DO 70 J = 1, N
            DO 60 I = 1, M
               WORK( I ) = WORK( I ) + ABS( A( I, J ) )
   60       CONTINUE
   70    CONTINUE
         VALUE = ZERO
         DO 80 I = 1, M
            TEMP = WORK( I )
            IF( VALUE.LT.TEMP .OR. SISNAN( TEMP ) ) VALUE = TEMP
   80    CONTINUE
      ELSE IF( ( LSAME( NORM, 'F' ) ) .OR. ( LSAME( NORM, 'E' ) ) ) THEN
*
*        Find normF(A).
*
         SCALE = ZERO
         SUM = ONE
         DO 90 J = 1, N
            CALL SLASSQ( M, A( 1, J ), 1, SCALE, SUM )
   90    CONTINUE
         VALUE = SCALE*SQRT( SUM )
      END IF
*
      SLANGE = VALUE
      RETURN
*
```

```
*     End of SLANGE
*
      END
```

———-

37.3.4 lisp code

— **slange.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun slange (norm m n a lda work)
    (declare (type (array single-float (*)) work a)
             (type (f2cl-lib:integer4) lda n m)
             (type (string *) norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (a single-float a-%data% a-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((scale 0.0) (sum 0.0) (value 0.0) (temp 0.0) (i 0) (j 0)
             (slange 0.0))
        (declare (type (f2cl-lib:integer4) i j)
                 (type (single-float) scale sum value temp slange))
        (cond
         ((= (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) 0)
          (setf value zero))
         ((multiple-value-bind (ret-val var-0 var-1)
              (lsame norm "M")
            (declare (ignore var-1))
            (when var-0 (setf norm var-0))
            ret-val)
          (setf value zero)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i m) nil)
                (tagbody
                  (setf temp
                          (abs
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%)))
                  (if (or (< value temp) (sisnan temp))
                      (setf value temp))
                 label10))
```

```
    label20)))
((or
  (multiple-value-bind (ret-val var-0 var-1)
      (lsame norm "O")
    (declare (ignore var-1))
    (when var-0 (setf norm var-0))
    ret-val)
  (f2cl-lib:fstring-= norm "1"))
 (setf value zero)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf sum zero)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i m) nil)
       (tagbody
         (setf sum
                 (+ sum
                    (abs
                     (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%))))
        label30))
     (if (or (< value sum) (sisnan sum))
         (setf value sum))
    label40)))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame norm "I")
   (declare (ignore var-1))
   (when var-0 (setf norm var-0))
   ret-val)
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i m) nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) zero)
    label50))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i m) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (+
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)
                  (abs
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%))))
        label60))
    label70))
```

```
          (setf value zero)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf temp (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
              (if (or (< value temp) (sisnan temp))
                  (setf value temp))
             label80)))
         ((or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame norm "F")
             (declare (ignore var-1))
             (when var-0 (setf norm var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame norm "E")
             (declare (ignore var-1))
             (when var-0 (setf norm var-0))
             ret-val))
          (setf scale zero) (setf sum one)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (slassq m
                   (f2cl-lib:array-slice a-%data% single-float (1 j)
                                         ((1 lda) (1 *)) a-%offset%)
                   1 scale sum)
                (declare (ignore var-1 var-2))
                (when var-0 (setf m var-0))
                (when var-3 (setf scale var-3))
                (when var-4 (setf sum var-4)))
             label90))
          (setf value (* scale (f2cl-lib:fsqrt sum)))))
        (setf slange value)
        (go end_label)
       end_label
        (return (values slange norm m nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slange
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
```

```
                                        (array single-float (*)))
                                      :return-values
                                      '(fortran-to_lisp::norm
                                        fortran-to_lisp::m nil nil nil
                                        nil)
                                      :calls '(fortran-to_lisp::sisnan))))
```

———-

37.4 slaqge

37.4.1 Axiom unit tests

— **slaqge.input** —

```
)set break resume
)sys rm -f slaqge.output
)spool slaqge.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

37.4.2 Axiom help page

— **slaqge.help** —

```
==================================================================
slaqge examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slaqge (M, N, A, LDA, R, C, ROWCND, COLCND, AMAX, EQUED)
```

```
          SLAQGE scales a general rectangular matrix, using row and column
          scaling factors computed by sgeequ.

Function/Subroutine Documentation
   subroutine slaqge (integer M, integer N, real, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) R, real, dimension( * ) C, real
       ROWCND, real COLCND, real AMAX, character EQUED)
       SLAQGE scales a general rectangular matrix, using row and column
       scaling factors computed by sgeequ.

Purpose:
   SLAQGE equilibrates a general M by N matrix A using the row and
   column scaling factors in the vectors R and C.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the M by N matrix A.
   On exit, the equilibrated matrix.  See EQUED for the form of
   the equilibrated matrix.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(M,1).

 R
   R is REAL array, dimension (M)
   The row scale factors for A.

 C
   C is REAL array, dimension (N)
   The column scale factors for A.

 ROWCND
   ROWCND is REAL
   Ratio of the smallest R(i) to the largest R(i).

 COLCND
   COLCND is REAL
   Ratio of the smallest C(i) to the largest C(i).
```

```
 AMAX
   AMAX is REAL
   Absolute value of largest matrix entry.

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done.
   = 'N':  No equilibration
   = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
   = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
   = 'B':  Both row and column equilibration, i.e., A has been
           replaced by diag(R) * A * diag(C).

Internal Parameters:

   THRESH is a threshold value used to decide if row or column scaling
   should be done based on the ratio of the row or column scaling
   factors.  If ROWCND < THRESH, row scaling is done, and if
   COLCND < THRESH, column scaling is done.

   LARGE and SMALL are threshold values used to decide if row scaling
   should be done based on the absolute size of the largest matrix
   element.  If AMAX > LARGE or AMAX < SMALL, row scaling is done.
```

———-

37.4.3 fortran code

— slaqge.f —

```
*  =====================================================================
      SUBROUTINE SLAQGE( M, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $                   EQUED )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED
      INTEGER            LDA, M, N
      REAL               AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), C( * ), R( * )
*     ..
```

```
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, THRESH
      PARAMETER          ( ONE = 1.0E+0, THRESH = 0.1E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               CJ, LARGE, SMALL
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( M.LE.0 .OR. N.LE.0 ) THEN
         EQUED = 'N'
         RETURN
      END IF
*
*     Initialize LARGE and SMALL.
*
      SMALL = SLAMCH( 'Safe minimum' ) / SLAMCH( 'Precision' )
      LARGE = ONE / SMALL
*
      IF( ROWCND.GE.THRESH .AND. AMAX.GE.SMALL .AND. AMAX.LE.LARGE )
     $     THEN
*
*        No row scaling
*
         IF( COLCND.GE.THRESH ) THEN
*
*           No column scaling
*
            EQUED = 'N'
         ELSE
*
*           Column scaling
*
            DO 20 J = 1, N
               CJ = C( J )
               DO 10 I = 1, M
                  A( I, J ) = CJ*A( I, J )
   10          CONTINUE
   20       CONTINUE
            EQUED = 'C'
```

```
         END IF
      ELSE IF( COLCND.GE.THRESH ) THEN
*
*        Row scaling, no column scaling
*
         DO 40 J = 1, N
            DO 30 I = 1, M
               A( I, J ) = R( I )*A( I, J )
   30       CONTINUE
   40    CONTINUE
         EQUED = 'R'
      ELSE
*
*        Row and column scaling
*
         DO 60 J = 1, N
            CJ = C( J )
            DO 50 I = 1, M
               A( I, J ) = CJ*R( I )*A( I, J )
   50       CONTINUE
   60    CONTINUE
         EQUED = 'B'
      END IF
*
      RETURN
*
*     End of SLAQGE
*
      END
```

———-

37.4.4 lisp code

— slaqge.lisp —

```
(let* ((one 1.0) (thresh 0.1))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.1 0.1) thresh)
           (ignorable one thresh))
  (defun slaqge (m n a lda r c rowcnd colcnd amax equed)
    (declare (type (string *) equed)
             (type (single-float) amax colcnd rowcnd)
             (type (array single-float (*)) c r a)
             (type (f2cl-lib:integer4) lda n m))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (r single-float r-%data% r-%offset%)
```

```
   (c single-float c-%data% c-%offset%)
   (equed character equed-%data% equed-%offset%))
(prog ((cj 0.0) (large 0.0) (small 0.0) (i 0) (j 0))
  (declare (type (single-float) cj large small)
           (type (f2cl-lib:integer4) i j))
  (cond
   ((or (<= m 0) (<= n 0))
    (f2cl-lib:f2cl-set-string equed "N" (string 1)) (go end_label)))
  (setf small (/ (slamch "Safe minimum") (slamch "Precision")))
  (setf large (/ one small))
  (cond
   ((and (>= rowcnd thresh) (>= amax small) (<= amax large))
    (cond
     ((>= colcnd thresh) (f2cl-lib:f2cl-set-string equed "N" (string 1)))
     (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf cj (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%)
                      (* cj
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)))
            label10))
        label20))
      (f2cl-lib:f2cl-set-string equed "C" (string 1)))))
   ((>= colcnd thresh)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                    (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)))
          label30))
      label40))
    (f2cl-lib:f2cl-set-string equed "R" (string 1)))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf cj (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
```

```
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i m) nil)
                (tagbody
                  (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%)
                          (* cj (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                            a-%offset%)))
                 label50))
             label60))
          (f2cl-lib:f2cl-set-string equed "B" (string 1))))
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil nil nil nil equed))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slaqge
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (single-float) (string))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
                                              nil fortran-to-lisp::equed)
                                            :calls 'nil)))
```

———-

37.5 stgex2

37.5.1 Axiom unit tests

— **stgex2.input** —

```
)set break resume
)sys rm -f stgex2.output
)spool stgex2.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

37.5.2 Axiom help page

— **stgex2.help** —

```
=====================================================================
stgex2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine stgex2 (WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z, LDZ, J1,
           N1, N2, WORK, LWORK, INFO)
           STGEX2 swaps adjacent diagonal blocks in an upper (quasi)
           triangular matrix pair by an orthogonal equivalence transformation.

Function/Subroutine Documentation
   subroutine stgex2 (logical WANTQ, logical WANTZ, integer N, real,
       dimension( lda, * ) A, integer LDA, real, dimension( ldb, * ) B,
       integer LDB, real, dimension( ldq, * ) Q, integer LDQ, real, dimension(
       ldz, * ) Z, integer LDZ, integer J1, integer N1, integer N2, real,
       dimension( * ) WORK, integer LWORK, integer INFO)
       STGEX2 swaps adjacent diagonal blocks in an upper (quasi) triangular
       matrix pair by an orthogonal equivalence transformation.

Purpose:
   STGEX2 swaps adjacent diagonal blocks (A11, B11) and (A22, B22)
   of size 1-by-1 or 2-by-2 in an upper (quasi) triangular matrix pair
   (A, B) by an orthogonal equivalence transformation.

   (A, B) must be in generalized real Schur canonical form (as returned
   by SGGES), i.e. A is block upper triangular with 1-by-1 and 2-by-2
   diagonal blocks. B is upper triangular.

   Optionally, the matrices Q and Z of generalized Schur vectors are
   updated.
```

```
   Q(in) * A(in) * Z(in)**T = Q(out) * A(out) * Z(out)**T
   Q(in) * B(in) * Z(in)**T = Q(out) * B(out) * Z(out)**T

Parameters:

 WANTQ
   WANTQ is LOGICAL
   .TRUE. : update the left transformation matrix Q;
   .FALSE.: do not update Q.

 WANTZ
   WANTZ is LOGICAL
   .TRUE. : update the right transformation matrix Z;
   .FALSE.: do not update Z.

 N
   N is INTEGER
   The order of the matrices A and B. N >= 0.

 A
   A is REAL arrays, dimensions (LDA,N)
   On entry, the matrix A in the pair (A, B).
   On exit, the updated matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A. LDA >= max(1,N).

 B
   B is REAL arrays, dimensions (LDB,N)
   On entry, the matrix B in the pair (A, B).
   On exit, the updated matrix B.

 LDB
   LDB is INTEGER
   The leading dimension of the array B. LDB >= max(1,N).

 Q
   Q is REAL array, dimension (LDZ,N)
   On entry, if WANTQ = .TRUE., the orthogonal matrix Q.
   On exit, the updated matrix Q.
   Not referenced if WANTQ = .FALSE..

 LDQ
   LDQ is INTEGER
   The leading dimension of the array Q. LDQ >= 1.
   If WANTQ = .TRUE., LDQ >= N.

 Z
```

```
  Z is REAL array, dimension (LDZ,N)
  On entry, if WANTZ =.TRUE., the orthogonal matrix Z.
  On exit, the updated matrix Z.
  Not referenced if WANTZ = .FALSE..

LDZ
  LDZ is INTEGER
  The leading dimension of the array Z. LDZ >= 1.
  If WANTZ = .TRUE., LDZ >= N.

J1
  J1 is INTEGER
  The index to the first block (A11, B11). 1 <= J1 <= N.

N1
  N1 is INTEGER
  The order of the first block (A11, B11). N1 = 0, 1 or 2.

N2
  N2 is INTEGER
  The order of the second block (A22, B22). N2 = 0, 1 or 2.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK)).

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  LWORK >=  MAX( N*(N2+N1), (N2+N1)*(N2+N1)*2 )

INFO
  INFO is INTEGER
  =0: Successful exit
  >0: If INFO = 1, the transformed matrix (A, B) would be
      too far from generalized Schur form; the blocks are
      not swapped and (A, B) and (Q, Z) are unchanged.
      The problem of swapping is too ill-conditioned.
  <0: If INFO = -16: LWORK is too small. Appropriate value
      for LWORK is returned in WORK(1).

Further Details:

  In the current code both weak and strong stability tests are
  performed. The user can omit the strong stability test by changing
  the internal logical parameter WANDS to .FALSE.. See ref. [2] for details.
```

———-

37.5.3 fortran code

— stgex2.f —

```
*  =====================================================================
      SUBROUTINE STGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                   LDZ, J1, N1, N2, WORK, LWORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      LOGICAL            WANTQ, WANTZ
      INTEGER            INFO, J1, LDA, LDB, LDQ, LDZ, LWORK, N, N1, N2
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), B( LDB, * ), Q( LDQ, * ),
     $                   WORK( * ), Z( LDZ, * )
*     ..
*
*  =====================================================================
*  Replaced various illegal calls to SCOPY by calls to SLASET, or by DO
*  loops. Sven Hammarling, 1/5/02.
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               TWENTY
      PARAMETER          ( TWENTY = 2.0E+01 )
      INTEGER            LDST
      PARAMETER          ( LDST = 4 )
      LOGICAL            WANDS
      PARAMETER          ( WANDS = .TRUE. )
*     ..
*     .. Local Scalars ..
      LOGICAL            STRONG, WEAK
      INTEGER            I, IDUM, LINFO, M
      REAL               BQRA21, BRQA21, DDUM, DNORM, DSCALE, DSUM, EPS,
     $                   F, G, SA, SB, SCALE, SMLNUM, SS, THRESH, WS
*     ..
*     .. Local Arrays ..
      INTEGER            IWORK( LDST )
      REAL               AI( 2 ), AR( 2 ), BE( 2 ), IR( LDST, LDST ),
     $                   IRCOP( LDST, LDST ), LI( LDST, LDST ),
     $                   LICOP( LDST, LDST ), S( LDST, LDST ),
     $                   SCPY( LDST, LDST ), T( LDST, LDST ),
     $                   TAUL( LDST ), TAUR( LDST ), TCPY( LDST, LDST )
```

```
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEMM, SGEQR2, SGERQ2, SLACPY, SLAGV2, SLARTG,
     $                   SLASET, SLASSQ, SORG2R, SORGR2, SORM2R, SORMR2,
     $                   SROT, SSCAL, STGSY2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
*     Quick return if possible
*
      IF( N.LE.1 .OR. N1.LE.0 .OR. N2.LE.0 )
     $   RETURN
      IF( N1.GT.N .OR. ( J1+N1 ).GT.N )
     $   RETURN
      M = N1 + N2
      IF( LWORK.LT.MAX( N*M, M*M*2 ) ) THEN
         INFO = -16
         WORK( 1 ) = MAX( N*M, M*M*2 )
         RETURN
      END IF
*
      WEAK = .FALSE.
      STRONG = .FALSE.
*
*     Make a local copy of selected block
*
      CALL SLASET( 'Full', LDST, LDST, ZERO, ZERO, LI, LDST )
      CALL SLASET( 'Full', LDST, LDST, ZERO, ZERO, IR, LDST )
      CALL SLACPY( 'Full', M, M, A( J1, J1 ), LDA, S, LDST )
      CALL SLACPY( 'Full', M, M, B( J1, J1 ), LDB, T, LDST )
*
*     Compute threshold for testing acceptance of swapping.
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' ) / EPS
      DSCALE = ZERO
      DSUM = ONE
      CALL SLACPY( 'Full', M, M, S, LDST, WORK, M )
      CALL SLASSQ( M*M, WORK, 1, DSCALE, DSUM )
      CALL SLACPY( 'Full', M, M, T, LDST, WORK, M )
      CALL SLASSQ( M*M, WORK, 1, DSCALE, DSUM )
```

```
      DNORM = DSCALE*SQRT( DSUM )
*
*     THRES has been changed from
*        THRESH = MAX( TEN*EPS*SA, SMLNUM )
*     to
*        THRESH = MAX( TWENTY*EPS*SA, SMLNUM )
*     on 04/01/10.
*     "Bug" reported by Ondra Kamenik, confirmed by Julie Langou, fixed by
*     Jim Demmel and Guillaume Revy. See forum post 1783.
*
      THRESH = MAX( TWENTY*EPS*DNORM, SMLNUM )
*
      IF( M.EQ.2 ) THEN
*
*        CASE 1: Swap 1-by-1 and 1-by-1 blocks.
*
*        Compute orthogonal QL and RQ that swap 1-by-1 and 1-by-1 blocks
*        using Givens rotations and perform the swap tentatively.
*
         F = S( 2, 2 )*T( 1, 1 ) - T( 2, 2 )*S( 1, 1 )
         G = S( 2, 2 )*T( 1, 2 ) - T( 2, 2 )*S( 1, 2 )
         SB = ABS( T( 2, 2 ) )
         SA = ABS( S( 2, 2 ) )
         CALL SLARTG( F, G, IR( 1, 2 ), IR( 1, 1 ), DDUM )
         IR( 2, 1 ) = -IR( 1, 2 )
         IR( 2, 2 ) = IR( 1, 1 )
         CALL SROT( 2, S( 1, 1 ), 1, S( 1, 2 ), 1, IR( 1, 1 ),
     $              IR( 2, 1 ) )
         CALL SROT( 2, T( 1, 1 ), 1, T( 1, 2 ), 1, IR( 1, 1 ),
     $              IR( 2, 1 ) )
         IF( SA.GE.SB ) THEN
            CALL SLARTG( S( 1, 1 ), S( 2, 1 ), LI( 1, 1 ), LI( 2, 1 ),
     $                   DDUM )
         ELSE
            CALL SLARTG( T( 1, 1 ), T( 2, 1 ), LI( 1, 1 ), LI( 2, 1 ),
     $                   DDUM )
         END IF
         CALL SROT( 2, S( 1, 1 ), LDST, S( 2, 1 ), LDST, LI( 1, 1 ),
     $              LI( 2, 1 ) )
         CALL SROT( 2, T( 1, 1 ), LDST, T( 2, 1 ), LDST, LI( 1, 1 ),
     $              LI( 2, 1 ) )
         LI( 2, 2 ) = LI( 1, 1 )
         LI( 1, 2 ) = -LI( 2, 1 )
*
*        Weak stability test:
*           |S21| + |T21| <= O(EPS * F-norm((S, T)))
*
         WS = ABS( S( 2, 1 ) ) + ABS( T( 2, 1 ) )
         WEAK = WS.LE.THRESH
         IF( .NOT.WEAK )
```

```
     $      GO TO 70
*
         IF( WANDS ) THEN
*
*           Strong stability test:
*           F-norm((A-QL**T*S*QR, B-QL**T*T*QR)) <= O(EPS*F-norm((A, B)))
*
            CALL SLACPY( 'Full', M, M, A( J1, J1 ), LDA, WORK( M*M+1 ),
     $                   M )
            CALL SGEMM( 'N', 'N', M, M, M, ONE, LI, LDST, S, LDST, ZERO,
     $                  WORK, M )
            CALL SGEMM( 'N', 'T', M, M, M, -ONE, WORK, M, IR, LDST, ONE,
     $                  WORK( M*M+1 ), M )
            DSCALE = ZERO
            DSUM = ONE
            CALL SLASSQ( M*M, WORK( M*M+1 ), 1, DSCALE, DSUM )
*
            CALL SLACPY( 'Full', M, M, B( J1, J1 ), LDB, WORK( M*M+1 ),
     $                   M )
            CALL SGEMM( 'N', 'N', M, M, M, ONE, LI, LDST, T, LDST, ZERO,
     $                  WORK, M )
            CALL SGEMM( 'N', 'T', M, M, M, -ONE, WORK, M, IR, LDST, ONE,
     $                  WORK( M*M+1 ), M )
            CALL SLASSQ( M*M, WORK( M*M+1 ), 1, DSCALE, DSUM )
            SS = DSCALE*SQRT( DSUM )
            STRONG = SS.LE.THRESH
            IF( .NOT.STRONG )
     $         GO TO 70
         END IF
*
*        Update (A(J1:J1+M-1, M+J1:N), B(J1:J1+M-1, M+J1:N)) and
*               (A(1:J1-1, J1:J1+M), B(1:J1-1, J1:J1+M)).
*
         CALL SROT( J1+1, A( 1, J1 ), 1, A( 1, J1+1 ), 1, IR( 1, 1 ),
     $              IR( 2, 1 ) )
         CALL SROT( J1+1, B( 1, J1 ), 1, B( 1, J1+1 ), 1, IR( 1, 1 ),
     $              IR( 2, 1 ) )
         CALL SROT( N-J1+1, A( J1, J1 ), LDA, A( J1+1, J1 ), LDA,
     $              LI( 1, 1 ), LI( 2, 1 ) )
         CALL SROT( N-J1+1, B( J1, J1 ), LDB, B( J1+1, J1 ), LDB,
     $              LI( 1, 1 ), LI( 2, 1 ) )
*
*        Set  N1-by-N2 (2,1) - blocks to ZERO.
*
         A( J1+1, J1 ) = ZERO
         B( J1+1, J1 ) = ZERO
*
*        Accumulate transformations into Q and Z if requested.
*
         IF( WANTZ )
```

```
     $      CALL SROT( N, Z( 1, J1 ), 1, Z( 1, J1+1 ), 1, IR( 1, 1 ),
     $                 IR( 2, 1 ) )
         IF( WANTQ )
     $      CALL SROT( N, Q( 1, J1 ), 1, Q( 1, J1+1 ), 1, LI( 1, 1 ),
     $                 LI( 2, 1 ) )
*
*        Exit with INFO = 0 if swap was successfully performed.
*
         RETURN
*
      ELSE
*
*        CASE 2: Swap 1-by-1 and 2-by-2 blocks, or 2-by-2
*                and 2-by-2 blocks.
*
*        Solve the generalized Sylvester equation
*                 S11 * R - L * S22 = SCALE * S12
*                 T11 * R - L * T22 = SCALE * T12
*        for R and L. Solutions in LI and IR.
*
         CALL SLACPY( 'Full', N1, N2, T( 1, N1+1 ), LDST, LI, LDST )
         CALL SLACPY( 'Full', N1, N2, S( 1, N1+1 ), LDST,
     $                IR( N2+1, N1+1 ), LDST )
         CALL STGSY2( 'N', 0, N1, N2, S, LDST, S( N1+1, N1+1 ), LDST,
     $                IR( N2+1, N1+1 ), LDST, T, LDST, T( N1+1, N1+1 ),
     $                LDST, LI, LDST, SCALE, DSUM, DSCALE, IWORK, IDUM,
     $                LINFO )
*
*        Compute orthogonal matrix QL:
*
*                    QL**T * LI = [ TL ]
*                                 [ 0  ]
*        where
*                    LI =  [      -L              ]
*                          [ SCALE * identity(N2) ]
*
         DO 10 I = 1, N2
            CALL SSCAL( N1, -ONE, LI( 1, I ), 1 )
            LI( N1+I, I ) = SCALE
   10    CONTINUE
         CALL SGEQR2( M, N2, LI, LDST, TAUL, WORK, LINFO )
         IF( LINFO.NE.0 )
     $      GO TO 70
         CALL SORG2R( M, M, N2, LI, LDST, TAUL, WORK, LINFO )
         IF( LINFO.NE.0 )
     $      GO TO 70
*
*        Compute orthogonal matrix RQ:
*
*                    IR * RQ**T =   [ 0  TR],
```

```
*
*         where IR = [ SCALE * identity(N1), R ]
*
         DO 20 I = 1, N1
            IR( N2+I, I ) = SCALE
   20    CONTINUE
         CALL SGERQ2( N1, M, IR( N2+1, 1 ), LDST, TAUR, WORK, LINFO )
         IF( LINFO.NE.0 )
     $      GO TO 70
         CALL SORGR2( M, M, N1, IR, LDST, TAUR, WORK, LINFO )
         IF( LINFO.NE.0 )
     $      GO TO 70
*
*        Perform the swapping tentatively:
*
         CALL SGEMM( 'T', 'N', M, M, M, ONE, LI, LDST, S, LDST, ZERO,
     $               WORK, M )
         CALL SGEMM( 'N', 'T', M, M, M, ONE, WORK, M, IR, LDST, ZERO, S,
     $               LDST )
         CALL SGEMM( 'T', 'N', M, M, M, ONE, LI, LDST, T, LDST, ZERO,
     $               WORK, M )
         CALL SGEMM( 'N', 'T', M, M, M, ONE, WORK, M, IR, LDST, ZERO, T,
     $               LDST )
         CALL SLACPY( 'F', M, M, S, LDST, SCPY, LDST )
         CALL SLACPY( 'F', M, M, T, LDST, TCPY, LDST )
         CALL SLACPY( 'F', M, M, IR, LDST, IRCOP, LDST )
         CALL SLACPY( 'F', M, M, LI, LDST, LICOP, LDST )
*
*        Triangularize the B-part by an RQ factorization.
*        Apply transformation (from left) to A-part, giving S.
*
         CALL SGERQ2( M, M, T, LDST, TAUR, WORK, LINFO )
         IF( LINFO.NE.0 )
     $      GO TO 70
         CALL SORMR2( 'R', 'T', M, M, M, T, LDST, TAUR, S, LDST, WORK,
     $                LINFO )
         IF( LINFO.NE.0 )
     $      GO TO 70
         CALL SORMR2( 'L', 'N', M, M, M, T, LDST, TAUR, IR, LDST, WORK,
     $                LINFO )
         IF( LINFO.NE.0 )
     $      GO TO 70
*
*        Compute F-norm(S21) in BRQA21. (T21 is 0.)
*
         DSCALE = ZERO
         DSUM = ONE
         DO 30 I = 1, N2
            CALL SLASSQ( N1, S( N2+1, I ), 1, DSCALE, DSUM )
   30    CONTINUE
```

```
      BRQA21 = DSCALE*SQRT( DSUM )
*
*     Triangularize the B-part by a QR factorization.
*     Apply transformation (from right) to A-part, giving S.
*
      CALL SGEQR2( M, M, TCPY, LDST, TAUL, WORK, LINFO )
      IF( LINFO.NE.0 )
     $   GO TO 70
      CALL SORM2R( 'L', 'T', M, M, M, TCPY, LDST, TAUL, SCPY, LDST,
     $             WORK, INFO )
      CALL SORM2R( 'R', 'N', M, M, M, TCPY, LDST, TAUL, LICOP, LDST,
     $             WORK, INFO )
      IF( LINFO.NE.0 )
     $   GO TO 70
*
*     Compute F-norm(S21) in BQRA21. (T21 is 0.)
*
      DSCALE = ZERO
      DSUM = ONE
      DO 40 I = 1, N2
         CALL SLASSQ( N1, SCPY( N2+1, I ), 1, DSCALE, DSUM )
   40 CONTINUE
      BQRA21 = DSCALE*SQRT( DSUM )
*
*     Decide which method to use.
*       Weak stability test:
*          F-norm(S21) <= O(EPS * F-norm((S, T)))
*
      IF( BQRA21.LE.BRQA21 .AND. BQRA21.LE.THRESH ) THEN
         CALL SLACPY( 'F', M, M, SCPY, LDST, S, LDST )
         CALL SLACPY( 'F', M, M, TCPY, LDST, T, LDST )
         CALL SLACPY( 'F', M, M, IRCOP, LDST, IR, LDST )
         CALL SLACPY( 'F', M, M, LICOP, LDST, LI, LDST )
      ELSE IF( BRQA21.GE.THRESH ) THEN
         GO TO 70
      END IF
*
*     Set lower triangle of B-part to zero
*
      CALL SLASET( 'Lower', M-1, M-1, ZERO, ZERO, T(2,1), LDST )
*
      IF( WANDS ) THEN
*
*        Strong stability test:
*           F-norm((A-QL*S*QR**T, B-QL*T*QR**T)) <= O(EPS*F-norm((A,B)))
*
         CALL SLACPY( 'Full', M, M, A( J1, J1 ), LDA, WORK( M*M+1 ),
     $                M )
         CALL SGEMM( 'N', 'N', M, M, M, ONE, LI, LDST, S, LDST, ZERO,
     $               WORK, M )
```

```
            CALL SGEMM( 'N', 'N', M, M, M, -ONE, WORK, M, IR, LDST, ONE,
     $                  WORK( M*M+1 ), M )
            DSCALE = ZERO
            DSUM = ONE
            CALL SLASSQ( M*M, WORK( M*M+1 ), 1, DSCALE, DSUM )
*
            CALL SLACPY( 'Full', M, M, B( J1, J1 ), LDB, WORK( M*M+1 ),
     $                   M )
            CALL SGEMM( 'N', 'N', M, M, M, ONE, LI, LDST, T, LDST, ZERO,
     $                  WORK, M )
            CALL SGEMM( 'N', 'N', M, M, M, -ONE, WORK, M, IR, LDST, ONE,
     $                  WORK( M*M+1 ), M )
            CALL SLASSQ( M*M, WORK( M*M+1 ), 1, DSCALE, DSUM )
            SS = DSCALE*SQRT( DSUM )
            STRONG = ( SS.LE.THRESH )
            IF( .NOT.STRONG )
     $         GO TO 70
*
         END IF
*
*        If the swap is accepted ("weakly" and "strongly"), apply the
*        transformations and set N1-by-N2 (2,1)-block to zero.
*
         CALL SLASET( 'Full', N1, N2, ZERO, ZERO, S(N2+1,1), LDST )
*
*        copy back M-by-M diagonal block starting at index J1 of (A, B)
*
         CALL SLACPY( 'F', M, M, S, LDST, A( J1, J1 ), LDA )
         CALL SLACPY( 'F', M, M, T, LDST, B( J1, J1 ), LDB )
         CALL SLASET( 'Full', LDST, LDST, ZERO, ZERO, T, LDST )
*
*        Standardize existing 2-by-2 blocks.
*
         CALL SLASET( 'Full', M, M, ZERO, ZERO, WORK, M )
         WORK( 1 ) = ONE
         T( 1, 1 ) = ONE
         IDUM = LWORK - M*M - 2
         IF( N2.GT.1 ) THEN
            CALL SLAGV2( A( J1, J1 ), LDA, B( J1, J1 ), LDB, AR, AI, BE,
     $                   WORK( 1 ), WORK( 2 ), T( 1, 1 ), T( 2, 1 ) )
            WORK( M+1 ) = -WORK( 2 )
            WORK( M+2 ) = WORK( 1 )
            T( N2, N2 ) = T( 1, 1 )
            T( 1, 2 ) = -T( 2, 1 )
         END IF
         WORK( M*M ) = ONE
         T( M, M ) = ONE
*
         IF( N1.GT.1 ) THEN
            CALL SLAGV2( A( J1+N2, J1+N2 ), LDA, B( J1+N2, J1+N2 ), LDB,
```

```
     $                  TAUR, TAUL, WORK( M*M+1 ), WORK( N2*M+N2+1 ),
     $                  WORK( N2*M+N2+2 ), T( N2+1, N2+1 ),
     $                  T( M, M-1 ) )
            WORK( M*M ) = WORK( N2*M+N2+1 )
            WORK( M*M-1 ) = -WORK( N2*M+N2+2 )
            T( M, M ) = T( N2+1, N2+1 )
            T( M-1, M ) = -T( M, M-1 )
         END IF
         CALL SGEMM( 'T', 'N', N2, N1, N2, ONE, WORK, M, A( J1, J1+N2 ),
     $               LDA, ZERO, WORK( M*M+1 ), N2 )
         CALL SLACPY( 'Full', N2, N1, WORK( M*M+1 ), N2, A( J1, J1+N2 ),
     $                LDA )
         CALL SGEMM( 'T', 'N', N2, N1, N2, ONE, WORK, M, B( J1, J1+N2 ),
     $               LDB, ZERO, WORK( M*M+1 ), N2 )
         CALL SLACPY( 'Full', N2, N1, WORK( M*M+1 ), N2, B( J1, J1+N2 ),
     $                LDB )
         CALL SGEMM( 'N', 'N', M, M, M, ONE, LI, LDST, WORK, M, ZERO,
     $               WORK( M*M+1 ), M )
         CALL SLACPY( 'Full', M, M, WORK( M*M+1 ), M, LI, LDST )
         CALL SGEMM( 'N', 'N', N2, N1, N1, ONE, A( J1, J1+N2 ), LDA,
     $               T( N2+1, N2+1 ), LDST, ZERO, WORK, N2 )
         CALL SLACPY( 'Full', N2, N1, WORK, N2, A( J1, J1+N2 ), LDA )
         CALL SGEMM( 'N', 'N', N2, N1, N1, ONE, B( J1, J1+N2 ), LDB,
     $               T( N2+1, N2+1 ), LDST, ZERO, WORK, N2 )
         CALL SLACPY( 'Full', N2, N1, WORK, N2, B( J1, J1+N2 ), LDB )
         CALL SGEMM( 'T', 'N', M, M, M, ONE, IR, LDST, T, LDST, ZERO,
     $               WORK, M )
         CALL SLACPY( 'Full', M, M, WORK, M, IR, LDST )
*
*        Accumulate transformations into Q and Z if requested.
*
         IF( WANTQ ) THEN
            CALL SGEMM( 'N', 'N', N, M, M, ONE, Q( 1, J1 ), LDQ, LI,
     $                  LDST, ZERO, WORK, N )
            CALL SLACPY( 'Full', N, M, WORK, N, Q( 1, J1 ), LDQ )
*
         END IF
*
         IF( WANTZ ) THEN
            CALL SGEMM( 'N', 'N', N, M, M, ONE, Z( 1, J1 ), LDZ, IR,
     $                  LDST, ZERO, WORK, N )
            CALL SLACPY( 'Full', N, M, WORK, N, Z( 1, J1 ), LDZ )
*
         END IF
*
*        Update (A(J1:J1+M-1, M+J1:N), B(J1:J1+M-1, M+J1:N)) and
*                (A(1:J1-1, J1:J1+M), B(1:J1-1, J1:J1+M)).
*
         I = J1 + M
         IF( I.LE.N ) THEN
```

```
            CALL SGEMM( 'T', 'N', M, N-I+1, M, ONE, LI, LDST,
     $                  A( J1, I ), LDA, ZERO, WORK, M )
            CALL SLACPY( 'Full', M, N-I+1, WORK, M, A( J1, I ), LDA )
            CALL SGEMM( 'T', 'N', M, N-I+1, M, ONE, LI, LDST,
     $                  B( J1, I ), LDB, ZERO, WORK, M )
            CALL SLACPY( 'Full', M, N-I+1, WORK, M, B( J1, I ), LDB )
         END IF
         I = J1 - 1
         IF( I.GT.0 ) THEN
            CALL SGEMM( 'N', 'N', I, M, M, ONE, A( 1, J1 ), LDA, IR,
     $                  LDST, ZERO, WORK, I )
            CALL SLACPY( 'Full', I, M, WORK, I, A( 1, J1 ), LDA )
            CALL SGEMM( 'N', 'N', I, M, M, ONE, B( 1, J1 ), LDB, IR,
     $                  LDST, ZERO, WORK, I )
            CALL SLACPY( 'Full', I, M, WORK, I, B( 1, J1 ), LDB )
         END IF
*
*        Exit with INFO = 0 if swap was successfully performed.
*
         RETURN
*
      END IF
*
*     Exit with INFO = 1 if swap was rejected.
*
   70 CONTINUE
*
      INFO = 1
      RETURN
*
*     End of STGEX2
*
      END
```

———-

37.5.4 lisp code

— stgex2.lisp —

```
(let* ((zero 0.0) (one 1.0) (twenty 20.0) (ldst 4) (wands f2cl-lib:%true%))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 20.0 20.0) twenty)
           (type (f2cl-lib:integer4 4 4) ldst)
           (type f2cl-lib:logical wands)
           (ignorable zero one twenty ldst wands))
  (defun stgex2
```

```
     (wantq wantz n a lda b ldb$ q ldq z ldz j1 n1 n2 work lwork info)
(declare (type (array single-float (*)) work z q b a)
         (type (f2cl-lib:integer4) info lwork n2 n1 j1 ldz ldq ldb$ lda n)
         (type f2cl-lib:logical wantz wantq))
(f2cl-lib:with-multi-array-data
    ((a single-float a-%data% a-%offset%)
     (b single-float b-%data% b-%offset%)
     (q single-float q-%data% q-%offset%)
     (z single-float z-%data% z-%offset%)
     (work single-float work-%data% work-%offset%))
  (prog ((ai (make-array 2 :element-type 'single-float))
         (ar (make-array 2 :element-type 'single-float))
         (be (make-array 2 :element-type 'single-float))
         (ir
          (make-array (the fixnum (reduce #'* (list ldst ldst)))
                      :element-type 'single-float))
         (ircop
          (make-array (the fixnum (reduce #'* (list ldst ldst)))
                      :element-type 'single-float))
         (li
          (make-array (the fixnum (reduce #'* (list ldst ldst)))
                      :element-type 'single-float))
         (licop
          (make-array (the fixnum (reduce #'* (list ldst ldst)))
                      :element-type 'single-float))
         (s
          (make-array (the fixnum (reduce #'* (list ldst ldst)))
                      :element-type 'single-float))
         (scpy
          (make-array (the fixnum (reduce #'* (list ldst ldst)))
                      :element-type 'single-float))
         (taul (make-array ldst :element-type 'single-float))
         (taur (make-array ldst :element-type 'single-float))
         (tcpy
          (make-array (the fixnum (reduce #'* (list ldst ldst)))
                      :element-type 'single-float))
         (iwork (make-array ldst :element-type 'f2cl-lib:integer4))
         (bqra21 0.0) (brqa21 0.0) (ddum 0.0) (dnorm 0.0) (dscale 0.0)
         (dsum 0.0) (eps 0.0) (f 0.0) (g 0.0) (sa 0.0) (sb 0.0) (scale 0.0)
         (smlnum 0.0) (ss 0.0) (thresh 0.0) (ws 0.0) (i 0) (idum 0)
         (linfo 0) (m 0) (strong nil) (weak nil)
         (t$
          (make-array (the fixnum (reduce #'* (list ldst ldst)))
                      :element-type 'single-float)))
    (declare (type (array single-float (2)) ai ar be)
             (type (array single-float (*)) t$ ir ircop li licop s scpy
              taul taur tcpy)
             (type (array f2cl-lib:integer4 (*)) iwork)
             (type (single-float) bqra21 brqa21 ddum dnorm dscale dsum eps
              f g sa sb scale smlnum ss thresh ws)
```

```
         (type (f2cl-lib:integer4) i idum linfo m)
         (type f2cl-lib:logical strong weak))
(setf info 0)
(if (or (<= n 1) (<= n1 0) (<= n2 0))
    (go end_label))
(if (or (> n1 n) (> (f2cl-lib:int-add j1 n1) n))
    (go end_label))
(setf m (f2cl-lib:int-add n1 n2))
(cond
 ((< lwork
     (max (the f2cl-lib:integer4 (f2cl-lib:int-mul n m))
          (the f2cl-lib:integer4 (f2cl-lib:int-mul m m 2))))
  (setf info -16)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce
           (the f2cl-lib:integer4
                (max (the f2cl-lib:integer4 (f2cl-lib:int-mul n m))
                     (the f2cl-lib:integer4 (f2cl-lib:int-mul m m 2))))
           'single-float))
  (go end_label)))
(setf weak f2cl-lib:%false%)
(setf strong f2cl-lib:%false%)
(slaset "Full" ldst ldst zero zero li ldst)
(slaset "Full" ldst ldst zero zero ir ldst)
(slacpy "Full" m m
 (f2cl-lib:array-slice a-%data% single-float (j1 j1) ((1 lda) (1 *))
                       a-%offset%)
 lda s ldst)
(slacpy "Full" m m
 (f2cl-lib:array-slice b-%data% single-float (j1 j1) ((1 ldb$) (1 *))
                       b-%offset%)
 ldb$ t$ ldst)
(setf eps (slamch "P"))
(setf smlnum (/ (slamch "S") eps))
(setf dscale zero)
(setf dsum one)
(slacpy "Full" m m s ldst work m)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (slassq (f2cl-lib:int-mul m m) work 1 dscale dsum)
  (declare (ignore var-0 var-1 var-2))
  (setf dscale var-3)
  (setf dsum var-4))
(slacpy "Full" m m t$ ldst work m)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (slassq (f2cl-lib:int-mul m m) work 1 dscale dsum)
  (declare (ignore var-0 var-1 var-2))
  (setf dscale var-3)
  (setf dsum var-4))
(setf dnorm (* dscale (f2cl-lib:fsqrt dsum)))
(setf thresh (max (* twenty eps dnorm) smlnum))
```

```
(cond
 ((= m 2)
  (setf f
          (-
           (* (f2cl-lib:fref s (2 2) ((1 ldst) (1 ldst)))
              (f2cl-lib:fref t$ (1 1) ((1 ldst) (1 ldst))))
           (* (f2cl-lib:fref t$ (2 2) ((1 ldst) (1 ldst)))
              (f2cl-lib:fref s (1 1) ((1 ldst) (1 ldst))))))
  (setf g
          (-
           (* (f2cl-lib:fref s (2 2) ((1 ldst) (1 ldst)))
              (f2cl-lib:fref t$ (1 2) ((1 ldst) (1 ldst))))
           (* (f2cl-lib:fref t$ (2 2) ((1 ldst) (1 ldst)))
              (f2cl-lib:fref s (1 2) ((1 ldst) (1 ldst))))))
  (setf sb (abs (f2cl-lib:fref t$ (2 2) ((1 ldst) (1 ldst)))))
  (setf sa (abs (f2cl-lib:fref s (2 2) ((1 ldst) (1 ldst)))))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartg f g (f2cl-lib:fref ir (1 2) ((1 ldst) (1 ldst)))
       (f2cl-lib:fref ir (1 1) ((1 ldst) (1 ldst))) ddum)
    (declare (ignore var-0 var-1))
    (setf (f2cl-lib:fref ir (1 2) ((1 ldst) (1 ldst))) var-2)
    (setf (f2cl-lib:fref ir (1 1) ((1 ldst) (1 ldst))) var-3)
    (setf ddum var-4))
  (setf (f2cl-lib:fref ir (2 1) ((1 ldst) (1 ldst)))
          (- (f2cl-lib:fref ir (1 2) ((1 ldst) (1 ldst)))))
  (setf (f2cl-lib:fref ir (2 2) ((1 ldst) (1 ldst)))
          (f2cl-lib:fref ir (1 1) ((1 ldst) (1 ldst))))
  (srot 2
   (f2cl-lib:array-slice s single-float (1 1) ((1 ldst) (1 ldst))) 1
   (f2cl-lib:array-slice s single-float (1 2) ((1 ldst) (1 ldst))) 1
   (f2cl-lib:array-slice ir single-float (1 1) ((1 ldst) (1 ldst)))
   (f2cl-lib:array-slice ir single-float (2 1) ((1 ldst) (1 ldst))))
  (srot 2
   (f2cl-lib:array-slice t$ single-float (1 1) ((1 ldst) (1 ldst))) 1
   (f2cl-lib:array-slice t$ single-float (1 2) ((1 ldst) (1 ldst))) 1
   (f2cl-lib:array-slice ir single-float (1 1) ((1 ldst) (1 ldst)))
   (f2cl-lib:array-slice ir single-float (2 1) ((1 ldst) (1 ldst))))
  (cond
   ((>= sa sb)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (slartg (f2cl-lib:fref s (1 1) ((1 ldst) (1 ldst)))
         (f2cl-lib:fref s (2 1) ((1 ldst) (1 ldst)))
         (f2cl-lib:fref li (1 1) ((1 ldst) (1 ldst)))
         (f2cl-lib:fref li (2 1) ((1 ldst) (1 ldst))) ddum)
      (declare (ignore var-0 var-1))
      (setf (f2cl-lib:fref li (1 1) ((1 ldst) (1 ldst))) var-2)
      (setf (f2cl-lib:fref li (2 1) ((1 ldst) (1 ldst))) var-3)
      (setf ddum var-4)))
   (t
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
      (slartg (f2cl-lib:fref t$ (1 1) ((1 ldst) (1 ldst)))
       (f2cl-lib:fref t$ (2 1) ((1 ldst) (1 ldst)))
       (f2cl-lib:fref li (1 1) ((1 ldst) (1 ldst)))
       (f2cl-lib:fref li (2 1) ((1 ldst) (1 ldst))) ddum)
    (declare (ignore var-0 var-1))
    (setf (f2cl-lib:fref li (1 1) ((1 ldst) (1 ldst))) var-2)
    (setf (f2cl-lib:fref li (2 1) ((1 ldst) (1 ldst))) var-3)
    (setf ddum var-4))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (srot 2
     (f2cl-lib:array-slice s single-float (1 1) ((1 ldst) (1 ldst)))
     ldst
     (f2cl-lib:array-slice s single-float (2 1) ((1 ldst) (1 ldst)))
     ldst
     (f2cl-lib:array-slice li single-float (1 1) ((1 ldst) (1 ldst)))
     (f2cl-lib:array-slice li single-float (2 1)
                           ((1 ldst) (1 ldst))))
  (declare (ignore var-0 var-1 var-3 var-5 var-6))
  (when var-2 (setf ldst var-2))
  (when var-4 (setf ldst var-4)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (srot 2
     (f2cl-lib:array-slice t$ single-float (1 1) ((1 ldst) (1 ldst)))
     ldst
     (f2cl-lib:array-slice t$ single-float (2 1) ((1 ldst) (1 ldst)))
     ldst
     (f2cl-lib:array-slice li single-float (1 1) ((1 ldst) (1 ldst)))
     (f2cl-lib:array-slice li single-float (2 1)
                           ((1 ldst) (1 ldst))))
  (declare (ignore var-0 var-1 var-3 var-5 var-6))
  (when var-2 (setf ldst var-2))
  (when var-4 (setf ldst var-4)))
(setf (f2cl-lib:fref li (2 2) ((1 ldst) (1 ldst)))
        (f2cl-lib:fref li (1 1) ((1 ldst) (1 ldst))))
(setf (f2cl-lib:fref li (1 2) ((1 ldst) (1 ldst)))
        (- (f2cl-lib:fref li (2 1) ((1 ldst) (1 ldst)))))
(setf ws
        (+ (abs (f2cl-lib:fref s (2 1) ((1 ldst) (1 ldst))))
           (abs (f2cl-lib:fref t$ (2 1) ((1 ldst) (1 ldst))))))
(setf weak (<= ws thresh))
(if (not weak)
    (go label70))
(cond
 (wands
  (slacpy "Full" m m
   (f2cl-lib:array-slice a-%data% single-float (j1 j1)
                         ((1 lda) (1 *)) a-%offset%)
   lda
   (f2cl-lib:array-slice work-%data% single-float
                         ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
```

```
                        work-%offset%)
 m)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (sgemm "N" "N" m m m one li ldst s ldst zero work m)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldst var-7))
  (when var-9 (setf ldst var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf m var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (sgemm "N" "T" m m m (- one) work m ir ldst one
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
     m)
  (declare (ignore var-0 var-1 var-5 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-7 (setf m var-7))
  (when var-9 (setf ldst var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf m var-12)))
(setf dscale zero) (setf dsum one)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (slassq (f2cl-lib:int-mul m m)
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
     1 dscale dsum)
  (declare (ignore var-0 var-1 var-2))
  (setf dscale var-3)
  (setf dsum var-4))
(slacpy "Full" m m
 (f2cl-lib:array-slice b-%data% single-float (j1 j1)
                       ((1 ldb$) (1 *)) b-%offset%)
 ldb$
 (f2cl-lib:array-slice work-%data% single-float
                       ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                       work-%offset%)
 m)
(multiple-value-bind
```

```
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (sgemm "N" "N" m m m one li ldst t$ ldst zero work m)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf m var-3))
      (when var-4 (setf m var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf ldst var-7))
      (when var-9 (setf ldst var-9))
      (when var-10 (setf zero var-10))
      (when var-12 (setf m var-12)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (sgemm "N" "T" m m m (- one) work m ir ldst one
         (f2cl-lib:array-slice work-%data% single-float
                               ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                               work-%offset%)
         m)
      (declare (ignore var-0 var-1 var-5 var-6 var-8 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf m var-3))
      (when var-4 (setf m var-4))
      (when var-7 (setf m var-7))
      (when var-9 (setf ldst var-9))
      (when var-10 (setf one var-10))
      (when var-12 (setf m var-12)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (slassq (f2cl-lib:int-mul m m)
         (f2cl-lib:array-slice work-%data% single-float
                               ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                               work-%offset%)
         1 dscale dsum)
      (declare (ignore var-0 var-1 var-2))
      (setf dscale var-3)
      (setf dsum var-4))
    (setf ss (* dscale (f2cl-lib:fsqrt dsum)))
    (setf strong (<= ss thresh))
    (if (not strong)
        (go label70))))
  (srot (f2cl-lib:int-add j1 1)
   (f2cl-lib:array-slice a-%data% single-float (1 j1) ((1 lda) (1 *))
                         a-%offset%)
   1
   (f2cl-lib:array-slice a-%data% single-float
                         (1 (f2cl-lib:int-add j1 1)) ((1 lda) (1 *))
                         a-%offset%)
   1 (f2cl-lib:array-slice ir single-float (1 1) ((1 ldst) (1 ldst)))
   (f2cl-lib:array-slice ir single-float (2 1) ((1 ldst) (1 ldst))))
```

```
(srot (f2cl-lib:int-add j1 1)
 (f2cl-lib:array-slice b-%data% single-float (1 j1) ((1 ldb$) (1 *))
                       b-%offset%)
 1
 (f2cl-lib:array-slice b-%data% single-float
                       (1 (f2cl-lib:int-add j1 1)) ((1 ldb$) (1 *))
                       b-%offset%)
 1 (f2cl-lib:array-slice ir single-float (1 1) ((1 ldst) (1 ldst)))
 (f2cl-lib:array-slice ir single-float (2 1) ((1 ldst) (1 ldst))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (srot (f2cl-lib:int-add (f2cl-lib:int-sub n j1) 1)
     (f2cl-lib:array-slice a-%data% single-float (j1 j1)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice a-%data% single-float ((+ j1 1) j1)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice li single-float (1 1) ((1 ldst) (1 ldst)))
     (f2cl-lib:array-slice li single-float (2 1)
                           ((1 ldst) (1 ldst))))
  (declare (ignore var-0 var-1 var-3 var-5 var-6))
  (when var-2 (setf lda var-2))
  (when var-4 (setf lda var-4)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (srot (f2cl-lib:int-add (f2cl-lib:int-sub n j1) 1)
     (f2cl-lib:array-slice b-%data% single-float (j1 j1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice b-%data% single-float ((+ j1 1) j1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice li single-float (1 1) ((1 ldst) (1 ldst)))
     (f2cl-lib:array-slice li single-float (2 1)
                           ((1 ldst) (1 ldst))))
  (declare (ignore var-0 var-1 var-3 var-5 var-6))
  (when var-2 (setf ldb$ var-2))
  (when var-4 (setf ldb$ var-4)))
(setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add j1 1) j1)
                     ((1 lda) (1 *)) a-%offset%)
        zero)
(setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add j1 1) j1)
                     ((1 ldb$) (1 *)) b-%offset%)
        zero)
(if wantz
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (srot n
         (f2cl-lib:array-slice z-%data% single-float (1 j1)
                               ((1 ldz) (1 *)) z-%offset%)
         1
         (f2cl-lib:array-slice z-%data% single-float
```

```
                                  (1 (f2cl-lib:int-add j1 1))
                                  ((1 ldz) (1 *)) z-%offset%)
            1
            (f2cl-lib:array-slice ir single-float (1 1)
                                  ((1 ldst) (1 ldst)))
            (f2cl-lib:array-slice ir single-float (2 1)
                                  ((1 ldst) (1 ldst))))
          (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
          (when var-0 (setf n var-0))))
   (if wantq
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
           (srot n
            (f2cl-lib:array-slice q-%data% single-float (1 j1)
                                  ((1 ldq) (1 *)) q-%offset%)
            1
            (f2cl-lib:array-slice q-%data% single-float
                                  (1 (f2cl-lib:int-add j1 1))
                                  ((1 ldq) (1 *)) q-%offset%)
            1
            (f2cl-lib:array-slice li single-float (1 1)
                                  ((1 ldst) (1 ldst)))
            (f2cl-lib:array-slice li single-float (2 1)
                                  ((1 ldst) (1 ldst))))
          (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
          (when var-0 (setf n var-0))))
   (go end_label))
  (t
   (slacpy "Full" n1 n2
    (f2cl-lib:array-slice t$ single-float (1 (f2cl-lib:int-add n1 1))
                          ((1 ldst) (1 ldst)))
    ldst li ldst)
   (slacpy "Full" n1 n2
    (f2cl-lib:array-slice s single-float (1 (f2cl-lib:int-add n1 1))
                          ((1 ldst) (1 ldst)))
    ldst
    (f2cl-lib:array-slice ir single-float
                          ((+ n2 1) (f2cl-lib:int-add n1 1))
                          ((1 ldst) (1 ldst)))
    ldst)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18
        var-19 var-20 var-21)
       (stgsy2 "N" 0 n1 n2 s ldst
        (f2cl-lib:array-slice s single-float
                              ((+ n1 1) (f2cl-lib:int-add n1 1))
                              ((1 ldst) (1 ldst)))
        ldst
        (f2cl-lib:array-slice ir single-float
                              ((+ n2 1) (f2cl-lib:int-add n1 1))
```

```
                              ((1 ldst) (1 ldst)))
      ldst t$ ldst
      (f2cl-lib:array-slice t$ single-float
                              ((+ n1 1) (f2cl-lib:int-add n1 1))
                              ((1 ldst) (1 ldst)))
      ldst li ldst scale dsum dscale iwork idum linfo)
  (declare
   (ignore var-0 var-1 var-4 var-6 var-8 var-10 var-12 var-14
    var-19))
  (when var-2 (setf n1 var-2))
  (when var-3 (setf n2 var-3))
  (when var-5 (setf ldst var-5))
  (when var-7 (setf ldst var-7))
  (when var-9 (setf ldst var-9))
  (when var-11 (setf ldst var-11))
  (when var-13 (setf ldst var-13))
  (when var-15 (setf ldst var-15))
  (when var-16 (setf scale var-16))
  (when var-17 (setf dsum var-17))
  (when var-18 (setf dscale var-18))
  (when var-20 (setf idum var-20))
  (when var-21 (setf linfo var-21)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n2) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal n1 (- one)
         (f2cl-lib:array-slice li single-float (1 i)
                               ((1 ldst) (1 ldst)))
         1)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf n1 var-0)))
    (setf (f2cl-lib:fref li ((f2cl-lib:int-add n1 i) i)
                          ((1 ldst) (1 ldst)))
            scale)
   label10))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (sgeqr2 m n2 li ldst taul work linfo)
  (declare (ignore var-0 var-1 var-2 var-4 var-5))
  (setf ldst var-3)
  (setf linfo var-6))
(if (/= linfo 0)
    (go label70))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (sorg2r m m n2 li ldst taul work linfo)
  (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6))
  (setf ldst var-4)
  (setf linfo var-7))
(if (/= linfo 0)
```

```
    (go label70))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n1) nil)
  (tagbody
    (setf (f2cl-lib:fref ir ((f2cl-lib:int-add n2 i) i)
                         ((1 ldst) (1 ldst)))
          scale)
   label20))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (sgerq2 n1 m
     (f2cl-lib:array-slice ir single-float ((+ n2 1) 1)
                           ((1 ldst) (1 ldst)))
     ldst taur work linfo)
  (declare (ignore var-0 var-1 var-2 var-4 var-5))
  (setf ldst var-3)
  (setf linfo var-6))
(if (/= linfo 0)
    (go label70))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (sorgr2 m m n1 ir ldst taur work linfo)
  (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6))
  (setf ldst var-4)
  (setf linfo var-7))
(if (/= linfo 0)
    (go label70))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (sgemm "T" "N" m m m one li ldst s ldst zero work m)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldst var-7))
  (when var-9 (setf ldst var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf m var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (sgemm "N" "T" m m m one work m ir ldst zero s ldst)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf m var-7))
  (when var-9 (setf ldst var-9))
```

```
  (when var-10 (setf zero var-10))
  (when var-12 (setf ldst var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (sgemm "T" "N" m m m one li ldst t$ ldst zero work m)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldst var-7))
  (when var-9 (setf ldst var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf m var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (sgemm "N" "T" m m m one work m ir ldst zero t$ ldst)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf m var-7))
  (when var-9 (setf ldst var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf ldst var-12)))
(slacpy "F" m m s ldst scpy ldst) (slacpy "F" m m t$ ldst tcpy ldst)
(slacpy "F" m m ir ldst ircop ldst)
(slacpy "F" m m li ldst licop ldst)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (sgerq2 m m t$ ldst taur work linfo)
  (declare (ignore var-0 var-1 var-2 var-4 var-5))
  (setf ldst var-3)
  (setf linfo var-6))
(if (/= linfo 0)
    (go label70))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (sormr2 "R" "T" m m m t$ ldst taur s ldst work linfo)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-8 var-10))
  (setf ldst var-6)
  (setf ldst var-9)
  (setf linfo var-11))
(if (/= linfo 0)
    (go label70))
(multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (sormr2 "L" "N" m m m t$ ldst taur ir ldst work linfo)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-8 var-10))
  (setf ldst var-6)
  (setf ldst var-9)
  (setf linfo var-11))
(if (/= linfo 0)
    (go label70))
(setf dscale zero) (setf dsum one)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n2) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (slassq n1
         (f2cl-lib:array-slice s single-float ((+ n2 1) i)
                               ((1 ldst) (1 ldst)))
         1 dscale dsum)
      (declare (ignore var-0 var-1 var-2))
      (setf dscale var-3)
      (setf dsum var-4))
   label30))
(setf brqa21 (* dscale (f2cl-lib:fsqrt dsum)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (sgeqr2 m m tcpy ldst taul work linfo)
  (declare (ignore var-0 var-1 var-2 var-4 var-5))
  (setf ldst var-3)
  (setf linfo var-6))
(if (/= linfo 0)
    (go label70))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (sorm2r "L" "T" m m m tcpy ldst taul scpy ldst work info)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
    var-10))
  (setf ldst var-9)
  (setf info var-11))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (sorm2r "R" "N" m m m tcpy ldst taul licop ldst work info)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
    var-10))
  (setf ldst var-9)
  (setf info var-11))
(if (/= linfo 0)
```

```
    (go label70))
(setf dscale zero) (setf dsum one)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n2) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (slassq n1
         (f2cl-lib:array-slice scpy single-float ((+ n2 1) i)
                               ((1 ldst) (1 ldst)))
         1 dscale dsum)
      (declare (ignore var-0 var-1 var-2))
      (setf dscale var-3)
      (setf dsum var-4))
   label40))
(setf bqra21 (* dscale (f2cl-lib:fsqrt dsum)))
(cond
 ((and (<= bqra21 brqa21) (<= bqra21 thresh))
  (slacpy "F" m m scpy ldst s ldst)
  (slacpy "F" m m tcpy ldst t$ ldst)
  (slacpy "F" m m ircop ldst ir ldst)
  (slacpy "F" m m licop ldst li ldst))
 ((>= brqa21 thresh) (go label70)))
(slaset "Lower" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1) zero
 zero
 (f2cl-lib:array-slice t$ single-float (2 1) ((1 ldst) (1 ldst)))
 ldst)
(cond
 (wands
  (slacpy "Full" m m
   (f2cl-lib:array-slice a-%data% single-float (j1 j1)
                         ((1 lda) (1 *)) a-%offset%)
   lda
   (f2cl-lib:array-slice work-%data% single-float
                         ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                         work-%offset%)
   m)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sgemm "N" "N" m m m one li ldst s ldst zero work m)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf m var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldst var-7))
    (when var-9 (setf ldst var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf m var-12)))
  (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (sgemm "N" "N" m m m (- one) work m ir ldst one
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
     m)
  (declare (ignore var-0 var-1 var-5 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-7 (setf m var-7))
  (when var-9 (setf ldst var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf m var-12)))
(setf dscale zero) (setf dsum one)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (slassq (f2cl-lib:int-mul m m)
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
     1 dscale dsum)
  (declare (ignore var-0 var-1 var-2))
  (setf dscale var-3)
  (setf dsum var-4))
(slacpy "Full" m m
 (f2cl-lib:array-slice b-%data% single-float (j1 j1)
                       ((1 ldb$) (1 *)) b-%offset%)
 ldb$
 (f2cl-lib:array-slice work-%data% single-float
                       ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                       work-%offset%)
 m)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (sgemm "N" "N" m m m one li ldst t$ ldst zero work m)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldst var-7))
  (when var-9 (setf ldst var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf m var-12)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (sgemm "N" "N" m m m (- one) work m ir ldst one
```

```
        (f2cl-lib:array-slice work-%data% single-float
                              ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                              work-%offset%)
        m)
      (declare (ignore var-0 var-1 var-5 var-6 var-8 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf m var-3))
      (when var-4 (setf m var-4))
      (when var-7 (setf m var-7))
      (when var-9 (setf ldst var-9))
      (when var-10 (setf one var-10))
      (when var-12 (setf m var-12)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (slassq (f2cl-lib:int-mul m m)
         (f2cl-lib:array-slice work-%data% single-float
                              ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                              work-%offset%)
         1 dscale dsum)
      (declare (ignore var-0 var-1 var-2))
      (setf dscale var-3)
      (setf dsum var-4))
    (setf ss (* dscale (f2cl-lib:fsqrt dsum)))
    (setf strong (<= ss thresh))
    (if (not strong)
        (go label70))))
  (slaset "Full" n1 n2 zero zero
   (f2cl-lib:array-slice s single-float ((+ n2 1) 1)
                         ((1 ldst) (1 ldst)))
   ldst)
  (slacpy "F" m m s ldst
   (f2cl-lib:array-slice a-%data% single-float (j1 j1) ((1 lda) (1 *))
                         a-%offset%)
   lda)
  (slacpy "F" m m t$ ldst
   (f2cl-lib:array-slice b-%data% single-float (j1 j1) ((1 ldb$) (1 *))
                         b-%offset%)
   ldb$)
  (slaset "Full" ldst ldst zero zero t$ ldst)
  (slaset "Full" m m zero zero work m)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) one)
  (setf (f2cl-lib:fref t$ (1 1) ((1 ldst) (1 ldst))) one)
  (setf idum (f2cl-lib:int-sub lwork (f2cl-lib:int-mul m m) 2))
  (cond
   ((> n2 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (slagv2
         (f2cl-lib:array-slice a-%data% single-float (j1 j1)
                              ((1 lda) (1 *)) a-%offset%)
```

```
        lda
        (f2cl-lib:array-slice b-%data% single-float (j1 j1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$ ar ai be
        (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (f2cl-lib:fref work-%data% (2) ((1 *)) work-%offset%)
        (f2cl-lib:fref t$ (1 1) ((1 ldst) (1 ldst)))
        (f2cl-lib:fref t$ (2 1) ((1 ldst) (1 ldst))))
    (declare (ignore var-0 var-2 var-4 var-5 var-6))
    (setf lda var-1)
    (setf ldb$ var-3)
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            var-7)
    (setf (f2cl-lib:fref work-%data% (2) ((1 *)) work-%offset%)
            var-8)
    (setf (f2cl-lib:fref t$ (1 1) ((1 ldst) (1 ldst))) var-9)
    (setf (f2cl-lib:fref t$ (2 1) ((1 ldst) (1 ldst))) var-10))
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add m 1)) ((1 *))
                       work-%offset%)
          (- (f2cl-lib:fref work-%data% (2) ((1 *)) work-%offset%)))
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add m 2)) ((1 *))
                       work-%offset%)
          (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%))
  (setf (f2cl-lib:fref t$ (n2 n2) ((1 ldst) (1 ldst)))
          (f2cl-lib:fref t$ (1 1) ((1 ldst) (1 ldst))))
  (setf (f2cl-lib:fref t$ (1 2) ((1 ldst) (1 ldst)))
          (- (f2cl-lib:fref t$ (2 1) ((1 ldst) (1 ldst)))))))
(setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-mul m m)) ((1 *))
                     work-%offset%)
        one)
(setf (f2cl-lib:fref t$ (m m) ((1 ldst) (1 ldst))) one)
(cond
 ((> n1 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (slagv2
       (f2cl-lib:array-slice a-%data% single-float
                             ((+ j1 n2) (f2cl-lib:int-add j1 n2))
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice b-%data% single-float
                             ((+ j1 n2) (f2cl-lib:int-add j1 n2))
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$ taur taul
       (f2cl-lib:array-slice work-%data% single-float
                             ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                             work-%offset%)
       (f2cl-lib:fref work-%data%
                      ((f2cl-lib:int-add (f2cl-lib:int-mul n2 m) n2
```

```
                                        1))
                        ((1 *)) work-%offset%)
        (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add (f2cl-lib:int-mul n2 m) n2
                                        2))
                        ((1 *)) work-%offset%)
        (f2cl-lib:fref t$
                        ((f2cl-lib:int-add n2 1)
                         (f2cl-lib:int-add n2 1))
                        ((1 ldst) (1 ldst)))
        (f2cl-lib:fref t$ (m (f2cl-lib:int-sub m 1))
                        ((1 ldst) (1 ldst))))
    (declare (ignore var-0 var-2 var-4 var-5 var-6))
    (setf lda var-1)
    (setf ldb$ var-3)
    (setf (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add (f2cl-lib:int-mul n2 m)
                                             n2 1))
                          ((1 *)) work-%offset%)
            var-7)
    (setf (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add (f2cl-lib:int-mul n2 m)
                                             n2 2))
                          ((1 *)) work-%offset%)
            var-8)
    (setf (f2cl-lib:fref t$
                          ((f2cl-lib:int-add n2 1)
                           (f2cl-lib:int-add n2 1))
                          ((1 ldst) (1 ldst)))
            var-9)
    (setf (f2cl-lib:fref t$ (m (f2cl-lib:int-sub m 1))
                          ((1 ldst) (1 ldst)))
            var-10))
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-mul m m)) ((1 *))
                        work-%offset%)
          (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add (f2cl-lib:int-mul n2 m)
                                             n2 1))
                          ((1 *)) work-%offset%))
  (setf (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-sub (f2cl-lib:int-mul m m) 1))
                        ((1 *)) work-%offset%)
          (-
           (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add (f2cl-lib:int-mul n2 m)
                                              n2 2))
                           ((1 *)) work-%offset%)))
  (setf (f2cl-lib:fref t$ (m m) ((1 ldst) (1 ldst)))
          (f2cl-lib:fref t$
                          ((f2cl-lib:int-add n2 1)
```

```
                          (f2cl-lib:int-add n2 1))
                         ((1 ldst) (1 ldst))))
  (setf (f2cl-lib:fref t$ ((f2cl-lib:int-sub m 1) m)
                        ((1 ldst) (1 ldst)))
          (-
           (f2cl-lib:fref t$ (m (f2cl-lib:int-sub m 1))
                          ((1 ldst) (1 ldst)))))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (sgemm "T" "N" n2 n1 n2 one work m
     (f2cl-lib:array-slice a-%data% single-float
                           (j1 (f2cl-lib:int-add j1 n2))
                           ((1 lda) (1 *)) a-%offset%)
     lda zero
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
     n2)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf n2 var-2))
  (when var-3 (setf n1 var-3))
  (when var-4 (setf n2 var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf m var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf n2 var-12)))
(slacpy "Full" n2 n1
 (f2cl-lib:array-slice work-%data% single-float
                       ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                       work-%offset%)
 n2
 (f2cl-lib:array-slice a-%data% single-float
                       (j1 (f2cl-lib:int-add j1 n2)) ((1 lda) (1 *))
                       a-%offset%)
 lda)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (sgemm "T" "N" n2 n1 n2 one work m
     (f2cl-lib:array-slice b-%data% single-float
                           (j1 (f2cl-lib:int-add j1 n2))
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ zero
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
     n2)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
```

```
  (when var-2 (setf n2 var-2))
  (when var-3 (setf n1 var-3))
  (when var-4 (setf n2 var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf m var-7))
  (when var-9 (setf ldb$ var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf n2 var-12)))
(slacpy "Full" n2 n1
 (f2cl-lib:array-slice work-%data% single-float
                       ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                       work-%offset%)
 n2
 (f2cl-lib:array-slice b-%data% single-float
                       (j1 (f2cl-lib:int-add j1 n2)) ((1 ldb$) (1 *))
                       b-%offset%)
 ldb$)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (sgemm "N" "N" m m m one li ldst work m zero
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                           work-%offset%)
     m)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldst var-7))
  (when var-9 (setf m var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf m var-12)))
(slacpy "Full" m m
 (f2cl-lib:array-slice work-%data% single-float
                       ((+ (f2cl-lib:int-mul m m) 1)) ((1 *))
                       work-%offset%)
 m li ldst)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (sgemm "N" "N" n2 n1 n1 one
     (f2cl-lib:array-slice a-%data% single-float
                           (j1 (f2cl-lib:int-add j1 n2))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice t$ single-float
                           ((+ n2 1) (f2cl-lib:int-add n2 1))
                           ((1 ldst) (1 ldst)))
```

```
      ldst zero work n2)
   (declare (ignore var-0 var-1 var-6 var-8 var-11))
   (when var-2 (setf n2 var-2))
   (when var-3 (setf n1 var-3))
   (when var-4 (setf n1 var-4))
   (when var-5 (setf one var-5))
   (when var-7 (setf lda var-7))
   (when var-9 (setf ldst var-9))
   (when var-10 (setf zero var-10))
   (when var-12 (setf n2 var-12)))
 (slacpy "Full" n2 n1 work n2
  (f2cl-lib:array-slice a-%data% single-float
                        (j1 (f2cl-lib:int-add j1 n2)) ((1 lda) (1 *))
                        a-%offset%)
  lda)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12)
     (sgemm "N" "N" n2 n1 n1 one
      (f2cl-lib:array-slice b-%data% single-float
                            (j1 (f2cl-lib:int-add j1 n2))
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$
      (f2cl-lib:array-slice t$ single-float
                            ((+ n2 1) (f2cl-lib:int-add n2 1))
                            ((1 ldst) (1 ldst)))
      ldst zero work n2)
   (declare (ignore var-0 var-1 var-6 var-8 var-11))
   (when var-2 (setf n2 var-2))
   (when var-3 (setf n1 var-3))
   (when var-4 (setf n1 var-4))
   (when var-5 (setf one var-5))
   (when var-7 (setf ldb$ var-7))
   (when var-9 (setf ldst var-9))
   (when var-10 (setf zero var-10))
   (when var-12 (setf n2 var-12)))
 (slacpy "Full" n2 n1 work n2
  (f2cl-lib:array-slice b-%data% single-float
                        (j1 (f2cl-lib:int-add j1 n2)) ((1 ldb$) (1 *))
                        b-%offset%)
  ldb$)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12)
     (sgemm "T" "N" m m m one ir ldst t$ ldst zero work m)
   (declare (ignore var-0 var-1 var-6 var-8 var-11))
   (when var-2 (setf m var-2))
   (when var-3 (setf m var-3))
   (when var-4 (setf m var-4))
   (when var-5 (setf one var-5))
```

```
  (when var-7 (setf ldst var-7))
  (when var-9 (setf ldst var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf m var-12)))
(slacpy "Full" m m work m ir ldst)
(cond
 (wantq
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sgemm "N" "N" n m m one
       (f2cl-lib:array-slice q-%data% single-float (1 j1)
                             ((1 ldq) (1 *)) q-%offset%)
       ldq li ldst zero work n)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldq var-7))
    (when var-9 (setf ldst var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf n var-12)))
  (slacpy "Full" n m work n
   (f2cl-lib:array-slice q-%data% single-float (1 j1) ((1 ldq) (1 *))
                         q-%offset%)
   ldq)))
(cond
 (wantz
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sgemm "N" "N" n m m one
       (f2cl-lib:array-slice z-%data% single-float (1 j1)
                             ((1 ldz) (1 *)) z-%offset%)
       ldz ir ldst zero work n)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldz var-7))
    (when var-9 (setf ldst var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf n var-12)))
  (slacpy "Full" n m work n
   (f2cl-lib:array-slice z-%data% single-float (1 j1) ((1 ldz) (1 *))
                         z-%offset%)
   ldz)))
(setf i (f2cl-lib:int-add j1 m))
```

```
(cond
 ((<= i n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sgemm "T" "N" m (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) m
       one li ldst
       (f2cl-lib:array-slice a-%data% single-float (j1 i)
                             ((1 lda) (1 *)) a-%offset%)
       lda zero work m)
    (declare (ignore var-0 var-1 var-3 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldst var-7))
    (when var-9 (setf lda var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf m var-12)))
  (slacpy "Full" m (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) work m
   (f2cl-lib:array-slice a-%data% single-float (j1 i) ((1 lda) (1 *))
                         a-%offset%)
   lda)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sgemm "T" "N" m (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) m
       one li ldst
       (f2cl-lib:array-slice b-%data% single-float (j1 i)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$ zero work m)
    (declare (ignore var-0 var-1 var-3 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldst var-7))
    (when var-9 (setf ldb$ var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf m var-12)))
  (slacpy "Full" m (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) work m
   (f2cl-lib:array-slice b-%data% single-float (j1 i)
                         ((1 ldb$) (1 *)) b-%offset%)
   ldb$)))
(setf i (f2cl-lib:int-sub j1 1))
(cond
 ((> i 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sgemm "N" "N" i m m one
       (f2cl-lib:array-slice a-%data% single-float (1 j1)
```

```
                                    ((1 lda) (1 *)) a-%offset%)
                 lda ir ldst zero work i)
               (declare (ignore var-0 var-1 var-6 var-8 var-11))
               (when var-2 (setf i var-2))
               (when var-3 (setf m var-3))
               (when var-4 (setf m var-4))
               (when var-5 (setf one var-5))
               (when var-7 (setf lda var-7))
               (when var-9 (setf ldst var-9))
               (when var-10 (setf zero var-10))
               (when var-12 (setf i var-12)))
              (slacpy "Full" i m work i
               (f2cl-lib:array-slice a-%data% single-float (1 j1) ((1 lda) (1 *))
                                     a-%offset%)
               lda)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                   var-10 var-11 var-12)
                  (sgemm "N" "N" i m m one
                   (f2cl-lib:array-slice b-%data% single-float (1 j1)
                                    ((1 ldb$) (1 *)) b-%offset%)
                   ldb$ ir ldst zero work i)
               (declare (ignore var-0 var-1 var-6 var-8 var-11))
               (when var-2 (setf i var-2))
               (when var-3 (setf m var-3))
               (when var-4 (setf m var-4))
               (when var-5 (setf one var-5))
               (when var-7 (setf ldb$ var-7))
               (when var-9 (setf ldst var-9))
               (when var-10 (setf zero var-10))
               (when var-12 (setf i var-12)))
              (slacpy "Full" i m work i
               (f2cl-lib:array-slice b-%data% single-float (1 j1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)))
            (go end_label)))
        label70
         (setf info 1)
         (go end_label)
        end_label
         (return
          (values nil nil n nil lda nil ldb$ nil ldq nil ldz nil n1 n2 nil nil
                  info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::stgex2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
```

```
'(fortran-to-lisp::logical
  fortran-to-lisp::logical
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(nil nil fortran-to-lisp::n nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldq nil
  fortran-to-lisp::ldz nil
  fortran-to-lisp::n1
  fortran-to-lisp::n2 nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::slagv2
  fortran-to-lisp::sorm2r
  fortran-to-lisp::sormr2
  fortran-to-lisp::sorgr2
  fortran-to-lisp::sgerq2
  fortran-to-lisp::sorg2r
  fortran-to-lisp::sgeqr2
  fortran-to-lisp::slartg
  fortran-to-lisp::slassq
  fortran-to-lisp::slacpy
  fortran-to-lisp::slaset))))
```

Chapter 38

LAPACK - General Matrices, Auxilliary Routines, Complex

38.1 cgesc2

38.1.1 Axiom unit tests

— **cgesc2.input** —

```
)set break resume
)sys rm -f cgesc2.output
)spool cgesc2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

38.1.2 Axiom help page

— **cgesc2.help** —

```
==========================================================================
cgesc2 examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgesc2 (N, A, LDA, RHS, IPIV, JPIV, SCALE)
           CGESC2 solves a system of linear equations using the LU
           factorization with complete pivoting computed by sgetc2.

Function/Subroutine Documentation
   subroutine cgesc2 (integer N, complex, dimension( lda, * ) A, integer LDA,
       complex, dimension( * ) RHS, integer, dimension( * ) IPIV, integer,
       dimension( * ) JPIV, real SCALE)
       CGESC2 solves a system of linear equations using the LU factorization
       with complete pivoting computed by sgetc2.

Purpose:
   CGESC2 solves a system of linear equations

      A * X = scale* RHS

   with a general N-by-N matrix A using the LU factorization with
   complete pivoting computed by CGETC2.

Parameters:

 N
   N is INTEGER
   The number of columns of the matrix A.

 A
   A is COMPLEX array, dimension (LDA, N)
   On entry, the  LU part of the factorization of the n-by-n
   matrix A computed by CGETC2:  A = P * L * U * Q

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1, N).

 RHS
   RHS is COMPLEX array, dimension N.
   On entry, the right hand side vector b.
   On exit, the solution vector X.

 IPIV
   IPIV is INTEGER array, dimension (N).
   The pivot indices; for 1 <= i <= N, row i of the
   matrix has been interchanged with row IPIV(i).
```

```
 JPIV
   JPIV is INTEGER array, dimension (N).
   The pivot indices; for 1 <= j <= N, column j of the
   matrix has been interchanged with column JPIV(j).

 SCALE
   SCALE is REAL
   On exit, SCALE contains the scale factor. SCALE is chosen
   0 <= SCALE <= 1 to prevent owerflow in the solution.
```

———-

38.1.3 fortran code

— **cgesc2.f** —

```
*  =====================================================================
      SUBROUTINE CGESC2( N, A, LDA, RHS, IPIV, JPIV, SCALE )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            LDA, N
      REAL               SCALE
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), JPIV( * )
      COMPLEX            A( LDA, * ), RHS( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE, TWO
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0, TWO = 2.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               BIGNUM, EPS, SMLNUM
      COMPLEX            TEMP
*     ..
*     .. External Subroutines ..
```

```
      EXTERNAL           CLASWP, CSCAL, SLABAD
*     ..
*     .. External Functions ..
      INTEGER            ICAMAX
      REAL               SLAMCH
      EXTERNAL           ICAMAX, SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, CMPLX, REAL
*     ..
*     .. Executable Statements ..
*
*     Set constant to control overflow
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' ) / EPS
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
*
*     Apply permutations IPIV to RHS
*
      CALL CLASWP( 1, RHS, LDA, 1, N-1, IPIV, 1 )
*
*     Solve for L part
*
      DO 20 I = 1, N - 1
         DO 10 J = I + 1, N
            RHS( J ) = RHS( J ) - A( J, I )*RHS( I )
   10    CONTINUE
   20 CONTINUE
*
*     Solve for U part
*
      SCALE = ONE
*
*     Check for scaling
*
      I = ICAMAX( N, RHS, 1 )
      IF( TWO*SMLNUM*ABS( RHS( I ) ).GT.ABS( A( N, N ) ) ) THEN
         TEMP = CMPLX( ONE / TWO, ZERO ) / ABS( RHS( I ) )
         CALL CSCAL( N, TEMP, RHS( 1 ), 1 )
         SCALE = SCALE*REAL( TEMP )
      END IF
      DO 40 I = N, 1, -1
         TEMP = CMPLX( ONE, ZERO ) / A( I, I )
         RHS( I ) = RHS( I )*TEMP
         DO 30 J = I + 1, N
            RHS( I ) = RHS( I ) - RHS( J )*( A( I, J )*TEMP )
   30    CONTINUE
   40 CONTINUE
```

```
*
*     Apply permutations JPIV to the solution (RHS)
*
      CALL CLASWP( 1, RHS, LDA, 1, N-1, JPIV, -1 )
      RETURN
*
*     End of CGESC2
*
      END
```

———-

38.1.4 lisp code

— **cgesc2.lisp** —

```
(let* ((zero 0.0) (one 1.0) (two 2.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (ignorable zero one two))
  (defun cgesc2 (n a lda rhs ipiv jpiv scale)
    (declare (type (single-float) scale)
             (type (array f2cl-lib:integer4 (*)) jpiv ipiv)
             (type (array f2cl-lib:complex8 (*)) rhs a)
             (type (f2cl-lib:integer4) lda n))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (rhs f2cl-lib:complex8 rhs-%data% rhs-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (jpiv f2cl-lib:integer4 jpiv-%data% jpiv-%offset%))
      (prog ((temp #C(0.0 0.0)) (bignum 0.0) (eps 0.0) (smlnum 0.0) (i 0)
             (j 0))
        (declare (type (f2cl-lib:complex8) temp)
                 (type (single-float) bignum eps smlnum)
                 (type (f2cl-lib:integer4) i j))
        (setf eps (slamch "P"))
        (setf smlnum (/ (slamch "S") eps))
        (setf bignum (/ one smlnum))
        (multiple-value-bind (var-0 var-1)
            (slabad smlnum bignum)
          (declare (ignore))
          (when var-0 (setf smlnum var-0))
          (when var-1 (setf bignum var-1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (claswp 1 rhs lda 1 (f2cl-lib:int-sub n 1) ipiv 1)
          (declare (ignore var-0 var-1 var-3 var-4 var-5 var-6))
          (when var-2 (setf lda var-2)))
```

```
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref rhs-%data% (j) ((1 *)) rhs-%offset%)
                (- (f2cl-lib:fref rhs-%data% (j) ((1 *)) rhs-%offset%)
                   (*
                    (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                   a-%offset%)
                    (f2cl-lib:fref rhs-%data% (i) ((1 *))
                                   rhs-%offset%))))
       label10))
   label20))
(setf scale one)
(setf i
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (icamax n rhs 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(cond
 ((> (* two smlnum (abs (f2cl-lib:fref rhs (i) ((1 *)))))
     (abs (f2cl-lib:fref a (n n) ((1 lda) (1 *)))))
  (setf temp
          (/ (f2cl-lib:cmplx (/ one two) zero)
             (abs
              (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%))))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (cscal n temp
       (f2cl-lib:array-slice rhs-%data% f2cl-lib:complex8 (1) ((1 *))
                             rhs-%offset%)
       1)
    (declare (ignore var-2 var-3))
    (when var-0 (setf n var-0))
    (when var-1 (setf temp var-1)))
  (setf scale (* scale (f2cl-lib:freal temp)))))
(f2cl-lib:fdo (i n (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf temp
            (/ (f2cl-lib:cmplx one zero)
               (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                              a-%offset%)))
    (setf (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%)
            (* (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%)
               temp))
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                  ((> j n) nil)
```

```
              (tagbody
                (setf (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%)
                        (- (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%)
                           (*
                            (f2cl-lib:fref rhs-%data% (j) ((1 *)) rhs-%offset%)
                            (*
                             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                            a-%offset%)
                             temp))))
               label30))
           label40))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (claswp 1 rhs lda 1 (f2cl-lib:int-sub n 1) jpiv -1)
          (declare (ignore var-0 var-1 var-3 var-4 var-5 var-6))
          (when var-2 (setf lda var-2)))
        (go end_label)
       end_label
        (return (values n nil lda nil nil nil scale))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgesc2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (single-float))
                                            :return-values
                                            '(fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::scale)
                                            :calls 'nil)))
```

———-

38.2 cgetc2

38.2.1 Axiom unit tests

— cgetc2.input —

```
)set break resume
)sys rm -f cgetc2.output
)spool cgetc2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

38.2.2 Axiom help page

— cgetc2.help —

```
======================================================================
cgetc2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgetc2 (N, A, LDA, IPIV, JPIV, INFO)
           CGETC2 computes the LU factorization with complete pivoting of the
           general n-by-n matrix.

Function/Subroutine Documentation
   subroutine cgetc2 (integer N, complex, dimension( lda, * ) A, integer LDA,
       integer, dimension( * ) IPIV, integer, dimension( * ) JPIV, integer
       INFO)
       CGETC2 computes the LU factorization with complete pivoting of the
       general n-by-n matrix.

Purpose:
   CGETC2 computes an LU factorization, using complete pivoting, of the
   n-by-n matrix A. The factorization has the form A = P * L * U * Q,
```

```
  where P and Q are permutation matrices, L is lower triangular with
  unit diagonal elements and U is upper triangular.

  This is a level 1 BLAS version of the algorithm.

Parameters:

N
  N is INTEGER
  The order of the matrix A. N >= 0.

A
  A is COMPLEX array, dimension (LDA, N)
  On entry, the n-by-n matrix to be factored.
  On exit, the factors L and U from the factorization
  A = P*L*U*Q; the unit diagonal elements of L are not stored.
  If U(k, k) appears to be less than SMIN, U(k, k) is given the
  value of SMIN, giving a nonsingular perturbed system.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1, N).

IPIV
  IPIV is INTEGER array, dimension (N).
  The pivot indices; for 1 <= i <= N, row i of the
  matrix has been interchanged with row IPIV(i).

JPIV
  JPIV is INTEGER array, dimension (N).
  The pivot indices; for 1 <= j <= N, column j of the
  matrix has been interchanged with column JPIV(j).

INFO
  INFO is INTEGER
  = 0: successful exit
  > 0: if INFO = k, U(k, k) is likely to produce overflow if
       one tries to solve for x in Ax = b. So U is perturbed
       to avoid the overflow.
```

38.2.3 fortran code

— cgetc2.f —

```
*  =====================================================================
      SUBROUTINE CGETC2( N, A, LDA, IPIV, JPIV, INFO )
*
*  -- LAPACK auxiliary routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), JPIV( * )
      COMPLEX            A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IP, IPV, J, JP, JPV
      REAL               BIGNUM, EPS, SMIN, SMLNUM, XMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGERU, CSWAP, SLABAD
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, CMPLX, MAX
*     ..
*     .. Executable Statements ..
*
*     Set constants to control overflow
*
      INFO = 0
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' ) / EPS
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
*
*     Factorize A using complete pivoting.
*     Set pivots less than SMIN to SMIN
*
      DO 40 I = 1, N - 1
```

```
*
*       Find max element in matrix A
*
        XMAX = ZERO
        DO 20 IP = I, N
           DO 10 JP = I, N
              IF( ABS( A( IP, JP ) ).GE.XMAX ) THEN
                 XMAX = ABS( A( IP, JP ) )
                 IPV = IP
                 JPV = JP
              END IF
   10      CONTINUE
   20   CONTINUE
        IF( I.EQ.1 )
     $     SMIN = MAX( EPS*XMAX, SMLNUM )
*
*       Swap rows
*
        IF( IPV.NE.I )
     $     CALL CSWAP( N, A( IPV, 1 ), LDA, A( I, 1 ), LDA )
        IPIV( I ) = IPV
*
*       Swap columns
*
        IF( JPV.NE.I )
     $     CALL CSWAP( N, A( 1, JPV ), 1, A( 1, I ), 1 )
        JPIV( I ) = JPV
*
*       Check for singularity
*
        IF( ABS( A( I, I ) ).LT.SMIN ) THEN
           INFO = I
           A( I, I ) = CMPLX( SMIN, ZERO )
        END IF
        DO 30 J = I + 1, N
           A( J, I ) = A( J, I ) / A( I, I )
   30   CONTINUE
        CALL CGERU( N-I, N-I, -CMPLX( ONE ), A( I+1, I ), 1,
     $              A( I, I+1 ), LDA, A( I+1, I+1 ), LDA )
   40 CONTINUE
*
      IF( ABS( A( N, N ) ).LT.SMIN ) THEN
         INFO = N
         A( N, N ) = CMPLX( SMIN, ZERO )
      END IF
*
*     Set last pivots to N
*
      IPIV( N ) = N
      JPIV( N ) = N
```

```
*
      RETURN
*
*     End of CGETC2
*
      END
```

———-

38.2.4 lisp code

— **cgetc2.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun cgetc2 (n a lda ipiv jpiv info)
    (declare (type (array f2cl-lib:integer4 (*)) jpiv ipiv)
             (type (array f2cl-lib:complex8 (*)) a)
             (type (f2cl-lib:integer4) info lda n))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (jpiv f2cl-lib:integer4 jpiv-%data% jpiv-%offset%))
      (prog ((bignum 0.0) (eps 0.0) (smin 0.0) (smlnum 0.0) (xmax 0.0) (i 0)
             (ip 0) (ipv 0) (j 0) (jp 0) (jpv 0))
        (declare (type (single-float) bignum eps smin smlnum xmax)
                 (type (f2cl-lib:integer4) i ip ipv j jp jpv))
        (setf info 0)
        (setf eps (slamch "P"))
        (setf smlnum (/ (slamch "S") eps))
        (setf bignum (/ one smlnum))
        (multiple-value-bind (var-0 var-1)
            (slabad smlnum bignum)
          (declare (ignore))
          (when var-0 (setf smlnum var-0))
          (when var-1 (setf bignum var-1)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
          (tagbody
            (setf xmax zero)
            (f2cl-lib:fdo (ip i (f2cl-lib:int-add ip 1))
                          ((> ip n) nil)
              (tagbody
                (f2cl-lib:fdo (jp i (f2cl-lib:int-add jp 1))
                              ((> jp n) nil)
                  (tagbody
```

```
        (cond
         ((>= (abs (f2cl-lib:fref a (ip jp) ((1 lda) (1 *)))) xmax)
          (setf xmax
                  (abs
                   (f2cl-lib:fref a-%data% (ip jp) ((1 lda) (1 *))
                                  a-%offset%)))
          (setf ipv ip) (setf jpv jp)))
       label10))
   label20))
(if (= i 1)
    (setf smin (max (* eps xmax) smlnum)))
(if (/= ipv i)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (cswap n
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (ipv 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-1 var-3))
      (when var-0 (setf n var-0))
      (when var-2 (setf lda var-2))
      (when var-4 (setf lda var-4))))
(setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) ipv)
(if (/= jpv i)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (cswap n
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 jpv)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 i)
                               ((1 lda) (1 *)) a-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0))))
(setf (f2cl-lib:fref jpiv-%data% (i) ((1 *)) jpiv-%offset%) jpv)
(cond
 ((< (abs (f2cl-lib:fref a (i i) ((1 lda) (1 *)))) smin)
  (setf info i)
  (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
          (f2cl-lib:cmplx smin zero))))
(f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *)) a-%offset%)
            (/
             (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                            a-%offset%)
             (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
```

```
                                        a-%offset%)))
                label30))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                  (cgeru (f2cl-lib:int-sub n i) (f2cl-lib:int-sub n i)
                   (- (f2cl-lib:cmplx one))
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 ((+ i 1) i)
                                         ((1 lda) (1 *)) a-%offset%)
                   1
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                         (i (f2cl-lib:int-add i 1))
                                         ((1 lda) (1 *)) a-%offset%)
                   lda
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                         ((+ i 1) (f2cl-lib:int-add i 1))
                                         ((1 lda) (1 *)) a-%offset%)
                   lda)
                (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7))
                (when var-6 (setf lda var-6))
                (when var-8 (setf lda var-8)))
            label40))
          (cond
           ((< (abs (f2cl-lib:fref a (n n) ((1 lda) (1 *)))) smin) (setf info n)
            (setf (f2cl-lib:fref a-%data% (n n) ((1 lda) (1 *)) a-%offset%)
                    (f2cl-lib:cmplx smin zero))))
          (setf (f2cl-lib:fref ipiv-%data% (n) ((1 *)) ipiv-%offset%) n)
          (setf (f2cl-lib:fref jpiv-%data% (n) ((1 *)) jpiv-%offset%) n)
          (go end_label)
         end_label
          (return (values n nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgetc2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::info)
```

```
                                        :calls 'nil)))
```

———-

38.3 clange

38.3.1 Axiom unit tests

— **clange.input** —

```
)set break resume
)sys rm -f clange.output
)spool clange.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

38.3.2 Axiom help page

— **clange.help** —

```
=====================================================================
clange examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       real function clange (NORM, M, N, A, LDA, WORK)
           CLANGE returns the value of the 1-norm, Frobenius norm, infinity-
           norm, or the largest absolute value of any element of a general
           rectangular matrix.

Function/Subroutine Documentation
```

```
   real function clange (character NORM, integer M, integer N, complex,
       dimension( lda, * ) A, integer LDA, real, dimension( * ) WORK)
       CLANGE returns the value of the 1-norm, Frobenius norm, infinity-norm,
       or the largest absolute value of any element of a general rectangular
       matrix.

Purpose:
   CLANGE  returns the value of the one norm,  or the Frobenius norm, or
   the  infinity norm,  or the  element of  largest absolute value  of a
   complex matrix A.

 Returns:

 CLANGE
   CLANGE = ( max(abs(A(i,j))), NORM = 'M' or 'm'
            (
            ( norm1(A),         NORM = '1', 'O' or 'o'
            (
            ( normI(A),         NORM = 'I' or 'i'
            (
            ( normF(A),         NORM = 'F', 'f', 'E' or 'e'

   where  norm1  denotes the  one norm of a matrix (maximum column sum),
   normI  denotes the  infinity norm  of a matrix  (maximum row sum) and
   normF  denotes the  Frobenius norm of a matrix (square root of sum of
   squares).  Note that  max(abs(A(i,j)))  is not a consistent matrix norm.

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies the value to be returned in CLANGE as described above.

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.  When M = 0,
   CLANGE is set to zero.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.  When N = 0,
   CLANGE is set to zero.

 A
   A is COMPLEX array, dimension (LDA,N)
   The m by n matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(M,1).
```

```
 WORK
   WORK is REAL array, dimension (MAX(1,LWORK)),
   where LWORK >= M when NORM = 'I'; otherwise, WORK is not referenced.
```

———-

38.3.3 fortran code

— **clange.f** —

```
*  =====================================================================
      REAL             FUNCTION CLANGE( NORM, M, N, A, LDA, WORK )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            LDA, M, N
*     ..
*     .. Array Arguments ..
      REAL               WORK( * )
      COMPLEX            A( LDA, * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               SCALE, SUM, VALUE, TEMP
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, SISNAN
      EXTERNAL           LSAME, SISNAN
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLASSQ
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MIN, SQRT
```

```
*     ..
*     .. Executable Statements ..
*
      IF( MIN( M, N ).EQ.0 ) THEN
         VALUE = ZERO
      ELSE IF( LSAME( NORM, 'M' ) ) THEN
*
*        Find max(abs(A(i,j))).
*
         VALUE = ZERO
         DO 20 J = 1, N
            DO 10 I = 1, M
               TEMP = ABS( A( I, J ) )
               IF( VALUE.LT.TEMP .OR. SISNAN( TEMP ) ) VALUE = TEMP
   10       CONTINUE
   20    CONTINUE
      ELSE IF( ( LSAME( NORM, 'O' ) ) .OR. ( NORM.EQ.'1' ) ) THEN
*
*        Find norm1(A).
*
         VALUE = ZERO
         DO 40 J = 1, N
            SUM = ZERO
            DO 30 I = 1, M
               SUM = SUM + ABS( A( I, J ) )
   30       CONTINUE
            IF( VALUE.LT.SUM .OR. SISNAN( SUM ) ) VALUE = SUM
   40    CONTINUE
      ELSE IF( LSAME( NORM, 'I' ) ) THEN
*
*        Find normI(A).
*
         DO 50 I = 1, M
            WORK( I ) = ZERO
   50    CONTINUE
         DO 70 J = 1, N
            DO 60 I = 1, M
               WORK( I ) = WORK( I ) + ABS( A( I, J ) )
   60       CONTINUE
   70    CONTINUE
         VALUE = ZERO
         DO 80 I = 1, M
            TEMP = WORK( I )
            IF( VALUE.LT.TEMP .OR. SISNAN( TEMP ) ) VALUE = TEMP
   80    CONTINUE
      ELSE IF( ( LSAME( NORM, 'F' ) ) .OR. ( LSAME( NORM, 'E' ) ) ) THEN
*
*        Find normF(A).
*
         SCALE = ZERO
```

```
         SUM = ONE
         DO 90 J = 1, N
            CALL CLASSQ( M, A( 1, J ), 1, SCALE, SUM )
   90    CONTINUE
         VALUE = SCALE*SQRT( SUM )
      END IF
*
      CLANGE = VALUE
      RETURN
*
*     End of CLANGE
*
      END
```

———-

38.3.4 lisp code

— **clange.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun clange (norm m n a lda work)
    (declare (type (array single-float (*)) work)
             (type (array f2cl-lib:complex8 (*)) a)
             (type (f2cl-lib:integer4) lda n m)
             (type (string *) norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((scale 0.0) (sum 0.0) (value 0.0) (temp 0.0) (i 0) (j 0)
             (clange 0.0))
        (declare (type (f2cl-lib:integer4) i j)
                 (type (single-float) scale sum value temp clange))
        (cond
         ((= (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) 0)
          (setf value zero))
         ((multiple-value-bind (ret-val var-0 var-1)
              (lsame norm "M")
            (declare (ignore var-1))
            (when var-0 (setf norm var-0))
            ret-val)
          (setf value zero)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
```

```
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf temp
                  (abs
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)))
          (if (or (< value temp)
                  (multiple-value-bind (ret-val var-0)
                      (sisnan temp)
                    (declare (ignore))
                    (when var-0 (setf temp var-0))
                    ret-val))
              (setf value temp))
         label10))
     label20)))
 ((or
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame norm "O")
     (declare (ignore var-1))
     (when var-0 (setf norm var-0))
     ret-val)
   (f2cl-lib:fstring-= norm "1"))
  (setf value zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf sum zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf sum
                  (+ sum
                     (abs
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))))
         label30))
      (if (or (< value sum)
              (multiple-value-bind (ret-val var-0)
                  (sisnan sum)
                (declare (ignore))
                (when var-0 (setf sum var-0))
                ret-val))
          (setf value sum))
     label40)))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame norm "I")
    (declare (ignore var-1))
    (when var-0 (setf norm var-0))
```

```
    ret-val)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) zero)
     label50))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (+
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                   (abs
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))))
         label60))
     label70))
  (setf value zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf temp (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
      (if (or (< value temp)
              (multiple-value-bind (ret-val var-0)
                  (sisnan temp)
                (declare (ignore))
                (when var-0 (setf temp var-0))
                ret-val))
          (setf value temp))
     label80)))
 ((or
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame norm "F")
     (declare (ignore var-1))
     (when var-0 (setf norm var-0))
     ret-val)
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame norm "E")
     (declare (ignore var-1))
     (when var-0 (setf norm var-0))
     ret-val))
  (setf scale zero) (setf sum one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
                 (classq m
                  (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 j)
                                        ((1 lda) (1 *)) a-%offset%)
                  1 scale sum)
                (declare (ignore var-1 var-2))
                (when var-0 (setf m var-0))
                (when var-3 (setf scale var-3))
                (when var-4 (setf sum var-4)))
            label90))
          (setf value (* scale (f2cl-lib:fsqrt sum)))))
        (setf clange value)
        (go end_label)
       end_label
        (return (values clange norm m nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::clange
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*)))
                                            :return-values
                                            '(fortran-to-lisp::norm
                                              fortran-to-lisp::m nil nil nil
                                              nil)
                                            :calls 'nil)))
```

———-

38.4 claqge

38.4.1 Axiom unit tests

— **claqge.input** —

```
)set break resume
)sys rm -f claqge.output
)spool claqge.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

38.4.2 Axiom help page

— claqge.help —

```
=====================================================================
claqge examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine claqge (M, N, A, LDA, R, C, ROWCND, COLCND, AMAX, EQUED)
           CLAQGE scales a general rectangular matrix, using row and column
           scaling factors computed by sgeequ.

Function/Subroutine Documentation
   subroutine claqge (integer M, integer N, complex, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) R, real, dimension( * ) C, real
       ROWCND, real COLCND, real AMAX, character EQUED)
       CLAQGE scales a general rectangular matrix, using row and column
       scaling factors computed by sgeequ.

Purpose:
   CLAQGE equilibrates a general M by N matrix A using the row and
   column scaling factors in the vectors R and C.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.
```

```
 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the M by N matrix A.
   On exit, the equilibrated matrix.  See EQUED for the form of
   the equilibrated matrix.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(M,1).

 R
   R is REAL array, dimension (M)
   The row scale factors for A.

 C
   C is REAL array, dimension (N)
   The column scale factors for A.

 ROWCND
   ROWCND is REAL
   Ratio of the smallest R(i) to the largest R(i).

 COLCND
   COLCND is REAL
   Ratio of the smallest C(i) to the largest C(i).

 AMAX
   AMAX is REAL
   Absolute value of largest matrix entry.

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done.
   = 'N':  No equilibration
   = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
   = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
   = 'B':  Both row and column equilibration, i.e., A has been
           replaced by diag(R) * A * diag(C).

Internal Parameters:

   THRESH is a threshold value used to decide if row or column scaling
   should be done based on the ratio of the row or column scaling
   factors.  If ROWCND < THRESH, row scaling is done, and if
   COLCND < THRESH, column scaling is done.

   LARGE and SMALL are threshold values used to decide if row scaling
   should be done based on the absolute size of the largest matrix
   element.  If AMAX > LARGE or AMAX < SMALL, row scaling is done.
```

———-

38.4.3 fortran code

— **claqge.f** —

```
*  ==================================================================
      SUBROUTINE CLAQGE( M, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $                   EQUED )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED
      INTEGER            LDA, M, N
      REAL               AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      REAL               C( * ), R( * )
      COMPLEX            A( LDA, * )
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
      REAL               ONE, THRESH
      PARAMETER          ( ONE = 1.0E+0, THRESH = 0.1E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               CJ, LARGE, SMALL
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( M.LE.0 .OR. N.LE.0 ) THEN
         EQUED = 'N'
         RETURN
      END IF
*
```

```
*     Initialize LARGE and SMALL.
*
      SMALL = SLAMCH( 'Safe minimum' ) / SLAMCH( 'Precision' )
      LARGE = ONE / SMALL
*
      IF( ROWCND.GE.THRESH .AND. AMAX.GE.SMALL .AND. AMAX.LE.LARGE )
     $     THEN
*
*        No row scaling
*
         IF( COLCND.GE.THRESH ) THEN
*
*           No column scaling
*
            EQUED = 'N'
         ELSE
*
*           Column scaling
*
            DO 20 J = 1, N
               CJ = C( J )
               DO 10 I = 1, M
                  A( I, J ) = CJ*A( I, J )
   10          CONTINUE
   20       CONTINUE
            EQUED = 'C'
         END IF
      ELSE IF( COLCND.GE.THRESH ) THEN
*
*        Row scaling, no column scaling
*
         DO 40 J = 1, N
            DO 30 I = 1, M
               A( I, J ) = R( I )*A( I, J )
   30       CONTINUE
   40    CONTINUE
         EQUED = 'R'
      ELSE
*
*        Row and column scaling
*
         DO 60 J = 1, N
            CJ = C( J )
            DO 50 I = 1, M
               A( I, J ) = CJ*R( I )*A( I, J )
   50       CONTINUE
   60    CONTINUE
         EQUED = 'B'
      END IF
*
```

```
      RETURN
*
*     End of CLAQGE
*
      END
```

———-

38.4.4 lisp code

— **claqge.lisp** —

```
(let* ((one 1.0) (thresh 0.1))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.1 0.1) thresh)
           (ignorable one thresh))
  (defun claqge (m n a lda r c rowcnd colcnd amax equed)
    (declare (type (string *) equed)
             (type (single-float) amax colcnd rowcnd)
             (type (array single-float (*)) c r)
             (type (array f2cl-lib:complex8 (*)) a)
             (type (f2cl-lib:integer4) lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (r single-float r-%data% r-%offset%)
         (c single-float c-%data% c-%offset%)
         (equed character equed-%data% equed-%offset%))
      (prog ((cj 0.0) (large 0.0) (small 0.0) (i 0) (j 0))
        (declare (type (single-float) cj large small)
                 (type (f2cl-lib:integer4) i j))
        (cond
         ((or (<= m 0) (<= n 0))
          (f2cl-lib:f2cl-set-string equed "N" (string 1)) (go end_label)))
        (setf small (/ (slamch "Safe minimum") (slamch "Precision")))
        (setf large (/ one small))
        (cond
         ((and (>= rowcnd thresh) (>= amax small) (<= amax large))
          (cond
           ((>= colcnd thresh) (f2cl-lib:f2cl-set-string equed "N" (string 1)))
           (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf cj (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i m) nil)
                  (tagbody
                    (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
```

```
                                        a-%offset%)
                              (* cj
                                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                                a-%offset%)))
                     label10))
                  label20))
              (f2cl-lib:f2cl-set-string equed "C" (string 1)))))
           ((>= colcnd thresh)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i m) nil)
                  (tagbody
                    (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                          (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                            a-%offset%)))
                  label30))
                label40))
            (f2cl-lib:f2cl-set-string equed "R" (string 1)))
           (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf cj (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i m) nil)
                  (tagbody
                    (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                          (* cj (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                            a-%offset%)))
                  label50))
                label60))
            (f2cl-lib:f2cl-set-string equed "B" (string 1))))
          (go end_label)
         end_label
          (return (values nil nil nil nil nil nil nil nil nil equed))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::claqge
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
```

```
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (single-float) (single-float)
  (single-float) (string))
:return-values
'(nil nil nil nil nil nil nil nil
  nil fortran-to-lisp::equed)
:calls 'nil)))
```

———-

38.5 ctgex2

38.5.1 Axiom unit tests

— **ctgex2.input** —

```
)set break resume
)sys rm -f ctgex2.output
)spool ctgex2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

38.5.2 Axiom help page

— **ctgex2.help** —

```
==================================================================
ctgex2 examples
==================================================================

==================================================================
Man Page Details
```

```
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ctgex2 (WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z, LDZ, J1,
           INFO)
           CTGEX2 swaps adjacent diagonal blocks in an upper (quasi)
           triangular matrix pair by an unitary equivalence transformation.

Function/Subroutine Documentation
   subroutine ctgex2 (logical WANTQ, logical WANTZ, integer N, complex,
       dimension( lda, * ) A, integer LDA, complex, dimension( ldb, * ) B,
       integer LDB, complex, dimension( ldq, * ) Q, integer LDQ, complex,
       dimension( ldz, * ) Z, integer LDZ, integer J1, integer INFO)
       CTGEX2 swaps adjacent diagonal blocks in an upper (quasi) triangular
       matrix pair by an unitary equivalence transformation.

Purpose:
   CTGEX2 swaps adjacent diagonal 1 by 1 blocks (A11,B11) and (A22,B22)
   in an upper triangular matrix pair (A, B) by an unitary equivalence
   transformation.

   (A, B) must be in generalized Schur canonical form, that is, A and
   B are both upper triangular.

   Optionally, the matrices Q and Z of generalized Schur vectors are updated.

   Q(in) * A(in) * Z(in)**H = Q(out) * A(out) * Z(out)**H
   Q(in) * B(in) * Z(in)**H = Q(out) * B(out) * Z(out)**H

Parameters:

 WANTQ
   WANTQ is LOGICAL
   .TRUE. : update the left transformation matrix Q;
   .FALSE.: do not update Q.

 WANTZ
   WANTZ is LOGICAL
   .TRUE. : update the right transformation matrix Z;
   .FALSE.: do not update Z.

 N
   N is INTEGER
   The order of the matrices A and B. N >= 0.

 A
   A is COMPLEX arrays, dimensions (LDA,N)
   On entry, the matrix A in the pair (A, B).
   On exit, the updated matrix A.
```

```
 LDA
   LDA is INTEGER
   The leading dimension of the array A. LDA >= max(1,N).

 B
   B is COMPLEX arrays, dimensions (LDB,N)
   On entry, the matrix B in the pair (A, B).
   On exit, the updated matrix B.

 LDB
   LDB is INTEGER
   The leading dimension of the array B. LDB >= max(1,N).

 Q
   Q is COMPLEX array, dimension (LDZ,N)
   If WANTQ = .TRUE, on entry, the unitary matrix Q. On exit,
   the updated matrix Q.
   Not referenced if WANTQ = .FALSE..

 LDQ
   LDQ is INTEGER
   The leading dimension of the array Q. LDQ >= 1;
   If WANTQ = .TRUE., LDQ >= N.

 Z
   Z is COMPLEX array, dimension (LDZ,N)
   If WANTZ = .TRUE, on entry, the unitary matrix Z. On exit,
   the updated matrix Z.
   Not referenced if WANTZ = .FALSE..

 LDZ
   LDZ is INTEGER
   The leading dimension of the array Z. LDZ >= 1;
   If WANTZ = .TRUE., LDZ >= N.

 J1
   J1 is INTEGER
   The index to the first block (A11, B11).

 INFO
   INFO is INTEGER
   =0:  Successful exit.
   =1:  The transformed matrix pair (A, B) would be too far
        from generalized Schur form; the problem is ill-conditioned.

Further Details:

   In the current code both weak and strong stability tests are
   performed. The user can omit the strong stability test by changing
```

the internal logical parameter WANDS to .FALSE.. See ref. [2] for details.

———-

38.5.3 fortran code

— **ctgex2.f** —

```
*  ===================================================================
      SUBROUTINE CTGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                   LDZ, J1, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      LOGICAL            WANTQ, WANTZ
      INTEGER            INFO, J1, LDA, LDB, LDQ, LDZ, N
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), B( LDB, * ), Q( LDQ, * ),
     $                   Z( LDZ, * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      COMPLEX            CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0E+0, 0.0E+0 ),
     $                   CONE = ( 1.0E+0, 0.0E+0 ) )
      REAL               TWENTY
      PARAMETER          ( TWENTY = 2.0E+1 )
      INTEGER            LDST
      PARAMETER          ( LDST = 2 )
      LOGICAL            WANDS
      PARAMETER          ( WANDS = .TRUE. )
*     ..
*     .. Local Scalars ..
      LOGICAL            STRONG, WEAK
      INTEGER            I, M
      REAL               CQ, CZ, EPS, SA, SB, SCALE, SMLNUM, SS, SUM,
     $                   THRESH, WS
      COMPLEX            CDUM, F, G, SQ, SZ
*     ..
*     .. Local Arrays ..
```

```
      COMPLEX            S( LDST, LDST ), T( LDST, LDST ), WORK( 8 )
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLACPY, CLARTG, CLASSQ, CROT
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, CONJG, MAX, REAL, SQRT
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
*     Quick return if possible
*
      IF( N.LE.1 )
     $   RETURN
*
      M = LDST
      WEAK = .FALSE.
      STRONG = .FALSE.
*
*     Make a local copy of selected block in (A, B)
*
      CALL CLACPY( 'Full', M, M, A( J1, J1 ), LDA, S, LDST )
      CALL CLACPY( 'Full', M, M, B( J1, J1 ), LDB, T, LDST )
*
*     Compute the threshold for testing the acceptance of swapping.
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' ) / EPS
      SCALE = REAL( CZERO )
      SUM = REAL( CONE )
      CALL CLACPY( 'Full', M, M, S, LDST, WORK, M )
      CALL CLACPY( 'Full', M, M, T, LDST, WORK( M*M+1 ), M )
      CALL CLASSQ( 2*M*M, WORK, 1, SCALE, SUM )
      SA = SCALE*SQRT( SUM )
*
*     THRES has been changed from
*        THRESH = MAX( TEN*EPS*SA, SMLNUM )
*     to
*        THRESH = MAX( TWENTY*EPS*SA, SMLNUM )
*     on 04/01/10.
*     "Bug" reported by Ondra Kamenik, confirmed by Julie Langou, fixed by
*     Jim Demmel and Guillaume Revy. See forum post 1783.
*
      THRESH = MAX( TWENTY*EPS*SA, SMLNUM )
```

```
*
*     Compute unitary QL and RQ that swap 1-by-1 and 1-by-1 blocks
*     using Givens rotations and perform the swap tentatively.
*
      F = S( 2, 2 )*T( 1, 1 ) - T( 2, 2 )*S( 1, 1 )
      G = S( 2, 2 )*T( 1, 2 ) - T( 2, 2 )*S( 1, 2 )
      SA = ABS( S( 2, 2 ) )
      SB = ABS( T( 2, 2 ) )
      CALL CLARTG( G, F, CZ, SZ, CDUM )
      SZ = -SZ
      CALL CROT( 2, S( 1, 1 ), 1, S( 1, 2 ), 1, CZ, CONJG( SZ ) )
      CALL CROT( 2, T( 1, 1 ), 1, T( 1, 2 ), 1, CZ, CONJG( SZ ) )
      IF( SA.GE.SB ) THEN
         CALL CLARTG( S( 1, 1 ), S( 2, 1 ), CQ, SQ, CDUM )
      ELSE
         CALL CLARTG( T( 1, 1 ), T( 2, 1 ), CQ, SQ, CDUM )
      END IF
      CALL CROT( 2, S( 1, 1 ), LDST, S( 2, 1 ), LDST, CQ, SQ )
      CALL CROT( 2, T( 1, 1 ), LDST, T( 2, 1 ), LDST, CQ, SQ )
*
*     Weak stability test: |S21| + |T21| <= O(EPS F-norm((S, T)))
*
      WS = ABS( S( 2, 1 ) ) + ABS( T( 2, 1 ) )
      WEAK = WS.LE.THRESH
      IF( .NOT.WEAK )
     $   GO TO 20
*
      IF( WANDS ) THEN
*
*        Strong stability test:
*           F-norm((A-QL**H*S*QR, B-QL**H*T*QR)) <= O(EPS*F-norm((A, B)))
*
         CALL CLACPY( 'Full', M, M, S, LDST, WORK, M )
         CALL CLACPY( 'Full', M, M, T, LDST, WORK( M*M+1 ), M )
         CALL CROT( 2, WORK, 1, WORK( 3 ), 1, CZ, -CONJG( SZ ) )
         CALL CROT( 2, WORK( 5 ), 1, WORK( 7 ), 1, CZ, -CONJG( SZ ) )
         CALL CROT( 2, WORK, 2, WORK( 2 ), 2, CQ, -SQ )
         CALL CROT( 2, WORK( 5 ), 2, WORK( 6 ), 2, CQ, -SQ )
         DO 10 I = 1, 2
            WORK( I ) = WORK( I ) - A( J1+I-1, J1 )
            WORK( I+2 ) = WORK( I+2 ) - A( J1+I-1, J1+1 )
            WORK( I+4 ) = WORK( I+4 ) - B( J1+I-1, J1 )
            WORK( I+6 ) = WORK( I+6 ) - B( J1+I-1, J1+1 )
   10    CONTINUE
         SCALE = REAL( CZERO )
         SUM = REAL( CONE )
         CALL CLASSQ( 2*M*M, WORK, 1, SCALE, SUM )
         SS = SCALE*SQRT( SUM )
         STRONG = SS.LE.THRESH
         IF( .NOT.STRONG )
```

```
     $      GO TO 20
      END IF
*
*     If the swap is accepted ("weakly" and "strongly"), apply the
*     equivalence transformations to the original matrix pair (A,B)
*
      CALL CROT( J1+1, A( 1, J1 ), 1, A( 1, J1+1 ), 1, CZ, CONJG( SZ ) )
      CALL CROT( J1+1, B( 1, J1 ), 1, B( 1, J1+1 ), 1, CZ, CONJG( SZ ) )
      CALL CROT( N-J1+1, A( J1, J1 ), LDA, A( J1+1, J1 ), LDA, CQ, SQ )
      CALL CROT( N-J1+1, B( J1, J1 ), LDB, B( J1+1, J1 ), LDB, CQ, SQ )
*
*     Set  N1 by N2 (2,1) blocks to 0
*
      A( J1+1, J1 ) = CZERO
      B( J1+1, J1 ) = CZERO
*
*     Accumulate transformations into Q and Z if requested.
*
      IF( WANTZ )
     $   CALL CROT( N, Z( 1, J1 ), 1, Z( 1, J1+1 ), 1, CZ, CONJG( SZ ) )
      IF( WANTQ )
     $   CALL CROT( N, Q( 1, J1 ), 1, Q( 1, J1+1 ), 1, CQ, CONJG( SQ ) )
*
*     Exit with INFO = 0 if swap was successfully performed.
*
      RETURN
*
*     Exit with INFO = 1 if swap was rejected.
*
   20 CONTINUE
      INFO = 1
      RETURN
*
*     End of CTGEX2
*
      END
```

———-

38.5.4 lisp code

— ctgex2.lisp —

```
(let* ((czero (f2cl-lib:cmplx 0.0 0.0))
       (cone (f2cl-lib:cmplx 1.0 0.0))
       (twenty 20.0)
       (ldst 2)
       (wands f2cl-lib:%true%))
```

```
(declare (type (f2cl-lib:complex8) czero)
         (type (f2cl-lib:complex8) cone)
         (type (single-float 20.0 20.0) twenty)
         (type (f2cl-lib:integer4 2 2) ldst)
         (type f2cl-lib:logical wands)
         (ignorable czero cone twenty ldst wands))
(defun ctgex2 (wantq wantz n a lda b ldb$ q ldq z ldz j1 info)
  (declare (type (array f2cl-lib:complex8 (*)) z q b a)
           (type (f2cl-lib:integer4) info j1 ldz ldq ldb$ lda n)
           (type f2cl-lib:logical wantz wantq))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex8 a-%data% a-%offset%)
       (b f2cl-lib:complex8 b-%data% b-%offset%)
       (q f2cl-lib:complex8 q-%data% q-%offset%)
       (z f2cl-lib:complex8 z-%data% z-%offset%))
    (prog ((s
            (make-array (the fixnum (reduce #'* (list ldst ldst)))
                        :element-type 'f2cl-lib:complex8))
           (work (make-array 8 :element-type 'f2cl-lib:complex8))
           (cdum #C(0.0 0.0)) (f #C(0.0 0.0)) (g #C(0.0 0.0))
           (sq #C(0.0 0.0)) (sz #C(0.0 0.0)) (cq 0.0) (cz 0.0) (eps 0.0)
           (sa 0.0) (sb 0.0) (scale 0.0) (smlnum 0.0) (ss 0.0) (sum 0.0)
           (thresh 0.0) (ws 0.0) (i 0) (m 0) (strong nil) (weak nil)
           (t$
            (make-array (the fixnum (reduce #'* (list ldst ldst)))
                        :element-type 'f2cl-lib:complex8)))
      (declare (type (array f2cl-lib:complex8 (*)) t$ s)
               (type (array f2cl-lib:complex8 (8)) work)
               (type (f2cl-lib:complex8) cdum f g sq sz)
               (type (single-float) cq cz eps sa sb scale smlnum ss sum
                thresh ws)
               (type (f2cl-lib:integer4) i m)
               (type f2cl-lib:logical strong weak))
      (setf info 0)
      (if (<= n 1)
          (go end_label))
      (setf m ldst)
      (setf weak f2cl-lib:%false%)
      (setf strong f2cl-lib:%false%)
      (clacpy "Full" m m
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (j1 j1)
                             ((1 lda) (1 *)) a-%offset%)
       lda s ldst)
      (clacpy "Full" m m
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (j1 j1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$ t$ ldst)
      (setf eps (slamch "P"))
      (setf smlnum (/ (slamch "S") eps))
      (setf scale (f2cl-lib:freal czero))
```

```
(setf sum (f2cl-lib:freal cone))
(clacpy "Full" m m s ldst work m)
(clacpy "Full" m m t$ ldst
 (f2cl-lib:array-slice work f2cl-lib:complex8
                       ((+ (f2cl-lib:int-mul m m) 1)) ((1 8)))
 m)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (classq (f2cl-lib:int-mul 2 m m) work 1 scale sum)
  (declare (ignore var-0 var-1 var-2))
  (setf scale var-3)
  (setf sum var-4))
(setf sa (* scale (f2cl-lib:fsqrt sum)))
(setf thresh (max (* twenty eps sa) smlnum))
(setf f
        (-
         (* (f2cl-lib:fref s (2 2) ((1 ldst) (1 ldst)))
            (f2cl-lib:fref t$ (1 1) ((1 ldst) (1 ldst))))
         (* (f2cl-lib:fref t$ (2 2) ((1 ldst) (1 ldst)))
            (f2cl-lib:fref s (1 1) ((1 ldst) (1 ldst))))))
(setf g
        (-
         (* (f2cl-lib:fref s (2 2) ((1 ldst) (1 ldst)))
            (f2cl-lib:fref t$ (1 2) ((1 ldst) (1 ldst))))
         (* (f2cl-lib:fref t$ (2 2) ((1 ldst) (1 ldst)))
            (f2cl-lib:fref s (1 2) ((1 ldst) (1 ldst))))))
(setf sa (abs (f2cl-lib:fref s (2 2) ((1 ldst) (1 ldst)))))
(setf sb (abs (f2cl-lib:fref t$ (2 2) ((1 ldst) (1 ldst)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (clartg g f cz sz cdum)
  (declare (ignore var-0 var-1))
  (setf cz var-2)
  (setf sz var-3)
  (setf cdum var-4))
(setf sz (- sz))
(crot 2
 (f2cl-lib:array-slice s f2cl-lib:complex8 (1 1) ((1 ldst) (1 ldst))) 1
 (f2cl-lib:array-slice s f2cl-lib:complex8 (1 2) ((1 ldst) (1 ldst))) 1
 cz (f2cl-lib:conjg sz))
(crot 2
 (f2cl-lib:array-slice t$ f2cl-lib:complex8 (1 1) ((1 ldst) (1 ldst)))
 1
 (f2cl-lib:array-slice t$ f2cl-lib:complex8 (1 2) ((1 ldst) (1 ldst)))
 1 cz (f2cl-lib:conjg sz))
(cond
 ((>= sa sb)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (clartg (f2cl-lib:fref s (1 1) ((1 ldst) (1 ldst)))
       (f2cl-lib:fref s (2 1) ((1 ldst) (1 ldst))) cq sq cdum)
    (declare (ignore var-0 var-1))
    (setf cq var-2)
```

```
    (setf sq var-3)
    (setf cdum var-4)))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (clartg (f2cl-lib:fref t$ (1 1) ((1 ldst) (1 ldst)))
       (f2cl-lib:fref t$ (2 1) ((1 ldst) (1 ldst))) cq sq cdum)
    (declare (ignore var-0 var-1))
    (setf cq var-2)
    (setf sq var-3)
    (setf cdum var-4))))
(crot 2
 (f2cl-lib:array-slice s f2cl-lib:complex8 (1 1) ((1 ldst) (1 ldst)))
 ldst
 (f2cl-lib:array-slice s f2cl-lib:complex8 (2 1) ((1 ldst) (1 ldst)))
 ldst cq sq)
(crot 2
 (f2cl-lib:array-slice t$ f2cl-lib:complex8 (1 1) ((1 ldst) (1 ldst)))
 ldst
 (f2cl-lib:array-slice t$ f2cl-lib:complex8 (2 1) ((1 ldst) (1 ldst)))
 ldst cq sq)
(setf ws
        (+ (abs (f2cl-lib:fref s (2 1) ((1 ldst) (1 ldst))))
           (abs (f2cl-lib:fref t$ (2 1) ((1 ldst) (1 ldst))))))
(setf weak (<= ws thresh))
(if (not weak)
    (go label20))
(cond
 (wands (clacpy "Full" m m s ldst work m)
  (clacpy "Full" m m t$ ldst
   (f2cl-lib:array-slice work f2cl-lib:complex8
                         ((+ (f2cl-lib:int-mul m m) 1)) ((1 8)))
   m)
  (crot 2 work 1
   (f2cl-lib:array-slice work f2cl-lib:complex8 (3) ((1 8))) 1 cz
   (- (f2cl-lib:conjg sz)))
  (crot 2 (f2cl-lib:array-slice work f2cl-lib:complex8 (5) ((1 8))) 1
   (f2cl-lib:array-slice work f2cl-lib:complex8 (7) ((1 8))) 1 cz
   (- (f2cl-lib:conjg sz)))
  (crot 2 work 2
   (f2cl-lib:array-slice work f2cl-lib:complex8 (2) ((1 8))) 2 cq
   (- sq))
  (crot 2 (f2cl-lib:array-slice work f2cl-lib:complex8 (5) ((1 8))) 2
   (f2cl-lib:array-slice work f2cl-lib:complex8 (6) ((1 8))) 2 cq
   (- sq))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i 2) nil)
    (tagbody
      (setf (f2cl-lib:fref work (i) ((1 8)))
              (- (f2cl-lib:fref work (i) ((1 8)))
                 (f2cl-lib:fref a-%data%
```

```
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-add j1 i) 1)
                                 j1)
                                ((1 lda) (1 *)) a-%offset%)))
        (setf (f2cl-lib:fref work ((f2cl-lib:int-add i 2)) ((1 8)))
                (- (f2cl-lib:fref work ((f2cl-lib:int-add i 2)) ((1 8)))
                   (f2cl-lib:fref a-%data%
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-add j1 i) 1)
                                 (f2cl-lib:int-add j1 1))
                                ((1 lda) (1 *)) a-%offset%)))
        (setf (f2cl-lib:fref work ((f2cl-lib:int-add i 4)) ((1 8)))
                (- (f2cl-lib:fref work ((f2cl-lib:int-add i 4)) ((1 8)))
                   (f2cl-lib:fref b-%data%
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-add j1 i) 1)
                                 j1)
                                ((1 ldb$) (1 *)) b-%offset%)))
        (setf (f2cl-lib:fref work ((f2cl-lib:int-add i 6)) ((1 8)))
                (- (f2cl-lib:fref work ((f2cl-lib:int-add i 6)) ((1 8)))
                   (f2cl-lib:fref b-%data%
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-add j1 i) 1)
                                 (f2cl-lib:int-add j1 1))
                                ((1 ldb$) (1 *)) b-%offset%)))
      label10))
    (setf scale (f2cl-lib:freal czero)) (setf sum (f2cl-lib:freal cone))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (classq (f2cl-lib:int-mul 2 m m) work 1 scale sum)
      (declare (ignore var-0 var-1 var-2))
      (setf scale var-3)
      (setf sum var-4))
    (setf ss (* scale (f2cl-lib:fsqrt sum))) (setf strong (<= ss thresh))
    (if (not strong)
        (go label20))))
  (crot (f2cl-lib:int-add j1 1)
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 j1)
                         ((1 lda) (1 *)) a-%offset%)
   1
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                         (1 (f2cl-lib:int-add j1 1)) ((1 lda) (1 *))
                         a-%offset%)
   1 cz (f2cl-lib:conjg sz))
  (crot (f2cl-lib:int-add j1 1)
   (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 j1)
                         ((1 ldb$) (1 *)) b-%offset%)
   1
   (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                         (1 (f2cl-lib:int-add j1 1)) ((1 ldb$) (1 *))
                         b-%offset%)
```

```
         1 cz (f2cl-lib:conjg sz))
        (crot (f2cl-lib:int-add (f2cl-lib:int-sub n j1) 1)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (j1 j1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 ((+ j1 1) j1)
                               ((1 lda) (1 *)) a-%offset%)
         lda cq sq)
        (crot (f2cl-lib:int-add (f2cl-lib:int-sub n j1) 1)
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (j1 j1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 ((+ j1 1) j1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$ cq sq)
        (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add j1 1) j1)
                             ((1 lda) (1 *)) a-%offset%)
                czero)
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add j1 1) j1)
                             ((1 ldb$) (1 *)) b-%offset%)
                czero)
        (if wantz
            (crot n
             (f2cl-lib:array-slice z-%data% f2cl-lib:complex8 (1 j1)
                                   ((1 ldz) (1 *)) z-%offset%)
             1
             (f2cl-lib:array-slice z-%data% f2cl-lib:complex8
                                   (1 (f2cl-lib:int-add j1 1)) ((1 ldz) (1 *))
                                   z-%offset%)
             1 cz (f2cl-lib:conjg sz)))
        (if wantq
            (crot n
             (f2cl-lib:array-slice q-%data% f2cl-lib:complex8 (1 j1)
                                   ((1 ldq) (1 *)) q-%offset%)
             1
             (f2cl-lib:array-slice q-%data% f2cl-lib:complex8
                                   (1 (f2cl-lib:int-add j1 1)) ((1 ldq) (1 *))
                                   q-%offset%)
             1 cq (f2cl-lib:conjg sq)))
        (go end_label)
       label20
        (setf info 1)
        (go end_label)
       end_label
        (return
         (values nil nil nil nil nil nil nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
```

```
(setf (gethash 'fortran-to-lisp::ctgex2
               fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '(fortran-to-lisp::logical
                                            fortran-to-lisp::logical
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4))
                                          :return-values
                                          '(nil nil nil nil nil nil nil nil
                                            nil nil nil nil
                                            fortran-to-lisp::info)
                                          :calls
                                          '(fortran-to-lisp::crot
                                            fortran-to-lisp::clartg
                                            fortran-to-lisp::classq
                                            fortran-to-lisp::clacpy))))
```

———-

Chapter 39

LAPACK - General Matrices, Auxilliary Routines, Complex16

39.1 zgesc2

39.1.1 Axiom unit tests

— **zgesc2.input** —

```
)set break resume
)sys rm -f zgesc2.output
)spool zgesc2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

39.1.2 Axiom help page

— **zgesc2.help** —

```
==========================================================================
zgesc2 examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgesc2 (N, A, LDA, RHS, IPIV, JPIV, SCALE)
           ZGESC2 solves a system of linear equations using the LU
           factorization with complete pivoting computed by sgetc2.

Function/Subroutine Documentation
   subroutine zgesc2 (integer N, complex*16, dimension( lda, * ) A, integer
       LDA, complex*16, dimension( * ) RHS, integer, dimension( * ) IPIV,
       integer, dimension( * ) JPIV, double precision SCALE)
       ZGESC2 solves a system of linear equations using the LU factorization
       with complete pivoting computed by sgetc2.

Purpose:
   ZGESC2 solves a system of linear equations

      A * X = scale* RHS

   with a general N-by-N matrix A using the LU factorization with
   complete pivoting computed by ZGETC2.

Parameters:

 N
   N is INTEGER
   The number of columns of the matrix A.

 A
   A is COMPLEX*16 array, dimension (LDA, N)
   On entry, the  LU part of the factorization of the n-by-n
   matrix A computed by ZGETC2:  A = P * L * U * Q

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1, N).

 RHS
   RHS is COMPLEX*16 array, dimension N.
   On entry, the right hand side vector b.
   On exit, the solution vector X.

 IPIV
   IPIV is INTEGER array, dimension (N).
   The pivot indices; for 1 <= i <= N, row i of the
   matrix has been interchanged with row IPIV(i).
```

```
 JPIV
   JPIV is INTEGER array, dimension (N).
   The pivot indices; for 1 <= j <= N, column j of the
   matrix has been interchanged with column JPIV(j).

 SCALE
   SCALE is DOUBLE PRECISION
   On exit, SCALE contains the scale factor. SCALE is chosen
   0 <= SCALE <= 1 to prevent owerflow in the solution.
```

39.1.3 fortran code

— zgesc2.f —

```
*  =====================================================================
      SUBROUTINE ZGESC2( N, A, LDA, RHS, IPIV, JPIV, SCALE )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            LDA, N
      DOUBLE PRECISION   SCALE
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), JPIV( * )
      COMPLEX*16         A( LDA, * ), RHS( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE, TWO
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0, TWO = 2.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   BIGNUM, EPS, SMLNUM
      COMPLEX*16         TEMP
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZLASWP, ZSCAL
```

```
*     ..
*     .. External Functions ..
      INTEGER            IZAMAX
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           IZAMAX, DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DCMPLX
*     ..
*     .. Executable Statements ..
*
*     Set constant to control overflow
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' ) / EPS
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
*
*     Apply permutations IPIV to RHS
*
      CALL ZLASWP( 1, RHS, LDA, 1, N-1, IPIV, 1 )
*
*     Solve for L part
*
      DO 20 I = 1, N - 1
         DO 10 J = I + 1, N
            RHS( J ) = RHS( J ) - A( J, I )*RHS( I )
   10    CONTINUE
   20 CONTINUE
*
*     Solve for U part
*
      SCALE = ONE
*
*     Check for scaling
*
      I = IZAMAX( N, RHS, 1 )
      IF( TWO*SMLNUM*ABS( RHS( I ) ).GT.ABS( A( N, N ) ) ) THEN
         TEMP = DCMPLX( ONE / TWO, ZERO ) / ABS( RHS( I ) )
         CALL ZSCAL( N, TEMP, RHS( 1 ), 1 )
         SCALE = SCALE*DBLE( TEMP )
      END IF
      DO 40 I = N, 1, -1
         TEMP = DCMPLX( ONE, ZERO ) / A( I, I )
         RHS( I ) = RHS( I )*TEMP
         DO 30 J = I + 1, N
            RHS( I ) = RHS( I ) - RHS( J )*( A( I, J )*TEMP )
   30    CONTINUE
   40 CONTINUE
*
```

```
*     Apply permutations JPIV to the solution (RHS)
*
      CALL ZLASWP( 1, RHS, LDA, 1, N-1, JPIV, -1 )
      RETURN
*
*     End of ZGESC2
*
      END
```

39.1.4 lisp code

— **zgesc2.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0) (two 2.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (ignorable zero one two))
  (defun zgesc2 (n a lda rhs ipiv jpiv scale)
    (declare (type (double-float) scale)
             (type (array f2cl-lib:integer4 (*)) jpiv ipiv)
             (type (array f2cl-lib:complex16 (*)) rhs a)
             (type (f2cl-lib:integer4) lda n))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (rhs f2cl-lib:complex16 rhs-%data% rhs-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (jpiv f2cl-lib:integer4 jpiv-%data% jpiv-%offset%))
      (prog ((temp #C(0.0d0 0.0d0)) (bignum 0.0d0) (eps 0.0d0) (smlnum 0.0d0)
             (i 0) (j 0))
        (declare (type (f2cl-lib:complex16) temp)
                 (type (double-float) bignum eps smlnum)
                 (type (f2cl-lib:integer4) i j))
        (setf eps (dlamch "P"))
        (setf smlnum (/ (dlamch "S") eps))
        (setf bignum (/ one smlnum))
        (multiple-value-bind (var-0 var-1)
            (dlabad smlnum bignum)
          (declare (ignore))
          (when var-0 (setf smlnum var-0))
          (when var-1 (setf bignum var-1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zlaswp 1 rhs lda 1 (f2cl-lib:int-sub n 1) ipiv 1)
          (declare (ignore var-0 var-1 var-3 var-4 var-5 var-6))
          (when var-2 (setf lda var-2)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref rhs-%data% (j) ((1 *)) rhs-%offset%)
                (- (f2cl-lib:fref rhs-%data% (j) ((1 *)) rhs-%offset%)
                   (*
                    (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                   a-%offset%)
                    (f2cl-lib:fref rhs-%data% (i) ((1 *))
                                   rhs-%offset%))))
       label10))
   label20))
(setf scale one)
(setf i
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (izamax n rhs 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(cond
 ((> (* two smlnum (abs (f2cl-lib:fref rhs (i) ((1 *)))))
     (abs (f2cl-lib:fref a (n n) ((1 lda) (1 *)))))
  (setf temp
          (/ (f2cl-lib:dcmplx (/ one two) zero)
             (abs
              (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%))))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (zscal n temp
       (f2cl-lib:array-slice rhs-%data% f2cl-lib:complex16 (1) ((1 *))
                             rhs-%offset%)
       1)
    (declare (ignore var-2 var-3))
    (when var-0 (setf n var-0))
    (when var-1 (setf temp var-1)))
  (setf scale (* scale (f2cl-lib:dble temp)))))
(f2cl-lib:fdo (i n (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf temp
            (/ (f2cl-lib:dcmplx one zero)
               (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                              a-%offset%)))
    (setf (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%)
            (* (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%)
               temp))
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
```

```
                (setf (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%)
                        (- (f2cl-lib:fref rhs-%data% (i) ((1 *)) rhs-%offset%)
                           (*
                            (f2cl-lib:fref rhs-%data% (j) ((1 *)) rhs-%offset%)
                            (*
                             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                            a-%offset%)
                             temp))))
               label30))
            label40))
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
             (zlaswp 1 rhs lda 1 (f2cl-lib:int-sub n 1) jpiv -1)
           (declare (ignore var-0 var-1 var-3 var-4 var-5 var-6))
           (when var-2 (setf lda var-2)))
         (go end_label)
        end_label
         (return (values n nil lda nil nil nil scale))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgesc2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (double-float))
                                            :return-values
                                            '(fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::scale)
                                            :calls 'nil)))
```

———-

39.2 zgetc2

39.2.1 Axiom unit tests

— **zgetc2.input** —

```
)set break resume
)sys rm -f zgetc2.output
)spool zgetc2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

39.2.2 Axiom help page

— **zgetc2.help** —

```
======================================================================
zgetc2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgetc2 (N, A, LDA, IPIV, JPIV, INFO)
           ZGETC2 computes the LU factorization with complete pivoting of the
           general n-by-n matrix.

Function/Subroutine Documentation
   subroutine zgetc2 (integer N, complex*16, dimension( lda, * ) A, integer
       LDA, integer, dimension( * ) IPIV, integer, dimension( * ) JPIV,
       integer INFO)
       ZGETC2 computes the LU factorization with complete pivoting of the
       general n-by-n matrix.

Purpose:
   ZGETC2 computes an LU factorization, using complete pivoting, of the
   n-by-n matrix A. The factorization has the form A = P * L * U * Q,
```

```
   where P and Q are permutation matrices, L is lower triangular with
   unit diagonal elements and U is upper triangular.

   This is a level 1 BLAS version of the algorithm.

Parameters:

 N
   N is INTEGER
   The order of the matrix A. N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA, N)
   On entry, the n-by-n matrix to be factored.
   On exit, the factors L and U from the factorization
   A = P*L*U*Q; the unit diagonal elements of L are not stored.
   If U(k, k) appears to be less than SMIN, U(k, k) is given the
   value of SMIN, giving a nonsingular perturbed system.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1, N).

 IPIV
   IPIV is INTEGER array, dimension (N).
   The pivot indices; for 1 <= i <= N, row i of the
   matrix has been interchanged with row IPIV(i).

 JPIV
   JPIV is INTEGER array, dimension (N).
   The pivot indices; for 1 <= j <= N, column j of the
   matrix has been interchanged with column JPIV(j).

 INFO
   INFO is INTEGER
   = 0: successful exit
   > 0: if INFO = k, U(k, k) is likely to produce overflow if
        one tries to solve for x in Ax = b. So U is perturbed
        to avoid the overflow.
```

39.2.3 fortran code

— zgetc2.f —

```
*  =====================================================================
```

```
      SUBROUTINE ZGETC2( N, A, LDA, IPIV, JPIV, INFO )
*
*  -- LAPACK auxiliary routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), JPIV( * )
      COMPLEX*16         A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IP, IPV, J, JP, JPV
      DOUBLE PRECISION   BIGNUM, EPS, SMIN, SMLNUM, XMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZGERU, ZSWAP
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DCMPLX, MAX
*     ..
*     .. Executable Statements ..
*
*     Set constants to control overflow
*
      INFO = 0
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' ) / EPS
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
*
*     Factorize A using complete pivoting.
*     Set pivots less than SMIN to SMIN
*
      DO 40 I = 1, N - 1
*
```

```
*        Find max element in matrix A
*
         XMAX = ZERO
         DO 20 IP = I, N
            DO 10 JP = I, N
               IF( ABS( A( IP, JP ) ).GE.XMAX ) THEN
                  XMAX = ABS( A( IP, JP ) )
                  IPV = IP
                  JPV = JP
               END IF
   10       CONTINUE
   20    CONTINUE
         IF( I.EQ.1 )
     $      SMIN = MAX( EPS*XMAX, SMLNUM )
*
*        Swap rows
*
         IF( IPV.NE.I )
     $      CALL ZSWAP( N, A( IPV, 1 ), LDA, A( I, 1 ), LDA )
         IPIV( I ) = IPV
*
*        Swap columns
*
         IF( JPV.NE.I )
     $      CALL ZSWAP( N, A( 1, JPV ), 1, A( 1, I ), 1 )
         JPIV( I ) = JPV
*
*        Check for singularity
*
         IF( ABS( A( I, I ) ).LT.SMIN ) THEN
            INFO = I
            A( I, I ) = DCMPLX( SMIN, ZERO )
         END IF
         DO 30 J = I + 1, N
            A( J, I ) = A( J, I ) / A( I, I )
   30    CONTINUE
         CALL ZGERU( N-I, N-I, -DCMPLX( ONE ), A( I+1, I ), 1,
     $               A( I, I+1 ), LDA, A( I+1, I+1 ), LDA )
   40 CONTINUE
*
      IF( ABS( A( N, N ) ).LT.SMIN ) THEN
         INFO = N
         A( N, N ) = DCMPLX( SMIN, ZERO )
      END IF
*
*     Set last pivots to N
*
      IPIV( N ) = N
      JPIV( N ) = N
*
```

```
      RETURN
*
*     End of ZGETC2
*
      END
```

———-

39.2.4 lisp code

— **zgetc2.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun zgetc2 (n a lda ipiv jpiv info)
    (declare (type (array f2cl-lib:integer4 (*)) jpiv ipiv)
             (type (array f2cl-lib:complex16 (*)) a)
             (type (f2cl-lib:integer4) info lda n))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (jpiv f2cl-lib:integer4 jpiv-%data% jpiv-%offset%))
      (prog ((bignum 0.0d0) (eps 0.0d0) (smin 0.0d0) (smlnum 0.0d0)
             (xmax 0.0d0) (i 0) (ip 0) (ipv 0) (j 0) (jp 0) (jpv 0))
        (declare (type (double-float) bignum eps smin smlnum xmax)
                 (type (f2cl-lib:integer4) i ip ipv j jp jpv))
        (setf info 0)
        (setf eps (dlamch "P"))
        (setf smlnum (/ (dlamch "S") eps))
        (setf bignum (/ one smlnum))
        (multiple-value-bind (var-0 var-1)
            (dlabad smlnum bignum)
          (declare (ignore))
          (when var-0 (setf smlnum var-0))
          (when var-1 (setf bignum var-1)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
          (tagbody
            (setf xmax zero)
            (f2cl-lib:fdo (ip i (f2cl-lib:int-add ip 1))
                          ((> ip n) nil)
              (tagbody
                (f2cl-lib:fdo (jp i (f2cl-lib:int-add jp 1))
                              ((> jp n) nil)
                  (tagbody
                    (cond
```

```
            ((>= (abs (f2cl-lib:fref a (ip jp) ((1 lda) (1 *)))) xmax)
             (setf xmax
                     (abs
                      (f2cl-lib:fref a-%data% (ip jp) ((1 lda) (1 *))
                                     a-%offset%)))
             (setf ipv ip) (setf jpv jp)))
          label10))
      label20))
   (if (= i 1)
       (setf smin (max (* eps xmax) smlnum)))
   (if (/= ipv i)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (zswap n
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (ipv 1)
                                  ((1 lda) (1 *)) a-%offset%)
            lda
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i 1)
                                  ((1 lda) (1 *)) a-%offset%)
            lda)
         (declare (ignore var-1 var-3))
         (when var-0 (setf n var-0))
         (when var-2 (setf lda var-2))
         (when var-4 (setf lda var-4))))
   (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) ipv)
   (if (/= jpv i)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (zswap n
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 jpv)
                                  ((1 lda) (1 *)) a-%offset%)
            1
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 i)
                                  ((1 lda) (1 *)) a-%offset%)
            1)
         (declare (ignore var-1 var-2 var-3 var-4))
         (when var-0 (setf n var-0))))
   (setf (f2cl-lib:fref jpiv-%data% (i) ((1 *)) jpiv-%offset%) jpv)
   (cond
    ((< (abs (f2cl-lib:fref a (i i) ((1 lda) (1 *)))) smin)
     (setf info i)
     (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
             (f2cl-lib:dcmplx smin zero))))
   (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *)) a-%offset%)
               (/
                (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                               a-%offset%)
                (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                               a-%offset%)))
```

```
              label30))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (zgeru (f2cl-lib:int-sub n i) (f2cl-lib:int-sub n i)
                 (- (f2cl-lib:dcmplx one))
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 ((+ i 1) i)
                                       ((1 lda) (1 *)) a-%offset%)
                 1
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                       (i (f2cl-lib:int-add i 1))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                       ((+ i 1) (f2cl-lib:int-add i 1))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7))
              (when var-6 (setf lda var-6))
              (when var-8 (setf lda var-8)))
           label40))
        (cond
         ((< (abs (f2cl-lib:fref a (n n) ((1 lda) (1 *)))) smin) (setf info n)
          (setf (f2cl-lib:fref a-%data% (n n) ((1 lda) (1 *)) a-%offset%)
                  (f2cl-lib:dcmplx smin zero))))
        (setf (f2cl-lib:fref ipiv-%data% (n) ((1 *)) ipiv-%offset%) n)
        (setf (f2cl-lib:fref jpiv-%data% (n) ((1 *)) jpiv-%offset%) n)
        (go end_label)
       end_label
        (return (values n nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgetc2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

39.3 zlange

39.3.1 Axiom unit tests

— **zlange.input** —

```
)set break resume
)sys rm -f zlange.output
)spool zlange.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

39.3.2 Axiom help page

— **zlange.help** —

```
======================================================================
zlange examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function zlange (NORM, M, N, A, LDA, WORK)
           ZLANGE returns the value of the 1-norm, Frobenius norm, infinity-
           norm, or the largest absolute value of any element of a general
           rectangular matrix.

Function/Subroutine Documentation
   double precision function zlange (character NORM, integer M, integer N,
```

```
      complex*16, dimension( lda, * ) A, integer LDA, double precision,
      dimension( * ) WORK)
      ZLANGE returns the value of the 1-norm, Frobenius norm, infinity-norm,
      or the largest absolute value of any element of a general rectangular
      matrix.

Purpose:
   ZLANGE  returns the value of the one norm,  or the Frobenius norm, or
   the  infinity norm,  or the  element of  largest absolute value  of a
   complex matrix A.

 Returns:

 ZLANGE
   ZLANGE = ( max(abs(A(i,j))), NORM = 'M' or 'm'
            (
            ( norm1(A),         NORM = '1', 'O' or 'o'
            (
            ( normI(A),         NORM = 'I' or 'i'
            (
            ( normF(A),         NORM = 'F', 'f', 'E' or 'e'

   where  norm1  denotes the  one norm of a matrix (maximum column sum),
   normI  denotes the  infinity norm  of a matrix  (maximum row sum) and
   normF  denotes the  Frobenius norm of a matrix (square root of sum of
   squares).  Note that  max(abs(A(i,j)))  is not a consistent matrix norm.

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies the value to be returned in ZLANGE as described above.

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.  When M = 0,
   ZLANGE is set to zero.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.  When N = 0,
   ZLANGE is set to zero.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   The m by n matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(M,1).
```

```
 WORK
   WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK)),
   where LWORK >= M when NORM = 'I'; otherwise, WORK is not referenced.
```

———-

39.3.3 fortran code

— **zlange.f** —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION ZLANGE( NORM, M, N, A, LDA, WORK )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   WORK( * )
      COMPLEX*16         A( LDA, * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   SCALE, SUM, VALUE, TEMP
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, DISNAN
      EXTERNAL           LSAME, DISNAN
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZLASSQ
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MIN, SQRT
```

```
*     ..
*     .. Executable Statements ..
*
      IF( MIN( M, N ).EQ.0 ) THEN
         VALUE = ZERO
      ELSE IF( LSAME( NORM, 'M' ) ) THEN
*
*        Find max(abs(A(i,j))).
*
         VALUE = ZERO
         DO 20 J = 1, N
            DO 10 I = 1, M
               TEMP = ABS( A( I, J ) )
               IF( VALUE.LT.TEMP .OR. DISNAN( TEMP ) ) VALUE = TEMP
   10       CONTINUE
   20    CONTINUE
      ELSE IF( ( LSAME( NORM, 'O' ) ) .OR. ( NORM.EQ.'1' ) ) THEN
*
*        Find norm1(A).
*
         VALUE = ZERO
         DO 40 J = 1, N
            SUM = ZERO
            DO 30 I = 1, M
               SUM = SUM + ABS( A( I, J ) )
   30       CONTINUE
            IF( VALUE.LT.SUM .OR. DISNAN( SUM ) ) VALUE = SUM
   40    CONTINUE
      ELSE IF( LSAME( NORM, 'I' ) ) THEN
*
*        Find normI(A).
*
         DO 50 I = 1, M
            WORK( I ) = ZERO
   50    CONTINUE
         DO 70 J = 1, N
            DO 60 I = 1, M
               WORK( I ) = WORK( I ) + ABS( A( I, J ) )
   60       CONTINUE
   70    CONTINUE
         VALUE = ZERO
         DO 80 I = 1, M
            TEMP = WORK( I )
            IF( VALUE.LT.TEMP .OR. DISNAN( TEMP ) ) VALUE = TEMP
   80    CONTINUE
      ELSE IF( ( LSAME( NORM, 'F' ) ) .OR. ( LSAME( NORM, 'E' ) ) ) THEN
*
*        Find normF(A).
*
         SCALE = ZERO
```

```
         SUM = ONE
         DO 90 J = 1, N
            CALL ZLASSQ( M, A( 1, J ), 1, SCALE, SUM )
   90    CONTINUE
         VALUE = SCALE*SQRT( SUM )
      END IF
*
      ZLANGE = VALUE
      RETURN
*
*     End of ZLANGE
*
      END
```

———-

39.3.4 lisp code

— zlange.lisp —

———-

39.4 zlaqge

39.4.1 Axiom unit tests

— zlaqge.input —

```
)set break resume
)sys rm -f zlaqge.output
)spool zlaqge.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

39.4.2 Axiom help page

— zlaqge.help —

```
====================================================================
zlaqge examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zlaqge (M, N, A, LDA, R, C, ROWCND, COLCND, AMAX, EQUED)
           ZLAQGE scales a general rectangular matrix, using row and column
           scaling factors computed by sgeequ.

Function/Subroutine Documentation
   subroutine zlaqge (integer M, integer N, complex*16, dimension( lda, * ) A,
       integer LDA, double precision, dimension( * ) R, double precision,
       dimension( * ) C, double precision ROWCND, double precision COLCND,
       double precision AMAX, character EQUED)
       ZLAQGE scales a general rectangular matrix, using row and column
       scaling factors computed by sgeequ.

Purpose:
   ZLAQGE equilibrates a general M by N matrix A using the row and
   column scaling factors in the vectors R and C.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the M by N matrix A.
   On exit, the equilibrated matrix.  See EQUED for the form of
   the equilibrated matrix.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(M,1).
```

```
 R
   R is DOUBLE PRECISION array, dimension (M)
   The row scale factors for A.

 C
   C is DOUBLE PRECISION array, dimension (N)
   The column scale factors for A.

 ROWCND
   ROWCND is DOUBLE PRECISION
   Ratio of the smallest R(i) to the largest R(i).

 COLCND
   COLCND is DOUBLE PRECISION
   Ratio of the smallest C(i) to the largest C(i).

 AMAX
   AMAX is DOUBLE PRECISION
   Absolute value of largest matrix entry.

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done.
   = 'N':  No equilibration
   = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
   = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
   = 'B':  Both row and column equilibration, i.e., A has been
           replaced by diag(R) * A * diag(C).

Internal Parameters:

   THRESH is a threshold value used to decide if row or column scaling
   should be done based on the ratio of the row or column scaling
   factors.  If ROWCND < THRESH, row scaling is done, and if
   COLCND < THRESH, column scaling is done.

   LARGE and SMALL are threshold values used to decide if row scaling
   should be done based on the absolute size of the largest matrix
   element.  If AMAX > LARGE or AMAX < SMALL, row scaling is done.
```

39.4.3 fortran code

— zlaqge.f —

```
*  =====================================================================
      SUBROUTINE ZLAQGE( M, N, A, LDA, R, C, ROWCND, COLCND, AMAX,
     $                   EQUED )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED
      INTEGER            LDA, M, N
      DOUBLE PRECISION   AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   C( * ), R( * )
      COMPLEX*16         A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, THRESH
      PARAMETER          ( ONE = 1.0D+0, THRESH = 0.1D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   CJ, LARGE, SMALL
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( M.LE.0 .OR. N.LE.0 ) THEN
         EQUED = 'N'
         RETURN
      END IF
*
*     Initialize LARGE and SMALL.
*
      SMALL = DLAMCH( 'Safe minimum' ) / DLAMCH( 'Precision' )
      LARGE = ONE / SMALL
*
      IF( ROWCND.GE.THRESH .AND. AMAX.GE.SMALL .AND. AMAX.LE.LARGE )
     $     THEN
*
```

```
*        No row scaling
*
         IF( COLCND.GE.THRESH ) THEN
*
*           No column scaling
*
            EQUED = 'N'
         ELSE
*
*           Column scaling
*
            DO 20 J = 1, N
               CJ = C( J )
               DO 10 I = 1, M
                  A( I, J ) = CJ*A( I, J )
   10          CONTINUE
   20       CONTINUE
            EQUED = 'C'
         END IF
      ELSE IF( COLCND.GE.THRESH ) THEN
*
*        Row scaling, no column scaling
*
         DO 40 J = 1, N
            DO 30 I = 1, M
               A( I, J ) = R( I )*A( I, J )
   30       CONTINUE
   40    CONTINUE
         EQUED = 'R'
      ELSE
*
*        Row and column scaling
*
         DO 60 J = 1, N
            CJ = C( J )
            DO 50 I = 1, M
               A( I, J ) = CJ*R( I )*A( I, J )
   50       CONTINUE
   60    CONTINUE
         EQUED = 'B'
      END IF
*
      RETURN
*
*     End of ZLAQGE
*
      END
```

39.4.4 lisp code

— zlaqge.lisp —

```
(let* ((one 1.0d0) (thresh 0.1d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.1d0 0.1d0) thresh)
           (ignorable one thresh))
  (defun zlaqge (m n a lda r c rowcnd colcnd amax equed)
    (declare (type (string *) equed)
             (type (double-float) amax colcnd rowcnd)
             (type (array double-float (*)) c r)
             (type (array f2cl-lib:complex16 (*)) a)
             (type (f2cl-lib:integer4) lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (r double-float r-%data% r-%offset%)
         (c double-float c-%data% c-%offset%)
         (equed character equed-%data% equed-%offset%))
      (prog ((cj 0.0d0) (large 0.0d0) (small 0.0d0) (i 0) (j 0))
        (declare (type (double-float) cj large small)
                 (type (f2cl-lib:integer4) i j))
        (cond
         ((or (<= m 0) (<= n 0))
          (f2cl-lib:f2cl-set-string equed "N" (string 1)) (go end_label)))
        (setf small (/ (dlamch "Safe minimum") (dlamch "Precision")))
        (setf large (/ one small))
        (cond
         ((and (>= rowcnd thresh) (>= amax small) (<= amax large))
          (cond
           ((>= colcnd thresh) (f2cl-lib:f2cl-set-string equed "N" (string 1)))
           (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf cj (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i m) nil)
                  (tagbody
                    (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                          (* cj
                             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                            a-%offset%)))
                  label10))
              label20))
            (f2cl-lib:f2cl-set-string equed "C" (string 1)))))
         ((>= colcnd thresh)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                    ((> j n) nil)
          (tagbody
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                      (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)))
               label30))
           label40))
         (f2cl-lib:f2cl-set-string equed "R" (string 1)))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf cj (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i m) nil)
               (tagbody
                 (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%)
                       (* cj (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)))
                label50))
           label60))
         (f2cl-lib:f2cl-set-string equed "B" (string 1))))
       (go end_label)
      end_label
       (return (values nil nil nil nil nil nil nil nil nil equed))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zlaqge
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (array double-float (*))
                                             (double-float) (double-float)
                                             (double-float) (string))
                                           :return-values
                                           '(nil nil nil nil nil nil nil nil
```

```
                                          nil fortran-to-lisp::equed)
                                        :calls 'nil)))
```

———-

39.5 ztgex2

39.5.1 Axiom unit tests

— **ztgex2.input** —

```
)set break resume
)sys rm -f ztgex2.output
)spool ztgex2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

39.5.2 Axiom help page

— **ztgex2.help** —

```
======================================================================
ztgex2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ztgex2 (WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z, LDZ, J1,
           INFO)
           ZTGEX2 swaps adjacent diagonal blocks in an upper (quasi)
           triangular matrix pair by an unitary equivalence transformation.
```

```
Function/Subroutine Documentation
   subroutine ztgex2 (logical WANTQ, logical WANTZ, integer N, complex*16,
       dimension( lda, * ) A, integer LDA, complex*16, dimension( ldb, * ) B,
       integer LDB, complex*16, dimension( ldq, * ) Q, integer LDQ,
       complex*16, dimension( ldz, * ) Z, integer LDZ, integer J1, integer
       INFO)
       ZTGEX2 swaps adjacent diagonal blocks in an upper (quasi) triangular
       matrix pair by an unitary equivalence transformation.

Purpose:
   ZTGEX2 swaps adjacent diagonal 1 by 1 blocks (A11,B11) and (A22,B22)
   in an upper triangular matrix pair (A, B) by an unitary equivalence
   transformation.

   (A, B) must be in generalized Schur canonical form, that is, A and
   B are both upper triangular.

   Optionally, the matrices Q and Z of generalized Schur vectors are
   updated.

   Q(in) * A(in) * Z(in)**H = Q(out) * A(out) * Z(out)**H
   Q(in) * B(in) * Z(in)**H = Q(out) * B(out) * Z(out)**H

Parameters:

 WANTQ
   WANTQ is LOGICAL
   .TRUE. : update the left transformation matrix Q;
   .FALSE.: do not update Q.

 WANTZ
   WANTZ is LOGICAL
   .TRUE. : update the right transformation matrix Z;
   .FALSE.: do not update Z.

 N
   N is INTEGER
   The order of the matrices A and B. N >= 0.

 A
   A is COMPLEX*16 arrays, dimensions (LDA,N)
   On entry, the matrix A in the pair (A, B).
   On exit, the updated matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A. LDA >= max(1,N).

 B
   B is COMPLEX*16 arrays, dimensions (LDB,N)
```

```
   On entry, the matrix B in the pair (A, B).
   On exit, the updated matrix B.

 LDB
   LDB is INTEGER
   The leading dimension of the array B. LDB >= max(1,N).

 Q
   Q is COMPLEX*16 array, dimension (LDZ,N)
   If WANTQ = .TRUE, on entry, the unitary matrix Q. On exit,
   the updated matrix Q.
   Not referenced if WANTQ = .FALSE..

 LDQ
   LDQ is INTEGER
   The leading dimension of the array Q. LDQ >= 1;
   If WANTQ = .TRUE., LDQ >= N.

 Z
   Z is COMPLEX*16 array, dimension (LDZ,N)
   If WANTZ = .TRUE, on entry, the unitary matrix Z. On exit,
   the updated matrix Z.
   Not referenced if WANTZ = .FALSE..

 LDZ
   LDZ is INTEGER
   The leading dimension of the array Z. LDZ >= 1;
   If WANTZ = .TRUE., LDZ >= N.

 J1
   J1 is INTEGER
   The index to the first block (A11, B11).

 INFO
   INFO is INTEGER
   =0:  Successful exit.
   =1:  The transformed matrix pair (A, B) would be too far
        from generalized Schur form; the problem is ill-conditioned.

Further Details:

   In the current code both weak and strong stability tests are
   performed. The user can omit the strong stability test by changing
   the internal logical parameter WANDS to .FALSE.. See ref. [2] for details.
```

———-

39.5.3 fortran code

— ztgex2.f —

```
*  =====================================================================
      SUBROUTINE ZTGEX2( WANTQ, WANTZ, N, A, LDA, B, LDB, Q, LDQ, Z,
     $                   LDZ, J1, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      LOGICAL            WANTQ, WANTZ
      INTEGER            INFO, J1, LDA, LDB, LDQ, LDZ, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), B( LDB, * ), Q( LDQ, * ),
     $                   Z( LDZ, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0D+0, 0.0D+0 ),
     $                   CONE = ( 1.0D+0, 0.0D+0 ) )
      DOUBLE PRECISION   TWENTY
      PARAMETER          ( TWENTY = 2.0D+1 )
      INTEGER            LDST
      PARAMETER          ( LDST = 2 )
      LOGICAL            WANDS
      PARAMETER          ( WANDS = .TRUE. )
*     ..
*     .. Local Scalars ..
      LOGICAL            DTRONG, WEAK
      INTEGER            I, M
      DOUBLE PRECISION   CQ, CZ, EPS, SA, SB, SCALE, SMLNUM, SS, SUM,
     $                   THRESH, WS
      COMPLEX*16         CDUM, F, G, SQ, SZ
*     ..
*     .. Local Arrays ..
      COMPLEX*16         S( LDST, LDST ), T( LDST, LDST ), WORK( 8 )
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
```

```
*     .. External Subroutines ..
      EXTERNAL           ZLACPY, ZLARTG, ZLASSQ, ZROT
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DCONJG, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
*     Quick return if possible
*
      IF( N.LE.1 )
     $   RETURN
*
      M = LDST
      WEAK = .FALSE.
      DTRONG = .FALSE.
*
*     Make a local copy of selected block in (A, B)
*
      CALL ZLACPY( 'Full', M, M, A( J1, J1 ), LDA, S, LDST )
      CALL ZLACPY( 'Full', M, M, B( J1, J1 ), LDB, T, LDST )
*
*     Compute the threshold for testing the acceptance of swapping.
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' ) / EPS
      SCALE = DBLE( CZERO )
      SUM = DBLE( CONE )
      CALL ZLACPY( 'Full', M, M, S, LDST, WORK, M )
      CALL ZLACPY( 'Full', M, M, T, LDST, WORK( M*M+1 ), M )
      CALL ZLASSQ( 2*M*M, WORK, 1, SCALE, SUM )
      SA = SCALE*SQRT( SUM )
*
*     THRES has been changed from
*        THRESH = MAX( TEN*EPS*SA, SMLNUM )
*     to
*        THRESH = MAX( TWENTY*EPS*SA, SMLNUM )
*     on 04/01/10.
*     "Bug" reported by Ondra Kamenik, confirmed by Julie Langou, fixed by
*     Jim Demmel and Guillaume Revy. See forum post 1783.
*
      THRESH = MAX( TWENTY*EPS*SA, SMLNUM )
*
*     Compute unitary QL and RQ that swap 1-by-1 and 1-by-1 blocks
*     using Givens rotations and perform the swap tentatively.
*
      F = S( 2, 2 )*T( 1, 1 ) - T( 2, 2 )*S( 1, 1 )
      G = S( 2, 2 )*T( 1, 2 ) - T( 2, 2 )*S( 1, 2 )
```

```
      SA = ABS( S( 2, 2 ) )
      SB = ABS( T( 2, 2 ) )
      CALL ZLARTG( G, F, CZ, SZ, CDUM )
      SZ = -SZ
      CALL ZROT( 2, S( 1, 1 ), 1, S( 1, 2 ), 1, CZ, DCONJG( SZ ) )
      CALL ZROT( 2, T( 1, 1 ), 1, T( 1, 2 ), 1, CZ, DCONJG( SZ ) )
      IF( SA.GE.SB ) THEN
         CALL ZLARTG( S( 1, 1 ), S( 2, 1 ), CQ, SQ, CDUM )
      ELSE
         CALL ZLARTG( T( 1, 1 ), T( 2, 1 ), CQ, SQ, CDUM )
      END IF
      CALL ZROT( 2, S( 1, 1 ), LDST, S( 2, 1 ), LDST, CQ, SQ )
      CALL ZROT( 2, T( 1, 1 ), LDST, T( 2, 1 ), LDST, CQ, SQ )
*
*     Weak stability test: |S21| + |T21| <= O(EPS F-norm((S, T)))
*
      WS = ABS( S( 2, 1 ) ) + ABS( T( 2, 1 ) )
      WEAK = WS.LE.THRESH
      IF( .NOT.WEAK )
     $   GO TO 20
*
      IF( WANDS ) THEN
*
*        Strong stability test:
*           F-norm((A-QL**H*S*QR, B-QL**H*T*QR)) <= O(EPS*F-norm((A, B)))
*
         CALL ZLACPY( 'Full', M, M, S, LDST, WORK, M )
         CALL ZLACPY( 'Full', M, M, T, LDST, WORK( M*M+1 ), M )
         CALL ZROT( 2, WORK, 1, WORK( 3 ), 1, CZ, -DCONJG( SZ ) )
         CALL ZROT( 2, WORK( 5 ), 1, WORK( 7 ), 1, CZ, -DCONJG( SZ ) )
         CALL ZROT( 2, WORK, 2, WORK( 2 ), 2, CQ, -SQ )
         CALL ZROT( 2, WORK( 5 ), 2, WORK( 6 ), 2, CQ, -SQ )
         DO 10 I = 1, 2
            WORK( I ) = WORK( I ) - A( J1+I-1, J1 )
            WORK( I+2 ) = WORK( I+2 ) - A( J1+I-1, J1+1 )
            WORK( I+4 ) = WORK( I+4 ) - B( J1+I-1, J1 )
            WORK( I+6 ) = WORK( I+6 ) - B( J1+I-1, J1+1 )
   10    CONTINUE
         SCALE = DBLE( CZERO )
         SUM = DBLE( CONE )
         CALL ZLASSQ( 2*M*M, WORK, 1, SCALE, SUM )
         SS = SCALE*SQRT( SUM )
         DTRONG = SS.LE.THRESH
         IF( .NOT.DTRONG )
     $      GO TO 20
      END IF
*
*     If the swap is accepted ("weakly" and "strongly"), apply the
*     equivalence transformations to the original matrix pair (A,B)
*
```

```
      CALL ZROT( J1+1, A( 1, J1 ), 1, A( 1, J1+1 ), 1, CZ,
     $           DCONJG( SZ ) )
      CALL ZROT( J1+1, B( 1, J1 ), 1, B( 1, J1+1 ), 1, CZ,
     $           DCONJG( SZ ) )
      CALL ZROT( N-J1+1, A( J1, J1 ), LDA, A( J1+1, J1 ), LDA, CQ, SQ )
      CALL ZROT( N-J1+1, B( J1, J1 ), LDB, B( J1+1, J1 ), LDB, CQ, SQ )
*
*     Set  N1 by N2 (2,1) blocks to 0
*
      A( J1+1, J1 ) = CZERO
      B( J1+1, J1 ) = CZERO
*
*     Accumulate transformations into Q and Z if requested.
*
      IF( WANTZ )
     $   CALL ZROT( N, Z( 1, J1 ), 1, Z( 1, J1+1 ), 1, CZ,
     $              DCONJG( SZ ) )
      IF( WANTQ )
     $   CALL ZROT( N, Q( 1, J1 ), 1, Q( 1, J1+1 ), 1, CQ,
     $              DCONJG( SQ ) )
*
*     Exit with INFO = 0 if swap was successfully performed.
*
      RETURN
*
*     Exit with INFO = 1 if swap was rejected.
*
   20 CONTINUE
      INFO = 1
      RETURN
*
*     End of ZTGEX2
*
      END
```

———-

39.5.4 lisp code

— **ztgex2.lisp** —

```
(let* ((czero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (cone (f2cl-lib:cmplx 1.0d0 0.0d0))
       (twenty 20.0d0)
       (ldst 2)
       (wands f2cl-lib:%true%))
  (declare (type (f2cl-lib:complex16) czero)
           (type (f2cl-lib:complex16) cone)
```

```
          (type (double-float 20.0d0 20.0d0) twenty)
          (type (f2cl-lib:integer4 2 2) ldst)
          (type f2cl-lib:logical wands)
          (ignorable czero cone twenty ldst wands))
 (defun ztgex2 (wantq wantz n a lda b ldb$ q ldq z ldz j1 info)
   (declare (type (array f2cl-lib:complex16 (*)) z q b a)
            (type (f2cl-lib:integer4) info j1 ldz ldq ldb$ lda n)
            (type f2cl-lib:logical wantz wantq))
   (f2cl-lib:with-multi-array-data
       ((a f2cl-lib:complex16 a-%data% a-%offset%)
        (b f2cl-lib:complex16 b-%data% b-%offset%)
        (q f2cl-lib:complex16 q-%data% q-%offset%)
        (z f2cl-lib:complex16 z-%data% z-%offset%))
     (prog ((s
             (make-array (the fixnum (reduce #'* (list ldst ldst)))
                         :element-type 'f2cl-lib:complex16))
            (work (make-array 8 :element-type 'f2cl-lib:complex16))
            (cdum #C(0.0d0 0.0d0)) (f #C(0.0d0 0.0d0)) (g #C(0.0d0 0.0d0))
            (sq #C(0.0d0 0.0d0)) (sz #C(0.0d0 0.0d0)) (cq 0.0d0) (cz 0.0d0)
            (eps 0.0d0) (sa 0.0d0) (sb 0.0d0) (scale 0.0d0) (smlnum 0.0d0)
            (ss 0.0d0) (sum 0.0d0) (thresh 0.0d0) (ws 0.0d0) (i 0) (m 0)
            (dtrong nil) (weak nil)
            (t$
             (make-array (the fixnum (reduce #'* (list ldst ldst)))
                         :element-type 'f2cl-lib:complex16)))
       (declare (type (array f2cl-lib:complex16 (*)) t$ s)
                (type (array f2cl-lib:complex16 (8)) work)
                (type (f2cl-lib:complex16) cdum f g sq sz)
                (type (double-float) cq cz eps sa sb scale smlnum ss sum
                 thresh ws)
                (type (f2cl-lib:integer4) i m)
                (type f2cl-lib:logical dtrong weak))
       (setf info 0)
       (if (<= n 1)
           (go end_label))
       (setf m ldst)
       (setf weak f2cl-lib:%false%)
       (setf dtrong f2cl-lib:%false%)
       (zlacpy "Full" m m
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (j1 j1)
                              ((1 lda) (1 *)) a-%offset%)
        lda s ldst)
       (zlacpy "Full" m m
        (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (j1 j1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$ t$ ldst)
       (setf eps (dlamch "P"))
       (setf smlnum (/ (dlamch "S") eps))
       (setf scale (f2cl-lib:dble czero))
       (setf sum (f2cl-lib:dble cone))
```

```
(zlacpy "Full" m m s ldst work m)
(zlacpy "Full" m m t$ ldst
 (f2cl-lib:array-slice work f2cl-lib:complex16
                       ((+ (f2cl-lib:int-mul m m) 1)) ((1 8)))
 m)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (zlassq (f2cl-lib:int-mul 2 m m) work 1 scale sum)
  (declare (ignore var-0 var-1 var-2))
  (setf scale var-3)
  (setf sum var-4))
(setf sa (* scale (f2cl-lib:fsqrt sum)))
(setf thresh (max (* twenty eps sa) smlnum))
(setf f
        (-
         (* (f2cl-lib:fref s (2 2) ((1 ldst) (1 ldst)))
            (f2cl-lib:fref t$ (1 1) ((1 ldst) (1 ldst))))
         (* (f2cl-lib:fref t$ (2 2) ((1 ldst) (1 ldst)))
            (f2cl-lib:fref s (1 1) ((1 ldst) (1 ldst))))))
(setf g
        (-
         (* (f2cl-lib:fref s (2 2) ((1 ldst) (1 ldst)))
            (f2cl-lib:fref t$ (1 2) ((1 ldst) (1 ldst))))
         (* (f2cl-lib:fref t$ (2 2) ((1 ldst) (1 ldst)))
            (f2cl-lib:fref s (1 2) ((1 ldst) (1 ldst))))))
(setf sa (abs (f2cl-lib:fref s (2 2) ((1 ldst) (1 ldst)))))
(setf sb (abs (f2cl-lib:fref t$ (2 2) ((1 ldst) (1 ldst)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (zlartg g f cz sz cdum)
  (declare (ignore var-0 var-1))
  (setf cz var-2)
  (setf sz var-3)
  (setf cdum var-4))
(setf sz (- sz))
(zrot 2
 (f2cl-lib:array-slice s f2cl-lib:complex16 (1 1) ((1 ldst) (1 ldst)))
 1
 (f2cl-lib:array-slice s f2cl-lib:complex16 (1 2) ((1 ldst) (1 ldst)))
 1 cz (f2cl-lib:dconjg sz))
(zrot 2
 (f2cl-lib:array-slice t$ f2cl-lib:complex16 (1 1) ((1 ldst) (1 ldst)))
 1
 (f2cl-lib:array-slice t$ f2cl-lib:complex16 (1 2) ((1 ldst) (1 ldst)))
 1 cz (f2cl-lib:dconjg sz))
(cond
 ((>= sa sb)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (zlartg (f2cl-lib:fref s (1 1) ((1 ldst) (1 ldst)))
       (f2cl-lib:fref s (2 1) ((1 ldst) (1 ldst))) cq sq cdum)
    (declare (ignore var-0 var-1))
    (setf cq var-2)
```

```
    (setf sq var-3)
    (setf cdum var-4)))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (zlartg (f2cl-lib:fref t$ (1 1) ((1 ldst) (1 ldst)))
       (f2cl-lib:fref t$ (2 1) ((1 ldst) (1 ldst))) cq sq cdum)
    (declare (ignore var-0 var-1))
    (setf cq var-2)
    (setf sq var-3)
    (setf cdum var-4))))
(zrot 2
 (f2cl-lib:array-slice s f2cl-lib:complex16 (1 1) ((1 ldst) (1 ldst)))
 ldst
 (f2cl-lib:array-slice s f2cl-lib:complex16 (2 1) ((1 ldst) (1 ldst)))
 ldst cq sq)
(zrot 2
 (f2cl-lib:array-slice t$ f2cl-lib:complex16 (1 1) ((1 ldst) (1 ldst)))
 ldst
 (f2cl-lib:array-slice t$ f2cl-lib:complex16 (2 1) ((1 ldst) (1 ldst)))
 ldst cq sq)
(setf ws
        (+ (abs (f2cl-lib:fref s (2 1) ((1 ldst) (1 ldst))))
           (abs (f2cl-lib:fref t$ (2 1) ((1 ldst) (1 ldst))))))
(setf weak (<= ws thresh))
(if (not weak)
    (go label20))
(cond
 (wands (zlacpy "Full" m m s ldst work m)
  (zlacpy "Full" m m t$ ldst
   (f2cl-lib:array-slice work f2cl-lib:complex16
                         ((+ (f2cl-lib:int-mul m m) 1)) ((1 8)))
   m)
  (zrot 2 work 1
   (f2cl-lib:array-slice work f2cl-lib:complex16 (3) ((1 8))) 1 cz
   (- (f2cl-lib:dconjg sz)))
  (zrot 2 (f2cl-lib:array-slice work f2cl-lib:complex16 (5) ((1 8))) 1
   (f2cl-lib:array-slice work f2cl-lib:complex16 (7) ((1 8))) 1 cz
   (- (f2cl-lib:dconjg sz)))
  (zrot 2 work 2
   (f2cl-lib:array-slice work f2cl-lib:complex16 (2) ((1 8))) 2 cq
   (- sq))
  (zrot 2 (f2cl-lib:array-slice work f2cl-lib:complex16 (5) ((1 8))) 2
   (f2cl-lib:array-slice work f2cl-lib:complex16 (6) ((1 8))) 2 cq
   (- sq))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i 2) nil)
    (tagbody
      (setf (f2cl-lib:fref work (i) ((1 8)))
              (- (f2cl-lib:fref work (i) ((1 8)))
                 (f2cl-lib:fref a-%data%
```

```
                                  ((f2cl-lib:int-sub
                                    (f2cl-lib:int-add j1 i) 1)
                                   j1)
                                  ((1 lda) (1 *)) a-%offset%)))
        (setf (f2cl-lib:fref work ((f2cl-lib:int-add i 2)) ((1 8)))
                (- (f2cl-lib:fref work ((f2cl-lib:int-add i 2)) ((1 8)))
                   (f2cl-lib:fref a-%data%
                                  ((f2cl-lib:int-sub
                                    (f2cl-lib:int-add j1 i) 1)
                                   (f2cl-lib:int-add j1 1))
                                  ((1 lda) (1 *)) a-%offset%)))
        (setf (f2cl-lib:fref work ((f2cl-lib:int-add i 4)) ((1 8)))
                (- (f2cl-lib:fref work ((f2cl-lib:int-add i 4)) ((1 8)))
                   (f2cl-lib:fref b-%data%
                                  ((f2cl-lib:int-sub
                                    (f2cl-lib:int-add j1 i) 1)
                                   j1)
                                  ((1 ldb$) (1 *)) b-%offset%)))
        (setf (f2cl-lib:fref work ((f2cl-lib:int-add i 6)) ((1 8)))
                (- (f2cl-lib:fref work ((f2cl-lib:int-add i 6)) ((1 8)))
                   (f2cl-lib:fref b-%data%
                                  ((f2cl-lib:int-sub
                                    (f2cl-lib:int-add j1 i) 1)
                                   (f2cl-lib:int-add j1 1))
                                  ((1 ldb$) (1 *)) b-%offset%)))
      label10))
    (setf scale (f2cl-lib:dble czero)) (setf sum (f2cl-lib:dble cone))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zlassq (f2cl-lib:int-mul 2 m m) work 1 scale sum)
      (declare (ignore var-0 var-1 var-2))
      (setf scale var-3)
      (setf sum var-4))
    (setf ss (* scale (f2cl-lib:fsqrt sum))) (setf dtrong (<= ss thresh))
    (if (not dtrong)
        (go label20))))
  (zrot (f2cl-lib:int-add j1 1)
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 j1)
                         ((1 lda) (1 *)) a-%offset%)
   1
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                         (1 (f2cl-lib:int-add j1 1)) ((1 lda) (1 *))
                         a-%offset%)
   1 cz (f2cl-lib:dconjg sz))
  (zrot (f2cl-lib:int-add j1 1)
   (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 j1)
                         ((1 ldb$) (1 *)) b-%offset%)
   1
   (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                         (1 (f2cl-lib:int-add j1 1)) ((1 ldb$) (1 *))
                         b-%offset%)
```

```
         1 cz (f2cl-lib:dconjg sz))
        (zrot (f2cl-lib:int-add (f2cl-lib:int-sub n j1) 1)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (j1 j1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 ((+ j1 1) j1)
                               ((1 lda) (1 *)) a-%offset%)
         lda cq sq)
        (zrot (f2cl-lib:int-add (f2cl-lib:int-sub n j1) 1)
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (j1 j1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 ((+ j1 1) j1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$ cq sq)
        (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add j1 1) j1)
                             ((1 lda) (1 *)) a-%offset%)
                czero)
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add j1 1) j1)
                             ((1 ldb$) (1 *)) b-%offset%)
                czero)
        (if wantz
            (zrot n
             (f2cl-lib:array-slice z-%data% f2cl-lib:complex16 (1 j1)
                                   ((1 ldz) (1 *)) z-%offset%)
             1
             (f2cl-lib:array-slice z-%data% f2cl-lib:complex16
                                   (1 (f2cl-lib:int-add j1 1)) ((1 ldz) (1 *))
                                   z-%offset%)
             1 cz (f2cl-lib:dconjg sz)))
        (if wantq
            (zrot n
             (f2cl-lib:array-slice q-%data% f2cl-lib:complex16 (1 j1)
                                   ((1 ldq) (1 *)) q-%offset%)
             1
             (f2cl-lib:array-slice q-%data% f2cl-lib:complex16
                                   (1 (f2cl-lib:int-add j1 1)) ((1 ldq) (1 *))
                                   q-%offset%)
             1 cq (f2cl-lib:dconjg sq)))
        (go end_label)
       label20
        (setf info 1)
        (go end_label)
       end_label
        (return
         (values nil nil nil nil nil nil nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
```

```
(setf (gethash 'fortran-to-lisp::ztgex2
               fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '(fortran-to-lisp::logical
                                            fortran-to-lisp::logical
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex16
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex16
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex16
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex16
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4))
                                          :return-values
                                          '(nil nil nil nil nil nil nil nil
                                            nil nil nil nil
                                            fortran-to-lisp::info)
                                          :calls
                                          '(fortran-to-lisp::zrot
                                            fortran-to-lisp::zlartg
                                            fortran-to-lisp::zlassq
                                            fortran-to-lisp::zlacpy))))
```

Chapter 40

LAPACK - General Band Matrix, Linear Solve, Double

40.1 dgbsv

40.1.1 Axiom unit tests

— dgbsv.input —

```
)set break resume
)sys rm -f dgbsv.output
)spool dgbsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

40.1.2 Axiom help page

— dgbsv.help —

```
======================================================================
dgbsv examples
======================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgbsv (N, KL, KU, NRHS, AB, LDAB, IPIV, B, LDB, INFO)
            DGBSV computes the solution to system of linear equations A * X =
           B for GB matrices (simple driver)

Function/Subroutine Documentation
   subroutine dgbsv (integer N, integer KL, integer KU, integer NRHS, double
       precision, dimension( ldab, * ) AB, integer LDAB, integer, dimension( *
       ) IPIV, double precision, dimension( ldb, * ) B, integer LDB, integer
       INFO)
        DGBSV computes the solution to system of linear equations A * X = B
       for GB matrices (simple driver)

Purpose:
   DGBSV computes the solution to a real system of linear equations
   A * X = B, where A is a band matrix of order N with KL subdiagonals
   and KU superdiagonals, and X and B are N-by-NRHS matrices.

   The LU decomposition with partial pivoting and row interchanges is
   used to factor A as A = L * U, where L is a product of permutation
   and unit lower triangular matrices with KL subdiagonals, and U is
   upper triangular with KL+KU superdiagonals.  The factored form of A
   is then used to solve the system of equations A * X = B.

Parameters:

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 AB
```

```
  AB is DOUBLE PRECISION array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows KL+1 to
  2*KL+KU+1; rows 1 to KL of the array need not be set.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(KL+KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+KL)
  On exit, details of the factorization: U is stored as an
  upper triangular band matrix with KL+KU superdiagonals in
  rows 1 to KL+KU+1, and the multipliers used during the
  factorization are stored in rows KL+KU+2 to 2*KL+KU+1.
  See below for further details.

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices that define the permutation matrix P;
  row i of the matrix was interchanged with row IPIV(i).

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, U(i,i) is exactly zero.  The factorization
        has been completed, but the factor U is exactly
        singular, and the solution has not been computed.

Further Details:

  The band storage scheme is illustrated by the following example, when
  M = N = 6, KL = 2, KU = 1:

       On entry:                       On exit:

           *    *    *    +    +    +       *    *    *   u14  u25  u36
           *    *    +    +    +    +       *    *   u13  u24  u35  u46
           *   a12  a23  a34  a45  a56       *   u12  u23  u34  u45  u56
          a11  a22  a33  a44  a55  a66     u11  u22  u33  u44  u55  u66
          a21  a32  a43  a54  a65   *      m21  m32  m43  m54  m65   *
```

```
          a31  a42  a53  a64   *    *      m31  m42  m53  m64   *    *

   Array elements marked * are not used by the routine; elements marked
   + need not be set on entry, but are required by the routine to store
   elements of U because of fill-in resulting from the row interchanges.
```

———-

40.1.3 fortran code

— dgbsv.f —

```
*  =====================================================================
      SUBROUTINE DGBSV( N, KL, KU, NRHS, AB, LDAB, IPIV, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, KL, KU, LDAB, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   AB( LDAB, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. External Subroutines ..
      EXTERNAL           DGBTRF, DGBTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( KL.LT.0 ) THEN
         INFO = -2
      ELSE IF( KU.LT.0 ) THEN
         INFO = -3
```

```
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.2*KL+KU+1 ) THEN
         INFO = -6
      ELSE IF( LDB.LT.MAX( N, 1 ) ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGBSV ', -INFO )
         RETURN
      END IF
*
*     Compute the LU factorization of the band matrix A.
*
      CALL DGBTRF( N, N, KL, KU, AB, LDAB, IPIV, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
         CALL DGBTRS( 'No transpose', N, KL, KU, NRHS, AB, LDAB, IPIV,
     $                B, LDB, INFO )
      END IF
      RETURN
*
*     End of DGBSV
*
      END
```

———-

40.1.4 lisp code

— dgbsv.lisp —

```
(defun dgbsv (n kl ku nrhs ab ldab ipiv b ldb$ info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array double-float (*)) b ab)
           (type (f2cl-lib:integer4) info ldb$ ldab nrhs ku kl n))
  (f2cl-lib:with-multi-array-data
      ((ab double-float ab-%data% ab-%offset%)
       (b double-float b-%data% b-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ()
      (declare)
      (setf info 0)
      (cond ((< n 0) (setf info -1)) ((< kl 0) (setf info -2))
            ((< ku 0) (setf info -3)) ((< nrhs 0) (setf info -4))
            ((< ldab (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
```

```
            (setf info -6))
           ((< ldb$ (max (the f2cl-lib:integer4 n) (the f2cl-lib:integer4 1)))
            (setf info -9)))
      (cond
       ((/= info 0) (xerbla "DGBSV " (f2cl-lib:int-sub info)) (go end_label)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (dgbtrf n n kl ku ab ldab ipiv info)
        (declare (ignore var-4 var-6))
        (when var-0 (setf n var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf kl var-2))
        (when var-3 (setf ku var-3))
        (when var-5 (setf ldab var-5))
        (when var-7 (setf info var-7)))
      (cond
       ((= info 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10)
            (dgbtrs "No transpose" n kl ku nrhs ab ldab ipiv b ldb$ info)
          (declare (ignore var-0 var-5 var-7 var-8))
          (when var-1 (setf n var-1))
          (when var-2 (setf kl var-2))
          (when var-3 (setf ku var-3))
          (when var-4 (setf nrhs var-4))
          (when var-6 (setf ldab var-6))
          (when var-9 (setf ldb$ var-9))
          (when var-10 (setf info var-10)))))
      (go end_label)
     end_label
      (return (values n kl ku nrhs nil ldab nil nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgbsv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n
```

```
                                                  fortran-to-lisp::kl
                                                  fortran-to-lisp::ku
                                                  fortran-to-lisp::nrhs nil
                                                  fortran-to-lisp::ldab nil nil
                                                  fortran-to-lisp::ldb$
                                                  fortran-to-lisp::info)
                                                :calls 'nil)))
```

———-

40.2 dgbsvx

40.2.1 Axiom unit tests

— **dgbsvx.input** —

```
)set break resume
)sys rm -f dgbsvx.output
)spool dgbsvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

40.2.2 Axiom help page

— **dgbsvx.help** —

```
====================================================================
dgbsvx examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
```

```
      subroutine dgbsvx (FACT, TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB,
          IPIV, EQUED, R, C, B, LDB, X, LDX, RCOND, FERR, BERR, WORK, IWORK,
          INFO)
           DGBSVX computes the solution to system of linear equations A * X =
          B for GB matrices

Function/Subroutine Documentation
   subroutine dgbsvx (character FACT, character TRANS, integer N, integer KL,
       integer KU, integer NRHS, double precision, dimension( ldab, * ) AB,
       integer LDAB, double precision, dimension( ldafb, * ) AFB, integer
       LDAFB, integer, dimension( * ) IPIV, character EQUED, double precision,
       dimension( * ) R, double precision, dimension( * ) C, double precision,
       dimension( ldb, * ) B, integer LDB, double precision, dimension( ldx, *
       ) X, integer LDX, double precision RCOND, double precision, dimension(
       * ) FERR, double precision, dimension( * ) BERR, double precision,
       dimension( * ) WORK, integer, dimension( * ) IWORK, integer INFO)
        DGBSVX computes the solution to system of linear equations A * X = B
       for GB matrices

Purpose:
   DGBSVX uses the LU factorization to compute the solution to a real
   system of linear equations A * X = B, A**T * X = B, or A**H * X = B,
   where A is a band matrix of order N with KL subdiagonals and KU
   superdiagonals, and X and B are N-by-NRHS matrices.

   Error bounds on the solution and a condition estimate are also
   provided.

Description:
   The following steps are performed by this subroutine:

   1. If FACT = 'E', real scaling factors are computed to equilibrate
      the system:
      TRANS = 'N':  diag(R)*A*diag(C)     *inv(diag(C))*X = diag(R)*B
      TRANS = 'T': (diag(R)*A*diag(C))**T *inv(diag(R))*X = diag(C)*B
      TRANS = 'C': (diag(R)*A*diag(C))**H *inv(diag(R))*X = diag(C)*B
      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(R)*A*diag(C) and B by diag(R)*B (if TRANS='N')
      or diag(C)*B (if TRANS = 'T' or 'C').

   2. If FACT = 'N' or 'E', the LU decomposition is used to factor the
      matrix A (after equilibration if FACT = 'E') as
         A = L * U,
      where L is a product of permutation and unit lower triangular
      matrices with KL subdiagonals, and U is upper triangular with
      KL+KU superdiagonals.

   3. If some U(i,i)=0, so that U is exactly singular, then the routine
      returns with INFO = i. Otherwise, the factored form of A is used
```

```
      to estimate the condition number of the matrix A.  If the
      reciprocal of the condition number is less than machine precision,
      INFO = N+1 is returned as a warning, but the routine still goes on
      to solve for X and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(C) (if TRANS = 'N') or diag(R) (if TRANS = 'T' or 'C') so
      that it solves the original system before equilibration.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AFB and IPIV contain the factored form of
           A.  If EQUED is not 'N', the matrix A has been
           equilibrated with scaling factors given by R and C.
           AB, AFB, and IPIV are not modified.
   = 'N':  The matrix A will be copied to AFB and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
           copied to AFB and factored.

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations.
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Transpose)

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 NRHS
   NRHS is INTEGER
```

```
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

AB
  AB is DOUBLE PRECISION array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

  If FACT = 'F' and EQUED is not 'N', then A must have been
  equilibrated by the scaling factors in R and/or C.  AB is not
  modified if FACT = 'F' or 'N', or if FACT = 'E' and
  EQUED = 'N' on exit.

  On exit, if EQUED .ne. 'N', A is scaled as follows:
  EQUED = 'R':  A := diag(R) * A
  EQUED = 'C':  A := A * diag(C)
  EQUED = 'B':  A := diag(R) * A * diag(C).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is DOUBLE PRECISION array, dimension (LDAFB,N)
  If FACT = 'F', then AFB is an input argument and on entry
  contains details of the LU factorization of the band matrix
  A, as computed by DGBTRF.  U is stored as an upper triangular
  band matrix with KL+KU superdiagonals in rows 1 to KL+KU+1,
  and the multipliers used during the factorization are stored
  in rows KL+KU+2 to 2*KL+KU+1.  If EQUED .ne. 'N', then AFB is
  the factored form of the equilibrated matrix A.

  If FACT = 'N', then AFB is an output argument and on exit
  returns details of the LU factorization of A.

  If FACT = 'E', then AFB is an output argument and on exit
  returns details of the LU factorization of the equilibrated
  matrix A (see the description of AB for the form of the
  equilibrated matrix).

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains the pivot indices from the factorization A = L*U
```

```
  as computed by DGBTRF; row i of the matrix was interchanged
  with row IPIV(i).

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = L*U
  of the original matrix A.

  If FACT = 'E', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = L*U
  of the equilibrated matrix A.

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
  = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
  = 'B':  Both row and column equilibration, i.e., A has been
          replaced by diag(R) * A * diag(C).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

R
  R is DOUBLE PRECISION array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
  EQUED = 'R' or 'B', each element of R must be positive.

C
  C is DOUBLE PRECISION array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if TRANS = 'N' and EQUED = 'R' or 'B', B is overwritten by
  diag(R)*B;
  if TRANS = 'T' or 'C' and EQUED = 'C' or 'B', B is
  overwritten by diag(C)*B.

LDB
  LDB is INTEGER
```

```
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X
  to the original system of equations.  Note that A and B are
  modified on exit if EQUED .ne. 'N', and the solution to the
  equilibrated system is inv(diag(C))*X if TRANS = 'N' and
  EQUED = 'C' or 'B', or inv(diag(R))*X if TRANS = 'T' or 'C'
  and EQUED = 'R' or 'B'.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  The estimate of the reciprocal condition number of the matrix
  A after equilibration (if done).  If RCOND is less than the
  machine precision (in particular, if RCOND = 0), the matrix
  is singular to working precision.  This condition is
  indicated by a return code of INFO > 0.

FERR
  FERR is DOUBLE PRECISION array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is DOUBLE PRECISION array, dimension (3*N)
  On exit, WORK(1) contains the reciprocal pivot growth
  factor norm(A)/norm(U). The "max absolute element" norm is
  used. If WORK(1) is much less than 1, then the stability
  of the LU factorization of the (equilibrated) matrix A
  could be poor. This also means that the solution X, condition
  estimator RCOND, and forward error bound FERR could be
  unreliable. If factorization fails with 0<INFO<=N, then
  WORK(1) contains the reciprocal pivot growth factor for the
```

```
  leading INFO columns of A.

 IWORK
  IWORK is INTEGER array, dimension (N)

 INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, and i is
  <= N:  U(i,i) is exactly zero.  The factorization
         has been completed, but the factor U is exactly
         singular, so the solution and error bounds
         could not be computed. RCOND = 0 is returned.
  = N+1: U is nonsingular, but RCOND is less than machine
         precision, meaning that the matrix is singular
         to working precision.  Nevertheless, the
         solution and error bounds are computed because
         there are a number of situations where the
         computed solution can be more accurate than the
         value of RCOND would suggest.
```

———-

40.2.3 fortran code

— dgbsvx.f —

```
*  =====================================================================
      SUBROUTINE DGBSVX( FACT, TRANS, N, KL, KU, NRHS, AB, LDAB, AFB,
     $                   LDAFB, IPIV, EQUED, R, C, B, LDB, X, LDX,
     $                   RCOND, FERR, BERR, WORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, TRANS
      INTEGER            INFO, KL, KU, LDAB, LDAFB, LDB, LDX, N, NRHS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      DOUBLE PRECISION   AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   BERR( * ), C( * ), FERR( * ), R( * ),
```

```
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            COLEQU, EQUIL, NOFACT, NOTRAN, ROWEQU
      CHARACTER          NORM
      INTEGER            I, INFEQU, J, J1, J2
      DOUBLE PRECISION   AMAX, ANORM, BIGNUM, COLCND, RCMAX, RCMIN,
     $                   ROWCND, RPVGRW, SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, DLANGB, DLANTB
      EXTERNAL           LSAME, DLAMCH, DLANGB, DLANTB
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DGBCON, DGBEQU, DGBRFS, DGBTRF, DGBTRS,
     $                   DLACPY, DLAQGB, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      NOTRAN = LSAME( TRANS, 'N' )
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         ROWEQU = .FALSE.
         COLEQU = .FALSE.
      ELSE
         ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
         COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         SMLNUM = DLAMCH( 'Safe minimum' )
         BIGNUM = ONE / SMLNUM
      END IF
*
*     Test the input parameters.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.LSAME( FACT, 'F' ) )
     $     THEN
         INFO = -1
```

```
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $         LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( KL.LT.0 ) THEN
         INFO = -4
      ELSE IF( KU.LT.0 ) THEN
         INFO = -5
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -6
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -8
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
         INFO = -10
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $         ( ROWEQU .OR. COLEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -12
      ELSE
         IF( ROWEQU ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 10 J = 1, N
               RCMIN = MIN( RCMIN, R( J ) )
               RCMAX = MAX( RCMAX, R( J ) )
10          CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -13
            ELSE IF( N.GT.0 ) THEN
               ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               ROWCND = ONE
            END IF
         END IF
         IF( COLEQU .AND. INFO.EQ.0 ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 20 J = 1, N
               RCMIN = MIN( RCMIN, C( J ) )
               RCMAX = MAX( RCMAX, C( J ) )
20          CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -14
            ELSE IF( N.GT.0 ) THEN
               COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               COLCND = ONE
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
```

```
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -16
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -18
            END IF
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGBSVX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*        Compute row and column scalings to equilibrate the matrix A.
*
         CALL DGBEQU( N, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $                AMAX, INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*           Equilibrate the matrix.
*
            CALL DLAQGB( N, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $                   AMAX, EQUED )
            ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
            COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         END IF
      END IF
*
*     Scale the right hand side.
*
      IF( NOTRAN ) THEN
         IF( ROWEQU ) THEN
            DO 40 J = 1, NRHS
               DO 30 I = 1, N
                  B( I, J ) = R( I )*B( I, J )
   30          CONTINUE
   40       CONTINUE
         END IF
      ELSE IF( COLEQU ) THEN
         DO 60 J = 1, NRHS
            DO 50 I = 1, N
               B( I, J ) = C( I )*B( I, J )
   50       CONTINUE
   60    CONTINUE
      END IF
*
      IF( NOFACT .OR. EQUIL ) THEN
*
```

```
*        Compute the LU factorization of the band matrix A.
*
         DO 70 J = 1, N
            J1 = MAX( J-KU, 1 )
            J2 = MIN( J+KL, N )
            CALL DCOPY( J2-J1+1, AB( KU+1-J+J1, J ), 1,
     $                  AFB( KL+KU+1-J+J1, J ), 1 )
   70    CONTINUE
*
         CALL DGBTRF( N, N, KL, KU, AFB, LDAFB, IPIV, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 ) THEN
*
*           Compute the reciprocal pivot growth factor of the
*           leading rank-deficient INFO columns of A.
*
            ANORM = ZERO
            DO 90 J = 1, INFO
               DO 80 I = MAX( KU+2-J, 1 ), MIN( N+KU+1-J, KL+KU+1 )
                  ANORM = MAX( ANORM, ABS( AB( I, J ) ) )
   80          CONTINUE
   90       CONTINUE
            RPVGRW = DLANTB( 'M', 'U', 'N', INFO, MIN( INFO-1, KL+KU ),
     $                       AFB( MAX( 1, KL+KU+2-INFO ), 1 ), LDAFB,
     $                       WORK )
            IF( RPVGRW.EQ.ZERO ) THEN
               RPVGRW = ONE
            ELSE
               RPVGRW = ANORM / RPVGRW
            END IF
            WORK( 1 ) = RPVGRW
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A and the
*     reciprocal pivot growth factor RPVGRW.
*
      IF( NOTRAN ) THEN
         NORM = '1'
      ELSE
         NORM = 'I'
      END IF
      ANORM = DLANGB( NORM, N, KL, KU, AB, LDAB, WORK )
      RPVGRW = DLANTB( 'M', 'U', 'N', N, KL+KU, AFB, LDAFB, WORK )
      IF( RPVGRW.EQ.ZERO ) THEN
         RPVGRW = ONE
```

```
      ELSE
         RPVGRW = DLANGB( 'M', N, KL, KU, AB, LDAB, WORK ) / RPVGRW
      END IF
*
*     Compute the reciprocal of the condition number of A.
*
      CALL DGBCON( NORM, N, KL, KU, AFB, LDAFB, IPIV, ANORM, RCOND,
     $             WORK, IWORK, INFO )
*
*     Compute the solution matrix X.
*
      CALL DLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL DGBTRS( TRANS, N, KL, KU, NRHS, AFB, LDAFB, IPIV, X, LDX,
     $             INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL DGBRFS( TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB, IPIV,
     $             B, LDB, X, LDX, FERR, BERR, WORK, IWORK, INFO )
*
*     Transform the solution matrix X to a solution of the original
*     system.
*
      IF( NOTRAN ) THEN
         IF( COLEQU ) THEN
            DO 110 J = 1, NRHS
               DO 100 I = 1, N
                  X( I, J ) = C( I )*X( I, J )
  100          CONTINUE
  110       CONTINUE
            DO 120 J = 1, NRHS
               FERR( J ) = FERR( J ) / COLCND
  120       CONTINUE
         END IF
      ELSE IF( ROWEQU ) THEN
         DO 140 J = 1, NRHS
            DO 130 I = 1, N
               X( I, J ) = R( I )*X( I, J )
  130       CONTINUE
  140    CONTINUE
         DO 150 J = 1, NRHS
            FERR( J ) = FERR( J ) / ROWCND
  150    CONTINUE
      END IF
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.DLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
```

```
*
      WORK( 1 ) = RPVGRW
      RETURN
*
*     End of DGBSVX
*
      END
```

———-

40.2.4 lisp code

— **dgbsvx.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dgbsvx
         (fact trans n kl ku nrhs ab ldab afb ldafb ipiv equed r c b ldb$ x ldx
          rcond ferr berr work iwork info)
    (declare (type (double-float) rcond)
             (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array double-float (*)) work berr ferr x b c r afb ab)
             (type (f2cl-lib:integer4) info ldx ldb$ ldafb ldab nrhs ku kl n)
             (type (string *) equed trans fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (trans character trans-%data% trans-%offset%)
         (equed character equed-%data% equed-%offset%)
         (ab double-float ab-%data% ab-%offset%)
         (afb double-float afb-%data% afb-%offset%)
         (r double-float r-%data% r-%offset%)
         (c double-float c-%data% c-%offset%)
         (b double-float b-%data% b-%offset%)
         (x double-float x-%data% x-%offset%)
         (ferr double-float ferr-%data% ferr-%offset%)
         (berr double-float berr-%data% berr-%offset%)
         (work double-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((amax 0.0d0) (anorm 0.0d0) (bignum 0.0d0) (colcnd 0.0d0)
             (rcmax 0.0d0) (rcmin 0.0d0) (rowcnd 0.0d0) (rpvgrw 0.0d0)
             (smlnum 0.0d0) (i 0) (infequ 0) (j 0) (j1 0) (j2 0)
             (norm
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (colequ nil) (equil nil) (nofact nil) (notran nil) (rowequ nil))
        (declare
```

```
 (type (double-float) amax anorm bignum colcnd rcmax rcmin rowcnd
  rpvgrw smlnum)
 (type (f2cl-lib:integer4) i infequ j j1 j2)
 (type (string 1) norm)
 (type f2cl-lib:logical colequ equil nofact notran rowequ))
(setf info 0)
(setf nofact
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "N")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val))
(setf equil
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "E")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val))
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
  (setf rowequ f2cl-lib:%false%) (setf colequ f2cl-lib:%false%))
 (t
  (setf rowequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "R")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)))
  (setf colequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "C")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
```

```
            (when var-0 (setf equed var-0))
            ret-val)))
  (setf smlnum (dlamch "Safe minimum")) (setf bignum (/ one smlnum))))
 (cond
  ((and (not nofact) (not equil)
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame fact "F")
           (declare (ignore var-1))
           (when var-0 (setf fact var-0))
           ret-val)))
   (setf info -1))
  ((and (not notran)
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame trans "T")
           (declare (ignore var-1))
           (when var-0 (setf trans var-0))
           ret-val))
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame trans "C")
           (declare (ignore var-1))
           (when var-0 (setf trans var-0))
           ret-val)))
   (setf info -2))
  ((< n 0) (setf info -3)) ((< kl 0) (setf info -4))
  ((< ku 0) (setf info -5)) ((< nrhs 0) (setf info -6))
  ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -8))
  ((< ldafb (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
   (setf info -10))
  ((and
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame fact "F")
      (declare (ignore var-1))
      (when var-0 (setf fact var-0))
      ret-val)
    (not
     (or rowequ colequ
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame equed "N")
           (declare (ignore var-1))
           (when var-0 (setf equed var-0))
           ret-val))))
   (setf info -12))
  (t
   (cond
    (rowequ (setf rcmin bignum) (setf rcmax zero)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
```

```
        (tagbody
          (setf rcmin
                  (min rcmin
                       (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
          (setf rcmax
                  (max rcmax
                       (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
         label10))
      (cond ((<= rcmin zero) (setf info -13))
            ((> n 0)
             (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum))))
            (t (setf rowcnd one)))))
    (cond
     ((and colequ (= info 0)) (setf rcmin bignum) (setf rcmax zero)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf rcmin
                  (min rcmin
                       (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
          (setf rcmax
                  (max rcmax
                       (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
         label20))
      (cond ((<= rcmin zero) (setf info -14))
            ((> n 0)
             (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum))))
            (t (setf colcnd one)))))
    (cond
     ((= info 0)
      (cond
       ((< ldb$
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -16))
       ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -18)))))))
  (cond
   ((/= info 0) (xerbla "DGBSVX" (f2cl-lib:int-sub info))
    (go end_label)))
  (cond
   (equil
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11)
        (dgbequ n n kl ku ab ldab r c rowcnd colcnd amax infequ)
      (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7))
      (setf rowcnd var-8)
      (setf colcnd var-9)
      (setf amax var-10)
      (setf infequ var-11))
```

```
      (cond
       ((= infequ 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11)
            (dlaqgb n n kl ku ab ldab r c rowcnd colcnd amax equed)
          (declare (ignore var-4 var-6 var-7))
          (when var-0 (setf n var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf kl var-2))
          (when var-3 (setf ku var-3))
          (when var-5 (setf ldab var-5))
          (when var-8 (setf rowcnd var-8))
          (when var-9 (setf colcnd var-9))
          (when var-10 (setf amax var-10))
          (when var-11 (setf equed var-11)))
        (setf rowequ
                (or
                 (multiple-value-bind (ret-val var-0 var-1)
                     (lsame equed "R")
                   (declare (ignore var-1))
                   (when var-0 (setf equed var-0))
                   ret-val)
                 (multiple-value-bind (ret-val var-0 var-1)
                     (lsame equed "B")
                   (declare (ignore var-1))
                   (when var-0 (setf equed var-0))
                   ret-val)))
        (setf colequ
                (or
                 (multiple-value-bind (ret-val var-0 var-1)
                     (lsame equed "C")
                   (declare (ignore var-1))
                   (when var-0 (setf equed var-0))
                   ret-val)
                 (multiple-value-bind (ret-val var-0 var-1)
                     (lsame equed "B")
                   (declare (ignore var-1))
                   (when var-0 (setf equed var-0))
                   ret-val)))))))
    (cond
     (notran
      (cond
       (rowequu
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j nrhs) nil)
          (tagbody
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
```

```
                (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                      b-%offset%)
                        (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                           (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                          b-%offset%)))
            label30))
        label40)))))
  (colequ
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j nrhs) nil)
     (tagbody
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                   (* (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)
                      (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                     b-%offset%)))
         label50))
     label60))))
 (cond
  ((or nofact equil)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf j1
               (max (the f2cl-lib:integer4 (f2cl-lib:int-sub j ku))
                    (the f2cl-lib:integer4 1)))
       (setf j2
               (min (the f2cl-lib:integer4 (f2cl-lib:int-add j kl))
                    (the f2cl-lib:integer4 n)))
       (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub j2 j1) 1)
        (f2cl-lib:array-slice ab-%data% double-float
                              ((+ ku 1 (f2cl-lib:int-sub j) j1) j)
                              ((1 ldab) (1 *)) ab-%offset%)
        1
        (f2cl-lib:array-slice afb-%data% double-float
                              ((+ kl ku 1 (f2cl-lib:int-sub j) j1) j)
                              ((1 ldafb) (1 *)) afb-%offset%)
        1)
     label70))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (dgbtrf n n kl ku afb ldafb ipiv info)
     (declare (ignore var-4 var-6))
     (when var-0 (setf n var-0))
     (when var-1 (setf n var-1))
     (when var-2 (setf kl var-2))
     (when var-3 (setf ku var-3))
```

```
  (when var-5 (setf ldafb var-5))
  (when var-7 (setf info var-7)))
(cond
 ((> info 0) (setf anorm zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j info) nil)
    (tagbody
      (f2cl-lib:fdo (i
                     (max
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add ku 2
                                             (f2cl-lib:int-sub j)))
                      (the f2cl-lib:integer4 1))
                     (f2cl-lib:int-add i 1))
                    ((> i
                        (min
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add n ku 1
                                                (f2cl-lib:int-sub
                                                 j)))
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add kl ku 1))))
                     nil)
        (tagbody
          (setf anorm
                  (max anorm
                       (abs
                        (f2cl-lib:fref ab-%data% (i j)
                                       ((1 ldab) (1 *))
                                       ab-%offset%))))
         label80))
    label90))
  (setf rpvgrw
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6
               var-7)
              (dlantb "M" "U" "N" info
               (min (the f2cl-lib:integer4 (f2cl-lib:int-sub info 1))
                    (the f2cl-lib:integer4 (f2cl-lib:int-add kl ku)))
               (f2cl-lib:array-slice afb-%data% double-float
                                     ((max 1
                                           (f2cl-lib:int-add kl ku 2
                                                             (f2cl-lib:int-sub
                                                              info)))
                                      1)
                                     ((1 ldafb) (1 *)) afb-%offset%)
               ldafb work)
            (declare (ignore var-0 var-1 var-2 var-4 var-5 var-7))
            (when var-3 (setf info var-3))
            (when var-6 (setf ldafb var-6))
```

```
                ret-val))
    (cond ((= rpvgrw zero) (setf rpvgrw one))
          (t (setf rpvgrw (/ anorm rpvgrw))))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) rpvgrw)
    (setf rcond zero) (go end_label)))))
(cond (notran (f2cl-lib:f2cl-set-string norm "1" (string 1)))
      (t (f2cl-lib:f2cl-set-string norm "I" (string 1))))
(setf anorm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dlangb norm n kl ku ab ldab work)
          (declare (ignore var-4 var-6))
          (when var-0 (setf norm var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf kl var-2))
          (when var-3 (setf ku var-3))
          (when var-5 (setf ldab var-5))
          ret-val))
(setf rpvgrw
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (dlantb "M" "U" "N" n (f2cl-lib:int-add kl ku) afb ldafb
             work)
          (declare (ignore var-0 var-1 var-2 var-4 var-5 var-7))
          (when var-3 (setf n var-3))
          (when var-6 (setf ldafb var-6))
          ret-val))
(cond ((= rpvgrw zero) (setf rpvgrw one))
      (t
       (setf rpvgrw
               (/
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (dlangb "M" n kl ku ab ldab work)
                  (declare (ignore var-0 var-4 var-6))
                  (when var-1 (setf n var-1))
                  (when var-2 (setf kl var-2))
                  (when var-3 (setf ku var-3))
                  (when var-5 (setf ldab var-5))
                  ret-val)
                rpvgrw))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11)
    (dgbcon norm n kl ku afb ldafb ipiv anorm rcond work iwork info)
  (declare (ignore var-2 var-3 var-4 var-6 var-7 var-9 var-10))
  (setf norm var-0)
  (setf n var-1)
  (setf ldafb var-5)
  (setf rcond var-8)
```

```
  (setf info var-11))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "Full" n nrhs b ldb$ x ldx)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf ldx var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgbtrs trans n kl ku nrhs afb ldafb ipiv x ldx info)
  (declare (ignore var-5 var-7 var-8))
  (when var-0 (setf trans var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf kl var-2))
  (when var-3 (setf ku var-3))
  (when var-4 (setf nrhs var-4))
  (when var-6 (setf ldafb var-6))
  (when var-9 (setf ldx var-9))
  (when var-10 (setf info var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18)
    (dgbrfs trans n kl ku nrhs ab ldab afb ldafb ipiv b ldb$ x ldx ferr
     berr work iwork info)
  (declare
   (ignore var-4 var-5 var-7 var-9 var-10 var-11 var-12 var-13 var-14
    var-15 var-16 var-17))
  (setf trans var-0)
  (setf n var-1)
  (setf kl var-2)
  (setf ku var-3)
  (setf ldab var-6)
  (setf ldafb var-8)
  (setf info var-18))
(cond
 (notran
  (cond
   (colequ
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                 x-%offset%)
                    (* (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)
                       (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
```

```
                                          x-%offset%)))
                  label100))
               label110))
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j nrhs) nil)
              (tagbody
                (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                        (/
                         (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                         colcnd))
               label120)))))
          (rowequ
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j nrhs) nil)
             (tagbody
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i n) nil)
                 (tagbody
                   (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                        x-%offset%)
                         (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                            (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                           x-%offset%)))
                 label130))
              label140))
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j nrhs) nil)
             (tagbody
               (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                       (/ (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                          rowcnd))
              label150))))
         (if (< rcond (dlamch "Epsilon"))
             (setf info (f2cl-lib:int-add n 1)))
         (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) rpvgrw)
         (go end_label)
        end_label
         (return
          (values fact trans n kl ku nrhs nil ldab nil ldafb nil equed nil nil
                  nil ldb$ nil ldx rcond nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgbsvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
```

```
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (string) (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (double-float)
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::fact
  fortran-to-lisp::trans
  fortran-to-lisp::n
  fortran-to-lisp::kl
  fortran-to-lisp::ku
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::ldab nil
  fortran-to-lisp::ldafb nil
  fortran-to-lisp::equed nil nil
  nil fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond nil nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::dgbrfs
  fortran-to-lisp::dgbcon
  fortran-to-lisp::dgbequ))))
```

40.3 dgbsvxx

40.3.1 Axiom unit tests

— **dgbsvxx.input** —

```
)set break resume
)sys rm -f dgbsvxx.output
)spool dgbsvxx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

40.3.2 Axiom help page [Demm05]

— **dgbsvxx.help** —

```
====================================================================
dgbsvxx examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgbsvxx (FACT, TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB,
           IPIV, EQUED, R, C, B, LDB, X, LDX, RCOND, RPVGRW, BERR, N_ERR_BNDS,
           ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, IWORK, INFO)
            DGBSVXX computes the solution to system of linear equations A * X
           = B for GB matrices

Function/Subroutine Documentation
   subroutine dgbsvxx (character FACT, character TRANS, integer N, integer KL,
       integer KU, integer NRHS, double precision, dimension( ldab, * ) AB,
       integer LDAB, double precision, dimension( ldafb, * ) AFB, integer
       LDAFB, integer, dimension( * ) IPIV, character EQUED, double precision,
       dimension( * ) R, double precision, dimension( * ) C, double precision,
       dimension( ldb, * ) B, integer LDB, double precision, dimension( ldx ,
       * ) X, integer LDX, double precision RCOND, double precision RPVGRW,
```

```
      double precision, dimension( * ) BERR, integer N_ERR_BNDS, double
      precision, dimension( nrhs, * ) ERR_BNDS_NORM, double precision,
      dimension( nrhs, * ) ERR_BNDS_COMP, integer NPARAMS, double precision,
      dimension( * ) PARAMS, double precision, dimension( * ) WORK, integer,
      dimension( * ) IWORK, integer INFO)
       DGBSVXX computes the solution to system of linear equations A * X = B
      for GB matrices

Purpose:
   DGBSVXX uses the LU factorization to compute the solution to a
   double precision system of linear equations  A * X = B,  where A is an
   N-by-N matrix and X and B are N-by-NRHS matrices.

   If requested, both normwise and maximum componentwise error bounds
   are returned. DGBSVXX will return a solution with a tiny
   guaranteed error (O(eps) where eps is the working machine
   precision) unless the matrix is very ill-conditioned, in which
   case a warning is returned. Relevant condition numbers also are
   calculated and returned.

   DGBSVXX accepts user-provided factorizations and equilibration
   factors; see the definitions of the FACT and EQUED options.
   Solving with refinement and using a factorization from a previous
   DGBSVXX call will also produce a solution with either O(eps)
   errors or warnings, but we cannot make that claim for general
   user-provided factorizations and equilibration factors if they
   differ from what DGBSVXX would itself produce.

Description:
   The following steps are performed:

   1. If FACT = 'E', double precision scaling factors are computed to
      equilibrate the system:

      TRANS = 'N':  diag(R)*A*diag(C)     *inv(diag(C))*X = diag(R)*B
      TRANS = 'T': (diag(R)*A*diag(C))**T *inv(diag(R))*X = diag(C)*B
      TRANS = 'C': (diag(R)*A*diag(C))**H *inv(diag(R))*X = diag(C)*B

      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(R)*A*diag(C) and B by diag(R)*B (if TRANS='N')
      or diag(C)*B (if TRANS = 'T' or 'C').

   2. If FACT = 'N' or 'E', the LU decomposition is used to factor
      the matrix A (after equilibration if FACT = 'E') as

      A = P * L * U,

      where P is a permutation matrix, L is a unit lower triangular
      matrix, and U is upper triangular.
```

```
   3. If some U(i,i)=0, so that U is exactly singular, then the
      routine returns with INFO = i. Otherwise, the factored form of A
      is used to estimate the condition number of the matrix A (see
      argument RCOND). If the reciprocal of the condition number is less
      than machine precision, the routine still goes on to solve for X
      and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. By default (unless PARAMS(LA_LINRX_ITREF_I) is set to zero),
      the routine will use iterative refinement to try to get a small
      error and error bounds.  Refinement calculates the residual to at
      least twice the working precision.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(C) (if TRANS = 'N') or diag(R) (if TRANS = 'T' or 'C') so
      that it solves the original system before equilibration.

      Some optional parameters are bundled in the PARAMS array.  These
      settings determine how refinement is performed, but often the
      defaults are acceptable.  If the defaults are acceptable, users
      can pass NPARAMS = 0 which prevents the source code from accessing
      the PARAMS argument.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AF and IPIV contain the factored form of A.
           If EQUED is not 'N', the matrix A has been
           equilibrated with scaling factors given by R and C.
           A, AF, and IPIV are not modified.
   = 'N':  The matrix A will be copied to AF and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
           copied to AF and factored.

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate Transpose = Transpose)

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.
```

```
KL
  KL is INTEGER
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

AB
  AB is DOUBLE PRECISION array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

  If FACT = 'F' and EQUED is not 'N', then AB must have been
  equilibrated by the scaling factors in R and/or C.  AB is not
  modified if FACT = 'F' or 'N', or if FACT = 'E' and
  EQUED = 'N' on exit.

  On exit, if EQUED .ne. 'N', A is scaled as follows:
  EQUED = 'R':  A := diag(R) * A
  EQUED = 'C':  A := A * diag(C)
  EQUED = 'B':  A := diag(R) * A * diag(C).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is DOUBLE PRECISION array, dimension (LDAFB,N)
  If FACT = 'F', then AFB is an input argument and on entry
  contains details of the LU factorization of the band matrix
  A, as computed by DGBTRF.  U is stored as an upper triangular
  band matrix with KL+KU superdiagonals in rows 1 to KL+KU+1,
  and the multipliers used during the factorization are stored
  in rows KL+KU+2 to 2*KL+KU+1.  If EQUED .ne. 'N', then AFB is
  the factored form of the equilibrated matrix A.

  If FACT = 'N', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then AF is an output argument and on exit
```

```
  returns the factors L and U from the factorization A = P*L*U
  of the equilibrated matrix A (see the description of A for
  the form of the equilibrated matrix).

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains the pivot indices from the factorization A = P*L*U
  as computed by DGETRF; row i of the matrix was interchanged
  with row IPIV(i).

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the equilibrated matrix A.

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
  = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
  = 'B':  Both row and column equilibration, i.e., A has been
          replaced by diag(R) * A * diag(C).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

R
  R is DOUBLE PRECISION array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
  EQUED = 'R' or 'B', each element of R must be positive.
  If R is output, each element of R is a power of the radix.
  If R is input, each element of R should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

C
```

```
  C is DOUBLE PRECISION array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.
  If C is output, each element of C is a power of the radix.
  If C is input, each element of C should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if TRANS = 'N' and EQUED = 'R' or 'B', B is overwritten by diag(R)*B;
  if TRANS = 'T' or 'C' and EQUED = 'C' or 'B', B is
     overwritten by diag(C)*B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  If INFO = 0, the N-by-NRHS solution matrix X to the original
  system of equations.  Note that A and B are modified on exit
  if EQUED .ne. 'N', and the solution to the equilibrated system is
  inv(diag(C))*X if TRANS = 'N' and EQUED = 'C' or 'B', or
  inv(diag(R))*X if TRANS = 'T' or 'C' and EQUED = 'R' or 'B'.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

RPVGRW
  RPVGRW is DOUBLE PRECISION
```

```
  Reciprocal pivot growth.  On exit, this contains the reciprocal
  pivot growth factor norm(A)/norm(U). The "max absolute element"
  norm is used.  If this is much less than 1, then the stability of
  the LU factorization of the (equilibrated) matrix A could be poor.
  This also means that the solution X, estimated condition numbers,
  and error bounds could be unreliable. If factorization fails with
  0<INFO<=N, then this contains the reciprocal pivot growth factor
  for the leading INFO columns of A.  In DGESVX, this quantity is
  returned in WORK(1).

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
           max_j (abs(XTRUE(j,i) - X(j,i)))
            ------------------------------
                   max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
```

```
      sqrt(n) * dlamch('Epsilon'). This error bound should only
      be trusted if the previous boolean is true.

   err = 3  Reciprocal condition number: Estimated normwise
      reciprocal condition number.  Compared with the threshold
      sqrt(n) * dlamch('Epsilon') to determine if the error
      estimate is "guaranteed". These reciprocal condition
      numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
      appropriately scaled matrix Z.
      Let Z = S*A, where S scales each row by a power of the
      radix so all absolute row sums of Z are approximately 1.

 ERR_BNDS_COMP
   ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
   For each right-hand side, this array contains information about
   various error bounds and condition numbers corresponding to the
   componentwise relative error, which is defined as follows:

   Componentwise relative error in the ith solution vector:
                   abs(XTRUE(j,i) - X(j,i))
             max_j ----------------------
                         abs(X(j,i))

   The array is indexed by the right-hand side i (on which the
   componentwise relative error depends), and the type of error
   information as described below. There currently are up to three
   pieces of information returned for each right-hand side. If
   componentwise accuracy is not requested (PARAMS(3) = 0.0), then
   ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
   the first (:,N_ERR_BNDS) entries are returned.

   The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
   right-hand side.

   The second index in ERR_BNDS_COMP(:,err) contains the following
   three fields:
   err = 1 "Trust/don't trust" boolean. Trust the answer if the
      reciprocal condition number is less than the threshold
      sqrt(n) * dlamch('Epsilon').

   err = 2 "Guaranteed" error bound: The estimated forward error,
      almost certainly within a factor of 10 of the true error
      so long as the next entry is greater than the threshold
      sqrt(n) * dlamch('Epsilon'). This error bound should only
      be trusted if the previous boolean is true.

   err = 3  Reciprocal condition number: Estimated componentwise
      reciprocal condition number.  Compared with the threshold
      sqrt(n) * dlamch('Epsilon') to determine if the error
      estimate is "guaranteed". These reciprocal condition
```

```
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is DOUBLE PRECISION array, dimension (NPARAMS)
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0D+0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the extra-precise refinement algorithm.
  (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is DOUBLE PRECISION array, dimension (4*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is
```

```
      guaranteed.
   < 0:  If INFO = -i, the i-th argument had an illegal value
   > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
      has been completed, but the factor U is exactly singular, so
      the solution and error bounds could not be computed. RCOND = 0
      is returned.
   = N+J: The solution corresponding to the Jth right-hand side is
      not guaranteed. The solutions corresponding to other right-
      hand sides K with K > J may not be guaranteed as well, but
      only the first such right-hand side is reported. If a small
      componentwise error is not requested (PARAMS(3) = 0.0) then
      the Jth right-hand side is the first with a normwise error
      bound that is not guaranteed (the smallest J such
      that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
      the Jth right-hand side is the first with either a normwise or
      componentwise error bound that is not guaranteed (the smallest
      J such that either ERR_BNDS_NORM(J,1) = 0.0 or
      ERR_BNDS_COMP(J,1) = 0.0). See the definition of
      ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
      about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

40.3.3 fortran code

— dgbsvxx.f —

```
*  ==================================================================
      SUBROUTINE DGBSVXX( FACT, TRANS, N, KL, KU, NRHS, AB, LDAB, AFB,
     $                    LDAFB, IPIV, EQUED, R, C, B, LDB, X, LDX,
     $                    RCOND, RPVGRW, BERR, N_ERR_BNDS,
     $                    ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $                    WORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, TRANS
      INTEGER            INFO, LDAB, LDAFB, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS, KL, KU
      DOUBLE PRECISION   RCOND, RPVGRW
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
```

```
      DOUBLE PRECISION   AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   X( LDX , * ),WORK( * )
      DOUBLE PRECISION   R( * ), C( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  =================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
*     ..
*     .. Local Scalars ..
      LOGICAL            COLEQU, EQUIL, NOFACT, NOTRAN, ROWEQU
      INTEGER            INFEQU, I, J
      DOUBLE PRECISION   AMAX, BIGNUM, COLCND, RCMAX, RCMIN,
     $                   ROWCND, SMLNUM
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, DLAMCH, DLA_GBRPVGRW
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, DLA_GBRPVGRW
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGBEQUB, DGBTRF, DGBTRS, DLACPY, DLAQGB,
     $                   XERBLA, DLASCL2, DGBRFSX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      NOTRAN = LSAME( TRANS, 'N' )
      SMLNUM = DLAMCH( 'Safe minimum' )
      BIGNUM = ONE / SMLNUM
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         ROWEQU = .FALSE.
```

```
         COLEQU = .FALSE.
      ELSE
         ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
         COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
      END IF
*
*     Default is failure.  If an input parameter is wrong or
*     factorization fails, make everything look horrible.  Only the
*     pivot growth is set here, the rest is initialized in DGBRFSX.
*
      RPVGRW = ZERO
*
*     Test the input parameters.  PARAMS is not tested until DGBRFSX.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.
     $     LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $        LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( KL.LT.0 ) THEN
         INFO = -4
      ELSE IF( KU.LT.0 ) THEN
         INFO = -5
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -6
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -8
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
         INFO = -10
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $        ( ROWEQU .OR. COLEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -12
      ELSE
         IF( ROWEQU ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 10 J = 1, N
               RCMIN = MIN( RCMIN, R( J ) )
               RCMAX = MAX( RCMAX, R( J ) )
 10         CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -13
            ELSE IF( N.GT.0 ) THEN
               ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               ROWCND = ONE
            END IF
```

```
         END IF
         IF( COLEQU .AND. INFO.EQ.0 ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 20 J = 1, N
               RCMIN = MIN( RCMIN, C( J ) )
               RCMAX = MAX( RCMAX, C( J ) )
 20         CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -14
            ELSE IF( N.GT.0 ) THEN
               COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               COLCND = ONE
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -15
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -16
            END IF
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGBSVXX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*     Compute row and column scalings to equilibrate the matrix A.
*
         CALL DGBEQUB( N, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $        AMAX, INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*     Equilibrate the matrix.
*
            CALL DLAQGB( N, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $           AMAX, EQUED )
            ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
            COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         END IF
*
*     If the scaling factors are not applied, set them to 1.0.
*
         IF ( .NOT.ROWEQU ) THEN
            DO J = 1, N
```

```
            R( J ) = 1.0D+0
         END DO
      END IF
      IF ( .NOT.COLEQU ) THEN
         DO J = 1, N
            C( J ) = 1.0D+0
         END DO
      END IF
   END IF
*
*     Scale the right hand side.
*
   IF( NOTRAN ) THEN
      IF( ROWEQU ) CALL DLASCL2(N, NRHS, R, B, LDB)
   ELSE
      IF( COLEQU ) CALL DLASCL2(N, NRHS, C, B, LDB)
   END IF
*
   IF( NOFACT .OR. EQUIL ) THEN
*
*       Compute the LU factorization of A.
*
      DO 40, J = 1, N
         DO 30, I = KL+1, 2*KL+KU+1
            AFB( I, J ) = AB( I-KL, J )
 30      CONTINUE
 40   CONTINUE
      CALL DGBTRF( N, N, KL, KU, AFB, LDAFB, IPIV, INFO )
*
*       Return if INFO is non-zero.
*
      IF( INFO.GT.0 ) THEN
*
*          Pivot in column INFO is exactly 0
*          Compute the reciprocal pivot growth factor of the
*          leading rank-deficient INFO columns of A.
*
         RPVGRW = DLA_GBRPVGRW( N, KL, KU, INFO, AB, LDAB, AFB,
     $           LDAFB )
         RETURN
      END IF
   END IF
*
*     Compute the reciprocal pivot growth factor RPVGRW.
*
   RPVGRW = DLA_GBRPVGRW( N, KL, KU, N, AB, LDAB, AFB, LDAFB )
*
*     Compute the solution matrix X.
*
   CALL DLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
```

```
      CALL DGBTRS( TRANS, N, KL, KU, NRHS, AFB, LDAFB, IPIV, X, LDX,
     $     INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL DGBRFSX( TRANS, EQUED, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB,
     $     IPIV, R, C, B, LDB, X, LDX, RCOND, BERR,
     $     N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $     WORK, IWORK, INFO )
*
*     Scale solutions.
*
      IF ( COLEQU .AND. NOTRAN ) THEN
         CALL DLASCL2 ( N, NRHS, C, X, LDX )
      ELSE IF ( ROWEQU .AND. .NOT.NOTRAN ) THEN
         CALL DLASCL2 ( N, NRHS, R, X, LDX )
      END IF
*
      RETURN
*
*     End of DGBSVXX
*
      END
```

———-

40.3.4 lisp code

— dgbsvxx.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dgbsvxx
         (fact trans n kl ku nrhs ab ldab afb ldafb ipiv equed r c b ldb$ x ldx
          rcond rpvgrw berr n_err_bnds err_bnds_norm err_bnds_comp nparams
          params work iwork info)
    (declare (type (double-float) rpvgrw rcond)
             (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array double-float (*)) work params err_bnds_comp
              err_bnds_norm berr x b c r afb ab)
             (type (f2cl-lib:integer4) info nparams n_err_bnds ldx ldb$ ldafb
              ldab nrhs ku kl n)
             (type (string *) equed trans fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
```

```
   (trans character trans-%data% trans-%offset%)
   (equed character equed-%data% equed-%offset%)
   (ab double-float ab-%data% ab-%offset%)
   (afb double-float afb-%data% afb-%offset%)
   (r double-float r-%data% r-%offset%)
   (c double-float c-%data% c-%offset%)
   (b double-float b-%data% b-%offset%)
   (x double-float x-%data% x-%offset%)
   (berr double-float berr-%data% berr-%offset%)
   (err_bnds_norm double-float err_bnds_norm-%data%
    err_bnds_norm-%offset%)
   (err_bnds_comp double-float err_bnds_comp-%data%
    err_bnds_comp-%offset%)
   (params double-float params-%data% params-%offset%)
   (work double-float work-%data% work-%offset%)
   (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
   (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
(prog ((amax 0.0d0) (bignum 0.0d0) (colcnd 0.0d0) (rcmax 0.0d0)
       (rcmin 0.0d0) (rowcnd 0.0d0) (smlnum 0.0d0) (infequ 0) (i 0) (j 0)
       (colequ nil) (equil nil) (nofact nil) (notran nil) (rowequ nil))
  (declare
   (type (double-float) amax bignum colcnd rcmax rcmin rowcnd smlnum)
   (type (f2cl-lib:integer4) infequ i j)
   (type f2cl-lib:logical colequ equil nofact notran rowequ))
  (setf info 0)
  (setf nofact
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame fact "N")
            (declare (ignore var-1))
            (when var-0 (setf fact var-0))
            ret-val))
  (setf equil
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame fact "E")
            (declare (ignore var-1))
            (when var-0 (setf fact var-0))
            ret-val))
  (setf notran
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame trans "N")
            (declare (ignore var-1))
            (when var-0 (setf trans var-0))
            ret-val))
  (setf smlnum (dlamch "Safe minimum"))
  (setf bignum (/ one smlnum))
  (cond
   ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
    (setf rowequ f2cl-lib:%false%) (setf colequ f2cl-lib:%false%))
   (t
    (setf rowequ
```

```
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "R")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "B")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)))
  (setf colequ
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "C")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "B")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)))))
(setf rpvgrw zero)
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< kl 0) (setf info -4))
 ((< ku 0) (setf info -5)) ((< nrhs 0) (setf info -6))
 ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -8))
```

```
((< ldafb (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
 (setf info -10))
((and
  (multiple-value-bind (ret-val var-0 var-1)
      (lsame fact "F")
    (declare (ignore var-1))
    (when var-0 (setf fact var-0))
    ret-val)
  (not
   (or rowequ colequ
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame equed "N")
         (declare (ignore var-1))
         (when var-0 (setf equed var-0))
         ret-val))))
 (setf info -12))
(t
 (cond
  (rowequ (setf rcmin bignum) (setf rcmax zero)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf rcmin
               (min rcmin
                    (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
       (setf rcmax
               (max rcmax
                    (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
      label10))
   (cond ((<= rcmin zero) (setf info -13))
         ((> n 0)
          (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum))))
         (t (setf rowcnd one)))))
 (cond
  ((and colequ (= info 0)) (setf rcmin bignum) (setf rcmax zero)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf rcmin
               (min rcmin
                    (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
       (setf rcmax
               (max rcmax
                    (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
      label20))
   (cond ((<= rcmin zero) (setf info -14))
         ((> n 0)
          (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum))))
         (t (setf colcnd one)))))
 (cond
```

```
    ((= info 0)
     (cond
      ((< ldb$
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -15))
      ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -16)))))))
(cond
 ((/= info 0) (xerbla "DGBSVXX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (equil
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11)
      (dgbequb n n kl ku ab ldab r c rowcnd colcnd amax infequ)
    (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7))
    (setf rowcnd var-8)
    (setf colcnd var-9)
    (setf amax var-10)
    (setf infequ var-11))
  (cond
   ((= infequ 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11)
        (dlaqgb n n kl ku ab ldab r c rowcnd colcnd amax equed)
      (declare (ignore var-4 var-6 var-7))
      (when var-0 (setf n var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf kl var-2))
      (when var-3 (setf ku var-3))
      (when var-5 (setf ldab var-5))
      (when var-8 (setf rowcnd var-8))
      (when var-9 (setf colcnd var-9))
      (when var-10 (setf amax var-10))
      (when var-11 (setf equed var-11)))
    (setf rowequ
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "R")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "B")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)))
    (setf colequ
```

```
              (or
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame equed "C")
                 (declare (ignore var-1))
                 (when var-0 (setf equed var-0))
                 ret-val)
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame equed "B")
                 (declare (ignore var-1))
                 (when var-0 (setf equed var-0))
                 ret-val)))))
  (cond
   ((not rowequ)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%) 1.0d0)
       label100000))))
  (cond
   ((not colequ)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) 1.0d0)
       label100001))))))
(cond
 (notran
  (if rowequ
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dlascl2 n nrhs r b ldb$)
        (declare (ignore var-2 var-3))
        (when var-0 (setf n var-0))
        (when var-1 (setf nrhs var-1))
        (when var-4 (setf ldb$ var-4)))))
 (t
  (if colequ
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dlascl2 n nrhs c b ldb$)
        (declare (ignore var-2 var-3))
        (when var-0 (setf n var-0))
        (when var-1 (setf nrhs var-1))
        (when var-4 (setf ldb$ var-4))))))
(cond
 ((or nofact equil)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i (f2cl-lib:int-add kl 1) (f2cl-lib:int-add i 1))
                    ((> i
                        (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku
```

```
                                                1))
                          nil)
              (tagbody
                (setf (f2cl-lib:fref afb-%data% (i j) ((1 ldafb) (1 *))
                                     afb-%offset%)
                        (f2cl-lib:fref ab-%data% ((f2cl-lib:int-sub i kl) j)
                                       ((1 ldab) (1 *)) ab-%offset%))
               label30))
           label40))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (dgbtrf n n kl ku afb ldafb ipiv info)
          (declare (ignore var-4 var-5 var-6))
          (setf n var-0)
          (setf n var-1)
          (setf kl var-2)
          (setf ku var-3)
          (setf info var-7))
        (cond
         ((> info 0)
          (setf rpvgrw
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6
                       var-7)
                      (dla_gbrpvgrw n kl ku info ab ldab afb ldafb)
                    (declare (ignore var-4 var-6))
                    (when var-0 (setf n var-0))
                    (when var-1 (setf kl var-1))
                    (when var-2 (setf ku var-2))
                    (when var-3 (setf info var-3))
                    (when var-5 (setf ldab var-5))
                    (when var-7 (setf ldafb var-7))
                    ret-val))
          (go end_label)))))
      (setf rpvgrw
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
                  (dla_gbrpvgrw n kl ku n ab ldab afb ldafb)
                (declare (ignore var-4 var-6))
                (when var-0 (setf n var-0))
                (when var-1 (setf kl var-1))
                (when var-2 (setf ku var-2))
                (when var-3 (setf n var-3))
                (when var-5 (setf ldab var-5))
                (when var-7 (setf ldafb var-7))
                ret-val))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlacpy "Full" n nrhs b ldb$ x ldx)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf n var-1))
```

```
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf ldx var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgbtrs trans n kl ku nrhs afb ldafb ipiv x ldx info)
  (declare (ignore var-2 var-3 var-5 var-7 var-8))
  (setf trans var-0)
  (setf n var-1)
  (setf nrhs var-4)
  (setf ldafb var-6)
  (setf ldx var-9)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19
     var-20 var-21 var-22 var-23 var-24 var-25 var-26)
    (dgbrfsx trans equed n kl ku nrhs ab ldab afb ldafb ipiv r c b ldb$
     x ldx rcond berr n_err_bnds err_bnds_norm err_bnds_comp nparams
     params work iwork info)
  (declare
   (ignore var-6 var-8 var-10 var-11 var-12 var-13 var-15 var-18 var-20
    var-21 var-23 var-24 var-25))
  (when var-0 (setf trans var-0))
  (when var-1 (setf equed var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf kl var-3))
  (when var-4 (setf ku var-4))
  (when var-5 (setf nrhs var-5))
  (when var-7 (setf ldab var-7))
  (when var-9 (setf ldafb var-9))
  (when var-14 (setf ldb$ var-14))
  (when var-16 (setf ldx var-16))
  (when var-17 (setf rcond var-17))
  (when var-19 (setf n_err_bnds var-19))
  (when var-22 (setf nparams var-22))
  (when var-26 (setf info var-26)))
(cond
 ((and colequ notran)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dlascl2 n nrhs c x ldx)
    (declare (ignore var-2 var-3))
    (when var-0 (setf n var-0))
    (when var-1 (setf nrhs var-1))
    (when var-4 (setf ldx var-4))))
 ((and rowequ (not notran))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dlascl2 n nrhs r x ldx)
    (declare (ignore var-2 var-3))
```

```
            (when var-0 (setf n var-0))
            (when var-1 (setf nrhs var-1))
            (when var-4 (setf ldx var-4)))))
        (go end_label)
       end_label
        (return
         (values fact trans n kl ku nrhs nil ldab nil ldafb nil equed nil nil
                 nil ldb$ nil ldx rcond rpvgrw nil n_err_bnds nil nil nparams
                 nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgbsvxx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (string) (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::fact
                                              fortran-to-lisp::trans
                                              fortran-to-lisp::n
                                              fortran-to-lisp::kl
                                              fortran-to-lisp::ku
```

```
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::ldab nil
  fortran-to-lisp::ldafb nil
  fortran-to-lisp::equed nil nil
  nil fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond
  fortran-to-lisp::rpvgrw nil
  fortran-to-lisp::n_err_bnds nil
  nil fortran-to-lisp::nparams nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::dgbtrs
  fortran-to-lisp::dgbtrf
  fortran-to-lisp::dgbequb))))
```

———-

Chapter 41

LAPACK - General Band Matrix, Linear Solve, Real

41.1 sgbsv

41.1.1 Axiom unit tests

— sgbsv.input —

```
)set break resume
)sys rm -f sgbsv.output
)spool sgbsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

41.1.2 Axiom help page

— sgbsv.help —

```
========================================================================
sgbsv examples
========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgbsv (N, KL, KU, NRHS, AB, LDAB, IPIV, B, LDB, INFO)
            SGBSV computes the solution to system of linear equations A * X =
           B for GB matrices (simple driver)

Function/Subroutine Documentation
   subroutine sgbsv (integer N, integer KL, integer KU, integer NRHS, real,
       dimension( ldab, * ) AB, integer LDAB, integer, dimension( * ) IPIV,
       real, dimension( ldb, * ) B, integer LDB, integer INFO)
        SGBSV computes the solution to system of linear equations A * X = B
       for GB matrices (simple driver)

Purpose:
   SGBSV computes the solution to a real system of linear equations
   A * X = B, where A is a band matrix of order N with KL subdiagonals
   and KU superdiagonals, and X and B are N-by-NRHS matrices.

   The LU decomposition with partial pivoting and row interchanges is
   used to factor A as A = L * U, where L is a product of permutation
   and unit lower triangular matrices with KL subdiagonals, and U is
   upper triangular with KL+KU superdiagonals.  The factored form of A
   is then used to solve the system of equations A * X = B.

Parameters:

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 AB
   AB is REAL array, dimension (LDAB,N)
```

```
  On entry, the matrix A in band storage, in rows KL+1 to
  2*KL+KU+1; rows 1 to KL of the array need not be set.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(KL+KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+KL)
  On exit, details of the factorization: U is stored as an
  upper triangular band matrix with KL+KU superdiagonals in
  rows 1 to KL+KU+1, and the multipliers used during the
  factorization are stored in rows KL+KU+2 to 2*KL+KU+1.
  See below for further details.

 LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.

 IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices that define the permutation matrix P;
  row i of the matrix was interchanged with row IPIV(i).

 B
  B is REAL array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

 LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

 INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, U(i,i) is exactly zero.  The factorization
        has been completed, but the factor U is exactly
        singular, and the solution has not been computed.

Further Details:

  The band storage scheme is illustrated by the following example, when
  M = N = 6, KL = 2, KU = 1:

       On entry:                      On exit:

           *    *    *    +    +    +       *    *    *   u14  u25  u36
           *    *    +    +    +    +       *    *   u13  u24  u35  u46
           *   a12  a23  a34  a45  a56       *   u12  u23  u34  u45  u56
          a11  a22  a33  a44  a55  a66      u11  u22  u33  u44  u55  u66
          a21  a32  a43  a54  a65   *      m21  m32  m43  m54  m65   *
          a31  a42  a53  a64   *    *      m31  m42  m53  m64   *    *
```

```
Array elements marked * are not used by the routine; elements marked
+ need not be set on entry, but are required by the routine to store
elements of U because of fill-in resulting from the row interchanges.
```

———-

41.1.3 fortran code

— **sgbsv.f** —

```
*  =====================================================================
      SUBROUTINE SGBSV( N, KL, KU, NRHS, AB, LDAB, IPIV, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, KL, KU, LDAB, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               AB( LDAB, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. External Subroutines ..
      EXTERNAL           SGBTRF, SGBTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( KL.LT.0 ) THEN
         INFO = -2
      ELSE IF( KU.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
```

```
         INFO = -4
      ELSE IF( LDAB.LT.2*KL+KU+1 ) THEN
         INFO = -6
      ELSE IF( LDB.LT.MAX( N, 1 ) ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGBSV ', -INFO )
         RETURN
      END IF
*
*     Compute the LU factorization of the band matrix A.
*
      CALL SGBTRF( N, N, KL, KU, AB, LDAB, IPIV, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
         CALL SGBTRS( 'No transpose', N, KL, KU, NRHS, AB, LDAB, IPIV,
     $                B, LDB, INFO )
      END IF
      RETURN
*
*     End of SGBSV
*
      END
```

———-

41.1.4 lisp code

— sgbsv.lisp —

```
(defun sgbsv (n kl ku nrhs ab ldab ipiv b ldb$ info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array single-float (*)) b ab)
           (type (f2cl-lib:integer4) info ldb$ ldab nrhs ku kl n))
  (f2cl-lib:with-multi-array-data
      ((ab single-float ab-%data% ab-%offset%)
       (b single-float b-%data% b-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ()
      (declare)
      (setf info 0)
      (cond ((< n 0) (setf info -1)) ((< kl 0) (setf info -2))
            ((< ku 0) (setf info -3)) ((< nrhs 0) (setf info -4))
            ((< ldab (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
             (setf info -6))
```

```
            ((< ldb$ (max (the f2cl-lib:integer4 n) (the f2cl-lib:integer4 1)))
             (setf info -9)))
      (cond
       ((/= info 0) (xerbla "SGBSV " (f2cl-lib:int-sub info)) (go end_label)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (sgbtrf n n kl ku ab ldab ipiv info)
        (declare (ignore var-4 var-6))
        (when var-0 (setf n var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf kl var-2))
        (when var-3 (setf ku var-3))
        (when var-5 (setf ldab var-5))
        (when var-7 (setf info var-7)))
      (cond
       ((= info 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10)
            (sgbtrs "No transpose" n kl ku nrhs ab ldab ipiv b ldb$ info)
          (declare (ignore var-0 var-5 var-7 var-8))
          (when var-1 (setf n var-1))
          (when var-2 (setf kl var-2))
          (when var-3 (setf ku var-3))
          (when var-4 (setf nrhs var-4))
          (when var-6 (setf ldab var-6))
          (when var-9 (setf ldb$ var-9))
          (when var-10 (setf info var-10)))))
      (go end_label)
     end_label
      (return (values n kl ku nrhs nil ldab nil nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgbsv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n
                                              fortran-to-lisp::kl
```

```
                                                  fortran-to-lisp::ku
                                                  fortran-to-lisp::nrhs nil
                                                  fortran-to-lisp::ldab nil nil
                                                  fortran-to-lisp::ldb$
                                                  fortran-to-lisp::info)
                                                :calls 'nil)))
```

———-

41.2 sgbsvx

41.2.1 Axiom unit tests

— **sgbsvx.input** —

```
)set break resume
)sys rm -f sgbsvx.output
)spool sgbsvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

41.2.2 Axiom help page

— **sgbsvx.help** —

```
==================================================================
sgbsvx examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgbsvx (FACT, TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB,
```

```
          IPIV, EQUED, R, C, B, LDB, X, LDX, RCOND, FERR, BERR, WORK, IWORK,
          INFO)
           SGBSVX computes the solution to system of linear equations A * X =
          B for GB matrices

Function/Subroutine Documentation
   subroutine sgbsvx (character FACT, character TRANS, integer N, integer KL,
       integer KU, integer NRHS, real, dimension( ldab, * ) AB, integer LDAB,
       real, dimension( ldafb, * ) AFB, integer LDAFB, integer, dimension( * )
       IPIV, character EQUED, real, dimension( * ) R, real, dimension( * ) C,
       real, dimension( ldb, * ) B, integer LDB, real, dimension( ldx, * ) X,
       integer LDX, real RCOND, real, dimension( * ) FERR, real, dimension( *
       ) BERR, real, dimension( * ) WORK, integer, dimension( * ) IWORK,
       integer INFO)
        SGBSVX computes the solution to system of linear equations A * X = B
       for GB matrices

Purpose:
   SGBSVX uses the LU factorization to compute the solution to a real
   system of linear equations A * X = B, A**T * X = B, or A**H * X = B,
   where A is a band matrix of order N with KL subdiagonals and KU
   superdiagonals, and X and B are N-by-NRHS matrices.

   Error bounds on the solution and a condition estimate are also
   provided.

Description:

   The following steps are performed by this subroutine:

   1. If FACT = 'E', real scaling factors are computed to equilibrate
      the system:
      TRANS = 'N':  diag(R)*A*diag(C)     *inv(diag(C))*X = diag(R)*B
      TRANS = 'T': (diag(R)*A*diag(C))**T *inv(diag(R))*X = diag(C)*B
      TRANS = 'C': (diag(R)*A*diag(C))**H *inv(diag(R))*X = diag(C)*B
      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(R)*A*diag(C) and B by diag(R)*B (if TRANS='N')
      or diag(C)*B (if TRANS = 'T' or 'C').

   2. If FACT = 'N' or 'E', the LU decomposition is used to factor the
      matrix A (after equilibration if FACT = 'E') as
         A = L * U,
      where L is a product of permutation and unit lower triangular
      matrices with KL subdiagonals, and U is upper triangular with
      KL+KU superdiagonals.

   3. If some U(i,i)=0, so that U is exactly singular, then the routine
      returns with INFO = i. Otherwise, the factored form of A is used
      to estimate the condition number of the matrix A.  If the
```

```
      reciprocal of the condition number is less than machine precision,
      INFO = N+1 is returned as a warning, but the routine still goes on
      to solve for X and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates
      for it.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(C) (if TRANS = 'N') or diag(R) (if TRANS = 'T' or 'C') so
      that it solves the original system before equilibration.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AFB and IPIV contain the factored form of
      A.  If EQUED is not 'N', the matrix A has been
      equilibrated with scaling factors given by R and C.
      AB, AFB, and IPIV are not modified.
   = 'N':  The matrix A will be copied to AFB and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
      copied to AFB and factored.

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations.
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Transpose)

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 NRHS
   NRHS is INTEGER
```

```
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

AB
  AB is REAL array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

  If FACT = 'F' and EQUED is not 'N', then A must have been
  equilibrated by the scaling factors in R and/or C.  AB is not
  modified if FACT = 'F' or 'N', or if FACT = 'E' and
  EQUED = 'N' on exit.

  On exit, if EQUED .ne. 'N', A is scaled as follows:
  EQUED = 'R':  A := diag(R) * A
  EQUED = 'C':  A := A * diag(C)
  EQUED = 'B':  A := diag(R) * A * diag(C).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is REAL array, dimension (LDAFB,N)
  If FACT = 'F', then AFB is an input argument and on entry
  contains details of the LU factorization of the band matrix
  A, as computed by SGBTRF.  U is stored as an upper triangular
  band matrix with KL+KU superdiagonals in rows 1 to KL+KU+1,
  and the multipliers used during the factorization are stored
  in rows KL+KU+2 to 2*KL+KU+1.  If EQUED .ne. 'N', then AFB is
  the factored form of the equilibrated matrix A.

  If FACT = 'N', then AFB is an output argument and on exit
  returns details of the LU factorization of A.

  If FACT = 'E', then AFB is an output argument and on exit
  returns details of the LU factorization of the equilibrated
  matrix A (see the description of AB for the form of the
  equilibrated matrix).

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains the pivot indices from the factorization A = L*U
```

```
  as computed by SGBTRF; row i of the matrix was interchanged
  with row IPIV(i).

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = L*U
  of the original matrix A.

  If FACT = 'E', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = L*U
  of the equilibrated matrix A.

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
  = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
  = 'B':  Both row and column equilibration, i.e., A has been
          replaced by diag(R) * A * diag(C).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

R
  R is REAL array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
  EQUED = 'R' or 'B', each element of R must be positive.

C
  C is REAL array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.

B
  B is REAL array, dimension (LDB,NRHS)
  On entry, the right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if TRANS = 'N' and EQUED = 'R' or 'B', B is overwritten by
  diag(R)*B;
  if TRANS = 'T' or 'C' and EQUED = 'C' or 'B', B is
  overwritten by diag(C)*B.

LDB
  LDB is INTEGER
```

```
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is REAL array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X
  to the original system of equations.  Note that A and B are
  modified on exit if EQUED .ne. 'N', and the solution to the
  equilibrated system is inv(diag(C))*X if TRANS = 'N' and
  EQUED = 'C' or 'B', or inv(diag(R))*X if TRANS = 'T' or 'C'
  and EQUED = 'R' or 'B'.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  The estimate of the reciprocal condition number of the matrix
  A after equilibration (if done).  If RCOND is less than the
  machine precision (in particular, if RCOND = 0), the matrix
  is singular to working precision.  This condition is
  indicated by a return code of INFO > 0.

FERR
  FERR is REAL array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is REAL array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is REAL array, dimension (3*N)
  On exit, WORK(1) contains the reciprocal pivot growth
  factor norm(A)/norm(U). The "max absolute element" norm is
  used. If WORK(1) is much less than 1, then the stability
  of the LU factorization of the (equilibrated) matrix A
  could be poor. This also means that the solution X, condition
  estimator RCOND, and forward error bound FERR could be
  unreliable. If factorization fails with 0<INFO<=N, then
  WORK(1) contains the reciprocal pivot growth factor for the
```

```
  leading INFO columns of A.

 IWORK
  IWORK is INTEGER array, dimension (N)

 INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, and i is
  <= N:  U(i,i) is exactly zero.  The factorization
     has been completed, but the factor U is exactly
     singular, so the solution and error bounds
     could not be computed. RCOND = 0 is returned.
  = N+1: U is nonsingular, but RCOND is less than machine
     precision, meaning that the matrix is singular
     to working precision.  Nevertheless, the
     solution and error bounds are computed because
     there are a number of situations where the
     computed solution can be more accurate than the
```

———-

41.2.3 fortran code

— **sgbsvx.f** —

```
*  =====================================================================
      SUBROUTINE SGBSVX( FACT, TRANS, N, KL, KU, NRHS, AB, LDAB, AFB,
     $                   LDAFB, IPIV, EQUED, R, C, B, LDB, X, LDX,
     $                   RCOND, FERR, BERR, WORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, TRANS
      INTEGER            INFO, KL, KU, LDAB, LDAFB, LDB, LDX, N, NRHS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      REAL               AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   BERR( * ), C( * ), FERR( * ), R( * ),
     $                   WORK( * ), X( LDX, * )
```

```
*     ..
*
*  =====================================================================
*  Moved setting of INFO = N+1 so INFO does not subsequently get
*  overwritten.  Sven, 17 Mar 05.
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            COLEQU, EQUIL, NOFACT, NOTRAN, ROWEQU
      CHARACTER          NORM
      INTEGER            I, INFEQU, J, J1, J2
      REAL               AMAX, ANORM, BIGNUM, COLCND, RCMAX, RCMIN,
     $                   ROWCND, RPVGRW, SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH, SLANGB, SLANTB
      EXTERNAL           LSAME, SLAMCH, SLANGB, SLANTB
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SGBCON, SGBEQU, SGBRFS, SGBTRF, SGBTRS,
     $                   SLACPY, SLAQGB, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      NOTRAN = LSAME( TRANS, 'N' )
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         ROWEQU = .FALSE.
         COLEQU = .FALSE.
      ELSE
         ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
         COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         SMLNUM = SLAMCH( 'Safe minimum' )
         BIGNUM = ONE / SMLNUM
      END IF
*
*     Test the input parameters.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.LSAME( FACT, 'F' ) )
```

```
$     THEN
      INFO = -1
   ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
$         LSAME( TRANS, 'C' ) ) THEN
      INFO = -2
   ELSE IF( N.LT.0 ) THEN
      INFO = -3
   ELSE IF( KL.LT.0 ) THEN
      INFO = -4
   ELSE IF( KU.LT.0 ) THEN
      INFO = -5
   ELSE IF( NRHS.LT.0 ) THEN
      INFO = -6
   ELSE IF( LDAB.LT.KL+KU+1 ) THEN
      INFO = -8
   ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
      INFO = -10
   ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
$         ( ROWEQU .OR. COLEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
      INFO = -12
   ELSE
      IF( ROWEQU ) THEN
         RCMIN = BIGNUM
         RCMAX = ZERO
         DO 10 J = 1, N
            RCMIN = MIN( RCMIN, R( J ) )
            RCMAX = MAX( RCMAX, R( J ) )
10       CONTINUE
         IF( RCMIN.LE.ZERO ) THEN
            INFO = -13
         ELSE IF( N.GT.0 ) THEN
            ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
         ELSE
            ROWCND = ONE
         END IF
      END IF
      IF( COLEQU .AND. INFO.EQ.0 ) THEN
         RCMIN = BIGNUM
         RCMAX = ZERO
         DO 20 J = 1, N
            RCMIN = MIN( RCMIN, C( J ) )
            RCMAX = MAX( RCMAX, C( J ) )
20       CONTINUE
         IF( RCMIN.LE.ZERO ) THEN
            INFO = -14
         ELSE IF( N.GT.0 ) THEN
            COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
         ELSE
            COLCND = ONE
         END IF
```

```
         END IF
         IF( INFO.EQ.0 ) THEN
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -16
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -18
            END IF
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGBSVX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*        Compute row and column scalings to equilibrate the matrix A.
*
         CALL SGBEQU( N, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $                AMAX, INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*           Equilibrate the matrix.
*
            CALL SLAQGB( N, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $                   AMAX, EQUED )
            ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
            COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         END IF
      END IF
*
*     Scale the right hand side.
*
      IF( NOTRAN ) THEN
         IF( ROWEQU ) THEN
            DO 40 J = 1, NRHS
               DO 30 I = 1, N
                  B( I, J ) = R( I )*B( I, J )
   30          CONTINUE
   40       CONTINUE
         END IF
      ELSE IF( COLEQU ) THEN
         DO 60 J = 1, NRHS
            DO 50 I = 1, N
               B( I, J ) = C( I )*B( I, J )
   50       CONTINUE
   60    CONTINUE
      END IF
*
```

```
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the LU factorization of the band matrix A.
*
         DO 70 J = 1, N
            J1 = MAX( J-KU, 1 )
            J2 = MIN( J+KL, N )
            CALL SCOPY( J2-J1+1, AB( KU+1-J+J1, J ), 1,
     $                  AFB( KL+KU+1-J+J1, J ), 1 )
   70    CONTINUE
*
         CALL SGBTRF( N, N, KL, KU, AFB, LDAFB, IPIV, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 ) THEN
*
*           Compute the reciprocal pivot growth factor of the
*           leading rank-deficient INFO columns of A.
*
            ANORM = ZERO
            DO 90 J = 1, INFO
               DO 80 I = MAX( KU+2-J, 1 ), MIN( N+KU+1-J, KL+KU+1 )
                  ANORM = MAX( ANORM, ABS( AB( I, J ) ) )
   80          CONTINUE
   90       CONTINUE
            RPVGRW = SLANTB( 'M', 'U', 'N', INFO, MIN( INFO-1, KL+KU ),
     $                       AFB( MAX( 1, KL+KU+2-INFO ), 1 ), LDAFB,
     $                       WORK )
            IF( RPVGRW.EQ.ZERO ) THEN
               RPVGRW = ONE
            ELSE
               RPVGRW = ANORM / RPVGRW
            END IF
            WORK( 1 ) = RPVGRW
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A and the
*     reciprocal pivot growth factor RPVGRW.
*
      IF( NOTRAN ) THEN
         NORM = '1'
      ELSE
         NORM = 'I'
      END IF
      ANORM = SLANGB( NORM, N, KL, KU, AB, LDAB, WORK )
      RPVGRW = SLANTB( 'M', 'U', 'N', N, KL+KU, AFB, LDAFB, WORK )
```

```
      IF( RPVGRW.EQ.ZERO ) THEN
         RPVGRW = ONE
      ELSE
         RPVGRW = SLANGB( 'M', N, KL, KU, AB, LDAB, WORK ) / RPVGRW
      END IF
*
*     Compute the reciprocal of the condition number of A.
*
      CALL SGBCON( NORM, N, KL, KU, AFB, LDAFB, IPIV, ANORM, RCOND,
     $             WORK, IWORK, INFO )
*
*     Compute the solution matrix X.
*
      CALL SLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL SGBTRS( TRANS, N, KL, KU, NRHS, AFB, LDAFB, IPIV, X, LDX,
     $             INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL SGBRFS( TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB, IPIV,
     $             B, LDB, X, LDX, FERR, BERR, WORK, IWORK, INFO )
*
*     Transform the solution matrix X to a solution of the original
*     system.
*
      IF( NOTRAN ) THEN
         IF( COLEQU ) THEN
            DO 110 J = 1, NRHS
               DO 100 I = 1, N
                  X( I, J ) = C( I )*X( I, J )
  100          CONTINUE
  110       CONTINUE
            DO 120 J = 1, NRHS
               FERR( J ) = FERR( J ) / COLCND
  120       CONTINUE
         END IF
      ELSE IF( ROWEQU ) THEN
         DO 140 J = 1, NRHS
            DO 130 I = 1, N
               X( I, J ) = R( I )*X( I, J )
  130       CONTINUE
  140    CONTINUE
         DO 150 J = 1, NRHS
            FERR( J ) = FERR( J ) / ROWCND
  150    CONTINUE
      END IF
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
```

```
      IF( RCOND.LT.SLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      WORK( 1 ) = RPVGRW
      RETURN
*
*     End of SGBSVX
*
      END
```

———-

41.2.4 lisp code

— **sgbsvx.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sgbsvx
         (fact trans n kl ku nrhs ab ldab afb ldafb ipiv equed r c b ldb$ x ldx
          rcond ferr berr work iwork info)
    (declare (type (single-float) rcond)
             (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array single-float (*)) work berr ferr x b c r afb ab)
             (type (f2cl-lib:integer4) info ldx ldb$ ldafb ldab nrhs ku kl n)
             (type (string *) equed trans fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (trans character trans-%data% trans-%offset%)
         (equed character equed-%data% equed-%offset%)
         (ab single-float ab-%data% ab-%offset%)
         (afb single-float afb-%data% afb-%offset%)
         (r single-float r-%data% r-%offset%)
         (c single-float c-%data% c-%offset%)
         (b single-float b-%data% b-%offset%)
         (x single-float x-%data% x-%offset%)
         (ferr single-float ferr-%data% ferr-%offset%)
         (berr single-float berr-%data% berr-%offset%)
         (work single-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((amax 0.0) (anorm 0.0) (bignum 0.0) (colcnd 0.0) (rcmax 0.0)
             (rcmin 0.0) (rowcnd 0.0) (rpvgrw 0.0) (smlnum 0.0) (i 0)
             (infequ 0) (j 0) (j1 0) (j2 0)
             (norm
              (make-array '(1) :element-type 'character :initial-element #\ ))
```

```
     (colequ nil) (equil nil) (nofact nil) (notran nil) (rowequ nil))
(declare
 (type (single-float) amax anorm bignum colcnd rcmax rcmin rowcnd
  rpvgrw smlnum)
 (type (f2cl-lib:integer4) i infequ j j1 j2)
 (type (string 1) norm)
 (type f2cl-lib:logical colequ equil nofact notran rowequ))
(setf info 0)
(setf nofact
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "N")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val))
(setf equil
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "E")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val))
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
  (setf rowequ f2cl-lib:%false%) (setf colequ f2cl-lib:%false%))
 (t
  (setf rowequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "R")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)))
  (setf colequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "C")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
```

```
                (lsame equed "B")
              (declare (ignore var-1))
              (when var-0 (setf equed var-0))
              ret-val)))
  (setf smlnum (slamch "Safe minimum")) (setf bignum (/ one smlnum))))
 (cond
  ((and (not nofact) (not equil)
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame fact "F")
           (declare (ignore var-1))
           (when var-0 (setf fact var-0))
           ret-val)))
   (setf info -1))
  ((and (not notran)
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame trans "T")
           (declare (ignore var-1))
           (when var-0 (setf trans var-0))
           ret-val))
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame trans "C")
           (declare (ignore var-1))
           (when var-0 (setf trans var-0))
           ret-val)))
   (setf info -2))
  ((< n 0) (setf info -3)) ((< kl 0) (setf info -4))
  ((< ku 0) (setf info -5)) ((< nrhs 0) (setf info -6))
  ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -8))
  ((< ldafb (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
   (setf info -10))
  ((and
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame fact "F")
      (declare (ignore var-1))
      (when var-0 (setf fact var-0))
      ret-val)
    (not
     (or rowequ colequ
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame equed "N")
           (declare (ignore var-1))
           (when var-0 (setf equed var-0))
           ret-val))))
   (setf info -12))
  (t
   (cond
    (rowequ (setf rcmin bignum) (setf rcmax zero)
```

```
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf rcmin
                (min rcmin
                     (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
        (setf rcmax
                (max rcmax
                     (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
       label10))
    (cond ((<= rcmin zero) (setf info -13))
          ((> n 0)
           (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum))))
          (t (setf rowcnd one)))))
  (cond
   ((and colequ (= info 0)) (setf rcmin bignum) (setf rcmax zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf rcmin
                (min rcmin
                     (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
        (setf rcmax
                (max rcmax
                     (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
       label20))
    (cond ((<= rcmin zero) (setf info -14))
          ((> n 0)
           (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum))))
          (t (setf colcnd one)))))
  (cond
   ((= info 0)
    (cond
     ((< ldb$
         (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -16))
     ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -18)))))))
(cond
 ((/= info 0) (xerbla "SGBSVX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (equil
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11)
      (sgbequ n n kl ku ab ldab r c rowcnd colcnd amax infequ)
    (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7))
    (setf rowcnd var-8)
    (setf colcnd var-9)
```

```
      (setf amax var-10)
      (setf infequ var-11))
    (cond
     ((= infequ 0)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11)
          (slaqgb n n kl ku ab ldab r c rowcnd colcnd amax equed)
        (declare (ignore var-4 var-6 var-7))
        (when var-0 (setf n var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf kl var-2))
        (when var-3 (setf ku var-3))
        (when var-5 (setf ldab var-5))
        (when var-8 (setf rowcnd var-8))
        (when var-9 (setf colcnd var-9))
        (when var-10 (setf amax var-10))
        (when var-11 (setf equed var-11)))
      (setf rowequ
              (or
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame equed "R")
                 (declare (ignore var-1))
                 (when var-0 (setf equed var-0))
                 ret-val)
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame equed "B")
                 (declare (ignore var-1))
                 (when var-0 (setf equed var-0))
                 ret-val)))
      (setf colequ
              (or
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame equed "C")
                 (declare (ignore var-1))
                 (when var-0 (setf equed var-0))
                 ret-val)
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame equed "B")
                 (declare (ignore var-1))
                 (when var-0 (setf equed var-0))
                 ret-val)))))))
  (cond
   (notran
    (cond
     (rowequ
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nrhs) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)
                    (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                       (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                      b-%offset%)))
             label30))
        label40)))))
  (colequ
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j nrhs) nil)
     (tagbody
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                 (* (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)
                    (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)))
          label50))
      label60))))
 (cond
  ((or nofact equil)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf j1
               (max (the f2cl-lib:integer4 (f2cl-lib:int-sub j ku))
                    (the f2cl-lib:integer4 1)))
       (setf j2
               (min (the f2cl-lib:integer4 (f2cl-lib:int-add j kl))
                    (the f2cl-lib:integer4 n)))
       (scopy (f2cl-lib:int-add (f2cl-lib:int-sub j2 j1) 1)
        (f2cl-lib:array-slice ab-%data% single-float
                              ((+ ku 1 (f2cl-lib:int-sub j) j1) j)
                              ((1 ldab) (1 *)) ab-%offset%)
        1
        (f2cl-lib:array-slice afb-%data% single-float
                              ((+ kl ku 1 (f2cl-lib:int-sub j) j1) j)
                              ((1 ldafb) (1 *)) afb-%offset%)
        1)
      label70))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (sgbtrf n n kl ku afb ldafb ipiv info)
     (declare (ignore var-4 var-6))
     (when var-0 (setf n var-0))
     (when var-1 (setf n var-1))
```

```
          (when var-2 (setf kl var-2))
          (when var-3 (setf ku var-3))
          (when var-5 (setf ldafb var-5))
          (when var-7 (setf info var-7)))
        (cond
         ((> info 0) (setf anorm zero)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j info) nil)
            (tagbody
              (f2cl-lib:fdo (i
                             (max
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add ku 2
                                                     (f2cl-lib:int-sub j)))
                              (the f2cl-lib:integer4 1))
                             (f2cl-lib:int-add i 1))
                            ((> i
                                (min
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add n ku 1
                                                        (f2cl-lib:int-sub
                                                         j)))
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add kl ku 1))))
                             nil)
              (tagbody
                (setf anorm
                        (max anorm
                             (abs
                              (f2cl-lib:fref ab-%data% (i j)
                                             ((1 ldab) (1 *))
                                             ab-%offset%))))
               label80))
            label90))
          (setf rpvgrw
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6
                       var-7)
                      (slantb "M" "U" "N" info
                       (min (the f2cl-lib:integer4 (f2cl-lib:int-sub info 1))
                            (the f2cl-lib:integer4 (f2cl-lib:int-add kl ku)))
                       (f2cl-lib:array-slice afb-%data% single-float
                                             ((max 1
                                                   (f2cl-lib:int-add kl ku 2
                                                                     (f2cl-lib:int-sub
                                                                      info)))
                                              1)
                                             ((1 ldafb) (1 *)) afb-%offset%)
                       ldafb work)
                    (declare (ignore var-0 var-1 var-2 var-4 var-5 var-7))
```

```
                (when var-3 (setf info var-3))
                (when var-6 (setf ldafb var-6))
                ret-val))
    (cond ((= rpvgrw zero) (setf rpvgrw one))
          (t (setf rpvgrw (/ anorm rpvgrw))))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) rpvgrw)
    (setf rcond zero) (go end_label)))))
(cond (notran (f2cl-lib:f2cl-set-string norm "1" (string 1)))
      (t (f2cl-lib:f2cl-set-string norm "I" (string 1))))
(setf anorm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (slangb norm n kl ku ab ldab work)
          (declare (ignore var-4 var-6))
          (when var-0 (setf norm var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf kl var-2))
          (when var-3 (setf ku var-3))
          (when var-5 (setf ldab var-5))
          ret-val))
(setf rpvgrw
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (slantb "M" "U" "N" n (f2cl-lib:int-add kl ku) afb ldafb
             work)
          (declare (ignore var-0 var-1 var-2 var-4 var-5 var-7))
          (when var-3 (setf n var-3))
          (when var-6 (setf ldafb var-6))
          ret-val))
(cond ((= rpvgrw zero) (setf rpvgrw one))
      (t
       (setf rpvgrw
               (/
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (slangb "M" n kl ku ab ldab work)
                  (declare (ignore var-0 var-4 var-6))
                  (when var-1 (setf n var-1))
                  (when var-2 (setf kl var-2))
                  (when var-3 (setf ku var-3))
                  (when var-5 (setf ldab var-5))
                  ret-val)
                rpvgrw))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11)
    (sgbcon norm n kl ku afb ldafb ipiv anorm rcond work iwork info)
  (declare (ignore var-2 var-3 var-4 var-6 var-7 var-9 var-10))
  (setf norm var-0)
  (setf n var-1)
```

```
  (setf ldafb var-5)
  (setf rcond var-8)
  (setf info var-11))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slacpy "Full" n nrhs b ldb$ x ldx)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf ldx var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (sgbtrs trans n kl ku nrhs afb ldafb ipiv x ldx info)
  (declare (ignore var-5 var-7 var-8))
  (when var-0 (setf trans var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf kl var-2))
  (when var-3 (setf ku var-3))
  (when var-4 (setf nrhs var-4))
  (when var-6 (setf ldafb var-6))
  (when var-9 (setf ldx var-9))
  (when var-10 (setf info var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18)
    (sgbrfs trans n kl ku nrhs ab ldab afb ldafb ipiv b ldb$ x ldx ferr
     berr work iwork info)
  (declare
   (ignore var-4 var-5 var-7 var-9 var-10 var-11 var-12 var-13 var-14
    var-15 var-16 var-17))
  (setf trans var-0)
  (setf n var-1)
  (setf kl var-2)
  (setf ku var-3)
  (setf ldab var-6)
  (setf ldafb var-8)
  (setf info var-18))
(cond
 (notran
  (cond
   (colequ
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                 x-%offset%)
```

```
                          (* (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)
                             (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                            x-%offset%)))
                  label100))
               label110))
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j nrhs) nil)
              (tagbody
                (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                      (/
                       (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                       colcnd))
               label120)))))
          (rowequ
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j nrhs) nil)
             (tagbody
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i n) nil)
                 (tagbody
                   (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                        x-%offset%)
                         (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                            (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                           x-%offset%)))
                  label130))
               label140))
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j nrhs) nil)
             (tagbody
               (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                     (/ (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                        rowcnd))
               label150))))
         (if (< rcond (slamch "Epsilon"))
             (setf info (f2cl-lib:int-add n 1)))
         (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) rpvgrw)
         (go end_label)
        end_label
         (return
          (values fact trans n kl ku nrhs nil ldab nil ldafb nil equed nil nil
                  nil ldb$ nil ldx rcond nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgbsvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
```

```
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (string) (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (single-float)
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::fact
  fortran-to-lisp::trans
  fortran-to-lisp::n
  fortran-to-lisp::kl
  fortran-to-lisp::ku
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::ldab nil
  fortran-to-lisp::ldafb nil
  fortran-to-lisp::equed nil nil
  nil fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond nil nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::sgbrfs
  fortran-to-lisp::sgbcon
  fortran-to-lisp::sgbequ))))
```

41.3 sgbsvxx

41.3.1 Axiom unit tests

— sgbsvxx.input —

```
)set break resume
)sys rm -f sgbsvxx.output
)spool sgbsvxx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

41.3.2 Axiom help page [Demm05]

— sgbsvxx.help —

```
======================================================================
sgbsvxx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgbsvxx (FACT, TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB,
           IPIV, EQUED, R, C, B, LDB, X, LDX, RCOND, RPVGRW, BERR, N_ERR_BNDS,
           ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, IWORK, INFO)
            SGBSVXX computes the solution to system of linear equations A * X
           = B for GB matrices

Function/Subroutine Documentation
   subroutine sgbsvxx (character FACT, character TRANS, integer N, integer KL,
       integer KU, integer NRHS, real, dimension( ldab, * ) AB, integer LDAB,
       real, dimension( ldafb, * ) AFB, integer LDAFB, integer, dimension( * )
       IPIV, character EQUED, real, dimension( * ) R, real, dimension( * ) C,
       real, dimension( ldb, * ) B, integer LDB, real, dimension( ldx , * ) X,
       integer LDX, real RCOND, real RPVGRW, real, dimension( * ) BERR,
       integer N_ERR_BNDS, real, dimension( nrhs, * ) ERR_BNDS_NORM, real,
```

```
      dimension( nrhs, * ) ERR_BNDS_COMP, integer NPARAMS, real, dimension( *
      ) PARAMS, real, dimension( * ) WORK, integer, dimension( * ) IWORK,
      integer INFO)
       SGBSVXX computes the solution to system of linear equations A * X = B
      for GB matrices

Purpose:
   SGBSVXX uses the LU factorization to compute the solution to a
   real system of linear equations  A * X = B,  where A is an
   N-by-N matrix and X and B are N-by-NRHS matrices.

   If requested, both normwise and maximum componentwise error bounds
   are returned. SGBSVXX will return a solution with a tiny
   guaranteed error (O(eps) where eps is the working machine
   precision) unless the matrix is very ill-conditioned, in which
   case a warning is returned. Relevant condition numbers also are
   calculated and returned.

   SGBSVXX accepts user-provided factorizations and equilibration
   factors; see the definitions of the FACT and EQUED options.
   Solving with refinement and using a factorization from a previous
   SGBSVXX call will also produce a solution with either O(eps)
   errors or warnings, but we cannot make that claim for general
   user-provided factorizations and equilibration factors if they
   differ from what SGBSVXX would itself produce.

Description:

   The following steps are performed:

   1. If FACT = 'E', real scaling factors are computed to equilibrate
      the system:

      TRANS = 'N':  diag(R)*A*diag(C)     *inv(diag(C))*X = diag(R)*B
      TRANS = 'T': (diag(R)*A*diag(C))**T *inv(diag(R))*X = diag(C)*B
      TRANS = 'C': (diag(R)*A*diag(C))**H *inv(diag(R))*X = diag(C)*B

      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(R)*A*diag(C) and B by diag(R)*B (if TRANS='N')
      or diag(C)*B (if TRANS = 'T' or 'C').

   2. If FACT = 'N' or 'E', the LU decomposition is used to factor
      the matrix A (after equilibration if FACT = 'E') as

      A = P * L * U,

      where P is a permutation matrix, L is a unit lower triangular
      matrix, and U is upper triangular.
```

```
   3. If some U(i,i)=0, so that U is exactly singular, then the
      routine returns with INFO = i. Otherwise, the factored form of A
      is used to estimate the condition number of the matrix A (see
      argument RCOND). If the reciprocal of the condition number is less
      than machine precision, the routine still goes on to solve for X
      and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. By default (unless PARAMS(LA_LINRX_ITREF_I) is set to zero),
      the routine will use iterative refinement to try to get a small
      error and error bounds.  Refinement calculates the residual to at
      least twice the working precision.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(C) (if TRANS = 'N') or diag(R) (if TRANS = 'T' or 'C') so
      that it solves the original system before equilibration.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AF and IPIV contain the factored form of A.
      If EQUED is not 'N', the matrix A has been
      equilibrated with scaling factors given by R and C.
      A, AF, and IPIV are not modified.
   = 'N':  The matrix A will be copied to AF and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
      copied to AF and factored.

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate Transpose = Transpose)

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.
```

```
KL
  KL is INTEGER
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

AB
  AB is REAL array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

  If FACT = 'F' and EQUED is not 'N', then AB must have been
  equilibrated by the scaling factors in R and/or C.  AB is not
  modified if FACT = 'F' or 'N', or if FACT = 'E' and
  EQUED = 'N' on exit.

  On exit, if EQUED .ne. 'N', A is scaled as follows:
  EQUED = 'R':  A := diag(R) * A
  EQUED = 'C':  A := A * diag(C)
  EQUED = 'B':  A := diag(R) * A * diag(C).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is REAL array, dimension (LDAFB,N)
  If FACT = 'F', then AFB is an input argument and on entry
  contains details of the LU factorization of the band matrix
  A, as computed by SGBTRF.  U is stored as an upper triangular
  band matrix with KL+KU superdiagonals in rows 1 to KL+KU+1,
  and the multipliers used during the factorization are stored
  in rows KL+KU+2 to 2*KL+KU+1.  If EQUED .ne. 'N', then AFB is
  the factored form of the equilibrated matrix A.

  If FACT = 'N', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
```

```
  of the equilibrated matrix A (see the description of A for
  the form of the equilibrated matrix).

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains the pivot indices from the factorization A = P*L*U
  as computed by SGETRF; row i of the matrix was interchanged
  with row IPIV(i).

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the equilibrated matrix A.

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
  = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
  = 'B':  Both row and column equilibration, i.e., A has been
          replaced by diag(R) * A * diag(C).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

R
  R is REAL array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
  EQUED = 'R' or 'B', each element of R must be positive.
  If R is output, each element of R is a power of the radix.
  If R is input, each element of R should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

C
  C is REAL array, dimension (N)
```

```
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.
  If C is output, each element of C is a power of the radix.
  If C is input, each element of C should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is REAL array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if TRANS = 'N' and EQUED = 'R' or 'B', B is overwritten by diag(R)*B;
  if TRANS = 'T' or 'C' and EQUED = 'C' or 'B', B is
     overwritten by diag(C)*B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is REAL array, dimension (LDX,NRHS)
  If INFO = 0, the N-by-NRHS solution matrix X to the original
  system of equations.  Note that A and B are modified on exit
  if EQUED .ne. 'N', and the solution to the equilibrated system is
  inv(diag(C))*X if TRANS = 'N' and EQUED = 'C' or 'B', or
  inv(diag(R))*X if TRANS = 'T' or 'C' and EQUED = 'R' or 'B'.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

RPVGRW
  RPVGRW is REAL
  Reciprocal pivot growth.  On exit, this contains the reciprocal
```

```
  pivot growth factor norm(A)/norm(U). The "max absolute element"
  norm is used.  If this is much less than 1, then the stability of
  the LU factorization of the (equilibrated) matrix A could be poor.
  This also means that the solution X, estimated condition numbers,
  and error bounds could be unreliable. If factorization fails with
  0<INFO<=N, then this contains the reciprocal pivot growth factor
  for the leading INFO columns of A.  In SGESVX, this quantity is
  returned in WORK(1).

BERR
  BERR is REAL array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
            max_j (abs(XTRUE(j,i) - X(j,i)))
             ------------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
```

```
      be trusted if the previous boolean is true.

   err = 3  Reciprocal condition number: Estimated normwise
      reciprocal condition number.  Compared with the threshold
      sqrt(n) * slamch('Epsilon') to determine if the error
      estimate is "guaranteed". These reciprocal condition
      numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
      appropriately scaled matrix Z.
      Let Z = S*A, where S scales each row by a power of the
      radix so all absolute row sums of Z are approximately 1.

 ERR_BNDS_COMP
   ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
   For each right-hand side, this array contains information about
   various error bounds and condition numbers corresponding to the
   componentwise relative error, which is defined as follows:

   Componentwise relative error in the ith solution vector:
                   abs(XTRUE(j,i) - X(j,i))
             max_j ----------------------
                         abs(X(j,i))

   The array is indexed by the right-hand side i (on which the
   componentwise relative error depends), and the type of error
   information as described below. There currently are up to three
   pieces of information returned for each right-hand side. If
   componentwise accuracy is not requested (PARAMS(3) = 0.0), then
   ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
   the first (:,N_ERR_BNDS) entries are returned.

   The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
   right-hand side.

   The second index in ERR_BNDS_COMP(:,err) contains the following
   three fields:
   err = 1 "Trust/don't trust" boolean. Trust the answer if the
      reciprocal condition number is less than the threshold
      sqrt(n) * slamch('Epsilon').

   err = 2 "Guaranteed" error bound: The estimated forward error,
      almost certainly within a factor of 10 of the true error
      so long as the next entry is greater than the threshold
      sqrt(n) * slamch('Epsilon'). This error bound should only
      be trusted if the previous boolean is true.

   err = 3  Reciprocal condition number: Estimated componentwise
      reciprocal condition number.  Compared with the threshold
      sqrt(n) * slamch('Epsilon') to determine if the error
      estimate is "guaranteed". These reciprocal condition
      numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
```

```
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is REAL array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
  (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is REAL array, dimension (4*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
```

```
= 0:  Successful exit. The solution to every right-hand side is
guaranteed.
< 0:  If INFO = -i, the i-th argument had an illegal value
> 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
   has been completed, but the factor U is exactly singular, so
   the solution and error bounds could not be computed. RCOND = 0
   is returned.
= N+J: The solution corresponding to the Jth right-hand side is
   not guaranteed. The solutions corresponding to other right-
   hand sides K with K > J may not be guaranteed as well, but
   only the first such right-hand side is reported. If a small
   componentwise error is not requested (PARAMS(3) = 0.0) then
   the Jth right-hand side is the first with a normwise error
   bound that is not guaranteed (the smallest J such
   that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
   the Jth right-hand side is the first with either a normwise or
   componentwise error bound that is not guaranteed (the smallest
   J such that either ERR_BNDS_NORM(J,1) = 0.0 or
   ERR_BNDS_COMP(J,1) = 0.0). See the definition of
   ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
   about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

———-

41.3.3 fortran code

— sgbsvxx.f —

```
*  ==================================================================
      SUBROUTINE SGBSVXX( FACT, TRANS, N, KL, KU, NRHS, AB, LDAB, AFB,
     $                    LDAFB, IPIV, EQUED, R, C, B, LDB, X, LDX,
     $                    RCOND, RPVGRW, BERR, N_ERR_BNDS,
     $                    ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $                    WORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, TRANS
      INTEGER            INFO, LDAB, LDAFB, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      REAL               RCOND, RPVGRW
*     ..
*     .. Array Arguments ..
```

```
      INTEGER            IPIV( * ), IWORK( * )
      REAL               AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   X( LDX , * ),WORK( * )
      REAL               R( * ), C( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  =================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
*     ..
*     .. Local Scalars ..
      LOGICAL            COLEQU, EQUIL, NOFACT, NOTRAN, ROWEQU
      INTEGER            INFEQU, I, J, KL, KU
      REAL               AMAX, BIGNUM, COLCND, RCMAX, RCMIN,
     $                   ROWCND, SMLNUM
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, SLAMCH, SLA_GBRPVGRW
      LOGICAL            LSAME
      REAL               SLAMCH, SLA_GBRPVGRW
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGBEQUB, SGBTRF, SGBTRS, SLACPY, SLAQGB,
     $                   XERBLA, SLASCL2, SGBRFSX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      NOTRAN = LSAME( TRANS, 'N' )
      SMLNUM = SLAMCH( 'Safe minimum' )
      BIGNUM = ONE / SMLNUM
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
```

```
         ROWEQU = .FALSE.
         COLEQU = .FALSE.
      ELSE
         ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
         COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
      END IF
*
*     Default is failure.  If an input parameter is wrong or
*     factorization fails, make everything look horrible.  Only the
*     pivot growth is set here, the rest is initialized in SGBRFSX.
*
      RPVGRW = ZERO
*
*     Test the input parameters.  PARAMS is not tested until SGBRFSX.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.
     $     LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $        LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( KL.LT.0 ) THEN
         INFO = -4
      ELSE IF( KU.LT.0 ) THEN
         INFO = -5
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -6
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -8
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
         INFO = -10
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $        ( ROWEQU .OR. COLEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -12
      ELSE
         IF( ROWEQU ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 10 J = 1, N
               RCMIN = MIN( RCMIN, R( J ) )
               RCMAX = MAX( RCMAX, R( J ) )
 10         CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -13
            ELSE IF( N.GT.0 ) THEN
               ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               ROWCND = ONE
```

```
         END IF
      END IF
      IF( COLEQU .AND. INFO.EQ.0 ) THEN
         RCMIN = BIGNUM
         RCMAX = ZERO
         DO 20 J = 1, N
            RCMIN = MIN( RCMIN, C( J ) )
            RCMAX = MAX( RCMAX, C( J ) )
20       CONTINUE
         IF( RCMIN.LE.ZERO ) THEN
            INFO = -14
         ELSE IF( N.GT.0 ) THEN
            COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
         ELSE
            COLCND = ONE
         END IF
      END IF
      IF( INFO.EQ.0 ) THEN
         IF( LDB.LT.MAX( 1, N ) ) THEN
            INFO = -15
         ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
            INFO = -16
         END IF
      END IF
   END IF
*
   IF( INFO.NE.0 ) THEN
      CALL XERBLA( 'SGBSVXX', -INFO )
      RETURN
   END IF
*
   IF( EQUIL ) THEN
*
*     Compute row and column scalings to equilibrate the matrix A.
*
      CALL SGBEQUB( N, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $        AMAX, INFEQU )
      IF( INFEQU.EQ.0 ) THEN
*
*     Equilibrate the matrix.
*
         CALL SLAQGB( N, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $           AMAX, EQUED )
         ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
         COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
      END IF
*
*     If the scaling factors are not applied, set them to 1.0.
*
      IF ( .NOT.ROWEQU ) THEN
```

```
            DO J = 1, N
               R( J ) = 1.0
            END DO
         END IF
         IF ( .NOT.COLEQU ) THEN
            DO J = 1, N
               C( J ) = 1.0
            END DO
         END IF
      END IF
*
*     Scale the right hand side.
*
      IF( NOTRAN ) THEN
         IF( ROWEQU ) CALL SLASCL2(N, NRHS, R, B, LDB)
      ELSE
         IF( COLEQU ) CALL SLASCL2(N, NRHS, C, B, LDB)
      END IF
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the LU factorization of A.
*
         DO 40, J = 1, N
            DO 30, I = KL+1, 2*KL+KU+1
               AFB( I, J ) = AB( I-KL, J )
 30         CONTINUE
 40      CONTINUE
         CALL SGBTRF( N, N, KL, KU, AFB, LDAFB, IPIV, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 ) THEN
*
*           Pivot in column INFO is exactly 0
*           Compute the reciprocal pivot growth factor of the
*           leading rank-deficient INFO columns of A.
*
            RPVGRW = SLA_GBRPVGRW( N, KL, KU, INFO, AB, LDAB, AFB,
     $           LDAFB )
            RETURN
         END IF
      END IF
*
*     Compute the reciprocal pivot growth factor RPVGRW.
*
      RPVGRW = SLA_GBRPVGRW( N, KL, KU, N, AB, LDAB, AFB, LDAFB )
*
*     Compute the solution matrix X.
*
```

```
      CALL SLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL SGBTRS( TRANS, N, KL, KU, NRHS, AFB, LDAFB, IPIV, X, LDX,
     $     INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL SGBRFSX( TRANS, EQUED, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB,
     $     IPIV, R, C, B, LDB, X, LDX, RCOND, BERR,
     $     N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $     WORK, IWORK, INFO )
*
*     Scale solutions.
*
      IF ( COLEQU .AND. NOTRAN ) THEN
         CALL SLASCL2 ( N, NRHS, C, X, LDX )
      ELSE IF ( ROWEQU .AND. .NOT.NOTRAN ) THEN
         CALL SLASCL2 ( N, NRHS, R, X, LDX )
      END IF
*
      RETURN
*
*     End of SGBSVXX
*
      END
```

———-

41.3.4 lisp code

— **sgbsvxx.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sgbsvxx
         (fact trans n kl ku nrhs ab ldab afb ldafb ipiv equed r c b ldb$ x ldx
          rcond rpvgrw berr n_err_bnds err_bnds_norm err_bnds_comp nparams
          params work iwork info)
    (declare (type (single-float) rpvgrw rcond)
             (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array single-float (*)) work params err_bnds_comp
              err_bnds_norm berr x b c r afb ab)
             (type (f2cl-lib:integer4) info nparams n_err_bnds ldx ldb$ ldafb
              ldab nrhs ku kl n)
             (type (string *) equed trans fact))
    (f2cl-lib:with-multi-array-data
```

```
  ((fact character fact-%data% fact-%offset%)
   (trans character trans-%data% trans-%offset%)
   (equed character equed-%data% equed-%offset%)
   (ab single-float ab-%data% ab-%offset%)
   (afb single-float afb-%data% afb-%offset%)
   (r single-float r-%data% r-%offset%)
   (c single-float c-%data% c-%offset%)
   (b single-float b-%data% b-%offset%)
   (x single-float x-%data% x-%offset%)
   (berr single-float berr-%data% berr-%offset%)
   (err_bnds_norm single-float err_bnds_norm-%data%
    err_bnds_norm-%offset%)
   (err_bnds_comp single-float err_bnds_comp-%data%
    err_bnds_comp-%offset%)
   (params single-float params-%data% params-%offset%)
   (work single-float work-%data% work-%offset%)
   (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
   (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
(prog ((amax 0.0) (bignum 0.0) (colcnd 0.0) (rcmax 0.0) (rcmin 0.0)
       (rowcnd 0.0) (smlnum 0.0) (infequ 0) (i 0) (j 0) (colequ nil)
       (equil nil) (nofact nil) (notran nil) (rowequ nil))
  (declare
   (type (single-float) amax bignum colcnd rcmax rcmin rowcnd smlnum)
   (type (f2cl-lib:integer4) infequ i j)
   (type f2cl-lib:logical colequ equil nofact notran rowequ))
  (setf info 0)
  (setf nofact
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame fact "N")
            (declare (ignore var-1))
            (when var-0 (setf fact var-0))
            ret-val))
  (setf equil
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame fact "E")
            (declare (ignore var-1))
            (when var-0 (setf fact var-0))
            ret-val))
  (setf notran
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame trans "N")
            (declare (ignore var-1))
            (when var-0 (setf trans var-0))
            ret-val))
  (setf smlnum (slamch "Safe minimum"))
  (setf bignum (/ one smlnum))
  (cond
   ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
    (setf rowequ f2cl-lib:%false%) (setf colequ f2cl-lib:%false%))
   (t
```

```
  (setf rowequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "R")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)))
  (setf colequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "C")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)))))
(setf rpvgrw zero)
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< kl 0) (setf info -4))
 ((< ku 0) (setf info -5)) ((< nrhs 0) (setf info -6))
```

```
((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -8))
((< ldafb (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
 (setf info -10))
((and
  (multiple-value-bind (ret-val var-0 var-1)
      (lsame fact "F")
    (declare (ignore var-1))
    (when var-0 (setf fact var-0))
    ret-val)
  (not
   (or rowequ colequ
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame equed "N")
         (declare (ignore var-1))
         (when var-0 (setf equed var-0))
         ret-val))))
 (setf info -12))
(t
 (cond
  (rowequ (setf rcmin bignum) (setf rcmax zero)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf rcmin
               (min rcmin
                    (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
       (setf rcmax
               (max rcmax
                    (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
      label10))
   (cond ((<= rcmin zero) (setf info -13))
         ((> n 0)
          (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum))))
         (t (setf rowcnd one)))))
 (cond
  ((and colequ (= info 0)) (setf rcmin bignum) (setf rcmax zero)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf rcmin
               (min rcmin
                    (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
       (setf rcmax
               (max rcmax
                    (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
      label20))
   (cond ((<= rcmin zero) (setf info -14))
         ((> n 0)
          (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum))))
         (t (setf colcnd one)))))
```

```
  (cond
   ((= info 0)
    (cond
     ((< ldb$
         (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -15))
     ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -16)))))))
(cond
 ((/= info 0) (xerbla "SGBSVXX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (equil
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11)
      (sgbequb n n kl ku ab ldab r c rowcnd colcnd amax infequ)
    (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7))
    (setf rowcnd var-8)
    (setf colcnd var-9)
    (setf amax var-10)
    (setf infequ var-11))
  (cond
   ((= infequ 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11)
        (slaqgb n n kl ku ab ldab r c rowcnd colcnd amax equed)
      (declare (ignore var-4 var-6 var-7))
      (when var-0 (setf n var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf kl var-2))
      (when var-3 (setf ku var-3))
      (when var-5 (setf ldab var-5))
      (when var-8 (setf rowcnd var-8))
      (when var-9 (setf colcnd var-9))
      (when var-10 (setf amax var-10))
      (when var-11 (setf equed var-11)))
    (setf rowequ
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "R")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "B")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)))
```

```
      (setf colequ
              (or
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame equed "C")
                 (declare (ignore var-1))
                 (when var-0 (setf equed var-0))
                 ret-val)
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame equed "B")
                 (declare (ignore var-1))
                 (when var-0 (setf equed var-0))
                 ret-val)))))
  (cond
   ((not rowequ)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%) 1.0)
       label100000))))
  (cond
   ((not colequ)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) 1.0)
       label100001))))))
(cond
 (notran
  (if rowequ
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (slascl2 n nrhs r b ldb$)
        (declare (ignore var-2 var-3))
        (when var-0 (setf n var-0))
        (when var-1 (setf nrhs var-1))
        (when var-4 (setf ldb$ var-4)))))
 (t
  (if colequ
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (slascl2 n nrhs c b ldb$)
        (declare (ignore var-2 var-3))
        (when var-0 (setf n var-0))
        (when var-1 (setf nrhs var-1))
        (when var-4 (setf ldb$ var-4))))))
(cond
 ((or nofact equil)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i (f2cl-lib:int-add kl 1) (f2cl-lib:int-add i 1))
                    ((> i
```

```
                          (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku
                                            1))
                      nil)
            (tagbody
              (setf (f2cl-lib:fref afb-%data% (i j) ((1 ldafb) (1 *))
                                   afb-%offset%)
                    (f2cl-lib:fref ab-%data% ((f2cl-lib:int-sub i kl) j)
                                   ((1 ldab) (1 *)) ab-%offset%))
             label30))
         label40))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (sgbtrf n n kl ku afb ldafb ipiv info)
        (declare (ignore var-4 var-5 var-6))
        (setf n var-0)
        (setf n var-1)
        (setf kl var-2)
        (setf ku var-3)
        (setf info var-7))
      (cond
       ((> info 0)
        (setf rpvgrw
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6
                     var-7)
                    (sla_gbrpvgrw n kl ku info ab ldab afb ldafb)
                  (declare (ignore var-4 var-6))
                  (when var-0 (setf n var-0))
                  (when var-1 (setf kl var-1))
                  (when var-2 (setf ku var-2))
                  (when var-3 (setf info var-3))
                  (when var-5 (setf ldab var-5))
                  (when var-7 (setf ldafb var-7))
                  ret-val))
        (go end_label)))))
    (setf rpvgrw
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
                (sla_gbrpvgrw n kl ku n ab ldab afb ldafb)
              (declare (ignore var-4 var-6))
              (when var-0 (setf n var-0))
              (when var-1 (setf kl var-1))
              (when var-2 (setf ku var-2))
              (when var-3 (setf n var-3))
              (when var-5 (setf ldab var-5))
              (when var-7 (setf ldafb var-7))
              ret-val))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slacpy "Full" n nrhs b ldb$ x ldx)
      (declare (ignore var-0 var-3 var-5))
```

```
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf ldx var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (sgbtrs trans n kl ku nrhs afb ldafb ipiv x ldx info)
  (declare (ignore var-2 var-3 var-5 var-7 var-8))
  (setf trans var-0)
  (setf n var-1)
  (setf nrhs var-4)
  (setf ldafb var-6)
  (setf ldx var-9)
  (setf info var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19
     var-20 var-21 var-22 var-23 var-24 var-25 var-26)
    (sgbrfsx trans equed n kl ku nrhs ab ldab afb ldafb ipiv r c b ldb$
     x ldx rcond berr n_err_bnds err_bnds_norm err_bnds_comp nparams
     params work iwork info)
  (declare
   (ignore var-6 var-8 var-10 var-11 var-12 var-13 var-15 var-18 var-20
    var-21 var-23 var-24 var-25))
  (when var-0 (setf trans var-0))
  (when var-1 (setf equed var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf kl var-3))
  (when var-4 (setf ku var-4))
  (when var-5 (setf nrhs var-5))
  (when var-7 (setf ldab var-7))
  (when var-9 (setf ldafb var-9))
  (when var-14 (setf ldb$ var-14))
  (when var-16 (setf ldx var-16))
  (when var-17 (setf rcond var-17))
  (when var-19 (setf n_err_bnds var-19))
  (when var-22 (setf nparams var-22))
  (when var-26 (setf info var-26)))
(cond
 ((and colequ notran)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slascl2 n nrhs c x ldx)
    (declare (ignore var-2 var-3))
    (when var-0 (setf n var-0))
    (when var-1 (setf nrhs var-1))
    (when var-4 (setf ldx var-4))))
 ((and rowequ (not notran))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slascl2 n nrhs r x ldx)
```

```
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf nrhs var-1))
            (when var-4 (setf ldx var-4)))))
        (go end_label)
       end_label
        (return
         (values fact trans n kl ku nrhs nil ldab nil ldafb nil equed nil nil
                 nil ldb$ nil ldx rcond rpvgrw nil n_err_bnds nil nil nparams
                 nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgbsvxx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (string) (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::fact
                                              fortran-to-lisp::trans
                                              fortran-to-lisp::n
                                              fortran-to-lisp::kl
```

```
  fortran-to-lisp::ku
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::ldab nil
  fortran-to-lisp::ldafb nil
  fortran-to-lisp::equed nil nil
  nil fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond
  fortran-to-lisp::rpvgrw nil
  fortran-to-lisp::n_err_bnds nil
  nil fortran-to-lisp::nparams nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::sgbtrs
  fortran-to-lisp::sgbtrf
  fortran-to-lisp::sgbequb))))
```

———-

Chapter 42

LAPACK - General Band Matrix, Linear Solve, Complex

42.1 cgbsv

42.1.1 Axiom unit tests

— cgbsv.input —

```
)set break resume
)sys rm -f cgbsv.output
)spool cgbsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

42.1.2 Axiom help page

— cgbsv.help —

```
==========================================================================
cgbsv examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgbsv (N, KL, KU, NRHS, AB, LDAB, IPIV, B, LDB, INFO)
            CGBSV computes the solution to system of linear equations A * X =
           B for GB matrices (simple driver)

Function/Subroutine Documentation
   subroutine cgbsv (integer N, integer KL, integer KU, integer NRHS, complex,
       dimension( ldab, * ) AB, integer LDAB, integer, dimension( * ) IPIV,
       complex, dimension( ldb, * ) B, integer LDB, integer INFO)
        CGBSV computes the solution to system of linear equations A * X = B
       for GB matrices (simple driver)

Purpose:
   CGBSV computes the solution to a complex system of linear equations
   A * X = B, where A is a band matrix of order N with KL subdiagonals
   and KU superdiagonals, and X and B are N-by-NRHS matrices.

   The LU decomposition with partial pivoting and row interchanges is
   used to factor A as A = L * U, where L is a product of permutation
   and unit lower triangular matrices with KL subdiagonals, and U is
   upper triangular with KL+KU superdiagonals.  The factored form of A
   is then used to solve the system of equations A * X = B.

Parameters:

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 AB
```

```
  AB is COMPLEX array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows KL+1 to
  2*KL+KU+1; rows 1 to KL of the array need not be set.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(KL+KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+KL)
  On exit, details of the factorization: U is stored as an
  upper triangular band matrix with KL+KU superdiagonals in
  rows 1 to KL+KU+1, and the multipliers used during the
  factorization are stored in rows KL+KU+2 to 2*KL+KU+1.
  See below for further details.

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices that define the permutation matrix P;
  row i of the matrix was interchanged with row IPIV(i).

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, U(i,i) is exactly zero.  The factorization
        has been completed, but the factor U is exactly
        singular, and the solution has not been computed.

The band storage scheme is illustrated by the following example, when
    M = N = 6, KL = 2, KU = 1:

    On entry:                       On exit:

        *    *    *    +    +    +       *    *    *   u14  u25  u36
        *    *    +    +    +    +       *    *   u13  u24  u35  u46
        *   a12  a23  a34  a45  a56      *   u12  u23  u34  u45  u56
       a11  a22  a33  a44  a55  a66     u11  u22  u33  u44  u55  u66
       a21  a32  a43  a54  a65   *      m21  m32  m43  m54  m65   *
       a31  a42  a53  a64   *    *      m31  m42  m53  m64   *    *
```

```
Array elements marked * are not used by the routine; elements marked
+ need not be set on entry, but are required by the routine to store
elements of U because of fill-in resulting from the row interchanges.
```

———-

42.1.3 fortran code

— **cgbsv.f** —

```
*  =====================================================================
      SUBROUTINE CGBSV( N, KL, KU, NRHS, AB, LDAB, IPIV, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, KL, KU, LDAB, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            AB( LDAB, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. External Subroutines ..
      EXTERNAL           CGBTRF, CGBTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( KL.LT.0 ) THEN
         INFO = -2
      ELSE IF( KU.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
```

```
      ELSE IF( LDAB.LT.2*KL+KU+1 ) THEN
         INFO = -6
      ELSE IF( LDB.LT.MAX( N, 1 ) ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGBSV ', -INFO )
         RETURN
      END IF
*
*     Compute the LU factorization of the band matrix A.
*
      CALL CGBTRF( N, N, KL, KU, AB, LDAB, IPIV, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
         CALL CGBTRS( 'No transpose', N, KL, KU, NRHS, AB, LDAB, IPIV,
     $                B, LDB, INFO )
      END IF
      RETURN
*
*     End of CGBSV
*
      END
```

———-

42.1.4 lisp code

— cgbsv.lisp —

```
(defun cgbsv (n kl ku nrhs ab ldab ipiv b ldb$ info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex8 (*)) b ab)
           (type (f2cl-lib:integer4) info ldb$ ldab nrhs ku kl n))
  (f2cl-lib:with-multi-array-data
      ((ab f2cl-lib:complex8 ab-%data% ab-%offset%)
       (b f2cl-lib:complex8 b-%data% b-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ()
      (declare)
      (setf info 0)
      (cond ((< n 0) (setf info -1)) ((< kl 0) (setf info -2))
            ((< ku 0) (setf info -3)) ((< nrhs 0) (setf info -4))
            ((< ldab (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
             (setf info -6))
            ((< ldb$ (max (the f2cl-lib:integer4 n) (the f2cl-lib:integer4 1)))
```

```
            (setf info -9)))
      (cond
       ((/= info 0) (xerbla "CGBSV " (f2cl-lib:int-sub info)) (go end_label)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (cgbtrf n n kl ku ab ldab ipiv info)
        (declare (ignore var-4 var-6))
        (when var-0 (setf n var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf kl var-2))
        (when var-3 (setf ku var-3))
        (when var-5 (setf ldab var-5))
        (when var-7 (setf info var-7)))
      (cond
       ((= info 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10)
            (cgbtrs "No transpose" n kl ku nrhs ab ldab ipiv b ldb$ info)
          (declare (ignore var-0 var-5 var-7 var-8))
          (when var-1 (setf n var-1))
          (when var-2 (setf kl var-2))
          (when var-3 (setf ku var-3))
          (when var-4 (setf nrhs var-4))
          (when var-6 (setf ldab var-6))
          (when var-9 (setf ldb$ var-9))
          (when var-10 (setf info var-10)))))
      (go end_label)
     end_label
      (return (values n kl ku nrhs nil ldab nil nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgbsv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n
```

```
                                                  fortran-to-lisp::kl
                                                  fortran-to-lisp::ku
                                                  fortran-to-lisp::nrhs nil
                                                  fortran-to-lisp::ldab nil nil
                                                  fortran-to-lisp::ldb$
                                                  fortran-to-lisp::info)
                                                :calls 'nil)))
```

———-

42.2 cgbsvx

42.2.1 Axiom unit tests

— **cgbsvx.input** —

```
)set break resume
)sys rm -f cgbsvx.output
)spool cgbsvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

42.2.2 Axiom help page

— **cgbsvx.help** —

```
==================================================================
cgbsvx examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
```

```
      subroutine cgbsvx (FACT, TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB,
          IPIV, EQUED, R, C, B, LDB, X, LDX, RCOND, FERR, BERR, WORK, RWORK,
          INFO)
           CGBSVX computes the solution to system of linear equations A * X =
          B for GB matrices

Function/Subroutine Documentation
   subroutine cgbsvx (character FACT, character TRANS, integer N, integer KL,
       integer KU, integer NRHS, complex, dimension( ldab, * ) AB, integer
       LDAB, complex, dimension( ldafb, * ) AFB, integer LDAFB, integer,
       dimension( * ) IPIV, character EQUED, real, dimension( * ) R, real,
       dimension( * ) C, complex, dimension( ldb, * ) B, integer LDB, complex,
       dimension( ldx, * ) X, integer LDX, real RCOND, real, dimension( * )
       FERR, real, dimension( * ) BERR, complex, dimension( * ) WORK, real,
       dimension( * ) RWORK, integer INFO)
        CGBSVX computes the solution to system of linear equations A * X = B
       for GB matrices

Purpose:
   CGBSVX uses the LU factorization to compute the solution to a complex
   system of linear equations A * X = B, A**T * X = B, or A**H * X = B,
   where A is a band matrix of order N with KL subdiagonals and KU
   superdiagonals, and X and B are N-by-NRHS matrices.

   Error bounds on the solution and a condition estimate are also
   provided.

Description:
   The following steps are performed by this subroutine:

   1. If FACT = 'E', real scaling factors are computed to equilibrate
      the system:

   TRANS = 'N':  diag(R)*A*diag(C)     *inv(diag(C))*X = diag(R)*B
   TRANS = 'T': (diag(R)*A*diag(C))**T *inv(diag(R))*X = diag(C)*B
   TRANS = 'C': (diag(R)*A*diag(C))**H *inv(diag(R))*X = diag(C)*B

   Whether or not the system will be equilibrated depends on the
   scaling of the matrix A, but if equilibration is used, A is
   overwritten by diag(R)*A*diag(C) and B by diag(R)*B (if TRANS='N')
   or diag(C)*B (if TRANS = 'T' or 'C').

   2. If FACT = 'N' or 'E', the LU decomposition is used to factor the
      matrix A (after equilibration if FACT = 'E') as A = L * U,
      where L is a product of permutation and unit lower triangular
      matrices with KL subdiagonals, and U is upper triangular with
      KL+KU superdiagonals.

   3. If some U(i,i)=0, so that U is exactly singular, then the routine
      returns with INFO = i. Otherwise, the factored form of A is used
```

```
      to estimate the condition number of the matrix A.  If the
      reciprocal of the condition number is less than machine precision,
      INFO = N+1 is returned as a warning, but the routine still goes on
      to solve for X and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(C) (if TRANS = 'N') or diag(R) (if TRANS = 'T' or 'C') so
      that it solves the original system before equilibration.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AFB and IPIV contain the factored form of
           A.  If EQUED is not 'N', the matrix A has been
           equilibrated with scaling factors given by R and C.
           AB, AFB, and IPIV are not modified.
   = 'N':  The matrix A will be copied to AFB and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
           copied to AFB and factored.

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations.
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose)

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 NRHS
```

```
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

AB
  AB is COMPLEX array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

  If FACT = 'F' and EQUED is not 'N', then A must have been
  equilibrated by the scaling factors in R and/or C.  AB is not
  modified if FACT = 'F' or 'N', or if FACT = 'E' and
  EQUED = 'N' on exit.

  On exit, if EQUED .ne. 'N', A is scaled as follows:
  EQUED = 'R':  A := diag(R) * A
  EQUED = 'C':  A := A * diag(C)
  EQUED = 'B':  A := diag(R) * A * diag(C).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is COMPLEX array, dimension (LDAFB,N)
  If FACT = 'F', then AFB is an input argument and on entry
  contains details of the LU factorization of the band matrix
  A, as computed by CGBTRF.  U is stored as an upper triangular
  band matrix with KL+KU superdiagonals in rows 1 to KL+KU+1,
  and the multipliers used during the factorization are stored
  in rows KL+KU+2 to 2*KL+KU+1.  If EQUED .ne. 'N', then AFB is
  the factored form of the equilibrated matrix A.

  If FACT = 'N', then AFB is an output argument and on exit
  returns details of the LU factorization of A.

  If FACT = 'E', then AFB is an output argument and on exit
  returns details of the LU factorization of the equilibrated
  matrix A (see the description of AB for the form of the
  equilibrated matrix).

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
```

```
  contains the pivot indices from the factorization A = L*U
  as computed by CGBTRF; row i of the matrix was interchanged
  with row IPIV(i).

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = L*U
  of the original matrix A.

  If FACT = 'E', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = L*U
  of the equilibrated matrix A.

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
  = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
  = 'B':  Both row and column equilibration, i.e., A has been
          replaced by diag(R) * A * diag(C).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

R
  R is REAL array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
  EQUED = 'R' or 'B', each element of R must be positive.

C
  C is REAL array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if TRANS = 'N' and EQUED = 'R' or 'B', B is overwritten by diag(R)*B;
  if TRANS = 'T' or 'C' and EQUED = 'C' or 'B', B is
     overwritten by diag(C)*B.

LDB
  LDB is INTEGER
```

```
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X
  to the original system of equations.  Note that A and B are
  modified on exit if EQUED .ne. 'N', and the solution to the
  equilibrated system is inv(diag(C))*X if TRANS = 'N' and
  EQUED = 'C' or 'B', or inv(diag(R))*X if TRANS = 'T' or 'C'
  and EQUED = 'R' or 'B'.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  The estimate of the reciprocal condition number of the matrix
  A after equilibration (if done).  If RCOND is less than the
  machine precision (in particular, if RCOND = 0), the matrix
  is singular to working precision.  This condition is
  indicated by a return code of INFO > 0.

FERR
  FERR is REAL array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is REAL array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is COMPLEX array, dimension (2*N)

RWORK
  RWORK is REAL array, dimension (N)
  On exit, RWORK(1) contains the reciprocal pivot growth
  factor norm(A)/norm(U). The "max absolute element" norm is
  used. If RWORK(1) is much less than 1, then the stability
  of the LU factorization of the (equilibrated) matrix A
  could be poor. This also means that the solution X, condition
```

```
  estimator RCOND, and forward error bound FERR could be
  unreliable. If factorization fails with 0<INFO<=N, then
  RWORK(1) contains the reciprocal pivot growth factor for the
  leading INFO columns of A.

 INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, and i is
         <= N:  U(i,i) is exactly zero.  The factorization
                has been completed, but the factor U is exactly
                singular, so the solution and error bounds
                could not be computed. RCOND = 0 is returned.
         = N+1: U is nonsingular, but RCOND is less than machine
                precision, meaning that the matrix is singular
                to working precision.  Nevertheless, the
                solution and error bounds are computed because
                there are a number of situations where the
                computed solution can be more accurate than the
                value of RCOND would suggest.
```

———-

42.2.3 fortran code

— cgbsvx.f —

```
*  ====================================================================
      SUBROUTINE CGBSVX( FACT, TRANS, N, KL, KU, NRHS, AB, LDAB, AFB,
     $                   LDAFB, IPIV, EQUED, R, C, B, LDB, X, LDX,
     $                   RCOND, FERR, BERR, WORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, TRANS
      INTEGER            INFO, KL, KU, LDAB, LDAFB, LDB, LDX, N, NRHS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               BERR( * ), C( * ), FERR( * ), R( * ),
     $                   RWORK( * )
```

```
      COMPLEX            AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*  Moved setting of INFO = N+1 so INFO does not subsequently get
*  overwritten.  Sven, 17 Mar 05.
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            COLEQU, EQUIL, NOFACT, NOTRAN, ROWEQU
      CHARACTER          NORM
      INTEGER            I, INFEQU, J, J1, J2
      REAL               AMAX, ANORM, BIGNUM, COLCND, RCMAX, RCMIN,
     $                   ROWCND, RPVGRW, SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               CLANGB, CLANTB, SLAMCH
      EXTERNAL           LSAME, CLANGB, CLANTB, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           CCOPY, CGBCON, CGBEQU, CGBRFS, CGBTRF, CGBTRS,
     $                   CLACPY, CLAQGB, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      NOTRAN = LSAME( TRANS, 'N' )
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         ROWEQU = .FALSE.
         COLEQU = .FALSE.
      ELSE
         ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
         COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         SMLNUM = SLAMCH( 'Safe minimum' )
         BIGNUM = ONE / SMLNUM
      END IF
*
*     Test the input parameters.
```

```
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.LSAME( FACT, 'F' ) )
     $     THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $         LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( KL.LT.0 ) THEN
         INFO = -4
      ELSE IF( KU.LT.0 ) THEN
         INFO = -5
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -6
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -8
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
         INFO = -10
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $         ( ROWEQU .OR. COLEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -12
      ELSE
         IF( ROWEQU ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 10 J = 1, N
               RCMIN = MIN( RCMIN, R( J ) )
               RCMAX = MAX( RCMAX, R( J ) )
   10       CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -13
            ELSE IF( N.GT.0 ) THEN
               ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               ROWCND = ONE
            END IF
         END IF
         IF( COLEQU .AND. INFO.EQ.0 ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 20 J = 1, N
               RCMIN = MIN( RCMIN, C( J ) )
               RCMAX = MAX( RCMAX, C( J ) )
   20       CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -14
            ELSE IF( N.GT.0 ) THEN
               COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
```

```
            COLCND = ONE
         END IF
      END IF
      IF( INFO.EQ.0 ) THEN
         IF( LDB.LT.MAX( 1, N ) ) THEN
            INFO = -16
         ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
            INFO = -18
         END IF
      END IF
   END IF
*
   IF( INFO.NE.0 ) THEN
      CALL XERBLA( 'CGBSVX', -INFO )
      RETURN
   END IF
*
   IF( EQUIL ) THEN
*
*     Compute row and column scalings to equilibrate the matrix A.
*
      CALL CGBEQU( N, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $                AMAX, INFEQU )
      IF( INFEQU.EQ.0 ) THEN
*
*        Equilibrate the matrix.
*
         CALL CLAQGB( N, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $                   AMAX, EQUED )
         ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
         COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
      END IF
   END IF
*
*  Scale the right hand side.
*
   IF( NOTRAN ) THEN
      IF( ROWEQU ) THEN
         DO 40 J = 1, NRHS
            DO 30 I = 1, N
               B( I, J ) = R( I )*B( I, J )
30          CONTINUE
40       CONTINUE
      END IF
   ELSE IF( COLEQU ) THEN
      DO 60 J = 1, NRHS
         DO 50 I = 1, N
            B( I, J ) = C( I )*B( I, J )
50       CONTINUE
60    CONTINUE
```

```
      END IF
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the LU factorization of the band matrix A.
*
         DO 70 J = 1, N
            J1 = MAX( J-KU, 1 )
            J2 = MIN( J+KL, N )
            CALL CCOPY( J2-J1+1, AB( KU+1-J+J1, J ), 1,
     $                  AFB( KL+KU+1-J+J1, J ), 1 )
   70    CONTINUE
*
         CALL CGBTRF( N, N, KL, KU, AFB, LDAFB, IPIV, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 ) THEN
*
*           Compute the reciprocal pivot growth factor of the
*           leading rank-deficient INFO columns of A.
*
            ANORM = ZERO
            DO 90 J = 1, INFO
               DO 80 I = MAX( KU+2-J, 1 ), MIN( N+KU+1-J, KL+KU+1 )
                  ANORM = MAX( ANORM, ABS( AB( I, J ) ) )
   80          CONTINUE
   90       CONTINUE
            RPVGRW = CLANTB( 'M', 'U', 'N', INFO, MIN( INFO-1, KL+KU ),
     $                       AFB( MAX( 1, KL+KU+2-INFO ), 1 ), LDAFB,
     $                       RWORK )
            IF( RPVGRW.EQ.ZERO ) THEN
               RPVGRW = ONE
            ELSE
               RPVGRW = ANORM / RPVGRW
            END IF
            RWORK( 1 ) = RPVGRW
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A and the
*     reciprocal pivot growth factor RPVGRW.
*
      IF( NOTRAN ) THEN
         NORM = '1'
      ELSE
         NORM = 'I'
      END IF
```

```
      ANORM = CLANGB( NORM, N, KL, KU, AB, LDAB, RWORK )
      RPVGRW = CLANTB( 'M', 'U', 'N', N, KL+KU, AFB, LDAFB, RWORK )
      IF( RPVGRW.EQ.ZERO ) THEN
         RPVGRW = ONE
      ELSE
         RPVGRW = CLANGB( 'M', N, KL, KU, AB, LDAB, RWORK ) / RPVGRW
      END IF
*
*     Compute the reciprocal of the condition number of A.
*
      CALL CGBCON( NORM, N, KL, KU, AFB, LDAFB, IPIV, ANORM, RCOND,
     $             WORK, RWORK, INFO )
*
*     Compute the solution matrix X.
*
      CALL CLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL CGBTRS( TRANS, N, KL, KU, NRHS, AFB, LDAFB, IPIV, X, LDX,
     $             INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL CGBRFS( TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB, IPIV,
     $             B, LDB, X, LDX, FERR, BERR, WORK, RWORK, INFO )
*
*     Transform the solution matrix X to a solution of the original
*     system.
*
      IF( NOTRAN ) THEN
         IF( COLEQU ) THEN
            DO 110 J = 1, NRHS
               DO 100 I = 1, N
                  X( I, J ) = C( I )*X( I, J )
  100          CONTINUE
  110       CONTINUE
            DO 120 J = 1, NRHS
               FERR( J ) = FERR( J ) / COLCND
  120       CONTINUE
         END IF
      ELSE IF( ROWEQU ) THEN
         DO 140 J = 1, NRHS
            DO 130 I = 1, N
               X( I, J ) = R( I )*X( I, J )
  130       CONTINUE
  140    CONTINUE
         DO 150 J = 1, NRHS
            FERR( J ) = FERR( J ) / ROWCND
  150    CONTINUE
      END IF
*
```

```
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.SLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      RWORK( 1 ) = RPVGRW
      RETURN
*
*     End of CGBSVX
*
      END
```

———-

42.2.4 lisp code

— **cgbsvx.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun cgbsvx
         (fact trans n kl ku nrhs ab ldab afb ldafb ipiv equed r c b ldb$ x ldx
          rcond ferr berr work rwork info)
    (declare (type (single-float) rcond)
             (type (array single-float (*)) rwork berr ferr c r)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) work x b afb ab)
             (type (f2cl-lib:integer4) info ldx ldb$ ldafb ldab nrhs ku kl n)
             (type (string *) equed trans fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (trans character trans-%data% trans-%offset%)
         (equed character equed-%data% equed-%offset%)
         (ab f2cl-lib:complex8 ab-%data% ab-%offset%)
         (afb f2cl-lib:complex8 afb-%data% afb-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (r single-float r-%data% r-%offset%)
         (c single-float c-%data% c-%offset%)
         (ferr single-float ferr-%data% ferr-%offset%)
         (berr single-float berr-%data% berr-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%))
      (prog ((amax 0.0) (anorm 0.0) (bignum 0.0) (colcnd 0.0) (rcmax 0.0)
             (rcmin 0.0) (rowcnd 0.0) (rpvgrw 0.0) (smlnum 0.0) (i 0)
```

```
     (infequ 0) (j 0) (j1 0) (j2 0)
     (norm
      (make-array '(1) :element-type 'character :initial-element #\ ))
     (colequ nil) (equil nil) (nofact nil) (notran nil) (rowequ nil))
(declare
 (type (single-float) amax anorm bignum colcnd rcmax rcmin rowcnd
  rpvgrw smlnum)
 (type (f2cl-lib:integer4) i infequ j j1 j2)
 (type (string 1) norm)
 (type f2cl-lib:logical colequ equil nofact notran rowequ))
(setf info 0)
(setf nofact
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "N")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val))
(setf equil
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "E")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val))
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
  (setf rowequ f2cl-lib:%false%) (setf colequ f2cl-lib:%false%))
 (t
  (setf rowequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "R")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)))
  (setf colequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "C")
             (declare (ignore var-1))
```

```
              (when var-0 (setf equed var-0))
              ret-val)
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame equed "B")
              (declare (ignore var-1))
              (when var-0 (setf equed var-0))
              ret-val)))
  (setf smlnum (slamch "Safe minimum")) (setf bignum (/ one smlnum))))
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< kl 0) (setf info -4))
 ((< ku 0) (setf info -5)) ((< nrhs 0) (setf info -6))
 ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -8))
 ((< ldafb (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
  (setf info -10))
 ((and
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame fact "F")
     (declare (ignore var-1))
     (when var-0 (setf fact var-0))
     ret-val)
   (not
    (or rowequ colequ
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame equed "N")
          (declare (ignore var-1))
          (when var-0 (setf equed var-0))
          ret-val))))
  (setf info -12))
```

```
(t
 (cond
  (rowequ (setf rcmin bignum) (setf rcmax zero)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf rcmin
               (min rcmin
                    (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
       (setf rcmax
               (max rcmax
                    (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
      label10))
   (cond ((<= rcmin zero) (setf info -13))
         ((> n 0)
          (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum))))
         (t (setf rowcnd one)))))
 (cond
  ((and colequ (= info 0)) (setf rcmin bignum) (setf rcmax zero)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf rcmin
               (min rcmin
                    (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
       (setf rcmax
               (max rcmax
                    (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
      label20))
   (cond ((<= rcmin zero) (setf info -14))
         ((> n 0)
          (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum))))
         (t (setf colcnd one)))))
 (cond
  ((= info 0)
   (cond
    ((< ldb$
        (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
     (setf info -16))
    ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
     (setf info -18)))))))
(cond
 ((/= info 0) (xerbla "CGBSVX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (equil
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11)
      (cgbequ n n kl ku ab ldab r c rowcnd colcnd amax infequ)
```

```
      (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7))
      (setf rowcnd var-8)
      (setf colcnd var-9)
      (setf amax var-10)
      (setf infequ var-11))
    (cond
     ((= infequ 0)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11)
          (claqgb n n kl ku ab ldab r c rowcnd colcnd amax equed)
        (declare (ignore var-4 var-6 var-7))
        (when var-0 (setf n var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf kl var-2))
        (when var-3 (setf ku var-3))
        (when var-5 (setf ldab var-5))
        (when var-8 (setf rowcnd var-8))
        (when var-9 (setf colcnd var-9))
        (when var-10 (setf amax var-10))
        (when var-11 (setf equed var-11)))
      (setf rowequ
              (or
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame equed "R")
                 (declare (ignore var-1))
                 (when var-0 (setf equed var-0))
                 ret-val)
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame equed "B")
                 (declare (ignore var-1))
                 (when var-0 (setf equed var-0))
                 ret-val)))
      (setf colequ
              (or
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame equed "C")
                 (declare (ignore var-1))
                 (when var-0 (setf equed var-0))
                 ret-val)
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame equed "B")
                 (declare (ignore var-1))
                 (when var-0 (setf equed var-0))
                 ret-val)))))))
  (cond
   (notran
    (cond
     (rowequ
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                    ((> j nrhs) nil)
          (tagbody
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                     b-%offset%)
                      (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                         (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                        b-%offset%)))
              label30))
           label40)))))
 (colequ
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)
                (* (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)))
         label50))
     label60))))
(cond
 ((or nofact equil)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf j1
              (max (the f2cl-lib:integer4 (f2cl-lib:int-sub j ku))
                   (the f2cl-lib:integer4 1)))
      (setf j2
              (min (the f2cl-lib:integer4 (f2cl-lib:int-add j kl))
                   (the f2cl-lib:integer4 n)))
      (ccopy (f2cl-lib:int-add (f2cl-lib:int-sub j2 j1) 1)
       (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                             ((+ ku 1 (f2cl-lib:int-sub j) j1) j)
                             ((1 ldab) (1 *)) ab-%offset%)
       1
       (f2cl-lib:array-slice afb-%data% f2cl-lib:complex8
                             ((+ kl ku 1 (f2cl-lib:int-sub j) j1) j)
                             ((1 ldafb) (1 *)) afb-%offset%)
       1)
     label70))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (cgbtrf n n kl ku afb ldafb ipiv info)
```

```
  (declare (ignore var-4 var-6))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf kl var-2))
  (when var-3 (setf ku var-3))
  (when var-5 (setf ldafb var-5))
  (when var-7 (setf info var-7)))
(cond
 ((> info 0) (setf anorm zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j info) nil)
    (tagbody
      (f2cl-lib:fdo (i
                     (max
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add ku 2
                                             (f2cl-lib:int-sub j)))
                      (the f2cl-lib:integer4 1))
                     (f2cl-lib:int-add i 1))
                    ((> i
                        (min
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add n ku 1
                                                (f2cl-lib:int-sub
                                                 j)))
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add kl ku 1))))
                     nil)
        (tagbody
          (setf anorm
                  (max anorm
                       (abs
                        (f2cl-lib:fref ab-%data% (i j)
                                       ((1 ldab) (1 *))
                                       ab-%offset%))))
         label80))
     label90))
  (setf rpvgrw
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6
               var-7)
              (clantb "M" "U" "N" info
               (min (the f2cl-lib:integer4 (f2cl-lib:int-sub info 1))
                    (the f2cl-lib:integer4 (f2cl-lib:int-add kl ku)))
               (f2cl-lib:array-slice afb-%data% f2cl-lib:complex8
                                     ((max 1
                                           (f2cl-lib:int-add kl ku 2
                                                             (f2cl-lib:int-sub
                                                              info)))
                                      1)
```

```
                                        ((1 ldafb) (1 *)) afb-%offset%)
                  ldafb rwork)
                (declare (ignore var-0 var-1 var-2 var-4 var-5 var-7))
                (when var-3 (setf info var-3))
                (when var-6 (setf ldafb var-6))
                ret-val))
    (cond ((= rpvgrw zero) (setf rpvgrw one))
          (t (setf rpvgrw (/ anorm rpvgrw))))
    (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
            rpvgrw)
    (setf rcond zero) (go end_label)))))
(cond (notran (f2cl-lib:f2cl-set-string norm "1" (string 1)))
      (t (f2cl-lib:f2cl-set-string norm "I" (string 1))))
(setf anorm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (clangb norm n kl ku ab ldab rwork)
          (declare (ignore var-4 var-6))
          (when var-0 (setf norm var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf kl var-2))
          (when var-3 (setf ku var-3))
          (when var-5 (setf ldab var-5))
          ret-val))
(setf rpvgrw
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (clantb "M" "U" "N" n (f2cl-lib:int-add kl ku) afb ldafb
             rwork)
          (declare (ignore var-0 var-1 var-2 var-4 var-5 var-7))
          (when var-3 (setf n var-3))
          (when var-6 (setf ldafb var-6))
          ret-val))
(cond ((= rpvgrw zero) (setf rpvgrw one))
      (t
       (setf rpvgrw
               (/
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (clangb "M" n kl ku ab ldab rwork)
                  (declare (ignore var-0 var-4 var-6))
                  (when var-1 (setf n var-1))
                  (when var-2 (setf kl var-2))
                  (when var-3 (setf ku var-3))
                  (when var-5 (setf ldab var-5))
                  ret-val)
                rpvgrw))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11)
```

```
    (cgbcon norm n kl ku afb ldafb ipiv anorm rcond work rwork info)
  (declare (ignore var-2 var-3 var-4 var-6 var-7 var-9 var-10))
  (setf norm var-0)
  (setf n var-1)
  (setf ldafb var-5)
  (setf rcond var-8)
  (setf info var-11))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "Full" n nrhs b ldb$ x ldx)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf ldx var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgbtrs trans n kl ku nrhs afb ldafb ipiv x ldx info)
  (declare (ignore var-5 var-7 var-8))
  (when var-0 (setf trans var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf kl var-2))
  (when var-3 (setf ku var-3))
  (when var-4 (setf nrhs var-4))
  (when var-6 (setf ldafb var-6))
  (when var-9 (setf ldx var-9))
  (when var-10 (setf info var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18)
    (cgbrfs trans n kl ku nrhs ab ldab afb ldafb ipiv b ldb$ x ldx ferr
     berr work rwork info)
  (declare
   (ignore var-4 var-5 var-7 var-9 var-10 var-11 var-12 var-13 var-14
    var-15 var-16 var-17))
  (setf trans var-0)
  (setf n var-1)
  (setf kl var-2)
  (setf ku var-3)
  (setf ldab var-6)
  (setf ldafb var-8)
  (setf info var-18))
(cond
 (notran
  (cond
   (colequ
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                              ((> i n) nil)
                    (tagbody
                      (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                           x-%offset%)
                            (* (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)
                               (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                              x-%offset%)))
                     label100))
                 label110))
              (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                            ((> j nrhs) nil)
                (tagbody
                  (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                          (/
                           (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                           colcnd))
                 label120)))))
            (rowequ
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j nrhs) nil)
               (tagbody
                 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                               ((> i n) nil)
                   (tagbody
                     (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                          x-%offset%)
                           (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                              (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                             x-%offset%)))
                    label130))
                label140))
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j nrhs) nil)
               (tagbody
                 (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                         (/ (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                            rowcnd))
                label150))))
            (if (< rcond (slamch "Epsilon"))
                (setf info (f2cl-lib:int-add n 1)))
            (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%) rpvgrw)
            (go end_label)
           end_label
            (return
             (values fact trans n kl ku nrhs nil ldab nil ldafb nil equed nil nil
                     nil ldb$ nil ldx rcond nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
```

```
(setf (gethash 'fortran-to-lisp::cgbsvx
               fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((string) (string)
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::integer4
                                             (*))
                                            (string) (array single-float (*))
                                            (array single-float (*))
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (single-float)
                                            (array single-float (*))
                                            (array single-float (*))
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (array single-float (*))
                                            (fortran-to-lisp::integer4))
                                          :return-values
                                          '(fortran-to-lisp::fact
                                            fortran-to-lisp::trans
                                            fortran-to-lisp::n
                                            fortran-to-lisp::kl
                                            fortran-to-lisp::ku
                                            fortran-to-lisp::nrhs nil
                                            fortran-to-lisp::ldab nil
                                            fortran-to-lisp::ldafb nil
                                            fortran-to-lisp::equed nil nil
                                            nil fortran-to-lisp::ldb$ nil
                                            fortran-to-lisp::ldx
                                            fortran-to-lisp::rcond nil nil
                                            nil nil fortran-to-lisp::info)
                                          :calls
                                          '(fortran-to-lisp::cgbrfs
                                            fortran-to-lisp::cgbcon
                                            fortran-to-lisp::cgbequ))))
```

———-

42.3 cgbsvxx

42.3.1 Axiom unit tests

— **cgbsvxx.input** —

```
)set break resume
)sys rm -f cgbsvxx.output
)spool cgbsvxx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

42.3.2 Axiom help page

— **cgbsvxx.help** —

```
==================================================================
cgbsvxx examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgbsvxx (FACT, TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB,
           IPIV, EQUED, R, C, B, LDB, X, LDX, RCOND, RPVGRW, BERR, N_ERR_BNDS,
           ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, RWORK, INFO)
            CGBSVXX computes the solution to system of linear equations A * X
           = B for GB matrices

Function/Subroutine Documentation
   subroutine cgbsvxx (character FACT, character TRANS, integer N, integer KL,
```

```
      integer KU, integer NRHS, complex, dimension( ldab, * ) AB, integer
      LDAB, complex, dimension( ldafb, * ) AFB, integer LDAFB, integer,
      dimension( * ) IPIV, character EQUED, real, dimension( * ) R, real,
      dimension( * ) C, complex, dimension( ldb, * ) B, integer LDB, complex,
      dimension( ldx , * ) X, integer LDX, real RCOND, real RPVGRW, real,
      dimension( * ) BERR, integer N_ERR_BNDS, real, dimension( nrhs, * )
      ERR_BNDS_NORM, real, dimension( nrhs, * ) ERR_BNDS_COMP, integer
      NPARAMS, real, dimension( * ) PARAMS, complex, dimension( * ) WORK,
      real, dimension( * ) RWORK, integer INFO)
       CGBSVXX computes the solution to system of linear equations A * X = B
      for GB matrices

Purpose:
   CGBSVXX uses the LU factorization to compute the solution to a
   complex system of linear equations  A * X = B,  where A is an
   N-by-N matrix and X and B are N-by-NRHS matrices.

   If requested, both normwise and maximum componentwise error bounds
   are returned. CGBSVXX will return a solution with a tiny
   guaranteed error (O(eps) where eps is the working machine
   precision) unless the matrix is very ill-conditioned, in which
   case a warning is returned. Relevant condition numbers also are
   calculated and returned.

   CGBSVXX accepts user-provided factorizations and equilibration
   factors; see the definitions of the FACT and EQUED options.
   Solving with refinement and using a factorization from a previous
   CGBSVXX call will also produce a solution with either O(eps)
   errors or warnings, but we cannot make that claim for general
   user-provided factorizations and equilibration factors if they
   differ from what CGBSVXX would itself produce.

Description:
   The following steps are performed:

   1. If FACT = 'E', real scaling factors are computed to equilibrate
      the system:

   TRANS = 'N':  diag(R)*A*diag(C)     *inv(diag(C))*X = diag(R)*B
   TRANS = 'T': (diag(R)*A*diag(C))**T *inv(diag(R))*X = diag(C)*B
   TRANS = 'C': (diag(R)*A*diag(C))**H *inv(diag(R))*X = diag(C)*B

   Whether or not the system will be equilibrated depends on the
   scaling of the matrix A, but if equilibration is used, A is
   overwritten by diag(R)*A*diag(C) and B by diag(R)*B (if TRANS='N')
   or diag(C)*B (if TRANS = 'T' or 'C').

   2. If FACT = 'N' or 'E', the LU decomposition is used to factor
      the matrix A (after equilibration if FACT = 'E') as
```

```
   A = P * L * U,

   where P is a permutation matrix, L is a unit lower triangular
   matrix, and U is upper triangular.

   3. If some U(i,i)=0, so that U is exactly singular, then the
   routine returns with INFO = i. Otherwise, the factored form of A
   is used to estimate the condition number of the matrix A (see
   argument RCOND). If the reciprocal of the condition number is less
   than machine precision, the routine still goes on to solve for X
   and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. By default (unless PARAMS(LA_LINRX_ITREF_I) is set to zero),
   the routine will use iterative refinement to try to get a small
   error and error bounds.  Refinement calculates the residual to at
   least twice the working precision.

   6. If equilibration was used, the matrix X is premultiplied by
   diag(C) (if TRANS = 'N') or diag(R) (if TRANS = 'T' or 'C') so
   that it solves the original system before equilibration.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AF and IPIV contain the factored form of A.
           If EQUED is not 'N', the matrix A has been
           equilibrated with scaling factors given by R and C.
           A, AF, and IPIV are not modified.
   = 'N':  The matrix A will be copied to AF and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
            copied to AF and factored.

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate Transpose = Transpose)
```

```
N
  N is INTEGER
  The number of linear equations, i.e., the order of the
  matrix A.  N >= 0.

KL
  KL is INTEGER
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

AB
  AB is COMPLEX array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

  If FACT = 'F' and EQUED is not 'N', then AB must have been
  equilibrated by the scaling factors in R and/or C.  AB is not
  modified if FACT = 'F' or 'N', or if FACT = 'E' and
  EQUED = 'N' on exit.

  On exit, if EQUED .ne. 'N', A is scaled as follows:
  EQUED = 'R':  A := diag(R) * A
  EQUED = 'C':  A := A * diag(C)
  EQUED = 'B':  A := diag(R) * A * diag(C).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is COMPLEX array, dimension (LDAFB,N)
  If FACT = 'F', then AFB is an input argument and on entry
  contains details of the LU factorization of the band matrix
  A, as computed by CGBTRF.  U is stored as an upper triangular
  band matrix with KL+KU superdiagonals in rows 1 to KL+KU+1,
  and the multipliers used during the factorization are stored
  in rows KL+KU+2 to 2*KL+KU+1.  If EQUED .ne. 'N', then AFB is
  the factored form of the equilibrated matrix A.
```

```
  If FACT = 'N', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the equilibrated matrix A (see the description of A for
  the form of the equilibrated matrix).

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains the pivot indices from the factorization A = P*L*U
  as computed by SGETRF; row i of the matrix was interchanged
  with row IPIV(i).

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the equilibrated matrix A.

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
  = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
  = 'B':  Both row and column equilibration, i.e., A has been
          replaced by diag(R) * A * diag(C).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

R
  R is REAL array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
  EQUED = 'R' or 'B', each element of R must be positive.
  If R is output, each element of R is a power of the radix.
  If R is input, each element of R should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
```

```
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

C
  C is REAL array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.
  If C is output, each element of C is a power of the radix.
  If C is input, each element of C should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if TRANS = 'N' and EQUED = 'R' or 'B', B is overwritten by diag(R)*B;
  if TRANS = 'T' or 'C' and EQUED = 'C' or 'B', B is
     overwritten by diag(C)*B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  If INFO = 0, the N-by-NRHS solution matrix X to the original
  system of equations.  Note that A and B are modified on exit
  if EQUED .ne. 'N', and the solution to the equilibrated system is
  inv(diag(C))*X if TRANS = 'N' and EQUED = 'C' or 'B', or
  inv(diag(R))*X if TRANS = 'T' or 'C' and EQUED = 'R' or 'B'.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
```

```
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-
  conditioned.

RPVGRW
  RPVGRW is REAL
  Reciprocal pivot growth.  On exit, this contains the reciprocal
  pivot growth factor norm(A)/norm(U). The "max absolute element"
  norm is used.  If this is much less than 1, then the stability of
  the LU factorization of the (equilibrated) matrix A could be poor.
  This also means that the solution X, estimated condition numbers,
  and error bounds could be unreliable. If factorization fails with
  0<INFO<=N, then this contains the reciprocal pivot growth factor
  for the leading INFO columns of A.  In SGESVX, this quantity is
  returned in WORK(1).

BERR
  BERR is REAL array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
             max_j (abs(XTRUE(j,i) - X(j,i)))
            ------------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
```

```
          reciprocal condition number is less than the threshold
          sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
          almost certainly within a factor of 10 of the true error
          so long as the next entry is greater than the threshold
          sqrt(n) * slamch('Epsilon'). This error bound should only
          be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
           reciprocal condition number.  Compared with the threshold
           sqrt(n) * slamch('Epsilon') to determine if the error
           estimate is "guaranteed". These reciprocal condition
           numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
           appropriately scaled matrix Z.
           Let Z = S*A, where S scales each row by a power of the
           radix so all absolute row sums of Z are approximately 1.

  See Lapack Working Note 165 for further details and extra cautions.

ERR_BNDS_COMP
  ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                    abs(XTRUE(j,i) - X(j,i))
             max_j ----------------------
                         abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
           reciprocal condition number is less than the threshold
           sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
          almost certainly within a factor of 10 of the true error
```

```
          so long as the next entry is greater than the threshold
          sqrt(n) * slamch('Epsilon'). This error bound should only
          be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
           reciprocal condition number.  Compared with the threshold
           sqrt(n) * slamch('Epsilon') to determine if the error
           estimate is "guaranteed". These reciprocal condition
           numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
           appropriately scaled matrix Z.
           Let Z = S*(A*diag(x)), where x is the solution for the
           current right-hand side and S scales each row of
           A*diag(x) by a power of the radix so all absolute row
           sums of Z are approximately 1.

  See Lapack Working Note 165 for further details and extra cautions.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is REAL array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.

Default: 1.0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
          (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.

  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
```

```
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.

  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is COMPLEX array, dimension (2*N)

RWORK
  RWORK is REAL array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is
        guaranteed.
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
        has been completed, but the factor U is exactly singular, so
        the solution and error bounds could not be computed. RCOND = 0
        is returned.
  = N+J: The solution corresponding to the Jth right-hand side is
         not guaranteed. The solutions corresponding to other right-
         hand sides K with K > J may not be guaranteed as well, but
         only the first such right-hand side is reported. If a small
         componentwise error is not requested (PARAMS(3) = 0.0) then
         the Jth right-hand side is the first with a normwise error
         bound that is not guaranteed (the smallest J such
         that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
         the Jth right-hand side is the first with either a normwise or
         componentwise error bound that is not guaranteed (the smallest
         J such that either ERR_BNDS_NORM(J,1) = 0.0 or
         ERR_BNDS_COMP(J,1) = 0.0). See the definition of
         ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
         about all of the right-hand sides check ERR_BNDS_NORM or
         ERR_BNDS_COMP.
```

———-

42.3.3 fortran code

— cgbsvxx.f —

```
*  ==================================================================
      SUBROUTINE CGBSVXX( FACT, TRANS, N, KL, KU, NRHS, AB, LDAB, AFB,
     $                    LDAFB, IPIV, EQUED, R, C, B, LDB, X, LDX,
     $                    RCOND, RPVGRW, BERR, N_ERR_BNDS,
```

```
     $                    ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $                    WORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, TRANS
      INTEGER            INFO, LDAB, LDAFB, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      REAL               RCOND, RPVGRW
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   X( LDX , * ),WORK( * )
      REAL               R( * ), C( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * ), RWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
*     ..
*     .. Local Scalars ..
      LOGICAL            COLEQU, EQUIL, NOFACT, NOTRAN, ROWEQU
      INTEGER            INFEQU, I, J, KL, KU
      REAL               AMAX, BIGNUM, COLCND, RCMAX, RCMIN,
     $                   ROWCND, SMLNUM
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, SLAMCH, CLA_GBRPVGRW
      LOGICAL            LSAME
      REAL               SLAMCH, CLA_GBRPVGRW
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGBEQUB, CGBTRF, CGBTRS, CLACPY, CLAQGB,
```

```
     $                   XERBLA, CLASCL2, CGBRFSX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      NOTRAN = LSAME( TRANS, 'N' )
      SMLNUM = SLAMCH( 'Safe minimum' )
      BIGNUM = ONE / SMLNUM
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         ROWEQU = .FALSE.
         COLEQU = .FALSE.
      ELSE
         ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
         COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
      END IF
*
*     Default is failure.  If an input parameter is wrong or
*     factorization fails, make everything look horrible.  Only the
*     pivot growth is set here, the rest is initialized in CGBRFSX.
*
      RPVGRW = ZERO
*
*     Test the input parameters.  PARAMS is not tested until SGERFSX.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.
     $     LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $        LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( KL.LT.0 ) THEN
         INFO = -4
      ELSE IF( KU.LT.0 ) THEN
         INFO = -5
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -6
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -8
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
         INFO = -10
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $        ( ROWEQU .OR. COLEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
```

```
         INFO = -12
      ELSE
         IF( ROWEQU ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 10 J = 1, N
               RCMIN = MIN( RCMIN, R( J ) )
               RCMAX = MAX( RCMAX, R( J ) )
 10         CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -13
            ELSE IF( N.GT.0 ) THEN
               ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               ROWCND = ONE
            END IF
         END IF
         IF( COLEQU .AND. INFO.EQ.0 ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 20 J = 1, N
               RCMIN = MIN( RCMIN, C( J ) )
               RCMAX = MAX( RCMAX, C( J ) )
 20         CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -14
            ELSE IF( N.GT.0 ) THEN
               COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               COLCND = ONE
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -15
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -16
            END IF
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGBSVXX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*     Compute row and column scalings to equilibrate the matrix A.
*
```

```
         CALL CGBEQUB( N, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $        AMAX, INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*     Equilibrate the matrix.
*
            CALL CLAQGB( N, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $           AMAX, EQUED )
            ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
            COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         END IF
*
*     If the scaling factors are not applied, set them to 1.0.
*
         IF ( .NOT.ROWEQU ) THEN
            DO J = 1, N
               R( J ) = 1.0
            END DO
         END IF
         IF ( .NOT.COLEQU ) THEN
            DO J = 1, N
               C( J ) = 1.0
            END DO
         END IF
      END IF
*
*     Scale the right-hand side.
*
      IF( NOTRAN ) THEN
         IF( ROWEQU ) CALL CLASCL2( N, NRHS, R, B, LDB )
      ELSE
         IF( COLEQU ) CALL CLASCL2( N, NRHS, C, B, LDB )
      END IF
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the LU factorization of A.
*
         DO 40, J = 1, N
            DO 30, I = KL+1, 2*KL+KU+1
               AFB( I, J ) = AB( I-KL, J )
 30         CONTINUE
 40      CONTINUE
         CALL CGBTRF( N, N, KL, KU, AFB, LDAFB, IPIV, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 ) THEN
*
*           Pivot in column INFO is exactly 0
```

```
*          Compute the reciprocal pivot growth factor of the
*          leading rank-deficient INFO columns of A.
*
           RPVGRW = CLA_GBRPVGRW( N, KL, KU, INFO, AB, LDAB, AFB,
     $           LDAFB )
           RETURN
        END IF
      END IF
*
*     Compute the reciprocal pivot growth factor RPVGRW.
*
      RPVGRW = CLA_GBRPVGRW( N, KL, KU, N, AB, LDAB, AFB, LDAFB )
*
*     Compute the solution matrix X.
*
      CALL CLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL CGBTRS( TRANS, N, KL, KU, NRHS, AFB, LDAFB, IPIV, X, LDX,
     $     INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL CGBRFSX( TRANS, EQUED, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB,
     $     IPIV, R, C, B, LDB, X, LDX, RCOND, BERR,
     $     N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $     WORK, RWORK, INFO )

*
*     Scale solutions.
*
      IF ( COLEQU .AND. NOTRAN ) THEN
         CALL CLASCL2( N, NRHS, C, X, LDX )
      ELSE IF ( ROWEQU .AND. .NOT.NOTRAN ) THEN
         CALL CLASCL2( N, NRHS, R, X, LDX )
      END IF
*
      RETURN
*
*     End of CGBSVXX
*
      END
```

———-

42.3.4 lisp code

— cgbsvxx.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun cgbsvxx
         (fact trans n kl ku nrhs ab ldab afb ldafb ipiv equed r c b ldb$ x ldx
          rcond rpvgrw berr n_err_bnds err_bnds_norm err_bnds_comp nparams
          params work rwork info)
    (declare (type (single-float) rpvgrw rcond)
             (type (array single-float (*)) rwork params err_bnds_comp
              err_bnds_norm berr c r)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) work x b afb ab)
             (type (f2cl-lib:integer4) info nparams n_err_bnds ldx ldb$ ldafb
              ldab nrhs ku kl n)
             (type (string *) equed trans fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (trans character trans-%data% trans-%offset%)
         (equed character equed-%data% equed-%offset%)
         (ab f2cl-lib:complex8 ab-%data% ab-%offset%)
         (afb f2cl-lib:complex8 afb-%data% afb-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (r single-float r-%data% r-%offset%)
         (c single-float c-%data% c-%offset%)
         (berr single-float berr-%data% berr-%offset%)
         (err_bnds_norm single-float err_bnds_norm-%data%
          err_bnds_norm-%offset%)
         (err_bnds_comp single-float err_bnds_comp-%data%
          err_bnds_comp-%offset%)
         (params single-float params-%data% params-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%))
      (prog ((amax 0.0) (bignum 0.0) (colcnd 0.0) (rcmax 0.0) (rcmin 0.0)
             (rowcnd 0.0) (smlnum 0.0) (infequ 0) (i 0) (j 0) (colequ nil)
             (equil nil) (nofact nil) (notran nil) (rowequ nil))
        (declare
         (type (single-float) amax bignum colcnd rcmax rcmin rowcnd smlnum)
         (type (f2cl-lib:integer4) infequ i j)
         (type f2cl-lib:logical colequ equil nofact notran rowequ))
        (setf info 0)
        (setf nofact
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame fact "N")
                  (declare (ignore var-1))
                  (when var-0 (setf fact var-0))
                  ret-val))
        (setf equil
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "E")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val))
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(setf smlnum (slamch "Safe minimum"))
(setf bignum (/ one smlnum))
(cond
 ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
  (setf rowequ f2cl-lib:%false%) (setf colequ f2cl-lib:%false%))
 (t
  (setf rowequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "R")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)))
  (setf colequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "C")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)))))
(setf rpvgrw zero)
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
```

```
 (setf info -1))
((and (not notran)
      (not
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame trans "T")
         (declare (ignore var-1))
         (when var-0 (setf trans var-0))
         ret-val))
      (not
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame trans "C")
         (declare (ignore var-1))
         (when var-0 (setf trans var-0))
         ret-val)))
 (setf info -2))
((< n 0) (setf info -3)) ((< kl 0) (setf info -4))
((< ku 0) (setf info -5)) ((< nrhs 0) (setf info -6))
((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -8))
((< ldafb (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
 (setf info -10))
((and
  (multiple-value-bind (ret-val var-0 var-1)
      (lsame fact "F")
    (declare (ignore var-1))
    (when var-0 (setf fact var-0))
    ret-val)
  (not
   (or rowequ colequ
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame equed "N")
         (declare (ignore var-1))
         (when var-0 (setf equed var-0))
         ret-val))))
 (setf info -12))
(t
 (cond
  (rowequ (setf rcmin bignum) (setf rcmax zero)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf rcmin
               (min rcmin
                    (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
       (setf rcmax
               (max rcmax
                    (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
      label10))
   (cond ((<= rcmin zero) (setf info -13))
         ((> n 0)
          (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum))))
```

```
            (t (setf rowcnd one)))))
    (cond
     ((and colequ (= info 0)) (setf rcmin bignum) (setf rcmax zero)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf rcmin
                  (min rcmin
                       (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
          (setf rcmax
                  (max rcmax
                       (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
         label20))
      (cond ((<= rcmin zero) (setf info -14))
            ((> n 0)
             (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum))))
            (t (setf colcnd one)))))
    (cond
     ((= info 0)
      (cond
       ((< ldb$
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -15))
       ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -16)))))))
  (cond
   ((/= info 0) (xerbla "CGBSVXX" (f2cl-lib:int-sub info))
    (go end_label)))
  (cond
   (equil
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11)
        (cgbequb n n kl ku ab ldab r c rowcnd colcnd amax infequ)
      (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7))
      (setf rowcnd var-8)
      (setf colcnd var-9)
      (setf amax var-10)
      (setf infequ var-11))
    (cond
     ((= infequ 0)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11)
          (claqgb n n kl ku ab ldab r c rowcnd colcnd amax equed)
        (declare (ignore var-4 var-6 var-7))
        (when var-0 (setf n var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf kl var-2))
        (when var-3 (setf ku var-3))
```

```
        (when var-5 (setf ldab var-5))
        (when var-8 (setf rowcnd var-8))
        (when var-9 (setf colcnd var-9))
        (when var-10 (setf amax var-10))
        (when var-11 (setf equed var-11)))
      (setf rowequ
              (or
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame equed "R")
                 (declare (ignore var-1))
                 (when var-0 (setf equed var-0))
                 ret-val)
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame equed "B")
                 (declare (ignore var-1))
                 (when var-0 (setf equed var-0))
                 ret-val)))
      (setf colequ
              (or
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame equed "C")
                 (declare (ignore var-1))
                 (when var-0 (setf equed var-0))
                 ret-val)
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame equed "B")
                 (declare (ignore var-1))
                 (when var-0 (setf equed var-0))
                 ret-val)))))
    (cond
     ((not rowequ)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%) 1.0)
         label100000))))
    (cond
     ((not colequ)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) 1.0)
         label100001))))))
  (cond
   (notran
    (if rowequ
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (clascl2 n nrhs r b ldb$)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n var-0))
```

```
            (when var-1 (setf nrhs var-1))
            (when var-4 (setf ldb$ var-4)))))
     (t
      (if colequ
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (clascl2 n nrhs c b ldb$)
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf nrhs var-1))
            (when var-4 (setf ldb$ var-4))))))
    (cond
     ((or nofact equil)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i (f2cl-lib:int-add kl 1) (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku
                                              1))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref afb-%data% (i j) ((1 ldafb) (1 *))
                                   afb-%offset%)
                    (f2cl-lib:fref ab-%data% ((f2cl-lib:int-sub i kl) j)
                                   ((1 ldab) (1 *)) ab-%offset%))
             label30))
         label40))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (cgbtrf n n kl ku afb ldafb ipiv info)
        (declare (ignore var-4 var-6))
        (when var-0 (setf n var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf kl var-2))
        (when var-3 (setf ku var-3))
        (when var-5 (setf ldafb var-5))
        (when var-7 (setf info var-7)))
      (cond
       ((> info 0)
        (setf rpvgrw
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6
                     var-7)
                    (cla_gbrpvgrw n kl ku info ab ldab afb ldafb)
                  (declare (ignore var-4 var-6))
                  (when var-0 (setf n var-0))
                  (when var-1 (setf kl var-1))
                  (when var-2 (setf ku var-2))
                  (when var-3 (setf info var-3))
                  (when var-5 (setf ldab var-5))
```

```
                (when var-7 (setf ldafb var-7))
                ret-val))
    (go end_label)))))
(setf rpvgrw
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (cla_gbrpvgrw n kl ku n ab ldab afb ldafb)
          (declare (ignore var-4 var-6))
          (when var-0 (setf n var-0))
          (when var-1 (setf kl var-1))
          (when var-2 (setf ku var-2))
          (when var-3 (setf n var-3))
          (when var-5 (setf ldab var-5))
          (when var-7 (setf ldafb var-7))
          ret-val))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "Full" n nrhs b ldb$ x ldx)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf ldx var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgbtrs trans n kl ku nrhs afb ldafb ipiv x ldx info)
  (declare (ignore var-5 var-7 var-8))
  (when var-0 (setf trans var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf kl var-2))
  (when var-3 (setf ku var-3))
  (when var-4 (setf nrhs var-4))
  (when var-6 (setf ldafb var-6))
  (when var-9 (setf ldx var-9))
  (when var-10 (setf info var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19
     var-20 var-21 var-22 var-23 var-24 var-25 var-26)
    (cgbrfsx trans equed n kl ku nrhs ab ldab afb ldafb ipiv r c b ldb$
     x ldx rcond berr n_err_bnds err_bnds_norm err_bnds_comp nparams
     params work rwork info)
  (declare
   (ignore var-6 var-8 var-10 var-11 var-12 var-13 var-15 var-18 var-20
    var-21 var-23 var-24 var-25))
  (when var-0 (setf trans var-0))
  (when var-1 (setf equed var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf kl var-3))
  (when var-4 (setf ku var-4))
```

```
          (when var-5 (setf nrhs var-5))
          (when var-7 (setf ldab var-7))
          (when var-9 (setf ldafb var-9))
          (when var-14 (setf ldb$ var-14))
          (when var-16 (setf ldx var-16))
          (when var-17 (setf rcond var-17))
          (when var-19 (setf n_err_bnds var-19))
          (when var-22 (setf nparams var-22))
          (when var-26 (setf info var-26)))
        (cond
         ((and colequ notran)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (clascl2 n nrhs c x ldx)
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf nrhs var-1))
            (when var-4 (setf ldx var-4))))
         ((and rowequ (not notran))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (clascl2 n nrhs r x ldx)
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf nrhs var-1))
            (when var-4 (setf ldx var-4)))))
        (go end_label)
       end_label
        (return
         (values fact trans n kl ku nrhs nil ldab nil ldafb nil equed nil nil
                 nil ldb$ nil ldx rcond rpvgrw nil n_err_bnds nil nil nparams
                 nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgbsvxx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
```

```
  (string) (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (single-float) (single-float)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array fortran-to-lisp::complex8
   (*))
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::fact
  fortran-to-lisp::trans
  fortran-to-lisp::n
  fortran-to-lisp::kl
  fortran-to-lisp::ku
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::ldab nil
  fortran-to-lisp::ldafb nil
  fortran-to-lisp::equed nil nil
  nil fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond
  fortran-to-lisp::rpvgrw nil
  fortran-to-lisp::n_err_bnds nil
  nil fortran-to-lisp::nparams nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::cgbequb))))
```

Chapter 43

LAPACK - General Band Matrix, Linear Solve, Complex16

43.1 zgbsv

43.1.1 Axiom unit tests

— zgbsv.input —

```
)set break resume
)sys rm -f zgbsv.output
)spool zgbsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

43.1.2 Axiom help page

— zgbsv.help —

```
==========================================================================
```

```
zgbsv examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgbsv (N, KL, KU, NRHS, AB, LDAB, IPIV, B, LDB, INFO)
            ZGBSV computes the solution to system of linear equations A * X =
           B for GB matrices (simple driver)

Function/Subroutine Documentation
   subroutine zgbsv (integer N, integer KL, integer KU, integer NRHS,
       complex*16, dimension( ldab, * ) AB, integer LDAB, integer, dimension(
       * ) IPIV, complex*16, dimension( ldb, * ) B, integer LDB, integer INFO)
        ZGBSV computes the solution to system of linear equations A * X = B
       for GB matrices (simple driver)

Purpose:
   ZGBSV computes the solution to a complex system of linear equations
   A * X = B, where A is a band matrix of order N with KL subdiagonals
   and KU superdiagonals, and X and B are N-by-NRHS matrices.

   The LU decomposition with partial pivoting and row interchanges is
   used to factor A as A = L * U, where L is a product of permutation
   and unit lower triangular matrices with KL subdiagonals, and U is
   upper triangular with KL+KU superdiagonals.  The factored form of A
   is then used to solve the system of equations A * X = B.

Parameters:

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.
```

```
 AB
   AB is COMPLEX*16 array, dimension (LDAB,N)
   On entry, the matrix A in band storage, in rows KL+1 to
   2*KL+KU+1; rows 1 to KL of the array need not be set.
   The j-th column of A is stored in the j-th column of the
   array AB as follows:
   AB(KL+KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+KL)
   On exit, details of the factorization: U is stored as an
   upper triangular band matrix with KL+KU superdiagonals in
   rows 1 to KL+KU+1, and the multipliers used during the
   factorization are stored in rows KL+KU+2 to 2*KL+KU+1.
   See below for further details.

 LDAB
   LDAB is INTEGER
   The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.

 IPIV
   IPIV is INTEGER array, dimension (N)
   The pivot indices that define the permutation matrix P;
   row i of the matrix was interchanged with row IPIV(i).

 B
   B is COMPLEX*16 array, dimension (LDB,NRHS)
   On entry, the N-by-NRHS right hand side matrix B.
   On exit, if INFO = 0, the N-by-NRHS solution matrix X.

 LDB
   LDB is INTEGER
   The leading dimension of the array B.  LDB >= max(1,N).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, U(i,i) is exactly zero.  The factorization
         has been completed, but the factor U is exactly
         singular, and the solution has not been computed.

Further Details:

   The band storage scheme is illustrated by the following example, when
   M = N = 6, KL = 2, KU = 1:

        On entry:                       On exit:

            *    *    *    +    +    +       *    *    *   u14  u25  u36
            *    *    +    +    +    +       *    *   u13  u24  u35  u46
            *   a12  a23  a34  a45  a56      *   u12  u23  u34  u45  u56
           a11  a22  a33  a44  a55  a66     u11  u22  u33  u44  u55  u66
```

```
          a21  a32  a43  a54  a65   *      m21  m32  m43  m54  m65   *
          a31  a42  a53  a64   *    *      m31  m42  m53  m64   *    *

   Array elements marked * are not used by the routine; elements marked
   + need not be set on entry, but are required by the routine to store
   elements of U because of fill-in resulting from the row interchanges.
```

43.1.3 fortran code

— zgbsv.f —

```
*  =====================================================================
      SUBROUTINE ZGBSV( N, KL, KU, NRHS, AB, LDAB, IPIV, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, KL, KU, LDAB, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         AB( LDAB, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGBTRF, ZGBTRS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( KL.LT.0 ) THEN
         INFO = -2
      ELSE IF( KU.LT.0 ) THEN
```

```
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.2*KL+KU+1 ) THEN
         INFO = -6
      ELSE IF( LDB.LT.MAX( N, 1 ) ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGBSV ', -INFO )
         RETURN
      END IF
*
*     Compute the LU factorization of the band matrix A.
*
      CALL ZGBTRF( N, N, KL, KU, AB, LDAB, IPIV, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
         CALL ZGBTRS( 'No transpose', N, KL, KU, NRHS, AB, LDAB, IPIV,
     $                B, LDB, INFO )
      END IF
      RETURN
*
*     End of ZGBSV
*
      END
```

———-

43.1.4 lisp code

— zgbsv.lisp —

```
(defun zgbsv (n kl ku nrhs ab ldab ipiv b ldb$ info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex16 (*)) b ab)
           (type (f2cl-lib:integer4) info ldb$ ldab nrhs ku kl n))
  (f2cl-lib:with-multi-array-data
      ((ab f2cl-lib:complex16 ab-%data% ab-%offset%)
       (b f2cl-lib:complex16 b-%data% b-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ()
      (declare)
      (setf info 0)
      (cond ((< n 0) (setf info -1)) ((< kl 0) (setf info -2))
            ((< ku 0) (setf info -3)) ((< nrhs 0) (setf info -4))
```

```
            ((< ldab (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
             (setf info -6))
            ((< ldb$ (max (the f2cl-lib:integer4 n) (the f2cl-lib:integer4 1)))
             (setf info -9)))
      (cond
       ((/= info 0) (xerbla "ZGBSV " (f2cl-lib:int-sub info)) (go end_label)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (zgbtrf n n kl ku ab ldab ipiv info)
        (declare (ignore var-4 var-6))
        (when var-0 (setf n var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf kl var-2))
        (when var-3 (setf ku var-3))
        (when var-5 (setf ldab var-5))
        (when var-7 (setf info var-7)))
      (cond
       ((= info 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10)
            (zgbtrs "No transpose" n kl ku nrhs ab ldab ipiv b ldb$ info)
          (declare (ignore var-0 var-5 var-7 var-8))
          (when var-1 (setf n var-1))
          (when var-2 (setf kl var-2))
          (when var-3 (setf ku var-3))
          (when var-4 (setf nrhs var-4))
          (when var-6 (setf ldab var-6))
          (when var-9 (setf ldb$ var-9))
          (when var-10 (setf info var-10)))))
      (go end_label)
     end_label
      (return (values n kl ku nrhs nil ldab nil nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgbsv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
```

```
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::n
  fortran-to-lisp::kl
  fortran-to-lisp::ku
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::ldab nil nil
  fortran-to-lisp::ldb$
  fortran-to-lisp::info)
:calls '(fortran-to-lisp::xerbla))))
```

———-

43.2 zgbsvx

43.2.1 Axiom unit tests

— **zgbsvx.input** —

```
)set break resume
)sys rm -f zgbsvx.output
)spool zgbsvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

43.2.2 Axiom help page

— **zgbsvx.help** —

```
==========================================================================
zgbsvx examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine zgbsvx (FACT, TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB,
           IPIV, EQUED, R, C, B, LDB, X, LDX, RCOND, FERR, BERR, WORK, RWORK,
           INFO)
            ZGBSVX computes the solution to system of linear equations A * X =
           B for GB matrices

Function/Subroutine Documentation
   subroutine zgbsvx (character FACT, character TRANS, integer N, integer KL,
       integer KU, integer NRHS, complex*16, dimension( ldab, * ) AB, integer
       LDAB, complex*16, dimension( ldafb, * ) AFB, integer LDAFB, integer,
       dimension( * ) IPIV, character EQUED, double precision, dimension( * )
       R, double precision, dimension( * ) C, complex*16, dimension( ldb, * )
       B, integer LDB, complex*16, dimension( ldx, * ) X, integer LDX, double
       precision RCOND, double precision, dimension( * ) FERR, double
       precision, dimension( * ) BERR, complex*16, dimension( * ) WORK, double
       precision, dimension( * ) RWORK, integer INFO)
        ZGBSVX computes the solution to system of linear equations A * X = B
       for GB matrices

Purpose:
   ZGBSVX uses the LU factorization to compute the solution to a complex
   system of linear equations A * X = B, A**T * X = B, or A**H * X = B,
   where A is a band matrix of order N with KL subdiagonals and KU
   superdiagonals, and X and B are N-by-NRHS matrices.

   Error bounds on the solution and a condition estimate are also provided.

Description:

   The following steps are performed by this subroutine:

   1. If FACT = 'E', real scaling factors are computed to equilibrate
      the system:
      TRANS = 'N':  diag(R)*A*diag(C)     *inv(diag(C))*X = diag(R)*B
      TRANS = 'T': (diag(R)*A*diag(C))**T *inv(diag(R))*X = diag(C)*B
      TRANS = 'C': (diag(R)*A*diag(C))**H *inv(diag(R))*X = diag(C)*B
      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(R)*A*diag(C) and B by diag(R)*B (if TRANS='N')
      or diag(C)*B (if TRANS = 'T' or 'C').

   2. If FACT = 'N' or 'E', the LU decomposition is used to factor the
      matrix A (after equilibration if FACT = 'E') as
      A = L * U,
      where L is a product of permutation and unit lower triangular
      matrices with KL subdiagonals, and U is upper triangular with
      KL+KU superdiagonals.
```

```
   3. If some U(i,i)=0, so that U is exactly singular, then the routine
      returns with INFO = i. Otherwise, the factored form of A is used
      to estimate the condition number of the matrix A.  If the
      reciprocal of the condition number is less than machine precision,
      INFO = N+1 is returned as a warning, but the routine still goes on
      to solve for X and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(C) (if TRANS = 'N') or diag(R) (if TRANS = 'T' or 'C') so
      that it solves the original system before equilibration.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AFB and IPIV contain the factored form of
      A.  If EQUED is not 'N', the matrix A has been
      equilibrated with scaling factors given by R and C.
      AB, AFB, and IPIV are not modified.
   = 'N':  The matrix A will be copied to AFB and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
      copied to AFB and factored.

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations.
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose)

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.
```

```
NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

AB
  AB is COMPLEX*16 array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

  If FACT = 'F' and EQUED is not 'N', then A must have been
  equilibrated by the scaling factors in R and/or C.  AB is not
  modified if FACT = 'F' or 'N', or if FACT = 'E' and
  EQUED = 'N' on exit.

  On exit, if EQUED .ne. 'N', A is scaled as follows:
  EQUED = 'R':  A := diag(R) * A
  EQUED = 'C':  A := A * diag(C)
  EQUED = 'B':  A := diag(R) * A * diag(C).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is COMPLEX*16 array, dimension (LDAFB,N)
  If FACT = 'F', then AFB is an input argument and on entry
  contains details of the LU factorization of the band matrix
  A, as computed by ZGBTRF.  U is stored as an upper triangular
  band matrix with KL+KU superdiagonals in rows 1 to KL+KU+1,
  and the multipliers used during the factorization are stored
  in rows KL+KU+2 to 2*KL+KU+1.  If EQUED .ne. 'N', then AFB is
  the factored form of the equilibrated matrix A.

  If FACT = 'N', then AFB is an output argument and on exit
  returns details of the LU factorization of A.

  If FACT = 'E', then AFB is an output argument and on exit
  returns details of the LU factorization of the equilibrated
  matrix A (see the description of AB for the form of the
  equilibrated matrix).

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL+KU+1.

IPIV
```

```
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains the pivot indices from the factorization A = L*U
  as computed by ZGBTRF; row i of the matrix was interchanged
  with row IPIV(i).

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = L*U
  of the original matrix A.

  If FACT = 'E', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = L*U
  of the equilibrated matrix A.

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
  = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
  = 'B':  Both row and column equilibration, i.e., A has been
     replaced by diag(R) * A * diag(C).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

R
  R is DOUBLE PRECISION array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
  EQUED = 'R' or 'B', each element of R must be positive.

C
  C is DOUBLE PRECISION array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  On entry, the right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if TRANS = 'N' and EQUED = 'R' or 'B', B is overwritten by diag(R)*B;
  if TRANS = 'T' or 'C' and EQUED = 'C' or 'B', B is
     overwritten by diag(C)*B.
```

```
LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X
  to the original system of equations.  Note that A and B are
  modified on exit if EQUED .ne. 'N', and the solution to the
  equilibrated system is inv(diag(C))*X if TRANS = 'N' and
  EQUED = 'C' or 'B', or inv(diag(R))*X if TRANS = 'T' or 'C'
  and EQUED = 'R' or 'B'.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  The estimate of the reciprocal condition number of the matrix
  A after equilibration (if done).  If RCOND is less than the
  machine precision (in particular, if RCOND = 0), the matrix
  is singular to working precision.  This condition is
  indicated by a return code of INFO > 0.

FERR
  FERR is DOUBLE PRECISION array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is COMPLEX*16 array, dimension (2*N)

RWORK
  RWORK is DOUBLE PRECISION array, dimension (N)
  On exit, RWORK(1) contains the reciprocal pivot growth
  factor norm(A)/norm(U). The "max absolute element" norm is
  used. If RWORK(1) is much less than 1, then the stability
```

```
  of the LU factorization of the (equilibrated) matrix A
  could be poor. This also means that the solution X, condition
  estimator RCOND, and forward error bound FERR could be
  unreliable. If factorization fails with 0<INFO<=N, then
  RWORK(1) contains the reciprocal pivot growth factor for the
  leading INFO columns of A.

 INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, and i is
  <= N:  U(i,i) is exactly zero.  The factorization
     has been completed, but the factor U is exactly
     singular, so the solution and error bounds
     could not be computed. RCOND = 0 is returned.
  = N+1: U is nonsingular, but RCOND is less than machine
     precision, meaning that the matrix is singular
     to working precision.  Nevertheless, the
     solution and error bounds are computed because
     there are a number of situations where the
     computed solution can be more accurate than the
     value of RCOND would suggest.
```

———-

43.2.3 fortran code

— zgbsvx.f —

```
*  ====================================================================
      SUBROUTINE ZGBSVX( FACT, TRANS, N, KL, KU, NRHS, AB, LDAB, AFB,
     $                   LDAFB, IPIV, EQUED, R, C, B, LDB, X, LDX,
     $                   RCOND, FERR, BERR, WORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, TRANS
      INTEGER            INFO, KL, KU, LDAB, LDAFB, LDB, LDX, N, NRHS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
```

```
      DOUBLE PRECISION   BERR( * ), C( * ), FERR( * ), R( * ),
     $                   RWORK( * )
      COMPLEX*16         AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*  Moved setting of INFO = N+1 so INFO does not subsequently get
*  overwritten.  Sven, 17 Mar 05.
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            COLEQU, EQUIL, NOFACT, NOTRAN, ROWEQU
      CHARACTER          NORM
      INTEGER            I, INFEQU, J, J1, J2
      DOUBLE PRECISION   AMAX, ANORM, BIGNUM, COLCND, RCMAX, RCMIN,
     $                   ROWCND, RPVGRW, SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, ZLANGB, ZLANTB
      EXTERNAL           LSAME, DLAMCH, ZLANGB, ZLANTB
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZCOPY, ZGBCON, ZGBEQU, ZGBRFS, ZGBTRF,
     $                   ZGBTRS, ZLACPY, ZLAQGB
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      NOTRAN = LSAME( TRANS, 'N' )
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         ROWEQU = .FALSE.
         COLEQU = .FALSE.
      ELSE
         ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
         COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         SMLNUM = DLAMCH( 'Safe minimum' )
         BIGNUM = ONE / SMLNUM
      END IF
```

```
*
*     Test the input parameters.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.LSAME( FACT, 'F' ) )
     $     THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $         LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( KL.LT.0 ) THEN
         INFO = -4
      ELSE IF( KU.LT.0 ) THEN
         INFO = -5
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -6
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -8
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
         INFO = -10
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $         ( ROWEQU .OR. COLEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -12
      ELSE
         IF( ROWEQU ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 10 J = 1, N
               RCMIN = MIN( RCMIN, R( J ) )
               RCMAX = MAX( RCMAX, R( J ) )
   10       CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -13
            ELSE IF( N.GT.0 ) THEN
               ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               ROWCND = ONE
            END IF
         END IF
         IF( COLEQU .AND. INFO.EQ.0 ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 20 J = 1, N
               RCMIN = MIN( RCMIN, C( J ) )
               RCMAX = MAX( RCMAX, C( J ) )
   20       CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -14
            ELSE IF( N.GT.0 ) THEN
```

```
            COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
         ELSE
            COLCND = ONE
         END IF
      END IF
      IF( INFO.EQ.0 ) THEN
         IF( LDB.LT.MAX( 1, N ) ) THEN
            INFO = -16
         ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
            INFO = -18
         END IF
      END IF
   END IF
*
   IF( INFO.NE.0 ) THEN
      CALL XERBLA( 'ZGBSVX', -INFO )
      RETURN
   END IF
*
   IF( EQUIL ) THEN
*
*     Compute row and column scalings to equilibrate the matrix A.
*
      CALL ZGBEQU( N, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $             AMAX, INFEQU )
      IF( INFEQU.EQ.0 ) THEN
*
*        Equilibrate the matrix.
*
         CALL ZLAQGB( N, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $                AMAX, EQUED )
         ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
         COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
      END IF
   END IF
*
*  Scale the right hand side.
*
   IF( NOTRAN ) THEN
      IF( ROWEQU ) THEN
         DO 40 J = 1, NRHS
            DO 30 I = 1, N
               B( I, J ) = R( I )*B( I, J )
30          CONTINUE
40       CONTINUE
      END IF
   ELSE IF( COLEQU ) THEN
      DO 60 J = 1, NRHS
         DO 50 I = 1, N
            B( I, J ) = C( I )*B( I, J )
```

```
   50       CONTINUE
   60    CONTINUE
      END IF
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the LU factorization of the band matrix A.
*
         DO 70 J = 1, N
            J1 = MAX( J-KU, 1 )
            J2 = MIN( J+KL, N )
            CALL ZCOPY( J2-J1+1, AB( KU+1-J+J1, J ), 1,
     $                  AFB( KL+KU+1-J+J1, J ), 1 )
   70    CONTINUE
*
         CALL ZGBTRF( N, N, KL, KU, AFB, LDAFB, IPIV, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 ) THEN
*
*           Compute the reciprocal pivot growth factor of the
*           leading rank-deficient INFO columns of A.
*
            ANORM = ZERO
            DO 90 J = 1, INFO
               DO 80 I = MAX( KU+2-J, 1 ), MIN( N+KU+1-J, KL+KU+1 )
                  ANORM = MAX( ANORM, ABS( AB( I, J ) ) )
   80          CONTINUE
   90       CONTINUE
            RPVGRW = ZLANTB( 'M', 'U', 'N', INFO, MIN( INFO-1, KL+KU ),
     $                       AFB( MAX( 1, KL+KU+2-INFO ), 1 ), LDAFB,
     $                       RWORK )
            IF( RPVGRW.EQ.ZERO ) THEN
               RPVGRW = ONE
            ELSE
               RPVGRW = ANORM / RPVGRW
            END IF
            RWORK( 1 ) = RPVGRW
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A and the
*     reciprocal pivot growth factor RPVGRW.
*
      IF( NOTRAN ) THEN
         NORM = '1'
      ELSE
```

```
         NORM = 'I'
      END IF
      ANORM = ZLANGB( NORM, N, KL, KU, AB, LDAB, RWORK )
      RPVGRW = ZLANTB( 'M', 'U', 'N', N, KL+KU, AFB, LDAFB, RWORK )
      IF( RPVGRW.EQ.ZERO ) THEN
         RPVGRW = ONE
      ELSE
         RPVGRW = ZLANGB( 'M', N, KL, KU, AB, LDAB, RWORK ) / RPVGRW
      END IF
*
*     Compute the reciprocal of the condition number of A.
*
      CALL ZGBCON( NORM, N, KL, KU, AFB, LDAFB, IPIV, ANORM, RCOND,
     $             WORK, RWORK, INFO )
*
*     Compute the solution matrix X.
*
      CALL ZLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL ZGBTRS( TRANS, N, KL, KU, NRHS, AFB, LDAFB, IPIV, X, LDX,
     $             INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL ZGBRFS( TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB, IPIV,
     $             B, LDB, X, LDX, FERR, BERR, WORK, RWORK, INFO )
*
*     Transform the solution matrix X to a solution of the original
*     system.
*
      IF( NOTRAN ) THEN
         IF( COLEQU ) THEN
            DO 110 J = 1, NRHS
               DO 100 I = 1, N
                  X( I, J ) = C( I )*X( I, J )
  100          CONTINUE
  110       CONTINUE
            DO 120 J = 1, NRHS
               FERR( J ) = FERR( J ) / COLCND
  120       CONTINUE
         END IF
      ELSE IF( ROWEQU ) THEN
         DO 140 J = 1, NRHS
            DO 130 I = 1, N
               X( I, J ) = R( I )*X( I, J )
  130       CONTINUE
  140    CONTINUE
         DO 150 J = 1, NRHS
            FERR( J ) = FERR( J ) / ROWCND
  150    CONTINUE
```

```
      END IF
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.DLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      RWORK( 1 ) = RPVGRW
      RETURN
*
*     End of ZGBSVX
*
      END
```

———-

43.2.4 lisp code

— **zgbsvx.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun zgbsvx
         (fact trans n kl ku nrhs ab ldab afb ldafb ipiv equed r c b ldb$ x ldx
          rcond ferr berr work rwork info)
    (declare (type (double-float) rcond)
             (type (array double-float (*)) rwork berr ferr c r)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) work x b afb ab)
             (type (f2cl-lib:integer4) info ldx ldb$ ldafb ldab nrhs ku kl n)
             (type (string *) equed trans fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (trans character trans-%data% trans-%offset%)
         (equed character equed-%data% equed-%offset%)
         (ab f2cl-lib:complex16 ab-%data% ab-%offset%)
         (afb f2cl-lib:complex16 afb-%data% afb-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (x f2cl-lib:complex16 x-%data% x-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (r double-float r-%data% r-%offset%)
         (c double-float c-%data% c-%offset%)
         (ferr double-float ferr-%data% ferr-%offset%)
         (berr double-float berr-%data% berr-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
```

```
(prog ((amax 0.0d0) (anorm 0.0d0) (bignum 0.0d0) (colcnd 0.0d0)
       (rcmax 0.0d0) (rcmin 0.0d0) (rowcnd 0.0d0) (rpvgrw 0.0d0)
       (smlnum 0.0d0) (i 0) (infequ 0) (j 0) (j1 0) (j2 0)
       (norm
        (make-array '(1) :element-type 'character :initial-element #\ ))
       (colequ nil) (equil nil) (nofact nil) (notran nil) (rowequ nil))
  (declare
   (type (double-float) amax anorm bignum colcnd rcmax rcmin rowcnd
    rpvgrw smlnum)
   (type (f2cl-lib:integer4) i infequ j j1 j2)
   (type (string 1) norm)
   (type f2cl-lib:logical colequ equil nofact notran rowequ))
  (setf info 0)
  (setf nofact
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame fact "N")
            (declare (ignore var-1))
            (when var-0 (setf fact var-0))
            ret-val))
  (setf equil
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame fact "E")
            (declare (ignore var-1))
            (when var-0 (setf fact var-0))
            ret-val))
  (setf notran
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame trans "N")
            (declare (ignore var-1))
            (when var-0 (setf trans var-0))
            ret-val))
  (cond
   ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
    (setf rowequ f2cl-lib:%false%) (setf colequ f2cl-lib:%false%))
   (t
    (setf rowequ
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "R")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame equed "B")
               (declare (ignore var-1))
               (when var-0 (setf equed var-0))
               ret-val)))
    (setf colequ
            (or
             (multiple-value-bind (ret-val var-0 var-1)
```

```
                (lsame equed "C")
              (declare (ignore var-1))
              (when var-0 (setf equed var-0))
              ret-val)
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame equed "B")
              (declare (ignore var-1))
              (when var-0 (setf equed var-0))
              ret-val)))
  (setf smlnum (dlamch "Safe minimum")) (setf bignum (/ one smlnum))))
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< kl 0) (setf info -4))
 ((< ku 0) (setf info -5)) ((< nrhs 0) (setf info -6))
 ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -8))
 ((< ldafb (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
  (setf info -10))
 ((and
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame fact "F")
     (declare (ignore var-1))
     (when var-0 (setf fact var-0))
     ret-val)
   (not
    (or rowequ colequ
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame equed "N")
          (declare (ignore var-1))
          (when var-0 (setf equed var-0))
```

```
            ret-val))))
    (setf info -12))
   (t
    (cond
     (rowequ (setf rcmin bignum) (setf rcmax zero)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf rcmin
                  (min rcmin
                       (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
          (setf rcmax
                  (max rcmax
                       (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
         label10))
      (cond ((<= rcmin zero) (setf info -13))
            ((> n 0)
             (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum))))
            (t (setf rowcnd one)))))
    (cond
     ((and colequ (= info 0)) (setf rcmin bignum) (setf rcmax zero)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf rcmin
                  (min rcmin
                       (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
          (setf rcmax
                  (max rcmax
                       (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
         label20))
      (cond ((<= rcmin zero) (setf info -14))
            ((> n 0)
             (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum))))
            (t (setf colcnd one)))))
    (cond
     ((= info 0)
      (cond
       ((< ldb$
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -16))
       ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -18)))))))
  (cond
   ((/= info 0) (xerbla "ZGBSVX" (f2cl-lib:int-sub info))
    (go end_label)))
  (cond
   (equil
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
          var-10 var-11)
         (zgbequ n n kl ku ab ldab r c rowcnd colcnd amax infequ)
       (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7))
       (setf rowcnd var-8)
       (setf colcnd var-9)
       (setf amax var-10)
       (setf infequ var-11))
     (cond
      ((= infequ 0)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10 var-11)
           (zlaqgb n n kl ku ab ldab r c rowcnd colcnd amax equed)
         (declare (ignore var-4 var-6 var-7))
         (when var-0 (setf n var-0))
         (when var-1 (setf n var-1))
         (when var-2 (setf kl var-2))
         (when var-3 (setf ku var-3))
         (when var-5 (setf ldab var-5))
         (when var-8 (setf rowcnd var-8))
         (when var-9 (setf colcnd var-9))
         (when var-10 (setf amax var-10))
         (when var-11 (setf equed var-11)))
       (setf rowequ
               (or
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame equed "R")
                  (declare (ignore var-1))
                  (when var-0 (setf equed var-0))
                  ret-val)
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame equed "B")
                  (declare (ignore var-1))
                  (when var-0 (setf equed var-0))
                  ret-val)))
       (setf colequ
               (or
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame equed "C")
                  (declare (ignore var-1))
                  (when var-0 (setf equed var-0))
                  ret-val)
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame equed "B")
                  (declare (ignore var-1))
                  (when var-0 (setf equed var-0))
                  ret-val)))))))
  (cond
   (notran
    (cond
```

```
    (rowequ
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j nrhs) nil)
       (tagbody
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)
                     (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                        (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)))
           label30))
       label40)))))
  (colequ
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j nrhs) nil)
     (tagbody
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                   (* (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)
                      (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                     b-%offset%)))
         label50))
     label60))))
 (cond
  ((or nofact equil)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf j1
               (max (the f2cl-lib:integer4 (f2cl-lib:int-sub j ku))
                    (the f2cl-lib:integer4 1)))
       (setf j2
               (min (the f2cl-lib:integer4 (f2cl-lib:int-add j kl))
                    (the f2cl-lib:integer4 n)))
       (zcopy (f2cl-lib:int-add (f2cl-lib:int-sub j2 j1) 1)
        (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                              ((+ ku 1 (f2cl-lib:int-sub j) j1) j)
                              ((1 ldab) (1 *)) ab-%offset%)
        1
        (f2cl-lib:array-slice afb-%data% f2cl-lib:complex16
                              ((+ kl ku 1 (f2cl-lib:int-sub j) j1) j)
                              ((1 ldafb) (1 *)) afb-%offset%)
        1)
     label70))
   (multiple-value-bind
```

```
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgbtrf n n kl ku afb ldafb ipiv info)
    (declare (ignore var-4 var-6))
    (when var-0 (setf n var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf kl var-2))
    (when var-3 (setf ku var-3))
    (when var-5 (setf ldafb var-5))
    (when var-7 (setf info var-7)))
  (cond
   ((> info 0) (setf anorm zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j info) nil)
      (tagbody
        (f2cl-lib:fdo (i
                       (max
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add ku 2
                                               (f2cl-lib:int-sub j)))
                        (the f2cl-lib:integer4 1))
                       (f2cl-lib:int-add i 1))
                      ((> i
                          (min
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add n ku 1
                                                  (f2cl-lib:int-sub
                                                   j)))
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add kl ku 1))))
                       nil)
          (tagbody
            (setf anorm
                    (max anorm
                         (abs
                          (f2cl-lib:fref ab-%data% (i j)
                                         ((1 ldab) (1 *))
                                         ab-%offset%))))
           label80))
       label90))
    (setf rpvgrw
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6
                 var-7)
                (zlantb "M" "U" "N" info
                 (min (the f2cl-lib:integer4 (f2cl-lib:int-sub info 1))
                      (the f2cl-lib:integer4 (f2cl-lib:int-add kl ku)))
                 (f2cl-lib:array-slice afb-%data% f2cl-lib:complex16
                                       ((max 1
                                             (f2cl-lib:int-add kl ku 2
                                                               (f2cl-lib:int-sub
```

```
                                                                    info)))
                                          1)
                                         ((1 ldafb) (1 *)) afb-%offset%)
                  ldafb rwork)
                (declare (ignore var-0 var-1 var-2 var-4 var-5 var-7))
                (when var-3 (setf info var-3))
                (when var-6 (setf ldafb var-6))
                ret-val))
    (cond ((= rpvgrw zero) (setf rpvgrw one))
          (t (setf rpvgrw (/ anorm rpvgrw))))
    (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
            rpvgrw)
    (setf rcond zero) (go end_label)))))
(cond (notran (f2cl-lib:f2cl-set-string norm "1" (string 1)))
      (t (f2cl-lib:f2cl-set-string norm "I" (string 1))))
(setf anorm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zlangb norm n kl ku ab ldab rwork)
          (declare (ignore var-4 var-6))
          (when var-0 (setf norm var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf kl var-2))
          (when var-3 (setf ku var-3))
          (when var-5 (setf ldab var-5))
          ret-val))
(setf rpvgrw
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (zlantb "M" "U" "N" n (f2cl-lib:int-add kl ku) afb ldafb
             rwork)
          (declare (ignore var-0 var-1 var-2 var-4 var-5 var-7))
          (when var-3 (setf n var-3))
          (when var-6 (setf ldafb var-6))
          ret-val))
(cond ((= rpvgrw zero) (setf rpvgrw one))
      (t
       (setf rpvgrw
               (/
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (zlangb "M" n kl ku ab ldab rwork)
                  (declare (ignore var-0 var-4 var-6))
                  (when var-1 (setf n var-1))
                  (when var-2 (setf kl var-2))
                  (when var-3 (setf ku var-3))
                  (when var-5 (setf ldab var-5))
                  ret-val)
                rpvgrw))))
(multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11)
    (zgbcon norm n kl ku afb ldafb ipiv anorm rcond work rwork info)
  (declare (ignore var-2 var-3 var-4 var-6 var-7 var-9 var-10))
  (setf norm var-0)
  (setf n var-1)
  (setf ldafb var-5)
  (setf rcond var-8)
  (setf info var-11))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "Full" n nrhs b ldb$ x ldx)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf ldx var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgbtrs trans n kl ku nrhs afb ldafb ipiv x ldx info)
  (declare (ignore var-5 var-7 var-8))
  (when var-0 (setf trans var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf kl var-2))
  (when var-3 (setf ku var-3))
  (when var-4 (setf nrhs var-4))
  (when var-6 (setf ldafb var-6))
  (when var-9 (setf ldx var-9))
  (when var-10 (setf info var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18)
    (zgbrfs trans n kl ku nrhs ab ldab afb ldafb ipiv b ldb$ x ldx ferr
     berr work rwork info)
  (declare
   (ignore var-4 var-5 var-7 var-9 var-10 var-11 var-12 var-13 var-14
    var-15 var-16 var-17))
  (setf trans var-0)
  (setf n var-1)
  (setf kl var-2)
  (setf ku var-3)
  (setf ldab var-6)
  (setf ldafb var-8)
  (setf info var-18))
(cond
 (notran
  (cond
   (colequ
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
```

```
              (tagbody
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                         x-%offset%)
                          (* (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)
                             (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                            x-%offset%)))
                  label100))
               label110))
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j nrhs) nil)
              (tagbody
                (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                      (/
                       (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                       colcnd))
               label120)))))
          (rowequ
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j nrhs) nil)
             (tagbody
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i n) nil)
                 (tagbody
                   (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                        x-%offset%)
                         (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                            (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                           x-%offset%)))
                 label130))
              label140))
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j nrhs) nil)
             (tagbody
               (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                     (/ (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                        rowcnd))
              label150))))
         (if (< rcond (dlamch "Epsilon"))
             (setf info (f2cl-lib:int-add n 1)))
         (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%) rpvgrw)
         (go end_label)
        end_label
         (return
          (values fact trans n kl ku nrhs nil ldab nil ldafb nil equed nil nil
                  nil ldb$ nil ldx rcond nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
```

```
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgbsvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (string) (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::fact
                                              fortran-to-lisp::trans
                                              fortran-to-lisp::n
                                              fortran-to-lisp::kl
                                              fortran-to-lisp::ku
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::ldab nil
                                              fortran-to-lisp::ldafb nil
                                              fortran-to-lisp::equed nil nil
                                              nil fortran-to-lisp::ldb$ nil
                                              fortran-to-lisp::ldx
                                              fortran-to-lisp::rcond nil nil
                                              nil nil fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::zgbrfs
                                              fortran-to-lisp::zgbcon
```

```
                                        fortran-to-lisp::zgbequ
                                        fortran-to-lisp::xerbla))))
```

———-

43.3 zgbsvxx

43.3.1 Axiom unit tests

— **zgbsvxx.input** —

```
)set break resume
)sys rm -f zgbsvxx.output
)spool zgbsvxx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

43.3.2 Axiom help page [Demm05]

— **zgbsvxx.help** —

```
======================================================================
zgbsvxx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgbsvxx (FACT, TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB,
           IPIV, EQUED, R, C, B, LDB, X, LDX, RCOND, RPVGRW, BERR, N_ERR_BNDS,
           ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, RWORK, INFO)
            ZGBSVXX computes the solution to system of linear equations A * X
           = B for GB matrices
```

```
Function/Subroutine Documentation
   subroutine zgbsvxx (character FACT, character TRANS, integer N, integer KL,
       integer KU, integer NRHS, complex*16, dimension( ldab, * ) AB, integer
       LDAB, complex*16, dimension( ldafb, * ) AFB, integer LDAFB, integer,
       dimension( * ) IPIV, character EQUED, double precision, dimension( * )
       R, double precision, dimension( * ) C, complex*16, dimension( ldb, * )
       B, integer LDB, complex*16, dimension( ldx , * ) X, integer LDX, double
       precision RCOND, double precision RPVGRW, double precision, dimension(
       * ) BERR, integer N_ERR_BNDS, double precision, dimension( nrhs, * )
       ERR_BNDS_NORM, double precision, dimension( nrhs, * ) ERR_BNDS_COMP,
       integer NPARAMS, double precision, dimension( * ) PARAMS, complex*16,
       dimension( * ) WORK, double precision, dimension( * ) RWORK, integer
       INFO)
        ZGBSVXX computes the solution to system of linear equations A * X = B
       for GB matrices

Purpose:
   ZGBSVXX uses the LU factorization to compute the solution to a
   complex*16 system of linear equations  A * X = B,  where A is an
   N-by-N matrix and X and B are N-by-NRHS matrices.

   If requested, both normwise and maximum componentwise error bounds
   are returned. ZGBSVXX will return a solution with a tiny
   guaranteed error (O(eps) where eps is the working machine
   precision) unless the matrix is very ill-conditioned, in which
   case a warning is returned. Relevant condition numbers also are
   calculated and returned.

   ZGBSVXX accepts user-provided factorizations and equilibration
   factors; see the definitions of the FACT and EQUED options.
   Solving with refinement and using a factorization from a previous
   ZGBSVXX call will also produce a solution with either O(eps)
   errors or warnings, but we cannot make that claim for general
   user-provided factorizations and equilibration factors if they
   differ from what ZGBSVXX would itself produce.

Description:

   The following steps are performed:

   1. If FACT = 'E', double precision scaling factors are computed to
      equilibrate the system:

      TRANS = 'N':  diag(R)*A*diag(C)     *inv(diag(C))*X = diag(R)*B
      TRANS = 'T': (diag(R)*A*diag(C))**T *inv(diag(R))*X = diag(C)*B
      TRANS = 'C': (diag(R)*A*diag(C))**H *inv(diag(R))*X = diag(C)*B

      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
```

```
      overwritten by diag(R)*A*diag(C) and B by diag(R)*B (if TRANS='N')
      or diag(C)*B (if TRANS = 'T' or 'C').

   2. If FACT = 'N' or 'E', the LU decomposition is used to factor
      the matrix A (after equilibration if FACT = 'E') as

      A = P * L * U,

      where P is a permutation matrix, L is a unit lower triangular
      matrix, and U is upper triangular.

   3. If some U(i,i)=0, so that U is exactly singular, then the
      routine returns with INFO = i. Otherwise, the factored form of A
      is used to estimate the condition number of the matrix A (see
      argument RCOND). If the reciprocal of the condition number is less
      than machine precision, the routine still goes on to solve for X
      and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. By default (unless PARAMS(LA_LINRX_ITREF_I) is set to zero),
      the routine will use iterative refinement to try to get a small
      error and error bounds.  Refinement calculates the residual to at
      least twice the working precision.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(C) (if TRANS = 'N') or diag(R) (if TRANS = 'T' or 'C') so
      that it solves the original system before equilibration.

      Some optional parameters are bundled in the PARAMS array.  These
      settings determine how refinement is performed, but often the
      defaults are acceptable.  If the defaults are acceptable, users
      can pass NPARAMS = 0 which prevents the source code from accessing
      the PARAMS argument.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AF and IPIV contain the factored form of A.
      If EQUED is not 'N', the matrix A has been
      equilibrated with scaling factors given by R and C.
      A, AF, and IPIV are not modified.
   = 'N':  The matrix A will be copied to AF and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
      copied to AF and factored.
```

```
TRANS
  TRANS is CHARACTER*1
  Specifies the form of the system of equations:
  = 'N':  A * X = B     (No transpose)
  = 'T':  A**T * X = B  (Transpose)
  = 'C':  A**H * X = B  (Conjugate Transpose = Transpose)

N
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

KL
  KL is INTEGER
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

AB
  AB is COMPLEX*16 array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

  If FACT = 'F' and EQUED is not 'N', then AB must have been
  equilibrated by the scaling factors in R and/or C.  AB is not
  modified if FACT = 'F' or 'N', or if FACT = 'E' and
  EQUED = 'N' on exit.

  On exit, if EQUED .ne. 'N', A is scaled as follows:
  EQUED = 'R':  A := diag(R) * A
  EQUED = 'C':  A := A * diag(C)
  EQUED = 'B':  A := diag(R) * A * diag(C).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is COMPLEX*16 array, dimension (LDAFB,N)
  If FACT = 'F', then AFB is an input argument and on entry
  contains details of the LU factorization of the band matrix
  A, as computed by ZGBTRF.  U is stored as an upper triangular
```

```
  band matrix with KL+KU superdiagonals in rows 1 to KL+KU+1,
  and the multipliers used during the factorization are stored
  in rows KL+KU+2 to 2*KL+KU+1.  If EQUED .ne. 'N', then AFB is
  the factored form of the equilibrated matrix A.

  If FACT = 'N', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then AF is an output argument and on exit
  returns the factors L and U from the factorization A = P*L*U
  of the equilibrated matrix A (see the description of A for
  the form of the equilibrated matrix).

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains the pivot indices from the factorization A = P*L*U
  as computed by DGETRF; row i of the matrix was interchanged
  with row IPIV(i).

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the original matrix A.

  If FACT = 'E', then IPIV is an output argument and on exit
  contains the pivot indices from the factorization A = P*L*U
  of the equilibrated matrix A.

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
  = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
  = 'B':  Both row and column equilibration, i.e., A has been
     replaced by diag(R) * A * diag(C).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

R
  R is DOUBLE PRECISION array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
```

```
  EQUED = 'R' or 'B', each element of R must be positive.
  If R is output, each element of R is a power of the radix.
  If R is input, each element of R should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

C
  C is DOUBLE PRECISION array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.
  If C is output, each element of C is a power of the radix.
  If C is input, each element of C should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if TRANS = 'N' and EQUED = 'R' or 'B', B is overwritten by diag(R)*B;
  if TRANS = 'T' or 'C' and EQUED = 'C' or 'B', B is
     overwritten by diag(C)*B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  If INFO = 0, the N-by-NRHS solution matrix X to the original
  system of equations.  Note that A and B are modified on exit
  if EQUED .ne. 'N', and the solution to the equilibrated system is
  inv(diag(C))*X if TRANS = 'N' and EQUED = 'C' or 'B', or
  inv(diag(R))*X if TRANS = 'T' or 'C' and EQUED = 'R' or 'B'.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
```

```
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

RPVGRW
  RPVGRW is DOUBLE PRECISION
  Reciprocal pivot growth.  On exit, this contains the reciprocal
  pivot growth factor norm(A)/norm(U). The "max absolute element"
  norm is used.  If this is much less than 1, then the stability of
  the LU factorization of the (equilibrated) matrix A could be poor.
  This also means that the solution X, estimated condition numbers,
  and error bounds could be unreliable. If factorization fails with
  0<INFO<=N, then this contains the reciprocal pivot growth factor
  for the leading INFO columns of A.  In DGESVX, this quantity is
  returned in WORK(1).

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
            max_j (abs(XTRUE(j,i) - X(j,i)))
             ------------------------------
                      max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.
```

```
  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * dlamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * dlamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.

ERR_BNDS_COMP
  ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                  abs(XTRUE(j,i) - X(j,i))
            max_j ----------------------
                        abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * dlamch('Epsilon').
```

```
  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * dlamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * dlamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is DOUBLE PRECISION array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0D+0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the extra-precise refinement algorithm.
  (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)
```

```
 WORK
   WORK is COMPLEX*16 array, dimension (2*N)

 RWORK
   RWORK is DOUBLE PRECISION array, dimension (2*N)

 INFO
   INFO is INTEGER
   = 0:  Successful exit. The solution to every right-hand side is guaranteed.
   < 0:  If INFO = -i, the i-th argument had an illegal value
   > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
      has been completed, but the factor U is exactly singular, so
      the solution and error bounds could not be computed. RCOND = 0
      is returned.
   = N+J: The solution corresponding to the Jth right-hand side is
      not guaranteed. The solutions corresponding to other right-
      hand sides K with K > J may not be guaranteed as well, but
      only the first such right-hand side is reported. If a small
      componentwise error is not requested (PARAMS(3) = 0.0) then
      the Jth right-hand side is the first with a normwise error
      bound that is not guaranteed (the smallest J such
      that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
      the Jth right-hand side is the first with either a normwise or
      componentwise error bound that is not guaranteed (the smallest
      J such that either ERR_BNDS_NORM(J,1) = 0.0 or
      ERR_BNDS_COMP(J,1) = 0.0). See the definition of
      ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
      about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

———-

43.3.3 fortran code

— **zgbsvxx.f** —

```
*  ==================================================================
      SUBROUTINE ZGBSVXX( FACT, TRANS, N, KL, KU, NRHS, AB, LDAB, AFB,
     $                    LDAFB, IPIV, EQUED, R, C, B, LDB, X, LDX,
     $                    RCOND, RPVGRW, BERR, N_ERR_BNDS,
     $                    ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $                    WORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
```

```
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, TRANS
      INTEGER            INFO, LDAB, LDAFB, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      DOUBLE PRECISION   RCOND, RPVGRW
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   X( LDX , * ),WORK( * )
      DOUBLE PRECISION   R( * ), C( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * ), RWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
*     ..
*     .. Local Scalars ..
      LOGICAL            COLEQU, EQUIL, NOFACT, NOTRAN, ROWEQU
      INTEGER            INFEQU, I, J, KL, KU
      DOUBLE PRECISION   AMAX, BIGNUM, COLCND, RCMAX, RCMIN,
     $                   ROWCND, SMLNUM
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, DLAMCH, ZLA_GBRPVGRW
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, ZLA_GBRPVGRW
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZGBEQUB, ZGBTRF, ZGBTRS, ZLACPY, ZLAQGB,
     $                   XERBLA, ZLASCL2, ZGBRFSX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
```

```
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      NOTRAN = LSAME( TRANS, 'N' )
      SMLNUM = DLAMCH( 'Safe minimum' )
      BIGNUM = ONE / SMLNUM
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         ROWEQU = .FALSE.
         COLEQU = .FALSE.
      ELSE
         ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
         COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
      END IF
*
*     Default is failure.  If an input parameter is wrong or
*     factorization fails, make everything look horrible.  Only the
*     pivot growth is set here, the rest is initialized in ZGBRFSX.
*
      RPVGRW = ZERO
*
*     Test the input parameters.  PARAMS is not tested until DGERFSX.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.
     $     LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $        LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( KL.LT.0 ) THEN
         INFO = -4
      ELSE IF( KU.LT.0 ) THEN
         INFO = -5
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -6
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -8
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
         INFO = -10
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $        ( ROWEQU .OR. COLEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -12
      ELSE
         IF( ROWEQU ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 10 J = 1, N
               RCMIN = MIN( RCMIN, R( J ) )
```

```
            RCMAX = MAX( RCMAX, R( J ) )
 10         CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -13
            ELSE IF( N.GT.0 ) THEN
               ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               ROWCND = ONE
            END IF
         END IF
         IF( COLEQU .AND. INFO.EQ.0 ) THEN
            RCMIN = BIGNUM
            RCMAX = ZERO
            DO 20 J = 1, N
               RCMIN = MIN( RCMIN, C( J ) )
               RCMAX = MAX( RCMAX, C( J ) )
 20         CONTINUE
            IF( RCMIN.LE.ZERO ) THEN
               INFO = -14
            ELSE IF( N.GT.0 ) THEN
               COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
            ELSE
               COLCND = ONE
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -15
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -16
            END IF
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGBSVXX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*     Compute row and column scalings to equilibrate the matrix A.
*
         CALL ZGBEQUB( N, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $        AMAX, INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*     Equilibrate the matrix.
*
            CALL ZLAQGB( N, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
```

```
     $           AMAX, EQUED )
            ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
            COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
         END IF
*
*     If the scaling factors are not applied, set them to 1.0.
*
         IF ( .NOT.ROWEQU ) THEN
            DO J = 1, N
               R( J ) = 1.0D+0
            END DO
         END IF
         IF ( .NOT.COLEQU ) THEN
            DO J = 1, N
               C( J ) = 1.0D+0
            END DO
         END IF
      END IF
*
*     Scale the right-hand side.
*
      IF( NOTRAN ) THEN
         IF( ROWEQU ) CALL ZLASCL2( N, NRHS, R, B, LDB )
      ELSE
         IF( COLEQU ) CALL ZLASCL2( N, NRHS, C, B, LDB )
      END IF
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the LU factorization of A.
*
         DO 40, J = 1, N
            DO 30, I = KL+1, 2*KL+KU+1
               AFB( I, J ) = AB( I-KL, J )
 30         CONTINUE
 40      CONTINUE
         CALL ZGBTRF( N, N, KL, KU, AFB, LDAFB, IPIV, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 ) THEN
*
*           Pivot in column INFO is exactly 0
*           Compute the reciprocal pivot growth factor of the
*           leading rank-deficient INFO columns of A.
*
            RPVGRW = ZLA_GBRPVGRW( N, KL, KU, INFO, AB, LDAB, AFB,
     $           LDAFB )
            RETURN
         END IF
```

```
      END IF
*
*     Compute the reciprocal pivot growth factor RPVGRW.
*
      RPVGRW = ZLA_GBRPVGRW( N, KL, KU, N, AB, LDAB, AFB, LDAFB )
*
*     Compute the solution matrix X.
*
      CALL ZLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL ZGBTRS( TRANS, N, KL, KU, NRHS, AFB, LDAFB, IPIV, X, LDX,
     $     INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL ZGBRFSX( TRANS, EQUED, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB,
     $     IPIV, R, C, B, LDB, X, LDX, RCOND, BERR,
     $     N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $     WORK, RWORK, INFO )

*
*     Scale solutions.
*
      IF ( COLEQU .AND. NOTRAN ) THEN
         CALL ZLASCL2( N, NRHS, C, X, LDX )
      ELSE IF ( ROWEQU .AND. .NOT.NOTRAN ) THEN
         CALL ZLASCL2( N, NRHS, R, X, LDX )
      END IF
*
      RETURN
*
*     End of ZGBSVXX
*
      END
```

———-

43.3.4 lisp code

— zgbsvxx.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun zgbsvxx
         (fact trans n kl ku nrhs ab ldab afb ldafb ipiv equed r c b ldb$ x ldx
          rcond rpvgrw berr n_err_bnds err_bnds_norm err_bnds_comp nparams
```

```
      params work rwork info)
(declare (type (double-float) rpvgrw rcond)
         (type (array double-float (*)) rwork params err_bnds_comp
          err_bnds_norm berr c r)
         (type (array f2cl-lib:integer4 (*)) ipiv)
         (type (array f2cl-lib:complex16 (*)) work x b afb ab)
         (type (f2cl-lib:integer4) info nparams n_err_bnds ldx ldb$ ldafb
          ldab nrhs ku kl n)
         (type (string *) equed trans fact))
(f2cl-lib:with-multi-array-data
    ((fact character fact-%data% fact-%offset%)
     (trans character trans-%data% trans-%offset%)
     (equed character equed-%data% equed-%offset%)
     (ab f2cl-lib:complex16 ab-%data% ab-%offset%)
     (afb f2cl-lib:complex16 afb-%data% afb-%offset%)
     (b f2cl-lib:complex16 b-%data% b-%offset%)
     (x f2cl-lib:complex16 x-%data% x-%offset%)
     (work f2cl-lib:complex16 work-%data% work-%offset%)
     (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
     (r double-float r-%data% r-%offset%)
     (c double-float c-%data% c-%offset%)
     (berr double-float berr-%data% berr-%offset%)
     (err_bnds_norm double-float err_bnds_norm-%data%
      err_bnds_norm-%offset%)
     (err_bnds_comp double-float err_bnds_comp-%data%
      err_bnds_comp-%offset%)
     (params double-float params-%data% params-%offset%)
     (rwork double-float rwork-%data% rwork-%offset%))
  (prog ((amax 0.0d0) (bignum 0.0d0) (colcnd 0.0d0) (rcmax 0.0d0)
         (rcmin 0.0d0) (rowcnd 0.0d0) (smlnum 0.0d0) (infequ 0) (i 0) (j 0)
         (colequ nil) (equil nil) (nofact nil) (notran nil) (rowequ nil))
    (declare
     (type (double-float) amax bignum colcnd rcmax rcmin rowcnd smlnum)
     (type (f2cl-lib:integer4) infequ i j)
     (type f2cl-lib:logical colequ equil nofact notran rowequ))
    (setf info 0)
    (setf nofact
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "N")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val))
    (setf equil
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "E")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val))
    (setf notran
            (multiple-value-bind (ret-val var-0 var-1)
```

```
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(setf smlnum (dlamch "Safe minimum"))
(setf bignum (/ one smlnum))
(cond
 ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
  (setf rowequ f2cl-lib:%false%) (setf colequ f2cl-lib:%false%))
 (t
  (setf rowequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "R")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)))
  (setf colequ
          (or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "C")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame equed "B")
             (declare (ignore var-1))
             (when var-0 (setf equed var-0))
             ret-val)))))
(setf rpvgrw zero)
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
```

```
          ret-val))
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame trans "C")
           (declare (ignore var-1))
           (when var-0 (setf trans var-0))
           ret-val)))
 (setf info -2))
((< n 0) (setf info -3)) ((< kl 0) (setf info -4))
((< ku 0) (setf info -5)) ((< nrhs 0) (setf info -6))
((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -8))
((< ldafb (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
 (setf info -10))
((and
  (multiple-value-bind (ret-val var-0 var-1)
      (lsame fact "F")
    (declare (ignore var-1))
    (when var-0 (setf fact var-0))
    ret-val)
  (not
   (or rowequ colequ
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame equed "N")
         (declare (ignore var-1))
         (when var-0 (setf equed var-0))
         ret-val))))
 (setf info -12))
(t
 (cond
  (rowequ (setf rcmin bignum) (setf rcmax zero)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf rcmin
               (min rcmin
                    (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
       (setf rcmax
               (max rcmax
                    (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%)))
      label10))
   (cond ((<= rcmin zero) (setf info -13))
         ((> n 0)
          (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum))))
         (t (setf rowcnd one)))))
 (cond
  ((and colequ (= info 0)) (setf rcmin bignum) (setf rcmax zero)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf rcmin
```

```
                    (min rcmin
                         (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
            (setf rcmax
                    (max rcmax
                         (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
           label20))
        (cond ((<= rcmin zero) (setf info -14))
              ((> n 0)
               (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum))))
              (t (setf colcnd one)))))
      (cond
       ((= info 0)
        (cond
         ((< ldb$
             (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -15))
         ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -16)))))))
    (cond
     ((/= info 0) (xerbla "ZGBSVXX" (f2cl-lib:int-sub info))
      (go end_label)))
    (cond
     (equil
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11)
          (zgbequb n n kl ku ab ldab r c rowcnd colcnd amax infequ)
        (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7))
        (setf rowcnd var-8)
        (setf colcnd var-9)
        (setf amax var-10)
        (setf infequ var-11))
      (cond
       ((= infequ 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11)
            (zlaqgb n n kl ku ab ldab r c rowcnd colcnd amax equed)
          (declare (ignore var-4 var-6 var-7))
          (when var-0 (setf n var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf kl var-2))
          (when var-3 (setf ku var-3))
          (when var-5 (setf ldab var-5))
          (when var-8 (setf rowcnd var-8))
          (when var-9 (setf colcnd var-9))
          (when var-10 (setf amax var-10))
          (when var-11 (setf equed var-11)))
        (setf rowequ
                (or
```

```
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame equed "R")
                  (declare (ignore var-1))
                  (when var-0 (setf equed var-0))
                  ret-val)
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame equed "B")
                  (declare (ignore var-1))
                  (when var-0 (setf equed var-0))
                  ret-val)))
      (setf colequ
              (or
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame equed "C")
                  (declare (ignore var-1))
                  (when var-0 (setf equed var-0))
                  ret-val)
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame equed "B")
                  (declare (ignore var-1))
                  (when var-0 (setf equed var-0))
                  ret-val)))))
    (cond
     ((not rowequ)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf (f2cl-lib:fref r-%data% (j) ((1 *)) r-%offset%) 1.0d0)
         label100000))))
    (cond
     ((not colequ)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) 1.0d0)
         label100001))))))
  (cond
   (notran
    (if rowequ
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (zlascl2 n nrhs r b ldb$)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-1 (setf nrhs var-1))
          (when var-4 (setf ldb$ var-4)))))
   (t
    (if colequ
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (zlascl2 n nrhs c b ldb$)
          (declare (ignore var-2 var-3))
```

```
        (when var-0 (setf n var-0))
        (when var-1 (setf nrhs var-1))
        (when var-4 (setf ldb$ var-4))))))
(cond
 ((or nofact equil)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i (f2cl-lib:int-add kl 1) (f2cl-lib:int-add i 1))
                    ((> i
                        (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku
                                          1))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref afb-%data% (i j) ((1 ldafb) (1 *))
                               afb-%offset%)
                (f2cl-lib:fref ab-%data% ((f2cl-lib:int-sub i kl) j)
                               ((1 ldab) (1 *)) ab-%offset%))
         label30))
     label40))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgbtrf n n kl ku afb ldafb ipiv info)
    (declare (ignore var-4 var-6))
    (when var-0 (setf n var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf kl var-2))
    (when var-3 (setf ku var-3))
    (when var-5 (setf ldafb var-5))
    (when var-7 (setf info var-7)))
  (cond
   ((> info 0)
    (setf rpvgrw
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6
                 var-7)
                (zla_gbrpvgrw n kl ku info ab ldab afb ldafb)
              (declare (ignore var-4 var-6))
              (when var-0 (setf n var-0))
              (when var-1 (setf kl var-1))
              (when var-2 (setf ku var-2))
              (when var-3 (setf info var-3))
              (when var-5 (setf ldab var-5))
              (when var-7 (setf ldafb var-7))
              ret-val))
    (go end_label)))))
(setf rpvgrw
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (zla_gbrpvgrw n kl ku n ab ldab afb ldafb)
```

```
          (declare (ignore var-4 var-6))
          (when var-0 (setf n var-0))
          (when var-1 (setf kl var-1))
          (when var-2 (setf ku var-2))
          (when var-3 (setf n var-3))
          (when var-5 (setf ldab var-5))
          (when var-7 (setf ldafb var-7))
          ret-val))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "Full" n nrhs b ldb$ x ldx)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf ldx var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgbtrs trans n kl ku nrhs afb ldafb ipiv x ldx info)
  (declare (ignore var-5 var-7 var-8))
  (when var-0 (setf trans var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf kl var-2))
  (when var-3 (setf ku var-3))
  (when var-4 (setf nrhs var-4))
  (when var-6 (setf ldafb var-6))
  (when var-9 (setf ldx var-9))
  (when var-10 (setf info var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19
     var-20 var-21 var-22 var-23 var-24 var-25 var-26)
    (zgbrfsx trans equed n kl ku nrhs ab ldab afb ldafb ipiv r c b ldb$
     x ldx rcond berr n_err_bnds err_bnds_norm err_bnds_comp nparams
     params work rwork info)
  (declare
   (ignore var-6 var-8 var-10 var-11 var-12 var-13 var-15 var-18 var-20
    var-21 var-23 var-24 var-25))
  (when var-0 (setf trans var-0))
  (when var-1 (setf equed var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf kl var-3))
  (when var-4 (setf ku var-4))
  (when var-5 (setf nrhs var-5))
  (when var-7 (setf ldab var-7))
  (when var-9 (setf ldafb var-9))
  (when var-14 (setf ldb$ var-14))
  (when var-16 (setf ldx var-16))
  (when var-17 (setf rcond var-17))
  (when var-19 (setf n_err_bnds var-19))
```

```
          (when var-22 (setf nparams var-22))
          (when var-26 (setf info var-26)))
        (cond
         ((and colequ notran)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (zlascl2 n nrhs c x ldx)
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf nrhs var-1))
            (when var-4 (setf ldx var-4))))
         ((and rowequ (not notran))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (zlascl2 n nrhs r x ldx)
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf nrhs var-1))
            (when var-4 (setf ldx var-4)))))
        (go end_label)
       end_label
        (return
         (values fact trans n kl ku nrhs nil ldab nil ldafb nil equed nil nil
                 nil ldb$ nil ldx rcond rpvgrw nil n_err_bnds nil nil nparams
                 nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgbsvxx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (string) (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
```

```
  (fortran-to-lisp::integer4)
  (double-float) (double-float)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array fortran-to-lisp::complex16
   (*))
  (array double-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::fact
  fortran-to-lisp::trans
  fortran-to-lisp::n
  fortran-to-lisp::kl
  fortran-to-lisp::ku
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::ldab nil
  fortran-to-lisp::ldafb nil
  fortran-to-lisp::equed nil nil
  nil fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond
  fortran-to-lisp::rpvgrw nil
  fortran-to-lisp::n_err_bnds nil
  nil fortran-to-lisp::nparams nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zgbequb
  fortran-to-lisp::xerbla))))
```

———-

Chapter 44

LAPACK - General Band Matrix, Comp. Routines, Double

44.1 dgbbrd

44.1.1 Axiom unit tests

— **dgbbrd.input** —

```
)set break resume
)sys rm -f dgbbrd.output
)spool dgbbrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

44.1.2 Axiom help page

— **dgbbrd.help** —

```
==========================================================================
```

```
dgbbrd examples
===================================================================

===================================================================
Man Page Details
===================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgbbrd (VECT, M, N, NCC, KL, KU, AB, LDAB, D, E, Q, LDQ, PT,
           LDPT, C, LDC, WORK, INFO)
           DGBBRD

Function/Subroutine Documentation
   subroutine dgbbrd (character VECT, integer M, integer N, integer NCC,
       integer KL, integer KU, double precision, dimension( ldab, * ) AB,
       integer LDAB, double precision, dimension( * ) D, double precision,
       dimension( * ) E, double precision, dimension( ldq, * ) Q, integer LDQ,
       double precision, dimension( ldpt, * ) PT, integer LDPT, double
       precision, dimension( ldc, * ) C, integer LDC, double precision,
       dimension( * ) WORK, integer INFO)
       DGBBRD

Purpose:
   DGBBRD reduces a real general m-by-n band matrix A to upper
   bidiagonal form B by an orthogonal transformation: Q**T * A * P = B.

   The routine computes B, and optionally forms Q or P**T, or computes
   Q**T*C for a given matrix C.

Parameters:

 VECT
   VECT is CHARACTER*1
   Specifies whether or not the matrices Q and P**T are to be
   formed.
   = 'N': do not form Q or P**T;
   = 'Q': form Q only;
   = 'P': form P**T only;
   = 'B': form both.

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 NCC
```

```
  NCC is INTEGER
  The number of columns of the matrix C.  NCC >= 0.

KL
  KL is INTEGER
  The number of subdiagonals of the matrix A. KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals of the matrix A. KU >= 0.

AB
  AB is DOUBLE PRECISION array, dimension (LDAB,N)
  On entry, the m-by-n band matrix A, stored in rows 1 to
  KL+KU+1. The j-th column of A is stored in the j-th column of
  the array AB as follows:
  AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl).
  On exit, A is overwritten by values generated during the reduction.

LDAB
  LDAB is INTEGER
  The leading dimension of the array A. LDAB >= KL+KU+1.

D
  D is DOUBLE PRECISION array, dimension (min(M,N))
  The diagonal elements of the bidiagonal matrix B.

E
  E is DOUBLE PRECISION array, dimension (min(M,N)-1)
  The superdiagonal elements of the bidiagonal matrix B.

Q
  Q is DOUBLE PRECISION array, dimension (LDQ,M)
  If VECT = 'Q' or 'B', the m-by-m orthogonal matrix Q.
  If VECT = 'N' or 'P', the array Q is not referenced.

LDQ
  LDQ is INTEGER
  The leading dimension of the array Q.
  LDQ >= max(1,M) if VECT = 'Q' or 'B'; LDQ >= 1 otherwise.

PT
  PT is DOUBLE PRECISION array, dimension (LDPT,N)
  If VECT = 'P' or 'B', the n-by-n orthogonal matrix P'.
  If VECT = 'N' or 'Q', the array PT is not referenced.

LDPT
  LDPT is INTEGER
  The leading dimension of the array PT.
  LDPT >= max(1,N) if VECT = 'P' or 'B'; LDPT >= 1 otherwise.
```

```
C
  C is DOUBLE PRECISION array, dimension (LDC,NCC)
  On entry, an m-by-ncc matrix C.
  On exit, C is overwritten by Q**T*C.
  C is not referenced if NCC = 0.

LDC
  LDC is INTEGER
  The leading dimension of the array C.
  LDC >= max(1,M) if NCC > 0; LDC >= 1 if NCC = 0.

WORK
  WORK is DOUBLE PRECISION array, dimension (2*max(M,N))

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

44.1.3 fortran code

— dgbbrd.f —

```
*  ==================================================================
      SUBROUTINE DGBBRD( VECT, M, N, NCC, KL, KU, AB, LDAB, D, E, Q,
     $                   LDQ, PT, LDPT, C, LDC, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          VECT
      INTEGER            INFO, KL, KU, LDAB, LDC, LDPT, LDQ, M, N, NCC
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   AB( LDAB, * ), C( LDC, * ), D( * ), E( * ),
     $                   PT( LDPT, * ), Q( LDQ, * ), WORK( * )
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
```

```
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            WANTB, WANTC, WANTPT, WANTQ
      INTEGER            I, INCA, J, J1, J2, KB, KB1, KK, KLM, KLU1,
     $                   KUN, L, MINMN, ML, ML0, MN, MU, MU0, NR, NRT
      DOUBLE PRECISION   RA, RB, RC, RS
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARGV, DLARTG, DLARTV, DLASET, DROT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      WANTB = LSAME( VECT, 'B' )
      WANTQ = LSAME( VECT, 'Q' ) .OR. WANTB
      WANTPT = LSAME( VECT, 'P' ) .OR. WANTB
      WANTC = NCC.GT.0
      KLU1 = KL + KU + 1
      INFO = 0
      IF( .NOT.WANTQ .AND. .NOT.WANTPT .AND. .NOT.LSAME( VECT, 'N' ) )
     $     THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NCC.LT.0 ) THEN
         INFO = -4
      ELSE IF( KL.LT.0 ) THEN
         INFO = -5
      ELSE IF( KU.LT.0 ) THEN
         INFO = -6
      ELSE IF( LDAB.LT.KLU1 ) THEN
         INFO = -8
      ELSE IF( LDQ.LT.1 .OR. WANTQ .AND. LDQ.LT.MAX( 1, M ) ) THEN
         INFO = -12
      ELSE IF( LDPT.LT.1 .OR. WANTPT .AND. LDPT.LT.MAX( 1, N ) ) THEN
         INFO = -14
      ELSE IF( LDC.LT.1 .OR. WANTC .AND. LDC.LT.MAX( 1, M ) ) THEN
         INFO = -16
```

```
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGBBRD', -INFO )
         RETURN
      END IF
*
*     Initialize Q and P**T to the unit matrix, if needed
*
      IF( WANTQ )
     $   CALL DLASET( 'Full', M, M, ZERO, ONE, Q, LDQ )
      IF( WANTPT )
     $   CALL DLASET( 'Full', N, N, ZERO, ONE, PT, LDPT )
*
*     Quick return if possible.
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
      MINMN = MIN( M, N )
*
      IF( KL+KU.GT.1 ) THEN
*
*        Reduce to upper bidiagonal form if KU > 0; if KU = 0, reduce
*        first to lower bidiagonal form and then transform to upper
*        bidiagonal
*
         IF( KU.GT.0 ) THEN
            ML0 = 1
            MU0 = 2
         ELSE
            ML0 = 2
            MU0 = 1
         END IF
*
*        Wherever possible, plane rotations are generated and applied in
*        vector operations of length NR over the index set J1:J2:KLU1.
*
*        The sines of the plane rotations are stored in WORK(1:max(m,n))
*        and the cosines in WORK(max(m,n)+1:2*max(m,n)).
*
         MN = MAX( M, N )
         KLM = MIN( M-1, KL )
         KUN = MIN( N-1, KU )
         KB = KLM + KUN
         KB1 = KB + 1
         INCA = KB1*LDAB
         NR = 0
         J1 = KLM + 2
         J2 = 1 - KUN
*
```

```
         DO 90 I = 1, MINMN
*
*           Reduce i-th column and i-th row of matrix to bidiagonal form
*
            ML = KLM + 1
            MU = KUN + 1
            DO 80 KK = 1, KB
               J1 = J1 + KB
               J2 = J2 + KB
*
*              generate plane rotations to annihilate nonzero elements
*              which have been created below the band
*
               IF( NR.GT.0 )
     $            CALL DLARGV( NR, AB( KLU1, J1-KLM-1 ), INCA,
     $                         WORK( J1 ), KB1, WORK( MN+J1 ), KB1 )
*
*              apply plane rotations from the left
*
               DO 10 L = 1, KB
                  IF( J2-KLM+L-1.GT.N ) THEN
                     NRT = NR - 1
                  ELSE
                     NRT = NR
                  END IF
                  IF( NRT.GT.0 )
     $               CALL DLARTV( NRT, AB( KLU1-L, J1-KLM+L-1 ), INCA,
     $                            AB( KLU1-L+1, J1-KLM+L-1 ), INCA,
     $                            WORK( MN+J1 ), WORK( J1 ), KB1 )
   10          CONTINUE
*
               IF( ML.GT.ML0 ) THEN
                  IF( ML.LE.M-I+1 ) THEN
*
*                    generate plane rotation to annihilate a(i+ml-1,i)
*                    within the band, and apply rotation from the left
*
                     CALL DLARTG( AB( KU+ML-1, I ), AB( KU+ML, I ),
     $                            WORK( MN+I+ML-1 ), WORK( I+ML-1 ),
     $                            RA )
                     AB( KU+ML-1, I ) = RA
                     IF( I.LT.N )
     $                  CALL DROT( MIN( KU+ML-2, N-I ),
     $                             AB( KU+ML-2, I+1 ), LDAB-1,
     $                             AB( KU+ML-1, I+1 ), LDAB-1,
     $                             WORK( MN+I+ML-1 ), WORK( I+ML-1 ) )
                  END IF
                  NR = NR + 1
                  J1 = J1 - KB1
               END IF
```

```
*
              IF( WANTQ ) THEN
*
*                accumulate product of plane rotations in Q
*
                DO 20 J = J1, J2, KB1
                   CALL DROT( M, Q( 1, J-1 ), 1, Q( 1, J ), 1,
     $                        WORK( MN+J ), WORK( J ) )
   20           CONTINUE
              END IF
*
              IF( WANTC ) THEN
*
*                apply plane rotations to C
*
                DO 30 J = J1, J2, KB1
                   CALL DROT( NCC, C( J-1, 1 ), LDC, C( J, 1 ), LDC,
     $                        WORK( MN+J ), WORK( J ) )
   30           CONTINUE
              END IF
*
              IF( J2+KUN.GT.N ) THEN
*
*                adjust J2 to keep within the bounds of the matrix
*
                NR = NR - 1
                J2 = J2 - KB1
              END IF
*
              DO 40 J = J1, J2, KB1
*
*                create nonzero element a(j-1,j+ku) above the band
*                and store it in WORK(n+1:2*n)
*
                WORK( J+KUN ) = WORK( J )*AB( 1, J+KUN )
                AB( 1, J+KUN ) = WORK( MN+J )*AB( 1, J+KUN )
   40         CONTINUE
*
*             generate plane rotations to annihilate nonzero elements
*             which have been generated above the band
*
              IF( NR.GT.0 )
     $           CALL DLARGV( NR, AB( 1, J1+KUN-1 ), INCA,
     $                        WORK( J1+KUN ), KB1, WORK( MN+J1+KUN ),
     $                        KB1 )
*
*             apply plane rotations from the right
*
              DO 50 L = 1, KB
                 IF( J2+L-1.GT.M ) THEN
```

```
                  NRT = NR - 1
               ELSE
                  NRT = NR
               END IF
               IF( NRT.GT.0 )
     $            CALL DLARTV( NRT, AB( L+1, J1+KUN-1 ), INCA,
     $                         AB( L, J1+KUN ), INCA,
     $                         WORK( MN+J1+KUN ), WORK( J1+KUN ),
     $                         KB1 )
   50          CONTINUE
*
               IF( ML.EQ.ML0 .AND. MU.GT.MU0 ) THEN
               IF( MU.LE.N-I+1 ) THEN
*
*                 generate plane rotation to annihilate a(i,i+mu-1)
*                 within the band, and apply rotation from the right
*
                  CALL DLARTG( AB( KU-MU+3, I+MU-2 ),
     $                         AB( KU-MU+2, I+MU-1 ),
     $                         WORK( MN+I+MU-1 ), WORK( I+MU-1 ),
     $                         RA )
                  AB( KU-MU+3, I+MU-2 ) = RA
                  CALL DROT( MIN( KL+MU-2, M-I ),
     $                       AB( KU-MU+4, I+MU-2 ), 1,
     $                       AB( KU-MU+3, I+MU-1 ), 1,
     $                       WORK( MN+I+MU-1 ), WORK( I+MU-1 ) )
               END IF
               NR = NR + 1
               J1 = J1 - KB1
               END IF
*
               IF( WANTPT ) THEN
*
*              accumulate product of plane rotations in P**T
*
               DO 60 J = J1, J2, KB1
                  CALL DROT( N, PT( J+KUN-1, 1 ), LDPT,
     $                       PT( J+KUN, 1 ), LDPT, WORK( MN+J+KUN ),
     $                       WORK( J+KUN ) )
   60          CONTINUE
               END IF
*
               IF( J2+KB.GT.M ) THEN
*
*              adjust J2 to keep within the bounds of the matrix
*
               NR = NR - 1
               J2 = J2 - KB1
               END IF
*
```

```
              DO 70 J = J1, J2, KB1
*
*                create nonzero element a(j+kl+ku,j+ku-1) below the
*                band and store it in WORK(1:n)
*
                 WORK( J+KB ) = WORK( J+KUN )*AB( KLU1, J+KUN )
                 AB( KLU1, J+KUN ) = WORK( MN+J+KUN )*AB( KLU1, J+KUN )
   70         CONTINUE
*
              IF( ML.GT.ML0 ) THEN
                 ML = ML - 1
              ELSE
                 MU = MU - 1
              END IF
   80       CONTINUE
   90    CONTINUE
      END IF
*
      IF( KU.EQ.0 .AND. KL.GT.0 ) THEN
*
*        A has been reduced to lower bidiagonal form
*
*        Transform lower bidiagonal form to upper bidiagonal by applying
*        plane rotations from the left, storing diagonal elements in D
*        and off-diagonal elements in E
*
         DO 100 I = 1, MIN( M-1, N )
            CALL DLARTG( AB( 1, I ), AB( 2, I ), RC, RS, RA )
            D( I ) = RA
            IF( I.LT.N ) THEN
               E( I ) = RS*AB( 1, I+1 )
               AB( 1, I+1 ) = RC*AB( 1, I+1 )
            END IF
            IF( WANTQ )
     $         CALL DROT( M, Q( 1, I ), 1, Q( 1, I+1 ), 1, RC, RS )
            IF( WANTC )
     $         CALL DROT( NCC, C( I, 1 ), LDC, C( I+1, 1 ), LDC, RC,
     $                    RS )
  100    CONTINUE
         IF( M.LE.N )
     $      D( M ) = AB( 1, M )
      ELSE IF( KU.GT.0 ) THEN
*
*        A has been reduced to upper bidiagonal form
*
         IF( M.LT.N ) THEN
*
*           Annihilate a(m,m+1) by applying plane rotations from the
*           right, storing diagonal elements in D and off-diagonal
*           elements in E
```

```
*
           RB = AB( KU, M+1 )
           DO 110 I = M, 1, -1
              CALL DLARTG( AB( KU+1, I ), RB, RC, RS, RA )
              D( I ) = RA
              IF( I.GT.1 ) THEN
                 RB = -RS*AB( KU, I )
                 E( I-1 ) = RC*AB( KU, I )
              END IF
              IF( WANTPT )
     $           CALL DROT( N, PT( I, 1 ), LDPT, PT( M+1, 1 ), LDPT,
     $                      RC, RS )
  110      CONTINUE
        ELSE
*
*          Copy off-diagonal elements to E and diagonal elements to D
*
           DO 120 I = 1, MINMN - 1
              E( I ) = AB( KU, I+1 )
  120      CONTINUE
           DO 130 I = 1, MINMN
              D( I ) = AB( KU+1, I )
  130      CONTINUE
        END IF
      ELSE
*
*       A is diagonal. Set elements of E to zero and copy diagonal
*       elements to D.
*
        DO 140 I = 1, MINMN - 1
           E( I ) = ZERO
  140   CONTINUE
        DO 150 I = 1, MINMN
           D( I ) = AB( 1, I )
  150   CONTINUE
      END IF
      RETURN
*
*     End of DGBBRD
*
      END
```

———-

44.1.4 lisp code

— dgbbrd.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dgbbrd (vect m n ncc kl ku ab ldab d e q ldq pt ldpt c ldc work info)
    (declare (type (array double-float (*)) work c pt q e d ab)
             (type (f2cl-lib:integer4) info ldc ldpt ldq ldab ku kl ncc n m)
             (type (string *) vect))
    (f2cl-lib:with-multi-array-data
        ((vect character vect-%data% vect-%offset%)
         (ab double-float ab-%data% ab-%offset%)
         (d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%)
         (q double-float q-%data% q-%offset%)
         (pt double-float pt-%data% pt-%offset%)
         (c double-float c-%data% c-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((ra 0.0d0) (rb 0.0d0) (rc 0.0d0) (rs 0.0d0) (i 0) (inca 0) (j 0)
             (j1 0) (j2 0) (kb 0) (kb1 0) (kk 0) (klm 0) (klu1 0) (kun 0) (l 0)
             (minmn 0) (ml 0) (ml0 0) (mn 0) (mu 0) (mu0 0) (nr 0) (nrt 0)
             (wantb nil) (wantc nil) (wantpt nil) (wantq nil))
        (declare (type (double-float) ra rb rc rs)
                 (type (f2cl-lib:integer4) i inca j j1 j2 kb kb1 kk klm klu1
                  kun l minmn ml ml0 mn mu mu0 nr nrt)
                 (type f2cl-lib:logical wantb wantc wantpt wantq))
        (setf wantb
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame vect "B")
                  (declare (ignore var-1))
                  (when var-0 (setf vect var-0))
                  ret-val))
        (setf wantq
                (or
                 (multiple-value-bind (ret-val var-0 var-1)
                     (lsame vect "Q")
                   (declare (ignore var-1))
                   (when var-0 (setf vect var-0))
                   ret-val)
                 wantb))
        (setf wantpt
                (or
                 (multiple-value-bind (ret-val var-0 var-1)
                     (lsame vect "P")
                   (declare (ignore var-1))
                   (when var-0 (setf vect var-0))
                   ret-val)
                 wantb))
        (setf wantc (> ncc 0))
        (setf klu1 (f2cl-lib:int-add kl ku 1))
        (setf info 0)
```

```
(cond
 ((and (not wantq) (not wantpt)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame vect "N")
          (declare (ignore var-1))
          (when var-0 (setf vect var-0))
          ret-val)))
  (setf info -1))
 ((< m 0) (setf info -2)) ((< n 0) (setf info -3))
 ((< ncc 0) (setf info -4)) ((< kl 0) (setf info -5))
 ((< ku 0) (setf info -6)) ((< ldab klu1) (setf info -8))
 ((or (< ldq 1)
      (and wantq
           (< ldq
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 m)))))
  (setf info -12))
 ((or (< ldpt 1)
      (and wantpt
           (< ldpt
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 n)))))
  (setf info -14))
 ((or (< ldc 1)
      (and wantc
           (< ldc
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 m)))))
  (setf info -16)))
(cond
 ((/= info 0) (xerbla "DGBBRD" (f2cl-lib:int-sub info))
  (go end_label)))
(if wantq
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "Full" m m zero one q ldq)
      (declare (ignore var-0 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf m var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldq var-6))))
(if wantpt
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "Full" n n zero one pt ldpt)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
```

```
      (when var-6 (setf ldpt var-6))))
(if (or (= m 0) (= n 0))
    (go end_label))
(setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(cond
 ((> (f2cl-lib:int-add kl ku) 1)
  (cond ((> ku 0) (setf ml0 1) (setf mu0 2))
        (t (setf ml0 2) (setf mu0 1)))
  (setf mn (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
  (setf klm
          (min (the f2cl-lib:integer4 (f2cl-lib:int-sub m 1))
               (the f2cl-lib:integer4 kl)))
  (setf kun
          (min (the f2cl-lib:integer4 (f2cl-lib:int-sub n 1))
               (the f2cl-lib:integer4 ku)))
  (setf kb (f2cl-lib:int-add klm kun))
  (setf kb1 (f2cl-lib:int-add kb 1))
  (setf inca (f2cl-lib:int-mul kb1 ldab)) (setf nr 0)
  (setf j1 (f2cl-lib:int-add klm 2)) (setf j2 (f2cl-lib:int-sub 1 kun))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i minmn) nil)
    (tagbody
      (setf ml (f2cl-lib:int-add klm 1))
      (setf mu (f2cl-lib:int-add kun 1))
      (f2cl-lib:fdo (kk 1 (f2cl-lib:int-add kk 1))
                    ((> kk kb) nil)
        (tagbody
          (setf j1 (f2cl-lib:int-add j1 kb))
          (setf j2 (f2cl-lib:int-add j2 kb))
          (if (> nr 0)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (dlargv nr
                   (f2cl-lib:array-slice ab-%data% double-float
                                         (klu1
                                          (f2cl-lib:int-sub j1 klm 1))
                                         ((1 ldab) (1 *)) ab-%offset%)
                   inca
                   (f2cl-lib:array-slice work-%data% double-float (j1)
                                         ((1 *)) work-%offset%)
                   kb1
                   (f2cl-lib:array-slice work-%data% double-float
                                         ((+ mn j1)) ((1 *))
                                         work-%offset%)
                   kb1)
                (declare (ignore var-1 var-3 var-5))
                (when var-0 (setf nr var-0))
                (when var-2 (setf inca var-2))
                (when var-4 (setf kb1 var-4))
                (when var-6 (setf kb1 var-6))))
```

```
(f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
              ((> l kb) nil)
  (tagbody
    (cond
     ((>
       (f2cl-lib:int-add j2 (f2cl-lib:int-sub klm) l
                          (f2cl-lib:int-sub 1))
       n)
      (setf nrt (f2cl-lib:int-sub nr 1)))
     (t (setf nrt nr)))
    (if (> nrt 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (dlartv nrt
             (f2cl-lib:array-slice ab-%data% double-float
                                   ((+ klu1
                                       (f2cl-lib:int-sub l))
                                    (f2cl-lib:int-sub
                                     (f2cl-lib:int-add
                                      (f2cl-lib:int-sub j1
                                                        klm)
                                      l)
                                     1))
                                   ((1 ldab) (1 *))
                                   ab-%offset%)
             inca
             (f2cl-lib:array-slice ab-%data% double-float
                                   ((+ klu1
                                       (f2cl-lib:int-sub l)
                                       1)
                                    (f2cl-lib:int-sub
                                     (f2cl-lib:int-add
                                      (f2cl-lib:int-sub j1
                                                        klm)
                                      l)
                                     1))
                                   ((1 ldab) (1 *))
                                   ab-%offset%)
             inca
             (f2cl-lib:array-slice work-%data% double-float
                                   ((+ mn j1)) ((1 *))
                                   work-%offset%)
             (f2cl-lib:array-slice work-%data% double-float
                                   (j1) ((1 *))
                                   work-%offset%)
             kb1)
          (declare (ignore var-1 var-3 var-5 var-6))
          (when var-0 (setf nrt var-0))
          (when var-2 (setf inca var-2))
          (when var-4 (setf inca var-4))
```

```
            (when var-7 (setf kb1 var-7))))
    label10))
  (cond
   ((> ml ml0)
    (cond
     ((<= ml (f2cl-lib:int-add m (f2cl-lib:int-sub i) 1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dlartg
           (f2cl-lib:array-slice ab-%data% double-float
                                 ((+ ku ml
                                     (f2cl-lib:int-sub 1))
                                  i)
                                 ((1 ldab) (1 *)) ab-%offset%)
           (f2cl-lib:array-slice ab-%data% double-float
                                 ((+ ku ml) i) ((1 ldab) (1 *))
                                 ab-%offset%)
           (f2cl-lib:array-slice work-%data% double-float
                                 ((+ mn i ml
                                     (f2cl-lib:int-sub 1)))
                                 ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% double-float
                                 ((+ i ml
                                     (f2cl-lib:int-sub 1)))
                                 ((1 *)) work-%offset%)
           ra)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf ra var-4)))
      (setf (f2cl-lib:fref ab-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add ku ml) 1)
                            i)
                           ((1 ldab) (1 *)) ab-%offset%)
            ra)
      (if (< i n)
          (drot
           (min
            (the f2cl-lib:integer4
                 (f2cl-lib:int-sub (f2cl-lib:int-add ku ml) 2))
            (the f2cl-lib:integer4 (f2cl-lib:int-sub n i)))
           (f2cl-lib:array-slice ab-%data% double-float
                                 ((+ ku ml
                                     (f2cl-lib:int-sub 2))
                                  (f2cl-lib:int-add i 1))
                                 ((1 ldab) (1 *)) ab-%offset%)
           (f2cl-lib:int-sub ldab 1)
           (f2cl-lib:array-slice ab-%data% double-float
                                 ((+ ku ml
                                     (f2cl-lib:int-sub 1))
                                  (f2cl-lib:int-add i 1))
                                 ((1 ldab) (1 *)) ab-%offset%)
```

```
            (f2cl-lib:int-sub ldab 1)
            (f2cl-lib:array-slice work-%data% double-float
                                  ((+ mn i ml
                                      (f2cl-lib:int-sub 1)))
                                  ((1 *)) work-%offset%)
            (f2cl-lib:array-slice work-%data% double-float
                                  ((+ i ml
                                      (f2cl-lib:int-sub 1)))
                                  ((1 *)) work-%offset%)))))
    (setf nr (f2cl-lib:int-add nr 1))
    (setf j1 (f2cl-lib:int-sub j1 kb1))))
  (cond
   (wantq
    (f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
                  ((> j j2) nil)
      (tagbody
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (drot m
             (f2cl-lib:array-slice q-%data% double-float
                                   (1 (f2cl-lib:int-sub j 1))
                                   ((1 ldq) (1 *)) q-%offset%)
             1
             (f2cl-lib:array-slice q-%data% double-float (1 j)
                                   ((1 ldq) (1 *)) q-%offset%)
             1
             (f2cl-lib:array-slice work-%data% double-float
                                   ((+ mn j)) ((1 *))
                                   work-%offset%)
             (f2cl-lib:array-slice work-%data% double-float (j)
                                   ((1 *)) work-%offset%))
          (declare
           (ignore var-1 var-2 var-3 var-4 var-5 var-6))
          (when var-0 (setf m var-0)))
       label20))))
  (cond
   (wantc
    (f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
                  ((> j j2) nil)
      (tagbody
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (drot ncc
             (f2cl-lib:array-slice c-%data% double-float
                                   ((+ j (f2cl-lib:int-sub 1))
                                    1)
                                   ((1 ldc) (1 *)) c-%offset%)
             ldc
             (f2cl-lib:array-slice c-%data% double-float (j 1)
                                   ((1 ldc) (1 *)) c-%offset%)
```

```
            ldc
            (f2cl-lib:array-slice work-%data% double-float
                                  ((+ mn j)) ((1 *))
                                  work-%offset%)
            (f2cl-lib:array-slice work-%data% double-float (j)
                                  ((1 *)) work-%offset%))
        (declare (ignore var-1 var-3 var-5 var-6))
        (when var-0 (setf ncc var-0))
        (when var-2 (setf ldc var-2))
        (when var-4 (setf ldc var-4)))
     label30))))
(cond
 ((> (f2cl-lib:int-add j2 kun) n)
  (setf nr (f2cl-lib:int-sub nr 1))
  (setf j2 (f2cl-lib:int-sub j2 kb1))))
(f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
              ((> j j2) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add j kun)) ((1 *))
                         work-%offset%)
            (*
             (f2cl-lib:fref work-%data% (j) ((1 *))
                            work-%offset%)
             (f2cl-lib:fref ab-%data%
                            (1 (f2cl-lib:int-add j kun))
                            ((1 ldab) (1 *)) ab-%offset%)))
    (setf (f2cl-lib:fref ab-%data%
                         (1 (f2cl-lib:int-add j kun))
                         ((1 ldab) (1 *)) ab-%offset%)
            (*
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add mn j)) ((1 *))
                            work-%offset%)
             (f2cl-lib:fref ab-%data%
                            (1 (f2cl-lib:int-add j kun))
                            ((1 ldab) (1 *)) ab-%offset%)))
   label40))
(if (> nr 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlargv nr
         (f2cl-lib:array-slice ab-%data% double-float
                               (1
                                (f2cl-lib:int-sub
                                 (f2cl-lib:int-add j1 kun)
                                 1))
                               ((1 ldab) (1 *)) ab-%offset%)
         inca
         (f2cl-lib:array-slice work-%data% double-float
```

```
                                  ((+ j1 kun)) ((1 *))
                                  work-%offset%)
            kb1
            (f2cl-lib:array-slice work-%data% double-float
                                  ((+ mn j1 kun)) ((1 *))
                                  work-%offset%)
            kb1)
        (declare (ignore var-1 var-3 var-5))
        (when var-0 (setf nr var-0))
        (when var-2 (setf inca var-2))
        (when var-4 (setf kb1 var-4))
        (when var-6 (setf kb1 var-6))))
(f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
              ((> l kb) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:int-add j2 l (f2cl-lib:int-sub 1)) m)
      (setf nrt (f2cl-lib:int-sub nr 1)))
     (t (setf nrt nr)))
    (if (> nrt 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (dlartv nrt
             (f2cl-lib:array-slice ab-%data% double-float
                                   ((+ l 1)
                                    (f2cl-lib:int-sub
                                     (f2cl-lib:int-add j1
                                                       kun)
                                    1))
                                   ((1 ldab) (1 *))
                                   ab-%offset%)
             inca
             (f2cl-lib:array-slice ab-%data% double-float
                                   (l
                                    (f2cl-lib:int-add j1
                                                      kun))
                                   ((1 ldab) (1 *))
                                   ab-%offset%)
             inca
             (f2cl-lib:array-slice work-%data% double-float
                                   ((+ mn j1 kun)) ((1 *))
                                   work-%offset%)
             (f2cl-lib:array-slice work-%data% double-float
                                   ((+ j1 kun)) ((1 *))
                                   work-%offset%)
             kb1)
            (declare (ignore var-1 var-3 var-5 var-6))
            (when var-0 (setf nrt var-0))
            (when var-2 (setf inca var-2))
            (when var-4 (setf inca var-4))
```

```
            (when var-7 (setf kb1 var-7))))
    label50))
  (cond
   ((and (= ml ml0) (> mu mu0))
    (cond
     ((<= mu (f2cl-lib:int-add n (f2cl-lib:int-sub i) 1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dlartg
           (f2cl-lib:array-slice ab-%data% double-float
                                 ((+ ku (f2cl-lib:int-sub mu)
                                     3)
                                  (f2cl-lib:int-sub
                                   (f2cl-lib:int-add i mu) 2))
                                 ((1 ldab) (1 *)) ab-%offset%)
           (f2cl-lib:array-slice ab-%data% double-float
                                 ((+ ku (f2cl-lib:int-sub mu)
                                     2)
                                  (f2cl-lib:int-sub
                                   (f2cl-lib:int-add i mu) 1))
                                 ((1 ldab) (1 *)) ab-%offset%)
           (f2cl-lib:array-slice work-%data% double-float
                                 ((+ mn i mu
                                     (f2cl-lib:int-sub 1)))
                                 ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% double-float
                                 ((+ i mu
                                     (f2cl-lib:int-sub 1)))
                                 ((1 *)) work-%offset%)
           ra)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf ra var-4)))
      (setf (f2cl-lib:fref ab-%data%
                           ((f2cl-lib:int-add
                             (f2cl-lib:int-sub ku mu) 3)
                            (f2cl-lib:int-sub
                             (f2cl-lib:int-add i mu) 2))
                           ((1 ldab) (1 *)) ab-%offset%)
            ra)
      (drot
       (min
        (the f2cl-lib:integer4
             (f2cl-lib:int-sub (f2cl-lib:int-add kl mu) 2))
        (the f2cl-lib:integer4 (f2cl-lib:int-sub m i)))
       (f2cl-lib:array-slice ab-%data% double-float
                             ((+ ku (f2cl-lib:int-sub mu) 4)
                              (f2cl-lib:int-sub
                               (f2cl-lib:int-add i mu) 2))
                             ((1 ldab) (1 *)) ab-%offset%)
       1
       (f2cl-lib:array-slice ab-%data% double-float
```

```
                              ((+ ku (f2cl-lib:int-sub mu) 3)
                               (f2cl-lib:int-sub
                                (f2cl-lib:int-add i mu) 1))
                              ((1 ldab) (1 *)) ab-%offset%)
      1
      (f2cl-lib:array-slice work-%data% double-float
                              ((+ mn i mu (f2cl-lib:int-sub 1)))
                              ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float
                              ((+ i mu (f2cl-lib:int-sub 1)))
                              ((1 *)) work-%offset%))))
  (setf nr (f2cl-lib:int-add nr 1))
  (setf j1 (f2cl-lib:int-sub j1 kb1))))
(cond
 (wantpt
  (f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
                ((> j j2) nil)
    (tagbody
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (drot n
           (f2cl-lib:array-slice pt-%data% double-float
                                 ((+ j kun
                                     (f2cl-lib:int-sub 1))
                                  1)
                                 ((1 ldpt) (1 *))
                                 pt-%offset%)
           ldpt
           (f2cl-lib:array-slice pt-%data% double-float
                                 ((+ j kun) 1)
                                 ((1 ldpt) (1 *))
                                 pt-%offset%)
           ldpt
           (f2cl-lib:array-slice work-%data% double-float
                                 ((+ mn j kun)) ((1 *))
                                 work-%offset%)
           (f2cl-lib:array-slice work-%data% double-float
                                 ((+ j kun)) ((1 *))
                                 work-%offset%))
        (declare (ignore var-1 var-3 var-5 var-6))
        (when var-0 (setf n var-0))
        (when var-2 (setf ldpt var-2))
        (when var-4 (setf ldpt var-4)))
     label60))))
(cond
 ((> (f2cl-lib:int-add j2 kb) m)
  (setf nr (f2cl-lib:int-sub nr 1))
  (setf j2 (f2cl-lib:int-sub j2 kb1))))
(f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
              ((> j j2) nil)
```

```
                (tagbody
                  (setf (f2cl-lib:fref work-%data%
                                       ((f2cl-lib:int-add j kb)) ((1 *))
                                       work-%offset%)
                        (*
                         (f2cl-lib:fref work-%data%
                                        ((f2cl-lib:int-add j kun))
                                        ((1 *)) work-%offset%)
                         (f2cl-lib:fref ab-%data%
                                        (klu1 (f2cl-lib:int-add j kun))
                                        ((1 ldab) (1 *)) ab-%offset%)))
                  (setf (f2cl-lib:fref ab-%data%
                                       (klu1 (f2cl-lib:int-add j kun))
                                       ((1 ldab) (1 *)) ab-%offset%)
                        (*
                         (f2cl-lib:fref work-%data%
                                        ((f2cl-lib:int-add mn j kun))
                                        ((1 *)) work-%offset%)
                         (f2cl-lib:fref ab-%data%
                                        (klu1 (f2cl-lib:int-add j kun))
                                        ((1 ldab) (1 *)) ab-%offset%)))
                label70))
              (cond ((> ml ml0) (setf ml (f2cl-lib:int-sub ml 1)))
                    (t (setf mu (f2cl-lib:int-sub mu 1))))
            label80))
        label90))))
(cond
 ((and (= ku 0) (> kl 0))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
                     (the f2cl-lib:integer4 n)))
                 nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dlartg
           (f2cl-lib:array-slice ab-%data% double-float (1 i)
                                 ((1 ldab) (1 *)) ab-%offset%)
           (f2cl-lib:array-slice ab-%data% double-float (2 i)
                                 ((1 ldab) (1 *)) ab-%offset%)
           rc rs ra)
        (declare (ignore var-0 var-1))
        (when var-2 (setf rc var-2))
        (when var-3 (setf rs var-3))
        (when var-4 (setf ra var-4)))
      (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) ra)
      (cond
       ((< i n)
```

```
          (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                  (* rs
                     (f2cl-lib:fref ab-%data% (1 (f2cl-lib:int-add i 1))
                                    ((1 ldab) (1 *)) ab-%offset%)))
          (setf (f2cl-lib:fref ab-%data% (1 (f2cl-lib:int-add i 1))
                               ((1 ldab) (1 *)) ab-%offset%)
                  (* rc
                     (f2cl-lib:fref ab-%data% (1 (f2cl-lib:int-add i 1))
                                    ((1 ldab) (1 *)) ab-%offset%)))))
      (if wantq
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (drot m
               (f2cl-lib:array-slice q-%data% double-float (1 i)
                                     ((1 ldq) (1 *)) q-%offset%)
               1
               (f2cl-lib:array-slice q-%data% double-float
                                     (1 (f2cl-lib:int-add i 1))
                                     ((1 ldq) (1 *)) q-%offset%)
               1 rc rs)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf m var-0))
            (when var-5 (setf rc var-5))
            (when var-6 (setf rs var-6))))
      (if wantc
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (drot ncc
               (f2cl-lib:array-slice c-%data% double-float (i 1)
                                     ((1 ldc) (1 *)) c-%offset%)
               ldc
               (f2cl-lib:array-slice c-%data% double-float ((+ i 1) 1)
                                     ((1 ldc) (1 *)) c-%offset%)
               ldc rc rs)
            (declare (ignore var-1 var-3))
            (when var-0 (setf ncc var-0))
            (when var-2 (setf ldc var-2))
            (when var-4 (setf ldc var-4))
            (when var-5 (setf rc var-5))
            (when var-6 (setf rs var-6))))
    label100))
 (if (<= m n)
     (setf (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%)
             (f2cl-lib:fref ab-%data% (1 m) ((1 ldab) (1 *))
                            ab-%offset%))))
((> ku 0)
 (cond
  ((< m n)
   (setf rb
           (f2cl-lib:fref ab-%data% (ku (f2cl-lib:int-add m 1))
```

```
                        ((1 ldab) (1 *)) ab-%offset%))
 (f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
               ((> i 1) nil)
   (tagbody
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (dlartg
          (f2cl-lib:array-slice ab-%data% double-float ((+ ku 1) i)
                                ((1 ldab) (1 *)) ab-%offset%)
          rb rc rs ra)
       (declare (ignore var-0))
       (when var-1 (setf rb var-1))
       (when var-2 (setf rc var-2))
       (when var-3 (setf rs var-3))
       (when var-4 (setf ra var-4)))
     (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) ra)
     (cond
      ((> i 1)
       (setf rb
               (* (- rs)
                  (f2cl-lib:fref ab-%data% (ku i) ((1 ldab) (1 *))
                                 ab-%offset%)))
       (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                             ((1 *)) e-%offset%)
               (* rc
                  (f2cl-lib:fref ab-%data% (ku i) ((1 ldab) (1 *))
                                 ab-%offset%)))))
     (if wantpt
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
             (drot n
              (f2cl-lib:array-slice pt-%data% double-float (i 1)
                                    ((1 ldpt) (1 *)) pt-%offset%)
              ldpt
              (f2cl-lib:array-slice pt-%data% double-float
                                    ((+ m 1) 1) ((1 ldpt) (1 *))
                                    pt-%offset%)
              ldpt rc rs)
           (declare (ignore var-1 var-3))
           (when var-0 (setf n var-0))
           (when var-2 (setf ldpt var-2))
           (when var-4 (setf ldpt var-4))
           (when var-5 (setf rc var-5))
           (when var-6 (setf rs var-6))))
    label110)))
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i (f2cl-lib:int-add minmn (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
```

```
                        (f2cl-lib:fref ab-%data% (ku (f2cl-lib:int-add i 1))
                                       ((1 ldab) (1 *)) ab-%offset%))
               label120))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i minmn) nil)
              (tagbody
                (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                        (f2cl-lib:fref ab-%data% ((f2cl-lib:int-add ku 1) i)
                                       ((1 ldab) (1 *)) ab-%offset%))
               label130)))))
          (t
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i (f2cl-lib:int-add minmn (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) zero)
             label140))
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i minmn) nil)
            (tagbody
              (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                      (f2cl-lib:fref ab-%data% (1 i) ((1 ldab) (1 *))
                                     ab-%offset%))
             label150))))
         (go end_label)
        end_label
         (return
          (values vect m n ncc nil nil nil nil nil nil nil ldq nil ldpt nil ldc
                  nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgbbrd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
```

```
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::vect
  fortran-to-lisp::m
  fortran-to-lisp::n
  fortran-to-lisp::ncc nil nil nil
  nil nil nil nil
  fortran-to-lisp::ldq nil
  fortran-to-lisp::ldpt nil
  fortran-to-lisp::ldc nil
  fortran-to-lisp::info)
:calls 'nil)))
```

44.2 dgbcon

44.2.1 Axiom unit tests

— dgbcon.input —

```
)set break resume
)sys rm -f dgbcon.output
)spool dgbcon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

44.2.2 Axiom help page

— dgbcon.help —

```
======================================================================
dgbcon examples
```

```
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgbcon (NORM, N, KL, KU, AB, LDAB, IPIV, ANORM, RCOND, WORK,
           IWORK, INFO)
           DGBCON

Function/Subroutine Documentation
   subroutine dgbcon (character NORM, integer N, integer KL, integer KU,
       double precision, dimension( ldab, * ) AB, integer LDAB, integer,
       dimension( * ) IPIV, double precision ANORM, double precision RCOND,
       double precision, dimension( * ) WORK, integer, dimension( * ) IWORK,
       integer INFO)
       DGBCON

Purpose:
   DGBCON estimates the reciprocal of the condition number of a real
   general band matrix A, in either the 1-norm or the infinity-norm,
   using the LU factorization computed by DGBTRF.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as
      RCOND = 1 / ( norm(A) * norm(inv(A)) ).

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies whether the 1-norm condition number or the
   infinity-norm condition number is required:
   = '1' or 'O':  1-norm;
   = 'I':         Infinity-norm.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.
```

```
AB
  AB is DOUBLE PRECISION array, dimension (LDAB,N)
  Details of the LU factorization of the band matrix A, as
  computed by DGBTRF.  U is stored as an upper triangular band
  matrix with KL+KU superdiagonals in rows 1 to KL+KU+1, and
  the multipliers used during the factorization are stored in
  rows KL+KU+2 to 2*KL+KU+1.

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= N, row i of the matrix was
  interchanged with row IPIV(i).

ANORM
  ANORM is DOUBLE PRECISION
  If NORM = '1' or 'O', the 1-norm of the original matrix A.
  If NORM = 'I', the infinity-norm of the original matrix A.

RCOND
  RCOND is DOUBLE PRECISION
  The reciprocal of the condition number of the matrix A,
  computed as RCOND = 1/(norm(A) * norm(inv(A))).

WORK
  WORK is DOUBLE PRECISION array, dimension (3*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
```

———-

44.2.3 fortran code

— dgbcon.f —

```
*  =====================================================================
      SUBROUTINE DGBCON( NORM, N, KL, KU, AB, LDAB, IPIV, ANORM, RCOND,
     $                   WORK, IWORK, INFO )
```

```
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            INFO, KL, KU, LDAB, N
      DOUBLE PRECISION   ANORM, RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      DOUBLE PRECISION   AB( LDAB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LNOTI, ONENRM
      CHARACTER          NORMIN
      INTEGER            IX, J, JP, KASE, KASE1, KD, LM
      DOUBLE PRECISION   AINVNM, SCALE, SMLNUM, T
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX
      DOUBLE PRECISION   DDOT, DLAMCH
      EXTERNAL           LSAME, IDAMAX, DDOT, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DLACN2, DLATBS, DRSCL, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      ONENRM = NORM.EQ.'1' .OR. LSAME( NORM, 'O' )
      IF( .NOT.ONENRM .AND. .NOT.LSAME( NORM, 'I' ) ) THEN
```

```
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.2*KL+KU+1 ) THEN
         INFO = -6
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGBCON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
      SMLNUM = DLAMCH( 'Safe minimum' )
*
*     Estimate the norm of inv(A).
*
      AINVNM = ZERO
      NORMIN = 'N'
      IF( ONENRM ) THEN
         KASE1 = 1
      ELSE
         KASE1 = 2
      END IF
      KD = KL + KU + 1
      LNOTI = KL.GT.0
      KASE = 0
   10 CONTINUE
      CALL DLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.KASE1 ) THEN
*
*           Multiply by inv(L).
*
            IF( LNOTI ) THEN
               DO 20 J = 1, N - 1
```

```
                 LM = MIN( KL, N-J )
                 JP = IPIV( J )
                 T = WORK( JP )
                 IF( JP.NE.J ) THEN
                    WORK( JP ) = WORK( J )
                    WORK( J ) = T
                 END IF
                 CALL DAXPY( LM, -T, AB( KD+1, J ), 1, WORK( J+1 ), 1 )
   20         CONTINUE
           END IF
*
*          Multiply by inv(U).
*
           CALL DLATBS( 'Upper', 'No transpose', 'Non-unit', NORMIN, N,
     $                  KL+KU, AB, LDAB, WORK, SCALE, WORK( 2*N+1 ),
     $                  INFO )
        ELSE
*
*          Multiply by inv(U**T).
*
           CALL DLATBS( 'Upper', 'Transpose', 'Non-unit', NORMIN, N,
     $                  KL+KU, AB, LDAB, WORK, SCALE, WORK( 2*N+1 ),
     $                  INFO )
*
*          Multiply by inv(L**T).
*
           IF( LNOTI ) THEN
              DO 30 J = N - 1, 1, -1
                 LM = MIN( KL, N-J )
                 WORK( J ) = WORK( J ) - DDOT( LM, AB( KD+1, J ), 1,
     $                       WORK( J+1 ), 1 )
                 JP = IPIV( J )
                 IF( JP.NE.J ) THEN
                    T = WORK( JP )
                    WORK( JP ) = WORK( J )
                    WORK( J ) = T
                 END IF
   30         CONTINUE
           END IF
        END IF
*
*       Divide X by 1/SCALE if doing so will not cause overflow.
*
        NORMIN = 'Y'
        IF( SCALE.NE.ONE ) THEN
           IX = IDAMAX( N, WORK, 1 )
           IF( SCALE.LT.ABS( WORK( IX ) )*SMLNUM .OR. SCALE.EQ.ZERO )
     $        GO TO 40
           CALL DRSCL( N, SCALE, WORK, 1 )
        END IF
```

```
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
   40 CONTINUE
      RETURN
*
*     End of DGBCON
*
      END
```

———-

44.2.4 lisp code

— dgbcon.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dgbcon (norm n kl ku ab ldab ipiv anorm rcond work iwork info)
    (declare (type (double-float) rcond anorm)
             (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array double-float (*)) work ab)
             (type (f2cl-lib:integer4) info ldab ku kl n)
             (type (string *) norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (ab double-float ab-%data% ab-%offset%)
         (work double-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (ainvnm 0.0d0) (scale 0.0d0) (smlnum 0.0d0) (ix 0) (j 0) (jp 0)
             (kase 0) (kase1 0) (kd 0) (lm 0)
             (normin
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (lnoti nil) (onenrm nil) (t$ 0.0d0))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (double-float) t$ ainvnm scale smlnum)
                 (type (f2cl-lib:integer4) ix j jp kase kase1 kd lm)
                 (type (string 1) normin)
                 (type f2cl-lib:logical lnoti onenrm))
```

```
  (setf info 0)
  (setf onenrm
          (or (f2cl-lib:fstring-= norm "1")
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame norm "O")
                (declare (ignore var-1))
                (when var-0 (setf norm var-0))
                ret-val)))
  (cond
   ((and (not onenrm)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame norm "I")
            (declare (ignore var-1))
            (when var-0 (setf norm var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2)) ((< kl 0) (setf info -3))
   ((< ku 0) (setf info -4))
   ((< ldab (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
    (setf info -6))
   ((< anorm zero) (setf info -8)))
  (cond
   ((/= info 0) (xerbla "DGBCON" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf rcond zero)
  (cond ((= n 0) (setf rcond one) (go end_label))
        ((= anorm zero) (go end_label)))
  (setf smlnum (dlamch "Safe minimum"))
  (setf ainvnm zero)
  (f2cl-lib:f2cl-set-string normin "N" (string 1))
  (cond (onenrm (setf kase1 1)) (t (setf kase1 2)))
  (setf kd (f2cl-lib:int-add kl ku 1))
  (setf lnoti (> kl 0))
  (setf kase 0)
 label10
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacn2 n
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1)) ((1 *))
                             work-%offset%)
       work iwork ainvnm kase isave)
    (declare (ignore var-1 var-2 var-3 var-6))
    (when var-0 (setf n var-0))
    (when var-4 (setf ainvnm var-4))
    (when var-5 (setf kase var-5)))
  (cond
   ((/= kase 0)
    (cond
     ((= kase kase1)
      (cond
```

```
(lnoti
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (setf lm
             (min (the f2cl-lib:integer4 kl)
                  (the f2cl-lib:integer4 (f2cl-lib:int-sub n j))))
     (setf jp
             (f2cl-lib:fref ipiv-%data% (j) ((1 *))
                            ipiv-%offset%))
     (setf t$
             (f2cl-lib:fref work-%data% (jp) ((1 *))
                            work-%offset%))
     (cond
      ((/= jp j)
       (setf (f2cl-lib:fref work-%data% (jp) ((1 *))
                            work-%offset%)
               (f2cl-lib:fref work-%data% (j) ((1 *))
                              work-%offset%))
       (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
               t$)))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
         (daxpy lm (- t$)
          (f2cl-lib:array-slice ab-%data% double-float
                                ((+ kd 1) j) ((1 ldab) (1 *))
                                ab-%offset%)
          1
          (f2cl-lib:array-slice work-%data% double-float ((+ j 1))
                                ((1 *)) work-%offset%)
          1)
       (declare (ignore var-1 var-2 var-3 var-4 var-5))
       (when var-0 (setf lm var-0)))
    label20))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (dlatbs "Upper" "No transpose" "Non-unit" normin n
     (f2cl-lib:int-add kl ku) ab ldab work scale
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                           work-%offset%)
     info)
  (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-10))
  (when var-3 (setf normin var-3))
  (when var-4 (setf n var-4))
  (when var-7 (setf ldab var-7))
  (when var-9 (setf scale var-9))
  (when var-11 (setf info var-11))))
(t
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (dlatbs "Upper" "Transpose" "Non-unit" normin n
     (f2cl-lib:int-add kl ku) ab ldab work scale
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                           work-%offset%)
     info)
  (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-10))
  (when var-3 (setf normin var-3))
  (when var-4 (setf n var-4))
  (when var-7 (setf ldab var-7))
  (when var-9 (setf scale var-9))
  (when var-11 (setf info var-11)))
(cond
 (lnoti
  (f2cl-lib:fdo (j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                 (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                ((> j 1) nil)
    (tagbody
      (setf lm
              (min (the f2cl-lib:integer4 kl)
                   (the f2cl-lib:integer4 (f2cl-lib:int-sub n j))))
      (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
              (-
               (f2cl-lib:fref work-%data% (j) ((1 *))
                              work-%offset%)
               (multiple-value-bind
                   (ret-val var-0 var-1 var-2 var-3 var-4)
                   (ddot lm
                    (f2cl-lib:array-slice ab-%data% double-float
                                          ((+ kd 1) j)
                                          ((1 ldab) (1 *))
                                          ab-%offset%)
                    1
                    (f2cl-lib:array-slice work-%data% double-float
                                          ((+ j 1)) ((1 *))
                                          work-%offset%)
                    1)
                 (declare (ignore var-1 var-2 var-3 var-4))
                 (when var-0 (setf lm var-0))
                 ret-val)))
      (setf jp
              (f2cl-lib:fref ipiv-%data% (j) ((1 *))
                             ipiv-%offset%))
      (cond
       ((/= jp j)
        (setf t$
                (f2cl-lib:fref work-%data% (jp) ((1 *))
```

```
                                        work-%offset%))
                    (setf (f2cl-lib:fref work-%data% (jp) ((1 *))
                                        work-%offset%)
                            (f2cl-lib:fref work-%data% (j) ((1 *))
                                        work-%offset%))
                    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                            t$)))
                label30))))))
          (f2cl-lib:f2cl-set-string normin "Y" (string 1))
          (cond
           ((/= scale one)
            (setf ix
                    (multiple-value-bind (ret-val var-0 var-1 var-2)
                        (idamax n work 1)
                      (declare (ignore var-1 var-2))
                      (when var-0 (setf n var-0))
                      ret-val))
            (if (or
                 (< scale
                    (*
                     (abs
                      (f2cl-lib:fref work-%data% (ix) ((1 *)) work-%offset%))
                     smlnum))
                 (= scale zero))
                (go label40))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (drscl n scale work 1)
              (declare (ignore var-2 var-3))
              (when var-0 (setf n var-0))
              (when var-1 (setf scale var-1)))))
          (go label10)))
         (if (/= ainvnm zero)
             (setf rcond (/ (/ one ainvnm) anorm)))
        label40
         (go end_label)
        end_label
         (return (values norm n nil nil nil ldab nil nil rcond nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgbcon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
```

```
  (array fortran-to-lisp::integer4
   (*))
  (double-float) (double-float)
  (array double-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::norm
  fortran-to-lisp::n nil nil nil
  fortran-to-lisp::ldab nil nil
  fortran-to-lisp::rcond nil nil
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

44.3 dgbequ

44.3.1 Axiom unit tests

— **dgbequ.input** —

```
)set break resume
)sys rm -f dgbequ.output
)spool dgbequ.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

44.3.2 Axiom help page

— **dgbequ.help** —

```
======================================================================
dgbequ examples
======================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgbequ (M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND, AMAX,
           INFO)
           DGBEQU

Function/Subroutine Documentation
   subroutine dgbequ (integer M, integer N, integer KL, integer KU, double
       precision, dimension( ldab, * ) AB, integer LDAB, double precision,
       dimension( * ) R, double precision, dimension( * ) C, double precision
       ROWCND, double precision COLCND, double precision AMAX, integer INFO)
       DGBEQU

Purpose:
   DGBEQU computes row and column scalings intended to equilibrate an
   M-by-N band matrix A and reduce its condition number.  R returns the
   row scale factors and C the column scale factors, chosen to try to
   make the largest element in each row and column of the matrix B with
   elements B(i,j)=R(i)*A(i,j)*C(j) have absolute value 1.

   R(i) and C(j) are restricted to be between SMLNUM = smallest safe
   number and BIGNUM = largest safe number.  Use of these scaling
   factors is not guaranteed to reduce the condition number of A but
   works well in practice.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 AB
   AB is DOUBLE PRECISION array, dimension (LDAB,N)
```

```
  The band matrix A, stored in rows 1 to KL+KU+1.  The j-th
  column of A is stored in the j-th column of the array AB as
  follows:
  AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

R
  R is DOUBLE PRECISION array, dimension (M)
  If INFO = 0, or INFO > M, R contains the row scale factors for A.

C
  C is DOUBLE PRECISION array, dimension (N)
  If INFO = 0, C contains the column scale factors for A.

ROWCND
  ROWCND is DOUBLE PRECISION
  If INFO = 0 or INFO > M, ROWCND contains the ratio of the
  smallest R(i) to the largest R(i).  If ROWCND >= 0.1 and
  AMAX is neither too large nor too small, it is not worth scaling by R.

COLCND
  COLCND is DOUBLE PRECISION
  If INFO = 0, COLCND contains the ratio of the smallest
  C(i) to the largest C(i).  If COLCND >= 0.1, it is not worth scaling by C.

AMAX
  AMAX is DOUBLE PRECISION
  Absolute value of largest matrix element.  If AMAX is very
  close to overflow or very close to underflow, the matrix should be scaled.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, and i is
    <= M:  the i-th row of A is exactly zero
    >  M:  the (i-M)-th column of A is exactly zero
```

———-

44.3.3 fortran code

— dgbequ.f —

```
*  =====================================================================
      SUBROUTINE DGBEQU( M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $                   AMAX, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, KL, KU, LDAB, M, N
      DOUBLE PRECISION   AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   AB( LDAB, * ), C( * ), R( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, KD
      DOUBLE PRECISION   BIGNUM, RCMAX, RCMIN, SMLNUM
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
```

```
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGBEQU', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         ROWCND = ONE
         COLCND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Get machine constants.
*
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
*
*     Compute row scale factors.
*
      DO 10 I = 1, M
         R( I ) = ZERO
   10 CONTINUE
*
*     Find the maximum element in each row.
*
      KD = KU + 1
      DO 30 J = 1, N
         DO 20 I = MAX( J-KU, 1 ), MIN( J+KL, M )
            R( I ) = MAX( R( I ), ABS( AB( KD+I-J, J ) ) )
   20    CONTINUE
   30 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 40 I = 1, M
         RCMAX = MAX( RCMAX, R( I ) )
         RCMIN = MIN( RCMIN, R( I ) )
   40 CONTINUE
      AMAX = RCMAX
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
```

```
*
         DO 50 I = 1, M
            IF( R( I ).EQ.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   50    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 60 I = 1, M
            R( I ) = ONE / MIN( MAX( R( I ), SMLNUM ), BIGNUM )
   60    CONTINUE
*
*        Compute ROWCND = min(R(I)) / max(R(I))
*
         ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
*     Compute column scale factors
*
      DO 70 J = 1, N
         C( J ) = ZERO
   70 CONTINUE
*
*     Find the maximum element in each column,
*     assuming the row scaling computed above.
*
      KD = KU + 1
      DO 90 J = 1, N
         DO 80 I = MAX( J-KU, 1 ), MIN( J+KL, M )
            C( J ) = MAX( C( J ), ABS( AB( KD+I-J, J ) )*R( I ) )
   80    CONTINUE
   90 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 100 J = 1, N
         RCMIN = MIN( RCMIN, C( J ) )
         RCMAX = MAX( RCMAX, C( J ) )
  100 CONTINUE
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 110 J = 1, N
```

```
            IF( C( J ).EQ.ZERO ) THEN
               INFO = M + J
               RETURN
            END IF
  110    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 120 J = 1, N
            C( J ) = ONE / MIN( MAX( C( J ), SMLNUM ), BIGNUM )
  120    CONTINUE
*
*        Compute COLCND = min(C(J)) / max(C(J))
*
         COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
      RETURN
*
*     End of DGBEQU
*
      END
```

———-

44.3.4 lisp code

— dgbequ.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dgbequ (m n kl ku ab ldab r c rowcnd colcnd amax info)
    (declare (type (double-float) amax colcnd rowcnd)
             (type (array double-float (*)) c r ab)
             (type (f2cl-lib:integer4) info ldab ku kl n m))
    (f2cl-lib:with-multi-array-data
        ((ab double-float ab-%data% ab-%offset%)
         (r double-float r-%data% r-%offset%)
         (c double-float c-%data% c-%offset%))
      (prog ((bignum 0.0d0) (rcmax 0.0d0) (rcmin 0.0d0) (smlnum 0.0d0) (i 0)
             (j 0) (kd 0))
        (declare (type (double-float) bignum rcmax rcmin smlnum)
                 (type (f2cl-lib:integer4) i j kd))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
```

```
      ((< kl 0) (setf info -3)) ((< ku 0) (setf info -4))
      ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -6)))
(cond
 ((/= info 0) (xerbla "DGBEQU" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= m 0) (= n 0)) (setf rowcnd one) (setf colcnd one)
  (setf amax zero) (go end_label)))
(setf smlnum (dlamch "S"))
(setf bignum (/ one smlnum))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%) zero)
   label10))
(setf kd (f2cl-lib:int-add ku 1))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                    (the f2cl-lib:integer4 1))
                   (f2cl-lib:int-add i 1))
                  ((> i
                      (min
                       (the f2cl-lib:integer4 (f2cl-lib:int-add j kl))
                       (the f2cl-lib:integer4 m)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                     (abs
                      (f2cl-lib:fref ab-%data%
                                     ((f2cl-lib:int-sub
                                       (f2cl-lib:int-add kd i) j)
                                      j)
                                     ((1 ldab) (1 *)) ab-%offset%))))
       label20))
   label30))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf rcmax
            (max rcmax
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
    (setf rcmin
```

```
            (min rcmin
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
   label40))
(setf amax rcmax)
(cond
 ((= rcmin zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref r (i) ((1 *))) zero) (setf info i)
        (go end_label)))
     label50)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
              (/ one
                 (min
                  (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                       smlnum)
                  bignum)))
     label60))
  (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) zero)
   label70))
(setf kd (f2cl-lib:int-add ku 1))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                    (the f2cl-lib:integer4 1))
                   (f2cl-lib:int-add i 1))
                  ((> i
                      (min
                       (the f2cl-lib:integer4 (f2cl-lib:int-add j kl))
                       (the f2cl-lib:integer4 m)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                     (*
                      (abs
```

```
                              (f2cl-lib:fref ab-%data%
                                             ((f2cl-lib:int-sub
                                               (f2cl-lib:int-add kd i) j)
                                              j)
                                             ((1 ldab) (1 *)) ab-%offset%))
                            (f2cl-lib:fref r-%data% (i) ((1 *))
                                           r-%offset%))))
                label80))
            label90))
        (setf rcmin bignum)
        (setf rcmax zero)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf rcmin
                    (min rcmin
                         (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
            (setf rcmax
                    (max rcmax
                         (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
            label100))
        (cond
         ((= rcmin zero)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (cond
               ((= (f2cl-lib:fref c (j) ((1 *))) zero)
                (setf info (f2cl-lib:int-add m j)) (go end_label)))
             label110)))
         (t
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                      (/ one
                         (min
                          (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                               smlnum)
                          bignum)))
             label120))
          (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
        (go end_label)
       end_label
        (return
         (values nil nil nil nil nil nil nil nil rowcnd colcnd amax info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
```

```
(setf (gethash 'fortran-to-lisp::dgbequ
               fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (array double-float (*))
                                            (fortran-to-lisp::integer4)
                                            (array double-float (*))
                                            (array double-float (*))
                                            (double-float) (double-float)
                                            (double-float)
                                            (fortran-to-lisp::integer4))
                                          :return-values
                                          '(nil nil nil nil nil nil nil nil
                                            fortran-to-lisp::rowcnd
                                            fortran-to-lisp::colcnd
                                            fortran-to-lisp::amax
                                            fortran-to-lisp::info)
                                          :calls 'nil)))
```

———-

44.4 dgbequb

44.4.1 Axiom unit tests

— **dgbequb.input** —

```
)set break resume
)sys rm -f dgbequb.output
)spool dgbequb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

44.4.2 Axiom help page

— dgbequb.help —

```
====================================================================
dgbequb examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgbequb (M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND, AMAX,
           INFO)
           DGBEQUB

Function/Subroutine Documentation
   subroutine dgbequb (integer M, integer N, integer KL, integer KU, double
       precision, dimension( ldab, * ) AB, integer LDAB, double precision,
       dimension( * ) R, double precision, dimension( * ) C, double precision
       ROWCND, double precision COLCND, double precision AMAX, integer INFO)
       DGBEQUB

Purpose:
   DGBEQUB computes row and column scalings intended to equilibrate an
   M-by-N matrix A and reduce its condition number.  R returns the row
   scale factors and C the column scale factors, chosen to try to make
   the largest element in each row and column of the matrix B with
   elements B(i,j)=R(i)*A(i,j)*C(j) have an absolute value of at most
   the radix.

   R(i) and C(j) are restricted to be a power of the radix between
   SMLNUM = smallest safe number and BIGNUM = largest safe number.  Use
   of these scaling factors is not guaranteed to reduce the condition
   number of A but works well in practice.

   This routine differs from DGEEQU by restricting the scaling factors
   to a power of the radix.  Baring over- and underflow, scaling by
   these factors introduces no additional rounding errors.  However, the
   scaled entries' magnitured are no longer approximately 1 but lie
   between sqrt(radix) and 1/sqrt(radix).

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.
```

```
N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

KL
  KL is INTEGER
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

AB
  AB is DOUBLE PRECISION array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

LDAB
  LDAB is INTEGER
  The leading dimension of the array A.  LDAB >= max(1,M).

R
  R is DOUBLE PRECISION array, dimension (M)
  If INFO = 0 or INFO > M, R contains the row scale factors for A.

C
  C is DOUBLE PRECISION array, dimension (N)
  If INFO = 0,  C contains the column scale factors for A.

ROWCND
  ROWCND is DOUBLE PRECISION
  If INFO = 0 or INFO > M, ROWCND contains the ratio of the
  smallest R(i) to the largest R(i).  If ROWCND >= 0.1 and
  AMAX is neither too large nor too small, it is not worth scaling by R.

COLCND
  COLCND is DOUBLE PRECISION
  If INFO = 0, COLCND contains the ratio of the smallest
  C(i) to the largest C(i).  If COLCND >= 0.1, it is not worth scaling by C.

AMAX
  AMAX is DOUBLE PRECISION
  Absolute value of largest matrix element.  If AMAX is very
  close to overflow or very close to underflow, the matrix should be scaled.

INFO
  INFO is INTEGER
```

```
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i,  and i is
     <= M:  the i-th row of A is exactly zero
     >  M:  the (i-M)-th column of A is exactly zero
```

44.4.3 fortran code

— dgbequb.f —

```
*  =====================================================================
      SUBROUTINE DGBEQUB( M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $                    AMAX, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, KL, KU, LDAB, M, N
      DOUBLE PRECISION   AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   AB( LDAB, * ), C( * ), R( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, KD
      DOUBLE PRECISION   BIGNUM, RCMAX, RCMIN, SMLNUM, RADIX, LOGRDX
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          ABS, MAX, MIN, LOG
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGBEQUB', -INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         ROWCND = ONE
         COLCND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Get machine constants.  Assume SMLNUM is a power of the radix.
*
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      RADIX = DLAMCH( 'B' )
      LOGRDX = LOG(RADIX)
*
*     Compute row scale factors.
*
      DO 10 I = 1, M
         R( I ) = ZERO
   10 CONTINUE
*
*     Find the maximum element in each row.
*
      KD = KU + 1
      DO 30 J = 1, N
         DO 20 I = MAX( J-KU, 1 ), MIN( J+KL, M )
```

```
            R( I ) = MAX( R( I ), ABS( AB( KD+I-J, J ) ) )
   20    CONTINUE
   30 CONTINUE
      DO I = 1, M
         IF( R( I ).GT.ZERO ) THEN
            R( I ) = RADIX**INT( LOG( R( I ) ) / LOGRDX )
         END IF
      END DO
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 40 I = 1, M
         RCMAX = MAX( RCMAX, R( I ) )
         RCMIN = MIN( RCMIN, R( I ) )
   40 CONTINUE
      AMAX = RCMAX
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 50 I = 1, M
            IF( R( I ).EQ.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   50    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 60 I = 1, M
            R( I ) = ONE / MIN( MAX( R( I ), SMLNUM ), BIGNUM )
   60    CONTINUE
*
*        Compute ROWCND = min(R(I)) / max(R(I)).
*
         ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
*     Compute column scale factors.
*
      DO 70 J = 1, N
         C( J ) = ZERO
   70 CONTINUE
*
*     Find the maximum element in each column,
*     assuming the row scaling computed above.
```

```
*
      DO 90 J = 1, N
         DO 80 I = MAX( J-KU, 1 ), MIN( J+KL, M )
            C( J ) = MAX( C( J ), ABS( AB( KD+I-J, J ) )*R( I ) )
   80    CONTINUE
         IF( C( J ).GT.ZERO ) THEN
            C( J ) = RADIX**INT( LOG( C( J ) ) / LOGRDX )
         END IF
   90 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 100 J = 1, N
         RCMIN = MIN( RCMIN, C( J ) )
         RCMAX = MAX( RCMAX, C( J ) )
  100 CONTINUE
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 110 J = 1, N
            IF( C( J ).EQ.ZERO ) THEN
               INFO = M + J
               RETURN
            END IF
  110    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 120 J = 1, N
            C( J ) = ONE / MIN( MAX( C( J ), SMLNUM ), BIGNUM )
  120    CONTINUE
*
*        Compute COLCND = min(C(J)) / max(C(J)).
*
         COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
      RETURN
*
*     End of DGBEQUB
*
      END
```

44.4.4 lisp code

— dgbequb.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dgbequb (m n kl ku ab ldab r c rowcnd colcnd amax info)
    (declare (type (double-float) amax colcnd rowcnd)
             (type (array double-float (*)) c r ab)
             (type (f2cl-lib:integer4) info ldab ku kl n m))
    (f2cl-lib:with-multi-array-data
        ((ab double-float ab-%data% ab-%offset%)
         (r double-float r-%data% r-%offset%)
         (c double-float c-%data% c-%offset%))
      (prog ((bignum 0.0d0) (rcmax 0.0d0) (rcmin 0.0d0) (smlnum 0.0d0)
             (radix 0.0d0) (logrdx 0.0d0) (i 0) (j 0) (kd 0))
        (declare (type (double-float) bignum rcmax rcmin smlnum radix logrdx)
                 (type (f2cl-lib:integer4) i j kd))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< kl 0) (setf info -3)) ((< ku 0) (setf info -4))
              ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -6)))
        (cond
         ((/= info 0) (xerbla "DGBEQUB" (f2cl-lib:int-sub info))
          (go end_label)))
        (cond
         ((or (= m 0) (= n 0)) (setf rowcnd one) (setf colcnd one)
          (setf amax zero) (go end_label)))
        (setf smlnum (dlamch "S"))
        (setf bignum (/ one smlnum))
        (setf radix (dlamch "B"))
        (setf logrdx (f2cl-lib:flog radix))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%) zero)
           label10))
        (setf kd (f2cl-lib:int-add ku 1))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (f2cl-lib:fdo (i
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                            (the f2cl-lib:integer4 1))
                           (f2cl-lib:int-add i 1))
```

```
                    ((> i
                        (min
                         (the f2cl-lib:integer4 (f2cl-lib:int-add j kl))
                         (the f2cl-lib:integer4 m)))
                     nil)
      (tagbody
        (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                     (abs
                      (f2cl-lib:fref ab-%data%
                                     ((f2cl-lib:int-sub
                                       (f2cl-lib:int-add kd i) j)
                                      j)
                                     ((1 ldab) (1 *)) ab-%offset%))))
       label20))
   label30))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref r (i) ((1 *))) zero)
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
              (expt radix
                    (f2cl-lib:int
                     (/
                      (f2cl-lib:flog
                       (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%))
                      logrdx))))))
   label100000))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf rcmax
            (max rcmax
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
    (setf rcmin
            (min rcmin
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
   label40))
(setf amax rcmax)
(cond
 ((= rcmin zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref r (i) ((1 *))) zero) (setf info i)
        (go end_label)))
```

```
    label50)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
              (/ one
                 (min
                  (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                       smlnum)
                  bignum)))
    label60))
  (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) zero)
   label70))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                    (the f2cl-lib:integer4 1))
                   (f2cl-lib:int-add i 1))
                  ((> i
                      (min
                       (the f2cl-lib:integer4 (f2cl-lib:int-add j kl))
                       (the f2cl-lib:integer4 m)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                     (*
                      (abs
                       (f2cl-lib:fref ab-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add kd i) j)
                                       j)
                                      ((1 ldab) (1 *)) ab-%offset%))
                      (f2cl-lib:fref r-%data% (i) ((1 *))
                                     r-%offset%))))
       label80))
    (cond
     ((> (f2cl-lib:fref c (j) ((1 *))) zero)
      (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
              (expt radix
                    (f2cl-lib:int
```

```
                              (/
                               (f2cl-lib:flog
                                (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
                               logrdx))))))
           label90))
        (setf rcmin bignum)
        (setf rcmax zero)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf rcmin
                    (min rcmin
                         (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
            (setf rcmax
                    (max rcmax
                         (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
           label100))
        (cond
         ((= rcmin zero)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (cond
               ((= (f2cl-lib:fref c (j) ((1 *))) zero)
                (setf info (f2cl-lib:int-add m j)) (go end_label)))
             label110)))
         (t
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                      (/ one
                         (min
                          (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                               smlnum)
                          bignum)))
             label120))
          (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
        (go end_label)
       end_label
        (return
         (values nil nil nil nil nil nil nil nil rowcnd colcnd amax info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgbequb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
```

```
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (double-float) (double-float)
  (double-float)
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil nil nil nil nil nil
  fortran-to-lisp::rowcnd
  fortran-to-lisp::colcnd
  fortran-to-lisp::amax
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

44.5 dgbrfs

44.5.1 Axiom unit tests

— **dgbrfs.input** —

```
)set break resume
)sys rm -f dgbrfs.output
)spool dgbrfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

44.5.2 Axiom help page

— **dgbrfs.help** —

```
==================================================================
```

```
dgbrfs examples
===================================================================

===================================================================
Man Page Details
===================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgbrfs (TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB, IPIV,
           B, LDB, X, LDX, FERR, BERR, WORK, IWORK, INFO)
           DGBRFS

Function/Subroutine Documentation
   subroutine dgbrfs (character TRANS, integer N, integer KL, integer KU,
       integer NRHS, double precision, dimension( ldab, * ) AB, integer LDAB,
       double precision, dimension( ldafb, * ) AFB, integer LDAFB, integer,
       dimension( * ) IPIV, double precision, dimension( ldb, * ) B, integer
       LDB, double precision, dimension( ldx, * ) X, integer LDX, double
       precision, dimension( * ) FERR, double precision, dimension( * ) BERR,
       double precision, dimension( * ) WORK, integer, dimension( * ) IWORK,
       integer INFO)
       DGBRFS

Purpose:
   DGBRFS improves the computed solution to a system of linear
   equations when the coefficient matrix is banded, and provides
   error bounds and backward error estimates for the solution.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose = Transpose)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.
```

```
NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

AB
  AB is DOUBLE PRECISION array, dimension (LDAB,N)
  The original band matrix A, stored in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(n,j+kl).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is DOUBLE PRECISION array, dimension (LDAFB,N)
  Details of the LU factorization of the band matrix A, as
  computed by DGBTRF.  U is stored as an upper triangular band
  matrix with KL+KU superdiagonals in rows 1 to KL+KU+1, and
  the multipliers used during the factorization are stored in
  rows KL+KU+2 to 2*KL+KU+1.

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL*KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from DGBTRF; for 1<=i<=N, row i of the
  matrix was interchanged with row IPIV(i).

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by DGBTRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).
```

```
 FERR
   FERR is DOUBLE PRECISION array, dimension (NRHS)
   The estimated forward error bound for each solution vector
   X(j) (the j-th column of the solution matrix X).
   If XTRUE is the true solution corresponding to X(j), FERR(j)
   is an estimated upper bound for the magnitude of the largest
   element in (X(j) - XTRUE) divided by the magnitude of the
   largest element in X(j).  The estimate is as reliable as
   the estimate for RCOND, and is almost always a slight
   overestimate of the true error.

 BERR
   BERR is DOUBLE PRECISION array, dimension (NRHS)
   The componentwise relative backward error of each solution
   vector X(j) (i.e., the smallest relative change in
   any element of A or B that makes X(j) an exact solution).

 WORK
   WORK is DOUBLE PRECISION array, dimension (3*N)

 IWORK
   IWORK is INTEGER array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

 ITMAX is the maximum number of steps of iterative refinement.
```

———-

44.5.3 fortran code

— dgbrfs.f —

```
*  ===================================================================
      SUBROUTINE DGBRFS( TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB,
     $                   IPIV, B, LDB, X, LDX, FERR, BERR, WORK, IWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
```

```
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, KL, KU, LDAB, LDAFB, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      DOUBLE PRECISION   AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   BERR( * ), FERR( * ), WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D+0 )
      DOUBLE PRECISION   THREE
      PARAMETER          ( THREE = 3.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      CHARACTER          TRANST
      INTEGER            COUNT, I, J, K, KASE, KK, NZ
      DOUBLE PRECISION   EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN, XK
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DCOPY, DGBMV, DGBTRS, DLACN2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
```

```
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -7
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
         INFO = -9
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -12
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -14
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGBRFS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
      IF( NOTRAN ) THEN
         TRANST = 'T'
      ELSE
         TRANST = 'N'
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = MIN( KL+KU+2, N+1 )
      EPS = DLAMCH( 'Epsilon' )
      SAFMIN = DLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
```

```
*     Do for each right hand side
*
      DO 140 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - op(A) * X,
*        where op(A) = A, A**T, or A**H, depending on TRANS.
*
         CALL DCOPY( N, B( 1, J ), 1, WORK( N+1 ), 1 )
         CALL DGBMV( TRANS, N, N, KL, KU, -ONE, AB, LDAB, X( 1, J ), 1,
     $               ONE, WORK( N+1 ), 1 )
*
*        Compute componentwise relative backward error from formula
*
*        max(i) ( abs(R(i)) / ( abs(op(A))*abs(X) + abs(B) )(i) )
*
*        where abs(Z) is the componentwise absolute value of the matrix
*        or vector Z.  If the i-th component of the denominator is less
*        than SAFE2, then SAFE1 is added to the i-th components of the
*        numerator and denominator before dividing.
*
         DO 30 I = 1, N
            WORK( I ) = ABS( B( I, J ) )
   30    CONTINUE
*
*        Compute abs(op(A))*abs(X) + abs(B).
*
         IF( NOTRAN ) THEN
            DO 50 K = 1, N
               KK = KU + 1 - K
               XK = ABS( X( K, J ) )
               DO 40 I = MAX( 1, K-KU ), MIN( N, K+KL )
                  WORK( I ) = WORK( I ) + ABS( AB( KK+I, K ) )*XK
   40          CONTINUE
   50       CONTINUE
         ELSE
            DO 70 K = 1, N
               S = ZERO
               KK = KU + 1 - K
               DO 60 I = MAX( 1, K-KU ), MIN( N, K+KL )
                  S = S + ABS( AB( KK+I, K ) )*ABS( X( I, J ) )
   60          CONTINUE
               WORK( K ) = WORK( K ) + S
   70       CONTINUE
         END IF
```

```
         S = ZERO
         DO 80 I = 1, N
            IF( WORK( I ).GT.SAFE2 ) THEN
               S = MAX( S, ABS( WORK( N+I ) ) / WORK( I ) )
            ELSE
               S = MAX( S, ( ABS( WORK( N+I ) )+SAFE1 ) /
     $             ( WORK( I )+SAFE1 ) )
            END IF
   80    CONTINUE
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
*           1) The residual BERR(J) is larger than machine epsilon, and
*           2) BERR(J) decreased by at least a factor of 2 during the
*              last iteration, and
*           3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
*
*           Update solution and try again.
*
            CALL DGBTRS( TRANS, N, KL, KU, 1, AFB, LDAFB, IPIV,
     $                   WORK( N+1 ), N, INFO )
            CALL DAXPY( N, ONE, WORK( N+1 ), 1, X( 1, J ), 1 )
            LSTRES = BERR( J )
            COUNT = COUNT + 1
            GO TO 20
         END IF
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(op(A)))*
*           ( abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
*          inv(op(A)) is the inverse of op(A)
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(op(A))*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(op(A))*abs(X) + abs(B) is less than SAFE2.
*
*        Use DLACN2 to estimate the infinity-norm of the matrix
*           inv(op(A)) * diag(W),
```

```
*        where W = abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) )))
*
         DO 90 I = 1, N
            IF( WORK( I ).GT.SAFE2 ) THEN
               WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I )
            ELSE
               WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I ) + SAFE1
            END IF
   90    CONTINUE
*
         KASE = 0
  100    CONTINUE
         CALL DLACN2( N, WORK( 2*N+1 ), WORK( N+1 ), IWORK, FERR( J ),
     $                KASE, ISAVE )
         IF( KASE.NE.0 ) THEN
            IF( KASE.EQ.1 ) THEN
*
*              Multiply by diag(W)*inv(op(A)**T).
*
               CALL DGBTRS( TRANST, N, KL, KU, 1, AFB, LDAFB, IPIV,
     $                      WORK( N+1 ), N, INFO )
               DO 110 I = 1, N
                  WORK( N+I ) = WORK( N+I )*WORK( I )
  110          CONTINUE
            ELSE
*
*              Multiply by inv(op(A))*diag(W).
*
               DO 120 I = 1, N
                  WORK( N+I ) = WORK( N+I )*WORK( I )
  120          CONTINUE
               CALL DGBTRS( TRANS, N, KL, KU, 1, AFB, LDAFB, IPIV,
     $                      WORK( N+1 ), N, INFO )
            END IF
            GO TO 100
         END IF
*
*        Normalize error.
*
         LSTRES = ZERO
         DO 130 I = 1, N
            LSTRES = MAX( LSTRES, ABS( X( I, J ) ) )
  130    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  140 CONTINUE
*
      RETURN
*
```

```
*     End of DGBRFS
*
      END
```

———-

44.5.4 lisp code

— **dgbrfs.lisp** —

```
(let* ((itmax 5) (zero 0.0d0) (one 1.0d0) (two 2.0d0) (three 3.0d0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 3.0d0 3.0d0) three)
           (ignorable itmax zero one two three))
  (defun dgbrfs
         (trans n kl ku nrhs ab ldab afb ldafb ipiv b ldb$ x ldx ferr berr work
          iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array double-float (*)) work berr ferr x b afb ab)
             (type (f2cl-lib:integer4) info ldx ldb$ ldafb ldab nrhs ku kl n)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (ab double-float ab-%data% ab-%offset%)
         (afb double-float afb-%data% afb-%offset%)
         (b double-float b-%data% b-%offset%)
         (x double-float x-%data% x-%offset%)
         (ferr double-float ferr-%data% ferr-%offset%)
         (berr double-float berr-%data% berr-%offset%)
         (work double-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (eps 0.0d0) (lstres 0.0d0) (s 0.0d0) (safe1 0.0d0) (safe2 0.0d0)
             (safmin 0.0d0) (xk 0.0d0) (count$ 0) (i 0) (j 0) (k 0) (kase 0)
             (kk 0) (nz 0)
             (transt
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (notran nil))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (double-float) eps lstres s safe1 safe2 safmin xk)
                 (type (f2cl-lib:integer4) count$ i j k kase kk nz)
                 (type (string 1) transt)
                 (type f2cl-lib:logical notran))
        (setf info 0)
```

```
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< kl 0) (setf info -3))
 ((< ku 0) (setf info -4)) ((< nrhs 0) (setf info -5))
 ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -7))
 ((< ldafb (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
  (setf info -9))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -12))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -14)))
(cond
 ((/= info 0) (xerbla "DGBRFS" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%) zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) zero)
     label10))
  (go end_label)))
(cond (notran (f2cl-lib:f2cl-set-string transt "T" (string 1)))
      (t (f2cl-lib:f2cl-set-string transt "N" (string 1))))
(setf nz
        (min (the f2cl-lib:integer4 (f2cl-lib:int-add kl ku 2))
             (the f2cl-lib:integer4 (f2cl-lib:int-add n 1))))
(setf eps (dlamch "Epsilon"))
(setf safmin (dlamch "Safe minimum"))
(setf safe1 (* nz safmin))
```

```
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy n
         (f2cl-lib:array-slice b-%data% double-float (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1
         (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                               ((1 *)) work-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (dgbmv trans n n kl ku (- one) ab ldab
         (f2cl-lib:array-slice x-%data% double-float (1 j)
                               ((1 ldx) (1 *)) x-%offset%)
         1 one
         (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                               ((1 *)) work-%offset%)
         1)
      (declare (ignore var-5 var-6 var-8 var-9 var-11 var-12))
      (when var-0 (setf trans var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf kl var-3))
      (when var-4 (setf ku var-4))
      (when var-7 (setf ldab var-7))
      (when var-10 (setf one var-10)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (abs
                 (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)))
       label30))
    (cond
     (notran
      (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                    ((> k n) nil)
        (tagbody
          (setf kk (f2cl-lib:int-sub (f2cl-lib:int-add ku 1) k))
          (setf xk
```

```
                (abs
                 (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                                x-%offset%)))
        (f2cl-lib:fdo (i
                       (max (the f2cl-lib:integer4 1)
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add k
                                                   (f2cl-lib:int-sub
                                                    ku))))
                       (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the f2cl-lib:integer4 n)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add k kl))))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)
                  (+
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                   (*
                    (abs
                     (f2cl-lib:fref ab-%data%
                                    ((f2cl-lib:int-add kk i) k)
                                    ((1 ldab) (1 *)) ab-%offset%))
                    xk)))
           label40))
      label50)))
  (t
   (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                 ((> k n) nil)
     (tagbody
       (setf s zero)
       (setf kk (f2cl-lib:int-sub (f2cl-lib:int-add ku 1) k))
       (f2cl-lib:fdo (i
                      (max (the f2cl-lib:integer4 1)
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add k
                                                  (f2cl-lib:int-sub
                                                   ku))))
                      (f2cl-lib:int-add i 1))
                     ((> i
                         (min (the f2cl-lib:integer4 n)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add k kl))))
                      nil)
         (tagbody
           (setf s
                 (+ s
```

```
                        (*
                         (abs
                          (f2cl-lib:fref ab-%data%
                                         ((f2cl-lib:int-add kk i) k)
                                         ((1 ldab) (1 *))
                                         ab-%offset%))
                         (abs
                          (f2cl-lib:fref x-%data% (i j)
                                         ((1 ldx) (1 *))
                                         x-%offset%)))))
            label60))
        (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                (+
                 (f2cl-lib:fref work-%data% (k) ((1 *))
                                work-%offset%)
                 s))
      label70))))
(setf s zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
      (setf s
              (max s
                   (/
                    (abs
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add n i))
                                    ((1 *)) work-%offset%))
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)))))
     (t
      (setf s
              (max s
                   (/
                    (+
                     (abs
                      (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-add n i))
                                     ((1 *)) work-%offset%))
                     safe1)
                    (+
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)
                     safe1))))))
   label80))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
```

```
        (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
        (<= count$ itmax))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dgbtrs trans n kl ku 1 afb ldafb ipiv
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 *)) work-%offset%)
       n info)
    (declare (ignore var-4 var-5 var-7 var-8))
    (when var-0 (setf trans var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf kl var-2))
    (when var-3 (setf ku var-3))
    (when var-6 (setf ldafb var-6))
    (when var-9 (setf n var-9))
    (when var-10 (setf info var-10)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (daxpy n one
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 *)) work-%offset%)
       1
       (f2cl-lib:array-slice x-%data% double-float (1 j)
                             ((1 ldx) (1 *)) x-%offset%)
       1)
    (declare (ignore var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))
    (when var-1 (setf one var-1)))
  (setf lstres
          (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
  (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (+
               (abs
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%))
               (* nz eps
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)))))
     (t
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (+
               (abs
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%))
```

```
                  (* nz eps
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%))
                  safe1))))
    label90))
 (setf kase 0)
label100
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlacn2 n
      (f2cl-lib:array-slice work-%data% double-float
                            ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                            ((1 *)) work-%offset%)
      iwork
      (f2cl-lib:array-slice ferr-%data% double-float (j) ((1 *))
                            ferr-%offset%)
      kase isave)
   (declare (ignore var-1 var-2 var-3 var-4 var-6))
   (when var-0 (setf n var-0))
   (when var-5 (setf kase var-5)))
 (cond
  ((/= kase 0)
   (cond
    ((= kase 1)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
          var-9 var-10)
         (dgbtrs transt n kl ku 1 afb ldafb ipiv
          (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                                ((1 *)) work-%offset%)
          n info)
       (declare (ignore var-4 var-5 var-7 var-8))
       (when var-0 (setf transt var-0))
       (when var-1 (setf n var-1))
       (when var-2 (setf kl var-2))
       (when var-3 (setf ku var-3))
       (when var-6 (setf ldafb var-6))
       (when var-9 (setf n var-9))
       (when var-10 (setf info var-10)))
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                              ((1 *)) work-%offset%)
                 (*
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add n i)) ((1 *))
                                 work-%offset%)
                  (f2cl-lib:fref work-%data% (i) ((1 *))
```

```
                                        work-%offset%)))
                 label110)))
              (t
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i n) nil)
                 (tagbody
                   (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                        ((1 *)) work-%offset%)
                         (*
                          (f2cl-lib:fref work-%data%
                                         ((f2cl-lib:int-add n i)) ((1 *))
                                         work-%offset%)
                          (f2cl-lib:fref work-%data% (i) ((1 *))
                                         work-%offset%)))
                 label120))
               (multiple-value-bind
                   (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                    var-9 var-10)
                   (dgbtrs trans n kl ku 1 afb ldafb ipiv
                    (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                                          ((1 *)) work-%offset%)
                    n info)
                 (declare (ignore var-4 var-5 var-7 var-8))
                 (when var-0 (setf trans var-0))
                 (when var-1 (setf n var-1))
                 (when var-2 (setf kl var-2))
                 (when var-3 (setf ku var-3))
                 (when var-6 (setf ldafb var-6))
                 (when var-9 (setf n var-9))
                 (when var-10 (setf info var-10)))))
            (go label100)))
          (setf lstres zero)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf lstres
                      (max lstres
                           (abs
                            (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                           x-%offset%))))
             label130))
          (if (/= lstres zero)
              (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                      (/
                       (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                       lstres)))
         label140))
       (go end_label)
      end_label
       (return
```

```
        (values trans n kl ku nil nil ldab nil ldafb nil nil nil nil nil nil
                nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgbrfs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::n
                                              fortran-to-lisp::kl
                                              fortran-to-lisp::ku nil nil
                                              fortran-to-lisp::ldab nil
                                              fortran-to-lisp::ldafb nil nil
                                              nil nil nil nil nil nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

44.6 dgbrfsx

44.6.1 Axiom unit tests

— dgbrfsx.input —

```
)set break resume
)sys rm -f dgbrfsx.output
)spool dgbrfsx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

44.6.2 Axiom help page [Demm05]

— dgbrfsx.help —

```
====================================================================
dgbrfsx examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgbrfsx (TRANS, EQUED, N, KL, KU, NRHS, AB, LDAB, AFB,
           LDAFB, IPIV, R, C, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS,
           ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, IWORK, INFO)
           DGBRFSX

Function/Subroutine Documentation
   subroutine dgbrfsx (character TRANS, character EQUED, integer N, integer
       KL, integer KU, integer NRHS, double precision, dimension( ldab, * )
       AB, integer LDAB, double precision, dimension( ldafb, * ) AFB, integer
       LDAFB, integer, dimension( * ) IPIV, double precision, dimension( * )
       R, double precision, dimension( * ) C, double precision, dimension(
       ldb, * ) B, integer LDB, double precision, dimension( ldx , * ) X,
       integer LDX, double precision RCOND, double precision, dimension( * )
       BERR, integer N_ERR_BNDS, double precision, dimension( nrhs, * )
```

```
      ERR_BNDS_NORM, double precision, dimension( nrhs, * ) ERR_BNDS_COMP,
      integer NPARAMS, double precision, dimension( * ) PARAMS, double
      precision, dimension( * ) WORK, integer, dimension( * ) IWORK, integer
      INFO)
      DGBRFSX

Purpose:
   DGBRFSX improves the computed solution to a system of linear
   equations and provides error bounds and backward error estimates
   for the solution.  In addition to normwise error bound, the code
   provides maximum componentwise error bound if possible.  See
   comments for ERR_BNDS_NORM and ERR_BNDS_COMP for details of the
   error bounds.

   The original system of linear equations may have been equilibrated
   before calling this routine, as described by arguments EQUED, R
   and C below. In this case, the solution and error bounds returned
   are for the original unequilibrated system.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
                the PARAMS argument.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose = Transpose)

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done to A
   before calling this routine. This is needed to compute
   the solution and error bounds correctly.
   = 'N':  No equilibration
   = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
   = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
   = 'B':  Both row and column equilibration, i.e., A has been
           replaced by diag(R) * A * diag(C).
           The right hand side B has been changed accordingly.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.
```

```
KL
  KL is INTEGER
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

AB
  AB is DOUBLE PRECISION array, dimension (LDAB,N)
  The original band matrix A, stored in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(n,j+kl).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is DOUBLE PRECISION array, dimension (LDAFB,N)
  Details of the LU factorization of the band matrix A, as
  computed by DGBTRF.  U is stored as an upper triangular band
  matrix with KL+KU superdiagonals in rows 1 to KL+KU+1, and
  the multipliers used during the factorization are stored in
  rows KL+KU+2 to 2*KL+KU+1.

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL*KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from DGETRF; for 1<=i<=N, row i of the
  matrix was interchanged with row IPIV(i).

R
  R is DOUBLE PRECISION array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
  EQUED = 'R' or 'B', each element of R must be positive.
  If R is output, each element of R is a power of the radix.
  If R is input, each element of R should be a power of the radix
```

```
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

C
  C is DOUBLE PRECISION array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.
  If C is output, each element of C is a power of the radix.
  If C is input, each element of C should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by DGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  Componentwise relative backward error.  This is the
```

```
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
            max_j (abs(XTRUE(j,i) - X(j,i)))
             ------------------------------
                   max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * dlamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * dlamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.
```

```
ERR_BNDS_COMP
  ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                 abs(XTRUE(j,i) - X(j,i))
            max_j ----------------------
                        abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * dlamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * dlamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.
```

```
PARAMS
  PARAMS is DOUBLE PRECISION array, dimension (NPARAMS)
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0D+0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
  (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is DOUBLE PRECISION array, dimension (4*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is guaranteed.
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
     has been completed, but the factor U is exactly singular, so
     the solution and error bounds could not be computed. RCOND = 0
     is returned.
  = N+J: The solution corresponding to the Jth right-hand side is
     not guaranteed. The solutions corresponding to other right-
     hand sides K with K > J may not be guaranteed as well, but
     only the first such right-hand side is reported. If a small
    componentwise error is not requested (PARAMS(3) = 0.0) then
```

```
the Jth right-hand side is the first with a normwise error
bound that is not guaranteed (the smallest J such
that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
the Jth right-hand side is the first with either a normwise or
componentwise error bound that is not guaranteed (the smallest
J such that either ERR_BNDS_NORM(J,1) = 0.0 or
ERR_BNDS_COMP(J,1) = 0.0). See the definition of
ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

———-

44.6.3 fortran code

— dgbrfsx.f —

```
*  ====================================================================
      SUBROUTINE DGBRFSX( TRANS, EQUED, N, KL, KU, NRHS, AB, LDAB, AFB,
     $                    LDAFB, IPIV, R, C, B, LDB, X, LDX, RCOND,
     $                    BERR, N_ERR_BNDS, ERR_BNDS_NORM,
     $                    ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, IWORK,
     $                    INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS, EQUED
      INTEGER            INFO, LDAB, LDAFB, LDB, LDX, N, KL, KU, NRHS,
     $                   NPARAMS, N_ERR_BNDS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      DOUBLE PRECISION   AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   X( LDX , * ),WORK( * )
      DOUBLE PRECISION   R( * ), C( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
```

```
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   ITREF_DEFAULT, ITHRESH_DEFAULT
      DOUBLE PRECISION   COMPONENTWISE_DEFAULT, RTHRESH_DEFAULT
      DOUBLE PRECISION   DZTHRESH_DEFAULT
      PARAMETER          ( ITREF_DEFAULT = 1.0D+0 )
      PARAMETER          ( ITHRESH_DEFAULT = 10.0D+0 )
      PARAMETER          ( COMPONENTWISE_DEFAULT = 1.0D+0 )
      PARAMETER          ( RTHRESH_DEFAULT = 0.5D+0 )
      PARAMETER          ( DZTHRESH_DEFAULT = 0.25D+0 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. Local Scalars ..
      CHARACTER(1)       NORM
      LOGICAL            ROWEQU, COLEQU, NOTRAN
      INTEGER            J, TRANS_TYPE, PREC_TYPE, REF_TYPE
      INTEGER            N_NORMS
      DOUBLE PRECISION   ANORM, RCOND_TMP
      DOUBLE PRECISION   ILLRCOND_THRESH, ERR_LBND, CWISE_WRONG
      LOGICAL            IGNORE_CWISE
      INTEGER            ITHRESH
      DOUBLE PRECISION   RTHRESH, UNSTABLE_THRESH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, DGBCON
      EXTERNAL           DLA_GBRFSX_EXTENDED
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, BLAS_FPINFO_X, ILATRANS, ILAPREC
      EXTERNAL           DLAMCH, DLANGB, DLA_GBRCOND
      DOUBLE PRECISION   DLAMCH, DLANGB, DLA_GBRCOND
      LOGICAL            LSAME
      INTEGER            BLAS_FPINFO_X
      INTEGER            ILATRANS, ILAPREC
*     ..
*     .. Executable Statements ..
*
*     Check the input parameters.
*
      INFO = 0
```

```
      TRANS_TYPE = ILATRANS( TRANS )
      REF_TYPE = INT( ITREF_DEFAULT )
      IF ( NPARAMS .GE. LA_LINRX_ITREF_I ) THEN
         IF ( PARAMS( LA_LINRX_ITREF_I ) .LT. 0.0D+0 ) THEN
            PARAMS( LA_LINRX_ITREF_I ) = ITREF_DEFAULT
         ELSE
            REF_TYPE = PARAMS( LA_LINRX_ITREF_I )
         END IF
      END IF
*
*     Set default parameters.
*
      ILLRCOND_THRESH = DBLE( N ) * DLAMCH( 'Epsilon' )
      ITHRESH = INT( ITHRESH_DEFAULT )
      RTHRESH = RTHRESH_DEFAULT
      UNSTABLE_THRESH = DZTHRESH_DEFAULT
      IGNORE_CWISE = COMPONENTWISE_DEFAULT .EQ. 0.0D+0
*
      IF ( NPARAMS.GE.LA_LINRX_ITHRESH_I ) THEN
         IF ( PARAMS( LA_LINRX_ITHRESH_I ).LT.0.0D+0 ) THEN
            PARAMS( LA_LINRX_ITHRESH_I ) = ITHRESH
         ELSE
            ITHRESH = INT( PARAMS( LA_LINRX_ITHRESH_I ) )
         END IF
      END IF
      IF ( NPARAMS.GE.LA_LINRX_CWISE_I ) THEN
         IF ( PARAMS( LA_LINRX_CWISE_I ).LT.0.0D+0 ) THEN
            IF ( IGNORE_CWISE ) THEN
               PARAMS( LA_LINRX_CWISE_I ) = 0.0D+0
            ELSE
               PARAMS( LA_LINRX_CWISE_I ) = 1.0D+0
            END IF
         ELSE
            IGNORE_CWISE = PARAMS( LA_LINRX_CWISE_I ) .EQ. 0.0D+0
         END IF
      END IF
      IF ( REF_TYPE .EQ. 0 .OR. N_ERR_BNDS .EQ. 0 ) THEN
         N_NORMS = 0
      ELSE IF ( IGNORE_CWISE ) THEN
         N_NORMS = 1
      ELSE
         N_NORMS = 2
      END IF
*
      NOTRAN = LSAME( TRANS, 'N' )
      ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
      COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
*
*     Test input parameters.
*
```

```
      IF( TRANS_TYPE.EQ.-1 ) THEN
        INFO = -1
      ELSE IF( .NOT.ROWEQU .AND. .NOT.COLEQU .AND.
     $         .NOT.LSAME( EQUED, 'N' ) ) THEN
        INFO = -2
      ELSE IF( N.LT.0 ) THEN
        INFO = -3
      ELSE IF( KL.LT.0 ) THEN
        INFO = -4
      ELSE IF( KU.LT.0 ) THEN
        INFO = -5
      ELSE IF( NRHS.LT.0 ) THEN
        INFO = -6
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
        INFO = -8
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
        INFO = -10
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
        INFO = -13
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
        INFO = -15
      END IF
      IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'DGBRFSX', -INFO )
        RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         RCOND = 1.0D+0
         DO J = 1, NRHS
            BERR( J ) = 0.0D+0
            IF ( N_ERR_BNDS .GE. 1 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
            IF ( N_ERR_BNDS .GE. 2 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 0.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 0.0D+0
            END IF
            IF ( N_ERR_BNDS .GE. 3 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 1.0D+0
            END IF
         END DO
         RETURN
      END IF
*
*     Default to failure.
```

```
*
      RCOND = 0.0D+0
      DO J = 1, NRHS
         BERR( J ) = 1.0D+0
         IF ( N_ERR_BNDS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
         END IF
         IF ( N_ERR_BNDS .GE. 2 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
         END IF
         IF ( N_ERR_BNDS .GE. 3 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 0.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 0.0D+0
         END IF
      END DO
*
*     Compute the norm of A and the reciprocal of the condition
*     number of A.
*
      IF( NOTRAN ) THEN
         NORM = 'I'
      ELSE
         NORM = '1'
      END IF
      ANORM = DLANGB( NORM, N, KL, KU, AB, LDAB, WORK )
      CALL DGBCON( NORM, N, KL, KU, AFB, LDAFB, IPIV, ANORM, RCOND,
     $     WORK, IWORK, INFO )
*
*     Perform refinement on each right-hand side
*
      IF (REF_TYPE .NE. 0) THEN

         PREC_TYPE = ILAPREC( 'E' )

         IF ( NOTRAN ) THEN
            CALL DLA_GBRFSX_EXTENDED( PREC_TYPE, TRANS_TYPE,  N, KL, KU,
     $           NRHS, AB, LDAB, AFB, LDAFB, IPIV, COLEQU, C, B,
     $           LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM,
     $           ERR_BNDS_COMP, WORK( N+1 ), WORK( 1 ), WORK( 2*N+1 ),
     $           WORK( 1 ), RCOND, ITHRESH, RTHRESH, UNSTABLE_THRESH,
     $           IGNORE_CWISE, INFO )
         ELSE
            CALL DLA_GBRFSX_EXTENDED( PREC_TYPE, TRANS_TYPE,  N, KL, KU,
     $           NRHS, AB, LDAB, AFB, LDAFB, IPIV, ROWEQU, R, B,
     $           LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM,
     $           ERR_BNDS_COMP, WORK( N+1 ), WORK( 1 ), WORK( 2*N+1 ),
     $           WORK( 1 ), RCOND, ITHRESH, RTHRESH, UNSTABLE_THRESH,
     $           IGNORE_CWISE, INFO )
```

```
         END IF
      END IF

      ERR_LBND = MAX( 10.0D+0, SQRT( DBLE( N ) ) ) * DLAMCH( 'Epsilon' )
      IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 1 ) THEN
*
*     Compute scaled normwise condition number cond(A*C).
*
         IF ( COLEQU .AND. NOTRAN ) THEN
            RCOND_TMP = DLA_GBRCOND( TRANS, N, KL, KU, AB, LDAB, AFB,
     $           LDAFB, IPIV, -1, C, INFO, WORK, IWORK )
         ELSE IF ( ROWEQU .AND. .NOT. NOTRAN ) THEN
            RCOND_TMP = DLA_GBRCOND( TRANS, N, KL, KU, AB, LDAB, AFB,
     $           LDAFB, IPIV, -1, R, INFO, WORK, IWORK )
         ELSE
            RCOND_TMP = DLA_GBRCOND( TRANS, N, KL, KU, AB, LDAB, AFB,
     $           LDAFB, IPIV, 0, R, INFO, WORK, IWORK )
         END IF
         DO J = 1, NRHS
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .GT. 1.0D+0 )
     $           ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 0.0D+0
               IF ( INFO .LE. N ) INFO = N + J
            ELSE IF ( ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .LT. ERR_LBND )
     $     THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF

         END DO
      END IF

      IF (N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 2) THEN
*
*     Compute componentwise condition number cond(A*diag(Y(:,J))) for
```

```
*     each right-hand side using the current solution as an estimate of
*     the true solution.  If the componentwise error estimate is too
*     large, then the solution is a lousy estimate of truth and the
*     estimated RCOND may be too optimistic.  To avoid misleading users,
*     the inverse condition number is set to 0.0 when the estimated
*     cwise error is at least CWISE_WRONG.
*
         CWISE_WRONG = SQRT( DLAMCH( 'Epsilon' ) )
         DO J = 1, NRHS
            IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .LT. CWISE_WRONG )
     $     THEN
               RCOND_TMP = DLA_GBRCOND( TRANS, N, KL, KU, AB, LDAB, AFB,
     $              LDAFB, IPIV, 1, X( 1, J ), INFO, WORK, IWORK )
            ELSE
               RCOND_TMP = 0.0D+0
            END IF
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .GT. 1.0D+0 )
     $           ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 0.0D+0
               IF ( PARAMS( LA_LINRX_CWISE_I ) .EQ. 1.0D+0
     $              .AND. INFO.LT.N + J ) INFO = N + J
            ELSE IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I )
     $              .LT. ERR_LBND ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF

         END DO
      END IF
*
      RETURN
*
*     End of DGBRFSX
*
      END
```

———-

44.6.4 lisp code

— **dgbrfsx.lisp** —

———-

44.7 dgbtf2

44.7.1 Axiom unit tests

— **dgbtf2.input** —

```
)set break resume
)sys rm -f dgbtf2.output
)spool dgbtf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

44.7.2 Axiom help page

— **dgbtf2.help** —

```
======================================================================
dgbtf2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
```

```
      subroutine dgbtf2 (M, N, KL, KU, AB, LDAB, IPIV, INFO)
          DGBTF2 computes the LU factorization of a general band matrix using
          the unblocked version of the algorithm.

Function/Subroutine Documentation
   subroutine dgbtf2 (integer M, integer N, integer KL, integer KU, double
       precision, dimension( ldab, * ) AB, integer LDAB, integer, dimension( *
       ) IPIV, integer INFO)
       DGBTF2 computes the LU factorization of a general band matrix using the
       unblocked version of the algorithm.

Purpose:
   DGBTF2 computes an LU factorization of a real m-by-n band matrix A
   using partial pivoting with row interchanges.

   This is the unblocked version of the algorithm, calling Level 2 BLAS.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 AB
   AB is DOUBLE PRECISION array, dimension (LDAB,N)
   On entry, the matrix A in band storage, in rows KL+1 to
   2*KL+KU+1; rows 1 to KL of the array need not be set.
   The j-th column of A is stored in the j-th column of the
   array AB as follows:
   AB(kl+ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl)

   On exit, details of the factorization: U is stored as an
   upper triangular band matrix with KL+KU superdiagonals in
   rows 1 to KL+KU+1, and the multipliers used during the
   factorization are stored in rows KL+KU+2 to 2*KL+KU+1.
   See below for further details.

 LDAB
```

```
   LDAB is INTEGER
   The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.

 IPIV
   IPIV is INTEGER array, dimension (min(M,N))
   The pivot indices; for 1 <= i <= min(M,N), row i of the
   matrix was interchanged with row IPIV(i).

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value
   > 0: if INFO = +i, U(i,i) is exactly zero. The factorization
        has been completed, but the factor U is exactly
        singular, and division by zero will occur if it is used
        to solve a system of equations.

Further Details:

   The band storage scheme is illustrated by the following example, when
   M = N = 6, KL = 2, KU = 1:

        On entry:                       On exit:

            *    *    *    +    +    +       *    *    *   u14  u25  u36
            *    *    +    +    +    +       *    *   u13  u24  u35  u46
            *   a12  a23  a34  a45  a56      *   u12  u23  u34  u45  u56
           a11  a22  a33  a44  a55  a66     u11  u22  u33  u44  u55  u66
           a21  a32  a43  a54  a65   *      m21  m32  m43  m54  m65   *
           a31  a42  a53  a64   *    *      m31  m42  m53  m64   *    *

   Array elements marked * are not used by the routine; elements marked
   + need not be set on entry, but are required by the routine to store
   elements of U, because of fill-in resulting from the row
   interchanges.
```

———-

44.7.3 fortran code

— dgbtf2.f —

```
*  =====================================================================
      SUBROUTINE DGBTF2( M, N, KL, KU, AB, LDAB, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
```

```
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, KL, KU, LDAB, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   AB( LDAB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, JP, JU, KM, KV
*     ..
*     .. External Functions ..
      INTEGER            IDAMAX
      EXTERNAL           IDAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGER, DSCAL, DSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     KV is the number of superdiagonals in the factor U, allowing for
*     fill-in.
*
      KV = KU + KL
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KV+1 ) THEN
         INFO = -6
```

```
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGBTF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Gaussian elimination with partial pivoting
*
*     Set fill-in elements in columns KU+2 to KV to zero.
*
      DO 20 J = KU + 2, MIN( KV, N )
         DO 10 I = KV - J + 2, KL
            AB( I, J ) = ZERO
   10    CONTINUE
   20 CONTINUE
*
*     JU is the index of the last column affected by the current stage
*     of the factorization.
*
      JU = 1
*
      DO 40 J = 1, MIN( M, N )
*
*        Set fill-in elements in column J+KV to zero.
*
         IF( J+KV.LE.N ) THEN
            DO 30 I = 1, KL
               AB( I, J+KV ) = ZERO
   30       CONTINUE
         END IF
*
*        Find pivot and test for singularity. KM is the number of
*        subdiagonal elements in the current column.
*
         KM = MIN( KL, M-J )
         JP = IDAMAX( KM+1, AB( KV+1, J ), 1 )
         IPIV( J ) = JP + J - 1
         IF( AB( KV+JP, J ).NE.ZERO ) THEN
            JU = MAX( JU, MIN( J+KU+JP-1, N ) )
*
*           Apply interchange to columns J to JU.
*
            IF( JP.NE.1 )
     $         CALL DSWAP( JU-J+1, AB( KV+JP, J ), LDAB-1,
     $                     AB( KV+1, J ), LDAB-1 )
```

```
*
            IF( KM.GT.0 ) THEN
*
*              Compute multipliers.
*
               CALL DSCAL( KM, ONE / AB( KV+1, J ), AB( KV+2, J ), 1 )
*
*              Update trailing submatrix within the band.
*
               IF( JU.GT.J )
     $            CALL DGER( KM, JU-J, -ONE, AB( KV+2, J ), 1,
     $                       AB( KV, J+1 ), LDAB-1, AB( KV+1, J+1 ),
     $                       LDAB-1 )
            END IF
         ELSE
*
*           If pivot is zero, set INFO to the index of the pivot
*           unless a zero pivot has already been found.
*
            IF( INFO.EQ.0 )
     $         INFO = J
         END IF
   40 CONTINUE
      RETURN
*
*     End of DGBTF2
*
      END
```

———-

44.7.4 lisp code

— **dgbtf2.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dgbtf2 (m n kl ku ab ldab ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array double-float (*)) ab)
             (type (f2cl-lib:integer4) info ldab ku kl n m))
    (f2cl-lib:with-multi-array-data
        ((ab double-float ab-%data% ab-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((i 0) (j 0) (jp 0) (ju 0) (km 0) (kv 0))
        (declare (type (f2cl-lib:integer4) i j jp ju km kv))
```

```
(setf kv (f2cl-lib:int-add ku kl))
(setf info 0)
(cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
      ((< kl 0) (setf info -3)) ((< ku 0) (setf info -4))
      ((< ldab (f2cl-lib:int-add kl kv 1)) (setf info -6)))
(cond
 ((/= info 0) (xerbla "DGBTF2" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= m 0) (= n 0))
    (go end_label))
(f2cl-lib:fdo (j (f2cl-lib:int-add ku 2) (f2cl-lib:int-add j 1))
              ((> j
                  (min (the f2cl-lib:integer4 kv)
                       (the f2cl-lib:integer4 n)))
               nil)
  (tagbody
    (f2cl-lib:fdo (i (f2cl-lib:int-add kv (f2cl-lib:int-sub j) 2)
                   (f2cl-lib:int-add i 1))
                  ((> i kl) nil)
      (tagbody
        (setf (f2cl-lib:fref ab-%data% (i j) ((1 ldab) (1 *))
                             ab-%offset%)
              zero)
       label10))
   label20))
(setf ju 1)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j
                  (min (the f2cl-lib:integer4 m)
                       (the f2cl-lib:integer4 n)))
               nil)
  (tagbody
    (cond
     ((<= (f2cl-lib:int-add j kv) n)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i kl) nil)
        (tagbody
          (setf (f2cl-lib:fref ab-%data% (i (f2cl-lib:int-add j kv))
                               ((1 ldab) (1 *)) ab-%offset%)
                zero)
         label30))))
    (setf km
            (min (the f2cl-lib:integer4 kl)
                 (the f2cl-lib:integer4 (f2cl-lib:int-sub m j))))
    (setf jp
            (idamax (f2cl-lib:int-add km 1)
             (f2cl-lib:array-slice ab-%data% double-float ((+ kv 1) j)
                                   ((1 ldab) (1 *)) ab-%offset%)
             1))
    (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
```

```
        (f2cl-lib:int-sub (f2cl-lib:int-add jp j) 1))
(cond
 ((/=
   (f2cl-lib:fref ab ((f2cl-lib:int-add kv jp) j) ((1 ldab) (1 *)))
   zero)
  (setf ju
          (max (the f2cl-lib:integer4 ju)
               (the f2cl-lib:integer4
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-sub
                           (f2cl-lib:int-add j ku jp) 1))
                     (the f2cl-lib:integer4 n)))))
  (if (/= jp 1)
      (dswap (f2cl-lib:int-add (f2cl-lib:int-sub ju j) 1)
       (f2cl-lib:array-slice ab-%data% double-float ((+ kv jp) j)
                             ((1 ldab) (1 *)) ab-%offset%)
       (f2cl-lib:int-sub ldab 1)
       (f2cl-lib:array-slice ab-%data% double-float ((+ kv 1) j)
                             ((1 ldab) (1 *)) ab-%offset%)
       (f2cl-lib:int-sub ldab 1)))
  (cond
   ((> km 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal km
         (/ one
            (f2cl-lib:fref ab-%data% ((f2cl-lib:int-add kv 1) j)
                           ((1 ldab) (1 *)) ab-%offset%))
         (f2cl-lib:array-slice ab-%data% double-float ((+ kv 2) j)
                               ((1 ldab) (1 *)) ab-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf km var-0)))
    (if (> ju j)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (dger km (f2cl-lib:int-sub ju j) (- one)
             (f2cl-lib:array-slice ab-%data% double-float
                                   ((+ kv 2) j) ((1 ldab) (1 *))
                                   ab-%offset%)
             1
             (f2cl-lib:array-slice ab-%data% double-float
                                   (kv (f2cl-lib:int-add j 1))
                                   ((1 ldab) (1 *)) ab-%offset%)
             (f2cl-lib:int-sub ldab 1)
             (f2cl-lib:array-slice ab-%data% double-float
                                   ((+ kv 1)
                                    (f2cl-lib:int-add j 1))
                                   ((1 ldab) (1 *)) ab-%offset%)
             (f2cl-lib:int-sub ldab 1))
```

```
                      (declare
                       (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7
                        var-8))
                      (when var-0 (setf km var-0)))))))
             (t
              (if (= info 0)
                  (setf info j))))
           label40))
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgbtf2
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::integer4
                                              (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(nil nil nil nil nil nil nil
                                             fortran-to-lisp::info)
                                           :calls 'nil)))
```

———-

44.8 dgbtrf

44.8.1 Axiom unit tests

— dgbtrf.input —

```
)set break resume
)sys rm -f dgbtrf.output
)spool dgbtrf.output
)set message test on
)set message auto off
```

```
)clear all

)spool
)lisp (bye)
```

———-

44.8.2 Axiom help page

— **dgbtrf.help** —

```
==================================================================
dgbtrf examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgbtrf (M, N, KL, KU, AB, LDAB, IPIV, INFO)
           DGBTRF

Function/Subroutine Documentation
   subroutine dgbtrf (integer M, integer N, integer KL, integer KU, double
       precision, dimension( ldab, * ) AB, integer LDAB, integer, dimension( *
       ) IPIV, integer INFO)
       DGBTRF

Purpose:
   DGBTRF computes an LU factorization of a real m-by-n band matrix A
   using partial pivoting with row interchanges.

   This is the blocked version of the algorithm, calling Level 3 BLAS.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 KL
   KL is INTEGER
```

```
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

AB
  AB is DOUBLE PRECISION array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows KL+1 to
  2*KL+KU+1; rows 1 to KL of the array need not be set.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(kl+ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl)

  On exit, details of the factorization: U is stored as an
  upper triangular band matrix with KL+KU superdiagonals in
  rows 1 to KL+KU+1, and the multipliers used during the
  factorization are stored in rows KL+KU+2 to 2*KL+KU+1.
  See below for further details.

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (min(M,N))
  The pivot indices; for 1 <= i <= min(M,N), row i of the
  matrix was interchanged with row IPIV(i).

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = +i, U(i,i) is exactly zero. The factorization
       has been completed, but the factor U is exactly
       singular, and division by zero will occur if it is used
       to solve a system of equations.

Further Details:

  The band storage scheme is illustrated by the following example, when
  M = N = 6, KL = 2, KU = 1:

       On entry:                       On exit:

           *    *    *    +    +    +       *    *    *   u14  u25  u36
           *    *    +    +    +    +       *    *   u13  u24  u35  u46
           *   a12  a23  a34  a45  a56       *   u12  u23  u34  u45  u56
          a11  a22  a33  a44  a55  a66     u11  u22  u33  u44  u55  u66
          a21  a32  a43  a54  a65   *      m21  m32  m43  m54  m65   *
```

```
          a31  a42  a53  a64   *    *      m31  m42  m53  m64   *    *

   Array elements marked * are not used by the routine; elements marked
   + need not be set on entry, but are required by the routine to store
   elements of U because of fill-in resulting from the row interchanges.
```

———-

44.8.3 fortran code

— dgbtrf.f —

```
*  =====================================================================
      SUBROUTINE DGBTRF( M, N, KL, KU, AB, LDAB, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, KL, KU, LDAB, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   AB( LDAB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
      INTEGER            NBMAX, LDWORK
      PARAMETER          ( NBMAX = 64, LDWORK = NBMAX+1 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, I2, I3, II, IP, J, J2, J3, JB, JJ, JM, JP,
     $                   JU, K2, KM, KV, NB, NW
      DOUBLE PRECISION   TEMP
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   WORK13( LDWORK, NBMAX ),
     $                   WORK31( LDWORK, NBMAX )
*     ..
*     .. External Functions ..
      INTEGER            IDAMAX, ILAENV
```

```
      EXTERNAL           IDAMAX, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DGBTF2, DGEMM, DGER, DLASWP, DSCAL,
     $                   DSWAP, DTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     KV is the number of superdiagonals in the factor U, allowing for
*     fill-in
*
      KV = KU + KL
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KV+1 ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGBTRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Determine the block size for this environment
*
      NB = ILAENV( 1, 'DGBTRF', ' ', M, N, KL, KU )
*
*     The block size must not exceed the limit set by the size of the
*     local arrays WORK13 and WORK31.
*
      NB = MIN( NB, NBMAX )
*
      IF( NB.LE.1 .OR. NB.GT.KL ) THEN
```

```
*
*        Use unblocked code
*
         CALL DGBTF2( M, N, KL, KU, AB, LDAB, IPIV, INFO )
      ELSE
*
*        Use blocked code
*
*        Zero the superdiagonal elements of the work array WORK13
*
         DO 20 J = 1, NB
            DO 10 I = 1, J - 1
               WORK13( I, J ) = ZERO
   10       CONTINUE
   20    CONTINUE
*
*        Zero the subdiagonal elements of the work array WORK31
*
         DO 40 J = 1, NB
            DO 30 I = J + 1, NB
               WORK31( I, J ) = ZERO
   30       CONTINUE
   40    CONTINUE
*
*        Gaussian elimination with partial pivoting
*
*        Set fill-in elements in columns KU+2 to KV to zero
*
         DO 60 J = KU + 2, MIN( KV, N )
            DO 50 I = KV - J + 2, KL
               AB( I, J ) = ZERO
   50       CONTINUE
   60    CONTINUE
*
*        JU is the index of the last column affected by the current
*        stage of the factorization
*
         JU = 1
*
         DO 180 J = 1, MIN( M, N ), NB
            JB = MIN( NB, MIN( M, N )-J+1 )
*
*           The active part of the matrix is partitioned
*
*              A11   A12   A13
*              A21   A22   A23
*              A31   A32   A33
*
*           Here A11, A21 and A31 denote the current block of JB columns
*           which is about to be factorized. The number of rows in the
```

```
*          partitioning are JB, I2, I3 respectively, and the numbers
*          of columns are JB, J2, J3. The superdiagonal elements of A13
*          and the subdiagonal elements of A31 lie outside the band.
*
           I2 = MIN( KL-JB, M-J-JB+1 )
           I3 = MIN( JB, M-J-KL+1 )
*
*          J2 and J3 are computed after JU has been updated.
*
*          Factorize the current block of JB columns
*
           DO 80 JJ = J, J + JB - 1
*
*             Set fill-in elements in column JJ+KV to zero
*
              IF( JJ+KV.LE.N ) THEN
                 DO 70 I = 1, KL
                    AB( I, JJ+KV ) = ZERO
   70            CONTINUE
              END IF
*
*             Find pivot and test for singularity. KM is the number of
*             subdiagonal elements in the current column.
*
              KM = MIN( KL, M-JJ )
              JP = IDAMAX( KM+1, AB( KV+1, JJ ), 1 )
              IPIV( JJ ) = JP + JJ - J
              IF( AB( KV+JP, JJ ).NE.ZERO ) THEN
                 JU = MAX( JU, MIN( JJ+KU+JP-1, N ) )
                 IF( JP.NE.1 ) THEN
*
*                   Apply interchange to columns J to J+JB-1
*
                    IF( JP+JJ-1.LT.J+KL ) THEN
*
                       CALL DSWAP( JB, AB( KV+1+JJ-J, J ), LDAB-1,
     $                              AB( KV+JP+JJ-J, J ), LDAB-1 )
                    ELSE
*
*                      The interchange affects columns J to JJ-1 of A31
*                      which are stored in the work array WORK31
*
                       CALL DSWAP( JJ-J, AB( KV+1+JJ-J, J ), LDAB-1,
     $                              WORK31( JP+JJ-J-KL, 1 ), LDWORK )
                       CALL DSWAP( J+JB-JJ, AB( KV+1, JJ ), LDAB-1,
     $                              AB( KV+JP, JJ ), LDAB-1 )
                    END IF
                 END IF
*
*                Compute multipliers
```

```
*
                 CALL DSCAL( KM, ONE / AB( KV+1, JJ ), AB( KV+2, JJ ),
     $                       1 )
*
*                Update trailing submatrix within the band and within
*                the current block. JM is the index of the last column
*                which needs to be updated.
*
                 JM = MIN( JU, J+JB-1 )
                 IF( JM.GT.JJ )
     $              CALL DGER( KM, JM-JJ, -ONE, AB( KV+2, JJ ), 1,
     $                         AB( KV, JJ+1 ), LDAB-1,
     $                         AB( KV+1, JJ+1 ), LDAB-1 )
              ELSE
*
*                If pivot is zero, set INFO to the index of the pivot
*                unless a zero pivot has already been found.
*
                 IF( INFO.EQ.0 )
     $              INFO = JJ
              END IF
*
*             Copy current column of A31 into the work array WORK31
*
              NW = MIN( JJ-J+1, I3 )
              IF( NW.GT.0 )
     $           CALL DCOPY( NW, AB( KV+KL+1-JJ+J, JJ ), 1,
     $                       WORK31( 1, JJ-J+1 ), 1 )
   80       CONTINUE
            IF( J+JB.LE.N ) THEN
*
*             Apply the row interchanges to the other blocks.
*
              J2 = MIN( JU-J+1, KV ) - JB
              J3 = MAX( 0, JU-J-KV+1 )
*
*             Use DLASWP to apply the row interchanges to A12, A22, and
*             A32.
*
              CALL DLASWP( J2, AB( KV+1-JB, J+JB ), LDAB-1, 1, JB,
     $                     IPIV( J ), 1 )
*
*             Adjust the pivot indices.
*
              DO 90 I = J, J + JB - 1
                 IPIV( I ) = IPIV( I ) + J - 1
   90         CONTINUE
*
*             Apply the row interchanges to A13, A23, and A33
*             columnwise.
```

```
*
              K2 = J - 1 + JB + J2
              DO 110 I = 1, J3
                 JJ = K2 + I
                 DO 100 II = J + I - 1, J + JB - 1
                    IP = IPIV( II )
                    IF( IP.NE.II ) THEN
                       TEMP = AB( KV+1+II-JJ, JJ )
                       AB( KV+1+II-JJ, JJ ) = AB( KV+1+IP-JJ, JJ )
                       AB( KV+1+IP-JJ, JJ ) = TEMP
                    END IF
  100            CONTINUE
  110         CONTINUE
*
*             Update the relevant part of the trailing submatrix
*
              IF( J2.GT.0 ) THEN
*
*                Update A12
*
                 CALL DTRSM( 'Left', 'Lower', 'No transpose', 'Unit',
     $                       JB, J2, ONE, AB( KV+1, J ), LDAB-1,
     $                       AB( KV+1-JB, J+JB ), LDAB-1 )
*
                 IF( I2.GT.0 ) THEN
*
*                   Update A22
*
                    CALL DGEMM( 'No transpose', 'No transpose', I2, J2,
     $                          JB, -ONE, AB( KV+1+JB, J ), LDAB-1,
     $                          AB( KV+1-JB, J+JB ), LDAB-1, ONE,
     $                          AB( KV+1, J+JB ), LDAB-1 )
                 END IF
*
                 IF( I3.GT.0 ) THEN
*
*                   Update A32
*
                    CALL DGEMM( 'No transpose', 'No transpose', I3, J2,
     $                          JB, -ONE, WORK31, LDWORK,
     $                          AB( KV+1-JB, J+JB ), LDAB-1, ONE,
     $                          AB( KV+KL+1-JB, J+JB ), LDAB-1 )
                 END IF
              END IF
*
              IF( J3.GT.0 ) THEN
*
*                Copy the lower triangle of A13 into the work array
*                WORK13
*
```

```
                 DO 130 JJ = 1, J3
                    DO 120 II = JJ, JB
                       WORK13( II, JJ ) = AB( II-JJ+1, JJ+J+KV-1 )
  120               CONTINUE
  130            CONTINUE
*
*                Update A13 in the work array
*
                 CALL DTRSM( 'Left', 'Lower', 'No transpose', 'Unit',
     $                       JB, J3, ONE, AB( KV+1, J ), LDAB-1,
     $                       WORK13, LDWORK )
*
                 IF( I2.GT.0 ) THEN
*
*                   Update A23
*
                    CALL DGEMM( 'No transpose', 'No transpose', I2, J3,
     $                          JB, -ONE, AB( KV+1+JB, J ), LDAB-1,
     $                          WORK13, LDWORK, ONE, AB( 1+JB, J+KV ),
     $                          LDAB-1 )
                 END IF
*
                 IF( I3.GT.0 ) THEN
*
*                   Update A33
*
                    CALL DGEMM( 'No transpose', 'No transpose', I3, J3,
     $                          JB, -ONE, WORK31, LDWORK, WORK13,
     $                          LDWORK, ONE, AB( 1+KL, J+KV ), LDAB-1 )
                 END IF
*
*                Copy the lower triangle of A13 back into place
*
                 DO 150 JJ = 1, J3
                    DO 140 II = JJ, JB
                       AB( II-JJ+1, JJ+J+KV-1 ) = WORK13( II, JJ )
  140               CONTINUE
  150            CONTINUE
              END IF
           ELSE
*
*             Adjust the pivot indices.
*
              DO 160 I = J, J + JB - 1
                 IPIV( I ) = IPIV( I ) + J - 1
  160         CONTINUE
           END IF
*
*          Partially undo the interchanges in the current block to
*          restore the upper triangular form of A31 and copy the upper
```

```
*          triangle of A31 back into place
*
           DO 170 JJ = J + JB - 1, J, -1
              JP = IPIV( JJ ) - JJ + 1
              IF( JP.NE.1 ) THEN
*
*                Apply interchange to columns J to JJ-1
*
                 IF( JP+JJ-1.LT.J+KL ) THEN
*
*                   The interchange does not affect A31
*
                    CALL DSWAP( JJ-J, AB( KV+1+JJ-J, J ), LDAB-1,
     $                           AB( KV+JP+JJ-J, J ), LDAB-1 )
                 ELSE
*
*                   The interchange does affect A31
*
                    CALL DSWAP( JJ-J, AB( KV+1+JJ-J, J ), LDAB-1,
     $                           WORK31( JP+JJ-J-KL, 1 ), LDWORK )
                 END IF
              END IF
*
*             Copy the current column of A31 back into place
*
              NW = MIN( I3, JJ-J+1 )
              IF( NW.GT.0 )
     $           CALL DCOPY( NW, WORK31( 1, JJ-J+1 ), 1,
     $                       AB( KV+KL+1-JJ+J, JJ ), 1 )
  170       CONTINUE
  180    CONTINUE
      END IF
*
      RETURN
*
*     End of DGBTRF
*
      END
```

———-

44.8.4 lisp code

— dgbtrf.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0) (nbmax 64) (ldwork (+ nbmax 1)))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
```

```
          (type (f2cl-lib:integer4 64 64) nbmax)
          (type (f2cl-lib:integer4) ldwork)
          (ignorable one zero nbmax ldwork))
 (defun dgbtrf (m n kl ku ab ldab ipiv info)
   (declare (type (array f2cl-lib:integer4 (*)) ipiv)
            (type (array double-float (*)) ab)
            (type (f2cl-lib:integer4) info ldab ku kl n m))
   (f2cl-lib:with-multi-array-data
       ((ab double-float ab-%data% ab-%offset%)
        (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
     (prog ((work13
             (make-array (the fixnum (reduce #'* (list ldwork nbmax)))
                         :element-type 'double-float))
            (work31
             (make-array (the fixnum (reduce #'* (list ldwork nbmax)))
                         :element-type 'double-float))
            (temp 0.0d0) (i 0) (i2 0) (i3 0) (ii 0) (ip 0) (j 0) (j2 0) (j3 0)
            (jb 0) (jj 0) (jm 0) (jp 0) (ju 0) (k2 0) (km 0) (kv 0) (nb 0)
            (nw 0))
       (declare (type (array double-float (*)) work13 work31)
                (type (double-float) temp)
                (type (f2cl-lib:integer4) i i2 i3 ii ip j j2 j3 jb jj jm jp ju
                 k2 km kv nb nw))
       (setf kv (f2cl-lib:int-add ku kl))
       (setf info 0)
       (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
             ((< kl 0) (setf info -3)) ((< ku 0) (setf info -4))
             ((< ldab (f2cl-lib:int-add kl kv 1)) (setf info -6)))
       (cond
        ((/= info 0) (xerbla "DGBTRF" (f2cl-lib:int-sub info))
         (go end_label)))
       (if (or (= m 0) (= n 0))
           (go end_label))
       (setf nb
               (multiple-value-bind
                   (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                   (ilaenv 1 "DGBTRF" " " m n kl ku)
                 (declare (ignore var-0 var-1 var-2))
                 (when var-3 (setf m var-3))
                 (when var-4 (setf n var-4))
                 (when var-5 (setf kl var-5))
                 (when var-6 (setf ku var-6))
                 ret-val))
       (setf nb
               (min (the f2cl-lib:integer4 nb) (the f2cl-lib:integer4 nbmax)))
       (cond
        ((or (<= nb 1) (> nb kl))
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
             (dgbtf2 m n kl ku ab ldab ipiv info)
```

```
    (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6))
    (setf info var-7)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j nb) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref work13 (i j) ((1 ldwork) (1 nbmax)))
                  zero)
         label10))
     label20))
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j nb) nil)
    (tagbody
      (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                    ((> i nb) nil)
        (tagbody
          (setf (f2cl-lib:fref work31 (i j) ((1 ldwork) (1 nbmax)))
                  zero)
         label30))
     label40))
   (f2cl-lib:fdo (j (f2cl-lib:int-add ku 2) (f2cl-lib:int-add j 1))
                 ((> j
                     (min (the f2cl-lib:integer4 kv)
                          (the f2cl-lib:integer4 n)))
                  nil)
    (tagbody
      (f2cl-lib:fdo (i (f2cl-lib:int-add kv (f2cl-lib:int-sub j) 2)
                     (f2cl-lib:int-add i 1))
                    ((> i kl) nil)
        (tagbody
          (setf (f2cl-lib:fref ab-%data% (i j) ((1 ldab) (1 *))
                                ab-%offset%)
                  zero)
         label50))
     label60))
   (setf ju 1)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                 ((> j
                     (min (the f2cl-lib:integer4 m)
                          (the f2cl-lib:integer4 n)))
                  nil)
    (tagbody
      (setf jb
              (min (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add
```

```
                    (f2cl-lib:int-sub
                     (min (the f2cl-lib:integer4 m)
                          (the f2cl-lib:integer4 n))
                     j)
                    1))))
(setf i2
        (min (the f2cl-lib:integer4 (f2cl-lib:int-sub kl jb))
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub m j jb)
                                    1))))
(setf i3
        (min (the f2cl-lib:integer4 jb)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub m j kl)
                                    1))))
(f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
              ((> jj
                  (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (cond
     ((<= (f2cl-lib:int-add jj kv) n)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i kl) nil)
        (tagbody
          (setf (f2cl-lib:fref ab-%data%
                               (i (f2cl-lib:int-add jj kv))
                               ((1 ldab) (1 *)) ab-%offset%)
                  zero)
         label70))))
    (setf km
            (min (the f2cl-lib:integer4 kl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-sub m jj))))
    (setf jp
            (idamax (f2cl-lib:int-add km 1)
             (f2cl-lib:array-slice ab-%data% double-float
                                   ((+ kv 1) jj) ((1 ldab) (1 *))
                                   ab-%offset%)
             1))
    (setf (f2cl-lib:fref ipiv-%data% (jj) ((1 *)) ipiv-%offset%)
            (f2cl-lib:int-sub (f2cl-lib:int-add jp jj) j))
    (cond
     ((/=
       (f2cl-lib:fref ab ((f2cl-lib:int-add kv jp) jj)
                      ((1 ldab) (1 *)))
       zero)
      (setf ju
              (max (the f2cl-lib:integer4 ju)
                   (the f2cl-lib:integer4
```

```
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-sub
                           (f2cl-lib:int-add jj ku jp) 1))
                     (the f2cl-lib:integer4 n)))))
(cond
 ((/= jp 1)
  (cond
   ((< (f2cl-lib:int-add jp jj (f2cl-lib:int-sub 1))
       (f2cl-lib:int-add j kl))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dswap jb
         (f2cl-lib:array-slice ab-%data% double-float
                               ((+ kv 1 jj
                                   (f2cl-lib:int-sub j))
                                j)
                               ((1 ldab) (1 *))
                               ab-%offset%)
         (f2cl-lib:int-sub ldab 1)
         (f2cl-lib:array-slice ab-%data% double-float
                               ((+ kv jp jj
                                   (f2cl-lib:int-sub j))
                                j)
                               ((1 ldab) (1 *))
                               ab-%offset%)
         (f2cl-lib:int-sub ldab 1))
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf jb var-0))))
   (t
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dswap (f2cl-lib:int-sub jj j)
         (f2cl-lib:array-slice ab-%data% double-float
                               ((+ kv 1 jj
                                   (f2cl-lib:int-sub j))
                                j)
                               ((1 ldab) (1 *))
                               ab-%offset%)
         (f2cl-lib:int-sub ldab 1)
         (f2cl-lib:array-slice work31 double-float
                               ((+ jp jj
                                   (f2cl-lib:int-sub j)
                                   (f2cl-lib:int-sub kl))
                                1)
                               ((1 ldwork) (1 nbmax)))
         ldwork)
      (declare (ignore var-0 var-1 var-2 var-3))
      (when var-4 (setf ldwork var-4)))
    (dswap (f2cl-lib:int-sub (f2cl-lib:int-add j jb) jj)
     (f2cl-lib:array-slice ab-%data% double-float
                           ((+ kv 1) jj) ((1 ldab) (1 *))
```

```
                              ab-%offset%)
      (f2cl-lib:int-sub ldab 1)
      (f2cl-lib:array-slice ab-%data% double-float
                              ((+ kv jp) jj) ((1 ldab) (1 *))
                              ab-%offset%)
      (f2cl-lib:int-sub ldab 1))))))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (dscal km
      (/ one
         (f2cl-lib:fref ab-%data%
                        ((f2cl-lib:int-add kv 1) jj)
                        ((1 ldab) (1 *)) ab-%offset%))
      (f2cl-lib:array-slice ab-%data% double-float
                              ((+ kv 2) jj) ((1 ldab) (1 *))
                              ab-%offset%)
      1)
   (declare (ignore var-1 var-2 var-3))
   (when var-0 (setf km var-0)))
 (setf jm
         (min (the f2cl-lib:integer4 ju)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-sub (f2cl-lib:int-add j jb)
                                     1))))
 (if (> jm jj)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
          var-8)
         (dger km (f2cl-lib:int-sub jm jj) (- one)
          (f2cl-lib:array-slice ab-%data% double-float
                                ((+ kv 2) jj)
                                ((1 ldab) (1 *))
                                ab-%offset%)
          1
          (f2cl-lib:array-slice ab-%data% double-float
                                (kv (f2cl-lib:int-add jj 1))
                                ((1 ldab) (1 *))
                                ab-%offset%)
          (f2cl-lib:int-sub ldab 1)
          (f2cl-lib:array-slice ab-%data% double-float
                                ((+ kv 1)
                                 (f2cl-lib:int-add jj 1))
                                ((1 ldab) (1 *))
                                ab-%offset%)
          (f2cl-lib:int-sub ldab 1))
       (declare
        (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7
         var-8))
       (when var-0 (setf km var-0)))))
(t
 (if (= info 0)
```

```
            (setf info jj))))
      (setf nw
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub jj j) 1))
               (the f2cl-lib:integer4 i3)))
      (if (> nw 0)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dcopy nw
               (f2cl-lib:array-slice ab-%data% double-float
                                     ((+ kv kl 1
                                         (f2cl-lib:int-sub jj) j)
                                      jj)
                                     ((1 ldab) (1 *)) ab-%offset%)
               1
               (f2cl-lib:array-slice work31 double-float
                                     (1
                                      (f2cl-lib:int-add
                                       (f2cl-lib:int-sub jj j) 1))
                                     ((1 ldwork) (1 nbmax)))
               1)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf nw var-0))))
    label80))
  (cond
   ((<= (f2cl-lib:int-add j jb) n)
    (setf j2
            (f2cl-lib:int-sub
             (min
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-sub ju j) 1))
              (the f2cl-lib:integer4 kv))
             jb))
    (setf j3
            (max (the f2cl-lib:integer4 0)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-sub ju j kv)
                                        1))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaswp j2
         (f2cl-lib:array-slice ab-%data% double-float
                               ((+ kv 1 (f2cl-lib:int-sub jb))
                                (f2cl-lib:int-add j jb))
                               ((1 ldab) (1 *)) ab-%offset%)
         (f2cl-lib:int-sub ldab 1) 1 jb
         (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (j)
                               ((1 *)) ipiv-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-5 var-6))
```

```
  (when var-0 (setf j2 var-0))
  (when var-4 (setf jb var-4)))
(f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
              ((> i
                  (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
            (f2cl-lib:int-sub
             (f2cl-lib:int-add
              (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                             ipiv-%offset%)
              j)
             1))
   label90))
(setf k2 (f2cl-lib:int-add (f2cl-lib:int-sub j 1) jb j2))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i j3) nil)
  (tagbody
    (setf jj (f2cl-lib:int-add k2 i))
    (f2cl-lib:fdo (ii
                   (f2cl-lib:int-add j i (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add ii 1))
                  ((> ii
                      (f2cl-lib:int-add j jb
                                        (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf ip
                (f2cl-lib:fref ipiv-%data% (ii) ((1 *))
                               ipiv-%offset%))
        (cond
         ((/= ip ii)
          (setf temp
                  (f2cl-lib:fref ab-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-add kv 1 ii)
                                   jj)
                                  jj)
                                 ((1 ldab) (1 *)) ab-%offset%))
          (setf (f2cl-lib:fref ab-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add kv 1 ii) jj)
                                jj)
                               ((1 ldab) (1 *)) ab-%offset%)
                  (f2cl-lib:fref ab-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-add kv 1 ip)
                                   jj)
                                  jj)
```

```
                                    ((1 ldab) (1 *)) ab-%offset%))
            (setf (f2cl-lib:fref ab-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-add kv 1 ip) jj)
                                  jj)
                                 ((1 ldab) (1 *)) ab-%offset%)
                  temp)))
        label100))
    label110))
  (cond
   ((> j2 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10)
        (dtrsm "Left" "Lower" "No transpose" "Unit" jb j2 one
         (f2cl-lib:array-slice ab-%data% double-float
                               ((+ kv 1) j) ((1 ldab) (1 *))
                               ab-%offset%)
         (f2cl-lib:int-sub ldab 1)
         (f2cl-lib:array-slice ab-%data% double-float
                               ((+ kv 1 (f2cl-lib:int-sub jb))
                                (f2cl-lib:int-add j jb))
                               ((1 ldab) (1 *)) ab-%offset%)
         (f2cl-lib:int-sub ldab 1))
      (declare
       (ignore var-0 var-1 var-2 var-3 var-7 var-8 var-9 var-10))
      (when var-4 (setf jb var-4))
      (when var-5 (setf j2 var-5))
      (when var-6 (setf one var-6)))
    (cond
     ((> i2 0)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (dgemm "No transpose" "No transpose" i2 j2 jb (- one)
           (f2cl-lib:array-slice ab-%data% double-float
                                 ((+ kv 1 jb) j) ((1 ldab) (1 *))
                                 ab-%offset%)
           (f2cl-lib:int-sub ldab 1)
           (f2cl-lib:array-slice ab-%data% double-float
                                 ((+ kv 1 (f2cl-lib:int-sub jb))
                                  (f2cl-lib:int-add j jb))
                                 ((1 ldab) (1 *)) ab-%offset%)
           (f2cl-lib:int-sub ldab 1) one
           (f2cl-lib:array-slice ab-%data% double-float
                                 ((+ kv 1)
                                  (f2cl-lib:int-add j jb))
                                 ((1 ldab) (1 *)) ab-%offset%)
           (f2cl-lib:int-sub ldab 1))
        (declare
```

```
        (ignore var-0 var-1 var-5 var-6 var-7 var-8 var-9 var-11
         var-12))
       (when var-2 (setf i2 var-2))
       (when var-3 (setf j2 var-3))
       (when var-4 (setf jb var-4))
       (when var-10 (setf one var-10)))))
  (cond
   ((> i3 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (dgemm "No transpose" "No transpose" i3 j2 jb (- one)
         work31 ldwork
         (f2cl-lib:array-slice ab-%data% double-float
                               ((+ kv 1 (f2cl-lib:int-sub jb))
                                (f2cl-lib:int-add j jb))
                               ((1 ldab) (1 *)) ab-%offset%)
         (f2cl-lib:int-sub ldab 1) one
         (f2cl-lib:array-slice ab-%data% double-float
                               ((+ kv kl 1
                                   (f2cl-lib:int-sub jb))
                                (f2cl-lib:int-add j jb))
                               ((1 ldab) (1 *)) ab-%offset%)
         (f2cl-lib:int-sub ldab 1))
      (declare
       (ignore var-0 var-1 var-5 var-6 var-8 var-9 var-11
        var-12))
      (when var-2 (setf i3 var-2))
      (when var-3 (setf j2 var-3))
      (when var-4 (setf jb var-4))
      (when var-7 (setf ldwork var-7))
      (when var-10 (setf one var-10)))))))
(cond
 ((> j3 0)
  (f2cl-lib:fdo (jj 1 (f2cl-lib:int-add jj 1))
                ((> jj j3) nil)
    (tagbody
      (f2cl-lib:fdo (ii jj (f2cl-lib:int-add ii 1))
                    ((> ii jb) nil)
        (tagbody
          (setf (f2cl-lib:fref work13 (ii jj)
                               ((1 ldwork) (1 nbmax)))
                  (f2cl-lib:fref ab-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-sub ii jj) 1)
                                  (f2cl-lib:int-sub
                                   (f2cl-lib:int-add jj j kv)
                                   1))
                                 ((1 ldab) (1 *)) ab-%offset%))
         label120))
```

```
   label130))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dtrsm "Left" "Lower" "No transpose" "Unit" jb j3 one
     (f2cl-lib:array-slice ab-%data% double-float
                           ((+ kv 1) j) ((1 ldab) (1 *))
                           ab-%offset%)
     (f2cl-lib:int-sub ldab 1) work13 ldwork)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-7 var-8 var-9))
  (when var-4 (setf jb var-4))
  (when var-5 (setf j3 var-5))
  (when var-6 (setf one var-6))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> i2 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "No transpose" "No transpose" i2 j3 jb (- one)
       (f2cl-lib:array-slice ab-%data% double-float
                             ((+ kv 1 jb) j) ((1 ldab) (1 *))
                             ab-%offset%)
       (f2cl-lib:int-sub ldab 1) work13 ldwork one
       (f2cl-lib:array-slice ab-%data% double-float
                             ((+ 1 jb)
                              (f2cl-lib:int-add j kv))
                             ((1 ldab) (1 *)) ab-%offset%)
       (f2cl-lib:int-sub ldab 1))
    (declare
     (ignore var-0 var-1 var-5 var-6 var-7 var-8 var-11
      var-12))
    (when var-2 (setf i2 var-2))
    (when var-3 (setf j3 var-3))
    (when var-4 (setf jb var-4))
    (when var-9 (setf ldwork var-9))
    (when var-10 (setf one var-10)))))
(cond
 ((> i3 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "No transpose" "No transpose" i3 j3 jb (- one)
       work31 ldwork work13 ldwork one
       (f2cl-lib:array-slice ab-%data% double-float
                             ((+ 1 kl)
                              (f2cl-lib:int-add j kv))
                             ((1 ldab) (1 *)) ab-%offset%)
       (f2cl-lib:int-sub ldab 1))
```

```
        (declare
         (ignore var-0 var-1 var-5 var-6 var-8 var-11 var-12))
        (when var-2 (setf i3 var-2))
        (when var-3 (setf j3 var-3))
        (when var-4 (setf jb var-4))
        (when var-7 (setf ldwork var-7))
        (when var-9 (setf ldwork var-9))
        (when var-10 (setf one var-10)))))
    (f2cl-lib:fdo (jj 1 (f2cl-lib:int-add jj 1))
                  ((> jj j3) nil)
      (tagbody
        (f2cl-lib:fdo (ii jj (f2cl-lib:int-add ii 1))
                      ((> ii jb) nil)
          (tagbody
            (setf (f2cl-lib:fref ab-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-sub ii jj) 1)
                                  (f2cl-lib:int-sub
                                   (f2cl-lib:int-add jj j kv) 1))
                                 ((1 ldab) (1 *)) ab-%offset%)
                  (f2cl-lib:fref work13 (ii jj)
                                 ((1 ldwork) (1 nbmax))))
          label140))
      label150)))))
 (t
  (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                ((> i
                    (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
              (f2cl-lib:int-sub
               (f2cl-lib:int-add
                (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                               ipiv-%offset%)
                j)
               1))
     label160))))
(f2cl-lib:fdo (jj (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add jj (f2cl-lib:int-sub 1)))
              ((> jj j) nil)
  (tagbody
    (setf jp
            (f2cl-lib:int-add
             (f2cl-lib:int-sub
              (f2cl-lib:fref ipiv-%data% (jj) ((1 *))
                             ipiv-%offset%)
              jj)
             1))
    (cond
```

```
((/= jp 1)
 (cond
  ((< (f2cl-lib:int-add jp jj (f2cl-lib:int-sub 1))
      (f2cl-lib:int-add j kl))
   (dswap (f2cl-lib:int-sub jj j)
    (f2cl-lib:array-slice ab-%data% double-float
                          ((+ kv 1 jj (f2cl-lib:int-sub j))
                           j)
                          ((1 ldab) (1 *)) ab-%offset%)
    (f2cl-lib:int-sub ldab 1)
    (f2cl-lib:array-slice ab-%data% double-float
                          ((+ kv jp jj (f2cl-lib:int-sub j))
                           j)
                          ((1 ldab) (1 *)) ab-%offset%)
    (f2cl-lib:int-sub ldab 1)))
  (t
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
       (dswap (f2cl-lib:int-sub jj j)
        (f2cl-lib:array-slice ab-%data% double-float
                              ((+ kv 1 jj
                                  (f2cl-lib:int-sub j))
                               j)
                              ((1 ldab) (1 *)) ab-%offset%)
        (f2cl-lib:int-sub ldab 1)
        (f2cl-lib:array-slice work31 double-float
                              ((+ jp jj (f2cl-lib:int-sub j)
                                  (f2cl-lib:int-sub kl))
                               1)
                              ((1 ldwork) (1 nbmax)))
        ldwork)
     (declare (ignore var-0 var-1 var-2 var-3))
     (when var-4 (setf ldwork var-4)))))))
(setf nw
       (min (the f2cl-lib:integer4 i3)
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-sub jj j)
                                   1))))
(if (> nw 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy nw
         (f2cl-lib:array-slice work31 double-float
                               (1
                                (f2cl-lib:int-add
                                 (f2cl-lib:int-sub jj j) 1))
                               ((1 ldwork) (1 nbmax)))
         1
         (f2cl-lib:array-slice ab-%data% double-float
                               ((+ kv kl 1
                                   (f2cl-lib:int-sub jj) j)
                                jj)
```

```
                                                  ((1 ldab) (1 *)) ab-%offset%)
                           1)
                        (declare (ignore var-1 var-2 var-3 var-4))
                        (when var-0 (setf nw var-0))))
                 label170))
              label180))))
         (go end_label)
        end_label
         (return (values m n kl ku nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgbtrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n
                                              fortran-to-lisp::kl
                                              fortran-to-lisp::ku nil nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::dgbtf2))))
```

———-

44.9 dgbtrs

44.9.1 Axiom unit tests

— dgbtrs.input —

```
)set break resume
)sys rm -f dgbtrs.output
)spool dgbtrs.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

44.9.2 Axiom help page

— **dgbtrs.help** —

```
=====================================================================
dgbtrs examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgbtrs (TRANS, N, KL, KU, NRHS, AB, LDAB, IPIV, B, LDB,
           INFO)
           DGBTRS

Function/Subroutine Documentation
   subroutine dgbtrs (character TRANS, integer N, integer KL, integer KU,
       integer NRHS, double precision, dimension( ldab, * ) AB, integer LDAB,
       integer, dimension( * ) IPIV, double precision, dimension( ldb, * ) B,
       integer LDB, integer INFO)
       DGBTRS

Purpose:
   DGBTRS solves a system of linear equations
      A * X = B  or  A**T * X = B
   with a general band matrix A using the LU factorization computed by DGBTRF.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations.
   = 'N':  A * X = B  (No transpose)
   = 'T':  A**T* X = B  (Transpose)
   = 'C':  A**T* X = B  (Conjugate transpose = Transpose)

 N
```

```
  N is INTEGER
  The order of the matrix A.  N >= 0.

KL
  KL is INTEGER
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrix B.  NRHS >= 0.

AB
  AB is DOUBLE PRECISION array, dimension (LDAB,N)
  Details of the LU factorization of the band matrix A, as
  computed by DGBTRF.  U is stored as an upper triangular band
  matrix with KL+KU superdiagonals in rows 1 to KL+KU+1, and
  the multipliers used during the factorization are stored in
  rows KL+KU+2 to 2*KL+KU+1.

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= N, row i of the matrix was
  interchanged with row IPIV(i).

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the right hand side matrix B.
  On exit, the solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
```

———-

44.9.3 fortran code

— dgbtrs.f —

```
*  =====================================================================
      SUBROUTINE DGBTRS( TRANS, N, KL, KU, NRHS, AB, LDAB, IPIV, B, LDB,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, KL, KU, LDAB, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   AB( LDAB, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LNOTI, NOTRAN
      INTEGER            I, J, KD, L, LM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEMV, DGER, DSWAP, DTBSV, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
```

```
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDAB.LT.( 2*KL+KU+1 ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -10
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGBTRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      KD = KU + KL + 1
      LNOTI = KL.GT.0
*
      IF( NOTRAN ) THEN
*
*        Solve  A*X = B.
*
*        Solve L*X = B, overwriting B with X.
*
*        L is represented as a product of permutations and unit lower
*        triangular matrices L = P(1) * L(1) * ... * P(n-1) * L(n-1),
*        where each transformation L(i) is a rank-one modification of
*        the identity matrix.
*
         IF( LNOTI ) THEN
            DO 10 J = 1, N - 1
               LM = MIN( KL, N-J )
               L = IPIV( J )
               IF( L.NE.J )
     $            CALL DSWAP( NRHS, B( L, 1 ), LDB, B( J, 1 ), LDB )
               CALL DGER( LM, NRHS, -ONE, AB( KD+1, J ), 1, B( J, 1 ),
     $                    LDB, B( J+1, 1 ), LDB )
   10       CONTINUE
         END IF
*
```

```
         DO 20 I = 1, NRHS
*
*           Solve U*X = B, overwriting B with X.
*
            CALL DTBSV( 'Upper', 'No transpose', 'Non-unit', N, KL+KU,
     $                  AB, LDAB, B( 1, I ), 1 )
   20    CONTINUE
*
      ELSE
*
*        Solve A**T*X = B.
*
         DO 30 I = 1, NRHS
*
*           Solve U**T*X = B, overwriting B with X.
*
            CALL DTBSV( 'Upper', 'Transpose', 'Non-unit', N, KL+KU, AB,
     $                  LDAB, B( 1, I ), 1 )
   30    CONTINUE
*
*        Solve L**T*X = B, overwriting B with X.
*
         IF( LNOTI ) THEN
            DO 40 J = N - 1, 1, -1
               LM = MIN( KL, N-J )
               CALL DGEMV( 'Transpose', LM, NRHS, -ONE, B( J+1, 1 ),
     $                     LDB, AB( KD+1, J ), 1, ONE, B( J, 1 ), LDB )
               L = IPIV( J )
               IF( L.NE.J )
     $            CALL DSWAP( NRHS, B( L, 1 ), LDB, B( J, 1 ), LDB )
   40       CONTINUE
         END IF
      END IF
      RETURN
*
*     End of DGBTRS
*
      END
```

———-

44.9.4 lisp code

— dgbtrs.lisp —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
```

```
(defun dgbtrs (trans n kl ku nrhs ab ldab ipiv b ldb$ info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array double-float (*)) b ab)
           (type (f2cl-lib:integer4) info ldb$ ldab nrhs ku kl n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (ab double-float ab-%data% ab-%offset%)
       (b double-float b-%data% b-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((i 0) (j 0) (kd 0) (l 0) (lm 0) (lnoti nil) (notran nil))
      (declare (type (f2cl-lib:integer4) i j kd l lm)
               (type f2cl-lib:logical lnoti notran))
      (setf info 0)
      (setf notran
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "N")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val))
      (cond
       ((and (not notran)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "T")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val))
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "C")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2)) ((< kl 0) (setf info -3))
       ((< ku 0) (setf info -4)) ((< nrhs 0) (setf info -5))
       ((< ldab (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
        (setf info -7))
       ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -10)))
      (cond
       ((/= info 0) (xerbla "DGBTRS" (f2cl-lib:int-sub info))
        (go end_label)))
      (if (or (= n 0) (= nrhs 0))
          (go end_label))
      (setf kd (f2cl-lib:int-add ku kl 1))
      (setf lnoti (> kl 0))
      (cond
       (notran
```

```
(cond
 (lnoti
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (setf lm
              (min (the f2cl-lib:integer4 kl)
                   (the f2cl-lib:integer4 (f2cl-lib:int-sub n j))))
      (setf l (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
      (if (/= l j)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dswap nrhs
               (f2cl-lib:array-slice b-%data% double-float (l 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% double-float (j 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dger lm nrhs (- one)
           (f2cl-lib:array-slice ab-%data% double-float ((+ kd 1) j)
                                 ((1 ldab) (1 *)) ab-%offset%)
           1
           (f2cl-lib:array-slice b-%data% double-float (j 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% double-float ((+ j 1) 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-2 var-3 var-4 var-5 var-7))
        (when var-0 (setf lm var-0))
        (when var-1 (setf nrhs var-1))
        (when var-6 (setf ldb$ var-6))
        (when var-8 (setf ldb$ var-8)))
     label10))))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nrhs) nil)
  (tagbody
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dtbsv "Upper" "No transpose" "Non-unit" n
         (f2cl-lib:int-add kl ku) ab ldab
         (f2cl-lib:array-slice b-%data% double-float (1 i)
                               ((1 ldb$) (1 *)) b-%offset%)
         1)
```

```
        (declare (ignore var-0 var-1 var-2 var-4 var-5 var-7 var-8))
        (when var-3 (setf n var-3))
        (when var-6 (setf ldab var-6)))
    label20)))
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i nrhs) nil)
   (tagbody
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (dtbsv "Upper" "Transpose" "Non-unit" n
          (f2cl-lib:int-add kl ku) ab ldab
          (f2cl-lib:array-slice b-%data% double-float (1 i)
                                ((1 ldb$) (1 *)) b-%offset%)
          1)
       (declare (ignore var-0 var-1 var-2 var-4 var-5 var-7 var-8))
       (when var-3 (setf n var-3))
       (when var-6 (setf ldab var-6)))
    label30))
 (cond
  (lnoti
   (f2cl-lib:fdo (j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                  (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                 ((> j 1) nil)
     (tagbody
       (setf lm
               (min (the f2cl-lib:integer4 kl)
                    (the f2cl-lib:integer4 (f2cl-lib:int-sub n j))))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
            var-9 var-10)
           (dgemv "Transpose" lm nrhs (- one)
            (f2cl-lib:array-slice b-%data% double-float ((+ j 1) 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$
            (f2cl-lib:array-slice ab-%data% double-float ((+ kd 1) j)
                                  ((1 ldab) (1 *)) ab-%offset%)
            1 one
            (f2cl-lib:array-slice b-%data% double-float (j 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
         (declare (ignore var-0 var-3 var-4 var-6 var-7 var-9))
         (when var-1 (setf lm var-1))
         (when var-2 (setf nrhs var-2))
         (when var-5 (setf ldb$ var-5))
         (when var-8 (setf one var-8))
         (when var-10 (setf ldb$ var-10)))
       (setf l (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
       (if (/= l j)
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
                        (dswap nrhs
                         (f2cl-lib:array-slice b-%data% double-float (l 1)
                                               ((1 ldb$) (1 *)) b-%offset%)
                         ldb$
                         (f2cl-lib:array-slice b-%data% double-float (j 1)
                                               ((1 ldb$) (1 *)) b-%offset%)
                         ldb$)
                       (declare (ignore var-1 var-3))
                       (when var-0 (setf nrhs var-0))
                       (when var-2 (setf ldb$ var-2))
                       (when var-4 (setf ldb$ var-4))))
               label40))))))
        (go end_label)
       end_label
        (return (values trans n nil nil nrhs nil ldab nil nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgbtrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::n nil nil
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::ldab nil nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

44.10 dggbak

44.10.1 Axiom unit tests

— **dggbak.input** —

```
)set break resume
)sys rm -f dggbak.output
)spool dggbak.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

44.10.2 Axiom help page

— **dggbak.help** —

```
======================================================================
dggbak examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dggbak (JOB, SIDE, N, ILO, IHI, LSCALE, RSCALE, M, V, LDV,
           INFO)
           DGGBAK

Function/Subroutine Documentation
   subroutine dggbak (character JOB, character SIDE, integer N, integer ILO,
       integer IHI, double precision, dimension( * ) LSCALE, double precision,
       dimension( * ) RSCALE, integer M, double precision, dimension( ldv, * )
       V, integer LDV, integer INFO)
       DGGBAK

Purpose:
   DGGBAK forms the right or left eigenvectors of a real generalized
   eigenvalue problem A*x = lambda*B*x, by backward transformation on
```

```
   the computed eigenvectors of the balanced pair of matrices output by DGGBAL.

Parameters:

 JOB
   JOB is CHARACTER*1
   Specifies the type of backward transformation required:
   = 'N':  do nothing, return immediately;
   = 'P':  do backward transformation for permutation only;
   = 'S':  do backward transformation for scaling only;
   = 'B':  do backward transformations for both permutation and scaling.
   JOB must be the same as the argument JOB supplied to DGGBAL.

 SIDE
   SIDE is CHARACTER*1
   = 'R':  V contains right eigenvectors;
   = 'L':  V contains left eigenvectors.

 N
   N is INTEGER
   The number of rows of the matrix V.  N >= 0.

 ILO
   ILO is INTEGER

 IHI
   IHI is INTEGER
   The integers ILO and IHI determined by DGGBAL.
   1 <= ILO <= IHI <= N, if N > 0; ILO=1 and IHI=0, if N=0.

 LSCALE
   LSCALE is DOUBLE PRECISION array, dimension (N)
   Details of the permutations and/or scaling factors applied
   to the left side of A and B, as returned by DGGBAL.

 RSCALE
   RSCALE is DOUBLE PRECISION array, dimension (N)
   Details of the permutations and/or scaling factors applied
   to the right side of A and B, as returned by DGGBAL.

 M
   M is INTEGER
   The number of columns of the matrix V.  M >= 0.

 V
   V is DOUBLE PRECISION array, dimension (LDV,M)
   On entry, the matrix of right or left eigenvectors to be
   transformed, as returned by DTGEVC.
   On exit, V is overwritten by the transformed eigenvectors.
```

```
 LDV
   LDV is INTEGER
   The leading dimension of the matrix V. LDV >= max(1,N).

 INFO
   INFO is INTEGER
   = 0:  successful exit.
   < 0:  if INFO = -i, the i-th argument had an illegal value.
```

44.10.3 fortran code

— dggbak.f —

```
*  =====================================================================
      SUBROUTINE DGGBAK( JOB, SIDE, N, ILO, IHI, LSCALE, RSCALE, M, V,
     $                   LDV, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOB, SIDE
      INTEGER            IHI, ILO, INFO, LDV, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   LSCALE( * ), RSCALE( * ), V( LDV, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LEFTV, RIGHTV
      INTEGER            I, K
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DSCAL, DSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, INT
```

```
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      RIGHTV = LSAME( SIDE, 'R' )
      LEFTV = LSAME( SIDE, 'L' )
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.RIGHTV .AND. .NOT.LEFTV ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( ILO.LT.1 ) THEN
         INFO = -4
      ELSE IF( N.EQ.0 .AND. IHI.EQ.0 .AND. ILO.NE.1 ) THEN
         INFO = -4
      ELSE IF( N.GT.0 .AND. ( IHI.LT.ILO .OR. IHI.GT.MAX( 1, N ) ) )
     $   THEN
         INFO = -5
      ELSE IF( N.EQ.0 .AND. ILO.EQ.1 .AND. IHI.NE.0 ) THEN
         INFO = -5
      ELSE IF( M.LT.0 ) THEN
         INFO = -8
      ELSE IF( LDV.LT.MAX( 1, N ) ) THEN
         INFO = -10
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGGBAK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
      IF( M.EQ.0 )
     $   RETURN
      IF( LSAME( JOB, 'N' ) )
     $   RETURN
*
      IF( ILO.EQ.IHI )
     $   GO TO 30
*
*     Backward balance
*
      IF( LSAME( JOB, 'S' ) .OR. LSAME( JOB, 'B' ) ) THEN
```

```
*
*       Backward transformation on right eigenvectors
*
        IF( RIGHTV ) THEN
           DO 10 I = ILO, IHI
              CALL DSCAL( M, RSCALE( I ), V( I, 1 ), LDV )
   10      CONTINUE
        END IF
*
*       Backward transformation on left eigenvectors
*
        IF( LEFTV ) THEN
           DO 20 I = ILO, IHI
              CALL DSCAL( M, LSCALE( I ), V( I, 1 ), LDV )
   20      CONTINUE
        END IF
      END IF
*
*     Backward permutation
*
   30 CONTINUE
      IF( LSAME( JOB, 'P' ) .OR. LSAME( JOB, 'B' ) ) THEN
*
*       Backward permutation on right eigenvectors
*
        IF( RIGHTV ) THEN
           IF( ILO.EQ.1 )
     $        GO TO 50
*
           DO 40 I = ILO - 1, 1, -1
              K = INT(RSCALE( I ))
              IF( K.EQ.I )
     $           GO TO 40
              CALL DSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   40      CONTINUE
*
   50      CONTINUE
           IF( IHI.EQ.N )
     $        GO TO 70
           DO 60 I = IHI + 1, N
              K = INT(RSCALE( I ))
              IF( K.EQ.I )
     $           GO TO 60
              CALL DSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   60      CONTINUE
        END IF
*
*       Backward permutation on left eigenvectors
*
   70   CONTINUE
```

```
         IF( LEFTV ) THEN
            IF( ILO.EQ.1 )
     $         GO TO 90
            DO 80 I = ILO - 1, 1, -1
               K = INT(LSCALE( I ))
               IF( K.EQ.I )
     $            GO TO 80
               CALL DSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   80       CONTINUE
*
   90       CONTINUE
            IF( IHI.EQ.N )
     $         GO TO 110
            DO 100 I = IHI + 1, N
               K = INT(LSCALE( I ))
               IF( K.EQ.I )
     $            GO TO 100
               CALL DSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
  100       CONTINUE
         END IF
      END IF
*
  110 CONTINUE
*
      RETURN
*
*     End of DGGBAK
*
      END
```

———-

44.10.4 lisp code

— dggbak.lisp —

```
(defun dggbak (job side n ilo ihi lscale rscale m v ldv info)
  (declare (type (array double-float (*)) v rscale lscale)
           (type (f2cl-lib:integer4) info ldv m ihi ilo n)
           (type (string *) side job))
  (f2cl-lib:with-multi-array-data
      ((job character job-%data% job-%offset%)
       (side character side-%data% side-%offset%)
       (lscale double-float lscale-%data% lscale-%offset%)
       (rscale double-float rscale-%data% rscale-%offset%)
       (v double-float v-%data% v-%offset%))
    (prog ((i 0) (k 0) (leftv nil) (rightv nil))
      (declare (type f2cl-lib:logical rightv leftv)
```

```
         (type (f2cl-lib:integer4) k i))
(setf rightv
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "R")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val))
(setf leftv
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "L")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val))
(setf info 0)
(cond
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "N")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "P")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "S")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "B")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)))
  (setf info -1))
 ((and (not rightv) (not leftv)) (setf info -2)) ((< n 0) (setf info -3))
 ((< ilo 1) (setf info -4))
 ((and (= n 0) (= ihi 0) (/= ilo 1)) (setf info -4))
 ((and (> n 0)
       (or (< ihi ilo)
           (> ihi
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 n)))))
  (setf info -5))
```

```
 ((and (= n 0) (= ilo 1) (/= ihi 0)) (setf info -5))
 ((< m 0) (setf info -8))
 ((< ldv (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -10)))
(cond
 ((/= info 0) (xerbla "DGGBAK" (f2cl-lib:int-sub info)) (go end_label)))
(if (= n 0)
    (go end_label))
(if (= m 0)
    (go end_label))
(if (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "N")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)
    (go end_label))
(if (= ilo ihi)
    (go label30))
(cond
 ((or
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame job "S")
     (declare (ignore var-1))
     (when var-0 (setf job var-0))
     ret-val)
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame job "B")
     (declare (ignore var-1))
     (when var-0 (setf job var-0))
     ret-val))
  (cond
   (rightv
    (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                  ((> i ihi) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal m
             (f2cl-lib:array-slice rscale-%data% double-float (i) ((1 *))
                                   rscale-%offset%)
             (f2cl-lib:array-slice v-%data% double-float (i 1)
                                   ((1 ldv) (1 *)) v-%offset%)
             ldv)
          (declare (ignore var-1 var-2))
          (when var-0 (setf m var-0))
          (when var-3 (setf ldv var-3)))
       label10))))
  (cond
   (leftv
    (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                  ((> i ihi) nil)
```

```
        (tagbody
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (dscal m
               (f2cl-lib:array-slice lscale-%data% double-float (i) ((1 *))
                                     lscale-%offset%)
               (f2cl-lib:array-slice v-%data% double-float (i 1)
                                     ((1 ldv) (1 *)) v-%offset%)
               ldv)
            (declare (ignore var-1 var-2))
            (when var-0 (setf m var-0))
            (when var-3 (setf ldv var-3)))
         label20))))))
label30
 (cond
  ((or
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "P")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "B")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (tagbody
     (cond
      (rightv
       (tagbody
         (if (= ilo 1)
             (go label50))
         (f2cl-lib:fdo (i (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))
                        (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                       ((> i 1) nil)
           (tagbody
             (setf k
                     (f2cl-lib:int
                      (f2cl-lib:fref rscale-%data% (i) ((1 *))
                                     rscale-%offset%)))
             (if (= k i)
                 (go label40))
             (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                 (dswap m
                  (f2cl-lib:array-slice v-%data% double-float (i 1)
                                        ((1 ldv) (1 *)) v-%offset%)
                  ldv
                  (f2cl-lib:array-slice v-%data% double-float (k 1)
                                        ((1 ldv) (1 *)) v-%offset%)
                  ldv)
               (declare (ignore var-1 var-3))
```

```
                (when var-0 (setf m var-0))
                (when var-2 (setf ldv var-2))
                (when var-4 (setf ldv var-4)))
            label40))
        label50
         (if (= ihi n)
             (go label70))
         (f2cl-lib:fdo (i (f2cl-lib:int-add ihi 1) (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf k
                     (f2cl-lib:int
                      (f2cl-lib:fref rscale-%data% (i) ((1 *))
                                     rscale-%offset%)))
             (if (= k i)
                 (go label60))
             (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                 (dswap m
                  (f2cl-lib:array-slice v-%data% double-float (i 1)
                                        ((1 ldv) (1 *)) v-%offset%)
                  ldv
                  (f2cl-lib:array-slice v-%data% double-float (k 1)
                                        ((1 ldv) (1 *)) v-%offset%)
                  ldv)
               (declare (ignore var-1 var-3))
               (when var-0 (setf m var-0))
               (when var-2 (setf ldv var-2))
               (when var-4 (setf ldv var-4)))
            label60)))))
    label70
     (cond
      (leftv
       (tagbody
         (if (= ilo 1)
             (go label90))
         (f2cl-lib:fdo (i (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))
                          (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                       ((> i 1) nil)
           (tagbody
             (setf k
                     (f2cl-lib:int
                      (f2cl-lib:fref lscale-%data% (i) ((1 *))
                                     lscale-%offset%)))
             (if (= k i)
                 (go label80))
             (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                 (dswap m
                  (f2cl-lib:array-slice v-%data% double-float (i 1)
                                        ((1 ldv) (1 *)) v-%offset%)
                  ldv
```

```
                      (f2cl-lib:array-slice v-%data% double-float (k 1)
                                            ((1 ldv) (1 *)) v-%offset%)
                      ldv)
                    (declare (ignore var-1 var-3))
                    (when var-0 (setf m var-0))
                    (when var-2 (setf ldv var-2))
                    (when var-4 (setf ldv var-4)))
                 label80))
              label90
               (if (= ihi n)
                   (go label110))
               (f2cl-lib:fdo (i (f2cl-lib:int-add ihi 1) (f2cl-lib:int-add i 1))
                             ((> i n) nil)
                 (tagbody
                   (setf k
                           (f2cl-lib:int
                            (f2cl-lib:fref lscale-%data% (i) ((1 *))
                                           lscale-%offset%)))
                   (if (= k i)
                       (go label100))
                   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                       (dswap m
                        (f2cl-lib:array-slice v-%data% double-float (i 1)
                                              ((1 ldv) (1 *)) v-%offset%)
                        ldv
                        (f2cl-lib:array-slice v-%data% double-float (k 1)
                                              ((1 ldv) (1 *)) v-%offset%)
                        ldv)
                     (declare (ignore var-1 var-3))
                     (when var-0 (setf m var-0))
                     (when var-2 (setf ldv var-2))
                     (when var-4 (setf ldv var-4)))
                 label100))))))))
     label110
      (go end_label)
     end_label
      (return (values job side nil nil nil nil nil m nil ldv info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dggbak
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
```

```
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::job
  fortran-to-lisp::side nil nil nil
  nil nil fortran-to-lisp::m nil
  fortran-to-lisp::ldv
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

44.11 dggbal

44.11.1 Axiom unit tests

— dggbal.input —

```
)set break resume
)sys rm -f dggbal.output
)spool dggbal.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

44.11.2 Axiom help page

— dggbal.help —

```
=====================================================================
dggbal examples
=====================================================================

=====================================================================
Man Page Details
```

```
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dggbal (JOB, N, A, LDA, B, LDB, ILO, IHI, LSCALE, RSCALE,
           WORK, INFO)
           DGGBAL

Function/Subroutine Documentation
   subroutine dggbal (character JOB, integer N, double precision, dimension(
       lda, * ) A, integer LDA, double precision, dimension( ldb, * ) B,
       integer LDB, integer ILO, integer IHI, double precision, dimension( * )
       LSCALE, double precision, dimension( * ) RSCALE, double precision,
       dimension( * ) WORK, integer INFO)
       DGGBAL

Purpose:
   DGGBAL balances a pair of general real matrices (A,B).  This
   involves, first, permuting A and B by similarity transformations to
   isolate eigenvalues in the first 1 to ILO$-$1 and last IHI+1 to N
   elements on the diagonal; and second, applying a diagonal similarity
   transformation to rows and columns ILO to IHI to make the rows
   and columns as close in norm as possible. Both steps are optional.

   Balancing may reduce the 1-norm of the matrices, and improve the
   accuracy of the computed eigenvalues and/or eigenvectors in the
   generalized eigenvalue problem A*x = lambda*B*x.

Parameters:

 JOB
   JOB is CHARACTER*1
   Specifies the operations to be performed on A and B:
   = 'N':  none:  simply set ILO = 1, IHI = N, LSCALE(I) = 1.0
   and RSCALE(I) = 1.0 for i = 1,...,N.
   = 'P':  permute only;
   = 'S':  scale only;
   = 'B':  both permute and scale.

 N
   N is INTEGER
   The order of the matrices A and B.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the input matrix A.
   On exit,  A is overwritten by the balanced matrix.
   If JOB = 'N', A is not referenced.

 LDA
```

```
  LDA is INTEGER
  The leading dimension of the array A. LDA >= max(1,N).

B
  B is DOUBLE PRECISION array, dimension (LDB,N)
  On entry, the input matrix B.
  On exit,  B is overwritten by the balanced matrix.
  If JOB = 'N', B is not referenced.

LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= max(1,N).

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER
  ILO and IHI are set to integers such that on exit
  A(i,j) = 0 and B(i,j) = 0 if i > j and
  j = 1,...,ILO-1 or i = IHI+1,...,N.
  If JOB = 'N' or 'S', ILO = 1 and IHI = N.

LSCALE
  LSCALE is DOUBLE PRECISION array, dimension (N)
  Details of the permutations and scaling factors applied
  to the left side of A and B.  If P(j) is the index of the
  row interchanged with row j, and D(j)
  is the scaling factor applied to row j, then
  LSCALE(j) = P(j)    for J = 1,...,ILO-1
            = D(j)    for J = ILO,...,IHI
            = P(j)    for J = IHI+1,...,N.
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

RSCALE
  RSCALE is DOUBLE PRECISION array, dimension (N)
  Details of the permutations and scaling factors applied
  to the right side of A and B.  If P(j) is the index of the
  column interchanged with column j, and D(j)
  is the scaling factor applied to column j, then
  LSCALE(j) = P(j)    for J = 1,...,ILO-1
            = D(j)    for J = ILO,...,IHI
            = P(j)    for J = IHI+1,...,N.
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

WORK
  WORK is DOUBLE PRECISION array, dimension (lwork)
  lwork must be at least max(1,6*N) when JOB = 'S' or 'B', and
```

```
  at least 1 when JOB = 'N' or 'P'.

 INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

44.11.3 fortran code

— dggbal.f —

```
*  =====================================================================
      SUBROUTINE DGGBAL( JOB, N, A, LDA, B, LDB, ILO, IHI, LSCALE,
     $                   RSCALE, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOB
      INTEGER            IHI, ILO, INFO, LDA, LDB, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * ), LSCALE( * ),
     $                   RSCALE( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, HALF, ONE
      PARAMETER          ( ZERO = 0.0D+0, HALF = 0.5D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   THREE, SCLFAC
      PARAMETER          ( THREE = 3.0D+0, SCLFAC = 1.0D+1 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, ICAB, IFLOW, IP1, IR, IRAB, IT, J, JC, JP1,
     $                   K, KOUNT, L, LCAB, LM1, LRAB, LSFMAX, LSFMIN,
     $                   M, NR, NRP2
      DOUBLE PRECISION   ALPHA, BASL, BETA, CAB, CMAX, COEF, COEF2,
     $                   COEF5, COR, EW, EWC, GAMMA, PGAMMA, RAB, SFMAX,
     $                   SFMIN, SUM, T, TA, TB, TC
*     ..
```

```
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX
      DOUBLE PRECISION   DDOT, DLAMCH
      EXTERNAL           LSAME, IDAMAX, DDOT, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DSCAL, DSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, INT, LOG10, MAX, MIN, SIGN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGGBAL', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         ILO = 1
         IHI = N
         RETURN
      END IF
*
      IF( N.EQ.1 ) THEN
         ILO = 1
         IHI = N
         LSCALE( 1 ) = ONE
         RSCALE( 1 ) = ONE
         RETURN
      END IF
*
      IF( LSAME( JOB, 'N' ) ) THEN
         ILO = 1
```

```
         IHI = N
         DO 10 I = 1, N
            LSCALE( I ) = ONE
            RSCALE( I ) = ONE
   10    CONTINUE
         RETURN
      END IF
*
      K = 1
      L = N
      IF( LSAME( JOB, 'S' ) )
     $   GO TO 190
*
      GO TO 30
*
*     Permute the matrices A and B to isolate the eigenvalues.
*
*     Find row with one nonzero in columns 1 through L
*
   20 CONTINUE
      L = LM1
      IF( L.NE.1 )
     $   GO TO 30
*
      RSCALE( 1 ) = ONE
      LSCALE( 1 ) = ONE
      GO TO 190
*
   30 CONTINUE
      LM1 = L - 1
      DO 80 I = L, 1, -1
         DO 40 J = 1, LM1
            JP1 = J + 1
            IF( A( I, J ).NE.ZERO .OR. B( I, J ).NE.ZERO )
     $         GO TO 50
   40    CONTINUE
         J = L
         GO TO 70
*
   50    CONTINUE
         DO 60 J = JP1, L
            IF( A( I, J ).NE.ZERO .OR. B( I, J ).NE.ZERO )
     $         GO TO 80
   60    CONTINUE
         J = JP1 - 1
*
   70    CONTINUE
         M = L
         IFLOW = 1
         GO TO 160
```

```
   80 CONTINUE
      GO TO 100
*
*     Find column with one nonzero in rows K through N
*
   90 CONTINUE
      K = K + 1
*
  100 CONTINUE
      DO 150 J = K, L
         DO 110 I = K, LM1
            IP1 = I + 1
            IF( A( I, J ).NE.ZERO .OR. B( I, J ).NE.ZERO )
     $         GO TO 120
  110    CONTINUE
         I = L
         GO TO 140
  120    CONTINUE
         DO 130 I = IP1, L
            IF( A( I, J ).NE.ZERO .OR. B( I, J ).NE.ZERO )
     $         GO TO 150
  130    CONTINUE
         I = IP1 - 1
  140    CONTINUE
         M = K
         IFLOW = 2
         GO TO 160
  150 CONTINUE
      GO TO 190
*
*     Permute rows M and I
*
  160 CONTINUE
      LSCALE( M ) = I
      IF( I.EQ.M )
     $   GO TO 170
      CALL DSWAP( N-K+1, A( I, K ), LDA, A( M, K ), LDA )
      CALL DSWAP( N-K+1, B( I, K ), LDB, B( M, K ), LDB )
*
*     Permute columns M and J
*
  170 CONTINUE
      RSCALE( M ) = J
      IF( J.EQ.M )
     $   GO TO 180
      CALL DSWAP( L, A( 1, J ), 1, A( 1, M ), 1 )
      CALL DSWAP( L, B( 1, J ), 1, B( 1, M ), 1 )
*
  180 CONTINUE
      GO TO ( 20, 90 )IFLOW
```

```
*
  190 CONTINUE
      ILO = K
      IHI = L
*
      IF( LSAME( JOB, 'P' ) ) THEN
         DO 195 I = ILO, IHI
            LSCALE( I ) = ONE
            RSCALE( I ) = ONE
  195    CONTINUE
         RETURN
      END IF
*
      IF( ILO.EQ.IHI )
     $   RETURN
*
*     Balance the submatrix in rows ILO to IHI.
*
      NR = IHI - ILO + 1
      DO 200 I = ILO, IHI
         RSCALE( I ) = ZERO
         LSCALE( I ) = ZERO
*
         WORK( I ) = ZERO
         WORK( I+N ) = ZERO
         WORK( I+2*N ) = ZERO
         WORK( I+3*N ) = ZERO
         WORK( I+4*N ) = ZERO
         WORK( I+5*N ) = ZERO
  200 CONTINUE
*
*     Compute right side vector in resulting linear equations
*
      BASL = LOG10( SCLFAC )
      DO 240 I = ILO, IHI
         DO 230 J = ILO, IHI
            TB = B( I, J )
            TA = A( I, J )
            IF( TA.EQ.ZERO )
     $         GO TO 210
            TA = LOG10( ABS( TA ) ) / BASL
  210       CONTINUE
            IF( TB.EQ.ZERO )
     $         GO TO 220
            TB = LOG10( ABS( TB ) ) / BASL
  220       CONTINUE
            WORK( I+4*N ) = WORK( I+4*N ) - TA - TB
            WORK( J+5*N ) = WORK( J+5*N ) - TA - TB
  230    CONTINUE
  240 CONTINUE
```

```
*
      COEF = ONE / DBLE( 2*NR )
      COEF2 = COEF*COEF
      COEF5 = HALF*COEF2
      NRP2 = NR + 2
      BETA = ZERO
      IT = 1
*
*     Start generalized conjugate gradient iteration
*
  250 CONTINUE
*
      GAMMA = DDOT( NR, WORK( ILO+4*N ), 1, WORK( ILO+4*N ), 1 ) +
     $        DDOT( NR, WORK( ILO+5*N ), 1, WORK( ILO+5*N ), 1 )
*
      EW = ZERO
      EWC = ZERO
      DO 260 I = ILO, IHI
         EW = EW + WORK( I+4*N )
         EWC = EWC + WORK( I+5*N )
  260 CONTINUE
*
      GAMMA = COEF*GAMMA - COEF2*( EW**2+EWC**2 ) - COEF5*( EW-EWC )**2
      IF( GAMMA.EQ.ZERO )
     $   GO TO 350
      IF( IT.NE.1 )
     $   BETA = GAMMA / PGAMMA
      T = COEF5*( EWC-THREE*EW )
      TC = COEF5*( EW-THREE*EWC )
*
      CALL DSCAL( NR, BETA, WORK( ILO ), 1 )
      CALL DSCAL( NR, BETA, WORK( ILO+N ), 1 )
*
      CALL DAXPY( NR, COEF, WORK( ILO+4*N ), 1, WORK( ILO+N ), 1 )
      CALL DAXPY( NR, COEF, WORK( ILO+5*N ), 1, WORK( ILO ), 1 )
*
      DO 270 I = ILO, IHI
         WORK( I ) = WORK( I ) + TC
         WORK( I+N ) = WORK( I+N ) + T
  270 CONTINUE
*
*     Apply matrix to vector
*
      DO 300 I = ILO, IHI
         KOUNT = 0
         SUM = ZERO
         DO 290 J = ILO, IHI
            IF( A( I, J ).EQ.ZERO )
     $         GO TO 280
            KOUNT = KOUNT + 1
```

```
            SUM = SUM + WORK( J )
  280       CONTINUE
            IF( B( I, J ).EQ.ZERO )
     $         GO TO 290
            KOUNT = KOUNT + 1
            SUM = SUM + WORK( J )
  290    CONTINUE
         WORK( I+2*N ) = DBLE( KOUNT )*WORK( I+N ) + SUM
  300 CONTINUE
*
      DO 330 J = ILO, IHI
         KOUNT = 0
         SUM = ZERO
         DO 320 I = ILO, IHI
            IF( A( I, J ).EQ.ZERO )
     $         GO TO 310
            KOUNT = KOUNT + 1
            SUM = SUM + WORK( I+N )
  310       CONTINUE
            IF( B( I, J ).EQ.ZERO )
     $         GO TO 320
            KOUNT = KOUNT + 1
            SUM = SUM + WORK( I+N )
  320    CONTINUE
         WORK( J+3*N ) = DBLE( KOUNT )*WORK( J ) + SUM
  330 CONTINUE
*
      SUM = DDOT( NR, WORK( ILO+N ), 1, WORK( ILO+2*N ), 1 ) +
     $      DDOT( NR, WORK( ILO ), 1, WORK( ILO+3*N ), 1 )
      ALPHA = GAMMA / SUM
*
*     Determine correction to current iteration
*
      CMAX = ZERO
      DO 340 I = ILO, IHI
         COR = ALPHA*WORK( I+N )
         IF( ABS( COR ).GT.CMAX )
     $      CMAX = ABS( COR )
         LSCALE( I ) = LSCALE( I ) + COR
         COR = ALPHA*WORK( I )
         IF( ABS( COR ).GT.CMAX )
     $      CMAX = ABS( COR )
         RSCALE( I ) = RSCALE( I ) + COR
  340 CONTINUE
      IF( CMAX.LT.HALF )
     $   GO TO 350
*
      CALL DAXPY( NR, -ALPHA, WORK( ILO+2*N ), 1, WORK( ILO+4*N ), 1 )
      CALL DAXPY( NR, -ALPHA, WORK( ILO+3*N ), 1, WORK( ILO+5*N ), 1 )
*
```

```
      PGAMMA = GAMMA
      IT = IT + 1
      IF( IT.LE.NRP2 )
     $   GO TO 250
*
*     End generalized conjugate gradient iteration
*
  350 CONTINUE
      SFMIN = DLAMCH( 'S' )
      SFMAX = ONE / SFMIN
      LSFMIN = INT( LOG10( SFMIN ) / BASL+ONE )
      LSFMAX = INT( LOG10( SFMAX ) / BASL )
      DO 360 I = ILO, IHI
         IRAB = IDAMAX( N-ILO+1, A( I, ILO ), LDA )
         RAB = ABS( A( I, IRAB+ILO-1 ) )
         IRAB = IDAMAX( N-ILO+1, B( I, ILO ), LDB )
         RAB = MAX( RAB, ABS( B( I, IRAB+ILO-1 ) ) )
         LRAB = INT( LOG10( RAB+SFMIN ) / BASL+ONE )
         IR = INT(LSCALE( I ) + SIGN( HALF, LSCALE( I ) ))
         IR = MIN( MAX( IR, LSFMIN ), LSFMAX, LSFMAX-LRAB )
         LSCALE( I ) = SCLFAC**IR
         ICAB = IDAMAX( IHI, A( 1, I ), 1 )
         CAB = ABS( A( ICAB, I ) )
         ICAB = IDAMAX( IHI, B( 1, I ), 1 )
         CAB = MAX( CAB, ABS( B( ICAB, I ) ) )
         LCAB = INT( LOG10( CAB+SFMIN ) / BASL+ONE )
         JC = INT(RSCALE( I ) + SIGN( HALF, RSCALE( I ) ))
         JC = MIN( MAX( JC, LSFMIN ), LSFMAX, LSFMAX-LCAB )
         RSCALE( I ) = SCLFAC**JC
  360 CONTINUE
*
*     Row scaling of matrices A and B
*
      DO 370 I = ILO, IHI
         CALL DSCAL( N-ILO+1, LSCALE( I ), A( I, ILO ), LDA )
         CALL DSCAL( N-ILO+1, LSCALE( I ), B( I, ILO ), LDB )
  370 CONTINUE
*
*     Column scaling of matrices A and B
*
      DO 380 J = ILO, IHI
         CALL DSCAL( IHI, RSCALE( J ), A( 1, J ), 1 )
         CALL DSCAL( IHI, RSCALE( J ), B( 1, J ), 1 )
  380 CONTINUE
*
      RETURN
*
*     End of DGGBAL
*
      END
```

———-

44.11.4 lisp code

— dggbal.lisp —

```
(let* ((zero 0.0d0) (half 0.5d0) (one 1.0d0) (three 3.0d0) (sclfac 10.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 0.5d0 0.5d0) half)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 3.0d0 3.0d0) three)
           (type (double-float 10.0d0 10.0d0) sclfac)
           (ignorable zero half one three sclfac))
  (defun dggbal (job n a lda b ldb$ ilo ihi lscale rscale work info)
    (declare (type (array double-float (*)) work rscale lscale b a)
             (type (f2cl-lib:integer4) info ihi ilo ldb$ lda n)
             (type (string *) job))
    (f2cl-lib:with-multi-array-data
        ((job character job-%data% job-%offset%)
         (a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%)
         (lscale double-float lscale-%data% lscale-%offset%)
         (rscale double-float rscale-%data% rscale-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((alpha 0.0d0) (basl 0.0d0) (beta 0.0d0) (cab 0.0d0) (cmax 0.0d0)
             (coef 0.0d0) (coef2 0.0d0) (coef5 0.0d0) (cor 0.0d0) (ew 0.0d0)
             (ewc 0.0d0) (gamma 0.0d0) (pgamma 0.0d0) (rab 0.0d0) (sfmax 0.0d0)
             (sfmin 0.0d0) (sum 0.0d0) (t$ 0.0d0) (ta 0.0d0) (tb 0.0d0)
             (tc 0.0d0) (i 0) (icab 0) (iflow 0) (ip1 0) (ir 0) (irab 0) (it 0)
             (j 0) (jc 0) (jp1 0) (k 0) (kount 0) (l 0) (lcab 0) (lm1 0)
             (lrab 0) (lsfmax 0) (lsfmin 0) (m 0) (nr 0) (nrp2 0))
        (declare
         (type (double-float) alpha basl beta cab cmax coef coef2 coef5 cor ew
          ewc gamma pgamma rab sfmax sfmin sum t$ ta tb tc)
         (type (f2cl-lib:integer4) i icab iflow ip1 ir irab it j jc jp1 k kount
          l lcab lm1 lrab lsfmax lsfmin m nr nrp2))
        (setf info 0)
        (cond
         ((and
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame job "N")
              (declare (ignore var-1))
              (when var-0 (setf job var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame job "P")
```

```
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "S")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "B")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6)))
(cond
 ((/= info 0) (xerbla "DGGBAL" (f2cl-lib:int-sub info))
  (go end_label)))
(cond ((= n 0) (setf ilo 1) (setf ihi n) (go end_label)))
(cond
 ((= n 1) (setf ilo 1) (setf ihi n)
  (setf (f2cl-lib:fref lscale-%data% (1) ((1 *)) lscale-%offset%) one)
  (setf (f2cl-lib:fref rscale-%data% (1) ((1 *)) rscale-%offset%) one)
  (go end_label)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame job "N")
    (declare (ignore var-1))
    (when var-0 (setf job var-0))
    ret-val)
  (setf ilo 1) (setf ihi n)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
              one)
      (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
              one)
     label10))
  (go end_label)))
(setf k 1)
(setf l n)
(if (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "S")
```

```
        (declare (ignore var-1))
        (when var-0 (setf job var-0))
        ret-val)
      (go label190))
  (go label30)
 label20
  (setf l lm1)
  (if (/= l 1)
      (go label30))
  (setf (f2cl-lib:fref rscale-%data% (1) ((1 *)) rscale-%offset%) one)
  (setf (f2cl-lib:fref lscale-%data% (1) ((1 *)) lscale-%offset%) one)
  (go label190)
 label30
  (setf lm1 (f2cl-lib:int-sub l 1))
  (f2cl-lib:fdo (i l (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                ((> i 1) nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j lm1) nil)
        (tagbody
          (setf jp1 (f2cl-lib:int-add j 1))
          (if (or
               (/=
                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                zero)
               (/=
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)
                zero))
              (go label50))
        label40))
      (setf j l)
      (go label70)
     label50
      (f2cl-lib:fdo (j jp1 (f2cl-lib:int-add j 1))
                    ((> j l) nil)
        (tagbody
          (if (or
               (/=
                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                zero)
               (/=
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)
                zero))
              (go label80))
        label60))
      (setf j (f2cl-lib:int-sub jp1 1))
     label70
      (setf m l)
```

```
     (setf iflow 1)
     (go label160)
    label80))
 (go label100)
label90
 (setf k (f2cl-lib:int-add k 1))
label100
 (f2cl-lib:fdo (j k (f2cl-lib:int-add j 1))
               ((> j l) nil)
   (tagbody
     (f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
                   ((> i lm1) nil)
       (tagbody
         (setf ip1 (f2cl-lib:int-add i 1))
         (if (or
              (/=
               (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
               zero)
              (/=
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)
               zero))
             (go label120))
        label110))
     (setf i l)
     (go label140)
    label120
     (f2cl-lib:fdo (i ip1 (f2cl-lib:int-add i 1))
                   ((> i l) nil)
       (tagbody
         (if (or
              (/=
               (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
               zero)
              (/=
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)
               zero))
             (go label150))
        label130))
     (setf i (f2cl-lib:int-sub ip1 1))
    label140
     (setf m k)
     (setf iflow 2)
     (go label160)
    label150))
 (go label190)
label160
 (setf (f2cl-lib:fref lscale-%data% (m) ((1 *)) lscale-%offset%)
        (coerce (the f2cl-lib:integer4 i) 'double-float))
```

```
 (if (= i m)
     (go label170))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (dswap (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
      (f2cl-lib:array-slice a-%data% double-float (i k) ((1 lda) (1 *))
                            a-%offset%)
      lda
      (f2cl-lib:array-slice a-%data% double-float (m k) ((1 lda) (1 *))
                            a-%offset%)
      lda)
   (declare (ignore var-0 var-1 var-3))
   (when var-2 (setf lda var-2))
   (when var-4 (setf lda var-4)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (dswap (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
      (f2cl-lib:array-slice b-%data% double-float (i k) ((1 ldb$) (1 *))
                            b-%offset%)
      ldb$
      (f2cl-lib:array-slice b-%data% double-float (m k) ((1 ldb$) (1 *))
                            b-%offset%)
      ldb$)
   (declare (ignore var-0 var-1 var-3))
   (when var-2 (setf ldb$ var-2))
   (when var-4 (setf ldb$ var-4)))
label170
 (setf (f2cl-lib:fref rscale-%data% (m) ((1 *)) rscale-%offset%)
         (coerce (the f2cl-lib:integer4 j) 'double-float))
 (if (= j m)
     (go label180))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (dswap l
      (f2cl-lib:array-slice a-%data% double-float (1 j) ((1 lda) (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice a-%data% double-float (1 m) ((1 lda) (1 *))
                            a-%offset%)
      1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf l var-0)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (dswap l
      (f2cl-lib:array-slice b-%data% double-float (1 j) ((1 ldb$) (1 *))
                            b-%offset%)
      1
      (f2cl-lib:array-slice b-%data% double-float (1 m) ((1 ldb$) (1 *))
                            b-%offset%)
      1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf l var-0)))
label180
```

```
 (f2cl-lib:computed-goto (label20 label90) iflow)
label190
 (setf ilo k)
 (setf ihi l)
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame job "P")
     (declare (ignore var-1))
     (when var-0 (setf job var-0))
     ret-val)
   (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                 ((> i ihi) nil)
     (tagbody
       (setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
               one)
       (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
               one)
      label195))
   (go end_label)))
 (if (= ilo ihi)
     (go end_label))
 (setf nr (f2cl-lib:int-add (f2cl-lib:int-sub ihi ilo) 1))
 (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
               ((> i ihi) nil)
   (tagbody
     (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
             zero)
     (setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
             zero)
     (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) zero)
     (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n)) ((1 *))
                          work-%offset%)
             zero)
     (setf (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add i (f2cl-lib:int-mul 2 n)))
                          ((1 *)) work-%offset%)
             zero)
     (setf (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add i (f2cl-lib:int-mul 3 n)))
                          ((1 *)) work-%offset%)
             zero)
     (setf (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add i (f2cl-lib:int-mul 4 n)))
                          ((1 *)) work-%offset%)
             zero)
     (setf (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add i (f2cl-lib:int-mul 5 n)))
                          ((1 *)) work-%offset%)
             zero)
    label200))
```

```
(setf basl (f2cl-lib:log10 sclfac))
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i ihi) nil)
  (tagbody
    (f2cl-lib:fdo (j ilo (f2cl-lib:int-add j 1))
                  ((> j ihi) nil)
      (tagbody
        (setf tb
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%))
        (setf ta
                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%))
        (if (= ta zero)
            (go label210))
        (setf ta (/ (f2cl-lib:log10 (abs ta)) basl))
       label210
        (if (= tb zero)
            (go label220))
        (setf tb (/ (f2cl-lib:log10 (abs tb)) basl))
       label220
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add i
                                                (f2cl-lib:int-mul 4
                                                                  n)))
                             ((1 *)) work-%offset%)
                (-
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add i
                                                   (f2cl-lib:int-mul 4
                                                                     n)))
                                ((1 *)) work-%offset%)
                 ta tb))
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add j
                                                (f2cl-lib:int-mul 5
                                                                  n)))
                             ((1 *)) work-%offset%)
                (-
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add j
                                                   (f2cl-lib:int-mul 5
                                                                     n)))
                                ((1 *)) work-%offset%)
                 ta tb))
       label230))
   label240))
(setf coef (/ one (f2cl-lib:dble (f2cl-lib:int-mul 2 nr))))
(setf coef2 (* coef coef))
(setf coef5 (* half coef2))
```

```
 (setf nrp2 (f2cl-lib:int-add nr 2))
 (setf beta zero)
 (setf it 1)
label250
 (setf gamma
         (+
          (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
              (ddot nr
               (f2cl-lib:array-slice work-%data% double-float
                                     ((+ ilo (f2cl-lib:int-mul 4 n)))
                                     ((1 *)) work-%offset%)
               1
               (f2cl-lib:array-slice work-%data% double-float
                                     ((+ ilo (f2cl-lib:int-mul 4 n)))
                                     ((1 *)) work-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf nr var-0))
            ret-val)
          (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
              (ddot nr
               (f2cl-lib:array-slice work-%data% double-float
                                     ((+ ilo (f2cl-lib:int-mul 5 n)))
                                     ((1 *)) work-%offset%)
               1
               (f2cl-lib:array-slice work-%data% double-float
                                     ((+ ilo (f2cl-lib:int-mul 5 n)))
                                     ((1 *)) work-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf nr var-0))
            ret-val)))
 (setf ew zero)
 (setf ewc zero)
 (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
               ((> i ihi) nil)
   (tagbody
     (setf ew
             (+ ew
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add i
                                                  (f2cl-lib:int-mul 4
                                                                    n)))
                               ((1 *)) work-%offset%)))
     (setf ewc
             (+ ewc
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add i
                                                  (f2cl-lib:int-mul 5
                                                                    n)))
```

```
                                ((1 *)) work-%offset%)))
   label260))
(setf gamma
        (- (* coef gamma) (* coef2 (+ (expt ew 2) (expt ewc 2)))
           (* coef5 (expt (- ew ewc) 2))))
(if (= gamma zero)
    (go label350))
(if (/= it 1)
    (setf beta (/ gamma pgamma)))
(setf t$ (* coef5 (- ewc (* three ew))))
(setf tc (* coef5 (- ew (* three ewc))))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (dscal nr beta
     (f2cl-lib:array-slice work-%data% double-float (ilo) ((1 *))
                           work-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf nr var-0))
  (when var-1 (setf beta var-1)))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (dscal nr beta
     (f2cl-lib:array-slice work-%data% double-float ((+ ilo n)) ((1 *))
                           work-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf nr var-0))
  (when var-1 (setf beta var-1)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (daxpy nr coef
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ ilo (f2cl-lib:int-mul 4 n))) ((1 *))
                           work-%offset%)
     1
     (f2cl-lib:array-slice work-%data% double-float ((+ ilo n)) ((1 *))
                           work-%offset%)
     1)
  (declare (ignore var-2 var-3 var-4 var-5))
  (when var-0 (setf nr var-0))
  (when var-1 (setf coef var-1)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (daxpy nr coef
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ ilo (f2cl-lib:int-mul 5 n))) ((1 *))
                           work-%offset%)
     1
     (f2cl-lib:array-slice work-%data% double-float (ilo) ((1 *))
                           work-%offset%)
     1)
  (declare (ignore var-2 var-3 var-4 var-5))
  (when var-0 (setf nr var-0))
```

```
  (when var-1 (setf coef var-1)))
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i ihi) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (+ (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               tc))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n)) ((1 *))
                         work-%offset%)
            (+
             (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n))
                            ((1 *)) work-%offset%)
             t$))
   label270))
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i ihi) nil)
  (tagbody
    (setf kount 0)
    (setf sum zero)
    (f2cl-lib:fdo (j ilo (f2cl-lib:int-add j 1))
                  ((> j ihi) nil)
      (tagbody
        (if (=
             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
             zero)
            (go label280))
        (setf kount (f2cl-lib:int-add kount 1))
        (setf sum
                (+ sum
                   (f2cl-lib:fref work-%data% (j) ((1 *))
                                  work-%offset%)))
       label280
        (if (=
             (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *)) b-%offset%)
             zero)
            (go label290))
        (setf kount (f2cl-lib:int-add kount 1))
        (setf sum
                (+ sum
                   (f2cl-lib:fref work-%data% (j) ((1 *))
                                  work-%offset%)))
       label290))
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add i (f2cl-lib:int-mul 2 n)))
                         ((1 *)) work-%offset%)
            (+
             (* (f2cl-lib:dble kount)
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n))
                               ((1 *)) work-%offset%))
             sum))
```

```
   label300))
(f2cl-lib:fdo (j ilo (f2cl-lib:int-add j 1))
              ((> j ihi) nil)
  (tagbody
    (setf kount 0)
    (setf sum zero)
    (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                  ((> i ihi) nil)
      (tagbody
        (if (=
             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
             zero)
            (go label310))
        (setf kount (f2cl-lib:int-add kount 1))
        (setf sum
                (+ sum
                   (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n))
                                  ((1 *)) work-%offset%)))
       label310
        (if (=
             (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *)) b-%offset%)
             zero)
            (go label320))
        (setf kount (f2cl-lib:int-add kount 1))
        (setf sum
                (+ sum
                   (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n))
                                  ((1 *)) work-%offset%)))
       label320))
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add j (f2cl-lib:int-mul 3 n)))
                         ((1 *)) work-%offset%)
            (+
             (* (f2cl-lib:dble kount)
                (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%))
             sum))
   label330))
(setf sum
        (+
         (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
             (ddot nr
              (f2cl-lib:array-slice work-%data% double-float
                                    ((+ ilo n)) ((1 *)) work-%offset%)
              1
              (f2cl-lib:array-slice work-%data% double-float
                                    ((+ ilo (f2cl-lib:int-mul 2 n)))
                                    ((1 *)) work-%offset%)
              1)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf nr var-0))
```

```
            ret-val)
          (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
              (ddot nr
               (f2cl-lib:array-slice work-%data% double-float (ilo)
                                     ((1 *)) work-%offset%)
               1
               (f2cl-lib:array-slice work-%data% double-float
                                     ((+ ilo (f2cl-lib:int-mul 3 n)))
                                     ((1 *)) work-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf nr var-0))
            ret-val)))
(setf alpha (/ gamma sum))
(setf cmax zero)
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i ihi) nil)
  (tagbody
    (setf cor
            (* alpha
               (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n))
                              ((1 *)) work-%offset%)))
    (if (> (abs cor) cmax)
        (setf cmax (abs cor)))
    (setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
            (+
             (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
             cor))
    (setf cor
            (* alpha
               (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)))
    (if (> (abs cor) cmax)
        (setf cmax (abs cor)))
    (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
            (+
             (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
             cor))
   label340))
(if (< cmax half)
    (go label350))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (daxpy nr (- alpha)
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ ilo (f2cl-lib:int-mul 2 n))) ((1 *))
                           work-%offset%)
     1
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ ilo (f2cl-lib:int-mul 4 n))) ((1 *))
                           work-%offset%)
     1)
```

```
    (declare (ignore var-1 var-2 var-3 var-4 var-5))
    (when var-0 (setf nr var-0)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (daxpy nr (- alpha)
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ ilo (f2cl-lib:int-mul 3 n))) ((1 *))
                             work-%offset%)
       1
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ ilo (f2cl-lib:int-mul 5 n))) ((1 *))
                             work-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4 var-5))
    (when var-0 (setf nr var-0)))
  (setf pgamma gamma)
  (setf it (f2cl-lib:int-add it 1))
  (if (<= it nrp2)
      (go label250))
 label350
  (setf sfmin (dlamch "S"))
  (setf sfmax (/ one sfmin))
  (setf lsfmin (f2cl-lib:int (+ (/ (f2cl-lib:log10 sfmin) basl) one)))
  (setf lsfmax (f2cl-lib:int (/ (f2cl-lib:log10 sfmax) basl)))
  (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                ((> i ihi) nil)
    (tagbody
      (setf irab
              (multiple-value-bind (ret-val var-0 var-1 var-2)
                  (idamax (f2cl-lib:int-add (f2cl-lib:int-sub n ilo) 1)
                   (f2cl-lib:array-slice a-%data% double-float (i ilo)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda)
                (declare (ignore var-0 var-1))
                (when var-2 (setf lda var-2))
                ret-val))
      (setf rab
              (abs
               (f2cl-lib:fref a-%data%
                              (i
                               (f2cl-lib:int-sub
                                (f2cl-lib:int-add irab ilo) 1))
                              ((1 lda) (1 *)) a-%offset%)))
      (setf irab
              (multiple-value-bind (ret-val var-0 var-1 var-2)
                  (idamax (f2cl-lib:int-add (f2cl-lib:int-sub n ilo) 1)
                   (f2cl-lib:array-slice b-%data% double-float (i ilo)
                                         ((1 ldb$) (1 *)) b-%offset%)
                   ldb$)
                (declare (ignore var-0 var-1))
                (when var-2 (setf ldb$ var-2))
```

```
          ret-val))
(setf rab
        (max rab
             (abs
              (f2cl-lib:fref b-%data%
                             (i
                              (f2cl-lib:int-sub
                               (f2cl-lib:int-add irab ilo) 1))
                             ((1 ldb$) (1 *)) b-%offset%))))
(setf lrab
        (f2cl-lib:int
         (+ (/ (f2cl-lib:log10 (+ rab sfmin)) basl) one)))
(setf ir
        (f2cl-lib:int
         (+
          (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
          (f2cl-lib:sign half
                         (f2cl-lib:fref lscale-%data% (i) ((1 *))
                                        lscale-%offset%)))))
(setf ir
        (min
         (the f2cl-lib:integer4
              (max (the f2cl-lib:integer4 ir)
                   (the f2cl-lib:integer4 lsfmin)))
         (the f2cl-lib:integer4 lsfmax)
         (the f2cl-lib:integer4 (f2cl-lib:int-sub lsfmax lrab))))
(setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
        (expt sclfac ir))
(setf icab
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (idamax ihi
             (f2cl-lib:array-slice a-%data% double-float (1 i)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf ihi var-0))
          ret-val))
(setf cab
        (abs
         (f2cl-lib:fref a-%data% (icab i) ((1 lda) (1 *))
                        a-%offset%)))
(setf icab
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (idamax ihi
             (f2cl-lib:array-slice b-%data% double-float (1 i)
                                   ((1 ldb$) (1 *)) b-%offset%)
             1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf ihi var-0))
          ret-val))
```

```
    (setf cab
            (max cab
                 (abs
                  (f2cl-lib:fref b-%data% (icab i) ((1 ldb$) (1 *))
                                 b-%offset%))))
    (setf lcab
            (f2cl-lib:int
             (+ (/ (f2cl-lib:log10 (+ cab sfmin)) basl) one)))
    (setf jc
            (f2cl-lib:int
             (+
              (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
              (f2cl-lib:sign half
                             (f2cl-lib:fref rscale-%data% (i) ((1 *))
                                            rscale-%offset%)))))
    (setf jc
            (min
             (the f2cl-lib:integer4
                  (max (the f2cl-lib:integer4 jc)
                       (the f2cl-lib:integer4 lsfmin)))
             (the f2cl-lib:integer4 lsfmax)
             (the f2cl-lib:integer4 (f2cl-lib:int-sub lsfmax lcab))))
    (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
            (expt sclfac jc))
   label360))
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i ihi) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal (f2cl-lib:int-add (f2cl-lib:int-sub n ilo) 1)
         (f2cl-lib:array-slice lscale-%data% double-float (i) ((1 *))
                               lscale-%offset%)
         (f2cl-lib:array-slice a-%data% double-float (i ilo)
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-0 var-1 var-2))
      (when var-3 (setf lda var-3)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal (f2cl-lib:int-add (f2cl-lib:int-sub n ilo) 1)
         (f2cl-lib:array-slice lscale-%data% double-float (i) ((1 *))
                               lscale-%offset%)
         (f2cl-lib:array-slice b-%data% double-float (i ilo)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-0 var-1 var-2))
      (when var-3 (setf ldb$ var-3)))
   label370))
(f2cl-lib:fdo (j ilo (f2cl-lib:int-add j 1))
              ((> j ihi) nil)
  (tagbody
```

```
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (dscal ihi
                 (f2cl-lib:array-slice rscale-%data% double-float (j) ((1 *))
                                       rscale-%offset%)
                 (f2cl-lib:array-slice a-%data% double-float (1 j)
                                       ((1 lda) (1 *)) a-%offset%)
                 1)
              (declare (ignore var-1 var-2 var-3))
              (when var-0 (setf ihi var-0)))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (dscal ihi
                 (f2cl-lib:array-slice rscale-%data% double-float (j) ((1 *))
                                       rscale-%offset%)
                 (f2cl-lib:array-slice b-%data% double-float (1 j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 1)
              (declare (ignore var-1 var-2 var-3))
              (when var-0 (setf ihi var-0)))
          label380))
       (go end_label)
      end_label
       (return (values job nil nil lda nil ldb$ ilo ihi nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dggbal
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string)
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (array double-float (*))
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(fortran-to-lisp::job nil nil
                                             fortran-to-lisp::lda nil
                                             fortran-to-lisp::ldb$
                                             fortran-to-lisp::ilo
                                             fortran-to-lisp::ihi nil nil nil
                                             fortran-to-lisp::info)
                                           :calls 'nil)))
```

———-

44.12 dla_gbamv

44.12.1 Axiom unit tests

— **dlagbamv.input** —

```
)set break resume
)sys rm -f dlagbamv.output
)spool dlagbamv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

44.12.2 Axiom help page

— **dla_gbamv.help** —

```
====================================================================
dla_gbamv examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dla_gbamv (TRANS, M, N, KL, KU, ALPHA, AB, LDAB, X, INCX,
           BETA, Y, INCY)
           DLA_GBAMV performs a matrix-vector operation to calculate error
           bounds.

Function/Subroutine Documentation
   subroutine dla_gbamv (integer TRANS, integer M, integer N, integer KL,
       integer KU, double precision ALPHA, double precision, dimension( ldab,
```

```
      * ) AB, integer LDAB, double precision, dimension( * ) X, integer INCX,
      double precision BETA, double precision, dimension( * ) Y, integer
      INCY)
      DLA_GBAMV performs a matrix-vector operation to calculate error bounds.

Purpose:
   DLA_GBAMV  performs one of the matrix-vector operations

        y := alpha*abs(A)*abs(x) + beta*abs(y),
   or   y := alpha*abs(A)**T*abs(x) + beta*abs(y),

   where alpha and beta are scalars, x and y are vectors and A is an
   m by n matrix.

   This function is primarily used in calculating error bounds.
   To protect against underflow during evaluation, components in
   the resulting vector are perturbed away from zero by (N+1)
   times the underflow threshold.  To prevent unnecessarily large
   errors for block-structure embedded in general matrices,
   "symbolically" zero components are not perturbed.  A zero
   entry is considered "symbolic" if all multiplications involved
   in computing that entry have at least one zero multiplicand.

Parameters:

 TRANS
   TRANS is INTEGER
   On entry, TRANS specifies the operation to be performed as
   follows:

   BLAS_NO_TRANS      y := alpha*abs(A)*abs(x) + beta*abs(y)
   BLAS_TRANS         y := alpha*abs(A**T)*abs(x) + beta*abs(y)
   BLAS_CONJ_TRANS    y := alpha*abs(A**T)*abs(x) + beta*abs(y)

   Unchanged on exit.

 M
   M is INTEGER
   On entry, M specifies the number of rows of the matrix A.
   M must be at least zero.
   Unchanged on exit.

 N
   N is INTEGER
   On entry, N specifies the number of columns of the matrix A.
   N must be at least zero.
   Unchanged on exit.

 KL
   KL is INTEGER
```

```
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

ALPHA
  ALPHA is DOUBLE PRECISION
  On entry, ALPHA specifies the scalar alpha.
  Unchanged on exit.

AB
  AB is DOUBLE PRECISION array of DIMENSION ( LDAB, n )
  Before entry, the leading m by n part of the array AB must
  contain the matrix of coefficients.
  Unchanged on exit.

LDAB
  LDAB is INTEGER
  On entry, LDA specifies the first dimension of AB as declared
  in the calling (sub) program. LDAB must be at least max( 1, m ).
  Unchanged on exit.

X
  X is DOUBLE PRECISION array, dimension
  ( 1 + ( n - 1 )*abs( INCX ) ) when TRANS = 'N' or 'n'
  and at least
  ( 1 + ( m - 1 )*abs( INCX ) ) otherwise.
  Before entry, the incremented array X must contain the vector x.
  Unchanged on exit.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of
  X. INCX must not be zero.
  Unchanged on exit.

BETA
  BETA is DOUBLE PRECISION
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.
  Unchanged on exit.

Y
  Y is DOUBLE PRECISION array, dimension
  ( 1 + ( m - 1 )*abs( INCY ) ) when TRANS = 'N' or 'n'
  and at least
  ( 1 + ( n - 1 )*abs( INCY ) ) otherwise.
  Before entry with BETA non-zero, the incremented array Y
  must contain the vector y. On exit, Y is overwritten by the
```

```
  updated vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of
  Y. INCY must not be zero.
  Unchanged on exit.
```

44.12.3 fortran code

— dla_gbamv.f —

```
*  =====================================================================
      SUBROUTINE DLA_GBAMV( TRANS, M, N, KL, KU, ALPHA, AB, LDAB, X,
     $                      INCX, BETA, Y, INCY )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   ALPHA, BETA
      INTEGER            INCX, INCY, LDAB, M, N, KL, KU, TRANS
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   AB( LDAB, * ), X( * ), Y( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            SYMB_ZERO
      DOUBLE PRECISION   TEMP, SAFE1
      INTEGER            I, INFO, IY, J, JX, KX, KY, LENX, LENY, KD, KE
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, DLAMCH
      DOUBLE PRECISION   DLAMCH
*     ..
*     .. External Functions ..
```

```
      EXTERNAL           ILATRANS
      INTEGER            ILATRANS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, ABS, SIGN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF     ( .NOT.( ( TRANS.EQ.ILATRANS( 'N' ) )
     $           .OR. ( TRANS.EQ.ILATRANS( 'T' ) )
     $           .OR. ( TRANS.EQ.ILATRANS( 'C' ) ) ) ) THEN
         INFO = 1
      ELSE IF( M.LT.0 )THEN
         INFO = 2
      ELSE IF( N.LT.0 )THEN
         INFO = 3
      ELSE IF( KL.LT.0 .OR. KL.GT.M-1 ) THEN
         INFO = 4
      ELSE IF( KU.LT.0 .OR. KU.GT.N-1 ) THEN
         INFO = 5
      ELSE IF( LDAB.LT.KL+KU+1 )THEN
         INFO = 6
      ELSE IF( INCX.EQ.0 )THEN
         INFO = 8
      ELSE IF( INCY.EQ.0 )THEN
         INFO = 11
      END IF
      IF( INFO.NE.0 )THEN
         CALL XERBLA( 'DLA_GBAMV ', INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( ( M.EQ.0 ).OR.( N.EQ.0 ).OR.
     $    ( ( ALPHA.EQ.ZERO ).AND.( BETA.EQ.ONE ) ) )
     $   RETURN
*
*     Set  LENX  and  LENY, the lengths of the vectors x and y, and set
*     up the start points in  X  and  Y.
*
      IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
         LENX = N
         LENY = M
      ELSE
         LENX = M
         LENY = N
```

```
      END IF
      IF( INCX.GT.0 )THEN
         KX = 1
      ELSE
         KX = 1 - ( LENX - 1 )*INCX
      END IF
      IF( INCY.GT.0 )THEN
         KY = 1
      ELSE
         KY = 1 - ( LENY - 1 )*INCY
      END IF
*
*     Set SAFE1 essentially to be the underflow threshold times the
*     number of additions in each row.
*
      SAFE1 = DLAMCH( 'Safe minimum' )
      SAFE1 = (N+1)*SAFE1
*
*     Form  y := alpha*abs(A)*abs(x) + beta*abs(y).
*
*     The O(M*N) SYMB_ZERO tests could be replaced by O(N) queries to
*     the inexact flag.  Still doesn't help change the iteration order
*     to per-column.
*
      KD = KU + 1
      KE = KL + 1
      IY = KY
      IF ( INCX.EQ.1 ) THEN
         IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
            DO I = 1, LENY
               IF ( BETA .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
                  Y( IY ) = 0.0D+0
               ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
               ELSE
                  SYMB_ZERO = .FALSE.
                  Y( IY ) = BETA * ABS( Y( IY ) )
               END IF
               IF ( ALPHA .NE. ZERO ) THEN
                  DO J = MAX( I-KL, 1 ), MIN( I+KU, LENX )
                     TEMP = ABS( AB( KD+I-J, J ) )
                     SYMB_ZERO = SYMB_ZERO .AND.
     $                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                     Y( IY ) = Y( IY ) + ALPHA*ABS( X( J ) )*TEMP
                  END DO
               END IF

               IF ( .NOT.SYMB_ZERO )
```

```
$              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )
          IY = IY + INCY
       END DO
    ELSE
       DO I = 1, LENY
          IF ( BETA .EQ. ZERO ) THEN
             SYMB_ZERO = .TRUE.
             Y( IY ) = 0.0D+0
          ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
             SYMB_ZERO = .TRUE.
          ELSE
             SYMB_ZERO = .FALSE.
             Y( IY ) = BETA * ABS( Y( IY ) )
          END IF
          IF ( ALPHA .NE. ZERO ) THEN
             DO J = MAX( I-KL, 1 ), MIN( I+KU, LENX )
                TEMP = ABS( AB( KE-I+J, I ) )
                SYMB_ZERO = SYMB_ZERO .AND.
$                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                Y( IY ) = Y( IY ) + ALPHA*ABS( X( J ) )*TEMP
             END DO
          END IF

          IF ( .NOT.SYMB_ZERO )
$              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )
          IY = IY + INCY
       END DO
    END IF
 ELSE
    IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
       DO I = 1, LENY
          IF ( BETA .EQ. ZERO ) THEN
             SYMB_ZERO = .TRUE.
             Y( IY ) = 0.0D+0
          ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
             SYMB_ZERO = .TRUE.
          ELSE
             SYMB_ZERO = .FALSE.
             Y( IY ) = BETA * ABS( Y( IY ) )
          END IF
          IF ( ALPHA .NE. ZERO ) THEN
             JX = KX
             DO J = MAX( I-KL, 1 ), MIN( I+KU, LENX )
                TEMP = ABS( AB( KD+I-J, J ) )
                SYMB_ZERO = SYMB_ZERO .AND.
$                    ( X( JX ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                Y( IY ) = Y( IY ) + ALPHA*ABS( X( JX ) )*TEMP
                JX = JX + INCX
```

```
               END DO
            END IF

            IF ( .NOT.SYMB_ZERO )
     $           Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

            IY = IY + INCY
         END DO
      ELSE
         DO I = 1, LENY
            IF ( BETA .EQ. ZERO ) THEN
               SYMB_ZERO = .TRUE.
               Y( IY ) = 0.0D+0
            ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
               SYMB_ZERO = .TRUE.
            ELSE
               SYMB_ZERO = .FALSE.
               Y( IY ) = BETA * ABS( Y( IY ) )
            END IF
            IF ( ALPHA .NE. ZERO ) THEN
               JX = KX
               DO J = MAX( I-KL, 1 ), MIN( I+KU, LENX )
                  TEMP = ABS( AB( KE-I+J, I ) )
                  SYMB_ZERO = SYMB_ZERO .AND.
     $                 ( X( JX ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                  Y( IY ) = Y( IY ) + ALPHA*ABS( X( JX ) )*TEMP
                  JX = JX + INCX
               END DO
            END IF

            IF ( .NOT.SYMB_ZERO )
     $           Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

            IY = IY + INCY
         END DO
      END IF

      END IF
*
      RETURN
*
*     End of DLA_GBAMV
*
      END
```

———-

44.12.4 lisp code

— dla_gbamv.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dla_gbamv (trans m n kl ku alpha ab ldab x incx beta y incy)
    (declare (type (array double-float (*)) y x ab)
             (type (double-float) beta alpha)
             (type (f2cl-lib:integer4) incy incx ldab ku kl n m trans))
    (f2cl-lib:with-multi-array-data
        ((ab double-float ab-%data% ab-%offset%)
         (x double-float x-%data% x-%offset%)
         (y double-float y-%data% y-%offset%))
      (prog ((i 0) (info 0) (iy 0) (j 0) (jx 0) (kx 0) (ky 0) (lenx 0) (leny 0)
             (kd 0) (ke 0) (temp 0.0d0) (safe1 0.0d0) (symb_zero nil))
        (declare
         (type (f2cl-lib:integer4) i info iy j jx kx ky lenx leny kd ke)
         (type (double-float) temp safe1)
         (type f2cl-lib:logical symb_zero))
        (setf info 0)
        (cond
         ((not
           (or (= trans (ilatrans "N")) (= trans (ilatrans "T"))
               (= trans (ilatrans "C"))))
          (setf info 1))
         ((< m 0) (setf info 2)) ((< n 0) (setf info 3))
         ((or (< kl 0) (> kl (f2cl-lib:int-add m (f2cl-lib:int-sub 1))))
          (setf info 4))
         ((or (< ku 0) (> ku (f2cl-lib:int-add n (f2cl-lib:int-sub 1))))
          (setf info 5))
         ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info 6))
         ((= incx 0) (setf info 8)) ((= incy 0) (setf info 11)))
        (cond
         ((/= info 0)
          (multiple-value-bind (var-0 var-1)
              (xerbla "DLA_GBAMV " info)
            (declare (ignore var-0))
            (when var-1 (setf info var-1)))
          (go end_label)))
        (if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
            (go end_label))
        (cond ((= trans (ilatrans "N")) (setf lenx n) (setf leny m))
              (t (setf lenx m) (setf leny n)))
        (cond ((> incx 0) (setf kx 1))
              (t
               (setf kx
```

```
                (f2cl-lib:int-sub 1
                                  (f2cl-lib:int-mul
                                   (f2cl-lib:int-sub lenx 1) incx)))))
(cond ((> incy 0) (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                  (f2cl-lib:int-mul
                                   (f2cl-lib:int-sub leny 1) incy)))))
(setf safe1 (dlamch "Safe minimum"))
(setf safe1 (* (f2cl-lib:int-add n 1) safe1))
(setf kd (f2cl-lib:int-add ku 1))
(setf ke (f2cl-lib:int-add kl 1))
(setf iy ky)
(cond
 ((= incx 1)
  (cond
   ((= trans (ilatrans "N"))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i leny) nil)
      (tagbody
        (cond
         ((= beta zero) (setf symb_zero f2cl-lib:%true%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                0.0d0))
         ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
          (setf symb_zero f2cl-lib:%true%))
         (t (setf symb_zero f2cl-lib:%false%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                (* beta
                   (abs
                    (f2cl-lib:fref y-%data% (iy) ((1 *))
                                   y-%offset%))))))
        (cond
         ((/= alpha zero)
          (f2cl-lib:fdo (j
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add i
                                                 (f2cl-lib:int-sub
                                                  kl)))
                          (the f2cl-lib:integer4 1))
                         (f2cl-lib:int-add j 1))
                        ((> j
                            (min
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add i ku))
                             (the f2cl-lib:integer4 lenx)))
                         nil)
            (tagbody
```

```
            (setf temp
                    (abs
                     (f2cl-lib:fref ab-%data%
                                    ((f2cl-lib:int-sub
                                      (f2cl-lib:int-add kd i) j)
                                     j)
                                    ((1 ldab) (1 *)) ab-%offset%)))
            (setf symb_zero
                    (and symb_zero
                         (or
                          (=
                           (f2cl-lib:fref x-%data% (j) ((1 *))
                                          x-%offset%)
                           zero)
                          (= temp zero))))
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+
                     (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (* alpha
                        (abs
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%))
                        temp)))
         label100001))))
     (if (not symb_zero)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (f2cl-lib:sign safe1
                                   (f2cl-lib:fref y-%data% (iy)
                                                  ((1 *))
                                                  y-%offset%)))))
     (setf iy (f2cl-lib:int-add iy incy))
    label100000)))
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i leny) nil)
   (tagbody
     (cond
      ((= beta zero) (setf symb_zero f2cl-lib:%true%)
       (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
               0.0d0))
      ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
       (setf symb_zero f2cl-lib:%true%))
      (t (setf symb_zero f2cl-lib:%false%)
       (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
               (* beta
                  (abs
                   (f2cl-lib:fref y-%data% (iy) ((1 *))
                                  y-%offset%))))))
     (cond
```

```
                ((/= alpha zero)
                 (f2cl-lib:fdo (j
                                (max
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add i
                                                        (f2cl-lib:int-sub
                                                         kl)))
                                 (the f2cl-lib:integer4 1))
                                (f2cl-lib:int-add j 1))
                               ((> j
                                   (min
                                    (the f2cl-lib:integer4
                                         (f2cl-lib:int-add i ku))
                                    (the f2cl-lib:integer4 lenx)))
                                nil)
                   (tagbody
                     (setf temp
                             (abs
                              (f2cl-lib:fref ab-%data%
                                             ((f2cl-lib:int-add
                                               (f2cl-lib:int-sub ke i) j)
                                              i)
                                             ((1 ldab) (1 *)) ab-%offset%)))
                     (setf symb_zero
                             (and symb_zero
                                  (or
                                   (=
                                    (f2cl-lib:fref x-%data% (j) ((1 *))
                                                   x-%offset%)
                                    zero)
                                   (= temp zero))))
                     (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                             (+
                              (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                              (* alpha
                                 (abs
                                  (f2cl-lib:fref x-%data% (j) ((1 *))
                                                 x-%offset%))
                                 temp)))
                    label100003))))
               (if (not symb_zero)
                   (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                            (f2cl-lib:sign safe1
                                           (f2cl-lib:fref y-%data% (iy)
                                                          ((1 *))
                                                          y-%offset%)))))
                (setf iy (f2cl-lib:int-add iy incy))
               label100002)))))
           (t
```

```
(cond
 ((= trans (ilatrans "N"))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i leny) nil)
    (tagbody
      (cond
       ((= beta zero) (setf symb_zero f2cl-lib:%true%)
        (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                0.0d0))
       ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
        (setf symb_zero f2cl-lib:%true%))
       (t (setf symb_zero f2cl-lib:%false%)
        (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                (* beta
                   (abs
                    (f2cl-lib:fref y-%data% (iy) ((1 *))
                                   y-%offset%))))))
      (cond
       ((/= alpha zero) (setf jx kx)
        (f2cl-lib:fdo (j
                       (max
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add i
                                               (f2cl-lib:int-sub
                                                kl)))
                        (the f2cl-lib:integer4 1))
                       (f2cl-lib:int-add j 1))
                      ((> j
                          (min
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add i ku))
                           (the f2cl-lib:integer4 lenx)))
                       nil)
          (tagbody
            (setf temp
                    (abs
                     (f2cl-lib:fref ab-%data%
                                    ((f2cl-lib:int-sub
                                      (f2cl-lib:int-add kd i) j)
                                     j)
                                    ((1 ldab) (1 *)) ab-%offset%)))
            (setf symb_zero
                    (and symb_zero
                         (or
                          (=
                           (f2cl-lib:fref x-%data% (jx) ((1 *))
                                          x-%offset%)
                           zero)
                          (= temp zero))))
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
```

```
                        (+
                         (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (* alpha
                            (abs
                             (f2cl-lib:fref x-%data% (jx) ((1 *))
                                            x-%offset%))
                            temp)))
                 (setf jx (f2cl-lib:int-add jx incx))
                label100005))))
        (if (not symb_zero)
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (f2cl-lib:sign safe1
                                      (f2cl-lib:fref y-%data% (iy)
                                                     ((1 *))
                                                     y-%offset%)))))
        (setf iy (f2cl-lib:int-add iy incy))
       label100004)))
    (t
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i leny) nil)
       (tagbody
         (cond
          ((= beta zero) (setf symb_zero f2cl-lib:%true%)
           (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 0.0d0))
          ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
           (setf symb_zero f2cl-lib:%true%))
          (t (setf symb_zero f2cl-lib:%false%)
           (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (* beta
                    (abs
                     (f2cl-lib:fref y-%data% (iy) ((1 *))
                                    y-%offset%))))))
         (cond
          ((/= alpha zero) (setf jx kx)
           (f2cl-lib:fdo (j
                          (max
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add i
                                                  (f2cl-lib:int-sub
                                                   kl)))
                           (the f2cl-lib:integer4 1))
                          (f2cl-lib:int-add j 1))
                         ((> j
                             (min
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add i ku))
                              (the f2cl-lib:integer4 lenx)))
                          nil)
```

```
                    (tagbody
                      (setf temp
                              (abs
                               (f2cl-lib:fref ab-%data%
                                              ((f2cl-lib:int-add
                                                (f2cl-lib:int-sub ke i) j)
                                               i)
                                              ((1 ldab) (1 *)) ab-%offset%)))
                      (setf symb_zero
                              (and symb_zero
                                   (or
                                    (=
                                     (f2cl-lib:fref x-%data% (jx) ((1 *))
                                                    x-%offset%)
                                     zero)
                                    (= temp zero))))
                      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                              (+
                               (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                               (* alpha
                                  (abs
                                   (f2cl-lib:fref x-%data% (jx) ((1 *))
                                                  x-%offset%))
                                  temp)))
                      (setf jx (f2cl-lib:int-add jx incx))
                     label100007))))
                (if (not symb_zero)
                    (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                            (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                               (f2cl-lib:sign safe1
                                              (f2cl-lib:fref y-%data% (iy)
                                                             ((1 *))
                                                             y-%offset%)))))
                (setf iy (f2cl-lib:int-add iy incy))
               label100006))))))
        (go end_label)
       end_label
        (return
         (values nil nil nil nil nil nil nil nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dla_gbamv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
```

```
                                        (fortran-to-lisp::integer4)
                                        (double-float)
                                        (array double-float (*))
                                        (fortran-to-lisp::integer4)
                                        (array double-float (*))
                                        (fortran-to-lisp::integer4)
                                        (double-float)
                                        (array double-float (*))
                                        (fortran-to-lisp::integer4))
                                      :return-values
                                      '(nil nil nil nil nil nil nil nil
                                        nil nil nil nil nil)
                                      :calls 'nil)))
```

———-

44.13 dla_gbrcond

44.13.1 Axiom unit tests

— **dlagbrcond.input** —

```
)set break resume
)sys rm -f dlagbrcond.output
)spool dlagbrcond.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

44.13.2 Axiom help page

— **dla_gbrcond.help** —

```
==========================================================================
dla_gbrcond examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function dla_gbrcond (TRANS, N, KL, KU, AB, LDAB, AFB,
           LDAFB, IPIV, CMODE, C, INFO, WORK, IWORK)
           DLA_GBRCOND estimates the Skeel condition number for a general
           banded matrix.

Function/Subroutine Documentation
   double precision function dla_gbrcond (character TRANS, integer N, integer
       KL, integer KU, double precision, dimension( ldab, * ) AB, integer
       LDAB, double precision, dimension( ldafb, * ) AFB, integer LDAFB,
       integer, dimension( * ) IPIV, integer CMODE, double precision,
       dimension( * ) C, integer INFO, double precision, dimension( * ) WORK,
       integer, dimension( * ) IWORK)
       DLA_GBRCOND estimates the Skeel condition number for a general banded
       matrix.

Purpose:
   DLA_GBRCOND Estimates the Skeel condition number of  op(A) * op2(C)
   where op2 is determined by CMODE as follows
   CMODE =  1    op2(C) = C
   CMODE =  0    op2(C) = I
   CMODE = -1    op2(C) = inv(C)
   The Skeel condition number  cond(A) = norminf( |inv(A)||A| )
   is computed by computing scaling factors R such that
   diag(R)*A*op2(C) is row equilibrated and computing the standard
   infinity-norm condition number.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate Transpose = Transpose)

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
```

```
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

AB
  AB is DOUBLE PRECISION array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is DOUBLE PRECISION array, dimension (LDAFB,N)
  Details of the LU factorization of the band matrix A, as
  computed by DGBTRF.  U is stored as an upper triangular
  band matrix with KL+KU superdiagonals in rows 1 to KL+KU+1,
  and the multipliers used during the factorization are stored
  in rows KL+KU+2 to 2*KL+KU+1.

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from the factorization A = P*L*U
  as computed by DGBTRF; row i of the matrix was interchanged
  with row IPIV(i).

CMODE
  CMODE is INTEGER
  Determines op2(C) in the formula op(A) * op2(C) as follows:
  CMODE =  1    op2(C) = C
  CMODE =  0    op2(C) = I
  CMODE = -1    op2(C) = inv(C)

C
  C is DOUBLE PRECISION array, dimension (N)
  The vector C in the formula op(A) * op2(C).

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  i > 0:  The ith argument is invalid.

WORK
  WORK is DOUBLE PRECISION array, dimension (5*N).
```

```
  Workspace.

IWORK
  IWORK is INTEGER array, dimension (N).
  Workspace.
```

44.13.3 fortran code

— dla_gbrcond.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION DLA_GBRCOND( TRANS, N, KL, KU, AB, LDAB,
     $                                       AFB, LDAFB, IPIV, CMODE, C,
     $                                       INFO, WORK, IWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            N, LDAB, LDAFB, INFO, KL, KU, CMODE
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * ), IPIV( * )
      DOUBLE PRECISION   AB( LDAB, * ), AFB( LDAFB, * ), WORK( * ),
     $                   C( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            NOTRANS
      INTEGER            KASE, I, J, KD, KE
      DOUBLE PRECISION   AINVNM, TMP
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
```

```
      EXTERNAL           DLACN2, DGBTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
      DLA_GBRCOND = 0.0D+0
*
      INFO = 0
      NOTRANS = LSAME( TRANS, 'N' )
      IF ( .NOT. NOTRANS .AND. .NOT. LSAME(TRANS, 'T')
     $     .AND. .NOT. LSAME(TRANS, 'C') ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 .OR. KL.GT.N-1 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 .OR. KU.GT.N-1 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -6
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLA_GBRCOND', -INFO )
         RETURN
      END IF
      IF( N.EQ.0 ) THEN
         DLA_GBRCOND = 1.0D+0
         RETURN
      END IF
*
*     Compute the equilibration matrix R such that
*     inv(R)*A*C has unit 1-norm.
*
      KD = KU + 1
      KE = KL + 1
      IF ( NOTRANS ) THEN
         DO I = 1, N
            TMP = 0.0D+0
               IF ( CMODE .EQ. 1 ) THEN
                  DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
                     TMP = TMP + ABS( AB( KD+I-J, J ) * C( J ) )
                  END DO
               ELSE IF ( CMODE .EQ. 0 ) THEN
                  DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
                     TMP = TMP + ABS( AB( KD+I-J, J ) )
                  END DO
```

```
            ELSE
               DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
                  TMP = TMP + ABS( AB( KD+I-J, J ) / C( J ) )
               END DO
            END IF
         WORK( 2*N+I ) = TMP
      END DO
   ELSE
      DO I = 1, N
         TMP = 0.0D+0
         IF ( CMODE .EQ. 1 ) THEN
            DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
               TMP = TMP + ABS( AB( KE-I+J, I ) * C( J ) )
            END DO
         ELSE IF ( CMODE .EQ. 0 ) THEN
            DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
               TMP = TMP + ABS( AB( KE-I+J, I ) )
            END DO
         ELSE
            DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
               TMP = TMP + ABS( AB( KE-I+J, I ) / C( J ) )
            END DO
         END IF
         WORK( 2*N+I ) = TMP
      END DO
   END IF
*
*     Estimate the norm of inv(op(A)).
*
      AINVNM = 0.0D+0

      KASE = 0
   10 CONTINUE
      CALL DLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * WORK( 2*N+I )
            END DO

            IF ( NOTRANS ) THEN
               CALL DGBTRS( 'No transpose', N, KL, KU, 1, AFB, LDAFB,
     $              IPIV, WORK, N, INFO )
            ELSE
               CALL DGBTRS( 'Transpose', N, KL, KU, 1, AFB, LDAFB, IPIV,
     $              WORK, N, INFO )
            END IF
```

```
*
*          Multiply by inv(C).
*
           IF ( CMODE .EQ. 1 ) THEN
              DO I = 1, N
                 WORK( I ) = WORK( I ) / C( I )
              END DO
           ELSE IF ( CMODE .EQ. -1 ) THEN
              DO I = 1, N
                 WORK( I ) = WORK( I ) * C( I )
              END DO
           END IF
        ELSE
*
*          Multiply by inv(C**T).
*
           IF ( CMODE .EQ. 1 ) THEN
              DO I = 1, N
                 WORK( I ) = WORK( I ) / C( I )
              END DO
           ELSE IF ( CMODE .EQ. -1 ) THEN
              DO I = 1, N
                 WORK( I ) = WORK( I ) * C( I )
              END DO
           END IF

           IF ( NOTRANS ) THEN
              CALL DGBTRS( 'Transpose', N, KL, KU, 1, AFB, LDAFB, IPIV,
     $              WORK, N, INFO )
           ELSE
              CALL DGBTRS( 'No transpose', N, KL, KU, 1, AFB, LDAFB,
     $              IPIV, WORK, N, INFO )
           END IF
*
*          Multiply by R.
*
           DO I = 1, N
              WORK( I ) = WORK( I ) * WORK( 2*N+I )
           END DO
        END IF
        GO TO 10
     END IF
*
*    Compute the estimate of the reciprocal condition number.
*
     IF( AINVNM .NE. 0.0D+0 )
     $   DLA_GBRCOND = ( 1.0D+0 / AINVNM )
*
     RETURN
*
```

```
      END
```

———-

44.13.4 lisp code

— **dla_gbrcond.lisp** —

```
(defun dla_gbrcond
       (trans n kl ku ab ldab afb ldafb ipiv cmode c info work iwork)
  (declare (type (array f2cl-lib:integer4 (*)) iwork ipiv)
           (type (array double-float (*)) work c afb ab)
           (type (f2cl-lib:integer4) info cmode ldafb ldab ku kl n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (ab double-float ab-%data% ab-%offset%)
       (afb double-float afb-%data% afb-%offset%)
       (c double-float c-%data% c-%offset%)
       (work double-float work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
    (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
           (ainvnm 0.0d0) (tmp 0.0d0) (kase 0) (i 0) (j 0) (kd 0) (ke 0)
           (notrans nil) (dla_gbrcond 0.0d0))
      (declare (type f2cl-lib:logical notrans)
               (type (f2cl-lib:integer4) ke kd j i kase)
               (type (double-float) dla_gbrcond tmp ainvnm)
               (type (array f2cl-lib:integer4 (3)) isave))
      (setf dla_gbrcond 0.0d0)
      (setf info 0)
      (setf notrans
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "N")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val))
      (cond
       ((and (not notrans)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "T")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val))
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "C")
```

```
            (declare (ignore var-1))
            (when var-0 (setf trans var-0))
            ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((or (< kl 0) (> kl (f2cl-lib:int-add n (f2cl-lib:int-sub 1))))
  (setf info -3))
 ((or (< ku 0) (> ku (f2cl-lib:int-add n (f2cl-lib:int-sub 1))))
  (setf info -4))
 ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -6))
 ((< ldafb (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
  (setf info -8)))
(cond
 ((/= info 0) (xerbla "DLA_GBRCOND" (f2cl-lib:int-sub info))
  (go end_label)))
(cond ((= n 0) (setf dla_gbrcond 1.0d0) (go end_label)))
(setf kd (f2cl-lib:int-add ku 1))
(setf ke (f2cl-lib:int-add kl 1))
(cond
 (notrans
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf tmp 0.0d0)
      (cond
       ((= cmode 1)
        (f2cl-lib:fdo (j
                       (max
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add i (f2cl-lib:int-sub kl)))
                        (the f2cl-lib:integer4 1))
                       (f2cl-lib:int-add j 1))
                      ((> j
                          (min
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add i ku))
                           (the f2cl-lib:integer4 n)))
                       nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (abs
                        (*
                         (f2cl-lib:fref ab-%data%
                                        ((f2cl-lib:int-sub
                                          (f2cl-lib:int-add kd i) j)
                                         j)
                                        ((1 ldab) (1 *)) ab-%offset%)
                         (f2cl-lib:fref c-%data% (j) ((1 *))
                                        c-%offset%)))))
```

```
    label100001)))
((= cmode 0)
 (f2cl-lib:fdo (j
                (max
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add i (f2cl-lib:int-sub kl)))
                 (the f2cl-lib:integer4 1))
                (f2cl-lib:int-add j 1))
               ((> j
                   (min
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add i ku))
                    (the f2cl-lib:integer4 n)))
                nil)
   (tagbody
     (setf tmp
             (+ tmp
                (abs
                 (f2cl-lib:fref ab-%data%
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-add kd i) j)
                                 j)
                                ((1 ldab) (1 *)) ab-%offset%))))
    label100002)))
(t
 (f2cl-lib:fdo (j
                (max
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add i (f2cl-lib:int-sub kl)))
                 (the f2cl-lib:integer4 1))
                (f2cl-lib:int-add j 1))
               ((> j
                   (min
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add i ku))
                    (the f2cl-lib:integer4 n)))
                nil)
   (tagbody
     (setf tmp
             (+ tmp
                (abs
                 (/
                  (f2cl-lib:fref ab-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-add kd i) j)
                                  j)
                                 ((1 ldab) (1 *)) ab-%offset%)
                  (f2cl-lib:fref c-%data% (j) ((1 *))
                                 c-%offset%)))))
    label100003))))
```

```
          (setf (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) i))
                               ((1 *)) work-%offset%)
                tmp)
      label100000)))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf tmp 0.0d0)
       (cond
        ((= cmode 1)
         (f2cl-lib:fdo (j
                        (max
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add i (f2cl-lib:int-sub kl)))
                         (the f2cl-lib:integer4 1))
                        (f2cl-lib:int-add j 1))
                       ((> j
                           (min
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add i ku))
                            (the f2cl-lib:integer4 n)))
                        nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (abs
                         (*
                          (f2cl-lib:fref ab-%data%
                                         ((f2cl-lib:int-add
                                           (f2cl-lib:int-sub ke i) j)
                                          i)
                                         ((1 ldab) (1 *)) ab-%offset%)
                          (f2cl-lib:fref c-%data% (j) ((1 *))
                                         c-%offset%)))))
             label100005)))
        ((= cmode 0)
         (f2cl-lib:fdo (j
                        (max
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add i (f2cl-lib:int-sub kl)))
                         (the f2cl-lib:integer4 1))
                        (f2cl-lib:int-add j 1))
                       ((> j
                           (min
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add i ku))
                            (the f2cl-lib:integer4 n)))
                        nil)
```

```
            (tagbody
              (setf tmp
                      (+ tmp
                         (abs
                          (f2cl-lib:fref ab-%data%
                                         ((f2cl-lib:int-add
                                           (f2cl-lib:int-sub ke i) j)
                                          i)
                                         ((1 ldab) (1 *)) ab-%offset%))))
             label100006)))
          (t
           (f2cl-lib:fdo (j
                          (max
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add i (f2cl-lib:int-sub kl)))
                           (the f2cl-lib:integer4 1))
                          (f2cl-lib:int-add j 1))
                         ((> j
                             (min
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add i ku))
                              (the f2cl-lib:integer4 n)))
                          nil)
            (tagbody
              (setf tmp
                      (+ tmp
                         (abs
                          (/
                           (f2cl-lib:fref ab-%data%
                                          ((f2cl-lib:int-add
                                            (f2cl-lib:int-sub ke i) j)
                                           i)
                                          ((1 ldab) (1 *)) ab-%offset%)
                           (f2cl-lib:fref c-%data% (j) ((1 *))
                                          c-%offset%)))))
             label100007))))
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) i))
                             ((1 *)) work-%offset%)
              tmp)
      label100004))))
  (setf ainvnm 0.0d0)
  (setf kase 0)
 label10
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacn2 n
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1)) ((1 *))
                             work-%offset%)
       work iwork ainvnm kase isave)
    (declare (ignore var-1 var-2 var-3 var-6))
```

```
      (setf n var-0)
      (setf ainvnm var-4)
      (setf kase var-5))
    (cond
     ((/= kase 0)
      (cond
       ((= kase 2)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                    (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                       (f2cl-lib:fref work-%data%
                                      ((f2cl-lib:int-add
                                        (f2cl-lib:int-mul 2 n) i))
                                      ((1 *)) work-%offset%)))
           label100008))
        (cond
         (notrans
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10)
              (dgbtrs "No transpose" n kl ku 1 afb ldafb ipiv work n info)
            (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
            (setf n var-1)
            (setf ldafb var-6)
            (setf n var-9)
            (setf info var-10)))
         (t
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10)
              (dgbtrs "Transpose" n kl ku 1 afb ldafb ipiv work n info)
            (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
            (setf n var-1)
            (setf ldafb var-6)
            (setf n var-9)
            (setf info var-10))))
        (cond
         ((= cmode 1)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                      (/
                       (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                       (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
             label100009)))
         ((= cmode (f2cl-lib:int-sub 1))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                 ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (*
                 (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
       label100010)))))
(t
 (cond
  ((= cmode 1)
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               (/
                (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
      label100011)))
  ((= cmode (f2cl-lib:int-sub 1))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               (*
                (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
      label100012))))
 (cond
  (notrans
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (dgbtrs "Transpose" n kl ku 1 afb ldafb ipiv work n info)
     (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
     (setf n var-1)
     (setf ldafb var-6)
     (setf n var-9)
     (setf info var-10)))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (dgbtrs "No transpose" n kl ku 1 afb ldafb ipiv work n info)
     (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
     (setf n var-1)
     (setf ldafb var-6)
     (setf n var-9)
     (setf info var-10))))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
```

```
            (tagbody
              (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                      (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                         (f2cl-lib:fref work-%data%
                                        ((f2cl-lib:int-add
                                          (f2cl-lib:int-mul 2 n) i))
                                        ((1 *)) work-%offset%)))
             label100013))))
        (go label10)))
      (if (/= ainvnm 0.0d0)
          (setf dla_gbrcond (/ 1.0d0 ainvnm)))
      (go end_label)
     end_label
      (return
       (values dla_gbrcond trans n nil nil nil nil nil ldafb nil nil nil info
               nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dla_gbrcond
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*)))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::n nil nil nil
                                              nil nil fortran-to-lisp::ldafb
                                              nil nil nil fortran-to-lisp::info
                                              nil nil)
                                            :calls
                                            '(fortran-to-lisp::dgbtrs
                                              fortran-to-lisp::dlacn2))))
```

———-

44.14 dla_gbrfsx_extended

44.14.1 Axiom unit tests

— **dlagbrfsxextended.input** —

```
)set break resume
)sys rm -f dlagbrfsxextended.output
)spool dlagbrfsxextended.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

44.14.2 Axiom help page [Demm05]

— **dla_gbrfsx_extended.help** —

```
====================================================================
dla_gbrfsx_extended examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dla_gbrfsx_extended (PREC_TYPE, TRANS_TYPE, N, KL, KU, NRHS,
           AB, LDAB, AFB, LDAFB, IPIV, COLEQU, C, B, LDB, Y, LDY, BERR_OUT,
           N_NORMS, ERR_BNDS_NORM, ERR_BNDS_COMP, RES, AYB, DY, Y_TAIL, RCOND,
           ITHRESH, RTHRESH, DZ_UB, IGNORE_CWISE, INFO)
           DLA_GBRFSX_EXTENDED improves the computed solution to a system of
           linear equations for general banded matrices by performing extra-
           precise iterative refinement and provides error bounds and backward
           error estimates for the solution.

Function/Subroutine Documentation
```

```
   subroutine dla_gbrfsx_extended (integer PREC_TYPE, integer TRANS_TYPE,
       integer N, integer KL, integer KU, integer NRHS, double precision,
       dimension( ldab, * ) AB, integer LDAB, double precision, dimension(
       ldafb, * ) AFB, integer LDAFB, integer, dimension( * ) IPIV, logical
       COLEQU, double precision, dimension( * ) C, double precision,
       dimension( ldb, * ) B, integer LDB, double precision, dimension( ldy, *
       ) Y, integer LDY, double precision, dimension(*) BERR_OUT, integer
       N_NORMS, double precision, dimension( nrhs, * ) ERR_BNDS_NORM, double
       precision, dimension( nrhs, * ) ERR_BNDS_COMP, double precision,
       dimension(*) RES, double precision, dimension(*) AYB, double precision,
       dimension(*) DY, double precision, dimension(*) Y_TAIL, double
       precision RCOND, integer ITHRESH, double precision RTHRESH, double
       precision DZ_UB, logical IGNORE_CWISE, integer INFO)
       DLA_GBRFSX_EXTENDED improves the computed solution to a system of
       linear equations for general banded matrices by performing extra-
       precise iterative refinement and provides error bounds and backward
       error estimates for the solution.

Purpose:
   DLA_GBRFSX_EXTENDED improves the computed solution to a system of
   linear equations by performing extra-precise iterative refinement
   and provides error bounds and backward error estimates for the solution.
   This subroutine is called by DGBRFSX to perform iterative refinement.
   In addition to normwise error bound, the code provides maximum
   componentwise error bound if possible. See comments for ERR_BNDS_NORM
   and ERR_BNDS_COMP for details of the error bounds. Note that this
   subroutine is only resonsible for setting the second fields of
   ERR_BNDS_NORM and ERR_BNDS_COMP.

Parameters:

   PREC_TYPE

   PREC_TYPE is INTEGER
   Specifies the intermediate precision to be used in refinement.
   The value is defined by ILAPREC(P) where P is a CHARACTER and
   P    = 'S':  Single
        = 'D':  Double
        = 'I':  Indigenous
        = 'X', 'E':  Extra

 TRANS_TYPE
   TRANS_TYPE is INTEGER
   Specifies the transposition operation on A.
   The value is defined by ILATRANS(T) where T is a CHARACTER and
   T    = 'N':  No transpose
        = 'T':  Transpose
        = 'C':  Conjugate transpose

 N
```

```
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

KL
  KL is INTEGER
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0

NRHS
  NRHS is INTEGER
  The number of right-hand-sides, i.e., the number of columns of the
  matrix B.

AB
  AB is DOUBLE PRECISION array, dimension (LDAB,N)
  On entry, the N-by-N matrix AB.

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDBA >= max(1,N).

AFB
  AFB is DOUBLE PRECISION array, dimension (LDAFB,N)
  The factors L and U from the factorization
  A = P*L*U as computed by DGBTRF.

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AF.  LDAFB >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from the factorization A = P*L*U
  as computed by DGBTRF; row i of the matrix was interchanged
  with row IPIV(i).

COLEQU
  COLEQU is LOGICAL
  If .TRUE. then column equilibration was done to A before calling
  this routine. This is needed to compute the solution and error
  bounds correctly.

C
  C is DOUBLE PRECISION array, dimension (N)
  The column scale factors for A. If COLEQU = .FALSE., C
  is not accessed. If C is input, each element of C should be a power
  of the radix to ensure a reliable solution and error estimates.
```

```
  Scaling by powers of the radix does not cause rounding errors unless
  the result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  The right-hand-side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

Y
  Y is DOUBLE PRECISION array, dimension (LDY,NRHS)
  On entry, the solution matrix X, as computed by DGBTRS.
  On exit, the improved solution matrix Y.

LDY
  LDY is INTEGER
  The leading dimension of the array Y.  LDY >= max(1,N).

BERR_OUT
  BERR_OUT is DOUBLE PRECISION array, dimension (NRHS)
  On exit, BERR_OUT(j) contains the componentwise relative backward
  error for right-hand-side j from the formula
  max(i) ( abs(RES(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
  where abs(Z) is the componentwise absolute value of the matrix
  or vector Z. This is computed by DLA_LIN_BERR.

N_NORMS
  N_NORMS is INTEGER
  Determines which error bounds to return (see ERR_BNDS_NORM
  and ERR_BNDS_COMP).
  If N_NORMS >= 1 return normwise error bounds.
  If N_NORMS >= 2 return componentwise error bounds.

ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
            max_j (abs(XTRUE(j,i) - X(j,i)))
             ------------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
```

```
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

ERR_BNDS_COMP
  ERR_BNDS_COMP is DOUBLE PRECISION array, dimension
  (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                 abs(XTRUE(j,i) - X(j,i))
            max_j ----------------------
                       abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.
```

```
  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

RES
  RES is DOUBLE PRECISION array, dimension (N)
  Workspace to hold the intermediate residual.

AYB
  AYB is DOUBLE PRECISION array, dimension (N)
  Workspace. This can be the same workspace passed for Y_TAIL.

DY
  DY is DOUBLE PRECISION array, dimension (N)
  Workspace to hold the intermediate solution.

Y_TAIL
  Y_TAIL is DOUBLE PRECISION array, dimension (N)
  Workspace to hold the trailing bits of the intermediate solution.

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.
```

```
 ITHRESH
   ITHRESH is INTEGER
   The maximum number of residual computations allowed for
   refinement. The default is 10. For 'aggressive' set to 100 to
   permit convergence using approximate factorizations or
   factorizations other than LU. If the factorization uses a
   technique other than Gaussian elimination, the guarantees in
   ERR_BNDS_NORM and ERR_BNDS_COMP may no longer be trustworthy.

 RTHRESH
   RTHRESH is DOUBLE PRECISION
   Determines when to stop refinement if the error estimate stops
   decreasing. Refinement will stop when the next solution no longer
   satisfies norm(dx_{i+1}) < RTHRESH * norm(dx_i) where norm(Z) is
   the infinity norm of Z. RTHRESH satisfies 0 < RTHRESH <= 1. The
   default value is 0.5. For 'aggressive' set to 0.9 to permit
   convergence on extremely ill-conditioned matrices. See LAWN 165
   for more details.

 DZ_UB
   DZ_UB is DOUBLE PRECISION
   Determines when to start considering componentwise convergence.
   Componentwise convergence is only considered after each component
   of the solution Y is stable, which we definte as the relative
   change in each component being less than DZ_UB. The default value
   is 0.25, requiring the first bit to be stable. See LAWN 165 for
   more details.

 IGNORE_CWISE
   IGNORE_CWISE is LOGICAL
   If .TRUE. then ignore componentwise convergence. Default value
   is .FALSE..

 INFO
   INFO is INTEGER
   = 0:  Successful exit.
   < 0:  if INFO = -i, the ith argument to DGBTRS had an illegal value
```

———-

44.14.3 fortran code

— dla_gbrfsx_extended.f —

```
*  =====================================================================
      SUBROUTINE DLA_GBRFSX_EXTENDED( PREC_TYPE, TRANS_TYPE, N, KL, KU,
```

```
     $                                NRHS, AB, LDAB, AFB, LDAFB, IPIV,
     $                                COLEQU, C, B, LDB, Y, LDY,
     $                                BERR_OUT, N_NORMS, ERR_BNDS_NORM,
     $                                ERR_BNDS_COMP, RES, AYB, DY,
     $                                Y_TAIL, RCOND, ITHRESH, RTHRESH,
     $                                DZ_UB, IGNORE_CWISE, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDAB, LDAFB, LDB, LDY, N, KL, KU, NRHS,
     $                   PREC_TYPE, TRANS_TYPE, N_NORMS, ITHRESH
      LOGICAL            COLEQU, IGNORE_CWISE
      DOUBLE PRECISION   RTHRESH, DZ_UB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   Y( LDY, * ), RES(*), DY(*), Y_TAIL(*)
      DOUBLE PRECISION   C( * ), AYB(*), RCOND, BERR_OUT(*),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      CHARACTER          TRANS
      INTEGER            CNT, I, J, M, X_STATE, Z_STATE, Y_PREC_STATE
      DOUBLE PRECISION   YK, DYK, YMIN, NORMY, NORMX, NORMDX, DXRAT,
     $                   DZRAT, PREVNORMDX, PREV_DZ_Z, DXRATMAX,
     $                   DZRATMAX, DX_X, DZ_Z, FINAL_DX_X, FINAL_DZ_Z,
     $                   EPS, HUGEVAL, INCR_THRESH
      LOGICAL            INCR_PREC
*     ..
*     .. Parameters ..
      INTEGER            UNSTABLE_STATE, WORKING_STATE, CONV_STATE,
     $                   NOPROG_STATE, BASE_RESIDUAL, EXTRA_RESIDUAL,
     $                   EXTRA_Y
      PARAMETER          ( UNSTABLE_STATE = 0, WORKING_STATE = 1,
     $                   CONV_STATE = 2, NOPROG_STATE = 3 )
      PARAMETER          ( BASE_RESIDUAL = 0, EXTRA_RESIDUAL = 1,
     $                   EXTRA_Y = 2 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
```

```
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DCOPY, DGBTRS, DGBMV, BLAS_DGBMV_X,
     $                   BLAS_DGBMV2_X, DLA_GBAMV, DLA_WWADDW, DLAMCH,
     $                   CHLA_TRANSTYPE, DLA_LIN_BERR
      DOUBLE PRECISION   DLAMCH
      CHARACTER          CHLA_TRANSTYPE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      IF (INFO.NE.0) RETURN
      TRANS = CHLA_TRANSTYPE(TRANS_TYPE)
      EPS = DLAMCH( 'Epsilon' )
      HUGEVAL = DLAMCH( 'Overflow' )
*     Force HUGEVAL to Inf
      HUGEVAL = HUGEVAL * HUGEVAL
*     Using HUGEVAL may lead to spurious underflows.
      INCR_THRESH = DBLE( N ) * EPS
      M = KL+KU+1

      DO J = 1, NRHS
         Y_PREC_STATE = EXTRA_RESIDUAL
         IF ( Y_PREC_STATE .EQ. EXTRA_Y ) THEN
            DO I = 1, N
               Y_TAIL( I ) = 0.0D+0
            END DO
         END IF

         DXRAT = 0.0D+0
         DXRATMAX = 0.0D+0
         DZRAT = 0.0D+0
         DZRATMAX = 0.0D+0
         FINAL_DX_X = HUGEVAL
         FINAL_DZ_Z = HUGEVAL
```

```
         PREVNORMDX = HUGEVAL
         PREV_DZ_Z = HUGEVAL
         DZ_Z = HUGEVAL
         DX_X = HUGEVAL

         X_STATE = WORKING_STATE
         Z_STATE = UNSTABLE_STATE
         INCR_PREC = .FALSE.

         DO CNT = 1, ITHRESH
*
*        Compute residual RES = B_s - op(A_s) * Y,
*            op(A) = A, A**T, or A**H depending on TRANS (and type).
*
            CALL DCOPY( N, B( 1, J ), 1, RES, 1 )
            IF ( Y_PREC_STATE .EQ. BASE_RESIDUAL ) THEN
               CALL DGBMV( TRANS, M, N, KL, KU, -1.0D+0, AB, LDAB,
     $              Y( 1, J ), 1, 1.0D+0, RES, 1 )
            ELSE IF ( Y_PREC_STATE .EQ. EXTRA_RESIDUAL ) THEN
               CALL BLAS_DGBMV_X( TRANS_TYPE, N, N, KL, KU,
     $              -1.0D+0, AB, LDAB, Y( 1, J ), 1, 1.0D+0, RES, 1,
     $              PREC_TYPE )
            ELSE
               CALL BLAS_DGBMV2_X( TRANS_TYPE, N, N, KL, KU, -1.0D+0,
     $              AB, LDAB, Y( 1, J ), Y_TAIL, 1, 1.0D+0, RES, 1,
     $              PREC_TYPE )
            END IF

!        XXX: RES is no longer needed.
            CALL DCOPY( N, RES, 1, DY, 1 )
            CALL DGBTRS( TRANS, N, KL, KU, 1, AFB, LDAFB, IPIV, DY, N,
     $           INFO )
*
*         Calculate relative changes DX_X, DZ_Z and ratios DXRAT, DZRAT.
*
            NORMX = 0.0D+0
            NORMY = 0.0D+0
            NORMDX = 0.0D+0
            DZ_Z = 0.0D+0
            YMIN = HUGEVAL

            DO I = 1, N
               YK = ABS( Y( I, J ) )
               DYK = ABS( DY( I ) )

               IF ( YK .NE. 0.0D+0 ) THEN
                  DZ_Z = MAX( DZ_Z, DYK / YK )
               ELSE IF ( DYK .NE. 0.0D+0 ) THEN
                  DZ_Z = HUGEVAL
               END IF
```

```
               YMIN = MIN( YMIN, YK )

               NORMY = MAX( NORMY, YK )

               IF ( COLEQU ) THEN
                  NORMX = MAX( NORMX, YK * C( I ) )
                  NORMDX = MAX( NORMDX, DYK * C( I ) )
               ELSE
                  NORMX = NORMY
                  NORMDX = MAX( NORMDX, DYK )
               END IF
            END DO

            IF ( NORMX .NE. 0.0D+0 ) THEN
               DX_X = NORMDX / NORMX
            ELSE IF ( NORMDX .EQ. 0.0D+0 ) THEN
               DX_X = 0.0D+0
            ELSE
               DX_X = HUGEVAL
            END IF

            DXRAT = NORMDX / PREVNORMDX
            DZRAT = DZ_Z / PREV_DZ_Z
*
*         Check termination criteria.
*
            IF ( .NOT.IGNORE_CWISE
     $           .AND. YMIN*RCOND .LT. INCR_THRESH*NORMY
     $           .AND. Y_PREC_STATE .LT. EXTRA_Y )
     $           INCR_PREC = .TRUE.

            IF ( X_STATE .EQ. NOPROG_STATE .AND. DXRAT .LE. RTHRESH )
     $           X_STATE = WORKING_STATE
            IF ( X_STATE .EQ. WORKING_STATE ) THEN
               IF ( DX_X .LE. EPS ) THEN
                  X_STATE = CONV_STATE
               ELSE IF ( DXRAT .GT. RTHRESH ) THEN
                  IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                     INCR_PREC = .TRUE.
                  ELSE
                     X_STATE = NOPROG_STATE
                  END IF
               ELSE
                  IF ( DXRAT .GT. DXRATMAX ) DXRATMAX = DXRAT
               END IF
               IF ( X_STATE .GT. WORKING_STATE ) FINAL_DX_X = DX_X
            END IF

            IF ( Z_STATE .EQ. UNSTABLE_STATE .AND. DZ_Z .LE. DZ_UB )
```

```
     $           Z_STATE = WORKING_STATE
            IF ( Z_STATE .EQ. NOPROG_STATE .AND. DZRAT .LE. RTHRESH )
     $           Z_STATE = WORKING_STATE
            IF ( Z_STATE .EQ. WORKING_STATE ) THEN
               IF ( DZ_Z .LE. EPS ) THEN
                  Z_STATE = CONV_STATE
               ELSE IF ( DZ_Z .GT. DZ_UB ) THEN
                  Z_STATE = UNSTABLE_STATE
                  DZRATMAX = 0.0D+0
                  FINAL_DZ_Z = HUGEVAL
               ELSE IF ( DZRAT .GT. RTHRESH ) THEN
                  IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                     INCR_PREC = .TRUE.
                  ELSE
                     Z_STATE = NOPROG_STATE
                  END IF
               ELSE
                  IF ( DZRAT .GT. DZRATMAX ) DZRATMAX = DZRAT
               END IF
               IF ( Z_STATE .GT. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
            END IF
*
*           Exit if both normwise and componentwise stopped working,
*           but if componentwise is unstable, let it go at least two
*           iterations.
*
            IF ( X_STATE.NE.WORKING_STATE ) THEN
               IF ( IGNORE_CWISE ) GOTO 666
               IF ( Z_STATE.EQ.NOPROG_STATE .OR. Z_STATE.EQ.CONV_STATE )
     $              GOTO 666
               IF ( Z_STATE.EQ.UNSTABLE_STATE .AND. CNT.GT.1 ) GOTO 666
            END IF

            IF ( INCR_PREC ) THEN
               INCR_PREC = .FALSE.
               Y_PREC_STATE = Y_PREC_STATE + 1
               DO I = 1, N
                  Y_TAIL( I ) = 0.0D+0
               END DO
            END IF

            PREVNORMDX = NORMDX
            PREV_DZ_Z = DZ_Z
*
*           Update soluton.
*
            IF (Y_PREC_STATE .LT. EXTRA_Y) THEN
               CALL DAXPY( N, 1.0D+0, DY, 1, Y(1,J), 1 )
            ELSE
               CALL DLA_WWADDW( N, Y(1,J), Y_TAIL, DY )
```

```
            END IF

         END DO
*        Target of "IF (Z_STOP .AND. X_STOP)".  Sun's f77 won't EXIT.
 666     CONTINUE
*
*     Set final_* when cnt hits ithresh.
*
         IF ( X_STATE .EQ. WORKING_STATE ) FINAL_DX_X = DX_X
         IF ( Z_STATE .EQ. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
*
*     Compute error bounds.
*
         IF ( N_NORMS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) =
     $           FINAL_DX_X / (1 - DXRATMAX)
         END IF
         IF (N_NORMS .GE. 2) THEN
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) =
     $           FINAL_DZ_Z / (1 - DZRATMAX)
         END IF
*
*     Compute componentwise relative backward error from formula
*         max(i) ( abs(R(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
*     where abs(Z) is the componentwise absolute value of the matrix
*     or vector Z.
*
*        Compute residual RES = B_s - op(A_s) * Y,
*            op(A) = A, A**T, or A**H depending on TRANS (and type).
*
         CALL DCOPY( N, B( 1, J ), 1, RES, 1 )
         CALL DGBMV(TRANS, N, N, KL, KU, -1.0D+0, AB, LDAB, Y(1,J),
     $        1, 1.0D+0, RES, 1 )

         DO I = 1, N
            AYB( I ) = ABS( B( I, J ) )
         END DO
*
*     Compute abs(op(A_s))*abs(Y) + abs(B_s).
*
        CALL DLA_GBAMV( TRANS_TYPE, N, N, KL, KU, 1.0D+0,
     $        AB, LDAB, Y(1, J), 1, 1.0D+0, AYB, 1 )

         CALL DLA_LIN_BERR( N, N, 1, RES, AYB, BERR_OUT( J ) )
*
*     End of loop for each RHS
*
      END DO
*
      RETURN
```

```
      END
```

———-

44.14.4 lisp code

— **dla_gbrfsx_extended.lisp** —

```
(let* ((unstable_state 0)
       (working_state 1)
       (conv_state 2)
       (noprog_state 3)
       (base_residual 0)
       (extra_residual 1)
       (extra_y 2)
       (la_linrx_err_i 2))
  (declare (type (f2cl-lib:integer4 0 0) unstable_state)
           (type (f2cl-lib:integer4 1 1) working_state)
           (type (f2cl-lib:integer4 2 2) conv_state)
           (type (f2cl-lib:integer4 3 3) noprog_state)
           (type (f2cl-lib:integer4 0 0) base_residual)
           (type (f2cl-lib:integer4 1 1) extra_residual)
           (type (f2cl-lib:integer4 2 2) extra_y)
           (type (f2cl-lib:integer4 2 2) la_linrx_err_i)
           (ignorable unstable_state working_state conv_state noprog_state
            base_residual extra_residual extra_y la_linrx_err_i))
  (defun dla_gbrfsx_extended
         (prec_type trans_type n kl ku nrhs ab ldab afb ldafb ipiv colequ c b
          ldb$ y ldy berr_out n_norms err_bnds_norm err_bnds_comp res ayb dy
          y_tail rcond ithresh rthresh dz_ub ignore_cwise info)
    (declare (type (double-float) dz_ub rthresh rcond)
             (type f2cl-lib:logical ignore_cwise colequ)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array double-float (*)) y_tail dy ayb res err_bnds_comp
              err_bnds_norm berr_out y b c afb ab)
             (type (f2cl-lib:integer4) info ithresh n_norms ldy ldb$ ldafb ldab
              nrhs ku kl n trans_type prec_type))
    (f2cl-lib:with-multi-array-data
        ((ab double-float ab-%data% ab-%offset%)
         (afb double-float afb-%data% afb-%offset%)
         (c double-float c-%data% c-%offset%)
         (b double-float b-%data% b-%offset%)
         (y double-float y-%data% y-%offset%)
         (berr_out double-float berr_out-%data% berr_out-%offset%)
         (err_bnds_norm double-float err_bnds_norm-%data%
          err_bnds_norm-%offset%)
         (err_bnds_comp double-float err_bnds_comp-%data%
          err_bnds_comp-%offset%)
```

```
    (res double-float res-%data% res-%offset%)
    (ayb double-float ayb-%data% ayb-%offset%)
    (dy double-float dy-%data% dy-%offset%)
    (y_tail double-float y_tail-%data% y_tail-%offset%)
    (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
  (prog ((incr_prec nil) (yk 0.0d0) (dyk 0.0d0) (ymin 0.0d0) (normy 0.0d0)
         (normx 0.0d0) (normdx 0.0d0) (dxrat 0.0d0) (dzrat 0.0d0)
         (prevnormdx 0.0d0) (prev_dz_z 0.0d0) (dxratmax 0.0d0)
         (dzratmax 0.0d0) (dx_x 0.0d0) (dz_z 0.0d0) (final_dx_x 0.0d0)
         (final_dz_z 0.0d0) (eps 0.0d0) (hugeval 0.0d0) (incr_thresh 0.0d0)
         (cnt 0) (i 0) (j 0) (m 0) (x_state 0) (z_state 0) (y_prec_state 0)
         (trans
          (make-array '(1) :element-type 'character :initial-element #\ )))
    (declare (type f2cl-lib:logical incr_prec)
             (type (double-float) yk dyk ymin normy normx normdx dxrat
              dzrat prevnormdx prev_dz_z dxratmax dzratmax dx_x dz_z
              final_dx_x final_dz_z eps hugeval incr_thresh)
             (type (f2cl-lib:integer4) cnt i j m x_state z_state
              y_prec_state)
             (type (string 1) trans))
    (if (/= info 0)
        (go end_label))
    (f2cl-lib:f2cl-set-string trans
                              (f2cl-lib:fref-string #'chla_transtype
                                                    (trans_type))
                              (string 1))
    (setf eps (dlamch "Epsilon"))
    (setf hugeval (dlamch "Overflow"))
    (setf hugeval (* hugeval hugeval))
    (setf incr_thresh (* (f2cl-lib:dble n) eps))
    (setf m (f2cl-lib:int-add kl ku 1))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (setf y_prec_state extra_residual)
        (cond
         ((= y_prec_state extra_y)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                                   y_tail-%offset%)
                    0.0d0)
             label100001))))
        (setf dxrat 0.0d0)
        (setf dxratmax 0.0d0)
        (setf dzrat 0.0d0)
        (setf dzratmax 0.0d0)
        (setf final_dx_x hugeval)
        (setf final_dz_z hugeval)
```

```
(setf prevnormdx hugeval)
(setf prev_dz_z hugeval)
(setf dz_z hugeval)
(setf dx_x hugeval)
(setf x_state working_state)
(setf z_state unstable_state)
(setf incr_prec f2cl-lib:%false%)
(f2cl-lib:fdo (cnt 1 (f2cl-lib:int-add cnt 1))
              ((> cnt ithresh) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy n
         (f2cl-lib:array-slice b-%data% double-float (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1 res 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (cond
     ((= y_prec_state base_residual)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (dgbmv trans m n kl ku -1.0d0 ab ldab
           (f2cl-lib:array-slice y-%data% double-float (1 j)
                                 ((1 ldy) (1 *)) y-%offset%)
           1 1.0d0 res 1)
        (declare
         (ignore var-5 var-6 var-8 var-9 var-10 var-11 var-12))
        (when var-0 (setf trans var-0))
        (when var-1 (setf m var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf kl var-3))
        (when var-4 (setf ku var-4))
        (when var-7 (setf ldab var-7))))
     ((= y_prec_state extra_residual)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12 var-13)
          (blas_dgbmv_x trans_type n n kl ku -1.0d0 ab ldab
           (f2cl-lib:array-slice y-%data% double-float (1 j)
                                 ((1 ldy) (1 *)) y-%offset%)
           1 1.0d0 res 1 prec_type)
        (declare
         (ignore var-5 var-6 var-8 var-9 var-10 var-11 var-12))
        (when var-0 (setf trans_type var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf kl var-3))
        (when var-4 (setf ku var-4))
        (when var-7 (setf ldab var-7))
```

```
    (when var-13 (setf prec_type var-13))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13 var-14)
      (blas_dgbmv2_x trans_type n n kl ku -1.0d0 ab ldab
       (f2cl-lib:array-slice y-%data% double-float (1 j)
                             ((1 ldy) (1 *)) y-%offset%)
       y_tail 1 1.0d0 res 1 prec_type)
    (declare
     (ignore var-5 var-6 var-8 var-9 var-10 var-11 var-12
      var-13))
    (when var-0 (setf trans_type var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf kl var-3))
    (when var-4 (setf ku var-4))
    (when var-7 (setf ldab var-7))
    (when var-14 (setf prec_type var-14)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dcopy n res 1 dy 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgbtrs trans n kl ku 1 afb ldafb ipiv dy n info)
  (declare (ignore var-2 var-3 var-4 var-5 var-7 var-8))
  (setf trans var-0)
  (setf n var-1)
  (setf ldafb var-6)
  (setf n var-9)
  (setf info var-10))
(setf normx 0.0d0)
(setf normy 0.0d0)
(setf normdx 0.0d0)
(setf dz_z 0.0d0)
(setf ymin hugeval)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf yk
            (abs
             (f2cl-lib:fref y-%data% (i j) ((1 ldy) (1 *))
                            y-%offset%)))
    (setf dyk
            (abs
             (f2cl-lib:fref dy-%data% (i) ((1 *))
                            dy-%offset%)))
    (cond ((/= yk 0.0d0) (setf dz_z (max dz_z (/ dyk yk))))
```

```
          ((/= dyk 0.0d0) (setf dz_z hugeval)))
    (setf ymin (min ymin yk))
    (setf normy (max normy yk))
    (cond
     (colequ
      (setf normx
              (max normx
                   (* yk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%))))
      (setf normdx
              (max normdx
                   (* dyk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%)))))
     (t (setf normx normy) (setf normdx (max normdx dyk))))
   label100003))
(cond ((/= normx 0.0d0) (setf dx_x (/ normdx normx)))
      ((= normdx 0.0d0) (setf dx_x 0.0d0))
      (t (setf dx_x hugeval)))
(setf dxrat (/ normdx prevnormdx))
(setf dzrat (/ dz_z prev_dz_z))
(if (and (not ignore_cwise)
         (< (* ymin rcond) (* incr_thresh normy))
         (< y_prec_state extra_y))
    (setf incr_prec f2cl-lib:%true%))
(if (and (= x_state noprog_state) (<= dxrat rthresh))
    (setf x_state working_state))
(cond
 ((= x_state working_state)
  (cond ((<= dx_x eps) (setf x_state conv_state))
        ((> dxrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf x_state noprog_state))))
        (t
         (if (> dxrat dxratmax)
             (setf dxratmax dxrat))))
  (if (> x_state working_state)
      (setf final_dx_x dx_x))))
(if (and (= z_state unstable_state) (<= dz_z dz_ub))
    (setf z_state working_state))
(if (and (= z_state noprog_state) (<= dzrat rthresh))
    (setf z_state working_state))
(cond
 ((= z_state working_state)
  (cond ((<= dz_z eps) (setf z_state conv_state))
        ((> dz_z dz_ub) (setf z_state unstable_state)
         (setf dzratmax 0.0d0) (setf final_dz_z hugeval))
```

```
                ((> dzrat rthresh)
                 (cond
                  ((/= y_prec_state extra_y)
                   (setf incr_prec f2cl-lib:%true%))
                  (t (setf z_state noprog_state))))
                (t
                 (if (> dzrat dzratmax)
                     (setf dzratmax dzrat))))
          (if (> z_state working_state)
              (setf final_dz_z dz_z))))
        (cond
         ((/= x_state working_state)
          (if ignore_cwise
              (go label666))
          (if (or (= z_state noprog_state) (= z_state conv_state))
              (go label666))
          (if (and (= z_state unstable_state) (> cnt 1))
              (go label666))))
        (cond
         (incr_prec (setf incr_prec f2cl-lib:%false%)
          (setf y_prec_state (f2cl-lib:int-add y_prec_state 1))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                                   y_tail-%offset%)
                    0.0d0)
             label100004))))
        (setf prevnormdx normdx)
        (setf prev_dz_z dz_z)
        (cond
         ((< y_prec_state extra_y)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
              (daxpy n 1.0d0 dy 1
               (f2cl-lib:array-slice y-%data% double-float (1 j)
                                     ((1 ldy) (1 *)) y-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3 var-4 var-5))
            (when var-0 (setf n var-0))))
         (t
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (dla_wwaddw n
               (f2cl-lib:array-slice y-%data% double-float (1 j)
                                     ((1 ldy) (1 *)) y-%offset%)
               y_tail dy)
            (declare (ignore var-1 var-2 var-3))
            (when var-0 (setf n var-0)))))
      label100002))
label666
 (if (= x_state working_state)
```

```
    (setf final_dx_x dx_x))
(if (= z_state working_state)
    (setf final_dz_z dz_z))
(cond
 ((>= n_norms 1)
  (setf (f2cl-lib:fref err_bnds_norm-%data% (j la_linrx_err_i)
                       ((1 nrhs) (1 *)) err_bnds_norm-%offset%)
          (/ final_dx_x (- 1 dxratmax)))))
(cond
 ((>= n_norms 2)
  (setf (f2cl-lib:fref err_bnds_comp-%data% (j la_linrx_err_i)
                       ((1 nrhs) (1 *)) err_bnds_comp-%offset%)
          (/ final_dz_z (- 1 dzratmax)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dcopy n
     (f2cl-lib:array-slice b-%data% double-float (1 j)
                           ((1 ldb$) (1 *)) b-%offset%)
     1 res 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dgbmv trans n n kl ku -1.0d0 ab ldab
     (f2cl-lib:array-slice y-%data% double-float (1 j)
                           ((1 ldy) (1 *)) y-%offset%)
     1 1.0d0 res 1)
  (declare (ignore var-5 var-6 var-8 var-9 var-10 var-11 var-12))
  (when var-0 (setf trans var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf kl var-3))
  (when var-4 (setf ku var-4))
  (when var-7 (setf ldab var-7)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref ayb-%data% (i) ((1 *)) ayb-%offset%)
            (abs
             (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                            b-%offset%)))
   label100005))
(dla_gbamv trans_type n n kl ku 1.0d0 ab ldab
 (f2cl-lib:array-slice y-%data% double-float (1 j) ((1 ldy) (1 *))
                       y-%offset%)
 1 1.0d0 ayb 1)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (dla_lin_berr n n 1 res ayb
     (f2cl-lib:array-slice berr_out-%data% double-float (j) ((1 *))
                           berr_out-%offset%))
```

```
              (declare (ignore var-2 var-3 var-4 var-5))
              (when var-0 (setf n var-0))
              (when var-1 (setf n var-1)))
           label100000))
        (go end_label)
       end_label
        (return
         (values prec_type trans_type n kl ku nil nil ldab nil ldafb nil nil
                 nil nil nil nil nil nil nil nil nil nil nil nil nil nil nil
                 nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dla_gbrfsx_extended
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              fortran-to-lisp::logical
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (double-float)
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              fortran-to-lisp::logical
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::prec_type
```

```
                                                  fortran-to-lisp::trans_type
                                                  fortran-to-lisp::n
                                                  fortran-to-lisp::kl
                                                  fortran-to-lisp::ku nil nil
                                                  fortran-to-lisp::ldab nil
                                                  fortran-to-lisp::ldafb nil nil
                                                  nil nil nil nil nil nil nil nil
                                                  nil nil nil nil nil nil nil nil
                                                  nil nil fortran-to-lisp::info)
                                                :calls
                                                '(fortran-to-lisp::dla_gbamv
                                                  fortran-to-lisp::dgbtrs))))
```

———-

44.15 dla_gbrpvgrw

44.15.1 Axiom unit tests

— dlagbrpvgrw.input —

```
)set break resume
)sys rm -f dlagbrpvgrw.output
)spool dlagbrpvgrw.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

44.15.2 Axiom help page

— dla_gbrpvgrw.help —

```
==========================================================================
dla_gbrpvgrw examples
==========================================================================

==========================================================================
```

```
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function dla_gbrpvgrw (N, KL, KU, NCOLS, AB, LDAB,
           AFB, LDAFB)
           DLA_GBRPVGRW computes the reciprocal pivot growth factor
           norm(A)/norm(U) for a general banded matrix.

Function/Subroutine Documentation
   double precision function dla_gbrpvgrw (integer N, integer KL, integer KU,
       integer NCOLS, double precision, dimension( ldab, * ) AB, integer LDAB,
       double precision, dimension( ldafb, * ) AFB, integer LDAFB)
       DLA_GBRPVGRW computes the reciprocal pivot growth factor
       norm(A)/norm(U) for a general banded matrix.

Purpose:
   DLA_GBRPVGRW computes the reciprocal pivot growth factor
   norm(A)/norm(U). The "max absolute element" norm is used. If this is
   much less than 1, the stability of the LU factorization of the
   (equilibrated) matrix A could be poor. This also means that the
   solution X, estimated condition numbers, and error bounds could be
   unreliable.

Parameters:

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 NCOLS
   NCOLS is INTEGER
   The number of columns of the matrix A.  NCOLS >= 0.

 AB
   AB is DOUBLE PRECISION array, dimension (LDAB,N)
   On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
   The j-th column of A is stored in the j-th column of the
   array AB as follows:
   AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)
```

```
LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is DOUBLE PRECISION array, dimension (LDAFB,N)
  Details of the LU factorization of the band matrix A, as
  computed by DGBTRF.  U is stored as an upper triangular
  band matrix with KL+KU superdiagonals in rows 1 to KL+KU+1,
  and the multipliers used during the factorization are stored
  in rows KL+KU+2 to 2*KL+KU+1.

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL+KU+1.
```

44.15.3 fortran code

— dla_gbrpvgrw.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION DLA_GBRPVGRW( N, KL, KU, NCOLS, AB,
     $                                        LDAB, AFB, LDAFB )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            N, KL, KU, NCOLS, LDAB, LDAFB
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   AB( LDAB, * ), AFB( LDAFB, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J, KD
      DOUBLE PRECISION   AMAX, UMAX, RPVGRW
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
```

```
*     .. Executable Statements ..
*
      RPVGRW = 1.0D+0

      KD = KU + 1
      DO J = 1, NCOLS
         AMAX = 0.0D+0
         UMAX = 0.0D+0
         DO I = MAX( J-KU, 1 ), MIN( J+KL, N )
            AMAX = MAX( ABS( AB( KD+I-J, J)), AMAX )
         END DO
         DO I = MAX( J-KU, 1 ), J
            UMAX = MAX( ABS( AFB( KD+I-J, J ) ), UMAX )
         END DO
         IF ( UMAX /= 0.0D+0 ) THEN
            RPVGRW = MIN( AMAX / UMAX, RPVGRW )
         END IF
      END DO
      DLA_GBRPVGRW = RPVGRW
      END
```

44.15.4 lisp code

— dla_gbrpvgrw.lisp —

```
(defun dla_gbrpvgrw (n kl ku ncols ab ldab afb ldafb)
  (declare (type (array double-float (*)) afb ab)
           (type (f2cl-lib:integer4) ldafb ldab ncols ku kl n))
  (f2cl-lib:with-multi-array-data
      ((ab double-float ab-%data% ab-%offset%)
       (afb double-float afb-%data% afb-%offset%))
    (prog ((amax 0.0d0) (umax 0.0d0) (rpvgrw 0.0d0) (i 0) (j 0) (kd 0)
           (dla_gbrpvgrw 0.0d0))
      (declare (type (f2cl-lib:integer4) kd j i)
               (type (double-float) dla_gbrpvgrw rpvgrw umax amax))
      (setf rpvgrw 1.0d0)
      (setf kd (f2cl-lib:int-add ku 1))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j ncols) nil)
        (tagbody
          (setf amax 0.0d0)
          (setf umax 0.0d0)
          (f2cl-lib:fdo (i
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
```

```
                        (the f2cl-lib:integer4 1))
                       (f2cl-lib:int-add i 1))
                      ((> i
                          (min
                           (the f2cl-lib:integer4 (f2cl-lib:int-add j kl))
                           (the f2cl-lib:integer4 n)))
                       nil)
            (tagbody
              (setf amax
                      (max
                       (abs
                        (f2cl-lib:fref ab-%data%
                                       ((f2cl-lib:int-sub
                                         (f2cl-lib:int-add kd i) j)
                                        j)
                                       ((1 ldab) (1 *)) ab-%offset%))
                       amax))
             label100001))
          (f2cl-lib:fdo (i
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                          (the f2cl-lib:integer4 1))
                         (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf umax
                      (max
                       (abs
                        (f2cl-lib:fref afb-%data%
                                       ((f2cl-lib:int-sub
                                         (f2cl-lib:int-add kd i) j)
                                        j)
                                       ((1 ldafb) (1 *)) afb-%offset%))
                       umax))
             label100002))
          (cond
           ((f2cl-lib:f2cl/ umax (= 0.0d0))
            (setf rpvgrw (min (/ amax umax) rpvgrw))))
         label100000))
      (setf dla_gbrpvgrw rpvgrw)
     end_label
      (return (values dla_gbrpvgrw nil nil nil nil nil nil nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dla_gbrpvgrw
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
```

```
'((fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil nil nil nil nil nil)
:calls 'nil)))
```

———-

44.16 dorgbr

44.16.1 Axiom unit tests

— dorgbr.input —

```
)set break resume
)sys rm -f dorgbr.output
)spool dorgbr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

44.16.2 Axiom help page

— dorgbr.help —

```
======================================================================
dorgbr examples
======================================================================

======================================================================
Man Page Details
```

```
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dorgbr (VECT, M, N, K, A, LDA, TAU, WORK, LWORK, INFO)
           DORGBR

Function/Subroutine Documentation
   subroutine dorgbr (character VECT, integer M, integer N, integer K, double
       precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( * ) TAU, double precision, dimension( * ) WORK, integer
       LWORK, integer INFO)
       DORGBR

Purpose:
   DORGBR generates one of the real orthogonal matrices Q or P**T
   determined by DGEBRD when reducing a real matrix A to bidiagonal
   form: A = Q * B * P**T.  Q and P**T are defined as products of
   elementary reflectors H(i) or G(i) respectively.

   If VECT = 'Q', A is assumed to have been an M-by-K matrix, and Q
   is of order M:
   if m >= k, Q = H(1) H(2) . . . H(k) and DORGBR returns the first n
   columns of Q, where m >= n >= k;
   if m < k, Q = H(1) H(2) . . . H(m-1) and DORGBR returns Q as an
   M-by-M matrix.

   If VECT = 'P', A is assumed to have been a K-by-N matrix, and P**T
   is of order N:
   if k < n, P**T = G(k) . . . G(2) G(1) and DORGBR returns the first m
   rows of P**T, where n >= m >= k;
   if k >= n, P**T = G(n-1) . . . G(2) G(1) and DORGBR returns P**T as
   an N-by-N matrix.

Parameters:

 VECT
   VECT is CHARACTER*1
   Specifies whether the matrix Q or the matrix P**T is
   required, as defined in the transformation applied by DGEBRD:
   = 'Q':  generate Q;
   = 'P':  generate P**T.

 M
   M is INTEGER
   The number of rows of the matrix Q or P**T to be returned.
   M >= 0.

 N
   N is INTEGER
```

```
  The number of columns of the matrix Q or P**T to be returned.
  N >= 0.
  If VECT = 'Q', M >= N >= min(M,K);
  if VECT = 'P', N >= M >= min(N,K).

K
  K is INTEGER
  If VECT = 'Q', the number of columns in the original M-by-K
  matrix reduced by DGEBRD.
  If VECT = 'P', the number of rows in the original K-by-N
  matrix reduced by DGEBRD.
  K >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the vectors which define the elementary reflectors,
  as returned by DGEBRD.
  On exit, the M-by-N matrix Q or P**T.

LDA
  LDA is INTEGER
  The leading dimension of the array A. LDA >= max(1,M).

TAU
  TAU is DOUBLE PRECISION array, dimension
     (min(M,K)) if VECT = 'Q'
     (min(N,K)) if VECT = 'P'
  TAU(i) must contain the scalar factor of the elementary
  reflector H(i) or G(i), which determines Q or P**T, as
  returned by DGEBRD in its array argument TAUQ or TAUP.

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK >= max(1,min(M,N)).
  For optimum performance LWORK >= min(M,N)*NB, where NB
  is the optimal blocksize.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

44.16.3 fortran code

— **dorgbr.f** —

```
*  =====================================================================
      SUBROUTINE DORGBR( VECT, M, N, K, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          VECT
      INTEGER            INFO, K, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, WANTQ
      INTEGER            I, IINFO, J, LWKOPT, MN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DORGLQ, DORGQR, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
```

```
*
      INFO = 0
      WANTQ = LSAME( VECT, 'Q' )
      MN = MIN( M, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.WANTQ .AND. .NOT.LSAME( VECT, 'P' ) ) THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 .OR. ( WANTQ .AND. ( N.GT.M .OR. N.LT.MIN( M,
     $         K ) ) ) .OR. ( .NOT.WANTQ .AND. ( M.GT.N .OR. M.LT.
     $         MIN( N, K ) ) ) ) THEN
         INFO = -3
      ELSE IF( K.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -6
      ELSE IF( LWORK.LT.MAX( 1, MN ) .AND. .NOT.LQUERY ) THEN
         INFO = -9
      END IF
*
      IF( INFO.EQ.0 ) THEN
         WORK( 1 ) = 1
         IF( WANTQ ) THEN
            IF( M.GE.K ) THEN
               CALL DORGQR( M, N, K, A, LDA, TAU, WORK, -1, IINFO )
            ELSE
               IF( M.GT.1 ) THEN
                  CALL DORGQR( M-1, M-1, M-1, A( 2, 2 ), LDA, TAU, WORK,
     $                         -1, IINFO )
               END IF
            END IF
         ELSE
            IF( K.LT.N ) THEN
               CALL DORGLQ( M, N, K, A, LDA, TAU, WORK, -1, IINFO )
            ELSE
               IF( N.GT.1 ) THEN
                  CALL DORGLQ( N-1, N-1, N-1, A( 2, 2 ), LDA, TAU, WORK,
     $                         -1, IINFO )
               END IF
            END IF
         END IF
         LWKOPT = WORK( 1 )
         LWKOPT = MAX (LWKOPT, MN)
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DORGBR', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
```

```
         WORK( 1 ) = LWKOPT
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      IF( WANTQ ) THEN
*
*        Form Q, determined by a call to DGEBRD to reduce an m-by-k
*        matrix
*
         IF( M.GE.K ) THEN
*
*           If m >= k, assume m >= n >= k
*
            CALL DORGQR( M, N, K, A, LDA, TAU, WORK, LWORK, IINFO )
*
         ELSE
*
*           If m < k, assume m = n
*
*           Shift the vectors which define the elementary reflectors one
*           column to the right, and set the first row and column of Q
*           to those of the unit matrix
*
            DO 20 J = M, 2, -1
               A( 1, J ) = ZERO
               DO 10 I = J + 1, M
                  A( I, J ) = A( I, J-1 )
   10          CONTINUE
   20       CONTINUE
            A( 1, 1 ) = ONE
            DO 30 I = 2, M
               A( I, 1 ) = ZERO
   30       CONTINUE
            IF( M.GT.1 ) THEN
*
               Form Q(2:m,2:m)
*
               CALL DORGQR( M-1, M-1, M-1, A( 2, 2 ), LDA, TAU, WORK,
     $                      LWORK, IINFO )
            END IF
         END IF
      ELSE
*
```

```
*        Form P**T, determined by a call to DGEBRD to reduce a k-by-n
*        matrix
*
         IF( K.LT.N ) THEN
*
*           If k < n, assume k <= m <= n
*
            CALL DORGLQ( M, N, K, A, LDA, TAU, WORK, LWORK, IINFO )
*
         ELSE
*
*           If k >= n, assume m = n
*
*           Shift the vectors which define the elementary reflectors one
*           row downward, and set the first row and column of P**T to
*           those of the unit matrix
*
            A( 1, 1 ) = ONE
            DO 40 I = 2, N
               A( I, 1 ) = ZERO
   40       CONTINUE
            DO 60 J = 2, N
               DO 50 I = J - 1, 2, -1
                  A( I, J ) = A( I-1, J )
   50          CONTINUE
               A( 1, J ) = ZERO
   60       CONTINUE
            IF( N.GT.1 ) THEN
*
*              Form P**T(2:n,2:n)
*
               CALL DORGLQ( N-1, N-1, N-1, A( 2, 2 ), LDA, TAU, WORK,
     $                      LWORK, IINFO )
            END IF
         END IF
      END IF
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of DORGBR
*
      END
```

———-

44.16.4 lisp code

— dorgbr.lisp —

———-

Chapter 45

LAPACK - General Band Matrix, Comp. Routines, Real

45.1 sgbbrd

45.1.1 Axiom unit tests

— sgbbrd.input —

```
)set break resume
)sys rm -f sgbbrd.output
)spool sgbbrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

45.1.2 Axiom help page

— sgbbrd.help —

```
==========================================================================
sgbbrd examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgbbrd (VECT, M, N, NCC, KL, KU, AB, LDAB, D, E, Q, LDQ, PT,
           LDPT, C, LDC, WORK, INFO)
           SGBBRD

Function/Subroutine Documentation
   subroutine sgbbrd (character VECT, integer M, integer N, integer NCC,
       integer KL, integer KU, real, dimension( ldab, * ) AB, integer LDAB,
       real, dimension( * ) D, real, dimension( * ) E, real, dimension( ldq, *
       ) Q, integer LDQ, real, dimension( ldpt, * ) PT, integer LDPT, real,
       dimension( ldc, * ) C, integer LDC, real, dimension( * ) WORK, integer
       INFO)
       SGBBRD

Purpose:
   SGBBRD reduces a real general m-by-n band matrix A to upper
   bidiagonal form B by an orthogonal transformation: Q**T * A * P = B.

   The routine computes B, and optionally forms Q or P**T, or computes
   Q**T*C for a given matrix C.

Parameters:

 VECT
   VECT is CHARACTER*1
   Specifies whether or not the matrices Q and P**T are to be
   formed.
   = 'N': do not form Q or P**T;
   = 'Q': form Q only;
   = 'P': form P**T only;
   = 'B': form both.

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 NCC
   NCC is INTEGER
   The number of columns of the matrix C.  NCC >= 0.
```

```
KL
  KL is INTEGER
  The number of subdiagonals of the matrix A. KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals of the matrix A. KU >= 0.

AB
  AB is REAL array, dimension (LDAB,N)
  On entry, the m-by-n band matrix A, stored in rows 1 to
  KL+KU+1. The j-th column of A is stored in the j-th column of
  the array AB as follows:
  AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl).
  On exit, A is overwritten by values generated during the reduction.

LDAB
  LDAB is INTEGER
  The leading dimension of the array A. LDAB >= KL+KU+1.

D
  D is REAL array, dimension (min(M,N))
  The diagonal elements of the bidiagonal matrix B.

E
  E is REAL array, dimension (min(M,N)-1)
  The superdiagonal elements of the bidiagonal matrix B.

Q
  Q is REAL array, dimension (LDQ,M)
  If VECT = 'Q' or 'B', the m-by-m orthogonal matrix Q.
  If VECT = 'N' or 'P', the array Q is not referenced.

LDQ
  LDQ is INTEGER
  The leading dimension of the array Q.
  LDQ >= max(1,M) if VECT = 'Q' or 'B'; LDQ >= 1 otherwise.

PT
  PT is REAL array, dimension (LDPT,N)
  If VECT = 'P' or 'B', the n-by-n orthogonal matrix P'.
  If VECT = 'N' or 'Q', the array PT is not referenced.

LDPT
  LDPT is INTEGER
  The leading dimension of the array PT.
  LDPT >= max(1,N) if VECT = 'P' or 'B'; LDPT >= 1 otherwise.

C
  C is REAL array, dimension (LDC,NCC)
```

```
  On entry, an m-by-ncc matrix C.
  On exit, C is overwritten by Q**T*C.
  C is not referenced if NCC = 0.

LDC
  LDC is INTEGER
  The leading dimension of the array C.
  LDC >= max(1,M) if NCC > 0; LDC >= 1 if NCC = 0.

WORK
  WORK is REAL array, dimension (2*max(M,N))

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

———-

45.1.3 fortran code

— **sgbbrd.f** —

```
*  =====================================================================
      SUBROUTINE SGBBRD( VECT, M, N, NCC, KL, KU, AB, LDAB, D, E, Q,
     $                   LDQ, PT, LDPT, C, LDC, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          VECT
      INTEGER            INFO, KL, KU, LDAB, LDC, LDPT, LDQ, M, N, NCC
*     ..
*     .. Array Arguments ..
      REAL               AB( LDAB, * ), C( LDC, * ), D( * ), E( * ),
     $                   PT( LDPT, * ), Q( LDQ, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
```

```
*     .. Local Scalars ..
      LOGICAL            WANTB, WANTC, WANTPT, WANTQ
      INTEGER            I, INCA, J, J1, J2, KB, KB1, KK, KLM, KLU1,
     $                   KUN, L, MINMN, ML, ML0, MN, MU, MU0, NR, NRT
      REAL               RA, RB, RC, RS
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLARGV, SLARTG, SLARTV, SLASET, SROT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      WANTB = LSAME( VECT, 'B' )
      WANTQ = LSAME( VECT, 'Q' ) .OR. WANTB
      WANTPT = LSAME( VECT, 'P' ) .OR. WANTB
      WANTC = NCC.GT.0
      KLU1 = KL + KU + 1
      INFO = 0
      IF( .NOT.WANTQ .AND. .NOT.WANTPT .AND. .NOT.LSAME( VECT, 'N' ) )
     $     THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NCC.LT.0 ) THEN
         INFO = -4
      ELSE IF( KL.LT.0 ) THEN
         INFO = -5
      ELSE IF( KU.LT.0 ) THEN
         INFO = -6
      ELSE IF( LDAB.LT.KLU1 ) THEN
         INFO = -8
      ELSE IF( LDQ.LT.1 .OR. WANTQ .AND. LDQ.LT.MAX( 1, M ) ) THEN
         INFO = -12
      ELSE IF( LDPT.LT.1 .OR. WANTPT .AND. LDPT.LT.MAX( 1, N ) ) THEN
         INFO = -14
      ELSE IF( LDC.LT.1 .OR. WANTC .AND. LDC.LT.MAX( 1, M ) ) THEN
         INFO = -16
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGBBRD', -INFO )
```

```
         RETURN
      END IF
*
*     Initialize Q and P**T to the unit matrix, if needed
*
      IF( WANTQ )
     $   CALL SLASET( 'Full', M, M, ZERO, ONE, Q, LDQ )
      IF( WANTPT )
     $   CALL SLASET( 'Full', N, N, ZERO, ONE, PT, LDPT )
*
*     Quick return if possible.
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
      MINMN = MIN( M, N )
*
      IF( KL+KU.GT.1 ) THEN
*
*        Reduce to upper bidiagonal form if KU > 0; if KU = 0, reduce
*        first to lower bidiagonal form and then transform to upper
*        bidiagonal
*
         IF( KU.GT.0 ) THEN
            ML0 = 1
            MU0 = 2
         ELSE
            ML0 = 2
            MU0 = 1
         END IF
*
*        Wherever possible, plane rotations are generated and applied in
*        vector operations of length NR over the index set J1:J2:KLU1.
*
*        The sines of the plane rotations are stored in WORK(1:max(m,n))
*        and the cosines in WORK(max(m,n)+1:2*max(m,n)).
*
         MN = MAX( M, N )
         KLM = MIN( M-1, KL )
         KUN = MIN( N-1, KU )
         KB = KLM + KUN
         KB1 = KB + 1
         INCA = KB1*LDAB
         NR = 0
         J1 = KLM + 2
         J2 = 1 - KUN
*
         DO 90 I = 1, MINMN
*
*           Reduce i-th column and i-th row of matrix to bidiagonal form
```

```
*
            ML = KLM + 1
            MU = KUN + 1
            DO 80 KK = 1, KB
               J1 = J1 + KB
               J2 = J2 + KB
*
*              generate plane rotations to annihilate nonzero elements
*              which have been created below the band
*
               IF( NR.GT.0 )
     $            CALL SLARGV( NR, AB( KLU1, J1-KLM-1 ), INCA,
     $                         WORK( J1 ), KB1, WORK( MN+J1 ), KB1 )
*
*              apply plane rotations from the left
*
               DO 10 L = 1, KB
                  IF( J2-KLM+L-1.GT.N ) THEN
                     NRT = NR - 1
                  ELSE
                     NRT = NR
                  END IF
                  IF( NRT.GT.0 )
     $               CALL SLARTV( NRT, AB( KLU1-L, J1-KLM+L-1 ), INCA,
     $                            AB( KLU1-L+1, J1-KLM+L-1 ), INCA,
     $                            WORK( MN+J1 ), WORK( J1 ), KB1 )
   10          CONTINUE
*
               IF( ML.GT.ML0 ) THEN
                  IF( ML.LE.M-I+1 ) THEN
*
*                    generate plane rotation to annihilate a(i+ml-1,i)
*                    within the band, and apply rotation from the left
*
                     CALL SLARTG( AB( KU+ML-1, I ), AB( KU+ML, I ),
     $                            WORK( MN+I+ML-1 ), WORK( I+ML-1 ),
     $                            RA )
                     AB( KU+ML-1, I ) = RA
                     IF( I.LT.N )
     $                  CALL SROT( MIN( KU+ML-2, N-I ),
     $                             AB( KU+ML-2, I+1 ), LDAB-1,
     $                             AB( KU+ML-1, I+1 ), LDAB-1,
     $                             WORK( MN+I+ML-1 ), WORK( I+ML-1 ) )
                  END IF
                  NR = NR + 1
                  J1 = J1 - KB1
               END IF
*
               IF( WANTQ ) THEN
*
```

```
*                 accumulate product of plane rotations in Q
*
                  DO 20 J = J1, J2, KB1
                     CALL SROT( M, Q( 1, J-1 ), 1, Q( 1, J ), 1,
     $                          WORK( MN+J ), WORK( J ) )
   20             CONTINUE
               END IF
*
               IF( WANTC ) THEN
*
*                 apply plane rotations to C
*
                  DO 30 J = J1, J2, KB1
                     CALL SROT( NCC, C( J-1, 1 ), LDC, C( J, 1 ), LDC,
     $                          WORK( MN+J ), WORK( J ) )
   30             CONTINUE
               END IF
*
               IF( J2+KUN.GT.N ) THEN
*
*                 adjust J2 to keep within the bounds of the matrix
*
                  NR = NR - 1
                  J2 = J2 - KB1
               END IF
*
               DO 40 J = J1, J2, KB1
*
*                 create nonzero element a(j-1,j+ku) above the band
*                 and store it in WORK(n+1:2*n)
*
                  WORK( J+KUN ) = WORK( J )*AB( 1, J+KUN )
                  AB( 1, J+KUN ) = WORK( MN+J )*AB( 1, J+KUN )
   40          CONTINUE
*
*              generate plane rotations to annihilate nonzero elements
*              which have been generated above the band
*
               IF( NR.GT.0 )
     $            CALL SLARGV( NR, AB( 1, J1+KUN-1 ), INCA,
     $                         WORK( J1+KUN ), KB1, WORK( MN+J1+KUN ),
     $                         KB1 )
*
*              apply plane rotations from the right
*
               DO 50 L = 1, KB
                  IF( J2+L-1.GT.M ) THEN
                     NRT = NR - 1
                  ELSE
                     NRT = NR
```

```
                 END IF
                 IF( NRT.GT.0 )
     $              CALL SLARTV( NRT, AB( L+1, J1+KUN-1 ), INCA,
     $                           AB( L, J1+KUN ), INCA,
     $                           WORK( MN+J1+KUN ), WORK( J1+KUN ),
     $                           KB1 )
   50          CONTINUE
*
               IF( ML.EQ.ML0 .AND. MU.GT.MU0 ) THEN
                 IF( MU.LE.N-I+1 ) THEN
*
*                   generate plane rotation to annihilate a(i,i+mu-1)
*                   within the band, and apply rotation from the right
*
                    CALL SLARTG( AB( KU-MU+3, I+MU-2 ),
     $                           AB( KU-MU+2, I+MU-1 ),
     $                           WORK( MN+I+MU-1 ), WORK( I+MU-1 ),
     $                           RA )
                    AB( KU-MU+3, I+MU-2 ) = RA
                    CALL SROT( MIN( KL+MU-2, M-I ),
     $                         AB( KU-MU+4, I+MU-2 ), 1,
     $                         AB( KU-MU+3, I+MU-1 ), 1,
     $                         WORK( MN+I+MU-1 ), WORK( I+MU-1 ) )
                 END IF
                 NR = NR + 1
                 J1 = J1 - KB1
               END IF
*
               IF( WANTPT ) THEN
*
*                accumulate product of plane rotations in P**T
*
                 DO 60 J = J1, J2, KB1
                    CALL SROT( N, PT( J+KUN-1, 1 ), LDPT,
     $                         PT( J+KUN, 1 ), LDPT, WORK( MN+J+KUN ),
     $                         WORK( J+KUN ) )
   60            CONTINUE
               END IF
*
               IF( J2+KB.GT.M ) THEN
*
*                adjust J2 to keep within the bounds of the matrix
*
                 NR = NR - 1
                 J2 = J2 - KB1
               END IF
*
               DO 70 J = J1, J2, KB1
*
*                create nonzero element a(j+kl+ku,j+ku-1) below the
```

```
*                 band and store it in WORK(1:n)
*
                  WORK( J+KB ) = WORK( J+KUN )*AB( KLU1, J+KUN )
                  AB( KLU1, J+KUN ) = WORK( MN+J+KUN )*AB( KLU1, J+KUN )
   70          CONTINUE
*
               IF( ML.GT.ML0 ) THEN
                  ML = ML - 1
               ELSE
                  MU = MU - 1
               END IF
   80       CONTINUE
   90    CONTINUE
      END IF
*
      IF( KU.EQ.0 .AND. KL.GT.0 ) THEN
*
*        A has been reduced to lower bidiagonal form
*
*        Transform lower bidiagonal form to upper bidiagonal by applying
*        plane rotations from the left, storing diagonal elements in D
*        and off-diagonal elements in E
*
         DO 100 I = 1, MIN( M-1, N )
            CALL SLARTG( AB( 1, I ), AB( 2, I ), RC, RS, RA )
            D( I ) = RA
            IF( I.LT.N ) THEN
               E( I ) = RS*AB( 1, I+1 )
               AB( 1, I+1 ) = RC*AB( 1, I+1 )
            END IF
            IF( WANTQ )
     $         CALL SROT( M, Q( 1, I ), 1, Q( 1, I+1 ), 1, RC, RS )
            IF( WANTC )
     $         CALL SROT( NCC, C( I, 1 ), LDC, C( I+1, 1 ), LDC, RC,
     $                    RS )
  100    CONTINUE
         IF( M.LE.N )
     $      D( M ) = AB( 1, M )
      ELSE IF( KU.GT.0 ) THEN
*
*        A has been reduced to upper bidiagonal form
*
         IF( M.LT.N ) THEN
*
*           Annihilate a(m,m+1) by applying plane rotations from the
*           right, storing diagonal elements in D and off-diagonal
*           elements in E
*
            RB = AB( KU, M+1 )
            DO 110 I = M, 1, -1
```

```
               CALL SLARTG( AB( KU+1, I ), RB, RC, RS, RA )
               D( I ) = RA
               IF( I.GT.1 ) THEN
                  RB = -RS*AB( KU, I )
                  E( I-1 ) = RC*AB( KU, I )
               END IF
               IF( WANTPT )
     $            CALL SROT( N, PT( I, 1 ), LDPT, PT( M+1, 1 ), LDPT,
     $                       RC, RS )
  110       CONTINUE
         ELSE
*
*           Copy off-diagonal elements to E and diagonal elements to D
*
            DO 120 I = 1, MINMN - 1
               E( I ) = AB( KU, I+1 )
  120       CONTINUE
            DO 130 I = 1, MINMN
               D( I ) = AB( KU+1, I )
  130       CONTINUE
         END IF
      ELSE
*
*        A is diagonal. Set elements of E to zero and copy diagonal
*        elements to D.
*
         DO 140 I = 1, MINMN - 1
            E( I ) = ZERO
  140    CONTINUE
         DO 150 I = 1, MINMN
            D( I ) = AB( 1, I )
  150    CONTINUE
      END IF
      RETURN
*
*     End of SGBBRD
*
      END
```

———-

45.1.4 lisp code

— sgbbrd.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
```

```
          (ignorable zero one))
  (defun sgbbrd (vect m n ncc kl ku ab ldab d e q ldq pt ldpt c ldc work info)
    (declare (type (array single-float (*)) work c pt q e d ab)
             (type (f2cl-lib:integer4) info ldc ldpt ldq ldab ku kl ncc n m)
             (type (string *) vect))
    (f2cl-lib:with-multi-array-data
        ((vect character vect-%data% vect-%offset%)
         (ab single-float ab-%data% ab-%offset%)
         (d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%)
         (q single-float q-%data% q-%offset%)
         (pt single-float pt-%data% pt-%offset%)
         (c single-float c-%data% c-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((ra 0.0) (rb 0.0) (rc 0.0) (rs 0.0) (i 0) (inca 0) (j 0) (j1 0)
             (j2 0) (kb 0) (kb1 0) (kk 0) (klm 0) (klu1 0) (kun 0) (l 0)
             (minmn 0) (ml 0) (ml0 0) (mn 0) (mu 0) (mu0 0) (nr 0) (nrt 0)
             (wantb nil) (wantc nil) (wantpt nil) (wantq nil))
        (declare (type (single-float) ra rb rc rs)
                 (type (f2cl-lib:integer4) i inca j j1 j2 kb kb1 kk klm klu1
                  kun l minmn ml ml0 mn mu mu0 nr nrt)
                 (type f2cl-lib:logical wantb wantc wantpt wantq))
        (setf wantb
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame vect "B")
                  (declare (ignore var-1))
                  (when var-0 (setf vect var-0))
                  ret-val))
        (setf wantq
                (or
                 (multiple-value-bind (ret-val var-0 var-1)
                     (lsame vect "Q")
                   (declare (ignore var-1))
                   (when var-0 (setf vect var-0))
                   ret-val)
                 wantb))
        (setf wantpt
                (or
                 (multiple-value-bind (ret-val var-0 var-1)
                     (lsame vect "P")
                   (declare (ignore var-1))
                   (when var-0 (setf vect var-0))
                   ret-val)
                 wantb))
        (setf wantc (> ncc 0))
        (setf klu1 (f2cl-lib:int-add kl ku 1))
        (setf info 0)
        (cond
         ((and (not wantq) (not wantpt)
               (not
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame vect "N")
          (declare (ignore var-1))
          (when var-0 (setf vect var-0))
          ret-val)))
  (setf info -1))
 ((< m 0) (setf info -2)) ((< n 0) (setf info -3))
 ((< ncc 0) (setf info -4)) ((< kl 0) (setf info -5))
 ((< ku 0) (setf info -6)) ((< ldab klu1) (setf info -8))
 ((or (< ldq 1)
      (and wantq
           (< ldq
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 m)))))
  (setf info -12))
 ((or (< ldpt 1)
      (and wantpt
           (< ldpt
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 n)))))
  (setf info -14))
 ((or (< ldc 1)
      (and wantc
           (< ldc
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 m)))))
  (setf info -16)))
(cond
 ((/= info 0) (xerbla "SGBBRD" (f2cl-lib:int-sub info))
  (go end_label)))
(if wantq
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "Full" m m zero one q ldq)
      (declare (ignore var-0 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf m var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldq var-6))))
(if wantpt
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "Full" n n zero one pt ldpt)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldpt var-6))))
(if (or (= m 0) (= n 0))
    (go end_label))
```

```
(setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(cond
 ((> (f2cl-lib:int-add kl ku) 1)
  (cond ((> ku 0) (setf ml0 1) (setf mu0 2))
        (t (setf ml0 2) (setf mu0 1)))
  (setf mn (max (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
  (setf klm
          (min (the f2cl-lib:integer4 (f2cl-lib:int-sub m 1))
               (the f2cl-lib:integer4 kl)))
  (setf kun
          (min (the f2cl-lib:integer4 (f2cl-lib:int-sub n 1))
               (the f2cl-lib:integer4 ku)))
  (setf kb (f2cl-lib:int-add klm kun))
  (setf kb1 (f2cl-lib:int-add kb 1))
  (setf inca (f2cl-lib:int-mul kb1 ldab)) (setf nr 0)
  (setf j1 (f2cl-lib:int-add klm 2)) (setf j2 (f2cl-lib:int-sub 1 kun))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i minmn) nil)
    (tagbody
      (setf ml (f2cl-lib:int-add klm 1))
      (setf mu (f2cl-lib:int-add kun 1))
      (f2cl-lib:fdo (kk 1 (f2cl-lib:int-add kk 1))
                    ((> kk kb) nil)
        (tagbody
          (setf j1 (f2cl-lib:int-add j1 kb))
          (setf j2 (f2cl-lib:int-add j2 kb))
          (if (> nr 0)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (slargv nr
                   (f2cl-lib:array-slice ab-%data% single-float
                                         (klu1
                                          (f2cl-lib:int-sub j1 klm 1))
                                         ((1 ldab) (1 *)) ab-%offset%)
                   inca
                   (f2cl-lib:array-slice work-%data% single-float (j1)
                                         ((1 *)) work-%offset%)
                   kb1
                   (f2cl-lib:array-slice work-%data% single-float
                                         ((+ mn j1)) ((1 *))
                                         work-%offset%)
                   kb1)
                (declare (ignore var-1 var-3 var-5))
                (when var-0 (setf nr var-0))
                (when var-2 (setf inca var-2))
                (when var-4 (setf kb1 var-4))
                (when var-6 (setf kb1 var-6))))
          (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                        ((> l kb) nil)
            (tagbody
```

```
      (cond
       ((>
         (f2cl-lib:int-add j2 (f2cl-lib:int-sub klm) l
                           (f2cl-lib:int-sub 1))
         n)
        (setf nrt (f2cl-lib:int-sub nr 1)))
       (t (setf nrt nr)))
      (if (> nrt 0)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
              (slartv nrt
               (f2cl-lib:array-slice ab-%data% single-float
                                     ((+ klu1
                                         (f2cl-lib:int-sub l))
                                      (f2cl-lib:int-sub
                                       (f2cl-lib:int-add
                                        (f2cl-lib:int-sub j1
                                                          klm)
                                        l)
                                       1))
                                     ((1 ldab) (1 *))
                                     ab-%offset%)
               inca
               (f2cl-lib:array-slice ab-%data% single-float
                                     ((+ klu1
                                         (f2cl-lib:int-sub l)
                                         1)
                                      (f2cl-lib:int-sub
                                       (f2cl-lib:int-add
                                        (f2cl-lib:int-sub j1
                                                          klm)
                                        l)
                                       1))
                                     ((1 ldab) (1 *))
                                     ab-%offset%)
               inca
               (f2cl-lib:array-slice work-%data% single-float
                                     ((+ mn j1)) ((1 *))
                                     work-%offset%)
               (f2cl-lib:array-slice work-%data% single-float
                                     (j1) ((1 *))
                                     work-%offset%)
               kb1)
            (declare (ignore var-1 var-3 var-5 var-6))
            (when var-0 (setf nrt var-0))
            (when var-2 (setf inca var-2))
            (when var-4 (setf inca var-4))
            (when var-7 (setf kb1 var-7))))
    label10))
  (cond
```

```
((> ml ml0)
 (cond
  ((<= ml (f2cl-lib:int-add m (f2cl-lib:int-sub i) 1))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
       (slartg
        (f2cl-lib:array-slice ab-%data% single-float
                              ((+ ku ml
                                  (f2cl-lib:int-sub 1))
                               i)
                              ((1 ldab) (1 *)) ab-%offset%)
        (f2cl-lib:array-slice ab-%data% single-float
                              ((+ ku ml) i) ((1 ldab) (1 *))
                              ab-%offset%)
        (f2cl-lib:array-slice work-%data% single-float
                              ((+ mn i ml
                                  (f2cl-lib:int-sub 1)))
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float
                              ((+ i ml
                                  (f2cl-lib:int-sub 1)))
                              ((1 *)) work-%offset%)
        ra)
     (declare (ignore var-0 var-1 var-2 var-3))
     (when var-4 (setf ra var-4)))
   (setf (f2cl-lib:fref ab-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add ku ml) 1)
                         i)
                        ((1 ldab) (1 *)) ab-%offset%)
           ra)
   (if (< i n)
       (srot
        (min
         (the f2cl-lib:integer4
              (f2cl-lib:int-sub (f2cl-lib:int-add ku ml) 2))
         (the f2cl-lib:integer4 (f2cl-lib:int-sub n i)))
        (f2cl-lib:array-slice ab-%data% single-float
                              ((+ ku ml
                                  (f2cl-lib:int-sub 2))
                               (f2cl-lib:int-add i 1))
                              ((1 ldab) (1 *)) ab-%offset%)
        (f2cl-lib:int-sub ldab 1)
        (f2cl-lib:array-slice ab-%data% single-float
                              ((+ ku ml
                                  (f2cl-lib:int-sub 1))
                               (f2cl-lib:int-add i 1))
                              ((1 ldab) (1 *)) ab-%offset%)
        (f2cl-lib:int-sub ldab 1)
        (f2cl-lib:array-slice work-%data% single-float
                              ((+ mn i ml
```

```
                                    (f2cl-lib:int-sub 1)))
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% single-float
                                ((+ i ml
                                    (f2cl-lib:int-sub 1)))
                                ((1 *)) work-%offset%)))))
  (setf nr (f2cl-lib:int-add nr 1))
  (setf j1 (f2cl-lib:int-sub j1 kb1))))
(cond
 (wantq
  (f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
                ((> j j2) nil)
    (tagbody
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (srot m
           (f2cl-lib:array-slice q-%data% single-float
                                 (1 (f2cl-lib:int-sub j 1))
                                 ((1 ldq) (1 *)) q-%offset%)
           1
           (f2cl-lib:array-slice q-%data% single-float (1 j)
                                 ((1 ldq) (1 *)) q-%offset%)
           1
           (f2cl-lib:array-slice work-%data% single-float
                                 ((+ mn j)) ((1 *))
                                 work-%offset%)
           (f2cl-lib:array-slice work-%data% single-float (j)
                                 ((1 *)) work-%offset%))
        (declare
         (ignore var-1 var-2 var-3 var-4 var-5 var-6))
        (when var-0 (setf m var-0)))
     label20))))
(cond
 (wantc
  (f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
                ((> j j2) nil)
    (tagbody
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (srot ncc
           (f2cl-lib:array-slice c-%data% single-float
                                 ((+ j (f2cl-lib:int-sub 1))
                                  1)
                                 ((1 ldc) (1 *)) c-%offset%)
           ldc
           (f2cl-lib:array-slice c-%data% single-float (j 1)
                                 ((1 ldc) (1 *)) c-%offset%)
           ldc
           (f2cl-lib:array-slice work-%data% single-float
                                 ((+ mn j)) ((1 *))
```

```
                                    work-%offset%)
            (f2cl-lib:array-slice work-%data% single-float (j)
                                    ((1 *)) work-%offset%))
        (declare (ignore var-1 var-3 var-5 var-6))
        (when var-0 (setf ncc var-0))
        (when var-2 (setf ldc var-2))
        (when var-4 (setf ldc var-4)))
     label30))))
(cond
 ((> (f2cl-lib:int-add j2 kun) n)
  (setf nr (f2cl-lib:int-sub nr 1))
  (setf j2 (f2cl-lib:int-sub j2 kb1))))
(f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
              ((> j j2) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add j kun)) ((1 *))
                         work-%offset%)
            (*
             (f2cl-lib:fref work-%data% (j) ((1 *))
                            work-%offset%)
             (f2cl-lib:fref ab-%data%
                            (1 (f2cl-lib:int-add j kun))
                            ((1 ldab) (1 *)) ab-%offset%)))
    (setf (f2cl-lib:fref ab-%data%
                         (1 (f2cl-lib:int-add j kun))
                         ((1 ldab) (1 *)) ab-%offset%)
            (*
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add mn j)) ((1 *))
                            work-%offset%)
             (f2cl-lib:fref ab-%data%
                            (1 (f2cl-lib:int-add j kun))
                            ((1 ldab) (1 *)) ab-%offset%)))
   label40))
(if (> nr 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slargv nr
         (f2cl-lib:array-slice ab-%data% single-float
                               (1
                                (f2cl-lib:int-sub
                                 (f2cl-lib:int-add j1 kun)
                                 1))
                               ((1 ldab) (1 *)) ab-%offset%)
         inca
         (f2cl-lib:array-slice work-%data% single-float
                               ((+ j1 kun)) ((1 *))
                               work-%offset%)
         kb1
```

```
          (f2cl-lib:array-slice work-%data% single-float
                                ((+ mn j1 kun)) ((1 *))
                                work-%offset%)
          kb1)
       (declare (ignore var-1 var-3 var-5))
       (when var-0 (setf nr var-0))
       (when var-2 (setf inca var-2))
       (when var-4 (setf kb1 var-4))
       (when var-6 (setf kb1 var-6))))
(f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
              ((> l kb) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:int-add j2 l (f2cl-lib:int-sub 1)) m)
      (setf nrt (f2cl-lib:int-sub nr 1)))
     (t (setf nrt nr)))
    (if (> nrt 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (slartv nrt
             (f2cl-lib:array-slice ab-%data% single-float
                                   ((+ l 1)
                                    (f2cl-lib:int-sub
                                     (f2cl-lib:int-add j1
                                                       kun)
                                     1))
                                   ((1 ldab) (1 *))
                                   ab-%offset%)
             inca
             (f2cl-lib:array-slice ab-%data% single-float
                                   (l
                                    (f2cl-lib:int-add j1
                                                      kun))
                                   ((1 ldab) (1 *))
                                   ab-%offset%)
             inca
             (f2cl-lib:array-slice work-%data% single-float
                                   ((+ mn j1 kun)) ((1 *))
                                   work-%offset%)
             (f2cl-lib:array-slice work-%data% single-float
                                   ((+ j1 kun)) ((1 *))
                                   work-%offset%)
             kb1)
          (declare (ignore var-1 var-3 var-5 var-6))
          (when var-0 (setf nrt var-0))
          (when var-2 (setf inca var-2))
          (when var-4 (setf inca var-4))
          (when var-7 (setf kb1 var-7))))
   label50))
(cond
```

```
((and (= ml ml0) (> mu mu0))
 (cond
  ((<= mu (f2cl-lib:int-add n (f2cl-lib:int-sub i) 1))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
       (slartg
        (f2cl-lib:array-slice ab-%data% single-float
                              ((+ ku (f2cl-lib:int-sub mu)
                                  3)
                               (f2cl-lib:int-sub
                                (f2cl-lib:int-add i mu) 2))
                              ((1 ldab) (1 *)) ab-%offset%)
        (f2cl-lib:array-slice ab-%data% single-float
                              ((+ ku (f2cl-lib:int-sub mu)
                                  2)
                               (f2cl-lib:int-sub
                                (f2cl-lib:int-add i mu) 1))
                              ((1 ldab) (1 *)) ab-%offset%)
        (f2cl-lib:array-slice work-%data% single-float
                              ((+ mn i mu
                                  (f2cl-lib:int-sub 1)))
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float
                              ((+ i mu
                                  (f2cl-lib:int-sub 1)))
                              ((1 *)) work-%offset%)
        ra)
     (declare (ignore var-0 var-1 var-2 var-3))
     (when var-4 (setf ra var-4)))
   (setf (f2cl-lib:fref ab-%data%
                        ((f2cl-lib:int-add
                          (f2cl-lib:int-sub ku mu) 3)
                         (f2cl-lib:int-sub
                          (f2cl-lib:int-add i mu) 2))
                        ((1 ldab) (1 *)) ab-%offset%)
         ra)
   (srot
    (min
     (the f2cl-lib:integer4
          (f2cl-lib:int-sub (f2cl-lib:int-add kl mu) 2))
     (the f2cl-lib:integer4 (f2cl-lib:int-sub m i)))
    (f2cl-lib:array-slice ab-%data% single-float
                          ((+ ku (f2cl-lib:int-sub mu) 4)
                           (f2cl-lib:int-sub
                            (f2cl-lib:int-add i mu) 2))
                          ((1 ldab) (1 *)) ab-%offset%)
    1
    (f2cl-lib:array-slice ab-%data% single-float
                          ((+ ku (f2cl-lib:int-sub mu) 3)
                           (f2cl-lib:int-sub
                            (f2cl-lib:int-add i mu) 1))
```

```
                              ((1 ldab) (1 *)) ab-%offset%)
      1
      (f2cl-lib:array-slice work-%data% single-float
                              ((+ mn i mu (f2cl-lib:int-sub 1)))
                              ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float
                              ((+ i mu (f2cl-lib:int-sub 1)))
                              ((1 *)) work-%offset%))))
  (setf nr (f2cl-lib:int-add nr 1))
  (setf j1 (f2cl-lib:int-sub j1 kb1))))
(cond
 (wantpt
  (f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
                ((> j j2) nil)
    (tagbody
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (srot n
           (f2cl-lib:array-slice pt-%data% single-float
                                 ((+ j kun
                                     (f2cl-lib:int-sub 1))
                                  1)
                                 ((1 ldpt) (1 *))
                                 pt-%offset%)
           ldpt
           (f2cl-lib:array-slice pt-%data% single-float
                                 ((+ j kun) 1)
                                 ((1 ldpt) (1 *))
                                 pt-%offset%)
           ldpt
           (f2cl-lib:array-slice work-%data% single-float
                                 ((+ mn j kun)) ((1 *))
                                 work-%offset%)
           (f2cl-lib:array-slice work-%data% single-float
                                 ((+ j kun)) ((1 *))
                                 work-%offset%))
        (declare (ignore var-1 var-3 var-5 var-6))
        (when var-0 (setf n var-0))
        (when var-2 (setf ldpt var-2))
        (when var-4 (setf ldpt var-4)))
     label60))))
(cond
 ((> (f2cl-lib:int-add j2 kb) m)
  (setf nr (f2cl-lib:int-sub nr 1))
  (setf j2 (f2cl-lib:int-sub j2 kb1))))
(f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
              ((> j j2) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add j kb)) ((1 *))
```

```
                                        work-%offset%)
                                (*
                                 (f2cl-lib:fref work-%data%
                                                ((f2cl-lib:int-add j kun))
                                                ((1 *)) work-%offset%)
                                 (f2cl-lib:fref ab-%data%
                                                (klu1 (f2cl-lib:int-add j kun))
                                                ((1 ldab) (1 *)) ab-%offset%)))
                      (setf (f2cl-lib:fref ab-%data%
                                           (klu1 (f2cl-lib:int-add j kun))
                                           ((1 ldab) (1 *)) ab-%offset%)
                                (*
                                 (f2cl-lib:fref work-%data%
                                                ((f2cl-lib:int-add mn j kun))
                                                ((1 *)) work-%offset%)
                                 (f2cl-lib:fref ab-%data%
                                                (klu1 (f2cl-lib:int-add j kun))
                                                ((1 ldab) (1 *)) ab-%offset%)))
                         label70))
                    (cond ((> ml ml0) (setf ml (f2cl-lib:int-sub ml 1)))
                          (t (setf mu (f2cl-lib:int-sub mu 1))))
                 label80))
             label90))))
        (cond
         ((and (= ku 0) (> kl 0))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i
                            (min
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
                             (the f2cl-lib:integer4 n)))
                         nil)
            (tagbody
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (slartg
                   (f2cl-lib:array-slice ab-%data% single-float (1 i)
                                         ((1 ldab) (1 *)) ab-%offset%)
                   (f2cl-lib:array-slice ab-%data% single-float (2 i)
                                         ((1 ldab) (1 *)) ab-%offset%)
                   rc rs ra)
                (declare (ignore var-0 var-1))
                (when var-2 (setf rc var-2))
                (when var-3 (setf rs var-3))
                (when var-4 (setf ra var-4)))
              (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) ra)
              (cond
               ((< i n)
                (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                        (* rs
                           (f2cl-lib:fref ab-%data% (1 (f2cl-lib:int-add i 1))
```

```
                                  ((1 ldab) (1 *)) ab-%offset%)))
          (setf (f2cl-lib:fref ab-%data% (1 (f2cl-lib:int-add i 1))
                               ((1 ldab) (1 *)) ab-%offset%)
                  (* rc
                     (f2cl-lib:fref ab-%data% (1 (f2cl-lib:int-add i 1))
                                  ((1 ldab) (1 *)) ab-%offset%)))))
      (if wantq
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (srot m
               (f2cl-lib:array-slice q-%data% single-float (1 i)
                                     ((1 ldq) (1 *)) q-%offset%)
               1
               (f2cl-lib:array-slice q-%data% single-float
                                     (1 (f2cl-lib:int-add i 1))
                                     ((1 ldq) (1 *)) q-%offset%)
               1 rc rs)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf m var-0))
            (when var-5 (setf rc var-5))
            (when var-6 (setf rs var-6))))
      (if wantc
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (srot ncc
               (f2cl-lib:array-slice c-%data% single-float (i 1)
                                     ((1 ldc) (1 *)) c-%offset%)
               ldc
               (f2cl-lib:array-slice c-%data% single-float ((+ i 1) 1)
                                     ((1 ldc) (1 *)) c-%offset%)
               ldc rc rs)
            (declare (ignore var-1 var-3))
            (when var-0 (setf ncc var-0))
            (when var-2 (setf ldc var-2))
            (when var-4 (setf ldc var-4))
            (when var-5 (setf rc var-5))
            (when var-6 (setf rs var-6))))
    label100))
 (if (<= m n)
     (setf (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%)
             (f2cl-lib:fref ab-%data% (1 m) ((1 ldab) (1 *))
                            ab-%offset%))))
((> ku 0)
 (cond
  ((< m n)
   (setf rb
           (f2cl-lib:fref ab-%data% (ku (f2cl-lib:int-add m 1))
                          ((1 ldab) (1 *)) ab-%offset%))
   (f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                 ((> i 1) nil)
```

```
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (slartg
           (f2cl-lib:array-slice ab-%data% single-float ((+ ku 1) i)
                                 ((1 ldab) (1 *)) ab-%offset%)
           rb rc rs ra)
        (declare (ignore var-0))
        (when var-1 (setf rb var-1))
        (when var-2 (setf rc var-2))
        (when var-3 (setf rs var-3))
        (when var-4 (setf ra var-4)))
      (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) ra)
      (cond
       ((> i 1)
        (setf rb
                (* (- rs)
                   (f2cl-lib:fref ab-%data% (ku i) ((1 ldab) (1 *))
                                  ab-%offset%)))
        (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                             ((1 *)) e-%offset%)
                (* rc
                   (f2cl-lib:fref ab-%data% (ku i) ((1 ldab) (1 *))
                                  ab-%offset%)))))
      (if wantpt
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (srot n
               (f2cl-lib:array-slice pt-%data% single-float (i 1)
                                     ((1 ldpt) (1 *)) pt-%offset%)
               ldpt
               (f2cl-lib:array-slice pt-%data% single-float
                                     ((+ m 1) 1) ((1 ldpt) (1 *))
                                     pt-%offset%)
               ldpt rc rs)
            (declare (ignore var-1 var-3))
            (when var-0 (setf n var-0))
            (when var-2 (setf ldpt var-2))
            (when var-4 (setf ldpt var-4))
            (when var-5 (setf rc var-5))
            (when var-6 (setf rs var-6))))
     label110)))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i (f2cl-lib:int-add minmn (f2cl-lib:int-sub 1)))
                  nil)
    (tagbody
      (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
              (f2cl-lib:fref ab-%data% (ku (f2cl-lib:int-add i 1))
                             ((1 ldab) (1 *)) ab-%offset%))
     label120))
```

```
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i minmn) nil)
              (tagbody
                (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                        (f2cl-lib:fref ab-%data% ((f2cl-lib:int-add ku 1) i)
                                       ((1 ldab) (1 *)) ab-%offset%))
               label130)))))
          (t
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i (f2cl-lib:int-add minmn (f2cl-lib:int-sub 1)))
                          nil)
            (tagbody
              (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) zero)
             label140))
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i minmn) nil)
            (tagbody
              (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                      (f2cl-lib:fref ab-%data% (1 i) ((1 ldab) (1 *))
                                     ab-%offset%))
             label150))))
         (go end_label)
        end_label
         (return
          (values vect m n ncc nil nil nil nil nil nil nil ldq nil ldpt nil ldc
                  nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgbbrd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
```

```
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::vect
  fortran-to-lisp::m
  fortran-to-lisp::n
  fortran-to-lisp::ncc nil nil nil
  nil nil nil nil
  fortran-to-lisp::ldq nil
  fortran-to-lisp::ldpt nil
  fortran-to-lisp::ldc nil
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

45.2 sgbcon

45.2.1 Axiom unit tests

— **sgbcon.input** —

```
)set break resume
)sys rm -f sgbcon.output
)spool sgbcon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

45.2.2 Axiom help page

— **sgbcon.help** —

```
==================================================================
sgbcon examples
==================================================================

==================================================================
```

```
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgbcon (NORM, N, KL, KU, AB, LDAB, IPIV, ANORM, RCOND, WORK,
           IWORK, INFO)
           SGBCON

Function/Subroutine Documentation
   subroutine sgbcon (character NORM, integer N, integer KL, integer KU, real,
       dimension( ldab, * ) AB, integer LDAB, integer, dimension( * ) IPIV,
       real ANORM, real RCOND, real, dimension( * ) WORK, integer, dimension(
       * ) IWORK, integer INFO)
       SGBCON

Purpose:
   SGBCON estimates the reciprocal of the condition number of a real
   general band matrix A, in either the 1-norm or the infinity-norm,
   using the LU factorization computed by SGBTRF.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as
   RCOND = 1 / ( norm(A) * norm(inv(A)) ).

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies whether the 1-norm condition number or the
   infinity-norm condition number is required:
   = '1' or 'O':  1-norm;
   = 'I':         Infinity-norm.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 AB
   AB is REAL array, dimension (LDAB,N)
   Details of the LU factorization of the band matrix A, as
   computed by SGBTRF.  U is stored as an upper triangular band
```

```
  matrix with KL+KU superdiagonals in rows 1 to KL+KU+1, and
  the multipliers used during the factorization are stored in
  rows KL+KU+2 to 2*KL+KU+1.

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= N, row i of the matrix was
  interchanged with row IPIV(i).

ANORM
  ANORM is REAL
  If NORM = '1' or 'O', the 1-norm of the original matrix A.
  If NORM = 'I', the infinity-norm of the original matrix A.

RCOND
  RCOND is REAL
  The reciprocal of the condition number of the matrix A,
  computed as RCOND = 1/(norm(A) * norm(inv(A))).

WORK
  WORK is REAL array, dimension (3*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
```

———-

45.2.3 fortran code

— sgbcon.f —

```
*  =====================================================================
      SUBROUTINE SGBCON( NORM, N, KL, KU, AB, LDAB, IPIV, ANORM, RCOND,
     $                   WORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
```

```
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            INFO, KL, KU, LDAB, N
      REAL               ANORM, RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      REAL               AB( LDAB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LNOTI, ONENRM
      CHARACTER          NORMIN
      INTEGER            IX, J, JP, KASE, KASE1, KD, LM
      REAL               AINVNM, SCALE, SMLNUM, T
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ISAMAX
      REAL               SDOT, SLAMCH
      EXTERNAL           LSAME, ISAMAX, SDOT, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           SAXPY, SLACN2, SLATBS, SRSCL, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      ONENRM = NORM.EQ.'1' .OR. LSAME( NORM, 'O' )
      IF( .NOT.ONENRM .AND. .NOT.LSAME( NORM, 'I' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
```

```
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.2*KL+KU+1 ) THEN
         INFO = -6
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGBCON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
      SMLNUM = SLAMCH( 'Safe minimum' )
*
*     Estimate the norm of inv(A).
*
      AINVNM = ZERO
      NORMIN = 'N'
      IF( ONENRM ) THEN
         KASE1 = 1
      ELSE
         KASE1 = 2
      END IF
      KD = KL + KU + 1
      LNOTI = KL.GT.0
      KASE = 0
   10 CONTINUE
      CALL SLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.KASE1 ) THEN
*
*           Multiply by inv(L).
*
            IF( LNOTI ) THEN
               DO 20 J = 1, N - 1
                  LM = MIN( KL, N-J )
                  JP = IPIV( J )
                  T = WORK( JP )
                  IF( JP.NE.J ) THEN
```

```
                  WORK( JP ) = WORK( J )
                  WORK( J ) = T
               END IF
               CALL SAXPY( LM, -T, AB( KD+1, J ), 1, WORK( J+1 ), 1 )
   20       CONTINUE
         END IF
*
*        Multiply by inv(U).
*
         CALL SLATBS( 'Upper', 'No transpose', 'Non-unit', NORMIN, N,
     $                KL+KU, AB, LDAB, WORK, SCALE, WORK( 2*N+1 ),
     $                INFO )
      ELSE
*
*        Multiply by inv(U**T).
*
         CALL SLATBS( 'Upper', 'Transpose', 'Non-unit', NORMIN, N,
     $                KL+KU, AB, LDAB, WORK, SCALE, WORK( 2*N+1 ),
     $                INFO )
*
*        Multiply by inv(L**T).
*
         IF( LNOTI ) THEN
            DO 30 J = N - 1, 1, -1
               LM = MIN( KL, N-J )
               WORK( J ) = WORK( J ) - SDOT( LM, AB( KD+1, J ), 1,
     $                     WORK( J+1 ), 1 )
               JP = IPIV( J )
               IF( JP.NE.J ) THEN
                  T = WORK( JP )
                  WORK( JP ) = WORK( J )
                  WORK( J ) = T
               END IF
   30       CONTINUE
         END IF
      END IF
*
*     Divide X by 1/SCALE if doing so will not cause overflow.
*
      NORMIN = 'Y'
      IF( SCALE.NE.ONE ) THEN
         IX = ISAMAX( N, WORK, 1 )
         IF( SCALE.LT.ABS( WORK( IX ) )*SMLNUM .OR. SCALE.EQ.ZERO )
     $      GO TO 40
         CALL SRSCL( N, SCALE, WORK, 1 )
      END IF
      GO TO 10
   END IF
*
*  Compute the estimate of the reciprocal condition number.
```

```
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
   40 CONTINUE
      RETURN
*
*     End of SGBCON
*
      END
```

———-

45.2.4 lisp code

— sgbcon.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun sgbcon (norm n kl ku ab ldab ipiv anorm rcond work iwork info)
    (declare (type (single-float) rcond anorm)
             (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array single-float (*)) work ab)
             (type (f2cl-lib:integer4) info ldab ku kl n)
             (type (string *) norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (ab single-float ab-%data% ab-%offset%)
         (work single-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (ainvnm 0.0) (scale 0.0) (smlnum 0.0) (ix 0) (j 0) (jp 0) (kase 0)
             (kase1 0) (kd 0) (lm 0)
             (normin
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (lnoti nil) (onenrm nil) (t$ 0.0))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (single-float) t$ ainvnm scale smlnum)
                 (type (f2cl-lib:integer4) ix j jp kase kase1 kd lm)
                 (type (string 1) normin)
                 (type f2cl-lib:logical lnoti onenrm))
        (setf info 0)
        (setf onenrm
                (or (f2cl-lib:fstring-= norm "1")
                    (multiple-value-bind (ret-val var-0 var-1)
```

```
                  (lsame norm "O")
                (declare (ignore var-1))
                (when var-0 (setf norm var-0))
                ret-val)))
 (cond
  ((and (not onenrm)
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame norm "I")
           (declare (ignore var-1))
           (when var-0 (setf norm var-0))
           ret-val)))
   (setf info -1))
  ((< n 0) (setf info -2)) ((< kl 0) (setf info -3))
  ((< ku 0) (setf info -4))
  ((< ldab (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
   (setf info -6))
  ((< anorm zero) (setf info -8)))
 (cond
  ((/= info 0) (xerbla "SGBCON" (f2cl-lib:int-sub info))
   (go end_label)))
 (setf rcond zero)
 (cond ((= n 0) (setf rcond one) (go end_label))
       ((= anorm zero) (go end_label)))
 (setf smlnum (slamch "Safe minimum"))
 (setf ainvnm zero)
 (f2cl-lib:f2cl-set-string normin "N" (string 1))
 (cond (onenrm (setf kase1 1)) (t (setf kase1 2)))
 (setf kd (f2cl-lib:int-add kl ku 1))
 (setf lnoti (> kl 0))
 (setf kase 0)
label10
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slacn2 n
      (f2cl-lib:array-slice work-%data% single-float ((+ n 1)) ((1 *))
                            work-%offset%)
      work iwork ainvnm kase isave)
   (declare (ignore var-1 var-2 var-3 var-6))
   (when var-0 (setf n var-0))
   (when var-4 (setf ainvnm var-4))
   (when var-5 (setf kase var-5)))
 (cond
  ((/= kase 0)
   (cond
    ((= kase kase1)
     (cond
      (lnoti
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                      nil)
```

```
          (tagbody
            (setf lm
                    (min (the f2cl-lib:integer4 kl)
                         (the f2cl-lib:integer4 (f2cl-lib:int-sub n j))))
            (setf jp
                    (f2cl-lib:fref ipiv-%data% (j) ((1 *))
                                   ipiv-%offset%))
            (setf t$
                    (f2cl-lib:fref work-%data% (jp) ((1 *))
                                   work-%offset%))
            (cond
             ((/= jp j)
              (setf (f2cl-lib:fref work-%data% (jp) ((1 *))
                                   work-%offset%)
                      (f2cl-lib:fref work-%data% (j) ((1 *))
                                     work-%offset%))
              (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                      t$)))
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                (saxpy lm (- t$)
                 (f2cl-lib:array-slice ab-%data% single-float
                                       ((+ kd 1) j) ((1 ldab) (1 *))
                                       ab-%offset%)
                 1
                 (f2cl-lib:array-slice work-%data% single-float ((+ j 1))
                                       ((1 *)) work-%offset%)
                 1)
              (declare (ignore var-1 var-2 var-3 var-4 var-5))
              (when var-0 (setf lm var-0)))
           label20))))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10 var-11)
           (slatbs "Upper" "No transpose" "Non-unit" normin n
            (f2cl-lib:int-add kl ku) ab ldab work scale
            (f2cl-lib:array-slice work-%data% single-float
                                  ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                                  work-%offset%)
            info)
         (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-10))
         (when var-3 (setf normin var-3))
         (when var-4 (setf n var-4))
         (when var-7 (setf ldab var-7))
         (when var-9 (setf scale var-9))
         (when var-11 (setf info var-11))))
      (t
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10 var-11)
           (slatbs "Upper" "Transpose" "Non-unit" normin n
```

```
      (f2cl-lib:int-add kl ku) ab ldab work scale
      (f2cl-lib:array-slice work-%data% single-float
                            ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                            work-%offset%)
      info)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-10))
    (when var-3 (setf normin var-3))
    (when var-4 (setf n var-4))
    (when var-7 (setf ldab var-7))
    (when var-9 (setf scale var-9))
    (when var-11 (setf info var-11)))
  (cond
   (lnoti
    (f2cl-lib:fdo (j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                  ((> j 1) nil)
      (tagbody
        (setf lm
                (min (the f2cl-lib:integer4 kl)
                     (the f2cl-lib:integer4 (f2cl-lib:int-sub n j))))
        (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                (-
                 (f2cl-lib:fref work-%data% (j) ((1 *))
                                work-%offset%)
                 (multiple-value-bind
                     (ret-val var-0 var-1 var-2 var-3 var-4)
                     (sdot lm
                      (f2cl-lib:array-slice ab-%data% single-float
                                            ((+ kd 1) j)
                                            ((1 ldab) (1 *))
                                            ab-%offset%)
                      1
                      (f2cl-lib:array-slice work-%data% single-float
                                            ((+ j 1)) ((1 *))
                                            work-%offset%)
                      1)
                   (declare (ignore var-1 var-2 var-3 var-4))
                   (when var-0 (setf lm var-0))
                   ret-val)))
        (setf jp
                (f2cl-lib:fref ipiv-%data% (j) ((1 *))
                               ipiv-%offset%))
        (cond
         ((/= jp j)
          (setf t$
                  (f2cl-lib:fref work-%data% (jp) ((1 *))
                                 work-%offset%))
          (setf (f2cl-lib:fref work-%data% (jp) ((1 *))
                               work-%offset%)
                  (f2cl-lib:fref work-%data% (j) ((1 *))
```

```
                                        work-%offset%))
                    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                            t$)))
                 label30))))))
          (f2cl-lib:f2cl-set-string normin "Y" (string 1))
          (cond
           ((/= scale one)
            (setf ix
                    (multiple-value-bind (ret-val var-0 var-1 var-2)
                        (isamax n work 1)
                      (declare (ignore var-1 var-2))
                      (when var-0 (setf n var-0))
                      ret-val))
            (if (or
                 (< scale
                    (*
                     (abs
                      (f2cl-lib:fref work-%data% (ix) ((1 *)) work-%offset%))
                     smlnum))
                 (= scale zero))
                (go label40))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (srscl n scale work 1)
              (declare (ignore var-2 var-3))
              (when var-0 (setf n var-0))
              (when var-1 (setf scale var-1)))))
          (go label10)))
        (if (/= ainvnm zero)
            (setf rcond (/ (/ one ainvnm) anorm)))
       label40
        (go end_label)
       end_label
        (return (values norm n nil nil nil ldab nil nil rcond nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgbcon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (single-float) (single-float)
                                              (array single-float (*))
```

```
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::norm
  fortran-to-lisp::n nil nil nil
  fortran-to-lisp::ldab nil nil
  fortran-to-lisp::rcond nil nil
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

45.3 sgbequ

45.3.1 Axiom unit tests

— **sgbequ.input** —

```
)set break resume
)sys rm -f sgbequ.output
)spool sgbequ.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

45.3.2 Axiom help page

— **sgbequ.help** —

```
==================================================================
sgbequ examples
==================================================================

==================================================================
Man Page Details
==================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine sgbequ (M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND, AMAX,
           INFO)
           SGBEQU

Function/Subroutine Documentation
   subroutine sgbequ (integer M, integer N, integer KL, integer KU, real,
       dimension( ldab, * ) AB, integer LDAB, real, dimension( * ) R, real,
       dimension( * ) C, real ROWCND, real COLCND, real AMAX, integer INFO)
       SGBEQU

Purpose:
   SGBEQU computes row and column scalings intended to equilibrate an
   M-by-N band matrix A and reduce its condition number.  R returns the
   row scale factors and C the column scale factors, chosen to try to
   make the largest element in each row and column of the matrix B with
   elements B(i,j)=R(i)*A(i,j)*C(j) have absolute value 1.

   R(i) and C(j) are restricted to be between SMLNUM = smallest safe
   number and BIGNUM = largest safe number.  Use of these scaling
   factors is not guaranteed to reduce the condition number of A but
   works well in practice.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 AB
   AB is REAL array, dimension (LDAB,N)
   The band matrix A, stored in rows 1 to KL+KU+1.  The j-th
   column of A is stored in the j-th column of the array AB as
   follows:
   AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl).
```

```
LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

R
  R is REAL array, dimension (M)
  If INFO = 0, or INFO > M, R contains the row scale factors for A.

C
  C is REAL array, dimension (N)
  If INFO = 0, C contains the column scale factors for A.

ROWCND
  ROWCND is REAL
  If INFO = 0 or INFO > M, ROWCND contains the ratio of the
  smallest R(i) to the largest R(i).  If ROWCND >= 0.1 and
  AMAX is neither too large nor too small, it is not worth scaling by R.

COLCND
  COLCND is REAL
  If INFO = 0, COLCND contains the ratio of the smallest
  C(i) to the largest C(i).  If COLCND >= 0.1, it is not worth scaling by C.

AMAX
  AMAX is REAL
  Absolute value of largest matrix element.  If AMAX is very
  close to overflow or very close to underflow, the matrix should be scaled.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, and i is
     <= M:  the i-th row of A is exactly zero
     >  M:  the (i-M)-th column of A is exactly zero
```

———-

45.3.3 fortran code

— **sgbequ.f** —

```
*  =====================================================================
      SUBROUTINE SGBEQU( M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $                   AMAX, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
```

```
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, KL, KU, LDAB, M, N
      REAL               AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      REAL               AB( LDAB, * ), C( * ), R( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, KD
      REAL               BIGNUM, RCMAX, RCMIN, SMLNUM
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGBEQU', -INFO )
```

```
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         ROWCND = ONE
         COLCND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Get machine constants.
*
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
*
*     Compute row scale factors.
*
      DO 10 I = 1, M
         R( I ) = ZERO
   10 CONTINUE
*
*     Find the maximum element in each row.
*
      KD = KU + 1
      DO 30 J = 1, N
         DO 20 I = MAX( J-KU, 1 ), MIN( J+KL, M )
            R( I ) = MAX( R( I ), ABS( AB( KD+I-J, J ) ) )
   20    CONTINUE
   30 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 40 I = 1, M
         RCMAX = MAX( RCMAX, R( I ) )
         RCMIN = MIN( RCMIN, R( I ) )
   40 CONTINUE
      AMAX = RCMAX
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 50 I = 1, M
            IF( R( I ).EQ.ZERO ) THEN
               INFO = I
               RETURN
```

```
            END IF
   50    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 60 I = 1, M
            R( I ) = ONE / MIN( MAX( R( I ), SMLNUM ), BIGNUM )
   60    CONTINUE
*
*        Compute ROWCND = min(R(I)) / max(R(I))
*
         ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
*     Compute column scale factors
*
      DO 70 J = 1, N
         C( J ) = ZERO
   70 CONTINUE
*
*     Find the maximum element in each column,
*     assuming the row scaling computed above.
*
      KD = KU + 1
      DO 90 J = 1, N
         DO 80 I = MAX( J-KU, 1 ), MIN( J+KL, M )
            C( J ) = MAX( C( J ), ABS( AB( KD+I-J, J ) )*R( I ) )
   80    CONTINUE
   90 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 100 J = 1, N
         RCMIN = MIN( RCMIN, C( J ) )
         RCMAX = MAX( RCMAX, C( J ) )
  100 CONTINUE
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 110 J = 1, N
            IF( C( J ).EQ.ZERO ) THEN
               INFO = M + J
               RETURN
            END IF
  110    CONTINUE
```

```
      ELSE
*
*        Invert the scale factors.
*
         DO 120 J = 1, N
            C( J ) = ONE / MIN( MAX( C( J ), SMLNUM ), BIGNUM )
  120    CONTINUE
*
*        Compute COLCND = min(C(J)) / max(C(J))
*
         COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
      RETURN
*
*     End of SGBEQU
*
      END
```

———-

45.3.4 lisp code

— sgbequ.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun sgbequ (m n kl ku ab ldab r c rowcnd colcnd amax info)
    (declare (type (single-float) amax colcnd rowcnd)
             (type (array single-float (*)) c r ab)
             (type (f2cl-lib:integer4) info ldab ku kl n m))
    (f2cl-lib:with-multi-array-data
        ((ab single-float ab-%data% ab-%offset%)
         (r single-float r-%data% r-%offset%)
         (c single-float c-%data% c-%offset%))
      (prog ((bignum 0.0) (rcmax 0.0) (rcmin 0.0) (smlnum 0.0) (i 0) (j 0)
             (kd 0))
        (declare (type (single-float) bignum rcmax rcmin smlnum)
                 (type (f2cl-lib:integer4) i j kd))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< kl 0) (setf info -3)) ((< ku 0) (setf info -4))
              ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -6)))
        (cond
         ((/= info 0) (xerbla "SGBEQU" (f2cl-lib:int-sub info))
          (go end_label)))
```

```
(cond
 ((or (= m 0) (= n 0)) (setf rowcnd one) (setf colcnd one)
  (setf amax zero) (go end_label)))
(setf smlnum (slamch "S"))
(setf bignum (/ one smlnum))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%) zero)
   label10))
(setf kd (f2cl-lib:int-add ku 1))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                    (the f2cl-lib:integer4 1))
                   (f2cl-lib:int-add i 1))
                  ((> i
                      (min
                       (the f2cl-lib:integer4 (f2cl-lib:int-add j kl))
                       (the f2cl-lib:integer4 m)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                     (abs
                      (f2cl-lib:fref ab-%data%
                                     ((f2cl-lib:int-sub
                                       (f2cl-lib:int-add kd i) j)
                                      j)
                                     ((1 ldab) (1 *)) ab-%offset%))))
       label20))
   label30))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf rcmax
            (max rcmax
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
    (setf rcmin
            (min rcmin
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
   label40))
(setf amax rcmax)
(cond
```

```
 ((= rcmin zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref r (i) ((1 *))) zero) (setf info i)
        (go end_label)))
     label50)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
              (/ one
                 (min
                  (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                       smlnum)
                  bignum)))
     label60))
  (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) zero)
   label70))
(setf kd (f2cl-lib:int-add ku 1))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                    (the f2cl-lib:integer4 1))
                   (f2cl-lib:int-add i 1))
                  ((> i
                      (min
                       (the f2cl-lib:integer4 (f2cl-lib:int-add j kl))
                       (the f2cl-lib:integer4 m)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                     (*
                      (abs
                       (f2cl-lib:fref ab-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add kd i) j)
                                       j)
                                      ((1 ldab) (1 *)) ab-%offset%))
```

```
                              (f2cl-lib:fref r-%data% (i) ((1 *))
                                             r-%offset%))))
                 label80))
             label90))
          (setf rcmin bignum)
          (setf rcmax zero)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf rcmin
                      (min rcmin
                           (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
              (setf rcmax
                      (max rcmax
                           (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
             label100))
          (cond
           ((= rcmin zero)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (cond
                 ((= (f2cl-lib:fref c (j) ((1 *))) zero)
                  (setf info (f2cl-lib:int-add m j)) (go end_label)))
               label110)))
           (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                        (/ one
                           (min
                            (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                                 smlnum)
                            bignum)))
               label120))
            (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
          (go end_label)
         end_label
          (return
           (values nil nil nil nil nil nil nil nil rowcnd colcnd amax info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgbequ
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
```

```
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (single-float) (single-float)
  (single-float)
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil nil nil nil nil nil
  fortran-to-lisp::rowcnd
  fortran-to-lisp::colcnd
  fortran-to-lisp::amax
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

45.4 sgbequb

45.4.1 Axiom unit tests

— **sgbequb.input** —

```
)set break resume
)sys rm -f sgbequb.output
)spool sgbequb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

45.4.2 Axiom help page

— **sgbequb.help** —

```
==================================================================
```

```
sgbequb examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgbequb (M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND, AMAX,
           INFO)
           SGBEQUB

Function/Subroutine Documentation
   subroutine sgbequb (integer M, integer N, integer KL, integer KU, real,
       dimension( ldab, * ) AB, integer LDAB, real, dimension( * ) R, real,
       dimension( * ) C, real ROWCND, real COLCND, real AMAX, integer INFO)
       SGBEQUB

Purpose:
   SGBEQUB computes row and column scalings intended to equilibrate an
   M-by-N matrix A and reduce its condition number.  R returns the row
   scale factors and C the column scale factors, chosen to try to make
   the largest element in each row and column of the matrix B with
   elements B(i,j)=R(i)*A(i,j)*C(j) have an absolute value of at most
   the radix.

   R(i) and C(j) are restricted to be a power of the radix between
   SMLNUM = smallest safe number and BIGNUM = largest safe number.  Use
   of these scaling factors is not guaranteed to reduce the condition
   number of A but works well in practice.

   This routine differs from SGEEQU by restricting the scaling factors
   to a power of the radix.  Baring over- and underflow, scaling by
   these factors introduces no additional rounding errors.  However, the
   scaled entries' magnitured are no longer approximately 1 but lie
   between sqrt(radix) and 1/sqrt(radix).

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 KL
   KL is INTEGER
```

```
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

AB
  AB is DOUBLE PRECISION array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

LDAB
  LDAB is INTEGER
  The leading dimension of the array A.  LDAB >= max(1,M).

R
  R is REAL array, dimension (M)
  If INFO = 0 or INFO > M, R contains the row scale factors for A.

C
  C is REAL array, dimension (N)
  If INFO = 0,  C contains the column scale factors for A.

ROWCND
  ROWCND is REAL
  If INFO = 0 or INFO > M, ROWCND contains the ratio of the
  smallest R(i) to the largest R(i).  If ROWCND >= 0.1 and
  AMAX is neither too large nor too small, it is not worth scaling by R.

COLCND
  COLCND is REAL
  If INFO = 0, COLCND contains the ratio of the smallest
  C(i) to the largest C(i).  If COLCND >= 0.1, it is not worth scaling by C.

AMAX
  AMAX is REAL
  Absolute value of largest matrix element.  If AMAX is very
  close to overflow or very close to underflow, the matrix should be scaled.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i,  and i is
     <= M:  the i-th row of A is exactly zero
     >  M:  the (i-M)-th column of A is exactly zero
```

———-

45.4.3 fortran code

— sgbequb.f —

```
*  =====================================================================
      SUBROUTINE SGBEQUB( M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $                    AMAX, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, KL, KU, LDAB, M, N
      REAL               AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      REAL               AB( LDAB, * ), C( * ), R( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, KD
      REAL               BIGNUM, RCMAX, RCMIN, SMLNUM, RADIX, LOGRDX
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, LOG
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
```

```
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGBEQUB', -INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         ROWCND = ONE
         COLCND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Get machine constants.  Assume SMLNUM is a power of the radix.
*
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      RADIX = SLAMCH( 'B' )
      LOGRDX = LOG(RADIX)
*
*     Compute row scale factors.
*
      DO 10 I = 1, M
         R( I ) = ZERO
   10 CONTINUE
*
*     Find the maximum element in each row.
*
      KD = KU + 1
      DO 30 J = 1, N
         DO 20 I = MAX( J-KU, 1 ), MIN( J+KL, M )
            R( I ) = MAX( R( I ), ABS( AB( KD+I-J, J ) ) )
   20    CONTINUE
   30 CONTINUE
      DO I = 1, M
         IF( R( I ).GT.ZERO ) THEN
            R( I ) = RADIX**INT( LOG( R( I ) ) / LOGRDX )
         END IF
```

```
      END DO
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 40 I = 1, M
         RCMAX = MAX( RCMAX, R( I ) )
         RCMIN = MIN( RCMIN, R( I ) )
   40 CONTINUE
      AMAX = RCMAX
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 50 I = 1, M
            IF( R( I ).EQ.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   50    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 60 I = 1, M
            R( I ) = ONE / MIN( MAX( R( I ), SMLNUM ), BIGNUM )
   60    CONTINUE
*
*        Compute ROWCND = min(R(I)) / max(R(I)).
*
         ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
*     Compute column scale factors.
*
      DO 70 J = 1, N
         C( J ) = ZERO
   70 CONTINUE
*
*     Find the maximum element in each column,
*     assuming the row scaling computed above.
*
      DO 90 J = 1, N
         DO 80 I = MAX( J-KU, 1 ), MIN( J+KL, M )
            C( J ) = MAX( C( J ), ABS( AB( KD+I-J, J ) )*R( I ) )
   80    CONTINUE
         IF( C( J ).GT.ZERO ) THEN
            C( J ) = RADIX**INT( LOG( C( J ) ) / LOGRDX )
```

```
         END IF
   90 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 100 J = 1, N
         RCMIN = MIN( RCMIN, C( J ) )
         RCMAX = MAX( RCMAX, C( J ) )
  100 CONTINUE
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 110 J = 1, N
            IF( C( J ).EQ.ZERO ) THEN
               INFO = M + J
               RETURN
            END IF
  110    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 120 J = 1, N
            C( J ) = ONE / MIN( MAX( C( J ), SMLNUM ), BIGNUM )
  120    CONTINUE
*
*        Compute COLCND = min(C(J)) / max(C(J)).
*
         COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
      RETURN
*
*     End of SGBEQUB
*
      END
```

———-

45.4.4 lisp code

— sgbequb.lisp —

```
(let* ((one 1.0) (zero 0.0))
```

```
(declare (type (single-float 1.0 1.0) one)
         (type (single-float 0.0 0.0) zero)
         (ignorable one zero))
(defun sgbequb (m n kl ku ab ldab r c rowcnd colcnd amax info)
  (declare (type (single-float) amax colcnd rowcnd)
           (type (array single-float (*)) c r ab)
           (type (f2cl-lib:integer4) info ldab ku kl n m))
  (f2cl-lib:with-multi-array-data
      ((ab single-float ab-%data% ab-%offset%)
       (r single-float r-%data% r-%offset%)
       (c single-float c-%data% c-%offset%))
    (prog ((bignum 0.0) (rcmax 0.0) (rcmin 0.0) (smlnum 0.0) (radix 0.0)
           (logrdx 0.0) (i 0) (j 0) (kd 0))
      (declare (type (single-float) bignum rcmax rcmin smlnum radix logrdx)
               (type (f2cl-lib:integer4) i j kd))
      (setf info 0)
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< kl 0) (setf info -3)) ((< ku 0) (setf info -4))
            ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -6)))
      (cond
       ((/= info 0) (xerbla "SGBEQUB" (f2cl-lib:int-sub info))
        (go end_label)))
      (cond
       ((or (= m 0) (= n 0)) (setf rowcnd one) (setf colcnd one)
        (setf amax zero) (go end_label)))
      (setf smlnum (slamch "S"))
      (setf bignum (/ one smlnum))
      (setf radix (slamch "B"))
      (setf logrdx (f2cl-lib:flog radix))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%) zero)
         label10))
      (setf kd (f2cl-lib:int-add ku 1))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                          (the f2cl-lib:integer4 1))
                         (f2cl-lib:int-add i 1))
                        ((> i
                            (min
                             (the f2cl-lib:integer4 (f2cl-lib:int-add j kl))
                             (the f2cl-lib:integer4 m)))
                         nil)
            (tagbody
```

```
        (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                     (abs
                      (f2cl-lib:fref ab-%data%
                                     ((f2cl-lib:int-sub
                                       (f2cl-lib:int-add kd i) j)
                                      j)
                                     ((1 ldab) (1 *)) ab-%offset%))))
      label20))
   label30))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref r (i) ((1 *))) zero)
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
              (expt radix
                    (f2cl-lib:int
                     (/
                      (f2cl-lib:flog
                       (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%))
                      logrdx))))))
   label100000))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf rcmax
            (max rcmax
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
    (setf rcmin
            (min rcmin
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
   label40))
(setf amax rcmax)
(cond
 ((= rcmin zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref r (i) ((1 *))) zero) (setf info i)
        (go end_label)))
     label50)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
```

```
                (/ one
                   (min
                    (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                         smlnum)
                    bignum)))
     label60))
  (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) zero)
   label70))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                    (the f2cl-lib:integer4 1))
                   (f2cl-lib:int-add i 1))
                  ((> i
                      (min
                       (the f2cl-lib:integer4 (f2cl-lib:int-add j kl))
                       (the f2cl-lib:integer4 m)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                     (*
                      (abs
                       (f2cl-lib:fref ab-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add kd i) j)
                                       j)
                                      ((1 ldab) (1 *)) ab-%offset%))
                      (f2cl-lib:fref r-%data% (i) ((1 *))
                                     r-%offset%))))
       label80))
    (cond
     ((> (f2cl-lib:fref c (j) ((1 *))) zero)
      (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
              (expt radix
                    (f2cl-lib:int
                     (/
                      (f2cl-lib:flog
                       (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
                      logrdx))))))
   label90))
(setf rcmin bignum)
```

```
        (setf rcmax zero)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf rcmin
                    (min rcmin
                         (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
            (setf rcmax
                    (max rcmax
                         (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
           label100))
        (cond
         ((= rcmin zero)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (cond
               ((= (f2cl-lib:fref c (j) ((1 *))) zero)
                (setf info (f2cl-lib:int-add m j)) (go end_label)))
             label110)))
         (t
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                      (/ one
                         (min
                          (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                               smlnum)
                          bignum)))
             label120))
          (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
        (go end_label)
       end_label
        (return
         (values nil nil nil nil nil nil nil nil rowcnd colcnd amax info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgbequb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
```

```
  (array single-float (*))
  (single-float) (single-float)
  (single-float)
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil nil nil nil nil nil
  fortran-to-lisp::rowcnd
  fortran-to-lisp::colcnd
  fortran-to-lisp::amax
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

45.5 sgbrfs

45.5.1 Axiom unit tests

— **sgbrfs.input** —

```
)set break resume
)sys rm -f sgbrfs.output
)spool sgbrfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

45.5.2 Axiom help page

— **sgbrfs.help** —

```
====================================================================
sgbrfs examples
====================================================================

====================================================================
Man Page Details
```

```
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgbrfs (TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB, IPIV,
           B, LDB, X, LDX, FERR, BERR, WORK, IWORK, INFO)
           SGBRFS

Function/Subroutine Documentation
   subroutine sgbrfs (character TRANS, integer N, integer KL, integer KU,
       integer NRHS, real, dimension( ldab, * ) AB, integer LDAB, real,
       dimension( ldafb, * ) AFB, integer LDAFB, integer, dimension( * ) IPIV,
       real, dimension( ldb, * ) B, integer LDB, real, dimension( ldx, * ) X,
       integer LDX, real, dimension( * ) FERR, real, dimension( * ) BERR,
       real, dimension( * ) WORK, integer, dimension( * ) IWORK, integer INFO)
       SGBRFS

Purpose:
   SGBRFS improves the computed solution to a system of linear
   equations when the coefficient matrix is banded, and provides
   error bounds and backward error estimates for the solution.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose = Transpose)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 AB
   AB is REAL array, dimension (LDAB,N)
```

```
  The original band matrix A, stored in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(n,j+kl).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is REAL array, dimension (LDAFB,N)
  Details of the LU factorization of the band matrix A, as
  computed by SGBTRF.  U is stored as an upper triangular band
  matrix with KL+KU superdiagonals in rows 1 to KL+KU+1, and
  the multipliers used during the factorization are stored in
  rows KL+KU+2 to 2*KL+KU+1.

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL*KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from SGBTRF; for 1<=i<=N, row i of the
  matrix was interchanged with row IPIV(i).

B
  B is REAL array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is REAL array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by SGBTRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

FERR
  FERR is REAL array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
```

```
   largest element in X(j).  The estimate is as reliable as
   the estimate for RCOND, and is almost always a slight
   overestimate of the true error.

 BERR
   BERR is REAL array, dimension (NRHS)
   The componentwise relative backward error of each solution
   vector X(j) (i.e., the smallest relative change in
   any element of A or B that makes X(j) an exact solution).

 WORK
   WORK is REAL array, dimension (3*N)

 IWORK
   IWORK is INTEGER array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

 ITMAX is the maximum number of steps of iterative refinement.
```

45.5.3 fortran code

— sgbrfs.f —

```
*  ===================================================================
      SUBROUTINE SGBRFS( TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB,
     $                   IPIV, B, LDB, X, LDX, FERR, BERR, WORK, IWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, KL, KU, LDAB, LDAFB, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
```

```
      REAL               AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   BERR( * ), FERR( * ), WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
      REAL               TWO
      PARAMETER          ( TWO = 2.0E+0 )
      REAL               THREE
      PARAMETER          ( THREE = 3.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      CHARACTER          TRANST
      INTEGER            COUNT, I, J, K, KASE, KK, NZ
      REAL               EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN, XK
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SAXPY, SCOPY, SGBMV, SGBTRS, SLACN2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH
      EXTERNAL           LSAME, SLAMCH
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
```

```
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -7
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
         INFO = -9
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -12
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -14
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGBRFS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
      IF( NOTRAN ) THEN
         TRANST = 'T'
      ELSE
         TRANST = 'N'
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = MIN( KL+KU+2, N+1 )
      EPS = SLAMCH( 'Epsilon' )
      SAFMIN = SLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 140 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
```

```
*
*       Loop until stopping criterion is satisfied.
*
*       Compute residual R = B - op(A) * X,
*       where op(A) = A, A**T, or A**H, depending on TRANS.
*
        CALL SCOPY( N, B( 1, J ), 1, WORK( N+1 ), 1 )
        CALL SGBMV( TRANS, N, N, KL, KU, -ONE, AB, LDAB, X( 1, J ), 1,
     $              ONE, WORK( N+1 ), 1 )
*
*       Compute componentwise relative backward error from formula
*
*       max(i) ( abs(R(i)) / ( abs(op(A))*abs(X) + abs(B) )(i) )
*
*       where abs(Z) is the componentwise absolute value of the matrix
*       or vector Z.  If the i-th component of the denominator is less
*       than SAFE2, then SAFE1 is added to the i-th components of the
*       numerator and denominator before dividing.
*
        DO 30 I = 1, N
           WORK( I ) = ABS( B( I, J ) )
   30   CONTINUE
*
*       Compute abs(op(A))*abs(X) + abs(B).
*
        IF( NOTRAN ) THEN
           DO 50 K = 1, N
              KK = KU + 1 - K
              XK = ABS( X( K, J ) )
              DO 40 I = MAX( 1, K-KU ), MIN( N, K+KL )
                 WORK( I ) = WORK( I ) + ABS( AB( KK+I, K ) )*XK
   40         CONTINUE
   50      CONTINUE
        ELSE
           DO 70 K = 1, N
              S = ZERO
              KK = KU + 1 - K
              DO 60 I = MAX( 1, K-KU ), MIN( N, K+KL )
                 S = S + ABS( AB( KK+I, K ) )*ABS( X( I, J ) )
   60         CONTINUE
              WORK( K ) = WORK( K ) + S
   70      CONTINUE
        END IF
        S = ZERO
        DO 80 I = 1, N
           IF( WORK( I ).GT.SAFE2 ) THEN
              S = MAX( S, ABS( WORK( N+I ) ) / WORK( I ) )
           ELSE
              S = MAX( S, ( ABS( WORK( N+I ) )+SAFE1 ) /
     $            ( WORK( I )+SAFE1 ) )
```

```
            END IF
   80    CONTINUE
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
*           1) The residual BERR(J) is larger than machine epsilon, and
*           2) BERR(J) decreased by at least a factor of 2 during the
*              last iteration, and
*           3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
*
*           Update solution and try again.
*
            CALL SGBTRS( TRANS, N, KL, KU, 1, AFB, LDAFB, IPIV,
     $                   WORK( N+1 ), N, INFO )
            CALL SAXPY( N, ONE, WORK( N+1 ), 1, X( 1, J ), 1 )
            LSTRES = BERR( J )
            COUNT = COUNT + 1
            GO TO 20
         END IF
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(op(A)))*
*           ( abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
*          inv(op(A)) is the inverse of op(A)
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(op(A))*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(op(A))*abs(X) + abs(B) is less than SAFE2.
*
*        Use SLACN2 to estimate the infinity-norm of the matrix
*           inv(op(A)) * diag(W),
*        where W = abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) )))
*
         DO 90 I = 1, N
            IF( WORK( I ).GT.SAFE2 ) THEN
               WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I )
            ELSE
               WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I ) + SAFE1
```

```
            END IF
   90    CONTINUE
*
         KASE = 0
  100    CONTINUE
         CALL SLACN2( N, WORK( 2*N+1 ), WORK( N+1 ), IWORK, FERR( J ),
     $               KASE, ISAVE )
         IF( KASE.NE.0 ) THEN
            IF( KASE.EQ.1 ) THEN
*
*              Multiply by diag(W)*inv(op(A)**T).
*
               CALL SGBTRS( TRANST, N, KL, KU, 1, AFB, LDAFB, IPIV,
     $                      WORK( N+1 ), N, INFO )
               DO 110 I = 1, N
                  WORK( N+I ) = WORK( N+I )*WORK( I )
  110          CONTINUE
            ELSE
*
*              Multiply by inv(op(A))*diag(W).
*
               DO 120 I = 1, N
                  WORK( N+I ) = WORK( N+I )*WORK( I )
  120          CONTINUE
               CALL SGBTRS( TRANS, N, KL, KU, 1, AFB, LDAFB, IPIV,
     $                      WORK( N+1 ), N, INFO )
            END IF
            GO TO 100
         END IF
*
*        Normalize error.
*
         LSTRES = ZERO
         DO 130 I = 1, N
            LSTRES = MAX( LSTRES, ABS( X( I, J ) ) )
  130    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  140 CONTINUE
*
      RETURN
*
*     End of SGBRFS
*
      END
```

———-

45.5.4 lisp code

— sgbrfs.lisp —

```
(let* ((itmax 5) (zero 0.0) (one 1.0) (two 2.0) (three 3.0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 3.0 3.0) three)
           (ignorable itmax zero one two three))
  (defun sgbrfs
         (trans n kl ku nrhs ab ldab afb ldafb ipiv b ldb$ x ldx ferr berr work
          iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array single-float (*)) work berr ferr x b afb ab)
             (type (f2cl-lib:integer4) info ldx ldb$ ldafb ldab nrhs ku kl n)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (ab single-float ab-%data% ab-%offset%)
         (afb single-float afb-%data% afb-%offset%)
         (b single-float b-%data% b-%offset%)
         (x single-float x-%data% x-%offset%)
         (ferr single-float ferr-%data% ferr-%offset%)
         (berr single-float berr-%data% berr-%offset%)
         (work single-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4)) (eps 0.0)
             (lstres 0.0) (s 0.0) (safe1 0.0) (safe2 0.0) (safmin 0.0) (xk 0.0)
             (count$ 0) (i 0) (j 0) (k 0) (kase 0) (kk 0) (nz 0)
             (transt
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (notran nil))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (single-float) eps lstres s safe1 safe2 safmin xk)
                 (type (f2cl-lib:integer4) count$ i j k kase kk nz)
                 (type (string 1) transt)
                 (type f2cl-lib:logical notran))
        (setf info 0)
        (setf notran
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame trans "N")
                  (declare (ignore var-1))
                  (when var-0 (setf trans var-0))
                  ret-val))
        (cond
         ((and (not notran)
```

```
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame trans "T")
           (declare (ignore var-1))
           (when var-0 (setf trans var-0))
           ret-val))
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame trans "C")
           (declare (ignore var-1))
           (when var-0 (setf trans var-0))
           ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< kl 0) (setf info -3))
 ((< ku 0) (setf info -4)) ((< nrhs 0) (setf info -5))
 ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -7))
 ((< ldafb (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
  (setf info -9))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -12))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -14)))
(cond
 ((/= info 0) (xerbla "SGBRFS" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%) zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) zero)
     label10))
  (go end_label)))
(cond (notran (f2cl-lib:f2cl-set-string transt "T" (string 1)))
      (t (f2cl-lib:f2cl-set-string transt "N" (string 1))))
(setf nz
        (min (the f2cl-lib:integer4 (f2cl-lib:int-add kl ku 2))
             (the f2cl-lib:integer4 (f2cl-lib:int-add n 1))))
(setf eps (slamch "Epsilon"))
(setf safmin (slamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
      (scopy n
       (f2cl-lib:array-slice b-%data% single-float (1 j)
                             ((1 ldb$) (1 *)) b-%offset%)
       1
       (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                             ((1 *)) work-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf n var-0)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sgbmv trans n n kl ku (- one) ab ldab
       (f2cl-lib:array-slice x-%data% single-float (1 j)
                             ((1 ldx) (1 *)) x-%offset%)
       1 one
       (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                             ((1 *)) work-%offset%)
       1)
    (declare (ignore var-5 var-6 var-8 var-9 var-11 var-12))
    (when var-0 (setf trans var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf kl var-3))
    (when var-4 (setf ku var-4))
    (when var-7 (setf ldab var-7))
    (when var-10 (setf one var-10)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (abs
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)))
     label30))
  (cond
   (notran
    (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                  ((> k n) nil)
      (tagbody
        (setf kk (f2cl-lib:int-sub (f2cl-lib:int-add ku 1) k))
        (setf xk
                (abs
                 (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                                x-%offset%)))
        (f2cl-lib:fdo (i
                       (max (the f2cl-lib:integer4 1)
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add k
                                                   (f2cl-lib:int-sub
```

```
                                                  ku))))
                          (f2cl-lib:int-add i 1))
                         ((> i
                             (min (the f2cl-lib:integer4 n)
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-add k kl))))
                          nil)
                 (tagbody
                   (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                        work-%offset%)
                         (+
                          (f2cl-lib:fref work-%data% (i) ((1 *))
                                         work-%offset%)
                          (*
                           (abs
                            (f2cl-lib:fref ab-%data%
                                           ((f2cl-lib:int-add kk i) k)
                                           ((1 ldab) (1 *)) ab-%offset%))
                           xk)))
                  label40))
              label50)))
          (t
           (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                         ((> k n) nil)
             (tagbody
               (setf s zero)
               (setf kk (f2cl-lib:int-sub (f2cl-lib:int-add ku 1) k))
               (f2cl-lib:fdo (i
                              (max (the f2cl-lib:integer4 1)
                                   (the f2cl-lib:integer4
                                        (f2cl-lib:int-add k
                                                          (f2cl-lib:int-sub
                                                           ku))))
                              (f2cl-lib:int-add i 1))
                             ((> i
                                 (min (the f2cl-lib:integer4 n)
                                      (the f2cl-lib:integer4
                                           (f2cl-lib:int-add k kl))))
                              nil)
                 (tagbody
                   (setf s
                         (+ s
                            (*
                             (abs
                              (f2cl-lib:fref ab-%data%
                                             ((f2cl-lib:int-add kk i) k)
                                             ((1 ldab) (1 *))
                                             ab-%offset%))
                             (abs
                              (f2cl-lib:fref x-%data% (i j)
```

```
                                        ((1 ldx) (1 *))
                                        x-%offset%)))))
          label60))
       (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
               (+
                (f2cl-lib:fref work-%data% (k) ((1 *))
                               work-%offset%)
                s))
      label70))))
(setf s zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
      (setf s
              (max s
                   (/
                    (abs
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add n i))
                                    ((1 *)) work-%offset%))
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)))))
     (t
      (setf s
              (max s
                   (/
                    (+
                     (abs
                      (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-add n i))
                                     ((1 *)) work-%offset%))
                     safe1)
                    (+
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)
                     safe1))))))
   label80))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (sgbtrs trans n kl ku 1 afb ldafb ipiv
       (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                             ((1 *)) work-%offset%)
```

```
        n info)
      (declare (ignore var-4 var-5 var-7 var-8))
      (when var-0 (setf trans var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf kl var-2))
      (when var-3 (setf ku var-3))
      (when var-6 (setf ldafb var-6))
      (when var-9 (setf n var-9))
      (when var-10 (setf info var-10)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (saxpy n one
         (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                               ((1 *)) work-%offset%)
         1
         (f2cl-lib:array-slice x-%data% single-float (1 j)
                               ((1 ldx) (1 *)) x-%offset%)
         1)
      (declare (ignore var-2 var-3 var-4 var-5))
      (when var-0 (setf n var-0))
      (when var-1 (setf one var-1)))
    (setf lstres
            (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
    (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (+
                 (abs
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                 ((1 *)) work-%offset%))
                 (* nz eps
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)))))
       (t
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (+
                 (abs
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                 ((1 *)) work-%offset%))
                 (* nz eps
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%))
                 safe1))))
     label90))
  (setf kase 0)
 label100
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
      (slacn2 n
       (f2cl-lib:array-slice work-%data% single-float
                             ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                             ((1 *)) work-%offset%)
       iwork
       (f2cl-lib:array-slice ferr-%data% single-float (j) ((1 *))
                             ferr-%offset%)
       kase isave)
    (declare (ignore var-1 var-2 var-3 var-4 var-6))
    (when var-0 (setf n var-0))
    (when var-5 (setf kase var-5)))
  (cond
   ((/= kase 0)
    (cond
     ((= kase 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (sgbtrs transt n kl ku 1 afb ldafb ipiv
           (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                                 ((1 *)) work-%offset%)
           n info)
        (declare (ignore var-4 var-5 var-7 var-8))
        (when var-0 (setf transt var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf kl var-2))
        (when var-3 (setf ku var-3))
        (when var-6 (setf ldafb var-6))
        (when var-9 (setf n var-9))
        (when var-10 (setf info var-10)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%)
                (*
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add n i)) ((1 *))
                                work-%offset%)
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)))
         label110)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%)
```

```
                          (*
                           (f2cl-lib:fref work-%data%
                                          ((f2cl-lib:int-add n i)) ((1 *))
                                          work-%offset%)
                           (f2cl-lib:fref work-%data% (i) ((1 *))
                                          work-%offset%)))
                   label120))
               (multiple-value-bind
                   (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                    var-9 var-10)
                   (sgbtrs trans n kl ku 1 afb ldafb ipiv
                    (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                                          ((1 *)) work-%offset%)
                    n info)
                 (declare (ignore var-4 var-5 var-7 var-8))
                 (when var-0 (setf trans var-0))
                 (when var-1 (setf n var-1))
                 (when var-2 (setf kl var-2))
                 (when var-3 (setf ku var-3))
                 (when var-6 (setf ldafb var-6))
                 (when var-9 (setf n var-9))
                 (when var-10 (setf info var-10)))))
             (go label100)))
           (setf lstres zero)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf lstres
                       (max lstres
                            (abs
                             (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                            x-%offset%))))
              label130))
           (if (/= lstres zero)
               (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                       (/
                        (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                        lstres)))
          label140))
       (go end_label)
      end_label
       (return
        (values trans n kl ku nil nil ldab nil ldafb nil nil nil nil nil nil
                nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgbrfs
                 fortran-to-lisp::*f2cl-function-info*)
```

```
(fortran-to-lisp::make-f2cl-finfo :arg-types
                                  '((string)
                                    (fortran-to-lisp::integer4)
                                    (fortran-to-lisp::integer4)
                                    (fortran-to-lisp::integer4)
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array fortran-to-lisp::integer4
                                     (*))
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (array single-float (*))
                                    (array single-float (*))
                                    (array fortran-to-lisp::integer4
                                     (*))
                                    (fortran-to-lisp::integer4))
                                  :return-values
                                  '(fortran-to-lisp::trans
                                    fortran-to-lisp::n
                                    fortran-to-lisp::kl
                                    fortran-to-lisp::ku nil nil
                                    fortran-to-lisp::ldab nil
                                    fortran-to-lisp::ldafb nil nil
                                    nil nil nil nil nil nil nil
                                    fortran-to-lisp::info)
                                  :calls 'nil)))
```

———-

45.6 sgbrfsx

45.6.1 Axiom unit tests

— sgbrfsx.input —

```
)set break resume
)sys rm -f sgbrfsx.output
)spool sgbrfsx.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

45.6.2 Axiom help page [Demm05]

— **sgbrfsx.help** —

```
==========================================================================
sgbrfsx examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgbrfsx (TRANS, EQUED, N, KL, KU, NRHS, AB, LDAB, AFB,
           LDAFB, IPIV, R, C, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS,
           ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, IWORK, INFO)
           SGBRFSX

Function/Subroutine Documentation
   subroutine sgbrfsx (character TRANS, character EQUED, integer N, integer
       KL, integer KU, integer NRHS, real, dimension( ldab, * ) AB, integer
       LDAB, real, dimension( ldafb, * ) AFB, integer LDAFB, integer,
       dimension( * ) IPIV, real, dimension( * ) R, real, dimension( * ) C,
       real, dimension( ldb, * ) B, integer LDB, real, dimension( ldx , * ) X,
       integer LDX, real RCOND, real, dimension( * ) BERR, integer N_ERR_BNDS,
       real, dimension( nrhs, * ) ERR_BNDS_NORM, real, dimension( nrhs, * )
       ERR_BNDS_COMP, integer NPARAMS, real, dimension( * ) PARAMS, real,
       dimension( * ) WORK, integer, dimension( * ) IWORK, integer INFO)
       SGBRFSX

Purpose:
   SGBRFSX improves the computed solution to a system of linear
   equations and provides error bounds and backward error estimates
   for the solution.  In addition to normwise error bound, the code
   provides maximum componentwise error bound if possible.  See
   comments for ERR_BNDS_NORM and ERR_BNDS_COMP for details of the
   error bounds.

   The original system of linear equations may have been equilibrated
```

```
   before calling this routine, as described by arguments EQUED, R
   and C below. In this case, the solution and error bounds returned
   are for the original unequilibrated system.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose = Transpose)

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done to A
   before calling this routine. This is needed to compute
   the solution and error bounds correctly.
   = 'N':  No equilibration
   = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
   = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
   = 'B':  Both row and column equilibration, i.e., A has been
           replaced by diag(R) * A * diag(C).
           The right hand side B has been changed accordingly.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 AB
   AB is DOUBLE PRECISION array, dimension (LDAB,N)
```

```
  The original band matrix A, stored in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(n,j+kl).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is DOUBLE PRECISION array, dimension (LDAFB,N)
  Details of the LU factorization of the band matrix A, as
  computed by DGBTRF.  U is stored as an upper triangular band
  matrix with KL+KU superdiagonals in rows 1 to KL+KU+1, and
  the multipliers used during the factorization are stored in
  rows KL+KU+2 to 2*KL+KU+1.

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL*KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from SGETRF; for 1<=i<=N, row i of the
  matrix was interchanged with row IPIV(i).

R
  R is REAL array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
  EQUED = 'R' or 'B', each element of R must be positive.
  If R is output, each element of R is a power of the radix.
  If R is input, each element of R should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

C
  C is REAL array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.
  If C is output, each element of C is a power of the radix.
  If C is input, each element of C should be a power of the radix
```

```
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is REAL array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is REAL array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by SGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

BERR
  BERR is REAL array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:
```

```
  Normwise relative error in the ith solution vector:
            max_j (abs(XTRUE(j,i) - X(j,i)))
             ----------------------------
                   max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.

ERR_BNDS_COMP
  ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                 abs(XTRUE(j,i) - X(j,i))
           max_j ----------------------
                        abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
```

```
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.


NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is REAL array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
  (other values are reserved for future use)
```

```
  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is REAL array, dimension (4*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is
  guaranteed.
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
     has been completed, but the factor U is exactly singular, so
     the solution and error bounds could not be computed. RCOND = 0
     is returned.
  = N+J: The solution corresponding to the Jth right-hand side is
     not guaranteed. The solutions corresponding to other right-
     hand sides K with K > J may not be guaranteed as well, but
     only the first such right-hand side is reported. If a small
     componentwise error is not requested (PARAMS(3) = 0.0) then
     the Jth right-hand side is the first with a normwise error
     bound that is not guaranteed (the smallest J such
     that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
     the Jth right-hand side is the first with either a normwise or
     componentwise error bound that is not guaranteed (the smallest
     J such that either ERR_BNDS_NORM(J,1) = 0.0 or
     ERR_BNDS_COMP(J,1) = 0.0). See the definition of
     ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
     about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

45.6.3 fortran code

— sgbrfsx.f —

```
*  ====================================================================
      SUBROUTINE SGBRFSX( TRANS, EQUED, N, KL, KU, NRHS, AB, LDAB, AFB,
     $                    LDAFB, IPIV, R, C, B, LDB, X, LDX, RCOND,
     $                    BERR, N_ERR_BNDS, ERR_BNDS_NORM,
     $                    ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, IWORK,
     $                    INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS, EQUED
      INTEGER            INFO, LDAB, LDAFB, LDB, LDX, N, KL, KU, NRHS,
     $                   NPARAMS, N_ERR_BNDS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      REAL               AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   X( LDX , * ),WORK( * )
      REAL               R( * ), C( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               ITREF_DEFAULT, ITHRESH_DEFAULT,
     $                   COMPONENTWISE_DEFAULT
      REAL               RTHRESH_DEFAULT, DZTHRESH_DEFAULT
      PARAMETER          ( ITREF_DEFAULT = 1.0 )
      PARAMETER          ( ITHRESH_DEFAULT = 10.0 )
      PARAMETER          ( COMPONENTWISE_DEFAULT = 1.0 )
      PARAMETER          ( RTHRESH_DEFAULT = 0.5 )
      PARAMETER          ( DZTHRESH_DEFAULT = 0.25 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
```

```
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. Local Scalars ..
      CHARACTER(1)       NORM
      LOGICAL            ROWEQU, COLEQU, NOTRAN
      INTEGER            J, TRANS_TYPE, PREC_TYPE, REF_TYPE
      INTEGER            N_NORMS
      REAL               ANORM, RCOND_TMP
      REAL               ILLRCOND_THRESH, ERR_LBND, CWISE_WRONG
      LOGICAL            IGNORE_CWISE
      INTEGER            ITHRESH
      REAL               RTHRESH, UNSTABLE_THRESH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, SGBCON
      EXTERNAL           SLA_GBRFSX_EXTENDED
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, BLAS_FPINFO_X, ILATRANS, ILAPREC
      EXTERNAL           SLAMCH, SLANGB, SLA_GBRCOND
      REAL               SLAMCH, SLANGB, SLA_GBRCOND
      LOGICAL            LSAME
      INTEGER            BLAS_FPINFO_X
      INTEGER            ILATRANS, ILAPREC
*     ..
*     .. Executable Statements ..
*
*     Check the input parameters.
*
      INFO = 0
      TRANS_TYPE = ILATRANS( TRANS )
      REF_TYPE = INT( ITREF_DEFAULT )
      IF ( NPARAMS .GE. LA_LINRX_ITREF_I ) THEN
         IF ( PARAMS( LA_LINRX_ITREF_I ) .LT. 0.0 ) THEN
            PARAMS( LA_LINRX_ITREF_I ) = ITREF_DEFAULT
         ELSE
            REF_TYPE = PARAMS( LA_LINRX_ITREF_I )
         END IF
      END IF
*
*     Set default parameters.
*
      ILLRCOND_THRESH = REAL( N ) * SLAMCH( 'Epsilon' )
      ITHRESH = INT( ITHRESH_DEFAULT )
```

```
      RTHRESH = RTHRESH_DEFAULT
      UNSTABLE_THRESH = DZTHRESH_DEFAULT
      IGNORE_CWISE = COMPONENTWISE_DEFAULT .EQ. 0.0
*
      IF ( NPARAMS.GE.LA_LINRX_ITHRESH_I ) THEN
         IF ( PARAMS( LA_LINRX_ITHRESH_I ).LT.0.0 ) THEN
            PARAMS( LA_LINRX_ITHRESH_I ) = ITHRESH
         ELSE
            ITHRESH = INT( PARAMS( LA_LINRX_ITHRESH_I ) )
         END IF
      END IF
      IF ( NPARAMS.GE.LA_LINRX_CWISE_I ) THEN
         IF ( PARAMS( LA_LINRX_CWISE_I ).LT.0.0 ) THEN
            IF ( IGNORE_CWISE ) THEN
               PARAMS( LA_LINRX_CWISE_I ) = 0.0
            ELSE
               PARAMS( LA_LINRX_CWISE_I ) = 1.0
            END IF
         ELSE
            IGNORE_CWISE = PARAMS( LA_LINRX_CWISE_I ) .EQ. 0.0
         END IF
      END IF
      IF ( REF_TYPE .EQ. 0 .OR. N_ERR_BNDS .EQ. 0 ) THEN
         N_NORMS = 0
      ELSE IF ( IGNORE_CWISE ) THEN
         N_NORMS = 1
      ELSE
         N_NORMS = 2
      END IF
*
      NOTRAN = LSAME( TRANS, 'N' )
      ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
      COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
*
*     Test input parameters.
*
      IF( TRANS_TYPE.EQ.-1 ) THEN
        INFO = -1
      ELSE IF( .NOT.ROWEQU .AND. .NOT.COLEQU .AND.
     $         .NOT.LSAME( EQUED, 'N' ) ) THEN
        INFO = -2
      ELSE IF( N.LT.0 ) THEN
        INFO = -3
      ELSE IF( KL.LT.0 ) THEN
        INFO = -4
      ELSE IF( KU.LT.0 ) THEN
        INFO = -5
      ELSE IF( NRHS.LT.0 ) THEN
        INFO = -6
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
```

```
        INFO = -8
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
        INFO = -10
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
        INFO = -13
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
        INFO = -15
      END IF
      IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'SGBRFSX', -INFO )
        RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         RCOND = 1.0
         DO J = 1, NRHS
            BERR( J ) = 0.0
            IF ( N_ERR_BNDS .GE. 1 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
            IF ( N_ERR_BNDS .GE. 2 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 0.0
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 0.0
            END IF
            IF ( N_ERR_BNDS .GE. 3 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 1.0
            END IF
         END DO
         RETURN
      END IF
*
*     Default to failure.
*
      RCOND = 0.0
      DO J = 1, NRHS
         BERR( J ) = 1.0
         IF ( N_ERR_BNDS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
            ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
         END IF
         IF ( N_ERR_BNDS .GE. 2 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
         END IF
         IF ( N_ERR_BNDS .GE. 3 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 0.0
```

```
            ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 0.0
         END IF
      END DO
*
*     Compute the norm of A and the reciprocal of the condition
*     number of A.
*
      IF( NOTRAN ) THEN
         NORM = 'I'
      ELSE
         NORM = '1'
      END IF
      ANORM = SLANGB( NORM, N, KL, KU, AB, LDAB, WORK )
      CALL SGBCON( NORM, N, KL, KU, AFB, LDAFB, IPIV, ANORM, RCOND,
     $     WORK, IWORK, INFO )
*
*     Perform refinement on each right-hand side
*
      IF (REF_TYPE .NE. 0) THEN

         PREC_TYPE = ILAPREC( 'D' )

         IF ( NOTRAN ) THEN
            CALL SLA_GBRFSX_EXTENDED( PREC_TYPE, TRANS_TYPE,  N, KL, KU,
     $           NRHS, AB, LDAB, AFB, LDAFB, IPIV, COLEQU, C, B,
     $           LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM,
     $           ERR_BNDS_COMP, WORK( N+1 ), WORK( 1 ), WORK( 2*N+1 ),
     $           WORK( 1 ), RCOND, ITHRESH, RTHRESH, UNSTABLE_THRESH,
     $           IGNORE_CWISE, INFO )
         ELSE
            CALL SLA_GBRFSX_EXTENDED( PREC_TYPE, TRANS_TYPE,  N, KL, KU,
     $           NRHS, AB, LDAB, AFB, LDAFB, IPIV, ROWEQU, R, B,
     $           LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM,
     $           ERR_BNDS_COMP, WORK( N+1 ), WORK( 1 ), WORK( 2*N+1 ),
     $           WORK( 1 ), RCOND, ITHRESH, RTHRESH, UNSTABLE_THRESH,
     $           IGNORE_CWISE, INFO )
         END IF
      END IF

      ERR_LBND = MAX( 10.0, SQRT( REAL( N ) ) ) * SLAMCH( 'Epsilon' )
      IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 1 ) THEN
*
*     Compute scaled normwise condition number cond(A*C).
*
         IF ( COLEQU .AND. NOTRAN ) THEN
            RCOND_TMP = SLA_GBRCOND( TRANS, N, KL, KU, AB, LDAB, AFB,
     $           LDAFB, IPIV, -1, C, INFO, WORK, IWORK )
         ELSE IF ( ROWEQU .AND. .NOT. NOTRAN ) THEN
            RCOND_TMP = SLA_GBRCOND( TRANS, N, KL, KU, AB, LDAB, AFB,
     $           LDAFB, IPIV, -1, R, INFO, WORK, IWORK )
```

```
         ELSE
            RCOND_TMP = SLA_GBRCOND( TRANS, N, KL, KU, AB, LDAB, AFB,
     $           LDAFB, IPIV, 0, R, INFO, WORK, IWORK )
         END IF
         DO J = 1, NRHS
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .GT. 1.0 )
     $           ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 0.0
               IF ( INFO .LE. N ) INFO = N + J
            ELSE IF ( ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .LT. ERR_LBND )
     $     THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF

         END DO
      END IF

      IF (N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 2) THEN
*
*     Compute componentwise condition number cond(A*diag(Y(:,J))) for
*     each right-hand side using the current solution as an estimate of
*     the true solution.  If the componentwise error estimate is too
*     large, then the solution is a lousy estimate of truth and the
*     estimated RCOND may be too optimistic.  To avoid misleading users,
*     the inverse condition number is set to 0.0 when the estimated
*     cwise error is at least CWISE_WRONG.
*
         CWISE_WRONG = SQRT( SLAMCH( 'Epsilon' ) )
         DO J = 1, NRHS
            IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .LT. CWISE_WRONG )
     $     THEN
               RCOND_TMP = SLA_GBRCOND( TRANS, N, KL, KU, AB, LDAB, AFB,
     $              LDAFB, IPIV, 1, X( 1, J ), INFO, WORK, IWORK )
            ELSE
```

```
               RCOND_TMP = 0.0
            END IF
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .GT. 1.0 )
     $           ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 0.0
               IF ( PARAMS( LA_LINRX_CWISE_I ) .EQ. 1.0
     $              .AND. INFO.LT.N + J ) INFO = N + J
            ELSE IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I )
     $              .LT. ERR_LBND ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF

         END DO
      END IF
*
      RETURN
*
*     End of SGBRFSX
*
      END
```

———-

45.6.4 lisp code

— sgbrfsx.lisp —

———-

45.7 sgbtf2

45.7.1 Axiom unit tests

— sgbtf2.input —

```
)set break resume
)sys rm -f sgbtf2.output
)spool sgbtf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

45.7.2 Axiom help page

— sgbtf2.help —

```
======================================================================
sgbtf2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgbtf2 (M, N, KL, KU, AB, LDAB, IPIV, INFO)
           SGBTF2 computes the LU factorization of a general band matrix using
           the unblocked version of the algorithm.

Function/Subroutine Documentation
   subroutine sgbtf2 (integer M, integer N, integer KL, integer KU, real,
       dimension( ldab, * ) AB, integer LDAB, integer, dimension( * ) IPIV,
       integer INFO)
       SGBTF2 computes the LU factorization of a general band matrix using the
       unblocked version of the algorithm.

Purpose:
   SGBTF2 computes an LU factorization of a real m-by-n band matrix A
   using partial pivoting with row interchanges.
```

```
  This is the unblocked version of the algorithm, calling Level 2 BLAS.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 AB
   AB is REAL array, dimension (LDAB,N)
   On entry, the matrix A in band storage, in rows KL+1 to
   2*KL+KU+1; rows 1 to KL of the array need not be set.
   The j-th column of A is stored in the j-th column of the
   array AB as follows:
   AB(kl+ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl)

   On exit, details of the factorization: U is stored as an
   upper triangular band matrix with KL+KU superdiagonals in
   rows 1 to KL+KU+1, and the multipliers used during the
   factorization are stored in rows KL+KU+2 to 2*KL+KU+1.
   See below for further details.

 LDAB
   LDAB is INTEGER
   The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.

 IPIV
   IPIV is INTEGER array, dimension (min(M,N))
   The pivot indices; for 1 <= i <= min(M,N), row i of the
   matrix was interchanged with row IPIV(i).

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value
   > 0: if INFO = +i, U(i,i) is exactly zero. The factorization
      has been completed, but the factor U is exactly
```

```
      singular, and division by zero will occur if it is used
      to solve a system of equations.

Further Details:

   The band storage scheme is illustrated by the following example, when
   M = N = 6, KL = 2, KU = 1:

        On entry:                       On exit:

            *    *    *    +    +    +       *    *    *   u14  u25  u36
            *    *    +    +    +    +       *    *   u13  u24  u35  u46
            *   a12  a23  a34  a45  a56      *   u12  u23  u34  u45  u56
           a11  a22  a33  a44  a55  a66     u11  u22  u33  u44  u55  u66
           a21  a32  a43  a54  a65   *      m21  m32  m43  m54  m65   *
           a31  a42  a53  a64   *    *      m31  m42  m53  m64   *    *

   Array elements marked * are not used by the routine; elements marked
   + need not be set on entry, but are required by the routine to store
   elements of U, because of fill-in resulting from the row interchanges.
```

———-

45.7.3 fortran code

— sgbtf2.f —

```
*  =====================================================================
      SUBROUTINE SGBTF2( M, N, KL, KU, AB, LDAB, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, KL, KU, LDAB, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               AB( LDAB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
```

```
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, JP, JU, KM, KV
*     ..
*     .. External Functions ..
      INTEGER            ISAMAX
      EXTERNAL           ISAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGER, SSCAL, SSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     KV is the number of superdiagonals in the factor U, allowing for
*     fill-in.
*
      KV = KU + KL
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KV+1 ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGBTF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Gaussian elimination with partial pivoting
*
*     Set fill-in elements in columns KU+2 to KV to zero.
*
```

```
      DO 20 J = KU + 2, MIN( KV, N )
         DO 10 I = KV - J + 2, KL
            AB( I, J ) = ZERO
   10    CONTINUE
   20 CONTINUE
*
*     JU is the index of the last column affected by the current stage
*     of the factorization.
*
      JU = 1
*
      DO 40 J = 1, MIN( M, N )
*
*        Set fill-in elements in column J+KV to zero.
*
         IF( J+KV.LE.N ) THEN
            DO 30 I = 1, KL
               AB( I, J+KV ) = ZERO
   30       CONTINUE
         END IF
*
*        Find pivot and test for singularity. KM is the number of
*        subdiagonal elements in the current column.
*
         KM = MIN( KL, M-J )
         JP = ISAMAX( KM+1, AB( KV+1, J ), 1 )
         IPIV( J ) = JP + J - 1
         IF( AB( KV+JP, J ).NE.ZERO ) THEN
            JU = MAX( JU, MIN( J+KU+JP-1, N ) )
*
*           Apply interchange to columns J to JU.
*
            IF( JP.NE.1 )
     $         CALL SSWAP( JU-J+1, AB( KV+JP, J ), LDAB-1,
     $                     AB( KV+1, J ), LDAB-1 )
*
            IF( KM.GT.0 ) THEN
*
*              Compute multipliers.
*
               CALL SSCAL( KM, ONE / AB( KV+1, J ), AB( KV+2, J ), 1 )
*
*              Update trailing submatrix within the band.
*
               IF( JU.GT.J )
     $            CALL SGER( KM, JU-J, -ONE, AB( KV+2, J ), 1,
     $                       AB( KV, J+1 ), LDAB-1, AB( KV+1, J+1 ),
     $                       LDAB-1 )
            END IF
         ELSE
```

```
*
*          If pivot is zero, set INFO to the index of the pivot
*          unless a zero pivot has already been found.
*
           IF( INFO.EQ.0 )
     $        INFO = J
        END IF
   40 CONTINUE
      RETURN
*
*     End of SGBTF2
*
      END
```

———-

45.7.4 lisp code

— **sgbtf2.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun sgbtf2 (m n kl ku ab ldab ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array single-float (*)) ab)
             (type (f2cl-lib:integer4) info ldab ku kl n m))
    (f2cl-lib:with-multi-array-data
        ((ab single-float ab-%data% ab-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((i 0) (j 0) (jp 0) (ju 0) (km 0) (kv 0))
        (declare (type (f2cl-lib:integer4) i j jp ju km kv))
        (setf kv (f2cl-lib:int-add ku kl))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< kl 0) (setf info -3)) ((< ku 0) (setf info -4))
              ((< ldab (f2cl-lib:int-add kl kv 1)) (setf info -6)))
        (cond
         ((/= info 0) (xerbla "SGBTF2" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (or (= m 0) (= n 0))
            (go end_label))
        (f2cl-lib:fdo (j (f2cl-lib:int-add ku 2) (f2cl-lib:int-add j 1))
                      ((> j
                          (min (the f2cl-lib:integer4 kv)
                               (the f2cl-lib:integer4 n)))
                       nil)
```

```
  (tagbody
    (f2cl-lib:fdo (i (f2cl-lib:int-add kv (f2cl-lib:int-sub j) 2)
                     (f2cl-lib:int-add i 1))
                  ((> i kl) nil)
      (tagbody
        (setf (f2cl-lib:fref ab-%data% (i j) ((1 ldab) (1 *))
                             ab-%offset%)
              zero)
       label10))
   label20))
(setf ju 1)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j
                  (min (the f2cl-lib:integer4 m)
                       (the f2cl-lib:integer4 n)))
               nil)
  (tagbody
    (cond
     ((<= (f2cl-lib:int-add j kv) n)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i kl) nil)
        (tagbody
          (setf (f2cl-lib:fref ab-%data% (i (f2cl-lib:int-add j kv))
                               ((1 ldab) (1 *)) ab-%offset%)
                zero)
         label30))))
    (setf km
            (min (the f2cl-lib:integer4 kl)
                 (the f2cl-lib:integer4 (f2cl-lib:int-sub m j))))
    (setf jp
            (isamax (f2cl-lib:int-add km 1)
             (f2cl-lib:array-slice ab-%data% single-float ((+ kv 1) j)
                                   ((1 ldab) (1 *)) ab-%offset%)
             1))
    (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
            (f2cl-lib:int-sub (f2cl-lib:int-add jp j) 1))
    (cond
     ((/=
       (f2cl-lib:fref ab ((f2cl-lib:int-add kv jp) j) ((1 ldab) (1 *)))
       zero)
      (setf ju
              (max (the f2cl-lib:integer4 ju)
                   (the f2cl-lib:integer4
                        (min
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-sub
                               (f2cl-lib:int-add j ku jp) 1))
                         (the f2cl-lib:integer4 n)))))
      (if (/= jp 1)
          (sswap (f2cl-lib:int-add (f2cl-lib:int-sub ju j) 1)
```

```
                 (f2cl-lib:array-slice ab-%data% single-float ((+ kv jp) j)
                                       ((1 ldab) (1 *)) ab-%offset%)
                 (f2cl-lib:int-sub ldab 1)
                 (f2cl-lib:array-slice ab-%data% single-float ((+ kv 1) j)
                                       ((1 ldab) (1 *)) ab-%offset%)
                 (f2cl-lib:int-sub ldab 1)))
            (cond
             ((> km 0)
              (multiple-value-bind (var-0 var-1 var-2 var-3)
                  (sscal km
                   (/ one
                      (f2cl-lib:fref ab-%data% ((f2cl-lib:int-add kv 1) j)
                                     ((1 ldab) (1 *)) ab-%offset%))
                   (f2cl-lib:array-slice ab-%data% single-float ((+ kv 2) j)
                                         ((1 ldab) (1 *)) ab-%offset%)
                   1)
                (declare (ignore var-1 var-2 var-3))
                (when var-0 (setf km var-0)))
              (if (> ju j)
                  (multiple-value-bind
                      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                      (sger km (f2cl-lib:int-sub ju j) (- one)
                       (f2cl-lib:array-slice ab-%data% single-float
                                             ((+ kv 2) j) ((1 ldab) (1 *))
                                             ab-%offset%)
                       1
                       (f2cl-lib:array-slice ab-%data% single-float
                                             (kv (f2cl-lib:int-add j 1))
                                             ((1 ldab) (1 *)) ab-%offset%)
                       (f2cl-lib:int-sub ldab 1)
                       (f2cl-lib:array-slice ab-%data% single-float
                                             ((+ kv 1)
                                              (f2cl-lib:int-add j 1))
                                             ((1 ldab) (1 *)) ab-%offset%)
                       (f2cl-lib:int-sub ldab 1))
                    (declare
                     (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7
                      var-8))
                    (when var-0 (setf km var-0)))))))
           (t
            (if (= info 0)
                (setf info j))))
        label40))
      (go end_label)
     end_label
      (return (values nil nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
```

```
(setf (gethash 'fortran-to-lisp::sgbtf2
               fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (array single-float (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::integer4
                                             (*))
                                            (fortran-to-lisp::integer4))
                                          :return-values
                                          '(nil nil nil nil nil nil nil
                                            fortran-to-lisp::info)
                                          :calls 'nil)))
```

———-

45.8 sgbtrf

45.8.1 Axiom unit tests

— **sgbtrf.input** —

```
)set break resume
)sys rm -f sgbtrf.output
)spool sgbtrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

45.8.2 Axiom help page

— **sgbtrf.help** —

```
====================================================================
```

```
sgbtrf examples
===================================================================

===================================================================
Man Page Details
===================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgbtrf (M, N, KL, KU, AB, LDAB, IPIV, INFO)
           SGBTRF

Function/Subroutine Documentation
   subroutine sgbtrf (integer M, integer N, integer KL, integer KU, real,
       dimension( ldab, * ) AB, integer LDAB, integer, dimension( * ) IPIV,
       integer INFO)
       SGBTRF

Purpose:
   SGBTRF computes an LU factorization of a real m-by-n band matrix A
   using partial pivoting with row interchanges.

   This is the blocked version of the algorithm, calling Level 3 BLAS.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 AB
   AB is REAL array, dimension (LDAB,N)
   On entry, the matrix A in band storage, in rows KL+1 to
   2*KL+KU+1; rows 1 to KL of the array need not be set.
   The j-th column of A is stored in the j-th column of the
   array AB as follows:
   AB(kl+ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl)
```

```
  On exit, details of the factorization: U is stored as an
  upper triangular band matrix with KL+KU superdiagonals in
  rows 1 to KL+KU+1, and the multipliers used during the
  factorization are stored in rows KL+KU+2 to 2*KL+KU+1.
  See below for further details.

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (min(M,N))
  The pivot indices; for 1 <= i <= min(M,N), row i of the
  matrix was interchanged with row IPIV(i).

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = +i, U(i,i) is exactly zero. The factorization
       has been completed, but the factor U is exactly
       singular, and division by zero will occur if it is used
       to solve a system of equations.

Further Details:

  The band storage scheme is illustrated by the following example, when
  M = N = 6, KL = 2, KU = 1:

       On entry:                       On exit:

           *    *    *    +    +    +       *    *    *   u14  u25  u36
           *    *    +    +    +    +       *    *   u13  u24  u35  u46
           *   a12  a23  a34  a45  a56       *   u12  u23  u34  u45  u56
          a11  a22  a33  a44  a55  a66     u11  u22  u33  u44  u55  u66
          a21  a32  a43  a54  a65   *      m21  m32  m43  m54  m65   *
          a31  a42  a53  a64   *    *      m31  m42  m53  m64   *    *

  Array elements marked * are not used by the routine; elements marked
  + need not be set on entry, but are required by the routine to store
  elements of U because of fill-in resulting from the row interchanges.
```

———-

45.8.3 fortran code

— sgbtrf.f —

```
*  =====================================================================
      SUBROUTINE SGBTRF( M, N, KL, KU, AB, LDAB, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, KL, KU, LDAB, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               AB( LDAB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
      INTEGER            NBMAX, LDWORK
      PARAMETER          ( NBMAX = 64, LDWORK = NBMAX+1 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, I2, I3, II, IP, J, J2, J3, JB, JJ, JM, JP,
     $                   JU, K2, KM, KV, NB, NW
      REAL               TEMP
*     ..
*     .. Local Arrays ..
      REAL               WORK13( LDWORK, NBMAX ),
     $                   WORK31( LDWORK, NBMAX )
*     ..
*     .. External Functions ..
      INTEGER            ILAENV, ISAMAX
      EXTERNAL           ILAENV, ISAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SGBTF2, SGEMM, SGER, SLASWP, SSCAL,
     $                   SSWAP, STRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     KV is the number of superdiagonals in the factor U, allowing for
*     fill-in
*
      KV = KU + KL
```

```
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KV+1 ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGBTRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Determine the block size for this environment
*
      NB = ILAENV( 1, 'SGBTRF', ' ', M, N, KL, KU )
*
*     The block size must not exceed the limit set by the size of the
*     local arrays WORK13 and WORK31.
*
      NB = MIN( NB, NBMAX )
*
      IF( NB.LE.1 .OR. NB.GT.KL ) THEN
*
*        Use unblocked code
*
         CALL SGBTF2( M, N, KL, KU, AB, LDAB, IPIV, INFO )
      ELSE
*
*        Use blocked code
*
*        Zero the superdiagonal elements of the work array WORK13
*
         DO 20 J = 1, NB
            DO 10 I = 1, J - 1
               WORK13( I, J ) = ZERO
   10       CONTINUE
   20    CONTINUE
```

```
*
*        Zero the subdiagonal elements of the work array WORK31
*
         DO 40 J = 1, NB
            DO 30 I = J + 1, NB
               WORK31( I, J ) = ZERO
   30       CONTINUE
   40    CONTINUE
*
*        Gaussian elimination with partial pivoting
*
*        Set fill-in elements in columns KU+2 to KV to zero
*
         DO 60 J = KU + 2, MIN( KV, N )
            DO 50 I = KV - J + 2, KL
               AB( I, J ) = ZERO
   50       CONTINUE
   60    CONTINUE
*
*        JU is the index of the last column affected by the current
*        stage of the factorization
*
         JU = 1
*
         DO 180 J = 1, MIN( M, N ), NB
            JB = MIN( NB, MIN( M, N )-J+1 )
*
*           The active part of the matrix is partitioned
*
*              A11   A12   A13
*              A21   A22   A23
*              A31   A32   A33
*
*           Here A11, A21 and A31 denote the current block of JB columns
*           which is about to be factorized. The number of rows in the
*           partitioning are JB, I2, I3 respectively, and the numbers
*           of columns are JB, J2, J3. The superdiagonal elements of A13
*           and the subdiagonal elements of A31 lie outside the band.
*
            I2 = MIN( KL-JB, M-J-JB+1 )
            I3 = MIN( JB, M-J-KL+1 )
*
*           J2 and J3 are computed after JU has been updated.
*
*           Factorize the current block of JB columns
*
            DO 80 JJ = J, J + JB - 1
*
*              Set fill-in elements in column JJ+KV to zero
*
```

```
               IF( JJ+KV.LE.N ) THEN
                  DO 70 I = 1, KL
                     AB( I, JJ+KV ) = ZERO
   70             CONTINUE
               END IF
*
*              Find pivot and test for singularity. KM is the number of
*              subdiagonal elements in the current column.
*
               KM = MIN( KL, M-JJ )
               JP = ISAMAX( KM+1, AB( KV+1, JJ ), 1 )
               IPIV( JJ ) = JP + JJ - J
               IF( AB( KV+JP, JJ ).NE.ZERO ) THEN
                  JU = MAX( JU, MIN( JJ+KU+JP-1, N ) )
                  IF( JP.NE.1 ) THEN
*
*                    Apply interchange to columns J to J+JB-1
*
                     IF( JP+JJ-1.LT.J+KL ) THEN
*
                        CALL SSWAP( JB, AB( KV+1+JJ-J, J ), LDAB-1,
     $                              AB( KV+JP+JJ-J, J ), LDAB-1 )
                     ELSE
*
*                       The interchange affects columns J to JJ-1 of A31
*                       which are stored in the work array WORK31
*
                        CALL SSWAP( JJ-J, AB( KV+1+JJ-J, J ), LDAB-1,
     $                              WORK31( JP+JJ-J-KL, 1 ), LDWORK )
                        CALL SSWAP( J+JB-JJ, AB( KV+1, JJ ), LDAB-1,
     $                              AB( KV+JP, JJ ), LDAB-1 )
                     END IF
                  END IF
*
*                 Compute multipliers
*
                  CALL SSCAL( KM, ONE / AB( KV+1, JJ ), AB( KV+2, JJ ),
     $                        1 )
*
*                 Update trailing submatrix within the band and within
*                 the current block. JM is the index of the last column
*                 which needs to be updated.
*
                  JM = MIN( JU, J+JB-1 )
                  IF( JM.GT.JJ )
     $               CALL SGER( KM, JM-JJ, -ONE, AB( KV+2, JJ ), 1,
     $                          AB( KV, JJ+1 ), LDAB-1,
     $                          AB( KV+1, JJ+1 ), LDAB-1 )
               ELSE
*
```

```
*                 If pivot is zero, set INFO to the index of the pivot
*                 unless a zero pivot has already been found.
*
                  IF( INFO.EQ.0 )
     $               INFO = JJ
               END IF
*
*              Copy current column of A31 into the work array WORK31
*
               NW = MIN( JJ-J+1, I3 )
               IF( NW.GT.0 )
     $            CALL SCOPY( NW, AB( KV+KL+1-JJ+J, JJ ), 1,
     $                        WORK31( 1, JJ-J+1 ), 1 )
   80       CONTINUE
            IF( J+JB.LE.N ) THEN
*
*              Apply the row interchanges to the other blocks.
*
               J2 = MIN( JU-J+1, KV ) - JB
               J3 = MAX( 0, JU-J-KV+1 )
*
*              Use SLASWP to apply the row interchanges to A12, A22, and
*              A32.
*
               CALL SLASWP( J2, AB( KV+1-JB, J+JB ), LDAB-1, 1, JB,
     $                      IPIV( J ), 1 )
*
*              Adjust the pivot indices.
*
               DO 90 I = J, J + JB - 1
                  IPIV( I ) = IPIV( I ) + J - 1
   90          CONTINUE
*
*              Apply the row interchanges to A13, A23, and A33
*              columnwise.
*
               K2 = J - 1 + JB + J2
               DO 110 I = 1, J3
                  JJ = K2 + I
                  DO 100 II = J + I - 1, J + JB - 1
                     IP = IPIV( II )
                     IF( IP.NE.II ) THEN
                        TEMP = AB( KV+1+II-JJ, JJ )
                        AB( KV+1+II-JJ, JJ ) = AB( KV+1+IP-JJ, JJ )
                        AB( KV+1+IP-JJ, JJ ) = TEMP
                     END IF
  100             CONTINUE
  110          CONTINUE
*
*              Update the relevant part of the trailing submatrix
```

```
*
               IF( J2.GT.0 ) THEN
*
*                 Update A12
*
                  CALL STRSM( 'Left', 'Lower', 'No transpose', 'Unit',
     $                        JB, J2, ONE, AB( KV+1, J ), LDAB-1,
     $                        AB( KV+1-JB, J+JB ), LDAB-1 )
*
                  IF( I2.GT.0 ) THEN
*
*                    Update A22
*
                     CALL SGEMM( 'No transpose', 'No transpose', I2, J2,
     $                           JB, -ONE, AB( KV+1+JB, J ), LDAB-1,
     $                           AB( KV+1-JB, J+JB ), LDAB-1, ONE,
     $                           AB( KV+1, J+JB ), LDAB-1 )
                  END IF
*
                  IF( I3.GT.0 ) THEN
*
*                    Update A32
*
                     CALL SGEMM( 'No transpose', 'No transpose', I3, J2,
     $                           JB, -ONE, WORK31, LDWORK,
     $                           AB( KV+1-JB, J+JB ), LDAB-1, ONE,
     $                           AB( KV+KL+1-JB, J+JB ), LDAB-1 )
                  END IF
               END IF
*
               IF( J3.GT.0 ) THEN
*
*                 Copy the lower triangle of A13 into the work array
*                 WORK13
*
                  DO 130 JJ = 1, J3
                     DO 120 II = JJ, JB
                        WORK13( II, JJ ) = AB( II-JJ+1, JJ+J+KV-1 )
  120                CONTINUE
  130             CONTINUE
*
*                 Update A13 in the work array
*
                  CALL STRSM( 'Left', 'Lower', 'No transpose', 'Unit',
     $                        JB, J3, ONE, AB( KV+1, J ), LDAB-1,
     $                        WORK13, LDWORK )
*
                  IF( I2.GT.0 ) THEN
*
*                    Update A23
```

```
*
                    CALL SGEMM( 'No transpose', 'No transpose', I2, J3,
     $                          JB, -ONE, AB( KV+1+JB, J ), LDAB-1,
     $                          WORK13, LDWORK, ONE, AB( 1+JB, J+KV ),
     $                          LDAB-1 )
                 END IF
*
                 IF( I3.GT.0 ) THEN
*
*                   Update A33
*
                    CALL SGEMM( 'No transpose', 'No transpose', I3, J3,
     $                          JB, -ONE, WORK31, LDWORK, WORK13,
     $                          LDWORK, ONE, AB( 1+KL, J+KV ), LDAB-1 )
                 END IF
*
*                Copy the lower triangle of A13 back into place
*
                 DO 150 JJ = 1, J3
                    DO 140 II = JJ, JB
                       AB( II-JJ+1, JJ+J+KV-1 ) = WORK13( II, JJ )
  140               CONTINUE
  150            CONTINUE
              END IF
           ELSE
*
*             Adjust the pivot indices.
*
              DO 160 I = J, J + JB - 1
                 IPIV( I ) = IPIV( I ) + J - 1
  160         CONTINUE
           END IF
*
*          Partially undo the interchanges in the current block to
*          restore the upper triangular form of A31 and copy the upper
*          triangle of A31 back into place
*
           DO 170 JJ = J + JB - 1, J, -1
              JP = IPIV( JJ ) - JJ + 1
              IF( JP.NE.1 ) THEN
*
*                Apply interchange to columns J to JJ-1
*
                 IF( JP+JJ-1.LT.J+KL ) THEN
*
*                   The interchange does not affect A31
*
                    CALL SSWAP( JJ-J, AB( KV+1+JJ-J, J ), LDAB-1,
     $                          AB( KV+JP+JJ-J, J ), LDAB-1 )
                 ELSE
```

```
*
*                    The interchange does affect A31
*
                     CALL SSWAP( JJ-J, AB( KV+1+JJ-J, J ), LDAB-1,
     $                           WORK31( JP+JJ-J-KL, 1 ), LDWORK )
                  END IF
               END IF
*
*              Copy the current column of A31 back into place
*
               NW = MIN( I3, JJ-J+1 )
               IF( NW.GT.0 )
     $            CALL SCOPY( NW, WORK31( 1, JJ-J+1 ), 1,
     $                        AB( KV+KL+1-JJ+J, JJ ), 1 )
  170       CONTINUE
  180    CONTINUE
      END IF
*
      RETURN
*
*     End of SGBTRF
*
      END
```

———-

45.8.4 lisp code

— sgbtrf.lisp —

```
(let* ((one 1.0) (zero 0.0) (nbmax 64) (ldwork (+ nbmax 1)))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (type (f2cl-lib:integer4 64 64) nbmax)
           (type (f2cl-lib:integer4) ldwork)
           (ignorable one zero nbmax ldwork))
  (defun sgbtrf (m n kl ku ab ldab ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array single-float (*)) ab)
             (type (f2cl-lib:integer4) info ldab ku kl n m))
    (f2cl-lib:with-multi-array-data
        ((ab single-float ab-%data% ab-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((work13
              (make-array (the fixnum (reduce #'* (list ldwork nbmax)))
                          :element-type 'single-float))
             (work31
              (make-array (the fixnum (reduce #'* (list ldwork nbmax)))
```

```
                  :element-type 'single-float))
     (temp 0.0) (i 0) (i2 0) (i3 0) (ii 0) (ip 0) (j 0) (j2 0) (j3 0)
     (jb 0) (jj 0) (jm 0) (jp 0) (ju 0) (k2 0) (km 0) (kv 0) (nb 0)
     (nw 0))
(declare (type (array single-float (*)) work13 work31)
         (type (single-float) temp)
         (type (f2cl-lib:integer4) i i2 i3 ii ip j j2 j3 jb jj jm jp ju
          k2 km kv nb nw))
(setf kv (f2cl-lib:int-add ku kl))
(setf info 0)
(cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
      ((< kl 0) (setf info -3)) ((< ku 0) (setf info -4))
      ((< ldab (f2cl-lib:int-add kl kv 1)) (setf info -6)))
(cond
 ((/= info 0) (xerbla "SGBTRF" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= m 0) (= n 0))
    (go end_label))
(setf nb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 1 "SGBTRF" " " m n kl ku)
          (declare (ignore var-0 var-1 var-2))
          (setf m var-3)
          (setf n var-4)
          (setf kl var-5)
          (setf ku var-6)
          ret-val))
(setf nb
        (min (the f2cl-lib:integer4 nb) (the f2cl-lib:integer4 nbmax)))
(cond
 ((or (<= nb 1) (> nb kl))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (sgbtf2 m n kl ku ab ldab ipiv info)
    (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6))
    (setf info var-7)))
 (t
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nb) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref work13 (i j) ((1 ldwork) (1 nbmax)))
                  zero)
         label10))
     label20))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                ((> j nb) nil)
  (tagbody
    (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                  ((> i nb) nil)
      (tagbody
        (setf (f2cl-lib:fref work31 (i j) ((1 ldwork) (1 nbmax)))
                zero)
       label30))
   label40))
(f2cl-lib:fdo (j (f2cl-lib:int-add ku 2) (f2cl-lib:int-add j 1))
              ((> j
                  (min (the f2cl-lib:integer4 kv)
                       (the f2cl-lib:integer4 n)))
               nil)
  (tagbody
    (f2cl-lib:fdo (i (f2cl-lib:int-add kv (f2cl-lib:int-sub j) 2)
                   (f2cl-lib:int-add i 1))
                  ((> i kl) nil)
      (tagbody
        (setf (f2cl-lib:fref ab-%data% (i j) ((1 ldab) (1 *))
                             ab-%offset%)
                zero)
       label50))
   label60))
(setf ju 1)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
              ((> j
                  (min (the f2cl-lib:integer4 m)
                       (the f2cl-lib:integer4 n)))
               nil)
  (tagbody
    (setf jb
            (min (the f2cl-lib:integer4 nb)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add
                       (f2cl-lib:int-sub
                        (min (the f2cl-lib:integer4 m)
                             (the f2cl-lib:integer4 n))
                        j)
                       1))))
    (setf i2
            (min (the f2cl-lib:integer4 (f2cl-lib:int-sub kl jb))
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-sub m j jb)
                                        1))))
    (setf i3
            (min (the f2cl-lib:integer4 jb)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-sub m j kl)
                                        1))))
```

```
(f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
              ((> jj
                  (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (cond
     ((<= (f2cl-lib:int-add jj kv) n)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i kl) nil)
        (tagbody
          (setf (f2cl-lib:fref ab-%data%
                               (i (f2cl-lib:int-add jj kv))
                               ((1 ldab) (1 *)) ab-%offset%)
                  zero)
        label70))))
    (setf km
            (min (the f2cl-lib:integer4 kl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-sub m jj))))
    (setf jp
            (isamax (f2cl-lib:int-add km 1)
             (f2cl-lib:array-slice ab-%data% single-float
                                   ((+ kv 1) jj) ((1 ldab) (1 *))
                                   ab-%offset%)
             1))
    (setf (f2cl-lib:fref ipiv-%data% (jj) ((1 *)) ipiv-%offset%)
            (f2cl-lib:int-sub (f2cl-lib:int-add jp jj) j))
    (cond
     ((/=
       (f2cl-lib:fref ab ((f2cl-lib:int-add kv jp) jj)
                      ((1 ldab) (1 *)))
       zero)
      (setf ju
              (max (the f2cl-lib:integer4 ju)
                   (the f2cl-lib:integer4
                        (min
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-sub
                               (f2cl-lib:int-add jj ku jp) 1))
                         (the f2cl-lib:integer4 n)))))
      (cond
       ((/= jp 1)
        (cond
         ((< (f2cl-lib:int-add jp jj (f2cl-lib:int-sub 1))
             (f2cl-lib:int-add j kl))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (sswap jb
               (f2cl-lib:array-slice ab-%data% single-float
                                     ((+ kv 1 jj
                                         (f2cl-lib:int-sub j))
```

```
                                    j)
                                   ((1 ldab) (1 *))
                                   ab-%offset%)
                (f2cl-lib:int-sub ldab 1)
                (f2cl-lib:array-slice ab-%data% single-float
                                   ((+ kv jp jj
                                       (f2cl-lib:int-sub j))
                                    j)
                                   ((1 ldab) (1 *))
                                   ab-%offset%)
                (f2cl-lib:int-sub ldab 1))
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf jb var-0))))
            (t
             (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                 (sswap (f2cl-lib:int-sub jj j)
                  (f2cl-lib:array-slice ab-%data% single-float
                                   ((+ kv 1 jj
                                       (f2cl-lib:int-sub j))
                                    j)
                                   ((1 ldab) (1 *))
                                   ab-%offset%)
                  (f2cl-lib:int-sub ldab 1)
                  (f2cl-lib:array-slice work31 single-float
                                   ((+ jp jj
                                       (f2cl-lib:int-sub j)
                                       (f2cl-lib:int-sub kl))
                                    1)
                                   ((1 ldwork) (1 nbmax)))
                  ldwork)
              (declare (ignore var-0 var-1 var-2 var-3))
              (when var-4 (setf ldwork var-4)))
             (sswap (f2cl-lib:int-sub (f2cl-lib:int-add j jb) jj)
              (f2cl-lib:array-slice ab-%data% single-float
                                 ((+ kv 1) jj) ((1 ldab) (1 *))
                                 ab-%offset%)
              (f2cl-lib:int-sub ldab 1)
              (f2cl-lib:array-slice ab-%data% single-float
                                 ((+ kv jp) jj) ((1 ldab) (1 *))
                                 ab-%offset%)
              (f2cl-lib:int-sub ldab 1))))))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal km
             (/ one
                (f2cl-lib:fref ab-%data%
                               ((f2cl-lib:int-add kv 1) jj)
                               ((1 ldab) (1 *)) ab-%offset%))
             (f2cl-lib:array-slice ab-%data% single-float
                                 ((+ kv 2) jj) ((1 ldab) (1 *))
                                 ab-%offset%)
```

```
      1)
    (declare (ignore var-1 var-2 var-3))
    (when var-0 (setf km var-0)))
  (setf jm
          (min (the f2cl-lib:integer4 ju)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-sub (f2cl-lib:int-add j jb)
                                      1))))
  (if (> jm jj)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
           var-8)
          (sger km (f2cl-lib:int-sub jm jj) (- one)
           (f2cl-lib:array-slice ab-%data% single-float
                                 ((+ kv 2) jj)
                                 ((1 ldab) (1 *))
                                 ab-%offset%)
           1
           (f2cl-lib:array-slice ab-%data% single-float
                                 (kv (f2cl-lib:int-add jj 1))
                                 ((1 ldab) (1 *))
                                 ab-%offset%)
           (f2cl-lib:int-sub ldab 1)
           (f2cl-lib:array-slice ab-%data% single-float
                                 ((+ kv 1)
                                  (f2cl-lib:int-add jj 1))
                                 ((1 ldab) (1 *))
                                 ab-%offset%)
           (f2cl-lib:int-sub ldab 1))
        (declare
         (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7
          var-8))
        (when var-0 (setf km var-0)))))
 (t
  (if (= info 0)
      (setf info jj))))
(setf nw
        (min
         (the f2cl-lib:integer4
              (f2cl-lib:int-add (f2cl-lib:int-sub jj j) 1))
         (the f2cl-lib:integer4 i3)))
(if (> nw 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy nw
         (f2cl-lib:array-slice ab-%data% single-float
                               ((+ kv kl 1
                                   (f2cl-lib:int-sub jj) j)
                                jj)
                               ((1 ldab) (1 *)) ab-%offset%)
         1
```

```
                (f2cl-lib:array-slice work31 single-float
                                      (1
                                       (f2cl-lib:int-add
                                        (f2cl-lib:int-sub jj j) 1))
                                      ((1 ldwork) (1 nbmax)))
                1)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf nw var-0))))
   label80))
(cond
 ((<= (f2cl-lib:int-add j jb) n)
  (setf j2
          (f2cl-lib:int-sub
           (min
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-sub ju j) 1))
            (the f2cl-lib:integer4 kv))
           jb))
  (setf j3
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub ju j kv)
                                      1))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaswp j2
       (f2cl-lib:array-slice ab-%data% single-float
                             ((+ kv 1 (f2cl-lib:int-sub jb))
                              (f2cl-lib:int-add j jb))
                             ((1 ldab) (1 *)) ab-%offset%)
       (f2cl-lib:int-sub ldab 1) 1 jb
       (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (j)
                             ((1 *)) ipiv-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-5 var-6))
    (when var-0 (setf j2 var-0))
    (when var-4 (setf jb var-4)))
  (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                ((> i
                    (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
              (f2cl-lib:int-sub
               (f2cl-lib:int-add
                (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                               ipiv-%offset%)
                j)
               1))
     label90))
```

```
(setf k2 (f2cl-lib:int-add (f2cl-lib:int-sub j 1) jb j2))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i j3) nil)
  (tagbody
    (setf jj (f2cl-lib:int-add k2 i))
    (f2cl-lib:fdo (ii
                   (f2cl-lib:int-add j i (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add ii 1))
                  ((> ii
                      (f2cl-lib:int-add j jb
                                        (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf ip
                (f2cl-lib:fref ipiv-%data% (ii) ((1 *))
                               ipiv-%offset%))
        (cond
         ((/= ip ii)
          (setf temp
                  (f2cl-lib:fref ab-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-add kv 1 ii)
                                   jj)
                                  jj)
                                 ((1 ldab) (1 *)) ab-%offset%))
          (setf (f2cl-lib:fref ab-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add kv 1 ii) jj)
                                jj)
                               ((1 ldab) (1 *)) ab-%offset%)
                  (f2cl-lib:fref ab-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-add kv 1 ip)
                                   jj)
                                  jj)
                                 ((1 ldab) (1 *)) ab-%offset%))
          (setf (f2cl-lib:fref ab-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add kv 1 ip) jj)
                                jj)
                               ((1 ldab) (1 *)) ab-%offset%)
                  temp)))
       label100))
   label110))
(cond
 ((> j2 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (strsm "Left" "Lower" "No transpose" "Unit" jb j2 one
```

```
      (f2cl-lib:array-slice ab-%data% single-float
                            ((+ kv 1) j) ((1 ldab) (1 *))
                            ab-%offset%)
      (f2cl-lib:int-sub ldab 1)
      (f2cl-lib:array-slice ab-%data% single-float
                            ((+ kv 1 (f2cl-lib:int-sub jb))
                             (f2cl-lib:int-add j jb))
                            ((1 ldab) (1 *)) ab-%offset%)
      (f2cl-lib:int-sub ldab 1))
  (declare
   (ignore var-0 var-1 var-2 var-3 var-7 var-8 var-9 var-10))
  (when var-4 (setf jb var-4))
  (when var-5 (setf j2 var-5))
  (when var-6 (setf one var-6)))
(cond
 ((> i2 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (sgemm "No transpose" "No transpose" i2 j2 jb (- one)
       (f2cl-lib:array-slice ab-%data% single-float
                             ((+ kv 1 jb) j) ((1 ldab) (1 *))
                             ab-%offset%)
       (f2cl-lib:int-sub ldab 1)
       (f2cl-lib:array-slice ab-%data% single-float
                             ((+ kv 1 (f2cl-lib:int-sub jb))
                              (f2cl-lib:int-add j jb))
                             ((1 ldab) (1 *)) ab-%offset%)
       (f2cl-lib:int-sub ldab 1) one
       (f2cl-lib:array-slice ab-%data% single-float
                             ((+ kv 1)
                              (f2cl-lib:int-add j jb))
                             ((1 ldab) (1 *)) ab-%offset%)
       (f2cl-lib:int-sub ldab 1))
    (declare
     (ignore var-0 var-1 var-5 var-6 var-7 var-8 var-9 var-11
      var-12))
    (when var-2 (setf i2 var-2))
    (when var-3 (setf j2 var-3))
    (when var-4 (setf jb var-4))
    (when var-10 (setf one var-10)))))
(cond
 ((> i3 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (sgemm "No transpose" "No transpose" i3 j2 jb (- one)
       work31 ldwork
       (f2cl-lib:array-slice ab-%data% single-float
                             ((+ kv 1 (f2cl-lib:int-sub jb))
```

```
                                 (f2cl-lib:int-add j jb))
                                ((1 ldab) (1 *)) ab-%offset%)
          (f2cl-lib:int-sub ldab 1) one
          (f2cl-lib:array-slice ab-%data% single-float
                                ((+ kv kl 1
                                    (f2cl-lib:int-sub jb))
                                 (f2cl-lib:int-add j jb))
                                ((1 ldab) (1 *)) ab-%offset%)
          (f2cl-lib:int-sub ldab 1))
      (declare
       (ignore var-0 var-1 var-5 var-6 var-8 var-9 var-11
        var-12))
      (when var-2 (setf i3 var-2))
      (when var-3 (setf j2 var-3))
      (when var-4 (setf jb var-4))
      (when var-7 (setf ldwork var-7))
      (when var-10 (setf one var-10)))))))
(cond
 ((> j3 0)
  (f2cl-lib:fdo (jj 1 (f2cl-lib:int-add jj 1))
                ((> jj j3) nil)
    (tagbody
      (f2cl-lib:fdo (ii jj (f2cl-lib:int-add ii 1))
                    ((> ii jb) nil)
        (tagbody
          (setf (f2cl-lib:fref work13 (ii jj)
                                ((1 ldwork) (1 nbmax)))
                  (f2cl-lib:fref ab-%data%
                                  ((f2cl-lib:int-add
                                    (f2cl-lib:int-sub ii jj) 1)
                                   (f2cl-lib:int-sub
                                    (f2cl-lib:int-add jj j kv)
                                    1))
                                  ((1 ldab) (1 *)) ab-%offset%))
          label120))
     label130))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (strsm "Left" "Lower" "No transpose" "Unit" jb j3 one
       (f2cl-lib:array-slice ab-%data% single-float
                             ((+ kv 1) j) ((1 ldab) (1 *))
                             ab-%offset%)
       (f2cl-lib:int-sub ldab 1) work13 ldwork)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-7 var-8 var-9))
    (when var-4 (setf jb var-4))
    (when var-5 (setf j3 var-5))
    (when var-6 (setf one var-6))
    (when var-10 (setf ldwork var-10)))
```

```
(cond
 ((> i2 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (sgemm "No transpose" "No transpose" i2 j3 jb (- one)
       (f2cl-lib:array-slice ab-%data% single-float
                             ((+ kv 1 jb) j) ((1 ldab) (1 *))
                             ab-%offset%)
       (f2cl-lib:int-sub ldab 1) work13 ldwork one
       (f2cl-lib:array-slice ab-%data% single-float
                             ((+ 1 jb)
                              (f2cl-lib:int-add j kv))
                             ((1 ldab) (1 *)) ab-%offset%)
       (f2cl-lib:int-sub ldab 1))
    (declare
     (ignore var-0 var-1 var-5 var-6 var-7 var-8 var-11
      var-12))
    (when var-2 (setf i2 var-2))
    (when var-3 (setf j3 var-3))
    (when var-4 (setf jb var-4))
    (when var-9 (setf ldwork var-9))
    (when var-10 (setf one var-10)))))
(cond
 ((> i3 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (sgemm "No transpose" "No transpose" i3 j3 jb (- one)
       work31 ldwork work13 ldwork one
       (f2cl-lib:array-slice ab-%data% single-float
                             ((+ 1 kl)
                              (f2cl-lib:int-add j kv))
                             ((1 ldab) (1 *)) ab-%offset%)
       (f2cl-lib:int-sub ldab 1))
    (declare
     (ignore var-0 var-1 var-5 var-6 var-8 var-11 var-12))
    (when var-2 (setf i3 var-2))
    (when var-3 (setf j3 var-3))
    (when var-4 (setf jb var-4))
    (when var-7 (setf ldwork var-7))
    (when var-9 (setf ldwork var-9))
    (when var-10 (setf one var-10)))))
(f2cl-lib:fdo (jj 1 (f2cl-lib:int-add jj 1))
              ((> jj j3) nil)
  (tagbody
    (f2cl-lib:fdo (ii jj (f2cl-lib:int-add ii 1))
                  ((> ii jb) nil)
      (tagbody
        (setf (f2cl-lib:fref ab-%data%
```

```
                                    ((f2cl-lib:int-add
                                      (f2cl-lib:int-sub ii jj) 1)
                                     (f2cl-lib:int-sub
                                      (f2cl-lib:int-add jj j kv) 1))
                                    ((1 ldab) (1 *)) ab-%offset%)
                        (f2cl-lib:fref work13 (ii jj)
                                      ((1 ldwork) (1 nbmax))))
              label140))
         label150)))))
  (t
   (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                 ((> i
                     (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
                  nil)
     (tagbody
       (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
               (f2cl-lib:int-sub
                (f2cl-lib:int-add
                 (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                                ipiv-%offset%)
                 j)
                1))
      label160))))
 (f2cl-lib:fdo (jj (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1))
                (f2cl-lib:int-add jj (f2cl-lib:int-sub 1)))
               ((> jj j) nil)
   (tagbody
     (setf jp
             (f2cl-lib:int-add
              (f2cl-lib:int-sub
               (f2cl-lib:fref ipiv-%data% (jj) ((1 *))
                              ipiv-%offset%)
               jj)
              1))
     (cond
      ((/= jp 1)
       (cond
        ((< (f2cl-lib:int-add jp jj (f2cl-lib:int-sub 1))
            (f2cl-lib:int-add j kl))
         (sswap (f2cl-lib:int-sub jj j)
          (f2cl-lib:array-slice ab-%data% single-float
                                ((+ kv 1 jj (f2cl-lib:int-sub j))
                                 j)
                                ((1 ldab) (1 *)) ab-%offset%)
          (f2cl-lib:int-sub ldab 1)
          (f2cl-lib:array-slice ab-%data% single-float
                                ((+ kv jp jj (f2cl-lib:int-sub j))
                                 j)
                                ((1 ldab) (1 *)) ab-%offset%)
          (f2cl-lib:int-sub ldab 1)))
```

```
                    (t
                     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                         (sswap (f2cl-lib:int-sub jj j)
                          (f2cl-lib:array-slice ab-%data% single-float
                                                ((+ kv 1 jj
                                                    (f2cl-lib:int-sub j))
                                                 j)
                                                ((1 ldab) (1 *)) ab-%offset%)
                          (f2cl-lib:int-sub ldab 1)
                          (f2cl-lib:array-slice work31 single-float
                                                ((+ jp jj (f2cl-lib:int-sub j)
                                                    (f2cl-lib:int-sub kl))
                                                 1)
                                                ((1 ldwork) (1 nbmax)))
                          ldwork)
                       (declare (ignore var-0 var-1 var-2 var-3))
                       (when var-4 (setf ldwork var-4)))))))
                 (setf nw
                         (min (the f2cl-lib:integer4 i3)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add (f2cl-lib:int-sub jj j)
                                                     1))))
                 (if (> nw 0)
                     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                         (scopy nw
                          (f2cl-lib:array-slice work31 single-float
                                                (1
                                                 (f2cl-lib:int-add
                                                  (f2cl-lib:int-sub jj j) 1))
                                                ((1 ldwork) (1 nbmax)))
                          1
                          (f2cl-lib:array-slice ab-%data% single-float
                                                ((+ kv kl 1
                                                    (f2cl-lib:int-sub jj) j)
                                                 jj)
                                                ((1 ldab) (1 *)) ab-%offset%)
                          1)
                       (declare (ignore var-1 var-2 var-3 var-4))
                       (when var-0 (setf nw var-0))))
                label170))
            label180))))
       (go end_label)
      end_label
       (return (values m n kl ku nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgbtrf
                 fortran-to-lisp::*f2cl-function-info*)
```

```
(fortran-to-lisp::make-f2cl-finfo :arg-types
                                  '((fortran-to-lisp::integer4)
                                    (fortran-to-lisp::integer4)
                                    (fortran-to-lisp::integer4)
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array fortran-to-lisp::integer4
                                     (*))
                                    (fortran-to-lisp::integer4))
                                  :return-values
                                  '(fortran-to-lisp::m
                                    fortran-to-lisp::n
                                    fortran-to-lisp::kl
                                    fortran-to-lisp::ku nil nil nil
                                    fortran-to-lisp::info)
                                  :calls
                                  '(fortran-to-lisp::sgbtf2
                                    fortran-to-lisp::ilaenv))))
```

———-

45.9 sgbtrs

45.9.1 Axiom unit tests

— **sgbtrs.input** —

```
)set break resume
)sys rm -f sgbtrs.output
)spool sgbtrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

45.9.2 Axiom help page

— **sgbtrs.help** —

```
====================================================================
sgbtrs examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgbtrs (TRANS, N, KL, KU, NRHS, AB, LDAB, IPIV, B, LDB,
           INFO)
           SGBTRS

Function/Subroutine Documentation
   subroutine sgbtrs (character TRANS, integer N, integer KL, integer KU,
       integer NRHS, real, dimension( ldab, * ) AB, integer LDAB, integer,
       dimension( * ) IPIV, real, dimension( ldb, * ) B, integer LDB, integer
       INFO)
       SGBTRS

Purpose:
   SGBTRS solves a system of linear equations
      A * X = B  or  A**T * X = B
   with a general band matrix A using the LU factorization computed by SGBTRF.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations.
   = 'N':  A * X = B  (No transpose)
   = 'T':  A**T* X = B  (Transpose)
   = 'C':  A**T* X = B  (Conjugate transpose = Transpose)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
```

```
  of the matrix B.  NRHS >= 0.

AB
  AB is REAL array, dimension (LDAB,N)
  Details of the LU factorization of the band matrix A, as
  computed by SGBTRF.  U is stored as an upper triangular band
  matrix with KL+KU superdiagonals in rows 1 to KL+KU+1, and
  the multipliers used during the factorization are stored in
  rows KL+KU+2 to 2*KL+KU+1.

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= N, row i of the matrix was
  interchanged with row IPIV(i).

B
  B is REAL array, dimension (LDB,NRHS)
  On entry, the right hand side matrix B.
  On exit, the solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
```

———-

45.9.3 fortran code

— sgbtrs.f —

```
*  =====================================================================
      SUBROUTINE SGBTRS( TRANS, N, KL, KU, NRHS, AB, LDAB, IPIV, B, LDB,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
```

```
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, KL, KU, LDAB, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               AB( LDAB, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LNOTI, NOTRAN
      INTEGER            I, J, KD, L, LM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEMV, SGER, SSWAP, STBSV, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDAB.LT.( 2*KL+KU+1 ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
```

```
         INFO = -10
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGBTRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      KD = KU + KL + 1
      LNOTI = KL.GT.0
*
      IF( NOTRAN ) THEN
*
*        Solve  A*X = B.
*
*        Solve L*X = B, overwriting B with X.
*
*        L is represented as a product of permutations and unit lower
*        triangular matrices L = P(1) * L(1) * ... * P(n-1) * L(n-1),
*        where each transformation L(i) is a rank-one modification of
*        the identity matrix.
*
         IF( LNOTI ) THEN
            DO 10 J = 1, N - 1
               LM = MIN( KL, N-J )
               L = IPIV( J )
               IF( L.NE.J )
     $            CALL SSWAP( NRHS, B( L, 1 ), LDB, B( J, 1 ), LDB )
               CALL SGER( LM, NRHS, -ONE, AB( KD+1, J ), 1, B( J, 1 ),
     $                    LDB, B( J+1, 1 ), LDB )
   10       CONTINUE
         END IF
*
         DO 20 I = 1, NRHS
*
*           Solve U*X = B, overwriting B with X.
*
            CALL STBSV( 'Upper', 'No transpose', 'Non-unit', N, KL+KU,
     $                  AB, LDAB, B( 1, I ), 1 )
   20    CONTINUE
*
      ELSE
*
*        Solve A**T*X = B.
*
         DO 30 I = 1, NRHS
```

```
*
*          Solve U**T*X = B, overwriting B with X.
*
           CALL STBSV( 'Upper', 'Transpose', 'Non-unit', N, KL+KU, AB,
     $                 LDAB, B( 1, I ), 1 )
   30    CONTINUE
*
*        Solve L**T*X = B, overwriting B with X.
*
         IF( LNOTI ) THEN
            DO 40 J = N - 1, 1, -1
               LM = MIN( KL, N-J )
               CALL SGEMV( 'Transpose', LM, NRHS, -ONE, B( J+1, 1 ),
     $                     LDB, AB( KD+1, J ), 1, ONE, B( J, 1 ), LDB )
               L = IPIV( J )
               IF( L.NE.J )
     $            CALL SSWAP( NRHS, B( L, 1 ), LDB, B( J, 1 ), LDB )
   40       CONTINUE
         END IF
      END IF
      RETURN
*
*     End of SGBTRS
*
      END
```

———-

45.9.4 lisp code

— **sgbtrs.lisp** —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun sgbtrs (trans n kl ku nrhs ab ldab ipiv b ldb$ info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array single-float (*)) b ab)
             (type (f2cl-lib:integer4) info ldb$ ldab nrhs ku kl n)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (ab single-float ab-%data% ab-%offset%)
         (b single-float b-%data% b-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((i 0) (j 0) (kd 0) (l 0) (lm 0) (lnoti nil) (notran nil))
        (declare (type (f2cl-lib:integer4) i j kd l lm)
                 (type f2cl-lib:logical lnoti notran))
```

```
(setf info 0)
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< kl 0) (setf info -3))
 ((< ku 0) (setf info -4)) ((< nrhs 0) (setf info -5))
 ((< ldab (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -10)))
(cond
 ((/= info 0) (xerbla "SGBTRS" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= n 0) (= nrhs 0))
    (go end_label))
(setf kd (f2cl-lib:int-add ku kl 1))
(setf lnoti (> kl 0))
(cond
 (notran
  (cond
   (lnoti
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (setf lm
                (min (the f2cl-lib:integer4 kl)
                     (the f2cl-lib:integer4 (f2cl-lib:int-sub n j))))
        (setf l (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
        (if (/= l j)
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (sswap nrhs
                 (f2cl-lib:array-slice b-%data% single-float (l 1)
```

```
                                        ((1 ldb$) (1 *)) b-%offset%)
                    ldb$
                    (f2cl-lib:array-slice b-%data% single-float (j 1)
                                        ((1 ldb$) (1 *)) b-%offset%)
                    ldb$)
                  (declare (ignore var-1 var-3))
                  (when var-0 (setf nrhs var-0))
                  (when var-2 (setf ldb$ var-2))
                  (when var-4 (setf ldb$ var-4))))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (sger lm nrhs (- one)
               (f2cl-lib:array-slice ab-%data% single-float ((+ kd 1) j)
                                     ((1 ldab) (1 *)) ab-%offset%)
               1
               (f2cl-lib:array-slice b-%data% single-float (j 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% single-float ((+ j 1) 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-2 var-3 var-4 var-5 var-7))
            (when var-0 (setf lm var-0))
            (when var-1 (setf nrhs var-1))
            (when var-6 (setf ldb$ var-6))
            (when var-8 (setf ldb$ var-8)))
        label10))))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i nrhs) nil)
   (tagbody
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (stbsv "Upper" "No transpose" "Non-unit" n
          (f2cl-lib:int-add kl ku) ab ldab
          (f2cl-lib:array-slice b-%data% single-float (1 i)
                                ((1 ldb$) (1 *)) b-%offset%)
          1)
       (declare (ignore var-0 var-1 var-2 var-4 var-5 var-7 var-8))
       (when var-3 (setf n var-3))
       (when var-6 (setf ldab var-6)))
    label20)))
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i nrhs) nil)
   (tagbody
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (stbsv "Upper" "Transpose" "Non-unit" n
          (f2cl-lib:int-add kl ku) ab ldab
          (f2cl-lib:array-slice b-%data% single-float (1 i)
```

```
                                  ((1 ldb$) (1 *)) b-%offset%)
            1)
          (declare (ignore var-0 var-1 var-2 var-4 var-5 var-7 var-8))
          (when var-3 (setf n var-3))
          (when var-6 (setf ldab var-6)))
       label30))
    (cond
     (lnoti
      (f2cl-lib:fdo (j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                     (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                    ((> j 1) nil)
        (tagbody
          (setf lm
                  (min (the f2cl-lib:integer4 kl)
                       (the f2cl-lib:integer4 (f2cl-lib:int-sub n j))))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10)
              (sgemv "Transpose" lm nrhs (- one)
               (f2cl-lib:array-slice b-%data% single-float ((+ j 1) 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice ab-%data% single-float ((+ kd 1) j)
                                     ((1 ldab) (1 *)) ab-%offset%)
               1 one
               (f2cl-lib:array-slice b-%data% single-float (j 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-0 var-3 var-4 var-6 var-7 var-9))
            (when var-1 (setf lm var-1))
            (when var-2 (setf nrhs var-2))
            (when var-5 (setf ldb$ var-5))
            (when var-8 (setf one var-8))
            (when var-10 (setf ldb$ var-10)))
          (setf l (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
          (if (/= l j)
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (sswap nrhs
                   (f2cl-lib:array-slice b-%data% single-float (l 1)
                                         ((1 ldb$) (1 *)) b-%offset%)
                   ldb$
                   (f2cl-lib:array-slice b-%data% single-float (j 1)
                                         ((1 ldb$) (1 *)) b-%offset%)
                   ldb$)
                (declare (ignore var-1 var-3))
                (when var-0 (setf nrhs var-0))
                (when var-2 (setf ldb$ var-2))
                (when var-4 (setf ldb$ var-4))))
        label40))))))
  (go end_label)
```

```
       end_label
        (return (values trans n nil nil nrhs nil ldab nil nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgbtrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::n nil nil
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::ldab nil nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

45.10 sggbak

45.10.1 Axiom unit tests

— sggbak.input —

```
)set break resume
)sys rm -f sggbak.output
)spool sggbak.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

45.10.2 Axiom help page

— **sggbak.help** —

```
==================================================================
sggbak examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sggbak (JOB, SIDE, N, ILO, IHI, LSCALE, RSCALE, M, V, LDV,
           INFO)
           SGGBAK

Function/Subroutine Documentation
   subroutine sggbak (character JOB, character SIDE, integer N, integer ILO,
       integer IHI, real, dimension( * ) LSCALE, real, dimension( * ) RSCALE,
       integer M, real, dimension( ldv, * ) V, integer LDV, integer INFO)
       SGGBAK

Purpose:
   SGGBAK forms the right or left eigenvectors of a real generalized
   eigenvalue problem A*x = lambda*B*x, by backward transformation on
   the computed eigenvectors of the balanced pair of matrices output by
   SGGBAL.

Parameters:

 JOB
   JOB is CHARACTER*1
   Specifies the type of backward transformation required:
   = 'N':  do nothing, return immediately;
   = 'P':  do backward transformation for permutation only;
   = 'S':  do backward transformation for scaling only;
   = 'B':  do backward transformations for both permutation and scaling.
   JOB must be the same as the argument JOB supplied to SGGBAL.

 SIDE
   SIDE is CHARACTER*1
```

```
  = 'R':  V contains right eigenvectors;
  = 'L':  V contains left eigenvectors.

N
  N is INTEGER
  The number of rows of the matrix V.  N >= 0.

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER
  The integers ILO and IHI determined by SGGBAL.
  1 <= ILO <= IHI <= N, if N > 0; ILO=1 and IHI=0, if N=0.

LSCALE
  LSCALE is REAL array, dimension (N)
  Details of the permutations and/or scaling factors applied
  to the left side of A and B, as returned by SGGBAL.

RSCALE
  RSCALE is REAL array, dimension (N)
  Details of the permutations and/or scaling factors applied
  to the right side of A and B, as returned by SGGBAL.

M
  M is INTEGER
  The number of columns of the matrix V.  M >= 0.

V
  V is REAL array, dimension (LDV,M)
  On entry, the matrix of right or left eigenvectors to be
  transformed, as returned by STGEVC.
  On exit, V is overwritten by the transformed eigenvectors.

LDV
  LDV is INTEGER
  The leading dimension of the matrix V. LDV >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

45.10.3 fortran code

— sggbak.f —

```
*  =====================================================================
      SUBROUTINE SGGBAK( JOB, SIDE, N, ILO, IHI, LSCALE, RSCALE, M, V,
     $                   LDV, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOB, SIDE
      INTEGER            IHI, ILO, INFO, LDV, M, N
*     ..
*     .. Array Arguments ..
      REAL               LSCALE( * ), RSCALE( * ), V( LDV, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LEFTV, RIGHTV
      INTEGER            I, K
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SSCAL, SSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      RIGHTV = LSAME( SIDE, 'R' )
      LEFTV = LSAME( SIDE, 'L' )
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.RIGHTV .AND. .NOT.LEFTV ) THEN
```

```
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( ILO.LT.1 ) THEN
         INFO = -4
      ELSE IF( N.EQ.0 .AND. IHI.EQ.0 .AND. ILO.NE.1 ) THEN
         INFO = -4
      ELSE IF( N.GT.0 .AND. ( IHI.LT.ILO .OR. IHI.GT.MAX( 1, N ) ) )
     $   THEN
         INFO = -5
      ELSE IF( N.EQ.0 .AND. ILO.EQ.1 .AND. IHI.NE.0 ) THEN
         INFO = -5
      ELSE IF( M.LT.0 ) THEN
         INFO = -8
      ELSE IF( LDV.LT.MAX( 1, N ) ) THEN
         INFO = -10
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGGBAK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
      IF( M.EQ.0 )
     $   RETURN
      IF( LSAME( JOB, 'N' ) )
     $   RETURN
*
      IF( ILO.EQ.IHI )
     $   GO TO 30
*
*     Backward balance
*
      IF( LSAME( JOB, 'S' ) .OR. LSAME( JOB, 'B' ) ) THEN
*
*        Backward transformation on right eigenvectors
*
         IF( RIGHTV ) THEN
            DO 10 I = ILO, IHI
               CALL SSCAL( M, RSCALE( I ), V( I, 1 ), LDV )
   10       CONTINUE
         END IF
*
*        Backward transformation on left eigenvectors
*
         IF( LEFTV ) THEN
            DO 20 I = ILO, IHI
```

```
               CALL SSCAL( M, LSCALE( I ), V( I, 1 ), LDV )
   20       CONTINUE
         END IF
      END IF
*
*     Backward permutation
*
   30 CONTINUE
      IF( LSAME( JOB, 'P' ) .OR. LSAME( JOB, 'B' ) ) THEN
*
*        Backward permutation on right eigenvectors
*
         IF( RIGHTV ) THEN
            IF( ILO.EQ.1 )
     $         GO TO 50
*
            DO 40 I = ILO - 1, 1, -1
               K = RSCALE( I )
               IF( K.EQ.I )
     $            GO TO 40
               CALL SSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   40       CONTINUE
*
   50       CONTINUE
            IF( IHI.EQ.N )
     $         GO TO 70
            DO 60 I = IHI + 1, N
               K = RSCALE( I )
               IF( K.EQ.I )
     $            GO TO 60
               CALL SSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   60       CONTINUE
         END IF
*
*        Backward permutation on left eigenvectors
*
   70    CONTINUE
         IF( LEFTV ) THEN
            IF( ILO.EQ.1 )
     $         GO TO 90
            DO 80 I = ILO - 1, 1, -1
               K = LSCALE( I )
               IF( K.EQ.I )
     $            GO TO 80
               CALL SSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   80       CONTINUE
*
   90       CONTINUE
            IF( IHI.EQ.N )
     $         GO TO 110
```

```
            DO 100 I = IHI + 1, N
               K = LSCALE( I )
               IF( K.EQ.I )
     $            GO TO 100
               CALL SSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
  100       CONTINUE
         END IF
      END IF
*
  110 CONTINUE
*
      RETURN
*
*     End of SGGBAK
*
      END
```

———-

45.10.4 lisp code

— sggbak.lisp —

```
(defun sggbak (job side n ilo ihi lscale rscale m v ldv info)
  (declare (type (array single-float (*)) v rscale lscale)
           (type (f2cl-lib:integer4) info ldv m ihi ilo n)
           (type (string *) side job))
  (f2cl-lib:with-multi-array-data
      ((job character job-%data% job-%offset%)
       (side character side-%data% side-%offset%)
       (lscale single-float lscale-%data% lscale-%offset%)
       (rscale single-float rscale-%data% rscale-%offset%)
       (v single-float v-%data% v-%offset%))
    (prog ((i 0) (k 0) (leftv nil) (rightv nil))
      (declare (type f2cl-lib:logical rightv leftv)
               (type (f2cl-lib:integer4) k i))
      (setf rightv
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame side "R")
                (declare (ignore var-1))
                (when var-0 (setf side var-0))
                ret-val))
      (setf leftv
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame side "L")
                (declare (ignore var-1))
                (when var-0 (setf side var-0))
                ret-val))
```

```
(setf info 0)
(cond
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "N")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "P")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "S")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "B")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)))
  (setf info -1))
 ((and (not rightv) (not leftv)) (setf info -2)) ((< n 0) (setf info -3))
 ((< ilo 1) (setf info -4))
 ((and (= n 0) (= ihi 0) (/= ilo 1)) (setf info -4))
 ((and (> n 0)
       (or (< ihi ilo)
           (> ihi
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 n)))))
  (setf info -5))
 ((and (= n 0) (= ilo 1) (/= ihi 0)) (setf info -5))
 ((< m 0) (setf info -8))
 ((< ldv (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -10)))
(cond
 ((/= info 0) (xerbla "SGGBAK" (f2cl-lib:int-sub info)) (go end_label)))
(if (= n 0)
    (go end_label))
(if (= m 0)
    (go end_label))
(if (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "N")
      (declare (ignore var-1))
```

```
        (when var-0 (setf job var-0))
        ret-val)
      (go end_label))
  (if (= ilo ihi)
      (go label30))
  (cond
   ((or
     (multiple-value-bind (ret-val var-0 var-1)
         (lsame job "S")
       (declare (ignore var-1))
       (when var-0 (setf job var-0))
       ret-val)
     (multiple-value-bind (ret-val var-0 var-1)
         (lsame job "B")
       (declare (ignore var-1))
       (when var-0 (setf job var-0))
       ret-val))
    (cond
     (rightv
      (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                    ((> i ihi) nil)
        (tagbody
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (sscal m
               (f2cl-lib:array-slice rscale-%data% single-float (i) ((1 *))
                                     rscale-%offset%)
               (f2cl-lib:array-slice v-%data% single-float (i 1)
                                     ((1 ldv) (1 *)) v-%offset%)
               ldv)
            (declare (ignore var-1 var-2))
            (when var-0 (setf m var-0))
            (when var-3 (setf ldv var-3)))
         label10))))
    (cond
     (leftv
      (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                    ((> i ihi) nil)
        (tagbody
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (sscal m
               (f2cl-lib:array-slice lscale-%data% single-float (i) ((1 *))
                                     lscale-%offset%)
               (f2cl-lib:array-slice v-%data% single-float (i 1)
                                     ((1 ldv) (1 *)) v-%offset%)
               ldv)
            (declare (ignore var-1 var-2))
            (when var-0 (setf m var-0))
            (when var-3 (setf ldv var-3)))
         label20))))))
label30
```

```
(cond
 ((or
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame job "P")
     (declare (ignore var-1))
     (when var-0 (setf job var-0))
     ret-val)
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame job "B")
     (declare (ignore var-1))
     (when var-0 (setf job var-0))
     ret-val))
  (tagbody
    (cond
     (rightv
      (tagbody
        (if (= ilo 1)
            (go label50))
        (f2cl-lib:fdo (i (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))
                       (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (setf k
                    (f2cl-lib:int
                     (f2cl-lib:fref rscale-%data% (i) ((1 *))
                                    rscale-%offset%)))
            (if (= k i)
                (go label40))
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (sswap m
                 (f2cl-lib:array-slice v-%data% single-float (i 1)
                                       ((1 ldv) (1 *)) v-%offset%)
                 ldv
                 (f2cl-lib:array-slice v-%data% single-float (k 1)
                                       ((1 ldv) (1 *)) v-%offset%)
                 ldv)
              (declare (ignore var-1 var-3))
              (when var-0 (setf m var-0))
              (when var-2 (setf ldv var-2))
              (when var-4 (setf ldv var-4)))
           label40))
       label50
        (if (= ihi n)
            (go label70))
        (f2cl-lib:fdo (i (f2cl-lib:int-add ihi 1) (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf k
                    (f2cl-lib:int
                     (f2cl-lib:fref rscale-%data% (i) ((1 *))
```

```
                                    rscale-%offset%)))
            (if (= k i)
                (go label60))
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (sswap m
                 (f2cl-lib:array-slice v-%data% single-float (i 1)
                                       ((1 ldv) (1 *)) v-%offset%)
                 ldv
                 (f2cl-lib:array-slice v-%data% single-float (k 1)
                                       ((1 ldv) (1 *)) v-%offset%)
                 ldv)
              (declare (ignore var-1 var-3))
              (when var-0 (setf m var-0))
              (when var-2 (setf ldv var-2))
              (when var-4 (setf ldv var-4)))
          label60)))))
label70
 (cond
  (leftv
   (tagbody
     (if (= ilo 1)
         (go label90))
     (f2cl-lib:fdo (i (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))
                    (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                   ((> i 1) nil)
       (tagbody
         (setf k
                 (f2cl-lib:int
                  (f2cl-lib:fref lscale-%data% (i) ((1 *))
                                 lscale-%offset%)))
         (if (= k i)
             (go label80))
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (sswap m
              (f2cl-lib:array-slice v-%data% single-float (i 1)
                                    ((1 ldv) (1 *)) v-%offset%)
              ldv
              (f2cl-lib:array-slice v-%data% single-float (k 1)
                                    ((1 ldv) (1 *)) v-%offset%)
              ldv)
           (declare (ignore var-1 var-3))
           (when var-0 (setf m var-0))
           (when var-2 (setf ldv var-2))
           (when var-4 (setf ldv var-4)))
         label80))
    label90
     (if (= ihi n)
         (go label110))
     (f2cl-lib:fdo (i (f2cl-lib:int-add ihi 1) (f2cl-lib:int-add i 1))
                   ((> i n) nil)
```

```
                (tagbody
                  (setf k
                          (f2cl-lib:int
                           (f2cl-lib:fref lscale-%data% (i) ((1 *))
                                          lscale-%offset%)))
                  (if (= k i)
                      (go label100))
                  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                      (sswap m
                       (f2cl-lib:array-slice v-%data% single-float (i 1)
                                             ((1 ldv) (1 *)) v-%offset%)
                       ldv
                       (f2cl-lib:array-slice v-%data% single-float (k 1)
                                             ((1 ldv) (1 *)) v-%offset%)
                       ldv)
                    (declare (ignore var-1 var-3))
                    (when var-0 (setf m var-0))
                    (when var-2 (setf ldv var-2))
                    (when var-4 (setf ldv var-4)))
                 label100))))))))
     label110
      (go end_label)
     end_label
      (return (values job side nil nil nil nil nil m nil ldv info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sggbak
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::job
                                              fortran-to-lisp::side nil nil nil
                                              nil nil fortran-to-lisp::m nil
                                              fortran-to-lisp::ldv
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

45.11 sggbal

45.11.1 Axiom unit tests

— **sggbal.input** —

```
)set break resume
)sys rm -f sggbal.output
)spool sggbal.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

45.11.2 Axiom help page

— **sggbal.help** —

```
====================================================================
sggbal examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sggbal (JOB, N, A, LDA, B, LDB, ILO, IHI, LSCALE, RSCALE,
           WORK, INFO)
           SGGBAL

Function/Subroutine Documentation
   subroutine sggbal (character JOB, integer N, real, dimension( lda, * ) A,
       integer LDA, real, dimension( ldb, * ) B, integer LDB, integer ILO,
       integer IHI, real, dimension( * ) LSCALE, real, dimension( * ) RSCALE,
       real, dimension( * ) WORK, integer INFO)
       SGGBAL
```

```
Purpose:
   SGGBAL balances a pair of general real matrices (A,B).  This
   involves, first, permuting A and B by similarity transformations to
   isolate eigenvalues in the first 1 to ILO$-$1 and last IHI+1 to N
   elements on the diagonal; and second, applying a diagonal similarity
   transformation to rows and columns ILO to IHI to make the rows
   and columns as close in norm as possible. Both steps are optional.

   Balancing may reduce the 1-norm of the matrices, and improve the
   accuracy of the computed eigenvalues and/or eigenvectors in the
   generalized eigenvalue problem A*x = lambda*B*x.

Parameters:

 JOB
   JOB is CHARACTER*1
   Specifies the operations to be performed on A and B:
   = 'N':  none:  simply set ILO = 1, IHI = N, LSCALE(I) = 1.0
           and RSCALE(I) = 1.0 for i = 1,...,N.
   = 'P':  permute only;
   = 'S':  scale only;
   = 'B':  both permute and scale.

 N
   N is INTEGER
   The order of the matrices A and B.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the input matrix A.
   On exit,  A is overwritten by the balanced matrix.
   If JOB = 'N', A is not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A. LDA >= max(1,N).

 B
   B is REAL array, dimension (LDB,N)
   On entry, the input matrix B.
   On exit,  B is overwritten by the balanced matrix.
   If JOB = 'N', B is not referenced.

 LDB
   LDB is INTEGER
   The leading dimension of the array B. LDB >= max(1,N).

 ILO
   ILO is INTEGER
```

```
IHI
  IHI is INTEGER
  ILO and IHI are set to integers such that on exit
  A(i,j) = 0 and B(i,j) = 0 if i > j and
  j = 1,...,ILO-1 or i = IHI+1,...,N.
  If JOB = 'N' or 'S', ILO = 1 and IHI = N.

LSCALE
  LSCALE is REAL array, dimension (N)
  Details of the permutations and scaling factors applied
  to the left side of A and B.  If P(j) is the index of the
  row interchanged with row j, and D(j)
  is the scaling factor applied to row j, then
  LSCALE(j) = P(j)    for J = 1,...,ILO-1
            = D(j)    for J = ILO,...,IHI
            = P(j)    for J = IHI+1,...,N.
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

RSCALE
  RSCALE is REAL array, dimension (N)
  Details of the permutations and scaling factors applied
  to the right side of A and B.  If P(j) is the index of the
  column interchanged with column j, and D(j)
  is the scaling factor applied to column j, then
  LSCALE(j) = P(j)    for J = 1,...,ILO-1
            = D(j)    for J = ILO,...,IHI
            = P(j)    for J = IHI+1,...,N.
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

WORK
  WORK is REAL array, dimension (lwork)
  lwork must be at least max(1,6*N) when JOB = 'S' or 'B', and
  at least 1 when JOB = 'N' or 'P'.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

———-

45.11.3 fortran code

— sggbal.f —

```
*  =====================================================================
      SUBROUTINE SGGBAL( JOB, N, A, LDA, B, LDB, ILO, IHI, LSCALE,
     $                   RSCALE, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOB
      INTEGER            IHI, ILO, INFO, LDA, LDB, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), B( LDB, * ), LSCALE( * ),
     $                   RSCALE( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, HALF, ONE
      PARAMETER          ( ZERO = 0.0E+0, HALF = 0.5E+0, ONE = 1.0E+0 )
      REAL               THREE, SCLFAC
      PARAMETER          ( THREE = 3.0E+0, SCLFAC = 1.0E+1 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, ICAB, IFLOW, IP1, IR, IRAB, IT, J, JC, JP1,
     $                   K, KOUNT, L, LCAB, LM1, LRAB, LSFMAX, LSFMIN,
     $                   M, NR, NRP2
      REAL               ALPHA, BASL, BETA, CAB, CMAX, COEF, COEF2,
     $                   COEF5, COR, EW, EWC, GAMMA, PGAMMA, RAB, SFMAX,
     $                   SFMIN, SUM, T, TA, TB, TC
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ISAMAX
      REAL               SDOT, SLAMCH
      EXTERNAL           LSAME, ISAMAX, SDOT, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           SAXPY, SSCAL, SSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, INT, LOG10, MAX, MIN, REAL, SIGN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
```

```
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGGBAL', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         ILO = 1
         IHI = N
         RETURN
      END IF
*
      IF( N.EQ.1 ) THEN
         ILO = 1
         IHI = N
         LSCALE( 1 ) = ONE
         RSCALE( 1 ) = ONE
         RETURN
      END IF
*
      IF( LSAME( JOB, 'N' ) ) THEN
         ILO = 1
         IHI = N
         DO 10 I = 1, N
            LSCALE( I ) = ONE
            RSCALE( I ) = ONE
   10    CONTINUE
         RETURN
      END IF
*
      K = 1
      L = N
      IF( LSAME( JOB, 'S' ) )
     $   GO TO 190
*
      GO TO 30
*
*     Permute the matrices A and B to isolate the eigenvalues.
```

```
*
*     Find row with one nonzero in columns 1 through L
*
   20 CONTINUE
      L = LM1
      IF( L.NE.1 )
     $   GO TO 30
*
      RSCALE( 1 ) = ONE
      LSCALE( 1 ) = ONE
      GO TO 190
*
   30 CONTINUE
      LM1 = L - 1
      DO 80 I = L, 1, -1
         DO 40 J = 1, LM1
            JP1 = J + 1
            IF( A( I, J ).NE.ZERO .OR. B( I, J ).NE.ZERO )
     $         GO TO 50
   40    CONTINUE
         J = L
         GO TO 70
*
   50    CONTINUE
         DO 60 J = JP1, L
            IF( A( I, J ).NE.ZERO .OR. B( I, J ).NE.ZERO )
     $         GO TO 80
   60    CONTINUE
         J = JP1 - 1
*
   70    CONTINUE
         M = L
         IFLOW = 1
         GO TO 160
   80 CONTINUE
      GO TO 100
*
*     Find column with one nonzero in rows K through N
*
   90 CONTINUE
      K = K + 1
*
  100 CONTINUE
      DO 150 J = K, L
         DO 110 I = K, LM1
            IP1 = I + 1
            IF( A( I, J ).NE.ZERO .OR. B( I, J ).NE.ZERO )
     $         GO TO 120
  110    CONTINUE
         I = L
```

```
         GO TO 140
  120    CONTINUE
         DO 130 I = IP1, L
            IF( A( I, J ).NE.ZERO .OR. B( I, J ).NE.ZERO )
     $         GO TO 150
  130    CONTINUE
         I = IP1 - 1
  140    CONTINUE
         M = K
         IFLOW = 2
         GO TO 160
  150 CONTINUE
      GO TO 190
*
*     Permute rows M and I
*
  160 CONTINUE
      LSCALE( M ) = I
      IF( I.EQ.M )
     $   GO TO 170
      CALL SSWAP( N-K+1, A( I, K ), LDA, A( M, K ), LDA )
      CALL SSWAP( N-K+1, B( I, K ), LDB, B( M, K ), LDB )
*
*     Permute columns M and J
*
  170 CONTINUE
      RSCALE( M ) = J
      IF( J.EQ.M )
     $   GO TO 180
      CALL SSWAP( L, A( 1, J ), 1, A( 1, M ), 1 )
      CALL SSWAP( L, B( 1, J ), 1, B( 1, M ), 1 )
*
  180 CONTINUE
      GO TO ( 20, 90 )IFLOW
*
  190 CONTINUE
      ILO = K
      IHI = L
*
      IF( LSAME( JOB, 'P' ) ) THEN
         DO 195 I = ILO, IHI
            LSCALE( I ) = ONE
            RSCALE( I ) = ONE
  195    CONTINUE
         RETURN
      END IF
*
      IF( ILO.EQ.IHI )
     $   RETURN
*
```

```
*     Balance the submatrix in rows ILO to IHI.
*
      NR = IHI - ILO + 1
      DO 200 I = ILO, IHI
         RSCALE( I ) = ZERO
         LSCALE( I ) = ZERO
*
         WORK( I ) = ZERO
         WORK( I+N ) = ZERO
         WORK( I+2*N ) = ZERO
         WORK( I+3*N ) = ZERO
         WORK( I+4*N ) = ZERO
         WORK( I+5*N ) = ZERO
  200 CONTINUE
*
*     Compute right side vector in resulting linear equations
*
      BASL = LOG10( SCLFAC )
      DO 240 I = ILO, IHI
         DO 230 J = ILO, IHI
            TB = B( I, J )
            TA = A( I, J )
            IF( TA.EQ.ZERO )
     $         GO TO 210
            TA = LOG10( ABS( TA ) ) / BASL
  210       CONTINUE
            IF( TB.EQ.ZERO )
     $         GO TO 220
            TB = LOG10( ABS( TB ) ) / BASL
  220       CONTINUE
            WORK( I+4*N ) = WORK( I+4*N ) - TA - TB
            WORK( J+5*N ) = WORK( J+5*N ) - TA - TB
  230    CONTINUE
  240 CONTINUE
*
      COEF = ONE / REAL( 2*NR )
      COEF2 = COEF*COEF
      COEF5 = HALF*COEF2
      NRP2 = NR + 2
      BETA = ZERO
      IT = 1
*
*     Start generalized conjugate gradient iteration
*
  250 CONTINUE
*
      GAMMA = SDOT( NR, WORK( ILO+4*N ), 1, WORK( ILO+4*N ), 1 ) +
     $        SDOT( NR, WORK( ILO+5*N ), 1, WORK( ILO+5*N ), 1 )
*
      EW = ZERO
```

```
      EWC = ZERO
      DO 260 I = ILO, IHI
         EW = EW + WORK( I+4*N )
         EWC = EWC + WORK( I+5*N )
  260 CONTINUE
*
      GAMMA = COEF*GAMMA - COEF2*( EW**2+EWC**2 ) - COEF5*( EW-EWC )**2
      IF( GAMMA.EQ.ZERO )
     $   GO TO 350
      IF( IT.NE.1 )
     $   BETA = GAMMA / PGAMMA
      T = COEF5*( EWC-THREE*EW )
      TC = COEF5*( EW-THREE*EWC )
*
      CALL SSCAL( NR, BETA, WORK( ILO ), 1 )
      CALL SSCAL( NR, BETA, WORK( ILO+N ), 1 )
*
      CALL SAXPY( NR, COEF, WORK( ILO+4*N ), 1, WORK( ILO+N ), 1 )
      CALL SAXPY( NR, COEF, WORK( ILO+5*N ), 1, WORK( ILO ), 1 )
*
      DO 270 I = ILO, IHI
         WORK( I ) = WORK( I ) + TC
         WORK( I+N ) = WORK( I+N ) + T
  270 CONTINUE
*
*     Apply matrix to vector
*
      DO 300 I = ILO, IHI
         KOUNT = 0
         SUM = ZERO
         DO 290 J = ILO, IHI
            IF( A( I, J ).EQ.ZERO )
     $         GO TO 280
            KOUNT = KOUNT + 1
            SUM = SUM + WORK( J )
  280       CONTINUE
            IF( B( I, J ).EQ.ZERO )
     $         GO TO 290
            KOUNT = KOUNT + 1
            SUM = SUM + WORK( J )
  290    CONTINUE
         WORK( I+2*N ) = REAL( KOUNT )*WORK( I+N ) + SUM
  300 CONTINUE
*
      DO 330 J = ILO, IHI
         KOUNT = 0
         SUM = ZERO
         DO 320 I = ILO, IHI
            IF( A( I, J ).EQ.ZERO )
     $         GO TO 310
```

```
            KOUNT = KOUNT + 1
            SUM = SUM + WORK( I+N )
  310       CONTINUE
            IF( B( I, J ).EQ.ZERO )
     $         GO TO 320
            KOUNT = KOUNT + 1
            SUM = SUM + WORK( I+N )
  320    CONTINUE
         WORK( J+3*N ) = REAL( KOUNT )*WORK( J ) + SUM
  330 CONTINUE
*
      SUM = SDOT( NR, WORK( ILO+N ), 1, WORK( ILO+2*N ), 1 ) +
     $      SDOT( NR, WORK( ILO ), 1, WORK( ILO+3*N ), 1 )
      ALPHA = GAMMA / SUM
*
*     Determine correction to current iteration
*
      CMAX = ZERO
      DO 340 I = ILO, IHI
         COR = ALPHA*WORK( I+N )
         IF( ABS( COR ).GT.CMAX )
     $      CMAX = ABS( COR )
         LSCALE( I ) = LSCALE( I ) + COR
         COR = ALPHA*WORK( I )
         IF( ABS( COR ).GT.CMAX )
     $      CMAX = ABS( COR )
         RSCALE( I ) = RSCALE( I ) + COR
  340 CONTINUE
      IF( CMAX.LT.HALF )
     $   GO TO 350
*
      CALL SAXPY( NR, -ALPHA, WORK( ILO+2*N ), 1, WORK( ILO+4*N ), 1 )
      CALL SAXPY( NR, -ALPHA, WORK( ILO+3*N ), 1, WORK( ILO+5*N ), 1 )
*
      PGAMMA = GAMMA
      IT = IT + 1
      IF( IT.LE.NRP2 )
     $   GO TO 250
*
*     End generalized conjugate gradient iteration
*
  350 CONTINUE
      SFMIN = SLAMCH( 'S' )
      SFMAX = ONE / SFMIN
      LSFMIN = INT( LOG10( SFMIN ) / BASL+ONE )
      LSFMAX = INT( LOG10( SFMAX ) / BASL )
      DO 360 I = ILO, IHI
         IRAB = ISAMAX( N-ILO+1, A( I, ILO ), LDA )
         RAB = ABS( A( I, IRAB+ILO-1 ) )
         IRAB = ISAMAX( N-ILO+1, B( I, ILO ), LDB )
```

```
         RAB = MAX( RAB, ABS( B( I, IRAB+ILO-1 ) ) )
         LRAB = INT( LOG10( RAB+SFMIN ) / BASL+ONE )
         IR = LSCALE( I ) + SIGN( HALF, LSCALE( I ) )
         IR = MIN( MAX( IR, LSFMIN ), LSFMAX, LSFMAX-LRAB )
         LSCALE( I ) = SCLFAC**IR
         ICAB = ISAMAX( IHI, A( 1, I ), 1 )
         CAB = ABS( A( ICAB, I ) )
         ICAB = ISAMAX( IHI, B( 1, I ), 1 )
         CAB = MAX( CAB, ABS( B( ICAB, I ) ) )
         LCAB = INT( LOG10( CAB+SFMIN ) / BASL+ONE )
         JC = RSCALE( I ) + SIGN( HALF, RSCALE( I ) )
         JC = MIN( MAX( JC, LSFMIN ), LSFMAX, LSFMAX-LCAB )
         RSCALE( I ) = SCLFAC**JC
  360 CONTINUE
*
*     Row scaling of matrices A and B
*
      DO 370 I = ILO, IHI
         CALL SSCAL( N-ILO+1, LSCALE( I ), A( I, ILO ), LDA )
         CALL SSCAL( N-ILO+1, LSCALE( I ), B( I, ILO ), LDB )
  370 CONTINUE
*
*     Column scaling of matrices A and B
*
      DO 380 J = ILO, IHI
         CALL SSCAL( IHI, RSCALE( J ), A( 1, J ), 1 )
         CALL SSCAL( IHI, RSCALE( J ), B( 1, J ), 1 )
  380 CONTINUE
*
      RETURN
*
*     End of SGGBAL
*
      END
```

———-

45.11.4 lisp code

— sggbal.lisp —

```
(let* ((zero 0.0) (half 0.5) (one 1.0) (three 3.0) (sclfac 10.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 0.5 0.5) half)
           (type (single-float 1.0 1.0) one)
           (type (single-float 3.0 3.0) three)
           (type (single-float 10.0 10.0) sclfac)
           (ignorable zero half one three sclfac))
```

```
(defun sggbal (job n a lda b ldb$ ilo ihi lscale rscale work info)
  (declare (type (array single-float (*)) work rscale lscale b a)
           (type (f2cl-lib:integer4) info ihi ilo ldb$ lda n)
           (type (string *) job))
  (f2cl-lib:with-multi-array-data
      ((job character job-%data% job-%offset%)
       (a single-float a-%data% a-%offset%)
       (b single-float b-%data% b-%offset%)
       (lscale single-float lscale-%data% lscale-%offset%)
       (rscale single-float rscale-%data% rscale-%offset%)
       (work single-float work-%data% work-%offset%))
    (prog ((alpha 0.0) (basl 0.0) (beta 0.0) (cab 0.0) (cmax 0.0) (coef 0.0)
           (coef2 0.0) (coef5 0.0) (cor 0.0) (ew 0.0) (ewc 0.0) (gamma 0.0)
           (pgamma 0.0) (rab 0.0) (sfmax 0.0) (sfmin 0.0) (sum 0.0) (t$ 0.0)
           (ta 0.0) (tb 0.0) (tc 0.0) (i 0) (icab 0) (iflow 0) (ip1 0) (ir 0)
           (irab 0) (it 0) (j 0) (jc 0) (jp1 0) (k 0) (kount 0) (l 0)
           (lcab 0) (lm1 0) (lrab 0) (lsfmax 0) (lsfmin 0) (m 0) (nr 0)
           (nrp2 0))
      (declare
       (type (single-float) alpha basl beta cab cmax coef coef2 coef5 cor ew
        ewc gamma pgamma rab sfmax sfmin sum t$ ta tb tc)
       (type (f2cl-lib:integer4) i icab iflow ip1 ir irab it j jc jp1 k kount
        l lcab lm1 lrab lsfmax lsfmin m nr nrp2))
      (setf info 0)
      (cond
       ((and
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame job "N")
            (declare (ignore var-1))
            (when var-0 (setf job var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame job "P")
            (declare (ignore var-1))
            (when var-0 (setf job var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame job "S")
            (declare (ignore var-1))
            (when var-0 (setf job var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame job "B")
            (declare (ignore var-1))
            (when var-0 (setf job var-0))
            ret-val)))
```

```
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4))
   ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -6)))
  (cond
   ((/= info 0) (xerbla "SGGBAL" (f2cl-lib:int-sub info))
    (go end_label)))
  (cond ((= n 0) (setf ilo 1) (setf ihi n) (go end_label)))
  (cond
   ((= n 1) (setf ilo 1) (setf ihi n)
    (setf (f2cl-lib:fref lscale-%data% (1) ((1 *)) lscale-%offset%) one)
    (setf (f2cl-lib:fref rscale-%data% (1) ((1 *)) rscale-%offset%) one)
    (go end_label)))
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame job "N")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)
    (setf ilo 1) (setf ihi n)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
                one)
        (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
                one)
       label10))
    (go end_label)))
  (setf k 1)
  (setf l n)
  (if (multiple-value-bind (ret-val var-0 var-1)
          (lsame job "S")
        (declare (ignore var-1))
        (when var-0 (setf job var-0))
        ret-val)
      (go label190))
  (go label30)
 label20
  (setf l lm1)
  (if (/= l 1)
      (go label30))
  (setf (f2cl-lib:fref rscale-%data% (1) ((1 *)) rscale-%offset%) one)
  (setf (f2cl-lib:fref lscale-%data% (1) ((1 *)) lscale-%offset%) one)
  (go label190)
 label30
  (setf lm1 (f2cl-lib:int-sub l 1))
  (f2cl-lib:fdo (i l (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
```

```
                ((> i 1) nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j lm1) nil)
        (tagbody
          (setf jp1 (f2cl-lib:int-add j 1))
          (if (or
               (/=
                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                zero)
               (/=
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)
                zero))
              (go label50))
         label40))
      (setf j l)
      (go label70)
     label50
      (f2cl-lib:fdo (j jp1 (f2cl-lib:int-add j 1))
                    ((> j l) nil)
        (tagbody
          (if (or
               (/=
                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                zero)
               (/=
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)
                zero))
              (go label80))
         label60))
      (setf j (f2cl-lib:int-sub jp1 1))
     label70
      (setf m l)
      (setf iflow 1)
      (go label160)
     label80))
  (go label100)
 label90
  (setf k (f2cl-lib:int-add k 1))
 label100
  (f2cl-lib:fdo (j k (f2cl-lib:int-add j 1))
                ((> j l) nil)
    (tagbody
      (f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
                    ((> i lm1) nil)
        (tagbody
          (setf ip1 (f2cl-lib:int-add i 1))
          (if (or
```

```
                 (/=
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                  zero)
                 (/=
                  (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  zero))
                (go label120))
           label110))
       (setf i l)
       (go label140)
      label120
       (f2cl-lib:fdo (i ip1 (f2cl-lib:int-add i 1))
                     ((> i l) nil)
         (tagbody
           (if (or
                (/=
                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                 zero)
                (/=
                 (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                 zero))
               (go label150))
           label130))
       (setf i (f2cl-lib:int-sub ip1 1))
      label140
       (setf m k)
       (setf iflow 2)
       (go label160)
      label150))
    (go label190)
   label160
    (setf (f2cl-lib:fref lscale-%data% (m) ((1 *)) lscale-%offset%)
          (coerce (the f2cl-lib:integer4 i) 'single-float))
    (if (= i m)
        (go label170))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (sswap (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
         (f2cl-lib:array-slice a-%data% single-float (i k) ((1 lda) (1 *))
                               a-%offset%)
         lda
         (f2cl-lib:array-slice a-%data% single-float (m k) ((1 lda) (1 *))
                               a-%offset%)
         lda)
      (declare (ignore var-0 var-1 var-3))
      (when var-2 (setf lda var-2))
      (when var-4 (setf lda var-4)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (sswap (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
```

```
      (f2cl-lib:array-slice b-%data% single-float (i k) ((1 ldb$) (1 *))
                            b-%offset%)
      ldb$
      (f2cl-lib:array-slice b-%data% single-float (m k) ((1 ldb$) (1 *))
                            b-%offset%)
      ldb$)
   (declare (ignore var-0 var-1 var-3))
   (when var-2 (setf ldb$ var-2))
   (when var-4 (setf ldb$ var-4)))
label170
 (setf (f2cl-lib:fref rscale-%data% (m) ((1 *)) rscale-%offset%)
        (coerce (the f2cl-lib:integer4 j) 'single-float))
 (if (= j m)
     (go label180))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (sswap l
      (f2cl-lib:array-slice a-%data% single-float (1 j) ((1 lda) (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice a-%data% single-float (1 m) ((1 lda) (1 *))
                            a-%offset%)
      1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf l var-0)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (sswap l
      (f2cl-lib:array-slice b-%data% single-float (1 j) ((1 ldb$) (1 *))
                            b-%offset%)
      1
      (f2cl-lib:array-slice b-%data% single-float (1 m) ((1 ldb$) (1 *))
                            b-%offset%)
      1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf l var-0)))
label180
 (f2cl-lib:computed-goto (label20 label90) iflow)
label190
 (setf ilo k)
 (setf ihi l)
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame job "P")
     (declare (ignore var-1))
     (when var-0 (setf job var-0))
     ret-val)
   (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                 ((> i ihi) nil)
     (tagbody
       (setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
               one)
```

```
      (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
              one)
     label195))
  (go end_label)))
(if (= ilo ihi)
    (go end_label))
(setf nr (f2cl-lib:int-add (f2cl-lib:int-sub ihi ilo) 1))
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i ihi) nil)
  (tagbody
    (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
            zero)
    (setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
            zero)
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) zero)
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n)) ((1 *))
                         work-%offset%)
            zero)
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add i (f2cl-lib:int-mul 2 n)))
                         ((1 *)) work-%offset%)
            zero)
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add i (f2cl-lib:int-mul 3 n)))
                         ((1 *)) work-%offset%)
            zero)
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add i (f2cl-lib:int-mul 4 n)))
                         ((1 *)) work-%offset%)
            zero)
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add i (f2cl-lib:int-mul 5 n)))
                         ((1 *)) work-%offset%)
            zero)
   label200))
(setf basl (f2cl-lib:log10 sclfac))
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i ihi) nil)
  (tagbody
    (f2cl-lib:fdo (j ilo (f2cl-lib:int-add j 1))
                  ((> j ihi) nil)
      (tagbody
        (setf tb
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%))
        (setf ta
                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%))
        (if (= ta zero)
            (go label210))
```

```
          (setf ta (/ (f2cl-lib:log10 (abs ta)) basl))
         label210
          (if (= tb zero)
              (go label220))
          (setf tb (/ (f2cl-lib:log10 (abs tb)) basl))
         label220
          (setf (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add i
                                                  (f2cl-lib:int-mul 4
                                                                    n)))
                               ((1 *)) work-%offset%)
                  (-
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add i
                                                     (f2cl-lib:int-mul 4
                                                                       n)))
                                  ((1 *)) work-%offset%)
                   ta tb))
          (setf (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add j
                                                  (f2cl-lib:int-mul 5
                                                                    n)))
                               ((1 *)) work-%offset%)
                  (-
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add j
                                                     (f2cl-lib:int-mul 5
                                                                       n)))
                                  ((1 *)) work-%offset%)
                   ta tb))
         label230))
     label240))
  (setf coef (/ one (f2cl-lib:freal (f2cl-lib:int-mul 2 nr))))
  (setf coef2 (* coef coef))
  (setf coef5 (* half coef2))
  (setf nrp2 (f2cl-lib:int-add nr 2))
  (setf beta zero)
  (setf it 1)
 label250
  (setf gamma
          (+
           (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
               (sdot nr
                (f2cl-lib:array-slice work-%data% single-float
                                      ((+ ilo (f2cl-lib:int-mul 4 n)))
                                      ((1 *)) work-%offset%)
                1
                (f2cl-lib:array-slice work-%data% single-float
                                      ((+ ilo (f2cl-lib:int-mul 4 n)))
                                      ((1 *)) work-%offset%)
```

```
                1)
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf nr var-0))
              ret-val)
            (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
                (sdot nr
                 (f2cl-lib:array-slice work-%data% single-float
                                       ((+ ilo (f2cl-lib:int-mul 5 n)))
                                       ((1 *)) work-%offset%)
                 1
                 (f2cl-lib:array-slice work-%data% single-float
                                       ((+ ilo (f2cl-lib:int-mul 5 n)))
                                       ((1 *)) work-%offset%)
                 1)
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf nr var-0))
              ret-val)))
(setf ew zero)
(setf ewc zero)
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i ihi) nil)
  (tagbody
    (setf ew
            (+ ew
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add i
                                                 (f2cl-lib:int-mul 4
                                                                   n)))
                              ((1 *)) work-%offset%)))
    (setf ewc
            (+ ewc
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add i
                                                 (f2cl-lib:int-mul 5
                                                                   n)))
                              ((1 *)) work-%offset%)))
   label260))
(setf gamma
        (- (* coef gamma) (* coef2 (+ (expt ew 2) (expt ewc 2)))
           (* coef5 (expt (- ew ewc) 2))))
(if (= gamma zero)
    (go label350))
(if (/= it 1)
    (setf beta (/ gamma pgamma)))
(setf t$ (* coef5 (- ewc (* three ew))))
(setf tc (* coef5 (- ew (* three ewc))))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (sscal nr beta
     (f2cl-lib:array-slice work-%data% single-float (ilo) ((1 *))
                           work-%offset%)
```

```
      1)
    (declare (ignore var-2 var-3))
    (when var-0 (setf nr var-0))
    (when var-1 (setf beta var-1)))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (sscal nr beta
       (f2cl-lib:array-slice work-%data% single-float ((+ ilo n)) ((1 *))
                             work-%offset%)
       1)
    (declare (ignore var-2 var-3))
    (when var-0 (setf nr var-0))
    (when var-1 (setf beta var-1)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (saxpy nr coef
       (f2cl-lib:array-slice work-%data% single-float
                             ((+ ilo (f2cl-lib:int-mul 4 n))) ((1 *))
                             work-%offset%)
       1
       (f2cl-lib:array-slice work-%data% single-float ((+ ilo n)) ((1 *))
                             work-%offset%)
       1)
    (declare (ignore var-2 var-3 var-4 var-5))
    (when var-0 (setf nr var-0))
    (when var-1 (setf coef var-1)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (saxpy nr coef
       (f2cl-lib:array-slice work-%data% single-float
                             ((+ ilo (f2cl-lib:int-mul 5 n))) ((1 *))
                             work-%offset%)
       1
       (f2cl-lib:array-slice work-%data% single-float (ilo) ((1 *))
                             work-%offset%)
       1)
    (declare (ignore var-2 var-3 var-4 var-5))
    (when var-0 (setf nr var-0))
    (when var-1 (setf coef var-1)))
  (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                ((> i ihi) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (+ (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 tc))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n)) ((1 *))
                           work-%offset%)
              (+
               (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n))
                              ((1 *)) work-%offset%)
               t$))
     label270))
  (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
```

```
                ((> i ihi) nil)
    (tagbody
      (setf kount 0)
      (setf sum zero)
      (f2cl-lib:fdo (j ilo (f2cl-lib:int-add j 1))
                    ((> j ihi) nil)
        (tagbody
          (if (=
               (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
               zero)
              (go label280))
          (setf kount (f2cl-lib:int-add kount 1))
          (setf sum
                  (+ sum
                     (f2cl-lib:fref work-%data% (j) ((1 *))
                                    work-%offset%)))
         label280
          (if (=
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *)) b-%offset%)
               zero)
              (go label290))
          (setf kount (f2cl-lib:int-add kount 1))
          (setf sum
                  (+ sum
                     (f2cl-lib:fref work-%data% (j) ((1 *))
                                    work-%offset%)))
         label290))
      (setf (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add i (f2cl-lib:int-mul 2 n)))
                           ((1 *)) work-%offset%)
            (+
             (* (f2cl-lib:freal kount)
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n))
                               ((1 *)) work-%offset%))
             sum))
     label300))
  (f2cl-lib:fdo (j ilo (f2cl-lib:int-add j 1))
                ((> j ihi) nil)
    (tagbody
      (setf kount 0)
      (setf sum zero)
      (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                    ((> i ihi) nil)
        (tagbody
          (if (=
               (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
               zero)
              (go label310))
          (setf kount (f2cl-lib:int-add kount 1))
          (setf sum
```

```
                 (+ sum
                    (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n))
                                   ((1 *)) work-%offset%)))
        label310
         (if (=
              (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *)) b-%offset%)
              zero)
             (go label320))
         (setf kount (f2cl-lib:int-add kount 1))
         (setf sum
                 (+ sum
                    (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n))
                                   ((1 *)) work-%offset%)))
        label320))
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add j (f2cl-lib:int-mul 3 n)))
                         ((1 *)) work-%offset%)
            (+
             (* (f2cl-lib:freal kount)
                (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%))
             sum))
   label330))
(setf sum
        (+
         (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
             (sdot nr
              (f2cl-lib:array-slice work-%data% single-float
                                    ((+ ilo n)) ((1 *)) work-%offset%)
              1
              (f2cl-lib:array-slice work-%data% single-float
                                    ((+ ilo (f2cl-lib:int-mul 2 n)))
                                    ((1 *)) work-%offset%)
              1)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf nr var-0))
            ret-val)
         (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
             (sdot nr
              (f2cl-lib:array-slice work-%data% single-float (ilo)
                                    ((1 *)) work-%offset%)
              1
              (f2cl-lib:array-slice work-%data% single-float
                                    ((+ ilo (f2cl-lib:int-mul 3 n)))
                                    ((1 *)) work-%offset%)
              1)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf nr var-0))
            ret-val)))
(setf alpha (/ gamma sum))
(setf cmax zero)
```

```
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i ihi) nil)
  (tagbody
    (setf cor
            (* alpha
               (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n))
                              ((1 *)) work-%offset%)))
    (if (> (abs cor) cmax)
        (setf cmax (abs cor)))
    (setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
            (+
             (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
             cor))
    (setf cor
            (* alpha
               (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)))
    (if (> (abs cor) cmax)
        (setf cmax (abs cor)))
    (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
            (+
             (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
             cor))
   label340))
(if (< cmax half)
    (go label350))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (saxpy nr (- alpha)
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ ilo (f2cl-lib:int-mul 2 n))) ((1 *))
                           work-%offset%)
     1
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ ilo (f2cl-lib:int-mul 4 n))) ((1 *))
                           work-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-4 var-5))
  (when var-0 (setf nr var-0)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (saxpy nr (- alpha)
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ ilo (f2cl-lib:int-mul 3 n))) ((1 *))
                           work-%offset%)
     1
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ ilo (f2cl-lib:int-mul 5 n))) ((1 *))
                           work-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-4 var-5))
  (when var-0 (setf nr var-0)))
(setf pgamma gamma)
```

```
 (setf it (f2cl-lib:int-add it 1))
 (if (<= it nrp2)
     (go label250))
label350
 (setf sfmin (slamch "S"))
 (setf sfmax (/ one sfmin))
 (setf lsfmin (f2cl-lib:int (+ (/ (f2cl-lib:log10 sfmin) basl) one)))
 (setf lsfmax (f2cl-lib:int (/ (f2cl-lib:log10 sfmax) basl)))
 (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
               ((> i ihi) nil)
   (tagbody
     (setf irab
             (multiple-value-bind (ret-val var-0 var-1 var-2)
                 (isamax (f2cl-lib:int-add (f2cl-lib:int-sub n ilo) 1)
                  (f2cl-lib:array-slice a-%data% single-float (i ilo)
                                        ((1 lda) (1 *)) a-%offset%)
                  lda)
               (declare (ignore var-0 var-1))
               (when var-2 (setf lda var-2))
               ret-val))
     (setf rab
             (abs
              (f2cl-lib:fref a-%data%
                             (i
                              (f2cl-lib:int-sub
                               (f2cl-lib:int-add irab ilo) 1))
                             ((1 lda) (1 *)) a-%offset%)))
     (setf irab
             (multiple-value-bind (ret-val var-0 var-1 var-2)
                 (isamax (f2cl-lib:int-add (f2cl-lib:int-sub n ilo) 1)
                  (f2cl-lib:array-slice b-%data% single-float (i ilo)
                                        ((1 ldb$) (1 *)) b-%offset%)
                  ldb$)
               (declare (ignore var-0 var-1))
               (when var-2 (setf ldb$ var-2))
               ret-val))
     (setf rab
             (max rab
                  (abs
                   (f2cl-lib:fref b-%data%
                                  (i
                                   (f2cl-lib:int-sub
                                    (f2cl-lib:int-add irab ilo) 1))
                                  ((1 ldb$) (1 *)) b-%offset%))))
     (setf lrab
             (f2cl-lib:int
              (+ (/ (f2cl-lib:log10 (+ rab sfmin)) basl) one)))
     (setf ir
             (f2cl-lib:int
              (+
```

```
          (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
          (f2cl-lib:sign half
                         (f2cl-lib:fref lscale-%data% (i) ((1 *))
                                        lscale-%offset%)))))
(setf ir
        (min
         (the f2cl-lib:integer4
              (max (the f2cl-lib:integer4 ir)
                   (the f2cl-lib:integer4 lsfmin)))
         (the f2cl-lib:integer4 lsfmax)
         (the f2cl-lib:integer4 (f2cl-lib:int-sub lsfmax lrab))))
(setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
        (expt sclfac ir))
(setf icab
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (isamax ihi
             (f2cl-lib:array-slice a-%data% single-float (1 i)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf ihi var-0))
          ret-val))
(setf cab
        (abs
         (f2cl-lib:fref a-%data% (icab i) ((1 lda) (1 *))
                        a-%offset%)))
(setf icab
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (isamax ihi
             (f2cl-lib:array-slice b-%data% single-float (1 i)
                                   ((1 ldb$) (1 *)) b-%offset%)
             1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf ihi var-0))
          ret-val))
(setf cab
        (max cab
             (abs
              (f2cl-lib:fref b-%data% (icab i) ((1 ldb$) (1 *))
                             b-%offset%))))
(setf lcab
        (f2cl-lib:int
         (+ (/ (f2cl-lib:log10 (+ cab sfmin)) basl) one)))
(setf jc
        (f2cl-lib:int
         (+
          (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
          (f2cl-lib:sign half
                         (f2cl-lib:fref rscale-%data% (i) ((1 *))
                                        rscale-%offset%)))))
```

```
    (setf jc
            (min
             (the f2cl-lib:integer4
                  (max (the f2cl-lib:integer4 jc)
                       (the f2cl-lib:integer4 lsfmin)))
             (the f2cl-lib:integer4 lsfmax)
             (the f2cl-lib:integer4 (f2cl-lib:int-sub lsfmax lcab))))
    (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
            (expt sclfac jc))
   label360))
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i ihi) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal (f2cl-lib:int-add (f2cl-lib:int-sub n ilo) 1)
         (f2cl-lib:array-slice lscale-%data% single-float (i) ((1 *))
                               lscale-%offset%)
         (f2cl-lib:array-slice a-%data% single-float (i ilo)
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-0 var-1 var-2))
      (when var-3 (setf lda var-3)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal (f2cl-lib:int-add (f2cl-lib:int-sub n ilo) 1)
         (f2cl-lib:array-slice lscale-%data% single-float (i) ((1 *))
                               lscale-%offset%)
         (f2cl-lib:array-slice b-%data% single-float (i ilo)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-0 var-1 var-2))
      (when var-3 (setf ldb$ var-3)))
   label370))
(f2cl-lib:fdo (j ilo (f2cl-lib:int-add j 1))
              ((> j ihi) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal ihi
         (f2cl-lib:array-slice rscale-%data% single-float (j) ((1 *))
                               rscale-%offset%)
         (f2cl-lib:array-slice a-%data% single-float (1 j)
                               ((1 lda) (1 *)) a-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf ihi var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal ihi
         (f2cl-lib:array-slice rscale-%data% single-float (j) ((1 *))
                               rscale-%offset%)
         (f2cl-lib:array-slice b-%data% single-float (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
```

```
                1)
             (declare (ignore var-1 var-2 var-3))
             (when var-0 (setf ihi var-0)))
          label380))
       (go end_label)
      end_label
       (return (values job nil nil lda nil ldb$ ilo ihi nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sggbal
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string)
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (array single-float (*))
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(fortran-to-lisp::job nil nil
                                             fortran-to-lisp::lda nil
                                             fortran-to-lisp::ldb$
                                             fortran-to-lisp::ilo
                                             fortran-to-lisp::ihi nil nil nil
                                             fortran-to-lisp::info)
                                           :calls 'nil)))
```

———-

45.12 sla_gbamv

45.12.1 Axiom unit tests

— slagbamv.input —

```
)set break resume
)sys rm -f slagbamv.output
```

```
)spool slagbamv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

45.12.2 Axiom help page

— **sla_gbamv.help** —

```
==================================================================
sla_gbamv examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sla_gbamv (TRANS, M, N, KL, KU, ALPHA, AB, LDAB, X, INCX,
           BETA, Y, INCY)
           SLA_GBAMV performs a matrix-vector operation to calculate error
           bounds.

Function/Subroutine Documentation
   subroutine sla_gbamv (integer TRANS, integer M, integer N, integer KL,
       integer KU, real ALPHA, real, dimension( ldab, * ) AB, integer LDAB,
       real, dimension( * ) X, integer INCX, real BETA, real, dimension( * )
       Y, integer INCY)
       SLA_GBAMV performs a matrix-vector operation to calculate error bounds.

Purpose:
   SLA_GBAMV  performs one of the matrix-vector operations

        y := alpha*abs(A)*abs(x) + beta*abs(y),
   or   y := alpha*abs(A)**T*abs(x) + beta*abs(y),

   where alpha and beta are scalars, x and y are vectors and A is an
   m by n matrix.

   This function is primarily used in calculating error bounds.
   To protect against underflow during evaluation, components in
   the resulting vector are perturbed away from zero by (N+1)
```

```
   times the underflow threshold.  To prevent unnecessarily large
   errors for block-structure embedded in general matrices,
   "symbolically" zero components are not perturbed.  A zero
   entry is considered "symbolic" if all multiplications involved
   in computing that entry have at least one zero multiplicand.

Parameters:

 TRANS
   TRANS is INTEGER
   On entry, TRANS specifies the operation to be performed as
   follows:

   BLAS_NO_TRANS      y := alpha*abs(A)*abs(x) + beta*abs(y)
   BLAS_TRANS         y := alpha*abs(A**T)*abs(x) + beta*abs(y)
   BLAS_CONJ_TRANS    y := alpha*abs(A**T)*abs(x) + beta*abs(y)

   Unchanged on exit.

 M
   M is INTEGER
   On entry, M specifies the number of rows of the matrix A.
   M must be at least zero.
   Unchanged on exit.

 N
   N is INTEGER
   On entry, N specifies the number of columns of the matrix A.
   N must be at least zero.
   Unchanged on exit.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 ALPHA
   ALPHA is REAL
   On entry, ALPHA specifies the scalar alpha.
   Unchanged on exit.

 AB
   AB is REAL array of DIMENSION ( LDAB, n )
   Before entry, the leading m by n part of the array AB must
   contain the matrix of coefficients.
   Unchanged on exit.
```

```
LDAB
  LDAB is INTEGER
  On entry, LDA specifies the first dimension of AB as declared
  in the calling (sub) program. LDAB must be at least
  max( 1, m ).
  Unchanged on exit.

X
  X is REAL array, dimension
  ( 1 + ( n - 1 )*abs( INCX ) ) when TRANS = 'N' or 'n'
  and at least
  ( 1 + ( m - 1 )*abs( INCX ) ) otherwise.
  Before entry, the incremented array X must contain the
  vector x.
  Unchanged on exit.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of
  X. INCX must not be zero.
  Unchanged on exit.

BETA
  BETA is REAL
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.
  Unchanged on exit.

Y
  Y is REAL array, dimension
  ( 1 + ( m - 1 )*abs( INCY ) ) when TRANS = 'N' or 'n'
  and at least
  ( 1 + ( n - 1 )*abs( INCY ) ) otherwise.
  Before entry with BETA non-zero, the incremented array Y
  must contain the vector y. On exit, Y is overwritten by the
  updated vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of
  Y. INCY must not be zero.
  Unchanged on exit.
```

———-

45.12.3 fortran code

— sla_gbamv.f —

```
*  =====================================================================
      SUBROUTINE SLA_GBAMV( TRANS, M, N, KL, KU, ALPHA, AB, LDAB, X,
     $                      INCX, BETA, Y, INCY )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      REAL               ALPHA, BETA
      INTEGER            INCX, INCY, LDAB, M, N, KL, KU, TRANS
*     ..
*     .. Array Arguments ..
      REAL               AB( LDAB, * ), X( * ), Y( * )
*     ..
*
*  =====================================================================
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            SYMB_ZERO
      REAL               TEMP, SAFE1
      INTEGER            I, INFO, IY, J, JX, KX, KY, LENX, LENY, KD, KE
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, SLAMCH
      REAL               SLAMCH
*     ..
*     .. External Functions ..
      EXTERNAL           ILATRANS
      INTEGER            ILATRANS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, ABS, SIGN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF     ( .NOT.( ( TRANS.EQ.ILATRANS( 'N' ) )
     $           .OR. ( TRANS.EQ.ILATRANS( 'T' ) )
```

```
     $           .OR. ( TRANS.EQ.ILATRANS( 'C' ) ) ) ) THEN
         INFO = 1
      ELSE IF( M.LT.0 )THEN
         INFO = 2
      ELSE IF( N.LT.0 )THEN
         INFO = 3
      ELSE IF( KL.LT.0 .OR. KL.GT.M-1 ) THEN
         INFO = 4
      ELSE IF( KU.LT.0 .OR. KU.GT.N-1 ) THEN
         INFO = 5
      ELSE IF( LDAB.LT.KL+KU+1 )THEN
         INFO = 6
      ELSE IF( INCX.EQ.0 )THEN
         INFO = 8
      ELSE IF( INCY.EQ.0 )THEN
         INFO = 11
      END IF
      IF( INFO.NE.0 )THEN
         CALL XERBLA( 'SLA_GBAMV ', INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( ( M.EQ.0 ).OR.( N.EQ.0 ).OR.
     $    ( ( ALPHA.EQ.ZERO ).AND.( BETA.EQ.ONE ) ) )
     $   RETURN
*
*     Set  LENX  and  LENY, the lengths of the vectors x and y, and set
*     up the start points in  X  and  Y.
*
      IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
         LENX = N
         LENY = M
      ELSE
         LENX = M
         LENY = N
      END IF
      IF( INCX.GT.0 )THEN
         KX = 1
      ELSE
         KX = 1 - ( LENX - 1 )*INCX
      END IF
      IF( INCY.GT.0 )THEN
         KY = 1
      ELSE
         KY = 1 - ( LENY - 1 )*INCY
      END IF
*
*     Set SAFE1 essentially to be the underflow threshold times the
```

```
*     number of additions in each row.
*
      SAFE1 = SLAMCH( 'Safe minimum' )
      SAFE1 = (N+1)*SAFE1
*
*     Form  y := alpha*abs(A)*abs(x) + beta*abs(y).
*
*     The O(M*N) SYMB_ZERO tests could be replaced by O(N) queries to
*     the inexact flag.  Still doesn't help change the iteration order
*     to per-column.
*
      KD = KU + 1
      KE = KL + 1
      IY = KY
      IF ( INCX.EQ.1 ) THEN
         IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
            DO I = 1, LENY
               IF ( BETA .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
                  Y( IY ) = 0.0
               ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
               ELSE
                  SYMB_ZERO = .FALSE.
                  Y( IY ) = BETA * ABS( Y( IY ) )
               END IF
               IF ( ALPHA .NE. ZERO ) THEN
                  DO J = MAX( I-KL, 1 ), MIN( I+KU, LENX )
                     TEMP = ABS( AB( KD+I-J, J ) )
                     SYMB_ZERO = SYMB_ZERO .AND.
     $                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                     Y( IY ) = Y( IY ) + ALPHA*ABS( X( J ) )*TEMP
                  END DO
               END IF

               IF ( .NOT.SYMB_ZERO )
     $              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )
               IY = IY + INCY
            END DO
         ELSE
            DO I = 1, LENY
               IF ( BETA .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
                  Y( IY ) = 0.0
               ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
               ELSE
                  SYMB_ZERO = .FALSE.
                  Y( IY ) = BETA * ABS( Y( IY ) )
```

```
         END IF
         IF ( ALPHA .NE. ZERO ) THEN
            DO J = MAX( I-KL, 1 ), MIN( I+KU, LENX )
               TEMP = ABS( AB( KE-I+J, I ) )
               SYMB_ZERO = SYMB_ZERO .AND.
$                   ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

               Y( IY ) = Y( IY ) + ALPHA*ABS( X( J ) )*TEMP
            END DO
         END IF

         IF ( .NOT.SYMB_ZERO )
$              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )
         IY = IY + INCY
      END DO
   END IF
 ELSE
   IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
      DO I = 1, LENY
         IF ( BETA .EQ. ZERO ) THEN
            SYMB_ZERO = .TRUE.
            Y( IY ) = 0.0
         ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
            SYMB_ZERO = .TRUE.
         ELSE
            SYMB_ZERO = .FALSE.
            Y( IY ) = BETA * ABS( Y( IY ) )
         END IF
         IF ( ALPHA .NE. ZERO ) THEN
            JX = KX
            DO J = MAX( I-KL, 1 ), MIN( I+KU, LENX )
               TEMP = ABS( AB( KD+I-J, J ) )
               SYMB_ZERO = SYMB_ZERO .AND.
$                   ( X( JX ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

               Y( IY ) = Y( IY ) + ALPHA*ABS( X( JX ) )*TEMP
               JX = JX + INCX
            END DO
         END IF

         IF ( .NOT.SYMB_ZERO )
$          Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

         IY = IY + INCY
      END DO
   ELSE
      DO I = 1, LENY
         IF ( BETA .EQ. ZERO ) THEN
            SYMB_ZERO = .TRUE.
            Y( IY ) = 0.0
```

```
               ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
               ELSE
                  SYMB_ZERO = .FALSE.
                  Y( IY ) = BETA * ABS( Y( IY ) )
               END IF
               IF ( ALPHA .NE. ZERO ) THEN
                  JX = KX
                  DO J = MAX( I-KL, 1 ), MIN( I+KU, LENX )
                     TEMP = ABS( AB( KE-I+J, I ) )
                     SYMB_ZERO = SYMB_ZERO .AND.
     $                    ( X( JX ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                     Y( IY ) = Y( IY ) + ALPHA*ABS( X( JX ) )*TEMP
                     JX = JX + INCX
                  END DO
               END IF

               IF ( .NOT.SYMB_ZERO )
     $           Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

               IY = IY + INCY
            END DO
         END IF

      END IF
*
      RETURN
*
*     End of SLA_GBAMV
*
      END
```

———-

45.12.4 lisp code

— **sla_gbamv.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun sla_gbamv (trans m n kl ku alpha ab ldab x incx beta y incy)
    (declare (type (array single-float (*)) y x ab)
             (type (single-float) beta alpha)
             (type (f2cl-lib:integer4) incy incx ldab ku kl n m trans))
    (f2cl-lib:with-multi-array-data
```

```
  ((ab single-float ab-%data% ab-%offset%)
   (x single-float x-%data% x-%offset%)
   (y single-float y-%data% y-%offset%))
(prog ((i 0) (info 0) (iy 0) (j 0) (jx 0) (kx 0) (ky 0) (lenx 0) (leny 0)
       (kd 0) (ke 0) (temp 0.0) (safe1 0.0) (symb_zero nil))
  (declare
   (type (f2cl-lib:integer4) i info iy j jx kx ky lenx leny kd ke)
   (type (single-float) temp safe1)
   (type f2cl-lib:logical symb_zero))
  (setf info 0)
  (cond
   ((not
     (or (= trans (ilatrans "N")) (= trans (ilatrans "T"))
         (= trans (ilatrans "C"))))
    (setf info 1))
   ((< m 0) (setf info 2)) ((< n 0) (setf info 3))
   ((or (< kl 0) (> kl (f2cl-lib:int-add m (f2cl-lib:int-sub 1))))
    (setf info 4))
   ((or (< ku 0) (> ku (f2cl-lib:int-add n (f2cl-lib:int-sub 1))))
    (setf info 5))
   ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info 6))
   ((= incx 0) (setf info 8)) ((= incy 0) (setf info 11)))
  (cond
   ((/= info 0)
    (multiple-value-bind (var-0 var-1)
        (xerbla "SLA_GBAMV " info)
      (declare (ignore var-0))
      (when var-1 (setf info var-1)))
    (go end_label)))
  (if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
      (go end_label))
  (cond ((= trans (ilatrans "N")) (setf lenx n) (setf leny m))
        (t (setf lenx m) (setf leny n)))
  (cond ((> incx 0) (setf kx 1))
        (t
         (setf kx
                 (f2cl-lib:int-sub 1
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-sub lenx 1) incx)))))
  (cond ((> incy 0) (setf ky 1))
        (t
         (setf ky
                 (f2cl-lib:int-sub 1
                                   (f2cl-lib:int-mul
                                    (f2cl-lib:int-sub leny 1) incy)))))
  (setf safe1 (slamch "Safe minimum"))
  (setf safe1 (* (f2cl-lib:int-add n 1) safe1))
  (setf kd (f2cl-lib:int-add ku 1))
  (setf ke (f2cl-lib:int-add kl 1))
  (setf iy ky)
```

```
(cond
 ((= incx 1)
  (cond
   ((= trans (ilatrans "N"))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i leny) nil)
      (tagbody
        (cond
         ((= beta zero) (setf symb_zero f2cl-lib:%true%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) 0.0))
         ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
          (setf symb_zero f2cl-lib:%true%))
         (t (setf symb_zero f2cl-lib:%false%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (* beta
                     (abs
                      (f2cl-lib:fref y-%data% (iy) ((1 *))
                                     y-%offset%))))))
        (cond
         ((/= alpha zero)
          (f2cl-lib:fdo (j
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add i
                                                 (f2cl-lib:int-sub
                                                  kl)))
                          (the f2cl-lib:integer4 1))
                         (f2cl-lib:int-add j 1))
                        ((> j
                            (min
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add i ku))
                             (the f2cl-lib:integer4 lenx)))
                         nil)
            (tagbody
              (setf temp
                      (abs
                       (f2cl-lib:fref ab-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add kd i) j)
                                       j)
                                      ((1 ldab) (1 *)) ab-%offset%)))
              (setf symb_zero
                      (and symb_zero
                           (or
                            (=
                             (f2cl-lib:fref x-%data% (j) ((1 *))
                                            x-%offset%)
                             zero)
                            (= temp zero))))
```

```
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (+
                       (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (* alpha
                          (abs
                           (f2cl-lib:fref x-%data% (j) ((1 *))
                                          x-%offset%))
                          temp)))
             label100001))))
        (if (not symb_zero)
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (f2cl-lib:sign safe1
                                      (f2cl-lib:fref y-%data% (iy)
                                                     ((1 *))
                                                     y-%offset%)))))
        (setf iy (f2cl-lib:int-add iy incy))
       label100000)))
    (t
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i leny) nil)
       (tagbody
         (cond
          ((= beta zero) (setf symb_zero f2cl-lib:%true%)
           (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) 0.0))
          ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
           (setf symb_zero f2cl-lib:%true%))
          (t (setf symb_zero f2cl-lib:%false%)
           (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                   (* beta
                      (abs
                       (f2cl-lib:fref y-%data% (iy) ((1 *))
                                      y-%offset%))))))
         (cond
          ((/= alpha zero)
           (f2cl-lib:fdo (j
                          (max
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add i
                                                  (f2cl-lib:int-sub
                                                   kl)))
                           (the f2cl-lib:integer4 1))
                          (f2cl-lib:int-add j 1))
                         ((> j
                             (min
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add i ku))
                              (the f2cl-lib:integer4 lenx)))
                          nil)
               (tagbody
```

```
                (setf temp
                        (abs
                         (f2cl-lib:fref ab-%data%
                                        ((f2cl-lib:int-add
                                          (f2cl-lib:int-sub ke i) j)
                                         i)
                                        ((1 ldab) (1 *)) ab-%offset%)))
                (setf symb_zero
                        (and symb_zero
                             (or
                              (=
                               (f2cl-lib:fref x-%data% (j) ((1 *))
                                              x-%offset%)
                               zero)
                              (= temp zero))))
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (* alpha
                            (abs
                             (f2cl-lib:fref x-%data% (j) ((1 *))
                                            x-%offset%))
                            temp)))
             label100003))))
        (if (not symb_zero)
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (f2cl-lib:sign safe1
                                      (f2cl-lib:fref y-%data% (iy)
                                                     ((1 *))
                                                     y-%offset%)))))
        (setf iy (f2cl-lib:int-add iy incy))
       label100002)))))
(t
 (cond
  ((= trans (ilatrans "N"))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i leny) nil)
     (tagbody
       (cond
        ((= beta zero) (setf symb_zero f2cl-lib:%true%)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) 0.0))
        ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
         (setf symb_zero f2cl-lib:%true%))
        (t (setf symb_zero f2cl-lib:%false%)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (* beta
                    (abs
                     (f2cl-lib:fref y-%data% (iy) ((1 *))
                                    y-%offset%))))))
```

```
(cond
 ((/= alpha zero) (setf jx kx)
  (f2cl-lib:fdo (j
                 (max
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add i
                                         (f2cl-lib:int-sub
                                          kl)))
                  (the f2cl-lib:integer4 1))
                 (f2cl-lib:int-add j 1))
                ((> j
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add i ku))
                     (the f2cl-lib:integer4 lenx)))
                 nil)
    (tagbody
      (setf temp
              (abs
               (f2cl-lib:fref ab-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add kd i) j)
                               j)
                              ((1 ldab) (1 *)) ab-%offset%)))
      (setf symb_zero
              (and symb_zero
                   (or
                    (=
                     (f2cl-lib:fref x-%data% (jx) ((1 *))
                                    x-%offset%)
                     zero)
                    (= temp zero))))
      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
              (+
               (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
               (* alpha
                  (abs
                   (f2cl-lib:fref x-%data% (jx) ((1 *))
                                  x-%offset%))
                  temp)))
      (setf jx (f2cl-lib:int-add jx incx))
     label100005))))
(if (not symb_zero)
    (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
            (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
               (f2cl-lib:sign safe1
                              (f2cl-lib:fref y-%data% (iy)
                                             ((1 *))
                                             y-%offset%)))))
(setf iy (f2cl-lib:int-add iy incy))
```

```
    label100004)))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i leny) nil)
     (tagbody
       (cond
        ((= beta zero) (setf symb_zero f2cl-lib:%true%)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) 0.0))
        ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
         (setf symb_zero f2cl-lib:%true%))
        (t (setf symb_zero f2cl-lib:%false%)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (* beta
                    (abs
                     (f2cl-lib:fref y-%data% (iy) ((1 *))
                                    y-%offset%))))))
       (cond
        ((/= alpha zero) (setf jx kx)
         (f2cl-lib:fdo (j
                        (max
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add i
                                                (f2cl-lib:int-sub
                                                 kl)))
                         (the f2cl-lib:integer4 1))
                        (f2cl-lib:int-add j 1))
                       ((> j
                           (min
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add i ku))
                            (the f2cl-lib:integer4 lenx)))
                        nil)
           (tagbody
             (setf temp
                     (abs
                      (f2cl-lib:fref ab-%data%
                                     ((f2cl-lib:int-add
                                       (f2cl-lib:int-sub ke i) j)
                                      i)
                                     ((1 ldab) (1 *)) ab-%offset%)))
             (setf symb_zero
                     (and symb_zero
                          (or
                           (=
                            (f2cl-lib:fref x-%data% (jx) ((1 *))
                                           x-%offset%)
                            zero)
                           (= temp zero))))
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (+
```

```
                              (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                              (* alpha
                                 (abs
                                  (f2cl-lib:fref x-%data% (jx) ((1 *))
                                                 x-%offset%))
                                 temp)))
                       (setf jx (f2cl-lib:int-add jx incx))
                      label100007))))
                 (if (not symb_zero)
                     (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                             (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                                (f2cl-lib:sign safe1
                                               (f2cl-lib:fref y-%data% (iy)
                                                              ((1 *))
                                                              y-%offset%)))))
                 (setf iy (f2cl-lib:int-add iy incy))
                label100006))))))
        (go end_label)
       end_label
        (return
         (values nil nil nil nil nil nil nil nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sla_gbamv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (single-float)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
                                              nil nil nil nil nil)
                                            :calls
                                            '(fortran-to-lisp::ilatrans))))
```

45.13 sla_gbrcond

45.13.1 Axiom unit tests

— slagbrcond.input —

```
)set break resume
)sys rm -f slagbrcond.output
)spool slagbrcond.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

45.13.2 Axiom help page

— sla_gbrcond.help —

```
====================================================================
sla_gbrcond examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       real function sla_gbrcond (TRANS, N, KL, KU, AB, LDAB, AFB, LDAFB,
           IPIV, CMODE, C, INFO, WORK, IWORK)
           SLA_GBRCOND estimates the Skeel condition number for a general
           banded matrix.

Function/Subroutine Documentation
   real function sla_gbrcond (character TRANS, integer N, integer KL, integer
       KU, real, dimension( ldab, * ) AB, integer LDAB, real, dimension(
       ldafb, * ) AFB, integer LDAFB, integer, dimension( * ) IPIV, integer
       CMODE, real, dimension( * ) C, integer INFO, real, dimension( * ) WORK,
       integer, dimension( * ) IWORK)
       SLA_GBRCOND estimates the Skeel condition number for a general banded
       matrix.
```

```
Purpose:
   SLA_GBRCOND Estimates the Skeel condition number of  op(A) * op2(C)
   where op2 is determined by CMODE as follows
   CMODE =  1    op2(C) = C
   CMODE =  0    op2(C) = I
   CMODE = -1    op2(C) = inv(C)
   The Skeel condition number  cond(A) = norminf( |inv(A)||A| )
   is computed by computing scaling factors R such that
   diag(R)*A*op2(C) is row equilibrated and computing the standard
   infinity-norm condition number.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate Transpose = Transpose)

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 AB
   AB is REAL array, dimension (LDAB,N)
   On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
   The j-th column of A is stored in the j-th column of the
   array AB as follows:
   AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

 LDAB
   LDAB is INTEGER
   The leading dimension of the array AB.  LDAB >= KL+KU+1.

 AFB
   AFB is REAL array, dimension (LDAFB,N)
   Details of the LU factorization of the band matrix A, as
   computed by SGBTRF.  U is stored as an upper triangular
   band matrix with KL+KU superdiagonals in rows 1 to KL+KU+1,
   and the multipliers used during the factorization are stored
   in rows KL+KU+2 to 2*KL+KU+1.
```

```
LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from the factorization A = P*L*U
  as computed by SGBTRF; row i of the matrix was interchanged
  with row IPIV(i).

CMODE
  CMODE is INTEGER
  Determines op2(C) in the formula op(A) * op2(C) as follows:
  CMODE =  1    op2(C) = C
  CMODE =  0    op2(C) = I
  CMODE = -1    op2(C) = inv(C)

C
  C is REAL array, dimension (N)
  The vector C in the formula op(A) * op2(C).

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  i > 0:  The ith argument is invalid.

WORK
  WORK is REAL array, dimension (5*N).
  Workspace.

IWORK
  IWORK is INTEGER array, dimension (N).
  Workspace.
```

———-

45.13.3 fortran code

— **sla_gbrcond.f** —

```
*  =====================================================================
      REAL FUNCTION SLA_GBRCOND( TRANS, N, KL, KU, AB, LDAB, AFB, LDAFB,
     $                           IPIV, CMODE, C, INFO, WORK, IWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
```

```
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            N, LDAB, LDAFB, INFO, KL, KU, CMODE
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * ), IPIV( * )
      REAL               AB( LDAB, * ), AFB( LDAFB, * ), WORK( * ),
     $                   C( * )
*    ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            NOTRANS
      INTEGER            KASE, I, J, KD, KE
      REAL               AINVNM, TMP
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLACN2, SGBTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
      SLA_GBRCOND = 0.0
*
      INFO = 0
      NOTRANS = LSAME( TRANS, 'N' )
      IF ( .NOT. NOTRANS .AND. .NOT. LSAME(TRANS, 'T')
     $     .AND. .NOT. LSAME(TRANS, 'C') ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 .OR. KL.GT.N-1 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 .OR. KU.GT.N-1 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -6
```

```
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SLA_GBRCOND', -INFO )
         RETURN
      END IF
      IF( N.EQ.0 ) THEN
         SLA_GBRCOND = 1.0
         RETURN
      END IF
*
*     Compute the equilibration matrix R such that
*     inv(R)*A*C has unit 1-norm.
*
      KD = KU + 1
      KE = KL + 1
      IF ( NOTRANS ) THEN
         DO I = 1, N
            TMP = 0.0
               IF ( CMODE .EQ. 1 ) THEN
               DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
                  TMP = TMP + ABS( AB( KD+I-J, J ) * C( J ) )
               END DO
               ELSE IF ( CMODE .EQ. 0 ) THEN
                  DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
                     TMP = TMP + ABS( AB( KD+I-J, J ) )
                  END DO
               ELSE
                  DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
                     TMP = TMP + ABS( AB( KD+I-J, J ) / C( J ) )
                  END DO
               END IF
            WORK( 2*N+I ) = TMP
         END DO
      ELSE
         DO I = 1, N
            TMP = 0.0
            IF ( CMODE .EQ. 1 ) THEN
               DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
                  TMP = TMP + ABS( AB( KE-I+J, I ) * C( J ) )
               END DO
            ELSE IF ( CMODE .EQ. 0 ) THEN
               DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
                  TMP = TMP + ABS( AB( KE-I+J, I ) )
               END DO
            ELSE
               DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
                  TMP = TMP + ABS( AB( KE-I+J, I ) / C( J ) )
               END DO
```

```
            END IF
            WORK( 2*N+I ) = TMP
         END DO
      END IF
*
*     Estimate the norm of inv(op(A)).
*
      AINVNM = 0.0

      KASE = 0
   10 CONTINUE
      CALL SLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * WORK( 2*N+I )
            END DO

            IF ( NOTRANS ) THEN
               CALL SGBTRS( 'No transpose', N, KL, KU, 1, AFB, LDAFB,
     $              IPIV, WORK, N, INFO )
            ELSE
               CALL SGBTRS( 'Transpose', N, KL, KU, 1, AFB, LDAFB, IPIV,
     $              WORK, N, INFO )
            END IF
*
*           Multiply by inv(C).
*
            IF ( CMODE .EQ. 1 ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) / C( I )
               END DO
            ELSE IF ( CMODE .EQ. -1 ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF
         ELSE
*
*           Multiply by inv(C**T).
*
            IF ( CMODE .EQ. 1 ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) / C( I )
               END DO
            ELSE IF ( CMODE .EQ. -1 ) THEN
               DO I = 1, N
```

```
                 WORK( I ) = WORK( I ) * C( I )
              END DO
           END IF

           IF ( NOTRANS ) THEN
              CALL SGBTRS( 'Transpose', N, KL, KU, 1, AFB, LDAFB, IPIV,
     $              WORK, N, INFO )
           ELSE
              CALL SGBTRS( 'No transpose', N, KL, KU, 1, AFB, LDAFB,
     $              IPIV, WORK, N, INFO )
           END IF
*
*          Multiply by R.
*
           DO I = 1, N
              WORK( I ) = WORK( I ) * WORK( 2*N+I )
           END DO
        END IF
        GO TO 10
     END IF
*
*    Compute the estimate of the reciprocal condition number.
*
     IF( AINVNM .NE. 0.0 )
     $   SLA_GBRCOND = ( 1.0 / AINVNM )
*
     RETURN
*
     END
```

———-

45.13.4 lisp code

— sla_gbrcond.lisp —

```
(defun sla_gbrcond
       (trans n kl ku ab ldab afb ldafb ipiv cmode c info work iwork)
  (declare (type (array f2cl-lib:integer4 (*)) iwork ipiv)
           (type (array single-float (*)) work c afb ab)
           (type (f2cl-lib:integer4) info cmode ldafb ldab ku kl n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (ab single-float ab-%data% ab-%offset%)
       (afb single-float afb-%data% afb-%offset%)
       (c single-float c-%data% c-%offset%)
       (work single-float work-%data% work-%offset%)
```

```
     (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
     (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
   (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4)) (ainvnm 0.0)
          (tmp 0.0) (kase 0) (i 0) (j 0) (kd 0) (ke 0) (notrans nil)
          (sla_gbrcond 0.0))
     (declare (type f2cl-lib:logical notrans)
              (type (f2cl-lib:integer4) ke kd j i kase)
              (type (single-float) sla_gbrcond tmp ainvnm)
              (type (array f2cl-lib:integer4 (3)) isave))
     (setf sla_gbrcond 0.0)
     (setf info 0)
     (setf notrans
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame trans "N")
               (declare (ignore var-1))
               (when var-0 (setf trans var-0))
               ret-val))
     (cond
      ((and (not notrans)
            (not
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame trans "T")
               (declare (ignore var-1))
               (when var-0 (setf trans var-0))
               ret-val))
            (not
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame trans "C")
               (declare (ignore var-1))
               (when var-0 (setf trans var-0))
               ret-val)))
       (setf info -1))
      ((< n 0) (setf info -2))
      ((or (< kl 0) (> kl (f2cl-lib:int-add n (f2cl-lib:int-sub 1))))
       (setf info -3))
      ((or (< ku 0) (> ku (f2cl-lib:int-add n (f2cl-lib:int-sub 1))))
       (setf info -4))
      ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -6))
      ((< ldafb (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
       (setf info -8)))
     (cond
      ((/= info 0) (xerbla "SLA_GBRCOND" (f2cl-lib:int-sub info))
       (go end_label)))
     (cond ((= n 0) (setf sla_gbrcond 1.0) (go end_label)))
     (setf kd (f2cl-lib:int-add ku 1))
     (setf ke (f2cl-lib:int-add kl 1))
     (cond
      (notrans
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
```

```
(tagbody
  (setf tmp 0.0)
  (cond
   ((= cmode 1)
    (f2cl-lib:fdo (j
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add i (f2cl-lib:int-sub kl)))
                    (the f2cl-lib:integer4 1))
                   (f2cl-lib:int-add j 1))
                  ((> j
                      (min
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add i ku))
                       (the f2cl-lib:integer4 n)))
                   nil)
      (tagbody
        (setf tmp
                (+ tmp
                   (abs
                    (*
                     (f2cl-lib:fref ab-%data%
                                    ((f2cl-lib:int-sub
                                      (f2cl-lib:int-add kd i) j)
                                     j)
                                    ((1 ldab) (1 *)) ab-%offset%)
                     (f2cl-lib:fref c-%data% (j) ((1 *))
                                    c-%offset%)))))
       label100001)))
   ((= cmode 0)
    (f2cl-lib:fdo (j
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add i (f2cl-lib:int-sub kl)))
                    (the f2cl-lib:integer4 1))
                   (f2cl-lib:int-add j 1))
                  ((> j
                      (min
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add i ku))
                       (the f2cl-lib:integer4 n)))
                   nil)
      (tagbody
        (setf tmp
                (+ tmp
                   (abs
                    (f2cl-lib:fref ab-%data%
                                   ((f2cl-lib:int-sub
                                     (f2cl-lib:int-add kd i) j)
                                    j)
```

```
                                        ((1 ldab) (1 *)) ab-%offset%))))
            label100002)))
        (t
         (f2cl-lib:fdo (j
                        (max
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add i (f2cl-lib:int-sub kl)))
                         (the f2cl-lib:integer4 1))
                        (f2cl-lib:int-add j 1))
                       ((> j
                           (min
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add i ku))
                            (the f2cl-lib:integer4 n)))
                        nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (abs
                         (/
                          (f2cl-lib:fref ab-%data%
                                         ((f2cl-lib:int-sub
                                           (f2cl-lib:int-add kd i) j)
                                          j)
                                         ((1 ldab) (1 *)) ab-%offset%)
                          (f2cl-lib:fref c-%data% (j) ((1 *))
                                         c-%offset%)))))
            label100003))))
      (setf (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) i))
                           ((1 *)) work-%offset%)
            tmp)
     label100000)))
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf tmp 0.0)
     (cond
      ((= cmode 1)
       (f2cl-lib:fdo (j
                      (max
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add i (f2cl-lib:int-sub kl)))
                       (the f2cl-lib:integer4 1))
                      (f2cl-lib:int-add j 1))
                     ((> j
                         (min
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add i ku))
```

```
                        (the f2cl-lib:integer4 n)))
                    nil)
      (tagbody
        (setf tmp
                (+ tmp
                   (abs
                    (*
                     (f2cl-lib:fref ab-%data%
                                    ((f2cl-lib:int-add
                                      (f2cl-lib:int-sub ke i) j)
                                     i)
                                    ((1 ldab) (1 *)) ab-%offset%)
                     (f2cl-lib:fref c-%data% (j) ((1 *))
                                    c-%offset%)))))
       label100005)))
    ((= cmode 0)
     (f2cl-lib:fdo (j
                    (max
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add i (f2cl-lib:int-sub kl)))
                     (the f2cl-lib:integer4 1))
                    (f2cl-lib:int-add j 1))
                   ((> j
                       (min
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add i ku))
                        (the f2cl-lib:integer4 n)))
                    nil)
      (tagbody
        (setf tmp
                (+ tmp
                   (abs
                    (f2cl-lib:fref ab-%data%
                                   ((f2cl-lib:int-add
                                     (f2cl-lib:int-sub ke i) j)
                                    i)
                                   ((1 ldab) (1 *)) ab-%offset%))))
       label100006)))
    (t
     (f2cl-lib:fdo (j
                    (max
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add i (f2cl-lib:int-sub kl)))
                     (the f2cl-lib:integer4 1))
                    (f2cl-lib:int-add j 1))
                   ((> j
                       (min
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add i ku))
                        (the f2cl-lib:integer4 n)))
```

```
                        nil)
            (tagbody
              (setf tmp
                      (+ tmp
                         (abs
                          (/
                           (f2cl-lib:fref ab-%data%
                                          ((f2cl-lib:int-add
                                            (f2cl-lib:int-sub ke i) j)
                                           i)
                                          ((1 ldab) (1 *)) ab-%offset%)
                           (f2cl-lib:fref c-%data% (j) ((1 *))
                                          c-%offset%)))))
             label100007))))
       (setf (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) i))
                            ((1 *)) work-%offset%)
             tmp)
     label100004))))
 (setf ainvnm 0.0)
 (setf kase 0)
label10
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slacn2 n
      (f2cl-lib:array-slice work-%data% single-float ((+ n 1)) ((1 *))
                            work-%offset%)
      work iwork ainvnm kase isave)
   (declare (ignore var-1 var-2 var-3 var-6))
   (setf n var-0)
   (setf ainvnm var-4)
   (setf kase var-5))
 (cond
  ((/= kase 0)
   (cond
    ((= kase 2)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                    (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-add
                                     (f2cl-lib:int-mul 2 n) i))
                                   ((1 *)) work-%offset%)))
        label100008))
     (cond
      (notrans
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10)
```

```
          (sgbtrs "No transpose" n kl ku 1 afb ldafb ipiv work n info)
        (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
        (setf n var-1)
        (setf ldafb var-6)
        (setf n var-9)
        (setf info var-10)))
      (t
       (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (sgbtrs "Transpose" n kl ku 1 afb ldafb ipiv work n info)
        (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
        (setf n var-1)
        (setf ldafb var-6)
        (setf n var-9)
        (setf info var-10))))
     (cond
      ((= cmode 1)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (/
                   (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                   (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
         label100009)))
      ((= cmode (f2cl-lib:int-sub 1))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (*
                   (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                   (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
         label100010)))))
    (t
     (cond
      ((= cmode 1)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (/
                   (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                   (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
         label100011)))
      ((= cmode (f2cl-lib:int-sub 1))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
        (tagbody
```

```
                (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                        (*
                         (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                         (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
              label100012))))
          (cond
           (notrans
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                 var-10)
                (sgbtrs "Transpose" n kl ku 1 afb ldafb ipiv work n info)
              (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
              (setf n var-1)
              (setf ldafb var-6)
              (setf n var-9)
              (setf info var-10)))
           (t
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                 var-10)
                (sgbtrs "No transpose" n kl ku 1 afb ldafb ipiv work n info)
              (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
              (setf n var-1)
              (setf ldafb var-6)
              (setf n var-9)
              (setf info var-10))))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                      (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                         (f2cl-lib:fref work-%data%
                                        ((f2cl-lib:int-add
                                          (f2cl-lib:int-mul 2 n) i))
                                        ((1 *)) work-%offset%)))
             label100013))))
        (go label10)))
      (if (/= ainvnm 0.0)
          (setf sla_gbrcond (/ 1.0 ainvnm)))
      (go end_label)
     end_label
      (return
       (values sla_gbrcond trans n nil nil nil nil nil ldafb nil nil nil info
               nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sla_gbrcond
                 fortran-to-lisp::*f2cl-function-info*)
```

```
(fortran-to-lisp::make-f2cl-finfo :arg-types
                                  '((string)
                                    (fortran-to-lisp::integer4)
                                    (fortran-to-lisp::integer4)
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array fortran-to-lisp::integer4
                                     (*))
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (array fortran-to-lisp::integer4
                                     (*)))
                                  :return-values
                                  '(fortran-to-lisp::trans
                                    fortran-to-lisp::n nil nil nil
                                    nil nil fortran-to-lisp::ldafb
                                    nil nil nil fortran-to-lisp::info
                                    nil nil)
                                  :calls
                                  '(fortran-to-lisp::sgbtrs
                                    fortran-to-lisp::slacn2))))
```

45.14 sla_gbrfsx_extended

45.14.1 Axiom unit tests

— slagbrfsxextended.input —

```
)set break resume
)sys rm -f slagbrfsxextended.output
)spool slagbrfsxextended.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

45.14.2 Axiom help page [Demm05]

— sla_gbrfsx_extended.help —

```
==========================================================================
sla_gbrfsx_extended examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sla_gbrfsx_extended (PREC_TYPE, TRANS_TYPE, N, KL, KU, NRHS,
           AB, LDAB, AFB, LDAFB, IPIV, COLEQU, C, B, LDB, Y, LDY, BERR_OUT,
           N_NORMS, ERR_BNDS_NORM, ERR_BNDS_COMP, RES, AYB, DY, Y_TAIL, RCOND,
           ITHRESH, RTHRESH, DZ_UB, IGNORE_CWISE, INFO)
           SLA_GBRFSX_EXTENDED improves the computed solution to a system of
           linear equations for general banded matrices by performing extra-
           precise iterative refinement and provides error bounds and backward
           error estimates for the solution.

Function/Subroutine Documentation
   subroutine sla_gbrfsx_extended (integer PREC_TYPE, integer TRANS_TYPE,
       integer N, integer KL, integer KU, integer NRHS, real, dimension( ldab,
       * ) AB, integer LDAB, real, dimension( ldafb, * ) AFB, integer LDAFB,
       integer, dimension( * ) IPIV, logical COLEQU, real, dimension( * ) C,
       real, dimension( ldb, * ) B, integer LDB, real, dimension( ldy, * ) Y,
       integer LDY, real, dimension(*) BERR_OUT, integer N_NORMS, real,
       dimension( nrhs, * ) ERR_BNDS_NORM, real, dimension( nrhs, * )
       ERR_BNDS_COMP, real, dimension(*) RES, real, dimension(*) AYB, real,
       dimension(*) DY, real, dimension(*) Y_TAIL, real RCOND, integer
       ITHRESH, real RTHRESH, real DZ_UB, logical IGNORE_CWISE, integer INFO)
       SLA_GBRFSX_EXTENDED improves the computed solution to a system of
       linear equations for general banded matrices by performing extra-
       precise iterative refinement and provides error bounds and backward
       error estimates for the solution.

Purpose:
   SLA_GBRFSX_EXTENDED improves the computed solution to a system of
   linear equations by performing extra-precise iterative refinement
   and provides error bounds and backward error estimates for the solution.
   This subroutine is called by SGBRFSX to perform iterative refinement.
   In addition to normwise error bound, the code provides maximum
   componentwise error bound if possible. See comments for ERR_BNDS_NORM
```

```
   and ERR_BNDS_COMP for details of the error bounds. Note that this
   subroutine is only resonsible for setting the second fields of
   ERR_BNDS_NORM and ERR_BNDS_COMP.

Parameters:

 PREC_TYPE
   PREC_TYPE is INTEGER
   Specifies the intermediate precision to be used in refinement.
   The value is defined by ILAPREC(P) where P is a CHARACTER and
   P    = 'S':  Single
        = 'D':  Double
        = 'I':  Indigenous
        = 'X', 'E':  Extra

 TRANS_TYPE
   TRANS_TYPE is INTEGER
   Specifies the transposition operation on A.
   The value is defined by ILATRANS(T) where T is a CHARACTER and
   T    = 'N':  No transpose
        = 'T':  Transpose
        = 'C':  Conjugate transpose

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0

 NRHS
   NRHS is INTEGER
   The number of right-hand-sides, i.e., the number of columns of the
   matrix B.

 AB
   AB is REAL array, dimension (LDAB,N)
   On entry, the N-by-N matrix AB.

 LDAB
   LDAB is INTEGER
   The leading dimension of the array AB.  LDAB >= max(1,N).

 AFB
   AFB is REAL array, dimension (LDAFB,N)
```

```
  The factors L and U from the factorization
  A = P*L*U as computed by SGBTRF.

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AF.  LDAFB >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from the factorization A = P*L*U
  as computed by SGBTRF; row i of the matrix was interchanged
  with row IPIV(i).

COLEQU
  COLEQU is LOGICAL
  If .TRUE. then column equilibration was done to A before calling
  this routine. This is needed to compute the solution and error
  bounds correctly.

C
  C is REAL array, dimension (N)
  The column scale factors for A. If COLEQU = .FALSE., C
  is not accessed. If C is input, each element of C should be a power
  of the radix to ensure a reliable solution and error estimates.
  Scaling by powers of the radix does not cause rounding errors unless
  the result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is REAL array, dimension (LDB,NRHS)
  The right-hand-side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

Y
  Y is REAL array, dimension (LDY,NRHS)
  On entry, the solution matrix X, as computed by SGBTRS.
  On exit, the improved solution matrix Y.

LDY
  LDY is INTEGER
  The leading dimension of the array Y.  LDY >= max(1,N).

BERR_OUT
  BERR_OUT is REAL array, dimension (NRHS)
  On exit, BERR_OUT(j) contains the componentwise relative backward
  error for right-hand-side j from the formula
```

```
  max(i) ( abs(RES(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
  where abs(Z) is the componentwise absolute value of the matrix
  or vector Z. This is computed by SLA_LIN_BERR.

N_NORMS
  N_NORMS is INTEGER
  Determines which error bounds to return (see ERR_BNDS_NORM
  and ERR_BNDS_COMP).
  If N_NORMS >= 1 return normwise error bounds.
  If N_NORMS >= 2 return componentwise error bounds.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
           max_j (abs(XTRUE(j,i) - X(j,i)))
            ------------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.
```

```
  This subroutine is only responsible for setting the second field above.

ERR_BNDS_COMP
  ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                 abs(XTRUE(j,i) - X(j,i))
           max_j ----------------------
                       abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

RES
```

```
  RES is REAL array, dimension (N)
  Workspace to hold the intermediate residual.

AYB
  AYB is REAL array, dimension (N)
  Workspace. This can be the same workspace passed for Y_TAIL.

DY
  DY is REAL array, dimension (N)
  Workspace to hold the intermediate solution.

Y_TAIL
  Y_TAIL is REAL array, dimension (N)
  Workspace to hold the trailing bits of the intermediate solution.

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

ITHRESH
  ITHRESH is INTEGER
  The maximum number of residual computations allowed for
  refinement. The default is 10. For 'aggressive' set to 100 to
  permit convergence using approximate factorizations or
  factorizations other than LU. If the factorization uses a
  technique other than Gaussian elimination, the guarantees in
  ERR_BNDS_NORM and ERR_BNDS_COMP may no longer be trustworthy.

RTHRESH
  RTHRESH is REAL
  Determines when to stop refinement if the error estimate stops
  decreasing. Refinement will stop when the next solution no longer
  satisfies norm(dx_{i+1}) < RTHRESH * norm(dx_i) where norm(Z) is
  the infinity norm of Z. RTHRESH satisfies 0 < RTHRESH <= 1. The
  default value is 0.5. For 'aggressive' set to 0.9 to permit
  convergence on extremely ill-conditioned matrices. See LAWN 165
  for more details.

DZ_UB
  DZ_UB is REAL
  Determines when to start considering componentwise convergence.
  Componentwise convergence is only considered after each component
  of the solution Y is stable, which we definte as the relative
  change in each component being less than DZ_UB. The default value
  is 0.25, requiring the first bit to be stable. See LAWN 165 for
```

```
  more details.

 IGNORE_CWISE
  IGNORE_CWISE is LOGICAL
  If .TRUE. then ignore componentwise convergence. Default value
  is .FALSE..

 INFO
  INFO is INTEGER
  = 0:  Successful exit.
  < 0:  if INFO = -i, the ith argument to SGBTRS had an illegal value
```

———-

45.14.3 fortran code

— **sla_gbrfsx_extended.f** —

```
*  ====================================================================
      SUBROUTINE SLA_GBRFSX_EXTENDED( PREC_TYPE, TRANS_TYPE, N, KL, KU,
     $                                NRHS, AB, LDAB, AFB, LDAFB, IPIV,
     $                                COLEQU, C, B, LDB, Y, LDY,
     $                                BERR_OUT, N_NORMS, ERR_BNDS_NORM,
     $                                ERR_BNDS_COMP, RES, AYB, DY,
     $                                Y_TAIL, RCOND, ITHRESH, RTHRESH,
     $                                DZ_UB, IGNORE_CWISE, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDAB, LDAFB, LDB, LDY, N, KL, KU, NRHS,
     $                   PREC_TYPE, TRANS_TYPE, N_NORMS, ITHRESH
      LOGICAL            COLEQU, IGNORE_CWISE
      REAL               RTHRESH, DZ_UB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   Y( LDY, * ), RES(*), DY(*), Y_TAIL(*)
      REAL               C( * ), AYB(*), RCOND, BERR_OUT(*),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
```

```
*  =====================================================================
*
*     .. Local Scalars ..
      CHARACTER          TRANS
      INTEGER            CNT, I, J, M, X_STATE, Z_STATE, Y_PREC_STATE
      REAL               YK, DYK, YMIN, NORMY, NORMX, NORMDX, DXRAT,
     $                   DZRAT, PREVNORMDX, PREV_DZ_Z, DXRATMAX,
     $                   DZRATMAX, DX_X, DZ_Z, FINAL_DX_X, FINAL_DZ_Z,
     $                   EPS, HUGEVAL, INCR_THRESH
      LOGICAL            INCR_PREC
*     ..
*     .. Parameters ..
      INTEGER            UNSTABLE_STATE, WORKING_STATE, CONV_STATE,
     $                   NOPROG_STATE, BASE_RESIDUAL, EXTRA_RESIDUAL,
     $                   EXTRA_Y
      PARAMETER          ( UNSTABLE_STATE = 0, WORKING_STATE = 1,
     $                   CONV_STATE = 2, NOPROG_STATE = 3 )
      PARAMETER          ( BASE_RESIDUAL = 0, EXTRA_RESIDUAL = 1,
     $                   EXTRA_Y = 2 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SAXPY, SCOPY, SGBTRS, SGBMV, BLAS_SGBMV_X,
     $                   BLAS_SGBMV2_X, SLA_GBAMV, SLA_WWADDW, SLAMCH,
     $                   CHLA_TRANSTYPE, SLA_LIN_BERR
      REAL               SLAMCH
      CHARACTER          CHLA_TRANSTYPE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      IF (INFO.NE.0) RETURN
```

```
      TRANS = CHLA_TRANSTYPE(TRANS_TYPE)
      EPS = SLAMCH( 'Epsilon' )
      HUGEVAL = SLAMCH( 'Overflow' )
*     Force HUGEVAL to Inf
      HUGEVAL = HUGEVAL * HUGEVAL
*     Using HUGEVAL may lead to spurious underflows.
      INCR_THRESH = REAL( N ) * EPS
      M = KL+KU+1

      DO J = 1, NRHS
         Y_PREC_STATE = EXTRA_RESIDUAL
         IF ( Y_PREC_STATE .EQ. EXTRA_Y ) THEN
            DO I = 1, N
               Y_TAIL( I ) = 0.0
            END DO
         END IF

         DXRAT = 0.0
         DXRATMAX = 0.0
         DZRAT = 0.0
         DZRATMAX = 0.0
         FINAL_DX_X = HUGEVAL
         FINAL_DZ_Z = HUGEVAL
         PREVNORMDX = HUGEVAL
         PREV_DZ_Z = HUGEVAL
         DZ_Z = HUGEVAL
         DX_X = HUGEVAL

         X_STATE = WORKING_STATE
         Z_STATE = UNSTABLE_STATE
         INCR_PREC = .FALSE.

         DO CNT = 1, ITHRESH
*
*        Compute residual RES = B_s - op(A_s) * Y,
*            op(A) = A, A**T, or A**H depending on TRANS (and type).
*
            CALL SCOPY( N, B( 1, J ), 1, RES, 1 )
            IF ( Y_PREC_STATE .EQ. BASE_RESIDUAL ) THEN
               CALL SGBMV( TRANS, M, N, KL, KU, -1.0, AB, LDAB,
     $              Y( 1, J ), 1, 1.0, RES, 1 )
            ELSE IF ( Y_PREC_STATE .EQ. EXTRA_RESIDUAL ) THEN
               CALL BLAS_SGBMV_X( TRANS_TYPE, N, N, KL, KU,
     $              -1.0, AB, LDAB, Y( 1, J ), 1, 1.0, RES, 1,
     $              PREC_TYPE )
            ELSE
               CALL BLAS_SGBMV2_X( TRANS_TYPE, N, N, KL, KU, -1.0,
     $              AB, LDAB, Y( 1, J ), Y_TAIL, 1, 1.0, RES, 1,
     $              PREC_TYPE )
            END IF
```

```
!        XXX: RES is no longer needed.
            CALL SCOPY( N, RES, 1, DY, 1 )
            CALL SGBTRS( TRANS, N, KL, KU, 1, AFB, LDAFB, IPIV, DY, N,
     $           INFO )
*
*         Calculate relative changes DX_X, DZ_Z and ratios DXRAT, DZRAT.
*
            NORMX = 0.0
            NORMY = 0.0
            NORMDX = 0.0
            DZ_Z = 0.0
            YMIN = HUGEVAL

            DO I = 1, N
               YK = ABS( Y( I, J ) )
               DYK = ABS( DY( I ) )

               IF ( YK .NE. 0.0 ) THEN
                  DZ_Z = MAX( DZ_Z, DYK / YK )
               ELSE IF ( DYK .NE. 0.0 ) THEN
                  DZ_Z = HUGEVAL
               END IF

               YMIN = MIN( YMIN, YK )

               NORMY = MAX( NORMY, YK )

               IF ( COLEQU ) THEN
                  NORMX = MAX( NORMX, YK * C( I ) )
                  NORMDX = MAX( NORMDX, DYK * C( I ) )
               ELSE
                  NORMX = NORMY
                  NORMDX = MAX( NORMDX, DYK )
               END IF
            END DO

            IF ( NORMX .NE. 0.0 ) THEN
               DX_X = NORMDX / NORMX
            ELSE IF ( NORMDX .EQ. 0.0 ) THEN
               DX_X = 0.0
            ELSE
               DX_X = HUGEVAL
            END IF

            DXRAT = NORMDX / PREVNORMDX
            DZRAT = DZ_Z / PREV_DZ_Z
*
*         Check termination criteria.
*
```

```
      IF ( .NOT.IGNORE_CWISE
     $     .AND. YMIN*RCOND .LT. INCR_THRESH*NORMY
     $     .AND. Y_PREC_STATE .LT. EXTRA_Y )
     $     INCR_PREC = .TRUE.

      IF ( X_STATE .EQ. NOPROG_STATE .AND. DXRAT .LE. RTHRESH )
     $     X_STATE = WORKING_STATE
      IF ( X_STATE .EQ. WORKING_STATE ) THEN
         IF ( DX_X .LE. EPS ) THEN
            X_STATE = CONV_STATE
         ELSE IF ( DXRAT .GT. RTHRESH ) THEN
            IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
               INCR_PREC = .TRUE.
            ELSE
               X_STATE = NOPROG_STATE
            END IF
         ELSE
            IF ( DXRAT .GT. DXRATMAX ) DXRATMAX = DXRAT
         END IF
         IF ( X_STATE .GT. WORKING_STATE ) FINAL_DX_X = DX_X
      END IF

      IF ( Z_STATE .EQ. UNSTABLE_STATE .AND. DZ_Z .LE. DZ_UB )
     $     Z_STATE = WORKING_STATE
      IF ( Z_STATE .EQ. NOPROG_STATE .AND. DZRAT .LE. RTHRESH )
     $     Z_STATE = WORKING_STATE
      IF ( Z_STATE .EQ. WORKING_STATE ) THEN
         IF ( DZ_Z .LE. EPS ) THEN
            Z_STATE = CONV_STATE
         ELSE IF ( DZ_Z .GT. DZ_UB ) THEN
            Z_STATE = UNSTABLE_STATE
            DZRATMAX = 0.0
            FINAL_DZ_Z = HUGEVAL
         ELSE IF ( DZRAT .GT. RTHRESH ) THEN
            IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
               INCR_PREC = .TRUE.
            ELSE
               Z_STATE = NOPROG_STATE
            END IF
         ELSE
            IF ( DZRAT .GT. DZRATMAX ) DZRATMAX = DZRAT
         END IF
         IF ( Z_STATE .GT. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
      END IF
*
*     Exit if both normwise and componentwise stopped working,
*     but if componentwise is unstable, let it go at least two
*     iterations.
*
      IF ( X_STATE.NE.WORKING_STATE ) THEN
```

```
               IF ( IGNORE_CWISE ) GOTO 666
               IF ( Z_STATE.EQ.NOPROG_STATE .OR. Z_STATE.EQ.CONV_STATE )
     $              GOTO 666
               IF ( Z_STATE.EQ.UNSTABLE_STATE .AND. CNT.GT.1 ) GOTO 666
            END IF

            IF ( INCR_PREC ) THEN
               INCR_PREC = .FALSE.
               Y_PREC_STATE = Y_PREC_STATE + 1
               DO I = 1, N
                  Y_TAIL( I ) = 0.0
               END DO
            END IF

            PREVNORMDX = NORMDX
            PREV_DZ_Z = DZ_Z
*
*           Update soluton.
*
            IF (Y_PREC_STATE .LT. EXTRA_Y) THEN
               CALL SAXPY( N, 1.0, DY, 1, Y(1,J), 1 )
            ELSE
               CALL SLA_WWADDW( N, Y(1,J), Y_TAIL, DY )
            END IF

         END DO
*        Target of "IF (Z_STOP .AND. X_STOP)".  Sun's f77 won't EXIT.
 666     CONTINUE
*
*     Set final_* when cnt hits ithresh.
*
         IF ( X_STATE .EQ. WORKING_STATE ) FINAL_DX_X = DX_X
         IF ( Z_STATE .EQ. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
*
*     Compute error bounds.
*
         IF ( N_NORMS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) =
     $           FINAL_DX_X / (1 - DXRATMAX)
         END IF
         IF (N_NORMS .GE. 2) THEN
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) =
     $           FINAL_DZ_Z / (1 - DZRATMAX)
         END IF
*
*     Compute componentwise relative backward error from formula
*         max(i) ( abs(R(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
*     where abs(Z) is the componentwise absolute value of the matrix
*     or vector Z.
*
```

```
*        Compute residual RES = B_s - op(A_s) * Y,
*            op(A) = A, A**T, or A**H depending on TRANS (and type).
*
         CALL SCOPY( N, B( 1, J ), 1, RES, 1 )
         CALL SGBMV(TRANS, N, N, KL, KU, -1.0, AB, LDAB, Y(1,J),
     $        1, 1.0, RES, 1 )

         DO I = 1, N
            AYB( I ) = ABS( B( I, J ) )
         END DO
*
*     Compute abs(op(A_s))*abs(Y) + abs(B_s).
*
        CALL SLA_GBAMV( TRANS_TYPE, N, N, KL, KU, 1.0,
     $        AB, LDAB, Y(1, J), 1, 1.0, AYB, 1 )

         CALL SLA_LIN_BERR( N, N, 1, RES, AYB, BERR_OUT( J ) )
*
*     End of loop for each RHS
*
      END DO
*
      RETURN
      END
```

———-

45.14.4 lisp code

— **sla_gbrfsx_extended.lisp** —

```
(let* ((unstable_state 0)
       (working_state 1)
       (conv_state 2)
       (noprog_state 3)
       (base_residual 0)
       (extra_residual 1)
       (extra_y 2)
       (la_linrx_err_i 2))
  (declare (type (f2cl-lib:integer4 0 0) unstable_state)
           (type (f2cl-lib:integer4 1 1) working_state)
           (type (f2cl-lib:integer4 2 2) conv_state)
           (type (f2cl-lib:integer4 3 3) noprog_state)
           (type (f2cl-lib:integer4 0 0) base_residual)
           (type (f2cl-lib:integer4 1 1) extra_residual)
           (type (f2cl-lib:integer4 2 2) extra_y)
           (type (f2cl-lib:integer4 2 2) la_linrx_err_i)
           (ignorable unstable_state working_state conv_state noprog_state
```

```
          base_residual extra_residual extra_y la_linrx_err_i))
(defun sla_gbrfsx_extended
       (prec_type trans_type n kl ku nrhs ab ldab afb ldafb ipiv colequ c b
        ldb$ y ldy berr_out n_norms err_bnds_norm err_bnds_comp res ayb dy
        y_tail rcond ithresh rthresh dz_ub ignore_cwise info)
  (declare (type (single-float) dz_ub rthresh rcond)
           (type f2cl-lib:logical ignore_cwise colequ)
           (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array single-float (*)) y_tail dy ayb res err_bnds_comp
            err_bnds_norm berr_out y b c afb ab)
           (type (f2cl-lib:integer4) info ithresh n_norms ldy ldb$ ldafb ldab
            nrhs ku kl n trans_type prec_type))
  (f2cl-lib:with-multi-array-data
      ((ab single-float ab-%data% ab-%offset%)
       (afb single-float afb-%data% afb-%offset%)
       (c single-float c-%data% c-%offset%)
       (b single-float b-%data% b-%offset%)
       (y single-float y-%data% y-%offset%)
       (berr_out single-float berr_out-%data% berr_out-%offset%)
       (err_bnds_norm single-float err_bnds_norm-%data%
        err_bnds_norm-%offset%)
       (err_bnds_comp single-float err_bnds_comp-%data%
        err_bnds_comp-%offset%)
       (res single-float res-%data% res-%offset%)
       (ayb single-float ayb-%data% ayb-%offset%)
       (dy single-float dy-%data% dy-%offset%)
       (y_tail single-float y_tail-%data% y_tail-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((incr_prec nil) (yk 0.0) (dyk 0.0) (ymin 0.0) (normy 0.0)
           (normx 0.0) (normdx 0.0) (dxrat 0.0) (dzrat 0.0) (prevnormdx 0.0)
           (prev_dz_z 0.0) (dxratmax 0.0) (dzratmax 0.0) (dx_x 0.0)
           (dz_z 0.0) (final_dx_x 0.0) (final_dz_z 0.0) (eps 0.0)
           (hugeval 0.0) (incr_thresh 0.0) (cnt 0) (i 0) (j 0) (m 0)
           (x_state 0) (z_state 0) (y_prec_state 0)
           (trans
            (make-array '(1) :element-type 'character :initial-element #\ )))
      (declare (type f2cl-lib:logical incr_prec)
               (type (single-float) yk dyk ymin normy normx normdx dxrat
                dzrat prevnormdx prev_dz_z dxratmax dzratmax dx_x dz_z
                final_dx_x final_dz_z eps hugeval incr_thresh)
               (type (f2cl-lib:integer4) cnt i j m x_state z_state
                y_prec_state)
               (type (string 1) trans))
      (if (/= info 0)
          (go end_label))
      (f2cl-lib:f2cl-set-string trans
                                (f2cl-lib:fref-string #'chla_transtype
                                                      (trans_type))
                                (string 1))
      (setf eps (slamch "Epsilon"))
```

```
(setf hugeval (slamch "Overflow"))
(setf hugeval (* hugeval hugeval))
(setf incr_thresh (* (f2cl-lib:freal n) eps))
(setf m (f2cl-lib:int-add kl ku 1))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf y_prec_state extra_residual)
    (cond
     ((= y_prec_state extra_y)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                               y_tail-%offset%)
                0.0)
         label100001))))
    (setf dxrat 0.0)
    (setf dxratmax 0.0)
    (setf dzrat 0.0)
    (setf dzratmax 0.0)
    (setf final_dx_x hugeval)
    (setf final_dz_z hugeval)
    (setf prevnormdx hugeval)
    (setf prev_dz_z hugeval)
    (setf dz_z hugeval)
    (setf dx_x hugeval)
    (setf x_state working_state)
    (setf z_state unstable_state)
    (setf incr_prec f2cl-lib:%false%)
    (f2cl-lib:fdo (cnt 1 (f2cl-lib:int-add cnt 1))
                  ((> cnt ithresh) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (scopy n
             (f2cl-lib:array-slice b-%data% single-float (1 j)
                                   ((1 ldb$) (1 *)) b-%offset%)
             1 res 1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf n var-0)))
        (cond
         ((= y_prec_state base_residual)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10 var-11 var-12)
              (sgbmv trans m n kl ku -1.0 ab ldab
               (f2cl-lib:array-slice y-%data% single-float (1 j)
                                     ((1 ldy) (1 *)) y-%offset%)
               1 1.0 res 1)
            (declare
```

```
     (ignore var-5 var-6 var-8 var-9 var-10 var-11 var-12))
    (when var-0 (setf trans var-0))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf kl var-3))
    (when var-4 (setf ku var-4))
    (when var-7 (setf ldab var-7))))
 ((= y_prec_state extra_residual)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13)
      (blas_sgbmv_x trans_type n n kl ku -1.0 ab ldab
       (f2cl-lib:array-slice y-%data% single-float (1 j)
                             ((1 ldy) (1 *)) y-%offset%)
       1 1.0 res 1 prec_type)
    (declare
     (ignore var-5 var-6 var-8 var-9 var-10 var-11 var-12))
    (when var-0 (setf trans_type var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf kl var-3))
    (when var-4 (setf ku var-4))
    (when var-7 (setf ldab var-7))
    (when var-13 (setf prec_type var-13))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13 var-14)
      (blas_sgbmv2_x trans_type n n kl ku -1.0 ab ldab
       (f2cl-lib:array-slice y-%data% single-float (1 j)
                             ((1 ldy) (1 *)) y-%offset%)
       y_tail 1 1.0 res 1 prec_type)
    (declare
     (ignore var-5 var-6 var-8 var-9 var-10 var-11 var-12
      var-13))
    (when var-0 (setf trans_type var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf kl var-3))
    (when var-4 (setf ku var-4))
    (when var-7 (setf ldab var-7))
    (when var-14 (setf prec_type var-14)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (scopy n res 1 dy 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (sgbtrs trans n kl ku 1 afb ldafb ipiv dy n info)
```

```
  (declare (ignore var-2 var-3 var-4 var-5 var-7 var-8))
  (setf trans var-0)
  (setf n var-1)
  (setf ldafb var-6)
  (setf n var-9)
  (setf info var-10))
(setf normx 0.0)
(setf normy 0.0)
(setf normdx 0.0)
(setf dz_z 0.0)
(setf ymin hugeval)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf yk
            (abs
             (f2cl-lib:fref y-%data% (i j) ((1 ldy) (1 *))
                            y-%offset%)))
    (setf dyk
            (abs
             (f2cl-lib:fref dy-%data% (i) ((1 *))
                            dy-%offset%)))
    (cond ((/= yk 0.0) (setf dz_z (max dz_z (/ dyk yk))))
          ((/= dyk 0.0) (setf dz_z hugeval)))
    (setf ymin (min ymin yk))
    (setf normy (max normy yk))
    (cond
     (colequ
      (setf normx
              (max normx
                   (* yk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%))))
      (setf normdx
              (max normdx
                   (* dyk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%)))))
     (t (setf normx normy) (setf normdx (max normdx dyk))))
   label100003))
(cond ((/= normx 0.0) (setf dx_x (/ normdx normx)))
      ((= normdx 0.0) (setf dx_x 0.0)) (t (setf dx_x hugeval)))
(setf dxrat (/ normdx prevnormdx))
(setf dzrat (/ dz_z prev_dz_z))
(if (and (not ignore_cwise)
         (< (* ymin rcond) (* incr_thresh normy))
         (< y_prec_state extra_y))
    (setf incr_prec f2cl-lib:%true%))
(if (and (= x_state noprog_state) (<= dxrat rthresh))
    (setf x_state working_state))
```

```
(cond
 ((= x_state working_state)
  (cond ((<= dx_x eps) (setf x_state conv_state))
        ((> dxrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf x_state noprog_state))))
        (t
         (if (> dxrat dxratmax)
             (setf dxratmax dxrat))))
  (if (> x_state working_state)
      (setf final_dx_x dx_x))))
(if (and (= z_state unstable_state) (<= dz_z dz_ub))
    (setf z_state working_state))
(if (and (= z_state noprog_state) (<= dzrat rthresh))
    (setf z_state working_state))
(cond
 ((= z_state working_state)
  (cond ((<= dz_z eps) (setf z_state conv_state))
        ((> dz_z dz_ub) (setf z_state unstable_state)
         (setf dzratmax 0.0) (setf final_dz_z hugeval))
        ((> dzrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf z_state noprog_state))))
        (t
         (if (> dzrat dzratmax)
             (setf dzratmax dzrat))))
  (if (> z_state working_state)
      (setf final_dz_z dz_z))))
(cond
 ((/= x_state working_state)
  (if ignore_cwise
      (go label666))
  (if (or (= z_state noprog_state) (= z_state conv_state))
      (go label666))
  (if (and (= z_state unstable_state) (> cnt 1))
      (go label666))))
(cond
 (incr_prec (setf incr_prec f2cl-lib:%false%)
  (setf y_prec_state (f2cl-lib:int-add y_prec_state 1))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                           y_tail-%offset%)
            0.0)
     label100004))))
```

```
        (setf prevnormdx normdx)
        (setf prev_dz_z dz_z)
        (cond
         ((< y_prec_state extra_y)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
              (saxpy n 1.0 dy 1
               (f2cl-lib:array-slice y-%data% single-float (1 j)
                                     ((1 ldy) (1 *)) y-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3 var-4 var-5))
            (when var-0 (setf n var-0))))
         (t
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (sla_wwaddw n
               (f2cl-lib:array-slice y-%data% single-float (1 j)
                                     ((1 ldy) (1 *)) y-%offset%)
               y_tail dy)
            (declare (ignore var-1 var-2 var-3))
            (when var-0 (setf n var-0)))))
      label100002))
  label666
   (if (= x_state working_state)
       (setf final_dx_x dx_x))
   (if (= z_state working_state)
       (setf final_dz_z dz_z))
   (cond
    ((>= n_norms 1)
     (setf (f2cl-lib:fref err_bnds_norm-%data% (j la_linrx_err_i)
                          ((1 nrhs) (1 *)) err_bnds_norm-%offset%)
           (/ final_dx_x (- 1 dxratmax)))))
   (cond
    ((>= n_norms 2)
     (setf (f2cl-lib:fref err_bnds_comp-%data% (j la_linrx_err_i)
                          ((1 nrhs) (1 *)) err_bnds_comp-%offset%)
           (/ final_dz_z (- 1 dzratmax)))))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
       (scopy n
        (f2cl-lib:array-slice b-%data% single-float (1 j)
                              ((1 ldb$) (1 *)) b-%offset%)
        1 res 1)
     (declare (ignore var-1 var-2 var-3 var-4))
     (when var-0 (setf n var-0)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12)
       (sgbmv trans n n kl ku -1.0 ab ldab
        (f2cl-lib:array-slice y-%data% single-float (1 j)
                              ((1 ldy) (1 *)) y-%offset%)
        1 1.0 res 1)
     (declare (ignore var-5 var-6 var-8 var-9 var-10 var-11 var-12))
```

```
              (when var-0 (setf trans var-0))
              (when var-1 (setf n var-1))
              (when var-2 (setf n var-2))
              (when var-3 (setf kl var-3))
              (when var-4 (setf ku var-4))
              (when var-7 (setf ldab var-7)))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref ayb-%data% (i) ((1 *)) ayb-%offset%)
                        (abs
                         (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                        b-%offset%)))
               label100005))
            (sla_gbamv trans_type n n kl ku 1.0 ab ldab
             (f2cl-lib:array-slice y-%data% single-float (1 j) ((1 ldy) (1 *))
                                   y-%offset%)
             1 1.0 ayb 1)
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                (sla_lin_berr n n 1 res ayb
                 (f2cl-lib:array-slice berr_out-%data% single-float (j) ((1 *))
                                       berr_out-%offset%))
              (declare (ignore var-2 var-3 var-4 var-5))
              (when var-0 (setf n var-0))
              (when var-1 (setf n var-1)))
          label100000))
       (go end_label)
      end_label
       (return
        (values prec_type trans_type n kl ku nil nil ldab nil ldafb nil nil
                nil nil nil nil nil nil nil nil nil nil nil nil nil nil nil
                nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sla_gbrfsx_extended
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::integer4
```

```
   (*))
  fortran-to-lisp::logical
  (array single-float (*))
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (single-float)
  (fortran-to-lisp::integer4)
  (single-float) (single-float)
  fortran-to-lisp::logical
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::prec_type
  fortran-to-lisp::trans_type
  fortran-to-lisp::n
  fortran-to-lisp::kl
  fortran-to-lisp::ku nil nil
  fortran-to-lisp::ldab nil
  fortran-to-lisp::ldafb nil nil
  nil nil nil nil nil nil nil nil
  nil nil nil nil nil nil nil nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::sla_gbamv
  fortran-to-lisp::sgbtrs))))
```

———-

45.15 sla_gbrpvgrw

45.15.1 Axiom unit tests

— slagbrpvgrw.input —

```
)set break resume
)sys rm -f slagbrpvgrw.output
```

```
)spool slagbrpvgrw.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

45.15.2 Axiom help page

— sla_gbrpvgrw.help —

```
=====================================================================
sla_gbrpvgrw examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       real function sla_gbrpvgrw (N, KL, KU, NCOLS, AB, LDAB, AFB, LDAFB)
           SLA_GBRPVGRW computes the reciprocal pivot growth factor
           norm(A)/norm(U) for a general banded matrix.

Function/Subroutine Documentation
   real function sla_gbrpvgrw (integer N, integer KL, integer KU, integer
       NCOLS, real, dimension( ldab, * ) AB, integer LDAB, real, dimension(
       ldafb, * ) AFB, integer LDAFB)
       SLA_GBRPVGRW computes the reciprocal pivot growth factor
       norm(A)/norm(U) for a general banded matrix.

Purpose:
   SLA_GBRPVGRW computes the reciprocal pivot growth factor
   norm(A)/norm(U). The "max absolute element" norm is used. If this is
   much less than 1, the stability of the LU factorization of the
   (equilibrated) matrix A could be poor. This also means that the
   solution X, estimated condition numbers, and error bounds could be
   unreliable.

Parameters:

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
```

```
  matrix A.  N >= 0.

KL
  KL is INTEGER
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

NCOLS
  NCOLS is INTEGER
  The number of columns of the matrix A.  NCOLS >= 0.

AB
  AB is REAL array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is REAL array, dimension (LDAFB,N)
  Details of the LU factorization of the band matrix A, as
  computed by SGBTRF.  U is stored as an upper triangular
  band matrix with KL+KU superdiagonals in rows 1 to KL+KU+1,
  and the multipliers used during the factorization are stored
  in rows KL+KU+2 to 2*KL+KU+1.

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL+KU+1.
```

———-

45.15.3 fortran code

— sla_gbrpvgrw.f —

```
*  =====================================================================
      REAL FUNCTION SLA_GBRPVGRW( N, KL, KU, NCOLS, AB, LDAB, AFB,
     $                            LDAFB )
*
```

```
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            N, KL, KU, NCOLS, LDAB, LDAFB
*     ..
*     .. Array Arguments ..
      REAL               AB( LDAB, * ), AFB( LDAFB, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J, KD
      REAL               AMAX, UMAX, RPVGRW
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      RPVGRW = 1.0

      KD = KU + 1
      DO J = 1, NCOLS
         AMAX = 0.0
         UMAX = 0.0
         DO I = MAX( J-KU, 1 ), MIN( J+KL, N )
            AMAX = MAX( ABS( AB( KD+I-J, J)), AMAX )
         END DO
         DO I = MAX( J-KU, 1 ), J
            UMAX = MAX( ABS( AFB( KD+I-J, J ) ), UMAX )
         END DO
         IF ( UMAX /= 0.0 ) THEN
            RPVGRW = MIN( AMAX / UMAX, RPVGRW )
         END IF
      END DO
      SLA_GBRPVGRW = RPVGRW
      END
```

———-

45.15.4 lisp code

— sla_gbrpvgrw.lisp —

```
(defun sla_gbrpvgrw (n kl ku ncols ab ldab afb ldafb)
  (declare (type (array single-float (*)) afb ab)
           (type (f2cl-lib:integer4) ldafb ldab ncols ku kl n))
  (f2cl-lib:with-multi-array-data
      ((ab single-float ab-%data% ab-%offset%)
       (afb single-float afb-%data% afb-%offset%))
    (prog ((amax 0.0) (umax 0.0) (rpvgrw 0.0) (i 0) (j 0) (kd 0)
           (sla_gbrpvgrw 0.0))
      (declare (type (f2cl-lib:integer4) kd j i)
               (type (single-float) sla_gbrpvgrw rpvgrw umax amax))
      (setf rpvgrw 1.0)
      (setf kd (f2cl-lib:int-add ku 1))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j ncols) nil)
        (tagbody
          (setf amax 0.0)
          (setf umax 0.0)
          (f2cl-lib:fdo (i
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                          (the f2cl-lib:integer4 1))
                         (f2cl-lib:int-add i 1))
                        ((> i
                            (min
                             (the f2cl-lib:integer4 (f2cl-lib:int-add j kl))
                             (the f2cl-lib:integer4 n)))
                         nil)
            (tagbody
              (setf amax
                      (max
                       (abs
                        (f2cl-lib:fref ab-%data%
                                       ((f2cl-lib:int-sub
                                         (f2cl-lib:int-add kd i) j)
                                        j)
                                       ((1 ldab) (1 *)) ab-%offset%))
                       amax))
             label100001))
          (f2cl-lib:fdo (i
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                          (the f2cl-lib:integer4 1))
                         (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf umax
                      (max
                       (abs
```

```
                        (f2cl-lib:fref afb-%data%
                                       ((f2cl-lib:int-sub
                                         (f2cl-lib:int-add kd i) j)
                                        j)
                                       ((1 ldafb) (1 *)) afb-%offset%))
                      umax))
            label100002))
          (cond
           ((f2cl-lib:f2cl/ umax (= 0.0))
            (setf rpvgrw (min (/ amax umax) rpvgrw))))
         label100000))
      (setf sla_gbrpvgrw rpvgrw)
     end_label
      (return (values sla_gbrpvgrw nil nil nil nil nil nil nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sla_gbrpvgrw
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil)
                                            :calls 'nil)))
```

———-

45.16 sorgbr

45.16.1 Axiom unit tests

— sorgbr.input —

```
)set break resume
)sys rm -f sorgbr.output
)spool sorgbr.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

45.16.2 Axiom help page

— **sorgbr.help** —

```
=====================================================================
sorgbr examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sorgbr (VECT, M, N, K, A, LDA, TAU, WORK, LWORK, INFO)
           SORGBR

Function/Subroutine Documentation
   subroutine sorgbr (character VECT, integer M, integer N, integer K, real,
       dimension( lda, * ) A, integer LDA, real, dimension( * ) TAU, real,
       dimension( * ) WORK, integer LWORK, integer INFO)
       SORGBR

Purpose:
   SORGBR generates one of the real orthogonal matrices Q or P**T
   determined by SGEBRD when reducing a real matrix A to bidiagonal
   form: A = Q * B * P**T.  Q and P**T are defined as products of
   elementary reflectors H(i) or G(i) respectively.

   If VECT = 'Q', A is assumed to have been an M-by-K matrix, and Q
   is of order M:
   if m >= k, Q = H(1) H(2) . . . H(k) and SORGBR returns the first n
   columns of Q, where m >= n >= k;
   if m < k, Q = H(1) H(2) . . . H(m-1) and SORGBR returns Q as an
   M-by-M matrix.

   If VECT = 'P', A is assumed to have been a K-by-N matrix, and P**T
   is of order N:
   if k < n, P**T = G(k) . . . G(2) G(1) and SORGBR returns the first m
   rows of P**T, where n >= m >= k;
```

```
   if k >= n, P**T = G(n-1) . . . G(2) G(1) and SORGBR returns P**T as
   an N-by-N matrix.

Parameters:

 VECT
   VECT is CHARACTER*1
   Specifies whether the matrix Q or the matrix P**T is
   required, as defined in the transformation applied by SGEBRD:
   = 'Q':  generate Q;
   = 'P':  generate P**T.

 M
   M is INTEGER
   The number of rows of the matrix Q or P**T to be returned.
   M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix Q or P**T to be returned.
   N >= 0.
   If VECT = 'Q', M >= N >= min(M,K);
   if VECT = 'P', N >= M >= min(N,K).

 K
   K is INTEGER
   If VECT = 'Q', the number of columns in the original M-by-K
   matrix reduced by SGEBRD.
   If VECT = 'P', the number of rows in the original K-by-N
   matrix reduced by SGEBRD.
   K >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the vectors which define the elementary reflectors,
   as returned by SGEBRD.
   On exit, the M-by-N matrix Q or P**T.

 LDA
   LDA is INTEGER
   The leading dimension of the array A. LDA >= max(1,M).

 TAU
   TAU is REAL array, dimension
      (min(M,K)) if VECT = 'Q'
      (min(N,K)) if VECT = 'P'
   TAU(i) must contain the scalar factor of the elementary
   reflector H(i) or G(i), which determines Q or P**T, as
   returned by SGEBRD in its array argument TAUQ or TAUP.
```

```
 WORK
   WORK is REAL array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK. LWORK >= max(1,min(M,N)).
   For optimum performance LWORK >= min(M,N)*NB, where NB
   is the optimal blocksize.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

45.16.3 fortran code

— sorgbr.f —

```
*  =====================================================================
      SUBROUTINE SORGBR( VECT, M, N, K, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          VECT
      INTEGER            INFO, K, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
```

```
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, WANTQ
      INTEGER            I, IINFO, J, LWKOPT, MN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           ILAENV, LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SORGLQ, SORGQR, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      WANTQ = LSAME( VECT, 'Q' )
      MN = MIN( M, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.WANTQ .AND. .NOT.LSAME( VECT, 'P' ) ) THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 .OR. ( WANTQ .AND. ( N.GT.M .OR. N.LT.MIN( M,
     $         K ) ) ) .OR. ( .NOT.WANTQ .AND. ( M.GT.N .OR. M.LT.
     $         MIN( N, K ) ) ) ) THEN
         INFO = -3
      ELSE IF( K.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -6
      ELSE IF( LWORK.LT.MAX( 1, MN ) .AND. .NOT.LQUERY ) THEN
         INFO = -9
      END IF
*
      IF( INFO.EQ.0 ) THEN
         WORK( 1 ) = 1
         IF( WANTQ ) THEN
            IF( M.GE.K ) THEN
               CALL SORGQR( M, N, K, A, LDA, TAU, WORK, -1, IINFO )
            ELSE
               IF( M.GT.1 ) THEN
                  CALL SORGQR( M-1, M-1, M-1, A( 2, 2 ), LDA, TAU, WORK,
     $                         -1, IINFO )
               END IF
```

```
            END IF
         ELSE
            IF( K.LT.N ) THEN
               CALL SORGLQ( M, N, K, A, LDA, TAU, WORK, -1, IINFO )
            ELSE
               IF( N.GT.1 ) THEN
                  CALL SORGLQ( N-1, N-1, N-1, A( 2, 2 ), LDA, TAU, WORK,
     $                         -1, IINFO )
               END IF
            END IF
         END IF
         LWKOPT = WORK( 1 )
         LWKOPT = MAX (LWKOPT, MN)
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SORGBR', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         WORK( 1 ) = LWKOPT
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      IF( WANTQ ) THEN
*
*        Form Q, determined by a call to SGEBRD to reduce an m-by-k
*        matrix
*
         IF( M.GE.K ) THEN
*
*           If m >= k, assume m >= n >= k
*
            CALL SORGQR( M, N, K, A, LDA, TAU, WORK, LWORK, IINFO )
*
         ELSE
*
*           If m < k, assume m = n
*
*           Shift the vectors which define the elementary reflectors one
*           column to the right, and set the first row and column of Q
*           to those of the unit matrix
*
            DO 20 J = M, 2, -1
```

```
               A( 1, J ) = ZERO
               DO 10 I = J + 1, M
                  A( I, J ) = A( I, J-1 )
   10          CONTINUE
   20       CONTINUE
            A( 1, 1 ) = ONE
            DO 30 I = 2, M
               A( I, 1 ) = ZERO
   30       CONTINUE
            IF( M.GT.1 ) THEN
*
*              Form Q(2:m,2:m)
*
               CALL SORGQR( M-1, M-1, M-1, A( 2, 2 ), LDA, TAU, WORK,
     $                       LWORK, IINFO )
            END IF
         END IF
      ELSE
*
*        Form P**T, determined by a call to SGEBRD to reduce a k-by-n
*        matrix
*
         IF( K.LT.N ) THEN
*
*           If k < n, assume k <= m <= n
*
            CALL SORGLQ( M, N, K, A, LDA, TAU, WORK, LWORK, IINFO )
*
         ELSE
*
*           If k >= n, assume m = n
*
*           Shift the vectors which define the elementary reflectors one
*           row downward, and set the first row and column of P**T to
*           those of the unit matrix
*
            A( 1, 1 ) = ONE
            DO 40 I = 2, N
               A( I, 1 ) = ZERO
   40       CONTINUE
            DO 60 J = 2, N
               DO 50 I = J - 1, 2, -1
                  A( I, J ) = A( I-1, J )
   50          CONTINUE
               A( 1, J ) = ZERO
   60       CONTINUE
            IF( N.GT.1 ) THEN
*
*              Form P**T(2:n,2:n)
*
```

```
            CALL SORGLQ( N-1, N-1, N-1, A( 2, 2 ), LDA, TAU, WORK,
     $                   LWORK, IINFO )
          END IF
        END IF
      END IF
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of SORGBR
*
      END
```

———-

45.16.4 lisp code

— **sorgbr.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sorgbr (vect m n k a lda tau work lwork info)
    (declare (type (array single-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lwork lda k n m)
             (type (string *) vect))
    (f2cl-lib:with-multi-array-data
        ((vect character vect-%data% vect-%offset%)
         (a single-float a-%data% a-%offset%)
         (tau single-float tau-%data% tau-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((i 0) (iinfo 0) (j 0) (lwkopt 0) (mn 0) (lquery nil) (wantq nil))
        (declare (type (f2cl-lib:integer4) i iinfo j lwkopt mn)
                 (type f2cl-lib:logical lquery wantq))
        (setf info 0)
        (setf wantq
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame vect "Q")
                  (declare (ignore var-1))
                  (when var-0 (setf vect var-0))
                  ret-val))
        (setf mn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond
         ((and (not wantq)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame vect "P")
```

```
            (declare (ignore var-1))
            (when var-0 (setf vect var-0))
            ret-val)))
  (setf info -1))
 ((< m 0) (setf info -2))
 ((or (< n 0)
      (and wantq
           (or (> n m)
               (< n
                  (min (the f2cl-lib:integer4 m)
                       (the f2cl-lib:integer4 k)))))
      (and (not wantq)
           (or (> m n)
               (< m
                  (min (the f2cl-lib:integer4 n)
                       (the f2cl-lib:integer4 k))))))
  (setf info -3))
 ((< k 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -6))
 ((and
   (< lwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 mn)))
   (not lquery))
  (setf info -9)))
(cond
 ((= info 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'single-float))
  (cond
   (wantq
    (cond
     ((>= m k)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (sorgqr m n k a lda tau work -1 iinfo)
        (declare (ignore var-3 var-5 var-6 var-7))
        (when var-0 (setf m var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf k var-2))
        (when var-4 (setf lda var-4))
        (when var-8 (setf iinfo var-8))))
     (t
      (cond
       ((> m 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (sorgqr (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
             (f2cl-lib:int-sub m 1)
             (f2cl-lib:array-slice a-%data% single-float (2 2)
                                   ((1 lda) (1 *)) a-%offset%)
```

```
              lda tau work -1 iinfo)
            (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
            (when var-4 (setf lda var-4))
            (when var-8 (setf iinfo var-8))))))))
    (t
     (cond
      ((< k n)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
           (sorglq m n k a lda tau work -1 iinfo)
         (declare (ignore var-3 var-5 var-6 var-7))
         (when var-0 (setf m var-0))
         (when var-1 (setf n var-1))
         (when var-2 (setf k var-2))
         (when var-4 (setf lda var-4))
         (when var-8 (setf iinfo var-8))))
      (t
       (cond
        ((> n 1)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
             (sorglq (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
              (f2cl-lib:int-sub n 1)
              (f2cl-lib:array-slice a-%data% single-float (2 2)
                                    ((1 lda) (1 *)) a-%offset%)
              lda tau work -1 iinfo)
           (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
           (when var-4 (setf lda var-4))
           (when var-8 (setf iinfo var-8)))))))))
  (setf lwkopt
          (f2cl-lib:int
           (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkopt)
               (the f2cl-lib:integer4 mn)))))
(cond
 ((/= info 0) (xerbla "SORGBR" (f2cl-lib:int-sub info)) (go end_label))
 (lquery
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
  (go end_label)))
(cond
 ((or (= m 0) (= n 0))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'single-float))
  (go end_label)))
(cond
 (wantq
  (cond
   ((>= m k)
```

```
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (sorgqr m n k a lda tau work lwork iinfo)
      (declare (ignore var-3 var-5 var-6))
      (when var-0 (setf m var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf k var-2))
      (when var-4 (setf lda var-4))
      (when var-7 (setf lwork var-7))
      (when var-8 (setf iinfo var-8))))
  (t
    (f2cl-lib:fdo (j m (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                  ((> j 2) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *)) a-%offset%)
                zero)
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                  (f2cl-lib:fref a-%data% (i (f2cl-lib:int-sub j 1))
                                   ((1 lda) (1 *)) a-%offset%))
           label10))
       label20))
    (setf (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)
            one)
    (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data% (i 1) ((1 lda) (1 *)) a-%offset%)
                zero)
       label30))
    (cond
     ((> m 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (sorgqr (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
           (f2cl-lib:int-sub m 1)
           (f2cl-lib:array-slice a-%data% single-float (2 2)
                                  ((1 lda) (1 *)) a-%offset%)
           lda tau work lwork iinfo)
        (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6))
        (when var-4 (setf lda var-4))
        (when var-7 (setf lwork var-7))
        (when var-8 (setf iinfo var-8))))))))
(t
 (cond
  ((< k n)
   (multiple-value-bind
```

```
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (sorglq m n k a lda tau work lwork iinfo)
        (declare (ignore var-3 var-5 var-6))
        (when var-0 (setf m var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf k var-2))
        (when var-4 (setf lda var-4))
        (when var-7 (setf lwork var-7))
        (when var-8 (setf iinfo var-8))))
    (t
     (setf (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)
             one)
     (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref a-%data% (i 1) ((1 lda) (1 *)) a-%offset%)
                 zero)
        label40))
     (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                          (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                         ((> i 2) nil)
           (tagbody
             (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%)
                     (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) j)
                                     ((1 lda) (1 *)) a-%offset%))
            label50))
         (setf (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *)) a-%offset%)
                 zero)
        label60))
     (cond
      ((> n 1)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
           (sorglq (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
            (f2cl-lib:int-sub n 1)
            (f2cl-lib:array-slice a-%data% single-float (2 2)
                                   ((1 lda) (1 *)) a-%offset%)
            lda tau work lwork iinfo)
         (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6))
         (when var-4 (setf lda var-4))
         (when var-7 (setf lwork var-7))
         (when var-8 (setf iinfo var-8)))))))))
 (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
         (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
 (go end_label)
end_label
```

```
        (return (values vect m n k nil lda nil nil lwork info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sorgbr
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::vect
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n
                                              fortran-to-lisp::k nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::lwork
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

Chapter 46

LAPACK - General Band Matrix, Comp. Routines, Complex

46.1 cgbbrd

46.1.1 Axiom unit tests

— **cgbbrd.input** —

```
)set break resume
)sys rm -f cgbbrd.output
)spool cgbbrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

46.1.2 Axiom help page

— **cgbbrd.help** —

```
==========================================================================
```

```
cgbbrd examples
===================================================================

===================================================================
Man Page Details
===================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgbbrd (VECT, M, N, NCC, KL, KU, AB, LDAB, D, E, Q, LDQ, PT,
           LDPT, C, LDC, WORK, RWORK, INFO)
           CGBBRD

Function/Subroutine Documentation
   subroutine cgbbrd (character VECT, integer M, integer N, integer NCC,
       integer KL, integer KU, complex, dimension( ldab, * ) AB, integer LDAB,
       real, dimension( * ) D, real, dimension( * ) E, complex, dimension(
       ldq, * ) Q, integer LDQ, complex, dimension( ldpt, * ) PT, integer
       LDPT, complex, dimension( ldc, * ) C, integer LDC, complex, dimension(
       * ) WORK, real, dimension( * ) RWORK, integer INFO)
       CGBBRD

Purpose:
   CGBBRD reduces a complex general m-by-n band matrix A to real upper
   bidiagonal form B by a unitary transformation: Q**H * A * P = B.

   The routine computes B, and optionally forms Q or P**H, or computes
   Q**H*C for a given matrix C.

Parameters:

 VECT
   VECT is CHARACTER*1
   Specifies whether or not the matrices Q and P**H are to be
   formed.
   = 'N': do not form Q or P**H;
   = 'Q': form Q only;
   = 'P': form P**H only;
   = 'B': form both.

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 NCC
   NCC is INTEGER
```

```
  The number of columns of the matrix C.  NCC >= 0.

KL
  KL is INTEGER
  The number of subdiagonals of the matrix A. KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals of the matrix A. KU >= 0.

AB
  AB is COMPLEX array, dimension (LDAB,N)
  On entry, the m-by-n band matrix A, stored in rows 1 to
  KL+KU+1. The j-th column of A is stored in the j-th column of
  the array AB as follows:
  AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl).
  On exit, A is overwritten by values generated during the reduction.

LDAB
  LDAB is INTEGER
  The leading dimension of the array A. LDAB >= KL+KU+1.

D
  D is REAL array, dimension (min(M,N))
  The diagonal elements of the bidiagonal matrix B.

E
  E is REAL array, dimension (min(M,N)-1)
  The superdiagonal elements of the bidiagonal matrix B.

Q
  Q is COMPLEX array, dimension (LDQ,M)
  If VECT = 'Q' or 'B', the m-by-m unitary matrix Q.
  If VECT = 'N' or 'P', the array Q is not referenced.

LDQ
  LDQ is INTEGER
  The leading dimension of the array Q.
  LDQ >= max(1,M) if VECT = 'Q' or 'B'; LDQ >= 1 otherwise.

PT
  PT is COMPLEX array, dimension (LDPT,N)
  If VECT = 'P' or 'B', the n-by-n unitary matrix P'.
  If VECT = 'N' or 'Q', the array PT is not referenced.

LDPT
  LDPT is INTEGER
  The leading dimension of the array PT.
  LDPT >= max(1,N) if VECT = 'P' or 'B'; LDPT >= 1 otherwise.
```

```
C
  C is COMPLEX array, dimension (LDC,NCC)
  On entry, an m-by-ncc matrix C.
  On exit, C is overwritten by Q**H*C.
  C is not referenced if NCC = 0.

LDC
  LDC is INTEGER
  The leading dimension of the array C.
  LDC >= max(1,M) if NCC > 0; LDC >= 1 if NCC = 0.

WORK
  WORK is COMPLEX array, dimension (max(M,N))

RWORK
  RWORK is REAL array, dimension (max(M,N))

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

46.1.3 fortran code

— cgbbrd.f —

```
*  ==================================================================
      SUBROUTINE CGBBRD( VECT, M, N, NCC, KL, KU, AB, LDAB, D, E, Q,
     $                   LDQ, PT, LDPT, C, LDC, WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          VECT
      INTEGER            INFO, KL, KU, LDAB, LDC, LDPT, LDQ, M, N, NCC
*     ..
*     .. Array Arguments ..
      REAL               D( * ), E( * ), RWORK( * )
      COMPLEX            AB( LDAB, * ), C( LDC, * ), PT( LDPT, * ),
     $                   Q( LDQ, * ), WORK( * )
*     ..
*
```

```
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
      COMPLEX            CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0E+0, 0.0E+0 ),
     $                   CONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            WANTB, WANTC, WANTPT, WANTQ
      INTEGER            I, INCA, J, J1, J2, KB, KB1, KK, KLM, KLU1,
     $                   KUN, L, MINMN, ML, ML0, MU, MU0, NR, NRT
      REAL               ABST, RC
      COMPLEX            RA, RB, RS, T
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLARGV, CLARTG, CLARTV, CLASET, CROT, CSCAL,
     $                   XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, CONJG, MAX, MIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      WANTB = LSAME( VECT, 'B' )
      WANTQ = LSAME( VECT, 'Q' ) .OR. WANTB
      WANTPT = LSAME( VECT, 'P' ) .OR. WANTB
      WANTC = NCC.GT.0
      KLU1 = KL + KU + 1
      INFO = 0
      IF( .NOT.WANTQ .AND. .NOT.WANTPT .AND. .NOT.LSAME( VECT, 'N' ) )
     $     THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NCC.LT.0 ) THEN
         INFO = -4
      ELSE IF( KL.LT.0 ) THEN
         INFO = -5
      ELSE IF( KU.LT.0 ) THEN
         INFO = -6
```

```
      ELSE IF( LDAB.LT.KLU1 ) THEN
         INFO = -8
      ELSE IF( LDQ.LT.1 .OR. WANTQ .AND. LDQ.LT.MAX( 1, M ) ) THEN
         INFO = -12
      ELSE IF( LDPT.LT.1 .OR. WANTPT .AND. LDPT.LT.MAX( 1, N ) ) THEN
         INFO = -14
      ELSE IF( LDC.LT.1 .OR. WANTC .AND. LDC.LT.MAX( 1, M ) ) THEN
         INFO = -16
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGBBRD', -INFO )
         RETURN
      END IF
*
*     Initialize Q and P**H to the unit matrix, if needed
*
      IF( WANTQ )
     $   CALL CLASET( 'Full', M, M, CZERO, CONE, Q, LDQ )
      IF( WANTPT )
     $   CALL CLASET( 'Full', N, N, CZERO, CONE, PT, LDPT )
*
*     Quick return if possible.
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
      MINMN = MIN( M, N )
*
      IF( KL+KU.GT.1 ) THEN
*
*        Reduce to upper bidiagonal form if KU > 0; if KU = 0, reduce
*        first to lower bidiagonal form and then transform to upper
*        bidiagonal
*
         IF( KU.GT.0 ) THEN
            ML0 = 1
            MU0 = 2
         ELSE
            ML0 = 2
            MU0 = 1
         END IF
*
*        Wherever possible, plane rotations are generated and applied in
*        vector operations of length NR over the index set J1:J2:KLU1.
*
*        The complex sines of the plane rotations are stored in WORK,
*        and the real cosines in RWORK.
*
         KLM = MIN( M-1, KL )
         KUN = MIN( N-1, KU )
```

```
      KB = KLM + KUN
      KB1 = KB + 1
      INCA = KB1*LDAB
      NR = 0
      J1 = KLM + 2
      J2 = 1 - KUN
*
      DO 90 I = 1, MINMN
*
*        Reduce i-th column and i-th row of matrix to bidiagonal form
*
         ML = KLM + 1
         MU = KUN + 1
         DO 80 KK = 1, KB
            J1 = J1 + KB
            J2 = J2 + KB
*
*           generate plane rotations to annihilate nonzero elements
*           which have been created below the band
*
            IF( NR.GT.0 )
     $         CALL CLARGV( NR, AB( KLU1, J1-KLM-1 ), INCA,
     $                      WORK( J1 ), KB1, RWORK( J1 ), KB1 )
*
*           apply plane rotations from the left
*
            DO 10 L = 1, KB
               IF( J2-KLM+L-1.GT.N ) THEN
                  NRT = NR - 1
               ELSE
                  NRT = NR
               END IF
               IF( NRT.GT.0 )
     $            CALL CLARTV( NRT, AB( KLU1-L, J1-KLM+L-1 ), INCA,
     $                         AB( KLU1-L+1, J1-KLM+L-1 ), INCA,
     $                         RWORK( J1 ), WORK( J1 ), KB1 )
   10       CONTINUE
*
            IF( ML.GT.ML0 ) THEN
               IF( ML.LE.M-I+1 ) THEN
*
*                 generate plane rotation to annihilate a(i+ml-1,i)
*                 within the band, and apply rotation from the left
*
                  CALL CLARTG( AB( KU+ML-1, I ), AB( KU+ML, I ),
     $                         RWORK( I+ML-1 ), WORK( I+ML-1 ), RA )
                  AB( KU+ML-1, I ) = RA
                  IF( I.LT.N )
     $               CALL CROT( MIN( KU+ML-2, N-I ),
     $                          AB( KU+ML-2, I+1 ), LDAB-1,
```

```
     $                           AB( KU+ML-1, I+1 ), LDAB-1,
     $                           RWORK( I+ML-1 ), WORK( I+ML-1 ) )
                 END IF
                 NR = NR + 1
                 J1 = J1 - KB1
              END IF
*
              IF( WANTQ ) THEN
*
*                accumulate product of plane rotations in Q
*
                 DO 20 J = J1, J2, KB1
                    CALL CROT( M, Q( 1, J-1 ), 1, Q( 1, J ), 1,
     $                         RWORK( J ), CONJG( WORK( J ) ) )
   20            CONTINUE
              END IF
*
              IF( WANTC ) THEN
*
*                apply plane rotations to C
*
                 DO 30 J = J1, J2, KB1
                    CALL CROT( NCC, C( J-1, 1 ), LDC, C( J, 1 ), LDC,
     $                         RWORK( J ), WORK( J ) )
   30            CONTINUE
              END IF
*
              IF( J2+KUN.GT.N ) THEN
*
*                adjust J2 to keep within the bounds of the matrix
*
                 NR = NR - 1
                 J2 = J2 - KB1
              END IF
*
              DO 40 J = J1, J2, KB1
*
*                create nonzero element a(j-1,j+ku) above the band
*                and store it in WORK(n+1:2*n)
*
                 WORK( J+KUN ) = WORK( J )*AB( 1, J+KUN )
                 AB( 1, J+KUN ) = RWORK( J )*AB( 1, J+KUN )
   40         CONTINUE
*
*             generate plane rotations to annihilate nonzero elements
*             which have been generated above the band
*
              IF( NR.GT.0 )
     $           CALL CLARGV( NR, AB( 1, J1+KUN-1 ), INCA,
     $                        WORK( J1+KUN ), KB1, RWORK( J1+KUN ),
```

```
     $                        KB1 )
*
*              apply plane rotations from the right
*
               DO 50 L = 1, KB
                  IF( J2+L-1.GT.M ) THEN
                     NRT = NR - 1
                  ELSE
                     NRT = NR
                  END IF
                  IF( NRT.GT.0 )
     $               CALL CLARTV( NRT, AB( L+1, J1+KUN-1 ), INCA,
     $                            AB( L, J1+KUN ), INCA,
     $                            RWORK( J1+KUN ), WORK( J1+KUN ), KB1 )
   50          CONTINUE
*
               IF( ML.EQ.ML0 .AND. MU.GT.MU0 ) THEN
                  IF( MU.LE.N-I+1 ) THEN
*
*                    generate plane rotation to annihilate a(i,i+mu-1)
*                    within the band, and apply rotation from the right
*
                     CALL CLARTG( AB( KU-MU+3, I+MU-2 ),
     $                            AB( KU-MU+2, I+MU-1 ),
     $                            RWORK( I+MU-1 ), WORK( I+MU-1 ), RA )
                     AB( KU-MU+3, I+MU-2 ) = RA
                     CALL CROT( MIN( KL+MU-2, M-I ),
     $                          AB( KU-MU+4, I+MU-2 ), 1,
     $                          AB( KU-MU+3, I+MU-1 ), 1,
     $                          RWORK( I+MU-1 ), WORK( I+MU-1 ) )
                  END IF
                  NR = NR + 1
                  J1 = J1 - KB1
               END IF
*
               IF( WANTPT ) THEN
*
*                 accumulate product of plane rotations in P**H
*
                  DO 60 J = J1, J2, KB1
                     CALL CROT( N, PT( J+KUN-1, 1 ), LDPT,
     $                          PT( J+KUN, 1 ), LDPT, RWORK( J+KUN ),
     $                          CONJG( WORK( J+KUN ) ) )
   60             CONTINUE
               END IF
*
               IF( J2+KB.GT.M ) THEN
*
*                 adjust J2 to keep within the bounds of the matrix
*
```

```
                 NR = NR - 1
                 J2 = J2 - KB1
              END IF
*
              DO 70 J = J1, J2, KB1
*
*                create nonzero element a(j+kl+ku,j+ku-1) below the
*                band and store it in WORK(1:n)
*
                 WORK( J+KB ) = WORK( J+KUN )*AB( KLU1, J+KUN )
                 AB( KLU1, J+KUN ) = RWORK( J+KUN )*AB( KLU1, J+KUN )
   70         CONTINUE
*
              IF( ML.GT.ML0 ) THEN
                 ML = ML - 1
              ELSE
                 MU = MU - 1
              END IF
   80       CONTINUE
   90    CONTINUE
      END IF
*
      IF( KU.EQ.0 .AND. KL.GT.0 ) THEN
*
*        A has been reduced to complex lower bidiagonal form
*
*        Transform lower bidiagonal form to upper bidiagonal by applying
*        plane rotations from the left, overwriting superdiagonal
*        elements on subdiagonal elements
*
         DO 100 I = 1, MIN( M-1, N )
            CALL CLARTG( AB( 1, I ), AB( 2, I ), RC, RS, RA )
            AB( 1, I ) = RA
            IF( I.LT.N ) THEN
               AB( 2, I ) = RS*AB( 1, I+1 )
               AB( 1, I+1 ) = RC*AB( 1, I+1 )
            END IF
            IF( WANTQ )
     $         CALL CROT( M, Q( 1, I ), 1, Q( 1, I+1 ), 1, RC,
     $                    CONJG( RS ) )
            IF( WANTC )
     $         CALL CROT( NCC, C( I, 1 ), LDC, C( I+1, 1 ), LDC, RC,
     $                    RS )
  100    CONTINUE
      ELSE
*
*        A has been reduced to complex upper bidiagonal form or is
*        diagonal
*
         IF( KU.GT.0 .AND. M.LT.N ) THEN
```

```
*
*          Annihilate a(m,m+1) by applying plane rotations from the
*          right
*
           RB = AB( KU, M+1 )
           DO 110 I = M, 1, -1
              CALL CLARTG( AB( KU+1, I ), RB, RC, RS, RA )
              AB( KU+1, I ) = RA
              IF( I.GT.1 ) THEN
                 RB = -CONJG( RS )*AB( KU, I )
                 AB( KU, I ) = RC*AB( KU, I )
              END IF
              IF( WANTPT )
     $           CALL CROT( N, PT( I, 1 ), LDPT, PT( M+1, 1 ), LDPT,
     $                      RC, CONJG( RS ) )
  110      CONTINUE
        END IF
     END IF
*
*    Make diagonal and superdiagonal elements real, storing them in D
*    and E
*
     T = AB( KU+1, 1 )
     DO 120 I = 1, MINMN
        ABST = ABS( T )
        D( I ) = ABST
        IF( ABST.NE.ZERO ) THEN
           T = T / ABST
        ELSE
           T = CONE
        END IF
        IF( WANTQ )
     $     CALL CSCAL( M, T, Q( 1, I ), 1 )
        IF( WANTC )
     $     CALL CSCAL( NCC, CONJG( T ), C( I, 1 ), LDC )
        IF( I.LT.MINMN ) THEN
           IF( KU.EQ.0 .AND. KL.EQ.0 ) THEN
              E( I ) = ZERO
              T = AB( 1, I+1 )
           ELSE
              IF( KU.EQ.0 ) THEN
                 T = AB( 2, I )*CONJG( T )
              ELSE
                 T = AB( KU, I+1 )*CONJG( T )
              END IF
              ABST = ABS( T )
              E( I ) = ABST
              IF( ABST.NE.ZERO ) THEN
                 T = T / ABST
              ELSE
```

```
                 T = CONE
              END IF
              IF( WANTPT )
     $           CALL CSCAL( N, T, PT( I+1, 1 ), LDPT )
              T = AB( KU+1, I+1 )*CONJG( T )
           END IF
        END IF
  120 CONTINUE
      RETURN
*
*     End of CGBBRD
*
      END
```

———-

46.1.4 lisp code

— **cgbbrd.lisp** —

———-

46.2 cgbcon

46.2.1 Axiom unit tests

— **cgbcon.input** —

```
)set break resume
)sys rm -f cgbcon.output
)spool cgbcon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

46.2.2 Axiom help page

— cgbcon.help —

```
==================================================================
cgbcon examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgbcon (NORM, N, KL, KU, AB, LDAB, IPIV, ANORM, RCOND, WORK,
           RWORK, INFO)
           CGBCON

Function/Subroutine Documentation
   subroutine cgbcon (character NORM, integer N, integer KL, integer KU,
       complex, dimension( ldab, * ) AB, integer LDAB, integer, dimension( * )
       IPIV, real ANORM, real RCOND, complex, dimension( * ) WORK, real,
       dimension( * ) RWORK, integer INFO)
       CGBCON

Purpose:
   CGBCON estimates the reciprocal of the condition number of a complex
   general band matrix A, in either the 1-norm or the infinity-norm,
   using the LU factorization computed by CGBTRF.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as
      RCOND = 1 / ( norm(A) * norm(inv(A)) ).

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies whether the 1-norm condition number or the
   infinity-norm condition number is required:
   = '1' or 'O':  1-norm;
   = 'I':         Infinity-norm.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
```

```
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

AB
  AB is COMPLEX array, dimension (LDAB,N)
  Details of the LU factorization of the band matrix A, as
  computed by CGBTRF.  U is stored as an upper triangular band
  matrix with KL+KU superdiagonals in rows 1 to KL+KU+1, and
  the multipliers used during the factorization are stored in
  rows KL+KU+2 to 2*KL+KU+1.

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= N, row i of the matrix was
  interchanged with row IPIV(i).

ANORM
  ANORM is REAL
  If NORM = '1' or 'O', the 1-norm of the original matrix A.
  If NORM = 'I', the infinity-norm of the original matrix A.

RCOND
  RCOND is REAL
  The reciprocal of the condition number of the matrix A,
  computed as RCOND = 1/(norm(A) * norm(inv(A))).

WORK
  WORK is COMPLEX array, dimension (2*N)

RWORK
  RWORK is REAL array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
```

———-

46.2.3 fortran code

— cgbcon.f —

```
*  =====================================================================
      SUBROUTINE CGBCON( NORM, N, KL, KU, AB, LDAB, IPIV, ANORM, RCOND,
     $                   WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            INFO, KL, KU, LDAB, N
      REAL               ANORM, RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               RWORK( * )
      COMPLEX            AB( LDAB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LNOTI, ONENRM
      CHARACTER          NORMIN
      INTEGER            IX, J, JP, KASE, KASE1, KD, LM
      REAL               AINVNM, SCALE, SMLNUM
      COMPLEX            T, ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ICAMAX
      REAL               SLAMCH
      COMPLEX            CDOTC
      EXTERNAL           LSAME, ICAMAX, SLAMCH, CDOTC
*     ..
*     .. External Subroutines ..
      EXTERNAL           CAXPY, CLACN2, CLATBS, CSRSCL, XERBLA
```

```
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, MIN, REAL
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      ONENRM = NORM.EQ.'1' .OR. LSAME( NORM, 'O' )
      IF( .NOT.ONENRM .AND. .NOT.LSAME( NORM, 'I' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.2*KL+KU+1 ) THEN
         INFO = -6
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGBCON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
      SMLNUM = SLAMCH( 'Safe minimum' )
*
*     Estimate the norm of inv(A).
*
      AINVNM = ZERO
      NORMIN = 'N'
```

```
      IF( ONENRM ) THEN
         KASE1 = 1
      ELSE
         KASE1 = 2
      END IF
      KD = KL + KU + 1
      LNOTI = KL.GT.0
      KASE = 0
   10 CONTINUE
      CALL CLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.KASE1 ) THEN
*
*           Multiply by inv(L).
*
            IF( LNOTI ) THEN
               DO 20 J = 1, N - 1
                  LM = MIN( KL, N-J )
                  JP = IPIV( J )
                  T = WORK( JP )
                  IF( JP.NE.J ) THEN
                     WORK( JP ) = WORK( J )
                     WORK( J ) = T
                  END IF
                  CALL CAXPY( LM, -T, AB( KD+1, J ), 1, WORK( J+1 ), 1 )
   20          CONTINUE
            END IF
*
*           Multiply by inv(U).
*
            CALL CLATBS( 'Upper', 'No transpose', 'Non-unit', NORMIN, N,
     $                   KL+KU, AB, LDAB, WORK, SCALE, RWORK, INFO )
         ELSE
*
*           Multiply by inv(U**H).
*
            CALL CLATBS( 'Upper', 'Conjugate transpose', 'Non-unit',
     $                   NORMIN, N, KL+KU, AB, LDAB, WORK, SCALE, RWORK,
     $                   INFO )
*
*           Multiply by inv(L**H).
*
            IF( LNOTI ) THEN
               DO 30 J = N - 1, 1, -1
                  LM = MIN( KL, N-J )
                  WORK( J ) = WORK( J ) - CDOTC( LM, AB( KD+1, J ), 1,
     $                        WORK( J+1 ), 1 )
                  JP = IPIV( J )
                  IF( JP.NE.J ) THEN
                     T = WORK( JP )
```

```
                    WORK( JP ) = WORK( J )
                    WORK( J ) = T
                 END IF
   30          CONTINUE
            END IF
         END IF
*
*        Divide X by 1/SCALE if doing so will not cause overflow.
*
         NORMIN = 'Y'
         IF( SCALE.NE.ONE ) THEN
            IX = ICAMAX( N, WORK, 1 )
            IF( SCALE.LT.CABS1( WORK( IX ) )*SMLNUM .OR. SCALE.EQ.ZERO )
     $         GO TO 40
            CALL CSRSCL( N, SCALE, WORK, 1 )
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
   40 CONTINUE
      RETURN
*
*     End of CGBCON
*
      END
```

———-

46.2.4 lisp code

— **cgbcon.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun cgbcon (norm n kl ku ab ldab ipiv anorm rcond work rwork info)
    (declare (type (array single-float (*)) rwork)
             (type (single-float) rcond anorm)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) work ab)
             (type (f2cl-lib:integer4) info ldab ku kl n)
             (type (string *) norm))
```

```
(f2cl-lib:with-multi-array-data
    ((norm character norm-%data% norm-%offset%)
     (ab f2cl-lib:complex8 ab-%data% ab-%offset%)
     (work f2cl-lib:complex8 work-%data% work-%offset%)
     (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
     (rwork single-float rwork-%data% rwork-%offset%))
  (labels ((cabs1 (zdum)
             (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
    (declare
     (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
      cabs1))
    (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
           (zdum #C(0.0 0.0)) (ainvnm 0.0) (scale 0.0) (smlnum 0.0) (ix 0)
           (j 0) (jp 0) (kase 0) (kase1 0) (kd 0) (lm 0)
           (normin
            (make-array '(1) :element-type 'character :initial-element
                        #\ ))
           (lnoti nil) (onenrm nil) (t$ #C(0.0 0.0)))
      (declare (type (array f2cl-lib:integer4 (3)) isave)
               (type (f2cl-lib:complex8) t$ zdum)
               (type (single-float) ainvnm scale smlnum)
               (type (f2cl-lib:integer4) ix j jp kase kase1 kd lm)
               (type (string 1) normin)
               (type f2cl-lib:logical lnoti onenrm))
      (setf info 0)
      (setf onenrm
              (or (f2cl-lib:fstring-= norm "1")
                  (multiple-value-bind (ret-val var-0 var-1)
                      (lsame norm "O")
                    (declare (ignore var-1))
                    (when var-0 (setf norm var-0))
                    ret-val)))
      (cond
       ((and (not onenrm)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame norm "I")
                (declare (ignore var-1))
                (when var-0 (setf norm var-0))
                ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2)) ((< kl 0) (setf info -3))
       ((< ku 0) (setf info -4))
       ((< ldab (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
        (setf info -6))
       ((< anorm zero) (setf info -8)))
      (cond
       ((/= info 0) (xerbla "CGBCON" (f2cl-lib:int-sub info))
        (go end_label)))
      (setf rcond zero)
```

```
 (cond ((= n 0) (setf rcond one) (go end_label))
       ((= anorm zero) (go end_label)))
 (setf smlnum (slamch "Safe minimum"))
 (setf ainvnm zero)
 (f2cl-lib:f2cl-set-string normin "N" (string 1))
 (cond (onenrm (setf kase1 1)) (t (setf kase1 2)))
 (setf kd (f2cl-lib:int-add kl ku 1))
 (setf lnoti (> kl 0))
 (setf kase 0)
label10
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
     (clacn2 n
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ n 1))
                            ((1 *)) work-%offset%)
      work ainvnm kase isave)
   (declare (ignore var-1 var-2 var-5))
   (when var-0 (setf n var-0))
   (when var-3 (setf ainvnm var-3))
   (when var-4 (setf kase var-4)))
 (cond
  ((/= kase 0)
   (cond
    ((= kase kase1)
     (cond
      (lnoti
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                      nil)
         (tagbody
           (setf lm
                   (min (the f2cl-lib:integer4 kl)
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-sub n j))))
           (setf jp
                   (f2cl-lib:fref ipiv-%data% (j) ((1 *))
                                  ipiv-%offset%))
           (setf t$
                   (f2cl-lib:fref work-%data% (jp) ((1 *))
                                  work-%offset%))
           (cond
            ((/= jp j)
             (setf (f2cl-lib:fref work-%data% (jp) ((1 *))
                                  work-%offset%)
                     (f2cl-lib:fref work-%data% (j) ((1 *))
                                    work-%offset%))
             (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                                  work-%offset%)
                     t$)))
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
               (caxpy lm (- t$)
```

```
              (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                    ((+ kd 1) j) ((1 ldab) (1 *))
                                    ab-%offset%)
              1
              (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                    ((+ j 1)) ((1 *)) work-%offset%)
              1)
            (declare (ignore var-1 var-2 var-3 var-4 var-5))
            (when var-0 (setf lm var-0)))
        label20))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11)
     (clatbs "Upper" "No transpose" "Non-unit" normin n
      (f2cl-lib:int-add kl ku) ab ldab work scale rwork info)
   (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-10))
   (when var-3 (setf normin var-3))
   (when var-4 (setf n var-4))
   (when var-7 (setf ldab var-7))
   (when var-9 (setf scale var-9))
   (when var-11 (setf info var-11))))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11)
     (clatbs "Upper" "Conjugate transpose" "Non-unit" normin n
      (f2cl-lib:int-add kl ku) ab ldab work scale rwork info)
   (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-10))
   (when var-3 (setf normin var-3))
   (when var-4 (setf n var-4))
   (when var-7 (setf ldab var-7))
   (when var-9 (setf scale var-9))
   (when var-11 (setf info var-11)))
 (cond
  (lnoti
   (f2cl-lib:fdo (j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                  (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                 ((> j 1) nil)
     (tagbody
       (setf lm
               (min (the f2cl-lib:integer4 kl)
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-sub n j))))
       (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
               (-
                (f2cl-lib:fref work-%data% (j) ((1 *))
                               work-%offset%)
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4)
                    (cdotc lm
```

```
                        (f2cl-lib:array-slice ab-%data%
                                              f2cl-lib:complex8
                                              ((+ kd 1) j)
                                              ((1 ldab) (1 *))
                                              ab-%offset%)
                        1
                        (f2cl-lib:array-slice work-%data%
                                              f2cl-lib:complex8
                                              ((+ j 1)) ((1 *))
                                              work-%offset%)
                        1)
                      (declare (ignore var-1 var-2 var-3 var-4))
                      (when var-0 (setf lm var-0))
                      ret-val)))
        (setf jp
                (f2cl-lib:fref ipiv-%data% (j) ((1 *))
                               ipiv-%offset%))
        (cond
         ((/= jp j)
          (setf t$
                  (f2cl-lib:fref work-%data% (jp) ((1 *))
                                 work-%offset%))
          (setf (f2cl-lib:fref work-%data% (jp) ((1 *))
                               work-%offset%)
                  (f2cl-lib:fref work-%data% (j) ((1 *))
                                 work-%offset%))
          (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                               work-%offset%)
                  t$)))
       label30))))))
(f2cl-lib:f2cl-set-string normin "Y" (string 1))
(cond
 ((/= scale one)
  (setf ix
          (multiple-value-bind (ret-val var-0 var-1 var-2)
              (icamax n work 1)
            (declare (ignore var-1 var-2))
            (when var-0 (setf n var-0))
            ret-val))
  (if (or
       (< scale
          (*
           (cabs1
            (f2cl-lib:fref work-%data% (ix) ((1 *)) work-%offset%))
           smlnum))
       (= scale zero))
      (go label40))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (csrscl n scale work 1)
    (declare (ignore var-2 var-3))
```

```
                (when var-0 (setf n var-0))
                (when var-1 (setf scale var-1)))))
            (go label10)))
          (if (/= ainvnm zero)
              (setf rcond (/ (/ one ainvnm) anorm)))
         label40
          (go end_label)
         end_label
          (return
           (values norm n nil nil nil ldab nil nil rcond nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgbcon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (single-float) (single-float)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::norm
                                              fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::ldab nil nil
                                              fortran-to-lisp::rcond nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

46.3 cgbequ

46.3.1 Axiom unit tests

— **cgbequ.input** —

```
)set break resume
)sys rm -f cgbequ.output
)spool cgbequ.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

46.3.2 Axiom help page

— **cgbequ.help** —

```
======================================================================
cgbequ examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgbequ (M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND, AMAX,
           INFO)
           CGBEQU

Function/Subroutine Documentation
   subroutine cgbequ (integer M, integer N, integer KL, integer KU, complex,
       dimension( ldab, * ) AB, integer LDAB, real, dimension( * ) R, real,
       dimension( * ) C, real ROWCND, real COLCND, real AMAX, integer INFO)
       CGBEQU

Purpose:
   CGBEQU computes row and column scalings intended to equilibrate an
   M-by-N band matrix A and reduce its condition number.  R returns the
   row scale factors and C the column scale factors, chosen to try to
```

```
  make the largest element in each row and column of the matrix B with
  elements B(i,j)=R(i)*A(i,j)*C(j) have absolute value 1.

  R(i) and C(j) are restricted to be between SMLNUM = smallest safe
  number and BIGNUM = largest safe number.  Use of these scaling
  factors is not guaranteed to reduce the condition number of A but
  works well in practice.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 AB
   AB is COMPLEX array, dimension (LDAB,N)
   The band matrix A, stored in rows 1 to KL+KU+1.  The j-th
   column of A is stored in the j-th column of the array AB as follows:
        AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl).

 LDAB
   LDAB is INTEGER
   The leading dimension of the array AB.  LDAB >= KL+KU+1.

 R
   R is REAL array, dimension (M)
   If INFO = 0, or INFO > M, R contains the row scale factors for A.

 C
   C is REAL array, dimension (N)
   If INFO = 0, C contains the column scale factors for A.

 ROWCND
   ROWCND is REAL
   If INFO = 0 or INFO > M, ROWCND contains the ratio of the
   smallest R(i) to the largest R(i).  If ROWCND >= 0.1 and
   AMAX is neither too large nor too small, it is not worth
   scaling by R.
```

```
COLCND
  COLCND is REAL
  If INFO = 0, COLCND contains the ratio of the smallest
  C(i) to the largest C(i).  If COLCND >= 0.1, it is not
  worth scaling by C.

AMAX
  AMAX is REAL
  Absolute value of largest matrix element.  If AMAX is very
  close to overflow or very close to underflow, the matrix
  should be scaled.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, and i is
        <= M:  the i-th row of A is exactly zero
        >  M:  the (i-M)-th column of A is exactly zero
```

46.3.3 fortran code

— cgbequ.f —

```
*  =====================================================================
      SUBROUTINE CGBEQU( M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $                   AMAX, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, KL, KU, LDAB, M, N
      REAL               AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      REAL               C( * ), R( * )
      COMPLEX            AB( LDAB, * )
*     ..
*
*  =====================================================================
*
```

```
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, KD
      REAL               BIGNUM, RCMAX, RCMIN, SMLNUM
      COMPLEX            ZDUM
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, MAX, MIN, REAL
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGBEQU', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         ROWCND = ONE
```

```
         COLCND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Get machine constants.
*
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
*
*     Compute row scale factors.
*
      DO 10 I = 1, M
         R( I ) = ZERO
   10 CONTINUE
*
*     Find the maximum element in each row.
*
      KD = KU + 1
      DO 30 J = 1, N
         DO 20 I = MAX( J-KU, 1 ), MIN( J+KL, M )
            R( I ) = MAX( R( I ), CABS1( AB( KD+I-J, J ) ) )
   20    CONTINUE
   30 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 40 I = 1, M
         RCMAX = MAX( RCMAX, R( I ) )
         RCMIN = MIN( RCMIN, R( I ) )
   40 CONTINUE
      AMAX = RCMAX
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 50 I = 1, M
            IF( R( I ).EQ.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   50    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 60 I = 1, M
```

```
            R( I ) = ONE / MIN( MAX( R( I ), SMLNUM ), BIGNUM )
   60    CONTINUE
*
*        Compute ROWCND = min(R(I)) / max(R(I))
*
         ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
*     Compute column scale factors
*
      DO 70 J = 1, N
         C( J ) = ZERO
   70 CONTINUE
*
*     Find the maximum element in each column,
*     assuming the row scaling computed above.
*
      KD = KU + 1
      DO 90 J = 1, N
         DO 80 I = MAX( J-KU, 1 ), MIN( J+KL, M )
            C( J ) = MAX( C( J ), CABS1( AB( KD+I-J, J ) )*R( I ) )
   80    CONTINUE
   90 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 100 J = 1, N
         RCMIN = MIN( RCMIN, C( J ) )
         RCMAX = MAX( RCMAX, C( J ) )
  100 CONTINUE
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 110 J = 1, N
            IF( C( J ).EQ.ZERO ) THEN
               INFO = M + J
               RETURN
            END IF
  110    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 120 J = 1, N
            C( J ) = ONE / MIN( MAX( C( J ), SMLNUM ), BIGNUM )
  120    CONTINUE
```

```
*
*        Compute COLCND = min(C(J)) / max(C(J))
*
         COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
      RETURN
*
*     End of CGBEQU
*
      END
```

———-

46.3.4 lisp code

— cgbequ.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun cgbequ (m n kl ku ab ldab r c rowcnd colcnd amax info)
    (declare (type (single-float) amax colcnd rowcnd)
             (type (array single-float (*)) c r)
             (type (array f2cl-lib:complex8 (*)) ab)
             (type (f2cl-lib:integer4) info ldab ku kl n m))
    (f2cl-lib:with-multi-array-data
        ((ab f2cl-lib:complex8 ab-%data% ab-%offset%)
         (r single-float r-%data% r-%offset%)
         (c single-float c-%data% c-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
          cabs1))
        (prog ((zdum #C(0.0 0.0)) (bignum 0.0) (rcmax 0.0) (rcmin 0.0)
               (smlnum 0.0) (i 0) (j 0) (kd 0))
          (declare (type (f2cl-lib:complex8) zdum)
                   (type (single-float) bignum rcmax rcmin smlnum)
                   (type (f2cl-lib:integer4) i j kd))
          (setf info 0)
          (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
                ((< kl 0) (setf info -3)) ((< ku 0) (setf info -4))
                ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -6)))
          (cond
           ((/= info 0) (xerbla "CGBEQU" (f2cl-lib:int-sub info))
            (go end_label)))
```

```
(cond
 ((or (= m 0) (= n 0)) (setf rowcnd one) (setf colcnd one)
  (setf amax zero) (go end_label)))
(setf smlnum (slamch "S"))
(setf bignum (/ one smlnum))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%) zero)
   label10))
(setf kd (f2cl-lib:int-add ku 1))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                    (the f2cl-lib:integer4 1))
                   (f2cl-lib:int-add i 1))
                  ((> i
                      (min
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add j kl))
                       (the f2cl-lib:integer4 m)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                     (cabs1
                      (f2cl-lib:fref ab-%data%
                                     ((f2cl-lib:int-sub
                                       (f2cl-lib:int-add kd i) j)
                                      j)
                                     ((1 ldab) (1 *)) ab-%offset%))))
       label20))
   label30))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf rcmax
            (max rcmax
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
    (setf rcmin
            (min rcmin
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
   label40))
(setf amax rcmax)
```

```
(cond
 ((= rcmin zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref r (i) ((1 *))) zero) (setf info i)
        (go end_label)))
     label50)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
              (/ one
                 (min
                  (max
                   (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                   smlnum)
                  bignum)))
     label60))
  (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) zero)
   label70))
(setf kd (f2cl-lib:int-add ku 1))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                    (the f2cl-lib:integer4 1))
                   (f2cl-lib:int-add i 1))
                  ((> i
                      (min
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add j kl))
                       (the f2cl-lib:integer4 m)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                     (*
                      (cabs1
                       (f2cl-lib:fref ab-%data%
                                      ((f2cl-lib:int-sub
```

```
                                                  (f2cl-lib:int-add kd i) j)
                                                 j)
                                                ((1 ldab) (1 *)) ab-%offset%))
                                  (f2cl-lib:fref r-%data% (i) ((1 *))
                                                 r-%offset%))))
                 label80))
              label90))
            (setf rcmin bignum)
            (setf rcmax zero)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf rcmin
                        (min rcmin
                             (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
                (setf rcmax
                        (max rcmax
                             (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
               label100))
            (cond
             ((= rcmin zero)
              (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                            ((> j n) nil)
                (tagbody
                  (cond
                   ((= (f2cl-lib:fref c (j) ((1 *))) zero)
                    (setf info (f2cl-lib:int-add m j)) (go end_label)))
                 label110)))
             (t
              (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                            ((> j n) nil)
                (tagbody
                  (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                          (/ one
                             (min
                              (max
                               (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                               smlnum)
                              bignum)))
                 label120))
              (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
            (go end_label)
           end_label
            (return
             (values nil nil nil nil nil nil nil nil rowcnd colcnd amax
                     info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
```

```
(setf (gethash 'fortran-to-lisp::cgbequ
               fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array single-float (*))
                                            (array single-float (*))
                                            (single-float) (single-float)
                                            (single-float)
                                            (fortran-to-lisp::integer4))
                                          :return-values
                                          '(nil nil nil nil nil nil nil nil
                                            fortran-to-lisp::rowcnd
                                            fortran-to-lisp::colcnd
                                            fortran-to-lisp::amax
                                            fortran-to-lisp::info)
                                          :calls 'nil)))
```

———-

46.4 cgbequb

46.4.1 Axiom unit tests

— **cgbequb.input** —

```
)set break resume
)sys rm -f cgbequb.output
)spool cgbequb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

46.4.2 Axiom help page

— cgbequb.help —

```
==================================================================
cgbequb examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgbequb (M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND, AMAX,
           INFO)
           CGBEQUB

Function/Subroutine Documentation
   subroutine cgbequb (integer M, integer N, integer KL, integer KU, complex,
       dimension( ldab, * ) AB, integer LDAB, real, dimension( * ) R, real,
       dimension( * ) C, real ROWCND, real COLCND, real AMAX, integer INFO)
       CGBEQUB

Purpose:

   CGBEQUB computes row and column scalings intended to equilibrate an
   M-by-N matrix A and reduce its condition number.  R returns the row
   scale factors and C the column scale factors, chosen to try to make
   the largest element in each row and column of the matrix B with
   elements B(i,j)=R(i)*A(i,j)*C(j) have an absolute value of at most
   the radix.

   R(i) and C(j) are restricted to be a power of the radix between
   SMLNUM = smallest safe number and BIGNUM = largest safe number.  Use
   of these scaling factors is not guaranteed to reduce the condition
   number of A but works well in practice.

   This routine differs from CGEEQU by restricting the scaling factors
   to a power of the radix.  Baring over- and underflow, scaling by
   these factors introduces no additional rounding errors.  However, the
   scaled entries' magnitured are no longer approximately 1 but lie
   between sqrt(radix) and 1/sqrt(radix).

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.
```

```
N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

KL
  KL is INTEGER
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

AB
  AB is DOUBLE PRECISION array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

LDAB
  LDAB is INTEGER
  The leading dimension of the array A.  LDAB >= max(1,M).

R
  R is REAL array, dimension (M)
  If INFO = 0 or INFO > M, R contains the row scale factors for A.

C
  C is REAL array, dimension (N)
  If INFO = 0,  C contains the column scale factors for A.

ROWCND
  ROWCND is REAL
  If INFO = 0 or INFO > M, ROWCND contains the ratio of the
  smallest R(i) to the largest R(i).  If ROWCND >= 0.1 and
  AMAX is neither too large nor too small, it is not worth
  scaling by R.

COLCND
  COLCND is REAL
  If INFO = 0, COLCND contains the ratio of the smallest
  C(i) to the largest C(i).  If COLCND >= 0.1, it is not
  worth scaling by C.

AMAX
  AMAX is REAL
  Absolute value of largest matrix element.  If AMAX is very
  close to overflow or very close to underflow, the matrix
  should be scaled.
```

```
 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i,  and i is
         <= M:  the i-th row of A is exactly zero
         >  M:  the (i-M)-th column of A is exactly zero
```

———-

46.4.3 fortran code

— cgbequb.f —

```
*  =====================================================================
      SUBROUTINE CGBEQUB( M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $                    AMAX, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, KL, KU, LDAB, M, N
      REAL               AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      REAL               C( * ), R( * )
      COMPLEX            AB( LDAB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, KD
      REAL               BIGNUM, RCMAX, RCMIN, SMLNUM, RADIX,
     $                   LOGRDX
      COMPLEX            ZDUM
*     ..
*     .. External Functions ..
      REAL               SLAMCH
```

```
      EXTERNAL           SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, LOG, REAL, AIMAG
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGBEQUB', -INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         ROWCND = ONE
         COLCND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Get machine constants.  Assume SMLNUM is a power of the radix.
*
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      RADIX = SLAMCH( 'B' )
      LOGRDX = LOG(RADIX)
```

```
*
*     Compute row scale factors.
*
      DO 10 I = 1, M
         R( I ) = ZERO
   10 CONTINUE
*
*     Find the maximum element in each row.
*
      KD = KU + 1
      DO 30 J = 1, N
         DO 20 I = MAX( J-KU, 1 ), MIN( J+KL, M )
            R( I ) = MAX( R( I ), CABS1( AB( KD+I-J, J ) ) )
   20    CONTINUE
   30 CONTINUE
      DO I = 1, M
         IF( R( I ).GT.ZERO ) THEN
            R( I ) = RADIX**INT( LOG( R( I ) ) / LOGRDX )
         END IF
      END DO
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 40 I = 1, M
         RCMAX = MAX( RCMAX, R( I ) )
         RCMIN = MIN( RCMIN, R( I ) )
   40 CONTINUE
      AMAX = RCMAX
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 50 I = 1, M
            IF( R( I ).EQ.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   50    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 60 I = 1, M
            R( I ) = ONE / MIN( MAX( R( I ), SMLNUM ), BIGNUM )
   60    CONTINUE
*
*        Compute ROWCND = min(R(I)) / max(R(I)).
```

```
*
         ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
*     Compute column scale factors.
*
      DO 70 J = 1, N
         C( J ) = ZERO
   70 CONTINUE
*
*     Find the maximum element in each column,
*     assuming the row scaling computed above.
*
      DO 90 J = 1, N
         DO 80 I = MAX( J-KU, 1 ), MIN( J+KL, M )
            C( J ) = MAX( C( J ), CABS1( AB( KD+I-J, J ) )*R( I ) )
   80    CONTINUE
         IF( C( J ).GT.ZERO ) THEN
            C( J ) = RADIX**INT( LOG( C( J ) ) / LOGRDX )
         END IF
   90 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 100 J = 1, N
         RCMIN = MIN( RCMIN, C( J ) )
         RCMAX = MAX( RCMAX, C( J ) )
  100 CONTINUE
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 110 J = 1, N
            IF( C( J ).EQ.ZERO ) THEN
               INFO = M + J
               RETURN
            END IF
  110    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 120 J = 1, N
            C( J ) = ONE / MIN( MAX( C( J ), SMLNUM ), BIGNUM )
  120    CONTINUE
*
*        Compute COLCND = min(C(J)) / max(C(J)).
```

```
*
         COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
      RETURN
*
*     End of CGBEQUB
*
      END
```

———-

46.4.4 lisp code

— **cgbequb.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun cgbequb (m n kl ku ab ldab r c rowcnd colcnd amax info)
    (declare (type (single-float) amax colcnd rowcnd)
             (type (array single-float (*)) c r)
             (type (array f2cl-lib:complex8 (*)) ab)
             (type (f2cl-lib:integer4) info ldab ku kl n m))
    (f2cl-lib:with-multi-array-data
        ((ab f2cl-lib:complex8 ab-%data% ab-%offset%)
         (r single-float r-%data% r-%offset%)
         (c single-float c-%data% c-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
          cabs1))
        (prog ((zdum #C(0.0 0.0)) (bignum 0.0) (rcmax 0.0) (rcmin 0.0)
               (smlnum 0.0) (radix 0.0) (logrdx 0.0) (i 0) (j 0) (kd 0))
          (declare (type (f2cl-lib:complex8) zdum)
                   (type (single-float) bignum rcmax rcmin smlnum radix logrdx)
                   (type (f2cl-lib:integer4) i j kd))
          (setf info 0)
          (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
                ((< kl 0) (setf info -3)) ((< ku 0) (setf info -4))
                ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -6)))
          (cond
           ((/= info 0) (xerbla "CGBEQUB" (f2cl-lib:int-sub info))
            (go end_label)))
          (cond
           ((or (= m 0) (= n 0)) (setf rowcnd one) (setf colcnd one)
```

```
  (setf amax zero) (go end_label)))
(setf smlnum (slamch "S"))
(setf bignum (/ one smlnum))
(setf radix (slamch "B"))
(setf logrdx (f2cl-lib:flog radix))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%) zero)
   label10))
(setf kd (f2cl-lib:int-add ku 1))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                    (the f2cl-lib:integer4 1))
                   (f2cl-lib:int-add i 1))
                  ((> i
                      (min
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add j kl))
                       (the f2cl-lib:integer4 m)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                     (cabs1
                      (f2cl-lib:fref ab-%data%
                                     ((f2cl-lib:int-sub
                                       (f2cl-lib:int-add kd i) j)
                                      j)
                                     ((1 ldab) (1 *)) ab-%offset%))))
       label20))
   label30))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref r (i) ((1 *))) zero)
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
              (expt radix
                    (f2cl-lib:int
                     (/
                      (f2cl-lib:flog
                       (f2cl-lib:fref r-%data% (i) ((1 *))
                                      r-%offset%))
                      logrdx))))))
```

```
   label100000))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf rcmax
            (max rcmax
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
    (setf rcmin
            (min rcmin
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
   label40))
(setf amax rcmax)
(cond
 ((= rcmin zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref r (i) ((1 *))) zero) (setf info i)
        (go end_label)))
     label50)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
              (/ one
                 (min
                  (max
                   (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                   smlnum)
                  bignum)))
     label60))
  (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) zero)
   label70))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                    (the f2cl-lib:integer4 1))
                   (f2cl-lib:int-add i 1))
```

```
                    ((> i
                        (min
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add j kl))
                         (the f2cl-lib:integer4 m)))
                     nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                     (*
                      (cabs1
                       (f2cl-lib:fref ab-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add kd i) j)
                                       j)
                                      ((1 ldab) (1 *)) ab-%offset%))
                      (f2cl-lib:fref r-%data% (i) ((1 *))
                                     r-%offset%))))
       label80))
    (cond
     ((> (f2cl-lib:fref c (j) ((1 *))) zero)
      (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
              (expt radix
                    (f2cl-lib:int
                     (/
                      (f2cl-lib:flog
                       (f2cl-lib:fref c-%data% (j) ((1 *))
                                      c-%offset%))
                      logrdx))))))
   label90))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf rcmin
            (min rcmin
                 (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
    (setf rcmax
            (max rcmax
                 (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
   label100))
(cond
 ((= rcmin zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref c (j) ((1 *))) zero)
        (setf info (f2cl-lib:int-add m j)) (go end_label)))
```

```
              label110)))
           (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                        (/ one
                           (min
                            (max
                             (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                             smlnum)
                            bignum)))
              label120))
            (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
          (go end_label)
         end_label
          (return
           (values nil nil nil nil nil nil nil nil rowcnd colcnd amax
                   info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgbequb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (single-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
                                              fortran-to-lisp::rowcnd
                                              fortran-to-lisp::colcnd
                                              fortran-to-lisp::amax
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

46.5 cgbrfs

46.5.1 Axiom unit tests

— cgbrfs.input —

```
)set break resume
)sys rm -f cgbrfs.output
)spool cgbrfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

46.5.2 Axiom help page

— cgbrfs.help —

```
======================================================================
cgbrfs examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgbrfs (TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB, IPIV,
           B, LDB, X, LDX, FERR, BERR, WORK, RWORK, INFO)
           CGBRFS

Function/Subroutine Documentation
   subroutine cgbrfs (character TRANS, integer N, integer KL, integer KU,
       integer NRHS, complex, dimension( ldab, * ) AB, integer LDAB, complex,
       dimension( ldafb, * ) AFB, integer LDAFB, integer, dimension( * ) IPIV,
       complex, dimension( ldb, * ) B, integer LDB, complex, dimension( ldx, *
       ) X, integer LDX, real, dimension( * ) FERR, real, dimension( * ) BERR,
       complex, dimension( * ) WORK, real, dimension( * ) RWORK, integer INFO)
       CGBRFS

Purpose:
```

```
  CGBRFS improves the computed solution to a system of linear
  equations when the coefficient matrix is banded, and provides
  error bounds and backward error estimates for the solution.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 AB
   AB is COMPLEX array, dimension (LDAB,N)
   The original band matrix A, stored in rows 1 to KL+KU+1.
   The j-th column of A is stored in the j-th column of the
   array AB as follows:
   AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(n,j+kl).

 LDAB
   LDAB is INTEGER
   The leading dimension of the array AB.  LDAB >= KL+KU+1.

 AFB
   AFB is COMPLEX array, dimension (LDAFB,N)
   Details of the LU factorization of the band matrix A, as
   computed by CGBTRF.  U is stored as an upper triangular band
   matrix with KL+KU superdiagonals in rows 1 to KL+KU+1, and
   the multipliers used during the factorization are stored in
   rows KL+KU+2 to 2*KL+KU+1.

 LDAFB
```

```
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL*KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from CGBTRF; for 1<=i<=N, row i of the
  matrix was interchanged with row IPIV(i).

B
  B is COMPLEX array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by CGBTRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

FERR
  FERR is REAL array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is REAL array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is COMPLEX array, dimension (2*N)

RWORK
  RWORK is REAL array, dimension (N)

INFO
  INFO is INTEGER
```

```
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

 ITMAX is the maximum number of steps of iterative refinement.
```

———-

46.5.3 fortran code

— **cgbrfs.f** —

```
*  =====================================================================
      SUBROUTINE CGBRFS( TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB,
     $                   IPIV, B, LDB, X, LDX, FERR, BERR, WORK, RWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, KL, KU, LDAB, LDAFB, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               BERR( * ), FERR( * ), RWORK( * )
      COMPLEX            AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
      COMPLEX            CONE
      PARAMETER          ( CONE = ( 1.0E+0, 0.0E+0 ) )
      REAL               TWO
      PARAMETER          ( TWO = 2.0E+0 )
      REAL               THREE
      PARAMETER          ( THREE = 3.0E+0 )
```

```
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      CHARACTER          TRANSN, TRANST
      INTEGER            COUNT, I, J, K, KASE, KK, NZ
      REAL               EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN, XK
      COMPLEX            ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           CAXPY, CCOPY, CGBMV, CGBTRS, CLACN2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, MAX, MIN, REAL
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH
      EXTERNAL           LSAME, SLAMCH
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -7
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
         INFO = -9
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
```

```
         INFO = -12
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -14
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGBRFS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
      IF( NOTRAN ) THEN
         TRANSN = 'N'
         TRANST = 'C'
      ELSE
         TRANSN = 'C'
         TRANST = 'N'
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = MIN( KL+KU+2, N+1 )
      EPS = SLAMCH( 'Epsilon' )
      SAFMIN = SLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 140 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - op(A) * X,
*        where op(A) = A, A**T, or A**H, depending on TRANS.
*
         CALL CCOPY( N, B( 1, J ), 1, WORK, 1 )
         CALL CGBMV( TRANS, N, N, KL, KU, -CONE, AB, LDAB, X( 1, J ), 1,
```

```
     $               CONE, WORK, 1 )
*
*        Compute componentwise relative backward error from formula
*
*        max(i) ( abs(R(i)) / ( abs(op(A))*abs(X) + abs(B) )(i) )
*
*        where abs(Z) is the componentwise absolute value of the matrix
*        or vector Z.  If the i-th component of the denominator is less
*        than SAFE2, then SAFE1 is added to the i-th components of the
*        numerator and denominator before dividing.
*
         DO 30 I = 1, N
            RWORK( I ) = CABS1( B( I, J ) )
   30    CONTINUE
*
*        Compute abs(op(A))*abs(X) + abs(B).
*
         IF( NOTRAN ) THEN
            DO 50 K = 1, N
               KK = KU + 1 - K
               XK = CABS1( X( K, J ) )
               DO 40 I = MAX( 1, K-KU ), MIN( N, K+KL )
                  RWORK( I ) = RWORK( I ) + CABS1( AB( KK+I, K ) )*XK
   40          CONTINUE
   50       CONTINUE
         ELSE
            DO 70 K = 1, N
               S = ZERO
               KK = KU + 1 - K
               DO 60 I = MAX( 1, K-KU ), MIN( N, K+KL )
                  S = S + CABS1( AB( KK+I, K ) )*CABS1( X( I, J ) )
   60          CONTINUE
               RWORK( K ) = RWORK( K ) + S
   70       CONTINUE
         END IF
         S = ZERO
         DO 80 I = 1, N
            IF( RWORK( I ).GT.SAFE2 ) THEN
               S = MAX( S, CABS1( WORK( I ) ) / RWORK( I ) )
            ELSE
               S = MAX( S, ( CABS1( WORK( I ) )+SAFE1 ) /
     $             ( RWORK( I )+SAFE1 ) )
            END IF
   80    CONTINUE
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
*           1) The residual BERR(J) is larger than machine epsilon, and
*           2) BERR(J) decreased by at least a factor of 2 during the
*              last iteration, and
```

```
*          3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
*
*          Update solution and try again.
*
           CALL CGBTRS( TRANS, N, KL, KU, 1, AFB, LDAFB, IPIV, WORK, N,
     $                  INFO )
           CALL CAXPY( N, CONE, WORK, 1, X( 1, J ), 1 )
           LSTRES = BERR( J )
           COUNT = COUNT + 1
           GO TO 20
         END IF
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(op(A)))*
*           ( abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
*          inv(op(A)) is the inverse of op(A)
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(op(A))*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(op(A))*abs(X) + abs(B) is less than SAFE2.
*
*        Use CLACN2 to estimate the infinity-norm of the matrix
*           inv(op(A)) * diag(W),
*        where W = abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) )))
*
         DO 90 I = 1, N
            IF( RWORK( I ).GT.SAFE2 ) THEN
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I )
            ELSE
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I ) +
     $                      SAFE1
            END IF
   90    CONTINUE
*
         KASE = 0
  100    CONTINUE
         CALL CLACN2( N, WORK( N+1 ), WORK, FERR( J ), KASE, ISAVE )
         IF( KASE.NE.0 ) THEN
```

```
            IF( KASE.EQ.1 ) THEN
*
*              Multiply by diag(W)*inv(op(A)**H).
*
               CALL CGBTRS( TRANST, N, KL, KU, 1, AFB, LDAFB, IPIV,
     $                      WORK, N, INFO )
               DO 110 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
  110          CONTINUE
            ELSE
*
*              Multiply by inv(op(A))*diag(W).
*
               DO 120 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
  120          CONTINUE
               CALL CGBTRS( TRANSN, N, KL, KU, 1, AFB, LDAFB, IPIV,
     $                      WORK, N, INFO )
            END IF
            GO TO 100
         END IF
*
*        Normalize error.
*
         LSTRES = ZERO
         DO 130 I = 1, N
            LSTRES = MAX( LSTRES, CABS1( X( I, J ) ) )
  130    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  140 CONTINUE
*
      RETURN
*
*     End of CGBRFS
*
      END
```

———-

46.5.4 lisp code

— cgbrfs.lisp —

```
(let* ((itmax 5)
       (zero 0.0)
       (cone (f2cl-lib:cmplx 1.0 0.0))
```

```
      (two 2.0)
      (three 3.0))
 (declare (type (f2cl-lib:integer4 5 5) itmax)
          (type (single-float 0.0 0.0) zero)
          (type (f2cl-lib:complex8) cone)
          (type (single-float 2.0 2.0) two)
          (type (single-float 3.0 3.0) three)
          (ignorable itmax zero cone two three))
 (defun cgbrfs
        (trans n kl ku nrhs ab ldab afb ldafb ipiv b ldb$ x ldx ferr berr work
         rwork info)
   (declare (type (array single-float (*)) rwork berr ferr)
            (type (array f2cl-lib:integer4 (*)) ipiv)
            (type (array f2cl-lib:complex8 (*)) work x b afb ab)
            (type (f2cl-lib:integer4) info ldx ldb$ ldafb ldab nrhs ku kl n)
            (type (string *) trans))
   (f2cl-lib:with-multi-array-data
       ((trans character trans-%data% trans-%offset%)
        (ab f2cl-lib:complex8 ab-%data% ab-%offset%)
        (afb f2cl-lib:complex8 afb-%data% afb-%offset%)
        (b f2cl-lib:complex8 b-%data% b-%offset%)
        (x f2cl-lib:complex8 x-%data% x-%offset%)
        (work f2cl-lib:complex8 work-%data% work-%offset%)
        (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
        (ferr single-float ferr-%data% ferr-%offset%)
        (berr single-float berr-%data% berr-%offset%)
        (rwork single-float rwork-%data% rwork-%offset%))
     (labels ((cabs1 (zdum)
                (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
       (declare
        (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
         cabs1))
       (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
              (zdum #C(0.0 0.0)) (eps 0.0) (lstres 0.0) (s 0.0) (safe1 0.0)
              (safe2 0.0) (safmin 0.0) (xk 0.0) (count$ 0) (i 0) (j 0) (k 0)
              (kase 0) (kk 0) (nz 0)
              (transn
               (make-array '(1) :element-type 'character :initial-element
                           #\ ))
              (transt
               (make-array '(1) :element-type 'character :initial-element
                           #\ ))
              (notran nil))
         (declare (type (array f2cl-lib:integer4 (3)) isave)
                  (type (f2cl-lib:complex8) zdum)
                  (type (single-float) eps lstres s safe1 safe2 safmin xk)
                  (type (f2cl-lib:integer4) count$ i j k kase kk nz)
                  (type (string 1) transn transt)
                  (type f2cl-lib:logical notran))
         (setf info 0)
```

```
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< kl 0) (setf info -3))
 ((< ku 0) (setf info -4)) ((< nrhs 0) (setf info -5))
 ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -7))
 ((< ldafb (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
  (setf info -9))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -12))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -14)))
(cond
 ((/= info 0) (xerbla "CGBRFS" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
              zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%)
              zero)
     label10))
  (go end_label)))
(cond
 (notran (f2cl-lib:f2cl-set-string transn "N" (string 1))
  (f2cl-lib:f2cl-set-string transt "C" (string 1)))
 (t (f2cl-lib:f2cl-set-string transn "C" (string 1))
  (f2cl-lib:f2cl-set-string transt "N" (string 1))))
(setf nz
```

```
        (min (the f2cl-lib:integer4 (f2cl-lib:int-add kl ku 2))
             (the f2cl-lib:integer4 (f2cl-lib:int-add n 1))))
(setf eps (slamch "Epsilon"))
(setf safmin (slamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (ccopy n
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1 work 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (cgbmv trans n n kl ku (- cone) ab ldab
         (f2cl-lib:array-slice x-%data% f2cl-lib:complex8 (1 j)
                               ((1 ldx) (1 *)) x-%offset%)
         1 cone work 1)
      (declare (ignore var-5 var-6 var-8 var-9 var-11 var-12))
      (when var-0 (setf trans var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf kl var-3))
      (when var-4 (setf ku var-4))
      (when var-7 (setf ldab var-7))
      (when var-10 (setf cone var-10)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                (cabs1
                 (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)))
       label30))
    (cond
     (notran
      (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                    ((> k n) nil)
        (tagbody
          (setf kk (f2cl-lib:int-sub (f2cl-lib:int-add ku 1) k))
          (setf xk
                  (cabs1
```

```
                (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                               x-%offset%)))
        (f2cl-lib:fdo (i
                       (max (the f2cl-lib:integer4 1)
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add k
                                                   (f2cl-lib:int-sub
                                                    ku))))
                       (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the f2cl-lib:integer4 n)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add k kl))))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                 rwork-%offset%)
                  (+
                   (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                  rwork-%offset%)
                   (*
                    (cabs1
                     (f2cl-lib:fref ab-%data%
                                    ((f2cl-lib:int-add kk i) k)
                                    ((1 ldab) (1 *))
                                    ab-%offset%))
                    xk)))
           label40))
       label50)))
    (t
     (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                   ((> k n) nil)
       (tagbody
         (setf s zero)
         (setf kk (f2cl-lib:int-sub (f2cl-lib:int-add ku 1) k))
         (f2cl-lib:fdo (i
                        (max (the f2cl-lib:integer4 1)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add k
                                                    (f2cl-lib:int-sub
                                                     ku))))
                        (f2cl-lib:int-add i 1))
                       ((> i
                           (min (the f2cl-lib:integer4 n)
                                (the f2cl-lib:integer4
                                     (f2cl-lib:int-add k kl))))
                        nil)
           (tagbody
             (setf s
                   (+ s
```

```
                        (*
                         (cabs1
                          (f2cl-lib:fref ab-%data%
                                         ((f2cl-lib:int-add kk i) k)
                                         ((1 ldab) (1 *))
                                         ab-%offset%))
                         (cabs1
                          (f2cl-lib:fref x-%data% (i j)
                                         ((1 ldx) (1 *))
                                         x-%offset%)))))
          label60))
       (setf (f2cl-lib:fref rwork-%data% (k) ((1 *))
                            rwork-%offset%)
             (+
              (f2cl-lib:fref rwork-%data% (k) ((1 *))
                             rwork-%offset%)
              s))
     label70))))
(setf s zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
      (setf s
              (max s
                   (/
                    (cabs1
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%))
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%)))))
     (t
      (setf s
              (max s
                   (/
                    (+
                     (cabs1
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%))
                     safe1)
                    (+
                     (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                    rwork-%offset%)
                     safe1))))))
   label80))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
```

```
        (<= count$ itmax))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10)
        (cgbtrs trans n kl ku 1 afb ldafb ipiv work n info)
      (declare (ignore var-4 var-5 var-7 var-8))
      (when var-0 (setf trans var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf kl var-2))
      (when var-3 (setf ku var-3))
      (when var-6 (setf ldafb var-6))
      (when var-9 (setf n var-9))
      (when var-10 (setf info var-10)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (caxpy n cone work 1
         (f2cl-lib:array-slice x-%data% f2cl-lib:complex8 (1 j)
                               ((1 ldx) (1 *)) x-%offset%)
         1)
      (declare (ignore var-2 var-3 var-4 var-5))
      (when var-0 (setf n var-0))
      (when var-1 (setf cone var-1)))
    (setf lstres
            (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
    (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                             rwork-%offset%)
                (+
                 (cabs1
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%))
                 (* nz eps
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%)))))
       (t
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                             rwork-%offset%)
                (+
                 (cabs1
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%))
                 (* nz eps
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%))
                 safe1))))
     label90))
```

```
 (setf kase 0)
label100
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
     (clacn2 n
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                            ((+ n 1)) ((1 *)) work-%offset%)
      work
      (f2cl-lib:array-slice ferr-%data% single-float (j) ((1 *))
                            ferr-%offset%)
      kase isave)
   (declare (ignore var-1 var-2 var-3 var-5))
   (when var-0 (setf n var-0))
   (when var-4 (setf kase var-4)))
 (cond
  ((/= kase 0)
   (cond
    ((= kase 1)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
          var-9 var-10)
         (cgbtrs transt n kl ku 1 afb ldafb ipiv work n info)
       (declare (ignore var-4 var-5 var-7 var-8))
       (when var-0 (setf transt var-0))
       (when var-1 (setf n var-1))
       (when var-2 (setf kl var-2))
       (when var-3 (setf ku var-3))
       (when var-6 (setf ldafb var-6))
       (when var-9 (setf n var-9))
       (when var-10 (setf info var-10)))
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                              work-%offset%)
                 (*
                  (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                 rwork-%offset%)
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)))
        label110)))
    (t
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                              work-%offset%)
                 (*
                  (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                 rwork-%offset%)
                  (f2cl-lib:fref work-%data% (i) ((1 *))
```

```
                                        work-%offset%)))
                    label120))
                  (multiple-value-bind
                      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                       var-9 var-10)
                      (cgbtrs transn n kl ku 1 afb ldafb ipiv work n info)
                    (declare (ignore var-4 var-5 var-7 var-8))
                    (when var-0 (setf transn var-0))
                    (when var-1 (setf n var-1))
                    (when var-2 (setf kl var-2))
                    (when var-3 (setf ku var-3))
                    (when var-6 (setf ldafb var-6))
                    (when var-9 (setf n var-9))
                    (when var-10 (setf info var-10)))))
                (go label100)))
              (setf lstres zero)
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i n) nil)
                (tagbody
                  (setf lstres
                          (max lstres
                               (cabs1
                                (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                               x-%offset%))))
                 label130))
              (if (/= lstres zero)
                  (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                          (/
                           (f2cl-lib:fref ferr-%data% (j) ((1 *))
                                          ferr-%offset%)
                           lstres)))
            label140))
          (go end_label)
        end_label
          (return
           (values trans n kl ku nil nil ldab nil ldafb nil nil nil nil nil nil
                   nil nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgbrfs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
```

```
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::complex8
   (*))
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::trans
  fortran-to-lisp::n
  fortran-to-lisp::kl
  fortran-to-lisp::ku nil nil
  fortran-to-lisp::ldab nil
  fortran-to-lisp::ldafb nil nil
  nil nil nil nil nil nil nil
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

46.6 cgbrfsx

46.6.1 Axiom unit tests

— cgbrfsx.input —

```
)set break resume
)sys rm -f cgbrfsx.output
)spool cgbrfsx.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

46.6.2 Axiom help page

— **cgbrfsx.help** —

```
==================================================================
cgbrfsx examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgbrfsx (TRANS, EQUED, N, KL, KU, NRHS, AB, LDAB, AFB,
           LDAFB, IPIV, R, C, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS,
           ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, RWORK, INFO)
           CGBRFSX

Function/Subroutine Documentation
   subroutine cgbrfsx (character TRANS, character EQUED, integer N, integer
       KL, integer KU, integer NRHS, complex, dimension( ldab, * ) AB, integer
       LDAB, complex, dimension( ldafb, * ) AFB, integer LDAFB, integer,
       dimension( * ) IPIV, real, dimension( * ) R, real, dimension( * ) C,
       complex, dimension( ldb, * ) B, integer LDB, complex, dimension( ldx ,
       * ) X, integer LDX, real RCOND, real, dimension( * ) BERR, integer
       N_ERR_BNDS, real, dimension( nrhs, * ) ERR_BNDS_NORM, real, dimension(
       nrhs, * ) ERR_BNDS_COMP, integer NPARAMS, real, dimension( * ) PARAMS,
       complex, dimension( * ) WORK, real, dimension( * ) RWORK, integer INFO)
       CGBRFSX

Purpose:
   CGBRFSX improves the computed solution to a system of linear
   equations and provides error bounds and backward error estimates
   for the solution.  In addition to normwise error bound, the code
   provides maximum componentwise error bound if possible.  See
   comments for ERR_BNDS_NORM and ERR_BNDS_COMP for details of the
   error bounds.

   The original system of linear equations may have been equilibrated
   before calling this routine, as described by arguments EQUED, R
   and C below. In this case, the solution and error bounds returned
   are for the original unequilibrated system.
```

```
   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose = Transpose)

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done to A
   before calling this routine. This is needed to compute
   the solution and error bounds correctly.
   = 'N':  No equilibration
   = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
   = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
   = 'B':  Both row and column equilibration, i.e., A has been
           replaced by diag(R) * A * diag(C).
   The right hand side B has been changed accordingly.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 AB
   AB is COMPLEX array, dimension (LDAB,N)
   The original band matrix A, stored in rows 1 to KL+KU+1.
   The j-th column of A is stored in the j-th column of the
   array AB as follows:
```

```
  AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(n,j+kl).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is COMPLEX array, dimension (LDAFB,N)
  Details of the LU factorization of the band matrix A, as
  computed by DGBTRF.  U is stored as an upper triangular band
  matrix with KL+KU superdiagonals in rows 1 to KL+KU+1, and
  the multipliers used during the factorization are stored in
  rows KL+KU+2 to 2*KL+KU+1.

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL*KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from SGETRF; for 1<=i<=N, row i of the
  matrix was interchanged with row IPIV(i).

R
  R is REAL array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
  EQUED = 'R' or 'B', each element of R must be positive.
  If R is output, each element of R is a power of the radix.
  If R is input, each element of R should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

C
  C is REAL array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.
  If C is output, each element of C is a power of the radix.
  If C is input, each element of C should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
```

```
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by SGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

BERR
  BERR is REAL array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
        max_j (abs(XTRUE(j,i) - X(j,i)))
```

```
     -----------------------------
           max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
          reciprocal condition number is less than the threshold
          sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
          almost certainly within a factor of 10 of the true error
          so long as the next entry is greater than the threshold
          sqrt(n) * slamch('Epsilon'). This error bound should only
          be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
          reciprocal condition number.  Compared with the threshold
          sqrt(n) * slamch('Epsilon') to determine if the error
          estimate is "guaranteed". These reciprocal condition
          numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
          appropriately scaled matrix Z.
          Let Z = S*A, where S scales each row by a power of the
          radix so all absolute row sums of Z are approximately 1.

  See Lapack Working Note 165 for further details and extra cautions.

ERR_BNDS_COMP

  ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
              abs(XTRUE(j,i) - X(j,i))
      max_j ----------------------
                   abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
```

```
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
          reciprocal condition number is less than the threshold
          sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
          almost certainly within a factor of 10 of the true error
          so long as the next entry is greater than the threshold
          sqrt(n) * slamch('Epsilon'). This error bound should only
          be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
          reciprocal condition number.  Compared with the threshold
          sqrt(n) * slamch('Epsilon') to determine if the error
          estimate is "guaranteed". These reciprocal condition
          numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
          appropriately scaled matrix Z.
          Let Z = S*(A*diag(x)), where x is the solution for the
          current right-hand side and S scales each row of
          A*diag(x) by a power of the radix so all absolute row
          sums of Z are approximately 1.

  See Lapack Working Note 165 for further details and extra cautions.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is REAL array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.

  Default: 1.0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
```

```
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
          (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.

  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.

  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is COMPLEX array, dimension (2*N)

RWORK
  RWORK is REAL array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is
        guaranteed.
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
        has been completed, but the factor U is exactly singular, so
        the solution and error bounds could not be computed. RCOND = 0
        is returned.
  = N+J: The solution corresponding to the Jth right-hand side is
          not guaranteed. The solutions corresponding to other right-
          hand sides K with K > J may not be guaranteed as well, but
          only the first such right-hand side is reported. If a small
          componentwise error is not requested (PARAMS(3) = 0.0) then
          the Jth right-hand side is the first with a normwise error
          bound that is not guaranteed (the smallest J such
          that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
          the Jth right-hand side is the first with either a normwise or
          componentwise error bound that is not guaranteed (the smallest
          J such that either ERR_BNDS_NORM(J,1) = 0.0 or
          ERR_BNDS_COMP(J,1) = 0.0). See the definition of
          ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
          about all of the right-hand sides check ERR_BNDS_NORM or
```

ERR_BNDS_COMP.

———-

46.6.3 fortran code

— **cgbrfsx.f** —

```
*  ==================================================================
      SUBROUTINE CGBRFSX( TRANS, EQUED, N, KL, KU, NRHS, AB, LDAB, AFB,
     $                    LDAFB, IPIV, R, C, B, LDB, X, LDX, RCOND,
     $                    BERR, N_ERR_BNDS, ERR_BNDS_NORM,
     $                    ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, RWORK,
     $                    INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS, EQUED
      INTEGER            INFO, LDAB, LDAFB, LDB, LDX, N, KL, KU, NRHS,
     $                   NPARAMS, N_ERR_BNDS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   X( LDX , * ),WORK( * )
      REAL               R( * ), C( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * ), RWORK( * )
*     ..
*
*  ================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               ITREF_DEFAULT, ITHRESH_DEFAULT,
     $                   COMPONENTWISE_DEFAULT
      REAL               RTHRESH_DEFAULT, DZTHRESH_DEFAULT
      PARAMETER          ( ITREF_DEFAULT = 1.0 )
      PARAMETER          ( ITHRESH_DEFAULT = 10.0 )
      PARAMETER          ( COMPONENTWISE_DEFAULT = 1.0 )
      PARAMETER          ( RTHRESH_DEFAULT = 0.5 )
```

```
      PARAMETER          ( DZTHRESH_DEFAULT = 0.25 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. Local Scalars ..
      CHARACTER(1)       NORM
      LOGICAL            ROWEQU, COLEQU, NOTRAN, IGNORE_CWISE
      INTEGER            J, TRANS_TYPE, PREC_TYPE, REF_TYPE, N_NORMS,
     $                   ITHRESH
      REAL               ANORM, RCOND_TMP, ILLRCOND_THRESH, ERR_LBND,
     $                   CWISE_WRONG, RTHRESH, UNSTABLE_THRESH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, CGBCON, CLA_GBRFSX_EXTENDED
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT, TRANSFER
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, BLAS_FPINFO_X, ILATRANS, ILAPREC
      EXTERNAL           SLAMCH, CLANGB, CLA_GBRCOND_X, CLA_GBRCOND_C
      REAL               SLAMCH, CLANGB, CLA_GBRCOND_X, CLA_GBRCOND_C
      LOGICAL            LSAME
      INTEGER            BLAS_FPINFO_X
      INTEGER            ILATRANS, ILAPREC
*     ..
*     .. Executable Statements ..
*
*     Check the input parameters.
*
      INFO = 0
      TRANS_TYPE = ILATRANS( TRANS )
      REF_TYPE = INT( ITREF_DEFAULT )
      IF ( NPARAMS .GE. LA_LINRX_ITREF_I ) THEN
         IF ( PARAMS( LA_LINRX_ITREF_I ) .LT. 0.0 ) THEN
            PARAMS( LA_LINRX_ITREF_I ) = ITREF_DEFAULT
         ELSE
            REF_TYPE = PARAMS( LA_LINRX_ITREF_I )
         END IF
      END IF
*
*     Set default parameters.
*
```

```
      ILLRCOND_THRESH = REAL( N ) * SLAMCH( 'Epsilon' )
      ITHRESH = INT( ITHRESH_DEFAULT )
      RTHRESH = RTHRESH_DEFAULT
      UNSTABLE_THRESH = DZTHRESH_DEFAULT
      IGNORE_CWISE = COMPONENTWISE_DEFAULT .EQ. 0.0
*
      IF ( NPARAMS.GE.LA_LINRX_ITHRESH_I ) THEN
         IF ( PARAMS( LA_LINRX_ITHRESH_I ).LT.0.0 ) THEN
            PARAMS( LA_LINRX_ITHRESH_I ) = ITHRESH
         ELSE
            ITHRESH = INT( PARAMS( LA_LINRX_ITHRESH_I ) )
         END IF
      END IF
      IF ( NPARAMS.GE.LA_LINRX_CWISE_I ) THEN
         IF ( PARAMS( LA_LINRX_CWISE_I ).LT.0.0 ) THEN
            IF ( IGNORE_CWISE ) THEN
               PARAMS( LA_LINRX_CWISE_I ) = 0.0
            ELSE
               PARAMS( LA_LINRX_CWISE_I ) = 1.0
            END IF
         ELSE
            IGNORE_CWISE = PARAMS( LA_LINRX_CWISE_I ) .EQ. 0.0
         END IF
      END IF
      IF ( REF_TYPE .EQ. 0 .OR. N_ERR_BNDS .EQ. 0 ) THEN
         N_NORMS = 0
      ELSE IF ( IGNORE_CWISE ) THEN
         N_NORMS = 1
      ELSE
         N_NORMS = 2
      END IF
*
      NOTRAN = LSAME( TRANS, 'N' )
      ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
      COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
*
*     Test input parameters.
*
      IF( TRANS_TYPE.EQ.-1 ) THEN
        INFO = -1
      ELSE IF( .NOT.ROWEQU .AND. .NOT.COLEQU .AND.
     $         .NOT.LSAME( EQUED, 'N' ) ) THEN
        INFO = -2
      ELSE IF( N.LT.0 ) THEN
        INFO = -3
      ELSE IF( KL.LT.0 ) THEN
        INFO = -4
      ELSE IF( KU.LT.0 ) THEN
        INFO = -5
      ELSE IF( NRHS.LT.0 ) THEN
```

```
        INFO = -6
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
        INFO = -8
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
        INFO = -10
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
        INFO = -13
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
        INFO = -15
      END IF
      IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'CGBRFSX', -INFO )
        RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         RCOND = 1.0
         DO J = 1, NRHS
            BERR( J ) = 0.0
            IF ( N_ERR_BNDS .GE. 1 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
            IF ( N_ERR_BNDS .GE. 2 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 0.0
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 0.0
            END IF
            IF ( N_ERR_BNDS .GE. 3 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 1.0
            END IF
         END DO
         RETURN
      END IF
*
*     Default to failure.
*
      RCOND = 0.0
      DO J = 1, NRHS
         BERR( J ) = 1.0
         IF ( N_ERR_BNDS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
            ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
         END IF
         IF ( N_ERR_BNDS .GE. 2 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
         END IF
```

```
         IF ( N_ERR_BNDS .GE. 3 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 0.0
            ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 0.0
         END IF
      END DO
*
*     Compute the norm of A and the reciprocal of the condition
*     number of A.
*
      IF( NOTRAN ) THEN
         NORM = 'I'
      ELSE
         NORM = '1'
      END IF
      ANORM = CLANGB( NORM, N, KL, KU, AB, LDAB, RWORK )
      CALL CGBCON( NORM, N, KL, KU, AFB, LDAFB, IPIV, ANORM, RCOND,
     $     WORK, RWORK, INFO )
*
*     Perform refinement on each right-hand side
*
      IF ( REF_TYPE .NE. 0 ) THEN

         PREC_TYPE = ILAPREC( 'D' )

         IF ( NOTRAN ) THEN
            CALL CLA_GBRFSX_EXTENDED( PREC_TYPE, TRANS_TYPE,  N, KL, KU,
     $           NRHS, AB, LDAB, AFB, LDAFB, IPIV, COLEQU, C, B,
     $           LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM,
     $           ERR_BNDS_COMP, WORK, RWORK, WORK(N+1),
     $           TRANSFER (RWORK(1:2*N), (/ (ZERO, ZERO) /), N),
     $           RCOND, ITHRESH, RTHRESH, UNSTABLE_THRESH, IGNORE_CWISE,
     $           INFO )
         ELSE
            CALL CLA_GBRFSX_EXTENDED( PREC_TYPE, TRANS_TYPE,  N, KL, KU,
     $           NRHS, AB, LDAB, AFB, LDAFB, IPIV, ROWEQU, R, B,
     $           LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM,
     $           ERR_BNDS_COMP, WORK, RWORK, WORK(N+1),
     $           TRANSFER (RWORK(1:2*N), (/ (ZERO, ZERO) /), N),
     $           RCOND, ITHRESH, RTHRESH, UNSTABLE_THRESH, IGNORE_CWISE,
     $           INFO )
         END IF
      END IF

      ERR_LBND = MAX( 10.0, SQRT( REAL( N ) ) ) * SLAMCH( 'Epsilon' )
      IF (N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 1) THEN
*
*     Compute scaled normwise condition number cond(A*C).
*
         IF ( COLEQU .AND. NOTRAN ) THEN
            RCOND_TMP = CLA_GBRCOND_C( TRANS, N, KL, KU, AB, LDAB, AFB,
```

```
     $           LDAFB, IPIV, C, .TRUE., INFO, WORK, RWORK )
         ELSE IF ( ROWEQU .AND. .NOT. NOTRAN ) THEN
            RCOND_TMP = CLA_GBRCOND_C( TRANS, N, KL, KU, AB, LDAB, AFB,
     $           LDAFB, IPIV, R, .TRUE., INFO, WORK, RWORK )
         ELSE
            RCOND_TMP = CLA_GBRCOND_C( TRANS, N, KL, KU, AB, LDAB, AFB,
     $           LDAFB, IPIV, C, .FALSE., INFO, WORK, RWORK )
         END IF
         DO J = 1, NRHS
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .GT. 1.0)
     $           ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 0.0
               IF ( INFO .LE. N ) INFO = N + J
            ELSE IF ( ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .LT. ERR_LBND )
     $              THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF

         END DO
      END IF

      IF (N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 2) THEN
*
*     Compute componentwise condition number cond(A*diag(Y(:,J))) for
*     each right-hand side using the current solution as an estimate of
*     the true solution.  If the componentwise error estimate is too
*     large, then the solution is a lousy estimate of truth and the
*     estimated RCOND may be too optimistic.  To avoid misleading users,
*     the inverse condition number is set to 0.0 when the estimated
*     cwise error is at least CWISE_WRONG.
*
         CWISE_WRONG = SQRT( SLAMCH( 'Epsilon' ) )
         DO J = 1, NRHS
            IF (ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .LT. CWISE_WRONG )
```

```
     $     THEN
              RCOND_TMP = CLA_GBRCOND_X( TRANS, N, KL, KU, AB, LDAB,
     $              AFB, LDAFB, IPIV, X( 1, J ), INFO, WORK, RWORK )
           ELSE
              RCOND_TMP = 0.0
           END IF
*
*     Cap the error at 1.0.
*
           IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .GT. 1.0 )
     $          ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
*
*     Threshold the error (see LAWN).
*
           IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
              ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
              ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 0.0
              IF ( PARAMS( LA_LINRX_CWISE_I ) .EQ. 1.0
     $              .AND. INFO.LT.N + J ) INFO = N + J
           ELSE IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I )
     $              .LT. ERR_LBND ) THEN
              ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = ERR_LBND
              ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
           END IF
*
*     Save the condition number.
*
           IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
              ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = RCOND_TMP
           END IF

        END DO
     END IF
*
     RETURN
*
*     End of CGBRFSX
*
     END
```

———-

46.6.4 lisp code

— cgbrfsx.lisp —

———-

46.7 cgbtf2

46.7.1 Axiom unit tests

— **cgbtf2.input** —

```
)set break resume
)sys rm -f cgbtf2.output
)spool cgbtf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

46.7.2 Axiom help page

— **cgbtf2.help** —

```
======================================================================
cgbtf2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgbtf2 (M, N, KL, KU, AB, LDAB, IPIV, INFO)
           CGBTF2 computes the LU factorization of a general band matrix using
           the unblocked version of the algorithm.

Function/Subroutine Documentation
   subroutine cgbtf2 (integer M, integer N, integer KL, integer KU, complex,
       dimension( ldab, * ) AB, integer LDAB, integer, dimension( * ) IPIV,
       integer INFO)
       CGBTF2 computes the LU factorization of a general band matrix using the
       unblocked version of the algorithm.
```

```
Purpose:
   CGBTF2 computes an LU factorization of a complex m-by-n band matrix
   A using partial pivoting with row interchanges.

   This is the unblocked version of the algorithm, calling Level 2 BLAS.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 AB
   AB is COMPLEX array, dimension (LDAB,N)
   On entry, the matrix A in band storage, in rows KL+1 to
   2*KL+KU+1; rows 1 to KL of the array need not be set.
   The j-th column of A is stored in the j-th column of the
   array AB as follows:
   AB(kl+ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl)

   On exit, details of the factorization: U is stored as an
   upper triangular band matrix with KL+KU superdiagonals in
   rows 1 to KL+KU+1, and the multipliers used during the
   factorization are stored in rows KL+KU+2 to 2*KL+KU+1.
   See below for further details.

 LDAB
   LDAB is INTEGER
   The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.

 IPIV
   IPIV is INTEGER array, dimension (min(M,N))
   The pivot indices; for 1 <= i <= min(M,N), row i of the
   matrix was interchanged with row IPIV(i).

 INFO
   INFO is INTEGER
```

```
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value
   > 0: if INFO = +i, U(i,i) is exactly zero. The factorization
        has been completed, but the factor U is exactly
        singular, and division by zero will occur if it is used
        to solve a system of equations.

 The band storage scheme is illustrated by the following example, when
 M = N = 6, KL = 2, KU = 1:

    On entry:                       On exit:

        *    *    *    +    +    +       *    *    *   u14  u25  u36
        *    *    +    +    +    +       *    *   u13  u24  u35  u46
        *   a12  a23  a34  a45  a56      *   u12  u23  u34  u45  u56
       a11  a22  a33  a44  a55  a66     u11  u22  u33  u44  u55  u66
       a21  a32  a43  a54  a65   *      m21  m32  m43  m54  m65   *
       a31  a42  a53  a64   *    *      m31  m42  m53  m64   *    *

   Array elements marked * are not used by the routine; elements marked
   + need not be set on entry, but are required by the routine to store
   elements of U, because of fill-in resulting from the row
   interchanges.
```

———-

46.7.3 fortran code

— cgbtf2.f —

```
*  =====================================================================
      SUBROUTINE CGBTF2( M, N, KL, KU, AB, LDAB, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, KL, KU, LDAB, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            AB( LDAB, * )
*     ..
*
*  =====================================================================
```

```
*
*     .. Parameters ..
      COMPLEX            ONE, ZERO
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ),
     $                   ZERO = ( 0.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, JP, JU, KM, KV
*     ..
*     .. External Functions ..
      INTEGER            ICAMAX
      EXTERNAL           ICAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGERU, CSCAL, CSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     KV is the number of superdiagonals in the factor U, allowing for
*     fill-in.
*
      KV = KU + KL
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KV+1 ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGBTF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
```

```
*     Gaussian elimination with partial pivoting
*
*     Set fill-in elements in columns KU+2 to KV to zero.
*
      DO 20 J = KU + 2, MIN( KV, N )
         DO 10 I = KV - J + 2, KL
            AB( I, J ) = ZERO
   10    CONTINUE
   20 CONTINUE
*
*     JU is the index of the last column affected by the current stage
*     of the factorization.
*
      JU = 1
*
      DO 40 J = 1, MIN( M, N )
*
*        Set fill-in elements in column J+KV to zero.
*
         IF( J+KV.LE.N ) THEN
            DO 30 I = 1, KL
               AB( I, J+KV ) = ZERO
   30       CONTINUE
         END IF
*
*        Find pivot and test for singularity. KM is the number of
*        subdiagonal elements in the current column.
*
         KM = MIN( KL, M-J )
         JP = ICAMAX( KM+1, AB( KV+1, J ), 1 )
         IPIV( J ) = JP + J - 1
         IF( AB( KV+JP, J ).NE.ZERO ) THEN
            JU = MAX( JU, MIN( J+KU+JP-1, N ) )
*
*           Apply interchange to columns J to JU.
*
            IF( JP.NE.1 )
     $         CALL CSWAP( JU-J+1, AB( KV+JP, J ), LDAB-1,
     $                     AB( KV+1, J ), LDAB-1 )
            IF( KM.GT.0 ) THEN
*
*              Compute multipliers.
*
               CALL CSCAL( KM, ONE / AB( KV+1, J ), AB( KV+2, J ), 1 )
*
*              Update trailing submatrix within the band.
*
               IF( JU.GT.J )
     $            CALL CGERU( KM, JU-J, -ONE, AB( KV+2, J ), 1,
     $                        AB( KV, J+1 ), LDAB-1, AB( KV+1, J+1 ),
```

```
     $                        LDAB-1 )
            END IF
         ELSE
*
*           If pivot is zero, set INFO to the index of the pivot
*           unless a zero pivot has already been found.
*
            IF( INFO.EQ.0 )
     $         INFO = J
         END IF
   40 CONTINUE
      RETURN
*
*     End of CGBTF2
*
      END
```

———-

46.7.4 lisp code

— cgbtf2.lisp —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)) (zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (ignorable one zero))
  (defun cgbtf2 (m n kl ku ab ldab ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) ab)
             (type (f2cl-lib:integer4) info ldab ku kl n m))
    (f2cl-lib:with-multi-array-data
        ((ab f2cl-lib:complex8 ab-%data% ab-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((i 0) (j 0) (jp 0) (ju 0) (km 0) (kv 0))
        (declare (type (f2cl-lib:integer4) i j jp ju km kv))
        (setf kv (f2cl-lib:int-add ku kl))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< kl 0) (setf info -3)) ((< ku 0) (setf info -4))
              ((< ldab (f2cl-lib:int-add kl kv 1)) (setf info -6)))
        (cond
         ((/= info 0) (xerbla "CGBTF2" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (or (= m 0) (= n 0))
            (go end_label))
        (f2cl-lib:fdo (j (f2cl-lib:int-add ku 2) (f2cl-lib:int-add j 1))
                      ((> j
```

```
                    (min (the f2cl-lib:integer4 kv)
                         (the f2cl-lib:integer4 n)))
                nil)
  (tagbody
    (f2cl-lib:fdo (i (f2cl-lib:int-add kv (f2cl-lib:int-sub j) 2)
                   (f2cl-lib:int-add i 1))
                  ((> i kl) nil)
      (tagbody
        (setf (f2cl-lib:fref ab-%data% (i j) ((1 ldab) (1 *))
                             ab-%offset%)
              zero)
       label10))
   label20))
(setf ju 1)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j
                  (min (the f2cl-lib:integer4 m)
                       (the f2cl-lib:integer4 n)))
               nil)
  (tagbody
    (cond
     ((<= (f2cl-lib:int-add j kv) n)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i kl) nil)
        (tagbody
          (setf (f2cl-lib:fref ab-%data% (i (f2cl-lib:int-add j kv))
                               ((1 ldab) (1 *)) ab-%offset%)
                zero)
         label30))))
    (setf km
            (min (the f2cl-lib:integer4 kl)
                 (the f2cl-lib:integer4 (f2cl-lib:int-sub m j))))
    (setf jp
            (icamax (f2cl-lib:int-add km 1)
             (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                   ((+ kv 1) j) ((1 ldab) (1 *))
                                   ab-%offset%)
             1))
    (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
            (f2cl-lib:int-sub (f2cl-lib:int-add jp j) 1))
    (cond
     ((/=
       (f2cl-lib:fref ab ((f2cl-lib:int-add kv jp) j) ((1 ldab) (1 *)))
       zero)
      (setf ju
              (max (the f2cl-lib:integer4 ju)
                   (the f2cl-lib:integer4
                        (min
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-sub
```

```
                              (f2cl-lib:int-add j ku jp) 1))
                        (the f2cl-lib:integer4 n)))))
    (if (/= jp 1)
        (cswap (f2cl-lib:int-add (f2cl-lib:int-sub ju j) 1)
         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                               ((+ kv jp) j) ((1 ldab) (1 *))
                               ab-%offset%)
         (f2cl-lib:int-sub ldab 1)
         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                               ((+ kv 1) j) ((1 ldab) (1 *))
                               ab-%offset%)
         (f2cl-lib:int-sub ldab 1)))
    (cond
     ((> km 0)
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (cscal km
           (/ one
              (f2cl-lib:fref ab-%data% ((f2cl-lib:int-add kv 1) j)
                             ((1 ldab) (1 *)) ab-%offset%))
           (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                 ((+ kv 2) j) ((1 ldab) (1 *))
                                 ab-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf km var-0)))
      (if (> ju j)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (cgeru km (f2cl-lib:int-sub ju j) (- one)
               (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                     ((+ kv 2) j) ((1 ldab) (1 *))
                                     ab-%offset%)
               1
               (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                     (kv (f2cl-lib:int-add j 1))
                                     ((1 ldab) (1 *)) ab-%offset%)
               (f2cl-lib:int-sub ldab 1)
               (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                     ((+ kv 1)
                                      (f2cl-lib:int-add j 1))
                                     ((1 ldab) (1 *)) ab-%offset%)
               (f2cl-lib:int-sub ldab 1))
            (declare
             (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7
              var-8))
            (when var-0 (setf km var-0)))))))
  (t
   (if (= info 0)
       (setf info j))))
label40))
```

```
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgbtf2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

46.8 cgbtrf

46.8.1 Axiom unit tests

— **cgbtrf.input** —

```
)set break resume
)sys rm -f cgbtrf.output
)spool cgbtrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

46.8.2 Axiom help page

— cgbtrf.help —

```
====================================================================
cgbtrf examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgbtrf (M, N, KL, KU, AB, LDAB, IPIV, INFO)
           CGBTRF

Function/Subroutine Documentation
   subroutine cgbtrf (integer M, integer N, integer KL, integer KU, complex,
       dimension( ldab, * ) AB, integer LDAB, integer, dimension( * ) IPIV,
       integer INFO)
       CGBTRF

Purpose:
   CGBTRF computes an LU factorization of a complex m-by-n band matrix A
   using partial pivoting with row interchanges.

   This is the blocked version of the algorithm, calling Level 3 BLAS.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 AB
   AB is COMPLEX array, dimension (LDAB,N)
```

```
  On entry, the matrix A in band storage, in rows KL+1 to
  2*KL+KU+1; rows 1 to KL of the array need not be set.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(kl+ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl)

  On exit, details of the factorization: U is stored as an
  upper triangular band matrix with KL+KU superdiagonals in
  rows 1 to KL+KU+1, and the multipliers used during the
  factorization are stored in rows KL+KU+2 to 2*KL+KU+1.
  See below for further details.

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (min(M,N))
  The pivot indices; for 1 <= i <= min(M,N), row i of the
  matrix was interchanged with row IPIV(i).

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = +i, U(i,i) is exactly zero. The factorization
       has been completed, but the factor U is exactly
       singular, and division by zero will occur if it is used
       to solve a system of equations.

The band storage scheme is illustrated by the following example, when
M = N = 6, KL = 2, KU = 1:

  On entry:                       On exit:

  *    *    *    +    +    +       *    *    *   u14  u25  u36
  *    *    +    +    +    +       *    *   u13  u24  u35  u46
  *   a12  a23  a34  a45  a56      *   u12  u23  u34  u45  u56
  a11  a22  a33  a44  a55  a66     u11  u22  u33  u44  u55  u66
  a21  a32  a43  a54  a65   *      m21  m32  m43  m54  m65   *
  a31  a42  a53  a64   *    *      m31  m42  m53  m64   *    *

  Array elements marked * are not used by the routine; elements marked
  + need not be set on entry, but are required by the routine to store
  elements of U because of fill-in resulting from the row interchanges.
```

46.8.3 fortran code

— cgbtrf.f —

```
*  =====================================================================
      SUBROUTINE CGBTRF( M, N, KL, KU, AB, LDAB, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, KL, KU, LDAB, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            AB( LDAB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ONE, ZERO
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ),
     $                   ZERO = ( 0.0E+0, 0.0E+0 ) )
      INTEGER            NBMAX, LDWORK
      PARAMETER          ( NBMAX = 64, LDWORK = NBMAX+1 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, I2, I3, II, IP, J, J2, J3, JB, JJ, JM, JP,
     $                   JU, K2, KM, KV, NB, NW
      COMPLEX            TEMP
*     ..
*     .. Local Arrays ..
      COMPLEX            WORK13( LDWORK, NBMAX ),
     $                   WORK31( LDWORK, NBMAX )
*     ..
*     .. External Functions ..
      INTEGER            ICAMAX, ILAENV
      EXTERNAL           ICAMAX, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           CCOPY, CGBTF2, CGEMM, CGERU, CLASWP, CSCAL,
     $                   CSWAP, CTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
```

```
*     .. Executable Statements ..
*
*     KV is the number of superdiagonals in the factor U, allowing for
*     fill-in
*
      KV = KU + KL
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KV+1 ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGBTRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Determine the block size for this environment
*
      NB = ILAENV( 1, 'CGBTRF', ' ', M, N, KL, KU )
*
*     The block size must not exceed the limit set by the size of the
*     local arrays WORK13 and WORK31.
*
      NB = MIN( NB, NBMAX )
*
      IF( NB.LE.1 .OR. NB.GT.KL ) THEN
*
*        Use unblocked code
*
         CALL CGBTF2( M, N, KL, KU, AB, LDAB, IPIV, INFO )
      ELSE
*
*        Use blocked code
*
*        Zero the superdiagonal elements of the work array WORK13
```

```
*
        DO 20 J = 1, NB
           DO 10 I = 1, J - 1
              WORK13( I, J ) = ZERO
   10      CONTINUE
   20   CONTINUE
*
*       Zero the subdiagonal elements of the work array WORK31
*
        DO 40 J = 1, NB
           DO 30 I = J + 1, NB
              WORK31( I, J ) = ZERO
   30      CONTINUE
   40   CONTINUE
*
*       Gaussian elimination with partial pivoting
*
*       Set fill-in elements in columns KU+2 to KV to zero
*
        DO 60 J = KU + 2, MIN( KV, N )
           DO 50 I = KV - J + 2, KL
              AB( I, J ) = ZERO
   50      CONTINUE
   60   CONTINUE
*
*       JU is the index of the last column affected by the current
*       stage of the factorization
*
        JU = 1
*
        DO 180 J = 1, MIN( M, N ), NB
           JB = MIN( NB, MIN( M, N )-J+1 )
*
*          The active part of the matrix is partitioned
*
              A11   A12   A13
*             A21   A22   A23
*             A31   A32   A33
*
*          Here A11, A21 and A31 denote the current block of JB columns
*          which is about to be factorized. The number of rows in the
*          partitioning are JB, I2, I3 respectively, and the numbers
*          of columns are JB, J2, J3. The superdiagonal elements of A13
*          and the subdiagonal elements of A31 lie outside the band.
*
           I2 = MIN( KL-JB, M-J-JB+1 )
           I3 = MIN( JB, M-J-KL+1 )
*
*          J2 and J3 are computed after JU has been updated.
*
```

```
*          Factorize the current block of JB columns
*
           DO 80 JJ = J, J + JB - 1
*
*             Set fill-in elements in column JJ+KV to zero
*
              IF( JJ+KV.LE.N ) THEN
                 DO 70 I = 1, KL
                    AB( I, JJ+KV ) = ZERO
   70            CONTINUE
              END IF
*
*             Find pivot and test for singularity. KM is the number of
*             subdiagonal elements in the current column.
*
              KM = MIN( KL, M-JJ )
              JP = ICAMAX( KM+1, AB( KV+1, JJ ), 1 )
              IPIV( JJ ) = JP + JJ - J
              IF( AB( KV+JP, JJ ).NE.ZERO ) THEN
                 JU = MAX( JU, MIN( JJ+KU+JP-1, N ) )
                 IF( JP.NE.1 ) THEN
*
*                   Apply interchange to columns J to J+JB-1
*
                    IF( JP+JJ-1.LT.J+KL ) THEN
*
                       CALL CSWAP( JB, AB( KV+1+JJ-J, J ), LDAB-1,
     $                             AB( KV+JP+JJ-J, J ), LDAB-1 )
                    ELSE
*
*                      The interchange affects columns J to JJ-1 of A31
*                      which are stored in the work array WORK31
*
                       CALL CSWAP( JJ-J, AB( KV+1+JJ-J, J ), LDAB-1,
     $                             WORK31( JP+JJ-J-KL, 1 ), LDWORK )
                       CALL CSWAP( J+JB-JJ, AB( KV+1, JJ ), LDAB-1,
     $                             AB( KV+JP, JJ ), LDAB-1 )
                    END IF
                 END IF
*
*                Compute multipliers
*
                 CALL CSCAL( KM, ONE / AB( KV+1, JJ ), AB( KV+2, JJ ),
     $                       1 )
*
*                Update trailing submatrix within the band and within
*                the current block. JM is the index of the last column
*                which needs to be updated.
*
                 JM = MIN( JU, J+JB-1 )
```

```
                 IF( JM.GT.JJ )
     $              CALL CGERU( KM, JM-JJ, -ONE, AB( KV+2, JJ ), 1,
     $                          AB( KV, JJ+1 ), LDAB-1,
     $                          AB( KV+1, JJ+1 ), LDAB-1 )
              ELSE
*
*                If pivot is zero, set INFO to the index of the pivot
*                unless a zero pivot has already been found.
*
                 IF( INFO.EQ.0 )
     $              INFO = JJ
              END IF
*
*             Copy current column of A31 into the work array WORK31
*
              NW = MIN( JJ-J+1, I3 )
              IF( NW.GT.0 )
     $           CALL CCOPY( NW, AB( KV+KL+1-JJ+J, JJ ), 1,
     $                       WORK31( 1, JJ-J+1 ), 1 )
   80       CONTINUE
            IF( J+JB.LE.N ) THEN
*
*             Apply the row interchanges to the other blocks.
*
              J2 = MIN( JU-J+1, KV ) - JB
              J3 = MAX( 0, JU-J-KV+1 )
*
*             Use CLASWP to apply the row interchanges to A12, A22, and
*             A32.
*
              CALL CLASWP( J2, AB( KV+1-JB, J+JB ), LDAB-1, 1, JB,
     $                     IPIV( J ), 1 )
*
*             Adjust the pivot indices.
*
              DO 90 I = J, J + JB - 1
                 IPIV( I ) = IPIV( I ) + J - 1
   90         CONTINUE
*
*             Apply the row interchanges to A13, A23, and A33
*             columnwise.
*
              K2 = J - 1 + JB + J2
              DO 110 I = 1, J3
                 JJ = K2 + I
                 DO 100 II = J + I - 1, J + JB - 1
                    IP = IPIV( II )
                    IF( IP.NE.II ) THEN
                       TEMP = AB( KV+1+II-JJ, JJ )
                       AB( KV+1+II-JJ, JJ ) = AB( KV+1+IP-JJ, JJ )
```

```
                     AB( KV+1+IP-JJ, JJ ) = TEMP
                  END IF
  100          CONTINUE
  110       CONTINUE
*
*           Update the relevant part of the trailing submatrix
*
            IF( J2.GT.0 ) THEN
*
*              Update A12
*
               CALL CTRSM( 'Left', 'Lower', 'No transpose', 'Unit',
     $                     JB, J2, ONE, AB( KV+1, J ), LDAB-1,
     $                     AB( KV+1-JB, J+JB ), LDAB-1 )
*
               IF( I2.GT.0 ) THEN
*
*                 Update A22
*
                  CALL CGEMM( 'No transpose', 'No transpose', I2, J2,
     $                        JB, -ONE, AB( KV+1+JB, J ), LDAB-1,
     $                        AB( KV+1-JB, J+JB ), LDAB-1, ONE,
     $                        AB( KV+1, J+JB ), LDAB-1 )
               END IF
*
               IF( I3.GT.0 ) THEN
*
*                 Update A32
*
                  CALL CGEMM( 'No transpose', 'No transpose', I3, J2,
     $                        JB, -ONE, WORK31, LDWORK,
     $                        AB( KV+1-JB, J+JB ), LDAB-1, ONE,
     $                        AB( KV+KL+1-JB, J+JB ), LDAB-1 )
               END IF
            END IF
*
            IF( J3.GT.0 ) THEN
*
*              Copy the lower triangle of A13 into the work array
*              WORK13
*
               DO 130 JJ = 1, J3
                  DO 120 II = JJ, JB
                     WORK13( II, JJ ) = AB( II-JJ+1, JJ+J+KV-1 )
  120             CONTINUE
  130          CONTINUE
*
*              Update A13 in the work array
*
               CALL CTRSM( 'Left', 'Lower', 'No transpose', 'Unit',
```

```
     $                       JB, J3, ONE, AB( KV+1, J ), LDAB-1,
     $                       WORK13, LDWORK )
*
                  IF( I2.GT.0 ) THEN
*
*                    Update A23
*
                     CALL CGEMM( 'No transpose', 'No transpose', I2, J3,
     $                           JB, -ONE, AB( KV+1+JB, J ), LDAB-1,
     $                           WORK13, LDWORK, ONE, AB( 1+JB, J+KV ),
     $                           LDAB-1 )
                  END IF
*
                  IF( I3.GT.0 ) THEN
*
*                    Update A33
*
                     CALL CGEMM( 'No transpose', 'No transpose', I3, J3,
     $                           JB, -ONE, WORK31, LDWORK, WORK13,
     $                           LDWORK, ONE, AB( 1+KL, J+KV ), LDAB-1 )
                  END IF
*
*                 Copy the lower triangle of A13 back into place
*
                  DO 150 JJ = 1, J3
                     DO 140 II = JJ, JB
                        AB( II-JJ+1, JJ+J+KV-1 ) = WORK13( II, JJ )
  140                CONTINUE
  150             CONTINUE
               END IF
            ELSE
*
*              Adjust the pivot indices.
*
               DO 160 I = J, J + JB - 1
                  IPIV( I ) = IPIV( I ) + J - 1
  160          CONTINUE
            END IF
*
*           Partially undo the interchanges in the current block to
*           restore the upper triangular form of A31 and copy the upper
*           triangle of A31 back into place
*
            DO 170 JJ = J + JB - 1, J, -1
               JP = IPIV( JJ ) - JJ + 1
               IF( JP.NE.1 ) THEN
*
*                 Apply interchange to columns J to JJ-1
*
                  IF( JP+JJ-1.LT.J+KL ) THEN
```

```
*
*                    The interchange does not affect A31
*
                     CALL CSWAP( JJ-J, AB( KV+1+JJ-J, J ), LDAB-1,
     $                           AB( KV+JP+JJ-J, J ), LDAB-1 )
                  ELSE
*
*                    The interchange does affect A31
*
                     CALL CSWAP( JJ-J, AB( KV+1+JJ-J, J ), LDAB-1,
     $                           WORK31( JP+JJ-J-KL, 1 ), LDWORK )
                  END IF
               END IF
*
*              Copy the current column of A31 back into place
*
               NW = MIN( I3, JJ-J+1 )
               IF( NW.GT.0 )
     $            CALL CCOPY( NW, WORK31( 1, JJ-J+1 ), 1,
     $                        AB( KV+KL+1-JJ+J, JJ ), 1 )
  170       CONTINUE
  180    CONTINUE
      END IF
*
      RETURN
*
*     End of CGBTRF
*
      END
```

———-

46.8.4 lisp code

— **cgbtrf.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0))
       (zero (f2cl-lib:cmplx 0.0 0.0))
       (nbmax 64)
       (ldwork (+ nbmax 1)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (type (f2cl-lib:integer4 64 64) nbmax)
           (type (f2cl-lib:integer4) ldwork)
           (ignorable one zero nbmax ldwork))
  (defun cgbtrf (m n kl ku ab ldab ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) ab)
```

```
          (type (f2cl-lib:integer4) info ldab ku kl n m))
   (f2cl-lib:with-multi-array-data
       ((ab f2cl-lib:complex8 ab-%data% ab-%offset%)
        (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
     (prog ((work13
             (make-array (the fixnum (reduce #'* (list ldwork nbmax)))
                         :element-type 'f2cl-lib:complex8))
            (work31
             (make-array (the fixnum (reduce #'* (list ldwork nbmax)))
                         :element-type 'f2cl-lib:complex8))
            (temp #C(0.0 0.0)) (i 0) (i2 0) (i3 0) (ii 0) (ip 0) (j 0) (j2 0)
            (j3 0) (jb 0) (jj 0) (jm 0) (jp 0) (ju 0) (k2 0) (km 0) (kv 0)
            (nb 0) (nw 0))
       (declare (type (array f2cl-lib:complex8 (*)) work13 work31)
                (type (f2cl-lib:complex8) temp)
                (type (f2cl-lib:integer4) i i2 i3 ii ip j j2 j3 jb jj jm jp ju
                 k2 km kv nb nw))
       (setf kv (f2cl-lib:int-add ku kl))
       (setf info 0)
       (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
             ((< kl 0) (setf info -3)) ((< ku 0) (setf info -4))
             ((< ldab (f2cl-lib:int-add kl kv 1)) (setf info -6)))
       (cond
        ((/= info 0) (xerbla "CGBTRF" (f2cl-lib:int-sub info))
         (go end_label)))
       (if (or (= m 0) (= n 0))
           (go end_label))
       (setf nb
               (multiple-value-bind
                   (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                   (ilaenv 1 "CGBTRF" " " m n kl ku)
                 (declare (ignore var-0 var-1 var-2))
                 (when var-3 (setf m var-3))
                 (when var-4 (setf n var-4))
                 (when var-5 (setf kl var-5))
                 (when var-6 (setf ku var-6))
                 ret-val))
       (setf nb
               (min (the f2cl-lib:integer4 nb) (the f2cl-lib:integer4 nbmax)))
       (cond
        ((or (<= nb 1) (> nb kl))
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
             (cgbtf2 m n kl ku ab ldab ipiv info)
           (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6))
           (setf info var-7)))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j nb) nil)
           (tagbody
```

```
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref work13 (i j) ((1 ldwork) (1 nbmax)))
                zero)
       label10))
   label20))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nb) nil)
  (tagbody
    (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                  ((> i nb) nil)
      (tagbody
        (setf (f2cl-lib:fref work31 (i j) ((1 ldwork) (1 nbmax)))
                zero)
       label30))
   label40))
(f2cl-lib:fdo (j (f2cl-lib:int-add ku 2) (f2cl-lib:int-add j 1))
              ((> j
                  (min (the f2cl-lib:integer4 kv)
                       (the f2cl-lib:integer4 n)))
               nil)
  (tagbody
    (f2cl-lib:fdo (i (f2cl-lib:int-add kv (f2cl-lib:int-sub j) 2)
                   (f2cl-lib:int-add i 1))
                  ((> i kl) nil)
      (tagbody
        (setf (f2cl-lib:fref ab-%data% (i j) ((1 ldab) (1 *))
                             ab-%offset%)
                zero)
       label50))
   label60))
(setf ju 1)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
              ((> j
                  (min (the f2cl-lib:integer4 m)
                       (the f2cl-lib:integer4 n)))
               nil)
  (tagbody
    (setf jb
            (min (the f2cl-lib:integer4 nb)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add
                       (f2cl-lib:int-sub
                        (min (the f2cl-lib:integer4 m)
                             (the f2cl-lib:integer4 n))
                        j)
                       1))))
    (setf i2
```

```
        (min (the f2cl-lib:integer4 (f2cl-lib:int-sub kl jb))
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub m j jb)
                                    1))))
(setf i3
        (min (the f2cl-lib:integer4 jb)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub m j kl)
                                    1))))
(f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
              ((> jj
                  (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (cond
     ((<= (f2cl-lib:int-add jj kv) n)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i kl) nil)
        (tagbody
          (setf (f2cl-lib:fref ab-%data%
                               (i (f2cl-lib:int-add jj kv))
                               ((1 ldab) (1 *)) ab-%offset%)
                zero)
         label70))))
    (setf km
            (min (the f2cl-lib:integer4 kl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-sub m jj))))
    (setf jp
            (icamax (f2cl-lib:int-add km 1)
             (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                   ((+ kv 1) jj) ((1 ldab) (1 *))
                                   ab-%offset%)
             1))
    (setf (f2cl-lib:fref ipiv-%data% (jj) ((1 *)) ipiv-%offset%)
            (f2cl-lib:int-sub (f2cl-lib:int-add jp jj) j))
    (cond
     ((/=
       (f2cl-lib:fref ab ((f2cl-lib:int-add kv jp) jj)
                      ((1 ldab) (1 *)))
       zero)
      (setf ju
              (max (the f2cl-lib:integer4 ju)
                   (the f2cl-lib:integer4
                        (min
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-sub
                               (f2cl-lib:int-add jj ku jp) 1))
                         (the f2cl-lib:integer4 n)))))
      (cond
```

```
((/= jp 1)
 (cond
  ((< (f2cl-lib:int-add jp jj (f2cl-lib:int-sub 1))
      (f2cl-lib:int-add j kl))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
       (cswap jb
        (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                              ((+ kv 1 jj
                                  (f2cl-lib:int-sub j))
                               j)
                              ((1 ldab) (1 *))
                              ab-%offset%)
        (f2cl-lib:int-sub ldab 1)
        (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                              ((+ kv jp jj
                                  (f2cl-lib:int-sub j))
                               j)
                              ((1 ldab) (1 *))
                              ab-%offset%)
        (f2cl-lib:int-sub ldab 1))
     (declare (ignore var-1 var-2 var-3 var-4))
     (when var-0 (setf jb var-0))))
  (t
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
       (cswap (f2cl-lib:int-sub jj j)
        (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                              ((+ kv 1 jj
                                  (f2cl-lib:int-sub j))
                               j)
                              ((1 ldab) (1 *))
                              ab-%offset%)
        (f2cl-lib:int-sub ldab 1)
        (f2cl-lib:array-slice work31 f2cl-lib:complex8
                              ((+ jp jj
                                  (f2cl-lib:int-sub j)
                                  (f2cl-lib:int-sub kl))
                               1)
                              ((1 ldwork) (1 nbmax)))
        ldwork)
     (declare (ignore var-0 var-1 var-2 var-3))
     (when var-4 (setf ldwork var-4)))
   (cswap (f2cl-lib:int-sub (f2cl-lib:int-add j jb) jj)
    (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                          ((+ kv 1) jj) ((1 ldab) (1 *))
                          ab-%offset%)
    (f2cl-lib:int-sub ldab 1)
    (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                          ((+ kv jp) jj) ((1 ldab) (1 *))
                          ab-%offset%)
    (f2cl-lib:int-sub ldab 1))))))
```

```
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (cscal km
       (/ one
          (f2cl-lib:fref ab-%data%
                         ((f2cl-lib:int-add kv 1) jj)
                         ((1 ldab) (1 *)) ab-%offset%))
       (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                             ((+ kv 2) jj) ((1 ldab) (1 *))
                             ab-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3))
    (when var-0 (setf km var-0)))
  (setf jm
          (min (the f2cl-lib:integer4 ju)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-sub (f2cl-lib:int-add j jb)
                                      1))))
  (if (> jm jj)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
           var-8)
          (cgeru km (f2cl-lib:int-sub jm jj) (- one)
           (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                 ((+ kv 2) jj)
                                 ((1 ldab) (1 *))
                                 ab-%offset%)
           1
           (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                 (kv (f2cl-lib:int-add jj 1))
                                 ((1 ldab) (1 *))
                                 ab-%offset%)
           (f2cl-lib:int-sub ldab 1)
           (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                 ((+ kv 1)
                                  (f2cl-lib:int-add jj 1))
                                 ((1 ldab) (1 *))
                                 ab-%offset%)
           (f2cl-lib:int-sub ldab 1))
         (declare
          (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7
           var-8))
         (when var-0 (setf km var-0)))))
 (t
  (if (= info 0)
      (setf info jj))))
(setf nw
         (min
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-sub jj j) 1))
          (the f2cl-lib:integer4 i3)))
```

```
      (if (> nw 0)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (ccopy nw
               (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                     ((+ kv kl 1
                                         (f2cl-lib:int-sub jj) j)
                                      jj)
                                     ((1 ldab) (1 *)) ab-%offset%)
               1
               (f2cl-lib:array-slice work31 f2cl-lib:complex8
                                     (1
                                      (f2cl-lib:int-add
                                       (f2cl-lib:int-sub jj j) 1))
                                     ((1 ldwork) (1 nbmax)))
               1)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf nw var-0))))
    label80))
  (cond
   ((<= (f2cl-lib:int-add j jb) n)
    (setf j2
            (f2cl-lib:int-sub
             (min
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-sub ju j) 1))
              (the f2cl-lib:integer4 kv))
             jb))
    (setf j3
            (max (the f2cl-lib:integer4 0)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-sub ju j kv)
                                        1))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claswp j2
         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                               ((+ kv 1 (f2cl-lib:int-sub jb))
                                (f2cl-lib:int-add j jb))
                               ((1 ldab) (1 *)) ab-%offset%)
         (f2cl-lib:int-sub ldab 1) 1 jb
         (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (j)
                               ((1 *)) ipiv-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-5 var-6))
      (when var-0 (setf j2 var-0))
      (when var-4 (setf jb var-4)))
    (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                  ((> i
                      (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
                   nil)
```

```
  (tagbody
    (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
            (f2cl-lib:int-sub
             (f2cl-lib:int-add
              (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                             ipiv-%offset%)
              j)
             1))
   label90))
(setf k2 (f2cl-lib:int-add (f2cl-lib:int-sub j 1) jb j2))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i j3) nil)
  (tagbody
    (setf jj (f2cl-lib:int-add k2 i))
    (f2cl-lib:fdo (ii
                   (f2cl-lib:int-add j i (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add ii 1))
                  ((> ii
                      (f2cl-lib:int-add j jb
                                        (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf ip
                (f2cl-lib:fref ipiv-%data% (ii) ((1 *))
                               ipiv-%offset%))
        (cond
         ((/= ip ii)
          (setf temp
                  (f2cl-lib:fref ab-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-add kv 1 ii)
                                   jj)
                                  jj)
                                 ((1 ldab) (1 *)) ab-%offset%))
          (setf (f2cl-lib:fref ab-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add kv 1 ii) jj)
                                jj)
                               ((1 ldab) (1 *)) ab-%offset%)
                  (f2cl-lib:fref ab-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-add kv 1 ip)
                                   jj)
                                  jj)
                                 ((1 ldab) (1 *)) ab-%offset%))
          (setf (f2cl-lib:fref ab-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add kv 1 ip) jj)
                                jj)
                               ((1 ldab) (1 *)) ab-%offset%)
```

```
                    temp)))
            label100))
        label110))
    (cond
     ((> j2 0)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (ctrsm "Left" "Lower" "No transpose" "Unit" jb j2 one
           (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                 ((+ kv 1) j) ((1 ldab) (1 *))
                                 ab-%offset%)
           (f2cl-lib:int-sub ldab 1)
           (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                 ((+ kv 1 (f2cl-lib:int-sub jb))
                                  (f2cl-lib:int-add j jb))
                                 ((1 ldab) (1 *)) ab-%offset%)
           (f2cl-lib:int-sub ldab 1))
        (declare
         (ignore var-0 var-1 var-2 var-3 var-7 var-8 var-9 var-10))
        (when var-4 (setf jb var-4))
        (when var-5 (setf j2 var-5))
        (when var-6 (setf one var-6)))
      (cond
       ((> i2 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12)
            (cgemm "No transpose" "No transpose" i2 j2 jb (- one)
             (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                   ((+ kv 1 jb) j) ((1 ldab) (1 *))
                                   ab-%offset%)
             (f2cl-lib:int-sub ldab 1)
             (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                   ((+ kv 1 (f2cl-lib:int-sub jb))
                                    (f2cl-lib:int-add j jb))
                                   ((1 ldab) (1 *)) ab-%offset%)
             (f2cl-lib:int-sub ldab 1) one
             (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                   ((+ kv 1)
                                    (f2cl-lib:int-add j jb))
                                   ((1 ldab) (1 *)) ab-%offset%)
             (f2cl-lib:int-sub ldab 1))
          (declare
           (ignore var-0 var-1 var-5 var-6 var-7 var-8 var-9 var-11
            var-12))
          (when var-2 (setf i2 var-2))
          (when var-3 (setf j2 var-3))
          (when var-4 (setf jb var-4))
          (when var-10 (setf one var-10)))))
```

```
  (cond
   ((> i3 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (cgemm "No transpose" "No transpose" i3 j2 jb (- one)
         work31 ldwork
         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                               ((+ kv 1 (f2cl-lib:int-sub jb))
                                (f2cl-lib:int-add j jb))
                               ((1 ldab) (1 *)) ab-%offset%)
         (f2cl-lib:int-sub ldab 1) one
         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                               ((+ kv kl 1
                                   (f2cl-lib:int-sub jb))
                                (f2cl-lib:int-add j jb))
                               ((1 ldab) (1 *)) ab-%offset%)
         (f2cl-lib:int-sub ldab 1))
      (declare
       (ignore var-0 var-1 var-5 var-6 var-8 var-9 var-11
        var-12))
      (when var-2 (setf i3 var-2))
      (when var-3 (setf j2 var-3))
      (when var-4 (setf jb var-4))
      (when var-7 (setf ldwork var-7))
      (when var-10 (setf one var-10)))))))
(cond
 ((> j3 0)
  (f2cl-lib:fdo (jj 1 (f2cl-lib:int-add jj 1))
                ((> jj j3) nil)
    (tagbody
      (f2cl-lib:fdo (ii jj (f2cl-lib:int-add ii 1))
                    ((> ii jb) nil)
        (tagbody
          (setf (f2cl-lib:fref work13 (ii jj)
                               ((1 ldwork) (1 nbmax)))
                  (f2cl-lib:fref ab-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-sub ii jj) 1)
                                  (f2cl-lib:int-sub
                                   (f2cl-lib:int-add jj j kv)
                                   1))
                                 ((1 ldab) (1 *)) ab-%offset%))
         label120))
     label130))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (ctrsm "Left" "Lower" "No transpose" "Unit" jb j3 one
       (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
```

```
                                ((+ kv 1) j) ((1 ldab) (1 *))
                                ab-%offset%)
                      (f2cl-lib:int-sub ldab 1) work13 ldwork)
                    (declare
                     (ignore var-0 var-1 var-2 var-3 var-7 var-8 var-9))
                    (when var-4 (setf jb var-4))
                    (when var-5 (setf j3 var-5))
                    (when var-6 (setf one var-6))
                    (when var-10 (setf ldwork var-10)))
                  (cond
                   ((> i2 0)
                    (multiple-value-bind
                        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                         var-9 var-10 var-11 var-12)
                        (cgemm "No transpose" "No transpose" i2 j3 jb (- one)
                         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                               ((+ kv 1 jb) j) ((1 ldab) (1 *))
                                               ab-%offset%)
                         (f2cl-lib:int-sub ldab 1) work13 ldwork one
                         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                               ((+ 1 jb)
                                                (f2cl-lib:int-add j kv))
                                               ((1 ldab) (1 *)) ab-%offset%)
                         (f2cl-lib:int-sub ldab 1))
                      (declare
                       (ignore var-0 var-1 var-5 var-6 var-7 var-8 var-11
                        var-12))
                      (when var-2 (setf i2 var-2))
                      (when var-3 (setf j3 var-3))
                      (when var-4 (setf jb var-4))
                      (when var-9 (setf ldwork var-9))
                      (when var-10 (setf one var-10)))))
                  (cond
                   ((> i3 0)
                    (multiple-value-bind
                        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                         var-9 var-10 var-11 var-12)
                        (cgemm "No transpose" "No transpose" i3 j3 jb (- one)
                         work31 ldwork work13 ldwork one
                         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                               ((+ 1 kl)
                                                (f2cl-lib:int-add j kv))
                                               ((1 ldab) (1 *)) ab-%offset%)
                         (f2cl-lib:int-sub ldab 1))
                      (declare
                       (ignore var-0 var-1 var-5 var-6 var-8 var-11 var-12))
                      (when var-2 (setf i3 var-2))
                      (when var-3 (setf j3 var-3))
                      (when var-4 (setf jb var-4))
                      (when var-7 (setf ldwork var-7))
```

```
        (when var-9 (setf ldwork var-9))
        (when var-10 (setf one var-10)))))
    (f2cl-lib:fdo (jj 1 (f2cl-lib:int-add jj 1))
                  ((> jj j3) nil)
      (tagbody
        (f2cl-lib:fdo (ii jj (f2cl-lib:int-add ii 1))
                      ((> ii jb) nil)
          (tagbody
            (setf (f2cl-lib:fref ab-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-sub ii jj) 1)
                                  (f2cl-lib:int-sub
                                   (f2cl-lib:int-add jj j kv) 1))
                                 ((1 ldab) (1 *)) ab-%offset%)
                  (f2cl-lib:fref work13 (ii jj)
                                 ((1 ldwork) (1 nbmax))))
          label140))
      label150)))))
 (t
  (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                ((> i
                    (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
              (f2cl-lib:int-sub
               (f2cl-lib:int-add
                (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                               ipiv-%offset%)
                j)
               1))
     label160))))
(f2cl-lib:fdo (jj (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add jj (f2cl-lib:int-sub 1)))
              ((> jj j) nil)
  (tagbody
    (setf jp
            (f2cl-lib:int-add
             (f2cl-lib:int-sub
              (f2cl-lib:fref ipiv-%data% (jj) ((1 *))
                             ipiv-%offset%)
              jj)
             1))
    (cond
     ((/= jp 1)
      (cond
       ((< (f2cl-lib:int-add jp jj (f2cl-lib:int-sub 1))
           (f2cl-lib:int-add j kl))
        (cswap (f2cl-lib:int-sub jj j)
         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
```

```
                                        ((+ kv 1 jj (f2cl-lib:int-sub j))
                                         j)
                                        ((1 ldab) (1 *)) ab-%offset%)
                      (f2cl-lib:int-sub ldab 1)
                      (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                        ((+ kv jp jj (f2cl-lib:int-sub j))
                                         j)
                                        ((1 ldab) (1 *)) ab-%offset%)
                      (f2cl-lib:int-sub ldab 1)))
                    (t
                     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                         (cswap (f2cl-lib:int-sub jj j)
                          (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                                ((+ kv 1 jj
                                                    (f2cl-lib:int-sub j))
                                                 j)
                                                ((1 ldab) (1 *)) ab-%offset%)
                          (f2cl-lib:int-sub ldab 1)
                          (f2cl-lib:array-slice work31 f2cl-lib:complex8
                                                ((+ jp jj (f2cl-lib:int-sub j)
                                                    (f2cl-lib:int-sub kl))
                                                 1)
                                                ((1 ldwork) (1 nbmax)))
                          ldwork)
                       (declare (ignore var-0 var-1 var-2 var-3))
                       (when var-4 (setf ldwork var-4)))))))
                (setf nw
                        (min (the f2cl-lib:integer4 i3)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-sub jj j)
                                                    1))))
                (if (> nw 0)
                    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                        (ccopy nw
                         (f2cl-lib:array-slice work31 f2cl-lib:complex8
                                               (1
                                                (f2cl-lib:int-add
                                                 (f2cl-lib:int-sub jj j) 1))
                                               ((1 ldwork) (1 nbmax)))
                         1
                         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                               ((+ kv kl 1
                                                   (f2cl-lib:int-sub jj) j)
                                                jj)
                                               ((1 ldab) (1 *)) ab-%offset%)
                         1)
                       (declare (ignore var-1 var-2 var-3 var-4))
                       (when var-0 (setf nw var-0))))
            label170))
        label180))))
```

```
        (go end_label)
       end_label
        (return (values m n kl ku nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgbtrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n
                                              fortran-to-lisp::kl
                                              fortran-to-lisp::ku nil nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::cgbtf2))))
```

46.9 cgbtrs

46.9.1 Axiom unit tests

— cgbtrs.input —

```
)set break resume
)sys rm -f cgbtrs.output
)spool cgbtrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

46.9.2 Axiom help page

— cgbtrs.help —

```
=====================================================================
cgbtrs examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgbtrs (TRANS, N, KL, KU, NRHS, AB, LDAB, IPIV, B, LDB,
           INFO)
           CGBTRS

Function/Subroutine Documentation
   subroutine cgbtrs (character TRANS, integer N, integer KL, integer KU,
       integer NRHS, complex, dimension( ldab, * ) AB, integer LDAB, integer,
       dimension( * ) IPIV, complex, dimension( ldb, * ) B, integer LDB,
       integer INFO)
       CGBTRS

Purpose:
   CGBTRS solves a system of linear equations
      A * X = B,  A**T * X = B,  or  A**H * X = B
   with a general band matrix A using the LU factorization computed
   by CGBTRF.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations.
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
```

```
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrix B.  NRHS >= 0.

AB
  AB is COMPLEX array, dimension (LDAB,N)
  Details of the LU factorization of the band matrix A, as
  computed by CGBTRF.  U is stored as an upper triangular band
  matrix with KL+KU superdiagonals in rows 1 to KL+KU+1, and
  the multipliers used during the factorization are stored in
  rows KL+KU+2 to 2*KL+KU+1.

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= N, row i of the matrix was
  interchanged with row IPIV(i).

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the right hand side matrix B.
  On exit, the solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

46.9.3 fortran code

— cgbtrs.f —

```
*  =====================================================================
      SUBROUTINE CGBTRS( TRANS, N, KL, KU, NRHS, AB, LDAB, IPIV, B, LDB,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, KL, KU, LDAB, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            AB( LDAB, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ONE
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LNOTI, NOTRAN
      INTEGER            I, J, KD, L, LM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEMV, CGERU, CLACGV, CSWAP, CTBSV, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
```

```
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDAB.LT.( 2*KL+KU+1 ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -10
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGBTRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      KD = KU + KL + 1
      LNOTI = KL.GT.0
*
      IF( NOTRAN ) THEN
*
*        Solve  A*X = B.
*
*        Solve L*X = B, overwriting B with X.
*
*        L is represented as a product of permutations and unit lower
*        triangular matrices L = P(1) * L(1) * ... * P(n-1) * L(n-1),
*        where each transformation L(i) is a rank-one modification of
*        the identity matrix.
*
         IF( LNOTI ) THEN
            DO 10 J = 1, N - 1
               LM = MIN( KL, N-J )
               L = IPIV( J )
               IF( L.NE.J )
     $            CALL CSWAP( NRHS, B( L, 1 ), LDB, B( J, 1 ), LDB )
               CALL CGERU( LM, NRHS, -ONE, AB( KD+1, J ), 1, B( J, 1 ),
     $                     LDB, B( J+1, 1 ), LDB )
   10       CONTINUE
         END IF
*
```

```
         DO 20 I = 1, NRHS
*
*           Solve U*X = B, overwriting B with X.
*
            CALL CTBSV( 'Upper', 'No transpose', 'Non-unit', N, KL+KU,
     $                  AB, LDAB, B( 1, I ), 1 )
   20    CONTINUE
*
      ELSE IF( LSAME( TRANS, 'T' ) ) THEN
*
*        Solve A**T * X = B.
*
         DO 30 I = 1, NRHS
*
*           Solve U**T * X = B, overwriting B with X.
*
            CALL CTBSV( 'Upper', 'Transpose', 'Non-unit', N, KL+KU, AB,
     $                  LDAB, B( 1, I ), 1 )
   30    CONTINUE
*
*        Solve L**T * X = B, overwriting B with X.
*
         IF( LNOTI ) THEN
            DO 40 J = N - 1, 1, -1
               LM = MIN( KL, N-J )
               CALL CGEMV( 'Transpose', LM, NRHS, -ONE, B( J+1, 1 ),
     $                     LDB, AB( KD+1, J ), 1, ONE, B( J, 1 ), LDB )
               L = IPIV( J )
               IF( L.NE.J )
     $            CALL CSWAP( NRHS, B( L, 1 ), LDB, B( J, 1 ), LDB )
   40       CONTINUE
         END IF
*
      ELSE
*
*        Solve A**H * X = B.
*
         DO 50 I = 1, NRHS
*
*           Solve U**H * X = B, overwriting B with X.
*
            CALL CTBSV( 'Upper', 'Conjugate transpose', 'Non-unit', N,
     $                  KL+KU, AB, LDAB, B( 1, I ), 1 )
   50    CONTINUE
*
*        Solve L**H * X = B, overwriting B with X.
*
         IF( LNOTI ) THEN
            DO 60 J = N - 1, 1, -1
               LM = MIN( KL, N-J )
```

```
               CALL CLACGV( NRHS, B( J, 1 ), LDB )
               CALL CGEMV( 'Conjugate transpose', LM, NRHS, -ONE,
     $                     B( J+1, 1 ), LDB, AB( KD+1, J ), 1, ONE,
     $                     B( J, 1 ), LDB )
               CALL CLACGV( NRHS, B( J, 1 ), LDB )
               L = IPIV( J )
               IF( L.NE.J )
     $            CALL CSWAP( NRHS, B( L, 1 ), LDB, B( J, 1 ), LDB )
   60       CONTINUE
         END IF
      END IF
      RETURN
*
*     End of CGBTRS
*
      END
```

———-

46.9.4 lisp code

— cgbtrs.lisp —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (ignorable one))
  (defun cgbtrs (trans n kl ku nrhs ab ldab ipiv b ldb$ info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) b ab)
             (type (f2cl-lib:integer4) info ldb$ ldab nrhs ku kl n)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (ab f2cl-lib:complex8 ab-%data% ab-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((i 0) (j 0) (kd 0) (l 0) (lm 0) (lnoti nil) (notran nil))
        (declare (type (f2cl-lib:integer4) i j kd l lm)
                 (type f2cl-lib:logical lnoti notran))
        (setf info 0)
        (setf notran
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame trans "N")
                  (declare (ignore var-1))
                  (when var-0 (setf trans var-0))
                  ret-val))
        (cond
         ((and (not notran)
```

```
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame trans "T")
           (declare (ignore var-1))
           (when var-0 (setf trans var-0))
           ret-val))
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame trans "C")
           (declare (ignore var-1))
           (when var-0 (setf trans var-0))
           ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< kl 0) (setf info -3))
 ((< ku 0) (setf info -4)) ((< nrhs 0) (setf info -5))
 ((< ldab (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -10)))
(cond
 ((/= info 0) (xerbla "CGBTRS" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= n 0) (= nrhs 0))
    (go end_label))
(setf kd (f2cl-lib:int-add ku kl 1))
(setf lnoti (> kl 0))
(cond
 (notran
  (cond
   (lnoti
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (setf lm
                (min (the f2cl-lib:integer4 kl)
                     (the f2cl-lib:integer4 (f2cl-lib:int-sub n j))))
        (setf l (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
        (if (/= l j)
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (cswap nrhs
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (l 1)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (j 1)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$)
              (declare (ignore var-1 var-3))
              (when var-0 (setf nrhs var-0))
              (when var-2 (setf ldb$ var-2))
              (when var-4 (setf ldb$ var-4))))
```

```
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (cgeru lm nrhs (- one)
             (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                   ((+ kd 1) j) ((1 ldab) (1 *))
                                   ab-%offset%)
             1
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (j 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                   ((+ j 1) 1) ((1 ldb$) (1 *))
                                   b-%offset%)
             ldb$)
          (declare (ignore var-2 var-3 var-4 var-5 var-7))
          (when var-0 (setf lm var-0))
          (when var-1 (setf nrhs var-1))
          (when var-6 (setf ldb$ var-6))
          (when var-8 (setf ldb$ var-8)))
      label10))))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i nrhs) nil)
   (tagbody
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (ctbsv "Upper" "No transpose" "Non-unit" n
          (f2cl-lib:int-add kl ku) ab ldab
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 i)
                                ((1 ldb$) (1 *)) b-%offset%)
          1)
       (declare (ignore var-0 var-1 var-2 var-4 var-5 var-7 var-8))
       (when var-3 (setf n var-3))
       (when var-6 (setf ldab var-6)))
    label20)))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame trans "T")
   (declare (ignore var-1))
   (when var-0 (setf trans var-0))
   ret-val)
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i nrhs) nil)
   (tagbody
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (ctbsv "Upper" "Transpose" "Non-unit" n
          (f2cl-lib:int-add kl ku) ab ldab
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 i)
                                ((1 ldb$) (1 *)) b-%offset%)
          1)
       (declare (ignore var-0 var-1 var-2 var-4 var-5 var-7 var-8))
```

```
        (when var-3 (setf n var-3))
        (when var-6 (setf ldab var-6)))
    label30))
 (cond
  (lnoti
   (f2cl-lib:fdo (j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                  (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                 ((> j 1) nil)
     (tagbody
       (setf lm
               (min (the f2cl-lib:integer4 kl)
                    (the f2cl-lib:integer4 (f2cl-lib:int-sub n j))))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
            var-9 var-10)
           (cgemv "Transpose" lm nrhs (- one)
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                  ((+ j 1) 1) ((1 ldb$) (1 *))
                                  b-%offset%)
            ldb$
            (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                  ((+ kd 1) j) ((1 ldab) (1 *))
                                  ab-%offset%)
            1 one
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (j 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
         (declare (ignore var-0 var-3 var-4 var-6 var-7 var-9))
         (when var-1 (setf lm var-1))
         (when var-2 (setf nrhs var-2))
         (when var-5 (setf ldb$ var-5))
         (when var-8 (setf one var-8))
         (when var-10 (setf ldb$ var-10)))
       (setf l (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
       (if (/= l j)
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
               (cswap nrhs
                (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (l 1)
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$
                (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (j 1)
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$)
             (declare (ignore var-1 var-3))
             (when var-0 (setf nrhs var-0))
             (when var-2 (setf ldb$ var-2))
             (when var-4 (setf ldb$ var-4))))
     label40)))))
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
              ((> i nrhs) nil)
  (tagbody
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (ctbsv "Upper" "Conjugate transpose" "Non-unit" n
         (f2cl-lib:int-add kl ku) ab ldab
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 i)
                               ((1 ldb$) (1 *)) b-%offset%)
         1)
      (declare (ignore var-0 var-1 var-2 var-4 var-5 var-7 var-8))
      (when var-3 (setf n var-3))
      (when var-6 (setf ldab var-6)))
   label50))
(cond
 (lnoti
  (f2cl-lib:fdo (j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                 (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                ((> j 1) nil)
    (tagbody
      (setf lm
              (min (the f2cl-lib:integer4 kl)
                   (the f2cl-lib:integer4 (f2cl-lib:int-sub n j))))
      (multiple-value-bind (var-0 var-1 var-2)
          (clacgv nrhs
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (j 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (cgemv "Conjugate transpose" lm nrhs (- one)
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                 ((+ j 1) 1) ((1 ldb$) (1 *))
                                 b-%offset%)
           ldb$
           (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                 ((+ kd 1) j) ((1 ldab) (1 *))
                                 ab-%offset%)
           1 one
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (j 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-3 var-4 var-6 var-7 var-9))
        (when var-1 (setf lm var-1))
        (when var-2 (setf nrhs var-2))
        (when var-5 (setf ldb$ var-5))
        (when var-8 (setf one var-8))
```

```
                  (when var-10 (setf ldb$ var-10)))
                (multiple-value-bind (var-0 var-1 var-2)
                    (clacgv nrhs
                     (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (j 1)
                                           ((1 ldb$) (1 *)) b-%offset%)
                     ldb$)
                  (declare (ignore var-1))
                  (when var-0 (setf nrhs var-0))
                  (when var-2 (setf ldb$ var-2)))
                (setf l (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
                (if (/= l j)
                    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                        (cswap nrhs
                         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (l 1)
                                               ((1 ldb$) (1 *)) b-%offset%)
                         ldb$
                         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (j 1)
                                               ((1 ldb$) (1 *)) b-%offset%)
                         ldb$)
                      (declare (ignore var-1 var-3))
                      (when var-0 (setf nrhs var-0))
                      (when var-2 (setf ldb$ var-2))
                      (when var-4 (setf ldb$ var-4))))
              label60))))))
       (go end_label)
      end_label
       (return (values trans n nil nil nrhs nil ldab nil nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgbtrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::trans
```

```
                                                  fortran-to-lisp::n nil nil
                                                  fortran-to-lisp::nrhs nil
                                                  fortran-to-lisp::ldab nil nil
                                                  fortran-to-lisp::ldb$
                                                  fortran-to-lisp::info)
                                                :calls 'nil)))
```

———-

46.10 cggbak

46.10.1 Axiom unit tests

— **cggbak.input** —

```
)set break resume
)sys rm -f cggbak.output
)spool cggbak.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

46.10.2 Axiom help page

— **cggbak.help** —

```
=====================================================================
cggbak examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cggbak (JOB, SIDE, N, ILO, IHI, LSCALE, RSCALE, M, V, LDV,
```

```
          INFO)
          CGGBAK

Function/Subroutine Documentation
   subroutine cggbak (character JOB, character SIDE, integer N, integer ILO,
       integer IHI, real, dimension( * ) LSCALE, real, dimension( * ) RSCALE,
       integer M, complex, dimension( ldv, * ) V, integer LDV, integer INFO)
       CGGBAK

Purpose:
   CGGBAK forms the right or left eigenvectors of a complex generalized
   eigenvalue problem A*x = lambda*B*x, by backward transformation on
   the computed eigenvectors of the balanced pair of matrices output by
   CGGBAL.

Parameters:

 JOB
   JOB is CHARACTER*1
   Specifies the type of backward transformation required:
   = 'N':  do nothing, return immediately;
   = 'P':  do backward transformation for permutation only;
   = 'S':  do backward transformation for scaling only;
   = 'B':  do backward transformations for both permutation and
           scaling.
   JOB must be the same as the argument JOB supplied to CGGBAL.

 SIDE
   SIDE is CHARACTER*1
   = 'R':  V contains right eigenvectors;
   = 'L':  V contains left eigenvectors.

 N
   N is INTEGER
   The number of rows of the matrix V.  N >= 0.

 ILO
   ILO is INTEGER

 IHI
   IHI is INTEGER
   The integers ILO and IHI determined by CGGBAL.
   1 <= ILO <= IHI <= N, if N > 0; ILO=1 and IHI=0, if N=0.

 LSCALE
   LSCALE is REAL array, dimension (N)
   Details of the permutations and/or scaling factors applied
   to the left side of A and B, as returned by CGGBAL.

 RSCALE
```

```
  RSCALE is REAL array, dimension (N)
  Details of the permutations and/or scaling factors applied
  to the right side of A and B, as returned by CGGBAL.

M
  M is INTEGER
  The number of columns of the matrix V.  M >= 0.

V
  V is COMPLEX array, dimension (LDV,M)
  On entry, the matrix of right or left eigenvectors to be
  transformed, as returned by CTGEVC.
  On exit, V is overwritten by the transformed eigenvectors.

LDV
  LDV is INTEGER
  The leading dimension of the matrix V. LDV >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

46.10.3 fortran code

— cggbak.f —

```
*  =====================================================================
      SUBROUTINE CGGBAK( JOB, SIDE, N, ILO, IHI, LSCALE, RSCALE, M, V,
     $                   LDV, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOB, SIDE
      INTEGER            IHI, ILO, INFO, LDV, M, N
*     ..
*     .. Array Arguments ..
      REAL               LSCALE( * ), RSCALE( * )
      COMPLEX            V( LDV, * )
*     ..
```

```
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LEFTV, RIGHTV
      INTEGER            I, K
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CSSCAL, CSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      RIGHTV = LSAME( SIDE, 'R' )
      LEFTV = LSAME( SIDE, 'L' )
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.RIGHTV .AND. .NOT.LEFTV ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( ILO.LT.1 ) THEN
         INFO = -4
      ELSE IF( N.EQ.0 .AND. IHI.EQ.0 .AND. ILO.NE.1 ) THEN
         INFO = -4
      ELSE IF( N.GT.0 .AND. ( IHI.LT.ILO .OR. IHI.GT.MAX( 1, N ) ) )
     $   THEN
         INFO = -5
      ELSE IF( N.EQ.0 .AND. ILO.EQ.1 .AND. IHI.NE.0 ) THEN
         INFO = -5
      ELSE IF( M.LT.0 ) THEN
         INFO = -8
      ELSE IF( LDV.LT.MAX( 1, N ) ) THEN
         INFO = -10
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGGBAK', -INFO )
         RETURN
      END IF
```

```
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
      IF( M.EQ.0 )
     $   RETURN
      IF( LSAME( JOB, 'N' ) )
     $   RETURN
*
      IF( ILO.EQ.IHI )
     $   GO TO 30
*
*     Backward balance
*
      IF( LSAME( JOB, 'S' ) .OR. LSAME( JOB, 'B' ) ) THEN
*
*        Backward transformation on right eigenvectors
*
         IF( RIGHTV ) THEN
            DO 10 I = ILO, IHI
               CALL CSSCAL( M, RSCALE( I ), V( I, 1 ), LDV )
   10       CONTINUE
         END IF
*
*        Backward transformation on left eigenvectors
*
         IF( LEFTV ) THEN
            DO 20 I = ILO, IHI
               CALL CSSCAL( M, LSCALE( I ), V( I, 1 ), LDV )
   20       CONTINUE
         END IF
      END IF
*
*     Backward permutation
*
   30 CONTINUE
      IF( LSAME( JOB, 'P' ) .OR. LSAME( JOB, 'B' ) ) THEN
*
*        Backward permutation on right eigenvectors
*
         IF( RIGHTV ) THEN
            IF( ILO.EQ.1 )
     $         GO TO 50
            DO 40 I = ILO - 1, 1, -1
               K = RSCALE( I )
               IF( K.EQ.I )
     $            GO TO 40
               CALL CSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   40       CONTINUE
```

```
*
   50       CONTINUE
            IF( IHI.EQ.N )
     $         GO TO 70
            DO 60 I = IHI + 1, N
               K = RSCALE( I )
               IF( K.EQ.I )
     $            GO TO 60
               CALL CSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   60       CONTINUE
         END IF
*
*        Backward permutation on left eigenvectors
*
   70    CONTINUE
         IF( LEFTV ) THEN
            IF( ILO.EQ.1 )
     $         GO TO 90
            DO 80 I = ILO - 1, 1, -1
               K = LSCALE( I )
               IF( K.EQ.I )
     $            GO TO 80
               CALL CSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   80       CONTINUE
*
   90       CONTINUE
            IF( IHI.EQ.N )
     $         GO TO 110
            DO 100 I = IHI + 1, N
               K = LSCALE( I )
               IF( K.EQ.I )
     $            GO TO 100
               CALL CSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
  100       CONTINUE
         END IF
      END IF
*
  110 CONTINUE
*
      RETURN
*
*     End of CGGBAK
*
      END
```

———-

46.10.4 lisp code

— cggbak.lisp —

```
(defun cggbak (job side n ilo ihi lscale rscale m v ldv info)
  (declare (type (array f2cl-lib:complex8 (*)) v)
           (type (array single-float (*)) rscale lscale)
           (type (f2cl-lib:integer4) info ldv m ihi ilo n)
           (type (string *) side job))
  (f2cl-lib:with-multi-array-data
      ((job character job-%data% job-%offset%)
       (side character side-%data% side-%offset%)
       (lscale single-float lscale-%data% lscale-%offset%)
       (rscale single-float rscale-%data% rscale-%offset%)
       (v f2cl-lib:complex8 v-%data% v-%offset%))
    (prog ((i 0) (k 0) (leftv nil) (rightv nil))
      (declare (type f2cl-lib:logical rightv leftv)
               (type (f2cl-lib:integer4) k i))
      (setf rightv
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame side "R")
                (declare (ignore var-1))
                (when var-0 (setf side var-0))
                ret-val))
      (setf leftv
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame side "L")
                (declare (ignore var-1))
                (when var-0 (setf side var-0))
                ret-val))
      (setf info 0)
      (cond
       ((and
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame job "N")
            (declare (ignore var-1))
            (when var-0 (setf job var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame job "P")
            (declare (ignore var-1))
            (when var-0 (setf job var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame job "S")
            (declare (ignore var-1))
```

```
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "B")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)))
  (setf info -1))
 ((and (not rightv) (not leftv)) (setf info -2)) ((< n 0) (setf info -3))
 ((< ilo 1) (setf info -4))
 ((and (= n 0) (= ihi 0) (/= ilo 1)) (setf info -4))
 ((and (> n 0)
       (or (< ihi ilo)
           (> ihi
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 n)))))
  (setf info -5))
 ((and (= n 0) (= ilo 1) (/= ihi 0)) (setf info -5))
 ((< m 0) (setf info -8))
 ((< ldv (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -10)))
(cond
 ((/= info 0) (xerbla "CGGBAK" (f2cl-lib:int-sub info)) (go end_label)))
(if (= n 0)
    (go end_label))
(if (= m 0)
    (go end_label))
(if (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "N")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)
    (go end_label))
(if (= ilo ihi)
    (go label30))
(cond
 ((or
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame job "S")
     (declare (ignore var-1))
     (when var-0 (setf job var-0))
     ret-val)
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame job "B")
     (declare (ignore var-1))
     (when var-0 (setf job var-0))
     ret-val))
  (cond
   (rightv
```

```
        (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                      ((> i ihi) nil)
          (tagbody
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (csscal m
                 (f2cl-lib:array-slice rscale-%data% single-float (i) ((1 *))
                                       rscale-%offset%)
                 (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (i 1)
                                       ((1 ldv) (1 *)) v-%offset%)
                 ldv)
              (declare (ignore var-1 var-2))
              (when var-0 (setf m var-0))
              (when var-3 (setf ldv var-3)))
           label10))))
      (cond
       (leftv
        (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                      ((> i ihi) nil)
          (tagbody
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (csscal m
                 (f2cl-lib:array-slice lscale-%data% single-float (i) ((1 *))
                                       lscale-%offset%)
                 (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (i 1)
                                       ((1 ldv) (1 *)) v-%offset%)
                 ldv)
              (declare (ignore var-1 var-2))
              (when var-0 (setf m var-0))
              (when var-3 (setf ldv var-3)))
           label20))))))
    label30
     (cond
      ((or
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame job "P")
          (declare (ignore var-1))
          (when var-0 (setf job var-0))
          ret-val)
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame job "B")
          (declare (ignore var-1))
          (when var-0 (setf job var-0))
          ret-val))
       (tagbody
         (cond
          (rightv
           (tagbody
             (if (= ilo 1)
                 (go label50))
             (f2cl-lib:fdo (i (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))
```

```
                        (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                       ((> i 1) nil)
          (tagbody
            (setf k
                    (f2cl-lib:int
                     (f2cl-lib:fref rscale-%data% (i) ((1 *))
                                    rscale-%offset%)))
            (if (= k i)
                (go label40))
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (cswap m
                 (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (i 1)
                                       ((1 ldv) (1 *)) v-%offset%)
                 ldv
                 (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (k 1)
                                       ((1 ldv) (1 *)) v-%offset%)
                 ldv)
              (declare (ignore var-1 var-3))
              (when var-0 (setf m var-0))
              (when var-2 (setf ldv var-2))
              (when var-4 (setf ldv var-4)))
           label40))
        label50
         (if (= ihi n)
             (go label70))
         (f2cl-lib:fdo (i (f2cl-lib:int-add ihi 1) (f2cl-lib:int-add i 1))
                       ((> i n) nil)
          (tagbody
            (setf k
                    (f2cl-lib:int
                     (f2cl-lib:fref rscale-%data% (i) ((1 *))
                                    rscale-%offset%)))
            (if (= k i)
                (go label60))
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (cswap m
                 (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (i 1)
                                       ((1 ldv) (1 *)) v-%offset%)
                 ldv
                 (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (k 1)
                                       ((1 ldv) (1 *)) v-%offset%)
                 ldv)
              (declare (ignore var-1 var-3))
              (when var-0 (setf m var-0))
              (when var-2 (setf ldv var-2))
              (when var-4 (setf ldv var-4)))
           label60)))))
    label70
     (cond
      (leftv
```

```
(tagbody
  (if (= ilo 1)
      (go label90))
  (f2cl-lib:fdo (i (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))
                 (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                ((> i 1) nil)
    (tagbody
      (setf k
              (f2cl-lib:int
               (f2cl-lib:fref lscale-%data% (i) ((1 *))
                              lscale-%offset%)))
      (if (= k i)
          (go label80))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (cswap m
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (i 1)
                                 ((1 ldv) (1 *)) v-%offset%)
           ldv
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (k 1)
                                 ((1 ldv) (1 *)) v-%offset%)
           ldv)
        (declare (ignore var-1 var-3))
        (when var-0 (setf m var-0))
        (when var-2 (setf ldv var-2))
        (when var-4 (setf ldv var-4)))
    label80))
 label90
  (if (= ihi n)
      (go label110))
  (f2cl-lib:fdo (i (f2cl-lib:int-add ihi 1) (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf k
              (f2cl-lib:int
               (f2cl-lib:fref lscale-%data% (i) ((1 *))
                              lscale-%offset%)))
      (if (= k i)
          (go label100))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (cswap m
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (i 1)
                                 ((1 ldv) (1 *)) v-%offset%)
           ldv
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex8 (k 1)
                                 ((1 ldv) (1 *)) v-%offset%)
           ldv)
        (declare (ignore var-1 var-3))
        (when var-0 (setf m var-0))
        (when var-2 (setf ldv var-2))
        (when var-4 (setf ldv var-4)))
```

```
                label100))))))))
     label110
      (go end_label)
     end_label
      (return (values job side nil nil nil nil nil m nil ldv info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cggbak
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::job
                                              fortran-to-lisp::side nil nil nil
                                              nil nil fortran-to-lisp::m nil
                                              fortran-to-lisp::ldv
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

46.11 cggbal

46.11.1 Axiom unit tests

— cggbal.input —

```
)set break resume
)sys rm -f cggbal.output
)spool cggbal.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

46.11.2 Axiom help page

— **cggbal.help** —

```
==================================================================
cggbal examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cggbal (JOB, N, A, LDA, B, LDB, ILO, IHI, LSCALE, RSCALE,
           WORK, INFO)
           CGGBAL

Function/Subroutine Documentation
   subroutine cggbal (character JOB, integer N, complex, dimension( lda, * )
       A, integer LDA, complex, dimension( ldb, * ) B, integer LDB, integer
       ILO, integer IHI, real, dimension( * ) LSCALE, real, dimension( * )
       RSCALE, real, dimension( * ) WORK, integer INFO)
       CGGBAL

Purpose:
   CGGBAL balances a pair of general complex matrices (A,B).  This
   involves, first, permuting A and B by similarity transformations to
   isolate eigenvalues in the first 1 to ILO$-$1 and last IHI+1 to N
   elements on the diagonal; and second, applying a diagonal similarity
   transformation to rows and columns ILO to IHI to make the rows
   and columns as close in norm as possible. Both steps are optional.

   Balancing may reduce the 1-norm of the matrices, and improve the
   accuracy of the computed eigenvalues and/or eigenvectors in the
   generalized eigenvalue problem A*x = lambda*B*x.

Parameters:

 JOB
   JOB is CHARACTER*1
   Specifies the operations to be performed on A and B:
```

```
  = 'N':  none:  simply set ILO = 1, IHI = N, LSCALE(I) = 1.0
          and RSCALE(I) = 1.0 for i=1,...,N;
  = 'P':  permute only;
  = 'S':  scale only;
  = 'B':  both permute and scale.

N
  N is INTEGER
  The order of the matrices A and B.  N >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the input matrix A.
  On exit, A is overwritten by the balanced matrix.
  If JOB = 'N', A is not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A. LDA >= max(1,N).

B
  B is COMPLEX array, dimension (LDB,N)
  On entry, the input matrix B.
  On exit, B is overwritten by the balanced matrix.
  If JOB = 'N', B is not referenced.

LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= max(1,N).

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER
  ILO and IHI are set to integers such that on exit
  A(i,j) = 0 and B(i,j) = 0 if i > j and
  j = 1,...,ILO-1 or i = IHI+1,...,N.
  If JOB = 'N' or 'S', ILO = 1 and IHI = N.

LSCALE
  LSCALE is REAL array, dimension (N)
  Details of the permutations and scaling factors applied
  to the left side of A and B.  If P(j) is the index of the
  row interchanged with row j, and D(j) is the scaling factor
  applied to row j, then
  LSCALE(j) = P(j)    for J = 1,...,ILO-1
            = D(j)    for J = ILO,...,IHI
            = P(j)    for J = IHI+1,...,N.
  The order in which the interchanges are made is N to IHI+1,
```

```
  then 1 to ILO-1.

RSCALE
  RSCALE is REAL array, dimension (N)
  Details of the permutations and scaling factors applied
  to the right side of A and B.  If P(j) is the index of the
  column interchanged with column j, and D(j) is the scaling
  factor applied to column j, then
  RSCALE(j) = P(j)    for J = 1,...,ILO-1
            = D(j)    for J = ILO,...,IHI
            = P(j)    for J = IHI+1,...,N.
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

WORK
  WORK is REAL array, dimension (lwork)
  lwork must be at least max(1,6*N) when JOB = 'S' or 'B', and
  at least 1 when JOB = 'N' or 'P'.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

———-

46.11.3 fortran code

— cggbal.f —

```
*  =====================================================================
      SUBROUTINE CGGBAL( JOB, N, A, LDA, B, LDB, ILO, IHI, LSCALE,
     $                   RSCALE, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOB
      INTEGER            IHI, ILO, INFO, LDA, LDB, N
*     ..
*     .. Array Arguments ..
      REAL               LSCALE( * ), RSCALE( * ), WORK( * )
      COMPLEX            A( LDA, * ), B( LDB, * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, HALF, ONE
      PARAMETER          ( ZERO = 0.0E+0, HALF = 0.5E+0, ONE = 1.0E+0 )
      REAL               THREE, SCLFAC
      PARAMETER          ( THREE = 3.0E+0, SCLFAC = 1.0E+1 )
      COMPLEX            CZERO
      PARAMETER          ( CZERO = ( 0.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, ICAB, IFLOW, IP1, IR, IRAB, IT, J, JC, JP1,
     $                   K, KOUNT, L, LCAB, LM1, LRAB, LSFMAX, LSFMIN,
     $                   M, NR, NRP2
      REAL               ALPHA, BASL, BETA, CAB, CMAX, COEF, COEF2,
     $                   COEF5, COR, EW, EWC, GAMMA, PGAMMA, RAB, SFMAX,
     $                   SFMIN, SUM, T, TA, TB, TC
      COMPLEX            CDUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ICAMAX
      REAL               SDOT, SLAMCH
      EXTERNAL           LSAME, ICAMAX, SDOT, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           CSSCAL, CSWAP, SAXPY, SSCAL, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, INT, LOG10, MAX, MIN, REAL, SIGN
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( CDUM ) = ABS( REAL( CDUM ) ) + ABS( AIMAG( CDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
```

```
         INFO = -4
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGGBAL', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         ILO = 1
         IHI = N
         RETURN
      END IF
*
      IF( N.EQ.1 ) THEN
         ILO = 1
         IHI = N
         LSCALE( 1 ) = ONE
         RSCALE( 1 ) = ONE
         RETURN
      END IF
*
      IF( LSAME( JOB, 'N' ) ) THEN
         ILO = 1
         IHI = N
         DO 10 I = 1, N
            LSCALE( I ) = ONE
            RSCALE( I ) = ONE
   10    CONTINUE
         RETURN
      END IF
*
      K = 1
      L = N
      IF( LSAME( JOB, 'S' ) )
     $   GO TO 190
*
      GO TO 30
*
*     Permute the matrices A and B to isolate the eigenvalues.
*
*     Find row with one nonzero in columns 1 through L
*
   20 CONTINUE
      L = LM1
      IF( L.NE.1 )
     $   GO TO 30
```

```
*
      RSCALE( 1 ) = ONE
      LSCALE( 1 ) = ONE
      GO TO 190
*
   30 CONTINUE
      LM1 = L - 1
      DO 80 I = L, 1, -1
         DO 40 J = 1, LM1
            JP1 = J + 1
            IF( A( I, J ).NE.CZERO .OR. B( I, J ).NE.CZERO )
     $         GO TO 50
   40    CONTINUE
         J = L
         GO TO 70
*
   50    CONTINUE
         DO 60 J = JP1, L
            IF( A( I, J ).NE.CZERO .OR. B( I, J ).NE.CZERO )
     $         GO TO 80
   60    CONTINUE
         J = JP1 - 1
*
   70    CONTINUE
         M = L
         IFLOW = 1
         GO TO 160
   80 CONTINUE
      GO TO 100
*
*     Find column with one nonzero in rows K through N
*
   90 CONTINUE
      K = K + 1
*
  100 CONTINUE
      DO 150 J = K, L
         DO 110 I = K, LM1
            IP1 = I + 1
            IF( A( I, J ).NE.CZERO .OR. B( I, J ).NE.CZERO )
     $         GO TO 120
  110    CONTINUE
         I = L
         GO TO 140
  120    CONTINUE
         DO 130 I = IP1, L
            IF( A( I, J ).NE.CZERO .OR. B( I, J ).NE.CZERO )
     $         GO TO 150
  130    CONTINUE
         I = IP1 - 1
```

```
  140    CONTINUE
         M = K
         IFLOW = 2
         GO TO 160
  150 CONTINUE
      GO TO 190
*
*     Permute rows M and I
*
  160 CONTINUE
      LSCALE( M ) = I
      IF( I.EQ.M )
     $   GO TO 170
      CALL CSWAP( N-K+1, A( I, K ), LDA, A( M, K ), LDA )
      CALL CSWAP( N-K+1, B( I, K ), LDB, B( M, K ), LDB )
*
*     Permute columns M and J
*
  170 CONTINUE
      RSCALE( M ) = J
      IF( J.EQ.M )
     $   GO TO 180
      CALL CSWAP( L, A( 1, J ), 1, A( 1, M ), 1 )
      CALL CSWAP( L, B( 1, J ), 1, B( 1, M ), 1 )
*
  180 CONTINUE
      GO TO ( 20, 90 )IFLOW
*
  190 CONTINUE
      ILO = K
      IHI = L
*
      IF( LSAME( JOB, 'P' ) ) THEN
         DO 195 I = ILO, IHI
            LSCALE( I ) = ONE
            RSCALE( I ) = ONE
  195    CONTINUE
         RETURN
      END IF
*
      IF( ILO.EQ.IHI )
     $   RETURN
*
*     Balance the submatrix in rows ILO to IHI.
*
      NR = IHI - ILO + 1
      DO 200 I = ILO, IHI
         RSCALE( I ) = ZERO
         LSCALE( I ) = ZERO
*
```

```
         WORK( I ) = ZERO
         WORK( I+N ) = ZERO
         WORK( I+2*N ) = ZERO
         WORK( I+3*N ) = ZERO
         WORK( I+4*N ) = ZERO
         WORK( I+5*N ) = ZERO
  200 CONTINUE
*
*     Compute right side vector in resulting linear equations
*
      BASL = LOG10( SCLFAC )
      DO 240 I = ILO, IHI
         DO 230 J = ILO, IHI
            IF( A( I, J ).EQ.CZERO ) THEN
               TA = ZERO
               GO TO 210
            END IF
            TA = LOG10( CABS1( A( I, J ) ) ) / BASL
*
  210       CONTINUE
            IF( B( I, J ).EQ.CZERO ) THEN
               TB = ZERO
               GO TO 220
            END IF
            TB = LOG10( CABS1( B( I, J ) ) ) / BASL
*
  220       CONTINUE
            WORK( I+4*N ) = WORK( I+4*N ) - TA - TB
            WORK( J+5*N ) = WORK( J+5*N ) - TA - TB
  230    CONTINUE
  240 CONTINUE
*
      COEF = ONE / REAL( 2*NR )
      COEF2 = COEF*COEF
      COEF5 = HALF*COEF2
      NRP2 = NR + 2
      BETA = ZERO
      IT = 1
*
*     Start generalized conjugate gradient iteration
*
  250 CONTINUE
*
      GAMMA = SDOT( NR, WORK( ILO+4*N ), 1, WORK( ILO+4*N ), 1 ) +
     $        SDOT( NR, WORK( ILO+5*N ), 1, WORK( ILO+5*N ), 1 )
*
      EW = ZERO
      EWC = ZERO
      DO 260 I = ILO, IHI
         EW = EW + WORK( I+4*N )
```

```
         EWC = EWC + WORK( I+5*N )
  260 CONTINUE
*
      GAMMA = COEF*GAMMA - COEF2*( EW**2+EWC**2 ) - COEF5*( EW-EWC )**2
      IF( GAMMA.EQ.ZERO )
     $   GO TO 350
      IF( IT.NE.1 )
     $   BETA = GAMMA / PGAMMA
      T = COEF5*( EWC-THREE*EW )
      TC = COEF5*( EW-THREE*EWC )
*
      CALL SSCAL( NR, BETA, WORK( ILO ), 1 )
      CALL SSCAL( NR, BETA, WORK( ILO+N ), 1 )
*
      CALL SAXPY( NR, COEF, WORK( ILO+4*N ), 1, WORK( ILO+N ), 1 )
      CALL SAXPY( NR, COEF, WORK( ILO+5*N ), 1, WORK( ILO ), 1 )
*
      DO 270 I = ILO, IHI
         WORK( I ) = WORK( I ) + TC
         WORK( I+N ) = WORK( I+N ) + T
  270 CONTINUE
*
*     Apply matrix to vector
*
      DO 300 I = ILO, IHI
         KOUNT = 0
         SUM = ZERO
         DO 290 J = ILO, IHI
            IF( A( I, J ).EQ.CZERO )
     $         GO TO 280
            KOUNT = KOUNT + 1
            SUM = SUM + WORK( J )
  280       CONTINUE
            IF( B( I, J ).EQ.CZERO )
     $         GO TO 290
            KOUNT = KOUNT + 1
            SUM = SUM + WORK( J )
  290    CONTINUE
         WORK( I+2*N ) = REAL( KOUNT )*WORK( I+N ) + SUM
  300 CONTINUE
*
      DO 330 J = ILO, IHI
         KOUNT = 0
         SUM = ZERO
         DO 320 I = ILO, IHI
            IF( A( I, J ).EQ.CZERO )
     $         GO TO 310
            KOUNT = KOUNT + 1
            SUM = SUM + WORK( I+N )
  310       CONTINUE
```

```
            IF( B( I, J ).EQ.CZERO )
     $         GO TO 320
            KOUNT = KOUNT + 1
            SUM = SUM + WORK( I+N )
  320    CONTINUE
         WORK( J+3*N ) = REAL( KOUNT )*WORK( J ) + SUM
  330 CONTINUE
*
      SUM = SDOT( NR, WORK( ILO+N ), 1, WORK( ILO+2*N ), 1 ) +
     $      SDOT( NR, WORK( ILO ), 1, WORK( ILO+3*N ), 1 )
      ALPHA = GAMMA / SUM
*
*     Determine correction to current iteration
*
      CMAX = ZERO
      DO 340 I = ILO, IHI
         COR = ALPHA*WORK( I+N )
         IF( ABS( COR ).GT.CMAX )
     $      CMAX = ABS( COR )
         LSCALE( I ) = LSCALE( I ) + COR
         COR = ALPHA*WORK( I )
         IF( ABS( COR ).GT.CMAX )
     $      CMAX = ABS( COR )
         RSCALE( I ) = RSCALE( I ) + COR
  340 CONTINUE
      IF( CMAX.LT.HALF )
     $   GO TO 350
*
      CALL SAXPY( NR, -ALPHA, WORK( ILO+2*N ), 1, WORK( ILO+4*N ), 1 )
      CALL SAXPY( NR, -ALPHA, WORK( ILO+3*N ), 1, WORK( ILO+5*N ), 1 )
*
      PGAMMA = GAMMA
      IT = IT + 1
      IF( IT.LE.NRP2 )
     $   GO TO 250
*
*     End generalized conjugate gradient iteration
*
  350 CONTINUE
      SFMIN = SLAMCH( 'S' )
      SFMAX = ONE / SFMIN
      LSFMIN = INT( LOG10( SFMIN ) / BASL+ONE )
      LSFMAX = INT( LOG10( SFMAX ) / BASL )
      DO 360 I = ILO, IHI
         IRAB = ICAMAX( N-ILO+1, A( I, ILO ), LDA )
         RAB = ABS( A( I, IRAB+ILO-1 ) )
         IRAB = ICAMAX( N-ILO+1, B( I, ILO ), LDB )
         RAB = MAX( RAB, ABS( B( I, IRAB+ILO-1 ) ) )
         LRAB = INT( LOG10( RAB+SFMIN ) / BASL+ONE )
         IR = LSCALE( I ) + SIGN( HALF, LSCALE( I ) )
```

```
         IR = MIN( MAX( IR, LSFMIN ), LSFMAX, LSFMAX-LRAB )
         LSCALE( I ) = SCLFAC**IR
         ICAB = ICAMAX( IHI, A( 1, I ), 1 )
         CAB = ABS( A( ICAB, I ) )
         ICAB = ICAMAX( IHI, B( 1, I ), 1 )
         CAB = MAX( CAB, ABS( B( ICAB, I ) ) )
         LCAB = INT( LOG10( CAB+SFMIN ) / BASL+ONE )
         JC = RSCALE( I ) + SIGN( HALF, RSCALE( I ) )
         JC = MIN( MAX( JC, LSFMIN ), LSFMAX, LSFMAX-LCAB )
         RSCALE( I ) = SCLFAC**JC
  360 CONTINUE
*
*     Row scaling of matrices A and B
*
      DO 370 I = ILO, IHI
         CALL CSSCAL( N-ILO+1, LSCALE( I ), A( I, ILO ), LDA )
         CALL CSSCAL( N-ILO+1, LSCALE( I ), B( I, ILO ), LDB )
  370 CONTINUE
*
*     Column scaling of matrices A and B
*
      DO 380 J = ILO, IHI
         CALL CSSCAL( IHI, RSCALE( J ), A( 1, J ), 1 )
         CALL CSSCAL( IHI, RSCALE( J ), B( 1, J ), 1 )
  380 CONTINUE
*
      RETURN
*
*     End of CGGBAL
*
      END
```

46.11.4 lisp code

— cggbal.lisp —

```
(let* ((zero 0.0)
       (half 0.5)
       (one 1.0)
       (three 3.0)
       (sclfac 10.0)
       (czero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 0.5 0.5) half)
           (type (single-float 1.0 1.0) one)
           (type (single-float 3.0 3.0) three)
```

```
          (type (single-float 10.0 10.0) sclfac)
          (type (f2cl-lib:complex8) czero)
          (ignorable zero half one three sclfac czero))
 (defun cggbal (job n a lda b ldb$ ilo ihi lscale rscale work info)
   (declare (type (array single-float (*)) work rscale lscale)
            (type (array f2cl-lib:complex8 (*)) b a)
            (type (f2cl-lib:integer4) info ihi ilo ldb$ lda n)
            (type (string *) job))
   (f2cl-lib:with-multi-array-data
       ((job character job-%data% job-%offset%)
        (a f2cl-lib:complex8 a-%data% a-%offset%)
        (b f2cl-lib:complex8 b-%data% b-%offset%)
        (lscale single-float lscale-%data% lscale-%offset%)
        (rscale single-float rscale-%data% rscale-%offset%)
        (work single-float work-%data% work-%offset%))
     (labels ((cabs1 (cdum)
                (+ (abs (f2cl-lib:freal cdum)) (abs (f2cl-lib:aimag cdum)))))
       (declare
        (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
         cabs1))
       (prog ((cdum #C(0.0 0.0)) (alpha 0.0) (basl 0.0) (beta 0.0) (cab 0.0)
              (cmax 0.0) (coef 0.0) (coef2 0.0) (coef5 0.0) (cor 0.0) (ew 0.0)
              (ewc 0.0) (gamma 0.0) (pgamma 0.0) (rab 0.0) (sfmax 0.0)
              (sfmin 0.0) (sum 0.0) (t$ 0.0) (ta 0.0) (tb 0.0) (tc 0.0) (i 0)
              (icab 0) (iflow 0) (ip1 0) (ir 0) (irab 0) (it 0) (j 0) (jc 0)
              (jp1 0) (k 0) (kount 0) (l 0) (lcab 0) (lm1 0) (lrab 0)
              (lsfmax 0) (lsfmin 0) (m 0) (nr 0) (nrp2 0))
         (declare (type (f2cl-lib:complex8) cdum)
                  (type (single-float) alpha basl beta cab cmax coef coef2
                   coef5 cor ew ewc gamma pgamma rab sfmax sfmin sum t$ ta tb
                   tc)
                  (type (f2cl-lib:integer4) i icab iflow ip1 ir irab it j jc
                   jp1 k kount l lcab lm1 lrab lsfmax lsfmin m nr nrp2))
         (setf info 0)
         (cond
          ((and
            (not
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame job "N")
               (declare (ignore var-1))
               (when var-0 (setf job var-0))
               ret-val))
            (not
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame job "P")
               (declare (ignore var-1))
               (when var-0 (setf job var-0))
               ret-val))
            (not
             (multiple-value-bind (ret-val var-0 var-1)
```

```
          (lsame job "S")
        (declare (ignore var-1))
        (when var-0 (setf job var-0))
        ret-val))
     (not
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame job "B")
        (declare (ignore var-1))
        (when var-0 (setf job var-0))
        ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4))
   ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -6)))
  (cond
   ((/= info 0) (xerbla "CGGBAL" (f2cl-lib:int-sub info))
    (go end_label)))
  (cond ((= n 0) (setf ilo 1) (setf ihi n) (go end_label)))
  (cond
   ((= n 1) (setf ilo 1) (setf ihi n)
    (setf (f2cl-lib:fref lscale-%data% (1) ((1 *)) lscale-%offset%)
            one)
    (setf (f2cl-lib:fref rscale-%data% (1) ((1 *)) rscale-%offset%)
            one)
    (go end_label)))
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame job "N")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)
    (setf ilo 1) (setf ihi n)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
                one)
        (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
                one)
       label10))
    (go end_label)))
  (setf k 1)
  (setf l n)
  (if (multiple-value-bind (ret-val var-0 var-1)
          (lsame job "S")
        (declare (ignore var-1))
        (when var-0 (setf job var-0))
        ret-val)
```

```
      (go label190))
  (go label30)
 label20
  (setf l lm1)
  (if (/= l 1)
      (go label30))
  (setf (f2cl-lib:fref rscale-%data% (1) ((1 *)) rscale-%offset%) one)
  (setf (f2cl-lib:fref lscale-%data% (1) ((1 *)) lscale-%offset%) one)
  (go label190)
 label30
  (setf lm1 (f2cl-lib:int-sub l 1))
  (f2cl-lib:fdo (i l (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                ((> i 1) nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j lm1) nil)
        (tagbody
          (setf jp1 (f2cl-lib:int-add j 1))
          (if (or
               (/=
                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%)
                czero)
               (/=
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)
                czero))
              (go label50))
         label40))
      (setf j l)
      (go label70)
     label50
      (f2cl-lib:fdo (j jp1 (f2cl-lib:int-add j 1))
                    ((> j l) nil)
        (tagbody
          (if (or
               (/=
                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%)
                czero)
               (/=
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)
                czero))
              (go label80))
         label60))
      (setf j (f2cl-lib:int-sub jp1 1))
     label70
      (setf m l)
      (setf iflow 1)
```

```
      (go label160)
     label80))
  (go label100)
 label90
  (setf k (f2cl-lib:int-add k 1))
 label100
  (f2cl-lib:fdo (j k (f2cl-lib:int-add j 1))
                ((> j l) nil)
    (tagbody
      (f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
                    ((> i lm1) nil)
        (tagbody
          (setf ip1 (f2cl-lib:int-add i 1))
          (if (or
               (/=
                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%)
                czero)
               (/=
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)
                czero))
              (go label120))
         label110))
      (setf i l)
      (go label140)
     label120
      (f2cl-lib:fdo (i ip1 (f2cl-lib:int-add i 1))
                    ((> i l) nil)
        (tagbody
          (if (or
               (/=
                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%)
                czero)
               (/=
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)
                czero))
              (go label150))
         label130))
      (setf i (f2cl-lib:int-sub ip1 1))
     label140
      (setf m k)
      (setf iflow 2)
      (go label160)
     label150))
  (go label190)
 label160
  (setf (f2cl-lib:fref lscale-%data% (m) ((1 *)) lscale-%offset%)
```

```
        (coerce (the f2cl-lib:integer4 i) 'single-float))
 (if (= i m)
     (go label170))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (cswap (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i k)
                            ((1 lda) (1 *)) a-%offset%)
      lda
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (m k)
                            ((1 lda) (1 *)) a-%offset%)
      lda)
   (declare (ignore var-0 var-1 var-3))
   (when var-2 (setf lda var-2))
   (when var-4 (setf lda var-4)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (cswap (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (i k)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (m k)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
   (declare (ignore var-0 var-1 var-3))
   (when var-2 (setf ldb$ var-2))
   (when var-4 (setf ldb$ var-4)))
label170
 (setf (f2cl-lib:fref rscale-%data% (m) ((1 *)) rscale-%offset%)
        (coerce (the f2cl-lib:integer4 j) 'single-float))
 (if (= j m)
     (go label180))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (cswap l
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 j)
                            ((1 lda) (1 *)) a-%offset%)
      1
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 m)
                            ((1 lda) (1 *)) a-%offset%)
      1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf l var-0)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (cswap l
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 j)
                            ((1 ldb$) (1 *)) b-%offset%)
      1
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 m)
                            ((1 ldb$) (1 *)) b-%offset%)
      1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf l var-0)))
```

```
label180
 (f2cl-lib:computed-goto (label20 label90) iflow)
label190
 (setf ilo k)
 (setf ihi l)
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame job "P")
     (declare (ignore var-1))
     (when var-0 (setf job var-0))
     ret-val)
   (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                 ((> i ihi) nil)
     (tagbody
       (setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
               one)
       (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
               one)
      label195))
   (go end_label)))
 (if (= ilo ihi)
     (go end_label))
 (setf nr (f2cl-lib:int-add (f2cl-lib:int-sub ihi ilo) 1))
 (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
               ((> i ihi) nil)
   (tagbody
     (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
             zero)
     (setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
             zero)
     (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) zero)
     (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n)) ((1 *))
                          work-%offset%)
             zero)
     (setf (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add i
                                             (f2cl-lib:int-mul 2 n)))
                          ((1 *)) work-%offset%)
             zero)
     (setf (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add i
                                             (f2cl-lib:int-mul 3 n)))
                          ((1 *)) work-%offset%)
             zero)
     (setf (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add i
                                             (f2cl-lib:int-mul 4 n)))
                          ((1 *)) work-%offset%)
             zero)
     (setf (f2cl-lib:fref work-%data%
```

```
                              ((f2cl-lib:int-add i
                                                 (f2cl-lib:int-mul 5 n)))
                              ((1 *)) work-%offset%)
                 zero)
      label200))
    (setf basl (f2cl-lib:log10 sclfac))
    (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                  ((> i ihi) nil)
      (tagbody
        (f2cl-lib:fdo (j ilo (f2cl-lib:int-add j 1))
                      ((> j ihi) nil)
          (tagbody
            (cond
             ((= (f2cl-lib:fref a (i j) ((1 lda) (1 *))) czero)
              (setf ta zero) (go label210)))
            (setf ta
                    (/
                     (f2cl-lib:log10
                      (cabs1
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)))
                     basl))
           label210
            (cond
             ((= (f2cl-lib:fref b (i j) ((1 ldb$) (1 *))) czero)
              (setf tb zero) (go label220)))
            (setf tb
                    (/
                     (f2cl-lib:log10
                      (cabs1
                       (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                      b-%offset%)))
                     basl))
           label220
            (setf (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add i
                                                    (f2cl-lib:int-mul 4
                                                                      n)))
                                 ((1 *)) work-%offset%)
                    (-
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add i
                                                       (f2cl-lib:int-mul
                                                        4 n)))
                                    ((1 *)) work-%offset%)
                     ta tb))
            (setf (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add j
                                                    (f2cl-lib:int-mul 5
                                                                      n)))
```

```
                                ((1 *)) work-%offset%)
                     (-
                      (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-add j
                                                        (f2cl-lib:int-mul
                                                         5 n)))
                                     ((1 *)) work-%offset%)
                      ta tb))
            label230))
        label240))
     (setf coef (/ one (f2cl-lib:freal (f2cl-lib:int-mul 2 nr))))
     (setf coef2 (* coef coef))
     (setf coef5 (* half coef2))
     (setf nrp2 (f2cl-lib:int-add nr 2))
     (setf beta zero)
     (setf it 1)
    label250
     (setf gamma
             (+
              (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
                  (sdot nr
                   (f2cl-lib:array-slice work-%data% single-float
                                         ((+ ilo (f2cl-lib:int-mul 4 n)))
                                         ((1 *)) work-%offset%)
                   1
                   (f2cl-lib:array-slice work-%data% single-float
                                         ((+ ilo (f2cl-lib:int-mul 4 n)))
                                         ((1 *)) work-%offset%)
                   1)
                (declare (ignore var-1 var-2 var-3 var-4))
                (when var-0 (setf nr var-0))
                ret-val)
              (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
                  (sdot nr
                   (f2cl-lib:array-slice work-%data% single-float
                                         ((+ ilo (f2cl-lib:int-mul 5 n)))
                                         ((1 *)) work-%offset%)
                   1
                   (f2cl-lib:array-slice work-%data% single-float
                                         ((+ ilo (f2cl-lib:int-mul 5 n)))
                                         ((1 *)) work-%offset%)
                   1)
                (declare (ignore var-1 var-2 var-3 var-4))
                (when var-0 (setf nr var-0))
                ret-val)))
     (setf ew zero)
     (setf ewc zero)
     (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                   ((> i ihi) nil)
       (tagbody
```

```
    (setf ew
            (+ ew
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add i
                                                 (f2cl-lib:int-mul 4
                                                                   n)))
                              ((1 *)) work-%offset%)))
    (setf ewc
            (+ ewc
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add i
                                                 (f2cl-lib:int-mul 5
                                                                   n)))
                              ((1 *)) work-%offset%)))
   label260))
(setf gamma
        (- (* coef gamma) (* coef2 (+ (expt ew 2) (expt ewc 2)))
           (* coef5 (expt (- ew ewc) 2))))
(if (= gamma zero)
    (go label350))
(if (/= it 1)
    (setf beta (/ gamma pgamma)))
(setf t$ (* coef5 (- ewc (* three ew))))
(setf tc (* coef5 (- ew (* three ewc))))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (sscal nr beta
     (f2cl-lib:array-slice work-%data% single-float (ilo) ((1 *))
                           work-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf nr var-0))
  (when var-1 (setf beta var-1)))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (sscal nr beta
     (f2cl-lib:array-slice work-%data% single-float ((+ ilo n))
                           ((1 *)) work-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf nr var-0))
  (when var-1 (setf beta var-1)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (saxpy nr coef
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ ilo (f2cl-lib:int-mul 4 n))) ((1 *))
                           work-%offset%)
     1
     (f2cl-lib:array-slice work-%data% single-float ((+ ilo n))
                           ((1 *)) work-%offset%)
     1)
  (declare (ignore var-2 var-3 var-4 var-5))
```

```
  (when var-0 (setf nr var-0))
  (when var-1 (setf coef var-1)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (saxpy nr coef
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ ilo (f2cl-lib:int-mul 5 n))) ((1 *))
                           work-%offset%)
     1
     (f2cl-lib:array-slice work-%data% single-float (ilo) ((1 *))
                           work-%offset%)
     1)
  (declare (ignore var-2 var-3 var-4 var-5))
  (when var-0 (setf nr var-0))
  (when var-1 (setf coef var-1)))
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i ihi) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (+ (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               tc))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n)) ((1 *))
                         work-%offset%)
            (+
             (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n))
                            ((1 *)) work-%offset%)
             t$))
   label270))
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i ihi) nil)
  (tagbody
    (setf kount 0)
    (setf sum zero)
    (f2cl-lib:fdo (j ilo (f2cl-lib:int-add j 1))
                  ((> j ihi) nil)
      (tagbody
        (if (=
             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                            a-%offset%)
             czero)
            (go label280))
        (setf kount (f2cl-lib:int-add kount 1))
        (setf sum
                (+ sum
                   (f2cl-lib:fref work-%data% (j) ((1 *))
                                  work-%offset%)))
       label280
        (if (=
             (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                            b-%offset%)
             czero)
```

```
                (go label290))
            (setf kount (f2cl-lib:int-add kount 1))
            (setf sum
                    (+ sum
                       (f2cl-lib:fref work-%data% (j) ((1 *))
                                      work-%offset%)))
          label290))
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add i
                                                (f2cl-lib:int-mul 2 n)))
                             ((1 *)) work-%offset%)
                (+
                 (* (f2cl-lib:freal kount)
                    (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n))
                                   ((1 *)) work-%offset%))
                 sum))
      label300))
    (f2cl-lib:fdo (j ilo (f2cl-lib:int-add j 1))
                  ((> j ihi) nil)
      (tagbody
        (setf kount 0)
        (setf sum zero)
        (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                      ((> i ihi) nil)
          (tagbody
            (if (=
                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                 czero)
                (go label310))
            (setf kount (f2cl-lib:int-add kount 1))
            (setf sum
                    (+ sum
                       (f2cl-lib:fref work-%data%
                                      ((f2cl-lib:int-add i n)) ((1 *))
                                      work-%offset%)))
          label310
            (if (=
                 (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                 czero)
                (go label320))
            (setf kount (f2cl-lib:int-add kount 1))
            (setf sum
                    (+ sum
                       (f2cl-lib:fref work-%data%
                                      ((f2cl-lib:int-add i n)) ((1 *))
                                      work-%offset%)))
          label320))
        (setf (f2cl-lib:fref work-%data%
```

```
                          ((f2cl-lib:int-add j
                                             (f2cl-lib:int-mul 3 n)))
                          ((1 *)) work-%offset%)
                (+
                 (* (f2cl-lib:freal kount)
                    (f2cl-lib:fref work-%data% (j) ((1 *))
                                   work-%offset%))
                 sum))
      label330))
    (setf sum
            (+
             (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
                 (sdot nr
                  (f2cl-lib:array-slice work-%data% single-float
                                        ((+ ilo n)) ((1 *))
                                        work-%offset%)
                  1
                  (f2cl-lib:array-slice work-%data% single-float
                                        ((+ ilo (f2cl-lib:int-mul 2 n)))
                                        ((1 *)) work-%offset%)
                  1)
               (declare (ignore var-1 var-2 var-3 var-4))
               (when var-0 (setf nr var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
                 (sdot nr
                  (f2cl-lib:array-slice work-%data% single-float (ilo)
                                        ((1 *)) work-%offset%)
                  1
                  (f2cl-lib:array-slice work-%data% single-float
                                        ((+ ilo (f2cl-lib:int-mul 3 n)))
                                        ((1 *)) work-%offset%)
                  1)
               (declare (ignore var-1 var-2 var-3 var-4))
               (when var-0 (setf nr var-0))
               ret-val)))
    (setf alpha (/ gamma sum))
    (setf cmax zero)
    (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                  ((> i ihi) nil)
      (tagbody
        (setf cor
                (* alpha
                   (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n))
                                  ((1 *)) work-%offset%)))
        (if (> (abs cor) cmax)
            (setf cmax (abs cor)))
        (setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
                (+
                 (f2cl-lib:fref lscale-%data% (i) ((1 *))
```

```
                                lscale-%offset%)
                 cor))
        (setf cor
                (* alpha
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)))
        (if (> (abs cor) cmax)
            (setf cmax (abs cor)))
        (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
                (+
                 (f2cl-lib:fref rscale-%data% (i) ((1 *))
                                rscale-%offset%)
                 cor))
       label340))
    (if (< cmax half)
        (go label350))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (saxpy nr (- alpha)
         (f2cl-lib:array-slice work-%data% single-float
                               ((+ ilo (f2cl-lib:int-mul 2 n))) ((1 *))
                               work-%offset%)
         1
         (f2cl-lib:array-slice work-%data% single-float
                               ((+ ilo (f2cl-lib:int-mul 4 n))) ((1 *))
                               work-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4 var-5))
      (when var-0 (setf nr var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (saxpy nr (- alpha)
         (f2cl-lib:array-slice work-%data% single-float
                               ((+ ilo (f2cl-lib:int-mul 3 n))) ((1 *))
                               work-%offset%)
         1
         (f2cl-lib:array-slice work-%data% single-float
                               ((+ ilo (f2cl-lib:int-mul 5 n))) ((1 *))
                               work-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4 var-5))
      (when var-0 (setf nr var-0)))
    (setf pgamma gamma)
    (setf it (f2cl-lib:int-add it 1))
    (if (<= it nrp2)
        (go label250))
   label350
    (setf sfmin (slamch "S"))
    (setf sfmax (/ one sfmin))
    (setf lsfmin (f2cl-lib:int (+ (/ (f2cl-lib:log10 sfmin) basl) one)))
    (setf lsfmax (f2cl-lib:int (/ (f2cl-lib:log10 sfmax) basl)))
    (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
```

```
            ((> i ihi) nil)
  (tagbody
    (setf irab
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (icamax (f2cl-lib:int-add (f2cl-lib:int-sub n ilo) 1)
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                       (i ilo) ((1 lda) (1 *))
                                       a-%offset%)
                 lda)
              (declare (ignore var-0 var-1))
              (when var-2 (setf lda var-2))
              ret-val))
    (setf rab
            (abs
             (f2cl-lib:fref a-%data%
                            (i
                             (f2cl-lib:int-sub
                              (f2cl-lib:int-add irab ilo) 1))
                            ((1 lda) (1 *)) a-%offset%)))
    (setf irab
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (icamax (f2cl-lib:int-add (f2cl-lib:int-sub n ilo) 1)
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                       (i ilo) ((1 ldb$) (1 *))
                                       b-%offset%)
                 ldb$)
              (declare (ignore var-0 var-1))
              (when var-2 (setf ldb$ var-2))
              ret-val))
    (setf rab
            (max rab
                 (abs
                  (f2cl-lib:fref b-%data%
                                 (i
                                  (f2cl-lib:int-sub
                                   (f2cl-lib:int-add irab ilo) 1))
                                 ((1 ldb$) (1 *)) b-%offset%))))
    (setf lrab
            (f2cl-lib:int
             (+ (/ (f2cl-lib:log10 (+ rab sfmin)) basl) one)))
    (setf ir
            (f2cl-lib:int
             (+
              (f2cl-lib:fref lscale-%data% (i) ((1 *))
                             lscale-%offset%)
              (f2cl-lib:sign half
                             (f2cl-lib:fref lscale-%data% (i) ((1 *))
                                            lscale-%offset%)))))
    (setf ir
            (min
```

```
          (the f2cl-lib:integer4
               (max (the f2cl-lib:integer4 ir)
                    (the f2cl-lib:integer4 lsfmin)))
          (the f2cl-lib:integer4 lsfmax)
          (the f2cl-lib:integer4 (f2cl-lib:int-sub lsfmax lrab))))
(setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
        (expt sclfac ir))
(setf icab
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (icamax ihi
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   (1 i) ((1 lda) (1 *))
                                   a-%offset%)
             1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf ihi var-0))
          ret-val))
(setf cab
        (abs
         (f2cl-lib:fref a-%data% (icab i) ((1 lda) (1 *))
                        a-%offset%)))
(setf icab
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (icamax ihi
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                   (1 i) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf ihi var-0))
          ret-val))
(setf cab
        (max cab
             (abs
              (f2cl-lib:fref b-%data% (icab i) ((1 ldb$) (1 *))
                             b-%offset%))))
(setf lcab
        (f2cl-lib:int
         (+ (/ (f2cl-lib:log10 (+ cab sfmin)) basl) one)))
(setf jc
        (f2cl-lib:int
         (+
          (f2cl-lib:fref rscale-%data% (i) ((1 *))
                         rscale-%offset%)
          (f2cl-lib:sign half
                         (f2cl-lib:fref rscale-%data% (i) ((1 *))
                                        rscale-%offset%)))))
(setf jc
        (min
         (the f2cl-lib:integer4
```

```
                    (max (the f2cl-lib:integer4 jc)
                         (the f2cl-lib:integer4 lsfmin)))
                (the f2cl-lib:integer4 lsfmax)
                (the f2cl-lib:integer4 (f2cl-lib:int-sub lsfmax lcab))))
      (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
              (expt sclfac jc))
    label360))
  (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                ((> i ihi) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (csscal (f2cl-lib:int-add (f2cl-lib:int-sub n ilo) 1)
           (f2cl-lib:array-slice lscale-%data% single-float (i) ((1 *))
                                 lscale-%offset%)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (i ilo)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2))
        (when var-3 (setf lda var-3)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (csscal (f2cl-lib:int-add (f2cl-lib:int-sub n ilo) 1)
           (f2cl-lib:array-slice lscale-%data% single-float (i) ((1 *))
                                 lscale-%offset%)
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (i ilo)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-1 var-2))
        (when var-3 (setf ldb$ var-3)))
    label370))
  (f2cl-lib:fdo (j ilo (f2cl-lib:int-add j 1))
                ((> j ihi) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (csscal ihi
           (f2cl-lib:array-slice rscale-%data% single-float (j) ((1 *))
                                 rscale-%offset%)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 j)
                                 ((1 lda) (1 *)) a-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf ihi var-0)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (csscal ihi
           (f2cl-lib:array-slice rscale-%data% single-float (j) ((1 *))
                                 rscale-%offset%)
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 j)
                                 ((1 ldb$) (1 *)) b-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf ihi var-0)))
```

```
            label380))
          (go end_label)
         end_label
          (return
           (values job nil nil lda nil ldb$ ilo ihi nil nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cggbal
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::job nil nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::ilo
                                              fortran-to-lisp::ihi nil nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

46.12 cla_gbamv

46.12.1 Axiom unit tests

— clagbamv.input —

```
)set break resume
)sys rm -f clagbamv.output
```

```
)spool clagbamv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

46.12.2 Axiom help page

— **cla_gbamv.help** —

```
==================================================================
cla_gbamv examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cla_gbamv (TRANS, M, N, KL, KU, ALPHA, AB, LDAB, X, INCX,
           BETA, Y, INCY)
           CLA_GBAMV performs a matrix-vector operation to calculate error
           bounds.

Function/Subroutine Documentation
   subroutine cla_gbamv (integer TRANS, integer M, integer N, integer KL,
       integer KU, real ALPHA, complex, dimension( ldab, * ) AB, integer LDAB,
       complex, dimension( * ) X, integer INCX, real BETA, real, dimension( *
       ) Y, integer INCY)
       CLA_GBAMV performs a matrix-vector operation to calculate error bounds.

Purpose:
   CLA_GBAMV  performs one of the matrix-vector operations

               y := alpha*abs(A)*abs(x) + beta*abs(y),
         or    y := alpha*abs(A)**T*abs(x) + beta*abs(y),

   where alpha and beta are scalars, x and y are vectors and A is an
   m by n matrix.

   This function is primarily used in calculating error bounds.
   To protect against underflow during evaluation, components in
   the resulting vector are perturbed away from zero by (N+1)
```

```
   times the underflow threshold.  To prevent unnecessarily large
   errors for block-structure embedded in general matrices,
   "symbolically" zero components are not perturbed.  A zero
   entry is considered "symbolic" if all multiplications involved
   in computing that entry have at least one zero multiplicand.

Parameters:

 TRANS
   TRANS is INTEGER
   On entry, TRANS specifies the operation to be performed as
   follows:

   BLAS_NO_TRANS      y := alpha*abs(A)*abs(x) + beta*abs(y)
   BLAS_TRANS         y := alpha*abs(A**T)*abs(x) + beta*abs(y)
   BLAS_CONJ_TRANS    y := alpha*abs(A**T)*abs(x) + beta*abs(y)

   Unchanged on exit.

 M
   M is INTEGER
   On entry, M specifies the number of rows of the matrix A.
   M must be at least zero.
   Unchanged on exit.

 N
   N is INTEGER
   On entry, N specifies the number of columns of the matrix A.
   N must be at least zero.
   Unchanged on exit.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 ALPHA
   ALPHA is REAL
   On entry, ALPHA specifies the scalar alpha.
   Unchanged on exit.

 AB
   AB is REAL array, dimension (LDAB,n)
   Before entry, the leading m by n part of the array AB must
   contain the matrix of coefficients.
   Unchanged on exit.
```

```
LDAB
  LDAB is INTEGER
  On entry, LDAB specifies the first dimension of AB as declared
  in the calling (sub) program. LDAB must be at least
  max( 1, m ).
  Unchanged on exit.

X
  X is REAL array, dimension
  ( 1 + ( n - 1 )*abs( INCX ) ) when TRANS = 'N' or 'n'
  and at least
  ( 1 + ( m - 1 )*abs( INCX ) ) otherwise.
  Before entry, the incremented array X must contain the
  vector x.
  Unchanged on exit.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of
  X. INCX must not be zero.
  Unchanged on exit.

BETA
  BETA is REAL
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.
  Unchanged on exit.

Y
  Y is REAL array, dimension
  ( 1 + ( m - 1 )*abs( INCY ) ) when TRANS = 'N' or 'n'
  and at least
  ( 1 + ( n - 1 )*abs( INCY ) ) otherwise.
  Before entry with BETA non-zero, the incremented array Y
  must contain the vector y. On exit, Y is overwritten by the
  updated vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of
  Y. INCY must not be zero.
  Unchanged on exit.
```

———-

46.12.3 fortran code

— cla_gbamv.f —

```
*  ===================================================================
      SUBROUTINE CLA_GBAMV( TRANS, M, N, KL, KU, ALPHA, AB, LDAB, X,
     $                      INCX, BETA, Y, INCY )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      REAL               ALPHA, BETA
      INTEGER            INCX, INCY, LDAB, M, N, KL, KU, TRANS
*     ..
*     .. Array Arguments ..
      COMPLEX            AB( LDAB, * ), X( * )
      REAL               Y( * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      COMPLEX            ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            SYMB_ZERO
      REAL               TEMP, SAFE1
      INTEGER            I, INFO, IY, J, JX, KX, KY, LENX, LENY, KD, KE
      COMPLEX            CDUM
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, SLAMCH
      REAL               SLAMCH
*     ..
*     .. External Functions ..
      EXTERNAL           ILATRANS
      INTEGER            ILATRANS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, ABS, REAL, AIMAG, SIGN
*     ..
*     .. Statement Functions
      REAL               CABS1
*     ..
*     .. Statement Function Definitions ..
```

```
      CABS1( CDUM ) = ABS( REAL( CDUM ) ) + ABS( AIMAG( CDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF     ( .NOT.( ( TRANS.EQ.ILATRANS( 'N' ) )
     $           .OR. ( TRANS.EQ.ILATRANS( 'T' ) )
     $           .OR. ( TRANS.EQ.ILATRANS( 'C' ) ) ) ) THEN
         INFO = 1
      ELSE IF( M.LT.0 )THEN
         INFO = 2
      ELSE IF( N.LT.0 )THEN
         INFO = 3
      ELSE IF( KL.LT.0 .OR. KL.GT.M-1 ) THEN
         INFO = 4
      ELSE IF( KU.LT.0 .OR. KU.GT.N-1 ) THEN
         INFO = 5
      ELSE IF( LDAB.LT.KL+KU+1 )THEN
         INFO = 6
      ELSE IF( INCX.EQ.0 )THEN
         INFO = 8
      ELSE IF( INCY.EQ.0 )THEN
         INFO = 11
      END IF
      IF( INFO.NE.0 )THEN
         CALL XERBLA( 'CLA_GBAMV ', INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( ( M.EQ.0 ).OR.( N.EQ.0 ).OR.
     $    ( ( ALPHA.EQ.ZERO ).AND.( BETA.EQ.ONE ) ) )
     $   RETURN
*
*     Set  LENX  and  LENY, the lengths of the vectors x and y, and set
*     up the start points in  X  and  Y.
*
      IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
         LENX = N
         LENY = M
      ELSE
         LENX = M
         LENY = N
      END IF
      IF( INCX.GT.0 )THEN
         KX = 1
      ELSE
```

```
         KX = 1 - ( LENX - 1 )*INCX
      END IF
      IF( INCY.GT.0 )THEN
         KY = 1
      ELSE
         KY = 1 - ( LENY - 1 )*INCY
      END IF
*
*     Set SAFE1 essentially to be the underflow threshold times the
*     number of additions in each row.
*
      SAFE1 = SLAMCH( 'Safe minimum' )
      SAFE1 = (N+1)*SAFE1
*
*     Form  y := alpha*abs(A)*abs(x) + beta*abs(y).
*
*     The O(M*N) SYMB_ZERO tests could be replaced by O(N) queries to
*     the inexact flag.  Still doesn't help change the iteration order
*     to per-column.
*
      KD = KU + 1
      KE = KL + 1
      IY = KY
      IF ( INCX.EQ.1 ) THEN
         IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
            DO I = 1, LENY
               IF ( BETA .EQ. 0.0 ) THEN
                  SYMB_ZERO = .TRUE.
                  Y( IY ) = 0.0
               ELSE IF ( Y( IY ) .EQ. 0.0 ) THEN
                  SYMB_ZERO = .TRUE.
               ELSE
                  SYMB_ZERO = .FALSE.
                  Y( IY ) = BETA * ABS( Y( IY ) )
               END IF
               IF ( ALPHA .NE. 0.0 ) THEN
                  DO J = MAX( I-KL, 1 ), MIN( I+KU, LENX )
                     TEMP = CABS1( AB( KD+I-J, J ) )
                     SYMB_ZERO = SYMB_ZERO .AND.
     $                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                     Y( IY ) = Y( IY ) + ALPHA*CABS1( X( J ) )*TEMP
                  END DO
               END IF

               IF ( .NOT.SYMB_ZERO)
     $              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

               IY = IY + INCY
            END DO
```

```
      ELSE
         DO I = 1, LENY
            IF ( BETA .EQ. 0.0 ) THEN
               SYMB_ZERO = .TRUE.
               Y( IY ) = 0.0
            ELSE IF ( Y( IY ) .EQ. 0.0 ) THEN
               SYMB_ZERO = .TRUE.
            ELSE
               SYMB_ZERO = .FALSE.
               Y( IY ) = BETA * ABS( Y( IY ) )
            END IF
            IF ( ALPHA .NE. 0.0 ) THEN
               DO J = MAX( I-KL, 1 ), MIN( I+KU, LENX )
                  TEMP = CABS1( AB( KE-I+J, I ) )
                  SYMB_ZERO = SYMB_ZERO .AND.
     $                 ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                  Y( IY ) = Y( IY ) + ALPHA*CABS1( X( J ) )*TEMP
               END DO
            END IF

            IF ( .NOT.SYMB_ZERO)
     $            Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

            IY = IY + INCY
         END DO
      END IF
   ELSE
      IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
         DO I = 1, LENY
            IF ( BETA .EQ. 0.0 ) THEN
               SYMB_ZERO = .TRUE.
               Y( IY ) = 0.0
            ELSE IF ( Y( IY ) .EQ. 0.0 ) THEN
               SYMB_ZERO = .TRUE.
            ELSE
               SYMB_ZERO = .FALSE.
               Y( IY ) = BETA * ABS( Y( IY ) )
            END IF
            IF ( ALPHA .NE. 0.0 ) THEN
               JX = KX
               DO J = MAX( I-KL, 1 ), MIN( I+KU, LENX )
                  TEMP = CABS1( AB( KD+I-J, J ) )
                  SYMB_ZERO = SYMB_ZERO .AND.
     $                 ( X( JX ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                  Y( IY ) = Y( IY ) + ALPHA*CABS1( X( JX ) )*TEMP
                  JX = JX + INCX
               END DO
            END IF
```

```
            IF ( .NOT.SYMB_ZERO )
     $           Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

            IY = IY + INCY
         END DO
      ELSE
         DO I = 1, LENY
            IF ( BETA .EQ. 0.0 ) THEN
               SYMB_ZERO = .TRUE.
               Y( IY ) = 0.0
            ELSE IF ( Y( IY ) .EQ. 0.0 ) THEN
               SYMB_ZERO = .TRUE.
            ELSE
               SYMB_ZERO = .FALSE.
               Y( IY ) = BETA * ABS( Y( IY ) )
            END IF
            IF ( ALPHA .NE. 0.0 ) THEN
               JX = KX
               DO J = MAX( I-KL, 1 ), MIN( I+KU, LENX )
                  TEMP = CABS1( AB( KE-I+J, I ) )
                  SYMB_ZERO = SYMB_ZERO .AND.
     $                 ( X( JX ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                  Y( IY ) = Y( IY ) + ALPHA*CABS1( X( JX ) )*TEMP
                  JX = JX + INCX
               END DO
            END IF

            IF ( .NOT.SYMB_ZERO )
     $           Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

            IY = IY + INCY
         END DO
      END IF

      END IF
*
      RETURN
*
*     End of CLA_GBAMV
*
      END
```

———-

46.12.4 lisp code

— cla_gbamv.lisp —

```
(let* ((one (coerce 1.0 'f2cl-lib:complex8))
       (zero (coerce 0.0 'f2cl-lib:complex8)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (ignorable one zero))
  (defun cla_gbamv (trans m n kl ku alpha ab ldab x incx beta y incy)
    (declare (type (array single-float (*)) y)
             (type (array f2cl-lib:complex8 (*)) x ab)
             (type (single-float) beta alpha)
             (type (f2cl-lib:integer4) incy incx ldab ku kl n m trans))
    (f2cl-lib:with-multi-array-data
        ((ab f2cl-lib:complex8 ab-%data% ab-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%)
         (y single-float y-%data% y-%offset%))
      (labels ((cabs1 (cdum)
                 (+ (abs (f2cl-lib:freal cdum)) (abs (f2cl-lib:aimag cdum)))))
        (declare
         (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
          cabs1))
        (prog ((cdum #C(0.0 0.0)) (i 0) (info 0) (iy 0) (j 0) (jx 0) (kx 0)
               (ky 0) (lenx 0) (leny 0) (kd 0) (ke 0) (temp 0.0) (safe1 0.0)
               (symb_zero nil))
          (declare (type (f2cl-lib:complex8) cdum)
                   (type (f2cl-lib:integer4) i info iy j jx kx ky lenx leny kd
                    ke)
                   (type (single-float) temp safe1)
                   (type f2cl-lib:logical symb_zero))
          (setf info 0)
          (cond
           ((not
             (or (= trans (ilatrans "N")) (= trans (ilatrans "T"))
                 (= trans (ilatrans "C"))))
            (setf info 1))
           ((< m 0) (setf info 2)) ((< n 0) (setf info 3))
           ((or (< kl 0) (> kl (f2cl-lib:int-add m (f2cl-lib:int-sub 1))))
            (setf info 4))
           ((or (< ku 0) (> ku (f2cl-lib:int-add n (f2cl-lib:int-sub 1))))
            (setf info 5))
           ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info 6))
           ((= incx 0) (setf info 8)) ((= incy 0) (setf info 11)))
          (cond
           ((/= info 0)
            (multiple-value-bind (var-0 var-1)
                (xerbla "CLA_GBAMV " info)
              (declare (ignore var-0))
```

```
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
    (go end_label))
(cond ((= trans (ilatrans "N")) (setf lenx n) (setf leny m))
      (t (setf lenx m) (setf leny n)))
(cond ((> incx 0) (setf kx 1))
      (t
       (setf kx
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub lenx 1) incx)))))
(cond ((> incy 0) (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub leny 1) incy)))))
(setf safe1 (slamch "Safe minimum"))
(setf safe1 (* (f2cl-lib:int-add n 1) safe1))
(setf kd (f2cl-lib:int-add ku 1))
(setf ke (f2cl-lib:int-add kl 1))
(setf iy ky)
(cond
 ((= incx 1)
  (cond
   ((= trans (ilatrans "N"))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i leny) nil)
      (tagbody
        (cond
         ((= beta 0.0) (setf symb_zero f2cl-lib:%true%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  0.0))
         ((= (f2cl-lib:fref y (iy) ((1 *))) 0.0)
          (setf symb_zero f2cl-lib:%true%))
         (t (setf symb_zero f2cl-lib:%false%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (* beta
                     (abs
                      (f2cl-lib:fref y-%data% (iy) ((1 *))
                                     y-%offset%))))))
        (cond
         ((/= alpha 0.0)
          (f2cl-lib:fdo (j
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add i
                                                 (f2cl-lib:int-sub
                                                  kl)))
```

```
                          (the f2cl-lib:integer4 1))
                         (f2cl-lib:int-add j 1))
                        ((> j
                            (min
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add i ku))
                             (the f2cl-lib:integer4 lenx)))
                         nil)
                (tagbody
                  (setf temp
                          (cabs1
                           (f2cl-lib:fref ab-%data%
                                          ((f2cl-lib:int-sub
                                            (f2cl-lib:int-add kd i) j)
                                           j)
                                          ((1 ldab) (1 *)) ab-%offset%)))
                  (setf symb_zero
                          (and symb_zero
                               (or
                                (=
                                 (f2cl-lib:fref x-%data% (j) ((1 *))
                                                x-%offset%)
                                 zero)
                                (= temp zero))))
                  (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                          (+
                           (f2cl-lib:fref y-%data% (iy) ((1 *))
                                          y-%offset%)
                           (* alpha
                              (cabs1
                               (f2cl-lib:fref x-%data% (j) ((1 *))
                                              x-%offset%))
                              temp)))
                 label100001))))
            (if (not symb_zero)
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (f2cl-lib:sign safe1
                                        (f2cl-lib:fref y-%data% (iy)
                                                       ((1 *))
                                                       y-%offset%)))))
            (setf iy (f2cl-lib:int-add iy incy))
           label100000)))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i leny) nil)
           (tagbody
             (cond
              ((= beta 0.0) (setf symb_zero f2cl-lib:%true%)
```

```
  (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
          0.0))
 ((= (f2cl-lib:fref y (iy) ((1 *))) 0.0)
  (setf symb_zero f2cl-lib:%true%))
 (t (setf symb_zero f2cl-lib:%false%)
  (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
          (* beta
             (abs
              (f2cl-lib:fref y-%data% (iy) ((1 *))
                             y-%offset%))))))
(cond
 ((/= alpha 0.0)
  (f2cl-lib:fdo (j
                 (max
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add i
                                         (f2cl-lib:int-sub
                                          kl)))
                  (the f2cl-lib:integer4 1))
                 (f2cl-lib:int-add j 1))
                ((> j
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add i ku))
                     (the f2cl-lib:integer4 lenx)))
                nil)
    (tagbody
      (setf temp
              (cabs1
               (f2cl-lib:fref ab-%data%
                              ((f2cl-lib:int-add
                                (f2cl-lib:int-sub ke i) j)
                               i)
                              ((1 ldab) (1 *)) ab-%offset%)))
      (setf symb_zero
              (and symb_zero
                   (or
                    (=
                     (f2cl-lib:fref x-%data% (j) ((1 *))
                                    x-%offset%)
                     zero)
                    (= temp zero))))
      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
              (+
               (f2cl-lib:fref y-%data% (iy) ((1 *))
                              y-%offset%)
               (* alpha
                  (cabs1
                   (f2cl-lib:fref x-%data% (j) ((1 *))
                                  x-%offset%))
```

```
                          temp)))
                 label100003))))
            (if (not symb_zero)
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (f2cl-lib:sign safe1
                                        (f2cl-lib:fref y-%data% (iy)
                                                       ((1 *))
                                                       y-%offset%)))))
            (setf iy (f2cl-lib:int-add iy incy))
           label100002)))))
      (t
       (cond
        ((= trans (ilatrans "N"))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i leny) nil)
           (tagbody
             (cond
              ((= beta 0.0) (setf symb_zero f2cl-lib:%true%)
               (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       0.0))
              ((= (f2cl-lib:fref y (iy) ((1 *))) 0.0)
               (setf symb_zero f2cl-lib:%true%))
              (t (setf symb_zero f2cl-lib:%false%)
               (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (* beta
                          (abs
                           (f2cl-lib:fref y-%data% (iy) ((1 *))
                                          y-%offset%))))))
             (cond
              ((/= alpha 0.0) (setf jx kx)
               (f2cl-lib:fdo (j
                              (max
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add i
                                                      (f2cl-lib:int-sub
                                                       kl)))
                               (the f2cl-lib:integer4 1))
                              (f2cl-lib:int-add j 1))
                             ((> j
                                 (min
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-add i ku))
                                  (the f2cl-lib:integer4 lenx)))
                              nil)
                 (tagbody
                   (setf temp
                           (cabs1
                            (f2cl-lib:fref ab-%data%
```

```
                                        ((f2cl-lib:int-sub
                                          (f2cl-lib:int-add kd i) j)
                                         j)
                                        ((1 ldab) (1 *)) ab-%offset%)))
                (setf symb_zero
                        (and symb_zero
                             (or
                              (=
                               (f2cl-lib:fref x-%data% (jx) ((1 *))
                                              x-%offset%)
                               zero)
                              (= temp zero))))
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data% (iy) ((1 *))
                                        y-%offset%)
                         (* alpha
                            (cabs1
                             (f2cl-lib:fref x-%data% (jx) ((1 *))
                                            x-%offset%))
                            temp)))
                (setf jx (f2cl-lib:int-add jx incx))
               label100005))))
          (if (not symb_zero)
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (+
                       (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (f2cl-lib:sign safe1
                                      (f2cl-lib:fref y-%data% (iy)
                                                     ((1 *))
                                                     y-%offset%)))))
          (setf iy (f2cl-lib:int-add iy incy))
         label100004)))
      (t
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i leny) nil)
         (tagbody
           (cond
            ((= beta 0.0) (setf symb_zero f2cl-lib:%true%)
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     0.0))
            ((= (f2cl-lib:fref y (iy) ((1 *))) 0.0)
             (setf symb_zero f2cl-lib:%true%))
            (t (setf symb_zero f2cl-lib:%false%)
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (* beta
                        (abs
                         (f2cl-lib:fref y-%data% (iy) ((1 *))
                                        y-%offset%))))))
           (cond
```

```
((/= alpha 0.0) (setf jx kx)
 (f2cl-lib:fdo (j
                (max
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add i
                                        (f2cl-lib:int-sub
                                         kl)))
                 (the f2cl-lib:integer4 1))
                (f2cl-lib:int-add j 1))
               ((> j
                   (min
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add i ku))
                    (the f2cl-lib:integer4 lenx)))
                nil)
   (tagbody
     (setf temp
             (cabs1
              (f2cl-lib:fref ab-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-sub ke i) j)
                              i)
                             ((1 ldab) (1 *)) ab-%offset%)))
     (setf symb_zero
             (and symb_zero
                  (or
                   (=
                    (f2cl-lib:fref x-%data% (jx) ((1 *))
                                   x-%offset%)
                    zero)
                   (= temp zero))))
     (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
             (+
              (f2cl-lib:fref y-%data% (iy) ((1 *))
                             y-%offset%)
              (* alpha
                 (cabs1
                  (f2cl-lib:fref x-%data% (jx) ((1 *))
                                 x-%offset%))
                 temp)))
     (setf jx (f2cl-lib:int-add jx incx))
    label100007))))
(if (not symb_zero)
    (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
            (+
             (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
             (f2cl-lib:sign safe1
                            (f2cl-lib:fref y-%data% (iy)
                                           ((1 *))
                                           y-%offset%)))))
```

```
                 (setf iy (f2cl-lib:int-add iy incy))
                label100006))))))
          (go end_label)
         end_label
          (return
           (values nil nil nil nil nil nil nil nil nil nil nil nil nil)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cla_gbamv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (single-float)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
                                              nil nil nil nil nil)
                                            :calls 'nil)))
```

———-

46.13 cla_gbrcond_c

46.13.1 Axiom unit tests

— clagbrcondc.input —

```
)set break resume
)sys rm -f clagbrcondc.output
)spool clagbrcondc.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

46.13.2 Axiom help page

— cla_gbrcond_c.help —

```
====================================================================
cla_gbrcond_c examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       real function cla_gbrcond_c (TRANS, N, KL, KU, AB, LDAB, AFB, LDAFB,
           IPIV, C, CAPPLY, INFO, WORK, RWORK)
           CLA_GBRCOND_C computes the infinity norm condition number of
           op(A)*inv(diag(c)) for general banded matrices.

Function/Subroutine Documentation
   real function cla_gbrcond_c (character TRANS, integer N, integer KL,
       integer KU, complex, dimension( ldab, * ) AB, integer LDAB, complex,
       dimension( ldafb, * ) AFB, integer LDAFB, integer, dimension( * ) IPIV,
       real, dimension( * ) C, logical CAPPLY, integer INFO, complex,
       dimension( * ) WORK, real, dimension( * ) RWORK)
       CLA_GBRCOND_C computes the infinity norm condition number of
       op(A)*inv(diag(c)) for general banded matrices.

Purpose:
   CLA_GBRCOND_C Computes the infinity norm condition number of
   op(A) * inv(diag(C)) where C is a REAL vector.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate Transpose = Transpose)
```

```
N
  N is INTEGER
  The number of linear equations, i.e., the order of the
  matrix A.  N >= 0.

KL
  KL is INTEGER
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

AB
  AB is COMPLEX array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is COMPLEX array, dimension (LDAFB,N)
  Details of the LU factorization of the band matrix A, as
  computed by CGBTRF.  U is stored as an upper triangular
  band matrix with KL+KU superdiagonals in rows 1 to KL+KU+1,
  and the multipliers used during the factorization are stored
  in rows KL+KU+2 to 2*KL+KU+1.

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from the factorization A = P*L*U
  as computed by CGBTRF; row i of the matrix was interchanged
  with row IPIV(i).

C
  C is REAL array, dimension (N)
  The vector C in the formula op(A) * inv(diag(C)).

CAPPLY
  CAPPLY is LOGICAL
  If .TRUE. then access the vector C in the formula above.
```

```
 INFO
   INFO is INTEGER
   = 0:  Successful exit.
   i > 0:  The ith argument is invalid.

 WORK
   WORK is COMPLEX array, dimension (2*N).
   Workspace.

 RWORK
   RWORK is REAL array, dimension (N).
   Workspace.
```

———-

46.13.3 fortran code

— **cla_gbrcond_c.f** —

```
*  =====================================================================
      REAL FUNCTION CLA_GBRCOND_C( TRANS, N, KL, KU, AB, LDAB, AFB,
     $                             LDAFB, IPIV, C, CAPPLY, INFO, WORK,
     $                             RWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      LOGICAL            CAPPLY
      INTEGER            N, KL, KU, KD, KE, LDAB, LDAFB, INFO
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            AB( LDAB, * ), AFB( LDAFB, * ), WORK( * )
      REAL               C( * ), RWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            NOTRANS
      INTEGER            KASE, I, J
```

```
      REAL               AINVNM, ANORM, TMP
      COMPLEX            ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLACN2, CGBTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
      CLA_GBRCOND_C = 0.0E+0
*
      INFO = 0
      NOTRANS = LSAME( TRANS, 'N' )
      IF ( .NOT. NOTRANS .AND. .NOT. LSAME( TRANS, 'T' ) .AND. .NOT.
     $     LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 .OR. KL.GT.N-1 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 .OR. KU.GT.N-1 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -6
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CLA_GBRCOND_C', -INFO )
         RETURN
      END IF
*
*     Compute norm of op(A)*op2(C).
*
      ANORM = 0.0E+0
      KD = KU + 1
```

```
      KE = KL + 1
      IF ( NOTRANS ) THEN
         DO I = 1, N
            TMP = 0.0E+0
            IF ( CAPPLY ) THEN
               DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
                  TMP = TMP + CABS1( AB( KD+I-J, J ) ) / C( J )
               END DO
            ELSE
               DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
                  TMP = TMP + CABS1( AB( KD+I-J, J ) )
               END DO
            END IF
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      ELSE
         DO I = 1, N
            TMP = 0.0E+0
            IF ( CAPPLY ) THEN
               DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
                  TMP = TMP + CABS1( AB( KE-I+J, I ) ) / C( J )
               END DO
            ELSE
               DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
                  TMP = TMP + CABS1( AB( KE-I+J, I ) )
               END DO
            END IF
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 ) THEN
         CLA_GBRCOND_C = 1.0E+0
         RETURN
      ELSE IF( ANORM .EQ. 0.0E+0 ) THEN
         RETURN
      END IF
*
*     Estimate the norm of inv(op(A)).
*
      AINVNM = 0.0E+0
*
      KASE = 0
   10 CONTINUE
      CALL CLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
```

```
         IF( KASE.EQ.2 ) THEN
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
*
            IF ( NOTRANS ) THEN
               CALL CGBTRS( 'No transpose', N, KL, KU, 1, AFB, LDAFB,
     $              IPIV, WORK, N, INFO )
            ELSE
               CALL CGBTRS( 'Conjugate transpose', N, KL, KU, 1, AFB,
     $              LDAFB, IPIV, WORK, N, INFO )
            ENDIF
*
*           Multiply by inv(C).
*
            IF ( CAPPLY ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF
         ELSE
*
*           Multiply by inv(C**H).
*
            IF ( CAPPLY ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF
*
            IF ( NOTRANS ) THEN
               CALL CGBTRS( 'Conjugate transpose', N, KL, KU, 1, AFB,
     $              LDAFB, IPIV,  WORK, N, INFO )
            ELSE
               CALL CGBTRS( 'No transpose', N, KL, KU, 1, AFB, LDAFB,
     $              IPIV, WORK, N, INFO )
            END IF
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
         END IF
         GO TO 10
      END IF
*
```

```
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM .NE. 0.0E+0 )
     $   CLA_GBRCOND_C = 1.0E+0 / AINVNM
*
      RETURN
*
      END
```

46.13.4 lisp code

— cla_gbrcond_c.lisp —

```
(defun cla_gbrcond_c
       (trans n kl ku ab ldab afb ldafb ipiv c capply info work rwork)
  (declare (type f2cl-lib:logical capply)
           (type (array single-float (*)) rwork c)
           (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex8 (*)) work afb ab)
           (type (f2cl-lib:integer4) info ldafb ldab ku kl n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (ab f2cl-lib:complex8 ab-%data% ab-%offset%)
       (afb f2cl-lib:complex8 afb-%data% afb-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (c single-float c-%data% c-%offset%)
       (rwork single-float rwork-%data% rwork-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
        cabs1))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (zdum #C(0.0 0.0)) (ainvnm 0.0) (anorm 0.0) (tmp 0.0) (kase 0)
             (i 0) (j 0) (notrans nil) (kd 0) (ke 0) (cla_gbrcond_c 0.0))
        (declare (type f2cl-lib:logical notrans)
                 (type (f2cl-lib:integer4) ke kd j i kase)
                 (type (single-float) cla_gbrcond_c tmp anorm ainvnm)
                 (type (f2cl-lib:complex8) zdum)
                 (type (array f2cl-lib:integer4 (3)) isave))
        (setf cla_gbrcond_c 0.0)
        (setf info 0)
        (setf notrans
                (multiple-value-bind (ret-val var-0 var-1)
```

```
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((and (not notrans)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((or (< kl 0) (> kl (f2cl-lib:int-add n (f2cl-lib:int-sub 1))))
  (setf info -3))
 ((or (< ku 0) (> ku (f2cl-lib:int-add n (f2cl-lib:int-sub 1))))
  (setf info -4))
 ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -6))
 ((< ldafb (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
  (setf info -8)))
(cond
 ((/= info 0) (xerbla "CLA_GBRCOND_C" (f2cl-lib:int-sub info))
  (go end_label)))
(setf anorm 0.0)
(setf kd (f2cl-lib:int-add ku 1))
(setf ke (f2cl-lib:int-add kl 1))
(cond
 (notrans
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf tmp 0.0)
      (cond
       (capply
        (f2cl-lib:fdo (j
                       (max
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add i
                                               (f2cl-lib:int-sub kl)))
                        (the f2cl-lib:integer4 1))
                       (f2cl-lib:int-add j 1))
                      ((> j
                          (min
```

```
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add i ku))
                              (the f2cl-lib:integer4 n)))
                          nil)
              (tagbody
                (setf tmp
                        (+ tmp
                           (/
                            (cabs1
                             (f2cl-lib:fref ab-%data%
                                            ((f2cl-lib:int-sub
                                              (f2cl-lib:int-add kd i) j)
                                             j)
                                            ((1 ldab) (1 *)) ab-%offset%))
                            (f2cl-lib:fref c-%data% (j) ((1 *))
                                           c-%offset%))))
               label100001)))
          (t
           (f2cl-lib:fdo (j
                          (max
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add i
                                                  (f2cl-lib:int-sub kl)))
                           (the f2cl-lib:integer4 1))
                          (f2cl-lib:int-add j 1))
                         ((> j
                             (min
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add i ku))
                              (the f2cl-lib:integer4 n)))
                          nil)
              (tagbody
                (setf tmp
                        (+ tmp
                           (cabs1
                            (f2cl-lib:fref ab-%data%
                                           ((f2cl-lib:int-sub
                                             (f2cl-lib:int-add kd i) j)
                                            j)
                                           ((1 ldab) (1 *)) ab-%offset%))))
               label100002))))
         (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                 tmp)
         (setf anorm (max anorm tmp))
        label100000)))
    (t
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf tmp 0.0)
```

```
(cond
 (capply
  (f2cl-lib:fdo (j
                 (max
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add i
                                         (f2cl-lib:int-sub kl)))
                  (the f2cl-lib:integer4 1))
                 (f2cl-lib:int-add j 1))
                ((> j
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add i ku))
                     (the f2cl-lib:integer4 n)))
                 nil)
    (tagbody
      (setf tmp
              (+ tmp
                 (/
                  (cabs1
                   (f2cl-lib:fref ab-%data%
                                  ((f2cl-lib:int-add
                                    (f2cl-lib:int-sub ke i) j)
                                   i)
                                  ((1 ldab) (1 *)) ab-%offset%))
                  (f2cl-lib:fref c-%data% (j) ((1 *))
                                 c-%offset%))))
     label100004)))
 (t
  (f2cl-lib:fdo (j
                 (max
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add i
                                         (f2cl-lib:int-sub kl)))
                  (the f2cl-lib:integer4 1))
                 (f2cl-lib:int-add j 1))
                ((> j
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add i ku))
                     (the f2cl-lib:integer4 n)))
                 nil)
    (tagbody
      (setf tmp
              (+ tmp
                 (cabs1
                  (f2cl-lib:fref ab-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-sub ke i) j)
                                  i)
```

```
                                        ((1 ldab) (1 *)) ab-%offset%))))
            label100005))))
       (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
               tmp)
       (setf anorm (max anorm tmp))
      label100003))))
 (cond ((= n 0) (setf cla_gbrcond_c 1.0) (go end_label))
       ((= anorm 0.0) (go end_label)))
 (setf ainvnm 0.0)
 (setf kase 0)
label10
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
     (clacn2 n
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ n 1))
                            ((1 *)) work-%offset%)
      work ainvnm kase isave)
   (declare (ignore var-1 var-2 var-5))
   (setf n var-0)
   (setf ainvnm var-3)
   (setf kase var-4))
 (cond
  ((/= kase 0)
   (cond
    ((= kase 2)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (*
                  (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                 rwork-%offset%)))
        label100006))
     (cond
      (notrans
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10)
           (cgbtrs "No transpose" n kl ku 1 afb ldafb ipiv work n info)
         (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
         (setf n var-1)
         (setf ldafb var-6)
         (setf n var-9)
         (setf info var-10)))
      (t
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10)
           (cgbtrs "Conjugate transpose" n kl ku 1 afb ldafb ipiv work n
            info)
```

```
        (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
        (setf n var-1)
        (setf ldafb var-6)
        (setf n var-9)
        (setf info var-10))))
    (cond
     (capply
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (*
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                   (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
         label100007)))))
  (t
   (cond
    (capply
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (*
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)
                  (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
         label100008))))
    (cond
     (notrans
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (cgbtrs "Conjugate transpose" n kl ku 1 afb ldafb ipiv work n
           info)
        (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
        (setf n var-1)
        (setf ldafb var-6)
        (setf n var-9)
        (setf info var-10)))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (cgbtrs "No transpose" n kl ku 1 afb ldafb ipiv work n info)
        (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
        (setf n var-1)
        (setf ldafb var-6)
        (setf n var-9)
        (setf info var-10))))
```

```
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                        (*
                         (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                         (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                        rwork-%offset%)))
               label100009))))
          (go label10)))
        (if (/= ainvnm 0.0)
            (setf cla_gbrcond_c (/ 1.0 ainvnm)))
        (go end_label)
       end_label
        (return
         (values cla_gbrcond_c trans n nil nil nil nil nil ldafb nil nil nil
                 info nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cla_gbrcond_c
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              fortran-to-lisp::logical
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array single-float (*)))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::n nil nil nil
                                              nil nil fortran-to-lisp::ldafb
                                              nil nil nil fortran-to-lisp::info
                                              nil nil)
                                            :calls
                                            '(fortran-to-lisp::cgbtrs
```

```
                                        fortran-to-lisp::clacn2))))
```

———-

46.14 cla_gbrcond_x

46.14.1 Axiom unit tests

— **clagbrcondx.input** —

```
)set break resume
)sys rm -f clagbrcondx.output
)spool clagbrcondx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

46.14.2 Axiom help page

— **cla_gbrcond_x.help** —

```
====================================================================
cla_gbrcond_x examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       real function cla_gbrcond_x (TRANS, N, KL, KU, AB, LDAB, AFB, LDAFB,
           IPIV, X, INFO, WORK, RWORK)
           CLA_GBRCOND_X computes the infinity norm condition number of
           op(A)*diag(x) for general banded matrices.

Function/Subroutine Documentation
```

```
   real function cla_gbrcond_x (character TRANS, integer N, integer KL,
       integer KU, complex, dimension( ldab, * ) AB, integer LDAB, complex,
       dimension( ldafb, * ) AFB, integer LDAFB, integer, dimension( * ) IPIV,
       complex, dimension( * ) X, integer INFO, complex, dimension( * ) WORK,
       real, dimension( * ) RWORK)
       CLA_GBRCOND_X computes the infinity norm condition number of
       op(A)*diag(x) for general banded matrices.

Purpose:
   CLA_GBRCOND_X Computes the infinity norm condition number of
   op(A) * diag(X) where X is a COMPLEX vector.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate Transpose = Transpose)

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 AB
   AB is COMPLEX array, dimension (LDAB,N)
   On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
   The j-th column of A is stored in the j-th column of the
   array AB as follows:
   AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

 LDAB
   LDAB is INTEGER
   The leading dimension of the array AB.  LDAB >= KL+KU+1.

 AFB
   AFB is COMPLEX array, dimension (LDAFB,N)
   Details of the LU factorization of the band matrix A, as
   computed by CGBTRF.  U is stored as an upper triangular
   band matrix with KL+KU superdiagonals in rows 1 to KL+KU+1,
```

```
  and the multipliers used during the factorization are stored
  in rows KL+KU+2 to 2*KL+KU+1.

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from the factorization A = P*L*U
  as computed by CGBTRF; row i of the matrix was interchanged
  with row IPIV(i).

X
  X is COMPLEX array, dimension (N)
  The vector X in the formula op(A) * diag(X).

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  i > 0:  The ith argument is invalid.

WORK
  WORK is COMPLEX array, dimension (2*N).
  Workspace.

RWORK
  RWORK is REAL array, dimension (N).
  Workspace.
```

46.14.3 fortran code

— cla_gbrcond_x.f —

```
*  =====================================================================
      REAL FUNCTION CLA_GBRCOND_X( TRANS, N, KL, KU, AB, LDAB, AFB,
     $                             LDAFB, IPIV, X, INFO, WORK, RWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
```

```
      CHARACTER          TRANS
      INTEGER            N, KL, KU, KD, KE, LDAB, LDAFB, INFO
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            AB( LDAB, * ), AFB( LDAFB, * ), WORK( * ),
     $                   X( * )
      REAL               RWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            NOTRANS
      INTEGER            KASE, I, J
      REAL               AINVNM, ANORM, TMP
      COMPLEX            ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLACN2, CGBTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      CLA_GBRCOND_X = 0.0E+0
*
      INFO = 0
      NOTRANS = LSAME( TRANS, 'N' )
      IF ( .NOT. NOTRANS .AND. .NOT. LSAME(TRANS, 'T') .AND. .NOT.
     $     LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 .OR. KL.GT.N-1 ) THEN
         INFO = -3
```

```
      ELSE IF( KU.LT.0 .OR. KU.GT.N-1 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -6
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CLA_GBRCOND_X', -INFO )
         RETURN
      END IF
*
*     Compute norm of op(A)*op2(C).
*
      KD = KU + 1
      KE = KL + 1
      ANORM = 0.0
      IF ( NOTRANS ) THEN
         DO I = 1, N
            TMP = 0.0E+0
            DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
               TMP = TMP + CABS1( AB( KD+I-J, J) * X( J ) )
            END DO
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      ELSE
         DO I = 1, N
            TMP = 0.0E+0
            DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
               TMP = TMP + CABS1( AB( KE-I+J, I ) * X( J ) )
            END DO
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 ) THEN
         CLA_GBRCOND_X = 1.0E+0
         RETURN
      ELSE IF( ANORM .EQ. 0.0E+0 ) THEN
         RETURN
      END IF
*
*     Estimate the norm of inv(op(A)).
*
      AINVNM = 0.0E+0
*
```

```
      KASE = 0
   10 CONTINUE
      CALL CLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
*
            IF ( NOTRANS ) THEN
               CALL CGBTRS( 'No transpose', N, KL, KU, 1, AFB, LDAFB,
     $              IPIV, WORK, N, INFO )
            ELSE
               CALL CGBTRS( 'Conjugate transpose', N, KL, KU, 1, AFB,
     $              LDAFB, IPIV, WORK, N, INFO )
            ENDIF
*
*           Multiply by inv(X).
*
            DO I = 1, N
               WORK( I ) = WORK( I ) / X( I )
            END DO
         ELSE
*
*           Multiply by inv(X**H).
*
            DO I = 1, N
               WORK( I ) = WORK( I ) / X( I )
            END DO
*
            IF ( NOTRANS ) THEN
               CALL CGBTRS( 'Conjugate transpose', N, KL, KU, 1, AFB,
     $              LDAFB, IPIV, WORK, N, INFO )
            ELSE
               CALL CGBTRS( 'No transpose', N, KL, KU, 1, AFB, LDAFB,
     $              IPIV, WORK, N, INFO )
            END IF
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
         END IF
         GO TO 10
      END IF
*
```

```
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM .NE. 0.0E+0 )
     $   CLA_GBRCOND_X = 1.0E+0 / AINVNM
*
      RETURN
*
      END
```

46.14.4 lisp code

— cla_gbrcond_x.lisp —

```
(defun cla_gbrcond_x (trans n kl ku ab ldab afb ldafb ipiv x info work rwork)
  (declare (type (array single-float (*)) rwork)
           (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex8 (*)) work x afb ab)
           (type (f2cl-lib:integer4) info ldafb ldab ku kl n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (ab f2cl-lib:complex8 ab-%data% ab-%offset%)
       (afb f2cl-lib:complex8 afb-%data% afb-%offset%)
       (x f2cl-lib:complex8 x-%data% x-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (rwork single-float rwork-%data% rwork-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
        cabs1))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (zdum #C(0.0 0.0)) (ainvnm 0.0) (anorm 0.0) (tmp 0.0) (kase 0)
             (i 0) (j 0) (notrans nil) (kd 0) (ke 0) (cla_gbrcond_x 0.0))
        (declare (type f2cl-lib:logical notrans)
                 (type (f2cl-lib:integer4) ke kd j i kase)
                 (type (single-float) cla_gbrcond_x tmp anorm ainvnm)
                 (type (f2cl-lib:complex8) zdum)
                 (type (array f2cl-lib:integer4 (3)) isave))
        (setf cla_gbrcond_x 0.0)
        (setf info 0)
        (setf notrans
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame trans "N")
                  (declare (ignore var-1))
```

```
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((and (not notrans)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((or (< kl 0) (> kl (f2cl-lib:int-add n (f2cl-lib:int-sub 1))))
  (setf info -3))
 ((or (< ku 0) (> ku (f2cl-lib:int-add n (f2cl-lib:int-sub 1))))
  (setf info -4))
 ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -6))
 ((< ldafb (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
  (setf info -8)))
(cond
 ((/= info 0) (xerbla "CLA_GBRCOND_X" (f2cl-lib:int-sub info))
  (go end_label)))
(setf kd (f2cl-lib:int-add ku 1))
(setf ke (f2cl-lib:int-add kl 1))
(setf anorm 0.0)
(cond
 (notrans
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf tmp 0.0)
      (f2cl-lib:fdo (j
                     (max
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add i (f2cl-lib:int-sub kl)))
                      (the f2cl-lib:integer4 1))
                     (f2cl-lib:int-add j 1))
                    ((> j
                        (min
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add i ku))
                         (the f2cl-lib:integer4 n)))
                     nil)
        (tagbody
```

```
            (setf tmp
                    (+ tmp
                       (cabs1
                        (*
                         (f2cl-lib:fref ab-%data%
                                        ((f2cl-lib:int-sub
                                          (f2cl-lib:int-add kd i) j)
                                         j)
                                        ((1 ldab) (1 *)) ab-%offset%)
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%)))))
        label100001))
      (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
              tmp)
      (setf anorm (max anorm tmp))
    label100000)))
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf tmp 0.0)
     (f2cl-lib:fdo (j
                    (max
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add i (f2cl-lib:int-sub kl)))
                     (the f2cl-lib:integer4 1))
                    (f2cl-lib:int-add j 1))
                   ((> j
                       (min
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add i ku))
                        (the f2cl-lib:integer4 n)))
                    nil)
       (tagbody
         (setf tmp
                 (+ tmp
                    (cabs1
                     (*
                      (f2cl-lib:fref ab-%data%
                                     ((f2cl-lib:int-add
                                       (f2cl-lib:int-sub ke i) j)
                                      i)
                                     ((1 ldab) (1 *)) ab-%offset%)
                      (f2cl-lib:fref x-%data% (j) ((1 *))
                                     x-%offset%)))))
        label100003))
      (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
              tmp)
      (setf anorm (max anorm tmp))
    label100002))))
```

```
 (cond ((= n 0) (setf cla_gbrcond_x 1.0) (go end_label))
       ((= anorm 0.0) (go end_label)))
 (setf ainvnm 0.0)
 (setf kase 0)
label10
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
     (clacn2 n
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ n 1))
                            ((1 *)) work-%offset%)
      work ainvnm kase isave)
   (declare (ignore var-1 var-2 var-5))
   (setf n var-0)
   (setf ainvnm var-3)
   (setf kase var-4))
 (cond
  ((/= kase 0)
   (cond
    ((= kase 2)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (*
                  (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                 rwork-%offset%)))
        label100004))
     (cond
      (notrans
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10)
           (cgbtrs "No transpose" n kl ku 1 afb ldafb ipiv work n info)
         (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
         (setf n var-1)
         (setf ldafb var-6)
         (setf n var-9)
         (setf info var-10)))
      (t
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10)
           (cgbtrs "Conjugate transpose" n kl ku 1 afb ldafb ipiv work n
            info)
         (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
         (setf n var-1)
         (setf ldafb var-6)
         (setf n var-9)
         (setf info var-10))))
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                    ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                    (/
                     (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                     (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)))
           label100005)))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                    (/
                     (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                     (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)))
           label100006))
         (cond
          (notrans
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                var-10)
               (cgbtrs "Conjugate transpose" n kl ku 1 afb ldafb ipiv work n
                info)
            (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
            (setf n var-1)
            (setf ldafb var-6)
            (setf n var-9)
            (setf info var-10)))
          (t
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                var-10)
               (cgbtrs "No transpose" n kl ku 1 afb ldafb ipiv work n info)
            (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
            (setf n var-1)
            (setf ldafb var-6)
            (setf n var-9)
            (setf info var-10))))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                    (*
                     (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                     (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                    rwork-%offset%)))
           label100007))))
      (go label10)))
    (if (/= ainvnm 0.0)
        (setf cla_gbrcond_x (/ 1.0 ainvnm)))
```

```
        (go end_label)
       end_label
        (return
         (values cla_gbrcond_x trans n nil nil nil nil nil ldafb nil nil info
                 nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cla_gbrcond_x
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array single-float (*)))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::n nil nil nil
                                              nil nil fortran-to-lisp::ldafb
                                              nil nil fortran-to-lisp::info nil
                                              nil)
                                            :calls
                                            '(fortran-to-lisp::cgbtrs
                                              fortran-to-lisp::clacn2))))
```

46.15 cla_gbrfsx_extended

46.15.1 Axiom unit tests

— clagbrfsxextended.input —

```
)set break resume
)sys rm -f clagbrfsxextended.output
)spool clagbrfsxextended.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

46.15.2 Axiom help page [Demm05]

— cla_gbrfsx_extended.help —

```
==================================================================
cla_gbrfsx_extended examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cla_gbrfsx_extended (PREC_TYPE, TRANS_TYPE, N, KL, KU, NRHS,
           AB, LDAB, AFB, LDAFB, IPIV, COLEQU, C, B, LDB, Y, LDY, BERR_OUT,
           N_NORMS, ERR_BNDS_NORM, ERR_BNDS_COMP, RES, AYB, DY, Y_TAIL, RCOND,
           ITHRESH, RTHRESH, DZ_UB, IGNORE_CWISE, INFO)
           CLA_GBRFSX_EXTENDED improves the computed solution to a system of
           linear equations for general banded matrices by performing extra-
           precise iterative refinement and provides error bounds and backward
           error estimates for the solution.

Function/Subroutine Documentation
   subroutine cla_gbrfsx_extended (integer PREC_TYPE, integer TRANS_TYPE,
       integer N, integer KL, integer KU, integer NRHS, complex, dimension(
       ldab, * ) AB, integer LDAB, complex, dimension( ldafb, * ) AFB, integer
       LDAFB, integer, dimension( * ) IPIV, logical COLEQU, real, dimension( *
```

```
      ) C, complex, dimension( ldb, * ) B, integer LDB, complex, dimension(
      ldy, * ) Y, integer LDY, real, dimension( * ) BERR_OUT, integer
      N_NORMS, real, dimension( nrhs, * ) ERR_BNDS_NORM, real, dimension(
      nrhs, * ) ERR_BNDS_COMP, complex, dimension( * ) RES, real,
      dimension(*) AYB, complex, dimension( * ) DY, complex, dimension( * )
      Y_TAIL, real RCOND, integer ITHRESH, real RTHRESH, real DZ_UB, logical
      IGNORE_CWISE, integer INFO)
      CLA_GBRFSX_EXTENDED improves the computed solution to a system of
      linear equations for general banded matrices by performing extra-
      precise iterative refinement and provides error bounds and backward
      error estimates for the solution.

Purpose:
   CLA_GBRFSX_EXTENDED improves the computed solution to a system of
   linear equations by performing extra-precise iterative refinement
   and provides error bounds and backward error estimates for the solution.
   This subroutine is called by CGBRFSX to perform iterative refinement.
   In addition to normwise error bound, the code provides maximum
   componentwise error bound if possible. See comments for ERR_BNDS_NORM
   and ERR_BNDS_COMP for details of the error bounds. Note that this
   subroutine is only resonsible for setting the second fields of
   ERR_BNDS_NORM and ERR_BNDS_COMP.

Parameters:

 PREC_TYPE
   PREC_TYPE is INTEGER
   Specifies the intermediate precision to be used in refinement.
   The value is defined by ILAPREC(P) where P is a CHARACTER and
   P    = 'S':  Single
        = 'D':  Double
        = 'I':  Indigenous
        = 'X', 'E':  Extra

 TRANS_TYPE
   TRANS_TYPE is INTEGER
   Specifies the transposition operation on A.
   The value is defined by ILATRANS(T) where T is a CHARACTER and
   T    = 'N':  No transpose
        = 'T':  Transpose
        = 'C':  Conjugate transpose

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.
```

```
KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0

NRHS
  NRHS is INTEGER
  The number of right-hand-sides, i.e., the number of columns of the
  matrix B.

AB
  AB is COMPLEX array, dimension (LDAB,N)
  On entry, the N-by-N matrix AB.

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= max(1,N).

AFB
  AFB is COMPLEX array, dimension (LDAF,N)
  The factors L and U from the factorization
  A = P*L*U as computed by CGBTRF.

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from the factorization A = P*L*U
  as computed by CGBTRF; row i of the matrix was interchanged
  with row IPIV(i).

COLEQU
  COLEQU is LOGICAL
  If .TRUE. then column equilibration was done to A before calling
  this routine. This is needed to compute the solution and error
  bounds correctly.

C
  C is REAL array, dimension (N)
  The column scale factors for A. If COLEQU = .FALSE., C
  is not accessed. If C is input, each element of C should be a power
  of the radix to ensure a reliable solution and error estimates.
  Scaling by powers of the radix does not cause rounding errors unless
  the result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
```

```
  B is COMPLEX array, dimension (LDB,NRHS)
  The right-hand-side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

Y
  Y is COMPLEX array, dimension (LDY,NRHS)
  On entry, the solution matrix X, as computed by CGBTRS.
  On exit, the improved solution matrix Y.

LDY
  LDY is INTEGER
  The leading dimension of the array Y.  LDY >= max(1,N).

BERR_OUT
  BERR_OUT is REAL array, dimension (NRHS)
  On exit, BERR_OUT(j) contains the componentwise relative backward
  error for right-hand-side j from the formula
  max(i) ( abs(RES(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
  where abs(Z) is the componentwise absolute value of the matrix
  or vector Z. This is computed by CLA_LIN_BERR.

N_NORMS
  N_NORMS is INTEGER
  Determines which error bounds to return (see ERR_BNDS_NORM
  and ERR_BNDS_COMP).
  If N_NORMS >= 1 return normwise error bounds.
  If N_NORMS >= 2 return componentwise error bounds.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
           max_j (abs(XTRUE(j,i) - X(j,i)))
           ------------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
```

```
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
          reciprocal condition number is less than the threshold
          sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
          almost certainly within a factor of 10 of the true error
          so long as the next entry is greater than the threshold
          sqrt(n) * slamch('Epsilon'). This error bound should only
          be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
           reciprocal condition number.  Compared with the threshold
           sqrt(n) * slamch('Epsilon') to determine if the error
           estimate is "guaranteed". These reciprocal condition
           numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
           appropriately scaled matrix Z.
           Let Z = S*A, where S scales each row by a power of the
           radix so all absolute row sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

ERR_BNDS_COMP
  ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                  abs(XTRUE(j,i) - X(j,i))
           max_j ----------------------
                        abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
          reciprocal condition number is less than the threshold
          sqrt(n) * slamch('Epsilon').
```

```
  err = 2 "Guaranteed" error bound: The estimated forward error,
          almost certainly within a factor of 10 of the true error
          so long as the next entry is greater than the threshold
          sqrt(n) * slamch('Epsilon'). This error bound should only
          be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
           reciprocal condition number.  Compared with the threshold
           sqrt(n) * slamch('Epsilon') to determine if the error
           estimate is "guaranteed". These reciprocal condition
           numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
           appropriately scaled matrix Z.
           Let Z = S*(A*diag(x)), where x is the solution for the
           current right-hand side and S scales each row of
           A*diag(x) by a power of the radix so all absolute row
           sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

RES
  RES is COMPLEX array, dimension (N)
  Workspace to hold the intermediate residual.

AYB
  AYB is REAL array, dimension (N)
  Workspace.

DY
  DY is COMPLEX array, dimension (N)
  Workspace to hold the intermediate solution.

Y_TAIL
  Y_TAIL is COMPLEX array, dimension (N)
  Workspace to hold the trailing bits of the intermediate solution.

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

ITHRESH
  ITHRESH is INTEGER
  The maximum number of residual computations allowed for
  refinement. The default is 10. For 'aggressive' set to 100 to
  permit convergence using approximate factorizations or
  factorizations other than LU. If the factorization uses a
```

```
  technique other than Gaussian elimination, the guarantees in
  ERR_BNDS_NORM and ERR_BNDS_COMP may no longer be trustworthy.

RTHRESH
  RTHRESH is REAL
  Determines when to stop refinement if the error estimate stops
  decreasing. Refinement will stop when the next solution no longer
  satisfies norm(dx_{i+1}) < RTHRESH * norm(dx_i) where norm(Z) is
  the infinity norm of Z. RTHRESH satisfies 0 < RTHRESH <= 1. The
  default value is 0.5. For 'aggressive' set to 0.9 to permit
  convergence on extremely ill-conditioned matrices. See LAWN 165
  for more details.

DZ_UB
  DZ_UB is REAL
  Determines when to start considering componentwise convergence.
  Componentwise convergence is only considered after each component
  of the solution Y is stable, which we definte as the relative
  change in each component being less than DZ_UB. The default value
  is 0.25, requiring the first bit to be stable. See LAWN 165 for
  more details.

IGNORE_CWISE
  IGNORE_CWISE is LOGICAL
  If .TRUE. then ignore componentwise convergence. Default value
  is .FALSE..

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  < 0:  if INFO = -i, the ith argument to CGBTRS had an illegal value
```

———-

46.15.3 fortran code

— cla_gbrfsx_extended.f —

```
*  ==================================================================
      SUBROUTINE CLA_GBRFSX_EXTENDED ( PREC_TYPE, TRANS_TYPE, N, KL, KU,
     $                                 NRHS, AB, LDAB, AFB, LDAFB, IPIV,
     $                                 COLEQU, C, B, LDB, Y, LDY,
     $                                 BERR_OUT, N_NORMS, ERR_BNDS_NORM,
     $                                 ERR_BNDS_COMP, RES, AYB, DY,
     $                                 Y_TAIL, RCOND, ITHRESH, RTHRESH,
     $                                 DZ_UB, IGNORE_CWISE, INFO )
```

```
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDAB, LDAFB, LDB, LDY, N, KL, KU, NRHS,
     $                   PREC_TYPE, TRANS_TYPE, N_NORMS, ITHRESH
      LOGICAL            COLEQU, IGNORE_CWISE
      REAL               RTHRESH, DZ_UB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   Y( LDY, * ), RES( * ), DY( * ), Y_TAIL( * )
      REAL               C( * ), AYB(*), RCOND, BERR_OUT( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      CHARACTER          TRANS
      INTEGER            CNT, I, J, M, X_STATE, Z_STATE, Y_PREC_STATE
      REAL               YK, DYK, YMIN, NORMY, NORMX, NORMDX, DXRAT,
     $                   DZRAT, PREVNORMDX, PREV_DZ_Z, DXRATMAX,
     $                   DZRATMAX, DX_X, DZ_Z, FINAL_DX_X, FINAL_DZ_Z,
     $                   EPS, HUGEVAL, INCR_THRESH
      LOGICAL            INCR_PREC
      COMPLEX            ZDUM
*     ..
*     .. Parameters ..
      INTEGER            UNSTABLE_STATE, WORKING_STATE, CONV_STATE,
     $                   NOPROG_STATE, BASE_RESIDUAL, EXTRA_RESIDUAL,
     $                   EXTRA_Y
      PARAMETER          ( UNSTABLE_STATE = 0, WORKING_STATE = 1,
     $                   CONV_STATE = 2, NOPROG_STATE = 3 )
      PARAMETER          ( BASE_RESIDUAL = 0, EXTRA_RESIDUAL = 1,
     $                   EXTRA_Y = 2 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
```

```
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           CAXPY, CCOPY, CGBTRS, CGBMV, BLAS_CGBMV_X,
     $                   BLAS_CGBMV2_X, CLA_GBAMV, CLA_WWADDW, SLAMCH,
     $                   CHLA_TRANSTYPE, CLA_LIN_BERR
      REAL               SLAMCH
      CHARACTER          CHLA_TRANSTYPE
*     ..
*     .. Intrinsic Functions..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      IF (INFO.NE.0) RETURN
      TRANS = CHLA_TRANSTYPE(TRANS_TYPE)
      EPS = SLAMCH( 'Epsilon' )
      HUGEVAL = SLAMCH( 'Overflow' )
*     Force HUGEVAL to Inf
      HUGEVAL = HUGEVAL * HUGEVAL
*     Using HUGEVAL may lead to spurious underflows.
      INCR_THRESH = REAL( N ) * EPS
      M = KL+KU+1

      DO J = 1, NRHS
         Y_PREC_STATE = EXTRA_RESIDUAL
         IF ( Y_PREC_STATE .EQ. EXTRA_Y ) then
            DO I = 1, N
               Y_TAIL( I ) = 0.0
            END DO
         END IF

         DXRAT = 0.0E+0
         DXRATMAX = 0.0E+0
         DZRAT = 0.0E+0
         DZRATMAX = 0.0E+0
         FINAL_DX_X = HUGEVAL
```

```
         FINAL_DZ_Z = HUGEVAL
         PREVNORMDX = HUGEVAL
         PREV_DZ_Z = HUGEVAL
         DZ_Z = HUGEVAL
         DX_X = HUGEVAL

         X_STATE = WORKING_STATE
         Z_STATE = UNSTABLE_STATE
         INCR_PREC = .FALSE.

         DO CNT = 1, ITHRESH
*
*        Compute residual RES = B_s - op(A_s) * Y,
*            op(A) = A, A**T, or A**H depending on TRANS (and type).
*
            CALL CCOPY( N, B( 1, J ), 1, RES, 1 )
            IF ( Y_PREC_STATE .EQ. BASE_RESIDUAL ) THEN
               CALL CGBMV( TRANS, M, N, KL, KU, (-1.0E+0,0.0E+0), AB,
     $              LDAB, Y( 1, J ), 1, (1.0E+0,0.0E+0), RES, 1 )
            ELSE IF ( Y_PREC_STATE .EQ. EXTRA_RESIDUAL ) THEN
               CALL BLAS_CGBMV_X( TRANS_TYPE, N, N, KL, KU,
     $              (-1.0E+0,0.0E+0), AB, LDAB, Y( 1, J ), 1,
     $              (1.0E+0,0.0E+0), RES, 1, PREC_TYPE )
            ELSE
               CALL BLAS_CGBMV2_X( TRANS_TYPE, N, N, KL, KU,
     $              (-1.0E+0,0.0E+0), AB, LDAB, Y( 1, J ), Y_TAIL, 1,
     $              (1.0E+0,0.0E+0), RES, 1, PREC_TYPE )
            END IF

!        XXX: RES is no longer needed.
            CALL CCOPY( N, RES, 1, DY, 1 )
            CALL CGBTRS( TRANS, N, KL, KU, 1, AFB, LDAFB, IPIV, DY, N,
     $           INFO )
*
*         Calculate relative changes DX_X, DZ_Z and ratios DXRAT, DZRAT.
*
            NORMX = 0.0E+0
            NORMY = 0.0E+0
            NORMDX = 0.0E+0
            DZ_Z = 0.0E+0
            YMIN = HUGEVAL

            DO I = 1, N
               YK = CABS1( Y( I, J ) )
               DYK = CABS1( DY( I ) )

               IF (YK .NE. 0.0) THEN
                  DZ_Z = MAX( DZ_Z, DYK / YK )
               ELSE IF ( DYK .NE. 0.0 ) THEN
                  DZ_Z = HUGEVAL
```

```
            END IF

            YMIN = MIN( YMIN, YK )

            NORMY = MAX( NORMY, YK )

            IF ( COLEQU ) THEN
               NORMX = MAX( NORMX, YK * C( I ) )
               NORMDX = MAX(NORMDX, DYK * C(I))
            ELSE
               NORMX = NORMY
               NORMDX = MAX( NORMDX, DYK )
            END IF
         END DO

         IF ( NORMX .NE. 0.0 ) THEN
            DX_X = NORMDX / NORMX
         ELSE IF ( NORMDX .EQ. 0.0 ) THEN
            DX_X = 0.0
         ELSE
            DX_X = HUGEVAL
         END IF

         DXRAT = NORMDX / PREVNORMDX
         DZRAT = DZ_Z / PREV_DZ_Z
*
*        Check termination criteria.
*
         IF (.NOT.IGNORE_CWISE
     $          .AND. YMIN*RCOND .LT. INCR_THRESH*NORMY
     $          .AND. Y_PREC_STATE .LT. EXTRA_Y )
     $          INCR_PREC = .TRUE.

         IF ( X_STATE .EQ. NOPROG_STATE .AND. DXRAT .LE. RTHRESH )
     $          X_STATE = WORKING_STATE
         IF ( X_STATE .EQ. WORKING_STATE ) THEN
            IF ( DX_X .LE. EPS ) THEN
               X_STATE = CONV_STATE
            ELSE IF ( DXRAT .GT. RTHRESH ) THEN
               IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                  INCR_PREC = .TRUE.
               ELSE
                  X_STATE = NOPROG_STATE
               END IF
            ELSE
               IF ( DXRAT .GT. DXRATMAX ) DXRATMAX = DXRAT
            END IF
            IF ( X_STATE .GT. WORKING_STATE ) FINAL_DX_X = DX_X
         END IF
```

```
            IF ( Z_STATE .EQ. UNSTABLE_STATE .AND. DZ_Z .LE. DZ_UB )
     $           Z_STATE = WORKING_STATE
            IF ( Z_STATE .EQ. NOPROG_STATE .AND. DZRAT .LE. RTHRESH )
     $           Z_STATE = WORKING_STATE
            IF ( Z_STATE .EQ. WORKING_STATE ) THEN
               IF ( DZ_Z .LE. EPS ) THEN
                  Z_STATE = CONV_STATE
               ELSE IF ( DZ_Z .GT. DZ_UB ) THEN
                  Z_STATE = UNSTABLE_STATE
                  DZRATMAX = 0.0
                  FINAL_DZ_Z = HUGEVAL
               ELSE IF ( DZRAT .GT. RTHRESH ) THEN
                  IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                     INCR_PREC = .TRUE.
                  ELSE
                     Z_STATE = NOPROG_STATE
                  END IF
               ELSE
                  IF ( DZRAT .GT. DZRATMAX ) DZRATMAX = DZRAT
               END IF
               IF ( Z_STATE .GT. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
            END IF
*
*           Exit if both normwise and componentwise stopped working,
*           but if componentwise is unstable, let it go at least two
*           iterations.
*
            IF ( X_STATE.NE.WORKING_STATE ) THEN
               IF ( IGNORE_CWISE ) GOTO 666
               IF ( Z_STATE.EQ.NOPROG_STATE .OR. Z_STATE.EQ.CONV_STATE )
     $              GOTO 666
               IF ( Z_STATE.EQ.UNSTABLE_STATE .AND. CNT.GT.1 ) GOTO 666
            END IF

            IF ( INCR_PREC ) THEN
               INCR_PREC = .FALSE.
               Y_PREC_STATE = Y_PREC_STATE + 1
               DO I = 1, N
                  Y_TAIL( I ) = 0.0
               END DO
            END IF

            PREVNORMDX = NORMDX
            PREV_DZ_Z = DZ_Z
*
*           Update soluton.
*
            IF ( Y_PREC_STATE .LT. EXTRA_Y ) THEN
               CALL CAXPY( N, (1.0E+0,0.0E+0), DY, 1, Y(1,J), 1 )
            ELSE
```

```
               CALL CLA_WWADDW( N, Y(1,J), Y_TAIL, DY )
            END IF

         END DO
*        Target of "IF (Z_STOP .AND. X_STOP)".  Sun's f77 won't EXIT.
 666     CONTINUE
*
*     Set final_* when cnt hits ithresh.
*
         IF ( X_STATE .EQ. WORKING_STATE ) FINAL_DX_X = DX_X
         IF ( Z_STATE .EQ. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
*
*     Compute error bounds.
*
         IF ( N_NORMS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) =
     $           FINAL_DX_X / (1 - DXRATMAX)
         END IF
         IF ( N_NORMS .GE. 2 ) THEN
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) =
     $           FINAL_DZ_Z / (1 - DZRATMAX)
         END IF
*
*     Compute componentwise relative backward error from formula
*         max(i) ( abs(R(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
*     where abs(Z) is the componentwise absolute value of the matrix
*     or vector Z.
*
*        Compute residual RES = B_s - op(A_s) * Y,
*            op(A) = A, A**T, or A**H depending on TRANS (and type).
*
         CALL CCOPY( N, B( 1, J ), 1, RES, 1 )
         CALL CGBMV( TRANS, N, N, KL, KU, (-1.0E+0,0.0E+0), AB, LDAB,
     $        Y(1,J), 1, (1.0E+0,0.0E+0), RES, 1 )

         DO I = 1, N
            AYB( I ) = CABS1( B( I, J ) )
         END DO
*
*     Compute abs(op(A_s))*abs(Y) + abs(B_s).
*
        CALL CLA_GBAMV( TRANS_TYPE, N, N, KL, KU, 1.0E+0,
     $        AB, LDAB, Y(1, J), 1, 1.0E+0, AYB, 1 )

         CALL CLA_LIN_BERR( N, N, 1, RES, AYB, BERR_OUT( J ) )
*
*     End of loop for each RHS.
*
      END DO
*
```

```
      RETURN
      END
```

———-

46.15.4 lisp code

— **cla_gbrfsx_extended.lisp** —

```
(let* ((unstable_state 0)
       (working_state 1)
       (conv_state 2)
       (noprog_state 3)
       (base_residual 0)
       (extra_residual 1)
       (extra_y 2)
       (la_linrx_err_i 2))
  (declare (type (f2cl-lib:integer4 0 0) unstable_state)
           (type (f2cl-lib:integer4 1 1) working_state)
           (type (f2cl-lib:integer4 2 2) conv_state)
           (type (f2cl-lib:integer4 3 3) noprog_state)
           (type (f2cl-lib:integer4 0 0) base_residual)
           (type (f2cl-lib:integer4 1 1) extra_residual)
           (type (f2cl-lib:integer4 2 2) extra_y)
           (type (f2cl-lib:integer4 2 2) la_linrx_err_i)
           (ignorable unstable_state working_state conv_state noprog_state
            base_residual extra_residual extra_y la_linrx_err_i))
  (defun cla_gbrfsx_extended
         (prec_type trans_type n kl ku nrhs ab ldab afb ldafb ipiv colequ c b
          ldb$ y ldy berr_out n_norms err_bnds_norm err_bnds_comp res ayb dy
          y_tail rcond ithresh rthresh dz_ub ignore_cwise info)
    (declare (type (single-float) dz_ub rthresh rcond)
             (type (array single-float (*)) ayb err_bnds_comp err_bnds_norm
              berr_out c)
             (type f2cl-lib:logical ignore_cwise colequ)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) y_tail dy res y b afb ab)
             (type (f2cl-lib:integer4) info ithresh n_norms ldy ldb$ ldafb ldab
              nrhs ku kl n trans_type prec_type))
    (f2cl-lib:with-multi-array-data
        ((ab f2cl-lib:complex8 ab-%data% ab-%offset%)
         (afb f2cl-lib:complex8 afb-%data% afb-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (y f2cl-lib:complex8 y-%data% y-%offset%)
         (res f2cl-lib:complex8 res-%data% res-%offset%)
         (dy f2cl-lib:complex8 dy-%data% dy-%offset%)
         (y_tail f2cl-lib:complex8 y_tail-%data% y_tail-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
```

```
    (c single-float c-%data% c-%offset%)
    (berr_out single-float berr_out-%data% berr_out-%offset%)
    (err_bnds_norm single-float err_bnds_norm-%data%
     err_bnds_norm-%offset%)
    (err_bnds_comp single-float err_bnds_comp-%data%
     err_bnds_comp-%offset%)
    (ayb single-float ayb-%data% ayb-%offset%))
  (labels ((cabs1 (zdum)
             (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
    (declare
     (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
      cabs1))
    (prog ((zdum #C(0.0 0.0)) (incr_prec nil) (yk 0.0) (dyk 0.0) (ymin 0.0)
           (normy 0.0) (normx 0.0) (normdx 0.0) (dxrat 0.0) (dzrat 0.0)
           (prevnormdx 0.0) (prev_dz_z 0.0) (dxratmax 0.0) (dzratmax 0.0)
           (dx_x 0.0) (dz_z 0.0) (final_dx_x 0.0) (final_dz_z 0.0)
           (eps 0.0) (hugeval 0.0) (incr_thresh 0.0) (cnt 0) (i 0) (j 0)
           (m 0) (x_state 0) (z_state 0) (y_prec_state 0)
           (trans
            (make-array '(1) :element-type 'character :initial-element
                        #\ )))
     (declare (type (f2cl-lib:complex8) zdum)
              (type f2cl-lib:logical incr_prec)
              (type (single-float) yk dyk ymin normy normx normdx dxrat
               dzrat prevnormdx prev_dz_z dxratmax dzratmax dx_x dz_z
               final_dx_x final_dz_z eps hugeval incr_thresh)
              (type (f2cl-lib:integer4) cnt i j m x_state z_state
               y_prec_state)
              (type (string 1) trans))
     (if (/= info 0)
         (go end_label))
     (f2cl-lib:f2cl-set-string trans
                               (f2cl-lib:fref-string #'chla_transtype
                                                     (trans_type))
                               (string 1))
     (setf eps (slamch "Epsilon"))
     (setf hugeval (slamch "Overflow"))
     (setf hugeval (* hugeval hugeval))
     (setf incr_thresh (* (f2cl-lib:freal n) eps))
     (setf m (f2cl-lib:int-add kl ku 1))
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j nrhs) nil)
       (tagbody
         (setf y_prec_state extra_residual)
         (cond
          ((= y_prec_state extra_y)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
```

```
                              y_tail-%offset%)
                 (coerce 0.0 'f2cl-lib:complex8))
     label100001))))
(setf dxrat 0.0)
(setf dxratmax 0.0)
(setf dzrat 0.0)
(setf dzratmax 0.0)
(setf final_dx_x hugeval)
(setf final_dz_z hugeval)
(setf prevnormdx hugeval)
(setf prev_dz_z hugeval)
(setf dz_z hugeval)
(setf dx_x hugeval)
(setf x_state working_state)
(setf z_state unstable_state)
(setf incr_prec f2cl-lib:%false%)
(f2cl-lib:fdo (cnt 1 (f2cl-lib:int-add cnt 1))
              ((> cnt ithresh) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (ccopy n
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1 res 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (cond
     ((= y_prec_state base_residual)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (cgbmv trans m n kl ku (f2cl-lib:cmplx -1.0 0.0) ab
           ldab
           (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                                 ((1 ldy) (1 *)) y-%offset%)
           1 (f2cl-lib:cmplx 1.0 0.0) res 1)
        (declare
         (ignore var-5 var-6 var-8 var-9 var-10 var-11 var-12))
        (when var-0 (setf trans var-0))
        (when var-1 (setf m var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf kl var-3))
        (when var-4 (setf ku var-4))
        (when var-7 (setf ldab var-7))))
     ((= y_prec_state extra_residual)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12 var-13)
          (blas_cgbmv_x trans_type n n kl ku
           (f2cl-lib:cmplx -1.0 0.0) ab ldab
```

```
        (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                              ((1 ldy) (1 *)) y-%offset%)
        1 (f2cl-lib:cmplx 1.0 0.0) res 1 prec_type)
      (declare
       (ignore var-5 var-6 var-8 var-9 var-10 var-11 var-12))
      (when var-0 (setf trans_type var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf kl var-3))
      (when var-4 (setf ku var-4))
      (when var-7 (setf ldab var-7))
      (when var-13 (setf prec_type var-13))))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10 var-11 var-12 var-13 var-14)
       (blas_cgbmv2_x trans_type n n kl ku
        (f2cl-lib:cmplx -1.0 0.0) ab ldab
        (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                              ((1 ldy) (1 *)) y-%offset%)
        y_tail 1 (f2cl-lib:cmplx 1.0 0.0) res 1 prec_type)
      (declare
       (ignore var-5 var-6 var-8 var-9 var-10 var-11 var-12
        var-13))
      (when var-0 (setf trans_type var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf kl var-3))
      (when var-4 (setf ku var-4))
      (when var-7 (setf ldab var-7))
      (when var-14 (setf prec_type var-14)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (ccopy n res 1 dy 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (cgbtrs trans n kl ku 1 afb ldafb ipiv dy n info)
  (declare (ignore var-2 var-3 var-4 var-5 var-7 var-8))
  (setf trans var-0)
  (setf n var-1)
  (setf ldafb var-6)
  (setf n var-9)
  (setf info var-10))
(setf normx 0.0)
(setf normy 0.0)
(setf normdx 0.0)
(setf dz_z 0.0)
(setf ymin hugeval)
```

```
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf yk
            (cabs1
             (f2cl-lib:fref y-%data% (i j) ((1 ldy) (1 *))
                            y-%offset%)))
    (setf dyk
            (cabs1
             (f2cl-lib:fref dy-%data% (i) ((1 *))
                            dy-%offset%)))
    (cond ((/= yk 0.0) (setf dz_z (max dz_z (/ dyk yk))))
          ((/= dyk 0.0) (setf dz_z hugeval)))
    (setf ymin (min ymin yk))
    (setf normy (max normy yk))
    (cond
     (colequ
      (setf normx
              (max normx
                   (* yk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%))))
      (setf normdx
              (max normdx
                   (* dyk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%)))))
     (t (setf normx normy) (setf normdx (max normdx dyk))))
   label100003))
(cond ((/= normx 0.0) (setf dx_x (/ normdx normx)))
      ((= normdx 0.0) (setf dx_x 0.0))
      (t (setf dx_x hugeval)))
(setf dxrat (/ normdx prevnormdx))
(setf dzrat (/ dz_z prev_dz_z))
(if (and (not ignore_cwise)
         (< (* ymin rcond) (* incr_thresh normy))
         (< y_prec_state extra_y))
    (setf incr_prec f2cl-lib:%true%))
(if (and (= x_state noprog_state) (<= dxrat rthresh))
    (setf x_state working_state))
(cond
 ((= x_state working_state)
  (cond ((<= dx_x eps) (setf x_state conv_state))
        ((> dxrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf x_state noprog_state))))
        (t
         (if (> dxrat dxratmax)
```

```
                (setf dxratmax dxrat))))
      (if (> x_state working_state)
          (setf final_dx_x dx_x))))
    (if (and (= z_state unstable_state) (<= dz_z dz_ub))
        (setf z_state working_state))
    (if (and (= z_state noprog_state) (<= dzrat rthresh))
        (setf z_state working_state))
    (cond
     ((= z_state working_state)
      (cond ((<= dz_z eps) (setf z_state conv_state))
            ((> dz_z dz_ub) (setf z_state unstable_state)
             (setf dzratmax 0.0) (setf final_dz_z hugeval))
            ((> dzrat rthresh)
             (cond
              ((/= y_prec_state extra_y)
               (setf incr_prec f2cl-lib:%true%))
              (t (setf z_state noprog_state))))
            (t
             (if (> dzrat dzratmax)
                 (setf dzratmax dzrat))))
      (if (> z_state working_state)
          (setf final_dz_z dz_z))))
    (cond
     ((/= x_state working_state)
      (if ignore_cwise
          (go label666))
      (if (or (= z_state noprog_state) (= z_state conv_state))
          (go label666))
      (if (and (= z_state unstable_state) (> cnt 1))
          (go label666))))
    (cond
     (incr_prec (setf incr_prec f2cl-lib:%false%)
      (setf y_prec_state (f2cl-lib:int-add y_prec_state 1))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                               y_tail-%offset%)
                (coerce 0.0 'f2cl-lib:complex8))
         label100004))))
    (setf prevnormdx normdx)
    (setf prev_dz_z dz_z)
    (cond
     ((< y_prec_state extra_y)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (caxpy n (f2cl-lib:cmplx 1.0 0.0) dy 1
           (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                                 ((1 ldy) (1 *)) y-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4 var-5))
```

```
          (when var-0 (setf n var-0))))
       (t
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (cla_wwaddw n
             (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                                   ((1 ldy) (1 *)) y-%offset%)
             y_tail dy)
          (declare (ignore var-1 var-2 var-3))
          (when var-0 (setf n var-0)))))
    label100002))
label666
 (if (= x_state working_state)
     (setf final_dx_x dx_x))
 (if (= z_state working_state)
     (setf final_dz_z dz_z))
 (cond
  ((>= n_norms 1)
   (setf (f2cl-lib:fref err_bnds_norm-%data% (j la_linrx_err_i)
                        ((1 nrhs) (1 *)) err_bnds_norm-%offset%)
         (/ final_dx_x (- 1 dxratmax)))))
 (cond
  ((>= n_norms 2)
   (setf (f2cl-lib:fref err_bnds_comp-%data% (j la_linrx_err_i)
                        ((1 nrhs) (1 *)) err_bnds_comp-%offset%)
         (/ final_dz_z (- 1 dzratmax)))))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (ccopy n
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 j)
                            ((1 ldb$) (1 *)) b-%offset%)
      1 res 1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf n var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12)
     (cgbmv trans n n kl ku (f2cl-lib:cmplx -1.0 0.0) ab ldab
      (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                            ((1 ldy) (1 *)) y-%offset%)
      1 (f2cl-lib:cmplx 1.0 0.0) res 1)
   (declare (ignore var-5 var-6 var-8 var-9 var-10 var-11 var-12))
   (when var-0 (setf trans var-0))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-3 (setf kl var-3))
   (when var-4 (setf ku var-4))
   (when var-7 (setf ldab var-7)))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf (f2cl-lib:fref ayb-%data% (i) ((1 *)) ayb-%offset%)
```

```
                        (cabs1
                         (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                        b-%offset%)))
                  label100005))
              (cla_gbamv trans_type n n kl ku 1.0 ab ldab
               (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                                     ((1 ldy) (1 *)) y-%offset%)
               1 1.0 ayb 1)
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                  (cla_lin_berr n n 1 res ayb
                   (f2cl-lib:array-slice berr_out-%data% single-float (j)
                                         ((1 *)) berr_out-%offset%))
                (declare (ignore var-2 var-3 var-4 var-5))
                (when var-0 (setf n var-0))
                (when var-1 (setf n var-1)))
            label100000))
          (go end_label)
         end_label
          (return
           (values prec_type trans_type n kl ku nil nil ldab nil ldafb nil nil
                   nil nil nil nil nil nil nil nil nil nil nil nil nil nil nil
                   nil nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cla_gbrfsx_extended
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              fortran-to-lisp::logical
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
```

```
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::complex8
   (*))
  (array single-float (*))
  (array fortran-to-lisp::complex8
   (*))
  (array fortran-to-lisp::complex8
   (*))
  (single-float)
  (fortran-to-lisp::integer4)
  (single-float) (single-float)
  fortran-to-lisp::logical
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::prec_type
  fortran-to-lisp::trans_type
  fortran-to-lisp::n
  fortran-to-lisp::kl
  fortran-to-lisp::ku nil nil
  fortran-to-lisp::ldab nil
  fortran-to-lisp::ldafb nil nil
  nil nil nil nil nil nil nil nil
  nil nil nil nil nil nil nil nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::cla_gbamv
  fortran-to-lisp::cgbtrs))))
```

———-

46.16 cla_gbrpvgrw

46.16.1 Axiom unit tests

— clagbrpvgrw.input —

```
)set break resume
)sys rm -f clagbrpvgrw.output
)spool clagbrpvgrw.output
)set message test on
)set message auto off
```

```
)clear all

)spool
)lisp (bye)
```

———-

46.16.2 Axiom help page

— **cla_gbrpvgrw.help** —

```
=====================================================================
cla_gbrpvgrw examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       real function cla_gbrpvgrw (N, KL, KU, NCOLS, AB, LDAB, AFB, LDAFB)
           CLA_GBRPVGRW computes the reciprocal pivot growth factor
           norm(A)/norm(U) for a general banded matrix.

Function/Subroutine Documentation
   real function cla_gbrpvgrw (integer N, integer KL, integer KU, integer
       NCOLS, complex, dimension( ldab, * ) AB, integer LDAB, complex,
       dimension( ldafb, * ) AFB, integer LDAFB)
       CLA_GBRPVGRW computes the reciprocal pivot growth factor
       norm(A)/norm(U) for a general banded matrix.

Purpose:
   CLA_GBRPVGRW computes the reciprocal pivot growth factor
   norm(A)/norm(U). The "max absolute element" norm is used. If this is
   much less than 1, the stability of the LU factorization of the
   (equilibrated) matrix A could be poor. This also means that the
   solution X, estimated condition numbers, and error bounds could be
   unreliable.

Parameters:

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 KL
```

```
  KL is INTEGER
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

NCOLS
  NCOLS is INTEGER
  The number of columns of the matrix A.  NCOLS >= 0.

AB
  AB is COMPLEX array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is COMPLEX array, dimension (LDAFB,N)
  Details of the LU factorization of the band matrix A, as
  computed by CGBTRF.  U is stored as an upper triangular
  band matrix with KL+KU superdiagonals in rows 1 to KL+KU+1,
  and the multipliers used during the factorization are stored
  in rows KL+KU+2 to 2*KL+KU+1.

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL+KU+1.
```

———-

46.16.3 fortran code

— cla_gbrpvgrw.f —

```
*  =====================================================================
      REAL FUNCTION CLA_GBRPVGRW( N, KL, KU, NCOLS, AB, LDAB, AFB,
     $                            LDAFB )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
```

```
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            N, KL, KU, NCOLS, LDAB, LDAFB
*     ..
*     .. Array Arguments ..
      COMPLEX            AB( LDAB, * ), AFB( LDAFB, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J, KD
      REAL               AMAX, UMAX, RPVGRW
      COMPLEX            ZDUM
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, REAL, AIMAG
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      RPVGRW = 1.0

      KD = KU + 1
      DO J = 1, NCOLS
         AMAX = 0.0
         UMAX = 0.0
         DO I = MAX( J-KU, 1 ), MIN( J+KL, N )
            AMAX = MAX( CABS1( AB( KD+I-J, J ) ), AMAX )
         END DO
         DO I = MAX( J-KU, 1 ), J
            UMAX = MAX( CABS1( AFB( KD+I-J, J ) ), UMAX )
         END DO
         IF ( UMAX /= 0.0 ) THEN
            RPVGRW = MIN( AMAX / UMAX, RPVGRW )
         END IF
      END DO
      CLA_GBRPVGRW = RPVGRW
      END
```

———-

46.16.4 lisp code

— cla_gbrpvgrw.lisp —

```
(defun cla_gbrpvgrw (n kl ku ncols ab ldab afb ldafb)
  (declare (type (array f2cl-lib:complex8 (*)) afb ab)
           (type (f2cl-lib:integer4) ldafb ldab ncols ku kl n))
  (f2cl-lib:with-multi-array-data
      ((ab f2cl-lib:complex8 ab-%data% ab-%offset%)
       (afb f2cl-lib:complex8 afb-%data% afb-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
        cabs1))
      (prog ((zdum #C(0.0 0.0)) (amax 0.0) (umax 0.0) (rpvgrw 0.0) (i 0) (j 0)
             (kd 0) (cla_gbrpvgrw 0.0))
        (declare (type (f2cl-lib:integer4) kd j i)
                 (type (single-float) cla_gbrpvgrw rpvgrw umax amax)
                 (type (f2cl-lib:complex8) zdum))
        (setf rpvgrw 1.0)
        (setf kd (f2cl-lib:int-add ku 1))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j ncols) nil)
          (tagbody
            (setf amax 0.0)
            (setf umax 0.0)
            (f2cl-lib:fdo (i
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                            (the f2cl-lib:integer4 1))
                           (f2cl-lib:int-add i 1))
                          ((> i
                              (min
                               (the f2cl-lib:integer4 (f2cl-lib:int-add j kl))
                               (the f2cl-lib:integer4 n)))
                           nil)
              (tagbody
                (setf amax
                        (max
                         (cabs1
                          (f2cl-lib:fref ab-%data%
                                         ((f2cl-lib:int-sub
                                           (f2cl-lib:int-add kd i) j)
                                          j)
                                         ((1 ldab) (1 *)) ab-%offset%))
                         amax))
               label100001))
```

```
            (f2cl-lib:fdo (i
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                            (the f2cl-lib:integer4 1))
                           (f2cl-lib:int-add i 1))
                          ((> i j) nil)
              (tagbody
                (setf umax
                        (max
                         (cabs1
                          (f2cl-lib:fref afb-%data%
                                         ((f2cl-lib:int-sub
                                           (f2cl-lib:int-add kd i) j)
                                          j)
                                         ((1 ldafb) (1 *)) afb-%offset%))
                         umax))
               label100002))
            (cond
             ((f2cl-lib:f2cl/ umax (= 0.0))
              (setf rpvgrw (min (/ amax umax) rpvgrw))))
           label100000))
        (setf cla_gbrpvgrw rpvgrw)
       end_label
        (return (values cla_gbrpvgrw nil nil nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cla_gbrpvgrw
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil)
                                            :calls 'nil)))
```

———-

46.17 cungbr

46.17.1 Axiom unit tests

— **cungbr.input** —

```
)set break resume
)sys rm -f cungbr.output
)spool cungbr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

46.17.2 Axiom help page

— **cungbr.help** —

```
======================================================================
cungbr examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cungbr (VECT, M, N, K, A, LDA, TAU, WORK, LWORK, INFO)
           CUNGBR

Function/Subroutine Documentation
   subroutine cungbr (character VECT, integer M, integer N, integer K,
       complex, dimension( lda, * ) A, integer LDA, complex, dimension( * )
       TAU, complex, dimension( * ) WORK, integer LWORK, integer INFO)
       CUNGBR

Purpose:
   CUNGBR generates one of the complex unitary matrices Q or P**H
   determined by CGEBRD when reducing a complex matrix A to bidiagonal
   form: A = Q * B * P**H.  Q and P**H are defined as products of
   elementary reflectors H(i) or G(i) respectively.
```

```
  If VECT = 'Q', A is assumed to have been an M-by-K matrix, and Q
  is of order M:
  if m >= k, Q = H(1) H(2) . . . H(k) and CUNGBR returns the first n
  columns of Q, where m >= n >= k;
  if m < k, Q = H(1) H(2) . . . H(m-1) and CUNGBR returns Q as an
  M-by-M matrix.

  If VECT = 'P', A is assumed to have been a K-by-N matrix, and P**H
  is of order N:
  if k < n, P**H = G(k) . . . G(2) G(1) and CUNGBR returns the first m
  rows of P**H, where n >= m >= k;
  if k >= n, P**H = G(n-1) . . . G(2) G(1) and CUNGBR returns P**H as
  an N-by-N matrix.

Parameters:

 VECT
   VECT is CHARACTER*1
   Specifies whether the matrix Q or the matrix P**H is
   required, as defined in the transformation applied by CGEBRD:
   = 'Q':  generate Q;
   = 'P':  generate P**H.

 M
   M is INTEGER
   The number of rows of the matrix Q or P**H to be returned.
   M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix Q or P**H to be returned.
   N >= 0.
   If VECT = 'Q', M >= N >= min(M,K);
   if VECT = 'P', N >= M >= min(N,K).

 K
   K is INTEGER
   If VECT = 'Q', the number of columns in the original M-by-K
   matrix reduced by CGEBRD.
   If VECT = 'P', the number of rows in the original K-by-N
   matrix reduced by CGEBRD.
   K >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the vectors which define the elementary reflectors,
   as returned by CGEBRD.
   On exit, the M-by-N matrix Q or P**H.
```

```
LDA
  LDA is INTEGER
  The leading dimension of the array A. LDA >= M.

TAU
  TAU is COMPLEX array, dimension
     (min(M,K)) if VECT = 'Q'
     (min(N,K)) if VECT = 'P'
  TAU(i) must contain the scalar factor of the elementary
  reflector H(i) or G(i), which determines Q or P**H, as
  returned by CGEBRD in its array argument TAUQ or TAUP.

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK >= max(1,min(M,N)).
  For optimum performance LWORK >= min(M,N)*NB, where NB
  is the optimal blocksize.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

46.17.3 fortran code

— cungbr.f —

```
*  ===================================================================
      SUBROUTINE CUNGBR( VECT, M, N, K, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
```

```
      CHARACTER          VECT
      INTEGER            INFO, K, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ZERO, ONE
      PARAMETER          ( ZERO = ( 0.0E+0, 0.0E+0 ),
     $                   ONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, WANTQ
      INTEGER            I, IINFO, J, LWKOPT, MN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           ILAENV, LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CUNGLQ, CUNGQR, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      WANTQ = LSAME( VECT, 'Q' )
      MN = MIN( M, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.WANTQ .AND. .NOT.LSAME( VECT, 'P' ) ) THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 .OR. ( WANTQ .AND. ( N.GT.M .OR. N.LT.MIN( M,
     $         K ) ) ) .OR. ( .NOT.WANTQ .AND. ( M.GT.N .OR. M.LT.
     $         MIN( N, K ) ) ) ) THEN
         INFO = -3
      ELSE IF( K.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -6
      ELSE IF( LWORK.LT.MAX( 1, MN ) .AND. .NOT.LQUERY ) THEN
```

```
         INFO = -9
      END IF
*
      IF( INFO.EQ.0 ) THEN
         WORK( 1 ) = 1
         IF( WANTQ ) THEN
            IF( M.GE.K ) THEN
               CALL CUNGQR( M, N, K, A, LDA, TAU, WORK, -1, IINFO )
            ELSE
               IF( M.GT.1 ) THEN
                  CALL CUNGQR( M-1, M-1, M-1, A( 2, 2 ), LDA, TAU, WORK,
     $                         -1, IINFO )
               END IF
            END IF
         ELSE
            IF( K.LT.N ) THEN
               CALL CUNGLQ( M, N, K, A, LDA, TAU, WORK, -1, IINFO )
            ELSE
               IF( N.GT.1 ) THEN
                  CALL CUNGLQ( N-1, N-1, N-1, A( 2, 2 ), LDA, TAU, WORK,
     $                         -1, IINFO )
               END IF
            END IF
         END IF
         LWKOPT = WORK( 1 )
         LWKOPT = MAX (LWKOPT, MN)
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CUNGBR', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         WORK( 1 ) = LWKOPT
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      IF( WANTQ ) THEN
*
*        Form Q, determined by a call to CGEBRD to reduce an m-by-k
*        matrix
*
         IF( M.GE.K ) THEN
*
```

```
*          If m >= k, assume m >= n >= k
*
           CALL CUNGQR( M, N, K, A, LDA, TAU, WORK, LWORK, IINFO )
*
        ELSE
*
*          If m < k, assume m = n
*
*          Shift the vectors which define the elementary reflectors one
*          column to the right, and set the first row and column of Q
*          to those of the unit matrix
*
           DO 20 J = M, 2, -1
              A( 1, J ) = ZERO
              DO 10 I = J + 1, M
                 A( I, J ) = A( I, J-1 )
   10         CONTINUE
   20      CONTINUE
           A( 1, 1 ) = ONE
           DO 30 I = 2, M
              A( I, 1 ) = ZERO
   30      CONTINUE
           IF( M.GT.1 ) THEN
*
*             Form Q(2:m,2:m)
*
              CALL CUNGQR( M-1, M-1, M-1, A( 2, 2 ), LDA, TAU, WORK,
     $                     LWORK, IINFO )
           END IF
        END IF
     ELSE
*
*       Form P**H, determined by a call to CGEBRD to reduce a k-by-n
*       matrix
*
        IF( K.LT.N ) THEN
*
*          If k < n, assume k <= m <= n
*
           CALL CUNGLQ( M, N, K, A, LDA, TAU, WORK, LWORK, IINFO )
*
        ELSE
*
*          If k >= n, assume m = n
*
*          Shift the vectors which define the elementary reflectors one
*          row downward, and set the first row and column of P**H to
*          those of the unit matrix
*
           A( 1, 1 ) = ONE
```

```
            DO 40 I = 2, N
               A( I, 1 ) = ZERO
   40       CONTINUE
            DO 60 J = 2, N
               DO 50 I = J - 1, 2, -1
                  A( I, J ) = A( I-1, J )
   50          CONTINUE
               A( 1, J ) = ZERO
   60       CONTINUE
            IF( N.GT.1 ) THEN
*
*              Form P**H(2:n,2:n)
*
               CALL CUNGLQ( N-1, N-1, N-1, A( 2, 2 ), LDA, TAU, WORK,
     $                      LWORK, IINFO )
            END IF
         END IF
      END IF
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of CUNGBR
*
      END
```

———-

46.17.4 lisp code

— cungbr.lisp —

```
(let* ((zero (f2cl-lib:cmplx 0.0 0.0)) (one (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:complex8) zero)
           (type (f2cl-lib:complex8) one)
           (ignorable zero one))
  (defun cungbr (vect m n k a lda tau work lwork info)
    (declare (type (array f2cl-lib:complex8 (*)) work tau a)
             (type (f2cl-lib:integer4) info lwork lda k n m)
             (type (string *) vect))
    (f2cl-lib:with-multi-array-data
        ((vect character vect-%data% vect-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (tau f2cl-lib:complex8 tau-%data% tau-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%))
      (prog ((i 0) (iinfo 0) (j 0) (lwkopt 0) (mn 0) (lquery nil) (wantq nil))
        (declare (type (f2cl-lib:integer4) i iinfo j lwkopt mn)
                 (type f2cl-lib:logical lquery wantq))
        (setf info 0)
```

```
(setf wantq
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame vect "Q")
          (declare (ignore var-1))
          (when var-0 (setf vect var-0))
          ret-val))
(setf mn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond
 ((and (not wantq)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame vect "P")
          (declare (ignore var-1))
          (when var-0 (setf vect var-0))
          ret-val)))
  (setf info -1))
 ((< m 0) (setf info -2))
 ((or (< n 0)
      (and wantq
           (or (> n m)
               (< n
                  (min (the f2cl-lib:integer4 m)
                       (the f2cl-lib:integer4 k)))))
      (and (not wantq)
           (or (> m n)
               (< m
                  (min (the f2cl-lib:integer4 n)
                       (the f2cl-lib:integer4 k))))))
  (setf info -3))
 ((< k 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -6))
 ((and
   (< lwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 mn)))
   (not lquery))
  (setf info -9)))
(cond
 ((= info 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce 1 'f2cl-lib:complex8))
  (cond
   (wantq
    (cond
     ((>= m k)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (cungqr m n k a lda tau work -1 iinfo)
        (declare (ignore var-3 var-5 var-6 var-7))
        (when var-0 (setf m var-0))
```

```
          (when var-1 (setf n var-1))
          (when var-2 (setf k var-2))
          (when var-4 (setf lda var-4))
          (when var-8 (setf iinfo var-8))))
        (t
         (cond
          ((> m 1)
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
               (cungqr (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
                (f2cl-lib:int-sub m 1)
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (2 2)
                                      ((1 lda) (1 *)) a-%offset%)
                lda tau work -1 iinfo)
             (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
             (when var-4 (setf lda var-4))
             (when var-8 (setf iinfo var-8))))))))
      (t
       (cond
        ((< k n)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
             (cunglq m n k a lda tau work -1 iinfo)
           (declare (ignore var-3 var-5 var-6 var-7))
           (when var-0 (setf m var-0))
           (when var-1 (setf n var-1))
           (when var-2 (setf k var-2))
           (when var-4 (setf lda var-4))
           (when var-8 (setf iinfo var-8))))
        (t
         (cond
          ((> n 1)
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
               (cunglq (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
                (f2cl-lib:int-sub n 1)
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (2 2)
                                      ((1 lda) (1 *)) a-%offset%)
                lda tau work -1 iinfo)
             (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
             (when var-4 (setf lda var-4))
             (when var-8 (setf iinfo var-8)))))))))
    (setf lwkopt
            (f2cl-lib:int
             (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)))
    (setf lwkopt
            (max (the f2cl-lib:integer4 lwkopt)
                 (the f2cl-lib:integer4 mn)))))
  (cond
   ((/= info 0) (xerbla "CUNGBR" (f2cl-lib:int-sub info)) (go end_label))
```

```
 (lquery
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwkopt 'f2cl-lib:complex8))
  (go end_label)))
(cond
 ((or (= m 0) (= n 0))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce 1 'f2cl-lib:complex8))
  (go end_label)))
(cond
 (wantq
  (cond
   ((>= m k)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (cungqr m n k a lda tau work lwork iinfo)
      (declare (ignore var-3 var-5 var-6))
      (when var-0 (setf m var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf k var-2))
      (when var-4 (setf lda var-4))
      (when var-7 (setf lwork var-7))
      (when var-8 (setf iinfo var-8))))
   (t
    (f2cl-lib:fdo (j m (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                  ((> j 2) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *)) a-%offset%)
                zero)
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                    (f2cl-lib:fref a-%data% (i (f2cl-lib:int-sub j 1))
                                    ((1 lda) (1 *)) a-%offset%))
           label10))
       label20))
    (setf (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)
            one)
    (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data% (i 1) ((1 lda) (1 *)) a-%offset%)
                zero)
       label30))
    (cond
     ((> m 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
```

```
          (cungqr (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
           (f2cl-lib:int-sub m 1)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (2 2)
                                 ((1 lda) (1 *)) a-%offset%)
           lda tau work lwork iinfo)
         (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6))
         (when var-4 (setf lda var-4))
         (when var-7 (setf lwork var-7))
         (when var-8 (setf iinfo var-8))))))))
(t
 (cond
  ((< k n)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (cunglq m n k a lda tau work lwork iinfo)
     (declare (ignore var-3 var-5 var-6))
     (when var-0 (setf m var-0))
     (when var-1 (setf n var-1))
     (when var-2 (setf k var-2))
     (when var-4 (setf lda var-4))
     (when var-7 (setf lwork var-7))
     (when var-8 (setf iinfo var-8))))
  (t
   (setf (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)
         one)
   (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf (f2cl-lib:fref a-%data% (i 1) ((1 lda) (1 *)) a-%offset%)
             zero)
      label40))
   (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                        (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     ((> i 2) nil)
         (tagbody
           (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                 (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) j)
                                ((1 lda) (1 *)) a-%offset%))
          label50))
       (setf (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *)) a-%offset%)
             zero)
      label60))
   (cond
    ((> n 1)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
```

```
                 (cunglq (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
                  (f2cl-lib:int-sub n 1)
                  (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (2 2)
                                        ((1 lda) (1 *)) a-%offset%)
                  lda tau work lwork iinfo)
                (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6))
                (when var-4 (setf lda var-4))
                (when var-7 (setf lwork var-7))
                (when var-8 (setf iinfo var-8)))))))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce lwkopt 'f2cl-lib:complex8))
        (go end_label)
       end_label
        (return (values vect m n k nil lda nil nil lwork info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cungbr
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::vect
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n
                                              fortran-to-lisp::k nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::lwork
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

Chapter 47

LAPACK - General Band Matrix, Comp. Routines, Complex16

47.1 zgbbrd

47.1.1 Axiom unit tests

— zgbbrd.input —

```
)set break resume
)sys rm -f zgbbrd.output
)spool zgbbrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

47.1.2 Axiom help page

— zgbbrd.help —

```
==========================================================================
```

```
zgbbrd examples
===================================================================

===================================================================
Man Page Details
===================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgbbrd (VECT, M, N, NCC, KL, KU, AB, LDAB, D, E, Q, LDQ, PT,
           LDPT, C, LDC, WORK, RWORK, INFO)
           ZGBBRD

Function/Subroutine Documentation
   subroutine zgbbrd (character VECT, integer M, integer N, integer NCC,
       integer KL, integer KU, complex*16, dimension( ldab, * ) AB, integer
       LDAB, double precision, dimension( * ) D, double precision, dimension(
       * ) E, complex*16, dimension( ldq, * ) Q, integer LDQ, complex*16,
       dimension( ldpt, * ) PT, integer LDPT, complex*16, dimension( ldc, * )
       C, integer LDC, complex*16, dimension( * ) WORK, double precision,
       dimension( * ) RWORK, integer INFO)
       ZGBBRD

Purpose:
   ZGBBRD reduces a complex general m-by-n band matrix A to real upper
   bidiagonal form B by a unitary transformation: Q**H * A * P = B.

   The routine computes B, and optionally forms Q or P**H, or computes
   Q**H*C for a given matrix C.

Parameters:

 VECT
   VECT is CHARACTER*1
   Specifies whether or not the matrices Q and P**H are to be
   formed.
   = 'N': do not form Q or P**H;
   = 'Q': form Q only;
   = 'P': form P**H only;
   = 'B': form both.

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 NCC
```

```
  NCC is INTEGER
  The number of columns of the matrix C.  NCC >= 0.

KL
  KL is INTEGER
  The number of subdiagonals of the matrix A. KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals of the matrix A. KU >= 0.

AB
  AB is COMPLEX*16 array, dimension (LDAB,N)
  On entry, the m-by-n band matrix A, stored in rows 1 to
  KL+KU+1. The j-th column of A is stored in the j-th column of
  the array AB as follows:
  AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl).
  On exit, A is overwritten by values generated during the
  reduction.

LDAB
  LDAB is INTEGER
  The leading dimension of the array A. LDAB >= KL+KU+1.

D
  D is DOUBLE PRECISION array, dimension (min(M,N))
  The diagonal elements of the bidiagonal matrix B.

E
  E is DOUBLE PRECISION array, dimension (min(M,N)-1)
  The superdiagonal elements of the bidiagonal matrix B.

Q
  Q is COMPLEX*16 array, dimension (LDQ,M)
  If VECT = 'Q' or 'B', the m-by-m unitary matrix Q.
  If VECT = 'N' or 'P', the array Q is not referenced.

LDQ
  LDQ is INTEGER
  The leading dimension of the array Q.
  LDQ >= max(1,M) if VECT = 'Q' or 'B'; LDQ >= 1 otherwise.

PT
  PT is COMPLEX*16 array, dimension (LDPT,N)
  If VECT = 'P' or 'B', the n-by-n unitary matrix P'.
  If VECT = 'N' or 'Q', the array PT is not referenced.

LDPT
  LDPT is INTEGER
  The leading dimension of the array PT.
```

```
  LDPT >= max(1,N) if VECT = 'P' or 'B'; LDPT >= 1 otherwise.

C
  C is COMPLEX*16 array, dimension (LDC,NCC)
  On entry, an m-by-ncc matrix C.
  On exit, C is overwritten by Q**H*C.
  C is not referenced if NCC = 0.

LDC
  LDC is INTEGER
  The leading dimension of the array C.
  LDC >= max(1,M) if NCC > 0; LDC >= 1 if NCC = 0.

WORK
  WORK is COMPLEX*16 array, dimension (max(M,N))

RWORK
  RWORK is DOUBLE PRECISION array, dimension (max(M,N))

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

———-

47.1.3 fortran code

— zgbbrd.f —

```
*  =====================================================================
      SUBROUTINE ZGBBRD( VECT, M, N, NCC, KL, KU, AB, LDAB, D, E, Q,
     $                   LDQ, PT, LDPT, C, LDC, WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          VECT
      INTEGER            INFO, KL, KU, LDAB, LDC, LDPT, LDQ, M, N, NCC
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), E( * ), RWORK( * )
      COMPLEX*16         AB( LDAB, * ), C( LDC, * ), PT( LDPT, * ),
     $                   Q( LDQ, * ), WORK( * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
      COMPLEX*16         CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0D+0, 0.0D+0 ),
     $                   CONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            WANTB, WANTC, WANTPT, WANTQ
      INTEGER            I, INCA, J, J1, J2, KB, KB1, KK, KLM, KLU1,
     $                   KUN, L, MINMN, ML, ML0, MU, MU0, NR, NRT
      DOUBLE PRECISION   ABST, RC
      COMPLEX*16         RA, RB, RS, T
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLARGV, ZLARTG, ZLARTV, ZLASET, ZROT,
     $                   ZSCAL
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DCONJG, MAX, MIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      WANTB = LSAME( VECT, 'B' )
      WANTQ = LSAME( VECT, 'Q' ) .OR. WANTB
      WANTPT = LSAME( VECT, 'P' ) .OR. WANTB
      WANTC = NCC.GT.0
      KLU1 = KL + KU + 1
      INFO = 0
      IF( .NOT.WANTQ .AND. .NOT.WANTPT .AND. .NOT.LSAME( VECT, 'N' ) )
     $     THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NCC.LT.0 ) THEN
         INFO = -4
      ELSE IF( KL.LT.0 ) THEN
         INFO = -5
```

```
      ELSE IF( KU.LT.0 ) THEN
         INFO = -6
      ELSE IF( LDAB.LT.KLU1 ) THEN
         INFO = -8
      ELSE IF( LDQ.LT.1 .OR. WANTQ .AND. LDQ.LT.MAX( 1, M ) ) THEN
         INFO = -12
      ELSE IF( LDPT.LT.1 .OR. WANTPT .AND. LDPT.LT.MAX( 1, N ) ) THEN
         INFO = -14
      ELSE IF( LDC.LT.1 .OR. WANTC .AND. LDC.LT.MAX( 1, M ) ) THEN
         INFO = -16
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGBBRD', -INFO )
         RETURN
      END IF
*
*     Initialize Q and P**H to the unit matrix, if needed
*
      IF( WANTQ )
     $   CALL ZLASET( 'Full', M, M, CZERO, CONE, Q, LDQ )
      IF( WANTPT )
     $   CALL ZLASET( 'Full', N, N, CZERO, CONE, PT, LDPT )
*
*     Quick return if possible.
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
      MINMN = MIN( M, N )
*
      IF( KL+KU.GT.1 ) THEN
*
*        Reduce to upper bidiagonal form if KU > 0; if KU = 0, reduce
*        first to lower bidiagonal form and then transform to upper
*        bidiagonal
*
         IF( KU.GT.0 ) THEN
            ML0 = 1
            MU0 = 2
         ELSE
            ML0 = 2
            MU0 = 1
         END IF
*
*        Wherever possible, plane rotations are generated and applied in
*        vector operations of length NR over the index set J1:J2:KLU1.
*
*        The complex sines of the plane rotations are stored in WORK,
*        and the real cosines in RWORK.
*
```

```
         KLM = MIN( M-1, KL )
         KUN = MIN( N-1, KU )
         KB = KLM + KUN
         KB1 = KB + 1
         INCA = KB1*LDAB
         NR = 0
         J1 = KLM + 2
         J2 = 1 - KUN
*
         DO 90 I = 1, MINMN
*
*           Reduce i-th column and i-th row of matrix to bidiagonal form
*
            ML = KLM + 1
            MU = KUN + 1
            DO 80 KK = 1, KB
               J1 = J1 + KB
               J2 = J2 + KB
*
*              generate plane rotations to annihilate nonzero elements
*              which have been created below the band
*
               IF( NR.GT.0 )
     $            CALL ZLARGV( NR, AB( KLU1, J1-KLM-1 ), INCA,
     $                         WORK( J1 ), KB1, RWORK( J1 ), KB1 )
*
*              apply plane rotations from the left
*
               DO 10 L = 1, KB
                  IF( J2-KLM+L-1.GT.N ) THEN
                     NRT = NR - 1
                  ELSE
                     NRT = NR
                  END IF
                  IF( NRT.GT.0 )
     $               CALL ZLARTV( NRT, AB( KLU1-L, J1-KLM+L-1 ), INCA,
     $                            AB( KLU1-L+1, J1-KLM+L-1 ), INCA,
     $                            RWORK( J1 ), WORK( J1 ), KB1 )
   10          CONTINUE
*
               IF( ML.GT.ML0 ) THEN
                  IF( ML.LE.M-I+1 ) THEN
*
*                    generate plane rotation to annihilate a(i+ml-1,i)
*                    within the band, and apply rotation from the left
*
                     CALL ZLARTG( AB( KU+ML-1, I ), AB( KU+ML, I ),
     $                            RWORK( I+ML-1 ), WORK( I+ML-1 ), RA )
                     AB( KU+ML-1, I ) = RA
                     IF( I.LT.N )
```

```
     $                 CALL ZROT( MIN( KU+ML-2, N-I ),
     $                            AB( KU+ML-2, I+1 ), LDAB-1,
     $                            AB( KU+ML-1, I+1 ), LDAB-1,
     $                            RWORK( I+ML-1 ), WORK( I+ML-1 ) )
                 END IF
                 NR = NR + 1
                 J1 = J1 - KB1
              END IF
*
              IF( WANTQ ) THEN
*
*                accumulate product of plane rotations in Q
*
                 DO 20 J = J1, J2, KB1
                    CALL ZROT( M, Q( 1, J-1 ), 1, Q( 1, J ), 1,
     $                         RWORK( J ), DCONJG( WORK( J ) ) )
   20            CONTINUE
              END IF
*
              IF( WANTC ) THEN
*
*                apply plane rotations to C
*
                 DO 30 J = J1, J2, KB1
                    CALL ZROT( NCC, C( J-1, 1 ), LDC, C( J, 1 ), LDC,
     $                         RWORK( J ), WORK( J ) )
   30            CONTINUE
              END IF
*
              IF( J2+KUN.GT.N ) THEN
*
*                adjust J2 to keep within the bounds of the matrix
*
                 NR = NR - 1
                 J2 = J2 - KB1
              END IF
*
              DO 40 J = J1, J2, KB1
*
*                create nonzero element a(j-1,j+ku) above the band
*                and store it in WORK(n+1:2*n)
*
                 WORK( J+KUN ) = WORK( J )*AB( 1, J+KUN )
                 AB( 1, J+KUN ) = RWORK( J )*AB( 1, J+KUN )
   40         CONTINUE
*
*             generate plane rotations to annihilate nonzero elements
*             which have been generated above the band
*
              IF( NR.GT.0 )
```

```
     $           CALL ZLARGV( NR, AB( 1, J1+KUN-1 ), INCA,
     $                        WORK( J1+KUN ), KB1, RWORK( J1+KUN ),
     $                        KB1 )
*
*              apply plane rotations from the right
*
               DO 50 L = 1, KB
                  IF( J2+L-1.GT.M ) THEN
                     NRT = NR - 1
                  ELSE
                     NRT = NR
                  END IF
                  IF( NRT.GT.0 )
     $               CALL ZLARTV( NRT, AB( L+1, J1+KUN-1 ), INCA,
     $                            AB( L, J1+KUN ), INCA,
     $                            RWORK( J1+KUN ), WORK( J1+KUN ), KB1 )
   50          CONTINUE
*
               IF( ML.EQ.ML0 .AND. MU.GT.MU0 ) THEN
                  IF( MU.LE.N-I+1 ) THEN
*
*                    generate plane rotation to annihilate a(i,i+mu-1)
*                    within the band, and apply rotation from the right
*
                     CALL ZLARTG( AB( KU-MU+3, I+MU-2 ),
     $                            AB( KU-MU+2, I+MU-1 ),
     $                            RWORK( I+MU-1 ), WORK( I+MU-1 ), RA )
                     AB( KU-MU+3, I+MU-2 ) = RA
                     CALL ZROT( MIN( KL+MU-2, M-I ),
     $                          AB( KU-MU+4, I+MU-2 ), 1,
     $                          AB( KU-MU+3, I+MU-1 ), 1,
     $                          RWORK( I+MU-1 ), WORK( I+MU-1 ) )
                  END IF
                  NR = NR + 1
                  J1 = J1 - KB1
               END IF
*
               IF( WANTPT ) THEN
*
*                 accumulate product of plane rotations in P**H
*
                  DO 60 J = J1, J2, KB1
                     CALL ZROT( N, PT( J+KUN-1, 1 ), LDPT,
     $                          PT( J+KUN, 1 ), LDPT, RWORK( J+KUN ),
     $                          DCONJG( WORK( J+KUN ) ) )
   60             CONTINUE
               END IF
*
               IF( J2+KB.GT.M ) THEN
*
```

```
*                 adjust J2 to keep within the bounds of the matrix
*
                  NR = NR - 1
                  J2 = J2 - KB1
               END IF
*
               DO 70 J = J1, J2, KB1
*
*                 create nonzero element a(j+kl+ku,j+ku-1) below the
*                 band and store it in WORK(1:n)
*
                  WORK( J+KB ) = WORK( J+KUN )*AB( KLU1, J+KUN )
                  AB( KLU1, J+KUN ) = RWORK( J+KUN )*AB( KLU1, J+KUN )
   70          CONTINUE
*
               IF( ML.GT.ML0 ) THEN
                  ML = ML - 1
               ELSE
                  MU = MU - 1
               END IF
   80       CONTINUE
   90    CONTINUE
      END IF
*
      IF( KU.EQ.0 .AND. KL.GT.0 ) THEN
*
*        A has been reduced to complex lower bidiagonal form
*
*        Transform lower bidiagonal form to upper bidiagonal by applying
*        plane rotations from the left, overwriting superdiagonal
*        elements on subdiagonal elements
*
         DO 100 I = 1, MIN( M-1, N )
            CALL ZLARTG( AB( 1, I ), AB( 2, I ), RC, RS, RA )
            AB( 1, I ) = RA
            IF( I.LT.N ) THEN
               AB( 2, I ) = RS*AB( 1, I+1 )
               AB( 1, I+1 ) = RC*AB( 1, I+1 )
            END IF
            IF( WANTQ )
     $         CALL ZROT( M, Q( 1, I ), 1, Q( 1, I+1 ), 1, RC,
     $                    DCONJG( RS ) )
            IF( WANTC )
     $         CALL ZROT( NCC, C( I, 1 ), LDC, C( I+1, 1 ), LDC, RC,
     $                    RS )
  100    CONTINUE
      ELSE
*
*        A has been reduced to complex upper bidiagonal form or is
*        diagonal
```

```
*
         IF( KU.GT.0 .AND. M.LT.N ) THEN
*
*           Annihilate a(m,m+1) by applying plane rotations from the
*           right
*
            RB = AB( KU, M+1 )
            DO 110 I = M, 1, -1
               CALL ZLARTG( AB( KU+1, I ), RB, RC, RS, RA )
               AB( KU+1, I ) = RA
               IF( I.GT.1 ) THEN
                  RB = -DCONJG( RS )*AB( KU, I )
                  AB( KU, I ) = RC*AB( KU, I )
               END IF
               IF( WANTPT )
     $            CALL ZROT( N, PT( I, 1 ), LDPT, PT( M+1, 1 ), LDPT,
     $                       RC, DCONJG( RS ) )
  110       CONTINUE
         END IF
      END IF
*
*     Make diagonal and superdiagonal elements real, storing them in D
*     and E
*
      T = AB( KU+1, 1 )
      DO 120 I = 1, MINMN
         ABST = ABS( T )
         D( I ) = ABST
         IF( ABST.NE.ZERO ) THEN
            T = T / ABST
         ELSE
            T = CONE
         END IF
         IF( WANTQ )
     $      CALL ZSCAL( M, T, Q( 1, I ), 1 )
         IF( WANTC )
     $      CALL ZSCAL( NCC, DCONJG( T ), C( I, 1 ), LDC )
         IF( I.LT.MINMN ) THEN
            IF( KU.EQ.0 .AND. KL.EQ.0 ) THEN
               E( I ) = ZERO
               T = AB( 1, I+1 )
            ELSE
               IF( KU.EQ.0 ) THEN
                  T = AB( 2, I )*DCONJG( T )
               ELSE
                  T = AB( KU, I+1 )*DCONJG( T )
               END IF
               ABST = ABS( T )
               E( I ) = ABST
               IF( ABST.NE.ZERO ) THEN
```

```
                 T = T / ABST
              ELSE
                 T = CONE
              END IF
              IF( WANTPT )
     $           CALL ZSCAL( N, T, PT( I+1, 1 ), LDPT )
              T = AB( KU+1, I+1 )*DCONJG( T )
           END IF
        END IF
  120 CONTINUE
      RETURN
*
*     End of ZGBBRD
*
      END
```

———-

47.1.4 lisp code

— zgbbrd.lisp —

```
(let* ((zero 0.0d0)
       (czero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (cone (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (f2cl-lib:complex16) czero)
           (type (f2cl-lib:complex16) cone)
           (ignorable zero czero cone))
  (defun zgbbrd
         (vect m n ncc kl ku ab ldab d e q ldq pt ldpt c ldc work rwork info)
    (declare (type (array double-float (*)) rwork e d)
             (type (array f2cl-lib:complex16 (*)) work c pt q ab)
             (type (f2cl-lib:integer4) info ldc ldpt ldq ldab ku kl ncc n m)
             (type (string *) vect))
    (f2cl-lib:with-multi-array-data
        ((vect character vect-%data% vect-%offset%)
         (ab f2cl-lib:complex16 ab-%data% ab-%offset%)
         (q f2cl-lib:complex16 q-%data% q-%offset%)
         (pt f2cl-lib:complex16 pt-%data% pt-%offset%)
         (c f2cl-lib:complex16 c-%data% c-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (prog ((ra #C(0.0d0 0.0d0)) (rb #C(0.0d0 0.0d0)) (rs #C(0.0d0 0.0d0))
             (abst 0.0d0) (rc 0.0d0) (i 0) (inca 0) (j 0) (j1 0) (j2 0) (kb 0)
             (kb1 0) (kk 0) (klm 0) (klu1 0) (kun 0) (l 0) (minmn 0) (ml 0)
```

```
     (ml0 0) (mu 0) (mu0 0) (nr 0) (nrt 0) (wantb nil) (wantc nil)
     (wantpt nil) (wantq nil) (t$ #C(0.0d0 0.0d0)))
(declare (type (f2cl-lib:complex16) t$ ra rb rs)
         (type (double-float) abst rc)
         (type (f2cl-lib:integer4) i inca j j1 j2 kb kb1 kk klm klu1
          kun l minmn ml ml0 mu mu0 nr nrt)
         (type f2cl-lib:logical wantb wantc wantpt wantq))
(setf wantb
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame vect "B")
          (declare (ignore var-1))
          (when var-0 (setf vect var-0))
          ret-val))
(setf wantq
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame vect "Q")
           (declare (ignore var-1))
           (when var-0 (setf vect var-0))
           ret-val)
         wantb))
(setf wantpt
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame vect "P")
           (declare (ignore var-1))
           (when var-0 (setf vect var-0))
           ret-val)
         wantb))
(setf wantc (> ncc 0))
(setf klu1 (f2cl-lib:int-add kl ku 1))
(setf info 0)
(cond
 ((and (not wantq) (not wantpt)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame vect "N")
          (declare (ignore var-1))
          (when var-0 (setf vect var-0))
          ret-val)))
  (setf info -1))
 ((< m 0) (setf info -2)) ((< n 0) (setf info -3))
 ((< ncc 0) (setf info -4)) ((< kl 0) (setf info -5))
 ((< ku 0) (setf info -6)) ((< ldab klu1) (setf info -8))
 ((or (< ldq 1)
      (and wantq
           (< ldq
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 m)))))
  (setf info -12))
```

```
 ((or (< ldpt 1)
      (and wantpt
           (< ldpt
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 n)))))
  (setf info -14))
 ((or (< ldc 1)
      (and wantc
           (< ldc
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 m)))))
  (setf info -16)))
(cond
 ((/= info 0) (xerbla "ZGBBRD" (f2cl-lib:int-sub info))
  (go end_label)))
(if wantq
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlaset "Full" m m czero cone q ldq)
      (declare (ignore var-0 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf m var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldq var-6))))
(if wantpt
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlaset "Full" n n czero cone pt ldpt)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldpt var-6))))
(if (or (= m 0) (= n 0))
    (go end_label))
(setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(cond
 ((> (f2cl-lib:int-add kl ku) 1)
  (cond ((> ku 0) (setf ml0 1) (setf mu0 2))
        (t (setf ml0 2) (setf mu0 1)))
  (setf klm
          (min (the f2cl-lib:integer4 (f2cl-lib:int-sub m 1))
               (the f2cl-lib:integer4 kl)))
  (setf kun
          (min (the f2cl-lib:integer4 (f2cl-lib:int-sub n 1))
               (the f2cl-lib:integer4 ku)))
  (setf kb (f2cl-lib:int-add klm kun))
  (setf kb1 (f2cl-lib:int-add kb 1))
  (setf inca (f2cl-lib:int-mul kb1 ldab)) (setf nr 0)
  (setf j1 (f2cl-lib:int-add klm 2)) (setf j2 (f2cl-lib:int-sub 1 kun))
```

```
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i minmn) nil)
  (tagbody
    (setf ml (f2cl-lib:int-add klm 1))
    (setf mu (f2cl-lib:int-add kun 1))
    (f2cl-lib:fdo (kk 1 (f2cl-lib:int-add kk 1))
                  ((> kk kb) nil)
      (tagbody
        (setf j1 (f2cl-lib:int-add j1 kb))
        (setf j2 (f2cl-lib:int-add j2 kb))
        (if (> nr 0)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (zlargv nr
                 (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                       (klu1
                                        (f2cl-lib:int-sub j1 klm 1))
                                       ((1 ldab) (1 *)) ab-%offset%)
                 inca
                 (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                       (j1) ((1 *)) work-%offset%)
                 kb1
                 (f2cl-lib:array-slice rwork-%data% double-float (j1)
                                       ((1 *)) rwork-%offset%)
                 kb1)
              (declare (ignore var-1 var-3 var-5))
              (when var-0 (setf nr var-0))
              (when var-2 (setf inca var-2))
              (when var-4 (setf kb1 var-4))
              (when var-6 (setf kb1 var-6))))
        (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                      ((> l kb) nil)
          (tagbody
            (cond
             ((>
               (f2cl-lib:int-add j2 (f2cl-lib:int-sub klm) l
                                  (f2cl-lib:int-sub 1))
               n)
              (setf nrt (f2cl-lib:int-sub nr 1)))
             (t (setf nrt nr)))
            (if (> nrt 0)
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
                    (zlartv nrt
                     (f2cl-lib:array-slice ab-%data%
                                           f2cl-lib:complex16
                                           ((+ klu1
                                               (f2cl-lib:int-sub 1))
                                            (f2cl-lib:int-sub
                                             (f2cl-lib:int-add
```

```
                                        (f2cl-lib:int-sub j1
                                                          klm)
                                       l)
                                      1))
                                  ((1 ldab) (1 *))
                                  ab-%offset%)
                     inca
                     (f2cl-lib:array-slice ab-%data%
                                  f2cl-lib:complex16
                                  ((+ klu1
                                      (f2cl-lib:int-sub l)
                                      1)
                                   (f2cl-lib:int-sub
                                    (f2cl-lib:int-add
                                     (f2cl-lib:int-sub j1
                                                       klm)
                                       l)
                                      1))
                                  ((1 ldab) (1 *))
                                  ab-%offset%)
                     inca
                     (f2cl-lib:array-slice rwork-%data% double-float
                                  (j1) ((1 *))
                                  rwork-%offset%)
                     (f2cl-lib:array-slice work-%data%
                                  f2cl-lib:complex16 (j1)
                                  ((1 *)) work-%offset%)
                     kb1)
                  (declare (ignore var-1 var-3 var-5 var-6))
                  (when var-0 (setf nrt var-0))
                  (when var-2 (setf inca var-2))
                  (when var-4 (setf inca var-4))
                  (when var-7 (setf kb1 var-7))))
            label10))
          (cond
           ((> ml ml0)
            (cond
             ((<= ml (f2cl-lib:int-add m (f2cl-lib:int-sub i) 1))
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (zlartg
                   (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                 ((+ ku ml
                                     (f2cl-lib:int-sub 1))
                                  i)
                                 ((1 ldab) (1 *)) ab-%offset%)
                   (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                 ((+ ku ml) i) ((1 ldab) (1 *))
                                 ab-%offset%)
                   (f2cl-lib:array-slice rwork-%data% double-float
                                 ((+ i ml
```

```
                                        (f2cl-lib:int-sub 1)))
                                    ((1 *)) rwork-%offset%)
                (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                    ((+ i ml
                                        (f2cl-lib:int-sub 1)))
                                    ((1 *)) work-%offset%)
                ra)
            (declare (ignore var-0 var-1 var-2 var-3))
            (when var-4 (setf ra var-4)))
          (setf (f2cl-lib:fref ab-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add ku ml) 1)
                                i)
                               ((1 ldab) (1 *)) ab-%offset%)
                  ra)
          (if (< i n)
              (zrot
                (min
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-sub (f2cl-lib:int-add ku ml) 2))
                 (the f2cl-lib:integer4 (f2cl-lib:int-sub n i)))
                (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                    ((+ ku ml
                                        (f2cl-lib:int-sub 2))
                                     (f2cl-lib:int-add i 1))
                                    ((1 ldab) (1 *)) ab-%offset%)
                (f2cl-lib:int-sub ldab 1)
                (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                    ((+ ku ml
                                        (f2cl-lib:int-sub 1))
                                     (f2cl-lib:int-add i 1))
                                    ((1 ldab) (1 *)) ab-%offset%)
                (f2cl-lib:int-sub ldab 1)
                (f2cl-lib:array-slice rwork-%data% double-float
                                    ((+ i ml
                                        (f2cl-lib:int-sub 1)))
                                    ((1 *)) rwork-%offset%)
                (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                    ((+ i ml
                                        (f2cl-lib:int-sub 1)))
                                    ((1 *)) work-%offset%)))))
  (setf nr (f2cl-lib:int-add nr 1))
  (setf j1 (f2cl-lib:int-sub j1 kb1))))
(cond
 (wantq
  (f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
                ((> j j2) nil)
    (tagbody
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
            (zrot m
             (f2cl-lib:array-slice q-%data% f2cl-lib:complex16
                                   (1 (f2cl-lib:int-sub j 1))
                                   ((1 ldq) (1 *)) q-%offset%)
             1
             (f2cl-lib:array-slice q-%data% f2cl-lib:complex16
                                   (1 j) ((1 ldq) (1 *))
                                   q-%offset%)
             1
             (f2cl-lib:array-slice rwork-%data% double-float
                                   (j) ((1 *)) rwork-%offset%)
             (f2cl-lib:dconjg
              (f2cl-lib:fref work-%data% (j) ((1 *))
                             work-%offset%)))
          (declare
           (ignore var-1 var-2 var-3 var-4 var-5 var-6))
          (when var-0 (setf m var-0)))
      label20))))
(cond
 (wantc
  (f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
                ((> j j2) nil)
    (tagbody
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zrot ncc
           (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                 ((+ j (f2cl-lib:int-sub 1))
                                  1)
                                 ((1 ldc) (1 *)) c-%offset%)
           ldc
           (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                 (j 1) ((1 ldc) (1 *))
                                 c-%offset%)
           ldc
           (f2cl-lib:array-slice rwork-%data% double-float
                                 (j) ((1 *)) rwork-%offset%)
           (f2cl-lib:array-slice work-%data%
                                 f2cl-lib:complex16 (j)
                                 ((1 *)) work-%offset%))
          (declare (ignore var-1 var-3 var-5 var-6))
          (when var-0 (setf ncc var-0))
          (when var-2 (setf ldc var-2))
          (when var-4 (setf ldc var-4)))
      label30))))
(cond
 ((> (f2cl-lib:int-add j2 kun) n)
  (setf nr (f2cl-lib:int-sub nr 1))
  (setf j2 (f2cl-lib:int-sub j2 kb1))))
(f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
```

```
                ((> j j2) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add j kun)) ((1 *))
                         work-%offset%)
          (*
           (f2cl-lib:fref work-%data% (j) ((1 *))
                          work-%offset%)
           (f2cl-lib:fref ab-%data%
                          (1 (f2cl-lib:int-add j kun))
                          ((1 ldab) (1 *)) ab-%offset%)))
    (setf (f2cl-lib:fref ab-%data%
                         (1 (f2cl-lib:int-add j kun))
                         ((1 ldab) (1 *)) ab-%offset%)
          (*
           (f2cl-lib:fref rwork-%data% (j) ((1 *))
                          rwork-%offset%)
           (f2cl-lib:fref ab-%data%
                          (1 (f2cl-lib:int-add j kun))
                          ((1 ldab) (1 *)) ab-%offset%)))
   label40))
(if (> nr 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlargv nr
         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                               (1
                                (f2cl-lib:int-sub
                                 (f2cl-lib:int-add j1 kun)
                                 1))
                               ((1 ldab) (1 *)) ab-%offset%)
         inca
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               ((+ j1 kun)) ((1 *))
                               work-%offset%)
         kb1
         (f2cl-lib:array-slice rwork-%data% double-float
                               ((+ j1 kun)) ((1 *))
                               rwork-%offset%)
         kb1)
      (declare (ignore var-1 var-3 var-5))
      (when var-0 (setf nr var-0))
      (when var-2 (setf inca var-2))
      (when var-4 (setf kb1 var-4))
      (when var-6 (setf kb1 var-6))))
(f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
              ((> l kb) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:int-add j2 l (f2cl-lib:int-sub 1)) m)
```

```
      (setf nrt (f2cl-lib:int-sub nr 1)))
     (t (setf nrt nr)))
    (if (> nrt 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (zlartv nrt
             (f2cl-lib:array-slice ab-%data%
                                   f2cl-lib:complex16
                                   ((+ l 1)
                                    (f2cl-lib:int-sub
                                     (f2cl-lib:int-add j1
                                                       kun)
                                     1))
                                   ((1 ldab) (1 *))
                                   ab-%offset%)
             inca
             (f2cl-lib:array-slice ab-%data%
                                   f2cl-lib:complex16
                                   (l
                                    (f2cl-lib:int-add j1
                                                      kun))
                                   ((1 ldab) (1 *))
                                   ab-%offset%)
             inca
             (f2cl-lib:array-slice rwork-%data% double-float
                                   ((+ j1 kun)) ((1 *))
                                   rwork-%offset%)
             (f2cl-lib:array-slice work-%data%
                                   f2cl-lib:complex16
                                   ((+ j1 kun)) ((1 *))
                                   work-%offset%)
             kb1)
           (declare (ignore var-1 var-3 var-5 var-6))
           (when var-0 (setf nrt var-0))
           (when var-2 (setf inca var-2))
           (when var-4 (setf inca var-4))
           (when var-7 (setf kb1 var-7))))
  label50))
(cond
 ((and (= ml ml0) (> mu mu0))
  (cond
   ((<= mu (f2cl-lib:int-add n (f2cl-lib:int-sub i) 1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zlartg
         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                               ((+ ku (f2cl-lib:int-sub mu)
                                   3)
                                (f2cl-lib:int-sub
                                 (f2cl-lib:int-add i mu) 2))
                               ((1 ldab) (1 *)) ab-%offset%)
```

```
            (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                  ((+ ku (f2cl-lib:int-sub mu)
                                      2)
                                   (f2cl-lib:int-sub
                                    (f2cl-lib:int-add i mu) 1))
                                  ((1 ldab) (1 *)) ab-%offset%)
            (f2cl-lib:array-slice rwork-%data% double-float
                                  ((+ i mu
                                      (f2cl-lib:int-sub 1)))
                                  ((1 *)) rwork-%offset%)
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                  ((+ i mu
                                      (f2cl-lib:int-sub 1)))
                                  ((1 *)) work-%offset%)
            ra)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf ra var-4)))
      (setf (f2cl-lib:fref ab-%data%
                           ((f2cl-lib:int-add
                             (f2cl-lib:int-sub ku mu) 3)
                            (f2cl-lib:int-sub
                             (f2cl-lib:int-add i mu) 2))
                           ((1 ldab) (1 *)) ab-%offset%)
            ra)
      (zrot
       (min
        (the f2cl-lib:integer4
             (f2cl-lib:int-sub (f2cl-lib:int-add kl mu) 2))
        (the f2cl-lib:integer4 (f2cl-lib:int-sub m i)))
       (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                             ((+ ku (f2cl-lib:int-sub mu) 4)
                              (f2cl-lib:int-sub
                               (f2cl-lib:int-add i mu) 2))
                             ((1 ldab) (1 *)) ab-%offset%)
       1
       (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                             ((+ ku (f2cl-lib:int-sub mu) 3)
                              (f2cl-lib:int-sub
                               (f2cl-lib:int-add i mu) 1))
                             ((1 ldab) (1 *)) ab-%offset%)
       1
       (f2cl-lib:array-slice rwork-%data% double-float
                             ((+ i mu (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             ((+ i mu (f2cl-lib:int-sub 1)))
                             ((1 *)) work-%offset%))))
    (setf nr (f2cl-lib:int-add nr 1))
    (setf j1 (f2cl-lib:int-sub j1 kb1))))
  (cond
```

```
(wantpt
 (f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
               ((> j j2) nil)
   (tagbody
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (zrot n
          (f2cl-lib:array-slice pt-%data% f2cl-lib:complex16
                                ((+ j kun
                                    (f2cl-lib:int-sub 1))
                                 1)
                                ((1 ldpt) (1 *))
                                pt-%offset%)
          ldpt
          (f2cl-lib:array-slice pt-%data% f2cl-lib:complex16
                                ((+ j kun) 1)
                                ((1 ldpt) (1 *))
                                pt-%offset%)
          ldpt
          (f2cl-lib:array-slice rwork-%data% double-float
                                ((+ j kun)) ((1 *))
                                rwork-%offset%)
          (f2cl-lib:dconjg
           (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add j kun)) ((1 *))
                          work-%offset%)))
       (declare (ignore var-1 var-3 var-5 var-6))
       (when var-0 (setf n var-0))
       (when var-2 (setf ldpt var-2))
       (when var-4 (setf ldpt var-4)))
    label60))))
(cond
 ((> (f2cl-lib:int-add j2 kb) m)
  (setf nr (f2cl-lib:int-sub nr 1))
  (setf j2 (f2cl-lib:int-sub j2 kb1))))
(f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
              ((> j j2) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add j kb)) ((1 *))
                         work-%offset%)
           (*
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add j kun))
                           ((1 *)) work-%offset%)
            (f2cl-lib:fref ab-%data%
                           (klu1 (f2cl-lib:int-add j kun))
                           ((1 ldab) (1 *)) ab-%offset%)))
    (setf (f2cl-lib:fref ab-%data%
                         (klu1 (f2cl-lib:int-add j kun))
```

```
                                        ((1 ldab) (1 *)) ab-%offset%)
                          (*
                           (f2cl-lib:fref rwork-%data%
                                          ((f2cl-lib:int-add j kun))
                                          ((1 *)) rwork-%offset%)
                           (f2cl-lib:fref ab-%data%
                                          (klu1 (f2cl-lib:int-add j kun))
                                          ((1 ldab) (1 *)) ab-%offset%)))
                label70))
              (cond ((> ml ml0) (setf ml (f2cl-lib:int-sub ml 1)))
                    (t (setf mu (f2cl-lib:int-sub mu 1))))
            label80))
        label90))))
    (cond
     ((and (= ku 0) (> kl 0))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (min
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
                         (the f2cl-lib:integer4 n)))
                     nil)
       (tagbody
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (zlartg
              (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16 (1 i)
                                    ((1 ldab) (1 *)) ab-%offset%)
              (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16 (2 i)
                                    ((1 ldab) (1 *)) ab-%offset%)
              rc rs ra)
           (declare (ignore var-0 var-1))
           (when var-2 (setf rc var-2))
           (when var-3 (setf rs var-3))
           (when var-4 (setf ra var-4)))
         (setf (f2cl-lib:fref ab-%data% (1 i) ((1 ldab) (1 *))
                              ab-%offset%)
               ra)
         (cond
          ((< i n)
           (setf (f2cl-lib:fref ab-%data% (2 i) ((1 ldab) (1 *))
                                ab-%offset%)
                 (* rs
                    (f2cl-lib:fref ab-%data% (1 (f2cl-lib:int-add i 1))
                                   ((1 ldab) (1 *)) ab-%offset%)))
           (setf (f2cl-lib:fref ab-%data% (1 (f2cl-lib:int-add i 1))
                                ((1 ldab) (1 *)) ab-%offset%)
                 (* rc
                    (f2cl-lib:fref ab-%data% (1 (f2cl-lib:int-add i 1))
                                   ((1 ldab) (1 *)) ab-%offset%)))))
         (if wantq
```

```
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (zrot m
                 (f2cl-lib:array-slice q-%data% f2cl-lib:complex16 (1 i)
                                       ((1 ldq) (1 *)) q-%offset%)
                 1
                 (f2cl-lib:array-slice q-%data% f2cl-lib:complex16
                                       (1 (f2cl-lib:int-add i 1))
                                       ((1 ldq) (1 *)) q-%offset%)
                 1 rc (f2cl-lib:dconjg rs))
              (declare (ignore var-1 var-2 var-3 var-4 var-6))
              (when var-0 (setf m var-0))
              (when var-5 (setf rc var-5))))
        (if wantc
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (zrot ncc
                 (f2cl-lib:array-slice c-%data% f2cl-lib:complex16 (i 1)
                                       ((1 ldc) (1 *)) c-%offset%)
                 ldc
                 (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                       ((+ i 1) 1) ((1 ldc) (1 *))
                                       c-%offset%)
                 ldc rc rs)
              (declare (ignore var-1 var-3))
              (when var-0 (setf ncc var-0))
              (when var-2 (setf ldc var-2))
              (when var-4 (setf ldc var-4))
              (when var-5 (setf rc var-5))
              (when var-6 (setf rs var-6))))
      label100)))
    (t
     (cond
      ((and (> ku 0) (< m n))
       (setf rb
               (f2cl-lib:fref ab-%data% (ku (f2cl-lib:int-add m 1))
                              ((1 ldab) (1 *)) ab-%offset%))
       (f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     ((> i 1) nil)
         (tagbody
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
               (zlartg
                (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                      ((+ ku 1) i) ((1 ldab) (1 *))
                                      ab-%offset%)
                rb rc rs ra)
             (declare (ignore var-0))
             (when var-1 (setf rb var-1))
             (when var-2 (setf rc var-2))
             (when var-3 (setf rs var-3))
```

```
            (when var-4 (setf ra var-4)))
          (setf (f2cl-lib:fref ab-%data% ((f2cl-lib:int-add ku 1) i)
                               ((1 ldab) (1 *)) ab-%offset%)
                ra)
          (cond
           ((> i 1)
            (setf rb
                    (* (- (f2cl-lib:dconjg rs))
                       (f2cl-lib:fref ab-%data% (ku i) ((1 ldab) (1 *))
                                      ab-%offset%)))
            (setf (f2cl-lib:fref ab-%data% (ku i) ((1 ldab) (1 *))
                                 ab-%offset%)
                    (* rc
                       (f2cl-lib:fref ab-%data% (ku i) ((1 ldab) (1 *))
                                      ab-%offset%)))))
          (if wantpt
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (zrot n
                   (f2cl-lib:array-slice pt-%data% f2cl-lib:complex16
                                         (i 1) ((1 ldpt) (1 *))
                                         pt-%offset%)
                   ldpt
                   (f2cl-lib:array-slice pt-%data% f2cl-lib:complex16
                                         ((+ m 1) 1) ((1 ldpt) (1 *))
                                         pt-%offset%)
                   ldpt rc (f2cl-lib:dconjg rs))
                (declare (ignore var-1 var-3 var-6))
                (when var-0 (setf n var-0))
                (when var-2 (setf ldpt var-2))
                (when var-4 (setf ldpt var-4))
                (when var-5 (setf rc var-5))))
        label110))))))
(setf t$
        (f2cl-lib:fref ab-%data% ((f2cl-lib:int-add ku 1) 1)
                       ((1 ldab) (1 *)) ab-%offset%))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i minmn) nil)
  (tagbody
    (setf abst (abs t$))
    (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) abst)
    (cond ((/= abst zero) (setf t$ (/ t$ abst))) (t (setf t$ cone)))
    (if wantq
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (zscal m t$
             (f2cl-lib:array-slice q-%data% f2cl-lib:complex16 (1 i)
                                   ((1 ldq) (1 *)) q-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
```

```
      (when var-1 (setf t$ var-1))))
(if wantc
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zscal ncc (f2cl-lib:dconjg t$)
         (f2cl-lib:array-slice c-%data% f2cl-lib:complex16 (i 1)
                               ((1 ldc) (1 *)) c-%offset%)
         ldc)
      (declare (ignore var-1 var-2))
      (when var-0 (setf ncc var-0))
      (when var-3 (setf ldc var-3))))
(cond
 ((< i minmn)
  (cond
   ((and (= ku 0) (= kl 0))
    (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) zero)
    (setf t$
            (f2cl-lib:fref ab-%data% (1 (f2cl-lib:int-add i 1))
                           ((1 ldab) (1 *)) ab-%offset%)))
   (t
    (cond
     ((= ku 0)
      (setf t$
              (*
               (f2cl-lib:fref ab-%data% (2 i) ((1 ldab) (1 *))
                              ab-%offset%)
               (f2cl-lib:dconjg t$))))
     (t
      (setf t$
              (*
               (f2cl-lib:fref ab-%data% (ku (f2cl-lib:int-add i 1))
                              ((1 ldab) (1 *)) ab-%offset%)
               (f2cl-lib:dconjg t$)))))
    (setf abst (abs t$))
    (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) abst)
    (cond ((/= abst zero) (setf t$ (/ t$ abst)))
          (t (setf t$ cone)))
    (if wantpt
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (zscal n t$
             (f2cl-lib:array-slice pt-%data% f2cl-lib:complex16
                                   ((+ i 1) 1) ((1 ldpt) (1 *))
                                   pt-%offset%)
             ldpt)
          (declare (ignore var-2))
          (when var-0 (setf n var-0))
          (when var-1 (setf t$ var-1))
          (when var-3 (setf ldpt var-3))))
    (setf t$
            (*
             (f2cl-lib:fref ab-%data%
```

```
                                        ((f2cl-lib:int-add ku 1)
                                         (f2cl-lib:int-add i 1))
                                        ((1 ldab) (1 *)) ab-%offset%)
                          (f2cl-lib:dconjg t$)))))))
           label120))
         (go end_label)
        end_label
         (return
          (values vect m n ncc nil nil nil nil nil nil nil ldq nil ldpt nil ldc
                  nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgbbrd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::vect
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n
                                              fortran-to-lisp::ncc nil nil nil
                                              nil nil nil nil
                                              fortran-to-lisp::ldq nil
                                              fortran-to-lisp::ldpt nil
                                              fortran-to-lisp::ldc nil nil
```

```
                                        fortran-to-lisp::info)
                                      :calls '(fortran-to-lisp::xerbla))))
```

———-

47.2 zgbcon

47.2.1 Axiom unit tests

— zgbcon.input —

```
)set break resume
)sys rm -f zgbcon.output
)spool zgbcon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

47.2.2 Axiom help page

— zgbcon.help —

```
======================================================================
zgbcon examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgbcon (NORM, N, KL, KU, AB, LDAB, IPIV, ANORM, RCOND, WORK,
           RWORK, INFO)
           ZGBCON

Function/Subroutine Documentation
```

```
   subroutine zgbcon (character NORM, integer N, integer KL, integer KU,
       complex*16, dimension( ldab, * ) AB, integer LDAB, integer, dimension(
       * ) IPIV, double precision ANORM, double precision RCOND, complex*16,
       dimension( * ) WORK, double precision, dimension( * ) RWORK, integer
       INFO)
       ZGBCON

Purpose:
   ZGBCON estimates the reciprocal of the condition number of a complex
   general band matrix A, in either the 1-norm or the infinity-norm,
   using the LU factorization computed by ZGBTRF.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as
   RCOND = 1 / ( norm(A) * norm(inv(A)) ).

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies whether the 1-norm condition number or the
   infinity-norm condition number is required:
   = '1' or 'O':  1-norm;
   = 'I':         Infinity-norm.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 AB
   AB is COMPLEX*16 array, dimension (LDAB,N)
   Details of the LU factorization of the band matrix A, as
   computed by ZGBTRF.  U is stored as an upper triangular band
   matrix with KL+KU superdiagonals in rows 1 to KL+KU+1, and
   the multipliers used during the factorization are stored in
   rows KL+KU+2 to 2*KL+KU+1.

 LDAB
   LDAB is INTEGER
   The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.

 IPIV
```

```
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= N, row i of the matrix was
  interchanged with row IPIV(i).

ANORM
  ANORM is DOUBLE PRECISION
  If NORM = '1' or 'O', the 1-norm of the original matrix A.
  If NORM = 'I', the infinity-norm of the original matrix A.

RCOND
  RCOND is DOUBLE PRECISION
  The reciprocal of the condition number of the matrix A,
  computed as RCOND = 1/(norm(A) * norm(inv(A))).

WORK
  WORK is COMPLEX*16 array, dimension (2*N)

RWORK
  RWORK is DOUBLE PRECISION array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
```

———-

47.2.3 fortran code

— **zgbcon.f** —

```
*  =====================================================================
      SUBROUTINE ZGBCON( NORM, N, KL, KU, AB, LDAB, IPIV, ANORM, RCOND,
     $                   WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            INFO, KL, KU, LDAB, N
      DOUBLE PRECISION   ANORM, RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
```

```
      DOUBLE PRECISION   RWORK( * )
      COMPLEX*16         AB( LDAB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LNOTI, ONENRM
      CHARACTER          NORMIN
      INTEGER            IX, J, JP, KASE, KASE1, KD, LM
      DOUBLE PRECISION   AINVNM, SCALE, SMLNUM
      COMPLEX*16         T, ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IZAMAX
      DOUBLE PRECISION   DLAMCH
      COMPLEX*16         ZDOTC
      EXTERNAL           LSAME, IZAMAX, DLAMCH, ZDOTC
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZAXPY, ZDRSCL, ZLACN2, ZLATBS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, MIN
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      ONENRM = NORM.EQ.'1' .OR. LSAME( NORM, 'O' )
      IF( .NOT.ONENRM .AND. .NOT.LSAME( NORM, 'I' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
```

```
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.2*KL+KU+1 ) THEN
         INFO = -6
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGBCON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
      SMLNUM = DLAMCH( 'Safe minimum' )
*
*     Estimate the norm of inv(A).
*
      AINVNM = ZERO
      NORMIN = 'N'
      IF( ONENRM ) THEN
         KASE1 = 1
      ELSE
         KASE1 = 2
      END IF
      KD = KL + KU + 1
      LNOTI = KL.GT.0
      KASE = 0
   10 CONTINUE
      CALL ZLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.KASE1 ) THEN
*
*           Multiply by inv(L).
*
            IF( LNOTI ) THEN
               DO 20 J = 1, N - 1
                  LM = MIN( KL, N-J )
                  JP = IPIV( J )
                  T = WORK( JP )
                  IF( JP.NE.J ) THEN
```

```
                  WORK( JP ) = WORK( J )
                  WORK( J ) = T
               END IF
               CALL ZAXPY( LM, -T, AB( KD+1, J ), 1, WORK( J+1 ), 1 )
   20       CONTINUE
         END IF
*
*        Multiply by inv(U).
*
         CALL ZLATBS( 'Upper', 'No transpose', 'Non-unit', NORMIN, N,
     $                KL+KU, AB, LDAB, WORK, SCALE, RWORK, INFO )
      ELSE
*
*        Multiply by inv(U**H).
*
         CALL ZLATBS( 'Upper', 'Conjugate transpose', 'Non-unit',
     $                NORMIN, N, KL+KU, AB, LDAB, WORK, SCALE, RWORK,
     $                INFO )
*
*        Multiply by inv(L**H).
*
         IF( LNOTI ) THEN
            DO 30 J = N - 1, 1, -1
               LM = MIN( KL, N-J )
               WORK( J ) = WORK( J ) - ZDOTC( LM, AB( KD+1, J ), 1,
     $                     WORK( J+1 ), 1 )
               JP = IPIV( J )
               IF( JP.NE.J ) THEN
                  T = WORK( JP )
                  WORK( JP ) = WORK( J )
                  WORK( J ) = T
               END IF
   30       CONTINUE
         END IF
      END IF
*
*     Divide X by 1/SCALE if doing so will not cause overflow.
*
      NORMIN = 'Y'
      IF( SCALE.NE.ONE ) THEN
         IX = IZAMAX( N, WORK, 1 )
         IF( SCALE.LT.CABS1( WORK( IX ) )*SMLNUM .OR. SCALE.EQ.ZERO )
     $      GO TO 40
         CALL ZDRSCL( N, SCALE, WORK, 1 )
      END IF
      GO TO 10
   END IF
*
*  Compute the estimate of the reciprocal condition number.
*
```

```
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
   40 CONTINUE
      RETURN
*
*     End of ZGBCON
*
      END
```

47.2.4 lisp code

— zgbcon.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun zgbcon (norm n kl ku ab ldab ipiv anorm rcond work rwork info)
    (declare (type (array double-float (*)) rwork)
             (type (double-float) rcond anorm)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) work ab)
             (type (f2cl-lib:integer4) info ldab ku kl n)
             (type (string *) norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (ab f2cl-lib:complex16 ab-%data% ab-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
          cabs1))
        (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
               (zdum #C(0.0d0 0.0d0)) (ainvnm 0.0d0) (scale 0.0d0)
               (smlnum 0.0d0) (ix 0) (j 0) (jp 0) (kase 0) (kase1 0) (kd 0)
               (lm 0)
               (normin
                (make-array '(1) :element-type 'character :initial-element
                            #\ ))
               (lnoti nil) (onenrm nil) (t$ #C(0.0d0 0.0d0)))
          (declare (type (array f2cl-lib:integer4 (3)) isave)
                   (type (f2cl-lib:complex16) t$ zdum)
```

```
          (type (double-float) ainvnm scale smlnum)
          (type (f2cl-lib:integer4) ix j jp kase kase1 kd lm)
          (type (string 1) normin)
          (type f2cl-lib:logical lnoti onenrm))
 (setf info 0)
 (setf onenrm
         (or (f2cl-lib:fstring-= norm "1")
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame norm "O")
               (declare (ignore var-1))
               (when var-0 (setf norm var-0))
               ret-val)))
 (cond
  ((and (not onenrm)
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame norm "I")
           (declare (ignore var-1))
           (when var-0 (setf norm var-0))
           ret-val)))
   (setf info -1))
  ((< n 0) (setf info -2)) ((< kl 0) (setf info -3))
  ((< ku 0) (setf info -4))
  ((< ldab (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
   (setf info -6))
  ((< anorm zero) (setf info -8)))
 (cond
  ((/= info 0) (xerbla "ZGBCON" (f2cl-lib:int-sub info))
   (go end_label)))
 (setf rcond zero)
 (cond ((= n 0) (setf rcond one) (go end_label))
       ((= anorm zero) (go end_label)))
 (setf smlnum (dlamch "Safe minimum"))
 (setf ainvnm zero)
 (f2cl-lib:f2cl-set-string normin "N" (string 1))
 (cond (onenrm (setf kase1 1)) (t (setf kase1 2)))
 (setf kd (f2cl-lib:int-add kl ku 1))
 (setf lnoti (> kl 0))
 (setf kase 0)
label10
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
     (zlacn2 n
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 ((+ n 1))
                            ((1 *)) work-%offset%)
      work ainvnm kase isave)
   (declare (ignore var-1 var-2 var-5))
   (when var-0 (setf n var-0))
   (when var-3 (setf ainvnm var-3))
   (when var-4 (setf kase var-4)))
 (cond
```

```
((/= kase 0)
 (cond
  ((= kase kase1)
   (cond
    (lnoti
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                    nil)
       (tagbody
         (setf lm
                 (min (the f2cl-lib:integer4 kl)
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-sub n j))))
         (setf jp
                 (f2cl-lib:fref ipiv-%data% (j) ((1 *))
                                ipiv-%offset%))
         (setf t$
                 (f2cl-lib:fref work-%data% (jp) ((1 *))
                                work-%offset%))
         (cond
          ((/= jp j)
           (setf (f2cl-lib:fref work-%data% (jp) ((1 *))
                                work-%offset%)
                   (f2cl-lib:fref work-%data% (j) ((1 *))
                                  work-%offset%))
           (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                                work-%offset%)
                   t$)))
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
             (zaxpy lm (- t$)
              (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                    ((+ kd 1) j) ((1 ldab) (1 *))
                                    ab-%offset%)
              1
              (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                    ((+ j 1)) ((1 *)) work-%offset%)
              1)
           (declare (ignore var-1 var-2 var-3 var-4 var-5))
           (when var-0 (setf lm var-0)))
       label20))))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11)
       (zlatbs "Upper" "No transpose" "Non-unit" normin n
        (f2cl-lib:int-add kl ku) ab ldab work scale rwork info)
     (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-10))
     (when var-3 (setf normin var-3))
     (when var-4 (setf n var-4))
     (when var-7 (setf ldab var-7))
     (when var-9 (setf scale var-9))
```

```
    (when var-11 (setf info var-11))))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11)
       (zlatbs "Upper" "Conjugate transpose" "Non-unit" normin n
        (f2cl-lib:int-add kl ku) ab ldab work scale rwork info)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-10))
    (when var-3 (setf normin var-3))
    (when var-4 (setf n var-4))
    (when var-7 (setf ldab var-7))
    (when var-9 (setf scale var-9))
    (when var-11 (setf info var-11)))
   (cond
    (lnoti
     (f2cl-lib:fdo (j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                    (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   ((> j 1) nil)
       (tagbody
         (setf lm
                 (min (the f2cl-lib:integer4 kl)
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-sub n j))))
         (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                 (-
                  (f2cl-lib:fref work-%data% (j) ((1 *))
                                 work-%offset%)
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4)
                      (zdotc lm
                       (f2cl-lib:array-slice ab-%data%
                                             f2cl-lib:complex16
                                             ((+ kd 1) j)
                                             ((1 ldab) (1 *))
                                             ab-%offset%)
                       1
                       (f2cl-lib:array-slice work-%data%
                                             f2cl-lib:complex16
                                             ((+ j 1)) ((1 *))
                                             work-%offset%)
                       1)
                    (declare (ignore var-1 var-2 var-3 var-4))
                    (when var-0 (setf lm var-0))
                    ret-val)))
         (setf jp
                 (f2cl-lib:fref ipiv-%data% (j) ((1 *))
                                ipiv-%offset%))
         (cond
          ((/= jp j)
           (setf t$
```

```
                        (f2cl-lib:fref work-%data% (jp) ((1 *))
                                       work-%offset%))
                    (setf (f2cl-lib:fref work-%data% (jp) ((1 *))
                                         work-%offset%)
                        (f2cl-lib:fref work-%data% (j) ((1 *))
                                       work-%offset%))
                    (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                                         work-%offset%)
                        t$)))
                label30))))))
          (f2cl-lib:f2cl-set-string normin "Y" (string 1))
          (cond
           ((/= scale one)
            (setf ix
                    (multiple-value-bind (ret-val var-0 var-1 var-2)
                        (izamax n work 1)
                      (declare (ignore var-1 var-2))
                      (when var-0 (setf n var-0))
                      ret-val))
            (if (or
                 (< scale
                    (*
                     (cabs1
                      (f2cl-lib:fref work-%data% (ix) ((1 *)) work-%offset%))
                     smlnum))
                 (= scale zero))
                (go label40))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (zdrscl n scale work 1)
              (declare (ignore var-1 var-2 var-3))
              (setf n var-0))))
          (go label10)))
         (if (/= ainvnm zero)
             (setf rcond (/ (/ one ainvnm) anorm)))
        label40
         (go end_label)
        end_label
         (return
          (values norm n nil nil nil ldab nil nil rcond nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgbcon
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
```

```
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (double-float) (double-float)
  (array fortran-to-lisp::complex16
   (*))
  (array double-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::norm
  fortran-to-lisp::n nil nil nil
  fortran-to-lisp::ldab nil nil
  fortran-to-lisp::rcond nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zdrscl
  fortran-to-lisp::xerbla))))
```

———-

47.3 zgbequ

47.3.1 Axiom unit tests

— **zgbequ.input** —

```
)set break resume
)sys rm -f zgbequ.output
)spool zgbequ.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

47.3.2 Axiom help page

— **zgbequ.help** —

```
====================================================================
zgbequ examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgbequ (M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND, AMAX,
           INFO)
           ZGBEQU

Function/Subroutine Documentation
   subroutine zgbequ (integer M, integer N, integer KL, integer KU,
       complex*16, dimension( ldab, * ) AB, integer LDAB, double precision,
       dimension( * ) R, double precision, dimension( * ) C, double precision
       ROWCND, double precision COLCND, double precision AMAX, integer INFO)
       ZGBEQU

Purpose:
   ZGBEQU computes row and column scalings intended to equilibrate an
   M-by-N band matrix A and reduce its condition number.  R returns the
   row scale factors and C the column scale factors, chosen to try to
   make the largest element in each row and column of the matrix B with
   elements B(i,j)=R(i)*A(i,j)*C(j) have absolute value 1.

   R(i) and C(j) are restricted to be between SMLNUM = smallest safe
   number and BIGNUM = largest safe number.  Use of these scaling
   factors is not guaranteed to reduce the condition number of A but
   works well in practice.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.
```

```
AB
  AB is COMPLEX*16 array, dimension (LDAB,N)
  The band matrix A, stored in rows 1 to KL+KU+1.  The j-th
  column of A is stored in the j-th column of the array AB as
  follows:
  AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

R
  R is DOUBLE PRECISION array, dimension (M)
  If INFO = 0, or INFO > M, R contains the row scale factors for A.

C
  C is DOUBLE PRECISION array, dimension (N)
  If INFO = 0, C contains the column scale factors for A.

ROWCND
  ROWCND is DOUBLE PRECISION
  If INFO = 0 or INFO > M, ROWCND contains the ratio of the
  smallest R(i) to the largest R(i).  If ROWCND >= 0.1 and
  AMAX is neither too large nor too small, it is not worth scaling by R.

COLCND
  COLCND is DOUBLE PRECISION
  If INFO = 0, COLCND contains the ratio of the smallest
  C(i) to the largest C(i).  If COLCND >= 0.1, it is not worth scaling by C.

AMAX
  AMAX is DOUBLE PRECISION
  Absolute value of largest matrix element.  If AMAX is very
  close to overflow or very close to underflow, the matrix should be scaled.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, and i is
    <= M:  the i-th row of A is exactly zero
    >  M:  the (i-M)-th column of A is exactly zero
```

———-

47.3.3 fortran code

— zgbequ.f —

```
*  ====================================================================
      SUBROUTINE ZGBEQU( M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $                   AMAX, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, KL, KU, LDAB, M, N
      DOUBLE PRECISION   AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   C( * ), R( * )
      COMPLEX*16         AB( LDAB, * )
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, KD
      DOUBLE PRECISION   BIGNUM, RCMAX, RCMIN, SMLNUM
      COMPLEX*16         ZDUM
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, MAX, MIN
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
```

```
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGBEQU', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         ROWCND = ONE
         COLCND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Get machine constants.
*
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
*
*     Compute row scale factors.
*
      DO 10 I = 1, M
         R( I ) = ZERO
   10 CONTINUE
*
*     Find the maximum element in each row.
*
      KD = KU + 1
      DO 30 J = 1, N
         DO 20 I = MAX( J-KU, 1 ), MIN( J+KL, M )
            R( I ) = MAX( R( I ), CABS1( AB( KD+I-J, J ) ) )
   20    CONTINUE
   30 CONTINUE
*
```

```
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 40 I = 1, M
         RCMAX = MAX( RCMAX, R( I ) )
         RCMIN = MIN( RCMIN, R( I ) )
   40 CONTINUE
      AMAX = RCMAX
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 50 I = 1, M
            IF( R( I ).EQ.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   50    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 60 I = 1, M
            R( I ) = ONE / MIN( MAX( R( I ), SMLNUM ), BIGNUM )
   60    CONTINUE
*
*        Compute ROWCND = min(R(I)) / max(R(I))
*
         ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
*     Compute column scale factors
*
      DO 70 J = 1, N
         C( J ) = ZERO
   70 CONTINUE
*
*     Find the maximum element in each column,
*     assuming the row scaling computed above.
*
      KD = KU + 1
      DO 90 J = 1, N
         DO 80 I = MAX( J-KU, 1 ), MIN( J+KL, M )
            C( J ) = MAX( C( J ), CABS1( AB( KD+I-J, J ) )*R( I ) )
   80    CONTINUE
   90 CONTINUE
*
*     Find the maximum and minimum scale factors.
```

```
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 100 J = 1, N
         RCMIN = MIN( RCMIN, C( J ) )
         RCMAX = MAX( RCMAX, C( J ) )
  100 CONTINUE
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 110 J = 1, N
            IF( C( J ).EQ.ZERO ) THEN
               INFO = M + J
               RETURN
            END IF
  110    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 120 J = 1, N
            C( J ) = ONE / MIN( MAX( C( J ), SMLNUM ), BIGNUM )
  120    CONTINUE
*
*        Compute COLCND = min(C(J)) / max(C(J))
*
         COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
      RETURN
*
*     End of ZGBEQU
*
      END
```

———-

47.3.4 lisp code

— zgbequ.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun zgbequ (m n kl ku ab ldab r c rowcnd colcnd amax info)
```

```
(declare (type (double-float) amax colcnd rowcnd)
         (type (array double-float (*)) c r)
         (type (array f2cl-lib:complex16 (*)) ab)
         (type (f2cl-lib:integer4) info ldab ku kl n m))
(f2cl-lib:with-multi-array-data
    ((ab f2cl-lib:complex16 ab-%data% ab-%offset%)
     (r double-float r-%data% r-%offset%)
     (c double-float c-%data% c-%offset%))
  (labels ((cabs1 (zdum)
             (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
    (declare
     (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
      cabs1))
    (prog ((zdum #C(0.0d0 0.0d0)) (bignum 0.0d0) (rcmax 0.0d0)
           (rcmin 0.0d0) (smlnum 0.0d0) (i 0) (j 0) (kd 0))
      (declare (type (f2cl-lib:complex16) zdum)
               (type (double-float) bignum rcmax rcmin smlnum)
               (type (f2cl-lib:integer4) i j kd))
      (setf info 0)
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< kl 0) (setf info -3)) ((< ku 0) (setf info -4))
            ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -6)))
      (cond
       ((/= info 0) (xerbla "ZGBEQU" (f2cl-lib:int-sub info))
        (go end_label)))
      (cond
       ((or (= m 0) (= n 0)) (setf rowcnd one) (setf colcnd one)
        (setf amax zero) (go end_label)))
      (setf smlnum (dlamch "S"))
      (setf bignum (/ one smlnum))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%) zero)
         label10))
      (setf kd (f2cl-lib:int-add ku 1))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                          (the f2cl-lib:integer4 1))
                         (f2cl-lib:int-add i 1))
                        ((> i
                            (min
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add j kl))
                             (the f2cl-lib:integer4 m)))
```

```
                    nil)
        (tagbody
          (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                  (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                       (cabs1
                        (f2cl-lib:fref ab-%data%
                                       ((f2cl-lib:int-sub
                                         (f2cl-lib:int-add kd i) j)
                                        j)
                                       ((1 ldab) (1 *)) ab-%offset%))))
         label20))
     label30))
  (setf rcmin bignum)
  (setf rcmax zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf rcmax
              (max rcmax
                   (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
      (setf rcmin
              (min rcmin
                   (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
     label40))
  (setf amax rcmax)
  (cond
   ((= rcmin zero)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (cond
         ((= (f2cl-lib:fref r (i) ((1 *))) zero) (setf info i)
          (go end_label)))
       label50)))
   (t
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                (/ one
                   (min
                    (max
                     (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                     smlnum)
                    bignum)))
       label60))
    (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
```

```
    (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) zero)
   label70))
(setf kd (f2cl-lib:int-add ku 1))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                    (the f2cl-lib:integer4 1))
                   (f2cl-lib:int-add i 1))
                  ((> i
                      (min
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add j kl))
                       (the f2cl-lib:integer4 m)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                     (*
                      (cabs1
                       (f2cl-lib:fref ab-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add kd i) j)
                                       j)
                                      ((1 ldab) (1 *)) ab-%offset%))
                      (f2cl-lib:fref r-%data% (i) ((1 *))
                                     r-%offset%))))
       label80))
   label90))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf rcmin
            (min rcmin
                 (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
    (setf rcmax
            (max rcmax
                 (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
   label100))
(cond
 ((= rcmin zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (cond
```

```
                 ((= (f2cl-lib:fref c (j) ((1 *))) zero)
                  (setf info (f2cl-lib:int-add m j)) (go end_label)))
               label110)))
           (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                        (/ one
                           (min
                            (max
                             (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                             smlnum)
                            bignum)))
               label120))
            (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
          (go end_label)
         end_label
          (return
           (values nil nil nil nil nil nil nil nil rowcnd colcnd amax
                   info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgbequ
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (double-float) (double-float)
                                              (double-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
                                              fortran-to-lisp::rowcnd
                                              fortran-to-lisp::colcnd
                                              fortran-to-lisp::amax
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

47.4 zgbequb

47.4.1 Axiom unit tests

— zgbequb.input —

```
)set break resume
)sys rm -f zgbequb.output
)spool zgbequb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

47.4.2 Axiom help page

— zgbequb.help —

```
======================================================================
zgbequb examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgbequb (M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND, AMAX,
           INFO)
           ZGBEQUB

Function/Subroutine Documentation
   subroutine zgbequb (integer M, integer N, integer KL, integer KU,
       complex*16, dimension( ldab, * ) AB, integer LDAB, double precision,
       dimension( * ) R, double precision, dimension( * ) C, double precision
       ROWCND, double precision COLCND, double precision AMAX, integer INFO)
       ZGBEQUB
```

```
Purpose:
   ZGBEQUB computes row and column scalings intended to equilibrate an
   M-by-N matrix A and reduce its condition number.  R returns the row
   scale factors and C the column scale factors, chosen to try to make
   the largest element in each row and column of the matrix B with
   elements B(i,j)=R(i)*A(i,j)*C(j) have an absolute value of at most
   the radix.

   R(i) and C(j) are restricted to be a power of the radix between
   SMLNUM = smallest safe number and BIGNUM = largest safe number.  Use
   of these scaling factors is not guaranteed to reduce the condition
   number of A but works well in practice.

   This routine differs from ZGEEQU by restricting the scaling factors
   to a power of the radix.  Baring over- and underflow, scaling by
   these factors introduces no additional rounding errors.  However, the
   scaled entries' magnitured are no longer approximately 1 but lie
   between sqrt(radix) and 1/sqrt(radix).

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 AB
   AB is DOUBLE PRECISION array, dimension (LDAB,N)
   On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
   The j-th column of A is stored in the j-th column of the
   array AB as follows:
   AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

 LDAB
   LDAB is INTEGER
   The leading dimension of the array A.  LDAB >= max(1,M).

 R
```

```
  R is DOUBLE PRECISION array, dimension (M)
  If INFO = 0 or INFO > M, R contains the row scale factors for A.

C
  C is DOUBLE PRECISION array, dimension (N)
  If INFO = 0,  C contains the column scale factors for A.

ROWCND
  ROWCND is DOUBLE PRECISION
  If INFO = 0 or INFO > M, ROWCND contains the ratio of the
  smallest R(i) to the largest R(i).  If ROWCND >= 0.1 and
  AMAX is neither too large nor too small, it is not worth scaling by R.

COLCND
  COLCND is DOUBLE PRECISION
  If INFO = 0, COLCND contains the ratio of the smallest
  C(i) to the largest C(i).  If COLCND >= 0.1, it is not worth scaling by C.

AMAX
  AMAX is DOUBLE PRECISION
  Absolute value of largest matrix element.  If AMAX is very
  close to overflow or very close to underflow, the matrix should be scaled.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i,  and i is
     <= M:  the i-th row of A is exactly zero
     >  M:  the (i-M)-th column of A is exactly zero
```

47.4.3 fortran code

— zgbequb.f —

```
*  =====================================================================
      SUBROUTINE ZGBEQUB( M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $                    AMAX, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
```

```
      INTEGER            INFO, KL, KU, LDAB, M, N
      DOUBLE PRECISION   AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   C( * ), R( * )
      COMPLEX*16         AB( LDAB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, KD
      DOUBLE PRECISION   BIGNUM, RCMAX, RCMIN, SMLNUM, RADIX,
     $                   LOGRDX
      COMPLEX*16         ZDUM
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, LOG, REAL, DIMAG
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
```

```
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGBEQUB', -INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         ROWCND = ONE
         COLCND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Get machine constants.  Assume SMLNUM is a power of the radix.
*
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      RADIX = DLAMCH( 'B' )
      LOGRDX = LOG(RADIX)
*
*     Compute row scale factors.
*
      DO 10 I = 1, M
         R( I ) = ZERO
   10 CONTINUE
*
*     Find the maximum element in each row.
*
      KD = KU + 1
      DO 30 J = 1, N
         DO 20 I = MAX( J-KU, 1 ), MIN( J+KL, M )
            R( I ) = MAX( R( I ), CABS1( AB( KD+I-J, J ) ) )
   20    CONTINUE
   30 CONTINUE
      DO I = 1, M
         IF( R( I ).GT.ZERO ) THEN
            R( I ) = RADIX**INT( LOG( R( I ) ) / LOGRDX )
         END IF
      END DO
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 40 I = 1, M
         RCMAX = MAX( RCMAX, R( I ) )
         RCMIN = MIN( RCMIN, R( I ) )
```

```
   40 CONTINUE
      AMAX = RCMAX
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 50 I = 1, M
            IF( R( I ).EQ.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   50    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 60 I = 1, M
            R( I ) = ONE / MIN( MAX( R( I ), SMLNUM ), BIGNUM )
   60    CONTINUE
*
*        Compute ROWCND = min(R(I)) / max(R(I)).
*
         ROWCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
*     Compute column scale factors.
*
      DO 70 J = 1, N
         C( J ) = ZERO
   70 CONTINUE
*
*     Find the maximum element in each column,
*     assuming the row scaling computed above.
*
      DO 90 J = 1, N
         DO 80 I = MAX( J-KU, 1 ), MIN( J+KL, M )
            C( J ) = MAX( C( J ), CABS1( AB( KD+I-J, J ) )*R( I ) )
   80    CONTINUE
         IF( C( J ).GT.ZERO ) THEN
            C( J ) = RADIX**INT( LOG( C( J ) ) / LOGRDX )
         END IF
   90 CONTINUE
*
*     Find the maximum and minimum scale factors.
*
      RCMIN = BIGNUM
      RCMAX = ZERO
      DO 100 J = 1, N
         RCMIN = MIN( RCMIN, C( J ) )
```

```
         RCMAX = MAX( RCMAX, C( J ) )
  100 CONTINUE
*
      IF( RCMIN.EQ.ZERO ) THEN
*
*        Find the first zero scale factor and return an error code.
*
         DO 110 J = 1, N
            IF( C( J ).EQ.ZERO ) THEN
               INFO = M + J
               RETURN
            END IF
  110    CONTINUE
      ELSE
*
*        Invert the scale factors.
*
         DO 120 J = 1, N
            C( J ) = ONE / MIN( MAX( C( J ), SMLNUM ), BIGNUM )
  120    CONTINUE
*
*        Compute COLCND = min(C(J)) / max(C(J)).
*
         COLCND = MAX( RCMIN, SMLNUM ) / MIN( RCMAX, BIGNUM )
      END IF
*
      RETURN
*
*     End of ZGBEQUB
*
      END
```

———-

47.4.4 lisp code

— zgbequb.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun zgbequb (m n kl ku ab ldab r c rowcnd colcnd amax info)
    (declare (type (double-float) amax colcnd rowcnd)
             (type (array double-float (*)) c r)
             (type (array f2cl-lib:complex16 (*)) ab)
             (type (f2cl-lib:integer4) info ldab ku kl n m))
    (f2cl-lib:with-multi-array-data
```

```
  ((ab f2cl-lib:complex16 ab-%data% ab-%offset%)
   (r double-float r-%data% r-%offset%)
   (c double-float c-%data% c-%offset%))
(labels ((cabs1 (zdum)
           (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
  (declare
   (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
    cabs1))
  (prog ((zdum #C(0.0d0 0.0d0)) (bignum 0.0d0) (rcmax 0.0d0)
         (rcmin 0.0d0) (smlnum 0.0d0) (radix 0.0d0) (logrdx 0.0d0) (i 0)
         (j 0) (kd 0))
    (declare (type (f2cl-lib:complex16) zdum)
             (type (double-float) bignum rcmax rcmin smlnum radix logrdx)
             (type (f2cl-lib:integer4) i j kd))
    (setf info 0)
    (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
          ((< kl 0) (setf info -3)) ((< ku 0) (setf info -4))
          ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -6)))
    (cond
     ((/= info 0) (xerbla "ZGBEQUB" (f2cl-lib:int-sub info))
      (go end_label)))
    (cond
     ((or (= m 0) (= n 0)) (setf rowcnd one) (setf colcnd one)
      (setf amax zero) (go end_label)))
    (setf smlnum (dlamch "S"))
    (setf bignum (/ one smlnum))
    (setf radix (dlamch "B"))
    (setf logrdx (f2cl-lib:flog radix))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%) zero)
       label10))
    (setf kd (f2cl-lib:int-add ku 1))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i
                       (max
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                        (the f2cl-lib:integer4 1))
                       (f2cl-lib:int-add i 1))
                      ((> i
                          (min
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add j kl))
                           (the f2cl-lib:integer4 m)))
                       nil)
         (tagbody
```

```
        (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                (max (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                     (cabs1
                      (f2cl-lib:fref ab-%data%
                                     ((f2cl-lib:int-sub
                                       (f2cl-lib:int-add kd i) j)
                                      j)
                                     ((1 ldab) (1 *)) ab-%offset%))))
      label20))
   label30))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref r (i) ((1 *))) zero)
      (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
              (expt radix
                    (f2cl-lib:int
                     (/
                      (f2cl-lib:flog
                       (f2cl-lib:fref r-%data% (i) ((1 *))
                                      r-%offset%))
                      logrdx))))))
   label100000))
(setf rcmin bignum)
(setf rcmax zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf rcmax
            (max rcmax
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
    (setf rcmin
            (min rcmin
                 (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)))
   label40))
(setf amax rcmax)
(cond
 ((= rcmin zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref r (i) ((1 *))) zero) (setf info i)
        (go end_label)))
     label50)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
```

```
        (setf (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                (/ one
                   (min
                    (max
                     (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                     smlnum)
                    bignum)))
      label60))
  (setf rowcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%) zero)
   label70))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                    (the f2cl-lib:integer4 1))
                   (f2cl-lib:int-add i 1))
                  ((> i
                      (min
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add j kl))
                       (the f2cl-lib:integer4 m)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                (max (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                     (*
                      (cabs1
                       (f2cl-lib:fref ab-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add kd i) j)
                                       j)
                                      ((1 ldab) (1 *)) ab-%offset%))
                      (f2cl-lib:fref r-%data% (i) ((1 *))
                                     r-%offset%))))
       label80))
    (cond
     ((> (f2cl-lib:fref c (j) ((1 *))) zero)
      (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
              (expt radix
                    (f2cl-lib:int
                     (/
                      (f2cl-lib:flog
                       (f2cl-lib:fref c-%data% (j) ((1 *))
```

```
                                                c-%offset%))
                                logrdx))))))
            label90))
          (setf rcmin bignum)
          (setf rcmax zero)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf rcmin
                      (min rcmin
                           (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
              (setf rcmax
                      (max rcmax
                           (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)))
            label100))
          (cond
           ((= rcmin zero)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (cond
                 ((= (f2cl-lib:fref c (j) ((1 *))) zero)
                  (setf info (f2cl-lib:int-add m j)) (go end_label)))
               label110)))
           (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                        (/ one
                           (min
                            (max
                             (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                             smlnum)
                            bignum)))
               label120))
            (setf colcnd (/ (max rcmin smlnum) (min rcmax bignum)))))
          (go end_label)
         end_label
          (return
           (values nil nil nil nil nil nil nil nil rowcnd colcnd amax
                   info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgbequb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
```

```
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (double-float) (double-float)
  (double-float)
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil nil nil nil nil nil
  fortran-to-lisp::rowcnd
  fortran-to-lisp::colcnd
  fortran-to-lisp::amax
  fortran-to-lisp::info)
:calls '(fortran-to-lisp::xerbla))))
```

———-

47.5 zgbrfs

47.5.1 Axiom unit tests

— zgbrfs.input —

```
)set break resume
)sys rm -f zgbrfs.output
)spool zgbrfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

47.5.2 Axiom help page

— zgbrfs.help —

```
====================================================================
zgbrfs examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgbrfs (TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB, IPIV,
           B, LDB, X, LDX, FERR, BERR, WORK, RWORK, INFO)
           ZGBRFS

Function/Subroutine Documentation
   subroutine zgbrfs (character TRANS, integer N, integer KL, integer KU,
       integer NRHS, complex*16, dimension( ldab, * ) AB, integer LDAB,
       complex*16, dimension( ldafb, * ) AFB, integer LDAFB, integer,
       dimension( * ) IPIV, complex*16, dimension( ldb, * ) B, integer LDB,
       complex*16, dimension( ldx, * ) X, integer LDX, double precision,
       dimension( * ) FERR, double precision, dimension( * ) BERR, complex*16,
       dimension( * ) WORK, double precision, dimension( * ) RWORK, integer
       INFO)
       ZGBRFS

Purpose:
   ZGBRFS improves the computed solution to a system of linear
   equations when the coefficient matrix is banded, and provides
   error bounds and backward error estimates for the solution.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.
```

```
NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

AB
  AB is COMPLEX*16 array, dimension (LDAB,N)
  The original band matrix A, stored in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(n,j+kl).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is COMPLEX*16 array, dimension (LDAFB,N)
  Details of the LU factorization of the band matrix A, as
  computed by ZGBTRF.  U is stored as an upper triangular band
  matrix with KL+KU superdiagonals in rows 1 to KL+KU+1, and
  the multipliers used during the factorization are stored in
  rows KL+KU+2 to 2*KL+KU+1.

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL*KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from ZGBTRF; for 1<=i<=N, row i of the
  matrix was interchanged with row IPIV(i).

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by ZGBTRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).
```

```
 FERR
   FERR is DOUBLE PRECISION array, dimension (NRHS)
   The estimated forward error bound for each solution vector
   X(j) (the j-th column of the solution matrix X).
   If XTRUE is the true solution corresponding to X(j), FERR(j)
   is an estimated upper bound for the magnitude of the largest
   element in (X(j) - XTRUE) divided by the magnitude of the
   largest element in X(j).  The estimate is as reliable as
   the estimate for RCOND, and is almost always a slight
   overestimate of the true error.

 BERR
   BERR is DOUBLE PRECISION array, dimension (NRHS)
   The componentwise relative backward error of each solution
   vector X(j) (i.e., the smallest relative change in
   any element of A or B that makes X(j) an exact solution).

 WORK
   WORK is COMPLEX*16 array, dimension (2*N)

 RWORK
   RWORK is DOUBLE PRECISION array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

 ITMAX is the maximum number of steps of iterative refinement.
```

———-

47.5.3 fortran code

— zgbrfs.f —

```
*  =====================================================================
      SUBROUTINE ZGBRFS( TRANS, N, KL, KU, NRHS, AB, LDAB, AFB, LDAFB,
     $                   IPIV, B, LDB, X, LDX, FERR, BERR, WORK, RWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
```

```
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, KL, KU, LDAB, LDAFB, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   BERR( * ), FERR( * ), RWORK( * )
      COMPLEX*16         AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
      COMPLEX*16         CONE
      PARAMETER          ( CONE = ( 1.0D+0, 0.0D+0 ) )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D+0 )
      DOUBLE PRECISION   THREE
      PARAMETER          ( THREE = 3.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      CHARACTER          TRANSN, TRANST
      INTEGER            COUNT, I, J, K, KASE, KK, NZ
      DOUBLE PRECISION   EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN, XK
      COMPLEX*16         ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZAXPY, ZCOPY, ZGBMV, ZGBTRS, ZLACN2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, MAX, MIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
```

```
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -7
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
         INFO = -9
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -12
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -14
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGBRFS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
      IF( NOTRAN ) THEN
         TRANSN = 'N'
         TRANST = 'C'
      ELSE
         TRANSN = 'C'
```

```
         TRANST = 'N'
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = MIN( KL+KU+2, N+1 )
      EPS = DLAMCH( 'Epsilon' )
      SAFMIN = DLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 140 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - op(A) * X,
*        where op(A) = A, A**T, or A**H, depending on TRANS.
*
         CALL ZCOPY( N, B( 1, J ), 1, WORK, 1 )
         CALL ZGBMV( TRANS, N, N, KL, KU, -CONE, AB, LDAB, X( 1, J ), 1,
     $               CONE, WORK, 1 )
*
*        Compute componentwise relative backward error from formula
*
*        max(i) ( abs(R(i)) / ( abs(op(A))*abs(X) + abs(B) )(i) )
*
*        where abs(Z) is the componentwise absolute value of the matrix
*        or vector Z.  If the i-th component of the denominator is less
*        than SAFE2, then SAFE1 is added to the i-th components of the
*        numerator and denominator before dividing.
*
         DO 30 I = 1, N
            RWORK( I ) = CABS1( B( I, J ) )
   30    CONTINUE
*
*        Compute abs(op(A))*abs(X) + abs(B).
*
         IF( NOTRAN ) THEN
            DO 50 K = 1, N
               KK = KU + 1 - K
               XK = CABS1( X( K, J ) )
               DO 40 I = MAX( 1, K-KU ), MIN( N, K+KL )
                  RWORK( I ) = RWORK( I ) + CABS1( AB( KK+I, K ) )*XK
   40          CONTINUE
```

```
   50       CONTINUE
         ELSE
            DO 70 K = 1, N
               S = ZERO
               KK = KU + 1 - K
               DO 60 I = MAX( 1, K-KU ), MIN( N, K+KL )
                  S = S + CABS1( AB( KK+I, K ) )*CABS1( X( I, J ) )
   60          CONTINUE
               RWORK( K ) = RWORK( K ) + S
   70       CONTINUE
         END IF
         S = ZERO
         DO 80 I = 1, N
            IF( RWORK( I ).GT.SAFE2 ) THEN
               S = MAX( S, CABS1( WORK( I ) ) / RWORK( I ) )
            ELSE
               S = MAX( S, ( CABS1( WORK( I ) )+SAFE1 ) /
     $             ( RWORK( I )+SAFE1 ) )
            END IF
   80    CONTINUE
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
*           1) The residual BERR(J) is larger than machine epsilon, and
*           2) BERR(J) decreased by at least a factor of 2 during the
*              last iteration, and
*           3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
*
*           Update solution and try again.
*
            CALL ZGBTRS( TRANS, N, KL, KU, 1, AFB, LDAFB, IPIV, WORK, N,
     $                   INFO )
            CALL ZAXPY( N, CONE, WORK, 1, X( 1, J ), 1 )
            LSTRES = BERR( J )
            COUNT = COUNT + 1
            GO TO 20
         END IF
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(op(A)))*
*           ( abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
*          inv(op(A)) is the inverse of op(A)
```

```
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(op(A))*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(op(A))*abs(X) + abs(B) is less than SAFE2.
*
*        Use ZLACN2 to estimate the infinity-norm of the matrix
*           inv(op(A)) * diag(W),
*        where W = abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) )))
*
         DO 90 I = 1, N
            IF( RWORK( I ).GT.SAFE2 ) THEN
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I )
            ELSE
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I ) +
     $                      SAFE1
            END IF
   90    CONTINUE
*
         KASE = 0
  100    CONTINUE
         CALL ZLACN2( N, WORK( N+1 ), WORK, FERR( J ), KASE, ISAVE )
         IF( KASE.NE.0 ) THEN
            IF( KASE.EQ.1 ) THEN
*
               Multiply by diag(W)*inv(op(A)**H).
*
               CALL ZGBTRS( TRANST, N, KL, KU, 1, AFB, LDAFB, IPIV,
     $                      WORK, N, INFO )
               DO 110 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
  110          CONTINUE
            ELSE
*
               Multiply by inv(op(A))*diag(W).
*
               DO 120 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
  120          CONTINUE
               CALL ZGBTRS( TRANSN, N, KL, KU, 1, AFB, LDAFB, IPIV,
     $                      WORK, N, INFO )
            END IF
            GO TO 100
         END IF
*
         Normalize error.
*
```

```
         LSTRES = ZERO
         DO 130 I = 1, N
            LSTRES = MAX( LSTRES, CABS1( X( I, J ) ) )
  130    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  140 CONTINUE
*
      RETURN
*
*     End of ZGBRFS
*
      END
```

———-

47.5.4 lisp code

— zgbrfs.lisp —

```
(let* ((itmax 5)
       (zero 0.0d0)
       (cone (f2cl-lib:cmplx 1.0d0 0.0d0))
       (two 2.0d0)
       (three 3.0d0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (f2cl-lib:complex16) cone)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 3.0d0 3.0d0) three)
           (ignorable itmax zero cone two three))
  (defun zgbrfs
         (trans n kl ku nrhs ab ldab afb ldafb ipiv b ldb$ x ldx ferr berr work
          rwork info)
    (declare (type (array double-float (*)) rwork berr ferr)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) work x b afb ab)
             (type (f2cl-lib:integer4) info ldx ldb$ ldafb ldab nrhs ku kl n)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (ab f2cl-lib:complex16 ab-%data% ab-%offset%)
         (afb f2cl-lib:complex16 afb-%data% afb-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (x f2cl-lib:complex16 x-%data% x-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
```

```
   (ferr double-float ferr-%data% ferr-%offset%)
   (berr double-float berr-%data% berr-%offset%)
   (rwork double-float rwork-%data% rwork-%offset%))
(labels ((cabs1 (zdum)
           (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
  (declare
   (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
    cabs1))
  (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
         (zdum #C(0.0d0 0.0d0)) (eps 0.0d0) (lstres 0.0d0) (s 0.0d0)
         (safe1 0.0d0) (safe2 0.0d0) (safmin 0.0d0) (xk 0.0d0) (count$ 0)
         (i 0) (j 0) (k 0) (kase 0) (kk 0) (nz 0)
         (transn
          (make-array '(1) :element-type 'character :initial-element
                      #\ ))
         (transt
          (make-array '(1) :element-type 'character :initial-element
                      #\ ))
         (notran nil))
    (declare (type (array f2cl-lib:integer4 (3)) isave)
             (type (f2cl-lib:complex16) zdum)
             (type (double-float) eps lstres s safe1 safe2 safmin xk)
             (type (f2cl-lib:integer4) count$ i j k kase kk nz)
             (type (string 1) transn transt)
             (type f2cl-lib:logical notran))
    (setf info 0)
    (setf notran
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "N")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val))
    (cond
     ((and (not notran)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "T")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "C")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val)))
      (setf info -1))
     ((< n 0) (setf info -2)) ((< kl 0) (setf info -3))
     ((< ku 0) (setf info -4)) ((< nrhs 0) (setf info -5))
     ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -7))
```

```
 ((< ldafb (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
  (setf info -9))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -12))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -14)))
(cond
 ((/= info 0) (xerbla "ZGBRFS" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
              zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%)
              zero)
     label10))
  (go end_label)))
(cond
 (notran (f2cl-lib:f2cl-set-string transn "N" (string 1))
  (f2cl-lib:f2cl-set-string transt "C" (string 1)))
 (t (f2cl-lib:f2cl-set-string transn "C" (string 1))
  (f2cl-lib:f2cl-set-string transt "N" (string 1))))
(setf nz
        (min (the f2cl-lib:integer4 (f2cl-lib:int-add kl ku 2))
             (the f2cl-lib:integer4 (f2cl-lib:int-add n 1))))
(setf eps (dlamch "Epsilon"))
(setf safmin (dlamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zcopy n
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1 work 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (zgbmv trans n n kl ku (- cone) ab ldab
         (f2cl-lib:array-slice x-%data% f2cl-lib:complex16 (1 j)
```

```
                          ((1 ldx) (1 *)) x-%offset%)
     1 cone work 1)
  (declare (ignore var-5 var-6 var-8 var-9 var-11 var-12))
  (when var-0 (setf trans var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf kl var-3))
  (when var-4 (setf ku var-4))
  (when var-7 (setf ldab var-7))
  (when var-10 (setf cone var-10)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
            (cabs1
             (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                            b-%offset%)))
   label30))
(cond
 (notran
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (setf kk (f2cl-lib:int-sub (f2cl-lib:int-add ku 1) k))
      (setf xk
              (cabs1
               (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                              x-%offset%)))
      (f2cl-lib:fdo (i
                     (max (the f2cl-lib:integer4 1)
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add k
                                                 (f2cl-lib:int-sub
                                                  ku))))
                     (f2cl-lib:int-add i 1))
                    ((> i
                        (min (the f2cl-lib:integer4 n)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add k kl))))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                               rwork-%offset%)
                  (+
                   (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                  rwork-%offset%)
                   (*
                    (cabs1
                     (f2cl-lib:fref ab-%data%
                                    ((f2cl-lib:int-add kk i) k)
```

```
                                        ((1 ldab) (1 *))
                                        ab-%offset%))
                        xk)))
            label40))
        label50)))
    (t
     (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                   ((> k n) nil)
       (tagbody
         (setf s zero)
         (setf kk (f2cl-lib:int-sub (f2cl-lib:int-add ku 1) k))
         (f2cl-lib:fdo (i
                        (max (the f2cl-lib:integer4 1)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add k
                                                    (f2cl-lib:int-sub
                                                     ku))))
                        (f2cl-lib:int-add i 1))
                       ((> i
                           (min (the f2cl-lib:integer4 n)
                                (the f2cl-lib:integer4
                                     (f2cl-lib:int-add k kl))))
                        nil)
           (tagbody
             (setf s
                     (+ s
                        (*
                         (cabs1
                          (f2cl-lib:fref ab-%data%
                                         ((f2cl-lib:int-add kk i) k)
                                         ((1 ldab) (1 *))
                                         ab-%offset%))
                         (cabs1
                          (f2cl-lib:fref x-%data% (i j)
                                         ((1 ldx) (1 *))
                                         x-%offset%)))))
            label60))
         (setf (f2cl-lib:fref rwork-%data% (k) ((1 *))
                              rwork-%offset%)
               (+
                (f2cl-lib:fref rwork-%data% (k) ((1 *))
                               rwork-%offset%)
                s))
        label70))))
    (setf s zero)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (cond
         ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
```

```
      (setf s
              (max s
                   (/
                    (cabs1
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%))
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%)))))
     (t
      (setf s
              (max s
                   (/
                    (+
                     (cabs1
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%))
                     safe1)
                    (+
                     (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                    rwork-%offset%)
                     safe1))))))
   label80))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (zgbtrs trans n kl ku 1 afb ldafb ipiv work n info)
    (declare (ignore var-4 var-5 var-7 var-8))
    (when var-0 (setf trans var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf kl var-2))
    (when var-3 (setf ku var-3))
    (when var-6 (setf ldafb var-6))
    (when var-9 (setf n var-9))
    (when var-10 (setf info var-10)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (zaxpy n cone work 1
       (f2cl-lib:array-slice x-%data% f2cl-lib:complex16 (1 j)
                             ((1 ldx) (1 *)) x-%offset%)
       1)
    (declare (ignore var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))
    (when var-1 (setf cone var-1)))
  (setf lstres
          (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
  (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
```

```
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (cond
      ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
       (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                            rwork-%offset%)
               (+
                (cabs1
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%))
                (* nz eps
                   (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                  rwork-%offset%)))))
      (t
       (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                            rwork-%offset%)
               (+
                (cabs1
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%))
                (* nz eps
                   (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                  rwork-%offset%))
                safe1))))
    label90))
 (setf kase 0)
label100
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
     (zlacn2 n
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            ((+ n 1)) ((1 *)) work-%offset%)
      work
      (f2cl-lib:array-slice ferr-%data% double-float (j) ((1 *))
                            ferr-%offset%)
      kase isave)
   (declare (ignore var-1 var-2 var-3 var-5))
   (when var-0 (setf n var-0))
   (when var-4 (setf kase var-4)))
 (cond
  ((/= kase 0)
   (cond
    ((= kase 1)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
          var-9 var-10)
         (zgbtrs transt n kl ku 1 afb ldafb ipiv work n info)
       (declare (ignore var-4 var-5 var-7 var-8))
       (when var-0 (setf transt var-0))
       (when var-1 (setf n var-1))
```

```
        (when var-2 (setf kl var-2))
        (when var-3 (setf ku var-3))
        (when var-6 (setf ldafb var-6))
        (when var-9 (setf n var-9))
        (when var-10 (setf info var-10)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                  (*
                   (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                  rwork-%offset%)
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)))
         label110)))
    (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                  (*
                   (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                  rwork-%offset%)
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)))
         label120))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (zgbtrs transn n kl ku 1 afb ldafb ipiv work n info)
        (declare (ignore var-4 var-5 var-7 var-8))
        (when var-0 (setf transn var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf kl var-2))
        (when var-3 (setf ku var-3))
        (when var-6 (setf ldafb var-6))
        (when var-9 (setf n var-9))
        (when var-10 (setf info var-10)))))
  (go label100)))
(setf lstres zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf lstres
            (max lstres
                 (cabs1
                  (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                 x-%offset%))))
```

```
                label130))
            (if (/= lstres zero)
                (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                        (/
                         (f2cl-lib:fref ferr-%data% (j) ((1 *))
                                        ferr-%offset%)
                         lstres)))
           label140))
         (go end_label)
        end_label
         (return
          (values trans n kl ku nil nil ldab nil ldafb nil nil nil nil nil nil
                  nil nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgbrfs
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::integer4
                                              (*))
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (array double-float (*))
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(fortran-to-lisp::trans
                                             fortran-to-lisp::n
                                             fortran-to-lisp::kl
                                             fortran-to-lisp::ku nil nil
```

```
                                        fortran-to-lisp::ldab nil
                                        fortran-to-lisp::ldafb nil nil
                                        nil nil nil nil nil nil nil
                                        fortran-to-lisp::info)
                                    :calls '(fortran-to-lisp::xerbla))))
```

———-

47.6 zgbrfsx

47.6.1 Axiom unit tests

— **zgbrfsx.input** —

```
)set break resume
)sys rm -f zgbrfsx.output
)spool zgbrfsx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

47.6.2 Axiom help page [Demm05]

— **zgbrfsx.help** —

```
======================================================================
zgbrfsx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgbrfsx (TRANS, EQUED, N, KL, KU, NRHS, AB, LDAB, AFB,
           LDAFB, IPIV, R, C, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS,
```

```
          ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, RWORK, INFO)
          ZGBRFSX

Function/Subroutine Documentation
   subroutine zgbrfsx (character TRANS, character EQUED, integer N, integer
       KL, integer KU, integer NRHS, complex*16, dimension( ldab, * ) AB,
       integer LDAB, complex*16, dimension( ldafb, * ) AFB, integer LDAFB,
       integer, dimension( * ) IPIV, double precision, dimension( * ) R,
       double precision, dimension( * ) C, complex*16, dimension( ldb, * ) B,
       integer LDB, complex*16, dimension( ldx , * ) X, integer LDX, double
       precision RCOND, double precision, dimension( * ) BERR, integer
       N_ERR_BNDS, double precision, dimension( nrhs, * ) ERR_BNDS_NORM,
       double precision, dimension( nrhs, * ) ERR_BNDS_COMP, integer NPARAMS,
       double precision, dimension( * ) PARAMS, complex*16, dimension( * )
       WORK, double precision, dimension( * ) RWORK, integer INFO)
       ZGBRFSX

Purpose:
   ZGBRFSX improves the computed solution to a system of linear
   equations and provides error bounds and backward error estimates
   for the solution.  In addition to normwise error bound, the code
   provides maximum componentwise error bound if possible.  See
   comments for ERR_BNDS_NORM and ERR_BNDS_COMP for details of the
   error bounds.

   The original system of linear equations may have been equilibrated
   before calling this routine, as described by arguments EQUED, R
   and C below. In this case, the solution and error bounds returned
   are for the original unequilibrated system.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose = Transpose)

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done to A
   before calling this routine. This is needed to compute
   the solution and error bounds correctly.
```

```
  = 'N':  No equilibration
  = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
  = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
  = 'B':  Both row and column equilibration, i.e., A has been
          replaced by diag(R) * A * diag(C).
          The right hand side B has been changed accordingly.

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

KL
  KL is INTEGER
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

AB
  AB is COMPLEX*16 array, dimension (LDAB,N)
  The original band matrix A, stored in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(n,j+kl).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is COMPLEX*16 array, dimension (LDAFB,N)
  Details of the LU factorization of the band matrix A, as
  computed by DGBTRF.  U is stored as an upper triangular band
  matrix with KL+KU superdiagonals in rows 1 to KL+KU+1, and
  the multipliers used during the factorization are stored in
  rows KL+KU+2 to 2*KL+KU+1.

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL*KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from DGETRF; for 1<=i<=N, row i of the
```

```
  matrix was interchanged with row IPIV(i).

R
  R is DOUBLE PRECISION array, dimension (N)
  The row scale factors for A.  If EQUED = 'R' or 'B', A is
  multiplied on the left by diag(R); if EQUED = 'N' or 'C', R
  is not accessed.  R is an input argument if FACT = 'F';
  otherwise, R is an output argument.  If FACT = 'F' and
  EQUED = 'R' or 'B', each element of R must be positive.
  If R is output, each element of R is a power of the radix.
  If R is input, each element of R should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

C
  C is DOUBLE PRECISION array, dimension (N)
  The column scale factors for A.  If EQUED = 'C' or 'B', A is
  multiplied on the right by diag(C); if EQUED = 'N' or 'R', C
  is not accessed.  C is an input argument if FACT = 'F';
  otherwise, C is an output argument.  If FACT = 'F' and
  EQUED = 'C' or 'B', each element of C must be positive.
  If C is output, each element of C is a power of the radix.
  If C is input, each element of C should be a power of the radix
  to ensure a reliable solution and error estimates. Scaling by
  powers of the radix does not cause rounding errors unless the
  result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by DGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
```

```
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

BERR
  BERR is COMPLEX*16 array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
         max_j (abs(XTRUE(j,i) - X(j,i)))
          ------------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
    reciprocal condition number is less than the threshold
    sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
    almost certainly within a factor of 10 of the true error
    so long as the next entry is greater than the threshold
    sqrt(n) * dlamch('Epsilon'). This error bound should only
    be trusted if the previous boolean is true.
```

```
  err = 3  Reciprocal condition number: Estimated normwise
    reciprocal condition number.  Compared with the threshold
    sqrt(n) * dlamch('Epsilon') to determine if the error
    estimate is "guaranteed". These reciprocal condition
    numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
    appropriately scaled matrix Z.
    Let Z = S*A, where S scales each row by a power of the
    radix so all absolute row sums of Z are approximately 1.

ERR_BNDS_COMP
  ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                 abs(XTRUE(j,i) - X(j,i))
           max_j ----------------------
                       abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
    reciprocal condition number is less than the threshold
    sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
    almost certainly within a factor of 10 of the true error
    so long as the next entry is greater than the threshold
    sqrt(n) * dlamch('Epsilon'). This error bound should only
    be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
    reciprocal condition number.  Compared with the threshold
    sqrt(n) * dlamch('Epsilon') to determine if the error
    estimate is "guaranteed". These reciprocal condition
    numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
    appropriately scaled matrix Z.
```

```
    Let Z = S*(A*diag(x)), where x is the solution for the
    current right-hand side and S scales each row of
    A*diag(x) by a power of the radix so all absolute row
    sums of Z are approximately 1.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is DOUBLE PRECISION array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0D+0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
  (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is COMPLEX*16 array, dimension (2*N)

RWORK
  RWORK is DOUBLE PRECISION array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is
```

```
   guaranteed.
   < 0:  If INFO = -i, the i-th argument had an illegal value
   > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
       has been completed, but the factor U is exactly singular, so
       the solution and error bounds could not be computed. RCOND = 0
       is returned.
   = N+J: The solution corresponding to the Jth right-hand side is
       not guaranteed. The solutions corresponding to other right-
       hand sides K with K > J may not be guaranteed as well, but
       only the first such right-hand side is reported. If a small
       componentwise error is not requested (PARAMS(3) = 0.0) then
       the Jth right-hand side is the first with a normwise error
       bound that is not guaranteed (the smallest J such
       that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
       the Jth right-hand side is the first with either a normwise or
       componentwise error bound that is not guaranteed (the smallest
       J such that either ERR_BNDS_NORM(J,1) = 0.0 or
       ERR_BNDS_COMP(J,1) = 0.0). See the definition of
       ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
       about all of the right-hand sides check ERR_BNDS_NORM or
       ERR_BNDS_COMP.
```

———-

47.6.3 fortran code

— **zgbrfsx.f** —

```
*  ======================================================================
      SUBROUTINE ZGBRFSX( TRANS, EQUED, N, KL, KU, NRHS, AB, LDAB, AFB,
     $                    LDAFB, IPIV, R, C, B, LDB, X, LDX, RCOND,
     $                    BERR, N_ERR_BNDS, ERR_BNDS_NORM,
     $                    ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, RWORK,
     $                    INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS, EQUED
      INTEGER            INFO, LDAB, LDAFB, LDB, LDX, N, KL, KU, NRHS,
     $                   NPARAMS, N_ERR_BNDS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
```

```
      INTEGER            IPIV( * )
      COMPLEX*16         AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   X( LDX , * ),WORK( * )
      DOUBLE PRECISION   R( * ), C( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * ), RWORK( * )
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   ITREF_DEFAULT, ITHRESH_DEFAULT
      DOUBLE PRECISION   COMPONENTWISE_DEFAULT, RTHRESH_DEFAULT
      DOUBLE PRECISION   DZTHRESH_DEFAULT
      PARAMETER          ( ITREF_DEFAULT = 1.0D+0 )
      PARAMETER          ( ITHRESH_DEFAULT = 10.0D+0 )
      PARAMETER          ( COMPONENTWISE_DEFAULT = 1.0D+0 )
      PARAMETER          ( RTHRESH_DEFAULT = 0.5D+0 )
      PARAMETER          ( DZTHRESH_DEFAULT = 0.25D+0 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. Local Scalars ..
      CHARACTER(1)       NORM
      LOGICAL            ROWEQU, COLEQU, NOTRAN, IGNORE_CWISE
      INTEGER            J, TRANS_TYPE, PREC_TYPE, REF_TYPE, N_NORMS,
     $                   ITHRESH
      DOUBLE PRECISION   ANORM, RCOND_TMP, ILLRCOND_THRESH, ERR_LBND,
     $                   CWISE_WRONG, RTHRESH, UNSTABLE_THRESH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGBCON, ZLA_GBRFSX_EXTENDED
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT, TRANSFER
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, BLAS_FPINFO_X, ILATRANS, ILAPREC
      EXTERNAL           DLAMCH, ZLANGB, ZLA_GBRCOND_X, ZLA_GBRCOND_C
      DOUBLE PRECISION   DLAMCH, ZLANGB, ZLA_GBRCOND_X, ZLA_GBRCOND_C
      LOGICAL            LSAME
```

```
      INTEGER            BLAS_FPINFO_X
      INTEGER            ILATRANS, ILAPREC
*     ..
*     .. Executable Statements ..
*
*     Check the input parameters.
*
      INFO = 0
      TRANS_TYPE = ILATRANS( TRANS )
      REF_TYPE = INT( ITREF_DEFAULT )
      IF ( NPARAMS .GE. LA_LINRX_ITREF_I ) THEN
         IF ( PARAMS( LA_LINRX_ITREF_I ) .LT. 0.0D+0 ) THEN
            PARAMS( LA_LINRX_ITREF_I ) = ITREF_DEFAULT
         ELSE
            REF_TYPE = PARAMS( LA_LINRX_ITREF_I )
         END IF
      END IF
*
*     Set default parameters.
*
      ILLRCOND_THRESH = DBLE( N ) * DLAMCH( 'Epsilon' )
      ITHRESH = INT( ITHRESH_DEFAULT )
      RTHRESH = RTHRESH_DEFAULT
      UNSTABLE_THRESH = DZTHRESH_DEFAULT
      IGNORE_CWISE = COMPONENTWISE_DEFAULT .EQ. 0.0D+0
*
      IF ( NPARAMS.GE.LA_LINRX_ITHRESH_I ) THEN
         IF ( PARAMS( LA_LINRX_ITHRESH_I ).LT.0.0D+0 ) THEN
            PARAMS( LA_LINRX_ITHRESH_I ) = ITHRESH
         ELSE
            ITHRESH = INT( PARAMS( LA_LINRX_ITHRESH_I ) )
         END IF
      END IF
      IF ( NPARAMS.GE.LA_LINRX_CWISE_I ) THEN
         IF ( PARAMS( LA_LINRX_CWISE_I ).LT.0.0D+0 ) THEN
            IF ( IGNORE_CWISE ) THEN
               PARAMS( LA_LINRX_CWISE_I ) = 0.0D+0
            ELSE
               PARAMS( LA_LINRX_CWISE_I ) = 1.0D+0
            END IF
         ELSE
            IGNORE_CWISE = PARAMS( LA_LINRX_CWISE_I ) .EQ. 0.0D+0
         END IF
      END IF
      IF ( REF_TYPE .EQ. 0 .OR. N_ERR_BNDS .EQ. 0 ) THEN
         N_NORMS = 0
      ELSE IF ( IGNORE_CWISE ) THEN
         N_NORMS = 1
      ELSE
         N_NORMS = 2
```

```
      END IF
*
      NOTRAN = LSAME( TRANS, 'N' )
      ROWEQU = LSAME( EQUED, 'R' ) .OR. LSAME( EQUED, 'B' )
      COLEQU = LSAME( EQUED, 'C' ) .OR. LSAME( EQUED, 'B' )
*
*     Test input parameters.
*
      IF( TRANS_TYPE.EQ.-1 ) THEN
        INFO = -1
      ELSE IF( .NOT.ROWEQU .AND. .NOT.COLEQU .AND.
     $         .NOT.LSAME( EQUED, 'N' ) ) THEN
        INFO = -2
      ELSE IF( N.LT.0 ) THEN
        INFO = -3
      ELSE IF( KL.LT.0 ) THEN
        INFO = -4
      ELSE IF( KU.LT.0 ) THEN
        INFO = -5
      ELSE IF( NRHS.LT.0 ) THEN
        INFO = -6
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
        INFO = -8
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
        INFO = -10
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
        INFO = -13
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
        INFO = -15
      END IF
      IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'ZGBRFSX', -INFO )
        RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         RCOND = 1.0D+0
         DO J = 1, NRHS
            BERR( J ) = 0.0D+0
            IF ( N_ERR_BNDS .GE. 1 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
            IF ( N_ERR_BNDS .GE. 2 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 0.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 0.0D+0
            END IF
            IF ( N_ERR_BNDS .GE. 3 ) THEN
```

```
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 1.0D+0
            END IF
         END DO
         RETURN
      END IF
*
*     Default to failure.
*
      RCOND = 0.0D+0
      DO J = 1, NRHS
         BERR( J ) = 1.0D+0
         IF ( N_ERR_BNDS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
         END IF
         IF ( N_ERR_BNDS .GE. 2 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
         END IF
         IF ( N_ERR_BNDS .GE. 3 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 0.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 0.0D+0
         END IF
      END DO
*
*     Compute the norm of A and the reciprocal of the condition
*     number of A.
*
      IF( NOTRAN ) THEN
         NORM = 'I'
      ELSE
         NORM = '1'
      END IF
      ANORM = ZLANGB( NORM, N, KL, KU, AB, LDAB, RWORK )
      CALL ZGBCON( NORM, N, KL, KU, AFB, LDAFB, IPIV, ANORM, RCOND,
     $     WORK, RWORK, INFO )
*
*     Perform refinement on each right-hand side
*
      IF ( REF_TYPE .NE. 0 ) THEN

         PREC_TYPE = ILAPREC( 'E' )

         IF ( NOTRAN ) THEN
            CALL ZLA_GBRFSX_EXTENDED( PREC_TYPE, TRANS_TYPE,  N, KL, KU,
     $           NRHS, AB, LDAB, AFB, LDAFB, IPIV, COLEQU, C, B,
     $           LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM,
     $           ERR_BNDS_COMP, WORK, RWORK, WORK(N+1),
     $           TRANSFER (RWORK(1:2*N), (/ (ZERO, ZERO) /), N),
```

```
     $           RCOND, ITHRESH, RTHRESH, UNSTABLE_THRESH, IGNORE_CWISE,
     $           INFO )
         ELSE
            CALL ZLA_GBRFSX_EXTENDED( PREC_TYPE, TRANS_TYPE,  N, KL, KU,
     $           NRHS, AB, LDAB, AFB, LDAFB, IPIV, ROWEQU, R, B,
     $           LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM,
     $           ERR_BNDS_COMP, WORK, RWORK, WORK(N+1),
     $           TRANSFER (RWORK(1:2*N), (/ (ZERO, ZERO) /), N),
     $           RCOND, ITHRESH, RTHRESH, UNSTABLE_THRESH, IGNORE_CWISE,
     $           INFO )
         END IF
      END IF

      ERR_LBND = MAX( 10.0D+0, SQRT( DBLE( N ) ) ) * DLAMCH( 'Epsilon' )
      IF (N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 1) THEN
*
*     Compute scaled normwise condition number cond(A*C).
*
         IF ( COLEQU .AND. NOTRAN ) THEN
            RCOND_TMP = ZLA_GBRCOND_C( TRANS, N, KL, KU, AB, LDAB, AFB,
     $           LDAFB, IPIV, C, .TRUE., INFO, WORK, RWORK )
         ELSE IF ( ROWEQU .AND. .NOT. NOTRAN ) THEN
            RCOND_TMP = ZLA_GBRCOND_C( TRANS, N, KL, KU, AB, LDAB, AFB,
     $           LDAFB, IPIV, R, .TRUE., INFO, WORK, RWORK )
         ELSE
            RCOND_TMP = ZLA_GBRCOND_C( TRANS, N, KL, KU, AB, LDAB, AFB,
     $           LDAFB, IPIV, C, .FALSE., INFO, WORK, RWORK )
         END IF
         DO J = 1, NRHS
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .GT. 1.0D+0)
     $           ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 0.0D+0
               IF ( INFO .LE. N ) INFO = N + J
            ELSE IF ( ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .LT. ERR_LBND )
     $              THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
*
*     Save the condition number.
*
```

```
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF

         END DO
      END IF

      IF (N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 2) THEN
*
*     Compute componentwise condition number cond(A*diag(Y(:,J))) for
*     each right-hand side using the current solution as an estimate of
*     the true solution.  If the componentwise error estimate is too
*     large, then the solution is a lousy estimate of truth and the
*     estimated RCOND may be too optimistic.  To avoid misleading users,
*     the inverse condition number is set to 0.0 when the estimated
*     cwise error is at least CWISE_WRONG.
*
         CWISE_WRONG = SQRT( DLAMCH( 'Epsilon' ) )
         DO J = 1, NRHS
            IF (ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .LT. CWISE_WRONG )
     $     THEN
               RCOND_TMP = ZLA_GBRCOND_X( TRANS, N, KL, KU, AB, LDAB,
     $              AFB, LDAFB, IPIV, X( 1, J ), INFO, WORK, RWORK )
            ELSE
               RCOND_TMP = 0.0D+0
            END IF
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .GT. 1.0D+0 )
     $           ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 0.0D+0
               IF ( PARAMS( LA_LINRX_CWISE_I ) .EQ. 1.0D+0
     $              .AND. INFO.LT.N + J ) INFO = N + J
            ELSE IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I )
     $              .LT. ERR_LBND ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = RCOND_TMP
```

```
            END IF

         END DO
      END IF
*
      RETURN
*
*     End of ZGBRFSX
*
      END
```

———-

47.6.4 lisp code

— **zgbrfsx.lisp** —

———-

47.7 zgbtf2

47.7.1 Axiom unit tests

— **zgbtf2.input** —

```
)set break resume
)sys rm -f zgbtf2.output
)spool zgbtf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

47.7.2 Axiom help page

— **zgbtf2.help** —

```
====================================================================
zgbtf2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgbtf2 (M, N, KL, KU, AB, LDAB, IPIV, INFO)
           ZGBTF2 computes the LU factorization of a general band matrix using
           the unblocked version of the algorithm.

Function/Subroutine Documentation
   subroutine zgbtf2 (integer M, integer N, integer KL, integer KU,
       complex*16, dimension( ldab, * ) AB, integer LDAB, integer, dimension(
       * ) IPIV, integer INFO)
       ZGBTF2 computes the LU factorization of a general band matrix using the
       unblocked version of the algorithm.

Purpose:
   ZGBTF2 computes an LU factorization of a complex m-by-n band matrix
   A using partial pivoting with row interchanges.

   This is the unblocked version of the algorithm, calling Level 2 BLAS.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 AB
   AB is COMPLEX*16 array, dimension (LDAB,N)
   On entry, the matrix A in band storage, in rows KL+1 to
   2*KL+KU+1; rows 1 to KL of the array need not be set.
   The j-th column of A is stored in the j-th column of the
```

```
  array AB as follows:
  AB(kl+ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl)

  On exit, details of the factorization: U is stored as an
  upper triangular band matrix with KL+KU superdiagonals in
  rows 1 to KL+KU+1, and the multipliers used during the
  factorization are stored in rows KL+KU+2 to 2*KL+KU+1.
  See below for further details.

 LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.

 IPIV
  IPIV is INTEGER array, dimension (min(M,N))
  The pivot indices; for 1 <= i <= min(M,N), row i of the
  matrix was interchanged with row IPIV(i).

 INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = +i, U(i,i) is exactly zero. The factorization
     has been completed, but the factor U is exactly
     singular, and division by zero will occur if it is used
     to solve a system of equations.

Further Details:

  The band storage scheme is illustrated by the following example, when
  M = N = 6, KL = 2, KU = 1:

       On entry:                       On exit:

           *    *    *    +    +    +       *    *    *   u14  u25  u36
           *    *    +    +    +    +       *    *   u13  u24  u35  u46
           *   a12  a23  a34  a45  a56       *   u12  u23  u34  u45  u56
          a11  a22  a33  a44  a55  a66     u11  u22  u33  u44  u55  u66
          a21  a32  a43  a54  a65   *      m21  m32  m43  m54  m65   *
          a31  a42  a53  a64   *    *      m31  m42  m53  m64   *    *

  Array elements marked * are not used by the routine; elements marked
  + need not be set on entry, but are required by the routine to store
  elements of U, because of fill-in resulting from the row interchanges.
```

47.7.3 fortran code

— zgbtf2.f —

```
*  =====================================================================
      SUBROUTINE ZGBTF2( M, N, KL, KU, AB, LDAB, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, KL, KU, LDAB, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         AB( LDAB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE, ZERO
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ),
     $                   ZERO = ( 0.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, JP, JU, KM, KV
*     ..
*     .. External Functions ..
      INTEGER            IZAMAX
      EXTERNAL           IZAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGERU, ZSCAL, ZSWAP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     KV is the number of superdiagonals in the factor U, allowing for
*     fill-in.
*
      KV = KU + KL
*
*     Test the input parameters.
*
```

```
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KV+1 ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGBTF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Gaussian elimination with partial pivoting
*
*     Set fill-in elements in columns KU+2 to KV to zero.
*
      DO 20 J = KU + 2, MIN( KV, N )
         DO 10 I = KV - J + 2, KL
            AB( I, J ) = ZERO
   10    CONTINUE
   20 CONTINUE
*
*     JU is the index of the last column affected by the current stage
*     of the factorization.
*
      JU = 1
*
      DO 40 J = 1, MIN( M, N )
*
*        Set fill-in elements in column J+KV to zero.
*
         IF( J+KV.LE.N ) THEN
            DO 30 I = 1, KL
               AB( I, J+KV ) = ZERO
   30       CONTINUE
         END IF
*
*        Find pivot and test for singularity. KM is the number of
*        subdiagonal elements in the current column.
*
```

```
         KM = MIN( KL, M-J )
         JP = IZAMAX( KM+1, AB( KV+1, J ), 1 )
         IPIV( J ) = JP + J - 1
         IF( AB( KV+JP, J ).NE.ZERO ) THEN
            JU = MAX( JU, MIN( J+KU+JP-1, N ) )
*
*           Apply interchange to columns J to JU.
*
            IF( JP.NE.1 )
     $         CALL ZSWAP( JU-J+1, AB( KV+JP, J ), LDAB-1,
     $                     AB( KV+1, J ), LDAB-1 )
            IF( KM.GT.0 ) THEN
*
*              Compute multipliers.
*
               CALL ZSCAL( KM, ONE / AB( KV+1, J ), AB( KV+2, J ), 1 )
*
*              Update trailing submatrix within the band.
*
               IF( JU.GT.J )
     $            CALL ZGERU( KM, JU-J, -ONE, AB( KV+2, J ), 1,
     $                        AB( KV, J+1 ), LDAB-1, AB( KV+1, J+1 ),
     $                        LDAB-1 )
            END IF
         ELSE
*
*           If pivot is zero, set INFO to the index of the pivot
*           unless a zero pivot has already been found.
*
            IF( INFO.EQ.0 )
     $         INFO = J
         END IF
   40 CONTINUE
      RETURN
*
*     End of ZGBTF2
*
      END
```

———-

47.7.4 lisp code

— **zgbtf2.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)) (zero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (type (f2cl-lib:complex16) zero)
```

```
          (ignorable one zero))
 (defun zgbtf2 (m n kl ku ab ldab ipiv info)
   (declare (type (array f2cl-lib:integer4 (*)) ipiv)
            (type (array f2cl-lib:complex16 (*)) ab)
            (type (f2cl-lib:integer4) info ldab ku kl n m))
   (f2cl-lib:with-multi-array-data
       ((ab f2cl-lib:complex16 ab-%data% ab-%offset%)
        (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
     (prog ((i 0) (j 0) (jp 0) (ju 0) (km 0) (kv 0))
       (declare (type (f2cl-lib:integer4) i j jp ju km kv))
       (setf kv (f2cl-lib:int-add ku kl))
       (setf info 0)
       (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
             ((< kl 0) (setf info -3)) ((< ku 0) (setf info -4))
             ((< ldab (f2cl-lib:int-add kl kv 1)) (setf info -6)))
       (cond
        ((/= info 0) (xerbla "ZGBTF2" (f2cl-lib:int-sub info))
         (go end_label)))
       (if (or (= m 0) (= n 0))
           (go end_label))
       (f2cl-lib:fdo (j (f2cl-lib:int-add ku 2) (f2cl-lib:int-add j 1))
                     ((> j
                         (min (the f2cl-lib:integer4 kv)
                              (the f2cl-lib:integer4 n)))
                      nil)
         (tagbody
           (f2cl-lib:fdo (i (f2cl-lib:int-add kv (f2cl-lib:int-sub j) 2)
                            (f2cl-lib:int-add i 1))
                         ((> i kl) nil)
             (tagbody
               (setf (f2cl-lib:fref ab-%data% (i j) ((1 ldab) (1 *))
                                    ab-%offset%)
                     zero)
              label10))
          label20))
       (setf ju 1)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j
                         (min (the f2cl-lib:integer4 m)
                              (the f2cl-lib:integer4 n)))
                      nil)
         (tagbody
           (cond
            ((<= (f2cl-lib:int-add j kv) n)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i kl) nil)
               (tagbody
                 (setf (f2cl-lib:fref ab-%data% (i (f2cl-lib:int-add j kv))
                                      ((1 ldab) (1 *)) ab-%offset%)
                       zero)
```

```
     label30))))
(setf km
        (min (the f2cl-lib:integer4 kl)
             (the f2cl-lib:integer4 (f2cl-lib:int-sub m j))))
(setf jp
        (izamax (f2cl-lib:int-add km 1)
         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                               ((+ kv 1) j) ((1 ldab) (1 *))
                               ab-%offset%)
         1))
(setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
        (f2cl-lib:int-sub (f2cl-lib:int-add jp j) 1))
(cond
 ((/=
   (f2cl-lib:fref ab ((f2cl-lib:int-add kv jp) j) ((1 ldab) (1 *)))
   zero)
  (setf ju
          (max (the f2cl-lib:integer4 ju)
               (the f2cl-lib:integer4
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-sub
                           (f2cl-lib:int-add j ku jp) 1))
                     (the f2cl-lib:integer4 n)))))
  (if (/= jp 1)
      (zswap (f2cl-lib:int-add (f2cl-lib:int-sub ju j) 1)
       (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                             ((+ kv jp) j) ((1 ldab) (1 *))
                             ab-%offset%)
       (f2cl-lib:int-sub ldab 1)
       (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                             ((+ kv 1) j) ((1 ldab) (1 *))
                             ab-%offset%)
       (f2cl-lib:int-sub ldab 1)))
  (cond
   ((> km 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zscal km
         (/ one
            (f2cl-lib:fref ab-%data% ((f2cl-lib:int-add kv 1) j)
                           ((1 ldab) (1 *)) ab-%offset%))
         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                               ((+ kv 2) j) ((1 ldab) (1 *))
                               ab-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf km var-0)))
    (if (> ju j)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
```

```
                        (zgeru km (f2cl-lib:int-sub ju j) (- one)
                         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                               ((+ kv 2) j) ((1 ldab) (1 *))
                                               ab-%offset%)
                         1
                         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                               (kv (f2cl-lib:int-add j 1))
                                               ((1 ldab) (1 *)) ab-%offset%)
                         (f2cl-lib:int-sub ldab 1)
                         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                               ((+ kv 1)
                                                (f2cl-lib:int-add j 1))
                                               ((1 ldab) (1 *)) ab-%offset%)
                         (f2cl-lib:int-sub ldab 1))
                       (declare
                        (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7
                         var-8))
                       (when var-0 (setf km var-0)))))))
             (t
              (if (= info 0)
                  (setf info j))))
           label40))
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgbtf2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

47.8 zgbtrf

47.8.1 Axiom unit tests

— zgbtrf.input —

```
)set break resume
)sys rm -f zgbtrf.output
)spool zgbtrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

47.8.2 Axiom help page

— zgbtrf.help —

```
======================================================================
zgbtrf examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgbtrf (M, N, KL, KU, AB, LDAB, IPIV, INFO)
           ZGBTRF

Function/Subroutine Documentation
   subroutine zgbtrf (integer M, integer N, integer KL, integer KU,
       complex*16, dimension( ldab, * ) AB, integer LDAB, integer, dimension(
       * ) IPIV, integer INFO)
       ZGBTRF

Purpose:
   ZGBTRF computes an LU factorization of a complex m-by-n band matrix A
   using partial pivoting with row interchanges.

   This is the blocked version of the algorithm, calling Level 3 BLAS.
```

```
Parameters:

M
  M is INTEGER
  The number of rows of the matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

KL
  KL is INTEGER
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

AB
  AB is COMPLEX*16 array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows KL+1 to
  2*KL+KU+1; rows 1 to KL of the array need not be set.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(kl+ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl)

  On exit, details of the factorization: U is stored as an
  upper triangular band matrix with KL+KU superdiagonals in
  rows 1 to KL+KU+1, and the multipliers used during the
  factorization are stored in rows KL+KU+2 to 2*KL+KU+1.
  See below for further details.

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (min(M,N))
  The pivot indices; for 1 <= i <= min(M,N), row i of the
  matrix was interchanged with row IPIV(i).

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = +i, U(i,i) is exactly zero. The factorization
     has been completed, but the factor U is exactly
     singular, and division by zero will occur if it is used
     to solve a system of equations.
```

```
Further Details:

   The band storage scheme is illustrated by the following example, when
   M = N = 6, KL = 2, KU = 1:

        On entry:                       On exit:

            *    *    *    +    +    +       *    *    *   u14  u25  u36
            *    *    +    +    +    +       *    *   u13  u24  u35  u46
            *   a12  a23  a34  a45  a56      *   u12  u23  u34  u45  u56
           a11  a22  a33  a44  a55  a66     u11  u22  u33  u44  u55  u66
           a21  a32  a43  a54  a65   *      m21  m32  m43  m54  m65   *
           a31  a42  a53  a64   *    *      m31  m42  m53  m64   *    *

   Array elements marked * are not used by the routine; elements marked
   + need not be set on entry, but are required by the routine to store
   elements of U because of fill-in resulting from the row interchanges.
```

47.8.3 fortran code

— zgbtrf.f —

```
*  =====================================================================
      SUBROUTINE ZGBTRF( M, N, KL, KU, AB, LDAB, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, KL, KU, LDAB, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         AB( LDAB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE, ZERO
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ),
     $                   ZERO = ( 0.0D+0, 0.0D+0 ) )
```

```
      INTEGER            NBMAX, LDWORK
      PARAMETER          ( NBMAX = 64, LDWORK = NBMAX+1 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, I2, I3, II, IP, J, J2, J3, JB, JJ, JM, JP,
     $                   JU, K2, KM, KV, NB, NW
      COMPLEX*16         TEMP
*     ..
*     .. Local Arrays ..
      COMPLEX*16         WORK13( LDWORK, NBMAX ),
     $                   WORK31( LDWORK, NBMAX )
*     ..
*     .. External Functions ..
      INTEGER            ILAENV, IZAMAX
      EXTERNAL           ILAENV, IZAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZCOPY, ZGBTF2, ZGEMM, ZGERU, ZLASWP,
     $                   ZSCAL, ZSWAP, ZTRSM
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     KV is the number of superdiagonals in the factor U, allowing for
*     fill-in
*
      KV = KU + KL
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KV+1 ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGBTRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
```

```
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Determine the block size for this environment
*
      NB = ILAENV( 1, 'ZGBTRF', ' ', M, N, KL, KU )
*
*     The block size must not exceed the limit set by the size of the
*     local arrays WORK13 and WORK31.
*
      NB = MIN( NB, NBMAX )
*
      IF( NB.LE.1 .OR. NB.GT.KL ) THEN
*
*        Use unblocked code
*
         CALL ZGBTF2( M, N, KL, KU, AB, LDAB, IPIV, INFO )
      ELSE
*
*        Use blocked code
*
*        Zero the superdiagonal elements of the work array WORK13
*
         DO 20 J = 1, NB
            DO 10 I = 1, J - 1
               WORK13( I, J ) = ZERO
   10       CONTINUE
   20    CONTINUE
*
*        Zero the subdiagonal elements of the work array WORK31
*
         DO 40 J = 1, NB
            DO 30 I = J + 1, NB
               WORK31( I, J ) = ZERO
   30       CONTINUE
   40    CONTINUE
*
*        Gaussian elimination with partial pivoting
*
*        Set fill-in elements in columns KU+2 to KV to zero
*
         DO 60 J = KU + 2, MIN( KV, N )
            DO 50 I = KV - J + 2, KL
               AB( I, J ) = ZERO
   50       CONTINUE
   60    CONTINUE
*
*        JU is the index of the last column affected by the current
*        stage of the factorization
```

```
*
         JU = 1
*
         DO 180 J = 1, MIN( M, N ), NB
            JB = MIN( NB, MIN( M, N )-J+1 )
*
*           The active part of the matrix is partitioned
*
*              A11   A12   A13
*              A21   A22   A23
*              A31   A32   A33
*
*           Here A11, A21 and A31 denote the current block of JB columns
*           which is about to be factorized. The number of rows in the
*           partitioning are JB, I2, I3 respectively, and the numbers
*           of columns are JB, J2, J3. The superdiagonal elements of A13
*           and the subdiagonal elements of A31 lie outside the band.
*
            I2 = MIN( KL-JB, M-J-JB+1 )
            I3 = MIN( JB, M-J-KL+1 )
*
*           J2 and J3 are computed after JU has been updated.
*
*           Factorize the current block of JB columns
*
            DO 80 JJ = J, J + JB - 1
*
*              Set fill-in elements in column JJ+KV to zero
*
               IF( JJ+KV.LE.N ) THEN
                  DO 70 I = 1, KL
                     AB( I, JJ+KV ) = ZERO
   70             CONTINUE
               END IF
*
*              Find pivot and test for singularity. KM is the number of
*              subdiagonal elements in the current column.
*
               KM = MIN( KL, M-JJ )
               JP = IZAMAX( KM+1, AB( KV+1, JJ ), 1 )
               IPIV( JJ ) = JP + JJ - J
               IF( AB( KV+JP, JJ ).NE.ZERO ) THEN
                  JU = MAX( JU, MIN( JJ+KU+JP-1, N ) )
                  IF( JP.NE.1 ) THEN
*
*                    Apply interchange to columns J to J+JB-1
*
                     IF( JP+JJ-1.LT.J+KL ) THEN
*
                        CALL ZSWAP( JB, AB( KV+1+JJ-J, J ), LDAB-1,
```

```
     $                             AB( KV+JP+JJ-J, J ), LDAB-1 )
                     ELSE
*
*                       The interchange affects columns J to JJ-1 of A31
*                       which are stored in the work array WORK31
*
                        CALL ZSWAP( JJ-J, AB( KV+1+JJ-J, J ), LDAB-1,
     $                              WORK31( JP+JJ-J-KL, 1 ), LDWORK )
                        CALL ZSWAP( J+JB-JJ, AB( KV+1, JJ ), LDAB-1,
     $                              AB( KV+JP, JJ ), LDAB-1 )
                     END IF
                  END IF
*
*                 Compute multipliers
*
                  CALL ZSCAL( KM, ONE / AB( KV+1, JJ ), AB( KV+2, JJ ),
     $                        1 )
*
*                 Update trailing submatrix within the band and within
*                 the current block. JM is the index of the last column
*                 which needs to be updated.
*
                  JM = MIN( JU, J+JB-1 )
                  IF( JM.GT.JJ )
     $               CALL ZGERU( KM, JM-JJ, -ONE, AB( KV+2, JJ ), 1,
     $                           AB( KV, JJ+1 ), LDAB-1,
     $                           AB( KV+1, JJ+1 ), LDAB-1 )
               ELSE
*
*                 If pivot is zero, set INFO to the index of the pivot
*                 unless a zero pivot has already been found.
*
                  IF( INFO.EQ.0 )
     $               INFO = JJ
               END IF
*
*              Copy current column of A31 into the work array WORK31
*
               NW = MIN( JJ-J+1, I3 )
               IF( NW.GT.0 )
     $            CALL ZCOPY( NW, AB( KV+KL+1-JJ+J, JJ ), 1,
     $                        WORK31( 1, JJ-J+1 ), 1 )
   80       CONTINUE
            IF( J+JB.LE.N ) THEN
*
*              Apply the row interchanges to the other blocks.
*
               J2 = MIN( JU-J+1, KV ) - JB
               J3 = MAX( 0, JU-J-KV+1 )
*
```

```
*              Use ZLASWP to apply the row interchanges to A12, A22, and
*              A32.
*
               CALL ZLASWP( J2, AB( KV+1-JB, J+JB ), LDAB-1, 1, JB,
     $                      IPIV( J ), 1 )
*
*              Adjust the pivot indices.
*
               DO 90 I = J, J + JB - 1
                  IPIV( I ) = IPIV( I ) + J - 1
   90          CONTINUE
*
*              Apply the row interchanges to A13, A23, and A33
*              columnwise.
*
               K2 = J - 1 + JB + J2
               DO 110 I = 1, J3
                  JJ = K2 + I
                  DO 100 II = J + I - 1, J + JB - 1
                     IP = IPIV( II )
                     IF( IP.NE.II ) THEN
                        TEMP = AB( KV+1+II-JJ, JJ )
                        AB( KV+1+II-JJ, JJ ) = AB( KV+1+IP-JJ, JJ )
                        AB( KV+1+IP-JJ, JJ ) = TEMP
                     END IF
  100             CONTINUE
  110          CONTINUE
*
*              Update the relevant part of the trailing submatrix
*
               IF( J2.GT.0 ) THEN
*
*                 Update A12
*
                  CALL ZTRSM( 'Left', 'Lower', 'No transpose', 'Unit',
     $                        JB, J2, ONE, AB( KV+1, J ), LDAB-1,
     $                        AB( KV+1-JB, J+JB ), LDAB-1 )
*
                  IF( I2.GT.0 ) THEN
*
*                    Update A22
*
                     CALL ZGEMM( 'No transpose', 'No transpose', I2, J2,
     $                           JB, -ONE, AB( KV+1+JB, J ), LDAB-1,
     $                           AB( KV+1-JB, J+JB ), LDAB-1, ONE,
     $                           AB( KV+1, J+JB ), LDAB-1 )
                  END IF
*
                  IF( I3.GT.0 ) THEN
*
```

```
*                    Update A32
*
                     CALL ZGEMM( 'No transpose', 'No transpose', I3, J2,
     $                           JB, -ONE, WORK31, LDWORK,
     $                           AB( KV+1-JB, J+JB ), LDAB-1, ONE,
     $                           AB( KV+KL+1-JB, J+JB ), LDAB-1 )
                  END IF
               END IF
*
               IF( J3.GT.0 ) THEN
*
*                 Copy the lower triangle of A13 into the work array
*                 WORK13
*
                  DO 130 JJ = 1, J3
                     DO 120 II = JJ, JB
                        WORK13( II, JJ ) = AB( II-JJ+1, JJ+J+KV-1 )
  120                CONTINUE
  130             CONTINUE
*
*                 Update A13 in the work array
*
                  CALL ZTRSM( 'Left', 'Lower', 'No transpose', 'Unit',
     $                        JB, J3, ONE, AB( KV+1, J ), LDAB-1,
     $                        WORK13, LDWORK )
*
                  IF( I2.GT.0 ) THEN
*
*                    Update A23
*
                     CALL ZGEMM( 'No transpose', 'No transpose', I2, J3,
     $                           JB, -ONE, AB( KV+1+JB, J ), LDAB-1,
     $                           WORK13, LDWORK, ONE, AB( 1+JB, J+KV ),
     $                           LDAB-1 )
                  END IF
*
                  IF( I3.GT.0 ) THEN
*
*                    Update A33
*
                     CALL ZGEMM( 'No transpose', 'No transpose', I3, J3,
     $                           JB, -ONE, WORK31, LDWORK, WORK13,
     $                           LDWORK, ONE, AB( 1+KL, J+KV ), LDAB-1 )
                  END IF
*
*                 Copy the lower triangle of A13 back into place
*
                  DO 150 JJ = 1, J3
                     DO 140 II = JJ, JB
                        AB( II-JJ+1, JJ+J+KV-1 ) = WORK13( II, JJ )
```

```
  140               CONTINUE
  150            CONTINUE
              END IF
           ELSE
*
*             Adjust the pivot indices.
*
              DO 160 I = J, J + JB - 1
                 IPIV( I ) = IPIV( I ) + J - 1
  160         CONTINUE
           END IF
*
*          Partially undo the interchanges in the current block to
*          restore the upper triangular form of A31 and copy the upper
*          triangle of A31 back into place
*
           DO 170 JJ = J + JB - 1, J, -1
              JP = IPIV( JJ ) - JJ + 1
              IF( JP.NE.1 ) THEN
*
                 Apply interchange to columns J to JJ-1
*
                 IF( JP+JJ-1.LT.J+KL ) THEN
*
                    The interchange does not affect A31
*
                    CALL ZSWAP( JJ-J, AB( KV+1+JJ-J, J ), LDAB-1,
     $                          AB( KV+JP+JJ-J, J ), LDAB-1 )
                 ELSE
*
                    The interchange does affect A31
*
                    CALL ZSWAP( JJ-J, AB( KV+1+JJ-J, J ), LDAB-1,
     $                          WORK31( JP+JJ-J-KL, 1 ), LDWORK )
                 END IF
              END IF
*
              Copy the current column of A31 back into place
*
              NW = MIN( I3, JJ-J+1 )
              IF( NW.GT.0 )
     $           CALL ZCOPY( NW, WORK31( 1, JJ-J+1 ), 1,
     $                       AB( KV+KL+1-JJ+J, JJ ), 1 )
  170       CONTINUE
  180    CONTINUE
      END IF
*
      RETURN
*
*     End of ZGBTRF
```

```
*
      END
```

———-

47.8.4 lisp code

— **zgbtrf.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0))
       (zero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (nbmax 64)
       (ldwork (+ nbmax 1)))
  (declare (type (f2cl-lib:complex16) one)
           (type (f2cl-lib:complex16) zero)
           (type (f2cl-lib:integer4 64 64) nbmax)
           (type (f2cl-lib:integer4) ldwork)
           (ignorable one zero nbmax ldwork))
  (defun zgbtrf (m n kl ku ab ldab ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) ab)
             (type (f2cl-lib:integer4) info ldab ku kl n m))
    (f2cl-lib:with-multi-array-data
        ((ab f2cl-lib:complex16 ab-%data% ab-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((work13
              (make-array (the fixnum (reduce #'* (list ldwork nbmax)))
                          :element-type 'f2cl-lib:complex16))
             (work31
              (make-array (the fixnum (reduce #'* (list ldwork nbmax)))
                          :element-type 'f2cl-lib:complex16))
             (temp #C(0.0d0 0.0d0)) (i 0) (i2 0) (i3 0) (ii 0) (ip 0) (j 0)
             (j2 0) (j3 0) (jb 0) (jj 0) (jm 0) (jp 0) (ju 0) (k2 0) (km 0)
             (kv 0) (nb 0) (nw 0))
        (declare (type (array f2cl-lib:complex16 (*)) work13 work31)
                 (type (f2cl-lib:complex16) temp)
                 (type (f2cl-lib:integer4) i i2 i3 ii ip j j2 j3 jb jj jm jp ju
                  k2 km kv nb nw))
        (setf kv (f2cl-lib:int-add ku kl))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< kl 0) (setf info -3)) ((< ku 0) (setf info -4))
              ((< ldab (f2cl-lib:int-add kl kv 1)) (setf info -6)))
        (cond
         ((/= info 0) (xerbla "ZGBTRF" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (or (= m 0) (= n 0))
            (go end_label))
```

```
(setf nb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 1 "ZGBTRF" " " m n kl ku)
          (declare (ignore var-0 var-1 var-2))
          (setf m var-3)
          (setf n var-4)
          (setf kl var-5)
          (setf ku var-6)
          ret-val))
(setf nb
        (min (the f2cl-lib:integer4 nb) (the f2cl-lib:integer4 nbmax)))
(cond
 ((or (<= nb 1) (> nb kl))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zgbtf2 m n kl ku ab ldab ipiv info)
    (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6))
    (setf info var-7)))
 (t
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nb) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref work13 (i j) ((1 ldwork) (1 nbmax)))
                  zero)
         label10))
     label20))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nb) nil)
    (tagbody
      (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                    ((> i nb) nil)
        (tagbody
          (setf (f2cl-lib:fref work31 (i j) ((1 ldwork) (1 nbmax)))
                  zero)
         label30))
     label40))
  (f2cl-lib:fdo (j (f2cl-lib:int-add ku 2) (f2cl-lib:int-add j 1))
                ((> j
                    (min (the f2cl-lib:integer4 kv)
                         (the f2cl-lib:integer4 n)))
                 nil)
    (tagbody
      (f2cl-lib:fdo (i (f2cl-lib:int-add kv (f2cl-lib:int-sub j) 2)
                       (f2cl-lib:int-add i 1))
                    ((> i kl) nil)
```

```
      (tagbody
        (setf (f2cl-lib:fref ab-%data% (i j) ((1 ldab) (1 *))
                             ab-%offset%)
              zero)
       label50))
   label60))
(setf ju 1)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
              ((> j
                  (min (the f2cl-lib:integer4 m)
                       (the f2cl-lib:integer4 n)))
               nil)
  (tagbody
    (setf jb
            (min (the f2cl-lib:integer4 nb)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add
                       (f2cl-lib:int-sub
                        (min (the f2cl-lib:integer4 m)
                             (the f2cl-lib:integer4 n))
                        j)
                       1))))
    (setf i2
            (min (the f2cl-lib:integer4 (f2cl-lib:int-sub kl jb))
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-sub m j jb)
                                        1))))
    (setf i3
            (min (the f2cl-lib:integer4 jb)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-sub m j kl)
                                        1))))
    (f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
                  ((> jj
                      (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (cond
         ((<= (f2cl-lib:int-add jj kv) n)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i kl) nil)
            (tagbody
              (setf (f2cl-lib:fref ab-%data%
                                   (i (f2cl-lib:int-add jj kv))
                                   ((1 ldab) (1 *)) ab-%offset%)
                    zero)
             label70))))
        (setf km
                (min (the f2cl-lib:integer4 kl)
                     (the f2cl-lib:integer4
```

```
                  (f2cl-lib:int-sub m jj))))
(setf jp
        (izamax (f2cl-lib:int-add km 1)
         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                               ((+ kv 1) jj) ((1 ldab) (1 *))
                               ab-%offset%)
         1))
(setf (f2cl-lib:fref ipiv-%data% (jj) ((1 *)) ipiv-%offset%)
        (f2cl-lib:int-sub (f2cl-lib:int-add jp jj) j))
(cond
 ((/=
   (f2cl-lib:fref ab ((f2cl-lib:int-add kv jp) jj)
                  ((1 ldab) (1 *)))
   zero)
  (setf ju
          (max (the f2cl-lib:integer4 ju)
               (the f2cl-lib:integer4
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-sub
                           (f2cl-lib:int-add jj ku jp) 1))
                     (the f2cl-lib:integer4 n)))))
  (cond
   ((/= jp 1)
    (cond
     ((< (f2cl-lib:int-add jp jj (f2cl-lib:int-sub 1))
         (f2cl-lib:int-add j kl))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (zswap jb
           (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                 ((+ kv 1 jj
                                     (f2cl-lib:int-sub j))
                                  j)
                                 ((1 ldab) (1 *))
                                 ab-%offset%)
           (f2cl-lib:int-sub ldab 1)
           (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                 ((+ kv jp jj
                                     (f2cl-lib:int-sub j))
                                  j)
                                 ((1 ldab) (1 *))
                                 ab-%offset%)
           (f2cl-lib:int-sub ldab 1))
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf jb var-0))))
     (t
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (zswap (f2cl-lib:int-sub jj j)
           (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                 ((+ kv 1 jj
```

```
                                    (f2cl-lib:int-sub j))
                                 j)
                                ((1 ldab) (1 *))
                                ab-%offset%)
          (f2cl-lib:int-sub ldab 1)
          (f2cl-lib:array-slice work31 f2cl-lib:complex16
                                ((+ jp jj
                                    (f2cl-lib:int-sub j)
                                    (f2cl-lib:int-sub kl))
                                 1)
                                ((1 ldwork) (1 nbmax)))
          ldwork)
       (declare (ignore var-0 var-1 var-2 var-3))
       (when var-4 (setf ldwork var-4)))
    (zswap (f2cl-lib:int-sub (f2cl-lib:int-add j jb) jj)
     (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                           ((+ kv 1) jj) ((1 ldab) (1 *))
                           ab-%offset%)
     (f2cl-lib:int-sub ldab 1)
     (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                           ((+ kv jp) jj) ((1 ldab) (1 *))
                           ab-%offset%)
     (f2cl-lib:int-sub ldab 1))))))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (zscal km
     (/ one
        (f2cl-lib:fref ab-%data%
                       ((f2cl-lib:int-add kv 1) jj)
                       ((1 ldab) (1 *)) ab-%offset%))
     (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                           ((+ kv 2) jj) ((1 ldab) (1 *))
                           ab-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3))
  (when var-0 (setf km var-0)))
(setf jm
        (min (the f2cl-lib:integer4 ju)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-sub (f2cl-lib:int-add j jb)
                                    1))))
(if (> jm jj)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
         var-8)
        (zgeru km (f2cl-lib:int-sub jm jj) (- one)
         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                               ((+ kv 2) jj)
                               ((1 ldab) (1 *))
                               ab-%offset%)
         1
```

```
                (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                      (kv (f2cl-lib:int-add jj 1))
                                      ((1 ldab) (1 *))
                                      ab-%offset%)
                (f2cl-lib:int-sub ldab 1)
                (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                      ((+ kv 1)
                                       (f2cl-lib:int-add jj 1))
                                      ((1 ldab) (1 *))
                                      ab-%offset%)
                (f2cl-lib:int-sub ldab 1))
              (declare
               (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7
                var-8))
              (when var-0 (setf km var-0)))))
      (t
       (if (= info 0)
           (setf info jj))))
    (setf nw
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub jj j) 1))
             (the f2cl-lib:integer4 i3)))
    (if (> nw 0)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (zcopy nw
             (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                   ((+ kv kl 1
                                       (f2cl-lib:int-sub jj) j)
                                    jj)
                                   ((1 ldab) (1 *)) ab-%offset%)
             1
             (f2cl-lib:array-slice work31 f2cl-lib:complex16
                                   (1
                                    (f2cl-lib:int-add
                                     (f2cl-lib:int-sub jj j) 1))
                                   ((1 ldwork) (1 nbmax)))
             1)
           (declare (ignore var-1 var-2 var-3 var-4))
           (when var-0 (setf nw var-0))))
   label80))
(cond
 ((<= (f2cl-lib:int-add j jb) n)
  (setf j2
          (f2cl-lib:int-sub
           (min
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-sub ju j) 1))
            (the f2cl-lib:integer4 kv))
           jb))
```

```
(setf j3
        (max (the f2cl-lib:integer4 0)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub ju j kv)
                                    1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaswp j2
     (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                           ((+ kv 1 (f2cl-lib:int-sub jb))
                            (f2cl-lib:int-add j jb))
                           ((1 ldab) (1 *)) ab-%offset%)
     (f2cl-lib:int-sub ldab 1) 1 jb
     (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (j)
                           ((1 *)) ipiv-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-5 var-6))
  (when var-0 (setf j2 var-0))
  (when var-4 (setf jb var-4)))
(f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
              ((> i
                  (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
            (f2cl-lib:int-sub
             (f2cl-lib:int-add
              (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                             ipiv-%offset%)
              j)
             1))
   label90))
(setf k2 (f2cl-lib:int-add (f2cl-lib:int-sub j 1) jb j2))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i j3) nil)
  (tagbody
    (setf jj (f2cl-lib:int-add k2 i))
    (f2cl-lib:fdo (ii
                   (f2cl-lib:int-add j i (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add ii 1))
                  ((> ii
                      (f2cl-lib:int-add j jb
                                        (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf ip
                (f2cl-lib:fref ipiv-%data% (ii) ((1 *))
                               ipiv-%offset%))
        (cond
         ((/= ip ii)
```

```
                (setf temp
                        (f2cl-lib:fref ab-%data%
                                       ((f2cl-lib:int-sub
                                         (f2cl-lib:int-add kv 1 ii)
                                         jj)
                                        jj)
                                       ((1 ldab) (1 *)) ab-%offset%))
                (setf (f2cl-lib:fref ab-%data%
                                     ((f2cl-lib:int-sub
                                       (f2cl-lib:int-add kv 1 ii) jj)
                                      jj)
                                     ((1 ldab) (1 *)) ab-%offset%)
                        (f2cl-lib:fref ab-%data%
                                       ((f2cl-lib:int-sub
                                         (f2cl-lib:int-add kv 1 ip)
                                         jj)
                                        jj)
                                       ((1 ldab) (1 *)) ab-%offset%))
                (setf (f2cl-lib:fref ab-%data%
                                     ((f2cl-lib:int-sub
                                       (f2cl-lib:int-add kv 1 ip) jj)
                                      jj)
                                     ((1 ldab) (1 *)) ab-%offset%)
                        temp)))
            label100))
        label110))
    (cond
     ((> j2 0)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (ztrsm "Left" "Lower" "No transpose" "Unit" jb j2 one
           (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                 ((+ kv 1) j) ((1 ldab) (1 *))
                                 ab-%offset%)
           (f2cl-lib:int-sub ldab 1)
           (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                 ((+ kv 1 (f2cl-lib:int-sub jb))
                                  (f2cl-lib:int-add j jb))
                                 ((1 ldab) (1 *)) ab-%offset%)
           (f2cl-lib:int-sub ldab 1))
        (declare
         (ignore var-0 var-1 var-2 var-3 var-7 var-8 var-9 var-10))
        (when var-4 (setf jb var-4))
        (when var-5 (setf j2 var-5))
        (when var-6 (setf one var-6)))
      (cond
       ((> i2 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
                var-9 var-10 var-11 var-12)
               (zgemm "No transpose" "No transpose" i2 j2 jb (- one)
                (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                      ((+ kv 1 jb) j) ((1 ldab) (1 *))
                                      ab-%offset%)
                (f2cl-lib:int-sub ldab 1)
                (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                      ((+ kv 1 (f2cl-lib:int-sub jb))
                                       (f2cl-lib:int-add j jb))
                                      ((1 ldab) (1 *)) ab-%offset%)
                (f2cl-lib:int-sub ldab 1) one
                (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                      ((+ kv 1)
                                       (f2cl-lib:int-add j jb))
                                      ((1 ldab) (1 *)) ab-%offset%)
                (f2cl-lib:int-sub ldab 1))
              (declare
               (ignore var-0 var-1 var-5 var-6 var-7 var-8 var-9 var-11
                var-12))
              (when var-2 (setf i2 var-2))
              (when var-3 (setf j2 var-3))
              (when var-4 (setf jb var-4))
              (when var-10 (setf one var-10)))))
          (cond
           ((> i3 0)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10 var-11 var-12)
                (zgemm "No transpose" "No transpose" i3 j2 jb (- one)
                 work31 ldwork
                 (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                       ((+ kv 1 (f2cl-lib:int-sub jb))
                                        (f2cl-lib:int-add j jb))
                                       ((1 ldab) (1 *)) ab-%offset%)
                 (f2cl-lib:int-sub ldab 1) one
                 (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                       ((+ kv kl 1
                                           (f2cl-lib:int-sub jb))
                                        (f2cl-lib:int-add j jb))
                                       ((1 ldab) (1 *)) ab-%offset%)
                 (f2cl-lib:int-sub ldab 1))
              (declare
               (ignore var-0 var-1 var-5 var-6 var-8 var-9 var-11
                var-12))
              (when var-2 (setf i3 var-2))
              (when var-3 (setf j2 var-3))
              (when var-4 (setf jb var-4))
              (when var-7 (setf ldwork var-7))
              (when var-10 (setf one var-10)))))))
        (cond
```

```
((> j3 0)
 (f2cl-lib:fdo (jj 1 (f2cl-lib:int-add jj 1))
               ((> jj j3) nil)
   (tagbody
     (f2cl-lib:fdo (ii jj (f2cl-lib:int-add ii 1))
                   ((> ii jb) nil)
       (tagbody
         (setf (f2cl-lib:fref work13 (ii jj)
                              ((1 ldwork) (1 nbmax)))
                 (f2cl-lib:fref ab-%data%
                                ((f2cl-lib:int-add
                                  (f2cl-lib:int-sub ii jj) 1)
                                 (f2cl-lib:int-sub
                                  (f2cl-lib:int-add jj j kv)
                                  1))
                                ((1 ldab) (1 *)) ab-%offset%))
        label120))
    label130))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10)
     (ztrsm "Left" "Lower" "No transpose" "Unit" jb j3 one
      (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                            ((+ kv 1) j) ((1 ldab) (1 *))
                            ab-%offset%)
      (f2cl-lib:int-sub ldab 1) work13 ldwork)
   (declare
    (ignore var-0 var-1 var-2 var-3 var-7 var-8 var-9))
   (when var-4 (setf jb var-4))
   (when var-5 (setf j3 var-5))
   (when var-6 (setf one var-6))
   (when var-10 (setf ldwork var-10)))
 (cond
  ((> i2 0)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10 var-11 var-12)
       (zgemm "No transpose" "No transpose" i2 j3 jb (- one)
        (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                              ((+ kv 1 jb) j) ((1 ldab) (1 *))
                              ab-%offset%)
        (f2cl-lib:int-sub ldab 1) work13 ldwork one
        (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                              ((+ 1 jb)
                               (f2cl-lib:int-add j kv))
                              ((1 ldab) (1 *)) ab-%offset%)
        (f2cl-lib:int-sub ldab 1))
     (declare
      (ignore var-0 var-1 var-5 var-6 var-7 var-8 var-11
       var-12))
```

```
            (when var-2 (setf i2 var-2))
            (when var-3 (setf j3 var-3))
            (when var-4 (setf jb var-4))
            (when var-9 (setf ldwork var-9))
            (when var-10 (setf one var-10)))))
        (cond
         ((> i3 0)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10 var-11 var-12)
              (zgemm "No transpose" "No transpose" i3 j3 jb (- one)
               work31 ldwork work13 ldwork one
               (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                     ((+ 1 kl)
                                      (f2cl-lib:int-add j kv))
                                     ((1 ldab) (1 *)) ab-%offset%)
               (f2cl-lib:int-sub ldab 1))
            (declare
             (ignore var-0 var-1 var-5 var-6 var-8 var-11 var-12))
            (when var-2 (setf i3 var-2))
            (when var-3 (setf j3 var-3))
            (when var-4 (setf jb var-4))
            (when var-7 (setf ldwork var-7))
            (when var-9 (setf ldwork var-9))
            (when var-10 (setf one var-10)))))
        (f2cl-lib:fdo (jj 1 (f2cl-lib:int-add jj 1))
                      ((> jj j3) nil)
          (tagbody
            (f2cl-lib:fdo (ii jj (f2cl-lib:int-add ii 1))
                          ((> ii jb) nil)
              (tagbody
                (setf (f2cl-lib:fref ab-%data%
                                     ((f2cl-lib:int-add
                                       (f2cl-lib:int-sub ii jj) 1)
                                      (f2cl-lib:int-sub
                                       (f2cl-lib:int-add jj j kv) 1))
                                     ((1 ldab) (1 *)) ab-%offset%)
                      (f2cl-lib:fref work13 (ii jj)
                                     ((1 ldwork) (1 nbmax))))
               label140))
          label150)))))
    (t
     (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                   ((> i
                       (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
                    nil)
       (tagbody
         (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
               (f2cl-lib:int-sub
                (f2cl-lib:int-add
```

```
                    (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                                   ipiv-%offset%)
                    j)
                   1))
     label160))))
(f2cl-lib:fdo (jj (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add jj (f2cl-lib:int-sub 1)))
              ((> jj j) nil)
  (tagbody
    (setf jp
            (f2cl-lib:int-add
             (f2cl-lib:int-sub
              (f2cl-lib:fref ipiv-%data% (jj) ((1 *))
                             ipiv-%offset%)
              jj)
             1))
    (cond
     ((/= jp 1)
      (cond
       ((< (f2cl-lib:int-add jp jj (f2cl-lib:int-sub 1))
           (f2cl-lib:int-add j kl))
        (zswap (f2cl-lib:int-sub jj j)
         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                               ((+ kv 1 jj (f2cl-lib:int-sub j))
                                j)
                               ((1 ldab) (1 *)) ab-%offset%)
         (f2cl-lib:int-sub ldab 1)
         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                               ((+ kv jp jj (f2cl-lib:int-sub j))
                                j)
                               ((1 ldab) (1 *)) ab-%offset%)
         (f2cl-lib:int-sub ldab 1)))
       (t
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (zswap (f2cl-lib:int-sub jj j)
             (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                   ((+ kv 1 jj
                                       (f2cl-lib:int-sub j))
                                    j)
                                   ((1 ldab) (1 *)) ab-%offset%)
             (f2cl-lib:int-sub ldab 1)
             (f2cl-lib:array-slice work31 f2cl-lib:complex16
                                   ((+ jp jj (f2cl-lib:int-sub j)
                                       (f2cl-lib:int-sub kl))
                                    1)
                                   ((1 ldwork) (1 nbmax)))
             ldwork)
          (declare (ignore var-0 var-1 var-2 var-3))
          (when var-4 (setf ldwork var-4)))))))
    (setf nw
```

```
                        (min (the f2cl-lib:integer4 i3)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-sub jj j)
                                                    1))))
                 (if (> nw 0)
                     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                         (zcopy nw
                          (f2cl-lib:array-slice work31 f2cl-lib:complex16
                                                (1
                                                 (f2cl-lib:int-add
                                                  (f2cl-lib:int-sub jj j) 1))
                                                ((1 ldwork) (1 nbmax)))
                          1
                          (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                                ((+ kv kl 1
                                                    (f2cl-lib:int-sub jj) j)
                                                 jj)
                                                ((1 ldab) (1 *)) ab-%offset%)
                          1)
                       (declare (ignore var-1 var-2 var-3 var-4))
                       (when var-0 (setf nw var-0))))
                label170))
            label180))))
       (go end_label)
      end_label
       (return (values m n kl ku nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgbtrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::m
                                              fortran-to-lisp::n
                                              fortran-to-lisp::kl
                                              fortran-to-lisp::ku nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
```

```
'(fortran-to_lisp::zgbtf2
  fortran-to_lisp::ilaenv
  fortran-to_lisp::xerbla))))
```

———-

47.9 zgbtrs

47.9.1 Axiom unit tests

— **zgbtrs.input** —

```
)set break resume
)sys rm -f zgbtrs.output
)spool zgbtrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

47.9.2 Axiom help page

— **zgbtrs.help** —

```
======================================================================
zgbtrs examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgbtrs (TRANS, N, KL, KU, NRHS, AB, LDAB, IPIV, B, LDB,
           INFO)
           ZGBTRS
```

```
Function/Subroutine Documentation
   subroutine zgbtrs (character TRANS, integer N, integer KL, integer KU,
       integer NRHS, complex*16, dimension( ldab, * ) AB, integer LDAB,
       integer, dimension( * ) IPIV, complex*16, dimension( ldb, * ) B,
       integer LDB, integer INFO)
       ZGBTRS

Purpose:
   ZGBTRS solves a system of linear equations
   A * X = B,  A**T * X = B,  or  A**H * X = B
   with a general band matrix A using the LU factorization computed by ZGBTRF.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations.
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 AB
   AB is COMPLEX*16 array, dimension (LDAB,N)
   Details of the LU factorization of the band matrix A, as
   computed by ZGBTRF.  U is stored as an upper triangular band
   matrix with KL+KU superdiagonals in rows 1 to KL+KU+1, and
   the multipliers used during the factorization are stored in
   rows KL+KU+2 to 2*KL+KU+1.

 LDAB
   LDAB is INTEGER
   The leading dimension of the array AB.  LDAB >= 2*KL+KU+1.
```

```
 IPIV
   IPIV is INTEGER array, dimension (N)
   The pivot indices; for 1 <= i <= N, row i of the matrix was
   interchanged with row IPIV(i).

 B
   B is COMPLEX*16 array, dimension (LDB,NRHS)
   On entry, the right hand side matrix B.
   On exit, the solution matrix X.

 LDB
   LDB is INTEGER
   The leading dimension of the array B.  LDB >= max(1,N).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

47.9.3 fortran code

— zgbtrs.f —

```
*  ===================================================================
      SUBROUTINE ZGBTRS( TRANS, N, KL, KU, NRHS, AB, LDAB, IPIV, B, LDB,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, KL, KU, LDAB, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         AB( LDAB, * ), B( LDB, * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE
```

```
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LNOTI, NOTRAN
      INTEGER            I, J, KD, L, LM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGEMV, ZGERU, ZLACGV, ZSWAP, ZTBSV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 ) THEN
         INFO = -4
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDAB.LT.( 2*KL+KU+1 ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -10
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGBTRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      KD = KU + KL + 1
      LNOTI = KL.GT.0
```

```
*
      IF( NOTRAN ) THEN
*
*        Solve  A*X = B.
*
*        Solve L*X = B, overwriting B with X.
*
*        L is represented as a product of permutations and unit lower
*        triangular matrices L = P(1) * L(1) * ... * P(n-1) * L(n-1),
*        where each transformation L(i) is a rank-one modification of
*        the identity matrix.
*
         IF( LNOTI ) THEN
            DO 10 J = 1, N - 1
               LM = MIN( KL, N-J )
               L = IPIV( J )
               IF( L.NE.J )
     $            CALL ZSWAP( NRHS, B( L, 1 ), LDB, B( J, 1 ), LDB )
               CALL ZGERU( LM, NRHS, -ONE, AB( KD+1, J ), 1, B( J, 1 ),
     $                     LDB, B( J+1, 1 ), LDB )
   10       CONTINUE
         END IF
*
         DO 20 I = 1, NRHS
*
*           Solve U*X = B, overwriting B with X.
*
            CALL ZTBSV( 'Upper', 'No transpose', 'Non-unit', N, KL+KU,
     $                  AB, LDAB, B( 1, I ), 1 )
   20    CONTINUE
*
      ELSE IF( LSAME( TRANS, 'T' ) ) THEN
*
*        Solve A**T * X = B.
*
         DO 30 I = 1, NRHS
*
*           Solve U**T * X = B, overwriting B with X.
*
            CALL ZTBSV( 'Upper', 'Transpose', 'Non-unit', N, KL+KU, AB,
     $                  LDAB, B( 1, I ), 1 )
   30    CONTINUE
*
*        Solve L**T * X = B, overwriting B with X.
*
         IF( LNOTI ) THEN
            DO 40 J = N - 1, 1, -1
               LM = MIN( KL, N-J )
               CALL ZGEMV( 'Transpose', LM, NRHS, -ONE, B( J+1, 1 ),
     $                     LDB, AB( KD+1, J ), 1, ONE, B( J, 1 ), LDB )
```

```
               L = IPIV( J )
               IF( L.NE.J )
     $            CALL ZSWAP( NRHS, B( L, 1 ), LDB, B( J, 1 ), LDB )
   40       CONTINUE
         END IF
*
      ELSE
*
*        Solve A**H * X = B.
*
         DO 50 I = 1, NRHS
*
*           Solve U**H * X = B, overwriting B with X.
*
            CALL ZTBSV( 'Upper', 'Conjugate transpose', 'Non-unit', N,
     $                  KL+KU, AB, LDAB, B( 1, I ), 1 )
   50    CONTINUE
*
*        Solve L**H * X = B, overwriting B with X.
*
         IF( LNOTI ) THEN
            DO 60 J = N - 1, 1, -1
               LM = MIN( KL, N-J )
               CALL ZLACGV( NRHS, B( J, 1 ), LDB )
               CALL ZGEMV( 'Conjugate transpose', LM, NRHS, -ONE,
     $                     B( J+1, 1 ), LDB, AB( KD+1, J ), 1, ONE,
     $                     B( J, 1 ), LDB )
               CALL ZLACGV( NRHS, B( J, 1 ), LDB )
               L = IPIV( J )
               IF( L.NE.J )
     $            CALL ZSWAP( NRHS, B( L, 1 ), LDB, B( J, 1 ), LDB )
   60       CONTINUE
         END IF
      END IF
      RETURN
*
*     End of ZGBTRS
*
      END
```

———-

47.9.4 lisp code

— zgbtrs.lisp —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
```

```
          (ignorable one))
 (defun zgbtrs (trans n kl ku nrhs ab ldab ipiv b ldb$ info)
   (declare (type (array f2cl-lib:integer4 (*)) ipiv)
            (type (array f2cl-lib:complex16 (*)) b ab)
            (type (f2cl-lib:integer4) info ldb$ ldab nrhs ku kl n)
            (type (string *) trans))
   (f2cl-lib:with-multi-array-data
       ((trans character trans-%data% trans-%offset%)
        (ab f2cl-lib:complex16 ab-%data% ab-%offset%)
        (b f2cl-lib:complex16 b-%data% b-%offset%)
        (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
     (prog ((i 0) (j 0) (kd 0) (l 0) (lm 0) (lnoti nil) (notran nil))
       (declare (type (f2cl-lib:integer4) i j kd l lm)
                (type f2cl-lib:logical lnoti notran))
       (setf info 0)
       (setf notran
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame trans "N")
                 (declare (ignore var-1))
                 (when var-0 (setf trans var-0))
                 ret-val))
       (cond
        ((and (not notran)
              (not
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame trans "T")
                 (declare (ignore var-1))
                 (when var-0 (setf trans var-0))
                 ret-val))
              (not
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame trans "C")
                 (declare (ignore var-1))
                 (when var-0 (setf trans var-0))
                 ret-val)))
         (setf info -1))
        ((< n 0) (setf info -2)) ((< kl 0) (setf info -3))
        ((< ku 0) (setf info -4)) ((< nrhs 0) (setf info -5))
        ((< ldab (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
         (setf info -7))
        ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -10)))
       (cond
        ((/= info 0) (xerbla "ZGBTRS" (f2cl-lib:int-sub info))
         (go end_label)))
       (if (or (= n 0) (= nrhs 0))
           (go end_label))
       (setf kd (f2cl-lib:int-add ku kl 1))
       (setf lnoti (> kl 0))
       (cond
```

```
(notran
 (cond
  (lnoti
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
     (tagbody
       (setf lm
               (min (the f2cl-lib:integer4 kl)
                    (the f2cl-lib:integer4 (f2cl-lib:int-sub n j))))
       (setf l (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
       (if (/= l j)
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
               (zswap nrhs
                (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                      (l 1) ((1 ldb$) (1 *))
                                      b-%offset%)
                ldb$
                (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                      (j 1) ((1 ldb$) (1 *))
                                      b-%offset%)
                ldb$)
             (declare (ignore var-1 var-3))
             (when var-0 (setf nrhs var-0))
             (when var-2 (setf ldb$ var-2))
             (when var-4 (setf ldb$ var-4))))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
           (zgeru lm nrhs (- one)
            (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                  ((+ kd 1) j) ((1 ldab) (1 *))
                                  ab-%offset%)
            1
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (j 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                  ((+ j 1) 1) ((1 ldb$) (1 *))
                                  b-%offset%)
            ldb$)
         (declare (ignore var-2 var-3 var-4 var-5 var-7))
         (when var-0 (setf lm var-0))
         (when var-1 (setf nrhs var-1))
         (when var-6 (setf ldb$ var-6))
         (when var-8 (setf ldb$ var-8)))
     label10))))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i nrhs) nil)
   (tagbody
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
```

```
            (ztbsv "Upper" "No transpose" "Non-unit" n
             (f2cl-lib:int-add kl ku) ab ldab
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 i)
                                   ((1 ldb$) (1 *)) b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-2 var-4 var-5 var-7 var-8))
          (when var-3 (setf n var-3))
          (when var-6 (setf ldab var-6)))
      label20)))
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame trans "T")
     (declare (ignore var-1))
     (when var-0 (setf trans var-0))
     ret-val)
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i nrhs) nil)
     (tagbody
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
           (ztbsv "Upper" "Transpose" "Non-unit" n
            (f2cl-lib:int-add kl ku) ab ldab
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 i)
                                  ((1 ldb$) (1 *)) b-%offset%)
            1)
          (declare (ignore var-0 var-1 var-2 var-4 var-5 var-7 var-8))
          (when var-3 (setf n var-3))
          (when var-6 (setf ldab var-6)))
      label30))
   (cond
    (lnoti
     (f2cl-lib:fdo (j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                    (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   ((> j 1) nil)
       (tagbody
         (setf lm
                 (min (the f2cl-lib:integer4 kl)
                      (the f2cl-lib:integer4 (f2cl-lib:int-sub n j))))
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9 var-10)
             (zgemv "Transpose" lm nrhs (- one)
              (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                    ((+ j 1) 1) ((1 ldb$) (1 *))
                                    b-%offset%)
              ldb$
              (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                    ((+ kd 1) j) ((1 ldab) (1 *))
                                    ab-%offset%)
              1 one
              (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (j 1)
```

```
                                    ((1 ldb$) (1 *)) b-%offset%)
                ldb$)
            (declare (ignore var-0 var-3 var-4 var-6 var-7 var-9))
            (when var-1 (setf lm var-1))
            (when var-2 (setf nrhs var-2))
            (when var-5 (setf ldb$ var-5))
            (when var-8 (setf one var-8))
            (when var-10 (setf ldb$ var-10)))
          (setf l (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
          (if (/= l j)
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (zswap nrhs
                   (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                         (l 1) ((1 ldb$) (1 *))
                                         b-%offset%)
                   ldb$
                   (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                         (j 1) ((1 ldb$) (1 *))
                                         b-%offset%)
                   ldb$)
                (declare (ignore var-1 var-3))
                (when var-0 (setf nrhs var-0))
                (when var-2 (setf ldb$ var-2))
                (when var-4 (setf ldb$ var-4))))
        label40)))))
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i nrhs) nil)
   (tagbody
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (ztbsv "Upper" "Conjugate transpose" "Non-unit" n
          (f2cl-lib:int-add kl ku) ab ldab
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 i)
                                ((1 ldb$) (1 *)) b-%offset%)
          1)
       (declare (ignore var-0 var-1 var-2 var-4 var-5 var-7 var-8))
       (when var-3 (setf n var-3))
       (when var-6 (setf ldab var-6)))
    label50))
 (cond
  (lnoti
   (f2cl-lib:fdo (j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                    (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                 ((> j 1) nil)
     (tagbody
       (setf lm
               (min (the f2cl-lib:integer4 kl)
                    (the f2cl-lib:integer4 (f2cl-lib:int-sub n j))))
       (multiple-value-bind (var-0 var-1 var-2)
```

```
    (zlacgv nrhs
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (j 1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$)
  (declare (ignore var-1))
  (when var-0 (setf nrhs var-0))
  (when var-2 (setf ldb$ var-2)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgemv "Conjugate transpose" lm nrhs (- one)
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                           ((+ j 1) 1) ((1 ldb$) (1 *))
                           b-%offset%)
     ldb$
     (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                           ((+ kd 1) j) ((1 ldab) (1 *))
                           ab-%offset%)
     1 one
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (j 1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$)
  (declare (ignore var-0 var-3 var-4 var-6 var-7 var-9))
  (when var-1 (setf lm var-1))
  (when var-2 (setf nrhs var-2))
  (when var-5 (setf ldb$ var-5))
  (when var-8 (setf one var-8))
  (when var-10 (setf ldb$ var-10)))
(multiple-value-bind (var-0 var-1 var-2)
    (zlacgv nrhs
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (j 1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$)
  (declare (ignore var-1))
  (when var-0 (setf nrhs var-0))
  (when var-2 (setf ldb$ var-2)))
(setf l (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
(if (/= l j)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zswap nrhs
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                               (l 1) ((1 ldb$) (1 *))
                               b-%offset%)
         ldb$
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                               (j 1) ((1 ldb$) (1 *))
                               b-%offset%)
         ldb$)
      (declare (ignore var-1 var-3))
      (when var-0 (setf nrhs var-0))
```

```
                    (when var-2 (setf ldb$ var-2))
                    (when var-4 (setf ldb$ var-4))))
              label60))))))
        (go end_label)
       end_label
        (return (values trans n nil nil nrhs nil ldab nil nil ldb$ info))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgbtrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::n nil nil
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::ldab nil nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

47.10 zggbak

47.10.1 Axiom unit tests

— zggbak.input —

```
)set break resume
```

```
)sys rm -f zggbak.output
)spool zggbak.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

47.10.2 Axiom help page

— zggbak.help —

```
==========================================================================
zggbak examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zggbak (JOB, SIDE, N, ILO, IHI, LSCALE, RSCALE, M, V, LDV,
           INFO)
           ZGGBAK

Function/Subroutine Documentation
   subroutine zggbak (character JOB, character SIDE, integer N, integer ILO,
       integer IHI, double precision, dimension( * ) LSCALE, double precision,
       dimension( * ) RSCALE, integer M, complex*16, dimension( ldv, * ) V,
       integer LDV, integer INFO)
       ZGGBAK

Purpose:
   ZGGBAK forms the right or left eigenvectors of a complex generalized
   eigenvalue problem A*x = lambda*B*x, by backward transformation on
   the computed eigenvectors of the balanced pair of matrices output by
   ZGGBAL.

Parameters:

 JOB
   JOB is CHARACTER*1
   Specifies the type of backward transformation required:
   = 'N':  do nothing, return immediately;
```

```
  = 'P':  do backward transformation for permutation only;
  = 'S':  do backward transformation for scaling only;
  = 'B':  do backward transformations for both permutation and scaling.
  JOB must be the same as the argument JOB supplied to ZGGBAL.

SIDE
  SIDE is CHARACTER*1
  = 'R':  V contains right eigenvectors;
  = 'L':  V contains left eigenvectors.

N
  N is INTEGER
  The number of rows of the matrix V.  N >= 0.

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER
  The integers ILO and IHI determined by ZGGBAL.
  1 <= ILO <= IHI <= N, if N > 0; ILO=1 and IHI=0, if N=0.

LSCALE
  LSCALE is DOUBLE PRECISION array, dimension (N)
  Details of the permutations and/or scaling factors applied
  to the left side of A and B, as returned by ZGGBAL.

RSCALE
  RSCALE is DOUBLE PRECISION array, dimension (N)
  Details of the permutations and/or scaling factors applied
  to the right side of A and B, as returned by ZGGBAL.

M
  M is INTEGER
  The number of columns of the matrix V.  M >= 0.

V
  V is COMPLEX*16 array, dimension (LDV,M)
  On entry, the matrix of right or left eigenvectors to be
  transformed, as returned by ZTGEVC.
  On exit, V is overwritten by the transformed eigenvectors.

LDV
  LDV is INTEGER
  The leading dimension of the matrix V. LDV >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

———-

47.10.3 fortran code

— **zggbak.f** —

```
*  =====================================================================
      SUBROUTINE ZGGBAK( JOB, SIDE, N, ILO, IHI, LSCALE, RSCALE, M, V,
     $                   LDV, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOB, SIDE
      INTEGER            IHI, ILO, INFO, LDV, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   LSCALE( * ), RSCALE( * )
      COMPLEX*16         V( LDV, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LEFTV, RIGHTV
      INTEGER            I, K
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZDSCAL, ZSWAP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, INT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      RIGHTV = LSAME( SIDE, 'R' )
      LEFTV = LSAME( SIDE, 'L' )
```

```
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.RIGHTV .AND. .NOT.LEFTV ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( ILO.LT.1 ) THEN
         INFO = -4
      ELSE IF( N.EQ.0 .AND. IHI.EQ.0 .AND. ILO.NE.1 ) THEN
         INFO = -4
      ELSE IF( N.GT.0 .AND. ( IHI.LT.ILO .OR. IHI.GT.MAX( 1, N ) ) )
     $   THEN
         INFO = -5
      ELSE IF( N.EQ.0 .AND. ILO.EQ.1 .AND. IHI.NE.0 ) THEN
         INFO = -5
      ELSE IF( M.LT.0 ) THEN
         INFO = -8
      ELSE IF( LDV.LT.MAX( 1, N ) ) THEN
         INFO = -10
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGGBAK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
      IF( M.EQ.0 )
     $   RETURN
      IF( LSAME( JOB, 'N' ) )
     $   RETURN
*
      IF( ILO.EQ.IHI )
     $   GO TO 30
*
*     Backward balance
*
      IF( LSAME( JOB, 'S' ) .OR. LSAME( JOB, 'B' ) ) THEN
*
*        Backward transformation on right eigenvectors
*
         IF( RIGHTV ) THEN
            DO 10 I = ILO, IHI
               CALL ZDSCAL( M, RSCALE( I ), V( I, 1 ), LDV )
   10       CONTINUE
```

```
         END IF
*
*        Backward transformation on left eigenvectors
*
         IF( LEFTV ) THEN
            DO 20 I = ILO, IHI
               CALL ZDSCAL( M, LSCALE( I ), V( I, 1 ), LDV )
   20       CONTINUE
         END IF
      END IF
*
*     Backward permutation
*
   30 CONTINUE
      IF( LSAME( JOB, 'P' ) .OR. LSAME( JOB, 'B' ) ) THEN
*
*        Backward permutation on right eigenvectors
*
         IF( RIGHTV ) THEN
            IF( ILO.EQ.1 )
     $         GO TO 50
            DO 40 I = ILO - 1, 1, -1
               K = INT(RSCALE( I ))
               IF( K.EQ.I )
     $            GO TO 40
               CALL ZSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   40       CONTINUE
*
   50       CONTINUE
            IF( IHI.EQ.N )
     $         GO TO 70
            DO 60 I = IHI + 1, N
               K = INT(RSCALE( I ))
               IF( K.EQ.I )
     $            GO TO 60
               CALL ZSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   60       CONTINUE
         END IF
*
*        Backward permutation on left eigenvectors
*
   70    CONTINUE
         IF( LEFTV ) THEN
            IF( ILO.EQ.1 )
     $         GO TO 90
            DO 80 I = ILO - 1, 1, -1
               K = INT(LSCALE( I ))
               IF( K.EQ.I )
     $            GO TO 80
               CALL ZSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
```

```
   80       CONTINUE
*
   90       CONTINUE
            IF( IHI.EQ.N )
     $         GO TO 110
            DO 100 I = IHI + 1, N
               K = INT(LSCALE( I ))
               IF( K.EQ.I )
     $            GO TO 100
               CALL ZSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
  100       CONTINUE
         END IF
      END IF
*
  110 CONTINUE
*
      RETURN
*
*     End of ZGGBAK
*
      END
```

———-

47.10.4 lisp code

— zggbak.lisp —

```
(defun zggbak (job side n ilo ihi lscale rscale m v ldv info)
  (declare (type (array f2cl-lib:complex16 (*)) v)
           (type (array double-float (*)) rscale lscale)
           (type (f2cl-lib:integer4) info ldv m ihi ilo n)
           (type (string *) side job))
  (f2cl-lib:with-multi-array-data
      ((job character job-%data% job-%offset%)
       (side character side-%data% side-%offset%)
       (lscale double-float lscale-%data% lscale-%offset%)
       (rscale double-float rscale-%data% rscale-%offset%)
       (v f2cl-lib:complex16 v-%data% v-%offset%))
    (prog ((i 0) (k 0) (leftv nil) (rightv nil))
      (declare (type f2cl-lib:logical rightv leftv)
               (type (f2cl-lib:integer4) k i))
      (setf rightv
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame side "R")
                (declare (ignore var-1))
                (when var-0 (setf side var-0))
                ret-val))
```

```
(setf leftv
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "L")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val))
(setf info 0)
(cond
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "N")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "P")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "S")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "B")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)))
  (setf info -1))
 ((and (not rightv) (not leftv)) (setf info -2)) ((< n 0) (setf info -3))
 ((< ilo 1) (setf info -4))
 ((and (= n 0) (= ihi 0) (/= ilo 1)) (setf info -4))
 ((and (> n 0)
       (or (< ihi ilo)
           (> ihi
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 n)))))
  (setf info -5))
 ((and (= n 0) (= ilo 1) (/= ihi 0)) (setf info -5))
 ((< m 0) (setf info -8))
 ((< ldv (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -10)))
(cond
 ((/= info 0) (xerbla "ZGGBAK" (f2cl-lib:int-sub info)) (go end_label)))
(if (= n 0)
```

```
    (go end_label))
(if (= m 0)
    (go end_label))
(if (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "N")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)
    (go end_label))
(if (= ilo ihi)
    (go label30))
(cond
 ((or
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame job "S")
     (declare (ignore var-1))
     (when var-0 (setf job var-0))
     ret-val)
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame job "B")
     (declare (ignore var-1))
     (when var-0 (setf job var-0))
     ret-val))
  (cond
   (rightv
    (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                  ((> i ihi) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (zdscal m
             (f2cl-lib:array-slice rscale-%data% double-float (i) ((1 *))
                                   rscale-%offset%)
             (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (i 1)
                                   ((1 ldv) (1 *)) v-%offset%)
             ldv)
          (declare (ignore var-1 var-2))
          (when var-0 (setf m var-0))
          (when var-3 (setf ldv var-3)))
       label10))))
  (cond
   (leftv
    (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                  ((> i ihi) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (zdscal m
             (f2cl-lib:array-slice lscale-%data% double-float (i) ((1 *))
                                   lscale-%offset%)
             (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (i 1)
                                   ((1 ldv) (1 *)) v-%offset%)
```

```
                ldv)
            (declare (ignore var-1 var-2))
            (when var-0 (setf m var-0))
            (when var-3 (setf ldv var-3)))
        label20))))))
label30
 (cond
  ((or
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "P")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "B")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (tagbody
     (cond
      (rightv
       (tagbody
         (if (= ilo 1)
             (go label50))
         (f2cl-lib:fdo (i (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))
                        (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                       ((> i 1) nil)
           (tagbody
             (setf k
                     (f2cl-lib:int
                      (f2cl-lib:fref rscale-%data% (i) ((1 *))
                                     rscale-%offset%)))
             (if (= k i)
                 (go label40))
             (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                 (zswap m
                  (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (i 1)
                                        ((1 ldv) (1 *)) v-%offset%)
                  ldv
                  (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (k 1)
                                        ((1 ldv) (1 *)) v-%offset%)
                  ldv)
               (declare (ignore var-1 var-3))
               (when var-0 (setf m var-0))
               (when var-2 (setf ldv var-2))
               (when var-4 (setf ldv var-4)))
           label40))
        label50
         (if (= ihi n)
             (go label70))
```

```
      (f2cl-lib:fdo (i (f2cl-lib:int-add ihi 1) (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf k
                  (f2cl-lib:int
                   (f2cl-lib:fref rscale-%data% (i) ((1 *))
                                  rscale-%offset%)))
          (if (= k i)
              (go label60))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (zswap m
               (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (i 1)
                                     ((1 ldv) (1 *)) v-%offset%)
               ldv
               (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (k 1)
                                     ((1 ldv) (1 *)) v-%offset%)
               ldv)
            (declare (ignore var-1 var-3))
            (when var-0 (setf m var-0))
            (when var-2 (setf ldv var-2))
            (when var-4 (setf ldv var-4)))
         label60)))))
label70
 (cond
  (leftv
   (tagbody
     (if (= ilo 1)
         (go label90))
     (f2cl-lib:fdo (i (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))
                      (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                   ((> i 1) nil)
       (tagbody
         (setf k
                 (f2cl-lib:int
                  (f2cl-lib:fref lscale-%data% (i) ((1 *))
                                 lscale-%offset%)))
         (if (= k i)
             (go label80))
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (zswap m
              (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (i 1)
                                    ((1 ldv) (1 *)) v-%offset%)
              ldv
              (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (k 1)
                                    ((1 ldv) (1 *)) v-%offset%)
              ldv)
           (declare (ignore var-1 var-3))
           (when var-0 (setf m var-0))
           (when var-2 (setf ldv var-2))
           (when var-4 (setf ldv var-4)))
```

```
                label80))
            label90
             (if (= ihi n)
                 (go label110))
             (f2cl-lib:fdo (i (f2cl-lib:int-add ihi 1) (f2cl-lib:int-add i 1))
                           ((> i n) nil)
               (tagbody
                 (setf k
                         (f2cl-lib:int
                          (f2cl-lib:fref lscale-%data% (i) ((1 *))
                                         lscale-%offset%)))
                 (if (= k i)
                     (go label100))
                 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                     (zswap m
                      (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (i 1)
                                            ((1 ldv) (1 *)) v-%offset%)
                      ldv
                      (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (k 1)
                                            ((1 ldv) (1 *)) v-%offset%)
                      ldv)
                   (declare (ignore var-1 var-3))
                   (when var-0 (setf m var-0))
                   (when var-2 (setf ldv var-2))
                   (when var-4 (setf ldv var-4)))
                label100))))))))
     label110
      (go end_label)
     end_label
      (return (values job side nil nil nil nil nil m nil ldv info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zggbak
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::job
```

```
                                        fortran-to-lisp::side nil nil nil
                                        nil nil fortran-to-lisp::m nil
                                        fortran-to-lisp::ldv
                                        fortran-to-lisp::info)
                                      :calls '(fortran-to-lisp::xerbla))))
```

———-

47.11 zggbal

47.11.1 Axiom unit tests

— **zggbal.input** —

```
)set break resume
)sys rm -f zggbal.output
)spool zggbal.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

47.11.2 Axiom help page

— **zggbal.help** —

```
=====================================================================
zggbal examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zggbal (JOB, N, A, LDA, B, LDB, ILO, IHI, LSCALE, RSCALE,
           WORK, INFO)
```

```
          ZGGBAL

Function/Subroutine Documentation
   subroutine zggbal (character JOB, integer N, complex*16, dimension( lda, *
       ) A, integer LDA, complex*16, dimension( ldb, * ) B, integer LDB,
       integer ILO, integer IHI, double precision, dimension( * ) LSCALE,
       double precision, dimension( * ) RSCALE, double precision, dimension( *
       ) WORK, integer INFO)
       ZGGBAL

Purpose:
   ZGGBAL balances a pair of general complex matrices (A,B).  This
   involves, first, permuting A and B by similarity transformations to
   isolate eigenvalues in the first 1 to ILO$-$1 and last IHI+1 to N
   elements on the diagonal; and second, applying a diagonal similarity
   transformation to rows and columns ILO to IHI to make the rows
   and columns as close in norm as possible. Both steps are optional.

   Balancing may reduce the 1-norm of the matrices, and improve the
   accuracy of the computed eigenvalues and/or eigenvectors in the
   generalized eigenvalue problem A*x = lambda*B*x.

Parameters:

 JOB
   JOB is CHARACTER*1
   Specifies the operations to be performed on A and B:
   = 'N':  none:  simply set ILO = 1, IHI = N, LSCALE(I) = 1.0
           and RSCALE(I) = 1.0 for i=1,...,N;
   = 'P':  permute only;
   = 'S':  scale only;
   = 'B':  both permute and scale.

 N
   N is INTEGER
   The order of the matrices A and B.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the input matrix A.
   On exit, A is overwritten by the balanced matrix.
   If JOB = 'N', A is not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A. LDA >= max(1,N).

 B
   B is COMPLEX*16 array, dimension (LDB,N)
   On entry, the input matrix B.
```

```
  On exit, B is overwritten by the balanced matrix.
  If JOB = 'N', B is not referenced.

LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= max(1,N).

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER
  ILO and IHI are set to integers such that on exit
  A(i,j) = 0 and B(i,j) = 0 if i > j and
  j = 1,...,ILO-1 or i = IHI+1,...,N.
  If JOB = 'N' or 'S', ILO = 1 and IHI = N.

LSCALE
  LSCALE is DOUBLE PRECISION array, dimension (N)
  Details of the permutations and scaling factors applied
  to the left side of A and B.  If P(j) is the index of the
  row interchanged with row j, and D(j) is the scaling factor
  applied to row j, then
  LSCALE(j) = P(j)    for J = 1,...,ILO-1
            = D(j)    for J = ILO,...,IHI
            = P(j)    for J = IHI+1,...,N.
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

RSCALE
  RSCALE is DOUBLE PRECISION array, dimension (N)
  Details of the permutations and scaling factors applied
  to the right side of A and B.  If P(j) is the index of the
  column interchanged with column j, and D(j) is the scaling
  factor applied to column j, then
  RSCALE(j) = P(j)    for J = 1,...,ILO-1
            = D(j)    for J = ILO,...,IHI
            = P(j)    for J = IHI+1,...,N.
  The order in which the interchanges are made is N to IHI+1,
  then 1 to ILO-1.

WORK
  WORK is REAL array, dimension (lwork)
  lwork must be at least max(1,6*N) when JOB = 'S' or 'B', and
  at least 1 when JOB = 'N' or 'P'.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

———-

47.11.3 fortran code

— **zggbal.f** —

```
*  =====================================================================
      SUBROUTINE ZGGBAL( JOB, N, A, LDA, B, LDB, ILO, IHI, LSCALE,
     $                   RSCALE, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOB
      INTEGER            IHI, ILO, INFO, LDA, LDB, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   LSCALE( * ), RSCALE( * ), WORK( * )
      COMPLEX*16         A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, HALF, ONE
      PARAMETER          ( ZERO = 0.0D+0, HALF = 0.5D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   THREE, SCLFAC
      PARAMETER          ( THREE = 3.0D+0, SCLFAC = 1.0D+1 )
      COMPLEX*16         CZERO
      PARAMETER          ( CZERO = ( 0.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, ICAB, IFLOW, IP1, IR, IRAB, IT, J, JC, JP1,
     $                   K, KOUNT, L, LCAB, LM1, LRAB, LSFMAX, LSFMIN,
     $                   M, NR, NRP2
      DOUBLE PRECISION   ALPHA, BASL, BETA, CAB, CMAX, COEF, COEF2,
     $                   COEF5, COR, EW, EWC, GAMMA, PGAMMA, RAB, SFMAX,
     $                   SFMIN, SUM, T, TA, TB, TC
      COMPLEX*16         CDUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IZAMAX
```

```
      DOUBLE PRECISION   DDOT, DLAMCH
      EXTERNAL           LSAME, IZAMAX, DDOT, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DSCAL, XERBLA, ZDSCAL, ZSWAP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, INT, LOG10, MAX, MIN, SIGN
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( CDUM ) = ABS( DBLE( CDUM ) ) + ABS( DIMAG( CDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGGBAL', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         ILO = 1
         IHI = N
         RETURN
      END IF
*
      IF( N.EQ.1 ) THEN
         ILO = 1
         IHI = N
         LSCALE( 1 ) = ONE
         RSCALE( 1 ) = ONE
         RETURN
      END IF
```

```
*
      IF( LSAME( JOB, 'N' ) ) THEN
         ILO = 1
         IHI = N
         DO 10 I = 1, N
            LSCALE( I ) = ONE
            RSCALE( I ) = ONE
   10    CONTINUE
         RETURN
      END IF
*
      K = 1
      L = N
      IF( LSAME( JOB, 'S' ) )
     $   GO TO 190
*
      GO TO 30
*
*     Permute the matrices A and B to isolate the eigenvalues.
*
*     Find row with one nonzero in columns 1 through L
*
   20 CONTINUE
      L = LM1
      IF( L.NE.1 )
     $   GO TO 30
*
      RSCALE( 1 ) = 1
      LSCALE( 1 ) = 1
      GO TO 190
*
   30 CONTINUE
      LM1 = L - 1
      DO 80 I = L, 1, -1
         DO 40 J = 1, LM1
            JP1 = J + 1
            IF( A( I, J ).NE.CZERO .OR. B( I, J ).NE.CZERO )
     $         GO TO 50
   40    CONTINUE
         J = L
         GO TO 70
*
   50    CONTINUE
         DO 60 J = JP1, L
            IF( A( I, J ).NE.CZERO .OR. B( I, J ).NE.CZERO )
     $         GO TO 80
   60    CONTINUE
         J = JP1 - 1
*
   70    CONTINUE
```

```
         M = L
         IFLOW = 1
         GO TO 160
   80 CONTINUE
      GO TO 100
*
*     Find column with one nonzero in rows K through N
*
   90 CONTINUE
      K = K + 1
*
  100 CONTINUE
      DO 150 J = K, L
         DO 110 I = K, LM1
            IP1 = I + 1
            IF( A( I, J ).NE.CZERO .OR. B( I, J ).NE.CZERO )
     $         GO TO 120
  110    CONTINUE
         I = L
         GO TO 140
  120    CONTINUE
         DO 130 I = IP1, L
            IF( A( I, J ).NE.CZERO .OR. B( I, J ).NE.CZERO )
     $         GO TO 150
  130    CONTINUE
         I = IP1 - 1
  140    CONTINUE
         M = K
         IFLOW = 2
         GO TO 160
  150 CONTINUE
      GO TO 190
*
*     Permute rows M and I
*
  160 CONTINUE
      LSCALE( M ) = I
      IF( I.EQ.M )
     $   GO TO 170
      CALL ZSWAP( N-K+1, A( I, K ), LDA, A( M, K ), LDA )
      CALL ZSWAP( N-K+1, B( I, K ), LDB, B( M, K ), LDB )
*
*     Permute columns M and J
*
  170 CONTINUE
      RSCALE( M ) = J
      IF( J.EQ.M )
     $   GO TO 180
      CALL ZSWAP( L, A( 1, J ), 1, A( 1, M ), 1 )
      CALL ZSWAP( L, B( 1, J ), 1, B( 1, M ), 1 )
```

```
*
  180 CONTINUE
      GO TO ( 20, 90 )IFLOW
*
  190 CONTINUE
      ILO = K
      IHI = L
*
      IF( LSAME( JOB, 'P' ) ) THEN
         DO 195 I = ILO, IHI
            LSCALE( I ) = ONE
            RSCALE( I ) = ONE
  195    CONTINUE
         RETURN
      END IF
*
      IF( ILO.EQ.IHI )
     $   RETURN
*
*     Balance the submatrix in rows ILO to IHI.
*
      NR = IHI - ILO + 1
      DO 200 I = ILO, IHI
         RSCALE( I ) = ZERO
         LSCALE( I ) = ZERO
*
         WORK( I ) = ZERO
         WORK( I+N ) = ZERO
         WORK( I+2*N ) = ZERO
         WORK( I+3*N ) = ZERO
         WORK( I+4*N ) = ZERO
         WORK( I+5*N ) = ZERO
  200 CONTINUE
*
*     Compute right side vector in resulting linear equations
*
      BASL = LOG10( SCLFAC )
      DO 240 I = ILO, IHI
         DO 230 J = ILO, IHI
            IF( A( I, J ).EQ.CZERO ) THEN
               TA = ZERO
               GO TO 210
            END IF
            TA = LOG10( CABS1( A( I, J ) ) ) / BASL
*
  210       CONTINUE
            IF( B( I, J ).EQ.CZERO ) THEN
               TB = ZERO
               GO TO 220
            END IF
```

```
            TB = LOG10( CABS1( B( I, J ) ) ) / BASL
*
  220       CONTINUE
            WORK( I+4*N ) = WORK( I+4*N ) - TA - TB
            WORK( J+5*N ) = WORK( J+5*N ) - TA - TB
  230    CONTINUE
  240 CONTINUE
*
      COEF = ONE / DBLE( 2*NR )
      COEF2 = COEF*COEF
      COEF5 = HALF*COEF2
      NRP2 = NR + 2
      BETA = ZERO
      IT = 1
*
*     Start generalized conjugate gradient iteration
*
  250 CONTINUE
*
      GAMMA = DDOT( NR, WORK( ILO+4*N ), 1, WORK( ILO+4*N ), 1 ) +
     $        DDOT( NR, WORK( ILO+5*N ), 1, WORK( ILO+5*N ), 1 )
*
      EW = ZERO
      EWC = ZERO
      DO 260 I = ILO, IHI
         EW = EW + WORK( I+4*N )
         EWC = EWC + WORK( I+5*N )
  260 CONTINUE
*
      GAMMA = COEF*GAMMA - COEF2*( EW**2+EWC**2 ) - COEF5*( EW-EWC )**2
      IF( GAMMA.EQ.ZERO )
     $   GO TO 350
      IF( IT.NE.1 )
     $   BETA = GAMMA / PGAMMA
      T = COEF5*( EWC-THREE*EW )
      TC = COEF5*( EW-THREE*EWC )
*
      CALL DSCAL( NR, BETA, WORK( ILO ), 1 )
      CALL DSCAL( NR, BETA, WORK( ILO+N ), 1 )
*
      CALL DAXPY( NR, COEF, WORK( ILO+4*N ), 1, WORK( ILO+N ), 1 )
      CALL DAXPY( NR, COEF, WORK( ILO+5*N ), 1, WORK( ILO ), 1 )
*
      DO 270 I = ILO, IHI
         WORK( I ) = WORK( I ) + TC
         WORK( I+N ) = WORK( I+N ) + T
  270 CONTINUE
*
*     Apply matrix to vector
*
```

```
      DO 300 I = ILO, IHI
         KOUNT = 0
         SUM = ZERO
         DO 290 J = ILO, IHI
            IF( A( I, J ).EQ.CZERO )
     $         GO TO 280
            KOUNT = KOUNT + 1
            SUM = SUM + WORK( J )
  280       CONTINUE
            IF( B( I, J ).EQ.CZERO )
     $         GO TO 290
            KOUNT = KOUNT + 1
            SUM = SUM + WORK( J )
  290    CONTINUE
         WORK( I+2*N ) = DBLE( KOUNT )*WORK( I+N ) + SUM
  300 CONTINUE
*
      DO 330 J = ILO, IHI
         KOUNT = 0
         SUM = ZERO
         DO 320 I = ILO, IHI
            IF( A( I, J ).EQ.CZERO )
     $         GO TO 310
            KOUNT = KOUNT + 1
            SUM = SUM + WORK( I+N )
  310       CONTINUE
            IF( B( I, J ).EQ.CZERO )
     $         GO TO 320
            KOUNT = KOUNT + 1
            SUM = SUM + WORK( I+N )
  320    CONTINUE
         WORK( J+3*N ) = DBLE( KOUNT )*WORK( J ) + SUM
  330 CONTINUE
*
      SUM = DDOT( NR, WORK( ILO+N ), 1, WORK( ILO+2*N ), 1 ) +
     $      DDOT( NR, WORK( ILO ), 1, WORK( ILO+3*N ), 1 )
      ALPHA = GAMMA / SUM
*
*     Determine correction to current iteration
*
      CMAX = ZERO
      DO 340 I = ILO, IHI
         COR = ALPHA*WORK( I+N )
         IF( ABS( COR ).GT.CMAX )
     $      CMAX = ABS( COR )
         LSCALE( I ) = LSCALE( I ) + COR
         COR = ALPHA*WORK( I )
         IF( ABS( COR ).GT.CMAX )
     $      CMAX = ABS( COR )
         RSCALE( I ) = RSCALE( I ) + COR
```

```
  340 CONTINUE
      IF( CMAX.LT.HALF )
     $   GO TO 350
*
      CALL DAXPY( NR, -ALPHA, WORK( ILO+2*N ), 1, WORK( ILO+4*N ), 1 )
      CALL DAXPY( NR, -ALPHA, WORK( ILO+3*N ), 1, WORK( ILO+5*N ), 1 )
*
      PGAMMA = GAMMA
      IT = IT + 1
      IF( IT.LE.NRP2 )
     $   GO TO 250
*
*     End generalized conjugate gradient iteration
*
  350 CONTINUE
      SFMIN = DLAMCH( 'S' )
      SFMAX = ONE / SFMIN
      LSFMIN = INT( LOG10( SFMIN ) / BASL+ONE )
      LSFMAX = INT( LOG10( SFMAX ) / BASL )
      DO 360 I = ILO, IHI
         IRAB = IZAMAX( N-ILO+1, A( I, ILO ), LDA )
         RAB = ABS( A( I, IRAB+ILO-1 ) )
         IRAB = IZAMAX( N-ILO+1, B( I, ILO ), LDB )
         RAB = MAX( RAB, ABS( B( I, IRAB+ILO-1 ) ) )
         LRAB = INT( LOG10( RAB+SFMIN ) / BASL+ONE )
         IR = INT(LSCALE( I ) + SIGN( HALF, LSCALE( I ) ))
         IR = MIN( MAX( IR, LSFMIN ), LSFMAX, LSFMAX-LRAB )
         LSCALE( I ) = SCLFAC**IR
         ICAB = IZAMAX( IHI, A( 1, I ), 1 )
         CAB = ABS( A( ICAB, I ) )
         ICAB = IZAMAX( IHI, B( 1, I ), 1 )
         CAB = MAX( CAB, ABS( B( ICAB, I ) ) )
         LCAB = INT( LOG10( CAB+SFMIN ) / BASL+ONE )
         JC = INT(RSCALE( I ) + SIGN( HALF, RSCALE( I ) ))
         JC = MIN( MAX( JC, LSFMIN ), LSFMAX, LSFMAX-LCAB )
         RSCALE( I ) = SCLFAC**JC
  360 CONTINUE
*
*     Row scaling of matrices A and B
*
      DO 370 I = ILO, IHI
         CALL ZDSCAL( N-ILO+1, LSCALE( I ), A( I, ILO ), LDA )
         CALL ZDSCAL( N-ILO+1, LSCALE( I ), B( I, ILO ), LDB )
  370 CONTINUE
*
*     Column scaling of matrices A and B
*
      DO 380 J = ILO, IHI
         CALL ZDSCAL( IHI, RSCALE( J ), A( 1, J ), 1 )
         CALL ZDSCAL( IHI, RSCALE( J ), B( 1, J ), 1 )
```

```
  380 CONTINUE
*
      RETURN
*
*     End of ZGGBAL
*
      END
```

———-

47.11.4 lisp code

— **zggbal.lisp** —

```
(let* ((zero 0.0d0)
       (half 0.5d0)
       (one 1.0d0)
       (three 3.0d0)
       (sclfac 10.0d0)
       (czero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 0.5d0 0.5d0) half)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 3.0d0 3.0d0) three)
           (type (double-float 10.0d0 10.0d0) sclfac)
           (type (f2cl-lib:complex16) czero)
           (ignorable zero half one three sclfac czero))
  (defun zggbal (job n a lda b ldb$ ilo ihi lscale rscale work info)
    (declare (type (array double-float (*)) work rscale lscale)
             (type (array f2cl-lib:complex16 (*)) b a)
             (type (f2cl-lib:integer4) info ihi ilo ldb$ lda n)
             (type (string *) job))
    (f2cl-lib:with-multi-array-data
        ((job character job-%data% job-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (lscale double-float lscale-%data% lscale-%offset%)
         (rscale double-float rscale-%data% rscale-%offset%)
         (work double-float work-%data% work-%offset%))
      (labels ((cabs1 (cdum)
                 (+ (abs (f2cl-lib:dble cdum)) (abs (f2cl-lib:dimag cdum)))))
        (declare
         (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
          cabs1))
        (prog ((cdum #C(0.0d0 0.0d0)) (alpha 0.0d0) (basl 0.0d0) (beta 0.0d0)
               (cab 0.0d0) (cmax 0.0d0) (coef 0.0d0) (coef2 0.0d0)
               (coef5 0.0d0) (cor 0.0d0) (ew 0.0d0) (ewc 0.0d0) (gamma 0.0d0)
               (pgamma 0.0d0) (rab 0.0d0) (sfmax 0.0d0) (sfmin 0.0d0)
```

```
     (sum 0.0d0) (t$ 0.0d0) (ta 0.0d0) (tb 0.0d0) (tc 0.0d0) (i 0)
     (icab 0) (iflow 0) (ip1 0) (ir 0) (irab 0) (it 0) (j 0) (jc 0)
     (jp1 0) (k 0) (kount 0) (l 0) (lcab 0) (lm1 0) (lrab 0)
     (lsfmax 0) (lsfmin 0) (m 0) (nr 0) (nrp2 0))
(declare (type (f2cl-lib:complex16) cdum)
         (type (double-float) alpha basl beta cab cmax coef coef2
          coef5 cor ew ewc gamma pgamma rab sfmax sfmin sum t$ ta tb
          tc)
         (type (f2cl-lib:integer4) i icab iflow ip1 ir irab it j jc
          jp1 k kount l lcab lm1 lrab lsfmax lsfmin m nr nrp2))
(setf info 0)
(cond
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "N")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "P")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "S")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "B")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6)))
(cond
 ((/= info 0) (xerbla "ZGGBAL" (f2cl-lib:int-sub info))
  (go end_label)))
(cond ((= n 0) (setf ilo 1) (setf ihi n) (go end_label)))
(cond
 ((= n 1) (setf ilo 1) (setf ihi n)
  (setf (f2cl-lib:fref lscale-%data% (1) ((1 *)) lscale-%offset%)
```

```
            one)
    (setf (f2cl-lib:fref rscale-%data% (1) ((1 *)) rscale-%offset%)
            one)
    (go end_label)))
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame job "N")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)
    (setf ilo 1) (setf ihi n)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
                one)
        (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
                one)
       label10))
    (go end_label)))
  (setf k 1)
  (setf l n)
  (if (multiple-value-bind (ret-val var-0 var-1)
          (lsame job "S")
        (declare (ignore var-1))
        (when var-0 (setf job var-0))
        ret-val)
      (go label190))
  (go label30)
 label20
  (setf l lm1)
  (if (/= l 1)
      (go label30))
  (setf (f2cl-lib:fref rscale-%data% (1) ((1 *)) rscale-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'double-float))
  (setf (f2cl-lib:fref lscale-%data% (1) ((1 *)) lscale-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'double-float))
  (go label190)
 label30
  (setf lm1 (f2cl-lib:int-sub l 1))
  (f2cl-lib:fdo (i l (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                ((> i 1) nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j lm1) nil)
        (tagbody
          (setf jp1 (f2cl-lib:int-add j 1))
          (if (or
               (/=
                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
```

```
                                a-%offset%)
                 czero)
                (/=
                 (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                 czero))
               (go label50))
           label40))
       (setf j l)
       (go label70)
      label50
       (f2cl-lib:fdo (j jp1 (f2cl-lib:int-add j 1))
                     ((> j l) nil)
         (tagbody
           (if (or
                (/=
                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                 czero)
                (/=
                 (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                 czero))
               (go label80))
           label60))
       (setf j (f2cl-lib:int-sub jp1 1))
      label70
       (setf m l)
       (setf iflow 1)
       (go label160)
      label80))
   (go label100)
  label90
   (setf k (f2cl-lib:int-add k 1))
  label100
   (f2cl-lib:fdo (j k (f2cl-lib:int-add j 1))
                 ((> j l) nil)
     (tagbody
       (f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
                     ((> i lm1) nil)
         (tagbody
           (setf ip1 (f2cl-lib:int-add i 1))
           (if (or
                (/=
                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                 czero)
                (/=
                 (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
```

```
                 czero))
               (go label120))
           label110))
       (setf i l)
       (go label140)
      label120
       (f2cl-lib:fdo (i ip1 (f2cl-lib:int-add i 1))
                     ((> i l) nil)
         (tagbody
           (if (or
                (/=
                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                 czero)
                (/=
                 (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                 czero))
               (go label150))
           label130))
       (setf i (f2cl-lib:int-sub ip1 1))
      label140
       (setf m k)
       (setf iflow 2)
       (go label160)
      label150))
  (go label190)
 label160
  (setf (f2cl-lib:fref lscale-%data% (m) ((1 *)) lscale-%offset%)
          (coerce (the f2cl-lib:integer4 i) 'double-float))
  (if (= i m)
      (go label170))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (zswap (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i k)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (m k)
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-3))
    (when var-2 (setf lda var-2))
    (when var-4 (setf lda var-4)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (zswap (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (i k)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (m k)
                             ((1 ldb$) (1 *)) b-%offset%)
```

```
      ldb$)
    (declare (ignore var-0 var-1 var-3))
    (when var-2 (setf ldb$ var-2))
    (when var-4 (setf ldb$ var-4)))
label170
 (setf (f2cl-lib:fref rscale-%data% (m) ((1 *)) rscale-%offset%)
         (coerce (the f2cl-lib:integer4 j) 'double-float))
 (if (= j m)
     (go label180))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (zswap l
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 j)
                            ((1 lda) (1 *)) a-%offset%)
      1
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 m)
                            ((1 lda) (1 *)) a-%offset%)
      1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf l var-0)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (zswap l
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 j)
                            ((1 ldb$) (1 *)) b-%offset%)
      1
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 m)
                            ((1 ldb$) (1 *)) b-%offset%)
      1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf l var-0)))
label180
 (f2cl-lib:computed-goto (label20 label90) iflow)
label190
 (setf ilo k)
 (setf ihi l)
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame job "P")
     (declare (ignore var-1))
     (when var-0 (setf job var-0))
     ret-val)
   (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                 ((> i ihi) nil)
     (tagbody
       (setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
               one)
       (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
               one)
      label195))
   (go end_label)))
 (if (= ilo ihi)
```

```
    (go end_label))
(setf nr (f2cl-lib:int-add (f2cl-lib:int-sub ihi ilo) 1))
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i ihi) nil)
  (tagbody
    (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
            zero)
    (setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
            zero)
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) zero)
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n)) ((1 *))
                         work-%offset%)
            zero)
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add i
                                            (f2cl-lib:int-mul 2 n)))
                         ((1 *)) work-%offset%)
            zero)
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add i
                                            (f2cl-lib:int-mul 3 n)))
                         ((1 *)) work-%offset%)
            zero)
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add i
                                            (f2cl-lib:int-mul 4 n)))
                         ((1 *)) work-%offset%)
            zero)
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add i
                                            (f2cl-lib:int-mul 5 n)))
                         ((1 *)) work-%offset%)
            zero)
   label200))
(setf basl (f2cl-lib:log10 sclfac))
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i ihi) nil)
  (tagbody
    (f2cl-lib:fdo (j ilo (f2cl-lib:int-add j 1))
                  ((> j ihi) nil)
      (tagbody
        (cond
         ((= (f2cl-lib:fref a (i j) ((1 lda) (1 *))) czero)
          (setf ta zero) (go label210)))
        (setf ta
                (/
                 (f2cl-lib:log10
                  (cabs1
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)))
```

```
                 basl))
        label210
         (cond
          ((= (f2cl-lib:fref b (i j) ((1 ldb$) (1 *))) czero)
           (setf tb zero) (go label220)))
         (setf tb
                (/
                 (f2cl-lib:log10
                  (cabs1
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)))
                 basl))
        label220
         (setf (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add i
                                                 (f2cl-lib:int-mul 4
                                                                   n)))
                              ((1 *)) work-%offset%)
                (-
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add i
                                                   (f2cl-lib:int-mul
                                                    4 n)))
                                ((1 *)) work-%offset%)
                 ta tb))
         (setf (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add j
                                                 (f2cl-lib:int-mul 5
                                                                   n)))
                              ((1 *)) work-%offset%)
                (-
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add j
                                                   (f2cl-lib:int-mul
                                                    5 n)))
                                ((1 *)) work-%offset%)
                 ta tb))
        label230))
    label240))
 (setf coef (/ one (f2cl-lib:dble (f2cl-lib:int-mul 2 nr))))
 (setf coef2 (* coef coef))
 (setf coef5 (* half coef2))
 (setf nrp2 (f2cl-lib:int-add nr 2))
 (setf beta zero)
 (setf it 1)
label250
 (setf gamma
        (+
         (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
             (ddot nr
```

```
                (f2cl-lib:array-slice work-%data% double-float
                                      ((+ ilo (f2cl-lib:int-mul 4 n)))
                                      ((1 *)) work-%offset%)
                1
                (f2cl-lib:array-slice work-%data% double-float
                                      ((+ ilo (f2cl-lib:int-mul 4 n)))
                                      ((1 *)) work-%offset%)
                1)
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf nr var-0))
              ret-val)
            (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
                (ddot nr
                (f2cl-lib:array-slice work-%data% double-float
                                      ((+ ilo (f2cl-lib:int-mul 5 n)))
                                      ((1 *)) work-%offset%)
                1
                (f2cl-lib:array-slice work-%data% double-float
                                      ((+ ilo (f2cl-lib:int-mul 5 n)))
                                      ((1 *)) work-%offset%)
                1)
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf nr var-0))
              ret-val)))
(setf ew zero)
(setf ewc zero)
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i ihi) nil)
  (tagbody
    (setf ew
            (+ ew
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add i
                                                 (f2cl-lib:int-mul 4
                                                                   n)))
                              ((1 *)) work-%offset%)))
    (setf ewc
            (+ ewc
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add i
                                                 (f2cl-lib:int-mul 5
                                                                   n)))
                              ((1 *)) work-%offset%)))
   label260))
(setf gamma
        (- (* coef gamma) (* coef2 (+ (expt ew 2) (expt ewc 2)))
           (* coef5 (expt (- ew ewc) 2))))
(if (= gamma zero)
    (go label350))
(if (/= it 1)
```

```
    (setf beta (/ gamma pgamma)))
(setf t$ (* coef5 (- ewc (* three ew))))
(setf tc (* coef5 (- ew (* three ewc))))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (dscal nr beta
     (f2cl-lib:array-slice work-%data% double-float (ilo) ((1 *))
                           work-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf nr var-0))
  (when var-1 (setf beta var-1)))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (dscal nr beta
     (f2cl-lib:array-slice work-%data% double-float ((+ ilo n))
                           ((1 *)) work-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf nr var-0))
  (when var-1 (setf beta var-1)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (daxpy nr coef
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ ilo (f2cl-lib:int-mul 4 n))) ((1 *))
                           work-%offset%)
     1
     (f2cl-lib:array-slice work-%data% double-float ((+ ilo n))
                           ((1 *)) work-%offset%)
     1)
  (declare (ignore var-2 var-3 var-4 var-5))
  (when var-0 (setf nr var-0))
  (when var-1 (setf coef var-1)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (daxpy nr coef
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ ilo (f2cl-lib:int-mul 5 n))) ((1 *))
                           work-%offset%)
     1
     (f2cl-lib:array-slice work-%data% double-float (ilo) ((1 *))
                           work-%offset%)
     1)
  (declare (ignore var-2 var-3 var-4 var-5))
  (when var-0 (setf nr var-0))
  (when var-1 (setf coef var-1)))
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i ihi) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (+ (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               tc))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n)) ((1 *))
```

```
                          work-%offset%)
                 (+
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n))
                                 ((1 *)) work-%offset%)
                  t$))
      label270))
   (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                 ((> i ihi) nil)
     (tagbody
       (setf kount 0)
       (setf sum zero)
       (f2cl-lib:fdo (j ilo (f2cl-lib:int-add j 1))
                     ((> j ihi) nil)
         (tagbody
           (if (=
                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%)
                czero)
               (go label280))
           (setf kount (f2cl-lib:int-add kount 1))
           (setf sum
                   (+ sum
                      (f2cl-lib:fref work-%data% (j) ((1 *))
                                     work-%offset%)))
          label280
           (if (=
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)
                czero)
               (go label290))
           (setf kount (f2cl-lib:int-add kount 1))
           (setf sum
                   (+ sum
                      (f2cl-lib:fref work-%data% (j) ((1 *))
                                     work-%offset%)))
          label290))
       (setf (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add i
                                               (f2cl-lib:int-mul 2 n)))
                            ((1 *)) work-%offset%)
             (+
              (* (f2cl-lib:dble kount)
                 (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n))
                                ((1 *)) work-%offset%))
              sum))
      label300))
   (f2cl-lib:fdo (j ilo (f2cl-lib:int-add j 1))
                 ((> j ihi) nil)
     (tagbody
       (setf kount 0)
```

```
    (setf sum zero)
    (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                  ((> i ihi) nil)
      (tagbody
        (if (=
             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                            a-%offset%)
             czero)
            (go label310))
        (setf kount (f2cl-lib:int-add kount 1))
        (setf sum
                (+ sum
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add i n)) ((1 *))
                                  work-%offset%)))
       label310
        (if (=
             (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                            b-%offset%)
             czero)
            (go label320))
        (setf kount (f2cl-lib:int-add kount 1))
        (setf sum
                (+ sum
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add i n)) ((1 *))
                                  work-%offset%)))
       label320))
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add j
                                            (f2cl-lib:int-mul 3 n)))
                         ((1 *)) work-%offset%)
            (+
             (* (f2cl-lib:dble kount)
                (f2cl-lib:fref work-%data% (j) ((1 *))
                               work-%offset%))
             sum))
   label330))
(setf sum
        (+
         (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
             (ddot nr
              (f2cl-lib:array-slice work-%data% double-float
                                    ((+ ilo n)) ((1 *))
                                    work-%offset%)
              1
              (f2cl-lib:array-slice work-%data% double-float
                                    ((+ ilo (f2cl-lib:int-mul 2 n)))
                                    ((1 *)) work-%offset%)
              1)
```

```
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf nr var-0))
            ret-val)
          (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
              (ddot nr
               (f2cl-lib:array-slice work-%data% double-float (ilo)
                                     ((1 *)) work-%offset%)
               1
               (f2cl-lib:array-slice work-%data% double-float
                                     ((+ ilo (f2cl-lib:int-mul 3 n)))
                                     ((1 *)) work-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf nr var-0))
            ret-val)))
(setf alpha (/ gamma sum))
(setf cmax zero)
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i ihi) nil)
  (tagbody
    (setf cor
            (* alpha
               (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n))
                              ((1 *)) work-%offset%)))
    (if (> (abs cor) cmax)
        (setf cmax (abs cor)))
    (setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
            (+
             (f2cl-lib:fref lscale-%data% (i) ((1 *))
                            lscale-%offset%)
             cor))
    (setf cor
            (* alpha
               (f2cl-lib:fref work-%data% (i) ((1 *))
                              work-%offset%)))
    (if (> (abs cor) cmax)
        (setf cmax (abs cor)))
    (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
            (+
             (f2cl-lib:fref rscale-%data% (i) ((1 *))
                            rscale-%offset%)
             cor))
   label340))
(if (< cmax half)
    (go label350))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (daxpy nr (- alpha)
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ ilo (f2cl-lib:int-mul 2 n))) ((1 *))
                           work-%offset%)
```

```
      1
      (f2cl-lib:array-slice work-%data% double-float
                            ((+ ilo (f2cl-lib:int-mul 4 n))) ((1 *))
                            work-%offset%)
      1)
    (declare (ignore var-1 var-2 var-3 var-4 var-5))
    (when var-0 (setf nr var-0)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (daxpy nr (- alpha)
      (f2cl-lib:array-slice work-%data% double-float
                            ((+ ilo (f2cl-lib:int-mul 3 n))) ((1 *))
                            work-%offset%)
      1
      (f2cl-lib:array-slice work-%data% double-float
                            ((+ ilo (f2cl-lib:int-mul 5 n))) ((1 *))
                            work-%offset%)
      1)
    (declare (ignore var-1 var-2 var-3 var-4 var-5))
    (when var-0 (setf nr var-0)))
  (setf pgamma gamma)
  (setf it (f2cl-lib:int-add it 1))
  (if (<= it nrp2)
      (go label250))
 label350
  (setf sfmin (dlamch "S"))
  (setf sfmax (/ one sfmin))
  (setf lsfmin (f2cl-lib:int (+ (/ (f2cl-lib:log10 sfmin) basl) one)))
  (setf lsfmax (f2cl-lib:int (/ (f2cl-lib:log10 sfmax) basl)))
  (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                ((> i ihi) nil)
    (tagbody
      (setf irab
              (multiple-value-bind (ret-val var-0 var-1 var-2)
                  (izamax (f2cl-lib:int-add (f2cl-lib:int-sub n ilo) 1)
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                         (i ilo) ((1 lda) (1 *))
                                         a-%offset%)
                   lda)
                (declare (ignore var-0 var-1))
                (when var-2 (setf lda var-2))
                ret-val))
      (setf rab
              (abs
               (f2cl-lib:fref a-%data%
                              (i
                               (f2cl-lib:int-sub
                                (f2cl-lib:int-add irab ilo) 1))
                              ((1 lda) (1 *)) a-%offset%)))
      (setf irab
              (multiple-value-bind (ret-val var-0 var-1 var-2)
```

```
            (izamax (f2cl-lib:int-add (f2cl-lib:int-sub n ilo) 1)
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                   (i ilo) ((1 ldb$) (1 *))
                                   b-%offset%)
             ldb$)
          (declare (ignore var-0 var-1))
          (when var-2 (setf ldb$ var-2))
          ret-val))
(setf rab
        (max rab
             (abs
              (f2cl-lib:fref b-%data%
                             (i
                              (f2cl-lib:int-sub
                               (f2cl-lib:int-add irab ilo) 1))
                             ((1 ldb$) (1 *)) b-%offset%))))
(setf lrab
        (f2cl-lib:int
         (+ (/ (f2cl-lib:log10 (+ rab sfmin)) basl) one)))
(setf ir
        (f2cl-lib:int
         (+
          (f2cl-lib:fref lscale-%data% (i) ((1 *))
                         lscale-%offset%)
          (f2cl-lib:sign half
                         (f2cl-lib:fref lscale-%data% (i) ((1 *))
                                        lscale-%offset%)))))
(setf ir
        (min
         (the f2cl-lib:integer4
              (max (the f2cl-lib:integer4 ir)
                   (the f2cl-lib:integer4 lsfmin)))
         (the f2cl-lib:integer4 lsfmax)
         (the f2cl-lib:integer4 (f2cl-lib:int-sub lsfmax lrab))))
(setf (f2cl-lib:fref lscale-%data% (i) ((1 *)) lscale-%offset%)
        (expt sclfac ir))
(setf icab
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (izamax ihi
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   (1 i) ((1 lda) (1 *))
                                   a-%offset%)
             1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf ihi var-0))
          ret-val))
(setf cab
        (abs
         (f2cl-lib:fref a-%data% (icab i) ((1 lda) (1 *))
                        a-%offset%)))
```

```
    (setf icab
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (izamax ihi
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                       (1 i) ((1 ldb$) (1 *))
                                       b-%offset%)
                 1)
              (declare (ignore var-1 var-2))
              (when var-0 (setf ihi var-0))
              ret-val))
    (setf cab
            (max cab
                 (abs
                  (f2cl-lib:fref b-%data% (icab i) ((1 ldb$) (1 *))
                                 b-%offset%))))
    (setf lcab
            (f2cl-lib:int
             (+ (/ (f2cl-lib:log10 (+ cab sfmin)) basl) one)))
    (setf jc
            (f2cl-lib:int
             (+
              (f2cl-lib:fref rscale-%data% (i) ((1 *))
                             rscale-%offset%)
              (f2cl-lib:sign half
                             (f2cl-lib:fref rscale-%data% (i) ((1 *))
                                            rscale-%offset%)))))
    (setf jc
            (min
             (the f2cl-lib:integer4
                  (max (the f2cl-lib:integer4 jc)
                       (the f2cl-lib:integer4 lsfmin)))
             (the f2cl-lib:integer4 lsfmax)
             (the f2cl-lib:integer4 (f2cl-lib:int-sub lsfmax lcab))))
    (setf (f2cl-lib:fref rscale-%data% (i) ((1 *)) rscale-%offset%)
            (expt sclfac jc))
   label360))
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i ihi) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zdscal (f2cl-lib:int-add (f2cl-lib:int-sub n ilo) 1)
         (f2cl-lib:array-slice lscale-%data% double-float (i) ((1 *))
                               lscale-%offset%)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i ilo)
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-0 var-1 var-2))
      (when var-3 (setf lda var-3)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zdscal (f2cl-lib:int-add (f2cl-lib:int-sub n ilo) 1)
```

```
                  (f2cl-lib:array-slice lscale-%data% double-float (i) ((1 *))
                                        lscale-%offset%)
                  (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (i ilo)
                                        ((1 ldb$) (1 *)) b-%offset%)
                  ldb$)
                (declare (ignore var-0 var-1 var-2))
                (when var-3 (setf ldb$ var-3)))
            label370))
          (f2cl-lib:fdo (j ilo (f2cl-lib:int-add j 1))
                        ((> j ihi) nil)
            (tagbody
              (multiple-value-bind (var-0 var-1 var-2 var-3)
                  (zdscal ihi
                  (f2cl-lib:array-slice rscale-%data% double-float (j) ((1 *))
                                        rscale-%offset%)
                  (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 j)
                                        ((1 lda) (1 *)) a-%offset%)
                  1)
                (declare (ignore var-1 var-2 var-3))
                (when var-0 (setf ihi var-0)))
              (multiple-value-bind (var-0 var-1 var-2 var-3)
                  (zdscal ihi
                  (f2cl-lib:array-slice rscale-%data% double-float (j) ((1 *))
                                        rscale-%offset%)
                  (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 j)
                                        ((1 ldb$) (1 *)) b-%offset%)
                  1)
                (declare (ignore var-1 var-2 var-3))
                (when var-0 (setf ihi var-0)))
            label380))
          (go end_label)
         end_label
          (return
           (values job nil nil lda nil ldb$ ilo ihi nil nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zggbal
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
```

```
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::job nil nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$
  fortran-to-lisp::ilo
  fortran-to-lisp::ihi nil nil nil
  fortran-to-lisp::info)
:calls '(fortran-to-lisp::xerbla))))
```

———-

47.12 zla_gbamv

47.12.1 Axiom unit tests

— zlagbamv.input —

```
)set break resume
)sys rm -f zlagbamv.output
)spool zlagbamv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

47.12.2 Axiom help page

— zla_gbamv.help —

```
======================================================================
zla_gbamv examples
======================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zla_gbamv (TRANS, M, N, KL, KU, ALPHA, AB, LDAB, X, INCX,
           BETA, Y, INCY)
           ZLA_GBAMV performs a matrix-vector operation to calculate error
           bounds.

Function/Subroutine Documentation
   subroutine zla_gbamv (integer TRANS, integer M, integer N, integer KL,
       integer KU, double precision ALPHA, complex*16, dimension( ldab, * )
       AB, integer LDAB, complex*16, dimension( * ) X, integer INCX, double
       precision BETA, double precision, dimension( * ) Y, integer INCY)
       ZLA_GBAMV performs a matrix-vector operation to calculate error bounds.

Purpose:
   ZLA_GBAMV  performs one of the matrix-vector operations

        y := alpha*abs(A)*abs(x) + beta*abs(y),
   or   y := alpha*abs(A)**T*abs(x) + beta*abs(y),

   where alpha and beta are scalars, x and y are vectors and A is an
   m by n matrix.

   This function is primarily used in calculating error bounds.
   To protect against underflow during evaluation, components in
   the resulting vector are perturbed away from zero by (N+1)
   times the underflow threshold.  To prevent unnecessarily large
   errors for block-structure embedded in general matrices,
   "symbolically" zero components are not perturbed.  A zero
   entry is considered "symbolic" if all multiplications involved
   in computing that entry have at least one zero multiplicand.

Parameters:

 TRANS
   TRANS is INTEGER
   On entry, TRANS specifies the operation to be performed as
   follows:

   BLAS_NO_TRANS      y := alpha*abs(A)*abs(x) + beta*abs(y)
   BLAS_TRANS         y := alpha*abs(A**T)*abs(x) + beta*abs(y)
   BLAS_CONJ_TRANS    y := alpha*abs(A**T)*abs(x) + beta*abs(y)

   Unchanged on exit.

 M
```

```
  M is INTEGER
  On entry, M specifies the number of rows of the matrix A.
  M must be at least zero.
  Unchanged on exit.

N
  N is INTEGER
  On entry, N specifies the number of columns of the matrix A.
  N must be at least zero.
  Unchanged on exit.

KL
  KL is INTEGER
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

ALPHA
  ALPHA is DOUBLE PRECISION
  On entry, ALPHA specifies the scalar alpha.
  Unchanged on exit.

AB
  AB is COMPLEX*16 array of DIMENSION ( LDAB, n )
  Before entry, the leading m by n part of the array AB must
  contain the matrix of coefficients.
  Unchanged on exit.

LDAB
  LDAB is INTEGER
  On entry, LDAB specifies the first dimension of AB as declared
  in the calling (sub) program. LDAB must be at least max( 1, m ).
  Unchanged on exit.

X
  X is COMPLEX*16 array, dimension
  ( 1 + ( n - 1 )*abs( INCX ) ) when TRANS = 'N' or 'n'
  and at least
  ( 1 + ( m - 1 )*abs( INCX ) ) otherwise.
  Before entry, the incremented array X must contain the vector x.
  Unchanged on exit.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of
  X. INCX must not be zero.
  Unchanged on exit.
```

```
 BETA
   BETA is DOUBLE PRECISION
   On entry, BETA specifies the scalar beta. When BETA is
   supplied as zero then Y need not be set on input.
   Unchanged on exit.

 Y
   Y is DOUBLE PRECISION array, dimension
   ( 1 + ( m - 1 )*abs( INCY ) ) when TRANS = 'N' or 'n'
   and at least
   ( 1 + ( n - 1 )*abs( INCY ) ) otherwise.
   Before entry with BETA non-zero, the incremented array Y
   must contain the vector y. On exit, Y is overwritten by the
   updated vector y.

 INCY
   INCY is INTEGER
   On entry, INCY specifies the increment for the elements of
   Y. INCY must not be zero.
   Unchanged on exit.
```

———-

47.12.3 fortran code

— zla_gbamv.f —

```
*  =====================================================================
      SUBROUTINE ZLA_GBAMV( TRANS, M, N, KL, KU, ALPHA, AB, LDAB, X,
     $                      INCX, BETA, Y, INCY )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   ALPHA, BETA
      INTEGER            INCX, INCY, LDAB, M, N, KL, KU, TRANS
*     ..
*     .. Array Arguments ..
      COMPLEX*16         AB( LDAB, * ), X( * )
      DOUBLE PRECISION   Y( * )
*     ..
*
*  =====================================================================
*
```

```
*     .. Parameters ..
      COMPLEX*16         ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            SYMB_ZERO
      DOUBLE PRECISION   TEMP, SAFE1
      INTEGER            I, INFO, IY, J, JX, KX, KY, LENX, LENY, KD, KE
      COMPLEX*16         CDUM
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, DLAMCH
      DOUBLE PRECISION   DLAMCH
*     ..
*     .. External Functions ..
      EXTERNAL           ILATRANS
      INTEGER            ILATRANS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, ABS, REAL, DIMAG, SIGN
*     ..
*     .. Statement Functions
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( CDUM ) = ABS( DBLE( CDUM ) ) + ABS( DIMAG( CDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF     ( .NOT.( ( TRANS.EQ.ILATRANS( 'N' ) )
     $           .OR. ( TRANS.EQ.ILATRANS( 'T' ) )
     $           .OR. ( TRANS.EQ.ILATRANS( 'C' ) ) ) ) THEN
         INFO = 1
      ELSE IF( M.LT.0 )THEN
         INFO = 2
      ELSE IF( N.LT.0 )THEN
         INFO = 3
      ELSE IF( KL.LT.0 .OR. KL.GT.M-1 ) THEN
         INFO = 4
      ELSE IF( KU.LT.0 .OR. KU.GT.N-1 ) THEN
         INFO = 5
      ELSE IF( LDAB.LT.KL+KU+1 )THEN
         INFO = 6
      ELSE IF( INCX.EQ.0 )THEN
         INFO = 8
      ELSE IF( INCY.EQ.0 )THEN
         INFO = 11
```

```
      END IF
      IF( INFO.NE.0 )THEN
         CALL XERBLA( 'ZLA_GBAMV ', INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( ( M.EQ.0 ).OR.( N.EQ.0 ).OR.
     $    ( ( ALPHA.EQ.ZERO ).AND.( BETA.EQ.ONE ) ) )
     $   RETURN
*
*     Set  LENX  and  LENY, the lengths of the vectors x and y, and set
*     up the start points in  X  and  Y.
*
      IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
         LENX = N
         LENY = M
      ELSE
         LENX = M
         LENY = N
      END IF
      IF( INCX.GT.0 )THEN
         KX = 1
      ELSE
         KX = 1 - ( LENX - 1 )*INCX
      END IF
      IF( INCY.GT.0 )THEN
         KY = 1
      ELSE
         KY = 1 - ( LENY - 1 )*INCY
      END IF
*
*     Set SAFE1 essentially to be the underflow threshold times the
*     number of additions in each row.
*
      SAFE1 = DLAMCH( 'Safe minimum' )
      SAFE1 = (N+1)*SAFE1
*
*     Form  y := alpha*abs(A)*abs(x) + beta*abs(y).
*
*     The O(M*N) SYMB_ZERO tests could be replaced by O(N) queries to
*     the inexact flag.  Still doesn't help change the iteration order
*     to per-column.
*
      KD = KU + 1
      KE = KL + 1
      IY = KY
      IF ( INCX.EQ.1 ) THEN
         IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
```

```
         DO I = 1, LENY
            IF ( BETA .EQ. 0.0D+0 ) THEN
               SYMB_ZERO = .TRUE.
               Y( IY ) = 0.0D+0
            ELSE IF ( Y( IY ) .EQ. 0.0D+0 ) THEN
               SYMB_ZERO = .TRUE.
            ELSE
               SYMB_ZERO = .FALSE.
               Y( IY ) = BETA * ABS( Y( IY ) )
            END IF
            IF ( ALPHA .NE. 0.0D+0 ) THEN
               DO J = MAX( I-KL, 1 ), MIN( I+KU, LENX )
                  TEMP = CABS1( AB( KD+I-J, J ) )
                  SYMB_ZERO = SYMB_ZERO .AND.
     $                 ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                  Y( IY ) = Y( IY ) + ALPHA*CABS1( X( J ) )*TEMP
               END DO
            END IF

            IF ( .NOT.SYMB_ZERO)
     $            Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

            IY = IY + INCY
         END DO
      ELSE
         DO I = 1, LENY
            IF ( BETA .EQ. 0.0D+0 ) THEN
               SYMB_ZERO = .TRUE.
               Y( IY ) = 0.0D+0
            ELSE IF ( Y( IY ) .EQ. 0.0D+0 ) THEN
               SYMB_ZERO = .TRUE.
            ELSE
               SYMB_ZERO = .FALSE.
               Y( IY ) = BETA * ABS( Y( IY ) )
            END IF
            IF ( ALPHA .NE. 0.0D+0 ) THEN
               DO J = MAX( I-KL, 1 ), MIN( I+KU, LENX )
                  TEMP = CABS1( AB( KE-I+J, I ) )
                  SYMB_ZERO = SYMB_ZERO .AND.
     $                 ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                  Y( IY ) = Y( IY ) + ALPHA*CABS1( X( J ) )*TEMP
               END DO
            END IF

            IF ( .NOT.SYMB_ZERO)
     $            Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

            IY = IY + INCY
```

```
         END DO
      END IF
   ELSE
      IF( TRANS.EQ.ILATRANS( 'N' ) )THEN
         DO I = 1, LENY
            IF ( BETA .EQ. 0.0D+0 ) THEN
               SYMB_ZERO = .TRUE.
               Y( IY ) = 0.0D+0
            ELSE IF ( Y( IY ) .EQ. 0.0D+0 ) THEN
               SYMB_ZERO = .TRUE.
            ELSE
               SYMB_ZERO = .FALSE.
               Y( IY ) = BETA * ABS( Y( IY ) )
            END IF
            IF ( ALPHA .NE. 0.0D+0 ) THEN
               JX = KX
               DO J = MAX( I-KL, 1 ), MIN( I+KU, LENX )
                  TEMP = CABS1( AB( KD+I-J, J ) )
                  SYMB_ZERO = SYMB_ZERO .AND.
$                      ( X( JX ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                  Y( IY ) = Y( IY ) + ALPHA*CABS1( X( JX ) )*TEMP
                  JX = JX + INCX
               END DO
            END IF

            IF ( .NOT.SYMB_ZERO )
$                Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

            IY = IY + INCY
         END DO
      ELSE
         DO I = 1, LENY
            IF ( BETA .EQ. 0.0D+0 ) THEN
               SYMB_ZERO = .TRUE.
               Y( IY ) = 0.0D+0
            ELSE IF ( Y( IY ) .EQ. 0.0D+0 ) THEN
               SYMB_ZERO = .TRUE.
            ELSE
               SYMB_ZERO = .FALSE.
               Y( IY ) = BETA * ABS( Y( IY ) )
            END IF
            IF ( ALPHA .NE. 0.0D+0 ) THEN
               JX = KX
               DO J = MAX( I-KL, 1 ), MIN( I+KU, LENX )
                  TEMP = CABS1( AB( KE-I+J, I ) )
                  SYMB_ZERO = SYMB_ZERO .AND.
$                      ( X( JX ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                  Y( IY ) = Y( IY ) + ALPHA*CABS1( X( JX ) )*TEMP
```

```
                 JX = JX + INCX
              END DO
           END IF

           IF ( .NOT.SYMB_ZERO )
     $          Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

           IY = IY + INCY
        END DO
      END IF

   END IF
*
   RETURN
*
*  End of ZLA_GBAMV
*
   END
```

47.12.4 lisp code

— zla_gbamv.lisp —

```
(let* ((one (coerce 1.0d0 'f2cl-lib:complex16))
       (zero (coerce 0.0d0 'f2cl-lib:complex16)))
  (declare (type (f2cl-lib:complex16) one)
           (type (f2cl-lib:complex16) zero)
           (ignorable one zero))
  (defun zla_gbamv (trans m n kl ku alpha ab ldab x incx beta y incy)
    (declare (type (array double-float (*)) y)
             (type (array f2cl-lib:complex16 (*)) x ab)
             (type (double-float) beta alpha)
             (type (f2cl-lib:integer4) incy incx ldab ku kl n m trans))
    (f2cl-lib:with-multi-array-data
        ((ab f2cl-lib:complex16 ab-%data% ab-%offset%)
         (x f2cl-lib:complex16 x-%data% x-%offset%)
         (y double-float y-%data% y-%offset%))
      (labels ((cabs1 (cdum)
                 (+ (abs (f2cl-lib:dble cdum)) (abs (f2cl-lib:dimag cdum)))))
        (declare
         (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
          cabs1))
        (prog ((cdum #C(0.0d0 0.0d0)) (i 0) (info 0) (iy 0) (j 0) (jx 0) (kx 0)
               (ky 0) (lenx 0) (leny 0) (kd 0) (ke 0) (temp 0.0d0)
               (safe1 0.0d0) (symb_zero nil))
          (declare (type (f2cl-lib:complex16) cdum)
```

```
          (type (f2cl-lib:integer4) i info iy j jx kx ky lenx leny kd
           ke)
          (type (double-float) temp safe1)
          (type f2cl-lib:logical symb_zero))
(setf info 0)
(cond
 ((not
   (or (= trans (ilatrans "N")) (= trans (ilatrans "T"))
       (= trans (ilatrans "C"))))
  (setf info 1))
 ((< m 0) (setf info 2)) ((< n 0) (setf info 3))
 ((or (< kl 0) (> kl (f2cl-lib:int-add m (f2cl-lib:int-sub 1))))
  (setf info 4))
 ((or (< ku 0) (> ku (f2cl-lib:int-add n (f2cl-lib:int-sub 1))))
  (setf info 5))
 ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info 6))
 ((= incx 0) (setf info 8)) ((= incy 0) (setf info 11)))
(cond ((/= info 0) (xerbla "ZLA_GBAMV " info) (go end_label)))
(if (or (= m 0) (= n 0) (and (= alpha zero) (= beta one)))
    (go end_label))
(cond ((= trans (ilatrans "N")) (setf lenx n) (setf leny m))
      (t (setf lenx m) (setf leny n)))
(cond ((> incx 0) (setf kx 1))
      (t
       (setf kx
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub lenx 1) incx)))))
(cond ((> incy 0) (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub leny 1) incy)))))
(setf safe1 (dlamch "Safe minimum"))
(setf safe1 (* (f2cl-lib:int-add n 1) safe1))
(setf kd (f2cl-lib:int-add ku 1))
(setf ke (f2cl-lib:int-add kl 1))
(setf iy ky)
(cond
 ((= incx 1)
  (cond
   ((= trans (ilatrans "N"))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i leny) nil)
      (tagbody
        (cond
         ((= beta 0.0d0) (setf symb_zero f2cl-lib:%true%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  0.0d0))
```

```
((= (f2cl-lib:fref y (iy) ((1 *))) 0.0d0)
 (setf symb_zero f2cl-lib:%true%))
(t (setf symb_zero f2cl-lib:%false%)
 (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
         (* beta
            (abs
             (f2cl-lib:fref y-%data% (iy) ((1 *))
                            y-%offset%))))))
(cond
 ((/= alpha 0.0d0)
  (f2cl-lib:fdo (j
                 (max
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add i
                                         (f2cl-lib:int-sub
                                          kl)))
                  (the f2cl-lib:integer4 1))
                 (f2cl-lib:int-add j 1))
                ((> j
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add i ku))
                     (the f2cl-lib:integer4 lenx)))
                nil)
    (tagbody
      (setf temp
              (cabs1
               (f2cl-lib:fref ab-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add kd i) j)
                               j)
                              ((1 ldab) (1 *)) ab-%offset%)))
      (setf symb_zero
              (and symb_zero
                   (or
                    (=
                     (f2cl-lib:fref x-%data% (j) ((1 *))
                                    x-%offset%)
                     zero)
                    (= temp zero))))
      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
              (+
               (f2cl-lib:fref y-%data% (iy) ((1 *))
                              y-%offset%)
               (* alpha
                  (cabs1
                   (f2cl-lib:fref x-%data% (j) ((1 *))
                                  x-%offset%))
                  temp)))
     label100001))))
```

```
      (if (not symb_zero)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (+
                   (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                   (f2cl-lib:sign safe1
                                  (f2cl-lib:fref y-%data% (iy)
                                                 ((1 *))
                                                 y-%offset%)))))
      (setf iy (f2cl-lib:int-add iy incy))
     label100000)))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i leny) nil)
     (tagbody
       (cond
        ((= beta 0.0d0) (setf symb_zero f2cl-lib:%true%)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 0.0d0))
        ((= (f2cl-lib:fref y (iy) ((1 *))) 0.0d0)
         (setf symb_zero f2cl-lib:%true%))
        (t (setf symb_zero f2cl-lib:%false%)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (* beta
                    (abs
                     (f2cl-lib:fref y-%data% (iy) ((1 *))
                                    y-%offset%))))))
       (cond
        ((/= alpha 0.0d0)
         (f2cl-lib:fdo (j
                        (max
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add i
                                                (f2cl-lib:int-sub
                                                 kl)))
                         (the f2cl-lib:integer4 1))
                        (f2cl-lib:int-add j 1))
                       ((> j
                           (min
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add i ku))
                            (the f2cl-lib:integer4 lenx)))
                        nil)
           (tagbody
             (setf temp
                     (cabs1
                      (f2cl-lib:fref ab-%data%
                                     ((f2cl-lib:int-add
                                       (f2cl-lib:int-sub ke i) j)
                                      i)
                                     ((1 ldab) (1 *)) ab-%offset%)))
```

```
                (setf symb_zero
                        (and symb_zero
                             (or
                              (=
                               (f2cl-lib:fref x-%data% (j) ((1 *))
                                              x-%offset%)
                               zero)
                              (= temp zero))))
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data% (iy) ((1 *))
                                        y-%offset%)
                         (* alpha
                            (cabs1
                             (f2cl-lib:fref x-%data% (j) ((1 *))
                                            x-%offset%))
                            temp)))
             label100003))))
        (if (not symb_zero)
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+
                     (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (f2cl-lib:sign safe1
                                    (f2cl-lib:fref y-%data% (iy)
                                                   ((1 *))
                                                   y-%offset%)))))
        (setf iy (f2cl-lib:int-add iy incy))
      label100002)))))
(t
 (cond
  ((= trans (ilatrans "N"))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i leny) nil)
     (tagbody
       (cond
        ((= beta 0.0d0) (setf symb_zero f2cl-lib:%true%)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 0.0d0))
        ((= (f2cl-lib:fref y (iy) ((1 *))) 0.0d0)
         (setf symb_zero f2cl-lib:%true%))
        (t (setf symb_zero f2cl-lib:%false%)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (* beta
                    (abs
                     (f2cl-lib:fref y-%data% (iy) ((1 *))
                                    y-%offset%))))))
       (cond
        ((/= alpha 0.0d0) (setf jx kx)
         (f2cl-lib:fdo (j
                        (max
```

```
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add i
                                                     (f2cl-lib:int-sub
                                                      kl)))
                              (the f2cl-lib:integer4 1))
                             (f2cl-lib:int-add j 1))
                            ((> j
                                (min
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add i ku))
                                 (the f2cl-lib:integer4 lenx)))
                             nil)
                 (tagbody
                   (setf temp
                           (cabs1
                            (f2cl-lib:fref ab-%data%
                                           ((f2cl-lib:int-sub
                                             (f2cl-lib:int-add kd i) j)
                                            j)
                                           ((1 ldab) (1 *)) ab-%offset%)))
                   (setf symb_zero
                           (and symb_zero
                                (or
                                 (=
                                  (f2cl-lib:fref x-%data% (jx) ((1 *))
                                                 x-%offset%)
                                  zero)
                                 (= temp zero))))
                   (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                           (+
                            (f2cl-lib:fref y-%data% (iy) ((1 *))
                                           y-%offset%)
                            (* alpha
                               (cabs1
                                (f2cl-lib:fref x-%data% (jx) ((1 *))
                                               x-%offset%))
                               temp)))
                   (setf jx (f2cl-lib:int-add jx incx))
                  label100005))))
             (if (not symb_zero)
                 (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (+
                          (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                          (f2cl-lib:sign safe1
                                         (f2cl-lib:fref y-%data% (iy)
                                                        ((1 *))
                                                        y-%offset%)))))
             (setf iy (f2cl-lib:int-add iy incy))
            label100004)))
        (t
```

```
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i leny) nil)
  (tagbody
    (cond
     ((= beta 0.0d0) (setf symb_zero f2cl-lib:%true%)
      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
              0.0d0))
     ((= (f2cl-lib:fref y (iy) ((1 *))) 0.0d0)
      (setf symb_zero f2cl-lib:%true%))
     (t (setf symb_zero f2cl-lib:%false%)
      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
              (* beta
                 (abs
                  (f2cl-lib:fref y-%data% (iy) ((1 *))
                                 y-%offset%))))))
    (cond
     ((/= alpha 0.0d0) (setf jx kx)
      (f2cl-lib:fdo (j
                     (max
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add i
                                             (f2cl-lib:int-sub
                                              kl)))
                      (the f2cl-lib:integer4 1))
                     (f2cl-lib:int-add j 1))
                    ((> j
                        (min
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add i ku))
                         (the f2cl-lib:integer4 lenx)))
                     nil)
        (tagbody
          (setf temp
                  (cabs1
                   (f2cl-lib:fref ab-%data%
                                  ((f2cl-lib:int-add
                                    (f2cl-lib:int-sub ke i) j)
                                   i)
                                  ((1 ldab) (1 *)) ab-%offset%)))
          (setf symb_zero
                  (and symb_zero
                       (or
                        (=
                         (f2cl-lib:fref x-%data% (jx) ((1 *))
                                        x-%offset%)
                         zero)
                        (= temp zero))))
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (+
                   (f2cl-lib:fref y-%data% (iy) ((1 *))
```

```
                                          y-%offset%)
                                 (* alpha
                                    (cabs1
                                     (f2cl-lib:fref x-%data% (jx) ((1 *))
                                                    x-%offset%))
                                    temp)))
                        (setf jx (f2cl-lib:int-add jx incx))
                       label100007))))
                  (if (not symb_zero)
                      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                              (+
                               (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                               (f2cl-lib:sign safe1
                                              (f2cl-lib:fref y-%data% (iy)
                                                             ((1 *))
                                                             y-%offset%)))))
                  (setf iy (f2cl-lib:int-add iy incy))
                 label100006))))))
           (go end_label)
          end_label
           (return
            (values nil nil nil nil nil nil nil nil nil nil nil nil nil)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zla_gbamv
                 fortran-to-lisp::*f2cl-function-info*)
           (fortran-to-lisp::make-f2cl-finfo :arg-types
                                             '((fortran-to-lisp::integer4)
                                               (fortran-to-lisp::integer4)
                                               (fortran-to-lisp::integer4)
                                               (fortran-to-lisp::integer4)
                                               (fortran-to-lisp::integer4)
                                               (double-float)
                                               (array fortran-to-lisp::complex16
                                                (*))
                                               (fortran-to-lisp::integer4)
                                               (array fortran-to-lisp::complex16
                                                (*))
                                               (fortran-to-lisp::integer4)
                                               (double-float)
                                               (array double-float (*))
                                               (fortran-to-lisp::integer4))
                                             :return-values
                                             '(nil nil nil nil nil nil nil nil
                                               nil nil nil nil nil)
                                             :calls
                                             '(fortran-to-lisp::xerbla
                                               fortran-to-lisp::ilatrans))))
```

———-

47.13 zla_gbrcond_c

47.13.1 Axiom unit tests

— **zlagbrcondc.input** —

```
)set break resume
)sys rm -f zlagbrcondc.output
)spool zlagbrcondc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

47.13.2 Axiom help page

— **zla_gbrcond_c.help** —

```
====================================================================
zla_gbrcond_c examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function zla_gbrcond_c (TRANS, N, KL, KU, AB, LDAB,
           AFB, LDAFB, IPIV, C, CAPPLY, INFO, WORK, RWORK)
           ZLA_GBRCOND_C computes the infinity norm condition number of
           op(A)*inv(diag(c)) for general banded matrices.

Function/Subroutine Documentation
   double precision function zla_gbrcond_c (character TRANS, integer N,
```

```
      integer KL, integer KU, complex*16, dimension( ldab, * ) AB, integer
      LDAB, complex*16, dimension( ldafb, * ) AFB, integer LDAFB, integer,
      dimension( * ) IPIV, double precision, dimension( * ) C, logical
      CAPPLY, integer INFO, complex*16, dimension( * ) WORK, double
      precision, dimension( * ) RWORK)
      ZLA_GBRCOND_C computes the infinity norm condition number of
      op(A)*inv(diag(c)) for general banded matrices.

Purpose:
   ZLA_GBRCOND_C Computes the infinity norm condition number of
   op(A) * inv(diag(C)) where C is a DOUBLE PRECISION vector.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate Transpose = Transpose)

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 AB
   AB is COMPLEX*16 array, dimension (LDAB,N)
   On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
   The j-th column of A is stored in the j-th column of the
   array AB as follows:
   AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

 LDAB
   LDAB is INTEGER
   The leading dimension of the array AB.  LDAB >= KL+KU+1.

 AFB
   AFB is COMPLEX*16 array, dimension (LDAFB,N)
   Details of the LU factorization of the band matrix A, as
   computed by ZGBTRF.  U is stored as an upper triangular
   band matrix with KL+KU superdiagonals in rows 1 to KL+KU+1,
```

```
  and the multipliers used during the factorization are stored
  in rows KL+KU+2 to 2*KL+KU+1.

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from the factorization A = P*L*U
  as computed by ZGBTRF; row i of the matrix was interchanged
  with row IPIV(i).

C
  C is DOUBLE PRECISION array, dimension (N)
  The vector C in the formula op(A) * inv(diag(C)).

CAPPLY
  CAPPLY is LOGICAL
  If .TRUE. then access the vector C in the formula above.

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  i > 0:  The ith argument is invalid.

WORK
  WORK is COMPLEX*16 array, dimension (2*N).
  Workspace.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (N).
  Workspace.
```

———-

47.13.3 fortran code

— zla_gbrcond_c.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION ZLA_GBRCOND_C( TRANS, N, KL, KU, AB,
     $                                          LDAB, AFB, LDAFB, IPIV,
     $                                          C, CAPPLY, INFO, WORK,
     $                                          RWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
```

```
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      LOGICAL            CAPPLY
      INTEGER            N, KL, KU, KD, KE, LDAB, LDAFB, INFO
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         AB( LDAB, * ), AFB( LDAFB, * ), WORK( * )
      DOUBLE PRECISION   C( * ), RWORK( * )
*
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            NOTRANS
      INTEGER            KASE, I, J
      DOUBLE PRECISION   AINVNM, ANORM, TMP
      COMPLEX*16         ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZLACN2, ZGBTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
      ZLA_GBRCOND_C = 0.0D+0
*
      INFO = 0
      NOTRANS = LSAME( TRANS, 'N' )
      IF ( .NOT. NOTRANS .AND. .NOT. LSAME( TRANS, 'T' ) .AND. .NOT.
     $     LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
```

```
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 .OR. KL.GT.N-1 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 .OR. KU.GT.N-1 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -6
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZLA_GBRCOND_C', -INFO )
         RETURN
      END IF
*
*     Compute norm of op(A)*op2(C).
*
      ANORM = 0.0D+0
      KD = KU + 1
      KE = KL + 1
      IF ( NOTRANS ) THEN
         DO I = 1, N
            TMP = 0.0D+0
            IF ( CAPPLY ) THEN
               DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
                  TMP = TMP + CABS1( AB( KD+I-J, J ) ) / C( J )
               END DO
            ELSE
               DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
                  TMP = TMP + CABS1( AB( KD+I-J, J ) )
               END DO
            END IF
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      ELSE
         DO I = 1, N
            TMP = 0.0D+0
            IF ( CAPPLY ) THEN
               DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
                  TMP = TMP + CABS1( AB( KE-I+J, I ) ) / C( J )
               END DO
            ELSE
               DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
                  TMP = TMP + CABS1( AB( KE-I+J, I ) )
               END DO
            END IF
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
```

```
         END DO
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 ) THEN
         ZLA_GBRCOND_C = 1.0D+0
         RETURN
      ELSE IF( ANORM .EQ. 0.0D+0 ) THEN
         RETURN
      END IF
*
*     Estimate the norm of inv(op(A)).
*
      AINVNM = 0.0D+0
*
      KASE = 0
   10 CONTINUE
      CALL ZLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
*
            IF ( NOTRANS ) THEN
               CALL ZGBTRS( 'No transpose', N, KL, KU, 1, AFB, LDAFB,
     $              IPIV, WORK, N, INFO )
            ELSE
               CALL ZGBTRS( 'Conjugate transpose', N, KL, KU, 1, AFB,
     $              LDAFB, IPIV, WORK, N, INFO )
            ENDIF
*
*           Multiply by inv(C).
*
            IF ( CAPPLY ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF
         ELSE
*
*           Multiply by inv(C**H).
*
            IF ( CAPPLY ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
```

```
            END DO
         END IF
*
         IF ( NOTRANS ) THEN
            CALL ZGBTRS( 'Conjugate transpose', N, KL, KU, 1, AFB,
     $           LDAFB, IPIV,  WORK, N, INFO )
         ELSE
            CALL ZGBTRS( 'No transpose', N, KL, KU, 1, AFB, LDAFB,
     $           IPIV, WORK, N, INFO )
         END IF
*
*        Multiply by R.
*
         DO I = 1, N
            WORK( I ) = WORK( I ) * RWORK( I )
         END DO
      END IF
      GO TO 10
   END IF
*
*  Compute the estimate of the reciprocal condition number.
*
   IF( AINVNM .NE. 0.0D+0 )
  $ ZLA_GBRCOND_C = 1.0D+0 / AINVNM
*
   RETURN
*
   END
```

———-

47.13.4 lisp code

— **zla_gbrcond_c.lisp** —

```
(defun zla_gbrcond_c
       (trans n kl ku ab ldab afb ldafb ipiv c capply info work rwork)
  (declare (type f2cl-lib:logical capply)
           (type (array double-float (*)) rwork c)
           (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex16 (*)) work afb ab)
           (type (f2cl-lib:integer4) info ldafb ldab ku kl n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (ab f2cl-lib:complex16 ab-%data% ab-%offset%)
       (afb f2cl-lib:complex16 afb-%data% afb-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%)
```

```
     (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
     (c double-float c-%data% c-%offset%)
     (rwork double-float rwork-%data% rwork-%offset%))
   (labels ((cabs1 (zdum)
              (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
     (declare
      (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
       cabs1))
     (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
            (zdum #C(0.0d0 0.0d0)) (ainvnm 0.0d0) (anorm 0.0d0) (tmp 0.0d0)
            (kase 0) (i 0) (j 0) (notrans nil) (kd 0) (ke 0)
            (zla_gbrcond_c 0.0d0))
       (declare (type f2cl-lib:logical notrans)
                (type (f2cl-lib:integer4) ke kd j i kase)
                (type (double-float) zla_gbrcond_c tmp anorm ainvnm)
                (type (f2cl-lib:complex16) zdum)
                (type (array f2cl-lib:integer4 (3)) isave))
       (setf zla_gbrcond_c 0.0d0)
       (setf info 0)
       (setf notrans
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame trans "N")
                 (declare (ignore var-1))
                 (when var-0 (setf trans var-0))
                 ret-val))
       (cond
        ((and (not notrans)
              (not
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame trans "T")
                 (declare (ignore var-1))
                 (when var-0 (setf trans var-0))
                 ret-val))
              (not
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame trans "C")
                 (declare (ignore var-1))
                 (when var-0 (setf trans var-0))
                 ret-val)))
         (setf info -1))
        ((< n 0) (setf info -2))
        ((or (< kl 0) (> kl (f2cl-lib:int-add n (f2cl-lib:int-sub 1))))
         (setf info -3))
        ((or (< ku 0) (> ku (f2cl-lib:int-add n (f2cl-lib:int-sub 1))))
         (setf info -4))
        ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -6))
        ((< ldafb (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
         (setf info -8)))
       (cond
        ((/= info 0) (xerbla "ZLA_GBRCOND_C" (f2cl-lib:int-sub info))
```

```
  (go end_label)))
(setf anorm 0.0d0)
(setf kd (f2cl-lib:int-add ku 1))
(setf ke (f2cl-lib:int-add kl 1))
(cond
 (notrans
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf tmp 0.0d0)
      (cond
       (capply
        (f2cl-lib:fdo (j
                       (max
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add i
                                               (f2cl-lib:int-sub kl)))
                        (the f2cl-lib:integer4 1))
                       (f2cl-lib:int-add j 1))
                      ((> j
                          (min
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add i ku))
                           (the f2cl-lib:integer4 n)))
                       nil)
          (tagbody
            (setf tmp
                  (+ tmp
                     (/
                      (cabs1
                       (f2cl-lib:fref ab-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add kd i) j)
                                       j)
                                      ((1 ldab) (1 *)) ab-%offset%))
                      (f2cl-lib:fref c-%data% (j) ((1 *))
                                     c-%offset%))))
           label100001)))
       (t
        (f2cl-lib:fdo (j
                       (max
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add i
                                               (f2cl-lib:int-sub kl)))
                        (the f2cl-lib:integer4 1))
                       (f2cl-lib:int-add j 1))
                      ((> j
                          (min
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add i ku))
```

```
                                (the f2cl-lib:integer4 n)))
                            nil)
                (tagbody
                  (setf tmp
                          (+ tmp
                             (cabs1
                              (f2cl-lib:fref ab-%data%
                                             ((f2cl-lib:int-sub
                                               (f2cl-lib:int-add kd i) j)
                                              j)
                                             ((1 ldab) (1 *)) ab-%offset%))))
                 label100002))))
             (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                     tmp)
             (setf anorm (max anorm tmp))
            label100000)))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf tmp 0.0d0)
             (cond
              (capply
               (f2cl-lib:fdo (j
                              (max
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add i
                                                      (f2cl-lib:int-sub kl)))
                               (the f2cl-lib:integer4 1))
                              (f2cl-lib:int-add j 1))
                             ((> j
                                 (min
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-add i ku))
                                  (the f2cl-lib:integer4 n)))
                              nil)
                (tagbody
                  (setf tmp
                          (+ tmp
                             (/
                              (cabs1
                               (f2cl-lib:fref ab-%data%
                                              ((f2cl-lib:int-add
                                                (f2cl-lib:int-sub ke i) j)
                                               i)
                                              ((1 ldab) (1 *)) ab-%offset%))
                              (f2cl-lib:fref c-%data% (j) ((1 *))
                                             c-%offset%))))
                 label100004)))
              (t
```

```
            (f2cl-lib:fdo (j
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add i
                                                   (f2cl-lib:int-sub kl)))
                            (the f2cl-lib:integer4 1))
                           (f2cl-lib:int-add j 1))
                          ((> j
                              (min
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add i ku))
                               (the f2cl-lib:integer4 n)))
                           nil)
              (tagbody
                (setf tmp
                        (+ tmp
                           (cabs1
                            (f2cl-lib:fref ab-%data%
                                           ((f2cl-lib:int-add
                                             (f2cl-lib:int-sub ke i) j)
                                            i)
                                           ((1 ldab) (1 *)) ab-%offset%))))
               label100005))))
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                tmp)
        (setf anorm (max anorm tmp))
       label100003))))
  (cond ((= n 0) (setf zla_gbrcond_c 1.0d0) (go end_label))
        ((= anorm 0.0d0) (go end_label)))
  (setf ainvnm 0.0d0)
  (setf kase 0)
 label10
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (zlacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 ((+ n 1))
                             ((1 *)) work-%offset%)
       work ainvnm kase isave)
    (declare (ignore var-1 var-2 var-5))
    (setf n var-0)
    (setf ainvnm var-3)
    (setf kase var-4))
  (cond
   ((/= kase 0)
    (cond
     ((= kase 2)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (*
```

```
                (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (f2cl-lib:fref rwork-%data% (i) ((1 *))
                               rwork-%offset%)))
    label100006))
  (cond
   (notrans
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (zgbtrs "No transpose" n kl ku 1 afb ldafb ipiv work n info)
      (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
      (setf n var-1)
      (setf ldafb var-6)
      (setf n var-9)
      (setf info var-10)))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (zgbtrs "Conjugate transpose" n kl ku 1 afb ldafb ipiv work n
         info)
      (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
      (setf n var-1)
      (setf ldafb var-6)
      (setf n var-9)
      (setf info var-10))))
  (cond
   (capply
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (*
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
      label100007)))))
 (t
  (cond
   (capply
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (*
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
      label100008))))
  (cond
```

```
            (notrans
             (multiple-value-bind
                 (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                  var-10)
                 (zgbtrs "Conjugate transpose" n kl ku 1 afb ldafb ipiv work n
                  info)
               (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
               (setf n var-1)
               (setf ldafb var-6)
               (setf n var-9)
               (setf info var-10)))
            (t
             (multiple-value-bind
                 (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                  var-10)
                 (zgbtrs "No transpose" n kl ku 1 afb ldafb ipiv work n info)
               (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
               (setf n var-1)
               (setf ldafb var-6)
               (setf n var-9)
               (setf info var-10))))
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                       (*
                        (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                        (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                       rwork-%offset%)))
              label100009))))
         (go label10)))
       (if (/= ainvnm 0.0d0)
           (setf zla_gbrcond_c (/ 1.0d0 ainvnm)))
       (go end_label)
      end_label
       (return
        (values zla_gbrcond_c trans n nil nil nil nil nil ldafb nil nil nil
                info nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zla_gbrcond_c
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
```

```
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (array double-float (*))
  fortran-to-lisp::logical
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (array double-float (*)))
:return-values
'(fortran-to-lisp::trans
  fortran-to-lisp::n nil nil nil
  nil nil fortran-to-lisp::ldafb
  nil nil nil fortran-to-lisp::info
  nil nil)
:calls
'(fortran-to-lisp::zgbtrs
  fortran-to-lisp::zlacn2
  fortran-to-lisp::xerbla))))
```

———-

47.14 zla_gbrcond_x

47.14.1 Axiom unit tests

— zlagbrcondx.input —

```
)set break resume
)sys rm -f zlagbrcondx.output
)spool zlagbrcondx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

47.14.2 Axiom help page

— zla_gbrcond_x.help —

```
====================================================================
zla_gbrcond_x examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function zla_gbrcond_x (TRANS, N, KL, KU, AB, LDAB,
           AFB, LDAFB, IPIV, X, INFO, WORK, RWORK)
           ZLA_GBRCOND_X computes the infinity norm condition number of
           op(A)*diag(x) for general banded matrices.

Function/Subroutine Documentation
   double precision function zla_gbrcond_x (character TRANS, integer N,
       integer KL, integer KU, complex*16, dimension( ldab, * ) AB, integer
       LDAB, complex*16, dimension( ldafb, * ) AFB, integer LDAFB, integer,
       dimension( * ) IPIV, complex*16, dimension( * ) X, integer INFO,
       complex*16, dimension( * ) WORK, double precision, dimension( * )
       RWORK)
       ZLA_GBRCOND_X computes the infinity norm condition number of
       op(A)*diag(x) for general banded matrices.

Purpose:
   ZLA_GBRCOND_X Computes the infinity norm condition number of
   op(A) * diag(X) where X is a COMPLEX*16 vector.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate Transpose = Transpose)

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 KL
   KL is INTEGER
```

```
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0.

AB
  AB is COMPLEX*16 array, dimension (LDAB,N)
  On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
  The j-th column of A is stored in the j-th column of the
  array AB as follows:
  AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

AFB
  AFB is COMPLEX*16 array, dimension (LDAFB,N)
  Details of the LU factorization of the band matrix A, as
  computed by ZGBTRF.  U is stored as an upper triangular
  band matrix with KL+KU superdiagonals in rows 1 to KL+KU+1,
  and the multipliers used during the factorization are stored
  in rows KL+KU+2 to 2*KL+KU+1.

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AFB.  LDAFB >= 2*KL+KU+1.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from the factorization A = P*L*U
  as computed by ZGBTRF; row i of the matrix was interchanged
  with row IPIV(i).

X
  X is COMPLEX*16 array, dimension (N)
  The vector X in the formula op(A) * diag(X).

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  i > 0:  The ith argument is invalid.

WORK
  WORK is COMPLEX*16 array, dimension (2*N).
  Workspace.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (N).
```

```
Workspace.
```

———-

47.14.3 fortran code

— **zla_gbrcond_x.f** —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION ZLA_GBRCOND_X( TRANS, N, KL, KU, AB,
     $                                         LDAB, AFB, LDAFB, IPIV,
     $                                         X, INFO, WORK, RWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            N, KL, KU, KD, KE, LDAB, LDAFB, INFO
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         AB( LDAB, * ), AFB( LDAFB, * ), WORK( * ),
     $                   X( * )
      DOUBLE PRECISION   RWORK( * )
*
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            NOTRANS
      INTEGER            KASE, I, J
      DOUBLE PRECISION   AINVNM, ANORM, TMP
      COMPLEX*16         ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZLACN2, ZGBTRS, XERBLA
*     ..
```

```
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      ZLA_GBRCOND_X = 0.0D+0
*
      INFO = 0
      NOTRANS = LSAME( TRANS, 'N' )
      IF ( .NOT. NOTRANS .AND. .NOT. LSAME(TRANS, 'T') .AND. .NOT.
     $     LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( KL.LT.0 .OR. KL.GT.N-1 ) THEN
         INFO = -3
      ELSE IF( KU.LT.0 .OR. KU.GT.N-1 ) THEN
         INFO = -4
      ELSE IF( LDAB.LT.KL+KU+1 ) THEN
         INFO = -6
      ELSE IF( LDAFB.LT.2*KL+KU+1 ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZLA_GBRCOND_X', -INFO )
         RETURN
      END IF
*
*     Compute norm of op(A)*op2(C).
*
      KD = KU + 1
      KE = KL + 1
      ANORM = 0.0D+0
      IF ( NOTRANS ) THEN
         DO I = 1, N
            TMP = 0.0D+0
            DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
               TMP = TMP + CABS1( AB( KD+I-J, J) * X( J ) )
            END DO
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      ELSE
         DO I = 1, N
```

```
            TMP = 0.0D+0
            DO J = MAX( I-KL, 1 ), MIN( I+KU, N )
               TMP = TMP + CABS1( AB( KE-I+J, I ) * X( J ) )
            END DO
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 ) THEN
         ZLA_GBRCOND_X = 1.0D+0
         RETURN
      ELSE IF( ANORM .EQ. 0.0D+0 ) THEN
         RETURN
      END IF
*
*     Estimate the norm of inv(op(A)).
*
      AINVNM = 0.0D+0
*
      KASE = 0
   10 CONTINUE
      CALL ZLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
*
            IF ( NOTRANS ) THEN
               CALL ZGBTRS( 'No transpose', N, KL, KU, 1, AFB, LDAFB,
     $              IPIV, WORK, N, INFO )
            ELSE
               CALL ZGBTRS( 'Conjugate transpose', N, KL, KU, 1, AFB,
     $              LDAFB, IPIV, WORK, N, INFO )
            ENDIF
*
*           Multiply by inv(X).
*
            DO I = 1, N
               WORK( I ) = WORK( I ) / X( I )
            END DO
         ELSE
*
*           Multiply by inv(X**H).
```

```
*
            DO I = 1, N
               WORK( I ) = WORK( I ) / X( I )
            END DO
*
            IF ( NOTRANS ) THEN
               CALL ZGBTRS( 'Conjugate transpose', N, KL, KU, 1, AFB,
     $              LDAFB, IPIV, WORK, N, INFO )
            ELSE
               CALL ZGBTRS( 'No transpose', N, KL, KU, 1, AFB, LDAFB,
     $              IPIV, WORK, N, INFO )
            END IF
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM .NE. 0.0D+0 )
     $   ZLA_GBRCOND_X = 1.0D+0 / AINVNM
*
      RETURN
*
      END
```

———-

47.14.4 lisp code

— zla_gbrcond_x.lisp —

```
(defun zla_gbrcond_x (trans n kl ku ab ldab afb ldafb ipiv x info work rwork)
  (declare (type (array double-float (*)) rwork)
           (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex16 (*)) work x afb ab)
           (type (f2cl-lib:integer4) info ldafb ldab ku kl n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (ab f2cl-lib:complex16 ab-%data% ab-%offset%)
       (afb f2cl-lib:complex16 afb-%data% afb-%offset%)
       (x f2cl-lib:complex16 x-%data% x-%offset%)
```

```
    (work f2cl-lib:complex16 work-%data% work-%offset%)
    (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
    (rwork double-float rwork-%data% rwork-%offset%))
  (labels ((cabs1 (zdum)
             (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
    (declare
     (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
      cabs1))
    (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
           (zdum #C(0.0d0 0.0d0)) (ainvnm 0.0d0) (anorm 0.0d0) (tmp 0.0d0)
           (kase 0) (i 0) (j 0) (notrans nil) (kd 0) (ke 0)
           (zla_gbrcond_x 0.0d0))
      (declare (type f2cl-lib:logical notrans)
               (type (f2cl-lib:integer4) ke kd j i kase)
               (type (double-float) zla_gbrcond_x tmp anorm ainvnm)
               (type (f2cl-lib:complex16) zdum)
               (type (array f2cl-lib:integer4 (3)) isave))
      (setf zla_gbrcond_x 0.0d0)
      (setf info 0)
      (setf notrans
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "N")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val))
      (cond
       ((and (not notrans)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "T")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val))
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "C")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((or (< kl 0) (> kl (f2cl-lib:int-add n (f2cl-lib:int-sub 1))))
        (setf info -3))
       ((or (< ku 0) (> ku (f2cl-lib:int-add n (f2cl-lib:int-sub 1))))
        (setf info -4))
       ((< ldab (f2cl-lib:int-add kl ku 1)) (setf info -6))
       ((< ldafb (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
        (setf info -8)))
      (cond
       ((/= info 0) (xerbla "ZLA_GBRCOND_X" (f2cl-lib:int-sub info))
```

```
  (go end_label)))
(setf kd (f2cl-lib:int-add ku 1))
(setf ke (f2cl-lib:int-add kl 1))
(setf anorm 0.0d0)
(cond
 (notrans
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf tmp 0.0d0)
      (f2cl-lib:fdo (j
                     (max
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add i (f2cl-lib:int-sub kl)))
                      (the f2cl-lib:integer4 1))
                     (f2cl-lib:int-add j 1))
                    ((> j
                        (min
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add i ku))
                         (the f2cl-lib:integer4 n)))
                     nil)
        (tagbody
          (setf tmp
                  (+ tmp
                     (cabs1
                      (*
                       (f2cl-lib:fref ab-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add kd i) j)
                                       j)
                                      ((1 ldab) (1 *)) ab-%offset%)
                       (f2cl-lib:fref x-%data% (j) ((1 *))
                                      x-%offset%)))))
          label100001))
      (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
              tmp)
      (setf anorm (max anorm tmp))
     label100000)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf tmp 0.0d0)
      (f2cl-lib:fdo (j
                     (max
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add i (f2cl-lib:int-sub kl)))
                      (the f2cl-lib:integer4 1))
                     (f2cl-lib:int-add j 1))
```

```
                        ((> j
                            (min
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add i ku))
                             (the f2cl-lib:integer4 n)))
                         nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (cabs1
                        (*
                         (f2cl-lib:fref ab-%data%
                                        ((f2cl-lib:int-add
                                          (f2cl-lib:int-sub ke i) j)
                                         i)
                                        ((1 ldab) (1 *)) ab-%offset%)
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%)))))
           label100003))
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                tmp)
        (setf anorm (max anorm tmp))
       label100002))))
 (cond ((= n 0) (setf zla_gbrcond_x 1.0d0) (go end_label))
       ((= anorm 0.0d0) (go end_label)))
 (setf ainvnm 0.0d0)
 (setf kase 0)
label10
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
     (zlacn2 n
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 ((+ n 1))
                            ((1 *)) work-%offset%)
      work ainvnm kase isave)
   (declare (ignore var-1 var-2 var-5))
   (setf n var-0)
   (setf ainvnm var-3)
   (setf kase var-4))
 (cond
  ((/= kase 0)
   (cond
    ((= kase 2)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (*
                  (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                 rwork-%offset%)))
        label100004))
```

```
(cond
 (notrans
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (zgbtrs "No transpose" n kl ku 1 afb ldafb ipiv work n info)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
    (setf n var-1)
    (setf ldafb var-6)
    (setf n var-9)
    (setf info var-10)))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (zgbtrs "Conjugate transpose" n kl ku 1 afb ldafb ipiv work n
       info)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
    (setf n var-1)
    (setf ldafb var-6)
    (setf n var-9)
    (setf info var-10))))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (/
             (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
             (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)))
   label100005)))
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
             (/
              (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)))
    label100006))
 (cond
  (notrans
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (zgbtrs "Conjugate transpose" n kl ku 1 afb ldafb ipiv work n
        info)
     (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
     (setf n var-1)
     (setf ldafb var-6)
     (setf n var-9)
```

```
                (setf info var-10)))
              (t
               (multiple-value-bind
                   (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                    var-10)
                   (zgbtrs "No transpose" n kl ku 1 afb ldafb ipiv work n info)
                 (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7 var-8))
                 (setf n var-1)
                 (setf ldafb var-6)
                 (setf n var-9)
                 (setf info var-10))))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                        (*
                         (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                         (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                        rwork-%offset%)))
               label100007))))
          (go label10)))
        (if (/= ainvnm 0.0d0)
            (setf zla_gbrcond_x (/ 1.0d0 ainvnm)))
        (go end_label)
       end_label
        (return
         (values zla_gbrcond_x trans n nil nil nil nil nil ldafb nil nil info
                 nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zla_gbrcond_x
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
```

```
  (array fortran-to-lisp::complex16
   (*))
  (array double-float (*)))
:return-values
'(fortran-to-lisp::trans
  fortran-to-lisp::n nil nil nil
  nil nil fortran-to-lisp::ldafb
  nil nil fortran-to-lisp::info nil
  nil)
:calls
'(fortran-to-lisp::zgbtrs
  fortran-to-lisp::zlacn2
  fortran-to-lisp::xerbla))))
```

———-

47.15 zla_gbrfsx_extended

47.15.1 Axiom unit tests

— **zlagbrfsxextended.input** —

```
)set break resume
)sys rm -f zlagbrfsxextended.output
)spool zlagbrfsxextended.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

47.15.2 Axiom help page [Demm05]

— **zla_gbrfsx_extended.help** —

```
==========================================================================
zla_gbrfsx_extended examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zla_gbrfsx_extended (PREC_TYPE, TRANS_TYPE, N, KL, KU, NRHS,
           AB, LDAB, AFB, LDAFB, IPIV, COLEQU, C, B, LDB, Y, LDY, BERR_OUT,
           N_NORMS, ERR_BNDS_NORM, ERR_BNDS_COMP, RES, AYB, DY, Y_TAIL, RCOND,
           ITHRESH, RTHRESH, DZ_UB, IGNORE_CWISE, INFO)
           ZLA_GBRFSX_EXTENDED improves the computed solution to a system of
           linear equations for general banded matrices by performing extra-
           precise iterative refinement and provides error bounds and backward
           error estimates for the solution.

Function/Subroutine Documentation
   subroutine zla_gbrfsx_extended (integer PREC_TYPE, integer TRANS_TYPE,
       integer N, integer KL, integer KU, integer NRHS, complex*16, dimension(
       ldab, * ) AB, integer LDAB, complex*16, dimension( ldafb, * ) AFB,
       integer LDAFB, integer, dimension( * ) IPIV, logical COLEQU, double
       precision, dimension( * ) C, complex*16, dimension( ldb, * ) B, integer
       LDB, complex*16, dimension( ldy, * ) Y, integer LDY, double precision,
       dimension( * ) BERR_OUT, integer N_NORMS, double precision, dimension(
       nrhs, * ) ERR_BNDS_NORM, double precision, dimension( nrhs, * )
       ERR_BNDS_COMP, complex*16, dimension( * ) RES, double precision,
       dimension(*) AYB, complex*16, dimension( * ) DY, complex*16, dimension(
       * ) Y_TAIL, double precision RCOND, integer ITHRESH, double precision
       RTHRESH, double precision DZ_UB, logical IGNORE_CWISE, integer INFO)
       ZLA_GBRFSX_EXTENDED improves the computed solution to a system of
       linear equations for general banded matrices by performing extra-
       precise iterative refinement and provides error bounds and backward
       error estimates for the solution.

Purpose:
   ZLA_GBRFSX_EXTENDED improves the computed solution to a system of
   linear equations by performing extra-precise iterative refinement
   and provides error bounds and backward error estimates for the solution.
   This subroutine is called by ZGBRFSX to perform iterative refinement.
   In addition to normwise error bound, the code provides maximum
   componentwise error bound if possible. See comments for ERR_BNDS_NORM
   and ERR_BNDS_COMP for details of the error bounds. Note that this
   subroutine is only resonsible for setting the second fields of
   ERR_BNDS_NORM and ERR_BNDS_COMP.

Parameters:

 PREC_TYPE
   PREC_TYPE is INTEGER
   Specifies the intermediate precision to be used in refinement.
   The value is defined by ILAPREC(P) where P is a CHARACTER and
```

```
  P    = 'S':  Single
       = 'D':  Double
       = 'I':  Indigenous
       = 'X', 'E':  Extra

TRANS_TYPE
  TRANS_TYPE is INTEGER
  Specifies the transposition operation on A.
  The value is defined by ILATRANS(T) where T is a CHARACTER and
  T    = 'N':  No transpose
       = 'T':  Transpose
       = 'C':  Conjugate transpose

N
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

KL
  KL is INTEGER
  The number of subdiagonals within the band of A.  KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals within the band of A.  KU >= 0

NRHS
  NRHS is INTEGER
  The number of right-hand-sides, i.e., the number of columns of the
  matrix B.

AB
  AB is COMPLEX*16 array, dimension (LDAB,N)
  On entry, the N-by-N matrix A.

LDAB
  LDAB is INTEGER
  The leading dimension of the array A.  LDAB >= max(1,N).

AFB
  AFB is COMPLEX*16 array, dimension (LDAF,N)
  The factors L and U from the factorization
  A = P*L*U as computed by ZGBTRF.

LDAFB
  LDAFB is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices from the factorization A = P*L*U
```

```
  as computed by ZGBTRF; row i of the matrix was interchanged
  with row IPIV(i).

COLEQU
  COLEQU is LOGICAL
  If .TRUE. then column equilibration was done to A before calling
  this routine. This is needed to compute the solution and error
  bounds correctly.

C
  C is DOUBLE PRECISION array, dimension (N)
  The column scale factors for A. If COLEQU = .FALSE., C
  is not accessed. If C is input, each element of C should be a power
  of the radix to ensure a reliable solution and error estimates.
  Scaling by powers of the radix does not cause rounding errors unless
  the result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  The right-hand-side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

Y
  Y is COMPLEX*16 array, dimension (LDY,NRHS)
  On entry, the solution matrix X, as computed by ZGBTRS.
  On exit, the improved solution matrix Y.

LDY
  LDY is INTEGER
  The leading dimension of the array Y.  LDY >= max(1,N).

BERR_OUT
  BERR_OUT is DOUBLE PRECISION array, dimension (NRHS)
  On exit, BERR_OUT(j) contains the componentwise relative backward
  error for right-hand-side j from the formula
  max(i) ( abs(RES(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
  where abs(Z) is the componentwise absolute value of the matrix
  or vector Z. This is computed by ZLA_LIN_BERR.

N_NORMS
  N_NORMS is INTEGER
  Determines which error bounds to return (see ERR_BNDS_NORM
  and ERR_BNDS_COMP).
  If N_NORMS >= 1 return normwise error bounds.
  If N_NORMS >= 2 return componentwise error bounds.
```

```
ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
            max_j (abs(XTRUE(j,i) - X(j,i)))
             ------------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

ERR_BNDS_COMP
  ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                     abs(XTRUE(j,i) - X(j,i))
```

```
                max_j ----------------------
                           abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

RES
  RES is COMPLEX*16 array, dimension (N)
  Workspace to hold the intermediate residual.

AYB
  AYB is DOUBLE PRECISION array, dimension (N)
  Workspace.

DY
  DY is COMPLEX*16 array, dimension (N)
  Workspace to hold the intermediate solution.
```

```
Y_TAIL
  Y_TAIL is COMPLEX*16 array, dimension (N)
  Workspace to hold the trailing bits of the intermediate solution.

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-
  conditioned.

ITHRESH
  ITHRESH is INTEGER
  The maximum number of residual computations allowed for
  refinement. The default is 10. For 'aggressive' set to 100 to
  permit convergence using approximate factorizations or
  factorizations other than LU. If the factorization uses a
  technique other than Gaussian elimination, the guarantees in
  ERR_BNDS_NORM and ERR_BNDS_COMP may no longer be trustworthy.

RTHRESH
  RTHRESH is DOUBLE PRECISION
  Determines when to stop refinement if the error estimate stops
  decreasing. Refinement will stop when the next solution no longer
  satisfies norm(dx_{i+1}) < RTHRESH * norm(dx_i) where norm(Z) is
  the infinity norm of Z. RTHRESH satisfies 0 < RTHRESH <= 1. The
  default value is 0.5. For 'aggressive' set to 0.9 to permit
  convergence on extremely ill-conditioned matrices. See LAWN 165
  for more details.

DZ_UB
  DZ_UB is DOUBLE PRECISION
  Determines when to start considering componentwise convergence.
  Componentwise convergence is only considered after each component
  of the solution Y is stable, which we definte as the relative
  change in each component being less than DZ_UB. The default value
  is 0.25, requiring the first bit to be stable. See LAWN 165 for
  more details.

IGNORE_CWISE
  IGNORE_CWISE is LOGICAL
  If .TRUE. then ignore componentwise convergence. Default value
  is .FALSE..

INFO
  INFO is INTEGER
```

```
= 0:  Successful exit.
< 0:  if INFO = -i, the ith argument to ZGBTRS had an illegal value
```

———-

47.15.3 fortran code

— **zla_gbrfsx_extended.f** —

```
*  =====================================================================
      SUBROUTINE ZLA_GBRFSX_EXTENDED( PREC_TYPE, TRANS_TYPE, N, KL, KU,
     $                                NRHS, AB, LDAB, AFB, LDAFB, IPIV,
     $                                COLEQU, C, B, LDB, Y, LDY,
     $                                BERR_OUT, N_NORMS, ERR_BNDS_NORM,
     $                                ERR_BNDS_COMP, RES, AYB, DY,
     $                                Y_TAIL, RCOND, ITHRESH, RTHRESH,
     $                                DZ_UB, IGNORE_CWISE, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDAB, LDAFB, LDB, LDY, N, KL, KU, NRHS,
     $                   PREC_TYPE, TRANS_TYPE, N_NORMS, ITHRESH
      LOGICAL            COLEQU, IGNORE_CWISE
      DOUBLE PRECISION   RTHRESH, DZ_UB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         AB( LDAB, * ), AFB( LDAFB, * ), B( LDB, * ),
     $                   Y( LDY, * ), RES( * ), DY( * ), Y_TAIL( * )
      DOUBLE PRECISION   C( * ), AYB(*), RCOND, BERR_OUT( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      CHARACTER          TRANS
      INTEGER            CNT, I, J, M, X_STATE, Z_STATE, Y_PREC_STATE
      DOUBLE PRECISION   YK, DYK, YMIN, NORMY, NORMX, NORMDX, DXRAT,
     $                   DZRAT, PREVNORMDX, PREV_DZ_Z, DXRATMAX,
     $                   DZRATMAX, DX_X, DZ_Z, FINAL_DX_X, FINAL_DZ_Z,
     $                   EPS, HUGEVAL, INCR_THRESH
```

```
      LOGICAL            INCR_PREC
      COMPLEX*16         ZDUM
*     ..
*     .. Parameters ..
      INTEGER            UNSTABLE_STATE, WORKING_STATE, CONV_STATE,
     $                   NOPROG_STATE, BASE_RESIDUAL, EXTRA_RESIDUAL,
     $                   EXTRA_Y
      PARAMETER          ( UNSTABLE_STATE = 0, WORKING_STATE = 1,
     $                   CONV_STATE = 2, NOPROG_STATE = 3 )
      PARAMETER          ( BASE_RESIDUAL = 0, EXTRA_RESIDUAL = 1,
     $                   EXTRA_Y = 2 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZAXPY, ZCOPY, ZGBTRS, ZGBMV, BLAS_ZGBMV_X,
     $                   BLAS_ZGBMV2_X, ZLA_GBAMV, ZLA_WWADDW, DLAMCH,
     $                   CHLA_TRANSTYPE, ZLA_LIN_BERR
      DOUBLE PRECISION   DLAMCH
      CHARACTER          CHLA_TRANSTYPE
*     ..
*     .. Intrinsic Functions..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      IF (INFO.NE.0) RETURN
      TRANS = CHLA_TRANSTYPE(TRANS_TYPE)
      EPS = DLAMCH( 'Epsilon' )
```

```
      HUGEVAL = DLAMCH( 'Overflow' )
*     Force HUGEVAL to Inf
      HUGEVAL = HUGEVAL * HUGEVAL
*     Using HUGEVAL may lead to spurious underflows.
      INCR_THRESH = DBLE( N ) * EPS
      M = KL+KU+1

      DO J = 1, NRHS
         Y_PREC_STATE = EXTRA_RESIDUAL
         IF ( Y_PREC_STATE .EQ. EXTRA_Y ) THEN
            DO I = 1, N
               Y_TAIL( I ) = 0.0D+0
            END DO
         END IF

         DXRAT = 0.0D+0
         DXRATMAX = 0.0D+0
         DZRAT = 0.0D+0
         DZRATMAX = 0.0D+0
         FINAL_DX_X = HUGEVAL
         FINAL_DZ_Z = HUGEVAL
         PREVNORMDX = HUGEVAL
         PREV_DZ_Z = HUGEVAL
         DZ_Z = HUGEVAL
         DX_X = HUGEVAL

         X_STATE = WORKING_STATE
         Z_STATE = UNSTABLE_STATE
         INCR_PREC = .FALSE.

         DO CNT = 1, ITHRESH
*
*        Compute residual RES = B_s - op(A_s) * Y,
*            op(A) = A, A**T, or A**H depending on TRANS (and type).
*
            CALL ZCOPY( N, B( 1, J ), 1, RES, 1 )
            IF ( Y_PREC_STATE .EQ. BASE_RESIDUAL ) THEN
               CALL ZGBMV( TRANS, M, N, KL, KU, (-1.0D+0,0.0D+0), AB,
     $              LDAB, Y( 1, J ), 1, (1.0D+0,0.0D+0), RES, 1 )
            ELSE IF ( Y_PREC_STATE .EQ. EXTRA_RESIDUAL ) THEN
               CALL BLAS_ZGBMV_X( TRANS_TYPE, N, N, KL, KU,
     $              (-1.0D+0,0.0D+0), AB, LDAB, Y( 1, J ), 1,
     $              (1.0D+0,0.0D+0), RES, 1, PREC_TYPE )
            ELSE
               CALL BLAS_ZGBMV2_X( TRANS_TYPE, N, N, KL, KU,
     $              (-1.0D+0,0.0D+0), AB, LDAB, Y( 1, J ), Y_TAIL, 1,
     $              (1.0D+0,0.0D+0), RES, 1, PREC_TYPE )
            END IF

!        XXX: RES is no longer needed.
```

```
            CALL ZCOPY( N, RES, 1, DY, 1 )
            CALL ZGBTRS( TRANS, N, KL, KU, 1, AFB, LDAFB, IPIV, DY, N,
     $           INFO )
*
*         Calculate relative changes DX_X, DZ_Z and ratios DXRAT, DZRAT.
*
            NORMX = 0.0D+0
            NORMY = 0.0D+0
            NORMDX = 0.0D+0
            DZ_Z = 0.0D+0
            YMIN = HUGEVAL

            DO I = 1, N
               YK = CABS1( Y( I, J ) )
               DYK = CABS1( DY( I ) )

               IF (YK .NE. 0.0D+0) THEN
                  DZ_Z = MAX( DZ_Z, DYK / YK )
               ELSE IF ( DYK .NE. 0.0D+0 ) THEN
                  DZ_Z = HUGEVAL
               END IF

               YMIN = MIN( YMIN, YK )

               NORMY = MAX( NORMY, YK )

               IF ( COLEQU ) THEN
                  NORMX = MAX( NORMX, YK * C( I ) )
                  NORMDX = MAX(NORMDX, DYK * C(I))
               ELSE
                  NORMX = NORMY
                  NORMDX = MAX( NORMDX, DYK )
               END IF
            END DO

            IF ( NORMX .NE. 0.0D+0 ) THEN
               DX_X = NORMDX / NORMX
            ELSE IF ( NORMDX .EQ. 0.0D+0 ) THEN
               DX_X = 0.0D+0
            ELSE
               DX_X = HUGEVAL
            END IF

            DXRAT = NORMDX / PREVNORMDX
            DZRAT = DZ_Z / PREV_DZ_Z
*
*         Check termination criteria.
*
            IF (.NOT.IGNORE_CWISE
     $           .AND. YMIN*RCOND .LT. INCR_THRESH*NORMY
```

```
     $           .AND. Y_PREC_STATE .LT. EXTRA_Y )
     $           INCR_PREC = .TRUE.

            IF ( X_STATE .EQ. NOPROG_STATE .AND. DXRAT .LE. RTHRESH )
     $           X_STATE = WORKING_STATE
            IF ( X_STATE .EQ. WORKING_STATE ) THEN
               IF ( DX_X .LE. EPS ) THEN
                  X_STATE = CONV_STATE
               ELSE IF ( DXRAT .GT. RTHRESH ) THEN
                  IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                     INCR_PREC = .TRUE.
                  ELSE
                     X_STATE = NOPROG_STATE
                  END IF
               ELSE
                  IF ( DXRAT .GT. DXRATMAX ) DXRATMAX = DXRAT
               END IF
               IF ( X_STATE .GT. WORKING_STATE ) FINAL_DX_X = DX_X
            END IF

            IF ( Z_STATE .EQ. UNSTABLE_STATE .AND. DZ_Z .LE. DZ_UB )
     $           Z_STATE = WORKING_STATE
            IF ( Z_STATE .EQ. NOPROG_STATE .AND. DZRAT .LE. RTHRESH )
     $           Z_STATE = WORKING_STATE
            IF ( Z_STATE .EQ. WORKING_STATE ) THEN
               IF ( DZ_Z .LE. EPS ) THEN
                  Z_STATE = CONV_STATE
               ELSE IF ( DZ_Z .GT. DZ_UB ) THEN
                  Z_STATE = UNSTABLE_STATE
                  DZRATMAX = 0.0D+0
                  FINAL_DZ_Z = HUGEVAL
               ELSE IF ( DZRAT .GT. RTHRESH ) THEN
                  IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                     INCR_PREC = .TRUE.
                  ELSE
                     Z_STATE = NOPROG_STATE
                  END IF
               ELSE
                  IF ( DZRAT .GT. DZRATMAX ) DZRATMAX = DZRAT
               END IF
               IF ( Z_STATE .GT. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
            END IF
*
*           Exit if both normwise and componentwise stopped working,
*           but if componentwise is unstable, let it go at least two
*           iterations.
*
            IF ( X_STATE.NE.WORKING_STATE ) THEN
               IF ( IGNORE_CWISE ) GOTO 666
               IF ( Z_STATE.EQ.NOPROG_STATE .OR. Z_STATE.EQ.CONV_STATE )
```

```
     $              GOTO 666
                IF ( Z_STATE.EQ.UNSTABLE_STATE .AND. CNT.GT.1 ) GOTO 666
            END IF

            IF ( INCR_PREC ) THEN
               INCR_PREC = .FALSE.
               Y_PREC_STATE = Y_PREC_STATE + 1
               DO I = 1, N
                  Y_TAIL( I ) = 0.0D+0
               END DO
            END IF

            PREVNORMDX = NORMDX
            PREV_DZ_Z = DZ_Z
*
*           Update soluton.
*
            IF ( Y_PREC_STATE .LT. EXTRA_Y ) THEN
               CALL ZAXPY( N, (1.0D+0,0.0D+0), DY, 1, Y(1,J), 1 )
            ELSE
               CALL ZLA_WWADDW( N, Y(1,J), Y_TAIL, DY )
            END IF

         END DO
*        Target of "IF (Z_STOP .AND. X_STOP)".  Sun's f77 won't EXIT.
 666     CONTINUE
*
*     Set final_* when cnt hits ithresh.
*
         IF ( X_STATE .EQ. WORKING_STATE ) FINAL_DX_X = DX_X
         IF ( Z_STATE .EQ. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
*
*     Compute error bounds.
*
         IF ( N_NORMS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) =
     $           FINAL_DX_X / (1 - DXRATMAX)
         END IF
         IF ( N_NORMS .GE. 2 ) THEN
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) =
     $           FINAL_DZ_Z / (1 - DZRATMAX)
         END IF
*
*     Compute componentwise relative backward error from formula
*         max(i) ( abs(R(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
*     where abs(Z) is the componentwise absolute value of the matrix
*     or vector Z.
*
*        Compute residual RES = B_s - op(A_s) * Y,
*            op(A) = A, A**T, or A**H depending on TRANS (and type).
```

```
*
         CALL ZCOPY( N, B( 1, J ), 1, RES, 1 )
         CALL ZGBMV( TRANS, N, N, KL, KU, (-1.0D+0,0.0D+0), AB, LDAB,
     $        Y(1,J), 1, (1.0D+0,0.0D+0), RES, 1 )

         DO I = 1, N
            AYB( I ) = CABS1( B( I, J ) )
         END DO
*
*     Compute abs(op(A_s))*abs(Y) + abs(B_s).
*
        CALL ZLA_GBAMV( TRANS_TYPE, N, N, KL, KU, 1.0D+0,
     $        AB, LDAB, Y(1, J), 1, 1.0D+0, AYB, 1 )

         CALL ZLA_LIN_BERR( N, N, 1, RES, AYB, BERR_OUT( J ) )
*
*     End of loop for each RHS.
*
      END DO
*
      RETURN
      END
```

———-

47.15.4 lisp code

— zla_gbrfsx_extended.lisp —

```
(let* ((unstable_state 0)
       (working_state 1)
       (conv_state 2)
       (noprog_state 3)
       (base_residual 0)
       (extra_residual 1)
       (extra_y 2)
       (la_linrx_err_i 2))
  (declare (type (f2cl-lib:integer4 0 0) unstable_state)
           (type (f2cl-lib:integer4 1 1) working_state)
           (type (f2cl-lib:integer4 2 2) conv_state)
           (type (f2cl-lib:integer4 3 3) noprog_state)
           (type (f2cl-lib:integer4 0 0) base_residual)
           (type (f2cl-lib:integer4 1 1) extra_residual)
           (type (f2cl-lib:integer4 2 2) extra_y)
           (type (f2cl-lib:integer4 2 2) la_linrx_err_i)
           (ignorable unstable_state working_state conv_state noprog_state
            base_residual extra_residual extra_y la_linrx_err_i))
  (defun zla_gbrfsx_extended
```

```
      (prec_type trans_type n kl ku nrhs ab ldab afb ldafb ipiv colequ c b
       ldb$ y ldy berr_out n_norms err_bnds_norm err_bnds_comp res ayb dy
       y_tail rcond ithresh rthresh dz_ub ignore_cwise info)
(declare (type (double-float) dz_ub rthresh rcond)
         (type (array double-float (*)) ayb err_bnds_comp err_bnds_norm
          berr_out c)
         (type f2cl-lib:logical ignore_cwise colequ)
         (type (array f2cl-lib:integer4 (*)) ipiv)
         (type (array f2cl-lib:complex16 (*)) y_tail dy res y b afb ab)
         (type (f2cl-lib:integer4) info ithresh n_norms ldy ldb$ ldafb ldab
          nrhs ku kl n trans_type prec_type))
(f2cl-lib:with-multi-array-data
    ((ab f2cl-lib:complex16 ab-%data% ab-%offset%)
     (afb f2cl-lib:complex16 afb-%data% afb-%offset%)
     (b f2cl-lib:complex16 b-%data% b-%offset%)
     (y f2cl-lib:complex16 y-%data% y-%offset%)
     (res f2cl-lib:complex16 res-%data% res-%offset%)
     (dy f2cl-lib:complex16 dy-%data% dy-%offset%)
     (y_tail f2cl-lib:complex16 y_tail-%data% y_tail-%offset%)
     (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
     (c double-float c-%data% c-%offset%)
     (berr_out double-float berr_out-%data% berr_out-%offset%)
     (err_bnds_norm double-float err_bnds_norm-%data%
      err_bnds_norm-%offset%)
     (err_bnds_comp double-float err_bnds_comp-%data%
      err_bnds_comp-%offset%)
     (ayb double-float ayb-%data% ayb-%offset%))
  (labels ((cabs1 (zdum)
             (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
    (declare
     (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
      cabs1))
    (prog ((zdum #C(0.0d0 0.0d0)) (incr_prec nil) (yk 0.0d0) (dyk 0.0d0)
           (ymin 0.0d0) (normy 0.0d0) (normx 0.0d0) (normdx 0.0d0)
           (dxrat 0.0d0) (dzrat 0.0d0) (prevnormdx 0.0d0) (prev_dz_z 0.0d0)
           (dxratmax 0.0d0) (dzratmax 0.0d0) (dx_x 0.0d0) (dz_z 0.0d0)
           (final_dx_x 0.0d0) (final_dz_z 0.0d0) (eps 0.0d0)
           (hugeval 0.0d0) (incr_thresh 0.0d0) (cnt 0) (i 0) (j 0) (m 0)
           (x_state 0) (z_state 0) (y_prec_state 0)
           (trans
            (make-array '(1) :element-type 'character :initial-element
                        #\ )))
     (declare (type (f2cl-lib:complex16) zdum)
              (type f2cl-lib:logical incr_prec)
              (type (double-float) yk dyk ymin normy normx normdx dxrat
               dzrat prevnormdx prev_dz_z dxratmax dzratmax dx_x dz_z
               final_dx_x final_dz_z eps hugeval incr_thresh)
              (type (f2cl-lib:integer4) cnt i j m x_state z_state
               y_prec_state)
              (type (string 1) trans))
```

```
(if (/= info 0)
    (go end_label))
(f2cl-lib:f2cl-set-string trans
                          (f2cl-lib:fref-string #'chla_transtype
                                                (trans_type))
                          (string 1))
(setf eps (dlamch "Epsilon"))
(setf hugeval (dlamch "Overflow"))
(setf hugeval (* hugeval hugeval))
(setf incr_thresh (* (f2cl-lib:dble n) eps))
(setf m (f2cl-lib:int-add kl ku 1))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf y_prec_state extra_residual)
    (cond
     ((= y_prec_state extra_y)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                               y_tail-%offset%)
                (coerce 0.0d0 'f2cl-lib:complex16))
         label100001))))
    (setf dxrat 0.0d0)
    (setf dxratmax 0.0d0)
    (setf dzrat 0.0d0)
    (setf dzratmax 0.0d0)
    (setf final_dx_x hugeval)
    (setf final_dz_z hugeval)
    (setf prevnormdx hugeval)
    (setf prev_dz_z hugeval)
    (setf dz_z hugeval)
    (setf dx_x hugeval)
    (setf x_state working_state)
    (setf z_state unstable_state)
    (setf incr_prec f2cl-lib:%false%)
    (f2cl-lib:fdo (cnt 1 (f2cl-lib:int-add cnt 1))
                  ((> cnt ithresh) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (zcopy n
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 j)
                                   ((1 ldb$) (1 *)) b-%offset%)
             1 res 1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf n var-0)))
        (cond
         ((= y_prec_state base_residual)
          (multiple-value-bind
```

```
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgbmv trans m n kl ku (f2cl-lib:cmplx -1.0d0 0.0d0) ab
       ldab
       (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                             (1 j) ((1 ldy) (1 *))
                             y-%offset%)
       1 (f2cl-lib:cmplx 1.0d0 0.0d0) res 1)
    (declare
     (ignore var-5 var-6 var-8 var-9 var-10 var-11 var-12))
    (when var-0 (setf trans var-0))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf kl var-3))
    (when var-4 (setf ku var-4))
    (when var-7 (setf ldab var-7))))
  ((= y_prec_state extra_residual)
   (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13)
      (blas_zgbmv_x trans_type n n kl ku
       (f2cl-lib:cmplx -1.0d0 0.0d0) ab ldab
       (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                             (1 j) ((1 ldy) (1 *))
                             y-%offset%)
       1 (f2cl-lib:cmplx 1.0d0 0.0d0) res 1 prec_type)
    (declare
     (ignore var-5 var-6 var-8 var-9 var-10 var-11 var-12))
    (when var-0 (setf trans_type var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf kl var-3))
    (when var-4 (setf ku var-4))
    (when var-7 (setf ldab var-7))
    (when var-13 (setf prec_type var-13))))
  (t
   (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13 var-14)
      (blas_zgbmv2_x trans_type n n kl ku
       (f2cl-lib:cmplx -1.0d0 0.0d0) ab ldab
       (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                             (1 j) ((1 ldy) (1 *))
                             y-%offset%)
       y_tail 1 (f2cl-lib:cmplx 1.0d0 0.0d0) res 1 prec_type)
    (declare
     (ignore var-5 var-6 var-8 var-9 var-10 var-11 var-12
      var-13))
    (when var-0 (setf trans_type var-0))
    (when var-1 (setf n var-1))
```

```
    (when var-2 (setf n var-2))
    (when var-3 (setf kl var-3))
    (when var-4 (setf ku var-4))
    (when var-7 (setf ldab var-7))
    (when var-14 (setf prec_type var-14)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (zcopy n res 1 dy 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (zgbtrs trans n kl ku 1 afb ldafb ipiv dy n info)
  (declare (ignore var-2 var-3 var-4 var-5 var-7 var-8))
  (setf trans var-0)
  (setf n var-1)
  (setf ldafb var-6)
  (setf n var-9)
  (setf info var-10))
(setf normx 0.0d0)
(setf normy 0.0d0)
(setf normdx 0.0d0)
(setf dz_z 0.0d0)
(setf ymin hugeval)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf yk
            (cabs1
             (f2cl-lib:fref y-%data% (i j) ((1 ldy) (1 *))
                            y-%offset%)))
    (setf dyk
            (cabs1
             (f2cl-lib:fref dy-%data% (i) ((1 *))
                            dy-%offset%)))
    (cond ((/= yk 0.0d0) (setf dz_z (max dz_z (/ dyk yk))))
          ((/= dyk 0.0d0) (setf dz_z hugeval)))
    (setf ymin (min ymin yk))
    (setf normy (max normy yk))
    (cond
     (colequ
      (setf normx
              (max normx
                   (* yk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%))))
      (setf normdx
              (max normdx
                   (* dyk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
```

```
                                        c-%offset%)))))
      (t (setf normx normy) (setf normdx (max normdx dyk))))
   label100003))
(cond ((/= normx 0.0d0) (setf dx_x (/ normdx normx)))
      ((= normdx 0.0d0) (setf dx_x 0.0d0))
      (t (setf dx_x hugeval)))
(setf dxrat (/ normdx prevnormdx))
(setf dzrat (/ dz_z prev_dz_z))
(if (and (not ignore_cwise)
         (< (* ymin rcond) (* incr_thresh normy))
         (< y_prec_state extra_y))
    (setf incr_prec f2cl-lib:%true%))
(if (and (= x_state noprog_state) (<= dxrat rthresh))
    (setf x_state working_state))
(cond
 ((= x_state working_state)
  (cond ((<= dx_x eps) (setf x_state conv_state))
        ((> dxrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf x_state noprog_state))))
        (t
         (if (> dxrat dxratmax)
             (setf dxratmax dxrat))))
  (if (> x_state working_state)
      (setf final_dx_x dx_x))))
(if (and (= z_state unstable_state) (<= dz_z dz_ub))
    (setf z_state working_state))
(if (and (= z_state noprog_state) (<= dzrat rthresh))
    (setf z_state working_state))
(cond
 ((= z_state working_state)
  (cond ((<= dz_z eps) (setf z_state conv_state))
        ((> dz_z dz_ub) (setf z_state unstable_state)
         (setf dzratmax 0.0d0) (setf final_dz_z hugeval))
        ((> dzrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf z_state noprog_state))))
        (t
         (if (> dzrat dzratmax)
             (setf dzratmax dzrat))))
  (if (> z_state working_state)
      (setf final_dz_z dz_z))))
(cond
 ((/= x_state working_state)
  (if ignore_cwise
      (go label666))
```

```
          (if (or (= z_state noprog_state) (= z_state conv_state))
              (go label666))
          (if (and (= z_state unstable_state) (> cnt 1))
              (go label666))))
        (cond
         (incr_prec (setf incr_prec f2cl-lib:%false%)
          (setf y_prec_state (f2cl-lib:int-add y_prec_state 1))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                                   y_tail-%offset%)
                      (coerce 0.0d0 'f2cl-lib:complex16))
             label100004))))
        (setf prevnormdx normdx)
        (setf prev_dz_z dz_z)
        (cond
         ((< y_prec_state extra_y)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
              (zaxpy n (f2cl-lib:cmplx 1.0d0 0.0d0) dy 1
               (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                                     (1 j) ((1 ldy) (1 *))
                                     y-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3 var-4 var-5))
            (when var-0 (setf n var-0))))
         (t
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (zla_wwaddw n
               (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                                     (1 j) ((1 ldy) (1 *))
                                     y-%offset%)
               y_tail dy)
            (declare (ignore var-1 var-2 var-3))
            (when var-0 (setf n var-0)))))
      label100002))
  label666
   (if (= x_state working_state)
       (setf final_dx_x dx_x))
   (if (= z_state working_state)
       (setf final_dz_z dz_z))
   (cond
    ((>= n_norms 1)
     (setf (f2cl-lib:fref err_bnds_norm-%data% (j la_linrx_err_i)
                          ((1 nrhs) (1 *)) err_bnds_norm-%offset%)
             (/ final_dx_x (- 1 dxratmax)))))
   (cond
    ((>= n_norms 2)
     (setf (f2cl-lib:fref err_bnds_comp-%data% (j la_linrx_err_i)
                          ((1 nrhs) (1 *)) err_bnds_comp-%offset%)
```

```
                (/ final_dz_z (- 1 dzratmax)))))
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (zcopy n
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 1 res 1)
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf n var-0)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                 var-10 var-11 var-12)
                (zgbmv trans n n kl ku (f2cl-lib:cmplx -1.0d0 0.0d0) ab ldab
                 (f2cl-lib:array-slice y-%data% f2cl-lib:complex16 (1 j)
                                       ((1 ldy) (1 *)) y-%offset%)
                 1 (f2cl-lib:cmplx 1.0d0 0.0d0) res 1)
              (declare (ignore var-5 var-6 var-8 var-9 var-10 var-11 var-12))
              (when var-0 (setf trans var-0))
              (when var-1 (setf n var-1))
              (when var-2 (setf n var-2))
              (when var-3 (setf kl var-3))
              (when var-4 (setf ku var-4))
              (when var-7 (setf ldab var-7)))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref ayb-%data% (i) ((1 *)) ayb-%offset%)
                        (cabs1
                         (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                        b-%offset%)))
               label100005))
            (zla_gbamv trans_type n n kl ku 1.0d0 ab ldab
             (f2cl-lib:array-slice y-%data% f2cl-lib:complex16 (1 j)
                                   ((1 ldy) (1 *)) y-%offset%)
             1 1.0d0 ayb 1)
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                (zla_lin_berr n n 1 res ayb
                 (f2cl-lib:array-slice berr_out-%data% double-float (j)
                                       ((1 *)) berr_out-%offset%))
              (declare (ignore var-2 var-3 var-4 var-5))
              (when var-0 (setf n var-0))
              (when var-1 (setf n var-1)))
           label100000))
         (go end_label)
        end_label
         (return
          (values prec_type trans_type n kl ku nil nil ldab nil ldafb nil nil
                  nil nil nil nil nil nil nil nil nil nil nil nil nil nil nil
                  nil nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
```

```
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zla_gbrfsx_extended
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              fortran-to-lisp::logical
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (double-float)
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              fortran-to-lisp::logical
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::prec_type
                                              fortran-to-lisp::trans_type
                                              fortran-to-lisp::n
                                              fortran-to-lisp::kl
                                              fortran-to-lisp::ku nil nil
                                              fortran-to-lisp::ldab nil
```

```
                                        fortran-to-lisp::ldafb nil nil
                                        nil nil nil nil nil nil nil nil
                                        nil nil nil nil nil nil nil nil
                                        nil nil fortran-to-lisp::info)
                                      :calls
                                      '(fortran-to-lisp::zla_gbamv
                                        fortran-to-lisp::zgbtrs))))
```

———-

47.16 zla_gbrpvgrw

47.16.1 Axiom unit tests

— **zlagbrpvgrw.input** —

```
)set break resume
)sys rm -f zlagbrpvgrw.output
)spool zlagbrpvgrw.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

47.16.2 Axiom help page

— **zla_gbrpvgrw.help** —

```
=====================================================================
zla_gbrpvgrw examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
```

```
      double precision function zla_gbrpvgrw (N, KL, KU, NCOLS, AB, LDAB,
          AFB, LDAFB)
          ZLA_GBRPVGRW computes the reciprocal pivot growth factor
          norm(A)/norm(U) for a general banded matrix.

Function/Subroutine Documentation
   double precision function zla_gbrpvgrw (integer N, integer KL, integer KU,
       integer NCOLS, complex*16, dimension( ldab, * ) AB, integer LDAB,
       complex*16, dimension( ldafb, * ) AFB, integer LDAFB)
       ZLA_GBRPVGRW computes the reciprocal pivot growth factor
       norm(A)/norm(U) for a general banded matrix.

Purpose:
   ZLA_GBRPVGRW computes the reciprocal pivot growth factor
   norm(A)/norm(U). The "max absolute element" norm is used. If this is
   much less than 1, the stability of the LU factorization of the
   (equilibrated) matrix A could be poor. This also means that the
   solution X, estimated condition numbers, and error bounds could be
   unreliable.

Parameters:

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 NCOLS
   NCOLS is INTEGER
   The number of columns of the matrix A.  NCOLS >= 0.

 AB
   AB is COMPLEX*16 array, dimension (LDAB,N)
   On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
   The j-th column of A is stored in the j-th column of the
   array AB as follows:
   AB(KU+1+i-j,j) = A(i,j) for max(1,j-KU)<=i<=min(N,j+kl)

 LDAB
   LDAB is INTEGER
   The leading dimension of the array AB.  LDAB >= KL+KU+1.
```

```
 AFB
   AFB is COMPLEX*16 array, dimension (LDAFB,N)
   Details of the LU factorization of the band matrix A, as
   computed by ZGBTRF.  U is stored as an upper triangular
   band matrix with KL+KU superdiagonals in rows 1 to KL+KU+1,
   and the multipliers used during the factorization are stored
   in rows KL+KU+2 to 2*KL+KU+1.

 LDAFB
   LDAFB is INTEGER
   The leading dimension of the array AFB.  LDAFB >= 2*KL+KU+1.
```

———-

47.16.3 fortran code

— zla_gbrpvgrw.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION ZLA_GBRPVGRW( N, KL, KU, NCOLS, AB,
     $                                        LDAB, AFB, LDAFB )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            N, KL, KU, NCOLS, LDAB, LDAFB
*     ..
*     .. Array Arguments ..
      COMPLEX*16         AB( LDAB, * ), AFB( LDAFB, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J, KD
      DOUBLE PRECISION   AMAX, UMAX, RPVGRW
      COMPLEX*16         ZDUM
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, REAL, DIMAG
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
```

```
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      RPVGRW = 1.0D+0

      KD = KU + 1
      DO J = 1, NCOLS
         AMAX = 0.0D+0
         UMAX = 0.0D+0
         DO I = MAX( J-KU, 1 ), MIN( J+KL, N )
            AMAX = MAX( CABS1( AB( KD+I-J, J ) ), AMAX )
         END DO
         DO I = MAX( J-KU, 1 ), J
            UMAX = MAX( CABS1( AFB( KD+I-J, J ) ), UMAX )
         END DO
         IF ( UMAX /= 0.0D+0 ) THEN
            RPVGRW = MIN( AMAX / UMAX, RPVGRW )
         END IF
      END DO
      ZLA_GBRPVGRW = RPVGRW
      END
```

———-

47.16.4 lisp code

— zla_gbrpvgrw.lisp —

```
(defun zla_gbrpvgrw (n kl ku ncols ab ldab afb ldafb)
  (declare (type (array f2cl-lib:complex16 (*)) afb ab)
           (type (f2cl-lib:integer4) ldafb ldab ncols ku kl n))
  (f2cl-lib:with-multi-array-data
      ((ab f2cl-lib:complex16 ab-%data% ab-%offset%)
       (afb f2cl-lib:complex16 afb-%data% afb-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
        cabs1))
      (prog ((zdum #C(0.0d0 0.0d0)) (amax 0.0d0) (umax 0.0d0) (rpvgrw 0.0d0)
             (i 0) (j 0) (kd 0) (zla_gbrpvgrw 0.0d0))
        (declare (type (f2cl-lib:integer4) kd j i)
                 (type (double-float) zla_gbrpvgrw rpvgrw umax amax)
                 (type (f2cl-lib:complex16) zdum))
        (setf rpvgrw 1.0d0)
        (setf kd (f2cl-lib:int-add ku 1))
```

```
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j ncols) nil)
  (tagbody
    (setf amax 0.0d0)
    (setf umax 0.0d0)
    (f2cl-lib:fdo (i
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                    (the f2cl-lib:integer4 1))
                   (f2cl-lib:int-add i 1))
                  ((> i
                      (min
                       (the f2cl-lib:integer4 (f2cl-lib:int-add j kl))
                       (the f2cl-lib:integer4 n)))
                   nil)
      (tagbody
        (setf amax
                (max
                 (cabs1
                  (f2cl-lib:fref ab-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-add kd i) j)
                                  j)
                                 ((1 ldab) (1 *)) ab-%offset%))
                 amax))
       label100001))
    (f2cl-lib:fdo (i
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add j (f2cl-lib:int-sub ku)))
                    (the f2cl-lib:integer4 1))
                   (f2cl-lib:int-add i 1))
                  ((> i j) nil)
      (tagbody
        (setf umax
                (max
                 (cabs1
                  (f2cl-lib:fref afb-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-add kd i) j)
                                  j)
                                 ((1 ldafb) (1 *)) afb-%offset%))
                 umax))
       label100002))
    (cond
     ((f2cl-lib:f2cl/ umax (= 0.0d0))
      (setf rpvgrw (min (/ amax umax) rpvgrw))))
   label100000))
(setf zla_gbrpvgrw rpvgrw)
```

```
       end_label
        (return (values zla_gbrpvgrw nil nil nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zla_gbrpvgrw
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil)
                                            :calls 'nil)))
```

———-

47.17 zungbr

47.17.1 Axiom unit tests

— **zungbr.input** —

```
)set break resume
)sys rm -f zungbr.output
)spool zungbr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

47.17.2 Axiom help page

— zungbr.help —

```
==================================================================
zungbr examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zungbr (VECT, M, N, K, A, LDA, TAU, WORK, LWORK, INFO)
           ZUNGBR

Function/Subroutine Documentation
   subroutine zungbr (character VECT, integer M, integer N, integer K,
       complex*16, dimension( lda, * ) A, integer LDA, complex*16, dimension(
       * ) TAU, complex*16, dimension( * ) WORK, integer LWORK, integer INFO)
       ZUNGBR

Purpose:
   ZUNGBR generates one of the complex unitary matrices Q or P**H
   determined by ZGEBRD when reducing a complex matrix A to bidiagonal
   form: A = Q * B * P**H.  Q and P**H are defined as products of
   elementary reflectors H(i) or G(i) respectively.

   If VECT = 'Q', A is assumed to have been an M-by-K matrix, and Q
   is of order M:
   if m >= k, Q = H(1) H(2) . . . H(k) and ZUNGBR returns the first n
   columns of Q, where m >= n >= k;
   if m < k, Q = H(1) H(2) . . . H(m-1) and ZUNGBR returns Q as an
   M-by-M matrix.

   If VECT = 'P', A is assumed to have been a K-by-N matrix, and P**H
   is of order N:
   if k < n, P**H = G(k) . . . G(2) G(1) and ZUNGBR returns the first m
   rows of P**H, where n >= m >= k;
   if k >= n, P**H = G(n-1) . . . G(2) G(1) and ZUNGBR returns P**H as
   an N-by-N matrix.

Parameters:

 VECT
   VECT is CHARACTER*1
   Specifies whether the matrix Q or the matrix P**H is
   required, as defined in the transformation applied by ZGEBRD:
```

```
  = 'Q':  generate Q;
  = 'P':  generate P**H.

M
  M is INTEGER
  The number of rows of the matrix Q or P**H to be returned.
  M >= 0.

N
  N is INTEGER
  The number of columns of the matrix Q or P**H to be returned.
  N >= 0.
  If VECT = 'Q', M >= N >= min(M,K);
  if VECT = 'P', N >= M >= min(N,K).

K
  K is INTEGER
  If VECT = 'Q', the number of columns in the original M-by-K
  matrix reduced by ZGEBRD.
  If VECT = 'P', the number of rows in the original K-by-N
  matrix reduced by ZGEBRD.
  K >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the vectors which define the elementary reflectors,
  as returned by ZGEBRD.
  On exit, the M-by-N matrix Q or P**H.

LDA
  LDA is INTEGER
  The leading dimension of the array A. LDA >= M.

TAU
  TAU is COMPLEX*16 array, dimension
     (min(M,K)) if VECT = 'Q'
     (min(N,K)) if VECT = 'P'
  TAU(i) must contain the scalar factor of the elementary
  reflector H(i) or G(i), which determines Q or P**H, as
  returned by ZGEBRD in its array argument TAUQ or TAUP.

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK >= max(1,min(M,N)).
  For optimum performance LWORK >= min(M,N)*NB, where NB
  is the optimal blocksize.
```

```
  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

47.17.3 fortran code

— **zungbr.f** —

```
*  =====================================================================
      SUBROUTINE ZUNGBR( VECT, M, N, K, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          VECT
      INTEGER            INFO, K, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ZERO, ONE
      PARAMETER          ( ZERO = ( 0.0D+0, 0.0D+0 ),
     $                   ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, WANTQ
      INTEGER            I, IINFO, J, LWKOPT, MN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
```

```
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZUNGLQ, ZUNGQR
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      WANTQ = LSAME( VECT, 'Q' )
      MN = MIN( M, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.WANTQ .AND. .NOT.LSAME( VECT, 'P' ) ) THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 .OR. ( WANTQ .AND. ( N.GT.M .OR. N.LT.MIN( M,
     $         K ) ) ) .OR. ( .NOT.WANTQ .AND. ( M.GT.N .OR. M.LT.
     $         MIN( N, K ) ) ) ) THEN
         INFO = -3
      ELSE IF( K.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -6
      ELSE IF( LWORK.LT.MAX( 1, MN ) .AND. .NOT.LQUERY ) THEN
         INFO = -9
      END IF
*
      IF( INFO.EQ.0 ) THEN
         WORK( 1 ) = 1
         IF( WANTQ ) THEN
            IF( M.GE.K ) THEN
               CALL ZUNGQR( M, N, K, A, LDA, TAU, WORK, -1, IINFO )
            ELSE
               IF( M.GT.1 ) THEN
                  CALL ZUNGQR( M-1, M-1, M-1, A( 2, 2 ), LDA, TAU, WORK,
     $                         -1, IINFO )
               END IF
            END IF
         ELSE
            IF( K.LT.N ) THEN
               CALL ZUNGLQ( M, N, K, A, LDA, TAU, WORK, -1, IINFO )
            ELSE
               IF( N.GT.1 ) THEN
                  CALL ZUNGLQ( N-1, N-1, N-1, A( 2, 2 ), LDA, TAU, WORK,
     $                         -1, IINFO )
```

```
            END IF
         END IF
      END IF
      LWKOPT = WORK( 1 )
      LWKOPT = MAX (LWKOPT, MN)
   END IF
*
   IF( INFO.NE.0 ) THEN
      CALL XERBLA( 'ZUNGBR', -INFO )
      RETURN
   ELSE IF( LQUERY ) THEN
      WORK( 1 ) = LWKOPT
      RETURN
   END IF
*
*     Quick return if possible
*
   IF( M.EQ.0 .OR. N.EQ.0 ) THEN
      WORK( 1 ) = 1
      RETURN
   END IF
*
   IF( WANTQ ) THEN
*
*        Form Q, determined by a call to ZGEBRD to reduce an m-by-k
*        matrix
*
      IF( M.GE.K ) THEN
*
*           If m >= k, assume m >= n >= k
*
            CALL ZUNGQR( M, N, K, A, LDA, TAU, WORK, LWORK, IINFO )
*
      ELSE
*
*           If m < k, assume m = n
*
*           Shift the vectors which define the elementary reflectors one
*           column to the right, and set the first row and column of Q
*           to those of the unit matrix
*
            DO 20 J = M, 2, -1
               A( 1, J ) = ZERO
               DO 10 I = J + 1, M
                  A( I, J ) = A( I, J-1 )
   10          CONTINUE
   20       CONTINUE
            A( 1, 1 ) = ONE
            DO 30 I = 2, M
               A( I, 1 ) = ZERO
```

```
   30       CONTINUE
            IF( M.GT.1 ) THEN
*
*              Form Q(2:m,2:m)
*
               CALL ZUNGQR( M-1, M-1, M-1, A( 2, 2 ), LDA, TAU, WORK,
     $                      LWORK, IINFO )
            END IF
         END IF
      ELSE
*
*        Form P**H, determined by a call to ZGEBRD to reduce a k-by-n
*        matrix
*
         IF( K.LT.N ) THEN
*
*           If k < n, assume k <= m <= n
*
            CALL ZUNGLQ( M, N, K, A, LDA, TAU, WORK, LWORK, IINFO )
*
         ELSE
*
*           If k >= n, assume m = n
*
*           Shift the vectors which define the elementary reflectors one
*           row downward, and set the first row and column of P**H to
*           those of the unit matrix
*
            A( 1, 1 ) = ONE
            DO 40 I = 2, N
               A( I, 1 ) = ZERO
   40       CONTINUE
            DO 60 J = 2, N
               DO 50 I = J - 1, 2, -1
                  A( I, J ) = A( I-1, J )
   50          CONTINUE
               A( 1, J ) = ZERO
   60       CONTINUE
            IF( N.GT.1 ) THEN
*
*              Form P**H(2:n,2:n)
*
               CALL ZUNGLQ( N-1, N-1, N-1, A( 2, 2 ), LDA, TAU, WORK,
     $                      LWORK, IINFO )
            END IF
         END IF
      END IF
      WORK( 1 ) = LWKOPT
      RETURN
*
```

```
*     End of ZUNGBR
*
      END
```

———-

47.17.4 lisp code

— **zungbr.lisp** —

```
(let* ((zero (f2cl-lib:cmplx 0.0d0 0.0d0)) (one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) zero)
           (type (f2cl-lib:complex16) one)
           (ignorable zero one))
  (defun zungbr (vect m n k a lda tau work lwork info)
    (declare (type (array f2cl-lib:complex16 (*)) work tau a)
             (type (f2cl-lib:integer4) info lwork lda k n m)
             (type (string *) vect))
    (f2cl-lib:with-multi-array-data
        ((vect character vect-%data% vect-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%))
      (prog ((i 0) (iinfo 0) (j 0) (lwkopt 0) (mn 0) (lquery nil) (wantq nil))
        (declare (type (f2cl-lib:integer4) i iinfo j lwkopt mn)
                 (type f2cl-lib:logical lquery wantq))
        (setf info 0)
        (setf wantq
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame vect "Q")
                  (declare (ignore var-1))
                  (when var-0 (setf vect var-0))
                  ret-val))
        (setf mn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond
         ((and (not wantq)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame vect "P")
                  (declare (ignore var-1))
                  (when var-0 (setf vect var-0))
                  ret-val)))
          (setf info -1))
         ((< m 0) (setf info -2))
         ((or (< n 0)
              (and wantq
                   (or (> n m)
```

```
                  (< n
                     (min (the f2cl-lib:integer4 m)
                          (the f2cl-lib:integer4 k)))))
        (and (not wantq)
             (or (> m n)
                 (< m
                    (min (the f2cl-lib:integer4 n)
                         (the f2cl-lib:integer4 k))))))
  (setf info -3))
 ((< k 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -6))
 ((and
   (< lwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 mn)))
   (not lquery))
  (setf info -9)))
(cond
 ((= info 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce 1 'f2cl-lib:complex16))
  (cond
   (wantq
    (cond
     ((>= m k)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (zungqr m n k a lda tau work -1 iinfo)
        (declare (ignore var-3 var-5 var-6 var-7))
        (when var-0 (setf m var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf k var-2))
        (when var-4 (setf lda var-4))
        (when var-8 (setf iinfo var-8))))
     (t
      (cond
       ((> m 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (zungqr (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
             (f2cl-lib:int-sub m 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (2 2)
                                   ((1 lda) (1 *)) a-%offset%)
             lda tau work -1 iinfo)
          (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
          (when var-4 (setf lda var-4))
          (when var-8 (setf iinfo var-8))))))))
   (t
    (cond
     ((< k n)
      (multiple-value-bind
```

```
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (zunglq m n k a lda tau work -1 iinfo)
          (declare (ignore var-3 var-5 var-6 var-7))
          (when var-0 (setf m var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf k var-2))
          (when var-4 (setf lda var-4))
          (when var-8 (setf iinfo var-8))))
       (t
        (cond
         ((> n 1)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (zunglq (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
               (f2cl-lib:int-sub n 1)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (2 2)
                                     ((1 lda) (1 *)) a-%offset%)
               lda tau work -1 iinfo)
            (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
            (when var-4 (setf lda var-4))
            (when var-8 (setf iinfo var-8)))))))))
  (setf lwkopt
          (f2cl-lib:int
           (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkopt)
               (the f2cl-lib:integer4 mn)))))
(cond
 ((/= info 0) (xerbla "ZUNGBR" (f2cl-lib:int-sub info)) (go end_label))
 (lquery
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwkopt 'f2cl-lib:complex16))
  (go end_label)))
(cond
 ((or (= m 0) (= n 0))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce 1 'f2cl-lib:complex16))
  (go end_label)))
(cond
 (wantq
  (cond
   ((>= m k)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (zungqr m n k a lda tau work lwork iinfo)
      (declare (ignore var-3 var-5 var-6))
      (when var-0 (setf m var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf k var-2))
      (when var-4 (setf lda var-4))
```

```
        (when var-7 (setf lwork var-7))
        (when var-8 (setf iinfo var-8))))
      (t
       (f2cl-lib:fdo (j m (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     ((> j 2) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *)) a-%offset%)
                  zero)
          (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%)
                      (f2cl-lib:fref a-%data% (i (f2cl-lib:int-sub j 1))
                                      ((1 lda) (1 *)) a-%offset%))
             label10))
         label20))
       (setf (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)
               one)
       (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i 1) ((1 lda) (1 *)) a-%offset%)
                  zero)
         label30))
       (cond
        ((> m 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (zungqr (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
             (f2cl-lib:int-sub m 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (2 2)
                                    ((1 lda) (1 *)) a-%offset%)
             lda tau work lwork iinfo)
          (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6))
          (when var-4 (setf lda var-4))
          (when var-7 (setf lwork var-7))
          (when var-8 (setf iinfo var-8))))))))
    (t
     (cond
      ((< k n)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
           (zunglq m n k a lda tau work lwork iinfo)
        (declare (ignore var-3 var-5 var-6))
        (when var-0 (setf m var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf k var-2))
        (when var-4 (setf lda var-4))
        (when var-7 (setf lwork var-7))
```

```
              (when var-8 (setf iinfo var-8))))
            (t
             (setf (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)
                     one)
             (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                           ((> i n) nil)
               (tagbody
                 (setf (f2cl-lib:fref a-%data% (i 1) ((1 lda) (1 *)) a-%offset%)
                         zero)
                label40))
             (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                                  (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                               ((> i 2) nil)
                   (tagbody
                     (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%)
                             (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) j)
                                            ((1 lda) (1 *)) a-%offset%))
                    label50))
                 (setf (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *)) a-%offset%)
                         zero)
                label60))
             (cond
              ((> n 1)
               (multiple-value-bind
                   (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                   (zunglq (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
                    (f2cl-lib:int-sub n 1)
                    (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (2 2)
                                          ((1 lda) (1 *)) a-%offset%)
                    lda tau work lwork iinfo)
                 (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6))
                 (when var-4 (setf lda var-4))
                 (when var-7 (setf lwork var-7))
                 (when var-8 (setf iinfo var-8)))))))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce lwkopt 'f2cl-lib:complex16))
        (go end_label)
       end_label
        (return (values vect m n k nil lda nil nil lwork info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zungbr
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
```

```
'((string)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::vect
  fortran-to-lisp::m
  fortran-to-lisp::n
  fortran-to-lisp::k nil
  fortran-to-lisp::lda nil nil
  fortran-to-lisp::lwork
  fortran-to-lisp::info)
:calls '(fortran-to-lisp::xerbla))))
```

Chapter 48

LAPACK - General Band Matrix, Auxilliary Routines, Double

48.1 dlangb

48.1.1 Axiom unit tests

— **dlangb.input** —

```
)set break resume
)sys rm -f dlangb.output
)spool dlangb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

48.1.2 Axiom help page

— **dlangb.help** —

```
====================================================================
```

```
dlangb examples
===================================================================

===================================================================
Man Page Details
===================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function dlangb (NORM, N, KL, KU, AB, LDAB, WORK)
           DLANGB returns the value of the 1-norm, Frobenius norm, infinity-
           norm, or the largest absolute value of any element of general band
           matrix.

Function/Subroutine Documentation
   double precision function dlangb (character NORM, integer N, integer KL,
       integer KU, double precision, dimension( ldab, * ) AB, integer LDAB,
       double precision, dimension( * ) WORK)
       DLANGB returns the value of the 1-norm, Frobenius norm, infinity-norm,
       or the largest absolute value of any element of general band matrix.

Purpose:
   DLANGB  returns the value of the one norm,  or the Frobenius norm, or
   the  infinity norm,  or the element of  largest absolute value  of an
   n by n band matrix  A,  with kl sub-diagonals and ku super-diagonals.

 Returns:

   DLANGB

       DLANGB = ( max(abs(A(i,j))), NORM = 'M' or 'm'
                (
                ( norm1(A),         NORM = '1', 'O' or 'o'
                (
                ( normI(A),         NORM = 'I' or 'i'
                (
                ( normF(A),         NORM = 'F', 'f', 'E' or 'e'

   where  norm1  denotes the  one norm of a matrix (maximum column sum),
   normI  denotes the  infinity norm  of a matrix  (maximum row sum) and
   normF  denotes the  Frobenius norm of a matrix (square root of sum of
   squares).  Note that  max(abs(A(i,j)))  is not a consistent matrix norm.

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies the value to be returned in DLANGB as described above.

 N
```

```
  N is INTEGER
  The order of the matrix A.  N >= 0.  When N = 0, DLANGB is set to zero.

KL
  KL is INTEGER
  The number of sub-diagonals of the matrix A.  KL >= 0.

KU
  KU is INTEGER
  The number of super-diagonals of the matrix A.  KU >= 0.

AB
  AB is DOUBLE PRECISION array, dimension (LDAB,N)
  The band matrix A, stored in rows 1 to KL+KU+1.  The j-th
  column of A is stored in the j-th column of the array AB as follows:
  AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(n,j+kl).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK)),
  where LWORK >= N when NORM = 'I'; otherwise, WORK is not referenced.
```

———-

48.1.3 fortran code

— dlangb.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION DLANGB( NORM, N, KL, KU, AB, LDAB,
     $                 WORK )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            KL, KU, LDAB, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   AB( LDAB, * ), WORK( * )
*     ..
```

```
*
* =====================================================================
*
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, K, L
      DOUBLE PRECISION   SCALE, SUM, VALUE, TEMP
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASSQ
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, DISNAN
      EXTERNAL           LSAME, DISNAN
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      IF( N.EQ.0 ) THEN
         VALUE = ZERO
      ELSE IF( LSAME( NORM, 'M' ) ) THEN
*
*        Find max(abs(A(i,j))).
*
         VALUE = ZERO
         DO 20 J = 1, N
            DO 10 I = MAX( KU+2-J, 1 ), MIN( N+KU+1-J, KL+KU+1 )
               TEMP = ABS( AB( I, J ) )
               IF( VALUE.LT.TEMP .OR. DISNAN( TEMP ) ) VALUE = TEMP
   10       CONTINUE
   20    CONTINUE
      ELSE IF( ( LSAME( NORM, 'O' ) ) .OR. ( NORM.EQ.'1' ) ) THEN
*
*        Find norm1(A).
*
         VALUE = ZERO
         DO 40 J = 1, N
            SUM = ZERO
            DO 30 I = MAX( KU+2-J, 1 ), MIN( N+KU+1-J, KL+KU+1 )
               SUM = SUM + ABS( AB( I, J ) )
   30       CONTINUE
            IF( VALUE.LT.SUM .OR. DISNAN( SUM ) ) VALUE = SUM
   40    CONTINUE
      ELSE IF( LSAME( NORM, 'I' ) ) THEN
```

```
*
*        Find normI(A).
*
         DO 50 I = 1, N
            WORK( I ) = ZERO
   50    CONTINUE
         DO 70 J = 1, N
            K = KU + 1 - J
            DO 60 I = MAX( 1, J-KU ), MIN( N, J+KL )
               WORK( I ) = WORK( I ) + ABS( AB( K+I, J ) )
   60       CONTINUE
   70    CONTINUE
         VALUE = ZERO
         DO 80 I = 1, N
            TEMP = WORK( I )
            IF( VALUE.LT.TEMP .OR. DISNAN( TEMP ) ) VALUE = TEMP
   80    CONTINUE
      ELSE IF( ( LSAME( NORM, 'F' ) ) .OR. ( LSAME( NORM, 'E' ) ) ) THEN
*
*        Find normF(A).
*
         SCALE = ZERO
         SUM = ONE
         DO 90 J = 1, N
            L = MAX( 1, J-KU )
            K = KU + 1 - J + L
            CALL DLASSQ( MIN( N, J+KL )-L+1, AB( K, J ), 1, SCALE, SUM )
   90    CONTINUE
         VALUE = SCALE*SQRT( SUM )
      END IF
*
      DLANGB = VALUE
      RETURN
*
*     End of DLANGB
*
      END
```

———-

48.1.4 lisp code

— dlangb.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
```

```
(defun dlangb (norm n kl ku ab ldab work)
  (declare (type (array double-float (*)) work ab)
           (type (f2cl-lib:integer4) ldab ku kl n)
           (type (string *) norm))
  (f2cl-lib:with-multi-array-data
      ((norm character norm-%data% norm-%offset%)
       (ab double-float ab-%data% ab-%offset%)
       (work double-float work-%data% work-%offset%))
    (prog ((scale 0.0d0) (sum 0.0d0) (value 0.0d0) (temp 0.0d0) (i 0) (j 0)
           (k 0) (l 0) (dlangb 0.0d0))
      (declare (type (f2cl-lib:integer4) i j k l)
               (type (double-float) scale sum value temp dlangb))
      (cond ((= n 0) (setf value zero))
            ((multiple-value-bind (ret-val var-0 var-1)
                 (lsame norm "M")
               (declare (ignore var-1))
               (when var-0 (setf norm var-0))
               ret-val)
             (setf value zero)
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (f2cl-lib:fdo (i
                                (max
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add ku 2
                                                        (f2cl-lib:int-sub
                                                         j)))
                                 (the f2cl-lib:integer4 1))
                                (f2cl-lib:int-add i 1))
                               ((> i
                                   (min
                                    (the f2cl-lib:integer4
                                         (f2cl-lib:int-add n ku 1
                                                           (f2cl-lib:int-sub
                                                            j)))
                                    (the f2cl-lib:integer4
                                         (f2cl-lib:int-add kl ku 1))))
                                nil)
                   (tagbody
                     (setf temp
                             (abs
                              (f2cl-lib:fref ab-%data% (i j) ((1 ldab) (1 *))
                                             ab-%offset%)))
                     (if (or (< value temp)
                             (multiple-value-bind (ret-val var-0)
                                 (disnan temp)
                               (declare (ignore))
                               (setf temp var-0)
                               ret-val))
```

```
                (setf value temp))
            label10))
        label20)))
    ((or
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame norm "O")
        (declare (ignore var-1))
        (when var-0 (setf norm var-0))
        ret-val)
      (f2cl-lib:fstring-= norm "1"))
     (setf value zero)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf sum zero)
         (f2cl-lib:fdo (i
                        (max
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add ku 2
                                                (f2cl-lib:int-sub
                                                 j)))
                         (the f2cl-lib:integer4 1))
                        (f2cl-lib:int-add i 1))
                       ((> i
                           (min
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add n ku 1
                                                   (f2cl-lib:int-sub
                                                    j)))
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add kl ku 1))))
                        nil)
           (tagbody
             (setf sum
                     (+ sum
                        (abs
                         (f2cl-lib:fref ab-%data% (i j)
                                        ((1 ldab) (1 *))
                                        ab-%offset%))))
            label30))
         (if (or (< value sum)
                 (multiple-value-bind (ret-val var-0)
                     (disnan sum)
                   (declare (ignore))
                   (setf sum var-0)
                   ret-val))
             (setf value sum))
        label40)))
    ((multiple-value-bind (ret-val var-0 var-1)
         (lsame norm "I")
```

```
  (declare (ignore var-1))
  (when var-0 (setf norm var-0))
  ret-val)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            zero)
   label50))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf k (f2cl-lib:int-sub (f2cl-lib:int-add ku 1) j))
    (f2cl-lib:fdo (i
                   (max (the f2cl-lib:integer4 1)
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add j
                                               (f2cl-lib:int-sub
                                                ku))))
                   (f2cl-lib:int-add i 1))
                  ((> i
                      (min (the f2cl-lib:integer4 n)
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add j kl))))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                             work-%offset%)
                (+
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 (abs
                  (f2cl-lib:fref ab-%data%
                                 ((f2cl-lib:int-add k i) j)
                                 ((1 ldab) (1 *))
                                 ab-%offset%))))
       label60))
   label70))
(setf value zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf temp
            (f2cl-lib:fref work-%data% (i) ((1 *))
                           work-%offset%))
    (if (or (< value temp)
            (multiple-value-bind (ret-val var-0)
                (disnan temp)
              (declare (ignore))
              (setf temp var-0)
```

```
                          ret-val))
                     (setf value temp))
                  label80)))
              ((or
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame norm "F")
                  (declare (ignore var-1))
                  (when var-0 (setf norm var-0))
                  ret-val)
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame norm "E")
                  (declare (ignore var-1))
                  (when var-0 (setf norm var-0))
                  ret-val))
               (setf scale zero) (setf sum one)
               (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                             ((> j n) nil)
                 (tagbody
                   (setf l
                           (max (the f2cl-lib:integer4 1)
                                (the f2cl-lib:integer4
                                     (f2cl-lib:int-sub j ku))))
                   (setf k
                           (f2cl-lib:int-add
                            (f2cl-lib:int-sub (f2cl-lib:int-add ku 1) j) l))
                   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                       (dlassq
                        (f2cl-lib:int-add
                         (f2cl-lib:int-sub
                          (min (the f2cl-lib:integer4 n)
                               (the f2cl-lib:integer4 (f2cl-lib:int-add j kl)))
                          l)
                         1)
                        (f2cl-lib:array-slice ab-%data% double-float (k j)
                                              ((1 ldab) (1 *)) ab-%offset%)
                        1 scale sum)
                     (declare (ignore var-0 var-1 var-2))
                     (when var-3 (setf scale var-3))
                     (when var-4 (setf sum var-4)))
                  label90))
               (setf value (* scale (f2cl-lib:fsqrt sum)))))
        (setf dlangb value)
        (go end_label)
       end_label
        (return (values dlangb norm nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlangb
```

```
        fortran-to-lisp::*f2cl-function-info*)
    (fortran-to-lisp::make-f2cl-finfo :arg-types
                                      '((string)
                                        (fortran-to-lisp::integer4)
                                        (fortran-to-lisp::integer4)
                                        (fortran-to-lisp::integer4)
                                        (array double-float (*))
                                        (fortran-to-lisp::integer4)
                                        (array double-float (*)))
                                      :return-values
                                      '(fortran-to-lisp::norm nil nil nil
                                        nil nil nil)
                                      :calls '(fortran-to-lisp::disnan))))
```

———-

48.2 dlaqgb

48.2.1 Axiom unit tests

— dlaqgb.input —

```
)set break resume
)sys rm -f dlaqgb.output
)spool dlaqgb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

48.2.2 Axiom help page

— dlaqgb.help —

```
======================================================================
dlaqgb examples
======================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlaqgb (M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND, AMAX,
           EQUED)
           DLAQGB scales a general band matrix, using row and column scaling
           factors computed by sgbequ.

Function/Subroutine Documentation
   subroutine dlaqgb (integer M, integer N, integer KL, integer KU, double
       precision, dimension( ldab, * ) AB, integer LDAB, double precision,
       dimension( * ) R, double precision, dimension( * ) C, double precision
       ROWCND, double precision COLCND, double precision AMAX, character
       EQUED)
       DLAQGB scales a general band matrix, using row and column scaling
       factors computed by sgbequ.

Purpose:
   DLAQGB equilibrates a general M by N band matrix A with KL
   subdiagonals and KU superdiagonals using the row and scaling factors
   in the vectors R and C.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 AB
   AB is DOUBLE PRECISION array, dimension (LDAB,N)
   On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
   The j-th column of A is stored in the j-th column of the
   array AB as follows:
   AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl)
```

```
   On exit, the equilibrated matrix, in the same storage format
   as A.  See EQUED for the form of the equilibrated matrix.

 LDAB
   LDAB is INTEGER
   The leading dimension of the array AB.  LDA >= KL+KU+1.

 R
   R is DOUBLE PRECISION array, dimension (M)
   The row scale factors for A.

 C
   C is DOUBLE PRECISION array, dimension (N)
   The column scale factors for A.

 ROWCND
   ROWCND is DOUBLE PRECISION
   Ratio of the smallest R(i) to the largest R(i).

 COLCND
   COLCND is DOUBLE PRECISION
   Ratio of the smallest C(i) to the largest C(i).

 AMAX
   AMAX is DOUBLE PRECISION
   Absolute value of largest matrix entry.

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done.
   = 'N':  No equilibration
   = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
   = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
   = 'B':  Both row and column equilibration, i.e., A has been
           replaced by diag(R) * A * diag(C).

Internal Parameters:

   THRESH is a threshold value used to decide if row or column scaling
   should be done based on the ratio of the row or column scaling
   factors.  If ROWCND < THRESH, row scaling is done, and if
   COLCND < THRESH, column scaling is done.

   LARGE and SMALL are threshold values used to decide if row scaling
   should be done based on the absolute size of the largest matrix
   element.  If AMAX > LARGE or AMAX < SMALL, row scaling is done.
```

48.2.3 fortran code

— dlaqgb.f —

```
*  =====================================================================
      SUBROUTINE DLAQGB( M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $                   AMAX, EQUED )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED
      INTEGER            KL, KU, LDAB, M, N
      DOUBLE PRECISION   AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   AB( LDAB, * ), C( * ), R( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, THRESH
      PARAMETER          ( ONE = 1.0D+0, THRESH = 0.1D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   CJ, LARGE, SMALL
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( M.LE.0 .OR. N.LE.0 ) THEN
         EQUED = 'N'
         RETURN
      END IF
*
*     Initialize LARGE and SMALL.
```

```
*
      SMALL = DLAMCH( 'Safe minimum' ) / DLAMCH( 'Precision' )
      LARGE = ONE / SMALL
*
      IF( ROWCND.GE.THRESH .AND. AMAX.GE.SMALL .AND. AMAX.LE.LARGE )
     $     THEN
*
*        No row scaling
*
         IF( COLCND.GE.THRESH ) THEN
*
*           No column scaling
*
            EQUED = 'N'
         ELSE
*
*           Column scaling
*
            DO 20 J = 1, N
               CJ = C( J )
               DO 10 I = MAX( 1, J-KU ), MIN( M, J+KL )
                  AB( KU+1+I-J, J ) = CJ*AB( KU+1+I-J, J )
   10          CONTINUE
   20       CONTINUE
            EQUED = 'C'
         END IF
      ELSE IF( COLCND.GE.THRESH ) THEN
*
*        Row scaling, no column scaling
*
         DO 40 J = 1, N
            DO 30 I = MAX( 1, J-KU ), MIN( M, J+KL )
               AB( KU+1+I-J, J ) = R( I )*AB( KU+1+I-J, J )
   30       CONTINUE
   40    CONTINUE
         EQUED = 'R'
      ELSE
*
*        Row and column scaling
*
         DO 60 J = 1, N
            CJ = C( J )
            DO 50 I = MAX( 1, J-KU ), MIN( M, J+KL )
               AB( KU+1+I-J, J ) = CJ*R( I )*AB( KU+1+I-J, J )
   50       CONTINUE
   60    CONTINUE
         EQUED = 'B'
      END IF
*
      RETURN
```

```
*
*     End of DLAQGB
*
      END
```

48.2.4 lisp code

— dlaqgb.lisp —

```
(let* ((one 1.0d0) (thresh 0.1d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.1d0 0.1d0) thresh)
           (ignorable one thresh))
  (defun dlaqgb (m n kl ku ab ldab r c rowcnd colcnd amax equed)
    (declare (type (string *) equed)
             (type (double-float) amax colcnd rowcnd)
             (type (array double-float (*)) c r ab)
             (type (f2cl-lib:integer4) ldab ku kl n m))
    (f2cl-lib:with-multi-array-data
        ((ab double-float ab-%data% ab-%offset%)
         (r double-float r-%data% r-%offset%)
         (c double-float c-%data% c-%offset%)
         (equed character equed-%data% equed-%offset%))
      (prog ((cj 0.0d0) (large 0.0d0) (small 0.0d0) (i 0) (j 0))
        (declare (type (double-float) cj large small)
                 (type (f2cl-lib:integer4) i j))
        (cond
         ((or (<= m 0) (<= n 0))
          (f2cl-lib:f2cl-set-string equed "N" (string 1)) (go end_label)))
        (setf small (/ (dlamch "Safe minimum") (dlamch "Precision")))
        (setf large (/ one small))
        (cond
         ((and (>= rowcnd thresh) (>= amax small) (<= amax large))
          (cond
           ((>= colcnd thresh) (f2cl-lib:f2cl-set-string equed "N" (string 1)))
           (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf cj (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
                (f2cl-lib:fdo (i
                               (max (the f2cl-lib:integer4 1)
                                    (the f2cl-lib:integer4
                                         (f2cl-lib:int-add j
                                                           (f2cl-lib:int-sub
                                                            ku))))
```

```
                        (f2cl-lib:int-add i 1))
                       ((> i
                           (min (the f2cl-lib:integer4 m)
                                (the f2cl-lib:integer4
                                     (f2cl-lib:int-add j kl))))
                        nil)
            (tagbody
              (setf (f2cl-lib:fref ab-%data%
                                   ((f2cl-lib:int-sub
                                     (f2cl-lib:int-add ku 1 i) j)
                                    j)
                                   ((1 ldab) (1 *)) ab-%offset%)
                    (* cj
                       (f2cl-lib:fref ab-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add ku 1 i) j)
                                       j)
                                      ((1 ldab) (1 *)) ab-%offset%)))
             label10))
          label20))
      (f2cl-lib:f2cl-set-string equed "C" (string 1)))))
    ((>= colcnd thresh)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i
                       (max (the f2cl-lib:integer4 1)
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add j
                                                   (f2cl-lib:int-sub
                                                    ku))))
                       (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the f2cl-lib:integer4 m)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add j kl))))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref ab-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-add ku 1 i) j)
                                  j)
                                 ((1 ldab) (1 *)) ab-%offset%)
                  (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                     (f2cl-lib:fref ab-%data%
                                    ((f2cl-lib:int-sub
                                      (f2cl-lib:int-add ku 1 i) j)
                                     j)
                                    ((1 ldab) (1 *)) ab-%offset%)))
           label30))
```

```
            label40))
          (f2cl-lib:f2cl-set-string equed "R" (string 1)))
         (t
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf cj (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
              (f2cl-lib:fdo (i
                             (max (the f2cl-lib:integer4 1)
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-add j
                                                         (f2cl-lib:int-sub
                                                          ku))))
                             (f2cl-lib:int-add i 1))
                            ((> i
                                (min (the f2cl-lib:integer4 m)
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add j kl))))
                             nil)
                (tagbody
                  (setf (f2cl-lib:fref ab-%data%
                                       ((f2cl-lib:int-sub
                                         (f2cl-lib:int-add ku 1 i) j)
                                        j)
                                       ((1 ldab) (1 *)) ab-%offset%)
                        (* cj (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                           (f2cl-lib:fref ab-%data%
                                          ((f2cl-lib:int-sub
                                            (f2cl-lib:int-add ku 1 i) j)
                                           j)
                                          ((1 ldab) (1 *)) ab-%offset%)))
                label50))
            label60))
          (f2cl-lib:f2cl-set-string equed "B" (string 1))))
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil nil nil nil nil nil equed))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlaqgb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
```

```
  (array double-float (*))
  (array double-float (*))
  (double-float) (double-float)
  (double-float) (string))
:return-values
'(nil nil nil nil nil nil nil nil
  nil nil nil
  fortran-to-lisp::equed)
:calls 'nil)))
```

———-

Chapter 49

LAPACK - General Band Matrix, Auxilliary Routines, Real

49.1 slangb

49.1.1 Axiom unit tests

— **slangb.input** —

```
)set break resume
)sys rm -f slangb.output
)spool slangb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

49.1.2 Axiom help page

— **slangb.help** —

```
==========================================================================
```

```
slangb examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       real function slangb (NORM, N, KL, KU, AB, LDAB, WORK)
           SLANGB returns the value of the 1-norm, Frobenius norm, infinity-
           norm, or the largest absolute value of any element of general band
           matrix.

Function/Subroutine Documentation
   real function slangb (character NORM, integer N, integer KL, integer KU,
       real, dimension( ldab, * ) AB, integer LDAB, real, dimension( * ) WORK)
       SLANGB returns the value of the 1-norm, Frobenius norm, infinity-norm,
       or the largest absolute value of any element of general band matrix.

Purpose:
   SLANGB  returns the value of the one norm,  or the Frobenius norm, or
   the  infinity norm,  or the element of  largest absolute value  of an
   n by n band matrix  A,  with kl sub-diagonals and ku super-diagonals.

 Returns:

   SLANGB

       SLANGB = ( max(abs(A(i,j))), NORM = 'M' or 'm'
                (
                ( norm1(A),         NORM = '1', 'O' or 'o'
                (
                ( normI(A),         NORM = 'I' or 'i'
                (
                ( normF(A),         NORM = 'F', 'f', 'E' or 'e'

   where  norm1  denotes the  one norm of a matrix (maximum column sum),
   normI  denotes the  infinity norm  of a matrix  (maximum row sum) and
   normF  denotes the  Frobenius norm of a matrix (square root of sum of
   squares).  Note that  max(abs(A(i,j)))  is not a consistent matrix norm.

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies the value to be returned in SLANGB as described above.

 N
   N is INTEGER
```

```
  The order of the matrix A.  N >= 0.  When N = 0, SLANGB is set to zero.

KL
  KL is INTEGER
  The number of sub-diagonals of the matrix A.  KL >= 0.

KU
  KU is INTEGER
  The number of super-diagonals of the matrix A.  KU >= 0.

AB
  AB is REAL array, dimension (LDAB,N)
  The band matrix A, stored in rows 1 to KL+KU+1.  The j-th
  column of A is stored in the j-th column of the array AB as follows:
  AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(n,j+kl).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK)),
  where LWORK >= N when NORM = 'I'; otherwise, WORK is not referenced.
```

———-

49.1.3 fortran code

— slangb.f —

```
*  =====================================================================
      REAL             FUNCTION SLANGB( NORM, N, KL, KU, AB, LDAB,
     $                 WORK )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            KL, KU, LDAB, N
*     ..
*     .. Array Arguments ..
      REAL               AB( LDAB, * ), WORK( * )
*     ..
*
```

```
* =====================================================================
*
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, K, L
      REAL               SCALE, SUM, VALUE, TEMP
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLASSQ
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, SISNAN
      EXTERNAL           LSAME, SISNAN
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      IF( N.EQ.0 ) THEN
         VALUE = ZERO
      ELSE IF( LSAME( NORM, 'M' ) ) THEN
*
*        Find max(abs(A(i,j))).
*
         VALUE = ZERO
         DO 20 J = 1, N
            DO 10 I = MAX( KU+2-J, 1 ), MIN( N+KU+1-J, KL+KU+1 )
               TEMP = ABS( AB( I, J ) )
               IF( VALUE.LT.TEMP .OR. SISNAN( TEMP ) ) VALUE = TEMP
   10       CONTINUE
   20    CONTINUE
      ELSE IF( ( LSAME( NORM, 'O' ) ) .OR. ( NORM.EQ.'1' ) ) THEN
*
*        Find norm1(A).
*
         VALUE = ZERO
         DO 40 J = 1, N
            SUM = ZERO
            DO 30 I = MAX( KU+2-J, 1 ), MIN( N+KU+1-J, KL+KU+1 )
               SUM = SUM + ABS( AB( I, J ) )
   30       CONTINUE
            IF( VALUE.LT.SUM .OR. SISNAN( SUM ) ) VALUE = SUM
   40    CONTINUE
      ELSE IF( LSAME( NORM, 'I' ) ) THEN
*
```

```
*        Find normI(A).
*
         DO 50 I = 1, N
            WORK( I ) = ZERO
   50    CONTINUE
         DO 70 J = 1, N
            K = KU + 1 - J
            DO 60 I = MAX( 1, J-KU ), MIN( N, J+KL )
               WORK( I ) = WORK( I ) + ABS( AB( K+I, J ) )
   60       CONTINUE
   70    CONTINUE
         VALUE = ZERO
         DO 80 I = 1, N
            TEMP = WORK( I )
            IF( VALUE.LT.TEMP .OR. SISNAN( TEMP ) ) VALUE = TEMP
   80    CONTINUE
      ELSE IF( ( LSAME( NORM, 'F' ) ) .OR. ( LSAME( NORM, 'E' ) ) ) THEN
*
*        Find normF(A).
*
         SCALE = ZERO
         SUM = ONE
         DO 90 J = 1, N
            L = MAX( 1, J-KU )
            K = KU + 1 - J + L
            CALL SLASSQ( MIN( N, J+KL )-L+1, AB( K, J ), 1, SCALE, SUM )
   90    CONTINUE
         VALUE = SCALE*SQRT( SUM )
      END IF
*
      SLANGB = VALUE
      RETURN
*
*     End of SLANGB
*
      END
```

———-

49.1.4 lisp code

— slangb.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun slangb (norm n kl ku ab ldab work)
```

```
(declare (type (array single-float (*)) work ab)
         (type (f2cl-lib:integer4) ldab ku kl n)
         (type (string *) norm))
(f2cl-lib:with-multi-array-data
    ((norm character norm-%data% norm-%offset%)
     (ab single-float ab-%data% ab-%offset%)
     (work single-float work-%data% work-%offset%))
  (prog ((scale 0.0) (sum 0.0) (value 0.0) (temp 0.0) (i 0) (j 0) (k 0)
         (l 0) (slangb 0.0))
    (declare (type (f2cl-lib:integer4) i j k l)
             (type (single-float) scale sum value temp slangb))
    (cond ((= n 0) (setf value zero))
          ((multiple-value-bind (ret-val var-0 var-1)
               (lsame norm "M")
             (declare (ignore var-1))
             (when var-0 (setf norm var-0))
             ret-val)
           (setf value zero)
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (f2cl-lib:fdo (i
                              (max
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add ku 2
                                                      (f2cl-lib:int-sub
                                                       j)))
                               (the f2cl-lib:integer4 1))
                              (f2cl-lib:int-add i 1))
                             ((> i
                                 (min
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-add n ku 1
                                                         (f2cl-lib:int-sub
                                                          j)))
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-add kl ku 1))))
                              nil)
                 (tagbody
                   (setf temp
                           (abs
                            (f2cl-lib:fref ab-%data% (i j) ((1 ldab) (1 *))
                                           ab-%offset%)))
                   (if (or (< value temp) (sisnan temp))
                       (setf value temp))
                  label10))
              label20)))
          ((or
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame norm "O")
```

```
    (declare (ignore var-1))
    (when var-0 (setf norm var-0))
    ret-val)
  (f2cl-lib:fstring-= norm "1"))
 (setf value zero)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf sum zero)
     (f2cl-lib:fdo (i
                    (max
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add ku 2
                                            (f2cl-lib:int-sub
                                             j)))
                     (the f2cl-lib:integer4 1))
                    (f2cl-lib:int-add i 1))
                   ((> i
                       (min
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add n ku 1
                                               (f2cl-lib:int-sub
                                                j)))
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add kl ku 1))))
                    nil)
       (tagbody
         (setf sum
                 (+ sum
                    (abs
                     (f2cl-lib:fref ab-%data% (i j)
                                    ((1 ldab) (1 *))
                                    ab-%offset%))))
        label30))
     (if (or (< value sum) (sisnan sum))
         (setf value sum))
    label40)))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame norm "I")
   (declare (ignore var-1))
   (when var-0 (setf norm var-0))
   ret-val)
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
             zero)
    label50))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
```

```
      (tagbody
        (setf k (f2cl-lib:int-sub (f2cl-lib:int-add ku 1) j))
        (f2cl-lib:fdo (i
                       (max (the f2cl-lib:integer4 1)
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add j
                                                   (f2cl-lib:int-sub
                                                    ku))))
                       (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the f2cl-lib:integer4 n)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add j kl))))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)
                  (+
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                   (abs
                    (f2cl-lib:fref ab-%data%
                                   ((f2cl-lib:int-add k i) j)
                                   ((1 ldab) (1 *))
                                   ab-%offset%))))
           label60))
       label70))
    (setf value zero)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf temp
                (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%))
        (if (or (< value temp) (sisnan temp))
            (setf value temp))
       label80)))
   ((or
     (multiple-value-bind (ret-val var-0 var-1)
         (lsame norm "F")
       (declare (ignore var-1))
       (when var-0 (setf norm var-0))
       ret-val)
     (multiple-value-bind (ret-val var-0 var-1)
         (lsame norm "E")
       (declare (ignore var-1))
       (when var-0 (setf norm var-0))
       ret-val))
    (setf scale zero) (setf sum one)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                    ((> j n) nil)
            (tagbody
              (setf l
                      (max (the f2cl-lib:integer4 1)
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-sub j ku))))
              (setf k
                      (f2cl-lib:int-add
                       (f2cl-lib:int-sub (f2cl-lib:int-add ku 1) j) l))
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (slassq
                   (f2cl-lib:int-add
                    (f2cl-lib:int-sub
                     (min (the f2cl-lib:integer4 n)
                          (the f2cl-lib:integer4 (f2cl-lib:int-add j kl)))
                     l)
                    1)
                   (f2cl-lib:array-slice ab-%data% single-float (k j)
                                         ((1 ldab) (1 *)) ab-%offset%)
                   1 scale sum)
                (declare (ignore var-0 var-1 var-2))
                (when var-3 (setf scale var-3))
                (when var-4 (setf sum var-4)))
             label90))
          (setf value (* scale (f2cl-lib:fsqrt sum)))))
     (setf slangb value)
     (go end_label)
    end_label
     (return (values slangb norm nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slangb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*)))
                                            :return-values
                                            '(fortran-to-lisp::norm nil nil nil
                                              nil nil nil)
                                            :calls '(fortran-to-lisp::sisnan))))
```

———-

49.2 slaqgb

49.2.1 Axiom unit tests

— **slaqgb.input** —

```
)set break resume
)sys rm -f slaqgb.output
)spool slaqgb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

49.2.2 Axiom help page

— **slaqgb.help** —

```
====================================================================
slaqgb examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slaqgb (M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND, AMAX,
           EQUED)
           SLAQGB scales a general band matrix, using row and column scaling
           factors computed by sgbequ.

Function/Subroutine Documentation
   subroutine slaqgb (integer M, integer N, integer KL, integer KU, real,
       dimension( ldab, * ) AB, integer LDAB, real, dimension( * ) R, real,
       dimension( * ) C, real ROWCND, real COLCND, real AMAX, character EQUED)
       SLAQGB scales a general band matrix, using row and column scaling
```

```
      factors computed by sgbequ.

Purpose:
   SLAQGB equilibrates a general M by N band matrix A with KL
   subdiagonals and KU superdiagonals using the row and scaling factors
   in the vectors R and C.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 AB
   AB is REAL array, dimension (LDAB,N)
   On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
   The j-th column of A is stored in the j-th column of the
   array AB as follows:
   AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl)

   On exit, the equilibrated matrix, in the same storage format
   as A.  See EQUED for the form of the equilibrated matrix.

 LDAB
   LDAB is INTEGER
   The leading dimension of the array AB.  LDA >= KL+KU+1.

 R
   R is REAL array, dimension (M)
   The row scale factors for A.

 C
   C is REAL array, dimension (N)
   The column scale factors for A.

 ROWCND
   ROWCND is REAL
   Ratio of the smallest R(i) to the largest R(i).
```

```
 COLCND
   COLCND is REAL
   Ratio of the smallest C(i) to the largest C(i).

 AMAX
   AMAX is REAL
   Absolute value of largest matrix entry.

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done.
   = 'N':  No equilibration
   = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
   = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
   = 'B':  Both row and column equilibration, i.e., A has been
           replaced by diag(R) * A * diag(C).

Internal Parameters:

   THRESH is a threshold value used to decide if row or column scaling
   should be done based on the ratio of the row or column scaling
   factors.  If ROWCND < THRESH, row scaling is done, and if
   COLCND < THRESH, column scaling is done.

   LARGE and SMALL are threshold values used to decide if row scaling
   should be done based on the absolute size of the largest matrix
   element.  If AMAX > LARGE or AMAX < SMALL, row scaling is done.
```

49.2.3 fortran code

— slaqgb.f —

```
*  =====================================================================
      SUBROUTINE SLAQGB( M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $                   AMAX, EQUED )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED
      INTEGER            KL, KU, LDAB, M, N
```

```
      REAL               AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      REAL               AB( LDAB, * ), C( * ), R( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, THRESH
      PARAMETER          ( ONE = 1.0E+0, THRESH = 0.1E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               CJ, LARGE, SMALL
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( M.LE.0 .OR. N.LE.0 ) THEN
         EQUED = 'N'
         RETURN
      END IF
*
*     Initialize LARGE and SMALL.
*
      SMALL = SLAMCH( 'Safe minimum' ) / SLAMCH( 'Precision' )
      LARGE = ONE / SMALL
*
      IF( ROWCND.GE.THRESH .AND. AMAX.GE.SMALL .AND. AMAX.LE.LARGE )
     $     THEN
*
*        No row scaling
*
         IF( COLCND.GE.THRESH ) THEN
*
*           No column scaling
*
            EQUED = 'N'
         ELSE
*
*           Column scaling
```

```
*
            DO 20 J = 1, N
               CJ = C( J )
               DO 10 I = MAX( 1, J-KU ), MIN( M, J+KL )
                  AB( KU+1+I-J, J ) = CJ*AB( KU+1+I-J, J )
   10          CONTINUE
   20       CONTINUE
            EQUED = 'C'
         END IF
      ELSE IF( COLCND.GE.THRESH ) THEN
*
*        Row scaling, no column scaling
*
         DO 40 J = 1, N
            DO 30 I = MAX( 1, J-KU ), MIN( M, J+KL )
               AB( KU+1+I-J, J ) = R( I )*AB( KU+1+I-J, J )
   30       CONTINUE
   40    CONTINUE
         EQUED = 'R'
      ELSE
*
*        Row and column scaling
*
         DO 60 J = 1, N
            CJ = C( J )
            DO 50 I = MAX( 1, J-KU ), MIN( M, J+KL )
               AB( KU+1+I-J, J ) = CJ*R( I )*AB( KU+1+I-J, J )
   50       CONTINUE
   60    CONTINUE
         EQUED = 'B'
      END IF
*
      RETURN
*
*     End of SLAQGB
*
      END
```

———-

49.2.4 lisp code

— slaqgb.lisp —

```
(let* ((one 1.0) (thresh 0.1))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.1 0.1) thresh)
           (ignorable one thresh))
```

```
(defun slaqgb (m n kl ku ab ldab r c rowcnd colcnd amax equed)
  (declare (type (string *) equed)
           (type (single-float) amax colcnd rowcnd)
           (type (array single-float (*)) c r ab)
           (type (f2cl-lib:integer4) ldab ku kl n m))
  (f2cl-lib:with-multi-array-data
      ((ab single-float ab-%data% ab-%offset%)
       (r single-float r-%data% r-%offset%)
       (c single-float c-%data% c-%offset%)
       (equed character equed-%data% equed-%offset%))
    (prog ((cj 0.0) (large 0.0) (small 0.0) (i 0) (j 0))
      (declare (type (single-float) cj large small)
               (type (f2cl-lib:integer4) i j))
      (cond
       ((or (<= m 0) (<= n 0))
        (f2cl-lib:f2cl-set-string equed "N" (string 1)) (go end_label)))
      (setf small (/ (slamch "Safe minimum") (slamch "Precision")))
      (setf large (/ one small))
      (cond
       ((and (>= rowcnd thresh) (>= amax small) (<= amax large))
        (cond
         ((>= colcnd thresh) (f2cl-lib:f2cl-set-string equed "N" (string 1)))
         (t
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf cj (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
              (f2cl-lib:fdo (i
                             (max (the f2cl-lib:integer4 1)
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-add j
                                                         (f2cl-lib:int-sub
                                                          ku))))
                             (f2cl-lib:int-add i 1))
                            ((> i
                                (min (the f2cl-lib:integer4 m)
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add j kl))))
                             nil)
                (tagbody
                  (setf (f2cl-lib:fref ab-%data%
                                       ((f2cl-lib:int-sub
                                         (f2cl-lib:int-add ku 1 i) j)
                                        j)
                                       ((1 ldab) (1 *)) ab-%offset%)
                        (* cj
                           (f2cl-lib:fref ab-%data%
                                          ((f2cl-lib:int-sub
                                            (f2cl-lib:int-add ku 1 i) j)
                                           j)
```

```
                                        ((1 ldab) (1 *)) ab-%offset%)))
            label10))
        label20))
    (f2cl-lib:f2cl-set-string equed "C" (string 1)))))
  ((>= colcnd thresh)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i
                     (max (the f2cl-lib:integer4 1)
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add j
                                                 (f2cl-lib:int-sub
                                                  ku))))
                     (f2cl-lib:int-add i 1))
                    ((> i
                        (min (the f2cl-lib:integer4 m)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add j kl))))
                     nil)
       (tagbody
         (setf (f2cl-lib:fref ab-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add ku 1 i) j)
                               j)
                              ((1 ldab) (1 *)) ab-%offset%)
                 (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                    (f2cl-lib:fref ab-%data%
                                   ((f2cl-lib:int-sub
                                     (f2cl-lib:int-add ku 1 i) j)
                                    j)
                                   ((1 ldab) (1 *)) ab-%offset%)))
        label30))
    label40))
   (f2cl-lib:f2cl-set-string equed "R" (string 1)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
    (tagbody
      (setf cj (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
      (f2cl-lib:fdo (i
                     (max (the f2cl-lib:integer4 1)
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add j
                                                 (f2cl-lib:int-sub
                                                  ku))))
                     (f2cl-lib:int-add i 1))
                    ((> i
                        (min (the f2cl-lib:integer4 m)
                             (the f2cl-lib:integer4
```

```
                                        (f2cl-lib:int-add j kl))))
                              nil)
                    (tagbody
                      (setf (f2cl-lib:fref ab-%data%
                                           ((f2cl-lib:int-sub
                                             (f2cl-lib:int-add ku 1 i) j)
                                            j)
                                           ((1 ldab) (1 *)) ab-%offset%)
                            (* cj (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                               (f2cl-lib:fref ab-%data%
                                              ((f2cl-lib:int-sub
                                                (f2cl-lib:int-add ku 1 i) j)
                                               j)
                                              ((1 ldab) (1 *)) ab-%offset%)))
                     label50))
                  label60))
              (f2cl-lib:f2cl-set-string equed "B" (string 1))))
            (go end_label)
           end_label
            (return (values nil nil nil nil nil nil nil nil nil nil nil equed))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slaqgb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (single-float) (string))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
                                              nil nil nil
                                              fortran-to-lisp::equed)
                                            :calls 'nil)))
```

———-

Chapter 50

LAPACK - General Band Matrix, Auxilliary Routines, Complex

50.1 clangb

50.1.1 Axiom unit tests

— **clangb.input** —

```
)set break resume
)sys rm -f clangb.output
)spool clangb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

50.1.2 Axiom help page

— **clangb.help** —

```
==========================================================================
```

```
clangb examples
===================================================================

===================================================================
Man Page Details
===================================================================

SYNOPSIS
   Functions/Subroutines
       real function clangb (NORM, N, KL, KU, AB, LDAB, WORK)
           CLANGB returns the value of the 1-norm, Frobenius norm, infinity-
           norm, or the largest absolute value of any element of general band
           matrix.

Function/Subroutine Documentation
   real function clangb (character NORM, integer N, integer KL, integer KU,
       complex, dimension( ldab, * ) AB, integer LDAB, real, dimension( * )
       WORK)
       CLANGB returns the value of the 1-norm, Frobenius norm, infinity-norm,
       or the largest absolute value of any element of general band matrix.

Purpose:
   CLANGB  returns the value of the one norm,  or the Frobenius norm, or
   the  infinity norm,  or the element of  largest absolute value  of an
   n by n band matrix  A,  with kl sub-diagonals and ku super-diagonals.

 Returns:

   CLANGB

       CLANGB = ( max(abs(A(i,j))), NORM = 'M' or 'm'
                (
                ( norm1(A),         NORM = '1', 'O' or 'o'
                (
                ( normI(A),         NORM = 'I' or 'i'
                (
                ( normF(A),         NORM = 'F', 'f', 'E' or 'e'

   where  norm1  denotes the  one norm of a matrix (maximum column sum),
   normI  denotes the  infinity norm  of a matrix  (maximum row sum) and
   normF  denotes the  Frobenius norm of a matrix (square root of sum of
   squares).  Note that  max(abs(A(i,j)))  is not a consistent matrix norm.

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies the value to be returned in CLANGB as described above.

 N
```

```
  N is INTEGER
  The order of the matrix A.  N >= 0.  When N = 0, CLANGB is set to zero.

KL
  KL is INTEGER
  The number of sub-diagonals of the matrix A.  KL >= 0.

KU
  KU is INTEGER
  The number of super-diagonals of the matrix A.  KU >= 0.

AB
  AB is COMPLEX array, dimension (LDAB,N)
  The band matrix A, stored in rows 1 to KL+KU+1.  The j-th
  column of A is stored in the j-th column of the array AB as follows:
  AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(n,j+kl).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK)),
  where LWORK >= N when NORM = 'I'; otherwise, WORK is not referenced.
```

———-

50.1.3 fortran code

— clangb.f —

```
*  =====================================================================
      REAL             FUNCTION CLANGB( NORM, N, KL, KU, AB, LDAB,
     $                 WORK )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            KL, KU, LDAB, N
*     ..
*     .. Array Arguments ..
      REAL               WORK( * )
      COMPLEX            AB( LDAB, * )
```

```
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, K, L
      REAL               SCALE, SUM, VALUE, TEMP
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, SISNAN
      EXTERNAL           LSAME, SISNAN
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLASSQ
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      IF( N.EQ.0 ) THEN
         VALUE = ZERO
      ELSE IF( LSAME( NORM, 'M' ) ) THEN
*
*        Find max(abs(A(i,j))).
*
         VALUE = ZERO
         DO 20 J = 1, N
            DO 10 I = MAX( KU+2-J, 1 ), MIN( N+KU+1-J, KL+KU+1 )
               TEMP = ABS( AB( I, J ) )
               IF( VALUE.LT.TEMP .OR. SISNAN( TEMP ) ) VALUE = TEMP
   10       CONTINUE
   20    CONTINUE
      ELSE IF( ( LSAME( NORM, 'O' ) ) .OR. ( NORM.EQ.'1' ) ) THEN
*
*        Find norm1(A).
*
         VALUE = ZERO
         DO 40 J = 1, N
            SUM = ZERO
            DO 30 I = MAX( KU+2-J, 1 ), MIN( N+KU+1-J, KL+KU+1 )
               SUM = SUM + ABS( AB( I, J ) )
   30       CONTINUE
            IF( VALUE.LT.SUM .OR. SISNAN( SUM ) ) VALUE = SUM
   40    CONTINUE
      ELSE IF( LSAME( NORM, 'I' ) ) THEN
```

```
*
*        Find normI(A).
*
         DO 50 I = 1, N
            WORK( I ) = ZERO
   50    CONTINUE
         DO 70 J = 1, N
            K = KU + 1 - J
            DO 60 I = MAX( 1, J-KU ), MIN( N, J+KL )
               WORK( I ) = WORK( I ) + ABS( AB( K+I, J ) )
   60       CONTINUE
   70    CONTINUE
         VALUE = ZERO
         DO 80 I = 1, N
            TEMP = WORK( I )
            IF( VALUE.LT.TEMP .OR. SISNAN( TEMP ) ) VALUE = TEMP
   80    CONTINUE
      ELSE IF( ( LSAME( NORM, 'F' ) ) .OR. ( LSAME( NORM, 'E' ) ) ) THEN
*
*        Find normF(A).
*
         SCALE = ZERO
         SUM = ONE
         DO 90 J = 1, N
            L = MAX( 1, J-KU )
            K = KU + 1 - J + L
            CALL CLASSQ( MIN( N, J+KL )-L+1, AB( K, J ), 1, SCALE, SUM )
   90    CONTINUE
         VALUE = SCALE*SQRT( SUM )
      END IF
*
      CLANGB = VALUE
      RETURN
*
*     End of CLANGB
*
      END
```

———-

50.1.4 lisp code

— clangb.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
```

```
(defun clangb (norm n kl ku ab ldab work)
  (declare (type (array single-float (*)) work)
           (type (array f2cl-lib:complex8 (*)) ab)
           (type (f2cl-lib:integer4) ldab ku kl n)
           (type (string *) norm))
  (f2cl-lib:with-multi-array-data
      ((norm character norm-%data% norm-%offset%)
       (ab f2cl-lib:complex8 ab-%data% ab-%offset%)
       (work single-float work-%data% work-%offset%))
    (prog ((scale 0.0) (sum 0.0) (value 0.0) (temp 0.0) (i 0) (j 0) (k 0)
           (l 0) (clangb 0.0))
      (declare (type (f2cl-lib:integer4) i j k l)
               (type (single-float) scale sum value temp clangb))
      (cond ((= n 0) (setf value zero))
            ((multiple-value-bind (ret-val var-0 var-1)
                 (lsame norm "M")
               (declare (ignore var-1))
               (when var-0 (setf norm var-0))
               ret-val)
             (setf value zero)
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (f2cl-lib:fdo (i
                                (max
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add ku 2
                                                        (f2cl-lib:int-sub
                                                         j)))
                                 (the f2cl-lib:integer4 1))
                                (f2cl-lib:int-add i 1))
                               ((> i
                                   (min
                                    (the f2cl-lib:integer4
                                         (f2cl-lib:int-add n ku 1
                                                           (f2cl-lib:int-sub
                                                            j)))
                                    (the f2cl-lib:integer4
                                         (f2cl-lib:int-add kl ku 1))))
                                nil)
                   (tagbody
                     (setf temp
                             (abs
                              (f2cl-lib:fref ab-%data% (i j) ((1 ldab) (1 *))
                                             ab-%offset%)))
                     (if (or (< value temp)
                             (multiple-value-bind (ret-val var-0)
                                 (sisnan temp)
                               (declare (ignore))
                               (when var-0 (setf temp var-0))
```

```
                    ret-val))
              (setf value temp))
         label10))
     label20)))
((or
  (multiple-value-bind (ret-val var-0 var-1)
      (lsame norm "O")
    (declare (ignore var-1))
    (when var-0 (setf norm var-0))
    ret-val)
  (f2cl-lib:fstring-= norm "1"))
 (setf value zero)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf sum zero)
     (f2cl-lib:fdo (i
                    (max
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add ku 2
                                            (f2cl-lib:int-sub
                                             j)))
                     (the f2cl-lib:integer4 1))
                    (f2cl-lib:int-add i 1))
                   ((> i
                       (min
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add n ku 1
                                               (f2cl-lib:int-sub
                                                j)))
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add kl ku 1))))
                    nil)
       (tagbody
         (setf sum
                 (+ sum
                    (abs
                     (f2cl-lib:fref ab-%data% (i j)
                                    ((1 ldab) (1 *))
                                    ab-%offset%))))
        label30))
     (if (or (< value sum)
             (multiple-value-bind (ret-val var-0)
                 (sisnan sum)
               (declare (ignore))
               (when var-0 (setf sum var-0))
               ret-val))
         (setf value sum))
     label40)))
((multiple-value-bind (ret-val var-0 var-1)
```

```
    (lsame norm "I")
  (declare (ignore var-1))
  (when var-0 (setf norm var-0))
  ret-val)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            zero)
   label50))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf k (f2cl-lib:int-sub (f2cl-lib:int-add ku 1) j))
    (f2cl-lib:fdo (i
                   (max (the f2cl-lib:integer4 1)
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add j
                                               (f2cl-lib:int-sub
                                                ku))))
                   (f2cl-lib:int-add i 1))
                  ((> i
                      (min (the f2cl-lib:integer4 n)
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add j kl))))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                             work-%offset%)
                (+
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 (abs
                  (f2cl-lib:fref ab-%data%
                                 ((f2cl-lib:int-add k i) j)
                                 ((1 ldab) (1 *))
                                 ab-%offset%))))
       label60))
   label70))
(setf value zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf temp
            (f2cl-lib:fref work-%data% (i) ((1 *))
                           work-%offset%))
    (if (or (< value temp)
            (multiple-value-bind (ret-val var-0)
                (sisnan temp)
              (declare (ignore))
```

```
                            (when var-0 (setf temp var-0))
                            ret-val))
                        (setf value temp))
                    label80)))
                ((or
                  (multiple-value-bind (ret-val var-0 var-1)
                      (lsame norm "F")
                    (declare (ignore var-1))
                    (when var-0 (setf norm var-0))
                    ret-val)
                  (multiple-value-bind (ret-val var-0 var-1)
                      (lsame norm "E")
                    (declare (ignore var-1))
                    (when var-0 (setf norm var-0))
                    ret-val))
                 (setf scale zero) (setf sum one)
                 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                               ((> j n) nil)
                   (tagbody
                     (setf l
                             (max (the f2cl-lib:integer4 1)
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-sub j ku))))
                     (setf k
                             (f2cl-lib:int-add
                              (f2cl-lib:int-sub (f2cl-lib:int-add ku 1) j) l))
                     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                         (classq
                          (f2cl-lib:int-add
                           (f2cl-lib:int-sub
                            (min (the f2cl-lib:integer4 n)
                                 (the f2cl-lib:integer4 (f2cl-lib:int-add j kl)))
                            l)
                           1)
                          (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8 (k j)
                                                ((1 ldab) (1 *)) ab-%offset%)
                          1 scale sum)
                       (declare (ignore var-0 var-1 var-2))
                       (when var-3 (setf scale var-3))
                       (when var-4 (setf sum var-4)))
                     label90))
                 (setf value (* scale (f2cl-lib:fsqrt sum)))))
        (setf clangb value)
        (go end_label)
       end_label
        (return (values clangb norm nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
```

```
(setf (gethash 'fortran-to-lisp::clangb
               fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((string)
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array single-float (*)))
                                          :return-values
                                          '(fortran-to-lisp::norm nil nil nil
                                            nil nil nil)
                                          :calls 'nil)))
```

50.2 claqgb

50.2.1 Axiom unit tests

— claqgb.input —

```
)set break resume
)sys rm -f claqgb.output
)spool claqgb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

50.2.2 Axiom help page

— claqgb.help —

```
=====================================================================
claqgb examples
```

```
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine claqgb (M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND, AMAX,
           EQUED)
           CLAQGB scales a general band matrix, using row and column scaling
           factors computed by sgbequ.

Function/Subroutine Documentation
   subroutine claqgb (integer M, integer N, integer KL, integer KU, complex,
       dimension( ldab, * ) AB, integer LDAB, real, dimension( * ) R, real,
       dimension( * ) C, real ROWCND, real COLCND, real AMAX, character EQUED)
       CLAQGB scales a general band matrix, using row and column scaling
       factors computed by sgbequ.

Purpose:
   CLAQGB equilibrates a general M by N band matrix A with KL
   subdiagonals and KU superdiagonals using the row and scaling factors
   in the vectors R and C.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 AB
   AB is COMPLEX array, dimension (LDAB,N)
   On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
   The j-th column of A is stored in the j-th column of the
   array AB as follows:
   AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl)
```

```
  On exit, the equilibrated matrix, in the same storage format
  as A.  See EQUED for the form of the equilibrated matrix.

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDA >= KL+KU+1.

R
  R is REAL array, dimension (M)
  The row scale factors for A.

C
  C is REAL array, dimension (N)
  The column scale factors for A.

ROWCND
  ROWCND is REAL
  Ratio of the smallest R(i) to the largest R(i).

COLCND
  COLCND is REAL
  Ratio of the smallest C(i) to the largest C(i).

AMAX
  AMAX is REAL
  Absolute value of largest matrix entry.

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration
  = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
  = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
  = 'B':  Both row and column equilibration, i.e., A has been
          replaced by diag(R) * A * diag(C).

Internal Parameters:

  THRESH is a threshold value used to decide if row or column scaling
  should be done based on the ratio of the row or column scaling
  factors.  If ROWCND < THRESH, row scaling is done, and if
  COLCND < THRESH, column scaling is done.

  LARGE and SMALL are threshold values used to decide if row scaling
  should be done based on the absolute size of the largest matrix
  element.  If AMAX > LARGE or AMAX < SMALL, row scaling is done.
```

50.2.3 fortran code

— claqgb.f —

```
*  =====================================================================
      SUBROUTINE CLAQGB( M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $                   AMAX, EQUED )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED
      INTEGER            KL, KU, LDAB, M, N
      REAL               AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      REAL               C( * ), R( * )
      COMPLEX            AB( LDAB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, THRESH
      PARAMETER          ( ONE = 1.0E+0, THRESH = 0.1E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               CJ, LARGE, SMALL
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( M.LE.0 .OR. N.LE.0 ) THEN
         EQUED = 'N'
         RETURN
      END IF
*
```

```
*     Initialize LARGE and SMALL.
*
      SMALL = SLAMCH( 'Safe minimum' ) / SLAMCH( 'Precision' )
      LARGE = ONE / SMALL
*
      IF( ROWCND.GE.THRESH .AND. AMAX.GE.SMALL .AND. AMAX.LE.LARGE )
     $     THEN
*
*        No row scaling
*
         IF( COLCND.GE.THRESH ) THEN
*
*           No column scaling
*
            EQUED = 'N'
         ELSE
*
*           Column scaling
*
            DO 20 J = 1, N
               CJ = C( J )
               DO 10 I = MAX( 1, J-KU ), MIN( M, J+KL )
                  AB( KU+1+I-J, J ) = CJ*AB( KU+1+I-J, J )
   10          CONTINUE
   20       CONTINUE
            EQUED = 'C'
         END IF
      ELSE IF( COLCND.GE.THRESH ) THEN
*
*        Row scaling, no column scaling
*
         DO 40 J = 1, N
            DO 30 I = MAX( 1, J-KU ), MIN( M, J+KL )
               AB( KU+1+I-J, J ) = R( I )*AB( KU+1+I-J, J )
   30       CONTINUE
   40    CONTINUE
         EQUED = 'R'
      ELSE
*
*        Row and column scaling
*
         DO 60 J = 1, N
            CJ = C( J )
            DO 50 I = MAX( 1, J-KU ), MIN( M, J+KL )
               AB( KU+1+I-J, J ) = CJ*R( I )*AB( KU+1+I-J, J )
   50       CONTINUE
   60    CONTINUE
         EQUED = 'B'
      END IF
*
```

```
      RETURN
*
*     End of CLAQGB
*
      END
```

———-

50.2.4 lisp code

— **claqgb.lisp** —

```
(let* ((one 1.0) (thresh 0.1))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.1 0.1) thresh)
           (ignorable one thresh))
  (defun claqgb (m n kl ku ab ldab r c rowcnd colcnd amax equed)
    (declare (type (string *) equed)
             (type (single-float) amax colcnd rowcnd)
             (type (array single-float (*)) c r)
             (type (array f2cl-lib:complex8 (*)) ab)
             (type (f2cl-lib:integer4) ldab ku kl n m))
    (f2cl-lib:with-multi-array-data
        ((ab f2cl-lib:complex8 ab-%data% ab-%offset%)
         (r single-float r-%data% r-%offset%)
         (c single-float c-%data% c-%offset%)
         (equed character equed-%data% equed-%offset%))
      (prog ((cj 0.0) (large 0.0) (small 0.0) (i 0) (j 0))
        (declare (type (single-float) cj large small)
                 (type (f2cl-lib:integer4) i j))
        (cond
         ((or (<= m 0) (<= n 0))
          (f2cl-lib:f2cl-set-string equed "N" (string 1)) (go end_label)))
        (setf small (/ (slamch "Safe minimum") (slamch "Precision")))
        (setf large (/ one small))
        (cond
         ((and (>= rowcnd thresh) (>= amax small) (<= amax large))
          (cond
           ((>= colcnd thresh) (f2cl-lib:f2cl-set-string equed "N" (string 1)))
           (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf cj (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
                (f2cl-lib:fdo (i
                               (max (the f2cl-lib:integer4 1)
                                    (the f2cl-lib:integer4
                                         (f2cl-lib:int-add j
```

```
                                                    (f2cl-lib:int-sub
                                                     ku))))
                          (f2cl-lib:int-add i 1))
                         ((> i
                             (min (the f2cl-lib:integer4 m)
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-add j kl))))
                          nil)
            (tagbody
              (setf (f2cl-lib:fref ab-%data%
                                   ((f2cl-lib:int-sub
                                     (f2cl-lib:int-add ku 1 i) j)
                                    j)
                                   ((1 ldab) (1 *)) ab-%offset%)
                    (* cj
                       (f2cl-lib:fref ab-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add ku 1 i) j)
                                       j)
                                      ((1 ldab) (1 *)) ab-%offset%)))
             label10))
          label20))
      (f2cl-lib:f2cl-set-string equed "C" (string 1)))))
    ((>= colcnd thresh)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i
                        (max (the f2cl-lib:integer4 1)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add j
                                                    (f2cl-lib:int-sub
                                                     ku))))
                        (f2cl-lib:int-add i 1))
                       ((> i
                           (min (the f2cl-lib:integer4 m)
                                (the f2cl-lib:integer4
                                     (f2cl-lib:int-add j kl))))
                        nil)
           (tagbody
             (setf (f2cl-lib:fref ab-%data%
                                  ((f2cl-lib:int-sub
                                    (f2cl-lib:int-add ku 1 i) j)
                                   j)
                                  ((1 ldab) (1 *)) ab-%offset%)
                   (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                      (f2cl-lib:fref ab-%data%
                                     ((f2cl-lib:int-sub
                                       (f2cl-lib:int-add ku 1 i) j)
                                      j)
```

```
                                        ((1 ldab) (1 *)) ab-%offset%)))
                 label30))
              label40))
           (f2cl-lib:f2cl-set-string equed "R" (string 1)))
          (t
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (setf cj (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
               (f2cl-lib:fdo (i
                              (max (the f2cl-lib:integer4 1)
                                   (the f2cl-lib:integer4
                                        (f2cl-lib:int-add j
                                                          (f2cl-lib:int-sub
                                                           ku))))
                              (f2cl-lib:int-add i 1))
                             ((> i
                                 (min (the f2cl-lib:integer4 m)
                                      (the f2cl-lib:integer4
                                           (f2cl-lib:int-add j kl))))
                              nil)
                 (tagbody
                   (setf (f2cl-lib:fref ab-%data%
                                        ((f2cl-lib:int-sub
                                          (f2cl-lib:int-add ku 1 i) j)
                                         j)
                                        ((1 ldab) (1 *)) ab-%offset%)
                         (* cj (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                              (f2cl-lib:fref ab-%data%
                                             ((f2cl-lib:int-sub
                                               (f2cl-lib:int-add ku 1 i) j)
                                              j)
                                             ((1 ldab) (1 *)) ab-%offset%)))
                 label50))
              label60))
           (f2cl-lib:f2cl-set-string equed "B" (string 1))))
         (go end_label)
        end_label
         (return (values nil nil nil nil nil nil nil nil nil nil nil equed))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::claqgb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
```

```
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (single-float) (single-float)
  (single-float) (string))
:return-values
'(nil nil nil nil nil nil nil nil
  nil nil nil
  fortran-to-lisp::equed)
:calls 'nil)))
```

Chapter 51

LAPACK - General Band Matrix, Auxilliary Routines, Complex16

51.1 zlangb

51.1.1 Axiom unit tests

— zlangb.input —

```
)set break resume
)sys rm -f zlangb.output
)spool zlangb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

51.1.2 Axiom help page

— zlangb.help —

```
==========================================================================
```

```
zlangb examples
===================================================================

===================================================================
Man Page Details
===================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function zlangb (NORM, N, KL, KU, AB, LDAB, WORK)
           ZLANGB returns the value of the 1-norm, Frobenius norm, infinity-
           norm, or the largest absolute value of any element of general band
           matrix.

Function/Subroutine Documentation
   double precision function zlangb (character NORM, integer N, integer KL,
       integer KU, complex*16, dimension( ldab, * ) AB, integer LDAB, double
       precision, dimension( * ) WORK)
       ZLANGB returns the value of the 1-norm, Frobenius norm, infinity-norm,
       or the largest absolute value of any element of general band matrix.

Purpose:
   ZLANGB  returns the value of the one norm,  or the Frobenius norm, or
   the  infinity norm,  or the element of  largest absolute value  of an
   n by n band matrix  A,  with kl sub-diagonals and ku super-diagonals.

 Returns:

   ZLANGB

       ZLANGB = ( max(abs(A(i,j))), NORM = 'M' or 'm'
                (
                ( norm1(A),         NORM = '1', 'O' or 'o'
                (
                ( normI(A),         NORM = 'I' or 'i'
                (
                ( normF(A),         NORM = 'F', 'f', 'E' or 'e'

   where  norm1  denotes the  one norm of a matrix (maximum column sum),
   normI  denotes the  infinity norm  of a matrix  (maximum row sum) and
   normF  denotes the  Frobenius norm of a matrix (square root of sum of
   squares).  Note that  max(abs(A(i,j)))  is not a consistent matrix norm.

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies the value to be returned in ZLANGB as described above.

 N
```

```
  N is INTEGER
  The order of the matrix A.  N >= 0.  When N = 0, ZLANGB is set to zero.

KL
  KL is INTEGER
  The number of sub-diagonals of the matrix A.  KL >= 0.

KU
  KU is INTEGER
  The number of super-diagonals of the matrix A.  KU >= 0.

AB
  AB is COMPLEX*16 array, dimension (LDAB,N)
  The band matrix A, stored in rows 1 to KL+KU+1.  The j-th
  column of A is stored in the j-th column of the array AB as follows:
  AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(n,j+kl).

LDAB
  LDAB is INTEGER
  The leading dimension of the array AB.  LDAB >= KL+KU+1.

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK)),
  where LWORK >= N when NORM = 'I'; otherwise, WORK is not referenced.
```

———-

51.1.3 fortran code

— zlangb.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION ZLANGB( NORM, N, KL, KU, AB, LDAB,
     $                 WORK )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            KL, KU, LDAB, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   WORK( * )
      COMPLEX*16         AB( LDAB, * )
```

```
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, K, L
      DOUBLE PRECISION   SCALE, SUM, VALUE, TEMP
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, DISNAN
      EXTERNAL           LSAME, DISNAN
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZLASSQ
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      IF( N.EQ.0 ) THEN
         VALUE = ZERO
      ELSE IF( LSAME( NORM, 'M' ) ) THEN
*
*        Find max(abs(A(i,j))).
*
         VALUE = ZERO
         DO 20 J = 1, N
            DO 10 I = MAX( KU+2-J, 1 ), MIN( N+KU+1-J, KL+KU+1 )
               TEMP = ABS( AB( I, J ) )
               IF( VALUE.LT.TEMP .OR. DISNAN( TEMP ) ) VALUE = TEMP
   10       CONTINUE
   20    CONTINUE
      ELSE IF( ( LSAME( NORM, 'O' ) ) .OR. ( NORM.EQ.'1' ) ) THEN
*
*        Find norm1(A).
*
         VALUE = ZERO
         DO 40 J = 1, N
            SUM = ZERO
            DO 30 I = MAX( KU+2-J, 1 ), MIN( N+KU+1-J, KL+KU+1 )
               SUM = SUM + ABS( AB( I, J ) )
   30       CONTINUE
            IF( VALUE.LT.SUM .OR. DISNAN( SUM ) ) VALUE = SUM
   40    CONTINUE
      ELSE IF( LSAME( NORM, 'I' ) ) THEN
```

```
*
*        Find normI(A).
*
         DO 50 I = 1, N
            WORK( I ) = ZERO
   50    CONTINUE
         DO 70 J = 1, N
            K = KU + 1 - J
            DO 60 I = MAX( 1, J-KU ), MIN( N, J+KL )
               WORK( I ) = WORK( I ) + ABS( AB( K+I, J ) )
   60       CONTINUE
   70    CONTINUE
         VALUE = ZERO
         DO 80 I = 1, N
            TEMP = WORK( I )
            IF( VALUE.LT.TEMP .OR. DISNAN( TEMP ) ) VALUE = TEMP
   80    CONTINUE
      ELSE IF( ( LSAME( NORM, 'F' ) ) .OR. ( LSAME( NORM, 'E' ) ) ) THEN
*
*        Find normF(A).
*
         SCALE = ZERO
         SUM = ONE
         DO 90 J = 1, N
            L = MAX( 1, J-KU )
            K = KU + 1 - J + L
            CALL ZLASSQ( MIN( N, J+KL )-L+1, AB( K, J ), 1, SCALE, SUM )
   90    CONTINUE
         VALUE = SCALE*SQRT( SUM )
      END IF
*
      ZLANGB = VALUE
      RETURN
*
*     End of ZLANGB
*
      END
```

———-

51.1.4 lisp code

— zlangb.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
```

```
(defun zlangb (norm n kl ku ab ldab work)
  (declare (type (array double-float (*)) work)
           (type (array f2cl-lib:complex16 (*)) ab)
           (type (f2cl-lib:integer4) ldab ku kl n)
           (type (string *) norm))
  (f2cl-lib:with-multi-array-data
      ((norm character norm-%data% norm-%offset%)
       (ab f2cl-lib:complex16 ab-%data% ab-%offset%)
       (work double-float work-%data% work-%offset%))
    (prog ((scale 0.0d0) (sum 0.0d0) (value 0.0d0) (temp 0.0d0) (i 0) (j 0)
           (k 0) (l 0) (zlangb 0.0d0))
      (declare (type (f2cl-lib:integer4) i j k l)
               (type (double-float) scale sum value temp zlangb))
      (cond ((= n 0) (setf value zero))
            ((multiple-value-bind (ret-val var-0 var-1)
                 (lsame norm "M")
               (declare (ignore var-1))
               (when var-0 (setf norm var-0))
               ret-val)
             (setf value zero)
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (f2cl-lib:fdo (i
                                (max
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add ku 2
                                                        (f2cl-lib:int-sub
                                                         j)))
                                 (the f2cl-lib:integer4 1))
                                (f2cl-lib:int-add i 1))
                               ((> i
                                   (min
                                    (the f2cl-lib:integer4
                                         (f2cl-lib:int-add n ku 1
                                                           (f2cl-lib:int-sub
                                                            j)))
                                    (the f2cl-lib:integer4
                                         (f2cl-lib:int-add kl ku 1))))
                                nil)
                 (tagbody
                   (setf temp
                           (abs
                            (f2cl-lib:fref ab-%data% (i j) ((1 ldab) (1 *))
                                           ab-%offset%)))
                   (if (or (< value temp)
                           (multiple-value-bind (ret-val var-0)
                               (disnan temp)
                             (declare (ignore))
                             (when var-0 (setf temp var-0))
```

```
                    ret-val))
              (setf value temp))
         label10))
     label20)))
((or
  (multiple-value-bind (ret-val var-0 var-1)
      (lsame norm "O")
    (declare (ignore var-1))
    (when var-0 (setf norm var-0))
    ret-val)
  (f2cl-lib:fstring-= norm "1"))
 (setf value zero)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf sum zero)
     (f2cl-lib:fdo (i
                    (max
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add ku 2
                                            (f2cl-lib:int-sub
                                             j)))
                     (the f2cl-lib:integer4 1))
                    (f2cl-lib:int-add i 1))
                   ((> i
                       (min
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add n ku 1
                                               (f2cl-lib:int-sub
                                                j)))
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add kl ku 1))))
                    nil)
       (tagbody
         (setf sum
                 (+ sum
                    (abs
                     (f2cl-lib:fref ab-%data% (i j)
                                    ((1 ldab) (1 *))
                                    ab-%offset%))))
        label30))
     (if (or (< value sum)
             (multiple-value-bind (ret-val var-0)
                 (disnan sum)
               (declare (ignore))
               (when var-0 (setf sum var-0))
               ret-val))
         (setf value sum))
     label40)))
((multiple-value-bind (ret-val var-0 var-1)
```

```
    (lsame norm "I")
  (declare (ignore var-1))
  (when var-0 (setf norm var-0))
  ret-val)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            zero)
   label50))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf k (f2cl-lib:int-sub (f2cl-lib:int-add ku 1) j))
    (f2cl-lib:fdo (i
                   (max (the f2cl-lib:integer4 1)
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add j
                                               (f2cl-lib:int-sub
                                                ku))))
                   (f2cl-lib:int-add i 1))
                  ((> i
                      (min (the f2cl-lib:integer4 n)
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add j kl))))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                             work-%offset%)
                (+
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 (abs
                  (f2cl-lib:fref ab-%data%
                                 ((f2cl-lib:int-add k i) j)
                                 ((1 ldab) (1 *))
                                 ab-%offset%))))
       label60))
   label70))
(setf value zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf temp
            (f2cl-lib:fref work-%data% (i) ((1 *))
                           work-%offset%))
    (if (or (< value temp)
            (multiple-value-bind (ret-val var-0)
                (disnan temp)
              (declare (ignore))
```

```
                          (when var-0 (setf temp var-0))
                          ret-val))
                     (setf value temp))
                 label80)))
             ((or
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame norm "F")
                 (declare (ignore var-1))
                 (when var-0 (setf norm var-0))
                 ret-val)
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame norm "E")
                 (declare (ignore var-1))
                 (when var-0 (setf norm var-0))
                 ret-val))
              (setf scale zero) (setf sum one)
              (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                            ((> j n) nil)
                (tagbody
                  (setf l
                          (max (the f2cl-lib:integer4 1)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-sub j ku))))
                  (setf k
                          (f2cl-lib:int-add
                           (f2cl-lib:int-sub (f2cl-lib:int-add ku 1) j) l))
                  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                      (zlassq
                       (f2cl-lib:int-add
                        (f2cl-lib:int-sub
                         (min (the f2cl-lib:integer4 n)
                              (the f2cl-lib:integer4 (f2cl-lib:int-add j kl)))
                         l)
                        1)
                       (f2cl-lib:array-slice ab-%data% f2cl-lib:complex16
                                             (k j) ((1 ldab) (1 *))
                                             ab-%offset%)
                       1 scale sum)
                    (declare (ignore var-0 var-1 var-2))
                    (when var-3 (setf scale var-3))
                    (when var-4 (setf sum var-4)))
                  label90))
              (setf value (* scale (f2cl-lib:fsqrt sum)))))
        (setf zlangb value)
        (go end_label)
       end_label
        (return (values zlangb norm nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
```

```
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zlangb
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*)))
                                           :return-values
                                           '(fortran-to-lisp::norm nil nil nil
                                             nil nil nil)
                                           :calls 'nil)))
```

———-

51.2 zlaqgb

51.2.1 Axiom unit tests

— **zlaqgb.input** —

```
)set break resume
)sys rm -f zlaqgb.output
)spool zlaqgb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

51.2.2 Axiom help page

— **zlaqgb.help** —

```
====================================================================
```

```
zlaqgb examples
===================================================================

===================================================================
Man Page Details
===================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zlaqgb (M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND, AMAX,
           EQUED)
           ZLAQGB scales a general band matrix, using row and column scaling
           factors computed by sgbequ.

Function/Subroutine Documentation
   subroutine zlaqgb (integer M, integer N, integer KL, integer KU,
       complex*16, dimension( ldab, * ) AB, integer LDAB, double precision,
       dimension( * ) R, double precision, dimension( * ) C, double precision
       ROWCND, double precision COLCND, double precision AMAX, character
       EQUED)
       ZLAQGB scales a general band matrix, using row and column scaling
       factors computed by sgbequ.

Purpose:
   ZLAQGB equilibrates a general M by N band matrix A with KL
   subdiagonals and KU superdiagonals using the row and scaling factors
   in the vectors R and C.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 KL
   KL is INTEGER
   The number of subdiagonals within the band of A.  KL >= 0.

 KU
   KU is INTEGER
   The number of superdiagonals within the band of A.  KU >= 0.

 AB
   AB is COMPLEX*16 array, dimension (LDAB,N)
   On entry, the matrix A in band storage, in rows 1 to KL+KU+1.
   The j-th column of A is stored in the j-th column of the
```

```
   array AB as follows:
   AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl)

   On exit, the equilibrated matrix, in the same storage format
   as A.  See EQUED for the form of the equilibrated matrix.

 LDAB
   LDAB is INTEGER
   The leading dimension of the array AB.  LDA >= KL+KU+1.

 R
   R is DOUBLE PRECISION array, dimension (M)
   The row scale factors for A.

 C
   C is DOUBLE PRECISION array, dimension (N)
   The column scale factors for A.

 ROWCND
   ROWCND is DOUBLE PRECISION
   Ratio of the smallest R(i) to the largest R(i).

 COLCND
   COLCND is DOUBLE PRECISION
   Ratio of the smallest C(i) to the largest C(i).

 AMAX
   AMAX is DOUBLE PRECISION
   Absolute value of largest matrix entry.

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done.
   = 'N':  No equilibration
   = 'R':  Row equilibration, i.e., A has been premultiplied by diag(R).
   = 'C':  Column equilibration, i.e., A has been postmultiplied by diag(C).
   = 'B':  Both row and column equilibration, i.e., A has been
           replaced by diag(R) * A * diag(C).

Internal Parameters:

   THRESH is a threshold value used to decide if row or column scaling
   should be done based on the ratio of the row or column scaling
   factors.  If ROWCND < THRESH, row scaling is done, and if
   COLCND < THRESH, column scaling is done.

   LARGE and SMALL are threshold values used to decide if row scaling
   should be done based on the absolute size of the largest matrix
   element.  If AMAX > LARGE or AMAX < SMALL, row scaling is done.
```

———-

51.2.3 fortran code

— zlaqgb.f —

```
*  ===================================================================
      SUBROUTINE ZLAQGB( M, N, KL, KU, AB, LDAB, R, C, ROWCND, COLCND,
     $                   AMAX, EQUED )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED
      INTEGER            KL, KU, LDAB, M, N
      DOUBLE PRECISION   AMAX, COLCND, ROWCND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   C( * ), R( * )
      COMPLEX*16         AB( LDAB, * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, THRESH
      PARAMETER          ( ONE = 1.0D+0, THRESH = 0.1D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   CJ, LARGE, SMALL
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( M.LE.0 .OR. N.LE.0 ) THEN
         EQUED = 'N'
```

```
         RETURN
      END IF
*
*     Initialize LARGE and SMALL.
*
      SMALL = DLAMCH( 'Safe minimum' ) / DLAMCH( 'Precision' )
      LARGE = ONE / SMALL
*
      IF( ROWCND.GE.THRESH .AND. AMAX.GE.SMALL .AND. AMAX.LE.LARGE )
     $     THEN
*
*        No row scaling
*
         IF( COLCND.GE.THRESH ) THEN
*
*           No column scaling
*
            EQUED = 'N'
         ELSE
*
*           Column scaling
*
            DO 20 J = 1, N
               CJ = C( J )
               DO 10 I = MAX( 1, J-KU ), MIN( M, J+KL )
                  AB( KU+1+I-J, J ) = CJ*AB( KU+1+I-J, J )
   10          CONTINUE
   20       CONTINUE
            EQUED = 'C'
         END IF
      ELSE IF( COLCND.GE.THRESH ) THEN
*
*        Row scaling, no column scaling
*
         DO 40 J = 1, N
            DO 30 I = MAX( 1, J-KU ), MIN( M, J+KL )
               AB( KU+1+I-J, J ) = R( I )*AB( KU+1+I-J, J )
   30       CONTINUE
   40    CONTINUE
         EQUED = 'R'
      ELSE
*
*        Row and column scaling
*
         DO 60 J = 1, N
            CJ = C( J )
            DO 50 I = MAX( 1, J-KU ), MIN( M, J+KL )
               AB( KU+1+I-J, J ) = CJ*R( I )*AB( KU+1+I-J, J )
   50       CONTINUE
   60    CONTINUE
```

```
         EQUED = 'B'
      END IF
*
      RETURN
*
*     End of ZLAQGB
*
      END
```

51.2.4 lisp code

— **zlaqgb.lisp** —

```
(let* ((one 1.0d0) (thresh 0.1d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.1d0 0.1d0) thresh)
           (ignorable one thresh))
  (defun zlaqgb (m n kl ku ab ldab r c rowcnd colcnd amax equed)
    (declare (type (string *) equed)
             (type (double-float) amax colcnd rowcnd)
             (type (array double-float (*)) c r)
             (type (array f2cl-lib:complex16 (*)) ab)
             (type (f2cl-lib:integer4) ldab ku kl n m))
    (f2cl-lib:with-multi-array-data
        ((ab f2cl-lib:complex16 ab-%data% ab-%offset%)
         (r double-float r-%data% r-%offset%)
         (c double-float c-%data% c-%offset%)
         (equed character equed-%data% equed-%offset%))
      (prog ((cj 0.0d0) (large 0.0d0) (small 0.0d0) (i 0) (j 0))
        (declare (type (double-float) cj large small)
                 (type (f2cl-lib:integer4) i j))
        (cond
         ((or (<= m 0) (<= n 0))
          (f2cl-lib:f2cl-set-string equed "N" (string 1)) (go end_label)))
        (setf small (/ (dlamch "Safe minimum") (dlamch "Precision")))
        (setf large (/ one small))
        (cond
         ((and (>= rowcnd thresh) (>= amax small) (<= amax large))
          (cond
           ((>= colcnd thresh) (f2cl-lib:f2cl-set-string equed "N" (string 1)))
           (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf cj (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
                (f2cl-lib:fdo (i
```

```
                          (max (the f2cl-lib:integer4 1)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add j
                                                      (f2cl-lib:int-sub
                                                       ku))))
                          (f2cl-lib:int-add i 1))
                         ((> i
                             (min (the f2cl-lib:integer4 m)
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-add j kl))))
                          nil)
                 (tagbody
                   (setf (f2cl-lib:fref ab-%data%
                                        ((f2cl-lib:int-sub
                                          (f2cl-lib:int-add ku 1 i) j)
                                         j)
                                        ((1 ldab) (1 *)) ab-%offset%)
                         (* cj
                            (f2cl-lib:fref ab-%data%
                                           ((f2cl-lib:int-sub
                                             (f2cl-lib:int-add ku 1 i) j)
                                            j)
                                           ((1 ldab) (1 *)) ab-%offset%)))
                  label10))
              label20))
            (f2cl-lib:f2cl-set-string equed "C" (string 1)))))
        ((>= colcnd thresh)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
            (tagbody
              (f2cl-lib:fdo (i
                             (max (the f2cl-lib:integer4 1)
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-add j
                                                         (f2cl-lib:int-sub
                                                          ku))))
                             (f2cl-lib:int-add i 1))
                            ((> i
                                (min (the f2cl-lib:integer4 m)
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add j kl))))
                             nil)
                 (tagbody
                   (setf (f2cl-lib:fref ab-%data%
                                        ((f2cl-lib:int-sub
                                          (f2cl-lib:int-add ku 1 i) j)
                                         j)
                                        ((1 ldab) (1 *)) ab-%offset%)
                         (* (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                            (f2cl-lib:fref ab-%data%
```

```
                                          ((f2cl-lib:int-sub
                                            (f2cl-lib:int-add ku 1 i) j)
                                           j)
                                          ((1 ldab) (1 *)) ab-%offset%)))
                 label30))
              label40))
           (f2cl-lib:f2cl-set-string equed "R" (string 1)))
          (t
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (setf cj (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
               (f2cl-lib:fdo (i
                              (max (the f2cl-lib:integer4 1)
                                   (the f2cl-lib:integer4
                                        (f2cl-lib:int-add j
                                                          (f2cl-lib:int-sub
                                                           ku))))
                              (f2cl-lib:int-add i 1))
                             ((> i
                                 (min (the f2cl-lib:integer4 m)
                                      (the f2cl-lib:integer4
                                           (f2cl-lib:int-add j kl))))
                              nil)
                 (tagbody
                   (setf (f2cl-lib:fref ab-%data%
                                        ((f2cl-lib:int-sub
                                          (f2cl-lib:int-add ku 1 i) j)
                                         j)
                                        ((1 ldab) (1 *)) ab-%offset%)
                         (* cj (f2cl-lib:fref r-%data% (i) ((1 *)) r-%offset%)
                            (f2cl-lib:fref ab-%data%
                                          ((f2cl-lib:int-sub
                                            (f2cl-lib:int-add ku 1 i) j)
                                           j)
                                          ((1 ldab) (1 *)) ab-%offset%)))
                 label50))
              label60))
           (f2cl-lib:f2cl-set-string equed "B" (string 1))))
         (go end_label)
        end_label
         (return (values nil nil nil nil nil nil nil nil nil nil nil equed))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zlaqgb
                 fortran-to-lisp::*f2cl-function-info*)
           (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((fortran-to-lisp::integer4)
```

```
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (double-float) (double-float)
  (double-float) (string))
:return-values
'(nil nil nil nil nil nil nil nil
  nil nil nil
  fortran-to-lisp::equed)
:calls 'nil)))
```

Chapter 52

LAPACK - Symmetric Matrix, Linear Solve, Double

52.1 dsysv

52.1.1 Axiom unit tests

— dsysv.input —

```
)set break resume
)sys rm -f dsysv.output
)spool dsysv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

52.1.2 Axiom help page

— dsysv.help —

```
==========================================================================
dsysv examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsysv (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK, LWORK,
           INFO)
            DSYSV computes the solution to system of linear equations A * X =
           B for SY matrices

Function/Subroutine Documentation
   subroutine dsysv (character UPLO, integer N, integer NRHS, double
       precision, dimension( lda, * ) A, integer LDA, integer, dimension( * )
       IPIV, double precision, dimension( ldb, * ) B, integer LDB, double
       precision, dimension( * ) WORK, integer LWORK, integer INFO)
        DSYSV computes the solution to system of linear equations A * X = B
       for SY matrices

Purpose:
   DSYSV computes the solution to a real system of linear equations
      A * X = B,
   where A is an N-by-N symmetric matrix and X and B are N-by-NRHS
   matrices.

   The diagonal pivoting method is used to factor A as
      A = U * D * U**T,  if UPLO = 'U', or
      A = L * D * L**T,  if UPLO = 'L',
   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, and D is symmetric and block diagonal with
   1-by-1 and 2-by-2 diagonal blocks.  The factored form of A is then
   used to solve the system of equations A * X = B.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.
```

```
A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if INFO = 0, the block diagonal matrix D and the
  multipliers used to obtain the factor U or L from the
  factorization A = U*D*U**T or A = L*D*L**T as computed by DSYTRF.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D, as
  determined by DSYTRF.  If IPIV(k) > 0, then rows and columns
  k and IPIV(k) were interchanged, and D(k,k) is a 1-by-1
  diagonal block.  If UPLO = 'U' and IPIV(k) = IPIV(k-1) < 0,
  then rows and columns k-1 and -IPIV(k) were interchanged and
  D(k-1:k,k-1:k) is a 2-by-2 diagonal block.  If UPLO = 'L' and
  IPIV(k) = IPIV(k+1) < 0, then rows and columns k+1 and
  -IPIV(k) were interchanged and D(k:k+1,k:k+1) is a 2-by-2
  diagonal block.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of WORK.  LWORK >= 1, and for best performance
  LWORK >= max(1,N*NB), where NB is the optimal blocksize for
  DSYTRF.
  for LWORK < N, TRS will be done with Level BLAS 2
  for LWORK >= N, TRS will be done with Level BLAS 3
```

```
  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = i, D(i,i) is exactly zero.  The factorization
       has been completed, but the block diagonal matrix D is
       exactly singular, so the solution could not be computed.
```

———-

52.1.3 fortran code

— **dsysv.f** —

```
*  =====================================================================
      SUBROUTINE DSYSV( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK,
     $                  LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, LWORK, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            LWKOPT
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
```

```
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, DSYTRF, DSYTRS, DSYTRS2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -10
      END IF
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            LWKOPT = 1
         ELSE
            CALL DSYTRF( UPLO, N, A, LDA, IPIV, WORK, -1, INFO )
            LWKOPT = WORK(1)
         END IF
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYSV ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Compute the factorization A = U*D*U**T or A = L*D*L**T.
*
      CALL DSYTRF( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
```

```
*
         IF ( LWORK.LT.N ) THEN
*
*        Solve with TRS ( Use Level BLAS 2)
*
            CALL DSYTRS( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
         ELSE
*
*        Solve with TRS2 ( Use Level BLAS 3)
*
            CALL DSYTRS2( UPLO,N,NRHS,A,LDA,IPIV,B,LDB,WORK,INFO )
*
         END IF
*
      END IF
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of DSYSV
*
      END
```

———-

52.1.4 lisp code

— **dsysv.lisp** —

```
(defun dsysv (uplo n nrhs a lda ipiv b ldb$ work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array double-float (*)) work b a)
           (type (f2cl-lib:integer4) info lwork ldb$ lda nrhs n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a double-float a-%data% a-%offset%)
       (b double-float b-%data% b-%offset%)
       (work double-float work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((lwkopt 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) lwkopt))
      (setf info 0)
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond
```

```
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and (< lwork 1) (not lquery)) (setf info -10)))
(cond
 ((= info 0)
  (cond ((= n 0) (setf lwkopt 1))
        (t
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
             (dsytrf uplo n a lda ipiv work -1 info)
           (declare (ignore var-2 var-4 var-5 var-6))
           (when var-0 (setf uplo var-0))
           (when var-1 (setf n var-1))
           (when var-3 (setf lda var-3))
           (when var-7 (setf info var-7)))
         (setf lwkopt
                 (f2cl-lib:int
                  (f2cl-lib:fref work-%data% (1) ((1 *))
                                 work-%offset%)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))))
(cond
 ((/= info 0) (xerbla "DSYSV " (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dsytrf uplo n a lda ipiv work lwork info)
  (declare (ignore var-2 var-4 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-6 (setf lwork var-6))
  (when var-7 (setf info var-7)))
(cond
```

```
      ((= info 0)
       (cond
        ((< lwork n)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
             (dsytrs uplo n nrhs a lda ipiv b ldb$ info)
           (declare (ignore var-3 var-5 var-6))
           (when var-0 (setf uplo var-0))
           (when var-1 (setf n var-1))
           (when var-2 (setf nrhs var-2))
           (when var-4 (setf lda var-4))
           (when var-7 (setf ldb$ var-7))
           (when var-8 (setf info var-8))))
        (t
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
             (dsytrs2 uplo n nrhs a lda ipiv b ldb$ work info)
           (declare (ignore var-3 var-5 var-6 var-8))
           (when var-0 (setf uplo var-0))
           (when var-1 (setf n var-1))
           (when var-2 (setf nrhs var-2))
           (when var-4 (setf lda var-4))
           (when var-7 (setf ldb$ var-7))
           (when var-9 (setf info var-9)))))))
     (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
             (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
     (go end_label)
    end_label
     (return (values uplo n nrhs nil lda nil nil ldb$ nil lwork info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsysv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
```

```
                                                  fortran-to-lisp::nrhs nil
                                                  fortran-to-lisp::lda nil nil
                                                  fortran-to-lisp::ldb$ nil
                                                  fortran-to-lisp::lwork
                                                  fortran-to-lisp::info)
                                                :calls 'nil)))
```

———-

52.2 dsysv_rook

52.2.1 Axiom unit tests

— **dsysvrook.input** —

```
)set break resume
)sys rm -f dsysvrook.output
)spool dsysvrook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

52.2.2 Axiom help page

— **dsysv_rook.help** —

```
==================================================================
dsysv_rook examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsysv_rook (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK,
```

```
          LWORK, INFO)
           DSYSV_ROOK computes the solution to system of linear equations A *
          X = B for SY matrices

Function/Subroutine Documentation
   subroutine dsysv_rook (character UPLO, integer N, integer NRHS, double
       precision, dimension( lda, * ) A, integer LDA, integer, dimension( * )
       IPIV, double precision, dimension( ldb, * ) B, integer LDB, double
       precision, dimension( * ) WORK, integer LWORK, integer INFO)
        DSYSV_ROOK computes the solution to system of linear equations A * X =
       B for SY matrices

Purpose:
   DSYSV_ROOK computes the solution to a real system of linear
   equations
   A * X = B,
   where A is an N-by-N symmetric matrix and X and B are N-by-NRHS
   matrices.

   The diagonal pivoting method is used to factor A as
   A = U * D * U**T,  if UPLO = 'U', or
   A = L * D * L**T,  if UPLO = 'L',
   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, and D is symmetric and block diagonal with
   1-by-1 and 2-by-2 diagonal blocks.

   DSYTRF_ROOK is called to compute the factorization of a real
   symmetric matrix A using the bounded Bunch-Kaufman ("rook") diagonal
   pivoting method.

   The factored form of A is then used to solve the system
   of equations A * X = B by calling DSYTRS_ROOK.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
```

```
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if INFO = 0, the block diagonal matrix D and the
  multipliers used to obtain the factor U or L from the
  factorization A = U*D*U**T or A = L*D*L**T as computed by DSYTRF_ROOK.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D,
  as determined by DSYTRF_ROOK.

  If UPLO = 'U':
     If IPIV(k) > 0, then rows and columns k and IPIV(k)
     were interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) < 0 and IPIV(k-1) < 0, then rows and
     columns k and -IPIV(k) were interchanged and rows and
     columns k-1 and -IPIV(k-1) were inerchaged,
     D(k-1:k,k-1:k) is a 2-by-2 diagonal block.

  If UPLO = 'L':
     If IPIV(k) > 0, then rows and columns k and IPIV(k)
     were interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) < 0 and IPIV(k+1) < 0, then rows and
     columns k and -IPIV(k) were interchanged and rows and
     columns k+1 and -IPIV(k+1) were inerchaged,
     D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

WORK
```

```
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of WORK.  LWORK >= 1, and for best performance
  LWORK >= max(1,N*NB), where NB is the optimal blocksize for
  DSYTRF_ROOK.

  TRS will be done with Level 2 BLAS

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = i, D(i,i) is exactly zero.  The factorization
       has been completed, but the block diagonal matrix D is
       exactly singular, so the solution could not be computed.
```

———-

52.2.3 fortran code

— **dsysv_rook.f** —

```
*  =====================================================================
      SUBROUTINE DSYSV_ROOK( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK,
     $                  LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, LWORK, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
```

```
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            LWKOPT
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, DSYTRF_ROOK, DSYTRS_ROOK
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -10
      END IF
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            LWKOPT = 1
         ELSE
            CALL DSYTRF_ROOK( UPLO, N, A, LDA, IPIV, WORK, -1, INFO )
            LWKOPT = WORK(1)
         END IF
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYSV_ROOK ', -INFO )
         RETURN
```

```
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Compute the factorization A = U*D*U**T or A = L*D*L**T.
*
      CALL DSYTRF_ROOK( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
*        Solve with TRS_ROOK ( Use Level 2 BLAS)
*
         CALL DSYTRS_ROOK( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
      END IF
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of DSYSV_ROOK
*
      END
```

———-

52.2.4 lisp code

— **dsysv_rook.lisp** —

```
(defun dsysv_rook (uplo n nrhs a lda ipiv b ldb$ work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array double-float (*)) work b a)
           (type (f2cl-lib:integer4) info lwork ldb$ lda nrhs n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a double-float a-%data% a-%offset%)
       (b double-float b-%data% b-%offset%)
       (work double-float work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((lwkopt 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) lwkopt))
      (setf info 0)
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond
```

```
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and (< lwork 1) (not lquery)) (setf info -10)))
(cond
 ((= info 0)
  (cond ((= n 0) (setf lwkopt 1))
        (t
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
             (dsytrf_rook uplo n a lda ipiv work -1 info)
           (declare (ignore var-2 var-4 var-5 var-6))
           (when var-0 (setf uplo var-0))
           (when var-1 (setf n var-1))
           (when var-3 (setf lda var-3))
           (when var-7 (setf info var-7)))
         (setf lwkopt
                 (f2cl-lib:int
                  (f2cl-lib:fref work-%data% (1) ((1 *))
                                 work-%offset%)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))))
(cond
 ((/= info 0) (xerbla "DSYSV_ROOK " (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dsytrf_rook uplo n a lda ipiv work lwork info)
  (declare (ignore var-2 var-4 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-6 (setf lwork var-6))
  (when var-7 (setf info var-7)))
```

```
      (cond
       ((= info 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (dsytrs_rook uplo n nrhs a lda ipiv b ldb$ info)
          (declare (ignore var-3 var-5 var-6))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf lda var-4))
          (when var-7 (setf ldb$ var-7))
          (when var-8 (setf info var-8)))))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
      (go end_label)
     end_label
      (return (values uplo n nrhs nil lda nil nil ldb$ nil lwork info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsysv_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::ldb$ nil
                                              fortran-to-lisp::lwork
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

52.3 dsysvx

52.3.1 Axiom unit tests

— **dsysvx.input** —

```
)set break resume
)sys rm -f dsysvx.output
)spool dsysvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

52.3.2 Axiom help page

— **dsysvx.help** —

```
======================================================================
dsysvx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsysvx (FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB,
           X, LDX, RCOND, FERR, BERR, WORK, LWORK, IWORK, INFO)
            DSYSVX computes the solution to system of linear equations A * X =
           B for SY matrices

Function/Subroutine Documentation
   subroutine dsysvx (character FACT, character UPLO, integer N, integer NRHS,
       double precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV,
       double precision, dimension( ldb, * ) B, integer LDB, double precision,
       dimension( ldx, * ) X, integer LDX, double precision RCOND, double
       precision, dimension( * ) FERR, double precision, dimension( * ) BERR,
       double precision, dimension( * ) WORK, integer LWORK, integer,
       dimension( * ) IWORK, integer INFO)
```

```
       DSYSVX computes the solution to system of linear equations A * X = B
      for SY matrices

Purpose:
   DSYSVX uses the diagonal pivoting factorization to compute the
   solution to a real system of linear equations A * X = B,
   where A is an N-by-N symmetric matrix and X and B are N-by-NRHS
   matrices.

   Error bounds on the solution and a condition estimate are also provided.

Description:

   The following steps are performed:

   1. If FACT = 'N', the diagonal pivoting method is used to factor A.
      The form of the factorization is
         A = U * D * U**T,  if UPLO = 'U', or
         A = L * D * L**T,  if UPLO = 'L',
      where U (or L) is a product of permutation and unit upper (lower)
      triangular matrices, and D is symmetric and block diagonal with
      1-by-1 and 2-by-2 diagonal blocks.

   2. If some D(i,i)=0, so that D is exactly singular, then the routine
      returns with INFO = i. Otherwise, the factored form of A is used
      to estimate the condition number of the matrix A.  If the
      reciprocal of the condition number is less than machine precision,
      INFO = N+1 is returned as a warning, but the routine still goes on
      to solve for X and compute error bounds as described below.

   3. The system of equations is solved for X using the factored form of A.

   4. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of A has been
   supplied on entry.
   = 'F':  On entry, AF and IPIV contain the factored form of
           A.  AF and IPIV will not be modified.
   = 'N':  The matrix A will be copied to AF and factored.

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.
```

```
N
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
  upper triangular part of A contains the upper triangular part
  of the matrix A, and the strictly lower triangular part of A
  is not referenced.  If UPLO = 'L', the leading N-by-N lower
  triangular part of A contains the lower triangular part of
  the matrix A, and the strictly upper triangular part of A is
  not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is DOUBLE PRECISION array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the block diagonal matrix D and the multipliers used
  to obtain the factor U or L from the factorization
  A = U*D*U**T or A = L*D*L**T as computed by DSYTRF.

  If FACT = 'N', then AF is an output argument and on exit
  returns the block diagonal matrix D and the multipliers used
  to obtain the factor U or L from the factorization
  A = U*D*U**T or A = L*D*L**T.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains details of the interchanges and the block structure
  of D, as determined by DSYTRF.
  If IPIV(k) > 0, then rows and columns k and IPIV(k) were
  interchanged and D(k,k) is a 1-by-1 diagonal block.
  If UPLO = 'U' and IPIV(k) = IPIV(k-1) < 0, then rows and
  columns k-1 and -IPIV(k) were interchanged and D(k-1:k,k-1:k)
  is a 2-by-2 diagonal block.  If UPLO = 'L' and IPIV(k) =
  IPIV(k+1) < 0, then rows and columns k+1 and -IPIV(k) were
```

```
  interchanged and D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

  If FACT = 'N', then IPIV is an output argument and on exit
  contains details of the interchanges and the block structure
  of D, as determined by DSYTRF.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  The N-by-NRHS right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  The estimate of the reciprocal condition number of the matrix
  A.  If RCOND is less than the machine precision (in
  particular, if RCOND = 0), the matrix is singular to working
  precision.  This condition is indicated by a return code of INFO > 0.

FERR
  FERR is DOUBLE PRECISION array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.
```

```
 LWORK
   LWORK is INTEGER
   The length of WORK.  LWORK >= max(1,3*N), and for best
   performance, when FACT = 'N', LWORK >= max(1,3*N,N*NB), where
   NB is the optimal blocksize for DSYTRF.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 IWORK
   IWORK is INTEGER array, dimension (N)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value
   > 0: if INFO = i, and i is
     <= N:  D(i,i) is exactly zero.  The factorization
        has been completed but the factor D is exactly
        singular, so the solution and error bounds could
        not be computed. RCOND = 0 is returned.
     = N+1: D is nonsingular, but RCOND is less than machine
        precision, meaning that the matrix is singular
        to working precision.  Nevertheless, the
        solution and error bounds are computed because
        there are a number of situations where the
        computed solution can be more accurate than the
        value of RCOND would suggest.
```

———-

52.3.3 fortran code

— dsysvx.f —

```
*  =====================================================================
      SUBROUTINE DSYSVX( FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B,
     $                   LDB, X, LDX, RCOND, FERR, BERR, WORK, LWORK,
     $                   IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
```

```
*     .. Scalar Arguments ..
      CHARACTER          FACT, UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, LWORK, N, NRHS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   BERR( * ), FERR( * ), WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, NOFACT
      INTEGER            LWKOPT, NB
      DOUBLE PRECISION   ANORM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANSY
      EXTERNAL           LSAME, ILAENV, DLAMCH, DLANSY
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLACPY, DSYCON, DSYRFS, DSYTRF, DSYTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.NOFACT .AND. .NOT.LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) )
     $          THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
```

```
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -11
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -13
      ELSE IF( LWORK.LT.MAX( 1, 3*N ) .AND. .NOT.LQUERY ) THEN
         INFO = -18
      END IF
*
      IF( INFO.EQ.0 ) THEN
         LWKOPT = MAX( 1, 3*N )
         IF( NOFACT ) THEN
            NB = ILAENV( 1, 'DSYTRF', UPLO, N, -1, -1, -1 )
            LWKOPT = MAX( LWKOPT, N*NB )
         END IF
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYSVX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
      IF( NOFACT ) THEN
*
*        Compute the factorization A = U*D*U**T or A = L*D*L**T.
*
         CALL DLACPY( UPLO, N, N, A, LDA, AF, LDAF )
         CALL DSYTRF( UPLO, N, AF, LDAF, IPIV, WORK, LWORK, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 )THEN
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A.
*
      ANORM = DLANSY( 'I', UPLO, N, A, LDA, WORK )
*
*     Compute the reciprocal of the condition number of A.
*
      CALL DSYCON( UPLO, N, AF, LDAF, IPIV, ANORM, RCOND, WORK, IWORK,
```

```
     $             INFO )
*
*     Compute the solution vectors X.
*
      CALL DLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL DSYTRS( UPLO, N, NRHS, AF, LDAF, IPIV, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solutions and
*     compute error bounds and backward error estimates for them.
*
      CALL DSYRFS( UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB, X,
     $             LDX, FERR, BERR, WORK, IWORK, INFO )
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.DLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of DSYSVX
*
      END
```

———-

52.3.4 lisp code

— **dsysvx.lisp** —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun dsysvx
         (fact uplo n nrhs a lda af ldaf ipiv b ldb$ x ldx rcond ferr berr work
          lwork iwork info)
    (declare (type (double-float) rcond)
             (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array double-float (*)) work berr ferr x b af a)
             (type (f2cl-lib:integer4) info lwork ldx ldb$ ldaf lda nrhs n)
             (type (string *) uplo fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (af double-float af-%data% af-%offset%)
```

```
    (b double-float b-%data% b-%offset%)
    (x double-float x-%data% x-%offset%)
    (ferr double-float ferr-%data% ferr-%offset%)
    (berr double-float berr-%data% berr-%offset%)
    (work double-float work-%data% work-%offset%)
    (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
    (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((anorm 0.0d0) (lwkopt 0) (nb 0) (lquery nil) (nofact nil))
    (declare (type (double-float) anorm)
             (type (f2cl-lib:integer4) lwkopt nb)
             (type f2cl-lib:logical lquery nofact))
    (setf info 0)
    (setf nofact
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "N")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val))
    (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
    (cond
     ((and (not nofact)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "F")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val)))
      (setf info -1))
     ((and
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
      (setf info -2))
     ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
     ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -6))
     ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -8))
     ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -11))
     ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
```

```
  (setf info -13))
 ((and
   (< lwork
      (max (the f2cl-lib:integer4 1)
           (the f2cl-lib:integer4 (f2cl-lib:int-mul 3 n))))
   (not lquery))
  (setf info -18)))
(cond
 ((= info 0)
  (setf lwkopt
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul 3 n))))
  (cond
   (nofact
    (setf nb
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 1 "DSYTRF" uplo n -1 -1 -1)
              (declare (ignore var-0 var-1 var-4 var-5 var-6))
              (when var-2 (setf uplo var-2))
              (when var-3 (setf n var-3))
              ret-val))
    (setf lwkopt
            (max (the f2cl-lib:integer4 lwkopt)
                 (the f2cl-lib:integer4 (f2cl-lib:int-mul n nb))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))))
(cond
 ((/= info 0) (xerbla "DSYSVX" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond
 (nofact
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy uplo n n a lda af ldaf)
    (declare (ignore var-3 var-5))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldaf var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dsytrf uplo n af ldaf ipiv work lwork info)
    (declare (ignore var-2 var-4 var-5))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf ldaf var-3))
    (when var-6 (setf lwork var-6))
    (when var-7 (setf info var-7)))
  (cond ((> info 0) (setf rcond zero) (go end_label)))))
```

```
(setf anorm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlansy "I" uplo n a lda work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf uplo var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dsycon uplo n af ldaf ipiv anorm rcond work iwork info)
  (declare (ignore var-2 var-4 var-7 var-8))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf ldaf var-3))
  (when var-5 (setf anorm var-5))
  (when var-6 (setf rcond var-6))
  (when var-9 (setf info var-9)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "Full" n nrhs b ldb$ x ldx)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf ldx var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dsytrs uplo n nrhs af ldaf ipiv x ldx info)
  (declare (ignore var-3 var-5 var-6))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldaf var-4))
  (when var-7 (setf ldx var-7))
  (when var-8 (setf info var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16)
    (dsyrfs uplo n nrhs a lda af ldaf ipiv b ldb$ x ldx ferr berr work
     iwork info)
  (declare
   (ignore var-2 var-3 var-5 var-7 var-8 var-9 var-10 var-11 var-12
    var-13 var-14 var-15))
  (setf uplo var-0)
  (setf n var-1)
  (setf lda var-4)
  (setf ldaf var-6)
  (setf info var-16))
(if (< rcond (dlamch "Epsilon"))
```

```
            (setf info (f2cl-lib:int-add n 1)))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
        (go end_label)
       end_label
        (return
         (values fact uplo n nrhs nil lda nil ldaf nil nil ldb$ nil ldx rcond
                 nil nil nil lwork nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsysvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::fact
                                              fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldaf nil nil
                                              fortran-to-lisp::ldb$ nil
                                              fortran-to-lisp::ldx
                                              fortran-to-lisp::rcond nil nil
                                              nil fortran-to-lisp::lwork nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::dsyrfs))))
```

Chapter 53

LAPACK - Symmetric Matrix, Linear Solve, Real

53.1 ssysv

53.1.1 Axiom unit tests

— ssysv.input —

```
)set break resume
)sys rm -f ssysv.output
)spool ssysv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

53.1.2 Axiom help page

— ssysv.help —

```
==========================================================================
ssysv examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssysv (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK, LWORK,
           INFO)
            SSYSV computes the solution to system of linear equations A * X =
           B for SY matrices

Function/Subroutine Documentation
   subroutine ssysv (character UPLO, integer N, integer NRHS, real, dimension(
       lda, * ) A, integer LDA, integer, dimension( * ) IPIV, real, dimension(
       ldb, * ) B, integer LDB, real, dimension( * ) WORK, integer LWORK,
       integer INFO)
        SSYSV computes the solution to system of linear equations A * X = B
       for SY matrices

Purpose:
   SSYSV computes the solution to a real system of linear equations
     A * X = B,
   where A is an N-by-N symmetric matrix and X and B are N-by-NRHS
   matrices.

   The diagonal pivoting method is used to factor A as
     A = U * D * U**T,  if UPLO = 'U', or
     A = L * D * L**T,  if UPLO = 'L',
   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, and D is symmetric and block diagonal with
   1-by-1 and 2-by-2 diagonal blocks.  The factored form of A is then
   used to solve the system of equations A * X = B.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.
```

```
A
  A is REAL array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if INFO = 0, the block diagonal matrix D and the
  multipliers used to obtain the factor U or L from the
  factorization A = U*D*U**T or A = L*D*L**T as computed by SSYTRF.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D, as
  determined by SSYTRF.  If IPIV(k) > 0, then rows and columns
  k and IPIV(k) were interchanged, and D(k,k) is a 1-by-1
  diagonal block.  If UPLO = 'U' and IPIV(k) = IPIV(k-1) < 0,
  then rows and columns k-1 and -IPIV(k) were interchanged and
  D(k-1:k,k-1:k) is a 2-by-2 diagonal block.  If UPLO = 'L' and
  IPIV(k) = IPIV(k+1) < 0, then rows and columns k+1 and
  -IPIV(k) were interchanged and D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

B
  B is REAL array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of WORK.  LWORK >= 1, and for best performance
  LWORK >= max(1,N*NB), where NB is the optimal blocksize for
  SSYTRF.
  for LWORK < N, TRS will be done with Level BLAS 2
  for LWORK >= N, TRS will be done with Level BLAS 3
```

```
  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = i, D(i,i) is exactly zero.  The factorization
       has been completed, but the block diagonal matrix D is
       exactly singular, so the solution could not be computed.
```

———-

53.1.3 fortran code

— **ssysv.f** —

```
*  =====================================================================
      SUBROUTINE SSYSV( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK,
     $                  LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, LWORK, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            LWKOPT
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
```

```
*     .. External Subroutines ..
      EXTERNAL           XERBLA, SSYTRF, SSYTRS, SSYTRS2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -10
      END IF
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            LWKOPT = 1
         ELSE
            CALL SSYTRF( UPLO, N, A, LDA, IPIV, WORK, -1, INFO )
            LWKOPT = WORK(1)
         END IF
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYSV ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Compute the factorization A = U*D*U**T or A = L*D*L**T.
*
      CALL SSYTRF( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
```

```
         IF ( LWORK.LT.N ) THEN
*
*        Solve with TRS ( Use Level BLAS 2)
*
            CALL SSYTRS( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
         ELSE
*
*        Solve with TRS2 ( Use Level BLAS 3)
*
            CALL SSYTRS2( UPLO,N,NRHS,A,LDA,IPIV,B,LDB,WORK,INFO )
*
         END IF
*
      END IF
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of SSYSV
*
      END
```

———-

53.1.4 lisp code

— **ssysv.lisp** —

```
(defun ssysv (uplo n nrhs a lda ipiv b ldb$ work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array single-float (*)) work b a)
           (type (f2cl-lib:integer4) info lwork ldb$ lda nrhs n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a single-float a-%data% a-%offset%)
       (b single-float b-%data% b-%offset%)
       (work single-float work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((lwkopt 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) lwkopt))
      (setf info 0)
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond
       ((and
```

```
    (not
     (multiple-value-bind (ret-val var-0 var-1)
         (lsame uplo "U")
       (declare (ignore var-1))
       (when var-0 (setf uplo var-0))
       ret-val))
    (not
     (multiple-value-bind (ret-val var-0 var-1)
         (lsame uplo "L")
       (declare (ignore var-1))
       (when var-0 (setf uplo var-0))
       ret-val)))
   (setf info -1))
  ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
  ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
   (setf info -5))
  ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
   (setf info -8))
  ((and (< lwork 1) (not lquery)) (setf info -10)))
 (cond
  ((= info 0)
   (cond ((= n 0) (setf lwkopt 1))
         (t
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
              (ssytrf uplo n a lda ipiv work -1 info)
            (declare (ignore var-2 var-4 var-5 var-6))
            (when var-0 (setf uplo var-0))
            (when var-1 (setf n var-1))
            (when var-3 (setf lda var-3))
            (when var-7 (setf info var-7)))
          (setf lwkopt
                  (f2cl-lib:int
                   (f2cl-lib:fref work-%data% (1) ((1 *))
                                  work-%offset%)))))
   (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
           (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))))
 (cond
  ((/= info 0) (xerbla "SSYSV " (f2cl-lib:int-sub info)) (go end_label))
  (lquery (go end_label)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (ssytrf uplo n a lda ipiv work lwork info)
   (declare (ignore var-2 var-4 var-5))
   (when var-0 (setf uplo var-0))
   (when var-1 (setf n var-1))
   (when var-3 (setf lda var-3))
   (when var-6 (setf lwork var-6))
   (when var-7 (setf info var-7)))
 (cond
  ((= info 0)
```

```
        (cond
         ((< lwork n)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (ssytrs uplo n nrhs a lda ipiv b ldb$ info)
            (declare (ignore var-3 var-5 var-6))
            (when var-0 (setf uplo var-0))
            (when var-1 (setf n var-1))
            (when var-2 (setf nrhs var-2))
            (when var-4 (setf lda var-4))
            (when var-7 (setf ldb$ var-7))
            (when var-8 (setf info var-8))))
         (t
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (ssytrs2 uplo n nrhs a lda ipiv b ldb$ work info)
            (declare (ignore var-3 var-5 var-6 var-8))
            (when var-0 (setf uplo var-0))
            (when var-1 (setf n var-1))
            (when var-2 (setf nrhs var-2))
            (when var-4 (setf lda var-4))
            (when var-7 (setf ldb$ var-7))
            (when var-9 (setf info var-9)))))))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
      (go end_label)
     end_label
      (return (values uplo n nrhs nil lda nil nil ldb$ nil lwork info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssysv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
```

```
                                        fortran-to-lisp::lda nil nil
                                        fortran-to-lisp::ldb$ nil
                                        fortran-to-lisp::lwork
                                        fortran-to-lisp::info)
                                      :calls 'nil)))
```

———-

53.2 ssysv_rook

53.2.1 Axiom unit tests

— **ssysvrook.input** —

```
)set break resume
)sys rm -f ssysvrook.output
)spool ssysvrook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

53.2.2 Axiom help page

— **ssysv_rook.help** —

```
====================================================================
ssysv_rook examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssysv_rook (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK,
           LWORK, INFO)
```

```
           SSYSV_ROOK computes the solution to system of linear equations A *
          X = B for SY matrices

Function/Subroutine Documentation
   subroutine ssysv_rook (character UPLO, integer N, integer NRHS, real,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV, real,
       dimension( ldb, * ) B, integer LDB, real, dimension( * ) WORK, integer
       LWORK, integer INFO)
        SSYSV_ROOK computes the solution to system of linear equations A * X =
       B for SY matrices

Purpose:
   SSYSV_ROOK computes the solution to a real system of linear equations
      A * X = B,
   where A is an N-by-N symmetric matrix and X and B are N-by-NRHS
   matrices.

   The diagonal pivoting method is used to factor A as
      A = U * D * U**T,  if UPLO = 'U', or
      A = L * D * L**T,  if UPLO = 'L',
   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, and D is symmetric and block diagonal with
   1-by-1 and 2-by-2 diagonal blocks.

   SSYTRF_ROOK is called to compute the factorization of a real
   symmetric matrix A using the bounded Bunch-Kaufman ("rook") diagonal
   pivoting method.

   The factored form of A is then used to solve the system
   of equations A * X = B by calling SSYTRS_ROOK.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
```

```
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if INFO = 0, the block diagonal matrix D and the
  multipliers used to obtain the factor U or L from the
  factorization A = U*D*U**T or A = L*D*L**T as computed by SSYTRF_ROOK.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D,
  as determined by SSYTRF_ROOK.

  If UPLO = 'U':
     If IPIV(k) > 0, then rows and columns k and IPIV(k)
     were interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) < 0 and IPIV(k-1) < 0, then rows and
     columns k and -IPIV(k) were interchanged and rows and
     columns k-1 and -IPIV(k-1) were inerchaged,
     D(k-1:k,k-1:k) is a 2-by-2 diagonal block.

  If UPLO = 'L':
     If IPIV(k) > 0, then rows and columns k and IPIV(k)
     were interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) < 0 and IPIV(k+1) < 0, then rows and
     columns k and -IPIV(k) were interchanged and rows and
     columns k+1 and -IPIV(k+1) were inerchaged,
     D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

B
  B is REAL array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.
```

```
 LWORK
   LWORK is INTEGER
   The length of WORK.  LWORK >= 1, and for best performance
   LWORK >= max(1,N*NB), where NB is the optimal blocksize for SSYTRF_ROOK.

   TRS will be done with Level 2 BLAS

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value
   > 0: if INFO = i, D(i,i) is exactly zero.  The factorization
        has been completed, but the block diagonal matrix D is
        exactly singular, so the solution could not be computed.
```

53.2.3 fortran code

— ssysv_rook.f —

```
*  =====================================================================
      SUBROUTINE SSYSV_ROOK( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK,
     $                  LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, LWORK, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
```

```
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            LWKOPT
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, SSYTRF_ROOK, SSYTRS_ROOK
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -10
      END IF
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            LWKOPT = 1
         ELSE
            CALL SSYTRF_ROOK( UPLO, N, A, LDA, IPIV, WORK, -1, INFO )
            LWKOPT = WORK(1)
         END IF
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYSV_ROOK ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
```

```
*
*     Compute the factorization A = U*D*U**T or A = L*D*L**T.
*
      CALL SSYTRF_ROOK( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
*        Solve with TRS_ROOK ( Use Level 2 BLAS)
*
         CALL SSYTRS_ROOK( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
      END IF
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of SSYSV_ROOK
*
      END
```

———-

53.2.4 lisp code

— **ssysv_rook.lisp** —

```
(defun ssysv_rook (uplo n nrhs a lda ipiv b ldb$ work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array single-float (*)) work b a)
           (type (f2cl-lib:integer4) info lwork ldb$ lda nrhs n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a single-float a-%data% a-%offset%)
       (b single-float b-%data% b-%offset%)
       (work single-float work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((lwkopt 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) lwkopt))
      (setf info 0)
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond
       ((and
         (not
          (multiple-value-bind (ret-val var-0 var-1)
```

```
          (lsame uplo "U")
        (declare (ignore var-1))
        (when var-0 (setf uplo var-0))
        ret-val))
     (not
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame uplo "L")
        (declare (ignore var-1))
        (when var-0 (setf uplo var-0))
        ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -5))
   ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -8))
   ((and (< lwork 1) (not lquery)) (setf info -10)))
  (cond
   ((= info 0)
    (cond ((= n 0) (setf lwkopt 1))
          (t
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
               (ssytrf_rook uplo n a lda ipiv work -1 info)
             (declare (ignore var-2 var-4 var-5 var-6))
             (when var-0 (setf uplo var-0))
             (when var-1 (setf n var-1))
             (when var-3 (setf lda var-3))
             (when var-7 (setf info var-7)))
           (setf lwkopt
                   (f2cl-lib:int
                    (f2cl-lib:fref work-%data% (1) ((1 *))
                                   work-%offset%)))))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))))
  (cond
   ((/= info 0) (xerbla "SSYSV_ROOK " (f2cl-lib:int-sub info))
    (go end_label))
   (lquery (go end_label)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (ssytrf_rook uplo n a lda ipiv work lwork info)
    (declare (ignore var-2 var-4 var-5))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf lda var-3))
    (when var-6 (setf lwork var-6))
    (when var-7 (setf info var-7)))
  (cond
   ((= info 0)
    (multiple-value-bind
```

```
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (ssytrs_rook uplo n nrhs a lda ipiv b ldb$ info)
          (declare (ignore var-3 var-5 var-6))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf lda var-4))
          (when var-7 (setf ldb$ var-7))
          (when var-8 (setf info var-8)))))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
      (go end_label)
     end_label
      (return (values uplo n nrhs nil lda nil nil ldb$ nil lwork info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssysv_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::ldb$ nil
                                              fortran-to-lisp::lwork
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

53.3 ssysvx

53.3.1 Axiom unit tests

— ssysvx.input —

```
)set break resume
)sys rm -f ssysvx.output
)spool ssysvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

53.3.2 Axiom help page

— ssysvx.help —

```
======================================================================
ssysvx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssysvx (FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB,
           X, LDX, RCOND, FERR, BERR, WORK, LWORK, IWORK, INFO)
            SSYSVX computes the solution to system of linear equations A * X =
           B for SY matrices

Function/Subroutine Documentation
   subroutine ssysvx (character FACT, character UPLO, integer N, integer NRHS,
       real, dimension( lda, * ) A, integer LDA, real, dimension( ldaf, * )
       AF, integer LDAF, integer, dimension( * ) IPIV, real, dimension( ldb, *
       ) B, integer LDB, real, dimension( ldx, * ) X, integer LDX, real RCOND,
       real, dimension( * ) FERR, real, dimension( * ) BERR, real, dimension(
       * ) WORK, integer LWORK, integer, dimension( * ) IWORK, integer INFO)
        SSYSVX computes the solution to system of linear equations A * X = B
       for SY matrices
```

```
Purpose:
   SSYSVX uses the diagonal pivoting factorization to compute the
   solution to a real system of linear equations A * X = B,
   where A is an N-by-N symmetric matrix and X and B are N-by-NRHS
   matrices.

   Error bounds on the solution and a condition estimate are also provided.

Description:

   The following steps are performed:

   1. If FACT = 'N', the diagonal pivoting method is used to factor A.
      The form of the factorization is
        A = U * D * U**T,  if UPLO = 'U', or
        A = L * D * L**T,  if UPLO = 'L',
      where U (or L) is a product of permutation and unit upper (lower)
      triangular matrices, and D is symmetric and block diagonal with
      1-by-1 and 2-by-2 diagonal blocks.

   2. If some D(i,i)=0, so that D is exactly singular, then the routine
      returns with INFO = i. Otherwise, the factored form of A is used
      to estimate the condition number of the matrix A.  If the
      reciprocal of the condition number is less than machine precision,
      INFO = N+1 is returned as a warning, but the routine still goes on
      to solve for X and compute error bounds as described below.

   3. The system of equations is solved for X using the factored form of A.

   4. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of A has been
   supplied on entry.
   = 'F':  On entry, AF and IPIV contain the factored form of A.
           AF and IPIV will not be modified.
   = 'N':  The matrix A will be copied to AF and factored.

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
```

```
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is REAL array, dimension (LDA,N)
  The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
  upper triangular part of A contains the upper triangular part
  of the matrix A, and the strictly lower triangular part of A
  is not referenced.  If UPLO = 'L', the leading N-by-N lower
  triangular part of A contains the lower triangular part of
  the matrix A, and the strictly upper triangular part of A is
  not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is REAL array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the block diagonal matrix D and the multipliers used
  to obtain the factor U or L from the factorization
  A = U*D*U**T or A = L*D*L**T as computed by SSYTRF.

  If FACT = 'N', then AF is an output argument and on exit
  returns the block diagonal matrix D and the multipliers used
  to obtain the factor U or L from the factorization
  A = U*D*U**T or A = L*D*L**T.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains details of the interchanges and the block structure
  of D, as determined by SSYTRF.
  If IPIV(k) > 0, then rows and columns k and IPIV(k) were
  interchanged and D(k,k) is a 1-by-1 diagonal block.
  If UPLO = 'U' and IPIV(k) = IPIV(k-1) < 0, then rows and
  columns k-1 and -IPIV(k) were interchanged and D(k-1:k,k-1:k)
  is a 2-by-2 diagonal block.  If UPLO = 'L' and IPIV(k) =
  IPIV(k+1) < 0, then rows and columns k+1 and -IPIV(k) were
  interchanged and D(k:k+1,k:k+1) is a 2-by-2 diagonal block.
```

```
  If FACT = 'N', then IPIV is an output argument and on exit
  contains details of the interchanges and the block structure
  of D, as determined by SSYTRF.

B
  B is REAL array, dimension (LDB,NRHS)
  The N-by-NRHS right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is REAL array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  The estimate of the reciprocal condition number of the matrix
  A.  If RCOND is less than the machine precision (in
  particular, if RCOND = 0), the matrix is singular to working
  precision.  This condition is indicated by a return code of INFO > 0.

FERR
  FERR is REAL array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is REAL array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
```

```
  The length of WORK.  LWORK >= max(1,3*N), and for best
  performance, when FACT = 'N', LWORK >= max(1,3*N,N*NB), where
  NB is the optimal blocksize for SSYTRF.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = i, and i is
    <= N:  D(i,i) is exactly zero.  The factorization
       has been completed but the factor D is exactly
       singular, so the solution and error bounds could
       not be computed. RCOND = 0 is returned.
    = N+1: D is nonsingular, but RCOND is less than machine
       precision, meaning that the matrix is singular
       to working precision.  Nevertheless, the
       solution and error bounds are computed because
       there are a number of situations where the
       computed solution can be more accurate than the
       value of RCOND would suggest.
```

53.3.3 fortran code

— ssysvx.f —

```
*  ==================================================================
      SUBROUTINE SSYSVX( FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B,
     $                   LDB, X, LDX, RCOND, FERR, BERR, WORK, LWORK,
     $                   IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          FACT, UPLO
```

```
      INTEGER            INFO, LDA, LDAF, LDB, LDX, LWORK, N, NRHS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      REAL               A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   BERR( * ), FERR( * ), WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, NOFACT
      INTEGER            LWKOPT, NB
      REAL               ANORM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               SLAMCH, SLANSY
      EXTERNAL           ILAENV, LSAME, SLAMCH, SLANSY
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLACPY, SSYCON, SSYRFS, SSYTRF, SSYTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.NOFACT .AND. .NOT.LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) )
     $          THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
```

```
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -11
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -13
      ELSE IF( LWORK.LT.MAX( 1, 3*N ) .AND. .NOT.LQUERY ) THEN
         INFO = -18
      END IF
*
      IF( INFO.EQ.0 ) THEN
         LWKOPT = MAX( 1, 3*N )
         IF( NOFACT ) THEN
            NB = ILAENV( 1, 'SSYTRF', UPLO, N, -1, -1, -1 )
            LWKOPT = MAX( LWKOPT, N*NB )
         END IF
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYSVX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
      IF( NOFACT ) THEN
*
*        Compute the factorization A = U*D*U**T or A = L*D*L**T.
*
         CALL SLACPY( UPLO, N, N, A, LDA, AF, LDAF )
         CALL SSYTRF( UPLO, N, AF, LDAF, IPIV, WORK, LWORK, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 )THEN
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A.
*
      ANORM = SLANSY( 'I', UPLO, N, A, LDA, WORK )
*
*     Compute the reciprocal of the condition number of A.
*
      CALL SSYCON( UPLO, N, AF, LDAF, IPIV, ANORM, RCOND, WORK, IWORK,
     $             INFO )
*
```

```
*     Compute the solution vectors X.
*
      CALL SLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL SSYTRS( UPLO, N, NRHS, AF, LDAF, IPIV, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solutions and
*     compute error bounds and backward error estimates for them.
*
      CALL SSYRFS( UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB, X,
     $             LDX, FERR, BERR, WORK, IWORK, INFO )
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.SLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of SSYSVX
*
      END
```

———-

53.3.4 lisp code

— **ssysvx.lisp** —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun ssysvx
         (fact uplo n nrhs a lda af ldaf ipiv b ldb$ x ldx rcond ferr berr work
          lwork iwork info)
    (declare (type (single-float) rcond)
             (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array single-float (*)) work berr ferr x b af a)
             (type (f2cl-lib:integer4) info lwork ldx ldb$ ldaf lda nrhs n)
             (type (string *) uplo fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (af single-float af-%data% af-%offset%)
         (b single-float b-%data% b-%offset%)
         (x single-float x-%data% x-%offset%)
```

```
    (ferr single-float ferr-%data% ferr-%offset%)
    (berr single-float berr-%data% berr-%offset%)
    (work single-float work-%data% work-%offset%)
    (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
    (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((anorm 0.0) (lwkopt 0) (nb 0) (lquery nil) (nofact nil))
    (declare (type (single-float) anorm)
             (type (f2cl-lib:integer4) lwkopt nb)
             (type f2cl-lib:logical lquery nofact))
    (setf info 0)
    (setf nofact
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "N")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val))
    (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
    (cond
     ((and (not nofact)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "F")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val)))
      (setf info -1))
     ((and
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
      (setf info -2))
     ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
     ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -6))
     ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -8))
     ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -11))
     ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -13))
     ((and
```

```
   (< lwork
      (max (the f2cl-lib:integer4 1)
           (the f2cl-lib:integer4 (f2cl-lib:int-mul 3 n))))
   (not lquery))
  (setf info -18)))
(cond
 ((= info 0)
  (setf lwkopt
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul 3 n))))
  (cond
   (nofact
    (setf nb
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 1 "SSYTRF" uplo n -1 -1 -1)
              (declare (ignore var-0 var-1 var-4 var-5 var-6))
              (setf uplo var-2)
              (setf n var-3)
              ret-val))
    (setf lwkopt
            (max (the f2cl-lib:integer4 lwkopt)
                 (the f2cl-lib:integer4 (f2cl-lib:int-mul n nb))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))))
(cond
 ((/= info 0) (xerbla "SSYSVX" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond
 (nofact
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy uplo n n a lda af ldaf)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
    (setf uplo var-0))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (ssytrf uplo n af ldaf ipiv work lwork info)
    (declare (ignore var-2 var-4 var-5))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf ldaf var-3))
    (when var-6 (setf lwork var-6))
    (when var-7 (setf info var-7)))
  (cond ((> info 0) (setf rcond zero) (go end_label)))))
(setf anorm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slansy "I" uplo n a lda work)
          (declare (ignore var-0 var-3 var-4 var-5))
          (setf uplo var-1)
```

```
                 (setf n var-2)
                 ret-val))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
            (ssycon uplo n af ldaf ipiv anorm rcond work iwork info)
          (declare (ignore var-2 var-4 var-5 var-7 var-8))
          (setf uplo var-0)
          (setf n var-1)
          (setf ldaf var-3)
          (setf rcond var-6)
          (setf info var-9))
        (slacpy "Full" n nrhs b ldb$ x ldx)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (ssytrs uplo n nrhs af ldaf ipiv x ldx info)
          (declare (ignore var-3 var-5 var-6))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldaf var-4))
          (when var-7 (setf ldx var-7))
          (when var-8 (setf info var-8)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
             var-11 var-12 var-13 var-14 var-15 var-16)
            (ssyrfs uplo n nrhs a lda af ldaf ipiv b ldb$ x ldx ferr berr work
             iwork info)
          (declare
           (ignore var-2 var-3 var-5 var-7 var-8 var-9 var-10 var-11 var-12
            var-13 var-14 var-15))
          (setf uplo var-0)
          (setf n var-1)
          (setf lda var-4)
          (setf ldaf var-6)
          (setf info var-16))
        (if (< rcond (slamch "Epsilon"))
            (setf info (f2cl-lib:int-add n 1)))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
        (go end_label)
       end_label
        (return
         (values fact uplo n nrhs nil lda nil ldaf nil nil nil nil ldx rcond
                 nil nil nil lwork nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssysvx
                 fortran-to-lisp::*f2cl-function-info*)
```

```
(fortran-to-lisp::make-f2cl-finfo :arg-types
                                  '((string) (string)
                                    (fortran-to-lisp::integer4)
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array fortran-to-lisp::integer4
                                     (*))
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (single-float)
                                    (array single-float (*))
                                    (array single-float (*))
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array fortran-to-lisp::integer4
                                     (*))
                                    (fortran-to-lisp::integer4))
                                  :return-values
                                  '(fortran-to-lisp::fact
                                    fortran-to-lisp::uplo
                                    fortran-to-lisp::n
                                    fortran-to-lisp::nrhs nil
                                    fortran-to-lisp::lda nil
                                    fortran-to-lisp::ldaf nil nil nil
                                    nil fortran-to-lisp::ldx
                                    fortran-to-lisp::rcond nil nil
                                    nil fortran-to-lisp::lwork nil
                                    fortran-to-lisp::info)
                                  :calls
                                  '(fortran-to-lisp::ssyrfs
                                    fortran-to-lisp::ssycon
                                    fortran-to-lisp::slansy
                                    fortran-to-lisp::slacpy
                                    fortran-to-lisp::ilaenv))))
```

Chapter 54

LAPACK - Symmetric Matrix, Linear Solve, Complex

54.1 csysv

54.1.1 Axiom unit tests

— **csysv.input** —

```
)set break resume
)sys rm -f csysv.output
)spool csysv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

54.1.2 Axiom help page

— **csysv.help** —

```
==========================================================================
csysv examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csysv (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK, LWORK,
           INFO)
            CSYSV computes the solution to system of linear equations A * X =
           B for SY matrices

Function/Subroutine Documentation
   subroutine csysv (character UPLO, integer N, integer NRHS, complex,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       complex, dimension( ldb, * ) B, integer LDB, complex, dimension( * )
       WORK, integer LWORK, integer INFO)
        CSYSV computes the solution to system of linear equations A * X = B
       for SY matrices

Purpose:
   CSYSV computes the solution to a complex system of linear equations
      A * X = B,
   where A is an N-by-N symmetric matrix and X and B are N-by-NRHS
   matrices.

   The diagonal pivoting method is used to factor A as
      A = U * D * U**T,  if UPLO = 'U', or
      A = L * D * L**T,  if UPLO = 'L',
   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, and D is symmetric and block diagonal with
   1-by-1 and 2-by-2 diagonal blocks.  The factored form of A is then
   used to solve the system of equations A * X = B.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.
```

```
A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if INFO = 0, the block diagonal matrix D and the
  multipliers used to obtain the factor U or L from the
  factorization A = U*D*U**T or A = L*D*L**T as computed by CSYTRF.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D, as
  determined by CSYTRF.  If IPIV(k) > 0, then rows and columns
  k and IPIV(k) were interchanged, and D(k,k) is a 1-by-1
  diagonal block.  If UPLO = 'U' and IPIV(k) = IPIV(k-1) < 0,
  then rows and columns k-1 and -IPIV(k) were interchanged and
  D(k-1:k,k-1:k) is a 2-by-2 diagonal block.  If UPLO = 'L' and
  IPIV(k) = IPIV(k+1) < 0, then rows and columns k+1 and
  -IPIV(k) were interchanged and D(k:k+1,k:k+1) is a 2-by-2
  diagonal block.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of WORK.  LWORK >= 1, and for best performance
  LWORK >= max(1,N*NB), where NB is the optimal blocksize for
  CSYTRF.
  for LWORK < N, TRS will be done with Level BLAS 2
  for LWORK >= N, TRS will be done with Level BLAS 3
```

```
  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = i, D(i,i) is exactly zero.  The factorization
       has been completed, but the block diagonal matrix D is
       exactly singular, so the solution could not be computed.
```

———-

54.1.3 fortran code

— **csysv.f** —

```
*  =====================================================================
      SUBROUTINE CSYSV( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK,
     $                  LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, LWORK, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            LWKOPT
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
```

```
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, CSYTRF, CSYTRS, CSYTRS2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -10
      END IF
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            LWKOPT = 1
         ELSE
            CALL CSYTRF( UPLO, N, A, LDA, IPIV, WORK, -1, INFO )
            LWKOPT = WORK(1)
         END IF
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CSYSV ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Compute the factorization A = U*D*U**T or A = L*D*L**T.
*
      CALL CSYTRF( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
```

```
*
         IF ( LWORK.LT.N ) THEN
*
*        Solve with TRS ( Use Level BLAS 2)
*
            CALL CSYTRS( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
         ELSE
*
*        Solve with TRS2 ( Use Level BLAS 3)
*
            CALL CSYTRS2( UPLO,N,NRHS,A,LDA,IPIV,B,LDB,WORK,INFO )
*
         END IF
*
      END IF
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of CSYSV
*
      END
```

———-

54.1.4 lisp code

— **csysv.lisp** —

```
(defun csysv (uplo n nrhs a lda ipiv b ldb$ work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex8 (*)) work b a)
           (type (f2cl-lib:integer4) info lwork ldb$ lda nrhs n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (b f2cl-lib:complex8 b-%data% b-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((lwkopt 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) lwkopt))
      (setf info 0)
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond
```

```
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and (< lwork 1) (not lquery)) (setf info -10)))
(cond
 ((= info 0)
  (cond ((= n 0) (setf lwkopt 1))
        (t
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
             (csytrf uplo n a lda ipiv work -1 info)
           (declare (ignore var-2 var-4 var-5 var-6))
           (when var-0 (setf uplo var-0))
           (when var-1 (setf n var-1))
           (when var-3 (setf lda var-3))
           (when var-7 (setf info var-7)))
         (setf lwkopt
                 (f2cl-lib:int
                  (f2cl-lib:fref work-%data% (1) ((1 *))
                                 work-%offset%)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwkopt 'f2cl-lib:complex8))))
(cond
 ((/= info 0) (xerbla "CSYSV " (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (csytrf uplo n a lda ipiv work lwork info)
  (declare (ignore var-2 var-4 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-6 (setf lwork var-6))
  (when var-7 (setf info var-7)))
(cond
```

```
      ((= info 0)
       (cond
        ((< lwork n)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
             (csytrs uplo n nrhs a lda ipiv b ldb$ info)
           (declare (ignore var-3 var-5 var-6))
           (when var-0 (setf uplo var-0))
           (when var-1 (setf n var-1))
           (when var-2 (setf nrhs var-2))
           (when var-4 (setf lda var-4))
           (when var-7 (setf ldb$ var-7))
           (when var-8 (setf info var-8))))
        (t
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
             (csytrs2 uplo n nrhs a lda ipiv b ldb$ work info)
           (declare (ignore var-3 var-5 var-6 var-8))
           (when var-0 (setf uplo var-0))
           (when var-1 (setf n var-1))
           (when var-2 (setf nrhs var-2))
           (when var-4 (setf lda var-4))
           (when var-7 (setf ldb$ var-7))
           (when var-9 (setf info var-9)))))))
     (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
             (coerce lwkopt 'f2cl-lib:complex8))
     (go end_label)
    end_label
     (return (values uplo n nrhs nil lda nil nil ldb$ nil lwork info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csysv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
```

```
:return-values
'(fortran-to_lisp::uplo
  fortran-to_lisp::n
  fortran-to_lisp::nrhs nil
  fortran-to_lisp::lda nil nil
  fortran-to_lisp::ldb$ nil
  fortran-to_lisp::lwork
  fortran-to_lisp::info)
:calls 'nil)))
```

———-

54.2 csysv_rook

54.2.1 Axiom unit tests

— **csysvrook.input** —

```
)set break resume
)sys rm -f csysvrook.output
)spool csysvrook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

54.2.2 Axiom help page

— **csysv_rook.help** —

```
==========================================================================
csysv_rook examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine csysv_rook (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK,
           LWORK, INFO)
            CSYSV_ROOK computes the solution to system of linear equations A *
           X = B for SY matrices

Function/Subroutine Documentation
   subroutine csysv_rook (character UPLO, integer N, integer NRHS, complex,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       complex, dimension( ldb, * ) B, integer LDB, complex, dimension( * )
       WORK, integer LWORK, integer INFO)
        CSYSV_ROOK computes the solution to system of linear equations A * X =
       B for SY matrices

Purpose:
   CSYSV_ROOK computes the solution to a complex system of linear
   equations
     A * X = B,
   where A is an N-by-N symmetric matrix and X and B are N-by-NRHS
   matrices.

   The diagonal pivoting method is used to factor A as
     A = U * D * U**T,  if UPLO = 'U', or
     A = L * D * L**T,  if UPLO = 'L',
   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, and D is symmetric and block diagonal with
   1-by-1 and 2-by-2 diagonal blocks.

   CSYTRF_ROOK is called to compute the factorization of a complex
   symmetric matrix A using the bounded Bunch-Kaufman ("rook") diagonal
   pivoting method.

   The factored form of A is then used to solve the system
   of equations A * X = B by calling CSYTRS_ROOK.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
```

```
  of the matrix B.  NRHS >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if INFO = 0, the block diagonal matrix D and the
  multipliers used to obtain the factor U or L from the
  factorization A = U*D*U**T or A = L*D*L**T as computed by CSYTRF_ROOK.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D,
  as determined by CSYTRF_ROOK.

  If UPLO = 'U':
    If IPIV(k) > 0, then rows and columns k and IPIV(k)
    were interchanged and D(k,k) is a 1-by-1 diagonal block.

    If IPIV(k) < 0 and IPIV(k-1) < 0, then rows and
    columns k and -IPIV(k) were interchanged and rows and
    columns k-1 and -IPIV(k-1) were inerchaged,
    D(k-1:k,k-1:k) is a 2-by-2 diagonal block.

  If UPLO = 'L':
    If IPIV(k) > 0, then rows and columns k and IPIV(k)
    were interchanged and D(k,k) is a 1-by-1 diagonal block.

    If IPIV(k) < 0 and IPIV(k+1) < 0, then rows and
    columns k and -IPIV(k) were interchanged and rows and
    columns k+1 and -IPIV(k+1) were inerchaged,
    D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
```

```
  The leading dimension of the array B.  LDB >= max(1,N).

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of WORK.  LWORK >= 1, and for best performance
  LWORK >= max(1,N*NB), where NB is the optimal blocksize for
  CSYTRF_ROOK.

  TRS will be done with Level 2 BLAS

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = i, D(i,i) is exactly zero.  The factorization
       has been completed, but the block diagonal matrix D is
       exactly singular, so the solution could not be computed.
```

54.2.3 fortran code

— csysv_rook.f —

```
*  =====================================================================
      SUBROUTINE CSYSV_ROOK( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK,
     $                  LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, LWORK, N, NRHS
*     ..
*     .. Array Arguments ..
```

```
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            LWKOPT
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, CSYTRF_ROOK, CSYTRS_ROOK
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -10
      END IF
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            LWKOPT = 1
         ELSE
            CALL CSYTRF_ROOK( UPLO, N, A, LDA, IPIV, WORK, -1, INFO )
            LWKOPT = WORK(1)
         END IF
         WORK( 1 ) = LWKOPT
      END IF
*
```

```
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CSYSV_ROOK ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Compute the factorization A = U*D*U**T or A = L*D*L**T.
*
      CALL CSYTRF_ROOK( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
*        Solve with TRS_ROOK ( Use Level 2 BLAS)
*
         CALL CSYTRS_ROOK( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
      END IF
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of CSYSV_ROOK
*
      END
```

———-

54.2.4 lisp code

— **csysv_rook.lisp** —

```
(defun csysv_rook (uplo n nrhs a lda ipiv b ldb$ work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex8 (*)) work b a)
           (type (f2cl-lib:integer4) info lwork ldb$ lda nrhs n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (b f2cl-lib:complex8 b-%data% b-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((lwkopt 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) lwkopt))
```

```
(setf info 0)
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and (< lwork 1) (not lquery)) (setf info -10)))
(cond
 ((= info 0)
  (cond ((= n 0) (setf lwkopt 1))
        (t
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
             (csytrf_rook uplo n a lda ipiv work -1 info)
           (declare (ignore var-2 var-4 var-5 var-6))
           (when var-0 (setf uplo var-0))
           (when var-1 (setf n var-1))
           (when var-3 (setf lda var-3))
           (when var-7 (setf info var-7)))
         (setf lwkopt
                 (f2cl-lib:int
                  (f2cl-lib:fref work-%data% (1) ((1 *))
                                 work-%offset%)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwkopt 'f2cl-lib:complex8))))
(cond
 ((/= info 0) (xerbla "CSYSV_ROOK " (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (csytrf_rook uplo n a lda ipiv work lwork info)
  (declare (ignore var-2 var-4 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
```

```
        (when var-3 (setf lda var-3))
        (when var-6 (setf lwork var-6))
        (when var-7 (setf info var-7)))
      (cond
       ((= info 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (csytrs_rook uplo n nrhs a lda ipiv b ldb$ info)
          (declare (ignore var-3 var-5 var-6))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf lda var-4))
          (when var-7 (setf ldb$ var-7))
          (when var-8 (setf info var-8)))))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce lwkopt 'f2cl-lib:complex8))
      (go end_label)
     end_label
      (return (values uplo n nrhs nil lda nil nil ldb$ nil lwork info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csysv_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::ldb$ nil
                                              fortran-to-lisp::lwork
                                              fortran-to-lisp::info)
```

```
                                        :calls 'nil)))
```

———-

54.3 csysvx

54.3.1 Axiom unit tests

— **csysvx.input** —

```
)set break resume
)sys rm -f csysvx.output
)spool csysvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

54.3.2 Axiom help page

— **csysvx.help** —

```
======================================================================
csysvx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csysvx (FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB,
           X, LDX, RCOND, FERR, BERR, WORK, LWORK, RWORK, INFO)
            CSYSVX computes the solution to system of linear equations A * X =
           B for SY matrices

Function/Subroutine Documentation
```

```
   subroutine csysvx (character FACT, character UPLO, integer N, integer NRHS,
       complex, dimension( lda, * ) A, integer LDA, complex, dimension( ldaf,
       * ) AF, integer LDAF, integer, dimension( * ) IPIV, complex, dimension(
       ldb, * ) B, integer LDB, complex, dimension( ldx, * ) X, integer LDX,
       real RCOND, real, dimension( * ) FERR, real, dimension( * ) BERR,
       complex, dimension( * ) WORK, integer LWORK, real, dimension( * )
       RWORK, integer INFO)
        CSYSVX computes the solution to system of linear equations A * X = B
       for SY matrices

Purpose:
   CSYSVX uses the diagonal pivoting factorization to compute the
   solution to a complex system of linear equations A * X = B,
   where A is an N-by-N symmetric matrix and X and B are N-by-NRHS
   matrices.

   Error bounds on the solution and a condition estimate are also provided.

Description:

   The following steps are performed:

   1. If FACT = 'N', the diagonal pivoting method is used to factor A.
      The form of the factorization is
        A = U * D * U**T,  if UPLO = 'U', or
        A = L * D * L**T,  if UPLO = 'L',
      where U (or L) is a product of permutation and unit upper (lower)
      triangular matrices, and D is symmetric and block diagonal with
      1-by-1 and 2-by-2 diagonal blocks.

   2. If some D(i,i)=0, so that D is exactly singular, then the routine
      returns with INFO = i. Otherwise, the factored form of A is used
      to estimate the condition number of the matrix A.  If the
      reciprocal of the condition number is less than machine precision,
      INFO = N+1 is returned as a warning, but the routine still goes on
      to solve for X and compute error bounds as described below.

   3. The system of equations is solved for X using the factored form of A.

   4. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of A has been
   supplied on entry.
   = 'F':  On entry, AF and IPIV contain the factored form of A.
           A, AF and IPIV will not be modified.
```

```
  = 'N':  The matrix A will be copied to AF and factored.

UPLO
  UPLO is CHARACTER*1
  = 'U':  Upper triangle of A is stored;
  = 'L':  Lower triangle of A is stored.

N
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
  upper triangular part of A contains the upper triangular part
  of the matrix A, and the strictly lower triangular part of A
  is not referenced.  If UPLO = 'L', the leading N-by-N lower
  triangular part of A contains the lower triangular part of
  the matrix A, and the strictly upper triangular part of A is
  not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the block diagonal matrix D and the multipliers used
  to obtain the factor U or L from the factorization
  A = U*D*U**T or A = L*D*L**T as computed by CSYTRF.

  If FACT = 'N', then AF is an output argument and on exit
  returns the block diagonal matrix D and the multipliers used
  to obtain the factor U or L from the factorization
  A = U*D*U**T or A = L*D*L**T.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains details of the interchanges and the block structure
```

```
  of D, as determined by CSYTRF.
  If IPIV(k) > 0, then rows and columns k and IPIV(k) were
  interchanged and D(k,k) is a 1-by-1 diagonal block.
  If UPLO = 'U' and IPIV(k) = IPIV(k-1) < 0, then rows and
  columns k-1 and -IPIV(k) were interchanged and D(k-1:k,k-1:k)
  is a 2-by-2 diagonal block.  If UPLO = 'L' and IPIV(k) =
  IPIV(k+1) < 0, then rows and columns k+1 and -IPIV(k) were
  interchanged and D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

  If FACT = 'N', then IPIV is an output argument and on exit
  contains details of the interchanges and the block structure
  of D, as determined by CSYTRF.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  The N-by-NRHS right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  The estimate of the reciprocal condition number of the matrix
  A.  If RCOND is less than the machine precision (in
  particular, if RCOND = 0), the matrix is singular to working
  precision.  This condition is indicated by a return code of INFO > 0.

FERR
  FERR is REAL array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is REAL array, dimension (NRHS)
  The componentwise relative backward error of each solution
```

```
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of WORK.  LWORK >= max(1,2*N), and for best
  performance, when FACT = 'N', LWORK >= max(1,2*N,N*NB), where
  NB is the optimal blocksize for CSYTRF.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is REAL array, dimension (N)

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = i, and i is
    <= N:  D(i,i) is exactly zero.  The factorization
       has been completed but the factor D is exactly
       singular, so the solution and error bounds could
       not be computed. RCOND = 0 is returned.
    = N+1: D is nonsingular, but RCOND is less than machine
       precision, meaning that the matrix is singular
       to working precision.  Nevertheless, the
       solution and error bounds are computed because
       there are a number of situations where the
       computed solution can be more accurate than the
       value of RCOND would suggest.
```

———-

54.3.3 fortran code

— **csysvx.f** —

```
*  =====================================================================
      SUBROUTINE CSYSVX( FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B,
     $                   LDB, X, LDX, RCOND, FERR, BERR, WORK, LWORK,
```

```
     $                   RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          FACT, UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, LWORK, N, NRHS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               BERR( * ), FERR( * ), RWORK( * )
      COMPLEX            A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, NOFACT
      INTEGER            LWKOPT, NB
      REAL               ANORM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               CLANSY, SLAMCH
      EXTERNAL           ILAENV, LSAME, CLANSY, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLACPY, CSYCON, CSYRFS, CSYTRF, CSYTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.NOFACT .AND. .NOT.LSAME( FACT, 'F' ) ) THEN
```

```
         INFO = -1
      ELSE IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) )
     $          THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -11
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -13
      ELSE IF( LWORK.LT.MAX( 1, 2*N ) .AND. .NOT.LQUERY ) THEN
         INFO = -18
      END IF
*
      IF( INFO.EQ.0 ) THEN
         LWKOPT = MAX( 1, 2*N )
         IF( NOFACT ) THEN
            NB = ILAENV( 1, 'CSYTRF', UPLO, N, -1, -1, -1 )
            LWKOPT = MAX( LWKOPT, N*NB )
         END IF
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CSYSVX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
      IF( NOFACT ) THEN
*
*        Compute the factorization A = U*D*U**T or A = L*D*L**T.
*
         CALL CLACPY( UPLO, N, N, A, LDA, AF, LDAF )
         CALL CSYTRF( UPLO, N, AF, LDAF, IPIV, WORK, LWORK, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 )THEN
            RCOND = ZERO
            RETURN
         END IF
      END IF
```

```
*
*     Compute the norm of the matrix A.
*
      ANORM = CLANSY( 'I', UPLO, N, A, LDA, RWORK )
*
*     Compute the reciprocal of the condition number of A.
*
      CALL CSYCON( UPLO, N, AF, LDAF, IPIV, ANORM, RCOND, WORK, INFO )
*
*     Compute the solution vectors X.
*
      CALL CLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL CSYTRS( UPLO, N, NRHS, AF, LDAF, IPIV, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solutions and
*     compute error bounds and backward error estimates for them.
*
      CALL CSYRFS( UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB, X,
     $             LDX, FERR, BERR, WORK, RWORK, INFO )
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.SLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of CSYSVX
*
      END
```

———-

54.3.4 lisp code

— **csysvx.lisp** —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun csysvx
         (fact uplo n nrhs a lda af ldaf ipiv b ldb$ x ldx rcond ferr berr work
          lwork rwork info)
    (declare (type (array single-float (*)) rwork berr ferr)
             (type (single-float) rcond)
             (type (array f2cl-lib:integer4 (*)) ipiv)
```

```
          (type (array f2cl-lib:complex8 (*)) work x b af a)
          (type (f2cl-lib:integer4) info lwork ldx ldb$ ldaf lda nrhs n)
          (type (string *) uplo fact))
  (f2cl-lib:with-multi-array-data
      ((fact character fact-%data% fact-%offset%)
       (uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (af f2cl-lib:complex8 af-%data% af-%offset%)
       (b f2cl-lib:complex8 b-%data% b-%offset%)
       (x f2cl-lib:complex8 x-%data% x-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (ferr single-float ferr-%data% ferr-%offset%)
       (berr single-float berr-%data% berr-%offset%)
       (rwork single-float rwork-%data% rwork-%offset%))
    (prog ((anorm 0.0) (lwkopt 0) (nb 0) (lquery nil) (nofact nil))
      (declare (type (single-float) anorm)
               (type (f2cl-lib:integer4) lwkopt nb)
               (type f2cl-lib:logical lquery nofact))
      (setf info 0)
      (setf nofact
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame fact "N")
                (declare (ignore var-1))
                (when var-0 (setf fact var-0))
                ret-val))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond
       ((and (not nofact)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame fact "F")
                (declare (ignore var-1))
                (when var-0 (setf fact var-0))
                ret-val)))
        (setf info -1))
       ((and
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
        (setf info -2))
```

```
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -11))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -13))
 ((and
   (< lwork
      (max (the f2cl-lib:integer4 1)
           (the f2cl-lib:integer4 (f2cl-lib:int-mul 2 n))))
   (not lquery))
  (setf info -18)))
(cond
 ((= info 0)
  (setf lwkopt
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul 2 n))))
  (cond
   (nofact
    (setf nb
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 1 "CSYTRF" uplo n -1 -1 -1)
              (declare (ignore var-0 var-1 var-4 var-5 var-6))
              (when var-2 (setf uplo var-2))
              (when var-3 (setf n var-3))
              ret-val))
    (setf lwkopt
            (max (the f2cl-lib:integer4 lwkopt)
                 (the f2cl-lib:integer4 (f2cl-lib:int-mul n nb))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwkopt 'f2cl-lib:complex8))))
(cond
 ((/= info 0) (xerbla "CSYSVX" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond
 (nofact
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy uplo n n a lda af ldaf)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
    (setf uplo var-0))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (csytrf uplo n af ldaf ipiv work lwork info)
    (declare (ignore var-2 var-4 var-5))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
```

```
        (when var-3 (setf ldaf var-3))
        (when var-6 (setf lwork var-6))
        (when var-7 (setf info var-7)))
      (cond ((> info 0) (setf rcond zero) (go end_label)))))
    (setf anorm
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                (clansy "I" uplo n a lda rwork)
              (declare (ignore var-0 var-3 var-4 var-5))
              (setf uplo var-1)
              (setf n var-2)
              ret-val))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (csycon uplo n af ldaf ipiv anorm rcond work info)
      (declare (ignore var-2 var-4 var-5 var-7))
      (setf uplo var-0)
      (setf n var-1)
      (setf ldaf var-3)
      (setf rcond var-6)
      (setf info var-8))
    (clacpy "Full" n nrhs b ldb$ x ldx)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (csytrs uplo n nrhs af ldaf ipiv x ldx info)
      (declare (ignore var-3 var-5 var-6))
      (when var-0 (setf uplo var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf nrhs var-2))
      (when var-4 (setf ldaf var-4))
      (when var-7 (setf ldx var-7))
      (when var-8 (setf info var-8)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
         var-11 var-12 var-13 var-14 var-15 var-16)
        (csyrfs uplo n nrhs a lda af ldaf ipiv b ldb$ x ldx ferr berr work
         rwork info)
      (declare
       (ignore var-2 var-3 var-4 var-5 var-7 var-8 var-9 var-10 var-11
        var-12 var-13 var-14 var-15))
      (setf uplo var-0)
      (setf n var-1)
      (setf ldaf var-6)
      (setf info var-16))
    (if (< rcond (slamch "Epsilon"))
        (setf info (f2cl-lib:int-add n 1)))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce lwkopt 'f2cl-lib:complex8))
    (go end_label)
   end_label
```

```
        (return
         (values fact uplo n nrhs nil nil nil ldaf nil nil nil nil ldx rcond
                 nil nil nil lwork nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csysvx
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string) (string)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::integer4
                                              (*))
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (single-float)
                                             (array single-float (*))
                                             (array single-float (*))
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(fortran-to-lisp::fact
                                             fortran-to-lisp::uplo
                                             fortran-to-lisp::n
                                             fortran-to-lisp::nrhs nil nil nil
                                             fortran-to-lisp::ldaf nil nil nil
                                             nil fortran-to-lisp::ldx
                                             fortran-to-lisp::rcond nil nil
                                             nil fortran-to-lisp::lwork nil
                                             fortran-to-lisp::info)
                                           :calls
                                           '(fortran-to-lisp::csyrfs
                                             fortran-to-lisp::csycon
                                             fortran-to-lisp::clansy
                                             fortran-to-lisp::clacpy))))
```

———-

54.4 csysvxx

54.4.1 Axiom unit tests

— **csysvxx.input** —

```
)set break resume
)sys rm -f csysvxx.output
)spool csysvxx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

54.4.2 Axiom help page

— **csysvxx.help** —

```
==================================================================
csysvxx examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csysvxx (FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, EQUED,
           S, B, LDB, X, LDX, RCOND, RPVGRW, BERR, N_ERR_BNDS, ERR_BNDS_NORM,
           ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, RWORK, INFO)
            CSYSVXX computes the solution to system of linear equations A * X
           = B for SY matrices

Function/Subroutine Documentation
```

```
   subroutine csysvxx (character FACT, character UPLO, integer N, integer
       NRHS, complex, dimension( lda, * ) A, integer LDA, complex, dimension(
       ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV, character
       EQUED, real, dimension( * ) S, complex, dimension( ldb, * ) B, integer
       LDB, complex, dimension( ldx, * ) X, integer LDX, real RCOND, real
       RPVGRW, real, dimension( * ) BERR, integer N_ERR_BNDS, real, dimension(
       nrhs, * ) ERR_BNDS_NORM, real, dimension( nrhs, * ) ERR_BNDS_COMP,
       integer NPARAMS, real, dimension( * ) PARAMS, complex, dimension( * )
       WORK, real, dimension( * ) RWORK, integer INFO)
        CSYSVXX computes the solution to system of linear equations A * X = B
       for SY matrices

Purpose:
   CSYSVXX uses the diagonal pivoting factorization to compute the
   solution to a complex system of linear equations A * X = B, where
   A is an N-by-N symmetric matrix and X and B are N-by-NRHS matrices.

   If requested, both normwise and maximum componentwise error bounds
   are returned. CSYSVXX will return a solution with a tiny
   guaranteed error (O(eps) where eps is the working machine
   precision) unless the matrix is very ill-conditioned, in which
   case a warning is returned. Relevant condition numbers also are
   calculated and returned.

   CSYSVXX accepts user-provided factorizations and equilibration
   factors; see the definitions of the FACT and EQUED options.
   Solving with refinement and using a factorization from a previous
   CSYSVXX call will also produce a solution with either O(eps)
   errors or warnings, but we cannot make that claim for general
   user-provided factorizations and equilibration factors if they
   differ from what CSYSVXX would itself produce.

Description:

   The following steps are performed:

   1. If FACT = 'E', real scaling factors are computed to equilibrate
      the system:

      diag(S)*A*diag(S)     *inv(diag(S))*X = diag(S)*B

      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(S)*A*diag(S) and B by diag(S)*B.

   2. If FACT = 'N' or 'E', the LU decomposition is used to factor
      the matrix A (after equilibration if FACT = 'E') as

      A = U * D * U**T,  if UPLO = 'U', or
      A = L * D * L**T,  if UPLO = 'L',
```

```
      where U (or L) is a product of permutation and unit upper (lower)
      triangular matrices, and D is symmetric and block diagonal with
      1-by-1 and 2-by-2 diagonal blocks.

   3. If some D(i,i)=0, so that D is exactly singular, then the
      routine returns with INFO = i. Otherwise, the factored form of A
      is used to estimate the condition number of the matrix A (see
      argument RCOND).  If the reciprocal of the condition number is
      less than machine precision, the routine still goes on to solve
      for X and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. By default (unless PARAMS(LA_LINRX_ITREF_I) is set to zero),
      the routine will use iterative refinement to try to get a small
      error and error bounds.  Refinement calculates the residual to at
      least twice the working precision.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(R) so that it solves the original system before equilibration.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AF and IPIV contain the factored form of A.
      If EQUED is not 'N', the matrix A has been
      equilibrated with scaling factors given by S.
      A, AF, and IPIV are not modified.
   = 'N':  The matrix A will be copied to AF and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
      copied to AF and factored.

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
```

```
  The number of linear equations, i.e., the order of the  matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
  upper triangular part of A contains the upper triangular
  part of the matrix A, and the strictly lower triangular
  part of A is not referenced.  If UPLO = 'L', the leading
  N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if FACT = 'E' and EQUED = 'Y', A is overwritten by
  diag(S)*A*diag(S).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the block diagonal matrix D and the multipliers
  used to obtain the factor U or L from the factorization A =
  U*D*U**T or A = L*D*L**T as computed by SSYTRF.

  If FACT = 'N', then AF is an output argument and on exit
  returns the block diagonal matrix D and the multipliers
  used to obtain the factor U or L from the factorization A =
  U*D*U**T or A = L*D*L**T.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains details of the interchanges and the block
  structure of D, as determined by SSYTRF.  If IPIV(k) > 0,
  then rows and columns k and IPIV(k) were interchanged and
  D(k,k) is a 1-by-1 diagonal block.  If UPLO = 'U' and
  IPIV(k) = IPIV(k-1) < 0, then rows and columns k-1 and
  -IPIV(k) were interchanged and D(k-1:k,k-1:k) is a 2-by-2
  diagonal block.  If UPLO = 'L' and IPIV(k) = IPIV(k+1) < 0,
```

```
  then rows and columns k+1 and -IPIV(k) were interchanged
  and D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

  If FACT = 'N', then IPIV is an output argument and on exit
  contains details of the interchanges and the block
  structure of D, as determined by SSYTRF.

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'Y':  Both row and column equilibration, i.e., A has been
  replaced by diag(S) * A * diag(S).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

S
  S is REAL array, dimension (N)
  The scale factors for A.  If EQUED = 'Y', A is multiplied on
  the left and right by diag(S).  S is an input argument if FACT =
  'F'; otherwise, S is an output argument.  If FACT = 'F' and EQUED
  = 'Y', each element of S must be positive.  If S is output, each
  element of S is a power of the radix. If S is input, each element
  of S should be a power of the radix to ensure a reliable solution
  and error estimates. Scaling by powers of the radix does not cause
  rounding errors unless the result underflows or overflows.
  Rounding errors during scaling lead to refining with a matrix that
  is not equivalent to the input matrix, producing error estimates
  that may not be reliable.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if EQUED = 'Y', B is overwritten by diag(S)*B;

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  If INFO = 0, the N-by-NRHS solution matrix X to the original
  system of equations.  Note that A and B are modified on exit if
  EQUED .ne. 'N', and the solution to the equilibrated system is
  inv(diag(S))*X.

LDX
  LDX is INTEGER
```

```
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

RPVGRW
  RPVGRW is REAL
  Reciprocal pivot growth.  On exit, this contains the reciprocal
  pivot growth factor norm(A)/norm(U). The "max absolute element"
  norm is used.  If this is much less than 1, then the stability of
  the LU factorization of the (equilibrated) matrix A could be poor.
  This also means that the solution X, estimated condition numbers,
  and error bounds could be unreliable. If factorization fails with
  0<INFO<=N, then this contains the reciprocal pivot growth factor
  for the leading INFO columns of A.

BERR
  BERR is REAL array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
            max_j (abs(XTRUE(j,i) - X(j,i)))
             ------------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.
```

```
  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.

ERR_BNDS_COMP

  ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                  abs(XTRUE(j,i) - X(j,i))
            max_j ----------------------
                       abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
```

```
      reciprocal condition number is less than the threshold
      sqrt(n) * slamch('Epsilon').

   err = 2 "Guaranteed" error bound: The estimated forward error,
      almost certainly within a factor of 10 of the true error
      so long as the next entry is greater than the threshold
      sqrt(n) * slamch('Epsilon'). This error bound should only
      be trusted if the previous boolean is true.

   err = 3  Reciprocal condition number: Estimated componentwise
      reciprocal condition number.  Compared with the threshold
      sqrt(n) * slamch('Epsilon') to determine if the error
      estimate is "guaranteed". These reciprocal condition
      numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
      appropriately scaled matrix Z.
      Let Z = S*(A*diag(x)), where x is the solution for the
      current right-hand side and S scales each row of
      A*diag(x) by a power of the radix so all absolute row
      sums of Z are approximately 1.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is REAL array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
  (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.
```

```
  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is COMPLEX array, dimension (2*N)

RWORK
  RWORK is REAL array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is
  guaranteed.
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
     has been completed, but the factor U is exactly singular, so
     the solution and error bounds could not be computed. RCOND = 0
     is returned.
  = N+J: The solution corresponding to the Jth right-hand side is
     not guaranteed. The solutions corresponding to other right-
     hand sides K with K > J may not be guaranteed as well, but
     only the first such right-hand side is reported. If a small
     componentwise error is not requested (PARAMS(3) = 0.0) then
     the Jth right-hand side is the first with a normwise error
     bound that is not guaranteed (the smallest J such
     that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
     the Jth right-hand side is the first with either a normwise or
     componentwise error bound that is not guaranteed (the smallest
     J such that either ERR_BNDS_NORM(J,1) = 0.0 or
     ERR_BNDS_COMP(J,1) = 0.0). See the definition of
     ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
     about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

———-

54.4.3 fortran code

— **csysvxx.f** —

```
*  ======================================================================
      SUBROUTINE CSYSVXX( FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $                    EQUED, S, B, LDB, X, LDX, RCOND, RPVGRW, BERR,
     $                    N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP,
     $                    NPARAMS, PARAMS, WORK, RWORK, INFO )
```

```
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      REAL               RCOND, RPVGRW
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX, * ), WORK( * )
      REAL               S( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * ), RWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
*     ..
*     .. Local Scalars ..
      LOGICAL            EQUIL, NOFACT, RCEQU
      INTEGER            INFEQU, J
      REAL               AMAX, BIGNUM, SMIN, SMAX, SCOND, SMLNUM
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, SLAMCH, CLA_SYRPVGRW
      LOGICAL            LSAME
      REAL               SLAMCH, CLA_SYRPVGRW
*     ..
*     .. External Subroutines ..
      EXTERNAL           CSYCON, CSYEQUB, CSYTRF, CSYTRS, CLACPY,
     $                   CLAQSY, XERBLA, CLASCL2, CSYRFSX
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      SMLNUM = SLAMCH( 'Safe minimum' )
      BIGNUM = ONE / SMLNUM
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         RCEQU = .FALSE.
      ELSE
         RCEQU = LSAME( EQUED, 'Y' )
      ENDIF
*
*     Default is failure.  If an input parameter is wrong or
*     factorization fails, make everything look horrible.  Only the
*     pivot growth is set here, the rest is initialized in CSYRFSX.
*
      RPVGRW = ZERO
*
*     Test the input parameters.  PARAMS is not tested until CSYRFSX.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.
     $     LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.LSAME(UPLO, 'U') .AND.
     $         .NOT.LSAME(UPLO, 'L') ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $        ( RCEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -10
      ELSE
         IF ( RCEQU ) THEN
            SMIN = BIGNUM
            SMAX = ZERO
            DO 10 J = 1, N
               SMIN = MIN( SMIN, S( J ) )
               SMAX = MAX( SMAX, S( J ) )
 10         CONTINUE
            IF( SMIN.LE.ZERO ) THEN
```

```
            INFO = -11
         ELSE IF( N.GT.0 ) THEN
            SCOND = MAX( SMIN, SMLNUM ) / MIN( SMAX, BIGNUM )
         ELSE
            SCOND = ONE
         END IF
      END IF
      IF( INFO.EQ.0 ) THEN
         IF( LDB.LT.MAX( 1, N ) ) THEN
            INFO = -13
         ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
            INFO = -15
         END IF
      END IF
   END IF
*
   IF( INFO.NE.0 ) THEN
      CALL XERBLA( 'CSYSVXX', -INFO )
      RETURN
   END IF
*
   IF( EQUIL ) THEN
*
*     Compute row and column scalings to equilibrate the matrix A.
*
      CALL CSYEQUB( UPLO, N, A, LDA, S, SCOND, AMAX, WORK, INFEQU )
      IF( INFEQU.EQ.0 ) THEN
*
*     Equilibrate the matrix.
*
         CALL CLAQSY( UPLO, N, A, LDA, S, SCOND, AMAX, EQUED )
         RCEQU = LSAME( EQUED, 'Y' )
      END IF

   END IF
*
*     Scale the right hand-side.
*
   IF( RCEQU ) CALL CLASCL2( N, NRHS, S, B, LDB )
*
   IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the LDL^T or UDU^T factorization of A.
*
      CALL CLACPY( UPLO, N, N, A, LDA, AF, LDAF )
      CALL CSYTRF( UPLO, N, AF, LDAF, IPIV, WORK, 5*MAX(1,N), INFO )
*
*        Return if INFO is non-zero.
*
      IF( INFO.GT.0 ) THEN
```

```
*
*          Pivot in column INFO is exactly 0
*          Compute the reciprocal pivot growth factor of the
*          leading rank-deficient INFO columns of A.
*
           IF ( N.GT.0 )
     $           RPVGRW = CLA_SYRPVGRW( UPLO, N, INFO, A, LDA, AF,
     $           LDAF, IPIV, RWORK )
           RETURN
        END IF
      END IF
*
*     Compute the reciprocal pivot growth factor RPVGRW.
*
      IF ( N.GT.0 )
     $     RPVGRW = CLA_SYRPVGRW( UPLO, N, INFO, A, LDA, AF, LDAF,
     $     IPIV, RWORK )
*
*     Compute the solution matrix X.
*
      CALL CLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL CSYTRS( UPLO, N, NRHS, AF, LDAF, IPIV, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL CSYRFSX( UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $     S, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS, ERR_BNDS_NORM,
     $     ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, RWORK, INFO )
*
*     Scale solutions.
*
      IF ( RCEQU ) THEN
         CALL CLASCL2 (N, NRHS, S, X, LDX )
      END IF
*
      RETURN
*
*     End of CSYSVXX
*
      END
```

———-

54.4.4 lisp code

— csysvxx.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun csysvxx
         (fact uplo n nrhs a lda af ldaf ipiv equed s b ldb$ x ldx rcond rpvgrw
          berr n_err_bnds err_bnds_norm err_bnds_comp nparams params work rwork
          info)
    (declare (type (single-float) rpvgrw rcond)
             (type (array single-float (*)) rwork params err_bnds_comp
              err_bnds_norm berr s)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) work x b af a)
             (type (f2cl-lib:integer4) info nparams n_err_bnds ldx ldb$ ldaf
              lda nrhs n)
             (type (string *) equed uplo fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (equed character equed-%data% equed-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (af f2cl-lib:complex8 af-%data% af-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (s single-float s-%data% s-%offset%)
         (berr single-float berr-%data% berr-%offset%)
         (err_bnds_norm single-float err_bnds_norm-%data%
          err_bnds_norm-%offset%)
         (err_bnds_comp single-float err_bnds_comp-%data%
          err_bnds_comp-%offset%)
         (params single-float params-%data% params-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%))
      (prog ((amax 0.0) (bignum 0.0) (smin 0.0) (smax 0.0) (scond 0.0)
             (smlnum 0.0) (infequ 0) (j 0) (equil nil) (nofact nil)
             (rcequ nil))
        (declare (type (single-float) amax bignum smin smax scond smlnum)
                 (type (f2cl-lib:integer4) infequ j)
                 (type f2cl-lib:logical equil nofact rcequ))
        (setf info 0)
        (setf nofact
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame fact "N")
                  (declare (ignore var-1))
                  (when var-0 (setf fact var-0))
                  ret-val))
        (setf equil
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame fact "E")
```

```
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val))
(setf smlnum (slamch "Safe minimum"))
(setf bignum (/ one smlnum))
(cond
 ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
  (setf rcequ f2cl-lib:%false%))
 (t
  (setf rcequ
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame equed "Y")
            (declare (ignore var-1))
            (when var-0 (setf equed var-0))
            ret-val))))
(setf rpvgrw zero)
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame fact "F")
     (declare (ignore var-1))
     (when var-0 (setf fact var-0))
     ret-val)
```

```
    (not
     (or rcequ
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame equed "N")
           (declare (ignore var-1))
           (when var-0 (setf equed var-0))
           ret-val))))
   (setf info -10))
  (t
   (cond
    (rcequ (setf smin bignum) (setf smax zero)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf smin
                 (min smin
                      (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
         (setf smax
                 (max smax
                      (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
        label10))
     (cond ((<= smin zero) (setf info -11))
           ((> n 0)
            (setf scond (/ (max smin smlnum) (min smax bignum))))
           (t (setf scond one)))))
   (cond
    ((= info 0)
     (cond
      ((< ldb$
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -13))
      ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -15)))))))
 (cond
  ((/= info 0) (xerbla "CSYSVXX" (f2cl-lib:int-sub info))
   (go end_label)))
 (cond
  (equil
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (csyequb uplo n a lda s scond amax work infequ)
     (declare (ignore var-1 var-2 var-3 var-4 var-7))
     (setf uplo var-0)
     (setf scond var-5)
     (setf amax var-6)
     (setf infequ var-8))
   (cond
    ((= infequ 0)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
```

```
          (claqsy uplo n a lda s scond amax equed)
        (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
        (setf uplo var-0)
        (setf equed var-7))
      (setf rcequ
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame equed "Y")
                (declare (ignore var-1))
                (when var-0 (setf equed var-0))
                ret-val))))))
  (if rcequ
      (clascl2 n nrhs s b ldb$))
  (cond
   ((or nofact equil)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (clacpy uplo n n a lda af ldaf)
      (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
      (setf uplo var-0))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (csytrf uplo n af ldaf ipiv work
         (f2cl-lib:int-mul 5
                           (max (the f2cl-lib:integer4 1)
                                (the f2cl-lib:integer4 n)))
         info)
      (declare (ignore var-2 var-4 var-5 var-6))
      (when var-0 (setf uplo var-0))
      (when var-1 (setf n var-1))
      (when var-3 (setf ldaf var-3))
      (when var-7 (setf info var-7)))
    (cond
     ((> info 0)
      (if (> n 0)
          (setf rpvgrw
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6
                       var-7 var-8)
                      (cla_syrpvgrw uplo n info a lda af ldaf ipiv rwork)
                    (declare
                     (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7
                      var-8))
                    (setf uplo var-0)
                    ret-val)))
      (go end_label)))))
  (if (> n 0)
      (setf rpvgrw
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6
                   var-7 var-8)
                  (cla_syrpvgrw uplo n info a lda af ldaf ipiv rwork)
```

```
                    (declare
                     (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7
                      var-8))
                    (setf uplo var-0)
                    ret-val)))
        (clacpy "Full" n nrhs b ldb$ x ldx)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (csytrs uplo n nrhs af ldaf ipiv x ldx info)
          (declare (ignore var-3 var-5 var-6))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldaf var-4))
          (when var-7 (setf ldx var-7))
          (when var-8 (setf info var-8)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
             var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19
             var-20 var-21 var-22 var-23)
            (csyrfsx uplo equed n nrhs a lda af ldaf ipiv s b ldb$ x ldx rcond
             berr n_err_bnds err_bnds_norm err_bnds_comp nparams params work
             rwork info)
          (declare
           (ignore var-4 var-6 var-8 var-9 var-10 var-12 var-15 var-17 var-18
            var-20 var-21 var-22))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf equed var-1))
          (when var-2 (setf n var-2))
          (when var-3 (setf nrhs var-3))
          (when var-5 (setf lda var-5))
          (when var-7 (setf ldaf var-7))
          (when var-11 (setf ldb$ var-11))
          (when var-13 (setf ldx var-13))
          (when var-14 (setf rcond var-14))
          (when var-16 (setf n_err_bnds var-16))
          (when var-19 (setf nparams var-19))
          (when var-23 (setf info var-23)))
        (cond (rcequ (clascl2 n nrhs s x ldx)))
        (go end_label)
       end_label
        (return
         (values fact uplo n nrhs nil lda nil ldaf nil equed nil nil ldb$ nil
                 ldx rcond rpvgrw nil n_err_bnds nil nil nparams nil nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csysvxx
```

```
        fortran-to-lisp::*f2cl-function-info*)
    (fortran-to-lisp::make-f2cl-finfo :arg-types
                                      '((string) (string)
                                        (fortran-to-lisp::integer4)
                                        (fortran-to-lisp::integer4)
                                        (array fortran-to-lisp::complex8
                                         (*))
                                        (fortran-to-lisp::integer4)
                                        (array fortran-to-lisp::complex8
                                         (*))
                                        (fortran-to-lisp::integer4)
                                        (array fortran-to-lisp::integer4
                                         (*))
                                        (string) (array single-float (*))
                                        (array fortran-to-lisp::complex8
                                         (*))
                                        (fortran-to-lisp::integer4)
                                        (array fortran-to-lisp::complex8
                                         (*))
                                        (fortran-to-lisp::integer4)
                                        (single-float) (single-float)
                                        (array single-float (*))
                                        (fortran-to-lisp::integer4)
                                        (array single-float (*))
                                        (array single-float (*))
                                        (fortran-to-lisp::integer4)
                                        (array single-float (*))
                                        (array fortran-to-lisp::complex8
                                         (*))
                                        (array single-float (*))
                                        (fortran-to-lisp::integer4))
                                      :return-values
                                      '(fortran-to-lisp::fact
                                        fortran-to-lisp::uplo
                                        fortran-to-lisp::n
                                        fortran-to-lisp::nrhs nil
                                        fortran-to-lisp::lda nil
                                        fortran-to-lisp::ldaf nil
                                        fortran-to-lisp::equed nil nil
                                        fortran-to-lisp::ldb$ nil
                                        fortran-to-lisp::ldx
                                        fortran-to-lisp::rcond
                                        fortran-to-lisp::rpvgrw nil
                                        fortran-to-lisp::n_err_bnds nil
                                        nil fortran-to-lisp::nparams nil
                                        nil nil fortran-to-lisp::info)
                                      :calls
                                      '(fortran-to-lisp::cla_syrpvgrw
                                        fortran-to-lisp::clacpy
                                        fortran-to-lisp::clascl2
```

```
fortran-to-lisp::claqsy
fortran-to-lisp::csyequb))))
```

Chapter 55

LAPACK - Symmetric Matrix, Linear Solve, Complex16

55.1 zsysv

55.1.1 Axiom unit tests

— zsysv.input —

```
)set break resume
)sys rm -f zsysv.output
)spool zsysv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

55.1.2 Axiom help page

— zsysv.help —

```
==========================================================================
zsysv examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zsysv (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK, LWORK,
           INFO)
            ZSYSV computes the solution to system of linear equations A * X =
           B for SY matrices

Function/Subroutine Documentation
   subroutine zsysv (character UPLO, integer N, integer NRHS, complex*16,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       complex*16, dimension( ldb, * ) B, integer LDB, complex*16, dimension(
       * ) WORK, integer LWORK, integer INFO)
        ZSYSV computes the solution to system of linear equations A * X = B
       for SY matrices

Purpose:
   ZSYSV computes the solution to a complex system of linear equations
     A * X = B,
   where A is an N-by-N symmetric matrix and X and B are N-by-NRHS
   matrices.

   The diagonal pivoting method is used to factor A as
     A = U * D * U**T,  if UPLO = 'U', or
     A = L * D * L**T,  if UPLO = 'L',
   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, and D is symmetric and block diagonal with
   1-by-1 and 2-by-2 diagonal blocks.  The factored form of A is then
   used to solve the system of equations A * X = B.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.
```

```
A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if INFO = 0, the block diagonal matrix D and the
  multipliers used to obtain the factor U or L from the
  factorization A = U*D*U**T or A = L*D*L**T as computed by ZSYTRF.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D, as
  determined by ZSYTRF.  If IPIV(k) > 0, then rows and columns
  k and IPIV(k) were interchanged, and D(k,k) is a 1-by-1
  diagonal block.  If UPLO = 'U' and IPIV(k) = IPIV(k-1) < 0,
  then rows and columns k-1 and -IPIV(k) were interchanged and
  D(k-1:k,k-1:k) is a 2-by-2 diagonal block.  If UPLO = 'L' and
  IPIV(k) = IPIV(k+1) < 0, then rows and columns k+1 and
  -IPIV(k) were interchanged and D(k:k+1,k:k+1) is a 2-by-2
  diagonal block.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of WORK.  LWORK >= 1, and for best performance
  LWORK >= max(1,N*NB), where NB is the optimal blocksize for
  ZSYTRF.
  for LWORK < N, TRS will be done with Level BLAS 2
  for LWORK >= N, TRS will be done with Level BLAS 3
```

```
  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = i, D(i,i) is exactly zero.  The factorization
       has been completed, but the block diagonal matrix D is
       exactly singular, so the solution could not be computed.
```

———-

55.1.3 fortran code

— **zsysv.f** —

```
*  =====================================================================
      SUBROUTINE ZSYSV( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK,
     $                  LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, LWORK, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            LWKOPT
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
```

```
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZSYTRF, ZSYTRS, ZSYTRS2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -10
      END IF
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            LWKOPT = 1
         ELSE
            CALL ZSYTRF( UPLO, N, A, LDA, IPIV, WORK, -1, INFO )
            LWKOPT = WORK(1)
         END IF
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYSV ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Compute the factorization A = U*D*U**T or A = L*D*L**T.
*
      CALL ZSYTRF( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
```

```
*
         IF ( LWORK.LT.N ) THEN
*
*        Solve with TRS ( Use Level BLAS 2)
*
            CALL ZSYTRS( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
         ELSE
*
*        Solve with TRS2 ( Use Level BLAS 3)
*
            CALL ZSYTRS2( UPLO,N,NRHS,A,LDA,IPIV,B,LDB,WORK,INFO )
*
         END IF
*
      END IF
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of ZSYSV
*
      END
```

———-

55.1.4 lisp code

— **zsysv.lisp** —

```
(defun zsysv (uplo n nrhs a lda ipiv b ldb$ work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex16 (*)) work b a)
           (type (f2cl-lib:integer4) info lwork ldb$ lda nrhs n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (b f2cl-lib:complex16 b-%data% b-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((lwkopt 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) lwkopt))
      (setf info 0)
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond
```

```
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and (< lwork 1) (not lquery)) (setf info -10)))
(cond
 ((= info 0)
  (cond ((= n 0) (setf lwkopt 1))
        (t
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
             (zsytrf uplo n a lda ipiv work -1 info)
           (declare (ignore var-2 var-4 var-5 var-6))
           (when var-0 (setf uplo var-0))
           (when var-1 (setf n var-1))
           (when var-3 (setf lda var-3))
           (when var-7 (setf info var-7)))
         (setf lwkopt
                 (f2cl-lib:int
                  (f2cl-lib:fref work-%data% (1) ((1 *))
                                 work-%offset%)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwkopt 'f2cl-lib:complex16))))
(cond
 ((/= info 0) (xerbla "ZSYSV " (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zsytrf uplo n a lda ipiv work lwork info)
  (declare (ignore var-2 var-4 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-6 (setf lwork var-6))
  (when var-7 (setf info var-7)))
(cond
```

```
      ((= info 0)
       (cond
        ((< lwork n)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
             (zsytrs uplo n nrhs a lda ipiv b ldb$ info)
           (declare (ignore var-3 var-5 var-6))
           (when var-0 (setf uplo var-0))
           (when var-1 (setf n var-1))
           (when var-2 (setf nrhs var-2))
           (when var-4 (setf lda var-4))
           (when var-7 (setf ldb$ var-7))
           (when var-8 (setf info var-8))))
        (t
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
             (zsytrs2 uplo n nrhs a lda ipiv b ldb$ work info)
           (declare (ignore var-3 var-5 var-6 var-8))
           (when var-0 (setf uplo var-0))
           (when var-1 (setf n var-1))
           (when var-2 (setf nrhs var-2))
           (when var-4 (setf lda var-4))
           (when var-7 (setf ldb$ var-7))
           (when var-9 (setf info var-9)))))))
     (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
             (coerce lwkopt 'f2cl-lib:complex16))
     (go end_label)
    end_label
     (return (values uplo n nrhs nil lda nil nil ldb$ nil lwork info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsysv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
```

```
:return-values
'(fortran-to-lisp::uplo
  fortran-to-lisp::n
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::lda nil nil
  fortran-to-lisp::ldb$ nil
  fortran-to-lisp::lwork
  fortran-to-lisp::info)
:calls '(fortran-to-lisp::xerbla))))
```

———-

55.2 zsysv_rook

55.2.1 Axiom unit tests

— **zsysvrook.input** —

```
)set break resume
)sys rm -f zsysvrook.output
)spool zsysvrook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

55.2.2 Axiom help page

— **zsysv_rook.help** —

```
==========================================================================
zsysv_rook examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine zsysv_rook (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK,
           LWORK, INFO)
            ZSYSV_ROOK computes the solution to system of linear equations A *
           X = B for SY matrices

Function/Subroutine Documentation
   subroutine zsysv_rook (character UPLO, integer N, integer NRHS, complex*16,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       complex*16, dimension( ldb, * ) B, integer LDB, complex*16, dimension(
       * ) WORK, integer LWORK, integer INFO)
        ZSYSV_ROOK computes the solution to system of linear equations A * X =
       B for SY matrices

Purpose:
   ZSYSV_ROOK computes the solution to a complex system of linear equations
     A * X = B,
   where A is an N-by-N symmetric matrix and X and B are N-by-NRHS
   matrices.

   The diagonal pivoting method is used to factor A as
     A = U * D * U**T,  if UPLO = 'U', or
     A = L * D * L**T,  if UPLO = 'L',
   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, and D is symmetric and block diagonal with
   1-by-1 and 2-by-2 diagonal blocks.

   ZSYTRF_ROOK is called to compute the factorization of a complex
   symmetric matrix A using the bounded Bunch-Kaufman ("rook") diagonal
   pivoting method.

   The factored form of A is then used to solve the system
   of equations A * X = B by calling ZSYTRS_ROOK.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.
```

```
A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if INFO = 0, the block diagonal matrix D and the
  multipliers used to obtain the factor U or L from the
  factorization A = U*D*U**T or A = L*D*L**T as computed by ZSYTRF_ROOK.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D,
  as determined by ZSYTRF_ROOK.

  If UPLO = 'U':
    If IPIV(k) > 0, then rows and columns k and IPIV(k)
    were interchanged and D(k,k) is a 1-by-1 diagonal block.

    If IPIV(k) < 0 and IPIV(k-1) < 0, then rows and
    columns k and -IPIV(k) were interchanged and rows and
    columns k-1 and -IPIV(k-1) were inerchaged,
    D(k-1:k,k-1:k) is a 2-by-2 diagonal block.

  If UPLO = 'L':
    If IPIV(k) > 0, then rows and columns k and IPIV(k)
    were interchanged and D(k,k) is a 1-by-1 diagonal block.

    If IPIV(k) < 0 and IPIV(k+1) < 0, then rows and
    columns k and -IPIV(k) were interchanged and rows and
    columns k+1 and -IPIV(k+1) were inerchaged,
    D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).
```

```
 WORK
   WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The length of WORK.  LWORK >= 1, and for best performance
   LWORK >= max(1,N*NB), where NB is the optimal blocksize for
   ZSYTRF_ROOK.

   TRS will be done with Level 2 BLAS

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value
   > 0: if INFO = i, D(i,i) is exactly zero.  The factorization
        has been completed, but the block diagonal matrix D is
        exactly singular, so the solution could not be computed.
```

55.2.3 fortran code

— zsysv_rook.f —

```
*  =====================================================================
      SUBROUTINE ZSYSV_ROOK( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK,
     $                  LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, LWORK, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
```

```
      COMPLEX*16         A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            LWKOPT
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZSYTRF_ROOK, ZSYTRS_ROOK
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -10
      END IF
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            LWKOPT = 1
         ELSE
            CALL ZSYTRF_ROOK( UPLO, N, A, LDA, IPIV, WORK, -1, INFO )
            LWKOPT = WORK(1)
         END IF
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
```

```
         CALL XERBLA( 'ZSYSV_ROOK ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Compute the factorization A = U*D*U**T or A = L*D*L**T.
*
      CALL ZSYTRF_ROOK( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
*        Solve with TRS_ROOK ( Use Level 2 BLAS)
*
         CALL ZSYTRS_ROOK( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
      END IF
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of ZSYSV_ROOK
*
      END
```

———-

55.2.4 lisp code

— **zsysv_rook.lisp** —

```
(defun zsysv_rook (uplo n nrhs a lda ipiv b ldb$ work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex16 (*)) work b a)
           (type (f2cl-lib:integer4) info lwork ldb$ lda nrhs n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (b f2cl-lib:complex16 b-%data% b-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((lwkopt 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) lwkopt))
      (setf info 0)
```

```
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and (< lwork 1) (not lquery)) (setf info -10)))
(cond
 ((= info 0)
  (cond ((= n 0) (setf lwkopt 1))
        (t
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
             (zsytrf_rook uplo n a lda ipiv work -1 info)
           (declare (ignore var-2 var-4 var-5 var-6))
           (when var-0 (setf uplo var-0))
           (when var-1 (setf n var-1))
           (when var-3 (setf lda var-3))
           (when var-7 (setf info var-7)))
         (setf lwkopt
                 (f2cl-lib:int
                  (f2cl-lib:fref work-%data% (1) ((1 *))
                                 work-%offset%)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwkopt 'f2cl-lib:complex16))))
(cond
 ((/= info 0) (xerbla "ZSYSV_ROOK " (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zsytrf_rook uplo n a lda ipiv work lwork info)
  (declare (ignore var-2 var-4 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
```

```
        (when var-6 (setf lwork var-6))
        (when var-7 (setf info var-7)))
      (cond
       ((= info 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (zsytrs_rook uplo n nrhs a lda ipiv b ldb$ info)
          (declare (ignore var-3 var-5 var-6))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf lda var-4))
          (when var-7 (setf ldb$ var-7))
          (when var-8 (setf info var-8)))))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce lwkopt 'f2cl-lib:complex16))
      (go end_label)
     end_label
      (return (values uplo n nrhs nil lda nil nil ldb$ nil lwork info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsysv_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::ldb$ nil
                                              fortran-to-lisp::lwork
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

55.3 zsysvx

55.3.1 Axiom unit tests

— **zsysvx.input** —

```
)set break resume
)sys rm -f zsysvx.output
)spool zsysvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

55.3.2 Axiom help page

— **zsysvx.help** —

```
==================================================================
zsysvx examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zsysvx (FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB,
           X, LDX, RCOND, FERR, BERR, WORK, LWORK, RWORK, INFO)
            ZSYSVX computes the solution to system of linear equations A * X =
           B for SY matrices

Function/Subroutine Documentation
   subroutine zsysvx (character FACT, character UPLO, integer N, integer NRHS,
```

```
      complex*16, dimension( lda, * ) A, integer LDA, complex*16, dimension(
      ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV, complex*16,
      dimension( ldb, * ) B, integer LDB, complex*16, dimension( ldx, * ) X,
      integer LDX, double precision RCOND, double precision, dimension( * )
      FERR, double precision, dimension( * ) BERR, complex*16, dimension( * )
      WORK, integer LWORK, double precision, dimension( * ) RWORK, integer
      INFO)
       ZSYSVX computes the solution to system of linear equations A * X = B
      for SY matrices

Purpose:
   ZSYSVX uses the diagonal pivoting factorization to compute the
   solution to a complex system of linear equations A * X = B,
   where A is an N-by-N symmetric matrix and X and B are N-by-NRHS
   matrices.

   Error bounds on the solution and a condition estimate are also provided.

Description:

   The following steps are performed:

   1. If FACT = 'N', the diagonal pivoting method is used to factor A.
      The form of the factorization is
        A = U * D * U**T,  if UPLO = 'U', or
        A = L * D * L**T,  if UPLO = 'L',
      where U (or L) is a product of permutation and unit upper (lower)
      triangular matrices, and D is symmetric and block diagonal with
      1-by-1 and 2-by-2 diagonal blocks.

   2. If some D(i,i)=0, so that D is exactly singular, then the routine
      returns with INFO = i. Otherwise, the factored form of A is used
      to estimate the condition number of the matrix A.  If the
      reciprocal of the condition number is less than machine precision,
      INFO = N+1 is returned as a warning, but the routine still goes on
      to solve for X and compute error bounds as described below.

   3. The system of equations is solved for X using the factored form of A.

   4. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of A has been
   supplied on entry.
   = 'F':  On entry, AF and IPIV contain the factored form of A.
           A, AF and IPIV will not be modified.
```

```
  = 'N':  The matrix A will be copied to AF and factored.

UPLO
  UPLO is CHARACTER*1
  = 'U':  Upper triangle of A is stored;
  = 'L':  Lower triangle of A is stored.

N
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
  upper triangular part of A contains the upper triangular part
  of the matrix A, and the strictly lower triangular part of A
  is not referenced.  If UPLO = 'L', the leading N-by-N lower
  triangular part of A contains the lower triangular part of
  the matrix A, and the strictly upper triangular part of A is
  not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX*16 array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the block diagonal matrix D and the multipliers used
  to obtain the factor U or L from the factorization
  A = U*D*U**T or A = L*D*L**T as computed by ZSYTRF.

  If FACT = 'N', then AF is an output argument and on exit
  returns the block diagonal matrix D and the multipliers used
  to obtain the factor U or L from the factorization
  A = U*D*U**T or A = L*D*L**T.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains details of the interchanges and the block structure
```

```
  of D, as determined by ZSYTRF.
  If IPIV(k) > 0, then rows and columns k and IPIV(k) were
  interchanged and D(k,k) is a 1-by-1 diagonal block.
  If UPLO = 'U' and IPIV(k) = IPIV(k-1) < 0, then rows and
  columns k-1 and -IPIV(k) were interchanged and D(k-1:k,k-1:k)
  is a 2-by-2 diagonal block.  If UPLO = 'L' and IPIV(k) =
  IPIV(k+1) < 0, then rows and columns k+1 and -IPIV(k) were
  interchanged and D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

  If FACT = 'N', then IPIV is an output argument and on exit
  contains details of the interchanges and the block structure
  of D, as determined by ZSYTRF.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  The N-by-NRHS right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  The estimate of the reciprocal condition number of the matrix
  A.  If RCOND is less than the machine precision (in
  particular, if RCOND = 0), the matrix is singular to working
  precision.  This condition is indicated by a return code of INFO > 0.

FERR
  FERR is DOUBLE PRECISION array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  The componentwise relative backward error of each solution
```

```
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of WORK.  LWORK >= max(1,2*N), and for best
  performance, when FACT = 'N', LWORK >= max(1,2*N,N*NB), where
  NB is the optimal blocksize for ZSYTRF.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (N)

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = i, and i is
    <= N:  D(i,i) is exactly zero.  The factorization
       has been completed but the factor D is exactly
       singular, so the solution and error bounds could
       not be computed. RCOND = 0 is returned.
    = N+1: D is nonsingular, but RCOND is less than machine
       precision, meaning that the matrix is singular
       to working precision.  Nevertheless, the
       solution and error bounds are computed because
       there are a number of situations where the
       computed solution can be more accurate than the
       value of RCOND would suggest.
```

———-

55.3.3 fortran code

— **zsysvx.f** —

```
*  =====================================================================
      SUBROUTINE ZSYSVX( FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B,
     $                   LDB, X, LDX, RCOND, FERR, BERR, WORK, LWORK,
```

```
     $                   RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          FACT, UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, LWORK, N, NRHS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   BERR( * ), FERR( * ), RWORK( * )
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, NOFACT
      INTEGER            LWKOPT, NB
      DOUBLE PRECISION   ANORM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, ZLANSY
      EXTERNAL           LSAME, ILAENV, DLAMCH, ZLANSY
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLACPY, ZSYCON, ZSYRFS, ZSYTRF, ZSYTRS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.NOFACT .AND. .NOT.LSAME( FACT, 'F' ) ) THEN
```

```
         INFO = -1
      ELSE IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) )
     $          THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -11
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -13
      ELSE IF( LWORK.LT.MAX( 1, 2*N ) .AND. .NOT.LQUERY ) THEN
         INFO = -18
      END IF
*
      IF( INFO.EQ.0 ) THEN
         LWKOPT = MAX( 1, 2*N )
         IF( NOFACT ) THEN
            NB = ILAENV( 1, 'ZSYTRF', UPLO, N, -1, -1, -1 )
            LWKOPT = MAX( LWKOPT, N*NB )
         END IF
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYSVX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
      IF( NOFACT ) THEN
*
*        Compute the factorization A = U*D*U**T or A = L*D*L**T.
*
         CALL ZLACPY( UPLO, N, N, A, LDA, AF, LDAF )
         CALL ZSYTRF( UPLO, N, AF, LDAF, IPIV, WORK, LWORK, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 )THEN
            RCOND = ZERO
            RETURN
         END IF
      END IF
```

```
*
*     Compute the norm of the matrix A.
*
      ANORM = ZLANSY( 'I', UPLO, N, A, LDA, RWORK )
*
*     Compute the reciprocal of the condition number of A.
*
      CALL ZSYCON( UPLO, N, AF, LDAF, IPIV, ANORM, RCOND, WORK, INFO )
*
*     Compute the solution vectors X.
*
      CALL ZLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL ZSYTRS( UPLO, N, NRHS, AF, LDAF, IPIV, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solutions and
*     compute error bounds and backward error estimates for them.
*
      CALL ZSYRFS( UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB, X,
     $             LDX, FERR, BERR, WORK, RWORK, INFO )
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.DLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of ZSYSVX
*
      END
```

———-

55.3.4 lisp code

— **zsysvx.lisp** —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun zsysvx
         (fact uplo n nrhs a lda af ldaf ipiv b ldb$ x ldx rcond ferr berr work
          lwork rwork info)
    (declare (type (array double-float (*)) rwork berr ferr)
             (type (double-float) rcond)
             (type (array f2cl-lib:integer4 (*)) ipiv)
```

```
          (type (array f2cl-lib:complex16 (*)) work x b af a)
          (type (f2cl-lib:integer4) info lwork ldx ldb$ ldaf lda nrhs n)
          (type (string *) uplo fact))
  (f2cl-lib:with-multi-array-data
      ((fact character fact-%data% fact-%offset%)
       (uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (af f2cl-lib:complex16 af-%data% af-%offset%)
       (b f2cl-lib:complex16 b-%data% b-%offset%)
       (x f2cl-lib:complex16 x-%data% x-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (ferr double-float ferr-%data% ferr-%offset%)
       (berr double-float berr-%data% berr-%offset%)
       (rwork double-float rwork-%data% rwork-%offset%))
    (prog ((anorm 0.0d0) (lwkopt 0) (nb 0) (lquery nil) (nofact nil))
      (declare (type (double-float) anorm)
               (type (f2cl-lib:integer4) lwkopt nb)
               (type f2cl-lib:logical lquery nofact))
      (setf info 0)
      (setf nofact
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame fact "N")
                (declare (ignore var-1))
                (when var-0 (setf fact var-0))
                ret-val))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond
       ((and (not nofact)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame fact "F")
                (declare (ignore var-1))
                (when var-0 (setf fact var-0))
                ret-val)))
        (setf info -1))
       ((and
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
        (setf info -2))
```

```
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -11))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -13))
 ((and
   (< lwork
      (max (the f2cl-lib:integer4 1)
           (the f2cl-lib:integer4 (f2cl-lib:int-mul 2 n))))
   (not lquery))
  (setf info -18)))
(cond
 ((= info 0)
  (setf lwkopt
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul 2 n))))
  (cond
   (nofact
    (setf nb
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 1 "ZSYTRF" uplo n -1 -1 -1)
              (declare (ignore var-0 var-1 var-4 var-5 var-6))
              (setf uplo var-2)
              (setf n var-3)
              ret-val))
    (setf lwkopt
            (max (the f2cl-lib:integer4 lwkopt)
                 (the f2cl-lib:integer4 (f2cl-lib:int-mul n nb))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwkopt 'f2cl-lib:complex16))))
(cond
 ((/= info 0) (xerbla "ZSYSVX" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond
 (nofact
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy uplo n n a lda af ldaf)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
    (setf uplo var-0))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zsytrf uplo n af ldaf ipiv work lwork info)
    (declare (ignore var-2 var-4 var-5))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
```

```
        (when var-3 (setf ldaf var-3))
        (when var-6 (setf lwork var-6))
        (when var-7 (setf info var-7)))
      (cond ((> info 0) (setf rcond zero) (go end_label)))))
    (setf anorm
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                (zlansy "I" uplo n a lda rwork)
              (declare (ignore var-0 var-3 var-4 var-5))
              (setf uplo var-1)
              (setf n var-2)
              ret-val))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (zsycon uplo n af ldaf ipiv anorm rcond work info)
      (declare (ignore var-2 var-4 var-5 var-7))
      (setf uplo var-0)
      (setf n var-1)
      (setf ldaf var-3)
      (setf rcond var-6)
      (setf info var-8))
    (zlacpy "Full" n nrhs b ldb$ x ldx)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (zsytrs uplo n nrhs af ldaf ipiv x ldx info)
      (declare (ignore var-3 var-5 var-6))
      (when var-0 (setf uplo var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf nrhs var-2))
      (when var-4 (setf ldaf var-4))
      (when var-7 (setf ldx var-7))
      (when var-8 (setf info var-8)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
         var-11 var-12 var-13 var-14 var-15 var-16)
        (zsyrfs uplo n nrhs a lda af ldaf ipiv b ldb$ x ldx ferr berr work
         rwork info)
      (declare
       (ignore var-2 var-3 var-4 var-5 var-7 var-8 var-9 var-10 var-11
        var-12 var-13 var-14 var-15))
      (setf uplo var-0)
      (setf n var-1)
      (setf ldaf var-6)
      (setf info var-16))
    (if (< rcond (dlamch "Epsilon"))
        (setf info (f2cl-lib:int-add n 1)))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce lwkopt 'f2cl-lib:complex16))
    (go end_label)
   end_label
```

```
        (return
         (values fact uplo n nrhs nil nil nil ldaf nil nil nil nil ldx rcond
                 nil nil nil lwork nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsysvx
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string) (string)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::integer4
                                              (*))
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (double-float)
                                             (array double-float (*))
                                             (array double-float (*))
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(fortran-to-lisp::fact
                                             fortran-to-lisp::uplo
                                             fortran-to-lisp::n
                                             fortran-to-lisp::nrhs nil nil nil
                                             fortran-to-lisp::ldaf nil nil nil
                                             nil fortran-to-lisp::ldx
                                             fortran-to-lisp::rcond nil nil
                                             nil fortran-to-lisp::lwork nil
                                             fortran-to-lisp::info)
                                           :calls
                                           '(fortran-to-lisp::zsyrfs
                                             fortran-to-lisp::zsycon
                                             fortran-to-lisp::zlansy
                                             fortran-to-lisp::zlacpy
```

```
                                        fortran-to-lisp::xerbla
                                        fortran-to-lisp::ilaenv))))
```

———-

55.4 zsysvxx

55.4.1 Axiom unit tests

— **zsysvxx.input** —

```
)set break resume
)sys rm -f zsysvxx.output
)spool zsysvxx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

55.4.2 Axiom help page [Demm05]

— **zsysvxx.help** —

```
======================================================================
zsysvxx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zsysvxx (FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, EQUED,
           S, B, LDB, X, LDX, RCOND, RPVGRW, BERR, N_ERR_BNDS, ERR_BNDS_NORM,
           ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, RWORK, INFO)
            ZSYSVXX computes the solution to system of linear equations A * X
           = B for SY matrices
```

```
Function/Subroutine Documentation
   subroutine zsysvxx (character FACT, character UPLO, integer N, integer
       NRHS, complex*16, dimension( lda, * ) A, integer LDA, complex*16,
       dimension( ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV,
       character EQUED, double precision, dimension( * ) S, complex*16,
       dimension( ldb, * ) B, integer LDB, complex*16, dimension( ldx, * ) X,
       integer LDX, double precision RCOND, double precision RPVGRW, double
       precision, dimension( * ) BERR, integer N_ERR_BNDS, double precision,
       dimension( nrhs, * ) ERR_BNDS_NORM, double precision, dimension( nrhs,
       * ) ERR_BNDS_COMP, integer NPARAMS, double precision, dimension( * )
       PARAMS, complex*16, dimension( * ) WORK, double precision, dimension( *
       ) RWORK, integer INFO)
        ZSYSVXX computes the solution to system of linear equations A * X = B
       for SY matrices

Purpose:
   ZSYSVXX uses the diagonal pivoting factorization to compute the
   solution to a complex*16 system of linear equations A * X = B, where
   A is an N-by-N symmetric matrix and X and B are N-by-NRHS
   matrices.

   If requested, both normwise and maximum componentwise error bounds
   are returned. ZSYSVXX will return a solution with a tiny
   guaranteed error (O(eps) where eps is the working machine
   precision) unless the matrix is very ill-conditioned, in which
   case a warning is returned. Relevant condition numbers also are
   calculated and returned.

   ZSYSVXX accepts user-provided factorizations and equilibration
   factors; see the definitions of the FACT and EQUED options.
   Solving with refinement and using a factorization from a previous
   ZSYSVXX call will also produce a solution with either O(eps)
   errors or warnings, but we cannot make that claim for general
   user-provided factorizations and equilibration factors if they
   differ from what ZSYSVXX would itself produce.

Description:

   The following steps are performed:

   1. If FACT = 'E', double precision scaling factors are computed to
      equilibrate the system:

      diag(S)*A*diag(S)     *inv(diag(S))*X = diag(S)*B

      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(S)*A*diag(S) and B by diag(S)*B.
```

```
   2. If FACT = 'N' or 'E', the LU decomposition is used to factor
      the matrix A (after equilibration if FACT = 'E') as

   A = U * D * U**T,  if UPLO = 'U', or
   A = L * D * L**T,  if UPLO = 'L',

      where U (or L) is a product of permutation and unit upper (lower)
      triangular matrices, and D is symmetric and block diagonal with
      1-by-1 and 2-by-2 diagonal blocks.

   3. If some D(i,i)=0, so that D is exactly singular, then the
      routine returns with INFO = i. Otherwise, the factored form of A
      is used to estimate the condition number of the matrix A (see
      argument RCOND).  If the reciprocal of the condition number is
      less than machine precision, the routine still goes on to solve
      for X and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. By default (unless PARAMS(LA_LINRX_ITREF_I) is set to zero),
      the routine will use iterative refinement to try to get a small
      error and error bounds.  Refinement calculates the residual to at
      least twice the working precision.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(R) so that it solves the original system before equilibration.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AF and IPIV contain the factored form of A.
           If EQUED is not 'N', the matrix A has been
           equilibrated with scaling factors given by S.
           A, AF, and IPIV are not modified.
   = 'N':  The matrix A will be copied to AF and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
           copied to AF and factored.

 UPLO
   UPLO is CHARACTER*1
```

```
  = 'U':  Upper triangle of A is stored;
  = 'L':  Lower triangle of A is stored.

N
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
  upper triangular part of A contains the upper triangular
  part of the matrix A, and the strictly lower triangular
  part of A is not referenced.  If UPLO = 'L', the leading
  N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if FACT = 'E' and EQUED = 'Y', A is overwritten by
  diag(S)*A*diag(S).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX*16 array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the block diagonal matrix D and the multipliers
  used to obtain the factor U or L from the factorization A =
  U*D*U**T or A = L*D*L**T as computed by DSYTRF.

  If FACT = 'N', then AF is an output argument and on exit
  returns the block diagonal matrix D and the multipliers
  used to obtain the factor U or L from the factorization A =
  U*D*U**T or A = L*D*L**T.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains details of the interchanges and the block
  structure of D, as determined by DSYTRF.  If IPIV(k) > 0,
```

```
  then rows and columns k and IPIV(k) were interchanged and
  D(k,k) is a 1-by-1 diagonal block.  If UPLO = 'U' and
  IPIV(k) = IPIV(k-1) < 0, then rows and columns k-1 and
  -IPIV(k) were interchanged and D(k-1:k,k-1:k) is a 2-by-2
  diagonal block.  If UPLO = 'L' and IPIV(k) = IPIV(k+1) < 0,
  then rows and columns k+1 and -IPIV(k) were interchanged
  and D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

  If FACT = 'N', then IPIV is an output argument and on exit
  contains details of the interchanges and the block
  structure of D, as determined by DSYTRF.

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'Y':  Both row and column equilibration, i.e., A has been
          replaced by diag(S) * A * diag(S).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

S
  S is DOUBLE PRECISION array, dimension (N)
  The scale factors for A.  If EQUED = 'Y', A is multiplied on
  the left and right by diag(S).  S is an input argument if FACT =
  'F'; otherwise, S is an output argument.  If FACT = 'F' and EQUED
  = 'Y', each element of S must be positive.  If S is output, each
  element of S is a power of the radix. If S is input, each element
  of S should be a power of the radix to ensure a reliable solution
  and error estimates. Scaling by powers of the radix does not cause
  rounding errors unless the result underflows or overflows.
  Rounding errors during scaling lead to refining with a matrix that
  is not equivalent to the input matrix, producing error estimates
  that may not be reliable.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if EQUED = 'Y', B is overwritten by diag(S)*B;

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  If INFO = 0, the N-by-NRHS solution matrix X to the original
  system of equations.  Note that A and B are modified on exit if
```

```
  EQUED .ne. 'N', and the solution to the equilibrated system is
  inv(diag(S))*X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

RPVGRW
  RPVGRW is DOUBLE PRECISION
  Reciprocal pivot growth.  On exit, this contains the reciprocal
  pivot growth factor norm(A)/norm(U). The "max absolute element"
  norm is used.  If this is much less than 1, then the stability of
  the LU factorization of the (equilibrated) matrix A could be poor.
  This also means that the solution X, estimated condition numbers,
  and error bounds could be unreliable. If factorization fails with
  0<INFO<=N, then this contains the reciprocal pivot growth factor
  for the leading INFO columns of A.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
           max_j (abs(XTRUE(j,i) - X(j,i)))
            ------------------------------
                  max_j abs(X(j,i))
```

```
  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * dlamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * dlamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.


ERR_BNDS_COMP
  ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                   abs(XTRUE(j,i) - X(j,i))
             max_j ----------------------
                         abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
```

```
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * dlamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * dlamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is DOUBLE PRECISION array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0D+0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the extra-precise refinement algorithm.
  (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
```

```
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be
              trustworthy.

 PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
   will attempt to find a solution with small componentwise
   relative error in the double-precision algorithm.  Positive
   is true, 0.0 is false.
   Default: 1.0 (attempt componentwise convergence)

 WORK
   WORK is COMPLEX*16 array, dimension (2*N)

 RWORK
   RWORK is DOUBLE PRECISION array, dimension (2*N)

 INFO
   INFO is INTEGER
   = 0:  Successful exit. The solution to every right-hand side is
   guaranteed.
   < 0:  If INFO = -i, the i-th argument had an illegal value
   > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
      has been completed, but the factor U is exactly singular, so
      the solution and error bounds could not be computed. RCOND = 0
      is returned.
   = N+J: The solution corresponding to the Jth right-hand side is
      not guaranteed. The solutions corresponding to other right-
      hand sides K with K > J may not be guaranteed as well, but
      only the first such right-hand side is reported. If a small
      componentwise error is not requested (PARAMS(3) = 0.0) then
      the Jth right-hand side is the first with a normwise error
      bound that is not guaranteed (the smallest J such
      that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
      the Jth right-hand side is the first with either a normwise or
      componentwise error bound that is not guaranteed (the smallest
      J such that either ERR_BNDS_NORM(J,1) = 0.0 or
      ERR_BNDS_COMP(J,1) = 0.0). See the definition of
      ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
      about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

———-

55.4.3 fortran code

— zsysvxx.f —

```
*  ==================================================================
```

```
      SUBROUTINE ZSYSVXX( FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $                    EQUED, S, B, LDB, X, LDX, RCOND, RPVGRW, BERR,
     $                    N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP,
     $                    NPARAMS, PARAMS, WORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      DOUBLE PRECISION   RCOND, RPVGRW
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX, * ), WORK( * )
      DOUBLE PRECISION   S( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * ), RWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
*     ..
*     .. Local Scalars ..
      LOGICAL            EQUIL, NOFACT, RCEQU
      INTEGER            INFEQU, J
      DOUBLE PRECISION   AMAX, BIGNUM, SMIN, SMAX, SCOND, SMLNUM
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, DLAMCH, ZLA_SYRPVGRW
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, ZLA_SYRPVGRW
*     ..
*     .. External Subroutines ..
```

```
      EXTERNAL           ZSYCON, ZSYEQUB, ZSYTRF, ZSYTRS, ZLACPY,
     $                   ZLAQSY, XERBLA, ZLASCL2, ZSYRFSX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      SMLNUM = DLAMCH( 'Safe minimum' )
      BIGNUM = ONE / SMLNUM
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         RCEQU = .FALSE.
      ELSE
         RCEQU = LSAME( EQUED, 'Y' )
      ENDIF
*
*     Default is failure.  If an input parameter is wrong or
*     factorization fails, make everything look horrible.  Only the
*     pivot growth is set here, the rest is initialized in ZSYRFSX.
*
      RPVGRW = ZERO
*
*     Test the input parameters.  PARAMS is not tested until ZSYRFSX.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.
     $     LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.LSAME(UPLO, 'U') .AND.
     $         .NOT.LSAME(UPLO, 'L') ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $        ( RCEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -10
      ELSE
         IF ( RCEQU ) THEN
            SMIN = BIGNUM
            SMAX = ZERO
            DO 10 J = 1, N
```

```
               SMIN = MIN( SMIN, S( J ) )
               SMAX = MAX( SMAX, S( J ) )
 10         CONTINUE
            IF( SMIN.LE.ZERO ) THEN
               INFO = -11
            ELSE IF( N.GT.0 ) THEN
               SCOND = MAX( SMIN, SMLNUM ) / MIN( SMAX, BIGNUM )
            ELSE
               SCOND = ONE
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -13
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -15
            END IF
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYSVXX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*     Compute row and column scalings to equilibrate the matrix A.
*
         CALL ZSYEQUB( UPLO, N, A, LDA, S, SCOND, AMAX, WORK, INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*     Equilibrate the matrix.
*
            CALL ZLAQSY( UPLO, N, A, LDA, S, SCOND, AMAX, EQUED )
            RCEQU = LSAME( EQUED, 'Y' )
         END IF

      END IF
*
*     Scale the right hand-side.
*
      IF( RCEQU ) CALL ZLASCL2( N, NRHS, S, B, LDB )
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the LDL^T or UDU^T factorization of A.
*
         CALL ZLACPY( UPLO, N, N, A, LDA, AF, LDAF )
         CALL ZSYTRF( UPLO, N, AF, LDAF, IPIV, WORK, 5*MAX(1,N), INFO )
```

```
*
*       Return if INFO is non-zero.
*
        IF( INFO.GT.0 ) THEN
*
*          Pivot in column INFO is exactly 0
*          Compute the reciprocal pivot growth factor of the
*          leading rank-deficient INFO columns of A.
*
           IF ( N.GT.0 )
     $           RPVGRW = ZLA_SYRPVGRW( UPLO, N, INFO, A, LDA, AF,
     $           LDAF, IPIV, RWORK )
           RETURN
        END IF
      END IF
*
*     Compute the reciprocal pivot growth factor RPVGRW.
*
      IF ( N.GT.0 )
     $     RPVGRW = ZLA_SYRPVGRW( UPLO, N, INFO, A, LDA, AF, LDAF,
     $     IPIV, RWORK )
*
*     Compute the solution matrix X.
*
      CALL ZLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL ZSYTRS( UPLO, N, NRHS, AF, LDAF, IPIV, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL ZSYRFSX( UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $     S, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS, ERR_BNDS_NORM,
     $     ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, RWORK, INFO )
*
*     Scale solutions.
*
      IF ( RCEQU ) THEN
         CALL ZLASCL2 (N, NRHS, S, X, LDX )
      END IF
*
      RETURN
*
*     End of ZSYSVXX
*
      END
```

———-

55.4.4 lisp code

— zsysvxx.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun zsysvxx
         (fact uplo n nrhs a lda af ldaf ipiv equed s b ldb$ x ldx rcond rpvgrw
          berr n_err_bnds err_bnds_norm err_bnds_comp nparams params work rwork
          info)
    (declare (type (double-float) rpvgrw rcond)
             (type (array double-float (*)) rwork params err_bnds_comp
              err_bnds_norm berr s)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) work x b af a)
             (type (f2cl-lib:integer4) info nparams n_err_bnds ldx ldb$ ldaf
              lda nrhs n)
             (type (string *) equed uplo fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (equed character equed-%data% equed-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (af f2cl-lib:complex16 af-%data% af-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (x f2cl-lib:complex16 x-%data% x-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (s double-float s-%data% s-%offset%)
         (berr double-float berr-%data% berr-%offset%)
         (err_bnds_norm double-float err_bnds_norm-%data%
          err_bnds_norm-%offset%)
         (err_bnds_comp double-float err_bnds_comp-%data%
          err_bnds_comp-%offset%)
         (params double-float params-%data% params-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (prog ((amax 0.0d0) (bignum 0.0d0) (smin 0.0d0) (smax 0.0d0)
             (scond 0.0d0) (smlnum 0.0d0) (infequ 0) (j 0) (equil nil)
             (nofact nil) (rcequ nil))
        (declare (type (double-float) amax bignum smin smax scond smlnum)
                 (type (f2cl-lib:integer4) infequ j)
                 (type f2cl-lib:logical equil nofact rcequ))
        (setf info 0)
        (setf nofact
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame fact "N")
                  (declare (ignore var-1))
```

```
          (when var-0 (setf fact var-0))
          ret-val))
(setf equil
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "E")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val))
(setf smlnum (dlamch "Safe minimum"))
(setf bignum (/ one smlnum))
(cond
 ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
  (setf rcequ f2cl-lib:%false%))
 (t
  (setf rcequ
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame equed "Y")
            (declare (ignore var-1))
            (when var-0 (setf equed var-0))
            ret-val))))
(setf rpvgrw zero)
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and
```

```
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame fact "F")
      (declare (ignore var-1))
      (when var-0 (setf fact var-0))
      ret-val)
    (not
     (or rcequ
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame equed "N")
           (declare (ignore var-1))
           (when var-0 (setf equed var-0))
           ret-val))))
  (setf info -10))
 (t
  (cond
   (rcequ (setf smin bignum) (setf smax zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf smin
                (min smin
                     (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
        (setf smax
                (max smax
                     (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
       label10))
    (cond ((<= smin zero) (setf info -11))
          ((> n 0)
           (setf scond (/ (max smin smlnum) (min smax bignum))))
          (t (setf scond one)))))
  (cond
   ((= info 0)
    (cond
     ((< ldb$
         (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -13))
     ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -15)))))))
(cond
 ((/= info 0) (xerbla "ZSYSVXX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (equil
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zsyequb uplo n a lda s scond amax work infequ)
    (declare (ignore var-1 var-2 var-3 var-4 var-7))
    (setf uplo var-0)
    (setf scond var-5)
    (setf amax var-6)
```

```
    (setf infequ var-8))
  (cond
   ((= infequ 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (zlaqsy uplo n a lda s scond amax equed)
      (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
      (setf uplo var-0)
      (setf equed var-7))
    (setf rcequ
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame equed "Y")
              (declare (ignore var-1))
              (when var-0 (setf equed var-0))
              ret-val))))))
(if rcequ
    (zlascl2 n nrhs s b ldb$))
(cond
 ((or nofact equil)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy uplo n n a lda af ldaf)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
    (setf uplo var-0))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zsytrf uplo n af ldaf ipiv work
       (f2cl-lib:int-mul 5
                         (max (the f2cl-lib:integer4 1)
                              (the f2cl-lib:integer4 n)))
       info)
    (declare (ignore var-2 var-4 var-5 var-6))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf ldaf var-3))
    (when var-7 (setf info var-7)))
  (cond
   ((> info 0)
    (if (> n 0)
        (setf rpvgrw
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6
                     var-7 var-8)
                    (zla_syrpvgrw uplo n info a lda af ldaf ipiv rwork)
                  (declare
                   (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7
                    var-8))
                  (setf uplo var-0)
                  ret-val)))
    (go end_label)))))
(if (> n 0)
```

```
      (setf rpvgrw
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6
                   var-7 var-8)
                  (zla_syrpvgrw uplo n info a lda af ldaf ipiv rwork)
                (declare
                 (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7
                  var-8))
                (setf uplo var-0)
                ret-val)))
  (zlacpy "Full" n nrhs b ldb$ x ldx)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zsytrs uplo n nrhs af ldaf ipiv x ldx info)
    (declare (ignore var-3 var-5 var-6))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf nrhs var-2))
    (when var-4 (setf ldaf var-4))
    (when var-7 (setf ldx var-7))
    (when var-8 (setf info var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
       var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19
       var-20 var-21 var-22 var-23)
      (zsyrfsx uplo equed n nrhs a lda af ldaf ipiv s b ldb$ x ldx rcond
       berr n_err_bnds err_bnds_norm err_bnds_comp nparams params work
       rwork info)
    (declare
     (ignore var-4 var-6 var-8 var-9 var-10 var-12 var-15 var-17 var-18
      var-20 var-21 var-22))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf equed var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf nrhs var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldaf var-7))
    (when var-11 (setf ldb$ var-11))
    (when var-13 (setf ldx var-13))
    (when var-14 (setf rcond var-14))
    (when var-16 (setf n_err_bnds var-16))
    (when var-19 (setf nparams var-19))
    (when var-23 (setf info var-23)))
  (cond (rcequ (zlascl2 n nrhs s x ldx)))
  (go end_label)
end_label
  (return
   (values fact uplo n nrhs nil lda nil ldaf nil equed nil nil ldb$ nil
           ldx rcond rpvgrw nil n_err_bnds nil nil nparams nil nil nil
           info))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsysvxx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (string) (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::fact
                                              fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldaf nil
                                              fortran-to-lisp::equed nil nil
                                              fortran-to-lisp::ldb$ nil
                                              fortran-to-lisp::ldx
                                              fortran-to-lisp::rcond
                                              fortran-to-lisp::rpvgrw nil
                                              fortran-to-lisp::n_err_bnds nil
                                              nil fortran-to-lisp::nparams nil
```

```
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zla_syrpvgrw
  fortran-to-lisp::zlacpy
  fortran-to-lisp::zlascl2
  fortran-to-lisp::zlaqsy
  fortran-to-lisp::zsyequb
  fortran-to-lisp::xerbla))))
```

———-

Chapter 56

LAPACK - Symmetric Matrix, Eigenvalue, Double

56.1 dsyev

56.1.1 Axiom unit tests

— **dsyev.input** —

```
)set break resume
)sys rm -f dsyev.output
)spool dsyev.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

56.1.2 Axiom help page

— **dsyev.help** —

```
==========================================================================
dsyev examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsyev (JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, INFO)
            DSYEV computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for SY matrices

Function/Subroutine Documentation
   subroutine dsyev (character JOBZ, character UPLO, integer N, double
       precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( * ) W, double precision, dimension( * ) WORK, integer LWORK,
       integer INFO)
        DSYEV computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for SY matrices

Purpose:
   DSYEV computes all eigenvalues and, optionally, eigenvectors of a
   real symmetric matrix A.

Parameters:

 JOBZ
   JOBZ is CHARACTER*1
   = 'N':  Compute eigenvalues only;
   = 'V':  Compute eigenvalues and eigenvectors.

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA, N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the
   leading N-by-N upper triangular part of A contains the
   upper triangular part of the matrix A.  If UPLO = 'L',
   the leading N-by-N lower triangular part of A contains
   the lower triangular part of the matrix A.
   On exit, if JOBZ = 'V', then if INFO = 0, A contains the
   orthonormal eigenvectors of the matrix A.
   If JOBZ = 'N', then on exit the lower triangle (if UPLO='L')
   or the upper triangle (if UPLO='U') of A, including the
```

```
  diagonal, is destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

W
  W is DOUBLE PRECISION array, dimension (N)
  If INFO = 0, the eigenvalues in ascending order.

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of the array WORK.  LWORK >= max(1,3*N-1).
  For optimal efficiency, LWORK >= (NB+2)*N,
  where NB is the blocksize for DSYTRD returned by ILAENV.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, the algorithm failed to converge; i
        off-diagonal elements of an intermediate tridiagonal
        form did not converge to zero.
```

———-

56.1.3 fortran code

— dsyev.f —

```
*  =====================================================================
      SUBROUTINE DSYEV( JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
```

```
*     .. Scalar Arguments ..
      CHARACTER          JOBZ, UPLO
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), W( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LOWER, LQUERY, WANTZ
      INTEGER            IINFO, IMAX, INDE, INDTAU, INDWRK, ISCALE,
     $                   LLWORK, LWKOPT, NB
      DOUBLE PRECISION   ANRM, BIGNUM, EPS, RMAX, RMIN, SAFMIN, SIGMA,
     $                   SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANSY
      EXTERNAL           LSAME, ILAENV, DLAMCH, DLANSY
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASCL, DORGTR, DSCAL, DSTEQR, DSTERF, DSYTRD,
     $                   XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      WANTZ = LSAME( JOBZ, 'V' )
      LOWER = LSAME( UPLO, 'L' )
      LQUERY = ( LWORK.EQ.-1 )
*
      INFO = 0
      IF( .NOT.( WANTZ .OR. LSAME( JOBZ, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( .NOT.( LOWER .OR. LSAME( UPLO, 'U' ) ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
```

```
         INFO = -5
      END IF
*
      IF( INFO.EQ.0 ) THEN
         NB = ILAENV( 1, 'DSYTRD', UPLO, N, -1, -1, -1 )
         LWKOPT = MAX( 1, ( NB+2 )*N )
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.MAX( 1, 3*N-1 ) .AND. .NOT.LQUERY )
     $      INFO = -8
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYEV ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         RETURN
      END IF
*
      IF( N.EQ.1 ) THEN
         W( 1 ) = A( 1, 1 )
         WORK( 1 ) = 2
         IF( WANTZ )
     $      A( 1, 1 ) = ONE
         RETURN
      END IF
*
*     Get machine constants.
*
      SAFMIN = DLAMCH( 'Safe minimum' )
      EPS = DLAMCH( 'Precision' )
      SMLNUM = SAFMIN / EPS
      BIGNUM = ONE / SMLNUM
      RMIN = SQRT( SMLNUM )
      RMAX = SQRT( BIGNUM )
*
*     Scale matrix to allowable range, if necessary.
*
      ANRM = DLANSY( 'M', UPLO, N, A, LDA, WORK )
      ISCALE = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.RMIN ) THEN
         ISCALE = 1
         SIGMA = RMIN / ANRM
      ELSE IF( ANRM.GT.RMAX ) THEN
```

```
         ISCALE = 1
         SIGMA = RMAX / ANRM
      END IF
      IF( ISCALE.EQ.1 )
     $   CALL DLASCL( UPLO, 0, 0, ONE, SIGMA, N, N, A, LDA, INFO )
*
*     Call DSYTRD to reduce symmetric matrix to tridiagonal form.
*
      INDE = 1
      INDTAU = INDE + N
      INDWRK = INDTAU + N
      LLWORK = LWORK - INDWRK + 1
      CALL DSYTRD( UPLO, N, A, LDA, W, WORK( INDE ), WORK( INDTAU ),
     $             WORK( INDWRK ), LLWORK, IINFO )
*
*     For eigenvalues only, call DSTERF.  For eigenvectors, first call
*     DORGTR to generate the orthogonal matrix, then call DSTEQR.
*
      IF( .NOT.WANTZ ) THEN
         CALL DSTERF( N, W, WORK( INDE ), INFO )
      ELSE
         CALL DORGTR( UPLO, N, A, LDA, WORK( INDTAU ), WORK( INDWRK ),
     $                LLWORK, IINFO )
         CALL DSTEQR( JOBZ, N, W, WORK( INDE ), A, LDA, WORK( INDTAU ),
     $                INFO )
      END IF
*
*     If matrix was scaled, then rescale eigenvalues appropriately.
*
      IF( ISCALE.EQ.1 ) THEN
         IF( INFO.EQ.0 ) THEN
            IMAX = N
         ELSE
            IMAX = INFO - 1
         END IF
         CALL DSCAL( IMAX, ONE / SIGMA, W, 1 )
      END IF
*
*     Set WORK(1) to optimal workspace size.
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of DSYEV
*
      END
```

56.1.4 lisp code

— dsyev.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dsyev (jobz uplo n a lda w work lwork info)
    (declare (type (array double-float (*)) work w a)
             (type (f2cl-lib:integer4) info lwork lda n)
             (type (string *) uplo jobz))
    (f2cl-lib:with-multi-array-data
        ((jobz character jobz-%data% jobz-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (w double-float w-%data% w-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((anrm 0.0d0) (bignum 0.0d0) (eps 0.0d0) (rmax 0.0d0) (rmin 0.0d0)
             (safmin 0.0d0) (sigma 0.0d0) (smlnum 0.0d0) (iinfo 0) (imax 0)
             (inde 0) (indtau 0) (indwrk 0) (iscale 0) (llwork 0) (lwkopt 0)
             (nb 0) (lower nil) (lquery nil) (wantz nil))
        (declare
         (type (double-float) anrm bignum eps rmax rmin safmin sigma smlnum)
         (type (f2cl-lib:integer4) iinfo imax inde indtau indwrk iscale llwork
          lwkopt nb)
         (type f2cl-lib:logical lower lquery wantz))
        (setf wantz
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame jobz "V")
                  (declare (ignore var-1))
                  (when var-0 (setf jobz var-0))
                  ret-val))
        (setf lower
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (setf info 0)
        (cond
         ((not
           (or wantz
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame jobz "N")
                 (declare (ignore var-1))
                 (when var-0 (setf jobz var-0))
                 ret-val)))
```

```
  (setf info -1))
 ((not
   (or lower
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame uplo "U")
         (declare (ignore var-1))
         (when var-0 (setf uplo var-0))
         ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5)))
(cond
 ((= info 0)
  (setf nb
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (ilaenv 1 "DSYTRD" uplo n -1 -1 -1)
            (declare (ignore var-0 var-1 var-4 var-5 var-6))
            (when var-2 (setf uplo var-2))
            (when var-3 (setf n var-3))
            ret-val))
  (setf lwkopt
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-mul (f2cl-lib:int-add nb 2) n))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
  (if (and
       (< lwork
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-sub (f2cl-lib:int-mul 3 n) 1))))
       (not lquery))
      (setf info -8))))
(cond
 ((/= info 0) (xerbla "DSYEV " (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond ((= n 0) (go end_label)))
(cond
 ((= n 1)
  (setf (f2cl-lib:fref w-%data% (1) ((1 *)) w-%offset%)
          (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 2) 'double-float))
  (if wantz
      (setf (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)
            one))
  (go end_label)))
(setf safmin (dlamch "Safe minimum"))
```

```
(setf eps (dlamch "Precision"))
(setf smlnum (/ safmin eps))
(setf bignum (/ one smlnum))
(setf rmin (f2cl-lib:fsqrt smlnum))
(setf rmax (f2cl-lib:fsqrt bignum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlansy "M" uplo n a lda work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf uplo var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iscale 0)
(cond
 ((and (> anrm zero) (< anrm rmin)) (setf iscale 1)
  (setf sigma (/ rmin anrm)))
 ((> anrm rmax) (setf iscale 1) (setf sigma (/ rmax anrm))))
(if (= iscale 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl uplo 0 0 one sigma n n a lda info)
      (declare (ignore var-1 var-2 var-7))
      (when var-0 (setf uplo var-0))
      (when var-3 (setf one var-3))
      (when var-4 (setf sigma var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf info var-9))))
(setf inde 1)
(setf indtau (f2cl-lib:int-add inde n))
(setf indwrk (f2cl-lib:int-add indtau n))
(setf llwork (f2cl-lib:int-add (f2cl-lib:int-sub lwork indwrk) 1))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dsytrd uplo n a lda w
     (f2cl-lib:array-slice work-%data% double-float (inde) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (indtau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (indwrk) ((1 *))
                           work-%offset%)
     llwork iinfo)
  (declare (ignore var-2 var-4 var-5 var-6 var-7))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-8 (setf llwork var-8))
```

```
  (when var-9 (setf iinfo var-9)))
(cond
 ((not wantz)
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dsterf n w
       (f2cl-lib:array-slice work-%data% double-float (inde) ((1 *))
                             work-%offset%)
       info)
    (declare (ignore var-1 var-2))
    (when var-0 (setf n var-0))
    (when var-3 (setf info var-3))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dorgtr uplo n a lda
       (f2cl-lib:array-slice work-%data% double-float (indtau) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (indwrk) ((1 *))
                             work-%offset%)
       llwork iinfo)
    (declare (ignore var-2 var-4 var-5))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf lda var-3))
    (when var-6 (setf llwork var-6))
    (when var-7 (setf iinfo var-7)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dsteqr jobz n w
       (f2cl-lib:array-slice work-%data% double-float (inde) ((1 *))
                             work-%offset%)
       a lda
       (f2cl-lib:array-slice work-%data% double-float (indtau) ((1 *))
                             work-%offset%)
       info)
    (declare (ignore var-2 var-3 var-4 var-6))
    (when var-0 (setf jobz var-0))
    (when var-1 (setf n var-1))
    (when var-5 (setf lda var-5))
    (when var-7 (setf info var-7)))))
(cond
 ((= iscale 1)
  (cond ((= info 0) (setf imax n))
        (t (setf imax (f2cl-lib:int-sub info 1))))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dscal imax (/ one sigma) w 1)
    (declare (ignore var-1 var-2 var-3))
    (when var-0 (setf imax var-0)))))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
```

```
        (go end_label)
       end_label
        (return (values jobz uplo n nil lda nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsyev fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::jobz
                                              fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

56.2 dsyevd

56.2.1 Axiom unit tests

— dsyevd.input —

```
)set break resume
)sys rm -f dsyevd.output
)spool dsyevd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

56.2.2 Axiom help page

— dsyevd.help —

```
=====================================================================
dsyevd examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsyevd (JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, IWORK,
           LIWORK, INFO)
            DSYEVD computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for SY matrices

Function/Subroutine Documentation
   subroutine dsyevd (character JOBZ, character UPLO, integer N, double
       precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( * ) W, double precision, dimension( * ) WORK, integer LWORK,
       integer, dimension( * ) IWORK, integer LIWORK, integer INFO)
        DSYEVD computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for SY matrices

Purpose:
   DSYEVD computes all eigenvalues and, optionally, eigenvectors of a
   real symmetric matrix A. If eigenvectors are desired, it uses a
   divide and conquer algorithm.

   The divide and conquer algorithm makes very mild assumptions about
   floating point arithmetic. It will work on machines with a guard
   digit in add/subtract, or on those binary machines without guard
   digits which subtract like the Cray X-MP, Cray Y-MP, Cray C-90, or
   Cray-2. It could conceivably fail on hexadecimal or decimal machines
   without guard digits, but we know of none.

   Because of large use of BLAS of level 3, DSYEVD needs N**2 more
   workspace than DSYEVX.

Parameters:

 JOBZ
   JOBZ is CHARACTER*1
   = 'N':  Compute eigenvalues only;
   = 'V':  Compute eigenvalues and eigenvectors.
```

```
UPLO
  UPLO is CHARACTER*1
  = 'U':  Upper triangle of A is stored;
  = 'L':  Lower triangle of A is stored.

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA, N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the
  leading N-by-N upper triangular part of A contains the
  upper triangular part of the matrix A.  If UPLO = 'L',
  the leading N-by-N lower triangular part of A contains
  the lower triangular part of the matrix A.
  On exit, if JOBZ = 'V', then if INFO = 0, A contains the
  orthonormal eigenvectors of the matrix A.
  If JOBZ = 'N', then on exit the lower triangle (if UPLO='L')
  or the upper triangle (if UPLO='U') of A, including the
  diagonal, is destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

W
  W is DOUBLE PRECISION array, dimension (N)
  If INFO = 0, the eigenvalues in ascending order.

WORK
  WORK is DOUBLE PRECISION array, dimension (LWORK)
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  If N <= 1,               LWORK must be at least 1.
  If JOBZ = 'N' and N > 1, LWORK must be at least 2*N+1.
  If JOBZ = 'V' and N > 1, LWORK must be at least
  1 + 6*N + 2*N**2.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal sizes of the WORK and IWORK
  arrays, returns these values as the first entries of the WORK
  and IWORK arrays, and no error message related to LWORK or
  LIWORK is issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (MAX(1,LIWORK))
```

```
  On exit, if INFO = 0, IWORK(1) returns the optimal LIWORK.

LIWORK
  LIWORK is INTEGER
  The dimension of the array IWORK.
  If N <= 1,                LIWORK must be at least 1.
  If JOBZ  = 'N' and N > 1, LIWORK must be at least 1.
  If JOBZ  = 'V' and N > 1, LIWORK must be at least 3 + 5*N.

  If LIWORK = -1, then a workspace query is assumed; the
  routine only calculates the optimal sizes of the WORK and
  IWORK arrays, returns these values as the first entries of
  the WORK and IWORK arrays, and no error message related to
  LWORK or LIWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i and JOBZ = 'N', then the algorithm failed
        to converge; i off-diagonal elements of an intermediate
        tridiagonal form did not converge to zero;
        if INFO = i and JOBZ = 'V', then the algorithm failed
        to compute an eigenvalue while working on the submatrix
        lying in rows and columns INFO/(N+1) through
        mod(INFO,N+1).
```

———-

56.2.3 fortran code

— **dsyevd.f** —

```
*  =====================================================================
      SUBROUTINE DSYEVD( JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, IWORK,
     $                   LIWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          JOBZ, UPLO
      INTEGER            INFO, LDA, LIWORK, LWORK, N
*     ..
*     .. Array Arguments ..
```

```
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), W( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
*
      LOGICAL            LOWER, LQUERY, WANTZ
      INTEGER            IINFO, INDE, INDTAU, INDWK2, INDWRK, ISCALE,
     $                   LIOPT, LIWMIN, LLWORK, LLWRK2, LOPT, LWMIN
      DOUBLE PRECISION   ANRM, BIGNUM, EPS, RMAX, RMIN, SAFMIN, SIGMA,
     $                   SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANSY
      EXTERNAL           LSAME, DLAMCH, DLANSY, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLACPY, DLASCL, DORMTR, DSCAL, DSTEDC, DSTERF,
     $                   DSYTRD, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      WANTZ = LSAME( JOBZ, 'V' )
      LOWER = LSAME( UPLO, 'L' )
      LQUERY = ( LWORK.EQ.-1 .OR. LIWORK.EQ.-1 )
*
      INFO = 0
      IF( .NOT.( WANTZ .OR. LSAME( JOBZ, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( .NOT.( LOWER .OR. LSAME( UPLO, 'U' ) ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      END IF
*
```

```
      IF( INFO.EQ.0 ) THEN
         IF( N.LE.1 ) THEN
            LIWMIN = 1
            LWMIN = 1
            LOPT = LWMIN
            LIOPT = LIWMIN
         ELSE
            IF( WANTZ ) THEN
               LIWMIN = 3 + 5*N
               LWMIN = 1 + 6*N + 2*N**2
            ELSE
               LIWMIN = 1
               LWMIN = 2*N + 1
            END IF
            LOPT = MAX( LWMIN, 2*N +
     $                  ILAENV( 1, 'DSYTRD', UPLO, N, -1, -1, -1 ) )
            LIOPT = LIWMIN
         END IF
         WORK( 1 ) = LOPT
         IWORK( 1 ) = LIOPT
*
         IF( LWORK.LT.LWMIN .AND. .NOT.LQUERY ) THEN
            INFO = -8
         ELSE IF( LIWORK.LT.LIWMIN .AND. .NOT.LQUERY ) THEN
            INFO = -10
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYEVD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
      IF( N.EQ.1 ) THEN
         W( 1 ) = A( 1, 1 )
         IF( WANTZ )
     $      A( 1, 1 ) = ONE
         RETURN
      END IF
*
*     Get machine constants.
*
      SAFMIN = DLAMCH( 'Safe minimum' )
```

```
      EPS = DLAMCH( 'Precision' )
      SMLNUM = SAFMIN / EPS
      BIGNUM = ONE / SMLNUM
      RMIN = SQRT( SMLNUM )
      RMAX = SQRT( BIGNUM )
*
*     Scale matrix to allowable range, if necessary.
*
      ANRM = DLANSY( 'M', UPLO, N, A, LDA, WORK )
      ISCALE = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.RMIN ) THEN
         ISCALE = 1
         SIGMA = RMIN / ANRM
      ELSE IF( ANRM.GT.RMAX ) THEN
         ISCALE = 1
         SIGMA = RMAX / ANRM
      END IF
      IF( ISCALE.EQ.1 )
     $   CALL DLASCL( UPLO, 0, 0, ONE, SIGMA, N, N, A, LDA, INFO )
*
*     Call DSYTRD to reduce symmetric matrix to tridiagonal form.
*
      INDE = 1
      INDTAU = INDE + N
      INDWRK = INDTAU + N
      LLWORK = LWORK - INDWRK + 1
      INDWK2 = INDWRK + N*N
      LLWRK2 = LWORK - INDWK2 + 1
*
      CALL DSYTRD( UPLO, N, A, LDA, W, WORK( INDE ), WORK( INDTAU ),
     $             WORK( INDWRK ), LLWORK, IINFO )
*
*     For eigenvalues only, call DSTERF.  For eigenvectors, first call
*     DSTEDC to generate the eigenvector matrix, WORK(INDWRK), of the
*     tridiagonal matrix, then call DORMTR to multiply it by the
*     Householder transformations stored in A.
*
      IF( .NOT.WANTZ ) THEN
         CALL DSTERF( N, W, WORK( INDE ), INFO )
      ELSE
         CALL DSTEDC( 'I', N, W, WORK( INDE ), WORK( INDWRK ), N,
     $                WORK( INDWK2 ), LLWRK2, IWORK, LIWORK, INFO )
         CALL DORMTR( 'L', UPLO, 'N', N, N, A, LDA, WORK( INDTAU ),
     $                WORK( INDWRK ), N, WORK( INDWK2 ), LLWRK2, IINFO )
         CALL DLACPY( 'A', N, N, WORK( INDWRK ), N, A, LDA )
      END IF
*
*     If matrix was scaled, then rescale eigenvalues appropriately.
*
      IF( ISCALE.EQ.1 )
```

```
     $   CALL DSCAL( N, ONE / SIGMA, W, 1 )
*
      WORK( 1 ) = LOPT
      IWORK( 1 ) = LIOPT
*
      RETURN
*
*     End of DSYEVD
*
      END
```

———-

56.2.4 lisp code

— **dsyevd.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dsyevd (jobz uplo n a lda w work lwork iwork liwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (array double-float (*)) work w a)
             (type (f2cl-lib:integer4) info liwork lwork lda n)
             (type (string *) uplo jobz))
    (f2cl-lib:with-multi-array-data
        ((jobz character jobz-%data% jobz-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (w double-float w-%data% w-%offset%)
         (work double-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((anrm 0.0d0) (bignum 0.0d0) (eps 0.0d0) (rmax 0.0d0) (rmin 0.0d0)
             (safmin 0.0d0) (sigma 0.0d0) (smlnum 0.0d0) (iinfo 0) (inde 0)
             (indtau 0) (indwk2 0) (indwrk 0) (iscale 0) (liopt 0) (liwmin 0)
             (llwork 0) (llwrk2 0) (lopt 0) (lwmin 0) (lower nil) (lquery nil)
             (wantz nil))
        (declare
         (type (double-float) anrm bignum eps rmax rmin safmin sigma smlnum)
         (type (f2cl-lib:integer4) iinfo inde indtau indwk2 indwrk iscale liopt
          liwmin llwork llwrk2 lopt lwmin)
         (type f2cl-lib:logical lower lquery wantz))
        (setf wantz
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame jobz "V")
                  (declare (ignore var-1))
                  (when var-0 (setf jobz var-0))
```

```
          ret-val))
(setf lower
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(setf lquery (or (= lwork -1) (= liwork -1)))
(setf info 0)
(cond
 ((not
   (or wantz
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobz "N")
         (declare (ignore var-1))
         (when var-0 (setf jobz var-0))
         ret-val)))
  (setf info -1))
 ((not
   (or lower
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame uplo "U")
         (declare (ignore var-1))
         (when var-0 (setf uplo var-0))
         ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5)))
(cond
 ((= info 0)
  (cond
   ((<= n 1) (setf liwmin 1) (setf lwmin 1) (setf lopt lwmin)
    (setf liopt liwmin))
   (t
    (cond
     (wantz (setf liwmin (f2cl-lib:int-add 3 (f2cl-lib:int-mul 5 n)))
      (setf lwmin
              (f2cl-lib:int-add 1 (f2cl-lib:int-mul 6 n)
                                (f2cl-lib:int-mul 2 (expt n 2)))))
     (t (setf liwmin 1)
      (setf lwmin (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) 1))))
    (setf lopt
            (max (the f2cl-lib:integer4 lwmin)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        (multiple-value-bind
                                            (ret-val var-0 var-1 var-2
                                             var-3 var-4 var-5 var-6)
                                            (ilaenv 1 "DSYTRD" uplo n
```

```
                                                  -1 -1 -1)
                                                (declare
                                                 (ignore var-0 var-1 var-4
                                                  var-5 var-6))
                                                (when var-2
                                                  (setf uplo var-2))
                                                (when var-3 (setf n var-3))
                                                ret-val)))))
    (setf liopt liwmin)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lopt) 'double-float))
  (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) liopt)
  (cond ((and (< lwork lwmin) (not lquery)) (setf info -8))
        ((and (< liwork liwmin) (not lquery)) (setf info -10)))))
(cond
 ((/= info 0) (xerbla "DSYEVD" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(cond
 ((= n 1)
  (setf (f2cl-lib:fref w-%data% (1) ((1 *)) w-%offset%)
          (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%))
  (if wantz
      (setf (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)
              one))
  (go end_label)))
(setf safmin (dlamch "Safe minimum"))
(setf eps (dlamch "Precision"))
(setf smlnum (/ safmin eps))
(setf bignum (/ one smlnum))
(setf rmin (f2cl-lib:fsqrt smlnum))
(setf rmax (f2cl-lib:fsqrt bignum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlansy "M" uplo n a lda work)
          (declare (ignore var-0 var-3 var-4 var-5))
          (setf uplo var-1)
          (setf n var-2)
          ret-val))
(setf iscale 0)
(cond
 ((and (> anrm zero) (< anrm rmin)) (setf iscale 1)
  (setf sigma (/ rmin anrm)))
 ((> anrm rmax) (setf iscale 1) (setf sigma (/ rmax anrm))))
(if (= iscale 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl uplo 0 0 one sigma n n a lda info)
```

```
      (declare (ignore var-1 var-2 var-5 var-6 var-7 var-8))
      (setf uplo var-0)
      (setf one var-3)
      (setf sigma var-4)
      (setf info var-9)))
(setf inde 1)
(setf indtau (f2cl-lib:int-add inde n))
(setf indwrk (f2cl-lib:int-add indtau n))
(setf llwork (f2cl-lib:int-add (f2cl-lib:int-sub lwork indwrk) 1))
(setf indwk2 (f2cl-lib:int-add indwrk (f2cl-lib:int-mul n n)))
(setf llwrk2 (f2cl-lib:int-add (f2cl-lib:int-sub lwork indwk2) 1))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dsytrd uplo n a lda w
     (f2cl-lib:array-slice work-%data% double-float (inde) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (indtau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (indwrk) ((1 *))
                           work-%offset%)
     llwork iinfo)
  (declare (ignore var-2 var-4 var-5 var-6 var-7))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-8 (setf llwork var-8))
  (when var-9 (setf iinfo var-9)))
(cond
 ((not wantz)
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dsterf n w
       (f2cl-lib:array-slice work-%data% double-float (inde) ((1 *))
                             work-%offset%)
       info)
    (declare (ignore var-0 var-1 var-2))
    (setf info var-3)))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dstedc "I" n w
       (f2cl-lib:array-slice work-%data% double-float (inde) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (indwrk) ((1 *))
                             work-%offset%)
       n
       (f2cl-lib:array-slice work-%data% double-float (indwk2) ((1 *))
                             work-%offset%)
       llwrk2 iwork liwork info)
    (declare (ignore var-0 var-2 var-3 var-4 var-6 var-7 var-8 var-9))
```

```
            (setf n var-1)
            (setf n var-5)
            (setf info var-10))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10 var-11 var-12)
              (dormtr "L" uplo "N" n n a lda
               (f2cl-lib:array-slice work-%data% double-float (indtau) ((1 *))
                                     work-%offset%)
               (f2cl-lib:array-slice work-%data% double-float (indwrk) ((1 *))
                                     work-%offset%)
               n
               (f2cl-lib:array-slice work-%data% double-float (indwk2) ((1 *))
                                     work-%offset%)
               llwrk2 iinfo)
            (declare (ignore var-0 var-2 var-5 var-7 var-8 var-10 var-11))
            (setf uplo var-1)
            (setf n var-3)
            (setf n var-4)
            (setf lda var-6)
            (setf n var-9)
            (setf iinfo var-12))
          (dlacpy "A" n n
           (f2cl-lib:array-slice work-%data% double-float (indwrk) ((1 *))
                                 work-%offset%)
           n a lda)))
        (if (= iscale 1)
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (dscal n (/ one sigma) w 1)
              (declare (ignore var-1 var-2 var-3))
              (when var-0 (setf n var-0))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lopt) 'double-float))
        (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) liopt)
        (go end_label)
       end_label
        (return (values jobz uplo n nil lda nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsyevd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
```

```
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::jobz
  fortran-to-lisp::uplo
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::dlacpy
  fortran-to-lisp::dormtr
  fortran-to-lisp::dstedc
  fortran-to-lisp::dsterf
  fortran-to-lisp::dlascl
  fortran-to-lisp::dlansy))))
```

———-

56.3 dsyevr

56.3.1 Axiom unit tests

— dsyevr.input —

```
)set break resume
)sys rm -f dsyevr.output
)spool dsyevr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

56.3.2 Axiom help page

— dsyevr.help —

```
====================================================================
dsyevr examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsyevr (JOBZ, RANGE, UPLO, N, A, LDA, VL, VU, IL, IU,
           ABSTOL, M, W, Z, LDZ, ISUPPZ, WORK, LWORK, IWORK, LIWORK, INFO)
            DSYEVR computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for SY matrices

Function/Subroutine Documentation
   subroutine dsyevr (character JOBZ, character RANGE, character UPLO, integer
       N, double precision, dimension( lda, * ) A, integer LDA, double
       precision VL, double precision VU, integer IL, integer IU, double
       precision ABSTOL, integer M, double precision, dimension( * ) W, double
       precision, dimension( ldz, * ) Z, integer LDZ, integer, dimension( * )
       ISUPPZ, double precision, dimension( * ) WORK, integer LWORK, integer,
       dimension( * ) IWORK, integer LIWORK, integer INFO)
        DSYEVR computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for SY matrices

       Purpose:
   DSYEVR computes selected eigenvalues and, optionally, eigenvectors
   of a real symmetric matrix A.  Eigenvalues and eigenvectors can be
   selected by specifying either a range of values or a range of
   indices for the desired eigenvalues.

   DSYEVR first reduces the matrix A to tridiagonal form T with a call
   to DSYTRD.  Then, whenever possible, DSYEVR calls DSTEMR to compute
   the eigenspectrum using Relatively Robust Representations.  DSTEMR
   computes eigenvalues by the dqds algorithm, while orthogonal
   eigenvectors are computed from various "good" L D L^T representations
   (also known as Relatively Robust Representations). Gram-Schmidt
   orthogonalization is avoided as far as possible. More specifically,
   the various steps of the algorithm are as follows.

   For each unreduced block (submatrix) of T,
   (a) Compute T - sigma I  = L D L^T, so that L and D
       define all the wanted eigenvalues to high relative accuracy.
       This means that small relative changes in the entries of D and L
       cause only small relative changes in the eigenvalues and
       eigenvectors. The standard (unfactored) representation of the
       tridiagonal matrix T does not have this property in general.
   (b) Compute the eigenvalues to suitable accuracy.
       If the eigenvectors are desired, the algorithm attains full
```

```
      accuracy of the computed eigenvalues only right before
      the corresponding vectors have to be computed, see steps c) and d).
  (c) For each cluster of close eigenvalues, select a new
      shift close to the cluster, find a new factorization, and refine
      the shifted eigenvalues to suitable accuracy.
  (d) For each eigenvalue with a large enough relative separation compute
      the corresponding eigenvector by forming a rank revealing twisted
      factorization. Go back to (c) for any clusters that remain.

  The desired accuracy of the output can be specified by the input
  parameter ABSTOL.

  Note 1 : DSYEVR calls DSTEMR when the full spectrum is requested
  on machines which conform to the ieee-754 floating point standard.
  DSYEVR calls DSTEBZ and SSTEIN on non-ieee machines and
  when partial spectrum requests are made.

  Normal execution of DSTEMR may create NaNs and infinities and hence
  may abort due to a floating point exception in environments which
  do not handle NaNs and infinities in the ieee standard default
  manner.

Parameters:

 JOBZ
   JOBZ is CHARACTER*1
   = 'N':  Compute eigenvalues only;
   = 'V':  Compute eigenvalues and eigenvectors.

 RANGE
   RANGE is CHARACTER*1
   = 'A': all eigenvalues will be found.
   = 'V': all eigenvalues in the half-open interval (VL,VU] will be found.
   = 'I': the IL-th through IU-th eigenvalues will be found.
   For RANGE = 'V' or 'I' and IU - IL < N - 1, DSTEBZ and DSTEIN are called

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA, N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the
   leading N-by-N upper triangular part of A contains the
   upper triangular part of the matrix A.  If UPLO = 'L',
```

```
  the leading N-by-N lower triangular part of A contains
  the lower triangular part of the matrix A.
  On exit, the lower triangle (if UPLO='L') or the upper
  triangle (if UPLO='U') of A, including the diagonal, is destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

VL
  VL is DOUBLE PRECISION

VU
  VU is DOUBLE PRECISION
  If RANGE='V', the lower and upper bounds of the interval to
  be searched for eigenvalues. VL < VU.
  Not referenced if RANGE = 'A' or 'I'.

IL
  IL is INTEGER

IU
  IU is INTEGER
  If RANGE='I', the indices (in ascending order) of the
  smallest and largest eigenvalues to be returned.
  1 <= IL <= IU <= N, if N > 0; IL = 1 and IU = 0 if N = 0.
  Not referenced if RANGE = 'A' or 'V'.

ABSTOL
  ABSTOL is DOUBLE PRECISION
  The absolute error tolerance for the eigenvalues.
  An approximate eigenvalue is accepted as converged
  when it is determined to lie in an interval [a,b]
  of width less than or equal to

  ABSTOL + EPS *   max( |a|,|b| ) ,

  where EPS is the machine precision.  If ABSTOL is less than
  or equal to zero, then  EPS*|T|  will be used in its place,
  where |T| is the 1-norm of the tridiagonal matrix obtained
  by reducing A to tridiagonal form.

  See "Computing Small Singular Values of Bidiagonal Matrices
  with Guaranteed High Relative Accuracy," by Demmel and
  Kahan, LAPACK Working Note #3.

  If high relative accuracy is important, set ABSTOL to
  DLAMCH( 'Safe minimum' ).  Doing so will guarantee that
  eigenvalues are computed to high relative accuracy when
  possible in future releases.  The current code does not
```

```
  make any guarantees about high relative accuracy, but
  future releases will. See J. Barlow and J. Demmel,
  "Computing Accurate Eigensystems of Scaled Diagonally
  Dominant Matrices", LAPACK Working Note #7, for a discussion
  of which matrices define their eigenvalues to high relative accuracy.

M
  M is INTEGER
  The total number of eigenvalues found.  0 <= M <= N.
  If RANGE = 'A', M = N, and if RANGE = 'I', M = IU-IL+1.

W
  W is DOUBLE PRECISION array, dimension (N)
  The first M elements contain the selected eigenvalues in
  ascending order.

Z
  Z is DOUBLE PRECISION array, dimension (LDZ, max(1,M))
  If JOBZ = 'V', then if INFO = 0, the first M columns of Z
  contain the orthonormal eigenvectors of the matrix A
  corresponding to the selected eigenvalues, with the i-th
  column of Z holding the eigenvector associated with W(i).
  If JOBZ = 'N', then Z is not referenced.
  Note: the user must ensure that at least max(1,M) columns are
  supplied in the array Z; if RANGE = 'V', the exact value of M
  is not known in advance and an upper bound must be used.
  Supplying N columns is always safe.

LDZ
  LDZ is INTEGER
  The leading dimension of the array Z.  LDZ >= 1, and if
  JOBZ = 'V', LDZ >= max(1,N).

ISUPPZ
  ISUPPZ is INTEGER array, dimension ( 2*max(1,M) )
  The support of the eigenvectors in Z, i.e., the indices
  indicating the nonzero elements in Z. The i-th eigenvector
  is nonzero only in elements ISUPPZ( 2*i-1 ) through ISUPPZ( 2*i ).
  Implemented only for RANGE = 'A' or 'I' and IU - IL = N - 1

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,26*N).
  For optimal efficiency, LWORK >= (NB+6)*N,
  where NB is the max of the blocksize for DSYTRD and DORMTR
  returned by ILAENV.
```

```
  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (MAX(1,LIWORK))
  On exit, if INFO = 0, IWORK(1) returns the optimal LWORK.

LIWORK
  LIWORK is INTEGER
  The dimension of the array IWORK.  LIWORK >= max(1,10*N).

  If LIWORK = -1, then a workspace query is assumed; the
  routine only calculates the optimal size of the IWORK array,
  returns this value as the first entry of the IWORK array, and
  no error message related to LIWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  Internal error
```

———-

56.3.3 fortran code

— **dsyevr.f** —

```
*  ===================================================================
      SUBROUTINE DSYEVR( JOBZ, RANGE, UPLO, N, A, LDA, VL, VU, IL, IU,
     $                   ABSTOL, M, W, Z, LDZ, ISUPPZ, WORK, LWORK,
     $                   IWORK, LIWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          JOBZ, RANGE, UPLO
      INTEGER            IL, INFO, IU, LDA, LDZ, LIWORK, LWORK, M, N
      DOUBLE PRECISION   ABSTOL, VL, VU
*     ..
*     .. Array Arguments ..
```

```
      INTEGER            ISUPPZ( * ), IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), W( * ), WORK( * ), Z( LDZ, * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE, TWO
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0, TWO = 2.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ALLEIG, INDEIG, LOWER, LQUERY, VALEIG, WANTZ,
     $                   TRYRAC
      CHARACTER          ORDER
      INTEGER            I, IEEEOK, IINFO, IMAX, INDD, INDDD, INDE,
     $                   INDEE, INDIBL, INDIFL, INDISP, INDIWO, INDTAU,
     $                   INDWK, INDWKN, ISCALE, J, JJ, LIWMIN,
     $                   LLWORK, LLWRKN, LWKOPT, LWMIN, NB, NSPLIT
      DOUBLE PRECISION   ABSTLL, ANRM, BIGNUM, EPS, RMAX, RMIN, SAFMIN,
     $                   SIGMA, SMLNUM, TMP1, VLL, VUU
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANSY
      EXTERNAL           LSAME, ILAENV, DLAMCH, DLANSY
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DORMTR, DSCAL, DSTEBZ, DSTEMR, DSTEIN,
     $                   DSTERF, DSWAP, DSYTRD, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      IEEEOK = ILAENV( 10, 'DSYEVR', 'N', 1, 2, 3, 4 )
*
      LOWER = LSAME( UPLO, 'L' )
      WANTZ = LSAME( JOBZ, 'V' )
      ALLEIG = LSAME( RANGE, 'A' )
      VALEIG = LSAME( RANGE, 'V' )
      INDEIG = LSAME( RANGE, 'I' )
*
      LQUERY = ( ( LWORK.EQ.-1 ) .OR. ( LIWORK.EQ.-1 ) )
*
      LWMIN = MAX( 1, 26*N )
      LIWMIN = MAX( 1, 10*N )
```

```
*
      INFO = 0
      IF( .NOT.( WANTZ .OR. LSAME( JOBZ, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( .NOT.( ALLEIG .OR. VALEIG .OR. INDEIG ) ) THEN
         INFO = -2
      ELSE IF( .NOT.( LOWER .OR. LSAME( UPLO, 'U' ) ) ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE
         IF( VALEIG ) THEN
            IF( N.GT.0 .AND. VU.LE.VL )
     $         INFO = -8
         ELSE IF( INDEIG ) THEN
            IF( IL.LT.1 .OR. IL.GT.MAX( 1, N ) ) THEN
               INFO = -9
            ELSE IF( IU.LT.MIN( N, IL ) .OR. IU.GT.N ) THEN
               INFO = -10
            END IF
         END IF
      END IF
      IF( INFO.EQ.0 ) THEN
         IF( LDZ.LT.1 .OR. ( WANTZ .AND. LDZ.LT.N ) ) THEN
            INFO = -15
         ELSE IF( LWORK.LT.LWMIN .AND. .NOT.LQUERY ) THEN
            INFO = -18
         ELSE IF( LIWORK.LT.LIWMIN .AND. .NOT.LQUERY ) THEN
            INFO = -20
         END IF
      END IF
*
      IF( INFO.EQ.0 ) THEN
         NB = ILAENV( 1, 'DSYTRD', UPLO, N, -1, -1, -1 )
         NB = MAX( NB, ILAENV( 1, 'DORMTR', UPLO, N, -1, -1, -1 ) )
         LWKOPT = MAX( ( NB+1 )*N, LWMIN )
         WORK( 1 ) = LWKOPT
         IWORK( 1 ) = LIWMIN
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYEVR', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
```

```
*
      M = 0
      IF( N.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      IF( N.EQ.1 ) THEN
         WORK( 1 ) = 7
         IF( ALLEIG .OR. INDEIG ) THEN
            M = 1
            W( 1 ) = A( 1, 1 )
         ELSE
            IF( VL.LT.A( 1, 1 ) .AND. VU.GE.A( 1, 1 ) ) THEN
               M = 1
               W( 1 ) = A( 1, 1 )
            END IF
         END IF
         IF( WANTZ ) THEN
            Z( 1, 1 ) = ONE
            ISUPPZ( 1 ) = 1
            ISUPPZ( 2 ) = 1
         END IF
         RETURN
      END IF
*
*     Get machine constants.
*
      SAFMIN = DLAMCH( 'Safe minimum' )
      EPS = DLAMCH( 'Precision' )
      SMLNUM = SAFMIN / EPS
      BIGNUM = ONE / SMLNUM
      RMIN = SQRT( SMLNUM )
      RMAX = MIN( SQRT( BIGNUM ), ONE / SQRT( SQRT( SAFMIN ) ) )
*
*     Scale matrix to allowable range, if necessary.
*
      ISCALE = 0
      ABSTLL = ABSTOL
      IF (VALEIG) THEN
         VLL = VL
         VUU = VU
      END IF
      ANRM = DLANSY( 'M', UPLO, N, A, LDA, WORK )
      IF( ANRM.GT.ZERO .AND. ANRM.LT.RMIN ) THEN
         ISCALE = 1
         SIGMA = RMIN / ANRM
      ELSE IF( ANRM.GT.RMAX ) THEN
         ISCALE = 1
         SIGMA = RMAX / ANRM
```

```
      END IF
      IF( ISCALE.EQ.1 ) THEN
         IF( LOWER ) THEN
            DO 10 J = 1, N
               CALL DSCAL( N-J+1, SIGMA, A( J, J ), 1 )
   10       CONTINUE
         ELSE
            DO 20 J = 1, N
               CALL DSCAL( J, SIGMA, A( 1, J ), 1 )
   20       CONTINUE
         END IF
         IF( ABSTOL.GT.0 )
     $      ABSTLL = ABSTOL*SIGMA
         IF( VALEIG ) THEN
            VLL = VL*SIGMA
            VUU = VU*SIGMA
         END IF
      END IF

*     Initialize indices into workspaces.  Note: The IWORK indices are
*     used only if DSTERF or DSTEMR fail.

*     WORK(INDTAU:INDTAU+N-1) stores the scalar factors of the
*     elementary reflectors used in DSYTRD.
      INDTAU = 1
*     WORK(INDD:INDD+N-1) stores the tridiagonal's diagonal entries.
      INDD = INDTAU + N
*     WORK(INDE:INDE+N-1) stores the off-diagonal entries of the
*     tridiagonal matrix from DSYTRD.
      INDE = INDD + N
*     WORK(INDDD:INDDD+N-1) is a copy of the diagonal entries over
*     -written by DSTEMR (the DSTERF path copies the diagonal to W).
      INDDD = INDE + N
*     WORK(INDEE:INDEE+N-1) is a copy of the off-diagonal entries over
*     -written while computing the eigenvalues in DSTERF and DSTEMR.
      INDEE = INDDD + N
*     INDWK is the starting offset of the left-over workspace, and
*     LLWORK is the remaining workspace size.
      INDWK = INDEE + N
      LLWORK = LWORK - INDWK + 1

*     IWORK(INDIBL:INDIBL+M-1) corresponds to IBLOCK in DSTEBZ and
*     stores the block indices of each of the M<=N eigenvalues.
      INDIBL = 1
*     IWORK(INDISP:INDISP+NSPLIT-1) corresponds to ISPLIT in DSTEBZ and
*     stores the starting and finishing indices of each block.
      INDISP = INDIBL + N
*     IWORK(INDIFL:INDIFL+N-1) stores the indices of eigenvectors
*     that corresponding to eigenvectors that fail to converge in
*     DSTEIN.  This information is discarded; if any fail, the driver
```

```
*     returns INFO > 0.
      INDIFL = INDISP + N
*     INDIWO is the offset of the remaining integer workspace.
      INDIWO = INDIFL + N

*
*     Call DSYTRD to reduce symmetric matrix to tridiagonal form.
*
      CALL DSYTRD( UPLO, N, A, LDA, WORK( INDD ), WORK( INDE ),
     $             WORK( INDTAU ), WORK( INDWK ), LLWORK, IINFO )
*
*     If all eigenvalues are desired
*     then call DSTERF or DSTEMR and DORMTR.
*
      IF( ( ALLEIG .OR. ( INDEIG .AND. IL.EQ.1 .AND. IU.EQ.N ) ) .AND.
     $    IEEEOK.EQ.1 ) THEN
         IF( .NOT.WANTZ ) THEN
            CALL DCOPY( N, WORK( INDD ), 1, W, 1 )
            CALL DCOPY( N-1, WORK( INDE ), 1, WORK( INDEE ), 1 )
            CALL DSTERF( N, W, WORK( INDEE ), INFO )
         ELSE
            CALL DCOPY( N-1, WORK( INDE ), 1, WORK( INDEE ), 1 )
            CALL DCOPY( N, WORK( INDD ), 1, WORK( INDDD ), 1 )
*
            IF (ABSTOL .LE. TWO*N*EPS) THEN
               TRYRAC = .TRUE.
            ELSE
               TRYRAC = .FALSE.
            END IF
            CALL DSTEMR( JOBZ, 'A', N, WORK( INDDD ), WORK( INDEE ),
     $                   VL, VU, IL, IU, M, W, Z, LDZ, N, ISUPPZ,
     $                   TRYRAC, WORK( INDWK ), LWORK, IWORK, LIWORK,
     $                   INFO )
*
*
*
*        Apply orthogonal matrix used in reduction to tridiagonal
*        form to eigenvectors returned by DSTEIN.
*
            IF( WANTZ .AND. INFO.EQ.0 ) THEN
               INDWKN = INDE
               LLWRKN = LWORK - INDWKN + 1
               CALL DORMTR( 'L', UPLO, 'N', N, M, A, LDA,
     $                      WORK( INDTAU ), Z, LDZ, WORK( INDWKN ),
     $                      LLWRKN, IINFO )
            END IF
         END IF
*
*
         IF( INFO.EQ.0 ) THEN
```

```
*           Everything worked.  Skip DSTEBZ/DSTEIN.  IWORK(:) are
*           undefined.
            M = N
            GO TO 30
         END IF
         INFO = 0
      END IF
*
*     Otherwise, call DSTEBZ and, if eigenvectors are desired, DSTEIN.
*     Also call DSTEBZ and DSTEIN if DSTEMR fails.
*
      IF( WANTZ ) THEN
         ORDER = 'B'
      ELSE
         ORDER = 'E'
      END IF

      CALL DSTEBZ( RANGE, ORDER, N, VLL, VUU, IL, IU, ABSTLL,
     $             WORK( INDD ), WORK( INDE ), M, NSPLIT, W,
     $             IWORK( INDIBL ), IWORK( INDISP ), WORK( INDWK ),
     $             IWORK( INDIWO ), INFO )
*
      IF( WANTZ ) THEN
         CALL DSTEIN( N, WORK( INDD ), WORK( INDE ), M, W,
     $                IWORK( INDIBL ), IWORK( INDISP ), Z, LDZ,
     $                WORK( INDWK ), IWORK( INDIWO ), IWORK( INDIFL ),
     $                INFO )
*
*        Apply orthogonal matrix used in reduction to tridiagonal
*        form to eigenvectors returned by DSTEIN.
*
         INDWKN = INDE
         LLWRKN = LWORK - INDWKN + 1
         CALL DORMTR( 'L', UPLO, 'N', N, M, A, LDA, WORK( INDTAU ), Z,
     $                LDZ, WORK( INDWKN ), LLWRKN, IINFO )
      END IF
*
*     If matrix was scaled, then rescale eigenvalues appropriately.
*
*  Jump here if DSTEMR/DSTEIN succeeded.
   30 CONTINUE
      IF( ISCALE.EQ.1 ) THEN
         IF( INFO.EQ.0 ) THEN
            IMAX = M
         ELSE
            IMAX = INFO - 1
         END IF
         CALL DSCAL( IMAX, ONE / SIGMA, W, 1 )
      END IF
*
```

```
*     If eigenvalues are not in order, then sort them, along with
*     eigenvectors.  Note: We do not sort the IFAIL portion of IWORK.
*     It may not be initialized (if DSTEMR/DSTEIN succeeded), and we do
*     not return this detailed information to the user.
*
      IF( WANTZ ) THEN
         DO 50 J = 1, M - 1
            I = 0
            TMP1 = W( J )
            DO 40 JJ = J + 1, M
               IF( W( JJ ).LT.TMP1 ) THEN
                  I = JJ
                  TMP1 = W( JJ )
               END IF
   40       CONTINUE
*
            IF( I.NE.0 ) THEN
               W( I ) = W( J )
               W( J ) = TMP1
               CALL DSWAP( N, Z( 1, I ), 1, Z( 1, J ), 1 )
            END IF
   50    CONTINUE
      END IF
*
*     Set WORK(1) to optimal workspace size.
*
      WORK( 1 ) = LWKOPT
      IWORK( 1 ) = LIWMIN
*
      RETURN
*
*     End of DSYEVR
*
      END
```

———-

56.3.4 lisp code

— dsyevr.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0) (two 2.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (ignorable zero one two))
  (defun dsyevr
         (jobz range uplo n a lda vl vu il iu abstol m w z ldz isuppz work
```

```
      lwork iwork liwork info)
(declare (type (array f2cl-lib:integer4 (*)) iwork isuppz)
         (type (double-float) abstol vu vl)
         (type (array double-float (*)) work z w a)
         (type (f2cl-lib:integer4) info liwork lwork ldz m iu il lda n)
         (type (string *) uplo range jobz))
(f2cl-lib:with-multi-array-data
    ((jobz character jobz-%data% jobz-%offset%)
     (range character range-%data% range-%offset%)
     (uplo character uplo-%data% uplo-%offset%)
     (a double-float a-%data% a-%offset%)
     (w double-float w-%data% w-%offset%)
     (z double-float z-%data% z-%offset%)
     (work double-float work-%data% work-%offset%)
     (isuppz f2cl-lib:integer4 isuppz-%data% isuppz-%offset%)
     (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((abstll 0.0d0) (anrm 0.0d0) (bignum 0.0d0) (eps 0.0d0)
         (rmax 0.0d0) (rmin 0.0d0) (safmin 0.0d0) (sigma 0.0d0)
         (smlnum 0.0d0) (tmp1 0.0d0) (vll 0.0d0) (vuu 0.0d0) (i 0)
         (ieeeok 0) (iinfo 0) (imax 0) (indd 0) (inddd 0) (inde 0)
         (indee 0) (indibl 0) (indifl 0) (indisp 0) (indiwo 0) (indtau 0)
         (indwk 0) (indwkn 0) (iscale 0) (j 0) (jj 0) (liwmin 0) (llwork 0)
         (llwrkn 0) (lwkopt 0) (lwmin 0) (nb 0) (nsplit 0)
         (order
          (make-array '(1) :element-type 'character :initial-element #\ ))
         (alleig nil) (indeig nil) (lower nil) (lquery nil) (valeig nil)
         (wantz nil) (tryrac nil))
    (declare
     (type (double-float) abstll anrm bignum eps rmax rmin safmin sigma
      smlnum tmp1 vll vuu)
     (type (f2cl-lib:integer4) i ieeeok iinfo imax indd inddd inde indee
      indibl indifl indisp indiwo indtau indwk indwkn iscale j jj liwmin
      llwork llwrkn lwkopt lwmin nb nsplit)
     (type (string 1) order)
     (type f2cl-lib:logical alleig indeig lower lquery valeig wantz
      tryrac))
    (setf ieeeok (ilaenv 10 "DSYEVR" "N" 1 2 3 4))
    (setf lower
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
    (setf wantz
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobz "V")
              (declare (ignore var-1))
              (when var-0 (setf jobz var-0))
              ret-val))
    (setf alleig
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "A")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf valeig
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "V")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf indeig
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "I")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf lquery (or (= lwork -1) (= liwork -1)))
(setf lwmin
        (max (the f2cl-lib:integer4 1)
             (the f2cl-lib:integer4 (f2cl-lib:int-mul 26 n))))
(setf liwmin
        (max (the f2cl-lib:integer4 1)
             (the f2cl-lib:integer4 (f2cl-lib:int-mul 10 n))))
(setf info 0)
(cond
 ((not
   (or wantz
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobz "N")
         (declare (ignore var-1))
         (when var-0 (setf jobz var-0))
         ret-val)))
  (setf info -1))
 ((not (or alleig valeig indeig)) (setf info -2))
 ((not
   (or lower
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame uplo "U")
         (declare (ignore var-1))
         (when var-0 (setf uplo var-0))
         ret-val)))
  (setf info -3))
 ((< n 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 (t
  (cond
   (valeig
    (if (and (> n 0) (<= vu vl))
```

```
        (setf info -8)))
   (indeig
    (cond
     ((or (< il 1)
          (> il
             (max (the f2cl-lib:integer4 1)
                  (the f2cl-lib:integer4 n))))
      (setf info -9))
     ((or
       (< iu
          (min (the f2cl-lib:integer4 n) (the f2cl-lib:integer4 il)))
       (> iu n))
      (setf info -10)))))))
(cond
 ((= info 0)
  (cond ((or (< ldz 1) (and wantz (< ldz n))) (setf info -15))
        ((and (< lwork lwmin) (not lquery)) (setf info -18))
        ((and (< liwork liwmin) (not lquery)) (setf info -20)))))
(cond
 ((= info 0)
  (setf nb
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (ilaenv 1 "DSYTRD" uplo n -1 -1 -1)
            (declare (ignore var-0 var-1 var-4 var-5 var-6))
            (when var-2 (setf uplo var-2))
            (when var-3 (setf n var-3))
            ret-val))
  (setf nb
          (max (the f2cl-lib:integer4 nb)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 1 "DORMTR" uplo n -1 -1 -1)
                      (declare (ignore var-0 var-1 var-4 var-5 var-6))
                      (when var-2 (setf uplo var-2))
                      (when var-3 (setf n var-3))
                      ret-val))))
  (setf lwkopt
          (max
           (the f2cl-lib:integer4
                (f2cl-lib:int-mul (f2cl-lib:int-add nb 1) n))
           (the f2cl-lib:integer4 lwmin)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
  (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%)
          liwmin)))
(cond
 ((/= info 0) (xerbla "DSYEVR" (f2cl-lib:int-sub info)) (go end_label))
```

```
 (lquery (go end_label)))
(setf m 0)
(cond
 ((= n 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'double-float))
  (go end_label)))
(cond
 ((= n 1)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 7) 'double-float))
  (cond
   ((or alleig indeig) (setf m 1)
    (setf (f2cl-lib:fref w-%data% (1) ((1 *)) w-%offset%)
            (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)))
   (t
    (cond
     ((and (< vl (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))
           (>= vu (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))))
      (setf m 1)
      (setf (f2cl-lib:fref w-%data% (1) ((1 *)) w-%offset%)
              (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *))
                             a-%offset%))))))
  (cond
   (wantz
    (setf (f2cl-lib:fref z-%data% (1 1) ((1 ldz) (1 *)) z-%offset%)
            one)
    (setf (f2cl-lib:fref isuppz-%data% (1) ((1 *)) isuppz-%offset%) 1)
    (setf (f2cl-lib:fref isuppz-%data% (2) ((1 *)) isuppz-%offset%)
            1)))
  (go end_label)))
(setf safmin (dlamch "Safe minimum"))
(setf eps (dlamch "Precision"))
(setf smlnum (/ safmin eps))
(setf bignum (/ one smlnum))
(setf rmin (f2cl-lib:fsqrt smlnum))
(setf rmax
        (min (f2cl-lib:fsqrt bignum)
             (/ one (f2cl-lib:fsqrt (f2cl-lib:fsqrt safmin)))))
(setf iscale 0)
(setf abstll abstol)
(cond (valeig (setf vll vl) (setf vuu vu)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlansy "M" uplo n a lda work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf uplo var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
```

```
          ret-val))
(cond
 ((and (> anrm zero) (< anrm rmin)) (setf iscale 1)
  (setf sigma (/ rmin anrm)))
 ((> anrm rmax) (setf iscale 1) (setf sigma (/ rmax anrm))))
(cond
 ((= iscale 1)
  (cond
   (lower
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1) sigma
             (f2cl-lib:array-slice a-%data% double-float (j j)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-0 var-2 var-3))
          (when var-1 (setf sigma var-1)))
       label10)))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal j sigma
             (f2cl-lib:array-slice a-%data% double-float (1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf j var-0))
          (when var-1 (setf sigma var-1)))
       label20))))
  (if (> abstol 0)
      (setf abstll (* abstol sigma)))
  (cond (valeig (setf vll (* vl sigma)) (setf vuu (* vu sigma))))))
(setf indtau 1)
(setf indd (f2cl-lib:int-add indtau n))
(setf inde (f2cl-lib:int-add indd n))
(setf inddd (f2cl-lib:int-add inde n))
(setf indee (f2cl-lib:int-add inddd n))
(setf indwk (f2cl-lib:int-add indee n))
(setf llwork (f2cl-lib:int-add (f2cl-lib:int-sub lwork indwk) 1))
(setf indibl 1)
(setf indisp (f2cl-lib:int-add indibl n))
(setf indifl (f2cl-lib:int-add indisp n))
(setf indiwo (f2cl-lib:int-add indifl n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dsytrd uplo n a lda
```

```
      (f2cl-lib:array-slice work-%data% double-float (indd) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (inde) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (indtau) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (indwk) ((1 *))
                            work-%offset%)
      llwork iinfo)
    (declare (ignore var-2 var-4 var-5 var-6 var-7))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf lda var-3))
    (when var-8 (setf llwork var-8))
    (when var-9 (setf iinfo var-9)))
  (cond
   ((and (or alleig (and indeig (= il 1) (= iu n))) (= ieeeok 1))
    (cond
     ((not wantz)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy n
           (f2cl-lib:array-slice work-%data% double-float (indd) ((1 *))
                                 work-%offset%)
           1 w 1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf n var-0)))
      (dcopy (f2cl-lib:int-sub n 1)
       (f2cl-lib:array-slice work-%data% double-float (inde) ((1 *))
                             work-%offset%)
       1
       (f2cl-lib:array-slice work-%data% double-float (indee) ((1 *))
                             work-%offset%)
       1)
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dsterf n w
           (f2cl-lib:array-slice work-%data% double-float (indee) ((1 *))
                                 work-%offset%)
           info)
        (declare (ignore var-1 var-2))
        (when var-0 (setf n var-0))
        (when var-3 (setf info var-3))))
     (t
      (dcopy (f2cl-lib:int-sub n 1)
       (f2cl-lib:array-slice work-%data% double-float (inde) ((1 *))
                             work-%offset%)
       1
       (f2cl-lib:array-slice work-%data% double-float (indee) ((1 *))
                             work-%offset%)
       1)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
    (dcopy n
     (f2cl-lib:array-slice work-%data% double-float (indd) ((1 *))
                           work-%offset%)
     1
     (f2cl-lib:array-slice work-%data% double-float (inddd) ((1 *))
                           work-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(cond ((<= abstol (* two n eps)) (setf tryrac f2cl-lib:%true%))
      (t (setf tryrac f2cl-lib:%false%)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18
     var-19 var-20)
    (dstemr jobz "A" n
     (f2cl-lib:array-slice work-%data% double-float (inddd) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (indee) ((1 *))
                           work-%offset%)
     vl vu il iu m w z ldz n isuppz tryrac
     (f2cl-lib:array-slice work-%data% double-float (indwk) ((1 *))
                           work-%offset%)
     lwork iwork liwork info)
  (declare
   (ignore var-1 var-3 var-4 var-10 var-11 var-14 var-16 var-18))
  (when var-0 (setf jobz var-0))
  (when var-2 (setf n var-2))
  (when var-5 (setf vl var-5))
  (when var-6 (setf vu var-6))
  (when var-7 (setf il var-7))
  (when var-8 (setf iu var-8))
  (when var-9 (setf m var-9))
  (when var-12 (setf ldz var-12))
  (when var-13 (setf n var-13))
  (when var-15 (setf tryrac var-15))
  (when var-17 (setf lwork var-17))
  (when var-19 (setf liwork var-19))
  (when var-20 (setf info var-20)))
(cond
 ((and wantz (= info 0)) (setf indwkn inde)
  (setf llwrkn
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork indwkn) 1))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dormtr "L" uplo "N" n m a lda
       (f2cl-lib:array-slice work-%data% double-float (indtau)
                             ((1 *)) work-%offset%)
       z ldz
```

```
            (f2cl-lib:array-slice work-%data% double-float (indwkn)
                                  ((1 *)) work-%offset%)
            llwrkn iinfo)
          (declare (ignore var-0 var-2 var-5 var-7 var-8 var-10))
          (when var-1 (setf uplo var-1))
          (when var-3 (setf n var-3))
          (when var-4 (setf m var-4))
          (when var-6 (setf lda var-6))
          (when var-9 (setf ldz var-9))
          (when var-11 (setf llwrkn var-11))
          (when var-12 (setf iinfo var-12)))))))
  (cond ((= info 0) (setf m n) (go label30))) (setf info 0)))
(cond (wantz (f2cl-lib:f2cl-set-string order "B" (string 1)))
      (t (f2cl-lib:f2cl-set-string order "E" (string 1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17)
    (dstebz range order n vll vuu il iu abstll
     (f2cl-lib:array-slice work-%data% double-float (indd) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (inde) ((1 *))
                           work-%offset%)
     m nsplit w
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indibl)
                           ((1 *)) iwork-%offset%)
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indisp)
                           ((1 *)) iwork-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (indwk) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indiwo)
                           ((1 *)) iwork-%offset%)
     info)
  (declare (ignore var-8 var-9 var-12 var-13 var-14 var-15 var-16))
  (when var-0 (setf range var-0))
  (when var-1 (setf order var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf vll var-3))
  (when var-4 (setf vuu var-4))
  (when var-5 (setf il var-5))
  (when var-6 (setf iu var-6))
  (when var-7 (setf abstll var-7))
  (when var-10 (setf m var-10))
  (when var-11 (setf nsplit var-11))
  (when var-17 (setf info var-17)))
(cond
 (wantz
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dstein n
```

```
            (f2cl-lib:array-slice work-%data% double-float (indd) ((1 *))
                                  work-%offset%)
            (f2cl-lib:array-slice work-%data% double-float (inde) ((1 *))
                                  work-%offset%)
            m w
            (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indibl)
                                  ((1 *)) iwork-%offset%)
            (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indisp)
                                  ((1 *)) iwork-%offset%)
            z ldz
            (f2cl-lib:array-slice work-%data% double-float (indwk) ((1 *))
                                  work-%offset%)
            (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indiwo)
                                  ((1 *)) iwork-%offset%)
            (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indifl)
                                  ((1 *)) iwork-%offset%)
            info)
          (declare
           (ignore var-1 var-2 var-4 var-5 var-6 var-7 var-9 var-10 var-11))
          (when var-0 (setf n var-0))
          (when var-3 (setf m var-3))
          (when var-8 (setf ldz var-8))
          (when var-12 (setf info var-12)))
        (setf indwkn inde)
        (setf llwrkn (f2cl-lib:int-add (f2cl-lib:int-sub lwork indwkn) 1))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11 var-12)
            (dormtr "L" uplo "N" n m a lda
             (f2cl-lib:array-slice work-%data% double-float (indtau) ((1 *))
                                   work-%offset%)
             z ldz
             (f2cl-lib:array-slice work-%data% double-float (indwkn) ((1 *))
                                   work-%offset%)
             llwrkn iinfo)
          (declare (ignore var-0 var-2 var-5 var-7 var-8 var-10))
          (when var-1 (setf uplo var-1))
          (when var-3 (setf n var-3))
          (when var-4 (setf m var-4))
          (when var-6 (setf lda var-6))
          (when var-9 (setf ldz var-9))
          (when var-11 (setf llwrkn var-11))
          (when var-12 (setf iinfo var-12)))))
    label30
     (cond
      ((= iscale 1)
       (cond ((= info 0) (setf imax m))
             (t (setf imax (f2cl-lib:int-sub info 1))))
       (multiple-value-bind (var-0 var-1 var-2 var-3)
           (dscal imax (/ one sigma) w 1)
```

```
            (declare (ignore var-1 var-2 var-3))
            (when var-0 (setf imax var-0)))))
        (cond
         (wantz
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
            (tagbody
              (setf i 0)
              (setf tmp1 (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%))
              (f2cl-lib:fdo (jj (f2cl-lib:int-add j 1) (f2cl-lib:int-add jj 1))
                            ((> jj m) nil)
                (tagbody
                  (cond
                   ((< (f2cl-lib:fref w (jj) ((1 *))) tmp1) (setf i jj)
                    (setf tmp1
                            (f2cl-lib:fref w-%data% (jj) ((1 *)) w-%offset%))))
                 label40))
              (cond
               ((/= i 0)
                (setf (f2cl-lib:fref w-%data% (i) ((1 *)) w-%offset%)
                        (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%))
                (setf (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%) tmp1)
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (dswap n
                     (f2cl-lib:array-slice z-%data% double-float (1 i)
                                           ((1 ldz) (1 *)) z-%offset%)
                     1
                     (f2cl-lib:array-slice z-%data% double-float (1 j)
                                           ((1 ldz) (1 *)) z-%offset%)
                     1)
                  (declare (ignore var-1 var-2 var-3 var-4))
                  (when var-0 (setf n var-0)))))
             label50))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
        (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) liwmin)
        (go end_label)
       end_label
        (return
         (values jobz range uplo n nil lda vl vu il iu nil m nil nil ldz nil
                 nil lwork nil liwork info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsyevr
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (fortran-to-lisp::integer4)
```

```
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (double-float) (double-float)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (double-float)
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::jobz
  fortran-to-lisp::range
  fortran-to-lisp::uplo
  fortran-to-lisp::n nil
  fortran-to-lisp::lda
  fortran-to-lisp::vl
  fortran-to-lisp::vu
  fortran-to-lisp::il
  fortran-to-lisp::iu nil
  fortran-to-lisp::m nil nil
  fortran-to-lisp::ldz nil nil
  fortran-to-lisp::lwork nil
  fortran-to-lisp::liwork
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

56.4 dsyevx

56.4.1 Axiom unit tests

— dsyevx.input —

```
)set break resume
)sys rm -f dsyevx.output
```

```
)spool dsyevx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

56.4.2 Axiom help page

— dsyevx.help —

```
=====================================================================
dsyevx examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsyevx (JOBZ, RANGE, UPLO, N, A, LDA, VL, VU, IL, IU,
           ABSTOL, M, W, Z, LDZ, WORK, LWORK, IWORK, IFAIL, INFO)
            DSYEVX computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for SY matrices

Function/Subroutine Documentation
   subroutine dsyevx (character JOBZ, character RANGE, character UPLO, integer
       N, double precision, dimension( lda, * ) A, integer LDA, double
       precision VL, double precision VU, integer IL, integer IU, double
       precision ABSTOL, integer M, double precision, dimension( * ) W, double
       precision, dimension( ldz, * ) Z, integer LDZ, double precision,
       dimension( * ) WORK, integer LWORK, integer, dimension( * ) IWORK,
       integer, dimension( * ) IFAIL, integer INFO)
        DSYEVX computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for SY matrices

Purpose:
   DSYEVX computes selected eigenvalues and, optionally, eigenvectors
   of a real symmetric matrix A.  Eigenvalues and eigenvectors can be
   selected by specifying either a range of values or a range of indices
   for the desired eigenvalues.

Parameters:
```

```
JOBZ
  JOBZ is CHARACTER*1
  = 'N':  Compute eigenvalues only;
  = 'V':  Compute eigenvalues and eigenvectors.

RANGE
  RANGE is CHARACTER*1
  = 'A': all eigenvalues will be found.
  = 'V': all eigenvalues in the half-open interval (VL,VU] will be found.
  = 'I': the IL-th through IU-th eigenvalues will be found.

UPLO
  UPLO is CHARACTER*1
  = 'U':  Upper triangle of A is stored;
  = 'L':  Lower triangle of A is stored.

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA, N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the
  leading N-by-N upper triangular part of A contains the
  upper triangular part of the matrix A.  If UPLO = 'L',
  the leading N-by-N lower triangular part of A contains
  the lower triangular part of the matrix A.
  On exit, the lower triangle (if UPLO='L') or the upper
  triangle (if UPLO='U') of A, including the diagonal, is destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

VL
  VL is DOUBLE PRECISION

VU
  VU is DOUBLE PRECISION
  If RANGE='V', the lower and upper bounds of the interval to
  be searched for eigenvalues. VL < VU.
  Not referenced if RANGE = 'A' or 'I'.

IL
  IL is INTEGER

IU
  IU is INTEGER
  If RANGE='I', the indices (in ascending order) of the
  smallest and largest eigenvalues to be returned.
```

```
  1 <= IL <= IU <= N, if N > 0; IL = 1 and IU = 0 if N = 0.
  Not referenced if RANGE = 'A' or 'V'.

ABSTOL
  ABSTOL is DOUBLE PRECISION
  The absolute error tolerance for the eigenvalues.
  An approximate eigenvalue is accepted as converged
  when it is determined to lie in an interval [a,b]
  of width less than or equal to

     ABSTOL + EPS *   max( |a|,|b| ) ,

  where EPS is the machine precision.  If ABSTOL is less than
  or equal to zero, then  EPS*|T|  will be used in its place,
  where |T| is the 1-norm of the tridiagonal matrix obtained
  by reducing A to tridiagonal form.

  Eigenvalues will be computed most accurately when ABSTOL is
  set to twice the underflow threshold 2*DLAMCH('S'), not zero.
  If this routine returns with INFO>0, indicating that some
  eigenvectors did not converge, try setting ABSTOL to
  2*DLAMCH('S').

M
  M is INTEGER
  The total number of eigenvalues found.  0 <= M <= N.
  If RANGE = 'A', M = N, and if RANGE = 'I', M = IU-IL+1.

W
  W is DOUBLE PRECISION array, dimension (N)
  On normal exit, the first M elements contain the selected
  eigenvalues in ascending order.

Z
  Z is DOUBLE PRECISION array, dimension (LDZ, max(1,M))
  If JOBZ = 'V', then if INFO = 0, the first M columns of Z
  contain the orthonormal eigenvectors of the matrix A
  corresponding to the selected eigenvalues, with the i-th
  column of Z holding the eigenvector associated with W(i).
  If an eigenvector fails to converge, then that column of Z
  contains the latest approximation to the eigenvector, and the
  index of the eigenvector is returned in IFAIL.
  If JOBZ = 'N', then Z is not referenced.
  Note: the user must ensure that at least max(1,M) columns are
  supplied in the array Z; if RANGE = 'V', the exact value of M
  is not known in advance and an upper bound must be used.

LDZ
  LDZ is INTEGER
  The leading dimension of the array Z.  LDZ >= 1, and if
```

```
  JOBZ = 'V', LDZ >= max(1,N).

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of the array WORK.  LWORK >= 1, when N <= 1;
  otherwise 8*N.
  For optimal efficiency, LWORK >= (NB+3)*N,
  where NB is the max of the blocksize for DSYTRD and DORMTR
  returned by ILAENV.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (5*N)

IFAIL
  IFAIL is INTEGER array, dimension (N)
  If JOBZ = 'V', then if INFO = 0, the first M elements of
  IFAIL are zero.  If INFO > 0, then IFAIL contains the
  indices of the eigenvectors that failed to converge.
  If JOBZ = 'N', then IFAIL is not referenced.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, then i eigenvectors failed to converge.
        Their indices are stored in array IFAIL.
```

———-

56.4.3 fortran code

— dsyevx.f —

```
*  =====================================================================
      SUBROUTINE DSYEVX( JOBZ, RANGE, UPLO, N, A, LDA, VL, VU, IL, IU,
     $                   ABSTOL, M, W, Z, LDZ, WORK, LWORK, IWORK,
     $                   IFAIL, INFO )
*
```

```
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOBZ, RANGE, UPLO
      INTEGER            IL, INFO, IU, LDA, LDZ, LWORK, M, N
      DOUBLE PRECISION   ABSTOL, VL, VU
*     ..
*     .. Array Arguments ..
      INTEGER            IFAIL( * ), IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), W( * ), WORK( * ), Z( LDZ, * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ALLEIG, INDEIG, LOWER, LQUERY, TEST, VALEIG,
     $                   WANTZ
      CHARACTER          ORDER
      INTEGER            I, IINFO, IMAX, INDD, INDE, INDEE, INDIBL,
     $                   INDISP, INDIWO, INDTAU, INDWKN, INDWRK, ISCALE,
     $                   ITMP1, J, JJ, LLWORK, LLWRKN, LWKMIN,
     $                   LWKOPT, NB, NSPLIT
      DOUBLE PRECISION   ABSTLL, ANRM, BIGNUM, EPS, RMAX, RMIN, SAFMIN,
     $                   SIGMA, SMLNUM, TMP1, VLL, VUU
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANSY
      EXTERNAL           LSAME, ILAENV, DLAMCH, DLANSY
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DLACPY, DORGTR, DORMTR, DSCAL, DSTEBZ,
     $                   DSTEIN, DSTEQR, DSTERF, DSWAP, DSYTRD, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      LOWER = LSAME( UPLO, 'L' )
```

```
      WANTZ = LSAME( JOBZ, 'V' )
      ALLEIG = LSAME( RANGE, 'A' )
      VALEIG = LSAME( RANGE, 'V' )
      INDEIG = LSAME( RANGE, 'I' )
      LQUERY = ( LWORK.EQ.-1 )
*
      INFO = 0
      IF( .NOT.( WANTZ .OR. LSAME( JOBZ, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( .NOT.( ALLEIG .OR. VALEIG .OR. INDEIG ) ) THEN
         INFO = -2
      ELSE IF( .NOT.( LOWER .OR. LSAME( UPLO, 'U' ) ) ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE
         IF( VALEIG ) THEN
            IF( N.GT.0 .AND. VU.LE.VL )
     $         INFO = -8
         ELSE IF( INDEIG ) THEN
            IF( IL.LT.1 .OR. IL.GT.MAX( 1, N ) ) THEN
               INFO = -9
            ELSE IF( IU.LT.MIN( N, IL ) .OR. IU.GT.N ) THEN
               INFO = -10
            END IF
         END IF
      END IF
      IF( INFO.EQ.0 ) THEN
         IF( LDZ.LT.1 .OR. ( WANTZ .AND. LDZ.LT.N ) ) THEN
            INFO = -15
         END IF
      END IF
*
      IF( INFO.EQ.0 ) THEN
         IF( N.LE.1 ) THEN
            LWKMIN = 1
            WORK( 1 ) = LWKMIN
         ELSE
            LWKMIN = 8*N
            NB = ILAENV( 1, 'DSYTRD', UPLO, N, -1, -1, -1 )
            NB = MAX( NB, ILAENV( 1, 'DORMTR', UPLO, N, -1, -1, -1 ) )
            LWKOPT = MAX( LWKMIN, ( NB + 3 )*N )
            WORK( 1 ) = LWKOPT
         END IF
*
         IF( LWORK.LT.LWKMIN .AND. .NOT.LQUERY )
     $      INFO = -17
      END IF
```

```
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYEVX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      M = 0
      IF( N.EQ.0 ) THEN
         RETURN
      END IF
*
      IF( N.EQ.1 ) THEN
         IF( ALLEIG .OR. INDEIG ) THEN
            M = 1
            W( 1 ) = A( 1, 1 )
         ELSE
            IF( VL.LT.A( 1, 1 ) .AND. VU.GE.A( 1, 1 ) ) THEN
               M = 1
               W( 1 ) = A( 1, 1 )
            END IF
         END IF
         IF( WANTZ )
     $      Z( 1, 1 ) = ONE
         RETURN
      END IF
*
*     Get machine constants.
*
      SAFMIN = DLAMCH( 'Safe minimum' )
      EPS = DLAMCH( 'Precision' )
      SMLNUM = SAFMIN / EPS
      BIGNUM = ONE / SMLNUM
      RMIN = SQRT( SMLNUM )
      RMAX = MIN( SQRT( BIGNUM ), ONE / SQRT( SQRT( SAFMIN ) ) )
*
*     Scale matrix to allowable range, if necessary.
*
      ISCALE = 0
      ABSTLL = ABSTOL
      IF( VALEIG ) THEN
         VLL = VL
         VUU = VU
      END IF
      ANRM = DLANSY( 'M', UPLO, N, A, LDA, WORK )
      IF( ANRM.GT.ZERO .AND. ANRM.LT.RMIN ) THEN
         ISCALE = 1
```

```
         SIGMA = RMIN / ANRM
      ELSE IF( ANRM.GT.RMAX ) THEN
         ISCALE = 1
         SIGMA = RMAX / ANRM
      END IF
      IF( ISCALE.EQ.1 ) THEN
         IF( LOWER ) THEN
            DO 10 J = 1, N
               CALL DSCAL( N-J+1, SIGMA, A( J, J ), 1 )
   10       CONTINUE
         ELSE
            DO 20 J = 1, N
               CALL DSCAL( J, SIGMA, A( 1, J ), 1 )
   20       CONTINUE
         END IF
         IF( ABSTOL.GT.0 )
     $      ABSTLL = ABSTOL*SIGMA
         IF( VALEIG ) THEN
            VLL = VL*SIGMA
            VUU = VU*SIGMA
         END IF
      END IF
*
*     Call DSYTRD to reduce symmetric matrix to tridiagonal form.
*
      INDTAU = 1
      INDE = INDTAU + N
      INDD = INDE + N
      INDWRK = INDD + N
      LLWORK = LWORK - INDWRK + 1
      CALL DSYTRD( UPLO, N, A, LDA, WORK( INDD ), WORK( INDE ),
     $             WORK( INDTAU ), WORK( INDWRK ), LLWORK, IINFO )
*
*     If all eigenvalues are desired and ABSTOL is less than or equal to
*     zero, then call DSTERF or DORGTR and SSTEQR.  If this fails for
*     some eigenvalue, then try DSTEBZ.
*
      TEST = .FALSE.
      IF( INDEIG ) THEN
         IF( IL.EQ.1 .AND. IU.EQ.N ) THEN
            TEST = .TRUE.
         END IF
      END IF
      IF( ( ALLEIG .OR. TEST ) .AND. ( ABSTOL.LE.ZERO ) ) THEN
         CALL DCOPY( N, WORK( INDD ), 1, W, 1 )
         INDEE = INDWRK + 2*N
         IF( .NOT.WANTZ ) THEN
            CALL DCOPY( N-1, WORK( INDE ), 1, WORK( INDEE ), 1 )
            CALL DSTERF( N, W, WORK( INDEE ), INFO )
         ELSE
```

```
            CALL DLACPY( 'A', N, N, A, LDA, Z, LDZ )
            CALL DORGTR( UPLO, N, Z, LDZ, WORK( INDTAU ),
     $                   WORK( INDWRK ), LLWORK, IINFO )
            CALL DCOPY( N-1, WORK( INDE ), 1, WORK( INDEE ), 1 )
            CALL DSTEQR( JOBZ, N, W, WORK( INDEE ), Z, LDZ,
     $                   WORK( INDWRK ), INFO )
            IF( INFO.EQ.0 ) THEN
               DO 30 I = 1, N
                  IFAIL( I ) = 0
   30          CONTINUE
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
            M = N
            GO TO 40
         END IF
         INFO = 0
      END IF
*
*     Otherwise, call DSTEBZ and, if eigenvectors are desired, SSTEIN.
*
      IF( WANTZ ) THEN
         ORDER = 'B'
      ELSE
         ORDER = 'E'
      END IF
      INDIBL = 1
      INDISP = INDIBL + N
      INDIWO = INDISP + N
      CALL DSTEBZ( RANGE, ORDER, N, VLL, VUU, IL, IU, ABSTLL,
     $             WORK( INDD ), WORK( INDE ), M, NSPLIT, W,
     $             IWORK( INDIBL ), IWORK( INDISP ), WORK( INDWRK ),
     $             IWORK( INDIWO ), INFO )
*
      IF( WANTZ ) THEN
         CALL DSTEIN( N, WORK( INDD ), WORK( INDE ), M, W,
     $                IWORK( INDIBL ), IWORK( INDISP ), Z, LDZ,
     $                WORK( INDWRK ), IWORK( INDIWO ), IFAIL, INFO )
*
*        Apply orthogonal matrix used in reduction to tridiagonal
*        form to eigenvectors returned by DSTEIN.
*
         INDWKN = INDE
         LLWRKN = LWORK - INDWKN + 1
         CALL DORMTR( 'L', UPLO, 'N', N, M, A, LDA, WORK( INDTAU ), Z,
     $                LDZ, WORK( INDWKN ), LLWRKN, IINFO )
      END IF
*
*     If matrix was scaled, then rescale eigenvalues appropriately.
*
```

```
   40 CONTINUE
      IF( ISCALE.EQ.1 ) THEN
         IF( INFO.EQ.0 ) THEN
            IMAX = M
         ELSE
            IMAX = INFO - 1
         END IF
         CALL DSCAL( IMAX, ONE / SIGMA, W, 1 )
      END IF
*
*     If eigenvalues are not in order, then sort them, along with
*     eigenvectors.
*
      IF( WANTZ ) THEN
         DO 60 J = 1, M - 1
            I = 0
            TMP1 = W( J )
            DO 50 JJ = J + 1, M
               IF( W( JJ ).LT.TMP1 ) THEN
                  I = JJ
                  TMP1 = W( JJ )
               END IF
   50       CONTINUE
*
            IF( I.NE.0 ) THEN
               ITMP1 = IWORK( INDIBL+I-1 )
               W( I ) = W( J )
               IWORK( INDIBL+I-1 ) = IWORK( INDIBL+J-1 )
               W( J ) = TMP1
               IWORK( INDIBL+J-1 ) = ITMP1
               CALL DSWAP( N, Z( 1, I ), 1, Z( 1, J ), 1 )
               IF( INFO.NE.0 ) THEN
                  ITMP1 = IFAIL( I )
                  IFAIL( I ) = IFAIL( J )
                  IFAIL( J ) = ITMP1
               END IF
            END IF
   60    CONTINUE
      END IF
*
*     Set WORK(1) to optimal workspace size.
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of DSYEVX
*
      END
```

———-

56.4.4 lisp code

— dsyevx.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dsyevx
         (jobz range uplo n a lda vl vu il iu abstol m w z ldz work lwork iwork
          ifail info)
    (declare (type (array f2cl-lib:integer4 (*)) ifail iwork)
             (type (double-float) abstol vu vl)
             (type (array double-float (*)) work z w a)
             (type (f2cl-lib:integer4) info lwork ldz m iu il lda n)
             (type (string *) uplo range jobz))
    (f2cl-lib:with-multi-array-data
        ((jobz character jobz-%data% jobz-%offset%)
         (range character range-%data% range-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (w double-float w-%data% w-%offset%)
         (z double-float z-%data% z-%offset%)
         (work double-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%)
         (ifail f2cl-lib:integer4 ifail-%data% ifail-%offset%))
      (prog ((abstll 0.0d0) (anrm 0.0d0) (bignum 0.0d0) (eps 0.0d0)
             (rmax 0.0d0) (rmin 0.0d0) (safmin 0.0d0) (sigma 0.0d0)
             (smlnum 0.0d0) (tmp1 0.0d0) (vll 0.0d0) (vuu 0.0d0) (i 0)
             (iinfo 0) (imax 0) (indd 0) (inde 0) (indee 0) (indibl 0)
             (indisp 0) (indiwo 0) (indtau 0) (indwkn 0) (indwrk 0) (iscale 0)
             (itmp1 0) (j 0) (jj 0) (llwork 0) (llwrkn 0) (lwkmin 0) (lwkopt 0)
             (nb 0) (nsplit 0)
             (order
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (alleig nil) (indeig nil) (lower nil) (lquery nil) (test nil)
             (valeig nil) (wantz nil))
        (declare
         (type (double-float) abstll anrm bignum eps rmax rmin safmin sigma
          smlnum tmp1 vll vuu)
         (type (f2cl-lib:integer4) i iinfo imax indd inde indee indibl indisp
          indiwo indtau indwkn indwrk iscale itmp1 j jj llwork llwrkn lwkmin
          lwkopt nb nsplit)
         (type (string 1) order)
         (type f2cl-lib:logical alleig indeig lower lquery test valeig wantz))
        (setf lower
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(setf wantz
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobz "V")
          (declare (ignore var-1))
          (when var-0 (setf jobz var-0))
          ret-val))
(setf alleig
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "A")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf valeig
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "V")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf indeig
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "I")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(setf info 0)
(cond
 ((not
   (or wantz
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobz "N")
         (declare (ignore var-1))
         (when var-0 (setf jobz var-0))
         ret-val)))
  (setf info -1))
 ((not (or alleig valeig indeig)) (setf info -2))
 ((not
   (or lower
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame uplo "U")
         (declare (ignore var-1))
         (when var-0 (setf uplo var-0))
         ret-val)))
  (setf info -3))
 ((< n 0) (setf info -4))
```

```
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 (t
  (cond
   (valeig
    (if (and (> n 0) (<= vu vl))
        (setf info -8)))
   (indeig
    (cond
     ((or (< il 1)
          (> il
             (max (the f2cl-lib:integer4 1)
                  (the f2cl-lib:integer4 n))))
      (setf info -9))
     ((or
       (< iu
          (min (the f2cl-lib:integer4 n) (the f2cl-lib:integer4 il)))
       (> iu n))
      (setf info -10)))))))
(cond
 ((= info 0)
  (cond ((or (< ldz 1) (and wantz (< ldz n))) (setf info -15)))))
(cond
 ((= info 0)
  (cond
   ((<= n 1) (setf lwkmin 1)
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce (the f2cl-lib:integer4 lwkmin) 'double-float)))
   (t (setf lwkmin (f2cl-lib:int-mul 8 n))
    (setf nb
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 1 "DSYTRD" uplo n -1 -1 -1)
              (declare (ignore var-0 var-1 var-4 var-5 var-6))
              (when var-2 (setf uplo var-2))
              (when var-3 (setf n var-3))
              ret-val))
    (setf nb
            (max (the f2cl-lib:integer4 nb)
                 (the f2cl-lib:integer4
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
                          (ilaenv 1 "DORMTR" uplo n -1 -1 -1)
                        (declare
                         (ignore var-0 var-1 var-4 var-5 var-6))
                        (when var-2 (setf uplo var-2))
                        (when var-3 (setf n var-3))
                        ret-val))))
    (setf lwkopt
```

```
            (max (the f2cl-lib:integer4 lwkmin)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-mul (f2cl-lib:int-add nb 3) n))))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))))
  (if (and (< lwork lwkmin) (not lquery))
      (setf info -17))))
(cond
 ((/= info 0) (xerbla "DSYEVX" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(setf m 0)
(cond ((= n 0) (go end_label)))
(cond
 ((= n 1)
  (cond
   ((or alleig indeig) (setf m 1)
    (setf (f2cl-lib:fref w-%data% (1) ((1 *)) w-%offset%)
            (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)))
   (t
    (cond
     ((and (< vl (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))
           (>= vu (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))))
      (setf m 1)
      (setf (f2cl-lib:fref w-%data% (1) ((1 *)) w-%offset%)
              (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *))
                             a-%offset%))))))
  (if wantz
      (setf (f2cl-lib:fref z-%data% (1 1) ((1 ldz) (1 *)) z-%offset%)
              one))
  (go end_label)))
(setf safmin (dlamch "Safe minimum"))
(setf eps (dlamch "Precision"))
(setf smlnum (/ safmin eps))
(setf bignum (/ one smlnum))
(setf rmin (f2cl-lib:fsqrt smlnum))
(setf rmax
        (min (f2cl-lib:fsqrt bignum)
             (/ one (f2cl-lib:fsqrt (f2cl-lib:fsqrt safmin)))))
(setf iscale 0)
(setf abstll abstol)
(cond (valeig (setf vll vl) (setf vuu vu)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlansy "M" uplo n a lda work)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf uplo var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
```

```
(cond
 ((and (> anrm zero) (< anrm rmin)) (setf iscale 1)
  (setf sigma (/ rmin anrm)))
 ((> anrm rmax) (setf iscale 1) (setf sigma (/ rmax anrm))))
(cond
 ((= iscale 1)
  (cond
   (lower
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1) sigma
             (f2cl-lib:array-slice a-%data% double-float (j j)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-0 var-2 var-3))
          (when var-1 (setf sigma var-1)))
       label10)))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal j sigma
             (f2cl-lib:array-slice a-%data% double-float (1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf j var-0))
          (when var-1 (setf sigma var-1)))
       label20))))
  (if (> abstol 0)
      (setf abstll (* abstol sigma)))
  (cond (valeig (setf vll (* vl sigma)) (setf vuu (* vu sigma))))))
(setf indtau 1)
(setf inde (f2cl-lib:int-add indtau n))
(setf indd (f2cl-lib:int-add inde n))
(setf indwrk (f2cl-lib:int-add indd n))
(setf llwork (f2cl-lib:int-add (f2cl-lib:int-sub lwork indwrk) 1))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dsytrd uplo n a lda
     (f2cl-lib:array-slice work-%data% double-float (indd) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (inde) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (indtau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (indwrk) ((1 *))
```

```
                            work-%offset%)
     llwork iinfo)
  (declare (ignore var-2 var-4 var-5 var-6 var-7))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-8 (setf llwork var-8))
  (when var-9 (setf iinfo var-9)))
(setf test f2cl-lib:%false%)
(cond
 (indeig (cond ((and (= il 1) (= iu n)) (setf test f2cl-lib:%true%)))))
(cond
 ((and (or alleig test) (<= abstol zero))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy n
       (f2cl-lib:array-slice work-%data% double-float (indd) ((1 *))
                              work-%offset%)
       1 w 1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf n var-0)))
  (setf indee (f2cl-lib:int-add indwrk (f2cl-lib:int-mul 2 n)))
  (cond
   ((not wantz)
    (dcopy (f2cl-lib:int-sub n 1)
     (f2cl-lib:array-slice work-%data% double-float (inde) ((1 *))
                            work-%offset%)
     1
     (f2cl-lib:array-slice work-%data% double-float (indee) ((1 *))
                            work-%offset%)
     1)
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dsterf n w
         (f2cl-lib:array-slice work-%data% double-float (indee) ((1 *))
                                work-%offset%)
         info)
      (declare (ignore var-1 var-2))
      (when var-0 (setf n var-0))
      (when var-3 (setf info var-3))))
   (t
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "A" n n a lda z ldz)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-6 (setf ldz var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (dorgtr uplo n z ldz
         (f2cl-lib:array-slice work-%data% double-float (indtau)
```

```
                                ((1 *)) work-%offset%)
            (f2cl-lib:array-slice work-%data% double-float (indwrk)
                                ((1 *)) work-%offset%)
            llwork iinfo)
        (declare (ignore var-2 var-4 var-5))
        (when var-0 (setf uplo var-0))
        (when var-1 (setf n var-1))
        (when var-3 (setf ldz var-3))
        (when var-6 (setf llwork var-6))
        (when var-7 (setf iinfo var-7)))
      (dcopy (f2cl-lib:int-sub n 1)
       (f2cl-lib:array-slice work-%data% double-float (inde) ((1 *))
                             work-%offset%)
       1
       (f2cl-lib:array-slice work-%data% double-float (indee) ((1 *))
                             work-%offset%)
       1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (dsteqr jobz n w
           (f2cl-lib:array-slice work-%data% double-float (indee) ((1 *))
                                 work-%offset%)
           z ldz
           (f2cl-lib:array-slice work-%data% double-float (indwrk)
                                 ((1 *)) work-%offset%)
           info)
        (declare (ignore var-2 var-3 var-4 var-6))
        (when var-0 (setf jobz var-0))
        (when var-1 (setf n var-1))
        (when var-5 (setf ldz var-5))
        (when var-7 (setf info var-7)))
      (cond
       ((= info 0)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref ifail-%data% (i) ((1 *)) ifail-%offset%)
                  0)
           label30))))))
  (cond ((= info 0) (setf m n) (go label40))) (setf info 0)))
(cond (wantz (f2cl-lib:f2cl-set-string order "B" (string 1)))
      (t (f2cl-lib:f2cl-set-string order "E" (string 1))))
(setf indibl 1)
(setf indisp (f2cl-lib:int-add indibl n))
(setf indiwo (f2cl-lib:int-add indisp n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17)
    (dstebz range order n vll vuu il iu abstll
     (f2cl-lib:array-slice work-%data% double-float (indd) ((1 *))
```

```
                          work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (inde) ((1 *))
                          work-%offset%)
      m nsplit w
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indibl)
                          ((1 *)) iwork-%offset%)
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indisp)
                          ((1 *)) iwork-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (indwrk) ((1 *))
                          work-%offset%)
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indiwo)
                          ((1 *)) iwork-%offset%)
      info)
    (declare (ignore var-8 var-9 var-12 var-13 var-14 var-15 var-16))
    (when var-0 (setf range var-0))
    (when var-1 (setf order var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf vll var-3))
    (when var-4 (setf vuu var-4))
    (when var-5 (setf il var-5))
    (when var-6 (setf iu var-6))
    (when var-7 (setf abstll var-7))
    (when var-10 (setf m var-10))
    (when var-11 (setf nsplit var-11))
    (when var-17 (setf info var-17)))
  (cond
   (wantz
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (dstein n
         (f2cl-lib:array-slice work-%data% double-float (indd) ((1 *))
                               work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (inde) ((1 *))
                               work-%offset%)
         m w
         (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indibl)
                               ((1 *)) iwork-%offset%)
         (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indisp)
                               ((1 *)) iwork-%offset%)
         z ldz
         (f2cl-lib:array-slice work-%data% double-float (indwrk) ((1 *))
                               work-%offset%)
         (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indiwo)
                               ((1 *)) iwork-%offset%)
         ifail info)
      (declare
       (ignore var-1 var-2 var-4 var-5 var-6 var-7 var-9 var-10 var-11))
      (when var-0 (setf n var-0))
      (when var-3 (setf m var-3))
```

```
      (when var-8 (setf ldz var-8))
      (when var-12 (setf info var-12)))
    (setf indwkn inde)
    (setf llwrkn (f2cl-lib:int-add (f2cl-lib:int-sub lwork indwkn) 1))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (dormtr "L" uplo "N" n m a lda
         (f2cl-lib:array-slice work-%data% double-float (indtau) ((1 *))
                               work-%offset%)
         z ldz
         (f2cl-lib:array-slice work-%data% double-float (indwkn) ((1 *))
                               work-%offset%)
         llwrkn iinfo)
      (declare (ignore var-0 var-2 var-5 var-7 var-8 var-10))
      (when var-1 (setf uplo var-1))
      (when var-3 (setf n var-3))
      (when var-4 (setf m var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldz var-9))
      (when var-11 (setf llwrkn var-11))
      (when var-12 (setf iinfo var-12)))))
label40
 (cond
  ((= iscale 1)
   (cond ((= info 0) (setf imax m))
         (t (setf imax (f2cl-lib:int-sub info 1))))
   (multiple-value-bind (var-0 var-1 var-2 var-3)
       (dscal imax (/ one sigma) w 1)
     (declare (ignore var-1 var-2 var-3))
     (when var-0 (setf imax var-0)))))
 (cond
  (wantz
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
     (tagbody
       (setf i 0)
       (setf tmp1 (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%))
       (f2cl-lib:fdo (jj (f2cl-lib:int-add j 1) (f2cl-lib:int-add jj 1))
                     ((> jj m) nil)
         (tagbody
           (cond
            ((< (f2cl-lib:fref w (jj) ((1 *))) tmp1) (setf i jj)
             (setf tmp1
                     (f2cl-lib:fref w-%data% (jj) ((1 *)) w-%offset%))))
          label50))
       (cond
        ((/= i 0)
         (setf itmp1
                 (f2cl-lib:fref iwork-%data%
```

```
                                  ((f2cl-lib:int-sub
                                    (f2cl-lib:int-add indibl i) 1))
                                  ((1 *)) iwork-%offset%))
                (setf (f2cl-lib:fref w-%data% (i) ((1 *)) w-%offset%)
                        (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%))
                (setf (f2cl-lib:fref iwork-%data%
                                    ((f2cl-lib:int-sub
                                      (f2cl-lib:int-add indibl i) 1))
                                    ((1 *)) iwork-%offset%)
                        (f2cl-lib:fref iwork-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add indibl j) 1))
                                      ((1 *)) iwork-%offset%))
                (setf (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%) tmp1)
                (setf (f2cl-lib:fref iwork-%data%
                                    ((f2cl-lib:int-sub
                                      (f2cl-lib:int-add indibl j) 1))
                                    ((1 *)) iwork-%offset%)
                        itmp1)
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (dswap n
                     (f2cl-lib:array-slice z-%data% double-float (1 i)
                                           ((1 ldz) (1 *)) z-%offset%)
                     1
                     (f2cl-lib:array-slice z-%data% double-float (1 j)
                                           ((1 ldz) (1 *)) z-%offset%)
                     1)
                  (declare (ignore var-1 var-2 var-3 var-4))
                  (when var-0 (setf n var-0)))
                (cond
                 ((/= info 0)
                  (setf itmp1
                          (f2cl-lib:fref ifail-%data% (i) ((1 *))
                                         ifail-%offset%))
                  (setf (f2cl-lib:fref ifail-%data% (i) ((1 *)) ifail-%offset%)
                          (f2cl-lib:fref ifail-%data% (j) ((1 *))
                                         ifail-%offset%))
                  (setf (f2cl-lib:fref ifail-%data% (j) ((1 *)) ifail-%offset%)
                          itmp1)))))
             label60))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
        (go end_label)
       end_label
        (return
         (values jobz range uplo n nil lda nil nil il iu nil m nil nil ldz nil
                 nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
```

```
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsyevx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (double-float)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::jobz
                                              fortran-to-lisp::range
                                              fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::il
                                              fortran-to-lisp::iu nil
                                              fortran-to-lisp::m nil nil
                                              fortran-to-lisp::ldz nil nil nil
                                              nil fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

56.5 dsygv

56.5.1 Axiom unit tests

— dsygv.input —

```
)set break resume
```

```
)sys rm -f dsygv.output
)spool dsygv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

56.5.2 Axiom help page

— dsygv.help —

```
=====================================================================
dsygv examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsygv (ITYPE, JOBZ, UPLO, N, A, LDA, B, LDB, W, WORK, LWORK,
           INFO)
           DSYGV

Function/Subroutine Documentation
   subroutine dsygv (integer ITYPE, character JOBZ, character UPLO, integer N,
       double precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldb, * ) B, integer LDB, double precision, dimension( * ) W,
       double precision, dimension( * ) WORK, integer LWORK, integer INFO)
       DSYGV

Purpose:
   DSYGV computes all the eigenvalues, and optionally, the eigenvectors
   of a real generalized symmetric-definite eigenproblem, of the form
   A*x=(lambda)*B*x,  A*Bx=(lambda)*x,  or B*A*x=(lambda)*x.
   Here A and B are assumed to be symmetric and B is also positive definite.

Parameters:

 ITYPE
   ITYPE is INTEGER
   Specifies the problem type to be solved:
   = 1:  A*x = (lambda)*B*x
```

```
  = 2:  A*B*x = (lambda)*x
  = 3:  B*A*x = (lambda)*x

JOBZ
  JOBZ is CHARACTER*1
  = 'N':  Compute eigenvalues only;
  = 'V':  Compute eigenvalues and eigenvectors.

UPLO
  UPLO is CHARACTER*1
  = 'U':  Upper triangles of A and B are stored;
  = 'L':  Lower triangles of A and B are stored.

N
  N is INTEGER
  The order of the matrices A and B.  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA, N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the
  leading N-by-N upper triangular part of A contains the
  upper triangular part of the matrix A.  If UPLO = 'L',
  the leading N-by-N lower triangular part of A contains
  the lower triangular part of the matrix A.

  On exit, if JOBZ = 'V', then if INFO = 0, A contains the
  matrix Z of eigenvectors.  The eigenvectors are normalized
  as follows:
  if ITYPE = 1 or 2, Z**T*B*Z = I;
  if ITYPE = 3, Z**T*inv(B)*Z = I.
  If JOBZ = 'N', then on exit the upper triangle (if UPLO='U')
  or the lower triangle (if UPLO='L') of A, including the
  diagonal, is destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

B
  B is DOUBLE PRECISION array, dimension (LDB, N)
  On entry, the symmetric positive definite matrix B.
  If UPLO = 'U', the leading N-by-N upper triangular part of B
  contains the upper triangular part of the matrix B.
  If UPLO = 'L', the leading N-by-N lower triangular part of B
  contains the lower triangular part of the matrix B.

  On exit, if INFO <= N, the part of B containing the matrix is
  overwritten by the triangular factor U or L from the Cholesky
  factorization B = U**T*U or B = L*L**T.
```

```
LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

W
  W is DOUBLE PRECISION array, dimension (N)
  If INFO = 0, the eigenvalues in ascending order.

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of the array WORK.  LWORK >= max(1,3*N-1).
  For optimal efficiency, LWORK >= (NB+2)*N,
  where NB is the blocksize for DSYTRD returned by ILAENV.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  DPOTRF or DSYEV returned an error code:
    <= N:  if INFO = i, DSYEV failed to converge;
           i off-diagonal elements of an intermediate
           tridiagonal form did not converge to zero;
    > N:   if INFO = N + i, for 1 <= i <= N, then the leading
           minor of order i of B is not positive definite.
           The factorization of B could not be completed and
           no eigenvalues or eigenvectors were computed.
```

———-

56.5.3 fortran code

— **dsygv.f** —

```
*  =====================================================================
      SUBROUTINE DSYGV( ITYPE, JOBZ, UPLO, N, A, LDA, B, LDB, W, WORK,
     $                  LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
```

```
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBZ, UPLO
      INTEGER            INFO, ITYPE, LDA, LDB, LWORK, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * ), W( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, UPPER, WANTZ
      CHARACTER          TRANS
      INTEGER            LWKMIN, LWKOPT, NB, NEIG
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DPOTRF, DSYEV, DSYGST, DTRMM, DTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      WANTZ = LSAME( JOBZ, 'V' )
      UPPER = LSAME( UPLO, 'U' )
      LQUERY = ( LWORK.EQ.-1 )
*
      INFO = 0
      IF( ITYPE.LT.1 .OR. ITYPE.GT.3 ) THEN
         INFO = -1
      ELSE IF( .NOT.( WANTZ .OR. LSAME( JOBZ, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( .NOT.( UPPER .OR. LSAME( UPLO, 'L' ) ) ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
```

```
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
*
      IF( INFO.EQ.0 ) THEN
         LWKMIN = MAX( 1, 3*N - 1 )
         NB = ILAENV( 1, 'DSYTRD', UPLO, N, -1, -1, -1 )
         LWKOPT = MAX( LWKMIN, ( NB + 2 )*N )
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.LWKMIN .AND. .NOT.LQUERY ) THEN
            INFO = -11
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYGV ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Form a Cholesky factorization of B.
*
      CALL DPOTRF( UPLO, N, B, LDB, INFO )
      IF( INFO.NE.0 ) THEN
         INFO = N + INFO
         RETURN
      END IF
*
*     Transform problem to standard eigenvalue problem and solve.
*
      CALL DSYGST( ITYPE, UPLO, N, A, LDA, B, LDB, INFO )
      CALL DSYEV( JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, INFO )
*
      IF( WANTZ ) THEN
*
*        Backtransform eigenvectors to the original problem.
*
         NEIG = N
         IF( INFO.GT.0 )
     $      NEIG = INFO - 1
```

```
         IF( ITYPE.EQ.1 .OR. ITYPE.EQ.2 ) THEN
*
*           For A*x=(lambda)*B*x and A*B*x=(lambda)*x;
*           backtransform eigenvectors: x = inv(L)**T*y or inv(U)*y
*
            IF( UPPER ) THEN
               TRANS = 'N'
            ELSE
               TRANS = 'T'
            END IF
*
            CALL DTRSM( 'Left', UPLO, TRANS, 'Non-unit', N, NEIG, ONE,
     $                  B, LDB, A, LDA )
*
         ELSE IF( ITYPE.EQ.3 ) THEN
*
*           For B*A*x=(lambda)*x;
*           backtransform eigenvectors: x = L*y or U**T*y
*
            IF( UPPER ) THEN
               TRANS = 'T'
            ELSE
               TRANS = 'N'
            END IF
*
            CALL DTRMM( 'Left', UPLO, TRANS, 'Non-unit', N, NEIG, ONE,
     $                  B, LDB, A, LDA )
         END IF
      END IF
*
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of DSYGV
*
      END
```

———-

56.5.4 lisp code

— dsygv.lisp —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dsygv (itype jobz uplo n a lda b ldb$ w work lwork info)
    (declare (type (array double-float (*)) work w b a)
```

```
          (type (string *) uplo jobz)
          (type (f2cl-lib:integer4) info lwork ldb$ lda n itype))
(f2cl-lib:with-multi-array-data
    ((jobz character jobz-%data% jobz-%offset%)
     (uplo character uplo-%data% uplo-%offset%)
     (a double-float a-%data% a-%offset%)
     (b double-float b-%data% b-%offset%)
     (w double-float w-%data% w-%offset%)
     (work double-float work-%data% work-%offset%))
  (prog ((lwkmin 0) (lwkopt 0) (nb 0) (neig 0)
         (trans
          (make-array '(1) :element-type 'character :initial-element #\ ))
         (lquery nil) (upper nil) (wantz nil))
    (declare (type (f2cl-lib:integer4) lwkmin lwkopt nb neig)
             (type (string 1) trans)
             (type f2cl-lib:logical lquery upper wantz))
    (setf wantz
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobz "V")
              (declare (ignore var-1))
              (when var-0 (setf jobz var-0))
              ret-val))
    (setf upper
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
    (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
    (setf info 0)
    (cond ((or (< itype 1) (> itype 3)) (setf info -1))
          ((not
            (or wantz
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame jobz "N")
                  (declare (ignore var-1))
                  (when var-0 (setf jobz var-0))
                  ret-val)))
           (setf info -2))
          ((not
            (or upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
           (setf info -3))
          ((< n 0) (setf info -4))
          ((< lda
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
```

```
        (setf info -6))
       ((< ldb$
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -8)))
(cond
 ((= info 0)
  (setf lwkmin
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-sub (f2cl-lib:int-mul 3 n) 1))))
  (setf nb
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (ilaenv 1 "DSYTRD" uplo n -1 -1 -1)
            (declare (ignore var-0 var-1 var-4 var-5 var-6))
            (when var-2 (setf uplo var-2))
            (when var-3 (setf n var-3))
            ret-val))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkmin)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-mul (f2cl-lib:int-add nb 2) n))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
  (cond ((and (< lwork lwkmin) (not lquery)) (setf info -11)))))
(cond
 ((/= info 0) (xerbla "DSYGV " (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dpotrf uplo n b ldb$ info)
  (declare (ignore var-2))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf ldb$ var-3))
  (when var-4 (setf info var-4)))
(cond
 ((/= info 0) (setf info (f2cl-lib:int-add n info)) (go end_label)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dsygst itype uplo n a lda b ldb$ info)
  (declare (ignore var-0 var-3 var-5))
  (setf uplo var-1)
  (setf n var-2)
  (setf lda var-4)
  (setf ldb$ var-6)
  (setf info var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dsyev jobz uplo n a lda w work lwork info)
```

```
          (declare (ignore var-3 var-5 var-6 var-7))
          (setf jobz var-0)
          (setf uplo var-1)
          (setf n var-2)
          (setf lda var-4)
          (setf info var-8))
        (cond
         (wantz (setf neig n)
          (if (> info 0)
              (setf neig (f2cl-lib:int-sub info 1)))
          (cond
           ((or (= itype 1) (= itype 2))
            (cond (upper (f2cl-lib:f2cl-set-string trans "N" (string 1)))
                  (t (f2cl-lib:f2cl-set-string trans "T" (string 1))))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                 var-10)
                (dtrsm "Left" uplo trans "Non-unit" n neig one b ldb$ a lda)
              (declare (ignore var-0 var-3 var-7 var-9))
              (when var-1 (setf uplo var-1))
              (when var-2 (setf trans var-2))
              (when var-4 (setf n var-4))
              (when var-5 (setf neig var-5))
              (when var-6 (setf one var-6))
              (when var-8 (setf ldb$ var-8))
              (when var-10 (setf lda var-10))))
           ((= itype 3)
            (cond (upper (f2cl-lib:f2cl-set-string trans "T" (string 1)))
                  (t (f2cl-lib:f2cl-set-string trans "N" (string 1))))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                 var-10)
                (dtrmm "Left" uplo trans "Non-unit" n neig one b ldb$ a lda)
              (declare (ignore var-0 var-3 var-7 var-9))
              (when var-1 (setf uplo var-1))
              (when var-2 (setf trans var-2))
              (when var-4 (setf n var-4))
              (when var-5 (setf neig var-5))
              (when var-6 (setf one var-6))
              (when var-8 (setf ldb$ var-8))
              (when var-10 (setf lda var-10)))))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
        (go end_label)
       end_label
        (return (values nil jobz uplo n nil lda nil ldb$ nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
```

```
(setf (gethash 'fortran-to-lisp::dsygv fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((fortran-to-lisp::integer4)
                                            (string) (string)
                                            (fortran-to-lisp::integer4)
                                            (array double-float (*))
                                            (fortran-to-lisp::integer4)
                                            (array double-float (*))
                                            (fortran-to-lisp::integer4)
                                            (array double-float (*))
                                            (array double-float (*))
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4))
                                          :return-values
                                          '(nil fortran-to-lisp::jobz
                                            fortran-to-lisp::uplo
                                            fortran-to-lisp::n nil
                                            fortran-to-lisp::lda nil
                                            fortran-to-lisp::ldb$ nil nil nil
                                            fortran-to-lisp::info)
                                          :calls
                                          '(fortran-to-lisp::dsyev
                                            fortran-to-lisp::dsygst))))
```

———-

56.6 dsygvd

56.6.1 Axiom unit tests

— **dsygvd.input** —

```
)set break resume
)sys rm -f dsygvd.output
)spool dsygvd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

56.6.2 Axiom help page

— dsygvd.help —

```
==================================================================
dsygvd examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsygvd (ITYPE, JOBZ, UPLO, N, A, LDA, B, LDB, W, WORK,
           LWORK, IWORK, LIWORK, INFO)
           DSYGVD

Function/Subroutine Documentation
   subroutine dsygvd (integer ITYPE, character JOBZ, character UPLO, integer
       N, double precision, dimension( lda, * ) A, integer LDA, double
       precision, dimension( ldb, * ) B, integer LDB, double precision,
       dimension( * ) W, double precision, dimension( * ) WORK, integer LWORK,
       integer, dimension( * ) IWORK, integer LIWORK, integer INFO)
       DSYGVD

Purpose:
   DSYGVD computes all the eigenvalues, and optionally, the eigenvectors
   of a real generalized symmetric-definite eigenproblem, of the form
   A*x=(lambda)*B*x,  A*Bx=(lambda)*x,  or B*A*x=(lambda)*x.  Here A and
   B are assumed to be symmetric and B is also positive definite.
   If eigenvectors are desired, it uses a divide and conquer algorithm.

   The divide and conquer algorithm makes very mild assumptions about
   floating point arithmetic. It will work on machines with a guard
   digit in add/subtract, or on those binary machines without guard
   digits which subtract like the Cray X-MP, Cray Y-MP, Cray C-90, or
   Cray-2. It could conceivably fail on hexadecimal or decimal machines
   without guard digits, but we know of none.

Parameters:

 ITYPE
   ITYPE is INTEGER
   Specifies the problem type to be solved:
   = 1:  A*x = (lambda)*B*x
   = 2:  A*B*x = (lambda)*x
   = 3:  B*A*x = (lambda)*x
```

```
JOBZ
  JOBZ is CHARACTER*1
  = 'N':  Compute eigenvalues only;
  = 'V':  Compute eigenvalues and eigenvectors.

UPLO
  UPLO is CHARACTER*1
  = 'U':  Upper triangles of A and B are stored;
  = 'L':  Lower triangles of A and B are stored.

N
  N is INTEGER
  The order of the matrices A and B.  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA, N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the
  leading N-by-N upper triangular part of A contains the
  upper triangular part of the matrix A.  If UPLO = 'L',
  the leading N-by-N lower triangular part of A contains
  the lower triangular part of the matrix A.

  On exit, if JOBZ = 'V', then if INFO = 0, A contains the
  matrix Z of eigenvectors.  The eigenvectors are normalized
  as follows:
  if ITYPE = 1 or 2, Z**T*B*Z = I;
  if ITYPE = 3, Z**T*inv(B)*Z = I.
  If JOBZ = 'N', then on exit the upper triangle (if UPLO='U')
  or the lower triangle (if UPLO='L') of A, including the
  diagonal, is destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

B
  B is DOUBLE PRECISION array, dimension (LDB, N)
  On entry, the symmetric matrix B.  If UPLO = 'U', the
  leading N-by-N upper triangular part of B contains the
  upper triangular part of the matrix B.  If UPLO = 'L',
  the leading N-by-N lower triangular part of B contains
  the lower triangular part of the matrix B.

  On exit, if INFO <= N, the part of B containing the matrix is
  overwritten by the triangular factor U or L from the Cholesky
  factorization B = U**T*U or B = L*L**T.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).
```

```
W
  W is DOUBLE PRECISION array, dimension (N)
  If INFO = 0, the eigenvalues in ascending order.

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  If N <= 1,               LWORK >= 1.
  If JOBZ = 'N' and N > 1, LWORK >= 2*N+1.
  If JOBZ = 'V' and N > 1, LWORK >= 1 + 6*N + 2*N**2.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal sizes of the WORK and IWORK
  arrays, returns these values as the first entries of the WORK
  and IWORK arrays, and no error message related to LWORK or
  LIWORK is issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (MAX(1,LIWORK))
  On exit, if INFO = 0, IWORK(1) returns the optimal LIWORK.

LIWORK
  LIWORK is INTEGER
  The dimension of the array IWORK.
  If N <= 1,                LIWORK >= 1.
  If JOBZ  = 'N' and N > 1, LIWORK >= 1.
  If JOBZ  = 'V' and N > 1, LIWORK >= 3 + 5*N.

  If LIWORK = -1, then a workspace query is assumed; the
  routine only calculates the optimal sizes of the WORK and
  IWORK arrays, returns these values as the first entries of
  the WORK and IWORK arrays, and no error message related to
  LWORK or LIWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  DPOTRF or DSYEVD returned an error code:
    <= N:  if INFO = i and JOBZ = 'N', then the algorithm
           failed to converge; i off-diagonal elements of an
           intermediate tridiagonal form did not converge to zero;
           if INFO = i and JOBZ = 'V', then the algorithm
           failed to compute an eigenvalue while working on
           the submatrix lying in rows and columns INFO/(N+1)
```

```
            through mod(INFO,N+1);
     > N:   if INFO = N + i, for 1 <= i <= N, then the leading
            minor of order i of B is not positive definite.
            The factorization of B could not be completed and
            no eigenvalues or eigenvectors were computed.

Further Details:

   Modified so that no backsubstitution is performed if DSYEVD fails to
   converge (NEIG in old code could be greater than N causing out of
   bounds reference to A - reported by Ralf Meyer).  Also corrected the
   description of INFO and the test on ITYPE. Sven, 16 Feb 05.
```

———-

56.6.3 fortran code

— **dsygvd.f** —

```
*  =====================================================================
      SUBROUTINE DSYGVD( ITYPE, JOBZ, UPLO, N, A, LDA, B, LDB, W, WORK,
     $                   LWORK, IWORK, LIWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBZ, UPLO
      INTEGER            INFO, ITYPE, LDA, LDB, LIWORK, LWORK, N
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * ), W( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, UPPER, WANTZ
      CHARACTER          TRANS
      INTEGER            LIOPT, LIWMIN, LOPT, LWMIN
```

```
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DPOTRF, DSYEVD, DSYGST, DTRMM, DTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      WANTZ = LSAME( JOBZ, 'V' )
      UPPER = LSAME( UPLO, 'U' )
      LQUERY = ( LWORK.EQ.-1 .OR. LIWORK.EQ.-1 )
*
      INFO = 0
      IF( N.LE.1 ) THEN
         LIWMIN = 1
         LWMIN = 1
      ELSE IF( WANTZ ) THEN
         LIWMIN = 3 + 5*N
         LWMIN = 1 + 6*N + 2*N**2
      ELSE
         LIWMIN = 1
         LWMIN = 2*N + 1
      END IF
      LOPT = LWMIN
      LIOPT = LIWMIN
      IF( ITYPE.LT.1 .OR. ITYPE.GT.3 ) THEN
         INFO = -1
      ELSE IF( .NOT.( WANTZ .OR. LSAME( JOBZ, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( .NOT.( UPPER .OR. LSAME( UPLO, 'L' ) ) ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
*
      IF( INFO.EQ.0 ) THEN
         WORK( 1 ) = LOPT
         IWORK( 1 ) = LIOPT
*
```

```
         IF( LWORK.LT.LWMIN .AND. .NOT.LQUERY ) THEN
            INFO = -11
         ELSE IF( LIWORK.LT.LIWMIN .AND. .NOT.LQUERY ) THEN
            INFO = -13
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYGVD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Form a Cholesky factorization of B.
*
      CALL DPOTRF( UPLO, N, B, LDB, INFO )
      IF( INFO.NE.0 ) THEN
         INFO = N + INFO
         RETURN
      END IF
*
*     Transform problem to standard eigenvalue problem and solve.
*
      CALL DSYGST( ITYPE, UPLO, N, A, LDA, B, LDB, INFO )
      CALL DSYEVD( JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, IWORK, LIWORK,
     $             INFO )
      LOPT = MAX( DBLE( LOPT ), DBLE( WORK( 1 ) ) )
      LIOPT = MAX( DBLE( LIOPT ), DBLE( IWORK( 1 ) ) )
*
      IF( WANTZ .AND. INFO.EQ.0 ) THEN
*
*        Backtransform eigenvectors to the original problem.
*
         IF( ITYPE.EQ.1 .OR. ITYPE.EQ.2 ) THEN
*
*           For A*x=(lambda)*B*x and A*B*x=(lambda)*x;
*           backtransform eigenvectors: x = inv(L)**T*y or inv(U)*y
*
            IF( UPPER ) THEN
               TRANS = 'N'
            ELSE
               TRANS = 'T'
            END IF
*
```

```
            CALL DTRSM( 'Left', UPLO, TRANS, 'Non-unit', N, N, ONE,
     $                  B, LDB, A, LDA )
*
         ELSE IF( ITYPE.EQ.3 ) THEN
*
*           For B*A*x=(lambda)*x;
*           backtransform eigenvectors: x = L*y or U**T*y
*
            IF( UPPER ) THEN
               TRANS = 'T'
            ELSE
               TRANS = 'N'
            END IF
*
            CALL DTRMM( 'Left', UPLO, TRANS, 'Non-unit', N, N, ONE,
     $                  B, LDB, A, LDA )
         END IF
      END IF
*
      WORK( 1 ) = LOPT
      IWORK( 1 ) = LIOPT
*
      RETURN
*
*     End of DSYGVD
*
      END
```

———-

56.6.4 lisp code

— dsygvd.lisp —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dsygvd (itype jobz uplo n a lda b ldb$ w work lwork iwork liwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (array double-float (*)) work w b a)
             (type (string *) uplo jobz)
             (type (f2cl-lib:integer4) info liwork lwork ldb$ lda n itype))
    (f2cl-lib:with-multi-array-data
        ((jobz character jobz-%data% jobz-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%)
         (w double-float w-%data% w-%offset%)
```

```
   (work double-float work-%data% work-%offset%)
   (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
(prog ((liopt 0) (liwmin 0) (lopt 0) (lwmin 0)
       (trans
        (make-array '(1) :element-type 'character :initial-element #\ ))
       (lquery nil) (upper nil) (wantz nil))
  (declare (type (f2cl-lib:integer4) liopt liwmin lopt lwmin)
           (type (string 1) trans)
           (type f2cl-lib:logical lquery upper wantz))
  (setf wantz
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame jobz "V")
            (declare (ignore var-1))
            (when var-0 (setf jobz var-0))
            ret-val))
  (setf upper
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
  (setf lquery (or (= lwork -1) (= liwork -1)))
  (setf info 0)
  (cond ((<= n 1) (setf liwmin 1) (setf lwmin 1))
        (wantz (setf liwmin (f2cl-lib:int-add 3 (f2cl-lib:int-mul 5 n)))
         (setf lwmin
                 (f2cl-lib:int-add 1 (f2cl-lib:int-mul 6 n)
                                   (f2cl-lib:int-mul 2 (expt n 2)))))
        (t (setf liwmin 1)
         (setf lwmin (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) 1))))
  (setf lopt lwmin)
  (setf liopt liwmin)
  (cond ((or (< itype 1) (> itype 3)) (setf info -1))
        ((not
          (or wantz
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame jobz "N")
                (declare (ignore var-1))
                (when var-0 (setf jobz var-0))
                ret-val)))
         (setf info -2))
        ((not
          (or upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val)))
         (setf info -3))
        ((< n 0) (setf info -4))
```

```
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -6))
      ((< ldb$
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -8)))
(cond
 ((= info 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lopt) 'double-float))
  (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) liopt)
  (cond ((and (< lwork lwmin) (not lquery)) (setf info -11))
        ((and (< liwork liwmin) (not lquery)) (setf info -13)))))
(cond
 ((/= info 0) (xerbla "DSYGVD" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dpotrf uplo n b ldb$ info)
  (declare (ignore var-2))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf ldb$ var-3))
  (when var-4 (setf info var-4)))
(cond
 ((/= info 0) (setf info (f2cl-lib:int-add n info)) (go end_label)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dsygst itype uplo n a lda b ldb$ info)
  (declare (ignore var-0 var-3 var-5))
  (setf uplo var-1)
  (setf n var-2)
  (setf lda var-4)
  (setf ldb$ var-6)
  (setf info var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dsyevd jobz uplo n a lda w work lwork iwork liwork info)
  (declare (ignore var-3 var-5 var-6 var-8))
  (when var-0 (setf jobz var-0))
  (when var-1 (setf uplo var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-7 (setf lwork var-7))
  (when var-9 (setf liwork var-9))
  (when var-10 (setf info var-10)))
(setf lopt
        (f2cl-lib:int
         (max (f2cl-lib:dble lopt)
```

```
                (f2cl-lib:dble
                 (f2cl-lib:fref work-%data% (1) ((1 *))
                                work-%offset%)))))
  (setf liopt
          (f2cl-lib:int
           (max (f2cl-lib:dble liopt)
                (f2cl-lib:dble
                 (f2cl-lib:fref iwork-%data% (1) ((1 *))
                                iwork-%offset%)))))
  (cond
   ((and wantz (= info 0))
    (cond
     ((or (= itype 1) (= itype 2))
      (cond (upper (f2cl-lib:f2cl-set-string trans "N" (string 1)))
            (t (f2cl-lib:f2cl-set-string trans "T" (string 1))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (dtrsm "Left" uplo trans "Non-unit" n n one b ldb$ a lda)
        (declare (ignore var-0 var-3 var-7 var-9))
        (when var-1 (setf uplo var-1))
        (when var-2 (setf trans var-2))
        (when var-4 (setf n var-4))
        (when var-5 (setf n var-5))
        (when var-6 (setf one var-6))
        (when var-8 (setf ldb$ var-8))
        (when var-10 (setf lda var-10))))
     ((= itype 3)
      (cond (upper (f2cl-lib:f2cl-set-string trans "T" (string 1)))
            (t (f2cl-lib:f2cl-set-string trans "N" (string 1))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (dtrmm "Left" uplo trans "Non-unit" n n one b ldb$ a lda)
        (declare (ignore var-0 var-3 var-7 var-9))
        (when var-1 (setf uplo var-1))
        (when var-2 (setf trans var-2))
        (when var-4 (setf n var-4))
        (when var-5 (setf n var-5))
        (when var-6 (setf one var-6))
        (when var-8 (setf ldb$ var-8))
        (when var-10 (setf lda var-10)))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lopt) 'double-float))
  (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) liopt)
  (go end_label)
 end_label
  (return
   (values nil jobz uplo n nil lda nil ldb$ nil nil lwork nil liwork
           info))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsygvd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil fortran-to-lisp::jobz
                                              fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$ nil nil
                                              fortran-to-lisp::lwork nil
                                              fortran-to-lisp::liwork
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::dsygst))))
```

———-

56.7 dsygvx

56.7.1 Axiom unit tests

— dsygvx.input —

```
)set break resume
)sys rm -f dsygvx.output
)spool dsygvx.output
)set message test on
)set message auto off
```

```
)clear all

)spool
)lisp (bye)
```

———-

56.7.2 Axiom help page

— **dsygvx.help** —

```
=====================================================================
dsygvx examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsygvx (ITYPE, JOBZ, RANGE, UPLO, N, A, LDA, B, LDB, VL, VU,
           IL, IU, ABSTOL, M, W, Z, LDZ, WORK, LWORK, IWORK, IFAIL, INFO)
           DSYGVX

Function/Subroutine Documentation
   subroutine dsygvx (integer ITYPE, character JOBZ, character RANGE,
       character UPLO, integer N, double precision, dimension( lda, * ) A,
       integer LDA, double precision, dimension( ldb, * ) B, integer LDB,
       double precision VL, double precision VU, integer IL, integer IU,
       double precision ABSTOL, integer M, double precision, dimension( * ) W,
       double precision, dimension( ldz, * ) Z, integer LDZ, double precision,
       dimension( * ) WORK, integer LWORK, integer, dimension( * ) IWORK,
       integer, dimension( * ) IFAIL, integer INFO)
       DSYGVX

Purpose:
   DSYGVX computes selected eigenvalues, and optionally, eigenvectors
   of a real generalized symmetric-definite eigenproblem, of the form
   A*x=(lambda)*B*x,  A*Bx=(lambda)*x,  or B*A*x=(lambda)*x.  Here A
   and B are assumed to be symmetric and B is also positive definite.
   Eigenvalues and eigenvectors can be selected by specifying either a
   range of values or a range of indices for the desired eigenvalues.

Parameters:

 ITYPE
   ITYPE is INTEGER
```

```
  Specifies the problem type to be solved:
  = 1:  A*x = (lambda)*B*x
  = 2:  A*B*x = (lambda)*x
  = 3:  B*A*x = (lambda)*x

JOBZ
  JOBZ is CHARACTER*1
  = 'N':  Compute eigenvalues only;
  = 'V':  Compute eigenvalues and eigenvectors.

RANGE
  RANGE is CHARACTER*1
  = 'A': all eigenvalues will be found.
  = 'V': all eigenvalues in the half-open interval (VL,VU] will be found.
  = 'I': the IL-th through IU-th eigenvalues will be found.

UPLO
  UPLO is CHARACTER*1
  = 'U':  Upper triangle of A and B are stored;
  = 'L':  Lower triangle of A and B are stored.

N
  N is INTEGER
  The order of the matrix pencil (A,B).  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA, N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the
  leading N-by-N upper triangular part of A contains the
  upper triangular part of the matrix A.  If UPLO = 'L',
  the leading N-by-N lower triangular part of A contains
  the lower triangular part of the matrix A.

  On exit, the lower triangle (if UPLO='L') or the upper
  triangle (if UPLO='U') of A, including the diagonal, is destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

B
  B is DOUBLE PRECISION array, dimension (LDB, N)
  On entry, the symmetric matrix B.  If UPLO = 'U', the
  leading N-by-N upper triangular part of B contains the
  upper triangular part of the matrix B.  If UPLO = 'L',
  the leading N-by-N lower triangular part of B contains
  the lower triangular part of the matrix B.

  On exit, if INFO <= N, the part of B containing the matrix is
  overwritten by the triangular factor U or L from the Cholesky
```

```
  factorization B = U**T*U or B = L*L**T.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

VL
  VL is DOUBLE PRECISION

VU
  VU is DOUBLE PRECISION
  If RANGE='V', the lower and upper bounds of the interval to
  be searched for eigenvalues. VL < VU.
  Not referenced if RANGE = 'A' or 'I'.

IL
  IL is INTEGER

IU
  IU is INTEGER
  If RANGE='I', the indices (in ascending order) of the
  smallest and largest eigenvalues to be returned.
  1 <= IL <= IU <= N, if N > 0; IL = 1 and IU = 0 if N = 0.
  Not referenced if RANGE = 'A' or 'V'.

ABSTOL
  ABSTOL is DOUBLE PRECISION
  The absolute error tolerance for the eigenvalues.
  An approximate eigenvalue is accepted as converged
  when it is determined to lie in an interval [a,b]
  of width less than or equal to

  ABSTOL + EPS *   max( |a|,|b| ) ,

  where EPS is the machine precision.  If ABSTOL is less than
  or equal to zero, then  EPS*|T|  will be used in its place,
  where |T| is the 1-norm of the tridiagonal matrix obtained
  by reducing C to tridiagonal form, where C is the symmetric
  matrix of the standard symmetric problem to which the
  generalized problem is transformed.

  Eigenvalues will be computed most accurately when ABSTOL is
  set to twice the underflow threshold 2*DLAMCH('S'), not zero.
  If this routine returns with INFO>0, indicating that some
  eigenvectors did not converge, try setting ABSTOL to 2*DLAMCH('S').

M
  M is INTEGER
  The total number of eigenvalues found.  0 <= M <= N.
  If RANGE = 'A', M = N, and if RANGE = 'I', M = IU-IL+1.
```

```
W
  W is DOUBLE PRECISION array, dimension (N)
  On normal exit, the first M elements contain the selected
  eigenvalues in ascending order.

Z
  Z is DOUBLE PRECISION array, dimension (LDZ, max(1,M))
  If JOBZ = 'N', then Z is not referenced.
  If JOBZ = 'V', then if INFO = 0, the first M columns of Z
  contain the orthonormal eigenvectors of the matrix A
  corresponding to the selected eigenvalues, with the i-th
  column of Z holding the eigenvector associated with W(i).
  The eigenvectors are normalized as follows:
  if ITYPE = 1 or 2, Z**T*B*Z = I;
  if ITYPE = 3, Z**T*inv(B)*Z = I.

  If an eigenvector fails to converge, then that column of Z
  contains the latest approximation to the eigenvector, and the
  index of the eigenvector is returned in IFAIL.
  Note: the user must ensure that at least max(1,M) columns are
  supplied in the array Z; if RANGE = 'V', the exact value of M
  is not known in advance and an upper bound must be used.

LDZ
  LDZ is INTEGER
  The leading dimension of the array Z.  LDZ >= 1, and if
  JOBZ = 'V', LDZ >= max(1,N).

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of the array WORK.  LWORK >= max(1,8*N).
  For optimal efficiency, LWORK >= (NB+3)*N,
  where NB is the blocksize for DSYTRD returned by ILAENV.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (5*N)

IFAIL
  IFAIL is INTEGER array, dimension (N)
  If JOBZ = 'V', then if INFO = 0, the first M elements of
```

```
  IFAIL are zero.  If INFO > 0, then IFAIL contains the
  indices of the eigenvectors that failed to converge.
  If JOBZ = 'N', then IFAIL is not referenced.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  DPOTRF or DSYEVX returned an error code:
  <= N:  if INFO = i, DSYEVX failed to converge;
         i eigenvectors failed to converge.  Their indices
         are stored in array IFAIL.
  > N:   if INFO = N + i, for 1 <= i <= N, then the leading
         minor of order i of B is not positive definite.
         The factorization of B could not be completed and
         no eigenvalues or eigenvectors were computed.
```

———-

56.7.3 fortran code

— dsygvx.f —

```
*  =====================================================================
      SUBROUTINE DSYGVX( ITYPE, JOBZ, RANGE, UPLO, N, A, LDA, B, LDB,
     $                   VL, VU, IL, IU, ABSTOL, M, W, Z, LDZ, WORK,
     $                   LWORK, IWORK, IFAIL, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBZ, RANGE, UPLO
      INTEGER            IL, INFO, ITYPE, IU, LDA, LDB, LDZ, LWORK, M, N
      DOUBLE PRECISION   ABSTOL, VL, VU
*     ..
*     .. Array Arguments ..
      INTEGER            IFAIL( * ), IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * ), W( * ), WORK( * ),
     $                   Z( LDZ, * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
```

```
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ALLEIG, INDEIG, LQUERY, UPPER, VALEIG, WANTZ
      CHARACTER          TRANS
      INTEGER            LWKMIN, LWKOPT, NB
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DPOTRF, DSYEVX, DSYGST, DTRMM, DTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      UPPER = LSAME( UPLO, 'U' )
      WANTZ = LSAME( JOBZ, 'V' )
      ALLEIG = LSAME( RANGE, 'A' )
      VALEIG = LSAME( RANGE, 'V' )
      INDEIG = LSAME( RANGE, 'I' )
      LQUERY = ( LWORK.EQ.-1 )
*
      INFO = 0
      IF( ITYPE.LT.1 .OR. ITYPE.GT.3 ) THEN
         INFO = -1
      ELSE IF( .NOT.( WANTZ .OR. LSAME( JOBZ, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( .NOT.( ALLEIG .OR. VALEIG .OR. INDEIG ) ) THEN
         INFO = -3
      ELSE IF( .NOT.( UPPER .OR. LSAME( UPLO, 'L' ) ) ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE
         IF( VALEIG ) THEN
            IF( N.GT.0 .AND. VU.LE.VL )
     $         INFO = -11
         ELSE IF( INDEIG ) THEN
```

```
            IF( IL.LT.1 .OR. IL.GT.MAX( 1, N ) ) THEN
               INFO = -12
            ELSE IF( IU.LT.MIN( N, IL ) .OR. IU.GT.N ) THEN
               INFO = -13
            END IF
         END IF
      END IF
      IF (INFO.EQ.0) THEN
         IF (LDZ.LT.1 .OR. (WANTZ .AND. LDZ.LT.N)) THEN
            INFO = -18
         END IF
      END IF
*
      IF( INFO.EQ.0 ) THEN
         LWKMIN = MAX( 1, 8*N )
         NB = ILAENV( 1, 'DSYTRD', UPLO, N, -1, -1, -1 )
         LWKOPT = MAX( LWKMIN, ( NB + 3 )*N )
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.LWKMIN .AND. .NOT.LQUERY ) THEN
            INFO = -20
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYGVX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      M = 0
      IF( N.EQ.0 ) THEN
         RETURN
      END IF
*
*     Form a Cholesky factorization of B.
*
      CALL DPOTRF( UPLO, N, B, LDB, INFO )
      IF( INFO.NE.0 ) THEN
         INFO = N + INFO
         RETURN
      END IF
*
*     Transform problem to standard eigenvalue problem and solve.
*
      CALL DSYGST( ITYPE, UPLO, N, A, LDA, B, LDB, INFO )
      CALL DSYEVX( JOBZ, RANGE, UPLO, N, A, LDA, VL, VU, IL, IU, ABSTOL,
```

```
     $             M, W, Z, LDZ, WORK, LWORK, IWORK, IFAIL, INFO )
*
      IF( WANTZ ) THEN
*
*        Backtransform eigenvectors to the original problem.
*
         IF( INFO.GT.0 )
     $      M = INFO - 1
         IF( ITYPE.EQ.1 .OR. ITYPE.EQ.2 ) THEN
*
*           For A*x=(lambda)*B*x and A*B*x=(lambda)*x;
*           backtransform eigenvectors: x = inv(L)**T*y or inv(U)*y
*
            IF( UPPER ) THEN
               TRANS = 'N'
            ELSE
               TRANS = 'T'
            END IF
*
            CALL DTRSM( 'Left', UPLO, TRANS, 'Non-unit', N, M, ONE, B,
     $                  LDB, Z, LDZ )
*
         ELSE IF( ITYPE.EQ.3 ) THEN
*
*           For B*A*x=(lambda)*x;
*           backtransform eigenvectors: x = L*y or U**T*y
*
            IF( UPPER ) THEN
               TRANS = 'T'
            ELSE
               TRANS = 'N'
            END IF
*
            CALL DTRMM( 'Left', UPLO, TRANS, 'Non-unit', N, M, ONE, B,
     $                  LDB, Z, LDZ )
         END IF
      END IF
*
*     Set WORK(1) to optimal workspace size.
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of DSYGVX
*
      END
```

———-

56.7.4 lisp code

— dsygvx.lisp —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dsygvx
         (itype jobz range uplo n a lda b ldb$ vl vu il iu abstol m w z ldz
          work lwork iwork ifail info)
    (declare (type (array f2cl-lib:integer4 (*)) ifail iwork)
             (type (double-float) abstol vu vl)
             (type (array double-float (*)) work z w b a)
             (type (string *) uplo range jobz)
             (type (f2cl-lib:integer4) info lwork ldz m iu il ldb$ lda n
              itype))
    (f2cl-lib:with-multi-array-data
        ((jobz character jobz-%data% jobz-%offset%)
         (range character range-%data% range-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%)
         (w double-float w-%data% w-%offset%)
         (z double-float z-%data% z-%offset%)
         (work double-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%)
         (ifail f2cl-lib:integer4 ifail-%data% ifail-%offset%))
      (prog ((lwkmin 0) (lwkopt 0) (nb 0)
             (trans
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (alleig nil) (indeig nil) (lquery nil) (upper nil) (valeig nil)
             (wantz nil))
        (declare (type (f2cl-lib:integer4) lwkmin lwkopt nb)
                 (type (string 1) trans)
                 (type f2cl-lib:logical alleig indeig lquery upper valeig
                  wantz))
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (setf wantz
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame jobz "V")
                  (declare (ignore var-1))
                  (when var-0 (setf jobz var-0))
                  ret-val))
        (setf alleig
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "A")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf valeig
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "V")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf indeig
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "I")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(setf info 0)
(cond ((or (< itype 1) (> itype 3)) (setf info -1))
      ((not
        (or wantz
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobz "N")
              (declare (ignore var-1))
              (when var-0 (setf jobz var-0))
              ret-val)))
       (setf info -2))
      ((not (or alleig valeig indeig)) (setf info -3))
      ((not
        (or upper
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
       (setf info -4))
      ((< n 0) (setf info -5))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -7))
      ((< ldb$
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -9))
      (t
       (cond
        (valeig
         (if (and (> n 0) (<= vu vl))
             (setf info -11)))
        (indeig
```

```
          (cond
           ((or (< il 1)
                (> il
                   (max (the f2cl-lib:integer4 1)
                        (the f2cl-lib:integer4 n))))
            (setf info -12))
           ((or
             (< iu
                (min (the f2cl-lib:integer4 n)
                     (the f2cl-lib:integer4 il)))
             (> iu n))
            (setf info -13)))))))
(cond
 ((= info 0)
  (cond ((or (< ldz 1) (and wantz (< ldz n))) (setf info -18)))))
(cond
 ((= info 0)
  (setf lwkmin
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul 8 n))))
  (setf nb
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (ilaenv 1 "DSYTRD" uplo n -1 -1 -1)
            (declare (ignore var-0 var-1 var-4 var-5 var-6))
            (when var-2 (setf uplo var-2))
            (when var-3 (setf n var-3))
            ret-val))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkmin)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-mul (f2cl-lib:int-add nb 3) n))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
  (cond ((and (< lwork lwkmin) (not lquery)) (setf info -20)))))
(cond
 ((/= info 0) (xerbla "DSYGVX" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(setf m 0)
(cond ((= n 0) (go end_label)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dpotrf uplo n b ldb$ info)
  (declare (ignore var-2))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf ldb$ var-3))
  (when var-4 (setf info var-4)))
(cond
 ((/= info 0) (setf info (f2cl-lib:int-add n info)) (go end_label)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
```

```
    (dsygst itype uplo n a lda b ldb$ info)
  (declare (ignore var-0 var-3 var-5))
  (setf uplo var-1)
  (setf n var-2)
  (setf lda var-4)
  (setf ldb$ var-6)
  (setf info var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19)
    (dsyevx jobz range uplo n a lda vl vu il iu abstol m w z ldz work
     lwork iwork ifail info)
  (declare
   (ignore var-4 var-6 var-7 var-10 var-12 var-13 var-15 var-16 var-17
    var-18))
  (setf jobz var-0)
  (setf range var-1)
  (setf uplo var-2)
  (setf n var-3)
  (setf lda var-5)
  (setf il var-8)
  (setf iu var-9)
  (setf m var-11)
  (setf ldz var-14)
  (setf info var-19))
(cond
 (wantz
  (if (> info 0)
      (setf m (f2cl-lib:int-sub info 1)))
  (cond
   ((or (= itype 1) (= itype 2))
    (cond (upper (f2cl-lib:f2cl-set-string trans "N" (string 1)))
          (t (f2cl-lib:f2cl-set-string trans "T" (string 1))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (dtrsm "Left" uplo trans "Non-unit" n m one b ldb$ z ldz)
      (declare (ignore var-0 var-3 var-7 var-9))
      (when var-1 (setf uplo var-1))
      (when var-2 (setf trans var-2))
      (when var-4 (setf n var-4))
      (when var-5 (setf m var-5))
      (when var-6 (setf one var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-10 (setf ldz var-10))))
   ((= itype 3)
    (cond (upper (f2cl-lib:f2cl-set-string trans "T" (string 1)))
          (t (f2cl-lib:f2cl-set-string trans "N" (string 1))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
                var-10)
               (dtrmm "Left" uplo trans "Non-unit" n m one b ldb$ z ldz)
             (declare (ignore var-0 var-3 var-7 var-9))
             (when var-1 (setf uplo var-1))
             (when var-2 (setf trans var-2))
             (when var-4 (setf n var-4))
             (when var-5 (setf m var-5))
             (when var-6 (setf one var-6))
             (when var-8 (setf ldb$ var-8))
             (when var-10 (setf ldz var-10)))))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
        (go end_label)
       end_label
        (return
         (values nil jobz range uplo n nil lda nil ldb$ nil nil il iu nil m nil
                 nil ldz nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsygvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (double-float)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil fortran-to-lisp::jobz
                                              fortran-to-lisp::range
                                              fortran-to-lisp::uplo
```

```
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$ nil nil
  fortran-to-lisp::il
  fortran-to-lisp::iu nil
  fortran-to-lisp::m nil nil
  fortran-to-lisp::ldz nil nil nil
  nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::dsyevx
  fortran-to-lisp::dsygst))))
```

Chapter 57

LAPACK - Symmetric Matrix, Eigenvalue, Real

57.1 ssyev

57.1.1 Axiom unit tests

— ssyev.input —

```
)set break resume
)sys rm -f ssyev.output
)spool ssyev.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

57.1.2 Axiom help page

— ssyev.help —

```
==========================================================================
ssyev examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssyev (JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, INFO)
            SSYEV computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for SY matrices

Function/Subroutine Documentation
   subroutine ssyev (character JOBZ, character UPLO, integer N, real,
       dimension( lda, * ) A, integer LDA, real, dimension( * ) W, real,
       dimension( * ) WORK, integer LWORK, integer INFO)
        SSYEV computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for SY matrices

Purpose:
   SSYEV computes all eigenvalues and, optionally, eigenvectors of a
   real symmetric matrix A.

Parameters:

 JOBZ
   JOBZ is CHARACTER*1
   = 'N':  Compute eigenvalues only;
   = 'V':  Compute eigenvalues and eigenvectors.

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA, N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the
   leading N-by-N upper triangular part of A contains the
   upper triangular part of the matrix A.  If UPLO = 'L',
   the leading N-by-N lower triangular part of A contains
   the lower triangular part of the matrix A.
   On exit, if JOBZ = 'V', then if INFO = 0, A contains the
   orthonormal eigenvectors of the matrix A.
   If JOBZ = 'N', then on exit the lower triangle (if UPLO='L')
   or the upper triangle (if UPLO='U') of A, including the
   diagonal, is destroyed.
```

```
 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 W
   W is REAL array, dimension (N)
   If INFO = 0, the eigenvalues in ascending order.

 WORK
   WORK is REAL array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The length of the array WORK.  LWORK >= max(1,3*N-1).
   For optimal efficiency, LWORK >= (NB+2)*N,
   where NB is the blocksize for SSYTRD returned by ILAENV.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, the algorithm failed to converge; i
         off-diagonal elements of an intermediate tridiagonal
         form did not converge to zero.
```

———-

57.1.3 fortran code

— **ssyev.f** —

```
*  =====================================================================
      SUBROUTINE SSYEV( JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
```

```
      CHARACTER          JOBZ, UPLO
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), W( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LOWER, LQUERY, WANTZ
      INTEGER            IINFO, IMAX, INDE, INDTAU, INDWRK, ISCALE,
     $                   LLWORK, LWKOPT, NB
      REAL               ANRM, BIGNUM, EPS, RMAX, RMIN, SAFMIN, SIGMA,
     $                   SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               SLAMCH, SLANSY
      EXTERNAL           ILAENV, LSAME, SLAMCH, SLANSY
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLASCL, SORGTR, SSCAL, SSTEQR, SSTERF, SSYTRD,
     $                   XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      WANTZ = LSAME( JOBZ, 'V' )
      LOWER = LSAME( UPLO, 'L' )
      LQUERY = ( LWORK.EQ.-1 )
*
      INFO = 0
      IF( .NOT.( WANTZ .OR. LSAME( JOBZ, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( .NOT.( LOWER .OR. LSAME( UPLO, 'U' ) ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
```

```
      END IF
*
      IF( INFO.EQ.0 ) THEN
         NB = ILAENV( 1, 'SSYTRD', UPLO, N, -1, -1, -1 )
         LWKOPT = MAX( 1, ( NB+2 )*N )
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.MAX( 1, 3*N-1 ) .AND. .NOT.LQUERY )
     $      INFO = -8
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYEV ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         RETURN
      END IF
*
      IF( N.EQ.1 ) THEN
         W( 1 ) = A( 1, 1 )
         WORK( 1 ) = 2
         IF( WANTZ )
     $      A( 1, 1 ) = ONE
         RETURN
      END IF
*
*     Get machine constants.
*
      SAFMIN = SLAMCH( 'Safe minimum' )
      EPS = SLAMCH( 'Precision' )
      SMLNUM = SAFMIN / EPS
      BIGNUM = ONE / SMLNUM
      RMIN = SQRT( SMLNUM )
      RMAX = SQRT( BIGNUM )
*
*     Scale matrix to allowable range, if necessary.
*
      ANRM = SLANSY( 'M', UPLO, N, A, LDA, WORK )
      ISCALE = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.RMIN ) THEN
         ISCALE = 1
         SIGMA = RMIN / ANRM
      ELSE IF( ANRM.GT.RMAX ) THEN
         ISCALE = 1
```

```
         SIGMA = RMAX / ANRM
      END IF
      IF( ISCALE.EQ.1 )
     $   CALL SLASCL( UPLO, 0, 0, ONE, SIGMA, N, N, A, LDA, INFO )
*
*     Call SSYTRD to reduce symmetric matrix to tridiagonal form.
*
      INDE = 1
      INDTAU = INDE + N
      INDWRK = INDTAU + N
      LLWORK = LWORK - INDWRK + 1
      CALL SSYTRD( UPLO, N, A, LDA, W, WORK( INDE ), WORK( INDTAU ),
     $             WORK( INDWRK ), LLWORK, IINFO )
*
*     For eigenvalues only, call SSTERF.  For eigenvectors, first call
*     SORGTR to generate the orthogonal matrix, then call SSTEQR.
*
      IF( .NOT.WANTZ ) THEN
         CALL SSTERF( N, W, WORK( INDE ), INFO )
      ELSE
         CALL SORGTR( UPLO, N, A, LDA, WORK( INDTAU ), WORK( INDWRK ),
     $                LLWORK, IINFO )
         CALL SSTEQR( JOBZ, N, W, WORK( INDE ), A, LDA, WORK( INDTAU ),
     $                INFO )
      END IF
*
*     If matrix was scaled, then rescale eigenvalues appropriately.
*
      IF( ISCALE.EQ.1 ) THEN
         IF( INFO.EQ.0 ) THEN
            IMAX = N
         ELSE
            IMAX = INFO - 1
         END IF
         CALL SSCAL( IMAX, ONE / SIGMA, W, 1 )
      END IF
*
*     Set WORK(1) to optimal workspace size.
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of SSYEV
*
      END
```

———-

57.1.4 lisp code

— ssyev.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun ssyev (jobz uplo n a lda w work lwork info)
    (declare (type (array single-float (*)) work w a)
             (type (f2cl-lib:integer4) info lwork lda n)
             (type (string *) uplo jobz))
    (f2cl-lib:with-multi-array-data
        ((jobz character jobz-%data% jobz-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (w single-float w-%data% w-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((anrm 0.0) (bignum 0.0) (eps 0.0) (rmax 0.0) (rmin 0.0)
             (safmin 0.0) (sigma 0.0) (smlnum 0.0) (iinfo 0) (imax 0) (inde 0)
             (indtau 0) (indwrk 0) (iscale 0) (llwork 0) (lwkopt 0) (nb 0)
             (lower nil) (lquery nil) (wantz nil))
        (declare
         (type (single-float) anrm bignum eps rmax rmin safmin sigma smlnum)
         (type (f2cl-lib:integer4) iinfo imax inde indtau indwrk iscale llwork
          lwkopt nb)
         (type f2cl-lib:logical lower lquery wantz))
        (setf wantz
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame jobz "V")
                  (declare (ignore var-1))
                  (when var-0 (setf jobz var-0))
                  ret-val))
        (setf lower
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (setf info 0)
        (cond
         ((not
           (or wantz
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame jobz "N")
                 (declare (ignore var-1))
                 (when var-0 (setf jobz var-0))
                 ret-val)))
```

```
  (setf info -1))
 ((not
   (or lower
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame uplo "U")
         (declare (ignore var-1))
         (when var-0 (setf uplo var-0))
         ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5)))
(cond
 ((= info 0)
  (setf nb
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (ilaenv 1 "SSYTRD" uplo n -1 -1 -1)
            (declare (ignore var-0 var-1 var-4 var-5 var-6))
            (setf uplo var-2)
            (setf n var-3)
            ret-val))
  (setf lwkopt
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-mul (f2cl-lib:int-add nb 2) n))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
  (if (and
       (< lwork
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-sub (f2cl-lib:int-mul 3 n) 1))))
       (not lquery))
      (setf info -8))))
(cond
 ((/= info 0) (xerbla "SSYEV " (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond ((= n 0) (go end_label)))
(cond
 ((= n 1)
  (setf (f2cl-lib:fref w-%data% (1) ((1 *)) w-%offset%)
          (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 2) 'single-float))
  (if wantz
      (setf (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)
            one))
  (go end_label)))
(setf safmin (slamch "Safe minimum"))
```

```
(setf eps (slamch "Precision"))
(setf smlnum (/ safmin eps))
(setf bignum (/ one smlnum))
(setf rmin (f2cl-lib:fsqrt smlnum))
(setf rmax (f2cl-lib:fsqrt bignum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slansy "M" uplo n a lda work)
          (declare (ignore var-0 var-3 var-4 var-5))
          (setf uplo var-1)
          (setf n var-2)
          ret-val))
(setf iscale 0)
(cond
 ((and (> anrm zero) (< anrm rmin)) (setf iscale 1)
  (setf sigma (/ rmin anrm)))
 ((> anrm rmax) (setf iscale 1) (setf sigma (/ rmax anrm))))
(if (= iscale 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl uplo 0 0 one sigma n n a lda info)
      (declare
       (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8))
      (setf uplo var-0)
      (setf info var-9)))
(setf inde 1)
(setf indtau (f2cl-lib:int-add inde n))
(setf indwrk (f2cl-lib:int-add indtau n))
(setf llwork (f2cl-lib:int-add (f2cl-lib:int-sub lwork indwrk) 1))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (ssytrd uplo n a lda w
     (f2cl-lib:array-slice work-%data% single-float (inde) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (indtau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (indwrk) ((1 *))
                           work-%offset%)
     llwork iinfo)
  (declare (ignore var-2 var-4 var-5 var-6 var-7))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-8 (setf llwork var-8))
  (when var-9 (setf iinfo var-9)))
(cond
 ((not wantz)
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (ssterf n w
```

```
              (f2cl-lib:array-slice work-%data% single-float (inde) ((1 *))
                                    work-%offset%)
              info)
            (declare (ignore var-0 var-1 var-2))
            (setf info var-3)))
          (t
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
               (sorgtr uplo n a lda
                (f2cl-lib:array-slice work-%data% single-float (indtau) ((1 *))
                                      work-%offset%)
                (f2cl-lib:array-slice work-%data% single-float (indwrk) ((1 *))
                                      work-%offset%)
                llwork iinfo)
            (declare (ignore var-1 var-2 var-4 var-5 var-6))
            (setf uplo var-0)
            (setf lda var-3)
            (setf iinfo var-7))
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
               (ssteqr jobz n w
                (f2cl-lib:array-slice work-%data% single-float (inde) ((1 *))
                                      work-%offset%)
                a lda
                (f2cl-lib:array-slice work-%data% single-float (indtau) ((1 *))
                                      work-%offset%)
                info)
            (declare (ignore var-2 var-3 var-4 var-5 var-6))
            (setf jobz var-0)
            (setf n var-1)
            (setf info var-7))))
         (cond
          ((= iscale 1)
           (cond ((= info 0) (setf imax n))
                 (t (setf imax (f2cl-lib:int-sub info 1))))
           (multiple-value-bind (var-0 var-1 var-2 var-3)
               (sscal imax (/ one sigma) w 1)
            (declare (ignore var-1 var-2 var-3))
            (when var-0 (setf imax var-0)))))
         (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                 (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
         (go end_label)
        end_label
         (return (values jobz uplo n nil lda nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssyev fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
```

```
'((string) (string)
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::jobz
  fortran-to-lisp::uplo
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::ssteqr
  fortran-to-lisp::sorgtr
  fortran-to-lisp::ssterf
  fortran-to-lisp::slascl
  fortran-to-lisp::slansy
  fortran-to-lisp::ilaenv))))
```

———-

57.2 ssyevd

57.2.1 Axiom unit tests

— **ssyevd.input** —

```
)set break resume
)sys rm -f ssyevd.output
)spool ssyevd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

57.2.2 Axiom help page

— ssyevd.help —

```
====================================================================
ssyevd examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssyevd (JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, IWORK,
           LIWORK, INFO)
            SSYEVD computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for SY matrices

Function/Subroutine Documentation
   subroutine ssyevd (character JOBZ, character UPLO, integer N, real,
       dimension( lda, * ) A, integer LDA, real, dimension( * ) W, real,
       dimension( * ) WORK, integer LWORK, integer, dimension( * ) IWORK,
       integer LIWORK, integer INFO)
        SSYEVD computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for SY matrices

Purpose:
   SSYEVD computes all eigenvalues and, optionally, eigenvectors of a
   real symmetric matrix A. If eigenvectors are desired, it uses a
   divide and conquer algorithm.

   The divide and conquer algorithm makes very mild assumptions about
   floating point arithmetic. It will work on machines with a guard
   digit in add/subtract, or on those binary machines without guard
   digits which subtract like the Cray X-MP, Cray Y-MP, Cray C-90, or
   Cray-2. It could conceivably fail on hexadecimal or decimal machines
   without guard digits, but we know of none.

   Because of large use of BLAS of level 3, SSYEVD needs N**2 more
   workspace than SSYEVX.

Parameters:

 JOBZ
   JOBZ is CHARACTER*1
   = 'N':  Compute eigenvalues only;
   = 'V':  Compute eigenvalues and eigenvectors.
```

```
UPLO
  UPLO is CHARACTER*1
  = 'U':  Upper triangle of A is stored;
  = 'L':  Lower triangle of A is stored.

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

A
  A is REAL array, dimension (LDA, N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the
  leading N-by-N upper triangular part of A contains the
  upper triangular part of the matrix A.  If UPLO = 'L',
  the leading N-by-N lower triangular part of A contains
  the lower triangular part of the matrix A.
  On exit, if JOBZ = 'V', then if INFO = 0, A contains the
  orthonormal eigenvectors of the matrix A.
  If JOBZ = 'N', then on exit the lower triangle (if UPLO='L')
  or the upper triangle (if UPLO='U') of A, including the
  diagonal, is destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

W
  W is REAL array, dimension (N)
  If INFO = 0, the eigenvalues in ascending order.

WORK
  WORK is REAL array, dimension (LWORK)
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  If N <= 1,               LWORK must be at least 1.
  If JOBZ = 'N' and N > 1, LWORK must be at least 2*N+1.
  If JOBZ = 'V' and N > 1, LWORK must be at least
  1 + 6*N + 2*N**2.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal sizes of the WORK and IWORK
  arrays, returns these values as the first entries of the WORK
  and IWORK arrays, and no error message related to LWORK or
  LIWORK is issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (MAX(1,LIWORK))
```

```
  On exit, if INFO = 0, IWORK(1) returns the optimal LIWORK.

LIWORK
  LIWORK is INTEGER
  The dimension of the array IWORK.
  If N <= 1,                LIWORK must be at least 1.
  If JOBZ  = 'N' and N > 1, LIWORK must be at least 1.
  If JOBZ  = 'V' and N > 1, LIWORK must be at least 3 + 5*N.

  If LIWORK = -1, then a workspace query is assumed; the
  routine only calculates the optimal sizes of the WORK and
  IWORK arrays, returns these values as the first entries of
  the WORK and IWORK arrays, and no error message related to
  LWORK or LIWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i and JOBZ = 'N', then the algorithm failed
        to converge; i off-diagonal elements of an intermediate
        tridiagonal form did not converge to zero;
        if INFO = i and JOBZ = 'V', then the algorithm failed
        to compute an eigenvalue while working on the submatrix
        lying in rows and columns INFO/(N+1) through mod(INFO,N+1).
```

———-

57.2.3 fortran code

— ssyevd.f —

```
*  =====================================================================
      SUBROUTINE SSYEVD( JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, IWORK,
     $                   LIWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOBZ, UPLO
      INTEGER            INFO, LDA, LIWORK, LWORK, N
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
```

```
      REAL               A( LDA, * ), W( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
*
      LOGICAL            LOWER, LQUERY, WANTZ
      INTEGER            IINFO, INDE, INDTAU, INDWK2, INDWRK, ISCALE,
     $                   LIOPT, LIWMIN, LLWORK, LLWRK2, LOPT, LWMIN
      REAL               ANRM, BIGNUM, EPS, RMAX, RMIN, SAFMIN, SIGMA,
     $                   SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               SLAMCH, SLANSY
      EXTERNAL           ILAENV, LSAME, SLAMCH, SLANSY
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLACPY, SLASCL, SORMTR, SSCAL, SSTEDC, SSTERF,
     $                   SSYTRD, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      WANTZ = LSAME( JOBZ, 'V' )
      LOWER = LSAME( UPLO, 'L' )
      LQUERY = ( LWORK.EQ.-1 .OR. LIWORK.EQ.-1 )
*
      INFO = 0
      IF( .NOT.( WANTZ .OR. LSAME( JOBZ, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( .NOT.( LOWER .OR. LSAME( UPLO, 'U' ) ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      END IF
*
      IF( INFO.EQ.0 ) THEN
```

```
         IF( N.LE.1 ) THEN
            LIWMIN = 1
            LWMIN = 1
            LOPT = LWMIN
            LIOPT = LIWMIN
         ELSE
            IF( WANTZ ) THEN
               LIWMIN = 3 + 5*N
               LWMIN = 1 + 6*N + 2*N**2
            ELSE
               LIWMIN = 1
               LWMIN = 2*N + 1
            END IF
            LOPT = MAX( LWMIN, 2*N +
     $                  ILAENV( 1, 'SSYTRD', UPLO, N, -1, -1, -1 ) )
            LIOPT = LIWMIN
         END IF
         WORK( 1 ) = LOPT
         IWORK( 1 ) = LIOPT
*
         IF( LWORK.LT.LWMIN .AND. .NOT.LQUERY ) THEN
            INFO = -8
         ELSE IF( LIWORK.LT.LIWMIN .AND. .NOT.LQUERY ) THEN
            INFO = -10
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYEVD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
      IF( N.EQ.1 ) THEN
         W( 1 ) = A( 1, 1 )
         IF( WANTZ )
     $      A( 1, 1 ) = ONE
         RETURN
      END IF
*
*     Get machine constants.
*
      SAFMIN = SLAMCH( 'Safe minimum' )
      EPS = SLAMCH( 'Precision' )
```

```
      SMLNUM = SAFMIN / EPS
      BIGNUM = ONE / SMLNUM
      RMIN = SQRT( SMLNUM )
      RMAX = SQRT( BIGNUM )
*
*     Scale matrix to allowable range, if necessary.
*
      ANRM = SLANSY( 'M', UPLO, N, A, LDA, WORK )
      ISCALE = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.RMIN ) THEN
         ISCALE = 1
         SIGMA = RMIN / ANRM
      ELSE IF( ANRM.GT.RMAX ) THEN
         ISCALE = 1
         SIGMA = RMAX / ANRM
      END IF
      IF( ISCALE.EQ.1 )
     $   CALL SLASCL( UPLO, 0, 0, ONE, SIGMA, N, N, A, LDA, INFO )
*
*     Call SSYTRD to reduce symmetric matrix to tridiagonal form.
*
      INDE = 1
      INDTAU = INDE + N
      INDWRK = INDTAU + N
      LLWORK = LWORK - INDWRK + 1
      INDWK2 = INDWRK + N*N
      LLWRK2 = LWORK - INDWK2 + 1
*
      CALL SSYTRD( UPLO, N, A, LDA, W, WORK( INDE ), WORK( INDTAU ),
     $             WORK( INDWRK ), LLWORK, IINFO )
*
*     For eigenvalues only, call SSTERF.  For eigenvectors, first call
*     SSTEDC to generate the eigenvector matrix, WORK(INDWRK), of the
*     tridiagonal matrix, then call SORMTR to multiply it by the
*     Householder transformations stored in A.
*
      IF( .NOT.WANTZ ) THEN
         CALL SSTERF( N, W, WORK( INDE ), INFO )
      ELSE
         CALL SSTEDC( 'I', N, W, WORK( INDE ), WORK( INDWRK ), N,
     $                WORK( INDWK2 ), LLWRK2, IWORK, LIWORK, INFO )
         CALL SORMTR( 'L', UPLO, 'N', N, N, A, LDA, WORK( INDTAU ),
     $                WORK( INDWRK ), N, WORK( INDWK2 ), LLWRK2, IINFO )
         CALL SLACPY( 'A', N, N, WORK( INDWRK ), N, A, LDA )
      END IF
*
*     If matrix was scaled, then rescale eigenvalues appropriately.
*
      IF( ISCALE.EQ.1 )
     $   CALL SSCAL( N, ONE / SIGMA, W, 1 )
```

```
*
      WORK( 1 ) = LOPT
      IWORK( 1 ) = LIOPT
*
      RETURN
*
*     End of SSYEVD
*
      END
```

———-

57.2.4 lisp code

— **ssyevd.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun ssyevd (jobz uplo n a lda w work lwork iwork liwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (array single-float (*)) work w a)
             (type (f2cl-lib:integer4) info liwork lwork lda n)
             (type (string *) uplo jobz))
    (f2cl-lib:with-multi-array-data
        ((jobz character jobz-%data% jobz-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (w single-float w-%data% w-%offset%)
         (work single-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((anrm 0.0) (bignum 0.0) (eps 0.0) (rmax 0.0) (rmin 0.0)
             (safmin 0.0) (sigma 0.0) (smlnum 0.0) (iinfo 0) (inde 0)
             (indtau 0) (indwk2 0) (indwrk 0) (iscale 0) (liopt 0) (liwmin 0)
             (llwork 0) (llwrk2 0) (lopt 0) (lwmin 0) (lower nil) (lquery nil)
             (wantz nil))
        (declare
         (type (single-float) anrm bignum eps rmax rmin safmin sigma smlnum)
         (type (f2cl-lib:integer4) iinfo inde indtau indwk2 indwrk iscale liopt
          liwmin llwork llwrk2 lopt lwmin)
         (type f2cl-lib:logical lower lquery wantz))
        (setf wantz
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame jobz "V")
                  (declare (ignore var-1))
                  (when var-0 (setf jobz var-0))
                  ret-val))
```

```
(setf lower
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(setf lquery (or (= lwork -1) (= liwork -1)))
(setf info 0)
(cond
 ((not
   (or wantz
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobz "N")
         (declare (ignore var-1))
         (when var-0 (setf jobz var-0))
         ret-val)))
  (setf info -1))
 ((not
   (or lower
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame uplo "U")
         (declare (ignore var-1))
         (when var-0 (setf uplo var-0))
         ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5)))
(cond
 ((= info 0)
  (cond
   ((<= n 1) (setf liwmin 1) (setf lwmin 1) (setf lopt lwmin)
    (setf liopt liwmin))
   (t
    (cond
     (wantz (setf liwmin (f2cl-lib:int-add 3 (f2cl-lib:int-mul 5 n)))
      (setf lwmin
              (f2cl-lib:int-add 1 (f2cl-lib:int-mul 6 n)
                                (f2cl-lib:int-mul 2 (expt n 2)))))
     (t (setf liwmin 1)
      (setf lwmin (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) 1))))
    (setf lopt
            (max (the f2cl-lib:integer4 lwmin)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                        (multiple-value-bind
                                            (ret-val var-0 var-1 var-2
                                             var-3 var-4 var-5 var-6)
                                            (ilaenv 1 "SSYTRD" uplo n
                                             -1 -1 -1)
```

```
                                                (declare
                                                 (ignore var-0 var-1 var-4
                                                  var-5 var-6))
                                                (setf uplo var-2)
                                                (setf n var-3)
                                                ret-val)))))
    (setf liopt liwmin)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lopt) 'single-float))
  (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) liopt)
  (cond ((and (< lwork lwmin) (not lquery)) (setf info -8))
        ((and (< liwork liwmin) (not lquery)) (setf info -10)))))
(cond
 ((/= info 0) (xerbla "SSYEVD" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(cond
 ((= n 1)
  (setf (f2cl-lib:fref w-%data% (1) ((1 *)) w-%offset%)
          (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%))
  (if wantz
      (setf (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)
              one))
  (go end_label)))
(setf safmin (slamch "Safe minimum"))
(setf eps (slamch "Precision"))
(setf smlnum (/ safmin eps))
(setf bignum (/ one smlnum))
(setf rmin (f2cl-lib:fsqrt smlnum))
(setf rmax (f2cl-lib:fsqrt bignum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slansy "M" uplo n a lda work)
          (declare (ignore var-0 var-3 var-4 var-5))
          (setf uplo var-1)
          (setf n var-2)
          ret-val))
(setf iscale 0)
(cond
 ((and (> anrm zero) (< anrm rmin)) (setf iscale 1)
  (setf sigma (/ rmin anrm)))
 ((> anrm rmax) (setf iscale 1) (setf sigma (/ rmax anrm))))
(if (= iscale 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slascl uplo 0 0 one sigma n n a lda info)
      (declare
       (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8))
```

```
      (setf uplo var-0)
      (setf info var-9)))
(setf inde 1)
(setf indtau (f2cl-lib:int-add inde n))
(setf indwrk (f2cl-lib:int-add indtau n))
(setf llwork (f2cl-lib:int-add (f2cl-lib:int-sub lwork indwrk) 1))
(setf indwk2 (f2cl-lib:int-add indwrk (f2cl-lib:int-mul n n)))
(setf llwrk2 (f2cl-lib:int-add (f2cl-lib:int-sub lwork indwk2) 1))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (ssytrd uplo n a lda w
     (f2cl-lib:array-slice work-%data% single-float (inde) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (indtau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (indwrk) ((1 *))
                           work-%offset%)
     llwork iinfo)
  (declare (ignore var-2 var-4 var-5 var-6 var-7))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-8 (setf llwork var-8))
  (when var-9 (setf iinfo var-9)))
(cond
 ((not wantz)
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (ssterf n w
       (f2cl-lib:array-slice work-%data% single-float (inde) ((1 *))
                             work-%offset%)
       info)
    (declare (ignore var-0 var-1 var-2))
    (setf info var-3)))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (sstedc "I" n w
       (f2cl-lib:array-slice work-%data% single-float (inde) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (indwrk) ((1 *))
                             work-%offset%)
       n
       (f2cl-lib:array-slice work-%data% single-float (indwk2) ((1 *))
                             work-%offset%)
       llwrk2 iwork liwork info)
    (declare (ignore var-0 var-2 var-3 var-4 var-6 var-7 var-8 var-9))
    (setf n var-1)
    (setf n var-5)
    (setf info var-10))
```

```
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10 var-11 var-12)
              (sormtr "L" uplo "N" n n a lda
               (f2cl-lib:array-slice work-%data% single-float (indtau) ((1 *))
                                     work-%offset%)
               (f2cl-lib:array-slice work-%data% single-float (indwrk) ((1 *))
                                     work-%offset%)
               n
               (f2cl-lib:array-slice work-%data% single-float (indwk2) ((1 *))
                                     work-%offset%)
               llwrk2 iinfo)
            (declare (ignore var-0 var-2 var-5 var-7 var-8 var-10 var-11))
            (setf uplo var-1)
            (setf n var-3)
            (setf n var-4)
            (setf lda var-6)
            (setf n var-9)
            (setf iinfo var-12))
          (slacpy "A" n n
           (f2cl-lib:array-slice work-%data% single-float (indwrk) ((1 *))
                                 work-%offset%)
           n a lda)))
        (if (= iscale 1)
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (sscal n (/ one sigma) w 1)
              (declare (ignore var-1 var-2 var-3))
              (when var-0 (setf n var-0))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lopt) 'single-float))
        (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) liopt)
        (go end_label)
       end_label
        (return (values jobz uplo n nil lda nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssyevd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
```

```
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::jobz
  fortran-to-lisp::uplo
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::slacpy
  fortran-to-lisp::sormtr
  fortran-to-lisp::sstedc
  fortran-to-lisp::ssterf
  fortran-to-lisp::slascl
  fortran-to-lisp::slansy
  fortran-to-lisp::ilaenv))))
```

———-

57.3 ssyevr

57.3.1 Axiom unit tests

— **ssyevr.input** —

```
)set break resume
)sys rm -f ssyevr.output
)spool ssyevr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

57.3.2 Axiom help page

— **ssyevr.help** —

```
==========================================================================
```

```
ssyevr examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssyevr (JOBZ, RANGE, UPLO, N, A, LDA, VL, VU, IL, IU,
           ABSTOL, M, W, Z, LDZ, ISUPPZ, WORK, LWORK, IWORK, LIWORK, INFO)
            SSYEVR computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for SY matrices

Function/Subroutine Documentation
   subroutine ssyevr (character JOBZ, character RANGE, character UPLO, integer
       N, real, dimension( lda, * ) A, integer LDA, real VL, real VU, integer
       IL, integer IU, real ABSTOL, integer M, real, dimension( * ) W, real,
       dimension( ldz, * ) Z, integer LDZ, integer, dimension( * ) ISUPPZ,
       real, dimension( * ) WORK, integer LWORK, integer, dimension( * )
       IWORK, integer LIWORK, integer INFO)
        SSYEVR computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for SY matrices

Purpose:
   SSYEVR computes selected eigenvalues and, optionally, eigenvectors
   of a real symmetric matrix A.  Eigenvalues and eigenvectors can be
   selected by specifying either a range of values or a range of
   indices for the desired eigenvalues.

   SSYEVR first reduces the matrix A to tridiagonal form T with a call
   to SSYTRD.  Then, whenever possible, SSYEVR calls SSTEMR to compute
   the eigenspectrum using Relatively Robust Representations.  SSTEMR
   computes eigenvalues by the dqds algorithm, while orthogonal
   eigenvectors are computed from various "good" L D L^T representations
   (also known as Relatively Robust Representations). Gram-Schmidt
   orthogonalization is avoided as far as possible. More specifically,
   the various steps of the algorithm are as follows.

   For each unreduced block (submatrix) of T,
   (a) Compute T - sigma I  = L D L^T, so that L and D
       define all the wanted eigenvalues to high relative accuracy.
       This means that small relative changes in the entries of D and L
       cause only small relative changes in the eigenvalues and
       eigenvectors. The standard (unfactored) representation of the
       tridiagonal matrix T does not have this property in general.
   (b) Compute the eigenvalues to suitable accuracy.
       If the eigenvectors are desired, the algorithm attains full
       accuracy of the computed eigenvalues only right before
       the corresponding vectors have to be computed, see steps c) and d).
```

```
   (c) For each cluster of close eigenvalues, select a new
       shift close to the cluster, find a new factorization, and refine
       the shifted eigenvalues to suitable accuracy.
   (d) For each eigenvalue with a large enough relative separation compute
       the corresponding eigenvector by forming a rank revealing twisted
       factorization. Go back to (c) for any clusters that remain.

   The desired accuracy of the output can be specified by the input
   parameter ABSTOL.

   Note 1 : SSYEVR calls SSTEMR when the full spectrum is requested
   on machines which conform to the ieee-754 floating point standard.
   SSYEVR calls SSTEBZ and SSTEIN on non-ieee machines and
   when partial spectrum requests are made.

   Normal execution of SSTEMR may create NaNs and infinities and
   hence may abort due to a floating point exception in environments
   which do not handle NaNs and infinities in the ieee standard default
   manner.

Parameters:

 JOBZ
   JOBZ is CHARACTER*1
   = 'N':  Compute eigenvalues only;
   = 'V':  Compute eigenvalues and eigenvectors.

 RANGE
   RANGE is CHARACTER*1
   = 'A': all eigenvalues will be found.
   = 'V': all eigenvalues in the half-open interval (VL,VU] will be found.
   = 'I': the IL-th through IU-th eigenvalues will be found.
   For RANGE = 'V' or 'I' and IU - IL < N - 1, SSTEBZ and SSTEIN are called

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA, N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the
   leading N-by-N upper triangular part of A contains the
   upper triangular part of the matrix A.  If UPLO = 'L',
   the leading N-by-N lower triangular part of A contains
   the lower triangular part of the matrix A.
```

```
  On exit, the lower triangle (if UPLO='L') or the upper
  triangle (if UPLO='U') of A, including the diagonal, is destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

VL
  VL is REAL

VU
  VU is REAL
  If RANGE='V', the lower and upper bounds of the interval to
  be searched for eigenvalues. VL < VU.
  Not referenced if RANGE = 'A' or 'I'.

IL
  IL is INTEGER

IU
  IU is INTEGER
  If RANGE='I', the indices (in ascending order) of the
  smallest and largest eigenvalues to be returned.
  1 <= IL <= IU <= N, if N > 0; IL = 1 and IU = 0 if N = 0.
  Not referenced if RANGE = 'A' or 'V'.

ABSTOL
  ABSTOL is REAL
  The absolute error tolerance for the eigenvalues.
  An approximate eigenvalue is accepted as converged
  when it is determined to lie in an interval [a,b]
  of width less than or equal to

  ABSTOL + EPS *   max( |a|,|b| ) ,

  where EPS is the machine precision.  If ABSTOL is less than
  or equal to zero, then  EPS*|T|  will be used in its place,
  where |T| is the 1-norm of the tridiagonal matrix obtained
  by reducing A to tridiagonal form.

  If high relative accuracy is important, set ABSTOL to
  SLAMCH( 'Safe minimum' ).  Doing so will guarantee that
  eigenvalues are computed to high relative accuracy when
  possible in future releases.  The current code does not
  make any guarantees about high relative accuracy, but
  future releases will. See J. Barlow and J. Demmel,
  "Computing Accurate Eigensystems of Scaled Diagonally
  Dominant Matrices", LAPACK Working Note #7, for a discussion
  of which matrices define their eigenvalues to high relative accuracy.
```

```
M
  M is INTEGER
  The total number of eigenvalues found.  0 <= M <= N.
  If RANGE = 'A', M = N, and if RANGE = 'I', M = IU-IL+1.

W
  W is REAL array, dimension (N)
  The first M elements contain the selected eigenvalues in
  ascending order.

Z
  Z is REAL array, dimension (LDZ, max(1,M))
  If JOBZ = 'V', then if INFO = 0, the first M columns of Z
  contain the orthonormal eigenvectors of the matrix A
  corresponding to the selected eigenvalues, with the i-th
  column of Z holding the eigenvector associated with W(i).
  If JOBZ = 'N', then Z is not referenced.
  Note: the user must ensure that at least max(1,M) columns are
  supplied in the array Z; if RANGE = 'V', the exact value of M
  is not known in advance and an upper bound must be used.
  Supplying N columns is always safe.

LDZ
  LDZ is INTEGER
  The leading dimension of the array Z.  LDZ >= 1, and if
  JOBZ = 'V', LDZ >= max(1,N).

ISUPPZ
  ISUPPZ is INTEGER array, dimension ( 2*max(1,M) )
  The support of the eigenvectors in Z, i.e., the indices
  indicating the nonzero elements in Z. The i-th eigenvector
  is nonzero only in elements ISUPPZ( 2*i-1 ) through ISUPPZ( 2*i ).
  Implemented only for RANGE = 'A' or 'I' and IU - IL = N - 1

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= max(1,26*N).
  For optimal efficiency, LWORK >= (NB+6)*N,
  where NB is the max of the blocksize for SSYTRD and SORMTR
  returned by ILAENV.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal sizes of the WORK and IWORK
  arrays, returns these values as the first entries of the WORK
  and IWORK arrays, and no error message related to LWORK or
  LIWORK is issued by XERBLA.
```

```
 IWORK
   IWORK is INTEGER array, dimension (MAX(1,LIWORK))
   On exit, if INFO = 0, IWORK(1) returns the optimal LWORK.

 LIWORK
   LIWORK is INTEGER
   The dimension of the array IWORK.  LIWORK >= max(1,10*N).

   If LIWORK = -1, then a workspace query is assumed; the
   routine only calculates the optimal sizes of the WORK and
   IWORK arrays, returns these values as the first entries of
   the WORK and IWORK arrays, and no error message related to
   LWORK or LIWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  Internal error
```

———-

57.3.3 fortran code

— ssyevr.f —

```
*  =====================================================================
      SUBROUTINE SSYEVR( JOBZ, RANGE, UPLO, N, A, LDA, VL, VU, IL, IU,
     $                   ABSTOL, M, W, Z, LDZ, ISUPPZ, WORK, LWORK,
     $                   IWORK, LIWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          JOBZ, RANGE, UPLO
      INTEGER            IL, INFO, IU, LDA, LDZ, LIWORK, LWORK, M, N
      REAL               ABSTOL, VL, VU
*     ..
*     .. Array Arguments ..
      INTEGER            ISUPPZ( * ), IWORK( * )
      REAL               A( LDA, * ), W( * ), WORK( * ), Z( LDZ, * )
*     ..
*
```

```
* =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE, TWO
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0, TWO = 2.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ALLEIG, INDEIG, LOWER, LQUERY, TEST, VALEIG,
     $                   WANTZ, TRYRAC
      CHARACTER          ORDER
      INTEGER            I, IEEEOK, IINFO, IMAX, INDD, INDDD, INDE,
     $                   INDEE, INDIBL, INDIFL, INDISP, INDIWO, INDTAU,
     $                   INDWK, INDWKN, ISCALE, J, JJ, LIWMIN,
     $                   LLWORK, LLWRKN, LWKOPT, LWMIN, NB, NSPLIT
      REAL               ABSTLL, ANRM, BIGNUM, EPS, RMAX, RMIN, SAFMIN,
     $                   SIGMA, SMLNUM, TMP1, VLL, VUU
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               SLAMCH, SLANSY
      EXTERNAL           LSAME, ILAENV, SLAMCH, SLANSY
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SORMTR, SSCAL, SSTEBZ, SSTEMR, SSTEIN,
     $                   SSTERF, SSWAP, SSYTRD, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      IEEEOK = ILAENV( 10, 'SSYEVR', 'N', 1, 2, 3, 4 )
*
      LOWER = LSAME( UPLO, 'L' )
      WANTZ = LSAME( JOBZ, 'V' )
      ALLEIG = LSAME( RANGE, 'A' )
      VALEIG = LSAME( RANGE, 'V' )
      INDEIG = LSAME( RANGE, 'I' )
*
      LQUERY = ( ( LWORK.EQ.-1 ) .OR. ( LIWORK.EQ.-1 ) )
*
      LWMIN = MAX( 1, 26*N )
      LIWMIN = MAX( 1, 10*N )
*
      INFO = 0
      IF( .NOT.( WANTZ .OR. LSAME( JOBZ, 'N' ) ) ) THEN
         INFO = -1
```

```
      ELSE IF( .NOT.( ALLEIG .OR. VALEIG .OR. INDEIG ) ) THEN
         INFO = -2
      ELSE IF( .NOT.( LOWER .OR. LSAME( UPLO, 'U' ) ) ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE
         IF( VALEIG ) THEN
            IF( N.GT.0 .AND. VU.LE.VL )
     $         INFO = -8
         ELSE IF( INDEIG ) THEN
            IF( IL.LT.1 .OR. IL.GT.MAX( 1, N ) ) THEN
               INFO = -9
            ELSE IF( IU.LT.MIN( N, IL ) .OR. IU.GT.N ) THEN
               INFO = -10
            END IF
         END IF
      END IF
      IF( INFO.EQ.0 ) THEN
         IF( LDZ.LT.1 .OR. ( WANTZ .AND. LDZ.LT.N ) ) THEN
            INFO = -15
         END IF
      END IF
*
      IF( INFO.EQ.0 ) THEN
         NB = ILAENV( 1, 'SSYTRD', UPLO, N, -1, -1, -1 )
         NB = MAX( NB, ILAENV( 1, 'SORMTR', UPLO, N, -1, -1, -1 ) )
         LWKOPT = MAX( ( NB+1 )*N, LWMIN )
         WORK( 1 ) = LWKOPT
         IWORK( 1 ) = LIWMIN
*
         IF( LWORK.LT.LWMIN .AND. .NOT.LQUERY ) THEN
            INFO = -18
         ELSE IF( LIWORK.LT.LIWMIN .AND. .NOT.LQUERY ) THEN
            INFO = -20
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYEVR', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      M = 0
```

```
      IF( N.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      IF( N.EQ.1 ) THEN
         WORK( 1 ) = 26
         IF( ALLEIG .OR. INDEIG ) THEN
            M = 1
            W( 1 ) = A( 1, 1 )
         ELSE
            IF( VL.LT.A( 1, 1 ) .AND. VU.GE.A( 1, 1 ) ) THEN
               M = 1
               W( 1 ) = A( 1, 1 )
            END IF
         END IF
         IF( WANTZ ) THEN
            Z( 1, 1 ) = ONE
            ISUPPZ( 1 ) = 1
            ISUPPZ( 2 ) = 1
         END IF
         RETURN
      END IF
*
*     Get machine constants.
*
      SAFMIN = SLAMCH( 'Safe minimum' )
      EPS = SLAMCH( 'Precision' )
      SMLNUM = SAFMIN / EPS
      BIGNUM = ONE / SMLNUM
      RMIN = SQRT( SMLNUM )
      RMAX = MIN( SQRT( BIGNUM ), ONE / SQRT( SQRT( SAFMIN ) ) )
*
*     Scale matrix to allowable range, if necessary.
*
      ISCALE = 0
      ABSTLL = ABSTOL
      IF (VALEIG) THEN
         VLL = VL
         VUU = VU
      END IF
      ANRM = SLANSY( 'M', UPLO, N, A, LDA, WORK )
      IF( ANRM.GT.ZERO .AND. ANRM.LT.RMIN ) THEN
         ISCALE = 1
         SIGMA = RMIN / ANRM
      ELSE IF( ANRM.GT.RMAX ) THEN
         ISCALE = 1
         SIGMA = RMAX / ANRM
      END IF
      IF( ISCALE.EQ.1 ) THEN
```

```
         IF( LOWER ) THEN
            DO 10 J = 1, N
               CALL SSCAL( N-J+1, SIGMA, A( J, J ), 1 )
   10       CONTINUE
         ELSE
            DO 20 J = 1, N
               CALL SSCAL( J, SIGMA, A( 1, J ), 1 )
   20       CONTINUE
         END IF
         IF( ABSTOL.GT.0 )
     $      ABSTLL = ABSTOL*SIGMA
         IF( VALEIG ) THEN
            VLL = VL*SIGMA
            VUU = VU*SIGMA
         END IF
      END IF

*     Initialize indices into workspaces.  Note: The IWORK indices are
*     used only if SSTERF or SSTEMR fail.

*     WORK(INDTAU:INDTAU+N-1) stores the scalar factors of the
*     elementary reflectors used in SSYTRD.
      INDTAU = 1
*     WORK(INDD:INDD+N-1) stores the tridiagonal's diagonal entries.
      INDD = INDTAU + N
*     WORK(INDE:INDE+N-1) stores the off-diagonal entries of the
*     tridiagonal matrix from SSYTRD.
      INDE = INDD + N
*     WORK(INDDD:INDDD+N-1) is a copy of the diagonal entries over
*     -written by SSTEMR (the SSTERF path copies the diagonal to W).
      INDDD = INDE + N
*     WORK(INDEE:INDEE+N-1) is a copy of the off-diagonal entries over
*     -written while computing the eigenvalues in SSTERF and SSTEMR.
      INDEE = INDDD + N
*     INDWK is the starting offset of the left-over workspace, and
*     LLWORK is the remaining workspace size.
      INDWK = INDEE + N
      LLWORK = LWORK - INDWK + 1

*     IWORK(INDIBL:INDIBL+M-1) corresponds to IBLOCK in SSTEBZ and
*     stores the block indices of each of the M<=N eigenvalues.
      INDIBL = 1
*     IWORK(INDISP:INDISP+NSPLIT-1) corresponds to ISPLIT in SSTEBZ and
*     stores the starting and finishing indices of each block.
      INDISP = INDIBL + N
*     IWORK(INDIFL:INDIFL+N-1) stores the indices of eigenvectors
*     that corresponding to eigenvectors that fail to converge in
*     SSTEIN.  This information is discarded; if any fail, the driver
*     returns INFO > 0.
      INDIFL = INDISP + N
```

```
*     INDIWO is the offset of the remaining integer workspace.
      INDIWO = INDIFL + N

*
*     Call SSYTRD to reduce symmetric matrix to tridiagonal form.
*
      CALL SSYTRD( UPLO, N, A, LDA, WORK( INDD ), WORK( INDE ),
     $             WORK( INDTAU ), WORK( INDWK ), LLWORK, IINFO )
*
*     If all eigenvalues are desired
*     then call SSTERF or SSTEMR and SORMTR.
*
      TEST = .FALSE.
      IF( INDEIG ) THEN
         IF( IL.EQ.1 .AND. IU.EQ.N ) THEN
            TEST = .TRUE.
         END IF
      END IF
      IF( ( ALLEIG.OR.TEST ) .AND. ( IEEEOK.EQ.1 ) ) THEN
         IF( .NOT.WANTZ ) THEN
            CALL SCOPY( N, WORK( INDD ), 1, W, 1 )
            CALL SCOPY( N-1, WORK( INDE ), 1, WORK( INDEE ), 1 )
            CALL SSTERF( N, W, WORK( INDEE ), INFO )
         ELSE
            CALL SCOPY( N-1, WORK( INDE ), 1, WORK( INDEE ), 1 )
            CALL SCOPY( N, WORK( INDD ), 1, WORK( INDDD ), 1 )
*
            IF (ABSTOL .LE. TWO*N*EPS) THEN
               TRYRAC = .TRUE.
            ELSE
               TRYRAC = .FALSE.
            END IF
            CALL SSTEMR( JOBZ, 'A', N, WORK( INDDD ), WORK( INDEE ),
     $                   VL, VU, IL, IU, M, W, Z, LDZ, N, ISUPPZ,
     $                   TRYRAC, WORK( INDWK ), LWORK, IWORK, LIWORK,
     $                   INFO )
*
*
*
*        Apply orthogonal matrix used in reduction to tridiagonal
*        form to eigenvectors returned by SSTEIN.
*
            IF( WANTZ .AND. INFO.EQ.0 ) THEN
               INDWKN = INDE
               LLWRKN = LWORK - INDWKN + 1
               CALL SORMTR( 'L', UPLO, 'N', N, M, A, LDA,
     $                      WORK( INDTAU ), Z, LDZ, WORK( INDWKN ),
     $                      LLWRKN, IINFO )
            END IF
         END IF
```

```
*
*
         IF( INFO.EQ.0 ) THEN
*           Everything worked.  Skip SSTEBZ/SSTEIN.  IWORK(:) are
*           undefined.
            M = N
            GO TO 30
         END IF
         INFO = 0
      END IF
*
*     Otherwise, call SSTEBZ and, if eigenvectors are desired, SSTEIN.
*     Also call SSTEBZ and SSTEIN if SSTEMR fails.
*
      IF( WANTZ ) THEN
         ORDER = 'B'
      ELSE
         ORDER = 'E'
      END IF

      CALL SSTEBZ( RANGE, ORDER, N, VLL, VUU, IL, IU, ABSTLL,
     $             WORK( INDD ), WORK( INDE ), M, NSPLIT, W,
     $             IWORK( INDIBL ), IWORK( INDISP ), WORK( INDWK ),
     $             IWORK( INDIWO ), INFO )
*
      IF( WANTZ ) THEN
         CALL SSTEIN( N, WORK( INDD ), WORK( INDE ), M, W,
     $                IWORK( INDIBL ), IWORK( INDISP ), Z, LDZ,
     $                WORK( INDWK ), IWORK( INDIWO ), IWORK( INDIFL ),
     $                INFO )
*
*        Apply orthogonal matrix used in reduction to tridiagonal
*        form to eigenvectors returned by SSTEIN.
*
         INDWKN = INDE
         LLWRKN = LWORK - INDWKN + 1
         CALL SORMTR( 'L', UPLO, 'N', N, M, A, LDA, WORK( INDTAU ), Z,
     $                LDZ, WORK( INDWKN ), LLWRKN, IINFO )
      END IF
*
*     If matrix was scaled, then rescale eigenvalues appropriately.
*
*  Jump here if SSTEMR/SSTEIN succeeded.
   30 CONTINUE
      IF( ISCALE.EQ.1 ) THEN
         IF( INFO.EQ.0 ) THEN
            IMAX = M
         ELSE
            IMAX = INFO - 1
         END IF
```

```
         CALL SSCAL( IMAX, ONE / SIGMA, W, 1 )
      END IF
*
*     If eigenvalues are not in order, then sort them, along with
*     eigenvectors.  Note: We do not sort the IFAIL portion of IWORK.
*     It may not be initialized (if SSTEMR/SSTEIN succeeded), and we do
*     not return this detailed information to the user.
*
      IF( WANTZ ) THEN
         DO 50 J = 1, M - 1
            I = 0
            TMP1 = W( J )
            DO 40 JJ = J + 1, M
               IF( W( JJ ).LT.TMP1 ) THEN
                  I = JJ
                  TMP1 = W( JJ )
               END IF
   40       CONTINUE
*
            IF( I.NE.0 ) THEN
               W( I ) = W( J )
               W( J ) = TMP1
               CALL SSWAP( N, Z( 1, I ), 1, Z( 1, J ), 1 )
            END IF
   50    CONTINUE
      END IF
*
*     Set WORK(1) to optimal workspace size.
*
      WORK( 1 ) = LWKOPT
      IWORK( 1 ) = LIWMIN
*
      RETURN
*
*     End of SSYEVR
*
      END
```

———-

57.3.4 lisp code

— ssyevr.lisp —

```
(let* ((zero 0.0) (one 1.0) (two 2.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
```

```
          (ignorable zero one two))
 (defun ssyevr
        (jobz range uplo n a lda vl vu il iu abstol m w z ldz isuppz work
         lwork iwork liwork info)
   (declare (type (array f2cl-lib:integer4 (*)) iwork isuppz)
            (type (single-float) abstol vu vl)
            (type (array single-float (*)) work z w a)
            (type (f2cl-lib:integer4) info liwork lwork ldz m iu il lda n)
            (type (string *) uplo range jobz))
   (f2cl-lib:with-multi-array-data
       ((jobz character jobz-%data% jobz-%offset%)
        (range character range-%data% range-%offset%)
        (uplo character uplo-%data% uplo-%offset%)
        (a single-float a-%data% a-%offset%)
        (w single-float w-%data% w-%offset%)
        (z single-float z-%data% z-%offset%)
        (work single-float work-%data% work-%offset%)
        (isuppz f2cl-lib:integer4 isuppz-%data% isuppz-%offset%)
        (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
     (prog ((abstll 0.0) (anrm 0.0) (bignum 0.0) (eps 0.0) (rmax 0.0)
            (rmin 0.0) (safmin 0.0) (sigma 0.0) (smlnum 0.0) (tmp1 0.0)
            (vll 0.0) (vuu 0.0) (i 0) (ieeeok 0) (iinfo 0) (imax 0) (indd 0)
            (inddd 0) (inde 0) (indee 0) (indibl 0) (indifl 0) (indisp 0)
            (indiwo 0) (indtau 0) (indwk 0) (indwkn 0) (iscale 0) (j 0) (jj 0)
            (liwmin 0) (llwork 0) (llwrkn 0) (lwkopt 0) (lwmin 0) (nb 0)
            (nsplit 0)
            (order
             (make-array '(1) :element-type 'character :initial-element #\ ))
            (alleig nil) (indeig nil) (lower nil) (lquery nil) (test nil)
            (valeig nil) (wantz nil) (tryrac nil))
       (declare
        (type (single-float) abstll anrm bignum eps rmax rmin safmin sigma
         smlnum tmp1 vll vuu)
        (type (f2cl-lib:integer4) i ieeeok iinfo imax indd inddd inde indee
         indibl indifl indisp indiwo indtau indwk indwkn iscale j jj liwmin
         llwork llwrkn lwkopt lwmin nb nsplit)
        (type (string 1) order)
        (type f2cl-lib:logical alleig indeig lower lquery test valeig wantz
         tryrac))
       (setf ieeeok (ilaenv 10 "SSYEVR" "N" 1 2 3 4))
       (setf lower
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame uplo "L")
                 (declare (ignore var-1))
                 (when var-0 (setf uplo var-0))
                 ret-val))
       (setf wantz
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame jobz "V")
                 (declare (ignore var-1))
```

```
          (when var-0 (setf jobz var-0))
          ret-val))
(setf alleig
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "A")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf valeig
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "V")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf indeig
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "I")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf lquery (or (= lwork -1) (= liwork -1)))
(setf lwmin
        (max (the f2cl-lib:integer4 1)
             (the f2cl-lib:integer4 (f2cl-lib:int-mul 26 n))))
(setf liwmin
        (max (the f2cl-lib:integer4 1)
             (the f2cl-lib:integer4 (f2cl-lib:int-mul 10 n))))
(setf info 0)
(cond
 ((not
   (or wantz
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobz "N")
         (declare (ignore var-1))
         (when var-0 (setf jobz var-0))
         ret-val)))
  (setf info -1))
 ((not (or alleig valeig indeig)) (setf info -2))
 ((not
   (or lower
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame uplo "U")
         (declare (ignore var-1))
         (when var-0 (setf uplo var-0))
         ret-val)))
  (setf info -3))
 ((< n 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 (t
```

```
  (cond
   (valeig
    (if (and (> n 0) (<= vu vl))
        (setf info -8)))
   (indeig
    (cond
     ((or (< il 1)
          (> il
             (max (the f2cl-lib:integer4 1)
                  (the f2cl-lib:integer4 n))))
      (setf info -9))
     ((or
       (< iu
          (min (the f2cl-lib:integer4 n) (the f2cl-lib:integer4 il)))
       (> iu n))
      (setf info -10)))))))
(cond
 ((= info 0)
  (cond ((or (< ldz 1) (and wantz (< ldz n))) (setf info -15)))))
(cond
 ((= info 0)
  (setf nb
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (ilaenv 1 "SSYTRD" uplo n -1 -1 -1)
            (declare (ignore var-0 var-1 var-4 var-5 var-6))
            (setf uplo var-2)
            (setf n var-3)
            ret-val))
  (setf nb
          (max (the f2cl-lib:integer4 nb)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 1 "SORMTR" uplo n -1 -1 -1)
                      (declare (ignore var-0 var-1 var-4 var-5 var-6))
                      (setf uplo var-2)
                      (setf n var-3)
                      ret-val))))
  (setf lwkopt
          (max
           (the f2cl-lib:integer4
                (f2cl-lib:int-mul (f2cl-lib:int-add nb 1) n))
           (the f2cl-lib:integer4 lwmin)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
  (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) liwmin)
  (cond ((and (< lwork lwmin) (not lquery)) (setf info -18))
        ((and (< liwork liwmin) (not lquery)) (setf info -20)))))
```

```
(cond
 ((/= info 0) (xerbla "SSYEVR" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(setf m 0)
(cond
 ((= n 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'single-float))
  (go end_label)))
(cond
 ((= n 1)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 26) 'single-float))
  (cond
   ((or alleig indeig) (setf m 1)
    (setf (f2cl-lib:fref w-%data% (1) ((1 *)) w-%offset%)
            (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)))
   (t
    (cond
     ((and (< vl (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))
           (>= vu (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))))
      (setf m 1)
      (setf (f2cl-lib:fref w-%data% (1) ((1 *)) w-%offset%)
              (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *))
                             a-%offset%))))))
  (cond
   (wantz
    (setf (f2cl-lib:fref z-%data% (1 1) ((1 ldz) (1 *)) z-%offset%)
            one)
    (setf (f2cl-lib:fref isuppz-%data% (1) ((1 *)) isuppz-%offset%) 1)
    (setf (f2cl-lib:fref isuppz-%data% (2) ((1 *)) isuppz-%offset%)
            1)))
  (go end_label)))
(setf safmin (slamch "Safe minimum"))
(setf eps (slamch "Precision"))
(setf smlnum (/ safmin eps))
(setf bignum (/ one smlnum))
(setf rmin (f2cl-lib:fsqrt smlnum))
(setf rmax
        (min (f2cl-lib:fsqrt bignum)
             (/ one (f2cl-lib:fsqrt (f2cl-lib:fsqrt safmin)))))
(setf iscale 0)
(setf abstll abstol)
(cond (valeig (setf vll vl) (setf vuu vu)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slansy "M" uplo n a lda work)
          (declare (ignore var-0 var-3 var-4 var-5))
          (setf uplo var-1)
```

```
          (setf n var-2)
          ret-val))
(cond
 ((and (> anrm zero) (< anrm rmin)) (setf iscale 1)
  (setf sigma (/ rmin anrm)))
 ((> anrm rmax) (setf iscale 1) (setf sigma (/ rmax anrm))))
(cond
 ((= iscale 1)
  (cond
   (lower
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1) sigma
             (f2cl-lib:array-slice a-%data% single-float (j j)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-0 var-2 var-3))
          (when var-1 (setf sigma var-1)))
       label10)))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal j sigma
             (f2cl-lib:array-slice a-%data% single-float (1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf j var-0))
          (when var-1 (setf sigma var-1)))
       label20))))
  (if (> abstol 0)
      (setf abstll (* abstol sigma)))
  (cond (valeig (setf vll (* vl sigma)) (setf vuu (* vu sigma))))))
(setf indtau 1)
(setf indd (f2cl-lib:int-add indtau n))
(setf inde (f2cl-lib:int-add indd n))
(setf inddd (f2cl-lib:int-add inde n))
(setf indee (f2cl-lib:int-add inddd n))
(setf indwk (f2cl-lib:int-add indee n))
(setf llwork (f2cl-lib:int-add (f2cl-lib:int-sub lwork indwk) 1))
(setf indibl 1)
(setf indisp (f2cl-lib:int-add indibl n))
(setf indifl (f2cl-lib:int-add indisp n))
(setf indiwo (f2cl-lib:int-add indifl n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
```

```
     (ssytrd uplo n a lda
      (f2cl-lib:array-slice work-%data% single-float (indd) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (inde) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (indtau) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (indwk) ((1 *))
                            work-%offset%)
      llwork iinfo)
    (declare (ignore var-2 var-4 var-5 var-6 var-7))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf lda var-3))
    (when var-8 (setf llwork var-8))
    (when var-9 (setf iinfo var-9)))
  (setf test f2cl-lib:%false%)
  (cond
   (indeig (cond ((and (= il 1) (= iu n)) (setf test f2cl-lib:%true%)))))
  (cond
   ((and (or alleig test) (= ieeeok 1))
    (cond
     ((not wantz)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (scopy n
           (f2cl-lib:array-slice work-%data% single-float (indd) ((1 *))
                                 work-%offset%)
           1 w 1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf n var-0)))
      (scopy (f2cl-lib:int-sub n 1)
       (f2cl-lib:array-slice work-%data% single-float (inde) ((1 *))
                             work-%offset%)
       1
       (f2cl-lib:array-slice work-%data% single-float (indee) ((1 *))
                             work-%offset%)
       1)
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (ssterf n w
           (f2cl-lib:array-slice work-%data% single-float (indee) ((1 *))
                                 work-%offset%)
           info)
        (declare (ignore var-0 var-1 var-2))
        (setf info var-3)))
     (t
      (scopy (f2cl-lib:int-sub n 1)
       (f2cl-lib:array-slice work-%data% single-float (inde) ((1 *))
                             work-%offset%)
       1
       (f2cl-lib:array-slice work-%data% single-float (indee) ((1 *))
```

```
                        work-%offset%)
 1)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (scopy n
     (f2cl-lib:array-slice work-%data% single-float (indd) ((1 *))
                           work-%offset%)
     1
     (f2cl-lib:array-slice work-%data% single-float (inddd) ((1 *))
                           work-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(cond ((<= abstol (* two n eps)) (setf tryrac f2cl-lib:%true%))
      (t (setf tryrac f2cl-lib:%false%)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18
     var-19 var-20)
    (sstemr jobz "A" n
     (f2cl-lib:array-slice work-%data% single-float (inddd) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (indee) ((1 *))
                           work-%offset%)
     vl vu il iu m w z ldz n isuppz tryrac
     (f2cl-lib:array-slice work-%data% single-float (indwk) ((1 *))
                           work-%offset%)
     lwork iwork liwork info)
  (declare
   (ignore var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-10 var-11
    var-13 var-14 var-16 var-17 var-18 var-19))
  (setf jobz var-0)
  (setf n var-2)
  (setf m var-9)
  (setf ldz var-12)
  (setf tryrac var-15)
  (setf info var-20))
(cond
 ((and wantz (= info 0)) (setf indwkn inde)
  (setf llwrkn
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork indwkn) 1))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sormtr "L" uplo "N" n m a lda
       (f2cl-lib:array-slice work-%data% single-float (indtau)
                             ((1 *)) work-%offset%)
       z ldz
       (f2cl-lib:array-slice work-%data% single-float (indwkn)
                             ((1 *)) work-%offset%)
       llwrkn iinfo)
```

```
        (declare (ignore var-0 var-2 var-5 var-7 var-8 var-10 var-11))
        (setf uplo var-1)
        (setf n var-3)
        (setf m var-4)
        (setf lda var-6)
        (setf ldz var-9)
        (setf iinfo var-12))))))
  (cond ((= info 0) (setf m n) (go label30))) (setf info 0)))
(cond (wantz (f2cl-lib:f2cl-set-string order "B" (string 1)))
      (t (f2cl-lib:f2cl-set-string order "E" (string 1))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17)
    (sstebz range order n vll vuu il iu abstll
     (f2cl-lib:array-slice work-%data% single-float (indd) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (inde) ((1 *))
                           work-%offset%)
     m nsplit w
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indibl)
                           ((1 *)) iwork-%offset%)
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indisp)
                           ((1 *)) iwork-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (indwk) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indiwo)
                           ((1 *)) iwork-%offset%)
     info)
  (declare
   (ignore var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-12 var-13
    var-14 var-15 var-16))
  (setf range var-0)
  (setf order var-1)
  (setf n var-2)
  (setf m var-10)
  (setf nsplit var-11)
  (setf info var-17))
(cond
 (wantz
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sstein n
       (f2cl-lib:array-slice work-%data% single-float (indd) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (inde) ((1 *))
                             work-%offset%)
       m w
       (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indibl)
                             ((1 *)) iwork-%offset%)
```

```
          (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indisp)
                                ((1 *)) iwork-%offset%)
          z ldz
          (f2cl-lib:array-slice work-%data% single-float (indwk) ((1 *))
                                work-%offset%)
          (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indiwo)
                                ((1 *)) iwork-%offset%)
          (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indifl)
                                ((1 *)) iwork-%offset%)
          info)
        (declare
         (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
          var-9 var-10 var-11))
        (setf info var-12))
      (setf indwkn inde)
      (setf llwrkn (f2cl-lib:int-add (f2cl-lib:int-sub lwork indwkn) 1))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (sormtr "L" uplo "N" n m a lda
           (f2cl-lib:array-slice work-%data% single-float (indtau) ((1 *))
                                 work-%offset%)
           z ldz
           (f2cl-lib:array-slice work-%data% single-float (indwkn) ((1 *))
                                 work-%offset%)
           llwrkn iinfo)
        (declare (ignore var-0 var-2 var-5 var-7 var-8 var-10 var-11))
        (setf uplo var-1)
        (setf n var-3)
        (setf m var-4)
        (setf lda var-6)
        (setf ldz var-9)
        (setf iinfo var-12))))
  label30
   (cond
    ((= iscale 1)
     (cond ((= info 0) (setf imax m))
           (t (setf imax (f2cl-lib:int-sub info 1))))
     (multiple-value-bind (var-0 var-1 var-2 var-3)
         (sscal imax (/ one sigma) w 1)
       (declare (ignore var-1 var-2 var-3))
       (when var-0 (setf imax var-0)))))
   (cond
    (wantz
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
       (tagbody
         (setf i 0)
         (setf tmp1 (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%))
         (f2cl-lib:fdo (jj (f2cl-lib:int-add j 1) (f2cl-lib:int-add jj 1))
```

```
                        ((> jj m) nil)
              (tagbody
                (cond
                 ((< (f2cl-lib:fref w (jj) ((1 *))) tmp1) (setf i jj)
                  (setf tmp1
                        (f2cl-lib:fref w-%data% (jj) ((1 *)) w-%offset%))))
               label40))
            (cond
             ((/= i 0)
              (setf (f2cl-lib:fref w-%data% (i) ((1 *)) w-%offset%)
                      (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%))
              (setf (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%) tmp1)
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (sswap n
                   (f2cl-lib:array-slice z-%data% single-float (1 i)
                                         ((1 ldz) (1 *)) z-%offset%)
                   1
                   (f2cl-lib:array-slice z-%data% single-float (1 j)
                                         ((1 ldz) (1 *)) z-%offset%)
                   1)
                (declare (ignore var-1 var-2 var-3 var-4))
                (when var-0 (setf n var-0)))))
          label50))))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
      (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) liwmin)
      (go end_label)
     end_label
      (return
       (values jobz range uplo n nil lda nil nil nil nil nil m nil nil ldz
               nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssyevr
                 fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((string) (string) (string)
                                            (fortran-to-lisp::integer4)
                                            (array single-float (*))
                                            (fortran-to-lisp::integer4)
                                            (single-float) (single-float)
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (single-float)
                                            (fortran-to-lisp::integer4)
                                            (array single-float (*))
                                            (array single-float (*))
                                            (fortran-to-lisp::integer4)
```

```
  (array fortran-to-lisp::integer4
   (*))
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::jobz
  fortran-to-lisp::range
  fortran-to-lisp::uplo
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil nil
  nil nil fortran-to-lisp::m nil
  nil fortran-to-lisp::ldz nil nil
  nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::sstein
  fortran-to-lisp::sstebz
  fortran-to-lisp::sormtr
  fortran-to-lisp::sstemr
  fortran-to-lisp::ssterf
  fortran-to-lisp::slansy
  fortran-to-lisp::ilaenv))))
```

———-

57.4 ssyevx

57.4.1 Axiom unit tests

— ssyevx.input —

```
)set break resume
)sys rm -f ssyevx.output
)spool ssyevx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

57.4.2 Axiom help page

— ssyevx.help —

```
==========================================================================
ssyevx examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssyevx (JOBZ, RANGE, UPLO, N, A, LDA, VL, VU, IL, IU,
           ABSTOL, M, W, Z, LDZ, WORK, LWORK, IWORK, IFAIL, INFO)
            SSYEVX computes the eigenvalues and, optionally, the left and/or
           right eigenvectors for SY matrices

Function/Subroutine Documentation
   subroutine ssyevx (character JOBZ, character RANGE, character UPLO, integer
       N, real, dimension( lda, * ) A, integer LDA, real VL, real VU, integer
       IL, integer IU, real ABSTOL, integer M, real, dimension( * ) W, real,
       dimension( ldz, * ) Z, integer LDZ, real, dimension( * ) WORK, integer
       LWORK, integer, dimension( * ) IWORK, integer, dimension( * ) IFAIL,
       integer INFO)
        SSYEVX computes the eigenvalues and, optionally, the left and/or right
       eigenvectors for SY matrices

Purpose:
   SSYEVX computes selected eigenvalues and, optionally, eigenvectors
   of a real symmetric matrix A.  Eigenvalues and eigenvectors can be
   selected by specifying either a range of values or a range of indices
   for the desired eigenvalues.

Parameters:

 JOBZ
   JOBZ is CHARACTER*1
   = 'N':  Compute eigenvalues only;
   = 'V':  Compute eigenvalues and eigenvectors.

 RANGE
   RANGE is CHARACTER*1
   = 'A': all eigenvalues will be found.
   = 'V': all eigenvalues in the half-open interval (VL,VU] will be found.
```

```
  = 'I': the IL-th through IU-th eigenvalues will be found.

UPLO
  UPLO is CHARACTER*1
  = 'U':  Upper triangle of A is stored;
  = 'L':  Lower triangle of A is stored.

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

A
  A is REAL array, dimension (LDA, N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the
  leading N-by-N upper triangular part of A contains the
  upper triangular part of the matrix A.  If UPLO = 'L',
  the leading N-by-N lower triangular part of A contains
  the lower triangular part of the matrix A.
  On exit, the lower triangle (if UPLO='L') or the upper
  triangle (if UPLO='U') of A, including the diagonal, is destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

VL
  VL is REAL

VU
  VU is REAL
  If RANGE='V', the lower and upper bounds of the interval to
  be searched for eigenvalues. VL < VU.
  Not referenced if RANGE = 'A' or 'I'.

IL
  IL is INTEGER

IU
  IU is INTEGER
  If RANGE='I', the indices (in ascending order) of the
  smallest and largest eigenvalues to be returned.
  1 <= IL <= IU <= N, if N > 0; IL = 1 and IU = 0 if N = 0.
  Not referenced if RANGE = 'A' or 'V'.

ABSTOL
  ABSTOL is REAL
  The absolute error tolerance for the eigenvalues.
  An approximate eigenvalue is accepted as converged
  when it is determined to lie in an interval [a,b]
  of width less than or equal to
```

```
  ABSTOL + EPS *   max( |a|,|b| ) ,

  where EPS is the machine precision.  If ABSTOL is less than
  or equal to zero, then  EPS*|T|  will be used in its place,
  where |T| is the 1-norm of the tridiagonal matrix obtained
  by reducing A to tridiagonal form.

  Eigenvalues will be computed most accurately when ABSTOL is
  set to twice the underflow threshold 2*SLAMCH('S'), not zero.
  If this routine returns with INFO>0, indicating that some
  eigenvectors did not converge, try setting ABSTOL to
  2*SLAMCH('S').

M
  M is INTEGER
  The total number of eigenvalues found.  0 <= M <= N.
  If RANGE = 'A', M = N, and if RANGE = 'I', M = IU-IL+1.

W
  W is REAL array, dimension (N)
  On normal exit, the first M elements contain the selected
  eigenvalues in ascending order.

Z
  Z is REAL array, dimension (LDZ, max(1,M))
  If JOBZ = 'V', then if INFO = 0, the first M columns of Z
  contain the orthonormal eigenvectors of the matrix A
  corresponding to the selected eigenvalues, with the i-th
  column of Z holding the eigenvector associated with W(i).
  If an eigenvector fails to converge, then that column of Z
  contains the latest approximation to the eigenvector, and the
  index of the eigenvector is returned in IFAIL.
  If JOBZ = 'N', then Z is not referenced.
  Note: the user must ensure that at least max(1,M) columns are
  supplied in the array Z; if RANGE = 'V', the exact value of M
  is not known in advance and an upper bound must be used.

LDZ
  LDZ is INTEGER
  The leading dimension of the array Z.  LDZ >= 1, and if
  JOBZ = 'V', LDZ >= max(1,N).

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of the array WORK.  LWORK >= 1, when N <= 1;
```

```
  otherwise 8*N.
  For optimal efficiency, LWORK >= (NB+3)*N,
  where NB is the max of the blocksize for SSYTRD and SORMTR
  returned by ILAENV.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (5*N)

IFAIL
  IFAIL is INTEGER array, dimension (N)
  If JOBZ = 'V', then if INFO = 0, the first M elements of
  IFAIL are zero.  If INFO > 0, then IFAIL contains the
  indices of the eigenvectors that failed to converge.
  If JOBZ = 'N', then IFAIL is not referenced.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, then i eigenvectors failed to converge.
        Their indices are stored in array IFAIL.
```

———-

57.4.3 fortran code

— **ssyevx.f** —

```
*  =====================================================================
      SUBROUTINE SSYEVX( JOBZ, RANGE, UPLO, N, A, LDA, VL, VU, IL, IU,
     $                   ABSTOL, M, W, Z, LDZ, WORK, LWORK, IWORK,
     $                   IFAIL, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOBZ, RANGE, UPLO
      INTEGER            IL, INFO, IU, LDA, LDZ, LWORK, M, N
      REAL               ABSTOL, VL, VU
```

```
*     ..
*     .. Array Arguments ..
      INTEGER            IFAIL( * ), IWORK( * )
      REAL               A( LDA, * ), W( * ), WORK( * ), Z( LDZ, * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ALLEIG, INDEIG, LOWER, LQUERY, TEST, VALEIG,
     $                   WANTZ
      CHARACTER          ORDER
      INTEGER            I, IINFO, IMAX, INDD, INDE, INDEE, INDIBL,
     $                   INDISP, INDIWO, INDTAU, INDWKN, INDWRK, ISCALE,
     $                   ITMP1, J, JJ, LLWORK, LLWRKN, LWKMIN,
     $                   LWKOPT, NB, NSPLIT
      REAL               ABSTLL, ANRM, BIGNUM, EPS, RMAX, RMIN, SAFMIN,
     $                   SIGMA, SMLNUM, TMP1, VLL, VUU
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               SLAMCH, SLANSY
      EXTERNAL           LSAME, ILAENV, SLAMCH, SLANSY
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SLACPY, SORGTR, SORMTR, SSCAL, SSTEBZ,
     $                   SSTEIN, SSTEQR, SSTERF, SSWAP, SSYTRD, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      LOWER = LSAME( UPLO, 'L' )
      WANTZ = LSAME( JOBZ, 'V' )
      ALLEIG = LSAME( RANGE, 'A' )
      VALEIG = LSAME( RANGE, 'V' )
      INDEIG = LSAME( RANGE, 'I' )
      LQUERY = ( LWORK.EQ.-1 )
*
      INFO = 0
      IF( .NOT.( WANTZ .OR. LSAME( JOBZ, 'N' ) ) ) THEN
         INFO = -1
```

```
      ELSE IF( .NOT.( ALLEIG .OR. VALEIG .OR. INDEIG ) ) THEN
         INFO = -2
      ELSE IF( .NOT.( LOWER .OR. LSAME( UPLO, 'U' ) ) ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE
         IF( VALEIG ) THEN
            IF( N.GT.0 .AND. VU.LE.VL )
     $         INFO = -8
         ELSE IF( INDEIG ) THEN
            IF( IL.LT.1 .OR. IL.GT.MAX( 1, N ) ) THEN
               INFO = -9
            ELSE IF( IU.LT.MIN( N, IL ) .OR. IU.GT.N ) THEN
               INFO = -10
            END IF
         END IF
      END IF
      IF( INFO.EQ.0 ) THEN
         IF( LDZ.LT.1 .OR. ( WANTZ .AND. LDZ.LT.N ) ) THEN
            INFO = -15
         END IF
      END IF
*
      IF( INFO.EQ.0 ) THEN
         IF( N.LE.1 ) THEN
            LWKMIN = 1
            WORK( 1 ) = LWKMIN
         ELSE
            LWKMIN = 8*N
            NB = ILAENV( 1, 'SSYTRD', UPLO, N, -1, -1, -1 )
            NB = MAX( NB, ILAENV( 1, 'SORMTR', UPLO, N, -1, -1, -1 ) )
            LWKOPT = MAX( LWKMIN, ( NB + 3 )*N )
            WORK( 1 ) = LWKOPT
         END IF
*
         IF( LWORK.LT.LWKMIN .AND. .NOT.LQUERY )
     $      INFO = -17
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYEVX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
```

```
*
      M = 0
      IF( N.EQ.0 ) THEN
         RETURN
      END IF
*
      IF( N.EQ.1 ) THEN
         IF( ALLEIG .OR. INDEIG ) THEN
            M = 1
            W( 1 ) = A( 1, 1 )
         ELSE
            IF( VL.LT.A( 1, 1 ) .AND. VU.GE.A( 1, 1 ) ) THEN
               M = 1
               W( 1 ) = A( 1, 1 )
            END IF
         END IF
         IF( WANTZ )
     $      Z( 1, 1 ) = ONE
         RETURN
      END IF
*
*     Get machine constants.
*
      SAFMIN = SLAMCH( 'Safe minimum' )
      EPS = SLAMCH( 'Precision' )
      SMLNUM = SAFMIN / EPS
      BIGNUM = ONE / SMLNUM
      RMIN = SQRT( SMLNUM )
      RMAX = MIN( SQRT( BIGNUM ), ONE / SQRT( SQRT( SAFMIN ) ) )
*
*     Scale matrix to allowable range, if necessary.
*
      ISCALE = 0
      ABSTLL = ABSTOL
      IF( VALEIG ) THEN
         VLL = VL
         VUU = VU
      END IF
      ANRM = SLANSY( 'M', UPLO, N, A, LDA, WORK )
      IF( ANRM.GT.ZERO .AND. ANRM.LT.RMIN ) THEN
         ISCALE = 1
         SIGMA = RMIN / ANRM
      ELSE IF( ANRM.GT.RMAX ) THEN
         ISCALE = 1
         SIGMA = RMAX / ANRM
      END IF
      IF( ISCALE.EQ.1 ) THEN
         IF( LOWER ) THEN
            DO 10 J = 1, N
               CALL SSCAL( N-J+1, SIGMA, A( J, J ), 1 )
```

```
   10       CONTINUE
         ELSE
            DO 20 J = 1, N
               CALL SSCAL( J, SIGMA, A( 1, J ), 1 )
   20       CONTINUE
         END IF
         IF( ABSTOL.GT.0 )
     $      ABSTLL = ABSTOL*SIGMA
         IF( VALEIG ) THEN
            VLL = VL*SIGMA
            VUU = VU*SIGMA
         END IF
      END IF
*
*     Call SSYTRD to reduce symmetric matrix to tridiagonal form.
*
      INDTAU = 1
      INDE = INDTAU + N
      INDD = INDE + N
      INDWRK = INDD + N
      LLWORK = LWORK - INDWRK + 1
      CALL SSYTRD( UPLO, N, A, LDA, WORK( INDD ), WORK( INDE ),
     $             WORK( INDTAU ), WORK( INDWRK ), LLWORK, IINFO )
*
*     If all eigenvalues are desired and ABSTOL is less than or equal to
*     zero, then call SSTERF or SORGTR and SSTEQR.  If this fails for
*     some eigenvalue, then try SSTEBZ.
*
      TEST = .FALSE.
      IF( INDEIG ) THEN
         IF( IL.EQ.1 .AND. IU.EQ.N ) THEN
            TEST = .TRUE.
         END IF
      END IF
      IF( ( ALLEIG .OR. TEST ) .AND. ( ABSTOL.LE.ZERO ) ) THEN
         CALL SCOPY( N, WORK( INDD ), 1, W, 1 )
         INDEE = INDWRK + 2*N
         IF( .NOT.WANTZ ) THEN
            CALL SCOPY( N-1, WORK( INDE ), 1, WORK( INDEE ), 1 )
            CALL SSTERF( N, W, WORK( INDEE ), INFO )
         ELSE
            CALL SLACPY( 'A', N, N, A, LDA, Z, LDZ )
            CALL SORGTR( UPLO, N, Z, LDZ, WORK( INDTAU ),
     $                   WORK( INDWRK ), LLWORK, IINFO )
            CALL SCOPY( N-1, WORK( INDE ), 1, WORK( INDEE ), 1 )
            CALL SSTEQR( JOBZ, N, W, WORK( INDEE ), Z, LDZ,
     $                   WORK( INDWRK ), INFO )
            IF( INFO.EQ.0 ) THEN
               DO 30 I = 1, N
                  IFAIL( I ) = 0
```

```
   30         CONTINUE
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
            M = N
            GO TO 40
         END IF
         INFO = 0
      END IF
*
*     Otherwise, call SSTEBZ and, if eigenvectors are desired, SSTEIN.
*
      IF( WANTZ ) THEN
         ORDER = 'B'
      ELSE
         ORDER = 'E'
      END IF
      INDIBL = 1
      INDISP = INDIBL + N
      INDIWO = INDISP + N
      CALL SSTEBZ( RANGE, ORDER, N, VLL, VUU, IL, IU, ABSTLL,
     $             WORK( INDD ), WORK( INDE ), M, NSPLIT, W,
     $             IWORK( INDIBL ), IWORK( INDISP ), WORK( INDWRK ),
     $             IWORK( INDIWO ), INFO )
*
      IF( WANTZ ) THEN
         CALL SSTEIN( N, WORK( INDD ), WORK( INDE ), M, W,
     $                IWORK( INDIBL ), IWORK( INDISP ), Z, LDZ,
     $                WORK( INDWRK ), IWORK( INDIWO ), IFAIL, INFO )
*
*        Apply orthogonal matrix used in reduction to tridiagonal
*        form to eigenvectors returned by SSTEIN.
*
         INDWKN = INDE
         LLWRKN = LWORK - INDWKN + 1
         CALL SORMTR( 'L', UPLO, 'N', N, M, A, LDA, WORK( INDTAU ), Z,
     $                LDZ, WORK( INDWKN ), LLWRKN, IINFO )
      END IF
*
*     If matrix was scaled, then rescale eigenvalues appropriately.
*
   40 CONTINUE
      IF( ISCALE.EQ.1 ) THEN
         IF( INFO.EQ.0 ) THEN
            IMAX = M
         ELSE
            IMAX = INFO - 1
         END IF
         CALL SSCAL( IMAX, ONE / SIGMA, W, 1 )
      END IF
```

```
*
*     If eigenvalues are not in order, then sort them, along with
*     eigenvectors.
*
      IF( WANTZ ) THEN
         DO 60 J = 1, M - 1
            I = 0
            TMP1 = W( J )
            DO 50 JJ = J + 1, M
               IF( W( JJ ).LT.TMP1 ) THEN
                  I = JJ
                  TMP1 = W( JJ )
               END IF
   50       CONTINUE
*
            IF( I.NE.0 ) THEN
               ITMP1 = IWORK( INDIBL+I-1 )
               W( I ) = W( J )
               IWORK( INDIBL+I-1 ) = IWORK( INDIBL+J-1 )
               W( J ) = TMP1
               IWORK( INDIBL+J-1 ) = ITMP1
               CALL SSWAP( N, Z( 1, I ), 1, Z( 1, J ), 1 )
               IF( INFO.NE.0 ) THEN
                  ITMP1 = IFAIL( I )
                  IFAIL( I ) = IFAIL( J )
                  IFAIL( J ) = ITMP1
               END IF
            END IF
   60    CONTINUE
      END IF
*
*     Set WORK(1) to optimal workspace size.
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of SSYEVX
*
      END
```

———-

57.4.4 lisp code

— **ssyevx.lisp** —

```
(let* ((zero 0.0) (one 1.0))
```

```
(declare (type (single-float 0.0 0.0) zero)
         (type (single-float 1.0 1.0) one)
         (ignorable zero one))
(defun ssyevx
       (jobz range uplo n a lda vl vu il iu abstol m w z ldz work lwork iwork
        ifail info)
  (declare (type (array f2cl-lib:integer4 (*)) ifail iwork)
           (type (single-float) abstol vu vl)
           (type (array single-float (*)) work z w a)
           (type (f2cl-lib:integer4) info lwork ldz m iu il lda n)
           (type (string *) uplo range jobz))
  (f2cl-lib:with-multi-array-data
      ((jobz character jobz-%data% jobz-%offset%)
       (range character range-%data% range-%offset%)
       (uplo character uplo-%data% uplo-%offset%)
       (a single-float a-%data% a-%offset%)
       (w single-float w-%data% w-%offset%)
       (z single-float z-%data% z-%offset%)
       (work single-float work-%data% work-%offset%)
       (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%)
       (ifail f2cl-lib:integer4 ifail-%data% ifail-%offset%))
    (prog ((abstll 0.0) (anrm 0.0) (bignum 0.0) (eps 0.0) (rmax 0.0)
           (rmin 0.0) (safmin 0.0) (sigma 0.0) (smlnum 0.0) (tmp1 0.0)
           (vll 0.0) (vuu 0.0) (i 0) (iinfo 0) (imax 0) (indd 0) (inde 0)
           (indee 0) (indibl 0) (indisp 0) (indiwo 0) (indtau 0) (indwkn 0)
           (indwrk 0) (iscale 0) (itmp1 0) (j 0) (jj 0) (llwork 0) (llwrkn 0)
           (lwkmin 0) (lwkopt 0) (nb 0) (nsplit 0)
           (order
            (make-array '(1) :element-type 'character :initial-element #\ ))
           (alleig nil) (indeig nil) (lower nil) (lquery nil) (test nil)
           (valeig nil) (wantz nil))
      (declare
       (type (single-float) abstll anrm bignum eps rmax rmin safmin sigma
        smlnum tmp1 vll vuu)
       (type (f2cl-lib:integer4) i iinfo imax indd inde indee indibl indisp
        indiwo indtau indwkn indwrk iscale itmp1 j jj llwork llwrkn lwkmin
        lwkopt nb nsplit)
       (type (string 1) order)
       (type f2cl-lib:logical alleig indeig lower lquery test valeig wantz))
      (setf lower
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (setf wantz
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame jobz "V")
                (declare (ignore var-1))
                (when var-0 (setf jobz var-0))
```

```
          ret-val))
(setf alleig
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "A")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf valeig
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "V")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf indeig
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame range "I")
          (declare (ignore var-1))
          (when var-0 (setf range var-0))
          ret-val))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(setf info 0)
(cond
 ((not
   (or wantz
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame jobz "N")
         (declare (ignore var-1))
         (when var-0 (setf jobz var-0))
         ret-val)))
  (setf info -1))
 ((not (or alleig valeig indeig)) (setf info -2))
 ((not
   (or lower
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame uplo "U")
         (declare (ignore var-1))
         (when var-0 (setf uplo var-0))
         ret-val)))
  (setf info -3))
 ((< n 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 (t
  (cond
   (valeig
    (if (and (> n 0) (<= vu vl))
        (setf info -8)))
   (indeig
    (cond
     ((or (< il 1)
```

```
            (> il
               (max (the f2cl-lib:integer4 1)
                    (the f2cl-lib:integer4 n))))
        (setf info -9))
       ((or
         (< iu
            (min (the f2cl-lib:integer4 n) (the f2cl-lib:integer4 il)))
         (> iu n))
        (setf info -10)))))))
(cond
 ((= info 0)
  (cond ((or (< ldz 1) (and wantz (< ldz n))) (setf info -15)))))
(cond
 ((= info 0)
  (cond
   ((<= n 1) (setf lwkmin 1)
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce (the f2cl-lib:integer4 lwkmin) 'single-float)))
   (t (setf lwkmin (f2cl-lib:int-mul 8 n))
    (setf nb
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 1 "SSYTRD" uplo n -1 -1 -1)
              (declare (ignore var-0 var-1 var-4 var-5 var-6))
              (setf uplo var-2)
              (setf n var-3)
              ret-val))
    (setf nb
            (max (the f2cl-lib:integer4 nb)
                 (the f2cl-lib:integer4
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
                          (ilaenv 1 "SORMTR" uplo n -1 -1 -1)
                        (declare
                         (ignore var-0 var-1 var-4 var-5 var-6))
                        (setf uplo var-2)
                        (setf n var-3)
                        ret-val))))
    (setf lwkopt
            (max (the f2cl-lib:integer4 lwkmin)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-mul (f2cl-lib:int-add nb 3) n))))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))))
  (if (and (< lwork lwkmin) (not lquery))
      (setf info -17))))
(cond
 ((/= info 0) (xerbla "SSYEVX" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
```

```
(setf m 0)
(cond ((= n 0) (go end_label)))
(cond
 ((= n 1)
  (cond
   ((or alleig indeig) (setf m 1)
    (setf (f2cl-lib:fref w-%data% (1) ((1 *)) w-%offset%)
            (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)))
   (t
    (cond
     ((and (< vl (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))
           (>= vu (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))))
      (setf m 1)
      (setf (f2cl-lib:fref w-%data% (1) ((1 *)) w-%offset%)
              (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *))
                              a-%offset%))))))
  (if wantz
      (setf (f2cl-lib:fref z-%data% (1 1) ((1 ldz) (1 *)) z-%offset%)
              one))
  (go end_label)))
(setf safmin (slamch "Safe minimum"))
(setf eps (slamch "Precision"))
(setf smlnum (/ safmin eps))
(setf bignum (/ one smlnum))
(setf rmin (f2cl-lib:fsqrt smlnum))
(setf rmax
        (min (f2cl-lib:fsqrt bignum)
             (/ one (f2cl-lib:fsqrt (f2cl-lib:fsqrt safmin)))))
(setf iscale 0)
(setf abstll abstol)
(cond (valeig (setf vll vl) (setf vuu vu)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slansy "M" uplo n a lda work)
          (declare (ignore var-0 var-3 var-4 var-5))
          (setf uplo var-1)
          (setf n var-2)
          ret-val))
(cond
 ((and (> anrm zero) (< anrm rmin)) (setf iscale 1)
  (setf sigma (/ rmin anrm)))
 ((> anrm rmax) (setf iscale 1) (setf sigma (/ rmax anrm))))
(cond
 ((= iscale 1)
  (cond
   (lower
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
```

```
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (sscal (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1) sigma
               (f2cl-lib:array-slice a-%data% single-float (j j)
                                     ((1 lda) (1 *)) a-%offset%)
               1)
            (declare (ignore var-0 var-2 var-3))
            (when var-1 (setf sigma var-1)))
        label10)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (multiple-value-bind (var-0 var-1 var-2 var-3)
             (sscal j sigma
              (f2cl-lib:array-slice a-%data% single-float (1 j)
                                    ((1 lda) (1 *)) a-%offset%)
              1)
            (declare (ignore var-2 var-3))
            (when var-0 (setf j var-0))
            (when var-1 (setf sigma var-1)))
        label20))))
  (if (> abstol 0)
      (setf abstll (* abstol sigma)))
  (cond (valeig (setf vll (* vl sigma)) (setf vuu (* vu sigma))))))
(setf indtau 1)
(setf inde (f2cl-lib:int-add indtau n))
(setf indd (f2cl-lib:int-add inde n))
(setf indwrk (f2cl-lib:int-add indd n))
(setf llwork (f2cl-lib:int-add (f2cl-lib:int-sub lwork indwrk) 1))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (ssytrd uplo n a lda
     (f2cl-lib:array-slice work-%data% single-float (indd) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (inde) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (indtau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (indwrk) ((1 *))
                           work-%offset%)
     llwork iinfo)
  (declare (ignore var-2 var-4 var-5 var-6 var-7))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-8 (setf llwork var-8))
  (when var-9 (setf iinfo var-9)))
(setf test f2cl-lib:%false%)
(cond
 (indeig (cond ((and (= il 1) (= iu n)) (setf test f2cl-lib:%true%)))))
```

```
(cond
 ((and (or alleig test) (<= abstol zero))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (scopy n
       (f2cl-lib:array-slice work-%data% single-float (indd) ((1 *))
                             work-%offset%)
       1 w 1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf n var-0)))
  (setf indee (f2cl-lib:int-add indwrk (f2cl-lib:int-mul 2 n)))
  (cond
   ((not wantz)
    (scopy (f2cl-lib:int-sub n 1)
     (f2cl-lib:array-slice work-%data% single-float (inde) ((1 *))
                           work-%offset%)
     1
     (f2cl-lib:array-slice work-%data% single-float (indee) ((1 *))
                           work-%offset%)
     1)
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (ssterf n w
         (f2cl-lib:array-slice work-%data% single-float (indee) ((1 *))
                               work-%offset%)
         info)
      (declare (ignore var-0 var-1 var-2))
      (setf info var-3)))
   (t (slacpy "A" n n a lda z ldz)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (sorgtr uplo n z ldz
         (f2cl-lib:array-slice work-%data% single-float (indtau)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% single-float (indwrk)
                               ((1 *)) work-%offset%)
         llwork iinfo)
      (declare (ignore var-1 var-2 var-4 var-5 var-6))
      (setf uplo var-0)
      (setf ldz var-3)
      (setf iinfo var-7))
    (scopy (f2cl-lib:int-sub n 1)
     (f2cl-lib:array-slice work-%data% single-float (inde) ((1 *))
                           work-%offset%)
     1
     (f2cl-lib:array-slice work-%data% single-float (indee) ((1 *))
                           work-%offset%)
     1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (ssteqr jobz n w
         (f2cl-lib:array-slice work-%data% single-float (indee) ((1 *))
```

```
                                work-%offset%)
            z ldz
            (f2cl-lib:array-slice work-%data% single-float (indwrk)
                                  ((1 *)) work-%offset%)
            info)
        (declare (ignore var-2 var-3 var-4 var-5 var-6))
        (setf jobz var-0)
        (setf n var-1)
        (setf info var-7))
      (cond
       ((= info 0)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref ifail-%data% (i) ((1 *)) ifail-%offset%)
                  0)
            label30))))))
    (cond ((= info 0) (setf m n) (go label40))) (setf info 0)))
  (cond (wantz (f2cl-lib:f2cl-set-string order "B" (string 1)))
        (t (f2cl-lib:f2cl-set-string order "E" (string 1))))
  (setf indibl 1)
  (setf indisp (f2cl-lib:int-add indibl n))
  (setf indiwo (f2cl-lib:int-add indisp n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
       var-11 var-12 var-13 var-14 var-15 var-16 var-17)
      (sstebz range order n vll vuu il iu abstll
       (f2cl-lib:array-slice work-%data% single-float (indd) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (inde) ((1 *))
                             work-%offset%)
       m nsplit w
       (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indibl)
                             ((1 *)) iwork-%offset%)
       (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indisp)
                             ((1 *)) iwork-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (indwrk) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indiwo)
                             ((1 *)) iwork-%offset%)
       info)
    (declare
     (ignore var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-12 var-13
      var-14 var-15 var-16))
    (setf range var-0)
    (setf order var-1)
    (setf n var-2)
    (setf m var-10)
    (setf nsplit var-11)
    (setf info var-17))
```

```
 (cond
  (wantz
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12)
       (sstein n
        (f2cl-lib:array-slice work-%data% single-float (indd) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (inde) ((1 *))
                              work-%offset%)
        m w
        (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indibl)
                              ((1 *)) iwork-%offset%)
        (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indisp)
                              ((1 *)) iwork-%offset%)
        z ldz
        (f2cl-lib:array-slice work-%data% single-float (indwrk) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (indiwo)
                              ((1 *)) iwork-%offset%)
        ifail info)
     (declare
      (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11))
     (setf info var-12))
   (setf indwkn inde)
   (setf llwrkn (f2cl-lib:int-add (f2cl-lib:int-sub lwork indwkn) 1))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12)
       (sormtr "L" uplo "N" n m a lda
        (f2cl-lib:array-slice work-%data% single-float (indtau) ((1 *))
                              work-%offset%)
        z ldz
        (f2cl-lib:array-slice work-%data% single-float (indwkn) ((1 *))
                              work-%offset%)
        llwrkn iinfo)
     (declare (ignore var-0 var-2 var-5 var-7 var-8 var-10 var-11))
     (setf uplo var-1)
     (setf n var-3)
     (setf m var-4)
     (setf lda var-6)
     (setf ldz var-9)
     (setf iinfo var-12))))
label40
 (cond
  ((= iscale 1)
   (cond ((= info 0) (setf imax m))
         (t (setf imax (f2cl-lib:int-sub info 1))))
   (multiple-value-bind (var-0 var-1 var-2 var-3)
```

```
        (sscal imax (/ one sigma) w 1)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf imax var-0)))))
  (cond
   (wantz
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (setf i 0)
        (setf tmp1 (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%))
        (f2cl-lib:fdo (jj (f2cl-lib:int-add j 1) (f2cl-lib:int-add jj 1))
                      ((> jj m) nil)
          (tagbody
            (cond
             ((< (f2cl-lib:fref w (jj) ((1 *))) tmp1) (setf i jj)
              (setf tmp1
                      (f2cl-lib:fref w-%data% (jj) ((1 *)) w-%offset%))))
           label50))
        (cond
         ((/= i 0)
          (setf itmp1
                  (f2cl-lib:fref iwork-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-add indibl i) 1))
                                 ((1 *)) iwork-%offset%))
          (setf (f2cl-lib:fref w-%data% (i) ((1 *)) w-%offset%)
                  (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%))
          (setf (f2cl-lib:fref iwork-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add indibl i) 1))
                               ((1 *)) iwork-%offset%)
                  (f2cl-lib:fref iwork-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-add indibl j) 1))
                                 ((1 *)) iwork-%offset%))
          (setf (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%) tmp1)
          (setf (f2cl-lib:fref iwork-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add indibl j) 1))
                               ((1 *)) iwork-%offset%)
                  itmp1)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (sswap n
               (f2cl-lib:array-slice z-%data% single-float (1 i)
                                     ((1 ldz) (1 *)) z-%offset%)
               1
               (f2cl-lib:array-slice z-%data% single-float (1 j)
                                     ((1 ldz) (1 *)) z-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3 var-4))
```

```
                 (when var-0 (setf n var-0)))
               (cond
                ((/= info 0)
                 (setf itmp1
                         (f2cl-lib:fref ifail-%data% (i) ((1 *))
                                        ifail-%offset%))
                 (setf (f2cl-lib:fref ifail-%data% (i) ((1 *)) ifail-%offset%)
                         (f2cl-lib:fref ifail-%data% (j) ((1 *))
                                        ifail-%offset%))
                 (setf (f2cl-lib:fref ifail-%data% (j) ((1 *)) ifail-%offset%)
                         itmp1)))))
             label60))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
        (go end_label)
       end_label
        (return
         (values jobz range uplo n nil lda nil nil nil nil nil m nil nil ldz
                 nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssyevx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (single-float)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::jobz
                                              fortran-to-lisp::range
                                              fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
```

```
  fortran-to-lisp::lda nil nil nil
  nil nil fortran-to-lisp::m nil
  nil fortran-to-lisp::ldz nil nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::sormtr
  fortran-to-lisp::sstein
  fortran-to-lisp::sstebz
  fortran-to-lisp::ssteqr
  fortran-to-lisp::sorgtr
  fortran-to-lisp::slacpy
  fortran-to-lisp::ssterf
  fortran-to-lisp::slansy
  fortran-to-lisp::ilaenv))))
```

———-

57.5 ssygv

57.5.1 Axiom unit tests

— **ssygv.input** —

```
)set break resume
)sys rm -f ssygv.output
)spool ssygv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

57.5.2 Axiom help page

— **ssygv.help** —

```
======================================================================
ssygv examples
======================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssygv (ITYPE, JOBZ, UPLO, N, A, LDA, B, LDB, W, WORK, LWORK,
           INFO)
           SSYGV

Function/Subroutine Documentation
   subroutine ssygv (integer ITYPE, character JOBZ, character UPLO, integer N,
       real, dimension( lda, * ) A, integer LDA, real, dimension( ldb, * ) B,
       integer LDB, real, dimension( * ) W, real, dimension( * ) WORK, integer
       LWORK, integer INFO)
       SSYGV

Purpose:
   SSYGV computes all the eigenvalues, and optionally, the eigenvectors
   of a real generalized symmetric-definite eigenproblem, of the form
   A*x=(lambda)*B*x,  A*Bx=(lambda)*x,  or B*A*x=(lambda)*x.
   Here A and B are assumed to be symmetric and B is also positive definite.

Parameters:

 ITYPE
   ITYPE is INTEGER
   Specifies the problem type to be solved:
   = 1:  A*x = (lambda)*B*x
   = 2:  A*B*x = (lambda)*x
   = 3:  B*A*x = (lambda)*x

 JOBZ
   JOBZ is CHARACTER*1
   = 'N':  Compute eigenvalues only;
   = 'V':  Compute eigenvalues and eigenvectors.

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangles of A and B are stored;
   = 'L':  Lower triangles of A and B are stored.

 N
   N is INTEGER
   The order of the matrices A and B.  N >= 0.

 A
   A is REAL array, dimension (LDA, N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the
```

```
  leading N-by-N upper triangular part of A contains the
  upper triangular part of the matrix A.  If UPLO = 'L',
  the leading N-by-N lower triangular part of A contains
  the lower triangular part of the matrix A.

  On exit, if JOBZ = 'V', then if INFO = 0, A contains the
  matrix Z of eigenvectors.  The eigenvectors are normalized
  as follows:
  if ITYPE = 1 or 2, Z**T*B*Z = I;
  if ITYPE = 3, Z**T*inv(B)*Z = I.
  If JOBZ = 'N', then on exit the upper triangle (if UPLO='U')
  or the lower triangle (if UPLO='L') of A, including the
  diagonal, is destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

B
  B is REAL array, dimension (LDB, N)
  On entry, the symmetric positive definite matrix B.
  If UPLO = 'U', the leading N-by-N upper triangular part of B
  contains the upper triangular part of the matrix B.
  If UPLO = 'L', the leading N-by-N lower triangular part of B
  contains the lower triangular part of the matrix B.

  On exit, if INFO <= N, the part of B containing the matrix is
  overwritten by the triangular factor U or L from the Cholesky
  factorization B = U**T*U or B = L*L**T.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

W
  W is REAL array, dimension (N)
  If INFO = 0, the eigenvalues in ascending order.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of the array WORK.  LWORK >= max(1,3*N-1).
  For optimal efficiency, LWORK >= (NB+2)*N,
  where NB is the blocksize for SSYTRD returned by ILAENV.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
```

```
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  SPOTRF or SSYEV returned an error code:
  <= N:  if INFO = i, SSYEV failed to converge;
         i off-diagonal elements of an intermediate
         tridiagonal form did not converge to zero;
  > N:   if INFO = N + i, for 1 <= i <= N, then the leading
         minor of order i of B is not positive definite.
         The factorization of B could not be completed and
         no eigenvalues or eigenvectors were computed.
```

———-

57.5.3 fortran code

— **ssygv.f** —

```
*  =====================================================================
      SUBROUTINE SSYGV( ITYPE, JOBZ, UPLO, N, A, LDA, B, LDB, W, WORK,
     $                  LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBZ, UPLO
      INTEGER            INFO, ITYPE, LDA, LDB, LWORK, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), B( LDB, * ), W( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, UPPER, WANTZ
```

```
      CHARACTER          TRANS
      INTEGER            LWKMIN, LWKOPT, NB, NEIG
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           ILAENV, LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SPOTRF, SSYEV, SSYGST, STRMM, STRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      WANTZ = LSAME( JOBZ, 'V' )
      UPPER = LSAME( UPLO, 'U' )
      LQUERY = ( LWORK.EQ.-1 )
*
      INFO = 0
      IF( ITYPE.LT.1 .OR. ITYPE.GT.3 ) THEN
         INFO = -1
      ELSE IF( .NOT.( WANTZ .OR. LSAME( JOBZ, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( .NOT.( UPPER .OR. LSAME( UPLO, 'L' ) ) ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
*
      IF( INFO.EQ.0 ) THEN
         LWKMIN = MAX( 1, 3*N - 1 )
         NB = ILAENV( 1, 'SSYTRD', UPLO, N, -1, -1, -1 )
         LWKOPT = MAX( LWKMIN, ( NB + 2 )*N )
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.LWKMIN .AND. .NOT.LQUERY ) THEN
            INFO = -11
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYGV ', -INFO )
```

```
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Form a Cholesky factorization of B.
*
      CALL SPOTRF( UPLO, N, B, LDB, INFO )
      IF( INFO.NE.0 ) THEN
         INFO = N + INFO
         RETURN
      END IF
*
*     Transform problem to standard eigenvalue problem and solve.
*
      CALL SSYGST( ITYPE, UPLO, N, A, LDA, B, LDB, INFO )
      CALL SSYEV( JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, INFO )
*
      IF( WANTZ ) THEN
*
*        Backtransform eigenvectors to the original problem.
*
         NEIG = N
         IF( INFO.GT.0 )
     $      NEIG = INFO - 1
         IF( ITYPE.EQ.1 .OR. ITYPE.EQ.2 ) THEN
*
*           For A*x=(lambda)*B*x and A*B*x=(lambda)*x;
*           backtransform eigenvectors: x = inv(L)**T*y or inv(U)*y
*
            IF( UPPER ) THEN
               TRANS = 'N'
            ELSE
               TRANS = 'T'
            END IF
*
            CALL STRSM( 'Left', UPLO, TRANS, 'Non-unit', N, NEIG, ONE,
     $                  B, LDB, A, LDA )
*
         ELSE IF( ITYPE.EQ.3 ) THEN
*
*           For B*A*x=(lambda)*x;
*           backtransform eigenvectors: x = L*y or U**T*y
*
            IF( UPPER ) THEN
```

```
               TRANS = 'T'
            ELSE
               TRANS = 'N'
            END IF
*
            CALL STRMM( 'Left', UPLO, TRANS, 'Non-unit', N, NEIG, ONE,
     $                  B, LDB, A, LDA )
         END IF
      END IF
*
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of SSYGV
*
      END
```

———-

57.5.4 lisp code

— **ssygv.lisp** —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun ssygv (itype jobz uplo n a lda b ldb$ w work lwork info)
    (declare (type (array single-float (*)) work w b a)
             (type (string *) uplo jobz)
             (type (f2cl-lib:integer4) info lwork ldb$ lda n itype))
    (f2cl-lib:with-multi-array-data
        ((jobz character jobz-%data% jobz-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%)
         (w single-float w-%data% w-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((lwkmin 0) (lwkopt 0) (nb 0) (neig 0)
             (trans
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (lquery nil) (upper nil) (wantz nil))
        (declare (type (f2cl-lib:integer4) lwkmin lwkopt nb neig)
                 (type (string 1) trans)
                 (type f2cl-lib:logical lquery upper wantz))
        (setf wantz
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame jobz "V")
                  (declare (ignore var-1))
```

```
          (when var-0 (setf jobz var-0))
          ret-val))
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(setf info 0)
(cond ((or (< itype 1) (> itype 3)) (setf info -1))
      ((not
        (or wantz
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobz "N")
              (declare (ignore var-1))
              (when var-0 (setf jobz var-0))
              ret-val)))
       (setf info -2))
      ((not
        (or upper
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
       (setf info -3))
      ((< n 0) (setf info -4))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -6))
      ((< ldb$
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -8)))
(cond
 ((= info 0)
  (setf lwkmin
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-sub (f2cl-lib:int-mul 3 n) 1))))
  (setf nb
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (ilaenv 1 "SSYTRD" uplo n -1 -1 -1)
            (declare (ignore var-0 var-1 var-4 var-5 var-6))
            (setf uplo var-2)
            (setf n var-3)
            ret-val))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkmin)
```

```
                (the f2cl-lib:integer4
                     (f2cl-lib:int-mul (f2cl-lib:int-add nb 2) n))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
  (cond ((and (< lwork lwkmin) (not lquery)) (setf info -11)))))
(cond
 ((/= info 0) (xerbla "SSYGV " (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (spotrf uplo n b ldb$ info)
  (declare (ignore var-2))
  (setf uplo var-0)
  (setf n var-1)
  (setf ldb$ var-3)
  (setf info var-4))
(cond
 ((/= info 0) (setf info (f2cl-lib:int-add n info)) (go end_label)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (ssygst itype uplo n a lda b ldb$ info)
  (declare (ignore var-0 var-3 var-5))
  (setf uplo var-1)
  (setf n var-2)
  (setf lda var-4)
  (setf ldb$ var-6)
  (setf info var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (ssyev jobz uplo n a lda w work lwork info)
  (declare (ignore var-3 var-5 var-6 var-7))
  (setf jobz var-0)
  (setf uplo var-1)
  (setf n var-2)
  (setf lda var-4)
  (setf info var-8))
(cond
 (wantz (setf neig n)
  (if (> info 0)
      (setf neig (f2cl-lib:int-sub info 1)))
  (cond
   ((or (= itype 1) (= itype 2))
    (cond (upper (f2cl-lib:f2cl-set-string trans "N" (string 1)))
          (t (f2cl-lib:f2cl-set-string trans "T" (string 1))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (strsm "Left" uplo trans "Non-unit" n neig one b ldb$ a lda)
      (declare (ignore var-0 var-3 var-7 var-9))
      (when var-1 (setf uplo var-1))
```

```
              (when var-2 (setf trans var-2))
              (when var-4 (setf n var-4))
              (when var-5 (setf neig var-5))
              (when var-6 (setf one var-6))
              (when var-8 (setf ldb$ var-8))
              (when var-10 (setf lda var-10))))
            ((= itype 3)
             (cond (upper (f2cl-lib:f2cl-set-string trans "T" (string 1)))
                   (t (f2cl-lib:f2cl-set-string trans "N" (string 1))))
             (multiple-value-bind
                 (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                  var-10)
                 (strmm "Left" uplo trans "Non-unit" n neig one b ldb$ a lda)
              (declare (ignore var-0 var-3 var-7 var-9))
              (when var-1 (setf uplo var-1))
              (when var-2 (setf trans var-2))
              (when var-4 (setf n var-4))
              (when var-5 (setf neig var-5))
              (when var-6 (setf one var-6))
              (when var-8 (setf ldb$ var-8))
              (when var-10 (setf lda var-10)))))))
         (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                 (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
         (go end_label)
        end_label
         (return (values nil jobz uplo n nil lda nil ldb$ nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssygv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil fortran-to-lisp::jobz
                                              fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$ nil nil nil
                                              fortran-to-lisp::info)
```

```
:calls
'(fortran-to_lisp::ssyev
  fortran-to-lisp::ssygst
  fortran-to-lisp::spotrf
  fortran-to-lisp::ilaenv))))
```

57.6 ssygvd

57.6.1 Axiom unit tests

— **ssygvd.input** —

```
)set break resume
)sys rm -f ssygvd.output
)spool ssygvd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

57.6.2 Axiom help page

— **ssygvd.help** —

```
=====================================================================
ssygvd examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssygvd (ITYPE, JOBZ, UPLO, N, A, LDA, B, LDB, W, WORK,
           LWORK, IWORK, LIWORK, INFO)
```

```
          SSYGVD

Function/Subroutine Documentation
   subroutine ssygvd (integer ITYPE, character JOBZ, character UPLO, integer
       N, real, dimension( lda, * ) A, integer LDA, real, dimension( ldb, * )
       B, integer LDB, real, dimension( * ) W, real, dimension( * ) WORK,
       integer LWORK, integer, dimension( * ) IWORK, integer LIWORK, integer
       INFO)
       SSYGVD

Purpose:
   SSYGVD computes all the eigenvalues, and optionally, the eigenvectors
   of a real generalized symmetric-definite eigenproblem, of the form
   A*x=(lambda)*B*x,  A*Bx=(lambda)*x,  or B*A*x=(lambda)*x.  Here A and
   B are assumed to be symmetric and B is also positive definite.
   If eigenvectors are desired, it uses a divide and conquer algorithm.

   The divide and conquer algorithm makes very mild assumptions about
   floating point arithmetic. It will work on machines with a guard
   digit in add/subtract, or on those binary machines without guard
   digits which subtract like the Cray X-MP, Cray Y-MP, Cray C-90, or
   Cray-2. It could conceivably fail on hexadecimal or decimal machines
   without guard digits, but we know of none.

Parameters:

 ITYPE
   ITYPE is INTEGER
   Specifies the problem type to be solved:
   = 1:  A*x = (lambda)*B*x
   = 2:  A*B*x = (lambda)*x
   = 3:  B*A*x = (lambda)*x

 JOBZ
   JOBZ is CHARACTER*1
   = 'N':  Compute eigenvalues only;
   = 'V':  Compute eigenvalues and eigenvectors.

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangles of A and B are stored;
   = 'L':  Lower triangles of A and B are stored.

 N
   N is INTEGER
   The order of the matrices A and B.  N >= 0.

 A
   A is REAL array, dimension (LDA, N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the
```

```
  leading N-by-N upper triangular part of A contains the
  upper triangular part of the matrix A.  If UPLO = 'L',
  the leading N-by-N lower triangular part of A contains
  the lower triangular part of the matrix A.

  On exit, if JOBZ = 'V', then if INFO = 0, A contains the
  matrix Z of eigenvectors.  The eigenvectors are normalized
  as follows:
  if ITYPE = 1 or 2, Z**T*B*Z = I;
  if ITYPE = 3, Z**T*inv(B)*Z = I.
  If JOBZ = 'N', then on exit the upper triangle (if UPLO='U')
  or the lower triangle (if UPLO='L') of A, including the
  diagonal, is destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

B
  B is REAL array, dimension (LDB, N)
  On entry, the symmetric matrix B.  If UPLO = 'U', the
  leading N-by-N upper triangular part of B contains the
  upper triangular part of the matrix B.  If UPLO = 'L',
  the leading N-by-N lower triangular part of B contains
  the lower triangular part of the matrix B.

  On exit, if INFO <= N, the part of B containing the matrix is
  overwritten by the triangular factor U or L from the Cholesky
  factorization B = U**T*U or B = L*L**T.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

W
  W is REAL array, dimension (N)
  If INFO = 0, the eigenvalues in ascending order.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  If N <= 1,               LWORK >= 1.
  If JOBZ = 'N' and N > 1, LWORK >= 2*N+1.
  If JOBZ = 'V' and N > 1, LWORK >= 1 + 6*N + 2*N**2.

  If LWORK = -1, then a workspace query is assumed; the routine
```

```
  only calculates the optimal sizes of the WORK and IWORK
  arrays, returns these values as the first entries of the WORK
  and IWORK arrays, and no error message related to LWORK or
  LIWORK is issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (MAX(1,LIWORK))
  On exit, if INFO = 0, IWORK(1) returns the optimal LIWORK.

LIWORK
  LIWORK is INTEGER
  The dimension of the array IWORK.
  If N <= 1,                LIWORK >= 1.
  If JOBZ  = 'N' and N > 1, LIWORK >= 1.
  If JOBZ  = 'V' and N > 1, LIWORK >= 3 + 5*N.

  If LIWORK = -1, then a workspace query is assumed; the
  routine only calculates the optimal sizes of the WORK and
  IWORK arrays, returns these values as the first entries of
  the WORK and IWORK arrays, and no error message related to
  LWORK or LIWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  SPOTRF or SSYEVD returned an error code:
  <= N:  if INFO = i and JOBZ = 'N', then the algorithm
     failed to converge; i off-diagonal elements of an
     intermediate tridiagonal form did not converge to zero;
     if INFO = i and JOBZ = 'V', then the algorithm
     failed to compute an eigenvalue while working on
     the submatrix lying in rows and columns INFO/(N+1)
     through mod(INFO,N+1);
  > N:   if INFO = N + i, for 1 <= i <= N, then the leading
     minor of order i of B is not positive definite.
     The factorization of B could not be completed and
     no eigenvalues or eigenvectors were computed.
```

———-

57.6.3 fortran code

— ssygvd.f —

```
*  =====================================================================
      SUBROUTINE SSYGVD( ITYPE, JOBZ, UPLO, N, A, LDA, B, LDB, W, WORK,
```

```
     $                   LWORK, IWORK, LIWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBZ, UPLO
      INTEGER            INFO, ITYPE, LDA, LDB, LIWORK, LWORK, N
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      REAL               A( LDA, * ), B( LDB, * ), W( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, UPPER, WANTZ
      CHARACTER          TRANS
      INTEGER            LIOPT, LIWMIN, LOPT, LWMIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SPOTRF, SSYEVD, SSYGST, STRMM, STRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, REAL
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      WANTZ = LSAME( JOBZ, 'V' )
      UPPER = LSAME( UPLO, 'U' )
      LQUERY = ( LWORK.EQ.-1 .OR. LIWORK.EQ.-1 )
*
      INFO = 0
      IF( N.LE.1 ) THEN
         LIWMIN = 1
         LWMIN = 1
      ELSE IF( WANTZ ) THEN
```

```
         LIWMIN = 3 + 5*N
         LWMIN = 1 + 6*N + 2*N**2
      ELSE
         LIWMIN = 1
         LWMIN = 2*N + 1
      END IF
      LOPT = LWMIN
      LIOPT = LIWMIN
      IF( ITYPE.LT.1 .OR. ITYPE.GT.3 ) THEN
         INFO = -1
      ELSE IF( .NOT.( WANTZ .OR. LSAME( JOBZ, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( .NOT.( UPPER .OR. LSAME( UPLO, 'L' ) ) ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
*
      IF( INFO.EQ.0 ) THEN
         WORK( 1 ) = LOPT
         IWORK( 1 ) = LIOPT
*
         IF( LWORK.LT.LWMIN .AND. .NOT.LQUERY ) THEN
            INFO = -11
         ELSE IF( LIWORK.LT.LIWMIN .AND. .NOT.LQUERY ) THEN
            INFO = -13
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYGVD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Form a Cholesky factorization of B.
*
      CALL SPOTRF( UPLO, N, B, LDB, INFO )
      IF( INFO.NE.0 ) THEN
         INFO = N + INFO
```

```
         RETURN
      END IF
*
*     Transform problem to standard eigenvalue problem and solve.
*
      CALL SSYGST( ITYPE, UPLO, N, A, LDA, B, LDB, INFO )
      CALL SSYEVD( JOBZ, UPLO, N, A, LDA, W, WORK, LWORK, IWORK, LIWORK,
     $             INFO )
      LOPT = MAX( REAL( LOPT ), REAL( WORK( 1 ) ) )
      LIOPT = MAX( REAL( LIOPT ), REAL( IWORK( 1 ) ) )
*
      IF( WANTZ .AND. INFO.EQ.0 ) THEN
*
*        Backtransform eigenvectors to the original problem.
*
         IF( ITYPE.EQ.1 .OR. ITYPE.EQ.2 ) THEN
*
*           For A*x=(lambda)*B*x and A*B*x=(lambda)*x;
*           backtransform eigenvectors: x = inv(L)**T*y or inv(U)*y
*
            IF( UPPER ) THEN
               TRANS = 'N'
            ELSE
               TRANS = 'T'
            END IF
*
            CALL STRSM( 'Left', UPLO, TRANS, 'Non-unit', N, N, ONE,
     $                  B, LDB, A, LDA )
*
         ELSE IF( ITYPE.EQ.3 ) THEN
*
*           For B*A*x=(lambda)*x;
*           backtransform eigenvectors: x = L*y or U**T*y
*
            IF( UPPER ) THEN
               TRANS = 'T'
            ELSE
               TRANS = 'N'
            END IF
*
            CALL STRMM( 'Left', UPLO, TRANS, 'Non-unit', N, N, ONE,
     $                  B, LDB, A, LDA )
         END IF
      END IF
*
      WORK( 1 ) = LOPT
      IWORK( 1 ) = LIOPT
*
      RETURN
*
```

```
*     End of SSYGVD
*
      END
```

———-

57.6.4 lisp code

— **ssygvd.lisp** —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun ssygvd (itype jobz uplo n a lda b ldb$ w work lwork iwork liwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (array single-float (*)) work w b a)
             (type (string *) uplo jobz)
             (type (f2cl-lib:integer4) info liwork lwork ldb$ lda n itype))
    (f2cl-lib:with-multi-array-data
        ((jobz character jobz-%data% jobz-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%)
         (w single-float w-%data% w-%offset%)
         (work single-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((liopt 0) (liwmin 0) (lopt 0) (lwmin 0)
             (trans
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (lquery nil) (upper nil) (wantz nil))
        (declare (type (f2cl-lib:integer4) liopt liwmin lopt lwmin)
                 (type (string 1) trans)
                 (type f2cl-lib:logical lquery upper wantz))
        (setf wantz
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame jobz "V")
                  (declare (ignore var-1))
                  (when var-0 (setf jobz var-0))
                  ret-val))
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (setf lquery (or (= lwork -1) (= liwork -1)))
        (setf info 0)
        (cond ((<= n 1) (setf liwmin 1) (setf lwmin 1))
```

```
      (wantz (setf liwmin (f2cl-lib:int-add 3 (f2cl-lib:int-mul 5 n)))
       (setf lwmin
               (f2cl-lib:int-add 1 (f2cl-lib:int-mul 6 n)
                                 (f2cl-lib:int-mul 2 (expt n 2)))))
      (t (setf liwmin 1)
       (setf lwmin (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) 1))))
(setf lopt lwmin)
(setf liopt liwmin)
(cond ((or (< itype 1) (> itype 3)) (setf info -1))
      ((not
        (or wantz
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobz "N")
              (declare (ignore var-1))
              (when var-0 (setf jobz var-0))
              ret-val)))
       (setf info -2))
      ((not
        (or upper
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
       (setf info -3))
      ((< n 0) (setf info -4))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -6))
      ((< ldb$
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -8)))
(cond
 ((= info 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lopt) 'single-float))
  (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) liopt)
  (cond ((and (< lwork lwmin) (not lquery)) (setf info -11))
        ((and (< liwork liwmin) (not lquery)) (setf info -13)))))
(cond
 ((/= info 0) (xerbla "SSYGVD" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (spotrf uplo n b ldb$ info)
  (declare (ignore var-2))
  (setf uplo var-0)
  (setf n var-1)
  (setf ldb$ var-3)
```

```
  (setf info var-4))
(cond
 ((/= info 0) (setf info (f2cl-lib:int-add n info)) (go end_label)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (ssygst itype uplo n a lda b ldb$ info)
  (declare (ignore var-0 var-3 var-5))
  (setf uplo var-1)
  (setf n var-2)
  (setf lda var-4)
  (setf ldb$ var-6)
  (setf info var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ssyevd jobz uplo n a lda w work lwork iwork liwork info)
  (declare (ignore var-3 var-5 var-6 var-7 var-8 var-9))
  (setf jobz var-0)
  (setf uplo var-1)
  (setf n var-2)
  (setf lda var-4)
  (setf info var-10))
(setf lopt
        (f2cl-lib:int
         (max (f2cl-lib:freal lopt)
              (f2cl-lib:freal
               (f2cl-lib:fref work-%data% (1) ((1 *))
                              work-%offset%)))))
(setf liopt
        (f2cl-lib:int
         (max (f2cl-lib:freal liopt)
              (f2cl-lib:freal
               (f2cl-lib:fref iwork-%data% (1) ((1 *))
                              iwork-%offset%)))))
(cond
 ((and wantz (= info 0))
  (cond
   ((or (= itype 1) (= itype 2))
    (cond (upper (f2cl-lib:f2cl-set-string trans "N" (string 1)))
          (t (f2cl-lib:f2cl-set-string trans "T" (string 1))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (strsm "Left" uplo trans "Non-unit" n n one b ldb$ a lda)
      (declare (ignore var-0 var-3 var-7 var-9))
      (when var-1 (setf uplo var-1))
      (when var-2 (setf trans var-2))
      (when var-4 (setf n var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf one var-6))
      (when var-8 (setf ldb$ var-8))
```

```
              (when var-10 (setf lda var-10))))
            ((= itype 3)
             (cond (upper (f2cl-lib:f2cl-set-string trans "T" (string 1)))
                   (t (f2cl-lib:f2cl-set-string trans "N" (string 1))))
             (multiple-value-bind
                 (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                  var-10)
                 (strmm "Left" uplo trans "Non-unit" n n one b ldb$ a lda)
               (declare (ignore var-0 var-3 var-7 var-9))
               (when var-1 (setf uplo var-1))
               (when var-2 (setf trans var-2))
               (when var-4 (setf n var-4))
               (when var-5 (setf n var-5))
               (when var-6 (setf one var-6))
               (when var-8 (setf ldb$ var-8))
               (when var-10 (setf lda var-10)))))))
         (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                 (coerce (the f2cl-lib:integer4 lopt) 'single-float))
         (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) liopt)
         (go end_label)
        end_label
         (return
          (values nil jobz uplo n nil lda nil ldb$ nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssygvd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil fortran-to-lisp::jobz
                                              fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$ nil nil nil
```

```
                                        nil nil fortran-to-lisp::info)
                                      :calls
                                      '(fortran-to-lisp::ssyevd
                                        fortran-to-lisp::ssygst
                                        fortran-to-lisp::spotrf))))
```

———-

57.7 ssygvx

57.7.1 Axiom unit tests

— **ssygvx.input** —

```
)set break resume
)sys rm -f ssygvx.output
)spool ssygvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

57.7.2 Axiom help page

— **ssygvx.help** —

```
======================================================================
ssygvx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssygvx (ITYPE, JOBZ, RANGE, UPLO, N, A, LDA, B, LDB, VL, VU,
           IL, IU, ABSTOL, M, W, Z, LDZ, WORK, LWORK, IWORK, IFAIL, INFO)
```

```
          SSYGVX

Function/Subroutine Documentation
   subroutine ssygvx (integer ITYPE, character JOBZ, character RANGE,
       character UPLO, integer N, real, dimension( lda, * ) A, integer LDA,
       real, dimension( ldb, * ) B, integer LDB, real VL, real VU, integer IL,
       integer IU, real ABSTOL, integer M, real, dimension( * ) W, real,
       dimension( ldz, * ) Z, integer LDZ, real, dimension( * ) WORK, integer
       LWORK, integer, dimension( * ) IWORK, integer, dimension( * ) IFAIL,
       integer INFO)
       SSYGVX

Purpose:
   SSYGVX computes selected eigenvalues, and optionally, eigenvectors
   of a real generalized symmetric-definite eigenproblem, of the form
   A*x=(lambda)*B*x,  A*Bx=(lambda)*x,  or B*A*x=(lambda)*x.  Here A
   and B are assumed to be symmetric and B is also positive definite.
   Eigenvalues and eigenvectors can be selected by specifying either a
   range of values or a range of indices for the desired eigenvalues.

Parameters:

 ITYPE
   ITYPE is INTEGER
   Specifies the problem type to be solved:
   = 1:  A*x = (lambda)*B*x
   = 2:  A*B*x = (lambda)*x
   = 3:  B*A*x = (lambda)*x

 JOBZ
   JOBZ is CHARACTER*1
   = 'N':  Compute eigenvalues only;
   = 'V':  Compute eigenvalues and eigenvectors.

 RANGE
   RANGE is CHARACTER*1
   = 'A': all eigenvalues will be found.
   = 'V': all eigenvalues in the half-open interval (VL,VU] will be found.
   = 'I': the IL-th through IU-th eigenvalues will be found.

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A and B are stored;
   = 'L':  Lower triangle of A and B are stored.

 N
   N is INTEGER
   The order of the matrix pencil (A,B).  N >= 0.

 A
```

```
  A is REAL array, dimension (LDA, N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the
  leading N-by-N upper triangular part of A contains the
  upper triangular part of the matrix A.  If UPLO = 'L',
  the leading N-by-N lower triangular part of A contains
  the lower triangular part of the matrix A.

  On exit, the lower triangle (if UPLO='L') or the upper
  triangle (if UPLO='U') of A, including the diagonal, is destroyed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

B
  B is REAL array, dimension (LDA, N)
  On entry, the symmetric matrix B.  If UPLO = 'U', the
  leading N-by-N upper triangular part of B contains the
  upper triangular part of the matrix B.  If UPLO = 'L',
  the leading N-by-N lower triangular part of B contains
  the lower triangular part of the matrix B.

  On exit, if INFO <= N, the part of B containing the matrix is
  overwritten by the triangular factor U or L from the Cholesky
  factorization B = U**T*U or B = L*L**T.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

VL
  VL is REAL

VU
  VU is REAL
  If RANGE='V', the lower and upper bounds of the interval to
  be searched for eigenvalues. VL < VU.
  Not referenced if RANGE = 'A' or 'I'.

IL
  IL is INTEGER

IU
  IU is INTEGER
  If RANGE='I', the indices (in ascending order) of the
  smallest and largest eigenvalues to be returned.
  1 <= IL <= IU <= N, if N > 0; IL = 1 and IU = 0 if N = 0.
  Not referenced if RANGE = 'A' or 'V'.

ABSTOL
```

```
  ABSTOL is REAL
  The absolute error tolerance for the eigenvalues.
  An approximate eigenvalue is accepted as converged
  when it is determined to lie in an interval [a,b]
  of width less than or equal to

  ABSTOL + EPS *   max( |a|,|b| ) ,

  where EPS is the machine precision.  If ABSTOL is less than
  or equal to zero, then  EPS*|T|  will be used in its place,
  where |T| is the 1-norm of the tridiagonal matrix obtained
  by reducing C to tridiagonal form, where C is the symmetric
  matrix of the standard symmetric problem to which the
  generalized problem is transformed.

  Eigenvalues will be computed most accurately when ABSTOL is
  set to twice the underflow threshold 2*DLAMCH('S'), not zero.
  If this routine returns with INFO>0, indicating that some
  eigenvectors did not converge, try setting ABSTOL to 2*SLAMCH('S').

M
  M is INTEGER
  The total number of eigenvalues found.  0 <= M <= N.
  If RANGE = 'A', M = N, and if RANGE = 'I', M = IU-IL+1.

W
  W is REAL array, dimension (N)
  On normal exit, the first M elements contain the selected
  eigenvalues in ascending order.

Z
  Z is REAL array, dimension (LDZ, max(1,M))
  If JOBZ = 'N', then Z is not referenced.
  If JOBZ = 'V', then if INFO = 0, the first M columns of Z
  contain the orthonormal eigenvectors of the matrix A
  corresponding to the selected eigenvalues, with the i-th
  column of Z holding the eigenvector associated with W(i).
  The eigenvectors are normalized as follows:
  if ITYPE = 1 or 2, Z**T*B*Z = I;
  if ITYPE = 3, Z**T*inv(B)*Z = I.

  If an eigenvector fails to converge, then that column of Z
  contains the latest approximation to the eigenvector, and the
  index of the eigenvector is returned in IFAIL.
  Note: the user must ensure that at least max(1,M) columns are
  supplied in the array Z; if RANGE = 'V', the exact value of M
  is not known in advance and an upper bound must be used.

LDZ
  LDZ is INTEGER
```

```
  The leading dimension of the array Z.  LDZ >= 1, and if
  JOBZ = 'V', LDZ >= max(1,N).

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of the array WORK.  LWORK >= max(1,8*N).
  For optimal efficiency, LWORK >= (NB+3)*N,
  where NB is the blocksize for SSYTRD returned by ILAENV.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (5*N)

IFAIL
  IFAIL is INTEGER array, dimension (N)
  If JOBZ = 'V', then if INFO = 0, the first M elements of
  IFAIL are zero.  If INFO > 0, then IFAIL contains the
  indices of the eigenvectors that failed to converge.
  If JOBZ = 'N', then IFAIL is not referenced.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  SPOTRF or SSYEVX returned an error code:
  <= N:  if INFO = i, SSYEVX failed to converge;
        i eigenvectors failed to converge.  Their indices
        are stored in array IFAIL.
  > N:   if INFO = N + i, for 1 <= i <= N, then the leading
        minor of order i of B is not positive definite.
        The factorization of B could not be completed and
        no eigenvalues or eigenvectors were computed.
```

———-

57.7.3 fortran code

— ssygvx.f —

```
*  =====================================================================
      SUBROUTINE SSYGVX( ITYPE, JOBZ, RANGE, UPLO, N, A, LDA, B, LDB,
     $                   VL, VU, IL, IU, ABSTOL, M, W, Z, LDZ, WORK,
     $                   LWORK, IWORK, IFAIL, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBZ, RANGE, UPLO
      INTEGER            IL, INFO, ITYPE, IU, LDA, LDB, LDZ, LWORK, M, N
      REAL               ABSTOL, VL, VU
*     ..
*     .. Array Arguments ..
      INTEGER            IFAIL( * ), IWORK( * )
      REAL               A( LDA, * ), B( LDB, * ), W( * ), WORK( * ),
     $                   Z( LDZ, * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ALLEIG, INDEIG, LQUERY, UPPER, VALEIG, WANTZ
      CHARACTER          TRANS
      INTEGER            LWKMIN, LWKOPT, NB
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           ILAENV, LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SPOTRF, SSYEVX, SSYGST, STRMM, STRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      UPPER = LSAME( UPLO, 'U' )
      WANTZ = LSAME( JOBZ, 'V' )
      ALLEIG = LSAME( RANGE, 'A' )
```

```
      VALEIG = LSAME( RANGE, 'V' )
      INDEIG = LSAME( RANGE, 'I' )
      LQUERY = ( LWORK.EQ.-1 )
*
      INFO = 0
      IF( ITYPE.LT.1 .OR. ITYPE.GT.3 ) THEN
         INFO = -1
      ELSE IF( .NOT.( WANTZ .OR. LSAME( JOBZ, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( .NOT.( ALLEIG .OR. VALEIG .OR. INDEIG ) ) THEN
         INFO = -3
      ELSE IF( .NOT.( UPPER .OR. LSAME( UPLO, 'L' ) ) ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE
         IF( VALEIG ) THEN
            IF( N.GT.0 .AND. VU.LE.VL )
     $         INFO = -11
         ELSE IF( INDEIG ) THEN
            IF( IL.LT.1 .OR. IL.GT.MAX( 1, N ) ) THEN
               INFO = -12
            ELSE IF( IU.LT.MIN( N, IL ) .OR. IU.GT.N ) THEN
               INFO = -13
            END IF
         END IF
      END IF
      IF (INFO.EQ.0) THEN
         IF (LDZ.LT.1 .OR. (WANTZ .AND. LDZ.LT.N)) THEN
            INFO = -18
         END IF
      END IF
*
      IF( INFO.EQ.0 ) THEN
         LWKMIN = MAX( 1, 8*N )
         NB = ILAENV( 1, 'SSYTRD', UPLO, N, -1, -1, -1 )
         LWKOPT = MAX( LWKMIN, ( NB + 3 )*N )
         WORK( 1 ) = LWKOPT
*
         IF( LWORK.LT.LWKMIN .AND. .NOT.LQUERY ) THEN
            INFO = -20
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYGVX', -INFO )
```

```
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      M = 0
      IF( N.EQ.0 ) THEN
         RETURN
      END IF
*
*     Form a Cholesky factorization of B.
*
      CALL SPOTRF( UPLO, N, B, LDB, INFO )
      IF( INFO.NE.0 ) THEN
         INFO = N + INFO
         RETURN
      END IF
*
*     Transform problem to standard eigenvalue problem and solve.
*
      CALL SSYGST( ITYPE, UPLO, N, A, LDA, B, LDB, INFO )
      CALL SSYEVX( JOBZ, RANGE, UPLO, N, A, LDA, VL, VU, IL, IU, ABSTOL,
     $             M, W, Z, LDZ, WORK, LWORK, IWORK, IFAIL, INFO )
*
      IF( WANTZ ) THEN
*
*        Backtransform eigenvectors to the original problem.
*
         IF( INFO.GT.0 )
     $      M = INFO - 1
         IF( ITYPE.EQ.1 .OR. ITYPE.EQ.2 ) THEN
*
*           For A*x=(lambda)*B*x and A*B*x=(lambda)*x;
*           backtransform eigenvectors: x = inv(L)**T*y or inv(U)*y
*
            IF( UPPER ) THEN
               TRANS = 'N'
            ELSE
               TRANS = 'T'
            END IF
*
            CALL STRSM( 'Left', UPLO, TRANS, 'Non-unit', N, M, ONE, B,
     $                  LDB, Z, LDZ )
*
         ELSE IF( ITYPE.EQ.3 ) THEN
*
*           For B*A*x=(lambda)*x;
*           backtransform eigenvectors: x = L*y or U**T*y
```

```
*
            IF( UPPER ) THEN
               TRANS = 'T'
            ELSE
               TRANS = 'N'
            END IF
*
            CALL STRMM( 'Left', UPLO, TRANS, 'Non-unit', N, M, ONE, B,
     $                  LDB, Z, LDZ )
         END IF
      END IF
*
*     Set WORK(1) to optimal workspace size.
*
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of SSYGVX
*
      END
```

———-

57.7.4 lisp code

— **ssygvx.lisp** —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun ssygvx
         (itype jobz range uplo n a lda b ldb$ vl vu il iu abstol m w z ldz
          work lwork iwork ifail info)
    (declare (type (array f2cl-lib:integer4 (*)) ifail iwork)
             (type (single-float) abstol vu vl)
             (type (array single-float (*)) work z w b a)
             (type (string *) uplo range jobz)
             (type (f2cl-lib:integer4) info lwork ldz m iu il ldb$ lda n
              itype))
    (f2cl-lib:with-multi-array-data
        ((jobz character jobz-%data% jobz-%offset%)
         (range character range-%data% range-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%)
         (w single-float w-%data% w-%offset%)
         (z single-float z-%data% z-%offset%)
```

```
   (work single-float work-%data% work-%offset%)
   (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%)
   (ifail f2cl-lib:integer4 ifail-%data% ifail-%offset%))
(prog ((lwkmin 0) (lwkopt 0) (nb 0)
       (trans
        (make-array '(1) :element-type 'character :initial-element #\ ))
       (alleig nil) (indeig nil) (lquery nil) (upper nil) (valeig nil)
       (wantz nil))
  (declare (type (f2cl-lib:integer4) lwkmin lwkopt nb)
           (type (string 1) trans)
           (type f2cl-lib:logical alleig indeig lquery upper valeig
            wantz))
  (setf upper
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
  (setf wantz
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame jobz "V")
            (declare (ignore var-1))
            (when var-0 (setf jobz var-0))
            ret-val))
  (setf alleig
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame range "A")
            (declare (ignore var-1))
            (when var-0 (setf range var-0))
            ret-val))
  (setf valeig
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame range "V")
            (declare (ignore var-1))
            (when var-0 (setf range var-0))
            ret-val))
  (setf indeig
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame range "I")
            (declare (ignore var-1))
            (when var-0 (setf range var-0))
            ret-val))
  (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
  (setf info 0)
  (cond ((or (< itype 1) (> itype 3)) (setf info -1))
        ((not
          (or wantz
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame jobz "N")
                (declare (ignore var-1))
```

```
                (when var-0 (setf jobz var-0))
                ret-val)))
         (setf info -2))
        ((not (or alleig valeig indeig)) (setf info -3))
        ((not
          (or upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val)))
         (setf info -4))
        ((< n 0) (setf info -5))
        ((< lda
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -7))
        ((< ldb$
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -9))
        (t
         (cond
          (valeig
           (if (and (> n 0) (<= vu vl))
               (setf info -11)))
          (indeig
           (cond
            ((or (< il 1)
                 (> il
                    (max (the f2cl-lib:integer4 1)
                         (the f2cl-lib:integer4 n))))
             (setf info -12))
            ((or
              (< iu
                 (min (the f2cl-lib:integer4 n)
                      (the f2cl-lib:integer4 il)))
              (> iu n))
             (setf info -13)))))))
(cond
 ((= info 0)
  (cond ((or (< ldz 1) (and wantz (< ldz n))) (setf info -18)))))
(cond
 ((= info 0)
  (setf lwkmin
          (max (the f2cl-lib:integer4 1)
               (the f2cl-lib:integer4 (f2cl-lib:int-mul 8 n))))
  (setf nb
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (ilaenv 1 "SSYTRD" uplo n -1 -1 -1)
            (declare (ignore var-0 var-1 var-4 var-5 var-6))
```

```
            (setf uplo var-2)
            (setf n var-3)
            ret-val))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkmin)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-mul (f2cl-lib:int-add nb 3) n))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
  (cond ((and (< lwork lwkmin) (not lquery)) (setf info -20)))))
(cond
 ((/= info 0) (xerbla "SSYGVX" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(setf m 0)
(cond ((= n 0) (go end_label)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (spotrf uplo n b ldb$ info)
  (declare (ignore var-2))
  (setf uplo var-0)
  (setf n var-1)
  (setf ldb$ var-3)
  (setf info var-4))
(cond
 ((/= info 0) (setf info (f2cl-lib:int-add n info)) (go end_label)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (ssygst itype uplo n a lda b ldb$ info)
  (declare (ignore var-0 var-3 var-5))
  (setf uplo var-1)
  (setf n var-2)
  (setf lda var-4)
  (setf ldb$ var-6)
  (setf info var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19)
    (ssyevx jobz range uplo n a lda vl vu il iu abstol m w z ldz work
     lwork iwork ifail info)
  (declare
   (ignore var-4 var-6 var-7 var-8 var-9 var-10 var-12 var-13 var-15
    var-16 var-17 var-18))
  (setf jobz var-0)
  (setf range var-1)
  (setf uplo var-2)
  (setf n var-3)
  (setf lda var-5)
  (setf m var-11)
  (setf ldz var-14)
  (setf info var-19))
(cond
 (wantz
```

```
          (if (> info 0)
              (setf m (f2cl-lib:int-sub info 1)))
          (cond
           ((or (= itype 1) (= itype 2))
            (cond (upper (f2cl-lib:f2cl-set-string trans "N" (string 1)))
                  (t (f2cl-lib:f2cl-set-string trans "T" (string 1))))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                 var-10)
                (strsm "Left" uplo trans "Non-unit" n m one b ldb$ z ldz)
              (declare (ignore var-0 var-3 var-7 var-9))
              (when var-1 (setf uplo var-1))
              (when var-2 (setf trans var-2))
              (when var-4 (setf n var-4))
              (when var-5 (setf m var-5))
              (when var-6 (setf one var-6))
              (when var-8 (setf ldb$ var-8))
              (when var-10 (setf ldz var-10))))
           ((= itype 3)
            (cond (upper (f2cl-lib:f2cl-set-string trans "T" (string 1)))
                  (t (f2cl-lib:f2cl-set-string trans "N" (string 1))))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                 var-10)
                (strmm "Left" uplo trans "Non-unit" n m one b ldb$ z ldz)
              (declare (ignore var-0 var-3 var-7 var-9))
              (when var-1 (setf uplo var-1))
              (when var-2 (setf trans var-2))
              (when var-4 (setf n var-4))
              (when var-5 (setf m var-5))
              (when var-6 (setf one var-6))
              (when var-8 (setf ldb$ var-8))
              (when var-10 (setf ldz var-10)))))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
        (go end_label)
       end_label
        (return
         (values nil jobz range uplo n nil lda nil ldb$ nil nil nil nil nil m
                 nil nil ldz nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssygvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (string) (string) (string)
                                              (fortran-to-lisp::integer4)
```

```
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (single-float) (single-float)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (single-float)
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(nil fortran-to-lisp::jobz
  fortran-to-lisp::range
  fortran-to-lisp::uplo
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$ nil nil nil
  nil nil fortran-to-lisp::m nil
  nil fortran-to-lisp::ldz nil nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::ssyevx
  fortran-to-lisp::ssygst
  fortran-to-lisp::spotrf
  fortran-to-lisp::ilaenv))))
```

Chapter 58

LAPACK - Symmetric Matrix, Comp. Routines, Double

58.1 dla_syamv

58.1.1 Axiom unit tests

— **dlasyamv.input** —

```
)set break resume
)sys rm -f dlasyamv.output
)spool dlasyamv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.1.2 Axiom help page

— **dla_syamv.help** —

```
==========================================================================
dla_syamv examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dla_syamv (UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)
           DLA_SYAMV computes a matrix-vector product using a symmetric
           indefinite matrix to calculate error bounds.

Function/Subroutine Documentation
   subroutine dla_syamv (integer UPLO, integer N, double precision ALPHA,
       double precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( * ) X, integer INCX, double precision BETA, double
       precision, dimension( * ) Y, integer INCY)
       DLA_SYAMV computes a matrix-vector product using a symmetric indefinite
       matrix to calculate error bounds.

Purpose:
   DLA_SYAMV  performs the matrix-vector operation

   y := alpha*abs(A)*abs(x) + beta*abs(y),

   where alpha and beta are scalars, x and y are vectors and A is an
   n by n symmetric matrix.

   This function is primarily used in calculating error bounds.
   To protect against underflow during evaluation, components in
   the resulting vector are perturbed away from zero by (N+1)
   times the underflow threshold.  To prevent unnecessarily large
   errors for block-structure embedded in general matrices,
   "symbolically" zero components are not perturbed.  A zero
   entry is considered "symbolic" if all multiplications involved
   in computing that entry have at least one zero multiplicand.

Parameters:

 UPLO
   UPLO is INTEGER
   On entry, UPLO specifies whether the upper or lower
   triangular part of the array A is to be referenced as
   follows:

   UPLO = BLAS_UPPER   Only the upper triangular part of A
   is to be referenced.

   UPLO = BLAS_LOWER   Only the lower triangular part of A
   is to be referenced.
```

```
  Unchanged on exit.

N
  N is INTEGER
  On entry, N specifies the number of columns of the matrix A.
  N must be at least zero.
  Unchanged on exit.

ALPHA
  ALPHA is DOUBLE PRECISION .
  On entry, ALPHA specifies the scalar alpha.
  Unchanged on exit.

A
  A is DOUBLE PRECISION array of DIMENSION ( LDA, n ).
  Before entry, the leading m by n part of the array A must
  contain the matrix of coefficients.
  Unchanged on exit.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, n ).
  Unchanged on exit.

X
  X is DOUBLE PRECISION array, dimension
  ( 1 + ( n - 1 )*abs( INCX ) )
  Before entry, the incremented array X must contain the vector x.
  Unchanged on exit.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of
  X. INCX must not be zero.
  Unchanged on exit.

BETA
  BETA is DOUBLE PRECISION .
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.
  Unchanged on exit.

Y
  Y is DOUBLE PRECISION array, dimension
  ( 1 + ( n - 1 )*abs( INCY ) )
  Before entry with BETA non-zero, the incremented array Y
  must contain the vector y. On exit, Y is overwritten by the
  updated vector y.
```

```
INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of
  Y. INCY must not be zero.
  Unchanged on exit.
```

58.1.3 fortran code

— dla_syamv.f —

```
*  =====================================================================
      SUBROUTINE DLA_SYAMV( UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y,
     $                      INCY )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   ALPHA, BETA
      INTEGER            INCX, INCY, LDA, N, UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), X( * ), Y( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            SYMB_ZERO
      DOUBLE PRECISION   TEMP, SAFE1
      INTEGER            I, INFO, IY, J, JX, KX, KY
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, DLAMCH
      DOUBLE PRECISION   DLAMCH
*     ..
*     .. External Functions ..
      EXTERNAL           ILAUPLO
      INTEGER            ILAUPLO
```

```
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, ABS, SIGN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF     ( UPLO.NE.ILAUPLO( 'U' ) .AND.
     $         UPLO.NE.ILAUPLO( 'L' ) ) THEN
         INFO = 1
      ELSE IF( N.LT.0 )THEN
         INFO = 2
      ELSE IF( LDA.LT.MAX( 1, N ) )THEN
         INFO = 5
      ELSE IF( INCX.EQ.0 )THEN
         INFO = 7
      ELSE IF( INCY.EQ.0 )THEN
         INFO = 10
      END IF
      IF( INFO.NE.0 )THEN
         CALL XERBLA( 'DSYMV ', INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( ( N.EQ.0 ).OR.( ( ALPHA.EQ.ZERO ).AND.( BETA.EQ.ONE ) ) )
     $   RETURN
*
*     Set up the start points in  X  and  Y.
*
      IF( INCX.GT.0 )THEN
         KX = 1
      ELSE
         KX = 1 - ( N - 1 )*INCX
      END IF
      IF( INCY.GT.0 )THEN
         KY = 1
      ELSE
         KY = 1 - ( N - 1 )*INCY
      END IF
*
*     Set SAFE1 essentially to be the underflow threshold times the
*     number of additions in each row.
*
      SAFE1 = DLAMCH( 'Safe minimum' )
      SAFE1 = (N+1)*SAFE1
*
```

```
*     Form  y := alpha*abs(A)*abs(x) + beta*abs(y).
*
*     The O(N^2) SYMB_ZERO tests could be replaced by O(N) queries to
*     the inexact flag.  Still doesn't help change the iteration order
*     to per-column.
*
      IY = KY
      IF ( INCX.EQ.1 ) THEN
         IF ( UPLO .EQ. ILAUPLO( 'U' ) ) THEN
            DO I = 1, N
               IF ( BETA .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
                  Y( IY ) = 0.0D+0
               ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
               ELSE
                  SYMB_ZERO = .FALSE.
                  Y( IY ) = BETA * ABS( Y( IY ) )
               END IF
               IF ( ALPHA .NE. ZERO ) THEN
                  DO J = 1, I
                     TEMP = ABS( A( J, I ) )
                     SYMB_ZERO = SYMB_ZERO .AND.
     $                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                     Y( IY ) = Y( IY ) + ALPHA*ABS( X( J ) )*TEMP
                  END DO
                  DO J = I+1, N
                     TEMP = ABS( A( I, J ) )
                     SYMB_ZERO = SYMB_ZERO .AND.
     $                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                     Y( IY ) = Y( IY ) + ALPHA*ABS( X( J ) )*TEMP
                  END DO
               END IF

               IF ( .NOT.SYMB_ZERO )
     $              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

               IY = IY + INCY
            END DO
         ELSE
            DO I = 1, N
               IF ( BETA .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
                  Y( IY ) = 0.0D+0
               ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
               ELSE
                  SYMB_ZERO = .FALSE.
```

```
            Y( IY ) = BETA * ABS( Y( IY ) )
         END IF
         IF ( ALPHA .NE. ZERO ) THEN
            DO J = 1, I
               TEMP = ABS( A( I, J ) )
               SYMB_ZERO = SYMB_ZERO .AND.
$                   ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

               Y( IY ) = Y( IY ) + ALPHA*ABS( X( J ) )*TEMP
            END DO
            DO J = I+1, N
               TEMP = ABS( A( J, I ) )
               SYMB_ZERO = SYMB_ZERO .AND.
$                   ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

               Y( IY ) = Y( IY ) + ALPHA*ABS( X( J ) )*TEMP
            END DO
         END IF

         IF ( .NOT.SYMB_ZERO )
$              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

         IY = IY + INCY
      END DO
   END IF
 ELSE
   IF ( UPLO .EQ. ILAUPLO( 'U' ) ) THEN
      DO I = 1, N
         IF ( BETA .EQ. ZERO ) THEN
            SYMB_ZERO = .TRUE.
            Y( IY ) = 0.0D+0
         ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
            SYMB_ZERO = .TRUE.
         ELSE
            SYMB_ZERO = .FALSE.
            Y( IY ) = BETA * ABS( Y( IY ) )
         END IF
         JX = KX
         IF ( ALPHA .NE. ZERO ) THEN
            DO J = 1, I
               TEMP = ABS( A( J, I ) )
               SYMB_ZERO = SYMB_ZERO .AND.
$                   ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

               Y( IY ) = Y( IY ) + ALPHA*ABS( X( JX ) )*TEMP
               JX = JX + INCX
            END DO
            DO J = I+1, N
               TEMP = ABS( A( I, J ) )
               SYMB_ZERO = SYMB_ZERO .AND.
```

```
$                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                  Y( IY ) = Y( IY ) + ALPHA*ABS( X( JX ) )*TEMP
                  JX = JX + INCX
               END DO
            END IF

            IF ( .NOT.SYMB_ZERO )
$              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

            IY = IY + INCY
         END DO
      ELSE
         DO I = 1, N
            IF ( BETA .EQ. ZERO ) THEN
               SYMB_ZERO = .TRUE.
               Y( IY ) = 0.0D+0
            ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
               SYMB_ZERO = .TRUE.
            ELSE
               SYMB_ZERO = .FALSE.
               Y( IY ) = BETA * ABS( Y( IY ) )
            END IF
            JX = KX
            IF ( ALPHA .NE. ZERO ) THEN
               DO J = 1, I
                  TEMP = ABS( A( I, J ) )
                  SYMB_ZERO = SYMB_ZERO .AND.
$                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                  Y( IY ) = Y( IY ) + ALPHA*ABS( X( JX ) )*TEMP
                  JX = JX + INCX
               END DO
               DO J = I+1, N
                  TEMP = ABS( A( J, I ) )
                  SYMB_ZERO = SYMB_ZERO .AND.
$                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                  Y( IY ) = Y( IY ) + ALPHA*ABS( X( JX ) )*TEMP
                  JX = JX + INCX
               END DO
            END IF

            IF ( .NOT.SYMB_ZERO )
$              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

            IY = IY + INCY
         END DO
      END IF
```

```
      END IF
*
      RETURN
*
*     End of DLA_SYAMV
*
      END
```

———-

58.1.4 lisp code

— **dla_syamv.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dla_syamv (uplo n alpha a lda x incx beta y incy)
    (declare (type (array double-float (*)) y x a)
             (type (double-float) beta alpha)
             (type (f2cl-lib:integer4) incy incx lda n uplo))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (x double-float x-%data% x-%offset%)
         (y double-float y-%data% y-%offset%))
      (prog ((i 0) (info 0) (iy 0) (j 0) (jx 0) (kx 0) (ky 0) (temp 0.0d0)
             (safe1 0.0d0) (symb_zero nil))
        (declare (type (f2cl-lib:integer4) i info iy j jx kx ky)
                 (type (double-float) temp safe1)
                 (type f2cl-lib:logical symb_zero))
        (setf info 0)
        (cond
         ((and (/= uplo (ilauplo "U")) (/= uplo (ilauplo "L"))) (setf info 1))
         ((< n 0) (setf info 2))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info 5))
         ((= incx 0) (setf info 7)) ((= incy 0) (setf info 10)))
        (cond
         ((/= info 0)
          (multiple-value-bind (var-0 var-1)
              (xerbla "DSYMV " info)
            (declare (ignore var-0))
            (when var-1 (setf info var-1)))
          (go end_label)))
        (if (or (= n 0) (and (= alpha zero) (= beta one)))
            (go end_label))
        (cond ((> incx 0) (setf kx 1))
```

```
      (t
       (setf kx
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incx)))))
(cond ((> incy 0) (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incy)))))
(setf safe1 (dlamch "Safe minimum"))
(setf safe1 (* (f2cl-lib:int-add n 1) safe1))
(setf iy ky)
(cond
 ((= incx 1)
  (cond
   ((= uplo (ilauplo "U"))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (cond
         ((= beta zero) (setf symb_zero f2cl-lib:%true%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                0.0d0))
         ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
          (setf symb_zero f2cl-lib:%true%))
         (t (setf symb_zero f2cl-lib:%false%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                (* beta
                   (abs
                    (f2cl-lib:fref y-%data% (iy) ((1 *))
                                   y-%offset%))))))
        (cond
         ((/= alpha zero)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j i) nil)
            (tagbody
              (setf temp
                      (abs
                       (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                      a-%offset%)))
              (setf symb_zero
                      (and symb_zero
                           (or
                            (=
                             (f2cl-lib:fref x-%data% (j) ((1 *))
                                            x-%offset%)
                             zero)
                            (= temp zero))))
```

```
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (* alpha
                            (abs
                             (f2cl-lib:fref x-%data% (j) ((1 *))
                                            x-%offset%))
                            temp)))
             label100001))
          (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                         (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf temp
                      (abs
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)))
              (setf symb_zero
                      (and symb_zero
                           (or
                            (=
                             (f2cl-lib:fref x-%data% (j) ((1 *))
                                            x-%offset%)
                             zero)
                            (= temp zero))))
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (+
                       (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (* alpha
                          (abs
                           (f2cl-lib:fref x-%data% (j) ((1 *))
                                          x-%offset%))
                          temp)))
             label100002))))
          (if (not symb_zero)
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (f2cl-lib:sign safe1
                                        (f2cl-lib:fref y-%data% (iy)
                                                       ((1 *))
                                                       y-%offset%)))))
          (setf iy (f2cl-lib:int-add iy incy))
         label100000)))
      (t
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (cond
            ((= beta zero) (setf symb_zero f2cl-lib:%true%)
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
```

```
          0.0d0))
 ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
  (setf symb_zero f2cl-lib:%true%))
 (t (setf symb_zero f2cl-lib:%false%)
  (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
          (* beta
             (abs
              (f2cl-lib:fref y-%data% (iy) ((1 *))
                             y-%offset%))))))
(cond
 ((/= alpha zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j i) nil)
    (tagbody
      (setf temp
              (abs
               (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                              a-%offset%)))
      (setf symb_zero
              (and symb_zero
                   (or
                    (=
                     (f2cl-lib:fref x-%data% (j) ((1 *))
                                    x-%offset%)
                     zero)
                    (= temp zero))))
      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
              (+
               (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
               (* alpha
                  (abs
                   (f2cl-lib:fref x-%data% (j) ((1 *))
                                  x-%offset%))
                  temp)))
     label100004))
  (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf temp
              (abs
               (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                              a-%offset%)))
      (setf symb_zero
              (and symb_zero
                   (or
                    (=
                     (f2cl-lib:fref x-%data% (j) ((1 *))
                                    x-%offset%)
                     zero)
```

```
                              (= temp zero))))
                 (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (+
                          (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                          (* alpha
                             (abs
                              (f2cl-lib:fref x-%data% (j) ((1 *))
                                             x-%offset%))
                             temp)))
                label100005))))
          (if (not symb_zero)
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (f2cl-lib:sign safe1
                                        (f2cl-lib:fref y-%data% (iy)
                                                       ((1 *))
                                                       y-%offset%)))))
          (setf iy (f2cl-lib:int-add iy incy))
         label100003)))))
(t
 (cond
  ((= uplo (ilauplo "U"))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (cond
        ((= beta zero) (setf symb_zero f2cl-lib:%true%)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 0.0d0))
        ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
         (setf symb_zero f2cl-lib:%true%))
        (t (setf symb_zero f2cl-lib:%false%)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (* beta
                    (abs
                     (f2cl-lib:fref y-%data% (iy) ((1 *))
                                    y-%offset%))))))
       (setf jx kx)
       (cond
        ((/= alpha zero)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j i) nil)
           (tagbody
             (setf temp
                     (abs
                      (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                     a-%offset%)))
             (setf symb_zero
                     (and symb_zero
                          (or
```

```
                        (=
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%)
                         zero)
                        (= temp zero))))
        (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                (+
                 (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (* alpha
                    (abs
                     (f2cl-lib:fref x-%data% (jx) ((1 *))
                                    x-%offset%))
                    temp)))
        (setf jx (f2cl-lib:int-add jx incx))
      label100007))
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                   (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp
                (abs
                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)))
        (setf symb_zero
                (and symb_zero
                     (or
                      (=
                       (f2cl-lib:fref x-%data% (j) ((1 *))
                                      x-%offset%)
                       zero)
                      (= temp zero))))
        (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                (+
                 (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (* alpha
                    (abs
                     (f2cl-lib:fref x-%data% (jx) ((1 *))
                                    x-%offset%))
                    temp)))
        (setf jx (f2cl-lib:int-add jx incx))
      label100008))))
  (if (not symb_zero)
      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
              (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (f2cl-lib:sign safe1
                                (f2cl-lib:fref y-%data% (iy)
                                               ((1 *))
                                               y-%offset%)))))
  (setf iy (f2cl-lib:int-add iy incy))
 label100006)))
```

```
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (cond
      ((= beta zero) (setf symb_zero f2cl-lib:%true%)
       (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
               0.0d0))
      ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
       (setf symb_zero f2cl-lib:%true%))
      (t (setf symb_zero f2cl-lib:%false%)
       (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
               (* beta
                  (abs
                   (f2cl-lib:fref y-%data% (iy) ((1 *))
                                  y-%offset%))))))
     (setf jx kx)
     (cond
      ((/= alpha zero)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j i) nil)
         (tagbody
           (setf temp
                   (abs
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%)))
           (setf symb_zero
                   (and symb_zero
                        (or
                         (=
                          (f2cl-lib:fref x-%data% (j) ((1 *))
                                         x-%offset%)
                          zero)
                         (= temp zero))))
           (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                   (+
                    (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (* alpha
                       (abs
                        (f2cl-lib:fref x-%data% (jx) ((1 *))
                                       x-%offset%))
                       temp)))
           (setf jx (f2cl-lib:int-add jx incx))
          label100010))
       (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                      (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (setf temp
                   (abs
```

```
                              (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                             a-%offset%)))
                      (setf symb_zero
                              (and symb_zero
                                   (or
                                    (=
                                     (f2cl-lib:fref x-%data% (j) ((1 *))
                                                    x-%offset%)
                                     zero)
                                    (= temp zero))))
                      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                              (+
                               (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                               (* alpha
                                  (abs
                                   (f2cl-lib:fref x-%data% (jx) ((1 *))
                                                  x-%offset%))
                                  temp)))
                      (setf jx (f2cl-lib:int-add jx incx))
                     label100011))))
                (if (not symb_zero)
                    (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                            (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                               (f2cl-lib:sign safe1
                                              (f2cl-lib:fref y-%data% (iy)
                                                             ((1 *))
                                                             y-%offset%)))))
                (setf iy (f2cl-lib:int-add iy incy))
               label100009))))))
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dla_syamv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (double-float)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
```

```
                                        '(nil nil nil nil nil nil nil nil
                                          nil nil)
                                        :calls 'nil)))
```

———-

58.2 dla_syrcond

58.2.1 Axiom unit tests

— **dlasyrcond.input** —

```
)set break resume
)sys rm -f dlasyrcond.output
)spool dlasyrcond.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.2.2 Axiom help page

— **dla_syrcond.help** —

```
====================================================================
dla_syrcond examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function dla_syrcond (UPLO, N, A, LDA, AF, LDAF, IPIV,
           CMODE, C, INFO, WORK, IWORK)
           DLA_SYRCOND estimates the Skeel condition number for a symmetric
           indefinite matrix.
```

```
Function/Subroutine Documentation
   double precision function dla_syrcond (character UPLO, integer N, double
       precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV,
       integer CMODE, double precision, dimension( * ) C, integer INFO, double
       precision, dimension( * ) WORK, integer, dimension( * ) IWORK)
       DLA_SYRCOND estimates the Skeel condition number for a symmetric
       indefinite matrix.

Purpose:
   DLA_SYRCOND estimates the Skeel condition number of  op(A) * op2(C)
   where op2 is determined by CMODE as follows
   CMODE =  1    op2(C) = C
   CMODE =  0    op2(C) = I
   CMODE = -1    op2(C) = inv(C)
   The Skeel condition number cond(A) = norminf( |inv(A)||A| )
   is computed by computing scaling factors R such that
   diag(R)*A*op2(C) is row equilibrated and computing the standard
   infinity-norm condition number.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is DOUBLE PRECISION array, dimension (LDAF,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by DSYTRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 IPIV
```

```
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by DSYTRF.

CMODE
  CMODE is INTEGER
  Determines op2(C) in the formula op(A) * op2(C) as follows:
  CMODE =  1    op2(C) = C
  CMODE =  0    op2(C) = I
  CMODE = -1    op2(C) = inv(C)

C
  C is DOUBLE PRECISION array, dimension (N)
  The vector C in the formula op(A) * op2(C).

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  i > 0:  The ith argument is invalid.

WORK
  WORK is DOUBLE PRECISION array, dimension (3*N).
  Workspace.

IWORK
  IWORK is INTEGER array, dimension (N).
  Workspace.
```

58.2.3 fortran code

— dla_syrcond.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION DLA_SYRCOND( UPLO, N, A, LDA, AF, LDAF,
     $                                       IPIV, CMODE, C, INFO, WORK,
     $                                       IWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            N, LDA, LDAF, INFO, CMODE
```

```
*     ..
*     .. Array Arguments
      INTEGER            IWORK( * ), IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * ), WORK( * ), C( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      CHARACTER          NORMIN
      INTEGER            KASE, I, J
      DOUBLE PRECISION   AINVNM, SMLNUM, TMP
      LOGICAL            UP
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, IDAMAX, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLACN2, DLATRS, DRSCL, XERBLA, DSYTRS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
      DLA_SYRCOND = 0.0D+0
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLA_SYRCOND', -INFO )
         RETURN
      END IF
      IF( N.EQ.0 ) THEN
         DLA_SYRCOND = 1.0D+0
         RETURN
      END IF
      UP = .FALSE.
```

```
      IF ( LSAME( UPLO, 'U' ) ) UP = .TRUE.
*
*     Compute the equilibration matrix R such that
*     inv(R)*A*C has unit 1-norm.
*
      IF ( UP ) THEN
         DO I = 1, N
            TMP = 0.0D+0
            IF ( CMODE .EQ. 1 ) THEN
               DO J = 1, I
                  TMP = TMP + ABS( A( J, I ) * C( J ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + ABS( A( I, J ) * C( J ) )
               END DO
            ELSE IF ( CMODE .EQ. 0 ) THEN
               DO J = 1, I
                  TMP = TMP + ABS( A( J, I ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + ABS( A( I, J ) )
               END DO
            ELSE
               DO J = 1, I
                  TMP = TMP + ABS( A( J, I ) / C( J ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + ABS( A( I, J ) / C( J ) )
               END DO
            END IF
            WORK( 2*N+I ) = TMP
         END DO
      ELSE
         DO I = 1, N
            TMP = 0.0D+0
            IF ( CMODE .EQ. 1 ) THEN
               DO J = 1, I
                  TMP = TMP + ABS( A( I, J ) * C( J ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + ABS( A( J, I ) * C( J ) )
               END DO
            ELSE IF ( CMODE .EQ. 0 ) THEN
               DO J = 1, I
                  TMP = TMP + ABS( A( I, J ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + ABS( A( J, I ) )
               END DO
            ELSE
```

```
               DO J = 1, I
                  TMP = TMP + ABS( A( I, J) / C( J ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + ABS( A( J, I) / C( J ) )
               END DO
            END IF
            WORK( 2*N+I ) = TMP
         END DO
      ENDIF
*
*     Estimate the norm of inv(op(A)).
*
      SMLNUM = DLAMCH( 'Safe minimum' )
      AINVNM = 0.0D+0
      NORMIN = 'N'

      KASE = 0
   10 CONTINUE
      CALL DLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * WORK( 2*N+I )
            END DO

            IF ( UP ) THEN
               CALL DSYTRS( 'U', N, 1, AF, LDAF, IPIV, WORK, N, INFO )
            ELSE
               CALL DSYTRS( 'L', N, 1, AF, LDAF, IPIV, WORK, N, INFO )
            ENDIF
*
*           Multiply by inv(C).
*
            IF ( CMODE .EQ. 1 ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) / C( I )
               END DO
            ELSE IF ( CMODE .EQ. -1 ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF
         ELSE
*
*           Multiply by inv(C**T).
*
```

```
            IF ( CMODE .EQ. 1 ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) / C( I )
               END DO
            ELSE IF ( CMODE .EQ. -1 ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF

            IF ( UP ) THEN
               CALL DSYTRS( 'U', N, 1, AF, LDAF, IPIV, WORK, N, INFO )
            ELSE
               CALL DSYTRS( 'L', N, 1, AF, LDAF, IPIV, WORK, N, INFO )
            ENDIF
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * WORK( 2*N+I )
            END DO
         END IF
*
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM .NE. 0.0D+0 )
     $   DLA_SYRCOND = ( 1.0D+0 / AINVNM )
*
      RETURN
*
      END
```

———-

58.2.4 lisp code

— **dla_syrcond.lisp** —

```
(defun dla_syrcond (uplo n a lda af ldaf ipiv cmode c info work iwork)
  (declare (type (array f2cl-lib:integer4 (*)) iwork ipiv)
           (type (array double-float (*)) work c af a)
           (type (f2cl-lib:integer4) info cmode ldaf lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
```

```
    (a double-float a-%data% a-%offset%)
    (af double-float af-%data% af-%offset%)
    (c double-float c-%data% c-%offset%)
    (work double-float work-%data% work-%offset%)
    (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
    (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4)) (up nil)
         (ainvnm 0.0d0) (smlnum 0.0d0) (tmp 0.0d0) (kase 0) (i 0) (j 0)
         (normin
          (make-array '(1) :element-type 'character :initial-element #\ ))
         (dla_syrcond 0.0d0))
    (declare (type (string 1) normin)
             (type (f2cl-lib:integer4) j i kase)
             (type (double-float) dla_syrcond tmp smlnum ainvnm)
             (type f2cl-lib:logical up)
             (type (array f2cl-lib:integer4 (3)) isave))
    (setf dla_syrcond 0.0d0)
    (setf info 0)
    (cond ((< n 0) (setf info -2))
          ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
           (setf info -4))
          ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
           (setf info -6)))
    (cond
     ((/= info 0) (xerbla "DLA_SYRCOND" (f2cl-lib:int-sub info))
      (go end_label)))
    (cond ((= n 0) (setf dla_syrcond 1.0d0) (go end_label)))
    (setf up f2cl-lib:%false%)
    (if (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)
        (setf up f2cl-lib:%true%))
    (cond
     (up
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf tmp 0.0d0)
          (cond
           ((= cmode 1)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j i) nil)
              (tagbody
                (setf tmp
                        (+ tmp
                           (abs
                            (*
                             (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
```

```
                                  a-%offset%)
                     (f2cl-lib:fref c-%data% (j) ((1 *))
                                  c-%offset%)))))
    label100001))
  (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf tmp
              (+ tmp
                 (abs
                  (*
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                   (f2cl-lib:fref c-%data% (j) ((1 *))
                                  c-%offset%)))))
    label100002)))
 ((= cmode 0)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j i) nil)
    (tagbody
      (setf tmp
              (+ tmp
                 (abs
                  (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                 a-%offset%))))
    label100003))
  (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf tmp
              (+ tmp
                 (abs
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%))))
    label100004)))
 (t
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j i) nil)
    (tagbody
      (setf tmp
              (+ tmp
                 (abs
                  (/
                   (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                  a-%offset%)
                   (f2cl-lib:fref c-%data% (j) ((1 *))
                                  c-%offset%)))))
    label100005))
  (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                ((> j n) nil)
```

```
            (tagbody
              (setf tmp
                      (+ tmp
                         (abs
                          (/
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%)
                           (f2cl-lib:fref c-%data% (j) ((1 *))
                                          c-%offset%)))))
             label100006))))
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) i))
                             ((1 *)) work-%offset%)
              tmp)
      label100000)))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf tmp 0.0d0)
       (cond
        ((= cmode 1)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j i) nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (abs
                         (*
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref c-%data% (j) ((1 *))
                                         c-%offset%)))))
            label100008))
         (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (abs
                         (*
                          (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref c-%data% (j) ((1 *))
                                         c-%offset%)))))
            label100009)))
        ((= cmode 0)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j i) nil)
           (tagbody
```

```
              (setf tmp
                      (+ tmp
                         (abs
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%))))
             label100010))
           (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (setf tmp
                       (+ tmp
                          (abs
                           (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                          a-%offset%))))
             label100011)))
          (t
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j i) nil)
             (tagbody
               (setf tmp
                       (+ tmp
                          (abs
                           (/
                            (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                           a-%offset%)
                            (f2cl-lib:fref c-%data% (j) ((1 *))
                                           c-%offset%)))))
             label100012))
           (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (setf tmp
                       (+ tmp
                          (abs
                           (/
                            (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                           a-%offset%)
                            (f2cl-lib:fref c-%data% (j) ((1 *))
                                           c-%offset%)))))
             label100013))))
         (setf (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) i))
                              ((1 *)) work-%offset%)
               tmp)
       label100007))))
  (setf smlnum (dlamch "Safe minimum"))
  (setf ainvnm 0.0d0)
  (f2cl-lib:f2cl-set-string normin "N" (string 1))
  (setf kase 0)
 label10
```

```
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacn2 n
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1)) ((1 *))
                           work-%offset%)
     work iwork ainvnm kase isave)
  (declare (ignore var-1 var-2 var-3 var-6))
  (setf n var-0)
  (setf ainvnm var-4)
  (setf kase var-5))
(cond
 ((/= kase 0)
  (cond
   ((= kase 2)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add
                                    (f2cl-lib:int-mul 2 n) i))
                                  ((1 *)) work-%offset%)))
       label100014))
    (cond
     (up
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dsytrs "U" n 1 af ldaf ipiv work n info)
        (declare (ignore var-0 var-2 var-3 var-5 var-6))
        (when var-1 (setf n var-1))
        (when var-4 (setf ldaf var-4))
        (when var-7 (setf n var-7))
        (when var-8 (setf info var-8))))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dsytrs "L" n 1 af ldaf ipiv work n info)
        (declare (ignore var-0 var-2 var-3 var-5 var-6))
        (when var-1 (setf n var-1))
        (when var-4 (setf ldaf var-4))
        (when var-7 (setf n var-7))
        (when var-8 (setf info var-8)))))
    (cond
     ((= cmode 1)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (/
                   (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
```

```
                 (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
        label100015)))
    ((= cmode (f2cl-lib:int-sub 1))
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (*
                  (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
        label100016)))))
  (t
   (cond
    ((= cmode 1)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (/
                  (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
        label100017)))
    ((= cmode (f2cl-lib:int-sub 1))
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (*
                  (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
        label100018))))
   (cond
    (up
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (dsytrs "U" n 1 af ldaf ipiv work n info)
       (declare (ignore var-0 var-2 var-3 var-5 var-6))
       (when var-1 (setf n var-1))
       (when var-4 (setf ldaf var-4))
       (when var-7 (setf n var-7))
       (when var-8 (setf info var-8))))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (dsytrs "L" n 1 af ldaf ipiv work n info)
       (declare (ignore var-0 var-2 var-3 var-5 var-6))
       (when var-1 (setf n var-1))
       (when var-4 (setf ldaf var-4))
       (when var-7 (setf n var-7))
       (when var-8 (setf info var-8)))))
```

```
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                      (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                         (f2cl-lib:fref work-%data%
                                        ((f2cl-lib:int-add
                                          (f2cl-lib:int-mul 2 n) i))
                                        ((1 *)) work-%offset%)))
             label100019))))
        (go label10)))
      (if (/= ainvnm 0.0d0)
          (setf dla_syrcond (/ 1.0d0 ainvnm)))
      (go end_label)
     end_label
      (return
       (values dla_syrcond uplo n nil nil nil ldaf nil nil nil info nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dla_syrcond
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*)))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::ldaf nil nil nil
                                              fortran-to-lisp::info nil nil)
                                            :calls '(fortran-to-lisp::dlacn2))))
```

58.3 dla_syrfsx_extended

58.3.1 Axiom unit tests

— **dlasyrfsxextended.input** —

```
)set break resume
)sys rm -f dlasyrfsxextended.output
)spool dlasyrfsxextended.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.3.2 Axiom help page [Demm05]

— **dla_syrfsx_extended.help** —

```
==================================================================
dla_syrfsx_extended examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dla_syrfsx_extended (PREC_TYPE, UPLO, N, NRHS, A, LDA, AF,
           LDAF, IPIV, COLEQU, C, B, LDB, Y, LDY, BERR_OUT, N_NORMS,
           ERR_BNDS_NORM, ERR_BNDS_COMP, RES, AYB, DY, Y_TAIL, RCOND, ITHRESH,
           RTHRESH, DZ_UB, IGNORE_CWISE, INFO)
           DLA_SYRFSX_EXTENDED improves the computed solution to a system of
           linear equations for symmetric indefinite matrices by performing
           extra-precise iterative refinement and provides error bounds and
           backward error estimates for the solution.

Function/Subroutine Documentation
   subroutine dla_syrfsx_extended (integer PREC_TYPE, character UPLO, integer
       N, integer NRHS, double precision, dimension( lda, * ) A, integer LDA,
       double precision, dimension( ldaf, * ) AF, integer LDAF, integer,
       dimension( * ) IPIV, logical COLEQU, double precision, dimension( * )
```

```
      C, double precision, dimension( ldb, * ) B, integer LDB, double
      precision, dimension( ldy, * ) Y, integer LDY, double precision,
      dimension( * ) BERR_OUT, integer N_NORMS, double precision, dimension(
      nrhs, * ) ERR_BNDS_NORM, double precision, dimension( nrhs, * )
      ERR_BNDS_COMP, double precision, dimension( * ) RES, double precision,
      dimension( * ) AYB, double precision, dimension( * ) DY, double
      precision, dimension( * ) Y_TAIL, double precision RCOND, integer
      ITHRESH, double precision RTHRESH, double precision DZ_UB, logical
      IGNORE_CWISE, integer INFO)
      DLA_SYRFSX_EXTENDED improves the computed solution to a system of
      linear equations for symmetric indefinite matrices by performing extra-
      precise iterative refinement and provides error bounds and backward
      error estimates for the solution.

Purpose:
   DLA_SYRFSX_EXTENDED improves the computed solution to a system of
   linear equations by performing extra-precise iterative refinement
   and provides error bounds and backward error estimates for the solution.
   This subroutine is called by DSYRFSX to perform iterative refinement.
   In addition to normwise error bound, the code provides maximum
   componentwise error bound if possible. See comments for ERR_BNDS_NORM
   and ERR_BNDS_COMP for details of the error bounds. Note that this
   subroutine is only resonsible for setting the second fields of
   ERR_BNDS_NORM and ERR_BNDS_COMP.

Parameters:

 PREC_TYPE
   PREC_TYPE is INTEGER
   Specifies the intermediate precision to be used in refinement.
   The value is defined by ILAPREC(P) where P is a CHARACTER and
   P    = 'S':  Single
        = 'D':  Double
        = 'I':  Indigenous
        = 'X', 'E':  Extra

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right-hand-sides, i.e., the number of columns of the
   matrix B.
```

```
A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the N-by-N matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is DOUBLE PRECISION array, dimension (LDAF,N)
  The block diagonal matrix D and the multipliers used to
  obtain the factor U or L as computed by DSYTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by DSYTRF.

COLEQU
  COLEQU is LOGICAL
  If .TRUE. then column equilibration was done to A before calling
  this routine. This is needed to compute the solution and error
  bounds correctly.

C
  C is DOUBLE PRECISION array, dimension (N)
  The column scale factors for A. If COLEQU = .FALSE., C
  is not accessed. If C is input, each element of C should be a power
  of the radix to ensure a reliable solution and error estimates.
  Scaling by powers of the radix does not cause rounding errors unless
  the result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  The right-hand-side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

Y
  Y is DOUBLE PRECISION array, dimension (LDY,NRHS)
  On entry, the solution matrix X, as computed by DSYTRS.
  On exit, the improved solution matrix Y.
```

```
LDY
  LDY is INTEGER
  The leading dimension of the array Y.  LDY >= max(1,N).

BERR_OUT
  BERR_OUT is DOUBLE PRECISION array, dimension (NRHS)
  On exit, BERR_OUT(j) contains the componentwise relative backward
  error for right-hand-side j from the formula
  max(i) ( abs(RES(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
  where abs(Z) is the componentwise absolute value of the matrix
  or vector Z. This is computed by DLA_LIN_BERR.

N_NORMS
  N_NORMS is INTEGER
  Determines which error bounds to return (see ERR_BNDS_NORM
  and ERR_BNDS_COMP).
  If N_NORMS >= 1 return normwise error bounds.
  If N_NORMS >= 2 return componentwise error bounds.

ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
           max_j (abs(XTRUE(j,i) - X(j,i)))
            ------------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.
```

```
  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

ERR_BNDS_COMP
  ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                 abs(XTRUE(j,i) - X(j,i))
            max_j ----------------------
                        abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
  almost certainly within a factor of 10 of the true error
  so long as the next entry is greater than the threshold
  sqrt(n) * slamch('Epsilon'). This error bound should only
  be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
  reciprocal condition number.  Compared with the threshold
  sqrt(n) * slamch('Epsilon') to determine if the error
  estimate is "guaranteed". These reciprocal condition
  numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
```

```
  appropriately scaled matrix Z.
  Let Z = S*(A*diag(x)), where x is the solution for the
  current right-hand side and S scales each row of
  A*diag(x) by a power of the radix so all absolute row
  sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

RES
  RES is DOUBLE PRECISION array, dimension (N)
  Workspace to hold the intermediate residual.

AYB
  AYB is DOUBLE PRECISION array, dimension (N)
  Workspace. This can be the same workspace passed for Y_TAIL.

DY
  DY is DOUBLE PRECISION array, dimension (N)
  Workspace to hold the intermediate solution.

Y_TAIL
  Y_TAIL is DOUBLE PRECISION array, dimension (N)
  Workspace to hold the trailing bits of the intermediate solution.

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

ITHRESH
  ITHRESH is INTEGER
  The maximum number of residual computations allowed for
  refinement. The default is 10. For 'aggressive' set to 100 to
  permit convergence using approximate factorizations or
  factorizations other than LU. If the factorization uses a
  technique other than Gaussian elimination, the guarantees in
  ERR_BNDS_NORM and ERR_BNDS_COMP may no longer be trustworthy.

RTHRESH
  RTHRESH is DOUBLE PRECISION
  Determines when to stop refinement if the error estimate stops
  decreasing. Refinement will stop when the next solution no longer
  satisfies norm(dx_{i+1}) < RTHRESH * norm(dx_i) where norm(Z) is
  the infinity norm of Z. RTHRESH satisfies 0 < RTHRESH <= 1. The
  default value is 0.5. For 'aggressive' set to 0.9 to permit
  convergence on extremely ill-conditioned matrices. See LAWN 165
```

```
  for more details.

 DZ_UB
   DZ_UB is DOUBLE PRECISION
   Determines when to start considering componentwise convergence.
   Componentwise convergence is only considered after each component
   of the solution Y is stable, which we definte as the relative
   change in each component being less than DZ_UB. The default value
   is 0.25, requiring the first bit to be stable. See LAWN 165 for
   more details.

 IGNORE_CWISE
   IGNORE_CWISE is LOGICAL
   If .TRUE. then ignore componentwise convergence. Default value
   is .FALSE..

 INFO
   INFO is INTEGER
   = 0:  Successful exit.
   < 0:  if INFO = -i, the ith argument to DLA_SYRFSX_EXTENDED had an illegal
         value
```

58.3.3 fortran code

— dla_syrfsx_extended.f —

```
*  =====================================================================
      SUBROUTINE DLA_SYRFSX_EXTENDED( PREC_TYPE, UPLO, N, NRHS, A, LDA,
     $                                AF, LDAF, IPIV, COLEQU, C, B, LDB,
     $                                Y, LDY, BERR_OUT, N_NORMS,
     $                                ERR_BNDS_NORM, ERR_BNDS_COMP, RES,
     $                                AYB, DY, Y_TAIL, RCOND, ITHRESH,
     $                                RTHRESH, DZ_UB, IGNORE_CWISE,
     $                                INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDAF, LDB, LDY, N, NRHS, PREC_TYPE,
     $                   N_NORMS, ITHRESH
      CHARACTER          UPLO
      LOGICAL            COLEQU, IGNORE_CWISE
```

```
      DOUBLE PRECISION   RTHRESH, DZ_UB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   Y( LDY, * ), RES( * ), DY( * ), Y_TAIL( * )
      DOUBLE PRECISION   C( * ), AYB( * ), RCOND, BERR_OUT( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            UPLO2, CNT, I, J, X_STATE, Z_STATE
      DOUBLE PRECISION   YK, DYK, YMIN, NORMY, NORMX, NORMDX, DXRAT,
     $                   DZRAT, PREVNORMDX, PREV_DZ_Z, DXRATMAX,
     $                   DZRATMAX, DX_X, DZ_Z, FINAL_DX_X, FINAL_DZ_Z,
     $                   EPS, HUGEVAL, INCR_THRESH
      LOGICAL            INCR_PREC, UPPER
*     ..
*     .. Parameters ..
      INTEGER            UNSTABLE_STATE, WORKING_STATE, CONV_STATE,
     $                   NOPROG_STATE, Y_PREC_STATE, BASE_RESIDUAL,
     $                   EXTRA_RESIDUAL, EXTRA_Y
      PARAMETER          ( UNSTABLE_STATE = 0, WORKING_STATE = 1,
     $                   CONV_STATE = 2, NOPROG_STATE = 3 )
      PARAMETER          ( BASE_RESIDUAL = 0, EXTRA_RESIDUAL = 1,
     $                   EXTRA_Y = 2 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           ILAUPLO
```

```
      INTEGER            ILAUPLO
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DCOPY, DSYTRS, DSYMV, BLAS_DSYMV_X,
     $                   BLAS_DSYMV2_X, DLA_SYAMV, DLA_WWADDW,
     $                   DLA_LIN_BERR
      DOUBLE PRECISION   DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -13
      ELSE IF( LDY.LT.MAX( 1, N ) ) THEN
         INFO = -15
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLA_SYRFSX_EXTENDED', -INFO )
         RETURN
      END IF
      EPS = DLAMCH( 'Epsilon' )
      HUGEVAL = DLAMCH( 'Overflow' )
*     Force HUGEVAL to Inf
      HUGEVAL = HUGEVAL * HUGEVAL
*     Using HUGEVAL may lead to spurious underflows.
      INCR_THRESH = DBLE( N )*EPS

      IF ( LSAME ( UPLO, 'L' ) ) THEN
         UPLO2 = ILAUPLO( 'L' )
      ELSE
         UPLO2 = ILAUPLO( 'U' )
      ENDIF

      DO J = 1, NRHS
         Y_PREC_STATE = EXTRA_RESIDUAL
         IF ( Y_PREC_STATE .EQ. EXTRA_Y ) THEN
```

```
            DO I = 1, N
               Y_TAIL( I ) = 0.0D+0
            END DO
         END IF

         DXRAT = 0.0D+0
         DXRATMAX = 0.0D+0
         DZRAT = 0.0D+0
         DZRATMAX = 0.0D+0
         FINAL_DX_X = HUGEVAL
         FINAL_DZ_Z = HUGEVAL
         PREVNORMDX = HUGEVAL
         PREV_DZ_Z = HUGEVAL
         DZ_Z = HUGEVAL
         DX_X = HUGEVAL

         X_STATE = WORKING_STATE
         Z_STATE = UNSTABLE_STATE
         INCR_PREC = .FALSE.

         DO CNT = 1, ITHRESH
*
*        Compute residual RES = B_s - op(A_s) * Y,
*            op(A) = A, A**T, or A**H depending on TRANS (and type).
*
            CALL DCOPY( N, B( 1, J ), 1, RES, 1 )
            IF (Y_PREC_STATE .EQ. BASE_RESIDUAL) THEN
               CALL DSYMV( UPLO, N, -1.0D+0, A, LDA, Y(1,J), 1,
     $              1.0D+0, RES, 1 )
            ELSE IF (Y_PREC_STATE .EQ. EXTRA_RESIDUAL) THEN
               CALL BLAS_DSYMV_X( UPLO2, N, -1.0D+0, A, LDA,
     $              Y( 1, J ), 1, 1.0D+0, RES, 1, PREC_TYPE )
            ELSE
               CALL BLAS_DSYMV2_X(UPLO2, N, -1.0D+0, A, LDA,
     $              Y(1, J), Y_TAIL, 1, 1.0D+0, RES, 1, PREC_TYPE)
            END IF

!         XXX: RES is no longer needed.
            CALL DCOPY( N, RES, 1, DY, 1 )
            CALL DSYTRS( UPLO, N, 1, AF, LDAF, IPIV, DY, N, INFO )
*
*         Calculate relative changes DX_X, DZ_Z and ratios DXRAT, DZRAT.
*
            NORMX = 0.0D+0
            NORMY = 0.0D+0
            NORMDX = 0.0D+0
            DZ_Z = 0.0D+0
            YMIN = HUGEVAL

            DO I = 1, N
```

```
               YK = ABS( Y( I, J ) )
               DYK = ABS( DY( I ) )

               IF ( YK .NE. 0.0D+0 ) THEN
                  DZ_Z = MAX( DZ_Z, DYK / YK )
               ELSE IF ( DYK .NE. 0.0D+0 ) THEN
                  DZ_Z = HUGEVAL
               END IF

               YMIN = MIN( YMIN, YK )

               NORMY = MAX( NORMY, YK )

               IF ( COLEQU ) THEN
                  NORMX = MAX( NORMX, YK * C( I ) )
                  NORMDX = MAX( NORMDX, DYK * C( I ) )
               ELSE
                  NORMX = NORMY
                  NORMDX = MAX(NORMDX, DYK)
               END IF
            END DO

            IF ( NORMX .NE. 0.0D+0 ) THEN
               DX_X = NORMDX / NORMX
            ELSE IF ( NORMDX .EQ. 0.0D+0 ) THEN
               DX_X = 0.0D+0
            ELSE
               DX_X = HUGEVAL
            END IF

            DXRAT = NORMDX / PREVNORMDX
            DZRAT = DZ_Z / PREV_DZ_Z
*
*         Check termination criteria.
*
            IF ( YMIN*RCOND .LT. INCR_THRESH*NORMY
     $           .AND. Y_PREC_STATE .LT. EXTRA_Y )
     $          INCR_PREC = .TRUE.

            IF ( X_STATE .EQ. NOPROG_STATE .AND. DXRAT .LE. RTHRESH )
     $          X_STATE = WORKING_STATE
            IF ( X_STATE .EQ. WORKING_STATE ) THEN
               IF ( DX_X .LE. EPS ) THEN
                  X_STATE = CONV_STATE
               ELSE IF ( DXRAT .GT. RTHRESH ) THEN
                  IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                     INCR_PREC = .TRUE.
                  ELSE
                     X_STATE = NOPROG_STATE
                  END IF
```

```
            ELSE
               IF ( DXRAT .GT. DXRATMAX ) DXRATMAX = DXRAT
            END IF
            IF ( X_STATE .GT. WORKING_STATE ) FINAL_DX_X = DX_X
         END IF

         IF ( Z_STATE .EQ. UNSTABLE_STATE .AND. DZ_Z .LE. DZ_UB )
     $         Z_STATE = WORKING_STATE
         IF ( Z_STATE .EQ. NOPROG_STATE .AND. DZRAT .LE. RTHRESH )
     $         Z_STATE = WORKING_STATE
         IF ( Z_STATE .EQ. WORKING_STATE ) THEN
            IF ( DZ_Z .LE. EPS ) THEN
               Z_STATE = CONV_STATE
            ELSE IF ( DZ_Z .GT. DZ_UB ) THEN
               Z_STATE = UNSTABLE_STATE
               DZRATMAX = 0.0D+0
               FINAL_DZ_Z = HUGEVAL
            ELSE IF ( DZRAT .GT. RTHRESH ) THEN
               IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                  INCR_PREC = .TRUE.
               ELSE
                  Z_STATE = NOPROG_STATE
               END IF
            ELSE
               IF ( DZRAT .GT. DZRATMAX ) DZRATMAX = DZRAT
            END IF
            IF ( Z_STATE .GT. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
         END IF

         IF ( X_STATE.NE.WORKING_STATE.AND.
     $         ( IGNORE_CWISE.OR.Z_STATE.NE.WORKING_STATE ) )
     $         GOTO 666

         IF ( INCR_PREC ) THEN
            INCR_PREC = .FALSE.
            Y_PREC_STATE = Y_PREC_STATE + 1
            DO I = 1, N
               Y_TAIL( I ) = 0.0D+0
            END DO
         END IF

         PREVNORMDX = NORMDX
         PREV_DZ_Z = DZ_Z
*
*        Update soluton.
*
         IF (Y_PREC_STATE .LT. EXTRA_Y) THEN
            CALL DAXPY( N, 1.0D+0, DY, 1, Y(1,J), 1 )
         ELSE
            CALL DLA_WWADDW( N, Y(1,J), Y_TAIL, DY )
```

```
            END IF

         END DO
*        Target of "IF (Z_STOP .AND. X_STOP)".  Sun's f77 won't EXIT.
 666     CONTINUE
*
*     Set final_* when cnt hits ithresh.
*
         IF ( X_STATE .EQ. WORKING_STATE ) FINAL_DX_X = DX_X
         IF ( Z_STATE .EQ. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
*
*     Compute error bounds.
*
         IF ( N_NORMS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) =
     $           FINAL_DX_X / (1 - DXRATMAX)
         END IF
         IF ( N_NORMS .GE. 2 ) THEN
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) =
     $           FINAL_DZ_Z / (1 - DZRATMAX)
         END IF
*
*     Compute componentwise relative backward error from formula
*         max(i) ( abs(R(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
*     where abs(Z) is the componentwise absolute value of the matrix
*     or vector Z.
*
*        Compute residual RES = B_s - op(A_s) * Y,
*            op(A) = A, A**T, or A**H depending on TRANS (and type).
         CALL DCOPY( N, B( 1, J ), 1, RES, 1 )
         CALL DSYMV( UPLO, N, -1.0D+0, A, LDA, Y(1,J), 1, 1.0D+0, RES,
     $     1 )

         DO I = 1, N
            AYB( I ) = ABS( B( I, J ) )
         END DO
*
*     Compute abs(op(A_s))*abs(Y) + abs(B_s).
*
         CALL DLA_SYAMV( UPLO2, N, 1.0D+0,
     $        A, LDA, Y(1, J), 1, 1.0D+0, AYB, 1 )

         CALL DLA_LIN_BERR( N, N, 1, RES, AYB, BERR_OUT( J ) )
*
*     End of loop for each RHS.
*
      END DO
*
      RETURN
      END
```

———-

58.3.4 lisp code

— dla_syrfsx_extended.lisp —

```
(let* ((unstable_state 0)
       (working_state 1)
       (conv_state 2)
       (noprog_state 3)
       (base_residual 0)
       (extra_residual 1)
       (extra_y 2)
       (la_linrx_err_i 2))
  (declare (type (f2cl-lib:integer4 0 0) unstable_state)
           (type (f2cl-lib:integer4 1 1) working_state)
           (type (f2cl-lib:integer4 2 2) conv_state)
           (type (f2cl-lib:integer4 3 3) noprog_state)
           (type (f2cl-lib:integer4 0 0) base_residual)
           (type (f2cl-lib:integer4 1 1) extra_residual)
           (type (f2cl-lib:integer4 2 2) extra_y)
           (type (f2cl-lib:integer4 2 2) la_linrx_err_i)
           (ignorable unstable_state working_state conv_state noprog_state
            base_residual extra_residual extra_y la_linrx_err_i))
  (defun dla_syrfsx_extended
         (prec_type uplo n nrhs a lda af ldaf ipiv colequ c b ldb$ y ldy
          berr_out n_norms err_bnds_norm err_bnds_comp res ayb dy y_tail rcond
          ithresh rthresh dz_ub ignore_cwise info)
    (declare (type (double-float) dz_ub rthresh rcond)
             (type f2cl-lib:logical ignore_cwise colequ)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array double-float (*)) y_tail dy ayb res err_bnds_comp
              err_bnds_norm berr_out y b c af a)
             (type (string *) uplo)
             (type (f2cl-lib:integer4) info ithresh n_norms ldy ldb$ ldaf lda
              nrhs n prec_type))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (af double-float af-%data% af-%offset%)
         (c double-float c-%data% c-%offset%)
         (b double-float b-%data% b-%offset%)
         (y double-float y-%data% y-%offset%)
         (berr_out double-float berr_out-%data% berr_out-%offset%)
         (err_bnds_norm double-float err_bnds_norm-%data%
          err_bnds_norm-%offset%)
         (err_bnds_comp double-float err_bnds_comp-%data%
          err_bnds_comp-%offset%)
```

```
    (res double-float res-%data% res-%offset%)
    (ayb double-float ayb-%data% ayb-%offset%)
    (dy double-float dy-%data% dy-%offset%)
    (y_tail double-float y_tail-%data% y_tail-%offset%)
    (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
  (prog ((y_prec_state 0) (incr_prec nil) (upper nil) (yk 0.0d0)
         (dyk 0.0d0) (ymin 0.0d0) (normy 0.0d0) (normx 0.0d0)
         (normdx 0.0d0) (dxrat 0.0d0) (dzrat 0.0d0) (prevnormdx 0.0d0)
         (prev_dz_z 0.0d0) (dxratmax 0.0d0) (dzratmax 0.0d0) (dx_x 0.0d0)
         (dz_z 0.0d0) (final_dx_x 0.0d0) (final_dz_z 0.0d0) (eps 0.0d0)
         (hugeval 0.0d0) (incr_thresh 0.0d0) (uplo2 0) (cnt 0) (i 0) (j 0)
         (x_state 0) (z_state 0))
    (declare (type f2cl-lib:logical incr_prec upper)
             (type (double-float) yk dyk ymin normy normx normdx dxrat
              dzrat prevnormdx prev_dz_z dxratmax dzratmax dx_x dz_z
              final_dx_x final_dz_z eps hugeval incr_thresh)
             (type (f2cl-lib:integer4) y_prec_state uplo2 cnt i j x_state
              z_state))
    (setf info 0)
    (setf upper
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
    (cond
     ((and (not upper)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
      (setf info -2))
     ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
     ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -6))
     ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -8))
     ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -13))
     ((< ldy (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -15)))
    (cond
     ((/= info 0) (xerbla "DLA_SYRFSX_EXTENDED" (f2cl-lib:int-sub info))
      (go end_label)))
    (setf eps (dlamch "Epsilon"))
    (setf hugeval (dlamch "Overflow"))
    (setf hugeval (* hugeval hugeval))
    (setf incr_thresh (* (f2cl-lib:dble n) eps))
```

```
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame uplo "L")
    (declare (ignore var-1))
    (when var-0 (setf uplo var-0))
    ret-val)
  (setf uplo2 (ilauplo "L")))
 (t (setf uplo2 (ilauplo "U"))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf y_prec_state extra_residual)
    (cond
     ((= y_prec_state extra_y)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                               y_tail-%offset%)
                0.0d0)
         label100001))))
    (setf dxrat 0.0d0)
    (setf dxratmax 0.0d0)
    (setf dzrat 0.0d0)
    (setf dzratmax 0.0d0)
    (setf final_dx_x hugeval)
    (setf final_dz_z hugeval)
    (setf prevnormdx hugeval)
    (setf prev_dz_z hugeval)
    (setf dz_z hugeval)
    (setf dx_x hugeval)
    (setf x_state working_state)
    (setf z_state unstable_state)
    (setf incr_prec f2cl-lib:%false%)
    (f2cl-lib:fdo (cnt 1 (f2cl-lib:int-add cnt 1))
                  ((> cnt ithresh) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dcopy n
             (f2cl-lib:array-slice b-%data% double-float (1 j)
                                   ((1 ldb$) (1 *)) b-%offset%)
             1 res 1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf n var-0)))
        (cond
         ((= y_prec_state base_residual)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9)
              (dsymv uplo n -1.0d0 a lda
```

```
        (f2cl-lib:array-slice y-%data% double-float (1 j)
                              ((1 ldy) (1 *)) y-%offset%)
        1 1.0d0 res 1)
     (declare
      (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
     (when var-0 (setf uplo var-0))
     (when var-1 (setf n var-1))
     (when var-4 (setf lda var-4))))
  ((= y_prec_state extra_residual)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (blas_dsymv_x uplo2 n -1.0d0 a lda
        (f2cl-lib:array-slice y-%data% double-float (1 j)
                              ((1 ldy) (1 *)) y-%offset%)
        1 1.0d0 res 1 prec_type)
     (declare
      (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
     (when var-0 (setf uplo2 var-0))
     (when var-1 (setf n var-1))
     (when var-4 (setf lda var-4))
     (when var-10 (setf prec_type var-10))))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10 var-11)
       (blas_dsymv2_x uplo2 n -1.0d0 a lda
        (f2cl-lib:array-slice y-%data% double-float (1 j)
                              ((1 ldy) (1 *)) y-%offset%)
        y_tail 1 1.0d0 res 1 prec_type)
     (declare
      (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9 var-10))
     (when var-0 (setf uplo2 var-0))
     (when var-1 (setf n var-1))
     (when var-4 (setf lda var-4))
     (when var-11 (setf prec_type var-11)))))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (dcopy n res 1 dy 1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf n var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (dsytrs uplo n 1 af ldaf ipiv dy n info)
   (declare (ignore var-2 var-3 var-5 var-6))
   (when var-0 (setf uplo var-0))
   (when var-1 (setf n var-1))
   (when var-4 (setf ldaf var-4))
   (when var-7 (setf n var-7))
   (when var-8 (setf info var-8)))
 (setf normx 0.0d0)
```

```
(setf normy 0.0d0)
(setf normdx 0.0d0)
(setf dz_z 0.0d0)
(setf ymin hugeval)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf yk
            (abs
             (f2cl-lib:fref y-%data% (i j) ((1 ldy) (1 *))
                            y-%offset%)))
    (setf dyk
            (abs
             (f2cl-lib:fref dy-%data% (i) ((1 *))
                            dy-%offset%)))
    (cond ((/= yk 0.0d0) (setf dz_z (max dz_z (/ dyk yk))))
          ((/= dyk 0.0d0) (setf dz_z hugeval)))
    (setf ymin (min ymin yk))
    (setf normy (max normy yk))
    (cond
     (colequ
      (setf normx
              (max normx
                   (* yk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%))))
      (setf normdx
              (max normdx
                   (* dyk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%)))))
     (t (setf normx normy) (setf normdx (max normdx dyk))))
   label100003))
(cond ((/= normx 0.0d0) (setf dx_x (/ normdx normx)))
      ((= normdx 0.0d0) (setf dx_x 0.0d0))
      (t (setf dx_x hugeval)))
(setf dxrat (/ normdx prevnormdx))
(setf dzrat (/ dz_z prev_dz_z))
(if (and (< (* ymin rcond) (* incr_thresh normy))
         (< y_prec_state extra_y))
    (setf incr_prec f2cl-lib:%true%))
(if (and (= x_state noprog_state) (<= dxrat rthresh))
    (setf x_state working_state))
(cond
 ((= x_state working_state)
  (cond ((<= dx_x eps) (setf x_state conv_state))
        ((> dxrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
```

```
          (t (setf x_state noprog_state))))
        (t
         (if (> dxrat dxratmax)
             (setf dxratmax dxrat))))
  (if (> x_state working_state)
      (setf final_dx_x dx_x))))
(if (and (= z_state unstable_state) (<= dz_z dz_ub))
    (setf z_state working_state))
(if (and (= z_state noprog_state) (<= dzrat rthresh))
    (setf z_state working_state))
(cond
 ((= z_state working_state)
  (cond ((<= dz_z eps) (setf z_state conv_state))
        ((> dz_z dz_ub) (setf z_state unstable_state)
         (setf dzratmax 0.0d0) (setf final_dz_z hugeval))
        ((> dzrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf z_state noprog_state))))
        (t
         (if (> dzrat dzratmax)
             (setf dzratmax dzrat))))
  (if (> z_state working_state)
      (setf final_dz_z dz_z))))
(if (and (/= x_state working_state)
         (or ignore_cwise (/= z_state working_state)))
    (go label666))
(cond
 (incr_prec (setf incr_prec f2cl-lib:%false%)
  (setf y_prec_state (f2cl-lib:int-add y_prec_state 1))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                           y_tail-%offset%)
            0.0d0)
     label100004))))
(setf prevnormdx normdx)
(setf prev_dz_z dz_z)
(cond
 ((< y_prec_state extra_y)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (daxpy n 1.0d0 dy 1
       (f2cl-lib:array-slice y-%data% double-float (1 j)
                             ((1 ldy) (1 *)) y-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))))
 (t
```

```
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dla_wwaddw n
             (f2cl-lib:array-slice y-%data% double-float (1 j)
                                   ((1 ldy) (1 *)) y-%offset%)
             y_tail dy)
          (declare (ignore var-1 var-2 var-3))
          (when var-0 (setf n var-0)))))
    label100002))
label666
 (if (= x_state working_state)
     (setf final_dx_x dx_x))
 (if (= z_state working_state)
     (setf final_dz_z dz_z))
 (cond
  ((>= n_norms 1)
   (setf (f2cl-lib:fref err_bnds_norm-%data% (j la_linrx_err_i)
                        ((1 nrhs) (1 *)) err_bnds_norm-%offset%)
           (/ final_dx_x (- 1 dxratmax)))))
 (cond
  ((>= n_norms 2)
   (setf (f2cl-lib:fref err_bnds_comp-%data% (j la_linrx_err_i)
                        ((1 nrhs) (1 *)) err_bnds_comp-%offset%)
           (/ final_dz_z (- 1 dzratmax)))))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (dcopy n
      (f2cl-lib:array-slice b-%data% double-float (1 j)
                            ((1 ldb$) (1 *)) b-%offset%)
      1 res 1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf n var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
     (dsymv uplo n -1.0d0 a lda
      (f2cl-lib:array-slice y-%data% double-float (1 j)
                            ((1 ldy) (1 *)) y-%offset%)
      1 1.0d0 res 1)
   (declare (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
   (when var-0 (setf uplo var-0))
   (when var-1 (setf n var-1))
   (when var-4 (setf lda var-4)))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf (f2cl-lib:fref ayb-%data% (i) ((1 *)) ayb-%offset%)
             (abs
              (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)))
    label100005))
 (dla_syamv uplo2 n 1.0d0 a lda
  (f2cl-lib:array-slice y-%data% double-float (1 j) ((1 ldy) (1 *))
```

```
                                y-%offset%)
            1 1.0d0 ayb 1)
           (dla_lin_berr n n 1 res ayb
            (f2cl-lib:array-slice berr_out-%data% double-float (j) ((1 *))
                                berr_out-%offset%))
          label100000))
        (go end_label)
       end_label
        (return
         (values prec_type uplo n nil nil lda nil ldaf nil nil nil nil nil nil
                 nil nil nil nil nil nil nil nil nil nil nil nil nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dla_syrfsx_extended
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              fortran-to-lisp::logical
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (double-float)
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              fortran-to-lisp::logical
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::prec_type
```

```
                                        fortran-to-lisp::uplo
                                        fortran-to-lisp::n nil nil
                                        fortran-to-lisp::lda nil
                                        fortran-to-lisp::ldaf nil nil nil
                                        nil nil nil nil nil nil nil nil
                                        nil nil nil nil nil nil nil nil
                                        nil fortran-to-lisp::info)
                                      :calls
                                      '(fortran-to-lisp::dla_lin_berr
                                        fortran-to-lisp::dla_syamv))))
```

———-

58.4 dla_syrpvgrw

58.4.1 Axiom unit tests

— **dlasyrpvgrw.input** —

```
)set break resume
)sys rm -f dlasyrpvgrw.output
)spool dlasyrpvgrw.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.4.2 Axiom help page

— **dla_syrpvgrw.help** —

```
==========================================================================
dla_syrpvgrw examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       double precision function dla_syrpvgrw (UPLO, N, INFO, A, LDA, AF,
           LDAF, IPIV, WORK)
           DLA_SYRPVGRW computes the reciprocal pivot growth factor
           norm(A)/norm(U) for a symmetric indefinite matrix.

Function/Subroutine Documentation
   double precision function dla_syrpvgrw (character*1 UPLO, integer N,
       integer INFO, double precision, dimension( lda, * ) A, integer LDA,
       double precision, dimension( ldaf, * ) AF, integer LDAF, integer,
       dimension( * ) IPIV, double precision, dimension( * ) WORK)
       DLA_SYRPVGRW computes the reciprocal pivot growth factor
       norm(A)/norm(U) for a symmetric indefinite matrix.

Purpose:
   DLA_SYRPVGRW computes the reciprocal pivot growth factor
   norm(A)/norm(U). The "max absolute element" norm is used. If this is
   much less than 1, the stability of the LU factorization of the
   (equilibrated) matrix A could be poor. This also means that the
   solution X, estimated condition numbers, and error bounds could be
   unreliable.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 INFO
   INFO is INTEGER
   The value of INFO returned from DSYTRF, .i.e., the pivot in
   column INFO is exactly 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is DOUBLE PRECISION array, dimension (LDAF,N)
```

```
  The block diagonal matrix D and the multipliers used to
  obtain the factor U or L as computed by DSYTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by DSYTRF.

WORK
  WORK is DOUBLE PRECISION array, dimension (2*N)
```

———-

58.4.3 fortran code

— dla_syrpvgrw.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION DLA_SYRPVGRW( UPLO, N, INFO, A, LDA, AF,
     $                                        LDAF, IPIV, WORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER*1        UPLO
      INTEGER            N, INFO, LDA, LDAF
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            NCOLS, I, J, K, KP
      DOUBLE PRECISION   AMAX, UMAX, RPVGRW, TMP
      LOGICAL            UPPER
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, DLASET
      LOGICAL            LSAME
*     ..
*     .. Executable Statements ..
*
      UPPER = LSAME( 'Upper', UPLO )
      IF ( INFO.EQ.0 ) THEN
         IF ( UPPER ) THEN
            NCOLS = 1
         ELSE
            NCOLS = N
         END IF
      ELSE
         NCOLS = INFO
      END IF

      RPVGRW = 1.0D+0
      DO I = 1, 2*N
         WORK( I ) = 0.0D+0
      END DO
*
*     Find the max magnitude entry of each column of A.  Compute the max
*     for all N columns so we can apply the pivot permutation while
*     looping below.  Assume a full factorization is the common case.
*
      IF ( UPPER ) THEN
         DO J = 1, N
            DO I = 1, J
               WORK( N+I ) = MAX( ABS( A( I, J ) ), WORK( N+I ) )
               WORK( N+J ) = MAX( ABS( A( I, J ) ), WORK( N+J ) )
            END DO
         END DO
      ELSE
         DO J = 1, N
            DO I = J, N
               WORK( N+I ) = MAX( ABS( A( I, J ) ), WORK( N+I ) )
               WORK( N+J ) = MAX( ABS( A( I, J ) ), WORK( N+J ) )
            END DO
         END DO
      END IF
*
*     Now find the max magnitude entry of each column of U or L.  Also
*     permute the magnitudes of A above so they're in the same order as
*     the factor.
*
*     The iteration orders and permutations were copied from dsytrs.
*     Calls to SSWAP would be severe overkill.
```

```
*
      IF ( UPPER ) THEN
         K = N
         DO WHILE ( K .LT. NCOLS .AND. K.GT.0 )
            IF ( IPIV( K ).GT.0 ) THEN
!              1x1 pivot
               KP = IPIV( K )
               IF ( KP .NE. K ) THEN
                  TMP = WORK( N+K )
                  WORK( N+K ) = WORK( N+KP )
                  WORK( N+KP ) = TMP
               END IF
               DO I = 1, K
                  WORK( K ) = MAX( ABS( AF( I, K ) ), WORK( K ) )
               END DO
               K = K - 1
            ELSE
!              2x2 pivot
               KP = -IPIV( K )
               TMP = WORK( N+K-1 )
               WORK( N+K-1 ) = WORK( N+KP )
               WORK( N+KP ) = TMP
               DO I = 1, K-1
                  WORK( K ) = MAX( ABS( AF( I, K ) ), WORK( K ) )
                  WORK( K-1 ) = MAX( ABS( AF( I, K-1 ) ), WORK( K-1 ) )
               END DO
               WORK( K ) = MAX( ABS( AF( K, K ) ), WORK( K ) )
               K = K - 2
            END IF
         END DO
         K = NCOLS
         DO WHILE ( K .LE. N )
            IF ( IPIV( K ).GT.0 ) THEN
               KP = IPIV( K )
               IF ( KP .NE. K ) THEN
                  TMP = WORK( N+K )
                  WORK( N+K ) = WORK( N+KP )
                  WORK( N+KP ) = TMP
               END IF
               K = K + 1
            ELSE
               KP = -IPIV( K )
               TMP = WORK( N+K )
               WORK( N+K ) = WORK( N+KP )
               WORK( N+KP ) = TMP
               K = K + 2
            END IF
         END DO
      ELSE
         K = 1
```

```
         DO WHILE ( K .LE. NCOLS )
            IF ( IPIV( K ).GT.0 ) THEN
!              1x1 pivot
               KP = IPIV( K )
               IF ( KP .NE. K ) THEN
                  TMP = WORK( N+K )
                  WORK( N+K ) = WORK( N+KP )
                  WORK( N+KP ) = TMP
               END IF
               DO I = K, N
                  WORK( K ) = MAX( ABS( AF( I, K ) ), WORK( K ) )
               END DO
               K = K + 1
            ELSE
!              2x2 pivot
               KP = -IPIV( K )
               TMP = WORK( N+K+1 )
               WORK( N+K+1 ) = WORK( N+KP )
               WORK( N+KP ) = TMP
               DO I = K+1, N
                  WORK( K ) = MAX( ABS( AF( I, K ) ), WORK( K ) )
                  WORK( K+1 ) = MAX( ABS( AF(I, K+1 ) ), WORK( K+1 ) )
               END DO
               WORK( K ) = MAX( ABS( AF( K, K ) ), WORK( K ) )
               K = K + 2
            END IF
         END DO
         K = NCOLS
         DO WHILE ( K .GE. 1 )
            IF ( IPIV( K ).GT.0 ) THEN
               KP = IPIV( K )
               IF ( KP .NE. K ) THEN
                  TMP = WORK( N+K )
                  WORK( N+K ) = WORK( N+KP )
                  WORK( N+KP ) = TMP
               END IF
               K = K - 1
            ELSE
               KP = -IPIV( K )
               TMP = WORK( N+K )
               WORK( N+K ) = WORK( N+KP )
               WORK( N+KP ) = TMP
               K = K - 2
            ENDIF
         END DO
      END IF
*
*     Compute the *inverse* of the max element growth factor.  Dividing
*     by zero would imply the largest entry of the factor's column is
*     zero.  Than can happen when either the column of A is zero or
```

```
*     massive pivots made the factor underflow to zero.  Neither counts
*     as growth in itself, so simply ignore terms with zero
*     denominators.
*
      IF ( UPPER ) THEN
         DO I = NCOLS, N
            UMAX = WORK( I )
            AMAX = WORK( N+I )
            IF ( UMAX /= 0.0D+0 ) THEN
               RPVGRW = MIN( AMAX / UMAX, RPVGRW )
            END IF
         END DO
      ELSE
         DO I = 1, NCOLS
            UMAX = WORK( I )
            AMAX = WORK( N+I )
            IF ( UMAX /= 0.0D+0 ) THEN
               RPVGRW = MIN( AMAX / UMAX, RPVGRW )
            END IF
         END DO
      END IF

      DLA_SYRPVGRW = RPVGRW
      END
```

———-

58.4.4 lisp code

— dla_syrpvgrw.lisp —

```
(defun dla_syrpvgrw (uplo n info a lda af ldaf ipiv work)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array double-float (*)) work af a)
           (type (f2cl-lib:integer4) ldaf lda info n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a double-float a-%data% a-%offset%)
       (af double-float af-%data% af-%offset%)
       (work double-float work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((upper nil) (amax 0.0d0) (umax 0.0d0) (rpvgrw 0.0d0) (tmp 0.0d0)
           (ncols 0) (i 0) (j 0) (k 0) (kp 0) (dla_syrpvgrw 0.0d0))
      (declare (type (f2cl-lib:integer4) kp k j i ncols)
               (type (double-float) dla_syrpvgrw tmp rpvgrw umax amax)
               (type f2cl-lib:logical upper))
      (setf upper
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame "Upper" uplo)
          (declare (ignore var-0))
          (when var-1 (setf uplo var-1))
          ret-val))
(cond ((= info 0) (cond (upper (setf ncols 1)) (t (setf ncols n))))
      (t (setf ncols info)))
(setf rpvgrw 1.0d0)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-mul 2 n)) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) 0.0d0)
   label100000))
(cond
 (upper
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i j) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                ((1 *)) work-%offset%)
                  (max
                   (abs
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))
                   (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                  ((1 *)) work-%offset%)))
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j))
                                ((1 *)) work-%offset%)
                  (max
                   (abs
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))
                   (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j))
                                  ((1 *)) work-%offset%)))
         label100002))
     label100001)))
 (t
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                ((1 *)) work-%offset%)
                  (max
                   (abs
```

```
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))
                       (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                      ((1 *)) work-%offset%)))
            (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j))
                                 ((1 *)) work-%offset%)
                    (max
                     (abs
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))
                     (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j))
                                    ((1 *)) work-%offset%)))
          label100004))
      label100003))))
  (cond
   (upper
    (tagbody
      (setf k n)
     label100005
      (if (not (and (< k ncols) (> k 0)))
          (go label100006))
      (cond
       ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
        (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
        (cond
         ((/= kp k)
          (setf tmp
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                                 ((1 *)) work-%offset%))
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                               work-%offset%)
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                                 ((1 *)) work-%offset%))
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                               ((1 *)) work-%offset%)
                  tmp)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i k) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                    (max
                     (abs
                      (f2cl-lib:fref af-%data% (i k) ((1 ldaf) (1 *))
                                     af-%offset%))
                     (f2cl-lib:fref work-%data% (k) ((1 *))
                                    work-%offset%)))
           label100007))
        (setf k (f2cl-lib:int-sub k 1)))
       (t
        (setf kp
```

```
            (f2cl-lib:int-sub
             (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
    (setf tmp
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-sub (f2cl-lib:int-add n k)
                                              1))
                           ((1 *)) work-%offset%))
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-sub (f2cl-lib:int-add n k) 1))
                         ((1 *)) work-%offset%)
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                           ((1 *)) work-%offset%))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp)) ((1 *))
                         work-%offset%)
            tmp)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                (max
                 (abs
                  (f2cl-lib:fref af-%data% (i k) ((1 ldaf) (1 *))
                                 af-%offset%))
                 (f2cl-lib:fref work-%data% (k) ((1 *))
                                work-%offset%)))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1))
                             ((1 *)) work-%offset%)
                (max
                 (abs
                  (f2cl-lib:fref af-%data% (i (f2cl-lib:int-sub k 1))
                                 ((1 ldaf) (1 *)) af-%offset%))
                 (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1))
                                ((1 *)) work-%offset%)))
       label100008))
    (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
            (max
             (abs
              (f2cl-lib:fref af-%data% (k k) ((1 ldaf) (1 *))
                             af-%offset%))
             (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)))
    (setf k (f2cl-lib:int-sub k 2))))
  (go label100005)
 label100006
  (setf k ncols)
 label100009
  (if (not (<= k n))
      (go label100010))
  (cond
   ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
    (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
```

```
      (cond
       ((/= kp k)
        (setf tmp
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                               ((1 *)) work-%offset%))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                             work-%offset%)
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                               ((1 *)) work-%offset%))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                             ((1 *)) work-%offset%)
                tmp)))
      (setf k (f2cl-lib:int-add k 1)))
     (t
      (setf kp
              (f2cl-lib:int-sub
               (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
      (setf tmp
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                             work-%offset%))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                           work-%offset%)
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                             ((1 *)) work-%offset%))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp)) ((1 *))
                           work-%offset%)
              tmp)
      (setf k (f2cl-lib:int-add k 2))))
    (go label100009)
   label100010))
 (t
  (tagbody
    (setf k 1)
   label100011
    (if (not (<= k ncols))
        (go label100012))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (cond
       ((/= kp k)
        (setf tmp
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                               ((1 *)) work-%offset%))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                             work-%offset%)
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                               ((1 *)) work-%offset%))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                             ((1 *)) work-%offset%)
```

```
          tmp)))
 (f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
             (max
              (abs
               (f2cl-lib:fref af-%data% (i k) ((1 ldaf) (1 *))
                              af-%offset%))
              (f2cl-lib:fref work-%data% (k) ((1 *))
                             work-%offset%)))
    label100013))
 (setf k (f2cl-lib:int-add k 1)))
(t
 (setf kp
         (f2cl-lib:int-sub
          (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
 (setf tmp
         (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k 1))
                        ((1 *)) work-%offset%))
 (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k 1)) ((1 *))
                      work-%offset%)
         (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                        ((1 *)) work-%offset%))
 (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp)) ((1 *))
                      work-%offset%)
         tmp)
 (f2cl-lib:fdo (i (f2cl-lib:int-add k 1) (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
             (max
              (abs
               (f2cl-lib:fref af-%data% (i k) ((1 ldaf) (1 *))
                              af-%offset%))
              (f2cl-lib:fref work-%data% (k) ((1 *))
                             work-%offset%)))
     (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1))
                          ((1 *)) work-%offset%)
             (max
              (abs
               (f2cl-lib:fref af-%data% (i (f2cl-lib:int-add k 1))
                              ((1 ldaf) (1 *)) af-%offset%))
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1))
                             ((1 *)) work-%offset%)))
    label100014))
 (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
         (max
          (abs
           (f2cl-lib:fref af-%data% (k k) ((1 ldaf) (1 *))
```

```
                                af-%offset%))
                 (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)))
        (setf k (f2cl-lib:int-add k 2))))
      (go label100011)
    label100012
      (setf k ncols)
    label100015
      (if (not (>= k 1))
          (go label100016))
      (cond
       ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
        (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
        (cond
         ((/= kp k)
          (setf tmp
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                                 ((1 *)) work-%offset%))
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                               work-%offset%)
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                                 ((1 *)) work-%offset%))
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                               ((1 *)) work-%offset%)
                  tmp)))
        (setf k (f2cl-lib:int-sub k 1)))
       (t
        (setf kp
                (f2cl-lib:int-sub
                 (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
        (setf tmp
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                               work-%offset%))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                             work-%offset%)
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                               ((1 *)) work-%offset%))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp)) ((1 *))
                             work-%offset%)
                tmp)
        (setf k (f2cl-lib:int-sub k 2))))
      (go label100015)
    label100016)))
  (cond
   (upper
    (f2cl-lib:fdo (i ncols (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf umax (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
        (setf amax
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i)) ((1 *))
```

```
                                  work-%offset%))
            (cond
             ((f2cl-lib:f2cl/ umax (= 0.0d0))
              (setf rpvgrw (min (/ amax umax) rpvgrw))))
           label100017)))
       (t
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i ncols) nil)
          (tagbody
            (setf umax (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
            (setf amax
                    (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i)) ((1 *))
                                   work-%offset%))
            (cond
             ((f2cl-lib:f2cl/ umax (= 0.0d0))
              (setf rpvgrw (min (/ amax umax) rpvgrw))))
           label100018))))
      (setf dla_syrpvgrw rpvgrw)
     end_label
      (return (values dla_syrpvgrw uplo nil nil nil nil nil nil nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dla_syrpvgrw
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*)))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil nil
                                              nil nil nil nil nil)
                                            :calls 'nil)))
```

———-

58.5 dlasyf

58.5.1 Axiom unit tests

— dlasyf.input —

```
)set break resume
)sys rm -f dlasyf.output
)spool dlasyf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.5.2 Axiom help page

— dlasyf.help —

```
======================================================================
dlasyf examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlasyf (UPLO, N, NB, KB, A, LDA, IPIV, W, LDW, INFO)
           DLASYF computes a partial factorization of a real symmetric matrix
           using the Bunch-Kaufman diagonal pivoting method.

Function/Subroutine Documentation
   subroutine dlasyf (character UPLO, integer N, integer NB, integer KB,
       double precision, dimension( lda, * ) A, integer LDA, integer,
       dimension( * ) IPIV, double precision, dimension( ldw, * ) W, integer
       LDW, integer INFO)
       DLASYF computes a partial factorization of a real symmetric matrix
       using the Bunch-Kaufman diagonal pivoting method.

Purpose:
   DLASYF computes a partial factorization of a real symmetric matrix A
```

```
  using the Bunch-Kaufman diagonal pivoting method. The partial
  factorization has the form:

      A  =  ( I  U12 ) ( A11  0  ) (  I       0    )  if UPLO = 'U', or:
            ( 0  U22 ) (  0   D  ) ( U12**T U22**T )

      A  =  ( L11  0 ) (  D   0  ) ( L11**T L21**T )  if UPLO = 'L'
            ( L21  I ) (  0  A22 ) (  0       I    )

  where the order of D is at most NB. The actual order is returned in
  the argument KB, and is either NB or NB-1, or N if N <= NB.

  DLASYF is an auxiliary routine called by DSYTRF. It uses blocked code
  (calling Level 3 BLAS) to update the submatrix A11 (if UPLO = 'U') or
  A22 (if UPLO = 'L').

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is stored:
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NB
   NB is INTEGER
   The maximum number of columns of the matrix A that should be
   factored.  NB should be at least 2 to allow for 2-by-2 pivot blocks.

 KB
   KB is INTEGER
   The number of columns of A that were actually factored.
   KB is either NB-1 or NB, or N if N <= NB.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   n-by-n upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading n-by-n lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.
   On exit, A contains details of the partial factorization.
```

```
LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D.

  If UPLO = 'U':
    Only the last KB elements of IPIV are set.

    If IPIV(k) > 0, then rows and columns k and IPIV(k) were
    interchanged and D(k,k) is a 1-by-1 diagonal block.

    If IPIV(k) = IPIV(k-1) < 0, then rows and columns
    k-1 and -IPIV(k) were interchanged and D(k-1:k,k-1:k)
    is a 2-by-2 diagonal block.

  If UPLO = 'L':
    Only the first KB elements of IPIV are set.

    If IPIV(k) > 0, then rows and columns k and IPIV(k) were
    interchanged and D(k,k) is a 1-by-1 diagonal block.

    If IPIV(k) = IPIV(k+1) < 0, then rows and columns
    k+1 and -IPIV(k) were interchanged and D(k:k+1,k:k+1)
    is a 2-by-2 diagonal block.

W
  W is DOUBLE PRECISION array, dimension (LDW,NB)

LDW
  LDW is INTEGER
  The leading dimension of the array W.  LDW >= max(1,N).

INFO
  INFO is INTEGER
  = 0: successful exit
  > 0: if INFO = k, D(k,k) is exactly zero.  The factorization
       has been completed, but the block diagonal matrix D is
       exactly singular.
```

58.5.3 fortran code

— dlasyf.f —

```
*  =====================================================================
      SUBROUTINE DLASYF( UPLO, N, NB, KB, A, LDA, IPIV, W, LDW, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, KB, LDA, LDW, N, NB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), W( LDW, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   EIGHT, SEVTEN
      PARAMETER          ( EIGHT = 8.0D+0, SEVTEN = 17.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            IMAX, J, JB, JJ, JMAX, JP, K, KK, KKW, KP,
     $                   KSTEP, KW
      DOUBLE PRECISION   ABSAKK, ALPHA, COLMAX, D11, D21, D22, R1,
     $                   ROWMAX, T
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX
      EXTERNAL           LSAME, IDAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DGEMM, DGEMV, DSCAL, DSWAP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
*     Initialize ALPHA for use in choosing pivot block size.
*
      ALPHA = ( ONE+SQRT( SEVTEN ) ) / EIGHT
*
```

```
      IF( LSAME( UPLO, 'U' ) ) THEN
*
*        Factorize the trailing columns of A using the upper triangle
*        of A and working backwards, and compute the matrix W = U12*D
*        for use in updating A11
*
*        K is the main loop index, decreasing from N in steps of 1 or 2
*
*        KW is the column of W which corresponds to column K of A
*
         K = N
   10    CONTINUE
         KW = NB + K - N
*
*        Exit from loop
*
         IF( ( K.LE.N-NB+1 .AND. NB.LT.N ) .OR. K.LT.1 )
     $      GO TO 30
*
*        Copy column K of A to column KW of W and update it
*
         CALL DCOPY( K, A( 1, K ), 1, W( 1, KW ), 1 )
         IF( K.LT.N )
     $      CALL DGEMV( 'No transpose', K, N-K, -ONE, A( 1, K+1 ), LDA,
     $                  W( K, KW+1 ), LDW, ONE, W( 1, KW ), 1 )
*
         KSTEP = 1
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = ABS( W( K, KW ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.GT.1 ) THEN
            IMAX = IDAMAX( K-1, W( 1, KW ), 1 )
            COLMAX = ABS( W( IMAX, KW ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( MAX( ABSAKK, COLMAX ).EQ.ZERO ) THEN
*
*           Column K is zero or underflow: set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
```

```
            KP = K
         ELSE
            IF( ABSAKK.GE.ALPHA*COLMAX ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
            ELSE
*
*              Copy column IMAX to column KW-1 of W and update it
*
               CALL DCOPY( IMAX, A( 1, IMAX ), 1, W( 1, KW-1 ), 1 )
               CALL DCOPY( K-IMAX, A( IMAX, IMAX+1 ), LDA,
     $                     W( IMAX+1, KW-1 ), 1 )
               IF( K.LT.N )
     $            CALL DGEMV( 'No transpose', K, N-K, -ONE, A( 1, K+1 ),
     $                        LDA, W( IMAX, KW+1 ), LDW, ONE,
     $                        W( 1, KW-1 ), 1 )
*
*              JMAX is the column-index of the largest off-diagonal
*              element in row IMAX, and ROWMAX is its absolute value
*
               JMAX = IMAX + IDAMAX( K-IMAX, W( IMAX+1, KW-1 ), 1 )
               ROWMAX = ABS( W( JMAX, KW-1 ) )
               IF( IMAX.GT.1 ) THEN
                  JMAX = IDAMAX( IMAX-1, W( 1, KW-1 ), 1 )
                  ROWMAX = MAX( ROWMAX, ABS( W( JMAX, KW-1 ) ) )
               END IF
*
               IF( ABSAKK.GE.ALPHA*COLMAX*( COLMAX / ROWMAX ) ) THEN
*
*                 no interchange, use 1-by-1 pivot block
*
                  KP = K
               ELSE IF( ABS( W( IMAX, KW-1 ) ).GE.ALPHA*ROWMAX ) THEN
*
*                 interchange rows and columns K and IMAX, use 1-by-1
*                 pivot block
*
                  KP = IMAX
*
*                 copy column KW-1 of W to column KW of W
*
                  CALL DCOPY( K, W( 1, KW-1 ), 1, W( 1, KW ), 1 )
               ELSE
*
*                 interchange rows and columns K-1 and IMAX, use 2-by-2
*                 pivot block
*
                  KP = IMAX
```

```
                 KSTEP = 2
              END IF
           END IF
*
*          ============================================================
*
*          KK is the column of A where pivoting step stopped
*
           KK = K - KSTEP + 1
*
*          KKW is the column of W which corresponds to column KK of A
*
           KKW = NB + KK - N
*
*          Interchange rows and columns KP and KK.
*          Updated column KP is already stored in column KKW of W.
*
           IF( KP.NE.KK ) THEN
*
*             Copy non-updated column KK to column KP of submatrix A
*             at step K. No need to copy element into column K
*             (or K and K-1 for 2-by-2 pivot) of A, since these columns
*             will be later overwritten.
*
              A( KP, KP ) = A( KK, KK )
              CALL DCOPY( KK-1-KP, A( KP+1, KK ), 1, A( KP, KP+1 ),
     $                    LDA )
              IF( KP.GT.1 )
     $           CALL DCOPY( KP-1, A( 1, KK ), 1, A( 1, KP ), 1 )
*
*             Interchange rows KK and KP in last K+1 to N columns of A
*             (columns K (or K and K-1 for 2-by-2 pivot) of A will be
*             later overwritten). Interchange rows KK and KP
*             in last KKW to NB columns of W.
*
              IF( K.LT.N )
     $           CALL DSWAP( N-K, A( KK, K+1 ), LDA, A( KP, K+1 ),
     $                       LDA )
              CALL DSWAP( N-KK+1, W( KK, KKW ), LDW, W( KP, KKW ),
     $                    LDW )
           END IF
*
           IF( KSTEP.EQ.1 ) THEN
*
*             1-by-1 pivot block D(k): column kw of W now holds
*
*             W(kw) = U(k)*D(k),
*
*             where U(k) is the k-th column of U
*
```

```
*              Store subdiag. elements of column U(k)
*              and 1-by-1 block D(k) in column k of A.
*              NOTE: Diagonal element U(k,k) is a UNIT element
*              and not stored.
*                 A(k,k) := D(k,k) = W(k,kw)
*                 A(1:k-1,k) := U(1:k-1,k) = W(1:k-1,kw)/D(k,k)
*
               CALL DCOPY( K, W( 1, KW ), 1, A( 1, K ), 1 )
               R1 = ONE / A( K, K )
               CALL DSCAL( K-1, R1, A( 1, K ), 1 )
*
            ELSE
*
*              2-by-2 pivot block D(k): columns kw and kw-1 of W now hold
*
*              ( W(kw-1) W(kw) ) = ( U(k-1) U(k) )*D(k)
*
*              where U(k) and U(k-1) are the k-th and (k-1)-th columns
*              of U
*
*              Store U(1:k-2,k-1) and U(1:k-2,k) and 2-by-2
*              block D(k-1:k,k-1:k) in columns k-1 and k of A.
*              NOTE: 2-by-2 diagonal block U(k-1:k,k-1:k) is a UNIT
*              block and not stored.
*                 A(k-1:k,k-1:k) := D(k-1:k,k-1:k) = W(k-1:k,kw-1:kw)
*                 A(1:k-2,k-1:k) := U(1:k-2,k:k-1:k) =
*                 = W(1:k-2,kw-1:kw) * ( D(k-1:k,k-1:k)**(-1) )
*
               IF( K.GT.2 ) THEN
*
*                 Compose the columns of the inverse of 2-by-2 pivot
*                 block D in the following way to reduce the number
*                 of FLOPS when we myltiply panel ( W(kw-1) W(kw) ) by
*                 this inverse
*
*                 D**(-1) = ( d11 d21 )**(-1) =
*                           ( d21 d22 )
*
*                 = 1/(d11*d22-d21**2) * ( ( d22 ) (-d21 ) ) =
*                                        ( (-d21 ) ( d11 ) )
*
*                 = 1/d21 * 1/((d11/d21)*(d22/d21)-1) *
*
*                   * ( ( d22/d21 ) (      -1 ) ) =
*                     ( (      -1 ) ( d11/d21 ) )
*
*                 = 1/d21 * 1/(D22*D11-1) * ( ( D11 ) (  -1 ) ) =
*                                           ( ( -1  ) ( D22 ) )
*
*                 = 1/d21 * T * ( ( D11 ) (  -1 ) )
```

```
*                              ( (  -1 ) ( D22 ) )
*
*                = D21 * ( ( D11 ) (  -1 ) )
*                        ( (  -1 ) ( D22 ) )
*
                 D21 = W( K-1, KW )
                 D11 = W( K, KW ) / D21
                 D22 = W( K-1, KW-1 ) / D21
                 T = ONE / ( D11*D22-ONE )
                 D21 = T / D21
*
*                Update elements in columns A(k-1) and A(k) as
*                dot products of rows of ( W(kw-1) W(kw) ) and columns
*                of D**(-1)
*
                 DO 20 J = 1, K - 2
                    A( J, K-1 ) = D21*( D11*W( J, KW-1 )-W( J, KW ) )
                    A( J, K ) = D21*( D22*W( J, KW )-W( J, KW-1 ) )
   20            CONTINUE
              END IF
*
*             Copy D(k) to A
*
              A( K-1, K-1 ) = W( K-1, KW-1 )
              A( K-1, K ) = W( K-1, KW )
              A( K, K ) = W( K, KW )
*
           END IF
*
        END IF
*
*       Store details of the interchanges in IPIV
*
        IF( KSTEP.EQ.1 ) THEN
           IPIV( K ) = KP
        ELSE
           IPIV( K ) = -KP
           IPIV( K-1 ) = -KP
        END IF
*
*       Decrease K and return to the start of the main loop
*
        K = K - KSTEP
        GO TO 10
*
   30   CONTINUE
*
*       Update the upper triangle of A11 (= A(1:k,1:k)) as
*
*       A11 := A11 - U12*D*U12**T = A11 - U12*W**T
```

```
*
*        computing blocks of NB columns at a time
*
         DO 50 J = ( ( K-1 ) / NB )*NB + 1, 1, -NB
            JB = MIN( NB, K-J+1 )
*
*           Update the upper triangle of the diagonal block
*
            DO 40 JJ = J, J + JB - 1
               CALL DGEMV( 'No transpose', JJ-J+1, N-K, -ONE,
     $                     A( J, K+1 ), LDA, W( JJ, KW+1 ), LDW, ONE,
     $                     A( J, JJ ), 1 )
   40       CONTINUE
*
*           Update the rectangular superdiagonal block
*
            CALL DGEMM( 'No transpose', 'Transpose', J-1, JB, N-K, -ONE,
     $                  A( 1, K+1 ), LDA, W( J, KW+1 ), LDW, ONE,
     $                  A( 1, J ), LDA )
   50    CONTINUE
*
*        Put U12 in standard form by partially undoing the interchanges
*        in columns k+1:n looping backwards from k+1 to n
*
         J = K + 1
   60    CONTINUE
*
*           Undo the interchanges (if any) of rows JJ and JP at each
*           step J
*
*           (Here, J is a diagonal index)
            JJ = J
            JP = IPIV( J )
            IF( JP.LT.0 ) THEN
               JP = -JP
*              (Here, J is a diagonal index)
               J = J + 1
            END IF
*           (NOTE: Here, J is used to determine row length. Length N-J+1
*           of the rows to swap back doesn't include diagonal element)
            J = J + 1
            IF( JP.NE.JJ .AND. J.LE.N )
     $         CALL DSWAP( N-J+1, A( JP, J ), LDA, A( JJ, J ), LDA )
         IF( J.LT.N )
     $      GO TO 60
*
*        Set KB to the number of columns factorized
*
         KB = N - K
*
```

```
      ELSE
*
*        Factorize the leading columns of A using the lower triangle
*        of A and working forwards, and compute the matrix W = L21*D
*        for use in updating A22
*
*        K is the main loop index, increasing from 1 in steps of 1 or 2
*
         K = 1
   70    CONTINUE
*
*        Exit from loop
*
         IF( ( K.GE.NB .AND. NB.LT.N ) .OR. K.GT.N )
     $      GO TO 90
*
*        Copy column K of A to column K of W and update it
*
         CALL DCOPY( N-K+1, A( K, K ), 1, W( K, K ), 1 )
         CALL DGEMV( 'No transpose', N-K+1, K-1, -ONE, A( K, 1 ), LDA,
     $               W( K, 1 ), LDW, ONE, W( K, K ), 1 )
*
         KSTEP = 1
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = ABS( W( K, K ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.LT.N ) THEN
            IMAX = K + IDAMAX( N-K, W( K+1, K ), 1 )
            COLMAX = ABS( W( IMAX, K ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( MAX( ABSAKK, COLMAX ).EQ.ZERO ) THEN
*
*           Column K is zero or underflow: set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
         ELSE
            IF( ABSAKK.GE.ALPHA*COLMAX ) THEN
*
```

```
*              no interchange, use 1-by-1 pivot block
*
               KP = K
            ELSE
*
*              Copy column IMAX to column K+1 of W and update it
*
               CALL DCOPY( IMAX-K, A( IMAX, K ), LDA, W( K, K+1 ), 1 )
               CALL DCOPY( N-IMAX+1, A( IMAX, IMAX ), 1, W( IMAX, K+1 ),
     $                     1 )
               CALL DGEMV( 'No transpose', N-K+1, K-1, -ONE, A( K, 1 ),
     $                     LDA, W( IMAX, 1 ), LDW, ONE, W( K, K+1 ), 1 )
*
*              JMAX is the column-index of the largest off-diagonal
*              element in row IMAX, and ROWMAX is its absolute value
*
               JMAX = K - 1 + IDAMAX( IMAX-K, W( K, K+1 ), 1 )
               ROWMAX = ABS( W( JMAX, K+1 ) )
               IF( IMAX.LT.N ) THEN
                  JMAX = IMAX + IDAMAX( N-IMAX, W( IMAX+1, K+1 ), 1 )
                  ROWMAX = MAX( ROWMAX, ABS( W( JMAX, K+1 ) ) )
               END IF
*
               IF( ABSAKK.GE.ALPHA*COLMAX*( COLMAX / ROWMAX ) ) THEN
*
*                 no interchange, use 1-by-1 pivot block
*
                  KP = K
               ELSE IF( ABS( W( IMAX, K+1 ) ).GE.ALPHA*ROWMAX ) THEN
*
*                 interchange rows and columns K and IMAX, use 1-by-1
*                 pivot block
*
                  KP = IMAX
*
*                 copy column K+1 of W to column K of W
*
                  CALL DCOPY( N-K+1, W( K, K+1 ), 1, W( K, K ), 1 )
               ELSE
*
*                 interchange rows and columns K+1 and IMAX, use 2-by-2
*                 pivot block
*
                  KP = IMAX
                  KSTEP = 2
               END IF
            END IF
*
*           ============================================================
*
```

```
*          KK is the column of A where pivoting step stopped
*
           KK = K + KSTEP - 1
*
*          Interchange rows and columns KP and KK.
*          Updated column KP is already stored in column KK of W.
*
           IF( KP.NE.KK ) THEN
*
*             Copy non-updated column KK to column KP of submatrix A
*             at step K. No need to copy element into column K
*             (or K and K+1 for 2-by-2 pivot) of A, since these columns
*             will be later overwritten.
*
              A( KP, KP ) = A( KK, KK )
              CALL DCOPY( KP-KK-1, A( KK+1, KK ), 1, A( KP, KK+1 ),
     $                    LDA )
              IF( KP.LT.N )
     $           CALL DCOPY( N-KP, A( KP+1, KK ), 1, A( KP+1, KP ), 1 )
*
*             Interchange rows KK and KP in first K-1 columns of A
*             (columns K (or K and K+1 for 2-by-2 pivot) of A will be
*             later overwritten). Interchange rows KK and KP
*             in first KK columns of W.
*
              IF( K.GT.1 )
     $           CALL DSWAP( K-1, A( KK, 1 ), LDA, A( KP, 1 ), LDA )
              CALL DSWAP( KK, W( KK, 1 ), LDW, W( KP, 1 ), LDW )
           END IF
*
           IF( KSTEP.EQ.1 ) THEN
*
*             1-by-1 pivot block D(k): column k of W now holds
*
*             W(k) = L(k)*D(k),
*
*             where L(k) is the k-th column of L
*
*             Store subdiag. elements of column L(k)
*             and 1-by-1 block D(k) in column k of A.
*             (NOTE: Diagonal element L(k,k) is a UNIT element
*             and not stored)
*                A(k,k) := D(k,k) = W(k,k)
*                A(k+1:N,k) := L(k+1:N,k) = W(k+1:N,k)/D(k,k)
*
              CALL DCOPY( N-K+1, W( K, K ), 1, A( K, K ), 1 )
              IF( K.LT.N ) THEN
                 R1 = ONE / A( K, K )
                 CALL DSCAL( N-K, R1, A( K+1, K ), 1 )
              END IF
```

```
*
            ELSE
*
*              2-by-2 pivot block D(k): columns k and k+1 of W now hold
*
*              ( W(k) W(k+1) ) = ( L(k) L(k+1) )*D(k)
*
*              where L(k) and L(k+1) are the k-th and (k+1)-th columns
*              of L
*
*              Store L(k+2:N,k) and L(k+2:N,k+1) and 2-by-2
*              block D(k:k+1,k:k+1) in columns k and k+1 of A.
*              (NOTE: 2-by-2 diagonal block L(k:k+1,k:k+1) is a UNIT
*              block and not stored)
*                 A(k:k+1,k:k+1) := D(k:k+1,k:k+1) = W(k:k+1,k:k+1)
*                 A(k+2:N,k:k+1) := L(k+2:N,k:k+1) =
*                 = W(k+2:N,k:k+1) * ( D(k:k+1,k:k+1)**(-1) )
*
               IF( K.LT.N-1 ) THEN
*
*                 Compose the columns of the inverse of 2-by-2 pivot
*                 block D in the following way to reduce the number
*                 of FLOPS when we myltiply panel ( W(k) W(k+1) ) by
*                 this inverse
*
*                 D**(-1) = ( d11 d21 )**(-1) =
*                           ( d21 d22 )
*
*                 = 1/(d11*d22-d21**2) * ( ( d22 ) (-d21 ) ) =
*                                        ( (-d21 ) ( d11 ) )
*
*                 = 1/d21 * 1/((d11/d21)*(d22/d21)-1) *
*
*                   * ( ( d22/d21 ) (      -1 ) ) =
*                     ( (      -1 ) ( d11/d21 ) )
*
*                 = 1/d21 * 1/(D22*D11-1) * ( ( D11 ) (  -1 ) ) =
*                                           ( ( -1  ) ( D22 ) )
*
*                 = 1/d21 * T * ( ( D11 ) (  -1 ) )
*                               ( (  -1 ) ( D22 ) )
*
*                 = D21 * ( ( D11 ) (  -1 ) )
*                         ( (  -1 ) ( D22 ) )
*
                  D21 = W( K+1, K )
                  D11 = W( K+1, K+1 ) / D21
                  D22 = W( K, K ) / D21
                  T = ONE / ( D11*D22-ONE )
                  D21 = T / D21
```

```
*
*                Update elements in columns A(k) and A(k+1) as
*                dot products of rows of ( W(k) W(k+1) ) and columns
*                of D**(-1)
*
                 DO 80 J = K + 2, N
                    A( J, K ) = D21*( D11*W( J, K )-W( J, K+1 ) )
                    A( J, K+1 ) = D21*( D22*W( J, K+1 )-W( J, K ) )
   80            CONTINUE
              END IF
*
*             Copy D(k) to A
*
              A( K, K ) = W( K, K )
              A( K+1, K ) = W( K+1, K )
              A( K+1, K+1 ) = W( K+1, K+1 )
*
           END IF
*
        END IF
*
*       Store details of the interchanges in IPIV
*
        IF( KSTEP.EQ.1 ) THEN
           IPIV( K ) = KP
        ELSE
           IPIV( K ) = -KP
           IPIV( K+1 ) = -KP
        END IF
*
*       Increase K and return to the start of the main loop
*
        K = K + KSTEP
        GO TO 70
*
   90   CONTINUE
*
*       Update the lower triangle of A22 (= A(k:n,k:n)) as
*
*       A22 := A22 - L21*D*L21**T = A22 - L21*W**T
*
*       computing blocks of NB columns at a time
*
        DO 110 J = K, N, NB
           JB = MIN( NB, N-J+1 )
*
*          Update the lower triangle of the diagonal block
*
           DO 100 JJ = J, J + JB - 1
              CALL DGEMV( 'No transpose', J+JB-JJ, K-1, -ONE,
```

```
     $                    A( JJ, 1 ), LDA, W( JJ, 1 ), LDW, ONE,
     $                    A( JJ, JJ ), 1 )
  100       CONTINUE
*
*           Update the rectangular subdiagonal block
*
            IF( J+JB.LE.N )
     $         CALL DGEMM( 'No transpose', 'Transpose', N-J-JB+1, JB,
     $                    K-1, -ONE, A( J+JB, 1 ), LDA, W( J, 1 ), LDW,
     $                    ONE, A( J+JB, J ), LDA )
  110    CONTINUE
*
*        Put L21 in standard form by partially undoing the interchanges
*        of rows in columns 1:k-1 looping backwards from k-1 to 1
*
         J = K - 1
  120    CONTINUE
*
*           Undo the interchanges (if any) of rows JJ and JP at each
*           step J
*
*           (Here, J is a diagonal index)
            JJ = J
            JP = IPIV( J )
            IF( JP.LT.0 ) THEN
               JP = -JP
*              (Here, J is a diagonal index)
               J = J - 1
            END IF
*           (NOTE: Here, J is used to determine row length. Length J
*           of the rows to swap back doesn't include diagonal element)
            J = J - 1
            IF( JP.NE.JJ .AND. J.GE.1 )
     $         CALL DSWAP( J, A( JP, 1 ), LDA, A( JJ, 1 ), LDA )
         IF( J.GT.1 )
     $      GO TO 120
*
*        Set KB to the number of columns factorized
*
         KB = K - 1
*
      END IF
      RETURN
*
*     End of DLASYF
*
      END
```

———-

58.5.4 lisp code

— dlasyf.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0) (eight 8.0d0) (sevten 17.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 8.0d0 8.0d0) eight)
           (type (double-float 17.0d0 17.0d0) sevten)
           (ignorable zero one eight sevten))
  (defun dlasyf (uplo n nb kb a lda ipiv w ldw info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array double-float (*)) w a)
             (type (f2cl-lib:integer4) info ldw lda kb nb n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (w double-float w-%data% w-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((absakk 0.0d0) (alpha 0.0d0) (colmax 0.0d0) (d11 0.0d0)
             (d21 0.0d0) (d22 0.0d0) (r1 0.0d0) (rowmax 0.0d0) (t$ 0.0d0)
             (imax 0) (j 0) (jb 0) (jj 0) (jmax 0) (jp 0) (k 0) (kk 0) (kkw 0)
             (kp 0) (kstep 0) (kw 0))
        (declare
         (type (double-float) absakk alpha colmax d11 d21 d22 r1 rowmax t$)
         (type (f2cl-lib:integer4) imax j jb jj jmax jp k kk kkw kp kstep kw))
        (setf info 0)
        (setf alpha (/ (+ one (f2cl-lib:fsqrt sevten)) eight))
        (cond
         ((multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)
          (tagbody
            (setf k n)
           label10
            (setf kw (f2cl-lib:int-sub (f2cl-lib:int-add nb k) n))
            (if (or
                 (and (<= k (f2cl-lib:int-add (f2cl-lib:int-sub n nb) 1))
                      (< nb n))
                 (< k 1))
                (go label30))
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (dcopy k
                 (f2cl-lib:array-slice a-%data% double-float (1 k)
                                       ((1 lda) (1 *)) a-%offset%)
                 1
```

```
      (f2cl-lib:array-slice w-%data% double-float (1 kw)
                            ((1 ldw) (1 *)) w-%offset%)
      1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf k var-0)))
(if (< k n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10)
        (dgemv "No transpose" k (f2cl-lib:int-sub n k) (- one)
         (f2cl-lib:array-slice a-%data% double-float
                               (1 (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice w-%data% double-float
                               (k (f2cl-lib:int-add kw 1))
                               ((1 ldw) (1 *)) w-%offset%)
         ldw one
         (f2cl-lib:array-slice w-%data% double-float (1 kw)
                               ((1 ldw) (1 *)) w-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3 var-4 var-6 var-9 var-10))
      (when var-1 (setf k var-1))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldw var-7))
      (when var-8 (setf one var-8))))
(setf kstep 1)
(setf absakk
        (abs
         (f2cl-lib:fref w-%data% (k kw) ((1 ldw) (1 *))
                        w-%offset%)))
(cond
 ((> k 1)
  (setf imax
          (idamax (f2cl-lib:int-sub k 1)
           (f2cl-lib:array-slice w-%data% double-float (1 kw)
                                 ((1 ldw) (1 *)) w-%offset%)
           1))
  (setf colmax
          (abs
           (f2cl-lib:fref w-%data% (imax kw) ((1 ldw) (1 *))
                          w-%offset%))))
 (t (setf colmax zero)))
(cond
 ((= (max absakk colmax) zero)
  (if (= info 0)
      (setf info k))
  (setf kp k))
 (t
  (cond ((>= absakk (* alpha colmax)) (setf kp k))
```

```
(t
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (dcopy imax
      (f2cl-lib:array-slice a-%data% double-float (1 imax)
                            ((1 lda) (1 *)) a-%offset%)
      1
      (f2cl-lib:array-slice w-%data% double-float
                            (1 (f2cl-lib:int-sub kw 1))
                            ((1 ldw) (1 *)) w-%offset%)
      1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf imax var-0)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (dcopy (f2cl-lib:int-sub k imax)
      (f2cl-lib:array-slice a-%data% double-float
                            (imax
                             (f2cl-lib:int-add imax 1))
                            ((1 lda) (1 *)) a-%offset%)
      lda
      (f2cl-lib:array-slice w-%data% double-float
                            ((+ imax 1)
                             (f2cl-lib:int-sub kw 1))
                            ((1 ldw) (1 *)) w-%offset%)
      1)
   (declare (ignore var-0 var-1 var-3 var-4))
   (when var-2 (setf lda var-2)))
 (if (< k n)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
          var-8 var-9 var-10)
         (dgemv "No transpose" k (f2cl-lib:int-sub n k)
          (- one)
          (f2cl-lib:array-slice a-%data% double-float
                                (1 (f2cl-lib:int-add k 1))
                                ((1 lda) (1 *)) a-%offset%)
          lda
          (f2cl-lib:array-slice w-%data% double-float
                                (imax
                                 (f2cl-lib:int-add kw 1))
                                ((1 ldw) (1 *)) w-%offset%)
          ldw one
          (f2cl-lib:array-slice w-%data% double-float
                                (1 (f2cl-lib:int-sub kw 1))
                                ((1 ldw) (1 *)) w-%offset%)
          1)
       (declare
        (ignore var-0 var-2 var-3 var-4 var-6 var-9
         var-10))
       (when var-1 (setf k var-1))
       (when var-5 (setf lda var-5))
```

```
      (when var-7 (setf ldw var-7))
      (when var-8 (setf one var-8))))
(setf jmax
        (f2cl-lib:int-add imax
                          (idamax
                           (f2cl-lib:int-sub k imax)
                           (f2cl-lib:array-slice w-%data%
                                                 double-float
                                                 ((+ imax
                                                     1)
                                                  (f2cl-lib:int-sub
                                                   kw 1))
                                                 ((1 ldw)
                                                  (1 *))
                                                 w-%offset%)
                           1)))
(setf rowmax
        (abs
         (f2cl-lib:fref w-%data%
                        (jmax (f2cl-lib:int-sub kw 1))
                        ((1 ldw) (1 *)) w-%offset%)))
(cond
 ((> imax 1)
  (setf jmax
          (idamax (f2cl-lib:int-sub imax 1)
           (f2cl-lib:array-slice w-%data% double-float
                                 (1
                                  (f2cl-lib:int-sub kw 1))
                                 ((1 ldw) (1 *))
                                 w-%offset%)
           1))
  (setf rowmax
          (max rowmax
               (abs
                (f2cl-lib:fref w-%data%
                               (jmax
                                (f2cl-lib:int-sub kw 1))
                               ((1 ldw) (1 *))
                               w-%offset%))))))
(cond
 ((>= absakk
      (* alpha colmax (f2cl-lib:f2cl/ colmax rowmax)))
  (setf kp k))
 ((>=
   (abs
    (f2cl-lib:fref w
                   (imax
                    (f2cl-lib:int-add kw
                                      (f2cl-lib:int-sub
                                       1)))
```

```
                                ((1 ldw) (1 *))))
                 (* alpha rowmax))
                (setf kp imax)
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (dcopy k
                     (f2cl-lib:array-slice w-%data% double-float
                                           (1 (f2cl-lib:int-sub kw 1))
                                           ((1 ldw) (1 *)) w-%offset%)
                     1
                     (f2cl-lib:array-slice w-%data% double-float (1 kw)
                                           ((1 ldw) (1 *)) w-%offset%)
                     1)
                  (declare (ignore var-1 var-2 var-3 var-4))
                  (when var-0 (setf k var-0))))
               (t (setf kp imax) (setf kstep 2)))))
(setf kk (f2cl-lib:int-add (f2cl-lib:int-sub k kstep) 1))
(setf kkw (f2cl-lib:int-sub (f2cl-lib:int-add nb kk) n))
(cond
 ((/= kp kk)
  (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                       a-%offset%)
          (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                         a-%offset%))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy (f2cl-lib:int-sub kk 1 kp)
       (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) kk)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% double-float
                             (kp (f2cl-lib:int-add kp 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (if (> kp 1)
      (dcopy (f2cl-lib:int-sub kp 1)
       (f2cl-lib:array-slice a-%data% double-float (1 kk)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% double-float (1 kp)
                             ((1 lda) (1 *)) a-%offset%)
       1))
  (if (< k n)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dswap (f2cl-lib:int-sub n k)
           (f2cl-lib:array-slice a-%data% double-float
                                 (kk (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice a-%data% double-float
```

```
                                    (kp (f2cl-lib:int-add k 1))
                                    ((1 lda) (1 *)) a-%offset%)
            lda)
          (declare (ignore var-0 var-1 var-3))
          (when var-2 (setf lda var-2))
          (when var-4 (setf lda var-4))))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dswap (f2cl-lib:int-add (f2cl-lib:int-sub n kk) 1)
       (f2cl-lib:array-slice w-%data% double-float (kk kkw)
                             ((1 ldw) (1 *)) w-%offset%)
       ldw
       (f2cl-lib:array-slice w-%data% double-float (kp kkw)
                             ((1 ldw) (1 *)) w-%offset%)
       ldw)
    (declare (ignore var-0 var-1 var-3))
    (when var-2 (setf ldw var-2))
    (when var-4 (setf ldw var-4)))))
(cond
 ((= kstep 1)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy k
       (f2cl-lib:array-slice w-%data% double-float (1 kw)
                             ((1 ldw) (1 *)) w-%offset%)
       1
       (f2cl-lib:array-slice a-%data% double-float (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf k var-0)))
  (setf r1
          (/ one
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dscal (f2cl-lib:int-sub k 1) r1
       (f2cl-lib:array-slice a-%data% double-float (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1)
    (declare (ignore var-0 var-2 var-3))
    (when var-1 (setf r1 var-1))))
 (t
  (cond
   ((> k 2)
    (setf d21
            (f2cl-lib:fref w-%data% ((f2cl-lib:int-sub k 1) kw)
                           ((1 ldw) (1 *)) w-%offset%))
    (setf d11
            (/
             (f2cl-lib:fref w-%data% (k kw) ((1 ldw) (1 *))
                            w-%offset%)
```

```
          d21))
  (setf d22
         (/
          (f2cl-lib:fref w-%data%
                         ((f2cl-lib:int-sub k 1)
                          (f2cl-lib:int-sub kw 1))
                         ((1 ldw) (1 *)) w-%offset%)
          d21))
  (setf t$ (/ one (- (* d11 d22) one))) (setf d21 (/ t$ d21))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j
                    (f2cl-lib:int-add k (f2cl-lib:int-sub 2)))
                 nil)
    (tagbody
      (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-sub k 1))
                           ((1 lda) (1 *)) a-%offset%)
              (* d21
                 (-
                  (* d11
                     (f2cl-lib:fref w-%data%
                                    (j (f2cl-lib:int-sub kw 1))
                                    ((1 ldw) (1 *))
                                    w-%offset%))
                  (f2cl-lib:fref w-%data% (j kw)
                                 ((1 ldw) (1 *)) w-%offset%))))
      (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                           a-%offset%)
              (* d21
                 (-
                  (* d22
                     (f2cl-lib:fref w-%data% (j kw)
                                    ((1 ldw) (1 *))
                                    w-%offset%))
                  (f2cl-lib:fref w-%data%
                                 (j (f2cl-lib:int-sub kw 1))
                                 ((1 ldw) (1 *)) w-%offset%))))
     label20))))
(setf (f2cl-lib:fref a-%data%
                     ((f2cl-lib:int-sub k 1)
                      (f2cl-lib:int-sub k 1))
                     ((1 lda) (1 *)) a-%offset%)
        (f2cl-lib:fref w-%data%
                       ((f2cl-lib:int-sub k 1)
                        (f2cl-lib:int-sub kw 1))
                       ((1 ldw) (1 *)) w-%offset%))
(setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                     ((1 lda) (1 *)) a-%offset%)
        (f2cl-lib:fref w-%data% ((f2cl-lib:int-sub k 1) kw)
                       ((1 ldw) (1 *)) w-%offset%))
(setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
```

```
                (f2cl-lib:fref w-%data% (k kw) ((1 ldw) (1 *))
                               w-%offset%))))))
 (cond
  ((= kstep 1)
   (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%) kp))
  (t
   (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
           (f2cl-lib:int-sub kp))
   (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                        ipiv-%offset%)
           (f2cl-lib:int-sub kp))))
 (setf k (f2cl-lib:int-sub k kstep))
 (go label10)
label30
 (f2cl-lib:fdo (j
                (+
                 (*
                  (the f2cl-lib:integer4 (truncate (+ k (- 1)) nb))
                  nb)
                 1)
                (f2cl-lib:int-add j (f2cl-lib:int-sub nb)))
               ((> j 1) nil)
   (tagbody
     (setf jb
             (min (the f2cl-lib:integer4 nb)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add (f2cl-lib:int-sub k j)
                                         1))))
     (f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
                   ((> jj
                       (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
                    nil)
       (tagbody
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9 var-10)
             (dgemv "No transpose"
              (f2cl-lib:int-add (f2cl-lib:int-sub jj j) 1)
              (f2cl-lib:int-sub n k) (- one)
              (f2cl-lib:array-slice a-%data% double-float
                                    (j (f2cl-lib:int-add k 1))
                                    ((1 lda) (1 *)) a-%offset%)
              lda
              (f2cl-lib:array-slice w-%data% double-float
                                    (jj (f2cl-lib:int-add kw 1))
                                    ((1 ldw) (1 *)) w-%offset%)
              ldw one
              (f2cl-lib:array-slice a-%data% double-float (j jj)
                                    ((1 lda) (1 *)) a-%offset%)
              1)
```

```
                (declare
                 (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9
                  var-10))
                (when var-5 (setf lda var-5))
                (when var-7 (setf ldw var-7))
                (when var-8 (setf one var-8)))
            label40))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12)
            (dgemm "No transpose" "Transpose" (f2cl-lib:int-sub j 1) jb
             (f2cl-lib:int-sub n k) (- one)
             (f2cl-lib:array-slice a-%data% double-float
                                   (1 (f2cl-lib:int-add k 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice w-%data% double-float
                                   (j (f2cl-lib:int-add kw 1))
                                   ((1 ldw) (1 *)) w-%offset%)
             ldw one
             (f2cl-lib:array-slice a-%data% double-float (1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare
           (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-8 var-11))
          (when var-3 (setf jb var-3))
          (when var-7 (setf lda var-7))
          (when var-9 (setf ldw var-9))
          (when var-10 (setf one var-10))
          (when var-12 (setf lda var-12)))
      label50))
  (setf j (f2cl-lib:int-add k 1))
 label60
  (setf jj j)
  (setf jp (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
  (cond
   ((< jp 0) (setf jp (f2cl-lib:int-sub jp))
    (setf j (f2cl-lib:int-add j 1))))
  (setf j (f2cl-lib:int-add j 1))
  (if (and (/= jp jj) (<= j n))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dswap (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1)
           (f2cl-lib:array-slice a-%data% double-float (jp j)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice a-%data% double-float (jj j)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-3))
        (when var-2 (setf lda var-2))
```

```
            (when var-4 (setf lda var-4))))
      (if (< j n)
          (go label60))
      (setf kb (f2cl-lib:int-sub n k))))
    (t
     (tagbody
      (setf k 1)
     label70
      (if (or (and (>= k nb) (< nb n)) (> k n))
          (go label90))
      (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
       (f2cl-lib:array-slice a-%data% double-float (k k) ((1 lda) (1 *))
                             a-%offset%)
       1
       (f2cl-lib:array-slice w-%data% double-float (k k) ((1 ldw) (1 *))
                             w-%offset%)
       1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (dgemv "No transpose"
           (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
           (f2cl-lib:int-sub k 1) (- one)
           (f2cl-lib:array-slice a-%data% double-float (k 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice w-%data% double-float (k 1)
                                 ((1 ldw) (1 *)) w-%offset%)
           ldw one
           (f2cl-lib:array-slice w-%data% double-float (k k)
                                 ((1 ldw) (1 *)) w-%offset%)
           1)
        (declare
         (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9 var-10))
        (when var-5 (setf lda var-5))
        (when var-7 (setf ldw var-7))
        (when var-8 (setf one var-8)))
      (setf kstep 1)
      (setf absakk
              (abs
               (f2cl-lib:fref w-%data% (k k) ((1 ldw) (1 *))
                              w-%offset%)))
      (cond
       ((< k n)
        (setf imax
                (f2cl-lib:int-add k
                                  (idamax (f2cl-lib:int-sub n k)
                                   (f2cl-lib:array-slice w-%data%
                                                         double-float
                                                         ((+ k 1) k)
```

```
                                                        ((1 ldw) (1 *))
                                                        w-%offset%)
                                 1)))
  (setf colmax
          (abs
           (f2cl-lib:fref w-%data% (imax k) ((1 ldw) (1 *))
                          w-%offset%))))
 (t (setf colmax zero)))
(cond
 ((= (max absakk colmax) zero)
  (if (= info 0)
      (setf info k))
  (setf kp k))
 (t
  (cond ((>= absakk (* alpha colmax)) (setf kp k))
        (t
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (dcopy (f2cl-lib:int-sub imax k)
              (f2cl-lib:array-slice a-%data% double-float (imax k)
                                    ((1 lda) (1 *)) a-%offset%)
              lda
              (f2cl-lib:array-slice w-%data% double-float
                                    (k (f2cl-lib:int-add k 1))
                                    ((1 ldw) (1 *)) w-%offset%)
              1)
           (declare (ignore var-0 var-1 var-3 var-4))
           (when var-2 (setf lda var-2)))
         (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub n imax) 1)
          (f2cl-lib:array-slice a-%data% double-float (imax imax)
                                ((1 lda) (1 *)) a-%offset%)
          1
          (f2cl-lib:array-slice w-%data% double-float
                                (imax (f2cl-lib:int-add k 1))
                                ((1 ldw) (1 *)) w-%offset%)
          1)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9 var-10)
             (dgemv "No transpose"
              (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
              (f2cl-lib:int-sub k 1) (- one)
              (f2cl-lib:array-slice a-%data% double-float (k 1)
                                    ((1 lda) (1 *)) a-%offset%)
              lda
              (f2cl-lib:array-slice w-%data% double-float (imax 1)
                                    ((1 ldw) (1 *)) w-%offset%)
              ldw one
              (f2cl-lib:array-slice w-%data% double-float
                                    (k (f2cl-lib:int-add k 1))
                                    ((1 ldw) (1 *)) w-%offset%)
```

```
     1)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9
    var-10))
  (when var-5 (setf lda var-5))
  (when var-7 (setf ldw var-7))
  (when var-8 (setf one var-8)))
(setf jmax
        (f2cl-lib:int-add (f2cl-lib:int-sub k 1)
                          (idamax
                           (f2cl-lib:int-sub imax k)
                           (f2cl-lib:array-slice w-%data%
                                                 double-float
                                                 (k
                                                  (f2cl-lib:int-add
                                                   k 1))
                                                 ((1 ldw)
                                                  (1 *))
                                                 w-%offset%)
                           1)))
(setf rowmax
        (abs
         (f2cl-lib:fref w-%data%
                        (jmax (f2cl-lib:int-add k 1))
                        ((1 ldw) (1 *)) w-%offset%)))
(cond
 ((< imax n)
  (setf jmax
          (f2cl-lib:int-add imax
                            (idamax
                             (f2cl-lib:int-sub n imax)
                             (f2cl-lib:array-slice
                              w-%data% double-float
                              ((+ imax 1)
                               (f2cl-lib:int-add k 1))
                              ((1 ldw) (1 *)) w-%offset%)
                             1)))
  (setf rowmax
          (max rowmax
               (abs
                (f2cl-lib:fref w-%data%
                               (jmax
                                (f2cl-lib:int-add k 1))
                               ((1 ldw) (1 *))
                               w-%offset%))))))
(cond
 ((>= absakk
      (* alpha colmax (f2cl-lib:f2cl/ colmax rowmax)))
  (setf kp k))
 ((>=
```

```
          (abs
           (f2cl-lib:fref w (imax (f2cl-lib:int-add k 1))
                          ((1 ldw) (1 *))))
          (* alpha rowmax))
         (setf kp imax)
         (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
          (f2cl-lib:array-slice w-%data% double-float
                                (k (f2cl-lib:int-add k 1))
                                ((1 ldw) (1 *)) w-%offset%)
          1
          (f2cl-lib:array-slice w-%data% double-float (k k)
                                ((1 ldw) (1 *)) w-%offset%)
          1))
        (t (setf kp imax) (setf kstep 2)))))
(setf kk (f2cl-lib:int-sub (f2cl-lib:int-add k kstep) 1))
(cond
 ((/= kp kk)
  (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                       a-%offset%)
          (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                         a-%offset%))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy (f2cl-lib:int-sub kp kk 1)
       (f2cl-lib:array-slice a-%data% double-float ((+ kk 1) kk)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% double-float
                             (kp (f2cl-lib:int-add kk 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (if (< kp n)
      (dcopy (f2cl-lib:int-sub n kp)
       (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) kk)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) kp)
                             ((1 lda) (1 *)) a-%offset%)
       1))
  (if (> k 1)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dswap (f2cl-lib:int-sub k 1)
           (f2cl-lib:array-slice a-%data% double-float (kk 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice a-%data% double-float (kp 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-3))
```

```
          (when var-2 (setf lda var-2))
          (when var-4 (setf lda var-4))))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dswap kk
         (f2cl-lib:array-slice w-%data% double-float (kk 1)
                               ((1 ldw) (1 *)) w-%offset%)
         ldw
         (f2cl-lib:array-slice w-%data% double-float (kp 1)
                               ((1 ldw) (1 *)) w-%offset%)
         ldw)
      (declare (ignore var-1 var-3))
      (when var-0 (setf kk var-0))
      (when var-2 (setf ldw var-2))
      (when var-4 (setf ldw var-4)))))
  (cond
   ((= kstep 1)
    (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
     (f2cl-lib:array-slice w-%data% double-float (k k)
                           ((1 ldw) (1 *)) w-%offset%)
     1
     (f2cl-lib:array-slice a-%data% double-float (k k)
                           ((1 lda) (1 *)) a-%offset%)
     1)
    (cond
     ((< k n)
      (setf r1
              (/ one
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal (f2cl-lib:int-sub n k) r1
           (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                                 ((1 lda) (1 *)) a-%offset%)
           1)
        (declare (ignore var-0 var-2 var-3))
        (when var-1 (setf r1 var-1))))))
   (t
    (cond
     ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
      (setf d21
              (f2cl-lib:fref w-%data% ((f2cl-lib:int-add k 1) k)
                             ((1 ldw) (1 *)) w-%offset%))
      (setf d11
              (/
               (f2cl-lib:fref w-%data%
                              ((f2cl-lib:int-add k 1)
                               (f2cl-lib:int-add k 1))
                              ((1 ldw) (1 *)) w-%offset%)
               d21))
      (setf d22
```

```
                 (/
                  (f2cl-lib:fref w-%data% (k k) ((1 ldw) (1 *))
                                 w-%offset%)
                  d21))
          (setf t$ (/ one (- (* d11 d22) one))) (setf d21 (/ t$ d21))
          (f2cl-lib:fdo (j (f2cl-lib:int-add k 2)
                         (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                   a-%offset%)
                      (* d21
                         (-
                          (* d11
                             (f2cl-lib:fref w-%data% (j k)
                                            ((1 ldw) (1 *))
                                            w-%offset%))
                          (f2cl-lib:fref w-%data%
                                         (j (f2cl-lib:int-add k 1))
                                         ((1 ldw) (1 *)) w-%offset%))))
              (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-add k 1))
                                   ((1 lda) (1 *)) a-%offset%)
                      (* d21
                         (-
                          (* d22
                             (f2cl-lib:fref w-%data%
                                            (j (f2cl-lib:int-add k 1))
                                            ((1 ldw) (1 *))
                                            w-%offset%))
                          (f2cl-lib:fref w-%data% (j k) ((1 ldw) (1 *))
                                         w-%offset%))))
            label80))))
      (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref w-%data% (k k) ((1 ldw) (1 *))
                             w-%offset%))
      (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                           ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref w-%data% ((f2cl-lib:int-add k 1) k)
                             ((1 ldw) (1 *)) w-%offset%))
      (setf (f2cl-lib:fref a-%data%
                           ((f2cl-lib:int-add k 1)
                            (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref w-%data%
                             ((f2cl-lib:int-add k 1)
                              (f2cl-lib:int-add k 1))
                             ((1 ldw) (1 *)) w-%offset%))))))
(cond
 ((= kstep 1)
  (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%) kp))
```

```
  (t
   (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
           (f2cl-lib:int-sub kp))
   (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1)) ((1 *))
                        ipiv-%offset%)
           (f2cl-lib:int-sub kp))))
 (setf k (f2cl-lib:int-add k kstep))
 (go label70)
label90
 (f2cl-lib:fdo (j k (f2cl-lib:int-add j nb))
               ((> j n) nil)
   (tagbody
     (setf jb
             (min (the f2cl-lib:integer4 nb)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add (f2cl-lib:int-sub n j)
                                         1))))
     (f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
                   ((> jj
                       (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
                    nil)
       (tagbody
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9 var-10)
             (dgemv "No transpose"
              (f2cl-lib:int-sub (f2cl-lib:int-add j jb) jj)
              (f2cl-lib:int-sub k 1) (- one)
              (f2cl-lib:array-slice a-%data% double-float (jj 1)
                                    ((1 lda) (1 *)) a-%offset%)
              lda
              (f2cl-lib:array-slice w-%data% double-float (jj 1)
                                    ((1 ldw) (1 *)) w-%offset%)
              ldw one
              (f2cl-lib:array-slice a-%data% double-float (jj jj)
                                    ((1 lda) (1 *)) a-%offset%)
              1)
           (declare
            (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9
             var-10))
           (when var-5 (setf lda var-5))
           (when var-7 (setf ldw var-7))
           (when var-8 (setf one var-8)))
       label100))
     (if (<= (f2cl-lib:int-add j jb) n)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9 var-10 var-11 var-12)
             (dgemm "No transpose" "Transpose"
              (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) jb
```

```
                        (f2cl-lib:int-sub k 1) (- one)
                        (f2cl-lib:array-slice a-%data% double-float
                                              ((+ j jb) 1) ((1 lda) (1 *))
                                              a-%offset%)
                        lda
                        (f2cl-lib:array-slice w-%data% double-float (j 1)
                                              ((1 ldw) (1 *)) w-%offset%)
                        ldw one
                        (f2cl-lib:array-slice a-%data% double-float
                                              ((+ j jb) j) ((1 lda) (1 *))
                                              a-%offset%)
                        lda)
                      (declare
                       (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-8
                        var-11))
                      (when var-3 (setf jb var-3))
                      (when var-7 (setf lda var-7))
                      (when var-9 (setf ldw var-9))
                      (when var-10 (setf one var-10))
                      (when var-12 (setf lda var-12))))
                 label110))
             (setf j (f2cl-lib:int-sub k 1))
            label120
             (setf jj j)
             (setf jp (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
             (cond
              ((< jp 0) (setf jp (f2cl-lib:int-sub jp))
               (setf j (f2cl-lib:int-sub j 1))))
             (setf j (f2cl-lib:int-sub j 1))
             (if (and (/= jp jj) (>= j 1))
                 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                     (dswap j
                      (f2cl-lib:array-slice a-%data% double-float (jp 1)
                                            ((1 lda) (1 *)) a-%offset%)
                      lda
                      (f2cl-lib:array-slice a-%data% double-float (jj 1)
                                            ((1 lda) (1 *)) a-%offset%)
                      lda)
                   (declare (ignore var-1 var-3))
                   (when var-0 (setf j var-0))
                   (when var-2 (setf lda var-2))
                   (when var-4 (setf lda var-4))))
             (if (> j 1)
                 (go label120))
             (setf kb (f2cl-lib:int-sub k 1)))))
         (go end_label)
        end_label
         (return (values uplo nil nil kb nil lda nil nil ldw info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
```

```
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlasyf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::kb nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::ldw
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

58.6 dlasyf_rook

58.6.1 Axiom unit tests

— **dlasyfrook.input** —

```
)set break resume
)sys rm -f dlasyfrook.output
)spool dlasyfrook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.6.2 Axiom help page

— dlasyf_rook.help —

```
=====================================================================
dlasyf_rook examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlasyf_rook (UPLO, N, NB, KB, A, LDA, IPIV, W, LDW, INFO)
           DLASYF_ROOK *> DLASYF_ROOK computes a partial factorization of a
           real symmetric matrix using the bounded Bunch-Kaufman ('rook')
           diagonal pivoting method.

Function/Subroutine Documentation
   subroutine dlasyf_rook (character UPLO, integer N, integer NB, integer KB,
       double precision, dimension( lda, * ) A, integer LDA, integer,
       dimension( * ) IPIV, double precision, dimension( ldw, * ) W, integer
       LDW, integer INFO)
       DLASYF_ROOK *> DLASYF_ROOK computes a partial factorization of a real
       symmetric matrix using the bounded Bunch-Kaufman ('rook') diagonal
       pivoting method.

Purpose:
   DLASYF_ROOK computes a partial factorization of a real symmetric
   matrix A using the bounded Bunch-Kaufman ("rook") diagonal
   pivoting method. The partial factorization has the form:

       A  =  ( I  U12 ) ( A11  0  ) (  I       0    )  if UPLO = 'U', or:
             ( 0  U22 ) (  0   D  ) ( U12**T U22**T )

       A  =  ( L11  0 ) (  D   0  ) ( L11**T L21**T )  if UPLO = 'L'
             ( L21  I ) (  0  A22 ) (  0       I    )

   where the order of D is at most NB. The actual order is returned in
   the argument KB, and is either NB or NB-1, or N if N <= NB.

   DLASYF_ROOK is an auxiliary routine called by DSYTRF_ROOK. It uses
   blocked code (calling Level 3 BLAS) to update the submatrix
   A11 (if UPLO = 'U') or A22 (if UPLO = 'L').

Parameters:

 UPLO
```

```
  UPLO is CHARACTER*1
  Specifies whether the upper or lower triangular part of the
  symmetric matrix A is stored:
  = 'U':  Upper triangular
  = 'L':  Lower triangular

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

NB
  NB is INTEGER
  The maximum number of columns of the matrix A that should be
  factored.  NB should be at least 2 to allow for 2-by-2 pivot blocks.

KB
  KB is INTEGER
  The number of columns of A that were actually factored.
  KB is either NB-1 or NB, or N if N <= NB.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  n-by-n upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading n-by-n lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.
  On exit, A contains details of the partial factorization.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D.

  If UPLO = 'U':
     Only the last KB elements of IPIV are set.

     If IPIV(k) > 0, then rows and columns k and IPIV(k) were
     interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) < 0 and IPIV(k-1) < 0, then rows and
     columns k and -IPIV(k) were interchanged and rows and
     columns k-1 and -IPIV(k-1) were inerchaged,
     D(k-1:k,k-1:k) is a 2-by-2 diagonal block.
```

```
   If UPLO = 'L':
      Only the first KB elements of IPIV are set.

      If IPIV(k) > 0, then rows and columns k and IPIV(k)
      were interchanged and D(k,k) is a 1-by-1 diagonal block.

      If IPIV(k) < 0 and IPIV(k+1) < 0, then rows and
      columns k and -IPIV(k) were interchanged and rows and
      columns k+1 and -IPIV(k+1) were inerchaged,
      D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

 W
   W is DOUBLE PRECISION array, dimension (LDW,NB)

 LDW
   LDW is INTEGER
   The leading dimension of the array W.  LDW >= max(1,N).

 INFO
   INFO is INTEGER
   = 0: successful exit
   > 0: if INFO = k, D(k,k) is exactly zero.  The factorization
        has been completed, but the block diagonal matrix D is
        exactly singular.
```

58.6.3 fortran code

— dlasyf_rook.f —

```
*  ===================================================================
      SUBROUTINE DLASYF_ROOK( UPLO, N, NB, KB, A, LDA, IPIV, W, LDW,
     $                        INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, KB, LDA, LDW, N, NB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), W( LDW, * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   EIGHT, SEVTEN
      PARAMETER          ( EIGHT = 8.0D+0, SEVTEN = 17.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            DONE
      INTEGER            IMAX, ITEMP, J, JB, JJ, JMAX, JP1, JP2, K, KK,
     $                   KW, KKW, KP, KSTEP, P, II

      DOUBLE PRECISION   ABSAKK, ALPHA, COLMAX, D11, D12, D21, D22,
     $                   DTEMP, R1, ROWMAX, T, SFMIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, IDAMAX, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DGEMM, DGEMV, DSCAL, DSWAP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
*     Initialize ALPHA for use in choosing pivot block size.
*
      ALPHA = ( ONE+SQRT( SEVTEN ) ) / EIGHT
*
*     Compute machine safe minimum
*
      SFMIN = DLAMCH( 'S' )
*
      IF( LSAME( UPLO, 'U' ) ) THEN
*
*        Factorize the trailing columns of A using the upper triangle
*        of A and working backwards, and compute the matrix W = U12*D
*        for use in updating A11
*
*        K is the main loop index, decreasing from N in steps of 1 or 2
*
```

```
         K = N
   10    CONTINUE
*
*        KW is the column of W which corresponds to column K of A
*
         KW = NB + K - N
*
*        Exit from loop
*
         IF( ( K.LE.N-NB+1 .AND. NB.LT.N ) .OR. K.LT.1 )
     $      GO TO 30
*
         KSTEP = 1
         P = K
*
*        Copy column K of A to column KW of W and update it
*
         CALL DCOPY( K, A( 1, K ), 1, W( 1, KW ), 1 )
         IF( K.LT.N )
     $      CALL DGEMV( 'No transpose', K, N-K, -ONE, A( 1, K+1 ),
     $                  LDA, W( K, KW+1 ), LDW, ONE, W( 1, KW ), 1 )
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = ABS( W( K, KW ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.GT.1 ) THEN
            IMAX = IDAMAX( K-1, W( 1, KW ), 1 )
            COLMAX = ABS( W( IMAX, KW ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( MAX( ABSAKK, COLMAX ).EQ.ZERO ) THEN
*
*           Column K is zero or underflow: set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
            CALL DCOPY( K, W( 1, KW ), 1, A( 1, K ), 1 )
         ELSE
*
*           ============================================================
*
```

```
*           Test for interchange
*
*           Equivalent to testing for ABSAKK.GE.ALPHA*COLMAX
*           (used to handle NaN and Inf)
*
            IF( .NOT.( ABSAKK.LT.ALPHA*COLMAX ) ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
*
            ELSE
*
               DONE = .FALSE.
*
*              Loop until pivot found
*
   12          CONTINUE
*
*                 Begin pivot search loop body
*
*
*                 Copy column IMAX to column KW-1 of W and update it
*
                  CALL DCOPY( IMAX, A( 1, IMAX ), 1, W( 1, KW-1 ), 1 )
                  CALL DCOPY( K-IMAX, A( IMAX, IMAX+1 ), LDA,
     $                        W( IMAX+1, KW-1 ), 1 )
*
                  IF( K.LT.N )
     $               CALL DGEMV( 'No transpose', K, N-K, -ONE,
     $                           A( 1, K+1 ), LDA, W( IMAX, KW+1 ), LDW,
     $                           ONE, W( 1, KW-1 ), 1 )
*
*                 JMAX is the column-index of the largest off-diagonal
*                 element in row IMAX, and ROWMAX is its absolute value.
*                 Determine both ROWMAX and JMAX.
*
                  IF( IMAX.NE.K ) THEN
                     JMAX = IMAX + IDAMAX( K-IMAX, W( IMAX+1, KW-1 ),
     $                                     1 )
                     ROWMAX = ABS( W( JMAX, KW-1 ) )
                  ELSE
                     ROWMAX = ZERO
                  END IF
*
                  IF( IMAX.GT.1 ) THEN
                     ITEMP = IDAMAX( IMAX-1, W( 1, KW-1 ), 1 )
                     DTEMP = ABS( W( ITEMP, KW-1 ) )
                     IF( DTEMP.GT.ROWMAX ) THEN
                        ROWMAX = DTEMP
```

```
                      JMAX = ITEMP
                   END IF
                END IF
*
*               Equivalent to testing for
*               ABS( W( IMAX, KW-1 ) ).GE.ALPHA*ROWMAX
*               (used to handle NaN and Inf)
*
                IF( .NOT.(ABS( W( IMAX, KW-1 ) ).LT.ALPHA*ROWMAX ) )
     $          THEN
*
*                  interchange rows and columns K and IMAX,
*                  use 1-by-1 pivot block
*
                   KP = IMAX
*
*                  copy column KW-1 of W to column KW of W
*
                   CALL DCOPY( K, W( 1, KW-1 ), 1, W( 1, KW ), 1 )
*
                   DONE = .TRUE.
*
*               Equivalent to testing for ROWMAX.EQ.COLMAX,
*               (used to handle NaN and Inf)
*
                ELSE IF( ( P.EQ.JMAX ) .OR. ( ROWMAX.LE.COLMAX ) )
     $          THEN
*
*                  interchange rows and columns K-1 and IMAX,
*                  use 2-by-2 pivot block
*
                   KP = IMAX
                   KSTEP = 2
                   DONE = .TRUE.
                ELSE
*
*                  Pivot not found: set params and repeat
*
                   P = IMAX
                   COLMAX = ROWMAX
                   IMAX = JMAX
*
*                  Copy updated JMAXth (next IMAXth) column to Kth of W
*
                   CALL DCOPY( K, W( 1, KW-1 ), 1, W( 1, KW ), 1 )
*
                END IF
*
*               End pivot search loop body
*
```

```
               IF( .NOT. DONE ) GOTO 12
*
            END IF
*
*           ============================================================
*
            KK = K - KSTEP + 1
*
*           KKW is the column of W which corresponds to column KK of A
*
            KKW = NB + KK - N
*
            IF( ( KSTEP.EQ.2 ) .AND. ( P.NE.K ) ) THEN
*
*              Copy non-updated column K to column P
*
               CALL DCOPY( K-P, A( P+1, K ), 1, A( P, P+1 ), LDA )
               CALL DCOPY( P, A( 1, K ), 1, A( 1, P ), 1 )
*
*              Interchange rows K and P in last N-K+1 columns of A
*              and last N-K+2 columns of W
*
               CALL DSWAP( N-K+1, A( K, K ), LDA, A( P, K ), LDA )
               CALL DSWAP( N-KK+1, W( K, KKW ), LDW, W( P, KKW ), LDW )
            END IF
*
*           Updated column KP is already stored in column KKW of W
*
            IF( KP.NE.KK ) THEN
*
*              Copy non-updated column KK to column KP
*
               A( KP, K ) = A( KK, K )
               CALL DCOPY( K-1-KP, A( KP+1, KK ), 1, A( KP, KP+1 ),
     $                     LDA )
               CALL DCOPY( KP, A( 1, KK ), 1, A( 1, KP ), 1 )
*
*              Interchange rows KK and KP in last N-KK+1 columns
*              of A and W
*
               CALL DSWAP( N-KK+1, A( KK, KK ), LDA, A( KP, KK ), LDA )
               CALL DSWAP( N-KK+1, W( KK, KKW ), LDW, W( KP, KKW ),
     $                     LDW )
            END IF
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column KW of W now holds
*
*              W(k) = U(k)*D(k)
```

```
*
*             where U(k) is the k-th column of U
*
*             Store U(k) in column k of A
*
              CALL DCOPY( K, W( 1, KW ), 1, A( 1, K ), 1 )
              IF( K.GT.1 ) THEN
                 IF( ABS( A( K, K ) ).GE.SFMIN ) THEN
                    R1 = ONE / A( K, K )
                    CALL DSCAL( K-1, R1, A( 1, K ), 1 )
                 ELSE IF( A( K, K ).NE.ZERO ) THEN
                    DO 14 II = 1, K - 1
                       A( II, K ) = A( II, K ) / A( K, K )
   14               CONTINUE
                 END IF
              END IF
*
           ELSE
*
*             2-by-2 pivot block D(k): columns KW and KW-1 of W now
*             hold
*
*             ( W(k-1) W(k) ) = ( U(k-1) U(k) )*D(k)
*
*             where U(k) and U(k-1) are the k-th and (k-1)-th columns
*             of U
*
              IF( K.GT.2 ) THEN
*
*                Store U(k) and U(k-1) in columns k and k-1 of A
*
                 D12 = W( K-1, KW )
                 D11 = W( K, KW ) / D12
                 D22 = W( K-1, KW-1 ) / D12
                 T = ONE / ( D11*D22-ONE )
                 DO 20 J = 1, K - 2
                    A( J, K-1 ) = T*( (D11*W( J, KW-1 )-W( J, KW ) ) /
     $                            D12 )
                    A( J, K ) = T*( ( D22*W( J, KW )-W( J, KW-1 ) ) /
     $                          D12 )
   20            CONTINUE
              END IF
*
*             Copy D(k) to A
*
              A( K-1, K-1 ) = W( K-1, KW-1 )
              A( K-1, K ) = W( K-1, KW )
              A( K, K ) = W( K, KW )
           END IF
        END IF
```

```
*
*        Store details of the interchanges in IPIV
*
         IF( KSTEP.EQ.1 ) THEN
            IPIV( K ) = KP
         ELSE
            IPIV( K ) = -P
            IPIV( K-1 ) = -KP
         END IF
*
*        Decrease K and return to the start of the main loop
*
         K = K - KSTEP
         GO TO 10
*
   30    CONTINUE
*
*        Update the upper triangle of A11 (= A(1:k,1:k)) as
*
*        A11 := A11 - U12*D*U12**T = A11 - U12*W**T
*
*        computing blocks of NB columns at a time
*
         DO 50 J = ( ( K-1 ) / NB )*NB + 1, 1, -NB
            JB = MIN( NB, K-J+1 )
*
*           Update the upper triangle of the diagonal block
*
            DO 40 JJ = J, J + JB - 1
               CALL DGEMV( 'No transpose', JJ-J+1, N-K, -ONE,
     $                     A( J, K+1 ), LDA, W( JJ, KW+1 ), LDW, ONE,
     $                     A( J, JJ ), 1 )
   40       CONTINUE
*
*           Update the rectangular superdiagonal block
*
            IF( J.GE.2 )
     $         CALL DGEMM( 'No transpose', 'Transpose', J-1, JB,
     $                  N-K, -ONE, A( 1, K+1 ), LDA, W( J, KW+1 ), LDW,
     $                  ONE, A( 1, J ), LDA )
   50    CONTINUE
*
*        Put U12 in standard form by partially undoing the interchanges
*        in columns k+1:n
*
         J = K + 1
   60    CONTINUE
*
            KSTEP = 1
            JP1 = 1
```

```
            JJ = J
            JP2 = IPIV( J )
            IF( JP2.LT.0 ) THEN
               JP2 = -JP2
               J = J + 1
               JP1 = -IPIV( J )
               KSTEP = 2
            END IF
*
            J = J + 1
            IF( JP2.NE.JJ .AND. J.LE.N )
     $         CALL DSWAP( N-J+1, A( JP2, J ), LDA, A( JJ, J ), LDA )
            JJ = J - 1
            IF( JP1.NE.JJ .AND. KSTEP.EQ.2 )
     $         CALL DSWAP( N-J+1, A( JP1, J ), LDA, A( JJ, J ), LDA )
         IF( J.LE.N )
     $      GO TO 60
*
*        Set KB to the number of columns factorized
*
         KB = N - K
*
      ELSE
*
*        Factorize the leading columns of A using the lower triangle
*        of A and working forwards, and compute the matrix W = L21*D
*        for use in updating A22
*
*        K is the main loop index, increasing from 1 in steps of 1 or 2
*
         K = 1
   70    CONTINUE
*
*        Exit from loop
*
         IF( ( K.GE.NB .AND. NB.LT.N ) .OR. K.GT.N )
     $      GO TO 90
*
         KSTEP = 1
         P = K
*
*        Copy column K of A to column K of W and update it
*
         CALL DCOPY( N-K+1, A( K, K ), 1, W( K, K ), 1 )
         IF( K.GT.1 )
     $      CALL DGEMV( 'No transpose', N-K+1, K-1, -ONE, A( K, 1 ),
     $                  LDA, W( K, 1 ), LDW, ONE, W( K, K ), 1 )
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
```

```
*
         ABSAKK = ABS( W( K, K ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.LT.N ) THEN
            IMAX = K + IDAMAX( N-K, W( K+1, K ), 1 )
            COLMAX = ABS( W( IMAX, K ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( MAX( ABSAKK, COLMAX ).EQ.ZERO ) THEN
*
*           Column K is zero or underflow: set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
            CALL DCOPY( N-K+1, W( K, K ), 1, A( K, K ), 1 )
         ELSE
*
*           ============================================================
*
*           Test for interchange
*
*           Equivalent to testing for ABSAKK.GE.ALPHA*COLMAX
*           (used to handle NaN and Inf)
*
            IF( .NOT.( ABSAKK.LT.ALPHA*COLMAX ) ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
*
            ELSE
*
               DONE = .FALSE.
*
*              Loop until pivot found
*
   72          CONTINUE
*
*                 Begin pivot search loop body
*
*
*                 Copy column IMAX to column K+1 of W and update it
*
```

```
                 CALL DCOPY( IMAX-K, A( IMAX, K ), LDA, W( K, K+1 ), 1)
                 CALL DCOPY( N-IMAX+1, A( IMAX, IMAX ), 1,
     $                      W( IMAX, K+1 ), 1 )
                 IF( K.GT.1 )
     $              CALL DGEMV( 'No transpose', N-K+1, K-1, -ONE,
     $                          A( K, 1 ), LDA, W( IMAX, 1 ), LDW,
     $                          ONE, W( K, K+1 ), 1 )
*
*                JMAX is the column-index of the largest off-diagonal
*                element in row IMAX, and ROWMAX is its absolute value.
*                Determine both ROWMAX and JMAX.
*
                 IF( IMAX.NE.K ) THEN
                    JMAX = K - 1 + IDAMAX( IMAX-K, W( K, K+1 ), 1 )
                    ROWMAX = ABS( W( JMAX, K+1 ) )
                 ELSE
                    ROWMAX = ZERO
                 END IF
*
                 IF( IMAX.LT.N ) THEN
                    ITEMP = IMAX + IDAMAX( N-IMAX, W( IMAX+1, K+1 ), 1)
                    DTEMP = ABS( W( ITEMP, K+1 ) )
                    IF( DTEMP.GT.ROWMAX ) THEN
                       ROWMAX = DTEMP
                       JMAX = ITEMP
                    END IF
                 END IF
*
*                Equivalent to testing for
*                ABS( W( IMAX, K+1 ) ).GE.ALPHA*ROWMAX
*                (used to handle NaN and Inf)
*
                 IF( .NOT.( ABS( W( IMAX, K+1 ) ).LT.ALPHA*ROWMAX ) )
     $           THEN
*
*                   interchange rows and columns K and IMAX,
*                   use 1-by-1 pivot block
*
                    KP = IMAX
*
*                   copy column K+1 of W to column K of W
*
                    CALL DCOPY( N-K+1, W( K, K+1 ), 1, W( K, K ), 1 )
*
                    DONE = .TRUE.
*
*                Equivalent to testing for ROWMAX.EQ.COLMAX,
*                (used to handle NaN and Inf)
*
                 ELSE IF( ( P.EQ.JMAX ) .OR. ( ROWMAX.LE.COLMAX ) )
```

```
     $            THEN
*
*                    interchange rows and columns K+1 and IMAX,
*                    use 2-by-2 pivot block
*
                     KP = IMAX
                     KSTEP = 2
                     DONE = .TRUE.
                  ELSE
*
*                    Pivot not found: set params and repeat
*
                     P = IMAX
                     COLMAX = ROWMAX
                     IMAX = JMAX
*
*                    Copy updated JMAXth (next IMAXth) column to Kth of W
*
                     CALL DCOPY( N-K+1, W( K, K+1 ), 1, W( K, K ), 1 )
*
                  END IF
*
*                 End pivot search loop body
*
               IF( .NOT. DONE ) GOTO 72
*
            END IF
*
*           ============================================================
*
            KK = K + KSTEP - 1
*
            IF( ( KSTEP.EQ.2 ) .AND. ( P.NE.K ) ) THEN
*
*              Copy non-updated column K to column P
*
               CALL DCOPY( P-K, A( K, K ), 1, A( P, K ), LDA )
               CALL DCOPY( N-P+1, A( P, K ), 1, A( P, P ), 1 )
*
*              Interchange rows K and P in first K columns of A
*              and first K+1 columns of W
*
               CALL DSWAP( K, A( K, 1 ), LDA, A( P, 1 ), LDA )
               CALL DSWAP( KK, W( K, 1 ), LDW, W( P, 1 ), LDW )
            END IF
*
*           Updated column KP is already stored in column KK of W
*
            IF( KP.NE.KK ) THEN
*
```

```
*              Copy non-updated column KK to column KP
*
               A( KP, K ) = A( KK, K )
               CALL DCOPY( KP-K-1, A( K+1, KK ), 1, A( KP, K+1 ), LDA )
               CALL DCOPY( N-KP+1, A( KP, KK ), 1, A( KP, KP ), 1 )
*
*              Interchange rows KK and KP in first KK columns of A and W
*
               CALL DSWAP( KK, A( KK, 1 ), LDA, A( KP, 1 ), LDA )
               CALL DSWAP( KK, W( KK, 1 ), LDW, W( KP, 1 ), LDW )
            END IF
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column k of W now holds
*
*              W(k) = L(k)*D(k)
*
*              where L(k) is the k-th column of L
*
*              Store L(k) in column k of A
*
               CALL DCOPY( N-K+1, W( K, K ), 1, A( K, K ), 1 )
               IF( K.LT.N ) THEN
                  IF( ABS( A( K, K ) ).GE.SFMIN ) THEN
                     R1 = ONE / A( K, K )
                     CALL DSCAL( N-K, R1, A( K+1, K ), 1 )
                  ELSE IF( A( K, K ).NE.ZERO ) THEN
                     DO 74 II = K + 1, N
                        A( II, K ) = A( II, K ) / A( K, K )
   74                CONTINUE
                  END IF
               END IF
*
            ELSE
*
*              2-by-2 pivot block D(k): columns k and k+1 of W now hold
*
*              ( W(k) W(k+1) ) = ( L(k) L(k+1) )*D(k)
*
*              where L(k) and L(k+1) are the k-th and (k+1)-th columns
*              of L
*
               IF( K.LT.N-1 ) THEN
*
*                 Store L(k) and L(k+1) in columns k and k+1 of A
*
                  D21 = W( K+1, K )
                  D11 = W( K+1, K+1 ) / D21
                  D22 = W( K, K ) / D21
```

```
                 T = ONE / ( D11*D22-ONE )
                 DO 80 J = K + 2, N
                    A( J, K ) = T*( ( D11*W( J, K )-W( J, K+1 ) ) /
     $                           D21 )
                    A( J, K+1 ) = T*( ( D22*W( J, K+1 )-W( J, K ) ) /
     $                             D21 )
   80            CONTINUE
              END IF
*
*             Copy D(k) to A
*
              A( K, K ) = W( K, K )
              A( K+1, K ) = W( K+1, K )
              A( K+1, K+1 ) = W( K+1, K+1 )
           END IF
        END IF
*
*       Store details of the interchanges in IPIV
*
        IF( KSTEP.EQ.1 ) THEN
           IPIV( K ) = KP
        ELSE
           IPIV( K ) = -P
           IPIV( K+1 ) = -KP
        END IF
*
*       Increase K and return to the start of the main loop
*
        K = K + KSTEP
        GO TO 70
*
   90   CONTINUE
*
*       Update the lower triangle of A22 (= A(k:n,k:n)) as
*
*       A22 := A22 - L21*D*L21**T = A22 - L21*W**T
*
*       computing blocks of NB columns at a time
*
        DO 110 J = K, N, NB
           JB = MIN( NB, N-J+1 )
*
*          Update the lower triangle of the diagonal block
*
           DO 100 JJ = J, J + JB - 1
              CALL DGEMV( 'No transpose', J+JB-JJ, K-1, -ONE,
     $                    A( JJ, 1 ), LDA, W( JJ, 1 ), LDW, ONE,
     $                    A( JJ, JJ ), 1 )
  100       CONTINUE
*
```

```
*           Update the rectangular subdiagonal block
*
            IF( J+JB.LE.N )
     $         CALL DGEMM( 'No transpose', 'Transpose', N-J-JB+1, JB,
     $                     K-1, -ONE, A( J+JB, 1 ), LDA, W( J, 1 ), LDW,
     $                     ONE, A( J+JB, J ), LDA )
  110    CONTINUE
*
*        Put L21 in standard form by partially undoing the interchanges
*        in columns 1:k-1
*
         J = K - 1
  120    CONTINUE
*
            KSTEP = 1
            JP1 = 1
            JJ = J
            JP2 = IPIV( J )
            IF( JP2.LT.0 ) THEN
               JP2 = -JP2
               J = J - 1
               JP1 = -IPIV( J )
               KSTEP = 2
            END IF
*
            J = J - 1
            IF( JP2.NE.JJ .AND. J.GE.1 )
     $         CALL DSWAP( J, A( JP2, 1 ), LDA, A( JJ, 1 ), LDA )
            JJ = J + 1
            IF( JP1.NE.JJ .AND. KSTEP.EQ.2 )
     $         CALL DSWAP( J, A( JP1, 1 ), LDA, A( JJ, 1 ), LDA )
         IF( J.GE.1 )
     $      GO TO 120
*
*        Set KB to the number of columns factorized
*
         KB = K - 1
*
      END IF
      RETURN
*
*     End of DLASYF_ROOK
*
      END
```

———-

58.6.4 lisp code

— dlasyf_rook.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0) (eight 8.0d0) (sevten 17.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 8.0d0 8.0d0) eight)
           (type (double-float 17.0d0 17.0d0) sevten)
           (ignorable zero one eight sevten))
  (defun dlasyf_rook (uplo n nb kb a lda ipiv w ldw info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array double-float (*)) w a)
             (type (f2cl-lib:integer4) info ldw lda kb nb n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (w double-float w-%data% w-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((absakk 0.0d0) (alpha 0.0d0) (colmax 0.0d0) (d11 0.0d0)
             (d12 0.0d0) (d21 0.0d0) (d22 0.0d0) (dtemp 0.0d0) (r1 0.0d0)
             (rowmax 0.0d0) (t$ 0.0d0) (sfmin 0.0d0) (imax 0) (itemp 0) (j 0)
             (jb 0) (jj 0) (jmax 0) (jp1 0) (jp2 0) (k 0) (kk 0) (kw 0) (kkw 0)
             (kp 0) (kstep 0) (p 0) (ii 0) (done nil))
        (declare
         (type (double-float) absakk alpha colmax d11 d12 d21 d22 dtemp r1
          rowmax t$ sfmin)
         (type (f2cl-lib:integer4) imax itemp j jb jj jmax jp1 jp2 k kk kw kkw
          kp kstep p ii)
         (type f2cl-lib:logical done))
        (setf info 0)
        (setf alpha (/ (+ one (f2cl-lib:fsqrt sevten)) eight))
        (setf sfmin (dlamch "S"))
        (cond
         ((multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)
          (tagbody
            (setf k n)
           label10
            (setf kw (f2cl-lib:int-sub (f2cl-lib:int-add nb k) n))
            (if (or
                 (and (<= k (f2cl-lib:int-add (f2cl-lib:int-sub n nb) 1))
                      (< nb n))
                 (< k 1))
                (go label30))
```

```
(setf kstep 1)
(setf p k)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dcopy k
     (f2cl-lib:array-slice a-%data% double-float (1 k)
                           ((1 lda) (1 *)) a-%offset%)
     1
     (f2cl-lib:array-slice w-%data% double-float (1 kw)
                           ((1 ldw) (1 *)) w-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf k var-0)))
(if (< k n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10)
        (dgemv "No transpose" k (f2cl-lib:int-sub n k) (- one)
         (f2cl-lib:array-slice a-%data% double-float
                               (1 (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice w-%data% double-float
                               (k (f2cl-lib:int-add kw 1))
                               ((1 ldw) (1 *)) w-%offset%)
         ldw one
         (f2cl-lib:array-slice w-%data% double-float (1 kw)
                               ((1 ldw) (1 *)) w-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3 var-4 var-6 var-9 var-10))
      (when var-1 (setf k var-1))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldw var-7))
      (when var-8 (setf one var-8))))
(setf absakk
        (abs
         (f2cl-lib:fref w-%data% (k kw) ((1 ldw) (1 *))
                        w-%offset%)))
(cond
 ((> k 1)
  (setf imax
          (idamax (f2cl-lib:int-sub k 1)
           (f2cl-lib:array-slice w-%data% double-float (1 kw)
                                 ((1 ldw) (1 *)) w-%offset%)
           1))
  (setf colmax
          (abs
           (f2cl-lib:fref w-%data% (imax kw) ((1 ldw) (1 *))
                          w-%offset%))))
 (t (setf colmax zero)))
(cond
```

```
((= (max absakk colmax) zero)
 (if (= info 0)
     (setf info k))
 (setf kp k)
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (dcopy k
      (f2cl-lib:array-slice w-%data% double-float (1 kw)
                            ((1 ldw) (1 *)) w-%offset%)
      1
      (f2cl-lib:array-slice a-%data% double-float (1 k)
                            ((1 lda) (1 *)) a-%offset%)
      1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf k var-0))))
(t
 (cond ((not (< absakk (* alpha colmax))) (setf kp k))
       (t
        (tagbody
          (setf done f2cl-lib:%false%)
         label12
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dcopy imax
               (f2cl-lib:array-slice a-%data% double-float
                                     (1 imax) ((1 lda) (1 *))
                                     a-%offset%)
               1
               (f2cl-lib:array-slice w-%data% double-float
                                     (1 (f2cl-lib:int-sub kw 1))
                                     ((1 ldw) (1 *)) w-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf imax var-0)))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dcopy (f2cl-lib:int-sub k imax)
               (f2cl-lib:array-slice a-%data% double-float
                                     (imax
                                      (f2cl-lib:int-add imax 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice w-%data% double-float
                                     ((+ imax 1)
                                      (f2cl-lib:int-sub kw 1))
                                     ((1 ldw) (1 *)) w-%offset%)
               1)
            (declare (ignore var-0 var-1 var-3 var-4))
            (when var-2 (setf lda var-2)))
          (if (< k n)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
                   var-8 var-9 var-10)
```

```
            (dgemv "No transpose" k (f2cl-lib:int-sub n k)
             (- one)
             (f2cl-lib:array-slice a-%data% double-float
                                   (1
                                    (f2cl-lib:int-add k 1))
                                   ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice w-%data% double-float
                                   (imax
                                    (f2cl-lib:int-add kw 1))
                                   ((1 ldw) (1 *))
                                   w-%offset%)
             ldw one
             (f2cl-lib:array-slice w-%data% double-float
                                   (1
                                    (f2cl-lib:int-sub kw 1))
                                   ((1 ldw) (1 *))
                                   w-%offset%)
             1)
          (declare
           (ignore var-0 var-2 var-3 var-4 var-6 var-9
            var-10))
          (when var-1 (setf k var-1))
          (when var-5 (setf lda var-5))
          (when var-7 (setf ldw var-7))
          (when var-8 (setf one var-8))))
    (cond
     ((/= imax k)
      (setf jmax
              (f2cl-lib:int-add imax
                                (idamax
                                 (f2cl-lib:int-sub k imax)
                                 (f2cl-lib:array-slice
                                  w-%data% double-float
                                  ((+ imax 1)
                                   (f2cl-lib:int-sub kw 1))
                                  ((1 ldw) (1 *))
                                  w-%offset%)
                                 1)))
      (setf rowmax
              (abs
               (f2cl-lib:fref w-%data%
                              (jmax (f2cl-lib:int-sub kw 1))
                              ((1 ldw) (1 *)) w-%offset%))))
     (t (setf rowmax zero)))
    (cond
     ((> imax 1)
      (setf itemp
              (idamax (f2cl-lib:int-sub imax 1)
```

```
            (f2cl-lib:array-slice w-%data% double-float
                                  (1
                                   (f2cl-lib:int-sub kw
                                                     1))
                                  ((1 ldw) (1 *))
                                  w-%offset%)
            1))
  (setf dtemp
          (abs
           (f2cl-lib:fref w-%data%
                          (itemp
                           (f2cl-lib:int-sub kw 1))
                          ((1 ldw) (1 *)) w-%offset%)))
  (cond
   ((> dtemp rowmax) (setf rowmax dtemp)
    (setf jmax itemp)))))
(cond
 ((not
   (<
    (abs
     (f2cl-lib:fref w
                    (imax
                     (f2cl-lib:int-add kw
                                       (f2cl-lib:int-sub
                                        1)))
                    ((1 ldw) (1 *))))
    (* alpha rowmax)))
  (setf kp imax)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy k
       (f2cl-lib:array-slice w-%data% double-float
                             (1 (f2cl-lib:int-sub kw 1))
                             ((1 ldw) (1 *)) w-%offset%)
       1
       (f2cl-lib:array-slice w-%data% double-float
                             (1 kw) ((1 ldw) (1 *))
                             w-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf k var-0)))
  (setf done f2cl-lib:%true%))
 ((or (= p jmax) (<= rowmax colmax)) (setf kp imax)
  (setf kstep 2) (setf done f2cl-lib:%true%))
 (t (setf p imax) (setf colmax rowmax) (setf imax jmax)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy k
       (f2cl-lib:array-slice w-%data% double-float
                             (1 (f2cl-lib:int-sub kw 1))
                             ((1 ldw) (1 *)) w-%offset%)
       1
```

```
                    (f2cl-lib:array-slice w-%data% double-float
                                          (1 kw) ((1 ldw) (1 *))
                                          w-%offset%)
                    1)
                (declare (ignore var-1 var-2 var-3 var-4))
                (when var-0 (setf k var-0)))))
            (if (not done)
                (go label12)))))
(setf kk (f2cl-lib:int-add (f2cl-lib:int-sub k kstep) 1))
(setf kkw (f2cl-lib:int-sub (f2cl-lib:int-add nb kk) n))
(cond
 ((and (= kstep 2) (/= p k))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy (f2cl-lib:int-sub k p)
       (f2cl-lib:array-slice a-%data% double-float ((+ p 1) k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% double-float
                             (p (f2cl-lib:int-add p 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy p
       (f2cl-lib:array-slice a-%data% double-float (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% double-float (1 p)
                             ((1 lda) (1 *)) a-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf p var-0)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dswap (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
       (f2cl-lib:array-slice a-%data% double-float (k k)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice a-%data% double-float (p k)
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-3))
    (when var-2 (setf lda var-2))
    (when var-4 (setf lda var-4)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dswap (f2cl-lib:int-add (f2cl-lib:int-sub n kk) 1)
       (f2cl-lib:array-slice w-%data% double-float (k kkw)
                             ((1 ldw) (1 *)) w-%offset%)
       ldw
       (f2cl-lib:array-slice w-%data% double-float (p kkw)
```

```
                                ((1 ldw) (1 *)) w-%offset%)
        ldw)
      (declare (ignore var-0 var-1 var-3))
      (when var-2 (setf ldw var-2))
      (when var-4 (setf ldw var-4)))))
  (cond
   ((/= kp kk)
    (setf (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                         a-%offset%)
          (f2cl-lib:fref a-%data% (kk k) ((1 lda) (1 *))
                           a-%offset%))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy (f2cl-lib:int-sub k 1 kp)
         (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) kk)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% double-float
                               (kp (f2cl-lib:int-add kp 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-0 var-1 var-2 var-3))
      (when var-4 (setf lda var-4)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy kp
         (f2cl-lib:array-slice a-%data% double-float (1 kk)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% double-float (1 kp)
                               ((1 lda) (1 *)) a-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf kp var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dswap (f2cl-lib:int-add (f2cl-lib:int-sub n kk) 1)
         (f2cl-lib:array-slice a-%data% double-float (kk kk)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice a-%data% double-float (kp kk)
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-0 var-1 var-3))
      (when var-2 (setf lda var-2))
      (when var-4 (setf lda var-4)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dswap (f2cl-lib:int-add (f2cl-lib:int-sub n kk) 1)
         (f2cl-lib:array-slice w-%data% double-float (kk kkw)
                               ((1 ldw) (1 *)) w-%offset%)
         ldw
         (f2cl-lib:array-slice w-%data% double-float (kp kkw)
                               ((1 ldw) (1 *)) w-%offset%)
```

```
        ldw)
      (declare (ignore var-0 var-1 var-3))
      (when var-2 (setf ldw var-2))
      (when var-4 (setf ldw var-4)))))
  (cond
   ((= kstep 1)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy k
         (f2cl-lib:array-slice w-%data% double-float (1 kw)
                               ((1 ldw) (1 *)) w-%offset%)
         1
         (f2cl-lib:array-slice a-%data% double-float (1 k)
                               ((1 lda) (1 *)) a-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf k var-0)))
    (cond
     ((> k 1)
      (cond
       ((>= (abs (f2cl-lib:fref a (k k) ((1 lda) (1 *)))) sfmin)
        (setf r1
                (/ one
                   (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                  a-%offset%)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal (f2cl-lib:int-sub k 1) r1
             (f2cl-lib:array-slice a-%data% double-float (1 k)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-0 var-2 var-3))
          (when var-1 (setf r1 var-1))))
       ((/= (f2cl-lib:fref a (k k) ((1 lda) (1 *))) zero)
        (f2cl-lib:fdo (ii 1 (f2cl-lib:int-add ii 1))
                      ((> ii
                          (f2cl-lib:int-add k
                                            (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
                                 a-%offset%)
                  (/
                   (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
                                  a-%offset%)
                   (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                  a-%offset%)))
          label14)))))))
   (t
    (cond
     ((> k 2)
      (setf d12
```

```
        (f2cl-lib:fref w-%data% ((f2cl-lib:int-sub k 1) kw)
                       ((1 ldw) (1 *)) w-%offset%))
  (setf d11
        (/
         (f2cl-lib:fref w-%data% (k kw) ((1 ldw) (1 *))
                        w-%offset%)
         d12))
  (setf d22
        (/
         (f2cl-lib:fref w-%data%
                        ((f2cl-lib:int-sub k 1)
                         (f2cl-lib:int-sub kw 1))
                        ((1 ldw) (1 *)) w-%offset%)
         d12))
  (setf t$ (/ one (- (* d11 d22) one)))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j
                    (f2cl-lib:int-add k (f2cl-lib:int-sub 2)))
                 nil)
    (tagbody
      (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-sub k 1))
                           ((1 lda) (1 *)) a-%offset%)
            (* t$
               (/
                (-
                 (* d11
                    (f2cl-lib:fref w-%data%
                                   (j
                                    (f2cl-lib:int-sub kw 1))
                                   ((1 ldw) (1 *))
                                   w-%offset%))
                 (f2cl-lib:fref w-%data% (j kw)
                                ((1 ldw) (1 *)) w-%offset%))
                d12)))
      (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                           a-%offset%)
            (* t$
               (/
                (-
                 (* d22
                    (f2cl-lib:fref w-%data% (j kw)
                                   ((1 ldw) (1 *))
                                   w-%offset%))
                 (f2cl-lib:fref w-%data%
                                (j (f2cl-lib:int-sub kw 1))
                                ((1 ldw) (1 *)) w-%offset%))
                d12)))
     label20))))
(setf (f2cl-lib:fref a-%data%
                     ((f2cl-lib:int-sub k 1)
```

```
                              (f2cl-lib:int-sub k 1))
                             ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref w-%data%
                               ((f2cl-lib:int-sub k 1)
                                (f2cl-lib:int-sub kw 1))
                               ((1 ldw) (1 *)) w-%offset%))
        (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                             ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref w-%data% ((f2cl-lib:int-sub k 1) kw)
                               ((1 ldw) (1 *)) w-%offset%))
        (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref w-%data% (k kw) ((1 ldw) (1 *))
                               w-%offset%))))))
    (cond
     ((= kstep 1)
      (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%) kp))
     (t
      (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
              (f2cl-lib:int-sub p))
      (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                           ipiv-%offset%)
              (f2cl-lib:int-sub kp))))
    (setf k (f2cl-lib:int-sub k kstep))
    (go label10)
   label30
    (f2cl-lib:fdo (j
                   (+
                    (*
                     (the f2cl-lib:integer4 (truncate (+ k (- 1)) nb))
                     nb)
                    1)
                   (f2cl-lib:int-add j (f2cl-lib:int-sub nb)))
                  ((> j 1) nil)
      (tagbody
        (setf jb
                (min (the f2cl-lib:integer4 nb)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-sub k j)
                                            1))))
        (f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
                      ((> jj
                          (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10)
                (dgemv "No transpose"
                 (f2cl-lib:int-add (f2cl-lib:int-sub jj j) 1)
                 (f2cl-lib:int-sub n k) (- one)
```

```
                (f2cl-lib:array-slice a-%data% double-float
                                      (j (f2cl-lib:int-add k 1))
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice w-%data% double-float
                                      (jj (f2cl-lib:int-add kw 1))
                                      ((1 ldw) (1 *)) w-%offset%)
                ldw one
                (f2cl-lib:array-slice a-%data% double-float (j jj)
                                      ((1 lda) (1 *)) a-%offset%)
                1)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9
                var-10))
              (when var-5 (setf lda var-5))
              (when var-7 (setf ldw var-7))
              (when var-8 (setf one var-8)))
           label40))
        (if (>= j 2)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10 var-11 var-12)
                (dgemm "No transpose" "Transpose"
                 (f2cl-lib:int-sub j 1) jb (f2cl-lib:int-sub n k)
                 (- one)
                 (f2cl-lib:array-slice a-%data% double-float
                                       (1 (f2cl-lib:int-add k 1))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice w-%data% double-float
                                       (j (f2cl-lib:int-add kw 1))
                                       ((1 ldw) (1 *)) w-%offset%)
                 ldw one
                 (f2cl-lib:array-slice a-%data% double-float (1 j)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare
               (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-8
                var-11))
              (when var-3 (setf jb var-3))
              (when var-7 (setf lda var-7))
              (when var-9 (setf ldw var-9))
              (when var-10 (setf one var-10))
              (when var-12 (setf lda var-12))))
       label50))
    (setf j (f2cl-lib:int-add k 1))
   label60
    (setf kstep 1)
    (setf jp1 1)
    (setf jj j)
```

```
    (setf jp2 (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
    (cond
     ((< jp2 0) (setf jp2 (f2cl-lib:int-sub jp2))
      (setf j (f2cl-lib:int-add j 1))
      (setf jp1
              (f2cl-lib:int-sub
               (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)))
      (setf kstep 2)))
    (setf j (f2cl-lib:int-add j 1))
    (if (and (/= jp2 jj) (<= j n))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dswap (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1)
             (f2cl-lib:array-slice a-%data% double-float (jp2 j)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice a-%data% double-float (jj j)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-1 var-3))
          (when var-2 (setf lda var-2))
          (when var-4 (setf lda var-4))))
    (setf jj (f2cl-lib:int-sub j 1))
    (if (and (/= jp1 jj) (= kstep 2))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dswap (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1)
             (f2cl-lib:array-slice a-%data% double-float (jp1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice a-%data% double-float (jj j)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-1 var-3))
          (when var-2 (setf lda var-2))
          (when var-4 (setf lda var-4))))
    (if (<= j n)
        (go label60))
    (setf kb (f2cl-lib:int-sub n k))))
  (t
   (tagbody
    (setf k 1)
   label70
    (if (or (and (>= k nb) (< nb n)) (> k n))
        (go label90))
    (setf kstep 1)
    (setf p k)
    (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
     (f2cl-lib:array-slice a-%data% double-float (k k) ((1 lda) (1 *))
                           a-%offset%)
     1
     (f2cl-lib:array-slice w-%data% double-float (k k) ((1 ldw) (1 *))
```

```
                        w-%offset%)
 1)
(if (> k 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10)
        (dgemv "No transpose"
         (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
         (f2cl-lib:int-sub k 1) (- one)
         (f2cl-lib:array-slice a-%data% double-float (k 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice w-%data% double-float (k 1)
                               ((1 ldw) (1 *)) w-%offset%)
         ldw one
         (f2cl-lib:array-slice w-%data% double-float (k k)
                               ((1 ldw) (1 *)) w-%offset%)
         1)
      (declare
       (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9 var-10))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldw var-7))
      (when var-8 (setf one var-8))))
(setf absakk
        (abs
         (f2cl-lib:fref w-%data% (k k) ((1 ldw) (1 *))
                        w-%offset%)))
(cond
 ((< k n)
  (setf imax
          (f2cl-lib:int-add k
                            (idamax (f2cl-lib:int-sub n k)
                             (f2cl-lib:array-slice w-%data%
                                                   double-float
                                                   ((+ k 1) k)
                                                   ((1 ldw) (1 *))
                                                   w-%offset%)
                             1)))
  (setf colmax
          (abs
           (f2cl-lib:fref w-%data% (imax k) ((1 ldw) (1 *))
                          w-%offset%))))
 (t (setf colmax zero)))
(cond
 ((= (max absakk colmax) zero)
  (if (= info 0)
      (setf info k))
  (setf kp k)
  (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
   (f2cl-lib:array-slice w-%data% double-float (k k)
```

```
                          ((1 ldw) (1 *)) w-%offset%)
  1
  (f2cl-lib:array-slice a-%data% double-float (k k)
                          ((1 lda) (1 *)) a-%offset%)
  1))
(t
 (cond ((not (< absakk (* alpha colmax))) (setf kp k))
       (t
        (tagbody
          (setf done f2cl-lib:%false%)
         label72
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dcopy (f2cl-lib:int-sub imax k)
               (f2cl-lib:array-slice a-%data% double-float
                                     (imax k) ((1 lda) (1 *))
                                     a-%offset%)
               lda
               (f2cl-lib:array-slice w-%data% double-float
                                     (k (f2cl-lib:int-add k 1))
                                     ((1 ldw) (1 *)) w-%offset%)
               1)
            (declare (ignore var-0 var-1 var-3 var-4))
            (when var-2 (setf lda var-2)))
          (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub n imax) 1)
           (f2cl-lib:array-slice a-%data% double-float (imax imax)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice w-%data% double-float
                                 (imax (f2cl-lib:int-add k 1))
                                 ((1 ldw) (1 *)) w-%offset%)
           1)
          (if (> k 1)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
                   var-8 var-9 var-10)
                  (dgemv "No transpose"
                   (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
                   (f2cl-lib:int-sub k 1) (- one)
                   (f2cl-lib:array-slice a-%data% double-float
                                         (k 1) ((1 lda) (1 *))
                                         a-%offset%)
                   lda
                   (f2cl-lib:array-slice w-%data% double-float
                                         (imax 1) ((1 ldw) (1 *))
                                         w-%offset%)
                   ldw one
                   (f2cl-lib:array-slice w-%data% double-float
                                         (k
                                          (f2cl-lib:int-add k 1))
                                         ((1 ldw) (1 *))
```

```
                                w-%offset%)
          1)
      (declare
       (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9
        var-10))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldw var-7))
      (when var-8 (setf one var-8))))
(cond
 ((/= imax k)
  (setf jmax
          (f2cl-lib:int-add (f2cl-lib:int-sub k 1)
                            (idamax
                             (f2cl-lib:int-sub imax k)
                             (f2cl-lib:array-slice
                              w-%data% double-float
                              (k (f2cl-lib:int-add k 1))
                              ((1 ldw) (1 *))
                              w-%offset%)
                             1)))
  (setf rowmax
          (abs
           (f2cl-lib:fref w-%data%
                          (jmax (f2cl-lib:int-add k 1))
                          ((1 ldw) (1 *)) w-%offset%))))
 (t (setf rowmax zero)))
(cond
 ((< imax n)
  (setf itemp
          (f2cl-lib:int-add imax
                            (idamax
                             (f2cl-lib:int-sub n imax)
                             (f2cl-lib:array-slice
                              w-%data% double-float
                              ((+ imax 1)
                               (f2cl-lib:int-add k 1))
                              ((1 ldw) (1 *))
                              w-%offset%)
                             1)))
  (setf dtemp
          (abs
           (f2cl-lib:fref w-%data%
                          (itemp (f2cl-lib:int-add k 1))
                          ((1 ldw) (1 *)) w-%offset%)))
  (cond
   ((> dtemp rowmax) (setf rowmax dtemp)
    (setf jmax itemp)))))
(cond
 ((not
   (<
```

```
                (abs
                 (f2cl-lib:fref w (imax (f2cl-lib:int-add k 1))
                                ((1 ldw) (1 *))))
                (* alpha rowmax)))
              (setf kp imax)
              (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
               (f2cl-lib:array-slice w-%data% double-float
                                     (k (f2cl-lib:int-add k 1))
                                     ((1 ldw) (1 *)) w-%offset%)
               1
               (f2cl-lib:array-slice w-%data% double-float (k k)
                                     ((1 ldw) (1 *)) w-%offset%)
               1)
              (setf done f2cl-lib:%true%))
             ((or (= p jmax) (<= rowmax colmax)) (setf kp imax)
              (setf kstep 2) (setf done f2cl-lib:%true%))
             (t (setf p imax) (setf colmax rowmax) (setf imax jmax)
              (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
               (f2cl-lib:array-slice w-%data% double-float
                                     (k (f2cl-lib:int-add k 1))
                                     ((1 ldw) (1 *)) w-%offset%)
               1
               (f2cl-lib:array-slice w-%data% double-float (k k)
                                     ((1 ldw) (1 *)) w-%offset%)
               1)))
            (if (not done)
                (go label72)))))
(setf kk (f2cl-lib:int-sub (f2cl-lib:int-add k kstep) 1))
(cond
 ((and (= kstep 2) (/= p k))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy (f2cl-lib:int-sub p k)
       (f2cl-lib:array-slice a-%data% double-float (k k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% double-float (p k)
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
   (f2cl-lib:array-slice a-%data% double-float (p k)
                         ((1 lda) (1 *)) a-%offset%)
   1
   (f2cl-lib:array-slice a-%data% double-float (p p)
                         ((1 lda) (1 *)) a-%offset%)
   1)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dswap k
       (f2cl-lib:array-slice a-%data% double-float (k 1)
```

```
                                ((1 lda) (1 *)) a-%offset%)
          lda
          (f2cl-lib:array-slice a-%data% double-float (p 1)
                                ((1 lda) (1 *)) a-%offset%)
          lda)
        (declare (ignore var-1 var-3))
        (when var-0 (setf k var-0))
        (when var-2 (setf lda var-2))
        (when var-4 (setf lda var-4)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dswap kk
          (f2cl-lib:array-slice w-%data% double-float (k 1)
                                ((1 ldw) (1 *)) w-%offset%)
          ldw
          (f2cl-lib:array-slice w-%data% double-float (p 1)
                                ((1 ldw) (1 *)) w-%offset%)
          ldw)
        (declare (ignore var-1 var-3))
        (when var-0 (setf kk var-0))
        (when var-2 (setf ldw var-2))
        (when var-4 (setf ldw var-4)))))
    (cond
     ((/= kp kk)
      (setf (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                           a-%offset%)
            (f2cl-lib:fref a-%data% (kk k) ((1 lda) (1 *))
                           a-%offset%))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy (f2cl-lib:int-sub kp k 1)
          (f2cl-lib:array-slice a-%data% double-float ((+ k 1) kk)
                                ((1 lda) (1 *)) a-%offset%)
          1
          (f2cl-lib:array-slice a-%data% double-float
                                (kp (f2cl-lib:int-add k 1))
                                ((1 lda) (1 *)) a-%offset%)
          lda)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf lda var-4)))
      (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub n kp) 1)
       (f2cl-lib:array-slice a-%data% double-float (kp kk)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% double-float (kp kp)
                             ((1 lda) (1 *)) a-%offset%)
       1)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dswap kk
          (f2cl-lib:array-slice a-%data% double-float (kk 1)
                                ((1 lda) (1 *)) a-%offset%)
          lda
```

```
          (f2cl-lib:array-slice a-%data% double-float (kp 1)
                                ((1 lda) (1 *)) a-%offset%)
          lda)
        (declare (ignore var-1 var-3))
        (when var-0 (setf kk var-0))
        (when var-2 (setf lda var-2))
        (when var-4 (setf lda var-4)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dswap kk
          (f2cl-lib:array-slice w-%data% double-float (kk 1)
                                ((1 ldw) (1 *)) w-%offset%)
          ldw
          (f2cl-lib:array-slice w-%data% double-float (kp 1)
                                ((1 ldw) (1 *)) w-%offset%)
          ldw)
        (declare (ignore var-1 var-3))
        (when var-0 (setf kk var-0))
        (when var-2 (setf ldw var-2))
        (when var-4 (setf ldw var-4)))))
    (cond
     ((= kstep 1)
      (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
       (f2cl-lib:array-slice w-%data% double-float (k k)
                             ((1 ldw) (1 *)) w-%offset%)
       1
       (f2cl-lib:array-slice a-%data% double-float (k k)
                             ((1 lda) (1 *)) a-%offset%)
       1)
      (cond
       ((< k n)
        (cond
         ((>= (abs (f2cl-lib:fref a (k k) ((1 lda) (1 *)))) sfmin)
          (setf r1
                  (/ one
                     (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                    a-%offset%)))
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (dscal (f2cl-lib:int-sub n k) r1
               (f2cl-lib:array-slice a-%data% double-float
                                     ((+ k 1) k) ((1 lda) (1 *))
                                     a-%offset%)
               1)
            (declare (ignore var-0 var-2 var-3))
            (when var-1 (setf r1 var-1))))
         ((/= (f2cl-lib:fref a (k k) ((1 lda) (1 *))) zero)
          (f2cl-lib:fdo (ii (f2cl-lib:int-add k 1)
                            (f2cl-lib:int-add ii 1))
                        ((> ii n) nil)
            (tagbody
              (setf (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
```

```
                                a-%offset%)
                     (/
                      (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
                                     a-%offset%)
                      (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                     a-%offset%)))
          label74)))))))
  (t
   (cond
    ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
     (setf d21
             (f2cl-lib:fref w-%data% ((f2cl-lib:int-add k 1) k)
                            ((1 ldw) (1 *)) w-%offset%))
     (setf d11
             (/
              (f2cl-lib:fref w-%data%
                             ((f2cl-lib:int-add k 1)
                              (f2cl-lib:int-add k 1))
                             ((1 ldw) (1 *)) w-%offset%)
              d21))
     (setf d22
             (/
              (f2cl-lib:fref w-%data% (k k) ((1 ldw) (1 *))
                             w-%offset%)
              d21))
     (setf t$ (/ one (- (* d11 d22) one)))
     (f2cl-lib:fdo (j (f2cl-lib:int-add k 2)
                    (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                              a-%offset%)
                 (* t$
                    (/
                     (-
                      (* d11
                         (f2cl-lib:fref w-%data% (j k)
                                        ((1 ldw) (1 *))
                                        w-%offset%))
                      (f2cl-lib:fref w-%data%
                                     (j (f2cl-lib:int-add k 1))
                                     ((1 ldw) (1 *)) w-%offset%))
                     d21)))
         (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-add k 1))
                              ((1 lda) (1 *)) a-%offset%)
                 (* t$
                    (/
                     (-
                      (* d22
                         (f2cl-lib:fref w-%data%
```

```
                                                (j (f2cl-lib:int-add k 1))
                                                ((1 ldw) (1 *))
                                                w-%offset%))
                                (f2cl-lib:fref w-%data% (j k)
                                               ((1 ldw) (1 *)) w-%offset%))
                              d21)))
                label80))))
            (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                    (f2cl-lib:fref w-%data% (k k) ((1 ldw) (1 *))
                                   w-%offset%))
            (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                                 ((1 lda) (1 *)) a-%offset%)
                    (f2cl-lib:fref w-%data% ((f2cl-lib:int-add k 1) k)
                                   ((1 ldw) (1 *)) w-%offset%))
            (setf (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add k 1)
                                  (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
                    (f2cl-lib:fref w-%data%
                                   ((f2cl-lib:int-add k 1)
                                    (f2cl-lib:int-add k 1))
                                   ((1 ldw) (1 *)) w-%offset%))))))
          (cond
           ((= kstep 1)
            (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%) kp))
           (t
            (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
                    (f2cl-lib:int-sub p))
            (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1)) ((1 *))
                                 ipiv-%offset%)
                    (f2cl-lib:int-sub kp))))
          (setf k (f2cl-lib:int-add k kstep))
          (go label70)
        label90
          (f2cl-lib:fdo (j k (f2cl-lib:int-add j nb))
                        ((> j n) nil)
            (tagbody
              (setf jb
                      (min (the f2cl-lib:integer4 nb)
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add (f2cl-lib:int-sub n j)
                                                  1))))
              (f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
                            ((> jj
                                (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
                             nil)
                (tagbody
                  (multiple-value-bind
                      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                       var-9 var-10)
```

```
                (dgemv "No transpose"
                 (f2cl-lib:int-sub (f2cl-lib:int-add j jb) jj)
                 (f2cl-lib:int-sub k 1) (- one)
                 (f2cl-lib:array-slice a-%data% double-float (jj 1)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice w-%data% double-float (jj 1)
                                       ((1 ldw) (1 *)) w-%offset%)
                 ldw one
                 (f2cl-lib:array-slice a-%data% double-float (jj jj)
                                       ((1 lda) (1 *)) a-%offset%)
                 1)
               (declare
                (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9
                 var-10))
               (when var-5 (setf lda var-5))
               (when var-7 (setf ldw var-7))
               (when var-8 (setf one var-8)))
            label100))
        (if (<= (f2cl-lib:int-add j jb) n)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10 var-11 var-12)
                (dgemm "No transpose" "Transpose"
                 (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) jb
                 (f2cl-lib:int-sub k 1) (- one)
                 (f2cl-lib:array-slice a-%data% double-float
                                       ((+ j jb) 1) ((1 lda) (1 *))
                                       a-%offset%)
                 lda
                 (f2cl-lib:array-slice w-%data% double-float (j 1)
                                       ((1 ldw) (1 *)) w-%offset%)
                 ldw one
                 (f2cl-lib:array-slice a-%data% double-float
                                       ((+ j jb) j) ((1 lda) (1 *))
                                       a-%offset%)
                 lda)
               (declare
                (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-8
                 var-11))
               (when var-3 (setf jb var-3))
               (when var-7 (setf lda var-7))
               (when var-9 (setf ldw var-9))
               (when var-10 (setf one var-10))
               (when var-12 (setf lda var-12))))
        label110))
    (setf j (f2cl-lib:int-sub k 1))
   label120
    (setf kstep 1)
    (setf jp1 1)
```

```
            (setf jj j)
            (setf jp2 (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
            (cond
             ((< jp2 0) (setf jp2 (f2cl-lib:int-sub jp2))
              (setf j (f2cl-lib:int-sub j 1))
              (setf jp1
                      (f2cl-lib:int-sub
                       (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)))
              (setf kstep 2)))
            (setf j (f2cl-lib:int-sub j 1))
            (if (and (/= jp2 jj) (>= j 1))
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (dswap j
                     (f2cl-lib:array-slice a-%data% double-float (jp2 1)
                                           ((1 lda) (1 *)) a-%offset%)
                     lda
                     (f2cl-lib:array-slice a-%data% double-float (jj 1)
                                           ((1 lda) (1 *)) a-%offset%)
                     lda)
                  (declare (ignore var-1 var-3))
                  (when var-0 (setf j var-0))
                  (when var-2 (setf lda var-2))
                  (when var-4 (setf lda var-4))))
            (setf jj (f2cl-lib:int-add j 1))
            (if (and (/= jp1 jj) (= kstep 2))
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (dswap j
                     (f2cl-lib:array-slice a-%data% double-float (jp1 1)
                                           ((1 lda) (1 *)) a-%offset%)
                     lda
                     (f2cl-lib:array-slice a-%data% double-float (jj 1)
                                           ((1 lda) (1 *)) a-%offset%)
                     lda)
                  (declare (ignore var-1 var-3))
                  (when var-0 (setf j var-0))
                  (when var-2 (setf lda var-2))
                  (when var-4 (setf lda var-4))))
            (if (>= j 1)
                (go label120))
            (setf kb (f2cl-lib:int-sub k 1)))))
        (go end_label)
       end_label
        (return (values uplo nil nil kb nil lda nil nil ldw info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlasyf_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
```

```
'((string)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::uplo nil nil
  fortran-to-lisp::kb nil
  fortran-to-lisp::lda nil nil
  fortran-to-lisp::ldw
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

58.7 dsycon

58.7.1 Axiom unit tests

— dsycon.input —

```
)set break resume
)sys rm -f dsycon.output
)spool dsycon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.7.2 Axiom help page

— dsycon.help —

```
====================================================================
dsycon examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsycon (UPLO, N, A, LDA, IPIV, ANORM, RCOND, WORK, IWORK,
           INFO)
           DSYCON

Function/Subroutine Documentation
   subroutine dsycon (character UPLO, integer N, double precision, dimension(
       lda, * ) A, integer LDA, integer, dimension( * ) IPIV, double precision
       ANORM, double precision RCOND, double precision, dimension( * ) WORK,
       integer, dimension( * ) IWORK, integer INFO)
       DSYCON

Purpose:
   DSYCON estimates the reciprocal of the condition number (in the
   1-norm) of a real symmetric matrix A using the factorization
   A = U*D*U**T or A = L*D*L**T computed by DSYTRF.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by DSYTRF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).
```

```
IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by DSYTRF.

ANORM
  ANORM is DOUBLE PRECISION
  The 1-norm of the original matrix A.

RCOND
  RCOND is DOUBLE PRECISION
  The reciprocal of the condition number of the matrix A,
  computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is an
  estimate of the 1-norm of inv(A) computed in this routine.

WORK
  WORK is DOUBLE PRECISION array, dimension (2*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

58.7.3 fortran code

— dsycon.f —

```
*  =====================================================================
      SUBROUTINE DSYCON( UPLO, N, A, LDA, IPIV, ANORM, RCOND, WORK,
     $                   IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
      DOUBLE PRECISION   ANORM, RCOND
*     ..
```

```
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, KASE
      DOUBLE PRECISION   AINVNM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLACN2, DSYTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYCON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
```

```
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.LE.ZERO ) THEN
         RETURN
      END IF
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
         DO 10 I = N, 1, -1
            IF( IPIV( I ).GT.0 .AND. A( I, I ).EQ.ZERO )
     $         RETURN
   10    CONTINUE
      ELSE
*
*        Lower triangular storage: examine D from top to bottom.
*
         DO 20 I = 1, N
            IF( IPIV( I ).GT.0 .AND. A( I, I ).EQ.ZERO )
     $         RETURN
   20    CONTINUE
      END IF
*
*     Estimate the 1-norm of the inverse.
*
      KASE = 0
   30 CONTINUE
      CALL DLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
*
*        Multiply by inv(L*D*L**T) or inv(U*D*U**T).
*
         CALL DSYTRS( UPLO, N, 1, A, LDA, IPIV, WORK, N, INFO )
         GO TO 30
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
      RETURN
*
*     End of DSYCON
```

```
*
      END
```

———-

58.7.4 lisp code

— **dsycon.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dsycon (uplo n a lda ipiv anorm rcond work iwork info)
    (declare (type (double-float) rcond anorm)
             (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array double-float (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (work double-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (ainvnm 0.0d0) (i 0) (kase 0) (upper nil))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (double-float) ainvnm)
                 (type (f2cl-lib:integer4) i kase)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2))
```

```
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4))
   ((< anorm zero) (setf info -6)))
  (cond
   ((/= info 0) (xerbla "DSYCON" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf rcond zero)
  (cond ((= n 0) (setf rcond one) (go end_label))
        ((<= anorm zero) (go end_label)))
  (cond
   (upper
    (f2cl-lib:fdo (i n (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
      (tagbody
        (if (and
             (> (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) 0)
             (= (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                zero))
            (go end_label))
      label10)))
   (t
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (if (and
             (> (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) 0)
             (= (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                zero))
            (go end_label))
      label20))))
  (setf kase 0)
 label30
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacn2 n
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1)) ((1 *))
                             work-%offset%)
       work iwork ainvnm kase isave)
    (declare (ignore var-1 var-2 var-3 var-6))
    (setf n var-0)
    (setf ainvnm var-4)
    (setf kase var-5))
  (cond
   ((/= kase 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dsytrs uplo n 1 a lda ipiv work n info)
      (declare (ignore var-2 var-3 var-5 var-6))
      (when var-0 (setf uplo var-0))
      (when var-1 (setf n var-1))
      (when var-4 (setf lda var-4))
```

```
            (when var-7 (setf n var-7))
            (when var-8 (setf info var-8)))
          (go label30)))
        (if (/= ainvnm zero)
            (setf rcond (/ (/ one ainvnm) anorm)))
        (go end_label)
       end_label
        (return (values uplo n nil lda nil nil rcond nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsycon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (double-float) (double-float)
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::rcond nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::dlacn2))))
```

———-

58.8 dsycon_rook

58.8.1 Axiom unit tests

— **dsyconrook.input** —

```
)set break resume
)sys rm -f dsyconrook.output
)spool dsyconrook.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

58.8.2 Axiom help page

— dsycon_rook.help —

```
=====================================================================
dsycon_rook examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsycon_rook (UPLO, N, A, LDA, IPIV, ANORM, RCOND, WORK,
           IWORK, INFO)
           DSYCON_ROOK

Function/Subroutine Documentation
   subroutine dsycon_rook (character UPLO, integer N, double precision,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       double precision ANORM, double precision RCOND, double precision,
       dimension( * ) WORK, integer, dimension( * ) IWORK, integer INFO)
       DSYCON_ROOK

Purpose:
   DSYCON_ROOK estimates the reciprocal of the condition number (in the
   1-norm) of a real symmetric matrix A using the factorization
   A = U*D*U**T or A = L*D*L**T computed by DSYTRF_ROOK.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
```

```
  = 'U':  Upper triangular, form is A = U*D*U**T;
  = 'L':  Lower triangular, form is A = L*D*L**T.

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  The block diagonal matrix D and the multipliers used to
  obtain the factor U or L as computed by DSYTRF_ROOK.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by DSYTRF_ROOK.

ANORM
  ANORM is DOUBLE PRECISION
  The 1-norm of the original matrix A.

RCOND
  RCOND is DOUBLE PRECISION
  The reciprocal of the condition number of the matrix A,
  computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is an
  estimate of the 1-norm of inv(A) computed in this routine.

WORK
  WORK is DOUBLE PRECISION array, dimension (2*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

58.8.3 fortran code

— dsycon_rook.f —

```
*  =====================================================================
      SUBROUTINE DSYCON_ROOK( UPLO, N, A, LDA, IPIV, ANORM, RCOND, WORK,
     $                   IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
      DOUBLE PRECISION   ANORM, RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, KASE
      DOUBLE PRECISION   AINVNM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLACN2, DSYTRS_ROOK, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
```

```
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYCON_ROOK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.LE.ZERO ) THEN
         RETURN
      END IF
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
         DO 10 I = N, 1, -1
            IF( IPIV( I ).GT.0 .AND. A( I, I ).EQ.ZERO )
     $         RETURN
   10    CONTINUE
      ELSE
*
*        Lower triangular storage: examine D from top to bottom.
*
         DO 20 I = 1, N
            IF( IPIV( I ).GT.0 .AND. A( I, I ).EQ.ZERO )
     $         RETURN
   20    CONTINUE
      END IF
*
*     Estimate the 1-norm of the inverse.
*
      KASE = 0
   30 CONTINUE
      CALL DLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
*
```

```
*        Multiply by inv(L*D*L**T) or inv(U*D*U**T).
*
         CALL DSYTRS_ROOK( UPLO, N, 1, A, LDA, IPIV, WORK, N, INFO )
         GO TO 30
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
      RETURN
*
*     End of DSYCON_ROOK
*
      END
```

58.8.4 lisp code

— dsycon_rook.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dsycon_rook (uplo n a lda ipiv anorm rcond work iwork info)
    (declare (type (double-float) rcond anorm)
             (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array double-float (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (work double-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (ainvnm 0.0d0) (i 0) (kase 0) (upper nil))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (double-float) ainvnm)
                 (type (f2cl-lib:integer4) i kase)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
```

```
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
  (cond
   ((and (not upper)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4))
   ((< anorm zero) (setf info -6)))
  (cond
   ((/= info 0) (xerbla "DSYCON_ROOK" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf rcond zero)
  (cond ((= n 0) (setf rcond one) (go end_label))
        ((<= anorm zero) (go end_label)))
  (cond
   (upper
    (f2cl-lib:fdo (i n (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
      (tagbody
        (if (and
             (> (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) 0)
             (= (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                zero))
            (go end_label))
      label10)))
   (t
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (if (and
             (> (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) 0)
             (= (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                zero))
            (go end_label))
      label20))))
  (setf kase 0)
 label30
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacn2 n
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1)) ((1 *))
                             work-%offset%)
```

```
            work iwork ainvnm kase isave)
          (declare (ignore var-1 var-2 var-3 var-6))
          (setf n var-0)
          (setf ainvnm var-4)
          (setf kase var-5))
        (cond
         ((/= kase 0)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (dsytrs_rook uplo n 1 a lda ipiv work n info)
            (declare (ignore var-2 var-3 var-5 var-6))
            (when var-0 (setf uplo var-0))
            (when var-1 (setf n var-1))
            (when var-4 (setf lda var-4))
            (when var-7 (setf n var-7))
            (when var-8 (setf info var-8)))
          (go label30)))
        (if (/= ainvnm zero)
            (setf rcond (/ (/ one ainvnm) anorm)))
        (go end_label)
       end_label
        (return (values uplo n nil lda nil nil rcond nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsycon_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (double-float) (double-float)
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::rcond nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::dlacn2))))
```

58.9 dsyconv

58.9.1 Axiom unit tests

— dsyconv.input —

```
)set break resume
)sys rm -f dsyconv.output
)spool dsyconv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

58.9.2 Axiom help page

— dsyconv.help —

```
======================================================================
dsyconv examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsyconv (UPLO, WAY, N, A, LDA, IPIV, E, INFO)
           DSYCONV

Function/Subroutine Documentation
   subroutine dsyconv (character UPLO, character WAY, integer N, double
       precision, dimension( lda, * ) A, integer LDA, integer, dimension( * )
       IPIV, double precision, dimension( * ) E, integer INFO)
       DSYCONV

Purpose:
```

```
  DSYCONV convert A given by TRF into L and D and vice-versa.
  Get Non-diag elements of D (returned in workspace) and
  apply or reverse permutation done in TRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 WAY
   WAY is CHARACTER*1
   = 'C': Convert
   = 'R': Revert

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by DSYTRF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by DSYTRF.

 E
   E is DOUBLE PRECISION array, dimension (N)
   E stores the supdiagonal/subdiagonal of the symmetric 1-by-1
   or 2-by-2 block diagonal matrix D in LDLT.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

58.9.3 fortran code

— dsyconv.f —

```
*  =====================================================================
      SUBROUTINE DSYCONV( UPLO, WAY, N, A, LDA, IPIV, E, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO, WAY
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     .. Local Scalars ..
      LOGICAL            UPPER, CONVERT
      INTEGER            I, IP, J
      DOUBLE PRECISION   TEMP
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      CONVERT = LSAME( WAY, 'C' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.CONVERT .AND. .NOT.LSAME( WAY, 'R' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
```

```
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5

      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYCONV', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*      A is UPPER
*
*      Convert A (A is upper)
*
*        Convert VALUE
*
         IF ( CONVERT ) THEN
            I=N
            E(1)=ZERO
            DO WHILE ( I .GT. 1 )
               IF( IPIV(I) .LT. 0 ) THEN
                  E(I)=A(I-1,I)
                  E(I-1)=ZERO
                  A(I-1,I)=ZERO
                  I=I-1
               ELSE
                  E(I)=ZERO
               ENDIF
               I=I-1
            END DO
*
*        Convert PERMUTATIONS
*
         I=N
         DO WHILE ( I .GE. 1 )
            IF( IPIV(I) .GT. 0) THEN
               IP=IPIV(I)
               IF( I .LT. N) THEN
                  DO 12 J= I+1,N
                    TEMP=A(IP,J)
                    A(IP,J)=A(I,J)
                    A(I,J)=TEMP
 12            CONTINUE
               ENDIF
```

```
            ELSE
              IP=-IPIV(I)
               IF( I .LT. N) THEN
             DO 13 J= I+1,N
                 TEMP=A(IP,J)
                 A(IP,J)=A(I-1,J)
                 A(I-1,J)=TEMP
 13            CONTINUE
                ENDIF
                I=I-1
           ENDIF
           I=I-1
        END DO

         ELSE
*
*      Revert A (A is upper)
*
*
*        Revert PERMUTATIONS
*
            I=1
            DO WHILE ( I .LE. N )
               IF( IPIV(I) .GT. 0 ) THEN
                  IP=IPIV(I)
                  IF( I .LT. N) THEN
                  DO J= I+1,N
                    TEMP=A(IP,J)
                    A(IP,J)=A(I,J)
                    A(I,J)=TEMP
                  END DO
                  ENDIF
               ELSE
                 IP=-IPIV(I)
                 I=I+1
                 IF( I .LT. N) THEN
                    DO J= I+1,N
                       TEMP=A(IP,J)
                       A(IP,J)=A(I-1,J)
                       A(I-1,J)=TEMP
                    END DO
                 ENDIF
               ENDIF
               I=I+1
            END DO
*
*        Revert VALUE
*
            I=N
            DO WHILE ( I .GT. 1 )
```

```
               IF( IPIV(I) .LT. 0 ) THEN
                  A(I-1,I)=E(I)
                  I=I-1
               ENDIF
               I=I-1
            END DO
         END IF
      ELSE
*
*      A is LOWER
*
         IF ( CONVERT ) THEN
*
*      Convert A (A is lower)
*
*
*        Convert VALUE
*
            I=1
            E(N)=ZERO
            DO WHILE ( I .LE. N )
               IF( I.LT.N .AND. IPIV(I) .LT. 0 ) THEN
                  E(I)=A(I+1,I)
                  E(I+1)=ZERO
                  A(I+1,I)=ZERO
                  I=I+1
               ELSE
                  E(I)=ZERO
               ENDIF
               I=I+1
            END DO
*
*        Convert PERMUTATIONS
*
         I=1
         DO WHILE ( I .LE. N )
            IF( IPIV(I) .GT. 0 ) THEN
               IP=IPIV(I)
               IF (I .GT. 1) THEN
               DO 22 J= 1,I-1
                 TEMP=A(IP,J)
                 A(IP,J)=A(I,J)
                 A(I,J)=TEMP
 22            CONTINUE
               ENDIF
            ELSE
              IP=-IPIV(I)
              IF (I .GT. 1) THEN
              DO 23 J= 1,I-1
                 TEMP=A(IP,J)
```

```
                A(IP,J)=A(I+1,J)
                A(I+1,J)=TEMP
 23           CONTINUE
              ENDIF
              I=I+1
           ENDIF
           I=I+1
        END DO
         ELSE
*
*      Revert A (A is lower)
*
*
*        Revert PERMUTATIONS
*
            I=N
            DO WHILE ( I .GE. 1 )
               IF( IPIV(I) .GT. 0 ) THEN
                  IP=IPIV(I)
                  IF (I .GT. 1) THEN
                     DO J= 1,I-1
                        TEMP=A(I,J)
                        A(I,J)=A(IP,J)
                        A(IP,J)=TEMP
                     END DO
                  ENDIF
               ELSE
                  IP=-IPIV(I)
                  I=I-1
                  IF (I .GT. 1) THEN
                     DO J= 1,I-1
                        TEMP=A(I+1,J)
                        A(I+1,J)=A(IP,J)
                        A(IP,J)=TEMP
                     END DO
                  ENDIF
               ENDIF
               I=I-1
            END DO
*
*        Revert VALUE
*
            I=1
            DO WHILE ( I .LE. N-1 )
               IF( IPIV(I) .LT. 0 ) THEN
                  A(I+1,I)=E(I)
                  I=I+1
               ENDIF
               I=I+1
            END DO
```

```
         END IF
      END IF

      RETURN
*
*     End of DSYCONV
*
      END
```

———-

58.9.4 lisp code

— **dsyconv.lisp** —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun dsyconv (uplo way n a lda ipiv e info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array double-float (*)) e a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) way uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (way character way-%data% way-%offset%)
         (a double-float a-%data% a-%offset%)
         (e double-float e-%data% e-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((temp 0.0d0) (i 0) (ip 0) (j 0) (upper nil) (convert nil))
        (declare (type (double-float) temp)
                 (type (f2cl-lib:integer4) i ip j)
                 (type f2cl-lib:logical upper convert))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (setf convert
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame way "C")
                  (declare (ignore var-1))
                  (when var-0 (setf way var-0))
                  ret-val))
        (cond
         ((and (not upper)
```

```
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame uplo "L")
           (declare (ignore var-1))
           (when var-0 (setf uplo var-0))
           ret-val)))
  (setf info -1))
 ((and (not convert)
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame way "R")
           (declare (ignore var-1))
           (when var-0 (setf way var-0))
           ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5)))
(cond
 ((/= info 0) (xerbla "DSYCONV" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(cond
 (upper
  (cond
   (convert
    (tagbody
      (setf i n)
      (setf (f2cl-lib:fref e-%data% (1) ((1 *)) e-%offset%) zero)
     label100000
      (if (not (> i 1))
          (go label100001))
      (cond
       ((< (f2cl-lib:fref ipiv (i) ((1 *))) 0)
        (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) i)
                                ((1 lda) (1 *)) a-%offset%))
        (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1)) ((1 *))
                              e-%offset%)
                zero)
        (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) i)
                              ((1 lda) (1 *)) a-%offset%)
                zero)
        (setf i (f2cl-lib:int-sub i 1)))
       (t (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) zero)))
      (setf i (f2cl-lib:int-sub i 1))
      (go label100000)
     label100001
      (setf i n)
```

```
label100002
 (if (not (>= i 1))
     (go label100003))
 (cond
  ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
   (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
   (cond
    ((< i n)
     (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                    (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp
                 (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                a-%offset%))
         (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                              a-%offset%)
                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%))
         (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                              a-%offset%)
                 temp)
       label12)))))
  (t
   (setf ip
           (f2cl-lib:int-sub
            (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                           ipiv-%offset%)))
   (cond
    ((< i n)
     (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                    (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf temp
                 (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                a-%offset%))
         (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                              a-%offset%)
                 (f2cl-lib:fref a-%data%
                                ((f2cl-lib:int-sub i 1) j)
                                ((1 lda) (1 *)) a-%offset%))
         (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) j)
                              ((1 lda) (1 *)) a-%offset%)
                 temp)
       label13))))
   (setf i (f2cl-lib:int-sub i 1))))
 (setf i (f2cl-lib:int-sub i 1))
 (go label100002)
label100003))
```

```
(t
 (tagbody
   (setf i 1)
  label100004
   (if (not (<= i n))
       (go label100005))
   (cond
    ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
     (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
     (cond
      ((< i n)
       (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                      (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (setf temp
                   (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                  a-%offset%))
           (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                a-%offset%)
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%))
           (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                   temp)
          label100006)))))
    (t
     (setf ip
             (f2cl-lib:int-sub
              (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                             ipiv-%offset%)))
     (setf i (f2cl-lib:int-add i 1))
     (cond
      ((< i n)
       (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                      (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (setf temp
                   (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                  a-%offset%))
           (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                a-%offset%)
                   (f2cl-lib:fref a-%data%
                                  ((f2cl-lib:int-sub i 1) j)
                                  ((1 lda) (1 *)) a-%offset%))
           (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) j)
                                ((1 lda) (1 *)) a-%offset%)
                   temp)
          label100007))))))
```

```
        (setf i (f2cl-lib:int-add i 1))
        (go label100004)
      label100005
        (setf i n)
      label100008
        (if (not (> i 1))
            (go label100009))
        (cond
         ((< (f2cl-lib:fref ipiv (i) ((1 *))) 0)
          (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) i)
                                ((1 lda) (1 *)) a-%offset%)
                  (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))
          (setf i (f2cl-lib:int-sub i 1))))
        (setf i (f2cl-lib:int-sub i 1))
        (go label100008)
      label100009))))
  (t
   (cond
    (convert
     (tagbody
       (setf i 1)
       (setf (f2cl-lib:fref e-%data% (n) ((1 *)) e-%offset%) zero)
      label100010
       (if (not (<= i n))
           (go label100011))
       (cond
        ((and (< i n) (< (f2cl-lib:fref ipiv (i) ((1 *))) 0))
         (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                 (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                                  ((1 lda) (1 *)) a-%offset%))
         (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                e-%offset%)
                 zero)
         (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                                ((1 lda) (1 *)) a-%offset%)
                 zero)
         (setf i (f2cl-lib:int-add i 1)))
        (t (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) zero)))
       (setf i (f2cl-lib:int-add i 1))
       (go label100010)
      label100011
       (setf i 1)
      label100012
       (if (not (<= i n))
           (go label100013))
       (cond
        ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
         (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
         (cond
          ((> i 1)
```

```
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j
                              (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                           nil)
              (tagbody
                (setf temp
                        (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                       a-%offset%))
                (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                     a-%offset%)
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%)
                        temp)
             label22)))))
        (t
         (setf ip
                 (f2cl-lib:int-sub
                  (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                                 ipiv-%offset%)))
         (cond
          ((> i 1)
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j
                             (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                          nil)
             (tagbody
               (setf temp
                       (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                      a-%offset%))
               (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                    a-%offset%)
                       (f2cl-lib:fref a-%data%
                                      ((f2cl-lib:int-add i 1) j)
                                      ((1 lda) (1 *)) a-%offset%))
               (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) j)
                                    ((1 lda) (1 *)) a-%offset%)
                       temp)
             label23))))
         (setf i (f2cl-lib:int-add i 1))))
       (setf i (f2cl-lib:int-add i 1))
       (go label100012)
      label100013))
    (t
     (tagbody
       (setf i n)
      label100014
       (if (not (>= i 1))
           (go label100015))
```

```
(cond
 ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
  (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
  (cond
   ((> i 1)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j
                      (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf temp
                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%))
        (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                             a-%offset%)
                (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                               a-%offset%))
        (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                             a-%offset%)
                temp)
       label100016)))))
 (t
  (setf ip
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                          ipiv-%offset%)))
  (setf i (f2cl-lib:int-sub i 1))
  (cond
   ((> i 1)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j
                      (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf temp
                (f2cl-lib:fref a-%data%
                               ((f2cl-lib:int-add i 1) j)
                               ((1 lda) (1 *)) a-%offset%))
        (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) j)
                             ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                               a-%offset%))
        (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                             a-%offset%)
                temp)
       label100017))))))
 (setf i (f2cl-lib:int-sub i 1))
 (go label100014)
label100015
 (setf i 1)
```

```
            label100018
             (if (not (<= i (f2cl-lib:int-sub n 1)))
                 (go label100019))
             (cond
              ((< (f2cl-lib:fref ipiv (i) ((1 *))) 0)
               (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                                     ((1 lda) (1 *)) a-%offset%)
                       (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))
               (setf i (f2cl-lib:int-add i 1))))
             (setf i (f2cl-lib:int-add i 1))
             (go label100018)
            label100019)))))
        (go end_label)
       end_label
        (return (values uplo way nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsyconv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::way nil nil nil
                                              nil nil fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

58.10 dsyequb

58.10.1 Axiom unit tests

— dsyequb.input —

```
)set break resume
```

```
)sys rm -f dsyequb.output
)spool dsyequb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.10.2 Axiom help page

— dsyequb.help —

```
=====================================================================
dsyequb examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsyequb (UPLO, N, A, LDA, S, SCOND, AMAX, WORK, INFO)
           DSYEQUB

Function/Subroutine Documentation
   subroutine dsyequb (character UPLO, integer N, double precision, dimension(
       lda, * ) A, integer LDA, double precision, dimension( * ) S, double
       precision SCOND, double precision AMAX, double precision, dimension( *
       ) WORK, integer INFO)
       DSYEQUB

Purpose:
   DSYEQUB computes row and column scalings intended to equilibrate a
   symmetric matrix A and reduce its condition number
   (with respect to the two-norm).  S contains the scale factors,
   S(i) = 1/sqrt(A(i,i)), chosen so that the scaled matrix B with
   elements B(i,j) = S(i)*A(i,j)*S(j) has ones on the diagonal.  This
   choice of S puts the condition number of B within a factor N of the
   smallest possible condition number over all possible diagonal scalings.

Parameters:

 UPLO
   UPLO is CHARACTER*1
```

```
  Specifies whether the details of the factorization are stored
  as an upper or lower triangular matrix.
  = 'U':  Upper triangular, form is A = U*D*U**T;
  = 'L':  Lower triangular, form is A = L*D*L**T.

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  The N-by-N symmetric matrix whose scaling
  factors are to be computed.  Only the diagonal elements of A
  are referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

S
  S is DOUBLE PRECISION array, dimension (N)
  If INFO = 0, S contains the scale factors for A.

SCOND
  SCOND is DOUBLE PRECISION
  If INFO = 0, S contains the ratio of the smallest S(i) to
  the largest S(i).  If SCOND >= 0.1 and AMAX is neither too
  large nor too small, it is not worth scaling by S.

AMAX
  AMAX is DOUBLE PRECISION
  Absolute value of largest matrix element.  If AMAX is very
  close to overflow or very close to underflow, the matrix
  should be scaled.

WORK
  WORK is DOUBLE PRECISION array, dimension (3*N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, the i-th diagonal element is nonpositive.
```

58.10.3 fortran code

— dsyequb.f —

```
*  =====================================================================
      SUBROUTINE DSYEQUB( UPLO, N, A, LDA, S, SCOND, AMAX, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, N
      DOUBLE PRECISION   AMAX, SCOND
      CHARACTER          UPLO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), S( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
      INTEGER            MAX_ITER
      PARAMETER          ( MAX_ITER = 100 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, ITER
      DOUBLE PRECISION   AVG, STD, TOL, C0, C1, C2, T, U, SI, D, BASE,
     $                   SMIN, SMAX, SMLNUM, BIGNUM, SCALE, SUMSQ
      LOGICAL            UP
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      LOGICAL            LSAME
      EXTERNAL           DLAMCH, LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASSQ
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, INT, LOG, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test input parameters.
```

```
*
      INFO = 0
      IF ( .NOT. ( LSAME( UPLO, 'U' ) .OR. LSAME( UPLO, 'L' ) ) ) THEN
        INFO = -1
      ELSE IF ( N .LT. 0 ) THEN
        INFO = -2
      ELSE IF ( LDA .LT. MAX( 1, N ) ) THEN
        INFO = -4
      END IF
      IF ( INFO .NE. 0 ) THEN
        CALL XERBLA( 'DSYEQUB', -INFO )
        RETURN
      END IF

      UP = LSAME( UPLO, 'U' )
      AMAX = ZERO
*
*     Quick return if possible.
*
      IF ( N .EQ. 0 ) THEN
        SCOND = ONE
        RETURN
      END IF

      DO I = 1, N
        S( I ) = ZERO
      END DO

      AMAX = ZERO
      IF ( UP ) THEN
         DO J = 1, N
            DO I = 1, J-1
               S( I ) = MAX( S( I ), ABS( A( I, J ) ) )
               S( J ) = MAX( S( J ), ABS( A( I, J ) ) )
               AMAX = MAX( AMAX, ABS( A(I, J) ) )
            END DO
            S( J ) = MAX( S( J ), ABS( A( J, J ) ) )
            AMAX = MAX( AMAX, ABS( A( J, J ) ) )
         END DO
      ELSE
         DO J = 1, N
            S( J ) = MAX( S( J ), ABS( A( J, J ) ) )
            AMAX = MAX( AMAX, ABS( A( J, J ) ) )
            DO I = J+1, N
               S( I ) = MAX( S( I ), ABS( A( I, J ) ) )
               S( J ) = MAX( S( J ), ABS( A( I, J ) ) )
               AMAX = MAX( AMAX, ABS( A( I, J ) ) )
            END DO
         END DO
      END IF
```

```
      DO J = 1, N
         S( J ) = 1.0D+0 / S( J )
      END DO

      TOL = ONE / SQRT(2.0D0 * N)

      DO ITER = 1, MAX_ITER
         SCALE = 0.0D+0
         SUMSQ = 0.0D+0
*       BETA = |A|S
        DO I = 1, N
           WORK(I) = ZERO
        END DO
        IF ( UP ) THEN
           DO J = 1, N
              DO I = 1, J-1
                 T = ABS( A( I, J ) )
                 WORK( I ) = WORK( I ) + ABS( A( I, J ) ) * S( J )
                 WORK( J ) = WORK( J ) + ABS( A( I, J ) ) * S( I )
              END DO
              WORK( J ) = WORK( J ) + ABS( A( J, J ) ) * S( J )
           END DO
        ELSE
           DO J = 1, N
              WORK( J ) = WORK( J ) + ABS( A( J, J ) ) * S( J )
              DO I = J+1, N
                 T = ABS( A( I, J ) )
                 WORK( I ) = WORK( I ) + ABS( A( I, J ) ) * S( J )
                 WORK( J ) = WORK( J ) + ABS( A( I, J ) ) * S( I )
              END DO
           END DO
        END IF

*       avg = s^T beta / n
        AVG = 0.0D+0
        DO I = 1, N
          AVG = AVG + S( I )*WORK( I )
        END DO
        AVG = AVG / N

        STD = 0.0D+0
        DO I = 2*N+1, 3*N
           WORK( I ) = S( I-2*N ) * WORK( I-2*N ) - AVG
        END DO
        CALL DLASSQ( N, WORK( 2*N+1 ), 1, SCALE, SUMSQ )
        STD = SCALE * SQRT( SUMSQ / N )

        IF ( STD .LT. TOL * AVG ) GOTO 999

        DO I = 1, N
```

```
          T = ABS( A( I, I ) )
          SI = S( I )
          C2 = ( N-1 ) * T
          C1 = ( N-2 ) * ( WORK( I ) - T*SI )
          C0 = -(T*SI)*SI + 2*WORK( I )*SI - N*AVG
          D = C1*C1 - 4*C0*C2

          IF ( D .LE. 0 ) THEN
            INFO = -1
            RETURN
          END IF
          SI = -2*C0 / ( C1 + SQRT( D ) )

          D = SI - S( I )
          U = ZERO
          IF ( UP ) THEN
            DO J = 1, I
              T = ABS( A( J, I ) )
              U = U + S( J )*T
              WORK( J ) = WORK( J ) + D*T
            END DO
            DO J = I+1,N
              T = ABS( A( I, J ) )
              U = U + S( J )*T
              WORK( J ) = WORK( J ) + D*T
            END DO
          ELSE
            DO J = 1, I
              T = ABS( A( I, J ) )
              U = U + S( J )*T
              WORK( J ) = WORK( J ) + D*T
            END DO
            DO J = I+1,N
              T = ABS( A( J, I ) )
              U = U + S( J )*T
              WORK( J ) = WORK( J ) + D*T
            END DO
          END IF

          AVG = AVG + ( U + WORK( I ) ) * D / N
          S( I ) = SI

        END DO

      END DO

999   CONTINUE

      SMLNUM = DLAMCH( 'SAFEMIN' )
      BIGNUM = ONE / SMLNUM
```

```
      SMIN = BIGNUM
      SMAX = ZERO
      T = ONE / SQRT(AVG)
      BASE = DLAMCH( 'B' )
      U = ONE / LOG( BASE )
      DO I = 1, N
        S( I ) = BASE ** INT( U * LOG( S( I ) * T ) )
        SMIN = MIN( SMIN, S( I ) )
        SMAX = MAX( SMAX, S( I ) )
      END DO
      SCOND = MAX( SMIN, SMLNUM ) / MIN( SMAX, BIGNUM )
*
      END
```

———-

58.10.4 lisp code

— dsyequb.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0) (max_iter 100))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (f2cl-lib:integer4 100 100) max_iter)
           (ignorable one zero max_iter))
  (defun dsyequb (uplo n a lda s scond amax work info)
    (declare (type (double-float) amax scond)
             (type (array double-float (*)) work s a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (s double-float s-%data% s-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((up nil) (avg 0.0d0) (std 0.0d0) (tol 0.0d0) (c0 0.0d0) (c1 0.0d0)
             (c2 0.0d0) (t$ 0.0d0) (u 0.0d0) (si 0.0d0) (d 0.0d0) (base 0.0d0)
             (smin 0.0d0) (smax 0.0d0) (smlnum 0.0d0) (bignum 0.0d0)
             (scale 0.0d0) (sumsq 0.0d0) (i 0) (j 0) (iter 0))
        (declare (type f2cl-lib:logical up)
                 (type (double-float) avg std tol c0 c1 c2 t$ u si d base smin
                  smax smlnum bignum scale sumsq)
                 (type (f2cl-lib:integer4) i j iter))
        (setf info 0)
        (cond
         ((not
           (or
            (multiple-value-bind (ret-val var-0 var-1)
```

```
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4)))
(cond
 ((/= info 0) (xerbla "DSYEQUB" (f2cl-lib:int-sub info))
  (go end_label)))
(setf up
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(setf amax zero)
(cond ((= n 0) (setf scond one) (go end_label)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%) zero)
   label100000))
(setf amax zero)
(cond
 (up
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                  (max (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                       (abs
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))))
          (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                  (max (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                       (abs
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))))
          (setf amax
```

```
                    (max amax
                         (abs
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%))))
            label100002))
        (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                (max (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                     (abs
                      (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%))))
        (setf amax
                (max amax
                     (abs
                      (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%))))
      label100001)))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                (max (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                     (abs
                      (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%))))
        (setf amax
                (max amax
                     (abs
                      (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%))))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                    (max (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                         (abs
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%))))
            (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                    (max (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                         (abs
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%))))
            (setf amax
                    (max amax
                         (abs
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%))))
            label100004))
        label100003))))
```

```
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
            (/ 1.0d0 (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
   label100005))
(setf tol (/ one (f2cl-lib:fsqrt (* 2.0d0 n))))
(f2cl-lib:fdo (iter 1 (f2cl-lib:int-add iter 1))
              ((> iter max_iter) nil)
  (tagbody
    (setf scale 0.0d0)
    (setf sumsq 0.0d0)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                zero)
       label100007))
    (cond
     (up
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf t$
                      (abs
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)))
              (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)
                      (+
                       (f2cl-lib:fref work-%data% (i) ((1 *))
                                      work-%offset%)
                       (*
                        (abs
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%))
                        (f2cl-lib:fref s-%data% (j) ((1 *))
                                        s-%offset%))))
              (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                                   work-%offset%)
                      (+
                       (f2cl-lib:fref work-%data% (j) ((1 *))
                                      work-%offset%)
                       (*
                        (abs
```

```
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))
                      (f2cl-lib:fref s-%data% (i) ((1 *))
                                     s-%offset%))))
          label100009))
      (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
              (+
               (f2cl-lib:fref work-%data% (j) ((1 *))
                              work-%offset%)
               (*
                (abs
                 (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                a-%offset%))
                (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))))
    label100008)))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
             (+
              (f2cl-lib:fref work-%data% (j) ((1 *))
                             work-%offset%)
              (*
               (abs
                (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                               a-%offset%))
               (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))))
     (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                      (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf t$
                 (abs
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%)))
         (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                              work-%offset%)
                 (+
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)
                  (*
                   (abs
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))
                   (f2cl-lib:fref s-%data% (j) ((1 *))
                                   s-%offset%))))
         (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                              work-%offset%)
                 (+
```

```
                    (f2cl-lib:fref work-%data% (j) ((1 *))
                                   work-%offset%)
                    (*
                     (abs
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))
                     (f2cl-lib:fref s-%data% (i) ((1 *))
                                    s-%offset%))))
         label100011))
     label100010))))
(setf avg 0.0d0)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf avg
            (+ avg
               (* (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%))))
   label100012))
(setf avg (/ avg n))
(setf std 0.0d0)
(f2cl-lib:fdo (i (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) 1)
               (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-mul 3 n)) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (-
             (*
              (f2cl-lib:fref s-%data%
                             ((f2cl-lib:int-add i
                                                (f2cl-lib:int-mul
                                                 -1 2 n)))
                             ((1 *)) s-%offset%)
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add i
                                                (f2cl-lib:int-mul
                                                 -1 2 n)))
                             ((1 *)) work-%offset%))
             avg))
   label100013))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dlassq n
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                           work-%offset%)
     1 scale sumsq)
  (declare (ignore var-0 var-1 var-2))
  (setf scale var-3)
  (setf sumsq var-4))
```

```
(setf std (* scale (f2cl-lib:fsqrt (/ sumsq n))))
(if (< std (* tol avg))
    (go label999))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf t$
            (abs
             (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                            a-%offset%)))
    (setf si (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%))
    (setf c2 (* (f2cl-lib:int-sub n 1) t$))
    (setf c1
            (* (f2cl-lib:int-sub n 2)
               (-
                (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                (* t$ si))))
    (setf c0
            (-
             (+ (* (- (* t$ si)) si)
                (* 2
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                   si))
             (* n avg)))
    (setf d (+ (* c1 c1) (* -4 c0 c2)))
    (cond ((<= d 0) (setf info -1) (go end_label)))
    (setf si (/ (* -2 c0) (+ c1 (f2cl-lib:fsqrt d))))
    (setf d (- si (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
    (setf u zero)
    (cond
     (up
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j i) nil)
        (tagbody
          (setf t$
                  (abs
                   (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                  a-%offset%)))
          (setf u
                  (+ u
                     (*
                      (f2cl-lib:fref s-%data% (j) ((1 *))
                                     s-%offset%)
                      t$)))
          (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                               work-%offset%)
                  (+
                   (f2cl-lib:fref work-%data% (j) ((1 *))
```

```
                                work-%offset%)
                 (* d t$)))
      label100015))
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                   (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf t$
                (abs
                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)))
        (setf u
                (+ u
                   (*
                    (f2cl-lib:fref s-%data% (j) ((1 *))
                                   s-%offset%)
                    t$)))
        (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                             work-%offset%)
                (+
                 (f2cl-lib:fref work-%data% (j) ((1 *))
                                work-%offset%)
                 (* d t$)))
      label100016)))
  (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j i) nil)
      (tagbody
        (setf t$
                (abs
                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)))
        (setf u
                (+ u
                   (*
                    (f2cl-lib:fref s-%data% (j) ((1 *))
                                   s-%offset%)
                    t$)))
        (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                             work-%offset%)
                (+
                 (f2cl-lib:fref work-%data% (j) ((1 *))
                                work-%offset%)
                 (* d t$)))
      label100017))
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                   (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf t$
```

```
                        (abs
                         (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                        a-%offset%)))
                (setf u
                        (+ u
                           (*
                            (f2cl-lib:fref s-%data% (j) ((1 *))
                                           s-%offset%)
                            t$)))
                (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                                     work-%offset%)
                        (+
                         (f2cl-lib:fref work-%data% (j) ((1 *))
                                        work-%offset%)
                         (* d t$)))
               label100018))))
          (setf avg
                  (+ avg
                     (/
                      (*
                       (+ u
                          (f2cl-lib:fref work-%data% (i) ((1 *))
                                         work-%offset%))
                       d)
                      n)))
          (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%) si)
         label100014))
    label100006))
label999
 (setf smlnum (dlamch "SAFEMIN"))
 (setf bignum (/ one smlnum))
 (setf smin bignum)
 (setf smax zero)
 (setf t$ (/ one (f2cl-lib:fsqrt avg)))
 (setf base (dlamch "B"))
 (setf u (/ one (f2cl-lib:flog base)))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
             (expt base
                   (f2cl-lib:int
                    (* u
                       (f2cl-lib:flog
                        (*
                         (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                         t$))))))
     (setf smin
             (min smin (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
     (setf smax
```

```
                    (max smax (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
           label100019))
        (setf scond (/ (max smin smlnum) (min smax bignum)))
       end_label
        (return (values uplo nil nil nil nil scond amax nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsyequb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (double-float) (double-float)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil nil
                                              nil fortran-to-lisp::scond
                                              fortran-to-lisp::amax nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::dlassq))))
```

———-

58.11 dsygs2

58.11.1 Axiom unit tests

— dsygs2.input —

```
)set break resume
)sys rm -f dsygs2.output
)spool dsygs2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.11.2 Axiom help page

— dsygs2.help —

```
=====================================================================
dsygs2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsygs2 (ITYPE, UPLO, N, A, LDA, B, LDB, INFO)
           DSYGS2 reduces a symmetric definite generalized eigenproblem to
           standard form, using the factorization results obtained from spotrf
           (unblocked algorithm).

Function/Subroutine Documentation
   subroutine dsygs2 (integer ITYPE, character UPLO, integer N, double
       precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldb, * ) B, integer LDB, integer INFO)
       DSYGS2 reduces a symmetric definite generalized eigenproblem to
       standard form, using the factorization results obtained from spotrf
       (unblocked algorithm).

Purpose:
   DSYGS2 reduces a real symmetric-definite generalized eigenproblem
   to standard form.

   If ITYPE = 1, the problem is A*x = lambda*B*x,
   and A is overwritten by inv(U**T)*A*inv(U) or inv(L)*A*inv(L**T)

   If ITYPE = 2 or 3, the problem is A*B*x = lambda*x or
   B*A*x = lambda*x, and A is overwritten by U*A*U**T or L**T *A*L.

   B must have been previously factorized as U**T *U or L*L**T by DPOTRF.

Parameters:

 ITYPE
   ITYPE is INTEGER
   = 1: compute inv(U**T)*A*inv(U) or inv(L)*A*inv(L**T);
   = 2 or 3: compute U*A*U**T or L**T *A*L.
```

```
UPLO
  UPLO is CHARACTER*1
  Specifies whether the upper or lower triangular part of the
  symmetric matrix A is stored, and how B has been factorized.
  = 'U':  Upper triangular
  = 'L':  Lower triangular

N
  N is INTEGER
  The order of the matrices A and B.  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  n by n upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading n by n lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if INFO = 0, the transformed matrix, stored in the
  same format as A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

B
  B is DOUBLE PRECISION array, dimension (LDB,N)
  The triangular factor from the Cholesky factorization of B,
  as returned by DPOTRF.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

———-

58.11.3 fortran code

— dsygs2.f —

```
*  =====================================================================
      SUBROUTINE DSYGS2( ITYPE, UPLO, N, A, LDA, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, ITYPE, LDA, LDB, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, HALF
      PARAMETER          ( ONE = 1.0D0, HALF = 0.5D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            K
      DOUBLE PRECISION   AKK, BKK, CT
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DSCAL, DSYR2, DTRMV, DTRSV, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( ITYPE.LT.1 .OR. ITYPE.GT.3 ) THEN
         INFO = -1
      ELSE IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
```

```
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYGS2', -INFO )
         RETURN
      END IF
*
      IF( ITYPE.EQ.1 ) THEN
         IF( UPPER ) THEN
*
*           Compute inv(U**T)*A*inv(U)
*
            DO 10 K = 1, N
*
*              Update the upper triangle of A(k:n,k:n)
*
               AKK = A( K, K )
               BKK = B( K, K )
               AKK = AKK / BKK**2
               A( K, K ) = AKK
               IF( K.LT.N ) THEN
                  CALL DSCAL( N-K, ONE / BKK, A( K, K+1 ), LDA )
                  CT = -HALF*AKK
                  CALL DAXPY( N-K, CT, B( K, K+1 ), LDB, A( K, K+1 ),
     $                        LDA )
                  CALL DSYR2( UPLO, N-K, -ONE, A( K, K+1 ), LDA,
     $                        B( K, K+1 ), LDB, A( K+1, K+1 ), LDA )
                  CALL DAXPY( N-K, CT, B( K, K+1 ), LDB, A( K, K+1 ),
     $                        LDA )
                  CALL DTRSV( UPLO, 'Transpose', 'Non-unit', N-K,
     $                        B( K+1, K+1 ), LDB, A( K, K+1 ), LDA )
               END IF
   10       CONTINUE
         ELSE
*
*           Compute inv(L)*A*inv(L**T)
*
            DO 20 K = 1, N
*
*              Update the lower triangle of A(k:n,k:n)
*
               AKK = A( K, K )
               BKK = B( K, K )
               AKK = AKK / BKK**2
               A( K, K ) = AKK
               IF( K.LT.N ) THEN
                  CALL DSCAL( N-K, ONE / BKK, A( K+1, K ), 1 )
                  CT = -HALF*AKK
```

```
                 CALL DAXPY( N-K, CT, B( K+1, K ), 1, A( K+1, K ), 1 )
                 CALL DSYR2( UPLO, N-K, -ONE, A( K+1, K ), 1,
     $                       B( K+1, K ), 1, A( K+1, K+1 ), LDA )
                 CALL DAXPY( N-K, CT, B( K+1, K ), 1, A( K+1, K ), 1 )
                 CALL DTRSV( UPLO, 'No transpose', 'Non-unit', N-K,
     $                       B( K+1, K+1 ), LDB, A( K+1, K ), 1 )
              END IF
   20       CONTINUE
         END IF
      ELSE
         IF( UPPER ) THEN
*
*           Compute U*A*U**T
*
            DO 30 K = 1, N
*
*              Update the upper triangle of A(1:k,1:k)
*
               AKK = A( K, K )
               BKK = B( K, K )
               CALL DTRMV( UPLO, 'No transpose', 'Non-unit', K-1, B,
     $                     LDB, A( 1, K ), 1 )
               CT = HALF*AKK
               CALL DAXPY( K-1, CT, B( 1, K ), 1, A( 1, K ), 1 )
               CALL DSYR2( UPLO, K-1, ONE, A( 1, K ), 1, B( 1, K ), 1,
     $                     A, LDA )
               CALL DAXPY( K-1, CT, B( 1, K ), 1, A( 1, K ), 1 )
               CALL DSCAL( K-1, BKK, A( 1, K ), 1 )
               A( K, K ) = AKK*BKK**2
   30       CONTINUE
         ELSE
*
*           Compute L**T *A*L
*
            DO 40 K = 1, N
*
*              Update the lower triangle of A(1:k,1:k)
*
               AKK = A( K, K )
               BKK = B( K, K )
               CALL DTRMV( UPLO, 'Transpose', 'Non-unit', K-1, B, LDB,
     $                     A( K, 1 ), LDA )
               CT = HALF*AKK
               CALL DAXPY( K-1, CT, B( K, 1 ), LDB, A( K, 1 ), LDA )
               CALL DSYR2( UPLO, K-1, ONE, A( K, 1 ), LDA, B( K, 1 ),
     $                     LDB, A, LDA )
               CALL DAXPY( K-1, CT, B( K, 1 ), LDB, A( K, 1 ), LDA )
               CALL DSCAL( K-1, BKK, A( K, 1 ), LDA )
               A( K, K ) = AKK*BKK**2
   40       CONTINUE
```

```
         END IF
      END IF
      RETURN
*
*     End of DSYGS2
*
      END
```

———-

58.11.4 lisp code

— **dsygs2.lisp** —

```
(let* ((one 1.0d0) (half 0.5d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.5d0 0.5d0) half)
           (ignorable one half))
  (defun dsygs2 (itype uplo n a lda b ldb$ info)
    (declare (type (array double-float (*)) b a)
             (type (string *) uplo)
             (type (f2cl-lib:integer4) info ldb$ lda n itype))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%))
      (prog ((akk 0.0d0) (bkk 0.0d0) (ct 0.0d0) (k 0) (upper nil))
        (declare (type (double-float) akk bkk ct)
                 (type (f2cl-lib:integer4) k)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond ((or (< itype 1) (> itype 3)) (setf info -1))
              ((and (not upper)
                    (not
                     (multiple-value-bind (ret-val var-0 var-1)
                         (lsame uplo "L")
                       (declare (ignore var-1))
                       (when var-0 (setf uplo var-0))
                       ret-val)))
               (setf info -2))
              ((< n 0) (setf info -3))
              ((< lda
```

```
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -5))
       ((< ldb$
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -7)))
 (cond
  ((/= info 0) (xerbla "DSYGS2" (f2cl-lib:int-sub info))
   (go end_label)))
 (cond
  ((= itype 1)
   (cond
    (upper
     (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                   ((> k n) nil)
       (tagbody
         (setf akk
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%))
         (setf bkk
                 (f2cl-lib:fref b-%data% (k k) ((1 ldb$) (1 *))
                                b-%offset%))
         (setf akk (/ akk (expt bkk 2)))
         (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                 akk)
         (cond
          ((< k n)
           (multiple-value-bind (var-0 var-1 var-2 var-3)
               (dscal (f2cl-lib:int-sub n k) (/ one bkk)
                (f2cl-lib:array-slice a-%data% double-float
                                      (k (f2cl-lib:int-add k 1))
                                      ((1 lda) (1 *)) a-%offset%)
                lda)
             (declare (ignore var-0 var-1 var-2))
             (when var-3 (setf lda var-3)))
           (setf ct (* (- half) akk))
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
               (daxpy (f2cl-lib:int-sub n k) ct
                (f2cl-lib:array-slice b-%data% double-float
                                      (k (f2cl-lib:int-add k 1))
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$
                (f2cl-lib:array-slice a-%data% double-float
                                      (k (f2cl-lib:int-add k 1))
                                      ((1 lda) (1 *)) a-%offset%)
                lda)
             (declare (ignore var-0 var-2 var-4))
             (when var-1 (setf ct var-1))
             (when var-3 (setf ldb$ var-3))
             (when var-5 (setf lda var-5)))
           (multiple-value-bind
```

```
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (dsyr2 uplo (f2cl-lib:int-sub n k) (- one)
                 (f2cl-lib:array-slice a-%data% double-float
                                       (k (f2cl-lib:int-add k 1))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice b-%data% double-float
                                       (k (f2cl-lib:int-add k 1))
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$
                 (f2cl-lib:array-slice a-%data% double-float
                                       ((+ k 1) (f2cl-lib:int-add k 1))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare (ignore var-1 var-2 var-3 var-5 var-7))
              (when var-0 (setf uplo var-0))
              (when var-4 (setf lda var-4))
              (when var-6 (setf ldb$ var-6))
              (when var-8 (setf lda var-8)))
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                (daxpy (f2cl-lib:int-sub n k) ct
                 (f2cl-lib:array-slice b-%data% double-float
                                       (k (f2cl-lib:int-add k 1))
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$
                 (f2cl-lib:array-slice a-%data% double-float
                                       (k (f2cl-lib:int-add k 1))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare (ignore var-0 var-2 var-4))
              (when var-1 (setf ct var-1))
              (when var-3 (setf ldb$ var-3))
              (when var-5 (setf lda var-5)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
                (dtrsv uplo "Transpose" "Non-unit" (f2cl-lib:int-sub n k)
                 (f2cl-lib:array-slice b-%data% double-float
                                       ((+ k 1) (f2cl-lib:int-add k 1))
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$
                 (f2cl-lib:array-slice a-%data% double-float
                                       (k (f2cl-lib:int-add k 1))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare (ignore var-1 var-2 var-3 var-4 var-6))
              (when var-0 (setf uplo var-0))
              (when var-5 (setf ldb$ var-5))
              (when var-7 (setf lda var-7)))))
        label10)))
    (t
```

```
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (setf akk
            (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                           a-%offset%))
    (setf bkk
            (f2cl-lib:fref b-%data% (k k) ((1 ldb$) (1 *))
                           b-%offset%))
    (setf akk (/ akk (expt bkk 2)))
    (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
            akk)
    (cond
     ((< k n)
      (dscal (f2cl-lib:int-sub n k) (/ one bkk)
       (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                             ((1 lda) (1 *)) a-%offset%)
       1)
      (setf ct (* (- half) akk))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (daxpy (f2cl-lib:int-sub n k) ct
           (f2cl-lib:array-slice b-%data% double-float ((+ k 1) k)
                                 ((1 ldb$) (1 *)) b-%offset%)
           1
           (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                                 ((1 lda) (1 *)) a-%offset%)
           1)
        (declare (ignore var-0 var-2 var-3 var-4 var-5))
        (when var-1 (setf ct var-1)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dsyr2 uplo (f2cl-lib:int-sub n k) (- one)
           (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice b-%data% double-float ((+ k 1) k)
                                 ((1 ldb$) (1 *)) b-%offset%)
           1
           (f2cl-lib:array-slice a-%data% double-float
                                 ((+ k 1) (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare
         (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7))
        (when var-0 (setf uplo var-0))
        (when var-8 (setf lda var-8)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (daxpy (f2cl-lib:int-sub n k) ct
           (f2cl-lib:array-slice b-%data% double-float ((+ k 1) k)
                                 ((1 ldb$) (1 *)) b-%offset%)
```

```
                1
                (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                                      ((1 lda) (1 *)) a-%offset%)
                1)
            (declare (ignore var-0 var-2 var-3 var-4 var-5))
            (when var-1 (setf ct var-1)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
              (dtrsv uplo "No transpose" "Non-unit"
               (f2cl-lib:int-sub n k)
               (f2cl-lib:array-slice b-%data% double-float
                                     ((+ k 1) (f2cl-lib:int-add k 1))
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                                     ((1 lda) (1 *)) a-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3 var-4 var-6 var-7))
            (when var-0 (setf uplo var-0))
            (when var-5 (setf ldb$ var-5)))))
      label20)))))
(t
 (cond
  (upper
   (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                 ((> k n) nil)
     (tagbody
       (setf akk
               (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                              a-%offset%))
       (setf bkk
               (f2cl-lib:fref b-%data% (k k) ((1 ldb$) (1 *))
                              b-%offset%))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
           (dtrmv uplo "No transpose" "Non-unit"
            (f2cl-lib:int-sub k 1) b ldb$
            (f2cl-lib:array-slice a-%data% double-float (1 k)
                                  ((1 lda) (1 *)) a-%offset%)
            1)
         (declare (ignore var-1 var-2 var-3 var-4 var-6 var-7))
         (when var-0 (setf uplo var-0))
         (when var-5 (setf ldb$ var-5)))
       (setf ct (* half akk))
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
           (daxpy (f2cl-lib:int-sub k 1) ct
            (f2cl-lib:array-slice b-%data% double-float (1 k)
                                  ((1 ldb$) (1 *)) b-%offset%)
            1
            (f2cl-lib:array-slice a-%data% double-float (1 k)
```

```
                                ((1 lda) (1 *)) a-%offset%)
            1)
          (declare (ignore var-0 var-2 var-3 var-4 var-5))
          (when var-1 (setf ct var-1)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (dsyr2 uplo (f2cl-lib:int-sub k 1) one
             (f2cl-lib:array-slice a-%data% double-float (1 k)
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice b-%data% double-float (1 k)
                                   ((1 ldb$) (1 *)) b-%offset%)
             1 a lda)
          (declare (ignore var-1 var-3 var-4 var-5 var-6 var-7))
          (when var-0 (setf uplo var-0))
          (when var-2 (setf one var-2))
          (when var-8 (setf lda var-8)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (daxpy (f2cl-lib:int-sub k 1) ct
             (f2cl-lib:array-slice b-%data% double-float (1 k)
                                   ((1 ldb$) (1 *)) b-%offset%)
             1
             (f2cl-lib:array-slice a-%data% double-float (1 k)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-0 var-2 var-3 var-4 var-5))
          (when var-1 (setf ct var-1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal (f2cl-lib:int-sub k 1) bkk
             (f2cl-lib:array-slice a-%data% double-float (1 k)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-0 var-2 var-3))
          (when var-1 (setf bkk var-1)))
        (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                (* akk (expt bkk 2)))
      label30)))
  (t
   (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                 ((> k n) nil)
     (tagbody
       (setf akk
               (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                              a-%offset%))
       (setf bkk
               (f2cl-lib:fref b-%data% (k k) ((1 ldb$) (1 *))
                              b-%offset%))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
           (dtrmv uplo "Transpose" "Non-unit" (f2cl-lib:int-sub k 1) b
```

```
     ldb$
     (f2cl-lib:array-slice a-%data% double-float (k 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-1 var-2 var-3 var-4 var-6))
  (when var-0 (setf uplo var-0))
  (when var-5 (setf ldb$ var-5))
  (when var-7 (setf lda var-7)))
(setf ct (* half akk))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (daxpy (f2cl-lib:int-sub k 1) ct
     (f2cl-lib:array-slice b-%data% double-float (k 1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice a-%data% double-float (k 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-2 var-4))
  (when var-1 (setf ct var-1))
  (when var-3 (setf ldb$ var-3))
  (when var-5 (setf lda var-5)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dsyr2 uplo (f2cl-lib:int-sub k 1) one
     (f2cl-lib:array-slice a-%data% double-float (k 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% double-float (k 1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ a lda)
  (declare (ignore var-1 var-3 var-5 var-7))
  (when var-0 (setf uplo var-0))
  (when var-2 (setf one var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldb$ var-6))
  (when var-8 (setf lda var-8)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (daxpy (f2cl-lib:int-sub k 1) ct
     (f2cl-lib:array-slice b-%data% double-float (k 1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice a-%data% double-float (k 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-2 var-4))
  (when var-1 (setf ct var-1))
  (when var-3 (setf ldb$ var-3))
  (when var-5 (setf lda var-5)))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (dscal (f2cl-lib:int-sub k 1) bkk
```

```
                    (f2cl-lib:array-slice a-%data% double-float (k 1)
                                          ((1 lda) (1 *)) a-%offset%)
                    lda)
                  (declare (ignore var-0 var-2))
                  (when var-1 (setf bkk var-1))
                  (when var-3 (setf lda var-3)))
                 (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                         (* akk (expt bkk 2)))
               label40))))))
        (go end_label)
       end_label
        (return (values nil uplo nil nil lda nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsygs2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

58.12 dsygst

58.12.1 Axiom unit tests

— dsygst.input —

```
)set break resume
)sys rm -f dsygst.output
)spool dsygst.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.12.2 Axiom help page

— **dsygst.help** —

```
=====================================================================
dsygst examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsygst (ITYPE, UPLO, N, A, LDA, B, LDB, INFO)
           DSYGST

Function/Subroutine Documentation
   subroutine dsygst (integer ITYPE, character UPLO, integer N, double
       precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldb, * ) B, integer LDB, integer INFO)
       DSYGST

Purpose:
   DSYGST reduces a real symmetric-definite generalized eigenproblem
   to standard form.

   If ITYPE = 1, the problem is A*x = lambda*B*x,
   and A is overwritten by inv(U**T)*A*inv(U) or inv(L)*A*inv(L**T)

   If ITYPE = 2 or 3, the problem is A*B*x = lambda*x or
   B*A*x = lambda*x, and A is overwritten by U*A*U**T or L**T*A*L.

   B must have been previously factorized as U**T*U or L*L**T by DPOTRF.

Parameters:

 ITYPE
   ITYPE is INTEGER
```

```
  = 1: compute inv(U**T)*A*inv(U) or inv(L)*A*inv(L**T);
  = 2 or 3: compute U*A*U**T or L**T*A*L.

UPLO
  UPLO is CHARACTER*1
  = 'U':  Upper triangle of A is stored and B is factored as U**T*U;
  = 'L':  Lower triangle of A is stored and B is factored as L*L**T.

N
  N is INTEGER
  The order of the matrices A and B.  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if INFO = 0, the transformed matrix, stored in the
  same format as A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

B
  B is DOUBLE PRECISION array, dimension (LDB,N)
  The triangular factor from the Cholesky factorization of B,
  as returned by DPOTRF.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

58.12.3 fortran code

— dsygst.f —

```
*  =====================================================================
      SUBROUTINE DSYGST( ITYPE, UPLO, N, A, LDA, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, ITYPE, LDA, LDB, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, HALF
      PARAMETER          ( ONE = 1.0D0, HALF = 0.5D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            K, KB, NB
*     ..
*     .. External Subroutines ..
      EXTERNAL           DSYGS2, DSYMM, DSYR2K, DTRMM, DTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( ITYPE.LT.1 .OR. ITYPE.GT.3 ) THEN
         INFO = -1
```

```
      ELSE IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYGST', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Determine the block size for this environment.
*
      NB = ILAENV( 1, 'DSYGST', UPLO, N, -1, -1, -1 )
*
      IF( NB.LE.1 .OR. NB.GE.N ) THEN
*
*        Use unblocked code
*
         CALL DSYGS2( ITYPE, UPLO, N, A, LDA, B, LDB, INFO )
      ELSE
*
*        Use blocked code
*
         IF( ITYPE.EQ.1 ) THEN
            IF( UPPER ) THEN
*
*              Compute inv(U**T)*A*inv(U)
*
               DO 10 K = 1, N, NB
                  KB = MIN( N-K+1, NB )
*
*                 Update the upper triangle of A(k:n,k:n)
*
                  CALL DSYGS2( ITYPE, UPLO, KB, A( K, K ), LDA,
     $                         B( K, K ), LDB, INFO )
                  IF( K+KB.LE.N ) THEN
                     CALL DTRSM( 'Left', UPLO, 'Transpose', 'Non-unit',
     $                           KB, N-K-KB+1, ONE, B( K, K ), LDB,
     $                           A( K, K+KB ), LDA )
                     CALL DSYMM( 'Left', UPLO, KB, N-K-KB+1, -HALF,
     $                           A( K, K ), LDA, B( K, K+KB ), LDB, ONE,
```

```
     $                          A( K, K+KB ), LDA )
                     CALL DSYR2K( UPLO, 'Transpose', N-K-KB+1, KB, -ONE,
     $                           A( K, K+KB ), LDA, B( K, K+KB ), LDB,
     $                           ONE, A( K+KB, K+KB ), LDA )
                     CALL DSYMM( 'Left', UPLO, KB, N-K-KB+1, -HALF,
     $                          A( K, K ), LDA, B( K, K+KB ), LDB, ONE,
     $                          A( K, K+KB ), LDA )
                     CALL DTRSM( 'Right', UPLO, 'No transpose',
     $                          'Non-unit', KB, N-K-KB+1, ONE,
     $                          B( K+KB, K+KB ), LDB, A( K, K+KB ),
     $                          LDA )
                  END IF
   10          CONTINUE
            ELSE
*
*             Compute inv(L)*A*inv(L**T)
*
              DO 20 K = 1, N, NB
                 KB = MIN( N-K+1, NB )
*
*                Update the lower triangle of A(k:n,k:n)
*
                 CALL DSYGS2( ITYPE, UPLO, KB, A( K, K ), LDA,
     $                        B( K, K ), LDB, INFO )
                 IF( K+KB.LE.N ) THEN
                    CALL DTRSM( 'Right', UPLO, 'Transpose', 'Non-unit',
     $                          N-K-KB+1, KB, ONE, B( K, K ), LDB,
     $                          A( K+KB, K ), LDA )
                    CALL DSYMM( 'Right', UPLO, N-K-KB+1, KB, -HALF,
     $                          A( K, K ), LDA, B( K+KB, K ), LDB, ONE,
     $                          A( K+KB, K ), LDA )
                    CALL DSYR2K( UPLO, 'No transpose', N-K-KB+1, KB,
     $                           -ONE, A( K+KB, K ), LDA, B( K+KB, K ),
     $                           LDB, ONE, A( K+KB, K+KB ), LDA )
                    CALL DSYMM( 'Right', UPLO, N-K-KB+1, KB, -HALF,
     $                          A( K, K ), LDA, B( K+KB, K ), LDB, ONE,
     $                          A( K+KB, K ), LDA )
                    CALL DTRSM( 'Left', UPLO, 'No transpose',
     $                          'Non-unit', N-K-KB+1, KB, ONE,
     $                          B( K+KB, K+KB ), LDB, A( K+KB, K ),
     $                          LDA )
                 END IF
   20         CONTINUE
            END IF
         ELSE
            IF( UPPER ) THEN
*
*             Compute U*A*U**T
*
              DO 30 K = 1, N, NB
```

```
                 KB = MIN( N-K+1, NB )
*
*                Update the upper triangle of A(1:k+kb-1,1:k+kb-1)
*
                 CALL DTRMM( 'Left', UPLO, 'No transpose', 'Non-unit',
     $                       K-1, KB, ONE, B, LDB, A( 1, K ), LDA )
                 CALL DSYMM( 'Right', UPLO, K-1, KB, HALF, A( K, K ),
     $                       LDA, B( 1, K ), LDB, ONE, A( 1, K ), LDA )
                 CALL DSYR2K( UPLO, 'No transpose', K-1, KB, ONE,
     $                        A( 1, K ), LDA, B( 1, K ), LDB, ONE, A,
     $                        LDA )
                 CALL DSYMM( 'Right', UPLO, K-1, KB, HALF, A( K, K ),
     $                       LDA, B( 1, K ), LDB, ONE, A( 1, K ), LDA )
                 CALL DTRMM( 'Right', UPLO, 'Transpose', 'Non-unit',
     $                       K-1, KB, ONE, B( K, K ), LDB, A( 1, K ),
     $                       LDA )
                 CALL DSYGS2( ITYPE, UPLO, KB, A( K, K ), LDA,
     $                        B( K, K ), LDB, INFO )
   30         CONTINUE
           ELSE
*
*             Compute L**T*A*L
*
              DO 40 K = 1, N, NB
                 KB = MIN( N-K+1, NB )
*
*                Update the lower triangle of A(1:k+kb-1,1:k+kb-1)
*
                 CALL DTRMM( 'Right', UPLO, 'No transpose', 'Non-unit',
     $                       KB, K-1, ONE, B, LDB, A( K, 1 ), LDA )
                 CALL DSYMM( 'Left', UPLO, KB, K-1, HALF, A( K, K ),
     $                       LDA, B( K, 1 ), LDB, ONE, A( K, 1 ), LDA )
                 CALL DSYR2K( UPLO, 'Transpose', K-1, KB, ONE,
     $                        A( K, 1 ), LDA, B( K, 1 ), LDB, ONE, A,
     $                        LDA )
                 CALL DSYMM( 'Left', UPLO, KB, K-1, HALF, A( K, K ),
     $                       LDA, B( K, 1 ), LDB, ONE, A( K, 1 ), LDA )
                 CALL DTRMM( 'Left', UPLO, 'Transpose', 'Non-unit', KB,
     $                       K-1, ONE, B( K, K ), LDB, A( K, 1 ), LDA )
                 CALL DSYGS2( ITYPE, UPLO, KB, A( K, K ), LDA,
     $                        B( K, K ), LDB, INFO )
   40         CONTINUE
           END IF
        END IF
      END IF
      RETURN
*
*     End of DSYGST
*
      END
```

———-

58.12.4 lisp code

— dsygst.lisp —

```
(let* ((one 1.0d0) (half 0.5d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.5d0 0.5d0) half)
           (ignorable one half))
  (defun dsygst (itype uplo n a lda b ldb$ info)
    (declare (type (array double-float (*)) b a)
             (type (string *) uplo)
             (type (f2cl-lib:integer4) info ldb$ lda n itype))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%))
      (prog ((k 0) (kb 0) (nb 0) (upper nil))
        (declare (type (f2cl-lib:integer4) k kb nb)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond ((or (< itype 1) (> itype 3)) (setf info -1))
              ((and (not upper)
                    (not
                     (multiple-value-bind (ret-val var-0 var-1)
                         (lsame uplo "L")
                       (declare (ignore var-1))
                       (when var-0 (setf uplo var-0))
                       ret-val)))
               (setf info -2))
              ((< n 0) (setf info -3))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -5))
              ((< ldb$
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -7)))
        (cond
         ((/= info 0) (xerbla "DSYGST" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (= n 0)
```

```
    (go end_label))
(setf nb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 1 "DSYGST" uplo n -1 -1 -1)
          (declare (ignore var-0 var-1 var-4 var-5 var-6))
          (when var-2 (setf uplo var-2))
          (when var-3 (setf n var-3))
          ret-val))
(cond
 ((or (<= nb 1) (>= nb n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dsygs2 itype uplo n a lda b ldb$ info)
    (declare (ignore var-0 var-2 var-3 var-5))
    (setf uplo var-1)
    (setf lda var-4)
    (setf ldb$ var-6)
    (setf info var-7)))
 (t
  (cond
   ((= itype 1)
    (cond
     (upper
      (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k nb))
                    ((> k n) nil)
        (tagbody
          (setf kb
                  (min
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1))
                   (the f2cl-lib:integer4 nb)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
              (dsygs2 itype uplo kb
               (f2cl-lib:array-slice a-%data% double-float (k k)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice b-%data% double-float (k k)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$ info)
            (declare (ignore var-0 var-2 var-3 var-5))
            (setf uplo var-1)
            (setf lda var-4)
            (setf ldb$ var-6)
            (setf info var-7))
          (cond
           ((<= (f2cl-lib:int-add k kb) n)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
     var-9 var-10)
    (dtrsm "Left" uplo "Transpose" "Non-unit" kb
     (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1) one
     (f2cl-lib:array-slice b-%data% double-float (k k)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice a-%data% double-float
                           (k (f2cl-lib:int-add k kb))
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-2 var-3 var-5 var-7 var-9))
  (when var-1 (setf uplo var-1))
  (when var-4 (setf kb var-4))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-10 (setf lda var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11)
    (dsymm "Left" uplo kb
     (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1)
     (- half)
     (f2cl-lib:array-slice a-%data% double-float (k k)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% double-float
                           (k (f2cl-lib:int-add k kb))
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ one
     (f2cl-lib:array-slice a-%data% double-float
                           (k (f2cl-lib:int-add k kb))
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-3 var-4 var-5 var-7 var-10))
  (when var-1 (setf uplo var-1))
  (when var-2 (setf kb var-2))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf lda var-11)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11)
    (dsyr2k uplo "Transpose"
     (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1) kb
     (- one)
     (f2cl-lib:array-slice a-%data% double-float
                           (k (f2cl-lib:int-add k kb))
                           ((1 lda) (1 *)) a-%offset%)
     lda
```

```
      (f2cl-lib:array-slice b-%data% double-float
                            (k (f2cl-lib:int-add k kb))
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$ one
      (f2cl-lib:array-slice a-%data% double-float
                            ((+ k kb)
                             (f2cl-lib:int-add k kb))
                            ((1 lda) (1 *)) a-%offset%)
      lda)
  (declare (ignore var-1 var-2 var-4 var-5 var-7 var-10))
  (when var-0 (setf uplo var-0))
  (when var-3 (setf kb var-3))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf lda var-11)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11)
    (dsymm "Left" uplo kb
     (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1)
     (- half)
     (f2cl-lib:array-slice a-%data% double-float (k k)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% double-float
                           (k (f2cl-lib:int-add k kb))
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ one
     (f2cl-lib:array-slice a-%data% double-float
                           (k (f2cl-lib:int-add k kb))
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-3 var-4 var-5 var-7 var-10))
  (when var-1 (setf uplo var-1))
  (when var-2 (setf kb var-2))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf lda var-11)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dtrsm "Right" uplo "No transpose" "Non-unit" kb
     (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1) one
     (f2cl-lib:array-slice b-%data% double-float
                           ((+ k kb)
                            (f2cl-lib:int-add k kb))
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
```

```
                (f2cl-lib:array-slice a-%data% double-float
                                      (k (f2cl-lib:int-add k kb))
                                      ((1 lda) (1 *)) a-%offset%)
                lda)
            (declare (ignore var-0 var-2 var-3 var-5 var-7 var-9))
            (when var-1 (setf uplo var-1))
            (when var-4 (setf kb var-4))
            (when var-6 (setf one var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-10 (setf lda var-10)))))
     label10)))
  (t
   (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k nb))
                 ((> k n) nil)
     (tagbody
       (setf kb
               (min
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1))
                (the f2cl-lib:integer4 nb)))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
           (dsygs2 itype uplo kb
            (f2cl-lib:array-slice a-%data% double-float (k k)
                                  ((1 lda) (1 *)) a-%offset%)
            lda
            (f2cl-lib:array-slice b-%data% double-float (k k)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$ info)
         (declare (ignore var-0 var-2 var-3 var-5))
         (setf uplo var-1)
         (setf lda var-4)
         (setf ldb$ var-6)
         (setf info var-7))
       (cond
        ((<= (f2cl-lib:int-add k kb) n)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9 var-10)
             (dtrsm "Right" uplo "Transpose" "Non-unit"
              (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1) kb one
              (f2cl-lib:array-slice b-%data% double-float (k k)
                                    ((1 ldb$) (1 *)) b-%offset%)
              ldb$
              (f2cl-lib:array-slice a-%data% double-float
                                    ((+ k kb) k) ((1 lda) (1 *))
                                    a-%offset%)
              lda)
           (declare (ignore var-0 var-2 var-3 var-4 var-7 var-9))
           (when var-1 (setf uplo var-1))
```

```
  (when var-5 (setf kb var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-10 (setf lda var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11)
    (dsymm "Right" uplo
     (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1) kb
     (- half)
     (f2cl-lib:array-slice a-%data% double-float (k k)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% double-float
                           ((+ k kb) k) ((1 ldb$) (1 *))
                           b-%offset%)
     ldb$ one
     (f2cl-lib:array-slice a-%data% double-float
                           ((+ k kb) k) ((1 lda) (1 *))
                           a-%offset%)
     lda)
  (declare (ignore var-0 var-2 var-4 var-5 var-7 var-10))
  (when var-1 (setf uplo var-1))
  (when var-3 (setf kb var-3))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf lda var-11)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11)
    (dsyr2k uplo "No transpose"
     (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1) kb
     (- one)
     (f2cl-lib:array-slice a-%data% double-float
                           ((+ k kb) k) ((1 lda) (1 *))
                           a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% double-float
                           ((+ k kb) k) ((1 ldb$) (1 *))
                           b-%offset%)
     ldb$ one
     (f2cl-lib:array-slice a-%data% double-float
                           ((+ k kb)
                            (f2cl-lib:int-add k kb))
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-1 var-2 var-4 var-5 var-7 var-10))
  (when var-0 (setf uplo var-0))
  (when var-3 (setf kb var-3))
```

```
                (when var-6 (setf lda var-6))
                (when var-8 (setf ldb$ var-8))
                (when var-9 (setf one var-9))
                (when var-11 (setf lda var-11)))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                   var-9 var-10 var-11)
                  (dsymm "Right" uplo
                   (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1) kb
                   (- half)
                   (f2cl-lib:array-slice a-%data% double-float (k k)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda
                   (f2cl-lib:array-slice b-%data% double-float
                                         ((+ k kb) k) ((1 ldb$) (1 *))
                                         b-%offset%)
                   ldb$ one
                   (f2cl-lib:array-slice a-%data% double-float
                                         ((+ k kb) k) ((1 lda) (1 *))
                                         a-%offset%)
                   lda)
                (declare (ignore var-0 var-2 var-4 var-5 var-7 var-10))
                (when var-1 (setf uplo var-1))
                (when var-3 (setf kb var-3))
                (when var-6 (setf lda var-6))
                (when var-8 (setf ldb$ var-8))
                (when var-9 (setf one var-9))
                (when var-11 (setf lda var-11)))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                   var-9 var-10)
                  (dtrsm "Left" uplo "No transpose" "Non-unit"
                   (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1) kb one
                   (f2cl-lib:array-slice b-%data% double-float
                                         ((+ k kb)
                                          (f2cl-lib:int-add k kb))
                                         ((1 ldb$) (1 *)) b-%offset%)
                   ldb$
                   (f2cl-lib:array-slice a-%data% double-float
                                         ((+ k kb) k) ((1 lda) (1 *))
                                         a-%offset%)
                   lda)
                (declare (ignore var-0 var-2 var-3 var-4 var-7 var-9))
                (when var-1 (setf uplo var-1))
                (when var-5 (setf kb var-5))
                (when var-6 (setf one var-6))
                (when var-8 (setf ldb$ var-8))
                (when var-10 (setf lda var-10)))))
        label20)))))
(t
```

```
(cond
 (upper
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k nb))
                ((> k n) nil)
    (tagbody
      (setf kb
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1))
               (the f2cl-lib:integer4 nb)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (dtrmm "Left" uplo "No transpose" "Non-unit"
           (f2cl-lib:int-sub k 1) kb one b ldb$
           (f2cl-lib:array-slice a-%data% double-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-2 var-3 var-4 var-7 var-9))
        (when var-1 (setf uplo var-1))
        (when var-5 (setf kb var-5))
        (when var-6 (setf one var-6))
        (when var-8 (setf ldb$ var-8))
        (when var-10 (setf lda var-10)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11)
          (dsymm "Right" uplo (f2cl-lib:int-sub k 1) kb half
           (f2cl-lib:array-slice a-%data% double-float (k k)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice b-%data% double-float (1 k)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$ one
           (f2cl-lib:array-slice a-%data% double-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-2 var-5 var-7 var-10))
        (when var-1 (setf uplo var-1))
        (when var-3 (setf kb var-3))
        (when var-4 (setf half var-4))
        (when var-6 (setf lda var-6))
        (when var-8 (setf ldb$ var-8))
        (when var-9 (setf one var-9))
        (when var-11 (setf lda var-11)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11)
          (dsyr2k uplo "No transpose" (f2cl-lib:int-sub k 1) kb one
           (f2cl-lib:array-slice a-%data% double-float (1 k)
```

```
                                ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice b-%data% double-float (1 k)
                                ((1 ldb$) (1 *)) b-%offset%)
                ldb$ one a lda)
            (declare (ignore var-1 var-2 var-5 var-7 var-10))
            (when var-0 (setf uplo var-0))
            (when var-3 (setf kb var-3))
            (when var-4 (setf one var-4))
            (when var-6 (setf lda var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf one var-9))
            (when var-11 (setf lda var-11)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10 var-11)
              (dsymm "Right" uplo (f2cl-lib:int-sub k 1) kb half
               (f2cl-lib:array-slice a-%data% double-float (k k)
                                ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice b-%data% double-float (1 k)
                                ((1 ldb$) (1 *)) b-%offset%)
               ldb$ one
               (f2cl-lib:array-slice a-%data% double-float (1 k)
                                ((1 lda) (1 *)) a-%offset%)
               lda)
            (declare (ignore var-0 var-2 var-5 var-7 var-10))
            (when var-1 (setf uplo var-1))
            (when var-3 (setf kb var-3))
            (when var-4 (setf half var-4))
            (when var-6 (setf lda var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf one var-9))
            (when var-11 (setf lda var-11)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10)
              (dtrmm "Right" uplo "Transpose" "Non-unit"
               (f2cl-lib:int-sub k 1) kb one
               (f2cl-lib:array-slice b-%data% double-float (k k)
                                ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice a-%data% double-float (1 k)
                                ((1 lda) (1 *)) a-%offset%)
               lda)
            (declare (ignore var-0 var-2 var-3 var-4 var-7 var-9))
            (when var-1 (setf uplo var-1))
            (when var-5 (setf kb var-5))
            (when var-6 (setf one var-6))
            (when var-8 (setf ldb$ var-8))
```

```
          (when var-10 (setf lda var-10)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (dsygs2 itype uplo kb
             (f2cl-lib:array-slice a-%data% double-float (k k)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice b-%data% double-float (k k)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$ info)
          (declare (ignore var-0 var-2 var-3 var-5))
          (setf uplo var-1)
          (setf lda var-4)
          (setf ldb$ var-6)
          (setf info var-7))
      label30)))
  (t
   (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k nb))
                 ((> k n) nil)
     (tagbody
       (setf kb
               (min
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1))
                (the f2cl-lib:integer4 nb)))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
            var-9 var-10)
           (dtrmm "Right" uplo "No transpose" "Non-unit" kb
            (f2cl-lib:int-sub k 1) one b ldb$
            (f2cl-lib:array-slice a-%data% double-float (k 1)
                                  ((1 lda) (1 *)) a-%offset%)
            lda)
         (declare (ignore var-0 var-2 var-3 var-5 var-7 var-9))
         (when var-1 (setf uplo var-1))
         (when var-4 (setf kb var-4))
         (when var-6 (setf one var-6))
         (when var-8 (setf ldb$ var-8))
         (when var-10 (setf lda var-10)))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
            var-9 var-10 var-11)
           (dsymm "Left" uplo kb (f2cl-lib:int-sub k 1) half
            (f2cl-lib:array-slice a-%data% double-float (k k)
                                  ((1 lda) (1 *)) a-%offset%)
            lda
            (f2cl-lib:array-slice b-%data% double-float (k 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$ one
            (f2cl-lib:array-slice a-%data% double-float (k 1)
```

```
                          ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-3 var-5 var-7 var-10))
  (when var-1 (setf uplo var-1))
  (when var-2 (setf kb var-2))
  (when var-4 (setf half var-4))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf lda var-11)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11)
    (dsyr2k uplo "Transpose" (f2cl-lib:int-sub k 1) kb one
     (f2cl-lib:array-slice a-%data% double-float (k 1)
                          ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% double-float (k 1)
                          ((1 ldb$) (1 *)) b-%offset%)
     ldb$ one a lda)
  (declare (ignore var-1 var-2 var-5 var-7 var-10))
  (when var-0 (setf uplo var-0))
  (when var-3 (setf kb var-3))
  (when var-4 (setf one var-4))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf lda var-11)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11)
    (dsymm "Left" uplo kb (f2cl-lib:int-sub k 1) half
     (f2cl-lib:array-slice a-%data% double-float (k k)
                          ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% double-float (k 1)
                          ((1 ldb$) (1 *)) b-%offset%)
     ldb$ one
     (f2cl-lib:array-slice a-%data% double-float (k 1)
                          ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-3 var-5 var-7 var-10))
  (when var-1 (setf uplo var-1))
  (when var-2 (setf kb var-2))
  (when var-4 (setf half var-4))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf lda var-11)))
(multiple-value-bind
```

```
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                     var-9 var-10)
                    (dtrmm "Left" uplo "Transpose" "Non-unit" kb
                     (f2cl-lib:int-sub k 1) one
                     (f2cl-lib:array-slice b-%data% double-float (k k)
                                           ((1 ldb$) (1 *)) b-%offset%)
                     ldb$
                     (f2cl-lib:array-slice a-%data% double-float (k 1)
                                           ((1 lda) (1 *)) a-%offset%)
                     lda)
                  (declare (ignore var-0 var-2 var-3 var-5 var-7 var-9))
                  (when var-1 (setf uplo var-1))
                  (when var-4 (setf kb var-4))
                  (when var-6 (setf one var-6))
                  (when var-8 (setf ldb$ var-8))
                  (when var-10 (setf lda var-10)))
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
                    (dsygs2 itype uplo kb
                     (f2cl-lib:array-slice a-%data% double-float (k k)
                                           ((1 lda) (1 *)) a-%offset%)
                     lda
                     (f2cl-lib:array-slice b-%data% double-float (k k)
                                           ((1 ldb$) (1 *)) b-%offset%)
                     ldb$ info)
                  (declare (ignore var-0 var-2 var-3 var-5))
                  (setf uplo var-1)
                  (setf lda var-4)
                  (setf ldb$ var-6)
                  (setf info var-7))
               label40))))))))
        (go end_label)
       end_label
        (return (values nil uplo n nil lda nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsygst
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
```

```
                                        '(nil fortran-to-lisp::uplo
                                          fortran-to-lisp::n nil
                                          fortran-to-lisp::lda nil
                                          fortran-to-lisp::ldb$
                                          fortran-to-lisp::info)
                                        :calls '(fortran-to-lisp::dsygs2))))
```

———-

58.13 dsyrfs

58.13.1 Axiom unit tests

— **dsyrfs.input** —

```
)set break resume
)sys rm -f dsyrfs.output
)spool dsyrfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.13.2 Axiom help page

— **dsyrfs.help** —

```
====================================================================
dsyrfs examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsyrfs (UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB, X,
```

```
          LDX, FERR, BERR, WORK, IWORK, INFO)
          DSYRFS

Function/Subroutine Documentation
   subroutine dsyrfs (character UPLO, integer N, integer NRHS, double
       precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV,
       double precision, dimension( ldb, * ) B, integer LDB, double precision,
       dimension( ldx, * ) X, integer LDX, double precision, dimension( * )
       FERR, double precision, dimension( * ) BERR, double precision,
       dimension( * ) WORK, integer, dimension( * ) IWORK, integer INFO)
       DSYRFS

Purpose:
   DSYRFS improves the computed solution to a system of linear
   equations when the coefficient matrix is symmetric indefinite, and
   provides error bounds and backward error estimates for the solution.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
   upper triangular part of A contains the upper triangular part
   of the matrix A, and the strictly lower triangular part of A
   is not referenced.  If UPLO = 'L', the leading N-by-N lower
   triangular part of A contains the lower triangular part of
   the matrix A, and the strictly upper triangular part of A is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is DOUBLE PRECISION array, dimension (LDAF,N)
```

```
  The factored form of the matrix A.  AF contains the block
  diagonal matrix D and the multipliers used to obtain the
  factor U or L from the factorization A = U*D*U**T or
  A = L*D*L**T as computed by DSYTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by DSYTRF.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by DSYTRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

FERR
  FERR is DOUBLE PRECISION array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is DOUBLE PRECISION array, dimension (3*N)
```

```
 IWORK
   IWORK is INTEGER array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

 ITMAX is the maximum number of steps of iterative refinement.
```

———-

58.13.3 fortran code

— **dsyrfs.f** —

```
*  =====================================================================
      SUBROUTINE DSYRFS( UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB,
     $                   X, LDX, FERR, BERR, WORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   BERR( * ), FERR( * ), WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
```

```
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D+0 )
      DOUBLE PRECISION   THREE
      PARAMETER          ( THREE = 3.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            COUNT, I, J, K, KASE, NZ
      DOUBLE PRECISION   EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN, XK
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DCOPY, DLACN2, DSYMV, DSYTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -10
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -12
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYRFS', -INFO )
         RETURN
      END IF
*
```

```
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = N + 1
      EPS = DLAMCH( 'Epsilon' )
      SAFMIN = DLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 140 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - A * X
*
         CALL DCOPY( N, B( 1, J ), 1, WORK( N+1 ), 1 )
         CALL DSYMV( UPLO, N, -ONE, A, LDA, X( 1, J ), 1, ONE,
     $               WORK( N+1 ), 1 )
*
*        Compute componentwise relative backward error from formula
*
*        max(i) ( abs(R(i)) / ( abs(A)*abs(X) + abs(B) )(i) )
*
*        where abs(Z) is the componentwise absolute value of the matrix
*        or vector Z.  If the i-th component of the denominator is less
*        than SAFE2, then SAFE1 is added to the i-th components of the
*        numerator and denominator before dividing.
*
         DO 30 I = 1, N
            WORK( I ) = ABS( B( I, J ) )
   30    CONTINUE
*
*        Compute abs(A)*abs(X) + abs(B).
*
         IF( UPPER ) THEN
```

```
         DO 50 K = 1, N
            S = ZERO
            XK = ABS( X( K, J ) )
            DO 40 I = 1, K - 1
               WORK( I ) = WORK( I ) + ABS( A( I, K ) )*XK
               S = S + ABS( A( I, K ) )*ABS( X( I, J ) )
 40         CONTINUE
            WORK( K ) = WORK( K ) + ABS( A( K, K ) )*XK + S
 50      CONTINUE
      ELSE
         DO 70 K = 1, N
            S = ZERO
            XK = ABS( X( K, J ) )
            WORK( K ) = WORK( K ) + ABS( A( K, K ) )*XK
            DO 60 I = K + 1, N
               WORK( I ) = WORK( I ) + ABS( A( I, K ) )*XK
               S = S + ABS( A( I, K ) )*ABS( X( I, J ) )
 60         CONTINUE
            WORK( K ) = WORK( K ) + S
 70      CONTINUE
      END IF
      S = ZERO
      DO 80 I = 1, N
         IF( WORK( I ).GT.SAFE2 ) THEN
            S = MAX( S, ABS( WORK( N+I ) ) / WORK( I ) )
         ELSE
            S = MAX( S, ( ABS( WORK( N+I ) )+SAFE1 ) /
     $           ( WORK( I )+SAFE1 ) )
         END IF
 80   CONTINUE
      BERR( J ) = S
*
*     Test stopping criterion. Continue iterating if
*        1) The residual BERR(J) is larger than machine epsilon, and
*        2) BERR(J) decreased by at least a factor of 2 during the
*           last iteration, and
*        3) At most ITMAX iterations tried.
*
      IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $     COUNT.LE.ITMAX ) THEN
*
*        Update solution and try again.
*
         CALL DSYTRS( UPLO, N, 1, AF, LDAF, IPIV, WORK( N+1 ), N,
     $                INFO )
         CALL DAXPY( N, ONE, WORK( N+1 ), 1, X( 1, J ), 1 )
         LSTRES = BERR( J )
         COUNT = COUNT + 1
         GO TO 20
      END IF
```

```
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(A))*
*           ( abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
*          inv(A) is the inverse of A
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(A)*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(A)*abs(X) + abs(B) is less than SAFE2.
*
*        Use DLACN2 to estimate the infinity-norm of the matrix
*           inv(A) * diag(W),
*        where W = abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) )))
*
         DO 90 I = 1, N
            IF( WORK( I ).GT.SAFE2 ) THEN
               WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I )
            ELSE
               WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I ) + SAFE1
            END IF
   90    CONTINUE
*
         KASE = 0
  100    CONTINUE
         CALL DLACN2( N, WORK( 2*N+1 ), WORK( N+1 ), IWORK, FERR( J ),
     $                KASE, ISAVE )
         IF( KASE.NE.0 ) THEN
            IF( KASE.EQ.1 ) THEN
*
*              Multiply by diag(W)*inv(A**T).
*
               CALL DSYTRS( UPLO, N, 1, AF, LDAF, IPIV, WORK( N+1 ), N,
     $                      INFO )
               DO 110 I = 1, N
                  WORK( N+I ) = WORK( I )*WORK( N+I )
  110          CONTINUE
            ELSE IF( KASE.EQ.2 ) THEN
*
*              Multiply by inv(A)*diag(W).
*
               DO 120 I = 1, N
```

```
                 WORK( N+I ) = WORK( I )*WORK( N+I )
  120          CONTINUE
               CALL DSYTRS( UPLO, N, 1, AF, LDAF, IPIV, WORK( N+1 ), N,
     $                      INFO )
            END IF
            GO TO 100
         END IF
*
*        Normalize error.
*
         LSTRES = ZERO
         DO 130 I = 1, N
            LSTRES = MAX( LSTRES, ABS( X( I, J ) ) )
  130    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  140 CONTINUE
*
      RETURN
*
*     End of DSYRFS
*
      END
```

———-

58.13.4 lisp code

— dsyrfs.lisp —

```
(let* ((itmax 5) (zero 0.0d0) (one 1.0d0) (two 2.0d0) (three 3.0d0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 3.0d0 3.0d0) three)
           (ignorable itmax zero one two three))
  (defun dsyrfs
         (uplo n nrhs a lda af ldaf ipiv b ldb$ x ldx ferr berr work iwork
          info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array double-float (*)) work berr ferr x b af a)
             (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
```

```
    (af double-float af-%data% af-%offset%)
    (b double-float b-%data% b-%offset%)
    (x double-float x-%data% x-%offset%)
    (ferr double-float ferr-%data% ferr-%offset%)
    (berr double-float berr-%data% berr-%offset%)
    (work double-float work-%data% work-%offset%)
    (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
    (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
         (eps 0.0d0) (lstres 0.0d0) (s 0.0d0) (safe1 0.0d0) (safe2 0.0d0)
         (safmin 0.0d0) (xk 0.0d0) (count$ 0) (i 0) (j 0) (k 0) (kase 0)
         (nz 0) (upper nil))
    (declare (type (array f2cl-lib:integer4 (3)) isave)
             (type (double-float) eps lstres s safe1 safe2 safmin xk)
             (type (f2cl-lib:integer4) count$ i j k kase nz)
             (type f2cl-lib:logical upper))
    (setf info 0)
    (setf upper
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
    (cond
     ((and (not upper)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
      (setf info -1))
     ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
     ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -5))
     ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -7))
     ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -10))
     ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -12)))
    (cond
     ((/= info 0) (xerbla "DSYRFS" (f2cl-lib:int-sub info))
      (go end_label)))
    (cond
     ((or (= n 0) (= nrhs 0))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nrhs) nil)
        (tagbody
          (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%) zero)
```

```
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) zero)
     label10))
  (go end_label)))
(setf nz (f2cl-lib:int-add n 1))
(setf eps (dlamch "Epsilon"))
(setf safmin (dlamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy n
         (f2cl-lib:array-slice b-%data% double-float (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1
         (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                               ((1 *)) work-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dsymv uplo n (- one) a lda
         (f2cl-lib:array-slice x-%data% double-float (1 j)
                               ((1 ldx) (1 *)) x-%offset%)
         1 one
         (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                               ((1 *)) work-%offset%)
         1)
      (declare (ignore var-2 var-3 var-5 var-6 var-8 var-9))
      (when var-0 (setf uplo var-0))
      (when var-1 (setf n var-1))
      (when var-4 (setf lda var-4))
      (when var-7 (setf one var-7)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (abs
                 (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)))
       label30))
    (cond
     (upper
      (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                    ((> k n) nil)
```

```
    (tagbody
      (setf s zero)
      (setf xk
              (abs
               (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                              x-%offset%)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                  (+
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                   (*
                    (abs
                     (f2cl-lib:fref a-%data% (i k) ((1 lda) (1 *))
                                    a-%offset%))
                    xk)))
          (setf s
                  (+ s
                     (*
                      (abs
                       (f2cl-lib:fref a-%data% (i k)
                                      ((1 lda) (1 *)) a-%offset%))
                      (abs
                       (f2cl-lib:fref x-%data% (i j)
                                      ((1 ldx) (1 *))
                                      x-%offset%)))))
        label40))
      (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
              (+
               (f2cl-lib:fref work-%data% (k) ((1 *))
                              work-%offset%)
               (*
                (abs
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%))
                xk)
               s))
    label50)))
(t
 (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
               ((> k n) nil)
   (tagbody
     (setf s zero)
     (setf xk
             (abs
```

```
                 (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                                x-%offset%)))
        (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                (+
                 (f2cl-lib:fref work-%data% (k) ((1 *))
                                work-%offset%)
                 (*
                  (abs
                   (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                  a-%offset%))
                  xk)))
        (f2cl-lib:fdo (i (f2cl-lib:int-add k 1)
                       (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)
                    (+
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)
                     (*
                      (abs
                       (f2cl-lib:fref a-%data% (i k) ((1 lda) (1 *))
                                      a-%offset%))
                      xk)))
            (setf s
                    (+ s
                       (*
                        (abs
                         (f2cl-lib:fref a-%data% (i k)
                                        ((1 lda) (1 *)) a-%offset%))
                        (abs
                         (f2cl-lib:fref x-%data% (i j)
                                        ((1 ldx) (1 *))
                                        x-%offset%)))))
           label60))
        (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                (+
                 (f2cl-lib:fref work-%data% (k) ((1 *))
                                work-%offset%)
                 s))
      label70))))
  (setf s zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
        (setf s
                (max s
```

```
                    (/
                     (abs
                      (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-add n i))
                                     ((1 *)) work-%offset%))
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)))))
     (t
      (setf s
              (max s
                   (/
                    (+
                     (abs
                      (f2cl-lib:fref work-%data%
                                      ((f2cl-lib:int-add n i))
                                      ((1 *)) work-%offset%))
                     safe1)
                    (+
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)
                     safe1))))))
   label80))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dsytrs uplo n 1 af ldaf ipiv
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 *)) work-%offset%)
       n info)
    (declare (ignore var-2 var-3 var-5 var-6))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf ldaf var-4))
    (when var-7 (setf n var-7))
    (when var-8 (setf info var-8)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (daxpy n one
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 *)) work-%offset%)
       1
       (f2cl-lib:array-slice x-%data% double-float (1 j)
                             ((1 ldx) (1 *)) x-%offset%)
       1)
    (declare (ignore var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))
    (when var-1 (setf one var-1)))
```

```
    (setf lstres
            (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
    (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (+
                 (abs
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                 ((1 *)) work-%offset%))
                 (* nz eps
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)))))
       (t
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (+
                 (abs
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                 ((1 *)) work-%offset%))
                 (* nz eps
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%))
                 safe1))))
     label90))
  (setf kase 0)
 label100
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacn2 n
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 *)) work-%offset%)
       iwork
       (f2cl-lib:array-slice ferr-%data% double-float (j) ((1 *))
                             ferr-%offset%)
       kase isave)
    (declare (ignore var-1 var-2 var-3 var-4 var-6))
    (when var-0 (setf n var-0))
    (when var-5 (setf kase var-5)))
  (cond
   ((/= kase 0)
    (cond
     ((= kase 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dsytrs uplo n 1 af ldaf ipiv
```

```
          (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                                ((1 *)) work-%offset%)
          n info)
        (declare (ignore var-2 var-3 var-5 var-6))
        (when var-0 (setf uplo var-0))
        (when var-1 (setf n var-1))
        (when var-4 (setf ldaf var-4))
        (when var-7 (setf n var-7))
        (when var-8 (setf info var-8)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%)
                  (*
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add n i)) ((1 *))
                                  work-%offset%)))
         label110)))
     ((= kase 2)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%)
                  (*
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add n i)) ((1 *))
                                  work-%offset%)))
         label120))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dsytrs uplo n 1 af ldaf ipiv
           (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                                 ((1 *)) work-%offset%)
           n info)
        (declare (ignore var-2 var-3 var-5 var-6))
        (when var-0 (setf uplo var-0))
        (when var-1 (setf n var-1))
        (when var-4 (setf ldaf var-4))
        (when var-7 (setf n var-7))
        (when var-8 (setf info var-8)))))
  (go label100)))
(setf lstres zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
```

```
              (tagbody
                (setf lstres
                        (max lstres
                             (abs
                              (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                             x-%offset%))))
               label130))
            (if (/= lstres zero)
                (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                        (/
                         (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                         lstres)))
           label140))
         (go end_label)
       end_label
         (return
          (values uplo n nil nil lda nil ldaf nil nil nil nil nil nil nil nil
                  nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsyrfs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldaf nil nil nil
                                              nil nil nil nil nil nil
```

```
                                        fortran-to-lisp::info)
                                      :calls 'nil)))
```

———-

58.14 dsyrfsx

58.14.1 Axiom unit tests

— **dsyrfsx.input** —

```
)set break resume
)sys rm -f dsyrfsx.output
)spool dsyrfsx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.14.2 Axiom help page [Demm05]

— **dsyrfsx.help** —

```
======================================================================
dsyrfsx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsyrfsx (UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF, IPIV, S, B,
           LDB, X, LDX, RCOND, BERR, N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP,
           NPARAMS, PARAMS, WORK, IWORK, INFO)
           DSYRFSX
```

```
Function/Subroutine Documentation
   subroutine dsyrfsx (character UPLO, character EQUED, integer N, integer
       NRHS, double precision, dimension( lda, * ) A, integer LDA, double
       precision, dimension( ldaf, * ) AF, integer LDAF, integer, dimension( *
       ) IPIV, double precision, dimension( * ) S, double precision,
       dimension( ldb, * ) B, integer LDB, double precision, dimension( ldx, *
       ) X, integer LDX, double precision RCOND, double precision, dimension(
       * ) BERR, integer N_ERR_BNDS, double precision, dimension( nrhs, * )
       ERR_BNDS_NORM, double precision, dimension( nrhs, * ) ERR_BNDS_COMP,
       integer NPARAMS, double precision, dimension( * ) PARAMS, double
       precision, dimension( * ) WORK, integer, dimension( * ) IWORK, integer
       INFO)
       DSYRFSX

Purpose:
   DSYRFSX improves the computed solution to a system of linear
   equations when the coefficient matrix is symmetric indefinite, and
   provides error bounds and backward error estimates for the
   solution.  In addition to normwise error bound, the code provides
   maximum componentwise error bound if possible.  See comments for
   ERR_BNDS_NORM and ERR_BNDS_COMP for details of the error bounds.

   The original system of linear equations may have been equilibrated
   before calling this routine, as described by arguments EQUED and S
   below. In this case, the solution and error bounds returned are
   for the original unequilibrated system.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done to A
   before calling this routine. This is needed to compute
   the solution and error bounds correctly.
   = 'N':  No equilibration
   = 'Y':  Both row and column equilibration, i.e., A has been
           replaced by diag(S) * A * diag(S).
           The right hand side B has been changed accordingly.
```

```
N
  N is INTEGER
  The order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
  upper triangular part of A contains the upper triangular
  part of the matrix A, and the strictly lower triangular
  part of A is not referenced.  If UPLO = 'L', the leading
  N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is DOUBLE PRECISION array, dimension (LDAF,N)
  The factored form of the matrix A.  AF contains the block
  diagonal matrix D and the multipliers used to obtain the
  factor U or L from the factorization A = U*D*U**T or A =
  L*D*L**T as computed by DSYTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by DSYTRF.

S
  S is DOUBLE PRECISION array, dimension (N)
  The scale factors for A.  If EQUED = 'Y', A is multiplied on
  the left and right by diag(S).  S is an input argument if FACT =
  'F'; otherwise, S is an output argument.  If FACT = 'F' and EQUED
  = 'Y', each element of S must be positive.  If S is output, each
  element of S is a power of the radix. If S is input, each element
  of S should be a power of the radix to ensure a reliable solution
  and error estimates. Scaling by powers of the radix does not cause
  rounding errors unless the result underflows or overflows.
  Rounding errors during scaling lead to refining with a matrix that
```

```
  is not equivalent to the input matrix, producing error estimates
  that may not be reliable.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by DGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
            max_j (abs(XTRUE(j,i) - X(j,i)))
```

```
            ----------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * dlamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * dlamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.

ERR_BNDS_COMP
  ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                 abs(XTRUE(j,i) - X(j,i))
            max_j ----------------------
                        abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.
```

```
  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * dlamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * dlamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is DOUBLE PRECISION array, dimension (NPARAMS)
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0D+0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
  (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
```

```
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is DOUBLE PRECISION array, dimension (4*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is guaranteed.
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
     has been completed, but the factor U is exactly singular, so
     the solution and error bounds could not be computed. RCOND = 0
     is returned.
  = N+J: The solution corresponding to the Jth right-hand side is
     not guaranteed. The solutions corresponding to other right-
     hand sides K with K > J may not be guaranteed as well, but
     only the first such right-hand side is reported. If a small
     componentwise error is not requested (PARAMS(3) = 0.0) then
     the Jth right-hand side is the first with a normwise error
     bound that is not guaranteed (the smallest J such
     that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
     the Jth right-hand side is the first with either a normwise or
     componentwise error bound that is not guaranteed (the smallest
     J such that either ERR_BNDS_NORM(J,1) = 0.0 or
     ERR_BNDS_COMP(J,1) = 0.0). See the definition of
     ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
     about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

———-

58.14.3 fortran code

— dsyrfsx.f —

```
*  =====================================================================
      SUBROUTINE DSYRFSX( UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $                    S, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS,
     $                    ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $                    WORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO, EQUED
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX, * ), WORK( * )
      DOUBLE PRECISION   S( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   ITREF_DEFAULT, ITHRESH_DEFAULT
      DOUBLE PRECISION   COMPONENTWISE_DEFAULT, RTHRESH_DEFAULT
      DOUBLE PRECISION   DZTHRESH_DEFAULT
      PARAMETER          ( ITREF_DEFAULT = 1.0D+0 )
      PARAMETER          ( ITHRESH_DEFAULT = 10.0D+0 )
      PARAMETER          ( COMPONENTWISE_DEFAULT = 1.0D+0 )
      PARAMETER          ( RTHRESH_DEFAULT = 0.5D+0 )
      PARAMETER          ( DZTHRESH_DEFAULT = 0.25D+0 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. Local Scalars ..
```

```
      CHARACTER(1)       NORM
      LOGICAL            RCEQU
      INTEGER            J, PREC_TYPE, REF_TYPE, N_NORMS
      DOUBLE PRECISION   ANORM, RCOND_TMP
      DOUBLE PRECISION   ILLRCOND_THRESH, ERR_LBND, CWISE_WRONG
      LOGICAL            IGNORE_CWISE
      INTEGER            ITHRESH
      DOUBLE PRECISION   RTHRESH, UNSTABLE_THRESH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, DSYCON, DLA_SYRFSX_EXTENDED
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, BLAS_FPINFO_X, ILATRANS, ILAPREC
      EXTERNAL           DLAMCH, DLANSY, DLA_SYRCOND
      DOUBLE PRECISION   DLAMCH, DLANSY, DLA_SYRCOND
      LOGICAL            LSAME
      INTEGER            BLAS_FPINFO_X
      INTEGER            ILATRANS, ILAPREC
*     ..
*     .. Executable Statements ..
*
*     Check the input parameters.
*
      INFO = 0
      REF_TYPE = INT( ITREF_DEFAULT )
      IF ( NPARAMS .GE. LA_LINRX_ITREF_I ) THEN
         IF ( PARAMS( LA_LINRX_ITREF_I ) .LT. 0.0D+0 ) THEN
            PARAMS( LA_LINRX_ITREF_I ) = ITREF_DEFAULT
         ELSE
            REF_TYPE = PARAMS( LA_LINRX_ITREF_I )
         END IF
      END IF
*
*     Set default parameters.
*
      ILLRCOND_THRESH = DBLE( N )*DLAMCH( 'Epsilon' )
      ITHRESH = INT( ITHRESH_DEFAULT )
      RTHRESH = RTHRESH_DEFAULT
      UNSTABLE_THRESH = DZTHRESH_DEFAULT
      IGNORE_CWISE = COMPONENTWISE_DEFAULT .EQ. 0.0D+0
*
      IF ( NPARAMS.GE.LA_LINRX_ITHRESH_I ) THEN
         IF ( PARAMS( LA_LINRX_ITHRESH_I ).LT.0.0D+0 ) THEN
            PARAMS( LA_LINRX_ITHRESH_I ) = ITHRESH
         ELSE
            ITHRESH = INT( PARAMS( LA_LINRX_ITHRESH_I ) )
```

```
         END IF
      END IF
      IF ( NPARAMS.GE.LA_LINRX_CWISE_I ) THEN
         IF ( PARAMS( LA_LINRX_CWISE_I ).LT.0.0D+0 ) THEN
            IF ( IGNORE_CWISE ) THEN
               PARAMS( LA_LINRX_CWISE_I ) = 0.0D+0
            ELSE
               PARAMS( LA_LINRX_CWISE_I ) = 1.0D+0
            END IF
         ELSE
            IGNORE_CWISE = PARAMS( LA_LINRX_CWISE_I ) .EQ. 0.0D+0
         END IF
      END IF
      IF ( REF_TYPE .EQ. 0 .OR. N_ERR_BNDS .EQ. 0 ) THEN
         N_NORMS = 0
      ELSE IF ( IGNORE_CWISE ) THEN
         N_NORMS = 1
      ELSE
         N_NORMS = 2
      END IF
*
      RCEQU = LSAME( EQUED, 'Y' )
*
*     Test input parameters.
*
      IF ( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
        INFO = -1
      ELSE IF( .NOT.RCEQU .AND. .NOT.LSAME( EQUED, 'N' ) ) THEN
        INFO = -2
      ELSE IF( N.LT.0 ) THEN
        INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
        INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
        INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
        INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
        INFO = -12
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
        INFO = -14
      END IF
      IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'DSYRFSX', -INFO )
        RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
```

```
         RCOND = 1.0D+0
         DO J = 1, NRHS
            BERR( J ) = 0.0D+0
            IF ( N_ERR_BNDS .GE. 1 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
            IF ( N_ERR_BNDS .GE. 2 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 0.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 0.0D+0
            END IF
            IF ( N_ERR_BNDS .GE. 3 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 1.0D+0
            END IF
         END DO
         RETURN
      END IF
*
*     Default to failure.
*
      RCOND = 0.0D+0
      DO J = 1, NRHS
         BERR( J ) = 1.0D+0
         IF ( N_ERR_BNDS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
         END IF
         IF ( N_ERR_BNDS .GE. 2 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
         END IF
         IF ( N_ERR_BNDS .GE. 3 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 0.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 0.0D+0
         END IF
      END DO
*
*     Compute the norm of A and the reciprocal of the condition
*     number of A.
*
      NORM = 'I'
      ANORM = DLANSY( NORM, UPLO, N, A, LDA, WORK )
      CALL DSYCON( UPLO, N, AF, LDAF, IPIV, ANORM, RCOND, WORK,
     $     IWORK, INFO )
*
*     Perform refinement on each right-hand side
*
      IF ( REF_TYPE .NE. 0 ) THEN
```

```
         PREC_TYPE = ILAPREC( 'E' )

         CALL DLA_SYRFSX_EXTENDED( PREC_TYPE, UPLO,  N,
     $        NRHS, A, LDA, AF, LDAF, IPIV, RCEQU, S, B,
     $        LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM, ERR_BNDS_COMP,
     $        WORK( N+1 ), WORK( 1 ), WORK( 2*N+1 ), WORK( 1 ), RCOND,
     $        ITHRESH, RTHRESH, UNSTABLE_THRESH, IGNORE_CWISE,
     $        INFO )
      END IF

      ERR_LBND = MAX( 10.0D+0, SQRT( DBLE( N ) ) )*DLAMCH( 'Epsilon' )
      IF (N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 1) THEN
*
*     Compute scaled normwise condition number cond(A*C).
*
         IF ( RCEQU ) THEN
            RCOND_TMP = DLA_SYRCOND( UPLO, N, A, LDA, AF, LDAF, IPIV,
     $           -1, S, INFO, WORK, IWORK )
         ELSE
            RCOND_TMP = DLA_SYRCOND( UPLO, N, A, LDA, AF, LDAF, IPIV,
     $           0, S, INFO, WORK, IWORK )
         END IF
         DO J = 1, NRHS
*
*     Cap the error at 1.0.
*
            IF (N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .GT. 1.0D+0)
     $           ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 0.0D+0
               IF ( INFO .LE. N ) INFO = N + J
            ELSE IF (ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .LT. ERR_LBND)
     $              THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
*
*     Save the condition number.
*
            IF (N_ERR_BNDS .GE. LA_LINRX_RCOND_I) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF
         END DO
      END IF
```

```
      IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 2 ) THEN
*
*     Compute componentwise condition number cond(A*diag(Y(:,J))) for
*     each right-hand side using the current solution as an estimate of
*     the true solution.  If the componentwise error estimate is too
*     large, then the solution is a lousy estimate of truth and the
*     estimated RCOND may be too optimistic.  To avoid misleading users,
*     the inverse condition number is set to 0.0 when the estimated
*     cwise error is at least CWISE_WRONG.
*
         CWISE_WRONG = SQRT( DLAMCH( 'Epsilon' ) )
         DO J = 1, NRHS
            IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .LT. CWISE_WRONG )
     $     THEN
               RCOND_TMP = DLA_SYRCOND( UPLO, N, A, LDA, AF, LDAF, IPIV,
     $              1, X(1,J), INFO, WORK, IWORK )
            ELSE
               RCOND_TMP = 0.0D+0
            END IF
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .GT. 1.0D+0 )
     $           ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 0.0D+0
               IF ( .NOT. IGNORE_CWISE
     $              .AND. INFO.LT.N + J ) INFO = N + J
            ELSE IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I )
     $              .LT. ERR_LBND ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF

         END DO
      END IF
*
      RETURN
*
```

```
*     End of DSYRFSX
*
      END
```

———-

58.14.4 lisp code

— **dsyrfsx.lisp** —

———-

58.15 dsytd2

58.15.1 Axiom unit tests

— **dsytd2.input** —

```
)set break resume
)sys rm -f dsytd2.output
)spool dsytd2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.15.2 Axiom help page

— **dsytd2.help** —

```
====================================================================
dsytd2 examples
====================================================================

====================================================================
Man Page Details
```

```
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsytd2 (UPLO, N, A, LDA, D, E, TAU, INFO)
           DSYTD2 reduces a symmetric matrix to real symmetric tridiagonal
           form by an orthogonal similarity transformation (unblocked
           algorithm).

Function/Subroutine Documentation
   subroutine dsytd2 (character UPLO, integer N, double precision, dimension(
       lda, * ) A, integer LDA, double precision, dimension( * ) D, double
       precision, dimension( * ) E, double precision, dimension( * ) TAU,
       integer INFO)
       DSYTD2 reduces a symmetric matrix to real symmetric tridiagonal form by
       an orthogonal similarity transformation (unblocked algorithm).

Purpose:
   DSYTD2 reduces a real symmetric matrix A to symmetric tridiagonal
   form T by an orthogonal similarity transformation: Q**T * A * Q = T.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is stored:
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   n-by-n upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading n-by-n lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.
   On exit, if UPLO = 'U', the diagonal and first superdiagonal
   of A are overwritten by the corresponding elements of the
   tridiagonal matrix T, and the elements above the first
   superdiagonal, with the array TAU, represent the orthogonal
   matrix Q as a product of elementary reflectors; if UPLO
   = 'L', the diagonal and first subdiagonal of A are over-
   written by the corresponding elements of the tridiagonal
```

```
  matrix T, and the elements below the first subdiagonal, with
  the array TAU, represent the orthogonal matrix Q as a product
  of elementary reflectors. See Further Details.

 LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

 D
  D is DOUBLE PRECISION array, dimension (N)
  The diagonal elements of the tridiagonal matrix T: D(i) = A(i,i).

 E
  E is DOUBLE PRECISION array, dimension (N-1)
  The off-diagonal elements of the tridiagonal matrix T:
  E(i) = A(i,i+1) if UPLO = 'U', E(i) = A(i+1,i) if UPLO = 'L'.

 TAU
  TAU is DOUBLE PRECISION array, dimension (N-1)
  The scalar factors of the elementary reflectors (see Further Details).

 INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value.

Further Details:

  If UPLO = 'U', the matrix Q is represented as a product of elementary
  reflectors

  Q = H(n-1) . . . H(2) H(1).

  Each H(i) has the form

  H(i) = I - tau * v * v**T

  where tau is a real scalar, and v is a real vector with
  v(i+1:n) = 0 and v(i) = 1; v(1:i-1) is stored on exit in
  A(1:i-1,i+1), and tau in TAU(i).

  If UPLO = 'L', the matrix Q is represented as a product of elementary
  reflectors

  Q = H(1) H(2) . . . H(n-1).

  Each H(i) has the form

  H(i) = I - tau * v * v**T
```

```
where tau is a real scalar, and v is a real vector with
v(1:i) = 0 and v(i+1) = 1; v(i+2:n) is stored on exit in A(i+2:n,i),
and tau in TAU(i).

The contents of A on exit are illustrated by the following examples
with n = 5:

     if UPLO = 'U':                       if UPLO = 'L':

      (  d   e   v2  v3  v4 )              (  d                  )
      (      d   e   v3  v4 )              (  e   d              )
      (          d   e   v4 )              (  v1  e   d          )
      (              d   e  )              (  v1  v2  e   d      )
      (                  d  )              (  v1  v2  v3  e   d  )

where d and e denote diagonal and off-diagonal elements of T, and vi
denotes an element of the vector defining H(i).
```

———-

58.15.3 fortran code

— dsytd2.f —

```
*  =====================================================================
      SUBROUTINE DSYTD2( UPLO, N, A, LDA, D, E, TAU, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), D( * ), E( * ), TAU( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO, HALF
      PARAMETER          ( ONE = 1.0D0, ZERO = 0.0D0,
     $                   HALF = 1.0D0 / 2.0D0 )
*     ..
```

```
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I
      DOUBLE PRECISION   ALPHA, TAUI
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DLARFG, DSYMV, DSYR2, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DDOT
      EXTERNAL           LSAME, DDOT
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYTD2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.LE.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*        Reduce the upper triangle of A
*
         DO 10 I = N - 1, 1, -1
*
*           Generate elementary reflector H(i) = I - tau * v * v**T
*           to annihilate A(1:i-1,i+1)
*
            CALL DLARFG( I, A( I, I+1 ), A( 1, I+1 ), 1, TAUI )
            E( I ) = A( I, I+1 )
```

```
*
            IF( TAUI.NE.ZERO ) THEN
*
*              Apply H(i) from both sides to A(1:i,1:i)
*
               A( I, I+1 ) = ONE
*
*              Compute  x := tau * A * v  storing x in TAU(1:i)
*
               CALL DSYMV( UPLO, I, TAUI, A, LDA, A( 1, I+1 ), 1, ZERO,
     $                     TAU, 1 )
*
*              Compute  w := x - 1/2 * tau * (x**T * v) * v
*
               ALPHA = -HALF*TAUI*DDOT( I, TAU, 1, A( 1, I+1 ), 1 )
               CALL DAXPY( I, ALPHA, A( 1, I+1 ), 1, TAU, 1 )
*
*              Apply the transformation as a rank-2 update:
*                 A := A - v * w**T - w * v**T
*
               CALL DSYR2( UPLO, I, -ONE, A( 1, I+1 ), 1, TAU, 1, A,
     $                     LDA )
*
               A( I, I+1 ) = E( I )
            END IF
            D( I+1 ) = A( I+1, I+1 )
            TAU( I ) = TAUI
   10    CONTINUE
         D( 1 ) = A( 1, 1 )
      ELSE
*
         Reduce the lower triangle of A
*
         DO 20 I = 1, N - 1
*
            Generate elementary reflector H(i) = I - tau * v * v**T
*           to annihilate A(i+2:n,i)
*
            CALL DLARFG( N-I, A( I+1, I ), A( MIN( I+2, N ), I ), 1,
     $                   TAUI )
            E( I ) = A( I+1, I )
*
            IF( TAUI.NE.ZERO ) THEN
*
*              Apply H(i) from both sides to A(i+1:n,i+1:n)
*
               A( I+1, I ) = ONE
*
*              Compute  x := tau * A * v  storing y in TAU(i:n-1)
*
```

```
              CALL DSYMV( UPLO, N-I, TAUI, A( I+1, I+1 ), LDA,
     $                    A( I+1, I ), 1, ZERO, TAU( I ), 1 )
*
*             Compute  w := x - 1/2 * tau * (x**T * v) * v
*
              ALPHA = -HALF*TAUI*DDOT( N-I, TAU( I ), 1, A( I+1, I ),
     $                1 )
              CALL DAXPY( N-I, ALPHA, A( I+1, I ), 1, TAU( I ), 1 )
*
*             Apply the transformation as a rank-2 update:
*                A := A - v * w**T - w * v**T
*
              CALL DSYR2( UPLO, N-I, -ONE, A( I+1, I ), 1, TAU( I ), 1,
     $                    A( I+1, I+1 ), LDA )
*
              A( I+1, I ) = E( I )
           END IF
           D( I ) = A( I, I )
           TAU( I ) = TAUI
   20    CONTINUE
         D( N ) = A( N, N )
      END IF
*
      RETURN
*
*     End of DSYTD2
*
      END
```

———-

58.15.4 lisp code

— **dsytd2.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0) (half (f2cl-lib:f2cl/ 1.0d0 2.0d0)))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float) half)
           (ignorable one zero half))
  (defun dsytd2 (uplo n a lda d e tau info)
    (declare (type (array double-float (*)) tau e d a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (d double-float d-%data% d-%offset%)
```

```
    (e double-float e-%data% e-%offset%)
    (tau double-float tau-%data% tau-%offset%))
  (prog ((alpha 0.0d0) (taui 0.0d0) (i 0) (upper nil))
    (declare (type (double-float) alpha taui)
             (type (f2cl-lib:integer4) i)
             (type f2cl-lib:logical upper))
    (setf info 0)
    (setf upper
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
    (cond
     ((and (not upper)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
      (setf info -1))
     ((< n 0) (setf info -2))
     ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -4)))
    (cond
     ((/= info 0) (xerbla "DSYTD2" (f2cl-lib:int-sub info))
      (go end_label)))
    (if (<= n 0)
        (go end_label))
    (cond
     (upper
      (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                     (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                    ((> i 1) nil)
        (tagbody
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dlarfg i
               (f2cl-lib:array-slice a-%data% double-float
                                     (i (f2cl-lib:int-add i 1))
                                     ((1 lda) (1 *)) a-%offset%)
               (f2cl-lib:array-slice a-%data% double-float
                                     (1 (f2cl-lib:int-add i 1))
                                     ((1 lda) (1 *)) a-%offset%)
               1 taui)
            (declare (ignore var-1 var-2 var-3))
            (when var-0 (setf i var-0))
            (when var-4 (setf taui var-4)))
          (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
```

```
                        ((1 lda) (1 *)) a-%offset%))
(cond
 ((/= taui zero)
  (setf (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
                        ((1 lda) (1 *)) a-%offset%)
          one)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (dsymv uplo i taui a lda
       (f2cl-lib:array-slice a-%data% double-float
                             (1 (f2cl-lib:int-add i 1))
                             ((1 lda) (1 *)) a-%offset%)
       1 zero tau 1)
    (declare (ignore var-3 var-5 var-6 var-8 var-9))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf i var-1))
    (when var-2 (setf taui var-2))
    (when var-4 (setf lda var-4))
    (when var-7 (setf zero var-7)))
  (setf alpha
          (* (- half) taui
             (multiple-value-bind
                 (ret-val var-0 var-1 var-2 var-3 var-4)
                 (ddot i tau 1
                  (f2cl-lib:array-slice a-%data% double-float
                                        (1
                                         (f2cl-lib:int-add i 1))
                                        ((1 lda) (1 *))
                                        a-%offset%)
                  1)
               (declare (ignore var-1 var-2 var-3 var-4))
               (when var-0 (setf i var-0))
               ret-val)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (daxpy i alpha
       (f2cl-lib:array-slice a-%data% double-float
                             (1 (f2cl-lib:int-add i 1))
                             ((1 lda) (1 *)) a-%offset%)
       1 tau 1)
    (declare (ignore var-2 var-3 var-4 var-5))
    (when var-0 (setf i var-0))
    (when var-1 (setf alpha var-1)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dsyr2 uplo i (- one)
       (f2cl-lib:array-slice a-%data% double-float
                             (1 (f2cl-lib:int-add i 1))
                             ((1 lda) (1 *)) a-%offset%)
       1 tau 1 a lda)
```

```
          (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf i var-1))
          (when var-8 (setf lda var-8)))
        (setf (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
                              ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))))
      (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                            d-%offset%)
              (f2cl-lib:fref a-%data%
                              ((f2cl-lib:int-add i 1)
                               (f2cl-lib:int-add i 1))
                              ((1 lda) (1 *)) a-%offset%))
      (setf (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%) taui)
     label10))
  (setf (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
          (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dlarfg (f2cl-lib:int-sub n i)
           (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                                 ((1 lda) (1 *)) a-%offset%)
           (f2cl-lib:array-slice a-%data% double-float
                                 ((min (f2cl-lib:int-add i 2) n) i)
                                 ((1 lda) (1 *)) a-%offset%)
           1 taui)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf taui var-4)))
      (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
              (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                              ((1 lda) (1 *)) a-%offset%))
      (cond
       ((/= taui zero)
        (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                              ((1 lda) (1 *)) a-%offset%)
                one)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9)
            (dsymv uplo (f2cl-lib:int-sub n i) taui
             (f2cl-lib:array-slice a-%data% double-float
                                   ((+ i 1) (f2cl-lib:int-add i 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                                   ((1 lda) (1 *)) a-%offset%)
             1 zero
```

```
        (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                              tau-%offset%)
        1)
      (declare (ignore var-1 var-3 var-5 var-6 var-8 var-9))
      (when var-0 (setf uplo var-0))
      (when var-2 (setf taui var-2))
      (when var-4 (setf lda var-4))
      (when var-7 (setf zero var-7)))
    (setf alpha
            (* (- half) taui
               (ddot (f2cl-lib:int-sub n i)
                (f2cl-lib:array-slice tau-%data% double-float (i)
                                      ((1 *)) tau-%offset%)
                1
                (f2cl-lib:array-slice a-%data% double-float
                                      ((+ i 1) i) ((1 lda) (1 *))
                                      a-%offset%)
                1)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (daxpy (f2cl-lib:int-sub n i) alpha
        (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                              ((1 lda) (1 *)) a-%offset%)
        1
        (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                              tau-%offset%)
        1)
      (declare (ignore var-0 var-2 var-3 var-4 var-5))
      (when var-1 (setf alpha var-1)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dsyr2 uplo (f2cl-lib:int-sub n i) (- one)
        (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                              ((1 lda) (1 *)) a-%offset%)
        1
        (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                              tau-%offset%)
        1
        (f2cl-lib:array-slice a-%data% double-float
                              ((+ i 1) (f2cl-lib:int-add i 1))
                              ((1 lda) (1 *)) a-%offset%)
        lda)
      (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7))
      (when var-0 (setf uplo var-0))
      (when var-8 (setf lda var-8)))
    (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                          ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))))
  (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
          (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                          a-%offset%))
```

```
              (setf (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%) taui)
             label20))
          (setf (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                  (f2cl-lib:fref a-%data% (n n) ((1 lda) (1 *)) a-%offset%))))
        (go end_label)
       end_label
        (return (values uplo nil nil lda nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsytd2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

58.16 dsytf2

58.16.1 Axiom unit tests

— dsytf2.input —

```
)set break resume
)sys rm -f dsytf2.output
)spool dsytf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.16.2 Axiom help page

— dsytf2.help —

```
==========================================================================
dsytf2 examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
dsytf2.f(3)                          LAPACK                        dsytf2.f(3)



NAME
       dsytf2.f -

SYNOPSIS
   Functions/Subroutines
       subroutine dsytf2 (UPLO, N, A, LDA, IPIV, INFO)
           DSYTF2 computes the factorization of a real symmetric indefinite
           matrix, using the diagonal pivoting method (unblocked algorithm).

Function/Subroutine Documentation
   subroutine dsytf2 (character UPLO, integer N, double precision, dimension(
       lda, * ) A, integer LDA, integer, dimension( * ) IPIV, integer INFO)
       DSYTF2 computes the factorization of a real symmetric indefinite
       matrix, using the diagonal pivoting method (unblocked algorithm).

Purpose:
   DSYTF2 computes the factorization of a real symmetric matrix A using
   the Bunch-Kaufman diagonal pivoting method:

   A = U*D*U**T  or  A = L*D*L**T

   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, U**T is the transpose of U, and D is symmetric and
   block diagonal with 1-by-1 and 2-by-2 diagonal blocks.

   This is the unblocked version of the algorithm, calling Level 2 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
```

```
  Specifies whether the upper or lower triangular part of the
  symmetric matrix A is stored:
  = 'U':  Upper triangular
  = 'L':  Lower triangular

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  n-by-n upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading n-by-n lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, the block diagonal matrix D and the multipliers used
  to obtain the factor U or L (see below for further details).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D.

  If UPLO = 'U':
    If IPIV(k) > 0, then rows and columns k and IPIV(k) were
    interchanged and D(k,k) is a 1-by-1 diagonal block.

    If IPIV(k) = IPIV(k-1) < 0, then rows and columns
    k-1 and -IPIV(k) were interchanged and D(k-1:k,k-1:k)
    is a 2-by-2 diagonal block.

  If UPLO = 'L':
    If IPIV(k) > 0, then rows and columns k and IPIV(k) were
    interchanged and D(k,k) is a 1-by-1 diagonal block.

    If IPIV(k) = IPIV(k+1) < 0, then rows and columns
    k+1 and -IPIV(k) were interchanged and D(k:k+1,k:k+1)
    is a 2-by-2 diagonal block.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -k, the k-th argument had an illegal value
```

```
   > 0: if INFO = k, D(k,k) is exactly zero.  The factorization
        has been completed, but the block diagonal matrix D is
        exactly singular, and division by zero will occur if it
        is used to solve a system of equations.

Further Details:

   If UPLO = 'U', then A = U*D*U**T, where
      U = P(n)*U(n)* ... *P(k)U(k)* ...,
   i.e., U is a product of terms P(k)*U(k), where k decreases from n to
   1 in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and U(k) is a unit upper triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                       (   I    v    0   )   k-s
             U(k) =    (   0    I    0   )   s
                       (   0    0    I   )   n-k
                          k-s   s   n-k

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(1:k-1,k).
   If s = 2, the upper triangle of D(k) overwrites A(k-1,k-1), A(k-1,k),
   and A(k,k), and v overwrites A(1:k-2,k-1:k).

   If UPLO = 'L', then A = L*D*L**T, where
      L = P(1)*L(1)* ... *P(k)*L(k)* ...,
   i.e., L is a product of terms P(k)*L(k), where k increases from 1 to
   n in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and L(k) is a unit lower triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                       (   I    0     0   )  k-1
             L(k) =    (   0    I     0   )  s
                       (   0    v     I   )  n-k-s+1
                          k-1   s  n-k-s+1

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(k+1:n,k).
   If s = 2, the lower triangle of D(k) overwrites A(k,k), A(k+1,k),
   and A(k+1,k+1), and v overwrites A(k+2:n,k:k+1).
```

———-

58.16.3 fortran code

— dsytf2.f —

```
*  =====================================================================
      SUBROUTINE DSYTF2( UPLO, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   EIGHT, SEVTEN
      PARAMETER          ( EIGHT = 8.0D+0, SEVTEN = 17.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, IMAX, J, JMAX, K, KK, KP, KSTEP
      DOUBLE PRECISION   ABSAKK, ALPHA, COLMAX, D11, D12, D21, D22, R1,
     $                   ROWMAX, T, WK, WKM1, WKP1
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, DISNAN
      INTEGER            IDAMAX
      EXTERNAL           LSAME, IDAMAX, DISNAN
*     ..
*     .. External Subroutines ..
      EXTERNAL           DSCAL, DSWAP, DSYR, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
```

```
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYTF2', -INFO )
         RETURN
      END IF
*
*     Initialize ALPHA for use in choosing pivot block size.
*
      ALPHA = ( ONE+SQRT( SEVTEN ) ) / EIGHT
*
      IF( UPPER ) THEN
*
*        Factorize A as U*D*U**T using the upper triangle of A
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        1 or 2
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop
*
         IF( K.LT.1 )
     $      GO TO 70
         KSTEP = 1
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = ABS( A( K, K ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.GT.1 ) THEN
            IMAX = IDAMAX( K-1, A( 1, K ), 1 )
            COLMAX = ABS( A( IMAX, K ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( (MAX( ABSAKK, COLMAX ).EQ.ZERO) .OR. DISNAN(ABSAKK) ) THEN
*
*           Column K is zero or underflow, or contains a NaN:
*           set INFO and continue
```

```
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
         ELSE
            IF( ABSAKK.GE.ALPHA*COLMAX ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
            ELSE
*
*              JMAX is the column-index of the largest off-diagonal
*              element in row IMAX, and ROWMAX is its absolute value
*
               JMAX = IMAX + IDAMAX( K-IMAX, A( IMAX, IMAX+1 ), LDA )
               ROWMAX = ABS( A( IMAX, JMAX ) )
               IF( IMAX.GT.1 ) THEN
                  JMAX = IDAMAX( IMAX-1, A( 1, IMAX ), 1 )
                  ROWMAX = MAX( ROWMAX, ABS( A( JMAX, IMAX ) ) )
               END IF
*
               IF( ABSAKK.GE.ALPHA*COLMAX*( COLMAX / ROWMAX ) ) THEN
*
*                 no interchange, use 1-by-1 pivot block
*
                  KP = K
               ELSE IF( ABS( A( IMAX, IMAX ) ).GE.ALPHA*ROWMAX ) THEN
*
*                 interchange rows and columns K and IMAX, use 1-by-1
*                 pivot block
*
                  KP = IMAX
               ELSE
*
*                 interchange rows and columns K-1 and IMAX, use 2-by-2
*                 pivot block
*
                  KP = IMAX
                  KSTEP = 2
               END IF
            END IF
*
            KK = K - KSTEP + 1
            IF( KP.NE.KK ) THEN
*
*              Interchange rows and columns KK and KP in the leading
*              submatrix A(1:k,1:k)
*
               CALL DSWAP( KP-1, A( 1, KK ), 1, A( 1, KP ), 1 )
```

```
               CALL DSWAP( KK-KP-1, A( KP+1, KK ), 1, A( KP, KP+1 ),
     $                     LDA )
               T = A( KK, KK )
               A( KK, KK ) = A( KP, KP )
               A( KP, KP ) = T
               IF( KSTEP.EQ.2 ) THEN
                  T = A( K-1, K )
                  A( K-1, K ) = A( KP, K )
                  A( KP, K ) = T
               END IF
            END IF
*
*           Update the leading submatrix
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column k now holds
*
*              W(k) = U(k)*D(k)
*
*              where U(k) is the k-th column of U
*
*              Perform a rank-1 update of A(1:k-1,1:k-1) as
*
*              A := A - U(k)*D(k)*U(k)**T = A - W(k)*1/D(k)*W(k)**T
*
               R1 = ONE / A( K, K )
               CALL DSYR( UPLO, K-1, -R1, A( 1, K ), 1, A, LDA )
*
*              Store U(k) in column k
*
               CALL DSCAL( K-1, R1, A( 1, K ), 1 )
            ELSE
*
*              2-by-2 pivot block D(k): columns k and k-1 now hold
*
*              ( W(k-1) W(k) ) = ( U(k-1) U(k) )*D(k)
*
*              where U(k) and U(k-1) are the k-th and (k-1)-th columns
*              of U
*
*              Perform a rank-2 update of A(1:k-2,1:k-2) as
*
*              A := A - ( U(k-1) U(k) )*D(k)*( U(k-1) U(k) )**T
*                 = A - ( W(k-1) W(k) )*inv(D(k))*( W(k-1) W(k) )**T
*
               IF( K.GT.2 ) THEN
*
                  D12 = A( K-1, K )
                  D22 = A( K-1, K-1 ) / D12
```

```
                 D11 = A( K, K ) / D12
                 T = ONE / ( D11*D22-ONE )
                 D12 = T / D12
*
                 DO 30 J = K - 2, 1, -1
                    WKM1 = D12*( D11*A( J, K-1 )-A( J, K ) )
                    WK = D12*( D22*A( J, K )-A( J, K-1 ) )
                    DO 20 I = J, 1, -1
                       A( I, J ) = A( I, J ) - A( I, K )*WK -
     $                            A( I, K-1 )*WKM1
   20               CONTINUE
                    A( J, K ) = WK
                    A( J, K-1 ) = WKM1
   30            CONTINUE
*
              END IF
*
           END IF
        END IF
*
*       Store details of the interchanges in IPIV
*
        IF( KSTEP.EQ.1 ) THEN
           IPIV( K ) = KP
        ELSE
           IPIV( K ) = -KP
           IPIV( K-1 ) = -KP
        END IF
*
*       Decrease K and return to the start of the main loop
*
        K = K - KSTEP
        GO TO 10
*
      ELSE
*
*       Factorize A as L*D*L**T using the lower triangle of A
*
*       K is the main loop index, increasing from 1 to N in steps of
*       1 or 2
*
        K = 1
   40   CONTINUE
*
*       If K > N, exit from loop
*
        IF( K.GT.N )
     $     GO TO 70
        KSTEP = 1
*
```

```
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = ABS( A( K, K ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.LT.N ) THEN
            IMAX = K + IDAMAX( N-K, A( K+1, K ), 1 )
            COLMAX = ABS( A( IMAX, K ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( (MAX( ABSAKK, COLMAX ).EQ.ZERO) .OR. DISNAN(ABSAKK) ) THEN
*
*           Column K is zero or underflow, or contains a NaN:
*           set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
         ELSE
            IF( ABSAKK.GE.ALPHA*COLMAX ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
            ELSE
*
*              JMAX is the column-index of the largest off-diagonal
*              element in row IMAX, and ROWMAX is its absolute value
*
               JMAX = K - 1 + IDAMAX( IMAX-K, A( IMAX, K ), LDA )
               ROWMAX = ABS( A( IMAX, JMAX ) )
               IF( IMAX.LT.N ) THEN
                  JMAX = IMAX + IDAMAX( N-IMAX, A( IMAX+1, IMAX ), 1 )
                  ROWMAX = MAX( ROWMAX, ABS( A( JMAX, IMAX ) ) )
               END IF
*
               IF( ABSAKK.GE.ALPHA*COLMAX*( COLMAX / ROWMAX ) ) THEN
*
*                 no interchange, use 1-by-1 pivot block
*
                  KP = K
               ELSE IF( ABS( A( IMAX, IMAX ) ).GE.ALPHA*ROWMAX ) THEN
*
*                 interchange rows and columns K and IMAX, use 1-by-1
```

```
*                 pivot block
*
                  KP = IMAX
               ELSE
*
*                 interchange rows and columns K+1 and IMAX, use 2-by-2
*                 pivot block
*
                  KP = IMAX
                  KSTEP = 2
               END IF
            END IF
*
            KK = K + KSTEP - 1
            IF( KP.NE.KK ) THEN
*
*              Interchange rows and columns KK and KP in the trailing
*              submatrix A(k:n,k:n)
*
               IF( KP.LT.N )
     $            CALL DSWAP( N-KP, A( KP+1, KK ), 1, A( KP+1, KP ), 1 )
               CALL DSWAP( KP-KK-1, A( KK+1, KK ), 1, A( KP, KK+1 ),
     $                     LDA )
               T = A( KK, KK )
               A( KK, KK ) = A( KP, KP )
               A( KP, KP ) = T
               IF( KSTEP.EQ.2 ) THEN
                  T = A( K+1, K )
                  A( K+1, K ) = A( KP, K )
                  A( KP, K ) = T
               END IF
            END IF
*
*           Update the trailing submatrix
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column k now holds
*
*              W(k) = L(k)*D(k)
*
*              where L(k) is the k-th column of L
*
               IF( K.LT.N ) THEN
*
*                 Perform a rank-1 update of A(k+1:n,k+1:n) as
*
*                 A := A - L(k)*D(k)*L(k)**T = A - W(k)*(1/D(k))*W(k)**T
*
                  D11 = ONE / A( K, K )
```

```
                 CALL DSYR( UPLO, N-K, -D11, A( K+1, K ), 1,
     $                      A( K+1, K+1 ), LDA )
*
*                Store L(k) in column K
*
                 CALL DSCAL( N-K, D11, A( K+1, K ), 1 )
              END IF
           ELSE
*
*             2-by-2 pivot block D(k)
*
              IF( K.LT.N-1 ) THEN
*
*                Perform a rank-2 update of A(k+2:n,k+2:n) as
*
*                A := A - ( (A(k) A(k+1))*D(k)**(-1) ) * (A(k) A(k+1))**T
*
*                where L(k) and L(k+1) are the k-th and (k+1)-th
*                columns of L
*
                 D21 = A( K+1, K )
                 D11 = A( K+1, K+1 ) / D21
                 D22 = A( K, K ) / D21
                 T = ONE / ( D11*D22-ONE )
                 D21 = T / D21
*
                 DO 60 J = K + 2, N
*
                    WK = D21*( D11*A( J, K )-A( J, K+1 ) )
                    WKP1 = D21*( D22*A( J, K+1 )-A( J, K ) )
*
                    DO 50 I = J, N
                       A( I, J ) = A( I, J ) - A( I, K )*WK -
     $                             A( I, K+1 )*WKP1
   50               CONTINUE
*
                    A( J, K ) = WK
                    A( J, K+1 ) = WKP1
*
   60            CONTINUE
              END IF
           END IF
        END IF
*
*       Store details of the interchanges in IPIV
*
        IF( KSTEP.EQ.1 ) THEN
           IPIV( K ) = KP
        ELSE
           IPIV( K ) = -KP
```

```
            IPIV( K+1 ) = -KP
         END IF
*
*        Increase K and return to the start of the main loop
*
         K = K + KSTEP
         GO TO 40
*
      END IF
*
   70 CONTINUE
*
      RETURN
*
*     End of DSYTF2
*
      END
```

———-

58.16.4 lisp code

— dsytf2.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0) (eight 8.0d0) (sevten 17.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 8.0d0 8.0d0) eight)
           (type (double-float 17.0d0 17.0d0) sevten)
           (ignorable zero one eight sevten))
  (defun dsytf2 (uplo n a lda ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array double-float (*)) a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((absakk 0.0d0) (alpha 0.0d0) (colmax 0.0d0) (d11 0.0d0)
             (d12 0.0d0) (d21 0.0d0) (d22 0.0d0) (r1 0.0d0) (rowmax 0.0d0)
             (t$ 0.0d0) (wk 0.0d0) (wkm1 0.0d0) (wkp1 0.0d0) (i 0) (imax 0)
             (j 0) (jmax 0) (k 0) (kk 0) (kp 0) (kstep 0) (upper nil))
        (declare
         (type (double-float) absakk alpha colmax d11 d12 d21 d22 r1 rowmax t$
          wk wkm1 wkp1)
         (type (f2cl-lib:integer4) i imax j jmax k kk kp kstep)
         (type f2cl-lib:logical upper))
```

```
(setf info 0)
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4)))
(cond
 ((/= info 0) (xerbla "DSYTF2" (f2cl-lib:int-sub info))
  (go end_label)))
(setf alpha (/ (+ one (f2cl-lib:fsqrt sevten)) eight))
(cond
 (upper
  (tagbody
    (setf k n)
   label10
    (if (< k 1)
        (go label70))
    (setf kstep 1)
    (setf absakk
            (abs
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)))
    (cond
     ((> k 1)
      (setf imax
              (idamax (f2cl-lib:int-sub k 1)
               (f2cl-lib:array-slice a-%data% double-float (1 k)
                                     ((1 lda) (1 *)) a-%offset%)
               1))
      (setf colmax
              (abs
               (f2cl-lib:fref a-%data% (imax k) ((1 lda) (1 *))
                              a-%offset%))))
     (t (setf colmax zero)))
    (cond
     ((or (= (max absakk colmax) zero)
          (multiple-value-bind (ret-val var-0)
```

```
          (disnan absakk)
        (declare (ignore))
        (when var-0 (setf absakk var-0))
        ret-val))
  (if (= info 0)
      (setf info k))
  (setf kp k))
 (t
  (cond ((>= absakk (* alpha colmax)) (setf kp k))
        (t
         (setf jmax
                 (f2cl-lib:int-add imax
                                   (multiple-value-bind
                                       (ret-val var-0 var-1 var-2)
                                       (idamax
                                        (f2cl-lib:int-sub k imax)
                                        (f2cl-lib:array-slice
                                         a-%data% double-float
                                         (imax
                                          (f2cl-lib:int-add imax
                                                            1))
                                         ((1 lda) (1 *))
                                         a-%offset%)
                                        lda)
                                     (declare (ignore var-0 var-1))
                                     (when var-2 (setf lda var-2))
                                     ret-val)))
         (setf rowmax
                 (abs
                  (f2cl-lib:fref a-%data% (imax jmax)
                                 ((1 lda) (1 *)) a-%offset%)))
         (cond
          ((> imax 1)
           (setf jmax
                   (idamax (f2cl-lib:int-sub imax 1)
                    (f2cl-lib:array-slice a-%data% double-float
                                          (1 imax) ((1 lda) (1 *))
                                          a-%offset%)
                    1))
           (setf rowmax
                   (max rowmax
                        (abs
                         (f2cl-lib:fref a-%data% (jmax imax)
                                        ((1 lda) (1 *))
                                        a-%offset%))))))
         (cond
          ((>= absakk
               (* alpha colmax (f2cl-lib:f2cl/ colmax rowmax)))
           (setf kp k))
          ((>= (abs (f2cl-lib:fref a (imax imax) ((1 lda) (1 *))))
```

```
                (* alpha rowmax))
            (setf kp imax))
           (t (setf kp imax) (setf kstep 2)))))
(setf kk (f2cl-lib:int-add (f2cl-lib:int-sub k kstep) 1))
(cond
 ((/= kp kk)
  (dswap (f2cl-lib:int-sub kp 1)
   (f2cl-lib:array-slice a-%data% double-float (1 kk)
                         ((1 lda) (1 *)) a-%offset%)
   1
   (f2cl-lib:array-slice a-%data% double-float (1 kp)
                         ((1 lda) (1 *)) a-%offset%)
   1)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dswap (f2cl-lib:int-sub kk kp 1)
       (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) kk)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% double-float
                             (kp (f2cl-lib:int-add kp 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (setf t$
          (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                       a-%offset%)
          (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                       a-%offset%)
          t$)
  (cond
   ((= kstep 2)
    (setf t$
            (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                           ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                         ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                         a-%offset%)
            t$)))))
(cond
 ((= kstep 1)
  (setf r1
          (/ one
```

```
            (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                           a-%offset%)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dsyr uplo (f2cl-lib:int-sub k 1) (- r1)
      (f2cl-lib:array-slice a-%data% double-float (1 k)
                            ((1 lda) (1 *)) a-%offset%)
      1 a lda)
   (declare (ignore var-1 var-2 var-3 var-4 var-5))
   (when var-0 (setf uplo var-0))
   (when var-6 (setf lda var-6)))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (dscal (f2cl-lib:int-sub k 1) r1
      (f2cl-lib:array-slice a-%data% double-float (1 k)
                            ((1 lda) (1 *)) a-%offset%)
      1)
   (declare (ignore var-0 var-2 var-3))
   (when var-1 (setf r1 var-1))))
(t
 (cond
  ((> k 2)
   (setf d12
           (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                          ((1 lda) (1 *)) a-%offset%))
   (setf d22
           (/
            (f2cl-lib:fref a-%data%
                           ((f2cl-lib:int-sub k 1)
                            (f2cl-lib:int-sub k 1))
                           ((1 lda) (1 *)) a-%offset%)
            d12))
   (setf d11
           (/
            (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                           a-%offset%)
            d12))
   (setf t$ (/ one (- (* d11 d22) one))) (setf d12 (/ t$ d12))
   (f2cl-lib:fdo (j (f2cl-lib:int-add k (f2cl-lib:int-sub 2))
                  (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                 ((> j 1) nil)
     (tagbody
       (setf wkm1
               (* d12
                  (-
                   (* d11
                      (f2cl-lib:fref a-%data%
                                     (j (f2cl-lib:int-sub k 1))
                                     ((1 lda) (1 *))
                                     a-%offset%))
                   (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
```

```
                                        a-%offset%))))
                (setf wk
                        (* d12
                           (-
                            (* d22
                               (f2cl-lib:fref a-%data% (j k)
                                              ((1 lda) (1 *))
                                              a-%offset%))
                            (f2cl-lib:fref a-%data%
                                           (j (f2cl-lib:int-sub k 1))
                                           ((1 lda) (1 *)) a-%offset%))))
                (f2cl-lib:fdo (i j
                               (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                              ((> i 1) nil)
                  (tagbody
                    (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                            (-
                             (f2cl-lib:fref a-%data% (i j)
                                            ((1 lda) (1 *)) a-%offset%)
                             (*
                              (f2cl-lib:fref a-%data% (i k)
                                             ((1 lda) (1 *)) a-%offset%)
                              wk)
                             (*
                              (f2cl-lib:fref a-%data%
                                             (i (f2cl-lib:int-sub k 1))
                                             ((1 lda) (1 *)) a-%offset%)
                              wkm1)))
                   label20))
                (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                     a-%offset%)
                        wk)
                (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-sub k 1))
                                     ((1 lda) (1 *)) a-%offset%)
                        wkm1)
              label30))))))))
   (cond
    ((= kstep 1)
     (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%) kp))
    (t
     (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
             (f2cl-lib:int-sub kp))
     (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                          ipiv-%offset%)
             (f2cl-lib:int-sub kp))))
   (setf k (f2cl-lib:int-sub k kstep))
   (go label10)))
(t
 (tagbody
```

```
 (setf k 1)
label40
 (if (> k n)
     (go label70))
 (setf kstep 1)
 (setf absakk
         (abs
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%)))
 (cond
  ((< k n)
   (setf imax
           (f2cl-lib:int-add k
                             (idamax (f2cl-lib:int-sub n k)
                              (f2cl-lib:array-slice a-%data%
                                                    double-float
                                                    ((+ k 1) k)
                                                    ((1 lda) (1 *))
                                                    a-%offset%)
                              1)))
   (setf colmax
           (abs
            (f2cl-lib:fref a-%data% (imax k) ((1 lda) (1 *))
                           a-%offset%))))
  (t (setf colmax zero)))
 (cond
  ((or (= (max absakk colmax) zero)
       (multiple-value-bind (ret-val var-0)
           (disnan absakk)
         (declare (ignore))
         (when var-0 (setf absakk var-0))
         ret-val))
   (if (= info 0)
       (setf info k))
   (setf kp k))
  (t
   (cond ((>= absakk (* alpha colmax)) (setf kp k))
         (t
          (setf jmax
                  (f2cl-lib:int-add (f2cl-lib:int-sub k 1)
                                    (multiple-value-bind
                                        (ret-val var-0 var-1 var-2)
                                        (idamax
                                         (f2cl-lib:int-sub imax k)
                                         (f2cl-lib:array-slice
                                          a-%data% double-float
                                          (imax k) ((1 lda) (1 *))
                                          a-%offset%)
                                         lda)
                                      (declare (ignore var-0 var-1))
```

```
                                (when var-2 (setf lda var-2))
                                ret-val)))
        (setf rowmax
                (abs
                 (f2cl-lib:fref a-%data% (imax jmax)
                                ((1 lda) (1 *)) a-%offset%)))
        (cond
         ((< imax n)
          (setf jmax
                  (f2cl-lib:int-add imax
                                (idamax
                                 (f2cl-lib:int-sub n imax)
                                 (f2cl-lib:array-slice
                                  a-%data% double-float
                                  ((+ imax 1) imax)
                                  ((1 lda) (1 *)) a-%offset%)
                                 1)))
          (setf rowmax
                  (max rowmax
                       (abs
                        (f2cl-lib:fref a-%data% (jmax imax)
                                       ((1 lda) (1 *))
                                       a-%offset%))))))
        (cond
         ((>= absakk
              (* alpha colmax (f2cl-lib:f2cl/ colmax rowmax)))
          (setf kp k))
         ((>= (abs (f2cl-lib:fref a (imax imax) ((1 lda) (1 *))))
              (* alpha rowmax))
          (setf kp imax))
         (t (setf kp imax) (setf kstep 2)))))
(setf kk (f2cl-lib:int-sub (f2cl-lib:int-add k kstep) 1))
(cond
 ((/= kp kk)
  (if (< kp n)
      (dswap (f2cl-lib:int-sub n kp)
       (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) kk)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) kp)
                             ((1 lda) (1 *)) a-%offset%)
       1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dswap (f2cl-lib:int-sub kp kk 1)
       (f2cl-lib:array-slice a-%data% double-float ((+ kk 1) kk)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% double-float
                             (kp (f2cl-lib:int-add kk 1))
                             ((1 lda) (1 *)) a-%offset%)
```

```
        lda)
      (declare (ignore var-0 var-1 var-2 var-3))
      (when var-4 (setf lda var-4)))
    (setf t$
            (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                         a-%offset%)
            (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%)
            t$)
    (cond
     ((= kstep 2)
      (setf t$
              (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                             ((1 lda) (1 *)) a-%offset%))
      (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                           ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                             a-%offset%))
      (setf (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                           a-%offset%)
              t$)))))
  (cond
   ((= kstep 1)
    (cond
     ((< k n)
      (setf d11
              (/ one
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dsyr uplo (f2cl-lib:int-sub n k) (- d11)
           (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% double-float
                                 ((+ k 1) (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-1 var-2 var-3 var-4 var-5))
        (when var-0 (setf uplo var-0))
        (when var-6 (setf lda var-6)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal (f2cl-lib:int-sub n k) d11
           (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                                 ((1 lda) (1 *)) a-%offset%)
```

```
          1)
        (declare (ignore var-0 var-2 var-3))
        (when var-1 (setf d11 var-1))))))
      (t
       (cond
        ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
         (setf d21
                 (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                                ((1 lda) (1 *)) a-%offset%))
         (setf d11
                 (/
                  (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add k 1)
                                  (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
                  d21))
         (setf d22
                 (/
                  (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                 a-%offset%)
                  d21))
         (setf t$ (/ one (- (* d11 d22) one))) (setf d21 (/ t$ d21))
         (f2cl-lib:fdo (j (f2cl-lib:int-add k 2)
                        (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf wk
                     (* d21
                        (-
                         (* d11
                            (f2cl-lib:fref a-%data% (j k)
                                           ((1 lda) (1 *))
                                           a-%offset%))
                         (f2cl-lib:fref a-%data%
                                        (j (f2cl-lib:int-add k 1))
                                        ((1 lda) (1 *)) a-%offset%))))
             (setf wkp1
                     (* d21
                        (-
                         (* d22
                            (f2cl-lib:fref a-%data%
                                           (j (f2cl-lib:int-add k 1))
                                           ((1 lda) (1 *))
                                           a-%offset%))
                         (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                        a-%offset%))))
             (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                           ((> i n) nil)
               (tagbody
                 (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
```

```
                                        a-%offset%)
                                 (-
                                  (f2cl-lib:fref a-%data% (i j)
                                                 ((1 lda) (1 *)) a-%offset%)
                                  (*
                                   (f2cl-lib:fref a-%data% (i k)
                                                  ((1 lda) (1 *)) a-%offset%)
                                   wk)
                                  (*
                                   (f2cl-lib:fref a-%data%
                                                  (i (f2cl-lib:int-add k 1))
                                                  ((1 lda) (1 *)) a-%offset%)
                                   wkp1)))
                          label50))
                      (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                           a-%offset%)
                            wk)
                      (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-add k 1))
                                           ((1 lda) (1 *)) a-%offset%)
                            wkp1)
                     label60))))))))
            (cond
             ((= kstep 1)
              (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%) kp))
             (t
              (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
                    (f2cl-lib:int-sub kp))
              (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1)) ((1 *))
                                   ipiv-%offset%)
                    (f2cl-lib:int-sub kp))))
            (setf k (f2cl-lib:int-add k kstep))
            (go label40))))
       label70
        (go end_label)
       end_label
        (return (values uplo nil nil lda nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsytf2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
```

```
                                        :return-values
                                        '(fortran-to-lisp::uplo nil nil
                                          fortran-to-lisp::lda nil
                                          fortran-to-lisp::info)
                                        :calls 'nil)))
```

———-

58.17 dsytf2_rook

58.17.1 Axiom unit tests

— **dsytf2rook.input** —

```
)set break resume
)sys rm -f dsytf2rook.output
)spool dsytf2rook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.17.2 Axiom help page

— **dsytf2_rook.help** —

```
==================================================================
dsytf2_rook examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsytf2_rook (UPLO, N, A, LDA, IPIV, INFO)
           DSYTF2_ROOK computes the factorization of a real symmetric
```

```
          indefinite matrix using the bounded Bunch-Kaufman ('rook') diagonal
          pivoting method (unblocked algorithm).

Function/Subroutine Documentation
   subroutine dsytf2_rook (character UPLO, integer N, double precision,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       integer INFO)
       DSYTF2_ROOK computes the factorization of a real symmetric indefinite
       matrix using the bounded Bunch-Kaufman ('rook') diagonal pivoting
       method (unblocked algorithm).

Purpose:
   DSYTF2_ROOK computes the factorization of a real symmetric matrix A
   using the bounded Bunch-Kaufman ("rook") diagonal pivoting method:

   A = U*D*U**T  or  A = L*D*L**T

   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, U**T is the transpose of U, and D is symmetric and
   block diagonal with 1-by-1 and 2-by-2 diagonal blocks.

   This is the unblocked version of the algorithm, calling Level 2 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is stored:
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   n-by-n upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading n-by-n lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

   On exit, the block diagonal matrix D and the multipliers used
   to obtain the factor U or L (see below for further details).

 LDA
```

```
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D.

   If UPLO = 'U':
      If IPIV(k) > 0, then rows and columns k and IPIV(k)
      were interchanged and D(k,k) is a 1-by-1 diagonal block.

      If IPIV(k) < 0 and IPIV(k-1) < 0, then rows and
      columns k and -IPIV(k) were interchanged and rows and
      columns k-1 and -IPIV(k-1) were inerchaged,
      D(k-1:k,k-1:k) is a 2-by-2 diagonal block.

   If UPLO = 'L':
      If IPIV(k) > 0, then rows and columns k and IPIV(k)
      were interchanged and D(k,k) is a 1-by-1 diagonal block.

      If IPIV(k) < 0 and IPIV(k+1) < 0, then rows and
      columns k and -IPIV(k) were interchanged and rows and
      columns k+1 and -IPIV(k+1) were inerchaged,
      D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -k, the k-th argument had an illegal value
   > 0: if INFO = k, D(k,k) is exactly zero.  The factorization
        has been completed, but the block diagonal matrix D is
        exactly singular, and division by zero will occur if it
        is used to solve a system of equations.

Further Details:

   If UPLO = 'U', then A = U*D*U**T, where
      U = P(n)*U(n)* ... *P(k)U(k)* ...,
   i.e., U is a product of terms P(k)*U(k), where k decreases from n to
   1 in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and U(k) is a unit upper triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                        (   I    v    0   )   k-s
                U(k) =  (   0    I    0   )   s
                        (   0    0    I   )   n-k
                           k-s   s   n-k

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(1:k-1,k).
```

```
   If s = 2, the upper triangle of D(k) overwrites A(k-1,k-1), A(k-1,k),
   and A(k,k), and v overwrites A(1:k-2,k-1:k).

   If UPLO = 'L', then A = L*D*L**T, where
      L = P(1)*L(1)* ... *P(k)*L(k)* ...,
   i.e., L is a product of terms P(k)*L(k), where k increases from 1 to
   n in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and L(k) is a unit lower triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                       (   I    0     0   )  k-1
             L(k) =    (   0    I     0   )  s
                       (   0    v     I   )  n-k-s+1
                          k-1   s  n-k-s+1

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(k+1:n,k).
   If s = 2, the lower triangle of D(k) overwrites A(k,k), A(k+1,k),
   and A(k+1,k+1), and v overwrites A(k+2:n,k:k+1).
```

58.17.3 fortran code

— dsytf2_rook.f —

```
*  =====================================================================
      SUBROUTINE DSYTRF_ROOK( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY, UPPER
```

```
      INTEGER            IINFO, IWS, J, K, KB, LDWORK, LWKOPT, NB, NBMIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASYF_ROOK, DSYTF2_ROOK, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
*
      IF( INFO.EQ.0 ) THEN
*
*        Determine the block size
*
         NB = ILAENV( 1, 'DSYTRF_ROOK', UPLO, N, -1, -1, -1 )
         LWKOPT = N*NB
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYTRF_ROOK', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
      NBMIN = 2
      LDWORK = N
      IF( NB.GT.1 .AND. NB.LT.N ) THEN
         IWS = LDWORK*NB
```

```
         IF( LWORK.LT.IWS ) THEN
            NB = MAX( LWORK / LDWORK, 1 )
            NBMIN = MAX( 2, ILAENV( 2, 'DSYTRF_ROOK',
     $                              UPLO, N, -1, -1, -1 ) )
         END IF
      ELSE
         IWS = 1
      END IF
      IF( NB.LT.NBMIN )
     $   NB = N
*
      IF( UPPER ) THEN
*
*        Factorize A as U*D*U**T using the upper triangle of A
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        KB, where KB is the number of columns factorized by DLASYF_ROOK;
*        KB is either NB or NB-1, or K for the last block
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop
*
         IF( K.LT.1 )
     $      GO TO 40
*
         IF( K.GT.NB ) THEN
*
*           Factorize columns k-kb+1:k of A and use blocked code to
*           update columns 1:k-kb
*
            CALL DLASYF_ROOK( UPLO, K, NB, KB, A, LDA,
     $                        IPIV, WORK, LDWORK, IINFO )
         ELSE
*
*           Use unblocked code to factorize columns 1:k of A
*
            CALL DSYTF2_ROOK( UPLO, K, A, LDA, IPIV, IINFO )
            KB = K
         END IF
*
*        Set INFO on the first occurrence of a zero pivot
*
         IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO
*
*        No need to adjust IPIV
*
*        Decrease K and return to the start of the main loop
```

```
*
         K = K - KB
         GO TO 10
*
      ELSE
*
*        Factorize A as L*D*L**T using the lower triangle of A
*
*        K is the main loop index, increasing from 1 to N in steps of
*        KB, where KB is the number of columns factorized by DLASYF_ROOK;
*        KB is either NB or NB-1, or N-K+1 for the last block
*
         K = 1
   20    CONTINUE
*
*        If K > N, exit from loop
*
         IF( K.GT.N )
     $      GO TO 40
*
         IF( K.LE.N-NB ) THEN
*
*           Factorize columns k:k+kb-1 of A and use blocked code to
*           update columns k+kb:n
*
            CALL DLASYF_ROOK( UPLO, N-K+1, NB, KB, A( K, K ), LDA,
     $                        IPIV( K ), WORK, LDWORK, IINFO )
         ELSE
*
*           Use unblocked code to factorize columns k:n of A
*
            CALL DSYTF2_ROOK( UPLO, N-K+1, A( K, K ), LDA, IPIV( K ),
     $                   IINFO )
            KB = N - K + 1
         END IF
*
*        Set INFO on the first occurrence of a zero pivot
*
         IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO + K - 1
*
*        Adjust IPIV
*
         DO 30 J = K, K + KB - 1
            IF( IPIV( J ).GT.0 ) THEN
               IPIV( J ) = IPIV( J ) + K - 1
            ELSE
               IPIV( J ) = IPIV( J ) - K + 1
            END IF
   30    CONTINUE
```

```
*
*        Increase K and return to the start of the main loop
*
         K = K + KB
         GO TO 20
*
      END IF
*
   40 CONTINUE
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of DSYTRF_ROOK
*
      END
```

———-

58.17.4 lisp code

— dsytf2_rook.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0) (eight 8.0d0) (sevten 17.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 8.0d0 8.0d0) eight)
           (type (double-float 17.0d0 17.0d0) sevten)
           (ignorable zero one eight sevten))
  (defun dsytf2_rook (uplo n a lda ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array double-float (*)) a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((absakk 0.0d0) (alpha 0.0d0) (colmax 0.0d0) (d11 0.0d0)
             (d12 0.0d0) (d21 0.0d0) (d22 0.0d0) (rowmax 0.0d0) (dtemp 0.0d0)
             (t$ 0.0d0) (wk 0.0d0) (wkm1 0.0d0) (wkp1 0.0d0) (sfmin 0.0d0)
             (i 0) (imax 0) (j 0) (jmax 0) (itemp 0) (k 0) (kk 0) (kp 0)
             (kstep 0) (p 0) (ii 0) (upper nil) (done nil))
        (declare
         (type (double-float) absakk alpha colmax d11 d12 d21 d22 rowmax dtemp
          t$ wk wkm1 wkp1 sfmin)
         (type (f2cl-lib:integer4) i imax j jmax itemp k kk kp kstep p ii)
         (type f2cl-lib:logical upper done))
        (setf info 0)
```

```
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4)))
(cond
 ((/= info 0) (xerbla "DSYTF2_ROOK" (f2cl-lib:int-sub info))
  (go end_label)))
(setf alpha (/ (+ one (f2cl-lib:fsqrt sevten)) eight))
(setf sfmin (dlamch "S"))
(cond
 (upper
  (tagbody
    (setf k n)
   label10
    (if (< k 1)
        (go label70))
    (setf kstep 1)
    (setf p k)
    (setf absakk
            (abs
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)))
    (cond
     ((> k 1)
      (setf imax
              (idamax (f2cl-lib:int-sub k 1)
               (f2cl-lib:array-slice a-%data% double-float (1 k)
                                     ((1 lda) (1 *)) a-%offset%)
               1))
      (setf colmax
              (abs
               (f2cl-lib:fref a-%data% (imax k) ((1 lda) (1 *))
                              a-%offset%))))
     (t (setf colmax zero)))
    (cond
     ((= (max absakk colmax) zero)
```

```
 (if (= info 0)
     (setf info k))
 (setf kp k))
(t
 (cond ((not (< absakk (* alpha colmax))) (setf kp k))
       (t
        (tagbody
          (setf done f2cl-lib:%false%)
         label12
          (cond
           ((/= imax k)
            (setf jmax
                    (f2cl-lib:int-add imax
                                      (multiple-value-bind
                                          (ret-val var-0 var-1
                                           var-2)
                                          (idamax
                                           (f2cl-lib:int-sub k
                                                             imax)
                                           (f2cl-lib:array-slice
                                            a-%data% double-float
                                            (imax
                                             (f2cl-lib:int-add
                                              imax 1))
                                            ((1 lda) (1 *))
                                            a-%offset%)
                                           lda)
                                        (declare
                                         (ignore var-0 var-1))
                                        (when var-2
                                          (setf lda var-2))
                                        ret-val)))
            (setf rowmax
                    (abs
                     (f2cl-lib:fref a-%data% (imax jmax)
                                    ((1 lda) (1 *)) a-%offset%))))
           (t (setf rowmax zero)))
          (cond
           ((> imax 1)
            (setf itemp
                    (idamax (f2cl-lib:int-sub imax 1)
                     (f2cl-lib:array-slice a-%data% double-float
                                           (1 imax)
                                           ((1 lda) (1 *))
                                           a-%offset%)
                     1))
            (setf dtemp
                    (abs
                     (f2cl-lib:fref a-%data% (itemp imax)
                                    ((1 lda) (1 *)) a-%offset%)))
```

```
            (cond
             ((> dtemp rowmax) (setf rowmax dtemp)
              (setf jmax itemp)))))
          (cond
           ((not
             (<
              (abs (f2cl-lib:fref a (imax imax) ((1 lda) (1 *))))
              (* alpha rowmax)))
            (setf kp imax) (setf done f2cl-lib:%true%))
           ((or (= p jmax) (<= rowmax colmax)) (setf kp imax)
            (setf kstep 2) (setf done f2cl-lib:%true%))
           (t (setf p imax) (setf colmax rowmax)
            (setf imax jmax)))
          (if (not done)
              (go label12)))))
(cond
 ((and (= kstep 2) (/= p k))
  (if (> p 1)
      (dswap (f2cl-lib:int-sub p 1)
       (f2cl-lib:array-slice a-%data% double-float (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% double-float (1 p)
                             ((1 lda) (1 *)) a-%offset%)
       1))
  (if (< p (f2cl-lib:int-sub k 1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dswap (f2cl-lib:int-sub k p 1)
           (f2cl-lib:array-slice a-%data% double-float
                                 ((+ p 1) k) ((1 lda) (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% double-float
                                 (p (f2cl-lib:int-add p 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf lda var-4))))
  (setf t$
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
          (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *)) a-%offset%)
          t$)))
(setf kk (f2cl-lib:int-add (f2cl-lib:int-sub k kstep) 1))
(cond
 ((/= kp kk)
  (if (> kp 1)
```

```
        (dswap (f2cl-lib:int-sub kp 1)
         (f2cl-lib:array-slice a-%data% double-float (1 kk)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% double-float (1 kp)
                               ((1 lda) (1 *)) a-%offset%)
         1))
    (if (and (> kk 1) (< kp (f2cl-lib:int-sub kk 1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dswap (f2cl-lib:int-sub kk kp 1)
             (f2cl-lib:array-slice a-%data% double-float
                                   ((+ kp 1) kk) ((1 lda) (1 *))
                                   a-%offset%)
             1
             (f2cl-lib:array-slice a-%data% double-float
                                   (kp (f2cl-lib:int-add kp 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-1 var-2 var-3))
          (when var-4 (setf lda var-4))))
    (setf t$
            (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                         a-%offset%)
            (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%)
            t$)
    (cond
     ((= kstep 2)
      (setf t$
              (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                             ((1 lda) (1 *)) a-%offset%))
      (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                           ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                             a-%offset%))
      (setf (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                           a-%offset%)
              t$)))))
  (cond
   ((= kstep 1)
    (cond
     ((> k 1)
      (cond
       ((>= (abs (f2cl-lib:fref a (k k) ((1 lda) (1 *)))) sfmin)
        (setf d11
                (/ one
```

```
                (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                               a-%offset%)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dsyr uplo (f2cl-lib:int-sub k 1) (- d11)
           (f2cl-lib:array-slice a-%data% double-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1 a lda)
        (declare (ignore var-1 var-2 var-3 var-4 var-5))
        (when var-0 (setf uplo var-0))
        (when var-6 (setf lda var-6)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal (f2cl-lib:int-sub k 1) d11
           (f2cl-lib:array-slice a-%data% double-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1)
        (declare (ignore var-0 var-2 var-3))
        (when var-1 (setf d11 var-1))))
     (t
      (setf d11
              (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                             a-%offset%))
      (f2cl-lib:fdo (ii 1 (f2cl-lib:int-add ii 1))
                    ((> ii
                        (f2cl-lib:int-add k
                                          (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
                               a-%offset%)
                (/
                 (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
                                a-%offset%)
                 d11))
         label16))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dsyr uplo (f2cl-lib:int-sub k 1) (- d11)
           (f2cl-lib:array-slice a-%data% double-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1 a lda)
        (declare (ignore var-1 var-2 var-3 var-4 var-5))
        (when var-0 (setf uplo var-0))
        (when var-6 (setf lda var-6))))))))
(t
 (cond
  ((> k 2)
   (setf d12
           (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                          ((1 lda) (1 *)) a-%offset%))
```

```
(setf d22
        (/
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-sub k 1)
                         (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         d12))
(setf d11
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         d12))
(setf t$ (/ one (- (* d11 d22) one)))
(f2cl-lib:fdo (j (f2cl-lib:int-add k (f2cl-lib:int-sub 2))
               (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j 1) nil)
  (tagbody
    (setf wkm1
            (* t$
               (-
                (* d11
                   (f2cl-lib:fref a-%data%
                                  (j (f2cl-lib:int-sub k 1))
                                  ((1 lda) (1 *))
                                  a-%offset%))
                (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                               a-%offset%))))
    (setf wk
            (* t$
               (-
                (* d22
                   (f2cl-lib:fref a-%data% (j k)
                                  ((1 lda) (1 *))
                                  a-%offset%))
                (f2cl-lib:fref a-%data%
                               (j (f2cl-lib:int-sub k 1))
                               ((1 lda) (1 *)) a-%offset%))))
    (f2cl-lib:fdo (i j
                   (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                             a-%offset%)
                (-
                 (f2cl-lib:fref a-%data% (i j)
                                ((1 lda) (1 *)) a-%offset%)
                 (*
                  (/
                   (f2cl-lib:fref a-%data% (i k)
                                  ((1 lda) (1 *)) a-%offset%)
```

```
                                d12)
                               wk)
                              (*
                               (/
                                (f2cl-lib:fref a-%data%
                                               (i (f2cl-lib:int-sub k 1))
                                               ((1 lda) (1 *)) a-%offset%)
                                d12)
                               wkm1)))
                   label20))
                (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                     a-%offset%)
                        (/ wk d12))
                (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-sub k 1))
                                     ((1 lda) (1 *)) a-%offset%)
                        (/ wkm1 d12))
              label30))))))))
    (cond
     ((= kstep 1)
      (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%) kp))
     (t
      (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
              (f2cl-lib:int-sub p))
      (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                           ipiv-%offset%)
              (f2cl-lib:int-sub kp))))
    (setf k (f2cl-lib:int-sub k kstep))
    (go label10)))
  (t
   (tagbody
    (setf k 1)
   label40
    (if (> k n)
        (go label70))
    (setf kstep 1)
    (setf p k)
    (setf absakk
            (abs
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)))
    (cond
     ((< k n)
      (setf imax
              (f2cl-lib:int-add k
                                (idamax (f2cl-lib:int-sub n k)
                                 (f2cl-lib:array-slice a-%data%
                                                       double-float
                                                       ((+ k 1) k)
                                                       ((1 lda) (1 *))
                                                       a-%offset%)
```

```
                                1)))
          (setf colmax
                  (abs
                   (f2cl-lib:fref a-%data% (imax k) ((1 lda) (1 *))
                                  a-%offset%))))
         (t (setf colmax zero)))
        (cond
         ((= (max absakk colmax) zero)
          (if (= info 0)
              (setf info k))
          (setf kp k))
         (t
          (cond ((not (< absakk (* alpha colmax))) (setf kp k))
                (t
                 (tagbody
                   (setf done f2cl-lib:%false%)
                  label42
                   (cond
                    ((/= imax k)
                     (setf jmax
                             (f2cl-lib:int-add (f2cl-lib:int-sub k 1)
                                               (multiple-value-bind
                                                   (ret-val var-0 var-1
                                                    var-2)
                                                   (idamax
                                                    (f2cl-lib:int-sub imax
                                                                      k)
                                                    (f2cl-lib:array-slice
                                                     a-%data% double-float
                                                     (imax k)
                                                     ((1 lda) (1 *))
                                                     a-%offset%)
                                                    lda)
                                                 (declare
                                                  (ignore var-0 var-1))
                                                 (when var-2
                                                   (setf lda var-2))
                                                 ret-val)))
                     (setf rowmax
                             (abs
                              (f2cl-lib:fref a-%data% (imax jmax)
                                             ((1 lda) (1 *)) a-%offset%))))
                    (t (setf rowmax zero)))
                   (cond
                    ((< imax n)
                     (setf itemp
                             (f2cl-lib:int-add imax
                                               (idamax
                                                (f2cl-lib:int-sub n imax)
                                                (f2cl-lib:array-slice
```

```
                                        a-%data% double-float
                                        ((+ imax 1) imax)
                                        ((1 lda) (1 *))
                                        a-%offset%)
                                       1)))
                (setf dtemp
                        (abs
                         (f2cl-lib:fref a-%data% (itemp imax)
                                        ((1 lda) (1 *)) a-%offset%)))
                (cond
                 ((> dtemp rowmax) (setf rowmax dtemp)
                  (setf jmax itemp)))))
              (cond
               ((not
                 (<
                  (abs (f2cl-lib:fref a (imax imax) ((1 lda) (1 *))))
                  (* alpha rowmax)))
                (setf kp imax) (setf done f2cl-lib:%true%))
               ((or (= p jmax) (<= rowmax colmax)) (setf kp imax)
                (setf kstep 2) (setf done f2cl-lib:%true%))
               (t (setf p imax) (setf colmax rowmax)
                (setf imax jmax)))
              (if (not done)
                  (go label42)))))
(cond
 ((and (= kstep 2) (/= p k))
  (if (< p n)
      (dswap (f2cl-lib:int-sub n p)
       (f2cl-lib:array-slice a-%data% double-float ((+ p 1) k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% double-float ((+ p 1) p)
                             ((1 lda) (1 *)) a-%offset%)
       1))
  (if (> p (f2cl-lib:int-add k 1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dswap (f2cl-lib:int-sub p k 1)
           (f2cl-lib:array-slice a-%data% double-float
                                 ((+ k 1) k) ((1 lda) (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% double-float
                                 (p (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf lda var-4))))
  (setf t$
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
```

```
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
          (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *)) a-%offset%)
          t$)))
(setf kk (f2cl-lib:int-sub (f2cl-lib:int-add k kstep) 1))
(cond
 ((/= kp kk)
  (if (< kp n)
      (dswap (f2cl-lib:int-sub n kp)
       (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) kk)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) kp)
                             ((1 lda) (1 *)) a-%offset%)
       1))
  (if (and (< kk n) (> kp (f2cl-lib:int-add kk 1)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dswap (f2cl-lib:int-sub kp kk 1)
           (f2cl-lib:array-slice a-%data% double-float
                                 ((+ kk 1) kk) ((1 lda) (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% double-float
                                 (kp (f2cl-lib:int-add kk 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf lda var-4))))
  (setf t$
          (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                       a-%offset%)
          (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                       a-%offset%)
          t$)
  (cond
   ((= kstep 2)
    (setf t$
            (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                           ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                         ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                         a-%offset%)
```

```
            t$)))))
(cond
 ((= kstep 1)
  (cond
   ((< k n)
    (cond
     ((>= (abs (f2cl-lib:fref a (k k) ((1 lda) (1 *)))) sfmin)
      (setf d11
              (/ one
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dsyr uplo (f2cl-lib:int-sub n k) (- d11)
           (f2cl-lib:array-slice a-%data% double-float
                                 ((+ k 1) k) ((1 lda) (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% double-float
                                 ((+ k 1) (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-1 var-2 var-3 var-4 var-5))
        (when var-0 (setf uplo var-0))
        (when var-6 (setf lda var-6)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal (f2cl-lib:int-sub n k) d11
           (f2cl-lib:array-slice a-%data% double-float
                                 ((+ k 1) k) ((1 lda) (1 *))
                                 a-%offset%)
           1)
        (declare (ignore var-0 var-2 var-3))
        (when var-1 (setf d11 var-1))))
     (t
      (setf d11
              (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                             a-%offset%))
      (f2cl-lib:fdo (ii (f2cl-lib:int-add k 1)
                        (f2cl-lib:int-add ii 1))
                    ((> ii n) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
                               a-%offset%)
                (/
                 (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
                                a-%offset%)
                 d11))
         label46))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
            (dsyr uplo (f2cl-lib:int-sub n k) (- d11)
             (f2cl-lib:array-slice a-%data% double-float
                                   ((+ k 1) k) ((1 lda) (1 *))
                                   a-%offset%)
             1
             (f2cl-lib:array-slice a-%data% double-float
                                   ((+ k 1) (f2cl-lib:int-add k 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-1 var-2 var-3 var-4 var-5))
          (when var-0 (setf uplo var-0))
          (when var-6 (setf lda var-6))))))))
   (t
    (cond
     ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
      (setf d21
              (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                             ((1 lda) (1 *)) a-%offset%))
      (setf d11
              (/
               (f2cl-lib:fref a-%data%
                              ((f2cl-lib:int-add k 1)
                               (f2cl-lib:int-add k 1))
                              ((1 lda) (1 *)) a-%offset%)
               d21))
      (setf d22
              (/
               (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                              a-%offset%)
               d21))
      (setf t$ (/ one (- (* d11 d22) one)))
      (f2cl-lib:fdo (j (f2cl-lib:int-add k 2)
                     (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf wk
                  (* t$
                     (-
                      (* d11
                         (f2cl-lib:fref a-%data% (j k)
                                        ((1 lda) (1 *))
                                        a-%offset%))
                      (f2cl-lib:fref a-%data%
                                     (j (f2cl-lib:int-add k 1))
                                     ((1 lda) (1 *)) a-%offset%))))
          (setf wkp1
                  (* t$
                     (-
                      (* d22
                         (f2cl-lib:fref a-%data%
```

```
                                                (j (f2cl-lib:int-add k 1))
                                                ((1 lda) (1 *))
                                                a-%offset%))
                                    (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                                   a-%offset%))))
                        (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                                      ((> i n) nil)
                          (tagbody
                            (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                                 a-%offset%)
                                  (-
                                   (f2cl-lib:fref a-%data% (i j)
                                                  ((1 lda) (1 *)) a-%offset%)
                                   (*
                                    (/
                                     (f2cl-lib:fref a-%data% (i k)
                                                    ((1 lda) (1 *)) a-%offset%)
                                     d21)
                                    wk)
                                   (*
                                    (/
                                     (f2cl-lib:fref a-%data%
                                                    (i (f2cl-lib:int-add k 1))
                                                    ((1 lda) (1 *)) a-%offset%)
                                     d21)
                                    wkp1)))
                           label50))
                        (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                             a-%offset%)
                              (/ wk d21))
                        (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-add k 1))
                                             ((1 lda) (1 *)) a-%offset%)
                              (/ wkp1 d21))
                      label60))))))))
            (cond
             ((= kstep 1)
              (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%) kp))
             (t
              (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
                    (f2cl-lib:int-sub p))
              (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1)) ((1 *))
                                   ipiv-%offset%)
                    (f2cl-lib:int-sub kp))))
            (setf k (f2cl-lib:int-add k kstep))
            (go label40))))
        label70
         (go end_label)
        end_label
         (return (values uplo nil nil lda nil info))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsytf2_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

58.18 dsytrd

58.18.1 Axiom unit tests

— dsytrd.input —

```
)set break resume
)sys rm -f dsytrd.output
)spool dsytrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.18.2 Axiom help page

— dsytrd.help —

```
====================================================================
dsytrd examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsytrd (UPLO, N, A, LDA, D, E, TAU, WORK, LWORK, INFO)
           DSYTRD

Function/Subroutine Documentation
   subroutine dsytrd (character UPLO, integer N, double precision, dimension(
       lda, * ) A, integer LDA, double precision, dimension( * ) D, double
       precision, dimension( * ) E, double precision, dimension( * ) TAU,
       double precision, dimension( * ) WORK, integer LWORK, integer INFO)
       DSYTRD

Purpose:
   DSYTRD reduces a real symmetric matrix A to real symmetric
   tridiagonal form T by an orthogonal similarity transformation:
   Q**T * A * Q = T.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   N-by-N upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading N-by-N lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.
   On exit, if UPLO = 'U', the diagonal and first superdiagonal
   of A are overwritten by the corresponding elements of the
   tridiagonal matrix T, and the elements above the first
   superdiagonal, with the array TAU, represent the orthogonal
   matrix Q as a product of elementary reflectors; if UPLO
```

```
  = 'L', the diagonal and first subdiagonal of A are over-
  written by the corresponding elements of the tridiagonal
  matrix T, and the elements below the first subdiagonal, with
  the array TAU, represent the orthogonal matrix Q as a product
  of elementary reflectors. See Further Details.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

D
  D is DOUBLE PRECISION array, dimension (N)
  The diagonal elements of the tridiagonal matrix T: D(i) = A(i,i).

E
  E is DOUBLE PRECISION array, dimension (N-1)
  The off-diagonal elements of the tridiagonal matrix T:
  E(i) = A(i,i+1) if UPLO = 'U', E(i) = A(i+1,i) if UPLO = 'L'.

TAU
  TAU is DOUBLE PRECISION array, dimension (N-1)
  The scalar factors of the elementary reflectors (see Further Details).

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= 1.
  For optimum performance LWORK >= N*NB, where NB is the
  optimal blocksize.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details:

  If UPLO = 'U', the matrix Q is represented as a product of elementary
  reflectors

  Q = H(n-1) . . . H(2) H(1).
```

```
Each H(i) has the form

H(i) = I - tau * v * v**T

where tau is a real scalar, and v is a real vector with
v(i+1:n) = 0 and v(i) = 1; v(1:i-1) is stored on exit in
A(1:i-1,i+1), and tau in TAU(i).

If UPLO = 'L', the matrix Q is represented as a product of elementary
reflectors

Q = H(1) H(2) . . . H(n-1).

Each H(i) has the form

H(i) = I - tau * v * v**T

where tau is a real scalar, and v is a real vector with
v(1:i) = 0 and v(i+1) = 1; v(i+2:n) is stored on exit in A(i+2:n,i),
and tau in TAU(i).

The contents of A on exit are illustrated by the following examples
with n = 5:

     if UPLO = 'U':                       if UPLO = 'L':

       (  d   e   v2  v3  v4 )              (  d                  )
       (      d   e   v3  v4 )              (  e   d              )
       (          d   e   v4 )              (  v1  e   d          )
       (              d   e  )              (  v1  v2  e   d      )
       (                  d  )              (  v1  v2  v3  e   d  )

where d and e denote diagonal and off-diagonal elements of T, and vi
denotes an element of the vector defining H(i).
```

———-

58.18.3 fortran code

— **dsytrd.f** —

```
*  =====================================================================
      SUBROUTINE DSYTRD( UPLO, N, A, LDA, D, E, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
```

```
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), D( * ), E( * ), TAU( * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, UPPER
      INTEGER            I, IINFO, IWS, J, KK, LDWORK, LWKOPT, NB,
     $                   NBMIN, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLATRD, DSYR2K, DSYTD2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -9
      END IF
*
```

```
      IF( INFO.EQ.0 ) THEN
*
*        Determine the block size.
*
         NB = ILAENV( 1, 'DSYTRD', UPLO, N, -1, -1, -1 )
         LWKOPT = N*NB
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYTRD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NX = N
      IWS = 1
      IF( NB.GT.1 .AND. NB.LT.N ) THEN
*
*        Determine when to cross over from blocked to unblocked code
*        (last block is always handled by unblocked code).
*
         NX = MAX( NB, ILAENV( 3, 'DSYTRD', UPLO, N, -1, -1, -1 ) )
         IF( NX.LT.N ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = N
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  determine the
*              minimum value of NB, and reduce NB or force use of
*              unblocked code by setting NX = N.
*
               NB = MAX( LWORK / LDWORK, 1 )
               NBMIN = ILAENV( 2, 'DSYTRD', UPLO, N, -1, -1, -1 )
               IF( NB.LT.NBMIN )
     $            NX = N
            END IF
         ELSE
            NX = N
```

```
         END IF
      ELSE
         NB = 1
      END IF
*
      IF( UPPER ) THEN
*
*        Reduce the upper triangle of A.
*        Columns 1:kk are handled by the unblocked method.
*
         KK = N - ( ( N-NX+NB-1 ) / NB )*NB
         DO 20 I = N - NB + 1, KK + 1, -NB
*
*           Reduce columns i:i+nb-1 to tridiagonal form and form the
*           matrix W which is needed to update the unreduced part of
*           the matrix
*
            CALL DLATRD( UPLO, I+NB-1, NB, A, LDA, E, TAU, WORK,
     $                   LDWORK )
*
*           Update the unreduced submatrix A(1:i-1,1:i-1), using an
*           update of the form:  A := A - V*W**T - W*V**T
*
            CALL DSYR2K( UPLO, 'No transpose', I-1, NB, -ONE, A( 1, I ),
     $                   LDA, WORK, LDWORK, ONE, A, LDA )
*
*           Copy superdiagonal elements back into A, and diagonal
*           elements into D
*
            DO 10 J = I, I + NB - 1
               A( J-1, J ) = E( J-1 )
               D( J ) = A( J, J )
   10       CONTINUE
   20    CONTINUE
*
*        Use unblocked code to reduce the last or only block
*
         CALL DSYTD2( UPLO, KK, A, LDA, D, E, TAU, IINFO )
      ELSE
*
*        Reduce the lower triangle of A
*
         DO 40 I = 1, N - NX, NB
*
*           Reduce columns i:i+nb-1 to tridiagonal form and form the
*           matrix W which is needed to update the unreduced part of
*           the matrix
*
            CALL DLATRD( UPLO, N-I+1, NB, A( I, I ), LDA, E( I ),
     $                   TAU( I ), WORK, LDWORK )
```

```
*
*          Update the unreduced submatrix A(i+ib:n,i+ib:n), using
*          an update of the form:  A := A - V*W**T - W*V**T
*
           CALL DSYR2K( UPLO, 'No transpose', N-I-NB+1, NB, -ONE,
     $                  A( I+NB, I ), LDA, WORK( NB+1 ), LDWORK, ONE,
     $                  A( I+NB, I+NB ), LDA )
*
*          Copy subdiagonal elements back into A, and diagonal
*          elements into D
*
           DO 30 J = I, I + NB - 1
              A( J+1, J ) = E( J )
              D( J ) = A( J, J )
   30      CONTINUE
   40   CONTINUE
*
*       Use unblocked code to reduce the last or only block
*
        CALL DSYTD2( UPLO, N-I+1, A( I, I ), LDA, D( I ), E( I ),
     $               TAU( I ), IINFO )
      END IF
*
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of DSYTRD
*
      END
```

———-

58.18.4 lisp code

— **dsytrd.lisp** —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dsytrd (uplo n a lda d e tau work lwork info)
    (declare (type (array double-float (*)) work tau e d a)
             (type (f2cl-lib:integer4) info lwork lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%)
```

```
   (tau double-float tau-%data% tau-%offset%)
   (work double-float work-%data% work-%offset%))
(prog ((i 0) (iinfo 0) (iws 0) (j 0) (kk 0) (ldwork 0) (lwkopt 0) (nb 0)
       (nbmin 0) (nx 0) (lquery nil) (upper nil))
  (declare
   (type (f2cl-lib:integer4) i iinfo iws j kk ldwork lwkopt nb nbmin nx)
   (type f2cl-lib:logical lquery upper))
  (setf info 0)
  (setf upper
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
  (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
  (cond
   ((and (not upper)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4))
   ((and (< lwork 1) (not lquery)) (setf info -9)))
  (cond
   ((= info 0)
    (setf nb
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 1 "DSYTRD" uplo n -1 -1 -1)
              (declare (ignore var-0 var-1 var-4 var-5 var-6))
              (when var-2 (setf uplo var-2))
              (when var-3 (setf n var-3))
              ret-val))
    (setf lwkopt (f2cl-lib:int-mul n nb))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))))
  (cond
   ((/= info 0) (xerbla "DSYTRD" (f2cl-lib:int-sub info)) (go end_label))
   (lquery (go end_label)))
  (cond
   ((= n 0)
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce (the f2cl-lib:integer4 1) 'double-float))
    (go end_label)))
  (setf nx n)
```

```
(setf iws 1)
(cond
 ((and (> nb 1) (< nb n))
  (setf nx
          (max (the f2cl-lib:integer4 nb)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "DSYTRD" uplo n -1 -1 -1)
                      (declare (ignore var-0 var-1 var-4 var-5 var-6))
                      (when var-2 (setf uplo var-2))
                      (when var-3 (setf n var-3))
                      ret-val))))
  (cond
   ((< nx n) (setf ldwork n) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (max (the f2cl-lib:integer4 (truncate lwork ldwork)) 1))
      (setf nbmin
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 2 "DSYTRD" uplo n -1 -1 -1)
                (declare (ignore var-0 var-1 var-4 var-5 var-6))
                (when var-2 (setf uplo var-2))
                (when var-3 (setf n var-3))
                ret-val))
      (if (< nb nbmin)
          (setf nx n)))))
   (t (setf nx n))))
 (t (setf nb 1)))
(cond
 (upper
  (setf kk
          (- n
             (*
              (the f2cl-lib:integer4
                   (truncate (- (+ (- n nx) nb) 1) nb))
              nb)))
  (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub nb) 1)
                 (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
                ((> i (f2cl-lib:int-add kk 1)) nil)
    (tagbody
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dlatrd uplo (f2cl-lib:int-sub (f2cl-lib:int-add i nb) 1) nb
           a lda e tau work ldwork)
        (declare (ignore var-1 var-3 var-5 var-6 var-7))
        (when var-0 (setf uplo var-0))
        (when var-2 (setf nb var-2))
```

```
        (when var-4 (setf lda var-4))
        (when var-8 (setf ldwork var-8)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11)
          (dsyr2k uplo "No transpose" (f2cl-lib:int-sub i 1) nb (- one)
           (f2cl-lib:array-slice a-%data% double-float (1 i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda work ldwork one a lda)
        (declare (ignore var-1 var-2 var-4 var-5 var-7 var-10))
        (when var-0 (setf uplo var-0))
        (when var-3 (setf nb var-3))
        (when var-6 (setf lda var-6))
        (when var-8 (setf ldwork var-8))
        (when var-9 (setf one var-9))
        (when var-11 (setf lda var-11)))
      (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                    ((> j (f2cl-lib:int-add i nb (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub j 1) j)
                               ((1 lda) (1 *)) a-%offset%)
                  (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub j 1))
                                 ((1 *)) e-%offset%))
          (setf (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                  (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                 a-%offset%))
         label10))
    label20))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dsytd2 uplo kk a lda d e tau iinfo)
    (declare (ignore var-1 var-2 var-4 var-5 var-6))
    (setf uplo var-0)
    (setf lda var-3)
    (setf iinfo var-7)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub nx))) nil)
    (tagbody
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dlatrd uplo (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) nb
           (f2cl-lib:array-slice a-%data% double-float (i i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice e-%data% double-float (i) ((1 *))
                                 e-%offset%)
           (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                 tau-%offset%)
```

```
            work ldwork)
          (declare (ignore var-1 var-3 var-5 var-6 var-7))
          (when var-0 (setf uplo var-0))
          (when var-2 (setf nb var-2))
          (when var-4 (setf lda var-4))
          (when var-8 (setf ldwork var-8)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11)
            (dsyr2k uplo "No transpose"
             (f2cl-lib:int-add (f2cl-lib:int-sub n i nb) 1) nb (- one)
             (f2cl-lib:array-slice a-%data% double-float ((+ i nb) i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice work-%data% double-float ((+ nb 1))
                                   ((1 *)) work-%offset%)
             ldwork one
             (f2cl-lib:array-slice a-%data% double-float
                                   ((+ i nb) (f2cl-lib:int-add i nb))
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-1 var-2 var-4 var-5 var-7 var-10))
          (when var-0 (setf uplo var-0))
          (when var-3 (setf nb var-3))
          (when var-6 (setf lda var-6))
          (when var-8 (setf ldwork var-8))
          (when var-9 (setf one var-9))
          (when var-11 (setf lda var-11)))
        (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                      ((> j (f2cl-lib:int-add i nb (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add j 1) j)
                                  ((1 lda) (1 *)) a-%offset%)
                  (f2cl-lib:fref e-%data% (j) ((1 *)) e-%offset%))
            (setf (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                  (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                 a-%offset%))
           label30))
       label40))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (dsytd2 uplo (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
         (f2cl-lib:array-slice a-%data% double-float (i i)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice d-%data% double-float (i) ((1 *))
                               d-%offset%)
         (f2cl-lib:array-slice e-%data% double-float (i) ((1 *))
                               e-%offset%)
```

```
              (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                    tau-%offset%)
              iinfo)
            (declare (ignore var-1 var-2 var-4 var-5 var-6))
            (setf uplo var-0)
            (setf lda var-3)
            (setf iinfo var-7))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
        (go end_label)
       end_label
        (return (values uplo n nil lda nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsytrd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              nil nil fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::dsytd2))))
```

———-

58.19 dsytrf

58.19.1 Axiom unit tests

— dsytrf.input —

```
)set break resume
```

```
)sys rm -f dsytrf.output
)spool dsytrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

58.19.2 Axiom help page

— dsytrf.help —

```
======================================================================
dsytrf examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsytrf (UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO)
           DSYTRF

Function/Subroutine Documentation
   subroutine dsytrf (character UPLO, integer N, double precision, dimension(
       lda, * ) A, integer LDA, integer, dimension( * ) IPIV, double
       precision, dimension( * ) WORK, integer LWORK, integer INFO)
       DSYTRF

Purpose:
   DSYTRF computes the factorization of a real symmetric matrix A using
   the Bunch-Kaufman diagonal pivoting method.  The form of the
   factorization is

   A = U*D*U**T  or  A = L*D*L**T

   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, and D is symmetric and block diagonal with
   1-by-1 and 2-by-2 diagonal blocks.

   This is the blocked version of the algorithm, calling Level 3 BLAS.

Parameters:
```

```
UPLO
  UPLO is CHARACTER*1
  = 'U':  Upper triangle of A is stored;
  = 'L':  Lower triangle of A is stored.

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, the block diagonal matrix D and the multipliers used
  to obtain the factor U or L (see below for further details).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D.
  If IPIV(k) > 0, then rows and columns k and IPIV(k) were
  interchanged and D(k,k) is a 1-by-1 diagonal block.
  If UPLO = 'U' and IPIV(k) = IPIV(k-1) < 0, then rows and
  columns k-1 and -IPIV(k) were interchanged and D(k-1:k,k-1:k)
  is a 2-by-2 diagonal block.  If UPLO = 'L' and IPIV(k) =
  IPIV(k+1) < 0, then rows and columns k+1 and -IPIV(k) were
  interchanged and D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of WORK.  LWORK >=1.  For best performance
  LWORK >= N*NB, where NB is the block size returned by ILAENV.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
```

```
  message related to LWORK is issued by XERBLA.

 INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, D(i,i) is exactly zero.  The factorization
        has been completed, but the block diagonal matrix D is
        exactly singular, and division by zero will occur if it
        is used to solve a system of equations.

Further Details:

  If UPLO = 'U', then A = U*D*U**T, where
     U = P(n)*U(n)* ... *P(k)U(k)* ...,
  i.e., U is a product of terms P(k)*U(k), where k decreases from n to
  1 in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
  and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
  defined by IPIV(k), and U(k) is a unit upper triangular matrix, such
  that if the diagonal block D(k) is of order s (s = 1 or 2), then

                      (   I    v    0   )   k-s
             U(k) =   (   0    I    0   )   s
                      (   0    0    I   )   n-k
                         k-s   s   n-k

  If s = 1, D(k) overwrites A(k,k), and v overwrites A(1:k-1,k).
  If s = 2, the upper triangle of D(k) overwrites A(k-1,k-1), A(k-1,k),
  and A(k,k), and v overwrites A(1:k-2,k-1:k).

  If UPLO = 'L', then A = L*D*L**T, where
     L = P(1)*L(1)* ... *P(k)*L(k)* ...,
  i.e., L is a product of terms P(k)*L(k), where k increases from 1 to
  n in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
  and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
  defined by IPIV(k), and L(k) is a unit lower triangular matrix, such
  that if the diagonal block D(k) is of order s (s = 1 or 2), then

                      (   I    0     0   )  k-1
             L(k) =   (   0    I     0   )  s
                      (   0    v     I   )  n-k-s+1
                         k-1   s  n-k-s+1

  If s = 1, D(k) overwrites A(k,k), and v overwrites A(k+1:n,k).
  If s = 2, the lower triangle of D(k) overwrites A(k,k), A(k+1,k),
  and A(k+1,k+1), and v overwrites A(k+2:n,k:k+1).
```

58.19.3 fortran code

— dsytrf.f —

```
*  =====================================================================
      SUBROUTINE DSYTRF( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY, UPPER
      INTEGER            IINFO, IWS, J, K, KB, LDWORK, LWKOPT, NB, NBMIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASYF, DSYTF2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
```

```
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
*
      IF( INFO.EQ.0 ) THEN
*
*        Determine the block size
*
         NB = ILAENV( 1, 'DSYTRF', UPLO, N, -1, -1, -1 )
         LWKOPT = N*NB
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYTRF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
      NBMIN = 2
      LDWORK = N
      IF( NB.GT.1 .AND. NB.LT.N ) THEN
         IWS = LDWORK*NB
         IF( LWORK.LT.IWS ) THEN
            NB = MAX( LWORK / LDWORK, 1 )
            NBMIN = MAX( 2, ILAENV( 2, 'DSYTRF', UPLO, N, -1, -1, -1 ) )
         END IF
      ELSE
         IWS = 1
      END IF
      IF( NB.LT.NBMIN )
     $   NB = N
*
      IF( UPPER ) THEN
*
*        Factorize A as U*D*U**T using the upper triangle of A
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        KB, where KB is the number of columns factorized by DLASYF;
*        KB is either NB or NB-1, or K for the last block
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop
*
         IF( K.LT.1 )
```

```
     $      GO TO 40
*
         IF( K.GT.NB ) THEN
*
*           Factorize columns k-kb+1:k of A and use blocked code to
*           update columns 1:k-kb
*
            CALL DLASYF( UPLO, K, NB, KB, A, LDA, IPIV, WORK, LDWORK,
     $                   IINFO )
         ELSE
*
*           Use unblocked code to factorize columns 1:k of A
*
            CALL DSYTF2( UPLO, K, A, LDA, IPIV, IINFO )
            KB = K
         END IF
*
*        Set INFO on the first occurrence of a zero pivot
*
         IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO
*
*        Decrease K and return to the start of the main loop
*
         K = K - KB
         GO TO 10
*
      ELSE
*
*        Factorize A as L*D*L**T using the lower triangle of A
*
*        K is the main loop index, increasing from 1 to N in steps of
*        KB, where KB is the number of columns factorized by DLASYF;
*        KB is either NB or NB-1, or N-K+1 for the last block
*
         K = 1
   20    CONTINUE
*
*        If K > N, exit from loop
*
         IF( K.GT.N )
     $      GO TO 40
*
         IF( K.LE.N-NB ) THEN
*
*           Factorize columns k:k+kb-1 of A and use blocked code to
*           update columns k+kb:n
*
            CALL DLASYF( UPLO, N-K+1, NB, KB, A( K, K ), LDA, IPIV( K ),
     $                   WORK, LDWORK, IINFO )
```

```
         ELSE
*
*           Use unblocked code to factorize columns k:n of A
*
            CALL DSYTF2( UPLO, N-K+1, A( K, K ), LDA, IPIV( K ), IINFO )
            KB = N - K + 1
         END IF
*
*        Set INFO on the first occurrence of a zero pivot
*
         IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO + K - 1
*
*        Adjust IPIV
*
         DO 30 J = K, K + KB - 1
            IF( IPIV( J ).GT.0 ) THEN
               IPIV( J ) = IPIV( J ) + K - 1
            ELSE
               IPIV( J ) = IPIV( J ) - K + 1
            END IF
   30    CONTINUE
*
*        Increase K and return to the start of the main loop
*
         K = K + KB
         GO TO 20
*
      END IF
*
   40 CONTINUE
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of DSYTRF
*
      END
```

———-

58.19.4 lisp code

— dsytrf.lisp —

```
(defun dsytrf (uplo n a lda ipiv work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array double-float (*)) work a)
           (type (f2cl-lib:integer4) info lwork lda n)
```

```
          (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a double-float a-%data% a-%offset%)
       (work double-float work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((iinfo 0) (iws 0) (j 0) (k 0) (kb 0) (ldwork 0) (lwkopt 0) (nb 0)
          (nbmin 0) (lquery nil) (upper nil))
      (declare (type f2cl-lib:logical upper lquery)
               (type (f2cl-lib:integer4) nbmin nb lwkopt ldwork kb k j iws
                iinfo))
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond
       ((and (not upper)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -4))
       ((and (< lwork 1) (not lquery)) (setf info -7)))
      (cond
       ((= info 0)
        (setf nb
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (ilaenv 1 "DSYTRF" uplo n -1 -1 -1)
                  (declare (ignore var-0 var-1 var-4 var-5 var-6))
                  (when var-2 (setf uplo var-2))
                  (when var-3 (setf n var-3))
                  ret-val))
        (setf lwkopt (f2cl-lib:int-mul n nb))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))))
      (cond
       ((/= info 0) (xerbla "DSYTRF" (f2cl-lib:int-sub info)) (go end_label))
       (lquery (go end_label)))
      (setf nbmin 2)
      (setf ldwork n)
```

```
(cond
 ((and (> nb 1) (< nb n)) (setf iws (f2cl-lib:int-mul ldwork nb))
  (cond
   ((< lwork iws)
    (setf nb (max (the f2cl-lib:integer4 (truncate lwork ldwork)) 1))
    (setf nbmin
            (max (the f2cl-lib:integer4 2)
                 (the f2cl-lib:integer4
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
                          (ilaenv 2 "DSYTRF" uplo n -1 -1 -1)
                        (declare (ignore var-0 var-1 var-4 var-5 var-6))
                        (when var-2 (setf uplo var-2))
                        (when var-3 (setf n var-3))
                        ret-val)))))))
 (t (setf iws 1)))
(if (< nb nbmin)
    (setf nb n))
(cond
 (upper
  (tagbody
    (setf k n)
   label10
    (if (< k 1)
        (go label40))
    (cond
     ((> k nb)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (dlasyf uplo k nb kb a lda ipiv work ldwork iinfo)
        (declare (ignore var-4 var-6 var-7))
        (when var-0 (setf uplo var-0))
        (when var-1 (setf k var-1))
        (when var-2 (setf nb var-2))
        (when var-3 (setf kb var-3))
        (when var-5 (setf lda var-5))
        (when var-8 (setf ldwork var-8))
        (when var-9 (setf iinfo var-9))))
     (t
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (dsytf2 uplo k a lda ipiv iinfo)
        (declare (ignore var-1 var-2 var-4))
        (setf uplo var-0)
        (setf lda var-3)
        (setf iinfo var-5))
      (setf kb k)))
    (if (and (= info 0) (> iinfo 0))
        (setf info iinfo))
    (setf k (f2cl-lib:int-sub k kb))
```

```
    (go label10)))
  (t
   (tagbody
     (setf k 1)
    label20
     (if (> k n)
         (go label40))
     (cond
      ((<= k (f2cl-lib:int-add n (f2cl-lib:int-sub nb)))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
           (dlasyf uplo (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1) nb kb
            (f2cl-lib:array-slice a-%data% double-float (k k)
                                  ((1 lda) (1 *)) a-%offset%)
            lda
            (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (k)
                                  ((1 *)) ipiv-%offset%)
            work ldwork iinfo)
         (declare (ignore var-1 var-4 var-6 var-7))
         (when var-0 (setf uplo var-0))
         (when var-2 (setf nb var-2))
         (when var-3 (setf kb var-3))
         (when var-5 (setf lda var-5))
         (when var-8 (setf ldwork var-8))
         (when var-9 (setf iinfo var-9))))
      (t
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
           (dsytf2 uplo (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
            (f2cl-lib:array-slice a-%data% double-float (k k)
                                  ((1 lda) (1 *)) a-%offset%)
            lda
            (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (k)
                                  ((1 *)) ipiv-%offset%)
            iinfo)
         (declare (ignore var-1 var-2 var-4))
         (setf uplo var-0)
         (setf lda var-3)
         (setf iinfo var-5))
       (setf kb (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1))))
     (if (and (= info 0) (> iinfo 0))
         (setf info (f2cl-lib:int-sub (f2cl-lib:int-add iinfo k) 1)))
     (f2cl-lib:fdo (j k (f2cl-lib:int-add j 1))
                   ((> j (f2cl-lib:int-add k kb (f2cl-lib:int-sub 1))) nil)
       (tagbody
         (cond
          ((> (f2cl-lib:fref ipiv (j) ((1 *))) 0)
           (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                 (f2cl-lib:int-sub
                  (f2cl-lib:int-add
                   (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
```

```
                          k)
                         1)))
                (t
                 (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                         (f2cl-lib:int-add
                          (f2cl-lib:int-sub
                           (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                           k)
                          1))))
             label30))
          (setf k (f2cl-lib:int-add k kb))
          (go label20))))
     label40
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
      (go end_label)
     end_label
      (return (values uplo n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsytrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::dsytf2))))
```

58.20 dsytrf_rook

58.20.1 Axiom unit tests

— **dsytrfrook.input** —

```
)set break resume
)sys rm -f dsytrfrook.output
)spool dsytrfrook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.20.2 Axiom help page

— **dsytrf_rook.help** —

```
====================================================================
dsytrf_rook examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsytrf_rook (UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO)
           DSYTRF_ROOK

Function/Subroutine Documentation
   subroutine dsytrf_rook (character UPLO, integer N, double precision,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       double precision, dimension( * ) WORK, integer LWORK, integer INFO)
       DSYTRF_ROOK

Purpose:
   DSYTRF_ROOK computes the factorization of a real symmetric matrix A
   using the bounded Bunch-Kaufman ("rook") diagonal pivoting method.
   The form of the factorization is
```

```
   A = U*D*U**T  or  A = L*D*L**T

   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, and D is symmetric and block diagonal with
   1-by-1 and 2-by-2 diagonal blocks.

   This is the blocked version of the algorithm, calling Level 3 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   N-by-N upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading N-by-N lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

   On exit, the block diagonal matrix D and the multipliers used
   to obtain the factor U or L (see below for further details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D.

   If UPLO = 'U':
      If IPIV(k) > 0, then rows and columns k and IPIV(k)
      were interchanged and D(k,k) is a 1-by-1 diagonal block.

      If IPIV(k) < 0 and IPIV(k-1) < 0, then rows and
      columns k and -IPIV(k) were interchanged and rows and
      columns k-1 and -IPIV(k-1) were inerchaged,
      D(k-1:k,k-1:k) is a 2-by-2 diagonal block.

   If UPLO = 'L':
```

```
      If IPIV(k) > 0, then rows and columns k and IPIV(k)
      were interchanged and D(k,k) is a 1-by-1 diagonal block.

      If IPIV(k) < 0 and IPIV(k+1) < 0, then rows and
      columns k and -IPIV(k) were interchanged and rows and
      columns k+1 and -IPIV(k+1) were inerchaged,
      D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

 WORK
   WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK)).
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The length of WORK.  LWORK >=1.  For best performance
   LWORK >= N*NB, where NB is the block size returned by ILAENV.

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, D(i,i) is exactly zero.  The factorization
         has been completed, but the block diagonal matrix D is
         exactly singular, and division by zero will occur if it
         is used to solve a system of equations.

Further Details:

   If UPLO = 'U', then A = U*D*U**T, where
      U = P(n)*U(n)* ... *P(k)U(k)* ...,
   i.e., U is a product of terms P(k)*U(k), where k decreases from n to
   1 in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and U(k) is a unit upper triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                       (   I    v    0   )   k-s
             U(k) =    (   0    I    0   )   s
                       (   0    0    I   )   n-k
                          k-s   s   n-k

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(1:k-1,k).
   If s = 2, the upper triangle of D(k) overwrites A(k-1,k-1), A(k-1,k),
   and A(k,k), and v overwrites A(1:k-2,k-1:k).
```

```
   If UPLO = 'L', then A = L*D*L**T, where
      L = P(1)*L(1)* ... *P(k)*L(k)* ...,
   i.e., L is a product of terms P(k)*L(k), where k increases from 1 to
   n in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and L(k) is a unit lower triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                       (   I    0     0   )  k-1
             L(k) =    (   0    I     0   )  s
                       (   0    v     I   )  n-k-s+1
                          k-1   s  n-k-s+1

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(k+1:n,k).
   If s = 2, the lower triangle of D(k) overwrites A(k,k), A(k+1,k),
   and A(k+1,k+1), and v overwrites A(k+2:n,k:k+1).
```

58.20.3 fortran code

— dsytrf_rook.f —

```
*  =====================================================================
      SUBROUTINE DSYTRF_ROOK( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY, UPPER
      INTEGER            IINFO, IWS, J, K, KB, LDWORK, LWKOPT, NB, NBMIN
*     ..
*     .. External Functions ..
```

```
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASYF_ROOK, DSYTF2_ROOK, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
*
      IF( INFO.EQ.0 ) THEN
*
*        Determine the block size
*
         NB = ILAENV( 1, 'DSYTRF_ROOK', UPLO, N, -1, -1, -1 )
         LWKOPT = N*NB
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYTRF_ROOK', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
      NBMIN = 2
      LDWORK = N
      IF( NB.GT.1 .AND. NB.LT.N ) THEN
         IWS = LDWORK*NB
         IF( LWORK.LT.IWS ) THEN
            NB = MAX( LWORK / LDWORK, 1 )
            NBMIN = MAX( 2, ILAENV( 2, 'DSYTRF_ROOK',
```

```
     $                              UPLO, N, -1, -1, -1 ) )
         END IF
      ELSE
         IWS = 1
      END IF
      IF( NB.LT.NBMIN )
     $   NB = N
*
      IF( UPPER ) THEN
*
*        Factorize A as U*D*U**T using the upper triangle of A
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        KB, where KB is the number of columns factorized by DLASYF_ROOK;
*        KB is either NB or NB-1, or K for the last block
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop
*
         IF( K.LT.1 )
     $      GO TO 40
*
         IF( K.GT.NB ) THEN
*
*           Factorize columns k-kb+1:k of A and use blocked code to
*           update columns 1:k-kb
*
            CALL DLASYF_ROOK( UPLO, K, NB, KB, A, LDA,
     $                        IPIV, WORK, LDWORK, IINFO )
         ELSE
*
*           Use unblocked code to factorize columns 1:k of A
*
            CALL DSYTF2_ROOK( UPLO, K, A, LDA, IPIV, IINFO )
            KB = K
         END IF
*
*        Set INFO on the first occurrence of a zero pivot
*
         IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO
*
*        No need to adjust IPIV
*
*        Decrease K and return to the start of the main loop
*
         K = K - KB
         GO TO 10
```

```
*
      ELSE
*
*        Factorize A as L*D*L**T using the lower triangle of A
*
*        K is the main loop index, increasing from 1 to N in steps of
*        KB, where KB is the number of columns factorized by DLASYF_ROOK;
*        KB is either NB or NB-1, or N-K+1 for the last block
*
         K = 1
   20    CONTINUE
*
*        If K > N, exit from loop
*
         IF( K.GT.N )
     $      GO TO 40
*
         IF( K.LE.N-NB ) THEN
*
*           Factorize columns k:k+kb-1 of A and use blocked code to
*           update columns k+kb:n
*
            CALL DLASYF_ROOK( UPLO, N-K+1, NB, KB, A( K, K ), LDA,
     $                        IPIV( K ), WORK, LDWORK, IINFO )
         ELSE
*
*           Use unblocked code to factorize columns k:n of A
*
            CALL DSYTF2_ROOK( UPLO, N-K+1, A( K, K ), LDA, IPIV( K ),
     $                   IINFO )
            KB = N - K + 1
         END IF
*
*        Set INFO on the first occurrence of a zero pivot
*
         IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO + K - 1
*
*        Adjust IPIV
*
         DO 30 J = K, K + KB - 1
            IF( IPIV( J ).GT.0 ) THEN
               IPIV( J ) = IPIV( J ) + K - 1
            ELSE
               IPIV( J ) = IPIV( J ) - K + 1
            END IF
   30    CONTINUE
*
*        Increase K and return to the start of the main loop
*
```

```
         K = K + KB
         GO TO 20
*
      END IF
*
   40 CONTINUE
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of DSYTRF_ROOK
*
      END
```

———-

58.20.4 lisp code

— dsytrf_rook.lisp —

```
(defun dsytrf_rook (uplo n a lda ipiv work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array double-float (*)) work a)
           (type (f2cl-lib:integer4) info lwork lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a double-float a-%data% a-%offset%)
       (work double-float work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((iinfo 0) (iws 0) (j 0) (k 0) (kb 0) (ldwork 0) (lwkopt 0) (nb 0)
           (nbmin 0) (lquery nil) (upper nil))
      (declare (type f2cl-lib:logical upper lquery)
               (type (f2cl-lib:integer4) nbmin nb lwkopt ldwork kb k j iws
                iinfo))
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond
       ((and (not upper)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
                (declare (ignore var-1))
```

```
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4))
 ((and (< lwork 1) (not lquery)) (setf info -7)))
(cond
 ((= info 0)
  (setf nb
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (ilaenv 1 "DSYTRF_ROOK" uplo n -1 -1 -1)
            (declare (ignore var-0 var-1 var-4 var-5 var-6))
            (when var-2 (setf uplo var-2))
            (when var-3 (setf n var-3))
            ret-val))
  (setf lwkopt (f2cl-lib:int-mul n nb))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))))
(cond
 ((/= info 0) (xerbla "DSYTRF_ROOK" (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(setf nbmin 2)
(setf ldwork n)
(cond
 ((and (> nb 1) (< nb n)) (setf iws (f2cl-lib:int-mul ldwork nb))
  (cond
   ((< lwork iws)
    (setf nb (max (the f2cl-lib:integer4 (truncate lwork ldwork)) 1))
    (setf nbmin
            (max (the f2cl-lib:integer4 2)
                 (the f2cl-lib:integer4
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
                          (ilaenv 2 "DSYTRF_ROOK" uplo n -1 -1 -1)
                        (declare (ignore var-0 var-1 var-4 var-5 var-6))
                        (when var-2 (setf uplo var-2))
                        (when var-3 (setf n var-3))
                        ret-val)))))))
 (t (setf iws 1)))
(if (< nb nbmin)
    (setf nb n))
(cond
 (upper
  (tagbody
    (setf k n)
   label10
```

```
    (if (< k 1)
        (go label40))
    (cond
     ((> k nb)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (dlasyf_rook uplo k nb kb a lda ipiv work ldwork iinfo)
        (declare (ignore var-4 var-6 var-7))
        (when var-0 (setf uplo var-0))
        (when var-1 (setf k var-1))
        (when var-2 (setf nb var-2))
        (when var-3 (setf kb var-3))
        (when var-5 (setf lda var-5))
        (when var-8 (setf ldwork var-8))
        (when var-9 (setf iinfo var-9))))
     (t
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (dsytf2_rook uplo k a lda ipiv iinfo)
        (declare (ignore var-1 var-2 var-4))
        (setf uplo var-0)
        (setf lda var-3)
        (setf iinfo var-5))
      (setf kb k)))
    (if (and (= info 0) (> iinfo 0))
        (setf info iinfo))
    (setf k (f2cl-lib:int-sub k kb))
    (go label10)))
(t
 (tagbody
   (setf k 1)
  label20
   (if (> k n)
       (go label40))
   (cond
    ((<= k (f2cl-lib:int-add n (f2cl-lib:int-sub nb)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (dlasyf_rook uplo (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
          nb kb
          (f2cl-lib:array-slice a-%data% double-float (k k)
                                ((1 lda) (1 *)) a-%offset%)
          lda
          (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (k)
                                ((1 *)) ipiv-%offset%)
          work ldwork iinfo)
       (declare (ignore var-1 var-4 var-6 var-7))
       (when var-0 (setf uplo var-0))
       (when var-2 (setf nb var-2))
       (when var-3 (setf kb var-3))
       (when var-5 (setf lda var-5))
```

```
              (when var-8 (setf ldwork var-8))
              (when var-9 (setf iinfo var-9))))
            (t
             (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                 (dsytf2_rook uplo (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
                  (f2cl-lib:array-slice a-%data% double-float (k k)
                                        ((1 lda) (1 *)) a-%offset%)
                  lda
                  (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (k)
                                        ((1 *)) ipiv-%offset%)
                  iinfo)
              (declare (ignore var-1 var-2 var-4))
              (setf uplo var-0)
              (setf lda var-3)
              (setf iinfo var-5))
             (setf kb (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1))))
           (if (and (= info 0) (> iinfo 0))
              (setf info (f2cl-lib:int-sub (f2cl-lib:int-add iinfo k) 1)))
           (f2cl-lib:fdo (j k (f2cl-lib:int-add j 1))
                         ((> j (f2cl-lib:int-add k kb (f2cl-lib:int-sub 1))) nil)
             (tagbody
               (cond
                ((> (f2cl-lib:fref ipiv (j) ((1 *))) 0)
                 (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                         (f2cl-lib:int-sub
                          (f2cl-lib:int-add
                           (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                           k)
                          1)))
                (t
                 (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                         (f2cl-lib:int-add
                          (f2cl-lib:int-sub
                           (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                           k)
                          1))))
               label30))
           (setf k (f2cl-lib:int-add k kb))
           (go label20))))
     label40
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
      (go end_label)
     end_label
      (return (values uplo n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsytrf_rook
```

```
        fortran-to-lisp::*f2cl-function-info*)
  (fortran-to-lisp::make-f2cl-finfo :arg-types
                                    '((string)
                                      (fortran-to-lisp::integer4)
                                      (array double-float (*))
                                      (fortran-to-lisp::integer4)
                                      (array fortran-to-lisp::integer4
                                       (*))
                                      (array double-float (*))
                                      (fortran-to-lisp::integer4)
                                      (fortran-to-lisp::integer4))
                                    :return-values
                                    '(fortran-to-lisp::uplo
                                      fortran-to-lisp::n nil
                                      fortran-to-lisp::lda nil nil nil
                                      fortran-to-lisp::info)
                                    :calls
                                    '(fortran-to-lisp::dsytf2_rook))))
```

———-

58.21 dsytri

58.21.1 Axiom unit tests

— dsytri.input —

```
)set break resume
)sys rm -f dsytri.output
)spool dsytri.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.21.2 Axiom help page

— dsytri.help —

```
====================================================================
dsytri examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsytri (UPLO, N, A, LDA, IPIV, WORK, INFO)
           DSYTRI

Function/Subroutine Documentation
   subroutine dsytri (character UPLO, integer N, double precision, dimension(
       lda, * ) A, integer LDA, integer, dimension( * ) IPIV, double
       precision, dimension( * ) WORK, integer INFO)
       DSYTRI

Purpose:
   DSYTRI computes the inverse of a real symmetric indefinite matrix
   A using the factorization A = U*D*U**T or A = L*D*L**T computed by DSYTRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the block diagonal matrix D and the multipliers
   used to obtain the factor U or L as computed by DSYTRF.

   On exit, if INFO = 0, the (symmetric) inverse of the original
   matrix.  If UPLO = 'U', the upper triangular part of the
   inverse is formed and the part of A below the diagonal is not
   referenced; if UPLO = 'L' the lower triangular part of the
   inverse is formed and the part of A above the diagonal is
   not referenced.

 LDA
   LDA is INTEGER
```

```
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by DSYTRF.

WORK
  WORK is DOUBLE PRECISION array, dimension (N)

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = i, D(i,i) = 0; the matrix is singular and its
       inverse could not be computed.
```

58.21.3 fortran code

— dsytri.f —

```
*  ===================================================================
      SUBROUTINE DSYTRI( UPLO, N, A, LDA, IPIV, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), WORK( * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
```

```
      LOGICAL            UPPER
      INTEGER            K, KP, KSTEP
      DOUBLE PRECISION   AK, AKKP1, AKP1, D, T, TEMP
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DDOT
      EXTERNAL           LSAME, DDOT
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DSWAP, DSYMV, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYTRI', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
         DO 10 INFO = N, 1, -1
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
   10    CONTINUE
      ELSE
*
```

```
*        Lower triangular storage: examine D from top to bottom.
*
         DO 20 INFO = 1, N
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
   20    CONTINUE
      END IF
      INFO = 0
*
      IF( UPPER ) THEN
*
*        Compute inv(A) from the factorization A = U*D*U**T.
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = 1
   30    CONTINUE
*
*        If K > N, exit from loop.
*
         IF( K.GT.N )
     $      GO TO 40
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Invert the diagonal block.
*
            A( K, K ) = ONE / A( K, K )
*
*           Compute column K of the inverse.
*
            IF( K.GT.1 ) THEN
               CALL DCOPY( K-1, A( 1, K ), 1, WORK, 1 )
               CALL DSYMV( UPLO, K-1, -ONE, A, LDA, WORK, 1, ZERO,
     $                     A( 1, K ), 1 )
               A( K, K ) = A( K, K ) - DDOT( K-1, WORK, 1, A( 1, K ),
     $                     1 )
            END IF
            KSTEP = 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Invert the diagonal block.
*
            T = ABS( A( K, K+1 ) )
            AK = A( K, K ) / T
```

```
            AKP1 = A( K+1, K+1 ) / T
            AKKP1 = A( K, K+1 ) / T
            D = T*( AK*AKP1-ONE )
            A( K, K ) = AKP1 / D
            A( K+1, K+1 ) = AK / D
            A( K, K+1 ) = -AKKP1 / D
*
*           Compute columns K and K+1 of the inverse.
*
            IF( K.GT.1 ) THEN
               CALL DCOPY( K-1, A( 1, K ), 1, WORK, 1 )
               CALL DSYMV( UPLO, K-1, -ONE, A, LDA, WORK, 1, ZERO,
     $                     A( 1, K ), 1 )
               A( K, K ) = A( K, K ) - DDOT( K-1, WORK, 1, A( 1, K ),
     $                     1 )
               A( K, K+1 ) = A( K, K+1 ) -
     $                       DDOT( K-1, A( 1, K ), 1, A( 1, K+1 ), 1 )
               CALL DCOPY( K-1, A( 1, K+1 ), 1, WORK, 1 )
               CALL DSYMV( UPLO, K-1, -ONE, A, LDA, WORK, 1, ZERO,
     $                     A( 1, K+1 ), 1 )
               A( K+1, K+1 ) = A( K+1, K+1 ) -
     $                         DDOT( K-1, WORK, 1, A( 1, K+1 ), 1 )
            END IF
            KSTEP = 2
         END IF
*
         KP = ABS( IPIV( K ) )
         IF( KP.NE.K ) THEN
*
*           Interchange rows and columns K and KP in the leading
*           submatrix A(1:k+1,1:k+1)
*
            CALL DSWAP( KP-1, A( 1, K ), 1, A( 1, KP ), 1 )
            CALL DSWAP( K-KP-1, A( KP+1, K ), 1, A( KP, KP+1 ), LDA )
            TEMP = A( K, K )
            A( K, K ) = A( KP, KP )
            A( KP, KP ) = TEMP
            IF( KSTEP.EQ.2 ) THEN
               TEMP = A( K, K+1 )
               A( K, K+1 ) = A( KP, K+1 )
               A( KP, K+1 ) = TEMP
            END IF
         END IF
*
         K = K + KSTEP
         GO TO 30
   40    CONTINUE
*
      ELSE
*
```

```
*        Compute inv(A) from the factorization A = L*D*L**T.
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = N
   50    CONTINUE
*
*        If K < 1, exit from loop.
*
         IF( K.LT.1 )
     $      GO TO 60
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Invert the diagonal block.
*
            A( K, K ) = ONE / A( K, K )
*
*           Compute column K of the inverse.
*
            IF( K.LT.N ) THEN
               CALL DCOPY( N-K, A( K+1, K ), 1, WORK, 1 )
               CALL DSYMV( UPLO, N-K, -ONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                     ZERO, A( K+1, K ), 1 )
               A( K, K ) = A( K, K ) - DDOT( N-K, WORK, 1, A( K+1, K ),
     $                     1 )
            END IF
            KSTEP = 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Invert the diagonal block.
*
            T = ABS( A( K, K-1 ) )
            AK = A( K-1, K-1 ) / T
            AKP1 = A( K, K ) / T
            AKKP1 = A( K, K-1 ) / T
            D = T*( AK*AKP1-ONE )
            A( K-1, K-1 ) = AKP1 / D
            A( K, K ) = AK / D
            A( K, K-1 ) = -AKKP1 / D
*
*           Compute columns K-1 and K of the inverse.
*
            IF( K.LT.N ) THEN
               CALL DCOPY( N-K, A( K+1, K ), 1, WORK, 1 )
```

```
               CALL DSYMV( UPLO, N-K, -ONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                     ZERO, A( K+1, K ), 1 )
               A( K, K ) = A( K, K ) - DDOT( N-K, WORK, 1, A( K+1, K ),
     $                     1 )
               A( K, K-1 ) = A( K, K-1 ) -
     $                       DDOT( N-K, A( K+1, K ), 1, A( K+1, K-1 ),
     $                       1 )
               CALL DCOPY( N-K, A( K+1, K-1 ), 1, WORK, 1 )
               CALL DSYMV( UPLO, N-K, -ONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                     ZERO, A( K+1, K-1 ), 1 )
               A( K-1, K-1 ) = A( K-1, K-1 ) -
     $                         DDOT( N-K, WORK, 1, A( K+1, K-1 ), 1 )
            END IF
            KSTEP = 2
         END IF
*
         KP = ABS( IPIV( K ) )
         IF( KP.NE.K ) THEN
*
*           Interchange rows and columns K and KP in the trailing
*           submatrix A(k-1:n,k-1:n)
*
            IF( KP.LT.N )
     $         CALL DSWAP( N-KP, A( KP+1, K ), 1, A( KP+1, KP ), 1 )
            CALL DSWAP( KP-K-1, A( K+1, K ), 1, A( KP, K+1 ), LDA )
            TEMP = A( K, K )
            A( K, K ) = A( KP, KP )
            A( KP, KP ) = TEMP
            IF( KSTEP.EQ.2 ) THEN
               TEMP = A( K, K-1 )
               A( K, K-1 ) = A( KP, K-1 )
               A( KP, K-1 ) = TEMP
            END IF
         END IF
*
         K = K - KSTEP
         GO TO 50
   60    CONTINUE
      END IF
*
      RETURN
*
*     End of DSYTRI
*
      END
```

———-

58.21.4 lisp code

— dsytri.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dsytri (uplo n a lda ipiv work info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array double-float (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (work double-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((ak 0.0d0) (akkp1 0.0d0) (akp1 0.0d0) (d 0.0d0) (t$ 0.0d0)
             (temp 0.0d0) (k 0) (kp 0) (kstep 0) (upper nil))
        (declare (type (double-float) ak akkp1 akp1 d t$ temp)
                 (type (f2cl-lib:integer4) k kp kstep)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -4)))
        (cond
         ((/= info 0) (xerbla "DSYTRI" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (= n 0)
            (go end_label))
        (cond
         (upper
```

```
  (f2cl-lib:fdo (info n (f2cl-lib:int-add info (f2cl-lib:int-sub 1)))
                ((> info 1) nil)
    (tagbody
      (if (and
           (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
              0)
           (=
            (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                           a-%offset%)
            zero))
          (go end_label))
     label10)))
 (t
  (f2cl-lib:fdo (info 1 (f2cl-lib:int-add info 1))
                ((> info n) nil)
    (tagbody
      (if (and
           (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
              0)
           (=
            (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                           a-%offset%)
            zero))
          (go end_label))
     label20))))
(setf info 0)
(cond
 (upper
  (tagbody
    (setf k 1)
   label30
    (if (> k n)
        (go label40))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
              (/ one
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%)))
      (cond
       ((> k 1)
        (dcopy (f2cl-lib:int-sub k 1)
         (f2cl-lib:array-slice a-%data% double-float (1 k)
                               ((1 lda) (1 *)) a-%offset%)
         1 work 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9)
            (dsymv uplo (f2cl-lib:int-sub k 1) (- one) a lda work 1
             zero
```

```
          (f2cl-lib:array-slice a-%data% double-float (1 k)
                                ((1 lda) (1 *)) a-%offset%)
          1)
      (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
      (when var-0 (setf uplo var-0))
      (when var-4 (setf lda var-4))
      (when var-7 (setf zero var-7)))
    (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
            (-
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)
             (ddot (f2cl-lib:int-sub k 1) work 1
              (f2cl-lib:array-slice a-%data% double-float (1 k)
                                    ((1 lda) (1 *)) a-%offset%)
              1)))))
  (setf kstep 1))
 (t
  (setf t$
          (abs
           (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                          ((1 lda) (1 *)) a-%offset%)))
  (setf ak
          (/
           (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                          a-%offset%)
           t$))
  (setf akp1
          (/
           (f2cl-lib:fref a-%data%
                          ((f2cl-lib:int-add k 1)
                           (f2cl-lib:int-add k 1))
                          ((1 lda) (1 *)) a-%offset%)
           t$))
  (setf akkp1
          (/
           (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                          ((1 lda) (1 *)) a-%offset%)
           t$))
  (setf d (* t$ (- (* ak akp1) one)))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
          (/ akp1 d))
  (setf (f2cl-lib:fref a-%data%
                       ((f2cl-lib:int-add k 1)
                        (f2cl-lib:int-add k 1))
                       ((1 lda) (1 *)) a-%offset%)
          (/ ak d))
  (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                       ((1 lda) (1 *)) a-%offset%)
          (/ (- akkp1) d))
  (cond
```

```
((> k 1)
 (dcopy (f2cl-lib:int-sub k 1)
  (f2cl-lib:array-slice a-%data% double-float (1 k)
                        ((1 lda) (1 *)) a-%offset%)
  1 work 1)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9)
     (dsymv uplo (f2cl-lib:int-sub k 1) (- one) a lda work 1
      zero
      (f2cl-lib:array-slice a-%data% double-float (1 k)
                            ((1 lda) (1 *)) a-%offset%)
      1)
   (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
   (when var-0 (setf uplo var-0))
   (when var-4 (setf lda var-4))
   (when var-7 (setf zero var-7)))
 (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
         (-
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%)
          (ddot (f2cl-lib:int-sub k 1) work 1
           (f2cl-lib:array-slice a-%data% double-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1)))
 (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                       ((1 lda) (1 *)) a-%offset%)
         (-
          (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                         ((1 lda) (1 *)) a-%offset%)
          (ddot (f2cl-lib:int-sub k 1)
           (f2cl-lib:array-slice a-%data% double-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% double-float
                                 (1 (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           1)))
 (dcopy (f2cl-lib:int-sub k 1)
  (f2cl-lib:array-slice a-%data% double-float
                        (1 (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
  1 work 1)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9)
     (dsymv uplo (f2cl-lib:int-sub k 1) (- one) a lda work 1
      zero
      (f2cl-lib:array-slice a-%data% double-float
                            (1 (f2cl-lib:int-add k 1))
```

```
                                  ((1 lda) (1 *)) a-%offset%)
          1)
        (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
        (when var-0 (setf uplo var-0))
        (when var-4 (setf lda var-4))
        (when var-7 (setf zero var-7)))
      (setf (f2cl-lib:fref a-%data%
                           ((f2cl-lib:int-add k 1)
                            (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
              (-
               (f2cl-lib:fref a-%data%
                              ((f2cl-lib:int-add k 1)
                               (f2cl-lib:int-add k 1))
                              ((1 lda) (1 *)) a-%offset%)
               (ddot (f2cl-lib:int-sub k 1) work 1
                (f2cl-lib:array-slice a-%data% double-float
                                      (1 (f2cl-lib:int-add k 1))
                                      ((1 lda) (1 *)) a-%offset%)
                1)))))
  (setf kstep 2)))
(setf kp
        (abs
         (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
(cond
 ((/= kp k)
  (dswap (f2cl-lib:int-sub kp 1)
   (f2cl-lib:array-slice a-%data% double-float (1 k)
                         ((1 lda) (1 *)) a-%offset%)
   1
   (f2cl-lib:array-slice a-%data% double-float (1 kp)
                         ((1 lda) (1 *)) a-%offset%)
   1)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dswap (f2cl-lib:int-sub k kp 1)
       (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% double-float
                             (kp (f2cl-lib:int-add kp 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (setf temp
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
          (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%))
```

```
      (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *)) a-%offset%)
             temp)
      (cond
       ((= kstep 2)
        (setf temp
                (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%))
        (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%))
        (setf (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
                temp)))))
    (setf k (f2cl-lib:int-add k kstep))
    (go label30)
   label40))
  (t
   (tagbody
    (setf k n)
   label50
    (if (< k 1)
        (go label60))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
             (/ one
                (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                               a-%offset%)))
      (cond
       ((< k n)
        (dcopy (f2cl-lib:int-sub n k)
         (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                               ((1 lda) (1 *)) a-%offset%)
         1 work 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9)
            (dsymv uplo (f2cl-lib:int-sub n k) (- one)
             (f2cl-lib:array-slice a-%data% double-float
                                   ((+ k 1) (f2cl-lib:int-add k 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda work 1 zero
             (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
          (when var-0 (setf uplo var-0))
          (when var-4 (setf lda var-4))
          (when var-7 (setf zero var-7)))
```

```
    (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
            (-
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)
             (ddot (f2cl-lib:int-sub n k) work 1
              (f2cl-lib:array-slice a-%data% double-float
                                    ((+ k 1) k) ((1 lda) (1 *))
                                    a-%offset%)
              1)))))
  (setf kstep 1))
 (t
  (setf t$
          (abs
           (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                          ((1 lda) (1 *)) a-%offset%)))
  (setf ak
          (/
           (f2cl-lib:fref a-%data%
                          ((f2cl-lib:int-sub k 1)
                           (f2cl-lib:int-sub k 1))
                          ((1 lda) (1 *)) a-%offset%)
           t$))
  (setf akp1
          (/
           (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                          a-%offset%)
           t$))
  (setf akkp1
          (/
           (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                          ((1 lda) (1 *)) a-%offset%)
           t$))
  (setf d (* t$ (- (* ak akp1) one)))
  (setf (f2cl-lib:fref a-%data%
                       ((f2cl-lib:int-sub k 1)
                        (f2cl-lib:int-sub k 1))
                       ((1 lda) (1 *)) a-%offset%)
          (/ akp1 d))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
          (/ ak d))
  (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                       ((1 lda) (1 *)) a-%offset%)
          (/ (- akkp1) d))
  (cond
   ((< k n)
    (dcopy (f2cl-lib:int-sub n k)
     (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                           ((1 lda) (1 *)) a-%offset%)
     1 work 1)
    (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dsymv uplo (f2cl-lib:int-sub n k) (- one)
     (f2cl-lib:array-slice a-%data% double-float
                           ((+ k 1) (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda work 1 zero
     (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                           ((1 lda) (1 *)) a-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
  (when var-0 (setf uplo var-0))
  (when var-4 (setf lda var-4))
  (when var-7 (setf zero var-7)))
(setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         (ddot (f2cl-lib:int-sub n k) work 1
          (f2cl-lib:array-slice a-%data% double-float
                                ((+ k 1) k) ((1 lda) (1 *))
                                a-%offset%)
          1)))
(setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                     ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         (ddot (f2cl-lib:int-sub n k)
          (f2cl-lib:array-slice a-%data% double-float
                                ((+ k 1) k) ((1 lda) (1 *))
                                a-%offset%)
          1
          (f2cl-lib:array-slice a-%data% double-float
                                ((+ k 1)
                                 (f2cl-lib:int-sub k 1))
                                ((1 lda) (1 *)) a-%offset%)
          1)))
(dcopy (f2cl-lib:int-sub n k)
 (f2cl-lib:array-slice a-%data% double-float
                       ((+ k 1) (f2cl-lib:int-sub k 1))
                       ((1 lda) (1 *)) a-%offset%)
 1 work 1)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dsymv uplo (f2cl-lib:int-sub n k) (- one)
     (f2cl-lib:array-slice a-%data% double-float
                           ((+ k 1) (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
```

```
          lda work 1 zero
          (f2cl-lib:array-slice a-%data% double-float
                                ((+ k 1) (f2cl-lib:int-sub k 1))
                                ((1 lda) (1 *)) a-%offset%)
          1)
        (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
        (when var-0 (setf uplo var-0))
        (when var-4 (setf lda var-4))
        (when var-7 (setf zero var-7)))
      (setf (f2cl-lib:fref a-%data%
                           ((f2cl-lib:int-sub k 1)
                            (f2cl-lib:int-sub k 1))
                           ((1 lda) (1 *)) a-%offset%)
              (-
               (f2cl-lib:fref a-%data%
                              ((f2cl-lib:int-sub k 1)
                               (f2cl-lib:int-sub k 1))
                              ((1 lda) (1 *)) a-%offset%)
               (ddot (f2cl-lib:int-sub n k) work 1
                (f2cl-lib:array-slice a-%data% double-float
                                      ((+ k 1)
                                       (f2cl-lib:int-sub k 1))
                                      ((1 lda) (1 *)) a-%offset%)
                1)))))
  (setf kstep 2)))
(setf kp
        (abs
         (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
(cond
 ((/= kp k)
  (if (< kp n)
      (dswap (f2cl-lib:int-sub n kp)
       (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) kp)
                             ((1 lda) (1 *)) a-%offset%)
       1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dswap (f2cl-lib:int-sub kp k 1)
       (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% double-float
                             (kp (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (setf temp
```

```
                (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                               a-%offset%))
        (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                               a-%offset%))
        (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *)) a-%offset%)
                temp)
        (cond
         ((= kstep 2)
          (setf temp
                  (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                                 ((1 lda) (1 *)) a-%offset%))
          (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                               ((1 lda) (1 *)) a-%offset%)
                  (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-sub k 1))
                                 ((1 lda) (1 *)) a-%offset%))
          (setf (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-sub k 1))
                               ((1 lda) (1 *)) a-%offset%)
                  temp)))))
      (setf k (f2cl-lib:int-sub k kstep))
      (go label50)
     label60)))
   (go end_label)
  end_label
   (return (values uplo nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsytri
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

58.22 dsytri2

58.22.1 Axiom unit tests

— dsytri2.input —

```
)set break resume
)sys rm -f dsytri2.output
)spool dsytri2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.22.2 Axiom help page

— dsytri2.help —

```
======================================================================
dsytri2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsytri2 (UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO)
           DSYTRI2

Function/Subroutine Documentation
   subroutine dsytri2 (character UPLO, integer N, double precision, dimension(
       lda, * ) A, integer LDA, integer, dimension( * ) IPIV, double
       precision, dimension( * ) WORK, integer LWORK, integer INFO)
       DSYTRI2

Purpose:
   DSYTRI2 computes the inverse of a DOUBLE PRECISION symmetric
   indefinite matrix A using the factorization A = U*D*U**T
   or A = L*D*L**T computed by DSYTRF. DSYTRI2 sets the
   LEADING DIMENSION of the workspace before calling DSYTRI2X
```

```
   that actually computes the inverse.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the NB diagonal matrix D and the multipliers
   used to obtain the factor U or L as computed by DSYTRF.

   On exit, if INFO = 0, the (symmetric) inverse of the original
   matrix.  If UPLO = 'U', the upper triangular part of the
   inverse is formed and the part of A below the diagonal is not
   referenced; if UPLO = 'L' the lower triangular part of the
   inverse is formed and the part of A above the diagonal is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the NB structure of D
   as determined by DSYTRF.

 WORK
   WORK is DOUBLE PRECISION array, dimension (N+NB+1)*(NB+3)

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.
   WORK is size >= (N+NB+1)*(NB+3)
   If LDWORK = -1, then a workspace query is assumed; the routine
   calculates:
   - the optimal size of the WORK array, returns
   this value as the first entry of the WORK array,
   - and no error message related to LDWORK is issued by XERBLA.

 INFO
```

```
INFO is INTEGER
= 0: successful exit
< 0: if INFO = -i, the i-th argument had an illegal value
> 0: if INFO = i, D(i,i) = 0; the matrix is singular and its
     inverse could not be computed.
```

58.22.3 fortran code

— dsytri2.f —

```
*  =====================================================================
      SUBROUTINE DSYTRI2( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            UPPER, LQUERY
      INTEGER            MINSIZE, NBMAX
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DSYTRI, DSYTRI2X
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
```

```
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      LQUERY = ( LWORK.EQ.-1 )
*     Get blocksize
      NBMAX = ILAENV( 1, 'DSYTRF', UPLO, N, -1, -1, -1 )
      IF ( NBMAX .GE. N ) THEN
         MINSIZE = N
      ELSE
         MINSIZE = (N+NBMAX+1)*(NBMAX+3)
      END IF
*
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF (LWORK .LT. MINSIZE .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
*
*     Quick return if possible
*
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYTRI2', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         WORK(1)=MINSIZE
         RETURN
      END IF
      IF( N.EQ.0 )
     $   RETURN

      IF( NBMAX .GE. N ) THEN
         CALL DSYTRI( UPLO, N, A, LDA, IPIV, WORK, INFO )
      ELSE
         CALL DSYTRI2X( UPLO, N, A, LDA, IPIV, WORK, NBMAX, INFO )
      END IF
      RETURN
*
*     End of DSYTRI2
*
      END
```

———-

58.22.4 lisp code

— dsytri2.lisp —

```
(defun dsytri2 (uplo n a lda ipiv work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array double-float (*)) work a)
           (type (f2cl-lib:integer4) info lwork lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a double-float a-%data% a-%offset%)
       (work double-float work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((minsize 0) (nbmax 0) (upper nil) (lquery nil))
      (declare (type f2cl-lib:logical lquery upper)
               (type (f2cl-lib:integer4) nbmax minsize))
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (setf nbmax
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 1 "DSYTRF" uplo n -1 -1 -1)
                (declare (ignore var-0 var-1 var-4 var-5 var-6))
                (when var-2 (setf uplo var-2))
                (when var-3 (setf n var-3))
                ret-val))
      (cond ((>= nbmax n) (setf minsize n))
            (t
             (setf minsize
                     (f2cl-lib:int-mul (f2cl-lib:int-add n nbmax 1)
                                       (f2cl-lib:int-add nbmax 3)))))
      (cond
       ((and (not upper)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
```

```
        (setf info -4))
       ((and (< lwork minsize) (not lquery)) (setf info -7)))
      (cond
       ((/= info 0) (xerbla "DSYTRI2" (f2cl-lib:int-sub info)) (go end_label))
       (lquery
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 minsize) 'double-float))
        (go end_label)))
      (if (= n 0)
          (go end_label))
      (cond
       ((>= nbmax n)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dsytri uplo n a lda ipiv work info)
          (declare (ignore var-2 var-4 var-5))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-6 (setf info var-6))))
       (t
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (dsytri2x uplo n a lda ipiv work nbmax info)
          (declare (ignore var-2 var-4 var-5))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-6 (setf nbmax var-6))
          (when var-7 (setf info var-7)))))
      (go end_label)
     end_label
      (return (values uplo n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsytri2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
```

```
                                        fortran-to-lisp::lda nil nil nil
                                        fortran-to-lisp::info)
                                      :calls 'nil)))
```

———-

58.23 dsytri2x

58.23.1 Axiom unit tests

— **dsytri2x.input** —

```
)set break resume
)sys rm -f dsytri2x.output
)spool dsytri2x.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.23.2 Axiom help page

— **dsytri2x.help** —

```
======================================================================
dsytri2x examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsytri2x (UPLO, N, A, LDA, IPIV, WORK, NB, INFO)
           DSYTRI2X

Function/Subroutine Documentation
```

```
   subroutine dsytri2x (character UPLO, integer N, double precision,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       double precision, dimension( n+nb+1,* ) WORK, integer NB, integer INFO)
       DSYTRI2X

Purpose:
   DSYTRI2X computes the inverse of a real symmetric indefinite matrix
   A using the factorization A = U*D*U**T or A = L*D*L**T computed by DSYTRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the NNB diagonal matrix D and the multipliers
   used to obtain the factor U or L as computed by DSYTRF.

   On exit, if INFO = 0, the (symmetric) inverse of the original
   matrix.  If UPLO = 'U', the upper triangular part of the
   inverse is formed and the part of A below the diagonal is not
   referenced; if UPLO = 'L' the lower triangular part of the
   inverse is formed and the part of A above the diagonal is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the NNB structure of D
   as determined by DSYTRF.

 WORK
   WORK is DOUBLE PRECISION array, dimension (N+NNB+1,NNB+3)

 NB
   NB is INTEGER
   Block size
```

```
 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value
   > 0: if INFO = i, D(i,i) = 0; the matrix is singular and its
        inverse could not be computed.
```

58.23.3 fortran code

— dsytri2x.f —

```
*  =====================================================================
      SUBROUTINE DSYTRI2X( UPLO, N, A, LDA, IPIV, WORK, NB, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N, NB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), WORK( N+NB+1,* )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, IINFO, IP, K, CUT, NNB
      INTEGER            COUNT
      INTEGER            J, U11, INVD

      DOUBLE PRECISION   AK, AKKP1, AKP1, D, T
      DOUBLE PRECISION   U01_I_J, U01_IP1_J
      DOUBLE PRECISION   U11_I_J, U11_IP1_J
*     ..
*     .. External Functions ..
```

```
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DSYCONV, XERBLA, DTRTRI
      EXTERNAL           DGEMM, DTRMM, DSYSWAPR
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
*
*     Quick return if possible
*
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYTRI2X', -INFO )
         RETURN
      END IF
      IF( N.EQ.0 )
     $   RETURN
*
*     Convert A
*     Workspace got Non-diag elements of D
*
      CALL DSYCONV( UPLO, 'C', N, A, LDA, IPIV, WORK, IINFO )
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
         DO INFO = N, 1, -1
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
         END DO
      ELSE
```

```
*
*        Lower triangular storage: examine D from top to bottom.
*
         DO INFO = 1, N
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
         END DO
      END IF
      INFO = 0
*
*  Splitting Workspace
*     U01 is a block (N,NB+1)
*     The first element of U01 is in WORK(1,1)
*     U11 is a block (NB+1,NB+1)
*     The first element of U11 is in WORK(N+1,1)
      U11 = N
*     INVD is a block (N,2)
*     The first element of INVD is in WORK(1,INVD)
      INVD = NB+2

      IF( UPPER ) THEN
*
*        invA = P * inv(U**T)*inv(D)*inv(U)*P**T.
*
        CALL DTRTRI( UPLO, 'U', N, A, LDA, INFO )
*
*       inv(D) and inv(D)*inv(U)
*
        K=1
        DO WHILE ( K .LE. N )
         IF( IPIV( K ).GT.0 ) THEN
*           1 x 1 diagonal NNB
             WORK(K,INVD) = ONE /  A( K, K )
             WORK(K,INVD+1) = 0
            K=K+1
         ELSE
*           2 x 2 diagonal NNB
             T = WORK(K+1,1)
             AK = A( K, K ) / T
             AKP1 = A( K+1, K+1 ) / T
             AKKP1 = WORK(K+1,1)  / T
             D = T*( AK*AKP1-ONE )
             WORK(K,INVD) = AKP1 / D
             WORK(K+1,INVD+1) = AK / D
             WORK(K,INVD+1) = -AKKP1 / D
             WORK(K+1,INVD) = -AKKP1 / D
            K=K+2
         END IF
        END DO
*
```

```
*       inv(U**T) = (inv(U))**T
*
*       inv(U**T)*inv(D)*inv(U)
*
        CUT=N
        DO WHILE (CUT .GT. 0)
           NNB=NB
           IF (CUT .LE. NNB) THEN
              NNB=CUT
           ELSE
              COUNT = 0
*             count negative elements,
              DO I=CUT+1-NNB,CUT
                  IF (IPIV(I) .LT. 0) COUNT=COUNT+1
              END DO
*             need a even number for a clear cut
              IF (MOD(COUNT,2) .EQ. 1) NNB=NNB+1
           END IF

           CUT=CUT-NNB
*
*          U01 Block
*
           DO I=1,CUT
             DO J=1,NNB
              WORK(I,J)=A(I,CUT+J)
             END DO
           END DO
*
*          U11 Block
*
           DO I=1,NNB
             WORK(U11+I,I)=ONE
             DO J=1,I-1
                WORK(U11+I,J)=ZERO
             END DO
             DO J=I+1,NNB
                WORK(U11+I,J)=A(CUT+I,CUT+J)
             END DO
           END DO
*
*          invD*U01
*
           I=1
           DO WHILE (I .LE. CUT)
             IF (IPIV(I) > 0) THEN
                DO J=1,NNB
                    WORK(I,J)=WORK(I,INVD)*WORK(I,J)
                END DO
                I=I+1
```

```
            ELSE
               DO J=1,NNB
                  U01_I_J = WORK(I,J)
                  U01_IP1_J = WORK(I+1,J)
                  WORK(I,J)=WORK(I,INVD)*U01_I_J+
     $                     WORK(I,INVD+1)*U01_IP1_J
                  WORK(I+1,J)=WORK(I+1,INVD)*U01_I_J+
     $                     WORK(I+1,INVD+1)*U01_IP1_J
               END DO
               I=I+2
            END IF
          END DO
*
*       invD1*U11
*
          I=1
          DO WHILE (I .LE. NNB)
            IF (IPIV(CUT+I) > 0) THEN
               DO J=I,NNB
                   WORK(U11+I,J)=WORK(CUT+I,INVD)*WORK(U11+I,J)
               END DO
               I=I+1
            ELSE
               DO J=I,NNB
                  U11_I_J = WORK(U11+I,J)
                  U11_IP1_J = WORK(U11+I+1,J)
               WORK(U11+I,J)=WORK(CUT+I,INVD)*WORK(U11+I,J) +
     $                     WORK(CUT+I,INVD+1)*WORK(U11+I+1,J)
               WORK(U11+I+1,J)=WORK(CUT+I+1,INVD)*U11_I_J+
     $                     WORK(CUT+I+1,INVD+1)*U11_IP1_J
               END DO
               I=I+2
            END IF
          END DO
*
*       U11**T*invD1*U11->U11
*
        CALL DTRMM('L','U','T','U',NNB, NNB,
     $             ONE,A(CUT+1,CUT+1),LDA,WORK(U11+1,1),N+NB+1)
*
         DO I=1,NNB
            DO J=I,NNB
              A(CUT+I,CUT+J)=WORK(U11+I,J)
            END DO
         END DO
*
*         U01**T*invD*U01->A(CUT+I,CUT+J)
*
         CALL DGEMM('T','N',NNB,NNB,CUT,ONE,A(1,CUT+1),LDA,
     $              WORK,N+NB+1, ZERO, WORK(U11+1,1), N+NB+1)
```

```
*
*        U11 =  U11**T*invD1*U11 + U01**T*invD*U01
*
         DO I=1,NNB
            DO J=I,NNB
              A(CUT+I,CUT+J)=A(CUT+I,CUT+J)+WORK(U11+I,J)
            END DO
         END DO
*
*        U01 =  U00**T*invD0*U01
*
         CALL DTRMM('L',UPLO,'T','U',CUT, NNB,
     $             ONE,A,LDA,WORK,N+NB+1)

*
*        Update U01
*
         DO I=1,CUT
           DO J=1,NNB
            A(I,CUT+J)=WORK(I,J)
           END DO
         END DO
*
*      Next Block
*
       END DO
*
*        Apply PERMUTATIONS P and P**T: P * inv(U**T)*inv(D)*inv(U) *P**T
*
            I=1
            DO WHILE ( I .LE. N )
               IF( IPIV(I) .GT. 0 ) THEN
                  IP=IPIV(I)
                 IF (I .LT. IP) CALL DSYSWAPR( UPLO, N, A, LDA, I ,IP )
                 IF (I .GT. IP) CALL DSYSWAPR( UPLO, N, A, LDA, IP ,I )
               ELSE
                 IP=-IPIV(I)
                 I=I+1
                 IF ( (I-1) .LT. IP)
     $                  CALL DSYSWAPR( UPLO, N, A, LDA, I-1 ,IP )
                 IF ( (I-1) .GT. IP)
     $                  CALL DSYSWAPR( UPLO, N, A, LDA, IP ,I-1 )
              ENDIF
               I=I+1
            END DO
      ELSE
*
*        LOWER...
*
```

```
*        invA = P * inv(U**T)*inv(D)*inv(U)*P**T.
*
         CALL DTRTRI( UPLO, 'U', N, A, LDA, INFO )
*
*       inv(D) and inv(D)*inv(U)
*
        K=N
        DO WHILE ( K .GE. 1 )
         IF( IPIV( K ).GT.0 ) THEN
*           1 x 1 diagonal NNB
             WORK(K,INVD) = ONE /  A( K, K )
             WORK(K,INVD+1) = 0
            K=K-1
         ELSE
*           2 x 2 diagonal NNB
             T = WORK(K-1,1)
             AK = A( K-1, K-1 ) / T
             AKP1 = A( K, K ) / T
             AKKP1 = WORK(K-1,1) / T
             D = T*( AK*AKP1-ONE )
             WORK(K-1,INVD) = AKP1 / D
             WORK(K,INVD) = AK / D
             WORK(K,INVD+1) = -AKKP1 / D
             WORK(K-1,INVD+1) = -AKKP1 / D
            K=K-2
         END IF
        END DO
*
*       inv(U**T) = (inv(U))**T
*
*       inv(U**T)*inv(D)*inv(U)
*
        CUT=0
        DO WHILE (CUT .LT. N)
           NNB=NB
           IF (CUT + NNB .GT. N) THEN
              NNB=N-CUT
           ELSE
              COUNT = 0
*             count negative elements,
              DO I=CUT+1,CUT+NNB
                  IF (IPIV(I) .LT. 0) COUNT=COUNT+1
              END DO
*             need a even number for a clear cut
              IF (MOD(COUNT,2) .EQ. 1) NNB=NNB+1
           END IF
*     L21 Block
           DO I=1,N-CUT-NNB
             DO J=1,NNB
              WORK(I,J)=A(CUT+NNB+I,CUT+J)
```

```
            END DO
          END DO
*     L11 Block
          DO I=1,NNB
            WORK(U11+I,I)=ONE
            DO J=I+1,NNB
               WORK(U11+I,J)=ZERO
            END DO
            DO J=1,I-1
               WORK(U11+I,J)=A(CUT+I,CUT+J)
            END DO
          END DO
*
*         invD*L21
*
          I=N-CUT-NNB
          DO WHILE (I .GE. 1)
            IF (IPIV(CUT+NNB+I) > 0) THEN
               DO J=1,NNB
                   WORK(I,J)=WORK(CUT+NNB+I,INVD)*WORK(I,J)
               END DO
               I=I-1
            ELSE
               DO J=1,NNB
                  U01_I_J = WORK(I,J)
                  U01_IP1_J = WORK(I-1,J)
                  WORK(I,J)=WORK(CUT+NNB+I,INVD)*U01_I_J+
     $                      WORK(CUT+NNB+I,INVD+1)*U01_IP1_J
                  WORK(I-1,J)=WORK(CUT+NNB+I-1,INVD+1)*U01_I_J+
     $                      WORK(CUT+NNB+I-1,INVD)*U01_IP1_J
               END DO
               I=I-2
            END IF
          END DO
*
*       invD1*L11
*
          I=NNB
          DO WHILE (I .GE. 1)
            IF (IPIV(CUT+I) > 0) THEN
               DO J=1,NNB
                   WORK(U11+I,J)=WORK(CUT+I,INVD)*WORK(U11+I,J)
               END DO
               I=I-1
            ELSE
               DO J=1,NNB
                  U11_I_J = WORK(U11+I,J)
                  U11_IP1_J = WORK(U11+I-1,J)
               WORK(U11+I,J)=WORK(CUT+I,INVD)*WORK(U11+I,J) +
     $                     WORK(CUT+I,INVD+1)*U11_IP1_J
```

```
                WORK(U11+I-1,J)=WORK(CUT+I-1,INVD+1)*U11_I_J+
     $                     WORK(CUT+I-1,INVD)*U11_IP1_J
                END DO
                I=I-2
             END IF
           END DO
*
*       L11**T*invD1*L11->L11
*
        CALL DTRMM('L',UPLO,'T','U',NNB, NNB,
     $             ONE,A(CUT+1,CUT+1),LDA,WORK(U11+1,1),N+NB+1)

*
         DO I=1,NNB
            DO J=1,I
              A(CUT+I,CUT+J)=WORK(U11+I,J)
            END DO
         END DO
*
        IF ( (CUT+NNB) .LT. N ) THEN
*
*         L21**T*invD2*L21->A(CUT+I,CUT+J)
*
         CALL DGEMM('T','N',NNB,NNB,N-NNB-CUT,ONE,A(CUT+NNB+1,CUT+1)
     $             ,LDA,WORK,N+NB+1, ZERO, WORK(U11+1,1), N+NB+1)

*
*        L11 =  L11**T*invD1*L11 + U01**T*invD*U01
*
         DO I=1,NNB
            DO J=1,I
              A(CUT+I,CUT+J)=A(CUT+I,CUT+J)+WORK(U11+I,J)
            END DO
         END DO
*
*        L01 =  L22**T*invD2*L21
*
         CALL DTRMM('L',UPLO,'T','U', N-NNB-CUT, NNB,
     $             ONE,A(CUT+NNB+1,CUT+NNB+1),LDA,WORK,N+NB+1)
*
*      Update L21
*
         DO I=1,N-CUT-NNB
           DO J=1,NNB
              A(CUT+NNB+I,CUT+J)=WORK(I,J)
           END DO
         END DO

       ELSE
*
```

```
*        L11 =  L11**T*invD1*L11
*
         DO I=1,NNB
            DO J=1,I
              A(CUT+I,CUT+J)=WORK(U11+I,J)
            END DO
         END DO
       END IF
*
*      Next Block
*
           CUT=CUT+NNB
       END DO
*
*        Apply PERMUTATIONS P and P**T: P * inv(U**T)*inv(D)*inv(U) *P**T
*
            I=N
            DO WHILE ( I .GE. 1 )
               IF( IPIV(I) .GT. 0 ) THEN
                  IP=IPIV(I)
                 IF (I .LT. IP) CALL DSYSWAPR( UPLO, N, A, LDA, I ,IP  )
                 IF (I .GT. IP) CALL DSYSWAPR( UPLO, N, A, LDA, IP ,I )
               ELSE
                 IP=-IPIV(I)
                 IF ( I .LT. IP) CALL DSYSWAPR( UPLO, N, A, LDA, I ,IP )
                 IF ( I .GT. IP) CALL DSYSWAPR( UPLO, N, A, LDA, IP, I )
                 I=I-1
               ENDIF
               I=I-1
            END DO
      END IF
*
      RETURN
*
*     End of DSYTRI2X
*
      END
```

———-

58.23.4 lisp code

— dsytri2x.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
```

```
          (ignorable one zero))
 (defun dsytri2x (uplo n a lda ipiv work nb info)
   (declare (type (array f2cl-lib:integer4 (*)) ipiv)
            (type (array double-float (*)) work a)
            (type (f2cl-lib:integer4) info nb lda n)
            (type (string *) uplo))
   (f2cl-lib:with-multi-array-data
       ((uplo character uplo-%data% uplo-%offset%)
        (a double-float a-%data% a-%offset%)
        (work double-float work-%data% work-%offset%)
        (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
     (prog ((u11_i_j 0.0d0) (u11_ip1_j 0.0d0) (u01_i_j 0.0d0)
            (u01_ip1_j 0.0d0) (ak 0.0d0) (akkp1 0.0d0) (akp1 0.0d0) (d 0.0d0)
            (t$ 0.0d0) (j 0) (u11 0) (invd 0) (count$ 0) (i 0) (iinfo 0)
            (ip 0) (k 0) (cut 0) (nnb 0) (upper nil))
       (declare
        (type (double-float) u11_i_j u11_ip1_j u01_i_j u01_ip1_j ak akkp1 akp1
         d t$)
        (type (f2cl-lib:integer4) j u11 invd count$ i iinfo ip k cut nnb)
        (type f2cl-lib:logical upper))
       (setf info 0)
       (setf upper
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame uplo "U")
                 (declare (ignore var-1))
                 (when var-0 (setf uplo var-0))
                 ret-val))
       (cond
        ((and (not upper)
              (not
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame uplo "L")
                 (declare (ignore var-1))
                 (when var-0 (setf uplo var-0))
                 ret-val)))
         (setf info -1))
        ((< n 0) (setf info -2))
        ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -4)))
       (cond
        ((/= info 0) (xerbla "DSYTRI2X" (f2cl-lib:int-sub info))
         (go end_label)))
       (if (= n 0)
           (go end_label))
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
           (dsyconv uplo "C" n a lda ipiv work iinfo)
         (declare (ignore var-1 var-3 var-5 var-6))
         (when var-0 (setf uplo var-0))
         (when var-2 (setf n var-2))
         (when var-4 (setf lda var-4))
```

```
  (when var-7 (setf iinfo var-7)))
 (cond
  (upper
   (f2cl-lib:fdo (info n (f2cl-lib:int-add info (f2cl-lib:int-sub 1)))
                 ((> info 1) nil)
     (tagbody
       (if (and
            (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
               0)
            (=
             (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                            a-%offset%)
             zero))
           (go end_label))
      label100000)))
  (t
   (f2cl-lib:fdo (info 1 (f2cl-lib:int-add info 1))
                 ((> info n) nil)
     (tagbody
       (if (and
            (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
               0)
            (=
             (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                            a-%offset%)
             zero))
           (go end_label))
      label100001))))
 (setf info 0)
 (setf u11 n)
 (setf invd (f2cl-lib:int-add nb 2))
 (cond
  (upper
   (tagbody
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
         (dtrtri uplo "U" n a lda info)
       (declare (ignore var-1 var-3))
       (when var-0 (setf uplo var-0))
       (when var-2 (setf n var-2))
       (when var-4 (setf lda var-4))
       (when var-5 (setf info var-5)))
     (setf k 1)
    label100002
     (if (not (<= k n))
         (go label100003))
     (cond
      ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
       (setf (f2cl-lib:fref work-%data% (k invd)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%)
```

```
        (/ one
           (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                          a-%offset%)))
 (setf (f2cl-lib:fref work-%data% (k (f2cl-lib:int-add invd 1))
                      ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                      work-%offset%)
        (coerce (the f2cl-lib:integer4 0) 'double-float))
 (setf k (f2cl-lib:int-add k 1)))
(t
 (setf t$
        (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1) 1)
                       ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                       work-%offset%))
 (setf ak
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         t$))
 (setf akp1
        (/
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-add k 1)
                         (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
 (setf akkp1
        (/
         (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1) 1)
                        ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                        work-%offset%)
         t$))
 (setf d (* t$ (- (* ak akp1) one)))
 (setf (f2cl-lib:fref work-%data% (k invd)
                      ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                      work-%offset%)
        (/ akp1 d))
 (setf (f2cl-lib:fref work-%data%
                      ((f2cl-lib:int-add k 1)
                       (f2cl-lib:int-add invd 1))
                      ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                      work-%offset%)
        (/ ak d))
 (setf (f2cl-lib:fref work-%data% (k (f2cl-lib:int-add invd 1))
                      ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                      work-%offset%)
        (/ (- akkp1) d))
 (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1) invd)
                      ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                      work-%offset%)
        (/ (- akkp1) d))
```

```
    (setf k (f2cl-lib:int-add k 2))))
  (go label100002)
 label100003
  (setf cut n)
 label100004
  (if (not (> cut 0))
      (go label100005))
  (setf nnb nb)
  (cond ((<= cut nnb) (setf nnb cut))
        (t (setf count$ 0)
         (f2cl-lib:fdo (i
                        (f2cl-lib:int-add cut 1
                                          (f2cl-lib:int-sub nnb))
                        (f2cl-lib:int-add i 1))
                       ((> i cut) nil)
           (tagbody
             (if (<
                  (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                                 ipiv-%offset%)
                  0)
                 (setf count$ (f2cl-lib:int-add count$ 1)))
            label100006))
         (if (= (mod count$ 2) 1)
             (setf nnb (f2cl-lib:int-add nnb 1)))))
  (setf cut (f2cl-lib:int-sub cut nnb))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i cut) nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nnb) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i j)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)
                (f2cl-lib:fref a-%data%
                               (i (f2cl-lib:int-add cut j))
                               ((1 lda) (1 *)) a-%offset%))
         label100008))
     label100007))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i nnb) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) i)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
            one)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
```

```
            (setf (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add u11 i) j)
                                 ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                 work-%offset%)
                  zero)
          label100010))
        (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                      ((> j nnb) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add u11 i) j)
                                 ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                 work-%offset%)
                  (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add cut i)
                                  (f2cl-lib:int-add cut j))
                                 ((1 lda) (1 *)) a-%offset%))
          label100011))
      label100009))
  (setf i 1)
 label100012
  (if (not (<= i cut))
      (go label100013))
  (cond
   ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
              (*
               (f2cl-lib:fref work-%data% (i invd)
                              ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                              work-%offset%)
               (f2cl-lib:fref work-%data% (i j)
                              ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                              work-%offset%)))
       label100014))
    (setf i (f2cl-lib:int-add i 1)))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf u01_i_j
              (f2cl-lib:fref work-%data% (i j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%))
        (setf u01_ip1_j
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1) j)
```

```
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%))
        (setf (f2cl-lib:fref work-%data% (i j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
              (+
               (*
                (f2cl-lib:fref work-%data% (i invd)
                               ((1 (f2cl-lib:int-add n nb 1))
                                (1 *))
                               work-%offset%)
                u01_i_j)
               (*
                (f2cl-lib:fref work-%data%
                               (i (f2cl-lib:int-add invd 1))
                               ((1 (f2cl-lib:int-add n nb 1))
                                (1 *))
                               work-%offset%)
                u01_ip1_j)))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1) j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
              (+
               (*
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add i 1) invd)
                               ((1 (f2cl-lib:int-add n nb 1))
                                (1 *))
                               work-%offset%)
                u01_i_j)
               (*
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add i 1)
                                (f2cl-lib:int-add invd 1))
                               ((1 (f2cl-lib:int-add n nb 1))
                                (1 *))
                               work-%offset%)
                u01_ip1_j)))
      label100015))
   (setf i (f2cl-lib:int-add i 2))))
 (go label100012)
label100013
 (setf i 1)
label100016
 (if (not (<= i nnb))
     (go label100017))
 (cond
  ((> (f2cl-lib:fref ipiv ((f2cl-lib:int-add cut i)) ((1 *))) 0)
   (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                 ((> j nnb) nil)
```

```
    (tagbody
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) j)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
              (*
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add cut i) invd)
                              ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                              work-%offset%)
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add u11 i) j)
                              ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                              work-%offset%)))
     label100018))
  (setf i (f2cl-lib:int-add i 1)))
 (t
  (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                ((> j nnb) nil)
    (tagbody
      (setf u11_i_j
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add u11 i) j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%))
      (setf u11_ip1_j
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add u11 i 1) j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) j)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
              (+
               (*
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add cut i) invd)
                               ((1 (f2cl-lib:int-add n nb 1))
                                (1 *))
                               work-%offset%)
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add u11 i) j)
                               ((1 (f2cl-lib:int-add n nb 1))
                                (1 *))
                               work-%offset%))
               (*
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add cut i)
                                (f2cl-lib:int-add invd 1))
                               ((1 (f2cl-lib:int-add n nb 1))
                                (1 *))
```

```
                                    work-%offset%)
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add u11 i 1) j)
                                    ((1 (f2cl-lib:int-add n nb 1))
                                     (1 *))
                                    work-%offset%))))
          (setf (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add u11 i 1) j)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)
                (+
                 (*
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add cut i 1) invd)
                                 ((1 (f2cl-lib:int-add n nb 1))
                                  (1 *))
                                 work-%offset%)
                  u11_i_j)
                 (*
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add cut i 1)
                                  (f2cl-lib:int-add invd 1))
                                 ((1 (f2cl-lib:int-add n nb 1))
                                  (1 *))
                                 work-%offset%)
                  u11_ip1_j)))
        label100019))
      (setf i (f2cl-lib:int-add i 2))))
    (go label100016)
   label100017
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (dtrmm "L" "U" "T" "U" nnb nnb one
         (f2cl-lib:array-slice a-%data% double-float
                               ((+ cut 1) (f2cl-lib:int-add cut 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice work-%data% double-float ((+ u11 1) 1)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)
         (f2cl-lib:int-add n nb 1))
      (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9 var-10))
      (when var-4 (setf nnb var-4))
      (when var-5 (setf nnb var-5))
      (when var-6 (setf one var-6))
      (when var-8 (setf lda var-8)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i nnb) nil)
      (tagbody
```

```
    (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data%
                             ((f2cl-lib:int-add cut i)
                              (f2cl-lib:int-add cut j))
                             ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add u11 i) j)
                               ((1 (f2cl-lib:int-add n nb 1))
                                (1 *))
                               work-%offset%))
       label100021))
   label100020))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dgemm "T" "N" nnb nnb cut one
     (f2cl-lib:array-slice a-%data% double-float
                           (1 (f2cl-lib:int-add cut 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda work (f2cl-lib:int-add n nb 1) zero
     (f2cl-lib:array-slice work-%data% double-float ((+ u11 1) 1)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
     (f2cl-lib:int-add n nb 1))
  (declare (ignore var-0 var-1 var-6 var-8 var-9 var-11 var-12))
  (when var-2 (setf nnb var-2))
  (when var-3 (setf nnb var-3))
  (when var-4 (setf cut var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf zero var-10)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nnb) nil)
  (tagbody
    (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data%
                             ((f2cl-lib:int-add cut i)
                              (f2cl-lib:int-add cut j))
                             ((1 lda) (1 *)) a-%offset%)
              (+
               (f2cl-lib:fref a-%data%
                              ((f2cl-lib:int-add cut i)
                               (f2cl-lib:int-add cut j))
                              ((1 lda) (1 *)) a-%offset%)
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add u11 i) j)
```

```
                                    ((1 (f2cl-lib:int-add n nb 1))
                                     (1 *))
                                    work-%offset%)))
          label100023))
      label100022))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dtrmm "L" uplo "T" "U" cut nnb one a lda work
       (f2cl-lib:int-add n nb 1))
    (declare (ignore var-0 var-2 var-3 var-7 var-9 var-10))
    (when var-1 (setf uplo var-1))
    (when var-4 (setf cut var-4))
    (when var-5 (setf nnb var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i cut) nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nnb) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add cut j))
                                ((1 lda) (1 *)) a-%offset%)
                  (f2cl-lib:fref work-%data% (i j)
                                  ((1 (f2cl-lib:int-add n nb 1))
                                   (1 *))
                                  work-%offset%))
          label100025))
      label100024))
  (go label100004)
 label100005
  (setf i 1)
 label100026
  (if (not (<= i n))
      (go label100027))
  (cond
   ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
    (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
    (if (< i ip)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (dsyswapr uplo n a lda i ip)
          (declare (ignore var-2))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-4 (setf i var-4))
          (when var-5 (setf ip var-5))))
    (if (> i ip)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
```

```
                (dsyswapr uplo n a lda ip i)
              (declare (ignore var-2))
              (when var-0 (setf uplo var-0))
              (when var-1 (setf n var-1))
              (when var-3 (setf lda var-3))
              (when var-4 (setf ip var-4))
              (when var-5 (setf i var-5)))))
      (t
       (setf ip
               (f2cl-lib:int-sub
                (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)))
       (setf i (f2cl-lib:int-add i 1))
       (if (< (f2cl-lib:int-sub i 1) ip)
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
               (dsyswapr uplo n a lda (f2cl-lib:int-sub i 1) ip)
             (declare (ignore var-2 var-4))
             (when var-0 (setf uplo var-0))
             (when var-1 (setf n var-1))
             (when var-3 (setf lda var-3))
             (when var-5 (setf ip var-5))))
       (if (> (f2cl-lib:int-sub i 1) ip)
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
               (dsyswapr uplo n a lda ip (f2cl-lib:int-sub i 1))
             (declare (ignore var-2 var-5))
             (when var-0 (setf uplo var-0))
             (when var-1 (setf n var-1))
             (when var-3 (setf lda var-3))
             (when var-4 (setf ip var-4))))))
    (setf i (f2cl-lib:int-add i 1))
    (go label100026)
   label100027))
  (t
   (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (dtrtri uplo "U" n a lda info)
      (declare (ignore var-1 var-3))
      (when var-0 (setf uplo var-0))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-5 (setf info var-5)))
    (setf k n)
   label100028
    (if (not (>= k 1))
        (go label100029))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf (f2cl-lib:fref work-%data% (k invd)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
              (/ one
```

```
            (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                           a-%offset%)))
 (setf (f2cl-lib:fref work-%data% (k (f2cl-lib:int-add invd 1))
                      ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                      work-%offset%)
         (coerce (the f2cl-lib:integer4 0) 'double-float))
 (setf k (f2cl-lib:int-sub k 1)))
(t
 (setf t$
         (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1) 1)
                        ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                        work-%offset%))
 (setf ak
         (/
          (f2cl-lib:fref a-%data%
                         ((f2cl-lib:int-sub k 1)
                          (f2cl-lib:int-sub k 1))
                         ((1 lda) (1 *)) a-%offset%)
          t$))
 (setf akp1
         (/
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%)
          t$))
 (setf akkp1
         (/
          (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1) 1)
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
          t$))
 (setf d (* t$ (- (* ak akp1) one)))
 (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1) invd)
                      ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                      work-%offset%)
         (/ akp1 d))
 (setf (f2cl-lib:fref work-%data% (k invd)
                      ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                      work-%offset%)
         (/ ak d))
 (setf (f2cl-lib:fref work-%data% (k (f2cl-lib:int-add invd 1))
                      ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                      work-%offset%)
         (/ (- akkp1) d))
 (setf (f2cl-lib:fref work-%data%
                      ((f2cl-lib:int-sub k 1)
                       (f2cl-lib:int-add invd 1))
                      ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                      work-%offset%)
         (/ (- akkp1) d))
 (setf k (f2cl-lib:int-sub k 2))))
```

```
 (go label100028)
label100029
 (setf cut 0)
label100030
 (if (not (< cut n))
     (go label100031))
 (setf nnb nb)
 (cond
  ((> (f2cl-lib:int-add cut nnb) n)
   (setf nnb (f2cl-lib:int-sub n cut)))
  (t (setf count$ 0)
   (f2cl-lib:fdo (i (f2cl-lib:int-add cut 1) (f2cl-lib:int-add i 1))
                 ((> i (f2cl-lib:int-add cut nnb)) nil)
     (tagbody
       (if (< (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
              0)
           (setf count$ (f2cl-lib:int-add count$ 1)))
      label100032))
   (if (= (mod count$ 2) 1)
       (setf nnb (f2cl-lib:int-add nnb 1)))))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i
                   (f2cl-lib:int-add n (f2cl-lib:int-sub cut)
                                     (f2cl-lib:int-sub nnb)))
                nil)
   (tagbody
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j nnb) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i j)
                              ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                              work-%offset%)
                 (f2cl-lib:fref a-%data%
                                ((f2cl-lib:int-add cut nnb i)
                                 (f2cl-lib:int-add cut j))
                                ((1 lda) (1 *)) a-%offset%))
       label100034))
   label100033))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i nnb) nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) i)
                          ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                          work-%offset%)
             one)
     (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                   ((> j nnb) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add u11 i) j)
```

```
                                  ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                  work-%offset%)
                     zero)
            label100036))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add u11 i) j)
                                  ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                  work-%offset%)
                     (f2cl-lib:fref a-%data%
                                    ((f2cl-lib:int-add cut i)
                                     (f2cl-lib:int-add cut j))
                                    ((1 lda) (1 *)) a-%offset%))
            label100037))
      label100035))
  (setf i (f2cl-lib:int-sub n cut nnb))
 label100038
  (if (not (>= i 1))
      (go label100039))
  (cond
   ((> (f2cl-lib:fref ipiv ((f2cl-lib:int-add cut nnb i)) ((1 *))) 0)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i j)
                              ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                              work-%offset%)
                 (*
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add cut nnb i) invd)
                                 ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                 work-%offset%)
                  (f2cl-lib:fref work-%data% (i j)
                                 ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                 work-%offset%)))
       label100040))
    (setf i (f2cl-lib:int-sub i 1)))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf u01_i_j
                (f2cl-lib:fref work-%data% (i j)
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%))
        (setf u01_ip1_j
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub i 1) j)
```

```
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%))
        (setf (f2cl-lib:fref work-%data% (i j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
              (+
               (*
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add cut nnb i) invd)
                               ((1 (f2cl-lib:int-add n nb 1))
                                (1 *))
                               work-%offset%)
                u01_i_j)
               (*
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add cut nnb i)
                                (f2cl-lib:int-add invd 1))
                               ((1 (f2cl-lib:int-add n nb 1))
                                (1 *))
                               work-%offset%)
                u01_ip1_j)))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub i 1) j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
              (+
               (*
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add cut nnb i) 1)
                                (f2cl-lib:int-add invd 1))
                               ((1 (f2cl-lib:int-add n nb 1))
                                (1 *))
                               work-%offset%)
                u01_i_j)
               (*
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add cut nnb i) 1)
                                invd)
                               ((1 (f2cl-lib:int-add n nb 1))
                                (1 *))
                               work-%offset%)
                u01_ip1_j)))
      label100041))
   (setf i (f2cl-lib:int-sub i 2))))
 (go label100038)
label100039
 (setf i nnb)
label100042
 (if (not (>= i 1))
```

```
    (go label100043))
(cond
 ((> (f2cl-lib:fref ipiv ((f2cl-lib:int-add cut i)) ((1 *))) 0)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nnb) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) j)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
            (*
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add cut i) invd)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%)
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add u11 i) j)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%)))
     label100044))
  (setf i (f2cl-lib:int-sub i 1)))
 (t
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nnb) nil)
    (tagbody
      (setf u11_i_j
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add u11 i) j)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%))
      (setf u11_ip1_j
            (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add u11 i) 1)
                            j)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) j)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
            (+
             (*
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add cut i) invd)
                             ((1 (f2cl-lib:int-add n nb 1))
                              (1 *))
                             work-%offset%)
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add u11 i) j)
                             ((1 (f2cl-lib:int-add n nb 1))
                              (1 *))
```

```
                                    work-%offset%))
                         (*
                          (f2cl-lib:fref work-%data%
                                         ((f2cl-lib:int-add cut i)
                                          (f2cl-lib:int-add invd 1))
                                         ((1 (f2cl-lib:int-add n nb 1))
                                          (1 *))
                                         work-%offset%)
                          u11_ip1_j)))
                (setf (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-sub
                                       (f2cl-lib:int-add u11 i) 1)
                                      j)
                                     ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                     work-%offset%)
                        (+
                         (*
                          (f2cl-lib:fref work-%data%
                                         ((f2cl-lib:int-sub
                                           (f2cl-lib:int-add cut i) 1)
                                          (f2cl-lib:int-add invd 1))
                                         ((1 (f2cl-lib:int-add n nb 1))
                                          (1 *))
                                         work-%offset%)
                          u11_i_j)
                         (*
                          (f2cl-lib:fref work-%data%
                                         ((f2cl-lib:int-sub
                                           (f2cl-lib:int-add cut i) 1)
                                          invd)
                                         ((1 (f2cl-lib:int-add n nb 1))
                                          (1 *))
                                         work-%offset%)
                          u11_ip1_j)))
               label100045))
             (setf i (f2cl-lib:int-sub i 2))))
           (go label100042)
          label100043
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                var-10)
               (dtrmm "L" uplo "T" "U" nnb nnb one
                (f2cl-lib:array-slice a-%data% double-float
                                      ((+ cut 1) (f2cl-lib:int-add cut 1))
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice work-%data% double-float ((+ u11 1) 1)
                                      ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                      work-%offset%)
                (f2cl-lib:int-add n nb 1))
```

```
    (declare (ignore var-0 var-2 var-3 var-7 var-9 var-10))
    (when var-1 (setf uplo var-1))
    (when var-4 (setf nnb var-4))
    (when var-5 (setf nnb var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i nnb) nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j i) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data%
                               ((f2cl-lib:int-add cut i)
                                (f2cl-lib:int-add cut j))
                               ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add u11 i) j)
                                 ((1 (f2cl-lib:int-add n nb 1))
                                  (1 *))
                                 work-%offset%))
        label100047))
     label100046))
  (cond
   ((< (f2cl-lib:int-add cut nnb) n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (dgemm "T" "N" nnb nnb (f2cl-lib:int-sub n nnb cut) one
         (f2cl-lib:array-slice a-%data% double-float
                               ((+ cut nnb 1)
                                (f2cl-lib:int-add cut 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda work (f2cl-lib:int-add n nb 1) zero
         (f2cl-lib:array-slice work-%data% double-float ((+ u11 1) 1)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)
         (f2cl-lib:int-add n nb 1))
      (declare
       (ignore var-0 var-1 var-4 var-6 var-8 var-9 var-11 var-12))
      (when var-2 (setf nnb var-2))
      (when var-3 (setf nnb var-3))
      (when var-5 (setf one var-5))
      (when var-7 (setf lda var-7))
      (when var-10 (setf zero var-10)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i nnb) nil)
      (tagbody
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j i) nil)
```

```
      (tagbody
        (setf (f2cl-lib:fref a-%data%
                             ((f2cl-lib:int-add cut i)
                              (f2cl-lib:int-add cut j))
                             ((1 lda) (1 *)) a-%offset%)
              (+
               (f2cl-lib:fref a-%data%
                              ((f2cl-lib:int-add cut i)
                               (f2cl-lib:int-add cut j))
                              ((1 lda) (1 *)) a-%offset%)
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add u11 i) j)
                              ((1 (f2cl-lib:int-add n nb 1))
                               (1 *))
                              work-%offset%)))
       label100049))
   label100048))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "L" uplo "T" "U" (f2cl-lib:int-sub n nnb cut) nnb one
     (f2cl-lib:array-slice a-%data% double-float
                           ((+ cut nnb 1)
                            (f2cl-lib:int-add cut nnb 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda work (f2cl-lib:int-add n nb 1))
  (declare (ignore var-0 var-2 var-3 var-4 var-7 var-9 var-10))
  (when var-1 (setf uplo var-1))
  (when var-5 (setf nnb var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i
                  (f2cl-lib:int-add n (f2cl-lib:int-sub cut)
                                    (f2cl-lib:int-sub nnb)))
               nil)
  (tagbody
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data%
                             ((f2cl-lib:int-add cut nnb i)
                              (f2cl-lib:int-add cut j))
                             ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref work-%data% (i j)
                             ((1 (f2cl-lib:int-add n nb 1))
                              (1 *))
                             work-%offset%))
       label100051))
   label100050)))
```

```
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i nnb) nil)
     (tagbody
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j i) nil)
         (tagbody
           (setf (f2cl-lib:fref a-%data%
                                ((f2cl-lib:int-add cut i)
                                 (f2cl-lib:int-add cut j))
                                ((1 lda) (1 *)) a-%offset%)
                 (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add u11 i) j)
                                  ((1 (f2cl-lib:int-add n nb 1))
                                   (1 *))
                                  work-%offset%))
          label100053))
      label100052))))
 (setf cut (f2cl-lib:int-add cut nnb))
 (go label100030)
label100031
 (setf i n)
label100054
 (if (not (>= i 1))
     (go label100055))
 (cond
  ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
   (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
   (if (< i ip)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
           (dsyswapr uplo n a lda i ip)
         (declare (ignore var-2))
         (when var-0 (setf uplo var-0))
         (when var-1 (setf n var-1))
         (when var-3 (setf lda var-3))
         (when var-4 (setf i var-4))
         (when var-5 (setf ip var-5))))
   (if (> i ip)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
           (dsyswapr uplo n a lda ip i)
         (declare (ignore var-2))
         (when var-0 (setf uplo var-0))
         (when var-1 (setf n var-1))
         (when var-3 (setf lda var-3))
         (when var-4 (setf ip var-4))
         (when var-5 (setf i var-5)))))
  (t
   (setf ip
           (f2cl-lib:int-sub
            (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)))
```

```
              (if (< i ip)
                  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                      (dsyswapr uplo n a lda i ip)
                    (declare (ignore var-2))
                    (when var-0 (setf uplo var-0))
                    (when var-1 (setf n var-1))
                    (when var-3 (setf lda var-3))
                    (when var-4 (setf i var-4))
                    (when var-5 (setf ip var-5))))
              (if (> i ip)
                  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                      (dsyswapr uplo n a lda ip i)
                    (declare (ignore var-2))
                    (when var-0 (setf uplo var-0))
                    (when var-1 (setf n var-1))
                    (when var-3 (setf lda var-3))
                    (when var-4 (setf ip var-4))
                    (when var-5 (setf i var-5))))
              (setf i (f2cl-lib:int-sub i 1))))
            (setf i (f2cl-lib:int-sub i 1))
            (go label100054)
           label100055)))
        (go end_label)
       end_label
        (return (values uplo n nil lda nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsytri2x
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

58.24 dsytri_rook

58.24.1 Axiom unit tests

— **dsytrirook.input** —

```
)set break resume
)sys rm -f dsytrirook.output
)spool dsytrirook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.24.2 Axiom help page

— **dsytri_rook.help** —

```
=====================================================================
dsytri_rook examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsytri_rook (UPLO, N, A, LDA, IPIV, WORK, INFO)
           DSYTRI_ROOK

Function/Subroutine Documentation
   subroutine dsytri_rook (character UPLO, integer N, double precision,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       double precision, dimension( * ) WORK, integer INFO)
       DSYTRI_ROOK

Purpose:
```

```
  DSYTRI_ROOK computes the inverse of a real symmetric
  matrix A using the factorization A = U*D*U**T or A = L*D*L**T
  computed by DSYTRF_ROOK.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the block diagonal matrix D and the multipliers
   used to obtain the factor U or L as computed by DSYTRF_ROOK.

   On exit, if INFO = 0, the (symmetric) inverse of the original
   matrix.  If UPLO = 'U', the upper triangular part of the
   inverse is formed and the part of A below the diagonal is not
   referenced; if UPLO = 'L' the lower triangular part of the
   inverse is formed and the part of A above the diagonal is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by DSYTRF_ROOK.

 WORK
   WORK is DOUBLE PRECISION array, dimension (N)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value
   > 0: if INFO = i, D(i,i) = 0; the matrix is singular and its
        inverse could not be computed.
```

58.24.3 fortran code

— dsytri_rook.f —

```
*  =====================================================================
      SUBROUTINE DSYTRI_ROOK( UPLO, N, A, LDA, IPIV, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            K, KP, KSTEP
      DOUBLE PRECISION   AK, AKKP1, AKP1, D, T, TEMP
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DDOT
      EXTERNAL           LSAME, DDOT
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DSWAP, DSYMV, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
```

```
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYTRI_ROOK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
         DO 10 INFO = N, 1, -1
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
   10    CONTINUE
      ELSE
*
*        Lower triangular storage: examine D from top to bottom.
*
         DO 20 INFO = 1, N
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
   20    CONTINUE
      END IF
      INFO = 0
*
      IF( UPPER ) THEN
*
*        Compute inv(A) from the factorization A = U*D*U**T.
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = 1
   30    CONTINUE
*
*        If K > N, exit from loop.
*
```

```
         IF( K.GT.N )
     $      GO TO 40
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Invert the diagonal block.
*
            A( K, K ) = ONE / A( K, K )
*
*           Compute column K of the inverse.
*
            IF( K.GT.1 ) THEN
               CALL DCOPY( K-1, A( 1, K ), 1, WORK, 1 )
               CALL DSYMV( UPLO, K-1, -ONE, A, LDA, WORK, 1, ZERO,
     $                     A( 1, K ), 1 )
               A( K, K ) = A( K, K ) - DDOT( K-1, WORK, 1, A( 1, K ),
     $                     1 )
            END IF
            KSTEP = 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Invert the diagonal block.
*
            T = ABS( A( K, K+1 ) )
            AK = A( K, K ) / T
            AKP1 = A( K+1, K+1 ) / T
            AKKP1 = A( K, K+1 ) / T
            D = T*( AK*AKP1-ONE )
            A( K, K ) = AKP1 / D
            A( K+1, K+1 ) = AK / D
            A( K, K+1 ) = -AKKP1 / D
*
*           Compute columns K and K+1 of the inverse.
*
            IF( K.GT.1 ) THEN
               CALL DCOPY( K-1, A( 1, K ), 1, WORK, 1 )
               CALL DSYMV( UPLO, K-1, -ONE, A, LDA, WORK, 1, ZERO,
     $                     A( 1, K ), 1 )
               A( K, K ) = A( K, K ) - DDOT( K-1, WORK, 1, A( 1, K ),
     $                     1 )
               A( K, K+1 ) = A( K, K+1 ) -
     $                       DDOT( K-1, A( 1, K ), 1, A( 1, K+1 ), 1 )
               CALL DCOPY( K-1, A( 1, K+1 ), 1, WORK, 1 )
               CALL DSYMV( UPLO, K-1, -ONE, A, LDA, WORK, 1, ZERO,
     $                     A( 1, K+1 ), 1 )
               A( K+1, K+1 ) = A( K+1, K+1 ) -
```

```
     $                        DDOT( K-1, WORK, 1, A( 1, K+1 ), 1 )
            END IF
            KSTEP = 2
         END IF
*
         IF( KSTEP.EQ.1 ) THEN
*
*           Interchange rows and columns K and IPIV(K) in the leading
*           submatrix A(1:k+1,1:k+1)
*
            KP = IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.GT.1 )
     $             CALL DSWAP( KP-1, A( 1, K ), 1, A( 1, KP ), 1 )
               CALL DSWAP( K-KP-1, A( KP+1, K ), 1, A( KP, KP+1 ), LDA )
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
            END IF
         ELSE
*
*           Interchange rows and columns K and K+1 with -IPIV(K) and
*           -IPIV(K+1)in the leading submatrix A(1:k+1,1:k+1)
*
            KP = -IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.GT.1 )
     $            CALL DSWAP( KP-1, A( 1, K ), 1, A( 1, KP ), 1 )
               CALL DSWAP( K-KP-1, A( KP+1, K ), 1, A( KP, KP+1 ), LDA )
*
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
               TEMP = A( K, K+1 )
               A( K, K+1 ) = A( KP, K+1 )
               A( KP, K+1 ) = TEMP
            END IF
*
            K = K + 1
            KP = -IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.GT.1 )
     $            CALL DSWAP( KP-1, A( 1, K ), 1, A( 1, KP ), 1 )
               CALL DSWAP( K-KP-1, A( KP+1, K ), 1, A( KP, KP+1 ), LDA )
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
            END IF
         END IF
*
```

```
         K = K + 1
         GO TO 30
   40    CONTINUE
*
      ELSE
*
*        Compute inv(A) from the factorization A = L*D*L**T.
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = N
   50    CONTINUE
*
*        If K < 1, exit from loop.
*
         IF( K.LT.1 )
     $      GO TO 60
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Invert the diagonal block.
*
            A( K, K ) = ONE / A( K, K )
*
*           Compute column K of the inverse.
*
            IF( K.LT.N ) THEN
               CALL DCOPY( N-K, A( K+1, K ), 1, WORK, 1 )
               CALL DSYMV( UPLO, N-K, -ONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                     ZERO, A( K+1, K ), 1 )
               A( K, K ) = A( K, K ) - DDOT( N-K, WORK, 1, A( K+1, K ),
     $                     1 )
            END IF
            KSTEP = 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Invert the diagonal block.
*
            T = ABS( A( K, K-1 ) )
            AK = A( K-1, K-1 ) / T
            AKP1 = A( K, K ) / T
            AKKP1 = A( K, K-1 ) / T
            D = T*( AK*AKP1-ONE )
            A( K-1, K-1 ) = AKP1 / D
            A( K, K ) = AK / D
```

```
            A( K, K-1 ) = -AKKP1 / D
*
*           Compute columns K-1 and K of the inverse.
*
            IF( K.LT.N ) THEN
               CALL DCOPY( N-K, A( K+1, K ), 1, WORK, 1 )
               CALL DSYMV( UPLO, N-K, -ONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                     ZERO, A( K+1, K ), 1 )
               A( K, K ) = A( K, K ) - DDOT( N-K, WORK, 1, A( K+1, K ),
     $                     1 )
               A( K, K-1 ) = A( K, K-1 ) -
     $                       DDOT( N-K, A( K+1, K ), 1, A( K+1, K-1 ),
     $                       1 )
               CALL DCOPY( N-K, A( K+1, K-1 ), 1, WORK, 1 )
               CALL DSYMV( UPLO, N-K, -ONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                     ZERO, A( K+1, K-1 ), 1 )
               A( K-1, K-1 ) = A( K-1, K-1 ) -
     $                         DDOT( N-K, WORK, 1, A( K+1, K-1 ), 1 )
            END IF
            KSTEP = 2
         END IF
*
         IF( KSTEP.EQ.1 ) THEN
*
*           Interchange rows and columns K and IPIV(K) in the trailing
*           submatrix A(k-1:n,k-1:n)
*
            KP = IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.LT.N )
     $            CALL DSWAP( N-KP, A( KP+1, K ), 1, A( KP+1, KP ), 1 )
               CALL DSWAP( KP-K-1, A( K+1, K ), 1, A( KP, K+1 ), LDA )
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
            END IF
         ELSE
*
*           Interchange rows and columns K and K-1 with -IPIV(K) and
*           -IPIV(K-1) in the trailing submatrix A(k-1:n,k-1:n)
*
            KP = -IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.LT.N )
     $            CALL DSWAP( N-KP, A( KP+1, K ), 1, A( KP+1, KP ), 1 )
               CALL DSWAP( KP-K-1, A( K+1, K ), 1, A( KP, K+1 ), LDA )
*
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
```

```
               TEMP = A( K, K-1 )
               A( K, K-1 ) = A( KP, K-1 )
               A( KP, K-1 ) = TEMP
            END IF
*
            K = K - 1
            KP = -IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.LT.N )
     $            CALL DSWAP( N-KP, A( KP+1, K ), 1, A( KP+1, KP ), 1 )
               CALL DSWAP( KP-K-1, A( K+1, K ), 1, A( KP, K+1 ), LDA )
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
            END IF
         END IF
*
         K = K - 1
         GO TO 50
   60    CONTINUE
      END IF
*
      RETURN
*
*     End of DSYTRI_ROOK
*
      END
```

———-

58.24.4 lisp code

— dsytri_rook.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dsytri_rook (uplo n a lda ipiv work info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array double-float (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (work double-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
```

```
(prog ((ak 0.0d0) (akkp1 0.0d0) (akp1 0.0d0) (d 0.0d0) (t$ 0.0d0)
       (temp 0.0d0) (k 0) (kp 0) (kstep 0) (upper nil))
  (declare (type (double-float) ak akkp1 akp1 d t$ temp)
           (type (f2cl-lib:integer4) k kp kstep)
           (type f2cl-lib:logical upper))
  (setf info 0)
  (setf upper
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
  (cond
   ((and (not upper)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4)))
  (cond
   ((/= info 0) (xerbla "DSYTRI_ROOK" (f2cl-lib:int-sub info))
    (go end_label)))
  (if (= n 0)
      (go end_label))
  (cond
   (upper
    (f2cl-lib:fdo (info n (f2cl-lib:int-add info (f2cl-lib:int-sub 1)))
                  ((> info 1) nil)
      (tagbody
        (if (and
             (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
                0)
             (=
              (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                             a-%offset%)
              zero))
            (go end_label))
      label10)))
   (t
    (f2cl-lib:fdo (info 1 (f2cl-lib:int-add info 1))
                  ((> info n) nil)
      (tagbody
        (if (and
             (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
                0)
```

```
            (=
             (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                            a-%offset%)
             zero))
           (go end_label))
     label20))))
(setf info 0)
(cond
 (upper
  (tagbody
    (setf k 1)
   label30
    (if (> k n)
        (go label40))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
              (/ one
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%)))
      (cond
       ((> k 1)
        (dcopy (f2cl-lib:int-sub k 1)
         (f2cl-lib:array-slice a-%data% double-float (1 k)
                               ((1 lda) (1 *)) a-%offset%)
         1 work 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9)
            (dsymv uplo (f2cl-lib:int-sub k 1) (- one) a lda work 1
             zero
             (f2cl-lib:array-slice a-%data% double-float (1 k)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
          (when var-0 (setf uplo var-0))
          (when var-4 (setf lda var-4))
          (when var-7 (setf zero var-7)))
        (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                (-
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%)
                 (ddot (f2cl-lib:int-sub k 1) work 1
                  (f2cl-lib:array-slice a-%data% double-float (1 k)
                                        ((1 lda) (1 *)) a-%offset%)
                  1)))))
      (setf kstep 1))
     (t
      (setf t$
              (abs
```

```
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)))
(setf ak
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         t$))
(setf akp1
        (/
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-add k 1)
                         (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
(setf akkp1
        (/
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
(setf d (* t$ (- (* ak akp1) one)))
(setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (/ akp1 d))
(setf (f2cl-lib:fref a-%data%
                     ((f2cl-lib:int-add k 1)
                      (f2cl-lib:int-add k 1))
                     ((1 lda) (1 *)) a-%offset%)
        (/ ak d))
(setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                     ((1 lda) (1 *)) a-%offset%)
        (/ (- akkp1) d))
(cond
 ((> k 1)
  (dcopy (f2cl-lib:int-sub k 1)
   (f2cl-lib:array-slice a-%data% double-float (1 k)
                         ((1 lda) (1 *)) a-%offset%)
   1 work 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (dsymv uplo (f2cl-lib:int-sub k 1) (- one) a lda work 1
       zero
       (f2cl-lib:array-slice a-%data% double-float (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
    (when var-0 (setf uplo var-0))
    (when var-4 (setf lda var-4))
    (when var-7 (setf zero var-7)))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
          (-
```

```
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%)
          (ddot (f2cl-lib:int-sub k 1) work 1
           (f2cl-lib:array-slice a-%data% double-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1)))
(setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (-
          (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                         ((1 lda) (1 *)) a-%offset%)
          (ddot (f2cl-lib:int-sub k 1)
           (f2cl-lib:array-slice a-%data% double-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% double-float
                                 (1 (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           1)))
(dcopy (f2cl-lib:int-sub k 1)
 (f2cl-lib:array-slice a-%data% double-float
                       (1 (f2cl-lib:int-add k 1))
                       ((1 lda) (1 *)) a-%offset%)
 1 work 1)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dsymv uplo (f2cl-lib:int-sub k 1) (- one) a lda work 1
     zero
     (f2cl-lib:array-slice a-%data% double-float
                           (1 (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
  (when var-0 (setf uplo var-0))
  (when var-4 (setf lda var-4))
  (when var-7 (setf zero var-7)))
(setf (f2cl-lib:fref a-%data%
                      ((f2cl-lib:int-add k 1)
                       (f2cl-lib:int-add k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (-
          (f2cl-lib:fref a-%data%
                         ((f2cl-lib:int-add k 1)
                          (f2cl-lib:int-add k 1))
                         ((1 lda) (1 *)) a-%offset%)
          (ddot (f2cl-lib:int-sub k 1) work 1
           (f2cl-lib:array-slice a-%data% double-float
                                 (1 (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
```

```
                1)))))
    (setf kstep 2)))
  (cond
   ((= kstep 1)
    (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
    (cond
     ((/= kp k)
      (if (> kp 1)
          (dswap (f2cl-lib:int-sub kp 1)
           (f2cl-lib:array-slice a-%data% double-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% double-float (1 kp)
                                 ((1 lda) (1 *)) a-%offset%)
           1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dswap (f2cl-lib:int-sub k kp 1)
           (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% double-float
                                 (kp (f2cl-lib:int-add kp 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf lda var-4)))
      (setf temp
              (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                             a-%offset%))
      (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                             a-%offset%))
      (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                           a-%offset%)
              temp))))
   (t
    (setf kp
            (f2cl-lib:int-sub
             (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
    (cond
     ((/= kp k)
      (if (> kp 1)
          (dswap (f2cl-lib:int-sub kp 1)
           (f2cl-lib:array-slice a-%data% double-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% double-float (1 kp)
                                 ((1 lda) (1 *)) a-%offset%)
           1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
        (dswap (f2cl-lib:int-sub k kp 1)
         (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) k)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% double-float
                               (kp (f2cl-lib:int-add kp 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-0 var-1 var-2 var-3))
      (when var-4 (setf lda var-4)))
    (setf temp
            (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%)
            temp)
    (setf temp
            (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                         ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-add k 1))
                         ((1 lda) (1 *)) a-%offset%)
            temp)))
  (setf k (f2cl-lib:int-add k 1))
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
  (cond
   ((/= kp k)
    (if (> kp 1)
        (dswap (f2cl-lib:int-sub kp 1)
         (f2cl-lib:array-slice a-%data% double-float (1 k)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% double-float (1 kp)
                               ((1 lda) (1 *)) a-%offset%)
         1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dswap (f2cl-lib:int-sub k kp 1)
         (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) k)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% double-float
                               (kp (f2cl-lib:int-add kp 1))
```

```
                                        ((1 lda) (1 *)) a-%offset%)
                  lda)
                (declare (ignore var-0 var-1 var-2 var-3))
                (when var-4 (setf lda var-4)))
              (setf temp
                      (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                      (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                                    a-%offset%)
                      temp)))))
        (setf k (f2cl-lib:int-add k 1))
        (go label30)
       label40))
      (t
       (tagbody
        (setf k n)
       label50
        (if (< k 1)
            (go label60))
        (cond
         ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
          (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                  (/ one
                     (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                     a-%offset%)))
          (cond
           ((< k n)
            (dcopy (f2cl-lib:int-sub n k)
             (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                                    ((1 lda) (1 *)) a-%offset%)
             1 work 1)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9)
                (dsymv uplo (f2cl-lib:int-sub n k) (- one)
                 (f2cl-lib:array-slice a-%data% double-float
                                        ((+ k 1) (f2cl-lib:int-add k 1))
                                        ((1 lda) (1 *)) a-%offset%)
                 lda work 1 zero
                 (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                                        ((1 lda) (1 *)) a-%offset%)
                 1)
              (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
              (when var-0 (setf uplo var-0))
              (when var-4 (setf lda var-4))
              (when var-7 (setf zero var-7)))
            (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
```

```
          (-
           (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                          a-%offset%)
           (ddot (f2cl-lib:int-sub n k) work 1
            (f2cl-lib:array-slice a-%data% double-float
                                  ((+ k 1) k) ((1 lda) (1 *))
                                  a-%offset%)
            1)))))
 (setf kstep 1))
(t
 (setf t$
        (abs
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)))
 (setf ak
        (/
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-sub k 1)
                         (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
 (setf akp1
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         t$))
 (setf akkp1
        (/
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
 (setf d (* t$ (- (* ak akp1) one)))
 (setf (f2cl-lib:fref a-%data%
                      ((f2cl-lib:int-sub k 1)
                       (f2cl-lib:int-sub k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (/ akp1 d))
 (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (/ ak d))
 (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (/ (- akkp1) d))
 (cond
  ((< k n)
   (dcopy (f2cl-lib:int-sub n k)
    (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                          ((1 lda) (1 *)) a-%offset%)
    1 work 1)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
     var-9)
    (dsymv uplo (f2cl-lib:int-sub n k) (- one)
     (f2cl-lib:array-slice a-%data% double-float
                           ((+ k 1) (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda work 1 zero
     (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                           ((1 lda) (1 *)) a-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
  (when var-0 (setf uplo var-0))
  (when var-4 (setf lda var-4))
  (when var-7 (setf zero var-7)))
(setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         (ddot (f2cl-lib:int-sub n k) work 1
          (f2cl-lib:array-slice a-%data% double-float
                                ((+ k 1) k) ((1 lda) (1 *))
                                a-%offset%)
          1)))
(setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                     ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         (ddot (f2cl-lib:int-sub n k)
          (f2cl-lib:array-slice a-%data% double-float
                                ((+ k 1) k) ((1 lda) (1 *))
                                a-%offset%)
          1
          (f2cl-lib:array-slice a-%data% double-float
                                ((+ k 1)
                                 (f2cl-lib:int-sub k 1))
                                ((1 lda) (1 *)) a-%offset%)
          1)))
(dcopy (f2cl-lib:int-sub n k)
 (f2cl-lib:array-slice a-%data% double-float
                       ((+ k 1) (f2cl-lib:int-sub k 1))
                       ((1 lda) (1 *)) a-%offset%)
 1 work 1)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dsymv uplo (f2cl-lib:int-sub n k) (- one)
     (f2cl-lib:array-slice a-%data% double-float
                           ((+ k 1) (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda work 1 zero
```

```
            (f2cl-lib:array-slice a-%data% double-float
                                  ((+ k 1) (f2cl-lib:int-sub k 1))
                                  ((1 lda) (1 *)) a-%offset%)
            1)
        (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
        (when var-0 (setf uplo var-0))
        (when var-4 (setf lda var-4))
        (when var-7 (setf zero var-7)))
      (setf (f2cl-lib:fref a-%data%
                           ((f2cl-lib:int-sub k 1)
                            (f2cl-lib:int-sub k 1))
                           ((1 lda) (1 *)) a-%offset%)
              (-
               (f2cl-lib:fref a-%data%
                              ((f2cl-lib:int-sub k 1)
                               (f2cl-lib:int-sub k 1))
                              ((1 lda) (1 *)) a-%offset%)
               (ddot (f2cl-lib:int-sub n k) work 1
                (f2cl-lib:array-slice a-%data% double-float
                                      ((+ k 1)
                                       (f2cl-lib:int-sub k 1))
                                      ((1 lda) (1 *)) a-%offset%)
                1)))))
    (setf kstep 2)))
  (cond
   ((= kstep 1)
    (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
    (cond
     ((/= kp k)
      (if (< kp n)
          (dswap (f2cl-lib:int-sub n kp)
           (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) kp)
                                 ((1 lda) (1 *)) a-%offset%)
           1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dswap (f2cl-lib:int-sub kp k 1)
           (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% double-float
                                 (kp (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf lda var-4)))
      (setf temp
              (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
```

```
                              a-%offset%))
    (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                              a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                            a-%offset%)
            temp))))
(t
 (setf kp
         (f2cl-lib:int-sub
          (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
 (cond
  ((/= kp k)
   (if (< kp n)
       (dswap (f2cl-lib:int-sub n kp)
        (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) k)
                              ((1 lda) (1 *)) a-%offset%)
        1
        (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) kp)
                              ((1 lda) (1 *)) a-%offset%)
        1))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
       (dswap (f2cl-lib:int-sub kp k 1)
        (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                              ((1 lda) (1 *)) a-%offset%)
        1
        (f2cl-lib:array-slice a-%data% double-float
                              (kp (f2cl-lib:int-add k 1))
                              ((1 lda) (1 *)) a-%offset%)
        lda)
     (declare (ignore var-0 var-1 var-2 var-3))
     (when var-4 (setf lda var-4)))
   (setf temp
           (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                              a-%offset%))
   (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
           (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                              a-%offset%))
   (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                            a-%offset%)
           temp)
   (setf temp
           (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                              ((1 lda) (1 *)) a-%offset%))
   (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                            ((1 lda) (1 *)) a-%offset%)
           (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-sub k 1))
                              ((1 lda) (1 *)) a-%offset%))
   (setf (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-sub k 1))
                            ((1 lda) (1 *)) a-%offset%)
```

```
                  temp)))
            (setf k (f2cl-lib:int-sub k 1))
            (setf kp
                    (f2cl-lib:int-sub
                     (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
            (cond
             ((/= kp k)
              (if (< kp n)
                  (dswap (f2cl-lib:int-sub n kp)
                   (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) k)
                                         ((1 lda) (1 *)) a-%offset%)
                   1
                   (f2cl-lib:array-slice a-%data% double-float ((+ kp 1) kp)
                                         ((1 lda) (1 *)) a-%offset%)
                   1))
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (dswap (f2cl-lib:int-sub kp k 1)
                   (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                                         ((1 lda) (1 *)) a-%offset%)
                   1
                   (f2cl-lib:array-slice a-%data% double-float
                                         (kp (f2cl-lib:int-add k 1))
                                         ((1 lda) (1 *)) a-%offset%)
                   lda)
                (declare (ignore var-0 var-1 var-2 var-3))
                (when var-4 (setf lda var-4)))
              (setf temp
                      (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                      (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                                   a-%offset%)
                      temp)))))
          (setf k (f2cl-lib:int-sub k 1))
          (go label50)
         label60)))
       (go end_label)
      end_label
       (return (values uplo nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsytri_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
```

```
  (array double-float (*))
  (fortran-to_lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (array double-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::uplo nil nil
  fortran-to-lisp::lda nil nil
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

58.25 dsytrs

58.25.1 Axiom unit tests

— **dsytrs.input** —

```
)set break resume
)sys rm -f dsytrs.output
)spool dsytrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.25.2 Axiom help page

— **dsytrs.help** —

```
=====================================================================
dsytrs examples
=====================================================================

=====================================================================
Man Page Details
```

```
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsytrs (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO)
           DSYTRS

Function/Subroutine Documentation
   subroutine dsytrs (character UPLO, integer N, integer NRHS, double
       precision, dimension( lda, * ) A, integer LDA, integer, dimension( * )
       IPIV, double precision, dimension( ldb, * ) B, integer LDB, integer
       INFO)
       DSYTRS

Purpose:
   DSYTRS solves a system of linear equations A*X = B with a real
   symmetric matrix A using the factorization A = U*D*U**T or
   A = L*D*L**T computed by DSYTRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by DSYTRF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by DSYTRF.
```

```
B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the right hand side matrix B.
  On exit, the solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

58.25.3 fortran code

— **dsytrs.f** —

```
*  =====================================================================
      SUBROUTINE DSYTRS( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            J, K, KP
```

```
      DOUBLE PRECISION   AK, AKM1, AKM1K, BK, BKM1, DENOM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEMV, DGER, DSCAL, DSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYTRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*        Solve A*X = B, where A = U*D*U**T.
*
*        First solve U*D*X = B, overwriting B with X.
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop.
```

```
*
         IF( K.LT.1 )
     $      GO TO 30
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
*           Multiply by inv(U(K)), where U(K) is the transformation
*           stored in column K of A.
*
            CALL DGER( K-1, NRHS, -ONE, A( 1, K ), 1, B( K, 1 ), LDB,
     $                 B( 1, 1 ), LDB )
*
*           Multiply by the inverse of the diagonal block.
*
            CALL DSCAL( NRHS, ONE / A( K, K ), B( K, 1 ), LDB )
            K = K - 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Interchange rows K-1 and -IPIV(K).
*
            KP = -IPIV( K )
            IF( KP.NE.K-1 )
     $         CALL DSWAP( NRHS, B( K-1, 1 ), LDB, B( KP, 1 ), LDB )
*
*           Multiply by inv(U(K)), where U(K) is the transformation
*           stored in columns K-1 and K of A.
*
            CALL DGER( K-2, NRHS, -ONE, A( 1, K ), 1, B( K, 1 ), LDB,
     $                 B( 1, 1 ), LDB )
            CALL DGER( K-2, NRHS, -ONE, A( 1, K-1 ), 1, B( K-1, 1 ),
     $                 LDB, B( 1, 1 ), LDB )
*
*           Multiply by the inverse of the diagonal block.
*
            AKM1K = A( K-1, K )
            AKM1 = A( K-1, K-1 ) / AKM1K
            AK = A( K, K ) / AKM1K
            DENOM = AKM1*AK - ONE
            DO 20 J = 1, NRHS
               BKM1 = B( K-1, J ) / AKM1K
```

```
               BK = B( K, J ) / AKM1K
               B( K-1, J ) = ( AK*BKM1-BK ) / DENOM
               B( K, J ) = ( AKM1*BK-BKM1 ) / DENOM
   20       CONTINUE
            K = K - 2
         END IF
*
         GO TO 10
   30    CONTINUE
*
*        Next solve U**T *X = B, overwriting B with X.
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = 1
   40    CONTINUE
*
*        If K > N, exit from loop.
*
         IF( K.GT.N )
     $      GO TO 50
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Multiply by inv(U**T(K)), where U(K) is the transformation
*           stored in column K of A.
*
            CALL DGEMV( 'Transpose', K-1, NRHS, -ONE, B, LDB, A( 1, K ),
     $                  1, ONE, B( K, 1 ), LDB )
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K = K + 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Multiply by inv(U**T(K+1)), where U(K+1) is the transformation
*           stored in columns K and K+1 of A.
*
            CALL DGEMV( 'Transpose', K-1, NRHS, -ONE, B, LDB, A( 1, K ),
     $                  1, ONE, B( K, 1 ), LDB )
            CALL DGEMV( 'Transpose', K-1, NRHS, -ONE, B, LDB,
     $                  A( 1, K+1 ), 1, ONE, B( K+1, 1 ), LDB )
```

```
*
*          Interchange rows K and -IPIV(K).
*
           KP = -IPIV( K )
           IF( KP.NE.K )
     $        CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
           K = K + 2
        END IF
*
        GO TO 40
   50   CONTINUE
*
      ELSE
*
*       Solve A*X = B, where A = L*D*L**T.
*
*       First solve L*D*X = B, overwriting B with X.
*
*       K is the main loop index, increasing from 1 to N in steps of
*       1 or 2, depending on the size of the diagonal blocks.
*
        K = 1
   60   CONTINUE
*
*       If K > N, exit from loop.
*
        IF( K.GT.N )
     $     GO TO 80
*
        IF( IPIV( K ).GT.0 ) THEN
*
*          1 x 1 diagonal block
*
*          Interchange rows K and IPIV(K).
*
           KP = IPIV( K )
           IF( KP.NE.K )
     $        CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
*          Multiply by inv(L(K)), where L(K) is the transformation
*          stored in column K of A.
*
           IF( K.LT.N )
     $        CALL DGER( N-K, NRHS, -ONE, A( K+1, K ), 1, B( K, 1 ),
     $                   LDB, B( K+1, 1 ), LDB )
*
*          Multiply by the inverse of the diagonal block.
*
           CALL DSCAL( NRHS, ONE / A( K, K ), B( K, 1 ), LDB )
           K = K + 1
```

```
         ELSE
*
*           2 x 2 diagonal block
*
*           Interchange rows K+1 and -IPIV(K).
*
            KP = -IPIV( K )
            IF( KP.NE.K+1 )
     $         CALL DSWAP( NRHS, B( K+1, 1 ), LDB, B( KP, 1 ), LDB )
*
*           Multiply by inv(L(K)), where L(K) is the transformation
*           stored in columns K and K+1 of A.
*
            IF( K.LT.N-1 ) THEN
               CALL DGER( N-K-1, NRHS, -ONE, A( K+2, K ), 1, B( K, 1 ),
     $                    LDB, B( K+2, 1 ), LDB )
               CALL DGER( N-K-1, NRHS, -ONE, A( K+2, K+1 ), 1,
     $                    B( K+1, 1 ), LDB, B( K+2, 1 ), LDB )
            END IF
*
*           Multiply by the inverse of the diagonal block.
*
            AKM1K = A( K+1, K )
            AKM1 = A( K, K ) / AKM1K
            AK = A( K+1, K+1 ) / AKM1K
            DENOM = AKM1*AK - ONE
            DO 70 J = 1, NRHS
               BKM1 = B( K, J ) / AKM1K
               BK = B( K+1, J ) / AKM1K
               B( K, J ) = ( AK*BKM1-BK ) / DENOM
               B( K+1, J ) = ( AKM1*BK-BKM1 ) / DENOM
   70       CONTINUE
            K = K + 2
         END IF
*
         GO TO 60
   80    CONTINUE
*
*        Next solve L**T *X = B, overwriting B with X.
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = N
   90    CONTINUE
*
*        If K < 1, exit from loop.
*
         IF( K.LT.1 )
     $      GO TO 100
```

```
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Multiply by inv(L**T(K)), where L(K) is the transformation
*           stored in column K of A.
*
            IF( K.LT.N )
     $         CALL DGEMV( 'Transpose', N-K, NRHS, -ONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K ), 1, ONE, B( K, 1 ), LDB )
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K = K - 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Multiply by inv(L**T(K-1)), where L(K-1) is the transformation
*           stored in columns K-1 and K of A.
*
            IF( K.LT.N ) THEN
               CALL DGEMV( 'Transpose', N-K, NRHS, -ONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K ), 1, ONE, B( K, 1 ), LDB )
               CALL DGEMV( 'Transpose', N-K, NRHS, -ONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K-1 ), 1, ONE, B( K-1, 1 ),
     $                     LDB )
            END IF
*
*           Interchange rows K and -IPIV(K).
*
            KP = -IPIV( K )
            IF( KP.NE.K )
     $         CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K = K - 2
         END IF
*
         GO TO 90
  100    CONTINUE
      END IF
*
      RETURN
*
*     End of DSYTRS
*
      END
```

———-

58.25.4 lisp code

— dsytrs.lisp —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dsytrs (uplo n nrhs a lda ipiv b ldb$ info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array double-float (*)) b a)
             (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((ak 0.0d0) (akm1 0.0d0) (akm1k 0.0d0) (bk 0.0d0) (bkm1 0.0d0)
             (denom 0.0d0) (j 0) (k 0) (kp 0) (upper nil))
        (declare (type (double-float) ak akm1 akm1k bk bkm1 denom)
                 (type (f2cl-lib:integer4) j k kp)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -5))
         ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -8)))
        (cond
         ((/= info 0) (xerbla "DSYTRS" (f2cl-lib:int-sub info))
          (go end_label)))
```

```
(if (or (= n 0) (= nrhs 0))
    (go end_label))
(cond
 (upper
  (tagbody
    (setf k n)
   label10
    (if (< k 1)
        (go label30))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dswap nrhs
               (f2cl-lib:array-slice b-%data% double-float (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dger (f2cl-lib:int-sub k 1) nrhs (- one)
           (f2cl-lib:array-slice a-%data% double-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice b-%data% double-float (k 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% double-float (1 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
        (when var-1 (setf nrhs var-1))
        (when var-6 (setf ldb$ var-6))
        (when var-8 (setf ldb$ var-8)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal nrhs
           (/ one
              (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                             a-%offset%))
           (f2cl-lib:array-slice b-%data% double-float (k 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-2))
```

```
    (when var-0 (setf nrhs var-0))
    (when var-3 (setf ldb$ var-3)))
  (setf k (f2cl-lib:int-sub k 1)))
 (t
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
  (if (/= kp (f2cl-lib:int-sub k 1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dswap nrhs
           (f2cl-lib:array-slice b-%data% double-float
                                 ((+ k (f2cl-lib:int-sub 1)) 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-3))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dger (f2cl-lib:int-sub k 2) nrhs (- one)
       (f2cl-lib:array-slice a-%data% double-float (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice b-%data% double-float (k 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice b-%data% double-float (1 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
    (when var-1 (setf nrhs var-1))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldb$ var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dger (f2cl-lib:int-sub k 2) nrhs (- one)
       (f2cl-lib:array-slice a-%data% double-float
                             (1 (f2cl-lib:int-sub k 1))
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice b-%data% double-float
                             ((+ k (f2cl-lib:int-sub 1)) 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice b-%data% double-float (1 1)
                             ((1 ldb$) (1 *)) b-%offset%)
```

```
        ldb$)
      (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
      (when var-1 (setf nrhs var-1))
      (when var-6 (setf ldb$ var-6))
      (when var-8 (setf ldb$ var-8)))
    (setf akm1k
            (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                           ((1 lda) (1 *)) a-%offset%))
    (setf akm1
            (/
             (f2cl-lib:fref a-%data%
                            ((f2cl-lib:int-sub k 1)
                             (f2cl-lib:int-sub k 1))
                            ((1 lda) (1 *)) a-%offset%)
             akm1k))
    (setf ak
            (/
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)
             akm1k))
    (setf denom (- (* akm1 ak) one))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (setf bkm1
                (/
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1) j)
                                ((1 ldb$) (1 *)) b-%offset%)
                 akm1k))
        (setf bk
                (/
                 (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                b-%offset%)
                 akm1k))
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)
                (/ (- (* ak bkm1) bk) denom))
        (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                             b-%offset%)
                (/ (- (* akm1 bk) bkm1) denom))
       label20))
    (setf k (f2cl-lib:int-sub k 2))))
  (go label10)
 label30
  (setf k 1)
 label40
  (if (> k n)
      (go label50))
  (cond
   ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
```

```
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (dgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- one) b ldb$
      (f2cl-lib:array-slice a-%data% double-float (1 k)
                            ((1 lda) (1 *)) a-%offset%)
      1 one
      (f2cl-lib:array-slice b-%data% double-float (k 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
   (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
   (when var-2 (setf nrhs var-2))
   (when var-5 (setf ldb$ var-5))
   (when var-8 (setf one var-8))
   (when var-10 (setf ldb$ var-10)))
 (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
 (if (/= kp k)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (dswap nrhs
          (f2cl-lib:array-slice b-%data% double-float (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
       (declare (ignore var-1 var-3))
       (when var-0 (setf nrhs var-0))
       (when var-2 (setf ldb$ var-2))
       (when var-4 (setf ldb$ var-4))))
 (setf k (f2cl-lib:int-add k 1)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (dgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- one) b ldb$
      (f2cl-lib:array-slice a-%data% double-float (1 k)
                            ((1 lda) (1 *)) a-%offset%)
      1 one
      (f2cl-lib:array-slice b-%data% double-float (k 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
   (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
   (when var-2 (setf nrhs var-2))
   (when var-5 (setf ldb$ var-5))
   (when var-8 (setf one var-8))
   (when var-10 (setf ldb$ var-10)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (dgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- one) b ldb$
```

```
                (f2cl-lib:array-slice a-%data% double-float
                                      (1 (f2cl-lib:int-add k 1))
                                      ((1 lda) (1 *)) a-%offset%)
                1 one
                (f2cl-lib:array-slice b-%data% double-float ((+ k 1) 1)
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$)
              (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
              (when var-2 (setf nrhs var-2))
              (when var-5 (setf ldb$ var-5))
              (when var-8 (setf one var-8))
              (when var-10 (setf ldb$ var-10)))
            (setf kp
                    (f2cl-lib:int-sub
                     (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
            (if (/= kp k)
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (dswap nrhs
                     (f2cl-lib:array-slice b-%data% double-float (k 1)
                                           ((1 ldb$) (1 *)) b-%offset%)
                     ldb$
                     (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                           ((1 ldb$) (1 *)) b-%offset%)
                     ldb$)
                  (declare (ignore var-1 var-3))
                  (when var-0 (setf nrhs var-0))
                  (when var-2 (setf ldb$ var-2))
                  (when var-4 (setf ldb$ var-4))))
            (setf k (f2cl-lib:int-add k 2))))
          (go label40)
         label50))
        (t
         (tagbody
          (setf k 1)
         label60
          (if (> k n)
              (go label80))
          (cond
           ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
            (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
            (if (/= kp k)
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (dswap nrhs
                     (f2cl-lib:array-slice b-%data% double-float (k 1)
                                           ((1 ldb$) (1 *)) b-%offset%)
                     ldb$
                     (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                           ((1 ldb$) (1 *)) b-%offset%)
                     ldb$)
                  (declare (ignore var-1 var-3))
```

```
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
  (if (< k n)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dger (f2cl-lib:int-sub n k) nrhs (- one)
           (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice b-%data% double-float (k 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% double-float ((+ k 1) 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
        (when var-1 (setf nrhs var-1))
        (when var-6 (setf ldb$ var-6))
        (when var-8 (setf ldb$ var-8))))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dscal nrhs
       (/ one
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
       (f2cl-lib:array-slice b-%data% double-float (k 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-1 var-2))
    (when var-0 (setf nrhs var-0))
    (when var-3 (setf ldb$ var-3)))
  (setf k (f2cl-lib:int-add k 1)))
 (t
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
  (if (/= kp (f2cl-lib:int-add k 1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dswap nrhs
           (f2cl-lib:array-slice b-%data% double-float ((+ k 1) 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-3))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
  (cond
```

```
((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (dger (f2cl-lib:int-sub n k 1) nrhs (- one)
      (f2cl-lib:array-slice a-%data% double-float ((+ k 2) k)
                            ((1 lda) (1 *)) a-%offset%)
      1
      (f2cl-lib:array-slice b-%data% double-float (k 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$
      (f2cl-lib:array-slice b-%data% double-float ((+ k 2) 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
   (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
   (when var-1 (setf nrhs var-1))
   (when var-6 (setf ldb$ var-6))
   (when var-8 (setf ldb$ var-8)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (dger (f2cl-lib:int-sub n k 1) nrhs (- one)
      (f2cl-lib:array-slice a-%data% double-float
                            ((+ k 2) (f2cl-lib:int-add k 1))
                            ((1 lda) (1 *)) a-%offset%)
      1
      (f2cl-lib:array-slice b-%data% double-float ((+ k 1) 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$
      (f2cl-lib:array-slice b-%data% double-float ((+ k 2) 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
   (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
   (when var-1 (setf nrhs var-1))
   (when var-6 (setf ldb$ var-6))
   (when var-8 (setf ldb$ var-8)))))
(setf akm1k
       (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                      ((1 lda) (1 *)) a-%offset%))
(setf akm1
       (/
        (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                       a-%offset%)
        akm1k))
(setf ak
       (/
        (f2cl-lib:fref a-%data%
                       ((f2cl-lib:int-add k 1)
                        (f2cl-lib:int-add k 1))
                       ((1 lda) (1 *)) a-%offset%)
        akm1k))
(setf denom (- (* akm1 ak) one))
```

```
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (setf bkm1
                (/
                 (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                b-%offset%)
                 akm1k))
        (setf bk
                (/
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                                ((1 ldb$) (1 *)) b-%offset%)
                 akm1k))
        (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                             b-%offset%)
                (/ (- (* ak bkm1) bk) denom))
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)
                (/ (- (* akm1 bk) bkm1) denom))
       label70))
    (setf k (f2cl-lib:int-add k 2))))
  (go label60)
 label80
  (setf k n)
 label90
  (if (< k 1)
      (go label100))
  (cond
   ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
    (if (< k n)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10)
            (dgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- one)
             (f2cl-lib:array-slice b-%data% double-float ((+ k 1) 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$
             (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                                   ((1 lda) (1 *)) a-%offset%)
             1 one
             (f2cl-lib:array-slice b-%data% double-float (k 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$)
          (declare
           (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
          (when var-2 (setf nrhs var-2))
          (when var-5 (setf ldb$ var-5))
          (when var-8 (setf one var-8))
          (when var-10 (setf ldb$ var-10))))
    (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
```

```
 (if (/= kp k)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (dswap nrhs
          (f2cl-lib:array-slice b-%data% double-float (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
       (declare (ignore var-1 var-3))
       (when var-0 (setf nrhs var-0))
       (when var-2 (setf ldb$ var-2))
       (when var-4 (setf ldb$ var-4))))
 (setf k (f2cl-lib:int-sub k 1)))
(t
 (cond
  ((< k n)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (dgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- one)
        (f2cl-lib:array-slice b-%data% double-float ((+ k 1) 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$
        (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                              ((1 lda) (1 *)) a-%offset%)
        1 one
        (f2cl-lib:array-slice b-%data% double-float (k 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$)
     (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
     (when var-2 (setf nrhs var-2))
     (when var-5 (setf ldb$ var-5))
     (when var-8 (setf one var-8))
     (when var-10 (setf ldb$ var-10)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (dgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- one)
        (f2cl-lib:array-slice b-%data% double-float ((+ k 1) 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$
        (f2cl-lib:array-slice a-%data% double-float
                              ((+ k 1) (f2cl-lib:int-sub k 1))
                              ((1 lda) (1 *)) a-%offset%)
        1 one
        (f2cl-lib:array-slice b-%data% double-float
                              ((+ k (f2cl-lib:int-sub 1)) 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$)
```

```
                (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
                (when var-2 (setf nrhs var-2))
                (when var-5 (setf ldb$ var-5))
                (when var-8 (setf one var-8))
                (when var-10 (setf ldb$ var-10)))))
            (setf kp
                    (f2cl-lib:int-sub
                     (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
            (if (/= kp k)
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (dswap nrhs
                     (f2cl-lib:array-slice b-%data% double-float (k 1)
                                           ((1 ldb$) (1 *)) b-%offset%)
                     ldb$
                     (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                           ((1 ldb$) (1 *)) b-%offset%)
                     ldb$)
                  (declare (ignore var-1 var-3))
                  (when var-0 (setf nrhs var-0))
                  (when var-2 (setf ldb$ var-2))
                  (when var-4 (setf ldb$ var-4))))
            (setf k (f2cl-lib:int-sub k 2))))
          (go label90)
         label100)))
       (go end_label)
      end_label
       (return (values uplo nil nrhs nil nil nil nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsytrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil
                                              fortran-to-lisp::nrhs nil nil nil
                                              nil fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

58.26 dsytrs2

58.26.1 Axiom unit tests

— **dsytrs2.input** —

```
)set break resume
)sys rm -f dsytrs2.output
)spool dsytrs2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.26.2 Axiom help page

— **dsytrs2.help** —

```
======================================================================
dsytrs2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsytrs2 (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK, INFO)
           DSYTRS2

Function/Subroutine Documentation
   subroutine dsytrs2 (character UPLO, integer N, integer NRHS, double
       precision, dimension( lda, * ) A, integer LDA, integer, dimension( * )
       IPIV, double precision, dimension( ldb, * ) B, integer LDB, double
```

```
      precision, dimension( * ) WORK, integer INFO)
      DSYTRS2

Purpose:
   DSYTRS2 solves a system of linear equations A*X = B with a real
   symmetric matrix A using the factorization A = U*D*U**T or
   A = L*D*L**T computed by DSYTRF and converted by DSYCONV.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by DSYTRF.
   Note that A is input / output. This might be counter-intuitive,
   and one may think that A is input only. A is input / output. This
   is because, at the start of the subroutine, we permute A in a
   "better" form and then we permute A back to its original form at the end.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by DSYTRF.

 B
   B is DOUBLE PRECISION array, dimension (LDB,NRHS)
   On entry, the right hand side matrix B.
   On exit, the solution matrix X.

 LDB
```

```
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

 WORK
  WORK is REAL array, dimension (N)

 INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

58.26.3 fortran code

— **dsytrs2.f** —

```
*  ===================================================================
      SUBROUTINE DSYTRS2( UPLO, N, NRHS, A, LDA, IPIV, B, LDB,
     $                    WORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, IINFO, J, K, KP
      DOUBLE PRECISION   AK, AKM1, AKM1K, BK, BKM1, DENOM
*     ..
*     .. External Functions ..
```

```
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DSCAL, DSYCONV, DSWAP, DTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYTRS2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
*     Convert A
*
      CALL DSYCONV( UPLO, 'C', N, A, LDA, IPIV, WORK, IINFO )
*
      IF( UPPER ) THEN
*
*        Solve A*X = B, where A = U*D*U**T.
*
*       P**T * B
        K=N
        DO WHILE ( K .GE. 1 )
         IF( IPIV( K ).GT.0 ) THEN
*           1 x 1 diagonal block
*           Interchange rows K and IPIV(K).
            KP = IPIV( K )
            IF( KP.NE.K )
```

```
     $         CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K=K-1
         ELSE
*           2 x 2 diagonal block
*           Interchange rows K-1 and -IPIV(K).
            KP = -IPIV( K )
            IF( KP.EQ.-IPIV( K-1 ) )
     $         CALL DSWAP( NRHS, B( K-1, 1 ), LDB, B( KP, 1 ), LDB )
            K=K-2
         END IF
        END DO
*
*  Compute (U \P**T * B) -> B    [ (U \P**T * B) ]
*
        CALL DTRSM('L','U','N','U',N,NRHS,ONE,A,LDA,B,LDB)
*
*  Compute D \ B -> B   [ D \ (U \P**T * B) ]
*
         I=N
         DO WHILE ( I .GE. 1 )
            IF( IPIV(I) .GT. 0 ) THEN
              CALL DSCAL( NRHS, ONE / A( I, I ), B( I, 1 ), LDB )
            ELSEIF ( I .GT. 1) THEN
               IF ( IPIV(I-1) .EQ. IPIV(I) ) THEN
                  AKM1K = WORK(I)
                  AKM1 = A( I-1, I-1 ) / AKM1K
                  AK = A( I, I ) / AKM1K
                  DENOM = AKM1*AK - ONE
                  DO 15 J = 1, NRHS
                     BKM1 = B( I-1, J ) / AKM1K
                     BK = B( I, J ) / AKM1K
                     B( I-1, J ) = ( AK*BKM1-BK ) / DENOM
                     B( I, J ) = ( AKM1*BK-BKM1 ) / DENOM
 15               CONTINUE
               I = I - 1
               ENDIF
            ENDIF
            I = I - 1
         END DO
*
*      Compute (U**T \ B) -> B   [ U**T \ (D \ (U \P**T * B) ) ]
*
         CALL DTRSM('L','U','T','U',N,NRHS,ONE,A,LDA,B,LDB)
*
*       P * B  [ P * (U**T \ (D \ (U \P**T * B) )) ]
*
        K=1
        DO WHILE ( K .LE. N )
         IF( IPIV( K ).GT.0 ) THEN
*           1 x 1 diagonal block
```

```
*          Interchange rows K and IPIV(K).
           KP = IPIV( K )
           IF( KP.NE.K )
     $        CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
           K=K+1
        ELSE
*          2 x 2 diagonal block
*          Interchange rows K-1 and -IPIV(K).
           KP = -IPIV( K )
           IF( K .LT. N .AND. KP.EQ.-IPIV( K+1 ) )
     $        CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
           K=K+2
        ENDIF
       END DO
*
      ELSE
*
*       Solve A*X = B, where A = L*D*L**T.
*
       P**T * B
       K=1
       DO WHILE ( K .LE. N )
        IF( IPIV( K ).GT.0 ) THEN
*          1 x 1 diagonal block
*          Interchange rows K and IPIV(K).
           KP = IPIV( K )
           IF( KP.NE.K )
     $        CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
           K=K+1
        ELSE
*          2 x 2 diagonal block
*          Interchange rows K and -IPIV(K+1).
           KP = -IPIV( K+1 )
           IF( KP.EQ.-IPIV( K ) )
     $        CALL DSWAP( NRHS, B( K+1, 1 ), LDB, B( KP, 1 ), LDB )
           K=K+2
        ENDIF
       END DO
*
*  Compute (L \P**T * B) -> B    [ (L \P**T * B) ]
*
       CALL DTRSM('L','L','N','U',N,NRHS,ONE,A,LDA,B,LDB)
*
*  Compute D \ B -> B   [ D \ (L \P**T * B) ]
*
        I=1
        DO WHILE ( I .LE. N )
           IF( IPIV(I) .GT. 0 ) THEN
             CALL DSCAL( NRHS, ONE / A( I, I ), B( I, 1 ), LDB )
           ELSE
```

```
                 AKM1K = WORK(I)
                 AKM1 = A( I, I ) / AKM1K
                 AK = A( I+1, I+1 ) / AKM1K
                 DENOM = AKM1*AK - ONE
                 DO 25 J = 1, NRHS
                    BKM1 = B( I, J ) / AKM1K
                    BK = B( I+1, J ) / AKM1K
                    B( I, J ) = ( AK*BKM1-BK ) / DENOM
                    B( I+1, J ) = ( AKM1*BK-BKM1 ) / DENOM
 25              CONTINUE
                 I = I + 1
            ENDIF
            I = I + 1
         END DO
*
*  Compute (L**T \ B) -> B   [ L**T \ (D \ (L \P**T * B) ) ]
*
        CALL DTRSM('L','L','T','U',N,NRHS,ONE,A,LDA,B,LDB)
*
*       P * B  [ P * (L**T \ (D \ (L \P**T * B) )) ]
*
        K=N
        DO WHILE ( K .GE. 1 )
         IF( IPIV( K ).GT.0 ) THEN
*           1 x 1 diagonal block
*           Interchange rows K and IPIV(K).
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K=K-1
         ELSE
*           2 x 2 diagonal block
*           Interchange rows K-1 and -IPIV(K).
            KP = -IPIV( K )
            IF( K.GT.1 .AND. KP.EQ.-IPIV( K-1 ) )
     $         CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K=K-2
         ENDIF
        END DO
*
      END IF
*
*     Revert A
*
      CALL DSYCONV( UPLO, 'R', N, A, LDA, IPIV, WORK, IINFO )
*
      RETURN
*
*     End of DSYTRS2
*
```

```
      END
```

———-

58.26.4 lisp code

— **dsytrs2.lisp** —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dsytrs2 (uplo n nrhs a lda ipiv b ldb$ work info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array double-float (*)) work b a)
             (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%)
         (work double-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((ak 0.0d0) (akm1 0.0d0) (akm1k 0.0d0) (bk 0.0d0) (bkm1 0.0d0)
             (denom 0.0d0) (i 0) (iinfo 0) (j 0) (k 0) (kp 0) (upper nil))
        (declare (type (double-float) ak akm1 akm1k bk bkm1 denom)
                 (type (f2cl-lib:integer4) i iinfo j k kp)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -5))
         ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -8)))
```

```
(cond
 ((/= info 0) (xerbla "DSYTRS2" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= n 0) (= nrhs 0))
    (go end_label))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dsyconv uplo "C" n a lda ipiv work iinfo)
  (declare (ignore var-1 var-3 var-5 var-6))
  (when var-0 (setf uplo var-0))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-7 (setf iinfo var-7)))
(cond
 (upper
  (tagbody
    (setf k n)
   label100000
    (if (not (>= k 1))
        (go label100001))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dswap nrhs
               (f2cl-lib:array-slice b-%data% double-float (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-sub k 1)))
     (t
      (setf kp
              (f2cl-lib:int-sub
               (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
      (if (= kp
             (f2cl-lib:int-sub
              (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                             ((1 *)) ipiv-%offset%)))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dswap nrhs
               (f2cl-lib:array-slice b-%data% double-float
                                     ((+ k (f2cl-lib:int-sub 1)) 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
```

```
              (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                    ((1 ldb$) (1 *)) b-%offset%)
              ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
    (setf k (f2cl-lib:int-sub k 2))))
  (go label100000)
 label100001
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dtrsm "L" "U" "N" "U" n nrhs one a lda b ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10)))
  (setf i n)
 label100002
  (if (not (>= i 1))
      (go label100003))
  (cond
   ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal nrhs
         (/ one
            (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                           a-%offset%))
         (f2cl-lib:array-slice b-%data% double-float (i 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-1 var-2))
      (when var-0 (setf nrhs var-0))
      (when var-3 (setf ldb$ var-3))))
   ((> i 1)
    (cond
     ((=
       (f2cl-lib:fref ipiv
                      ((f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((1 *)))
       (f2cl-lib:fref ipiv (i) ((1 *))))
      (setf akm1k
              (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
      (setf akm1
              (/
               (f2cl-lib:fref a-%data%
                              ((f2cl-lib:int-sub i 1)
```

```
                                (f2cl-lib:int-sub i 1))
                               ((1 lda) (1 *)) a-%offset%)
                akm1k))
        (setf ak
                (/
                 (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                               a-%offset%)
                 akm1k))
        (setf denom (- (* akm1 ak) one))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j nrhs) nil)
          (tagbody
            (setf bkm1
                    (/
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                    ((1 ldb$) (1 *)) b-%offset%)
                     akm1k))
            (setf bk
                    (/
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)
                     akm1k))
            (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                    (/ (- (* ak bkm1) bk) denom))
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                    (/ (- (* akm1 bk) bkm1) denom))
           label15))
        (setf i (f2cl-lib:int-sub i 1))))))
    (setf i (f2cl-lib:int-sub i 1))
    (go label100002)
  label100003
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (dtrsm "L" "U" "T" "U" n nrhs one a lda b ldb$)
      (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
      (when var-4 (setf n var-4))
      (when var-5 (setf nrhs var-5))
      (when var-6 (setf one var-6))
      (when var-8 (setf lda var-8))
      (when var-10 (setf ldb$ var-10)))
    (setf k 1)
  label100004
    (if (not (<= k n))
        (go label100005))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
```

```
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dswap nrhs
               (f2cl-lib:array-slice b-%data% double-float (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-add k 1)))
     (t
      (setf kp
              (f2cl-lib:int-sub
               (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
      (if (and (< k n)
               (= kp
                  (f2cl-lib:int-sub
                   (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
                                  ((1 *)) ipiv-%offset%))))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dswap nrhs
               (f2cl-lib:array-slice b-%data% double-float (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-add k 2))))
    (go label100004)
   label100005))
 (t
  (tagbody
    (setf k 1)
   label100006
    (if (not (<= k n))
        (go label100007))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dswap nrhs
```

```
                (f2cl-lib:array-slice b-%data% double-float (k 1)
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$
                (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$)
              (declare (ignore var-1 var-3))
              (when var-0 (setf nrhs var-0))
              (when var-2 (setf ldb$ var-2))
              (when var-4 (setf ldb$ var-4))))
    (setf k (f2cl-lib:int-add k 1)))
   (t
    (setf kp
            (f2cl-lib:int-sub
             (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
                            ((1 *)) ipiv-%offset%)))
    (if (= kp
           (f2cl-lib:int-sub
            (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dswap nrhs
             (f2cl-lib:array-slice b-%data% double-float ((+ k 1) 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$
             (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
    (setf k (f2cl-lib:int-add k 2))))
  (go label100006)
 label100007
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dtrsm "L" "L" "N" "U" n nrhs one a lda b ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10)))
  (setf i 1)
 label100008
  (if (not (<= i n))
      (go label100009))
  (cond
   ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
```

```
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dscal nrhs
       (/ one
          (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                         a-%offset%))
       (f2cl-lib:array-slice b-%data% double-float (i 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-1 var-2))
    (when var-0 (setf nrhs var-0))
    (when var-3 (setf ldb$ var-3))))
 (t
  (setf akm1k
          (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
  (setf akm1
          (/
           (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                          a-%offset%)
           akm1k))
  (setf ak
          (/
           (f2cl-lib:fref a-%data%
                          ((f2cl-lib:int-add i 1)
                           (f2cl-lib:int-add i 1))
                          ((1 lda) (1 *)) a-%offset%)
           akm1k))
  (setf denom (- (* akm1 ak) one))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf bkm1
              (/
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)
               akm1k))
      (setf bk
              (/
               (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                              ((1 ldb$) (1 *)) b-%offset%)
               akm1k))
      (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                           b-%offset%)
              (/ (- (* ak bkm1) bk) denom))
      (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                           ((1 ldb$) (1 *)) b-%offset%)
              (/ (- (* akm1 bk) bkm1) denom))
     label25))
  (setf i (f2cl-lib:int-add i 1))))
(setf i (f2cl-lib:int-add i 1))
(go label100008)
```

```
label100009
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (dtrsm "L" "L" "T" "U" n nrhs one a lda b ldb$)
   (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
   (when var-4 (setf n var-4))
   (when var-5 (setf nrhs var-5))
   (when var-6 (setf one var-6))
   (when var-8 (setf lda var-8))
   (when var-10 (setf ldb$ var-10)))
 (setf k n)
label100010
 (if (not (>= k 1))
     (go label100011))
 (cond
  ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
   (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
   (if (/= kp k)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (dswap nrhs
            (f2cl-lib:array-slice b-%data% double-float (k 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$
            (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
         (declare (ignore var-1 var-3))
         (when var-0 (setf nrhs var-0))
         (when var-2 (setf ldb$ var-2))
         (when var-4 (setf ldb$ var-4))))
   (setf k (f2cl-lib:int-sub k 1)))
  (t
   (setf kp
           (f2cl-lib:int-sub
            (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
   (if (and (> k 1)
            (= kp
               (f2cl-lib:int-sub
                (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                               ((1 *)) ipiv-%offset%))))
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (dswap nrhs
            (f2cl-lib:array-slice b-%data% double-float (k 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$
            (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
         (declare (ignore var-1 var-3))
```

```
                    (when var-0 (setf nrhs var-0))
                    (when var-2 (setf ldb$ var-2))
                    (when var-4 (setf ldb$ var-4))))
              (setf k (f2cl-lib:int-sub k 2))))
            (go label100010)
           label100011)))
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
             (dsyconv uplo "R" n a lda ipiv work iinfo)
           (declare (ignore var-1 var-3 var-5 var-6))
           (when var-0 (setf uplo var-0))
           (when var-2 (setf n var-2))
           (when var-4 (setf lda var-4))
           (when var-7 (setf iinfo var-7)))
         (go end_label)
        end_label
         (return (values uplo n nrhs nil lda nil nil ldb$ nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsytrs2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::ldb$ nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

58.27 dsytrs_rook

58.27.1 Axiom unit tests

— dsytrsrook.input —

```
)set break resume
)sys rm -f dsytrsrook.output
)spool dsytrsrook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.27.2 Axiom help page

— dsytrs_rook.help —

```
====================================================================
dsytrs_rook examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsytrs_rook (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO)
           DSYTRS_ROOK

Function/Subroutine Documentation
   subroutine dsytrs_rook (character UPLO, integer N, integer NRHS, double
       precision, dimension( lda, * ) A, integer LDA, integer, dimension( * )
       IPIV, double precision, dimension( ldb, * ) B, integer LDB, integer
       INFO)
       DSYTRS_ROOK

Purpose:
   DSYTRS_ROOK solves a system of linear equations A*X = B with
   a real symmetric matrix A using the factorization A = U*D*U**T or
   A = L*D*L**T computed by DSYTRF_ROOK.
```

```
Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by DSYTRF_ROOK.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by DSYTRF_ROOK.

 B
   B is DOUBLE PRECISION array, dimension (LDB,NRHS)
   On entry, the right hand side matrix B.
   On exit, the solution matrix X.

 LDB
   LDB is INTEGER
   The leading dimension of the array B.  LDB >= max(1,N).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

58.27.3 fortran code

— dsytrs_rook.f —

```
*  =====================================================================
      SUBROUTINE DSYTRS_ROOK( UPLO, N, NRHS, A, LDA, IPIV, B, LDB,
     $                        INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            J, K, KP
      DOUBLE PRECISION   AK, AKM1, AKM1K, BK, BKM1, DENOM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEMV, DGER, DSCAL, DSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
```

```
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSYTRS_ROOK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*        Solve A*X = B, where A = U*D*U**T.
*
*        First solve U*D*X = B, overwriting B with X.
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop.
*
         IF( K.LT.1 )
     $      GO TO 30
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
*           Multiply by inv(U(K)), where U(K) is the transformation
*           stored in column K of A.
*
            CALL DGER( K-1, NRHS, -ONE, A( 1, K ), 1, B( K, 1 ), LDB,
```

```
     $                B( 1, 1 ), LDB )
*
*          Multiply by the inverse of the diagonal block.
*
           CALL DSCAL( NRHS, ONE / A( K, K ), B( K, 1 ), LDB )
           K = K - 1
        ELSE
*
*          2 x 2 diagonal block
*
*          Interchange rows K and -IPIV(K) THEN K-1 and -IPIV(K-1)
*
           KP = -IPIV( K )
           IF( KP.NE.K )
     $        CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
           KP = -IPIV( K-1 )
           IF( KP.NE.K-1 )
     $        CALL DSWAP( NRHS, B( K-1, 1 ), LDB, B( KP, 1 ), LDB )
*
*          Multiply by inv(U(K)), where U(K) is the transformation
*          stored in columns K-1 and K of A.
*
           IF( K.GT.2 ) THEN
              CALL DGER( K-2, NRHS, -ONE, A( 1, K ), 1, B( K, 1 ),
     $                   LDB, B( 1, 1 ), LDB )
              CALL DGER( K-2, NRHS, -ONE, A( 1, K-1 ), 1, B( K-1, 1 ),
     $                   LDB, B( 1, 1 ), LDB )
           END IF
*
*          Multiply by the inverse of the diagonal block.
*
           AKM1K = A( K-1, K )
           AKM1 = A( K-1, K-1 ) / AKM1K
           AK = A( K, K ) / AKM1K
           DENOM = AKM1*AK - ONE
           DO 20 J = 1, NRHS
              BKM1 = B( K-1, J ) / AKM1K
              BK = B( K, J ) / AKM1K
              B( K-1, J ) = ( AK*BKM1-BK ) / DENOM
              B( K, J ) = ( AKM1*BK-BKM1 ) / DENOM
   20      CONTINUE
           K = K - 2
        END IF
*
        GO TO 10
   30   CONTINUE
*
*       Next solve U**T *X = B, overwriting B with X.
*
```

```
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = 1
   40    CONTINUE
*
*        If K > N, exit from loop.
*
         IF( K.GT.N )
     $      GO TO 50
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Multiply by inv(U**T(K)), where U(K) is the transformation
*           stored in column K of A.
*
            IF( K.GT.1 )
     $         CALL DGEMV( 'Transpose', K-1, NRHS, -ONE, B,
     $                     LDB, A( 1, K ), 1, ONE, B( K, 1 ), LDB )
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K = K + 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Multiply by inv(U**T(K+1)), where U(K+1) is the transformation
*           stored in columns K and K+1 of A.
*
            IF( K.GT.1 ) THEN
               CALL DGEMV( 'Transpose', K-1, NRHS, -ONE, B,
     $                     LDB, A( 1, K ), 1, ONE, B( K, 1 ), LDB )
               CALL DGEMV( 'Transpose', K-1, NRHS, -ONE, B,
     $                     LDB, A( 1, K+1 ), 1, ONE, B( K+1, 1 ), LDB )
            END IF
*
*           Interchange rows K and -IPIV(K) THEN K+1 and -IPIV(K+1).
*
            KP = -IPIV( K )
            IF( KP.NE.K )
     $         CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
            KP = -IPIV( K+1 )
            IF( KP.NE.K+1 )
```

```
     $         CALL DSWAP( NRHS, B( K+1, 1 ), LDB, B( KP, 1 ), LDB )
*
           K = K + 2
        END IF
*
        GO TO 40
   50   CONTINUE
*
      ELSE
*
*       Solve A*X = B, where A = L*D*L**T.
*
*       First solve L*D*X = B, overwriting B with X.
*
*       K is the main loop index, increasing from 1 to N in steps of
*       1 or 2, depending on the size of the diagonal blocks.
*
        K = 1
   60   CONTINUE
*
*       If K > N, exit from loop.
*
        IF( K.GT.N )
     $     GO TO 80
*
        IF( IPIV( K ).GT.0 ) THEN
*
*          1 x 1 diagonal block
*
*          Interchange rows K and IPIV(K).
*
           KP = IPIV( K )
           IF( KP.NE.K )
     $        CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
*          Multiply by inv(L(K)), where L(K) is the transformation
*          stored in column K of A.
*
           IF( K.LT.N )
     $        CALL DGER( N-K, NRHS, -ONE, A( K+1, K ), 1, B( K, 1 ),
     $                   LDB, B( K+1, 1 ), LDB )
*
*          Multiply by the inverse of the diagonal block.
*
           CALL DSCAL( NRHS, ONE / A( K, K ), B( K, 1 ), LDB )
           K = K + 1
        ELSE
*
*          2 x 2 diagonal block
*
```

```
*           Interchange rows K and -IPIV(K) THEN K+1 and -IPIV(K+1)
*
            KP = -IPIV( K )
            IF( KP.NE.K )
     $         CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
            KP = -IPIV( K+1 )
            IF( KP.NE.K+1 )
     $         CALL DSWAP( NRHS, B( K+1, 1 ), LDB, B( KP, 1 ), LDB )
*
*           Multiply by inv(L(K)), where L(K) is the transformation
*           stored in columns K and K+1 of A.
*
            IF( K.LT.N-1 ) THEN
               CALL DGER( N-K-1, NRHS, -ONE, A( K+2, K ), 1, B( K, 1 ),
     $                    LDB, B( K+2, 1 ), LDB )
               CALL DGER( N-K-1, NRHS, -ONE, A( K+2, K+1 ), 1,
     $                    B( K+1, 1 ), LDB, B( K+2, 1 ), LDB )
            END IF
*
*           Multiply by the inverse of the diagonal block.
*
            AKM1K = A( K+1, K )
            AKM1 = A( K, K ) / AKM1K
            AK = A( K+1, K+1 ) / AKM1K
            DENOM = AKM1*AK - ONE
            DO 70 J = 1, NRHS
               BKM1 = B( K, J ) / AKM1K
               BK = B( K+1, J ) / AKM1K
               B( K, J ) = ( AK*BKM1-BK ) / DENOM
               B( K+1, J ) = ( AKM1*BK-BKM1 ) / DENOM
   70       CONTINUE
            K = K + 2
         END IF
*
         GO TO 60
   80    CONTINUE
*
*        Next solve L**T *X = B, overwriting B with X.
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = N
   90    CONTINUE
*
*        If K < 1, exit from loop.
*
         IF( K.LT.1 )
     $      GO TO 100
```

```
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Multiply by inv(L**T(K)), where L(K) is the transformation
*           stored in column K of A.
*
            IF( K.LT.N )
     $         CALL DGEMV( 'Transpose', N-K, NRHS, -ONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K ), 1, ONE, B( K, 1 ), LDB )
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K = K - 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Multiply by inv(L**T(K-1)), where L(K-1) is the transformation
*           stored in columns K-1 and K of A.
*
            IF( K.LT.N ) THEN
               CALL DGEMV( 'Transpose', N-K, NRHS, -ONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K ), 1, ONE, B( K, 1 ), LDB )
               CALL DGEMV( 'Transpose', N-K, NRHS, -ONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K-1 ), 1, ONE, B( K-1, 1 ),
     $                     LDB )
            END IF
*
*           Interchange rows K and -IPIV(K) THEN K-1 and -IPIV(K-1)
*
            KP = -IPIV( K )
            IF( KP.NE.K )
     $         CALL DSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
            KP = -IPIV( K-1 )
            IF( KP.NE.K-1 )
     $         CALL DSWAP( NRHS, B( K-1, 1 ), LDB, B( KP, 1 ), LDB )
*
            K = K - 2
         END IF
*
         GO TO 90
  100    CONTINUE
      END IF
*
```

```
      RETURN
*
*     End of DSYTRS_ROOK
*
      END
```

———-

58.27.4 lisp code

— **dsytrs_rook.lisp** —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dsytrs_rook (uplo n nrhs a lda ipiv b ldb$ info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array double-float (*)) b a)
             (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((ak 0.0d0) (akm1 0.0d0) (akm1k 0.0d0) (bk 0.0d0) (bkm1 0.0d0)
             (denom 0.0d0) (j 0) (k 0) (kp 0) (upper nil))
        (declare (type (double-float) ak akm1 akm1k bk bkm1 denom)
                 (type (f2cl-lib:integer4) j k kp)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
```

```
  (setf info -5))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8)))
(cond
 ((/= info 0) (xerbla "DSYTRS_ROOK" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= n 0) (= nrhs 0))
    (go end_label))
(cond
 (upper
  (tagbody
    (setf k n)
   label10
    (if (< k 1)
        (go label30))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dswap nrhs
               (f2cl-lib:array-slice b-%data% double-float (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dger (f2cl-lib:int-sub k 1) nrhs (- one)
           (f2cl-lib:array-slice a-%data% double-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice b-%data% double-float (k 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% double-float (1 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
        (when var-1 (setf nrhs var-1))
        (when var-6 (setf ldb$ var-6))
        (when var-8 (setf ldb$ var-8)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal nrhs
           (/ one
```

```
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
       (f2cl-lib:array-slice b-%data% double-float (k 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-1 var-2))
    (when var-0 (setf nrhs var-0))
    (when var-3 (setf ldb$ var-3)))
  (setf k (f2cl-lib:int-sub k 1)))
 (t
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
  (if (/= kp k)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dswap nrhs
           (f2cl-lib:array-slice b-%data% double-float (k 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-3))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                          ((1 *)) ipiv-%offset%)))
  (if (/= kp (f2cl-lib:int-sub k 1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dswap nrhs
           (f2cl-lib:array-slice b-%data% double-float
                                 ((+ k (f2cl-lib:int-sub 1)) 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-3))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
  (cond
   ((> k 2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dger (f2cl-lib:int-sub k 2) nrhs (- one)
         (f2cl-lib:array-slice a-%data% double-float (1 k)
```

```
                                ((1 lda) (1 *)) a-%offset%)
        1
        (f2cl-lib:array-slice b-%data% double-float (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
        ldb$
        (f2cl-lib:array-slice b-%data% double-float (1 1)
                                ((1 ldb$) (1 *)) b-%offset%)
        ldb$)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
    (when var-1 (setf nrhs var-1))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldb$ var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dger (f2cl-lib:int-sub k 2) nrhs (- one)
        (f2cl-lib:array-slice a-%data% double-float
                                (1 (f2cl-lib:int-sub k 1))
                                ((1 lda) (1 *)) a-%offset%)
        1
        (f2cl-lib:array-slice b-%data% double-float
                                ((+ k (f2cl-lib:int-sub 1)) 1)
                                ((1 ldb$) (1 *)) b-%offset%)
        ldb$
        (f2cl-lib:array-slice b-%data% double-float (1 1)
                                ((1 ldb$) (1 *)) b-%offset%)
        ldb$)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
    (when var-1 (setf nrhs var-1))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldb$ var-8)))))
(setf akm1k
        (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                        ((1 lda) (1 *)) a-%offset%))
(setf akm1
        (/
         (f2cl-lib:fref a-%data%
                         ((f2cl-lib:int-sub k 1)
                          (f2cl-lib:int-sub k 1))
                         ((1 lda) (1 *)) a-%offset%)
         akm1k))
(setf ak
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%)
         akm1k))
(setf denom (- (* akm1 ak) one))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf bkm1
```

```
                (/
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1) j)
                                ((1 ldb$) (1 *)) b-%offset%)
                 akm1k))
        (setf bk
                (/
                 (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                b-%offset%)
                 akm1k))
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)
                (/ (- (* ak bkm1) bk) denom))
        (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                             b-%offset%)
                (/ (- (* akm1 bk) bkm1) denom))
      label20))
   (setf k (f2cl-lib:int-sub k 2))))
 (go label10)
label30
 (setf k 1)
label40
 (if (> k n)
     (go label50))
 (cond
  ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
   (if (> k 1)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
            var-9 var-10)
           (dgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- one) b
            ldb$
            (f2cl-lib:array-slice a-%data% double-float (1 k)
                                  ((1 lda) (1 *)) a-%offset%)
            1 one
            (f2cl-lib:array-slice b-%data% double-float (k 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
         (declare
          (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
         (when var-2 (setf nrhs var-2))
         (when var-5 (setf ldb$ var-5))
         (when var-8 (setf one var-8))
         (when var-10 (setf ldb$ var-10))))
   (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
   (if (/= kp k)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (dswap nrhs
            (f2cl-lib:array-slice b-%data% double-float (k 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$
```

```
          (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
        (declare (ignore var-1 var-3))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
 (setf k (f2cl-lib:int-add k 1)))
(t
 (cond
  ((> k 1)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (dgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- one) b
        ldb$
        (f2cl-lib:array-slice a-%data% double-float (1 k)
                              ((1 lda) (1 *)) a-%offset%)
        1 one
        (f2cl-lib:array-slice b-%data% double-float (k 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$)
     (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
     (when var-2 (setf nrhs var-2))
     (when var-5 (setf ldb$ var-5))
     (when var-8 (setf one var-8))
     (when var-10 (setf ldb$ var-10)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (dgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- one) b
        ldb$
        (f2cl-lib:array-slice a-%data% double-float
                              (1 (f2cl-lib:int-add k 1))
                              ((1 lda) (1 *)) a-%offset%)
        1 one
        (f2cl-lib:array-slice b-%data% double-float ((+ k 1) 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$)
     (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
     (when var-2 (setf nrhs var-2))
     (when var-5 (setf ldb$ var-5))
     (when var-8 (setf one var-8))
     (when var-10 (setf ldb$ var-10)))))
 (setf kp
         (f2cl-lib:int-sub
          (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
 (if (/= kp k)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (dswap nrhs
```

```
                (f2cl-lib:array-slice b-%data% double-float (k 1)
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$
                (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$)
              (declare (ignore var-1 var-3))
              (when var-0 (setf nrhs var-0))
              (when var-2 (setf ldb$ var-2))
              (when var-4 (setf ldb$ var-4))))
      (setf kp
              (f2cl-lib:int-sub
                (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
                               ((1 *)) ipiv-%offset%)))
      (if (/= kp (f2cl-lib:int-add k 1))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dswap nrhs
                (f2cl-lib:array-slice b-%data% double-float ((+ k 1) 1)
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$
                (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$)
              (declare (ignore var-1 var-3))
              (when var-0 (setf nrhs var-0))
              (when var-2 (setf ldb$ var-2))
              (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-add k 2))))
    (go label40)
   label50))
 (t
  (tagbody
    (setf k 1)
   label60
    (if (> k n)
        (go label80))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dswap nrhs
                (f2cl-lib:array-slice b-%data% double-float (k 1)
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$
                (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$)
              (declare (ignore var-1 var-3))
              (when var-0 (setf nrhs var-0))
```

```
      (when var-2 (setf ldb$ var-2))
      (when var-4 (setf ldb$ var-4))))
 (if (< k n)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (dger (f2cl-lib:int-sub n k) nrhs (- one)
          (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                                ((1 lda) (1 *)) a-%offset%)
          1
          (f2cl-lib:array-slice b-%data% double-float (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice b-%data% double-float ((+ k 1) 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
      (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
      (when var-1 (setf nrhs var-1))
      (when var-6 (setf ldb$ var-6))
      (when var-8 (setf ldb$ var-8))))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (dscal nrhs
      (/ one
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%))
      (f2cl-lib:array-slice b-%data% double-float (k 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
   (declare (ignore var-1 var-2))
   (when var-0 (setf nrhs var-0))
   (when var-3 (setf ldb$ var-3)))
 (setf k (f2cl-lib:int-add k 1)))
(t
 (setf kp
         (f2cl-lib:int-sub
          (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
 (if (/= kp k)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (dswap nrhs
          (f2cl-lib:array-slice b-%data% double-float (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
      (declare (ignore var-1 var-3))
      (when var-0 (setf nrhs var-0))
      (when var-2 (setf ldb$ var-2))
      (when var-4 (setf ldb$ var-4))))
 (setf kp
         (f2cl-lib:int-sub
```

```
          (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
                         ((1 *)) ipiv-%offset%)))
(if (/= kp (f2cl-lib:int-add k 1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dswap nrhs
         (f2cl-lib:array-slice b-%data% double-float ((+ k 1) 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice b-%data% double-float (kp 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-1 var-3))
      (when var-0 (setf nrhs var-0))
      (when var-2 (setf ldb$ var-2))
      (when var-4 (setf ldb$ var-4))))
(cond
 ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dger (f2cl-lib:int-sub n k 1) nrhs (- one)
       (f2cl-lib:array-slice a-%data% double-float ((+ k 2) k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice b-%data% double-float (k 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice b-%data% double-float ((+ k 2) 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
    (when var-1 (setf nrhs var-1))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldb$ var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dger (f2cl-lib:int-sub n k 1) nrhs (- one)
       (f2cl-lib:array-slice a-%data% double-float
                             ((+ k 2) (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice b-%data% double-float ((+ k 1) 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice b-%data% double-float ((+ k 2) 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
    (when var-1 (setf nrhs var-1))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldb$ var-8)))))
```

```
    (setf akm1k
            (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                           ((1 lda) (1 *)) a-%offset%))
    (setf akm1
            (/
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)
             akm1k))
    (setf ak
            (/
             (f2cl-lib:fref a-%data%
                            ((f2cl-lib:int-add k 1)
                             (f2cl-lib:int-add k 1))
                            ((1 lda) (1 *)) a-%offset%)
             akm1k))
    (setf denom (- (* akm1 ak) one))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (setf bkm1
                (/
                 (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                b-%offset%)
                 akm1k))
        (setf bk
                (/
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                                ((1 ldb$) (1 *)) b-%offset%)
                 akm1k))
        (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                             b-%offset%)
                (/ (- (* ak bkm1) bk) denom))
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)
                (/ (- (* akm1 bk) bkm1) denom))
       label70))
    (setf k (f2cl-lib:int-add k 2))))
  (go label60)
 label80
  (setf k n)
 label90
  (if (< k 1)
      (go label100))
  (cond
   ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
    (if (< k n)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10)
            (dgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- one)
```

```
          (f2cl-lib:array-slice b-%data% double-float ((+ k 1) 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                                ((1 lda) (1 *)) a-%offset%)
          1 one
          (f2cl-lib:array-slice b-%data% double-float (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
        (declare
         (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
        (when var-2 (setf nrhs var-2))
        (when var-5 (setf ldb$ var-5))
        (when var-8 (setf one var-8))
        (when var-10 (setf ldb$ var-10))))
  (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
  (if (/= kp k)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dswap nrhs
          (f2cl-lib:array-slice b-%data% double-float (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice b-%data% double-float (kp 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
        (declare (ignore var-1 var-3))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
  (setf k (f2cl-lib:int-sub k 1)))
 (t
  (cond
   ((< k n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10)
        (dgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- one)
         (f2cl-lib:array-slice b-%data% double-float ((+ k 1) 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice a-%data% double-float ((+ k 1) k)
                               ((1 lda) (1 *)) a-%offset%)
         1 one
         (f2cl-lib:array-slice b-%data% double-float (k 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
      (when var-2 (setf nrhs var-2))
      (when var-5 (setf ldb$ var-5))
      (when var-8 (setf one var-8))
```

```
    (when var-10 (setf ldb$ var-10)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (dgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- one)
       (f2cl-lib:array-slice b-%data% double-float ((+ k 1) 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice a-%data% double-float
                             ((+ k 1) (f2cl-lib:int-sub k 1))
                             ((1 lda) (1 *)) a-%offset%)
       1 one
       (f2cl-lib:array-slice b-%data% double-float
                             ((+ k (f2cl-lib:int-sub 1)) 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
    (when var-2 (setf nrhs var-2))
    (when var-5 (setf ldb$ var-5))
    (when var-8 (setf one var-8))
    (when var-10 (setf ldb$ var-10)))))
(setf kp
        (f2cl-lib:int-sub
         (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
(if (/= kp k)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dswap nrhs
         (f2cl-lib:array-slice b-%data% double-float (k 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice b-%data% double-float (kp 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-1 var-3))
      (when var-0 (setf nrhs var-0))
      (when var-2 (setf ldb$ var-2))
      (when var-4 (setf ldb$ var-4))))
(setf kp
        (f2cl-lib:int-sub
         (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                        ((1 *)) ipiv-%offset%)))
(if (/= kp (f2cl-lib:int-sub k 1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dswap nrhs
         (f2cl-lib:array-slice b-%data% double-float
                               ((+ k (f2cl-lib:int-sub 1)) 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice b-%data% double-float (kp 1)
                               ((1 ldb$) (1 *)) b-%offset%)
```

```
                      ldb$)
                    (declare (ignore var-1 var-3))
                    (when var-0 (setf nrhs var-0))
                    (when var-2 (setf ldb$ var-2))
                    (when var-4 (setf ldb$ var-4))))
              (setf k (f2cl-lib:int-sub k 2))))
            (go label90)
           label100)))
        (go end_label)
       end_label
        (return (values uplo nil nrhs nil nil nil nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsytrs_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil
                                              fortran-to-lisp::nrhs nil nil nil
                                              nil fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

58.28 dtgsyl

58.28.1 Axiom unit tests

— dtgsyl.input —

```
)set break resume
)sys rm -f dtgsyl.output
```

```
)spool dtgsyl.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.28.2 Axiom help page

— **dtgsyl.help** —

```
=====================================================================
dtgsyl examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dtgsyl (TRANS, IJOB, M, N, A, LDA, B, LDB, C, LDC, D, LDD,
           E, LDE, F, LDF, SCALE, DIF, WORK, LWORK, IWORK, INFO)
           DTGSYL

Function/Subroutine Documentation
   subroutine dtgsyl (character TRANS, integer IJOB, integer M, integer N,
       double precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldb, * ) B, integer LDB, double precision, dimension( ldc, *
       ) C, integer LDC, double precision, dimension( ldd, * ) D, integer LDD,
       double precision, dimension( lde, * ) E, integer LDE, double precision,
       dimension( ldf, * ) F, integer LDF, double precision SCALE, double
       precision DIF, double precision, dimension( * ) WORK, integer LWORK,
       integer, dimension( * ) IWORK, integer INFO)
       DTGSYL

Purpose:
   DTGSYL solves the generalized Sylvester equation:

                   A * R - L * B = scale * C                 (1)
                   D * R - L * E = scale * F

   where R and L are unknown m-by-n matrices, (A, D), (B, E) and
   (C, F) are given matrix pairs of size m-by-m, n-by-n and m-by-n,
   respectively, with real entries. (A, D) and (B, E) must be in
```

```
  generalized (real) Schur canonical form, i.e. A, B are upper quasi
  triangular and D, E are upper triangular.

  The solution (R, L) overwrites (C, F). 0 <= SCALE <= 1 is an output
  scaling factor chosen to avoid overflow.

  In matrix notation (1) is equivalent to solve  Zx = scale b, where
  Z is defined as

                 Z = [ kron(In, A)  -kron(B**T, Im) ]         (2)
                     [ kron(In, D)  -kron(E**T, Im) ].

  Here Ik is the identity matrix of size k and X**T is the transpose of
  X. kron(X, Y) is the Kronecker product between the matrices X and Y.

  If TRANS = 'T', DTGSYL solves the transposed system Z**T*y = scale*b,
  which is equivalent to solve for R and L in

                  A**T * R + D**T * L = scale * C           (3)
                  R * B**T + L * E**T = scale * -F

  This case (TRANS = 'T') is used to compute an one-norm-based estimate
  of Dif[(A,D), (B,E)], the separation between the matrix pairs (A,D)
  and (B,E), using DLACON.

  If IJOB >= 1, DTGSYL computes a Frobenius norm-based estimate
  of Dif[(A,D),(B,E)]. That is, the reciprocal of a lower bound on the
  reciprocal of the smallest singular value of Z. See [1-2] for more
  information.

  This is a level 3 BLAS algorithm.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   = 'N', solve the generalized Sylvester equation (1).
   = 'T', solve the 'transposed' system (3).

 IJOB
   IJOB is INTEGER
   Specifies what kind of functionality to be performed.
   =0: solve (1) only.
   =1: The functionality of 0 and 3.
   =2: The functionality of 0 and 4.
   =3: Only an estimate of Dif[(A,D), (B,E)] is computed.
       (look ahead strategy IJOB  = 1 is used).
   =4: Only an estimate of Dif[(A,D), (B,E)] is computed.
       ( DGECON on sub-systems is used ).
   Not referenced if TRANS = 'T'.
```

```
M
  M is INTEGER
  The order of the matrices A and D, and the row dimension of
  the matrices C, F, R and L.

N
  N is INTEGER
  The order of the matrices B and E, and the column dimension
  of the matrices C, F, R and L.

A
  A is DOUBLE PRECISION array, dimension (LDA, M)
  The upper quasi triangular matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A. LDA >= max(1, M).

B
  B is DOUBLE PRECISION array, dimension (LDB, N)
  The upper quasi triangular matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= max(1, N).

C
  C is DOUBLE PRECISION array, dimension (LDC, N)
  On entry, C contains the right-hand-side of the first matrix
  equation in (1) or (3).
  On exit, if IJOB = 0, 1 or 2, C has been overwritten by
  the solution R. If IJOB = 3 or 4 and TRANS = 'N', C holds R,
  the solution achieved during the computation of the Dif-estimate.

LDC
  LDC is INTEGER
  The leading dimension of the array C. LDC >= max(1, M).

D
  D is DOUBLE PRECISION array, dimension (LDD, M)
  The upper triangular matrix D.

LDD
  LDD is INTEGER
  The leading dimension of the array D. LDD >= max(1, M).

E
  E is DOUBLE PRECISION array, dimension (LDE, N)
  The upper triangular matrix E.
```

```
LDE
  LDE is INTEGER
  The leading dimension of the array E. LDE >= max(1, N).

F
  F is DOUBLE PRECISION array, dimension (LDF, N)
  On entry, F contains the right-hand-side of the second matrix
  equation in (1) or (3).
  On exit, if IJOB = 0, 1 or 2, F has been overwritten by
  the solution L. If IJOB = 3 or 4 and TRANS = 'N', F holds L,
  the solution achieved during the computation of the Dif-estimate.

LDF
  LDF is INTEGER
  The leading dimension of the array F. LDF >= max(1, M).

DIF
  DIF is DOUBLE PRECISION
  On exit DIF is the reciprocal of a lower bound of the
  reciprocal of the Dif-function, i.e. DIF is an upper bound of
  Dif[(A,D), (B,E)] = sigma_min(Z), where Z as in (2).
  IF IJOB = 0 or TRANS = 'T', DIF is not touched.

SCALE
  SCALE is DOUBLE PRECISION
  On exit SCALE is the scaling factor in (1) or (3).
  If 0 < SCALE < 1, C and F hold the solutions R and L, resp.,
  to a slightly perturbed system but the input matrices A, B, D
  and E have not been changed. If SCALE = 0, C and F hold the
  solutions R and L, respectively, to the homogeneous system
  with C = F = 0. Normally, SCALE = 1.

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK > = 1.
  If IJOB = 1 or 2 and TRANS = 'N', LWORK >= max(1,2*M*N).

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (M+N+6)
```

```
 INFO
   INFO is INTEGER
   =0: successful exit
   <0: If INFO = -i, the i-th argument had an illegal value.
   >0: (A, D) and (B, E) have common or close eigenvalues.
```

———-

58.28.3 fortran code

— **dtgsyl.f** —

```
*  =====================================================================
      SUBROUTINE DTGSYL( TRANS, IJOB, M, N, A, LDA, B, LDB, C, LDC, D,
     $                   LDD, E, LDE, F, LDF, SCALE, DIF, WORK, LWORK,
     $                   IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            IJOB, INFO, LDA, LDB, LDC, LDD, LDE, LDF,
     $                   LWORK, M, N
      DOUBLE PRECISION   DIF, SCALE
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * ), C( LDC, * ),
     $                   D( LDD, * ), E( LDE, * ), F( LDF, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*  Replaced various illegal calls to DCOPY by calls to DLASET.
*  Sven Hammarling, 1/5/02.
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, NOTRAN
      INTEGER            I, IE, IFUNC, IROUND, IS, ISOLVE, J, JE, JS, K,
     $                   LINFO, LWMIN, MB, NB, P, PPQQ, PQ, Q
```

```
      DOUBLE PRECISION   DSCALE, DSUM, SCALE2, SCALOC
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEMM, DLACPY, DLASET, DSCAL, DTGSY2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode and test input parameters
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
*
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) ) THEN
         INFO = -1
      ELSE IF( NOTRAN ) THEN
         IF( ( IJOB.LT.0 ) .OR. ( IJOB.GT.4 ) ) THEN
            INFO = -2
         END IF
      END IF
      IF( INFO.EQ.0 ) THEN
         IF( M.LE.0 ) THEN
            INFO = -3
         ELSE IF( N.LE.0 ) THEN
            INFO = -4
         ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
            INFO = -6
         ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
            INFO = -8
         ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
            INFO = -10
         ELSE IF( LDD.LT.MAX( 1, M ) ) THEN
            INFO = -12
         ELSE IF( LDE.LT.MAX( 1, N ) ) THEN
            INFO = -14
         ELSE IF( LDF.LT.MAX( 1, M ) ) THEN
            INFO = -16
         END IF
      END IF
*
      IF( INFO.EQ.0 ) THEN
         IF( NOTRAN ) THEN
```

```
            IF( IJOB.EQ.1 .OR. IJOB.EQ.2 ) THEN
               LWMIN = MAX( 1, 2*M*N )
            ELSE
               LWMIN = 1
            END IF
         ELSE
            LWMIN = 1
         END IF
         WORK( 1 ) = LWMIN
*
         IF( LWORK.LT.LWMIN .AND. .NOT.LQUERY ) THEN
            INFO = -20
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DTGSYL', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         SCALE = 1
         IF( NOTRAN ) THEN
            IF( IJOB.NE.0 ) THEN
               DIF = 0
            END IF
         END IF
         RETURN
      END IF
*
*     Determine optimal block sizes MB and NB
*
      MB = ILAENV( 2, 'DTGSYL', TRANS, M, N, -1, -1 )
      NB = ILAENV( 5, 'DTGSYL', TRANS, M, N, -1, -1 )
*
      ISOLVE = 1
      IFUNC = 0
      IF( NOTRAN ) THEN
         IF( IJOB.GE.3 ) THEN
            IFUNC = IJOB - 2
            CALL DLASET( 'F', M, N, ZERO, ZERO, C, LDC )
            CALL DLASET( 'F', M, N, ZERO, ZERO, F, LDF )
         ELSE IF( IJOB.GE.1 ) THEN
            ISOLVE = 2
         END IF
      END IF
```

```
*
      IF( ( MB.LE.1 .AND. NB.LE.1 ) .OR. ( MB.GE.M .AND. NB.GE.N ) )
     $     THEN
*
         DO 30 IROUND = 1, ISOLVE
*
*           Use unblocked Level 2 solver
*
            DSCALE = ZERO
            DSUM = ONE
            PQ = 0
            CALL DTGSY2( TRANS, IFUNC, M, N, A, LDA, B, LDB, C, LDC, D,
     $                   LDD, E, LDE, F, LDF, SCALE, DSUM, DSCALE,
     $                   IWORK, PQ, INFO )
            IF( DSCALE.NE.ZERO ) THEN
               IF( IJOB.EQ.1 .OR. IJOB.EQ.3 ) THEN
                  DIF = SQRT( DBLE( 2*M*N ) ) / ( DSCALE*SQRT( DSUM ) )
               ELSE
                  DIF = SQRT( DBLE( PQ ) ) / ( DSCALE*SQRT( DSUM ) )
               END IF
            END IF
*
            IF( ISOLVE.EQ.2 .AND. IROUND.EQ.1 ) THEN
               IF( NOTRAN ) THEN
                  IFUNC = IJOB
               END IF
               SCALE2 = SCALE
               CALL DLACPY( 'F', M, N, C, LDC, WORK, M )
               CALL DLACPY( 'F', M, N, F, LDF, WORK( M*N+1 ), M )
               CALL DLASET( 'F', M, N, ZERO, ZERO, C, LDC )
               CALL DLASET( 'F', M, N, ZERO, ZERO, F, LDF )
            ELSE IF( ISOLVE.EQ.2 .AND. IROUND.EQ.2 ) THEN
               CALL DLACPY( 'F', M, N, WORK, M, C, LDC )
               CALL DLACPY( 'F', M, N, WORK( M*N+1 ), M, F, LDF )
               SCALE = SCALE2
            END IF
   30    CONTINUE
*
         RETURN
      END IF
*
*     Determine block structure of A
*
      P = 0
      I = 1
   40 CONTINUE
      IF( I.GT.M )
     $   GO TO 50
      P = P + 1
      IWORK( P ) = I
```

```
      I = I + MB
      IF( I.GE.M )
     $   GO TO 50
      IF( A( I, I-1 ).NE.ZERO )
     $   I = I + 1
      GO TO 40
   50 CONTINUE
*
      IWORK( P+1 ) = M + 1
      IF( IWORK( P ).EQ.IWORK( P+1 ) )
     $   P = P - 1
*
*     Determine block structure of B
*
      Q = P + 1
      J = 1
   60 CONTINUE
      IF( J.GT.N )
     $   GO TO 70
      Q = Q + 1
      IWORK( Q ) = J
      J = J + NB
      IF( J.GE.N )
     $   GO TO 70
      IF( B( J, J-1 ).NE.ZERO )
     $   J = J + 1
      GO TO 60
   70 CONTINUE
*
      IWORK( Q+1 ) = N + 1
      IF( IWORK( Q ).EQ.IWORK( Q+1 ) )
     $   Q = Q - 1
*
      IF( NOTRAN ) THEN
*
         DO 150 IROUND = 1, ISOLVE
*
*           Solve (I, J)-subsystem
*               A(I, I) * R(I, J) - L(I, J) * B(J, J) = C(I, J)
*               D(I, I) * R(I, J) - L(I, J) * E(J, J) = F(I, J)
*           for I = P, P - 1,..., 1; J = 1, 2,..., Q
*
            DSCALE = ZERO
            DSUM = ONE
            PQ = 0
            SCALE = ONE
            DO 130 J = P + 2, Q
               JS = IWORK( J )
               JE = IWORK( J+1 ) - 1
               NB = JE - JS + 1
```

```
               DO 120 I = P, 1, -1
                  IS = IWORK( I )
                  IE = IWORK( I+1 ) - 1
                  MB = IE - IS + 1
                  PPQQ = 0
                  CALL DTGSY2( TRANS, IFUNC, MB, NB, A( IS, IS ), LDA,
     $                         B( JS, JS ), LDB, C( IS, JS ), LDC,
     $                         D( IS, IS ), LDD, E( JS, JS ), LDE,
     $                         F( IS, JS ), LDF, SCALOC, DSUM, DSCALE,
     $                         IWORK( Q+2 ), PPQQ, LINFO )
                  IF( LINFO.GT.0 )
     $               INFO = LINFO
*
                  PQ = PQ + PPQQ
                  IF( SCALOC.NE.ONE ) THEN
                     DO 80 K = 1, JS - 1
                        CALL DSCAL( M, SCALOC, C( 1, K ), 1 )
                        CALL DSCAL( M, SCALOC, F( 1, K ), 1 )
   80                CONTINUE
                     DO 90 K = JS, JE
                        CALL DSCAL( IS-1, SCALOC, C( 1, K ), 1 )
                        CALL DSCAL( IS-1, SCALOC, F( 1, K ), 1 )
   90                CONTINUE
                     DO 100 K = JS, JE
                        CALL DSCAL( M-IE, SCALOC, C( IE+1, K ), 1 )
                        CALL DSCAL( M-IE, SCALOC, F( IE+1, K ), 1 )
  100                CONTINUE
                     DO 110 K = JE + 1, N
                        CALL DSCAL( M, SCALOC, C( 1, K ), 1 )
                        CALL DSCAL( M, SCALOC, F( 1, K ), 1 )
  110                CONTINUE
                     SCALE = SCALE*SCALOC
                  END IF
*
*                 Substitute R(I, J) and L(I, J) into remaining
*                 equation.
*
                  IF( I.GT.1 ) THEN
                     CALL DGEMM( 'N', 'N', IS-1, NB, MB, -ONE,
     $                           A( 1, IS ), LDA, C( IS, JS ), LDC, ONE,
     $                           C( 1, JS ), LDC )
                     CALL DGEMM( 'N', 'N', IS-1, NB, MB, -ONE,
     $                           D( 1, IS ), LDD, C( IS, JS ), LDC, ONE,
     $                           F( 1, JS ), LDF )
                  END IF
                  IF( J.LT.Q ) THEN
                     CALL DGEMM( 'N', 'N', MB, N-JE, NB, ONE,
     $                           F( IS, JS ), LDF, B( JS, JE+1 ), LDB,
     $                           ONE, C( IS, JE+1 ), LDC )
                     CALL DGEMM( 'N', 'N', MB, N-JE, NB, ONE,
```

```
     $                          F( IS, JS ), LDF, E( JS, JE+1 ), LDE,
     $                          ONE, F( IS, JE+1 ), LDF )
                 END IF
  120         CONTINUE
  130      CONTINUE
           IF( DSCALE.NE.ZERO ) THEN
              IF( IJOB.EQ.1 .OR. IJOB.EQ.3 ) THEN
                 DIF = SQRT( DBLE( 2*M*N ) ) / ( DSCALE*SQRT( DSUM ) )
              ELSE
                 DIF = SQRT( DBLE( PQ ) ) / ( DSCALE*SQRT( DSUM ) )
              END IF
           END IF
           IF( ISOLVE.EQ.2 .AND. IROUND.EQ.1 ) THEN
              IF( NOTRAN ) THEN
                 IFUNC = IJOB
              END IF
              SCALE2 = SCALE
              CALL DLACPY( 'F', M, N, C, LDC, WORK, M )
              CALL DLACPY( 'F', M, N, F, LDF, WORK( M*N+1 ), M )
              CALL DLASET( 'F', M, N, ZERO, ZERO, C, LDC )
              CALL DLASET( 'F', M, N, ZERO, ZERO, F, LDF )
           ELSE IF( ISOLVE.EQ.2 .AND. IROUND.EQ.2 ) THEN
              CALL DLACPY( 'F', M, N, WORK, M, C, LDC )
              CALL DLACPY( 'F', M, N, WORK( M*N+1 ), M, F, LDF )
              SCALE = SCALE2
           END IF
  150    CONTINUE
*
      ELSE
*
*        Solve transposed (I, J)-subsystem
*             A(I, I)**T * R(I, J)  + D(I, I)**T * L(I, J)  =  C(I, J)
*             R(I, J)  * B(J, J)**T + L(I, J)  * E(J, J)**T = -F(I, J)
*        for I = 1,2,..., P; J = Q, Q-1,..., 1
*
         SCALE = ONE
         DO 210 I = 1, P
            IS = IWORK( I )
            IE = IWORK( I+1 ) - 1
            MB = IE - IS + 1
            DO 200 J = Q, P + 2, -1
               JS = IWORK( J )
               JE = IWORK( J+1 ) - 1
               NB = JE - JS + 1
               CALL DTGSY2( TRANS, IFUNC, MB, NB, A( IS, IS ), LDA,
     $                      B( JS, JS ), LDB, C( IS, JS ), LDC,
     $                      D( IS, IS ), LDD, E( JS, JS ), LDE,
     $                      F( IS, JS ), LDF, SCALOC, DSUM, DSCALE,
     $                      IWORK( Q+2 ), PPQQ, LINFO )
               IF( LINFO.GT.0 )
```

```
     $            INFO = LINFO
               IF( SCALOC.NE.ONE ) THEN
                  DO 160 K = 1, JS - 1
                     CALL DSCAL( M, SCALOC, C( 1, K ), 1 )
                     CALL DSCAL( M, SCALOC, F( 1, K ), 1 )
  160             CONTINUE
                  DO 170 K = JS, JE
                     CALL DSCAL( IS-1, SCALOC, C( 1, K ), 1 )
                     CALL DSCAL( IS-1, SCALOC, F( 1, K ), 1 )
  170             CONTINUE
                  DO 180 K = JS, JE
                     CALL DSCAL( M-IE, SCALOC, C( IE+1, K ), 1 )
                     CALL DSCAL( M-IE, SCALOC, F( IE+1, K ), 1 )
  180             CONTINUE
                  DO 190 K = JE + 1, N
                     CALL DSCAL( M, SCALOC, C( 1, K ), 1 )
                     CALL DSCAL( M, SCALOC, F( 1, K ), 1 )
  190             CONTINUE
                  SCALE = SCALE*SCALOC
               END IF
*
*              Substitute R(I, J) and L(I, J) into remaining equation.
*
               IF( J.GT.P+2 ) THEN
                  CALL DGEMM( 'N', 'T', MB, JS-1, NB, ONE, C( IS, JS ),
     $                        LDC, B( 1, JS ), LDB, ONE, F( IS, 1 ),
     $                        LDF )
                  CALL DGEMM( 'N', 'T', MB, JS-1, NB, ONE, F( IS, JS ),
     $                        LDF, E( 1, JS ), LDE, ONE, F( IS, 1 ),
     $                        LDF )
               END IF
               IF( I.LT.P ) THEN
                  CALL DGEMM( 'T', 'N', M-IE, NB, MB, -ONE,
     $                        A( IS, IE+1 ), LDA, C( IS, JS ), LDC, ONE,
     $                        C( IE+1, JS ), LDC )
                  CALL DGEMM( 'T', 'N', M-IE, NB, MB, -ONE,
     $                        D( IS, IE+1 ), LDD, F( IS, JS ), LDF, ONE,
     $                        C( IE+1, JS ), LDC )
               END IF
  200       CONTINUE
  210    CONTINUE
*
      END IF
*
      WORK( 1 ) = LWMIN
*
      RETURN
*
*     End of DTGSYL
*
```

```
      END
```

———-

58.28.4 lisp code

— **dtgsyl.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dtgsyl
         (trans ijob m n a lda b ldb$ c ldc d ldd e lde f ldf scale dif work
          lwork iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (double-float) dif scale)
             (type (array double-float (*)) work f e d c b a)
             (type (f2cl-lib:integer4) info lwork ldf lde ldd ldc ldb$ lda n m
              ijob)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%)
         (c double-float c-%data% c-%offset%)
         (d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%)
         (f double-float f-%data% f-%offset%)
         (work double-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((dscale 0.0d0) (dsum 0.0d0) (scale2 0.0d0) (scaloc 0.0d0) (i 0)
             (ie 0) (ifunc 0) (iround 0) (is 0) (isolve 0) (j 0) (je 0) (js 0)
             (k 0) (linfo 0) (lwmin 0) (mb 0) (nb 0) (p 0) (ppqq 0) (pq 0)
             (q 0) (lquery nil) (notran nil))
        (declare (type (double-float) dscale dsum scale2 scaloc)
                 (type (f2cl-lib:integer4) i ie ifunc iround is isolve j je js
                  k linfo lwmin mb nb p ppqq pq q)
                 (type f2cl-lib:logical lquery notran))
        (setf info 0)
        (setf notran
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame trans "N")
                  (declare (ignore var-1))
                  (when var-0 (setf trans var-0))
                  ret-val))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond
```

```
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -1))
 (notran (cond ((or (< ijob 0) (> ijob 4)) (setf info -2)))))
(cond
 ((= info 0)
  (cond ((<= m 0) (setf info -3)) ((<= n 0) (setf info -4))
        ((< lda
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -6))
        ((< ldb$
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -8))
        ((< ldc
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -10))
        ((< ldd
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -12))
        ((< lde
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -14))
        ((< ldf
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -16)))))
(cond
 ((= info 0)
  (cond
   (notran
    (cond
     ((or (= ijob 1) (= ijob 2))
      (setf lwmin
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 (f2cl-lib:int-mul 2 m n)))))
     (t (setf lwmin 1))))
   (t (setf lwmin 1)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwmin) 'double-float))
  (cond ((and (< lwork lwmin) (not lquery)) (setf info -20)))))
(cond
 ((/= info 0) (xerbla "DTGSYL" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond
 ((or (= m 0) (= n 0))
  (setf scale (coerce (the f2cl-lib:integer4 1) 'double-float))
```

```
  (cond
   (notran
    (cond
     ((/= ijob 0)
      (setf dif (coerce (the f2cl-lib:integer4 0) 'double-float))))))
  (go end_label)))
(setf mb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 2 "DTGSYL" trans m n -1 -1)
          (declare (ignore var-0 var-1 var-5 var-6))
          (when var-2 (setf trans var-2))
          (when var-3 (setf m var-3))
          (when var-4 (setf n var-4))
          ret-val))
(setf nb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 5 "DTGSYL" trans m n -1 -1)
          (declare (ignore var-0 var-1 var-5 var-6))
          (when var-2 (setf trans var-2))
          (when var-3 (setf m var-3))
          (when var-4 (setf n var-4))
          ret-val))
(setf isolve 1)
(setf ifunc 0)
(cond
 (notran
  (cond
   ((>= ijob 3) (setf ifunc (f2cl-lib:int-sub ijob 2))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "F" m n zero zero c ldc)
      (declare (ignore var-0 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldc var-6)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "F" m n zero zero f ldf)
      (declare (ignore var-0 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldf var-6))))
   ((>= ijob 1) (setf isolve 2)))))
(cond
 ((or (and (<= mb 1) (<= nb 1)) (and (>= mb m) (>= nb n)))
  (f2cl-lib:fdo (iround 1 (f2cl-lib:int-add iround 1))
```

```
            ((> iround isolve) nil)
(tagbody
  (setf dscale zero)
  (setf dsum one)
  (setf pq 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17
       var-18 var-19 var-20 var-21)
      (dtgsy2 trans ifunc m n a lda b ldb$ c ldc d ldd e lde f ldf
       scale dsum dscale iwork pq info)
    (declare
     (ignore var-3 var-4 var-6 var-8 var-10 var-12 var-14 var-19))
    (setf trans var-0)
    (setf ifunc var-1)
    (setf m var-2)
    (setf lda var-5)
    (setf ldb$ var-7)
    (setf ldc var-9)
    (setf ldd var-11)
    (setf lde var-13)
    (setf ldf var-15)
    (setf scale var-16)
    (setf dsum var-17)
    (setf dscale var-18)
    (setf pq var-20)
    (setf info var-21))
  (cond
   ((/= dscale zero)
    (cond
     ((or (= ijob 1) (= ijob 3))
      (setf dif
              (/
               (f2cl-lib:fsqrt
                (f2cl-lib:dble (f2cl-lib:int-mul 2 m n)))
               (* dscale (f2cl-lib:fsqrt dsum)))))
     (t
      (setf dif
              (/ (f2cl-lib:fsqrt (f2cl-lib:dble pq))
                 (* dscale (f2cl-lib:fsqrt dsum))))))))
  (cond
   ((and (= isolve 2) (= iround 1))
    (cond (notran (setf ifunc ijob))) (setf scale2 scale)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "F" m n c ldc work m)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf ldc var-4))
```

```
    (when var-6 (setf m var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "F" m n f ldf
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ (f2cl-lib:int-mul m n) 1))
                             ((1 *)) work-%offset%)
       m)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldf var-4))
    (when var-6 (setf m var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "F" m n zero zero c ldc)
    (declare (ignore var-0 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldc var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "F" m n zero zero f ldf)
    (declare (ignore var-0 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldf var-6))))
 ((and (= isolve 2) (= iround 2))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "F" m n work m c ldc)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf m var-4))
    (when var-6 (setf ldc var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "F" m n
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ (f2cl-lib:int-mul m n) 1))
                             ((1 *)) work-%offset%)
       m f ldf)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
```

```
            (when var-4 (setf m var-4))
            (when var-6 (setf ldf var-6)))
          (setf scale scale2)))
      label30))
    (go end_label)))
  (setf p 0)
  (setf i 1)
 label40
  (if (> i m)
      (go label50))
  (setf p (f2cl-lib:int-add p 1))
  (setf (f2cl-lib:fref iwork-%data% (p) ((1 *)) iwork-%offset%) i)
  (setf i (f2cl-lib:int-add i mb))
  (if (>= i m)
      (go label50))
  (if (/=
       (f2cl-lib:fref a-%data% (i (f2cl-lib:int-sub i 1)) ((1 lda) (1 *))
                      a-%offset%)
       zero)
      (setf i (f2cl-lib:int-add i 1)))
  (go label40)
 label50
  (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add p 1)) ((1 *))
                       iwork-%offset%)
          (f2cl-lib:int-add m 1))
  (if (= (f2cl-lib:fref iwork-%data% (p) ((1 *)) iwork-%offset%)
         (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add p 1)) ((1 *))
                        iwork-%offset%))
      (setf p (f2cl-lib:int-sub p 1)))
  (setf q (f2cl-lib:int-add p 1))
  (setf j 1)
 label60
  (if (> j n)
      (go label70))
  (setf q (f2cl-lib:int-add q 1))
  (setf (f2cl-lib:fref iwork-%data% (q) ((1 *)) iwork-%offset%) j)
  (setf j (f2cl-lib:int-add j nb))
  (if (>= j n)
      (go label70))
  (if (/=
       (f2cl-lib:fref b-%data% (j (f2cl-lib:int-sub j 1))
                      ((1 ldb$) (1 *)) b-%offset%)
       zero)
      (setf j (f2cl-lib:int-add j 1)))
  (go label60)
 label70
  (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add q 1)) ((1 *))
                       iwork-%offset%)
          (f2cl-lib:int-add n 1))
  (if (= (f2cl-lib:fref iwork-%data% (q) ((1 *)) iwork-%offset%)
```

```
          (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add q 1)) ((1 *))
                         iwork-%offset%))
      (setf q (f2cl-lib:int-sub q 1)))
  (cond
   (notran
    (f2cl-lib:fdo (iround 1 (f2cl-lib:int-add iround 1))
                  ((> iround isolve) nil)
      (tagbody
        (setf dscale zero)
        (setf dsum one)
        (setf pq 0)
        (setf scale one)
        (f2cl-lib:fdo (j (f2cl-lib:int-add p 2) (f2cl-lib:int-add j 1))
                      ((> j q) nil)
          (tagbody
            (setf js
                    (f2cl-lib:fref iwork-%data% (j) ((1 *))
                                   iwork-%offset%))
            (setf je
                    (f2cl-lib:int-sub
                     (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add j 1))
                                    ((1 *)) iwork-%offset%)
                     1))
            (setf nb (f2cl-lib:int-add (f2cl-lib:int-sub je js) 1))
            (f2cl-lib:fdo (i p (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                          ((> i 1) nil)
              (tagbody
                (setf is
                        (f2cl-lib:fref iwork-%data% (i) ((1 *))
                                       iwork-%offset%))
                (setf ie
                        (f2cl-lib:int-sub
                         (f2cl-lib:fref iwork-%data%
                                        ((f2cl-lib:int-add i 1)) ((1 *))
                                        iwork-%offset%)
                         1))
                (setf mb (f2cl-lib:int-add (f2cl-lib:int-sub ie is) 1))
                (setf ppqq 0)
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
                     var-8 var-9 var-10 var-11 var-12 var-13 var-14
                     var-15 var-16 var-17 var-18 var-19 var-20 var-21)
                    (dtgsy2 trans ifunc mb nb
                     (f2cl-lib:array-slice a-%data% double-float (is is)
                                           ((1 lda) (1 *)) a-%offset%)
                     lda
                     (f2cl-lib:array-slice b-%data% double-float (js js)
                                           ((1 ldb$) (1 *)) b-%offset%)
                     ldb$
                     (f2cl-lib:array-slice c-%data% double-float (is js)
```

```
                              ((1 ldc) (1 *)) c-%offset%)
        ldc
        (f2cl-lib:array-slice d-%data% double-float (is is)
                              ((1 ldd) (1 *)) d-%offset%)
        ldd
        (f2cl-lib:array-slice e-%data% double-float (js js)
                              ((1 lde) (1 *)) e-%offset%)
        lde
        (f2cl-lib:array-slice f-%data% double-float (is js)
                              ((1 ldf) (1 *)) f-%offset%)
        ldf scaloc dsum dscale
        (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                              ((+ q 2)) ((1 *))
                              iwork-%offset%)
        ppqq linfo)
    (declare
     (ignore var-3 var-4 var-6 var-8 var-10 var-12 var-14
      var-19))
    (setf trans var-0)
    (setf ifunc var-1)
    (setf mb var-2)
    (setf lda var-5)
    (setf ldb$ var-7)
    (setf ldc var-9)
    (setf ldd var-11)
    (setf lde var-13)
    (setf ldf var-15)
    (setf scaloc var-16)
    (setf dsum var-17)
    (setf dscale var-18)
    (setf ppqq var-20)
    (setf linfo var-21))
  (if (> linfo 0)
      (setf info linfo))
  (setf pq (f2cl-lib:int-add pq ppqq))
  (cond
   ((/= scaloc one)
    (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                  ((> k
                      (f2cl-lib:int-add js
                                        (f2cl-lib:int-sub
                                         1)))
                   nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal m scaloc
             (f2cl-lib:array-slice c-%data% double-float
                                   (1 k) ((1 ldc) (1 *))
                                   c-%offset%)
             1)
```

```
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal m scaloc
           (f2cl-lib:array-slice f-%data% double-float
                                 (1 k) ((1 ldf) (1 *))
                                 f-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
    label80))
  (f2cl-lib:fdo (k js (f2cl-lib:int-add k 1))
                ((> k je) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal (f2cl-lib:int-sub is 1) scaloc
           (f2cl-lib:array-slice c-%data% double-float
                                 (1 k) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-0 var-2 var-3))
        (when var-1 (setf scaloc var-1)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal (f2cl-lib:int-sub is 1) scaloc
           (f2cl-lib:array-slice f-%data% double-float
                                 (1 k) ((1 ldf) (1 *))
                                 f-%offset%)
           1)
        (declare (ignore var-0 var-2 var-3))
        (when var-1 (setf scaloc var-1)))
    label90))
  (f2cl-lib:fdo (k js (f2cl-lib:int-add k 1))
                ((> k je) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal (f2cl-lib:int-sub m ie) scaloc
           (f2cl-lib:array-slice c-%data% double-float
                                 ((+ ie 1) k)
                                 ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-0 var-2 var-3))
        (when var-1 (setf scaloc var-1)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal (f2cl-lib:int-sub m ie) scaloc
           (f2cl-lib:array-slice f-%data% double-float
                                 ((+ ie 1) k)
                                 ((1 ldf) (1 *))
```

```
                                    f-%offset%)
                 1)
               (declare (ignore var-0 var-2 var-3))
               (when var-1 (setf scaloc var-1)))
           label100))
        (f2cl-lib:fdo (k (f2cl-lib:int-add je 1)
                       (f2cl-lib:int-add k 1))
                      ((> k n) nil)
          (tagbody
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (dscal m scaloc
                 (f2cl-lib:array-slice c-%data% double-float
                                       (1 k) ((1 ldc) (1 *))
                                       c-%offset%)
                 1)
               (declare (ignore var-2 var-3))
               (when var-0 (setf m var-0))
               (when var-1 (setf scaloc var-1)))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (dscal m scaloc
                 (f2cl-lib:array-slice f-%data% double-float
                                       (1 k) ((1 ldf) (1 *))
                                       f-%offset%)
                 1)
               (declare (ignore var-2 var-3))
               (when var-0 (setf m var-0))
               (when var-1 (setf scaloc var-1)))
           label110))
        (setf scale (* scale scaloc))))
      (cond
       ((> i 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
             var-8 var-9 var-10 var-11 var-12)
            (dgemm "N" "N" (f2cl-lib:int-sub is 1) nb mb
             (- one)
             (f2cl-lib:array-slice a-%data% double-float (1 is)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice c-%data% double-float
                                   (is js) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc one
             (f2cl-lib:array-slice c-%data% double-float (1 js)
                                   ((1 ldc) (1 *)) c-%offset%)
             ldc)
          (declare
           (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
          (when var-3 (setf nb var-3))
          (when var-4 (setf mb var-4))
```

```
      (when var-7 (setf lda var-7))
      (when var-9 (setf ldc var-9))
      (when var-10 (setf one var-10))
      (when var-12 (setf ldc var-12)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
         var-8 var-9 var-10 var-11 var-12)
        (dgemm "N" "N" (f2cl-lib:int-sub is 1) nb mb
         (- one)
         (f2cl-lib:array-slice d-%data% double-float (1 is)
                               ((1 ldd) (1 *)) d-%offset%)
         ldd
         (f2cl-lib:array-slice c-%data% double-float
                               (is js) ((1 ldc) (1 *))
                               c-%offset%)
         ldc one
         (f2cl-lib:array-slice f-%data% double-float (1 js)
                               ((1 ldf) (1 *)) f-%offset%)
         ldf)
      (declare
       (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
      (when var-3 (setf nb var-3))
      (when var-4 (setf mb var-4))
      (when var-7 (setf ldd var-7))
      (when var-9 (setf ldc var-9))
      (when var-10 (setf one var-10))
      (when var-12 (setf ldf var-12)))))
  (cond
   ((< j q)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
         var-8 var-9 var-10 var-11 var-12)
        (dgemm "N" "N" mb (f2cl-lib:int-sub n je) nb one
         (f2cl-lib:array-slice f-%data% double-float
                               (is js) ((1 ldf) (1 *))
                               f-%offset%)
         ldf
         (f2cl-lib:array-slice b-%data% double-float
                               (js (f2cl-lib:int-add je 1))
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$ one
         (f2cl-lib:array-slice c-%data% double-float
                               (is (f2cl-lib:int-add je 1))
                               ((1 ldc) (1 *)) c-%offset%)
         ldc)
      (declare
       (ignore var-0 var-1 var-3 var-6 var-8 var-11))
      (when var-2 (setf mb var-2))
      (when var-4 (setf nb var-4))
      (when var-5 (setf one var-5))
```

```
                (when var-7 (setf ldf var-7))
                (when var-9 (setf ldb$ var-9))
                (when var-10 (setf one var-10))
                (when var-12 (setf ldc var-12)))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
                   var-8 var-9 var-10 var-11 var-12)
                  (dgemm "N" "N" mb (f2cl-lib:int-sub n je) nb one
                   (f2cl-lib:array-slice f-%data% double-float
                                         (is js) ((1 ldf) (1 *))
                                         f-%offset%)
                   ldf
                   (f2cl-lib:array-slice e-%data% double-float
                                         (js (f2cl-lib:int-add je 1))
                                         ((1 lde) (1 *)) e-%offset%)
                   lde one
                   (f2cl-lib:array-slice f-%data% double-float
                                         (is (f2cl-lib:int-add je 1))
                                         ((1 ldf) (1 *)) f-%offset%)
                   ldf)
                (declare
                 (ignore var-0 var-1 var-3 var-6 var-8 var-11))
                (when var-2 (setf mb var-2))
                (when var-4 (setf nb var-4))
                (when var-5 (setf one var-5))
                (when var-7 (setf ldf var-7))
                (when var-9 (setf lde var-9))
                (when var-10 (setf one var-10))
                (when var-12 (setf ldf var-12)))))
          label120))
      label130))
    (cond
     ((/= dscale zero)
      (cond
       ((or (= ijob 1) (= ijob 3))
        (setf dif
                (/
                 (f2cl-lib:fsqrt
                  (f2cl-lib:dble (f2cl-lib:int-mul 2 m n)))
                 (* dscale (f2cl-lib:fsqrt dsum)))))
       (t
        (setf dif
                (/ (f2cl-lib:fsqrt (f2cl-lib:dble pq))
                   (* dscale (f2cl-lib:fsqrt dsum))))))))
    (cond
     ((and (= isolve 2) (= iround 1))
      (cond (notran (setf ifunc ijob))) (setf scale2 scale)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlacpy "F" m n c ldc work m)
```

```
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldc var-4))
    (when var-6 (setf m var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "F" m n f ldf
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ (f2cl-lib:int-mul m n) 1))
                             ((1 *)) work-%offset%)
       m)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldf var-4))
    (when var-6 (setf m var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "F" m n zero zero c ldc)
    (declare (ignore var-0 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldc var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "F" m n zero zero f ldf)
    (declare (ignore var-0 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldf var-6))))
 ((and (= isolve 2) (= iround 2))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "F" m n work m c ldc)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf m var-4))
    (when var-6 (setf ldc var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "F" m n
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ (f2cl-lib:int-mul m n) 1))
                             ((1 *)) work-%offset%)
```

```
            m f ldf)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf m var-4))
          (when var-6 (setf ldf var-6)))
        (setf scale scale2)))
    label150)))
 (t (setf scale one)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i p) nil)
    (tagbody
      (setf is (f2cl-lib:fref iwork-%data% (i) ((1 *)) iwork-%offset%))
      (setf ie
              (f2cl-lib:int-sub
               (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add i 1))
                              ((1 *)) iwork-%offset%)
               1))
      (setf mb (f2cl-lib:int-add (f2cl-lib:int-sub ie is) 1))
      (f2cl-lib:fdo (j q (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                    ((> j (f2cl-lib:int-add p 2)) nil)
        (tagbody
          (setf js
                  (f2cl-lib:fref iwork-%data% (j) ((1 *))
                                 iwork-%offset%))
          (setf je
                  (f2cl-lib:int-sub
                   (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add j 1))
                                  ((1 *)) iwork-%offset%)
                   1))
          (setf nb (f2cl-lib:int-add (f2cl-lib:int-sub je js) 1))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
               var-17 var-18 var-19 var-20 var-21)
              (dtgsy2 trans ifunc mb nb
               (f2cl-lib:array-slice a-%data% double-float (is is)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice b-%data% double-float (js js)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice c-%data% double-float (is js)
                                     ((1 ldc) (1 *)) c-%offset%)
               ldc
               (f2cl-lib:array-slice d-%data% double-float (is is)
                                     ((1 ldd) (1 *)) d-%offset%)
               ldd
               (f2cl-lib:array-slice e-%data% double-float (js js)
                                     ((1 lde) (1 *)) e-%offset%)
```

```
      lde
      (f2cl-lib:array-slice f-%data% double-float (is js)
                            ((1 ldf) (1 *)) f-%offset%)
      ldf scaloc dsum dscale
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                            ((+ q 2)) ((1 *)) iwork-%offset%)
      ppqq linfo)
  (declare
   (ignore var-3 var-4 var-6 var-8 var-10 var-12 var-14
    var-19))
  (setf trans var-0)
  (setf ifunc var-1)
  (setf mb var-2)
  (setf lda var-5)
  (setf ldb$ var-7)
  (setf ldc var-9)
  (setf ldd var-11)
  (setf lde var-13)
  (setf ldf var-15)
  (setf scaloc var-16)
  (setf dsum var-17)
  (setf dscale var-18)
  (setf ppqq var-20)
  (setf linfo var-21))
(if (> linfo 0)
    (setf info linfo))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k
                    (f2cl-lib:int-add js
                                      (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal m scaloc
           (f2cl-lib:array-slice c-%data% double-float (1 k)
                                 ((1 ldc) (1 *)) c-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal m scaloc
           (f2cl-lib:array-slice f-%data% double-float (1 k)
                                 ((1 ldf) (1 *)) f-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
```

```
   label160))
(f2cl-lib:fdo (k js (f2cl-lib:int-add k 1))
              ((> k je) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal (f2cl-lib:int-sub is 1) scaloc
         (f2cl-lib:array-slice c-%data% double-float (1 k)
                               ((1 ldc) (1 *)) c-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3))
      (when var-1 (setf scaloc var-1)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal (f2cl-lib:int-sub is 1) scaloc
         (f2cl-lib:array-slice f-%data% double-float (1 k)
                               ((1 ldf) (1 *)) f-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3))
      (when var-1 (setf scaloc var-1)))
   label170))
(f2cl-lib:fdo (k js (f2cl-lib:int-add k 1))
              ((> k je) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal (f2cl-lib:int-sub m ie) scaloc
         (f2cl-lib:array-slice c-%data% double-float
                               ((+ ie 1) k) ((1 ldc) (1 *))
                               c-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3))
      (when var-1 (setf scaloc var-1)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal (f2cl-lib:int-sub m ie) scaloc
         (f2cl-lib:array-slice f-%data% double-float
                               ((+ ie 1) k) ((1 ldf) (1 *))
                               f-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3))
      (when var-1 (setf scaloc var-1)))
   label180))
(f2cl-lib:fdo (k (f2cl-lib:int-add je 1)
               (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal m scaloc
         (f2cl-lib:array-slice c-%data% double-float (1 k)
                               ((1 ldc) (1 *)) c-%offset%)
         1)
      (declare (ignore var-2 var-3))
      (when var-0 (setf m var-0))
```

```
          (when var-1 (setf scaloc var-1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal m scaloc
             (f2cl-lib:array-slice f-%data% double-float (1 k)
                                   ((1 ldf) (1 *)) f-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
      label190))
  (setf scale (* scale scaloc))))
(cond
 ((> j (f2cl-lib:int-add p 2))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "N" "T" mb (f2cl-lib:int-sub js 1) nb one
       (f2cl-lib:array-slice c-%data% double-float (is js)
                             ((1 ldc) (1 *)) c-%offset%)
       ldc
       (f2cl-lib:array-slice b-%data% double-float (1 js)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$ one
       (f2cl-lib:array-slice f-%data% double-float (is 1)
                             ((1 ldf) (1 *)) f-%offset%)
       ldf)
    (declare (ignore var-0 var-1 var-3 var-6 var-8 var-11))
    (when var-2 (setf mb var-2))
    (when var-4 (setf nb var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldc var-7))
    (when var-9 (setf ldb$ var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldf var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "N" "T" mb (f2cl-lib:int-sub js 1) nb one
       (f2cl-lib:array-slice f-%data% double-float (is js)
                             ((1 ldf) (1 *)) f-%offset%)
       ldf
       (f2cl-lib:array-slice e-%data% double-float (1 js)
                             ((1 lde) (1 *)) e-%offset%)
       lde one
       (f2cl-lib:array-slice f-%data% double-float (is 1)
                             ((1 ldf) (1 *)) f-%offset%)
       ldf)
    (declare (ignore var-0 var-1 var-3 var-6 var-8 var-11))
    (when var-2 (setf mb var-2))
    (when var-4 (setf nb var-4))
```

```
    (when var-5 (setf one var-5))
    (when var-7 (setf ldf var-7))
    (when var-9 (setf lde var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldf var-12)))))
(cond
 ((< i p)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "T" "N" (f2cl-lib:int-sub m ie) nb mb (- one)
       (f2cl-lib:array-slice a-%data% double-float
                             (is (f2cl-lib:int-add ie 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice c-%data% double-float (is js)
                             ((1 ldc) (1 *)) c-%offset%)
       ldc one
       (f2cl-lib:array-slice c-%data% double-float
                             ((+ ie 1) js) ((1 ldc) (1 *))
                             c-%offset%)
       ldc)
    (declare
     (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
    (when var-3 (setf nb var-3))
    (when var-4 (setf mb var-4))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldc var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldc var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "T" "N" (f2cl-lib:int-sub m ie) nb mb (- one)
       (f2cl-lib:array-slice d-%data% double-float
                             (is (f2cl-lib:int-add ie 1))
                             ((1 ldd) (1 *)) d-%offset%)
       ldd
       (f2cl-lib:array-slice f-%data% double-float (is js)
                             ((1 ldf) (1 *)) f-%offset%)
       ldf one
       (f2cl-lib:array-slice c-%data% double-float
                             ((+ ie 1) js) ((1 ldc) (1 *))
                             c-%offset%)
       ldc)
    (declare
     (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
    (when var-3 (setf nb var-3))
    (when var-4 (setf mb var-4))
    (when var-7 (setf ldd var-7))
```

```
                    (when var-9 (setf ldf var-9))
                    (when var-10 (setf one var-10))
                    (when var-12 (setf ldc var-12)))))
                label200))
            label210))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwmin) 'double-float))
        (go end_label)
       end_label
        (return
         (values trans nil m n nil lda nil ldb$ nil ldc nil ldd nil lde nil ldf
                 scale dif nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dtgsyl
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4)
                                             (double-float) (double-float)
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::integer4
                                              (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(fortran-to-lisp::trans nil
                                             fortran-to-lisp::m
                                             fortran-to-lisp::n nil
                                             fortran-to-lisp::lda nil
                                             fortran-to-lisp::ldb$ nil
                                             fortran-to-lisp::ldc nil
                                             fortran-to-lisp::ldd nil
                                             fortran-to-lisp::lde nil
```

```
                                                  fortran-to-lisp::ldf
                                                  fortran-to-lisp::scale
                                                  fortran-to-lisp::dif nil nil nil
                                                  fortran-to-lisp::info)
                                                :calls '(fortran-to-lisp::dtgsy2))))
```

———-

58.29 dtrsyl

58.29.1 Axiom unit tests

— **dtrsyl.input** —

```
)set break resume
)sys rm -f dtrsyl.output
)spool dtrsyl.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

58.29.2 Axiom help page

— **dtrsyl.help** —

```
======================================================================
dtrsyl examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dtrsyl (TRANA, TRANB, ISGN, M, N, A, LDA, B, LDB, C, LDC,
           SCALE, INFO)
```

```
          DTRSYL

Function/Subroutine Documentation
   subroutine dtrsyl (character TRANA, character TRANB, integer ISGN, integer
       M, integer N, double precision, dimension( lda, * ) A, integer LDA,
       double precision, dimension( ldb, * ) B, integer LDB, double precision,
       dimension( ldc, * ) C, integer LDC, double precision SCALE, integer
       INFO)
       DTRSYL

Purpose:
   DTRSYL solves the real Sylvester matrix equation:

      op(A)*X + X*op(B) = scale*C or
      op(A)*X - X*op(B) = scale*C,

   where op(A) = A or A**T, and  A and B are both upper quasi-
   triangular. A is M-by-M and B is N-by-N; the right hand side C and
   the solution X are M-by-N; and scale is an output scale factor, set
   <= 1 to avoid overflow in X.

   A and B must be in Schur canonical form (as returned by DHSEQR), that
   is, block upper triangular with 1-by-1 and 2-by-2 diagonal blocks;
   each 2-by-2 diagonal block has its diagonal elements equal and its
   off-diagonal elements of opposite sign.

Parameters:

 TRANA
   TRANA is CHARACTER*1
   Specifies the option op(A):
   = 'N': op(A) = A    (No transpose)
   = 'T': op(A) = A**T (Transpose)
   = 'C': op(A) = A**H (Conjugate transpose = Transpose)

 TRANB
   TRANB is CHARACTER*1
   Specifies the option op(B):
   = 'N': op(B) = B    (No transpose)
   = 'T': op(B) = B**T (Transpose)
   = 'C': op(B) = B**H (Conjugate transpose = Transpose)

 ISGN
   ISGN is INTEGER
   Specifies the sign in the equation:
   = +1: solve op(A)*X + X*op(B) = scale*C
   = -1: solve op(A)*X - X*op(B) = scale*C

 M
   M is INTEGER
```

```
  The order of the matrix A, and the number of rows in the
  matrices X and C. M >= 0.

N
  N is INTEGER
  The order of the matrix B, and the number of columns in the
  matrices X and C. N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,M)
  The upper quasi-triangular matrix A, in Schur canonical form.

LDA
  LDA is INTEGER
  The leading dimension of the array A. LDA >= max(1,M).

B
  B is DOUBLE PRECISION array, dimension (LDB,N)
  The upper quasi-triangular matrix B, in Schur canonical form.

LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= max(1,N).

C
  C is DOUBLE PRECISION array, dimension (LDC,N)
  On entry, the M-by-N right hand side matrix C.
  On exit, C is overwritten by the solution matrix X.

LDC
  LDC is INTEGER
  The leading dimension of the array C. LDC >= max(1,M)

SCALE
  SCALE is DOUBLE PRECISION
  The scale factor, scale, set <= 1 to avoid overflow in X.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  = 1: A and B have common or very close eigenvalues; perturbed
       values were used to solve the equation (but the matrices
       A and B are unchanged).
```

58.29.3 fortran code

— dtrsyl.f —

```
*  =====================================================================
      SUBROUTINE DTRSYL( TRANA, TRANB, ISGN, M, N, A, LDA, B, LDB, C,
     $                   LDC, SCALE, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANA, TRANB
      INTEGER            INFO, ISGN, LDA, LDB, LDC, M, N
      DOUBLE PRECISION   SCALE
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * ), C( LDC, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRNA, NOTRNB
      INTEGER            IERR, J, K, K1, K2, KNEXT, L, L1, L2, LNEXT
      DOUBLE PRECISION   A11, BIGNUM, DA11, DB, EPS, SCALOC, SGN, SMIN,
     $                   SMLNUM, SUML, SUMR, XNORM
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   DUM( 1 ), VEC( 2, 2 ), X( 2, 2 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DDOT, DLAMCH, DLANGE
      EXTERNAL           LSAME, DDOT, DLAMCH, DLANGE
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, DLALN2, DLASY2, DSCAL, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, MAX, MIN
*     ..
*     .. Executable Statements ..
```

```
*
*     Decode and Test input parameters
*
      NOTRNA = LSAME( TRANA, 'N' )
      NOTRNB = LSAME( TRANB, 'N' )
*
      INFO = 0
      IF( .NOT.NOTRNA .AND. .NOT.LSAME( TRANA, 'T' ) .AND. .NOT.
     $    LSAME( TRANA, 'C' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRNB .AND. .NOT.LSAME( TRANB, 'T' ) .AND. .NOT.
     $         LSAME( TRANB, 'C' ) ) THEN
         INFO = -2
      ELSE IF( ISGN.NE.1 .AND. ISGN.NE.-1 ) THEN
         INFO = -3
      ELSE IF( M.LT.0 ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
         INFO = -11
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DTRSYL', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      SCALE = ONE
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Set constants to control overflow
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SMLNUM*DBLE( M*N ) / EPS
      BIGNUM = ONE / SMLNUM
*
      SMIN = MAX( SMLNUM, EPS*DLANGE( 'M', M, M, A, LDA, DUM ),
     $       EPS*DLANGE( 'M', N, N, B, LDB, DUM ) )
*
      SGN = ISGN
```

```
*
      IF( NOTRNA .AND. NOTRNB ) THEN
*
*        Solve    A*X + ISGN*X*B = scale*C.
*
*        The (K,L)th block of X is determined starting from
*        bottom-left corner column by column by
*
*         A(K,K)*X(K,L) + ISGN*X(K,L)*B(L,L) = C(K,L) - R(K,L)
*
*        Where
*                  M                          L-1
*        R(K,L) = SUM [A(K,I)*X(I,L)] + ISGN*SUM [X(K,J)*B(J,L)].
*                I=K+1                        J=1
*
*        Start column loop (index = L)
*        L1 (L2) : column index of the first (first) row of X(K,L).
*
         LNEXT = 1
         DO 60 L = 1, N
            IF( L.LT.LNEXT )
     $         GO TO 60
            IF( L.EQ.N ) THEN
               L1 = L
               L2 = L
            ELSE
               IF( B( L+1, L ).NE.ZERO ) THEN
                  L1 = L
                  L2 = L + 1
                  LNEXT = L + 2
               ELSE
                  L1 = L
                  L2 = L
                  LNEXT = L + 1
               END IF
            END IF
*
            Start row loop (index = K)
            K1 (K2): row index of the first (last) row of X(K,L).
*
            KNEXT = M
            DO 50 K = M, 1, -1
               IF( K.GT.KNEXT )
     $            GO TO 50
               IF( K.EQ.1 ) THEN
                  K1 = K
                  K2 = K
               ELSE
                  IF( A( K, K-1 ).NE.ZERO ) THEN
                     K1 = K - 1
```

```
                  K2 = K
                  KNEXT = K - 2
               ELSE
                  K1 = K
                  K2 = K
                  KNEXT = K - 1
               END IF
            END IF
*
            IF( L1.EQ.L2 .AND. K1.EQ.K2 ) THEN
               SUML = DDOT( M-K1, A( K1, MIN( K1+1, M ) ), LDA,
     $                C( MIN( K1+1, M ), L1 ), 1 )
               SUMR = DDOT( L1-1, C( K1, 1 ), LDC, B( 1, L1 ), 1 )
               VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
               SCALOC = ONE
*
               A11 = A( K1, K1 ) + SGN*B( L1, L1 )
               DA11 = ABS( A11 )
               IF( DA11.LE.SMIN ) THEN
                  A11 = SMIN
                  DA11 = SMIN
                  INFO = 1
               END IF
               DB = ABS( VEC( 1, 1 ) )
               IF( DA11.LT.ONE .AND. DB.GT.ONE ) THEN
                  IF( DB.GT.BIGNUM*DA11 )
     $               SCALOC = ONE / DB
               END IF
               X( 1, 1 ) = ( VEC( 1, 1 )*SCALOC ) / A11
*
               IF( SCALOC.NE.ONE ) THEN
                  DO 10 J = 1, N
                     CALL DSCAL( M, SCALOC, C( 1, J ), 1 )
   10             CONTINUE
                  SCALE = SCALE*SCALOC
               END IF
               C( K1, L1 ) = X( 1, 1 )
*
            ELSE IF( L1.EQ.L2 .AND. K1.NE.K2 ) THEN
*
               SUML = DDOT( M-K2, A( K1, MIN( K2+1, M ) ), LDA,
     $                C( MIN( K2+1, M ), L1 ), 1 )
               SUMR = DDOT( L1-1, C( K1, 1 ), LDC, B( 1, L1 ), 1 )
               VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
*
               SUML = DDOT( M-K2, A( K2, MIN( K2+1, M ) ), LDA,
     $                C( MIN( K2+1, M ), L1 ), 1 )
               SUMR = DDOT( L1-1, C( K2, 1 ), LDC, B( 1, L1 ), 1 )
               VEC( 2, 1 ) = C( K2, L1 ) - ( SUML+SGN*SUMR )
*
```

```
                 CALL DLALN2( .FALSE., 2, 1, SMIN, ONE, A( K1, K1 ),
     $                        LDA, ONE, ONE, VEC, 2, -SGN*B( L1, L1 ),
     $                        ZERO, X, 2, SCALOC, XNORM, IERR )
                 IF( IERR.NE.0 )
     $              INFO = 1
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 20 J = 1, N
                       CALL DSCAL( M, SCALOC, C( 1, J ), 1 )
   20               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
                 C( K1, L1 ) = X( 1, 1 )
                 C( K2, L1 ) = X( 2, 1 )
*
              ELSE IF( L1.NE.L2 .AND. K1.EQ.K2 ) THEN
*
                 SUML = DDOT( M-K1, A( K1, MIN( K1+1, M ) ), LDA,
     $                  C( MIN( K1+1, M ), L1 ), 1 )
                 SUMR = DDOT( L1-1, C( K1, 1 ), LDC, B( 1, L1 ), 1 )
                 VEC( 1, 1 ) = SGN*( C( K1, L1 )-( SUML+SGN*SUMR ) )
*
                 SUML = DDOT( M-K1, A( K1, MIN( K1+1, M ) ), LDA,
     $                  C( MIN( K1+1, M ), L2 ), 1 )
                 SUMR = DDOT( L1-1, C( K1, 1 ), LDC, B( 1, L2 ), 1 )
                 VEC( 2, 1 ) = SGN*( C( K1, L2 )-( SUML+SGN*SUMR ) )
*
                 CALL DLALN2( .TRUE., 2, 1, SMIN, ONE, B( L1, L1 ),
     $                        LDB, ONE, ONE, VEC, 2, -SGN*A( K1, K1 ),
     $                        ZERO, X, 2, SCALOC, XNORM, IERR )
                 IF( IERR.NE.0 )
     $              INFO = 1
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 30 J = 1, N
                       CALL DSCAL( M, SCALOC, C( 1, J ), 1 )
   30               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
                 C( K1, L1 ) = X( 1, 1 )
                 C( K1, L2 ) = X( 2, 1 )
*
              ELSE IF( L1.NE.L2 .AND. K1.NE.K2 ) THEN
*
                 SUML = DDOT( M-K2, A( K1, MIN( K2+1, M ) ), LDA,
     $                  C( MIN( K2+1, M ), L1 ), 1 )
                 SUMR = DDOT( L1-1, C( K1, 1 ), LDC, B( 1, L1 ), 1 )
                 VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
*
                 SUML = DDOT( M-K2, A( K1, MIN( K2+1, M ) ), LDA,
```

```
     $                 C( MIN( K2+1, M ), L2 ), 1 )
                 SUMR = DDOT( L1-1, C( K1, 1 ), LDC, B( 1, L2 ), 1 )
                 VEC( 1, 2 ) = C( K1, L2 ) - ( SUML+SGN*SUMR )
*
                 SUML = DDOT( M-K2, A( K2, MIN( K2+1, M ) ), LDA,
     $                 C( MIN( K2+1, M ), L1 ), 1 )
                 SUMR = DDOT( L1-1, C( K2, 1 ), LDC, B( 1, L1 ), 1 )
                 VEC( 2, 1 ) = C( K2, L1 ) - ( SUML+SGN*SUMR )
*
                 SUML = DDOT( M-K2, A( K2, MIN( K2+1, M ) ), LDA,
     $                 C( MIN( K2+1, M ), L2 ), 1 )
                 SUMR = DDOT( L1-1, C( K2, 1 ), LDC, B( 1, L2 ), 1 )
                 VEC( 2, 2 ) = C( K2, L2 ) - ( SUML+SGN*SUMR )
*
                 CALL DLASY2( .FALSE., .FALSE., ISGN, 2, 2,
     $                        A( K1, K1 ), LDA, B( L1, L1 ), LDB, VEC,
     $                        2, SCALOC, X, 2, XNORM, IERR )
                 IF( IERR.NE.0 )
     $              INFO = 1
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 40 J = 1, N
                       CALL DSCAL( M, SCALOC, C( 1, J ), 1 )
   40               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
                 C( K1, L1 ) = X( 1, 1 )
                 C( K1, L2 ) = X( 1, 2 )
                 C( K2, L1 ) = X( 2, 1 )
                 C( K2, L2 ) = X( 2, 2 )
              END IF
*
   50       CONTINUE
*
   60    CONTINUE
*
      ELSE IF( .NOT.NOTRNA .AND. NOTRNB ) THEN
*
*        Solve    A**T *X + ISGN*X*B = scale*C.
*
*        The (K,L)th block of X is determined starting from
*        upper-left corner column by column by
*
*          A(K,K)**T*X(K,L) + ISGN*X(K,L)*B(L,L) = C(K,L) - R(K,L)
*
*        Where
*                   K-1        T                    L-1
*          R(K,L) = SUM [A(I,K)**T*X(I,L)] +ISGN*SUM [X(K,J)*B(J,L)]
*                   I=1                           J=1
*
```

```
*        Start column loop (index = L)
*        L1 (L2): column index of the first (last) row of X(K,L)
*
         LNEXT = 1
         DO 120 L = 1, N
            IF( L.LT.LNEXT )
     $         GO TO 120
            IF( L.EQ.N ) THEN
               L1 = L
               L2 = L
            ELSE
               IF( B( L+1, L ).NE.ZERO ) THEN
                  L1 = L
                  L2 = L + 1
                  LNEXT = L + 2
               ELSE
                  L1 = L
                  L2 = L
                  LNEXT = L + 1
               END IF
            END IF
*
*           Start row loop (index = K)
*           K1 (K2): row index of the first (last) row of X(K,L)
*
            KNEXT = 1
            DO 110 K = 1, M
               IF( K.LT.KNEXT )
     $            GO TO 110
               IF( K.EQ.M ) THEN
                  K1 = K
                  K2 = K
               ELSE
                  IF( A( K+1, K ).NE.ZERO ) THEN
                     K1 = K
                     K2 = K + 1
                     KNEXT = K + 2
                  ELSE
                     K1 = K
                     K2 = K
                     KNEXT = K + 1
                  END IF
               END IF
*
               IF( L1.EQ.L2 .AND. K1.EQ.K2 ) THEN
                  SUML = DDOT( K1-1, A( 1, K1 ), 1, C( 1, L1 ), 1 )
                  SUMR = DDOT( L1-1, C( K1, 1 ), LDC, B( 1, L1 ), 1 )
                  VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
                  SCALOC = ONE
*
```

```
                 A11 = A( K1, K1 ) + SGN*B( L1, L1 )
                 DA11 = ABS( A11 )
                 IF( DA11.LE.SMIN ) THEN
                    A11 = SMIN
                    DA11 = SMIN
                    INFO = 1
                 END IF
                 DB = ABS( VEC( 1, 1 ) )
                 IF( DA11.LT.ONE .AND. DB.GT.ONE ) THEN
                    IF( DB.GT.BIGNUM*DA11 )
     $                 SCALOC = ONE / DB
                 END IF
                 X( 1, 1 ) = ( VEC( 1, 1 )*SCALOC ) / A11
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 70 J = 1, N
                       CALL DSCAL( M, SCALOC, C( 1, J ), 1 )
   70               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
                 C( K1, L1 ) = X( 1, 1 )
*
              ELSE IF( L1.EQ.L2 .AND. K1.NE.K2 ) THEN
*
                 SUML = DDOT( K1-1, A( 1, K1 ), 1, C( 1, L1 ), 1 )
                 SUMR = DDOT( L1-1, C( K1, 1 ), LDC, B( 1, L1 ), 1 )
                 VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
*
                 SUML = DDOT( K1-1, A( 1, K2 ), 1, C( 1, L1 ), 1 )
                 SUMR = DDOT( L1-1, C( K2, 1 ), LDC, B( 1, L1 ), 1 )
                 VEC( 2, 1 ) = C( K2, L1 ) - ( SUML+SGN*SUMR )
*
                 CALL DLALN2( .TRUE., 2, 1, SMIN, ONE, A( K1, K1 ),
     $                        LDA, ONE, ONE, VEC, 2, -SGN*B( L1, L1 ),
     $                        ZERO, X, 2, SCALOC, XNORM, IERR )
                 IF( IERR.NE.0 )
     $              INFO = 1
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 80 J = 1, N
                       CALL DSCAL( M, SCALOC, C( 1, J ), 1 )
   80               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
                 C( K1, L1 ) = X( 1, 1 )
                 C( K2, L1 ) = X( 2, 1 )
*
              ELSE IF( L1.NE.L2 .AND. K1.EQ.K2 ) THEN
*
                 SUML = DDOT( K1-1, A( 1, K1 ), 1, C( 1, L1 ), 1 )
```

```
                 SUMR = DDOT( L1-1, C( K1, 1 ), LDC, B( 1, L1 ), 1 )
                 VEC( 1, 1 ) = SGN*( C( K1, L1 )-( SUML+SGN*SUMR ) )
*
                 SUML = DDOT( K1-1, A( 1, K1 ), 1, C( 1, L2 ), 1 )
                 SUMR = DDOT( L1-1, C( K1, 1 ), LDC, B( 1, L2 ), 1 )
                 VEC( 2, 1 ) = SGN*( C( K1, L2 )-( SUML+SGN*SUMR ) )
*
                 CALL DLALN2( .TRUE., 2, 1, SMIN, ONE, B( L1, L1 ),
     $                        LDB, ONE, ONE, VEC, 2, -SGN*A( K1, K1 ),
     $                        ZERO, X, 2, SCALOC, XNORM, IERR )
                 IF( IERR.NE.0 )
     $              INFO = 1
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 90 J = 1, N
                       CALL DSCAL( M, SCALOC, C( 1, J ), 1 )
   90               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
                 C( K1, L1 ) = X( 1, 1 )
                 C( K1, L2 ) = X( 2, 1 )
*
              ELSE IF( L1.NE.L2 .AND. K1.NE.K2 ) THEN
*
                 SUML = DDOT( K1-1, A( 1, K1 ), 1, C( 1, L1 ), 1 )
                 SUMR = DDOT( L1-1, C( K1, 1 ), LDC, B( 1, L1 ), 1 )
                 VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
*
                 SUML = DDOT( K1-1, A( 1, K1 ), 1, C( 1, L2 ), 1 )
                 SUMR = DDOT( L1-1, C( K1, 1 ), LDC, B( 1, L2 ), 1 )
                 VEC( 1, 2 ) = C( K1, L2 ) - ( SUML+SGN*SUMR )
*
                 SUML = DDOT( K1-1, A( 1, K2 ), 1, C( 1, L1 ), 1 )
                 SUMR = DDOT( L1-1, C( K2, 1 ), LDC, B( 1, L1 ), 1 )
                 VEC( 2, 1 ) = C( K2, L1 ) - ( SUML+SGN*SUMR )
*
                 SUML = DDOT( K1-1, A( 1, K2 ), 1, C( 1, L2 ), 1 )
                 SUMR = DDOT( L1-1, C( K2, 1 ), LDC, B( 1, L2 ), 1 )
                 VEC( 2, 2 ) = C( K2, L2 ) - ( SUML+SGN*SUMR )
*
                 CALL DLASY2( .TRUE., .FALSE., ISGN, 2, 2, A( K1, K1 ),
     $                        LDA, B( L1, L1 ), LDB, VEC, 2, SCALOC, X,
     $                        2, XNORM, IERR )
                 IF( IERR.NE.0 )
     $              INFO = 1
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 100 J = 1, N
                       CALL DSCAL( M, SCALOC, C( 1, J ), 1 )
  100               CONTINUE
```

```
                   SCALE = SCALE*SCALOC
                END IF
                C( K1, L1 ) = X( 1, 1 )
                C( K1, L2 ) = X( 1, 2 )
                C( K2, L1 ) = X( 2, 1 )
                C( K2, L2 ) = X( 2, 2 )
             END IF
*
  110       CONTINUE
  120    CONTINUE
*
      ELSE IF( .NOT.NOTRNA .AND. .NOT.NOTRNB ) THEN
*
*        Solve    A**T*X + ISGN*X*B**T = scale*C.
*
*        The (K,L)th block of X is determined starting from
*        top-right corner column by column by
*
*           A(K,K)**T*X(K,L) + ISGN*X(K,L)*B(L,L)**T = C(K,L) - R(K,L)
*
*        Where
*                     K-1                            N
*            R(K,L) = SUM [A(I,K)**T*X(I,L)] + ISGN*SUM [X(K,J)*B(L,J)**T].
*                     I=1                          J=L+1
*
*        Start column loop (index = L)
*        L1 (L2): column index of the first (last) row of X(K,L)
*
         LNEXT = N
         DO 180 L = N, 1, -1
            IF( L.GT.LNEXT )
     $         GO TO 180
            IF( L.EQ.1 ) THEN
               L1 = L
               L2 = L
            ELSE
               IF( B( L, L-1 ).NE.ZERO ) THEN
                  L1 = L - 1
                  L2 = L
                  LNEXT = L - 2
               ELSE
                  L1 = L
                  L2 = L
                  LNEXT = L - 1
               END IF
            END IF
*
            Start row loop (index = K)
            K1 (K2): row index of the first (last) row of X(K,L)
*
```

```
            KNEXT = 1
            DO 170 K = 1, M
               IF( K.LT.KNEXT )
     $            GO TO 170
               IF( K.EQ.M ) THEN
                  K1 = K
                  K2 = K
               ELSE
                  IF( A( K+1, K ).NE.ZERO ) THEN
                     K1 = K
                     K2 = K + 1
                     KNEXT = K + 2
                  ELSE
                     K1 = K
                     K2 = K
                     KNEXT = K + 1
                  END IF
               END IF
*
               IF( L1.EQ.L2 .AND. K1.EQ.K2 ) THEN
                  SUML = DDOT( K1-1, A( 1, K1 ), 1, C( 1, L1 ), 1 )
                  SUMR = DDOT( N-L1, C( K1, MIN( L1+1, N ) ), LDC,
     $                   B( L1, MIN( L1+1, N ) ), LDB )
                  VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
                  SCALOC = ONE
*
                  A11 = A( K1, K1 ) + SGN*B( L1, L1 )
                  DA11 = ABS( A11 )
                  IF( DA11.LE.SMIN ) THEN
                     A11 = SMIN
                     DA11 = SMIN
                     INFO = 1
                  END IF
                  DB = ABS( VEC( 1, 1 ) )
                  IF( DA11.LT.ONE .AND. DB.GT.ONE ) THEN
                     IF( DB.GT.BIGNUM*DA11 )
     $                  SCALOC = ONE / DB
                  END IF
                  X( 1, 1 ) = ( VEC( 1, 1 )*SCALOC ) / A11
*
                  IF( SCALOC.NE.ONE ) THEN
                     DO 130 J = 1, N
                        CALL DSCAL( M, SCALOC, C( 1, J ), 1 )
  130                CONTINUE
                     SCALE = SCALE*SCALOC
                  END IF
                  C( K1, L1 ) = X( 1, 1 )
*
               ELSE IF( L1.EQ.L2 .AND. K1.NE.K2 ) THEN
*
```

```
                 SUML = DDOT( K1-1, A( 1, K1 ), 1, C( 1, L1 ), 1 )
                 SUMR = DDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                  B( L1, MIN( L2+1, N ) ), LDB )
                 VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
*
                 SUML = DDOT( K1-1, A( 1, K2 ), 1, C( 1, L1 ), 1 )
                 SUMR = DDOT( N-L2, C( K2, MIN( L2+1, N ) ), LDC,
     $                  B( L1, MIN( L2+1, N ) ), LDB )
                 VEC( 2, 1 ) = C( K2, L1 ) - ( SUML+SGN*SUMR )
*
                 CALL DLALN2( .TRUE., 2, 1, SMIN, ONE, A( K1, K1 ),
     $                        LDA, ONE, ONE, VEC, 2, -SGN*B( L1, L1 ),
     $                        ZERO, X, 2, SCALOC, XNORM, IERR )
                 IF( IERR.NE.0 )
     $              INFO = 1
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 140 J = 1, N
                       CALL DSCAL( M, SCALOC, C( 1, J ), 1 )
  140               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
                 C( K1, L1 ) = X( 1, 1 )
                 C( K2, L1 ) = X( 2, 1 )
*
              ELSE IF( L1.NE.L2 .AND. K1.EQ.K2 ) THEN
*
                 SUML = DDOT( K1-1, A( 1, K1 ), 1, C( 1, L1 ), 1 )
                 SUMR = DDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                  B( L1, MIN( L2+1, N ) ), LDB )
                 VEC( 1, 1 ) = SGN*( C( K1, L1 )-( SUML+SGN*SUMR ) )
*
                 SUML = DDOT( K1-1, A( 1, K1 ), 1, C( 1, L2 ), 1 )
                 SUMR = DDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                  B( L2, MIN( L2+1, N ) ), LDB )
                 VEC( 2, 1 ) = SGN*( C( K1, L2 )-( SUML+SGN*SUMR ) )
*
                 CALL DLALN2( .FALSE., 2, 1, SMIN, ONE, B( L1, L1 ),
     $                        LDB, ONE, ONE, VEC, 2, -SGN*A( K1, K1 ),
     $                        ZERO, X, 2, SCALOC, XNORM, IERR )
                 IF( IERR.NE.0 )
     $              INFO = 1
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 150 J = 1, N
                       CALL DSCAL( M, SCALOC, C( 1, J ), 1 )
  150               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
                 C( K1, L1 ) = X( 1, 1 )
```

```
                 C( K1, L2 ) = X( 2, 1 )
*
              ELSE IF( L1.NE.L2 .AND. K1.NE.K2 ) THEN
*
                 SUML = DDOT( K1-1, A( 1, K1 ), 1, C( 1, L1 ), 1 )
                 SUMR = DDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                  B( L1, MIN( L2+1, N ) ), LDB )
                 VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
*
                 SUML = DDOT( K1-1, A( 1, K1 ), 1, C( 1, L2 ), 1 )
                 SUMR = DDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                  B( L2, MIN( L2+1, N ) ), LDB )
                 VEC( 1, 2 ) = C( K1, L2 ) - ( SUML+SGN*SUMR )
*
                 SUML = DDOT( K1-1, A( 1, K2 ), 1, C( 1, L1 ), 1 )
                 SUMR = DDOT( N-L2, C( K2, MIN( L2+1, N ) ), LDC,
     $                  B( L1, MIN( L2+1, N ) ), LDB )
                 VEC( 2, 1 ) = C( K2, L1 ) - ( SUML+SGN*SUMR )
*
                 SUML = DDOT( K1-1, A( 1, K2 ), 1, C( 1, L2 ), 1 )
                 SUMR = DDOT( N-L2, C( K2, MIN( L2+1, N ) ), LDC,
     $                  B( L2, MIN( L2+1, N ) ), LDB )
                 VEC( 2, 2 ) = C( K2, L2 ) - ( SUML+SGN*SUMR )
*
                 CALL DLASY2( .TRUE., .TRUE., ISGN, 2, 2, A( K1, K1 ),
     $                        LDA, B( L1, L1 ), LDB, VEC, 2, SCALOC, X,
     $                        2, XNORM, IERR )
                 IF( IERR.NE.0 )
     $              INFO = 1
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 160 J = 1, N
                       CALL DSCAL( M, SCALOC, C( 1, J ), 1 )
  160               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
                 C( K1, L1 ) = X( 1, 1 )
                 C( K1, L2 ) = X( 1, 2 )
                 C( K2, L1 ) = X( 2, 1 )
                 C( K2, L2 ) = X( 2, 2 )
              END IF
*
  170       CONTINUE
  180    CONTINUE
*
      ELSE IF( NOTRNA .AND. .NOT.NOTRNB ) THEN
*
*        Solve    A*X + ISGN*X*B**T = scale*C.
*
*        The (K,L)th block of X is determined starting from
```

```
*        bottom-right corner column by column by
*
*            A(K,K)*X(K,L) + ISGN*X(K,L)*B(L,L)**T = C(K,L) - R(K,L)
*
*        Where
*                      M                          N
*            R(K,L) = SUM [A(K,I)*X(I,L)] + ISGN*SUM [X(K,J)*B(L,J)**T].
*                   I=K+1                       J=L+1
*
*        Start column loop (index = L)
*        L1 (L2): column index of the first (last) row of X(K,L)
*
         LNEXT = N
         DO 240 L = N, 1, -1
            IF( L.GT.LNEXT )
     $         GO TO 240
            IF( L.EQ.1 ) THEN
               L1 = L
               L2 = L
            ELSE
               IF( B( L, L-1 ).NE.ZERO ) THEN
                  L1 = L - 1
                  L2 = L
                  LNEXT = L - 2
               ELSE
                  L1 = L
                  L2 = L
                  LNEXT = L - 1
               END IF
            END IF
*
*           Start row loop (index = K)
*           K1 (K2): row index of the first (last) row of X(K,L)
*
            KNEXT = M
            DO 230 K = M, 1, -1
               IF( K.GT.KNEXT )
     $            GO TO 230
               IF( K.EQ.1 ) THEN
                  K1 = K
                  K2 = K
               ELSE
                  IF( A( K, K-1 ).NE.ZERO ) THEN
                     K1 = K - 1
                     K2 = K
                     KNEXT = K - 2
                  ELSE
                     K1 = K
                     K2 = K
                     KNEXT = K - 1
```

```
                 END IF
              END IF
*
              IF( L1.EQ.L2 .AND. K1.EQ.K2 ) THEN
                 SUML = DDOT( M-K1, A( K1, MIN( K1+1, M ) ), LDA,
     $                   C( MIN( K1+1, M ), L1 ), 1 )
                 SUMR = DDOT( N-L1, C( K1, MIN( L1+1, N ) ), LDC,
     $                   B( L1, MIN( L1+1, N ) ), LDB )
                 VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
                 SCALOC = ONE
*
                 A11 = A( K1, K1 ) + SGN*B( L1, L1 )
                 DA11 = ABS( A11 )
                 IF( DA11.LE.SMIN ) THEN
                    A11 = SMIN
                    DA11 = SMIN
                    INFO = 1
                 END IF
                 DB = ABS( VEC( 1, 1 ) )
                 IF( DA11.LT.ONE .AND. DB.GT.ONE ) THEN
                    IF( DB.GT.BIGNUM*DA11 )
     $                 SCALOC = ONE / DB
                 END IF
                 X( 1, 1 ) = ( VEC( 1, 1 )*SCALOC ) / A11
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 190 J = 1, N
                       CALL DSCAL( M, SCALOC, C( 1, J ), 1 )
  190               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
                 C( K1, L1 ) = X( 1, 1 )
*
              ELSE IF( L1.EQ.L2 .AND. K1.NE.K2 ) THEN
*
                 SUML = DDOT( M-K2, A( K1, MIN( K2+1, M ) ), LDA,
     $                   C( MIN( K2+1, M ), L1 ), 1 )
                 SUMR = DDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                   B( L1, MIN( L2+1, N ) ), LDB )
                 VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
*
                 SUML = DDOT( M-K2, A( K2, MIN( K2+1, M ) ), LDA,
     $                   C( MIN( K2+1, M ), L1 ), 1 )
                 SUMR = DDOT( N-L2, C( K2, MIN( L2+1, N ) ), LDC,
     $                   B( L1, MIN( L2+1, N ) ), LDB )
                 VEC( 2, 1 ) = C( K2, L1 ) - ( SUML+SGN*SUMR )
*
                 CALL DLALN2( .FALSE., 2, 1, SMIN, ONE, A( K1, K1 ),
     $                        LDA, ONE, ONE, VEC, 2, -SGN*B( L1, L1 ),
     $                        ZERO, X, 2, SCALOC, XNORM, IERR )
```

```
                 IF( IERR.NE.0 )
     $              INFO = 1
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 200 J = 1, N
                       CALL DSCAL( M, SCALOC, C( 1, J ), 1 )
  200               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
                 C( K1, L1 ) = X( 1, 1 )
                 C( K2, L1 ) = X( 2, 1 )
*
              ELSE IF( L1.NE.L2 .AND. K1.EQ.K2 ) THEN
*
                 SUML = DDOT( M-K1, A( K1, MIN( K1+1, M ) ), LDA,
     $                  C( MIN( K1+1, M ), L1 ), 1 )
                 SUMR = DDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                  B( L1, MIN( L2+1, N ) ), LDB )
                 VEC( 1, 1 ) = SGN*( C( K1, L1 )-( SUML+SGN*SUMR ) )
*
                 SUML = DDOT( M-K1, A( K1, MIN( K1+1, M ) ), LDA,
     $                  C( MIN( K1+1, M ), L2 ), 1 )
                 SUMR = DDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                  B( L2, MIN( L2+1, N ) ), LDB )
                 VEC( 2, 1 ) = SGN*( C( K1, L2 )-( SUML+SGN*SUMR ) )
*
                 CALL DLALN2( .FALSE., 2, 1, SMIN, ONE, B( L1, L1 ),
     $                        LDB, ONE, ONE, VEC, 2, -SGN*A( K1, K1 ),
     $                        ZERO, X, 2, SCALOC, XNORM, IERR )
                 IF( IERR.NE.0 )
     $              INFO = 1
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 210 J = 1, N
                       CALL DSCAL( M, SCALOC, C( 1, J ), 1 )
  210               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
                 C( K1, L1 ) = X( 1, 1 )
                 C( K1, L2 ) = X( 2, 1 )
*
              ELSE IF( L1.NE.L2 .AND. K1.NE.K2 ) THEN
*
                 SUML = DDOT( M-K2, A( K1, MIN( K2+1, M ) ), LDA,
     $                  C( MIN( K2+1, M ), L1 ), 1 )
                 SUMR = DDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                  B( L1, MIN( L2+1, N ) ), LDB )
                 VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
*
                 SUML = DDOT( M-K2, A( K1, MIN( K2+1, M ) ), LDA,
```

```
     $                  C( MIN( K2+1, M ), L2 ), 1 )
                SUMR = DDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                 B( L2, MIN( L2+1, N ) ), LDB )
                VEC( 1, 2 ) = C( K1, L2 ) - ( SUML+SGN*SUMR )
*
                SUML = DDOT( M-K2, A( K2, MIN( K2+1, M ) ), LDA,
     $                  C( MIN( K2+1, M ), L1 ), 1 )
                SUMR = DDOT( N-L2, C( K2, MIN( L2+1, N ) ), LDC,
     $                 B( L1, MIN( L2+1, N ) ), LDB )
                VEC( 2, 1 ) = C( K2, L1 ) - ( SUML+SGN*SUMR )
*
                SUML = DDOT( M-K2, A( K2, MIN( K2+1, M ) ), LDA,
     $                  C( MIN( K2+1, M ), L2 ), 1 )
                SUMR = DDOT( N-L2, C( K2, MIN( L2+1, N ) ), LDC,
     $                 B( L2, MIN( L2+1, N ) ), LDB )
                VEC( 2, 2 ) = C( K2, L2 ) - ( SUML+SGN*SUMR )
*
                CALL DLASY2( .FALSE., .TRUE., ISGN, 2, 2, A( K1, K1 ),
     $                       LDA, B( L1, L1 ), LDB, VEC, 2, SCALOC, X,
     $                       2, XNORM, IERR )
                IF( IERR.NE.0 )
     $             INFO = 1
*
                IF( SCALOC.NE.ONE ) THEN
                   DO 220 J = 1, N
                      CALL DSCAL( M, SCALOC, C( 1, J ), 1 )
  220              CONTINUE
                   SCALE = SCALE*SCALOC
                END IF
                C( K1, L1 ) = X( 1, 1 )
                C( K1, L2 ) = X( 1, 2 )
                C( K2, L1 ) = X( 2, 1 )
                C( K2, L2 ) = X( 2, 2 )
             END IF
*
  230       CONTINUE
  240    CONTINUE
*
      END IF
*
      RETURN
*
*     End of DTRSYL
*
      END
```

———-

58.29.4 lisp code

— dtrsyl.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dtrsyl (trana tranb isgn m n a lda b ldb$ c ldc scale info)
    (declare (type (double-float) scale)
             (type (array double-float (*)) c b a)
             (type (f2cl-lib:integer4) info ldc ldb$ lda n m isgn)
             (type (string *) tranb trana))
    (f2cl-lib:with-multi-array-data
        ((trana character trana-%data% trana-%offset%)
         (tranb character tranb-%data% tranb-%offset%)
         (a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%)
         (c double-float c-%data% c-%offset%))
      (prog ((dum (make-array 1 :element-type 'double-float))
             (vec (make-array 4 :element-type 'double-float))
             (x (make-array 4 :element-type 'double-float)) (a11 0.0d0)
             (bignum 0.0d0) (da11 0.0d0) (db 0.0d0) (eps 0.0d0) (scaloc 0.0d0)
             (sgn 0.0d0) (smin 0.0d0) (smlnum 0.0d0) (suml 0.0d0) (sumr 0.0d0)
             (xnorm 0.0d0) (ierr 0) (j 0) (k 0) (k1 0) (k2 0) (knext 0) (l 0)
             (l1 0) (l2 0) (lnext 0) (notrna nil) (notrnb nil))
        (declare (type (array double-float (1)) dum)
                 (type (array double-float (4)) vec x)
                 (type (double-float) a11 bignum da11 db eps scaloc sgn smin
                  smlnum suml sumr xnorm)
                 (type (f2cl-lib:integer4) ierr j k k1 k2 knext l l1 l2 lnext)
                 (type f2cl-lib:logical notrna notrnb))
        (setf notrna
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame trana "N")
                  (declare (ignore var-1))
                  (when var-0 (setf trana var-0))
                  ret-val))
        (setf notrnb
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame tranb "N")
                  (declare (ignore var-1))
                  (when var-0 (setf tranb var-0))
                  ret-val))
        (setf info 0)
        (cond
         ((and (not notrna)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
```

```
              (lsame trana "T")
            (declare (ignore var-1))
            (when var-0 (setf trana var-0))
            ret-val))
        (not
         (multiple-value-bind (ret-val var-0 var-1)
              (lsame trana "C")
            (declare (ignore var-1))
            (when var-0 (setf trana var-0))
            ret-val)))
  (setf info -1))
 ((and (not notrnb)
        (not
         (multiple-value-bind (ret-val var-0 var-1)
              (lsame tranb "T")
            (declare (ignore var-1))
            (when var-0 (setf tranb var-0))
            ret-val))
        (not
         (multiple-value-bind (ret-val var-0 var-1)
              (lsame tranb "C")
            (declare (ignore var-1))
            (when var-0 (setf tranb var-0))
            ret-val)))
  (setf info -2))
 ((and (/= isgn 1) (/= isgn (f2cl-lib:int-sub 1))) (setf info -3))
 ((< m 0) (setf info -4)) ((< n 0) (setf info -5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9))
 ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -11)))
(cond
 ((/= info 0) (xerbla "DTRSYL" (f2cl-lib:int-sub info))
  (go end_label)))
(setf scale one)
(if (or (= m 0) (= n 0))
    (go end_label))
(setf eps (dlamch "P"))
(setf smlnum (dlamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (* smlnum (f2cl-lib:dble (f2cl-lib:int-mul m n))) eps))
(setf bignum (/ one smlnum))
(setf smin
```

```
          (max smlnum
               (* eps
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                      (dlange "M" m m a lda dum)
                    (declare (ignore var-0 var-3 var-5))
                    (when var-1 (setf m var-1))
                    (when var-2 (setf m var-2))
                    (when var-4 (setf lda var-4))
                    ret-val))
               (* eps
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                      (dlange "M" n n b ldb$ dum)
                    (declare (ignore var-0 var-3 var-5))
                    (when var-1 (setf n var-1))
                    (when var-2 (setf n var-2))
                    (when var-4 (setf ldb$ var-4))
                    ret-val))))
(setf sgn (coerce (the f2cl-lib:integer4 isgn) 'double-float))
(cond
 ((and notrna notrnb) (setf lnext 1)
  (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                ((> l n) nil)
    (tagbody
      (if (< l lnext)
          (go label60))
      (cond ((= l n) (setf l1 l) (setf l2 l))
            (t
             (cond
              ((/=
                (f2cl-lib:fref b ((f2cl-lib:int-add l 1) l)
                               ((1 ldb$) (1 *)))
                zero)
               (setf l1 l) (setf l2 (f2cl-lib:int-add l 1))
               (setf lnext (f2cl-lib:int-add l 2)))
              (t (setf l1 l) (setf l2 l)
               (setf lnext (f2cl-lib:int-add l 1))))))
      (setf knext m)
      (f2cl-lib:fdo (k m (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                    ((> k 1) nil)
        (tagbody
          (if (> k knext)
              (go label50))
          (cond ((= k 1) (setf k1 k) (setf k2 k))
                (t
                 (cond
                  ((/=
                    (f2cl-lib:fref a
                                   (k
```

```
                                (f2cl-lib:int-add k
                                                  (f2cl-lib:int-sub
                                                   1)))
                              ((1 lda) (1 *)))
                 zero)
                (setf k1 (f2cl-lib:int-sub k 1)) (setf k2 k)
                (setf knext (f2cl-lib:int-sub k 2)))
               (t (setf k1 k) (setf k2 k)
                (setf knext (f2cl-lib:int-sub k 1))))))
        (cond
         ((and (= l1 l2) (= k1 k2))
          (setf suml
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4)
                      (ddot (f2cl-lib:int-sub m k1)
                       (f2cl-lib:array-slice a-%data% double-float
                                             (k1
                                              (min
                                               (the f2cl-lib:integer4
                                                    (f2cl-lib:int-add
                                                     k1 1))
                                               (the f2cl-lib:integer4
                                                    m)))
                                             ((1 lda) (1 *))
                                             a-%offset%)
                       lda
                       (f2cl-lib:array-slice c-%data% double-float
                                             ((min
                                               (f2cl-lib:int-add k1
                                                                 1)
                                               m)
                                              l1)
                                             ((1 ldc) (1 *))
                                             c-%offset%)
                       1)
                     (declare (ignore var-0 var-1 var-3 var-4))
                     (when var-2 (setf lda var-2))
                     ret-val))
          (setf sumr
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4)
                      (ddot (f2cl-lib:int-sub l1 1)
                       (f2cl-lib:array-slice c-%data% double-float
                                             (k1 1) ((1 ldc) (1 *))
                                             c-%offset%)
                       ldc
                       (f2cl-lib:array-slice b-%data% double-float
                                             (1 l1) ((1 ldb$) (1 *))
                                             b-%offset%)
                       1)
```

```
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
 (setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
 (setf scaloc one)
 (setf a11
        (+
         (f2cl-lib:fref a-%data% (k1 k1) ((1 lda) (1 *))
                        a-%offset%)
         (* sgn
            (f2cl-lib:fref b-%data% (l1 l1)
                           ((1 ldb$) (1 *)) b-%offset%))))
 (setf da11 (abs a11))
 (cond
  ((<= da11 smin) (setf a11 smin) (setf da11 smin)
   (setf info 1)))
 (setf db (abs (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))))
 (cond
  ((and (< da11 one) (> db one))
   (if (> db (* bignum da11))
       (setf scaloc (/ one db)))))
 (setf (f2cl-lib:fref x (1 1) ((1 2) (1 2)))
        (/
         (* (f2cl-lib:fref vec (1 1) ((1 2) (1 2))) scaloc)
         a11))
 (cond
  ((/= scaloc one)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2 var-3)
           (dscal m scaloc
            (f2cl-lib:array-slice c-%data% double-float
                                  (1 j) ((1 ldc) (1 *))
                                  c-%offset%)
            1)
         (declare (ignore var-2 var-3))
         (when var-0 (setf m var-0))
         (when var-1 (setf scaloc var-1)))
      label10))
   (setf scale (* scale scaloc))))
 (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                      c-%offset%)
        (f2cl-lib:fref x (1 1) ((1 2) (1 2)))))
((and (= l1 l2) (/= k1 k2))
 (setf suml
```

```
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub m k2)
             (f2cl-lib:array-slice a-%data% double-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           k2 1))
                                     (the f2cl-lib:integer4
                                          m)))
                                   ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice c-%data% double-float
                                   ((min
                                     (f2cl-lib:int-add k2
                                                       1)
                                     m)
                                    l1)
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf lda var-2))
          ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k1 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                   (1 l1) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub m k2)
```

```
                (f2cl-lib:array-slice a-%data% double-float
                                      (k2
                                       (min
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-add
                                              k2 1))
                                        (the f2cl-lib:integer4
                                             m)))
                                      ((1 lda) (1 *))
                                      a-%offset%)
                lda
                (f2cl-lib:array-slice c-%data% double-float
                                      ((min
                                        (f2cl-lib:int-add k2
                                                          1)
                                        m)
                                       l1)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                1)
            (declare (ignore var-0 var-1 var-3 var-4))
            (when var-2 (setf lda var-2))
            ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k2 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                   (1 l1) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
            (declare (ignore var-0 var-1 var-3 var-4))
            (when var-2 (setf ldc var-2))
            ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
    (dlaln2 f2cl-lib:%false% 2 1 smin one
     (f2cl-lib:array-slice a-%data% double-float (k1 k1)
                           ((1 lda) (1 *)) a-%offset%)
```

```
      lda one one vec 2
      (* (- sgn)
         (f2cl-lib:fref b-%data% (l1 l1) ((1 ldb$) (1 *))
                        b-%offset%))
      zero x 2 scaloc xnorm ierr)
   (declare
    (ignore var-0 var-1 var-2 var-5 var-9 var-10 var-11
     var-13 var-14))
   (when var-3 (setf smin var-3))
   (when var-4 (setf one var-4))
   (when var-6 (setf lda var-6))
   (when var-7 (setf one var-7))
   (when var-8 (setf one var-8))
   (when var-12 (setf zero var-12))
   (when var-15 (setf scaloc var-15))
   (when var-16 (setf xnorm var-16))
   (when var-17 (setf ierr var-17)))
 (if (/= ierr 0)
     (setf info 1))
 (cond
  ((/= scaloc one)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2 var-3)
           (dscal m scaloc
            (f2cl-lib:array-slice c-%data% double-float
                                  (1 j) ((1 ldc) (1 *))
                                  c-%offset%)
            1)
         (declare (ignore var-2 var-3))
         (when var-0 (setf m var-0))
         (when var-1 (setf scaloc var-1)))
      label20))
   (setf scale (* scale scaloc))))
 (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                      c-%offset%)
         (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
 (setf (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                      c-%offset%)
         (f2cl-lib:fref x (2 1) ((1 2) (1 2)))))
((and (/= l1 l2) (= k1 k2))
 (setf suml
         (multiple-value-bind
             (ret-val var-0 var-1 var-2 var-3 var-4)
             (ddot (f2cl-lib:int-sub m k1)
              (f2cl-lib:array-slice a-%data% double-float
                                    (k1
                                     (min
                                      (the f2cl-lib:integer4
```

```
                                                (f2cl-lib:int-add
                                                 k1 1))
                                          (the f2cl-lib:integer4
                                               m)))
                                       ((1 lda) (1 *))
                                       a-%offset%)
                     lda
                     (f2cl-lib:array-slice c-%data% double-float
                                       ((min
                                         (f2cl-lib:int-add k1
                                                           1)
                                         m)
                                        l1)
                                       ((1 ldc) (1 *))
                                       c-%offset%)
                     1)
                  (declare (ignore var-0 var-1 var-3 var-4))
                  (when var-2 (setf lda var-2))
                  ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% double-float
                                       (k1 1) ((1 ldc) (1 *))
                                       c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                       (1 l1) ((1 ldb$) (1 *))
                                       b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (* sgn
           (-
            (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                           c-%offset%)
            (+ suml (* sgn sumr)))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub m k1)
             (f2cl-lib:array-slice a-%data% double-float
                                       (k1
                                        (min
                                         (the f2cl-lib:integer4
                                              (f2cl-lib:int-add
                                               k1 1))
```

```
                                        (the f2cl-lib:integer4
                                             m)))
                                      ((1 lda) (1 *))
                                      a-%offset%)
                 lda
                 (f2cl-lib:array-slice c-%data% double-float
                                      ((min
                                        (f2cl-lib:int-add k1
                                                          1)
                                        m)
                                       l2)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                 1)
              (declare (ignore var-0 var-1 var-3 var-4))
              (when var-2 (setf lda var-2))
              ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k1 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                   (1 l2) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (* sgn
           (-
            (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                           c-%offset%)
            (+ suml (* sgn sumr)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
    (dlaln2 f2cl-lib:%true% 2 1 smin one
     (f2cl-lib:array-slice b-%data% double-float (l1 l1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ one one vec 2
     (* (- sgn)
        (f2cl-lib:fref a-%data% (k1 k1) ((1 lda) (1 *))
                       a-%offset%))
     zero x 2 scaloc xnorm ierr)
```

```
  (declare
   (ignore var-0 var-1 var-2 var-5 var-9 var-10 var-11
    var-13 var-14))
  (when var-3 (setf smin var-3))
  (when var-4 (setf one var-4))
  (when var-6 (setf ldb$ var-6))
  (when var-7 (setf one var-7))
  (when var-8 (setf one var-8))
  (when var-12 (setf zero var-12))
  (when var-15 (setf scaloc var-15))
  (when var-16 (setf xnorm var-16))
  (when var-17 (setf ierr var-17)))
(if (/= ierr 0)
    (setf info 1))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal m scaloc
           (f2cl-lib:array-slice c-%data% double-float
                                 (1 j) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
     label30))
  (setf scale (* scale scaloc))))
(setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                      c-%offset%)
        (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
(setf (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                      c-%offset%)
        (f2cl-lib:fref x (2 1) ((1 2) (1 2)))))
((and (/= l1 l2) (/= k1 k2))
 (setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub m k2)
             (f2cl-lib:array-slice a-%data% double-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           k2 1))
                                     (the f2cl-lib:integer4
                                          m)))
                                   ((1 lda) (1 *))
```

```
                                        a-%offset%)
                 lda
                 (f2cl-lib:array-slice c-%data% double-float
                                        ((min
                                          (f2cl-lib:int-add k2
                                                            1)
                                          m)
                                         l1)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                 1)
              (declare (ignore var-0 var-1 var-3 var-4))
              (when var-2 (setf lda var-2))
              ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% double-float
                                        (k1 1) ((1 ldc) (1 *))
                                        c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                        (1 l1) ((1 ldb$) (1 *))
                                        b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub m k2)
             (f2cl-lib:array-slice a-%data% double-float
                                        (k1
                                         (min
                                          (the f2cl-lib:integer4
                                               (f2cl-lib:int-add
                                                k2 1))
                                          (the f2cl-lib:integer4
                                               m)))
                                        ((1 lda) (1 *))
                                        a-%offset%)
             lda
             (f2cl-lib:array-slice c-%data% double-float
```

```
                                        ((min
                                          (f2cl-lib:int-add k2
                                                            1)
                                          m)
                                         l2)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                     1)
                  (declare (ignore var-0 var-1 var-3 var-4))
                  (when var-2 (setf lda var-2))
                  ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k1 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                   (1 l2) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (1 2) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub m k2)
             (f2cl-lib:array-slice a-%data% double-float
                                   (k2
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           k2 1))
                                     (the f2cl-lib:integer4
                                          m)))
                                   ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice c-%data% double-float
                                   ((min
                                     (f2cl-lib:int-add k2
                                                       1)
```

```
                                        m)
                                       l1)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                 1)
              (declare (ignore var-0 var-1 var-3 var-4))
              (when var-2 (setf lda var-2))
              ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k2 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                   (1 l1) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub m k2)
             (f2cl-lib:array-slice a-%data% double-float
                                   (k2
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           k2 1))
                                     (the f2cl-lib:integer4
                                          m)))
                                   ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice c-%data% double-float
                                   ((min
                                     (f2cl-lib:int-add k2
                                                       1)
                                     m)
                                    l2)
                                   ((1 ldc) (1 *))
```

```
                                    c-%offset%)
                 1)
              (declare (ignore var-0 var-1 var-3 var-4))
              (when var-2 (setf lda var-2))
              ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k2 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                   (1 l2) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (2 2) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l2) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15)
    (dlasy2 f2cl-lib:%false% f2cl-lib:%false% isgn 2 2
     (f2cl-lib:array-slice a-%data% double-float (k1 k1)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% double-float (l1 l1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ vec 2 scaloc x 2 xnorm ierr)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-12 var-13))
  (when var-2 (setf isgn var-2))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-11 (setf scaloc var-11))
  (when var-14 (setf xnorm var-14))
  (when var-15 (setf ierr var-15)))
(if (/= ierr 0)
    (setf info 1))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
```

```
                    (tagbody
                      (multiple-value-bind (var-0 var-1 var-2 var-3)
                          (dscal m scaloc
                           (f2cl-lib:array-slice c-%data% double-float
                                                 (1 j) ((1 ldc) (1 *))
                                                 c-%offset%)
                           1)
                        (declare (ignore var-2 var-3))
                        (when var-0 (setf m var-0))
                        (when var-1 (setf scaloc var-1)))
                     label40))
                  (setf scale (* scale scaloc))))
                (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                                     c-%offset%)
                        (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
                (setf (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                                     c-%offset%)
                        (f2cl-lib:fref x (1 2) ((1 2) (1 2))))
                (setf (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                                     c-%offset%)
                        (f2cl-lib:fref x (2 1) ((1 2) (1 2))))
                (setf (f2cl-lib:fref c-%data% (k2 l2) ((1 ldc) (1 *))
                                     c-%offset%)
                        (f2cl-lib:fref x (2 2) ((1 2) (1 2))))))
            label50))
        label60)))
    ((and (not notrna) notrnb) (setf lnext 1)
     (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                   ((> l n) nil)
       (tagbody
         (if (< l lnext)
             (go label120))
         (cond ((= l n) (setf l1 l) (setf l2 l))
               (t
                (cond
                 ((/=
                   (f2cl-lib:fref b ((f2cl-lib:int-add l 1) l)
                                  ((1 ldb$) (1 *)))
                   zero)
                  (setf l1 l) (setf l2 (f2cl-lib:int-add l 1))
                  (setf lnext (f2cl-lib:int-add l 2)))
                 (t (setf l1 l) (setf l2 l)
                  (setf lnext (f2cl-lib:int-add l 1))))))
         (setf knext 1)
         (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                       ((> k m) nil)
           (tagbody
             (if (< k knext)
                 (go label110))
             (cond ((= k m) (setf k1 k) (setf k2 k))
```

```
        (t
         (cond
          ((/=
            (f2cl-lib:fref a ((f2cl-lib:int-add k 1) k)
                           ((1 lda) (1 *)))
            zero)
           (setf k1 k) (setf k2 (f2cl-lib:int-add k 1))
           (setf knext (f2cl-lib:int-add k 2)))
          (t (setf k1 k) (setf k2 k)
           (setf knext (f2cl-lib:int-add k 1))))))
(cond
 ((and (= l1 l2) (= k1 k2))
  (setf suml
          (ddot (f2cl-lib:int-sub k1 1)
           (f2cl-lib:array-slice a-%data% double-float (1 k1)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice c-%data% double-float (1 l1)
                                 ((1 ldc) (1 *)) c-%offset%)
           1))
  (setf sumr
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4)
              (ddot (f2cl-lib:int-sub l1 1)
               (f2cl-lib:array-slice c-%data% double-float
                                     (k1 1) ((1 ldc) (1 *))
                                     c-%offset%)
               ldc
               (f2cl-lib:array-slice b-%data% double-float
                                     (1 l1) ((1 ldb$) (1 *))
                                     b-%offset%)
               1)
            (declare (ignore var-0 var-1 var-3 var-4))
            (when var-2 (setf ldc var-2))
            ret-val))
  (setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
          (-
           (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                          c-%offset%)
           (+ suml (* sgn sumr))))
  (setf scaloc one)
  (setf a11
          (+
           (f2cl-lib:fref a-%data% (k1 k1) ((1 lda) (1 *))
                          a-%offset%)
           (* sgn
              (f2cl-lib:fref b-%data% (l1 l1)
                             ((1 ldb$) (1 *)) b-%offset%))))
  (setf da11 (abs a11))
  (cond
```

```
 ((<= da11 smin) (setf a11 smin) (setf da11 smin)
  (setf info 1)))
(setf db (abs (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))))
(cond
 ((and (< da11 one) (> db one))
  (if (> db (* bignum da11))
      (setf scaloc (/ one db)))))
(setf (f2cl-lib:fref x (1 1) ((1 2) (1 2)))
        (/
         (* (f2cl-lib:fref vec (1 1) ((1 2) (1 2))) scaloc)
         a11))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal m scaloc
           (f2cl-lib:array-slice c-%data% double-float
                                 (1 j) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
     label70))
  (setf scale (* scale scaloc))))
(setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref x (1 1) ((1 2) (1 2)))))
((and (= l1 l2) (/= k1 k2))
 (setf suml
        (ddot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% double-float (1 k1)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% double-float (1 l1)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
 (setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k1 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                   (1 l1) ((1 ldb$) (1 *))
                                   b-%offset%)
```

```
            1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (ddot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% double-float (1 k2)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% double-float (1 l1)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k2 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                   (1 l1) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
    (dlaln2 f2cl-lib:%true% 2 1 smin one
     (f2cl-lib:array-slice a-%data% double-float (k1 k1)
                           ((1 lda) (1 *)) a-%offset%)
     lda one one vec 2
     (* (- sgn)
        (f2cl-lib:fref b-%data% (l1 l1) ((1 ldb$) (1 *))
                       b-%offset%))
     zero x 2 scaloc xnorm ierr)
  (declare
```

```
    (ignore var-0 var-1 var-2 var-5 var-9 var-10 var-11
     var-13 var-14))
   (when var-3 (setf smin var-3))
   (when var-4 (setf one var-4))
   (when var-6 (setf lda var-6))
   (when var-7 (setf one var-7))
   (when var-8 (setf one var-8))
   (when var-12 (setf zero var-12))
   (when var-15 (setf scaloc var-15))
   (when var-16 (setf xnorm var-16))
   (when var-17 (setf ierr var-17)))
  (if (/= ierr 0)
      (setf info 1))
  (cond
   ((/= scaloc one)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal m scaloc
             (f2cl-lib:array-slice c-%data% double-float
                                   (1 j) ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
      label80))
    (setf scale (* scale scaloc))))
  (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                       c-%offset%)
          (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
  (setf (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                       c-%offset%)
          (f2cl-lib:fref x (2 1) ((1 2) (1 2)))))
 ((and (/= l1 l2) (= k1 k2))
  (setf suml
          (ddot (f2cl-lib:int-sub k1 1)
           (f2cl-lib:array-slice a-%data% double-float (1 k1)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice c-%data% double-float (1 l1)
                                 ((1 ldc) (1 *)) c-%offset%)
           1))
  (setf sumr
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4)
              (ddot (f2cl-lib:int-sub l1 1)
               (f2cl-lib:array-slice c-%data% double-float
                                     (k1 1) ((1 ldc) (1 *))
```

```
                                    c-%offset%)
                ldc
                (f2cl-lib:array-slice b-%data% double-float
                                    (1 l1) ((1 ldb$) (1 *))
                                    b-%offset%)
                1)
            (declare (ignore var-0 var-1 var-3 var-4))
            (when var-2 (setf ldc var-2))
            ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (* sgn
           (-
            (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                           c-%offset%)
            (+ suml (* sgn sumr)))))
(setf suml
        (ddot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% double-float (1 k1)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% double-float (1 l2)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% double-float
                                    (k1 1) ((1 ldc) (1 *))
                                    c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                    (1 l2) ((1 ldb$) (1 *))
                                    b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (* sgn
           (-
            (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                           c-%offset%)
            (+ suml (* sgn sumr)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
    (dlaln2 f2cl-lib:%true% 2 1 smin one
     (f2cl-lib:array-slice b-%data% double-float (l1 l1)
```

```
                                ((1 ldb$) (1 *)) b-%offset%)
        ldb$ one one vec 2
        (* (- sgn)
           (f2cl-lib:fref a-%data% (k1 k1) ((1 lda) (1 *))
                          a-%offset%))
        zero x 2 scaloc xnorm ierr)
      (declare
       (ignore var-0 var-1 var-2 var-5 var-9 var-10 var-11
        var-13 var-14))
      (when var-3 (setf smin var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldb$ var-6))
      (when var-7 (setf one var-7))
      (when var-8 (setf one var-8))
      (when var-12 (setf zero var-12))
      (when var-15 (setf scaloc var-15))
      (when var-16 (setf xnorm var-16))
      (when var-17 (setf ierr var-17)))
    (if (/= ierr 0)
        (setf info 1))
    (cond
     ((/= scaloc one)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (dscal m scaloc
               (f2cl-lib:array-slice c-%data% double-float
                                     (1 j) ((1 ldc) (1 *))
                                     c-%offset%)
               1)
            (declare (ignore var-2 var-3))
            (when var-0 (setf m var-0))
            (when var-1 (setf scaloc var-1)))
        label90))
      (setf scale (* scale scaloc))))
    (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                         c-%offset%)
          (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
    (setf (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                         c-%offset%)
          (f2cl-lib:fref x (2 1) ((1 2) (1 2)))))
  ((and (/= l1 l2) (/= k1 k2))
    (setf suml
          (ddot (f2cl-lib:int-sub k1 1)
           (f2cl-lib:array-slice a-%data% double-float (1 k1)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice c-%data% double-float (1 l1)
                                 ((1 ldc) (1 *)) c-%offset%)
```

```
         1))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k1 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                   (1 l1) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (ddot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% double-float (1 k1)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% double-float (1 l2)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k1 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                   (1 l2) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (1 2) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
```

```
        (ddot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% double-float (1 k2)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% double-float (1 l1)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k2 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                   (1 l1) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (ddot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% double-float (1 k2)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% double-float (1 l2)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k2 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                   (1 l2) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
```

```
(setf (f2cl-lib:fref vec (2 2) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l2) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15)
    (dlasy2 f2cl-lib:%true% f2cl-lib:%false% isgn 2 2
     (f2cl-lib:array-slice a-%data% double-float (k1 k1)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% double-float (l1 l1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ vec 2 scaloc x 2 xnorm ierr)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-12 var-13))
  (when var-2 (setf isgn var-2))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-11 (setf scaloc var-11))
  (when var-14 (setf xnorm var-14))
  (when var-15 (setf ierr var-15)))
(if (/= ierr 0)
    (setf info 1))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal m scaloc
           (f2cl-lib:array-slice c-%data% double-float
                                 (1 j) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
     label100))
  (setf scale (* scale scaloc))))
(setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
(setf (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref x (1 2) ((1 2) (1 2))))
(setf (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                     c-%offset%)
```

```
                        (f2cl-lib:fref x (2 1) ((1 2) (1 2))))
                  (setf (f2cl-lib:fref c-%data% (k2 l2) ((1 ldc) (1 *))
                                       c-%offset%)
                        (f2cl-lib:fref x (2 2) ((1 2) (1 2))))))
            label110))
        label120)))
    ((and (not notrna) (not notrnb)) (setf lnext n)
     (f2cl-lib:fdo (l n (f2cl-lib:int-add l (f2cl-lib:int-sub 1)))
                   ((> l 1) nil)
       (tagbody
         (if (> l lnext)
             (go label180))
         (cond ((= l 1) (setf l1 l) (setf l2 l))
               (t
                (cond
                 ((/=
                   (f2cl-lib:fref b
                                  (l
                                   (f2cl-lib:int-add l
                                                     (f2cl-lib:int-sub
                                                      1)))
                                  ((1 ldb$) (1 *)))
                   zero)
                  (setf l1 (f2cl-lib:int-sub l 1)) (setf l2 l)
                  (setf lnext (f2cl-lib:int-sub l 2)))
                 (t (setf l1 l) (setf l2 l)
                  (setf lnext (f2cl-lib:int-sub l 1))))))
         (setf knext 1)
         (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                       ((> k m) nil)
           (tagbody
             (if (< k knext)
                 (go label170))
             (cond ((= k m) (setf k1 k) (setf k2 k))
                   (t
                    (cond
                     ((/=
                       (f2cl-lib:fref a ((f2cl-lib:int-add k 1) k)
                                      ((1 lda) (1 *)))
                       zero)
                      (setf k1 k) (setf k2 (f2cl-lib:int-add k 1))
                      (setf knext (f2cl-lib:int-add k 2)))
                     (t (setf k1 k) (setf k2 k)
                      (setf knext (f2cl-lib:int-add k 1))))))
             (cond
              ((and (= l1 l2) (= k1 k2))
               (setf suml
                       (ddot (f2cl-lib:int-sub k1 1)
                        (f2cl-lib:array-slice a-%data% double-float (1 k1)
                                              ((1 lda) (1 *)) a-%offset%)
```

```
         1
         (f2cl-lib:array-slice c-%data% double-float (1 l1)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub n l1)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l1 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                   (l1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l1 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldb$) (1 *))
                                   b-%offset%)
             ldb$)
          (declare (ignore var-0 var-1 var-3))
          (when var-2 (setf ldc var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf scaloc one)
(setf a11
        (+
         (f2cl-lib:fref a-%data% (k1 k1) ((1 lda) (1 *))
                        a-%offset%)
         (* sgn
            (f2cl-lib:fref b-%data% (l1 l1)
                           ((1 ldb$) (1 *)) b-%offset%))))
(setf da11 (abs a11))
(cond
 ((<= da11 smin) (setf a11 smin) (setf da11 smin)
```

```
  (setf info 1)))
 (setf db (abs (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))))
 (cond
  ((and (< da11 one) (> db one))
   (if (> db (* bignum da11))
       (setf scaloc (/ one db)))))
 (setf (f2cl-lib:fref x (1 1) ((1 2) (1 2)))
         (/
          (* (f2cl-lib:fref vec (1 1) ((1 2) (1 2))) scaloc)
          a11))
 (cond
  ((/= scaloc one)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2 var-3)
           (dscal m scaloc
            (f2cl-lib:array-slice c-%data% double-float
                                  (1 j) ((1 ldc) (1 *))
                                  c-%offset%)
            1)
         (declare (ignore var-2 var-3))
         (when var-0 (setf m var-0))
         (when var-1 (setf scaloc var-1)))
      label130))
   (setf scale (* scale scaloc))))
 (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                      c-%offset%)
         (f2cl-lib:fref x (1 1) ((1 2) (1 2)))))
((and (= l1 l2) (/= k1 k2))
 (setf suml
         (ddot (f2cl-lib:int-sub k1 1)
          (f2cl-lib:array-slice a-%data% double-float (1 k1)
                                ((1 lda) (1 *)) a-%offset%)
          1
          (f2cl-lib:array-slice c-%data% double-float (1 l1)
                                ((1 ldc) (1 *)) c-%offset%)
          1))
 (setf sumr
         (multiple-value-bind
             (ret-val var-0 var-1 var-2 var-3 var-4)
             (ddot (f2cl-lib:int-sub n l2)
              (f2cl-lib:array-slice c-%data% double-float
                                    (k1
                                     (min
                                      (the f2cl-lib:integer4
                                           (f2cl-lib:int-add
                                            l2 1))
                                      (the f2cl-lib:integer4
                                           n)))
```

```
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                        ldc
                        (f2cl-lib:array-slice b-%data% double-float
                                        (l1
                                         (min
                                          (the f2cl-lib:integer4
                                               (f2cl-lib:int-add
                                                l2 1))
                                          (the f2cl-lib:integer4
                                               n)))
                                        ((1 ldb$) (1 *))
                                        b-%offset%)
                        ldb$)
                     (declare (ignore var-0 var-1 var-3))
                     (when var-2 (setf ldc var-2))
                     (when var-4 (setf ldb$ var-4))
                     ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (ddot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% double-float (1 k2)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% double-float (1 l1)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% double-float
                                        (k2
                                         (min
                                          (the f2cl-lib:integer4
                                               (f2cl-lib:int-add
                                                l2 1))
                                          (the f2cl-lib:integer4
                                               n)))
                                        ((1 ldc) (1 *))
                                        c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                        (l1
                                         (min
                                          (the f2cl-lib:integer4
```

```
                                                (f2cl-lib:int-add
                                                 l2 1))
                                          (the f2cl-lib:integer4
                                                n)))
                                       ((1 ldb$) (1 *))
                                       b-%offset%)
                 ldb$)
              (declare (ignore var-0 var-1 var-3))
              (when var-2 (setf ldc var-2))
              (when var-4 (setf ldb$ var-4))
              ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
    (dlaln2 f2cl-lib:%true% 2 1 smin one
     (f2cl-lib:array-slice a-%data% double-float (k1 k1)
                           ((1 lda) (1 *)) a-%offset%)
     lda one one vec 2
     (* (- sgn)
        (f2cl-lib:fref b-%data% (l1 l1) ((1 ldb$) (1 *))
                       b-%offset%))
     zero x 2 scaloc xnorm ierr)
  (declare
   (ignore var-0 var-1 var-2 var-5 var-9 var-10 var-11
    var-13 var-14))
  (when var-3 (setf smin var-3))
  (when var-4 (setf one var-4))
  (when var-6 (setf lda var-6))
  (when var-7 (setf one var-7))
  (when var-8 (setf one var-8))
  (when var-12 (setf zero var-12))
  (when var-15 (setf scaloc var-15))
  (when var-16 (setf xnorm var-16))
  (when var-17 (setf ierr var-17)))
(if (/= ierr 0)
    (setf info 1))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal m scaloc
           (f2cl-lib:array-slice c-%data% double-float
```

```
                                    (1 j) ((1 ldc) (1 *))
                                    c-%offset%)
                  1)
            (declare (ignore var-2 var-3))
            (when var-0 (setf m var-0))
            (when var-1 (setf scaloc var-1)))
        label140))
      (setf scale (* scale scaloc))))
    (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                         c-%offset%)
            (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
    (setf (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                         c-%offset%)
            (f2cl-lib:fref x (2 1) ((1 2) (1 2)))))
   ((and (/= l1 l2) (= k1 k2))
    (setf suml
            (ddot (f2cl-lib:int-sub k1 1)
             (f2cl-lib:array-slice a-%data% double-float (1 k1)
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice c-%data% double-float (1 l1)
                                   ((1 ldc) (1 *)) c-%offset%)
             1))
    (setf sumr
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4)
                (ddot (f2cl-lib:int-sub n l2)
                 (f2cl-lib:array-slice c-%data% double-float
                                       (k1
                                        (min
                                         (the f2cl-lib:integer4
                                              (f2cl-lib:int-add
                                               l2 1))
                                         (the f2cl-lib:integer4
                                              n)))
                                       ((1 ldc) (1 *))
                                       c-%offset%)
                 ldc
                 (f2cl-lib:array-slice b-%data% double-float
                                       (l1
                                        (min
                                         (the f2cl-lib:integer4
                                              (f2cl-lib:int-add
                                               l2 1))
                                         (the f2cl-lib:integer4
                                              n)))
                                       ((1 ldb$) (1 *))
                                       b-%offset%)
                 ldb$)
              (declare (ignore var-0 var-1 var-3))
```

```
          (when var-2 (setf ldc var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (* sgn
           (-
            (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                           c-%offset%)
            (+ suml (* sgn sumr)))))
(setf suml
        (ddot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% double-float (1 k1)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% double-float (1 l2)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                   (l2
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldb$) (1 *))
                                   b-%offset%)
             ldb$)
          (declare (ignore var-0 var-1 var-3))
          (when var-2 (setf ldc var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (* sgn
           (-
```

```
            (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                           c-%offset%)
            (+ suml (* sgn sumr)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
    (dlaln2 f2cl-lib:%false% 2 1 smin one
     (f2cl-lib:array-slice b-%data% double-float (l1 l1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ one one vec 2
     (* (- sgn)
        (f2cl-lib:fref a-%data% (k1 k1) ((1 lda) (1 *))
                       a-%offset%))
     zero x 2 scaloc xnorm ierr)
  (declare
   (ignore var-0 var-1 var-2 var-5 var-9 var-10 var-11
    var-13 var-14))
  (when var-3 (setf smin var-3))
  (when var-4 (setf one var-4))
  (when var-6 (setf ldb$ var-6))
  (when var-7 (setf one var-7))
  (when var-8 (setf one var-8))
  (when var-12 (setf zero var-12))
  (when var-15 (setf scaloc var-15))
  (when var-16 (setf xnorm var-16))
  (when var-17 (setf ierr var-17)))
(if (/= ierr 0)
    (setf info 1))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal m scaloc
           (f2cl-lib:array-slice c-%data% double-float
                                 (1 j) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
     label150))
  (setf scale (* scale scaloc))))
(setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
(setf (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                     c-%offset%)
```

```
        (f2cl-lib:fref x (2 1) ((1 2) (1 2)))))
((and (/= l1 l2) (/= k1 k2))
 (setf suml
        (ddot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% double-float (1 k1)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% double-float (1 l1)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
 (setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                   (l1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldb$) (1 *))
                                   b-%offset%)
             ldb$)
           (declare (ignore var-0 var-1 var-3))
           (when var-2 (setf ldc var-2))
           (when var-4 (setf ldb$ var-4))
           ret-val))
 (setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
 (setf suml
        (ddot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% double-float (1 k1)
                               ((1 lda) (1 *)) a-%offset%)
         1
```

```
         (f2cl-lib:array-slice c-%data% double-float (1 l2)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                   (l2
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldb$) (1 *))
                                   b-%offset%)
             ldb$)
          (declare (ignore var-0 var-1 var-3))
          (when var-2 (setf ldc var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf (f2cl-lib:fref vec (1 2) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (ddot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% double-float (1 k2)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% double-float (1 l1)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub n l2)
```

```
                (f2cl-lib:array-slice c-%data% double-float
                                      (k2
                                       (min
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-add
                                              l2 1))
                                        (the f2cl-lib:integer4
                                             n)))
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                ldc
                (f2cl-lib:array-slice b-%data% double-float
                                      (l1
                                       (min
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-add
                                              l2 1))
                                        (the f2cl-lib:integer4
                                             n)))
                                      ((1 ldb$) (1 *))
                                      b-%offset%)
                ldb$)
            (declare (ignore var-0 var-1 var-3))
            (when var-2 (setf ldc var-2))
            (when var-4 (setf ldb$ var-4))
            ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (ddot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% double-float (1 k2)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% double-float (1 l2)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% double-float
                                      (k2
                                       (min
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-add
                                              l2 1))
                                        (the f2cl-lib:integer4
```

```
                                        n)))
                                  ((1 ldc) (1 *))
                                  c-%offset%)
                ldc
                (f2cl-lib:array-slice b-%data% double-float
                                  (l2
                                   (min
                                    (the f2cl-lib:integer4
                                         (f2cl-lib:int-add
                                          l2 1))
                                    (the f2cl-lib:integer4
                                         n)))
                                  ((1 ldb$) (1 *))
                                  b-%offset%)
                ldb$)
            (declare (ignore var-0 var-1 var-3))
            (when var-2 (setf ldc var-2))
            (when var-4 (setf ldb$ var-4))
            ret-val))
(setf (f2cl-lib:fref vec (2 2) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l2) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15)
    (dlasy2 f2cl-lib:%true% f2cl-lib:%true% isgn 2 2
     (f2cl-lib:array-slice a-%data% double-float (k1 k1)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% double-float (l1 l1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ vec 2 scaloc x 2 xnorm ierr)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-12 var-13))
  (when var-2 (setf isgn var-2))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-11 (setf scaloc var-11))
  (when var-14 (setf xnorm var-14))
  (when var-15 (setf ierr var-15)))
(if (/= ierr 0)
    (setf info 1))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
```

```
                    (multiple-value-bind (var-0 var-1 var-2 var-3)
                        (dscal m scaloc
                         (f2cl-lib:array-slice c-%data% double-float
                                               (1 j) ((1 ldc) (1 *))
                                               c-%offset%)
                         1)
                      (declare (ignore var-2 var-3))
                      (when var-0 (setf m var-0))
                      (when var-1 (setf scaloc var-1)))
                  label160))
                (setf scale (* scale scaloc))))
              (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                                    c-%offset%)
                      (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
              (setf (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                                    c-%offset%)
                      (f2cl-lib:fref x (1 2) ((1 2) (1 2))))
              (setf (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                                    c-%offset%)
                      (f2cl-lib:fref x (2 1) ((1 2) (1 2))))
              (setf (f2cl-lib:fref c-%data% (k2 l2) ((1 ldc) (1 *))
                                    c-%offset%)
                      (f2cl-lib:fref x (2 2) ((1 2) (1 2))))))
          label170))
      label180)))
  ((and notrna (not notrnb)) (setf lnext n)
   (f2cl-lib:fdo (l n (f2cl-lib:int-add l (f2cl-lib:int-sub 1)))
                 ((> l 1) nil)
     (tagbody
       (if (> l lnext)
           (go label240))
       (cond ((= l 1) (setf l1 l) (setf l2 l))
             (t
              (cond
               ((/=
                 (f2cl-lib:fref b
                                (l
                                 (f2cl-lib:int-add l
                                                   (f2cl-lib:int-sub
                                                    1)))
                                ((1 ldb$) (1 *)))
                 zero)
                (setf l1 (f2cl-lib:int-sub l 1)) (setf l2 l)
                (setf lnext (f2cl-lib:int-sub l 2)))
               (t (setf l1 l) (setf l2 l)
                (setf lnext (f2cl-lib:int-sub l 1))))))
       (setf knext m)
       (f2cl-lib:fdo (k m (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                     ((> k 1) nil)
         (tagbody
```

```
(if (> k knext)
    (go label230))
(cond ((= k 1) (setf k1 k) (setf k2 k))
      (t
       (cond
        ((/=
          (f2cl-lib:fref a
                         (k
                          (f2cl-lib:int-add k
                                            (f2cl-lib:int-sub
                                             1)))
                         ((1 lda) (1 *)))
          zero)
         (setf k1 (f2cl-lib:int-sub k 1)) (setf k2 k)
         (setf knext (f2cl-lib:int-sub k 2)))
        (t (setf k1 k) (setf k2 k)
         (setf knext (f2cl-lib:int-sub k 1))))))
(cond
 ((and (= l1 l2) (= k1 k2))
  (setf suml
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4)
              (ddot (f2cl-lib:int-sub m k1)
               (f2cl-lib:array-slice a-%data% double-float
                                     (k1
                                      (min
                                       (the f2cl-lib:integer4
                                            (f2cl-lib:int-add
                                             k1 1))
                                       (the f2cl-lib:integer4
                                            m)))
                                     ((1 lda) (1 *))
                                     a-%offset%)
               lda
               (f2cl-lib:array-slice c-%data% double-float
                                     ((min
                                       (f2cl-lib:int-add k1
                                                         1)
                                       m)
                                      l1)
                                     ((1 ldc) (1 *))
                                     c-%offset%)
               1)
            (declare (ignore var-0 var-1 var-3 var-4))
            (when var-2 (setf lda var-2))
            ret-val))
  (setf sumr
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4)
              (ddot (f2cl-lib:int-sub n l1)
```

```
                (f2cl-lib:array-slice c-%data% double-float
                                      (k1
                                       (min
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-add
                                              l1 1))
                                        (the f2cl-lib:integer4
                                             n)))
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                ldc
                (f2cl-lib:array-slice b-%data% double-float
                                      (l1
                                       (min
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-add
                                              l1 1))
                                        (the f2cl-lib:integer4
                                             n)))
                                      ((1 ldb$) (1 *))
                                      b-%offset%)
                ldb$)
             (declare (ignore var-0 var-1 var-3))
             (when var-2 (setf ldc var-2))
             (when var-4 (setf ldb$ var-4))
             ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf scaloc one)
(setf a11
        (+
         (f2cl-lib:fref a-%data% (k1 k1) ((1 lda) (1 *))
                        a-%offset%)
         (* sgn
            (f2cl-lib:fref b-%data% (l1 l1)
                           ((1 ldb$) (1 *)) b-%offset%))))
(setf da11 (abs a11))
(cond
 ((<= da11 smin) (setf a11 smin) (setf da11 smin)
  (setf info 1)))
(setf db (abs (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))))
(cond
 ((and (< da11 one) (> db one))
  (if (> db (* bignum da11))
      (setf scaloc (/ one db)))))
(setf (f2cl-lib:fref x (1 1) ((1 2) (1 2)))
        (/
```

```
          (* (f2cl-lib:fref vec (1 1) ((1 2) (1 2))) scaloc)
          a11))
 (cond
  ((/= scaloc one)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2 var-3)
           (dscal m scaloc
            (f2cl-lib:array-slice c-%data% double-float
                                  (1 j) ((1 ldc) (1 *))
                                  c-%offset%)
            1)
         (declare (ignore var-2 var-3))
         (when var-0 (setf m var-0))
         (when var-1 (setf scaloc var-1)))
      label190))
   (setf scale (* scale scaloc))))
 (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                      c-%offset%)
         (f2cl-lib:fref x (1 1) ((1 2) (1 2)))))
((and (= l1 l2) (/= k1 k2))
 (setf suml
         (multiple-value-bind
             (ret-val var-0 var-1 var-2 var-3 var-4)
             (ddot (f2cl-lib:int-sub m k2)
              (f2cl-lib:array-slice a-%data% double-float
                                    (k1
                                     (min
                                      (the f2cl-lib:integer4
                                           (f2cl-lib:int-add
                                            k2 1))
                                      (the f2cl-lib:integer4
                                           m)))
                                    ((1 lda) (1 *))
                                    a-%offset%)
              lda
              (f2cl-lib:array-slice c-%data% double-float
                                    ((min
                                      (f2cl-lib:int-add k2
                                                        1)
                                      m)
                                     l1)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
              1)
           (declare (ignore var-0 var-1 var-3 var-4))
           (when var-2 (setf lda var-2))
           ret-val))
 (setf sumr
```

```
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                   (l1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldb$) (1 *))
                                   b-%offset%)
             ldb$)
          (declare (ignore var-0 var-1 var-3))
          (when var-2 (setf ldc var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub m k2)
             (f2cl-lib:array-slice a-%data% double-float
                                   (k2
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           k2 1))
                                     (the f2cl-lib:integer4
                                          m)))
                                   ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice c-%data% double-float
```

```
                                        ((min
                                          (f2cl-lib:int-add k2
                                                            1)
                                          m)
                                         l1)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                     1)
                  (declare (ignore var-0 var-1 var-3 var-4))
                  (when var-2 (setf lda var-2))
                  ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% double-float
                                        (k2
                                         (min
                                          (the f2cl-lib:integer4
                                               (f2cl-lib:int-add
                                                l2 1))
                                          (the f2cl-lib:integer4
                                               n)))
                                        ((1 ldc) (1 *))
                                        c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                        (l1
                                         (min
                                          (the f2cl-lib:integer4
                                               (f2cl-lib:int-add
                                                l2 1))
                                          (the f2cl-lib:integer4
                                               n)))
                                        ((1 ldb$) (1 *))
                                        b-%offset%)
             ldb$)
          (declare (ignore var-0 var-1 var-3))
          (when var-2 (setf ldc var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
```

```
      (dlaln2 f2cl-lib:%false% 2 1 smin one
       (f2cl-lib:array-slice a-%data% double-float (k1 k1)
                             ((1 lda) (1 *)) a-%offset%)
       lda one one vec 2
       (* (- sgn)
          (f2cl-lib:fref b-%data% (l1 l1) ((1 ldb$) (1 *))
                         b-%offset%))
       zero x 2 scaloc xnorm ierr)
    (declare
     (ignore var-0 var-1 var-2 var-5 var-9 var-10 var-11
      var-13 var-14))
    (when var-3 (setf smin var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf lda var-6))
    (when var-7 (setf one var-7))
    (when var-8 (setf one var-8))
    (when var-12 (setf zero var-12))
    (when var-15 (setf scaloc var-15))
    (when var-16 (setf xnorm var-16))
    (when var-17 (setf ierr var-17)))
  (if (/= ierr 0)
      (setf info 1))
  (cond
   ((/= scaloc one)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal m scaloc
             (f2cl-lib:array-slice c-%data% double-float
                                   (1 j) ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
       label200))
    (setf scale (* scale scaloc))))
  (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                       c-%offset%)
        (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
  (setf (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                       c-%offset%)
        (f2cl-lib:fref x (2 1) ((1 2) (1 2)))))
 ((and (/= l1 l2) (= k1 k2))
  (setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub m k1)
             (f2cl-lib:array-slice a-%data% double-float
```

```
                                        (k1
                                         (min
                                          (the f2cl-lib:integer4
                                               (f2cl-lib:int-add
                                                k1 1))
                                          (the f2cl-lib:integer4
                                               m)))
                                        ((1 lda) (1 *))
                                        a-%offset%)
                        lda
                        (f2cl-lib:array-slice c-%data% double-float
                                        ((min
                                          (f2cl-lib:int-add k1
                                                            1)
                                          m)
                                         l1)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                        1)
                      (declare (ignore var-0 var-1 var-3 var-4))
                      (when var-2 (setf lda var-2))
                      ret-val))
            (setf sumr
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4)
                        (ddot (f2cl-lib:int-sub n l2)
                         (f2cl-lib:array-slice c-%data% double-float
                                        (k1
                                         (min
                                          (the f2cl-lib:integer4
                                               (f2cl-lib:int-add
                                                l2 1))
                                          (the f2cl-lib:integer4
                                               n)))
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                         ldc
                         (f2cl-lib:array-slice b-%data% double-float
                                        (l1
                                         (min
                                          (the f2cl-lib:integer4
                                               (f2cl-lib:int-add
                                                l2 1))
                                          (the f2cl-lib:integer4
                                               n)))
                                        ((1 ldb$) (1 *))
                                        b-%offset%)
                         ldb$)
                      (declare (ignore var-0 var-1 var-3))
                      (when var-2 (setf ldc var-2))
```

```
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (* sgn
           (-
            (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                           c-%offset%)
            (+ suml (* sgn sumr)))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub m k1)
             (f2cl-lib:array-slice a-%data% double-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           k1 1))
                                     (the f2cl-lib:integer4
                                          m)))
                                   ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice c-%data% double-float
                                   ((min
                                     (f2cl-lib:int-add k1
                                                       1)
                                     m)
                                    l2)
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf lda var-2))
          ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
```

```
                (f2cl-lib:array-slice b-%data% double-float
                                      (l2
                                       (min
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-add
                                              l2 1))
                                        (the f2cl-lib:integer4
                                             n)))
                                      ((1 ldb$) (1 *))
                                      b-%offset%)
                ldb$)
            (declare (ignore var-0 var-1 var-3))
            (when var-2 (setf ldc var-2))
            (when var-4 (setf ldb$ var-4))
            ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (* sgn
           (-
            (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                           c-%offset%)
            (+ suml (* sgn sumr)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
    (dlaln2 f2cl-lib:%false% 2 1 smin one
     (f2cl-lib:array-slice b-%data% double-float (l1 l1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ one one vec 2
     (* (- sgn)
        (f2cl-lib:fref a-%data% (k1 k1) ((1 lda) (1 *))
                       a-%offset%))
     zero x 2 scaloc xnorm ierr)
  (declare
   (ignore var-0 var-1 var-2 var-5 var-9 var-10 var-11
    var-13 var-14))
  (when var-3 (setf smin var-3))
  (when var-4 (setf one var-4))
  (when var-6 (setf ldb$ var-6))
  (when var-7 (setf one var-7))
  (when var-8 (setf one var-8))
  (when var-12 (setf zero var-12))
  (when var-15 (setf scaloc var-15))
  (when var-16 (setf xnorm var-16))
  (when var-17 (setf ierr var-17)))
(if (/= ierr 0)
    (setf info 1))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                ((> j n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal m scaloc
           (f2cl-lib:array-slice c-%data% double-float
                                 (1 j) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
     label210))
  (setf scale (* scale scaloc))))
 (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                      c-%offset%)
        (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
 (setf (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                      c-%offset%)
        (f2cl-lib:fref x (2 1) ((1 2) (1 2)))))
((and (/= l1 l2) (/= k1 k2))
 (setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub m k2)
             (f2cl-lib:array-slice a-%data% double-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           k2 1))
                                     (the f2cl-lib:integer4
                                          m)))
                                   ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice c-%data% double-float
                                   ((min
                                     (f2cl-lib:int-add k2
                                                       1)
                                     m)
                                    l1)
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             1)
           (declare (ignore var-0 var-1 var-3 var-4))
           (when var-2 (setf lda var-2))
           ret-val))
 (setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
```

```
                    (ddot (f2cl-lib:int-sub n l2)
                     (f2cl-lib:array-slice c-%data% double-float
                                           (k1
                                            (min
                                             (the f2cl-lib:integer4
                                                  (f2cl-lib:int-add
                                                   l2 1))
                                             (the f2cl-lib:integer4
                                                  n)))
                                           ((1 ldc) (1 *))
                                           c-%offset%)
                     ldc
                     (f2cl-lib:array-slice b-%data% double-float
                                           (l1
                                            (min
                                             (the f2cl-lib:integer4
                                                  (f2cl-lib:int-add
                                                   l2 1))
                                             (the f2cl-lib:integer4
                                                  n)))
                                           ((1 ldb$) (1 *))
                                           b-%offset%)
                     ldb$)
                   (declare (ignore var-0 var-1 var-3))
                   (when var-2 (setf ldc var-2))
                   (when var-4 (setf ldb$ var-4))
                   ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub m k2)
             (f2cl-lib:array-slice a-%data% double-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           k2 1))
                                     (the f2cl-lib:integer4
                                          m)))
                                   ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice c-%data% double-float
                                   ((min
                                     (f2cl-lib:int-add k2
```

```
                                                                  1)
                                        m)
                                       l2)
                                     ((1 ldc) (1 *))
                                     c-%offset%)
                 1)
              (declare (ignore var-0 var-1 var-3 var-4))
              (when var-2 (setf lda var-2))
              ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                   (l2
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldb$) (1 *))
                                   b-%offset%)
             ldb$)
              (declare (ignore var-0 var-1 var-3))
              (when var-2 (setf ldc var-2))
              (when var-4 (setf ldb$ var-4))
              ret-val))
(setf (f2cl-lib:fref vec (1 2) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub m k2)
             (f2cl-lib:array-slice a-%data% double-float
                                   (k2
```

```
                                        (min
                                         (the f2cl-lib:integer4
                                              (f2cl-lib:int-add
                                               k2 1))
                                         (the f2cl-lib:integer4
                                              m)))
                                       ((1 lda) (1 *))
                                       a-%offset%)
                   lda
                   (f2cl-lib:array-slice c-%data% double-float
                                       ((min
                                         (f2cl-lib:int-add k2
                                                           1)
                                         m)
                                        l1)
                                       ((1 ldc) (1 *))
                                       c-%offset%)
                   1)
                 (declare (ignore var-0 var-1 var-3 var-4))
                 (when var-2 (setf lda var-2))
                 ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% double-float
                                       (k2
                                        (min
                                         (the f2cl-lib:integer4
                                              (f2cl-lib:int-add
                                               l2 1))
                                         (the f2cl-lib:integer4
                                              n)))
                                       ((1 ldc) (1 *))
                                       c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                       (l1
                                        (min
                                         (the f2cl-lib:integer4
                                              (f2cl-lib:int-add
                                               l2 1))
                                         (the f2cl-lib:integer4
                                              n)))
                                       ((1 ldb$) (1 *))
                                       b-%offset%)
             ldb$)
            (declare (ignore var-0 var-1 var-3))
            (when var-2 (setf ldc var-2))
            (when var-4 (setf ldb$ var-4))
```

```
          ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub m k2)
             (f2cl-lib:array-slice a-%data% double-float
                                   (k2
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           k2 1))
                                     (the f2cl-lib:integer4
                                          m)))
                                   ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice c-%data% double-float
                                   ((min
                                     (f2cl-lib:int-add k2
                                                       1)
                                     m)
                                    l2)
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf lda var-2))
          ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% double-float
                                   (k2
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% double-float
                                   (l2
```

```
                                        (min
                                         (the f2cl-lib:integer4
                                              (f2cl-lib:int-add
                                               l2 1))
                                         (the f2cl-lib:integer4
                                              n)))
                                      ((1 ldb$) (1 *))
                                      b-%offset%)
                 ldb$)
              (declare (ignore var-0 var-1 var-3))
              (when var-2 (setf ldc var-2))
              (when var-4 (setf ldb$ var-4))
              ret-val))
(setf (f2cl-lib:fref vec (2 2) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l2) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15)
    (dlasy2 f2cl-lib:%false% f2cl-lib:%true% isgn 2 2
     (f2cl-lib:array-slice a-%data% double-float (k1 k1)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% double-float (l1 l1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ vec 2 scaloc x 2 xnorm ierr)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-12 var-13))
  (when var-2 (setf isgn var-2))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-11 (setf scaloc var-11))
  (when var-14 (setf xnorm var-14))
  (when var-15 (setf ierr var-15)))
(if (/= ierr 0)
    (setf info 1))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal m scaloc
           (f2cl-lib:array-slice c-%data% double-float
                                 (1 j) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
```

```
                        (declare (ignore var-2 var-3))
                        (when var-0 (setf m var-0))
                        (when var-1 (setf scaloc var-1)))
                    label220))
                  (setf scale (* scale scaloc))))
                (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                                     c-%offset%)
                        (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
                (setf (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                                     c-%offset%)
                        (f2cl-lib:fref x (1 2) ((1 2) (1 2))))
                (setf (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                                     c-%offset%)
                        (f2cl-lib:fref x (2 1) ((1 2) (1 2))))
                (setf (f2cl-lib:fref c-%data% (k2 l2) ((1 ldc) (1 *))
                                     c-%offset%)
                        (f2cl-lib:fref x (2 2) ((1 2) (1 2))))))
            label230))
        label240))))
    (go end_label)
   end_label
    (return
     (values trana tranb isgn m n nil lda nil ldb$ nil ldc scale info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dtrsyl
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::trana
                                              fortran-to-lisp::tranb
                                              fortran-to-lisp::isgn
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$ nil
```

```
  fortran-to-lisp::ldc
  fortran-to-lisp::scale
  fortran-to-lisp::info)
:calls 'nil)))
```

Chapter 59

LAPACK - Symmetric Matrix, Comp. Routines, Real

59.1 sla_syamv

59.1.1 Axiom unit tests

— slasyamv.input —

```
)set break resume
)sys rm -f slasyamv.output
)spool slasyamv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.1.2 Axiom help page

— sla_syamv.help —

```
==========================================================================
sla_syamv examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sla_syamv (UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)
           SLA_SYAMV computes a matrix-vector product using a symmetric
           indefinite matrix to calculate error bounds.

Function/Subroutine Documentation
   subroutine sla_syamv (integer UPLO, integer N, real ALPHA, real, dimension(
       lda, * ) A, integer LDA, real, dimension( * ) X, integer INCX, real
       BETA, real, dimension( * ) Y, integer INCY)
       SLA_SYAMV computes a matrix-vector product using a symmetric indefinite
       matrix to calculate error bounds.

Purpose:
   SLA_SYAMV  performs the matrix-vector operation

   y := alpha*abs(A)*abs(x) + beta*abs(y),

   where alpha and beta are scalars, x and y are vectors and A is an
   n by n symmetric matrix.

   This function is primarily used in calculating error bounds.
   To protect against underflow during evaluation, components in
   the resulting vector are perturbed away from zero by (N+1)
   times the underflow threshold.  To prevent unnecessarily large
   errors for block-structure embedded in general matrices,
   "symbolically" zero components are not perturbed.  A zero
   entry is considered "symbolic" if all multiplications involved
   in computing that entry have at least one zero multiplicand.

Parameters:

 UPLO
   UPLO is INTEGER
   On entry, UPLO specifies whether the upper or lower
   triangular part of the array A is to be referenced as
   follows:

   UPLO = BLAS_UPPER   Only the upper triangular part of A is to be referenced.
   UPLO = BLAS_LOWER   Only the lower triangular part of A is to be referenced.

 Unchanged on exit.

 N
   N is INTEGER
```

```
  On entry, N specifies the number of columns of the matrix A.
  N must be at least zero.
  Unchanged on exit.

ALPHA
  ALPHA is REAL .
  On entry, ALPHA specifies the scalar alpha.
  Unchanged on exit.

A
  A is REAL array of DIMENSION ( LDA, n ).
  Before entry, the leading m by n part of the array A must
  contain the matrix of coefficients.
  Unchanged on exit.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, n ).
  Unchanged on exit.

X
  X is REAL array, dimension
  ( 1 + ( n - 1 )*abs( INCX ) )
  Before entry, the incremented array X must contain the vector x.
  Unchanged on exit.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of
  X. INCX must not be zero.
  Unchanged on exit.

BETA
  BETA is REAL .
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.
  Unchanged on exit.

Y
  Y is REAL array, dimension
  ( 1 + ( n - 1 )*abs( INCY ) )
  Before entry with BETA non-zero, the incremented array Y
  must contain the vector y. On exit, Y is overwritten by the
  updated vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of
  Y. INCY must not be zero.
```

```
Unchanged on exit.
```

———-

59.1.3 fortran code

— **sla_syamv.f** —

```
*  =====================================================================
      SUBROUTINE SLA_SYAMV( UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y,
     $                      INCY )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      REAL               ALPHA, BETA
      INTEGER            INCX, INCY, LDA, N, UPLO
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), X( * ), Y( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            SYMB_ZERO
      REAL               TEMP, SAFE1
      INTEGER            I, INFO, IY, J, JX, KX, KY
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, SLAMCH
      REAL               SLAMCH
*     ..
*     .. External Functions ..
      EXTERNAL           ILAUPLO
      INTEGER            ILAUPLO
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, ABS, SIGN
*     ..
```

```
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF     ( UPLO.NE.ILAUPLO( 'U' ) .AND.
     $         UPLO.NE.ILAUPLO( 'L' ) ) THEN
         INFO = 1
      ELSE IF( N.LT.0 )THEN
         INFO = 2
      ELSE IF( LDA.LT.MAX( 1, N ) )THEN
         INFO = 5
      ELSE IF( INCX.EQ.0 )THEN
         INFO = 7
      ELSE IF( INCY.EQ.0 )THEN
         INFO = 10
      END IF
      IF( INFO.NE.0 )THEN
         CALL XERBLA( 'SSYMV ', INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( ( N.EQ.0 ).OR.( ( ALPHA.EQ.ZERO ).AND.( BETA.EQ.ONE ) ) )
     $   RETURN
*
*     Set up the start points in  X  and  Y.
*
      IF( INCX.GT.0 )THEN
         KX = 1
      ELSE
         KX = 1 - ( N - 1 )*INCX
      END IF
      IF( INCY.GT.0 )THEN
         KY = 1
      ELSE
         KY = 1 - ( N - 1 )*INCY
      END IF
*
*     Set SAFE1 essentially to be the underflow threshold times the
*     number of additions in each row.
*
      SAFE1 = SLAMCH( 'Safe minimum' )
      SAFE1 = (N+1)*SAFE1
*
*     Form  y := alpha*abs(A)*abs(x) + beta*abs(y).
*
*     The O(N^2) SYMB_ZERO tests could be replaced by O(N) queries to
*     the inexact flag.  Still doesn't help change the iteration order
```

```
*     to per-column.
*
      IY = KY
      IF ( INCX.EQ.1 ) THEN
         IF ( UPLO .EQ. ILAUPLO( 'U' ) ) THEN
            DO I = 1, N
               IF ( BETA .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
                  Y( IY ) = 0.0
               ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
               ELSE
                  SYMB_ZERO = .FALSE.
                  Y( IY ) = BETA * ABS( Y( IY ) )
               END IF
               IF ( ALPHA .NE. ZERO ) THEN
                  DO J = 1, I
                     TEMP = ABS( A( J, I ) )
                     SYMB_ZERO = SYMB_ZERO .AND.
     $                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                     Y( IY ) = Y( IY ) + ALPHA*ABS( X( J ) )*TEMP
                  END DO
                  DO J = I+1, N
                     TEMP = ABS( A( I, J ) )
                     SYMB_ZERO = SYMB_ZERO .AND.
     $                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                     Y( IY ) = Y( IY ) + ALPHA*ABS( X( J ) )*TEMP
                  END DO
               END IF

               IF ( .NOT.SYMB_ZERO )
     $              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

               IY = IY + INCY
            END DO
         ELSE
            DO I = 1, N
               IF ( BETA .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
                  Y( IY ) = 0.0
               ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
               ELSE
                  SYMB_ZERO = .FALSE.
                  Y( IY ) = BETA * ABS( Y( IY ) )
               END IF
               IF ( ALPHA .NE. ZERO ) THEN
                  DO J = 1, I
```

```
                TEMP = ABS( A( I, J ) )
                SYMB_ZERO = SYMB_ZERO .AND.
$                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                Y( IY ) = Y( IY ) + ALPHA*ABS( X( J ) )*TEMP
             END DO
             DO J = I+1, N
                TEMP = ABS( A( J, I ) )
                SYMB_ZERO = SYMB_ZERO .AND.
$                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                Y( IY ) = Y( IY ) + ALPHA*ABS( X( J ) )*TEMP
             END DO
          END IF

          IF ( .NOT.SYMB_ZERO )
$              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

          IY = IY + INCY
       END DO
    END IF
 ELSE
    IF ( UPLO .EQ. ILAUPLO( 'U' ) ) THEN
       DO I = 1, N
          IF ( BETA .EQ. ZERO ) THEN
             SYMB_ZERO = .TRUE.
             Y( IY ) = 0.0
          ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
             SYMB_ZERO = .TRUE.
          ELSE
             SYMB_ZERO = .FALSE.
             Y( IY ) = BETA * ABS( Y( IY ) )
          END IF
          JX = KX
          IF ( ALPHA .NE. ZERO ) THEN
             DO J = 1, I
                TEMP = ABS( A( J, I ) )
                SYMB_ZERO = SYMB_ZERO .AND.
$                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                Y( IY ) = Y( IY ) + ALPHA*ABS( X( JX ) )*TEMP
                JX = JX + INCX
             END DO
             DO J = I+1, N
                TEMP = ABS( A( I, J ) )
                SYMB_ZERO = SYMB_ZERO .AND.
$                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                Y( IY ) = Y( IY ) + ALPHA*ABS( X( JX ) )*TEMP
                JX = JX + INCX
```

```
               END DO
            END IF

            IF ( .NOT.SYMB_ZERO )
     $           Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

            IY = IY + INCY
         END DO
      ELSE
         DO I = 1, N
            IF ( BETA .EQ. ZERO ) THEN
               SYMB_ZERO = .TRUE.
               Y( IY ) = 0.0
            ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
               SYMB_ZERO = .TRUE.
            ELSE
               SYMB_ZERO = .FALSE.
               Y( IY ) = BETA * ABS( Y( IY ) )
            END IF
            JX = KX
            IF ( ALPHA .NE. ZERO ) THEN
               DO J = 1, I
                  TEMP = ABS( A( I, J ) )
                  SYMB_ZERO = SYMB_ZERO .AND.
     $                 ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                  Y( IY ) = Y( IY ) + ALPHA*ABS( X( JX ) )*TEMP
                  JX = JX + INCX
               END DO
               DO J = I+1, N
                  TEMP = ABS( A( J, I ) )
                  SYMB_ZERO = SYMB_ZERO .AND.
     $                 ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                  Y( IY ) = Y( IY ) + ALPHA*ABS( X( JX ) )*TEMP
                  JX = JX + INCX
               END DO
            END IF

            IF ( .NOT.SYMB_ZERO )
     $           Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

            IY = IY + INCY
         END DO
      END IF

   END IF
*
   RETURN
*
```

```
*     End of SLA_SYAMV
*
      END
```

———-

59.1.4 lisp code

— **sla_syamv.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun sla_syamv (uplo n alpha a lda x incx beta y incy)
    (declare (type (array single-float (*)) y x a)
             (type (single-float) beta alpha)
             (type (f2cl-lib:integer4) incy incx lda n uplo))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (x single-float x-%data% x-%offset%)
         (y single-float y-%data% y-%offset%))
      (prog ((i 0) (info 0) (iy 0) (j 0) (jx 0) (kx 0) (ky 0) (temp 0.0)
             (safe1 0.0) (symb_zero nil))
        (declare (type (f2cl-lib:integer4) i info iy j jx kx ky)
                 (type (single-float) temp safe1)
                 (type f2cl-lib:logical symb_zero))
        (setf info 0)
        (cond
         ((and (/= uplo (ilauplo "U")) (/= uplo (ilauplo "L"))) (setf info 1))
         ((< n 0) (setf info 2))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info 5))
         ((= incx 0) (setf info 7)) ((= incy 0) (setf info 10)))
        (cond
         ((/= info 0)
          (multiple-value-bind (var-0 var-1)
              (xerbla "SSYMV " info)
            (declare (ignore var-0))
            (when var-1 (setf info var-1)))
          (go end_label)))
        (if (or (= n 0) (and (= alpha zero) (= beta one)))
            (go end_label))
        (cond ((> incx 0) (setf kx 1))
              (t
               (setf kx
                       (f2cl-lib:int-sub 1
                                         (f2cl-lib:int-mul
```

```
                                  (f2cl-lib:int-sub n 1) incx)))))
(cond ((> incy 0) (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incy)))))
(setf safe1 (slamch "Safe minimum"))
(setf safe1 (* (f2cl-lib:int-add n 1) safe1))
(setf iy ky)
(cond
 ((= incx 1)
  (cond
   ((= uplo (ilauplo "U"))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (cond
         ((= beta zero) (setf symb_zero f2cl-lib:%true%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) 0.0))
         ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
          (setf symb_zero f2cl-lib:%true%))
         (t (setf symb_zero f2cl-lib:%false%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (* beta
                     (abs
                      (f2cl-lib:fref y-%data% (iy) ((1 *))
                                     y-%offset%))))))
        (cond
         ((/= alpha zero)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j i) nil)
            (tagbody
              (setf temp
                      (abs
                       (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                      a-%offset%)))
              (setf symb_zero
                      (and symb_zero
                           (or
                            (=
                             (f2cl-lib:fref x-%data% (j) ((1 *))
                                            x-%offset%)
                             zero)
                            (= temp zero))))
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (+
                       (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (* alpha
                          (abs
```

```
                          (f2cl-lib:fref x-%data% (j) ((1 *))
                                         x-%offset%))
                        temp)))
           label100001))
        (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                       (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf temp
                    (abs
                     (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%)))
            (setf symb_zero
                    (and symb_zero
                         (or
                          (=
                           (f2cl-lib:fref x-%data% (j) ((1 *))
                                          x-%offset%)
                           zero)
                          (= temp zero))))
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+
                     (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (* alpha
                        (abs
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%))
                        temp)))
           label100002))))
      (if (not symb_zero)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (f2cl-lib:sign safe1
                                    (f2cl-lib:fref y-%data% (iy)
                                                   ((1 *))
                                                   y-%offset%)))))
      (setf iy (f2cl-lib:int-add iy incy))
     label100000)))
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (cond
      ((= beta zero) (setf symb_zero f2cl-lib:%true%)
       (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) 0.0))
      ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
       (setf symb_zero f2cl-lib:%true%))
      (t (setf symb_zero f2cl-lib:%false%)
       (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
               (* beta
```

```
                (abs
                 (f2cl-lib:fref y-%data% (iy) ((1 *))
                                y-%offset%))))))
(cond
 ((/= alpha zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j i) nil)
    (tagbody
      (setf temp
              (abs
               (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                              a-%offset%)))
      (setf symb_zero
              (and symb_zero
                   (or
                    (=
                     (f2cl-lib:fref x-%data% (j) ((1 *))
                                    x-%offset%)
                     zero)
                    (= temp zero))))
      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
              (+
               (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
               (* alpha
                  (abs
                   (f2cl-lib:fref x-%data% (j) ((1 *))
                                  x-%offset%))
                  temp)))
     label100004))
  (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf temp
              (abs
               (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                              a-%offset%)))
      (setf symb_zero
              (and symb_zero
                   (or
                    (=
                     (f2cl-lib:fref x-%data% (j) ((1 *))
                                    x-%offset%)
                     zero)
                    (= temp zero))))
      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
              (+
               (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
               (* alpha
                  (abs
```

```
                              (f2cl-lib:fref x-%data% (j) ((1 *))
                                             x-%offset%))
                            temp)))
                 label100005))))
            (if (not symb_zero)
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                           (f2cl-lib:sign safe1
                                          (f2cl-lib:fref y-%data% (iy)
                                                         ((1 *))
                                                         y-%offset%)))))
            (setf iy (f2cl-lib:int-add iy incy))
           label100003)))))
      (t
       (cond
        ((= uplo (ilauplo "U"))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (cond
              ((= beta zero) (setf symb_zero f2cl-lib:%true%)
               (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) 0.0))
              ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
               (setf symb_zero f2cl-lib:%true%))
              (t (setf symb_zero f2cl-lib:%false%)
               (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (* beta
                          (abs
                           (f2cl-lib:fref y-%data% (iy) ((1 *))
                                          y-%offset%))))))
             (setf jx kx)
             (cond
              ((/= alpha zero)
               (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                             ((> j i) nil)
                 (tagbody
                   (setf temp
                           (abs
                            (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                           a-%offset%)))
                   (setf symb_zero
                           (and symb_zero
                                (or
                                 (=
                                  (f2cl-lib:fref x-%data% (j) ((1 *))
                                                 x-%offset%)
                                  zero)
                                 (= temp zero))))
                   (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                           (+
```

```
                        (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (* alpha
                           (abs
                            (f2cl-lib:fref x-%data% (jx) ((1 *))
                                           x-%offset%))
                           temp)))
            (setf jx (f2cl-lib:int-add jx incx))
           label100007))
        (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                       (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf temp
                    (abs
                     (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%)))
            (setf symb_zero
                    (and symb_zero
                         (or
                          (=
                           (f2cl-lib:fref x-%data% (j) ((1 *))
                                          x-%offset%)
                           zero)
                          (= temp zero))))
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+
                     (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (* alpha
                        (abs
                         (f2cl-lib:fref x-%data% (jx) ((1 *))
                                        x-%offset%))
                        temp)))
            (setf jx (f2cl-lib:int-add jx incx))
           label100008))))
      (if (not symb_zero)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                   (f2cl-lib:sign safe1
                                  (f2cl-lib:fref y-%data% (iy)
                                                 ((1 *))
                                                 y-%offset%)))))
      (setf iy (f2cl-lib:int-add iy incy))
     label100006)))
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (cond
      ((= beta zero) (setf symb_zero f2cl-lib:%true%)
       (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) 0.0))
```

```
 ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
  (setf symb_zero f2cl-lib:%true%))
 (t (setf symb_zero f2cl-lib:%false%)
  (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
          (* beta
             (abs
              (f2cl-lib:fref y-%data% (iy) ((1 *))
                             y-%offset%))))))
(setf jx kx)
(cond
 ((/= alpha zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j i) nil)
    (tagbody
      (setf temp
              (abs
               (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                              a-%offset%)))
      (setf symb_zero
              (and symb_zero
                   (or
                    (=
                     (f2cl-lib:fref x-%data% (j) ((1 *))
                                    x-%offset%)
                     zero)
                    (= temp zero))))
      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
              (+
               (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
               (* alpha
                  (abs
                   (f2cl-lib:fref x-%data% (jx) ((1 *))
                                  x-%offset%))
                  temp)))
      (setf jx (f2cl-lib:int-add jx incx))
     label100010))
  (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf temp
              (abs
               (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                              a-%offset%)))
      (setf symb_zero
              (and symb_zero
                   (or
                    (=
                     (f2cl-lib:fref x-%data% (j) ((1 *))
                                    x-%offset%)
```

```
                                zero)
                               (= temp zero))))
                    (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                            (+
                             (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                             (* alpha
                                (abs
                                 (f2cl-lib:fref x-%data% (jx) ((1 *))
                                                x-%offset%))
                                temp)))
                    (setf jx (f2cl-lib:int-add jx incx))
                   label100011))))
              (if (not symb_zero)
                  (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                          (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                             (f2cl-lib:sign safe1
                                            (f2cl-lib:fref y-%data% (iy)
                                                           ((1 *))
                                                           y-%offset%)))))
              (setf iy (f2cl-lib:int-add iy incy))
             label100009))))))
       (go end_label)
      end_label
       (return (values nil nil nil nil nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sla_syamv
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (single-float)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (single-float)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(nil nil nil nil nil nil nil nil
                                             nil nil)
                                           :calls
                                           '(fortran-to-lisp::ilauplo))))
```

———-

59.2 sla_syrcond

59.2.1 Axiom unit tests

— **slasyrcond.input** —

```
)set break resume
)sys rm -f slasyrcond.output
)spool slasyrcond.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.2.2 Axiom help page

— **sla_syrcond.help** —

```
======================================================================
sla_syrcond examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       real function sla_syrcond (UPLO, N, A, LDA, AF, LDAF, IPIV, CMODE, C,
           INFO, WORK, IWORK)
           SLA_SYRCOND estimates the Skeel condition number for a symmetric
           indefinite matrix.

Function/Subroutine Documentation
   real function sla_syrcond (character UPLO, integer N, real, dimension( lda,
       * ) A, integer LDA, real, dimension( ldaf, * ) AF, integer LDAF,
       integer, dimension( * ) IPIV, integer CMODE, real, dimension( * ) C,
       integer INFO, real, dimension( * ) WORK, integer, dimension( * ) IWORK)
       SLA_SYRCOND estimates the Skeel condition number for a symmetric
       indefinite matrix.

Purpose:
```

```
  SLA_SYRCOND estimates the Skeel condition number of  op(A) * op2(C)
  where op2 is determined by CMODE as follows
  CMODE =  1    op2(C) = C
  CMODE =  0    op2(C) = I
  CMODE = -1    op2(C) = inv(C)
  The Skeel condition number cond(A) = norminf( |inv(A)||A| )
  is computed by computing scaling factors R such that
  diag(R)*A*op2(C) is row equilibrated and computing the standard
  infinity-norm condition number.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is REAL array, dimension (LDAF,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by SSYTRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by SSYTRF.

 CMODE
   CMODE is INTEGER
   Determines op2(C) in the formula op(A) * op2(C) as follows:
   CMODE =  1    op2(C) = C
   CMODE =  0    op2(C) = I
   CMODE = -1    op2(C) = inv(C)
```

```
C
  C is REAL array, dimension (N)
  The vector C in the formula op(A) * op2(C).

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  i > 0:  The ith argument is invalid.

WORK
  WORK is REAL array, dimension (3*N).
  Workspace.

IWORK
  IWORK is INTEGER array, dimension (N).
  Workspace.
```

59.2.3 fortran code

— sla_syrcond.f —

```
*  ====================================================================
      REAL FUNCTION SLA_SYRCOND( UPLO, N, A, LDA, AF, LDAF, IPIV, CMODE,
     $                           C, INFO, WORK, IWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            N, LDA, LDAF, INFO, CMODE
*     ..
*     .. Array Arguments
      INTEGER            IWORK( * ), IPIV( * )
      REAL               A( LDA, * ), AF( LDAF, * ), WORK( * ), C( * )
*     ..
*
*  ====================================================================
*
*     .. Local Scalars ..
      CHARACTER          NORMIN
      INTEGER            KASE, I, J
      REAL               AINVNM, SMLNUM, TMP
```

```
      LOGICAL            UP
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ISAMAX
      REAL               SLAMCH
      EXTERNAL           LSAME, ISAMAX, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLACN2, SLATRS, SRSCL, XERBLA, SSYTRS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
      SLA_SYRCOND = 0.0
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SLA_SYRCOND', -INFO )
         RETURN
      END IF
      IF( N.EQ.0 ) THEN
         SLA_SYRCOND = 1.0
         RETURN
      END IF
      UP = .FALSE.
      IF ( LSAME( UPLO, 'U' ) ) UP = .TRUE.
*
*     Compute the equilibration matrix R such that
*     inv(R)*A*C has unit 1-norm.
*
      IF ( UP ) THEN
         DO I = 1, N
            TMP = 0.0
            IF ( CMODE .EQ. 1 ) THEN
               DO J = 1, I
                  TMP = TMP + ABS( A( J, I ) * C( J ) )
               END DO
```

```
            DO J = I+1, N
               TMP = TMP + ABS( A( I, J ) * C( J ) )
            END DO
         ELSE IF ( CMODE .EQ. 0 ) THEN
            DO J = 1, I
               TMP = TMP + ABS( A( J, I ) )
            END DO
            DO J = I+1, N
               TMP = TMP + ABS( A( I, J ) )
            END DO
         ELSE
            DO J = 1, I
               TMP = TMP + ABS( A( J, I ) / C( J ) )
            END DO
            DO J = I+1, N
               TMP = TMP + ABS( A( I, J ) / C( J ) )
            END DO
         END IF
         WORK( 2*N+I ) = TMP
      END DO
   ELSE
      DO I = 1, N
         TMP = 0.0
         IF ( CMODE .EQ. 1 ) THEN
            DO J = 1, I
               TMP = TMP + ABS( A( I, J ) * C( J ) )
            END DO
            DO J = I+1, N
               TMP = TMP + ABS( A( J, I ) * C( J ) )
            END DO
         ELSE IF ( CMODE .EQ. 0 ) THEN
            DO J = 1, I
               TMP = TMP + ABS( A( I, J ) )
            END DO
            DO J = I+1, N
               TMP = TMP + ABS( A( J, I ) )
            END DO
         ELSE
            DO J = 1, I
               TMP = TMP + ABS( A( I, J) / C( J ) )
            END DO
            DO J = I+1, N
               TMP = TMP + ABS( A( J, I) / C( J ) )
            END DO
         END IF
         WORK( 2*N+I ) = TMP
      END DO
   ENDIF
*
*     Estimate the norm of inv(op(A)).
```

```
*
      SMLNUM = SLAMCH( 'Safe minimum' )
      AINVNM = 0.0
      NORMIN = 'N'

      KASE = 0
   10 CONTINUE
      CALL SLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * WORK( 2*N+I )
            END DO

            IF ( UP ) THEN
               CALL SSYTRS( 'U', N, 1, AF, LDAF, IPIV, WORK, N, INFO )
            ELSE
               CALL SSYTRS( 'L', N, 1, AF, LDAF, IPIV, WORK, N, INFO )
            ENDIF
*
*           Multiply by inv(C).
*
            IF ( CMODE .EQ. 1 ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) / C( I )
               END DO
            ELSE IF ( CMODE .EQ. -1 ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF
         ELSE
*
*           Multiply by inv(C**T).
*
            IF ( CMODE .EQ. 1 ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) / C( I )
               END DO
            ELSE IF ( CMODE .EQ. -1 ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF

            IF ( UP ) THEN
               CALL SSYTRS( 'U', N, 1, AF, LDAF, IPIV, WORK, N, INFO )
```

```
            ELSE
               CALL SSYTRS( 'L', N, 1, AF, LDAF, IPIV, WORK, N, INFO )
            ENDIF
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * WORK( 2*N+I )
            END DO
         END IF
*
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM .NE. 0.0 )
     $   SLA_SYRCOND = ( 1.0 / AINVNM )
*
      RETURN
*
      END
```

———-

59.2.4 lisp code

— sla_syrcond.lisp —

```
(defun sla_syrcond (uplo n a lda af ldaf ipiv cmode c info work iwork)
  (declare (type (array f2cl-lib:integer4 (*)) iwork ipiv)
           (type (array single-float (*)) work c af a)
           (type (f2cl-lib:integer4) info cmode ldaf lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a single-float a-%data% a-%offset%)
       (af single-float af-%data% af-%offset%)
       (c single-float c-%data% c-%offset%)
       (work single-float work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
    (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4)) (up nil)
           (ainvnm 0.0) (smlnum 0.0) (tmp 0.0) (kase 0) (i 0) (j 0)
           (normin
            (make-array '(1) :element-type 'character :initial-element #\ ))
           (sla_syrcond 0.0))
      (declare (type (string 1) normin)
```

```
         (type (f2cl-lib:integer4) j i kase)
         (type (single-float) sla_syrcond tmp smlnum ainvnm)
         (type f2cl-lib:logical up)
         (type (array f2cl-lib:integer4 (3)) isave))
(setf sla_syrcond 0.0)
(setf info 0)
(cond ((< n 0) (setf info -2))
      ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -4))
      ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -6)))
(cond
 ((/= info 0) (xerbla "SLA_SYRCOND" (f2cl-lib:int-sub info))
  (go end_label)))
(cond ((= n 0) (setf sla_syrcond 1.0) (go end_label)))
(setf up f2cl-lib:%false%)
(if (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)
    (setf up f2cl-lib:%true%))
(cond
 (up
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf tmp 0.0)
      (cond
       ((= cmode 1)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j i) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (abs
                        (*
                         (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                        a-%offset%)
                         (f2cl-lib:fref c-%data% (j) ((1 *))
                                        c-%offset%)))))
           label100001))
        (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (abs
                        (*
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
```

```
                                    a-%offset%)
                      (f2cl-lib:fref c-%data% (j) ((1 *))
                                    c-%offset%)))))
        label100002)))
    ((= cmode 0)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j i) nil)
       (tagbody
         (setf tmp
                 (+ tmp
                    (abs
                     (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                    a-%offset%))))
        label100003))
     (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf tmp
                 (+ tmp
                    (abs
                     (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%))))
        label100004)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j i) nil)
       (tagbody
         (setf tmp
                 (+ tmp
                    (abs
                     (/
                      (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                     a-%offset%)
                      (f2cl-lib:fref c-%data% (j) ((1 *))
                                     c-%offset%)))))
        label100005))
     (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf tmp
                 (+ tmp
                    (abs
                     (/
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%)
                      (f2cl-lib:fref c-%data% (j) ((1 *))
                                     c-%offset%)))))
        label100006))))
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) i))
```

```
                          ((1 *)) work-%offset%)
             tmp)
    label100000)))
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf tmp 0.0)
     (cond
      ((= cmode 1)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j i) nil)
         (tagbody
           (setf tmp
                   (+ tmp
                      (abs
                       (*
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%)
                        (f2cl-lib:fref c-%data% (j) ((1 *))
                                       c-%offset%)))))
          label100008))
       (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (setf tmp
                   (+ tmp
                      (abs
                       (*
                        (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                       a-%offset%)
                        (f2cl-lib:fref c-%data% (j) ((1 *))
                                       c-%offset%)))))
          label100009)))
      ((= cmode 0)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j i) nil)
         (tagbody
           (setf tmp
                   (+ tmp
                      (abs
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%))))
          label100010))
       (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (setf tmp
                   (+ tmp
                      (abs
```

```
                          (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                         a-%offset%))))
             label100011)))
          (t
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j i) nil)
             (tagbody
               (setf tmp
                       (+ tmp
                          (abs
                           (/
                            (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                           a-%offset%)
                            (f2cl-lib:fref c-%data% (j) ((1 *))
                                           c-%offset%)))))
             label100012))
           (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (setf tmp
                       (+ tmp
                          (abs
                           (/
                            (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                           a-%offset%)
                            (f2cl-lib:fref c-%data% (j) ((1 *))
                                           c-%offset%)))))
             label100013))))
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) i))
                             ((1 *)) work-%offset%)
              tmp)
      label100007))))
  (setf smlnum (slamch "Safe minimum"))
  (setf ainvnm 0.0)
  (f2cl-lib:f2cl-set-string normin "N" (string 1))
  (setf kase 0)
 label10
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacn2 n
       (f2cl-lib:array-slice work-%data% single-float ((+ n 1)) ((1 *))
                             work-%offset%)
       work iwork ainvnm kase isave)
    (declare (ignore var-1 var-2 var-3 var-6))
    (setf n var-0)
    (setf ainvnm var-4)
    (setf kase var-5))
  (cond
   ((/= kase 0)
    (cond
```

```
((= kase 2)
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
             (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add
                                 (f2cl-lib:int-mul 2 n) i))
                               ((1 *)) work-%offset%)))
    label100014))
 (cond
  (up
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (ssytrs "U" n 1 af ldaf ipiv work n info)
     (declare (ignore var-0 var-2 var-3 var-5 var-6))
     (when var-1 (setf n var-1))
     (when var-4 (setf ldaf var-4))
     (when var-7 (setf n var-7))
     (when var-8 (setf info var-8))))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (ssytrs "L" n 1 af ldaf ipiv work n info)
     (declare (ignore var-0 var-2 var-3 var-5 var-6))
     (when var-1 (setf n var-1))
     (when var-4 (setf ldaf var-4))
     (when var-7 (setf n var-7))
     (when var-8 (setf info var-8)))))
 (cond
  ((= cmode 1)
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               (/
                (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
      label100015)))
  ((= cmode (f2cl-lib:int-sub 1))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               (*
                (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
      label100016)))))
(t
```

```
      (cond
       ((= cmode 1)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                    (/
                     (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                     (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
           label100017)))
       ((= cmode (f2cl-lib:int-sub 1))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                    (*
                     (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                     (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
           label100018))))
      (cond
       (up
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (ssytrs "U" n 1 af ldaf ipiv work n info)
          (declare (ignore var-0 var-2 var-3 var-5 var-6))
          (when var-1 (setf n var-1))
          (when var-4 (setf ldaf var-4))
          (when var-7 (setf n var-7))
          (when var-8 (setf info var-8))))
       (t
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (ssytrs "L" n 1 af ldaf ipiv work n info)
          (declare (ignore var-0 var-2 var-3 var-5 var-6))
          (when var-1 (setf n var-1))
          (when var-4 (setf ldaf var-4))
          (when var-7 (setf n var-7))
          (when var-8 (setf info var-8)))))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add
                                      (f2cl-lib:int-mul 2 n) i))
                                    ((1 *)) work-%offset%)))
         label100019))))
    (go label10)))
  (if (/= ainvnm 0.0)
```

```
          (setf sla_syrcond (/ 1.0 ainvnm)))
      (go end_label)
     end_label
      (return
       (values sla_syrcond uplo n nil nil nil ldaf nil nil nil info nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sla_syrcond
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*)))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::ldaf nil nil nil
                                              fortran-to-lisp::info nil nil)
                                            :calls '(fortran-to-lisp::slacn2))))
```

———-

59.3 sla_syrfsx_extended

59.3.1 Axiom unit tests

— slasyrfsxextended.input —

```
)set break resume
)sys rm -f slasyrfsxextended.output
)spool slasyrfsxextended.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.3.2 Axiom help page [Demm05]

— **sla_syrfsx_extended.help** —

```
==========================================================================
sla_syrfsx_extended examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sla_syrfsx_extended (PREC_TYPE, UPLO, N, NRHS, A, LDA, AF,
           LDAF, IPIV, COLEQU, C, B, LDB, Y, LDY, BERR_OUT, N_NORMS,
           ERR_BNDS_NORM, ERR_BNDS_COMP, RES, AYB, DY, Y_TAIL, RCOND, ITHRESH,
           RTHRESH, DZ_UB, IGNORE_CWISE, INFO)
           SLA_SYRFSX_EXTENDED improves the computed solution to a system of
           linear equations for symmetric indefinite matrices by performing
           extra-precise iterative refinement and provides error bounds and
           backward error estimates for the solution.

Function/Subroutine Documentation
   subroutine sla_syrfsx_extended (integer PREC_TYPE, character UPLO, integer
       N, integer NRHS, real, dimension( lda, * ) A, integer LDA, real,
       dimension( ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV,
       logical COLEQU, real, dimension( * ) C, real, dimension( ldb, * ) B,
       integer LDB, real, dimension( ldy, * ) Y, integer LDY, real, dimension(
       * ) BERR_OUT, integer N_NORMS, real, dimension( nrhs, * )
       ERR_BNDS_NORM, real, dimension( nrhs, * ) ERR_BNDS_COMP, real,
       dimension( * ) RES, real, dimension( * ) AYB, real, dimension( * ) DY,
       real, dimension( * ) Y_TAIL, real RCOND, integer ITHRESH, real RTHRESH,
       real DZ_UB, logical IGNORE_CWISE, integer INFO)
       SLA_SYRFSX_EXTENDED improves the computed solution to a system of
       linear equations for symmetric indefinite matrices by performing extra-
       precise iterative refinement and provides error bounds and backward
       error estimates for the solution.

Purpose:
```

```
  SLA_SYRFSX_EXTENDED improves the computed solution to a system of
  linear equations by performing extra-precise iterative refinement
  and provides error bounds and backward error estimates for the solution.
  This subroutine is called by SSYRFSX to perform iterative refinement.
  In addition to normwise error bound, the code provides maximum
  componentwise error bound if possible. See comments for ERR_BNDS_NORM
  and ERR_BNDS_COMP for details of the error bounds. Note that this
  subroutine is only resonsible for setting the second fields of
  ERR_BNDS_NORM and ERR_BNDS_COMP.

Parameters:

 PREC_TYPE
   PREC_TYPE is INTEGER
   Specifies the intermediate precision to be used in refinement.
   The value is defined by ILAPREC(P) where P is a CHARACTER and
   P    = 'S':  Single
        = 'D':  Double
        = 'I':  Indigenous
        = 'X', 'E':  Extra

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right-hand-sides, i.e., the number of columns of the
   matrix B.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is REAL array, dimension (LDAF,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by SSYTRF.

 LDAF
```

```
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by SSYTRF.

COLEQU
  COLEQU is LOGICAL
  If .TRUE. then column equilibration was done to A before calling
  this routine. This is needed to compute the solution and error
  bounds correctly.

C
  C is REAL array, dimension (N)
  The column scale factors for A. If COLEQU = .FALSE., C
  is not accessed. If C is input, each element of C should be a power
  of the radix to ensure a reliable solution and error estimates.
  Scaling by powers of the radix does not cause rounding errors unless
  the result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is REAL array, dimension (LDB,NRHS)
  The right-hand-side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

Y
  Y is REAL array, dimension (LDY,NRHS)
  On entry, the solution matrix X, as computed by SSYTRS.
  On exit, the improved solution matrix Y.

LDY
  LDY is INTEGER
  The leading dimension of the array Y.  LDY >= max(1,N).

BERR_OUT
  BERR_OUT is REAL array, dimension (NRHS)
  On exit, BERR_OUT(j) contains the componentwise relative backward
  error for right-hand-side j from the formula
  max(i) ( abs(RES(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
  where abs(Z) is the componentwise absolute value of the matrix
  or vector Z. This is computed by SLA_LIN_BERR.

N_NORMS
```

```
  N_NORMS is INTEGER
  Determines which error bounds to return (see ERR_BNDS_NORM
  and ERR_BNDS_COMP).
  If N_NORMS >= 1 return normwise error bounds.
  If N_NORMS >= 2 return componentwise error bounds.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
            max_j (abs(XTRUE(j,i) - X(j,i)))
             ------------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

ERR_BNDS_COMP
  ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
```

```
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
              abs(XTRUE(j,i) - X(j,i))
         max_j ----------------------
                    abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

RES
  RES is REAL array, dimension (N)
  Workspace to hold the intermediate residual.
```

```
AYB
  AYB is REAL array, dimension (N)
  Workspace. This can be the same workspace passed for Y_TAIL.

DY
  DY is REAL array, dimension (N)
  Workspace to hold the intermediate solution.

Y_TAIL
  Y_TAIL is REAL array, dimension (N)
  Workspace to hold the trailing bits of the intermediate solution.

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

ITHRESH
  ITHRESH is INTEGER
  The maximum number of residual computations allowed for
  refinement. The default is 10. For 'aggressive' set to 100 to
  permit convergence using approximate factorizations or
  factorizations other than LU. If the factorization uses a
  technique other than Gaussian elimination, the guarantees in
  ERR_BNDS_NORM and ERR_BNDS_COMP may no longer be trustworthy.

RTHRESH
  RTHRESH is REAL
  Determines when to stop refinement if the error estimate stops
  decreasing. Refinement will stop when the next solution no longer
  satisfies norm(dx_{i+1}) < RTHRESH * norm(dx_i) where norm(Z) is
  the infinity norm of Z. RTHRESH satisfies 0 < RTHRESH <= 1. The
  default value is 0.5. For 'aggressive' set to 0.9 to permit
  convergence on extremely ill-conditioned matrices. See LAWN 165
  for more details.

DZ_UB
  DZ_UB is REAL
  Determines when to start considering componentwise convergence.
  Componentwise convergence is only considered after each component
  of the solution Y is stable, which we definte as the relative
  change in each component being less than DZ_UB. The default value
  is 0.25, requiring the first bit to be stable. See LAWN 165 for
  more details.

IGNORE_CWISE
```

```
  IGNORE_CWISE is LOGICAL
  If .TRUE. then ignore componentwise convergence. Default value
  is .FALSE..

 INFO
  INFO is INTEGER
  = 0:  Successful exit.
  < 0:  if INFO = -i, the ith argument to SLA_SYRFSX_EXTENDED had an illegal
  value
```

———-

59.3.3 fortran code

— sla_syrfsx_extended.f —

```
*  =====================================================================
      SUBROUTINE SLA_SYRFSX_EXTENDED( PREC_TYPE, UPLO, N, NRHS, A, LDA,
     $                                AF, LDAF, IPIV, COLEQU, C, B, LDB,
     $                                Y, LDY, BERR_OUT, N_NORMS,
     $                                ERR_BNDS_NORM, ERR_BNDS_COMP, RES,
     $                                AYB, DY, Y_TAIL, RCOND, ITHRESH,
     $                                RTHRESH, DZ_UB, IGNORE_CWISE,
     $                                INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDAF, LDB, LDY, N, NRHS, PREC_TYPE,
     $                   N_NORMS, ITHRESH
      CHARACTER          UPLO
      LOGICAL            COLEQU, IGNORE_CWISE
      REAL               RTHRESH, DZ_UB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   Y( LDY, * ), RES( * ), DY( * ), Y_TAIL( * )
      REAL               C( * ), AYB( * ), RCOND, BERR_OUT( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  =====================================================================
```

```
*
*     .. Local Scalars ..
      INTEGER            UPLO2, CNT, I, J, X_STATE, Z_STATE
      REAL               YK, DYK, YMIN, NORMY, NORMX, NORMDX, DXRAT,
     $                   DZRAT, PREVNORMDX, PREV_DZ_Z, DXRATMAX,
     $                   DZRATMAX, DX_X, DZ_Z, FINAL_DX_X, FINAL_DZ_Z,
     $                   EPS, HUGEVAL, INCR_THRESH
      LOGICAL            INCR_PREC, UPPER
*     ..
*     .. Parameters ..
      INTEGER            UNSTABLE_STATE, WORKING_STATE, CONV_STATE,
     $                   NOPROG_STATE, Y_PREC_STATE, BASE_RESIDUAL,
     $                   EXTRA_RESIDUAL, EXTRA_Y
      PARAMETER          ( UNSTABLE_STATE = 0, WORKING_STATE = 1,
     $                   CONV_STATE = 2, NOPROG_STATE = 3 )
      PARAMETER          ( BASE_RESIDUAL = 0, EXTRA_RESIDUAL = 1,
     $                   EXTRA_Y = 2 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           ILAUPLO
      INTEGER            ILAUPLO
*     ..
*     .. External Subroutines ..
      EXTERNAL           SAXPY, SCOPY, SSYTRS, SSYMV, BLAS_SSYMV_X,
     $                   BLAS_SSYMV2_X, SLA_SYAMV, SLA_WWADDW,
     $                   SLA_LIN_BERR
      REAL               SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
```

```
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -13
      ELSE IF( LDY.LT.MAX( 1, N ) ) THEN
         INFO = -15
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SLA_SYRFSX_EXTENDED', -INFO )
         RETURN
      END IF
      EPS = SLAMCH( 'Epsilon' )
      HUGEVAL = SLAMCH( 'Overflow' )
*     Force HUGEVAL to Inf
      HUGEVAL = HUGEVAL * HUGEVAL
*     Using HUGEVAL may lead to spurious underflows.
      INCR_THRESH = REAL( N )*EPS

      IF ( LSAME ( UPLO, 'L' ) ) THEN
         UPLO2 = ILAUPLO( 'L' )
      ELSE
         UPLO2 = ILAUPLO( 'U' )
      ENDIF

      DO J = 1, NRHS
         Y_PREC_STATE = EXTRA_RESIDUAL
         IF ( Y_PREC_STATE .EQ. EXTRA_Y ) THEN
            DO I = 1, N
               Y_TAIL( I ) = 0.0
            END DO
         END IF

         DXRAT = 0.0
         DXRATMAX = 0.0
         DZRAT = 0.0
         DZRATMAX = 0.0
         FINAL_DX_X = HUGEVAL
         FINAL_DZ_Z = HUGEVAL
         PREVNORMDX = HUGEVAL
```

```
         PREV_DZ_Z = HUGEVAL
         DZ_Z = HUGEVAL
         DX_X = HUGEVAL

         X_STATE = WORKING_STATE
         Z_STATE = UNSTABLE_STATE
         INCR_PREC = .FALSE.

         DO CNT = 1, ITHRESH
*
*        Compute residual RES = B_s - op(A_s) * Y,
*            op(A) = A, A**T, or A**H depending on TRANS (and type).
*
            CALL SCOPY( N, B( 1, J ), 1, RES, 1 )
            IF (Y_PREC_STATE .EQ. BASE_RESIDUAL) THEN
               CALL SSYMV( UPLO, N, -1.0, A, LDA, Y(1,J), 1,
     $              1.0, RES, 1 )
            ELSE IF (Y_PREC_STATE .EQ. EXTRA_RESIDUAL) THEN
               CALL BLAS_SSYMV_X( UPLO2, N, -1.0, A, LDA,
     $              Y( 1, J ), 1, 1.0, RES, 1, PREC_TYPE )
            ELSE
               CALL BLAS_SSYMV2_X(UPLO2, N, -1.0, A, LDA,
     $              Y(1, J), Y_TAIL, 1, 1.0, RES, 1, PREC_TYPE)
            END IF

!         XXX: RES is no longer needed.
            CALL SCOPY( N, RES, 1, DY, 1 )
            CALL SSYTRS( UPLO, N, 1, AF, LDAF, IPIV, DY, N, INFO )
*
*         Calculate relative changes DX_X, DZ_Z and ratios DXRAT, DZRAT.
*
            NORMX = 0.0
            NORMY = 0.0
            NORMDX = 0.0
            DZ_Z = 0.0
            YMIN = HUGEVAL

            DO I = 1, N
               YK = ABS( Y( I, J ) )
               DYK = ABS( DY( I ) )

               IF ( YK .NE. 0.0 ) THEN
                  DZ_Z = MAX( DZ_Z, DYK / YK )
               ELSE IF ( DYK .NE. 0.0 ) THEN
                  DZ_Z = HUGEVAL
               END IF

               YMIN = MIN( YMIN, YK )

               NORMY = MAX( NORMY, YK )
```

```
               IF ( COLEQU ) THEN
                  NORMX = MAX( NORMX, YK * C( I ) )
                  NORMDX = MAX( NORMDX, DYK * C( I ) )
               ELSE
                  NORMX = NORMY
                  NORMDX = MAX(NORMDX, DYK)
               END IF
            END DO

            IF ( NORMX .NE. 0.0 ) THEN
               DX_X = NORMDX / NORMX
            ELSE IF ( NORMDX .EQ. 0.0 ) THEN
               DX_X = 0.0
            ELSE
               DX_X = HUGEVAL
            END IF

            DXRAT = NORMDX / PREVNORMDX
            DZRAT = DZ_Z / PREV_DZ_Z
*
*         Check termination criteria.
*
            IF ( YMIN*RCOND .LT. INCR_THRESH*NORMY
     $           .AND. Y_PREC_STATE .LT. EXTRA_Y )
     $           INCR_PREC = .TRUE.

            IF ( X_STATE .EQ. NOPROG_STATE .AND. DXRAT .LE. RTHRESH )
     $           X_STATE = WORKING_STATE
            IF ( X_STATE .EQ. WORKING_STATE ) THEN
               IF ( DX_X .LE. EPS ) THEN
                  X_STATE = CONV_STATE
               ELSE IF ( DXRAT .GT. RTHRESH ) THEN
                  IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                     INCR_PREC = .TRUE.
                  ELSE
                     X_STATE = NOPROG_STATE
                  END IF
               ELSE
                  IF ( DXRAT .GT. DXRATMAX ) DXRATMAX = DXRAT
               END IF
               IF ( X_STATE .GT. WORKING_STATE ) FINAL_DX_X = DX_X
            END IF

            IF ( Z_STATE .EQ. UNSTABLE_STATE .AND. DZ_Z .LE. DZ_UB )
     $           Z_STATE = WORKING_STATE
            IF ( Z_STATE .EQ. NOPROG_STATE .AND. DZRAT .LE. RTHRESH )
     $           Z_STATE = WORKING_STATE
            IF ( Z_STATE .EQ. WORKING_STATE ) THEN
               IF ( DZ_Z .LE. EPS ) THEN
```

```
                 Z_STATE = CONV_STATE
              ELSE IF ( DZ_Z .GT. DZ_UB ) THEN
                 Z_STATE = UNSTABLE_STATE
                 DZRATMAX = 0.0
                 FINAL_DZ_Z = HUGEVAL
              ELSE IF ( DZRAT .GT. RTHRESH ) THEN
                 IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                    INCR_PREC = .TRUE.
                 ELSE
                    Z_STATE = NOPROG_STATE
                 END IF
              ELSE
                 IF ( DZRAT .GT. DZRATMAX ) DZRATMAX = DZRAT
              END IF
              IF ( Z_STATE .GT. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
           END IF

           IF ( X_STATE.NE.WORKING_STATE.AND.
     $           ( IGNORE_CWISE.OR.Z_STATE.NE.WORKING_STATE ) )
     $           GOTO 666

           IF ( INCR_PREC ) THEN
              INCR_PREC = .FALSE.
              Y_PREC_STATE = Y_PREC_STATE + 1
              DO I = 1, N
                 Y_TAIL( I ) = 0.0
              END DO
           END IF

           PREVNORMDX = NORMDX
           PREV_DZ_Z = DZ_Z
*
*          Update soluton.
*
           IF (Y_PREC_STATE .LT. EXTRA_Y) THEN
              CALL SAXPY( N, 1.0, DY, 1, Y(1,J), 1 )
           ELSE
              CALL SLA_WWADDW( N, Y(1,J), Y_TAIL, DY )
           END IF

        END DO
*       Target of "IF (Z_STOP .AND. X_STOP)".  Sun's f77 won't EXIT.
 666    CONTINUE
*
*     Set final_* when cnt hits ithresh.
*
        IF ( X_STATE .EQ. WORKING_STATE ) FINAL_DX_X = DX_X
        IF ( Z_STATE .EQ. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
*
*     Compute error bounds.
```

```
*
         IF ( N_NORMS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) =
     $           FINAL_DX_X / (1 - DXRATMAX)
         END IF
         IF ( N_NORMS .GE. 2 ) THEN
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) =
     $           FINAL_DZ_Z / (1 - DZRATMAX)
         END IF
*
*     Compute componentwise relative backward error from formula
*         max(i) ( abs(R(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
*     where abs(Z) is the componentwise absolute value of the matrix
*     or vector Z.
*
*        Compute residual RES = B_s - op(A_s) * Y,
*            op(A) = A, A**T, or A**H depending on TRANS (and type).
         CALL SCOPY( N, B( 1, J ), 1, RES, 1 )
         CALL SSYMV( UPLO, N, -1.0, A, LDA, Y(1,J), 1, 1.0, RES, 1 )

         DO I = 1, N
            AYB( I ) = ABS( B( I, J ) )
         END DO
*
*     Compute abs(op(A_s))*abs(Y) + abs(B_s).
*
         CALL SLA_SYAMV( UPLO2, N, 1.0,
     $        A, LDA, Y(1, J), 1, 1.0, AYB, 1 )

         CALL SLA_LIN_BERR( N, N, 1, RES, AYB, BERR_OUT( J ) )
*
*     End of loop for each RHS.
*
      END DO
*
      RETURN
      END
```

———-

59.3.4 lisp code

— sla_syrfsx_extended.lisp —

```
(let* ((unstable_state 0)
       (working_state 1)
       (conv_state 2)
       (noprog_state 3)
```

```
      (base_residual 0)
      (extra_residual 1)
      (extra_y 2)
      (la_linrx_err_i 2))
 (declare (type (f2cl-lib:integer4 0 0) unstable_state)
          (type (f2cl-lib:integer4 1 1) working_state)
          (type (f2cl-lib:integer4 2 2) conv_state)
          (type (f2cl-lib:integer4 3 3) noprog_state)
          (type (f2cl-lib:integer4 0 0) base_residual)
          (type (f2cl-lib:integer4 1 1) extra_residual)
          (type (f2cl-lib:integer4 2 2) extra_y)
          (type (f2cl-lib:integer4 2 2) la_linrx_err_i)
          (ignorable unstable_state working_state conv_state noprog_state
           base_residual extra_residual extra_y la_linrx_err_i))
 (defun sla_syrfsx_extended
        (prec_type uplo n nrhs a lda af ldaf ipiv colequ c b ldb$ y ldy
         berr_out n_norms err_bnds_norm err_bnds_comp res ayb dy y_tail rcond
         ithresh rthresh dz_ub ignore_cwise info)
   (declare (type (single-float) dz_ub rthresh rcond)
            (type f2cl-lib:logical ignore_cwise colequ)
            (type (array f2cl-lib:integer4 (*)) ipiv)
            (type (array single-float (*)) y_tail dy ayb res err_bnds_comp
             err_bnds_norm berr_out y b c af a)
            (type (string *) uplo)
            (type (f2cl-lib:integer4) info ithresh n_norms ldy ldb$ ldaf lda
             nrhs n prec_type))
   (f2cl-lib:with-multi-array-data
       ((uplo character uplo-%data% uplo-%offset%)
        (a single-float a-%data% a-%offset%)
        (af single-float af-%data% af-%offset%)
        (c single-float c-%data% c-%offset%)
        (b single-float b-%data% b-%offset%)
        (y single-float y-%data% y-%offset%)
        (berr_out single-float berr_out-%data% berr_out-%offset%)
        (err_bnds_norm single-float err_bnds_norm-%data%
         err_bnds_norm-%offset%)
        (err_bnds_comp single-float err_bnds_comp-%data%
         err_bnds_comp-%offset%)
        (res single-float res-%data% res-%offset%)
        (ayb single-float ayb-%data% ayb-%offset%)
        (dy single-float dy-%data% dy-%offset%)
        (y_tail single-float y_tail-%data% y_tail-%offset%)
        (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
     (prog ((y_prec_state 0) (incr_prec nil) (upper nil) (yk 0.0) (dyk 0.0)
            (ymin 0.0) (normy 0.0) (normx 0.0) (normdx 0.0) (dxrat 0.0)
            (dzrat 0.0) (prevnormdx 0.0) (prev_dz_z 0.0) (dxratmax 0.0)
            (dzratmax 0.0) (dx_x 0.0) (dz_z 0.0) (final_dx_x 0.0)
            (final_dz_z 0.0) (eps 0.0) (hugeval 0.0) (incr_thresh 0.0)
            (uplo2 0) (cnt 0) (i 0) (j 0) (x_state 0) (z_state 0))
       (declare (type f2cl-lib:logical incr_prec upper)
```

```
          (type (single-float) yk dyk ymin normy normx normdx dxrat
           dzrat prevnormdx prev_dz_z dxratmax dzratmax dx_x dz_z
           final_dx_x final_dz_z eps hugeval incr_thresh)
          (type (f2cl-lib:integer4) y_prec_state uplo2 cnt i j x_state
           z_state))
(setf info 0)
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -13))
 ((< ldy (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -15)))
(cond
 ((/= info 0) (xerbla "SLA_SYRFSX_EXTENDED" (f2cl-lib:int-sub info))
  (go end_label)))
(setf eps (slamch "Epsilon"))
(setf hugeval (slamch "Overflow"))
(setf hugeval (* hugeval hugeval))
(setf incr_thresh (* (f2cl-lib:freal n) eps))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame uplo "L")
    (declare (ignore var-1))
    (when var-0 (setf uplo var-0))
    ret-val)
  (setf uplo2 (ilauplo "L")))
 (t (setf uplo2 (ilauplo "U"))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf y_prec_state extra_residual)
    (cond
```

```
 ((= y_prec_state extra_y)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                           y_tail-%offset%)
              0.0)
     label100001))))
(setf dxrat 0.0)
(setf dxratmax 0.0)
(setf dzrat 0.0)
(setf dzratmax 0.0)
(setf final_dx_x hugeval)
(setf final_dz_z hugeval)
(setf prevnormdx hugeval)
(setf prev_dz_z hugeval)
(setf dz_z hugeval)
(setf dx_x hugeval)
(setf x_state working_state)
(setf z_state unstable_state)
(setf incr_prec f2cl-lib:%false%)
(f2cl-lib:fdo (cnt 1 (f2cl-lib:int-add cnt 1))
              ((> cnt ithresh) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy n
         (f2cl-lib:array-slice b-%data% single-float (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1 res 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (cond
     ((= y_prec_state base_residual)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9)
          (ssymv uplo n -1.0 a lda
           (f2cl-lib:array-slice y-%data% single-float (1 j)
                                 ((1 ldy) (1 *)) y-%offset%)
           1 1.0 res 1)
        (declare
         (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
        (when var-0 (setf uplo var-0))
        (when var-1 (setf n var-1))
        (when var-4 (setf lda var-4))))
     ((= y_prec_state extra_residual)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (blas_ssymv_x uplo2 n -1.0 a lda
```

```
        (f2cl-lib:array-slice y-%data% single-float (1 j)
                              ((1 ldy) (1 *)) y-%offset%)
        1 1.0 res 1 prec_type)
    (declare
     (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
    (when var-0 (setf uplo2 var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf lda var-4))
    (when var-10 (setf prec_type var-10))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11)
      (blas_ssymv2_x uplo2 n -1.0 a lda
       (f2cl-lib:array-slice y-%data% single-float (1 j)
                              ((1 ldy) (1 *)) y-%offset%)
       y_tail 1 1.0 res 1 prec_type)
    (declare
     (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9 var-10))
    (when var-0 (setf uplo2 var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf lda var-4))
    (when var-11 (setf prec_type var-11)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (scopy n res 1 dy 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (ssytrs uplo n 1 af ldaf ipiv dy n info)
  (declare (ignore var-2 var-3 var-5 var-6))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-4 (setf ldaf var-4))
  (when var-7 (setf n var-7))
  (when var-8 (setf info var-8)))
(setf normx 0.0)
(setf normy 0.0)
(setf normdx 0.0)
(setf dz_z 0.0)
(setf ymin hugeval)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf yk
            (abs
             (f2cl-lib:fref y-%data% (i j) ((1 ldy) (1 *))
                            y-%offset%)))
    (setf dyk
            (abs
```

```
                (f2cl-lib:fref dy-%data% (i) ((1 *))
                               dy-%offset%)))
      (cond ((/= yk 0.0) (setf dz_z (max dz_z (/ dyk yk))))
            ((/= dyk 0.0) (setf dz_z hugeval)))
      (setf ymin (min ymin yk))
      (setf normy (max normy yk))
      (cond
       (colequ
        (setf normx
                (max normx
                     (* yk
                        (f2cl-lib:fref c-%data% (i) ((1 *))
                                       c-%offset%))))
        (setf normdx
                (max normdx
                     (* dyk
                        (f2cl-lib:fref c-%data% (i) ((1 *))
                                       c-%offset%)))))
       (t (setf normx normy) (setf normdx (max normdx dyk))))
    label100003))
(cond ((/= normx 0.0) (setf dx_x (/ normdx normx)))
      ((= normdx 0.0) (setf dx_x 0.0)) (t (setf dx_x hugeval)))
(setf dxrat (/ normdx prevnormdx))
(setf dzrat (/ dz_z prev_dz_z))
(if (and (< (* ymin rcond) (* incr_thresh normy))
         (< y_prec_state extra_y))
    (setf incr_prec f2cl-lib:%true%))
(if (and (= x_state noprog_state) (<= dxrat rthresh))
    (setf x_state working_state))
(cond
 ((= x_state working_state)
  (cond ((<= dx_x eps) (setf x_state conv_state))
        ((> dxrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf x_state noprog_state))))
        (t
         (if (> dxrat dxratmax)
             (setf dxratmax dxrat))))
  (if (> x_state working_state)
      (setf final_dx_x dx_x))))
(if (and (= z_state unstable_state) (<= dz_z dz_ub))
    (setf z_state working_state))
(if (and (= z_state noprog_state) (<= dzrat rthresh))
    (setf z_state working_state))
(cond
 ((= z_state working_state)
  (cond ((<= dz_z eps) (setf z_state conv_state))
        ((> dz_z dz_ub) (setf z_state unstable_state)
```

```
                 (setf dzratmax 0.0) (setf final_dz_z hugeval))
                ((> dzrat rthresh)
                 (cond
                  ((/= y_prec_state extra_y)
                   (setf incr_prec f2cl-lib:%true%))
                  (t (setf z_state noprog_state))))
                (t
                 (if (> dzrat dzratmax)
                     (setf dzratmax dzrat))))
          (if (> z_state working_state)
              (setf final_dz_z dz_z))))
        (if (and (/= x_state working_state)
                 (or ignore_cwise (/= z_state working_state)))
            (go label666))
        (cond
         (incr_prec (setf incr_prec f2cl-lib:%false%)
          (setf y_prec_state (f2cl-lib:int-add y_prec_state 1))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                                   y_tail-%offset%)
                    0.0)
             label100004))))
        (setf prevnormdx normdx)
        (setf prev_dz_z dz_z)
        (cond
         ((< y_prec_state extra_y)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
              (saxpy n 1.0 dy 1
               (f2cl-lib:array-slice y-%data% single-float (1 j)
                                     ((1 ldy) (1 *)) y-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3 var-4 var-5))
            (when var-0 (setf n var-0))))
         (t
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (sla_wwaddw n
               (f2cl-lib:array-slice y-%data% single-float (1 j)
                                     ((1 ldy) (1 *)) y-%offset%)
               y_tail dy)
            (declare (ignore var-1 var-2 var-3))
            (when var-0 (setf n var-0)))))
      label100002))
  label666
   (if (= x_state working_state)
       (setf final_dx_x dx_x))
   (if (= z_state working_state)
       (setf final_dz_z dz_z))
   (cond
```

```
            ((>= n_norms 1)
             (setf (f2cl-lib:fref err_bnds_norm-%data% (j la_linrx_err_i)
                                   ((1 nrhs) (1 *)) err_bnds_norm-%offset%)
                   (/ final_dx_x (- 1 dxratmax)))))
           (cond
            ((>= n_norms 2)
             (setf (f2cl-lib:fref err_bnds_comp-%data% (j la_linrx_err_i)
                                   ((1 nrhs) (1 *)) err_bnds_comp-%offset%)
                   (/ final_dz_z (- 1 dzratmax)))))
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
               (scopy n
                (f2cl-lib:array-slice b-%data% single-float (1 j)
                                      ((1 ldb$) (1 *)) b-%offset%)
                1 res 1)
             (declare (ignore var-1 var-2 var-3 var-4))
             (when var-0 (setf n var-0)))
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
               (ssymv uplo n -1.0 a lda
                (f2cl-lib:array-slice y-%data% single-float (1 j)
                                      ((1 ldy) (1 *)) y-%offset%)
                1 1.0 res 1)
             (declare (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
             (when var-0 (setf uplo var-0))
             (when var-1 (setf n var-1))
             (when var-4 (setf lda var-4)))
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref ayb-%data% (i) ((1 *)) ayb-%offset%)
                       (abs
                        (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)))
              label100005))
           (sla_syamv uplo2 n 1.0 a lda
            (f2cl-lib:array-slice y-%data% single-float (1 j) ((1 ldy) (1 *))
                                  y-%offset%)
            1 1.0 ayb 1)
           (sla_lin_berr n n 1 res ayb
            (f2cl-lib:array-slice berr_out-%data% single-float (j) ((1 *))
                                  berr_out-%offset%))
         label100000))
       (go end_label)
      end_label
       (return
        (values prec_type uplo n nil nil lda nil ldaf nil nil nil nil nil nil
                nil nil nil nil nil nil nil nil nil nil nil nil nil nil
                info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
```

```
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sla_syrfsx_extended
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              fortran-to-lisp::logical
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (single-float)
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              fortran-to-lisp::logical
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::prec_type
                                              fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldaf nil nil nil
                                              nil nil nil nil nil nil nil nil
                                              nil nil nil nil nil nil nil nil
                                              nil fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::sla_lin_berr
                                              fortran-to-lisp::sla_syamv
                                              fortran-to-lisp::ilauplo))))
```

59.4 sla_syrpvgrw

59.4.1 Axiom unit tests

— slasyrpvgrw.input —

```
)set break resume
)sys rm -f slasyrpvgrw.output
)spool slasyrpvgrw.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

59.4.2 Axiom help page

— sla_syrpvgrw.help —

```
==================================================================
sla_syrpvgrw examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       real function sla_syrpvgrw (UPLO, N, INFO, A, LDA, AF, LDAF, IPIV,
           WORK)
           SLA_SYRPVGRW computes the reciprocal pivot growth factor
           norm(A)/norm(U) for a symmetric indefinite matrix.

Function/Subroutine Documentation
   real function sla_syrpvgrw (character*1 UPLO, integer N, integer INFO,
       real, dimension( lda, * ) A, integer LDA, real, dimension( ldaf, * )
       AF, integer LDAF, integer, dimension( * ) IPIV, real, dimension( * )
       WORK)
```

```
      SLA_SYRPVGRW computes the reciprocal pivot growth factor
      norm(A)/norm(U) for a symmetric indefinite matrix.

Purpose:
   SLA_SYRPVGRW computes the reciprocal pivot growth factor
   norm(A)/norm(U). The "max absolute element" norm is used. If this is
   much less than 1, the stability of the LU factorization of the
   (equilibrated) matrix A could be poor. This also means that the
   solution X, estimated condition numbers, and error bounds could be
   unreliable.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 INFO
   INFO is INTEGER
   The value of INFO returned from SSYTRF, .i.e., the pivot in
   column INFO is exactly 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is REAL array, dimension (LDAF,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by SSYTRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by SSYTRF.

 WORK
```

```
WORK is REAL array, dimension (2*N)
```

———-

59.4.3 fortran code

— **sla_syrpvgrw.f** —

```
*  =====================================================================
      REAL FUNCTION SLA_SYRPVGRW( UPLO, N, INFO, A, LDA, AF, LDAF, IPIV,
     $                            WORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER*1        UPLO
      INTEGER            N, INFO, LDA, LDAF
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), AF( LDAF, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            NCOLS, I, J, K, KP
      REAL               AMAX, UMAX, RPVGRW, TMP
      LOGICAL            UPPER
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, SLASET
      LOGICAL            LSAME
*     ..
*     .. Executable Statements ..
*
      UPPER = LSAME( 'Upper', UPLO )
      IF ( INFO.EQ.0 ) THEN
         IF ( UPPER ) THEN
            NCOLS = 1
         ELSE
```

```
            NCOLS = N
         END IF
      ELSE
         NCOLS = INFO
      END IF

      RPVGRW = 1.0
      DO I = 1, 2*N
         WORK( I ) = 0.0
      END DO
*
*     Find the max magnitude entry of each column of A.  Compute the max
*     for all N columns so we can apply the pivot permutation while
*     looping below.  Assume a full factorization is the common case.
*
      IF ( UPPER ) THEN
         DO J = 1, N
            DO I = 1, J
               WORK( N+I ) = MAX( ABS( A( I, J ) ), WORK( N+I ) )
               WORK( N+J ) = MAX( ABS( A( I, J ) ), WORK( N+J ) )
            END DO
         END DO
      ELSE
         DO J = 1, N
            DO I = J, N
               WORK( N+I ) = MAX( ABS( A( I, J ) ), WORK( N+I ) )
               WORK( N+J ) = MAX( ABS( A( I, J ) ), WORK( N+J ) )
            END DO
         END DO
      END IF
*
*     Now find the max magnitude entry of each column of U or L.  Also
*     permute the magnitudes of A above so they're in the same order as
*     the factor.
*
*     The iteration orders and permutations were copied from ssytrs.
*     Calls to SSWAP would be severe overkill.
*
      IF ( UPPER ) THEN
         K = N
         DO WHILE ( K .LT. NCOLS .AND. K.GT.0 )
            IF ( IPIV( K ).GT.0 ) THEN
!              1x1 pivot
               KP = IPIV( K )
               IF ( KP .NE. K ) THEN
                  TMP = WORK( N+K )
                  WORK( N+K ) = WORK( N+KP )
                  WORK( N+KP ) = TMP
               END IF
               DO I = 1, K
```

```
                 WORK( K ) = MAX( ABS( AF( I, K ) ), WORK( K ) )
              END DO
              K = K - 1
           ELSE
!             2x2 pivot
              KP = -IPIV( K )
              TMP = WORK( N+K-1 )
              WORK( N+K-1 ) = WORK( N+KP )
              WORK( N+KP ) = TMP
              DO I = 1, K-1
                 WORK( K ) = MAX( ABS( AF( I, K ) ), WORK( K ) )
                 WORK( K-1 ) = MAX( ABS( AF( I, K-1 ) ), WORK( K-1 ) )
              END DO
              WORK( K ) = MAX( ABS( AF( K, K ) ), WORK( K ) )
              K = K - 2
           END IF
        END DO
        K = NCOLS
        DO WHILE ( K .LE. N )
           IF ( IPIV( K ).GT.0 ) THEN
              KP = IPIV( K )
              IF ( KP .NE. K ) THEN
                 TMP = WORK( N+K )
                 WORK( N+K ) = WORK( N+KP )
                 WORK( N+KP ) = TMP
              END IF
              K = K + 1
           ELSE
              KP = -IPIV( K )
              TMP = WORK( N+K )
              WORK( N+K ) = WORK( N+KP )
              WORK( N+KP ) = TMP
              K = K + 2
           END IF
        END DO
     ELSE
        K = 1
        DO WHILE ( K .LE. NCOLS )
           IF ( IPIV( K ).GT.0 ) THEN
!             1x1 pivot
              KP = IPIV( K )
              IF ( KP .NE. K ) THEN
                 TMP = WORK( N+K )
                 WORK( N+K ) = WORK( N+KP )
                 WORK( N+KP ) = TMP
              END IF
              DO I = K, N
                 WORK( K ) = MAX( ABS( AF( I, K ) ), WORK( K ) )
              END DO
              K = K + 1
```

```
            ELSE
!              2x2 pivot
               KP = -IPIV( K )
               TMP = WORK( N+K+1 )
               WORK( N+K+1 ) = WORK( N+KP )
               WORK( N+KP ) = TMP
               DO I = K+1, N
                  WORK( K ) = MAX( ABS( AF( I, K ) ), WORK( K ) )
                  WORK( K+1 ) = MAX( ABS( AF(I, K+1 ) ), WORK( K+1 ) )
               END DO
               WORK( K ) = MAX( ABS( AF( K, K ) ), WORK( K ) )
               K = K + 2
            END IF
         END DO
         K = NCOLS
         DO WHILE ( K .GE. 1 )
            IF ( IPIV( K ).GT.0 ) THEN
               KP = IPIV( K )
               IF ( KP .NE. K ) THEN
                  TMP = WORK( N+K )
                  WORK( N+K ) = WORK( N+KP )
                  WORK( N+KP ) = TMP
               END IF
               K = K - 1
            ELSE
               KP = -IPIV( K )
               TMP = WORK( N+K )
               WORK( N+K ) = WORK( N+KP )
               WORK( N+KP ) = TMP
               K = K - 2
            ENDIF
         END DO
      END IF
*
*     Compute the *inverse* of the max element growth factor.  Dividing
*     by zero would imply the largest entry of the factor's column is
*     zero.  Than can happen when either the column of A is zero or
*     massive pivots made the factor underflow to zero.  Neither counts
*     as growth in itself, so simply ignore terms with zero
*     denominators.
*
      IF ( UPPER ) THEN
         DO I = NCOLS, N
            UMAX = WORK( I )
            AMAX = WORK( N+I )
            IF ( UMAX /= 0.0 ) THEN
               RPVGRW = MIN( AMAX / UMAX, RPVGRW )
            END IF
         END DO
      ELSE
```

```
         DO I = 1, NCOLS
            UMAX = WORK( I )
            AMAX = WORK( N+I )
            IF ( UMAX /= 0.0 ) THEN
               RPVGRW = MIN( AMAX / UMAX, RPVGRW )
            END IF
         END DO
      END IF

      SLA_SYRPVGRW = RPVGRW
      END
```

———-

59.4.4 lisp code

— sla_syrpvgrw.lisp —

```
(defun sla_syrpvgrw (uplo n info a lda af ldaf ipiv work)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array single-float (*)) work af a)
           (type (f2cl-lib:integer4) ldaf lda info n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a single-float a-%data% a-%offset%)
       (af single-float af-%data% af-%offset%)
       (work single-float work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((upper nil) (amax 0.0) (umax 0.0) (rpvgrw 0.0) (tmp 0.0) (ncols 0)
           (i 0) (j 0) (k 0) (kp 0) (sla_syrpvgrw 0.0))
      (declare (type (f2cl-lib:integer4) kp k j i ncols)
               (type (single-float) sla_syrpvgrw tmp rpvgrw umax amax)
               (type f2cl-lib:logical upper))
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame "Upper" uplo)
                (declare (ignore var-0))
                (when var-1 (setf uplo var-1))
                ret-val))
      (cond ((= info 0) (cond (upper (setf ncols 1)) (t (setf ncols n))))
            (t (setf ncols info)))
      (setf rpvgrw 1.0)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i (f2cl-lib:int-mul 2 n)) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) 0.0)
         label100000))
```

```
(cond
 (upper
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i j) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                ((1 *)) work-%offset%)
                  (max
                   (abs
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))
                   (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                  ((1 *)) work-%offset%)))
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j))
                                ((1 *)) work-%offset%)
                  (max
                   (abs
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))
                   (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j))
                                  ((1 *)) work-%offset%)))
        label100002))
    label100001)))
 (t
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                ((1 *)) work-%offset%)
                  (max
                   (abs
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))
                   (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                  ((1 *)) work-%offset%)))
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j))
                                ((1 *)) work-%offset%)
                  (max
                   (abs
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))
                   (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j))
                                  ((1 *)) work-%offset%)))
        label100004))
```

```
    label100003))))
(cond
 (upper
  (tagbody
    (setf k n)
   label100005
    (if (not (and (< k ncols) (> k 0)))
        (go label100006))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (cond
       ((/= kp k)
        (setf tmp
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                               ((1 *)) work-%offset%))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                             work-%offset%)
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                               ((1 *)) work-%offset%))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                             ((1 *)) work-%offset%)
                tmp)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i k) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                  (max
                   (abs
                    (f2cl-lib:fref af-%data% (i k) ((1 ldaf) (1 *))
                                   af-%offset%))
                   (f2cl-lib:fref work-%data% (k) ((1 *))
                                  work-%offset%)))
         label100007))
      (setf k (f2cl-lib:int-sub k 1)))
     (t
      (setf kp
              (f2cl-lib:int-sub
               (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
      (setf tmp
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-sub (f2cl-lib:int-add n k)
                                                1))
                             ((1 *)) work-%offset%))
      (setf (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-sub (f2cl-lib:int-add n k) 1))
                           ((1 *)) work-%offset%)
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                             ((1 *)) work-%offset%))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp)) ((1 *))
```

```
                        work-%offset%)
            tmp)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                (max
                 (abs
                  (f2cl-lib:fref af-%data% (i k) ((1 ldaf) (1 *))
                                 af-%offset%))
                 (f2cl-lib:fref work-%data% (k) ((1 *))
                                work-%offset%)))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1))
                             ((1 *)) work-%offset%)
                (max
                 (abs
                  (f2cl-lib:fref af-%data% (i (f2cl-lib:int-sub k 1))
                                 ((1 ldaf) (1 *)) af-%offset%))
                 (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1))
                                ((1 *)) work-%offset%)))
       label100008))
    (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
            (max
             (abs
              (f2cl-lib:fref af-%data% (k k) ((1 ldaf) (1 *))
                             af-%offset%))
             (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)))
    (setf k (f2cl-lib:int-sub k 2))))
  (go label100005)
 label100006
  (setf k ncols)
 label100009
  (if (not (<= k n))
      (go label100010))
  (cond
   ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
    (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
    (cond
     ((/= kp k)
      (setf tmp
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                             ((1 *)) work-%offset%))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                           work-%offset%)
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                             ((1 *)) work-%offset%))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                           ((1 *)) work-%offset%)
              tmp)))
    (setf k (f2cl-lib:int-add k 1)))
```

```
    (t
     (setf kp
             (f2cl-lib:int-sub
              (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
     (setf tmp
             (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                            work-%offset%))
     (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                          work-%offset%)
             (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                            ((1 *)) work-%offset%))
     (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp)) ((1 *))
                          work-%offset%)
             tmp)
     (setf k (f2cl-lib:int-add k 2))))
   (go label100009)
  label100010))
(t
 (tagbody
   (setf k 1)
  label100011
   (if (not (<= k ncols))
       (go label100012))
   (cond
    ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
     (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
     (cond
      ((/= kp k)
       (setf tmp
               (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                              ((1 *)) work-%offset%))
       (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                            work-%offset%)
               (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                              ((1 *)) work-%offset%))
       (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                            ((1 *)) work-%offset%)
               tmp)))
     (f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                 (max
                  (abs
                   (f2cl-lib:fref af-%data% (i k) ((1 ldaf) (1 *))
                                  af-%offset%))
                  (f2cl-lib:fref work-%data% (k) ((1 *))
                                 work-%offset%)))
        label100013))
     (setf k (f2cl-lib:int-add k 1)))
```

```
   (t
    (setf kp
            (f2cl-lib:int-sub
             (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
    (setf tmp
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k 1))
                           ((1 *)) work-%offset%))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k 1)) ((1 *))
                         work-%offset%)
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                           ((1 *)) work-%offset%))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp)) ((1 *))
                         work-%offset%)
            tmp)
    (f2cl-lib:fdo (i (f2cl-lib:int-add k 1) (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                (max
                 (abs
                  (f2cl-lib:fref af-%data% (i k) ((1 ldaf) (1 *))
                                 af-%offset%))
                 (f2cl-lib:fref work-%data% (k) ((1 *))
                                work-%offset%)))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1))
                             ((1 *)) work-%offset%)
                (max
                 (abs
                  (f2cl-lib:fref af-%data% (i (f2cl-lib:int-add k 1))
                                 ((1 ldaf) (1 *)) af-%offset%))
                 (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1))
                                ((1 *)) work-%offset%)))
       label100014))
    (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
            (max
             (abs
              (f2cl-lib:fref af-%data% (k k) ((1 ldaf) (1 *))
                             af-%offset%))
             (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)))
    (setf k (f2cl-lib:int-add k 2))))
  (go label100011)
 label100012
  (setf k ncols)
 label100015
  (if (not (>= k 1))
      (go label100016))
  (cond
   ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
    (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
    (cond
```

```
        ((/= kp k)
         (setf tmp
                 (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                                ((1 *)) work-%offset%))
         (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                              work-%offset%)
                 (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                                ((1 *)) work-%offset%))
         (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                              ((1 *)) work-%offset%)
                 tmp)))
       (setf k (f2cl-lib:int-sub k 1)))
      (t
       (setf kp
               (f2cl-lib:int-sub
                (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
       (setf tmp
               (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                              work-%offset%))
       (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                            work-%offset%)
               (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                              ((1 *)) work-%offset%))
       (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp)) ((1 *))
                            work-%offset%)
               tmp)
       (setf k (f2cl-lib:int-sub k 2))))
     (go label100015)
    label100016)))
  (cond
   (upper
    (f2cl-lib:fdo (i ncols (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf umax (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
        (setf amax
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i)) ((1 *))
                               work-%offset%))
        (cond
         ((f2cl-lib:f2cl/ umax (= 0.0))
          (setf rpvgrw (min (/ amax umax) rpvgrw))))
      label100017)))
   (t
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i ncols) nil)
      (tagbody
        (setf umax (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
        (setf amax
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i)) ((1 *))
                               work-%offset%))
```

```
            (cond
             ((f2cl-lib:f2cl/ umax (= 0.0))
              (setf rpvgrw (min (/ amax umax) rpvgrw))))
           label100018))))
      (setf sla_syrpvgrw rpvgrw)
     end_label
      (return (values sla_syrpvgrw uplo nil nil nil nil nil nil nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sla_syrpvgrw
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*)))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil nil
                                              nil nil nil nil nil)
                                            :calls 'nil)))
```

———-

59.5 slasyf

59.5.1 Axiom unit tests

— slasyf.input —

```
)set break resume
)sys rm -f slasyf.output
)spool slasyf.output
)set message test on
)set message auto off
)clear all

)spool
```

```
)lisp (bye)
```

———-

59.5.2 Axiom help page

— **slasyf.help** —

```
======================================================================
slasyf examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slasyf (UPLO, N, NB, KB, A, LDA, IPIV, W, LDW, INFO)
           SLASYF computes a partial factorization of a real symmetric matrix
           using the Bunch-Kaufman diagonal pivoting method.

Function/Subroutine Documentation
   subroutine slasyf (character UPLO, integer N, integer NB, integer KB, real,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV, real,
       dimension( ldw, * ) W, integer LDW, integer INFO)
       SLASYF computes a partial factorization of a real symmetric matrix
       using the Bunch-Kaufman diagonal pivoting method.

Purpose:
   SLASYF computes a partial factorization of a real symmetric matrix A
   using the Bunch-Kaufman diagonal pivoting method. The partial
   factorization has the form:

       A  =  ( I  U12 ) ( A11  0  ) (  I       0    )  if UPLO = 'U', or:
             ( 0  U22 ) (  0   D  ) ( U12**T U22**T )

       A  =  ( L11  0 ) (  D   0  ) ( L11**T L21**T )  if UPLO = 'L'
             ( L21  I ) (  0  A22 ) (  0       I    )

   where the order of D is at most NB. The actual order is returned in
   the argument KB, and is either NB or NB-1, or N if N <= NB.

   SLASYF is an auxiliary routine called by SSYTRF. It uses blocked code
   (calling Level 3 BLAS) to update the submatrix A11 (if UPLO = 'U') or
   A22 (if UPLO = 'L').

Parameters:
```

```
UPLO
  UPLO is CHARACTER*1
  Specifies whether the upper or lower triangular part of the
  symmetric matrix A is stored:
  = 'U':  Upper triangular
  = 'L':  Lower triangular

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

NB
  NB is INTEGER
  The maximum number of columns of the matrix A that should be
  factored.  NB should be at least 2 to allow for 2-by-2 pivot blocks.

KB
  KB is INTEGER
  The number of columns of A that were actually factored.
  KB is either NB-1 or NB, or N if N <= NB.

A
  A is REAL array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  n-by-n upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading n-by-n lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.
  On exit, A contains details of the partial factorization.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D.

  If UPLO = 'U':
     Only the last KB elements of IPIV are set.

     If IPIV(k) > 0, then rows and columns k and IPIV(k) were
     interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) = IPIV(k-1) < 0, then rows and columns
     k-1 and -IPIV(k) were interchanged and D(k-1:k,k-1:k)
     is a 2-by-2 diagonal block.
```

```
  If UPLO = 'L':
     Only the first KB elements of IPIV are set.

     If IPIV(k) > 0, then rows and columns k and IPIV(k) were
     interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) = IPIV(k+1) < 0, then rows and columns
     k+1 and -IPIV(k) were interchanged and D(k:k+1,k:k+1)
     is a 2-by-2 diagonal block.

W
  W is REAL array, dimension (LDW,NB)

LDW
  LDW is INTEGER
  The leading dimension of the array W.  LDW >= max(1,N).

INFO
  INFO is INTEGER
  = 0: successful exit
  > 0: if INFO = k, D(k,k) is exactly zero.  The factorization
       has been completed, but the block diagonal matrix D is
       exactly singular.
```

———-

59.5.3 fortran code

— **slasyf.f** —

```
*  =====================================================================
      SUBROUTINE SLASYF( UPLO, N, NB, KB, A, LDA, IPIV, W, LDW, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, KB, LDA, LDW, N, NB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), W( LDW, * )
*     ..
```

```
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               EIGHT, SEVTEN
      PARAMETER          ( EIGHT = 8.0E+0, SEVTEN = 17.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            IMAX, J, JB, JJ, JMAX, JP, K, KK, KKW, KP,
     $                   KSTEP, KW
      REAL               ABSAKK, ALPHA, COLMAX, D11, D21, D22, R1,
     $                   ROWMAX, T
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ISAMAX
      EXTERNAL           LSAME, ISAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SGEMM, SGEMV, SSCAL, SSWAP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
*     Initialize ALPHA for use in choosing pivot block size.
*
      ALPHA = ( ONE+SQRT( SEVTEN ) ) / EIGHT
*
      IF( LSAME( UPLO, 'U' ) ) THEN
*
*        Factorize the trailing columns of A using the upper triangle
*        of A and working backwards, and compute the matrix W = U12*D
*        for use in updating A11
*
*        K is the main loop index, decreasing from N in steps of 1 or 2
*
*        KW is the column of W which corresponds to column K of A
*
         K = N
   10    CONTINUE
         KW = NB + K - N
*
*        Exit from loop
*
```

```
         IF( ( K.LE.N-NB+1 .AND. NB.LT.N ) .OR. K.LT.1 )
     $      GO TO 30
*
*        Copy column K of A to column KW of W and update it
*
         CALL SCOPY( K, A( 1, K ), 1, W( 1, KW ), 1 )
         IF( K.LT.N )
     $      CALL SGEMV( 'No transpose', K, N-K, -ONE, A( 1, K+1 ), LDA,
     $                  W( K, KW+1 ), LDW, ONE, W( 1, KW ), 1 )
*
         KSTEP = 1
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = ABS( W( K, KW ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.GT.1 ) THEN
            IMAX = ISAMAX( K-1, W( 1, KW ), 1 )
            COLMAX = ABS( W( IMAX, KW ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( MAX( ABSAKK, COLMAX ).EQ.ZERO ) THEN
*
*           Column K is zero or underflow: set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
         ELSE
            IF( ABSAKK.GE.ALPHA*COLMAX ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
            ELSE
*
*              Copy column IMAX to column KW-1 of W and update it
*
               CALL SCOPY( IMAX, A( 1, IMAX ), 1, W( 1, KW-1 ), 1 )
               CALL SCOPY( K-IMAX, A( IMAX, IMAX+1 ), LDA,
     $                     W( IMAX+1, KW-1 ), 1 )
               IF( K.LT.N )
     $            CALL SGEMV( 'No transpose', K, N-K, -ONE, A( 1, K+1 ),
```

```
     $                       LDA, W( IMAX, KW+1 ), LDW, ONE,
     $                       W( 1, KW-1 ), 1 )
*
*             JMAX is the column-index of the largest off-diagonal
*             element in row IMAX, and ROWMAX is its absolute value
*
              JMAX = IMAX + ISAMAX( K-IMAX, W( IMAX+1, KW-1 ), 1 )
              ROWMAX = ABS( W( JMAX, KW-1 ) )
              IF( IMAX.GT.1 ) THEN
                 JMAX = ISAMAX( IMAX-1, W( 1, KW-1 ), 1 )
                 ROWMAX = MAX( ROWMAX, ABS( W( JMAX, KW-1 ) ) )
              END IF
*
              IF( ABSAKK.GE.ALPHA*COLMAX*( COLMAX / ROWMAX ) ) THEN
*
*                no interchange, use 1-by-1 pivot block
*
                 KP = K
              ELSE IF( ABS( W( IMAX, KW-1 ) ).GE.ALPHA*ROWMAX ) THEN
*
*                interchange rows and columns K and IMAX, use 1-by-1
*                pivot block
*
                 KP = IMAX
*
*                copy column KW-1 of W to column KW of W
*
                 CALL SCOPY( K, W( 1, KW-1 ), 1, W( 1, KW ), 1 )
              ELSE
*
*                interchange rows and columns K-1 and IMAX, use 2-by-2
*                pivot block
*
                 KP = IMAX
                 KSTEP = 2
              END IF
           END IF
*
*          ============================================================
*
*          KK is the column of A where pivoting step stopped
*
           KK = K - KSTEP + 1
*
*          KKW is the column of W which corresponds to column KK of A
*
           KKW = NB + KK - N
*
*          Interchange rows and columns KP and KK.
*          Updated column KP is already stored in column KKW of W.
```

```
*
            IF( KP.NE.KK ) THEN
*
*              Copy non-updated column KK to column KP of submatrix A
*              at step K. No need to copy element into column K
*              (or K and K-1 for 2-by-2 pivot) of A, since these columns
*              will be later overwritten.
*
               A( KP, KP ) = A( KK, KK )
               CALL SCOPY( KK-1-KP, A( KP+1, KK ), 1, A( KP, KP+1 ),
     $                     LDA )
               IF( KP.GT.1 )
     $            CALL SCOPY( KP-1, A( 1, KK ), 1, A( 1, KP ), 1 )
*
*              Interchange rows KK and KP in last K+1 to N columns of A
*              (columns K (or K and K-1 for 2-by-2 pivot) of A will be
*              later overwritten). Interchange rows KK and KP
*              in last KKW to NB columns of W.
*
               IF( K.LT.N )
     $            CALL SSWAP( N-K, A( KK, K+1 ), LDA, A( KP, K+1 ),
     $                        LDA )
               CALL SSWAP( N-KK+1, W( KK, KKW ), LDW, W( KP, KKW ),
     $                     LDW )
            END IF
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column kw of W now holds
*
*              W(kw) = U(k)*D(k),
*
*              where U(k) is the k-th column of U
*
*              Store subdiag. elements of column U(k)
*              and 1-by-1 block D(k) in column k of A.
*              NOTE: Diagonal element U(k,k) is a UNIT element
*              and not stored.
*                 A(k,k) := D(k,k) = W(k,kw)
*                 A(1:k-1,k) := U(1:k-1,k) = W(1:k-1,kw)/D(k,k)
*
               CALL SCOPY( K, W( 1, KW ), 1, A( 1, K ), 1 )
               R1 = ONE / A( K, K )
               CALL SSCAL( K-1, R1, A( 1, K ), 1 )
*
            ELSE
*
*              2-by-2 pivot block D(k): columns kw and kw-1 of W now hold
*
*              ( W(kw-1) W(kw) ) = ( U(k-1) U(k) )*D(k)
```

```
*
*              where U(k) and U(k-1) are the k-th and (k-1)-th columns
*              of U
*
*              Store U(1:k-2,k-1) and U(1:k-2,k) and 2-by-2
*              block D(k-1:k,k-1:k) in columns k-1 and k of A.
*              NOTE: 2-by-2 diagonal block U(k-1:k,k-1:k) is a UNIT
*              block and not stored.
*                 A(k-1:k,k-1:k) := D(k-1:k,k-1:k) = W(k-1:k,kw-1:kw)
*                 A(1:k-2,k-1:k) := U(1:k-2,k:k-1:k) =
*                 = W(1:k-2,kw-1:kw) * ( D(k-1:k,k-1:k)**(-1) )
*
               IF( K.GT.2 ) THEN
*
*                 Compose the columns of the inverse of 2-by-2 pivot
*                 block D in the following way to reduce the number
*                 of FLOPS when we myltiply panel ( W(kw-1) W(kw) ) by
*                 this inverse
*
*                 D**(-1) = ( d11 d21 )**(-1) =
*                           ( d21 d22 )
*
*                 = 1/(d11*d22-d21**2) * ( ( d22 ) (-d21 ) ) =
*                                        ( (-d21 ) ( d11 ) )
*
*                 = 1/d21 * 1/((d11/d21)*(d22/d21)-1) *
*
*                   * ( ( d22/d21 ) (      -1 ) ) =
*                     ( (      -1 ) ( d11/d21 ) )
*
*                 = 1/d21 * 1/(D22*D11-1) * ( ( D11 ) (  -1 ) ) =
*                                           ( ( -1  ) ( D22 ) )
*
*                 = 1/d21 * T * ( ( D11 ) (  -1 ) )
*                               ( (  -1 ) ( D22 ) )
*
*                 = D21 * ( ( D11 ) (  -1 ) )
*                         ( (  -1 ) ( D22 ) )
*
                  D21 = W( K-1, KW )
                  D11 = W( K, KW ) / D21
                  D22 = W( K-1, KW-1 ) / D21
                  T = ONE / ( D11*D22-ONE )
                  D21 = T / D21
*
*                 Update elements in columns A(k-1) and A(k) as
*                 dot products of rows of ( W(kw-1) W(kw) ) and columns
*                 of D**(-1)
*
                  DO 20 J = 1, K - 2
```

```
                  A( J, K-1 ) = D21*( D11*W( J, KW-1 )-W( J, KW ) )
                  A( J, K ) = D21*( D22*W( J, KW )-W( J, KW-1 ) )
   20            CONTINUE
              END IF
*
*             Copy D(k) to A
*
              A( K-1, K-1 ) = W( K-1, KW-1 )
              A( K-1, K ) = W( K-1, KW )
              A( K, K ) = W( K, KW )
*
           END IF
*
        END IF
*
*       Store details of the interchanges in IPIV
*
        IF( KSTEP.EQ.1 ) THEN
           IPIV( K ) = KP
        ELSE
           IPIV( K ) = -KP
           IPIV( K-1 ) = -KP
        END IF
*
*       Decrease K and return to the start of the main loop
*
        K = K - KSTEP
        GO TO 10
*
   30   CONTINUE
*
*       Update the upper triangle of A11 (= A(1:k,1:k)) as
*
*       A11 := A11 - U12*D*U12**T = A11 - U12*W**T
*
*       computing blocks of NB columns at a time
*
        DO 50 J = ( ( K-1 ) / NB )*NB + 1, 1, -NB
           JB = MIN( NB, K-J+1 )
*
*          Update the upper triangle of the diagonal block
*
           DO 40 JJ = J, J + JB - 1
              CALL SGEMV( 'No transpose', JJ-J+1, N-K, -ONE,
     $                    A( J, K+1 ), LDA, W( JJ, KW+1 ), LDW, ONE,
     $                    A( J, JJ ), 1 )
   40      CONTINUE
*
*          Update the rectangular superdiagonal block
*
```

```
            CALL SGEMM( 'No transpose', 'Transpose', J-1, JB, N-K, -ONE,
     $                  A( 1, K+1 ), LDA, W( J, KW+1 ), LDW, ONE,
     $                  A( 1, J ), LDA )
   50    CONTINUE
*
*        Put U12 in standard form by partially undoing the interchanges
*        in columns k+1:n looping backwards from k+1 to n
*
         J = K + 1
   60    CONTINUE
*
*           Undo the interchanges (if any) of rows JJ and JP at each
*           step J
*
*           (Here, J is a diagonal index)
            JJ = J
            JP = IPIV( J )
            IF( JP.LT.0 ) THEN
               JP = -JP
*              (Here, J is a diagonal index)
               J = J + 1
            END IF
*           (NOTE: Here, J is used to determine row length. Length N-J+1
*           of the rows to swap back doesn't include diagonal element)
            J = J + 1
            IF( JP.NE.JJ .AND. J.LE.N )
     $         CALL SSWAP( N-J+1, A( JP, J ), LDA, A( JJ, J ), LDA )
         IF( J.LT.N )
     $      GO TO 60
*
*        Set KB to the number of columns factorized
*
         KB = N - K
*
      ELSE
*
*        Factorize the leading columns of A using the lower triangle
*        of A and working forwards, and compute the matrix W = L21*D
*        for use in updating A22
*
*        K is the main loop index, increasing from 1 in steps of 1 or 2
*
         K = 1
   70    CONTINUE
*
*        Exit from loop
*
         IF( ( K.GE.NB .AND. NB.LT.N ) .OR. K.GT.N )
     $      GO TO 90
*
```

```
*        Copy column K of A to column K of W and update it
*
         CALL SCOPY( N-K+1, A( K, K ), 1, W( K, K ), 1 )
         CALL SGEMV( 'No transpose', N-K+1, K-1, -ONE, A( K, 1 ), LDA,
     $               W( K, 1 ), LDW, ONE, W( K, K ), 1 )
*
         KSTEP = 1
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = ABS( W( K, K ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.LT.N ) THEN
            IMAX = K + ISAMAX( N-K, W( K+1, K ), 1 )
            COLMAX = ABS( W( IMAX, K ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( MAX( ABSAKK, COLMAX ).EQ.ZERO ) THEN
*
*           Column K is zero or underflow: set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
         ELSE
            IF( ABSAKK.GE.ALPHA*COLMAX ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
            ELSE
*
*              Copy column IMAX to column K+1 of W and update it
*
               CALL SCOPY( IMAX-K, A( IMAX, K ), LDA, W( K, K+1 ), 1 )
               CALL SCOPY( N-IMAX+1, A( IMAX, IMAX ), 1, W( IMAX, K+1 ),
     $                     1 )
               CALL SGEMV( 'No transpose', N-K+1, K-1, -ONE, A( K, 1 ),
     $                     LDA, W( IMAX, 1 ), LDW, ONE, W( K, K+1 ), 1 )
*
*              JMAX is the column-index of the largest off-diagonal
*              element in row IMAX, and ROWMAX is its absolute value
*
```

```
               JMAX = K - 1 + ISAMAX( IMAX-K, W( K, K+1 ), 1 )
               ROWMAX = ABS( W( JMAX, K+1 ) )
               IF( IMAX.LT.N ) THEN
                  JMAX = IMAX + ISAMAX( N-IMAX, W( IMAX+1, K+1 ), 1 )
                  ROWMAX = MAX( ROWMAX, ABS( W( JMAX, K+1 ) ) )
               END IF
*
               IF( ABSAKK.GE.ALPHA*COLMAX*( COLMAX / ROWMAX ) ) THEN
*
*                 no interchange, use 1-by-1 pivot block
*
                  KP = K
               ELSE IF( ABS( W( IMAX, K+1 ) ).GE.ALPHA*ROWMAX ) THEN
*
*                 interchange rows and columns K and IMAX, use 1-by-1
*                 pivot block
*
                  KP = IMAX
*
*                 copy column K+1 of W to column K of W
*
                  CALL SCOPY( N-K+1, W( K, K+1 ), 1, W( K, K ), 1 )
               ELSE
*
*                 interchange rows and columns K+1 and IMAX, use 2-by-2
*                 pivot block
*
                  KP = IMAX
                  KSTEP = 2
               END IF
            END IF
*
*           ============================================================
*
*           KK is the column of A where pivoting step stopped
*
            KK = K + KSTEP - 1
*
*           Interchange rows and columns KP and KK.
*           Updated column KP is already stored in column KK of W.
*
            IF( KP.NE.KK ) THEN
*
*              Copy non-updated column KK to column KP of submatrix A
*              at step K. No need to copy element into column K
*              (or K and K+1 for 2-by-2 pivot) of A, since these columns
*              will be later overwritten.
*
               A( KP, KP ) = A( KK, KK )
               CALL SCOPY( KP-KK-1, A( KK+1, KK ), 1, A( KP, KK+1 ),
```

```
     $                    LDA )
                IF( KP.LT.N )
     $             CALL SCOPY( N-KP, A( KP+1, KK ), 1, A( KP+1, KP ), 1 )
*
*               Interchange rows KK and KP in first K-1 columns of A
*               (columns K (or K and K+1 for 2-by-2 pivot) of A will be
*               later overwritten). Interchange rows KK and KP
*               in first KK columns of W.
*
                IF( K.GT.1 )
     $             CALL SSWAP( K-1, A( KK, 1 ), LDA, A( KP, 1 ), LDA )
                CALL SSWAP( KK, W( KK, 1 ), LDW, W( KP, 1 ), LDW )
             END IF
*
             IF( KSTEP.EQ.1 ) THEN
*
*               1-by-1 pivot block D(k): column k of W now holds
*
*               W(k) = L(k)*D(k),
*
*               where L(k) is the k-th column of L
*
*               Store subdiag. elements of column L(k)
*               and 1-by-1 block D(k) in column k of A.
*               (NOTE: Diagonal element L(k,k) is a UNIT element
*               and not stored)
*                  A(k,k) := D(k,k) = W(k,k)
*                  A(k+1:N,k) := L(k+1:N,k) = W(k+1:N,k)/D(k,k)
*
                CALL SCOPY( N-K+1, W( K, K ), 1, A( K, K ), 1 )
                IF( K.LT.N ) THEN
                   R1 = ONE / A( K, K )
                   CALL SSCAL( N-K, R1, A( K+1, K ), 1 )
                END IF
*
             ELSE
*
*               2-by-2 pivot block D(k): columns k and k+1 of W now hold
*
*               ( W(k) W(k+1) ) = ( L(k) L(k+1) )*D(k)
*
*               where L(k) and L(k+1) are the k-th and (k+1)-th columns
*               of L
*
*               Store L(k+2:N,k) and L(k+2:N,k+1) and 2-by-2
*               block D(k:k+1,k:k+1) in columns k and k+1 of A.
*               (NOTE: 2-by-2 diagonal block L(k:k+1,k:k+1) is a UNIT
*               block and not stored)
*                  A(k:k+1,k:k+1) := D(k:k+1,k:k+1) = W(k:k+1,k:k+1)
*                  A(k+2:N,k:k+1) := L(k+2:N,k:k+1) =
```

```
*                 = W(k+2:N,k:k+1) * ( D(k:k+1,k:k+1)**(-1) )
*
               IF( K.LT.N-1 ) THEN
*
*                 Compose the columns of the inverse of 2-by-2 pivot
*                 block D in the following way to reduce the number
*                 of FLOPS when we myltiply panel ( W(k) W(k+1) ) by
*                 this inverse
*
*                 D**(-1) = ( d11 d21 )**(-1) =
*                           ( d21 d22 )
*
*                 = 1/(d11*d22-d21**2) * ( ( d22 ) (-d21 ) ) =
*                                        ( (-d21 ) ( d11 ) )
*
*                 = 1/d21 * 1/((d11/d21)*(d22/d21)-1) *
*
*                   * ( ( d22/d21 ) (      -1 ) ) =
*                     ( (      -1 ) ( d11/d21 ) )
*
*                 = 1/d21 * 1/(D22*D11-1) * ( ( D11 ) (  -1 ) ) =
*                                           ( ( -1  ) ( D22 ) )
*
*                 = 1/d21 * T * ( ( D11 ) (  -1 ) )
*                               ( (  -1 ) ( D22 ) )
*
*                 = D21 * ( ( D11 ) (  -1 ) )
*                         ( (  -1 ) ( D22 ) )
*
                  D21 = W( K+1, K )
                  D11 = W( K+1, K+1 ) / D21
                  D22 = W( K, K ) / D21
                  T = ONE / ( D11*D22-ONE )
                  D21 = T / D21
*
*                 Update elements in columns A(k) and A(k+1) as
*                 dot products of rows of ( W(k) W(k+1) ) and columns
*                 of D**(-1)
*
                  DO 80 J = K + 2, N
                     A( J, K ) = D21*( D11*W( J, K )-W( J, K+1 ) )
                     A( J, K+1 ) = D21*( D22*W( J, K+1 )-W( J, K ) )
   80             CONTINUE
               END IF
*
*              Copy D(k) to A
*
               A( K, K ) = W( K, K )
               A( K+1, K ) = W( K+1, K )
               A( K+1, K+1 ) = W( K+1, K+1 )
```

```
*
            END IF
*
         END IF
*
*        Store details of the interchanges in IPIV
*
         IF( KSTEP.EQ.1 ) THEN
            IPIV( K ) = KP
         ELSE
            IPIV( K ) = -KP
            IPIV( K+1 ) = -KP
         END IF
*
*        Increase K and return to the start of the main loop
*
         K = K + KSTEP
         GO TO 70
*
   90    CONTINUE
*
*        Update the lower triangle of A22 (= A(k:n,k:n)) as
*
*        A22 := A22 - L21*D*L21**T = A22 - L21*W**T
*
*        computing blocks of NB columns at a time
*
         DO 110 J = K, N, NB
            JB = MIN( NB, N-J+1 )
*
*           Update the lower triangle of the diagonal block
*
            DO 100 JJ = J, J + JB - 1
               CALL SGEMV( 'No transpose', J+JB-JJ, K-1, -ONE,
     $                     A( JJ, 1 ), LDA, W( JJ, 1 ), LDW, ONE,
     $                     A( JJ, JJ ), 1 )
  100       CONTINUE
*
*           Update the rectangular subdiagonal block
*
            IF( J+JB.LE.N )
     $         CALL SGEMM( 'No transpose', 'Transpose', N-J-JB+1, JB,
     $                     K-1, -ONE, A( J+JB, 1 ), LDA, W( J, 1 ), LDW,
     $                     ONE, A( J+JB, J ), LDA )
  110    CONTINUE
*
*        Put L21 in standard form by partially undoing the interchanges
*        of rows in columns 1:k-1 looping backwards from k-1 to 1
*
         J = K - 1
```

```
  120    CONTINUE
*
*           Undo the interchanges (if any) of rows JJ and JP at each
*           step J
*
*           (Here, J is a diagonal index)
            JJ = J
            JP = IPIV( J )
            IF( JP.LT.0 ) THEN
               JP = -JP
*              (Here, J is a diagonal index)
               J = J - 1
            END IF
*           (NOTE: Here, J is used to determine row length. Length J
*           of the rows to swap back doesn't include diagonal element)
            J = J - 1
            IF( JP.NE.JJ .AND. J.GE.1 )
     $         CALL SSWAP( J, A( JP, 1 ), LDA, A( JJ, 1 ), LDA )
         IF( J.GT.1 )
     $      GO TO 120
*
*        Set KB to the number of columns factorized
*
         KB = K - 1
*
      END IF
      RETURN
*
*     End of SLASYF
*
      END
```

———-

59.5.4 lisp code

— **slasyf.lisp** —

```
(let* ((zero 0.0) (one 1.0) (eight 8.0) (sevten 17.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 8.0 8.0) eight)
           (type (single-float 17.0 17.0) sevten)
           (ignorable zero one eight sevten))
  (defun slasyf (uplo n nb kb a lda ipiv w ldw info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array single-float (*)) w a)
             (type (f2cl-lib:integer4) info ldw lda kb nb n)
```

```
          (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (w single-float w-%data% w-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((absakk 0.0) (alpha 0.0) (colmax 0.0) (d11 0.0) (d21 0.0)
             (d22 0.0) (r1 0.0) (rowmax 0.0) (t$ 0.0) (imax 0) (j 0) (jb 0)
             (jj 0) (jmax 0) (jp 0) (k 0) (kk 0) (kkw 0) (kp 0) (kstep 0)
             (kw 0))
        (declare
         (type (single-float) absakk alpha colmax d11 d21 d22 r1 rowmax t$)
         (type (f2cl-lib:integer4) imax j jb jj jmax jp k kk kkw kp kstep kw))
        (setf info 0)
        (setf alpha (/ (+ one (f2cl-lib:fsqrt sevten)) eight))
        (cond
         ((multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)
          (tagbody
            (setf k n)
           label10
            (setf kw (f2cl-lib:int-sub (f2cl-lib:int-add nb k) n))
            (if (or
                 (and (<= k (f2cl-lib:int-add (f2cl-lib:int-sub n nb) 1))
                      (< nb n))
                 (< k 1))
                (go label30))
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (scopy k
                 (f2cl-lib:array-slice a-%data% single-float (1 k)
                                       ((1 lda) (1 *)) a-%offset%)
                 1
                 (f2cl-lib:array-slice w-%data% single-float (1 kw)
                                       ((1 ldw) (1 *)) w-%offset%)
                 1)
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf k var-0)))
            (if (< k n)
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                     var-9 var-10)
                    (sgemv "No transpose" k (f2cl-lib:int-sub n k) (- one)
                     (f2cl-lib:array-slice a-%data% single-float
                                           (1 (f2cl-lib:int-add k 1))
                                           ((1 lda) (1 *)) a-%offset%)
                     lda
                     (f2cl-lib:array-slice w-%data% single-float
```

```
                                (k (f2cl-lib:int-add kw 1))
                                ((1 ldw) (1 *)) w-%offset%)
          ldw one
          (f2cl-lib:array-slice w-%data% single-float (1 kw)
                                ((1 ldw) (1 *)) w-%offset%)
          1)
      (declare (ignore var-0 var-2 var-3 var-4 var-6 var-9 var-10))
      (when var-1 (setf k var-1))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldw var-7))
      (when var-8 (setf one var-8))))
(setf kstep 1)
(setf absakk
        (abs
         (f2cl-lib:fref w-%data% (k kw) ((1 ldw) (1 *))
                        w-%offset%)))
(cond
 ((> k 1)
  (setf imax
          (isamax (f2cl-lib:int-sub k 1)
           (f2cl-lib:array-slice w-%data% single-float (1 kw)
                                 ((1 ldw) (1 *)) w-%offset%)
           1))
  (setf colmax
          (abs
           (f2cl-lib:fref w-%data% (imax kw) ((1 ldw) (1 *))
                          w-%offset%))))
 (t (setf colmax zero)))
(cond
 ((= (max absakk colmax) zero)
  (if (= info 0)
      (setf info k))
  (setf kp k))
 (t
  (cond ((>= absakk (* alpha colmax)) (setf kp k))
        (t
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (scopy imax
              (f2cl-lib:array-slice a-%data% single-float (1 imax)
                                    ((1 lda) (1 *)) a-%offset%)
              1
              (f2cl-lib:array-slice w-%data% single-float
                                    (1 (f2cl-lib:int-sub kw 1))
                                    ((1 ldw) (1 *)) w-%offset%)
              1)
           (declare (ignore var-1 var-2 var-3 var-4))
           (when var-0 (setf imax var-0)))
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (scopy (f2cl-lib:int-sub k imax)
              (f2cl-lib:array-slice a-%data% single-float
```

```
                                (imax
                                 (f2cl-lib:int-add imax 1))
                                ((1 lda) (1 *)) a-%offset%)
      lda
      (f2cl-lib:array-slice w-%data% single-float
                                ((+ imax 1)
                                 (f2cl-lib:int-sub kw 1))
                                ((1 ldw) (1 *)) w-%offset%)
      1)
  (declare (ignore var-0 var-1 var-3 var-4))
  (when var-2 (setf lda var-2)))
(if (< k n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
         var-8 var-9 var-10)
        (sgemv "No transpose" k (f2cl-lib:int-sub n k)
         (- one)
         (f2cl-lib:array-slice a-%data% single-float
                               (1 (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice w-%data% single-float
                               (imax
                                (f2cl-lib:int-add kw 1))
                               ((1 ldw) (1 *)) w-%offset%)
         ldw one
         (f2cl-lib:array-slice w-%data% single-float
                               (1 (f2cl-lib:int-sub kw 1))
                               ((1 ldw) (1 *)) w-%offset%)
         1)
      (declare
       (ignore var-0 var-2 var-3 var-4 var-6 var-9
        var-10))
      (when var-1 (setf k var-1))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldw var-7))
      (when var-8 (setf one var-8))))
(setf jmax
        (f2cl-lib:int-add imax
                          (isamax
                           (f2cl-lib:int-sub k imax)
                           (f2cl-lib:array-slice w-%data%
                                                 single-float
                                                 ((+ imax
                                                     1)
                                                  (f2cl-lib:int-sub
                                                   kw 1))
                                                 ((1 ldw)
                                                  (1 *))
                                                 w-%offset%)
```

```
                            1)))
(setf rowmax
        (abs
         (f2cl-lib:fref w-%data%
                        (jmax (f2cl-lib:int-sub kw 1))
                        ((1 ldw) (1 *)) w-%offset%)))
(cond
 ((> imax 1)
  (setf jmax
          (isamax (f2cl-lib:int-sub imax 1)
           (f2cl-lib:array-slice w-%data% single-float
                                 (1
                                  (f2cl-lib:int-sub kw 1))
                                 ((1 ldw) (1 *))
                                 w-%offset%)
           1))
  (setf rowmax
          (max rowmax
               (abs
                (f2cl-lib:fref w-%data%
                               (jmax
                                (f2cl-lib:int-sub kw 1))
                               ((1 ldw) (1 *))
                               w-%offset%))))))
(cond
 ((>= absakk
      (* alpha colmax (f2cl-lib:f2cl/ colmax rowmax)))
  (setf kp k))
 ((>=
   (abs
    (f2cl-lib:fref w
                   (imax
                    (f2cl-lib:int-add kw
                                      (f2cl-lib:int-sub
                                       1)))
                   ((1 ldw) (1 *))))
   (* alpha rowmax))
  (setf kp imax)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (scopy k
       (f2cl-lib:array-slice w-%data% single-float
                             (1 (f2cl-lib:int-sub kw 1))
                             ((1 ldw) (1 *)) w-%offset%)
       1
       (f2cl-lib:array-slice w-%data% single-float (1 kw)
                             ((1 ldw) (1 *)) w-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf k var-0))))
 (t (setf kp imax) (setf kstep 2)))))
```

```
(setf kk (f2cl-lib:int-add (f2cl-lib:int-sub k kstep) 1))
(setf kkw (f2cl-lib:int-sub (f2cl-lib:int-add nb kk) n))
(cond
 ((/= kp kk)
  (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                       a-%offset%)
        (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                       a-%offset%))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (scopy (f2cl-lib:int-sub kk 1 kp)
       (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) kk)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% single-float
                             (kp (f2cl-lib:int-add kp 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (if (> kp 1)
      (scopy (f2cl-lib:int-sub kp 1)
       (f2cl-lib:array-slice a-%data% single-float (1 kk)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% single-float (1 kp)
                             ((1 lda) (1 *)) a-%offset%)
       1))
  (if (< k n)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (sswap (f2cl-lib:int-sub n k)
           (f2cl-lib:array-slice a-%data% single-float
                                 (kk (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice a-%data% single-float
                                 (kp (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-3))
        (when var-2 (setf lda var-2))
        (when var-4 (setf lda var-4))))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (sswap (f2cl-lib:int-add (f2cl-lib:int-sub n kk) 1)
       (f2cl-lib:array-slice w-%data% single-float (kk kkw)
                             ((1 ldw) (1 *)) w-%offset%)
       ldw
       (f2cl-lib:array-slice w-%data% single-float (kp kkw)
                             ((1 ldw) (1 *)) w-%offset%)
       ldw)
    (declare (ignore var-0 var-1 var-3))
```

```
      (when var-2 (setf ldw var-2))
      (when var-4 (setf ldw var-4)))))
  (cond
   ((= kstep 1)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy k
         (f2cl-lib:array-slice w-%data% single-float (1 kw)
                               ((1 ldw) (1 *)) w-%offset%)
         1
         (f2cl-lib:array-slice a-%data% single-float (1 k)
                               ((1 lda) (1 *)) a-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf k var-0)))
    (setf r1
            (/ one
               (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                              a-%offset%)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal (f2cl-lib:int-sub k 1) r1
         (f2cl-lib:array-slice a-%data% single-float (1 k)
                               ((1 lda) (1 *)) a-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3))
      (when var-1 (setf r1 var-1))))
   (t
    (cond
     ((> k 2)
      (setf d21
              (f2cl-lib:fref w-%data% ((f2cl-lib:int-sub k 1) kw)
                             ((1 ldw) (1 *)) w-%offset%))
      (setf d11
              (/
               (f2cl-lib:fref w-%data% (k kw) ((1 ldw) (1 *))
                              w-%offset%)
               d21))
      (setf d22
              (/
               (f2cl-lib:fref w-%data%
                              ((f2cl-lib:int-sub k 1)
                               (f2cl-lib:int-sub kw 1))
                              ((1 ldw) (1 *)) w-%offset%)
               d21))
      (setf t$ (/ one (- (* d11 d22) one))) (setf d21 (/ t$ d21))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j
                        (f2cl-lib:int-add k (f2cl-lib:int-sub 2)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-sub k 1))
```

```
                                    ((1 lda) (1 *)) a-%offset%)
                        (* d21
                           (-
                            (* d11
                               (f2cl-lib:fref w-%data%
                                              (j (f2cl-lib:int-sub kw 1))
                                              ((1 ldw) (1 *))
                                              w-%offset%))
                            (f2cl-lib:fref w-%data% (j kw)
                                           ((1 ldw) (1 *)) w-%offset%))))
                (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                    a-%offset%)
                        (* d21
                           (-
                            (* d22
                               (f2cl-lib:fref w-%data% (j kw)
                                              ((1 ldw) (1 *))
                                              w-%offset%))
                            (f2cl-lib:fref w-%data%
                                           (j (f2cl-lib:int-sub kw 1))
                                           ((1 ldw) (1 *)) w-%offset%))))
              label20))))
        (setf (f2cl-lib:fref a-%data%
                             ((f2cl-lib:int-sub k 1)
                              (f2cl-lib:int-sub k 1))
                             ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref w-%data%
                               ((f2cl-lib:int-sub k 1)
                                (f2cl-lib:int-sub kw 1))
                               ((1 ldw) (1 *)) w-%offset%))
        (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                             ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref w-%data% ((f2cl-lib:int-sub k 1) kw)
                               ((1 ldw) (1 *)) w-%offset%))
        (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref w-%data% (k kw) ((1 ldw) (1 *))
                               w-%offset%))))))
    (cond
     ((= kstep 1)
      (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%) kp))
     (t
      (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
              (f2cl-lib:int-sub kp))
      (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                           ipiv-%offset%)
              (f2cl-lib:int-sub kp))))
    (setf k (f2cl-lib:int-sub k kstep))
    (go label10)
   label30
    (f2cl-lib:fdo (j
```

```
                (+
                 (*
                  (the f2cl-lib:integer4 (truncate (+ k (- 1)) nb))
                  nb)
                 1)
                (f2cl-lib:int-add j (f2cl-lib:int-sub nb)))
               ((> j 1) nil)
(tagbody
  (setf jb
          (min (the f2cl-lib:integer4 nb)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub k j)
                                      1))))
  (f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
                ((> jj
                    (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (sgemv "No transpose"
           (f2cl-lib:int-add (f2cl-lib:int-sub jj j) 1)
           (f2cl-lib:int-sub n k) (- one)
           (f2cl-lib:array-slice a-%data% single-float
                                 (j (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice w-%data% single-float
                                 (jj (f2cl-lib:int-add kw 1))
                                 ((1 ldw) (1 *)) w-%offset%)
           ldw one
           (f2cl-lib:array-slice a-%data% single-float (j jj)
                                 ((1 lda) (1 *)) a-%offset%)
           1)
        (declare
         (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9
          var-10))
        (when var-5 (setf lda var-5))
        (when var-7 (setf ldw var-7))
        (when var-8 (setf one var-8)))
    label40))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (sgemm "No transpose" "Transpose" (f2cl-lib:int-sub j 1) jb
       (f2cl-lib:int-sub n k) (- one)
       (f2cl-lib:array-slice a-%data% single-float
                             (1 (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
```

```
            lda
            (f2cl-lib:array-slice w-%data% single-float
                                  (j (f2cl-lib:int-add kw 1))
                                  ((1 ldw) (1 *)) w-%offset%)
            ldw one
            (f2cl-lib:array-slice a-%data% single-float (1 j)
                                  ((1 lda) (1 *)) a-%offset%)
            lda)
          (declare
           (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-8 var-11))
          (when var-3 (setf jb var-3))
          (when var-7 (setf lda var-7))
          (when var-9 (setf ldw var-9))
          (when var-10 (setf one var-10))
          (when var-12 (setf lda var-12)))
       label50))
   (setf j (f2cl-lib:int-add k 1))
  label60
   (setf jj j)
   (setf jp (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
   (cond
    ((< jp 0) (setf jp (f2cl-lib:int-sub jp))
     (setf j (f2cl-lib:int-add j 1))))
   (setf j (f2cl-lib:int-add j 1))
   (if (and (/= jp jj) (<= j n))
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (sswap (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1)
            (f2cl-lib:array-slice a-%data% single-float (jp j)
                                  ((1 lda) (1 *)) a-%offset%)
            lda
            (f2cl-lib:array-slice a-%data% single-float (jj j)
                                  ((1 lda) (1 *)) a-%offset%)
            lda)
          (declare (ignore var-0 var-1 var-3))
          (when var-2 (setf lda var-2))
          (when var-4 (setf lda var-4))))
   (if (< j n)
       (go label60))
   (setf kb (f2cl-lib:int-sub n k))))
(t
 (tagbody
   (setf k 1)
  label70
   (if (or (and (>= k nb) (< nb n)) (> k n))
       (go label90))
   (scopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
    (f2cl-lib:array-slice a-%data% single-float (k k) ((1 lda) (1 *))
                          a-%offset%)
    1
    (f2cl-lib:array-slice w-%data% single-float (k k) ((1 ldw) (1 *))
```

```
                        w-%offset%)
 1)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (sgemv "No transpose"
     (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
     (f2cl-lib:int-sub k 1) (- one)
     (f2cl-lib:array-slice a-%data% single-float (k 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice w-%data% single-float (k 1)
                           ((1 ldw) (1 *)) w-%offset%)
     ldw one
     (f2cl-lib:array-slice w-%data% single-float (k k)
                           ((1 ldw) (1 *)) w-%offset%)
     1)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9 var-10))
  (when var-5 (setf lda var-5))
  (when var-7 (setf ldw var-7))
  (when var-8 (setf one var-8)))
(setf kstep 1)
(setf absakk
        (abs
         (f2cl-lib:fref w-%data% (k k) ((1 ldw) (1 *))
                        w-%offset%)))
(cond
 ((< k n)
  (setf imax
          (f2cl-lib:int-add k
                            (isamax (f2cl-lib:int-sub n k)
                             (f2cl-lib:array-slice w-%data%
                                                   single-float
                                                   ((+ k 1) k)
                                                   ((1 ldw) (1 *))
                                                   w-%offset%)
                             1)))
  (setf colmax
          (abs
           (f2cl-lib:fref w-%data% (imax k) ((1 ldw) (1 *))
                          w-%offset%))))
 (t (setf colmax zero)))
(cond
 ((= (max absakk colmax) zero)
  (if (= info 0)
      (setf info k))
  (setf kp k))
 (t
  (cond ((>= absakk (* alpha colmax)) (setf kp k))
```

```
(t
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (scopy (f2cl-lib:int-sub imax k)
      (f2cl-lib:array-slice a-%data% single-float (imax k)
                            ((1 lda) (1 *)) a-%offset%)
      lda
      (f2cl-lib:array-slice w-%data% single-float
                            (k (f2cl-lib:int-add k 1))
                            ((1 ldw) (1 *)) w-%offset%)
      1)
   (declare (ignore var-0 var-1 var-3 var-4))
   (when var-2 (setf lda var-2)))
 (scopy (f2cl-lib:int-add (f2cl-lib:int-sub n imax) 1)
  (f2cl-lib:array-slice a-%data% single-float (imax imax)
                        ((1 lda) (1 *)) a-%offset%)
  1
  (f2cl-lib:array-slice w-%data% single-float
                        (imax (f2cl-lib:int-add k 1))
                        ((1 ldw) (1 *)) w-%offset%)
  1)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10)
     (sgemv "No transpose"
      (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
      (f2cl-lib:int-sub k 1) (- one)
      (f2cl-lib:array-slice a-%data% single-float (k 1)
                            ((1 lda) (1 *)) a-%offset%)
      lda
      (f2cl-lib:array-slice w-%data% single-float (imax 1)
                            ((1 ldw) (1 *)) w-%offset%)
      ldw one
      (f2cl-lib:array-slice w-%data% single-float
                            (k (f2cl-lib:int-add k 1))
                            ((1 ldw) (1 *)) w-%offset%)
      1)
   (declare
    (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9
     var-10))
   (when var-5 (setf lda var-5))
   (when var-7 (setf ldw var-7))
   (when var-8 (setf one var-8)))
 (setf jmax
         (f2cl-lib:int-add (f2cl-lib:int-sub k 1)
                           (isamax
                            (f2cl-lib:int-sub imax k)
                            (f2cl-lib:array-slice w-%data%
                                                  single-float
                                                  (k
                                                   (f2cl-lib:int-add
```

```
                                                                  k 1))
                                                                ((1 ldw)
                                                                 (1 *))
                                                                w-%offset%)
                                  1)))
          (setf rowmax
                  (abs
                   (f2cl-lib:fref w-%data%
                                  (jmax (f2cl-lib:int-add k 1))
                                  ((1 ldw) (1 *)) w-%offset%)))
          (cond
           ((< imax n)
            (setf jmax
                    (f2cl-lib:int-add imax
                                      (isamax
                                       (f2cl-lib:int-sub n imax)
                                       (f2cl-lib:array-slice
                                        w-%data% single-float
                                        ((+ imax 1)
                                         (f2cl-lib:int-add k 1))
                                        ((1 ldw) (1 *)) w-%offset%)
                                       1)))
            (setf rowmax
                    (max rowmax
                         (abs
                          (f2cl-lib:fref w-%data%
                                         (jmax
                                          (f2cl-lib:int-add k 1))
                                         ((1 ldw) (1 *))
                                         w-%offset%))))))
          (cond
           ((>= absakk
                (* alpha colmax (f2cl-lib:f2cl/ colmax rowmax)))
            (setf kp k))
           ((>=
             (abs
              (f2cl-lib:fref w (imax (f2cl-lib:int-add k 1))
                             ((1 ldw) (1 *))))
             (* alpha rowmax))
            (setf kp imax)
            (scopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
             (f2cl-lib:array-slice w-%data% single-float
                                   (k (f2cl-lib:int-add k 1))
                                   ((1 ldw) (1 *)) w-%offset%)
             1
             (f2cl-lib:array-slice w-%data% single-float (k k)
                                   ((1 ldw) (1 *)) w-%offset%)
             1))
           (t (setf kp imax) (setf kstep 2)))))
        (setf kk (f2cl-lib:int-sub (f2cl-lib:int-add k kstep) 1))
```

```
(cond
 ((/= kp kk)
  (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                       a-%offset%)
        (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                         a-%offset%))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (scopy (f2cl-lib:int-sub kp kk 1)
       (f2cl-lib:array-slice a-%data% single-float ((+ kk 1) kk)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% single-float
                             (kp (f2cl-lib:int-add kk 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (if (< kp n)
      (scopy (f2cl-lib:int-sub n kp)
       (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) kk)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) kp)
                             ((1 lda) (1 *)) a-%offset%)
       1))
  (if (> k 1)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (sswap (f2cl-lib:int-sub k 1)
           (f2cl-lib:array-slice a-%data% single-float (kk 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice a-%data% single-float (kp 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-3))
        (when var-2 (setf lda var-2))
        (when var-4 (setf lda var-4))))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (sswap kk
       (f2cl-lib:array-slice w-%data% single-float (kk 1)
                             ((1 ldw) (1 *)) w-%offset%)
       ldw
       (f2cl-lib:array-slice w-%data% single-float (kp 1)
                             ((1 ldw) (1 *)) w-%offset%)
       ldw)
    (declare (ignore var-1 var-3))
    (when var-0 (setf kk var-0))
    (when var-2 (setf ldw var-2))
    (when var-4 (setf ldw var-4)))))
(cond
```

```
((= kstep 1)
 (scopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
  (f2cl-lib:array-slice w-%data% single-float (k k)
                        ((1 ldw) (1 *)) w-%offset%)
  1
  (f2cl-lib:array-slice a-%data% single-float (k k)
                        ((1 lda) (1 *)) a-%offset%)
  1)
 (cond
  ((< k n)
   (setf r1
           (/ one
              (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                             a-%offset%)))
   (multiple-value-bind (var-0 var-1 var-2 var-3)
       (sscal (f2cl-lib:int-sub n k) r1
        (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                              ((1 lda) (1 *)) a-%offset%)
        1)
     (declare (ignore var-0 var-2 var-3))
     (when var-1 (setf r1 var-1))))))
(t
 (cond
  ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
   (setf d21
           (f2cl-lib:fref w-%data% ((f2cl-lib:int-add k 1) k)
                          ((1 ldw) (1 *)) w-%offset%))
   (setf d11
           (/
            (f2cl-lib:fref w-%data%
                           ((f2cl-lib:int-add k 1)
                            (f2cl-lib:int-add k 1))
                           ((1 ldw) (1 *)) w-%offset%)
            d21))
   (setf d22
           (/
            (f2cl-lib:fref w-%data% (k k) ((1 ldw) (1 *))
                           w-%offset%)
            d21))
   (setf t$ (/ one (- (* d11 d22) one))) (setf d21 (/ t$ d21))
   (f2cl-lib:fdo (j (f2cl-lib:int-add k 2)
                  (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                            a-%offset%)
               (* d21
                  (-
                   (* d11
                      (f2cl-lib:fref w-%data% (j k)
```

```
                                          ((1 ldw) (1 *))
                                          w-%offset%))
                         (f2cl-lib:fref w-%data%
                                        (j (f2cl-lib:int-add k 1))
                                        ((1 ldw) (1 *)) w-%offset%))))
            (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
                    (* d21
                       (-
                        (* d22
                           (f2cl-lib:fref w-%data%
                                          (j (f2cl-lib:int-add k 1))
                                          ((1 ldw) (1 *))
                                          w-%offset%))
                         (f2cl-lib:fref w-%data% (j k) ((1 ldw) (1 *))
                                        w-%offset%))))
          label80))))
      (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref w-%data% (k k) ((1 ldw) (1 *))
                             w-%offset%))
      (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                           ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref w-%data% ((f2cl-lib:int-add k 1) k)
                             ((1 ldw) (1 *)) w-%offset%))
      (setf (f2cl-lib:fref a-%data%
                           ((f2cl-lib:int-add k 1)
                            (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref w-%data%
                             ((f2cl-lib:int-add k 1)
                              (f2cl-lib:int-add k 1))
                             ((1 ldw) (1 *)) w-%offset%))))))
  (cond
   ((= kstep 1)
    (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%) kp))
   (t
    (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
            (f2cl-lib:int-sub kp))
    (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1)) ((1 *))
                         ipiv-%offset%)
            (f2cl-lib:int-sub kp))))
  (setf k (f2cl-lib:int-add k kstep))
  (go label70)
 label90
  (f2cl-lib:fdo (j k (f2cl-lib:int-add j nb))
                ((> j n) nil)
    (tagbody
      (setf jb
              (min (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
```

```
                    (f2cl-lib:int-add (f2cl-lib:int-sub n j)
                                      1))))
(f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
              ((> jj
                  (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10)
        (sgemv "No transpose"
         (f2cl-lib:int-sub (f2cl-lib:int-add j jb) jj)
         (f2cl-lib:int-sub k 1) (- one)
         (f2cl-lib:array-slice a-%data% single-float (jj 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice w-%data% single-float (jj 1)
                               ((1 ldw) (1 *)) w-%offset%)
         ldw one
         (f2cl-lib:array-slice a-%data% single-float (jj jj)
                               ((1 lda) (1 *)) a-%offset%)
         1)
      (declare
       (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9
        var-10))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldw var-7))
      (when var-8 (setf one var-8)))
   label100))
(if (<= (f2cl-lib:int-add j jb) n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (sgemm "No transpose" "Transpose"
         (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) jb
         (f2cl-lib:int-sub k 1) (- one)
         (f2cl-lib:array-slice a-%data% single-float
                               ((+ j jb) 1) ((1 lda) (1 *))
                               a-%offset%)
         lda
         (f2cl-lib:array-slice w-%data% single-float (j 1)
                               ((1 ldw) (1 *)) w-%offset%)
         ldw one
         (f2cl-lib:array-slice a-%data% single-float
                               ((+ j jb) j) ((1 lda) (1 *))
                               a-%offset%)
         lda)
      (declare
       (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-8
        var-11))
```

```
                    (when var-3 (setf jb var-3))
                    (when var-7 (setf lda var-7))
                    (when var-9 (setf ldw var-9))
                    (when var-10 (setf one var-10))
                    (when var-12 (setf lda var-12))))
               label110))
            (setf j (f2cl-lib:int-sub k 1))
           label120
            (setf jj j)
            (setf jp (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
            (cond
             ((< jp 0) (setf jp (f2cl-lib:int-sub jp))
              (setf j (f2cl-lib:int-sub j 1))))
            (setf j (f2cl-lib:int-sub j 1))
            (if (and (/= jp jj) (>= j 1))
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (sswap j
                     (f2cl-lib:array-slice a-%data% single-float (jp 1)
                                           ((1 lda) (1 *)) a-%offset%)
                     lda
                     (f2cl-lib:array-slice a-%data% single-float (jj 1)
                                           ((1 lda) (1 *)) a-%offset%)
                     lda)
                  (declare (ignore var-1 var-3))
                  (when var-0 (setf j var-0))
                  (when var-2 (setf lda var-2))
                  (when var-4 (setf lda var-4))))
            (if (> j 1)
                (go label120))
            (setf kb (f2cl-lib:int-sub k 1)))))
        (go end_label)
       end_label
        (return (values uplo nil nil kb nil lda nil nil ldw info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slasyf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
```

```
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::uplo nil nil
  fortran-to-lisp::kb nil
  fortran-to-lisp::lda nil nil
  fortran-to-lisp::ldw
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

59.6 slasyf_rook

59.6.1 Axiom unit tests

— **slasyfrook.input** —

```
)set break resume
)sys rm -f slasyfrook.output
)spool slasyfrook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.6.2 Axiom help page

— **slasyf_rook.help** —

```
==================================================================
slasyf_rook examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
```

```
   Functions/Subroutines
       subroutine slasyf_rook (UPLO, N, NB, KB, A, LDA, IPIV, W, LDW, INFO)
           SLASYF_ROOK computes a partial factorization of a real symmetric
           matrix using the bounded Bunch-Kaufman ('rook') diagonal pivoting
           method.

Function/Subroutine Documentation
   subroutine slasyf_rook (character UPLO, integer N, integer NB, integer KB,
       real, dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       real, dimension( ldw, * ) W, integer LDW, integer INFO)
       SLASYF_ROOK computes a partial factorization of a real symmetric matrix
       using the bounded Bunch-Kaufman ('rook') diagonal pivoting method.

Purpose:
   SLASYF_ROOK computes a partial factorization of a real symmetric
   matrix A using the bounded Bunch-Kaufman ("rook") diagonal
   pivoting method. The partial factorization has the form:

       A  =  ( I  U12 ) ( A11  0  ) (  I       0    )  if UPLO = 'U', or:
             ( 0  U22 ) (  0   D  ) ( U12**T U22**T )

       A  =  ( L11  0 ) (  D   0  ) ( L11**T L21**T )  if UPLO = 'L'
             ( L21  I ) (  0  A22 ) (  0       I    )

   where the order of D is at most NB. The actual order is returned in
   the argument KB, and is either NB or NB-1, or N if N <= NB.

   SLASYF_ROOK is an auxiliary routine called by SSYTRF_ROOK. It uses
   blocked code (calling Level 3 BLAS) to update the submatrix
   A11 (if UPLO = 'U') or A22 (if UPLO = 'L').

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is stored:
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NB
   NB is INTEGER
   The maximum number of columns of the matrix A that should be
   factored.  NB should be at least 2 to allow for 2-by-2 pivot blocks.

 KB
```

```
  KB is INTEGER
  The number of columns of A that were actually factored.
  KB is either NB-1 or NB, or N if N <= NB.

A
  A is REAL array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  n-by-n upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading n-by-n lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.
  On exit, A contains details of the partial factorization.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D.

  If UPLO = 'U':
     Only the last KB elements of IPIV are set.

     If IPIV(k) > 0, then rows and columns k and IPIV(k) were
     interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) < 0 and IPIV(k-1) < 0, then rows and
     columns k and -IPIV(k) were interchanged and rows and
     columns k-1 and -IPIV(k-1) were inerchaged,
     D(k-1:k,k-1:k) is a 2-by-2 diagonal block.

  If UPLO = 'L':
     Only the first KB elements of IPIV are set.

     If IPIV(k) > 0, then rows and columns k and IPIV(k)
     were interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) < 0 and IPIV(k+1) < 0, then rows and
     columns k and -IPIV(k) were interchanged and rows and
     columns k+1 and -IPIV(k+1) were inerchaged,
     D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

W
  W is REAL array, dimension (LDW,NB)

LDW
  LDW is INTEGER
```

```
  The leading dimension of the array W.  LDW >= max(1,N).

 INFO
  INFO is INTEGER
  = 0: successful exit
  > 0: if INFO = k, D(k,k) is exactly zero.  The factorization
       has been completed, but the block diagonal matrix D is
       exactly singular.
```

59.6.3 fortran code

— slasyf_rook.f —

```
*  =====================================================================
      SUBROUTINE SLASYF_ROOK( UPLO, N, NB, KB, A, LDA, IPIV, W, LDW,
     $                        INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, KB, LDA, LDW, N, NB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), W( LDW, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               EIGHT, SEVTEN
      PARAMETER          ( EIGHT = 8.0E+0, SEVTEN = 17.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            DONE
      INTEGER            IMAX, ITEMP, J, JB, JJ, JMAX, JP1, JP2, K, KK,
     $                   KW, KKW, KP, KSTEP, P, II

      REAL               ABSAKK, ALPHA, COLMAX, D11, D12, D21, D22,
```

```
     $                   STEMP, R1, ROWMAX, T, SFMIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ISAMAX
      REAL               SLAMCH
      EXTERNAL           LSAME, ISAMAX, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SGEMM, SGEMV, SSCAL, SSWAP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
*     Initialize ALPHA for use in choosing pivot block size.
*
      ALPHA = ( ONE+SQRT( SEVTEN ) ) / EIGHT
*
*     Compute machine safe minimum
*
      SFMIN = SLAMCH( 'S' )
*
      IF( LSAME( UPLO, 'U' ) ) THEN
*
*        Factorize the trailing columns of A using the upper triangle
*        of A and working backwards, and compute the matrix W = U12*D
*        for use in updating A11
*
*        K is the main loop index, decreasing from N in steps of 1 or 2
*
         K = N
   10    CONTINUE
*
*        KW is the column of W which corresponds to column K of A
*
         KW = NB + K - N
*
*        Exit from loop
*
         IF( ( K.LE.N-NB+1 .AND. NB.LT.N ) .OR. K.LT.1 )
     $      GO TO 30
*
         KSTEP = 1
         P = K
*
*        Copy column K of A to column KW of W and update it
```

```
*
         CALL SCOPY( K, A( 1, K ), 1, W( 1, KW ), 1 )
         IF( K.LT.N )
     $      CALL SGEMV( 'No transpose', K, N-K, -ONE, A( 1, K+1 ),
     $                  LDA, W( K, KW+1 ), LDW, ONE, W( 1, KW ), 1 )
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = ABS( W( K, KW ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.GT.1 ) THEN
            IMAX = ISAMAX( K-1, W( 1, KW ), 1 )
            COLMAX = ABS( W( IMAX, KW ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( MAX( ABSAKK, COLMAX ).EQ.ZERO ) THEN
*
*           Column K is zero or underflow: set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
            CALL SCOPY( K, W( 1, KW ), 1, A( 1, K ), 1 )
         ELSE
*
*           ============================================================
*
*           Test for interchange
*
*           Equivalent to testing for ABSAKK.GE.ALPHA*COLMAX
*           (used to handle NaN and Inf)
*
            IF( .NOT.( ABSAKK.LT.ALPHA*COLMAX ) ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
*
            ELSE
*
               DONE = .FALSE.
*
*              Loop until pivot found
```

```
*
   12          CONTINUE
*
*                 Begin pivot search loop body
*
*
*                 Copy column IMAX to column KW-1 of W and update it
*
                  CALL SCOPY( IMAX, A( 1, IMAX ), 1, W( 1, KW-1 ), 1 )
                  CALL SCOPY( K-IMAX, A( IMAX, IMAX+1 ), LDA,
     $                        W( IMAX+1, KW-1 ), 1 )
*
                  IF( K.LT.N )
     $               CALL SGEMV( 'No transpose', K, N-K, -ONE,
     $                           A( 1, K+1 ), LDA, W( IMAX, KW+1 ), LDW,
     $                           ONE, W( 1, KW-1 ), 1 )
*
*                 JMAX is the column-index of the largest off-diagonal
*                 element in row IMAX, and ROWMAX is its absolute value.
*                 Determine both ROWMAX and JMAX.
*
                  IF( IMAX.NE.K ) THEN
                     JMAX = IMAX + ISAMAX( K-IMAX, W( IMAX+1, KW-1 ),
     $                                     1 )
                     ROWMAX = ABS( W( JMAX, KW-1 ) )
                  ELSE
                     ROWMAX = ZERO
                  END IF
*
                  IF( IMAX.GT.1 ) THEN
                     ITEMP = ISAMAX( IMAX-1, W( 1, KW-1 ), 1 )
                     STEMP = ABS( W( ITEMP, KW-1 ) )
                     IF( STEMP.GT.ROWMAX ) THEN
                        ROWMAX = STEMP
                        JMAX = ITEMP
                     END IF
                  END IF
*
*                 Equivalent to testing for
*                 ABS( W( IMAX, KW-1 ) ).GE.ALPHA*ROWMAX
*                 (used to handle NaN and Inf)
*
                  IF( .NOT.(ABS( W( IMAX, KW-1 ) ).LT.ALPHA*ROWMAX ) )
     $            THEN
*
*                    interchange rows and columns K and IMAX,
*                    use 1-by-1 pivot block
*
                     KP = IMAX
*
```

```
*                    copy column KW-1 of W to column KW of W
*
                     CALL SCOPY( K, W( 1, KW-1 ), 1, W( 1, KW ), 1 )
*
                     DONE = .TRUE.
*
                  Equivalent to testing for ROWMAX.EQ.COLMAX,
*                 (used to handle NaN and Inf)
*
                  ELSE IF( ( P.EQ.JMAX ) .OR. ( ROWMAX.LE.COLMAX ) )
     $            THEN
*
                     interchange rows and columns K-1 and IMAX,
*                    use 2-by-2 pivot block
*
                     KP = IMAX
                     KSTEP = 2
                     DONE = .TRUE.
                  ELSE
*
                     Pivot not found: set params and repeat
*
                     P = IMAX
                     COLMAX = ROWMAX
                     IMAX = JMAX
*
                     Copy updated JMAXth (next IMAXth) column to Kth of W
*
                     CALL SCOPY( K, W( 1, KW-1 ), 1, W( 1, KW ), 1 )
*
                  END IF
*
                  End pivot search loop body
*
               IF( .NOT. DONE ) GOTO 12
*
            END IF
*
            ============================================================
*
            KK = K - KSTEP + 1
*
            KKW is the column of W which corresponds to column KK of A
*
            KKW = NB + KK - N
*
            IF( ( KSTEP.EQ.2 ) .AND. ( P.NE.K ) ) THEN
*
               Copy non-updated column K to column P
*
```

```
               CALL SCOPY( K-P, A( P+1, K ), 1, A( P, P+1 ), LDA )
               CALL SCOPY( P, A( 1, K ), 1, A( 1, P ), 1 )
*
*              Interchange rows K and P in last N-K+1 columns of A
*              and last N-K+2 columns of W
*
               CALL SSWAP( N-K+1, A( K, K ), LDA, A( P, K ), LDA )
               CALL SSWAP( N-KK+1, W( K, KKW ), LDW, W( P, KKW ), LDW )
            END IF
*
*           Updated column KP is already stored in column KKW of W
*
            IF( KP.NE.KK ) THEN
*
*              Copy non-updated column KK to column KP
*
               A( KP, K ) = A( KK, K )
               CALL SCOPY( K-1-KP, A( KP+1, KK ), 1, A( KP, KP+1 ),
     $                     LDA )
               CALL SCOPY( KP, A( 1, KK ), 1, A( 1, KP ), 1 )
*
*              Interchange rows KK and KP in last N-KK+1 columns
*              of A and W
*
               CALL SSWAP( N-KK+1, A( KK, KK ), LDA, A( KP, KK ), LDA )
               CALL SSWAP( N-KK+1, W( KK, KKW ), LDW, W( KP, KKW ),
     $                     LDW )
            END IF
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column KW of W now holds
*
*              W(k) = U(k)*D(k)
*
*              where U(k) is the k-th column of U
*
*              Store U(k) in column k of A
*
               CALL SCOPY( K, W( 1, KW ), 1, A( 1, K ), 1 )
               IF( K.GT.1 ) THEN
                  IF( ABS( A( K, K ) ).GE.SFMIN ) THEN
                     R1 = ONE / A( K, K )
                     CALL SSCAL( K-1, R1, A( 1, K ), 1 )
                  ELSE IF( A( K, K ).NE.ZERO ) THEN
                     DO 14 II = 1, K - 1
                        A( II, K ) = A( II, K ) / A( K, K )
   14                CONTINUE
                  END IF
               END IF
```

```
*
            ELSE
*
*              2-by-2 pivot block D(k): columns KW and KW-1 of W now
*              hold
*
*              ( W(k-1) W(k) ) = ( U(k-1) U(k) )*D(k)
*
*              where U(k) and U(k-1) are the k-th and (k-1)-th columns
*              of U
*
               IF( K.GT.2 ) THEN
*
*                 Store U(k) and U(k-1) in columns k and k-1 of A
*
                  D12 = W( K-1, KW )
                  D11 = W( K, KW ) / D12
                  D22 = W( K-1, KW-1 ) / D12
                  T = ONE / ( D11*D22-ONE )
                  DO 20 J = 1, K - 2
                     A( J, K-1 ) = T*( (D11*W( J, KW-1 )-W( J, KW ) ) /
     $                               D12 )
                     A( J, K ) = T*( ( D22*W( J, KW )-W( J, KW-1 ) ) /
     $                            D12 )
   20             CONTINUE
               END IF
*
*              Copy D(k) to A
*
               A( K-1, K-1 ) = W( K-1, KW-1 )
               A( K-1, K ) = W( K-1, KW )
               A( K, K ) = W( K, KW )
            END IF
         END IF
*
*        Store details of the interchanges in IPIV
*
         IF( KSTEP.EQ.1 ) THEN
            IPIV( K ) = KP
         ELSE
            IPIV( K ) = -P
            IPIV( K-1 ) = -KP
         END IF
*
*        Decrease K and return to the start of the main loop
*
         K = K - KSTEP
         GO TO 10
*
   30    CONTINUE
```

```
*
*        Update the upper triangle of A11 (= A(1:k,1:k)) as
*
*        A11 := A11 - U12*D*U12**T = A11 - U12*W**T
*
*        computing blocks of NB columns at a time
*
         DO 50 J = ( ( K-1 ) / NB )*NB + 1, 1, -NB
            JB = MIN( NB, K-J+1 )
*
*           Update the upper triangle of the diagonal block
*
            DO 40 JJ = J, J + JB - 1
               CALL SGEMV( 'No transpose', JJ-J+1, N-K, -ONE,
     $                     A( J, K+1 ), LDA, W( JJ, KW+1 ), LDW, ONE,
     $                     A( J, JJ ), 1 )
   40       CONTINUE
*
*           Update the rectangular superdiagonal block
*
            IF( J.GE.2 )
     $         CALL SGEMM( 'No transpose', 'Transpose', J-1, JB,
     $                  N-K, -ONE, A( 1, K+1 ), LDA, W( J, KW+1 ), LDW,
     $                  ONE, A( 1, J ), LDA )
   50    CONTINUE
*
*        Put U12 in standard form by partially undoing the interchanges
*        in columns k+1:n
*
         J = K + 1
   60    CONTINUE
*
            KSTEP = 1
            JP1 = 1
            JJ = J
            JP2 = IPIV( J )
            IF( JP2.LT.0 ) THEN
               JP2 = -JP2
               J = J + 1
               JP1 = -IPIV( J )
               KSTEP = 2
            END IF
*
            J = J + 1
            IF( JP2.NE.JJ .AND. J.LE.N )
     $         CALL SSWAP( N-J+1, A( JP2, J ), LDA, A( JJ, J ), LDA )
            JJ = J - 1
            IF( JP1.NE.JJ .AND. KSTEP.EQ.2 )
     $         CALL SSWAP( N-J+1, A( JP1, J ), LDA, A( JJ, J ), LDA )
         IF( J.LE.N )
```

```
     $      GO TO 60
*
*       Set KB to the number of columns factorized
*
        KB = N - K
*
      ELSE
*
*       Factorize the leading columns of A using the lower triangle
*       of A and working forwards, and compute the matrix W = L21*D
*       for use in updating A22
*
*       K is the main loop index, increasing from 1 in steps of 1 or 2
*
        K = 1
   70   CONTINUE
*
*       Exit from loop
*
        IF( ( K.GE.NB .AND. NB.LT.N ) .OR. K.GT.N )
     $      GO TO 90
*
        KSTEP = 1
        P = K
*
*       Copy column K of A to column K of W and update it
*
        CALL SCOPY( N-K+1, A( K, K ), 1, W( K, K ), 1 )
        IF( K.GT.1 )
     $      CALL SGEMV( 'No transpose', N-K+1, K-1, -ONE, A( K, 1 ),
     $                  LDA, W( K, 1 ), LDW, ONE, W( K, K ), 1 )
*
*       Determine rows and columns to be interchanged and whether
*       a 1-by-1 or 2-by-2 pivot block will be used
*
        ABSAKK = ABS( W( K, K ) )
*
*       IMAX is the row-index of the largest off-diagonal element in
*       column K, and COLMAX is its absolute value.
*       Determine both COLMAX and IMAX.
*
        IF( K.LT.N ) THEN
           IMAX = K + ISAMAX( N-K, W( K+1, K ), 1 )
           COLMAX = ABS( W( IMAX, K ) )
        ELSE
           COLMAX = ZERO
        END IF
*
        IF( MAX( ABSAKK, COLMAX ).EQ.ZERO ) THEN
*
```

```
*           Column K is zero or underflow: set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
            CALL SCOPY( N-K+1, W( K, K ), 1, A( K, K ), 1 )
         ELSE
*
*           ============================================================
*
*           Test for interchange
*
*           Equivalent to testing for ABSAKK.GE.ALPHA*COLMAX
*           (used to handle NaN and Inf)
*
            IF( .NOT.( ABSAKK.LT.ALPHA*COLMAX ) ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
*
            ELSE
*
               DONE = .FALSE.
*
*              Loop until pivot found
*
   72          CONTINUE
*
*                 Begin pivot search loop body
*
*
*                 Copy column IMAX to column K+1 of W and update it
*
                  CALL SCOPY( IMAX-K, A( IMAX, K ), LDA, W( K, K+1 ), 1)
                  CALL SCOPY( N-IMAX+1, A( IMAX, IMAX ), 1,
     $                        W( IMAX, K+1 ), 1 )
                  IF( K.GT.1 )
     $               CALL SGEMV( 'No transpose', N-K+1, K-1, -ONE,
     $                           A( K, 1 ), LDA, W( IMAX, 1 ), LDW,
     $                           ONE, W( K, K+1 ), 1 )
*
*                 JMAX is the column-index of the largest off-diagonal
*                 element in row IMAX, and ROWMAX is its absolute value.
*                 Determine both ROWMAX and JMAX.
*
                  IF( IMAX.NE.K ) THEN
                     JMAX = K - 1 + ISAMAX( IMAX-K, W( K, K+1 ), 1 )
                     ROWMAX = ABS( W( JMAX, K+1 ) )
                  ELSE
```

```
                 ROWMAX = ZERO
              END IF
*
              IF( IMAX.LT.N ) THEN
                 ITEMP = IMAX + ISAMAX( N-IMAX, W( IMAX+1, K+1 ), 1)
                 STEMP = ABS( W( ITEMP, K+1 ) )
                 IF( STEMP.GT.ROWMAX ) THEN
                    ROWMAX = STEMP
                    JMAX = ITEMP
                 END IF
              END IF
*
*             Equivalent to testing for
*             ABS( W( IMAX, K+1 ) ).GE.ALPHA*ROWMAX
*             (used to handle NaN and Inf)
*
              IF( .NOT.( ABS( W( IMAX, K+1 ) ).LT.ALPHA*ROWMAX ) )
     $        THEN
*
*                interchange rows and columns K and IMAX,
*                use 1-by-1 pivot block
*
                 KP = IMAX
*
*                copy column K+1 of W to column K of W
*
                 CALL SCOPY( N-K+1, W( K, K+1 ), 1, W( K, K ), 1 )
*
                 DONE = .TRUE.
*
*             Equivalent to testing for ROWMAX.EQ.COLMAX,
*             (used to handle NaN and Inf)
*
              ELSE IF( ( P.EQ.JMAX ) .OR. ( ROWMAX.LE.COLMAX ) )
     $        THEN
*
*                interchange rows and columns K+1 and IMAX,
*                use 2-by-2 pivot block
*
                 KP = IMAX
                 KSTEP = 2
                 DONE = .TRUE.
              ELSE
*
*                Pivot not found: set params and repeat
*
                 P = IMAX
                 COLMAX = ROWMAX
                 IMAX = JMAX
*
```

```
*                    Copy updated JMAXth (next IMAXth) column to Kth of W
*
                     CALL SCOPY( N-K+1, W( K, K+1 ), 1, W( K, K ), 1 )
*
                  END IF
*
*                 End pivot search loop body
*
               IF( .NOT. DONE ) GOTO 72
*
            END IF
*
*           ============================================================
*
            KK = K + KSTEP - 1
*
            IF( ( KSTEP.EQ.2 ) .AND. ( P.NE.K ) ) THEN
*
*              Copy non-updated column K to column P
*
               CALL SCOPY( P-K, A( K, K ), 1, A( P, K ), LDA )
               CALL SCOPY( N-P+1, A( P, K ), 1, A( P, P ), 1 )
*
*              Interchange rows K and P in first K columns of A
*              and first K+1 columns of W
*
               CALL SSWAP( K, A( K, 1 ), LDA, A( P, 1 ), LDA )
               CALL SSWAP( KK, W( K, 1 ), LDW, W( P, 1 ), LDW )
            END IF
*
*           Updated column KP is already stored in column KK of W
*
            IF( KP.NE.KK ) THEN
*
*              Copy non-updated column KK to column KP
*
               A( KP, K ) = A( KK, K )
               CALL SCOPY( KP-K-1, A( K+1, KK ), 1, A( KP, K+1 ), LDA )
               CALL SCOPY( N-KP+1, A( KP, KK ), 1, A( KP, KP ), 1 )
*
*              Interchange rows KK and KP in first KK columns of A and W
*
               CALL SSWAP( KK, A( KK, 1 ), LDA, A( KP, 1 ), LDA )
               CALL SSWAP( KK, W( KK, 1 ), LDW, W( KP, 1 ), LDW )
            END IF
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column k of W now holds
*
```

```
*             W(k) = L(k)*D(k)
*
*             where L(k) is the k-th column of L
*
*             Store L(k) in column k of A
*
              CALL SCOPY( N-K+1, W( K, K ), 1, A( K, K ), 1 )
              IF( K.LT.N ) THEN
                 IF( ABS( A( K, K ) ).GE.SFMIN ) THEN
                    R1 = ONE / A( K, K )
                    CALL SSCAL( N-K, R1, A( K+1, K ), 1 )
                 ELSE IF( A( K, K ).NE.ZERO ) THEN
                    DO 74 II = K + 1, N
                       A( II, K ) = A( II, K ) / A( K, K )
   74               CONTINUE
                 END IF
              END IF
*
           ELSE
*
*             2-by-2 pivot block D(k): columns k and k+1 of W now hold
*
*             ( W(k) W(k+1) ) = ( L(k) L(k+1) )*D(k)
*
*             where L(k) and L(k+1) are the k-th and (k+1)-th columns
*             of L
*
              IF( K.LT.N-1 ) THEN
*
*                Store L(k) and L(k+1) in columns k and k+1 of A
*
                 D21 = W( K+1, K )
                 D11 = W( K+1, K+1 ) / D21
                 D22 = W( K, K ) / D21
                 T = ONE / ( D11*D22-ONE )
                 DO 80 J = K + 2, N
                    A( J, K ) = T*( ( D11*W( J, K )-W( J, K+1 ) ) /
     $                          D21 )
                    A( J, K+1 ) = T*( ( D22*W( J, K+1 )-W( J, K ) ) /
     $                            D21 )
   80            CONTINUE
              END IF
*
*             Copy D(k) to A
*
              A( K, K ) = W( K, K )
              A( K+1, K ) = W( K+1, K )
              A( K+1, K+1 ) = W( K+1, K+1 )
           END IF
        END IF
```

```
*
*        Store details of the interchanges in IPIV
*
         IF( KSTEP.EQ.1 ) THEN
            IPIV( K ) = KP
         ELSE
            IPIV( K ) = -P
            IPIV( K+1 ) = -KP
         END IF
*
*        Increase K and return to the start of the main loop
*
         K = K + KSTEP
         GO TO 70
*
   90    CONTINUE
*
*        Update the lower triangle of A22 (= A(k:n,k:n)) as
*
*        A22 := A22 - L21*D*L21**T = A22 - L21*W**T
*
*        computing blocks of NB columns at a time
*
         DO 110 J = K, N, NB
            JB = MIN( NB, N-J+1 )
*
*           Update the lower triangle of the diagonal block
*
            DO 100 JJ = J, J + JB - 1
               CALL SGEMV( 'No transpose', J+JB-JJ, K-1, -ONE,
     $                     A( JJ, 1 ), LDA, W( JJ, 1 ), LDW, ONE,
     $                     A( JJ, JJ ), 1 )
  100       CONTINUE
*
*           Update the rectangular subdiagonal block
*
            IF( J+JB.LE.N )
     $         CALL SGEMM( 'No transpose', 'Transpose', N-J-JB+1, JB,
     $                     K-1, -ONE, A( J+JB, 1 ), LDA, W( J, 1 ), LDW,
     $                     ONE, A( J+JB, J ), LDA )
  110    CONTINUE
*
*        Put L21 in standard form by partially undoing the interchanges
*        in columns 1:k-1
*
         J = K - 1
  120    CONTINUE
*
            KSTEP = 1
            JP1 = 1
```

```
            JJ = J
            JP2 = IPIV( J )
            IF( JP2.LT.0 ) THEN
               JP2 = -JP2
               J = J - 1
               JP1 = -IPIV( J )
               KSTEP = 2
            END IF
*
            J = J - 1
            IF( JP2.NE.JJ .AND. J.GE.1 )
     $         CALL SSWAP( J, A( JP2, 1 ), LDA, A( JJ, 1 ), LDA )
            JJ = J + 1
            IF( JP1.NE.JJ .AND. KSTEP.EQ.2 )
     $         CALL SSWAP( J, A( JP1, 1 ), LDA, A( JJ, 1 ), LDA )
         IF( J.GE.1 )
     $      GO TO 120
*
*        Set KB to the number of columns factorized
*
         KB = K - 1
*
      END IF
      RETURN
*
*     End of SLASYF_ROOK
*
      END
```

———-

59.6.4 lisp code

— **slasyf_rook.lisp** —

```
(let* ((zero 0.0) (one 1.0) (eight 8.0) (sevten 17.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 8.0 8.0) eight)
           (type (single-float 17.0 17.0) sevten)
           (ignorable zero one eight sevten))
  (defun slasyf_rook (uplo n nb kb a lda ipiv w ldw info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array single-float (*)) w a)
             (type (f2cl-lib:integer4) info ldw lda kb nb n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
```

```
   (a single-float a-%data% a-%offset%)
   (w single-float w-%data% w-%offset%)
   (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
(prog ((absakk 0.0) (alpha 0.0) (colmax 0.0) (d11 0.0) (d12 0.0)
       (d21 0.0) (d22 0.0) (stemp 0.0) (r1 0.0) (rowmax 0.0) (t$ 0.0)
       (sfmin 0.0) (imax 0) (itemp 0) (j 0) (jb 0) (jj 0) (jmax 0)
       (jp1 0) (jp2 0) (k 0) (kk 0) (kw 0) (kkw 0) (kp 0) (kstep 0) (p 0)
       (ii 0) (done nil))
  (declare
   (type (single-float) absakk alpha colmax d11 d12 d21 d22 stemp r1
    rowmax t$ sfmin)
   (type (f2cl-lib:integer4) imax itemp j jb jj jmax jp1 jp2 k kk kw kkw
    kp kstep p ii)
   (type f2cl-lib:logical done))
  (setf info 0)
  (setf alpha (/ (+ one (f2cl-lib:fsqrt sevten)) eight))
  (setf sfmin (slamch "S"))
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)
    (tagbody
      (setf k n)
     label10
      (setf kw (f2cl-lib:int-sub (f2cl-lib:int-add nb k) n))
      (if (or
           (and (<= k (f2cl-lib:int-add (f2cl-lib:int-sub n nb) 1))
                (< nb n))
           (< k 1))
          (go label30))
      (setf kstep 1)
      (setf p k)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (scopy k
           (f2cl-lib:array-slice a-%data% single-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice w-%data% single-float (1 kw)
                                 ((1 ldw) (1 *)) w-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf k var-0)))
      (if (< k n)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10)
              (sgemv "No transpose" k (f2cl-lib:int-sub n k) (- one)
               (f2cl-lib:array-slice a-%data% single-float
```

```
                                (1 (f2cl-lib:int-add k 1))
                                ((1 lda) (1 *)) a-%offset%)
          lda
          (f2cl-lib:array-slice w-%data% single-float
                                (k (f2cl-lib:int-add kw 1))
                                ((1 ldw) (1 *)) w-%offset%)
          ldw one
          (f2cl-lib:array-slice w-%data% single-float (1 kw)
                                ((1 ldw) (1 *)) w-%offset%)
          1)
      (declare (ignore var-0 var-2 var-3 var-4 var-6 var-9 var-10))
      (when var-1 (setf k var-1))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldw var-7))
      (when var-8 (setf one var-8))))
(setf absakk
        (abs
         (f2cl-lib:fref w-%data% (k kw) ((1 ldw) (1 *))
                        w-%offset%)))
(cond
 ((> k 1)
  (setf imax
          (isamax (f2cl-lib:int-sub k 1)
           (f2cl-lib:array-slice w-%data% single-float (1 kw)
                                 ((1 ldw) (1 *)) w-%offset%)
           1))
  (setf colmax
          (abs
           (f2cl-lib:fref w-%data% (imax kw) ((1 ldw) (1 *))
                          w-%offset%))))
 (t (setf colmax zero)))
(cond
 ((= (max absakk colmax) zero)
  (if (= info 0)
      (setf info k))
  (setf kp k)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (scopy k
       (f2cl-lib:array-slice w-%data% single-float (1 kw)
                             ((1 ldw) (1 *)) w-%offset%)
       1
       (f2cl-lib:array-slice a-%data% single-float (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf k var-0))))
 (t
  (cond ((not (< absakk (* alpha colmax))) (setf kp k))
        (t
         (tagbody
```

```
 (setf done f2cl-lib:%false%)
label12
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (scopy imax
      (f2cl-lib:array-slice a-%data% single-float
                            (1 imax) ((1 lda) (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice w-%data% single-float
                            (1 (f2cl-lib:int-sub kw 1))
                            ((1 ldw) (1 *)) w-%offset%)
      1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf imax var-0)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (scopy (f2cl-lib:int-sub k imax)
      (f2cl-lib:array-slice a-%data% single-float
                            (imax
                             (f2cl-lib:int-add imax 1))
                            ((1 lda) (1 *)) a-%offset%)
      lda
      (f2cl-lib:array-slice w-%data% single-float
                            ((+ imax 1)
                             (f2cl-lib:int-sub kw 1))
                            ((1 ldw) (1 *)) w-%offset%)
      1)
   (declare (ignore var-0 var-1 var-3 var-4))
   (when var-2 (setf lda var-2)))
 (if (< k n)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
          var-8 var-9 var-10)
         (sgemv "No transpose" k (f2cl-lib:int-sub n k)
          (- one)
          (f2cl-lib:array-slice a-%data% single-float
                                (1
                                 (f2cl-lib:int-add k 1))
                                ((1 lda) (1 *))
                                a-%offset%)
          lda
          (f2cl-lib:array-slice w-%data% single-float
                                (imax
                                 (f2cl-lib:int-add kw 1))
                                ((1 ldw) (1 *))
                                w-%offset%)
          ldw one
          (f2cl-lib:array-slice w-%data% single-float
                                (1
                                 (f2cl-lib:int-sub kw 1))
                                ((1 ldw) (1 *))
```

```
                                w-%offset%)
         1)
      (declare
       (ignore var-0 var-2 var-3 var-4 var-6 var-9
        var-10))
      (when var-1 (setf k var-1))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldw var-7))
      (when var-8 (setf one var-8))))
(cond
 ((/= imax k)
  (setf jmax
          (f2cl-lib:int-add imax
                            (isamax
                             (f2cl-lib:int-sub k imax)
                             (f2cl-lib:array-slice
                              w-%data% single-float
                              ((+ imax 1)
                               (f2cl-lib:int-sub kw 1))
                              ((1 ldw) (1 *))
                              w-%offset%)
                             1)))
  (setf rowmax
          (abs
           (f2cl-lib:fref w-%data%
                          (jmax (f2cl-lib:int-sub kw 1))
                          ((1 ldw) (1 *)) w-%offset%))))
 (t (setf rowmax zero)))
(cond
 ((> imax 1)
  (setf itemp
          (isamax (f2cl-lib:int-sub imax 1)
           (f2cl-lib:array-slice w-%data% single-float
                                 (1
                                  (f2cl-lib:int-sub kw
                                                    1))
                                 ((1 ldw) (1 *))
                                 w-%offset%)
           1))
  (setf stemp
          (abs
           (f2cl-lib:fref w-%data%
                          (itemp
                           (f2cl-lib:int-sub kw 1))
                          ((1 ldw) (1 *)) w-%offset%)))
  (cond
   ((> stemp rowmax) (setf rowmax stemp)
    (setf jmax itemp)))))
(cond
 ((not
```

```
                 (<
                  (abs
                   (f2cl-lib:fref w
                                  (imax
                                   (f2cl-lib:int-add kw
                                                     (f2cl-lib:int-sub
                                                      1)))
                                  ((1 ldw) (1 *))))
                  (* alpha rowmax)))
                (setf kp imax)
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (scopy k
                     (f2cl-lib:array-slice w-%data% single-float
                                           (1 (f2cl-lib:int-sub kw 1))
                                           ((1 ldw) (1 *)) w-%offset%)
                     1
                     (f2cl-lib:array-slice w-%data% single-float
                                           (1 kw) ((1 ldw) (1 *))
                                           w-%offset%)
                     1)
                  (declare (ignore var-1 var-2 var-3 var-4))
                  (when var-0 (setf k var-0)))
                (setf done f2cl-lib:%true%))
               ((or (= p jmax) (<= rowmax colmax)) (setf kp imax)
                (setf kstep 2) (setf done f2cl-lib:%true%))
               (t (setf p imax) (setf colmax rowmax) (setf imax jmax)
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (scopy k
                     (f2cl-lib:array-slice w-%data% single-float
                                           (1 (f2cl-lib:int-sub kw 1))
                                           ((1 ldw) (1 *)) w-%offset%)
                     1
                     (f2cl-lib:array-slice w-%data% single-float
                                           (1 kw) ((1 ldw) (1 *))
                                           w-%offset%)
                     1)
                  (declare (ignore var-1 var-2 var-3 var-4))
                  (when var-0 (setf k var-0)))))
              (if (not done)
                  (go label12)))))
(setf kk (f2cl-lib:int-add (f2cl-lib:int-sub k kstep) 1))
(setf kkw (f2cl-lib:int-sub (f2cl-lib:int-add nb kk) n))
(cond
 ((and (= kstep 2) (/= p k))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (scopy (f2cl-lib:int-sub k p)
       (f2cl-lib:array-slice a-%data% single-float ((+ p 1) k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% single-float
```

```
                                (p (f2cl-lib:int-add p 1))
                                ((1 lda) (1 *)) a-%offset%)
          lda)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf lda var-4)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (scopy p
           (f2cl-lib:array-slice a-%data% single-float (1 k)
                                ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% single-float (1 p)
                                ((1 lda) (1 *)) a-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf p var-0)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (sswap (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
           (f2cl-lib:array-slice a-%data% single-float (k k)
                                ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice a-%data% single-float (p k)
                                ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-3))
        (when var-2 (setf lda var-2))
        (when var-4 (setf lda var-4)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (sswap (f2cl-lib:int-add (f2cl-lib:int-sub n kk) 1)
           (f2cl-lib:array-slice w-%data% single-float (k kkw)
                                ((1 ldw) (1 *)) w-%offset%)
           ldw
           (f2cl-lib:array-slice w-%data% single-float (p kkw)
                                ((1 ldw) (1 *)) w-%offset%)
           ldw)
        (declare (ignore var-0 var-1 var-3))
        (when var-2 (setf ldw var-2))
        (when var-4 (setf ldw var-4)))))
    (cond
     ((/= kp kk)
      (setf (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                           a-%offset%)
            (f2cl-lib:fref a-%data% (kk k) ((1 lda) (1 *))
                             a-%offset%))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (scopy (f2cl-lib:int-sub k 1 kp)
           (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) kk)
                                ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% single-float
                                (kp (f2cl-lib:int-add kp 1))
```

```
                              ((1 lda) (1 *)) a-%offset%)
        lda)
      (declare (ignore var-0 var-1 var-2 var-3))
      (when var-4 (setf lda var-4)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy kp
         (f2cl-lib:array-slice a-%data% single-float (1 kk)
                              ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% single-float (1 kp)
                              ((1 lda) (1 *)) a-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf kp var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (sswap (f2cl-lib:int-add (f2cl-lib:int-sub n kk) 1)
         (f2cl-lib:array-slice a-%data% single-float (kk kk)
                              ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice a-%data% single-float (kp kk)
                              ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-0 var-1 var-3))
      (when var-2 (setf lda var-2))
      (when var-4 (setf lda var-4)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (sswap (f2cl-lib:int-add (f2cl-lib:int-sub n kk) 1)
         (f2cl-lib:array-slice w-%data% single-float (kk kkw)
                              ((1 ldw) (1 *)) w-%offset%)
         ldw
         (f2cl-lib:array-slice w-%data% single-float (kp kkw)
                              ((1 ldw) (1 *)) w-%offset%)
         ldw)
      (declare (ignore var-0 var-1 var-3))
      (when var-2 (setf ldw var-2))
      (when var-4 (setf ldw var-4)))))
  (cond
   ((= kstep 1)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy k
         (f2cl-lib:array-slice w-%data% single-float (1 kw)
                              ((1 ldw) (1 *)) w-%offset%)
         1
         (f2cl-lib:array-slice a-%data% single-float (1 k)
                              ((1 lda) (1 *)) a-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf k var-0)))
    (cond
     ((> k 1)
```

```
      (cond
       ((>= (abs (f2cl-lib:fref a (k k) ((1 lda) (1 *)))) sfmin)
        (setf r1
                (/ one
                   (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                  a-%offset%)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal (f2cl-lib:int-sub k 1) r1
             (f2cl-lib:array-slice a-%data% single-float (1 k)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-0 var-2 var-3))
          (when var-1 (setf r1 var-1))))
       ((/= (f2cl-lib:fref a (k k) ((1 lda) (1 *))) zero)
        (f2cl-lib:fdo (ii 1 (f2cl-lib:int-add ii 1))
                      ((> ii
                          (f2cl-lib:int-add k
                                            (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
                                 a-%offset%)
                  (/
                   (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
                                  a-%offset%)
                   (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                  a-%offset%)))
         label14)))))))
    (t
     (cond
      ((> k 2)
       (setf d12
               (f2cl-lib:fref w-%data% ((f2cl-lib:int-sub k 1) kw)
                              ((1 ldw) (1 *)) w-%offset%))
       (setf d11
               (/
                (f2cl-lib:fref w-%data% (k kw) ((1 ldw) (1 *))
                               w-%offset%)
                d12))
       (setf d22
               (/
                (f2cl-lib:fref w-%data%
                               ((f2cl-lib:int-sub k 1)
                                (f2cl-lib:int-sub kw 1))
                               ((1 ldw) (1 *)) w-%offset%)
                d12))
       (setf t$ (/ one (- (* d11 d22) one)))
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j
                         (f2cl-lib:int-add k (f2cl-lib:int-sub 2)))
```

```
                    nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-sub k 1))
                                 ((1 lda) (1 *)) a-%offset%)
                  (* t$
                     (/
                      (-
                       (* d11
                          (f2cl-lib:fref w-%data%
                                         (j
                                          (f2cl-lib:int-sub kw 1))
                                         ((1 ldw) (1 *))
                                         w-%offset%))
                       (f2cl-lib:fref w-%data% (j kw)
                                      ((1 ldw) (1 *)) w-%offset%))
                      d12)))
            (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                 a-%offset%)
                  (* t$
                     (/
                      (-
                       (* d22
                          (f2cl-lib:fref w-%data% (j kw)
                                         ((1 ldw) (1 *))
                                         w-%offset%))
                       (f2cl-lib:fref w-%data%
                                      (j (f2cl-lib:int-sub kw 1))
                                      ((1 ldw) (1 *)) w-%offset%))
                      d12)))
          label20))))
      (setf (f2cl-lib:fref a-%data%
                           ((f2cl-lib:int-sub k 1)
                            (f2cl-lib:int-sub k 1))
                           ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref w-%data%
                           ((f2cl-lib:int-sub k 1)
                            (f2cl-lib:int-sub kw 1))
                           ((1 ldw) (1 *)) w-%offset%))
      (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                           ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref w-%data% ((f2cl-lib:int-sub k 1) kw)
                           ((1 ldw) (1 *)) w-%offset%))
      (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref w-%data% (k kw) ((1 ldw) (1 *))
                           w-%offset%))))))
  (cond
   ((= kstep 1)
    (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%) kp))
   (t
    (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
```

```
            (f2cl-lib:int-sub p))
      (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                           ipiv-%offset%)
            (f2cl-lib:int-sub kp))))
    (setf k (f2cl-lib:int-sub k kstep))
    (go label10)
   label30
    (f2cl-lib:fdo (j
                   (+
                    (*
                     (the f2cl-lib:integer4 (truncate (+ k (- 1)) nb))
                     nb)
                    1)
                   (f2cl-lib:int-add j (f2cl-lib:int-sub nb)))
                  ((> j 1) nil)
      (tagbody
        (setf jb
                (min (the f2cl-lib:integer4 nb)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-sub k j)
                                            1))))
        (f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
                      ((> jj
                          (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10)
                (sgemv "No transpose"
                 (f2cl-lib:int-add (f2cl-lib:int-sub jj j) 1)
                 (f2cl-lib:int-sub n k) (- one)
                 (f2cl-lib:array-slice a-%data% single-float
                                       (j (f2cl-lib:int-add k 1))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice w-%data% single-float
                                       (jj (f2cl-lib:int-add kw 1))
                                       ((1 ldw) (1 *)) w-%offset%)
                 ldw one
                 (f2cl-lib:array-slice a-%data% single-float (j jj)
                                       ((1 lda) (1 *)) a-%offset%)
                 1)
                (declare
                 (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9
                  var-10))
                (when var-5 (setf lda var-5))
                (when var-7 (setf ldw var-7))
                (when var-8 (setf one var-8)))
          label40))
```

```
        (if (>= j 2)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10 var-11 var-12)
                (sgemm "No transpose" "Transpose"
                 (f2cl-lib:int-sub j 1) jb (f2cl-lib:int-sub n k)
                 (- one)
                 (f2cl-lib:array-slice a-%data% single-float
                                       (1 (f2cl-lib:int-add k 1))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice w-%data% single-float
                                       (j (f2cl-lib:int-add kw 1))
                                       ((1 ldw) (1 *)) w-%offset%)
                 ldw one
                 (f2cl-lib:array-slice a-%data% single-float (1 j)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare
               (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-8
                var-11))
              (when var-3 (setf jb var-3))
              (when var-7 (setf lda var-7))
              (when var-9 (setf ldw var-9))
              (when var-10 (setf one var-10))
              (when var-12 (setf lda var-12))))
    label50))
 (setf j (f2cl-lib:int-add k 1))
label60
 (setf kstep 1)
 (setf jp1 1)
 (setf jj j)
 (setf jp2 (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
 (cond
  ((< jp2 0) (setf jp2 (f2cl-lib:int-sub jp2))
   (setf j (f2cl-lib:int-add j 1))
   (setf jp1
           (f2cl-lib:int-sub
            (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)))
   (setf kstep 2)))
 (setf j (f2cl-lib:int-add j 1))
 (if (and (/= jp2 jj) (<= j n))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (sswap (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1)
          (f2cl-lib:array-slice a-%data% single-float (jp2 j)
                                ((1 lda) (1 *)) a-%offset%)
          lda
          (f2cl-lib:array-slice a-%data% single-float (jj j)
                                ((1 lda) (1 *)) a-%offset%)
          lda)
```

```
          (declare (ignore var-0 var-1 var-3))
          (when var-2 (setf lda var-2))
          (when var-4 (setf lda var-4))))
    (setf jj (f2cl-lib:int-sub j 1))
    (if (and (/= jp1 jj) (= kstep 2))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (sswap (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1)
             (f2cl-lib:array-slice a-%data% single-float (jp1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice a-%data% single-float (jj j)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-1 var-3))
          (when var-2 (setf lda var-2))
          (when var-4 (setf lda var-4))))
    (if (<= j n)
        (go label60))
    (setf kb (f2cl-lib:int-sub n k))))
(t
 (tagbody
   (setf k 1)
  label70
   (if (or (and (>= k nb) (< nb n)) (> k n))
       (go label90))
   (setf kstep 1)
   (setf p k)
   (scopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
    (f2cl-lib:array-slice a-%data% single-float (k k) ((1 lda) (1 *))
                          a-%offset%)
    1
    (f2cl-lib:array-slice w-%data% single-float (k k) ((1 ldw) (1 *))
                          w-%offset%)
    1)
   (if (> k 1)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
            var-9 var-10)
           (sgemv "No transpose"
            (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
            (f2cl-lib:int-sub k 1) (- one)
            (f2cl-lib:array-slice a-%data% single-float (k 1)
                                  ((1 lda) (1 *)) a-%offset%)
            lda
            (f2cl-lib:array-slice w-%data% single-float (k 1)
                                  ((1 ldw) (1 *)) w-%offset%)
            ldw one
            (f2cl-lib:array-slice w-%data% single-float (k k)
                                  ((1 ldw) (1 *)) w-%offset%)
            1)
```

```
      (declare
       (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9 var-10))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldw var-7))
      (when var-8 (setf one var-8))))
(setf absakk
        (abs
         (f2cl-lib:fref w-%data% (k k) ((1 ldw) (1 *))
                        w-%offset%)))
(cond
 ((< k n)
  (setf imax
          (f2cl-lib:int-add k
                            (isamax (f2cl-lib:int-sub n k)
                             (f2cl-lib:array-slice w-%data%
                                                   single-float
                                                   ((+ k 1) k)
                                                   ((1 ldw) (1 *))
                                                   w-%offset%)
                             1)))
  (setf colmax
          (abs
           (f2cl-lib:fref w-%data% (imax k) ((1 ldw) (1 *))
                          w-%offset%))))
 (t (setf colmax zero)))
(cond
 ((= (max absakk colmax) zero)
  (if (= info 0)
      (setf info k))
  (setf kp k)
  (scopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
   (f2cl-lib:array-slice w-%data% single-float (k k)
                         ((1 ldw) (1 *)) w-%offset%)
   1
   (f2cl-lib:array-slice a-%data% single-float (k k)
                         ((1 lda) (1 *)) a-%offset%)
   1))
 (t
  (cond ((not (< absakk (* alpha colmax))) (setf kp k))
        (t
         (tagbody
           (setf done f2cl-lib:%false%)
          label72
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
               (scopy (f2cl-lib:int-sub imax k)
                (f2cl-lib:array-slice a-%data% single-float
                                      (imax k) ((1 lda) (1 *))
                                      a-%offset%)
                lda
                (f2cl-lib:array-slice w-%data% single-float
```

```
                            (k (f2cl-lib:int-add k 1))
                            ((1 ldw) (1 *)) w-%offset%)
     1)
  (declare (ignore var-0 var-1 var-3 var-4))
  (when var-2 (setf lda var-2)))
(scopy (f2cl-lib:int-add (f2cl-lib:int-sub n imax) 1)
 (f2cl-lib:array-slice a-%data% single-float (imax imax)
                       ((1 lda) (1 *)) a-%offset%)
 1
 (f2cl-lib:array-slice w-%data% single-float
                       (imax (f2cl-lib:int-add k 1))
                       ((1 ldw) (1 *)) w-%offset%)
 1)
(if (> k 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
         var-8 var-9 var-10)
        (sgemv "No transpose"
         (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
         (f2cl-lib:int-sub k 1) (- one)
         (f2cl-lib:array-slice a-%data% single-float
                               (k 1) ((1 lda) (1 *))
                               a-%offset%)
         lda
         (f2cl-lib:array-slice w-%data% single-float
                               (imax 1) ((1 ldw) (1 *))
                               w-%offset%)
         ldw one
         (f2cl-lib:array-slice w-%data% single-float
                               (k
                                (f2cl-lib:int-add k 1))
                               ((1 ldw) (1 *))
                               w-%offset%)
         1)
      (declare
       (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9
        var-10))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldw var-7))
      (when var-8 (setf one var-8))))
(cond
 ((/= imax k)
  (setf jmax
          (f2cl-lib:int-add (f2cl-lib:int-sub k 1)
                            (isamax
                             (f2cl-lib:int-sub imax k)
                             (f2cl-lib:array-slice
                              w-%data% single-float
                              (k (f2cl-lib:int-add k 1))
                              ((1 ldw) (1 *))
```

```
                                w-%offset%)
                               1)))
  (setf rowmax
          (abs
           (f2cl-lib:fref w-%data%
                          (jmax (f2cl-lib:int-add k 1))
                          ((1 ldw) (1 *)) w-%offset%))))
 (t (setf rowmax zero)))
(cond
 ((< imax n)
  (setf itemp
          (f2cl-lib:int-add imax
                            (isamax
                             (f2cl-lib:int-sub n imax)
                             (f2cl-lib:array-slice
                              w-%data% single-float
                              ((+ imax 1)
                               (f2cl-lib:int-add k 1))
                              ((1 ldw) (1 *))
                              w-%offset%)
                             1)))
  (setf stemp
          (abs
           (f2cl-lib:fref w-%data%
                          (itemp (f2cl-lib:int-add k 1))
                          ((1 ldw) (1 *)) w-%offset%)))
  (cond
   ((> stemp rowmax) (setf rowmax stemp)
    (setf jmax itemp)))))
(cond
 ((not
   (<
    (abs
     (f2cl-lib:fref w (imax (f2cl-lib:int-add k 1))
                    ((1 ldw) (1 *))))
    (* alpha rowmax)))
  (setf kp imax)
  (scopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
   (f2cl-lib:array-slice w-%data% single-float
                         (k (f2cl-lib:int-add k 1))
                         ((1 ldw) (1 *)) w-%offset%)
   1
   (f2cl-lib:array-slice w-%data% single-float (k k)
                         ((1 ldw) (1 *)) w-%offset%)
   1)
  (setf done f2cl-lib:%true%))
 ((or (= p jmax) (<= rowmax colmax)) (setf kp imax)
  (setf kstep 2) (setf done f2cl-lib:%true%))
 (t (setf p imax) (setf colmax rowmax) (setf imax jmax)
  (scopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
```

```
            (f2cl-lib:array-slice w-%data% single-float
                                  (k (f2cl-lib:int-add k 1))
                                  ((1 ldw) (1 *)) w-%offset%)
            1
            (f2cl-lib:array-slice w-%data% single-float (k k)
                                  ((1 ldw) (1 *)) w-%offset%)
            1)))
         (if (not done)
             (go label72)))))
(setf kk (f2cl-lib:int-sub (f2cl-lib:int-add k kstep) 1))
(cond
 ((and (= kstep 2) (/= p k))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (scopy (f2cl-lib:int-sub p k)
       (f2cl-lib:array-slice a-%data% single-float (k k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% single-float (p k)
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (scopy (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
   (f2cl-lib:array-slice a-%data% single-float (p k)
                         ((1 lda) (1 *)) a-%offset%)
   1
   (f2cl-lib:array-slice a-%data% single-float (p p)
                         ((1 lda) (1 *)) a-%offset%)
   1)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (sswap k
       (f2cl-lib:array-slice a-%data% single-float (k 1)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice a-%data% single-float (p 1)
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-1 var-3))
    (when var-0 (setf k var-0))
    (when var-2 (setf lda var-2))
    (when var-4 (setf lda var-4)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (sswap kk
       (f2cl-lib:array-slice w-%data% single-float (k 1)
                             ((1 ldw) (1 *)) w-%offset%)
       ldw
       (f2cl-lib:array-slice w-%data% single-float (p 1)
                             ((1 ldw) (1 *)) w-%offset%)
       ldw)
    (declare (ignore var-1 var-3))
```

```
      (when var-0 (setf kk var-0))
      (when var-2 (setf ldw var-2))
      (when var-4 (setf ldw var-4)))))
  (cond
   ((/= kp kk)
    (setf (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                         a-%offset%)
          (f2cl-lib:fref a-%data% (kk k) ((1 lda) (1 *))
                         a-%offset%))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy (f2cl-lib:int-sub kp k 1)
         (f2cl-lib:array-slice a-%data% single-float ((+ k 1) kk)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% single-float
                               (kp (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-0 var-1 var-2 var-3))
      (when var-4 (setf lda var-4)))
    (scopy (f2cl-lib:int-add (f2cl-lib:int-sub n kp) 1)
     (f2cl-lib:array-slice a-%data% single-float (kp kk)
                           ((1 lda) (1 *)) a-%offset%)
     1
     (f2cl-lib:array-slice a-%data% single-float (kp kp)
                           ((1 lda) (1 *)) a-%offset%)
     1)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (sswap kk
         (f2cl-lib:array-slice a-%data% single-float (kk 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice a-%data% single-float (kp 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-1 var-3))
      (when var-0 (setf kk var-0))
      (when var-2 (setf lda var-2))
      (when var-4 (setf lda var-4)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (sswap kk
         (f2cl-lib:array-slice w-%data% single-float (kk 1)
                               ((1 ldw) (1 *)) w-%offset%)
         ldw
         (f2cl-lib:array-slice w-%data% single-float (kp 1)
                               ((1 ldw) (1 *)) w-%offset%)
         ldw)
      (declare (ignore var-1 var-3))
      (when var-0 (setf kk var-0))
      (when var-2 (setf ldw var-2))
```

```
    (when var-4 (setf ldw var-4)))))
(cond
 ((= kstep 1)
  (scopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
   (f2cl-lib:array-slice w-%data% single-float (k k)
                         ((1 ldw) (1 *)) w-%offset%)
   1
   (f2cl-lib:array-slice a-%data% single-float (k k)
                         ((1 lda) (1 *)) a-%offset%)
   1)
  (cond
   ((< k n)
    (cond
     ((>= (abs (f2cl-lib:fref a (k k) ((1 lda) (1 *)))) sfmin)
      (setf r1
              (/ one
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal (f2cl-lib:int-sub n k) r1
           (f2cl-lib:array-slice a-%data% single-float
                                 ((+ k 1) k) ((1 lda) (1 *))
                                 a-%offset%)
           1)
        (declare (ignore var-0 var-2 var-3))
        (when var-1 (setf r1 var-1))))
     ((/= (f2cl-lib:fref a (k k) ((1 lda) (1 *))) zero)
      (f2cl-lib:fdo (ii (f2cl-lib:int-add k 1)
                     (f2cl-lib:int-add ii 1))
                    ((> ii n) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
                               a-%offset%)
                  (/
                   (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
                                  a-%offset%)
                   (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                  a-%offset%)))
         label74)))))))
 (t
  (cond
   ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
    (setf d21
            (f2cl-lib:fref w-%data% ((f2cl-lib:int-add k 1) k)
                           ((1 ldw) (1 *)) w-%offset%))
    (setf d11
            (/
             (f2cl-lib:fref w-%data%
                            ((f2cl-lib:int-add k 1)
                             (f2cl-lib:int-add k 1))
```

```
                          ((1 ldw) (1 *)) w-%offset%)
          d21))
  (setf d22
        (/
         (f2cl-lib:fref w-%data% (k k) ((1 ldw) (1 *))
                        w-%offset%)
         d21))
  (setf t$ (/ one (- (* d11 d22) one)))
  (f2cl-lib:fdo (j (f2cl-lib:int-add k 2)
                 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                           a-%offset%)
            (* t$
               (/
                (-
                 (* d11
                    (f2cl-lib:fref w-%data% (j k)
                                   ((1 ldw) (1 *))
                                   w-%offset%))
                 (f2cl-lib:fref w-%data%
                                (j (f2cl-lib:int-add k 1))
                                ((1 ldw) (1 *)) w-%offset%))
                d21)))
      (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
            (* t$
               (/
                (-
                 (* d22
                    (f2cl-lib:fref w-%data%
                                   (j (f2cl-lib:int-add k 1))
                                   ((1 ldw) (1 *))
                                   w-%offset%))
                 (f2cl-lib:fref w-%data% (j k)
                                ((1 ldw) (1 *)) w-%offset%))
                d21)))
     label80))))
(setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (f2cl-lib:fref w-%data% (k k) ((1 ldw) (1 *))
                       w-%offset%))
(setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                      ((1 lda) (1 *)) a-%offset%)
        (f2cl-lib:fref w-%data% ((f2cl-lib:int-add k 1) k)
                       ((1 ldw) (1 *)) w-%offset%))
(setf (f2cl-lib:fref a-%data%
                      ((f2cl-lib:int-add k 1)
                       (f2cl-lib:int-add k 1))
                      ((1 lda) (1 *)) a-%offset%)
```

```
                (f2cl-lib:fref w-%data%
                               ((f2cl-lib:int-add k 1)
                                (f2cl-lib:int-add k 1))
                               ((1 ldw) (1 *)) w-%offset%))))))
  (cond
   ((= kstep 1)
    (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%) kp))
   (t
    (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
            (f2cl-lib:int-sub p))
    (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1)) ((1 *))
                         ipiv-%offset%)
            (f2cl-lib:int-sub kp))))
  (setf k (f2cl-lib:int-add k kstep))
  (go label70)
 label90
  (f2cl-lib:fdo (j k (f2cl-lib:int-add j nb))
                ((> j n) nil)
    (tagbody
      (setf jb
              (min (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub n j)
                                          1))))
      (f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
                    ((> jj
                        (f2cl-lib:int-add j jb (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10)
              (sgemv "No transpose"
               (f2cl-lib:int-sub (f2cl-lib:int-add j jb) jj)
               (f2cl-lib:int-sub k 1) (- one)
               (f2cl-lib:array-slice a-%data% single-float (jj 1)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice w-%data% single-float (jj 1)
                                     ((1 ldw) (1 *)) w-%offset%)
               ldw one
               (f2cl-lib:array-slice a-%data% single-float (jj jj)
                                     ((1 lda) (1 *)) a-%offset%)
               1)
             (declare
              (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9
               var-10))
             (when var-5 (setf lda var-5))
             (when var-7 (setf ldw var-7))
             (when var-8 (setf one var-8)))
```

```
          label100))
      (if (<= (f2cl-lib:int-add j jb) n)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10 var-11 var-12)
              (sgemm "No transpose" "Transpose"
               (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) jb
               (f2cl-lib:int-sub k 1) (- one)
               (f2cl-lib:array-slice a-%data% single-float
                                     ((+ j jb) 1) ((1 lda) (1 *))
                                     a-%offset%)
               lda
               (f2cl-lib:array-slice w-%data% single-float (j 1)
                                     ((1 ldw) (1 *)) w-%offset%)
               ldw one
               (f2cl-lib:array-slice a-%data% single-float
                                     ((+ j jb) j) ((1 lda) (1 *))
                                     a-%offset%)
               lda)
            (declare
             (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-8
              var-11))
            (when var-3 (setf jb var-3))
            (when var-7 (setf lda var-7))
            (when var-9 (setf ldw var-9))
            (when var-10 (setf one var-10))
            (when var-12 (setf lda var-12))))
    label110))
 (setf j (f2cl-lib:int-sub k 1))
label120
 (setf kstep 1)
 (setf jp1 1)
 (setf jj j)
 (setf jp2 (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
 (cond
  ((< jp2 0) (setf jp2 (f2cl-lib:int-sub jp2))
   (setf j (f2cl-lib:int-sub j 1))
   (setf jp1
           (f2cl-lib:int-sub
            (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)))
   (setf kstep 2)))
 (setf j (f2cl-lib:int-sub j 1))
 (if (and (/= jp2 jj) (>= j 1))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (sswap j
          (f2cl-lib:array-slice a-%data% single-float (jp2 1)
                                ((1 lda) (1 *)) a-%offset%)
          lda
          (f2cl-lib:array-slice a-%data% single-float (jj 1)
                                ((1 lda) (1 *)) a-%offset%)
```

```
                    lda)
                  (declare (ignore var-1 var-3))
                  (when var-0 (setf j var-0))
                  (when var-2 (setf lda var-2))
                  (when var-4 (setf lda var-4))))
            (setf jj (f2cl-lib:int-add j 1))
            (if (and (/= jp1 jj) (= kstep 2))
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (sswap j
                     (f2cl-lib:array-slice a-%data% single-float (jp1 1)
                                           ((1 lda) (1 *)) a-%offset%)
                     lda
                     (f2cl-lib:array-slice a-%data% single-float (jj 1)
                                           ((1 lda) (1 *)) a-%offset%)
                     lda)
                  (declare (ignore var-1 var-3))
                  (when var-0 (setf j var-0))
                  (when var-2 (setf lda var-2))
                  (when var-4 (setf lda var-4))))
            (if (>= j 1)
                (go label120))
            (setf kb (f2cl-lib:int-sub k 1)))))
        (go end_label)
       end_label
        (return (values uplo nil nil kb nil lda nil nil ldw info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slasyf_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::kb nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::ldw
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

59.7 ssycon

59.7.1 Axiom unit tests

— **ssycon.input** —

```
)set break resume
)sys rm -f ssycon.output
)spool ssycon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.7.2 Axiom help page

— **ssycon.help** —

```
====================================================================
ssycon examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssycon (UPLO, N, A, LDA, IPIV, ANORM, RCOND, WORK, IWORK,
           INFO)
           SSYCON

Function/Subroutine Documentation
   subroutine ssycon (character UPLO, integer N, real, dimension( lda, * ) A,
       integer LDA, integer, dimension( * ) IPIV, real ANORM, real RCOND,
```

```
      real, dimension( * ) WORK, integer, dimension( * ) IWORK, integer INFO)
      SSYCON

Purpose:
   SSYCON estimates the reciprocal of the condition number (in the
   1-norm) of a real symmetric matrix A using the factorization
   A = U*D*U**T or A = L*D*L**T computed by SSYTRF.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by SSYTRF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by SSYTRF.

 ANORM
   ANORM is REAL
   The 1-norm of the original matrix A.

 RCOND
   RCOND is REAL
   The reciprocal of the condition number of the matrix A,
   computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is an
   estimate of the 1-norm of inv(A) computed in this routine.

 WORK
   WORK is REAL array, dimension (2*N)
```

```
 IWORK
   IWORK is INTEGER array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

59.7.3 fortran code

— **ssycon.f** —

```
*  =====================================================================
      SUBROUTINE SSYCON( UPLO, N, A, LDA, IPIV, ANORM, RCOND, WORK,
     $                   IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
      REAL               ANORM, RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      REAL               A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, KASE
      REAL               AINVNM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
```

```
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLACN2, SSYTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYCON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.LE.ZERO ) THEN
         RETURN
      END IF
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
         DO 10 I = N, 1, -1
            IF( IPIV( I ).GT.0 .AND. A( I, I ).EQ.ZERO )
     $         RETURN
```

```
   10    CONTINUE
      ELSE
*
*        Lower triangular storage: examine D from top to bottom.
*
         DO 20 I = 1, N
            IF( IPIV( I ).GT.0 .AND. A( I, I ).EQ.ZERO )
     $         RETURN
   20    CONTINUE
      END IF
*
*     Estimate the 1-norm of the inverse.
*
      KASE = 0
   30 CONTINUE
      CALL SLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
*
*        Multiply by inv(L*D*L**T) or inv(U*D*U**T).
*
         CALL SSYTRS( UPLO, N, 1, A, LDA, IPIV, WORK, N, INFO )
         GO TO 30
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
      RETURN
*
*     End of SSYCON
*
      END
```

59.7.4 lisp code

— ssycon.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun ssycon (uplo n a lda ipiv anorm rcond work iwork info)
    (declare (type (single-float) rcond anorm)
             (type (array f2cl-lib:integer4 (*)) iwork ipiv)
```

```
          (type (array single-float (*)) work a)
          (type (f2cl-lib:integer4) info lda n)
          (type (string *) uplo))
(f2cl-lib:with-multi-array-data
    ((uplo character uplo-%data% uplo-%offset%)
     (a single-float a-%data% a-%offset%)
     (work single-float work-%data% work-%offset%)
     (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
     (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
         (ainvnm 0.0) (i 0) (kase 0) (upper nil))
    (declare (type (array f2cl-lib:integer4 (3)) isave)
             (type (single-float) ainvnm)
             (type (f2cl-lib:integer4) i kase)
             (type f2cl-lib:logical upper))
    (setf info 0)
    (setf upper
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
    (cond
     ((and (not upper)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
      (setf info -1))
     ((< n 0) (setf info -2))
     ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -4))
     ((< anorm zero) (setf info -6)))
    (cond
     ((/= info 0) (xerbla "SSYCON" (f2cl-lib:int-sub info))
      (go end_label)))
    (setf rcond zero)
    (cond ((= n 0) (setf rcond one) (go end_label))
          ((<= anorm zero) (go end_label)))
    (cond
     (upper
      (f2cl-lib:fdo (i n (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                    ((> i 1) nil)
        (tagbody
          (if (and
               (> (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) 0)
               (= (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                  zero))
```

```
                 (go end_label))
             label10)))
          (t
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (if (and
                    (> (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) 0)
                    (= (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                       zero))
                 (go end_label))
             label20))))
         (setf kase 0)
        label30
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
             (slacn2 n
              (f2cl-lib:array-slice work-%data% single-float ((+ n 1)) ((1 *))
                                    work-%offset%)
              work iwork ainvnm kase isave)
           (declare (ignore var-1 var-2 var-3 var-6))
           (setf n var-0)
           (setf ainvnm var-4)
           (setf kase var-5))
         (cond
          ((/= kase 0)
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
               (ssytrs uplo n 1 a lda ipiv work n info)
             (declare (ignore var-2 var-3 var-5 var-6))
             (when var-0 (setf uplo var-0))
             (when var-1 (setf n var-1))
             (when var-4 (setf lda var-4))
             (when var-7 (setf n var-7))
             (when var-8 (setf info var-8)))
           (go label30)))
         (if (/= ainvnm zero)
             (setf rcond (/ (/ one ainvnm) anorm)))
         (go end_label)
        end_label
         (return (values uplo n nil lda nil nil rcond nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssycon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
```

```
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (single-float) (single-float)
  (array single-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::uplo
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil
  fortran-to-lisp::rcond nil nil
  fortran-to-lisp::info)
:calls '(fortran-to-lisp::slacn2))))
```

59.8 ssycon_rook

59.8.1 Axiom unit tests

— ssyconrook.input —

```
)set break resume
)sys rm -f ssyconrook.output
)spool ssyconrook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

59.8.2 Axiom help page

— ssycon_rook.help —

```
======================================================================
ssycon_rook examples
```

```
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssycon_rook (UPLO, N, A, LDA, IPIV, ANORM, RCOND, WORK,
           IWORK, INFO)
           SSYCON_ROOK

Function/Subroutine Documentation
   subroutine ssycon_rook (character UPLO, integer N, real, dimension( lda, *
       ) A, integer LDA, integer, dimension( * ) IPIV, real ANORM, real RCOND,
       real, dimension( * ) WORK, integer, dimension( * ) IWORK, integer INFO)
       SSYCON_ROOK

Purpose:
   SSYCON_ROOK estimates the reciprocal of the condition number (in the
   1-norm) of a real symmetric matrix A using the factorization
   A = U*D*U**T or A = L*D*L**T computed by SSYTRF_ROOK.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by SSYTRF_ROOK.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
```

```
  Details of the interchanges and the block structure of D
  as determined by SSYTRF_ROOK.

ANORM
  ANORM is REAL
  The 1-norm of the original matrix A.

RCOND
  RCOND is REAL
  The reciprocal of the condition number of the matrix A,
  computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is an
  estimate of the 1-norm of inv(A) computed in this routine.

WORK
  WORK is REAL array, dimension (2*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

59.8.3 fortran code

— **ssycon_rook.f** —

```
*  =====================================================================
      SUBROUTINE SSYCON_ROOK( UPLO, N, A, LDA, IPIV, ANORM, RCOND, WORK,
     $                   IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
      REAL               ANORM, RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      REAL               A( LDA, * ), WORK( * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, KASE
      REAL               AINVNM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLACN2, SSYTRS_ROOK, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYCON_ROOK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
```

```
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.LE.ZERO ) THEN
         RETURN
      END IF
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
         DO 10 I = N, 1, -1
            IF( IPIV( I ).GT.0 .AND. A( I, I ).EQ.ZERO )
     $         RETURN
   10    CONTINUE
      ELSE
*
*        Lower triangular storage: examine D from top to bottom.
*
         DO 20 I = 1, N
            IF( IPIV( I ).GT.0 .AND. A( I, I ).EQ.ZERO )
     $         RETURN
   20    CONTINUE
      END IF
*
*     Estimate the 1-norm of the inverse.
*
      KASE = 0
   30 CONTINUE
      CALL SLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
*
*        Multiply by inv(L*D*L**T) or inv(U*D*U**T).
*
         CALL SSYTRS_ROOK( UPLO, N, 1, A, LDA, IPIV, WORK, N, INFO )
         GO TO 30
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
      RETURN
*
*     End of SSYCON_ROOK
*
      END
```

———-

59.8.4 lisp code

— ssycon_rook.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun ssycon_rook (uplo n a lda ipiv anorm rcond work iwork info)
    (declare (type (single-float) rcond anorm)
             (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array single-float (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (work single-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (ainvnm 0.0) (i 0) (kase 0) (upper nil))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (single-float) ainvnm)
                 (type (f2cl-lib:integer4) i kase)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -4))
         ((< anorm zero) (setf info -6)))
```

```
 (cond
  ((/= info 0) (xerbla "SSYCON_ROOK" (f2cl-lib:int-sub info))
   (go end_label)))
 (setf rcond zero)
 (cond ((= n 0) (setf rcond one) (go end_label))
       ((<= anorm zero) (go end_label)))
 (cond
  (upper
   (f2cl-lib:fdo (i n (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                 ((> i 1) nil)
     (tagbody
       (if (and
            (> (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) 0)
            (= (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
               zero))
           (go end_label))
      label10)))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (if (and
            (> (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) 0)
            (= (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
               zero))
           (go end_label))
      label20))))
 (setf kase 0)
label30
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slacn2 n
      (f2cl-lib:array-slice work-%data% single-float ((+ n 1)) ((1 *))
                            work-%offset%)
      work iwork ainvnm kase isave)
   (declare (ignore var-1 var-2 var-3 var-6))
   (setf n var-0)
   (setf ainvnm var-4)
   (setf kase var-5))
 (cond
  ((/= kase 0)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (ssytrs_rook uplo n 1 a lda ipiv work n info)
     (declare (ignore var-2 var-3 var-5 var-6))
     (when var-0 (setf uplo var-0))
     (when var-1 (setf n var-1))
     (when var-4 (setf lda var-4))
     (when var-7 (setf n var-7))
     (when var-8 (setf info var-8)))
   (go label30)))
```

```
        (if (/= ainvnm zero)
            (setf rcond (/ (/ one ainvnm) anorm)))
        (go end_label)
       end_label
        (return (values uplo n nil lda nil nil rcond nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssycon_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (single-float) (single-float)
                                              (array single-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::rcond nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::slacn2))))
```

———-

59.9 ssyconv

59.9.1 Axiom unit tests

— **ssyconv.input** —

```
)set break resume
)sys rm -f ssyconv.output
)spool ssyconv.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

59.9.2 Axiom help page

— **ssyconv.help** —

```
==================================================================
ssyconv examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssyconv (UPLO, WAY, N, A, LDA, IPIV, E, INFO)
           SSYCONV

Function/Subroutine Documentation
   subroutine ssyconv (character UPLO, character WAY, integer N, real,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV, real,
       dimension( * ) E, integer INFO)
       SSYCONV

Purpose:
   SSYCONV convert A given by TRF into L and D and vice-versa.
   Get Non-diag elements of D (returned in workspace) and
   apply or reverse permutation done in TRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 WAY
   WAY is CHARACTER*1
   = 'C': Convert
   = 'R': Revert
```

```
N
  N is INTEGER
  The order of the matrix A.  N >= 0.

A
  A is REAL array, dimension (LDA,N)
  The block diagonal matrix D and the multipliers used to
  obtain the factor U or L as computed by SSYTRF.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by SSYTRF.

E
  E is REAL array, dimension (N)
  E stores the supdiagonal/subdiagonal of the symmetric 1-by-1
  or 2-by-2 block diagonal matrix D in LDLT.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

59.9.3 fortran code

— **ssyconv.f** —

```
*  =====================================================================
      SUBROUTINE SSYCONV( UPLO, WAY, N, A, LDA, IPIV, E, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO, WAY
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
```

```
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     .. Local Scalars ..
      LOGICAL            UPPER, CONVERT
      INTEGER            I, IP, J
      REAL               TEMP
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      CONVERT = LSAME( WAY, 'C' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.CONVERT .AND. .NOT.LSAME( WAY, 'R' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5

      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYCONV', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*      A is UPPER
*
```

```
*      Convert A (A is upper)
*
*        Convert VALUE
*
         IF ( CONVERT ) THEN
            I=N
            E(1)=ZERO
            DO WHILE ( I .GT. 1 )
               IF( IPIV(I) .LT. 0 ) THEN
                  E(I)=A(I-1,I)
                  E(I-1)=ZERO
                  A(I-1,I)=ZERO
                  I=I-1
               ELSE
                  E(I)=ZERO
               ENDIF
               I=I-1
            END DO
*
*        Convert PERMUTATIONS
*
         I=N
         DO WHILE ( I .GE. 1 )
            IF( IPIV(I) .GT. 0) THEN
               IP=IPIV(I)
               IF( I .LT. N) THEN
                  DO 12 J= I+1,N
                    TEMP=A(IP,J)
                    A(IP,J)=A(I,J)
                    A(I,J)=TEMP
 12             CONTINUE
                ENDIF
            ELSE
              IP=-IPIV(I)
               IF( I .LT. N) THEN
             DO 13 J= I+1,N
                 TEMP=A(IP,J)
                 A(IP,J)=A(I-1,J)
                 A(I-1,J)=TEMP
 13             CONTINUE
                 ENDIF
                 I=I-1
           ENDIF
           I=I-1
        END DO

         ELSE
*
*      Revert A (A is upper)
*
```

```
*
*        Revert PERMUTATIONS
*
            I=1
            DO WHILE ( I .LE. N )
               IF( IPIV(I) .GT. 0 ) THEN
                  IP=IPIV(I)
                  IF( I .LT. N) THEN
                  DO J= I+1,N
                    TEMP=A(IP,J)
                    A(IP,J)=A(I,J)
                    A(I,J)=TEMP
                  END DO
                  ENDIF
               ELSE
                 IP=-IPIV(I)
                 I=I+1
                 IF( I .LT. N) THEN
                    DO J= I+1,N
                       TEMP=A(IP,J)
                       A(IP,J)=A(I-1,J)
                       A(I-1,J)=TEMP
                    END DO
                 ENDIF
               ENDIF
               I=I+1
            END DO
*
*        Revert VALUE
*
            I=N
            DO WHILE ( I .GT. 1 )
               IF( IPIV(I) .LT. 0 ) THEN
                  A(I-1,I)=E(I)
                  I=I-1
               ENDIF
               I=I-1
            END DO
         END IF
      ELSE
*
*      A is LOWER
*
         IF ( CONVERT ) THEN
*
*      Convert A (A is lower)
*
*
*        Convert VALUE
*
```

```
            I=1
            E(N)=ZERO
            DO WHILE ( I .LE. N )
               IF( I.LT.N .AND. IPIV(I) .LT. 0 ) THEN
                  E(I)=A(I+1,I)
                  E(I+1)=ZERO
                  A(I+1,I)=ZERO
                  I=I+1
               ELSE
                  E(I)=ZERO
               ENDIF
               I=I+1
            END DO
*
*        Convert PERMUTATIONS
*
         I=1
         DO WHILE ( I .LE. N )
            IF( IPIV(I) .GT. 0 ) THEN
               IP=IPIV(I)
               IF (I .GT. 1) THEN
               DO 22 J= 1,I-1
                 TEMP=A(IP,J)
                 A(IP,J)=A(I,J)
                 A(I,J)=TEMP
 22            CONTINUE
               ENDIF
            ELSE
              IP=-IPIV(I)
              IF (I .GT. 1) THEN
              DO 23 J= 1,I-1
                 TEMP=A(IP,J)
                 A(IP,J)=A(I+1,J)
                 A(I+1,J)=TEMP
 23           CONTINUE
              ENDIF
              I=I+1
           ENDIF
           I=I+1
        END DO
         ELSE
*
*      Revert A (A is lower)
*
*
*        Revert PERMUTATIONS
*
            I=N
            DO WHILE ( I .GE. 1 )
               IF( IPIV(I) .GT. 0 ) THEN
```

```
                IP=IPIV(I)
                IF (I .GT. 1) THEN
                   DO J= 1,I-1
                      TEMP=A(I,J)
                      A(I,J)=A(IP,J)
                      A(IP,J)=TEMP
                   END DO
                ENDIF
             ELSE
                IP=-IPIV(I)
                I=I-1
                IF (I .GT. 1) THEN
                   DO J= 1,I-1
                      TEMP=A(I+1,J)
                      A(I+1,J)=A(IP,J)
                      A(IP,J)=TEMP
                   END DO
                ENDIF
             ENDIF
             I=I-1
          END DO
*
*        Revert VALUE
*
          I=1
          DO WHILE ( I .LE. N-1 )
             IF( IPIV(I) .LT. 0 ) THEN
                A(I+1,I)=E(I)
                I=I+1
             ENDIF
             I=I+1
          END DO
       END IF
    END IF

    RETURN
*
*   End of SSYCONV
*
    END
```

———-

59.9.4 lisp code

— **ssyconv.lisp** —

```
(let* ((zero 0.0))
```

```
(declare (type (single-float 0.0 0.0) zero)
         (ignorable zero))
(defun ssyconv (uplo way n a lda ipiv e info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array single-float (*)) e a)
           (type (f2cl-lib:integer4) info lda n)
           (type (string *) way uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (way character way-%data% way-%offset%)
       (a single-float a-%data% a-%offset%)
       (e single-float e-%data% e-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((temp 0.0) (i 0) (ip 0) (j 0) (upper nil) (convert nil))
      (declare (type (single-float) temp)
               (type (f2cl-lib:integer4) i ip j)
               (type f2cl-lib:logical upper convert))
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (setf convert
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame way "C")
                (declare (ignore var-1))
                (when var-0 (setf way var-0))
                ret-val))
      (cond
       ((and (not upper)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val)))
        (setf info -1))
       ((and (not convert)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame way "R")
                (declare (ignore var-1))
                (when var-0 (setf way var-0))
                ret-val)))
        (setf info -2))
       ((< n 0) (setf info -3))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -5)))
```

```
(cond
 ((/= info 0) (xerbla "SSYCONV" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(cond
 (upper
  (cond
   (convert
    (tagbody
      (setf i n)
      (setf (f2cl-lib:fref e-%data% (1) ((1 *)) e-%offset%) zero)
     label100000
      (if (not (> i 1))
          (go label100001))
      (cond
       ((< (f2cl-lib:fref ipiv (i) ((1 *))) 0)
        (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) i)
                               ((1 lda) (1 *)) a-%offset%))
        (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1)) ((1 *))
                             e-%offset%)
                zero)
        (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) i)
                             ((1 lda) (1 *)) a-%offset%)
                zero)
        (setf i (f2cl-lib:int-sub i 1)))
       (t (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) zero)))
      (setf i (f2cl-lib:int-sub i 1))
      (go label100000)
     label100001
      (setf i n)
     label100002
      (if (not (>= i 1))
          (go label100003))
      (cond
       ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
        (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
        (cond
         ((< i n)
          (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                           (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf temp
                      (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                   a-%offset%)
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
```

```
                                    a-%offset%))
              (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%)
                      temp)
            label12)))))
      (t
       (setf ip
               (f2cl-lib:int-sub
                (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                               ipiv-%offset%)))
       (cond
        ((< i n)
         (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                          (f2cl-lib:int-add j 1))
                        ((> j n) nil)
           (tagbody
             (setf temp
                     (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                    a-%offset%))
             (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                  a-%offset%)
                     (f2cl-lib:fref a-%data%
                                    ((f2cl-lib:int-sub i 1) j)
                                    ((1 lda) (1 *)) a-%offset%))
             (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) j)
                                  ((1 lda) (1 *)) a-%offset%)
                     temp)
            label13))))
       (setf i (f2cl-lib:int-sub i 1))))
     (setf i (f2cl-lib:int-sub i 1))
     (go label100002)
    label100003))
  (t
   (tagbody
     (setf i 1)
    label100004
     (if (not (<= i n))
         (go label100005))
     (cond
      ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
       (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
       (cond
        ((< i n)
         (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                          (f2cl-lib:int-add j 1))
                        ((> j n) nil)
           (tagbody
             (setf temp
                     (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                    a-%offset%))
```

```
                (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                      a-%offset%)
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%))
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                        temp)
              label100006)))))
       (t
        (setf ip
                (f2cl-lib:int-sub
                 (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                                ipiv-%offset%)))
        (setf i (f2cl-lib:int-add i 1))
        (cond
         ((< i n)
          (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                         (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf temp
                        (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                        a-%offset%))
              (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                      a-%offset%)
                        (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-sub i 1) j)
                                        ((1 lda) (1 *)) a-%offset%))
              (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) j)
                                      ((1 lda) (1 *)) a-%offset%)
                        temp)
              label100007))))))
       (setf i (f2cl-lib:int-add i 1))
       (go label100004)
      label100005
       (setf i n)
      label100008
       (if (not (> i 1))
           (go label100009))
       (cond
        ((< (f2cl-lib:fref ipiv (i) ((1 *))) 0)
         (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) i)
                               ((1 lda) (1 *)) a-%offset%)
                 (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))
         (setf i (f2cl-lib:int-sub i 1))))
       (setf i (f2cl-lib:int-sub i 1))
       (go label100008)
      label100009))))
  (t
   (cond
```

```
(convert
 (tagbody
   (setf i 1)
   (setf (f2cl-lib:fref e-%data% (n) ((1 *)) e-%offset%) zero)
  label100010
   (if (not (<= i n))
       (go label100011))
   (cond
    ((and (< i n) (< (f2cl-lib:fref ipiv (i) ((1 *))) 0))
     (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
             (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                             ((1 lda) (1 *)) a-%offset%))
     (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                           e-%offset%)
             zero)
     (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                           ((1 lda) (1 *)) a-%offset%)
             zero)
     (setf i (f2cl-lib:int-add i 1)))
    (t (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) zero)))
   (setf i (f2cl-lib:int-add i 1))
   (go label100010)
  label100011
   (setf i 1)
  label100012
   (if (not (<= i n))
       (go label100013))
   (cond
    ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
     (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
     (cond
      ((> i 1)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j
                         (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      nil)
         (tagbody
           (setf temp
                   (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                  a-%offset%))
           (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                a-%offset%)
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%))
           (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                   temp)
          label22)))))
    (t
     (setf ip
```

```
                (f2cl-lib:int-sub
                 (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                                ipiv-%offset%)))
      (cond
       ((> i 1)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j
                          (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf temp
                    (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                   a-%offset%))
            (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                 a-%offset%)
                    (f2cl-lib:fref a-%data%
                                   ((f2cl-lib:int-add i 1) j)
                                   ((1 lda) (1 *)) a-%offset%))
            (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) j)
                                 ((1 lda) (1 *)) a-%offset%)
                    temp)
           label23))))
      (setf i (f2cl-lib:int-add i 1))))
    (setf i (f2cl-lib:int-add i 1))
    (go label100012)
   label100013))
  (t
   (tagbody
     (setf i n)
    label100014
     (if (not (>= i 1))
         (go label100015))
     (cond
      ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
       (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
       (cond
        ((> i 1)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j
                           (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                        nil)
           (tagbody
             (setf temp
                     (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%))
             (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                     (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                    a-%offset%))
             (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
```

```
                                        a-%offset%)
                                temp)
                        label100016)))))
                  (t
                   (setf ip
                           (f2cl-lib:int-sub
                            (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                                           ipiv-%offset%)))
                   (setf i (f2cl-lib:int-sub i 1))
                   (cond
                    ((> i 1)
                     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                                   ((> j
                                       (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                                    nil)
                       (tagbody
                         (setf temp
                                 (f2cl-lib:fref a-%data%
                                                ((f2cl-lib:int-add i 1) j)
                                                ((1 lda) (1 *)) a-%offset%))
                         (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) j)
                                              ((1 lda) (1 *)) a-%offset%)
                                 (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                                a-%offset%))
                         (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                              a-%offset%)
                                 temp)
                        label100017))))))
                 (setf i (f2cl-lib:int-sub i 1))
                 (go label100014)
                label100015
                 (setf i 1)
                label100018
                 (if (not (<= i (f2cl-lib:int-sub n 1)))
                     (go label100019))
                 (cond
                  ((< (f2cl-lib:fref ipiv (i) ((1 *))) 0)
                   (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                                        ((1 lda) (1 *)) a-%offset%)
                           (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))
                   (setf i (f2cl-lib:int-add i 1))))
                 (setf i (f2cl-lib:int-add i 1))
                 (go label100018)
                label100019)))))
          (go end_label)
         end_label
          (return (values uplo way nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
```

```
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssyconv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::way nil nil nil
                                              nil nil fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

59.10 ssyequb

59.10.1 Axiom unit tests

— **ssyequb.input** —

```
)set break resume
)sys rm -f ssyequb.output
)spool ssyequb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.10.2 Axiom help page

— **ssyequb.help** —

```
==================================================================
```

```
ssyequb examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssyequb (UPLO, N, A, LDA, S, SCOND, AMAX, WORK, INFO)
           SSYEQUB

Function/Subroutine Documentation
   subroutine ssyequb (character UPLO, integer N, real, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) S, real SCOND, real AMAX, real,
       dimension( * ) WORK, integer INFO)
       SSYEQUB

Purpose:
   SSYEQUB computes row and column scalings intended to equilibrate a
   symmetric matrix A and reduce its condition number
   (with respect to the two-norm).  S contains the scale factors,
   S(i) = 1/sqrt(A(i,i)), chosen so that the scaled matrix B with
   elements B(i,j) = S(i)*A(i,j)*S(j) has ones on the diagonal.  This
   choice of S puts the condition number of B within a factor N of the
   smallest possible condition number over all possible diagonal scalings.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   The N-by-N symmetric matrix whose scaling
   factors are to be computed.  Only the diagonal elements of A
    are referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).
```

```
 S
   S is REAL array, dimension (N)
   If INFO = 0, S contains the scale factors for A.

 SCOND
   SCOND is REAL
   If INFO = 0, S contains the ratio of the smallest S(i) to
   the largest S(i).  If SCOND >= 0.1 and AMAX is neither too
   large nor too small, it is not worth scaling by S.

 AMAX
   AMAX is REAL
   Absolute value of largest matrix element.  If AMAX is very
   close to overflow or very close to underflow, the matrix
   should be scaled.

 WORK
   WORK is REAL array, dimension (3*N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, the i-th diagonal element is nonpositive.
```

———-

59.10.3 fortran code

— **ssyequb.f** —

```
*  =====================================================================
      SUBROUTINE SSYEQUB( UPLO, N, A, LDA, S, SCOND, AMAX, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, N
      REAL               AMAX, SCOND
      CHARACTER          UPLO
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), S( * ), WORK( * )
*     ..
```

```
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
      INTEGER            MAX_ITER
      PARAMETER          ( MAX_ITER = 100 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, ITER
      REAL               AVG, STD, TOL, C0, C1, C2, T, U, SI, D, BASE,
     $                   SMIN, SMAX, SMLNUM, BIGNUM, SCALE, SUMSQ
      LOGICAL            UP
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      LOGICAL            LSAME
      EXTERNAL           LSAME, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLASSQ
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, INT, LOG, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test input parameters.
*
      INFO = 0
      IF ( .NOT. ( LSAME( UPLO, 'U' ) .OR. LSAME( UPLO, 'L' ) ) ) THEN
        INFO = -1
      ELSE IF ( N .LT. 0 ) THEN
        INFO = -2
      ELSE IF ( LDA .LT. MAX( 1, N ) ) THEN
        INFO = -4
      END IF
      IF ( INFO .NE. 0 ) THEN
        CALL XERBLA( 'SSYEQUB', -INFO )
        RETURN
      END IF

      UP = LSAME( UPLO, 'U' )
      AMAX = ZERO
*
*     Quick return if possible.
*
      IF ( N .EQ. 0 ) THEN
        SCOND = ONE
```

```
      RETURN
    END IF

    DO I = 1, N
      S( I ) = ZERO
    END DO

    AMAX = ZERO
    IF ( UP ) THEN
       DO J = 1, N
          DO I = 1, J-1
             S( I ) = MAX( S( I ), ABS( A( I, J ) ) )
             S( J ) = MAX( S( J ), ABS( A( I, J ) ) )
             AMAX = MAX( AMAX, ABS( A(I, J) ) )
          END DO
          S( J ) = MAX( S( J ), ABS( A( J, J ) ) )
          AMAX = MAX( AMAX, ABS( A( J, J ) ) )
       END DO
    ELSE
       DO J = 1, N
          S( J ) = MAX( S( J ), ABS( A( J, J ) ) )
          AMAX = MAX( AMAX, ABS( A( J, J ) ) )
          DO I = J+1, N
             S( I ) = MAX( S( I ), ABS( A( I, J ) ) )
             S( J ) = MAX( S( J ), ABS( A( I, J ) ) )
             AMAX = MAX( AMAX, ABS( A( I, J ) ) )
          END DO
       END DO
    END IF
    DO J = 1, N
       S( J ) = 1.0 / S( J )
    END DO

    TOL = ONE / SQRT(2.0E0 * N)

    DO ITER = 1, MAX_ITER
       SCALE = 0.0
       SUMSQ = 0.0
*     BETA = |A|S
      DO I = 1, N
         WORK(I) = ZERO
      END DO
      IF ( UP ) THEN
         DO J = 1, N
            DO I = 1, J-1
               T = ABS( A( I, J ) )
               WORK( I ) = WORK( I ) + ABS( A( I, J ) ) * S( J )
               WORK( J ) = WORK( J ) + ABS( A( I, J ) ) * S( I )
            END DO
            WORK( J ) = WORK( J ) + ABS( A( J, J ) ) * S( J )
```

```
         END DO
      ELSE
         DO J = 1, N
            WORK( J ) = WORK( J ) + ABS( A( J, J ) ) * S( J )
            DO I = J+1, N
               T = ABS( A( I, J ) )
               WORK( I ) = WORK( I ) + ABS( A( I, J ) ) * S( J )
               WORK( J ) = WORK( J ) + ABS( A( I, J ) ) * S( I )
            END DO
         END DO
      END IF

*     avg = s^T beta / n
      AVG = 0.0
      DO I = 1, N
        AVG = AVG + S( I )*WORK( I )
      END DO
      AVG = AVG / N

      STD = 0.0
      DO I = 2*N+1, 3*N
         WORK( I ) = S( I-2*N ) * WORK( I-2*N ) - AVG
      END DO
      CALL SLASSQ( N, WORK( 2*N+1 ), 1, SCALE, SUMSQ )
      STD = SCALE * SQRT( SUMSQ / N )

      IF ( STD .LT. TOL * AVG ) GOTO 999

      DO I = 1, N
        T = ABS( A( I, I ) )
        SI = S( I )
        C2 = ( N-1 ) * T
        C1 = ( N-2 ) * ( WORK( I ) - T*SI )
        C0 = -(T*SI)*SI + 2*WORK( I )*SI - N*AVG
        D = C1*C1 - 4*C0*C2

        IF ( D .LE. 0 ) THEN
          INFO = -1
          RETURN
        END IF
        SI = -2*C0 / ( C1 + SQRT( D ) )

        D = SI - S( I )
        U = ZERO
        IF ( UP ) THEN
          DO J = 1, I
            T = ABS( A( J, I ) )
            U = U + S( J )*T
            WORK( J ) = WORK( J ) + D*T
          END DO
```

```
          DO J = I+1,N
            T = ABS( A( I, J ) )
            U = U + S( J )*T
            WORK( J ) = WORK( J ) + D*T
          END DO
        ELSE
          DO J = 1, I
            T = ABS( A( I, J ) )
            U = U + S( J )*T
            WORK( J ) = WORK( J ) + D*T
          END DO
          DO J = I+1,N
            T = ABS( A( J, I ) )
            U = U + S( J )*T
            WORK( J ) = WORK( J ) + D*T
          END DO
        END IF

        AVG = AVG + ( U + WORK( I ) ) * D / N
        S( I ) = SI

      END DO

    END DO

 999  CONTINUE

      SMLNUM = SLAMCH( 'SAFEMIN' )
      BIGNUM = ONE / SMLNUM
      SMIN = BIGNUM
      SMAX = ZERO
      T = ONE / SQRT(AVG)
      BASE = SLAMCH( 'B' )
      U = ONE / LOG( BASE )
      DO I = 1, N
        S( I ) = BASE ** INT( U * LOG( S( I ) * T ) )
        SMIN = MIN( SMIN, S( I ) )
        SMAX = MAX( SMAX, S( I ) )
      END DO
      SCOND = MAX( SMIN, SMLNUM ) / MIN( SMAX, BIGNUM )
*
      END
```

———-

59.10.4 lisp code

— ssyequb.lisp —

```
(let* ((one 1.0) (zero 0.0) (max_iter 100))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (type (f2cl-lib:integer4 100 100) max_iter)
           (ignorable one zero max_iter))
  (defun ssyequb (uplo n a lda s scond amax work info)
    (declare (type (single-float) amax scond)
             (type (array single-float (*)) work s a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (s single-float s-%data% s-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((up nil) (avg 0.0) (std 0.0) (tol 0.0) (c0 0.0) (c1 0.0) (c2 0.0)
             (t$ 0.0) (u 0.0) (si 0.0) (d 0.0) (base 0.0) (smin 0.0) (smax 0.0)
             (smlnum 0.0) (bignum 0.0) (scale 0.0) (sumsq 0.0) (i 0) (j 0)
             (iter 0))
        (declare (type f2cl-lib:logical up)
                 (type (single-float) avg std tol c0 c1 c2 t$ u si d base smin
                  smax smlnum bignum scale sumsq)
                 (type (f2cl-lib:integer4) i j iter))
        (setf info 0)
        (cond
         ((not
           (or
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -4)))
        (cond
         ((/= info 0) (xerbla "SSYEQUB" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf up
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
```

```
(setf amax zero)
(cond ((= n 0) (setf scond one) (go end_label)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%) zero)
   label100000))
(setf amax zero)
(cond
 (up
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                  (max (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                       (abs
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))))
          (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                  (max (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                       (abs
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))))
          (setf amax
                  (max amax
                       (abs
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))))
         label100002))
      (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
              (max (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                   (abs
                    (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                   a-%offset%))))
      (setf amax
              (max amax
                   (abs
                    (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                   a-%offset%))))
     label100001)))
 (t
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
              (max (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
```

```
                    (abs
                     (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                    a-%offset%))))
        (setf amax
                (max amax
                     (abs
                      (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%))))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                    (max (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                         (abs
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%))))
            (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                    (max (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                         (abs
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%))))
            (setf amax
                    (max amax
                         (abs
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%))))
           label100004))
      label100003))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
            (/ 1.0 (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
   label100005))
(setf tol (/ one (f2cl-lib:fsqrt (* 2.0 n))))
(f2cl-lib:fdo (iter 1 (f2cl-lib:int-add iter 1))
              ((> iter max_iter) nil)
  (tagbody
    (setf scale 0.0)
    (setf sumsq 0.0)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                zero)
       label100007))
    (cond
     (up
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
```

```
          (tagbody
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i
                              (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                           nil)
              (tagbody
                (setf t$
                        (abs
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)))
                (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%)
                        (+
                         (f2cl-lib:fref work-%data% (i) ((1 *))
                                        work-%offset%)
                         (*
                          (abs
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%))
                          (f2cl-lib:fref s-%data% (j) ((1 *))
                                         s-%offset%))))
                (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                                     work-%offset%)
                        (+
                         (f2cl-lib:fref work-%data% (j) ((1 *))
                                        work-%offset%)
                         (*
                          (abs
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%))
                          (f2cl-lib:fref s-%data% (i) ((1 *))
                                         s-%offset%))))
              label100009))
            (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                    (+
                     (f2cl-lib:fref work-%data% (j) ((1 *))
                                    work-%offset%)
                     (*
                      (abs
                       (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                      a-%offset%))
                      (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))))
          label100008)))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                     (+
                      (f2cl-lib:fref work-%data% (j) ((1 *))
```

```
                                work-%offset%)
                  (*
                   (abs
                    (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                   a-%offset%))
                   (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                       (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf t$
                    (abs
                     (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%)))
            (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)
                    (+
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)
                     (*
                      (abs
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%))
                      (f2cl-lib:fref s-%data% (j) ((1 *))
                                      s-%offset%))))
            (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                                 work-%offset%)
                    (+
                     (f2cl-lib:fref work-%data% (j) ((1 *))
                                    work-%offset%)
                     (*
                      (abs
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%))
                      (f2cl-lib:fref s-%data% (i) ((1 *))
                                      s-%offset%))))
          label100011))
      label100010))))
(setf avg 0.0)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf avg
            (+ avg
               (* (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%))))
   label100012))
(setf avg (/ avg n))
(setf std 0.0)
```

```
(f2cl-lib:fdo (i (f2cl-lib:int-add (f2cl-lib:int-mul 2 n) 1)
               (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-mul 3 n)) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (-
             (*
              (f2cl-lib:fref s-%data%
                             ((f2cl-lib:int-add i
                                                (f2cl-lib:int-mul
                                                 -1 2 n)))
                             ((1 *)) s-%offset%)
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add i
                                                (f2cl-lib:int-mul
                                                 -1 2 n)))
                             ((1 *)) work-%offset%))
             avg))
   label100013))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (slassq n
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                           work-%offset%)
     1 scale sumsq)
  (declare (ignore var-0 var-1 var-2))
  (setf scale var-3)
  (setf sumsq var-4))
(setf std (* scale (f2cl-lib:fsqrt (/ sumsq n))))
(if (< std (* tol avg))
    (go label999))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf t$
            (abs
             (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                            a-%offset%)))
    (setf si (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%))
    (setf c2 (* (f2cl-lib:int-sub n 1) t$))
    (setf c1
            (* (f2cl-lib:int-sub n 2)
               (-
                (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                (* t$ si))))
    (setf c0
            (-
             (+ (* (- (* t$ si)) si)
                (* 2
```

```
                (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                si))
          (* n avg)))
(setf d (+ (* c1 c1) (* -4 c0 c2)))
(cond ((<= d 0) (setf info -1) (go end_label)))
(setf si (/ (* -2 c0) (+ c1 (f2cl-lib:fsqrt d))))
(setf d (- si (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
(setf u zero)
(cond
 (up
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j i) nil)
    (tagbody
      (setf t$
              (abs
               (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                              a-%offset%)))
      (setf u
              (+ u
                 (*
                  (f2cl-lib:fref s-%data% (j) ((1 *))
                                 s-%offset%)
                  t$)))
      (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                           work-%offset%)
              (+
               (f2cl-lib:fref work-%data% (j) ((1 *))
                              work-%offset%)
               (* d t$)))
     label100015))
  (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf t$
              (abs
               (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                              a-%offset%)))
      (setf u
              (+ u
                 (*
                  (f2cl-lib:fref s-%data% (j) ((1 *))
                                 s-%offset%)
                  t$)))
      (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                           work-%offset%)
              (+
               (f2cl-lib:fref work-%data% (j) ((1 *))
                              work-%offset%)
```

```
                 (* d t$)))
     label100016)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j i) nil)
     (tagbody
       (setf t$
               (abs
                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%)))
       (setf u
               (+ u
                  (*
                   (f2cl-lib:fref s-%data% (j) ((1 *))
                                  s-%offset%)
                   t$)))
       (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                            work-%offset%)
               (+
                (f2cl-lib:fref work-%data% (j) ((1 *))
                               work-%offset%)
                (* d t$)))
     label100017))
   (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                  (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf t$
               (abs
                (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                               a-%offset%)))
       (setf u
               (+ u
                  (*
                   (f2cl-lib:fref s-%data% (j) ((1 *))
                                  s-%offset%)
                   t$)))
       (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                            work-%offset%)
               (+
                (f2cl-lib:fref work-%data% (j) ((1 *))
                               work-%offset%)
                (* d t$)))
     label100018))))
(setf avg
        (+ avg
           (/
            (*
             (+ u
                (f2cl-lib:fref work-%data% (i) ((1 *))
```

```
                                                work-%offset%))
                                d)
                               n)))
                     (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%) si)
                    label100014))
                 label100006))
        label999
         (setf smlnum (slamch "SAFEMIN"))
         (setf bignum (/ one smlnum))
         (setf smin bignum)
         (setf smax zero)
         (setf t$ (/ one (f2cl-lib:fsqrt avg)))
         (setf base (slamch "B"))
         (setf u (/ one (f2cl-lib:flog base)))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                     (expt base
                           (f2cl-lib:int
                            (* u
                               (f2cl-lib:flog
                                (*
                                 (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                                 t$))))))
             (setf smin
                     (min smin (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
             (setf smax
                     (max smax (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
                 label100019))
         (setf scond (/ (max smin smlnum) (min smax bignum)))
        end_label
         (return (values uplo nil nil nil nil scond amax nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssyequb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil nil
```

```
                                                  nil fortran-to-lisp::scond
                                                  fortran-to-lisp::amax nil
                                                  fortran-to-lisp::info)
                                                :calls '(fortran-to-lisp::slassq))))
```

———-

59.11 ssygs2

59.11.1 Axiom unit tests

— **ssygs2.input** —

```
)set break resume
)sys rm -f ssygs2.output
)spool ssygs2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.11.2 Axiom help page

— **ssygs2.help** —

```
====================================================================
ssygs2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssygs2 (ITYPE, UPLO, N, A, LDA, B, LDB, INFO)
           SSYGS2 reduces a symmetric definite generalized eigenproblem to
           standard form, using the factorization results obtained from spotrf
```

```
          (unblocked algorithm).

Function/Subroutine Documentation
   subroutine ssygs2 (integer ITYPE, character UPLO, integer N, real,
       dimension( lda, * ) A, integer LDA, real, dimension( ldb, * ) B,
       integer LDB, integer INFO)
       SSYGS2 reduces a symmetric definite generalized eigenproblem to
       standard form, using the factorization results obtained from spotrf
       (unblocked algorithm).

Purpose:
   SSYGS2 reduces a real symmetric-definite generalized eigenproblem
   to standard form.

   If ITYPE = 1, the problem is A*x = lambda*B*x,
   and A is overwritten by inv(U**T)*A*inv(U) or inv(L)*A*inv(L**T)

   If ITYPE = 2 or 3, the problem is A*B*x = lambda*x or
   B*A*x = lambda*x, and A is overwritten by U*A*U**T or L**T *A*L.

   B must have been previously factorized as U**T *U or L*L**T by SPOTRF.

Parameters:

 ITYPE
   ITYPE is INTEGER
   = 1: compute inv(U**T)*A*inv(U) or inv(L)*A*inv(L**T);
   = 2 or 3: compute U*A*U**T or L**T *A*L.

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is stored, and how B has been factorized.
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
   The order of the matrices A and B.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   n by n upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading n by n lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.
```

```
  On exit, if INFO = 0, the transformed matrix, stored in the
  same format as A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

B
  B is REAL array, dimension (LDB,N)
  The triangular factor from the Cholesky factorization of B,
  as returned by SPOTRF.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

———-

59.11.3 fortran code

— **ssygs2.f** —

```
*  =====================================================================
      SUBROUTINE SSYGS2( ITYPE, UPLO, N, A, LDA, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, ITYPE, LDA, LDB, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, HALF
```

```
      PARAMETER          ( ONE = 1.0, HALF = 0.5 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            K
      REAL               AKK, BKK, CT
*     ..
*     .. External Subroutines ..
      EXTERNAL           SAXPY, SSCAL, SSYR2, STRMV, STRSV, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( ITYPE.LT.1 .OR. ITYPE.GT.3 ) THEN
         INFO = -1
      ELSE IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYGS2', -INFO )
         RETURN
      END IF
*
      IF( ITYPE.EQ.1 ) THEN
         IF( UPPER ) THEN
*
*           Compute inv(U**T)*A*inv(U)
*
            DO 10 K = 1, N
*
*              Update the upper triangle of A(k:n,k:n)
*
               AKK = A( K, K )
               BKK = B( K, K )
```

```
               AKK = AKK / BKK**2
               A( K, K ) = AKK
               IF( K.LT.N ) THEN
                  CALL SSCAL( N-K, ONE / BKK, A( K, K+1 ), LDA )
                  CT = -HALF*AKK
                  CALL SAXPY( N-K, CT, B( K, K+1 ), LDB, A( K, K+1 ),
     $                        LDA )
                  CALL SSYR2( UPLO, N-K, -ONE, A( K, K+1 ), LDA,
     $                        B( K, K+1 ), LDB, A( K+1, K+1 ), LDA )
                  CALL SAXPY( N-K, CT, B( K, K+1 ), LDB, A( K, K+1 ),
     $                        LDA )
                  CALL STRSV( UPLO, 'Transpose', 'Non-unit', N-K,
     $                        B( K+1, K+1 ), LDB, A( K, K+1 ), LDA )
               END IF
   10       CONTINUE
         ELSE
*
*           Compute inv(L)*A*inv(L**T)
*
            DO 20 K = 1, N
*
*              Update the lower triangle of A(k:n,k:n)
*
               AKK = A( K, K )
               BKK = B( K, K )
               AKK = AKK / BKK**2
               A( K, K ) = AKK
               IF( K.LT.N ) THEN
                  CALL SSCAL( N-K, ONE / BKK, A( K+1, K ), 1 )
                  CT = -HALF*AKK
                  CALL SAXPY( N-K, CT, B( K+1, K ), 1, A( K+1, K ), 1 )
                  CALL SSYR2( UPLO, N-K, -ONE, A( K+1, K ), 1,
     $                        B( K+1, K ), 1, A( K+1, K+1 ), LDA )
                  CALL SAXPY( N-K, CT, B( K+1, K ), 1, A( K+1, K ), 1 )
                  CALL STRSV( UPLO, 'No transpose', 'Non-unit', N-K,
     $                        B( K+1, K+1 ), LDB, A( K+1, K ), 1 )
               END IF
   20       CONTINUE
         END IF
      ELSE
         IF( UPPER ) THEN
*
*           Compute U*A*U**T
*
            DO 30 K = 1, N
*
*              Update the upper triangle of A(1:k,1:k)
*
               AKK = A( K, K )
               BKK = B( K, K )
```

```
               CALL STRMV( UPLO, 'No transpose', 'Non-unit', K-1, B,
     $                     LDB, A( 1, K ), 1 )
               CT = HALF*AKK
               CALL SAXPY( K-1, CT, B( 1, K ), 1, A( 1, K ), 1 )
               CALL SSYR2( UPLO, K-1, ONE, A( 1, K ), 1, B( 1, K ), 1,
     $                     A, LDA )
               CALL SAXPY( K-1, CT, B( 1, K ), 1, A( 1, K ), 1 )
               CALL SSCAL( K-1, BKK, A( 1, K ), 1 )
               A( K, K ) = AKK*BKK**2
   30       CONTINUE
         ELSE
*
*           Compute L**T *A*L
*
            DO 40 K = 1, N
*
*              Update the lower triangle of A(1:k,1:k)
*
               AKK = A( K, K )
               BKK = B( K, K )
               CALL STRMV( UPLO, 'Transpose', 'Non-unit', K-1, B, LDB,
     $                     A( K, 1 ), LDA )
               CT = HALF*AKK
               CALL SAXPY( K-1, CT, B( K, 1 ), LDB, A( K, 1 ), LDA )
               CALL SSYR2( UPLO, K-1, ONE, A( K, 1 ), LDA, B( K, 1 ),
     $                     LDB, A, LDA )
               CALL SAXPY( K-1, CT, B( K, 1 ), LDB, A( K, 1 ), LDA )
               CALL SSCAL( K-1, BKK, A( K, 1 ), LDA )
               A( K, K ) = AKK*BKK**2
   40       CONTINUE
         END IF
      END IF
      RETURN
*
*     End of SSYGS2
*
      END
```

———-

59.11.4 lisp code

— **ssygs2.lisp** —

```
(let* ((one 1.0) (half 0.5))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.5 0.5) half)
           (ignorable one half))
```

```
(defun ssygs2 (itype uplo n a lda b ldb$ info)
  (declare (type (array single-float (*)) b a)
           (type (string *) uplo)
           (type (f2cl-lib:integer4) info ldb$ lda n itype))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a single-float a-%data% a-%offset%)
       (b single-float b-%data% b-%offset%))
    (prog ((akk 0.0) (bkk 0.0) (ct 0.0) (k 0) (upper nil))
      (declare (type (single-float) akk bkk ct)
               (type (f2cl-lib:integer4) k)
               (type f2cl-lib:logical upper))
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (cond ((or (< itype 1) (> itype 3)) (setf info -1))
            ((and (not upper)
                  (not
                   (multiple-value-bind (ret-val var-0 var-1)
                       (lsame uplo "L")
                     (declare (ignore var-1))
                     (when var-0 (setf uplo var-0))
                     ret-val)))
             (setf info -2))
            ((< n 0) (setf info -3))
            ((< lda
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -5))
            ((< ldb$
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -7)))
      (cond
       ((/= info 0) (xerbla "SSYGS2" (f2cl-lib:int-sub info))
        (go end_label)))
      (cond
       ((= itype 1)
        (cond
         (upper
          (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                        ((> k n) nil)
            (tagbody
              (setf akk
                      (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                     a-%offset%))
              (setf bkk
                      (f2cl-lib:fref b-%data% (k k) ((1 ldb$) (1 *))
```

```
                        b-%offset%))
(setf akk (/ akk (expt bkk 2)))
(setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        akk)
(cond
 ((< k n)
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (sscal (f2cl-lib:int-sub n k) (/ one bkk)
       (f2cl-lib:array-slice a-%data% single-float
                             (k (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2))
    (when var-3 (setf lda var-3)))
  (setf ct (* (- half) akk))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (saxpy (f2cl-lib:int-sub n k) ct
       (f2cl-lib:array-slice b-%data% single-float
                             (k (f2cl-lib:int-add k 1))
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice a-%data% single-float
                             (k (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-2 var-4))
    (when var-1 (setf ct var-1))
    (when var-3 (setf ldb$ var-3))
    (when var-5 (setf lda var-5)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (ssyr2 uplo (f2cl-lib:int-sub n k) (- one)
       (f2cl-lib:array-slice a-%data% single-float
                             (k (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice b-%data% single-float
                             (k (f2cl-lib:int-add k 1))
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice a-%data% single-float
                             ((+ k 1) (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-1 var-2 var-3 var-5 var-7))
    (when var-0 (setf uplo var-0))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf lda var-8)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
```

```
                (saxpy (f2cl-lib:int-sub n k) ct
                 (f2cl-lib:array-slice b-%data% single-float
                                       (k (f2cl-lib:int-add k 1))
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$
                 (f2cl-lib:array-slice a-%data% single-float
                                       (k (f2cl-lib:int-add k 1))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
               (declare (ignore var-0 var-2 var-4))
               (when var-1 (setf ct var-1))
               (when var-3 (setf ldb$ var-3))
               (when var-5 (setf lda var-5)))
             (multiple-value-bind
                 (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
                 (strsv uplo "Transpose" "Non-unit" (f2cl-lib:int-sub n k)
                 (f2cl-lib:array-slice b-%data% single-float
                                       ((+ k 1) (f2cl-lib:int-add k 1))
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$
                 (f2cl-lib:array-slice a-%data% single-float
                                       (k (f2cl-lib:int-add k 1))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
               (declare (ignore var-1 var-2 var-3 var-4 var-6))
               (when var-0 (setf uplo var-0))
               (when var-5 (setf ldb$ var-5))
               (when var-7 (setf lda var-7)))))
         label10)))
      (t
       (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                     ((> k n) nil)
         (tagbody
           (setf akk
                   (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                  a-%offset%))
           (setf bkk
                   (f2cl-lib:fref b-%data% (k k) ((1 ldb$) (1 *))
                                  b-%offset%))
           (setf akk (/ akk (expt bkk 2)))
           (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                   akk)
           (cond
            ((< k n)
             (sscal (f2cl-lib:int-sub n k) (/ one bkk)
              (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                                    ((1 lda) (1 *)) a-%offset%)
              1)
             (setf ct (* (- half) akk))
             (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
```

```
    (saxpy (f2cl-lib:int-sub n k) ct
     (f2cl-lib:array-slice b-%data% single-float ((+ k 1) k)
                           ((1 ldb$) (1 *)) b-%offset%)
     1
     (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                           ((1 lda) (1 *)) a-%offset%)
     1)
  (declare (ignore var-0 var-2 var-3 var-4 var-5))
  (when var-1 (setf ct var-1)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (ssyr2 uplo (f2cl-lib:int-sub n k) (- one)
     (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                           ((1 lda) (1 *)) a-%offset%)
     1
     (f2cl-lib:array-slice b-%data% single-float ((+ k 1) k)
                           ((1 ldb$) (1 *)) b-%offset%)
     1
     (f2cl-lib:array-slice a-%data% single-float
                           ((+ k 1) (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare
   (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7))
  (when var-0 (setf uplo var-0))
  (when var-8 (setf lda var-8)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (saxpy (f2cl-lib:int-sub n k) ct
     (f2cl-lib:array-slice b-%data% single-float ((+ k 1) k)
                           ((1 ldb$) (1 *)) b-%offset%)
     1
     (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                           ((1 lda) (1 *)) a-%offset%)
     1)
  (declare (ignore var-0 var-2 var-3 var-4 var-5))
  (when var-1 (setf ct var-1)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (strsv uplo "No transpose" "Non-unit"
     (f2cl-lib:int-sub n k)
     (f2cl-lib:array-slice b-%data% single-float
                           ((+ k 1) (f2cl-lib:int-add k 1))
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                           ((1 lda) (1 *)) a-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-4 var-6 var-7))
  (when var-0 (setf uplo var-0))
  (when var-5 (setf ldb$ var-5)))))
```

```
      label20)))))
(t
 (cond
  (upper
   (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                 ((> k n) nil)
     (tagbody
       (setf akk
               (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                              a-%offset%))
       (setf bkk
               (f2cl-lib:fref b-%data% (k k) ((1 ldb$) (1 *))
                              b-%offset%))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
           (strmv uplo "No transpose" "Non-unit"
            (f2cl-lib:int-sub k 1) b ldb$
            (f2cl-lib:array-slice a-%data% single-float (1 k)
                                  ((1 lda) (1 *)) a-%offset%)
            1)
         (declare (ignore var-1 var-2 var-3 var-4 var-6 var-7))
         (when var-0 (setf uplo var-0))
         (when var-5 (setf ldb$ var-5)))
       (setf ct (* half akk))
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
           (saxpy (f2cl-lib:int-sub k 1) ct
            (f2cl-lib:array-slice b-%data% single-float (1 k)
                                  ((1 ldb$) (1 *)) b-%offset%)
            1
            (f2cl-lib:array-slice a-%data% single-float (1 k)
                                  ((1 lda) (1 *)) a-%offset%)
            1)
         (declare (ignore var-0 var-2 var-3 var-4 var-5))
         (when var-1 (setf ct var-1)))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
           (ssyr2 uplo (f2cl-lib:int-sub k 1) one
            (f2cl-lib:array-slice a-%data% single-float (1 k)
                                  ((1 lda) (1 *)) a-%offset%)
            1
            (f2cl-lib:array-slice b-%data% single-float (1 k)
                                  ((1 ldb$) (1 *)) b-%offset%)
            1 a lda)
         (declare (ignore var-1 var-3 var-4 var-5 var-6 var-7))
         (when var-0 (setf uplo var-0))
         (when var-2 (setf one var-2))
         (when var-8 (setf lda var-8)))
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
           (saxpy (f2cl-lib:int-sub k 1) ct
            (f2cl-lib:array-slice b-%data% single-float (1 k)
```

```
                                  ((1 ldb$) (1 *)) b-%offset%)
            1
            (f2cl-lib:array-slice a-%data% single-float (1 k)
                                  ((1 lda) (1 *)) a-%offset%)
            1)
          (declare (ignore var-0 var-2 var-3 var-4 var-5))
          (when var-1 (setf ct var-1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal (f2cl-lib:int-sub k 1) bkk
            (f2cl-lib:array-slice a-%data% single-float (1 k)
                                  ((1 lda) (1 *)) a-%offset%)
            1)
          (declare (ignore var-0 var-2 var-3))
          (when var-1 (setf bkk var-1)))
        (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                (* akk (expt bkk 2)))
      label30)))
  (t
   (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                 ((> k n) nil)
     (tagbody
       (setf akk
               (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                              a-%offset%))
       (setf bkk
               (f2cl-lib:fref b-%data% (k k) ((1 ldb$) (1 *))
                              b-%offset%))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
           (strmv uplo "Transpose" "Non-unit" (f2cl-lib:int-sub k 1) b
            ldb$
            (f2cl-lib:array-slice a-%data% single-float (k 1)
                                  ((1 lda) (1 *)) a-%offset%)
            lda)
         (declare (ignore var-1 var-2 var-3 var-4 var-6))
         (when var-0 (setf uplo var-0))
         (when var-5 (setf ldb$ var-5))
         (when var-7 (setf lda var-7)))
       (setf ct (* half akk))
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
           (saxpy (f2cl-lib:int-sub k 1) ct
            (f2cl-lib:array-slice b-%data% single-float (k 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$
            (f2cl-lib:array-slice a-%data% single-float (k 1)
                                  ((1 lda) (1 *)) a-%offset%)
            lda)
         (declare (ignore var-0 var-2 var-4))
         (when var-1 (setf ct var-1))
         (when var-3 (setf ldb$ var-3))
```

```
                (when var-5 (setf lda var-5)))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                  (ssyr2 uplo (f2cl-lib:int-sub k 1) one
                   (f2cl-lib:array-slice a-%data% single-float (k 1)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda
                   (f2cl-lib:array-slice b-%data% single-float (k 1)
                                         ((1 ldb$) (1 *)) b-%offset%)
                   ldb$ a lda)
                (declare (ignore var-1 var-3 var-5 var-7))
                (when var-0 (setf uplo var-0))
                (when var-2 (setf one var-2))
                (when var-4 (setf lda var-4))
                (when var-6 (setf ldb$ var-6))
                (when var-8 (setf lda var-8)))
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                  (saxpy (f2cl-lib:int-sub k 1) ct
                   (f2cl-lib:array-slice b-%data% single-float (k 1)
                                         ((1 ldb$) (1 *)) b-%offset%)
                   ldb$
                   (f2cl-lib:array-slice a-%data% single-float (k 1)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda)
                (declare (ignore var-0 var-2 var-4))
                (when var-1 (setf ct var-1))
                (when var-3 (setf ldb$ var-3))
                (when var-5 (setf lda var-5)))
              (multiple-value-bind (var-0 var-1 var-2 var-3)
                  (sscal (f2cl-lib:int-sub k 1) bkk
                   (f2cl-lib:array-slice a-%data% single-float (k 1)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda)
                (declare (ignore var-0 var-2))
                (when var-1 (setf bkk var-1))
                (when var-3 (setf lda var-3)))
              (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                      (* akk (expt bkk 2)))
            label40))))))
       (go end_label)
      end_label
       (return (values nil uplo nil nil lda nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssygs2
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
```

```
  (string)
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(nil fortran-to-lisp::uplo nil nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

59.12 ssygst

59.12.1 Axiom unit tests

— **ssygst.input** —

```
)set break resume
)sys rm -f ssygst.output
)spool ssygst.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.12.2 Axiom help page

— **ssygst.help** —

```
======================================================================
ssygst examples
======================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssygst (ITYPE, UPLO, N, A, LDA, B, LDB, INFO)
           SSYGST

Function/Subroutine Documentation
   subroutine ssygst (integer ITYPE, character UPLO, integer N, real,
       dimension( lda, * ) A, integer LDA, real, dimension( ldb, * ) B,
       integer LDB, integer INFO)
       SSYGST

Purpose:
   SSYGST reduces a real symmetric-definite generalized eigenproblem
   to standard form.

   If ITYPE = 1, the problem is A*x = lambda*B*x,
   and A is overwritten by inv(U**T)*A*inv(U) or inv(L)*A*inv(L**T)

   If ITYPE = 2 or 3, the problem is A*B*x = lambda*x or
   B*A*x = lambda*x, and A is overwritten by U*A*U**T or L**T*A*L.

   B must have been previously factorized as U**T*U or L*L**T by SPOTRF.

Parameters:

 ITYPE
   ITYPE is INTEGER
   = 1: compute inv(U**T)*A*inv(U) or inv(L)*A*inv(L**T);
   = 2 or 3: compute U*A*U**T or L**T*A*L.

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored and B is factored as U**T*U;
   = 'L':  Lower triangle of A is stored and B is factored as L*L**T.

 N
   N is INTEGER
   The order of the matrices A and B.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   N-by-N upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading N-by-N lower triangular part of A contains the lower
```

```
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if INFO = 0, the transformed matrix, stored in the
  same format as A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

B
  B is REAL array, dimension (LDB,N)
  The triangular factor from the Cholesky factorization of B,
  as returned by SPOTRF.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

59.12.3 fortran code

— **ssygst.f** —

```
*  =====================================================================
      SUBROUTINE SSYGST( ITYPE, UPLO, N, A, LDA, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, ITYPE, LDA, LDB, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
```

```
*
*     .. Parameters ..
      REAL               ONE, HALF
      PARAMETER          ( ONE = 1.0, HALF = 0.5 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            K, KB, NB
*     ..
*     .. External Subroutines ..
      EXTERNAL           SSYGS2, SSYMM, SSYR2K, STRMM, STRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( ITYPE.LT.1 .OR. ITYPE.GT.3 ) THEN
         INFO = -1
      ELSE IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYGST', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Determine the block size for this environment.
*
      NB = ILAENV( 1, 'SSYGST', UPLO, N, -1, -1, -1 )
```

```
*
      IF( NB.LE.1 .OR. NB.GE.N ) THEN
*
*        Use unblocked code
*
         CALL SSYGS2( ITYPE, UPLO, N, A, LDA, B, LDB, INFO )
      ELSE
*
*        Use blocked code
*
         IF( ITYPE.EQ.1 ) THEN
            IF( UPPER ) THEN
*
               Compute inv(U**T)*A*inv(U)
*
               DO 10 K = 1, N, NB
                  KB = MIN( N-K+1, NB )
*
                  Update the upper triangle of A(k:n,k:n)
*
                  CALL SSYGS2( ITYPE, UPLO, KB, A( K, K ), LDA,
     $                         B( K, K ), LDB, INFO )
                  IF( K+KB.LE.N ) THEN
                     CALL STRSM( 'Left', UPLO, 'Transpose', 'Non-unit',
     $                           KB, N-K-KB+1, ONE, B( K, K ), LDB,
     $                           A( K, K+KB ), LDA )
                     CALL SSYMM( 'Left', UPLO, KB, N-K-KB+1, -HALF,
     $                           A( K, K ), LDA, B( K, K+KB ), LDB, ONE,
     $                           A( K, K+KB ), LDA )
                     CALL SSYR2K( UPLO, 'Transpose', N-K-KB+1, KB, -ONE,
     $                            A( K, K+KB ), LDA, B( K, K+KB ), LDB,
     $                            ONE, A( K+KB, K+KB ), LDA )
                     CALL SSYMM( 'Left', UPLO, KB, N-K-KB+1, -HALF,
     $                           A( K, K ), LDA, B( K, K+KB ), LDB, ONE,
     $                           A( K, K+KB ), LDA )
                     CALL STRSM( 'Right', UPLO, 'No transpose',
     $                           'Non-unit', KB, N-K-KB+1, ONE,
     $                           B( K+KB, K+KB ), LDB, A( K, K+KB ),
     $                           LDA )
                  END IF
   10          CONTINUE
            ELSE
*
               Compute inv(L)*A*inv(L**T)
*
               DO 20 K = 1, N, NB
                  KB = MIN( N-K+1, NB )
*
                  Update the lower triangle of A(k:n,k:n)
*
```

```
                 CALL SSYGS2( ITYPE, UPLO, KB, A( K, K ), LDA,
     $                        B( K, K ), LDB, INFO )
                 IF( K+KB.LE.N ) THEN
                    CALL STRSM( 'Right', UPLO, 'Transpose', 'Non-unit',
     $                          N-K-KB+1, KB, ONE, B( K, K ), LDB,
     $                          A( K+KB, K ), LDA )
                    CALL SSYMM( 'Right', UPLO, N-K-KB+1, KB, -HALF,
     $                          A( K, K ), LDA, B( K+KB, K ), LDB, ONE,
     $                          A( K+KB, K ), LDA )
                    CALL SSYR2K( UPLO, 'No transpose', N-K-KB+1, KB,
     $                           -ONE, A( K+KB, K ), LDA, B( K+KB, K ),
     $                           LDB, ONE, A( K+KB, K+KB ), LDA )
                    CALL SSYMM( 'Right', UPLO, N-K-KB+1, KB, -HALF,
     $                          A( K, K ), LDA, B( K+KB, K ), LDB, ONE,
     $                          A( K+KB, K ), LDA )
                    CALL STRSM( 'Left', UPLO, 'No transpose',
     $                          'Non-unit', N-K-KB+1, KB, ONE,
     $                          B( K+KB, K+KB ), LDB, A( K+KB, K ),
     $                          LDA )
                 END IF
   20         CONTINUE
           END IF
        ELSE
           IF( UPPER ) THEN
*
*             Compute U*A*U**T
*
              DO 30 K = 1, N, NB
                 KB = MIN( N-K+1, NB )
*
*                Update the upper triangle of A(1:k+kb-1,1:k+kb-1)
*
                 CALL STRMM( 'Left', UPLO, 'No transpose', 'Non-unit',
     $                       K-1, KB, ONE, B, LDB, A( 1, K ), LDA )
                 CALL SSYMM( 'Right', UPLO, K-1, KB, HALF, A( K, K ),
     $                       LDA, B( 1, K ), LDB, ONE, A( 1, K ), LDA )
                 CALL SSYR2K( UPLO, 'No transpose', K-1, KB, ONE,
     $                        A( 1, K ), LDA, B( 1, K ), LDB, ONE, A,
     $                        LDA )
                 CALL SSYMM( 'Right', UPLO, K-1, KB, HALF, A( K, K ),
     $                       LDA, B( 1, K ), LDB, ONE, A( 1, K ), LDA )
                 CALL STRMM( 'Right', UPLO, 'Transpose', 'Non-unit',
     $                       K-1, KB, ONE, B( K, K ), LDB, A( 1, K ),
     $                       LDA )
                 CALL SSYGS2( ITYPE, UPLO, KB, A( K, K ), LDA,
     $                        B( K, K ), LDB, INFO )
   30         CONTINUE
           ELSE
*
*             Compute L**T*A*L
```

```
*
               DO 40 K = 1, N, NB
                  KB = MIN( N-K+1, NB )
*
*                 Update the lower triangle of A(1:k+kb-1,1:k+kb-1)
*
                  CALL STRMM( 'Right', UPLO, 'No transpose', 'Non-unit',
     $                        KB, K-1, ONE, B, LDB, A( K, 1 ), LDA )
                  CALL SSYMM( 'Left', UPLO, KB, K-1, HALF, A( K, K ),
     $                        LDA, B( K, 1 ), LDB, ONE, A( K, 1 ), LDA )
                  CALL SSYR2K( UPLO, 'Transpose', K-1, KB, ONE,
     $                         A( K, 1 ), LDA, B( K, 1 ), LDB, ONE, A,
     $                         LDA )
                  CALL SSYMM( 'Left', UPLO, KB, K-1, HALF, A( K, K ),
     $                        LDA, B( K, 1 ), LDB, ONE, A( K, 1 ), LDA )
                  CALL STRMM( 'Left', UPLO, 'Transpose', 'Non-unit', KB,
     $                        K-1, ONE, B( K, K ), LDB, A( K, 1 ), LDA )
                  CALL SSYGS2( ITYPE, UPLO, KB, A( K, K ), LDA,
     $                         B( K, K ), LDB, INFO )
   40          CONTINUE
            END IF
         END IF
      END IF
      RETURN
*
*     End of SSYGST
*
      END
```

———-

59.12.4 lisp code

— **ssygst.lisp** —

```
(let* ((one 1.0) (half 0.5))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.5 0.5) half)
           (ignorable one half))
  (defun ssygst (itype uplo n a lda b ldb$ info)
    (declare (type (array single-float (*)) b a)
             (type (string *) uplo)
             (type (f2cl-lib:integer4) info ldb$ lda n itype))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%))
      (prog ((k 0) (kb 0) (nb 0) (upper nil))
```

```
(declare (type (f2cl-lib:integer4) k kb nb)
         (type f2cl-lib:logical upper))
(setf info 0)
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(cond ((or (< itype 1) (> itype 3)) (setf info -1))
      ((and (not upper)
            (not
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame uplo "L")
               (declare (ignore var-1))
               (when var-0 (setf uplo var-0))
               ret-val)))
       (setf info -2))
      ((< n 0) (setf info -3))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -5))
      ((< ldb$
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -7)))
(cond
 ((/= info 0) (xerbla "SSYGST" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(setf nb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 1 "SSYGST" uplo n -1 -1 -1)
          (declare (ignore var-0 var-1 var-4 var-5 var-6))
          (setf uplo var-2)
          (setf n var-3)
          ret-val))
(cond
 ((or (<= nb 1) (>= nb n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (ssygs2 itype uplo n a lda b ldb$ info)
    (declare (ignore var-0 var-2 var-3 var-5))
    (setf uplo var-1)
    (setf lda var-4)
    (setf ldb$ var-6)
    (setf info var-7)))
 (t
  (cond
```

```
((= itype 1)
 (cond
  (upper
   (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k nb))
                 ((> k n) nil)
     (tagbody
       (setf kb
               (min
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1))
                (the f2cl-lib:integer4 nb)))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
           (ssygs2 itype uplo kb
            (f2cl-lib:array-slice a-%data% single-float (k k)
                                  ((1 lda) (1 *)) a-%offset%)
            lda
            (f2cl-lib:array-slice b-%data% single-float (k k)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$ info)
         (declare (ignore var-0 var-2 var-3 var-5))
         (setf uplo var-1)
         (setf lda var-4)
         (setf ldb$ var-6)
         (setf info var-7))
       (cond
        ((<= (f2cl-lib:int-add k kb) n)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9 var-10)
             (strsm "Left" uplo "Transpose" "Non-unit" kb
              (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1) one
              (f2cl-lib:array-slice b-%data% single-float (k k)
                                    ((1 ldb$) (1 *)) b-%offset%)
              ldb$
              (f2cl-lib:array-slice a-%data% single-float
                                    (k (f2cl-lib:int-add k kb))
                                    ((1 lda) (1 *)) a-%offset%)
              lda)
           (declare (ignore var-0 var-2 var-3 var-5 var-7 var-9))
           (when var-1 (setf uplo var-1))
           (when var-4 (setf kb var-4))
           (when var-6 (setf one var-6))
           (when var-8 (setf ldb$ var-8))
           (when var-10 (setf lda var-10)))
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9 var-10 var-11)
             (ssymm "Left" uplo kb
              (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1)
```

```
      (- half)
      (f2cl-lib:array-slice a-%data% single-float (k k)
                            ((1 lda) (1 *)) a-%offset%)
      lda
      (f2cl-lib:array-slice b-%data% single-float
                            (k (f2cl-lib:int-add k kb))
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$ one
      (f2cl-lib:array-slice a-%data% single-float
                            (k (f2cl-lib:int-add k kb))
                            ((1 lda) (1 *)) a-%offset%)
      lda)
  (declare (ignore var-0 var-3 var-4 var-5 var-7 var-10))
  (when var-1 (setf uplo var-1))
  (when var-2 (setf kb var-2))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf lda var-11)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11)
    (ssyr2k uplo "Transpose"
     (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1) kb
     (- one)
     (f2cl-lib:array-slice a-%data% single-float
                           (k (f2cl-lib:int-add k kb))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% single-float
                           (k (f2cl-lib:int-add k kb))
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ one
     (f2cl-lib:array-slice a-%data% single-float
                           ((+ k kb)
                            (f2cl-lib:int-add k kb))
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-1 var-2 var-4 var-5 var-7 var-10))
  (when var-0 (setf uplo var-0))
  (when var-3 (setf kb var-3))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf lda var-11)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11)
    (ssymm "Left" uplo kb
     (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1)
```

```
                (- half)
                (f2cl-lib:array-slice a-%data% single-float (k k)
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice b-%data% single-float
                                      (k (f2cl-lib:int-add k kb))
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$ one
                (f2cl-lib:array-slice a-%data% single-float
                                      (k (f2cl-lib:int-add k kb))
                                      ((1 lda) (1 *)) a-%offset%)
                lda)
              (declare (ignore var-0 var-3 var-4 var-5 var-7 var-10))
              (when var-1 (setf uplo var-1))
              (when var-2 (setf kb var-2))
              (when var-6 (setf lda var-6))
              (when var-8 (setf ldb$ var-8))
              (when var-9 (setf one var-9))
              (when var-11 (setf lda var-11)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10)
                (strsm "Right" uplo "No transpose" "Non-unit" kb
                 (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1) one
                 (f2cl-lib:array-slice b-%data% single-float
                                       ((+ k kb)
                                        (f2cl-lib:int-add k kb))
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$
                 (f2cl-lib:array-slice a-%data% single-float
                                       (k (f2cl-lib:int-add k kb))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare (ignore var-0 var-2 var-3 var-5 var-7 var-9))
              (when var-1 (setf uplo var-1))
              (when var-4 (setf kb var-4))
              (when var-6 (setf one var-6))
              (when var-8 (setf ldb$ var-8))
              (when var-10 (setf lda var-10)))))
        label10)))
    (t
     (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k nb))
                   ((> k n) nil)
       (tagbody
         (setf kb
                 (min
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1))
                  (the f2cl-lib:integer4 nb)))
         (multiple-value-bind
```

```
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (ssygs2 itype uplo kb
       (f2cl-lib:array-slice a-%data% single-float (k k)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice b-%data% single-float (k k)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$ info)
    (declare (ignore var-0 var-2 var-3 var-5))
    (setf uplo var-1)
    (setf lda var-4)
    (setf ldb$ var-6)
    (setf info var-7))
  (cond
   ((<= (f2cl-lib:int-add k kb) n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10)
        (strsm "Right" uplo "Transpose" "Non-unit"
         (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1) kb one
         (f2cl-lib:array-slice b-%data% single-float (k k)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice a-%data% single-float
                               ((+ k kb) k) ((1 lda) (1 *))
                               a-%offset%)
         lda)
      (declare (ignore var-0 var-2 var-3 var-4 var-7 var-9))
      (when var-1 (setf uplo var-1))
      (when var-5 (setf kb var-5))
      (when var-6 (setf one var-6))
      (when var-8 (setf ldb$ var-8))
      (when var-10 (setf lda var-10)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11)
        (ssymm "Right" uplo
         (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1) kb
         (- half)
         (f2cl-lib:array-slice a-%data% single-float (k k)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice b-%data% single-float
                               ((+ k kb) k) ((1 ldb$) (1 *))
                               b-%offset%)
         ldb$ one
         (f2cl-lib:array-slice a-%data% single-float
                               ((+ k kb) k) ((1 lda) (1 *))
                               a-%offset%)
         lda)
```

```
  (declare (ignore var-0 var-2 var-4 var-5 var-7 var-10))
  (when var-1 (setf uplo var-1))
  (when var-3 (setf kb var-3))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf lda var-11)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11)
    (ssyr2k uplo "No transpose"
     (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1) kb
     (- one)
     (f2cl-lib:array-slice a-%data% single-float
                           ((+ k kb) k) ((1 lda) (1 *))
                           a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% single-float
                           ((+ k kb) k) ((1 ldb$) (1 *))
                           b-%offset%)
     ldb$ one
     (f2cl-lib:array-slice a-%data% single-float
                           ((+ k kb)
                            (f2cl-lib:int-add k kb))
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-1 var-2 var-4 var-5 var-7 var-10))
  (when var-0 (setf uplo var-0))
  (when var-3 (setf kb var-3))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf lda var-11)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11)
    (ssymm "Right" uplo
     (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1) kb
     (- half)
     (f2cl-lib:array-slice a-%data% single-float (k k)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% single-float
                           ((+ k kb) k) ((1 ldb$) (1 *))
                           b-%offset%)
     ldb$ one
     (f2cl-lib:array-slice a-%data% single-float
                           ((+ k kb) k) ((1 lda) (1 *))
                           a-%offset%)
     lda)
```

```
                (declare (ignore var-0 var-2 var-4 var-5 var-7 var-10))
                (when var-1 (setf uplo var-1))
                (when var-3 (setf kb var-3))
                (when var-6 (setf lda var-6))
                (when var-8 (setf ldb$ var-8))
                (when var-9 (setf one var-9))
                (when var-11 (setf lda var-11)))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                   var-9 var-10)
                  (strsm "Left" uplo "No transpose" "Non-unit"
                   (f2cl-lib:int-add (f2cl-lib:int-sub n k kb) 1) kb one
                   (f2cl-lib:array-slice b-%data% single-float
                                         ((+ k kb)
                                          (f2cl-lib:int-add k kb))
                                         ((1 ldb$) (1 *)) b-%offset%)
                   ldb$
                   (f2cl-lib:array-slice a-%data% single-float
                                         ((+ k kb) k) ((1 lda) (1 *))
                                         a-%offset%)
                   lda)
                (declare (ignore var-0 var-2 var-3 var-4 var-7 var-9))
                (when var-1 (setf uplo var-1))
                (when var-5 (setf kb var-5))
                (when var-6 (setf one var-6))
                (when var-8 (setf ldb$ var-8))
                (when var-10 (setf lda var-10)))))
          label20)))))
     (t
      (cond
       (upper
        (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k nb))
                      ((> k n) nil)
          (tagbody
            (setf kb
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1))
                     (the f2cl-lib:integer4 nb)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10)
                (strmm "Left" uplo "No transpose" "Non-unit"
                 (f2cl-lib:int-sub k 1) kb one b ldb$
                 (f2cl-lib:array-slice a-%data% single-float (1 k)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare (ignore var-0 var-2 var-3 var-4 var-7 var-9))
              (when var-1 (setf uplo var-1))
              (when var-5 (setf kb var-5))
```

```
  (when var-6 (setf one var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-10 (setf lda var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11)
    (ssymm "Right" uplo (f2cl-lib:int-sub k 1) kb half
     (f2cl-lib:array-slice a-%data% single-float (k k)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% single-float (1 k)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ one
     (f2cl-lib:array-slice a-%data% single-float (1 k)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-2 var-5 var-7 var-10))
  (when var-1 (setf uplo var-1))
  (when var-3 (setf kb var-3))
  (when var-4 (setf half var-4))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf lda var-11)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11)
    (ssyr2k uplo "No transpose" (f2cl-lib:int-sub k 1) kb one
     (f2cl-lib:array-slice a-%data% single-float (1 k)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% single-float (1 k)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ one a lda)
  (declare (ignore var-1 var-2 var-5 var-7 var-10))
  (when var-0 (setf uplo var-0))
  (when var-3 (setf kb var-3))
  (when var-4 (setf one var-4))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf lda var-11)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11)
    (ssymm "Right" uplo (f2cl-lib:int-sub k 1) kb half
     (f2cl-lib:array-slice a-%data% single-float (k k)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% single-float (1 k)
```

```
                                ((1 ldb$) (1 *)) b-%offset%)
                ldb$ one
                (f2cl-lib:array-slice a-%data% single-float (1 k)
                                ((1 lda) (1 *)) a-%offset%)
                lda)
            (declare (ignore var-0 var-2 var-5 var-7 var-10))
            (when var-1 (setf uplo var-1))
            (when var-3 (setf kb var-3))
            (when var-4 (setf half var-4))
            (when var-6 (setf lda var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-9 (setf one var-9))
            (when var-11 (setf lda var-11)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10)
              (strmm "Right" uplo "Transpose" "Non-unit"
               (f2cl-lib:int-sub k 1) kb one
               (f2cl-lib:array-slice b-%data% single-float (k k)
                                ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice a-%data% single-float (1 k)
                                ((1 lda) (1 *)) a-%offset%)
               lda)
            (declare (ignore var-0 var-2 var-3 var-4 var-7 var-9))
            (when var-1 (setf uplo var-1))
            (when var-5 (setf kb var-5))
            (when var-6 (setf one var-6))
            (when var-8 (setf ldb$ var-8))
            (when var-10 (setf lda var-10)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
              (ssygs2 itype uplo kb
               (f2cl-lib:array-slice a-%data% single-float (k k)
                                ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice b-%data% single-float (k k)
                                ((1 ldb$) (1 *)) b-%offset%)
               ldb$ info)
            (declare (ignore var-0 var-2 var-3 var-5))
            (setf uplo var-1)
            (setf lda var-4)
            (setf ldb$ var-6)
            (setf info var-7))
        label30)))
    (t
     (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k nb))
                   ((> k n) nil)
       (tagbody
         (setf kb
```

```
        (min
         (the f2cl-lib:integer4
              (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1))
         (the f2cl-lib:integer4 nb)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (strmm "Right" uplo "No transpose" "Non-unit" kb
     (f2cl-lib:int-sub k 1) one b ldb$
     (f2cl-lib:array-slice a-%data% single-float (k 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-2 var-3 var-5 var-7 var-9))
  (when var-1 (setf uplo var-1))
  (when var-4 (setf kb var-4))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-10 (setf lda var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11)
    (ssymm "Left" uplo kb (f2cl-lib:int-sub k 1) half
     (f2cl-lib:array-slice a-%data% single-float (k k)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% single-float (k 1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ one
     (f2cl-lib:array-slice a-%data% single-float (k 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-3 var-5 var-7 var-10))
  (when var-1 (setf uplo var-1))
  (when var-2 (setf kb var-2))
  (when var-4 (setf half var-4))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf lda var-11)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11)
    (ssyr2k uplo "Transpose" (f2cl-lib:int-sub k 1) kb one
     (f2cl-lib:array-slice a-%data% single-float (k 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% single-float (k 1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ one a lda)
  (declare (ignore var-1 var-2 var-5 var-7 var-10))
```

```
  (when var-0 (setf uplo var-0))
  (when var-3 (setf kb var-3))
  (when var-4 (setf one var-4))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf lda var-11)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11)
    (ssymm "Left" uplo kb (f2cl-lib:int-sub k 1) half
     (f2cl-lib:array-slice a-%data% single-float (k k)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% single-float (k 1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ one
     (f2cl-lib:array-slice a-%data% single-float (k 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-3 var-5 var-7 var-10))
  (when var-1 (setf uplo var-1))
  (when var-2 (setf kb var-2))
  (when var-4 (setf half var-4))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf lda var-11)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (strmm "Left" uplo "Transpose" "Non-unit" kb
     (f2cl-lib:int-sub k 1) one
     (f2cl-lib:array-slice b-%data% single-float (k k)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice a-%data% single-float (k 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-2 var-3 var-5 var-7 var-9))
  (when var-1 (setf uplo var-1))
  (when var-4 (setf kb var-4))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldb$ var-8))
  (when var-10 (setf lda var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (ssygs2 itype uplo kb
     (f2cl-lib:array-slice a-%data% single-float (k k)
                           ((1 lda) (1 *)) a-%offset%)
```

```
                      lda
                      (f2cl-lib:array-slice b-%data% single-float (k k)
                                            ((1 ldb$) (1 *)) b-%offset%)
                      ldb$ info)
                    (declare (ignore var-0 var-2 var-3 var-5))
                    (setf uplo var-1)
                    (setf lda var-4)
                    (setf ldb$ var-6)
                    (setf info var-7))
                  label40))))))))
        (go end_label)
       end_label
        (return (values nil uplo n nil lda nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssygst
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::ssygs2
                                              fortran-to-lisp::ilaenv))))
```

———-

59.13 ssyrfs

59.13.1 Axiom unit tests

— ssyrfs.input —

```
)set break resume
)sys rm -f ssyrfs.output
)spool ssyrfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.13.2 Axiom help page

— **ssyrfs.help** —

```
==========================================================================
ssyrfs examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssyrfs (UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB, X,
           LDX, FERR, BERR, WORK, IWORK, INFO)
           SSYRFS

Function/Subroutine Documentation
   subroutine ssyrfs (character UPLO, integer N, integer NRHS, real,
       dimension( lda, * ) A, integer LDA, real, dimension( ldaf, * ) AF,
       integer LDAF, integer, dimension( * ) IPIV, real, dimension( ldb, * )
       B, integer LDB, real, dimension( ldx, * ) X, integer LDX, real,
       dimension( * ) FERR, real, dimension( * ) BERR, real, dimension( * )
       WORK, integer, dimension( * ) IWORK, integer INFO)
       SSYRFS

Purpose:
   SSYRFS improves the computed solution to a system of linear
   equations when the coefficient matrix is symmetric indefinite, and
   provides error bounds and backward error estimates for the solution.

Parameters:

 UPLO
   UPLO is CHARACTER*1
```

```
  = 'U':  Upper triangle of A is stored;
  = 'L':  Lower triangle of A is stored.

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is REAL array, dimension (LDA,N)
  The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
  upper triangular part of A contains the upper triangular part
  of the matrix A, and the strictly lower triangular part of A
  is not referenced.  If UPLO = 'L', the leading N-by-N lower
  triangular part of A contains the lower triangular part of
  the matrix A, and the strictly upper triangular part of A is
  not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is REAL array, dimension (LDAF,N)
  The factored form of the matrix A.  AF contains the block
  diagonal matrix D and the multipliers used to obtain the
  factor U or L from the factorization A = U*D*U**T or
  A = L*D*L**T as computed by SSYTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by SSYTRF.

B
  B is REAL array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).
```

```
 X
   X is REAL array, dimension (LDX,NRHS)
   On entry, the solution matrix X, as computed by SSYTRS.
   On exit, the improved solution matrix X.

 LDX
   LDX is INTEGER
   The leading dimension of the array X.  LDX >= max(1,N).

 FERR
   FERR is REAL array, dimension (NRHS)
   The estimated forward error bound for each solution vector
   X(j) (the j-th column of the solution matrix X).
   If XTRUE is the true solution corresponding to X(j), FERR(j)
   is an estimated upper bound for the magnitude of the largest
   element in (X(j) - XTRUE) divided by the magnitude of the
   largest element in X(j).  The estimate is as reliable as
   the estimate for RCOND, and is almost always a slight
   overestimate of the true error.

 BERR
   BERR is REAL array, dimension (NRHS)
   The componentwise relative backward error of each solution
   vector X(j) (i.e., the smallest relative change in
   any element of A or B that makes X(j) an exact solution).

 WORK
   WORK is REAL array, dimension (3*N)

 IWORK
   IWORK is INTEGER array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

   ITMAX is the maximum number of steps of iterative refinement.
```

———-

59.13.3 fortran code

— ssyrfs.f —

```
*  =====================================================================
      SUBROUTINE SSYRFS( UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB,
     $                   X, LDX, FERR, BERR, WORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      REAL               A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   BERR( * ), FERR( * ), WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
      REAL               TWO
      PARAMETER          ( TWO = 2.0E+0 )
      REAL               THREE
      PARAMETER          ( THREE = 3.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            COUNT, I, J, K, KASE, NZ
      REAL               EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN, XK
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SAXPY, SCOPY, SLACN2, SSYMV, SSYTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH
```

```
      EXTERNAL           LSAME, SLAMCH
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -10
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -12
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYRFS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = N + 1
      EPS = SLAMCH( 'Epsilon' )
      SAFMIN = SLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 140 J = 1, NRHS
*
```

```
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - A * X
*
         CALL SCOPY( N, B( 1, J ), 1, WORK( N+1 ), 1 )
         CALL SSYMV( UPLO, N, -ONE, A, LDA, X( 1, J ), 1, ONE,
     $               WORK( N+1 ), 1 )
*
*        Compute componentwise relative backward error from formula
*
*        max(i) ( abs(R(i)) / ( abs(A)*abs(X) + abs(B) )(i) )
*
*        where abs(Z) is the componentwise absolute value of the matrix
*        or vector Z.  If the i-th component of the denominator is less
*        than SAFE2, then SAFE1 is added to the i-th components of the
*        numerator and denominator before dividing.
*
         DO 30 I = 1, N
            WORK( I ) = ABS( B( I, J ) )
   30    CONTINUE
*
*        Compute abs(A)*abs(X) + abs(B).
*
         IF( UPPER ) THEN
            DO 50 K = 1, N
               S = ZERO
               XK = ABS( X( K, J ) )
               DO 40 I = 1, K - 1
                  WORK( I ) = WORK( I ) + ABS( A( I, K ) )*XK
                  S = S + ABS( A( I, K ) )*ABS( X( I, J ) )
   40          CONTINUE
               WORK( K ) = WORK( K ) + ABS( A( K, K ) )*XK + S
   50       CONTINUE
         ELSE
            DO 70 K = 1, N
               S = ZERO
               XK = ABS( X( K, J ) )
               WORK( K ) = WORK( K ) + ABS( A( K, K ) )*XK
               DO 60 I = K + 1, N
                  WORK( I ) = WORK( I ) + ABS( A( I, K ) )*XK
                  S = S + ABS( A( I, K ) )*ABS( X( I, J ) )
   60          CONTINUE
               WORK( K ) = WORK( K ) + S
   70       CONTINUE
         END IF
         S = ZERO
```

```
      DO 80 I = 1, N
         IF( WORK( I ).GT.SAFE2 ) THEN
            S = MAX( S, ABS( WORK( N+I ) ) / WORK( I ) )
         ELSE
            S = MAX( S, ( ABS( WORK( N+I ) )+SAFE1 ) /
     $           ( WORK( I )+SAFE1 ) )
         END IF
   80 CONTINUE
      BERR( J ) = S
*
*     Test stopping criterion. Continue iterating if
*        1) The residual BERR(J) is larger than machine epsilon, and
*        2) BERR(J) decreased by at least a factor of 2 during the
*           last iteration, and
*        3) At most ITMAX iterations tried.
*
      IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $     COUNT.LE.ITMAX ) THEN
*
*        Update solution and try again.
*
         CALL SSYTRS( UPLO, N, 1, AF, LDAF, IPIV, WORK( N+1 ), N,
     $                INFO )
         CALL SAXPY( N, ONE, WORK( N+1 ), 1, X( 1, J ), 1 )
         LSTRES = BERR( J )
         COUNT = COUNT + 1
         GO TO 20
      END IF
*
*     Bound error from formula
*
*     norm(X - XTRUE) / norm(X) .le. FERR =
*     norm( abs(inv(A))*
*        ( abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) ))) / norm(X)
*
*     where
*       norm(Z) is the magnitude of the largest component of Z
*       inv(A) is the inverse of A
*       abs(Z) is the componentwise absolute value of the matrix or
*          vector Z
*       NZ is the maximum number of nonzeros in any row of A, plus 1
*       EPS is machine epsilon
*
*     The i-th component of abs(R)+NZ*EPS*(abs(A)*abs(X)+abs(B))
*     is incremented by SAFE1 if the i-th component of
*     abs(A)*abs(X) + abs(B) is less than SAFE2.
*
*     Use SLACN2 to estimate the infinity-norm of the matrix
*        inv(A) * diag(W),
*     where W = abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) )))
```

```
*
         DO 90 I = 1, N
            IF( WORK( I ).GT.SAFE2 ) THEN
               WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I )
            ELSE
               WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I ) + SAFE1
            END IF
   90    CONTINUE
*
         KASE = 0
  100    CONTINUE
         CALL SLACN2( N, WORK( 2*N+1 ), WORK( N+1 ), IWORK, FERR( J ),
     $                KASE, ISAVE )
         IF( KASE.NE.0 ) THEN
            IF( KASE.EQ.1 ) THEN
*
*              Multiply by diag(W)*inv(A**T).
*
               CALL SSYTRS( UPLO, N, 1, AF, LDAF, IPIV, WORK( N+1 ), N,
     $                      INFO )
               DO 110 I = 1, N
                  WORK( N+I ) = WORK( I )*WORK( N+I )
  110          CONTINUE
            ELSE IF( KASE.EQ.2 ) THEN
*
*              Multiply by inv(A)*diag(W).
*
               DO 120 I = 1, N
                  WORK( N+I ) = WORK( I )*WORK( N+I )
  120          CONTINUE
               CALL SSYTRS( UPLO, N, 1, AF, LDAF, IPIV, WORK( N+1 ), N,
     $                      INFO )
            END IF
            GO TO 100
         END IF
*
*        Normalize error.
*
         LSTRES = ZERO
         DO 130 I = 1, N
            LSTRES = MAX( LSTRES, ABS( X( I, J ) ) )
  130    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  140 CONTINUE
*
      RETURN
*
*     End of SSYRFS
```

```
*
      END
```

———-

59.13.4 lisp code

— **ssyrfs.lisp** —

```
(let* ((itmax 5) (zero 0.0) (one 1.0) (two 2.0) (three 3.0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 3.0 3.0) three)
           (ignorable itmax zero one two three))
  (defun ssyrfs
         (uplo n nrhs a lda af ldaf ipiv b ldb$ x ldx ferr berr work iwork
          info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array single-float (*)) work berr ferr x b af a)
             (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (af single-float af-%data% af-%offset%)
         (b single-float b-%data% b-%offset%)
         (x single-float x-%data% x-%offset%)
         (ferr single-float ferr-%data% ferr-%offset%)
         (berr single-float berr-%data% berr-%offset%)
         (work single-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4)) (eps 0.0)
             (lstres 0.0) (s 0.0) (safe1 0.0) (safe2 0.0) (safmin 0.0) (xk 0.0)
             (count$ 0) (i 0) (j 0) (k 0) (kase 0) (nz 0) (upper nil))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (single-float) eps lstres s safe1 safe2 safmin xk)
                 (type (f2cl-lib:integer4) count$ i j k kase nz)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
```

```
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -10))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -12)))
(cond
 ((/= info 0) (xerbla "SSYRFS" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%) zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) zero)
     label10))
  (go end_label)))
(setf nz (f2cl-lib:int-add n 1))
(setf eps (slamch "Epsilon"))
(setf safmin (slamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy n
         (f2cl-lib:array-slice b-%data% single-float (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1
         (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                               ((1 *)) work-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
```

```
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (ssymv uplo n (- one) a lda
     (f2cl-lib:array-slice x-%data% single-float (1 j)
                           ((1 ldx) (1 *)) x-%offset%)
     1 one
     (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                           ((1 *)) work-%offset%)
     1)
  (declare (ignore var-2 var-3 var-5 var-6 var-8 var-9))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-4 (setf lda var-4))
  (when var-7 (setf one var-7)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (abs
             (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                            b-%offset%)))
   label30))
(cond
 (upper
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (setf s zero)
      (setf xk
              (abs
               (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                              x-%offset%)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                  (+
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                   (*
                    (abs
                     (f2cl-lib:fref a-%data% (i k) ((1 lda) (1 *))
                                    a-%offset%))
                    xk)))
          (setf s
                  (+ s
```

```
                        (*
                         (abs
                          (f2cl-lib:fref a-%data% (i k)
                                         ((1 lda) (1 *)) a-%offset%))
                         (abs
                          (f2cl-lib:fref x-%data% (i j)
                                         ((1 ldx) (1 *))
                                         x-%offset%)))))
            label40))
        (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                (+
                 (f2cl-lib:fref work-%data% (k) ((1 *))
                                work-%offset%)
                 (*
                  (abs
                   (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                  a-%offset%))
                  xk)
                 s))
      label50)))
  (t
   (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                 ((> k n) nil)
     (tagbody
       (setf s zero)
       (setf xk
               (abs
                (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                               x-%offset%)))
       (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
               (+
                (f2cl-lib:fref work-%data% (k) ((1 *))
                               work-%offset%)
                (*
                 (abs
                  (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                 a-%offset%))
                 xk)))
       (f2cl-lib:fdo (i (f2cl-lib:int-add k 1)
                      (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                   (+
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)
                    (*
                     (abs
                      (f2cl-lib:fref a-%data% (i k) ((1 lda) (1 *))
```

```
                                        a-%offset%))
                        xk)))
            (setf s
                    (+ s
                       (*
                        (abs
                         (f2cl-lib:fref a-%data% (i k)
                                        ((1 lda) (1 *)) a-%offset%))
                        (abs
                         (f2cl-lib:fref x-%data% (i j)
                                        ((1 ldx) (1 *))
                                        x-%offset%)))))
          label60))
       (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
               (+
                (f2cl-lib:fref work-%data% (k) ((1 *))
                               work-%offset%)
                s))
     label70))))
(setf s zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
      (setf s
              (max s
                   (/
                    (abs
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add n i))
                                    ((1 *)) work-%offset%))
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)))))
     (t
      (setf s
              (max s
                   (/
                    (+
                     (abs
                      (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-add n i))
                                     ((1 *)) work-%offset%))
                     safe1)
                    (+
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)
                     safe1))))))
   label80))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
```

```
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (ssytrs uplo n 1 af ldaf ipiv
       (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                             ((1 *)) work-%offset%)
       n info)
    (declare (ignore var-2 var-3 var-5 var-6))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf ldaf var-4))
    (when var-7 (setf n var-7))
    (when var-8 (setf info var-8)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (saxpy n one
       (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                             ((1 *)) work-%offset%)
       1
       (f2cl-lib:array-slice x-%data% single-float (1 j)
                             ((1 ldx) (1 *)) x-%offset%)
       1)
    (declare (ignore var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))
    (when var-1 (setf one var-1)))
  (setf lstres
          (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
  (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (+
               (abs
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%))
               (* nz eps
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)))))
     (t
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (+
               (abs
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%))
               (* nz eps
```

```
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%))
                safe1))))
   label90))
 (setf kase 0)
label100
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slacn2 n
      (f2cl-lib:array-slice work-%data% single-float
                            ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                            ((1 *)) work-%offset%)
      iwork (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
      kase isave)
   (declare (ignore var-1 var-2 var-3 var-6))
   (setf n var-0)
   (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
           var-4)
   (setf kase var-5))
 (cond
  ((/= kase 0)
   (cond
    ((= kase 1)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (ssytrs uplo n 1 af ldaf ipiv
          (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                                ((1 *)) work-%offset%)
          n info)
       (declare (ignore var-2 var-3 var-5 var-6))
       (when var-0 (setf uplo var-0))
       (when var-1 (setf n var-1))
       (when var-4 (setf ldaf var-4))
       (when var-7 (setf n var-7))
       (when var-8 (setf info var-8)))
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                              ((1 *)) work-%offset%)
                (*
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add n i)) ((1 *))
                                work-%offset%)))
        label110)))
    ((= kase 2)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                          ((> i n) nil)
                  (tagbody
                    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                         ((1 *)) work-%offset%)
                            (*
                             (f2cl-lib:fref work-%data% (i) ((1 *))
                                            work-%offset%)
                             (f2cl-lib:fref work-%data%
                                            ((f2cl-lib:int-add n i)) ((1 *))
                                            work-%offset%)))
                   label120))
                 (multiple-value-bind
                     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                     (ssytrs uplo n 1 af ldaf ipiv
                      (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                                            ((1 *)) work-%offset%)
                      n info)
                   (declare (ignore var-2 var-3 var-5 var-6))
                   (when var-0 (setf uplo var-0))
                   (when var-1 (setf n var-1))
                   (when var-4 (setf ldaf var-4))
                   (when var-7 (setf n var-7))
                   (when var-8 (setf info var-8)))))
             (go label100)))
           (setf lstres zero)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf lstres
                       (max lstres
                            (abs
                             (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                            x-%offset%))))
              label130))
           (if (/= lstres zero)
               (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                       (/
                        (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                        lstres)))
          label140))
       (go end_label)
      end_label
       (return
        (values uplo n nil nil lda nil ldaf nil nil nil nil nil nil nil nil
                nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssyrfs
```

```
        fortran-to-lisp::*f2cl-function-info*)
  (fortran-to-lisp::make-f2cl-finfo :arg-types
                                    '((string)
                                      (fortran-to-lisp::integer4)
                                      (fortran-to-lisp::integer4)
                                      (array single-float (*))
                                      (fortran-to-lisp::integer4)
                                      (array single-float (*))
                                      (fortran-to-lisp::integer4)
                                      (array fortran-to-lisp::integer4
                                       (*))
                                      (array single-float (*))
                                      (fortran-to-lisp::integer4)
                                      (array single-float (*))
                                      (fortran-to-lisp::integer4)
                                      (array single-float (*))
                                      (array single-float (*))
                                      (array single-float (*))
                                      (array fortran-to-lisp::integer4
                                       (*))
                                      (fortran-to-lisp::integer4))
                                    :return-values
                                    '(fortran-to-lisp::uplo
                                      fortran-to-lisp::n nil nil
                                      fortran-to-lisp::lda nil
                                      fortran-to-lisp::ldaf nil nil nil
                                      nil nil nil nil nil nil
                                      fortran-to-lisp::info)
                                    :calls '(fortran-to-lisp::slacn2))))
```

———-

59.14 ssyrfsx

59.14.1 Axiom unit tests

— ssyrfsx.input —

```
)set break resume
)sys rm -f ssyrfsx.output
)spool ssyrfsx.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

59.14.2 Axiom help page [Demm05]

— ssyrfsx.help —

```
==========================================================================
ssyrfsx examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssyrfsx (UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF, IPIV, S, B,
           LDB, X, LDX, RCOND, BERR, N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP,
           NPARAMS, PARAMS, WORK, IWORK, INFO)
           SSYRFSX

Function/Subroutine Documentation
   subroutine ssyrfsx (character UPLO, character EQUED, integer N, integer
       NRHS, real, dimension( lda, * ) A, integer LDA, real, dimension( ldaf,
       * ) AF, integer LDAF, integer, dimension( * ) IPIV, real, dimension( *
       ) S, real, dimension( ldb, * ) B, integer LDB, real, dimension( ldx, *
       ) X, integer LDX, real RCOND, real, dimension( * ) BERR, integer
       N_ERR_BNDS, real, dimension( nrhs, * ) ERR_BNDS_NORM, real, dimension(
       nrhs, * ) ERR_BNDS_COMP, integer NPARAMS, real, dimension( * ) PARAMS,
       real, dimension( * ) WORK, integer, dimension( * ) IWORK, integer INFO)
       SSYRFSX

Purpose:
   SSYRFSX improves the computed solution to a system of linear
   equations when the coefficient matrix is symmetric indefinite, and
   provides error bounds and backward error estimates for the
   solution.  In addition to normwise error bound, the code provides
   maximum componentwise error bound if possible.  See comments for
   ERR_BNDS_NORM and ERR_BNDS_COMP for details of the error bounds.

   The original system of linear equations may have been equilibrated
   before calling this routine, as described by arguments EQUED and S
   below. In this case, the solution and error bounds returned are
   for the original unequilibrated system.
```

```
   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done to A
   before calling this routine. This is needed to compute
   the solution and error bounds correctly.
   = 'N':  No equilibration
   = 'Y':  Both row and column equilibration, i.e., A has been
           replaced by diag(S) * A * diag(S).
   The right hand side B has been changed accordingly.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 A
   A is REAL array, dimension (LDA,N)
   The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
   upper triangular part of A contains the upper triangular
   part of the matrix A, and the strictly lower triangular
   part of A is not referenced.  If UPLO = 'L', the leading
   N-by-N lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is REAL array, dimension (LDAF,N)
   The factored form of the matrix A.  AF contains the block
   diagonal matrix D and the multipliers used to obtain the
```

```
  factor U or L from the factorization A = U*D*U**T or A =
  L*D*L**T as computed by SSYTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by SSYTRF.

S
  S is REAL array, dimension (N)
  The scale factors for A.  If EQUED = 'Y', A is multiplied on
  the left and right by diag(S).  S is an input argument if FACT =
  'F'; otherwise, S is an output argument.  If FACT = 'F' and EQUED
  = 'Y', each element of S must be positive.  If S is output, each
  element of S is a power of the radix. If S is input, each element
  of S should be a power of the radix to ensure a reliable solution
  and error estimates. Scaling by powers of the radix does not cause
  rounding errors unless the result underflows or overflows.
  Rounding errors during scaling lead to refining with a matrix that
  is not equivalent to the input matrix, producing error estimates
  that may not be reliable.

B
  B is REAL array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is REAL array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by SGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
```

```
  if this number is very small and the matrix appears ill-conditioned.

BERR
  BERR is REAL array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
         max_j (abs(XTRUE(j,i) - X(j,i)))
          ------------------------------
                max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
```

```
      numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
      appropriately scaled matrix Z.
      Let Z = S*A, where S scales each row by a power of the
      radix so all absolute row sums of Z are approximately 1.

 ERR_BNDS_COMP
   ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
   For each right-hand side, this array contains information about
   various error bounds and condition numbers corresponding to the
   componentwise relative error, which is defined as follows:

   Componentwise relative error in the ith solution vector:
                  abs(XTRUE(j,i) - X(j,i))
             max_j ----------------------
                         abs(X(j,i))

   The array is indexed by the right-hand side i (on which the
   componentwise relative error depends), and the type of error
   information as described below. There currently are up to three
   pieces of information returned for each right-hand side. If
   componentwise accuracy is not requested (PARAMS(3) = 0.0), then
   ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
   the first (:,N_ERR_BNDS) entries are returned.

   The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
   right-hand side.

   The second index in ERR_BNDS_COMP(:,err) contains the following
   three fields:

   err = 1 "Trust/don't trust" boolean. Trust the answer if the
      reciprocal condition number is less than the threshold
      sqrt(n) * slamch('Epsilon').

   err = 2 "Guaranteed" error bound: The estimated forward error,
      almost certainly within a factor of 10 of the true error
      so long as the next entry is greater than the threshold
      sqrt(n) * slamch('Epsilon'). This error bound should only
      be trusted if the previous boolean is true.

   err = 3  Reciprocal condition number: Estimated componentwise
      reciprocal condition number.  Compared with the threshold
      sqrt(n) * slamch('Epsilon') to determine if the error
      estimate is "guaranteed". These reciprocal condition
      numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
      appropriately scaled matrix Z.
      Let Z = S*(A*diag(x)), where x is the solution for the
      current right-hand side and S scales each row of
      A*diag(x) by a power of the radix so all absolute row
      sums of Z are approximately 1.
```

```
NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is REAL array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
  (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be
              trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is REAL array, dimension (4*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is
  guaranteed.
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
```

```
      has been completed, but the factor U is exactly singular, so
      the solution and error bounds could not be computed. RCOND = 0
      is returned.
   = N+J: The solution corresponding to the Jth right-hand side is
      not guaranteed. The solutions corresponding to other right-
      hand sides K with K > J may not be guaranteed as well, but
      only the first such right-hand side is reported. If a small
      componentwise error is not requested (PARAMS(3) = 0.0) then
      the Jth right-hand side is the first with a normwise error
      bound that is not guaranteed (the smallest J such
      that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
      the Jth right-hand side is the first with either a normwise or
      componentwise error bound that is not guaranteed (the smallest
      J such that either ERR_BNDS_NORM(J,1) = 0.0 or
      ERR_BNDS_COMP(J,1) = 0.0). See the definition of
      ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
      about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

———-

59.14.3 fortran code

— **ssyrfsx.f** —

```
*  ======================================================================
      SUBROUTINE SSYRFSX( UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $                    S, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS,
     $                    ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $                    WORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO, EQUED
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      REAL               A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX, * ), WORK( * )
      REAL               S( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
```

```
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  =================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               ITREF_DEFAULT, ITHRESH_DEFAULT,
     $                   COMPONENTWISE_DEFAULT
      REAL               RTHRESH_DEFAULT, DZTHRESH_DEFAULT
      PARAMETER          ( ITREF_DEFAULT = 1.0 )
      PARAMETER          ( ITHRESH_DEFAULT = 10.0 )
      PARAMETER          ( COMPONENTWISE_DEFAULT = 1.0 )
      PARAMETER          ( RTHRESH_DEFAULT = 0.5 )
      PARAMETER          ( DZTHRESH_DEFAULT = 0.25 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. Local Scalars ..
      CHARACTER(1)       NORM
      LOGICAL            RCEQU
      INTEGER            J, PREC_TYPE, REF_TYPE, N_NORMS
      REAL               ANORM, RCOND_TMP
      REAL               ILLRCOND_THRESH, ERR_LBND, CWISE_WRONG
      LOGICAL            IGNORE_CWISE
      INTEGER            ITHRESH
      REAL               RTHRESH, UNSTABLE_THRESH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, SSYCON, SLA_SYRFSX_EXTENDED
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, BLAS_FPINFO_X, ILATRANS, ILAPREC
      EXTERNAL           SLAMCH, SLANSY, SLA_SYRCOND
      REAL               SLAMCH, SLANSY, SLA_SYRCOND
      LOGICAL            LSAME
      INTEGER            BLAS_FPINFO_X
      INTEGER            ILATRANS, ILAPREC
*     ..
```

```
*     .. Executable Statements ..
*
*     Check the input parameters.
*
      INFO = 0
      REF_TYPE = INT( ITREF_DEFAULT )
      IF ( NPARAMS .GE. LA_LINRX_ITREF_I ) THEN
         IF ( PARAMS( LA_LINRX_ITREF_I ) .LT. 0.0 ) THEN
            PARAMS( LA_LINRX_ITREF_I ) = ITREF_DEFAULT
         ELSE
            REF_TYPE = PARAMS( LA_LINRX_ITREF_I )
         END IF
      END IF
*
*     Set default parameters.
*
      ILLRCOND_THRESH = REAL( N )*SLAMCH( 'Epsilon' )
      ITHRESH = INT( ITHRESH_DEFAULT )
      RTHRESH = RTHRESH_DEFAULT
      UNSTABLE_THRESH = DZTHRESH_DEFAULT
      IGNORE_CWISE = COMPONENTWISE_DEFAULT .EQ. 0.0
*
      IF ( NPARAMS.GE.LA_LINRX_ITHRESH_I ) THEN
         IF ( PARAMS( LA_LINRX_ITHRESH_I ).LT.0.0 ) THEN
            PARAMS( LA_LINRX_ITHRESH_I ) = ITHRESH
         ELSE
            ITHRESH = INT( PARAMS( LA_LINRX_ITHRESH_I ) )
         END IF
      END IF
      IF ( NPARAMS.GE.LA_LINRX_CWISE_I ) THEN
         IF ( PARAMS( LA_LINRX_CWISE_I ).LT.0.0 ) THEN
            IF ( IGNORE_CWISE ) THEN
               PARAMS( LA_LINRX_CWISE_I ) = 0.0
            ELSE
               PARAMS( LA_LINRX_CWISE_I ) = 1.0
            END IF
         ELSE
            IGNORE_CWISE = PARAMS( LA_LINRX_CWISE_I ) .EQ. 0.0
         END IF
      END IF
      IF ( REF_TYPE .EQ. 0 .OR. N_ERR_BNDS .EQ. 0 ) THEN
         N_NORMS = 0
      ELSE IF ( IGNORE_CWISE ) THEN
         N_NORMS = 1
      ELSE
         N_NORMS = 2
      END IF
*
      RCEQU = LSAME( EQUED, 'Y' )
*
```

```
*     Test input parameters.
*
      IF ( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
        INFO = -1
      ELSE IF( .NOT.RCEQU .AND. .NOT.LSAME( EQUED, 'N' ) ) THEN
        INFO = -2
      ELSE IF( N.LT.0 ) THEN
        INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
        INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
        INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
        INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
        INFO = -12
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
        INFO = -14
      END IF
      IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'SSYRFSX', -INFO )
        RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         RCOND = 1.0
         DO J = 1, NRHS
            BERR( J ) = 0.0
            IF ( N_ERR_BNDS .GE. 1 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
            IF ( N_ERR_BNDS .GE. 2 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 0.0
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 0.0
            END IF
            IF ( N_ERR_BNDS .GE. 3 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 1.0
            END IF
         END DO
         RETURN
      END IF
*
*     Default to failure.
*
      RCOND = 0.0
      DO J = 1, NRHS
```

```
         BERR( J ) = 1.0
         IF ( N_ERR_BNDS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
            ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
         END IF
         IF ( N_ERR_BNDS .GE. 2 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
         END IF
         IF ( N_ERR_BNDS .GE. 3 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 0.0
            ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 0.0
         END IF
      END DO
*
*     Compute the norm of A and the reciprocal of the condition
*     number of A.
*
      NORM = 'I'
      ANORM = SLANSY( NORM, UPLO, N, A, LDA, WORK )
      CALL SSYCON( UPLO, N, AF, LDAF, IPIV, ANORM, RCOND, WORK,
     $     IWORK, INFO )
*
*     Perform refinement on each right-hand side
*
      IF ( REF_TYPE .NE. 0 ) THEN

         PREC_TYPE = ILAPREC( 'D' )

         CALL SLA_SYRFSX_EXTENDED( PREC_TYPE, UPLO,  N,
     $        NRHS, A, LDA, AF, LDAF, IPIV, RCEQU, S, B,
     $        LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM, ERR_BNDS_COMP,
     $        WORK( N+1 ), WORK( 1 ), WORK( 2*N+1 ), WORK( 1 ), RCOND,
     $        ITHRESH, RTHRESH, UNSTABLE_THRESH, IGNORE_CWISE,
     $        INFO )
      END IF

      ERR_LBND = MAX( 10.0, SQRT( REAL( N ) ) )*SLAMCH( 'Epsilon' )
      IF (N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 1) THEN
*
*     Compute scaled normwise condition number cond(A*C).
*
         IF ( RCEQU ) THEN
            RCOND_TMP = SLA_SYRCOND( UPLO, N, A, LDA, AF, LDAF, IPIV,
     $           -1, S, INFO, WORK, IWORK )
         ELSE
            RCOND_TMP = SLA_SYRCOND( UPLO, N, A, LDA, AF, LDAF, IPIV,
     $           0, S, INFO, WORK, IWORK )
         END IF
         DO J = 1, NRHS
```

```
*
*     Cap the error at 1.0.
*
            IF (N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .GT. 1.0)
     $           ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 0.0
               IF ( INFO .LE. N ) INFO = N + J
            ELSE IF (ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .LT. ERR_LBND)
     $              THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
*
*     Save the condition number.
*
            IF (N_ERR_BNDS .GE. LA_LINRX_RCOND_I) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF
         END DO
      END IF

      IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 2 ) THEN
*
*     Compute componentwise condition number cond(A*diag(Y(:,J))) for
*     each right-hand side using the current solution as an estimate of
*     the true solution.  If the componentwise error estimate is too
*     large, then the solution is a lousy estimate of truth and the
*     estimated RCOND may be too optimistic.  To avoid misleading users,
*     the inverse condition number is set to 0.0 when the estimated
*     cwise error is at least CWISE_WRONG.
*
         CWISE_WRONG = SQRT( SLAMCH( 'Epsilon' ) )
         DO J = 1, NRHS
            IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .LT. CWISE_WRONG )
     $     THEN
               RCOND_TMP = SLA_SYRCOND( UPLO, N, A, LDA, AF, LDAF, IPIV,
     $              1, X(1,J), INFO, WORK, IWORK )
            ELSE
               RCOND_TMP = 0.0
            END IF
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
```

```
     $           .AND. ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .GT. 1.0 )
     $           ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 0.0
               IF ( .NOT. IGNORE_CWISE
     $              .AND. INFO.LT.N + J ) INFO = N + J
            ELSE IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I )
     $              .LT. ERR_LBND ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF

         END DO
      END IF
*
      RETURN
*
*     End of SSYRFSX
*
      END
```

———-

59.14.4 lisp code

— ssyrfsx.lisp —

———-

59.15 ssytd2

59.15.1 Axiom unit tests

— ssytd2.input —

```
)set break resume
)sys rm -f ssytd2.output
)spool ssytd2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.15.2 Axiom help page

— ssytd2.help —

```
======================================================================
ssytd2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssytd2 (UPLO, N, A, LDA, D, E, TAU, INFO)
           SSYTD2 reduces a symmetric matrix to real symmetric tridiagonal
           form by an orthogonal similarity transformation (unblocked
           algorithm).

Function/Subroutine Documentation
   subroutine ssytd2 (character UPLO, integer N, real, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) D, real, dimension( * ) E, real,
       dimension( * ) TAU, integer INFO)
       SSYTD2 reduces a symmetric matrix to real symmetric tridiagonal form by
       an orthogonal similarity transformation (unblocked algorithm).

Purpose:
   SSYTD2 reduces a real symmetric matrix A to symmetric tridiagonal
```

```
  form T by an orthogonal similarity transformation: Q**T * A * Q = T.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is stored:
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   n-by-n upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading n-by-n lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.
   On exit, if UPLO = 'U', the diagonal and first superdiagonal
   of A are overwritten by the corresponding elements of the
   tridiagonal matrix T, and the elements above the first
   superdiagonal, with the array TAU, represent the orthogonal
   matrix Q as a product of elementary reflectors; if UPLO
   = 'L', the diagonal and first subdiagonal of A are over-
   written by the corresponding elements of the tridiagonal
   matrix T, and the elements below the first subdiagonal, with
   the array TAU, represent the orthogonal matrix Q as a product
   of elementary reflectors. See Further Details.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 D
   D is REAL array, dimension (N)
   The diagonal elements of the tridiagonal matrix T:
   D(i) = A(i,i).

 E
   E is REAL array, dimension (N-1)
   The off-diagonal elements of the tridiagonal matrix T:
   E(i) = A(i,i+1) if UPLO = 'U', E(i) = A(i+1,i) if UPLO = 'L'.

 TAU
```

```
   TAU is REAL array, dimension (N-1)
   The scalar factors of the elementary reflectors (see Further Details).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value.

Further Details:

   If UPLO = 'U', the matrix Q is represented as a product of elementary
   reflectors

               Q = H(n-1) . . . H(2) H(1).

   Each H(i) has the form

               H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(i+1:n) = 0 and v(i) = 1; v(1:i-1) is stored on exit in
   A(1:i-1,i+1), and tau in TAU(i).

   If UPLO = 'L', the matrix Q is represented as a product of elementary
   reflectors

               Q = H(1) H(2) . . . H(n-1).

   Each H(i) has the form

               H(i) = I - tau * v * v**T

   where tau is a real scalar, and v is a real vector with
   v(1:i) = 0 and v(i+1) = 1; v(i+2:n) is stored on exit in A(i+2:n,i),
   and tau in TAU(i).

   The contents of A on exit are illustrated by the following examples
   with n = 5:

        if UPLO = 'U':                        if UPLO = 'L':

          (  d   e   v2  v3  v4 )               (  d                  )
          (      d   e   v3  v4 )               (  e   d              )
          (          d   e   v4 )               (  v1  e   d          )
          (              d   e  )               (  v1  v2  e   d      )
          (                  d  )               (  v1  v2  v3  e   d  )

   where d and e denote diagonal and off-diagonal elements of T, and vi
   denotes an element of the vector defining H(i).
```

———-

59.15.3 fortran code

— **ssytd2.f** —

```
*  =====================================================================
      SUBROUTINE SSYTD2( UPLO, N, A, LDA, D, E, TAU, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), D( * ), E( * ), TAU( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO, HALF
      PARAMETER          ( ONE = 1.0, ZERO = 0.0, HALF = 1.0 / 2.0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I
      REAL               ALPHA, TAUI
*     ..
*     .. External Subroutines ..
      EXTERNAL           SAXPY, SLARFG, SSYMV, SSYR2, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SDOT
      EXTERNAL           LSAME, SDOT
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
```

```
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYTD2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.LE.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*        Reduce the upper triangle of A
*
         DO 10 I = N - 1, 1, -1
*
*           Generate elementary reflector H(i) = I - tau * v * v**T
*           to annihilate A(1:i-1,i+1)
*
            CALL SLARFG( I, A( I, I+1 ), A( 1, I+1 ), 1, TAUI )
            E( I ) = A( I, I+1 )
*
            IF( TAUI.NE.ZERO ) THEN
*
               Apply H(i) from both sides to A(1:i,1:i)
*
               A( I, I+1 ) = ONE
*
               Compute  x := tau * A * v  storing x in TAU(1:i)
*
               CALL SSYMV( UPLO, I, TAUI, A, LDA, A( 1, I+1 ), 1, ZERO,
     $                     TAU, 1 )
*
               Compute  w := x - 1/2 * tau * (x**T * v) * v
*
               ALPHA = -HALF*TAUI*SDOT( I, TAU, 1, A( 1, I+1 ), 1 )
               CALL SAXPY( I, ALPHA, A( 1, I+1 ), 1, TAU, 1 )
*
               Apply the transformation as a rank-2 update:
*                 A := A - v * w**T - w * v**T
*
```

```
               CALL SSYR2( UPLO, I, -ONE, A( 1, I+1 ), 1, TAU, 1, A,
     $                     LDA )
*
               A( I, I+1 ) = E( I )
            END IF
            D( I+1 ) = A( I+1, I+1 )
            TAU( I ) = TAUI
   10    CONTINUE
         D( 1 ) = A( 1, 1 )
      ELSE
*
*        Reduce the lower triangle of A
*
         DO 20 I = 1, N - 1
*
*           Generate elementary reflector H(i) = I - tau * v * v**T
*           to annihilate A(i+2:n,i)
*
            CALL SLARFG( N-I, A( I+1, I ), A( MIN( I+2, N ), I ), 1,
     $                   TAUI )
            E( I ) = A( I+1, I )
*
            IF( TAUI.NE.ZERO ) THEN
*
*              Apply H(i) from both sides to A(i+1:n,i+1:n)
*
               A( I+1, I ) = ONE
*
*              Compute  x := tau * A * v  storing y in TAU(i:n-1)
*
               CALL SSYMV( UPLO, N-I, TAUI, A( I+1, I+1 ), LDA,
     $                     A( I+1, I ), 1, ZERO, TAU( I ), 1 )
*
*              Compute  w := x - 1/2 * tau * (x**T * v) * v
*
               ALPHA = -HALF*TAUI*SDOT( N-I, TAU( I ), 1, A( I+1, I ),
     $                 1 )
               CALL SAXPY( N-I, ALPHA, A( I+1, I ), 1, TAU( I ), 1 )
*
*              Apply the transformation as a rank-2 update:
*                 A := A - v * w**T - w * v**T
*
               CALL SSYR2( UPLO, N-I, -ONE, A( I+1, I ), 1, TAU( I ), 1,
     $                     A( I+1, I+1 ), LDA )
*
               A( I+1, I ) = E( I )
            END IF
            D( I ) = A( I, I )
            TAU( I ) = TAUI
   20    CONTINUE
```

```
         D( N ) = A( N, N )
      END IF
*
      RETURN
*
*     End of SSYTD2
*
      END
```

———-

59.15.4 lisp code

— **ssytd2.lisp** —

```
(let* ((one 1.0) (zero 0.0) (half (f2cl-lib:f2cl/ 1.0 2.0)))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (type (single-float) half)
           (ignorable one zero half))
  (defun ssytd2 (uplo n a lda d e tau info)
    (declare (type (array single-float (*)) tau e d a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%)
         (tau single-float tau-%data% tau-%offset%))
      (prog ((alpha 0.0) (taui 0.0) (i 0) (upper nil))
        (declare (type (single-float) alpha taui)
                 (type (f2cl-lib:integer4) i)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
```

```
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4)))
(cond
 ((/= info 0) (xerbla "SSYTD2" (f2cl-lib:int-sub info))
  (go end_label)))
(if (<= n 0)
    (go end_label))
(cond
 (upper
  (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                 (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                ((> i 1) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (slarfg i
           (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
                          ((1 lda) (1 *)) a-%offset%)
           (f2cl-lib:array-slice a-%data% single-float
                                 (1 (f2cl-lib:int-add i 1))
                                 ((1 lda) (1 *)) a-%offset%)
           1 taui)
        (declare (ignore var-0 var-2 var-3))
        (setf (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
                             ((1 lda) (1 *)) a-%offset%)
              var-1)
        (setf taui var-4))
      (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
            (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
                             ((1 lda) (1 *)) a-%offset%))
      (cond
       ((/= taui zero)
        (setf (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
                             ((1 lda) (1 *)) a-%offset%)
              one)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9)
            (ssymv uplo i taui a lda
             (f2cl-lib:array-slice a-%data% single-float
                                   (1 (f2cl-lib:int-add i 1))
                                   ((1 lda) (1 *)) a-%offset%)
             1 zero tau 1)
          (declare (ignore var-3 var-5 var-6 var-8 var-9))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf i var-1))
          (when var-2 (setf taui var-2))
          (when var-4 (setf lda var-4))
```

```
          (when var-7 (setf zero var-7)))
        (setf alpha
                (* (- half) taui
                   (multiple-value-bind
                       (ret-val var-0 var-1 var-2 var-3 var-4)
                       (sdot i tau 1
                        (f2cl-lib:array-slice a-%data% single-float
                                              (1
                                               (f2cl-lib:int-add i 1))
                                              ((1 lda) (1 *))
                                              a-%offset%)
                        1)
                      (declare (ignore var-1 var-2 var-3 var-4))
                      (when var-0 (setf i var-0))
                      ret-val)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (saxpy i alpha
             (f2cl-lib:array-slice a-%data% single-float
                                   (1 (f2cl-lib:int-add i 1))
                                   ((1 lda) (1 *)) a-%offset%)
             1 tau 1)
          (declare (ignore var-2 var-3 var-4 var-5))
          (when var-0 (setf i var-0))
          (when var-1 (setf alpha var-1)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (ssyr2 uplo i (- one)
             (f2cl-lib:array-slice a-%data% single-float
                                   (1 (f2cl-lib:int-add i 1))
                                   ((1 lda) (1 *)) a-%offset%)
             1 tau 1 a lda)
          (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf i var-1))
          (when var-8 (setf lda var-8)))
        (setf (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
                             ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))))
      (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                           d-%offset%)
              (f2cl-lib:fref a-%data%
                             ((f2cl-lib:int-add i 1)
                              (f2cl-lib:int-add i 1))
                             ((1 lda) (1 *)) a-%offset%))
      (setf (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%) taui)
     label10))
  (setf (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
        (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
            ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
(tagbody
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slarfg (f2cl-lib:int-sub n i)
       (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                      ((1 lda) (1 *)) a-%offset%)
       (f2cl-lib:array-slice a-%data% single-float
                             ((min (f2cl-lib:int-add i 2) n) i)
                             ((1 lda) (1 *)) a-%offset%)
       1 taui)
    (declare (ignore var-0 var-2 var-3))
    (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                         ((1 lda) (1 *)) a-%offset%)
          var-1)
    (setf taui var-4))
  (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
        (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                         ((1 lda) (1 *)) a-%offset%))
  (cond
   ((/= taui zero)
    (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                         ((1 lda) (1 *)) a-%offset%)
          one)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9)
        (ssymv uplo (f2cl-lib:int-sub n i) taui
         (f2cl-lib:array-slice a-%data% single-float
                               ((+ i 1) (f2cl-lib:int-add i 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice a-%data% single-float ((+ i 1) i)
                               ((1 lda) (1 *)) a-%offset%)
         1 zero
         (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                               tau-%offset%)
         1)
      (declare (ignore var-1 var-3 var-5 var-6 var-8 var-9))
      (when var-0 (setf uplo var-0))
      (when var-2 (setf taui var-2))
      (when var-4 (setf lda var-4))
      (when var-7 (setf zero var-7)))
    (setf alpha
            (* (- half) taui
               (sdot (f2cl-lib:int-sub n i)
                (f2cl-lib:array-slice tau-%data% single-float (i)
                                      ((1 *)) tau-%offset%)
                1
                (f2cl-lib:array-slice a-%data% single-float
                                      ((+ i 1) i) ((1 lda) (1 *))
```

```
                                                a-%offset%)
                          1)))
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                    (saxpy (f2cl-lib:int-sub n i) alpha
                     (f2cl-lib:array-slice a-%data% single-float ((+ i 1) i)
                                           ((1 lda) (1 *)) a-%offset%)
                     1
                     (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                           tau-%offset%)
                     1)
                  (declare (ignore var-0 var-2 var-3 var-4 var-5))
                  (when var-1 (setf alpha var-1)))
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                    (ssyr2 uplo (f2cl-lib:int-sub n i) (- one)
                     (f2cl-lib:array-slice a-%data% single-float ((+ i 1) i)
                                           ((1 lda) (1 *)) a-%offset%)
                     1
                     (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                           tau-%offset%)
                     1
                     (f2cl-lib:array-slice a-%data% single-float
                                           ((+ i 1) (f2cl-lib:int-add i 1))
                                           ((1 lda) (1 *)) a-%offset%)
                     lda)
                  (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7))
                  (when var-0 (setf uplo var-0))
                  (when var-8 (setf lda var-8)))
                (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                                      ((1 lda) (1 *)) a-%offset%)
                        (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))))
              (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                      (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                      a-%offset%))
              (setf (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%) taui)
            label20))
          (setf (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                  (f2cl-lib:fref a-%data% (n n) ((1 lda) (1 *)) a-%offset%))))
       (go end_label)
      end_label
       (return (values uplo nil nil lda nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssytd2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
```

```
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::uplo nil nil
  fortran-to-lisp::lda nil nil nil
  fortran-to-lisp::info)
:calls '(fortran-to-lisp::slarfg))))
```

———-

59.16 ssytf2

59.16.1 Axiom unit tests

— **ssytf2.input** —

```
)set break resume
)sys rm -f ssytf2.output
)spool ssytf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.16.2 Axiom help page

— **ssytf2.help** —

```
==================================================================
ssytf2 examples
==================================================================

==================================================================
Man Page Details
```

```
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssytf2 (UPLO, N, A, LDA, IPIV, INFO)
           SSYTF2 computes the factorization of a real symmetric indefinite
           matrix, using the diagonal pivoting method (unblocked algorithm).

Function/Subroutine Documentation
   subroutine ssytf2 (character UPLO, integer N, real, dimension( lda, * ) A,
       integer LDA, integer, dimension( * ) IPIV, integer INFO)
       SSYTF2 computes the factorization of a real symmetric indefinite
       matrix, using the diagonal pivoting method (unblocked algorithm).

Purpose:
   SSYTF2 computes the factorization of a real symmetric matrix A using
   the Bunch-Kaufman diagonal pivoting method:

               A = U*D*U**T  or  A = L*D*L**T

   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, U**T is the transpose of U, and D is symmetric and
   block diagonal with 1-by-1 and 2-by-2 diagonal blocks.

   This is the unblocked version of the algorithm, calling Level 2 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is stored:
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   n-by-n upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading n-by-n lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

   On exit, the block diagonal matrix D and the multipliers used
```

```
   to obtain the factor U or L (see below for further details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D.

   If UPLO = 'U':
      If IPIV(k) > 0, then rows and columns k and IPIV(k) were
      interchanged and D(k,k) is a 1-by-1 diagonal block.

      If IPIV(k) = IPIV(k-1) < 0, then rows and columns
      k-1 and -IPIV(k) were interchanged and D(k-1:k,k-1:k)
      is a 2-by-2 diagonal block.

   If UPLO = 'L':
      If IPIV(k) > 0, then rows and columns k and IPIV(k) were
      interchanged and D(k,k) is a 1-by-1 diagonal block.

      If IPIV(k) = IPIV(k+1) < 0, then rows and columns
      k+1 and -IPIV(k) were interchanged and D(k:k+1,k:k+1)
      is a 2-by-2 diagonal block.

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -k, the k-th argument had an illegal value
   > 0: if INFO = k, D(k,k) is exactly zero.  The factorization
      has been completed, but the block diagonal matrix D is
      exactly singular, and division by zero will occur if it
      is used to solve a system of equations.

Further Details:

   If UPLO = 'U', then A = U*D*U**T, where
      U = P(n)*U(n)* ... *P(k)U(k)* ...,
   i.e., U is a product of terms P(k)*U(k), where k decreases from n to
   1 in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and U(k) is a unit upper triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                         (   I    v    0   )   k-s
                U(k) =   (   0    I    0   )   s
                         (   0    0    I   )   n-k
                            k-s   s   n-k
```

```
If s = 1, D(k) overwrites A(k,k), and v overwrites A(1:k-1,k).
If s = 2, the upper triangle of D(k) overwrites A(k-1,k-1), A(k-1,k),
and A(k,k), and v overwrites A(1:k-2,k-1:k).

If UPLO = 'L', then A = L*D*L**T, where
   L = P(1)*L(1)* ... *P(k)*L(k)* ...,
i.e., L is a product of terms P(k)*L(k), where k increases from 1 to
n in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
defined by IPIV(k), and L(k) is a unit lower triangular matrix, such
that if the diagonal block D(k) is of order s (s = 1 or 2), then

                     (   I    0     0   )  k-1
             L(k) =  (   0    I     0   )  s
                     (   0    v     I   )  n-k-s+1
                        k-1   s  n-k-s+1

If s = 1, D(k) overwrites A(k,k), and v overwrites A(k+1:n,k).
If s = 2, the lower triangle of D(k) overwrites A(k,k), A(k+1,k),
and A(k+1,k+1), and v overwrites A(k+2:n,k:k+1).
```

———-

59.16.3 fortran code

— **ssytf2.f** —

```
*  =====================================================================
      SUBROUTINE SSYTF2( UPLO, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
```

```
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               EIGHT, SEVTEN
      PARAMETER          ( EIGHT = 8.0E+0, SEVTEN = 17.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, IMAX, J, JMAX, K, KK, KP, KSTEP
      REAL               ABSAKK, ALPHA, COLMAX, D11, D12, D21, D22, R1,
     $                   ROWMAX, T, WK, WKM1, WKP1
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, SISNAN
      INTEGER            ISAMAX
      EXTERNAL           LSAME, ISAMAX, SISNAN
*     ..
*     .. External Subroutines ..
      EXTERNAL           SSCAL, SSWAP, SSYR, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYTF2', -INFO )
         RETURN
      END IF
*
*     Initialize ALPHA for use in choosing pivot block size.
*
      ALPHA = ( ONE+SQRT( SEVTEN ) ) / EIGHT
*
      IF( UPPER ) THEN
*
*        Factorize A as U*D*U**T using the upper triangle of A
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        1 or 2
```

```
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop
*
         IF( K.LT.1 )
     $      GO TO 70
         KSTEP = 1
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = ABS( A( K, K ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.GT.1 ) THEN
            IMAX = ISAMAX( K-1, A( 1, K ), 1 )
            COLMAX = ABS( A( IMAX, K ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( (MAX( ABSAKK, COLMAX ).EQ.ZERO) .OR. SISNAN(ABSAKK) ) THEN
*
*           Column K is zero or underflow, or contains a NaN:
*           set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
         ELSE
            IF( ABSAKK.GE.ALPHA*COLMAX ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
            ELSE
*
*              JMAX is the column-index of the largest off-diagonal
*              element in row IMAX, and ROWMAX is its absolute value
*
               JMAX = IMAX + ISAMAX( K-IMAX, A( IMAX, IMAX+1 ), LDA )
               ROWMAX = ABS( A( IMAX, JMAX ) )
               IF( IMAX.GT.1 ) THEN
                  JMAX = ISAMAX( IMAX-1, A( 1, IMAX ), 1 )
                  ROWMAX = MAX( ROWMAX, ABS( A( JMAX, IMAX ) ) )
```

```
               END IF
*
               IF( ABSAKK.GE.ALPHA*COLMAX*( COLMAX / ROWMAX ) ) THEN
*
*                 no interchange, use 1-by-1 pivot block
*
                  KP = K
               ELSE IF( ABS( A( IMAX, IMAX ) ).GE.ALPHA*ROWMAX ) THEN
*
*                 interchange rows and columns K and IMAX, use 1-by-1
*                 pivot block
*
                  KP = IMAX
               ELSE
*
*                 interchange rows and columns K-1 and IMAX, use 2-by-2
*                 pivot block
*
                  KP = IMAX
                  KSTEP = 2
               END IF
            END IF
*
            KK = K - KSTEP + 1
            IF( KP.NE.KK ) THEN
*
*              Interchange rows and columns KK and KP in the leading
*              submatrix A(1:k,1:k)
*
               CALL SSWAP( KP-1, A( 1, KK ), 1, A( 1, KP ), 1 )
               CALL SSWAP( KK-KP-1, A( KP+1, KK ), 1, A( KP, KP+1 ),
     $                     LDA )
               T = A( KK, KK )
               A( KK, KK ) = A( KP, KP )
               A( KP, KP ) = T
               IF( KSTEP.EQ.2 ) THEN
                  T = A( K-1, K )
                  A( K-1, K ) = A( KP, K )
                  A( KP, K ) = T
               END IF
            END IF
*
*           Update the leading submatrix
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column k now holds
*
*              W(k) = U(k)*D(k)
*
```

```
*              where U(k) is the k-th column of U
*
*              Perform a rank-1 update of A(1:k-1,1:k-1) as
*
*              A := A - U(k)*D(k)*U(k)**T = A - W(k)*1/D(k)*W(k)**T
*
               R1 = ONE / A( K, K )
               CALL SSYR( UPLO, K-1, -R1, A( 1, K ), 1, A, LDA )
*
*              Store U(k) in column k
*
               CALL SSCAL( K-1, R1, A( 1, K ), 1 )
            ELSE
*
*              2-by-2 pivot block D(k): columns k and k-1 now hold
*
*              ( W(k-1) W(k) ) = ( U(k-1) U(k) )*D(k)
*
*              where U(k) and U(k-1) are the k-th and (k-1)-th columns
*              of U
*
*              Perform a rank-2 update of A(1:k-2,1:k-2) as
*
*              A := A - ( U(k-1) U(k) )*D(k)*( U(k-1) U(k) )**T
*                 = A - ( W(k-1) W(k) )*inv(D(k))*( W(k-1) W(k) )**T
*
               IF( K.GT.2 ) THEN
*
                  D12 = A( K-1, K )
                  D22 = A( K-1, K-1 ) / D12
                  D11 = A( K, K ) / D12
                  T = ONE / ( D11*D22-ONE )
                  D12 = T / D12
*
                  DO 30 J = K - 2, 1, -1
                     WKM1 = D12*( D11*A( J, K-1 )-A( J, K ) )
                     WK = D12*( D22*A( J, K )-A( J, K-1 ) )
                     DO 20 I = J, 1, -1
                        A( I, J ) = A( I, J ) - A( I, K )*WK -
     $                              A( I, K-1 )*WKM1
   20                CONTINUE
                     A( J, K ) = WK
                     A( J, K-1 ) = WKM1
   30             CONTINUE
*
               END IF
*
            END IF
         END IF
*
```

```
*        Store details of the interchanges in IPIV
*
         IF( KSTEP.EQ.1 ) THEN
            IPIV( K ) = KP
         ELSE
            IPIV( K ) = -KP
            IPIV( K-1 ) = -KP
         END IF
*
*        Decrease K and return to the start of the main loop
*
         K = K - KSTEP
         GO TO 10
*
      ELSE
*
*        Factorize A as L*D*L**T using the lower triangle of A
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2
*
         K = 1
   40    CONTINUE
*
*        If K > N, exit from loop
*
         IF( K.GT.N )
     $      GO TO 70
         KSTEP = 1
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = ABS( A( K, K ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.LT.N ) THEN
            IMAX = K + ISAMAX( N-K, A( K+1, K ), 1 )
            COLMAX = ABS( A( IMAX, K ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( (MAX( ABSAKK, COLMAX ).EQ.ZERO) .OR. SISNAN(ABSAKK) ) THEN
*
*           Column K is zero or underflow, or contains a NaN:
*           set INFO and continue
```

```
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
         ELSE
            IF( ABSAKK.GE.ALPHA*COLMAX ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
            ELSE
*
*              JMAX is the column-index of the largest off-diagonal
*              element in row IMAX, and ROWMAX is its absolute value
*
               JMAX = K - 1 + ISAMAX( IMAX-K, A( IMAX, K ), LDA )
               ROWMAX = ABS( A( IMAX, JMAX ) )
               IF( IMAX.LT.N ) THEN
                  JMAX = IMAX + ISAMAX( N-IMAX, A( IMAX+1, IMAX ), 1 )
                  ROWMAX = MAX( ROWMAX, ABS( A( JMAX, IMAX ) ) )
               END IF
*
               IF( ABSAKK.GE.ALPHA*COLMAX*( COLMAX / ROWMAX ) ) THEN
*
*                 no interchange, use 1-by-1 pivot block
*
                  KP = K
               ELSE IF( ABS( A( IMAX, IMAX ) ).GE.ALPHA*ROWMAX ) THEN
*
*                 interchange rows and columns K and IMAX, use 1-by-1
*                 pivot block
*
                  KP = IMAX
               ELSE
*
*                 interchange rows and columns K+1 and IMAX, use 2-by-2
*                 pivot block
*
                  KP = IMAX
                  KSTEP = 2
               END IF
            END IF
*
            KK = K + KSTEP - 1
            IF( KP.NE.KK ) THEN
*
*              Interchange rows and columns KK and KP in the trailing
*              submatrix A(k:n,k:n)
*
               IF( KP.LT.N )
```

```
     $           CALL SSWAP( N-KP, A( KP+1, KK ), 1, A( KP+1, KP ), 1 )
              CALL SSWAP( KP-KK-1, A( KK+1, KK ), 1, A( KP, KK+1 ),
     $                    LDA )
              T = A( KK, KK )
              A( KK, KK ) = A( KP, KP )
              A( KP, KP ) = T
              IF( KSTEP.EQ.2 ) THEN
                 T = A( K+1, K )
                 A( K+1, K ) = A( KP, K )
                 A( KP, K ) = T
              END IF
           END IF
*
*          Update the trailing submatrix
*
           IF( KSTEP.EQ.1 ) THEN
*
*             1-by-1 pivot block D(k): column k now holds
*
*             W(k) = L(k)*D(k)
*
*             where L(k) is the k-th column of L
*
              IF( K.LT.N ) THEN
*
*                Perform a rank-1 update of A(k+1:n,k+1:n) as
*
*                A := A - L(k)*D(k)*L(k)**T = A - W(k)*(1/D(k))*W(k)**T
*
                 D11 = ONE / A( K, K )
                 CALL SSYR( UPLO, N-K, -D11, A( K+1, K ), 1,
     $                      A( K+1, K+1 ), LDA )
*
*                Store L(k) in column K
*
                 CALL SSCAL( N-K, D11, A( K+1, K ), 1 )
              END IF
           ELSE
*
*             2-by-2 pivot block D(k)
*
              IF( K.LT.N-1 ) THEN
*
*                Perform a rank-2 update of A(k+2:n,k+2:n) as
*
*                A := A - ( (A(k) A(k+1))*D(k)**(-1) ) * (A(k) A(k+1))**T
*
*                where L(k) and L(k+1) are the k-th and (k+1)-th
*                columns of L
*
```

```
                 D21 = A( K+1, K )
                 D11 = A( K+1, K+1 ) / D21
                 D22 = A( K, K ) / D21
                 T = ONE / ( D11*D22-ONE )
                 D21 = T / D21
*
                 DO 60 J = K + 2, N
*
                    WK = D21*( D11*A( J, K )-A( J, K+1 ) )
                    WKP1 = D21*( D22*A( J, K+1 )-A( J, K ) )
*
                    DO 50 I = J, N
                       A( I, J ) = A( I, J ) - A( I, K )*WK -
     $                             A( I, K+1 )*WKP1
   50               CONTINUE
*
                    A( J, K ) = WK
                    A( J, K+1 ) = WKP1
*
   60            CONTINUE
              END IF
           END IF
        END IF
*
*       Store details of the interchanges in IPIV
*
        IF( KSTEP.EQ.1 ) THEN
           IPIV( K ) = KP
        ELSE
           IPIV( K ) = -KP
           IPIV( K+1 ) = -KP
        END IF
*
*       Increase K and return to the start of the main loop
*
        K = K + KSTEP
        GO TO 40
*
      END IF
*
   70 CONTINUE
*
      RETURN
*
*     End of SSYTF2
*
      END
```

59.16.4 lisp code

— ssytf2.lisp —

```
(let* ((zero 0.0) (one 1.0) (eight 8.0) (sevten 17.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 8.0 8.0) eight)
           (type (single-float 17.0 17.0) sevten)
           (ignorable zero one eight sevten))
  (defun ssytf2 (uplo n a lda ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array single-float (*)) a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((absakk 0.0) (alpha 0.0) (colmax 0.0) (d11 0.0) (d12 0.0)
             (d21 0.0) (d22 0.0) (r1 0.0) (rowmax 0.0) (t$ 0.0) (wk 0.0)
             (wkm1 0.0) (wkp1 0.0) (i 0) (imax 0) (j 0) (jmax 0) (k 0) (kk 0)
             (kp 0) (kstep 0) (upper nil))
        (declare
         (type (single-float) absakk alpha colmax d11 d12 d21 d22 r1 rowmax t$
          wk wkm1 wkp1)
         (type (f2cl-lib:integer4) i imax j jmax k kk kp kstep)
         (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -4)))
        (cond
         ((/= info 0) (xerbla "SSYTF2" (f2cl-lib:int-sub info))
```

```
  (go end_label)))
(setf alpha (/ (+ one (f2cl-lib:fsqrt sevten)) eight))
(cond
 (upper
  (tagbody
    (setf k n)
   label10
    (if (< k 1)
        (go label70))
    (setf kstep 1)
    (setf absakk
            (abs
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)))
    (cond
     ((> k 1)
      (setf imax
              (isamax (f2cl-lib:int-sub k 1)
               (f2cl-lib:array-slice a-%data% single-float (1 k)
                                     ((1 lda) (1 *)) a-%offset%)
               1))
      (setf colmax
              (abs
               (f2cl-lib:fref a-%data% (imax k) ((1 lda) (1 *))
                              a-%offset%))))
     (t (setf colmax zero)))
    (cond
     ((or (= (max absakk colmax) zero) (sisnan absakk))
      (if (= info 0)
          (setf info k))
      (setf kp k))
     (t
      (cond ((>= absakk (* alpha colmax)) (setf kp k))
            (t
             (setf jmax
                     (f2cl-lib:int-add imax
                                       (multiple-value-bind
                                           (ret-val var-0 var-1 var-2)
                                           (isamax
                                            (f2cl-lib:int-sub k imax)
                                            (f2cl-lib:array-slice
                                             a-%data% single-float
                                             (imax
                                              (f2cl-lib:int-add imax
                                                                1))
                                             ((1 lda) (1 *))
                                             a-%offset%)
                                            lda)
                                         (declare (ignore var-0 var-1))
                                         (when var-2 (setf lda var-2))
```

```
                                ret-val)))
        (setf rowmax
                (abs
                 (f2cl-lib:fref a-%data% (imax jmax)
                                ((1 lda) (1 *)) a-%offset%)))
        (cond
         ((> imax 1)
          (setf jmax
                  (isamax (f2cl-lib:int-sub imax 1)
                   (f2cl-lib:array-slice a-%data% single-float
                                         (1 imax) ((1 lda) (1 *))
                                         a-%offset%)
                   1))
          (setf rowmax
                  (max rowmax
                       (abs
                        (f2cl-lib:fref a-%data% (jmax imax)
                                       ((1 lda) (1 *))
                                       a-%offset%))))))
        (cond
         ((>= absakk
              (* alpha colmax (f2cl-lib:f2cl/ colmax rowmax)))
          (setf kp k))
         ((>= (abs (f2cl-lib:fref a (imax imax) ((1 lda) (1 *))))
              (* alpha rowmax))
          (setf kp imax))
         (t (setf kp imax) (setf kstep 2)))))
(setf kk (f2cl-lib:int-add (f2cl-lib:int-sub k kstep) 1))
(cond
 ((/= kp kk)
  (sswap (f2cl-lib:int-sub kp 1)
   (f2cl-lib:array-slice a-%data% single-float (1 kk)
                         ((1 lda) (1 *)) a-%offset%)
   1
   (f2cl-lib:array-slice a-%data% single-float (1 kp)
                         ((1 lda) (1 *)) a-%offset%)
   1)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (sswap (f2cl-lib:int-sub kk kp 1)
       (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) kk)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% single-float
                             (kp (f2cl-lib:int-add kp 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (setf t$
          (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
```

```
                          a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                       a-%offset%)
          (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                          a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                       a-%offset%)
          t$)
  (cond
   ((= kstep 2)
    (setf t$
            (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                            ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                          ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                            a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                          a-%offset%)
            t$)))))
(cond
 ((= kstep 1)
  (setf r1
          (/ one
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                             a-%offset%)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (ssyr uplo (f2cl-lib:int-sub k 1) (- r1)
       (f2cl-lib:array-slice a-%data% single-float (1 k)
                              ((1 lda) (1 *)) a-%offset%)
       1 a lda)
    (declare (ignore var-1 var-2 var-3 var-4 var-5))
    (when var-0 (setf uplo var-0))
    (when var-6 (setf lda var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (sscal (f2cl-lib:int-sub k 1) r1
       (f2cl-lib:array-slice a-%data% single-float (1 k)
                              ((1 lda) (1 *)) a-%offset%)
       1)
    (declare (ignore var-0 var-2 var-3))
    (when var-1 (setf r1 var-1))))
 (t
  (cond
   ((> k 2)
    (setf d12
            (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                            ((1 lda) (1 *)) a-%offset%))
    (setf d22
            (/
```

```
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-sub k 1)
                         (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         d12))
(setf d11
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         d12))
(setf t$ (/ one (- (* d11 d22) one))) (setf d12 (/ t$ d12))
(f2cl-lib:fdo (j (f2cl-lib:int-add k (f2cl-lib:int-sub 2))
               (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j 1) nil)
  (tagbody
    (setf wkm1
            (* d12
               (-
                (* d11
                   (f2cl-lib:fref a-%data%
                                  (j (f2cl-lib:int-sub k 1))
                                  ((1 lda) (1 *))
                                  a-%offset%))
                (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                               a-%offset%))))
    (setf wk
            (* d12
               (-
                (* d22
                   (f2cl-lib:fref a-%data% (j k)
                                  ((1 lda) (1 *))
                                  a-%offset%))
                (f2cl-lib:fref a-%data%
                               (j (f2cl-lib:int-sub k 1))
                               ((1 lda) (1 *)) a-%offset%))))
    (f2cl-lib:fdo (i j
                   (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                             a-%offset%)
              (-
               (f2cl-lib:fref a-%data% (i j)
                              ((1 lda) (1 *)) a-%offset%)
               (*
                (f2cl-lib:fref a-%data% (i k)
                               ((1 lda) (1 *)) a-%offset%)
                wk)
               (*
                (f2cl-lib:fref a-%data%
```

```
                                                (i (f2cl-lib:int-sub k 1))
                                                ((1 lda) (1 *)) a-%offset%)
                                  wkm1)))
                        label20))
                     (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                          a-%offset%)
                           wk)
                     (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-sub k 1))
                                          ((1 lda) (1 *)) a-%offset%)
                           wkm1)
                  label30))))))))
          (cond
           ((= kstep 1)
            (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%) kp))
           (t
            (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
                  (f2cl-lib:int-sub kp))
            (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                                 ipiv-%offset%)
                  (f2cl-lib:int-sub kp))))
          (setf k (f2cl-lib:int-sub k kstep))
          (go label10)))
        (t
         (tagbody
          (setf k 1)
         label40
          (if (> k n)
              (go label70))
          (setf kstep 1)
          (setf absakk
                (abs
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%)))
          (cond
           ((< k n)
            (setf imax
                  (f2cl-lib:int-add k
                                    (isamax (f2cl-lib:int-sub n k)
                                     (f2cl-lib:array-slice a-%data%
                                                           single-float
                                                           ((+ k 1) k)
                                                           ((1 lda) (1 *))
                                                           a-%offset%)
                                     1)))
            (setf colmax
                  (abs
                   (f2cl-lib:fref a-%data% (imax k) ((1 lda) (1 *))
                                  a-%offset%))))
           (t (setf colmax zero)))
          (cond
```

```
((or (= (max absakk colmax) zero) (sisnan absakk))
 (if (= info 0)
     (setf info k))
 (setf kp k))
(t
 (cond ((>= absakk (* alpha colmax)) (setf kp k))
       (t
        (setf jmax
                (f2cl-lib:int-add (f2cl-lib:int-sub k 1)
                                  (multiple-value-bind
                                      (ret-val var-0 var-1 var-2)
                                      (isamax
                                       (f2cl-lib:int-sub imax k)
                                       (f2cl-lib:array-slice
                                        a-%data% single-float
                                        (imax k) ((1 lda) (1 *))
                                        a-%offset%)
                                       lda)
                                    (declare (ignore var-0 var-1))
                                    (when var-2 (setf lda var-2))
                                    ret-val)))
        (setf rowmax
                (abs
                 (f2cl-lib:fref a-%data% (imax jmax)
                                ((1 lda) (1 *)) a-%offset%)))
        (cond
         ((< imax n)
          (setf jmax
                  (f2cl-lib:int-add imax
                                    (isamax
                                     (f2cl-lib:int-sub n imax)
                                     (f2cl-lib:array-slice
                                      a-%data% single-float
                                      ((+ imax 1) imax)
                                      ((1 lda) (1 *)) a-%offset%)
                                     1)))
          (setf rowmax
                  (max rowmax
                       (abs
                        (f2cl-lib:fref a-%data% (jmax imax)
                                       ((1 lda) (1 *))
                                       a-%offset%))))))
        (cond
         ((>= absakk
              (* alpha colmax (f2cl-lib:f2cl/ colmax rowmax)))
          (setf kp k))
         ((>= (abs (f2cl-lib:fref a (imax imax) ((1 lda) (1 *))))
              (* alpha rowmax))
          (setf kp imax))
         (t (setf kp imax) (setf kstep 2)))))
```

```
(setf kk (f2cl-lib:int-sub (f2cl-lib:int-add k kstep) 1))
(cond
 ((/= kp kk)
  (if (< kp n)
      (sswap (f2cl-lib:int-sub n kp)
       (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) kk)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) kp)
                             ((1 lda) (1 *)) a-%offset%)
       1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (sswap (f2cl-lib:int-sub kp kk 1)
       (f2cl-lib:array-slice a-%data% single-float ((+ kk 1) kk)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% single-float
                             (kp (f2cl-lib:int-add kk 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (setf t$
          (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                       a-%offset%)
          (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                       a-%offset%)
          t$)
  (cond
   ((= kstep 2)
    (setf t$
            (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                           ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                         ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                         a-%offset%)
            t$)))))
(cond
 ((= kstep 1)
  (cond
   ((< k n)
    (setf d11
            (/ one
```

```
                (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                               a-%offset%)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (ssyr uplo (f2cl-lib:int-sub n k) (- d11)
         (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% single-float
                               ((+ k 1) (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-1 var-2 var-3 var-4 var-5))
      (when var-0 (setf uplo var-0))
      (when var-6 (setf lda var-6)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal (f2cl-lib:int-sub n k) d11
         (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                               ((1 lda) (1 *)) a-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3))
      (when var-1 (setf d11 var-1))))))
(t
 (cond
  ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
   (setf d21
           (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                          ((1 lda) (1 *)) a-%offset%))
   (setf d11
           (/
            (f2cl-lib:fref a-%data%
                           ((f2cl-lib:int-add k 1)
                            (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
            d21))
   (setf d22
           (/
            (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                           a-%offset%)
            d21))
   (setf t$ (/ one (- (* d11 d22) one))) (setf d21 (/ t$ d21))
   (f2cl-lib:fdo (j (f2cl-lib:int-add k 2)
                  (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf wk
               (* d21
                  (-
                   (* d11
                      (f2cl-lib:fref a-%data% (j k)
```

```
                                        ((1 lda) (1 *))
                                        a-%offset%))
                          (f2cl-lib:fref a-%data%
                                       (j (f2cl-lib:int-add k 1))
                                       ((1 lda) (1 *)) a-%offset%))))
                (setf wkp1
                        (* d21
                           (-
                            (* d22
                               (f2cl-lib:fref a-%data%
                                        (j (f2cl-lib:int-add k 1))
                                        ((1 lda) (1 *))
                                        a-%offset%))
                            (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                       a-%offset%))))
                (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                              ((> i n) nil)
                  (tagbody
                    (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                          (-
                           (f2cl-lib:fref a-%data% (i j)
                                          ((1 lda) (1 *)) a-%offset%)
                           (*
                            (f2cl-lib:fref a-%data% (i k)
                                           ((1 lda) (1 *)) a-%offset%)
                            wk)
                           (*
                            (f2cl-lib:fref a-%data%
                                           (i (f2cl-lib:int-add k 1))
                                           ((1 lda) (1 *)) a-%offset%)
                            wkp1)))
                  label50))
                (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                     a-%offset%)
                        wk)
                (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-add k 1))
                                     ((1 lda) (1 *)) a-%offset%)
                        wkp1)
              label60))))))))
        (cond
         ((= kstep 1)
          (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%) kp))
         (t
          (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
                  (f2cl-lib:int-sub kp))
          (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1)) ((1 *))
                               ipiv-%offset%)
                  (f2cl-lib:int-sub kp))))
        (setf k (f2cl-lib:int-add k kstep))
```

```
            (go label40))))
       label70
        (go end_label)
       end_label
        (return (values uplo nil nil lda nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssytf2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::sisnan))))
```

———-

59.17 ssytf2_rook

59.17.1 Axiom unit tests

— **ssytf2rook.input** —

```
)set break resume
)sys rm -f ssytf2rook.output
)spool ssytf2rook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.17.2 Axiom help page

— ssytf2_rook.help —

```
=====================================================================
ssytf2_rook examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssytf2_rook (UPLO, N, A, LDA, IPIV, INFO)
           SSYTF2_ROOK computes the factorization of a real symmetric
           indefinite matrix using the bounded Bunch-Kaufman ('rook') diagonal
           pivoting method (unblocked algorithm).

Function/Subroutine Documentation
   subroutine ssytf2_rook (character UPLO, integer N, real, dimension( lda, *
       ) A, integer LDA, integer, dimension( * ) IPIV, integer INFO)
       SSYTF2_ROOK computes the factorization of a real symmetric indefinite
       matrix using the bounded Bunch-Kaufman ('rook') diagonal pivoting
       method (unblocked algorithm).

Purpose:
   SSYTF2_ROOK computes the factorization of a real symmetric matrix A
   using the bounded Bunch-Kaufman ("rook") diagonal pivoting method:

   A = U*D*U**T  or  A = L*D*L**T

   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, U**T is the transpose of U, and D is symmetric and
   block diagonal with 1-by-1 and 2-by-2 diagonal blocks.

   This is the unblocked version of the algorithm, calling Level 2 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is stored:
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
```

```
  The order of the matrix A.  N >= 0.

A
  A is REAL array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  n-by-n upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading n-by-n lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, the block diagonal matrix D and the multipliers used
  to obtain the factor U or L (see below for further details).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D.

  If UPLO = 'U':
     If IPIV(k) > 0, then rows and columns k and IPIV(k)
     were interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) < 0 and IPIV(k-1) < 0, then rows and
     columns k and -IPIV(k) were interchanged and rows and
     columns k-1 and -IPIV(k-1) were inerchaged,
     D(k-1:k,k-1:k) is a 2-by-2 diagonal block.

  If UPLO = 'L':
     If IPIV(k) > 0, then rows and columns k and IPIV(k)
     were interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) < 0 and IPIV(k+1) < 0, then rows and
     columns k and -IPIV(k) were interchanged and rows and
     columns k+1 and -IPIV(k+1) were inerchaged,
     D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -k, the k-th argument had an illegal value
  > 0: if INFO = k, D(k,k) is exactly zero.  The factorization
     has been completed, but the block diagonal matrix D is
     exactly singular, and division by zero will occur if it
     is used to solve a system of equations.
```

```
Further Details:
   If UPLO = 'U', then A = U*D*U**T, where
      U = P(n)*U(n)* ... *P(k)U(k)* ...,
   i.e., U is a product of terms P(k)*U(k), where k decreases from n to
   1 in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and U(k) is a unit upper triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                       (   I    v    0   )   k-s
             U(k) =    (   0    I    0   )   s
                       (   0    0    I   )   n-k
                          k-s   s   n-k

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(1:k-1,k).
   If s = 2, the upper triangle of D(k) overwrites A(k-1,k-1), A(k-1,k),
   and A(k,k), and v overwrites A(1:k-2,k-1:k).

   If UPLO = 'L', then A = L*D*L**T, where
      L = P(1)*L(1)* ... *P(k)*L(k)* ...,
   i.e., L is a product of terms P(k)*L(k), where k increases from 1 to
   n in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and L(k) is a unit lower triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                       (   I    0     0   )  k-1
             L(k) =    (   0    I     0   )  s
                       (   0    v     I   )  n-k-s+1
                          k-1   s  n-k-s+1

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(k+1:n,k).
   If s = 2, the lower triangle of D(k) overwrites A(k,k), A(k+1,k),
   and A(k+1,k+1), and v overwrites A(k+2:n,k:k+1).
```

———-

59.17.3 fortran code

— **ssytf2_rook.f** —

```
*  =====================================================================
      SUBROUTINE SSYTF2_ROOK( UPLO, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
```

```
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               EIGHT, SEVTEN
      PARAMETER          ( EIGHT = 8.0E+0, SEVTEN = 17.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER, DONE
      INTEGER            I, IMAX, J, JMAX, ITEMP, K, KK, KP, KSTEP,
     $                   P, II
      REAL               ABSAKK, ALPHA, COLMAX, D11, D12, D21, D22,
     $                   ROWMAX, STEMP, T, WK, WKM1, WKP1, SFMIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ISAMAX
      REAL               SLAMCH
      EXTERNAL           LSAME, ISAMAX, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           SSCAL, SSWAP, SSYR, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
```

```
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYTF2_ROOK', -INFO )
         RETURN
      END IF
*
*     Initialize ALPHA for use in choosing pivot block size.
*
      ALPHA = ( ONE+SQRT( SEVTEN ) ) / EIGHT
*
*     Compute machine safe minimum
*
      SFMIN = SLAMCH( 'S' )
*
      IF( UPPER ) THEN
*
*        Factorize A as U*D*U**T using the upper triangle of A
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        1 or 2
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop
*
         IF( K.LT.1 )
     $      GO TO 70
         KSTEP = 1
         P = K
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = ABS( A( K, K ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.GT.1 ) THEN
            IMAX = ISAMAX( K-1, A( 1, K ), 1 )
            COLMAX = ABS( A( IMAX, K ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( (MAX( ABSAKK, COLMAX ).EQ.ZERO) ) THEN
*
*           Column K is zero or underflow: set INFO and continue
```

```
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
         ELSE
*
            Test for interchange
*
            Equivalent to testing for (used to handle NaN and Inf)
            ABSAKK.GE.ALPHA*COLMAX
*
            IF( .NOT.( ABSAKK.LT.ALPHA*COLMAX ) ) THEN
*
               no interchange,
               use 1-by-1 pivot block
*
               KP = K
            ELSE
*
               DONE = .FALSE.
*
               Loop until pivot found
*
   12          CONTINUE
*
                  Begin pivot search loop body
*
                  JMAX is the column-index of the largest off-diagonal
                  element in row IMAX, and ROWMAX is its absolute value.
                  Determine both ROWMAX and JMAX.
*
                  IF( IMAX.NE.K ) THEN
                     JMAX = IMAX + ISAMAX( K-IMAX, A( IMAX, IMAX+1 ),
     $                                     LDA )
                     ROWMAX = ABS( A( IMAX, JMAX ) )
                  ELSE
                     ROWMAX = ZERO
                  END IF
*
                  IF( IMAX.GT.1 ) THEN
                     ITEMP = ISAMAX( IMAX-1, A( 1, IMAX ), 1 )
                     STEMP = ABS( A( ITEMP, IMAX ) )
                     IF( STEMP.GT.ROWMAX ) THEN
                        ROWMAX = STEMP
                        JMAX = ITEMP
                     END IF
                  END IF
*
                  Equivalent to testing for (used to handle NaN and Inf)
                  ABS( A( IMAX, IMAX ) ).GE.ALPHA*ROWMAX
```

```
*
                 IF( .NOT.( ABS( A( IMAX, IMAX ) ).LT.ALPHA*ROWMAX ) )
     $           THEN
*
*                   interchange rows and columns K and IMAX,
*                   use 1-by-1 pivot block
*
                    KP = IMAX
                    DONE = .TRUE.
*
*                Equivalent to testing for ROWMAX .EQ. COLMAX,
*                used to handle NaN and Inf
*
                 ELSE IF( ( P.EQ.JMAX ).OR.( ROWMAX.LE.COLMAX ) ) THEN
*
*                   interchange rows and columns K+1 and IMAX,
*                   use 2-by-2 pivot block
*
                    KP = IMAX
                    KSTEP = 2
                    DONE = .TRUE.
                 ELSE
*
*                   Pivot NOT found, set variables and repeat
*
                    P = IMAX
                    COLMAX = ROWMAX
                    IMAX = JMAX
                 END IF
*
*                End pivot search loop body
*
              IF( .NOT. DONE ) GOTO 12
*
           END IF
*
*          Swap TWO rows and TWO columns
*
*          First swap
*
           IF( ( KSTEP.EQ.2 ) .AND. ( P.NE.K ) ) THEN
*
*             Interchange rows and column K and P in the leading
*             submatrix A(1:k,1:k) if we have a 2-by-2 pivot
*
              IF( P.GT.1 )
     $           CALL SSWAP( P-1, A( 1, K ), 1, A( 1, P ), 1 )
              IF( P.LT.(K-1) )
     $           CALL SSWAP( K-P-1, A( P+1, K ), 1, A( P, P+1 ),
     $                    LDA )
```

```
               T = A( K, K )
               A( K, K ) = A( P, P )
               A( P, P ) = T
            END IF
*
*           Second swap
*
            KK = K - KSTEP + 1
            IF( KP.NE.KK ) THEN
*
*              Interchange rows and columns KK and KP in the leading
*              submatrix A(1:k,1:k)
*
               IF( KP.GT.1 )
     $            CALL SSWAP( KP-1, A( 1, KK ), 1, A( 1, KP ), 1 )
               IF( ( KK.GT.1 ) .AND. ( KP.LT.(KK-1) ) )
     $            CALL SSWAP( KK-KP-1, A( KP+1, KK ), 1, A( KP, KP+1 ),
     $                      LDA )
               T = A( KK, KK )
               A( KK, KK ) = A( KP, KP )
               A( KP, KP ) = T
               IF( KSTEP.EQ.2 ) THEN
                  T = A( K-1, K )
                  A( K-1, K ) = A( KP, K )
                  A( KP, K ) = T
               END IF
            END IF
*
*           Update the leading submatrix
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column k now holds
*
*              W(k) = U(k)*D(k)
*
*              where U(k) is the k-th column of U
*
               IF( K.GT.1 ) THEN
*
*                 Perform a rank-1 update of A(1:k-1,1:k-1) and
*                 store U(k) in column k
*
                  IF( ABS( A( K, K ) ).GE.SFMIN ) THEN
*
*                    Perform a rank-1 update of A(1:k-1,1:k-1) as
*                    A := A - U(k)*D(k)*U(k)**T
*                       = A - W(k)*1/D(k)*W(k)**T
*
                     D11 = ONE / A( K, K )
```

```
                  CALL SSYR( UPLO, K-1, -D11, A( 1, K ), 1, A, LDA )
*
*                 Store U(k) in column k
*
                  CALL SSCAL( K-1, D11, A( 1, K ), 1 )
               ELSE
*
*                 Store L(k) in column K
*
                  D11 = A( K, K )
                  DO 16 II = 1, K - 1
                     A( II, K ) = A( II, K ) / D11
   16             CONTINUE
*
*                 Perform a rank-1 update of A(k+1:n,k+1:n) as
*                 A := A - U(k)*D(k)*U(k)**T
*                    = A - W(k)*(1/D(k))*W(k)**T
*                    = A - (W(k)/D(k))*(D(k))*(W(k)/D(K))**T
*
                  CALL SSYR( UPLO, K-1, -D11, A( 1, K ), 1, A, LDA )
               END IF
            END IF
*
         ELSE
*
            2-by-2 pivot block D(k): columns k and k-1 now hold
*
            ( W(k-1) W(k) ) = ( U(k-1) U(k) )*D(k)
*
            where U(k) and U(k-1) are the k-th and (k-1)-th columns
            of U
*
            Perform a rank-2 update of A(1:k-2,1:k-2) as
*
            A := A - ( U(k-1) U(k) )*D(k)*( U(k-1) U(k) )**T
               = A - ( ( A(k-1)A(k) )*inv(D(k)) ) * ( A(k-1)A(k) )**T
*
            and store L(k) and L(k+1) in columns k and k+1
*
            IF( K.GT.2 ) THEN
*
               D12 = A( K-1, K )
               D22 = A( K-1, K-1 ) / D12
               D11 = A( K, K ) / D12
               T = ONE / ( D11*D22-ONE )
*
               DO 30 J = K - 2, 1, -1
*
                  WKM1 = T*( D11*A( J, K-1 )-A( J, K ) )
                  WK = T*( D22*A( J, K )-A( J, K-1 ) )
```

```
*
                    DO 20 I = J, 1, -1
                       A( I, J ) = A( I, J ) - (A( I, K ) / D12 )*WK -
     $                            ( A( I, K-1 ) / D12 )*WKM1
   20               CONTINUE
*
*                   Store U(k) and U(k-1) in cols k and k-1 for row J
*
                    A( J, K ) = WK / D12
                    A( J, K-1 ) = WKM1 / D12
*
   30            CONTINUE
*
              END IF
*
           END IF
        END IF
*
*       Store details of the interchanges in IPIV
*
        IF( KSTEP.EQ.1 ) THEN
           IPIV( K ) = KP
        ELSE
           IPIV( K ) = -P
           IPIV( K-1 ) = -KP
        END IF
*
*       Decrease K and return to the start of the main loop
*
        K = K - KSTEP
        GO TO 10
*
      ELSE
*
*       Factorize A as L*D*L**T using the lower triangle of A
*
*       K is the main loop index, increasing from 1 to N in steps of
*       1 or 2
*
        K = 1
   40   CONTINUE
*
*       If K > N, exit from loop
*
        IF( K.GT.N )
     $     GO TO 70
        KSTEP = 1
        P = K
*
*       Determine rows and columns to be interchanged and whether
```

```
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = ABS( A( K, K ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.LT.N ) THEN
            IMAX = K + ISAMAX( N-K, A( K+1, K ), 1 )
            COLMAX = ABS( A( IMAX, K ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( ( MAX( ABSAKK, COLMAX ).EQ.ZERO ) ) THEN
*
*           Column K is zero or underflow: set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
         ELSE
*
*           Test for interchange
*
*           Equivalent to testing for (used to handle NaN and Inf)
*           ABSAKK.GE.ALPHA*COLMAX
*
            IF( .NOT.( ABSAKK.LT.ALPHA*COLMAX ) ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
            ELSE
*
               DONE = .FALSE.
*
*              Loop until pivot found
*
   42          CONTINUE
*
*                 Begin pivot search loop body
*
*                 JMAX is the column-index of the largest off-diagonal
*                 element in row IMAX, and ROWMAX is its absolute value.
*                 Determine both ROWMAX and JMAX.
*
                  IF( IMAX.NE.K ) THEN
                     JMAX = K - 1 + ISAMAX( IMAX-K, A( IMAX, K ), LDA )
```

```
                  ROWMAX = ABS( A( IMAX, JMAX ) )
               ELSE
                  ROWMAX = ZERO
               END IF
*
               IF( IMAX.LT.N ) THEN
                  ITEMP = IMAX + ISAMAX( N-IMAX, A( IMAX+1, IMAX ),
     $                                   1 )
                  STEMP = ABS( A( ITEMP, IMAX ) )
                  IF( STEMP.GT.ROWMAX ) THEN
                     ROWMAX = STEMP
                     JMAX = ITEMP
                  END IF
               END IF
*
*              Equivalent to testing for (used to handle NaN and Inf)
*              ABS( A( IMAX, IMAX ) ).GE.ALPHA*ROWMAX
*
               IF( .NOT.( ABS( A( IMAX, IMAX ) ).LT.ALPHA*ROWMAX ) )
     $         THEN
*
*                 interchange rows and columns K and IMAX,
*                 use 1-by-1 pivot block
*
                  KP = IMAX
                  DONE = .TRUE.
*
*              Equivalent to testing for ROWMAX .EQ. COLMAX,
*              used to handle NaN and Inf
*
               ELSE IF( ( P.EQ.JMAX ).OR.( ROWMAX.LE.COLMAX ) ) THEN
*
*                 interchange rows and columns K+1 and IMAX,
*                 use 2-by-2 pivot block
*
                  KP = IMAX
                  KSTEP = 2
                  DONE = .TRUE.
               ELSE
*
*                 Pivot NOT found, set variables and repeat
*
                  P = IMAX
                  COLMAX = ROWMAX
                  IMAX = JMAX
               END IF
*
*              End pivot search loop body
*
            IF( .NOT. DONE ) GOTO 42
```

```
*
            END IF
*
*           Swap TWO rows and TWO columns
*
*           First swap
*
            IF( ( KSTEP.EQ.2 ) .AND. ( P.NE.K ) ) THEN
*
*              Interchange rows and column K and P in the trailing
*              submatrix A(k:n,k:n) if we have a 2-by-2 pivot
*
               IF( P.LT.N )
     $            CALL SSWAP( N-P, A( P+1, K ), 1, A( P+1, P ), 1 )
               IF( P.GT.(K+1) )
     $            CALL SSWAP( P-K-1, A( K+1, K ), 1, A( P, K+1 ), LDA )
               T = A( K, K )
               A( K, K ) = A( P, P )
               A( P, P ) = T
            END IF
*
*           Second swap
*
            KK = K + KSTEP - 1
            IF( KP.NE.KK ) THEN
*
*              Interchange rows and columns KK and KP in the trailing
*              submatrix A(k:n,k:n)
*
               IF( KP.LT.N )
     $            CALL SSWAP( N-KP, A( KP+1, KK ), 1, A( KP+1, KP ), 1 )
               IF( ( KK.LT.N ) .AND. ( KP.GT.(KK+1) ) )
     $            CALL SSWAP( KP-KK-1, A( KK+1, KK ), 1, A( KP, KK+1 ),
     $                     LDA )
               T = A( KK, KK )
               A( KK, KK ) = A( KP, KP )
               A( KP, KP ) = T
               IF( KSTEP.EQ.2 ) THEN
                  T = A( K+1, K )
                  A( K+1, K ) = A( KP, K )
                  A( KP, K ) = T
               END IF
            END IF
*
*           Update the trailing submatrix
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column k now holds
*
```

```
*              W(k) = L(k)*D(k)
*
*              where L(k) is the k-th column of L
*
               IF( K.LT.N ) THEN
*
*              Perform a rank-1 update of A(k+1:n,k+1:n) and
*              store L(k) in column k
*
                  IF( ABS( A( K, K ) ).GE.SFMIN ) THEN
*
*                    Perform a rank-1 update of A(k+1:n,k+1:n) as
*                    A := A - L(k)*D(k)*L(k)**T
*                       = A - W(k)*(1/D(k))*W(k)**T
*
                     D11 = ONE / A( K, K )
                     CALL SSYR( UPLO, N-K, -D11, A( K+1, K ), 1,
     $                          A( K+1, K+1 ), LDA )
*
*                    Store L(k) in column k
*
                     CALL SSCAL( N-K, D11, A( K+1, K ), 1 )
                  ELSE
*
*                    Store L(k) in column k
*
                     D11 = A( K, K )
                     DO 46 II = K + 1, N
                        A( II, K ) = A( II, K ) / D11
   46                CONTINUE
*
*                    Perform a rank-1 update of A(k+1:n,k+1:n) as
*                    A := A - L(k)*D(k)*L(k)**T
*                       = A - W(k)*(1/D(k))*W(k)**T
*                       = A - (W(k)/D(k))*(D(k))*(W(k)/D(K))**T
*
                     CALL SSYR( UPLO, N-K, -D11, A( K+1, K ), 1,
     $                          A( K+1, K+1 ), LDA )
                  END IF
               END IF
*
            ELSE
*
*              2-by-2 pivot block D(k): columns k and k+1 now hold
*
*              ( W(k) W(k+1) ) = ( L(k) L(k+1) )*D(k)
*
*              where L(k) and L(k+1) are the k-th and (k+1)-th columns
*              of L
*
```

```
*
*             Perform a rank-2 update of A(k+2:n,k+2:n) as
*
*             A := A - ( L(k) L(k+1) ) * D(k) * ( L(k) L(k+1) )**T
*                = A - ( ( A(k)A(k+1) )*inv(D(k) ) * ( A(k)A(k+1) )**T
*
*             and store L(k) and L(k+1) in columns k and k+1
*
              IF( K.LT.N-1 ) THEN
*
                 D21 = A( K+1, K )
                 D11 = A( K+1, K+1 ) / D21
                 D22 = A( K, K ) / D21
                 T = ONE / ( D11*D22-ONE )
*
                 DO 60 J = K + 2, N
*
*                   Compute  D21 * ( W(k)W(k+1) ) * inv(D(k)) for row J
*
                    WK = T*( D11*A( J, K )-A( J, K+1 ) )
                    WKP1 = T*( D22*A( J, K+1 )-A( J, K ) )
*
*                   Perform a rank-2 update of A(k+2:n,k+2:n)
*
                    DO 50 I = J, N
                       A( I, J ) = A( I, J ) - ( A( I, K ) / D21 )*WK -
     $                             ( A( I, K+1 ) / D21 )*WKP1
   50               CONTINUE
*
*                   Store L(k) and L(k+1) in cols k and k+1 for row J
*
                    A( J, K ) = WK / D21
                    A( J, K+1 ) = WKP1 / D21
*
   60            CONTINUE
*
              END IF
*
           END IF
        END IF
*
*       Store details of the interchanges in IPIV
*
        IF( KSTEP.EQ.1 ) THEN
           IPIV( K ) = KP
        ELSE
           IPIV( K ) = -P
           IPIV( K+1 ) = -KP
        END IF
*
```

```
*        Increase K and return to the start of the main loop
*
         K = K + KSTEP
         GO TO 40
*
      END IF
*
   70 CONTINUE
*
      RETURN
*
*     End of SSYTF2_ROOK
*
      END
```

59.17.4 lisp code

— ssytf2_rook.lisp —

```
(let* ((zero 0.0) (one 1.0) (eight 8.0) (sevten 17.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 8.0 8.0) eight)
           (type (single-float 17.0 17.0) sevten)
           (ignorable zero one eight sevten))
  (defun ssytf2_rook (uplo n a lda ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array single-float (*)) a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((absakk 0.0) (alpha 0.0) (colmax 0.0) (d11 0.0) (d12 0.0)
             (d21 0.0) (d22 0.0) (rowmax 0.0) (stemp 0.0) (t$ 0.0) (wk 0.0)
             (wkm1 0.0) (wkp1 0.0) (sfmin 0.0) (i 0) (imax 0) (j 0) (jmax 0)
             (itemp 0) (k 0) (kk 0) (kp 0) (kstep 0) (p 0) (ii 0) (upper nil)
             (done nil))
        (declare
         (type (single-float) absakk alpha colmax d11 d12 d21 d22 rowmax stemp
          t$ wk wkm1 wkp1 sfmin)
         (type (f2cl-lib:integer4) i imax j jmax itemp k kk kp kstep p ii)
         (type f2cl-lib:logical upper done))
        (setf info 0)
        (setf upper
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4)))
(cond
 ((/= info 0) (xerbla "SSYTF2_ROOK" (f2cl-lib:int-sub info))
  (go end_label)))
(setf alpha (/ (+ one (f2cl-lib:fsqrt sevten)) eight))
(setf sfmin (slamch "S"))
(cond
 (upper
  (tagbody
    (setf k n)
   label10
    (if (< k 1)
        (go label70))
    (setf kstep 1)
    (setf p k)
    (setf absakk
            (abs
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)))
    (cond
     ((> k 1)
      (setf imax
              (isamax (f2cl-lib:int-sub k 1)
               (f2cl-lib:array-slice a-%data% single-float (1 k)
                                     ((1 lda) (1 *)) a-%offset%)
               1))
      (setf colmax
              (abs
               (f2cl-lib:fref a-%data% (imax k) ((1 lda) (1 *))
                              a-%offset%))))
     (t (setf colmax zero)))
    (cond
     ((= (max absakk colmax) zero)
      (if (= info 0)
```

```
        (setf info k))
    (setf kp k))
   (t
    (cond ((not (< absakk (* alpha colmax))) (setf kp k))
          (t
           (tagbody
             (setf done f2cl-lib:%false%)
            label12
             (cond
              ((/= imax k)
               (setf jmax
                       (f2cl-lib:int-add imax
                                         (multiple-value-bind
                                             (ret-val var-0 var-1
                                              var-2)
                                             (isamax
                                              (f2cl-lib:int-sub k
                                                                imax)
                                              (f2cl-lib:array-slice
                                               a-%data% single-float
                                               (imax
                                                (f2cl-lib:int-add
                                                 imax 1))
                                               ((1 lda) (1 *))
                                               a-%offset%)
                                              lda)
                                           (declare
                                            (ignore var-0 var-1))
                                           (when var-2
                                             (setf lda var-2))
                                           ret-val)))
               (setf rowmax
                       (abs
                        (f2cl-lib:fref a-%data% (imax jmax)
                                       ((1 lda) (1 *)) a-%offset%))))
              (t (setf rowmax zero)))
             (cond
              ((> imax 1)
               (setf itemp
                       (isamax (f2cl-lib:int-sub imax 1)
                        (f2cl-lib:array-slice a-%data% single-float
                                              (1 imax)
                                              ((1 lda) (1 *))
                                              a-%offset%)
                        1))
               (setf stemp
                       (abs
                        (f2cl-lib:fref a-%data% (itemp imax)
                                       ((1 lda) (1 *)) a-%offset%)))
               (cond
```

```
                ((> stemp rowmax) (setf rowmax stemp)
                 (setf jmax itemp)))))
            (cond
             ((not
               (<
                (abs (f2cl-lib:fref a (imax imax) ((1 lda) (1 *))))
                (* alpha rowmax)))
              (setf kp imax) (setf done f2cl-lib:%true%))
             ((or (= p jmax) (<= rowmax colmax)) (setf kp imax)
              (setf kstep 2) (setf done f2cl-lib:%true%))
             (t (setf p imax) (setf colmax rowmax)
              (setf imax jmax)))
            (if (not done)
                (go label12)))))
(cond
 ((and (= kstep 2) (/= p k))
  (if (> p 1)
      (sswap (f2cl-lib:int-sub p 1)
       (f2cl-lib:array-slice a-%data% single-float (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% single-float (1 p)
                             ((1 lda) (1 *)) a-%offset%)
       1))
  (if (< p (f2cl-lib:int-sub k 1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (sswap (f2cl-lib:int-sub k p 1)
           (f2cl-lib:array-slice a-%data% single-float
                                 ((+ p 1) k) ((1 lda) (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% single-float
                                 (p (f2cl-lib:int-add p 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf lda var-4))))
  (setf t$
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
          (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *)) a-%offset%)
          t$)))
(setf kk (f2cl-lib:int-add (f2cl-lib:int-sub k kstep) 1))
(cond
 ((/= kp kk)
  (if (> kp 1)
      (sswap (f2cl-lib:int-sub kp 1)
```

```
        (f2cl-lib:array-slice a-%data% single-float (1 kk)
                              ((1 lda) (1 *)) a-%offset%)
        1
        (f2cl-lib:array-slice a-%data% single-float (1 kp)
                              ((1 lda) (1 *)) a-%offset%)
        1))
  (if (and (> kk 1) (< kp (f2cl-lib:int-sub kk 1)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (sswap (f2cl-lib:int-sub kk kp 1)
           (f2cl-lib:array-slice a-%data% single-float
                                 ((+ kp 1) kk) ((1 lda) (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% single-float
                                 (kp (f2cl-lib:int-add kp 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
         (declare (ignore var-0 var-1 var-2 var-3))
         (when var-4 (setf lda var-4))))
  (setf t$
          (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                       a-%offset%)
          (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                       a-%offset%)
          t$)
  (cond
   ((= kstep 2)
    (setf t$
            (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                           ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                         ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                         a-%offset%)
            t$)))))
(cond
 ((= kstep 1)
  (cond
   ((> k 1)
    (cond
     ((>= (abs (f2cl-lib:fref a (k k) ((1 lda) (1 *)))) sfmin)
      (setf d11
              (/ one
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
```

```
                                a-%offset%)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ssyr uplo (f2cl-lib:int-sub k 1) (- d11)
             (f2cl-lib:array-slice a-%data% single-float (1 k)
                                   ((1 lda) (1 *)) a-%offset%)
             1 a lda)
          (declare (ignore var-1 var-2 var-3 var-4 var-5))
          (when var-0 (setf uplo var-0))
          (when var-6 (setf lda var-6)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal (f2cl-lib:int-sub k 1) d11
             (f2cl-lib:array-slice a-%data% single-float (1 k)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-0 var-2 var-3))
          (when var-1 (setf d11 var-1))))
       (t
        (setf d11
                (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                               a-%offset%))
        (f2cl-lib:fdo (ii 1 (f2cl-lib:int-add ii 1))
                      ((> ii
                          (f2cl-lib:int-add k
                                            (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
                                 a-%offset%)
                  (/
                   (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
                                  a-%offset%)
                   d11))
           label16))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ssyr uplo (f2cl-lib:int-sub k 1) (- d11)
             (f2cl-lib:array-slice a-%data% single-float (1 k)
                                   ((1 lda) (1 *)) a-%offset%)
             1 a lda)
          (declare (ignore var-1 var-2 var-3 var-4 var-5))
          (when var-0 (setf uplo var-0))
          (when var-6 (setf lda var-6))))))))
(t
 (cond
  ((> k 2)
   (setf d12
           (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                          ((1 lda) (1 *)) a-%offset%))
   (setf d22
```

```
        (/
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-sub k 1)
                         (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         d12))
(setf d11
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         d12))
(setf t$ (/ one (- (* d11 d22) one)))
(f2cl-lib:fdo (j (f2cl-lib:int-add k (f2cl-lib:int-sub 2))
               (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j 1) nil)
  (tagbody
    (setf wkm1
            (* t$
               (-
                (* d11
                   (f2cl-lib:fref a-%data%
                                  (j (f2cl-lib:int-sub k 1))
                                  ((1 lda) (1 *))
                                  a-%offset%))
                (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                               a-%offset%))))
    (setf wk
            (* t$
               (-
                (* d22
                   (f2cl-lib:fref a-%data% (j k)
                                  ((1 lda) (1 *))
                                  a-%offset%))
                (f2cl-lib:fref a-%data%
                               (j (f2cl-lib:int-sub k 1))
                               ((1 lda) (1 *)) a-%offset%))))
    (f2cl-lib:fdo (i j
                   (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                             a-%offset%)
                (-
                 (f2cl-lib:fref a-%data% (i j)
                                ((1 lda) (1 *)) a-%offset%)
                 (*
                  (/
                   (f2cl-lib:fref a-%data% (i k)
                                  ((1 lda) (1 *)) a-%offset%)
                   d12)
```

```
                              wk)
                             (*
                              (/
                               (f2cl-lib:fref a-%data%
                                              (i (f2cl-lib:int-sub k 1))
                                              ((1 lda) (1 *)) a-%offset%)
                               d12)
                              wkm1)))
                   label20))
                (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                     a-%offset%)
                        (/ wk d12))
                (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-sub k 1))
                                     ((1 lda) (1 *)) a-%offset%)
                        (/ wkm1 d12))
              label30))))))))
    (cond
     ((= kstep 1)
      (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%) kp))
     (t
      (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
              (f2cl-lib:int-sub p))
      (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                           ipiv-%offset%)
              (f2cl-lib:int-sub kp))))
    (setf k (f2cl-lib:int-sub k kstep))
    (go label10)))
  (t
   (tagbody
    (setf k 1)
   label40
    (if (> k n)
        (go label70))
    (setf kstep 1)
    (setf p k)
    (setf absakk
            (abs
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)))
    (cond
     ((< k n)
      (setf imax
              (f2cl-lib:int-add k
                                (isamax (f2cl-lib:int-sub n k)
                                 (f2cl-lib:array-slice a-%data%
                                                       single-float
                                                       ((+ k 1) k)
                                                       ((1 lda) (1 *))
                                                       a-%offset%)
                                 1)))
```

```
  (setf colmax
          (abs
           (f2cl-lib:fref a-%data% (imax k) ((1 lda) (1 *))
                          a-%offset%))))
 (t (setf colmax zero)))
(cond
 ((= (max absakk colmax) zero)
  (if (= info 0)
      (setf info k))
  (setf kp k))
 (t
  (cond ((not (< absakk (* alpha colmax))) (setf kp k))
        (t
         (tagbody
           (setf done f2cl-lib:%false%)
          label42
           (cond
            ((/= imax k)
             (setf jmax
                     (f2cl-lib:int-add (f2cl-lib:int-sub k 1)
                                       (multiple-value-bind
                                           (ret-val var-0 var-1
                                            var-2)
                                           (isamax
                                            (f2cl-lib:int-sub imax
                                                              k)
                                            (f2cl-lib:array-slice
                                             a-%data% single-float
                                             (imax k)
                                             ((1 lda) (1 *))
                                             a-%offset%)
                                            lda)
                                         (declare
                                          (ignore var-0 var-1))
                                         (when var-2
                                           (setf lda var-2))
                                         ret-val)))
             (setf rowmax
                     (abs
                      (f2cl-lib:fref a-%data% (imax jmax)
                                     ((1 lda) (1 *)) a-%offset%))))
            (t (setf rowmax zero)))
           (cond
            ((< imax n)
             (setf itemp
                     (f2cl-lib:int-add imax
                                       (isamax
                                        (f2cl-lib:int-sub n imax)
                                        (f2cl-lib:array-slice
                                         a-%data% single-float
```

```
                                        ((+ imax 1) imax)
                                        ((1 lda) (1 *))
                                        a-%offset%)
                                       1)))
                (setf stemp
                        (abs
                         (f2cl-lib:fref a-%data% (itemp imax)
                                        ((1 lda) (1 *)) a-%offset%)))
                (cond
                 ((> stemp rowmax) (setf rowmax stemp)
                  (setf jmax itemp)))))
              (cond
               ((not
                 (<
                  (abs (f2cl-lib:fref a (imax imax) ((1 lda) (1 *))))
                  (* alpha rowmax)))
                (setf kp imax) (setf done f2cl-lib:%true%))
               ((or (= p jmax) (<= rowmax colmax)) (setf kp imax)
                (setf kstep 2) (setf done f2cl-lib:%true%))
               (t (setf p imax) (setf colmax rowmax)
                (setf imax jmax)))
              (if (not done)
                  (go label42)))))
(cond
 ((and (= kstep 2) (/= p k))
  (if (< p n)
      (sswap (f2cl-lib:int-sub n p)
       (f2cl-lib:array-slice a-%data% single-float ((+ p 1) k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% single-float ((+ p 1) p)
                             ((1 lda) (1 *)) a-%offset%)
       1))
  (if (> p (f2cl-lib:int-add k 1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (sswap (f2cl-lib:int-sub p k 1)
           (f2cl-lib:array-slice a-%data% single-float
                                 ((+ k 1) k) ((1 lda) (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% single-float
                                 (p (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf lda var-4))))
  (setf t$
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
```

```
          (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *)) a-%offset%)
          t$)))
(setf kk (f2cl-lib:int-sub (f2cl-lib:int-add k kstep) 1))
(cond
 ((/= kp kk)
  (if (< kp n)
      (sswap (f2cl-lib:int-sub n kp)
       (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) kk)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) kp)
                             ((1 lda) (1 *)) a-%offset%)
       1))
  (if (and (< kk n) (> kp (f2cl-lib:int-add kk 1)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (sswap (f2cl-lib:int-sub kp kk 1)
           (f2cl-lib:array-slice a-%data% single-float
                                 ((+ kk 1) kk) ((1 lda) (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% single-float
                                 (kp (f2cl-lib:int-add kk 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf lda var-4))))
  (setf t$
          (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                       a-%offset%)
          (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                       a-%offset%)
          t$)
  (cond
   ((= kstep 2)
    (setf t$
            (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                           ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                         ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                         a-%offset%)
            t$)))))
```

```
(cond
 ((= kstep 1)
  (cond
   ((< k n)
    (cond
     ((>= (abs (f2cl-lib:fref a (k k) ((1 lda) (1 *)))) sfmin)
      (setf d11
              (/ one
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (ssyr uplo (f2cl-lib:int-sub n k) (- d11)
           (f2cl-lib:array-slice a-%data% single-float
                                 ((+ k 1) k) ((1 lda) (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% single-float
                                 ((+ k 1) (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-1 var-2 var-3 var-4 var-5))
        (when var-0 (setf uplo var-0))
        (when var-6 (setf lda var-6)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal (f2cl-lib:int-sub n k) d11
           (f2cl-lib:array-slice a-%data% single-float
                                 ((+ k 1) k) ((1 lda) (1 *))
                                 a-%offset%)
           1)
        (declare (ignore var-0 var-2 var-3))
        (when var-1 (setf d11 var-1))))
     (t
      (setf d11
              (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                             a-%offset%))
      (f2cl-lib:fdo (ii (f2cl-lib:int-add k 1)
                     (f2cl-lib:int-add ii 1))
                    ((> ii n) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
                               a-%offset%)
                (/
                 (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
                                a-%offset%)
                 d11))
         label46))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (ssyr uplo (f2cl-lib:int-sub n k) (- d11)
```

```
            (f2cl-lib:array-slice a-%data% single-float
                                  ((+ k 1) k) ((1 lda) (1 *))
                                  a-%offset%)
            1
            (f2cl-lib:array-slice a-%data% single-float
                                  ((+ k 1) (f2cl-lib:int-add k 1))
                                  ((1 lda) (1 *)) a-%offset%)
            lda)
        (declare (ignore var-1 var-2 var-3 var-4 var-5))
        (when var-0 (setf uplo var-0))
        (when var-6 (setf lda var-6))))))))
(t
 (cond
  ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
   (setf d21
           (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                          ((1 lda) (1 *)) a-%offset%))
   (setf d11
           (/
            (f2cl-lib:fref a-%data%
                           ((f2cl-lib:int-add k 1)
                            (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
            d21))
   (setf d22
           (/
            (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                           a-%offset%)
            d21))
   (setf t$ (/ one (- (* d11 d22) one)))
   (f2cl-lib:fdo (j (f2cl-lib:int-add k 2)
                  (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf wk
               (* t$
                  (-
                   (* d11
                      (f2cl-lib:fref a-%data% (j k)
                                     ((1 lda) (1 *))
                                     a-%offset%))
                   (f2cl-lib:fref a-%data%
                                  (j (f2cl-lib:int-add k 1))
                                  ((1 lda) (1 *)) a-%offset%))))
       (setf wkp1
               (* t$
                  (-
                   (* d22
                      (f2cl-lib:fref a-%data%
                                     (j (f2cl-lib:int-add k 1))
```

```
                                                  ((1 lda) (1 *))
                                                  a-%offset%))
                                     (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                                    a-%offset%))))
                         (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                                       ((> i n) nil)
                           (tagbody
                             (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                                  a-%offset%)
                                     (-
                                      (f2cl-lib:fref a-%data% (i j)
                                                     ((1 lda) (1 *)) a-%offset%)
                                      (*
                                       (/
                                        (f2cl-lib:fref a-%data% (i k)
                                                       ((1 lda) (1 *)) a-%offset%)
                                        d21)
                                       wk)
                                      (*
                                       (/
                                        (f2cl-lib:fref a-%data%
                                                       (i (f2cl-lib:int-add k 1))
                                                       ((1 lda) (1 *)) a-%offset%)
                                        d21)
                                       wkp1)))
                              label50))
                         (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                              a-%offset%)
                                 (/ wk d21))
                         (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-add k 1))
                                              ((1 lda) (1 *)) a-%offset%)
                                 (/ wkp1 d21))
                       label60))))))))
                 (cond
                  ((= kstep 1)
                   (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%) kp))
                  (t
                   (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
                         (f2cl-lib:int-sub p))
                   (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1)) ((1 *))
                                        ipiv-%offset%)
                         (f2cl-lib:int-sub kp))))
                 (setf k (f2cl-lib:int-add k kstep))
                 (go label40))))
        label70
         (go end_label)
        end_label
         (return (values uplo nil nil lda nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
```

```
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssytf2_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

59.18 ssytrd

59.18.1 Axiom unit tests

— **ssytrd.input** —

```
)set break resume
)sys rm -f ssytrd.output
)spool ssytrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.18.2 Axiom help page

— **ssytrd.help** —

```
==================================================================
```

```
ssytrd examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssytrd (UPLO, N, A, LDA, D, E, TAU, WORK, LWORK, INFO)
           SSYTRD

Function/Subroutine Documentation
   subroutine ssytrd (character UPLO, integer N, real, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) D, real, dimension( * ) E, real,
       dimension( * ) TAU, real, dimension( * ) WORK, integer LWORK, integer
       INFO)
       SSYTRD

Purpose:
   SSYTRD reduces a real symmetric matrix A to real symmetric
   tridiagonal form T by an orthogonal similarity transformation:
   Q**T * A * Q = T.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   N-by-N upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading N-by-N lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.
   On exit, if UPLO = 'U', the diagonal and first superdiagonal
   of A are overwritten by the corresponding elements of the
   tridiagonal matrix T, and the elements above the first
   superdiagonal, with the array TAU, represent the orthogonal
   matrix Q as a product of elementary reflectors; if UPLO
   = 'L', the diagonal and first subdiagonal of A are over-
```

```
  written by the corresponding elements of the tridiagonal
  matrix T, and the elements below the first subdiagonal, with
  the array TAU, represent the orthogonal matrix Q as a product
  of elementary reflectors. See Further Details.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

D
  D is REAL array, dimension (N)
  The diagonal elements of the tridiagonal matrix T: D(i) = A(i,i).

E
  E is REAL array, dimension (N-1)
  The off-diagonal elements of the tridiagonal matrix T:
  E(i) = A(i,i+1) if UPLO = 'U', E(i) = A(i+1,i) if UPLO = 'L'.

TAU
  TAU is REAL array, dimension (N-1)
  The scalar factors of the elementary reflectors (see Further Details).

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.  LWORK >= 1.
  For optimum performance LWORK >= N*NB, where NB is the
  optimal blocksize.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details:

  If UPLO = 'U', the matrix Q is represented as a product of elementary
  reflectors

              Q = H(n-1) . . . H(2) H(1).

  Each H(i) has the form
```

```
             H(i) = I - tau * v * v**T

where tau is a real scalar, and v is a real vector with
v(i+1:n) = 0 and v(i) = 1; v(1:i-1) is stored on exit in
A(1:i-1,i+1), and tau in TAU(i).

If UPLO = 'L', the matrix Q is represented as a product of elementary
reflectors

             Q = H(1) H(2) . . . H(n-1).

Each H(i) has the form

             H(i) = I - tau * v * v**T

where tau is a real scalar, and v is a real vector with
v(1:i) = 0 and v(i+1) = 1; v(i+2:n) is stored on exit in A(i+2:n,i),
and tau in TAU(i).

The contents of A on exit are illustrated by the following examples
with n = 5:

     if UPLO = 'U':                       if UPLO = 'L':

       (  d   e   v2  v3  v4 )              (  d                  )
       (      d   e   v3  v4 )              (  e   d              )
       (          d   e   v4 )              (  v1  e   d          )
       (              d   e  )              (  v1  v2  e   d      )
       (                  d  )              (  v1  v2  v3  e   d  )

where d and e denote diagonal and off-diagonal elements of T, and vi
denotes an element of the vector defining H(i).
```

———-

59.18.3 fortran code

— **ssytrd.f** —

```
*  ===================================================================
      SUBROUTINE SSYTRD( UPLO, N, A, LDA, D, E, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
```

```
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), D( * ), E( * ), TAU( * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, UPPER
      INTEGER            I, IINFO, IWS, J, KK, LDWORK, LWKOPT, NB,
     $                   NBMIN, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLATRD, SSYR2K, SSYTD2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -9
      END IF
*
      IF( INFO.EQ.0 ) THEN
```

```
*
*        Determine the block size.
*
         NB = ILAENV( 1, 'SSYTRD', UPLO, N, -1, -1, -1 )
         LWKOPT = N*NB
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYTRD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NX = N
      IWS = 1
      IF( NB.GT.1 .AND. NB.LT.N ) THEN
*
*        Determine when to cross over from blocked to unblocked code
*        (last block is always handled by unblocked code).
*
         NX = MAX( NB, ILAENV( 3, 'SSYTRD', UPLO, N, -1, -1, -1 ) )
         IF( NX.LT.N ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = N
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  determine the
*              minimum value of NB, and reduce NB or force use of
*              unblocked code by setting NX = N.
*
               NB = MAX( LWORK / LDWORK, 1 )
               NBMIN = ILAENV( 2, 'SSYTRD', UPLO, N, -1, -1, -1 )
               IF( NB.LT.NBMIN )
     $            NX = N
            END IF
         ELSE
            NX = N
         END IF
```

```
      ELSE
         NB = 1
      END IF
*
      IF( UPPER ) THEN
*
*        Reduce the upper triangle of A.
*        Columns 1:kk are handled by the unblocked method.
*
         KK = N - ( ( N-NX+NB-1 ) / NB )*NB
         DO 20 I = N - NB + 1, KK + 1, -NB
*
*           Reduce columns i:i+nb-1 to tridiagonal form and form the
*           matrix W which is needed to update the unreduced part of
*           the matrix
*
            CALL SLATRD( UPLO, I+NB-1, NB, A, LDA, E, TAU, WORK,
     $                   LDWORK )
*
*           Update the unreduced submatrix A(1:i-1,1:i-1), using an
*           update of the form:  A := A - V*W**T - W*V**T
*
            CALL SSYR2K( UPLO, 'No transpose', I-1, NB, -ONE, A( 1, I ),
     $                   LDA, WORK, LDWORK, ONE, A, LDA )
*
*           Copy superdiagonal elements back into A, and diagonal
*           elements into D
*
            DO 10 J = I, I + NB - 1
               A( J-1, J ) = E( J-1 )
               D( J ) = A( J, J )
   10       CONTINUE
   20    CONTINUE
*
*        Use unblocked code to reduce the last or only block
*
         CALL SSYTD2( UPLO, KK, A, LDA, D, E, TAU, IINFO )
      ELSE
*
*        Reduce the lower triangle of A
*
         DO 40 I = 1, N - NX, NB
*
*           Reduce columns i:i+nb-1 to tridiagonal form and form the
*           matrix W which is needed to update the unreduced part of
*           the matrix
*
            CALL SLATRD( UPLO, N-I+1, NB, A( I, I ), LDA, E( I ),
     $                   TAU( I ), WORK, LDWORK )
*
```

```
*          Update the unreduced submatrix A(i+ib:n,i+ib:n), using
*          an update of the form:  A := A - V*W**T - W*V**T
*
           CALL SSYR2K( UPLO, 'No transpose', N-I-NB+1, NB, -ONE,
     $                  A( I+NB, I ), LDA, WORK( NB+1 ), LDWORK, ONE,
     $                  A( I+NB, I+NB ), LDA )
*
*          Copy subdiagonal elements back into A, and diagonal
*          elements into D
*
           DO 30 J = I, I + NB - 1
              A( J+1, J ) = E( J )
              D( J ) = A( J, J )
   30      CONTINUE
   40   CONTINUE
*
*       Use unblocked code to reduce the last or only block
*
        CALL SSYTD2( UPLO, N-I+1, A( I, I ), LDA, D( I ), E( I ),
     $               TAU( I ), IINFO )
      END IF
*
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of SSYTRD
*
      END
```

———-

59.18.4 lisp code

— **ssytrd.lisp** —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun ssytrd (uplo n a lda d e tau work lwork info)
    (declare (type (array single-float (*)) work tau e d a)
             (type (f2cl-lib:integer4) info lwork lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%)
         (tau single-float tau-%data% tau-%offset%)
```

```
   (work single-float work-%data% work-%offset%))
(prog ((i 0) (iinfo 0) (iws 0) (j 0) (kk 0) (ldwork 0) (lwkopt 0) (nb 0)
       (nbmin 0) (nx 0) (lquery nil) (upper nil))
  (declare
   (type (f2cl-lib:integer4) i iinfo iws j kk ldwork lwkopt nb nbmin nx)
   (type f2cl-lib:logical lquery upper))
  (setf info 0)
  (setf upper
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
  (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
  (cond
   ((and (not upper)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4))
   ((and (< lwork 1) (not lquery)) (setf info -9)))
  (cond
   ((= info 0)
    (setf nb
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 1 "SSYTRD" uplo n -1 -1 -1)
              (declare (ignore var-0 var-1 var-4 var-5 var-6))
              (setf uplo var-2)
              (setf n var-3)
              ret-val))
    (setf lwkopt (f2cl-lib:int-mul n nb))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))))
  (cond
   ((/= info 0) (xerbla "SSYTRD" (f2cl-lib:int-sub info)) (go end_label))
   (lquery (go end_label)))
  (cond
   ((= n 0)
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce (the f2cl-lib:integer4 1) 'single-float))
    (go end_label)))
  (setf nx n)
  (setf iws 1)
```

```
(cond
 ((and (> nb 1) (< nb n))
  (setf nx
          (max (the f2cl-lib:integer4 nb)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "SSYTRD" uplo n -1 -1 -1)
                      (declare (ignore var-0 var-1 var-4 var-5 var-6))
                      (setf uplo var-2)
                      (setf n var-3)
                      ret-val))))
  (cond
   ((< nx n) (setf ldwork n) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (max (the f2cl-lib:integer4 (truncate lwork ldwork)) 1))
      (setf nbmin
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 2 "SSYTRD" uplo n -1 -1 -1)
                (declare (ignore var-0 var-1 var-4 var-5 var-6))
                (setf uplo var-2)
                (setf n var-3)
                ret-val))
      (if (< nb nbmin)
          (setf nx n)))))
   (t (setf nx n))))
 (t (setf nb 1)))
(cond
 (upper
  (setf kk
          (- n
             (*
              (the f2cl-lib:integer4
                   (truncate (- (+ (- n nx) nb) 1) nb))
              nb)))
  (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub nb) 1)
                 (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
                ((> i (f2cl-lib:int-add kk 1)) nil)
    (tagbody
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (slatrd uplo (f2cl-lib:int-sub (f2cl-lib:int-add i nb) 1) nb
           a lda e tau work ldwork)
        (declare (ignore var-1 var-2 var-3 var-5 var-6 var-7))
        (setf uplo var-0)
        (setf lda var-4)
        (setf ldwork var-8))
```

```
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11)
          (ssyr2k uplo "No transpose" (f2cl-lib:int-sub i 1) nb (- one)
           (f2cl-lib:array-slice a-%data% single-float (1 i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda work ldwork one a lda)
        (declare (ignore var-1 var-2 var-4 var-5 var-7 var-10))
        (when var-0 (setf uplo var-0))
        (when var-3 (setf nb var-3))
        (when var-6 (setf lda var-6))
        (when var-8 (setf ldwork var-8))
        (when var-9 (setf one var-9))
        (when var-11 (setf lda var-11)))
      (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                    ((> j (f2cl-lib:int-add i nb (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub j 1) j)
                               ((1 lda) (1 *)) a-%offset%)
                  (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub j 1))
                                 ((1 *)) e-%offset%))
          (setf (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                  (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                 a-%offset%))
         label10))
    label20))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (ssytd2 uplo kk a lda d e tau iinfo)
   (declare (ignore var-1 var-2 var-4 var-5 var-6))
   (setf uplo var-0)
   (setf lda var-3)
   (setf iinfo var-7)))
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
               ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub nx))) nil)
   (tagbody
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (slatrd uplo (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) nb
          (f2cl-lib:array-slice a-%data% single-float (i i)
                                ((1 lda) (1 *)) a-%offset%)
          lda
          (f2cl-lib:array-slice e-%data% single-float (i) ((1 *))
                                e-%offset%)
          (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                                tau-%offset%)
          work ldwork)
       (declare (ignore var-1 var-2 var-3 var-5 var-6 var-7))
```

```
        (setf uplo var-0)
        (setf lda var-4)
        (setf ldwork var-8))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11)
          (ssyr2k uplo "No transpose"
           (f2cl-lib:int-add (f2cl-lib:int-sub n i nb) 1) nb (- one)
           (f2cl-lib:array-slice a-%data% single-float ((+ i nb) i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice work-%data% single-float ((+ nb 1))
                                 ((1 *)) work-%offset%)
           ldwork one
           (f2cl-lib:array-slice a-%data% single-float
                                 ((+ i nb) (f2cl-lib:int-add i nb))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-1 var-2 var-4 var-5 var-7 var-10))
        (when var-0 (setf uplo var-0))
        (when var-3 (setf nb var-3))
        (when var-6 (setf lda var-6))
        (when var-8 (setf ldwork var-8))
        (when var-9 (setf one var-9))
        (when var-11 (setf lda var-11)))
      (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                    ((> j (f2cl-lib:int-add i nb (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add j 1) j)
                               ((1 lda) (1 *)) a-%offset%)
                  (f2cl-lib:fref e-%data% (j) ((1 *)) e-%offset%))
          (setf (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                  (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                 a-%offset%))
         label30))
    label40))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (ssytd2 uplo (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
       (f2cl-lib:array-slice a-%data% single-float (i i)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice d-%data% single-float (i) ((1 *))
                             d-%offset%)
       (f2cl-lib:array-slice e-%data% single-float (i) ((1 *))
                             e-%offset%)
       (f2cl-lib:array-slice tau-%data% single-float (i) ((1 *))
                             tau-%offset%)
       iinfo)
```

```
            (declare (ignore var-1 var-2 var-4 var-5 var-6))
            (setf uplo var-0)
            (setf lda var-3)
            (setf iinfo var-7))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
        (go end_label)
       end_label
        (return (values uplo n nil lda nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssytrd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              nil nil fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::ssytd2
                                              fortran-to-lisp::slatrd
                                              fortran-to-lisp::ilaenv))))
```

———-

59.19 ssytrf

59.19.1 Axiom unit tests

— ssytrf.input —

```
)set break resume
```

```
)sys rm -f ssytrf.output
)spool ssytrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

59.19.2 Axiom help page

— ssytrf.help —

```
==================================================================
ssytrf examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssytrf (UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO)
           SSYTRF

Function/Subroutine Documentation
   subroutine ssytrf (character UPLO, integer N, real, dimension( lda, * ) A,
       integer LDA, integer, dimension( * ) IPIV, real, dimension( * ) WORK,
       integer LWORK, integer INFO)
       SSYTRF

Purpose:
   SSYTRF computes the factorization of a real symmetric matrix A using
   the Bunch-Kaufman diagonal pivoting method.  The form of the
   factorization is

   A = U*D*U**T  or  A = L*D*L**T

   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, and D is symmetric and block diagonal with
   1-by-1 and 2-by-2 diagonal blocks.

   This is the blocked version of the algorithm, calling Level 3 BLAS.

Parameters:
```

```
UPLO
  UPLO is CHARACTER*1
  = 'U':  Upper triangle of A is stored;
  = 'L':  Lower triangle of A is stored.

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

A
  A is REAL array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, the block diagonal matrix D and the multipliers used
  to obtain the factor U or L (see below for further details).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D.
  If IPIV(k) > 0, then rows and columns k and IPIV(k) were
  interchanged and D(k,k) is a 1-by-1 diagonal block.
  If UPLO = 'U' and IPIV(k) = IPIV(k-1) < 0, then rows and
  columns k-1 and -IPIV(k) were interchanged and D(k-1:k,k-1:k)
  is a 2-by-2 diagonal block.  If UPLO = 'L' and IPIV(k) =
  IPIV(k+1) < 0, then rows and columns k+1 and -IPIV(k) were
  interchanged and D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of WORK.  LWORK >=1.  For best performance
  LWORK >= N*NB, where NB is the block size returned by ILAENV.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
```

```
  message related to LWORK is issued by XERBLA.

 INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, D(i,i) is exactly zero.  The factorization
     has been completed, but the block diagonal matrix D is
     exactly singular, and division by zero will occur if it
     is used to solve a system of equations.

Further Details:

  If UPLO = 'U', then A = U*D*U**T, where
     U = P(n)*U(n)* ... *P(k)U(k)* ...,
  i.e., U is a product of terms P(k)*U(k), where k decreases from n to
  1 in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
  and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
  defined by IPIV(k), and U(k) is a unit upper triangular matrix, such
  that if the diagonal block D(k) is of order s (s = 1 or 2), then

                      (   I    v    0   )   k-s
             U(k) =   (   0    I    0   )   s
                      (   0    0    I   )   n-k
                         k-s   s   n-k

  If s = 1, D(k) overwrites A(k,k), and v overwrites A(1:k-1,k).
  If s = 2, the upper triangle of D(k) overwrites A(k-1,k-1), A(k-1,k),
  and A(k,k), and v overwrites A(1:k-2,k-1:k).

  If UPLO = 'L', then A = L*D*L**T, where
     L = P(1)*L(1)* ... *P(k)*L(k)* ...,
  i.e., L is a product of terms P(k)*L(k), where k increases from 1 to
  n in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
  and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
  defined by IPIV(k), and L(k) is a unit lower triangular matrix, such
  that if the diagonal block D(k) is of order s (s = 1 or 2), then

                      (   I    0     0   )  k-1
             L(k) =   (   0    I     0   )  s
                      (   0    v     I   )  n-k-s+1
                         k-1   s  n-k-s+1

  If s = 1, D(k) overwrites A(k,k), and v overwrites A(k+1:n,k).
  If s = 2, the lower triangle of D(k) overwrites A(k,k), A(k+1,k),
  and A(k+1,k+1), and v overwrites A(k+2:n,k:k+1).
```

59.19.3 fortran code

— ssytrf.f —

```
*  =====================================================================
      SUBROUTINE SSYTRF( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY, UPPER
      INTEGER            IINFO, IWS, J, K, KB, LDWORK, LWKOPT, NB, NBMIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLASYF, SSYTF2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
```

```
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
*
      IF( INFO.EQ.0 ) THEN
*
*        Determine the block size
*
         NB = ILAENV( 1, 'SSYTRF', UPLO, N, -1, -1, -1 )
         LWKOPT = N*NB
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYTRF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
      NBMIN = 2
      LDWORK = N
      IF( NB.GT.1 .AND. NB.LT.N ) THEN
         IWS = LDWORK*NB
         IF( LWORK.LT.IWS ) THEN
            NB = MAX( LWORK / LDWORK, 1 )
            NBMIN = MAX( 2, ILAENV( 2, 'SSYTRF', UPLO, N, -1, -1, -1 ) )
         END IF
      ELSE
         IWS = 1
      END IF
      IF( NB.LT.NBMIN )
     $   NB = N
*
      IF( UPPER ) THEN
*
*        Factorize A as U*D*U**T using the upper triangle of A
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        KB, where KB is the number of columns factorized by SLASYF;
*        KB is either NB or NB-1, or K for the last block
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop
*
         IF( K.LT.1 )
```

```
     $      GO TO 40
*
         IF( K.GT.NB ) THEN
*
*           Factorize columns k-kb+1:k of A and use blocked code to
*           update columns 1:k-kb
*
            CALL SLASYF( UPLO, K, NB, KB, A, LDA, IPIV, WORK, LDWORK,
     $                   IINFO )
         ELSE
*
*           Use unblocked code to factorize columns 1:k of A
*
            CALL SSYTF2( UPLO, K, A, LDA, IPIV, IINFO )
            KB = K
         END IF
*
*        Set INFO on the first occurrence of a zero pivot
*
         IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO
*
*        Decrease K and return to the start of the main loop
*
         K = K - KB
         GO TO 10
*
      ELSE
*
*        Factorize A as L*D*L**T using the lower triangle of A
*
*        K is the main loop index, increasing from 1 to N in steps of
*        KB, where KB is the number of columns factorized by SLASYF;
*        KB is either NB or NB-1, or N-K+1 for the last block
*
         K = 1
   20    CONTINUE
*
*        If K > N, exit from loop
*
         IF( K.GT.N )
     $      GO TO 40
*
         IF( K.LE.N-NB ) THEN
*
*           Factorize columns k:k+kb-1 of A and use blocked code to
*           update columns k+kb:n
*
            CALL SLASYF( UPLO, N-K+1, NB, KB, A( K, K ), LDA, IPIV( K ),
     $                   WORK, LDWORK, IINFO )
```

```
         ELSE
*
*           Use unblocked code to factorize columns k:n of A
*
            CALL SSYTF2( UPLO, N-K+1, A( K, K ), LDA, IPIV( K ), IINFO )
            KB = N - K + 1
         END IF
*
*        Set INFO on the first occurrence of a zero pivot
*
         IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO + K - 1
*
*        Adjust IPIV
*
         DO 30 J = K, K + KB - 1
            IF( IPIV( J ).GT.0 ) THEN
               IPIV( J ) = IPIV( J ) + K - 1
            ELSE
               IPIV( J ) = IPIV( J ) - K + 1
            END IF
   30    CONTINUE
*
*        Increase K and return to the start of the main loop
*
         K = K + KB
         GO TO 20
*
      END IF
*
   40 CONTINUE
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of SSYTRF
*
      END
```

———-

59.19.4 lisp code

— ssytrf.lisp —

```
(defun ssytrf (uplo n a lda ipiv work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array single-float (*)) work a)
           (type (f2cl-lib:integer4) info lwork lda n)
```

```
          (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a single-float a-%data% a-%offset%)
       (work single-float work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((iinfo 0) (iws 0) (j 0) (k 0) (kb 0) (ldwork 0) (lwkopt 0) (nb 0)
          (nbmin 0) (lquery nil) (upper nil))
      (declare (type f2cl-lib:logical upper lquery)
               (type (f2cl-lib:integer4) nbmin nb lwkopt ldwork kb k j iws
                iinfo))
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond
       ((and (not upper)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -4))
       ((and (< lwork 1) (not lquery)) (setf info -7)))
      (cond
       ((= info 0)
        (setf nb
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (ilaenv 1 "SSYTRF" uplo n -1 -1 -1)
                  (declare (ignore var-0 var-1 var-4 var-5 var-6))
                  (setf uplo var-2)
                  (setf n var-3)
                  ret-val))
        (setf lwkopt (f2cl-lib:int-mul n nb))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))))
      (cond
       ((/= info 0) (xerbla "SSYTRF" (f2cl-lib:int-sub info)) (go end_label))
       (lquery (go end_label)))
      (setf nbmin 2)
      (setf ldwork n)
```

```
(cond
 ((and (> nb 1) (< nb n)) (setf iws (f2cl-lib:int-mul ldwork nb))
  (cond
   ((< lwork iws)
    (setf nb (max (the f2cl-lib:integer4 (truncate lwork ldwork)) 1))
    (setf nbmin
            (max (the f2cl-lib:integer4 2)
                 (the f2cl-lib:integer4
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
                          (ilaenv 2 "SSYTRF" uplo n -1 -1 -1)
                        (declare (ignore var-0 var-1 var-4 var-5 var-6))
                        (setf uplo var-2)
                        (setf n var-3)
                        ret-val)))))))
 (t (setf iws 1)))
(if (< nb nbmin)
    (setf nb n))
(cond
 (upper
  (tagbody
    (setf k n)
   label10
    (if (< k 1)
        (go label40))
    (cond
     ((> k nb)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (slasyf uplo k nb kb a lda ipiv work ldwork iinfo)
        (declare (ignore var-1 var-2 var-4 var-6 var-7))
        (setf uplo var-0)
        (setf kb var-3)
        (setf lda var-5)
        (setf ldwork var-8)
        (setf iinfo var-9)))
     (t
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (ssytf2 uplo k a lda ipiv iinfo)
        (declare (ignore var-1 var-2 var-4))
        (setf uplo var-0)
        (setf lda var-3)
        (setf iinfo var-5))
      (setf kb k)))
    (if (and (= info 0) (> iinfo 0))
        (setf info iinfo))
    (setf k (f2cl-lib:int-sub k kb))
    (go label10)))
 (t
```

```
(tagbody
  (setf k 1)
 label20
  (if (> k n)
      (go label40))
  (cond
   ((<= k (f2cl-lib:int-add n (f2cl-lib:int-sub nb)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (slasyf uplo (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1) nb kb
         (f2cl-lib:array-slice a-%data% single-float (k k)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (k)
                               ((1 *)) ipiv-%offset%)
         work ldwork iinfo)
      (declare (ignore var-1 var-2 var-4 var-6 var-7))
      (setf uplo var-0)
      (setf kb var-3)
      (setf lda var-5)
      (setf ldwork var-8)
      (setf iinfo var-9)))
   (t
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (ssytf2 uplo (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
         (f2cl-lib:array-slice a-%data% single-float (k k)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (k)
                               ((1 *)) ipiv-%offset%)
         iinfo)
      (declare (ignore var-1 var-2 var-4))
      (setf uplo var-0)
      (setf lda var-3)
      (setf iinfo var-5))
    (setf kb (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1))))
  (if (and (= info 0) (> iinfo 0))
      (setf info (f2cl-lib:int-sub (f2cl-lib:int-add iinfo k) 1)))
  (f2cl-lib:fdo (j k (f2cl-lib:int-add j 1))
                ((> j (f2cl-lib:int-add k kb (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (cond
       ((> (f2cl-lib:fref ipiv (j) ((1 *))) 0)
        (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                (f2cl-lib:int-sub
                 (f2cl-lib:int-add
                  (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                  k)
                 1)))
       (t
```

```
                (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                        (f2cl-lib:int-add
                         (f2cl-lib:int-sub
                          (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                          k)
                         1))))
            label30))
          (setf k (f2cl-lib:int-add k kb))
          (go label20))))
     label40
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
      (go end_label)
     end_label
      (return (values uplo n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssytrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::ssytf2
                                              fortran-to-lisp::slasyf
                                              fortran-to-lisp::ilaenv))))
```

59.20 ssytrf_rook

59.20.1 Axiom unit tests

— ssytrfrook.input —

```
)set break resume
)sys rm -f ssytrfrook.output
)spool ssytrfrook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.20.2 Axiom help page

— ssytrf_rook.help —

```
====================================================================
ssytrf_rook examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssytrf_rook (UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO)
           SSYTRF_ROOK

Function/Subroutine Documentation
   subroutine ssytrf_rook (character UPLO, integer N, real, dimension( lda, *
       ) A, integer LDA, integer, dimension( * ) IPIV, real, dimension( * )
       WORK, integer LWORK, integer INFO)
       SSYTRF_ROOK

Purpose:
   SSYTRF_ROOK computes the factorization of a real symmetric matrix A
   using the bounded Bunch-Kaufman ("rook") diagonal pivoting method.
   The form of the factorization is
```

```
   A = U*D*U**T  or  A = L*D*L**T

   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, and D is symmetric and block diagonal with
   1-by-1 and 2-by-2 diagonal blocks.

   This is the blocked version of the algorithm, calling Level 3 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   N-by-N upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading N-by-N lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

   On exit, the block diagonal matrix D and the multipliers used
   to obtain the factor U or L (see below for further details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D.

   If UPLO = 'U':
      If IPIV(k) > 0, then rows and columns k and IPIV(k)
      were interchanged and D(k,k) is a 1-by-1 diagonal block.

      If IPIV(k) < 0 and IPIV(k-1) < 0, then rows and
      columns k and -IPIV(k) were interchanged and rows and
      columns k-1 and -IPIV(k-1) were inerchaged,
      D(k-1:k,k-1:k) is a 2-by-2 diagonal block.

   If UPLO = 'L':
```

```
     If IPIV(k) > 0, then rows and columns k and IPIV(k)
     were interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) < 0 and IPIV(k+1) < 0, then rows and
     columns k and -IPIV(k) were interchanged and rows and
     columns k+1 and -IPIV(k+1) were inerchaged,
     D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK)).
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of WORK.  LWORK >=1.  For best performance
  LWORK >= N*NB, where NB is the block size returned by ILAENV.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, D(i,i) is exactly zero.  The factorization
     has been completed, but the block diagonal matrix D is
     exactly singular, and division by zero will occur if it
     is used to solve a system of equations.

Further Details:

  If UPLO = 'U', then A = U*D*U**T, where
     U = P(n)*U(n)* ... *P(k)U(k)* ...,
  i.e., U is a product of terms P(k)*U(k), where k decreases from n to
  1 in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
  and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
  defined by IPIV(k), and U(k) is a unit upper triangular matrix, such
  that if the diagonal block D(k) is of order s (s = 1 or 2), then

                     (   I    v    0   )   k-s
             U(k) =  (   0    I    0   )   s
                     (   0    0    I   )   n-k
                        k-s   s   n-k

  If s = 1, D(k) overwrites A(k,k), and v overwrites A(1:k-1,k).
  If s = 2, the upper triangle of D(k) overwrites A(k-1,k-1), A(k-1,k),
  and A(k,k), and v overwrites A(1:k-2,k-1:k).
```

```
If UPLO = 'L', then A = L*D*L**T, where
   L = P(1)*L(1)* ... *P(k)*L(k)* ...,
i.e., L is a product of terms P(k)*L(k), where k increases from 1 to
n in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
defined by IPIV(k), and L(k) is a unit lower triangular matrix, such
that if the diagonal block D(k) is of order s (s = 1 or 2), then

                     (   I    0     0   )  k-1
             L(k) =  (   0    I     0   )  s
                     (   0    v     I   )  n-k-s+1
                        k-1   s  n-k-s+1

If s = 1, D(k) overwrites A(k,k), and v overwrites A(k+1:n,k).
If s = 2, the lower triangle of D(k) overwrites A(k,k), A(k+1,k),
and A(k+1,k+1), and v overwrites A(k+2:n,k:k+1).
```

59.20.3 fortran code

— ssytrf_rook.f —

```
*  =====================================================================
      SUBROUTINE SSYTRF_ROOK( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY, UPPER
      INTEGER            IINFO, IWS, J, K, KB, LDWORK, LWKOPT, NB, NBMIN
*     ..
*     .. External Functions ..
```

```
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLASYF_ROOK, SSYTF2_ROOK, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
*
      IF( INFO.EQ.0 ) THEN
*
*        Determine the block size
*
         NB = ILAENV( 1, 'SSYTRF_ROOK', UPLO, N, -1, -1, -1 )
         LWKOPT = N*NB
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYTRF_ROOK', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
      NBMIN = 2
      LDWORK = N
      IF( NB.GT.1 .AND. NB.LT.N ) THEN
         IWS = LDWORK*NB
         IF( LWORK.LT.IWS ) THEN
            NB = MAX( LWORK / LDWORK, 1 )
            NBMIN = MAX( 2, ILAENV( 2, 'SSYTRF_ROOK',
```

```
     $                              UPLO, N, -1, -1, -1 ) )
         END IF
      ELSE
         IWS = 1
      END IF
      IF( NB.LT.NBMIN )
     $   NB = N
*
      IF( UPPER ) THEN
*
*        Factorize A as U*D*U**T using the upper triangle of A
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        KB, where KB is the number of columns factorized by SLASYF_ROOK;
*        KB is either NB or NB-1, or K for the last block
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop
*
         IF( K.LT.1 )
     $      GO TO 40
*
         IF( K.GT.NB ) THEN
*
*           Factorize columns k-kb+1:k of A and use blocked code to
*           update columns 1:k-kb
*
            CALL SLASYF_ROOK( UPLO, K, NB, KB, A, LDA,
     $                        IPIV, WORK, LDWORK, IINFO )
         ELSE
*
*           Use unblocked code to factorize columns 1:k of A
*
            CALL SSYTF2_ROOK( UPLO, K, A, LDA, IPIV, IINFO )
            KB = K
         END IF
*
*        Set INFO on the first occurrence of a zero pivot
*
         IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO
*
*        No need to adjust IPIV
*
*        Decrease K and return to the start of the main loop
*
         K = K - KB
         GO TO 10
```

```
*
      ELSE
*
*        Factorize A as L*D*L**T using the lower triangle of A
*
*        K is the main loop index, increasing from 1 to N in steps of
*        KB, where KB is the number of columns factorized by SLASYF_ROOK;
*        KB is either NB or NB-1, or N-K+1 for the last block
*
         K = 1
   20    CONTINUE
*
*        If K > N, exit from loop
*
         IF( K.GT.N )
     $      GO TO 40
*
         IF( K.LE.N-NB ) THEN
*
*          Factorize columns k:k+kb-1 of A and use blocked code to
*          update columns k+kb:n
*
            CALL SLASYF_ROOK( UPLO, N-K+1, NB, KB, A( K, K ), LDA,
     $                        IPIV( K ), WORK, LDWORK, IINFO )
         ELSE
*
*          Use unblocked code to factorize columns k:n of A
*
            CALL SSYTF2_ROOK( UPLO, N-K+1, A( K, K ), LDA, IPIV( K ),
     $                   IINFO )
            KB = N - K + 1
         END IF
*
*        Set INFO on the first occurrence of a zero pivot
*
         IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO + K - 1
*
*        Adjust IPIV
*
         DO 30 J = K, K + KB - 1
            IF( IPIV( J ).GT.0 ) THEN
               IPIV( J ) = IPIV( J ) + K - 1
            ELSE
               IPIV( J ) = IPIV( J ) - K + 1
            END IF
   30    CONTINUE
*
*        Increase K and return to the start of the main loop
*
```

```
         K = K + KB
         GO TO 20
*
      END IF
*
   40 CONTINUE
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of SSYTRF_ROOK
*
      END
```

———-

59.20.4 lisp code

— **ssytrf_rook.lisp** —

```
(defun ssytrf_rook (uplo n a lda ipiv work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array single-float (*)) work a)
           (type (f2cl-lib:integer4) info lwork lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a single-float a-%data% a-%offset%)
       (work single-float work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((iinfo 0) (iws 0) (j 0) (k 0) (kb 0) (ldwork 0) (lwkopt 0) (nb 0)
           (nbmin 0) (lquery nil) (upper nil))
      (declare (type f2cl-lib:logical upper lquery)
               (type (f2cl-lib:integer4) nbmin nb lwkopt ldwork kb k j iws
                iinfo))
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond
       ((and (not upper)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
                (declare (ignore var-1))
```

```
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4))
 ((and (< lwork 1) (not lquery)) (setf info -7)))
(cond
 ((= info 0)
  (setf nb
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (ilaenv 1 "SSYTRF_ROOK" uplo n -1 -1 -1)
            (declare (ignore var-0 var-1 var-4 var-5 var-6))
            (setf uplo var-2)
            (setf n var-3)
            ret-val))
  (setf lwkopt (f2cl-lib:int-mul n nb))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))))
(cond
 ((/= info 0) (xerbla "SSYTRF_ROOK" (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(setf nbmin 2)
(setf ldwork n)
(cond
 ((and (> nb 1) (< nb n)) (setf iws (f2cl-lib:int-mul ldwork nb))
  (cond
   ((< lwork iws)
    (setf nb (max (the f2cl-lib:integer4 (truncate lwork ldwork)) 1))
    (setf nbmin
            (max (the f2cl-lib:integer4 2)
                 (the f2cl-lib:integer4
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
                          (ilaenv 2 "SSYTRF_ROOK" uplo n -1 -1 -1)
                        (declare (ignore var-0 var-1 var-4 var-5 var-6))
                        (setf uplo var-2)
                        (setf n var-3)
                        ret-val)))))))
 (t (setf iws 1)))
(if (< nb nbmin)
    (setf nb n))
(cond
 (upper
  (tagbody
    (setf k n)
   label10
```

```
    (if (< k 1)
        (go label40))
    (cond
     ((> k nb)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (slasyf_rook uplo k nb kb a lda ipiv work ldwork iinfo)
        (declare (ignore var-1 var-2 var-4 var-6 var-7))
        (setf uplo var-0)
        (setf kb var-3)
        (setf lda var-5)
        (setf ldwork var-8)
        (setf iinfo var-9)))
     (t
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (ssytf2_rook uplo k a lda ipiv iinfo)
        (declare (ignore var-1 var-2 var-4))
        (setf uplo var-0)
        (setf lda var-3)
        (setf iinfo var-5))
      (setf kb k)))
    (if (and (= info 0) (> iinfo 0))
        (setf info iinfo))
    (setf k (f2cl-lib:int-sub k kb))
    (go label10)))
  (t
   (tagbody
    (setf k 1)
   label20
    (if (> k n)
        (go label40))
    (cond
     ((<= k (f2cl-lib:int-add n (f2cl-lib:int-sub nb)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (slasyf_rook uplo (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
           nb kb
           (f2cl-lib:array-slice a-%data% single-float (k k)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (k)
                                 ((1 *)) ipiv-%offset%)
           work ldwork iinfo)
        (declare (ignore var-1 var-2 var-4 var-6 var-7))
        (setf uplo var-0)
        (setf kb var-3)
        (setf lda var-5)
        (setf ldwork var-8)
        (setf iinfo var-9)))
     (t
```

```
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                (ssytf2_rook uplo (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
                 (f2cl-lib:array-slice a-%data% single-float (k k)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (k)
                                       ((1 *)) ipiv-%offset%)
                 iinfo)
              (declare (ignore var-1 var-2 var-4))
              (setf uplo var-0)
              (setf lda var-3)
              (setf iinfo var-5))
            (setf kb (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1))))
          (if (and (= info 0) (> iinfo 0))
              (setf info (f2cl-lib:int-sub (f2cl-lib:int-add iinfo k) 1)))
          (f2cl-lib:fdo (j k (f2cl-lib:int-add j 1))
                        ((> j (f2cl-lib:int-add k kb (f2cl-lib:int-sub 1))) nil)
            (tagbody
              (cond
               ((> (f2cl-lib:fref ipiv (j) ((1 *))) 0)
                (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                        (f2cl-lib:int-sub
                         (f2cl-lib:int-add
                          (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                          k)
                         1)))
               (t
                (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                        (f2cl-lib:int-add
                         (f2cl-lib:int-sub
                          (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                          k)
                         1))))
             label30))
          (setf k (f2cl-lib:int-add k kb))
          (go label20))))
     label40
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 lwkopt) 'single-float))
      (go end_label)
     end_label
      (return (values uplo n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssytrf_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
```

```
                                        (fortran-to-lisp::integer4)
                                        (array single-float (*))
                                        (fortran-to-lisp::integer4)
                                        (array fortran-to-lisp::integer4
                                         (*))
                                        (array single-float (*))
                                        (fortran-to-lisp::integer4)
                                        (fortran-to-lisp::integer4))
                                      :return-values
                                      '(fortran-to-lisp::uplo
                                        fortran-to-lisp::n nil
                                        fortran-to-lisp::lda nil nil nil
                                        fortran-to-lisp::info)
                                      :calls
                                      '(fortran-to-lisp::ssytf2_rook
                                        fortran-to-lisp::slasyf_rook
                                        fortran-to-lisp::ilaenv))))
```

———-

59.21 ssytri

59.21.1 Axiom unit tests

— **ssytri.input** —

```
)set break resume
)sys rm -f ssytri.output
)spool ssytri.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.21.2 Axiom help page

— **ssytri.help** —

```
==================================================================
```

```
ssytri examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssytri (UPLO, N, A, LDA, IPIV, WORK, INFO)
           SSYTRI

Function/Subroutine Documentation
   subroutine ssytri (character UPLO, integer N, real, dimension( lda, * ) A,
       integer LDA, integer, dimension( * ) IPIV, real, dimension( * ) WORK,
       integer INFO)
       SSYTRI

Purpose:
   SSYTRI computes the inverse of a real symmetric indefinite matrix
   A using the factorization A = U*D*U**T or A = L*D*L**T computed by SSYTRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the block diagonal matrix D and the multipliers
   used to obtain the factor U or L as computed by SSYTRF.

   On exit, if INFO = 0, the (symmetric) inverse of the original
   matrix.  If UPLO = 'U', the upper triangular part of the
   inverse is formed and the part of A below the diagonal is not
   referenced; if UPLO = 'L' the lower triangular part of the
   inverse is formed and the part of A above the diagonal is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).
```

```
 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by SSYTRF.

 WORK
   WORK is REAL array, dimension (N)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value
   > 0: if INFO = i, D(i,i) = 0; the matrix is singular and its
        inverse could not be computed.
```

———-

59.21.3 fortran code

— **ssytri.f** —

```
*  =====================================================================
      SUBROUTINE SSYTRI( UPLO, N, A, LDA, IPIV, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
```

```
      INTEGER            K, KP, KSTEP
      REAL               AK, AKKP1, AKP1, D, T, TEMP
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SDOT
      EXTERNAL           LSAME, SDOT
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SSWAP, SSYMV, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYTRI', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
         DO 10 INFO = N, 1, -1
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
   10    CONTINUE
      ELSE
*
*        Lower triangular storage: examine D from top to bottom.
```

```
*
         DO 20 INFO = 1, N
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
   20    CONTINUE
      END IF
      INFO = 0
*
      IF( UPPER ) THEN
*
*        Compute inv(A) from the factorization A = U*D*U**T.
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = 1
   30    CONTINUE
*
*        If K > N, exit from loop.
*
         IF( K.GT.N )
     $      GO TO 40
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Invert the diagonal block.
*
            A( K, K ) = ONE / A( K, K )
*
*           Compute column K of the inverse.
*
            IF( K.GT.1 ) THEN
               CALL SCOPY( K-1, A( 1, K ), 1, WORK, 1 )
               CALL SSYMV( UPLO, K-1, -ONE, A, LDA, WORK, 1, ZERO,
     $                     A( 1, K ), 1 )
               A( K, K ) = A( K, K ) - SDOT( K-1, WORK, 1, A( 1, K ),
     $                     1 )
            END IF
            KSTEP = 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Invert the diagonal block.
*
            T = ABS( A( K, K+1 ) )
            AK = A( K, K ) / T
            AKP1 = A( K+1, K+1 ) / T
```

```
            AKKP1 = A( K, K+1 ) / T
            D = T*( AK*AKP1-ONE )
            A( K, K ) = AKP1 / D
            A( K+1, K+1 ) = AK / D
            A( K, K+1 ) = -AKKP1 / D
*
*           Compute columns K and K+1 of the inverse.
*
            IF( K.GT.1 ) THEN
               CALL SCOPY( K-1, A( 1, K ), 1, WORK, 1 )
               CALL SSYMV( UPLO, K-1, -ONE, A, LDA, WORK, 1, ZERO,
     $                     A( 1, K ), 1 )
               A( K, K ) = A( K, K ) - SDOT( K-1, WORK, 1, A( 1, K ),
     $                     1 )
               A( K, K+1 ) = A( K, K+1 ) -
     $                       SDOT( K-1, A( 1, K ), 1, A( 1, K+1 ), 1 )
               CALL SCOPY( K-1, A( 1, K+1 ), 1, WORK, 1 )
               CALL SSYMV( UPLO, K-1, -ONE, A, LDA, WORK, 1, ZERO,
     $                     A( 1, K+1 ), 1 )
               A( K+1, K+1 ) = A( K+1, K+1 ) -
     $                         SDOT( K-1, WORK, 1, A( 1, K+1 ), 1 )
            END IF
            KSTEP = 2
         END IF
*
         KP = ABS( IPIV( K ) )
         IF( KP.NE.K ) THEN
*
*           Interchange rows and columns K and KP in the leading
*           submatrix A(1:k+1,1:k+1)
*
            CALL SSWAP( KP-1, A( 1, K ), 1, A( 1, KP ), 1 )
            CALL SSWAP( K-KP-1, A( KP+1, K ), 1, A( KP, KP+1 ), LDA )
            TEMP = A( K, K )
            A( K, K ) = A( KP, KP )
            A( KP, KP ) = TEMP
            IF( KSTEP.EQ.2 ) THEN
               TEMP = A( K, K+1 )
               A( K, K+1 ) = A( KP, K+1 )
               A( KP, K+1 ) = TEMP
            END IF
         END IF
*
         K = K + KSTEP
         GO TO 30
   40    CONTINUE
*
      ELSE
*
*        Compute inv(A) from the factorization A = L*D*L**T.
```

```
*
*       K is the main loop index, increasing from 1 to N in steps of
*       1 or 2, depending on the size of the diagonal blocks.
*
        K = N
   50   CONTINUE
*
*       If K < 1, exit from loop.
*
        IF( K.LT.1 )
     $     GO TO 60
*
        IF( IPIV( K ).GT.0 ) THEN
*
*          1 x 1 diagonal block
*
*          Invert the diagonal block.
*
           A( K, K ) = ONE / A( K, K )
*
*          Compute column K of the inverse.
*
           IF( K.LT.N ) THEN
              CALL SCOPY( N-K, A( K+1, K ), 1, WORK, 1 )
              CALL SSYMV( UPLO, N-K, -ONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                    ZERO, A( K+1, K ), 1 )
              A( K, K ) = A( K, K ) - SDOT( N-K, WORK, 1, A( K+1, K ),
     $                    1 )
           END IF
           KSTEP = 1
        ELSE
*
*          2 x 2 diagonal block
*
*          Invert the diagonal block.
*
           T = ABS( A( K, K-1 ) )
           AK = A( K-1, K-1 ) / T
           AKP1 = A( K, K ) / T
           AKKP1 = A( K, K-1 ) / T
           D = T*( AK*AKP1-ONE )
           A( K-1, K-1 ) = AKP1 / D
           A( K, K ) = AK / D
           A( K, K-1 ) = -AKKP1 / D
*
*          Compute columns K-1 and K of the inverse.
*
           IF( K.LT.N ) THEN
              CALL SCOPY( N-K, A( K+1, K ), 1, WORK, 1 )
              CALL SSYMV( UPLO, N-K, -ONE, A( K+1, K+1 ), LDA, WORK, 1,
```

```
     $                     ZERO, A( K+1, K ), 1 )
               A( K, K ) = A( K, K ) - SDOT( N-K, WORK, 1, A( K+1, K ),
     $                     1 )
               A( K, K-1 ) = A( K, K-1 ) -
     $                       SDOT( N-K, A( K+1, K ), 1, A( K+1, K-1 ),
     $                       1 )
               CALL SCOPY( N-K, A( K+1, K-1 ), 1, WORK, 1 )
               CALL SSYMV( UPLO, N-K, -ONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                     ZERO, A( K+1, K-1 ), 1 )
               A( K-1, K-1 ) = A( K-1, K-1 ) -
     $                         SDOT( N-K, WORK, 1, A( K+1, K-1 ), 1 )
            END IF
            KSTEP = 2
         END IF
*
         KP = ABS( IPIV( K ) )
         IF( KP.NE.K ) THEN
*
*           Interchange rows and columns K and KP in the trailing
*           submatrix A(k-1:n,k-1:n)
*
            IF( KP.LT.N )
     $         CALL SSWAP( N-KP, A( KP+1, K ), 1, A( KP+1, KP ), 1 )
            CALL SSWAP( KP-K-1, A( K+1, K ), 1, A( KP, K+1 ), LDA )
            TEMP = A( K, K )
            A( K, K ) = A( KP, KP )
            A( KP, KP ) = TEMP
            IF( KSTEP.EQ.2 ) THEN
               TEMP = A( K, K-1 )
               A( K, K-1 ) = A( KP, K-1 )
               A( KP, K-1 ) = TEMP
            END IF
         END IF
*
         K = K - KSTEP
         GO TO 50
   60    CONTINUE
      END IF
*
      RETURN
*
*     End of SSYTRI
*
      END
```

———-

59.21.4 lisp code

— ssytri.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun ssytri (uplo n a lda ipiv work info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array single-float (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (work single-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((ak 0.0) (akkp1 0.0) (akp1 0.0) (d 0.0) (t$ 0.0) (temp 0.0) (k 0)
             (kp 0) (kstep 0) (upper nil))
        (declare (type (single-float) ak akkp1 akp1 d t$ temp)
                 (type (f2cl-lib:integer4) k kp kstep)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -4)))
        (cond
         ((/= info 0) (xerbla "SSYTRI" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (= n 0)
            (go end_label))
        (cond
         (upper
```

```
  (f2cl-lib:fdo (info n (f2cl-lib:int-add info (f2cl-lib:int-sub 1)))
                ((> info 1) nil)
    (tagbody
      (if (and
           (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
              0)
           (=
            (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                           a-%offset%)
            zero))
          (go end_label))
     label10)))
 (t
  (f2cl-lib:fdo (info 1 (f2cl-lib:int-add info 1))
                ((> info n) nil)
    (tagbody
      (if (and
           (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
              0)
           (=
            (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                           a-%offset%)
            zero))
          (go end_label))
     label20))))
(setf info 0)
(cond
 (upper
  (tagbody
    (setf k 1)
   label30
    (if (> k n)
        (go label40))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
              (/ one
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%)))
      (cond
       ((> k 1)
        (scopy (f2cl-lib:int-sub k 1)
         (f2cl-lib:array-slice a-%data% single-float (1 k)
                               ((1 lda) (1 *)) a-%offset%)
         1 work 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9)
            (ssymv uplo (f2cl-lib:int-sub k 1) (- one) a lda work 1
             zero
```

```
          (f2cl-lib:array-slice a-%data% single-float (1 k)
                                ((1 lda) (1 *)) a-%offset%)
          1)
        (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
        (when var-0 (setf uplo var-0))
        (when var-4 (setf lda var-4))
        (when var-7 (setf zero var-7)))
      (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
              (-
               (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                              a-%offset%)
               (sdot (f2cl-lib:int-sub k 1) work 1
                (f2cl-lib:array-slice a-%data% single-float (1 k)
                                      ((1 lda) (1 *)) a-%offset%)
                1)))))
    (setf kstep 1))
   (t
    (setf t$
            (abs
             (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                            ((1 lda) (1 *)) a-%offset%)))
    (setf ak
            (/
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)
             t$))
    (setf akp1
            (/
             (f2cl-lib:fref a-%data%
                            ((f2cl-lib:int-add k 1)
                             (f2cl-lib:int-add k 1))
                            ((1 lda) (1 *)) a-%offset%)
             t$))
    (setf akkp1
            (/
             (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                            ((1 lda) (1 *)) a-%offset%)
             t$))
    (setf d (* t$ (- (* ak akp1) one)))
    (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
            (/ akp1 d))
    (setf (f2cl-lib:fref a-%data%
                         ((f2cl-lib:int-add k 1)
                          (f2cl-lib:int-add k 1))
                         ((1 lda) (1 *)) a-%offset%)
            (/ ak d))
    (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                         ((1 lda) (1 *)) a-%offset%)
            (/ (- akkp1) d))
    (cond
```

```
((> k 1)
 (scopy (f2cl-lib:int-sub k 1)
  (f2cl-lib:array-slice a-%data% single-float (1 k)
                        ((1 lda) (1 *)) a-%offset%)
  1 work 1)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9)
     (ssymv uplo (f2cl-lib:int-sub k 1) (- one) a lda work 1
      zero
      (f2cl-lib:array-slice a-%data% single-float (1 k)
                            ((1 lda) (1 *)) a-%offset%)
      1)
   (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
   (when var-0 (setf uplo var-0))
   (when var-4 (setf lda var-4))
   (when var-7 (setf zero var-7)))
 (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
         (-
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%)
          (sdot (f2cl-lib:int-sub k 1) work 1
           (f2cl-lib:array-slice a-%data% single-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1)))
 (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                       ((1 lda) (1 *)) a-%offset%)
         (-
          (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                         ((1 lda) (1 *)) a-%offset%)
          (sdot (f2cl-lib:int-sub k 1)
           (f2cl-lib:array-slice a-%data% single-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% single-float
                                 (1 (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           1)))
 (scopy (f2cl-lib:int-sub k 1)
  (f2cl-lib:array-slice a-%data% single-float
                        (1 (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
  1 work 1)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9)
     (ssymv uplo (f2cl-lib:int-sub k 1) (- one) a lda work 1
      zero
      (f2cl-lib:array-slice a-%data% single-float
                            (1 (f2cl-lib:int-add k 1))
```

```
                                  ((1 lda) (1 *)) a-%offset%)
          1)
       (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
       (when var-0 (setf uplo var-0))
       (when var-4 (setf lda var-4))
       (when var-7 (setf zero var-7)))
     (setf (f2cl-lib:fref a-%data%
                          ((f2cl-lib:int-add k 1)
                           (f2cl-lib:int-add k 1))
                          ((1 lda) (1 *)) a-%offset%)
             (-
              (f2cl-lib:fref a-%data%
                             ((f2cl-lib:int-add k 1)
                              (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
              (sdot (f2cl-lib:int-sub k 1) work 1
               (f2cl-lib:array-slice a-%data% single-float
                                     (1 (f2cl-lib:int-add k 1))
                                     ((1 lda) (1 *)) a-%offset%)
               1)))))
  (setf kstep 2)))
(setf kp
        (abs
         (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
(cond
 ((/= kp k)
  (sswap (f2cl-lib:int-sub kp 1)
   (f2cl-lib:array-slice a-%data% single-float (1 k)
                         ((1 lda) (1 *)) a-%offset%)
   1
   (f2cl-lib:array-slice a-%data% single-float (1 kp)
                         ((1 lda) (1 *)) a-%offset%)
   1)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (sswap (f2cl-lib:int-sub k kp 1)
       (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% single-float
                             (kp (f2cl-lib:int-add kp 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (setf temp
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
          (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%))
```

```
      (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *)) a-%offset%)
             temp)
      (cond
       ((= kstep 2)
        (setf temp
                (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%))
        (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%))
        (setf (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
                temp)))))
    (setf k (f2cl-lib:int-add k kstep))
    (go label30)
   label40))
  (t
   (tagbody
    (setf k n)
   label50
    (if (< k 1)
        (go label60))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
             (/ one
                (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                               a-%offset%)))
      (cond
       ((< k n)
        (scopy (f2cl-lib:int-sub n k)
         (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                               ((1 lda) (1 *)) a-%offset%)
         1 work 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9)
            (ssymv uplo (f2cl-lib:int-sub n k) (- one)
             (f2cl-lib:array-slice a-%data% single-float
                                   ((+ k 1) (f2cl-lib:int-add k 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda work 1 zero
             (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
          (when var-0 (setf uplo var-0))
          (when var-4 (setf lda var-4))
          (when var-7 (setf zero var-7)))
```

```
    (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
            (-
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)
             (sdot (f2cl-lib:int-sub n k) work 1
              (f2cl-lib:array-slice a-%data% single-float
                                    ((+ k 1) k) ((1 lda) (1 *))
                                    a-%offset%)
              1)))))
  (setf kstep 1))
 (t
  (setf t$
          (abs
           (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                          ((1 lda) (1 *)) a-%offset%)))
  (setf ak
          (/
           (f2cl-lib:fref a-%data%
                          ((f2cl-lib:int-sub k 1)
                           (f2cl-lib:int-sub k 1))
                          ((1 lda) (1 *)) a-%offset%)
           t$))
  (setf akp1
          (/
           (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                          a-%offset%)
           t$))
  (setf akkp1
          (/
           (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                          ((1 lda) (1 *)) a-%offset%)
           t$))
  (setf d (* t$ (- (* ak akp1) one)))
  (setf (f2cl-lib:fref a-%data%
                       ((f2cl-lib:int-sub k 1)
                        (f2cl-lib:int-sub k 1))
                       ((1 lda) (1 *)) a-%offset%)
          (/ akp1 d))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
          (/ ak d))
  (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                       ((1 lda) (1 *)) a-%offset%)
          (/ (- akkp1) d))
  (cond
   ((< k n)
    (scopy (f2cl-lib:int-sub n k)
     (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                           ((1 lda) (1 *)) a-%offset%)
     1 work 1)
    (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (ssymv uplo (f2cl-lib:int-sub n k) (- one)
     (f2cl-lib:array-slice a-%data% single-float
                           ((+ k 1) (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda work 1 zero
     (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                           ((1 lda) (1 *)) a-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
  (when var-0 (setf uplo var-0))
  (when var-4 (setf lda var-4))
  (when var-7 (setf zero var-7)))
(setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         (sdot (f2cl-lib:int-sub n k) work 1
          (f2cl-lib:array-slice a-%data% single-float
                                ((+ k 1) k) ((1 lda) (1 *))
                                a-%offset%)
          1)))
(setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         (sdot (f2cl-lib:int-sub n k)
          (f2cl-lib:array-slice a-%data% single-float
                                ((+ k 1) k) ((1 lda) (1 *))
                                a-%offset%)
          1
          (f2cl-lib:array-slice a-%data% single-float
                                ((+ k 1)
                                 (f2cl-lib:int-sub k 1))
                                ((1 lda) (1 *)) a-%offset%)
          1)))
(scopy (f2cl-lib:int-sub n k)
 (f2cl-lib:array-slice a-%data% single-float
                       ((+ k 1) (f2cl-lib:int-sub k 1))
                       ((1 lda) (1 *)) a-%offset%)
 1 work 1)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (ssymv uplo (f2cl-lib:int-sub n k) (- one)
     (f2cl-lib:array-slice a-%data% single-float
                           ((+ k 1) (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
```

```
          lda work 1 zero
          (f2cl-lib:array-slice a-%data% single-float
                                ((+ k 1) (f2cl-lib:int-sub k 1))
                                ((1 lda) (1 *)) a-%offset%)
          1)
      (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
      (when var-0 (setf uplo var-0))
      (when var-4 (setf lda var-4))
      (when var-7 (setf zero var-7)))
    (setf (f2cl-lib:fref a-%data%
                         ((f2cl-lib:int-sub k 1)
                          (f2cl-lib:int-sub k 1))
                         ((1 lda) (1 *)) a-%offset%)
            (-
             (f2cl-lib:fref a-%data%
                            ((f2cl-lib:int-sub k 1)
                             (f2cl-lib:int-sub k 1))
                            ((1 lda) (1 *)) a-%offset%)
             (sdot (f2cl-lib:int-sub n k) work 1
              (f2cl-lib:array-slice a-%data% single-float
                                    ((+ k 1)
                                     (f2cl-lib:int-sub k 1))
                                    ((1 lda) (1 *)) a-%offset%)
              1)))))
  (setf kstep 2)))
(setf kp
        (abs
         (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
(cond
 ((/= kp k)
  (if (< kp n)
      (sswap (f2cl-lib:int-sub n kp)
       (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) kp)
                             ((1 lda) (1 *)) a-%offset%)
       1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (sswap (f2cl-lib:int-sub kp k 1)
       (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% single-float
                             (kp (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (setf temp
```

```
                (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                               a-%offset%))
        (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                               a-%offset%))
        (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *)) a-%offset%)
                temp)
        (cond
         ((= kstep 2)
          (setf temp
                  (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                                 ((1 lda) (1 *)) a-%offset%))
          (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                               ((1 lda) (1 *)) a-%offset%)
                  (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-sub k 1))
                                 ((1 lda) (1 *)) a-%offset%))
          (setf (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-sub k 1))
                               ((1 lda) (1 *)) a-%offset%)
                  temp)))))
      (setf k (f2cl-lib:int-sub k kstep))
      (go label50)
     label60)))
   (go end_label)
  end_label
   (return (values uplo nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssytri
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

59.22 ssytri2

59.22.1 Axiom unit tests

— **ssytri2.input** —

```
)set break resume
)sys rm -f ssytri2.output
)spool ssytri2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.22.2 Axiom help page

— **ssytri2.help** —

```
======================================================================
ssytri2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssytri2 (UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO)
           SSYTRI2

Function/Subroutine Documentation
   subroutine ssytri2 (character UPLO, integer N, real, dimension( lda, * ) A,
       integer LDA, integer, dimension( * ) IPIV, real, dimension( * ) WORK,
       integer LWORK, integer INFO)
       SSYTRI2

Purpose:
   SSYTRI2 computes the inverse of a REAL symmetric indefinite matrix
   A using the factorization A = U*D*U**T or A = L*D*L**T computed by
   SSYTRF. SSYTRI2 sets the LEADING DIMENSION of the workspace
   before calling SSYTRI2X that actually computes the inverse.
```

```
Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the NB diagonal matrix D and the multipliers
   used to obtain the factor U or L as computed by SSYTRF.

   On exit, if INFO = 0, the (symmetric) inverse of the original
   matrix.  If UPLO = 'U', the upper triangular part of the
   inverse is formed and the part of A below the diagonal is not
   referenced; if UPLO = 'L' the lower triangular part of the
   inverse is formed and the part of A above the diagonal is not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the NB structure of D
   as determined by SSYTRF.

 WORK
   WORK is REAL array, dimension (N+NB+1)*(NB+3)

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.
   WORK is size >= (N+NB+1)*(NB+3)
   If LDWORK = -1, then a workspace query is assumed; the routine
   calculates:
   - the optimal size of the WORK array, returns
   this value as the first entry of the WORK array,
   - and no error message related to LDWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
   = 0: successful exit
```

```
< 0: if INFO = -i, the i-th argument had an illegal value
> 0: if INFO = i, D(i,i) = 0; the matrix is singular and its
     inverse could not be computed.
```

59.22.3 fortran code

— ssytri2.f —

```
*  =====================================================================
      SUBROUTINE SSYTRI2( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            UPPER, LQUERY
      INTEGER            MINSIZE, NBMAX
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           SSYTRI, SSYTRI2X
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
```

```
      LQUERY = ( LWORK.EQ.-1 )
*     Get blocksize
      NBMAX = ILAENV( 1, 'SSYTRF', UPLO, N, -1, -1, -1 )
      IF ( NBMAX .GE. N ) THEN
         MINSIZE = N
      ELSE
         MINSIZE = (N+NBMAX+1)*(NBMAX+3)
      END IF
*
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF (LWORK .LT. MINSIZE .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
*
*     Quick return if possible
*
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYTRI2', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         WORK(1)=MINSIZE
         RETURN
      END IF
      IF( N.EQ.0 )
     $   RETURN

      IF( NBMAX .GE. N ) THEN
         CALL SSYTRI( UPLO, N, A, LDA, IPIV, WORK, INFO )
      ELSE
         CALL SSYTRI2X( UPLO, N, A, LDA, IPIV, WORK, NBMAX, INFO )
      END IF
      RETURN
*
*     End of SSYTRI2
*
      END
```

———-

59.22.4 lisp code

— ssytri2.lisp —

```
(defun ssytri2 (uplo n a lda ipiv work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array single-float (*)) work a)
           (type (f2cl-lib:integer4) info lwork lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a single-float a-%data% a-%offset%)
       (work single-float work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((minsize 0) (nbmax 0) (upper nil) (lquery nil))
      (declare (type f2cl-lib:logical lquery upper)
               (type (f2cl-lib:integer4) nbmax minsize))
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (setf nbmax
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 1 "SSYTRF" uplo n -1 -1 -1)
                (declare (ignore var-0 var-1 var-4 var-5 var-6))
                (setf uplo var-2)
                (setf n var-3)
                ret-val))
      (cond ((>= nbmax n) (setf minsize n))
            (t
             (setf minsize
                     (f2cl-lib:int-mul (f2cl-lib:int-add n nbmax 1)
                                       (f2cl-lib:int-add nbmax 3)))))
      (cond
       ((and (not upper)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -4))
       ((and (< lwork minsize) (not lquery)) (setf info -7)))
      (cond
       ((/= info 0) (xerbla "SSYTRI2" (f2cl-lib:int-sub info)) (go end_label))
       (lquery
```

```
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 minsize) 'single-float))
        (go end_label)))
      (if (= n 0)
          (go end_label))
      (cond
       ((>= nbmax n)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ssytri uplo n a lda ipiv work info)
          (declare (ignore var-2 var-4 var-5))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-6 (setf info var-6))))
       (t
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (ssytri2x uplo n a lda ipiv work nbmax info)
          (declare (ignore var-2 var-4 var-5))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-6 (setf nbmax var-6))
          (when var-7 (setf info var-7)))))
      (go end_label)
     end_label
      (return (values uplo n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssytri2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::ilaenv))))
```

———-

59.23 ssytri2x

59.23.1 Axiom unit tests

— **ssytri2x.input** —

```
)set break resume
)sys rm -f ssytri2x.output
)spool ssytri2x.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.23.2 Axiom help page

— **ssytri2x.help** —

```
====================================================================
ssytri2x examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssytri2x (UPLO, N, A, LDA, IPIV, WORK, NB, INFO)
           SSYTRI2X

Function/Subroutine Documentation
   subroutine ssytri2x (character UPLO, integer N, real, dimension( lda, * )
       A, integer LDA, integer, dimension( * ) IPIV, real, dimension( n+nb+1,*
       ) WORK, integer NB, integer INFO)
       SSYTRI2X

Purpose:
```

```
  SSYTRI2X computes the inverse of a real symmetric indefinite matrix
  A using the factorization A = U*D*U**T or A = L*D*L**T computed by SSYTRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the NNB diagonal matrix D and the multipliers
   used to obtain the factor U or L as computed by SSYTRF.

   On exit, if INFO = 0, the (symmetric) inverse of the original
   matrix.  If UPLO = 'U', the upper triangular part of the
   inverse is formed and the part of A below the diagonal is not
   referenced; if UPLO = 'L' the lower triangular part of the
   inverse is formed and the part of A above the diagonal is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the NNB structure of D
   as determined by SSYTRF.

 WORK
   WORK is REAL array, dimension (N+NNB+1,NNB+3)

 NB
   NB is INTEGER
   Block size

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value
   > 0: if INFO = i, D(i,i) = 0; the matrix is singular and its
        inverse could not be computed.
```

———-

59.23.3 fortran code

— **ssytri2x.f** —

```
*  =====================================================================
      SUBROUTINE SSYTRI2X( UPLO, N, A, LDA, IPIV, WORK, NB, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N, NB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), WORK( N+NB+1,* )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, IINFO, IP, K, CUT, NNB
      INTEGER            COUNT
      INTEGER            J, U11, INVD

      REAL               AK, AKKP1, AKP1, D, T
      REAL               U01_I_J, U01_IP1_J
      REAL               U11_I_J, U11_IP1_J
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SSYCONV, XERBLA, STRTRI
      EXTERNAL           SGEMM, STRMM, SSYSWAPR
```

```
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
*
*     Quick return if possible
*
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYTRI2X', -INFO )
         RETURN
      END IF
      IF( N.EQ.0 )
     $   RETURN
*
*     Convert A
*     Workspace got Non-diag elements of D
*
      CALL SSYCONV( UPLO, 'C', N, A, LDA, IPIV, WORK, IINFO )
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
         DO INFO = N, 1, -1
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
         END DO
      ELSE
*
*        Lower triangular storage: examine D from top to bottom.
*
         DO INFO = 1, N
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
```

```
         END DO
      END IF
      INFO = 0
*
*  Splitting Workspace
*     U01 is a block (N,NB+1)
*     The first element of U01 is in WORK(1,1)
*     U11 is a block (NB+1,NB+1)
*     The first element of U11 is in WORK(N+1,1)
      U11 = N
*     INVD is a block (N,2)
*     The first element of INVD is in WORK(1,INVD)
      INVD = NB+2

      IF( UPPER ) THEN
*
*        invA = P * inv(U**T)*inv(D)*inv(U)*P**T.
*
        CALL STRTRI( UPLO, 'U', N, A, LDA, INFO )
*
*       inv(D) and inv(D)*inv(U)
*
        K=1
        DO WHILE ( K .LE. N )
         IF( IPIV( K ).GT.0 ) THEN
*           1 x 1 diagonal NNB
             WORK(K,INVD) = ONE /  A( K, K )
             WORK(K,INVD+1) = 0
            K=K+1
         ELSE
*           2 x 2 diagonal NNB
             T = WORK(K+1,1)
             AK = A( K, K ) / T
             AKP1 = A( K+1, K+1 ) / T
             AKKP1 = WORK(K+1,1)  / T
             D = T*( AK*AKP1-ONE )
             WORK(K,INVD) = AKP1 / D
             WORK(K+1,INVD+1) = AK / D
             WORK(K,INVD+1) = -AKKP1 / D
             WORK(K+1,INVD) = -AKKP1 / D
            K=K+2
         END IF
        END DO
*
*       inv(U**T) = (inv(U))**T
*
*       inv(U**T)*inv(D)*inv(U)
*
        CUT=N
        DO WHILE (CUT .GT. 0)
```

```
          NNB=NB
          IF (CUT .LE. NNB) THEN
             NNB=CUT
          ELSE
             COUNT = 0
*            count negative elements,
             DO I=CUT+1-NNB,CUT
                 IF (IPIV(I) .LT. 0) COUNT=COUNT+1
             END DO
*            need a even number for a clear cut
             IF (MOD(COUNT,2) .EQ. 1) NNB=NNB+1
          END IF

          CUT=CUT-NNB
*
*         U01 Block
*
          DO I=1,CUT
            DO J=1,NNB
             WORK(I,J)=A(I,CUT+J)
            END DO
          END DO
*
*         U11 Block
*
          DO I=1,NNB
            WORK(U11+I,I)=ONE
            DO J=1,I-1
               WORK(U11+I,J)=ZERO
            END DO
            DO J=I+1,NNB
               WORK(U11+I,J)=A(CUT+I,CUT+J)
            END DO
          END DO
*
*         invD*U01
*
          I=1
          DO WHILE (I .LE. CUT)
            IF (IPIV(I) > 0) THEN
               DO J=1,NNB
                   WORK(I,J)=WORK(I,INVD)*WORK(I,J)
               END DO
               I=I+1
            ELSE
               DO J=1,NNB
                  U01_I_J = WORK(I,J)
                  U01_IP1_J = WORK(I+1,J)
                  WORK(I,J)=WORK(I,INVD)*U01_I_J+
     $                     WORK(I,INVD+1)*U01_IP1_J
```

```
                 WORK(I+1,J)=WORK(I+1,INVD)*U01_I_J+
     $                    WORK(I+1,INVD+1)*U01_IP1_J
              END DO
              I=I+2
           END IF
        END DO
*
*       invD1*U11
*
        I=1
        DO WHILE (I .LE. NNB)
           IF (IPIV(CUT+I) > 0) THEN
              DO J=I,NNB
                  WORK(U11+I,J)=WORK(CUT+I,INVD)*WORK(U11+I,J)
              END DO
              I=I+1
           ELSE
              DO J=I,NNB
                 U11_I_J = WORK(U11+I,J)
                 U11_IP1_J = WORK(U11+I+1,J)
              WORK(U11+I,J)=WORK(CUT+I,INVD)*WORK(U11+I,J) +
     $                    WORK(CUT+I,INVD+1)*WORK(U11+I+1,J)
              WORK(U11+I+1,J)=WORK(CUT+I+1,INVD)*U11_I_J+
     $                    WORK(CUT+I+1,INVD+1)*U11_IP1_J
              END DO
              I=I+2
           END IF
        END DO
*
*      U11**T*invD1*U11->U11
*
       CALL STRMM('L','U','T','U',NNB, NNB,
     $            ONE,A(CUT+1,CUT+1),LDA,WORK(U11+1,1),N+NB+1)
*
        DO I=1,NNB
           DO J=I,NNB
             A(CUT+I,CUT+J)=WORK(U11+I,J)
           END DO
        END DO
*
*         U01**T*invD*U01->A(CUT+I,CUT+J)
*
        CALL SGEMM('T','N',NNB,NNB,CUT,ONE,A(1,CUT+1),LDA,
     $              WORK,N+NB+1, ZERO, WORK(U11+1,1), N+NB+1)
*
*       U11 =  U11**T*invD1*U11 + U01**T*invD*U01
*
        DO I=1,NNB
           DO J=I,NNB
             A(CUT+I,CUT+J)=A(CUT+I,CUT+J)+WORK(U11+I,J)
```

```
            END DO
         END DO
*
*        U01 =  U00**T*invD0*U01
*
         CALL STRMM('L',UPLO,'T','U',CUT, NNB,
     $             ONE,A,LDA,WORK,N+NB+1)

*
*        Update U01
*
         DO I=1,CUT
           DO J=1,NNB
            A(I,CUT+J)=WORK(I,J)
           END DO
         END DO
*
*      Next Block
*
       END DO
*
*        Apply PERMUTATIONS P and P**T: P * inv(U**T)*inv(D)*inv(U) *P**T
*
            I=1
            DO WHILE ( I .LE. N )
               IF( IPIV(I) .GT. 0 ) THEN
                  IP=IPIV(I)
                 IF (I .LT. IP) CALL SSYSWAPR( UPLO, N, A, LDA, I ,IP )
                 IF (I .GT. IP) CALL SSYSWAPR( UPLO, N, A, LDA, IP ,I )
               ELSE
                 IP=-IPIV(I)
                 I=I+1
                 IF ( (I-1) .LT. IP)
     $                  CALL SSYSWAPR( UPLO, N, A, LDA, I-1 ,IP )
                 IF ( (I-1) .GT. IP)
     $                  CALL SSYSWAPR( UPLO, N, A, LDA, IP ,I-1 )
              ENDIF
               I=I+1
            END DO
      ELSE
*
*        LOWER...
*
*        invA = P * inv(U**T)*inv(D)*inv(U)*P**T.
*
         CALL STRTRI( UPLO, 'U', N, A, LDA, INFO )
*
*       inv(D) and inv(D)*inv(U)
*
        K=N
```

```
      DO WHILE ( K .GE. 1 )
       IF( IPIV( K ).GT.0 ) THEN
*         1 x 1 diagonal NNB
           WORK(K,INVD) = ONE /  A( K, K )
           WORK(K,INVD+1) = 0
          K=K-1
       ELSE
*         2 x 2 diagonal NNB
           T = WORK(K-1,1)
           AK = A( K-1, K-1 ) / T
           AKP1 = A( K, K ) / T
           AKKP1 = WORK(K-1,1) / T
           D = T*( AK*AKP1-ONE )
           WORK(K-1,INVD) = AKP1 / D
           WORK(K,INVD) = AK / D
           WORK(K,INVD+1) = -AKKP1 / D
           WORK(K-1,INVD+1) = -AKKP1 / D
          K=K-2
       END IF
      END DO
*
*     inv(U**T) = (inv(U))**T
*
*     inv(U**T)*inv(D)*inv(U)
*
      CUT=0
      DO WHILE (CUT .LT. N)
         NNB=NB
         IF (CUT + NNB .GT. N) THEN
            NNB=N-CUT
         ELSE
            COUNT = 0
*           count negative elements,
            DO I=CUT+1,CUT+NNB
                IF (IPIV(I) .LT. 0) COUNT=COUNT+1
            END DO
*           need a even number for a clear cut
            IF (MOD(COUNT,2) .EQ. 1) NNB=NNB+1
         END IF
*    L21 Block
         DO I=1,N-CUT-NNB
           DO J=1,NNB
            WORK(I,J)=A(CUT+NNB+I,CUT+J)
           END DO
         END DO
*    L11 Block
         DO I=1,NNB
           WORK(U11+I,I)=ONE
           DO J=I+1,NNB
              WORK(U11+I,J)=ZERO
```

```
            END DO
            DO J=1,I-1
               WORK(U11+I,J)=A(CUT+I,CUT+J)
            END DO
          END DO
*
*         invD*L21
*
          I=N-CUT-NNB
          DO WHILE (I .GE. 1)
            IF (IPIV(CUT+NNB+I) > 0) THEN
               DO J=1,NNB
                   WORK(I,J)=WORK(CUT+NNB+I,INVD)*WORK(I,J)
               END DO
               I=I-1
            ELSE
               DO J=1,NNB
                  U01_I_J = WORK(I,J)
                  U01_IP1_J = WORK(I-1,J)
                  WORK(I,J)=WORK(CUT+NNB+I,INVD)*U01_I_J+
     $                      WORK(CUT+NNB+I,INVD+1)*U01_IP1_J
                  WORK(I-1,J)=WORK(CUT+NNB+I-1,INVD+1)*U01_I_J+
     $                      WORK(CUT+NNB+I-1,INVD)*U01_IP1_J
               END DO
               I=I-2
            END IF
          END DO
*
*       invD1*L11
*
          I=NNB
          DO WHILE (I .GE. 1)
            IF (IPIV(CUT+I) > 0) THEN
               DO J=1,NNB
                   WORK(U11+I,J)=WORK(CUT+I,INVD)*WORK(U11+I,J)
               END DO
               I=I-1
            ELSE
               DO J=1,NNB
                  U11_I_J = WORK(U11+I,J)
                  U11_IP1_J = WORK(U11+I-1,J)
               WORK(U11+I,J)=WORK(CUT+I,INVD)*WORK(U11+I,J) +
     $                     WORK(CUT+I,INVD+1)*U11_IP1_J
               WORK(U11+I-1,J)=WORK(CUT+I-1,INVD+1)*U11_I_J+
     $                     WORK(CUT+I-1,INVD)*U11_IP1_J
               END DO
               I=I-2
            END IF
          END DO
*
```

```
*       L11**T*invD1*L11->L11
*
        CALL STRMM('L',UPLO,'T','U',NNB, NNB,
     $             ONE,A(CUT+1,CUT+1),LDA,WORK(U11+1,1),N+NB+1)

*
         DO I=1,NNB
            DO J=1,I
              A(CUT+I,CUT+J)=WORK(U11+I,J)
            END DO
         END DO
*
        IF ( (CUT+NNB) .LT. N ) THEN
*
*         L21**T*invD2*L21->A(CUT+I,CUT+J)
*
         CALL SGEMM('T','N',NNB,NNB,N-NNB-CUT,ONE,A(CUT+NNB+1,CUT+1)
     $             ,LDA,WORK,N+NB+1, ZERO, WORK(U11+1,1), N+NB+1)

*
*        L11 =  L11**T*invD1*L11 + U01**T*invD*U01
*
         DO I=1,NNB
            DO J=1,I
              A(CUT+I,CUT+J)=A(CUT+I,CUT+J)+WORK(U11+I,J)
            END DO
         END DO
*
*        L01 =  L22**T*invD2*L21
*
         CALL STRMM('L',UPLO,'T','U', N-NNB-CUT, NNB,
     $             ONE,A(CUT+NNB+1,CUT+NNB+1),LDA,WORK,N+NB+1)
*
*      Update L21
*
         DO I=1,N-CUT-NNB
           DO J=1,NNB
              A(CUT+NNB+I,CUT+J)=WORK(I,J)
           END DO
         END DO

       ELSE
*
*        L11 =  L11**T*invD1*L11
*
         DO I=1,NNB
            DO J=1,I
              A(CUT+I,CUT+J)=WORK(U11+I,J)
            END DO
         END DO
```

```
      END IF
*
*     Next Block
*
          CUT=CUT+NNB
      END DO
*
*       Apply PERMUTATIONS P and P**T: P * inv(U**T)*inv(D)*inv(U) *P**T
*
           I=N
           DO WHILE ( I .GE. 1 )
              IF( IPIV(I) .GT. 0 ) THEN
                 IP=IPIV(I)
                IF (I .LT. IP) CALL SSYSWAPR( UPLO, N, A, LDA, I ,IP  )
                IF (I .GT. IP) CALL SSYSWAPR( UPLO, N, A, LDA, IP ,I )
              ELSE
                IP=-IPIV(I)
                IF ( I .LT. IP) CALL SSYSWAPR( UPLO, N, A, LDA, I ,IP )
                IF ( I .GT. IP) CALL SSYSWAPR( UPLO, N, A, LDA, IP ,I )
                I=I-1
              ENDIF
              I=I-1
           END DO
     END IF
*
     RETURN
*
*    End of SSYTRI2X
*
     END
```

———-

59.23.4 lisp code

— **ssytri2x.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun ssytri2x (uplo n a lda ipiv work nb info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array single-float (*)) work a)
             (type (f2cl-lib:integer4) info nb lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
```

```
  ((uplo character uplo-%data% uplo-%offset%)
   (a single-float a-%data% a-%offset%)
   (work single-float work-%data% work-%offset%)
   (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
(prog ((u11_i_j 0.0) (u11_ip1_j 0.0) (u01_i_j 0.0) (u01_ip1_j 0.0)
       (ak 0.0) (akkp1 0.0) (akp1 0.0) (d 0.0) (t$ 0.0) (j 0) (u11 0)
       (invd 0) (count$ 0) (i 0) (iinfo 0) (ip 0) (k 0) (cut 0) (nnb 0)
       (upper nil))
  (declare
   (type (single-float) u11_i_j u11_ip1_j u01_i_j u01_ip1_j ak akkp1 akp1
    d t$)
   (type (f2cl-lib:integer4) j u11 invd count$ i iinfo ip k cut nnb)
   (type f2cl-lib:logical upper))
  (setf info 0)
  (setf upper
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
  (cond
   ((and (not upper)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4)))
  (cond
   ((/= info 0) (xerbla "SSYTRI2X" (f2cl-lib:int-sub info))
    (go end_label)))
  (if (= n 0)
      (go end_label))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (ssyconv uplo "C" n a lda ipiv work iinfo)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
    (setf uplo var-0)
    (setf iinfo var-7))
  (cond
   (upper
    (f2cl-lib:fdo (info n (f2cl-lib:int-add info (f2cl-lib:int-sub 1)))
                  ((> info 1) nil)
      (tagbody
        (if (and
             (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
                0)
```

```
            (=
             (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                            a-%offset%)
             zero))
           (go end_label))
      label100000)))
  (t
   (f2cl-lib:fdo (info 1 (f2cl-lib:int-add info 1))
                 ((> info n) nil)
     (tagbody
       (if (and
            (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
               0)
            (=
             (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                            a-%offset%)
             zero))
           (go end_label))
      label100001))))
 (setf info 0)
 (setf u11 n)
 (setf invd (f2cl-lib:int-add nb 2))
 (cond
  (upper
   (tagbody
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
         (strtri uplo "U" n a lda info)
       (declare (ignore var-1 var-3))
       (when var-0 (setf uplo var-0))
       (when var-2 (setf n var-2))
       (when var-4 (setf lda var-4))
       (when var-5 (setf info var-5)))
     (setf k 1)
    label100002
     (if (not (<= k n))
         (go label100003))
     (cond
      ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
       (setf (f2cl-lib:fref work-%data% (k invd)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%)
               (/ one
                  (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                 a-%offset%)))
       (setf (f2cl-lib:fref work-%data% (k (f2cl-lib:int-add invd 1))
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%)
               (coerce (the f2cl-lib:integer4 0) 'single-float))
       (setf k (f2cl-lib:int-add k 1)))
      (t
```

```
    (setf t$
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1) 1)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%))
    (setf ak
            (/
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)
             t$))
    (setf akp1
            (/
             (f2cl-lib:fref a-%data%
                            ((f2cl-lib:int-add k 1)
                             (f2cl-lib:int-add k 1))
                            ((1 lda) (1 *)) a-%offset%)
             t$))
    (setf akkp1
            (/
             (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1) 1)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%)
             t$))
    (setf d (* t$ (- (* ak akp1) one)))
    (setf (f2cl-lib:fref work-%data% (k invd)
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
            (/ akp1 d))
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add k 1)
                          (f2cl-lib:int-add invd 1))
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
            (/ ak d))
    (setf (f2cl-lib:fref work-%data% (k (f2cl-lib:int-add invd 1))
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
            (/ (- akkp1) d))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1) invd)
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
            (/ (- akkp1) d))
    (setf k (f2cl-lib:int-add k 2))))
  (go label100002)
label100003
  (setf cut n)
label100004
  (if (not (> cut 0))
      (go label100005))
  (setf nnb nb)
  (cond ((<= cut nnb) (setf nnb cut))
```

```
      (t (setf count$ 0)
       (f2cl-lib:fdo (i
                      (f2cl-lib:int-add cut 1
                                        (f2cl-lib:int-sub nnb))
                      (f2cl-lib:int-add i 1))
                     ((> i cut) nil)
         (tagbody
           (if (<
                (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                               ipiv-%offset%)
                0)
               (setf count$ (f2cl-lib:int-add count$ 1)))
          label100006))
       (if (= (mod count$ 2) 1)
           (setf nnb (f2cl-lib:int-add nnb 1)))))
(setf cut (f2cl-lib:int-sub cut nnb))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i cut) nil)
  (tagbody
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
              (f2cl-lib:fref a-%data%
                             (i (f2cl-lib:int-add cut j))
                             ((1 lda) (1 *)) a-%offset%))
       label100008))
   label100007))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nnb) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) i)
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
          one)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add u11 i) j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
              zero)
       label100010))
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
```

```
          (setf (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add u11 i) j)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)
                  (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add cut i)
                                  (f2cl-lib:int-add cut j))
                                 ((1 lda) (1 *)) a-%offset%))
        label100011))
    label100009))
 (setf i 1)
label100012
 (if (not (<= i cut))
     (go label100013))
 (cond
  ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j nnb) nil)
     (tagbody
       (setf (f2cl-lib:fref work-%data% (i j)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%)
               (*
                (f2cl-lib:fref work-%data% (i invd)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)
                (f2cl-lib:fref work-%data% (i j)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)))
      label100014))
   (setf i (f2cl-lib:int-add i 1)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j nnb) nil)
     (tagbody
       (setf u01_i_j
               (f2cl-lib:fref work-%data% (i j)
                              ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                              work-%offset%))
       (setf u01_ip1_j
               (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1) j)
                              ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                              work-%offset%))
       (setf (f2cl-lib:fref work-%data% (i j)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%)
               (+
                (*
                 (f2cl-lib:fref work-%data% (i invd)
                                ((1 (f2cl-lib:int-add n nb 1))
```

```
                                  (1 *))
                                 work-%offset%)
                   u01_i_j)
                  (*
                   (f2cl-lib:fref work-%data%
                                 (i (f2cl-lib:int-add invd 1))
                                 ((1 (f2cl-lib:int-add n nb 1))
                                  (1 *))
                                 work-%offset%)
                   u01_ip1_j)))
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1) j)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)
                 (+
                  (*
                   (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add i 1) invd)
                                 ((1 (f2cl-lib:int-add n nb 1))
                                  (1 *))
                                 work-%offset%)
                   u01_i_j)
                  (*
                   (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add i 1)
                                  (f2cl-lib:int-add invd 1))
                                 ((1 (f2cl-lib:int-add n nb 1))
                                  (1 *))
                                 work-%offset%)
                   u01_ip1_j)))
        label100015))
    (setf i (f2cl-lib:int-add i 2))))
  (go label100012)
 label100013
  (setf i 1)
 label100016
  (if (not (<= i nnb))
      (go label100017))
  (cond
   ((> (f2cl-lib:fref ipiv ((f2cl-lib:int-add cut i)) ((1 *))) 0)
    (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
                 (*
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add cut i) invd)
                                 ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                 work-%offset%)
```

```
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add u11 i) j)
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%)))
    label100018))
 (setf i (f2cl-lib:int-add i 1)))
(t
 (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
               ((> j nnb) nil)
   (tagbody
     (setf u11_i_j
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add u11 i) j)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%))
     (setf u11_ip1_j
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add u11 i 1) j)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%))
     (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) j)
                          ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                          work-%offset%)
             (+
              (*
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add cut i) invd)
                              ((1 (f2cl-lib:int-add n nb 1))
                               (1 *))
                              work-%offset%)
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add u11 i) j)
                              ((1 (f2cl-lib:int-add n nb 1))
                               (1 *))
                              work-%offset%))
              (*
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add cut i)
                               (f2cl-lib:int-add invd 1))
                              ((1 (f2cl-lib:int-add n nb 1))
                               (1 *))
                              work-%offset%)
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add u11 i 1) j)
                              ((1 (f2cl-lib:int-add n nb 1))
                               (1 *))
                              work-%offset%))))
     (setf (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add u11 i 1) j)
                          ((1 (f2cl-lib:int-add n nb 1)) (1 *))
```

```
                                work-%offset%)
                        (+
                         (*
                          (f2cl-lib:fref work-%data%
                                         ((f2cl-lib:int-add cut i 1) invd)
                                         ((1 (f2cl-lib:int-add n nb 1))
                                          (1 *))
                                         work-%offset%)
                          u11_i_j)
                         (*
                          (f2cl-lib:fref work-%data%
                                         ((f2cl-lib:int-add cut i 1)
                                          (f2cl-lib:int-add invd 1))
                                         ((1 (f2cl-lib:int-add n nb 1))
                                          (1 *))
                                         work-%offset%)
                          u11_ip1_j)))
              label100019))
        (setf i (f2cl-lib:int-add i 2))))
      (go label100016)
     label100017
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (strmm "L" "U" "T" "U" nnb nnb one
           (f2cl-lib:array-slice a-%data% single-float
                                 ((+ cut 1) (f2cl-lib:int-add cut 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice work-%data% single-float ((+ u11 1) 1)
                                 ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                 work-%offset%)
           (f2cl-lib:int-add n nb 1))
        (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9 var-10))
        (when var-4 (setf nnb var-4))
        (when var-5 (setf nnb var-5))
        (when var-6 (setf one var-6))
        (when var-8 (setf lda var-8)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i nnb) nil)
        (tagbody
          (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                        ((> j nnb) nil)
            (tagbody
              (setf (f2cl-lib:fref a-%data%
                                   ((f2cl-lib:int-add cut i)
                                    (f2cl-lib:int-add cut j))
                                   ((1 lda) (1 *)) a-%offset%)
                    (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-add u11 i) j)
```

```
                                  ((1 (f2cl-lib:int-add n nb 1))
                                   (1 *))
                                  work-%offset%))
        label100021))
    label100020))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (sgemm "T" "N" nnb nnb cut one
     (f2cl-lib:array-slice a-%data% single-float
                           (1 (f2cl-lib:int-add cut 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda work (f2cl-lib:int-add n nb 1) zero
     (f2cl-lib:array-slice work-%data% single-float ((+ u11 1) 1)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
     (f2cl-lib:int-add n nb 1))
  (declare (ignore var-0 var-1 var-6 var-8 var-9 var-11 var-12))
  (when var-2 (setf nnb var-2))
  (when var-3 (setf nnb var-3))
  (when var-4 (setf cut var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf zero var-10)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nnb) nil)
  (tagbody
    (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data%
                             ((f2cl-lib:int-add cut i)
                              (f2cl-lib:int-add cut j))
                             ((1 lda) (1 *)) a-%offset%)
                (+
                 (f2cl-lib:fref a-%data%
                                ((f2cl-lib:int-add cut i)
                                 (f2cl-lib:int-add cut j))
                                ((1 lda) (1 *)) a-%offset%)
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add u11 i) j)
                                ((1 (f2cl-lib:int-add n nb 1))
                                 (1 *))
                                work-%offset%)))
        label100023))
    label100022))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (strmm "L" uplo "T" "U" cut nnb one a lda work
```

```
        (f2cl-lib:int-add n nb 1))
      (declare (ignore var-0 var-2 var-3 var-7 var-9 var-10))
      (when var-1 (setf uplo var-1))
      (when var-4 (setf cut var-4))
      (when var-5 (setf nnb var-5))
      (when var-6 (setf one var-6))
      (when var-8 (setf lda var-8)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i cut) nil)
      (tagbody
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j nnb) nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add cut j))
                                  ((1 lda) (1 *)) a-%offset%)
                    (f2cl-lib:fref work-%data% (i j)
                                   ((1 (f2cl-lib:int-add n nb 1))
                                    (1 *))
                                   work-%offset%))
           label100025))
       label100024))
    (go label100004)
   label100005
    (setf i 1)
   label100026
    (if (not (<= i n))
        (go label100027))
    (cond
     ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
      (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
      (if (< i ip)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
              (ssyswapr uplo n a lda i ip)
            (declare (ignore var-2))
            (when var-0 (setf uplo var-0))
            (when var-1 (setf n var-1))
            (when var-3 (setf lda var-3))
            (when var-4 (setf i var-4))
            (when var-5 (setf ip var-5))))
      (if (> i ip)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
              (ssyswapr uplo n a lda ip i)
            (declare (ignore var-2))
            (when var-0 (setf uplo var-0))
            (when var-1 (setf n var-1))
            (when var-3 (setf lda var-3))
            (when var-4 (setf ip var-4))
            (when var-5 (setf i var-5)))))
     (t
      (setf ip
```

```
                (f2cl-lib:int-sub
                 (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)))
      (setf i (f2cl-lib:int-add i 1))
      (if (< (f2cl-lib:int-sub i 1) ip)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
              (ssyswapr uplo n a lda (f2cl-lib:int-sub i 1) ip)
            (declare (ignore var-2 var-4))
            (when var-0 (setf uplo var-0))
            (when var-1 (setf n var-1))
            (when var-3 (setf lda var-3))
            (when var-5 (setf ip var-5))))
      (if (> (f2cl-lib:int-sub i 1) ip)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
              (ssyswapr uplo n a lda ip (f2cl-lib:int-sub i 1))
            (declare (ignore var-2 var-5))
            (when var-0 (setf uplo var-0))
            (when var-1 (setf n var-1))
            (when var-3 (setf lda var-3))
            (when var-4 (setf ip var-4))))))
    (setf i (f2cl-lib:int-add i 1))
    (go label100026)
   label100027))
 (t
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (strtri uplo "U" n a lda info)
      (declare (ignore var-1 var-3))
      (when var-0 (setf uplo var-0))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-5 (setf info var-5)))
    (setf k n)
   label100028
    (if (not (>= k 1))
        (go label100029))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf (f2cl-lib:fref work-%data% (k invd)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
                (/ one
                   (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                  a-%offset%)))
      (setf (f2cl-lib:fref work-%data% (k (f2cl-lib:int-add invd 1))
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
                (coerce (the f2cl-lib:integer4 0) 'single-float))
      (setf k (f2cl-lib:int-sub k 1)))
     (t
      (setf t$
```

```
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1) 1)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%))
    (setf ak
            (/
             (f2cl-lib:fref a-%data%
                            ((f2cl-lib:int-sub k 1)
                             (f2cl-lib:int-sub k 1))
                            ((1 lda) (1 *)) a-%offset%)
             t$))
    (setf akp1
            (/
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)
             t$))
    (setf akkp1
            (/
             (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1) 1)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%)
             t$))
    (setf d (* t$ (- (* ak akp1) one)))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1) invd)
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
            (/ akp1 d))
    (setf (f2cl-lib:fref work-%data% (k invd)
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
            (/ ak d))
    (setf (f2cl-lib:fref work-%data% (k (f2cl-lib:int-add invd 1))
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
            (/ (- akkp1) d))
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-sub k 1)
                          (f2cl-lib:int-add invd 1))
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
            (/ (- akkp1) d))
    (setf k (f2cl-lib:int-sub k 2))))
  (go label100028)
 label100029
  (setf cut 0)
 label100030
  (if (not (< cut n))
      (go label100031))
  (setf nnb nb)
  (cond
   ((> (f2cl-lib:int-add cut nnb) n)
```

```
  (setf nnb (f2cl-lib:int-sub n cut)))
 (t (setf count$ 0)
  (f2cl-lib:fdo (i (f2cl-lib:int-add cut 1) (f2cl-lib:int-add i 1))
                ((> i (f2cl-lib:int-add cut nnb)) nil)
    (tagbody
      (if (< (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
             0)
          (setf count$ (f2cl-lib:int-add count$ 1)))
     label100032))
  (if (= (mod count$ 2) 1)
      (setf nnb (f2cl-lib:int-add nnb 1)))))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i
                  (f2cl-lib:int-add n (f2cl-lib:int-sub cut)
                                    (f2cl-lib:int-sub nnb)))
               nil)
  (tagbody
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
              (f2cl-lib:fref a-%data%
                             ((f2cl-lib:int-add cut nnb i)
                              (f2cl-lib:int-add cut j))
                             ((1 lda) (1 *)) a-%offset%))
       label100034))
   label100033))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nnb) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) i)
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
          one)
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add u11 i) j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
              zero)
       label100036))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data%
```

```
                                ((f2cl-lib:int-add u11 i) j)
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%)
                   (f2cl-lib:fref a-%data%
                                  ((f2cl-lib:int-add cut i)
                                   (f2cl-lib:int-add cut j))
                                  ((1 lda) (1 *)) a-%offset%))
         label100037))
     label100035))
  (setf i (f2cl-lib:int-sub n cut nnb))
 label100038
  (if (not (>= i 1))
      (go label100039))
  (cond
   ((> (f2cl-lib:fref ipiv ((f2cl-lib:int-add cut nnb i)) ((1 *))) 0)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
              (*
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add cut nnb i) invd)
                              ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                              work-%offset%)
               (f2cl-lib:fref work-%data% (i j)
                              ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                              work-%offset%)))
       label100040))
    (setf i (f2cl-lib:int-sub i 1)))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf u01_i_j
              (f2cl-lib:fref work-%data% (i j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%))
        (setf u01_ip1_j
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub i 1) j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%))
        (setf (f2cl-lib:fref work-%data% (i j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
              (+
               (*
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add cut nnb i) invd)
```

```
                                    ((1 (f2cl-lib:int-add n nb 1))
                                     (1 *))
                                    work-%offset%)
                     u01_i_j)
                    (*
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add cut nnb i)
                                     (f2cl-lib:int-add invd 1))
                                    ((1 (f2cl-lib:int-add n nb 1))
                                     (1 *))
                                    work-%offset%)
                     u01_ip1_j)))
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub i 1) j)
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%)
                  (+
                   (*
                    (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-sub
                                     (f2cl-lib:int-add cut nnb i) 1)
                                    (f2cl-lib:int-add invd 1))
                                   ((1 (f2cl-lib:int-add n nb 1))
                                    (1 *))
                                   work-%offset%)
                    u01_i_j)
                   (*
                    (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-sub
                                     (f2cl-lib:int-add cut nnb i) 1)
                                    invd)
                                   ((1 (f2cl-lib:int-add n nb 1))
                                    (1 *))
                                   work-%offset%)
                    u01_ip1_j)))
        label100041))
      (setf i (f2cl-lib:int-sub i 2))))
    (go label100038)
  label100039
    (setf i nnb)
  label100042
    (if (not (>= i 1))
        (go label100043))
    (cond
     ((> (f2cl-lib:fref ipiv ((f2cl-lib:int-add cut i)) ((1 *))) 0)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nnb) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) j)
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%)
```

```
                (*
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add cut i) invd)
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%)
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add u11 i) j)
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%)))
    label100044))
 (setf i (f2cl-lib:int-sub i 1)))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j nnb) nil)
   (tagbody
     (setf u11_i_j
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add u11 i) j)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%))
     (setf u11_ip1_j
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add u11 i) 1)
                             j)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%))
     (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) j)
                          ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                          work-%offset%)
             (+
              (*
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add cut i) invd)
                              ((1 (f2cl-lib:int-add n nb 1))
                               (1 *))
                              work-%offset%)
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add u11 i) j)
                              ((1 (f2cl-lib:int-add n nb 1))
                               (1 *))
                              work-%offset%))
              (*
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add cut i)
                               (f2cl-lib:int-add invd 1))
                              ((1 (f2cl-lib:int-add n nb 1))
                               (1 *))
                              work-%offset%)
               u11_ip1_j)))
```

```
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-sub
                               (f2cl-lib:int-add u11 i) 1)
                              j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
              (+
               (*
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add cut i) 1)
                                (f2cl-lib:int-add invd 1))
                               ((1 (f2cl-lib:int-add n nb 1))
                                (1 *))
                               work-%offset%)
                u11_i_j)
               (*
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add cut i) 1)
                                invd)
                               ((1 (f2cl-lib:int-add n nb 1))
                                (1 *))
                               work-%offset%)
                u11_ip1_j)))
      label100045))
    (setf i (f2cl-lib:int-sub i 2))))
  (go label100042)
 label100043
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (strmm "L" uplo "T" "U" nnb nnb one
       (f2cl-lib:array-slice a-%data% single-float
                             ((+ cut 1) (f2cl-lib:int-add cut 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice work-%data% single-float ((+ u11 1) 1)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
       (f2cl-lib:int-add n nb 1))
    (declare (ignore var-0 var-2 var-3 var-7 var-9 var-10))
    (when var-1 (setf uplo var-1))
    (when var-4 (setf nnb var-4))
    (when var-5 (setf nnb var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i nnb) nil)
    (tagbody
```

```
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j i) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data%
                               ((f2cl-lib:int-add cut i)
                                (f2cl-lib:int-add cut j))
                               ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add u11 i) j)
                               ((1 (f2cl-lib:int-add n nb 1))
                                (1 *))
                               work-%offset%))
         label100047))
    label100046))
  (cond
   ((< (f2cl-lib:int-add cut nnb) n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (sgemm "T" "N" nnb nnb (f2cl-lib:int-sub n nnb cut) one
         (f2cl-lib:array-slice a-%data% single-float
                               ((+ cut nnb 1)
                                (f2cl-lib:int-add cut 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda work (f2cl-lib:int-add n nb 1) zero
         (f2cl-lib:array-slice work-%data% single-float ((+ u11 1) 1)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)
         (f2cl-lib:int-add n nb 1))
      (declare
       (ignore var-0 var-1 var-4 var-6 var-8 var-9 var-11 var-12))
      (when var-2 (setf nnb var-2))
      (when var-3 (setf nnb var-3))
      (when var-5 (setf one var-5))
      (when var-7 (setf lda var-7))
      (when var-10 (setf zero var-10)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i nnb) nil)
      (tagbody
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j i) nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add cut i)
                                  (f2cl-lib:int-add cut j))
                                 ((1 lda) (1 *)) a-%offset%)
                  (+
                   (f2cl-lib:fref a-%data%
                                  ((f2cl-lib:int-add cut i)
                                   (f2cl-lib:int-add cut j))
```

```
                                        ((1 lda) (1 *)) a-%offset%)
                        (f2cl-lib:fref work-%data%
                                        ((f2cl-lib:int-add u11 i) j)
                                        ((1 (f2cl-lib:int-add n nb 1))
                                         (1 *))
                                        work-%offset%)))
          label100049))
      label100048))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (strmm "L" uplo "T" "U" (f2cl-lib:int-sub n nnb cut) nnb one
         (f2cl-lib:array-slice a-%data% single-float
                               ((+ cut nnb 1)
                                (f2cl-lib:int-add cut nnb 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda work (f2cl-lib:int-add n nb 1))
      (declare (ignore var-0 var-2 var-3 var-4 var-7 var-9 var-10))
      (when var-1 (setf uplo var-1))
      (when var-5 (setf nnb var-5))
      (when var-6 (setf one var-6))
      (when var-8 (setf lda var-8)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i
                      (f2cl-lib:int-add n (f2cl-lib:int-sub cut)
                                        (f2cl-lib:int-sub nnb)))
                   nil)
      (tagbody
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j nnb) nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add cut nnb i)
                                  (f2cl-lib:int-add cut j))
                                 ((1 lda) (1 *)) a-%offset%)
                  (f2cl-lib:fref work-%data% (i j)
                                 ((1 (f2cl-lib:int-add n nb 1))
                                  (1 *))
                                 work-%offset%))
          label100051))
      label100050)))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i nnb) nil)
     (tagbody
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j i) nil)
         (tagbody
           (setf (f2cl-lib:fref a-%data%
                                ((f2cl-lib:int-add cut i)
```

```
                                    (f2cl-lib:int-add cut j))
                                  ((1 lda) (1 *)) a-%offset%)
                      (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-add u11 i) j)
                                     ((1 (f2cl-lib:int-add n nb 1))
                                      (1 *))
                                     work-%offset%))
            label100053))
        label100052))))
  (setf cut (f2cl-lib:int-add cut nnb))
  (go label100030)
 label100031
  (setf i n)
 label100054
  (if (not (>= i 1))
      (go label100055))
  (cond
   ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
    (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
    (if (< i ip)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (ssyswapr uplo n a lda i ip)
          (declare (ignore var-2))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-4 (setf i var-4))
          (when var-5 (setf ip var-5))))
    (if (> i ip)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (ssyswapr uplo n a lda ip i)
          (declare (ignore var-2))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-4 (setf ip var-4))
          (when var-5 (setf i var-5)))))
   (t
    (setf ip
            (f2cl-lib:int-sub
             (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)))
    (if (< i ip)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (ssyswapr uplo n a lda i ip)
          (declare (ignore var-2))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-4 (setf i var-4))
          (when var-5 (setf ip var-5))))
```

```
              (if (> i ip)
                  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                      (ssyswapr uplo n a lda ip i)
                    (declare (ignore var-2))
                    (when var-0 (setf uplo var-0))
                    (when var-1 (setf n var-1))
                    (when var-3 (setf lda var-3))
                    (when var-4 (setf ip var-4))
                    (when var-5 (setf i var-5))))
              (setf i (f2cl-lib:int-sub i 1))))
            (setf i (f2cl-lib:int-sub i 1))
            (go label100054)
           label100055)))
        (go end_label)
       end_label
        (return (values uplo n nil lda nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssytri2x
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::ssyconv))))
```

59.24 ssytri_rook

59.24.1 Axiom unit tests

— **ssytrirook.input** —

```
)set break resume
)sys rm -f ssytrirook.output
)spool ssytrirook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.24.2 Axiom help page

— **ssytri_rook.help** —

```
====================================================================
ssytri_rook examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssytri_rook (UPLO, N, A, LDA, IPIV, WORK, INFO)
           SSYTRI_ROOK

Function/Subroutine Documentation
   subroutine ssytri_rook (character UPLO, integer N, real, dimension( lda, *
       ) A, integer LDA, integer, dimension( * ) IPIV, real, dimension( * )
       WORK, integer INFO)
       SSYTRI_ROOK

Purpose:
   SSYTRI_ROOK computes the inverse of a real symmetric
   matrix A using the factorization A = U*D*U**T or A = L*D*L**T
   computed by SSYTRF_ROOK.
```

```
Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the block diagonal matrix D and the multipliers
   used to obtain the factor U or L as computed by SSYTRF_ROOK.

   On exit, if INFO = 0, the (symmetric) inverse of the original
   matrix.  If UPLO = 'U', the upper triangular part of the
   inverse is formed and the part of A below the diagonal is not
   referenced; if UPLO = 'L' the lower triangular part of the
   inverse is formed and the part of A above the diagonal is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by SSYTRF_ROOK.

 WORK
   WORK is REAL array, dimension (N)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value
   > 0: if INFO = i, D(i,i) = 0; the matrix is singular and its
        inverse could not be computed.
```

59.24.3 fortran code

— ssytri_rook.f —

```
*  =====================================================================
      SUBROUTINE SSYTRI_ROOK( UPLO, N, A, LDA, IPIV, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            K, KP, KSTEP
      REAL               AK, AKKP1, AKP1, D, T, TEMP
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SDOT
      EXTERNAL           LSAME, SDOT
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SSWAP, SSYMV, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
```

```
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYTRI_ROOK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
         DO 10 INFO = N, 1, -1
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
   10    CONTINUE
      ELSE
*
*        Lower triangular storage: examine D from top to bottom.
*
         DO 20 INFO = 1, N
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
   20    CONTINUE
      END IF
      INFO = 0
*
      IF( UPPER ) THEN
*
*        Compute inv(A) from the factorization A = U*D*U**T.
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = 1
   30    CONTINUE
*
*        If K > N, exit from loop.
*
```

```
         IF( K.GT.N )
     $      GO TO 40
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Invert the diagonal block.
*
            A( K, K ) = ONE / A( K, K )
*
*           Compute column K of the inverse.
*
            IF( K.GT.1 ) THEN
               CALL SCOPY( K-1, A( 1, K ), 1, WORK, 1 )
               CALL SSYMV( UPLO, K-1, -ONE, A, LDA, WORK, 1, ZERO,
     $                     A( 1, K ), 1 )
               A( K, K ) = A( K, K ) - SDOT( K-1, WORK, 1, A( 1, K ),
     $                     1 )
            END IF
            KSTEP = 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Invert the diagonal block.
*
            T = ABS( A( K, K+1 ) )
            AK = A( K, K ) / T
            AKP1 = A( K+1, K+1 ) / T
            AKKP1 = A( K, K+1 ) / T
            D = T*( AK*AKP1-ONE )
            A( K, K ) = AKP1 / D
            A( K+1, K+1 ) = AK / D
            A( K, K+1 ) = -AKKP1 / D
*
*           Compute columns K and K+1 of the inverse.
*
            IF( K.GT.1 ) THEN
               CALL SCOPY( K-1, A( 1, K ), 1, WORK, 1 )
               CALL SSYMV( UPLO, K-1, -ONE, A, LDA, WORK, 1, ZERO,
     $                     A( 1, K ), 1 )
               A( K, K ) = A( K, K ) - SDOT( K-1, WORK, 1, A( 1, K ),
     $                     1 )
               A( K, K+1 ) = A( K, K+1 ) -
     $                       SDOT( K-1, A( 1, K ), 1, A( 1, K+1 ), 1 )
               CALL SCOPY( K-1, A( 1, K+1 ), 1, WORK, 1 )
               CALL SSYMV( UPLO, K-1, -ONE, A, LDA, WORK, 1, ZERO,
     $                     A( 1, K+1 ), 1 )
               A( K+1, K+1 ) = A( K+1, K+1 ) -
```

```
     $                        SDOT( K-1, WORK, 1, A( 1, K+1 ), 1 )
            END IF
            KSTEP = 2
         END IF
*
         IF( KSTEP.EQ.1 ) THEN
*
*           Interchange rows and columns K and IPIV(K) in the leading
*           submatrix A(1:k+1,1:k+1)
*
            KP = IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.GT.1 )
     $             CALL SSWAP( KP-1, A( 1, K ), 1, A( 1, KP ), 1 )
               CALL SSWAP( K-KP-1, A( KP+1, K ), 1, A( KP, KP+1 ), LDA )
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
            END IF
         ELSE
*
*           Interchange rows and columns K and K+1 with -IPIV(K) and
*           -IPIV(K+1)in the leading submatrix A(1:k+1,1:k+1)
*
            KP = -IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.GT.1 )
     $            CALL SSWAP( KP-1, A( 1, K ), 1, A( 1, KP ), 1 )
               CALL SSWAP( K-KP-1, A( KP+1, K ), 1, A( KP, KP+1 ), LDA )
*
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
               TEMP = A( K, K+1 )
               A( K, K+1 ) = A( KP, K+1 )
               A( KP, K+1 ) = TEMP
            END IF
*
            K = K + 1
            KP = -IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.GT.1 )
     $            CALL SSWAP( KP-1, A( 1, K ), 1, A( 1, KP ), 1 )
               CALL SSWAP( K-KP-1, A( KP+1, K ), 1, A( KP, KP+1 ), LDA )
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
            END IF
         END IF
*
```

```
         K = K + 1
         GO TO 30
   40    CONTINUE
*
      ELSE
*
*        Compute inv(A) from the factorization A = L*D*L**T.
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = N
   50    CONTINUE
*
*        If K < 1, exit from loop.
*
         IF( K.LT.1 )
     $      GO TO 60
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Invert the diagonal block.
*
            A( K, K ) = ONE / A( K, K )
*
*           Compute column K of the inverse.
*
            IF( K.LT.N ) THEN
               CALL SCOPY( N-K, A( K+1, K ), 1, WORK, 1 )
               CALL SSYMV( UPLO, N-K, -ONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                     ZERO, A( K+1, K ), 1 )
               A( K, K ) = A( K, K ) - SDOT( N-K, WORK, 1, A( K+1, K ),
     $                     1 )
            END IF
            KSTEP = 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Invert the diagonal block.
*
            T = ABS( A( K, K-1 ) )
            AK = A( K-1, K-1 ) / T
            AKP1 = A( K, K ) / T
            AKKP1 = A( K, K-1 ) / T
            D = T*( AK*AKP1-ONE )
            A( K-1, K-1 ) = AKP1 / D
            A( K, K ) = AK / D
```

```
            A( K, K-1 ) = -AKKP1 / D
*
*           Compute columns K-1 and K of the inverse.
*
            IF( K.LT.N ) THEN
               CALL SCOPY( N-K, A( K+1, K ), 1, WORK, 1 )
               CALL SSYMV( UPLO, N-K, -ONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                     ZERO, A( K+1, K ), 1 )
               A( K, K ) = A( K, K ) - SDOT( N-K, WORK, 1, A( K+1, K ),
     $                     1 )
               A( K, K-1 ) = A( K, K-1 ) -
     $                       SDOT( N-K, A( K+1, K ), 1, A( K+1, K-1 ),
     $                       1 )
               CALL SCOPY( N-K, A( K+1, K-1 ), 1, WORK, 1 )
               CALL SSYMV( UPLO, N-K, -ONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                     ZERO, A( K+1, K-1 ), 1 )
               A( K-1, K-1 ) = A( K-1, K-1 ) -
     $                         SDOT( N-K, WORK, 1, A( K+1, K-1 ), 1 )
            END IF
            KSTEP = 2
         END IF
*
         IF( KSTEP.EQ.1 ) THEN
*
*           Interchange rows and columns K and IPIV(K) in the trailing
*           submatrix A(k-1:n,k-1:n)
*
            KP = IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.LT.N )
     $            CALL SSWAP( N-KP, A( KP+1, K ), 1, A( KP+1, KP ), 1 )
               CALL SSWAP( KP-K-1, A( K+1, K ), 1, A( KP, K+1 ), LDA )
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
            END IF
         ELSE
*
*           Interchange rows and columns K and K-1 with -IPIV(K) and
*           -IPIV(K-1) in the trailing submatrix A(k-1:n,k-1:n)
*
            KP = -IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.LT.N )
     $            CALL SSWAP( N-KP, A( KP+1, K ), 1, A( KP+1, KP ), 1 )
               CALL SSWAP( KP-K-1, A( K+1, K ), 1, A( KP, K+1 ), LDA )
*
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
```

```
               TEMP = A( K, K-1 )
               A( K, K-1 ) = A( KP, K-1 )
               A( KP, K-1 ) = TEMP
            END IF
*
            K = K - 1
            KP = -IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.LT.N )
     $            CALL SSWAP( N-KP, A( KP+1, K ), 1, A( KP+1, KP ), 1 )
               CALL SSWAP( KP-K-1, A( K+1, K ), 1, A( KP, K+1 ), LDA )
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
            END IF
         END IF
*
         K = K - 1
         GO TO 50
   60    CONTINUE
      END IF
*
      RETURN
*
*     End of SSYTRI_ROOK
*
      END
```

———-

59.24.4 lisp code

— ssytri_rook.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun ssytri_rook (uplo n a lda ipiv work info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array single-float (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (work single-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
```

```
(prog ((ak 0.0) (akkp1 0.0) (akp1 0.0) (d 0.0) (t$ 0.0) (temp 0.0) (k 0)
       (kp 0) (kstep 0) (upper nil))
  (declare (type (single-float) ak akkp1 akp1 d t$ temp)
           (type (f2cl-lib:integer4) k kp kstep)
           (type f2cl-lib:logical upper))
  (setf info 0)
  (setf upper
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
  (cond
   ((and (not upper)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4)))
  (cond
   ((/= info 0) (xerbla "SSYTRI_ROOK" (f2cl-lib:int-sub info))
    (go end_label)))
  (if (= n 0)
      (go end_label))
  (cond
   (upper
    (f2cl-lib:fdo (info n (f2cl-lib:int-add info (f2cl-lib:int-sub 1)))
                  ((> info 1) nil)
      (tagbody
        (if (and
             (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
                0)
             (=
              (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                             a-%offset%)
              zero))
            (go end_label))
      label10)))
   (t
    (f2cl-lib:fdo (info 1 (f2cl-lib:int-add info 1))
                  ((> info n) nil)
      (tagbody
        (if (and
             (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
                0)
```

```
            (=
             (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                            a-%offset%)
             zero))
           (go end_label))
     label20))))
(setf info 0)
(cond
 (upper
  (tagbody
    (setf k 1)
   label30
    (if (> k n)
        (go label40))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
              (/ one
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%)))
      (cond
       ((> k 1)
        (scopy (f2cl-lib:int-sub k 1)
         (f2cl-lib:array-slice a-%data% single-float (1 k)
                               ((1 lda) (1 *)) a-%offset%)
         1 work 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9)
            (ssymv uplo (f2cl-lib:int-sub k 1) (- one) a lda work 1
             zero
             (f2cl-lib:array-slice a-%data% single-float (1 k)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
          (when var-0 (setf uplo var-0))
          (when var-4 (setf lda var-4))
          (when var-7 (setf zero var-7)))
        (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                (-
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%)
                 (sdot (f2cl-lib:int-sub k 1) work 1
                  (f2cl-lib:array-slice a-%data% single-float (1 k)
                                        ((1 lda) (1 *)) a-%offset%)
                  1)))))
      (setf kstep 1))
     (t
      (setf t$
              (abs
```

```
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)))
(setf ak
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         t$))
(setf akp1
        (/
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-add k 1)
                         (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
(setf akkp1
        (/
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
(setf d (* t$ (- (* ak akp1) one)))
(setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (/ akp1 d))
(setf (f2cl-lib:fref a-%data%
                     ((f2cl-lib:int-add k 1)
                      (f2cl-lib:int-add k 1))
                     ((1 lda) (1 *)) a-%offset%)
        (/ ak d))
(setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                     ((1 lda) (1 *)) a-%offset%)
        (/ (- akkp1) d))
(cond
 ((> k 1)
  (scopy (f2cl-lib:int-sub k 1)
   (f2cl-lib:array-slice a-%data% single-float (1 k)
                         ((1 lda) (1 *)) a-%offset%)
   1 work 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (ssymv uplo (f2cl-lib:int-sub k 1) (- one) a lda work 1
       zero
       (f2cl-lib:array-slice a-%data% single-float (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
    (when var-0 (setf uplo var-0))
    (when var-4 (setf lda var-4))
    (when var-7 (setf zero var-7)))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
          (-
```

```
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%)
          (sdot (f2cl-lib:int-sub k 1) work 1
           (f2cl-lib:array-slice a-%data% single-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1)))
(setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (-
          (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                         ((1 lda) (1 *)) a-%offset%)
          (sdot (f2cl-lib:int-sub k 1)
           (f2cl-lib:array-slice a-%data% single-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% single-float
                                 (1 (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           1)))
(scopy (f2cl-lib:int-sub k 1)
 (f2cl-lib:array-slice a-%data% single-float
                       (1 (f2cl-lib:int-add k 1))
                       ((1 lda) (1 *)) a-%offset%)
 1 work 1)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (ssymv uplo (f2cl-lib:int-sub k 1) (- one) a lda work 1
     zero
     (f2cl-lib:array-slice a-%data% single-float
                           (1 (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
  (when var-0 (setf uplo var-0))
  (when var-4 (setf lda var-4))
  (when var-7 (setf zero var-7)))
(setf (f2cl-lib:fref a-%data%
                      ((f2cl-lib:int-add k 1)
                       (f2cl-lib:int-add k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (-
          (f2cl-lib:fref a-%data%
                         ((f2cl-lib:int-add k 1)
                          (f2cl-lib:int-add k 1))
                         ((1 lda) (1 *)) a-%offset%)
          (sdot (f2cl-lib:int-sub k 1) work 1
           (f2cl-lib:array-slice a-%data% single-float
                                 (1 (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
```

```
                1)))))
    (setf kstep 2)))
  (cond
   ((= kstep 1)
    (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
    (cond
     ((/= kp k)
      (if (> kp 1)
          (sswap (f2cl-lib:int-sub kp 1)
           (f2cl-lib:array-slice a-%data% single-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% single-float (1 kp)
                                 ((1 lda) (1 *)) a-%offset%)
           1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (sswap (f2cl-lib:int-sub k kp 1)
           (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% single-float
                                 (kp (f2cl-lib:int-add kp 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf lda var-4)))
      (setf temp
              (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                             a-%offset%))
      (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                             a-%offset%))
      (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                           a-%offset%)
              temp))))
   (t
    (setf kp
            (f2cl-lib:int-sub
             (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
    (cond
     ((/= kp k)
      (if (> kp 1)
          (sswap (f2cl-lib:int-sub kp 1)
           (f2cl-lib:array-slice a-%data% single-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% single-float (1 kp)
                                 ((1 lda) (1 *)) a-%offset%)
           1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
        (sswap (f2cl-lib:int-sub k kp 1)
         (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) k)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% single-float
                               (kp (f2cl-lib:int-add kp 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-0 var-1 var-2 var-3))
      (when var-4 (setf lda var-4)))
    (setf temp
            (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%)
            temp)
    (setf temp
            (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                         ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-add k 1))
                         ((1 lda) (1 *)) a-%offset%)
            temp)))
  (setf k (f2cl-lib:int-add k 1))
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
  (cond
   ((/= kp k)
    (if (> kp 1)
        (sswap (f2cl-lib:int-sub kp 1)
         (f2cl-lib:array-slice a-%data% single-float (1 k)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% single-float (1 kp)
                               ((1 lda) (1 *)) a-%offset%)
         1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (sswap (f2cl-lib:int-sub k kp 1)
         (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) k)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% single-float
                               (kp (f2cl-lib:int-add kp 1))
```

```
                                    ((1 lda) (1 *)) a-%offset%)
                lda)
              (declare (ignore var-0 var-1 var-2 var-3))
              (when var-4 (setf lda var-4)))
            (setf temp
                    (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                   a-%offset%))
            (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                    (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                                   a-%offset%))
            (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                                 a-%offset%)
                    temp)))))
      (setf k (f2cl-lib:int-add k 1))
      (go label30)
     label40))
   (t
    (tagbody
      (setf k n)
     label50
      (if (< k 1)
          (go label60))
      (cond
       ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
        (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                (/ one
                   (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                  a-%offset%)))
        (cond
         ((< k n)
          (scopy (f2cl-lib:int-sub n k)
           (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                                 ((1 lda) (1 *)) a-%offset%)
           1 work 1)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9)
              (ssymv uplo (f2cl-lib:int-sub n k) (- one)
               (f2cl-lib:array-slice a-%data% single-float
                                     ((+ k 1) (f2cl-lib:int-add k 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda work 1 zero
               (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                                     ((1 lda) (1 *)) a-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
            (when var-0 (setf uplo var-0))
            (when var-4 (setf lda var-4))
            (when var-7 (setf zero var-7)))
          (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
```

```
          (-
           (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                          a-%offset%)
           (sdot (f2cl-lib:int-sub n k) work 1
            (f2cl-lib:array-slice a-%data% single-float
                                  ((+ k 1) k) ((1 lda) (1 *))
                                  a-%offset%)
            1)))))
 (setf kstep 1))
(t
 (setf t$
        (abs
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)))
 (setf ak
        (/
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-sub k 1)
                         (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
 (setf akp1
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         t$))
 (setf akkp1
        (/
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
 (setf d (* t$ (- (* ak akp1) one)))
 (setf (f2cl-lib:fref a-%data%
                      ((f2cl-lib:int-sub k 1)
                       (f2cl-lib:int-sub k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (/ akp1 d))
 (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (/ ak d))
 (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (/ (- akkp1) d))
 (cond
  ((< k n)
   (scopy (f2cl-lib:int-sub n k)
    (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                          ((1 lda) (1 *)) a-%offset%)
    1 work 1)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
     var-9)
    (ssymv uplo (f2cl-lib:int-sub n k) (- one)
     (f2cl-lib:array-slice a-%data% single-float
                           ((+ k 1) (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda work 1 zero
     (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                           ((1 lda) (1 *)) a-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
  (when var-0 (setf uplo var-0))
  (when var-4 (setf lda var-4))
  (when var-7 (setf zero var-7)))
(setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         (sdot (f2cl-lib:int-sub n k) work 1
          (f2cl-lib:array-slice a-%data% single-float
                                ((+ k 1) k) ((1 lda) (1 *))
                                a-%offset%)
          1)))
(setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                     ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         (sdot (f2cl-lib:int-sub n k)
          (f2cl-lib:array-slice a-%data% single-float
                                ((+ k 1) k) ((1 lda) (1 *))
                                a-%offset%)
          1
          (f2cl-lib:array-slice a-%data% single-float
                                ((+ k 1)
                                 (f2cl-lib:int-sub k 1))
                                ((1 lda) (1 *)) a-%offset%)
          1)))
(scopy (f2cl-lib:int-sub n k)
 (f2cl-lib:array-slice a-%data% single-float
                       ((+ k 1) (f2cl-lib:int-sub k 1))
                       ((1 lda) (1 *)) a-%offset%)
 1 work 1)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (ssymv uplo (f2cl-lib:int-sub n k) (- one)
     (f2cl-lib:array-slice a-%data% single-float
                           ((+ k 1) (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda work 1 zero
```

```
            (f2cl-lib:array-slice a-%data% single-float
                                  ((+ k 1) (f2cl-lib:int-sub k 1))
                                  ((1 lda) (1 *)) a-%offset%)
            1)
        (declare (ignore var-1 var-2 var-3 var-5 var-6 var-8 var-9))
        (when var-0 (setf uplo var-0))
        (when var-4 (setf lda var-4))
        (when var-7 (setf zero var-7)))
      (setf (f2cl-lib:fref a-%data%
                           ((f2cl-lib:int-sub k 1)
                            (f2cl-lib:int-sub k 1))
                           ((1 lda) (1 *)) a-%offset%)
              (-
               (f2cl-lib:fref a-%data%
                              ((f2cl-lib:int-sub k 1)
                               (f2cl-lib:int-sub k 1))
                              ((1 lda) (1 *)) a-%offset%)
               (sdot (f2cl-lib:int-sub n k) work 1
                (f2cl-lib:array-slice a-%data% single-float
                                      ((+ k 1)
                                       (f2cl-lib:int-sub k 1))
                                      ((1 lda) (1 *)) a-%offset%)
                1)))))
    (setf kstep 2)))
  (cond
   ((= kstep 1)
    (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
    (cond
     ((/= kp k)
      (if (< kp n)
          (sswap (f2cl-lib:int-sub n kp)
           (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) kp)
                                 ((1 lda) (1 *)) a-%offset%)
           1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (sswap (f2cl-lib:int-sub kp k 1)
           (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% single-float
                                 (kp (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf lda var-4)))
      (setf temp
              (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
```

```
                              a-%offset%))
      (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                              a-%offset%))
      (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                            a-%offset%)
              temp))))
  (t
   (setf kp
           (f2cl-lib:int-sub
            (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
   (cond
    ((/= kp k)
     (if (< kp n)
         (sswap (f2cl-lib:int-sub n kp)
          (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) k)
                                ((1 lda) (1 *)) a-%offset%)
          1
          (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) kp)
                                ((1 lda) (1 *)) a-%offset%)
          1))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (sswap (f2cl-lib:int-sub kp k 1)
          (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                                ((1 lda) (1 *)) a-%offset%)
          1
          (f2cl-lib:array-slice a-%data% single-float
                                (kp (f2cl-lib:int-add k 1))
                                ((1 lda) (1 *)) a-%offset%)
          lda)
       (declare (ignore var-0 var-1 var-2 var-3))
       (when var-4 (setf lda var-4)))
     (setf temp
              (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                              a-%offset%))
     (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                              a-%offset%))
     (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                            a-%offset%)
              temp)
     (setf temp
              (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                              ((1 lda) (1 *)) a-%offset%))
     (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                            ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-sub k 1))
                              ((1 lda) (1 *)) a-%offset%))
     (setf (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-sub k 1))
                            ((1 lda) (1 *)) a-%offset%)
```

```
                  temp)))
            (setf k (f2cl-lib:int-sub k 1))
            (setf kp
                    (f2cl-lib:int-sub
                     (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
            (cond
             ((/= kp k)
              (if (< kp n)
                  (sswap (f2cl-lib:int-sub n kp)
                   (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) k)
                                         ((1 lda) (1 *)) a-%offset%)
                   1
                   (f2cl-lib:array-slice a-%data% single-float ((+ kp 1) kp)
                                         ((1 lda) (1 *)) a-%offset%)
                   1))
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (sswap (f2cl-lib:int-sub kp k 1)
                   (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                                         ((1 lda) (1 *)) a-%offset%)
                   1
                   (f2cl-lib:array-slice a-%data% single-float
                                         (kp (f2cl-lib:int-add k 1))
                                         ((1 lda) (1 *)) a-%offset%)
                   lda)
                (declare (ignore var-0 var-1 var-2 var-3))
                (when var-4 (setf lda var-4)))
              (setf temp
                      (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                      (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                                   a-%offset%)
                      temp)))))
           (setf k (f2cl-lib:int-sub k 1))
           (go label50)
          label60)))
       (go end_label)
      end_label
       (return (values uplo nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssytri_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
```

```
                                        (array single-float (*))
                                        (fortran-to-lisp::integer4)
                                        (array fortran-to-lisp::integer4
                                         (*))
                                        (array single-float (*))
                                        (fortran-to-lisp::integer4))
                                      :return-values
                                      '(fortran-to-lisp::uplo nil nil
                                        fortran-to-lisp::lda nil nil
                                        fortran-to-lisp::info)
                                      :calls 'nil)))
```

———-

59.25 ssytrs

59.25.1 Axiom unit tests

— **ssytrs.input** —

```
)set break resume
)sys rm -f ssytrs.output
)spool ssytrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.25.2 Axiom help page

— **ssytrs.help** —

```
=====================================================================
ssytrs examples
=====================================================================

=====================================================================
Man Page Details
```

```
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssytrs (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO)
           SSYTRS

Function/Subroutine Documentation
   subroutine ssytrs (character UPLO, integer N, integer NRHS, real,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV, real,
       dimension( ldb, * ) B, integer LDB, integer INFO)
       SSYTRS

Purpose:
   SSYTRS solves a system of linear equations A*X = B with a real
   symmetric matrix A using the factorization A = U*D*U**T or
   A = L*D*L**T computed by SSYTRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is REAL array, dimension (LDA,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by SSYTRF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by SSYTRF.
```

```
B
  B is REAL array, dimension (LDB,NRHS)
  On entry, the right hand side matrix B.
  On exit, the solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

59.25.3 fortran code

— **ssytrs.f** —

```
*  =====================================================================
      SUBROUTINE SSYTRS( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            J, K, KP
      REAL               AK, AKM1, AKM1K, BK, BKM1, DENOM
```

```
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEMV, SGER, SSCAL, SSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYTRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*        Solve A*X = B, where A = U*D*U**T.
*
*        First solve U*D*X = B, overwriting B with X.
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop.
*
```

```
         IF( K.LT.1 )
     $      GO TO 30
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
*           Multiply by inv(U(K)), where U(K) is the transformation
*           stored in column K of A.
*
            CALL SGER( K-1, NRHS, -ONE, A( 1, K ), 1, B( K, 1 ), LDB,
     $                 B( 1, 1 ), LDB )
*
*           Multiply by the inverse of the diagonal block.
*
            CALL SSCAL( NRHS, ONE / A( K, K ), B( K, 1 ), LDB )
            K = K - 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Interchange rows K-1 and -IPIV(K).
*
            KP = -IPIV( K )
            IF( KP.NE.K-1 )
     $         CALL SSWAP( NRHS, B( K-1, 1 ), LDB, B( KP, 1 ), LDB )
*
*           Multiply by inv(U(K)), where U(K) is the transformation
*           stored in columns K-1 and K of A.
*
            CALL SGER( K-2, NRHS, -ONE, A( 1, K ), 1, B( K, 1 ), LDB,
     $                 B( 1, 1 ), LDB )
            CALL SGER( K-2, NRHS, -ONE, A( 1, K-1 ), 1, B( K-1, 1 ),
     $                 LDB, B( 1, 1 ), LDB )
*
*           Multiply by the inverse of the diagonal block.
*
            AKM1K = A( K-1, K )
            AKM1 = A( K-1, K-1 ) / AKM1K
            AK = A( K, K ) / AKM1K
            DENOM = AKM1*AK - ONE
            DO 20 J = 1, NRHS
               BKM1 = B( K-1, J ) / AKM1K
               BK = B( K, J ) / AKM1K
```

```
               B( K-1, J ) = ( AK*BKM1-BK ) / DENOM
               B( K, J ) = ( AKM1*BK-BKM1 ) / DENOM
   20       CONTINUE
            K = K - 2
         END IF
*
         GO TO 10
   30    CONTINUE
*
*        Next solve U**T *X = B, overwriting B with X.
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = 1
   40    CONTINUE
*
*        If K > N, exit from loop.
*
         IF( K.GT.N )
     $      GO TO 50
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Multiply by inv(U**T(K)), where U(K) is the transformation
*           stored in column K of A.
*
            CALL SGEMV( 'Transpose', K-1, NRHS, -ONE, B, LDB, A( 1, K ),
     $                  1, ONE, B( K, 1 ), LDB )
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K = K + 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Multiply by inv(U**T(K+1)), where U(K+1) is the transformation
*           stored in columns K and K+1 of A.
*
            CALL SGEMV( 'Transpose', K-1, NRHS, -ONE, B, LDB, A( 1, K ),
     $                  1, ONE, B( K, 1 ), LDB )
            CALL SGEMV( 'Transpose', K-1, NRHS, -ONE, B, LDB,
     $                  A( 1, K+1 ), 1, ONE, B( K+1, 1 ), LDB )
*
```

```
*          Interchange rows K and -IPIV(K).
*
           KP = -IPIV( K )
           IF( KP.NE.K )
     $        CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
           K = K + 2
        END IF
*
        GO TO 40
   50   CONTINUE
*
     ELSE
*
*       Solve A*X = B, where A = L*D*L**T.
*
*       First solve L*D*X = B, overwriting B with X.
*
*       K is the main loop index, increasing from 1 to N in steps of
*       1 or 2, depending on the size of the diagonal blocks.
*
        K = 1
   60   CONTINUE
*
*       If K > N, exit from loop.
*
        IF( K.GT.N )
     $     GO TO 80
*
        IF( IPIV( K ).GT.0 ) THEN
*
*          1 x 1 diagonal block
*
*          Interchange rows K and IPIV(K).
*
           KP = IPIV( K )
           IF( KP.NE.K )
     $        CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
*          Multiply by inv(L(K)), where L(K) is the transformation
*          stored in column K of A.
*
           IF( K.LT.N )
     $        CALL SGER( N-K, NRHS, -ONE, A( K+1, K ), 1, B( K, 1 ),
     $                   LDB, B( K+1, 1 ), LDB )
*
*          Multiply by the inverse of the diagonal block.
*
           CALL SSCAL( NRHS, ONE / A( K, K ), B( K, 1 ), LDB )
           K = K + 1
        ELSE
```

```
*
*          2 x 2 diagonal block
*
*          Interchange rows K+1 and -IPIV(K).
*
           KP = -IPIV( K )
           IF( KP.NE.K+1 )
     $        CALL SSWAP( NRHS, B( K+1, 1 ), LDB, B( KP, 1 ), LDB )
*
*          Multiply by inv(L(K)), where L(K) is the transformation
*          stored in columns K and K+1 of A.
*
           IF( K.LT.N-1 ) THEN
              CALL SGER( N-K-1, NRHS, -ONE, A( K+2, K ), 1, B( K, 1 ),
     $                   LDB, B( K+2, 1 ), LDB )
              CALL SGER( N-K-1, NRHS, -ONE, A( K+2, K+1 ), 1,
     $                   B( K+1, 1 ), LDB, B( K+2, 1 ), LDB )
           END IF
*
*          Multiply by the inverse of the diagonal block.
*
           AKM1K = A( K+1, K )
           AKM1 = A( K, K ) / AKM1K
           AK = A( K+1, K+1 ) / AKM1K
           DENOM = AKM1*AK - ONE
           DO 70 J = 1, NRHS
              BKM1 = B( K, J ) / AKM1K
              BK = B( K+1, J ) / AKM1K
              B( K, J ) = ( AK*BKM1-BK ) / DENOM
              B( K+1, J ) = ( AKM1*BK-BKM1 ) / DENOM
   70      CONTINUE
           K = K + 2
        END IF
*
        GO TO 60
   80   CONTINUE
*
*       Next solve L**T *X = B, overwriting B with X.
*
*       K is the main loop index, decreasing from N to 1 in steps of
*       1 or 2, depending on the size of the diagonal blocks.
*
        K = N
   90   CONTINUE
*
*       If K < 1, exit from loop.
*
        IF( K.LT.1 )
     $     GO TO 100
*
```

```
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Multiply by inv(L**T(K)), where L(K) is the transformation
*           stored in column K of A.
*
            IF( K.LT.N )
     $         CALL SGEMV( 'Transpose', N-K, NRHS, -ONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K ), 1, ONE, B( K, 1 ), LDB )
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K = K - 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Multiply by inv(L**T(K-1)), where L(K-1) is the transformation
*           stored in columns K-1 and K of A.
*
            IF( K.LT.N ) THEN
               CALL SGEMV( 'Transpose', N-K, NRHS, -ONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K ), 1, ONE, B( K, 1 ), LDB )
               CALL SGEMV( 'Transpose', N-K, NRHS, -ONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K-1 ), 1, ONE, B( K-1, 1 ),
     $                     LDB )
            END IF
*
*           Interchange rows K and -IPIV(K).
*
            KP = -IPIV( K )
            IF( KP.NE.K )
     $         CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K = K - 2
         END IF
*
         GO TO 90
  100    CONTINUE
      END IF
*
      RETURN
*
*     End of SSYTRS
*
      END
```

———-

59.25.4 lisp code

— **ssytrs.lisp** —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun ssytrs (uplo n nrhs a lda ipiv b ldb$ info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array single-float (*)) b a)
             (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((ak 0.0) (akm1 0.0) (akm1k 0.0) (bk 0.0) (bkm1 0.0) (denom 0.0)
             (j 0) (k 0) (kp 0) (upper nil))
        (declare (type (single-float) ak akm1 akm1k bk bkm1 denom)
                 (type (f2cl-lib:integer4) j k kp)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -5))
         ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -8)))
        (cond
         ((/= info 0) (xerbla "SSYTRS" (f2cl-lib:int-sub info))
          (go end_label)))
```

```
(if (or (= n 0) (= nrhs 0))
    (go end_label))
(cond
 (upper
  (tagbody
    (setf k n)
   label10
    (if (< k 1)
        (go label30))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (sswap nrhs
               (f2cl-lib:array-slice b-%data% single-float (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (sger (f2cl-lib:int-sub k 1) nrhs (- one)
           (f2cl-lib:array-slice a-%data% single-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice b-%data% single-float (k 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% single-float (1 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
        (when var-1 (setf nrhs var-1))
        (when var-6 (setf ldb$ var-6))
        (when var-8 (setf ldb$ var-8)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal nrhs
           (/ one
              (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                             a-%offset%))
           (f2cl-lib:array-slice b-%data% single-float (k 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-2))
```

```
    (when var-0 (setf nrhs var-0))
    (when var-3 (setf ldb$ var-3)))
  (setf k (f2cl-lib:int-sub k 1)))
 (t
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
  (if (/= kp (f2cl-lib:int-sub k 1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (sswap nrhs
           (f2cl-lib:array-slice b-%data% single-float
                                 ((+ k (f2cl-lib:int-sub 1)) 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-3))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (sger (f2cl-lib:int-sub k 2) nrhs (- one)
       (f2cl-lib:array-slice a-%data% single-float (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice b-%data% single-float (k 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice b-%data% single-float (1 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
    (when var-1 (setf nrhs var-1))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldb$ var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (sger (f2cl-lib:int-sub k 2) nrhs (- one)
       (f2cl-lib:array-slice a-%data% single-float
                             (1 (f2cl-lib:int-sub k 1))
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice b-%data% single-float
                             ((+ k (f2cl-lib:int-sub 1)) 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice b-%data% single-float (1 1)
                             ((1 ldb$) (1 *)) b-%offset%)
```

```
          ldb$)
        (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
        (when var-1 (setf nrhs var-1))
        (when var-6 (setf ldb$ var-6))
        (when var-8 (setf ldb$ var-8)))
      (setf akm1k
              (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                             ((1 lda) (1 *)) a-%offset%))
      (setf akm1
              (/
               (f2cl-lib:fref a-%data%
                              ((f2cl-lib:int-sub k 1)
                               (f2cl-lib:int-sub k 1))
                              ((1 lda) (1 *)) a-%offset%)
               akm1k))
      (setf ak
              (/
               (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                              a-%offset%)
               akm1k))
      (setf denom (- (* akm1 ak) one))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nrhs) nil)
        (tagbody
          (setf bkm1
                  (/
                   (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1) j)
                                  ((1 ldb$) (1 *)) b-%offset%)
                   akm1k))
          (setf bk
                  (/
                   (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                  b-%offset%)
                   akm1k))
          (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1) j)
                               ((1 ldb$) (1 *)) b-%offset%)
                  (/ (- (* ak bkm1) bk) denom))
          (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                               b-%offset%)
                  (/ (- (* akm1 bk) bkm1) denom))
         label20))
      (setf k (f2cl-lib:int-sub k 2))))
    (go label10)
  label30
    (setf k 1)
  label40
    (if (> k n)
        (go label50))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
```

```
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (sgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- one) b ldb$
      (f2cl-lib:array-slice a-%data% single-float (1 k)
                            ((1 lda) (1 *)) a-%offset%)
      1 one
      (f2cl-lib:array-slice b-%data% single-float (k 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
   (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
   (when var-2 (setf nrhs var-2))
   (when var-5 (setf ldb$ var-5))
   (when var-8 (setf one var-8))
   (when var-10 (setf ldb$ var-10)))
 (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
 (if (/= kp k)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (sswap nrhs
          (f2cl-lib:array-slice b-%data% single-float (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
       (declare (ignore var-1 var-3))
       (when var-0 (setf nrhs var-0))
       (when var-2 (setf ldb$ var-2))
       (when var-4 (setf ldb$ var-4))))
 (setf k (f2cl-lib:int-add k 1)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (sgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- one) b ldb$
      (f2cl-lib:array-slice a-%data% single-float (1 k)
                            ((1 lda) (1 *)) a-%offset%)
      1 one
      (f2cl-lib:array-slice b-%data% single-float (k 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
   (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
   (when var-2 (setf nrhs var-2))
   (when var-5 (setf ldb$ var-5))
   (when var-8 (setf one var-8))
   (when var-10 (setf ldb$ var-10)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (sgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- one) b ldb$
```

```
            (f2cl-lib:array-slice a-%data% single-float
                                  (1 (f2cl-lib:int-add k 1))
                                  ((1 lda) (1 *)) a-%offset%)
            1 one
            (f2cl-lib:array-slice b-%data% single-float ((+ k 1) 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
          (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
          (when var-2 (setf nrhs var-2))
          (when var-5 (setf ldb$ var-5))
          (when var-8 (setf one var-8))
          (when var-10 (setf ldb$ var-10)))
        (setf kp
                (f2cl-lib:int-sub
                 (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
        (if (/= kp k)
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (sswap nrhs
                 (f2cl-lib:array-slice b-%data% single-float (k 1)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$
                 (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$)
              (declare (ignore var-1 var-3))
              (when var-0 (setf nrhs var-0))
              (when var-2 (setf ldb$ var-2))
              (when var-4 (setf ldb$ var-4))))
        (setf k (f2cl-lib:int-add k 2))))
      (go label40)
     label50))
   (t
    (tagbody
      (setf k 1)
     label60
      (if (> k n)
          (go label80))
      (cond
       ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
        (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
        (if (/= kp k)
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (sswap nrhs
                 (f2cl-lib:array-slice b-%data% single-float (k 1)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$
                 (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$)
              (declare (ignore var-1 var-3))
```

```
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
  (if (< k n)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (sger (f2cl-lib:int-sub n k) nrhs (- one)
           (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice b-%data% single-float (k 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% single-float ((+ k 1) 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
        (when var-1 (setf nrhs var-1))
        (when var-6 (setf ldb$ var-6))
        (when var-8 (setf ldb$ var-8))))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (sscal nrhs
       (/ one
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
       (f2cl-lib:array-slice b-%data% single-float (k 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-1 var-2))
    (when var-0 (setf nrhs var-0))
    (when var-3 (setf ldb$ var-3)))
  (setf k (f2cl-lib:int-add k 1)))
 (t
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
  (if (/= kp (f2cl-lib:int-add k 1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (sswap nrhs
           (f2cl-lib:array-slice b-%data% single-float ((+ k 1) 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-3))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
  (cond
```

```
((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (sger (f2cl-lib:int-sub n k 1) nrhs (- one)
      (f2cl-lib:array-slice a-%data% single-float ((+ k 2) k)
                            ((1 lda) (1 *)) a-%offset%)
      1
      (f2cl-lib:array-slice b-%data% single-float (k 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$
      (f2cl-lib:array-slice b-%data% single-float ((+ k 2) 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
   (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
   (when var-1 (setf nrhs var-1))
   (when var-6 (setf ldb$ var-6))
   (when var-8 (setf ldb$ var-8)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (sger (f2cl-lib:int-sub n k 1) nrhs (- one)
      (f2cl-lib:array-slice a-%data% single-float
                            ((+ k 2) (f2cl-lib:int-add k 1))
                            ((1 lda) (1 *)) a-%offset%)
      1
      (f2cl-lib:array-slice b-%data% single-float ((+ k 1) 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$
      (f2cl-lib:array-slice b-%data% single-float ((+ k 2) 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
   (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
   (when var-1 (setf nrhs var-1))
   (when var-6 (setf ldb$ var-6))
   (when var-8 (setf ldb$ var-8)))))
(setf akm1k
       (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                      ((1 lda) (1 *)) a-%offset%))
(setf akm1
       (/
        (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                       a-%offset%)
        akm1k))
(setf ak
       (/
        (f2cl-lib:fref a-%data%
                       ((f2cl-lib:int-add k 1)
                        (f2cl-lib:int-add k 1))
                       ((1 lda) (1 *)) a-%offset%)
        akm1k))
(setf denom (- (* akm1 ak) one))
```

```
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (setf bkm1
                (/
                 (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                b-%offset%)
                 akm1k))
        (setf bk
                (/
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                                ((1 ldb$) (1 *)) b-%offset%)
                 akm1k))
        (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                             b-%offset%)
                (/ (- (* ak bkm1) bk) denom))
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)
                (/ (- (* akm1 bk) bkm1) denom))
       label70))
    (setf k (f2cl-lib:int-add k 2))))
  (go label60)
 label80
  (setf k n)
 label90
  (if (< k 1)
      (go label100))
  (cond
   ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
    (if (< k n)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10)
            (sgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- one)
             (f2cl-lib:array-slice b-%data% single-float ((+ k 1) 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$
             (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                                   ((1 lda) (1 *)) a-%offset%)
             1 one
             (f2cl-lib:array-slice b-%data% single-float (k 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$)
          (declare
           (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
          (when var-2 (setf nrhs var-2))
          (when var-5 (setf ldb$ var-5))
          (when var-8 (setf one var-8))
          (when var-10 (setf ldb$ var-10))))
    (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
```

```
 (if (/= kp k)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (sswap nrhs
          (f2cl-lib:array-slice b-%data% single-float (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
       (declare (ignore var-1 var-3))
       (when var-0 (setf nrhs var-0))
       (when var-2 (setf ldb$ var-2))
       (when var-4 (setf ldb$ var-4))))
 (setf k (f2cl-lib:int-sub k 1)))
(t
 (cond
  ((< k n)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (sgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- one)
        (f2cl-lib:array-slice b-%data% single-float ((+ k 1) 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$
        (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                              ((1 lda) (1 *)) a-%offset%)
        1 one
        (f2cl-lib:array-slice b-%data% single-float (k 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$)
     (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
     (when var-2 (setf nrhs var-2))
     (when var-5 (setf ldb$ var-5))
     (when var-8 (setf one var-8))
     (when var-10 (setf ldb$ var-10)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (sgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- one)
        (f2cl-lib:array-slice b-%data% single-float ((+ k 1) 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$
        (f2cl-lib:array-slice a-%data% single-float
                              ((+ k 1) (f2cl-lib:int-sub k 1))
                              ((1 lda) (1 *)) a-%offset%)
        1 one
        (f2cl-lib:array-slice b-%data% single-float
                              ((+ k (f2cl-lib:int-sub 1)) 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$)
```

```
                (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
                (when var-2 (setf nrhs var-2))
                (when var-5 (setf ldb$ var-5))
                (when var-8 (setf one var-8))
                (when var-10 (setf ldb$ var-10)))))
            (setf kp
                    (f2cl-lib:int-sub
                     (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
            (if (/= kp k)
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (sswap nrhs
                     (f2cl-lib:array-slice b-%data% single-float (k 1)
                                           ((1 ldb$) (1 *)) b-%offset%)
                     ldb$
                     (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                           ((1 ldb$) (1 *)) b-%offset%)
                     ldb$)
                  (declare (ignore var-1 var-3))
                  (when var-0 (setf nrhs var-0))
                  (when var-2 (setf ldb$ var-2))
                  (when var-4 (setf ldb$ var-4))))
            (setf k (f2cl-lib:int-sub k 2))))
          (go label90)
         label100)))
       (go end_label)
      end_label
       (return (values uplo nil nrhs nil nil nil nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssytrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil
                                              fortran-to-lisp::nrhs nil nil nil
                                              nil fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

59.26 ssytrs2

59.26.1 Axiom unit tests

— **ssytrs2.input** —

```
)set break resume
)sys rm -f ssytrs2.output
)spool ssytrs2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.26.2 Axiom help page

— **ssytrs2.help** —

```
====================================================================
ssytrs2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssytrs2 (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK, INFO)
           SSYTRS2

Function/Subroutine Documentation
   subroutine ssytrs2 (character UPLO, integer N, integer NRHS, real,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV, real,
       dimension( ldb, * ) B, integer LDB, real, dimension( * ) WORK, integer
```

```
      INFO)
      SSYTRS2

Purpose:
   SSYTRS2 solves a system of linear equations A*X = B with a real
   symmetric matrix A using the factorization A = U*D*U**T or
   A = L*D*L**T computed by SSYTRF and converted by SSYCONV.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is REAL array, dimension (LDA,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by SSYTRF.
   Note that A is input / output. This might be counter-intuitive,
   and one may think that A is input only. A is input / output. This
   is because, at the start of the subroutine, we permute A in a
   "better" form and then we permute A back to its original form at
   the end.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by SSYTRF.

 B
   B is REAL array, dimension (LDB,NRHS)
   On entry, the right hand side matrix B.
   On exit, the solution matrix X.
```

```
LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

WORK
  WORK is REAL array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

59.26.3 fortran code

— **ssytrs2.f** —

```
*  =====================================================================
      SUBROUTINE SSYTRS2( UPLO, N, NRHS, A, LDA, IPIV, B, LDB,
     $                    WORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, IINFO, J, K, KP
      REAL               AK, AKM1, AKM1K, BK, BKM1, DENOM
*     ..
```

```
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SSCAL, SSYCONV, SSWAP, STRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYTRS2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
*     Convert A
*
      CALL SSYCONV( UPLO, 'C', N, A, LDA, IPIV, WORK, IINFO )
*
      IF( UPPER ) THEN
*
*        Solve A*X = B, where A = U*D*U**T.
*
*       P**T * B
        K=N
        DO WHILE ( K .GE. 1 )
         IF( IPIV( K ).GT.0 ) THEN
*           1 x 1 diagonal block
*           Interchange rows K and IPIV(K).
            KP = IPIV( K )
```

```
            IF( KP.NE.K )
     $         CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K=K-1
         ELSE
*           2 x 2 diagonal block
*           Interchange rows K-1 and -IPIV(K).
            KP = -IPIV( K )
            IF( KP.EQ.-IPIV( K-1 ) )
     $         CALL SSWAP( NRHS, B( K-1, 1 ), LDB, B( KP, 1 ), LDB )
            K=K-2
         END IF
        END DO
*
*  Compute (U \P**T * B) -> B    [ (U \P**T * B) ]
*
        CALL STRSM('L','U','N','U',N,NRHS,ONE,A,LDA,B,LDB)
*
*  Compute D \ B -> B   [ D \ (U \P**T * B) ]
*
         I=N
         DO WHILE ( I .GE. 1 )
            IF( IPIV(I) .GT. 0 ) THEN
              CALL SSCAL( NRHS, ONE / A( I, I ), B( I, 1 ), LDB )
            ELSEIF ( I .GT. 1) THEN
               IF ( IPIV(I-1) .EQ. IPIV(I) ) THEN
                  AKM1K = WORK(I)
                  AKM1 = A( I-1, I-1 ) / AKM1K
                  AK = A( I, I ) / AKM1K
                  DENOM = AKM1*AK - ONE
                  DO 15 J = 1, NRHS
                     BKM1 = B( I-1, J ) / AKM1K
                     BK = B( I, J ) / AKM1K
                     B( I-1, J ) = ( AK*BKM1-BK ) / DENOM
                     B( I, J ) = ( AKM1*BK-BKM1 ) / DENOM
 15               CONTINUE
               I = I - 1
               ENDIF
            ENDIF
            I = I - 1
         END DO
*
*      Compute (U**T \ B) -> B   [ U**T \ (D \ (U \P**T * B) ) ]
*
         CALL STRSM('L','U','T','U',N,NRHS,ONE,A,LDA,B,LDB)
*
*       P * B  [ P * (U**T \ (D \ (U \P**T * B) )) ]
*
        K=1
        DO WHILE ( K .LE. N )
         IF( IPIV( K ).GT.0 ) THEN
```

```
*          1 x 1 diagonal block
*          Interchange rows K and IPIV(K).
           KP = IPIV( K )
           IF( KP.NE.K )
     $        CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
           K=K+1
        ELSE
*          2 x 2 diagonal block
*          Interchange rows K-1 and -IPIV(K).
           KP = -IPIV( K )
           IF( K .LT. N .AND. KP.EQ.-IPIV( K+1 ) )
     $        CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
           K=K+2
        ENDIF
       END DO
*
      ELSE
*
*       Solve A*X = B, where A = L*D*L**T.
*
*      P**T * B
       K=1
       DO WHILE ( K .LE. N )
        IF( IPIV( K ).GT.0 ) THEN
*          1 x 1 diagonal block
*          Interchange rows K and IPIV(K).
           KP = IPIV( K )
           IF( KP.NE.K )
     $        CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
           K=K+1
        ELSE
*          2 x 2 diagonal block
*          Interchange rows K and -IPIV(K+1).
           KP = -IPIV( K+1 )
           IF( KP.EQ.-IPIV( K ) )
     $        CALL SSWAP( NRHS, B( K+1, 1 ), LDB, B( KP, 1 ), LDB )
           K=K+2
        ENDIF
       END DO
*
*  Compute (L \P**T * B) -> B    [ (L \P**T * B) ]
*
       CALL STRSM('L','L','N','U',N,NRHS,ONE,A,LDA,B,LDB)
*
*  Compute D \ B -> B   [ D \ (L \P**T * B) ]
*
        I=1
        DO WHILE ( I .LE. N )
           IF( IPIV(I) .GT. 0 ) THEN
             CALL SSCAL( NRHS, ONE / A( I, I ), B( I, 1 ), LDB )
```

```
           ELSE
                 AKM1K = WORK(I)
                 AKM1 = A( I, I ) / AKM1K
                 AK = A( I+1, I+1 ) / AKM1K
                 DENOM = AKM1*AK - ONE
                 DO 25 J = 1, NRHS
                    BKM1 = B( I, J ) / AKM1K
                    BK = B( I+1, J ) / AKM1K
                    B( I, J ) = ( AK*BKM1-BK ) / DENOM
                    B( I+1, J ) = ( AKM1*BK-BKM1 ) / DENOM
 25              CONTINUE
                 I = I + 1
           ENDIF
           I = I + 1
        END DO
*
*  Compute (L**T \ B) -> B   [ L**T \ (D \ (L \P**T * B) ) ]
*
       CALL STRSM('L','L','T','U',N,NRHS,ONE,A,LDA,B,LDB)
*
*      P * B  [ P * (L**T \ (D \ (L \P**T * B) )) ]
*
       K=N
       DO WHILE ( K .GE. 1 )
        IF( IPIV( K ).GT.0 ) THEN
*          1 x 1 diagonal block
*          Interchange rows K and IPIV(K).
           KP = IPIV( K )
           IF( KP.NE.K )
     $        CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
           K=K-1
        ELSE
*          2 x 2 diagonal block
*          Interchange rows K-1 and -IPIV(K).
           KP = -IPIV( K )
           IF( K.GT.1 .AND. KP.EQ.-IPIV( K-1 ) )
     $        CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
           K=K-2
        ENDIF
       END DO
*
     END IF
*
*    Revert A
*
     CALL SSYCONV( UPLO, 'R', N, A, LDA, IPIV, WORK, IINFO )
*
     RETURN
*
*    End of SSYTRS2
```

```
*
      END
```

———-

59.26.4 lisp code

— **ssytrs2.lisp** —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun ssytrs2 (uplo n nrhs a lda ipiv b ldb$ work info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array single-float (*)) work b a)
             (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%)
         (work single-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((ak 0.0) (akm1 0.0) (akm1k 0.0) (bk 0.0) (bkm1 0.0) (denom 0.0)
             (i 0) (iinfo 0) (j 0) (k 0) (kp 0) (upper nil))
        (declare (type (single-float) ak akm1 akm1k bk bkm1 denom)
                 (type (f2cl-lib:integer4) i iinfo j k kp)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -5))
         ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
```

```
  (setf info -8)))
(cond
 ((/= info 0) (xerbla "SSYTRS2" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= n 0) (= nrhs 0))
    (go end_label))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (ssyconv uplo "C" n a lda ipiv work iinfo)
  (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
  (setf uplo var-0)
  (setf iinfo var-7))
(cond
 (upper
  (tagbody
    (setf k n)
   label100000
    (if (not (>= k 1))
        (go label100001))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (sswap nrhs
               (f2cl-lib:array-slice b-%data% single-float (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-sub k 1)))
     (t
      (setf kp
              (f2cl-lib:int-sub
               (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
      (if (= kp
             (f2cl-lib:int-sub
              (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                             ((1 *)) ipiv-%offset%)))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (sswap nrhs
               (f2cl-lib:array-slice b-%data% single-float
                                     ((+ k (f2cl-lib:int-sub 1)) 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% single-float (kp 1)
```

```
                                    ((1 ldb$) (1 *)) b-%offset%)
              ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
    (setf k (f2cl-lib:int-sub k 2))))
  (go label100000)
 label100001
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (strsm "L" "U" "N" "U" n nrhs one a lda b ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10)))
  (setf i n)
 label100002
  (if (not (>= i 1))
      (go label100003))
  (cond
   ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal nrhs
         (/ one
            (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                           a-%offset%))
         (f2cl-lib:array-slice b-%data% single-float (i 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-1 var-2))
      (when var-0 (setf nrhs var-0))
      (when var-3 (setf ldb$ var-3))))
   ((> i 1)
    (cond
     ((=
       (f2cl-lib:fref ipiv
                      ((f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((1 *)))
       (f2cl-lib:fref ipiv (i) ((1 *))))
      (setf akm1k
              (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
      (setf akm1
              (/
               (f2cl-lib:fref a-%data%
                              ((f2cl-lib:int-sub i 1)
                               (f2cl-lib:int-sub i 1))
```

```
                                ((1 lda) (1 *)) a-%offset%)
                akm1k))
        (setf ak
                (/
                 (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                a-%offset%)
                akm1k))
        (setf denom (- (* akm1 ak) one))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j nrhs) nil)
          (tagbody
            (setf bkm1
                    (/
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                    ((1 ldb$) (1 *)) b-%offset%)
                     akm1k))
            (setf bk
                    (/
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)
                     akm1k))
            (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                    (/ (- (* ak bkm1) bk) denom))
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                    (/ (- (* akm1 bk) bkm1) denom))
           label15))
        (setf i (f2cl-lib:int-sub i 1))))))
    (setf i (f2cl-lib:int-sub i 1))
    (go label100002)
  label100003
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (strsm "L" "U" "T" "U" n nrhs one a lda b ldb$)
      (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
      (when var-4 (setf n var-4))
      (when var-5 (setf nrhs var-5))
      (when var-6 (setf one var-6))
      (when var-8 (setf lda var-8))
      (when var-10 (setf ldb$ var-10)))
    (setf k 1)
  label100004
    (if (not (<= k n))
        (go label100005))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (if (/= kp k)
```

```
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (sswap nrhs
                 (f2cl-lib:array-slice b-%data% single-float (k 1)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$
                 (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$)
              (declare (ignore var-1 var-3))
              (when var-0 (setf nrhs var-0))
              (when var-2 (setf ldb$ var-2))
              (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-add k 1)))
     (t
      (setf kp
              (f2cl-lib:int-sub
               (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
      (if (and (< k n)
               (= kp
                  (f2cl-lib:int-sub
                   (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
                                  ((1 *)) ipiv-%offset%))))
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (sswap nrhs
                 (f2cl-lib:array-slice b-%data% single-float (k 1)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$
                 (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$)
              (declare (ignore var-1 var-3))
              (when var-0 (setf nrhs var-0))
              (when var-2 (setf ldb$ var-2))
              (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-add k 2))))
    (go label100004)
   label100005))
  (t
   (tagbody
     (setf k 1)
    label100006
     (if (not (<= k n))
         (go label100007))
     (cond
      ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
       (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
       (if (/= kp k)
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
               (sswap nrhs
                (f2cl-lib:array-slice b-%data% single-float (k 1)
```

```
                                    ((1 ldb$) (1 *)) b-%offset%)
                ldb$
                (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                    ((1 ldb$) (1 *)) b-%offset%)
                ldb$)
              (declare (ignore var-1 var-3))
              (when var-0 (setf nrhs var-0))
              (when var-2 (setf ldb$ var-2))
              (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-add k 1)))
     (t
      (setf kp
              (f2cl-lib:int-sub
               (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
                              ((1 *)) ipiv-%offset%)))
      (if (= kp
             (f2cl-lib:int-sub
              (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (sswap nrhs
               (f2cl-lib:array-slice b-%data% single-float ((+ k 1) 1)
                                    ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                    ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
              (declare (ignore var-1 var-3))
              (when var-0 (setf nrhs var-0))
              (when var-2 (setf ldb$ var-2))
              (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-add k 2))))
    (go label100006)
   label100007
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (strsm "L" "L" "N" "U" n nrhs one a lda b ldb$)
      (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
      (when var-4 (setf n var-4))
      (when var-5 (setf nrhs var-5))
      (when var-6 (setf one var-6))
      (when var-8 (setf lda var-8))
      (when var-10 (setf ldb$ var-10)))
    (setf i 1)
   label100008
    (if (not (<= i n))
        (go label100009))
    (cond
     ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
      (multiple-value-bind (var-0 var-1 var-2 var-3)
```

```
        (sscal nrhs
         (/ one
            (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                           a-%offset%))
         (f2cl-lib:array-slice b-%data% single-float (i 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-1 var-2))
      (when var-0 (setf nrhs var-0))
      (when var-3 (setf ldb$ var-3))))
    (t
     (setf akm1k
             (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
     (setf akm1
             (/
              (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                             a-%offset%)
              akm1k))
     (setf ak
             (/
              (f2cl-lib:fref a-%data%
                             ((f2cl-lib:int-add i 1)
                              (f2cl-lib:int-add i 1))
                             ((1 lda) (1 *)) a-%offset%)
              akm1k))
     (setf denom (- (* akm1 ak) one))
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j nrhs) nil)
       (tagbody
         (setf bkm1
                 (/
                  (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  akm1k))
         (setf bk
                 (/
                  (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                  akm1k))
         (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)
                 (/ (- (* ak bkm1) bk) denom))
         (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                              ((1 ldb$) (1 *)) b-%offset%)
                 (/ (- (* akm1 bk) bkm1) denom))
        label25))
     (setf i (f2cl-lib:int-add i 1))))
   (setf i (f2cl-lib:int-add i 1))
   (go label100008)
  label100009
```

```
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (strsm "L" "L" "T" "U" n nrhs one a lda b ldb$)
   (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
   (when var-4 (setf n var-4))
   (when var-5 (setf nrhs var-5))
   (when var-6 (setf one var-6))
   (when var-8 (setf lda var-8))
   (when var-10 (setf ldb$ var-10)))
 (setf k n)
label100010
 (if (not (>= k 1))
     (go label100011))
 (cond
  ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
   (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
   (if (/= kp k)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (sswap nrhs
            (f2cl-lib:array-slice b-%data% single-float (k 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$
            (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
         (declare (ignore var-1 var-3))
         (when var-0 (setf nrhs var-0))
         (when var-2 (setf ldb$ var-2))
         (when var-4 (setf ldb$ var-4))))
   (setf k (f2cl-lib:int-sub k 1)))
  (t
   (setf kp
           (f2cl-lib:int-sub
            (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
   (if (and (> k 1)
            (= kp
               (f2cl-lib:int-sub
                (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                               ((1 *)) ipiv-%offset%))))
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (sswap nrhs
            (f2cl-lib:array-slice b-%data% single-float (k 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$
            (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
         (declare (ignore var-1 var-3))
         (when var-0 (setf nrhs var-0))
```

```
                    (when var-2 (setf ldb$ var-2))
                    (when var-4 (setf ldb$ var-4))))
              (setf k (f2cl-lib:int-sub k 2))))
            (go label100010)
           label100011)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (ssyconv uplo "R" n a lda ipiv work iinfo)
          (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
          (setf uplo var-0)
          (setf iinfo var-7))
        (go end_label)
       end_label
        (return (values uplo n nrhs nil lda nil nil ldb$ nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssytrs2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::ldb$ nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::ssyconv))))
```

———-

59.27 ssytrs_rook

59.27.1 Axiom unit tests

— **ssytrsrook.input** —

```
)set break resume
)sys rm -f ssytrsrook.output
)spool ssytrsrook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.27.2 Axiom help page

— **ssytrs_rook.help** —

```
====================================================================
ssytrs_rook examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssytrs_rook (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO)
           SSYTRS_ROOK

Function/Subroutine Documentation
   subroutine ssytrs_rook (character UPLO, integer N, integer NRHS, real,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV, real,
       dimension( ldb, * ) B, integer LDB, integer INFO)
       SSYTRS_ROOK

Purpose:
   SSYTRS_ROOK solves a system of linear equations A*X = B with
   a real symmetric matrix A using the factorization A = U*D*U**T or
   A = L*D*L**T computed by SSYTRF_ROOK.
```

```
Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is REAL array, dimension (LDA,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by SSYTRF_ROOK.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by SSYTRF_ROOK.

 B
   B is REAL array, dimension (LDB,NRHS)
   On entry, the right hand side matrix B.
   On exit, the solution matrix X.

 LDB
   LDB is INTEGER
   The leading dimension of the array B.  LDB >= max(1,N).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

59.27.3 fortran code

— ssytrs_rook.f —

```
*  =====================================================================
      SUBROUTINE SSYTRS_ROOK( UPLO, N, NRHS, A, LDA, IPIV, B, LDB,
     $                        INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            J, K, KP
      REAL               AK, AKM1, AKM1K, BK, BKM1, DENOM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEMV, SGER, SSCAL, SSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
```

```
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SSYTRS_ROOK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*        Solve A*X = B, where A = U*D*U**T.
*
*        First solve U*D*X = B, overwriting B with X.
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop.
*
         IF( K.LT.1 )
     $      GO TO 30
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
*           Multiply by inv(U(K)), where U(K) is the transformation
*           stored in column K of A.
*
            CALL SGER( K-1, NRHS, -ONE, A( 1, K ), 1, B( K, 1 ), LDB,
```

```
     $                B( 1, 1 ), LDB )
*
*          Multiply by the inverse of the diagonal block.
*
           CALL SSCAL( NRHS, ONE / A( K, K ), B( K, 1 ), LDB )
           K = K - 1
        ELSE
*
*          2 x 2 diagonal block
*
*          Interchange rows K and -IPIV(K) THEN K-1 and -IPIV(K-1)
*
           KP = -IPIV( K )
           IF( KP.NE.K )
     $        CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
           KP = -IPIV( K-1 )
           IF( KP.NE.K-1 )
     $        CALL SSWAP( NRHS, B( K-1, 1 ), LDB, B( KP, 1 ), LDB )
*
*          Multiply by inv(U(K)), where U(K) is the transformation
*          stored in columns K-1 and K of A.
*
           IF( K.GT.2 ) THEN
              CALL SGER( K-2, NRHS, -ONE, A( 1, K ), 1, B( K, 1 ),
     $                   LDB, B( 1, 1 ), LDB )
              CALL SGER( K-2, NRHS, -ONE, A( 1, K-1 ), 1, B( K-1, 1 ),
     $                   LDB, B( 1, 1 ), LDB )
           END IF
*
*          Multiply by the inverse of the diagonal block.
*
           AKM1K = A( K-1, K )
           AKM1 = A( K-1, K-1 ) / AKM1K
           AK = A( K, K ) / AKM1K
           DENOM = AKM1*AK - ONE
           DO 20 J = 1, NRHS
              BKM1 = B( K-1, J ) / AKM1K
              BK = B( K, J ) / AKM1K
              B( K-1, J ) = ( AK*BKM1-BK ) / DENOM
              B( K, J ) = ( AKM1*BK-BKM1 ) / DENOM
   20      CONTINUE
           K = K - 2
        END IF
*
        GO TO 10
   30   CONTINUE
*
*       Next solve U**T *X = B, overwriting B with X.
*
```

```
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = 1
   40    CONTINUE
*
*        If K > N, exit from loop.
*
         IF( K.GT.N )
     $      GO TO 50
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Multiply by inv(U**T(K)), where U(K) is the transformation
*           stored in column K of A.
*
            IF( K.GT.1 )
     $         CALL SGEMV( 'Transpose', K-1, NRHS, -ONE, B,
     $                     LDB, A( 1, K ), 1, ONE, B( K, 1 ), LDB )
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K = K + 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Multiply by inv(U**T(K+1)), where U(K+1) is the transformation
*           stored in columns K and K+1 of A.
*
            IF( K.GT.1 ) THEN
               CALL SGEMV( 'Transpose', K-1, NRHS, -ONE, B,
     $                     LDB, A( 1, K ), 1, ONE, B( K, 1 ), LDB )
               CALL SGEMV( 'Transpose', K-1, NRHS, -ONE, B,
     $                     LDB, A( 1, K+1 ), 1, ONE, B( K+1, 1 ), LDB )
            END IF
*
*           Interchange rows K and -IPIV(K) THEN K+1 and -IPIV(K+1).
*
            KP = -IPIV( K )
            IF( KP.NE.K )
     $         CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
            KP = -IPIV( K+1 )
            IF( KP.NE.K+1 )
```

```
     $         CALL SSWAP( NRHS, B( K+1, 1 ), LDB, B( KP, 1 ), LDB )
*
           K = K + 2
        END IF
*
        GO TO 40
   50   CONTINUE
*
      ELSE
*
*       Solve A*X = B, where A = L*D*L**T.
*
*       First solve L*D*X = B, overwriting B with X.
*
*       K is the main loop index, increasing from 1 to N in steps of
*       1 or 2, depending on the size of the diagonal blocks.
*
        K = 1
   60   CONTINUE
*
*       If K > N, exit from loop.
*
        IF( K.GT.N )
     $     GO TO 80
*
        IF( IPIV( K ).GT.0 ) THEN
*
*          1 x 1 diagonal block
*
*          Interchange rows K and IPIV(K).
*
           KP = IPIV( K )
           IF( KP.NE.K )
     $        CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
*          Multiply by inv(L(K)), where L(K) is the transformation
*          stored in column K of A.
*
           IF( K.LT.N )
     $        CALL SGER( N-K, NRHS, -ONE, A( K+1, K ), 1, B( K, 1 ),
     $                   LDB, B( K+1, 1 ), LDB )
*
*          Multiply by the inverse of the diagonal block.
*
           CALL SSCAL( NRHS, ONE / A( K, K ), B( K, 1 ), LDB )
           K = K + 1
        ELSE
*
*          2 x 2 diagonal block
*
```

```
*           Interchange rows K and -IPIV(K) THEN K+1 and -IPIV(K+1)
*
            KP = -IPIV( K )
            IF( KP.NE.K )
     $         CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
            KP = -IPIV( K+1 )
            IF( KP.NE.K+1 )
     $         CALL SSWAP( NRHS, B( K+1, 1 ), LDB, B( KP, 1 ), LDB )
*
*           Multiply by inv(L(K)), where L(K) is the transformation
*           stored in columns K and K+1 of A.
*
            IF( K.LT.N-1 ) THEN
               CALL SGER( N-K-1, NRHS, -ONE, A( K+2, K ), 1, B( K, 1 ),
     $                    LDB, B( K+2, 1 ), LDB )
               CALL SGER( N-K-1, NRHS, -ONE, A( K+2, K+1 ), 1,
     $                    B( K+1, 1 ), LDB, B( K+2, 1 ), LDB )
            END IF
*
*           Multiply by the inverse of the diagonal block.
*
            AKM1K = A( K+1, K )
            AKM1 = A( K, K ) / AKM1K
            AK = A( K+1, K+1 ) / AKM1K
            DENOM = AKM1*AK - ONE
            DO 70 J = 1, NRHS
               BKM1 = B( K, J ) / AKM1K
               BK = B( K+1, J ) / AKM1K
               B( K, J ) = ( AK*BKM1-BK ) / DENOM
               B( K+1, J ) = ( AKM1*BK-BKM1 ) / DENOM
   70       CONTINUE
            K = K + 2
         END IF
*
         GO TO 60
   80    CONTINUE
*
*        Next solve L**T *X = B, overwriting B with X.
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = N
   90    CONTINUE
*
*        If K < 1, exit from loop.
*
         IF( K.LT.1 )
     $      GO TO 100
```

```
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Multiply by inv(L**T(K)), where L(K) is the transformation
*           stored in column K of A.
*
            IF( K.LT.N )
     $         CALL SGEMV( 'Transpose', N-K, NRHS, -ONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K ), 1, ONE, B( K, 1 ), LDB )
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K = K - 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Multiply by inv(L**T(K-1)), where L(K-1) is the transformation
*           stored in columns K-1 and K of A.
*
            IF( K.LT.N ) THEN
               CALL SGEMV( 'Transpose', N-K, NRHS, -ONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K ), 1, ONE, B( K, 1 ), LDB )
               CALL SGEMV( 'Transpose', N-K, NRHS, -ONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K-1 ), 1, ONE, B( K-1, 1 ),
     $                     LDB )
            END IF
*
*           Interchange rows K and -IPIV(K) THEN K-1 and -IPIV(K-1)
*
            KP = -IPIV( K )
            IF( KP.NE.K )
     $         CALL SSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
            KP = -IPIV( K-1 )
            IF( KP.NE.K-1 )
     $         CALL SSWAP( NRHS, B( K-1, 1 ), LDB, B( KP, 1 ), LDB )
*
            K = K - 2
         END IF
*
         GO TO 90
  100    CONTINUE
      END IF
*
```

```
      RETURN
*
*     End of SSYTRS_ROOK
*
      END
```

———-

59.27.4 lisp code

— **ssytrs_rook.lisp** —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun ssytrs_rook (uplo n nrhs a lda ipiv b ldb$ info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array single-float (*)) b a)
             (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((ak 0.0) (akm1 0.0) (akm1k 0.0) (bk 0.0) (bkm1 0.0) (denom 0.0)
             (j 0) (k 0) (kp 0) (upper nil))
        (declare (type (single-float) ak akm1 akm1k bk bkm1 denom)
                 (type (f2cl-lib:integer4) j k kp)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
```

```
  (setf info -5))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8)))
(cond
 ((/= info 0) (xerbla "SSYTRS_ROOK" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= n 0) (= nrhs 0))
    (go end_label))
(cond
 (upper
  (tagbody
    (setf k n)
   label10
    (if (< k 1)
        (go label30))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (sswap nrhs
               (f2cl-lib:array-slice b-%data% single-float (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (sger (f2cl-lib:int-sub k 1) nrhs (- one)
           (f2cl-lib:array-slice a-%data% single-float (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice b-%data% single-float (k 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% single-float (1 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
        (when var-1 (setf nrhs var-1))
        (when var-6 (setf ldb$ var-6))
        (when var-8 (setf ldb$ var-8)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal nrhs
           (/ one
```

```
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
       (f2cl-lib:array-slice b-%data% single-float (k 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-1 var-2))
    (when var-0 (setf nrhs var-0))
    (when var-3 (setf ldb$ var-3)))
  (setf k (f2cl-lib:int-sub k 1)))
 (t
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
  (if (/= kp k)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (sswap nrhs
           (f2cl-lib:array-slice b-%data% single-float (k 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-3))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                          ((1 *)) ipiv-%offset%)))
  (if (/= kp (f2cl-lib:int-sub k 1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (sswap nrhs
           (f2cl-lib:array-slice b-%data% single-float
                                 ((+ k (f2cl-lib:int-sub 1)) 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-3))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
  (cond
   ((> k 2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (sger (f2cl-lib:int-sub k 2) nrhs (- one)
         (f2cl-lib:array-slice a-%data% single-float (1 k)
```

```
                                ((1 lda) (1 *)) a-%offset%)
        1
        (f2cl-lib:array-slice b-%data% single-float (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
        ldb$
        (f2cl-lib:array-slice b-%data% single-float (1 1)
                                ((1 ldb$) (1 *)) b-%offset%)
        ldb$)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
    (when var-1 (setf nrhs var-1))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldb$ var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (sger (f2cl-lib:int-sub k 2) nrhs (- one)
        (f2cl-lib:array-slice a-%data% single-float
                                (1 (f2cl-lib:int-sub k 1))
                                ((1 lda) (1 *)) a-%offset%)
        1
        (f2cl-lib:array-slice b-%data% single-float
                                ((+ k (f2cl-lib:int-sub 1)) 1)
                                ((1 ldb$) (1 *)) b-%offset%)
        ldb$
        (f2cl-lib:array-slice b-%data% single-float (1 1)
                                ((1 ldb$) (1 *)) b-%offset%)
        ldb$)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
    (when var-1 (setf nrhs var-1))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldb$ var-8)))))
(setf akm1k
        (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                        ((1 lda) (1 *)) a-%offset%))
(setf akm1
        (/
         (f2cl-lib:fref a-%data%
                         ((f2cl-lib:int-sub k 1)
                          (f2cl-lib:int-sub k 1))
                         ((1 lda) (1 *)) a-%offset%)
         akm1k))
(setf ak
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%)
         akm1k))
(setf denom (- (* akm1 ak) one))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf bkm1
```

```
                 (/
                  (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                  akm1k))
         (setf bk
                 (/
                  (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  akm1k))
         (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1) j)
                              ((1 ldb$) (1 *)) b-%offset%)
                 (/ (- (* ak bkm1) bk) denom))
         (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                              b-%offset%)
                 (/ (- (* akm1 bk) bkm1) denom))
       label20))
    (setf k (f2cl-lib:int-sub k 2))))
  (go label10)
 label30
  (setf k 1)
 label40
  (if (> k n)
      (go label50))
  (cond
   ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
    (if (> k 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10)
            (sgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- one) b
             ldb$
             (f2cl-lib:array-slice a-%data% single-float (1 k)
                                   ((1 lda) (1 *)) a-%offset%)
             1 one
             (f2cl-lib:array-slice b-%data% single-float (k 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$)
          (declare
           (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
          (when var-2 (setf nrhs var-2))
          (when var-5 (setf ldb$ var-5))
          (when var-8 (setf one var-8))
          (when var-10 (setf ldb$ var-10))))
    (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
    (if (/= kp k)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (sswap nrhs
             (f2cl-lib:array-slice b-%data% single-float (k 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$
```

```
          (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
        (declare (ignore var-1 var-3))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
 (setf k (f2cl-lib:int-add k 1)))
(t
 (cond
  ((> k 1)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (sgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- one) b
        ldb$
        (f2cl-lib:array-slice a-%data% single-float (1 k)
                              ((1 lda) (1 *)) a-%offset%)
        1 one
        (f2cl-lib:array-slice b-%data% single-float (k 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$)
     (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
     (when var-2 (setf nrhs var-2))
     (when var-5 (setf ldb$ var-5))
     (when var-8 (setf one var-8))
     (when var-10 (setf ldb$ var-10)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (sgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- one) b
        ldb$
        (f2cl-lib:array-slice a-%data% single-float
                              (1 (f2cl-lib:int-add k 1))
                              ((1 lda) (1 *)) a-%offset%)
        1 one
        (f2cl-lib:array-slice b-%data% single-float ((+ k 1) 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$)
     (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
     (when var-2 (setf nrhs var-2))
     (when var-5 (setf ldb$ var-5))
     (when var-8 (setf one var-8))
     (when var-10 (setf ldb$ var-10)))))
 (setf kp
         (f2cl-lib:int-sub
          (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
 (if (/= kp k)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (sswap nrhs
```

```
                (f2cl-lib:array-slice b-%data% single-float (k 1)
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$
                (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$)
              (declare (ignore var-1 var-3))
              (when var-0 (setf nrhs var-0))
              (when var-2 (setf ldb$ var-2))
              (when var-4 (setf ldb$ var-4))))
      (setf kp
              (f2cl-lib:int-sub
                (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
                               ((1 *)) ipiv-%offset%)))
      (if (/= kp (f2cl-lib:int-add k 1))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (sswap nrhs
                (f2cl-lib:array-slice b-%data% single-float ((+ k 1) 1)
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$
                (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$)
              (declare (ignore var-1 var-3))
              (when var-0 (setf nrhs var-0))
              (when var-2 (setf ldb$ var-2))
              (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-add k 2))))
    (go label40)
   label50))
 (t
  (tagbody
    (setf k 1)
   label60
    (if (> k n)
        (go label80))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (sswap nrhs
                (f2cl-lib:array-slice b-%data% single-float (k 1)
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$
                (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$)
              (declare (ignore var-1 var-3))
              (when var-0 (setf nrhs var-0))
```

```
      (when var-2 (setf ldb$ var-2))
      (when var-4 (setf ldb$ var-4))))
 (if (< k n)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (sger (f2cl-lib:int-sub n k) nrhs (- one)
          (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                                ((1 lda) (1 *)) a-%offset%)
          1
          (f2cl-lib:array-slice b-%data% single-float (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice b-%data% single-float ((+ k 1) 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
      (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
      (when var-1 (setf nrhs var-1))
      (when var-6 (setf ldb$ var-6))
      (when var-8 (setf ldb$ var-8))))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (sscal nrhs
      (/ one
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%))
      (f2cl-lib:array-slice b-%data% single-float (k 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
   (declare (ignore var-1 var-2))
   (when var-0 (setf nrhs var-0))
   (when var-3 (setf ldb$ var-3)))
 (setf k (f2cl-lib:int-add k 1)))
(t
 (setf kp
         (f2cl-lib:int-sub
          (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
 (if (/= kp k)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (sswap nrhs
          (f2cl-lib:array-slice b-%data% single-float (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
      (declare (ignore var-1 var-3))
      (when var-0 (setf nrhs var-0))
      (when var-2 (setf ldb$ var-2))
      (when var-4 (setf ldb$ var-4))))
 (setf kp
         (f2cl-lib:int-sub
```

```
          (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
                         ((1 *)) ipiv-%offset%)))
(if (/= kp (f2cl-lib:int-add k 1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (sswap nrhs
         (f2cl-lib:array-slice b-%data% single-float ((+ k 1) 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice b-%data% single-float (kp 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-1 var-3))
      (when var-0 (setf nrhs var-0))
      (when var-2 (setf ldb$ var-2))
      (when var-4 (setf ldb$ var-4))))
(cond
 ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (sger (f2cl-lib:int-sub n k 1) nrhs (- one)
       (f2cl-lib:array-slice a-%data% single-float ((+ k 2) k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice b-%data% single-float (k 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice b-%data% single-float ((+ k 2) 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
    (when var-1 (setf nrhs var-1))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldb$ var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (sger (f2cl-lib:int-sub n k 1) nrhs (- one)
       (f2cl-lib:array-slice a-%data% single-float
                             ((+ k 2) (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice b-%data% single-float ((+ k 1) 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice b-%data% single-float ((+ k 2) 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
    (when var-1 (setf nrhs var-1))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldb$ var-8)))))
```

```
    (setf akm1k
            (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                           ((1 lda) (1 *)) a-%offset%))
    (setf akm1
            (/
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)
             akm1k))
    (setf ak
            (/
             (f2cl-lib:fref a-%data%
                            ((f2cl-lib:int-add k 1)
                             (f2cl-lib:int-add k 1))
                            ((1 lda) (1 *)) a-%offset%)
             akm1k))
    (setf denom (- (* akm1 ak) one))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (setf bkm1
                (/
                 (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                b-%offset%)
                 akm1k))
        (setf bk
                (/
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                                ((1 ldb$) (1 *)) b-%offset%)
                 akm1k))
        (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                             b-%offset%)
                (/ (- (* ak bkm1) bk) denom))
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)
                (/ (- (* akm1 bk) bkm1) denom))
       label70))
    (setf k (f2cl-lib:int-add k 2))))
  (go label60)
 label80
  (setf k n)
 label90
  (if (< k 1)
      (go label100))
  (cond
   ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
    (if (< k n)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10)
            (sgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- one)
```

```
            (f2cl-lib:array-slice b-%data% single-float ((+ k 1) 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$
            (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                                  ((1 lda) (1 *)) a-%offset%)
            1 one
            (f2cl-lib:array-slice b-%data% single-float (k 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
          (declare
           (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
          (when var-2 (setf nrhs var-2))
          (when var-5 (setf ldb$ var-5))
          (when var-8 (setf one var-8))
          (when var-10 (setf ldb$ var-10))))
   (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
   (if (/= kp k)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (sswap nrhs
            (f2cl-lib:array-slice b-%data% single-float (k 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$
            (f2cl-lib:array-slice b-%data% single-float (kp 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
          (declare (ignore var-1 var-3))
          (when var-0 (setf nrhs var-0))
          (when var-2 (setf ldb$ var-2))
          (when var-4 (setf ldb$ var-4))))
   (setf k (f2cl-lib:int-sub k 1)))
  (t
   (cond
    ((< k n)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
          var-9 var-10)
         (sgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- one)
          (f2cl-lib:array-slice b-%data% single-float ((+ k 1) 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice a-%data% single-float ((+ k 1) k)
                                ((1 lda) (1 *)) a-%offset%)
          1 one
          (f2cl-lib:array-slice b-%data% single-float (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
       (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
       (when var-2 (setf nrhs var-2))
       (when var-5 (setf ldb$ var-5))
       (when var-8 (setf one var-8))
```

```
    (when var-10 (setf ldb$ var-10)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (sgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- one)
       (f2cl-lib:array-slice b-%data% single-float ((+ k 1) 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice a-%data% single-float
                             ((+ k 1) (f2cl-lib:int-sub k 1))
                             ((1 lda) (1 *)) a-%offset%)
       1 one
       (f2cl-lib:array-slice b-%data% single-float
                             ((+ k (f2cl-lib:int-sub 1)) 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
    (when var-2 (setf nrhs var-2))
    (when var-5 (setf ldb$ var-5))
    (when var-8 (setf one var-8))
    (when var-10 (setf ldb$ var-10)))))
(setf kp
        (f2cl-lib:int-sub
         (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
(if (/= kp k)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (sswap nrhs
         (f2cl-lib:array-slice b-%data% single-float (k 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice b-%data% single-float (kp 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-1 var-3))
      (when var-0 (setf nrhs var-0))
      (when var-2 (setf ldb$ var-2))
      (when var-4 (setf ldb$ var-4))))
(setf kp
        (f2cl-lib:int-sub
         (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                        ((1 *)) ipiv-%offset%)))
(if (/= kp (f2cl-lib:int-sub k 1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (sswap nrhs
         (f2cl-lib:array-slice b-%data% single-float
                               ((+ k (f2cl-lib:int-sub 1)) 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice b-%data% single-float (kp 1)
                               ((1 ldb$) (1 *)) b-%offset%)
```

```
                      ldb$)
                    (declare (ignore var-1 var-3))
                    (when var-0 (setf nrhs var-0))
                    (when var-2 (setf ldb$ var-2))
                    (when var-4 (setf ldb$ var-4))))
              (setf k (f2cl-lib:int-sub k 2))))
            (go label90)
           label100)))
        (go end_label)
       end_label
        (return (values uplo nil nrhs nil nil nil nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ssytrs_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil
                                              fortran-to-lisp::nrhs nil nil nil
                                              nil fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

59.28 stgsyl

59.28.1 Axiom unit tests

— stgsyl.input —

```
)set break resume
)sys rm -f stgsyl.output
```

```
)spool stgsyl.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.28.2 Axiom help page

— **stgsyl.help** —

```
==========================================================================
stgsyl examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine stgsyl (TRANS, IJOB, M, N, A, LDA, B, LDB, C, LDC, D, LDD,
           E, LDE, F, LDF, SCALE, DIF, WORK, LWORK, IWORK, INFO)
           STGSYL

Function/Subroutine Documentation
   subroutine stgsyl (character TRANS, integer IJOB, integer M, integer N,
       real, dimension( lda, * ) A, integer LDA, real, dimension( ldb, * ) B,
       integer LDB, real, dimension( ldc, * ) C, integer LDC, real, dimension(
       ldd, * ) D, integer LDD, real, dimension( lde, * ) E, integer LDE,
       real, dimension( ldf, * ) F, integer LDF, real SCALE, real DIF, real,
       dimension( * ) WORK, integer LWORK, integer, dimension( * ) IWORK,
       integer INFO)
       STGSYL

Purpose:
   STGSYL solves the generalized Sylvester equation:

                   A * R - L * B = scale * C                 (1)
                   D * R - L * E = scale * F

   where R and L are unknown m-by-n matrices, (A, D), (B, E) and
   (C, F) are given matrix pairs of size m-by-m, n-by-n and m-by-n,
   respectively, with real entries. (A, D) and (B, E) must be in
   generalized (real) Schur canonical form, i.e. A, B are upper quasi
```

```
  triangular and D, E are upper triangular.

  The solution (R, L) overwrites (C, F). 0 <= SCALE <= 1 is an output
  scaling factor chosen to avoid overflow.

  In matrix notation (1) is equivalent to solve  Zx = scale b, where
  Z is defined as

                 Z = [ kron(In, A)  -kron(B**T, Im) ]         (2)
                     [ kron(In, D)  -kron(E**T, Im) ].

  Here Ik is the identity matrix of size k and X**T is the transpose of
  X. kron(X, Y) is the Kronecker product between the matrices X and Y.

  If TRANS = 'T', STGSYL solves the transposed system Z**T*y = scale*b,
  which is equivalent to solve for R and L in

                  A**T * R + D**T * L = scale * C           (3)
                  R * B**T + L * E**T = scale * -F

  This case (TRANS = 'T') is used to compute an one-norm-based estimate
  of Dif[(A,D), (B,E)], the separation between the matrix pairs (A,D)
  and (B,E), using SLACON.

  If IJOB >= 1, STGSYL computes a Frobenius norm-based estimate
  of Dif[(A,D),(B,E)]. That is, the reciprocal of a lower bound on the
  reciprocal of the smallest singular value of Z. See [1-2] for more
  information.

  This is a level 3 BLAS algorithm.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   = 'N', solve the generalized Sylvester equation (1).
   = 'T', solve the 'transposed' system (3).

 IJOB
   IJOB is INTEGER
   Specifies what kind of functionality to be performed.
   =0: solve (1) only.
   =1: The functionality of 0 and 3.
   =2: The functionality of 0 and 4.
   =3: Only an estimate of Dif[(A,D), (B,E)] is computed.
       (look ahead strategy IJOB  = 1 is used).
   =4: Only an estimate of Dif[(A,D), (B,E)] is computed.
       ( SGECON on sub-systems is used ).
   Not referenced if TRANS = 'T'.
```

```
M
  M is INTEGER
  The order of the matrices A and D, and the row dimension of
  the matrices C, F, R and L.

N
  N is INTEGER
  The order of the matrices B and E, and the column dimension
  of the matrices C, F, R and L.

A
  A is REAL array, dimension (LDA, M)
  The upper quasi triangular matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A. LDA >= max(1, M).

B
  B is REAL array, dimension (LDB, N)
  The upper quasi triangular matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= max(1, N).

C
  C is REAL array, dimension (LDC, N)
  On entry, C contains the right-hand-side of the first matrix
  equation in (1) or (3).
  On exit, if IJOB = 0, 1 or 2, C has been overwritten by
  the solution R. If IJOB = 3 or 4 and TRANS = 'N', C holds R,
  the solution achieved during the computation of the Dif-estimate.

LDC
  LDC is INTEGER
  The leading dimension of the array C. LDC >= max(1, M).

D
  D is REAL array, dimension (LDD, M)
  The upper triangular matrix D.

LDD
  LDD is INTEGER
  The leading dimension of the array D. LDD >= max(1, M).

E
  E is REAL array, dimension (LDE, N)
  The upper triangular matrix E.
```

```
LDE
  LDE is INTEGER
  The leading dimension of the array E. LDE >= max(1, N).

F
  F is REAL array, dimension (LDF, N)
  On entry, F contains the right-hand-side of the second matrix
  equation in (1) or (3).
  On exit, if IJOB = 0, 1 or 2, F has been overwritten by
  the solution L. If IJOB = 3 or 4 and TRANS = 'N', F holds L,
  the solution achieved during the computation of the Dif-estimate.

LDF
  LDF is INTEGER
  The leading dimension of the array F. LDF >= max(1, M).

DIF
  DIF is REAL
  On exit DIF is the reciprocal of a lower bound of the
  reciprocal of the Dif-function, i.e. DIF is an upper bound of
  Dif[(A,D), (B,E)] = sigma_min(Z), where Z as in (2).
  IF IJOB = 0 or TRANS = 'T', DIF is not touched.

SCALE
  SCALE is REAL
  On exit SCALE is the scaling factor in (1) or (3).
  If 0 < SCALE < 1, C and F hold the solutions R and L, resp.,
  to a slightly perturbed system but the input matrices A, B, D
  and E have not been changed. If SCALE = 0, C and F hold the
  solutions R and L, respectively, to the homogeneous system
  with C = F = 0. Normally, SCALE = 1.

WORK
  WORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK > = 1.
  If IJOB = 1 or 2 and TRANS = 'N', LWORK >= max(1,2*M*N).

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (M+N+6)

INFO
```

```
INFO is INTEGER
=0: successful exit
<0: If INFO = -i, the i-th argument had an illegal value.
>0: (A, D) and (B, E) have common or close eigenvalues.
```

59.28.3 fortran code

— stgsyl.f —

```
*  =====================================================================
      SUBROUTINE STGSYL( TRANS, IJOB, M, N, A, LDA, B, LDB, C, LDC, D,
     $                   LDD, E, LDE, F, LDF, SCALE, DIF, WORK, LWORK,
     $                   IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            IJOB, INFO, LDA, LDB, LDC, LDD, LDE, LDF,
     $                   LWORK, M, N
      REAL               DIF, SCALE
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      REAL               A( LDA, * ), B( LDB, * ), C( LDC, * ),
     $                   D( LDD, * ), E( LDE, * ), F( LDF, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*  Replaced various illegal calls to SCOPY by calls to SLASET.
*  Sven Hammarling, 1/5/02.
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, NOTRAN
      INTEGER            I, IE, IFUNC, IROUND, IS, ISOLVE, J, JE, JS, K,
     $                   LINFO, LWMIN, MB, NB, P, PPQQ, PQ, Q
      REAL               DSCALE, DSUM, SCALE2, SCALOC
```

```
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEMM, SLACPY, SLASET, SSCAL, STGSY2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, REAL, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode and test input parameters
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
*
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) ) THEN
         INFO = -1
      ELSE IF( NOTRAN ) THEN
         IF( ( IJOB.LT.0 ) .OR. ( IJOB.GT.4 ) ) THEN
            INFO = -2
         END IF
      END IF
      IF( INFO.EQ.0 ) THEN
         IF( M.LE.0 ) THEN
            INFO = -3
         ELSE IF( N.LE.0 ) THEN
            INFO = -4
         ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
            INFO = -6
         ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
            INFO = -8
         ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
            INFO = -10
         ELSE IF( LDD.LT.MAX( 1, M ) ) THEN
            INFO = -12
         ELSE IF( LDE.LT.MAX( 1, N ) ) THEN
            INFO = -14
         ELSE IF( LDF.LT.MAX( 1, M ) ) THEN
            INFO = -16
         END IF
      END IF
*
      IF( INFO.EQ.0 ) THEN
         IF( NOTRAN ) THEN
            IF( IJOB.EQ.1 .OR. IJOB.EQ.2 ) THEN
```

```
            LWMIN = MAX( 1, 2*M*N )
         ELSE
            LWMIN = 1
         END IF
      ELSE
         LWMIN = 1
      END IF
      WORK( 1 ) = LWMIN
*
      IF( LWORK.LT.LWMIN .AND. .NOT.LQUERY ) THEN
         INFO = -20
      END IF
   END IF
*
   IF( INFO.NE.0 ) THEN
      CALL XERBLA( 'STGSYL', -INFO )
      RETURN
   ELSE IF( LQUERY ) THEN
      RETURN
   END IF
*
*     Quick return if possible
*
   IF( M.EQ.0 .OR. N.EQ.0 ) THEN
      SCALE = 1
      IF( NOTRAN ) THEN
         IF( IJOB.NE.0 ) THEN
            DIF = 0
         END IF
      END IF
      RETURN
   END IF
*
*     Determine optimal block sizes MB and NB
*
   MB = ILAENV( 2, 'STGSYL', TRANS, M, N, -1, -1 )
   NB = ILAENV( 5, 'STGSYL', TRANS, M, N, -1, -1 )
*
   ISOLVE = 1
   IFUNC = 0
   IF( NOTRAN ) THEN
      IF( IJOB.GE.3 ) THEN
         IFUNC = IJOB - 2
         CALL SLASET( 'F', M, N, ZERO, ZERO, C, LDC )
         CALL SLASET( 'F', M, N, ZERO, ZERO, F, LDF )
      ELSE IF( IJOB.GE.1 .AND. NOTRAN ) THEN
         ISOLVE = 2
      END IF
   END IF
*
```

```
      IF( ( MB.LE.1 .AND. NB.LE.1 ) .OR. ( MB.GE.M .AND. NB.GE.N ) )
     $     THEN
*
         DO 30 IROUND = 1, ISOLVE
*
*           Use unblocked Level 2 solver
*
            DSCALE = ZERO
            DSUM = ONE
            PQ = 0
            CALL STGSY2( TRANS, IFUNC, M, N, A, LDA, B, LDB, C, LDC, D,
     $                   LDD, E, LDE, F, LDF, SCALE, DSUM, DSCALE,
     $                   IWORK, PQ, INFO )
            IF( DSCALE.NE.ZERO ) THEN
               IF( IJOB.EQ.1 .OR. IJOB.EQ.3 ) THEN
                  DIF = SQRT( REAL( 2*M*N ) ) / ( DSCALE*SQRT( DSUM ) )
               ELSE
                  DIF = SQRT( REAL( PQ ) ) / ( DSCALE*SQRT( DSUM ) )
               END IF
            END IF
*
            IF( ISOLVE.EQ.2 .AND. IROUND.EQ.1 ) THEN
               IF( NOTRAN ) THEN
                  IFUNC = IJOB
               END IF
               SCALE2 = SCALE
               CALL SLACPY( 'F', M, N, C, LDC, WORK, M )
               CALL SLACPY( 'F', M, N, F, LDF, WORK( M*N+1 ), M )
               CALL SLASET( 'F', M, N, ZERO, ZERO, C, LDC )
               CALL SLASET( 'F', M, N, ZERO, ZERO, F, LDF )
            ELSE IF( ISOLVE.EQ.2 .AND. IROUND.EQ.2 ) THEN
               CALL SLACPY( 'F', M, N, WORK, M, C, LDC )
               CALL SLACPY( 'F', M, N, WORK( M*N+1 ), M, F, LDF )
               SCALE = SCALE2
            END IF
   30    CONTINUE
*
         RETURN
      END IF
*
*     Determine block structure of A
*
      P = 0
      I = 1
   40 CONTINUE
      IF( I.GT.M )
     $   GO TO 50
      P = P + 1
      IWORK( P ) = I
      I = I + MB
```

```
      IF( I.GE.M )
     $   GO TO 50
      IF( A( I, I-1 ).NE.ZERO )
     $   I = I + 1
      GO TO 40
   50 CONTINUE
*
      IWORK( P+1 ) = M + 1
      IF( IWORK( P ).EQ.IWORK( P+1 ) )
     $   P = P - 1
*
*     Determine block structure of B
*
      Q = P + 1
      J = 1
   60 CONTINUE
      IF( J.GT.N )
     $   GO TO 70
      Q = Q + 1
      IWORK( Q ) = J
      J = J + NB
      IF( J.GE.N )
     $   GO TO 70
      IF( B( J, J-1 ).NE.ZERO )
     $   J = J + 1
      GO TO 60
   70 CONTINUE
*
      IWORK( Q+1 ) = N + 1
      IF( IWORK( Q ).EQ.IWORK( Q+1 ) )
     $   Q = Q - 1
*
      IF( NOTRAN ) THEN
*
         DO 150 IROUND = 1, ISOLVE
*
*           Solve (I, J)-subsystem
*               A(I, I) * R(I, J) - L(I, J) * B(J, J) = C(I, J)
*               D(I, I) * R(I, J) - L(I, J) * E(J, J) = F(I, J)
*           for I = P, P - 1,..., 1; J = 1, 2,..., Q
*
            DSCALE = ZERO
            DSUM = ONE
            PQ = 0
            SCALE = ONE
            DO 130 J = P + 2, Q
               JS = IWORK( J )
               JE = IWORK( J+1 ) - 1
               NB = JE - JS + 1
               DO 120 I = P, 1, -1
```

```
                 IS = IWORK( I )
                 IE = IWORK( I+1 ) - 1
                 MB = IE - IS + 1
                 PPQQ = 0
                 CALL STGSY2( TRANS, IFUNC, MB, NB, A( IS, IS ), LDA,
     $                        B( JS, JS ), LDB, C( IS, JS ), LDC,
     $                        D( IS, IS ), LDD, E( JS, JS ), LDE,
     $                        F( IS, JS ), LDF, SCALOC, DSUM, DSCALE,
     $                        IWORK( Q+2 ), PPQQ, LINFO )
                 IF( LINFO.GT.0 )
     $              INFO = LINFO
*
                 PQ = PQ + PPQQ
                 IF( SCALOC.NE.ONE ) THEN
                    DO 80 K = 1, JS - 1
                       CALL SSCAL( M, SCALOC, C( 1, K ), 1 )
                       CALL SSCAL( M, SCALOC, F( 1, K ), 1 )
   80               CONTINUE
                    DO 90 K = JS, JE
                       CALL SSCAL( IS-1, SCALOC, C( 1, K ), 1 )
                       CALL SSCAL( IS-1, SCALOC, F( 1, K ), 1 )
   90               CONTINUE
                    DO 100 K = JS, JE
                       CALL SSCAL( M-IE, SCALOC, C( IE+1, K ), 1 )
                       CALL SSCAL( M-IE, SCALOC, F( IE+1, K ), 1 )
  100               CONTINUE
                    DO 110 K = JE + 1, N
                       CALL SSCAL( M, SCALOC, C( 1, K ), 1 )
                       CALL SSCAL( M, SCALOC, F( 1, K ), 1 )
  110               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
*
*                Substitute R(I, J) and L(I, J) into remaining
*                equation.
*
                 IF( I.GT.1 ) THEN
                    CALL SGEMM( 'N', 'N', IS-1, NB, MB, -ONE,
     $                          A( 1, IS ), LDA, C( IS, JS ), LDC, ONE,
     $                          C( 1, JS ), LDC )
                    CALL SGEMM( 'N', 'N', IS-1, NB, MB, -ONE,
     $                          D( 1, IS ), LDD, C( IS, JS ), LDC, ONE,
     $                          F( 1, JS ), LDF )
                 END IF
                 IF( J.LT.Q ) THEN
                    CALL SGEMM( 'N', 'N', MB, N-JE, NB, ONE,
     $                          F( IS, JS ), LDF, B( JS, JE+1 ), LDB,
     $                          ONE, C( IS, JE+1 ), LDC )
                    CALL SGEMM( 'N', 'N', MB, N-JE, NB, ONE,
     $                          F( IS, JS ), LDF, E( JS, JE+1 ), LDE,
```

```
     $                          ONE, F( IS, JE+1 ), LDF )
                   END IF
  120           CONTINUE
  130        CONTINUE
             IF( DSCALE.NE.ZERO ) THEN
                IF( IJOB.EQ.1 .OR. IJOB.EQ.3 ) THEN
                   DIF = SQRT( REAL( 2*M*N ) ) / ( DSCALE*SQRT( DSUM ) )
                ELSE
                   DIF = SQRT( REAL( PQ ) ) / ( DSCALE*SQRT( DSUM ) )
                END IF
             END IF
             IF( ISOLVE.EQ.2 .AND. IROUND.EQ.1 ) THEN
                IF( NOTRAN ) THEN
                   IFUNC = IJOB
                END IF
                SCALE2 = SCALE
                CALL SLACPY( 'F', M, N, C, LDC, WORK, M )
                CALL SLACPY( 'F', M, N, F, LDF, WORK( M*N+1 ), M )
                CALL SLASET( 'F', M, N, ZERO, ZERO, C, LDC )
                CALL SLASET( 'F', M, N, ZERO, ZERO, F, LDF )
             ELSE IF( ISOLVE.EQ.2 .AND. IROUND.EQ.2 ) THEN
                CALL SLACPY( 'F', M, N, WORK, M, C, LDC )
                CALL SLACPY( 'F', M, N, WORK( M*N+1 ), M, F, LDF )
                SCALE = SCALE2
             END IF
  150     CONTINUE
*
      ELSE
*
*        Solve transposed (I, J)-subsystem
*             A(I, I)**T * R(I, J)  + D(I, I)**T * L(I, J)  =  C(I, J)
*             R(I, J)  * B(J, J)**T + L(I, J)  * E(J, J)**T = -F(I, J)
*        for I = 1,2,..., P; J = Q, Q-1,..., 1
*
         SCALE = ONE
         DO 210 I = 1, P
            IS = IWORK( I )
            IE = IWORK( I+1 ) - 1
            MB = IE - IS + 1
            DO 200 J = Q, P + 2, -1
               JS = IWORK( J )
               JE = IWORK( J+1 ) - 1
               NB = JE - JS + 1
               CALL STGSY2( TRANS, IFUNC, MB, NB, A( IS, IS ), LDA,
     $                      B( JS, JS ), LDB, C( IS, JS ), LDC,
     $                      D( IS, IS ), LDD, E( JS, JS ), LDE,
     $                      F( IS, JS ), LDF, SCALOC, DSUM, DSCALE,
     $                      IWORK( Q+2 ), PPQQ, LINFO )
               IF( LINFO.GT.0 )
     $            INFO = LINFO
```

```
               IF( SCALOC.NE.ONE ) THEN
                  DO 160 K = 1, JS - 1
                     CALL SSCAL( M, SCALOC, C( 1, K ), 1 )
                     CALL SSCAL( M, SCALOC, F( 1, K ), 1 )
  160             CONTINUE
                  DO 170 K = JS, JE
                     CALL SSCAL( IS-1, SCALOC, C( 1, K ), 1 )
                     CALL SSCAL( IS-1, SCALOC, F( 1, K ), 1 )
  170             CONTINUE
                  DO 180 K = JS, JE
                     CALL SSCAL( M-IE, SCALOC, C( IE+1, K ), 1 )
                     CALL SSCAL( M-IE, SCALOC, F( IE+1, K ), 1 )
  180             CONTINUE
                  DO 190 K = JE + 1, N
                     CALL SSCAL( M, SCALOC, C( 1, K ), 1 )
                     CALL SSCAL( M, SCALOC, F( 1, K ), 1 )
  190             CONTINUE
                  SCALE = SCALE*SCALOC
               END IF
*
*              Substitute R(I, J) and L(I, J) into remaining equation.
*
               IF( J.GT.P+2 ) THEN
                  CALL SGEMM( 'N', 'T', MB, JS-1, NB, ONE, C( IS, JS ),
     $                        LDC, B( 1, JS ), LDB, ONE, F( IS, 1 ),
     $                        LDF )
                  CALL SGEMM( 'N', 'T', MB, JS-1, NB, ONE, F( IS, JS ),
     $                        LDF, E( 1, JS ), LDE, ONE, F( IS, 1 ),
     $                        LDF )
               END IF
               IF( I.LT.P ) THEN
                  CALL SGEMM( 'T', 'N', M-IE, NB, MB, -ONE,
     $                        A( IS, IE+1 ), LDA, C( IS, JS ), LDC, ONE,
     $                        C( IE+1, JS ), LDC )
                  CALL SGEMM( 'T', 'N', M-IE, NB, MB, -ONE,
     $                        D( IS, IE+1 ), LDD, F( IS, JS ), LDF, ONE,
     $                        C( IE+1, JS ), LDC )
               END IF
  200       CONTINUE
  210    CONTINUE
*
      END IF
*
      WORK( 1 ) = LWMIN
*
      RETURN
*
*     End of STGSYL
*
      END
```

———-

59.28.4 lisp code

— **stgsyl.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun stgsyl
         (trans ijob m n a lda b ldb$ c ldc d ldd e lde f ldf scale dif work
          lwork iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (single-float) dif scale)
             (type (array single-float (*)) work f e d c b a)
             (type (f2cl-lib:integer4) info lwork ldf lde ldd ldc ldb$ lda n m
              ijob)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%)
         (c single-float c-%data% c-%offset%)
         (d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%)
         (f single-float f-%data% f-%offset%)
         (work single-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((dscale 0.0) (dsum 0.0) (scale2 0.0) (scaloc 0.0) (i 0) (ie 0)
             (ifunc 0) (iround 0) (is 0) (isolve 0) (j 0) (je 0) (js 0) (k 0)
             (linfo 0) (lwmin 0) (mb 0) (nb 0) (p 0) (ppqq 0) (pq 0) (q 0)
             (lquery nil) (notran nil))
        (declare (type (single-float) dscale dsum scale2 scaloc)
                 (type (f2cl-lib:integer4) i ie ifunc iround is isolve j je js
                  k linfo lwmin mb nb p ppqq pq q)
                 (type f2cl-lib:logical lquery notran))
        (setf info 0)
        (setf notran
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame trans "N")
                  (declare (ignore var-1))
                  (when var-0 (setf trans var-0))
                  ret-val))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond
         ((and (not notran)
               (not
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -1))
 (notran (cond ((or (< ijob 0) (> ijob 4)) (setf info -2)))))
(cond
 ((= info 0)
  (cond ((<= m 0) (setf info -3)) ((<= n 0) (setf info -4))
        ((< lda
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -6))
        ((< ldb$
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -8))
        ((< ldc
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -10))
        ((< ldd
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -12))
        ((< lde
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -14))
        ((< ldf
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -16)))))
(cond
 ((= info 0)
  (cond
   (notran
    (cond
     ((or (= ijob 1) (= ijob 2))
      (setf lwmin
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 (f2cl-lib:int-mul 2 m n)))))
     (t (setf lwmin 1))))
   (t (setf lwmin 1)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwmin) 'single-float))
  (cond ((and (< lwork lwmin) (not lquery)) (setf info -20)))))
(cond
 ((/= info 0) (xerbla "STGSYL" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond
 ((or (= m 0) (= n 0))
  (setf scale (coerce (the f2cl-lib:integer4 1) 'single-float))
  (cond
   (notran
```

```
    (cond
     ((/= ijob 0)
      (setf dif (coerce (the f2cl-lib:integer4 0) 'single-float))))))
  (go end_label)))
(setf mb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 2 "STGSYL" trans m n -1 -1)
          (declare (ignore var-0 var-1 var-5 var-6))
          (setf trans var-2)
          (setf m var-3)
          (setf n var-4)
          ret-val))
(setf nb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 5 "STGSYL" trans m n -1 -1)
          (declare (ignore var-0 var-1 var-5 var-6))
          (setf trans var-2)
          (setf m var-3)
          (setf n var-4)
          ret-val))
(setf isolve 1)
(setf ifunc 0)
(cond
 (notran
  (cond
   ((>= ijob 3) (setf ifunc (f2cl-lib:int-sub ijob 2))
    (slaset "F" m n zero zero c ldc) (slaset "F" m n zero zero f ldf))
   ((and (>= ijob 1) notran) (setf isolve 2)))))
(cond
 ((or (and (<= mb 1) (<= nb 1)) (and (>= mb m) (>= nb n)))
  (f2cl-lib:fdo (iround 1 (f2cl-lib:int-add iround 1))
                ((> iround isolve) nil)
    (tagbody
      (setf dscale zero)
      (setf dsum one)
      (setf pq 0)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17
           var-18 var-19 var-20 var-21)
          (stgsy2 trans ifunc m n a lda b ldb$ c ldc d ldd e lde f ldf
           scale dsum dscale iwork pq info)
        (declare
         (ignore var-1 var-3 var-4 var-6 var-8 var-10 var-12 var-14
          var-19))
        (setf trans var-0)
        (setf m var-2)
        (setf lda var-5)
```

```
          (setf ldb$ var-7)
          (setf ldc var-9)
          (setf ldd var-11)
          (setf lde var-13)
          (setf ldf var-15)
          (setf scale var-16)
          (setf dsum var-17)
          (setf dscale var-18)
          (setf pq var-20)
          (setf info var-21))
        (cond
         ((/= dscale zero)
          (cond
           ((or (= ijob 1) (= ijob 3))
            (setf dif
                    (/
                     (f2cl-lib:fsqrt
                      (f2cl-lib:freal (f2cl-lib:int-mul 2 m n)))
                     (* dscale (f2cl-lib:fsqrt dsum)))))
           (t
            (setf dif
                    (/ (f2cl-lib:fsqrt (f2cl-lib:freal pq))
                       (* dscale (f2cl-lib:fsqrt dsum))))))))
        (cond
         ((and (= isolve 2) (= iround 1))
          (cond (notran (setf ifunc ijob))) (setf scale2 scale)
          (slacpy "F" m n c ldc work m)
          (slacpy "F" m n f ldf
           (f2cl-lib:array-slice work-%data% single-float
                                 ((+ (f2cl-lib:int-mul m n) 1)) ((1 *))
                                 work-%offset%)
           m)
          (slaset "F" m n zero zero c ldc)
          (slaset "F" m n zero zero f ldf))
         ((and (= isolve 2) (= iround 2)) (slacpy "F" m n work m c ldc)
          (slacpy "F" m n
           (f2cl-lib:array-slice work-%data% single-float
                                 ((+ (f2cl-lib:int-mul m n) 1)) ((1 *))
                                 work-%offset%)
           m f ldf)
          (setf scale scale2)))
      label30))
    (go end_label)))
  (setf p 0)
  (setf i 1)
 label40
  (if (> i m)
      (go label50))
  (setf p (f2cl-lib:int-add p 1))
  (setf (f2cl-lib:fref iwork-%data% (p) ((1 *)) iwork-%offset%) i)
```

```
 (setf i (f2cl-lib:int-add i mb))
 (if (>= i m)
     (go label50))
 (if (/=
      (f2cl-lib:fref a-%data% (i (f2cl-lib:int-sub i 1)) ((1 lda) (1 *))
                     a-%offset%)
      zero)
     (setf i (f2cl-lib:int-add i 1)))
 (go label40)
label50
 (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add p 1)) ((1 *))
                      iwork-%offset%)
         (f2cl-lib:int-add m 1))
 (if (= (f2cl-lib:fref iwork-%data% (p) ((1 *)) iwork-%offset%)
        (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add p 1)) ((1 *))
                       iwork-%offset%))
     (setf p (f2cl-lib:int-sub p 1)))
 (setf q (f2cl-lib:int-add p 1))
 (setf j 1)
label60
 (if (> j n)
     (go label70))
 (setf q (f2cl-lib:int-add q 1))
 (setf (f2cl-lib:fref iwork-%data% (q) ((1 *)) iwork-%offset%) j)
 (setf j (f2cl-lib:int-add j nb))
 (if (>= j n)
     (go label70))
 (if (/=
      (f2cl-lib:fref b-%data% (j (f2cl-lib:int-sub j 1))
                     ((1 ldb$) (1 *)) b-%offset%)
      zero)
     (setf j (f2cl-lib:int-add j 1)))
 (go label60)
label70
 (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add q 1)) ((1 *))
                      iwork-%offset%)
         (f2cl-lib:int-add n 1))
 (if (= (f2cl-lib:fref iwork-%data% (q) ((1 *)) iwork-%offset%)
        (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add q 1)) ((1 *))
                       iwork-%offset%))
     (setf q (f2cl-lib:int-sub q 1)))
 (cond
  (notran
   (f2cl-lib:fdo (iround 1 (f2cl-lib:int-add iround 1))
                 ((> iround isolve) nil)
     (tagbody
       (setf dscale zero)
       (setf dsum one)
       (setf pq 0)
       (setf scale one)
```

```
(f2cl-lib:fdo (j (f2cl-lib:int-add p 2) (f2cl-lib:int-add j 1))
              ((> j q) nil)
  (tagbody
    (setf js
            (f2cl-lib:fref iwork-%data% (j) ((1 *))
                           iwork-%offset%))
    (setf je
            (f2cl-lib:int-sub
             (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add j 1))
                            ((1 *)) iwork-%offset%)
             1))
    (setf nb (f2cl-lib:int-add (f2cl-lib:int-sub je js) 1))
    (f2cl-lib:fdo (i p (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
      (tagbody
        (setf is
                (f2cl-lib:fref iwork-%data% (i) ((1 *))
                               iwork-%offset%))
        (setf ie
                (f2cl-lib:int-sub
                 (f2cl-lib:fref iwork-%data%
                                ((f2cl-lib:int-add i 1)) ((1 *))
                                iwork-%offset%)
                 1))
        (setf mb (f2cl-lib:int-add (f2cl-lib:int-sub ie is) 1))
        (setf ppqq 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
             var-8 var-9 var-10 var-11 var-12 var-13 var-14
             var-15 var-16 var-17 var-18 var-19 var-20 var-21)
            (stgsy2 trans ifunc mb nb
             (f2cl-lib:array-slice a-%data% single-float (is is)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice b-%data% single-float (js js)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$
             (f2cl-lib:array-slice c-%data% single-float (is js)
                                   ((1 ldc) (1 *)) c-%offset%)
             ldc
             (f2cl-lib:array-slice d-%data% single-float (is is)
                                   ((1 ldd) (1 *)) d-%offset%)
             ldd
             (f2cl-lib:array-slice e-%data% single-float (js js)
                                   ((1 lde) (1 *)) e-%offset%)
             lde
             (f2cl-lib:array-slice f-%data% single-float (is js)
                                   ((1 ldf) (1 *)) f-%offset%)
             ldf scaloc dsum dscale
             (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
```

```
                                ((+ q 2)) ((1 *))
                                iwork-%offset%)
      ppqq linfo)
   (declare
    (ignore var-1 var-3 var-4 var-6 var-8 var-10 var-12
     var-14 var-19))
   (setf trans var-0)
   (setf mb var-2)
   (setf lda var-5)
   (setf ldb$ var-7)
   (setf ldc var-9)
   (setf ldd var-11)
   (setf lde var-13)
   (setf ldf var-15)
   (setf scaloc var-16)
   (setf dsum var-17)
   (setf dscale var-18)
   (setf ppqq var-20)
   (setf linfo var-21))
 (if (> linfo 0)
     (setf info linfo))
 (setf pq (f2cl-lib:int-add pq ppqq))
 (cond
  ((/= scaloc one)
   (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                 ((> k
                     (f2cl-lib:int-add js
                                       (f2cl-lib:int-sub
                                        1)))
                  nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2 var-3)
           (sscal m scaloc
            (f2cl-lib:array-slice c-%data% single-float
                                  (1 k) ((1 ldc) (1 *))
                                  c-%offset%)
            1)
         (declare (ignore var-2 var-3))
         (when var-0 (setf m var-0))
         (when var-1 (setf scaloc var-1)))
       (multiple-value-bind (var-0 var-1 var-2 var-3)
           (sscal m scaloc
            (f2cl-lib:array-slice f-%data% single-float
                                  (1 k) ((1 ldf) (1 *))
                                  f-%offset%)
            1)
         (declare (ignore var-2 var-3))
         (when var-0 (setf m var-0))
         (when var-1 (setf scaloc var-1)))
      label80))
```

```
(f2cl-lib:fdo (k js (f2cl-lib:int-add k 1))
              ((> k je) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal (f2cl-lib:int-sub is 1) scaloc
         (f2cl-lib:array-slice c-%data% single-float
                               (1 k) ((1 ldc) (1 *))
                               c-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3))
      (when var-1 (setf scaloc var-1)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal (f2cl-lib:int-sub is 1) scaloc
         (f2cl-lib:array-slice f-%data% single-float
                               (1 k) ((1 ldf) (1 *))
                               f-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3))
      (when var-1 (setf scaloc var-1)))
   label90))
(f2cl-lib:fdo (k js (f2cl-lib:int-add k 1))
              ((> k je) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal (f2cl-lib:int-sub m ie) scaloc
         (f2cl-lib:array-slice c-%data% single-float
                               ((+ ie 1) k)
                               ((1 ldc) (1 *))
                               c-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3))
      (when var-1 (setf scaloc var-1)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal (f2cl-lib:int-sub m ie) scaloc
         (f2cl-lib:array-slice f-%data% single-float
                               ((+ ie 1) k)
                               ((1 ldf) (1 *))
                               f-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3))
      (when var-1 (setf scaloc var-1)))
   label100))
(f2cl-lib:fdo (k (f2cl-lib:int-add je 1)
               (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal m scaloc
         (f2cl-lib:array-slice c-%data% single-float
                               (1 k) ((1 ldc) (1 *))
```

```
                                    c-%offset%)
                 1)
               (declare (ignore var-2 var-3))
               (when var-0 (setf m var-0))
               (when var-1 (setf scaloc var-1)))
             (multiple-value-bind (var-0 var-1 var-2 var-3)
                 (sscal m scaloc
                  (f2cl-lib:array-slice f-%data% single-float
                                        (1 k) ((1 ldf) (1 *))
                                        f-%offset%)
                 1)
               (declare (ignore var-2 var-3))
               (when var-0 (setf m var-0))
               (when var-1 (setf scaloc var-1)))
           label110))
         (setf scale (* scale scaloc))))
       (cond
        ((> i 1)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
              var-8 var-9 var-10 var-11 var-12)
             (sgemm "N" "N" (f2cl-lib:int-sub is 1) nb mb
              (- one)
              (f2cl-lib:array-slice a-%data% single-float (1 is)
                                    ((1 lda) (1 *)) a-%offset%)
              lda
              (f2cl-lib:array-slice c-%data% single-float
                                    (is js) ((1 ldc) (1 *))
                                    c-%offset%)
              ldc one
              (f2cl-lib:array-slice c-%data% single-float (1 js)
                                    ((1 ldc) (1 *)) c-%offset%)
              ldc)
           (declare
            (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
           (when var-3 (setf nb var-3))
           (when var-4 (setf mb var-4))
           (when var-7 (setf lda var-7))
           (when var-9 (setf ldc var-9))
           (when var-10 (setf one var-10))
           (when var-12 (setf ldc var-12)))
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
              var-8 var-9 var-10 var-11 var-12)
             (sgemm "N" "N" (f2cl-lib:int-sub is 1) nb mb
              (- one)
              (f2cl-lib:array-slice d-%data% single-float (1 is)
                                    ((1 ldd) (1 *)) d-%offset%)
              ldd
              (f2cl-lib:array-slice c-%data% single-float
```

```
                                (is js) ((1 ldc) (1 *))
                                c-%offset%)
          ldc one
          (f2cl-lib:array-slice f-%data% single-float (1 js)
                                ((1 ldf) (1 *)) f-%offset%)
          ldf)
        (declare
         (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
        (when var-3 (setf nb var-3))
        (when var-4 (setf mb var-4))
        (when var-7 (setf ldd var-7))
        (when var-9 (setf ldc var-9))
        (when var-10 (setf one var-10))
        (when var-12 (setf ldf var-12)))))
    (cond
     ((< j q)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
           var-8 var-9 var-10 var-11 var-12)
          (sgemm "N" "N" mb (f2cl-lib:int-sub n je) nb one
           (f2cl-lib:array-slice f-%data% single-float
                                 (is js) ((1 ldf) (1 *))
                                 f-%offset%)
           ldf
           (f2cl-lib:array-slice b-%data% single-float
                                 (js (f2cl-lib:int-add je 1))
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$ one
           (f2cl-lib:array-slice c-%data% single-float
                                 (is (f2cl-lib:int-add je 1))
                                 ((1 ldc) (1 *)) c-%offset%)
           ldc)
        (declare
         (ignore var-0 var-1 var-3 var-6 var-8 var-11))
        (when var-2 (setf mb var-2))
        (when var-4 (setf nb var-4))
        (when var-5 (setf one var-5))
        (when var-7 (setf ldf var-7))
        (when var-9 (setf ldb$ var-9))
        (when var-10 (setf one var-10))
        (when var-12 (setf ldc var-12)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
           var-8 var-9 var-10 var-11 var-12)
          (sgemm "N" "N" mb (f2cl-lib:int-sub n je) nb one
           (f2cl-lib:array-slice f-%data% single-float
                                 (is js) ((1 ldf) (1 *))
                                 f-%offset%)
           ldf
           (f2cl-lib:array-slice e-%data% single-float
```

```
                                        (js (f2cl-lib:int-add je 1))
                                        ((1 lde) (1 *)) e-%offset%)
                    lde one
                    (f2cl-lib:array-slice f-%data% single-float
                                        (is (f2cl-lib:int-add je 1))
                                        ((1 ldf) (1 *)) f-%offset%)
                    ldf)
                 (declare
                  (ignore var-0 var-1 var-3 var-6 var-8 var-11))
                 (when var-2 (setf mb var-2))
                 (when var-4 (setf nb var-4))
                 (when var-5 (setf one var-5))
                 (when var-7 (setf ldf var-7))
                 (when var-9 (setf lde var-9))
                 (when var-10 (setf one var-10))
                 (when var-12 (setf ldf var-12)))))
            label120))
        label130))
     (cond
      ((/= dscale zero)
       (cond
        ((or (= ijob 1) (= ijob 3))
         (setf dif
                 (/
                  (f2cl-lib:fsqrt
                   (f2cl-lib:freal (f2cl-lib:int-mul 2 m n)))
                  (* dscale (f2cl-lib:fsqrt dsum)))))
        (t
         (setf dif
                 (/ (f2cl-lib:fsqrt (f2cl-lib:freal pq))
                    (* dscale (f2cl-lib:fsqrt dsum))))))))
     (cond
      ((and (= isolve 2) (= iround 1))
       (cond (notran (setf ifunc ijob))) (setf scale2 scale)
       (slacpy "F" m n c ldc work m)
       (slacpy "F" m n f ldf
        (f2cl-lib:array-slice work-%data% single-float
                              ((+ (f2cl-lib:int-mul m n) 1)) ((1 *))
                              work-%offset%)
        m)
       (slaset "F" m n zero zero c ldc)
       (slaset "F" m n zero zero f ldf))
      ((and (= isolve 2) (= iround 2)) (slacpy "F" m n work m c ldc)
       (slacpy "F" m n
        (f2cl-lib:array-slice work-%data% single-float
                              ((+ (f2cl-lib:int-mul m n) 1)) ((1 *))
                              work-%offset%)
        m f ldf)
       (setf scale scale2)))
    label150)))
```

```
(t (setf scale one)
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i p) nil)
   (tagbody
     (setf is (f2cl-lib:fref iwork-%data% (i) ((1 *)) iwork-%offset%))
     (setf ie
             (f2cl-lib:int-sub
              (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add i 1))
                             ((1 *)) iwork-%offset%)
              1))
     (setf mb (f2cl-lib:int-add (f2cl-lib:int-sub ie is) 1))
     (f2cl-lib:fdo (j q (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   ((> j (f2cl-lib:int-add p 2)) nil)
       (tagbody
         (setf js
                 (f2cl-lib:fref iwork-%data% (j) ((1 *))
                                iwork-%offset%))
         (setf je
                 (f2cl-lib:int-sub
                  (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add j 1))
                                 ((1 *)) iwork-%offset%)
                  1))
         (setf nb (f2cl-lib:int-add (f2cl-lib:int-sub je js) 1))
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
              var-17 var-18 var-19 var-20 var-21)
             (stgsy2 trans ifunc mb nb
              (f2cl-lib:array-slice a-%data% single-float (is is)
                                    ((1 lda) (1 *)) a-%offset%)
              lda
              (f2cl-lib:array-slice b-%data% single-float (js js)
                                    ((1 ldb$) (1 *)) b-%offset%)
              ldb$
              (f2cl-lib:array-slice c-%data% single-float (is js)
                                    ((1 ldc) (1 *)) c-%offset%)
              ldc
              (f2cl-lib:array-slice d-%data% single-float (is is)
                                    ((1 ldd) (1 *)) d-%offset%)
              ldd
              (f2cl-lib:array-slice e-%data% single-float (js js)
                                    ((1 lde) (1 *)) e-%offset%)
              lde
              (f2cl-lib:array-slice f-%data% single-float (is js)
                                    ((1 ldf) (1 *)) f-%offset%)
              ldf scaloc dsum dscale
              (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                                    ((+ q 2)) ((1 *)) iwork-%offset%)
              ppqq linfo)
           (declare
```

```
   (ignore var-1 var-3 var-4 var-6 var-8 var-10 var-12 var-14
    var-19))
  (setf trans var-0)
  (setf mb var-2)
  (setf lda var-5)
  (setf ldb$ var-7)
  (setf ldc var-9)
  (setf ldd var-11)
  (setf lde var-13)
  (setf ldf var-15)
  (setf scaloc var-16)
  (setf dsum var-17)
  (setf dscale var-18)
  (setf ppqq var-20)
  (setf linfo var-21))
(if (> linfo 0)
    (setf info linfo))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k
                    (f2cl-lib:int-add js
                                      (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal m scaloc
           (f2cl-lib:array-slice c-%data% single-float (1 k)
                                 ((1 ldc) (1 *)) c-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal m scaloc
           (f2cl-lib:array-slice f-%data% single-float (1 k)
                                 ((1 ldf) (1 *)) f-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
     label160))
  (f2cl-lib:fdo (k js (f2cl-lib:int-add k 1))
                ((> k je) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal (f2cl-lib:int-sub is 1) scaloc
           (f2cl-lib:array-slice c-%data% single-float (1 k)
                                 ((1 ldc) (1 *)) c-%offset%)
           1)
```

```
        (declare (ignore var-0 var-2 var-3))
        (when var-1 (setf scaloc var-1)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal (f2cl-lib:int-sub is 1) scaloc
           (f2cl-lib:array-slice f-%data% single-float (1 k)
                                 ((1 ldf) (1 *)) f-%offset%)
           1)
        (declare (ignore var-0 var-2 var-3))
        (when var-1 (setf scaloc var-1)))
    label170))
(f2cl-lib:fdo (k js (f2cl-lib:int-add k 1))
              ((> k je) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal (f2cl-lib:int-sub m ie) scaloc
         (f2cl-lib:array-slice c-%data% single-float
                               ((+ ie 1) k) ((1 ldc) (1 *))
                               c-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3))
      (when var-1 (setf scaloc var-1)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal (f2cl-lib:int-sub m ie) scaloc
         (f2cl-lib:array-slice f-%data% single-float
                               ((+ ie 1) k) ((1 ldf) (1 *))
                               f-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3))
      (when var-1 (setf scaloc var-1)))
    label180))
(f2cl-lib:fdo (k (f2cl-lib:int-add je 1)
               (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal m scaloc
         (f2cl-lib:array-slice c-%data% single-float (1 k)
                               ((1 ldc) (1 *)) c-%offset%)
         1)
      (declare (ignore var-2 var-3))
      (when var-0 (setf m var-0))
      (when var-1 (setf scaloc var-1)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (sscal m scaloc
         (f2cl-lib:array-slice f-%data% single-float (1 k)
                               ((1 ldf) (1 *)) f-%offset%)
         1)
      (declare (ignore var-2 var-3))
      (when var-0 (setf m var-0))
      (when var-1 (setf scaloc var-1)))
```

```
      label190))
    (setf scale (* scale scaloc))))
  (cond
   ((> j (f2cl-lib:int-add p 2))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (sgemm "N" "T" mb (f2cl-lib:int-sub js 1) nb one
         (f2cl-lib:array-slice c-%data% single-float (is js)
                               ((1 ldc) (1 *)) c-%offset%)
         ldc
         (f2cl-lib:array-slice b-%data% single-float (1 js)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$ one
         (f2cl-lib:array-slice f-%data% single-float (is 1)
                               ((1 ldf) (1 *)) f-%offset%)
         ldf)
      (declare (ignore var-0 var-1 var-3 var-6 var-8 var-11))
      (when var-2 (setf mb var-2))
      (when var-4 (setf nb var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf ldc var-7))
      (when var-9 (setf ldb$ var-9))
      (when var-10 (setf one var-10))
      (when var-12 (setf ldf var-12)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (sgemm "N" "T" mb (f2cl-lib:int-sub js 1) nb one
         (f2cl-lib:array-slice f-%data% single-float (is js)
                               ((1 ldf) (1 *)) f-%offset%)
         ldf
         (f2cl-lib:array-slice e-%data% single-float (1 js)
                               ((1 lde) (1 *)) e-%offset%)
         lde one
         (f2cl-lib:array-slice f-%data% single-float (is 1)
                               ((1 ldf) (1 *)) f-%offset%)
         ldf)
      (declare (ignore var-0 var-1 var-3 var-6 var-8 var-11))
      (when var-2 (setf mb var-2))
      (when var-4 (setf nb var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf ldf var-7))
      (when var-9 (setf lde var-9))
      (when var-10 (setf one var-10))
      (when var-12 (setf ldf var-12)))))
  (cond
   ((< i p)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
                    var-9 var-10 var-11 var-12)
                   (sgemm "T" "N" (f2cl-lib:int-sub m ie) nb mb (- one)
                    (f2cl-lib:array-slice a-%data% single-float
                                          (is (f2cl-lib:int-add ie 1))
                                          ((1 lda) (1 *)) a-%offset%)
                    lda
                    (f2cl-lib:array-slice c-%data% single-float (is js)
                                          ((1 ldc) (1 *)) c-%offset%)
                    ldc one
                    (f2cl-lib:array-slice c-%data% single-float
                                          ((+ ie 1) js) ((1 ldc) (1 *))
                                          c-%offset%)
                    ldc)
                  (declare
                   (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
                  (when var-3 (setf nb var-3))
                  (when var-4 (setf mb var-4))
                  (when var-7 (setf lda var-7))
                  (when var-9 (setf ldc var-9))
                  (when var-10 (setf one var-10))
                  (when var-12 (setf ldc var-12)))
                (multiple-value-bind
                   (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                    var-9 var-10 var-11 var-12)
                   (sgemm "T" "N" (f2cl-lib:int-sub m ie) nb mb (- one)
                    (f2cl-lib:array-slice d-%data% single-float
                                          (is (f2cl-lib:int-add ie 1))
                                          ((1 ldd) (1 *)) d-%offset%)
                    ldd
                    (f2cl-lib:array-slice f-%data% single-float (is js)
                                          ((1 ldf) (1 *)) f-%offset%)
                    ldf one
                    (f2cl-lib:array-slice c-%data% single-float
                                          ((+ ie 1) js) ((1 ldc) (1 *))
                                          c-%offset%)
                    ldc)
                  (declare
                   (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
                  (when var-3 (setf nb var-3))
                  (when var-4 (setf mb var-4))
                  (when var-7 (setf ldd var-7))
                  (when var-9 (setf ldf var-9))
                  (when var-10 (setf one var-10))
                  (when var-12 (setf ldc var-12)))))
            label200))
       label210))))
 (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce (the f2cl-lib:integer4 lwmin) 'single-float))
 (go end_label)
end_label
```

```
        (return
         (values trans nil m n nil lda nil ldb$ nil ldc nil ldd nil lde nil ldf
                 scale dif nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::stgsyl
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (single-float) (single-float)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::integer4
                                              (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(fortran-to-lisp::trans nil
                                             fortran-to-lisp::m
                                             fortran-to-lisp::n nil
                                             fortran-to-lisp::lda nil
                                             fortran-to-lisp::ldb$ nil
                                             fortran-to-lisp::ldc nil
                                             fortran-to-lisp::ldd nil
                                             fortran-to-lisp::lde nil
                                             fortran-to-lisp::ldf
                                             fortran-to-lisp::scale
                                             fortran-to-lisp::dif nil nil nil
                                             fortran-to-lisp::info)
                                           :calls
                                           '(fortran-to-lisp::slacpy
                                             fortran-to-lisp::stgsy2
                                             fortran-to-lisp::slaset
                                             fortran-to-lisp::ilaenv))))
```

———-

59.29 strsyl

59.29.1 Axiom unit tests

— **strsyl.input** —

```
)set break resume
)sys rm -f strsyl.output
)spool strsyl.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

59.29.2 Axiom help page

— **strsyl.help** —

```
==================================================================
strsyl examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine strsyl (TRANA, TRANB, ISGN, M, N, A, LDA, B, LDB, C, LDC,
           SCALE, INFO)
           STRSYL

Function/Subroutine Documentation
   subroutine strsyl (character TRANA, character TRANB, integer ISGN, integer
       M, integer N, real, dimension( lda, * ) A, integer LDA, real,
```

```
      dimension( ldb, * ) B, integer LDB, real, dimension( ldc, * ) C,
      integer LDC, real SCALE, integer INFO)
      STRSYL

Purpose:
   STRSYL solves the real Sylvester matrix equation:

               op(A)*X + X*op(B) = scale*C or
               op(A)*X - X*op(B) = scale*C,

   where op(A) = A or A**T, and  A and B are both upper quasi-
   triangular. A is M-by-M and B is N-by-N; the right hand side C and
   the solution X are M-by-N; and scale is an output scale factor, set
   <= 1 to avoid overflow in X.

   A and B must be in Schur canonical form (as returned by SHSEQR), that
   is, block upper triangular with 1-by-1 and 2-by-2 diagonal blocks;
   each 2-by-2 diagonal block has its diagonal elements equal and its
   off-diagonal elements of opposite sign.

Parameters:

 TRANA
   TRANA is CHARACTER*1
   Specifies the option op(A):
   = 'N': op(A) = A    (No transpose)
   = 'T': op(A) = A**T (Transpose)
   = 'C': op(A) = A**H (Conjugate transpose = Transpose)

 TRANB
   TRANB is CHARACTER*1
   Specifies the option op(B):
   = 'N': op(B) = B    (No transpose)
   = 'T': op(B) = B**T (Transpose)
   = 'C': op(B) = B**H (Conjugate transpose = Transpose)

 ISGN
   ISGN is INTEGER
   Specifies the sign in the equation:
   = +1: solve op(A)*X + X*op(B) = scale*C
   = -1: solve op(A)*X - X*op(B) = scale*C

 M
   M is INTEGER
   The order of the matrix A, and the number of rows in the
   matrices X and C. M >= 0.

 N
   N is INTEGER
   The order of the matrix B, and the number of columns in the
```

```
  matrices X and C. N >= 0.

A
  A is REAL array, dimension (LDA,M)
  The upper quasi-triangular matrix A, in Schur canonical form.

LDA
  LDA is INTEGER
  The leading dimension of the array A. LDA >= max(1,M).

B
  B is REAL array, dimension (LDB,N)
  The upper quasi-triangular matrix B, in Schur canonical form.

LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= max(1,N).

C
  C is REAL array, dimension (LDC,N)
  On entry, the M-by-N right hand side matrix C.
  On exit, C is overwritten by the solution matrix X.

LDC
  LDC is INTEGER
  The leading dimension of the array C. LDC >= max(1,M)

SCALE
  SCALE is REAL
  The scale factor, scale, set <= 1 to avoid overflow in X.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  = 1: A and B have common or very close eigenvalues; perturbed
       values were used to solve the equation (but the matrices
       A and B are unchanged).
```

59.29.3 fortran code

— strsyl.f —

```
*  =====================================================================
      SUBROUTINE STRSYL( TRANA, TRANB, ISGN, M, N, A, LDA, B, LDB, C,
```

```
     $                   LDC, SCALE, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANA, TRANB
      INTEGER            INFO, ISGN, LDA, LDB, LDC, M, N
      REAL               SCALE
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), B( LDB, * ), C( LDC, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRNA, NOTRNB
      INTEGER            IERR, J, K, K1, K2, KNEXT, L, L1, L2, LNEXT
      REAL               A11, BIGNUM, DA11, DB, EPS, SCALOC, SGN, SMIN,
     $                   SMLNUM, SUML, SUMR, XNORM
*     ..
*     .. Local Arrays ..
      REAL               DUM( 1 ), VEC( 2, 2 ), X( 2, 2 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SDOT, SLAMCH, SLANGE
      EXTERNAL           LSAME, SDOT, SLAMCH, SLANGE
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLABAD, SLALN2, SLASY2, SSCAL, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, REAL
*     ..
*     .. Executable Statements ..
*
*     Decode and Test input parameters
*
      NOTRNA = LSAME( TRANA, 'N' )
      NOTRNB = LSAME( TRANB, 'N' )
*
      INFO = 0
```

```
      IF( .NOT.NOTRNA .AND. .NOT.LSAME( TRANA, 'T' ) .AND. .NOT.
     $    LSAME( TRANA, 'C' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRNB .AND. .NOT.LSAME( TRANB, 'T' ) .AND. .NOT.
     $         LSAME( TRANB, 'C' ) ) THEN
         INFO = -2
      ELSE IF( ISGN.NE.1 .AND. ISGN.NE.-1 ) THEN
         INFO = -3
      ELSE IF( M.LT.0 ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
         INFO = -11
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'STRSYL', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      SCALE = ONE
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Set constants to control overflow
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
      SMLNUM = SMLNUM*REAL( M*N ) / EPS
      BIGNUM = ONE / SMLNUM
*
      SMIN = MAX( SMLNUM, EPS*SLANGE( 'M', M, M, A, LDA, DUM ),
     $       EPS*SLANGE( 'M', N, N, B, LDB, DUM ) )
*
      SGN = ISGN
*
      IF( NOTRNA .AND. NOTRNB ) THEN
*
*        Solve    A*X + ISGN*X*B = scale*C.
*
*        The (K,L)th block of X is determined starting from
*        bottom-left corner column by column by
```

```
*
*         A(K,K)*X(K,L) + ISGN*X(K,L)*B(L,L) = C(K,L) - R(K,L)
*
*        Where
*                  M                          L-1
*        R(K,L) = SUM [A(K,I)*X(I,L)] + ISGN*SUM [X(K,J)*B(J,L)].
*                I=K+1                        J=1
*
*        Start column loop (index = L)
*        L1 (L2) : column index of the first (first) row of X(K,L).
*
         LNEXT = 1
         DO 70 L = 1, N
            IF( L.LT.LNEXT )
     $         GO TO 70
            IF( L.EQ.N ) THEN
               L1 = L
               L2 = L
            ELSE
               IF( B( L+1, L ).NE.ZERO ) THEN
                  L1 = L
                  L2 = L + 1
                  LNEXT = L + 2
               ELSE
                  L1 = L
                  L2 = L
                  LNEXT = L + 1
               END IF
            END IF
*
*           Start row loop (index = K)
*           K1 (K2): row index of the first (last) row of X(K,L).
*
            KNEXT = M
            DO 60 K = M, 1, -1
               IF( K.GT.KNEXT )
     $            GO TO 60
               IF( K.EQ.1 ) THEN
                  K1 = K
                  K2 = K
               ELSE
                  IF( A( K, K-1 ).NE.ZERO ) THEN
                     K1 = K - 1
                     K2 = K
                     KNEXT = K - 2
                  ELSE
                     K1 = K
                     K2 = K
                     KNEXT = K - 1
                  END IF
```

```
            END IF
*
            IF( L1.EQ.L2 .AND. K1.EQ.K2 ) THEN
               SUML = SDOT( M-K1, A( K1, MIN( K1+1, M ) ), LDA,
     $                      C( MIN( K1+1, M ), L1 ), 1 )
               SUMR = SDOT( L1-1, C( K1, 1 ), LDC, B( 1, L1 ), 1 )
               VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
               SCALOC = ONE
*
               A11 = A( K1, K1 ) + SGN*B( L1, L1 )
               DA11 = ABS( A11 )
               IF( DA11.LE.SMIN ) THEN
                  A11 = SMIN
                  DA11 = SMIN
                  INFO = 1
               END IF
               DB = ABS( VEC( 1, 1 ) )
               IF( DA11.LT.ONE .AND. DB.GT.ONE ) THEN
                  IF( DB.GT.BIGNUM*DA11 )
     $               SCALOC = ONE / DB
               END IF
               X( 1, 1 ) = ( VEC( 1, 1 )*SCALOC ) / A11
*
               IF( SCALOC.NE.ONE ) THEN
                  DO 10 J = 1, N
                     CALL SSCAL( M, SCALOC, C( 1, J ), 1 )
   10             CONTINUE
                  SCALE = SCALE*SCALOC
               END IF
               C( K1, L1 ) = X( 1, 1 )
*
            ELSE IF( L1.EQ.L2 .AND. K1.NE.K2 ) THEN
*
               SUML = SDOT( M-K2, A( K1, MIN( K2+1, M ) ), LDA,
     $                      C( MIN( K2+1, M ), L1 ), 1 )
               SUMR = SDOT( L1-1, C( K1, 1 ), LDC, B( 1, L1 ), 1 )
               VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
*
               SUML = SDOT( M-K2, A( K2, MIN( K2+1, M ) ), LDA,
     $                      C( MIN( K2+1, M ), L1 ), 1 )
               SUMR = SDOT( L1-1, C( K2, 1 ), LDC, B( 1, L1 ), 1 )
               VEC( 2, 1 ) = C( K2, L1 ) - ( SUML+SGN*SUMR )
*
               CALL SLALN2( .FALSE., 2, 1, SMIN, ONE, A( K1, K1 ),
     $                      LDA, ONE, ONE, VEC, 2, -SGN*B( L1, L1 ),
     $                      ZERO, X, 2, SCALOC, XNORM, IERR )
               IF( IERR.NE.0 )
     $            INFO = 1
*
               IF( SCALOC.NE.ONE ) THEN
```

```
                 DO 20 J = 1, N
                    CALL SSCAL( M, SCALOC, C( 1, J ), 1 )
   20            CONTINUE
                 SCALE = SCALE*SCALOC
              END IF
              C( K1, L1 ) = X( 1, 1 )
              C( K2, L1 ) = X( 2, 1 )
*
           ELSE IF( L1.NE.L2 .AND. K1.EQ.K2 ) THEN
*
              SUML = SDOT( M-K1, A( K1, MIN( K1+1, M ) ), LDA,
     $                     C( MIN( K1+1, M ), L1 ), 1 )
              SUMR = SDOT( L1-1, C( K1, 1 ), LDC, B( 1, L1 ), 1 )
              VEC( 1, 1 ) = SGN*( C( K1, L1 )-( SUML+SGN*SUMR ) )
*
              SUML = SDOT( M-K1, A( K1, MIN( K1+1, M ) ), LDA,
     $                     C( MIN( K1+1, M ), L2 ), 1 )
              SUMR = SDOT( L1-1, C( K1, 1 ), LDC, B( 1, L2 ), 1 )
              VEC( 2, 1 ) = SGN*( C( K1, L2 )-( SUML+SGN*SUMR ) )
*
              CALL SLALN2( .TRUE., 2, 1, SMIN, ONE, B( L1, L1 ),
     $                     LDB, ONE, ONE, VEC, 2, -SGN*A( K1, K1 ),
     $                     ZERO, X, 2, SCALOC, XNORM, IERR )
              IF( IERR.NE.0 )
     $           INFO = 1
*
              IF( SCALOC.NE.ONE ) THEN
                 DO 40 J = 1, N
                    CALL SSCAL( M, SCALOC, C( 1, J ), 1 )
   40            CONTINUE
                 SCALE = SCALE*SCALOC
              END IF
              C( K1, L1 ) = X( 1, 1 )
              C( K1, L2 ) = X( 2, 1 )
*
           ELSE IF( L1.NE.L2 .AND. K1.NE.K2 ) THEN
*
              SUML = SDOT( M-K2, A( K1, MIN( K2+1, M ) ), LDA,
     $                     C( MIN( K2+1, M ), L1 ), 1 )
              SUMR = SDOT( L1-1, C( K1, 1 ), LDC, B( 1, L1 ), 1 )
              VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
*
              SUML = SDOT( M-K2, A( K1, MIN( K2+1, M ) ), LDA,
     $                     C( MIN( K2+1, M ), L2 ), 1 )
              SUMR = SDOT( L1-1, C( K1, 1 ), LDC, B( 1, L2 ), 1 )
              VEC( 1, 2 ) = C( K1, L2 ) - ( SUML+SGN*SUMR )
*
              SUML = SDOT( M-K2, A( K2, MIN( K2+1, M ) ), LDA,
     $                     C( MIN( K2+1, M ), L1 ), 1 )
              SUMR = SDOT( L1-1, C( K2, 1 ), LDC, B( 1, L1 ), 1 )
```

```
                 VEC( 2, 1 ) = C( K2, L1 ) - ( SUML+SGN*SUMR )
*
                 SUML = SDOT( M-K2, A( K2, MIN( K2+1, M ) ), LDA,
     $                       C( MIN( K2+1, M ), L2 ), 1 )
                 SUMR = SDOT( L1-1, C( K2, 1 ), LDC, B( 1, L2 ), 1 )
                 VEC( 2, 2 ) = C( K2, L2 ) - ( SUML+SGN*SUMR )
*
                 CALL SLASY2( .FALSE., .FALSE., ISGN, 2, 2,
     $                       A( K1, K1 ), LDA, B( L1, L1 ), LDB, VEC,
     $                       2, SCALOC, X, 2, XNORM, IERR )
                 IF( IERR.NE.0 )
     $              INFO = 1
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 50 J = 1, N
                       CALL SSCAL( M, SCALOC, C( 1, J ), 1 )
   50               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
                 C( K1, L1 ) = X( 1, 1 )
                 C( K1, L2 ) = X( 1, 2 )
                 C( K2, L1 ) = X( 2, 1 )
                 C( K2, L2 ) = X( 2, 2 )
              END IF
*
   60       CONTINUE
*
   70    CONTINUE
*
      ELSE IF( .NOT.NOTRNA .AND. NOTRNB ) THEN
*
*        Solve    A**T *X + ISGN*X*B = scale*C.
*
*        The (K,L)th block of X is determined starting from
*        upper-left corner column by column by
*
*          A(K,K)**T*X(K,L) + ISGN*X(K,L)*B(L,L) = C(K,L) - R(K,L)
*
*        Where
*                   K-1                          L-1
*          R(K,L) = SUM [A(I,K)**T*X(I,L)] +ISGN*SUM [X(K,J)*B(J,L)]
*                   I=1                          J=1
*
*        Start column loop (index = L)
*        L1 (L2): column index of the first (last) row of X(K,L)
*
         LNEXT = 1
         DO 130 L = 1, N
            IF( L.LT.LNEXT )
     $         GO TO 130
```

```
      IF( L.EQ.N ) THEN
         L1 = L
         L2 = L
      ELSE
         IF( B( L+1, L ).NE.ZERO ) THEN
            L1 = L
            L2 = L + 1
            LNEXT = L + 2
         ELSE
            L1 = L
            L2 = L
            LNEXT = L + 1
         END IF
      END IF
*
*     Start row loop (index = K)
*     K1 (K2): row index of the first (last) row of X(K,L)
*
      KNEXT = 1
      DO 120 K = 1, M
         IF( K.LT.KNEXT )
     $      GO TO 120
         IF( K.EQ.M ) THEN
            K1 = K
            K2 = K
         ELSE
            IF( A( K+1, K ).NE.ZERO ) THEN
               K1 = K
               K2 = K + 1
               KNEXT = K + 2
            ELSE
               K1 = K
               K2 = K
               KNEXT = K + 1
            END IF
         END IF
*
         IF( L1.EQ.L2 .AND. K1.EQ.K2 ) THEN
            SUML = SDOT( K1-1, A( 1, K1 ), 1, C( 1, L1 ), 1 )
            SUMR = SDOT( L1-1, C( K1, 1 ), LDC, B( 1, L1 ), 1 )
            VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
            SCALOC = ONE
*
            A11 = A( K1, K1 ) + SGN*B( L1, L1 )
            DA11 = ABS( A11 )
            IF( DA11.LE.SMIN ) THEN
               A11 = SMIN
               DA11 = SMIN
               INFO = 1
            END IF
```

```
                 DB = ABS( VEC( 1, 1 ) )
                 IF( DA11.LT.ONE .AND. DB.GT.ONE ) THEN
                    IF( DB.GT.BIGNUM*DA11 )
     $                 SCALOC = ONE / DB
                 END IF
                 X( 1, 1 ) = ( VEC( 1, 1 )*SCALOC ) / A11
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 80 J = 1, N
                       CALL SSCAL( M, SCALOC, C( 1, J ), 1 )
   80               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
                 C( K1, L1 ) = X( 1, 1 )
*
              ELSE IF( L1.EQ.L2 .AND. K1.NE.K2 ) THEN
*
                 SUML = SDOT( K1-1, A( 1, K1 ), 1, C( 1, L1 ), 1 )
                 SUMR = SDOT( L1-1, C( K1, 1 ), LDC, B( 1, L1 ), 1 )
                 VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
*
                 SUML = SDOT( K1-1, A( 1, K2 ), 1, C( 1, L1 ), 1 )
                 SUMR = SDOT( L1-1, C( K2, 1 ), LDC, B( 1, L1 ), 1 )
                 VEC( 2, 1 ) = C( K2, L1 ) - ( SUML+SGN*SUMR )
*
                 CALL SLALN2( .TRUE., 2, 1, SMIN, ONE, A( K1, K1 ),
     $                        LDA, ONE, ONE, VEC, 2, -SGN*B( L1, L1 ),
     $                        ZERO, X, 2, SCALOC, XNORM, IERR )
                 IF( IERR.NE.0 )
     $              INFO = 1
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 90 J = 1, N
                       CALL SSCAL( M, SCALOC, C( 1, J ), 1 )
   90               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
                 C( K1, L1 ) = X( 1, 1 )
                 C( K2, L1 ) = X( 2, 1 )
*
              ELSE IF( L1.NE.L2 .AND. K1.EQ.K2 ) THEN
*
                 SUML = SDOT( K1-1, A( 1, K1 ), 1, C( 1, L1 ), 1 )
                 SUMR = SDOT( L1-1, C( K1, 1 ), LDC, B( 1, L1 ), 1 )
                 VEC( 1, 1 ) = SGN*( C( K1, L1 )-( SUML+SGN*SUMR ) )
*
                 SUML = SDOT( K1-1, A( 1, K1 ), 1, C( 1, L2 ), 1 )
                 SUMR = SDOT( L1-1, C( K1, 1 ), LDC, B( 1, L2 ), 1 )
                 VEC( 2, 1 ) = SGN*( C( K1, L2 )-( SUML+SGN*SUMR ) )
*
```

```
                 CALL SLALN2( .TRUE., 2, 1, SMIN, ONE, B( L1, L1 ),
     $                        LDB, ONE, ONE, VEC, 2, -SGN*A( K1, K1 ),
     $                        ZERO, X, 2, SCALOC, XNORM, IERR )
                 IF( IERR.NE.0 )
     $              INFO = 1
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 100 J = 1, N
                       CALL SSCAL( M, SCALOC, C( 1, J ), 1 )
  100               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
                 C( K1, L1 ) = X( 1, 1 )
                 C( K1, L2 ) = X( 2, 1 )
*
              ELSE IF( L1.NE.L2 .AND. K1.NE.K2 ) THEN
*
                 SUML = SDOT( K1-1, A( 1, K1 ), 1, C( 1, L1 ), 1 )
                 SUMR = SDOT( L1-1, C( K1, 1 ), LDC, B( 1, L1 ), 1 )
                 VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
*
                 SUML = SDOT( K1-1, A( 1, K1 ), 1, C( 1, L2 ), 1 )
                 SUMR = SDOT( L1-1, C( K1, 1 ), LDC, B( 1, L2 ), 1 )
                 VEC( 1, 2 ) = C( K1, L2 ) - ( SUML+SGN*SUMR )
*
                 SUML = SDOT( K1-1, A( 1, K2 ), 1, C( 1, L1 ), 1 )
                 SUMR = SDOT( L1-1, C( K2, 1 ), LDC, B( 1, L1 ), 1 )
                 VEC( 2, 1 ) = C( K2, L1 ) - ( SUML+SGN*SUMR )
*
                 SUML = SDOT( K1-1, A( 1, K2 ), 1, C( 1, L2 ), 1 )
                 SUMR = SDOT( L1-1, C( K2, 1 ), LDC, B( 1, L2 ), 1 )
                 VEC( 2, 2 ) = C( K2, L2 ) - ( SUML+SGN*SUMR )
*
                 CALL SLASY2( .TRUE., .FALSE., ISGN, 2, 2, A( K1, K1 ),
     $                        LDA, B( L1, L1 ), LDB, VEC, 2, SCALOC, X,
     $                        2, XNORM, IERR )
                 IF( IERR.NE.0 )
     $              INFO = 1
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 110 J = 1, N
                       CALL SSCAL( M, SCALOC, C( 1, J ), 1 )
  110               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
                 C( K1, L1 ) = X( 1, 1 )
                 C( K1, L2 ) = X( 1, 2 )
                 C( K2, L1 ) = X( 2, 1 )
                 C( K2, L2 ) = X( 2, 2 )
              END IF
```

```
*
  120       CONTINUE
  130    CONTINUE
*
      ELSE IF( .NOT.NOTRNA .AND. .NOT.NOTRNB ) THEN
*
*        Solve    A**T*X + ISGN*X*B**T = scale*C.
*
*        The (K,L)th block of X is determined starting from
*        top-right corner column by column by
*
*           A(K,K)**T*X(K,L) + ISGN*X(K,L)*B(L,L)**T = C(K,L) - R(K,L)
*
*        Where
*                     K-1                             N
*            R(K,L) = SUM [A(I,K)**T*X(I,L)] + ISGN*SUM [X(K,J)*B(L,J)**T].
*                     I=1                           J=L+1
*
*        Start column loop (index = L)
*        L1 (L2): column index of the first (last) row of X(K,L)
*
         LNEXT = N
         DO 190 L = N, 1, -1
            IF( L.GT.LNEXT )
     $         GO TO 190
            IF( L.EQ.1 ) THEN
               L1 = L
               L2 = L
            ELSE
               IF( B( L, L-1 ).NE.ZERO ) THEN
                  L1 = L - 1
                  L2 = L
                  LNEXT = L - 2
               ELSE
                  L1 = L
                  L2 = L
                  LNEXT = L - 1
               END IF
            END IF
*
*           Start row loop (index = K)
*           K1 (K2): row index of the first (last) row of X(K,L)
*
            KNEXT = 1
            DO 180 K = 1, M
               IF( K.LT.KNEXT )
     $            GO TO 180
               IF( K.EQ.M ) THEN
                  K1 = K
                  K2 = K
```

```
            ELSE
               IF( A( K+1, K ).NE.ZERO ) THEN
                  K1 = K
                  K2 = K + 1
                  KNEXT = K + 2
               ELSE
                  K1 = K
                  K2 = K
                  KNEXT = K + 1
               END IF
            END IF
*
            IF( L1.EQ.L2 .AND. K1.EQ.K2 ) THEN
               SUML = SDOT( K1-1, A( 1, K1 ), 1, C( 1, L1 ), 1 )
               SUMR = SDOT( N-L1, C( K1, MIN( L1+1, N ) ), LDC,
     $                      B( L1, MIN( L1+1, N ) ), LDB )
               VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
               SCALOC = ONE
*
               A11 = A( K1, K1 ) + SGN*B( L1, L1 )
               DA11 = ABS( A11 )
               IF( DA11.LE.SMIN ) THEN
                  A11 = SMIN
                  DA11 = SMIN
                  INFO = 1
               END IF
               DB = ABS( VEC( 1, 1 ) )
               IF( DA11.LT.ONE .AND. DB.GT.ONE ) THEN
                  IF( DB.GT.BIGNUM*DA11 )
     $               SCALOC = ONE / DB
               END IF
               X( 1, 1 ) = ( VEC( 1, 1 )*SCALOC ) / A11
*
               IF( SCALOC.NE.ONE ) THEN
                  DO 140 J = 1, N
                     CALL SSCAL( M, SCALOC, C( 1, J ), 1 )
  140             CONTINUE
                  SCALE = SCALE*SCALOC
               END IF
               C( K1, L1 ) = X( 1, 1 )
*
            ELSE IF( L1.EQ.L2 .AND. K1.NE.K2 ) THEN
*
               SUML = SDOT( K1-1, A( 1, K1 ), 1, C( 1, L1 ), 1 )
               SUMR = SDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                      B( L1, MIN( L2+1, N ) ), LDB )
               VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
*
               SUML = SDOT( K1-1, A( 1, K2 ), 1, C( 1, L1 ), 1 )
               SUMR = SDOT( N-L2, C( K2, MIN( L2+1, N ) ), LDC,
```

```
     $                       B( L1, MIN( L2+1, N ) ), LDB )
                  VEC( 2, 1 ) = C( K2, L1 ) - ( SUML+SGN*SUMR )
*
                  CALL SLALN2( .TRUE., 2, 1, SMIN, ONE, A( K1, K1 ),
     $                       LDA, ONE, ONE, VEC, 2, -SGN*B( L1, L1 ),
     $                       ZERO, X, 2, SCALOC, XNORM, IERR )
                  IF( IERR.NE.0 )
     $               INFO = 1
*
                  IF( SCALOC.NE.ONE ) THEN
                     DO 150 J = 1, N
                        CALL SSCAL( M, SCALOC, C( 1, J ), 1 )
  150                CONTINUE
                     SCALE = SCALE*SCALOC
                  END IF
                  C( K1, L1 ) = X( 1, 1 )
                  C( K2, L1 ) = X( 2, 1 )
*
               ELSE IF( L1.NE.L2 .AND. K1.EQ.K2 ) THEN
*
                  SUML = SDOT( K1-1, A( 1, K1 ), 1, C( 1, L1 ), 1 )
                  SUMR = SDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                       B( L1, MIN( L2+1, N ) ), LDB )
                  VEC( 1, 1 ) = SGN*( C( K1, L1 )-( SUML+SGN*SUMR ) )
*
                  SUML = SDOT( K1-1, A( 1, K1 ), 1, C( 1, L2 ), 1 )
                  SUMR = SDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                       B( L2, MIN( L2+1, N ) ), LDB )
                  VEC( 2, 1 ) = SGN*( C( K1, L2 )-( SUML+SGN*SUMR ) )
*
                  CALL SLALN2( .FALSE., 2, 1, SMIN, ONE, B( L1, L1 ),
     $                       LDB, ONE, ONE, VEC, 2, -SGN*A( K1, K1 ),
     $                       ZERO, X, 2, SCALOC, XNORM, IERR )
                  IF( IERR.NE.0 )
     $               INFO = 1
*
                  IF( SCALOC.NE.ONE ) THEN
                     DO 160 J = 1, N
                        CALL SSCAL( M, SCALOC, C( 1, J ), 1 )
  160                CONTINUE
                     SCALE = SCALE*SCALOC
                  END IF
                  C( K1, L1 ) = X( 1, 1 )
                  C( K1, L2 ) = X( 2, 1 )
*
               ELSE IF( L1.NE.L2 .AND. K1.NE.K2 ) THEN
*
                  SUML = SDOT( K1-1, A( 1, K1 ), 1, C( 1, L1 ), 1 )
                  SUMR = SDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                       B( L1, MIN( L2+1, N ) ), LDB )
```

```
                 VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
*
                 SUML = SDOT( K1-1, A( 1, K1 ), 1, C( 1, L2 ), 1 )
                 SUMR = SDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                       B( L2, MIN( L2+1, N ) ), LDB )
                 VEC( 1, 2 ) = C( K1, L2 ) - ( SUML+SGN*SUMR )
*
                 SUML = SDOT( K1-1, A( 1, K2 ), 1, C( 1, L1 ), 1 )
                 SUMR = SDOT( N-L2, C( K2, MIN( L2+1, N ) ), LDC,
     $                       B( L1, MIN( L2+1, N ) ), LDB )
                 VEC( 2, 1 ) = C( K2, L1 ) - ( SUML+SGN*SUMR )
*
                 SUML = SDOT( K1-1, A( 1, K2 ), 1, C( 1, L2 ), 1 )
                 SUMR = SDOT( N-L2, C( K2, MIN( L2+1, N ) ), LDC,
     $                       B( L2, MIN(L2+1, N ) ), LDB )
                 VEC( 2, 2 ) = C( K2, L2 ) - ( SUML+SGN*SUMR )
*
                 CALL SLASY2( .TRUE., .TRUE., ISGN, 2, 2, A( K1, K1 ),
     $                       LDA, B( L1, L1 ), LDB, VEC, 2, SCALOC, X,
     $                       2, XNORM, IERR )
                 IF( IERR.NE.0 )
     $              INFO = 1
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 170 J = 1, N
                       CALL SSCAL( M, SCALOC, C( 1, J ), 1 )
  170               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
                 C( K1, L1 ) = X( 1, 1 )
                 C( K1, L2 ) = X( 1, 2 )
                 C( K2, L1 ) = X( 2, 1 )
                 C( K2, L2 ) = X( 2, 2 )
              END IF
*
  180       CONTINUE
  190    CONTINUE
*
      ELSE IF( NOTRNA .AND. .NOT.NOTRNB ) THEN
*
*        Solve    A*X + ISGN*X*B**T = scale*C.
*
*        The (K,L)th block of X is determined starting from
*        bottom-right corner column by column by
*
*            A(K,K)*X(K,L) + ISGN*X(K,L)*B(L,L)**T = C(K,L) - R(K,L)
*
*        Where
*                      M                          N
*            R(K,L) = SUM [A(K,I)*X(I,L)] + ISGN*SUM [X(K,J)*B(L,J)**T].
```

```
*                   I=K+1                       J=L+1
*
*        Start column loop (index = L)
*        L1 (L2): column index of the first (last) row of X(K,L)
*
         LNEXT = N
         DO 250 L = N, 1, -1
            IF( L.GT.LNEXT )
     $         GO TO 250
            IF( L.EQ.1 ) THEN
               L1 = L
               L2 = L
            ELSE
               IF( B( L, L-1 ).NE.ZERO ) THEN
                  L1 = L - 1
                  L2 = L
                  LNEXT = L - 2
               ELSE
                  L1 = L
                  L2 = L
                  LNEXT = L - 1
               END IF
            END IF
*
*           Start row loop (index = K)
*           K1 (K2): row index of the first (last) row of X(K,L)
*
            KNEXT = M
            DO 240 K = M, 1, -1
               IF( K.GT.KNEXT )
     $            GO TO 240
               IF( K.EQ.1 ) THEN
                  K1 = K
                  K2 = K
               ELSE
                  IF( A( K, K-1 ).NE.ZERO ) THEN
                     K1 = K - 1
                     K2 = K
                     KNEXT = K - 2
                  ELSE
                     K1 = K
                     K2 = K
                     KNEXT = K - 1
                  END IF
               END IF
*
               IF( L1.EQ.L2 .AND. K1.EQ.K2 ) THEN
                  SUML = SDOT( M-K1, A( K1, MIN(K1+1, M ) ), LDA,
     $                   C( MIN( K1+1, M ), L1 ), 1 )
                  SUMR = SDOT( N-L1, C( K1, MIN( L1+1, N ) ), LDC,
```

```
     $                       B( L1, MIN( L1+1, N ) ), LDB )
                  VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
                  SCALOC = ONE
*
                  A11 = A( K1, K1 ) + SGN*B( L1, L1 )
                  DA11 = ABS( A11 )
                  IF( DA11.LE.SMIN ) THEN
                     A11 = SMIN
                     DA11 = SMIN
                     INFO = 1
                  END IF
                  DB = ABS( VEC( 1, 1 ) )
                  IF( DA11.LT.ONE .AND. DB.GT.ONE ) THEN
                     IF( DB.GT.BIGNUM*DA11 )
     $                  SCALOC = ONE / DB
                  END IF
                  X( 1, 1 ) = ( VEC( 1, 1 )*SCALOC ) / A11
*
                  IF( SCALOC.NE.ONE ) THEN
                     DO 200 J = 1, N
                        CALL SSCAL( M, SCALOC, C( 1, J ), 1 )
  200                CONTINUE
                     SCALE = SCALE*SCALOC
                  END IF
                  C( K1, L1 ) = X( 1, 1 )
*
               ELSE IF( L1.EQ.L2 .AND. K1.NE.K2 ) THEN
*
                  SUML = SDOT( M-K2, A( K1, MIN( K2+1, M ) ), LDA,
     $                       C( MIN( K2+1, M ), L1 ), 1 )
                  SUMR = SDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                       B( L1, MIN( L2+1, N ) ), LDB )
                  VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
*
                  SUML = SDOT( M-K2, A( K2, MIN( K2+1, M ) ), LDA,
     $                       C( MIN( K2+1, M ), L1 ), 1 )
                  SUMR = SDOT( N-L2, C( K2, MIN( L2+1, N ) ), LDC,
     $                       B( L1, MIN( L2+1, N ) ), LDB )
                  VEC( 2, 1 ) = C( K2, L1 ) - ( SUML+SGN*SUMR )
*
                  CALL SLALN2( .FALSE., 2, 1, SMIN, ONE, A( K1, K1 ),
     $                       LDA, ONE, ONE, VEC, 2, -SGN*B( L1, L1 ),
     $                       ZERO, X, 2, SCALOC, XNORM, IERR )
                  IF( IERR.NE.0 )
     $               INFO = 1
*
                  IF( SCALOC.NE.ONE ) THEN
                     DO 210 J = 1, N
                        CALL SSCAL( M, SCALOC, C( 1, J ), 1 )
  210                CONTINUE
```

```
                 SCALE = SCALE*SCALOC
              END IF
              C( K1, L1 ) = X( 1, 1 )
              C( K2, L1 ) = X( 2, 1 )
*
           ELSE IF( L1.NE.L2 .AND. K1.EQ.K2 ) THEN
*
              SUML = SDOT( M-K1, A( K1, MIN( K1+1, M ) ), LDA,
     $                     C( MIN( K1+1, M ), L1 ), 1 )
              SUMR = SDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                     B( L1, MIN( L2+1, N ) ), LDB )
              VEC( 1, 1 ) = SGN*( C( K1, L1 )-( SUML+SGN*SUMR ) )
*
              SUML = SDOT( M-K1, A( K1, MIN( K1+1, M ) ), LDA,
     $                     C( MIN( K1+1, M ), L2 ), 1 )
              SUMR = SDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                     B( L2, MIN( L2+1, N ) ), LDB )
              VEC( 2, 1 ) = SGN*( C( K1, L2 )-( SUML+SGN*SUMR ) )
*
              CALL SLALN2( .FALSE., 2, 1, SMIN, ONE, B( L1, L1 ),
     $                     LDB, ONE, ONE, VEC, 2, -SGN*A( K1, K1 ),
     $                     ZERO, X, 2, SCALOC, XNORM, IERR )
              IF( IERR.NE.0 )
     $           INFO = 1
*
              IF( SCALOC.NE.ONE ) THEN
                 DO 220 J = 1, N
                    CALL SSCAL( M, SCALOC, C( 1, J ), 1 )
  220            CONTINUE
                 SCALE = SCALE*SCALOC
              END IF
              C( K1, L1 ) = X( 1, 1 )
              C( K1, L2 ) = X( 2, 1 )
*
           ELSE IF( L1.NE.L2 .AND. K1.NE.K2 ) THEN
*
              SUML = SDOT( M-K2, A( K1, MIN( K2+1, M ) ), LDA,
     $                     C( MIN( K2+1, M ), L1 ), 1 )
              SUMR = SDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                     B( L1, MIN( L2+1, N ) ), LDB )
              VEC( 1, 1 ) = C( K1, L1 ) - ( SUML+SGN*SUMR )
*
              SUML = SDOT( M-K2, A( K1, MIN( K2+1, M ) ), LDA,
     $                     C( MIN( K2+1, M ), L2 ), 1 )
              SUMR = SDOT( N-L2, C( K1, MIN( L2+1, N ) ), LDC,
     $                     B( L2, MIN( L2+1, N ) ), LDB )
              VEC( 1, 2 ) = C( K1, L2 ) - ( SUML+SGN*SUMR )
*
              SUML = SDOT( M-K2, A( K2, MIN( K2+1, M ) ), LDA,
     $                     C( MIN( K2+1, M ), L1 ), 1 )
```

```
                 SUMR = SDOT( N-L2, C( K2, MIN( L2+1, N ) ), LDC,
     $                       B( L1, MIN( L2+1, N ) ), LDB )
                 VEC( 2, 1 ) = C( K2, L1 ) - ( SUML+SGN*SUMR )
*
                 SUML = SDOT( M-K2, A( K2, MIN( K2+1, M ) ), LDA,
     $                       C( MIN( K2+1, M ), L2 ), 1 )
                 SUMR = SDOT( N-L2, C( K2, MIN( L2+1, N ) ), LDC,
     $                       B( L2, MIN( L2+1, N ) ), LDB )
                 VEC( 2, 2 ) = C( K2, L2 ) - ( SUML+SGN*SUMR )
*
                 CALL SLASY2( .FALSE., .TRUE., ISGN, 2, 2, A( K1, K1 ),
     $                       LDA, B( L1, L1 ), LDB, VEC, 2, SCALOC, X,
     $                       2, XNORM, IERR )
                 IF( IERR.NE.0 )
     $              INFO = 1
*
                 IF( SCALOC.NE.ONE ) THEN
                    DO 230 J = 1, N
                       CALL SSCAL( M, SCALOC, C( 1, J ), 1 )
  230               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
                 C( K1, L1 ) = X( 1, 1 )
                 C( K1, L2 ) = X( 1, 2 )
                 C( K2, L1 ) = X( 2, 1 )
                 C( K2, L2 ) = X( 2, 2 )
              END IF
*
  240       CONTINUE
  250    CONTINUE
*
      END IF
*
      RETURN
*
*     End of STRSYL
*
      END
```

———-

59.29.4 lisp code

— strsyl.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
```

```
          (ignorable zero one))
  (defun strsyl (trana tranb isgn m n a lda b ldb$ c ldc scale info)
    (declare (type (single-float) scale)
             (type (array single-float (*)) c b a)
             (type (f2cl-lib:integer4) info ldc ldb$ lda n m isgn)
             (type (string *) tranb trana))
    (f2cl-lib:with-multi-array-data
        ((trana character trana-%data% trana-%offset%)
         (tranb character tranb-%data% tranb-%offset%)
         (a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%)
         (c single-float c-%data% c-%offset%))
      (prog ((dum (make-array 1 :element-type 'single-float))
             (vec (make-array 4 :element-type 'single-float))
             (x (make-array 4 :element-type 'single-float)) (a11 0.0)
             (bignum 0.0) (da11 0.0) (db 0.0) (eps 0.0) (scaloc 0.0) (sgn 0.0)
             (smin 0.0) (smlnum 0.0) (suml 0.0) (sumr 0.0) (xnorm 0.0) (ierr 0)
             (j 0) (k 0) (k1 0) (k2 0) (knext 0) (l 0) (l1 0) (l2 0) (lnext 0)
             (notrna nil) (notrnb nil))
        (declare (type (array single-float (1)) dum)
                 (type (array single-float (4)) vec x)
                 (type (single-float) a11 bignum da11 db eps scaloc sgn smin
                  smlnum suml sumr xnorm)
                 (type (f2cl-lib:integer4) ierr j k k1 k2 knext l l1 l2 lnext)
                 (type f2cl-lib:logical notrna notrnb))
        (setf notrna
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame trana "N")
                  (declare (ignore var-1))
                  (when var-0 (setf trana var-0))
                  ret-val))
        (setf notrnb
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame tranb "N")
                  (declare (ignore var-1))
                  (when var-0 (setf tranb var-0))
                  ret-val))
        (setf info 0)
        (cond
         ((and (not notrna)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame trana "T")
                  (declare (ignore var-1))
                  (when var-0 (setf trana var-0))
                  ret-val))
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame trana "C")
                  (declare (ignore var-1))
```

```
          (when var-0 (setf trana var-0))
          ret-val)))
  (setf info -1))
 ((and (not notrnb)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame tranb "T")
          (declare (ignore var-1))
          (when var-0 (setf tranb var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame tranb "C")
          (declare (ignore var-1))
          (when var-0 (setf tranb var-0))
          ret-val)))
  (setf info -2))
 ((and (/= isgn 1) (/= isgn (f2cl-lib:int-sub 1))) (setf info -3))
 ((< m 0) (setf info -4)) ((< n 0) (setf info -5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9))
 ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -11)))
(cond
 ((/= info 0) (xerbla "STRSYL" (f2cl-lib:int-sub info))
  (go end_label)))
(setf scale one)
(if (or (= m 0) (= n 0))
    (go end_label))
(setf eps (slamch "P"))
(setf smlnum (slamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (slabad smlnum bignum)
  (declare (ignore))
  (setf smlnum var-0)
  (setf bignum var-1))
(setf smlnum
        (/ (* smlnum (f2cl-lib:freal (f2cl-lib:int-mul m n))) eps))
(setf bignum (/ one smlnum))
(setf smin
        (max smlnum
             (* eps
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                    (slange "M" m m a lda dum)
                  (declare (ignore var-0 var-2 var-3 var-4 var-5))
                  (setf m var-1)
```

```
                    ret-val))
                (* eps
                   (multiple-value-bind
                       (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                       (slange "M" n n b ldb$ dum)
                     (declare (ignore var-0 var-2 var-3 var-4 var-5))
                     (setf n var-1)
                     ret-val))))
(setf sgn (coerce (the f2cl-lib:integer4 isgn) 'single-float))
(cond
 ((and notrna notrnb) (setf lnext 1)
  (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                ((> l n) nil)
    (tagbody
      (if (< l lnext)
          (go label70))
      (cond ((= l n) (setf l1 l) (setf l2 l))
            (t
             (cond
              ((/=
                (f2cl-lib:fref b ((f2cl-lib:int-add l 1) l)
                               ((1 ldb$) (1 *)))
                zero)
               (setf l1 l) (setf l2 (f2cl-lib:int-add l 1))
               (setf lnext (f2cl-lib:int-add l 2)))
              (t (setf l1 l) (setf l2 l)
               (setf lnext (f2cl-lib:int-add l 1))))))
      (setf knext m)
      (f2cl-lib:fdo (k m (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                    ((> k 1) nil)
        (tagbody
          (if (> k knext)
              (go label60))
          (cond ((= k 1) (setf k1 k) (setf k2 k))
                (t
                 (cond
                  ((/=
                    (f2cl-lib:fref a
                                   (k
                                    (f2cl-lib:int-add k
                                                      (f2cl-lib:int-sub
                                                       1)))
                                   ((1 lda) (1 *)))
                    zero)
                   (setf k1 (f2cl-lib:int-sub k 1)) (setf k2 k)
                   (setf knext (f2cl-lib:int-sub k 2)))
                  (t (setf k1 k) (setf k2 k)
                   (setf knext (f2cl-lib:int-sub k 1))))))
          (cond
           ((and (= l1 l2) (= k1 k2))
```

```
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub m k1)
             (f2cl-lib:array-slice a-%data% single-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           k1 1))
                                     (the f2cl-lib:integer4
                                          m)))
                                   ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice c-%data% single-float
                                   ((min
                                     (f2cl-lib:int-add k1
                                                       1)
                                     m)
                                    l1)
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf lda var-2))
          ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k1 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                   (1 l1) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf scaloc one)
(setf a11
        (+
```

```
          (f2cl-lib:fref a-%data% (k1 k1) ((1 lda) (1 *))
                         a-%offset%)
          (* sgn
             (f2cl-lib:fref b-%data% (l1 l1)
                            ((1 ldb$) (1 *)) b-%offset%))))
 (setf da11 (abs a11))
 (cond
  ((<= da11 smin) (setf a11 smin) (setf da11 smin)
   (setf info 1)))
 (setf db (abs (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))))
 (cond
  ((and (< da11 one) (> db one))
   (if (> db (* bignum da11))
       (setf scaloc (/ one db)))))
 (setf (f2cl-lib:fref x (1 1) ((1 2) (1 2)))
         (/
          (* (f2cl-lib:fref vec (1 1) ((1 2) (1 2))) scaloc)
          a11))
 (cond
  ((/= scaloc one)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2 var-3)
           (sscal m scaloc
            (f2cl-lib:array-slice c-%data% single-float
                                  (1 j) ((1 ldc) (1 *))
                                  c-%offset%)
            1)
         (declare (ignore var-2 var-3))
         (when var-0 (setf m var-0))
         (when var-1 (setf scaloc var-1)))
      label10))
   (setf scale (* scale scaloc))))
 (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                       c-%offset%)
         (f2cl-lib:fref x (1 1) ((1 2) (1 2)))))
((and (= l1 l2) (/= k1 k2))
 (setf suml
         (multiple-value-bind
             (ret-val var-0 var-1 var-2 var-3 var-4)
             (sdot (f2cl-lib:int-sub m k2)
              (f2cl-lib:array-slice a-%data% single-float
                                    (k1
                                     (min
                                      (the f2cl-lib:integer4
                                           (f2cl-lib:int-add
                                            k2 1))
                                      (the f2cl-lib:integer4
                                           m)))
```

```
                                        ((1 lda) (1 *))
                                        a-%offset%)
                    lda
                    (f2cl-lib:array-slice c-%data% single-float
                                        ((min
                                          (f2cl-lib:int-add k2
                                                            1)
                                          m)
                                         l1)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                    1)
                  (declare (ignore var-0 var-1 var-3 var-4))
                  (when var-2 (setf lda var-2))
                  ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% single-float
                                        (k1 1) ((1 ldc) (1 *))
                                        c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                        (1 l1) ((1 ldb$) (1 *))
                                        b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub m k2)
             (f2cl-lib:array-slice a-%data% single-float
                                        (k2
                                         (min
                                          (the f2cl-lib:integer4
                                               (f2cl-lib:int-add
                                                k2 1))
                                          (the f2cl-lib:integer4
                                               m)))
                                        ((1 lda) (1 *))
                                        a-%offset%)
             lda
```

```
                (f2cl-lib:array-slice c-%data% single-float
                                      ((min
                                        (f2cl-lib:int-add k2
                                                          1)
                                        m)
                                       l1)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                1)
            (declare (ignore var-0 var-1 var-3 var-4))
            (when var-2 (setf lda var-2))
            ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k2 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                   (1 l1) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
    (slaln2 f2cl-lib:%false% 2 1 smin one
     (f2cl-lib:array-slice a-%data% single-float (k1 k1)
                           ((1 lda) (1 *)) a-%offset%)
     lda one one vec 2
     (* (- sgn)
        (f2cl-lib:fref b-%data% (l1 l1) ((1 ldb$) (1 *))
                       b-%offset%))
     zero x 2 scaloc xnorm ierr)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
    var-8 var-9 var-10 var-11 var-12 var-13 var-14))
  (setf scaloc var-15)
  (setf xnorm var-16)
  (setf ierr var-17))
```

```
(if (/= ierr 0)
    (setf info 1))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal m scaloc
           (f2cl-lib:array-slice c-%data% single-float
                                 (1 j) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
     label20))
  (setf scale (* scale scaloc))))
(setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
(setf (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref x (2 1) ((1 2) (1 2)))))
((and (/= l1 l2) (= k1 k2))
 (setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub m k1)
             (f2cl-lib:array-slice a-%data% single-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           k1 1))
                                     (the f2cl-lib:integer4
                                          m)))
                                   ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice c-%data% single-float
                                   ((min
                                     (f2cl-lib:int-add k1
                                                       1)
                                     m)
                                    l1)
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             1)
           (declare (ignore var-0 var-1 var-3 var-4))
```

```
          (when var-2 (setf lda var-2))
          ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k1 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                   (1 l1) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (* sgn
           (-
            (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                           c-%offset%)
            (+ suml (* sgn sumr)))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub m k1)
             (f2cl-lib:array-slice a-%data% single-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           k1 1))
                                     (the f2cl-lib:integer4
                                          m)))
                                   ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice c-%data% single-float
                                   ((min
                                     (f2cl-lib:int-add k1
                                                       1)
                                     m)
                                    l2)
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf lda var-2))
          ret-val))
```

```
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k1 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                   (1 l2) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (* sgn
           (-
            (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                           c-%offset%)
            (+ suml (* sgn sumr)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
    (slaln2 f2cl-lib:%true% 2 1 smin one
     (f2cl-lib:array-slice b-%data% single-float (l1 l1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ one one vec 2
     (* (- sgn)
        (f2cl-lib:fref a-%data% (k1 k1) ((1 lda) (1 *))
                       a-%offset%))
     zero x 2 scaloc xnorm ierr)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
    var-8 var-9 var-10 var-11 var-12 var-13 var-14))
  (setf scaloc var-15)
  (setf xnorm var-16)
  (setf ierr var-17))
(if (/= ierr 0)
    (setf info 1))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal m scaloc
           (f2cl-lib:array-slice c-%data% single-float
                                 (1 j) ((1 ldc) (1 *))
```

```
                                    c-%offset%)
                 1)
            (declare (ignore var-2 var-3))
            (when var-0 (setf m var-0))
            (when var-1 (setf scaloc var-1)))
        label40))
      (setf scale (* scale scaloc))))
    (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                         c-%offset%)
            (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
    (setf (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                         c-%offset%)
            (f2cl-lib:fref x (2 1) ((1 2) (1 2)))))
   ((and (/= l1 l2) (/= k1 k2))
    (setf suml
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4)
                (sdot (f2cl-lib:int-sub m k2)
                 (f2cl-lib:array-slice a-%data% single-float
                                       (k1
                                        (min
                                         (the f2cl-lib:integer4
                                              (f2cl-lib:int-add
                                               k2 1))
                                         (the f2cl-lib:integer4
                                              m)))
                                       ((1 lda) (1 *))
                                       a-%offset%)
                 lda
                 (f2cl-lib:array-slice c-%data% single-float
                                       ((min
                                         (f2cl-lib:int-add k2
                                                           1)
                                         m)
                                        l1)
                                       ((1 ldc) (1 *))
                                       c-%offset%)
                 1)
              (declare (ignore var-0 var-1 var-3 var-4))
              (when var-2 (setf lda var-2))
              ret-val))
    (setf sumr
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4)
                (sdot (f2cl-lib:int-sub l1 1)
                 (f2cl-lib:array-slice c-%data% single-float
                                       (k1 1) ((1 ldc) (1 *))
                                       c-%offset%)
                 ldc
                 (f2cl-lib:array-slice b-%data% single-float
```

```
                                        (1 l1) ((1 ldb$) (1 *))
                                        b-%offset%)
                    1)
                  (declare (ignore var-0 var-1 var-3 var-4))
                  (when var-2 (setf ldc var-2))
                  ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub m k2)
             (f2cl-lib:array-slice a-%data% single-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           k2 1))
                                     (the f2cl-lib:integer4
                                          m)))
                                   ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice c-%data% single-float
                                   ((min
                                     (f2cl-lib:int-add k2
                                                       1)
                                     m)
                                    l2)
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf lda var-2))
          ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k1 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                   (1 l2) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
```

```
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (1 2) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub m k2)
             (f2cl-lib:array-slice a-%data% single-float
                                   (k2
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           k2 1))
                                     (the f2cl-lib:integer4
                                          m)))
                                   ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice c-%data% single-float
                                   ((min
                                     (f2cl-lib:int-add k2
                                                       1)
                                     m)
                                    l1)
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf lda var-2))
          ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k2 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                   (1 l1) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
```

```
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub m k2)
             (f2cl-lib:array-slice a-%data% single-float
                                   (k2
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           k2 1))
                                     (the f2cl-lib:integer4
                                          m)))
                                   ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice c-%data% single-float
                                   ((min
                                     (f2cl-lib:int-add k2
                                                       1)
                                     m)
                                    l2)
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf lda var-2))
          ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k2 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                   (1 l2) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf (f2cl-lib:fref vec (2 2) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l2) ((1 ldc) (1 *))
```

```
                                c-%offset%)
                     (+ suml (* sgn sumr))))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10 var-11 var-12 var-13 var-14 var-15)
                (slasy2 f2cl-lib:%false% f2cl-lib:%false% isgn 2 2
                 (f2cl-lib:array-slice a-%data% single-float (k1 k1)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice b-%data% single-float (l1 l1)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$ vec 2 scaloc x 2 xnorm ierr)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
                var-8 var-9 var-10 var-12 var-13))
              (setf scaloc var-11)
              (setf xnorm var-14)
              (setf ierr var-15))
            (if (/= ierr 0)
                (setf info 1))
            (cond
             ((/= scaloc one)
              (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                            ((> j n) nil)
                (tagbody
                  (multiple-value-bind (var-0 var-1 var-2 var-3)
                      (sscal m scaloc
                       (f2cl-lib:array-slice c-%data% single-float
                                             (1 j) ((1 ldc) (1 *))
                                             c-%offset%)
                       1)
                    (declare (ignore var-2 var-3))
                    (when var-0 (setf m var-0))
                    (when var-1 (setf scaloc var-1)))
                 label50))
              (setf scale (* scale scaloc))))
            (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                                 c-%offset%)
                    (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
            (setf (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                                 c-%offset%)
                    (f2cl-lib:fref x (1 2) ((1 2) (1 2))))
            (setf (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                                 c-%offset%)
                    (f2cl-lib:fref x (2 1) ((1 2) (1 2))))
            (setf (f2cl-lib:fref c-%data% (k2 l2) ((1 ldc) (1 *))
                                 c-%offset%)
                    (f2cl-lib:fref x (2 2) ((1 2) (1 2))))))
        label60))
    label70)))
```

```
((and (not notrna) notrnb) (setf lnext 1)
 (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
               ((> l n) nil)
   (tagbody
     (if (< l lnext)
         (go label130))
     (cond ((= l n) (setf l1 l) (setf l2 l))
           (t
            (cond
             ((/=
               (f2cl-lib:fref b ((f2cl-lib:int-add l 1) l)
                              ((1 ldb$) (1 *)))
               zero)
              (setf l1 l) (setf l2 (f2cl-lib:int-add l 1))
              (setf lnext (f2cl-lib:int-add l 2)))
             (t (setf l1 l) (setf l2 l)
              (setf lnext (f2cl-lib:int-add l 1))))))
     (setf knext 1)
     (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                   ((> k m) nil)
       (tagbody
         (if (< k knext)
             (go label120))
         (cond ((= k m) (setf k1 k) (setf k2 k))
               (t
                (cond
                 ((/=
                   (f2cl-lib:fref a ((f2cl-lib:int-add k 1) k)
                                  ((1 lda) (1 *)))
                   zero)
                  (setf k1 k) (setf k2 (f2cl-lib:int-add k 1))
                  (setf knext (f2cl-lib:int-add k 2)))
                 (t (setf k1 k) (setf k2 k)
                  (setf knext (f2cl-lib:int-add k 1))))))
         (cond
          ((and (= l1 l2) (= k1 k2))
           (setf suml
                   (sdot (f2cl-lib:int-sub k1 1)
                    (f2cl-lib:array-slice a-%data% single-float (1 k1)
                                          ((1 lda) (1 *)) a-%offset%)
                    1
                    (f2cl-lib:array-slice c-%data% single-float (1 l1)
                                          ((1 ldc) (1 *)) c-%offset%)
                    1))
           (setf sumr
                   (multiple-value-bind
                       (ret-val var-0 var-1 var-2 var-3 var-4)
                       (sdot (f2cl-lib:int-sub l1 1)
                        (f2cl-lib:array-slice c-%data% single-float
                                              (k1 1) ((1 ldc) (1 *))
```

```
                                    c-%offset%)
                ldc
                (f2cl-lib:array-slice b-%data% single-float
                                      (1 l1) ((1 ldb$) (1 *))
                                      b-%offset%)
                1)
             (declare (ignore var-0 var-1 var-3 var-4))
             (when var-2 (setf ldc var-2))
             ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf scaloc one)
(setf a11
        (+
         (f2cl-lib:fref a-%data% (k1 k1) ((1 lda) (1 *))
                        a-%offset%)
         (* sgn
            (f2cl-lib:fref b-%data% (l1 l1)
                           ((1 ldb$) (1 *)) b-%offset%))))
(setf da11 (abs a11))
(cond
 ((<= da11 smin) (setf a11 smin) (setf da11 smin)
  (setf info 1)))
(setf db (abs (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))))
(cond
 ((and (< da11 one) (> db one))
  (if (> db (* bignum da11))
      (setf scaloc (/ one db)))))
(setf (f2cl-lib:fref x (1 1) ((1 2) (1 2)))
        (/
         (* (f2cl-lib:fref vec (1 1) ((1 2) (1 2))) scaloc)
         a11))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal m scaloc
           (f2cl-lib:array-slice c-%data% single-float
                                 (1 j) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
     label80))
```

```
  (setf scale (* scale scaloc))))
 (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                      c-%offset%)
        (f2cl-lib:fref x (1 1) ((1 2) (1 2)))))
((and (= l1 l2) (/= k1 k2))
 (setf suml
        (sdot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% single-float (1 k1)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% single-float (1 l1)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
 (setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k1 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                   (1 l1) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
 (setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
 (setf suml
        (sdot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% single-float (1 k2)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% single-float (1 l1)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
 (setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k2 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
```

```
                                        (1 l1) ((1 ldb$) (1 *))
                                        b-%offset%)
                  1)
              (declare (ignore var-0 var-1 var-3 var-4))
              (when var-2 (setf ldc var-2))
              ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
    (slaln2 f2cl-lib:%true% 2 1 smin one
     (f2cl-lib:array-slice a-%data% single-float (k1 k1)
                           ((1 lda) (1 *)) a-%offset%)
     lda one one vec 2
     (* (- sgn)
        (f2cl-lib:fref b-%data% (l1 l1) ((1 ldb$) (1 *))
                       b-%offset%))
     zero x 2 scaloc xnorm ierr)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
    var-8 var-9 var-10 var-11 var-12 var-13 var-14))
  (setf scaloc var-15)
  (setf xnorm var-16)
  (setf ierr var-17))
(if (/= ierr 0)
    (setf info 1))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal m scaloc
           (f2cl-lib:array-slice c-%data% single-float
                                 (1 j) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
     label90))
  (setf scale (* scale scaloc))))
(setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
```

```
 (setf (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                      c-%offset%)
        (f2cl-lib:fref x (2 1) ((1 2) (1 2)))))
((and (/= l1 l2) (= k1 k2))
 (setf suml
        (sdot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% single-float (1 k1)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% single-float (1 l1)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
 (setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k1 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                   (1 l1) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
 (setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (* sgn
           (-
            (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                           c-%offset%)
            (+ suml (* sgn sumr)))))
 (setf suml
        (sdot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% single-float (1 k1)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% single-float (1 l2)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
 (setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k1 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
```

```
                                    (1 l2) ((1 ldb$) (1 *))
                                    b-%offset%)
                 1)
              (declare (ignore var-0 var-1 var-3 var-4))
              (when var-2 (setf ldc var-2))
              ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (* sgn
           (-
            (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                           c-%offset%)
            (+ suml (* sgn sumr)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
    (slaln2 f2cl-lib:%true% 2 1 smin one
     (f2cl-lib:array-slice b-%data% single-float (l1 l1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ one one vec 2
     (* (- sgn)
        (f2cl-lib:fref a-%data% (k1 k1) ((1 lda) (1 *))
                       a-%offset%))
     zero x 2 scaloc xnorm ierr)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
    var-8 var-9 var-10 var-11 var-12 var-13 var-14))
  (setf scaloc var-15)
  (setf xnorm var-16)
  (setf ierr var-17))
(if (/= ierr 0)
    (setf info 1))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal m scaloc
           (f2cl-lib:array-slice c-%data% single-float
                                 (1 j) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
     label100))
  (setf scale (* scale scaloc))))
(setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                     c-%offset%)
```

```
        (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
 (setf (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                      c-%offset%)
        (f2cl-lib:fref x (2 1) ((1 2) (1 2)))))
((and (/= l1 l2) (/= k1 k2))
 (setf suml
        (sdot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% single-float (1 k1)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% single-float (1 l1)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
 (setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k1 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                   (1 l1) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
 (setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
 (setf suml
        (sdot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% single-float (1 k1)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% single-float (1 l2)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
 (setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k1 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
```

```
                                    (1 l2) ((1 ldb$) (1 *))
                                    b-%offset%)
                1)
             (declare (ignore var-0 var-1 var-3 var-4))
             (when var-2 (setf ldc var-2))
             ret-val))
(setf (f2cl-lib:fref vec (1 2) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (sdot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% single-float (1 k2)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% single-float (1 l1)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub l1 1)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k2 1) ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                   (1 l1) ((1 ldb$) (1 *))
                                   b-%offset%)
             1)
           (declare (ignore var-0 var-1 var-3 var-4))
           (when var-2 (setf ldc var-2))
           ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (sdot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% single-float (1 k2)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% single-float (1 l2)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
```

```
                (sdot (f2cl-lib:int-sub l1 1)
                 (f2cl-lib:array-slice c-%data% single-float
                                       (k2 1) ((1 ldc) (1 *))
                                       c-%offset%)
                 ldc
                 (f2cl-lib:array-slice b-%data% single-float
                                       (1 l2) ((1 ldb$) (1 *))
                                       b-%offset%)
                 1)
               (declare (ignore var-0 var-1 var-3 var-4))
               (when var-2 (setf ldc var-2))
               ret-val))
(setf (f2cl-lib:fref vec (2 2) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l2) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15)
    (slasy2 f2cl-lib:%true% f2cl-lib:%false% isgn 2 2
     (f2cl-lib:array-slice a-%data% single-float (k1 k1)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% single-float (l1 l1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ vec 2 scaloc x 2 xnorm ierr)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
    var-8 var-9 var-10 var-12 var-13))
  (setf scaloc var-11)
  (setf xnorm var-14)
  (setf ierr var-15))
(if (/= ierr 0)
    (setf info 1))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal m scaloc
           (f2cl-lib:array-slice c-%data% single-float
                                 (1 j) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
     label110))
```

```
                (setf scale (* scale scaloc))))
              (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                                    c-%offset%)
                      (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
              (setf (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                                    c-%offset%)
                      (f2cl-lib:fref x (1 2) ((1 2) (1 2))))
              (setf (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                                    c-%offset%)
                      (f2cl-lib:fref x (2 1) ((1 2) (1 2))))
              (setf (f2cl-lib:fref c-%data% (k2 l2) ((1 ldc) (1 *))
                                    c-%offset%)
                      (f2cl-lib:fref x (2 2) ((1 2) (1 2))))))
          label120))
     label130)))
  ((and (not notrna) (not notrnb)) (setf lnext n)
   (f2cl-lib:fdo (l n (f2cl-lib:int-add l (f2cl-lib:int-sub 1)))
                 ((> l 1) nil)
     (tagbody
       (if (> l lnext)
           (go label190))
       (cond ((= l 1) (setf l1 l) (setf l2 l))
             (t
              (cond
               ((/=
                 (f2cl-lib:fref b
                                (l
                                 (f2cl-lib:int-add l
                                                   (f2cl-lib:int-sub
                                                    1)))
                                ((1 ldb$) (1 *)))
                 zero)
                (setf l1 (f2cl-lib:int-sub l 1)) (setf l2 l)
                (setf lnext (f2cl-lib:int-sub l 2)))
               (t (setf l1 l) (setf l2 l)
                (setf lnext (f2cl-lib:int-sub l 1))))))
       (setf knext 1)
       (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                     ((> k m) nil)
         (tagbody
           (if (< k knext)
               (go label180))
           (cond ((= k m) (setf k1 k) (setf k2 k))
                 (t
                  (cond
                   ((/=
                     (f2cl-lib:fref a ((f2cl-lib:int-add k 1) k)
                                    ((1 lda) (1 *)))
                     zero)
                    (setf k1 k) (setf k2 (f2cl-lib:int-add k 1))
```

```
                (setf knext (f2cl-lib:int-add k 2)))
               (t (setf k1 k) (setf k2 k)
                (setf knext (f2cl-lib:int-add k 1))))))
        (cond
         ((and (= l1 l2) (= k1 k2))
          (setf suml
                  (sdot (f2cl-lib:int-sub k1 1)
                   (f2cl-lib:array-slice a-%data% single-float (1 k1)
                                         ((1 lda) (1 *)) a-%offset%)
                   1
                   (f2cl-lib:array-slice c-%data% single-float (1 l1)
                                         ((1 ldc) (1 *)) c-%offset%)
                   1))
          (setf sumr
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4)
                      (sdot (f2cl-lib:int-sub n l1)
                       (f2cl-lib:array-slice c-%data% single-float
                                             (k1
                                              (min
                                               (the f2cl-lib:integer4
                                                    (f2cl-lib:int-add
                                                     l1 1))
                                               (the f2cl-lib:integer4
                                                    n)))
                                             ((1 ldc) (1 *))
                                             c-%offset%)
                       ldc
                       (f2cl-lib:array-slice b-%data% single-float
                                             (l1
                                              (min
                                               (the f2cl-lib:integer4
                                                    (f2cl-lib:int-add
                                                     l1 1))
                                               (the f2cl-lib:integer4
                                                    n)))
                                             ((1 ldb$) (1 *))
                                             b-%offset%)
                       ldb$)
                     (declare (ignore var-0 var-1 var-3))
                     (when var-2 (setf ldc var-2))
                     (when var-4 (setf ldb$ var-4))
                     ret-val))
          (setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
                  (-
                   (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                                  c-%offset%)
                   (+ suml (* sgn sumr))))
          (setf scaloc one)
          (setf a11
```

```
        (+
         (f2cl-lib:fref a-%data% (k1 k1) ((1 lda) (1 *))
                        a-%offset%)
         (* sgn
            (f2cl-lib:fref b-%data% (l1 l1)
                           ((1 ldb$) (1 *)) b-%offset%))))
 (setf da11 (abs a11))
 (cond
  ((<= da11 smin) (setf a11 smin) (setf da11 smin)
   (setf info 1)))
 (setf db (abs (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))))
 (cond
  ((and (< da11 one) (> db one))
   (if (> db (* bignum da11))
       (setf scaloc (/ one db)))))
 (setf (f2cl-lib:fref x (1 1) ((1 2) (1 2)))
        (/
         (* (f2cl-lib:fref vec (1 1) ((1 2) (1 2))) scaloc)
         a11))
 (cond
  ((/= scaloc one)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2 var-3)
           (sscal m scaloc
            (f2cl-lib:array-slice c-%data% single-float
                                  (1 j) ((1 ldc) (1 *))
                                  c-%offset%)
            1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
      label140))
   (setf scale (* scale scaloc))))
 (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                      c-%offset%)
        (f2cl-lib:fref x (1 1) ((1 2) (1 2)))))
((and (= l1 l2) (/= k1 k2))
 (setf suml
        (sdot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% single-float (1 k1)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% single-float (1 l1)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
 (setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
```

```
                (sdot (f2cl-lib:int-sub n l2)
                 (f2cl-lib:array-slice c-%data% single-float
                                       (k1
                                        (min
                                         (the f2cl-lib:integer4
                                              (f2cl-lib:int-add
                                               l2 1))
                                         (the f2cl-lib:integer4
                                              n)))
                                       ((1 ldc) (1 *))
                                       c-%offset%)
                 ldc
                 (f2cl-lib:array-slice b-%data% single-float
                                       (l1
                                        (min
                                         (the f2cl-lib:integer4
                                              (f2cl-lib:int-add
                                               l2 1))
                                         (the f2cl-lib:integer4
                                              n)))
                                       ((1 ldb$) (1 *))
                                       b-%offset%)
                 ldb$)
                (declare (ignore var-0 var-1 var-3))
                (when var-2 (setf ldc var-2))
                (when var-4 (setf ldb$ var-4))
                ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (sdot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% single-float (1 k2)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% single-float (1 l1)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k2
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
```

```
                                        (the f2cl-lib:integer4
                                             n)))
                                     ((1 ldc) (1 *))
                                     c-%offset%)
                  ldc
                  (f2cl-lib:array-slice b-%data% single-float
                                     (l1
                                      (min
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-add
                                              l2 1))
                                        (the f2cl-lib:integer4
                                             n)))
                                     ((1 ldb$) (1 *))
                                     b-%offset%)
                  ldb$)
               (declare (ignore var-0 var-1 var-3))
               (when var-2 (setf ldc var-2))
               (when var-4 (setf ldb$ var-4))
               ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
    (slaln2 f2cl-lib:%true% 2 1 smin one
     (f2cl-lib:array-slice a-%data% single-float (k1 k1)
                           ((1 lda) (1 *)) a-%offset%)
     lda one one vec 2
     (* (- sgn)
        (f2cl-lib:fref b-%data% (l1 l1) ((1 ldb$) (1 *))
                       b-%offset%))
     zero x 2 scaloc xnorm ierr)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
    var-8 var-9 var-10 var-11 var-12 var-13 var-14))
  (setf scaloc var-15)
  (setf xnorm var-16)
  (setf ierr var-17))
(if (/= ierr 0)
    (setf info 1))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
```

```
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal m scaloc
             (f2cl-lib:array-slice c-%data% single-float
                                   (1 j) ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
      label150))
    (setf scale (* scale scaloc))))
  (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                       c-%offset%)
          (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
  (setf (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                       c-%offset%)
          (f2cl-lib:fref x (2 1) ((1 2) (1 2)))))
 ((and (/= l1 l2) (= k1 k2))
  (setf suml
          (sdot (f2cl-lib:int-sub k1 1)
           (f2cl-lib:array-slice a-%data% single-float (1 k1)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice c-%data% single-float (1 l1)
                                 ((1 ldc) (1 *)) c-%offset%)
           1))
  (setf sumr
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4)
              (sdot (f2cl-lib:int-sub n l2)
               (f2cl-lib:array-slice c-%data% single-float
                                     (k1
                                      (min
                                       (the f2cl-lib:integer4
                                            (f2cl-lib:int-add
                                             l2 1))
                                       (the f2cl-lib:integer4
                                            n)))
                                     ((1 ldc) (1 *))
                                     c-%offset%)
               ldc
               (f2cl-lib:array-slice b-%data% single-float
                                     (l1
                                      (min
                                       (the f2cl-lib:integer4
                                            (f2cl-lib:int-add
                                             l2 1))
                                       (the f2cl-lib:integer4
                                            n)))
                                     ((1 ldb$) (1 *))
```

```
                                    b-%offset%)
              ldb$)
           (declare (ignore var-0 var-1 var-3))
           (when var-2 (setf ldc var-2))
           (when var-4 (setf ldb$ var-4))
           ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (* sgn
           (-
            (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                           c-%offset%)
            (+ suml (* sgn sumr)))))
(setf suml
        (sdot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% single-float (1 k1)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% single-float (1 l2)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                   (l2
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldb$) (1 *))
                                   b-%offset%)
             ldb$)
           (declare (ignore var-0 var-1 var-3))
           (when var-2 (setf ldc var-2))
           (when var-4 (setf ldb$ var-4))
           ret-val))
```

```
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (* sgn
           (-
            (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                           c-%offset%)
            (+ suml (* sgn sumr)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
    (slaln2 f2cl-lib:%false% 2 1 smin one
     (f2cl-lib:array-slice b-%data% single-float (l1 l1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ one one vec 2
     (* (- sgn)
        (f2cl-lib:fref a-%data% (k1 k1) ((1 lda) (1 *))
                       a-%offset%))
     zero x 2 scaloc xnorm ierr)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
    var-8 var-9 var-10 var-11 var-12 var-13 var-14))
  (setf scaloc var-15)
  (setf xnorm var-16)
  (setf ierr var-17))
(if (/= ierr 0)
    (setf info 1))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal m scaloc
           (f2cl-lib:array-slice c-%data% single-float
                                 (1 j) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
     label160))
  (setf scale (* scale scaloc))))
(setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
(setf (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref x (2 1) ((1 2) (1 2)))))
((and (/= l1 l2) (/= k1 k2))
 (setf suml
```

```
        (sdot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% single-float (1 k1)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% single-float (1 l1)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                   (l1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldb$) (1 *))
                                   b-%offset%)
             ldb$)
          (declare (ignore var-0 var-1 var-3))
          (when var-2 (setf ldc var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (sdot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% single-float (1 k1)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% single-float (1 l2)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
```

```
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                   (l2
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldb$) (1 *))
                                   b-%offset%)
             ldb$)
          (declare (ignore var-0 var-1 var-3))
          (when var-2 (setf ldc var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf (f2cl-lib:fref vec (1 2) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (sdot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% single-float (1 k2)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% single-float (1 l1)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k2
                                    (min
```

```
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-add
                                              l2 1))
                                        (the f2cl-lib:integer4
                                             n)))
                                     ((1 ldc) (1 *))
                                     c-%offset%)
                 ldc
                 (f2cl-lib:array-slice b-%data% single-float
                                     (l1
                                      (min
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-add
                                              l2 1))
                                        (the f2cl-lib:integer4
                                             n)))
                                     ((1 ldb$) (1 *))
                                     b-%offset%)
                 ldb$)
              (declare (ignore var-0 var-1 var-3))
              (when var-2 (setf ldc var-2))
              (when var-4 (setf ldb$ var-4))
              ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (sdot (f2cl-lib:int-sub k1 1)
         (f2cl-lib:array-slice a-%data% single-float (1 k2)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% single-float (1 l2)
                               ((1 ldc) (1 *)) c-%offset%)
         1))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% single-float
                                     (k2
                                      (min
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-add
                                              l2 1))
                                        (the f2cl-lib:integer4
                                             n)))
                                     ((1 ldc) (1 *))
                                     c-%offset%)
```

```
                ldc
                (f2cl-lib:array-slice b-%data% single-float
                                      (l2
                                       (min
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-add
                                              l2 1))
                                        (the f2cl-lib:integer4
                                             n)))
                                      ((1 ldb$) (1 *))
                                      b-%offset%)
                ldb$)
            (declare (ignore var-0 var-1 var-3))
            (when var-2 (setf ldc var-2))
            (when var-4 (setf ldb$ var-4))
            ret-val))
(setf (f2cl-lib:fref vec (2 2) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l2) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15)
    (slasy2 f2cl-lib:%true% f2cl-lib:%true% isgn 2 2
     (f2cl-lib:array-slice a-%data% single-float (k1 k1)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% single-float (l1 l1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ vec 2 scaloc x 2 xnorm ierr)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
    var-8 var-9 var-10 var-12 var-13))
  (setf scaloc var-11)
  (setf xnorm var-14)
  (setf ierr var-15))
(if (/= ierr 0)
    (setf info 1))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal m scaloc
           (f2cl-lib:array-slice c-%data% single-float
                                 (1 j) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
```

```
                    (declare (ignore var-2 var-3))
                    (when var-0 (setf m var-0))
                    (when var-1 (setf scaloc var-1)))
                label170))
              (setf scale (* scale scaloc))))
            (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                                  c-%offset%)
                    (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
            (setf (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                                  c-%offset%)
                    (f2cl-lib:fref x (1 2) ((1 2) (1 2))))
            (setf (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                                  c-%offset%)
                    (f2cl-lib:fref x (2 1) ((1 2) (1 2))))
            (setf (f2cl-lib:fref c-%data% (k2 l2) ((1 ldc) (1 *))
                                  c-%offset%)
                    (f2cl-lib:fref x (2 2) ((1 2) (1 2))))))
        label180))
    label190)))
((and notrna (not notrnb)) (setf lnext n)
 (f2cl-lib:fdo (l n (f2cl-lib:int-add l (f2cl-lib:int-sub 1)))
               ((> l 1) nil)
   (tagbody
     (if (> l lnext)
         (go label250))
     (cond ((= l 1) (setf l1 l) (setf l2 l))
           (t
            (cond
             ((/=
               (f2cl-lib:fref b
                              (l
                               (f2cl-lib:int-add l
                                                 (f2cl-lib:int-sub
                                                  1)))
                              ((1 ldb$) (1 *)))
               zero)
              (setf l1 (f2cl-lib:int-sub l 1)) (setf l2 l)
              (setf lnext (f2cl-lib:int-sub l 2)))
             (t (setf l1 l) (setf l2 l)
              (setf lnext (f2cl-lib:int-sub l 1))))))
     (setf knext m)
     (f2cl-lib:fdo (k m (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                   ((> k 1) nil)
       (tagbody
         (if (> k knext)
             (go label240))
         (cond ((= k 1) (setf k1 k) (setf k2 k))
               (t
                (cond
                 ((/=
```

```
                (f2cl-lib:fref a
                               (k
                                (f2cl-lib:int-add k
                                                  (f2cl-lib:int-sub
                                                   1)))
                               ((1 lda) (1 *)))
                zero)
               (setf k1 (f2cl-lib:int-sub k 1)) (setf k2 k)
               (setf knext (f2cl-lib:int-sub k 2)))
              (t (setf k1 k) (setf k2 k)
               (setf knext (f2cl-lib:int-sub k 1))))))
(cond
 ((and (= l1 l2) (= k1 k2))
  (setf suml
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4)
              (sdot (f2cl-lib:int-sub m k1)
               (f2cl-lib:array-slice a-%data% single-float
                                     (k1
                                      (min
                                       (the f2cl-lib:integer4
                                            (f2cl-lib:int-add
                                             k1 1))
                                       (the f2cl-lib:integer4
                                            m)))
                                     ((1 lda) (1 *))
                                     a-%offset%)
               lda
               (f2cl-lib:array-slice c-%data% single-float
                                     ((min
                                       (f2cl-lib:int-add k1
                                                         1)
                                       m)
                                      l1)
                                     ((1 ldc) (1 *))
                                     c-%offset%)
               1)
             (declare (ignore var-0 var-1 var-3 var-4))
             (when var-2 (setf lda var-2))
             ret-val))
  (setf sumr
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4)
              (sdot (f2cl-lib:int-sub n l1)
               (f2cl-lib:array-slice c-%data% single-float
                                     (k1
                                      (min
                                       (the f2cl-lib:integer4
                                            (f2cl-lib:int-add
                                             l1 1))
```

```
                                        (the f2cl-lib:integer4
                                             n)))
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                 ldc
                 (f2cl-lib:array-slice b-%data% single-float
                                      (l1
                                       (min
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-add
                                              l1 1))
                                        (the f2cl-lib:integer4
                                             n)))
                                      ((1 ldb$) (1 *))
                                      b-%offset%)
                 ldb$)
              (declare (ignore var-0 var-1 var-3))
              (when var-2 (setf ldc var-2))
              (when var-4 (setf ldb$ var-4))
              ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf scaloc one)
(setf a11
        (+
         (f2cl-lib:fref a-%data% (k1 k1) ((1 lda) (1 *))
                        a-%offset%)
         (* sgn
            (f2cl-lib:fref b-%data% (l1 l1)
                           ((1 ldb$) (1 *)) b-%offset%))))
(setf da11 (abs a11))
(cond
 ((<= da11 smin) (setf a11 smin) (setf da11 smin)
  (setf info 1)))
(setf db (abs (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))))
(cond
 ((and (< da11 one) (> db one))
  (if (> db (* bignum da11))
      (setf scaloc (/ one db)))))
(setf (f2cl-lib:fref x (1 1) ((1 2) (1 2)))
        (/
         (* (f2cl-lib:fref vec (1 1) ((1 2) (1 2))) scaloc)
         a11))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
```

```
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal m scaloc
             (f2cl-lib:array-slice c-%data% single-float
                                   (1 j) ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
       label200))
    (setf scale (* scale scaloc))))
  (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                       c-%offset%)
          (f2cl-lib:fref x (1 1) ((1 2) (1 2)))))
 ((and (= l1 l2) (/= k1 k2))
  (setf suml
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4)
              (sdot (f2cl-lib:int-sub m k2)
               (f2cl-lib:array-slice a-%data% single-float
                                     (k1
                                      (min
                                       (the f2cl-lib:integer4
                                            (f2cl-lib:int-add
                                             k2 1))
                                       (the f2cl-lib:integer4
                                            m)))
                                     ((1 lda) (1 *))
                                     a-%offset%)
               lda
               (f2cl-lib:array-slice c-%data% single-float
                                     ((min
                                       (f2cl-lib:int-add k2
                                                         1)
                                       m)
                                      l1)
                                     ((1 ldc) (1 *))
                                     c-%offset%)
               1)
            (declare (ignore var-0 var-1 var-3 var-4))
            (when var-2 (setf lda var-2))
            ret-val))
  (setf sumr
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4)
              (sdot (f2cl-lib:int-sub n l2)
               (f2cl-lib:array-slice c-%data% single-float
                                     (k1
                                      (min
```

```
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-add
                                              l2 1))
                                        (the f2cl-lib:integer4
                                             n)))
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                 ldc
                 (f2cl-lib:array-slice b-%data% single-float
                                      (l1
                                       (min
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-add
                                              l2 1))
                                        (the f2cl-lib:integer4
                                             n)))
                                      ((1 ldb$) (1 *))
                                      b-%offset%)
                 ldb$)
              (declare (ignore var-0 var-1 var-3))
              (when var-2 (setf ldc var-2))
              (when var-4 (setf ldb$ var-4))
              ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub m k2)
             (f2cl-lib:array-slice a-%data% single-float
                                      (k2
                                       (min
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-add
                                              k2 1))
                                        (the f2cl-lib:integer4
                                             m)))
                                      ((1 lda) (1 *))
                                      a-%offset%)
                 lda
                 (f2cl-lib:array-slice c-%data% single-float
                                      ((min
                                        (f2cl-lib:int-add k2
                                                          1)
                                        m)
                                       l1)
                                      ((1 ldc) (1 *))
```

```
                                        c-%offset%)
                 1)
              (declare (ignore var-0 var-1 var-3 var-4))
              (when var-2 (setf lda var-2))
              ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k2
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                   (l1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldb$) (1 *))
                                   b-%offset%)
             ldb$)
              (declare (ignore var-0 var-1 var-3))
              (when var-2 (setf ldc var-2))
              (when var-4 (setf ldb$ var-4))
              ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
    (slaln2 f2cl-lib:%false% 2 1 smin one
     (f2cl-lib:array-slice a-%data% single-float (k1 k1)
                           ((1 lda) (1 *)) a-%offset%)
     lda one one vec 2
     (* (- sgn)
        (f2cl-lib:fref b-%data% (l1 l1) ((1 ldb$) (1 *))
```

```
                        b-%offset%))
      zero x 2 scaloc xnorm ierr)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
      var-8 var-9 var-10 var-11 var-12 var-13 var-14))
    (setf scaloc var-15)
    (setf xnorm var-16)
    (setf ierr var-17))
  (if (/= ierr 0)
      (setf info 1))
  (cond
   ((/= scaloc one)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal m scaloc
             (f2cl-lib:array-slice c-%data% single-float
                                   (1 j) ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
      label210))
    (setf scale (* scale scaloc))))
  (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                       c-%offset%)
          (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
  (setf (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                       c-%offset%)
          (f2cl-lib:fref x (2 1) ((1 2) (1 2)))))
 ((and (/= l1 l2) (= k1 k2))
  (setf suml
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4)
              (sdot (f2cl-lib:int-sub m k1)
               (f2cl-lib:array-slice a-%data% single-float
                                     (k1
                                      (min
                                       (the f2cl-lib:integer4
                                            (f2cl-lib:int-add
                                             k1 1))
                                       (the f2cl-lib:integer4
                                            m)))
                                     ((1 lda) (1 *))
                                     a-%offset%)
               lda
               (f2cl-lib:array-slice c-%data% single-float
                                     ((min
```

```
                                        (f2cl-lib:int-add k1
                                                          1)
                                       m)
                                      l1)
                                     ((1 ldc) (1 *))
                                     c-%offset%)
                 1)
              (declare (ignore var-0 var-1 var-3 var-4))
              (when var-2 (setf lda var-2))
              ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                   (l1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldb$) (1 *))
                                   b-%offset%)
             ldb$)
              (declare (ignore var-0 var-1 var-3))
              (when var-2 (setf ldc var-2))
              (when var-4 (setf ldb$ var-4))
              ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (* sgn
           (-
            (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                           c-%offset%)
            (+ suml (* sgn sumr)))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub m k1)
```

```
                    (f2cl-lib:array-slice a-%data% single-float
                                          (k1
                                           (min
                                            (the f2cl-lib:integer4
                                                 (f2cl-lib:int-add
                                                  k1 1))
                                            (the f2cl-lib:integer4
                                                 m)))
                                          ((1 lda) (1 *))
                                          a-%offset%)
                    lda
                    (f2cl-lib:array-slice c-%data% single-float
                                          ((min
                                            (f2cl-lib:int-add k1
                                                              1)
                                            m)
                                           l2)
                                          ((1 ldc) (1 *))
                                          c-%offset%)
                    1)
                  (declare (ignore var-0 var-1 var-3 var-4))
                  (when var-2 (setf lda var-2))
                  ret-val))
          (setf sumr
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4)
                      (sdot (f2cl-lib:int-sub n l2)
                       (f2cl-lib:array-slice c-%data% single-float
                                             (k1
                                              (min
                                               (the f2cl-lib:integer4
                                                    (f2cl-lib:int-add
                                                     l2 1))
                                               (the f2cl-lib:integer4
                                                    n)))
                                             ((1 ldc) (1 *))
                                             c-%offset%)
                       ldc
                       (f2cl-lib:array-slice b-%data% single-float
                                             (l2
                                              (min
                                               (the f2cl-lib:integer4
                                                    (f2cl-lib:int-add
                                                     l2 1))
                                               (the f2cl-lib:integer4
                                                    n)))
                                             ((1 ldb$) (1 *))
                                             b-%offset%)
                       ldb$)
                  (declare (ignore var-0 var-1 var-3))
```

```
          (when var-2 (setf ldc var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (* sgn
           (-
            (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                           c-%offset%)
            (+ suml (* sgn sumr)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
    (slaln2 f2cl-lib:%false% 2 1 smin one
     (f2cl-lib:array-slice b-%data% single-float (l1 l1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$ one one vec 2
     (* (- sgn)
        (f2cl-lib:fref a-%data% (k1 k1) ((1 lda) (1 *))
                       a-%offset%))
     zero x 2 scaloc xnorm ierr)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
    var-8 var-9 var-10 var-11 var-12 var-13 var-14))
  (setf scaloc var-15)
  (setf xnorm var-16)
  (setf ierr var-17))
(if (/= ierr 0)
    (setf info 1))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal m scaloc
           (f2cl-lib:array-slice c-%data% single-float
                                 (1 j) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
     label220))
  (setf scale (* scale scaloc))))
(setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
(setf (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                     c-%offset%)
```

```
        (f2cl-lib:fref x (2 1) ((1 2) (1 2)))))
((and (/= l1 l2) (/= k1 k2))
 (setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub m k2)
             (f2cl-lib:array-slice a-%data% single-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           k2 1))
                                     (the f2cl-lib:integer4
                                          m)))
                                   ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice c-%data% single-float
                                   ((min
                                     (f2cl-lib:int-add k2
                                                       1)
                                     m)
                                    l1)
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf lda var-2))
          ret-val))
 (setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                   (l1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
```

```
                                        (the f2cl-lib:integer4
                                             n)))
                                    ((1 ldb$) (1 *))
                                    b-%offset%)
                ldb$)
            (declare (ignore var-0 var-1 var-3))
            (when var-2 (setf ldc var-2))
            (when var-4 (setf ldb$ var-4))
            ret-val))
(setf (f2cl-lib:fref vec (1 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub m k2)
             (f2cl-lib:array-slice a-%data% single-float
                                    (k1
                                     (min
                                      (the f2cl-lib:integer4
                                           (f2cl-lib:int-add
                                            k2 1))
                                      (the f2cl-lib:integer4
                                           m)))
                                    ((1 lda) (1 *))
                                    a-%offset%)
             lda
             (f2cl-lib:array-slice c-%data% single-float
                                    ((min
                                      (f2cl-lib:int-add k2
                                                        1)
                                      m)
                                     l2)
                                    ((1 ldc) (1 *))
                                    c-%offset%)
             1)
            (declare (ignore var-0 var-1 var-3 var-4))
            (when var-2 (setf lda var-2))
            ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% single-float
                                    (k1
                                     (min
                                      (the f2cl-lib:integer4
                                           (f2cl-lib:int-add
```

```
                                                 l2 1))
                                            (the f2cl-lib:integer4
                                                 n)))
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                        ldc
                        (f2cl-lib:array-slice b-%data% single-float
                                        (l2
                                         (min
                                          (the f2cl-lib:integer4
                                               (f2cl-lib:int-add
                                                l2 1))
                                          (the f2cl-lib:integer4
                                               n)))
                                        ((1 ldb$) (1 *))
                                        b-%offset%)
                        ldb$)
                    (declare (ignore var-0 var-1 var-3))
                    (when var-2 (setf ldc var-2))
                    (when var-4 (setf ldb$ var-4))
                    ret-val))
(setf (f2cl-lib:fref vec (1 2) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub m k2)
             (f2cl-lib:array-slice a-%data% single-float
                                        (k2
                                         (min
                                          (the f2cl-lib:integer4
                                               (f2cl-lib:int-add
                                                k2 1))
                                          (the f2cl-lib:integer4
                                               m)))
                                        ((1 lda) (1 *))
                                        a-%offset%)
             lda
             (f2cl-lib:array-slice c-%data% single-float
                                        ((min
                                          (f2cl-lib:int-add k2
                                                            1)
                                          m)
                                         l1)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
             1)
```

```
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf lda var-2))
          ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% single-float
                                   (k2
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                   (l1
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           l2 1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldb$) (1 *))
                                   b-%offset%)
             ldb$)
          (declare (ignore var-0 var-1 var-3))
          (when var-2 (setf ldc var-2))
          (when var-4 (setf ldb$ var-4))
          ret-val))
(setf (f2cl-lib:fref vec (2 1) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf suml
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub m k2)
             (f2cl-lib:array-slice a-%data% single-float
                                   (k2
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add
                                           k2 1))
                                     (the f2cl-lib:integer4
                                          m)))
```

```
                                        ((1 lda) (1 *))
                                        a-%offset%)
                    lda
                    (f2cl-lib:array-slice c-%data% single-float
                                        ((min
                                          (f2cl-lib:int-add k2
                                                            1)
                                          m)
                                         l2)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                    1)
                  (declare (ignore var-0 var-1 var-3 var-4))
                  (when var-2 (setf lda var-2))
                  ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (sdot (f2cl-lib:int-sub n l2)
             (f2cl-lib:array-slice c-%data% single-float
                                        (k2
                                         (min
                                          (the f2cl-lib:integer4
                                               (f2cl-lib:int-add
                                                l2 1))
                                          (the f2cl-lib:integer4
                                               n)))
                                        ((1 ldc) (1 *))
                                        c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% single-float
                                        (l2
                                         (min
                                          (the f2cl-lib:integer4
                                               (f2cl-lib:int-add
                                                l2 1))
                                          (the f2cl-lib:integer4
                                               n)))
                                        ((1 ldb$) (1 *))
                                        b-%offset%)
             ldb$)
            (declare (ignore var-0 var-1 var-3))
            (when var-2 (setf ldc var-2))
            (when var-4 (setf ldb$ var-4))
            ret-val))
(setf (f2cl-lib:fref vec (2 2) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref c-%data% (k2 l2) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
```

```
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                     var-9 var-10 var-11 var-12 var-13 var-14 var-15)
                    (slasy2 f2cl-lib:%false% f2cl-lib:%true% isgn 2 2
                     (f2cl-lib:array-slice a-%data% single-float (k1 k1)
                                           ((1 lda) (1 *)) a-%offset%)
                     lda
                     (f2cl-lib:array-slice b-%data% single-float (l1 l1)
                                           ((1 ldb$) (1 *)) b-%offset%)
                     ldb$ vec 2 scaloc x 2 xnorm ierr)
                  (declare
                   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
                    var-8 var-9 var-10 var-12 var-13))
                  (setf scaloc var-11)
                  (setf xnorm var-14)
                  (setf ierr var-15))
                (if (/= ierr 0)
                    (setf info 1))
                (cond
                 ((/= scaloc one)
                  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                                ((> j n) nil)
                    (tagbody
                      (multiple-value-bind (var-0 var-1 var-2 var-3)
                          (sscal m scaloc
                           (f2cl-lib:array-slice c-%data% single-float
                                                 (1 j) ((1 ldc) (1 *))
                                                 c-%offset%)
                           1)
                        (declare (ignore var-2 var-3))
                        (when var-0 (setf m var-0))
                        (when var-1 (setf scaloc var-1)))
                     label230))
                  (setf scale (* scale scaloc))))
                (setf (f2cl-lib:fref c-%data% (k1 l1) ((1 ldc) (1 *))
                                     c-%offset%)
                        (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
                (setf (f2cl-lib:fref c-%data% (k1 l2) ((1 ldc) (1 *))
                                     c-%offset%)
                        (f2cl-lib:fref x (1 2) ((1 2) (1 2))))
                (setf (f2cl-lib:fref c-%data% (k2 l1) ((1 ldc) (1 *))
                                     c-%offset%)
                        (f2cl-lib:fref x (2 1) ((1 2) (1 2))))
                (setf (f2cl-lib:fref c-%data% (k2 l2) ((1 ldc) (1 *))
                                     c-%offset%)
                        (f2cl-lib:fref x (2 2) ((1 2) (1 2))))))
            label240))
        label250))))
 (go end_label)
end_label
```

```
        (return
         (values trana tranb nil m n nil lda nil ldb$ nil ldc scale info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::strsyl
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::trana
                                              fortran-to-lisp::tranb nil
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$ nil
                                              fortran-to-lisp::ldc
                                              fortran-to-lisp::scale
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::slasy2
                                              fortran-to-lisp::slaln2
                                              fortran-to-lisp::slange
                                              fortran-to-lisp::slabad))))
```

Chapter 60

LAPACK - Symmetric Matrix, Comp. Routines, Complex

60.1 cla_syamv

60.1.1 Axiom unit tests

— **clasyamv.input** —

```
)set break resume
)sys rm -f clasyamv.output
)spool clasyamv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.1.2 Axiom help page

— **cla_syamv.help** —

```
==========================================================================
cla_syamv examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cla_syamv (UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)
           CLA_SYAMV computes a matrix-vector product using a symmetric
           indefinite matrix to calculate error bounds.

Function/Subroutine Documentation
   subroutine cla_syamv (integer UPLO, integer N, real ALPHA, complex,
       dimension( lda, * ) A, integer LDA, complex, dimension( * ) X, integer
       INCX, real BETA, real, dimension( * ) Y, integer INCY)
       CLA_SYAMV computes a matrix-vector product using a symmetric indefinite
       matrix to calculate error bounds.

Purpose:
   CLA_SYAMV  performs the matrix-vector operation

                    y := alpha*abs(A)*abs(x) + beta*abs(y),

   where alpha and beta are scalars, x and y are vectors and A is an
   n by n symmetric matrix.

   This function is primarily used in calculating error bounds.
   To protect against underflow during evaluation, components in
   the resulting vector are perturbed away from zero by (N+1)
   times the underflow threshold.  To prevent unnecessarily large
   errors for block-structure embedded in general matrices,
   "symbolically" zero components are not perturbed.  A zero
   entry is considered "symbolic" if all multiplications involved
   in computing that entry have at least one zero multiplicand.

Parameters:

 UPLO
   UPLO is INTEGER
   On entry, UPLO specifies whether the upper or lower
   triangular part of the array A is to be referenced as
   follows:

   UPLO = BLAS_UPPER   Only the upper triangular part of A is to be referenced.
   UPLO = BLAS_LOWER   Only the lower triangular part of A is to be referenced.

   Unchanged on exit.

 N
   N is INTEGER
```

```
  On entry, N specifies the number of columns of the matrix A.
  N must be at least zero.
  Unchanged on exit.

ALPHA
  ALPHA is REAL .
  On entry, ALPHA specifies the scalar alpha.
  Unchanged on exit.

A
  A is COMPLEX array of DIMENSION ( LDA, n ).
  Before entry, the leading m by n part of the array A must
  contain the matrix of coefficients.
  Unchanged on exit.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, n ).
  Unchanged on exit.

X
  X is COMPLEX array, dimension
  ( 1 + ( n - 1 )*abs( INCX ) )
  Before entry, the incremented array X must contain the vector x.
  Unchanged on exit.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of
  X. INCX must not be zero.
  Unchanged on exit.

BETA
  BETA is REAL .
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.
  Unchanged on exit.

Y
  Y is REAL array, dimension
  ( 1 + ( n - 1 )*abs( INCY ) )
  Before entry with BETA non-zero, the incremented array Y
  must contain the vector y. On exit, Y is overwritten by the
  updated vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of
  Y. INCY must not be zero.
```

```
Unchanged on exit.
```

———-

60.1.3 fortran code

— **cla_syamv.f** —

```
*  ===================================================================
      SUBROUTINE CLA_SYAMV( UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y,
     $                      INCY )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      REAL               ALPHA, BETA
      INTEGER            INCX, INCY, LDA, N
      INTEGER            UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), X( * )
      REAL               Y( * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            SYMB_ZERO
      REAL               TEMP, SAFE1
      INTEGER            I, INFO, IY, J, JX, KX, KY
      COMPLEX            ZDUM
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, SLAMCH
      REAL               SLAMCH
*     ..
*     .. External Functions ..
      EXTERNAL           ILAUPLO
      INTEGER            ILAUPLO
*     ..
```

```
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, ABS, SIGN, REAL, AIMAG
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( REAL ( ZDUM ) ) + ABS( AIMAG ( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF     ( UPLO.NE.ILAUPLO( 'U' ) .AND.
     $         UPLO.NE.ILAUPLO( 'L' ) )THEN
         INFO = 1
      ELSE IF( N.LT.0 )THEN
         INFO = 2
      ELSE IF( LDA.LT.MAX( 1, N ) )THEN
         INFO = 5
      ELSE IF( INCX.EQ.0 )THEN
         INFO = 7
      ELSE IF( INCY.EQ.0 )THEN
         INFO = 10
      END IF
      IF( INFO.NE.0 )THEN
         CALL XERBLA( 'SSYMV ', INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( ( N.EQ.0 ).OR.( ( ALPHA.EQ.ZERO ).AND.( BETA.EQ.ONE ) ) )
     $   RETURN
*
*     Set up the start points in  X  and  Y.
*
      IF( INCX.GT.0 )THEN
         KX = 1
      ELSE
         KX = 1 - ( N - 1 )*INCX
      END IF
      IF( INCY.GT.0 )THEN
         KY = 1
      ELSE
         KY = 1 - ( N - 1 )*INCY
      END IF
*
*     Set SAFE1 essentially to be the underflow threshold times the
```

```
*     number of additions in each row.
*
      SAFE1 = SLAMCH( 'Safe minimum' )
      SAFE1 = (N+1)*SAFE1
*
*     Form  y := alpha*abs(A)*abs(x) + beta*abs(y).
*
*     The O(N^2) SYMB_ZERO tests could be replaced by O(N) queries to
*     the inexact flag.  Still doesn't help change the iteration order
*     to per-column.
*
      IY = KY
      IF ( INCX.EQ.1 ) THEN
         IF ( UPLO .EQ. ILAUPLO( 'U' ) ) THEN
            DO I = 1, N
               IF ( BETA .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
                  Y( IY ) = 0.0
               ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
               ELSE
                  SYMB_ZERO = .FALSE.
                  Y( IY ) = BETA * ABS( Y( IY ) )
               END IF
               IF ( ALPHA .NE. ZERO ) THEN
                  DO J = 1, I
                     TEMP = CABS1( A( J, I ) )
                     SYMB_ZERO = SYMB_ZERO .AND.
     $                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                     Y( IY ) = Y( IY ) + ALPHA*CABS1( X( J ) )*TEMP
                  END DO
                  DO J = I+1, N
                     TEMP = CABS1( A( I, J ) )
                     SYMB_ZERO = SYMB_ZERO .AND.
     $                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                     Y( IY ) = Y( IY ) + ALPHA*CABS1( X( J ) )*TEMP
                  END DO
               END IF

               IF ( .NOT.SYMB_ZERO )
     $              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

               IY = IY + INCY
            END DO
         ELSE
            DO I = 1, N
               IF ( BETA .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
```

```
            Y( IY ) = 0.0
         ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
            SYMB_ZERO = .TRUE.
         ELSE
            SYMB_ZERO = .FALSE.
            Y( IY ) = BETA * ABS( Y( IY ) )
         END IF
         IF ( ALPHA .NE. ZERO ) THEN
            DO J = 1, I
               TEMP = CABS1( A( I, J ) )
               SYMB_ZERO = SYMB_ZERO .AND.
$                   ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

               Y( IY ) = Y( IY ) + ALPHA*CABS1( X( J ) )*TEMP
            END DO
            DO J = I+1, N
               TEMP = CABS1( A( J, I ) )
               SYMB_ZERO = SYMB_ZERO .AND.
$                   ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

               Y( IY ) = Y( IY ) + ALPHA*CABS1( X( J ) )*TEMP
            END DO
         END IF

         IF ( .NOT.SYMB_ZERO )
$              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

         IY = IY + INCY
      END DO
   END IF
 ELSE
   IF ( UPLO .EQ. ILAUPLO( 'U' ) ) THEN
      DO I = 1, N
         IF ( BETA .EQ. ZERO ) THEN
            SYMB_ZERO = .TRUE.
            Y( IY ) = 0.0
         ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
            SYMB_ZERO = .TRUE.
         ELSE
            SYMB_ZERO = .FALSE.
            Y( IY ) = BETA * ABS( Y( IY ) )
         END IF
         JX = KX
         IF ( ALPHA .NE. ZERO ) THEN
            DO J = 1, I
               TEMP = CABS1( A( J, I ) )
               SYMB_ZERO = SYMB_ZERO .AND.
$                   ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

               Y( IY ) = Y( IY ) + ALPHA*CABS1( X( JX ) )*TEMP
```

```
                JX = JX + INCX
             END DO
             DO J = I+1, N
                TEMP = CABS1( A( I, J ) )
                SYMB_ZERO = SYMB_ZERO .AND.
$                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                Y( IY ) = Y( IY ) + ALPHA*CABS1( X( JX ) )*TEMP
                JX = JX + INCX
             END DO
          END IF

          IF ( .NOT.SYMB_ZERO )
$              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

          IY = IY + INCY
       END DO
    ELSE
       DO I = 1, N
          IF ( BETA .EQ. ZERO ) THEN
             SYMB_ZERO = .TRUE.
             Y( IY ) = 0.0
          ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
             SYMB_ZERO = .TRUE.
          ELSE
             SYMB_ZERO = .FALSE.
             Y( IY ) = BETA * ABS( Y( IY ) )
          END IF
          JX = KX
          IF ( ALPHA .NE. ZERO ) THEN
             DO J = 1, I
                TEMP = CABS1( A( I, J ) )
                SYMB_ZERO = SYMB_ZERO .AND.
$                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                Y( IY ) = Y( IY ) + ALPHA*CABS1( X( JX ) )*TEMP
                JX = JX + INCX
             END DO
             DO J = I+1, N
                TEMP = CABS1( A( J, I ) )
                SYMB_ZERO = SYMB_ZERO .AND.
$                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                Y( IY ) = Y( IY ) + ALPHA*CABS1( X( JX ) )*TEMP
                JX = JX + INCX
             END DO
          END IF

          IF ( .NOT.SYMB_ZERO )
$              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )
```

```
               IY = IY + INCY
            END DO
         END IF

      END IF
*
      RETURN
*
*     End of CLA_SYAMV
*
      END
```

———-

60.1.4 lisp code

— **cla_syamv.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun cla_syamv (uplo n alpha a lda x incx beta y incy)
    (declare (type (array single-float (*)) y)
             (type (array f2cl-lib:complex8 (*)) x a)
             (type (single-float) beta alpha)
             (type (f2cl-lib:integer4) incy incx lda n uplo))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex8 a-%data% a-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%)
         (y single-float y-%data% y-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
          cabs1))
        (prog ((zdum #C(0.0 0.0)) (i 0) (info 0) (iy 0) (j 0) (jx 0) (kx 0)
               (ky 0) (temp 0.0) (safe1 0.0) (symb_zero nil))
          (declare (type (f2cl-lib:complex8) zdum)
                   (type (f2cl-lib:integer4) i info iy j jx kx ky)
                   (type (single-float) temp safe1)
                   (type f2cl-lib:logical symb_zero))
          (setf info 0)
          (cond
           ((and (/= uplo (ilauplo "U")) (/= uplo (ilauplo "L")))
            (setf info 1))
           ((< n 0) (setf info 2))
```

```
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info 5))
 ((= incx 0) (setf info 7)) ((= incy 0) (setf info 10)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "SSYMV " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= n 0) (and (= alpha zero) (= beta one)))
    (go end_label))
(cond ((> incx 0) (setf kx 1))
      (t
       (setf kx
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incx)))))
(cond ((> incy 0) (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incy)))))
(setf safe1 (slamch "Safe minimum"))
(setf safe1 (* (f2cl-lib:int-add n 1) safe1))
(setf iy ky)
(cond
 ((= incx 1)
  (cond
   ((= uplo (ilauplo "U"))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (cond
         ((= beta zero) (setf symb_zero f2cl-lib:%true%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  0.0))
         ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
          (setf symb_zero f2cl-lib:%true%))
         (t (setf symb_zero f2cl-lib:%false%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (* beta
                     (abs
                      (f2cl-lib:fref y-%data% (iy) ((1 *))
                                     y-%offset%))))))
        (cond
         ((/= alpha zero)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j i) nil)
```

```
      (tagbody
        (setf temp
                (cabs1
                 (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                a-%offset%)))
        (setf symb_zero
                (and symb_zero
                     (or
                      (=
                       (f2cl-lib:fref x-%data% (j) ((1 *))
                                      x-%offset%)
                       zero)
                      (= temp zero))))
        (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                (+
                 (f2cl-lib:fref y-%data% (iy) ((1 *))
                                y-%offset%)
                 (* alpha
                    (cabs1
                     (f2cl-lib:fref x-%data% (j) ((1 *))
                                    x-%offset%))
                    temp)))
       label100001))
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                   (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp
                (cabs1
                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)))
        (setf symb_zero
                (and symb_zero
                     (or
                      (=
                       (f2cl-lib:fref x-%data% (j) ((1 *))
                                      x-%offset%)
                       zero)
                      (= temp zero))))
        (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                (+
                 (f2cl-lib:fref y-%data% (iy) ((1 *))
                                y-%offset%)
                 (* alpha
                    (cabs1
                     (f2cl-lib:fref x-%data% (j) ((1 *))
                                    x-%offset%))
                    temp)))
       label100002))))
  (if (not symb_zero)
```

```
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+
                     (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (f2cl-lib:sign safe1
                                    (f2cl-lib:fref y-%data% (iy)
                                                   ((1 *))
                                                   y-%offset%)))))
        (setf iy (f2cl-lib:int-add iy incy))
       label100000)))
   (t
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (cond
         ((= beta zero) (setf symb_zero f2cl-lib:%true%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  0.0))
         ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
          (setf symb_zero f2cl-lib:%true%))
         (t (setf symb_zero f2cl-lib:%false%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (* beta
                     (abs
                      (f2cl-lib:fref y-%data% (iy) ((1 *))
                                     y-%offset%))))))
        (cond
         ((/= alpha zero)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j i) nil)
            (tagbody
              (setf temp
                      (cabs1
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)))
              (setf symb_zero
                      (and symb_zero
                           (or
                            (=
                             (f2cl-lib:fref x-%data% (j) ((1 *))
                                            x-%offset%)
                             zero)
                            (= temp zero))))
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (+
                       (f2cl-lib:fref y-%data% (iy) ((1 *))
                                      y-%offset%)
                       (* alpha
                          (cabs1
                           (f2cl-lib:fref x-%data% (j) ((1 *))
                                          x-%offset%))
```

```
                              temp)))
                 label100004))
              (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                             (f2cl-lib:int-add j 1))
                            ((> j n) nil)
                (tagbody
                  (setf temp
                          (cabs1
                           (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                          a-%offset%)))
                  (setf symb_zero
                          (and symb_zero
                               (or
                                (=
                                 (f2cl-lib:fref x-%data% (j) ((1 *))
                                                x-%offset%)
                                 zero)
                                (= temp zero))))
                  (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                          (+
                           (f2cl-lib:fref y-%data% (iy) ((1 *))
                                          y-%offset%)
                           (* alpha
                              (cabs1
                               (f2cl-lib:fref x-%data% (j) ((1 *))
                                              x-%offset%))
                              temp)))
                 label100005))))
            (if (not symb_zero)
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (f2cl-lib:sign safe1
                                        (f2cl-lib:fref y-%data% (iy)
                                                       ((1 *))
                                                       y-%offset%)))))
            (setf iy (f2cl-lib:int-add iy incy))
           label100003)))))
      (t
       (cond
        ((= uplo (ilauplo "U"))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (cond
              ((= beta zero) (setf symb_zero f2cl-lib:%true%)
               (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     0.0))
              ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
               (setf symb_zero f2cl-lib:%true%))
```

```
 (t (setf symb_zero f2cl-lib:%false%)
  (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
          (* beta
             (abs
              (f2cl-lib:fref y-%data% (iy) ((1 *))
                             y-%offset%))))))
(setf jx kx)
(cond
 ((/= alpha zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j i) nil)
    (tagbody
      (setf temp
              (cabs1
               (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                              a-%offset%)))
      (setf symb_zero
              (and symb_zero
                   (or
                    (=
                     (f2cl-lib:fref x-%data% (j) ((1 *))
                                    x-%offset%)
                     zero)
                    (= temp zero))))
      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
              (+
               (f2cl-lib:fref y-%data% (iy) ((1 *))
                              y-%offset%)
               (* alpha
                  (cabs1
                   (f2cl-lib:fref x-%data% (jx) ((1 *))
                                  x-%offset%))
                  temp)))
      (setf jx (f2cl-lib:int-add jx incx))
     label100007))
  (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf temp
              (cabs1
               (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                              a-%offset%)))
      (setf symb_zero
              (and symb_zero
                   (or
                    (=
                     (f2cl-lib:fref x-%data% (j) ((1 *))
                                    x-%offset%)
                     zero)
```

```
                              (= temp zero))))
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data% (iy) ((1 *))
                                        y-%offset%)
                         (* alpha
                            (cabs1
                             (f2cl-lib:fref x-%data% (jx) ((1 *))
                                            x-%offset%))
                            temp)))
                (setf jx (f2cl-lib:int-add jx incx))
               label100008))))
          (if (not symb_zero)
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (+
                       (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (f2cl-lib:sign safe1
                                      (f2cl-lib:fref y-%data% (iy)
                                                     ((1 *))
                                                     y-%offset%)))))
          (setf iy (f2cl-lib:int-add iy incy))
         label100006)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (cond
           ((= beta zero) (setf symb_zero f2cl-lib:%true%)
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    0.0))
           ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
            (setf symb_zero f2cl-lib:%true%))
           (t (setf symb_zero f2cl-lib:%false%)
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (* beta
                       (abs
                        (f2cl-lib:fref y-%data% (iy) ((1 *))
                                       y-%offset%))))))
          (setf jx kx)
          (cond
           ((/= alpha zero)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j i) nil)
              (tagbody
                (setf temp
                        (cabs1
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)))
                (setf symb_zero
                        (and symb_zero
```

```
                    (or
                     (=
                      (f2cl-lib:fref x-%data% (j) ((1 *))
                                     x-%offset%)
                      zero)
                     (= temp zero))))
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (+
                   (f2cl-lib:fref y-%data% (iy) ((1 *))
                                  y-%offset%)
                   (* alpha
                      (cabs1
                       (f2cl-lib:fref x-%data% (jx) ((1 *))
                                      x-%offset%))
                      temp)))
          (setf jx (f2cl-lib:int-add jx incx))
         label100010))
        (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                       (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf temp
                    (cabs1
                     (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                    a-%offset%)))
            (setf symb_zero
                    (and symb_zero
                         (or
                          (=
                           (f2cl-lib:fref x-%data% (j) ((1 *))
                                          x-%offset%)
                           zero)
                          (= temp zero))))
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+
                     (f2cl-lib:fref y-%data% (iy) ((1 *))
                                    y-%offset%)
                     (* alpha
                        (cabs1
                         (f2cl-lib:fref x-%data% (jx) ((1 *))
                                        x-%offset%))
                        temp)))
            (setf jx (f2cl-lib:int-add jx incx))
           label100011))))
      (if (not symb_zero)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (+
                   (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                   (f2cl-lib:sign safe1
                                  (f2cl-lib:fref y-%data% (iy)
```

```
                                                        ((1 *))
                                                        y-%offset%)))))
                 (setf iy (f2cl-lib:int-add iy incy))
                label100009))))))
          (go end_label)
         end_label
          (return (values nil nil nil nil nil nil nil nil nil nil)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cla_syamv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (single-float)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
                                              nil nil)
                                            :calls 'nil)))
```

———-

60.2 cla_syrcond_c

60.2.1 Axiom unit tests

— clasyrcondc.input —

```
)set break resume
)sys rm -f clasyrcondc.output
)spool clasyrcondc.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

60.2.2 Axiom help page

— **cla_syrcond_c.help** —

```
=====================================================================
cla_syrcond_c examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       real function cla_syrcond_c (UPLO, N, A, LDA, AF, LDAF, IPIV, C,
           CAPPLY, INFO, WORK, RWORK)
           CLA_SYRCOND_C computes the infinity norm condition number of
           op(A)*inv(diag(c)) for symmetric indefinite matrices.

Function/Subroutine Documentation
   real function cla_syrcond_c (character UPLO, integer N, complex, dimension(
       lda, * ) A, integer LDA, complex, dimension( ldaf, * ) AF, integer
       LDAF, integer, dimension( * ) IPIV, real, dimension( * ) C, logical
       CAPPLY, integer INFO, complex, dimension( * ) WORK, real, dimension( *
       ) RWORK)
       CLA_SYRCOND_C computes the infinity norm condition number of
       op(A)*inv(diag(c)) for symmetric indefinite matrices.

Purpose:
   CLA_SYRCOND_C Computes the infinity norm condition number of
   op(A) * inv(diag(C)) where C is a REAL vector.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.
```

```
A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the N-by-N matrix A

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX array, dimension (LDAF,N)
  The block diagonal matrix D and the multipliers used to
  obtain the factor U or L as computed by CSYTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by CSYTRF.

C
  C is REAL array, dimension (N)
  The vector C in the formula op(A) * inv(diag(C)).

CAPPLY
  CAPPLY is LOGICAL
  If .TRUE. then access the vector C in the formula above.

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  i > 0:  The ith argument is invalid.

WORK
  WORK is COMPLEX array, dimension (2*N).
  Workspace.

RWORK
  RWORK is REAL array, dimension (N).
  Workspace.
```

———-

60.2.3 fortran code

— cla_syrcond_c.f —

```
*  ====================================================================
      REAL FUNCTION CLA_SYRCOND_C( UPLO, N, A, LDA, AF, LDAF, IPIV, C,
     $                             CAPPLY, INFO, WORK, RWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      LOGICAL            CAPPLY
      INTEGER            N, LDA, LDAF, INFO
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), AF( LDAF, * ), WORK( * )
      REAL               C( * ), RWORK( * )
*     ..
*
*  ====================================================================
*
*     .. Local Scalars ..
      INTEGER            KASE
      REAL               AINVNM, ANORM, TMP
      INTEGER            I, J
      LOGICAL            UP, UPPER
      COMPLEX            ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLACN2, CSYTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Statement Functions ..
      REAL CABS1
*     ..
```

```
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      CLA_SYRCOND_C = 0.0E+0
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CLA_SYRCOND_C', -INFO )
         RETURN
      END IF
      UP = .FALSE.
      IF ( LSAME( UPLO, 'U' ) ) UP = .TRUE.
*
*     Compute norm of op(A)*op2(C).
*
      ANORM = 0.0E+0
      IF ( UP ) THEN
         DO I = 1, N
            TMP = 0.0E+0
            IF ( CAPPLY ) THEN
               DO J = 1, I
                  TMP = TMP + CABS1( A( J, I ) ) / C( J )
               END DO
               DO J = I+1, N
                  TMP = TMP + CABS1( A( I, J ) ) / C( J )
               END DO
            ELSE
               DO J = 1, I
                  TMP = TMP + CABS1( A( J, I ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + CABS1( A( I, J ) )
               END DO
            END IF
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      ELSE
```

```
         DO I = 1, N
            TMP = 0.0E+0
            IF ( CAPPLY ) THEN
               DO J = 1, I
                  TMP = TMP + CABS1( A( I, J ) ) / C( J )
               END DO
               DO J = I+1, N
                  TMP = TMP + CABS1( A( J, I ) ) / C( J )
               END DO
            ELSE
               DO J = 1, I
                  TMP = TMP + CABS1( A( I, J ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + CABS1( A( J, I ) )
               END DO
            END IF
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 ) THEN
         CLA_SYRCOND_C = 1.0E+0
         RETURN
      ELSE IF( ANORM .EQ. 0.0E+0 ) THEN
         RETURN
      END IF
*
*     Estimate the norm of inv(op(A)).
*
      AINVNM = 0.0E+0
*
      KASE = 0
   10 CONTINUE
      CALL CLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
*
            IF ( UP ) THEN
               CALL CSYTRS( 'U', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
```

```
            ELSE
               CALL CSYTRS( 'L', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ENDIF
*
*           Multiply by inv(C).
*
            IF ( CAPPLY ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF
         ELSE
*
*           Multiply by inv(C**T).
*
            IF ( CAPPLY ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF
*
            IF ( UP ) THEN
               CALL CSYTRS( 'U', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ELSE
               CALL CSYTRS( 'L', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            END IF
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM .NE. 0.0E+0 )
     $   CLA_SYRCOND_C = 1.0E+0 / AINVNM
*
      RETURN
*
      END
```

60.2.4 lisp code

— cla_syrcond_c.lisp —

```
(defun cla_syrcond_c (uplo n a lda af ldaf ipiv c capply info work rwork)
  (declare (type f2cl-lib:logical capply)
           (type (array single-float (*)) rwork c)
           (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex8 (*)) work af a)
           (type (f2cl-lib:integer4) info ldaf lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (af f2cl-lib:complex8 af-%data% af-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (c single-float c-%data% c-%offset%)
       (rwork single-float rwork-%data% rwork-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
        cabs1))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (zdum #C(0.0 0.0)) (up nil) (upper nil) (i 0) (j 0) (ainvnm 0.0)
             (anorm 0.0) (tmp 0.0) (kase 0) (cla_syrcond_c 0.0))
        (declare (type (single-float) cla_syrcond_c tmp anorm ainvnm)
                 (type (f2cl-lib:integer4) kase j i)
                 (type f2cl-lib:logical upper up)
                 (type (f2cl-lib:complex8) zdum)
                 (type (array f2cl-lib:integer4 (3)) isave))
        (setf cla_syrcond_c 0.0)
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
```

```
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6)))
(cond
 ((/= info 0) (xerbla "CLA_SYRCOND_C" (f2cl-lib:int-sub info))
  (go end_label)))
(setf up f2cl-lib:%false%)
(if (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)
    (setf up f2cl-lib:%true%))
(setf anorm 0.0)
(cond
 (up
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf tmp 0.0)
      (cond
       (capply
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j i) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (/
                        (cabs1
                         (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                        a-%offset%))
                        (f2cl-lib:fref c-%data% (j) ((1 *))
                                       c-%offset%))))
           label100001))
        (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (/
                        (cabs1
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%))
                        (f2cl-lib:fref c-%data% (j) ((1 *))
                                       c-%offset%))))
           label100002)))
       (t
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                        ((> j i) nil)
                (tagbody
                  (setf tmp
                          (+ tmp
                             (cabs1
                              (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                             a-%offset%))))
                 label100003))
              (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                        ((> j n) nil)
                (tagbody
                  (setf tmp
                          (+ tmp
                             (cabs1
                              (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                             a-%offset%))))
                 label100004))))
            (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                    tmp)
            (setf anorm (max anorm tmp))
           label100000)))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf tmp 0.0)
             (cond
              (capply
               (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                             ((> j i) nil)
                 (tagbody
                   (setf tmp
                           (+ tmp
                              (/
                               (cabs1
                                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                               a-%offset%))
                               (f2cl-lib:fref c-%data% (j) ((1 *))
                                             c-%offset%))))
                 label100006))
              (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                            ((> j n) nil)
                 (tagbody
                   (setf tmp
                           (+ tmp
                              (/
                               (cabs1
                                (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                               a-%offset%))
                               (f2cl-lib:fref c-%data% (j) ((1 *))
```

```
                                        c-%offset%))))
            label100007)))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j i) nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (cabs1
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%))))
            label100008))
         (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (cabs1
                         (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                        a-%offset%))))
            label100009))))
       (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
              tmp)
       (setf anorm (max anorm tmp))
      label100005))))
 (cond ((= n 0) (setf cla_syrcond_c 1.0) (go end_label))
       ((= anorm 0.0) (go end_label)))
 (setf ainvnm 0.0)
 (setf kase 0)
label10
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
     (clacn2 n
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ n 1))
                            ((1 *)) work-%offset%)
      work ainvnm kase isave)
   (declare (ignore var-1 var-2 var-5))
   (setf n var-0)
   (setf ainvnm var-3)
   (setf kase var-4))
 (cond
  ((/= kase 0)
   (cond
    ((= kase 2)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (*
                  (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref rwork-%data% (i) ((1 *))
```

```
                                rwork-%offset%)))
    label100010))
  (cond
   (up
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (csytrs "U" n 1 af ldaf ipiv work n info)
      (declare (ignore var-0 var-2 var-3 var-5 var-6))
      (when var-1 (setf n var-1))
      (when var-4 (setf ldaf var-4))
      (when var-7 (setf n var-7))
      (when var-8 (setf info var-8))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (csytrs "L" n 1 af ldaf ipiv work n info)
      (declare (ignore var-0 var-2 var-3 var-5 var-6))
      (when var-1 (setf n var-1))
      (when var-4 (setf ldaf var-4))
      (when var-7 (setf n var-7))
      (when var-8 (setf info var-8)))))
  (cond
   (capply
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (*
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
      label100011)))))
 (t
  (cond
   (capply
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (*
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
      label100012))))
  (cond
   (up
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (csytrs "U" n 1 af ldaf ipiv work n info)
      (declare (ignore var-0 var-2 var-3 var-5 var-6))
```

```
                (when var-1 (setf n var-1))
                (when var-4 (setf ldaf var-4))
                (when var-7 (setf n var-7))
                (when var-8 (setf info var-8))))
              (t
               (multiple-value-bind
                   (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                   (csytrs "L" n 1 af ldaf ipiv work n info)
                (declare (ignore var-0 var-2 var-3 var-5 var-6))
                (when var-1 (setf n var-1))
                (when var-4 (setf ldaf var-4))
                (when var-7 (setf n var-7))
                (when var-8 (setf info var-8)))))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
               (tagbody
                (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                        (*
                         (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                         (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                        rwork-%offset%)))
                label100013))))
          (go label10)))
        (if (/= ainvnm 0.0)
            (setf cla_syrcond_c (/ 1.0 ainvnm)))
        (go end_label)
       end_label
        (return
         (values cla_syrcond_c uplo n nil nil nil ldaf nil nil nil info nil
                 nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cla_syrcond_c
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              fortran-to-lisp::logical
                                              (fortran-to-lisp::integer4)
```

```
  (array fortran-to-lisp::complex8
   (*))
  (array single-float (*)))
:return-values
'(fortran-to-lisp::uplo
  fortran-to-lisp::n nil nil nil
  fortran-to-lisp::ldaf nil nil nil
  fortran-to-lisp::info nil nil)
:calls '(fortran-to-lisp::clacn2))))
```

———-

60.3 cla_syrcond_x

60.3.1 Axiom unit tests

— **clasyrcondx.input** —

```
)set break resume
)sys rm -f clasyrcondx.output
)spool clasyrcondx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.3.2 Axiom help page

— **cla_syrcond_x.help** —

```
====================================================================
cla_syrcond_x examples
====================================================================

====================================================================
Man Page Details
====================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       real function cla_syrcond_x (UPLO, N, A, LDA, AF, LDAF, IPIV, X, INFO,
           WORK, RWORK)
           CLA_SYRCOND_X computes the infinity norm condition number of
           op(A)*diag(x) for symmetric indefinite matrices.

Function/Subroutine Documentation
   real function cla_syrcond_x (character UPLO, integer N, complex, dimension(
       lda, * ) A, integer LDA, complex, dimension( ldaf, * ) AF, integer
       LDAF, integer, dimension( * ) IPIV, complex, dimension( * ) X, integer
       INFO, complex, dimension( * ) WORK, real, dimension( * ) RWORK)
       CLA_SYRCOND_X computes the infinity norm condition number of
       op(A)*diag(x) for symmetric indefinite matrices.

Purpose:
   CLA_SYRCOND_X Computes the infinity norm condition number of
   op(A) * diag(X) where X is a COMPLEX vector.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is COMPLEX array, dimension (LDAF,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by CSYTRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
```

```
  as determined by CSYTRF.

X
  X is COMPLEX array, dimension (N)
  The vector X in the formula op(A) * diag(X).

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  i > 0:  The ith argument is invalid.

WORK
  WORK is COMPLEX array, dimension (2*N).
  Workspace.

RWORK
  RWORK is REAL array, dimension (N).
  Workspace.
```

60.3.3 fortran code

— cla_syrcond_x.f —

```
*  =====================================================================
      REAL FUNCTION CLA_SYRCOND_X( UPLO, N, A, LDA, AF, LDAF, IPIV, X,
     $                             INFO, WORK, RWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            N, LDA, LDAF, INFO
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), AF( LDAF, * ), WORK( * ), X( * )
      REAL               RWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
```

```
      INTEGER            KASE
      REAL               AINVNM, ANORM, TMP
      INTEGER            I, J
      LOGICAL            UP, UPPER
      COMPLEX            ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLACN2, CSYTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      CLA_SYRCOND_X = 0.0E+0
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF ( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CLA_SYRCOND_X', -INFO )
         RETURN
      END IF
      UP = .FALSE.
      IF ( LSAME( UPLO, 'U' ) ) UP = .TRUE.
*
*     Compute norm of op(A)*op2(C).
*
      ANORM = 0.0
```

```
      IF ( UP ) THEN
         DO I = 1, N
            TMP = 0.0E+0
            DO J = 1, I
               TMP = TMP + CABS1( A( J, I ) * X( J ) )
            END DO
            DO J = I+1, N
               TMP = TMP + CABS1( A( I, J ) * X( J ) )
            END DO
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      ELSE
         DO I = 1, N
            TMP = 0.0E+0
            DO J = 1, I
               TMP = TMP + CABS1( A( I, J ) * X( J ) )
            END DO
            DO J = I+1, N
               TMP = TMP + CABS1( A( J, I ) * X( J ) )
            END DO
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 ) THEN
         CLA_SYRCOND_X = 1.0E+0
         RETURN
      ELSE IF( ANORM .EQ. 0.0E+0 ) THEN
         RETURN
      END IF
*
*     Estimate the norm of inv(op(A)).
*
      AINVNM = 0.0E+0
*
      KASE = 0
   10 CONTINUE
      CALL CLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
```

```
*
            IF ( UP ) THEN
               CALL CSYTRS( 'U', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ELSE
               CALL CSYTRS( 'L', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ENDIF
*
*           Multiply by inv(X).
*
            DO I = 1, N
               WORK( I ) = WORK( I ) / X( I )
            END DO
         ELSE
*
*           Multiply by inv(X**T).
*
            DO I = 1, N
               WORK( I ) = WORK( I ) / X( I )
            END DO
*
            IF ( UP ) THEN
               CALL CSYTRS( 'U', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ELSE
               CALL CSYTRS( 'L', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            END IF
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM .NE. 0.0E+0 )
     $   CLA_SYRCOND_X = 1.0E+0 / AINVNM
*
      RETURN
*
      END
```

———

60.3.4 lisp code

— cla_syrcond_x.lisp —

```
(defun cla_syrcond_x (uplo n a lda af ldaf ipiv x info work rwork)
  (declare (type (array single-float (*)) rwork)
           (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex8 (*)) work x af a)
           (type (f2cl-lib:integer4) info ldaf lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (af f2cl-lib:complex8 af-%data% af-%offset%)
       (x f2cl-lib:complex8 x-%data% x-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (rwork single-float rwork-%data% rwork-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
        cabs1))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (zdum #C(0.0 0.0)) (up nil) (upper nil) (i 0) (j 0) (ainvnm 0.0)
             (anorm 0.0) (tmp 0.0) (kase 0) (cla_syrcond_x 0.0))
        (declare (type (single-float) cla_syrcond_x tmp anorm ainvnm)
                 (type (f2cl-lib:integer4) kase j i)
                 (type f2cl-lib:logical upper up)
                 (type (f2cl-lib:complex8) zdum)
                 (type (array f2cl-lib:integer4 (3)) isave))
        (setf cla_syrcond_x 0.0)
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2))
```

```
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6)))
(cond
 ((/= info 0) (xerbla "CLA_SYRCOND_X" (f2cl-lib:int-sub info))
  (go end_label)))
(setf up f2cl-lib:%false%)
(if (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)
    (setf up f2cl-lib:%true%))
(setf anorm 0.0)
(cond
 (up
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf tmp 0.0)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j i) nil)
        (tagbody
          (setf tmp
                  (+ tmp
                     (cabs1
                      (*
                       (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                      a-%offset%)
                       (f2cl-lib:fref x-%data% (j) ((1 *))
                                      x-%offset%)))))
         label100001))
      (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf tmp
                  (+ tmp
                     (cabs1
                      (*
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                       (f2cl-lib:fref x-%data% (j) ((1 *))
                                      x-%offset%)))))
         label100002))
      (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
              tmp)
      (setf anorm (max anorm tmp))
     label100000)))
 (t
```

```
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf tmp 0.0)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j i) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (cabs1
                        (*
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%)))))
           label100004))
        (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (cabs1
                        (*
                         (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                        a-%offset%)
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%)))))
           label100005))
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                tmp)
        (setf anorm (max anorm tmp))
       label100003))))
  (cond ((= n 0) (setf cla_syrcond_x 1.0) (go end_label))
        ((= anorm 0.0) (go end_label)))
  (setf ainvnm 0.0)
  (setf kase 0)
 label10
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (clacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ n 1))
                             ((1 *)) work-%offset%)
       work ainvnm kase isave)
    (declare (ignore var-1 var-2 var-5))
    (setf n var-0)
    (setf ainvnm var-3)
    (setf kase var-4))
  (cond
   ((/= kase 0)
    (cond
     ((= kase 2)
```

```
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (*
             (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
             (f2cl-lib:fref rwork-%data% (i) ((1 *))
                            rwork-%offset%)))
   label100006))
(cond
 (up
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (csytrs "U" n 1 af ldaf ipiv work n info)
    (declare (ignore var-0 var-2 var-3 var-5 var-6))
    (when var-1 (setf n var-1))
    (when var-4 (setf ldaf var-4))
    (when var-7 (setf n var-7))
    (when var-8 (setf info var-8))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (csytrs "L" n 1 af ldaf ipiv work n info)
    (declare (ignore var-0 var-2 var-3 var-5 var-6))
    (when var-1 (setf n var-1))
    (when var-4 (setf ldaf var-4))
    (when var-7 (setf n var-7))
    (when var-8 (setf info var-8)))))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (/
             (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
             (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)))
   label100007)))
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
             (/
              (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)))
    label100008))
 (cond
  (up
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (csytrs "U" n 1 af ldaf ipiv work n info)
```

```
                (declare (ignore var-0 var-2 var-3 var-5 var-6))
                (when var-1 (setf n var-1))
                (when var-4 (setf ldaf var-4))
                (when var-7 (setf n var-7))
                (when var-8 (setf info var-8))))
              (t
               (multiple-value-bind
                   (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                   (csytrs "L" n 1 af ldaf ipiv work n info)
                (declare (ignore var-0 var-2 var-3 var-5 var-6))
                (when var-1 (setf n var-1))
                (when var-4 (setf ldaf var-4))
                (when var-7 (setf n var-7))
                (when var-8 (setf info var-8)))))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
               (tagbody
                (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                        (*
                         (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                         (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                        rwork-%offset%)))
                label100009))))
          (go label10)))
        (if (/= ainvnm 0.0)
            (setf cla_syrcond_x (/ 1.0 ainvnm)))
        (go end_label)
       end_label
        (return
         (values cla_syrcond_x uplo n nil nil nil ldaf nil nil info nil
                 nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cla_syrcond_x
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
```

```
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (array single-float (*)))
:return-values
'(fortran-to-lisp::uplo
  fortran-to-lisp::n nil nil nil
  fortran-to-lisp::ldaf nil nil
  fortran-to-lisp::info nil nil)
:calls '(fortran-to-lisp::clacn2))))
```

———-

60.4 cla_syrfsx_extended

60.4.1 Axiom unit tests

— **clasyrfsxextended.input** —

```
)set break resume
)sys rm -f clasyrfsxextended.output
)spool clasyrfsxextended.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.4.2 Axiom help page [Demm05]

— **cla_syrfsx_extended.help** —

```
==========================================================================
cla_syrfsx_extended examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine cla_syrfsx_extended (PREC_TYPE, UPLO, N, NRHS, A, LDA, AF,
           LDAF, IPIV, COLEQU, C, B, LDB, Y, LDY, BERR_OUT, N_NORMS,
           ERR_BNDS_NORM, ERR_BNDS_COMP, RES, AYB, DY, Y_TAIL, RCOND, ITHRESH,
           RTHRESH, DZ_UB, IGNORE_CWISE, INFO)
           CLA_SYRFSX_EXTENDED improves the computed solution to a system of
           linear equations for symmetric indefinite matrices by performing
           extra-precise iterative refinement and provides error bounds and
           backward error estimates for the solution.

Function/Subroutine Documentation
   subroutine cla_syrfsx_extended (integer PREC_TYPE, character UPLO, integer
       N, integer NRHS, complex, dimension( lda, * ) A, integer LDA, complex,
       dimension( ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV,
       logical COLEQU, real, dimension( * ) C, complex, dimension( ldb, * ) B,
       integer LDB, complex, dimension( ldy, * ) Y, integer LDY, real,
       dimension( * ) BERR_OUT, integer N_NORMS, real, dimension( nrhs, * )
       ERR_BNDS_NORM, real, dimension( nrhs, * ) ERR_BNDS_COMP, complex,
       dimension( * ) RES, real, dimension( * ) AYB, complex, dimension( * )
       DY, complex, dimension( * ) Y_TAIL, real RCOND, integer ITHRESH, real
       RTHRESH, real DZ_UB, logical IGNORE_CWISE, integer INFO)
       CLA_SYRFSX_EXTENDED improves the computed solution to a system of
       linear equations for symmetric indefinite matrices by performing extra-
       precise iterative refinement and provides error bounds and backward
       error estimates for the solution.

Purpose:
   CLA_SYRFSX_EXTENDED improves the computed solution to a system of
   linear equations by performing extra-precise iterative refinement
   and provides error bounds and backward error estimates for the solution.
   This subroutine is called by CSYRFSX to perform iterative refinement.
   In addition to normwise error bound, the code provides maximum
   componentwise error bound if possible. See comments for ERR_BNDS_NORM
   and ERR_BNDS_COMP for details of the error bounds. Note that this
   subroutine is only resonsible for setting the second fields of
   ERR_BNDS_NORM and ERR_BNDS_COMP.

Parameters:

 PREC_TYPE
   PREC_TYPE is INTEGER
   Specifies the intermediate precision to be used in refinement.
   The value is defined by ILAPREC(P) where P is a CHARACTER and
   P    = 'S':  Single
        = 'D':  Double
        = 'I':  Indigenous
        = 'X', 'E':  Extra
```

```
UPLO
  UPLO is CHARACTER*1
  = 'U':  Upper triangle of A is stored;
  = 'L':  Lower triangle of A is stored.

N
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right-hand-sides, i.e., the number of columns of the
  matrix B.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the N-by-N matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX array, dimension (LDAF,N)
  The block diagonal matrix D and the multipliers used to
  obtain the factor U or L as computed by CSYTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by CSYTRF.

COLEQU
  COLEQU is LOGICAL
  If .TRUE. then column equilibration was done to A before calling
  this routine. This is needed to compute the solution and error
  bounds correctly.

C
  C is REAL array, dimension (N)
  The column scale factors for A. If COLEQU = .FALSE., C
  is not accessed. If C is input, each element of C should be a power
  of the radix to ensure a reliable solution and error estimates.
  Scaling by powers of the radix does not cause rounding errors unless
  the result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
```

```
  input matrix, producing error estimates that may not be reliable.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  The right-hand-side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

Y
  Y is COMPLEX array, dimension (LDY,NRHS)
  On entry, the solution matrix X, as computed by CSYTRS.
  On exit, the improved solution matrix Y.

LDY
  LDY is INTEGER
  The leading dimension of the array Y.  LDY >= max(1,N).

BERR_OUT
  BERR_OUT is REAL array, dimension (NRHS)
  On exit, BERR_OUT(j) contains the componentwise relative backward
  error for right-hand-side j from the formula
     max(i) ( abs(RES(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
  where abs(Z) is the componentwise absolute value of the matrix
  or vector Z. This is computed by CLA_LIN_BERR.

N_NORMS
  N_NORMS is INTEGER
  Determines which error bounds to return (see ERR_BNDS_NORM
  and ERR_BNDS_COMP).
  If N_NORMS >= 1 return normwise error bounds.
  If N_NORMS >= 2 return componentwise error bounds.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
          max_j (abs(XTRUE(j,i) - X(j,i)))
           ------------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
```

```
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

ERR_BNDS_COMP
  ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                  abs(XTRUE(j,i) - X(j,i))
            max_j ----------------------
                        abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:
```

```
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
    reciprocal condition number is less than the threshold
    sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
    almost certainly within a factor of 10 of the true error
    so long as the next entry is greater than the threshold
    sqrt(n) * slamch('Epsilon'). This error bound should only
    be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
    reciprocal condition number.  Compared with the threshold
    sqrt(n) * slamch('Epsilon') to determine if the error
    estimate is "guaranteed". These reciprocal condition
    numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
    appropriately scaled matrix Z.
    Let Z = S*(A*diag(x)), where x is the solution for the
    current right-hand side and S scales each row of
    A*diag(x) by a power of the radix so all absolute row
    sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

RES
  RES is COMPLEX array, dimension (N)
  Workspace to hold the intermediate residual.

AYB
  AYB is REAL array, dimension (N)
  Workspace.

DY
  DY is COMPLEX array, dimension (N)
  Workspace to hold the intermediate solution.

Y_TAIL
  Y_TAIL is COMPLEX array, dimension (N)
  Workspace to hold the trailing bits of the intermediate solution.

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

ITHRESH
```

```
  ITHRESH is INTEGER
  The maximum number of residual computations allowed for
  refinement. The default is 10. For 'aggressive' set to 100 to
  permit convergence using approximate factorizations or
  factorizations other than LU. If the factorization uses a
  technique other than Gaussian elimination, the guarantees in
  ERR_BNDS_NORM and ERR_BNDS_COMP may no longer be trustworthy.

RTHRESH
  RTHRESH is REAL
  Determines when to stop refinement if the error estimate stops
  decreasing. Refinement will stop when the next solution no longer
  satisfies norm(dx_{i+1}) < RTHRESH * norm(dx_i) where norm(Z) is
  the infinity norm of Z. RTHRESH satisfies 0 < RTHRESH <= 1. The
  default value is 0.5. For 'aggressive' set to 0.9 to permit
  convergence on extremely ill-conditioned matrices. See LAWN 165
  for more details.

DZ_UB
  DZ_UB is REAL
  Determines when to start considering componentwise convergence.
  Componentwise convergence is only considered after each component
  of the solution Y is stable, which we definte as the relative
  change in each component being less than DZ_UB. The default value
  is 0.25, requiring the first bit to be stable. See LAWN 165 for
  more details.

IGNORE_CWISE
  IGNORE_CWISE is LOGICAL
  If .TRUE. then ignore componentwise convergence. Default value
  is .FALSE..

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  < 0:  if INFO = -i, the ith argument to CLA_SYRFSX_EXTENDED had an illegal
  value
```

———-

60.4.3 fortran code

— cla_syrfsx_extended.f —

```
*  ====================================================================
      SUBROUTINE CLA_SYRFSX_EXTENDED( PREC_TYPE, UPLO, N, NRHS, A, LDA,
     $                                AF, LDAF, IPIV, COLEQU, C, B, LDB,
```

```
     $                                Y, LDY, BERR_OUT, N_NORMS,
     $                                ERR_BNDS_NORM, ERR_BNDS_COMP, RES,
     $                                AYB, DY, Y_TAIL, RCOND, ITHRESH,
     $                                RTHRESH, DZ_UB, IGNORE_CWISE,
     $                                INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDAF, LDB, LDY, N, NRHS, PREC_TYPE,
     $                   N_NORMS, ITHRESH
      CHARACTER          UPLO
      LOGICAL            COLEQU, IGNORE_CWISE
      REAL               RTHRESH, DZ_UB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   Y( LDY, * ), RES( * ), DY( * ), Y_TAIL( * )
      REAL               C( * ), AYB( * ), RCOND, BERR_OUT( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            UPLO2, CNT, I, J, X_STATE, Z_STATE,
     $                   Y_PREC_STATE
      REAL               YK, DYK, YMIN, NORMY, NORMX, NORMDX, DXRAT,
     $                   DZRAT, PREVNORMDX, PREV_DZ_Z, DXRATMAX,
     $                   DZRATMAX, DX_X, DZ_Z, FINAL_DX_X, FINAL_DZ_Z,
     $                   EPS, HUGEVAL, INCR_THRESH
      LOGICAL            INCR_PREC, UPPER
      COMPLEX            ZDUM
*     ..
*     .. Parameters ..
      INTEGER            UNSTABLE_STATE, WORKING_STATE, CONV_STATE,
     $                   NOPROG_STATE, BASE_RESIDUAL, EXTRA_RESIDUAL,
     $                   EXTRA_Y
      PARAMETER          ( UNSTABLE_STATE = 0, WORKING_STATE = 1,
     $                   CONV_STATE = 2, NOPROG_STATE = 3 )
      PARAMETER          ( BASE_RESIDUAL = 0, EXTRA_RESIDUAL = 1,
     $                   EXTRA_Y = 2 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
```

```
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           ILAUPLO
      INTEGER            ILAUPLO
*     ..
*     .. External Subroutines ..
      EXTERNAL           CAXPY, CCOPY, CSYTRS, CSYMV, BLAS_CSYMV_X,
     $                   BLAS_CSYMV2_X, CLA_SYAMV, CLA_WWADDW,
     $                   CLA_LIN_BERR
      REAL               SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, REAL, AIMAG, MAX, MIN
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
```

```
         INFO = -13
      ELSE IF( LDY.LT.MAX( 1, N ) ) THEN
         INFO = -15
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CLA_SYRFSX_EXTENDED', -INFO )
         RETURN
      END IF
      EPS = SLAMCH( 'Epsilon' )
      HUGEVAL = SLAMCH( 'Overflow' )
*     Force HUGEVAL to Inf
      HUGEVAL = HUGEVAL * HUGEVAL
*     Using HUGEVAL may lead to spurious underflows.
      INCR_THRESH = REAL( N ) * EPS

      IF ( LSAME ( UPLO, 'L' ) ) THEN
         UPLO2 = ILAUPLO( 'L' )
      ELSE
         UPLO2 = ILAUPLO( 'U' )
      ENDIF

      DO J = 1, NRHS
         Y_PREC_STATE = EXTRA_RESIDUAL
         IF ( Y_PREC_STATE .EQ. EXTRA_Y ) THEN
            DO I = 1, N
               Y_TAIL( I ) = 0.0
            END DO
         END IF

         DXRAT = 0.0
         DXRATMAX = 0.0
         DZRAT = 0.0
         DZRATMAX = 0.0
         FINAL_DX_X = HUGEVAL
         FINAL_DZ_Z = HUGEVAL
         PREVNORMDX = HUGEVAL
         PREV_DZ_Z = HUGEVAL
         DZ_Z = HUGEVAL
         DX_X = HUGEVAL

         X_STATE = WORKING_STATE
         Z_STATE = UNSTABLE_STATE
         INCR_PREC = .FALSE.

         DO CNT = 1, ITHRESH
*
*         Compute residual RES = B_s - op(A_s) * Y,
*             op(A) = A, A**T, or A**H depending on TRANS (and type).
*
            CALL CCOPY( N, B( 1, J ), 1, RES, 1 )
```

```
            IF ( Y_PREC_STATE .EQ. BASE_RESIDUAL ) THEN
               CALL CSYMV( UPLO, N, CMPLX(-1.0), A, LDA, Y(1,J), 1,
     $              CMPLX(1.0), RES, 1 )
            ELSE IF ( Y_PREC_STATE .EQ. EXTRA_RESIDUAL ) THEN
               CALL BLAS_CSYMV_X( UPLO2, N, CMPLX(-1.0), A, LDA,
     $              Y( 1, J ), 1, CMPLX(1.0), RES, 1, PREC_TYPE )
            ELSE
               CALL BLAS_CSYMV2_X(UPLO2, N, CMPLX(-1.0), A, LDA,
     $              Y(1, J), Y_TAIL, 1, CMPLX(1.0), RES, 1, PREC_TYPE)
            END IF

!         XXX: RES is no longer needed.
            CALL CCOPY( N, RES, 1, DY, 1 )
            CALL CSYTRS( UPLO, N, 1, AF, LDAF, IPIV, DY, N, INFO )
*
*         Calculate relative changes DX_X, DZ_Z and ratios DXRAT, DZRAT.
*
            NORMX = 0.0
            NORMY = 0.0
            NORMDX = 0.0
            DZ_Z = 0.0
            YMIN = HUGEVAL

            DO I = 1, N
               YK = CABS1( Y( I, J ) )
               DYK = CABS1( DY( I ) )

               IF ( YK .NE. 0.0 ) THEN
                  DZ_Z = MAX( DZ_Z, DYK / YK )
               ELSE IF ( DYK .NE. 0.0 ) THEN
                  DZ_Z = HUGEVAL
               END IF

               YMIN = MIN( YMIN, YK )

               NORMY = MAX( NORMY, YK )

               IF ( COLEQU ) THEN
                  NORMX = MAX( NORMX, YK * C( I ) )
                  NORMDX = MAX( NORMDX, DYK * C( I ) )
               ELSE
                  NORMX = NORMY
                  NORMDX = MAX( NORMDX, DYK )
               END IF
            END DO

            IF ( NORMX .NE. 0.0 ) THEN
               DX_X = NORMDX / NORMX
            ELSE IF ( NORMDX .EQ. 0.0 ) THEN
               DX_X = 0.0
```

```
            ELSE
               DX_X = HUGEVAL
            END IF

            DXRAT = NORMDX / PREVNORMDX
            DZRAT = DZ_Z / PREV_DZ_Z
*
*         Check termination criteria.
*
            IF ( YMIN*RCOND .LT. INCR_THRESH*NORMY
     $           .AND. Y_PREC_STATE .LT. EXTRA_Y )
     $           INCR_PREC = .TRUE.

            IF ( X_STATE .EQ. NOPROG_STATE .AND. DXRAT .LE. RTHRESH )
     $           X_STATE = WORKING_STATE
            IF ( X_STATE .EQ. WORKING_STATE ) THEN
               IF ( DX_X .LE. EPS ) THEN
                  X_STATE = CONV_STATE
               ELSE IF ( DXRAT .GT. RTHRESH ) THEN
                  IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                     INCR_PREC = .TRUE.
                  ELSE
                     X_STATE = NOPROG_STATE
                  END IF
               ELSE
                  IF (DXRAT .GT. DXRATMAX) DXRATMAX = DXRAT
               END IF
               IF ( X_STATE .GT. WORKING_STATE ) FINAL_DX_X = DX_X
            END IF

            IF ( Z_STATE .EQ. UNSTABLE_STATE .AND. DZ_Z .LE. DZ_UB )
     $           Z_STATE = WORKING_STATE
            IF ( Z_STATE .EQ. NOPROG_STATE .AND. DZRAT .LE. RTHRESH )
     $           Z_STATE = WORKING_STATE
            IF ( Z_STATE .EQ. WORKING_STATE ) THEN
               IF ( DZ_Z .LE. EPS ) THEN
                  Z_STATE = CONV_STATE
               ELSE IF ( DZ_Z .GT. DZ_UB ) THEN
                  Z_STATE = UNSTABLE_STATE
                  DZRATMAX = 0.0
                  FINAL_DZ_Z = HUGEVAL
               ELSE IF ( DZRAT .GT. RTHRESH ) THEN
                  IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                     INCR_PREC = .TRUE.
                  ELSE
                     Z_STATE = NOPROG_STATE
                  END IF
               ELSE
                  IF ( DZRAT .GT. DZRATMAX ) DZRATMAX = DZRAT
               END IF
```

```
               IF ( Z_STATE .GT. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
            END IF

            IF ( X_STATE.NE.WORKING_STATE.AND.
     $           ( IGNORE_CWISE.OR.Z_STATE.NE.WORKING_STATE ) )
     $           GOTO 666

            IF ( INCR_PREC ) THEN
               INCR_PREC = .FALSE.
               Y_PREC_STATE = Y_PREC_STATE + 1
               DO I = 1, N
                  Y_TAIL( I ) = 0.0
               END DO
            END IF

            PREVNORMDX = NORMDX
            PREV_DZ_Z = DZ_Z
*
*           Update soluton.
*
            IF ( Y_PREC_STATE .LT. EXTRA_Y ) THEN
               CALL CAXPY( N, CMPLX(1.0), DY, 1, Y(1,J), 1 )
            ELSE
               CALL CLA_WWADDW( N, Y(1,J), Y_TAIL, DY )
            END IF

         END DO
*        Target of "IF (Z_STOP .AND. X_STOP)".  Sun's f77 won't EXIT.
 666     CONTINUE
*
*     Set final_* when cnt hits ithresh.
*
         IF ( X_STATE .EQ. WORKING_STATE ) FINAL_DX_X = DX_X
         IF ( Z_STATE .EQ. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
*
*     Compute error bounds.
*
         IF ( N_NORMS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) =
     $           FINAL_DX_X / (1 - DXRATMAX)
         END IF
         IF ( N_NORMS .GE. 2 ) THEN
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) =
     $           FINAL_DZ_Z / (1 - DZRATMAX)
         END IF
*
*     Compute componentwise relative backward error from formula
*         max(i) ( abs(R(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
*     where abs(Z) is the componentwise absolute value of the matrix
*     or vector Z.
```

```
*
*        Compute residual RES = B_s - op(A_s) * Y,
*            op(A) = A, A**T, or A**H depending on TRANS (and type).
*
         CALL CCOPY( N, B( 1, J ), 1, RES, 1 )
         CALL CSYMV( UPLO, N, CMPLX(-1.0), A, LDA, Y(1,J), 1,
     $        CMPLX(1.0), RES, 1 )

         DO I = 1, N
            AYB( I ) = CABS1( B( I, J ) )
         END DO
*
*     Compute abs(op(A_s))*abs(Y) + abs(B_s).
*
         CALL CLA_SYAMV ( UPLO2, N, 1.0,
     $        A, LDA, Y(1, J), 1, 1.0, AYB, 1 )

         CALL CLA_LIN_BERR ( N, N, 1, RES, AYB, BERR_OUT( J ) )
*
*     End of loop for each RHS.
*
      END DO
*
      RETURN
      END
```

———-

60.4.4 lisp code

— cla_syrfsx_extended.lisp —

```
(let* ((unstable_state 0)
       (working_state 1)
       (conv_state 2)
       (noprog_state 3)
       (base_residual 0)
       (extra_residual 1)
       (extra_y 2)
       (la_linrx_err_i 2))
  (declare (type (f2cl-lib:integer4 0 0) unstable_state)
           (type (f2cl-lib:integer4 1 1) working_state)
           (type (f2cl-lib:integer4 2 2) conv_state)
           (type (f2cl-lib:integer4 3 3) noprog_state)
           (type (f2cl-lib:integer4 0 0) base_residual)
           (type (f2cl-lib:integer4 1 1) extra_residual)
           (type (f2cl-lib:integer4 2 2) extra_y)
           (type (f2cl-lib:integer4 2 2) la_linrx_err_i)
```

```
          (ignorable unstable_state working_state conv_state noprog_state
           base_residual extra_residual extra_y la_linrx_err_i))
(defun cla_syrfsx_extended
       (prec_type uplo n nrhs a lda af ldaf ipiv colequ c b ldb$ y ldy
        berr_out n_norms err_bnds_norm err_bnds_comp res ayb dy y_tail rcond
        ithresh rthresh dz_ub ignore_cwise info)
  (declare (type (single-float) dz_ub rthresh rcond)
           (type (array single-float (*)) ayb err_bnds_comp err_bnds_norm
            berr_out c)
           (type f2cl-lib:logical ignore_cwise colequ)
           (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex8 (*)) y_tail dy res y b af a)
           (type (string *) uplo)
           (type (f2cl-lib:integer4) info ithresh n_norms ldy ldb$ ldaf lda
            nrhs n prec_type))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (af f2cl-lib:complex8 af-%data% af-%offset%)
       (b f2cl-lib:complex8 b-%data% b-%offset%)
       (y f2cl-lib:complex8 y-%data% y-%offset%)
       (res f2cl-lib:complex8 res-%data% res-%offset%)
       (dy f2cl-lib:complex8 dy-%data% dy-%offset%)
       (y_tail f2cl-lib:complex8 y_tail-%data% y_tail-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (c single-float c-%data% c-%offset%)
       (berr_out single-float berr_out-%data% berr_out-%offset%)
       (err_bnds_norm single-float err_bnds_norm-%data%
        err_bnds_norm-%offset%)
       (err_bnds_comp single-float err_bnds_comp-%data%
        err_bnds_comp-%offset%)
       (ayb single-float ayb-%data% ayb-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
        cabs1))
      (prog ((zdum #C(0.0 0.0)) (incr_prec nil) (upper nil) (yk 0.0)
             (dyk 0.0) (ymin 0.0) (normy 0.0) (normx 0.0) (normdx 0.0)
             (dxrat 0.0) (dzrat 0.0) (prevnormdx 0.0) (prev_dz_z 0.0)
             (dxratmax 0.0) (dzratmax 0.0) (dx_x 0.0) (dz_z 0.0)
             (final_dx_x 0.0) (final_dz_z 0.0) (eps 0.0) (hugeval 0.0)
             (incr_thresh 0.0) (uplo2 0) (cnt 0) (i 0) (j 0) (x_state 0)
             (z_state 0) (y_prec_state 0))
       (declare (type (f2cl-lib:complex8) zdum)
                (type f2cl-lib:logical incr_prec upper)
                (type (single-float) yk dyk ymin normy normx normdx dxrat
                 dzrat prevnormdx prev_dz_z dxratmax dzratmax dx_x dz_z
                 final_dx_x final_dz_z eps hugeval incr_thresh)
                (type (f2cl-lib:integer4) uplo2 cnt i j x_state z_state
```

```
          y_prec_state))
(setf info 0)
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -13))
 ((< ldy (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -15)))
(cond
 ((/= info 0) (xerbla "CLA_SYRFSX_EXTENDED" (f2cl-lib:int-sub info))
  (go end_label)))
(setf eps (slamch "Epsilon"))
(setf hugeval (slamch "Overflow"))
(setf hugeval (* hugeval hugeval))
(setf incr_thresh (* (f2cl-lib:freal n) eps))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame uplo "L")
    (declare (ignore var-1))
    (when var-0 (setf uplo var-0))
    ret-val)
  (setf uplo2 (ilauplo "L")))
 (t (setf uplo2 (ilauplo "U"))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf y_prec_state extra_residual)
    (cond
     ((= y_prec_state extra_y)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
```

```
      (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                           y_tail-%offset%)
              (coerce 0.0 'f2cl-lib:complex8))
     label100001))))
(setf dxrat 0.0)
(setf dxratmax 0.0)
(setf dzrat 0.0)
(setf dzratmax 0.0)
(setf final_dx_x hugeval)
(setf final_dz_z hugeval)
(setf prevnormdx hugeval)
(setf prev_dz_z hugeval)
(setf dz_z hugeval)
(setf dx_x hugeval)
(setf x_state working_state)
(setf z_state unstable_state)
(setf incr_prec f2cl-lib:%false%)
(f2cl-lib:fdo (cnt 1 (f2cl-lib:int-add cnt 1))
              ((> cnt ithresh) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (ccopy n
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1 res 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (cond
     ((= y_prec_state base_residual)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9)
          (csymv uplo n (f2cl-lib:cmplx -1.0) a lda
           (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                                 ((1 ldy) (1 *)) y-%offset%)
           1 (f2cl-lib:cmplx 1.0) res 1)
        (declare
         (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
        (when var-0 (setf uplo var-0))
        (when var-1 (setf n var-1))
        (when var-4 (setf lda var-4))))
     ((= y_prec_state extra_residual)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (blas_csymv_x uplo2 n (f2cl-lib:cmplx -1.0) a lda
           (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                                 ((1 ldy) (1 *)) y-%offset%)
           1 (f2cl-lib:cmplx 1.0) res 1 prec_type)
        (declare
```

```
     (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
    (when var-0 (setf uplo2 var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf lda var-4))
    (when var-10 (setf prec_type var-10))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11)
      (blas_csymv2_x uplo2 n (f2cl-lib:cmplx -1.0) a lda
       (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                             ((1 ldy) (1 *)) y-%offset%)
       y_tail 1 (f2cl-lib:cmplx 1.0) res 1 prec_type)
    (declare
     (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9
      var-10))
    (when var-0 (setf uplo2 var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf lda var-4))
    (when var-11 (setf prec_type var-11)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (ccopy n res 1 dy 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (csytrs uplo n 1 af ldaf ipiv dy n info)
  (declare (ignore var-2 var-3 var-5 var-6))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-4 (setf ldaf var-4))
  (when var-7 (setf n var-7))
  (when var-8 (setf info var-8)))
(setf normx 0.0)
(setf normy 0.0)
(setf normdx 0.0)
(setf dz_z 0.0)
(setf ymin hugeval)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf yk
            (cabs1
             (f2cl-lib:fref y-%data% (i j) ((1 ldy) (1 *))
                            y-%offset%)))
    (setf dyk
            (cabs1
             (f2cl-lib:fref dy-%data% (i) ((1 *))
                            dy-%offset%)))
    (cond ((/= yk 0.0) (setf dz_z (max dz_z (/ dyk yk))))
```

```
          ((/= dyk 0.0) (setf dz_z hugeval)))
    (setf ymin (min ymin yk))
    (setf normy (max normy yk))
    (cond
     (colequ
      (setf normx
              (max normx
                   (* yk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%))))
      (setf normdx
              (max normdx
                   (* dyk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%)))))
     (t (setf normx normy) (setf normdx (max normdx dyk))))
   label100003))
(cond ((/= normx 0.0) (setf dx_x (/ normdx normx)))
      ((= normdx 0.0) (setf dx_x 0.0))
      (t (setf dx_x hugeval)))
(setf dxrat (/ normdx prevnormdx))
(setf dzrat (/ dz_z prev_dz_z))
(if (and (< (* ymin rcond) (* incr_thresh normy))
         (< y_prec_state extra_y))
    (setf incr_prec f2cl-lib:%true%))
(if (and (= x_state noprog_state) (<= dxrat rthresh))
    (setf x_state working_state))
(cond
 ((= x_state working_state)
  (cond ((<= dx_x eps) (setf x_state conv_state))
        ((> dxrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf x_state noprog_state))))
        (t
         (if (> dxrat dxratmax)
             (setf dxratmax dxrat))))
  (if (> x_state working_state)
      (setf final_dx_x dx_x))))
(if (and (= z_state unstable_state) (<= dz_z dz_ub))
    (setf z_state working_state))
(if (and (= z_state noprog_state) (<= dzrat rthresh))
    (setf z_state working_state))
(cond
 ((= z_state working_state)
  (cond ((<= dz_z eps) (setf z_state conv_state))
        ((> dz_z dz_ub) (setf z_state unstable_state)
         (setf dzratmax 0.0) (setf final_dz_z hugeval))
        ((> dzrat rthresh)
```

```
                (cond
                 ((/= y_prec_state extra_y)
                  (setf incr_prec f2cl-lib:%true%))
                 (t (setf z_state noprog_state))))
               (t
                (if (> dzrat dzratmax)
                    (setf dzratmax dzrat))))
          (if (> z_state working_state)
              (setf final_dz_z dz_z))))
        (if (and (/= x_state working_state)
                 (or ignore_cwise (/= z_state working_state)))
            (go label666))
        (cond
         (incr_prec (setf incr_prec f2cl-lib:%false%)
          (setf y_prec_state (f2cl-lib:int-add y_prec_state 1))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                                   y_tail-%offset%)
                    (coerce 0.0 'f2cl-lib:complex8))
             label100004))))
        (setf prevnormdx normdx)
        (setf prev_dz_z dz_z)
        (cond
         ((< y_prec_state extra_y)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
              (caxpy n (f2cl-lib:cmplx 1.0) dy 1
               (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                                     ((1 ldy) (1 *)) y-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3 var-4 var-5))
            (when var-0 (setf n var-0))))
         (t
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (cla_wwaddw n
               (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                                     ((1 ldy) (1 *)) y-%offset%)
               y_tail dy)
            (declare (ignore var-1 var-2 var-3))
            (when var-0 (setf n var-0)))))
      label100002))
  label666
   (if (= x_state working_state)
       (setf final_dx_x dx_x))
   (if (= z_state working_state)
       (setf final_dz_z dz_z))
   (cond
    ((>= n_norms 1)
     (setf (f2cl-lib:fref err_bnds_norm-%data% (j la_linrx_err_i)
```

```
                                ((1 nrhs) (1 *)) err_bnds_norm-%offset%)
                      (/ final_dx_x (- 1 dxratmax)))))
            (cond
             ((>= n_norms 2)
              (setf (f2cl-lib:fref err_bnds_comp-%data% (j la_linrx_err_i)
                                ((1 nrhs) (1 *)) err_bnds_comp-%offset%)
                      (/ final_dz_z (- 1 dzratmax)))))
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (ccopy n
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 1 res 1)
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf n var-0)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                (csymv uplo n (f2cl-lib:cmplx -1.0) a lda
                 (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                                       ((1 ldy) (1 *)) y-%offset%)
                 1 (f2cl-lib:cmplx 1.0) res 1)
              (declare (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
              (when var-0 (setf uplo var-0))
              (when var-1 (setf n var-1))
              (when var-4 (setf lda var-4)))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref ayb-%data% (i) ((1 *)) ayb-%offset%)
                        (cabs1
                         (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                        b-%offset%)))
               label100005))
            (cla_syamv uplo2 n 1.0 a lda
             (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                                ((1 ldy) (1 *)) y-%offset%)
             1 1.0 ayb 1)
            (cla_lin_berr n n 1 res ayb
             (f2cl-lib:array-slice berr_out-%data% single-float (j) ((1 *))
                                berr_out-%offset%))
           label100000))
         (go end_label)
        end_label
         (return
          (values prec_type uplo n nil nil lda nil ldaf nil nil nil nil nil
                  nil nil nil nil nil nil nil nil nil nil nil nil nil nil nil
                  info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
```

```
(setf (gethash 'fortran-to-lisp::cla_syrfsx_extended
               fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((fortran-to-lisp::integer4)
                                            (string)
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::integer4
                                             (*))
                                            fortran-to-lisp::logical
                                            (array single-float (*))
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array single-float (*))
                                            (fortran-to-lisp::integer4)
                                            (array single-float (*))
                                            (array single-float (*))
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (array single-float (*))
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (single-float)
                                            (fortran-to-lisp::integer4)
                                            (single-float) (single-float)
                                            fortran-to-lisp::logical
                                            (fortran-to-lisp::integer4))
                                          :return-values
                                          '(fortran-to-lisp::prec_type
                                            fortran-to-lisp::uplo
                                            fortran-to-lisp::n nil nil
                                            fortran-to-lisp::lda nil
                                            fortran-to-lisp::ldaf nil nil nil
                                            nil nil nil nil nil nil nil nil
                                            nil nil nil nil nil nil nil nil
                                            nil fortran-to-lisp::info)
                                          :calls
                                          '(fortran-to-lisp::cla_lin_berr
```

```
                                        fortran-to-lisp::cla_syamv))))
```

———-

60.5 cla_syrpvgrw

60.5.1 Axiom unit tests

— **clasyrpvgrw.input** —

```
)set break resume
)sys rm -f clasyrpvgrw.output
)spool clasyrpvgrw.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.5.2 Axiom help page

— **cla_syrpvgrw.help** —

```
======================================================================
cla_syrpvgrw examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       real function cla_syrpvgrw (UPLO, N, INFO, A, LDA, AF, LDAF, IPIV,
           WORK)
           CLA_SYRPVGRW computes the reciprocal pivot growth factor
           norm(A)/norm(U) for a symmetric indefinite matrix.

Function/Subroutine Documentation
```

```
   real function cla_syrpvgrw (character*1 UPLO, integer N, integer INFO,
       complex, dimension( lda, * ) A, integer LDA, complex, dimension( ldaf,
       * ) AF, integer LDAF, integer, dimension( * ) IPIV, real, dimension( *
       ) WORK)
       CLA_SYRPVGRW computes the reciprocal pivot growth factor
       norm(A)/norm(U) for a symmetric indefinite matrix.

Purpose:
   CLA_SYRPVGRW computes the reciprocal pivot growth factor
   norm(A)/norm(U). The "max absolute element" norm is used. If this is
   much less than 1, the stability of the LU factorization of the
   (equilibrated) matrix A could be poor. This also means that the
   solution X, estimated condition numbers, and error bounds could be
   unreliable.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 INFO
   INFO is INTEGER
   The value of INFO returned from CSYTRF, .i.e., the pivot in
   column INFO is exactly 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is COMPLEX array, dimension (LDAF,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by CSYTRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 IPIV
```

```
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by CSYTRF.

 WORK
   WORK is REAL array, dimension (2*N)
```

60.5.3 fortran code

— cla_syrpvgrw.f —

```
*  =====================================================================
      REAL FUNCTION CLA_SYRPVGRW( UPLO, N, INFO, A, LDA, AF, LDAF, IPIV,
     $                            WORK )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER*1        UPLO
      INTEGER            N, INFO, LDA, LDAF
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), AF( LDAF, * )
      REAL               WORK( * )
      INTEGER            IPIV( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            NCOLS, I, J, K, KP
      REAL               AMAX, UMAX, RPVGRW, TMP
      LOGICAL            UPPER
      COMPLEX            ZDUM
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, REAL, AIMAG, MAX, MIN
*     ..
*     .. External Subroutines ..
      EXTERNAL           LSAME, CLASET
      LOGICAL            LSAME
*     ..
```

```
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( REAL ( ZDUM ) ) + ABS( AIMAG ( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      UPPER = LSAME( 'Upper', UPLO )
      IF ( INFO.EQ.0 ) THEN
         IF ( UPPER ) THEN
            NCOLS = 1
         ELSE
            NCOLS = N
         END IF
      ELSE
         NCOLS = INFO
      END IF

      RPVGRW = 1.0
      DO I = 1, 2*N
         WORK( I ) = 0.0
      END DO
*
*     Find the max magnitude entry of each column of A.  Compute the max
*     for all N columns so we can apply the pivot permutation while
*     looping below.  Assume a full factorization is the common case.
*
      IF ( UPPER ) THEN
         DO J = 1, N
            DO I = 1, J
               WORK( N+I ) = MAX( CABS1( A( I, J ) ), WORK( N+I ) )
               WORK( N+J ) = MAX( CABS1( A( I, J ) ), WORK( N+J ) )
            END DO
         END DO
      ELSE
         DO J = 1, N
            DO I = J, N
               WORK( N+I ) = MAX( CABS1( A( I, J ) ), WORK( N+I ) )
               WORK( N+J ) = MAX( CABS1( A( I, J ) ), WORK( N+J ) )
            END DO
         END DO
      END IF
*
*     Now find the max magnitude entry of each column of U or L.  Also
*     permute the magnitudes of A above so they're in the same order as
*     the factor.
*
*     The iteration orders and permutations were copied from csytrs.
*     Calls to SSWAP would be severe overkill.
```

```
*
      IF ( UPPER ) THEN
         K = N
         DO WHILE ( K .LT. NCOLS .AND. K.GT.0 )
            IF ( IPIV( K ).GT.0 ) THEN
!              1x1 pivot
               KP = IPIV( K )
               IF ( KP .NE. K ) THEN
                  TMP = WORK( N+K )
                  WORK( N+K ) = WORK( N+KP )
                  WORK( N+KP ) = TMP
               END IF
               DO I = 1, K
                  WORK( K ) = MAX( CABS1( AF( I, K ) ), WORK( K ) )
               END DO
               K = K - 1
            ELSE
!              2x2 pivot
               KP = -IPIV( K )
               TMP = WORK( N+K-1 )
               WORK( N+K-1 ) = WORK( N+KP )
               WORK( N+KP ) = TMP
               DO I = 1, K-1
                  WORK( K ) = MAX( CABS1( AF( I, K ) ), WORK( K ) )
                  WORK( K-1 ) =
     $                 MAX( CABS1( AF( I, K-1 ) ), WORK( K-1 ) )
               END DO
               WORK( K ) = MAX( CABS1( AF( K, K ) ), WORK( K ) )
               K = K - 2
            END IF
         END DO
         K = NCOLS
         DO WHILE ( K .LE. N )
            IF ( IPIV( K ).GT.0 ) THEN
               KP = IPIV( K )
               IF ( KP .NE. K ) THEN
                  TMP = WORK( N+K )
                  WORK( N+K ) = WORK( N+KP )
                  WORK( N+KP ) = TMP
               END IF
               K = K + 1
            ELSE
               KP = -IPIV( K )
               TMP = WORK( N+K )
               WORK( N+K ) = WORK( N+KP )
               WORK( N+KP ) = TMP
               K = K + 2
            END IF
         END DO
      ELSE
```

```
         K = 1
         DO WHILE ( K .LE. NCOLS )
            IF ( IPIV( K ).GT.0 ) THEN
!              1x1 pivot
               KP = IPIV( K )
               IF ( KP .NE. K ) THEN
                  TMP = WORK( N+K )
                  WORK( N+K ) = WORK( N+KP )
                  WORK( N+KP ) = TMP
               END IF
               DO I = K, N
                  WORK( K ) = MAX( CABS1( AF( I, K ) ), WORK( K ) )
               END DO
               K = K + 1
            ELSE
!              2x2 pivot
               KP = -IPIV( K )
               TMP = WORK( N+K+1 )
               WORK( N+K+1 ) = WORK( N+KP )
               WORK( N+KP ) = TMP
               DO I = K+1, N
                  WORK( K ) = MAX( CABS1( AF( I, K ) ), WORK( K ) )
                  WORK( K+1 ) =
     $                 MAX( CABS1( AF( I, K+1 ) ), WORK( K+1 ) )
               END DO
               WORK( K ) = MAX( CABS1( AF( K, K ) ), WORK( K ) )
               K = K + 2
            END IF
         END DO
         K = NCOLS
         DO WHILE ( K .GE. 1 )
            IF ( IPIV( K ).GT.0 ) THEN
               KP = IPIV( K )
               IF ( KP .NE. K ) THEN
                  TMP = WORK( N+K )
                  WORK( N+K ) = WORK( N+KP )
                  WORK( N+KP ) = TMP
               END IF
               K = K - 1
            ELSE
               KP = -IPIV( K )
               TMP = WORK( N+K )
               WORK( N+K ) = WORK( N+KP )
               WORK( N+KP ) = TMP
               K = K - 2
            ENDIF
         END DO
      END IF
*
*     Compute the *inverse* of the max element growth factor.  Dividing
```

```
*     by zero would imply the largest entry of the factor's column is
*     zero.  Than can happen when either the column of A is zero or
*     massive pivots made the factor underflow to zero.  Neither counts
*     as growth in itself, so simply ignore terms with zero
*     denominators.
*
      IF ( UPPER ) THEN
         DO I = NCOLS, N
            UMAX = WORK( I )
            AMAX = WORK( N+I )
            IF ( UMAX /= 0.0 ) THEN
               RPVGRW = MIN( AMAX / UMAX, RPVGRW )
            END IF
         END DO
      ELSE
         DO I = 1, NCOLS
            UMAX = WORK( I )
            AMAX = WORK( N+I )
            IF ( UMAX /= 0.0 ) THEN
               RPVGRW = MIN( AMAX / UMAX, RPVGRW )
            END IF
         END DO
      END IF

      CLA_SYRPVGRW = RPVGRW
      END
```

———-

60.5.4 lisp code

— **cla_syrpvgrw.lisp** —

```
(defun cla_syrpvgrw (uplo n info a lda af ldaf ipiv work)
  (declare (type (array single-float (*)) work)
           (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex8 (*)) af a)
           (type (f2cl-lib:integer4) ldaf lda info n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (af f2cl-lib:complex8 af-%data% af-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (work single-float work-%data% work-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
      (declare
```

```
 (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
  cabs1))
(prog ((zdum #C(0.0 0.0)) (upper nil) (amax 0.0) (umax 0.0) (rpvgrw 0.0)
       (tmp 0.0) (ncols 0) (i 0) (j 0) (k 0) (kp 0) (cla_syrpvgrw 0.0))
  (declare (type (f2cl-lib:integer4) kp k j i ncols)
           (type (single-float) cla_syrpvgrw tmp rpvgrw umax amax)
           (type f2cl-lib:logical upper)
           (type (f2cl-lib:complex8) zdum))
  (setf upper
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame "Upper" uplo)
            (declare (ignore var-0))
            (when var-1 (setf uplo var-1))
            ret-val))
  (cond ((= info 0) (cond (upper (setf ncols 1)) (t (setf ncols n))))
        (t (setf ncols info)))
  (setf rpvgrw 1.0)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i (f2cl-lib:int-mul 2 n)) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) 0.0)
     label100000))
  (cond
   (upper
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i j) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                 ((1 *)) work-%offset%)
                    (max
                     (cabs1
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))
                     (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                    ((1 *)) work-%offset%)))
            (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j))
                                 ((1 *)) work-%offset%)
                    (max
                     (cabs1
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))
                     (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j))
                                    ((1 *)) work-%offset%)))
           label100002))
       label100001)))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                 ((1 *)) work-%offset%)
                  (max
                   (cabs1
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))
                   (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                  ((1 *)) work-%offset%)))
            (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j))
                                 ((1 *)) work-%offset%)
                  (max
                   (cabs1
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))
                   (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j))
                                  ((1 *)) work-%offset%)))
          label100004))
      label100003))))
  (cond
   (upper
    (tagbody
      (setf k n)
     label100005
      (if (not (and (< k ncols) (> k 0)))
          (go label100006))
      (cond
       ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
        (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
        (cond
         ((/= kp k)
          (setf tmp
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                               ((1 *)) work-%offset%))
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                               ((1 *)) work-%offset%)
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                               ((1 *)) work-%offset%))
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                               ((1 *)) work-%offset%)
                tmp)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i k) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                  (max
```

```
                (cabs1
                 (f2cl-lib:fref af-%data% (i k) ((1 ldaf) (1 *))
                                af-%offset%))
                (f2cl-lib:fref work-%data% (k) ((1 *))
                               work-%offset%)))
    label100007))
 (setf k (f2cl-lib:int-sub k 1)))
(t
 (setf kp
         (f2cl-lib:int-sub
          (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
 (setf tmp
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-sub (f2cl-lib:int-add n k)
                                           1))
                        ((1 *)) work-%offset%))
 (setf (f2cl-lib:fref work-%data%
                      ((f2cl-lib:int-sub (f2cl-lib:int-add n k)
                                         1))
                      ((1 *)) work-%offset%)
         (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                        ((1 *)) work-%offset%))
 (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                      ((1 *)) work-%offset%)
         tmp)
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
             (max
              (cabs1
               (f2cl-lib:fref af-%data% (i k) ((1 ldaf) (1 *))
                              af-%offset%))
              (f2cl-lib:fref work-%data% (k) ((1 *))
                             work-%offset%)))
     (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1))
                          ((1 *)) work-%offset%)
             (max
              (cabs1
               (f2cl-lib:fref af-%data% (i (f2cl-lib:int-sub k 1))
                              ((1 ldaf) (1 *)) af-%offset%))
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1))
                             ((1 *)) work-%offset%)))
    label100008))
 (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
         (max
          (cabs1
           (f2cl-lib:fref af-%data% (k k) ((1 ldaf) (1 *))
                          af-%offset%))
```

```
                (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)))
      (setf k (f2cl-lib:int-sub k 2))))
    (go label100005)
   label100006
    (setf k ncols)
   label100009
    (if (not (<= k n))
        (go label100010))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (cond
       ((/= kp k)
        (setf tmp
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                               ((1 *)) work-%offset%))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                             ((1 *)) work-%offset%)
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                               ((1 *)) work-%offset%))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                             ((1 *)) work-%offset%)
                tmp)))
      (setf k (f2cl-lib:int-add k 1)))
     (t
      (setf kp
              (f2cl-lib:int-sub
               (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
      (setf tmp
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                             ((1 *)) work-%offset%))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                           work-%offset%)
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                             ((1 *)) work-%offset%))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                           ((1 *)) work-%offset%)
              tmp)
      (setf k (f2cl-lib:int-add k 2))))
    (go label100009)
   label100010))
  (t
   (tagbody
    (setf k 1)
   label100011
    (if (not (<= k ncols))
        (go label100012))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
```

```
(cond
 ((/= kp k)
  (setf tmp
          (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                         ((1 *)) work-%offset%))
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                       ((1 *)) work-%offset%)
          (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                         ((1 *)) work-%offset%))
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                       ((1 *)) work-%offset%)
          tmp)))
(f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
            (max
             (cabs1
              (f2cl-lib:fref af-%data% (i k) ((1 ldaf) (1 *))
                             af-%offset%))
             (f2cl-lib:fref work-%data% (k) ((1 *))
                            work-%offset%)))
   label100013))
(setf k (f2cl-lib:int-add k 1)))
(t
 (setf kp
        (f2cl-lib:int-sub
         (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
 (setf tmp
        (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k 1))
                       ((1 *)) work-%offset%))
 (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k 1))
                      ((1 *)) work-%offset%)
        (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                       ((1 *)) work-%offset%))
 (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                      ((1 *)) work-%offset%)
        tmp)
 (f2cl-lib:fdo (i (f2cl-lib:int-add k 1) (f2cl-lib:int-add i 1))
               ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
            (max
             (cabs1
              (f2cl-lib:fref af-%data% (i k) ((1 ldaf) (1 *))
                             af-%offset%))
             (f2cl-lib:fref work-%data% (k) ((1 *))
                            work-%offset%)))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1))
                         ((1 *)) work-%offset%)
```

```
                 (max
                  (cabs1
                   (f2cl-lib:fref af-%data% (i (f2cl-lib:int-add k 1))
                                  ((1 ldaf) (1 *)) af-%offset%))
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1))
                                 ((1 *)) work-%offset%)))
       label100014))
    (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
            (max
             (cabs1
              (f2cl-lib:fref af-%data% (k k) ((1 ldaf) (1 *))
                             af-%offset%))
             (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)))
    (setf k (f2cl-lib:int-add k 2))))
  (go label100011)
 label100012
  (setf k ncols)
 label100015
  (if (not (>= k 1))
      (go label100016))
  (cond
   ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
    (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
    (cond
     ((/= kp k)
      (setf tmp
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                             ((1 *)) work-%offset%))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                           ((1 *)) work-%offset%)
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                             ((1 *)) work-%offset%))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                           ((1 *)) work-%offset%)
              tmp)))
    (setf k (f2cl-lib:int-sub k 1)))
   (t
    (setf kp
            (f2cl-lib:int-sub
             (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
    (setf tmp
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                           ((1 *)) work-%offset%))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                         work-%offset%)
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                           ((1 *)) work-%offset%))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                         ((1 *)) work-%offset%)
            tmp)
```

```
              (setf k (f2cl-lib:int-sub k 2))))
            (go label100015)
           label100016)))
        (cond
         (upper
          (f2cl-lib:fdo (i ncols (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf umax (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
              (setf amax
                      (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                     ((1 *)) work-%offset%))
              (cond
               ((f2cl-lib:f2cl/ umax (= 0.0))
                (setf rpvgrw (min (/ amax umax) rpvgrw))))
             label100017)))
         (t
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i ncols) nil)
            (tagbody
              (setf umax (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
              (setf amax
                      (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                     ((1 *)) work-%offset%))
              (cond
               ((f2cl-lib:f2cl/ umax (= 0.0))
                (setf rpvgrw (min (/ amax umax) rpvgrw))))
             label100018))))
        (setf cla_syrpvgrw rpvgrw)
       end_label
        (return (values cla_syrpvgrw uplo nil nil nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cla_syrpvgrw
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::integer4
                                              (*))
                                             (array single-float (*)))
```

```
                                        :return-values
                                        '(fortran-to-lisp::uplo nil nil nil
                                          nil nil nil nil nil)
                                        :calls 'nil)))
```

———-

60.6 clasyf

60.6.1 Axiom unit tests

— **clasyf.input** —

```
)set break resume
)sys rm -f clasyf.output
)spool clasyf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.6.2 Axiom help page

— **clasyf.help** —

```
====================================================================
clasyf examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine clasyf (UPLO, N, NB, KB, A, LDA, IPIV, W, LDW, INFO)
           CLASYF computes a partial factorization of a complex symmetric
           matrix using the Bunch-Kaufman diagonal pivoting method.
```

```
Function/Subroutine Documentation
   subroutine clasyf (character UPLO, integer N, integer NB, integer KB,
       complex, dimension( lda, * ) A, integer LDA, integer, dimension( * )
       IPIV, complex, dimension( ldw, * ) W, integer LDW, integer INFO)
       CLASYF computes a partial factorization of a complex symmetric matrix
       using the Bunch-Kaufman diagonal pivoting method.

Purpose:
   CLASYF computes a partial factorization of a complex symmetric matrix
   A using the Bunch-Kaufman diagonal pivoting method. The partial
   factorization has the form:

       A  =  ( I  U12 ) ( A11  0  ) (  I       0    )  if UPLO = 'U', or:
             ( 0  U22 ) (  0   D  ) ( U12**T U22**T )

       A  =  ( L11  0 ) ( D    0  ) ( L11**T L21**T )  if UPLO = 'L'
             ( L21  I ) ( 0   A22 ) (  0       I    )

   where the order of D is at most NB. The actual order is returned in
   the argument KB, and is either NB or NB-1, or N if N <= NB.
   Note that U**T denotes the transpose of U.

   CLASYF is an auxiliary routine called by CSYTRF. It uses blocked code
   (calling Level 3 BLAS) to update the submatrix A11 (if UPLO = 'U') or
   A22 (if UPLO = 'L').

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is stored:
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NB
   NB is INTEGER
   The maximum number of columns of the matrix A that should be
   factored.  NB should be at least 2 to allow for 2-by-2 pivot blocks.

 KB
   KB is INTEGER
   The number of columns of A that were actually factored.
   KB is either NB-1 or NB, or N if N <= NB.
```

```
A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  n-by-n upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading n-by-n lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.
  On exit, A contains details of the partial factorization.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D.

  If UPLO = 'U':
     Only the last KB elements of IPIV are set.

     If IPIV(k) > 0, then rows and columns k and IPIV(k) were
     interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) = IPIV(k-1) < 0, then rows and columns
     k-1 and -IPIV(k) were interchanged and D(k-1:k,k-1:k)
     is a 2-by-2 diagonal block.

  If UPLO = 'L':
     Only the first KB elements of IPIV are set.

     If IPIV(k) > 0, then rows and columns k and IPIV(k) were
     interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) = IPIV(k+1) < 0, then rows and columns
     k+1 and -IPIV(k) were interchanged and D(k:k+1,k:k+1)
     is a 2-by-2 diagonal block.

W
  W is COMPLEX array, dimension (LDW,NB)

LDW
  LDW is INTEGER
  The leading dimension of the array W.  LDW >= max(1,N).

INFO
  INFO is INTEGER
  = 0: successful exit
  > 0: if INFO = k, D(k,k) is exactly zero.  The factorization
```

```
       has been completed, but the block diagonal matrix D is
       exactly singular.
```

———-

60.6.3 fortran code

— **clasyf.f** —

```
*  =====================================================================
      SUBROUTINE CLASYF( UPLO, N, NB, KB, A, LDA, IPIV, W, LDW, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, KB, LDA, LDW, N, NB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), W( LDW, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               EIGHT, SEVTEN
      PARAMETER          ( EIGHT = 8.0E+0, SEVTEN = 17.0E+0 )
      COMPLEX            CONE
      PARAMETER          ( CONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            IMAX, J, JB, JJ, JMAX, JP, K, KK, KKW, KP,
     $                   KSTEP, KW
      REAL               ABSAKK, ALPHA, COLMAX, ROWMAX
      COMPLEX            D11, D21, D22, R1, T, Z
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ICAMAX
      EXTERNAL           LSAME, ICAMAX
*     ..
```

```
*     .. External Subroutines ..
      EXTERNAL           CCOPY, CGEMM, CGEMV, CSCAL, CSWAP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, MAX, MIN, REAL, SQRT
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( Z ) = ABS( REAL( Z ) ) + ABS( AIMAG( Z ) )
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
*     Initialize ALPHA for use in choosing pivot block size.
*
      ALPHA = ( ONE+SQRT( SEVTEN ) ) / EIGHT
*
      IF( LSAME( UPLO, 'U' ) ) THEN
*
*        Factorize the trailing columns of A using the upper triangle
*        of A and working backwards, and compute the matrix W = U12*D
*        for use in updating A11
*
*        K is the main loop index, decreasing from N in steps of 1 or 2
*
*        KW is the column of W which corresponds to column K of A
*
         K = N
   10    CONTINUE
         KW = NB + K - N
*
*        Exit from loop
*
         IF( ( K.LE.N-NB+1 .AND. NB.LT.N ) .OR. K.LT.1 )
     $      GO TO 30
*
*        Copy column K of A to column KW of W and update it
*
         CALL CCOPY( K, A( 1, K ), 1, W( 1, KW ), 1 )
         IF( K.LT.N )
     $      CALL CGEMV( 'No transpose', K, N-K, -CONE, A( 1, K+1 ), LDA,
     $                  W( K, KW+1 ), LDW, CONE, W( 1, KW ), 1 )
*
         KSTEP = 1
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
```

```
*
         ABSAKK = CABS1( W( K, KW ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.GT.1 ) THEN
            IMAX = ICAMAX( K-1, W( 1, KW ), 1 )
            COLMAX = CABS1( W( IMAX, KW ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( MAX( ABSAKK, COLMAX ).EQ.ZERO ) THEN
*
*           Column K is zero or underflow: set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
         ELSE
            IF( ABSAKK.GE.ALPHA*COLMAX ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
            ELSE
*
*              Copy column IMAX to column KW-1 of W and update it
*
               CALL CCOPY( IMAX, A( 1, IMAX ), 1, W( 1, KW-1 ), 1 )
               CALL CCOPY( K-IMAX, A( IMAX, IMAX+1 ), LDA,
     $                     W( IMAX+1, KW-1 ), 1 )
               IF( K.LT.N )
     $            CALL CGEMV( 'No transpose', K, N-K, -CONE,
     $                        A( 1, K+1 ), LDA, W( IMAX, KW+1 ), LDW,
     $                        CONE, W( 1, KW-1 ), 1 )
*
*              JMAX is the column-index of the largest off-diagonal
*              element in row IMAX, and ROWMAX is its absolute value
*
               JMAX = IMAX + ICAMAX( K-IMAX, W( IMAX+1, KW-1 ), 1 )
               ROWMAX = CABS1( W( JMAX, KW-1 ) )
               IF( IMAX.GT.1 ) THEN
                  JMAX = ICAMAX( IMAX-1, W( 1, KW-1 ), 1 )
                  ROWMAX = MAX( ROWMAX, CABS1( W( JMAX, KW-1 ) ) )
               END IF
*
               IF( ABSAKK.GE.ALPHA*COLMAX*( COLMAX / ROWMAX ) ) THEN
```

```
*
*                no interchange, use 1-by-1 pivot block
*
                 KP = K
              ELSE IF( CABS1( W( IMAX, KW-1 ) ).GE.ALPHA*ROWMAX ) THEN
*
*                interchange rows and columns K and IMAX, use 1-by-1
*                pivot block
*
                 KP = IMAX
*
*                copy column KW-1 of W to column KW of W
*
                 CALL CCOPY( K, W( 1, KW-1 ), 1, W( 1, KW ), 1 )
              ELSE
*
*                interchange rows and columns K-1 and IMAX, use 2-by-2
*                pivot block
*
                 KP = IMAX
                 KSTEP = 2
              END IF
           END IF
*
*          ============================================================
*
*          KK is the column of A where pivoting step stopped
*
           KK = K - KSTEP + 1
*
*          KKW is the column of W which corresponds to column KK of A
*
           KKW = NB + KK - N
*
*          Interchange rows and columns KP and KK.
*          Updated column KP is already stored in column KKW of W.
*
           IF( KP.NE.KK ) THEN
*
*             Copy non-updated column KK to column KP of submatrix A
*             at step K. No need to copy element into column K
*             (or K and K-1 for 2-by-2 pivot) of A, since these columns
*             will be later overwritten.
*
              A( KP, KP ) = A( KK, KK )
              CALL CCOPY( KK-1-KP, A( KP+1, KK ), 1, A( KP, KP+1 ),
     $                    LDA )
              IF( KP.GT.1 )
     $           CALL CCOPY( KP-1, A( 1, KK ), 1, A( 1, KP ), 1 )
*
```

```
*              Interchange rows KK and KP in last K+1 to N columns of A
*              (columns K (or K and K-1 for 2-by-2 pivot) of A will be
*              later overwritten). Interchange rows KK and KP
*              in last KKW to NB columns of W.
*
               IF( K.LT.N )
     $            CALL CSWAP( N-K, A( KK, K+1 ), LDA, A( KP, K+1 ),
     $                        LDA )
               CALL CSWAP( N-KK+1, W( KK, KKW ), LDW, W( KP, KKW ),
     $                     LDW )
            END IF
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column kw of W now holds
*
*              W(kw) = U(k)*D(k),
*
*              where U(k) is the k-th column of U
*
*              Store subdiag. elements of column U(k)
*              and 1-by-1 block D(k) in column k of A.
*              NOTE: Diagonal element U(k,k) is a UNIT element
*              and not stored.
*                 A(k,k) := D(k,k) = W(k,kw)
*                 A(1:k-1,k) := U(1:k-1,k) = W(1:k-1,kw)/D(k,k)
*
               CALL CCOPY( K, W( 1, KW ), 1, A( 1, K ), 1 )
               R1 = CONE / A( K, K )
               CALL CSCAL( K-1, R1, A( 1, K ), 1 )
*
            ELSE
*
*              2-by-2 pivot block D(k): columns kw and kw-1 of W now hold
*
*              ( W(kw-1) W(kw) ) = ( U(k-1) U(k) )*D(k)
*
*              where U(k) and U(k-1) are the k-th and (k-1)-th columns
*              of U
*
*              Store U(1:k-2,k-1) and U(1:k-2,k) and 2-by-2
*              block D(k-1:k,k-1:k) in columns k-1 and k of A.
*              NOTE: 2-by-2 diagonal block U(k-1:k,k-1:k) is a UNIT
*              block and not stored.
*                 A(k-1:k,k-1:k) := D(k-1:k,k-1:k) = W(k-1:k,kw-1:kw)
*                 A(1:k-2,k-1:k) := U(1:k-2,k:k-1:k) =
*                 = W(1:k-2,kw-1:kw) * ( D(k-1:k,k-1:k)**(-1) )
*
               IF( K.GT.2 ) THEN
*
```

```
*                Compose the columns of the inverse of 2-by-2 pivot
*                block D in the following way to reduce the number
*                of FLOPS when we myltiply panel ( W(kw-1) W(kw) ) by
*                this inverse
*
*                D**(-1) = ( d11 d21 )**(-1) =
*                          ( d21 d22 )
*
*                = 1/(d11*d22-d21**2) * ( ( d22 ) (-d21 ) ) =
*                                       ( (-d21 ) ( d11 ) )
*
*                = 1/d21 * 1/((d11/d21)*(d22/d21)-1) *
*
*                  * ( ( d22/d21 ) (      -1 ) ) =
*                    ( (      -1 ) ( d11/d21 ) )
*
*                = 1/d21 * 1/(D22*D11-1) * ( ( D11 ) (  -1 ) ) =
*                                          ( ( -1  ) ( D22 ) )
*
*                = 1/d21 * T * ( ( D11 ) (  -1 ) )
*                              ( (  -1 ) ( D22 ) )
*
*                = D21 * ( ( D11 ) (  -1 ) )
*                        ( (  -1 ) ( D22 ) )
*
                 D21 = W( K-1, KW )
                 D11 = W( K, KW ) / D21
                 D22 = W( K-1, KW-1 ) / D21
                 T = CONE / ( D11*D22-CONE )
*
*                Update elements in columns A(k-1) and A(k) as
*                dot products of rows of ( W(kw-1) W(kw) ) and columns
*                of D**(-1)
*
                 D21 = T / D21
                 DO 20 J = 1, K - 2
                    A( J, K-1 ) = D21*( D11*W( J, KW-1 )-W( J, KW ) )
                    A( J, K ) = D21*( D22*W( J, KW )-W( J, KW-1 ) )
   20            CONTINUE
              END IF
*
*             Copy D(k) to A
*
              A( K-1, K-1 ) = W( K-1, KW-1 )
              A( K-1, K ) = W( K-1, KW )
              A( K, K ) = W( K, KW )
*
           END IF
*
        END IF
```

```
*
*        Store details of the interchanges in IPIV
*
         IF( KSTEP.EQ.1 ) THEN
            IPIV( K ) = KP
         ELSE
            IPIV( K ) = -KP
            IPIV( K-1 ) = -KP
         END IF
*
*        Decrease K and return to the start of the main loop
*
         K = K - KSTEP
         GO TO 10
*
   30    CONTINUE
*
*        Update the upper triangle of A11 (= A(1:k,1:k)) as
*
*        A11 := A11 - U12*D*U12**T = A11 - U12*W**T
*
*        computing blocks of NB columns at a time
*
         DO 50 J = ( ( K-1 ) / NB )*NB + 1, 1, -NB
            JB = MIN( NB, K-J+1 )
*
*           Update the upper triangle of the diagonal block
*
            DO 40 JJ = J, J + JB - 1
               CALL CGEMV( 'No transpose', JJ-J+1, N-K, -CONE,
     $                     A( J, K+1 ), LDA, W( JJ, KW+1 ), LDW, CONE,
     $                     A( J, JJ ), 1 )
   40       CONTINUE
*
*           Update the rectangular superdiagonal block
*
            CALL CGEMM( 'No transpose', 'Transpose', J-1, JB, N-K,
     $                  -CONE, A( 1, K+1 ), LDA, W( J, KW+1 ), LDW,
     $                  CONE, A( 1, J ), LDA )
   50    CONTINUE
*
*        Put U12 in standard form by partially undoing the interchanges
*        in columns k+1:n looping backwards from k+1 to n
*
         J = K + 1
   60    CONTINUE
*
*           Undo the interchanges (if any) of rows JJ and JP at each
*           step J
*
```

```
*           (Here, J is a diagonal index)
            JJ = J
            JP = IPIV( J )
            IF( JP.LT.0 ) THEN
               JP = -JP
*              (Here, J is a diagonal index)
               J = J + 1
            END IF
*           (NOTE: Here, J is used to determine row length. Length N-J+1
*           of the rows to swap back doesn't include diagonal element)
            J = J + 1
            IF( JP.NE.JJ .AND. J.LE.N )
     $         CALL CSWAP( N-J+1, A( JP, J ), LDA, A( JJ, J ), LDA )
         IF( J.LT.N )
     $      GO TO 60
*
*        Set KB to the number of columns factorized
*
         KB = N - K
*
      ELSE
*
*        Factorize the leading columns of A using the lower triangle
*        of A and working forwards, and compute the matrix W = L21*D
*        for use in updating A22
*
*        K is the main loop index, increasing from 1 in steps of 1 or 2
*
         K = 1
   70    CONTINUE
*
*        Exit from loop
*
         IF( ( K.GE.NB .AND. NB.LT.N ) .OR. K.GT.N )
     $      GO TO 90
*
*        Copy column K of A to column K of W and update it
*
         CALL CCOPY( N-K+1, A( K, K ), 1, W( K, K ), 1 )
         CALL CGEMV( 'No transpose', N-K+1, K-1, -CONE, A( K, 1 ), LDA,
     $               W( K, 1 ), LDW, CONE, W( K, K ), 1 )
*
         KSTEP = 1
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = CABS1( W( K, K ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
```

```
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.LT.N ) THEN
            IMAX = K + ICAMAX( N-K, W( K+1, K ), 1 )
            COLMAX = CABS1( W( IMAX, K ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( MAX( ABSAKK, COLMAX ).EQ.ZERO ) THEN
*
*           Column K is zero or underflow: set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
         ELSE
            IF( ABSAKK.GE.ALPHA*COLMAX ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
            ELSE
*
*              Copy column IMAX to column K+1 of W and update it
*
               CALL CCOPY( IMAX-K, A( IMAX, K ), LDA, W( K, K+1 ), 1 )
               CALL CCOPY( N-IMAX+1, A( IMAX, IMAX ), 1, W( IMAX, K+1 ),
     $                     1 )
               CALL CGEMV( 'No transpose', N-K+1, K-1, -CONE, A( K, 1 ),
     $                     LDA, W( IMAX, 1 ), LDW, CONE, W( K, K+1 ),
     $                     1 )
*
*              JMAX is the column-index of the largest off-diagonal
*              element in row IMAX, and ROWMAX is its absolute value
*
               JMAX = K - 1 + ICAMAX( IMAX-K, W( K, K+1 ), 1 )
               ROWMAX = CABS1( W( JMAX, K+1 ) )
               IF( IMAX.LT.N ) THEN
                  JMAX = IMAX + ICAMAX( N-IMAX, W( IMAX+1, K+1 ), 1 )
                  ROWMAX = MAX( ROWMAX, CABS1( W( JMAX, K+1 ) ) )
               END IF
*
               IF( ABSAKK.GE.ALPHA*COLMAX*( COLMAX / ROWMAX ) ) THEN
*
*                 no interchange, use 1-by-1 pivot block
*
                  KP = K
               ELSE IF( CABS1( W( IMAX, K+1 ) ).GE.ALPHA*ROWMAX ) THEN
```

```
*
*                 interchange rows and columns K and IMAX, use 1-by-1
*                 pivot block
*
                  KP = IMAX
*
*                 copy column K+1 of W to column K of W
*
                  CALL CCOPY( N-K+1, W( K, K+1 ), 1, W( K, K ), 1 )
               ELSE
*
*                 interchange rows and columns K+1 and IMAX, use 2-by-2
*                 pivot block
*
                  KP = IMAX
                  KSTEP = 2
               END IF
            END IF
*
*           ==========================================================
*
*           KK is the column of A where pivoting step stopped
*
            KK = K + KSTEP - 1
*
*           Interchange rows and columns KP and KK.
*           Updated column KP is already stored in column KK of W.
*
            IF( KP.NE.KK ) THEN
*
*              Copy non-updated column KK to column KP of submatrix A
*              at step K. No need to copy element into column K
*              (or K and K+1 for 2-by-2 pivot) of A, since these columns
*              will be later overwritten.
*
               A( KP, KP ) = A( KK, KK )
               CALL CCOPY( KP-KK-1, A( KK+1, KK ), 1, A( KP, KK+1 ),
     $                     LDA )
               IF( KP.LT.N )
     $            CALL CCOPY( N-KP, A( KP+1, KK ), 1, A( KP+1, KP ), 1 )
*
*              Interchange rows KK and KP in first K-1 columns of A
*              (columns K (or K and K+1 for 2-by-2 pivot) of A will be
*              later overwritten). Interchange rows KK and KP
*              in first KK columns of W.
*
               IF( K.GT.1 )
     $            CALL CSWAP( K-1, A( KK, 1 ), LDA, A( KP, 1 ), LDA )
               CALL CSWAP( KK, W( KK, 1 ), LDW, W( KP, 1 ), LDW )
            END IF
```

```
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column k of W now holds
*
*              W(k) = L(k)*D(k),
*
*              where L(k) is the k-th column of L
*
*              Store subdiag. elements of column L(k)
*              and 1-by-1 block D(k) in column k of A.
*              (NOTE: Diagonal element L(k,k) is a UNIT element
*              and not stored)
*                 A(k,k) := D(k,k) = W(k,k)
*                 A(k+1:N,k) := L(k+1:N,k) = W(k+1:N,k)/D(k,k)
*
               CALL CCOPY( N-K+1, W( K, K ), 1, A( K, K ), 1 )
               IF( K.LT.N ) THEN
                  R1 = CONE / A( K, K )
                  CALL CSCAL( N-K, R1, A( K+1, K ), 1 )
               END IF
*
            ELSE
*
*              2-by-2 pivot block D(k): columns k and k+1 of W now hold
*
*              ( W(k) W(k+1) ) = ( L(k) L(k+1) )*D(k)
*
*              where L(k) and L(k+1) are the k-th and (k+1)-th columns
*              of L
*
*              Store L(k+2:N,k) and L(k+2:N,k+1) and 2-by-2
*              block D(k:k+1,k:k+1) in columns k and k+1 of A.
*              (NOTE: 2-by-2 diagonal block L(k:k+1,k:k+1) is a UNIT
*              block and not stored)
*                 A(k:k+1,k:k+1) := D(k:k+1,k:k+1) = W(k:k+1,k:k+1)
*                 A(k+2:N,k:k+1) := L(k+2:N,k:k+1) =
*                 = W(k+2:N,k:k+1) * ( D(k:k+1,k:k+1)**(-1) )
*
               IF( K.LT.N-1 ) THEN
*
*                 Compose the columns of the inverse of 2-by-2 pivot
*                 block D in the following way to reduce the number
*                 of FLOPS when we myltiply panel ( W(k) W(k+1) ) by
*                 this inverse
*
*                 D**(-1) = ( d11 d21 )**(-1) =
*                           ( d21 d22 )
*
*                 = 1/(d11*d22-d21**2) * ( ( d22 ) (-d21 ) ) =
```

```
*                                       ( (-d21 ) ( d11 ) )
*
*                 = 1/d21 * 1/((d11/d21)*(d22/d21)-1) *
*
*                   * ( ( d22/d21 ) (      -1 ) ) =
*                     ( (      -1 ) ( d11/d21 ) )
*
*                 = 1/d21 * 1/(D22*D11-1) * ( ( D11 ) (  -1 ) ) =
*                                           ( ( -1  ) ( D22 ) )
*
*                 = 1/d21 * T * ( ( D11 ) (  -1 ) )
*                               ( (  -1 ) ( D22 ) )
*
*                 = D21 * ( ( D11 ) (  -1 ) )
*                         ( (  -1 ) ( D22 ) )
*
                  D21 = W( K+1, K )
                  D11 = W( K+1, K+1 ) / D21
                  D22 = W( K, K ) / D21
                  T = CONE / ( D11*D22-CONE )
                  D21 = T / D21
*
*                 Update elements in columns A(k) and A(k+1) as
*                 dot products of rows of ( W(k) W(k+1) ) and columns
*                 of D**(-1)
*
                  DO 80 J = K + 2, N
                     A( J, K ) = D21*( D11*W( J, K )-W( J, K+1 ) )
                     A( J, K+1 ) = D21*( D22*W( J, K+1 )-W( J, K ) )
   80             CONTINUE
               END IF
*
*              Copy D(k) to A
*
               A( K, K ) = W( K, K )
               A( K+1, K ) = W( K+1, K )
               A( K+1, K+1 ) = W( K+1, K+1 )
*
            END IF
*
         END IF
*
*        Store details of the interchanges in IPIV
*
         IF( KSTEP.EQ.1 ) THEN
            IPIV( K ) = KP
         ELSE
            IPIV( K ) = -KP
            IPIV( K+1 ) = -KP
         END IF
```

```
*
*        Increase K and return to the start of the main loop
*
         K = K + KSTEP
         GO TO 70
*
   90    CONTINUE
*
*        Update the lower triangle of A22 (= A(k:n,k:n)) as
*
*        A22 := A22 - L21*D*L21**T = A22 - L21*W**T
*
*        computing blocks of NB columns at a time
*
         DO 110 J = K, N, NB
            JB = MIN( NB, N-J+1 )
*
*           Update the lower triangle of the diagonal block
*
            DO 100 JJ = J, J + JB - 1
               CALL CGEMV( 'No transpose', J+JB-JJ, K-1, -CONE,
     $                     A( JJ, 1 ), LDA, W( JJ, 1 ), LDW, CONE,
     $                     A( JJ, JJ ), 1 )
  100       CONTINUE
*
*           Update the rectangular subdiagonal block
*
            IF( J+JB.LE.N )
     $         CALL CGEMM( 'No transpose', 'Transpose', N-J-JB+1, JB,
     $                     K-1, -CONE, A( J+JB, 1 ), LDA, W( J, 1 ),
     $                     LDW, CONE, A( J+JB, J ), LDA )
  110    CONTINUE
*
*        Put L21 in standard form by partially undoing the interchanges
*        of rows in columns 1:k-1 looping backwards from k-1 to 1
*
         J = K - 1
  120    CONTINUE
*
*           Undo the interchanges (if any) of rows JJ and JP at each
*           step J
*
*           (Here, J is a diagonal index)
            JJ = J
            JP = IPIV( J )
            IF( JP.LT.0 ) THEN
               JP = -JP
*              (Here, J is a diagonal index)
               J = J - 1
            END IF
```

```
*           (NOTE: Here, J is used to determine row length. Length J
*           of the rows to swap back doesn't include diagonal element)
            J = J - 1
            IF( JP.NE.JJ .AND. J.GE.1 )
     $         CALL CSWAP( J, A( JP, 1 ), LDA, A( JJ, 1 ), LDA )
         IF( J.GT.1 )
     $      GO TO 120
*
*        Set KB to the number of columns factorized
*
         KB = K - 1
*
      END IF
      RETURN
*
*     End of CLASYF
*
      END
```

———-

60.6.4 lisp code

— **clasyf.lisp** —

```
(let* ((zero 0.0)
       (one 1.0)
       (eight 8.0)
       (sevten 17.0)
       (cone (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 8.0 8.0) eight)
           (type (single-float 17.0 17.0) sevten)
           (type (f2cl-lib:complex8) cone)
           (ignorable zero one eight sevten cone))
  (defun clasyf (uplo n nb kb a lda ipiv w ldw info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) w a)
             (type (f2cl-lib:integer4) info ldw lda kb nb n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (w f2cl-lib:complex8 w-%data% w-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (labels ((cabs1 (z)
                 (+ (abs (f2cl-lib:freal z)) (abs (f2cl-lib:aimag z)))))
```

```
(declare
 (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
  cabs1))
(prog ((d11 #C(0.0 0.0)) (d21 #C(0.0 0.0)) (d22 #C(0.0 0.0))
       (r1 #C(0.0 0.0)) (t$ #C(0.0 0.0)) (z #C(0.0 0.0)) (absakk 0.0)
       (alpha 0.0) (colmax 0.0) (rowmax 0.0) (imax 0) (j 0) (jb 0)
       (jj 0) (jmax 0) (jp 0) (k 0) (kk 0) (kkw 0) (kp 0) (kstep 0)
       (kw 0))
  (declare (type (f2cl-lib:complex8) d11 d21 d22 r1 t$ z)
           (type (single-float) absakk alpha colmax rowmax)
           (type (f2cl-lib:integer4) imax j jb jj jmax jp k kk kkw kp
            kstep kw))
  (setf info 0)
  (setf alpha (/ (+ one (f2cl-lib:fsqrt sevten)) eight))
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)
    (tagbody
      (setf k n)
     label10
      (setf kw (f2cl-lib:int-sub (f2cl-lib:int-add nb k) n))
      (if (or
           (and (<= k (f2cl-lib:int-add (f2cl-lib:int-sub n nb) 1))
                (< nb n))
           (< k 1))
          (go label30))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (ccopy k
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (1 kw)
                                 ((1 ldw) (1 *)) w-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf k var-0)))
      (if (< k n)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10)
              (cgemv "No transpose" k (f2cl-lib:int-sub n k) (- cone)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                     (1 (f2cl-lib:int-add k 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                     (k (f2cl-lib:int-add kw 1))
```

```
                                ((1 ldw) (1 *)) w-%offset%)
          ldw cone
          (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (1 kw)
                                ((1 ldw) (1 *)) w-%offset%)
          1)
      (declare
       (ignore var-0 var-2 var-3 var-4 var-6 var-9 var-10))
      (when var-1 (setf k var-1))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldw var-7))
      (when var-8 (setf cone var-8))))
(setf kstep 1)
(setf absakk
        (cabs1
         (f2cl-lib:fref w-%data% (k kw) ((1 ldw) (1 *))
                        w-%offset%)))
(cond
 ((> k 1)
  (setf imax
          (icamax (f2cl-lib:int-sub k 1)
           (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                 (1 kw) ((1 ldw) (1 *))
                                 w-%offset%)
           1))
  (setf colmax
          (cabs1
           (f2cl-lib:fref w-%data% (imax kw) ((1 ldw) (1 *))
                          w-%offset%))))
 (t (setf colmax zero)))
(cond
 ((= (max absakk colmax) zero)
  (if (= info 0)
      (setf info k))
  (setf kp k))
 (t
  (cond ((>= absakk (* alpha colmax)) (setf kp k))
        (t
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (ccopy imax
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                    (1 imax) ((1 lda) (1 *))
                                    a-%offset%)
              1
              (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                    (1 (f2cl-lib:int-sub kw 1))
                                    ((1 ldw) (1 *)) w-%offset%)
              1)
           (declare (ignore var-1 var-2 var-3 var-4))
           (when var-0 (setf imax var-0)))
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
    (ccopy (f2cl-lib:int-sub k imax)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                           (imax
                            (f2cl-lib:int-add imax 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                           ((+ imax 1)
                            (f2cl-lib:int-sub kw 1))
                           ((1 ldw) (1 *)) w-%offset%)
     1)
  (declare (ignore var-0 var-1 var-3 var-4))
  (when var-2 (setf lda var-2)))
(if (< k n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
         var-8 var-9 var-10)
        (cgemv "No transpose" k (f2cl-lib:int-sub n k)
         (- cone)
         (f2cl-lib:array-slice a-%data%
                               f2cl-lib:complex8
                               (1
                                (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *))
                               a-%offset%)
         lda
         (f2cl-lib:array-slice w-%data%
                               f2cl-lib:complex8
                               (imax
                                (f2cl-lib:int-add kw 1))
                               ((1 ldw) (1 *))
                               w-%offset%)
         ldw cone
         (f2cl-lib:array-slice w-%data%
                               f2cl-lib:complex8
                               (1
                                (f2cl-lib:int-sub kw 1))
                               ((1 ldw) (1 *))
                               w-%offset%)
         1)
      (declare
       (ignore var-0 var-2 var-3 var-4 var-6 var-9
        var-10))
      (when var-1 (setf k var-1))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldw var-7))
      (when var-8 (setf cone var-8))))
(setf jmax
        (f2cl-lib:int-add imax
                          (icamax
```

```
                            (f2cl-lib:int-sub k imax)
                            (f2cl-lib:array-slice
                             w-%data% f2cl-lib:complex8
                             ((+ imax 1)
                              (f2cl-lib:int-sub kw 1))
                             ((1 ldw) (1 *)) w-%offset%)
                            1)))
(setf rowmax
        (cabs1
         (f2cl-lib:fref w-%data%
                        (jmax (f2cl-lib:int-sub kw 1))
                        ((1 ldw) (1 *)) w-%offset%)))
(cond
 ((> imax 1)
  (setf jmax
          (icamax (f2cl-lib:int-sub imax 1)
           (f2cl-lib:array-slice w-%data%
                                 f2cl-lib:complex8
                                 (1
                                  (f2cl-lib:int-sub kw
                                                    1))
                                 ((1 ldw) (1 *))
                                 w-%offset%)
           1))
  (setf rowmax
          (max rowmax
               (cabs1
                (f2cl-lib:fref w-%data%
                               (jmax
                                (f2cl-lib:int-sub kw 1))
                               ((1 ldw) (1 *))
                               w-%offset%))))))
(cond
 ((>= absakk
      (* alpha colmax (f2cl-lib:f2cl/ colmax rowmax)))
  (setf kp k))
 ((>=
   (cabs1
    (f2cl-lib:fref w
                   (imax
                    (f2cl-lib:int-add kw
                                      (f2cl-lib:int-sub
                                       1)))
                   ((1 ldw) (1 *))))
   (* alpha rowmax))
  (setf kp imax)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (ccopy k
       (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                             (1 (f2cl-lib:int-sub kw 1))
```

```
                                        ((1 ldw) (1 *)) w-%offset%)
                  1
                  (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                        (1 kw) ((1 ldw) (1 *))
                                        w-%offset%)
                  1)
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf k var-0))))
           (t (setf kp imax) (setf kstep 2)))))
(setf kk (f2cl-lib:int-add (f2cl-lib:int-sub k kstep) 1))
(setf kkw (f2cl-lib:int-sub (f2cl-lib:int-add nb kk) n))
(cond
 ((/= kp kk)
  (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                       a-%offset%)
          (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                         a-%offset%))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (ccopy (f2cl-lib:int-sub kk 1 kp)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ kp 1) kk) ((1 lda) (1 *))
                             a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             (kp (f2cl-lib:int-add kp 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (if (> kp 1)
      (ccopy (f2cl-lib:int-sub kp 1)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 kk)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 kp)
                             ((1 lda) (1 *)) a-%offset%)
       1))
  (if (< k n)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (cswap (f2cl-lib:int-sub n k)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 (kk (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 (kp (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-3))
        (when var-2 (setf lda var-2))
```

```
        (when var-4 (setf lda var-4))))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (cswap (f2cl-lib:int-add (f2cl-lib:int-sub n kk) 1)
       (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                             (kk kkw) ((1 ldw) (1 *))
                             w-%offset%)
       ldw
       (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                             (kp kkw) ((1 ldw) (1 *))
                             w-%offset%)
       ldw)
    (declare (ignore var-0 var-1 var-3))
    (when var-2 (setf ldw var-2))
    (when var-4 (setf ldw var-4)))))
(cond
 ((= kstep 1)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (ccopy k
       (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (1 kw)
                             ((1 ldw) (1 *)) w-%offset%)
       1
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf k var-0)))
  (setf r1
          (/ cone
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (cscal (f2cl-lib:int-sub k 1) r1
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1)
    (declare (ignore var-0 var-2 var-3))
    (when var-1 (setf r1 var-1))))
 (t
  (cond
   ((> k 2)
    (setf d21
            (f2cl-lib:fref w-%data% ((f2cl-lib:int-sub k 1) kw)
                           ((1 ldw) (1 *)) w-%offset%))
    (setf d11
            (/
             (f2cl-lib:fref w-%data% (k kw) ((1 ldw) (1 *))
                            w-%offset%)
             d21))
    (setf d22
            (/
```

```
            (f2cl-lib:fref w-%data%
                           ((f2cl-lib:int-sub k 1)
                            (f2cl-lib:int-sub kw 1))
                           ((1 ldw) (1 *)) w-%offset%)
            d21))
  (setf t$ (/ cone (- (* d11 d22) cone)))
  (setf d21 (/ t$ d21))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j
                    (f2cl-lib:int-add k
                                      (f2cl-lib:int-sub 2)))
                 nil)
    (tagbody
      (setf (f2cl-lib:fref a-%data%
                           (j (f2cl-lib:int-sub k 1))
                           ((1 lda) (1 *)) a-%offset%)
              (* d21
                 (-
                  (* d11
                     (f2cl-lib:fref w-%data%
                                    (j
                                     (f2cl-lib:int-sub kw 1))
                                    ((1 ldw) (1 *))
                                    w-%offset%))
                  (f2cl-lib:fref w-%data% (j kw)
                                 ((1 ldw) (1 *))
                                 w-%offset%))))
      (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                           a-%offset%)
              (* d21
                 (-
                  (* d22
                     (f2cl-lib:fref w-%data% (j kw)
                                    ((1 ldw) (1 *))
                                    w-%offset%))
                  (f2cl-lib:fref w-%data%
                                 (j (f2cl-lib:int-sub kw 1))
                                 ((1 ldw) (1 *))
                                 w-%offset%))))
     label20))))
(setf (f2cl-lib:fref a-%data%
                     ((f2cl-lib:int-sub k 1)
                      (f2cl-lib:int-sub k 1))
                     ((1 lda) (1 *)) a-%offset%)
        (f2cl-lib:fref w-%data%
                       ((f2cl-lib:int-sub k 1)
                        (f2cl-lib:int-sub kw 1))
                       ((1 ldw) (1 *)) w-%offset%))
(setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                     ((1 lda) (1 *)) a-%offset%)
```

```
                (f2cl-lib:fref w-%data% ((f2cl-lib:int-sub k 1) kw)
                               ((1 ldw) (1 *)) w-%offset%))
        (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                             a-%offset%)
                (f2cl-lib:fref w-%data% (k kw) ((1 ldw) (1 *))
                               w-%offset%))))))
  (cond
   ((= kstep 1)
    (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
            kp))
   (t
    (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
            (f2cl-lib:int-sub kp))
    (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                         ((1 *)) ipiv-%offset%)
            (f2cl-lib:int-sub kp))))
  (setf k (f2cl-lib:int-sub k kstep))
  (go label10)
 label30
  (f2cl-lib:fdo (j
                 (+
                  (*
                   (the f2cl-lib:integer4
                        (truncate (+ k (- 1)) nb))
                   nb)
                  1)
                 (f2cl-lib:int-add j (f2cl-lib:int-sub nb)))
                ((> j 1) nil)
    (tagbody
      (setf jb
              (min (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub k j)
                                          1))))
      (f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
                    ((> jj
                        (f2cl-lib:int-add j jb
                                          (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
               var-8 var-9 var-10)
              (cgemv "No transpose"
               (f2cl-lib:int-add (f2cl-lib:int-sub jj j) 1)
               (f2cl-lib:int-sub n k) (- cone)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                     (j (f2cl-lib:int-add k 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda
```

```
                (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                      (jj (f2cl-lib:int-add kw 1))
                                      ((1 ldw) (1 *)) w-%offset%)
                ldw cone
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                      (j jj) ((1 lda) (1 *))
                                      a-%offset%)
                1)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9
                var-10))
              (when var-5 (setf lda var-5))
              (when var-7 (setf ldw var-7))
              (when var-8 (setf cone var-8)))
            label40))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12)
            (cgemm "No transpose" "Transpose" (f2cl-lib:int-sub j 1)
             jb (f2cl-lib:int-sub n k) (- cone)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   (1 (f2cl-lib:int-add k 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                   (j (f2cl-lib:int-add kw 1))
                                   ((1 ldw) (1 *)) w-%offset%)
             ldw cone
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare
           (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-8 var-11))
          (when var-3 (setf jb var-3))
          (when var-7 (setf lda var-7))
          (when var-9 (setf ldw var-9))
          (when var-10 (setf cone var-10))
          (when var-12 (setf lda var-12)))
      label50))
  (setf j (f2cl-lib:int-add k 1))
 label60
  (setf jj j)
  (setf jp (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
  (cond
   ((< jp 0) (setf jp (f2cl-lib:int-sub jp))
    (setf j (f2cl-lib:int-add j 1))))
  (setf j (f2cl-lib:int-add j 1))
  (if (and (/= jp jj) (<= j n))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (cswap (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1)
```

```
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (jp j)
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (jj j)
                                      ((1 lda) (1 *)) a-%offset%)
                lda)
              (declare (ignore var-0 var-1 var-3))
              (when var-2 (setf lda var-2))
              (when var-4 (setf lda var-4))))
      (if (< j n)
          (go label60))
      (setf kb (f2cl-lib:int-sub n k))))
    (t
     (tagbody
       (setf k 1)
      label70
       (if (or (and (>= k nb) (< nb n)) (> k n))
           (go label90))
       (ccopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (k k)
                              ((1 lda) (1 *)) a-%offset%)
        1
        (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (k k)
                              ((1 ldw) (1 *)) w-%offset%)
        1)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10)
           (cgemv "No transpose"
            (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
            (f2cl-lib:int-sub k 1) (- cone)
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (k 1)
                                  ((1 lda) (1 *)) a-%offset%)
            lda
            (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (k 1)
                                  ((1 ldw) (1 *)) w-%offset%)
            ldw cone
            (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (k k)
                                  ((1 ldw) (1 *)) w-%offset%)
            1)
         (declare
          (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9 var-10))
         (when var-5 (setf lda var-5))
         (when var-7 (setf ldw var-7))
         (when var-8 (setf cone var-8)))
       (setf kstep 1)
       (setf absakk
               (cabs1
                (f2cl-lib:fref w-%data% (k k) ((1 ldw) (1 *))
                               w-%offset%)))
```

```
(cond
 ((< k n)
  (setf imax
          (f2cl-lib:int-add k
                            (icamax (f2cl-lib:int-sub n k)
                             (f2cl-lib:array-slice w-%data%
                                                   f2cl-lib:complex8
                                                   ((+ k 1) k)
                                                   ((1 ldw)
                                                    (1 *))
                                                   w-%offset%)
                             1)))
  (setf colmax
          (cabs1
           (f2cl-lib:fref w-%data% (imax k) ((1 ldw) (1 *))
                          w-%offset%))))
 (t (setf colmax zero)))
(cond
 ((= (max absakk colmax) zero)
  (if (= info 0)
      (setf info k))
  (setf kp k))
 (t
  (cond ((>= absakk (* alpha colmax)) (setf kp k))
        (t
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (ccopy (f2cl-lib:int-sub imax k)
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                    (imax k) ((1 lda) (1 *))
                                    a-%offset%)
              lda
              (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                    (k (f2cl-lib:int-add k 1))
                                    ((1 ldw) (1 *)) w-%offset%)
              1)
           (declare (ignore var-0 var-1 var-3 var-4))
           (when var-2 (setf lda var-2)))
         (ccopy (f2cl-lib:int-add (f2cl-lib:int-sub n imax) 1)
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                (imax imax) ((1 lda) (1 *))
                                a-%offset%)
          1
          (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                (imax (f2cl-lib:int-add k 1))
                                ((1 ldw) (1 *)) w-%offset%)
          1)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
              var-8 var-9 var-10)
             (cgemv "No transpose"
```

```
      (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
      (f2cl-lib:int-sub k 1) (- cone)
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                            (k 1) ((1 lda) (1 *))
                            a-%offset%)
      lda
      (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                            (imax 1) ((1 ldw) (1 *))
                            w-%offset%)
      ldw cone
      (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                            (k (f2cl-lib:int-add k 1))
                            ((1 ldw) (1 *)) w-%offset%)
      1)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9
    var-10))
  (when var-5 (setf lda var-5))
  (when var-7 (setf ldw var-7))
  (when var-8 (setf cone var-8)))
(setf jmax
        (f2cl-lib:int-add (f2cl-lib:int-sub k 1)
                          (icamax
                           (f2cl-lib:int-sub imax k)
                           (f2cl-lib:array-slice
                            w-%data% f2cl-lib:complex8
                            (k (f2cl-lib:int-add k 1))
                            ((1 ldw) (1 *)) w-%offset%)
                           1)))
(setf rowmax
        (cabs1
         (f2cl-lib:fref w-%data%
                        (jmax (f2cl-lib:int-add k 1))
                        ((1 ldw) (1 *)) w-%offset%)))
(cond
 ((< imax n)
  (setf jmax
          (f2cl-lib:int-add imax
                            (icamax
                             (f2cl-lib:int-sub n imax)
                             (f2cl-lib:array-slice
                              w-%data% f2cl-lib:complex8
                              ((+ imax 1)
                               (f2cl-lib:int-add k 1))
                              ((1 ldw) (1 *))
                              w-%offset%)
                             1)))
  (setf rowmax
          (max rowmax
               (cabs1
```

```
                                (f2cl-lib:fref w-%data%
                                               (jmax
                                                (f2cl-lib:int-add k 1))
                                               ((1 ldw) (1 *))
                                               w-%offset%))))))
                        (cond
                         ((>= absakk
                              (* alpha colmax (f2cl-lib:f2cl/ colmax rowmax)))
                          (setf kp k))
                         ((>=
                           (cabs1
                            (f2cl-lib:fref w (imax (f2cl-lib:int-add k 1))
                                           ((1 ldw) (1 *))))
                           (* alpha rowmax))
                          (setf kp imax)
                          (ccopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
                           (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                                 (k (f2cl-lib:int-add k 1))
                                                 ((1 ldw) (1 *)) w-%offset%)
                           1
                           (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                                 (k k) ((1 ldw) (1 *))
                                                 w-%offset%)
                           1))
                         (t (setf kp imax) (setf kstep 2)))))
                (setf kk (f2cl-lib:int-sub (f2cl-lib:int-add k kstep) 1))
                (cond
                 ((/= kp kk)
                  (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                                        a-%offset%)
                          (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                                         a-%offset%))
                  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                      (ccopy (f2cl-lib:int-sub kp kk 1)
                       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                             ((+ kk 1) kk) ((1 lda) (1 *))
                                             a-%offset%)
                       1
                       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                             (kp (f2cl-lib:int-add kk 1))
                                             ((1 lda) (1 *)) a-%offset%)
                       lda)
                    (declare (ignore var-0 var-1 var-2 var-3))
                    (when var-4 (setf lda var-4)))
                  (if (< kp n)
                      (ccopy (f2cl-lib:int-sub n kp)
                       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                             ((+ kp 1) kk) ((1 lda) (1 *))
                                             a-%offset%)
                       1
```

```
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                              ((+ kp 1) kp) ((1 lda) (1 *))
                              a-%offset%)
        1))
    (if (> k 1)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (cswap (f2cl-lib:int-sub k 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   (kk 1) ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   (kp 1) ((1 lda) (1 *))
                                   a-%offset%)
             lda)
          (declare (ignore var-0 var-1 var-3))
          (when var-2 (setf lda var-2))
          (when var-4 (setf lda var-4))))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (cswap kk
         (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (kk 1)
                               ((1 ldw) (1 *)) w-%offset%)
         ldw
         (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (kp 1)
                               ((1 ldw) (1 *)) w-%offset%)
         ldw)
      (declare (ignore var-1 var-3))
      (when var-0 (setf kk var-0))
      (when var-2 (setf ldw var-2))
      (when var-4 (setf ldw var-4)))))
  (cond
   ((= kstep 1)
    (ccopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
     (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (k k)
                           ((1 ldw) (1 *)) w-%offset%)
     1
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (k k)
                           ((1 lda) (1 *)) a-%offset%)
     1)
    (cond
     ((< k n)
      (setf r1
              (/ cone
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (cscal (f2cl-lib:int-sub n k) r1
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 ((+ k 1) k) ((1 lda) (1 *))
                                 a-%offset%)
```

```
          1)
      (declare (ignore var-0 var-2 var-3))
      (when var-1 (setf r1 var-1))))))
(t
 (cond
  ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
   (setf d21
           (f2cl-lib:fref w-%data% ((f2cl-lib:int-add k 1) k)
                          ((1 ldw) (1 *)) w-%offset%))
   (setf d11
           (/
            (f2cl-lib:fref w-%data%
                           ((f2cl-lib:int-add k 1)
                            (f2cl-lib:int-add k 1))
                           ((1 ldw) (1 *)) w-%offset%)
            d21))
   (setf d22
           (/
            (f2cl-lib:fref w-%data% (k k) ((1 ldw) (1 *))
                           w-%offset%)
            d21))
   (setf t$ (/ cone (- (* d11 d22) cone)))
   (setf d21 (/ t$ d21))
   (f2cl-lib:fdo (j (f2cl-lib:int-add k 2)
                  (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                            a-%offset%)
             (* d21
                (-
                 (* d11
                    (f2cl-lib:fref w-%data% (j k)
                                   ((1 ldw) (1 *))
                                   w-%offset%))
                 (f2cl-lib:fref w-%data%
                                (j (f2cl-lib:int-add k 1))
                                ((1 ldw) (1 *))
                                w-%offset%))))
       (setf (f2cl-lib:fref a-%data%
                            (j (f2cl-lib:int-add k 1))
                            ((1 lda) (1 *)) a-%offset%)
             (* d21
                (-
                 (* d22
                    (f2cl-lib:fref w-%data%
                                   (j
                                    (f2cl-lib:int-add k 1))
                                   ((1 ldw) (1 *))
                                   w-%offset%))
```

```
                        (f2cl-lib:fref w-%data% (j k)
                                       ((1 ldw) (1 *))
                                       w-%offset%))))
          label80))))
      (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                           a-%offset%)
              (f2cl-lib:fref w-%data% (k k) ((1 ldw) (1 *))
                             w-%offset%))
      (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                           ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref w-%data% ((f2cl-lib:int-add k 1) k)
                             ((1 ldw) (1 *)) w-%offset%))
      (setf (f2cl-lib:fref a-%data%
                           ((f2cl-lib:int-add k 1)
                            (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref w-%data%
                             ((f2cl-lib:int-add k 1)
                              (f2cl-lib:int-add k 1))
                             ((1 ldw) (1 *)) w-%offset%))))))
  (cond
   ((= kstep 1)
    (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
          kp))
   (t
    (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
          (f2cl-lib:int-sub kp))
    (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
                         ((1 *)) ipiv-%offset%)
          (f2cl-lib:int-sub kp))))
  (setf k (f2cl-lib:int-add k kstep))
  (go label70)
 label90
  (f2cl-lib:fdo (j k (f2cl-lib:int-add j nb))
                ((> j n) nil)
    (tagbody
      (setf jb
              (min (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub n j)
                                          1))))
      (f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
                    ((> jj
                        (f2cl-lib:int-add j jb
                                          (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
               var-8 var-9 var-10)
```

```
            (cgemv "No transpose"
             (f2cl-lib:int-sub (f2cl-lib:int-add j jb) jj)
             (f2cl-lib:int-sub k 1) (- cone)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   (jj 1) ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                   (jj 1) ((1 ldw) (1 *))
                                   w-%offset%)
             ldw cone
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   (jj jj) ((1 lda) (1 *))
                                   a-%offset%)
             1)
          (declare
           (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9
            var-10))
          (when var-5 (setf lda var-5))
          (when var-7 (setf ldw var-7))
          (when var-8 (setf cone var-8)))
      label100))
   (if (<= (f2cl-lib:int-add j jb) n)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
            var-8 var-9 var-10 var-11 var-12)
           (cgemm "No transpose" "Transpose"
            (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) jb
            (f2cl-lib:int-sub k 1) (- cone)
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                  ((+ j jb) 1) ((1 lda) (1 *))
                                  a-%offset%)
            lda
            (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                  (j 1) ((1 ldw) (1 *))
                                  w-%offset%)
            ldw cone
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                  ((+ j jb) j) ((1 lda) (1 *))
                                  a-%offset%)
            lda)
          (declare
           (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-8
            var-11))
          (when var-3 (setf jb var-3))
          (when var-7 (setf lda var-7))
          (when var-9 (setf ldw var-9))
          (when var-10 (setf cone var-10))
          (when var-12 (setf lda var-12))))
label110))
```

```
              (setf j (f2cl-lib:int-sub k 1))
             label120
              (setf jj j)
              (setf jp (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
              (cond
               ((< jp 0) (setf jp (f2cl-lib:int-sub jp))
                (setf j (f2cl-lib:int-sub j 1))))
              (setf j (f2cl-lib:int-sub j 1))
              (if (and (/= jp jj) (>= j 1))
                  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                      (cswap j
                       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (jp 1)
                                             ((1 lda) (1 *)) a-%offset%)
                       lda
                       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (jj 1)
                                             ((1 lda) (1 *)) a-%offset%)
                       lda)
                    (declare (ignore var-1 var-3))
                    (when var-0 (setf j var-0))
                    (when var-2 (setf lda var-2))
                    (when var-4 (setf lda var-4))))
              (if (> j 1)
                  (go label120))
              (setf kb (f2cl-lib:int-sub k 1)))))
          (go end_label)
         end_label
          (return (values uplo nil nil kb nil lda nil nil ldw info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::clasyf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::kb nil
```

```
                                          fortran-to-lisp::lda nil nil
                                          fortran-to-lisp::ldw
                                          fortran-to-lisp::info)
                                        :calls 'nil)))
```

———-

60.7 clasyf_rook

60.7.1 Axiom unit tests

— **clasyfrook.input** —

```
)set break resume
)sys rm -f clasyfrook.output
)spool clasyfrook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.7.2 Axiom help page

— **clasyf_rook.help** —

```
====================================================================
clasyf_rook examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine clasyf_rook (UPLO, N, NB, KB, A, LDA, IPIV, W, LDW, INFO)
           CLASYF_ROOK computes a partial factorization of a complex symmetric
           matrix using the bounded Bunch-Kaufman ('rook') diagonal pivoting
```

```
          method.

Function/Subroutine Documentation
   subroutine clasyf_rook (character UPLO, integer N, integer NB, integer KB,
       complex, dimension( lda, * ) A, integer LDA, integer, dimension( * )
       IPIV, complex, dimension( ldw, * ) W, integer LDW, integer INFO)
       CLASYF_ROOK computes a partial factorization of a complex symmetric
       matrix using the bounded Bunch-Kaufman ('rook') diagonal pivoting
       method.

Purpose:
   CLASYF_ROOK computes a partial factorization of a complex symmetric
   matrix A using the bounded Bunch-Kaufman ("rook") diagonal
   pivoting method. The partial factorization has the form:

       A  =  ( I  U12 ) ( A11  0  ) (  I       0    )  if UPLO = 'U', or:
             ( 0  U22 ) (  0   D  ) ( U12**T U22**T )

       A  =  ( L11  0 ) (  D   0  ) ( L11**T L21**T )  if UPLO = 'L'
             ( L21  I ) (  0  A22 ) (  0       I    )

   where the order of D is at most NB. The actual order is returned in
   the argument KB, and is either NB or NB-1, or N if N <= NB.

   CLASYF_ROOK is an auxiliary routine called by CSYTRF_ROOK. It uses
   blocked code (calling Level 3 BLAS) to update the submatrix
   A11 (if UPLO = 'U') or A22 (if UPLO = 'L').

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is stored:
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NB
   NB is INTEGER
   The maximum number of columns of the matrix A that should be
   factored.  NB should be at least 2 to allow for 2-by-2 pivot blocks.

 KB
   KB is INTEGER
   The number of columns of A that were actually factored.
   KB is either NB-1 or NB, or N if N <= NB.
```

```
A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  n-by-n upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading n-by-n lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.
  On exit, A contains details of the partial factorization.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D.

  If UPLO = 'U':
     Only the last KB elements of IPIV are set.

     If IPIV(k) > 0, then rows and columns k and IPIV(k) were
     interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) < 0 and IPIV(k-1) < 0, then rows and
     columns k and -IPIV(k) were interchanged and rows and
     columns k-1 and -IPIV(k-1) were inerchaged,
     D(k-1:k,k-1:k) is a 2-by-2 diagonal block.

  If UPLO = 'L':
     Only the first KB elements of IPIV are set.

     If IPIV(k) > 0, then rows and columns k and IPIV(k)
     were interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) < 0 and IPIV(k+1) < 0, then rows and
     columns k and -IPIV(k) were interchanged and rows and
     columns k+1 and -IPIV(k+1) were inerchaged,
     D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

W
  W is COMPLEX array, dimension (LDW,NB)

LDW
  LDW is INTEGER
  The leading dimension of the array W.  LDW >= max(1,N).

INFO
```

```
INFO is INTEGER
= 0: successful exit
> 0: if INFO = k, D(k,k) is exactly zero.  The factorization
     has been completed, but the block diagonal matrix D is
     exactly singular.
```

———-

60.7.3 fortran code

— **clasyf_rook.f** —

```
*  ===================================================================
      SUBROUTINE CLASYF_ROOK( UPLO, N, NB, KB, A, LDA, IPIV, W, LDW,
     $                        INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, KB, LDA, LDW, N, NB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), W( LDW, * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               EIGHT, SEVTEN
      PARAMETER          ( EIGHT = 8.0E+0, SEVTEN = 17.0E+0 )
      COMPLEX            CONE, CZERO
      PARAMETER          ( CONE = ( 1.0E+0, 0.0E+0 ),
     $                   CZERO = ( 0.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            DONE
      INTEGER            IMAX, ITEMP, J, JB, JJ, JMAX, JP1, JP2, K, KK,
     $                   KW, KKW, KP, KSTEP, P, II
      REAL               ABSAKK, ALPHA, COLMAX, ROWMAX, STEMP, SFMIN
      COMPLEX            D11, D12, D21, D22, R1, T, Z
```

```
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ICAMAX
      REAL               SLAMCH
      EXTERNAL           LSAME, ICAMAX, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           CCOPY, CGEMM, CGEMV, CSCAL, CSWAP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT, AIMAG, REAL
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( Z ) = ABS( REAL( Z ) ) + ABS( AIMAG( Z ) )
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
*     Initialize ALPHA for use in choosing pivot block size.
*
      ALPHA = ( ONE+SQRT( SEVTEN ) ) / EIGHT
*
*     Compute machine safe minimum
*
      SFMIN = SLAMCH( 'S' )
*
      IF( LSAME( UPLO, 'U' ) ) THEN
*
*        Factorize the trailing columns of A using the upper triangle
*        of A and working backwards, and compute the matrix W = U12*D
*        for use in updating A11
*
*        K is the main loop index, decreasing from N in steps of 1 or 2
*
         K = N
   10    CONTINUE
*
*        KW is the column of W which corresponds to column K of A
*
         KW = NB + K - N
*
*        Exit from loop
*
         IF( ( K.LE.N-NB+1 .AND. NB.LT.N ) .OR. K.LT.1 )
     $      GO TO 30
```

```
*
         KSTEP = 1
         P = K
*
*        Copy column K of A to column KW of W and update it
*
         CALL CCOPY( K, A( 1, K ), 1, W( 1, KW ), 1 )
         IF( K.LT.N )
     $      CALL CGEMV( 'No transpose', K, N-K, -CONE, A( 1, K+1 ),
     $                  LDA, W( K, KW+1 ), LDW, CONE, W( 1, KW ), 1 )
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = CABS1( W( K, KW ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.GT.1 ) THEN
            IMAX = ICAMAX( K-1, W( 1, KW ), 1 )
            COLMAX = CABS1( W( IMAX, KW ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( MAX( ABSAKK, COLMAX ).EQ.ZERO ) THEN
*
*           Column K is zero or underflow: set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
            CALL CCOPY( K, W( 1, KW ), 1, A( 1, K ), 1 )
         ELSE
*
*           ============================================================
*
*           Test for interchange
*
*           Equivalent to testing for ABSAKK.GE.ALPHA*COLMAX
*           (used to handle NaN and Inf)
*
            IF( .NOT.( ABSAKK.LT.ALPHA*COLMAX ) ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
*
```

```
            ELSE
*
               DONE = .FALSE.
*
*              Loop until pivot found
*
   12          CONTINUE
*
*                 Begin pivot search loop body
*
*
*                 Copy column IMAX to column KW-1 of W and update it
*
                  CALL CCOPY( IMAX, A( 1, IMAX ), 1, W( 1, KW-1 ), 1 )
                  CALL CCOPY( K-IMAX, A( IMAX, IMAX+1 ), LDA,
     $                        W( IMAX+1, KW-1 ), 1 )
*
                  IF( K.LT.N )
     $               CALL CGEMV( 'No transpose', K, N-K, -CONE,
     $                           A( 1, K+1 ), LDA, W( IMAX, KW+1 ), LDW,
     $                           CONE, W( 1, KW-1 ), 1 )
*
*                 JMAX is the column-index of the largest off-diagonal
*                 element in row IMAX, and ROWMAX is its absolute value.
*                 Determine both ROWMAX and JMAX.
*
                  IF( IMAX.NE.K ) THEN
                     JMAX = IMAX + ICAMAX( K-IMAX, W( IMAX+1, KW-1 ),
     $                                     1 )
                     ROWMAX = CABS1( W( JMAX, KW-1 ) )
                  ELSE
                     ROWMAX = ZERO
                  END IF
*
                  IF( IMAX.GT.1 ) THEN
                     ITEMP = ICAMAX( IMAX-1, W( 1, KW-1 ), 1 )
                     STEMP = CABS1( W( ITEMP, KW-1 ) )
                     IF( STEMP.GT.ROWMAX ) THEN
                        ROWMAX = STEMP
                        JMAX = ITEMP
                     END IF
                  END IF
*
*                 Equivalent to testing for
*                 CABS1( W( IMAX, KW-1 ) ).GE.ALPHA*ROWMAX
*                 (used to handle NaN and Inf)
*
                  IF( .NOT.(CABS1( W( IMAX, KW-1 ) ).LT.ALPHA*ROWMAX ) )
     $            THEN
*
```

```
*                    interchange rows and columns K and IMAX,
*                    use 1-by-1 pivot block
*
                     KP = IMAX
*
*                    copy column KW-1 of W to column KW of W
*
                     CALL CCOPY( K, W( 1, KW-1 ), 1, W( 1, KW ), 1 )
*
                     DONE = .TRUE.
*
*                 Equivalent to testing for ROWMAX.EQ.COLMAX,
*                 (used to handle NaN and Inf)
*
                  ELSE IF( ( P.EQ.JMAX ) .OR. ( ROWMAX.LE.COLMAX ) )
     $            THEN
*
*                    interchange rows and columns K-1 and IMAX,
*                    use 2-by-2 pivot block
*
                     KP = IMAX
                     KSTEP = 2
                     DONE = .TRUE.
                  ELSE
*
*                    Pivot not found: set params and repeat
*
                     P = IMAX
                     COLMAX = ROWMAX
                     IMAX = JMAX
*
*                    Copy updated JMAXth (next IMAXth) column to Kth of W
*
                     CALL CCOPY( K, W( 1, KW-1 ), 1, W( 1, KW ), 1 )
*
                  END IF
*
*                 End pivot search loop body
*
               IF( .NOT. DONE ) GOTO 12
*
            END IF
*
*           ============================================================
*
            KK = K - KSTEP + 1
*
*           KKW is the column of W which corresponds to column KK of A
*
            KKW = NB + KK - N
```

```
*
            IF( ( KSTEP.EQ.2 ) .AND. ( P.NE.K ) ) THEN
*
*              Copy non-updated column K to column P
*
               CALL CCOPY( K-P, A( P+1, K ), 1, A( P, P+1 ), LDA )
               CALL CCOPY( P, A( 1, K ), 1, A( 1, P ), 1 )
*
*              Interchange rows K and P in last N-K+1 columns of A
*              and last N-K+2 columns of W
*
               CALL CSWAP( N-K+1, A( K, K ), LDA, A( P, K ), LDA )
               CALL CSWAP( N-KK+1, W( K, KKW ), LDW, W( P, KKW ), LDW )
            END IF
*
*           Updated column KP is already stored in column KKW of W
*
            IF( KP.NE.KK ) THEN
*
*              Copy non-updated column KK to column KP
*
               A( KP, K ) = A( KK, K )
               CALL CCOPY( K-1-KP, A( KP+1, KK ), 1, A( KP, KP+1 ),
     $                     LDA )
               CALL CCOPY( KP, A( 1, KK ), 1, A( 1, KP ), 1 )
*
*              Interchange rows KK and KP in last N-KK+1 columns
*              of A and W
*
               CALL CSWAP( N-KK+1, A( KK, KK ), LDA, A( KP, KK ), LDA )
               CALL CSWAP( N-KK+1, W( KK, KKW ), LDW, W( KP, KKW ),
     $                     LDW )
            END IF
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column KW of W now holds
*
*              W(k) = U(k)*D(k)
*
*              where U(k) is the k-th column of U
*
*              Store U(k) in column k of A
*
               CALL CCOPY( K, W( 1, KW ), 1, A( 1, K ), 1 )
               IF( K.GT.1 ) THEN
                  IF( CABS1( A( K, K ) ).GE.SFMIN ) THEN
                     R1 = CONE / A( K, K )
                     CALL CSCAL( K-1, R1, A( 1, K ), 1 )
                  ELSE IF( A( K, K ).NE.CZERO ) THEN
```

```
                  DO 14 II = 1, K - 1
                     A( II, K ) = A( II, K ) / A( K, K )
   14             CONTINUE
                END IF
             END IF
*
          ELSE
*
*            2-by-2 pivot block D(k): columns KW and KW-1 of W now
*            hold
*
*            ( W(k-1) W(k) ) = ( U(k-1) U(k) )*D(k)
*
*            where U(k) and U(k-1) are the k-th and (k-1)-th columns
*            of U
*
             IF( K.GT.2 ) THEN
*
*               Store U(k) and U(k-1) in columns k and k-1 of A
*
                D12 = W( K-1, KW )
                D11 = W( K, KW ) / D12
                D22 = W( K-1, KW-1 ) / D12
                T = CONE / ( D11*D22-CONE )
                DO 20 J = 1, K - 2
                   A( J, K-1 ) = T*( (D11*W( J, KW-1 )-W( J, KW ) ) /
     $                           D12 )
                   A( J, K ) = T*( ( D22*W( J, KW )-W( J, KW-1 ) ) /
     $                         D12 )
   20           CONTINUE
             END IF
*
*            Copy D(k) to A
*
             A( K-1, K-1 ) = W( K-1, KW-1 )
             A( K-1, K ) = W( K-1, KW )
             A( K, K ) = W( K, KW )
          END IF
       END IF
*
*      Store details of the interchanges in IPIV
*
       IF( KSTEP.EQ.1 ) THEN
          IPIV( K ) = KP
       ELSE
          IPIV( K ) = -P
          IPIV( K-1 ) = -KP
       END IF
*
*      Decrease K and return to the start of the main loop
```

```
*
         K = K - KSTEP
         GO TO 10
*
   30    CONTINUE
*
*        Update the upper triangle of A11 (= A(1:k,1:k)) as
*
*        A11 := A11 - U12*D*U12**T = A11 - U12*W**T
*
*        computing blocks of NB columns at a time
*
         DO 50 J = ( ( K-1 ) / NB )*NB + 1, 1, -NB
            JB = MIN( NB, K-J+1 )
*
*           Update the upper triangle of the diagonal block
*
            DO 40 JJ = J, J + JB - 1
               CALL CGEMV( 'No transpose', JJ-J+1, N-K, -CONE,
     $                     A( J, K+1 ), LDA, W( JJ, KW+1 ), LDW, CONE,
     $                     A( J, JJ ), 1 )
   40       CONTINUE
*
*           Update the rectangular superdiagonal block
*
            IF( J.GE.2 )
     $         CALL CGEMM( 'No transpose', 'Transpose', J-1, JB,
     $                  N-K, -CONE, A( 1, K+1 ), LDA, W( J, KW+1 ), LDW,
     $                  CONE, A( 1, J ), LDA )
   50    CONTINUE
*
*        Put U12 in standard form by partially undoing the interchanges
*        in columns k+1:n
*
         J = K + 1
   60    CONTINUE
*
            KSTEP = 1
            JP1 = 1
            JJ = J
            JP2 = IPIV( J )
            IF( JP2.LT.0 ) THEN
               JP2 = -JP2
               J = J + 1
               JP1 = -IPIV( J )
               KSTEP = 2
            END IF
*
            J = J + 1
            IF( JP2.NE.JJ .AND. J.LE.N )
```

```
     $         CALL CSWAP( N-J+1, A( JP2, J ), LDA, A( JJ, J ), LDA )
            JJ = J - 1
            IF( JP1.NE.JJ .AND. KSTEP.EQ.2 )
     $         CALL CSWAP( N-J+1, A( JP1, J ), LDA, A( JJ, J ), LDA )
         IF( J.LE.N )
     $      GO TO 60
*
*        Set KB to the number of columns factorized
*
         KB = N - K
*
      ELSE
*
*        Factorize the leading columns of A using the lower triangle
*        of A and working forwards, and compute the matrix W = L21*D
*        for use in updating A22
*
*        K is the main loop index, increasing from 1 in steps of 1 or 2
*
         K = 1
   70    CONTINUE
*
*        Exit from loop
*
         IF( ( K.GE.NB .AND. NB.LT.N ) .OR. K.GT.N )
     $      GO TO 90
*
         KSTEP = 1
         P = K
*
*        Copy column K of A to column K of W and update it
*
         CALL CCOPY( N-K+1, A( K, K ), 1, W( K, K ), 1 )
         IF( K.GT.1 )
     $      CALL CGEMV( 'No transpose', N-K+1, K-1, -CONE, A( K, 1 ),
     $                  LDA, W( K, 1 ), LDW, CONE, W( K, K ), 1 )
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = CABS1( W( K, K ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.LT.N ) THEN
            IMAX = K + ICAMAX( N-K, W( K+1, K ), 1 )
            COLMAX = CABS1( W( IMAX, K ) )
         ELSE
```

```
            COLMAX = ZERO
         END IF
*
         IF( MAX( ABSAKK, COLMAX ).EQ.ZERO ) THEN
*
*           Column K is zero or underflow: set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
            CALL CCOPY( N-K+1, W( K, K ), 1, A( K, K ), 1 )
         ELSE
*
*           ============================================================
*
*           Test for interchange
*
*           Equivalent to testing for ABSAKK.GE.ALPHA*COLMAX
*           (used to handle NaN and Inf)
*
            IF( .NOT.( ABSAKK.LT.ALPHA*COLMAX ) ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
*
            ELSE
*
               DONE = .FALSE.
*
*              Loop until pivot found
*
   72          CONTINUE
*
*                 Begin pivot search loop body
*
*
*                 Copy column IMAX to column K+1 of W and update it
*
                  CALL CCOPY( IMAX-K, A( IMAX, K ), LDA, W( K, K+1 ), 1)
                  CALL CCOPY( N-IMAX+1, A( IMAX, IMAX ), 1,
     $                        W( IMAX, K+1 ), 1 )
                  IF( K.GT.1 )
     $               CALL CGEMV( 'No transpose', N-K+1, K-1, -CONE,
     $                           A( K, 1 ), LDA, W( IMAX, 1 ), LDW,
     $                           CONE, W( K, K+1 ), 1 )
*
*                 JMAX is the column-index of the largest off-diagonal
*                 element in row IMAX, and ROWMAX is its absolute value.
*                 Determine both ROWMAX and JMAX.
```

```
*
                  IF( IMAX.NE.K ) THEN
                     JMAX = K - 1 + ICAMAX( IMAX-K, W( K, K+1 ), 1 )
                     ROWMAX = CABS1( W( JMAX, K+1 ) )
                  ELSE
                     ROWMAX = ZERO
                  END IF
*
                  IF( IMAX.LT.N ) THEN
                     ITEMP = IMAX + ICAMAX( N-IMAX, W( IMAX+1, K+1 ), 1)
                     STEMP = CABS1( W( ITEMP, K+1 ) )
                     IF( STEMP.GT.ROWMAX ) THEN
                        ROWMAX = STEMP
                        JMAX = ITEMP
                     END IF
                  END IF
*
*                 Equivalent to testing for
*                 CABS1( W( IMAX, K+1 ) ).GE.ALPHA*ROWMAX
*                 (used to handle NaN and Inf)
*
                  IF( .NOT.( CABS1( W( IMAX, K+1 ) ).LT.ALPHA*ROWMAX ) )
     $            THEN
*
*                    interchange rows and columns K and IMAX,
*                    use 1-by-1 pivot block
*
                     KP = IMAX
*
*                    copy column K+1 of W to column K of W
*
                     CALL CCOPY( N-K+1, W( K, K+1 ), 1, W( K, K ), 1 )
*
                     DONE = .TRUE.
*
*                 Equivalent to testing for ROWMAX.EQ.COLMAX,
*                 (used to handle NaN and Inf)
*
                  ELSE IF( ( P.EQ.JMAX ) .OR. ( ROWMAX.LE.COLMAX ) )
     $            THEN
*
*                    interchange rows and columns K+1 and IMAX,
*                    use 2-by-2 pivot block
*
                     KP = IMAX
                     KSTEP = 2
                     DONE = .TRUE.
                  ELSE
*
*                    Pivot not found: set params and repeat
```

```
*
                    P = IMAX
                    COLMAX = ROWMAX
                    IMAX = JMAX
*
*                   Copy updated JMAXth (next IMAXth) column to Kth of W
*
                    CALL CCOPY( N-K+1, W( K, K+1 ), 1, W( K, K ), 1 )
*
                 END IF
*
*                End pivot search loop body
*
              IF( .NOT. DONE ) GOTO 72
*
           END IF
*
*          ==========================================================
*
           KK = K + KSTEP - 1
*
           IF( ( KSTEP.EQ.2 ) .AND. ( P.NE.K ) ) THEN
*
*             Copy non-updated column K to column P
*
              CALL CCOPY( P-K, A( K, K ), 1, A( P, K ), LDA )
              CALL CCOPY( N-P+1, A( P, K ), 1, A( P, P ), 1 )
*
*             Interchange rows K and P in first K columns of A
*             and first K+1 columns of W
*
              CALL CSWAP( K, A( K, 1 ), LDA, A( P, 1 ), LDA )
              CALL CSWAP( KK, W( K, 1 ), LDW, W( P, 1 ), LDW )
           END IF
*
*          Updated column KP is already stored in column KK of W
*
           IF( KP.NE.KK ) THEN
*
*             Copy non-updated column KK to column KP
*
              A( KP, K ) = A( KK, K )
              CALL CCOPY( KP-K-1, A( K+1, KK ), 1, A( KP, K+1 ), LDA )
              CALL CCOPY( N-KP+1, A( KP, KK ), 1, A( KP, KP ), 1 )
*
*             Interchange rows KK and KP in first KK columns of A and W
*
              CALL CSWAP( KK, A( KK, 1 ), LDA, A( KP, 1 ), LDA )
              CALL CSWAP( KK, W( KK, 1 ), LDW, W( KP, 1 ), LDW )
           END IF
```

```
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column k of W now holds
*
*              W(k) = L(k)*D(k)
*
*              where L(k) is the k-th column of L
*
*              Store L(k) in column k of A
*
               CALL CCOPY( N-K+1, W( K, K ), 1, A( K, K ), 1 )
               IF( K.LT.N ) THEN
                  IF( CABS1( A( K, K ) ).GE.SFMIN ) THEN
                     R1 = CONE / A( K, K )
                     CALL CSCAL( N-K, R1, A( K+1, K ), 1 )
                  ELSE IF( A( K, K ).NE.CZERO ) THEN
                     DO 74 II = K + 1, N
                        A( II, K ) = A( II, K ) / A( K, K )
   74                CONTINUE
                  END IF
               END IF
*
            ELSE
*
*              2-by-2 pivot block D(k): columns k and k+1 of W now hold
*
*              ( W(k) W(k+1) ) = ( L(k) L(k+1) )*D(k)
*
*              where L(k) and L(k+1) are the k-th and (k+1)-th columns
*              of L
*
               IF( K.LT.N-1 ) THEN
*
*                 Store L(k) and L(k+1) in columns k and k+1 of A
*
                  D21 = W( K+1, K )
                  D11 = W( K+1, K+1 ) / D21
                  D22 = W( K, K ) / D21
                  T = CONE / ( D11*D22-CONE )
                  DO 80 J = K + 2, N
                     A( J, K ) = T*( ( D11*W( J, K )-W( J, K+1 ) ) /
     $                           D21 )
                     A( J, K+1 ) = T*( ( D22*W( J, K+1 )-W( J, K ) ) /
     $                             D21 )
   80             CONTINUE
               END IF
*
*              Copy D(k) to A
*
```

```
               A( K, K ) = W( K, K )
               A( K+1, K ) = W( K+1, K )
               A( K+1, K+1 ) = W( K+1, K+1 )
            END IF
         END IF
*
*        Store details of the interchanges in IPIV
*
         IF( KSTEP.EQ.1 ) THEN
            IPIV( K ) = KP
         ELSE
            IPIV( K ) = -P
            IPIV( K+1 ) = -KP
         END IF
*
*        Increase K and return to the start of the main loop
*
         K = K + KSTEP
         GO TO 70
*
   90    CONTINUE
*
*        Update the lower triangle of A22 (= A(k:n,k:n)) as
*
*        A22 := A22 - L21*D*L21**T = A22 - L21*W**T
*
*        computing blocks of NB columns at a time
*
         DO 110 J = K, N, NB
            JB = MIN( NB, N-J+1 )
*
*           Update the lower triangle of the diagonal block
*
            DO 100 JJ = J, J + JB - 1
               CALL CGEMV( 'No transpose', J+JB-JJ, K-1, -CONE,
     $                     A( JJ, 1 ), LDA, W( JJ, 1 ), LDW, CONE,
     $                     A( JJ, JJ ), 1 )
  100       CONTINUE
*
*           Update the rectangular subdiagonal block
*
            IF( J+JB.LE.N )
     $         CALL CGEMM( 'No transpose', 'Transpose', N-J-JB+1, JB,
     $                    K-1, -CONE, A( J+JB, 1 ), LDA, W( J, 1 ), LDW,
     $                    CONE, A( J+JB, J ), LDA )
  110    CONTINUE
*
*        Put L21 in standard form by partially undoing the interchanges
*        in columns 1:k-1
*
```

```
         J = K - 1
  120    CONTINUE
*
            KSTEP = 1
            JP1 = 1
            JJ = J
            JP2 = IPIV( J )
            IF( JP2.LT.0 ) THEN
               JP2 = -JP2
               J = J - 1
               JP1 = -IPIV( J )
               KSTEP = 2
            END IF
*
            J = J - 1
            IF( JP2.NE.JJ .AND. J.GE.1 )
     $         CALL CSWAP( J, A( JP2, 1 ), LDA, A( JJ, 1 ), LDA )
            JJ = J + 1
            IF( JP1.NE.JJ .AND. KSTEP.EQ.2 )
     $         CALL CSWAP( J, A( JP1, 1 ), LDA, A( JJ, 1 ), LDA )
         IF( J.GE.1 )
     $      GO TO 120
*
*        Set KB to the number of columns factorized
*
         KB = K - 1
*
      END IF
      RETURN
*
*     End of CLASYF_ROOK
*
      END
```

———-

60.7.4 lisp code

— clasyf_rook.lisp —

```
(let* ((zero 0.0)
       (one 1.0)
       (eight 8.0)
       (sevten 17.0)
       (cone (f2cl-lib:cmplx 1.0 0.0))
       (czero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
```

```
          (type (single-float 8.0 8.0) eight)
          (type (single-float 17.0 17.0) sevten)
          (type (f2cl-lib:complex8) cone)
          (type (f2cl-lib:complex8) czero)
          (ignorable zero one eight sevten cone czero))
 (defun clasyf_rook (uplo n nb kb a lda ipiv w ldw info)
   (declare (type (array f2cl-lib:integer4 (*)) ipiv)
            (type (array f2cl-lib:complex8 (*)) w a)
            (type (f2cl-lib:integer4) info ldw lda kb nb n)
            (type (string *) uplo))
   (f2cl-lib:with-multi-array-data
       ((uplo character uplo-%data% uplo-%offset%)
        (a f2cl-lib:complex8 a-%data% a-%offset%)
        (w f2cl-lib:complex8 w-%data% w-%offset%)
        (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
     (labels ((cabs1 (z)
                (+ (abs (f2cl-lib:freal z)) (abs (f2cl-lib:aimag z)))))
       (declare
        (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
         cabs1))
       (prog ((d11 #C(0.0 0.0)) (d12 #C(0.0 0.0)) (d21 #C(0.0 0.0))
              (d22 #C(0.0 0.0)) (r1 #C(0.0 0.0)) (t$ #C(0.0 0.0))
              (z #C(0.0 0.0)) (absakk 0.0) (alpha 0.0) (colmax 0.0)
              (rowmax 0.0) (stemp 0.0) (sfmin 0.0) (imax 0) (itemp 0) (j 0)
              (jb 0) (jj 0) (jmax 0) (jp1 0) (jp2 0) (k 0) (kk 0) (kw 0)
              (kkw 0) (kp 0) (kstep 0) (p 0) (ii 0) (done nil))
         (declare (type (f2cl-lib:complex8) d11 d12 d21 d22 r1 t$ z)
                  (type (single-float) absakk alpha colmax rowmax stemp sfmin)
                  (type (f2cl-lib:integer4) imax itemp j jb jj jmax jp1 jp2 k
                   kk kw kkw kp kstep p ii)
                  (type f2cl-lib:logical done))
         (setf info 0)
         (setf alpha (/ (+ one (f2cl-lib:fsqrt sevten)) eight))
         (setf sfmin (slamch "S"))
         (cond
          ((multiple-value-bind (ret-val var-0 var-1)
               (lsame uplo "U")
             (declare (ignore var-1))
             (when var-0 (setf uplo var-0))
             ret-val)
           (tagbody
             (setf k n)
            label10
             (setf kw (f2cl-lib:int-sub (f2cl-lib:int-add nb k) n))
             (if (or
                  (and (<= k (f2cl-lib:int-add (f2cl-lib:int-sub n nb) 1))
                       (< nb n))
                  (< k 1))
                 (go label30))
             (setf kstep 1)
```

```
(setf p k)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (ccopy k
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                           ((1 lda) (1 *)) a-%offset%)
     1
     (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (1 kw)
                           ((1 ldw) (1 *)) w-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf k var-0)))
(if (< k n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10)
        (cgemv "No transpose" k (f2cl-lib:int-sub n k) (- cone)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               (1 (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                               (k (f2cl-lib:int-add kw 1))
                               ((1 ldw) (1 *)) w-%offset%)
         ldw cone
         (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (1 kw)
                               ((1 ldw) (1 *)) w-%offset%)
         1)
      (declare
       (ignore var-0 var-2 var-3 var-4 var-6 var-9 var-10))
      (when var-1 (setf k var-1))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldw var-7))
      (when var-8 (setf cone var-8))))
(setf absakk
        (cabs1
         (f2cl-lib:fref w-%data% (k kw) ((1 ldw) (1 *))
                        w-%offset%)))
(cond
 ((> k 1)
  (setf imax
          (icamax (f2cl-lib:int-sub k 1)
           (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                 (1 kw) ((1 ldw) (1 *))
                                 w-%offset%)
           1))
  (setf colmax
          (cabs1
           (f2cl-lib:fref w-%data% (imax kw) ((1 ldw) (1 *))
                          w-%offset%))))
 (t (setf colmax zero)))
```

```
(cond
 ((= (max absakk colmax) zero)
  (if (= info 0)
      (setf info k))
  (setf kp k)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (ccopy k
       (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (1 kw)
                             ((1 ldw) (1 *)) w-%offset%)
       1
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf k var-0))))
 (t
  (cond ((not (< absakk (* alpha colmax))) (setf kp k))
        (t
         (tagbody
           (setf done f2cl-lib:%false%)
          label12
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
               (ccopy imax
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                      (1 imax) ((1 lda) (1 *))
                                      a-%offset%)
                1
                (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                      (1 (f2cl-lib:int-sub kw 1))
                                      ((1 ldw) (1 *)) w-%offset%)
                1)
             (declare (ignore var-1 var-2 var-3 var-4))
             (when var-0 (setf imax var-0)))
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
               (ccopy (f2cl-lib:int-sub k imax)
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                      (imax
                                       (f2cl-lib:int-add imax 1))
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                      ((+ imax 1)
                                       (f2cl-lib:int-sub kw 1))
                                      ((1 ldw) (1 *)) w-%offset%)
                1)
             (declare (ignore var-0 var-1 var-3 var-4))
             (when var-2 (setf lda var-2)))
           (if (< k n)
               (multiple-value-bind
                   (var-0 var-1 var-2 var-3 var-4 var-5 var-6
```

```
                 var-7 var-8 var-9 var-10)
                (cgemv "No transpose" k (f2cl-lib:int-sub n k)
                 (- cone)
                 (f2cl-lib:array-slice a-%data%
                                       f2cl-lib:complex8
                                       (1
                                        (f2cl-lib:int-add k
                                                          1))
                                       ((1 lda) (1 *))
                                       a-%offset%)
                 lda
                 (f2cl-lib:array-slice w-%data%
                                       f2cl-lib:complex8
                                       (imax
                                        (f2cl-lib:int-add kw
                                                          1))
                                       ((1 ldw) (1 *))
                                       w-%offset%)
                 ldw cone
                 (f2cl-lib:array-slice w-%data%
                                       f2cl-lib:complex8
                                       (1
                                        (f2cl-lib:int-sub kw
                                                          1))
                                       ((1 ldw) (1 *))
                                       w-%offset%)
                 1)
               (declare
                (ignore var-0 var-2 var-3 var-4 var-6 var-9
                 var-10))
               (when var-1 (setf k var-1))
               (when var-5 (setf lda var-5))
               (when var-7 (setf ldw var-7))
               (when var-8 (setf cone var-8))))
        (cond
         ((/= imax k)
          (setf jmax
                  (f2cl-lib:int-add imax
                                    (icamax
                                     (f2cl-lib:int-sub k imax)
                                     (f2cl-lib:array-slice
                                      w-%data%
                                      f2cl-lib:complex8
                                      ((+ imax 1)
                                       (f2cl-lib:int-sub kw
                                                         1))
                                      ((1 ldw) (1 *))
                                      w-%offset%)
                                     1)))
          (setf rowmax
```

```
          (cabs1
           (f2cl-lib:fref w-%data%
                          (jmax
                           (f2cl-lib:int-sub kw 1))
                          ((1 ldw) (1 *))
                          w-%offset%))))
 (t (setf rowmax zero)))
(cond
 ((> imax 1)
  (setf itemp
          (icamax (f2cl-lib:int-sub imax 1)
           (f2cl-lib:array-slice w-%data%
                                 f2cl-lib:complex8
                                 (1
                                  (f2cl-lib:int-sub kw
                                                    1))
                                 ((1 ldw) (1 *))
                                 w-%offset%)
           1))
  (setf stemp
          (cabs1
           (f2cl-lib:fref w-%data%
                          (itemp
                           (f2cl-lib:int-sub kw 1))
                          ((1 ldw) (1 *))
                          w-%offset%)))
  (cond
   ((> stemp rowmax) (setf rowmax stemp)
    (setf jmax itemp)))))
(cond
 ((not
   (<
    (cabs1
     (f2cl-lib:fref w
                    (imax
                     (f2cl-lib:int-add kw
                                       (f2cl-lib:int-sub
                                        1)))
                    ((1 ldw) (1 *))))
    (* alpha rowmax)))
  (setf kp imax)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (ccopy k
       (f2cl-lib:array-slice w-%data%
                             f2cl-lib:complex8
                             (1
                              (f2cl-lib:int-sub kw 1))
                             ((1 ldw) (1 *))
                             w-%offset%)
       1
```

```
                    (f2cl-lib:array-slice w-%data%
                                          f2cl-lib:complex8 (1 kw)
                                          ((1 ldw) (1 *))
                                          w-%offset%)
                    1)
                 (declare (ignore var-1 var-2 var-3 var-4))
                 (when var-0 (setf k var-0)))
                (setf done f2cl-lib:%true%))
               ((or (= p jmax) (<= rowmax colmax)) (setf kp imax)
                (setf kstep 2) (setf done f2cl-lib:%true%))
               (t (setf p imax) (setf colmax rowmax)
                (setf imax jmax)
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (ccopy k
                    (f2cl-lib:array-slice w-%data%
                                          f2cl-lib:complex8
                                          (1
                                           (f2cl-lib:int-sub kw 1))
                                          ((1 ldw) (1 *))
                                          w-%offset%)
                    1
                    (f2cl-lib:array-slice w-%data%
                                          f2cl-lib:complex8 (1 kw)
                                          ((1 ldw) (1 *))
                                          w-%offset%)
                    1)
                 (declare (ignore var-1 var-2 var-3 var-4))
                 (when var-0 (setf k var-0)))))
             (if (not done)
                 (go label12)))))
(setf kk (f2cl-lib:int-add (f2cl-lib:int-sub k kstep) 1))
(setf kkw (f2cl-lib:int-sub (f2cl-lib:int-add nb kk) n))
(cond
 ((and (= kstep 2) (/= p k))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (ccopy (f2cl-lib:int-sub k p)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ p 1) k) ((1 lda) (1 *))
                             a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             (p (f2cl-lib:int-add p 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (ccopy p
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                             ((1 lda) (1 *)) a-%offset%)
```

```
                1
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 p)
                                      ((1 lda) (1 *)) a-%offset%)
                1)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf p var-0)))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (cswap (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (k k)
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (p k)
                                      ((1 lda) (1 *)) a-%offset%)
                lda)
            (declare (ignore var-0 var-1 var-3))
            (when var-2 (setf lda var-2))
            (when var-4 (setf lda var-4)))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (cswap (f2cl-lib:int-add (f2cl-lib:int-sub n kk) 1)
                (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (k kkw)
                                      ((1 ldw) (1 *)) w-%offset%)
                ldw
                (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (p kkw)
                                      ((1 ldw) (1 *)) w-%offset%)
                ldw)
            (declare (ignore var-0 var-1 var-3))
            (when var-2 (setf ldw var-2))
            (when var-4 (setf ldw var-4)))))
        (cond
         ((/= kp kk)
          (setf (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                               a-%offset%)
                (f2cl-lib:fref a-%data% (kk k) ((1 lda) (1 *))
                               a-%offset%))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (ccopy (f2cl-lib:int-sub k 1 kp)
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                      ((+ kp 1) kk) ((1 lda) (1 *))
                                      a-%offset%)
                1
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                      (kp (f2cl-lib:int-add kp 1))
                                      ((1 lda) (1 *)) a-%offset%)
                lda)
            (declare (ignore var-0 var-1 var-2 var-3))
            (when var-4 (setf lda var-4)))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (ccopy kp
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 kk)
                                      ((1 lda) (1 *)) a-%offset%)
```

```
          1
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 kp)
                                ((1 lda) (1 *)) a-%offset%)
          1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf kp var-0)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (cswap (f2cl-lib:int-add (f2cl-lib:int-sub n kk) 1)
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (kk kk)
                                ((1 lda) (1 *)) a-%offset%)
          lda
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (kp kk)
                                ((1 lda) (1 *)) a-%offset%)
          lda)
        (declare (ignore var-0 var-1 var-3))
        (when var-2 (setf lda var-2))
        (when var-4 (setf lda var-4)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (cswap (f2cl-lib:int-add (f2cl-lib:int-sub n kk) 1)
          (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                (kk kkw) ((1 ldw) (1 *))
                                w-%offset%)
          ldw
          (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                (kp kkw) ((1 ldw) (1 *))
                                w-%offset%)
          ldw)
        (declare (ignore var-0 var-1 var-3))
        (when var-2 (setf ldw var-2))
        (when var-4 (setf ldw var-4)))))
    (cond
     ((= kstep 1)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (ccopy k
          (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (1 kw)
                                ((1 ldw) (1 *)) w-%offset%)
          1
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                                ((1 lda) (1 *)) a-%offset%)
          1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf k var-0)))
      (cond
       ((> k 1)
        (cond
         ((>= (cabs1 (f2cl-lib:fref a (k k) ((1 lda) (1 *))))
              sfmin)
          (setf r1
                  (/ cone
                     (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
```

```
                                a-%offset%)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (cscal (f2cl-lib:int-sub k 1) r1
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   (1 k) ((1 lda) (1 *))
                                   a-%offset%)
             1)
          (declare (ignore var-0 var-2 var-3))
          (when var-1 (setf r1 var-1))))
       ((/= (f2cl-lib:fref a (k k) ((1 lda) (1 *))) czero)
        (f2cl-lib:fdo (ii 1 (f2cl-lib:int-add ii 1))
                      ((> ii
                          (f2cl-lib:int-add k
                                            (f2cl-lib:int-sub
                                             1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
                                 a-%offset%)
                  (/
                   (f2cl-lib:fref a-%data% (ii k)
                                  ((1 lda) (1 *)) a-%offset%)
                   (f2cl-lib:fref a-%data% (k k)
                                  ((1 lda) (1 *)) a-%offset%)))
           label14)))))))
(t
 (cond
  ((> k 2)
   (setf d12
           (f2cl-lib:fref w-%data% ((f2cl-lib:int-sub k 1) kw)
                          ((1 ldw) (1 *)) w-%offset%))
   (setf d11
           (/
            (f2cl-lib:fref w-%data% (k kw) ((1 ldw) (1 *))
                           w-%offset%)
            d12))
   (setf d22
           (/
            (f2cl-lib:fref w-%data%
                           ((f2cl-lib:int-sub k 1)
                            (f2cl-lib:int-sub kw 1))
                           ((1 ldw) (1 *)) w-%offset%)
            d12))
   (setf t$ (/ cone (- (* d11 d22) cone)))
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j
                     (f2cl-lib:int-add k
                                       (f2cl-lib:int-sub 2)))
                  nil)
     (tagbody
```

```
                (setf (f2cl-lib:fref a-%data%
                                     (j (f2cl-lib:int-sub k 1))
                                     ((1 lda) (1 *)) a-%offset%)
                        (* t$
                           (/
                            (-
                             (* d11
                                (f2cl-lib:fref w-%data%
                                               (j
                                                (f2cl-lib:int-sub kw
                                                                  1))
                                               ((1 ldw) (1 *))
                                               w-%offset%))
                             (f2cl-lib:fref w-%data% (j kw)
                                            ((1 ldw) (1 *))
                                            w-%offset%))
                            d12)))
                (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                     a-%offset%)
                        (* t$
                           (/
                            (-
                             (* d22
                                (f2cl-lib:fref w-%data% (j kw)
                                               ((1 ldw) (1 *))
                                               w-%offset%))
                             (f2cl-lib:fref w-%data%
                                            (j (f2cl-lib:int-sub kw 1))
                                            ((1 ldw) (1 *))
                                            w-%offset%))
                            d12)))
               label20))))
          (setf (f2cl-lib:fref a-%data%
                               ((f2cl-lib:int-sub k 1)
                                (f2cl-lib:int-sub k 1))
                               ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref w-%data%
                               ((f2cl-lib:int-sub k 1)
                                (f2cl-lib:int-sub kw 1))
                               ((1 ldw) (1 *)) w-%offset%))
          (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                               ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref w-%data% ((f2cl-lib:int-sub k 1) kw)
                               ((1 ldw) (1 *)) w-%offset%))
          (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                               a-%offset%)
                (f2cl-lib:fref w-%data% (k kw) ((1 ldw) (1 *))
                               w-%offset%))))))
      (cond
       ((= kstep 1)
```

```
    (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
            kp))
   (t
    (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
            (f2cl-lib:int-sub p))
    (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                         ((1 *)) ipiv-%offset%)
            (f2cl-lib:int-sub kp))))
  (setf k (f2cl-lib:int-sub k kstep))
  (go label10)
 label30
  (f2cl-lib:fdo (j
                 (+
                  (*
                   (the f2cl-lib:integer4
                        (truncate (+ k (- 1)) nb))
                   nb)
                  1)
                 (f2cl-lib:int-add j (f2cl-lib:int-sub nb)))
                ((> j 1) nil)
    (tagbody
      (setf jb
              (min (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub k j)
                                          1))))
      (f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
                    ((> jj
                        (f2cl-lib:int-add j jb
                                          (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
               var-8 var-9 var-10)
              (cgemv "No transpose"
               (f2cl-lib:int-add (f2cl-lib:int-sub jj j) 1)
               (f2cl-lib:int-sub n k) (- cone)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                     (j (f2cl-lib:int-add k 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                     (jj (f2cl-lib:int-add kw 1))
                                     ((1 ldw) (1 *)) w-%offset%)
               ldw cone
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                     (j jj) ((1 lda) (1 *))
                                     a-%offset%)
               1)
```

```
            (declare
             (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9
              var-10))
            (when var-5 (setf lda var-5))
            (when var-7 (setf ldw var-7))
            (when var-8 (setf cone var-8)))
         label40))
      (if (>= j 2)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
               var-8 var-9 var-10 var-11 var-12)
              (cgemm "No transpose" "Transpose"
               (f2cl-lib:int-sub j 1) jb (f2cl-lib:int-sub n k)
               (- cone)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                     (1 (f2cl-lib:int-add k 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                     (j (f2cl-lib:int-add kw 1))
                                     ((1 ldw) (1 *)) w-%offset%)
               ldw cone
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                     (1 j) ((1 lda) (1 *))
                                     a-%offset%)
               lda)
            (declare
             (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-8
              var-11))
            (when var-3 (setf jb var-3))
            (when var-7 (setf lda var-7))
            (when var-9 (setf ldw var-9))
            (when var-10 (setf cone var-10))
            (when var-12 (setf lda var-12))))
    label50))
 (setf j (f2cl-lib:int-add k 1))
label60
 (setf kstep 1)
 (setf jp1 1)
 (setf jj j)
 (setf jp2 (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
 (cond
  ((< jp2 0) (setf jp2 (f2cl-lib:int-sub jp2))
   (setf j (f2cl-lib:int-add j 1))
   (setf jp1
           (f2cl-lib:int-sub
            (f2cl-lib:fref ipiv-%data% (j) ((1 *))
                           ipiv-%offset%)))
   (setf kstep 2)))
 (setf j (f2cl-lib:int-add j 1))
```

```
    (if (and (/= jp2 jj) (<= j n))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (cswap (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (jp2 j)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (jj j)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-1 var-3))
          (when var-2 (setf lda var-2))
          (when var-4 (setf lda var-4))))
    (setf jj (f2cl-lib:int-sub j 1))
    (if (and (/= jp1 jj) (= kstep 2))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (cswap (f2cl-lib:int-add (f2cl-lib:int-sub n j) 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (jp1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (jj j)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-1 var-3))
          (when var-2 (setf lda var-2))
          (when var-4 (setf lda var-4))))
    (if (<= j n)
        (go label60))
    (setf kb (f2cl-lib:int-sub n k))))
(t
 (tagbody
   (setf k 1)
  label70
   (if (or (and (>= k nb) (< nb n)) (> k n))
       (go label90))
   (setf kstep 1)
   (setf p k)
   (ccopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
    (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (k k)
                          ((1 lda) (1 *)) a-%offset%)
    1
    (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (k k)
                          ((1 ldw) (1 *)) w-%offset%)
    1)
   (if (> k 1)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
            var-9 var-10)
           (cgemv "No transpose"
            (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
            (f2cl-lib:int-sub k 1) (- cone)
```

```
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (k 1)
                                ((1 lda) (1 *)) a-%offset%)
          lda
          (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (k 1)
                                ((1 ldw) (1 *)) w-%offset%)
          ldw cone
          (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (k k)
                                ((1 ldw) (1 *)) w-%offset%)
          1)
      (declare
       (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9 var-10))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldw var-7))
      (when var-8 (setf cone var-8))))
(setf absakk
        (cabs1
         (f2cl-lib:fref w-%data% (k k) ((1 ldw) (1 *))
                        w-%offset%)))
(cond
 ((< k n)
  (setf imax
          (f2cl-lib:int-add k
                            (icamax (f2cl-lib:int-sub n k)
                             (f2cl-lib:array-slice w-%data%
                                                   f2cl-lib:complex8
                                                   ((+ k 1) k)
                                                   ((1 ldw)
                                                    (1 *))
                                                   w-%offset%)
                             1)))
  (setf colmax
          (cabs1
           (f2cl-lib:fref w-%data% (imax k) ((1 ldw) (1 *))
                          w-%offset%))))
 (t (setf colmax zero)))
(cond
 ((= (max absakk colmax) zero)
  (if (= info 0)
      (setf info k))
  (setf kp k)
  (ccopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
   (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (k k)
                         ((1 ldw) (1 *)) w-%offset%)
   1
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (k k)
                         ((1 lda) (1 *)) a-%offset%)
   1))
 (t
  (cond ((not (< absakk (* alpha colmax))) (setf kp k))
        (t
```

```
(tagbody
  (setf done f2cl-lib:%false%)
 label72
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (ccopy (f2cl-lib:int-sub imax k)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             (imax k) ((1 lda) (1 *))
                             a-%offset%)
       lda
       (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                             (k (f2cl-lib:int-add k 1))
                             ((1 ldw) (1 *)) w-%offset%)
       1)
    (declare (ignore var-0 var-1 var-3 var-4))
    (when var-2 (setf lda var-2)))
  (ccopy (f2cl-lib:int-add (f2cl-lib:int-sub n imax) 1)
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                         (imax imax) ((1 lda) (1 *))
                         a-%offset%)
   1
   (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                         (imax (f2cl-lib:int-add k 1))
                         ((1 ldw) (1 *)) w-%offset%)
   1)
  (if (> k 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6
           var-7 var-8 var-9 var-10)
          (cgemv "No transpose"
           (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
           (f2cl-lib:int-sub k 1) (- cone)
           (f2cl-lib:array-slice a-%data%
                                 f2cl-lib:complex8 (k 1)
                                 ((1 lda) (1 *))
                                 a-%offset%)
           lda
           (f2cl-lib:array-slice w-%data%
                                 f2cl-lib:complex8
                                 (imax 1)
                                 ((1 ldw) (1 *))
                                 w-%offset%)
           ldw cone
           (f2cl-lib:array-slice w-%data%
                                 f2cl-lib:complex8
                                 (k
                                  (f2cl-lib:int-add k
                                                    1))
                                 ((1 ldw) (1 *))
                                 w-%offset%)
           1)
```

```
      (declare
       (ignore var-0 var-1 var-2 var-3 var-4 var-6
        var-9 var-10))
      (when var-5 (setf lda var-5))
      (when var-7 (setf ldw var-7))
      (when var-8 (setf cone var-8))))
(cond
 ((/= imax k)
  (setf jmax
          (f2cl-lib:int-add (f2cl-lib:int-sub k 1)
                            (icamax
                             (f2cl-lib:int-sub imax k)
                             (f2cl-lib:array-slice
                              w-%data%
                              f2cl-lib:complex8
                              (k
                               (f2cl-lib:int-add k 1))
                              ((1 ldw) (1 *))
                              w-%offset%)
                             1)))
  (setf rowmax
          (cabs1
           (f2cl-lib:fref w-%data%
                          (jmax
                           (f2cl-lib:int-add k 1))
                          ((1 ldw) (1 *))
                          w-%offset%))))
 (t (setf rowmax zero)))
(cond
 ((< imax n)
  (setf itemp
          (f2cl-lib:int-add imax
                            (icamax
                             (f2cl-lib:int-sub n imax)
                             (f2cl-lib:array-slice
                              w-%data%
                              f2cl-lib:complex8
                              ((+ imax 1)
                               (f2cl-lib:int-add k 1))
                              ((1 ldw) (1 *))
                              w-%offset%)
                             1)))
  (setf stemp
          (cabs1
           (f2cl-lib:fref w-%data%
                          (itemp
                           (f2cl-lib:int-add k 1))
                          ((1 ldw) (1 *))
                          w-%offset%)))
  (cond
```

```
                ((> stemp rowmax) (setf rowmax stemp)
                 (setf jmax itemp)))))
              (cond
               ((not
                 (<
                  (cabs1
                   (f2cl-lib:fref w (imax (f2cl-lib:int-add k 1))
                                  ((1 ldw) (1 *))))
                  (* alpha rowmax)))
                (setf kp imax)
                (ccopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
                 (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                       (k (f2cl-lib:int-add k 1))
                                       ((1 ldw) (1 *)) w-%offset%)
                 1
                 (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                       (k k) ((1 ldw) (1 *))
                                       w-%offset%)
                 1)
                (setf done f2cl-lib:%true%))
               ((or (= p jmax) (<= rowmax colmax)) (setf kp imax)
                (setf kstep 2) (setf done f2cl-lib:%true%))
               (t (setf p imax) (setf colmax rowmax)
                (setf imax jmax)
                (ccopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
                 (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                       (k (f2cl-lib:int-add k 1))
                                       ((1 ldw) (1 *)) w-%offset%)
                 1
                 (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                       (k k) ((1 ldw) (1 *))
                                       w-%offset%)
                 1)))
              (if (not done)
                  (go label72)))))
            (setf kk (f2cl-lib:int-sub (f2cl-lib:int-add k kstep) 1))
            (cond
             ((and (= kstep 2) (/= p k))
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (ccopy (f2cl-lib:int-sub p k)
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (k k)
                                         ((1 lda) (1 *)) a-%offset%)
                   1
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (p k)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda)
                (declare (ignore var-0 var-1 var-2 var-3))
                (when var-4 (setf lda var-4)))
              (ccopy (f2cl-lib:int-add (f2cl-lib:int-sub n p) 1)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (p k)
```

```
                        ((1 lda) (1 *)) a-%offset%)
    1
    (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (p p)
                        ((1 lda) (1 *)) a-%offset%)
    1)
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
       (cswap k
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (k 1)
                              ((1 lda) (1 *)) a-%offset%)
        lda
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (p 1)
                              ((1 lda) (1 *)) a-%offset%)
        lda)
     (declare (ignore var-1 var-3))
     (when var-0 (setf k var-0))
     (when var-2 (setf lda var-2))
     (when var-4 (setf lda var-4)))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
       (cswap kk
        (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (k 1)
                              ((1 ldw) (1 *)) w-%offset%)
        ldw
        (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (p 1)
                              ((1 ldw) (1 *)) w-%offset%)
        ldw)
     (declare (ignore var-1 var-3))
     (when var-0 (setf kk var-0))
     (when var-2 (setf ldw var-2))
     (when var-4 (setf ldw var-4)))))
 (cond
  ((/= kp kk)
   (setf (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                        a-%offset%)
         (f2cl-lib:fref a-%data% (kk k) ((1 lda) (1 *))
                        a-%offset%))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
       (ccopy (f2cl-lib:int-sub kp k 1)
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                              ((+ k 1) kk) ((1 lda) (1 *))
                              a-%offset%)
        1
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                              (kp (f2cl-lib:int-add k 1))
                              ((1 lda) (1 *)) a-%offset%)
        lda)
     (declare (ignore var-0 var-1 var-2 var-3))
     (when var-4 (setf lda var-4)))
   (ccopy (f2cl-lib:int-add (f2cl-lib:int-sub n kp) 1)
    (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (kp kk)
                        ((1 lda) (1 *)) a-%offset%)
```

```
     1
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (kp kp)
                           ((1 lda) (1 *)) a-%offset%)
     1)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (cswap kk
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (kk 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (kp 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-1 var-3))
      (when var-0 (setf kk var-0))
      (when var-2 (setf lda var-2))
      (when var-4 (setf lda var-4)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (cswap kk
         (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (kk 1)
                               ((1 ldw) (1 *)) w-%offset%)
         ldw
         (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (kp 1)
                               ((1 ldw) (1 *)) w-%offset%)
         ldw)
      (declare (ignore var-1 var-3))
      (when var-0 (setf kk var-0))
      (when var-2 (setf ldw var-2))
      (when var-4 (setf ldw var-4)))))
  (cond
   ((= kstep 1)
    (ccopy (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
     (f2cl-lib:array-slice w-%data% f2cl-lib:complex8 (k k)
                           ((1 ldw) (1 *)) w-%offset%)
     1
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (k k)
                           ((1 lda) (1 *)) a-%offset%)
     1)
    (cond
     ((< k n)
      (cond
       ((>= (cabs1 (f2cl-lib:fref a (k k) ((1 lda) (1 *))))
            sfmin)
        (setf r1
                (/ cone
                   (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                  a-%offset%)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (cscal (f2cl-lib:int-sub n k) r1
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   ((+ k 1) k) ((1 lda) (1 *))
```

```
                                a-%offset%)
            1)
         (declare (ignore var-0 var-2 var-3))
         (when var-1 (setf r1 var-1))))
      ((/= (f2cl-lib:fref a (k k) ((1 lda) (1 *))) czero)
       (f2cl-lib:fdo (ii (f2cl-lib:int-add k 1)
                      (f2cl-lib:int-add ii 1))
                     ((> ii n) nil)
         (tagbody
           (setf (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
                                a-%offset%)
                 (/
                  (f2cl-lib:fref a-%data% (ii k)
                                 ((1 lda) (1 *)) a-%offset%)
                  (f2cl-lib:fref a-%data% (k k)
                                 ((1 lda) (1 *)) a-%offset%)))
        label74)))))))
(t
 (cond
  ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
   (setf d21
           (f2cl-lib:fref w-%data% ((f2cl-lib:int-add k 1) k)
                          ((1 ldw) (1 *)) w-%offset%))
   (setf d11
           (/
            (f2cl-lib:fref w-%data%
                           ((f2cl-lib:int-add k 1)
                            (f2cl-lib:int-add k 1))
                           ((1 ldw) (1 *)) w-%offset%)
            d21))
   (setf d22
           (/
            (f2cl-lib:fref w-%data% (k k) ((1 ldw) (1 *))
                           w-%offset%)
            d21))
   (setf t$ (/ cone (- (* d11 d22) cone)))
   (f2cl-lib:fdo (j (f2cl-lib:int-add k 2)
                  (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                            a-%offset%)
             (* t$
                (/
                 (-
                  (* d11
                     (f2cl-lib:fref w-%data% (j k)
                                    ((1 ldw) (1 *))
                                    w-%offset%))
                  (f2cl-lib:fref w-%data%
```

```
                                        (j (f2cl-lib:int-add k 1))
                                        ((1 ldw) (1 *))
                                        w-%offset%))
                        d21)))
            (setf (f2cl-lib:fref a-%data%
                                 (j (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
                    (* t$
                       (/
                        (-
                         (* d22
                            (f2cl-lib:fref w-%data%
                                           (j
                                            (f2cl-lib:int-add k 1))
                                           ((1 ldw) (1 *))
                                           w-%offset%))
                         (f2cl-lib:fref w-%data% (j k)
                                        ((1 ldw) (1 *))
                                        w-%offset%))
                        d21)))
          label80))))
     (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                          a-%offset%)
             (f2cl-lib:fref w-%data% (k k) ((1 ldw) (1 *))
                            w-%offset%))
     (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                          ((1 lda) (1 *)) a-%offset%)
             (f2cl-lib:fref w-%data% ((f2cl-lib:int-add k 1) k)
                            ((1 ldw) (1 *)) w-%offset%))
     (setf (f2cl-lib:fref a-%data%
                          ((f2cl-lib:int-add k 1)
                           (f2cl-lib:int-add k 1))
                          ((1 lda) (1 *)) a-%offset%)
             (f2cl-lib:fref w-%data%
                            ((f2cl-lib:int-add k 1)
                             (f2cl-lib:int-add k 1))
                            ((1 ldw) (1 *)) w-%offset%))))))
 (cond
  ((= kstep 1)
   (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
           kp))
  (t
   (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
           (f2cl-lib:int-sub p))
   (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
                        ((1 *)) ipiv-%offset%)
           (f2cl-lib:int-sub kp))))
 (setf k (f2cl-lib:int-add k kstep))
 (go label70)
label90
```

```
(f2cl-lib:fdo (j k (f2cl-lib:int-add j nb))
              ((> j n) nil)
  (tagbody
    (setf jb
            (min (the f2cl-lib:integer4 nb)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-sub n j)
                                        1))))
    (f2cl-lib:fdo (jj j (f2cl-lib:int-add jj 1))
                  ((> jj
                      (f2cl-lib:int-add j jb
                                        (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
             var-8 var-9 var-10)
            (cgemv "No transpose"
             (f2cl-lib:int-sub (f2cl-lib:int-add j jb) jj)
             (f2cl-lib:int-sub k 1) (- cone)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   (jj 1) ((1 lda) (1 *))
                                   a-%offset%)
             lda
             (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                   (jj 1) ((1 ldw) (1 *))
                                   w-%offset%)
             ldw cone
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   (jj jj) ((1 lda) (1 *))
                                   a-%offset%)
             1)
          (declare
           (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9
            var-10))
          (when var-5 (setf lda var-5))
          (when var-7 (setf ldw var-7))
          (when var-8 (setf cone var-8)))
       label100))
    (if (<= (f2cl-lib:int-add j jb) n)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
             var-8 var-9 var-10 var-11 var-12)
            (cgemm "No transpose" "Transpose"
             (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) jb
             (f2cl-lib:int-sub k 1) (- cone)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   ((+ j jb) 1) ((1 lda) (1 *))
                                   a-%offset%)
             lda
```

```
                (f2cl-lib:array-slice w-%data% f2cl-lib:complex8
                                      (j 1) ((1 ldw) (1 *))
                                      w-%offset%)
                ldw cone
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                      ((+ j jb) j) ((1 lda) (1 *))
                                      a-%offset%)
                lda)
              (declare
               (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-8
                var-11))
              (when var-3 (setf jb var-3))
              (when var-7 (setf lda var-7))
              (when var-9 (setf ldw var-9))
              (when var-10 (setf cone var-10))
              (when var-12 (setf lda var-12))))
    label110))
 (setf j (f2cl-lib:int-sub k 1))
label120
 (setf kstep 1)
 (setf jp1 1)
 (setf jj j)
 (setf jp2 (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%))
 (cond
  ((< jp2 0) (setf jp2 (f2cl-lib:int-sub jp2))
   (setf j (f2cl-lib:int-sub j 1))
   (setf jp1
           (f2cl-lib:int-sub
            (f2cl-lib:fref ipiv-%data% (j) ((1 *))
                           ipiv-%offset%)))
   (setf kstep 2)))
 (setf j (f2cl-lib:int-sub j 1))
 (if (and (/= jp2 jj) (>= j 1))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (cswap j
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (jp2 1)
                                ((1 lda) (1 *)) a-%offset%)
          lda
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (jj 1)
                                ((1 lda) (1 *)) a-%offset%)
          lda)
       (declare (ignore var-1 var-3))
       (when var-0 (setf j var-0))
       (when var-2 (setf lda var-2))
       (when var-4 (setf lda var-4))))
 (setf jj (f2cl-lib:int-add j 1))
 (if (and (/= jp1 jj) (= kstep 2))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (cswap j
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (jp1 1)
```

```
                                        ((1 lda) (1 *)) a-%offset%)
                      lda
                      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (jj 1)
                                        ((1 lda) (1 *)) a-%offset%)
                      lda)
                    (declare (ignore var-1 var-3))
                    (when var-0 (setf j var-0))
                    (when var-2 (setf lda var-2))
                    (when var-4 (setf lda var-4))))
                (if (>= j 1)
                    (go label120))
                (setf kb (f2cl-lib:int-sub k 1)))))
          (go end_label)
         end_label
          (return (values uplo nil nil kb nil lda nil nil ldw info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::clasyf_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::kb nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::ldw
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

60.8 csycon

60.8.1 Axiom unit tests

— csycon.input —

```
)set break resume
)sys rm -f csycon.output
)spool csycon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.8.2 Axiom help page

— csycon.help —

```
======================================================================
csycon examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csycon (UPLO, N, A, LDA, IPIV, ANORM, RCOND, WORK, INFO)
           CSYCON

Function/Subroutine Documentation
   subroutine csycon (character UPLO, integer N, complex, dimension( lda, * )
       A, integer LDA, integer, dimension( * ) IPIV, real ANORM, real RCOND,
       complex, dimension( * ) WORK, integer INFO)
       CSYCON

Purpose:
   CSYCON estimates the reciprocal of the condition number (in the
   1-norm) of a complex symmetric matrix A using the factorization
   A = U*D*U**T or A = L*D*L**T computed by CSYTRF.
```

```
   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by CSYTRF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by CSYTRF.

 ANORM
   ANORM is REAL
   The 1-norm of the original matrix A.

 RCOND
   RCOND is REAL
   The reciprocal of the condition number of the matrix A,
   computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is an
   estimate of the 1-norm of inv(A) computed in this routine.

 WORK
   WORK is COMPLEX array, dimension (2*N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

          ———————-
```

60.8.3 fortran code

— csycon.f —

```
*  =====================================================================
      SUBROUTINE CSYCON( UPLO, N, A, LDA, IPIV, ANORM, RCOND, WORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
      REAL               ANORM, RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, KASE
      REAL               AINVNM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLACN2, CSYTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
```

```
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CSYCON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.LE.ZERO ) THEN
         RETURN
      END IF
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
         DO 10 I = N, 1, -1
            IF( IPIV( I ).GT.0 .AND. A( I, I ).EQ.ZERO )
     $         RETURN
   10    CONTINUE
      ELSE
*
*        Lower triangular storage: examine D from top to bottom.
*
         DO 20 I = 1, N
            IF( IPIV( I ).GT.0 .AND. A( I, I ).EQ.ZERO )
     $         RETURN
   20    CONTINUE
      END IF
*
*     Estimate the 1-norm of the inverse.
*
```

```
      KASE = 0
   30 CONTINUE
      CALL CLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
*
*        Multiply by inv(L*D*L**T) or inv(U*D*U**T).
*
         CALL CSYTRS( UPLO, N, 1, A, LDA, IPIV, WORK, N, INFO )
         GO TO 30
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
      RETURN
*
*     End of CSYCON
*
      END
```

———-

60.8.4 lisp code

— **csycon.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun csycon (uplo n a lda ipiv anorm rcond work info)
    (declare (type (single-float) rcond anorm)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (ainvnm 0.0) (i 0) (kase 0) (upper nil))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (single-float) ainvnm)
                 (type (f2cl-lib:integer4) i kase)
```

```
          (type f2cl-lib:logical upper))
  (setf info 0)
  (setf upper
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame uplo "U")
           (declare (ignore var-1))
           (when var-0 (setf uplo var-0))
           ret-val))
  (cond
   ((and (not upper)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4))
   ((< anorm zero) (setf info -6)))
  (cond
   ((/= info 0) (xerbla "CSYCON" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf rcond zero)
  (cond ((= n 0) (setf rcond one) (go end_label))
        ((<= anorm zero) (go end_label)))
  (cond
   (upper
    (f2cl-lib:fdo (i n (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
      (tagbody
        (if (and
             (> (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) 0)
             (= (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                zero))
           (go end_label))
       label10)))
   (t
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (if (and
             (> (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) 0)
             (= (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                zero))
           (go end_label))
       label20))))
  (setf kase 0)
 label30
```

```
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (clacn2 n
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ n 1))
                                   ((1 *)) work-%offset%)
             work ainvnm kase isave)
          (declare (ignore var-1 var-2 var-5))
          (setf n var-0)
          (setf ainvnm var-3)
          (setf kase var-4))
        (cond
         ((/= kase 0)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (csytrs uplo n 1 a lda ipiv work n info)
            (declare (ignore var-2 var-3 var-5 var-6))
            (when var-0 (setf uplo var-0))
            (when var-1 (setf n var-1))
            (when var-4 (setf lda var-4))
            (when var-7 (setf n var-7))
            (when var-8 (setf info var-8)))
          (go label30)))
        (if (/= ainvnm zero)
            (setf rcond (/ (/ one ainvnm) anorm)))
        (go end_label)
       end_label
        (return (values uplo n nil lda nil nil rcond nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csycon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (single-float) (single-float)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::rcond nil
                                              fortran-to-lisp::info)
```

```
                                        :calls '(fortran-to-lisp::clacn2))))
```

———-

60.9 csycon_rook

60.9.1 Axiom unit tests

— **csyconrook.input** —

```
)set break resume
)sys rm -f csyconrook.output
)spool csyconrook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.9.2 Axiom help page

— **csycon_rook.help** —

```
====================================================================
csycon_rook examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csycon_rook (UPLO, N, A, LDA, IPIV, ANORM, RCOND, WORK,
           INFO)
           CSYCON_ROOK

Function/Subroutine Documentation
   subroutine csycon_rook (character UPLO, integer N, complex, dimension( lda,
```

```
      * ) A, integer LDA, integer, dimension( * ) IPIV, real ANORM, real
      RCOND, complex, dimension( * ) WORK, integer INFO)
      CSYCON_ROOK

Purpose:
   CSYCON_ROOK estimates the reciprocal of the condition number (in the
   1-norm) of a complex symmetric matrix A using the factorization
   A = U*D*U**T or A = L*D*L**T computed by CSYTRF_ROOK.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by CSYTRF_ROOK.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by CSYTRF_ROOK.

 ANORM
   ANORM is REAL
   The 1-norm of the original matrix A.

 RCOND
   RCOND is REAL
   The reciprocal of the condition number of the matrix A,
   computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is an
   estimate of the 1-norm of inv(A) computed in this routine.

 WORK
```

```
  WORK is COMPLEX array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

60.9.3 fortran code

— **csycon_rook.f** —

```
*  ===================================================================
      SUBROUTINE CSYCON_ROOK( UPLO, N, A, LDA, IPIV, ANORM, RCOND, WORK,
     $                        INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
      REAL               ANORM, RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), WORK( * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
      COMPLEX            CZERO
      PARAMETER          ( CZERO = ( 0.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, KASE
      REAL               AINVNM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
```

```
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLACN2, CSYTRS_ROOK, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CSYCON_ROOK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.LE.ZERO ) THEN
         RETURN
      END IF
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
         DO 10 I = N, 1, -1
            IF( IPIV( I ).GT.0 .AND. A( I, I ).EQ.CZERO )
     $         RETURN
```

```
   10    CONTINUE
      ELSE
*
*        Lower triangular storage: examine D from top to bottom.
*
         DO 20 I = 1, N
            IF( IPIV( I ).GT.0 .AND. A( I, I ).EQ.CZERO )
     $         RETURN
   20    CONTINUE
      END IF
*
*     Estimate the 1-norm of the inverse.
*
      KASE = 0
   30 CONTINUE
      CALL CLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
*
*        Multiply by inv(L*D*L**T) or inv(U*D*U**T).
*
         CALL CSYTRS_ROOK( UPLO, N, 1, A, LDA, IPIV, WORK, N, INFO )
         GO TO 30
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
      RETURN
*
*     End of CSYCON_ROOK
*
      END
```

60.9.4 lisp code

— csycon_rook.lisp —

```
(let* ((one 1.0) (zero 0.0) (czero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (type (f2cl-lib:complex8) czero)
           (ignorable one zero czero))
  (defun csycon_rook (uplo n a lda ipiv anorm rcond work info)
    (declare (type (single-float) rcond anorm)
```

```
          (type (array f2cl-lib:integer4 (*)) ipiv)
          (type (array f2cl-lib:complex8 (*)) work a)
          (type (f2cl-lib:integer4) info lda n)
          (type (string *) uplo))
(f2cl-lib:with-multi-array-data
    ((uplo character uplo-%data% uplo-%offset%)
     (a f2cl-lib:complex8 a-%data% a-%offset%)
     (work f2cl-lib:complex8 work-%data% work-%offset%)
     (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
  (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
         (ainvnm 0.0) (i 0) (kase 0) (upper nil))
    (declare (type (array f2cl-lib:integer4 (3)) isave)
             (type (single-float) ainvnm)
             (type (f2cl-lib:integer4) i kase)
             (type f2cl-lib:logical upper))
    (setf info 0)
    (setf upper
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
    (cond
     ((and (not upper)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
      (setf info -1))
     ((< n 0) (setf info -2))
     ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -4))
     ((< anorm zero) (setf info -6)))
    (cond
     ((/= info 0) (xerbla "CSYCON_ROOK" (f2cl-lib:int-sub info))
      (go end_label)))
    (setf rcond zero)
    (cond ((= n 0) (setf rcond one) (go end_label))
          ((<= anorm zero) (go end_label)))
    (cond
     (upper
      (f2cl-lib:fdo (i n (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                    ((> i 1) nil)
        (tagbody
          (if (and
               (> (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) 0)
               (= (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                  czero))
```

```
                 (go end_label))
             label10)))
          (t
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (if (and
                    (> (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) 0)
                    (= (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                       czero))
                 (go end_label))
             label20))))
         (setf kase 0)
        label30
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
             (clacn2 n
              (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ n 1))
                                    ((1 *)) work-%offset%)
              work ainvnm kase isave)
           (declare (ignore var-1 var-2 var-5))
           (setf n var-0)
           (setf ainvnm var-3)
           (setf kase var-4))
         (cond
          ((/= kase 0)
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
               (csytrs_rook uplo n 1 a lda ipiv work n info)
             (declare (ignore var-2 var-3 var-5 var-6))
             (when var-0 (setf uplo var-0))
             (when var-1 (setf n var-1))
             (when var-4 (setf lda var-4))
             (when var-7 (setf n var-7))
             (when var-8 (setf info var-8)))
           (go label30)))
         (if (/= ainvnm zero)
             (setf rcond (/ (/ one ainvnm) anorm)))
         (go end_label)
        end_label
         (return (values uplo n nil lda nil nil rcond nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csycon_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
```

```
    (*))
   (fortran-to-lisp::integer4)
   (array fortran-to-lisp::integer4
    (*))
   (single-float) (single-float)
   (array fortran-to-lisp::complex8
    (*))
   (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::uplo
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil
  fortran-to-lisp::rcond nil
  fortran-to-lisp::info)
:calls '(fortran-to-lisp::clacn2))))
```

———-

60.10 csyconv

60.10.1 Axiom unit tests

— **csyconv.input** —

```
)set break resume
)sys rm -f csyconv.output
)spool csyconv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.10.2 Axiom help page

— **csyconv.help** —

```
=====================================================================
csyconv examples
```

```
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csyconv (UPLO, WAY, N, A, LDA, IPIV, E, INFO)
           CSYCONV

Function/Subroutine Documentation
   subroutine csyconv (character UPLO, character WAY, integer N, complex,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       complex, dimension( * ) E, integer INFO)
       CSYCONV

Purpose:
   CSYCONV convert A given by TRF into L and D and vice-versa.
   Get Non-diag elements of D (returned in workspace) and
   apply or reverse permutation done in TRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 WAY
   WAY is CHARACTER*1
   = 'C': Convert
   = 'R': Revert

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by CSYTRF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
```

```
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by CSYTRF.

E
  E is COMPLEX array, dimension (N)
  E stores the supdiagonal/subdiagonal of the symmetric 1-by-1
  or 2-by-2 block diagonal matrix D in LDLT.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

60.10.3 fortran code

— **csyconv.f** —

```
*  =====================================================================
      SUBROUTINE CSYCONV( UPLO, WAY, N, A, LDA, IPIV, E, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO, WAY
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ZERO
      PARAMETER          ( ZERO = (0.0E+0,0.0E+0) )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*
```

```
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     .. Local Scalars ..
      LOGICAL            UPPER, CONVERT
      INTEGER            I, IP, J
      COMPLEX            TEMP
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      CONVERT = LSAME( WAY, 'C' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.CONVERT .AND. .NOT.LSAME( WAY, 'R' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5

      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CSYCONV', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*      A is UPPER
*
*      Convert A (A is upper)
*
*        Convert VALUE
*
         IF ( CONVERT ) THEN
            I=N
            E(1)=ZERO
            DO WHILE ( I .GT. 1 )
               IF( IPIV(I) .LT. 0 ) THEN
                  E(I)=A(I-1,I)
                  E(I-1)=ZERO
                  A(I-1,I)=ZERO
                  I=I-1
               ELSE
```

```
                 E(I)=ZERO
              ENDIF
              I=I-1
           END DO
*
*        Convert PERMUTATIONS
*
         I=N
         DO WHILE ( I .GE. 1 )
            IF( IPIV(I) .GT. 0) THEN
               IP=IPIV(I)
               IF( I .LT. N) THEN
                  DO 12 J= I+1,N
                    TEMP=A(IP,J)
                    A(IP,J)=A(I,J)
                    A(I,J)=TEMP
 12            CONTINUE
               ENDIF
            ELSE
              IP=-IPIV(I)
               IF( I .LT. N) THEN
             DO 13 J= I+1,N
                 TEMP=A(IP,J)
                 A(IP,J)=A(I-1,J)
                 A(I-1,J)=TEMP
 13            CONTINUE
                ENDIF
                I=I-1
           ENDIF
           I=I-1
        END DO

         ELSE
*
*      Revert A (A is upper)
*
*
*        Revert PERMUTATIONS
*
            I=1
            DO WHILE ( I .LE. N )
               IF( IPIV(I) .GT. 0 ) THEN
                  IP=IPIV(I)
                  IF( I .LT. N) THEN
                  DO J= I+1,N
                    TEMP=A(IP,J)
                    A(IP,J)=A(I,J)
                    A(I,J)=TEMP
                  END DO
                  ENDIF
```

```
               ELSE
                 IP=-IPIV(I)
                 I=I+1
                 IF( I .LT. N) THEN
                    DO J= I+1,N
                       TEMP=A(IP,J)
                       A(IP,J)=A(I-1,J)
                       A(I-1,J)=TEMP
                    END DO
                 ENDIF
               ENDIF
               I=I+1
            END DO
*
*        Revert VALUE
*
            I=N
            DO WHILE ( I .GT. 1 )
               IF( IPIV(I) .LT. 0 ) THEN
                  A(I-1,I)=E(I)
                  I=I-1
               ENDIF
               I=I-1
            END DO
         END IF
      ELSE
*
*      A is LOWER
*
         IF ( CONVERT ) THEN
*
*      Convert A (A is lower)
*
*
*        Convert VALUE
*
            I=1
            E(N)=ZERO
            DO WHILE ( I .LE. N )
               IF( I.LT.N .AND. IPIV(I) .LT. 0 ) THEN
                  E(I)=A(I+1,I)
                  E(I+1)=ZERO
                  A(I+1,I)=ZERO
                  I=I+1
               ELSE
                  E(I)=ZERO
               ENDIF
               I=I+1
            END DO
*
```

```
*        Convert PERMUTATIONS
*
         I=1
         DO WHILE ( I .LE. N )
            IF( IPIV(I) .GT. 0 ) THEN
               IP=IPIV(I)
               IF (I .GT. 1) THEN
               DO 22 J= 1,I-1
                 TEMP=A(IP,J)
                 A(IP,J)=A(I,J)
                 A(I,J)=TEMP
 22            CONTINUE
               ENDIF
            ELSE
              IP=-IPIV(I)
              IF (I .GT. 1) THEN
              DO 23 J= 1,I-1
                 TEMP=A(IP,J)
                 A(IP,J)=A(I+1,J)
                 A(I+1,J)=TEMP
 23           CONTINUE
              ENDIF
              I=I+1
          ENDIF
          I=I+1
       END DO
        ELSE
*
*      Revert A (A is lower)
*
*
*        Revert PERMUTATIONS
*
            I=N
            DO WHILE ( I .GE. 1 )
               IF( IPIV(I) .GT. 0 ) THEN
                  IP=IPIV(I)
                  IF (I .GT. 1) THEN
                     DO J= 1,I-1
                        TEMP=A(I,J)
                        A(I,J)=A(IP,J)
                        A(IP,J)=TEMP
                     END DO
                  ENDIF
               ELSE
                  IP=-IPIV(I)
                  I=I-1
                  IF (I .GT. 1) THEN
                     DO J= 1,I-1
                        TEMP=A(I+1,J)
```

```
                      A(I+1,J)=A(IP,J)
                      A(IP,J)=TEMP
                   END DO
                ENDIF
             ENDIF
             I=I-1
          END DO
*
*        Revert VALUE
*
          I=1
          DO WHILE ( I .LE. N-1 )
             IF( IPIV(I) .LT. 0 ) THEN
                A(I+1,I)=E(I)
                I=I+1
             ENDIF
             I=I+1
          END DO
       END IF
    END IF

    RETURN
*
*     End of CSYCONV
*
    END
```

———-

60.10.4 lisp code

— **csyconv.lisp** —

```
(let* ((zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) zero)
           (ignorable zero))
  (defun csyconv (uplo way n a lda ipiv e info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) e a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) way uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (way character way-%data% way-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (e f2cl-lib:complex8 e-%data% e-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((temp #C(0.0 0.0)) (i 0) (ip 0) (j 0) (upper nil) (convert nil))
```

```
(declare (type (f2cl-lib:complex8) temp)
         (type (f2cl-lib:integer4) i ip j)
         (type f2cl-lib:logical upper convert))
(setf info 0)
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(setf convert
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame way "C")
          (declare (ignore var-1))
          (when var-0 (setf way var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((and (not convert)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame way "R")
          (declare (ignore var-1))
          (when var-0 (setf way var-0))
          ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5)))
(cond
 ((/= info 0) (xerbla "CSYCONV" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(cond
 (upper
  (cond
   (convert
    (tagbody
      (setf i n)
      (setf (f2cl-lib:fref e-%data% (1) ((1 *)) e-%offset%) zero)
     label100000
      (if (not (> i 1))
```

```
      (go label100001))
  (cond
   ((< (f2cl-lib:fref ipiv (i) ((1 *))) 0)
    (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
            (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) i)
                           ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1)) ((1 *))
                         e-%offset%)
            zero)
    (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) i)
                         ((1 lda) (1 *)) a-%offset%)
            zero)
    (setf i (f2cl-lib:int-sub i 1)))
   (t (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) zero)))
  (setf i (f2cl-lib:int-sub i 1))
  (go label100000)
 label100001
  (setf i n)
 label100002
  (if (not (>= i 1))
      (go label100003))
  (cond
   ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
    (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
    (cond
     ((< i n)
      (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                     (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf temp
                  (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                 a-%offset%))
          (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                               a-%offset%)
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%))
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%)
                  temp)
        label12)))))
   (t
    (setf ip
            (f2cl-lib:int-sub
             (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                            ipiv-%offset%)))
    (cond
     ((< i n)
      (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                     (f2cl-lib:int-add j 1))
```

```
                    ((> j n) nil)
            (tagbody
              (setf temp
                      (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                   a-%offset%)
                      (f2cl-lib:fref a-%data%
                                     ((f2cl-lib:int-sub i 1) j)
                                     ((1 lda) (1 *)) a-%offset%))
              (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) j)
                                   ((1 lda) (1 *)) a-%offset%)
                      temp)
             label13))))
      (setf i (f2cl-lib:int-sub i 1))))
    (setf i (f2cl-lib:int-sub i 1))
    (go label100002)
   label100003))
  (t
   (tagbody
     (setf i 1)
    label100004
     (if (not (<= i n))
         (go label100005))
     (cond
      ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
       (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
       (cond
        ((< i n)
         (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                          (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf temp
                     (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                    a-%offset%))
             (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                  a-%offset%)
                     (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%))
             (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                     temp)
            label100006)))))
      (t
       (setf ip
               (f2cl-lib:int-sub
                (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                               ipiv-%offset%)))
       (setf i (f2cl-lib:int-add i 1))
```

```
          (cond
           ((< i n)
            (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                           (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf temp
                        (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                       a-%offset%))
                (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                     a-%offset%)
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-sub i 1) j)
                                       ((1 lda) (1 *)) a-%offset%))
                (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) j)
                                     ((1 lda) (1 *)) a-%offset%)
                        temp)
               label100007))))))
      (setf i (f2cl-lib:int-add i 1))
      (go label100004)
     label100005
      (setf i n)
     label100008
      (if (not (> i 1))
          (go label100009))
      (cond
       ((< (f2cl-lib:fref ipiv (i) ((1 *))) 0)
        (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) i)
                             ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))
        (setf i (f2cl-lib:int-sub i 1))))
      (setf i (f2cl-lib:int-sub i 1))
      (go label100008)
     label100009))))
 (t
  (cond
   (convert
    (tagbody
      (setf i 1)
      (setf (f2cl-lib:fref e-%data% (n) ((1 *)) e-%offset%) zero)
     label100010
      (if (not (<= i n))
          (go label100011))
      (cond
       ((and (< i n) (< (f2cl-lib:fref ipiv (i) ((1 *))) 0))
        (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                               ((1 lda) (1 *)) a-%offset%))
        (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                             e-%offset%)
```

```
            zero)
    (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                          ((1 lda) (1 *)) a-%offset%)
            zero)
    (setf i (f2cl-lib:int-add i 1)))
   (t (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) zero)))
  (setf i (f2cl-lib:int-add i 1))
  (go label100010)
 label100011
  (setf i 1)
 label100012
  (if (not (<= i n))
      (go label100013))
  (cond
   ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
    (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
    (cond
     ((> i 1)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j
                        (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf temp
                  (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                 a-%offset%))
          (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                               a-%offset%)
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%))
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%)
                  temp)
         label22)))))
   (t
    (setf ip
            (f2cl-lib:int-sub
             (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                            ipiv-%offset%)))
    (cond
     ((> i 1)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j
                        (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf temp
                  (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                 a-%offset%))
          (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
```

```
                                  a-%offset%)
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add i 1) j)
                                       ((1 lda) (1 *)) a-%offset%))
                (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) j)
                                  ((1 lda) (1 *)) a-%offset%)
                        temp)
                label23))))
          (setf i (f2cl-lib:int-add i 1))))
        (setf i (f2cl-lib:int-add i 1))
        (go label100012)
       label100013))
      (t
       (tagbody
        (setf i n)
       label100014
        (if (not (>= i 1))
            (go label100015))
        (cond
         ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
          (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
          (cond
           ((> i 1)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j
                              (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                           nil)
              (tagbody
                (setf temp
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                        (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                       a-%offset%))
                (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                  a-%offset%)
                        temp)
               label100016)))))
         (t
          (setf ip
                  (f2cl-lib:int-sub
                   (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                                  ipiv-%offset%)))
          (setf i (f2cl-lib:int-sub i 1))
          (cond
           ((> i 1)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j
                              (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
```

```
                                nil)
                      (tagbody
                        (setf temp
                                (f2cl-lib:fref a-%data%
                                               ((f2cl-lib:int-add i 1) j)
                                               ((1 lda) (1 *)) a-%offset%))
                        (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) j)
                                             ((1 lda) (1 *)) a-%offset%)
                                (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                               a-%offset%))
                        (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                             a-%offset%)
                                temp)
                       label100017))))))
                 (setf i (f2cl-lib:int-sub i 1))
                 (go label100014)
                label100015
                 (setf i 1)
                label100018
                 (if (not (<= i (f2cl-lib:int-sub n 1)))
                     (go label100019))
                 (cond
                  ((< (f2cl-lib:fref ipiv (i) ((1 *))) 0)
                   (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                                         ((1 lda) (1 *)) a-%offset%)
                           (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))
                   (setf i (f2cl-lib:int-add i 1))))
                 (setf i (f2cl-lib:int-add i 1))
                 (go label100018)
                label100019)))))
          (go end_label)
         end_label
          (return (values uplo way nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csyconv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4))
```

```
                                        :return-values
                                        '(fortran-to-lisp::uplo
                                          fortran-to-lisp::way nil nil nil
                                          nil nil fortran-to-lisp::info)
                                        :calls 'nil)))
```

———-

60.11 csyequb

60.11.1 Axiom unit tests

— **csyequb.input** —

```
)set break resume
)sys rm -f csyequb.output
)spool csyequb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.11.2 Axiom help page

— **csyequb.help** —

```
======================================================================
csyequb examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csyequb (UPLO, N, A, LDA, S, SCOND, AMAX, WORK, INFO)
           CSYEQUB
```

```
Function/Subroutine Documentation
   subroutine csyequb (character UPLO, integer N, complex, dimension( lda, * )
       A, integer LDA, real, dimension( * ) S, real SCOND, real AMAX, complex,
       dimension( * ) WORK, integer INFO)
       CSYEQUB

Purpose:
   CSYEQUB computes row and column scalings intended to equilibrate a
   symmetric matrix A and reduce its condition number
   (with respect to the two-norm).  S contains the scale factors,
   S(i) = 1/sqrt(A(i,i)), chosen so that the scaled matrix B with
   elements B(i,j) = S(i)*A(i,j)*S(j) has ones on the diagonal.  This
   choice of S puts the condition number of B within a factor N of the
   smallest possible condition number over all possible diagonal scalings.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   The N-by-N symmetric matrix whose scaling
   factors are to be computed.  Only the diagonal elements of A
   are referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 S
   S is REAL array, dimension (N)
   If INFO = 0, S contains the scale factors for A.

 SCOND
   SCOND is REAL
   If INFO = 0, S contains the ratio of the smallest S(i) to
   the largest S(i).  If SCOND >= 0.1 and AMAX is neither too
   large nor too small, it is not worth scaling by S.

 AMAX
```

```
   AMAX is REAL
   Absolute value of largest matrix element.  If AMAX is very
   close to overflow or very close to underflow, the matrix should be scaled.

 WORK
   WORK is COMPLEX array, dimension (3*N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, the i-th diagonal element is nonpositive.
```

———-

60.11.3 fortran code

— **csyequb.f** —

```
*  =====================================================================
      SUBROUTINE CSYEQUB( UPLO, N, A, LDA, S, SCOND, AMAX, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, N
      REAL               AMAX, SCOND
      CHARACTER          UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), WORK( * )
      REAL               S( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E0, ZERO = 0.0E0 )
      INTEGER            MAX_ITER
      PARAMETER          ( MAX_ITER = 100 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, ITER
```

```
      REAL               AVG, STD, TOL, C0, C1, C2, T, U, SI, D, BASE,
     $                   SMIN, SMAX, SMLNUM, BIGNUM, SCALE, SUMSQ
      LOGICAL            UP
      COMPLEX            ZDUM
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      LOGICAL            LSAME
      EXTERNAL           LSAME, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLASSQ
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, INT, LOG, MAX, MIN, REAL, SQRT
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     Statement Function Definitions
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF ( .NOT. ( LSAME( UPLO, 'U' ) .OR. LSAME( UPLO, 'L' ) ) ) THEN
        INFO = -1
      ELSE IF ( N .LT. 0 ) THEN
        INFO = -2
      ELSE IF ( LDA .LT. MAX( 1, N ) ) THEN
        INFO = -4
      END IF
      IF ( INFO .NE. 0 ) THEN
        CALL XERBLA( 'CSYEQUB', -INFO )
        RETURN
      END IF

      UP = LSAME( UPLO, 'U' )
      AMAX = ZERO
*
*     Quick return if possible.
*
      IF ( N .EQ. 0 ) THEN
        SCOND = ONE
        RETURN
      END IF

      DO I = 1, N
```

```
        S( I ) = ZERO
      END DO

      AMAX = ZERO
      IF ( UP ) THEN
         DO J = 1, N
            DO I = 1, J-1
               S( I ) = MAX( S( I ), CABS1( A( I, J ) ) )
               S( J ) = MAX( S( J ), CABS1( A( I, J ) ) )
               AMAX = MAX( AMAX, CABS1( A( I, J ) ) )
            END DO
            S( J ) = MAX( S( J ), CABS1( A( J, J) ) )
            AMAX = MAX( AMAX, CABS1( A( J, J ) ) )
         END DO
      ELSE
         DO J = 1, N
            S( J ) = MAX( S( J ), CABS1( A( J, J ) ) )
            AMAX = MAX( AMAX, CABS1( A( J, J ) ) )
            DO I = J+1, N
               S( I ) = MAX( S( I ), CABS1( A( I, J ) ) )
               S( J ) = MAX( S( J ), CABS1 (A( I, J ) ) )
               AMAX = MAX( AMAX, CABS1( A( I, J ) ) )
            END DO
         END DO
      END IF
      DO J = 1, N
         S( J ) = 1.0 / S( J )
      END DO

      TOL = ONE / SQRT( 2.0E0 * N )

      DO ITER = 1, MAX_ITER
         SCALE = 0.0
         SUMSQ = 0.0
*       beta = |A|s
        DO I = 1, N
           WORK( I ) = ZERO
        END DO
        IF ( UP ) THEN
           DO J = 1, N
              DO I = 1, J-1
                 T = CABS1( A( I, J ) )
                 WORK( I ) = WORK( I ) + CABS1( A( I, J ) ) * S( J )
                 WORK( J ) = WORK( J ) + CABS1( A( I, J ) ) * S( I )
              END DO
              WORK( J ) = WORK( J ) + CABS1( A( J, J ) ) * S( J )
           END DO
        ELSE
           DO J = 1, N
              WORK( J ) = WORK( J ) + CABS1( A( J, J ) ) * S( J )
```

```
            DO I = J+1, N
               T = CABS1( A( I, J ) )
               WORK( I ) = WORK( I ) + CABS1( A( I, J ) ) * S( J )
               WORK( J ) = WORK( J ) + CABS1( A( I, J ) ) * S( I )
            END DO
         END DO
      END IF

*     avg = s^T beta / n
      AVG = 0.0
      DO I = 1, N
        AVG = AVG + S( I )*WORK( I )
      END DO
      AVG = AVG / N

      STD = 0.0
      DO I = N+1, 2*N
         WORK( I ) = S( I-N ) * WORK( I-N ) - AVG
      END DO
      CALL CLASSQ( N, WORK( N+1 ), 1, SCALE, SUMSQ )
      STD = SCALE * SQRT( SUMSQ / N )

      IF ( STD .LT. TOL * AVG ) GOTO 999

      DO I = 1, N
        T = CABS1( A( I, I ) )
        SI = S( I )
        C2 = ( N-1 ) * T
        C1 = ( N-2 ) * ( WORK( I ) - T*SI )
        C0 = -(T*SI)*SI + 2*WORK( I )*SI - N*AVG
        D = C1*C1 - 4*C0*C2

        IF ( D .LE. 0 ) THEN
          INFO = -1
          RETURN
        END IF
        SI = -2*C0 / ( C1 + SQRT( D ) )

        D = SI - S( I )
        U = ZERO
        IF ( UP ) THEN
          DO J = 1, I
            T = CABS1( A( J, I ) )
            U = U + S( J )*T
            WORK( J ) = WORK( J ) + D*T
          END DO
          DO J = I+1,N
            T = CABS1( A( I, J ) )
            U = U + S( J )*T
            WORK( J ) = WORK( J ) + D*T
```

```
          END DO
        ELSE
          DO J = 1, I
            T = CABS1( A( I, J ) )
            U = U + S( J )*T
            WORK( J ) = WORK( J ) + D*T
          END DO
          DO J = I+1,N
            T = CABS1( A( J, I ) )
            U = U + S( J )*T
            WORK( J ) = WORK( J ) + D*T
          END DO
        END IF
        AVG = AVG + ( U + WORK( I ) ) * D / N
        S( I ) = SI
      END DO
    END DO

 999  CONTINUE

      SMLNUM = SLAMCH( 'SAFEMIN' )
      BIGNUM = ONE / SMLNUM
      SMIN = BIGNUM
      SMAX = ZERO
      T = ONE / SQRT( AVG )
      BASE = SLAMCH( 'B' )
      U = ONE / LOG( BASE )
      DO I = 1, N
        S( I ) = BASE ** INT( U * LOG( S( I ) * T ) )
        SMIN = MIN( SMIN, S( I ) )
        SMAX = MAX( SMAX, S( I ) )
      END DO
      SCOND = MAX( SMIN, SMLNUM ) / MIN( SMAX, BIGNUM )
*
      END
```

———-

60.11.4 lisp code

— csyequb.lisp —

```
(let* ((one 1.0) (zero 0.0) (max_iter 100))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (type (f2cl-lib:integer4 100 100) max_iter)
           (ignorable one zero max_iter))
  (defun csyequb (uplo n a lda s scond amax work info)
```

```
(declare (type (single-float) amax scond)
         (type (array single-float (*)) s)
         (type (array f2cl-lib:complex8 (*)) work a)
         (type (f2cl-lib:integer4) info lda n)
         (type (string *) uplo))
(f2cl-lib:with-multi-array-data
    ((uplo character uplo-%data% uplo-%offset%)
     (a f2cl-lib:complex8 a-%data% a-%offset%)
     (work f2cl-lib:complex8 work-%data% work-%offset%)
     (s single-float s-%data% s-%offset%))
  (labels ((cabs1 (zdum)
             (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
    (declare
     (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
      cabs1))
    (prog ((zdum #C(0.0 0.0)) (up nil) (avg 0.0) (std 0.0) (tol 0.0)
           (c0 0.0) (c1 0.0) (c2 0.0) (t$ 0.0) (u 0.0) (si 0.0) (d 0.0)
           (base 0.0) (smin 0.0) (smax 0.0) (smlnum 0.0) (bignum 0.0)
           (scale 0.0) (sumsq 0.0) (i 0) (j 0) (iter 0))
      (declare (type (f2cl-lib:complex8) zdum)
               (type f2cl-lib:logical up)
               (type (single-float) avg std tol c0 c1 c2 t$ u si d base
                smin smax smlnum bignum scale sumsq)
               (type (f2cl-lib:integer4) i j iter))
      (setf info 0)
      (cond
       ((not
         (or
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -4)))
      (cond
       ((/= info 0) (xerbla "CSYEQUB" (f2cl-lib:int-sub info))
        (go end_label)))
      (setf up
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
```

```
          ret-val))
(setf amax zero)
(cond ((= n 0) (setf scond one) (go end_label)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%) zero)
   label100000))
(setf amax zero)
(cond
 (up
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                  (max
                   (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                   (cabs1
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))))
          (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                  (max
                   (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                   (cabs1
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))))
          (setf amax
                  (max amax
                       (cabs1
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))))
        label100002))
      (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
              (max (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                   (cabs1
                    (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                   a-%offset%))))
      (setf amax
              (max amax
                   (cabs1
                    (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                   a-%offset%))))
    label100001)))
 (t
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
```

```
      (tagbody
        (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                (max (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                     (cabs1
                      (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%))))
        (setf amax
                (max amax
                     (cabs1
                      (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%))))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                    (max
                     (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                     (cabs1
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))))
            (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                    (max
                     (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                     (cabs1
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))))
            (setf amax
                    (max amax
                         (cabs1
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%))))
          label100004))
      label100003))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
            (/ 1.0 (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
   label100005))
(setf tol (/ one (f2cl-lib:fsqrt (* 2.0 n))))
(f2cl-lib:fdo (iter 1 (f2cl-lib:int-add iter 1))
              ((> iter max_iter) nil)
  (tagbody
    (setf scale 0.0)
    (setf sumsq 0.0)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (coerce zero 'f2cl-lib:complex8))
```

```
   label100007))
 (cond
  (up
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i
                         (f2cl-lib:int-add j
                                           (f2cl-lib:int-sub 1)))
                      nil)
         (tagbody
           (setf t$
                   (cabs1
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%)))
           (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                   (+
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)
                    (*
                     (cabs1
                      (f2cl-lib:fref a-%data% (i j)
                                     ((1 lda) (1 *)) a-%offset%))
                     (f2cl-lib:fref s-%data% (j) ((1 *))
                                    s-%offset%))))
           (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                                work-%offset%)
                   (+
                    (f2cl-lib:fref work-%data% (j) ((1 *))
                                   work-%offset%)
                    (*
                     (cabs1
                      (f2cl-lib:fref a-%data% (i j)
                                     ((1 lda) (1 *)) a-%offset%))
                     (f2cl-lib:fref s-%data% (i) ((1 *))
                                    s-%offset%))))
         label100009))
       (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
               (+
                (f2cl-lib:fref work-%data% (j) ((1 *))
                               work-%offset%)
                (*
                 (cabs1
                  (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                 a-%offset%))
                 (f2cl-lib:fref s-%data% (j) ((1 *))
                                s-%offset%))))
     label100008)))
```

```
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
             (+
              (f2cl-lib:fref work-%data% (j) ((1 *))
                             work-%offset%)
              (*
               (cabs1
                (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                               a-%offset%))
               (f2cl-lib:fref s-%data% (j) ((1 *))
                              s-%offset%))))
     (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                    (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf t$
                 (cabs1
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%)))
         (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                              work-%offset%)
                 (+
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)
                  (*
                   (cabs1
                    (f2cl-lib:fref a-%data% (i j)
                                   ((1 lda) (1 *)) a-%offset%))
                   (f2cl-lib:fref s-%data% (j) ((1 *))
                                  s-%offset%))))
         (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                              work-%offset%)
                 (+
                  (f2cl-lib:fref work-%data% (j) ((1 *))
                                 work-%offset%)
                  (*
                   (cabs1
                    (f2cl-lib:fref a-%data% (i j)
                                   ((1 lda) (1 *)) a-%offset%))
                   (f2cl-lib:fref s-%data% (i) ((1 *))
                                  s-%offset%))))
       label100011))
   label100010))))
(setf avg 0.0)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
```

```
    (setf avg
            (coerce
             (realpart
              (+ avg
                 (*
                  (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%))))
             'single-float))
   label100012))
(setf avg (/ avg n))
(setf std 0.0)
(f2cl-lib:fdo (i (f2cl-lib:int-add n 1) (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-mul 2 n)) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (-
             (*
              (f2cl-lib:fref s-%data% ((f2cl-lib:int-sub i n))
                             ((1 *)) s-%offset%)
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub i n))
                             ((1 *)) work-%offset%))
             avg))
   label100013))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (classq n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           ((+ n 1)) ((1 *)) work-%offset%)
     1 scale sumsq)
  (declare (ignore var-0 var-1 var-2))
  (setf scale var-3)
  (setf sumsq var-4))
(setf std (* scale (f2cl-lib:fsqrt (/ sumsq n))))
(if (< std (* tol avg))
    (go label999))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf t$
            (cabs1
             (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                            a-%offset%)))
    (setf si (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%))
    (setf c2 (* (f2cl-lib:int-sub n 1) t$))
    (setf c1
            (coerce
             (realpart
              (* (f2cl-lib:int-sub n 2)
                 (-
                  (f2cl-lib:fref work-%data% (i) ((1 *))
```

```
                                work-%offset%)
                (* t$ si))))
           'single-float))
(setf c0
        (coerce
         (realpart
          (-
           (+ (* (- (* t$ si)) si)
              (* 2
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 si))
           (* n avg)))
         'single-float))
(setf d (+ (* c1 c1) (* -4 c0 c2)))
(cond ((<= d 0) (setf info -1) (go end_label)))
(setf si (/ (* -2 c0) (+ c1 (f2cl-lib:fsqrt d))))
(setf d
        (- si
           (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
(setf u zero)
(cond
 (up
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j i) nil)
    (tagbody
      (setf t$
              (cabs1
               (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                              a-%offset%)))
      (setf u
              (+ u
                 (*
                  (f2cl-lib:fref s-%data% (j) ((1 *))
                                 s-%offset%)
                  t$)))
      (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                           work-%offset%)
              (+
               (f2cl-lib:fref work-%data% (j) ((1 *))
                              work-%offset%)
               (* d t$)))
     label100015))
  (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf t$
              (cabs1
               (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
```

```
                                a-%offset%)))
        (setf u
                (+ u
                   (*
                    (f2cl-lib:fref s-%data% (j) ((1 *))
                                   s-%offset%)
                    t$)))
        (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                             work-%offset%)
                (+
                 (f2cl-lib:fref work-%data% (j) ((1 *))
                                work-%offset%)
                 (* d t$)))
      label100016)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j i) nil)
       (tagbody
        (setf t$
                (cabs1
                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)))
        (setf u
                (+ u
                   (*
                    (f2cl-lib:fref s-%data% (j) ((1 *))
                                   s-%offset%)
                    t$)))
        (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                             work-%offset%)
                (+
                 (f2cl-lib:fref work-%data% (j) ((1 *))
                                work-%offset%)
                 (* d t$)))
      label100017))
     (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                    (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
        (setf t$
                (cabs1
                 (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                a-%offset%)))
        (setf u
                (+ u
                   (*
                    (f2cl-lib:fref s-%data% (j) ((1 *))
                                   s-%offset%)
                    t$)))
        (setf (f2cl-lib:fref work-%data% (j) ((1 *))
```

```
                                        work-%offset%)
                                 (+
                                  (f2cl-lib:fref work-%data% (j) ((1 *))
                                                 work-%offset%)
                                  (* d t$)))
                       label100018))))
                 (setf avg
                         (coerce
                          (realpart
                           (+ avg
                              (/
                               (*
                                (+ u
                                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                                  work-%offset%))
                                d)
                               n)))
                          'single-float))
                 (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%) si)
                label100014))
            label100006))
        label999
         (setf smlnum (slamch "SAFEMIN"))
         (setf bignum (/ one smlnum))
         (setf smin bignum)
         (setf smax zero)
         (setf t$ (/ one (f2cl-lib:fsqrt avg)))
         (setf base (slamch "B"))
         (setf u (/ one (f2cl-lib:flog base)))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                     (expt base
                           (f2cl-lib:int
                            (* u
                               (f2cl-lib:flog
                                (*
                                 (f2cl-lib:fref s-%data% (i) ((1 *))
                                                s-%offset%)
                                 t$))))))
             (setf smin
                     (min smin
                          (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
             (setf smax
                     (max smax
                          (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
            label100019))
         (setf scond (/ (max smin smlnum) (min smax bignum)))
        end_label
```

```
          (return (values uplo nil nil nil nil scond amax nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csyequb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil nil
                                              nil fortran-to-lisp::scond
                                              fortran-to-lisp::amax nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::classq))))
```

———-

60.12 csyrfs

60.12.1 Axiom unit tests

— csyrfs.input —

```
)set break resume
)sys rm -f csyrfs.output
)spool csyrfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.12.2 Axiom help page

— csyrfs.help —

```
=====================================================================
csyrfs examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csyrfs (UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB, X,
           LDX, FERR, BERR, WORK, RWORK, INFO)
           CSYRFS

Function/Subroutine Documentation
   subroutine csyrfs (character UPLO, integer N, integer NRHS, complex,
       dimension( lda, * ) A, integer LDA, complex, dimension( ldaf, * ) AF,
       integer LDAF, integer, dimension( * ) IPIV, complex, dimension( ldb, *
       ) B, integer LDB, complex, dimension( ldx, * ) X, integer LDX, real,
       dimension( * ) FERR, real, dimension( * ) BERR, complex, dimension( * )
       WORK, real, dimension( * ) RWORK, integer INFO)
       CSYRFS

Purpose:
   CSYRFS improves the computed solution to a system of linear
   equations when the coefficient matrix is symmetric indefinite, and
   provides error bounds and backward error estimates for the solution.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 A
```

```
  A is COMPLEX array, dimension (LDA,N)
  The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
  upper triangular part of A contains the upper triangular part
  of the matrix A, and the strictly lower triangular part of A
  is not referenced.  If UPLO = 'L', the leading N-by-N lower
  triangular part of A contains the lower triangular part of
  the matrix A, and the strictly upper triangular part of A is
  not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX array, dimension (LDAF,N)
  The factored form of the matrix A.  AF contains the block
  diagonal matrix D and the multipliers used to obtain the
  factor U or L from the factorization A = U*D*U**T or
  A = L*D*L**T as computed by CSYTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by CSYTRF.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by CSYTRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

FERR
  FERR is REAL array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
```

```
   If XTRUE is the true solution corresponding to X(j), FERR(j)
   is an estimated upper bound for the magnitude of the largest
   element in (X(j) - XTRUE) divided by the magnitude of the
   largest element in X(j).  The estimate is as reliable as
   the estimate for RCOND, and is almost always a slight
   overestimate of the true error.

 BERR
   BERR is REAL array, dimension (NRHS)
   The componentwise relative backward error of each solution
   vector X(j) (i.e., the smallest relative change in
   any element of A or B that makes X(j) an exact solution).

 WORK
   WORK is COMPLEX array, dimension (2*N)

 RWORK
   RWORK is REAL array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

   ITMAX is the maximum number of steps of iterative refinement.
```

60.12.3 fortran code

— csyrfs.f —

```
*  =====================================================================
      SUBROUTINE CSYRFS( UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB,
     $                   X, LDX, FERR, BERR, WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
*     ..
```

```
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               BERR( * ), FERR( * ), RWORK( * )
      COMPLEX            A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
      COMPLEX            ONE
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ) )
      REAL               TWO
      PARAMETER          ( TWO = 2.0E+0 )
      REAL               THREE
      PARAMETER          ( THREE = 3.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            COUNT, I, J, K, KASE, NZ
      REAL               EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN, XK
      COMPLEX            ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           CAXPY, CCOPY, CLACN2, CSYMV, CSYTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, MAX, REAL
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH
      EXTERNAL           LSAME, SLAMCH
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
```

```
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -10
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -12
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CSYRFS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = N + 1
      EPS = SLAMCH( 'Epsilon' )
      SAFMIN = SLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 140 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
```

```
*
*        Compute residual R = B - A * X
*
         CALL CCOPY( N, B( 1, J ), 1, WORK, 1 )
         CALL CSYMV( UPLO, N, -ONE, A, LDA, X( 1, J ), 1, ONE, WORK, 1 )
*
*        Compute componentwise relative backward error from formula
*
*        max(i) ( abs(R(i)) / ( abs(A)*abs(X) + abs(B) )(i) )
*
*        where abs(Z) is the componentwise absolute value of the matrix
*        or vector Z.  If the i-th component of the denominator is less
*        than SAFE2, then SAFE1 is added to the i-th components of the
*        numerator and denominator before dividing.
*
         DO 30 I = 1, N
            RWORK( I ) = CABS1( B( I, J ) )
   30    CONTINUE
*
*        Compute abs(A)*abs(X) + abs(B).
*
         IF( UPPER ) THEN
            DO 50 K = 1, N
               S = ZERO
               XK = CABS1( X( K, J ) )
               DO 40 I = 1, K - 1
                  RWORK( I ) = RWORK( I ) + CABS1( A( I, K ) )*XK
                  S = S + CABS1( A( I, K ) )*CABS1( X( I, J ) )
   40          CONTINUE
               RWORK( K ) = RWORK( K ) + CABS1( A( K, K ) )*XK + S
   50       CONTINUE
         ELSE
            DO 70 K = 1, N
               S = ZERO
               XK = CABS1( X( K, J ) )
               RWORK( K ) = RWORK( K ) + CABS1( A( K, K ) )*XK
               DO 60 I = K + 1, N
                  RWORK( I ) = RWORK( I ) + CABS1( A( I, K ) )*XK
                  S = S + CABS1( A( I, K ) )*CABS1( X( I, J ) )
   60          CONTINUE
               RWORK( K ) = RWORK( K ) + S
   70       CONTINUE
         END IF
         S = ZERO
         DO 80 I = 1, N
            IF( RWORK( I ).GT.SAFE2 ) THEN
               S = MAX( S, CABS1( WORK( I ) ) / RWORK( I ) )
            ELSE
               S = MAX( S, ( CABS1( WORK( I ) )+SAFE1 ) /
     $             ( RWORK( I )+SAFE1 ) )
```

```
            END IF
   80    CONTINUE
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
*           1) The residual BERR(J) is larger than machine epsilon, and
*           2) BERR(J) decreased by at least a factor of 2 during the
*              last iteration, and
*           3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
*
*           Update solution and try again.
*
            CALL CSYTRS( UPLO, N, 1, AF, LDAF, IPIV, WORK, N, INFO )
            CALL CAXPY( N, ONE, WORK, 1, X( 1, J ), 1 )
            LSTRES = BERR( J )
            COUNT = COUNT + 1
            GO TO 20
         END IF
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(A))*
*           ( abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
*          inv(A) is the inverse of A
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(A)*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(A)*abs(X) + abs(B) is less than SAFE2.
*
*        Use CLACN2 to estimate the infinity-norm of the matrix
*           inv(A) * diag(W),
*        where W = abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) )))
*
         DO 90 I = 1, N
            IF( RWORK( I ).GT.SAFE2 ) THEN
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I )
            ELSE
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I ) +
     $                      SAFE1
```

```
            END IF
   90    CONTINUE
*
         KASE = 0
  100    CONTINUE
         CALL CLACN2( N, WORK( N+1 ), WORK, FERR( J ), KASE, ISAVE )
         IF( KASE.NE.0 ) THEN
            IF( KASE.EQ.1 ) THEN
*
*              Multiply by diag(W)*inv(A**T).
*
               CALL CSYTRS( UPLO, N, 1, AF, LDAF, IPIV, WORK, N, INFO )
               DO 110 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
  110          CONTINUE
            ELSE IF( KASE.EQ.2 ) THEN
*
*              Multiply by inv(A)*diag(W).
*
               DO 120 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
  120          CONTINUE
               CALL CSYTRS( UPLO, N, 1, AF, LDAF, IPIV, WORK, N, INFO )
            END IF
            GO TO 100
         END IF
*
*        Normalize error.
*
         LSTRES = ZERO
         DO 130 I = 1, N
            LSTRES = MAX( LSTRES, CABS1( X( I, J ) ) )
  130    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  140 CONTINUE
*
      RETURN
*
*     End of CSYRFS
*
      END
```

———-

60.12.4 lisp code

— csyrfs.lisp —

```
(let* ((itmax 5)
       (zero 0.0)
       (one (f2cl-lib:cmplx 1.0 0.0))
       (two 2.0)
       (three 3.0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (single-float 0.0 0.0) zero)
           (type (f2cl-lib:complex8) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 3.0 3.0) three)
           (ignorable itmax zero one two three))
  (defun csyrfs
         (uplo n nrhs a lda af ldaf ipiv b ldb$ x ldx ferr berr work rwork
          info)
    (declare (type (array single-float (*)) rwork berr ferr)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) work x b af a)
             (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (af f2cl-lib:complex8 af-%data% af-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (ferr single-float ferr-%data% ferr-%offset%)
         (berr single-float berr-%data% berr-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
          cabs1))
        (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
               (zdum #C(0.0 0.0)) (eps 0.0) (lstres 0.0) (s 0.0) (safe1 0.0)
               (safe2 0.0) (safmin 0.0) (xk 0.0) (count$ 0) (i 0) (j 0) (k 0)
               (kase 0) (nz 0) (upper nil))
          (declare (type (array f2cl-lib:integer4 (3)) isave)
                   (type (f2cl-lib:complex8) zdum)
                   (type (single-float) eps lstres s safe1 safe2 safmin xk)
                   (type (f2cl-lib:integer4) count$ i j k kase nz)
                   (type f2cl-lib:logical upper))
          (setf info 0)
```

```
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -10))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -12)))
(cond
 ((/= info 0) (xerbla "CSYRFS" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
              zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%)
              zero)
     label10))
  (go end_label)))
(setf nz (f2cl-lib:int-add n 1))
(setf eps (slamch "Epsilon"))
(setf safmin (slamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
    (ccopy n
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 j)
                           ((1 ldb$) (1 *)) b-%offset%)
     1 work 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (csymv uplo n (- one) a lda
     (f2cl-lib:array-slice x-%data% f2cl-lib:complex8 (1 j)
                           ((1 ldx) (1 *)) x-%offset%)
     1 one work 1)
  (declare
   (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
    var-9))
  (setf uplo var-0))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
            (cabs1
             (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                            b-%offset%)))
   label30))
(cond
 (upper
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (setf s zero)
      (setf xk
              (cabs1
               (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                              x-%offset%)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (f2cl-lib:int-add k
                                          (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                               rwork-%offset%)
                (+
                 (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                rwork-%offset%)
                 (*
                  (cabs1
                   (f2cl-lib:fref a-%data% (i k)
                                  ((1 lda) (1 *)) a-%offset%))
                  xk)))
```

```
            (setf s
                    (+ s
                       (*
                        (cabs1
                         (f2cl-lib:fref a-%data% (i k)
                                        ((1 lda) (1 *))
                                        a-%offset%))
                        (cabs1
                         (f2cl-lib:fref x-%data% (i j)
                                        ((1 ldx) (1 *))
                                        x-%offset%)))))
          label40))
      (setf (f2cl-lib:fref rwork-%data% (k) ((1 *))
                           rwork-%offset%)
            (+
             (f2cl-lib:fref rwork-%data% (k) ((1 *))
                            rwork-%offset%)
             (*
              (cabs1
               (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                              a-%offset%))
              xk)
             s))
    label50)))
(t
 (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
               ((> k n) nil)
   (tagbody
     (setf s zero)
     (setf xk
            (cabs1
             (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                            x-%offset%)))
     (setf (f2cl-lib:fref rwork-%data% (k) ((1 *))
                          rwork-%offset%)
            (+
             (f2cl-lib:fref rwork-%data% (k) ((1 *))
                            rwork-%offset%)
             (*
              (cabs1
               (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                              a-%offset%))
              xk)))
     (f2cl-lib:fdo (i (f2cl-lib:int-add k 1)
                    (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                              rwork-%offset%)
               (+
```

```
                        (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                       rwork-%offset%)
                        (*
                         (cabs1
                          (f2cl-lib:fref a-%data% (i k)
                                         ((1 lda) (1 *)) a-%offset%))
                         xk)))
                (setf s
                       (+ s
                          (*
                           (cabs1
                            (f2cl-lib:fref a-%data% (i k)
                                           ((1 lda) (1 *))
                                           a-%offset%))
                           (cabs1
                            (f2cl-lib:fref x-%data% (i j)
                                           ((1 ldx) (1 *))
                                           x-%offset%)))))
            label60))
        (setf (f2cl-lib:fref rwork-%data% (k) ((1 *))
                             rwork-%offset%)
              (+
               (f2cl-lib:fref rwork-%data% (k) ((1 *))
                              rwork-%offset%)
               s))
      label70))))
(setf s zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
      (setf s
              (max s
                   (/
                    (cabs1
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%))
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%)))))
     (t
      (setf s
              (max s
                   (/
                    (+
                     (cabs1
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%))
                     safe1)
                    (+
```

```
                        (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                       rwork-%offset%)
                        safe1))))))
   label80))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (csytrs uplo n 1 af ldaf ipiv work n info)
    (declare (ignore var-2 var-3 var-5 var-6))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf ldaf var-4))
    (when var-7 (setf n var-7))
    (when var-8 (setf info var-8)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (caxpy n one work 1
       (f2cl-lib:array-slice x-%data% f2cl-lib:complex8 (1 j)
                             ((1 ldx) (1 *)) x-%offset%)
       1)
    (declare (ignore var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))
    (when var-1 (setf one var-1)))
  (setf lstres
          (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
  (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
      (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                           rwork-%offset%)
              (+
               (cabs1
                (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%))
               (* nz eps
                  (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                 rwork-%offset%)))))
     (t
      (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                           rwork-%offset%)
              (+
               (cabs1
                (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%))
```

```
                    (* nz eps
                       (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                      rwork-%offset%))
                    safe1))))
     label90))
  (setf kase 0)
 label100
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (clacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             ((+ n 1)) ((1 *)) work-%offset%)
       work (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
       kase isave)
    (declare (ignore var-1 var-2 var-5))
    (setf n var-0)
    (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
            var-3)
    (setf kase var-4))
  (cond
   ((/= kase 0)
    (cond
     ((= kase 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (csytrs uplo n 1 af ldaf ipiv work n info)
        (declare (ignore var-2 var-3 var-5 var-6))
        (when var-0 (setf uplo var-0))
        (when var-1 (setf n var-1))
        (when var-4 (setf ldaf var-4))
        (when var-7 (setf n var-7))
        (when var-8 (setf info var-8)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                (*
                 (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                rwork-%offset%)
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)))
         label110)))
     ((= kase 2)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                (*
                 (f2cl-lib:fref rwork-%data% (i) ((1 *))
```

```
                                          rwork-%offset%)
                              (f2cl-lib:fref work-%data% (i) ((1 *))
                                          work-%offset%)))
                     label120))
                  (multiple-value-bind
                      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                      (csytrs uplo n 1 af ldaf ipiv work n info)
                    (declare (ignore var-2 var-3 var-5 var-6))
                    (when var-0 (setf uplo var-0))
                    (when var-1 (setf n var-1))
                    (when var-4 (setf ldaf var-4))
                    (when var-7 (setf n var-7))
                    (when var-8 (setf info var-8)))))
                (go label100)))
              (setf lstres zero)
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i n) nil)
                (tagbody
                  (setf lstres
                          (max lstres
                               (cabs1
                                (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                               x-%offset%))))
                 label130))
              (if (/= lstres zero)
                  (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                          (/
                           (f2cl-lib:fref ferr-%data% (j) ((1 *))
                                          ferr-%offset%)
                           lstres)))
            label140))
          (go end_label)
         end_label
          (return
           (values uplo n nil nil nil nil ldaf nil nil nil nil nil nil nil nil
                   nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csyrfs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
```

```
  (*))
 (fortran-to-lisp::integer4)
 (array fortran-to-lisp::integer4
  (*))
 (array fortran-to-lisp::complex8
  (*))
 (fortran-to-lisp::integer4)
 (array fortran-to-lisp::complex8
  (*))
 (fortran-to-lisp::integer4)
 (array single-float (*))
 (array single-float (*))
 (array fortran-to-lisp::complex8
  (*))
 (array single-float (*))
 (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::uplo
  fortran-to-lisp::n nil nil nil
  nil fortran-to-lisp::ldaf nil nil
  nil nil nil nil nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::clacn2
  fortran-to-lisp::csymv))))
```

———-

60.13 csyrfsx

60.13.1 Axiom unit tests

— **csyrfsx.input** —

```
)set break resume
)sys rm -f csyrfsx.output
)spool csyrfsx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.13.2 Axiom help page [Demm05]

— csyrfsx.help —

```
==================================================================
csyrfsx examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csyrfsx (UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF, IPIV, S, B,
           LDB, X, LDX, RCOND, BERR, N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP,
           NPARAMS, PARAMS, WORK, RWORK, INFO)
           CSYRFSX

Function/Subroutine Documentation
   subroutine csyrfsx (character UPLO, character EQUED, integer N, integer
       NRHS, complex, dimension( lda, * ) A, integer LDA, complex, dimension(
       ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV, real,
       dimension( * ) S, complex, dimension( ldb, * ) B, integer LDB, complex,
       dimension( ldx, * ) X, integer LDX, real RCOND, real, dimension( * )
       BERR, integer N_ERR_BNDS, real, dimension( nrhs, * ) ERR_BNDS_NORM,
       real, dimension( nrhs, * ) ERR_BNDS_COMP, integer NPARAMS, real,
       dimension( * ) PARAMS, complex, dimension( * ) WORK, real, dimension( *
       ) RWORK, integer INFO)
       CSYRFSX

Purpose:
   CSYRFSX improves the computed solution to a system of linear
   equations when the coefficient matrix is symmetric indefinite, and
   provides error bounds and backward error estimates for the
   solution.  In addition to normwise error bound, the code provides
   maximum componentwise error bound if possible.  See comments for
   ERR_BNDS_NORM and ERR_BNDS_COMP for details of the error bounds.

   The original system of linear equations may have been equilibrated
   before calling this routine, as described by arguments EQUED and S
   below. In this case, the solution and error bounds returned are
   for the original unequilibrated system.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.
```

```
Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done to A
   before calling this routine. This is needed to compute
   the solution and error bounds correctly.
   = 'N':  No equilibration
   = 'Y':  Both row and column equilibration, i.e., A has been
           replaced by diag(S) * A * diag(S).
           The right hand side B has been changed accordingly.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
   upper triangular part of A contains the upper triangular
   part of the matrix A, and the strictly lower triangular
   part of A is not referenced.  If UPLO = 'L', the leading
   N-by-N lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is COMPLEX array, dimension (LDAF,N)
   The factored form of the matrix A.  AF contains the block
   diagonal matrix D and the multipliers used to obtain the
   factor U or L from the factorization A = U*D*U**T or A =
   L*D*L**T as computed by SSYTRF.

 LDAF
   LDAF is INTEGER
```

```
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by SSYTRF.

S
  S is REAL array, dimension (N)
  The scale factors for A.  If EQUED = 'Y', A is multiplied on
  the left and right by diag(S).  S is an input argument if FACT =
  'F'; otherwise, S is an output argument.  If FACT = 'F' and EQUED
  = 'Y', each element of S must be positive.  If S is output, each
  element of S is a power of the radix. If S is input, each element
  of S should be a power of the radix to ensure a reliable solution
  and error estimates. Scaling by powers of the radix does not cause
  rounding errors unless the result underflows or overflows.
  Rounding errors during scaling lead to refining with a matrix that
  is not equivalent to the input matrix, producing error estimates
  that may not be reliable.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by SGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

BERR
  BERR is REAL array, dimension (NRHS)
  Componentwise relative backward error.  This is the
```

```
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
          max_j (abs(XTRUE(j,i) - X(j,i)))
           ------------------------------
                 max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.
```

```
ERR_BNDS_COMP
  ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                 abs(XTRUE(j,i) - X(j,i))
            max_j ----------------------
                        abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
    reciprocal condition number is less than the threshold
    sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
    almost certainly within a factor of 10 of the true error
    so long as the next entry is greater than the threshold
    sqrt(n) * slamch('Epsilon'). This error bound should only
    be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
    reciprocal condition number.  Compared with the threshold
    sqrt(n) * slamch('Epsilon') to determine if the error
    estimate is "guaranteed". These reciprocal condition
    numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
    appropriately scaled matrix Z.
    Let Z = S*(A*diag(x)), where x is the solution for the
    current right-hand side and S scales each row of
    A*diag(x) by a power of the radix so all absolute row
    sums of Z are approximately 1.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.
```

```
PARAMS
  PARAMS is REAL array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
  (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is COMPLEX array, dimension (2*N)

RWORK
  RWORK is REAL array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is
  guaranteed.
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
     has been completed, but the factor U is exactly singular, so
     the solution and error bounds could not be computed. RCOND = 0
     is returned.
  = N+J: The solution corresponding to the Jth right-hand side is
     not guaranteed. The solutions corresponding to other right-
     hand sides K with K > J may not be guaranteed as well, but
```

```
      only the first such right-hand side is reported. If a small
      componentwise error is not requested (PARAMS(3) = 0.0) then
      the Jth right-hand side is the first with a normwise error
      bound that is not guaranteed (the smallest J such
      that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
      the Jth right-hand side is the first with either a normwise or
      componentwise error bound that is not guaranteed (the smallest
      J such that either ERR_BNDS_NORM(J,1) = 0.0 or
      ERR_BNDS_COMP(J,1) = 0.0). See the definition of
      ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
      about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

———-

60.13.3 fortran code

— **csyrfsx.f** —

```
*  =====================================================================
      SUBROUTINE CSYRFSX( UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $                    S, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS,
     $                    ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $                    WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO, EQUED
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX, * ), WORK( * )
      REAL               S( * ), PARAMS( * ), BERR( * ), RWORK( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
```

```
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               ITREF_DEFAULT, ITHRESH_DEFAULT,
     $                   COMPONENTWISE_DEFAULT
      REAL               RTHRESH_DEFAULT, DZTHRESH_DEFAULT
      PARAMETER          ( ITREF_DEFAULT = 1.0 )
      PARAMETER          ( ITHRESH_DEFAULT = 10.0 )
      PARAMETER          ( COMPONENTWISE_DEFAULT = 1.0 )
      PARAMETER          ( RTHRESH_DEFAULT = 0.5 )
      PARAMETER          ( DZTHRESH_DEFAULT = 0.25 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. Local Scalars ..
      CHARACTER(1)       NORM
      LOGICAL            RCEQU
      INTEGER            J, PREC_TYPE, REF_TYPE
      INTEGER            N_NORMS
      REAL               ANORM, RCOND_TMP
      REAL               ILLRCOND_THRESH, ERR_LBND, CWISE_WRONG
      LOGICAL            IGNORE_CWISE
      INTEGER            ITHRESH
      REAL               RTHRESH, UNSTABLE_THRESH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, CSYCON, CLA_SYRFSX_EXTENDED
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT, TRANSFER
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, BLAS_FPINFO_X, ILATRANS, ILAPREC
      EXTERNAL           SLAMCH, CLANSY, CLA_SYRCOND_X, CLA_SYRCOND_C
      REAL               SLAMCH, CLANSY, CLA_SYRCOND_X, CLA_SYRCOND_C
      LOGICAL            LSAME
      INTEGER            BLAS_FPINFO_X
      INTEGER            ILATRANS, ILAPREC
*     ..
*     .. Executable Statements ..
*
*     Check the input parameters.
*
      INFO = 0
```

```
      REF_TYPE = INT( ITREF_DEFAULT )
      IF ( NPARAMS .GE. LA_LINRX_ITREF_I ) THEN
         IF ( PARAMS( LA_LINRX_ITREF_I ) .LT. 0.0 ) THEN
            PARAMS( LA_LINRX_ITREF_I ) = ITREF_DEFAULT
         ELSE
            REF_TYPE = PARAMS( LA_LINRX_ITREF_I )
         END IF
      END IF
*
*     Set default parameters.
*
      ILLRCOND_THRESH = REAL( N ) * SLAMCH( 'Epsilon' )
      ITHRESH = INT( ITHRESH_DEFAULT )
      RTHRESH = RTHRESH_DEFAULT
      UNSTABLE_THRESH = DZTHRESH_DEFAULT
      IGNORE_CWISE = COMPONENTWISE_DEFAULT .EQ. 0.0
*
      IF ( NPARAMS.GE.LA_LINRX_ITHRESH_I ) THEN
         IF ( PARAMS( LA_LINRX_ITHRESH_I ).LT.0.0 ) THEN
            PARAMS( LA_LINRX_ITHRESH_I ) = ITHRESH
         ELSE
            ITHRESH = INT( PARAMS( LA_LINRX_ITHRESH_I ) )
         END IF
      END IF
      IF ( NPARAMS.GE.LA_LINRX_CWISE_I ) THEN
         IF ( PARAMS( LA_LINRX_CWISE_I ).LT.0.0 ) THEN
            IF ( IGNORE_CWISE ) THEN
               PARAMS( LA_LINRX_CWISE_I ) = 0.0
            ELSE
               PARAMS( LA_LINRX_CWISE_I ) = 1.0
            END IF
         ELSE
            IGNORE_CWISE = PARAMS( LA_LINRX_CWISE_I ) .EQ. 0.0
         END IF
      END IF
      IF ( REF_TYPE .EQ. 0 .OR. N_ERR_BNDS .EQ. 0 ) THEN
         N_NORMS = 0
      ELSE IF ( IGNORE_CWISE ) THEN
         N_NORMS = 1
      ELSE
         N_NORMS = 2
      END IF
*
      RCEQU = LSAME( EQUED, 'Y' )
*
*     Test input parameters.
*
      IF ( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
        INFO = -1
      ELSE IF( .NOT.RCEQU .AND. .NOT.LSAME( EQUED, 'N' ) ) THEN
```

```
        INFO = -2
      ELSE IF( N.LT.0 ) THEN
        INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
        INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
        INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
        INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
        INFO = -12
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
        INFO = -14
      END IF
      IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'CSYRFSX', -INFO )
        RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         RCOND = 1.0
         DO J = 1, NRHS
            BERR( J ) = 0.0
            IF ( N_ERR_BNDS .GE. 1 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
            IF ( N_ERR_BNDS .GE. 2 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 0.0
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 0.0
            END IF
            IF ( N_ERR_BNDS .GE. 3 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 1.0
            END IF
         END DO
         RETURN
      END IF
*
*     Default to failure.
*
      RCOND = 0.0
      DO J = 1, NRHS
         BERR( J ) = 1.0
         IF ( N_ERR_BNDS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
            ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
         END IF
```

```
         IF ( N_ERR_BNDS .GE. 2 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
         END IF
         IF ( N_ERR_BNDS .GE. 3 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 0.0
            ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 0.0
         END IF
      END DO
*
*     Compute the norm of A and the reciprocal of the condition
*     number of A.
*
      NORM = 'I'
      ANORM = CLANSY( NORM, UPLO, N, A, LDA, RWORK )
      CALL CSYCON( UPLO, N, AF, LDAF, IPIV, ANORM, RCOND, WORK,
     $     INFO )
*
*     Perform refinement on each right-hand side
*
      IF ( REF_TYPE .NE. 0 ) THEN

         PREC_TYPE = ILAPREC( 'D' )

         CALL CLA_SYRFSX_EXTENDED( PREC_TYPE, UPLO,  N,
     $        NRHS, A, LDA, AF, LDAF, IPIV, RCEQU, S, B,
     $        LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM, ERR_BNDS_COMP,
     $        WORK, RWORK, WORK(N+1),
     $        TRANSFER (RWORK(1:2*N), (/ (ZERO, ZERO) /), N), RCOND,
     $        ITHRESH, RTHRESH, UNSTABLE_THRESH, IGNORE_CWISE,
     $        INFO )
      END IF

      ERR_LBND = MAX( 10.0, SQRT( REAL( N ) ) ) * SLAMCH( 'Epsilon' )
      IF (N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 1) THEN
*
*     Compute scaled normwise condition number cond(A*C).
*
         IF ( RCEQU ) THEN
            RCOND_TMP = CLA_SYRCOND_C( UPLO, N, A, LDA, AF, LDAF, IPIV,
     $           S, .TRUE., INFO, WORK, RWORK )
         ELSE
            RCOND_TMP = CLA_SYRCOND_C( UPLO, N, A, LDA, AF, LDAF, IPIV,
     $           S, .FALSE., INFO, WORK, RWORK )
         END IF
         DO J = 1, NRHS
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
```

```
     $           .AND. ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .GT. 1.0 )
     $           ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 0.0
               IF ( INFO .LE. N ) INFO = N + J
            ELSE IF ( ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .LT. ERR_LBND )
     $              THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF
         END DO
      END IF

      IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 2 ) THEN
*
*     Compute componentwise condition number cond(A*diag(Y(:,J))) for
*     each right-hand side using the current solution as an estimate of
*     the true solution.  If the componentwise error estimate is too
*     large, then the solution is a lousy estimate of truth and the
*     estimated RCOND may be too optimistic.  To avoid misleading users,
*     the inverse condition number is set to 0.0 when the estimated
*     cwise error is at least CWISE_WRONG.
*
         CWISE_WRONG = SQRT( SLAMCH( 'Epsilon' ) )
         DO J = 1, NRHS
            IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .LT. CWISE_WRONG )
     $     THEN
               RCOND_TMP = CLA_SYRCOND_X( UPLO, N, A, LDA, AF, LDAF,
     $         IPIV, X(1,J), INFO, WORK, RWORK )
            ELSE
               RCOND_TMP = 0.0
            END IF
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .GT. 1.0 )
     $           ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0

*
```

```
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 0.0
               IF ( .NOT. IGNORE_CWISE
     $              .AND. INFO.LT.N + J ) INFO = N + J
            ELSE IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I )
     $              .LT. ERR_LBND ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF

         END DO
      END IF
*
      RETURN
*
*     End of CSYRFSX
*
      END
```

———-

60.13.4 lisp code

— **csyrfsx.lisp** —

———-

60.14 csytf2

60.14.1 Axiom unit tests

— **csytf2.input** —

```
)set break resume
```

```
)sys rm -f csytf2.output
)spool csytf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

60.14.2 Axiom help page

— csytf2.help —

```
======================================================================
csytf2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csytf2 (UPLO, N, A, LDA, IPIV, INFO)
           CSYTF2 computes the factorization of a real symmetric indefinite
           matrix, using the diagonal pivoting method (unblocked algorithm).

Function/Subroutine Documentation
   subroutine csytf2 (character UPLO, integer N, complex, dimension( lda, * )
       A, integer LDA, integer, dimension( * ) IPIV, integer INFO)
       CSYTF2 computes the factorization of a real symmetric indefinite
       matrix, using the diagonal pivoting method (unblocked algorithm).

Purpose:
   CSYTF2 computes the factorization of a complex symmetric matrix A
   using the Bunch-Kaufman diagonal pivoting method:

   A = U*D*U**T  or  A = L*D*L**T

   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, U**T is the transpose of U, and D is symmetric and
   block diagonal with 1-by-1 and 2-by-2 diagonal blocks.

   This is the unblocked version of the algorithm, calling Level 2 BLAS.

Parameters:
```

```
UPLO
  UPLO is CHARACTER*1
  Specifies whether the upper or lower triangular part of the
  symmetric matrix A is stored:
  = 'U':  Upper triangular
  = 'L':  Lower triangular

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  n-by-n upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading n-by-n lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, the block diagonal matrix D and the multipliers used
  to obtain the factor U or L (see below for further details).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D.

  If UPLO = 'U':
     If IPIV(k) > 0, then rows and columns k and IPIV(k) were
     interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) = IPIV(k-1) < 0, then rows and columns
     k-1 and -IPIV(k) were interchanged and D(k-1:k,k-1:k)
     is a 2-by-2 diagonal block.

  If UPLO = 'L':
     If IPIV(k) > 0, then rows and columns k and IPIV(k) were
     interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) = IPIV(k+1) < 0, then rows and columns
     k+1 and -IPIV(k) were interchanged and D(k:k+1,k:k+1)
     is a 2-by-2 diagonal block.

INFO
```

```
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -k, the k-th argument had an illegal value
   > 0: if INFO = k, D(k,k) is exactly zero.  The factorization
      has been completed, but the block diagonal matrix D is
      exactly singular, and division by zero will occur if it
      is used to solve a system of equations.

Further Details:
   If UPLO = 'U', then A = U*D*U**T, where
      U = P(n)*U(n)* ... *P(k)U(k)* ...,
   i.e., U is a product of terms P(k)*U(k), where k decreases from n to
   1 in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and U(k) is a unit upper triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                       (   I    v    0   )   k-s
             U(k) =    (   0    I    0   )   s
                       (   0    0    I   )   n-k
                          k-s   s   n-k

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(1:k-1,k).
   If s = 2, the upper triangle of D(k) overwrites A(k-1,k-1), A(k-1,k),
   and A(k,k), and v overwrites A(1:k-2,k-1:k).

   If UPLO = 'L', then A = L*D*L**T, where
      L = P(1)*L(1)* ... *P(k)*L(k)* ...,
   i.e., L is a product of terms P(k)*L(k), where k increases from 1 to
   n in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and L(k) is a unit lower triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                       (   I    0     0   )  k-1
             L(k) =    (   0    I     0   )  s
                       (   0    v     I   )  n-k-s+1
                          k-1   s  n-k-s+1

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(k+1:n,k).
   If s = 2, the lower triangle of D(k) overwrites A(k,k), A(k+1,k),
   and A(k+1,k+1), and v overwrites A(k+2:n,k:k+1).
```

———-

60.14.3 fortran code

— csytf2.f —

```
*  ==================================================================
      SUBROUTINE CSYTF2( UPLO, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * )
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               EIGHT, SEVTEN
      PARAMETER          ( EIGHT = 8.0E+0, SEVTEN = 17.0E+0 )
      COMPLEX            CONE
      PARAMETER          ( CONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, IMAX, J, JMAX, K, KK, KP, KSTEP
      REAL               ABSAKK, ALPHA, COLMAX, ROWMAX
      COMPLEX            D11, D12, D21, D22, R1, T, WK, WKM1, WKP1, Z
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, SISNAN
      INTEGER            ICAMAX
      EXTERNAL           LSAME, ICAMAX, SISNAN
*     ..
*     .. External Subroutines ..
      EXTERNAL           CSCAL, CSWAP, CSYR, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, MAX, REAL, SQRT
*     ..
*     .. Statement Functions ..
```

```
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( Z ) = ABS( REAL( Z ) ) + ABS( AIMAG( Z ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CSYTF2', -INFO )
         RETURN
      END IF
*
*     Initialize ALPHA for use in choosing pivot block size.
*
      ALPHA = ( ONE+SQRT( SEVTEN ) ) / EIGHT
*
      IF( UPPER ) THEN
*
*        Factorize A as U*D*U**T using the upper triangle of A
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        1 or 2
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop
*
         IF( K.LT.1 )
     $      GO TO 70
         KSTEP = 1
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = CABS1( A( K, K ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
```

```
*        Determine both COLMAX and IMAX.
*
         IF( K.GT.1 ) THEN
            IMAX = ICAMAX( K-1, A( 1, K ), 1 )
            COLMAX = CABS1( A( IMAX, K ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( MAX( ABSAKK, COLMAX ).EQ.ZERO .OR. SISNAN(ABSAKK) ) THEN
*
*           Column K is zero or underflow, or contains a NaN:
*           set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
         ELSE
            IF( ABSAKK.GE.ALPHA*COLMAX ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
            ELSE
*
*              JMAX is the column-index of the largest off-diagonal
*              element in row IMAX, and ROWMAX is its absolute value
*
               JMAX = IMAX + ICAMAX( K-IMAX, A( IMAX, IMAX+1 ), LDA )
               ROWMAX = CABS1( A( IMAX, JMAX ) )
               IF( IMAX.GT.1 ) THEN
                  JMAX = ICAMAX( IMAX-1, A( 1, IMAX ), 1 )
                  ROWMAX = MAX( ROWMAX, CABS1( A( JMAX, IMAX ) ) )
               END IF
*
               IF( ABSAKK.GE.ALPHA*COLMAX*( COLMAX / ROWMAX ) ) THEN
*
*                 no interchange, use 1-by-1 pivot block
*
                  KP = K
               ELSE IF( CABS1( A( IMAX, IMAX ) ).GE.ALPHA*ROWMAX ) THEN
*
*                 interchange rows and columns K and IMAX, use 1-by-1
*                 pivot block
*
                  KP = IMAX
               ELSE
*
*                 interchange rows and columns K-1 and IMAX, use 2-by-2
*                 pivot block
```

```
*
                 KP = IMAX
                 KSTEP = 2
              END IF
           END IF
*
           KK = K - KSTEP + 1
           IF( KP.NE.KK ) THEN
*
*             Interchange rows and columns KK and KP in the leading
*             submatrix A(1:k,1:k)
*
              CALL CSWAP( KP-1, A( 1, KK ), 1, A( 1, KP ), 1 )
              CALL CSWAP( KK-KP-1, A( KP+1, KK ), 1, A( KP, KP+1 ),
     $                    LDA )
              T = A( KK, KK )
              A( KK, KK ) = A( KP, KP )
              A( KP, KP ) = T
              IF( KSTEP.EQ.2 ) THEN
                 T = A( K-1, K )
                 A( K-1, K ) = A( KP, K )
                 A( KP, K ) = T
              END IF
           END IF
*
*          Update the leading submatrix
*
           IF( KSTEP.EQ.1 ) THEN
*
*             1-by-1 pivot block D(k): column k now holds
*
*             W(k) = U(k)*D(k)
*
*             where U(k) is the k-th column of U
*
*             Perform a rank-1 update of A(1:k-1,1:k-1) as
*
*             A := A - U(k)*D(k)*U(k)**T = A - W(k)*1/D(k)*W(k)**T
*
              R1 = CONE / A( K, K )
              CALL CSYR( UPLO, K-1, -R1, A( 1, K ), 1, A, LDA )
*
*             Store U(k) in column k
*
              CALL CSCAL( K-1, R1, A( 1, K ), 1 )
           ELSE
*
*             2-by-2 pivot block D(k): columns k and k-1 now hold
*
*             ( W(k-1) W(k) ) = ( U(k-1) U(k) )*D(k)
```

```
*
*              where U(k) and U(k-1) are the k-th and (k-1)-th columns
*              of U
*
*              Perform a rank-2 update of A(1:k-2,1:k-2) as
*
*              A := A - ( U(k-1) U(k) )*D(k)*( U(k-1) U(k) )**T
*                 = A - ( W(k-1) W(k) )*inv(D(k))*( W(k-1) W(k) )**T
*
               IF( K.GT.2 ) THEN
*
                  D12 = A( K-1, K )
                  D22 = A( K-1, K-1 ) / D12
                  D11 = A( K, K ) / D12
                  T = CONE / ( D11*D22-CONE )
                  D12 = T / D12
*
                  DO 30 J = K - 2, 1, -1
                     WKM1 = D12*( D11*A( J, K-1 )-A( J, K ) )
                     WK = D12*( D22*A( J, K )-A( J, K-1 ) )
                     DO 20 I = J, 1, -1
                        A( I, J ) = A( I, J ) - A( I, K )*WK -
     $                             A( I, K-1 )*WKM1
   20                CONTINUE
                     A( J, K ) = WK
                     A( J, K-1 ) = WKM1
   30             CONTINUE
*
               END IF
*
            END IF
         END IF
*
*        Store details of the interchanges in IPIV
*
         IF( KSTEP.EQ.1 ) THEN
            IPIV( K ) = KP
         ELSE
            IPIV( K ) = -KP
            IPIV( K-1 ) = -KP
         END IF
*
*        Decrease K and return to the start of the main loop
*
         K = K - KSTEP
         GO TO 10
*
      ELSE
*
*        Factorize A as L*D*L**T using the lower triangle of A
```

```
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2
*
         K = 1
   40    CONTINUE
*
*        If K > N, exit from loop
*
         IF( K.GT.N )
     $      GO TO 70
         KSTEP = 1
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = CABS1( A( K, K ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.LT.N ) THEN
            IMAX = K + ICAMAX( N-K, A( K+1, K ), 1 )
            COLMAX = CABS1( A( IMAX, K ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( MAX( ABSAKK, COLMAX ).EQ.ZERO .OR. SISNAN(ABSAKK) ) THEN
*
*           Column K is zero or underflow, or contains a NaN:
*           set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
         ELSE
            IF( ABSAKK.GE.ALPHA*COLMAX ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
            ELSE
*
*              JMAX is the column-index of the largest off-diagonal
*              element in row IMAX, and ROWMAX is its absolute value
*
               JMAX = K - 1 + ICAMAX( IMAX-K, A( IMAX, K ), LDA )
               ROWMAX = CABS1( A( IMAX, JMAX ) )
```

```
               IF( IMAX.LT.N ) THEN
                  JMAX = IMAX + ICAMAX( N-IMAX, A( IMAX+1, IMAX ), 1 )
                  ROWMAX = MAX( ROWMAX, CABS1( A( JMAX, IMAX ) ) )
               END IF
*
               IF( ABSAKK.GE.ALPHA*COLMAX*( COLMAX / ROWMAX ) ) THEN
*
*                 no interchange, use 1-by-1 pivot block
*
                  KP = K
               ELSE IF( CABS1( A( IMAX, IMAX ) ).GE.ALPHA*ROWMAX ) THEN
*
*                 interchange rows and columns K and IMAX, use 1-by-1
*                 pivot block
*
                  KP = IMAX
               ELSE
*
*                 interchange rows and columns K+1 and IMAX, use 2-by-2
*                 pivot block
*
                  KP = IMAX
                  KSTEP = 2
               END IF
            END IF
*
            KK = K + KSTEP - 1
            IF( KP.NE.KK ) THEN
*
*              Interchange rows and columns KK and KP in the trailing
*              submatrix A(k:n,k:n)
*
               IF( KP.LT.N )
     $            CALL CSWAP( N-KP, A( KP+1, KK ), 1, A( KP+1, KP ), 1 )
               CALL CSWAP( KP-KK-1, A( KK+1, KK ), 1, A( KP, KK+1 ),
     $                     LDA )
               T = A( KK, KK )
               A( KK, KK ) = A( KP, KP )
               A( KP, KP ) = T
               IF( KSTEP.EQ.2 ) THEN
                  T = A( K+1, K )
                  A( K+1, K ) = A( KP, K )
                  A( KP, K ) = T
               END IF
            END IF
*
*           Update the trailing submatrix
*
            IF( KSTEP.EQ.1 ) THEN
*
```

```
*              1-by-1 pivot block D(k): column k now holds
*
*              W(k) = L(k)*D(k)
*
*              where L(k) is the k-th column of L
*
               IF( K.LT.N ) THEN
*
*                 Perform a rank-1 update of A(k+1:n,k+1:n) as
*
*                 A := A - L(k)*D(k)*L(k)**T = A - W(k)*(1/D(k))*W(k)**T
*
                  R1 = CONE / A( K, K )
                  CALL CSYR( UPLO, N-K, -R1, A( K+1, K ), 1,
     $                       A( K+1, K+1 ), LDA )
*
*                 Store L(k) in column K
*
                  CALL CSCAL( N-K, R1, A( K+1, K ), 1 )
               END IF
            ELSE
*
*              2-by-2 pivot block D(k)
*
               IF( K.LT.N-1 ) THEN
*
*                 Perform a rank-2 update of A(k+2:n,k+2:n) as
*
*                 A := A - ( L(k) L(k+1) )*D(k)*( L(k) L(k+1) )**T
*                    = A - ( W(k) W(k+1) )*inv(D(k))*( W(k) W(k+1) )**T
*
*                 where L(k) and L(k+1) are the k-th and (k+1)-th
*                 columns of L
*
                  D21 = A( K+1, K )
                  D11 = A( K+1, K+1 ) / D21
                  D22 = A( K, K ) / D21
                  T = CONE / ( D11*D22-CONE )
                  D21 = T / D21
*
                  DO 60 J = K + 2, N
                     WK = D21*( D11*A( J, K )-A( J, K+1 ) )
                     WKP1 = D21*( D22*A( J, K+1 )-A( J, K ) )
                     DO 50 I = J, N
                        A( I, J ) = A( I, J ) - A( I, K )*WK -
     $                              A( I, K+1 )*WKP1
   50                CONTINUE
                     A( J, K ) = WK
                     A( J, K+1 ) = WKP1
   60             CONTINUE
```

```
               END IF
            END IF
         END IF
*
*        Store details of the interchanges in IPIV
*
         IF( KSTEP.EQ.1 ) THEN
            IPIV( K ) = KP
         ELSE
            IPIV( K ) = -KP
            IPIV( K+1 ) = -KP
         END IF
*
*        Increase K and return to the start of the main loop
*
         K = K + KSTEP
         GO TO 40
*
      END IF
*
   70 CONTINUE
      RETURN
*
*     End of CSYTF2
*
      END
```

———-

60.14.4 lisp code

— **csytf2.lisp** —

```
(let* ((zero 0.0)
       (one 1.0)
       (eight 8.0)
       (sevten 17.0)
       (cone (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 8.0 8.0) eight)
           (type (single-float 17.0 17.0) sevten)
           (type (f2cl-lib:complex8) cone)
           (ignorable zero one eight sevten cone))
  (defun csytf2 (uplo n a lda ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) a)
             (type (f2cl-lib:integer4) info lda n)
```

```
          (type (string *) uplo))
(f2cl-lib:with-multi-array-data
    ((uplo character uplo-%data% uplo-%offset%)
     (a f2cl-lib:complex8 a-%data% a-%offset%)
     (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
  (labels ((cabs1 (z)
             (+ (abs (f2cl-lib:freal z)) (abs (f2cl-lib:aimag z)))))
    (declare
     (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
      cabs1))
    (prog ((d11 #C(0.0 0.0)) (d12 #C(0.0 0.0)) (d21 #C(0.0 0.0))
           (d22 #C(0.0 0.0)) (r1 #C(0.0 0.0)) (t$ #C(0.0 0.0))
           (wk #C(0.0 0.0)) (wkm1 #C(0.0 0.0)) (wkp1 #C(0.0 0.0))
           (z #C(0.0 0.0)) (absakk 0.0) (alpha 0.0) (colmax 0.0)
           (rowmax 0.0) (i 0) (imax 0) (j 0) (jmax 0) (k 0) (kk 0) (kp 0)
           (kstep 0) (upper nil))
      (declare
       (type (f2cl-lib:complex8) d11 d12 d21 d22 r1 t$ wk wkm1 wkp1 z)
       (type (single-float) absakk alpha colmax rowmax)
       (type (f2cl-lib:integer4) i imax j jmax k kk kp kstep)
       (type f2cl-lib:logical upper))
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (cond
       ((and (not upper)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -4)))
      (cond
       ((/= info 0) (xerbla "CSYTF2" (f2cl-lib:int-sub info))
        (go end_label)))
      (setf alpha (/ (+ one (f2cl-lib:fsqrt sevten)) eight))
      (cond
       (upper
        (tagbody
          (setf k n)
         label10
          (if (< k 1)
```

```
    (go label70))
(setf kstep 1)
(setf absakk
        (cabs1
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)))
(cond
 ((> k 1)
  (setf imax
          (icamax (f2cl-lib:int-sub k 1)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1))
  (setf colmax
          (cabs1
           (f2cl-lib:fref a-%data% (imax k) ((1 lda) (1 *))
                          a-%offset%))))
 (t (setf colmax zero)))
(cond
 ((or (= (max absakk colmax) zero)
      (multiple-value-bind (ret-val var-0)
          (sisnan absakk)
        (declare (ignore))
        (when var-0 (setf absakk var-0))
        ret-val))
  (if (= info 0)
      (setf info k))
  (setf kp k))
 (t
  (cond ((>= absakk (* alpha colmax)) (setf kp k))
        (t
         (setf jmax
                 (f2cl-lib:int-add imax
                                   (multiple-value-bind
                                       (ret-val var-0 var-1
                                        var-2)
                                       (icamax
                                        (f2cl-lib:int-sub k imax)
                                        (f2cl-lib:array-slice
                                         a-%data%
                                         f2cl-lib:complex8
                                         (imax
                                          (f2cl-lib:int-add imax
                                                            1))
                                         ((1 lda) (1 *))
                                         a-%offset%)
                                        lda)
                                     (declare
                                      (ignore var-0 var-1))
                                     (when var-2
```

```
                                    (setf lda var-2))
                                  ret-val)))
        (setf rowmax
                (cabs1
                 (f2cl-lib:fref a-%data% (imax jmax)
                                ((1 lda) (1 *)) a-%offset%)))
        (cond
         ((> imax 1)
          (setf jmax
                  (icamax (f2cl-lib:int-sub imax 1)
                   (f2cl-lib:array-slice a-%data%
                                         f2cl-lib:complex8
                                         (1 imax)
                                         ((1 lda) (1 *))
                                         a-%offset%)
                   1))
          (setf rowmax
                  (max rowmax
                       (cabs1
                        (f2cl-lib:fref a-%data% (jmax imax)
                                       ((1 lda) (1 *))
                                       a-%offset%))))))
        (cond
         ((>= absakk
              (* alpha colmax (f2cl-lib:f2cl/ colmax rowmax)))
          (setf kp k))
         ((>=
           (cabs1 (f2cl-lib:fref a (imax imax) ((1 lda) (1 *))))
           (* alpha rowmax))
          (setf kp imax))
         (t (setf kp imax) (setf kstep 2)))))
(setf kk (f2cl-lib:int-add (f2cl-lib:int-sub k kstep) 1))
(cond
 ((/= kp kk)
  (cswap (f2cl-lib:int-sub kp 1)
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 kk)
                         ((1 lda) (1 *)) a-%offset%)
   1
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 kp)
                         ((1 lda) (1 *)) a-%offset%)
   1)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (cswap (f2cl-lib:int-sub kk kp 1)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ kp 1) kk) ((1 lda) (1 *))
                             a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             (kp (f2cl-lib:int-add kp 1))
                             ((1 lda) (1 *)) a-%offset%)
```

```
        lda)
      (declare (ignore var-0 var-1 var-2 var-3))
      (when var-4 (setf lda var-4)))
    (setf t$
            (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                         a-%offset%)
            (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%)
            t$)
    (cond
     ((= kstep 2)
      (setf t$
              (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                             ((1 lda) (1 *)) a-%offset%))
      (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                           ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                             a-%offset%))
      (setf (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                           a-%offset%)
              t$)))))
  (cond
   ((= kstep 1)
    (setf r1
            (/ cone
               (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                              a-%offset%)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (csyr uplo (f2cl-lib:int-sub k 1) (- r1)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                               ((1 lda) (1 *)) a-%offset%)
         1 a lda)
      (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
      (setf uplo var-0))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (cscal (f2cl-lib:int-sub k 1) r1
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                               ((1 lda) (1 *)) a-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3))
      (when var-1 (setf r1 var-1))))
   (t
    (cond
     ((> k 2)
      (setf d12
```

```
        (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                       ((1 lda) (1 *)) a-%offset%))
(setf d22
        (/
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-sub k 1)
                         (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         d12))
(setf d11
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         d12))
(setf t$ (/ cone (- (* d11 d22) cone)))
(setf d12 (/ t$ d12))
(f2cl-lib:fdo (j (f2cl-lib:int-add k (f2cl-lib:int-sub 2))
               (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j 1) nil)
  (tagbody
    (setf wkm1
            (* d12
               (-
                (* d11
                   (f2cl-lib:fref a-%data%
                                  (j
                                   (f2cl-lib:int-sub k 1))
                                  ((1 lda) (1 *))
                                  a-%offset%))
                (f2cl-lib:fref a-%data% (j k)
                               ((1 lda) (1 *))
                               a-%offset%))))
    (setf wk
            (* d12
               (-
                (* d22
                   (f2cl-lib:fref a-%data% (j k)
                                  ((1 lda) (1 *))
                                  a-%offset%))
                (f2cl-lib:fref a-%data%
                               (j (f2cl-lib:int-sub k 1))
                               ((1 lda) (1 *))
                               a-%offset%))))
    (f2cl-lib:fdo (i j
                   (f2cl-lib:int-add i
                                     (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                             a-%offset%)
```

```
                                (-
                                 (f2cl-lib:fref a-%data% (i j)
                                                ((1 lda) (1 *)) a-%offset%)
                                 (*
                                  (f2cl-lib:fref a-%data% (i k)
                                                 ((1 lda) (1 *))
                                                 a-%offset%)
                                  wk)
                                 (*
                                  (f2cl-lib:fref a-%data%
                                                 (i (f2cl-lib:int-sub k 1))
                                                 ((1 lda) (1 *))
                                                 a-%offset%)
                                  wkm1)))
                           label20))
                     (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                          a-%offset%)
                           wk)
                     (setf (f2cl-lib:fref a-%data%
                                          (j (f2cl-lib:int-sub k 1))
                                          ((1 lda) (1 *)) a-%offset%)
                           wkm1)
                  label30))))))))
            (cond
             ((= kstep 1)
              (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
                    kp))
             (t
              (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
                    (f2cl-lib:int-sub kp))
              (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                                   ((1 *)) ipiv-%offset%)
                    (f2cl-lib:int-sub kp))))
            (setf k (f2cl-lib:int-sub k kstep))
            (go label10)))
          (t
           (tagbody
             (setf k 1)
            label40
             (if (> k n)
                 (go label70))
             (setf kstep 1)
             (setf absakk
                     (cabs1
                      (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                     a-%offset%)))
             (cond
              ((< k n)
               (setf imax
                     (f2cl-lib:int-add k
```

```
                                (icamax (f2cl-lib:int-sub n k)
                                 (f2cl-lib:array-slice a-%data%
                                                       f2cl-lib:complex8
                                                       ((+ k 1) k)
                                                       ((1 lda)
                                                        (1 *))
                                                       a-%offset%)
                                 1)))
                  (setf colmax
                          (cabs1
                           (f2cl-lib:fref a-%data% (imax k) ((1 lda) (1 *))
                                          a-%offset%))))
                 (t (setf colmax zero)))
                (cond
                 ((or (= (max absakk colmax) zero)
                      (multiple-value-bind (ret-val var-0)
                          (sisnan absakk)
                        (declare (ignore))
                        (when var-0 (setf absakk var-0))
                        ret-val))
                  (if (= info 0)
                      (setf info k))
                  (setf kp k))
                 (t
                  (cond ((>= absakk (* alpha colmax)) (setf kp k))
                        (t
                         (setf jmax
                                 (f2cl-lib:int-add (f2cl-lib:int-sub k 1)
                                                   (multiple-value-bind
                                                       (ret-val var-0 var-1
                                                        var-2)
                                                       (icamax
                                                        (f2cl-lib:int-sub imax k)
                                                        (f2cl-lib:array-slice
                                                         a-%data%
                                                         f2cl-lib:complex8
                                                         (imax k) ((1 lda) (1 *))
                                                         a-%offset%)
                                                        lda)
                                                     (declare
                                                      (ignore var-0 var-1))
                                                     (when var-2
                                                       (setf lda var-2))
                                                     ret-val)))
                         (setf rowmax
                                 (cabs1
                                  (f2cl-lib:fref a-%data% (imax jmax)
                                                 ((1 lda) (1 *)) a-%offset%)))
                         (cond
                          ((< imax n)
```

```
          (setf jmax
                  (f2cl-lib:int-add imax
                                    (icamax
                                     (f2cl-lib:int-sub n imax)
                                     (f2cl-lib:array-slice
                                      a-%data% f2cl-lib:complex8
                                      ((+ imax 1) imax)
                                      ((1 lda) (1 *))
                                      a-%offset%)
                                     1)))
          (setf rowmax
                  (max rowmax
                       (cabs1
                        (f2cl-lib:fref a-%data% (jmax imax)
                                       ((1 lda) (1 *))
                                       a-%offset%))))))
        (cond
         ((>= absakk
              (* alpha colmax (f2cl-lib:f2cl/ colmax rowmax)))
          (setf kp k))
         ((>=
           (cabs1 (f2cl-lib:fref a (imax imax) ((1 lda) (1 *))))
           (* alpha rowmax))
          (setf kp imax))
         (t (setf kp imax) (setf kstep 2)))))
(setf kk (f2cl-lib:int-sub (f2cl-lib:int-add k kstep) 1))
(cond
 ((/= kp kk)
  (if (< kp n)
      (cswap (f2cl-lib:int-sub n kp)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ kp 1) kk) ((1 lda) (1 *))
                             a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ kp 1) kp) ((1 lda) (1 *))
                             a-%offset%)
       1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (cswap (f2cl-lib:int-sub kp kk 1)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ kk 1) kk) ((1 lda) (1 *))
                             a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             (kp (f2cl-lib:int-add kk 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
```

```
  (setf t$
          (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                       a-%offset%)
          (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                       a-%offset%)
          t$)
  (cond
   ((= kstep 2)
    (setf t$
            (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                           ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                         ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                         a-%offset%)
            t$)))))
(cond
 ((= kstep 1)
  (cond
   ((< k n)
    (setf r1
            (/ cone
               (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                              a-%offset%)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (csyr uplo (f2cl-lib:int-sub n k) (- r1)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               ((+ k 1) k) ((1 lda) (1 *))
                               a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               ((+ k 1) (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
      (setf uplo var-0))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (cscal (f2cl-lib:int-sub n k) r1
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               ((+ k 1) k) ((1 lda) (1 *))
                               a-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3))
```

```
      (when var-1 (setf r1 var-1))))))
(t
 (cond
  ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
   (setf d21
           (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                          ((1 lda) (1 *)) a-%offset%))
   (setf d11
           (/
            (f2cl-lib:fref a-%data%
                           ((f2cl-lib:int-add k 1)
                            (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
            d21))
   (setf d22
           (/
            (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                           a-%offset%)
            d21))
   (setf t$ (/ cone (- (* d11 d22) cone)))
   (setf d21 (/ t$ d21))
   (f2cl-lib:fdo (j (f2cl-lib:int-add k 2)
                  (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf wk
               (* d21
                  (-
                   (* d11
                      (f2cl-lib:fref a-%data% (j k)
                                     ((1 lda) (1 *))
                                     a-%offset%))
                   (f2cl-lib:fref a-%data%
                                  (j (f2cl-lib:int-add k 1))
                                  ((1 lda) (1 *))
                                  a-%offset%))))
       (setf wkp1
               (* d21
                  (-
                   (* d22
                      (f2cl-lib:fref a-%data%
                                     (j
                                      (f2cl-lib:int-add k 1))
                                     ((1 lda) (1 *))
                                     a-%offset%))
                   (f2cl-lib:fref a-%data% (j k)
                                  ((1 lda) (1 *))
                                  a-%offset%))))
       (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                     ((> i n) nil)
```

```
                          (tagbody
                            (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                                 a-%offset%)
                                  (-
                                   (f2cl-lib:fref a-%data% (i j)
                                                  ((1 lda) (1 *)) a-%offset%)
                                   (*
                                    (f2cl-lib:fref a-%data% (i k)
                                                   ((1 lda) (1 *))
                                                   a-%offset%)
                                    wk)
                                   (*
                                    (f2cl-lib:fref a-%data%
                                                   (i (f2cl-lib:int-add k 1))
                                                   ((1 lda) (1 *))
                                                   a-%offset%)
                                    wkp1)))
                             label50))
                        (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                             a-%offset%)
                              wk)
                        (setf (f2cl-lib:fref a-%data%
                                             (j (f2cl-lib:int-add k 1))
                                             ((1 lda) (1 *)) a-%offset%)
                              wkp1)
                       label60))))))))
                (cond
                 ((= kstep 1)
                  (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
                        kp))
                 (t
                  (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
                        (f2cl-lib:int-sub kp))
                  (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
                                       ((1 *)) ipiv-%offset%)
                        (f2cl-lib:int-sub kp))))
                (setf k (f2cl-lib:int-add k kstep))
                (go label40))))
          label70
           (go end_label)
          end_label
           (return (values uplo nil nil lda nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csytf2
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string)
```

```
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::uplo nil nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::info)
:calls '(fortran-to-lisp::csyr))))
```

———-

60.15 csytf2_rook

60.15.1 Axiom unit tests

— **csytf2rook.input** —

```
)set break resume
)sys rm -f csytf2rook.output
)spool csytf2rook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.15.2 Axiom help page

— **csytf2_rook.help** —

```
==========================================================================
csytf2_rook examples
==========================================================================

==========================================================================
```

```
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csytf2_rook (UPLO, N, A, LDA, IPIV, INFO)
           CSYTF2_ROOK computes the factorization of a complex symmetric
           indefinite matrix using the bounded Bunch-Kaufman ('rook') diagonal
           pivoting method (unblocked algorithm).

Function/Subroutine Documentation
   subroutine csytf2_rook (character UPLO, integer N, complex, dimension( lda,
       * ) A, integer LDA, integer, dimension( * ) IPIV, integer INFO)
       CSYTF2_ROOK computes the factorization of a complex symmetric
       indefinite matrix using the bounded Bunch-Kaufman ('rook') diagonal
       pivoting method (unblocked algorithm).

Purpose:
   CSYTF2_ROOK computes the factorization of a complex symmetric matrix A
   using the bounded Bunch-Kaufman ("rook") diagonal pivoting method:

   A = U*D*U**T  or  A = L*D*L**T

   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, U**T is the transpose of U, and D is symmetric and
   block diagonal with 1-by-1 and 2-by-2 diagonal blocks.

   This is the unblocked version of the algorithm, calling Level 2 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is stored:
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   n-by-n upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading n-by-n lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
```

```
   triangular part of A is not referenced.

   On exit, the block diagonal matrix D and the multipliers used
   to obtain the factor U or L (see below for further details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D.

   If UPLO = 'U':
     If IPIV(k) > 0, then rows and columns k and IPIV(k)
     were interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) < 0 and IPIV(k-1) < 0, then rows and
     columns k and -IPIV(k) were interchanged and rows and
     columns k-1 and -IPIV(k-1) were inerchaged,
     D(k-1:k,k-1:k) is a 2-by-2 diagonal block.

   If UPLO = 'L':
     If IPIV(k) > 0, then rows and columns k and IPIV(k)
     were interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) < 0 and IPIV(k+1) < 0, then rows and
     columns k and -IPIV(k) were interchanged and rows and
     columns k+1 and -IPIV(k+1) were inerchaged,
     D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -k, the k-th argument had an illegal value
   > 0: if INFO = k, D(k,k) is exactly zero.  The factorization
      has been completed, but the block diagonal matrix D is
      exactly singular, and division by zero will occur if it
      is used to solve a system of equations.

Further Details:

   If UPLO = 'U', then A = U*D*U**T, where
      U = P(n)*U(n)* ... *P(k)U(k)* ...,
   i.e., U is a product of terms P(k)*U(k), where k decreases from n to
   1 in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and U(k) is a unit upper triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then
```

```
                    (   I    v    0   )   k-s
             U(k) =  (   0    I    0   )   s
                    (   0    0    I   )   n-k
                       k-s   s   n-k

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(1:k-1,k).
   If s = 2, the upper triangle of D(k) overwrites A(k-1,k-1), A(k-1,k),
   and A(k,k), and v overwrites A(1:k-2,k-1:k).

   If UPLO = 'L', then A = L*D*L**T, where
      L = P(1)*L(1)* ... *P(k)*L(k)* ...,
   i.e., L is a product of terms P(k)*L(k), where k increases from 1 to
   n in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and L(k) is a unit lower triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                    (   I    0     0   )  k-1
             L(k) =  (   0    I     0   )  s
                    (   0    v     I   )  n-k-s+1
                       k-1   s  n-k-s+1

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(k+1:n,k).
   If s = 2, the lower triangle of D(k) overwrites A(k,k), A(k+1,k),
   and A(k+1,k+1), and v overwrites A(k+2:n,k:k+1).
```

———-

60.15.3 fortran code

— **csytf2_rook.f** —

```
*  =====================================================================
      SUBROUTINE CSYTF2_ROOK( UPLO, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               EIGHT, SEVTEN
      PARAMETER          ( EIGHT = 8.0E+0, SEVTEN = 17.0E+0 )
      COMPLEX            CONE
      PARAMETER          ( CONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER, DONE
      INTEGER            I, IMAX, J, JMAX, ITEMP, K, KK, KP, KSTEP,
     $                   P, II
      REAL               ABSAKK, ALPHA, COLMAX, ROWMAX, STEMP, SFMIN
      COMPLEX            D11, D12, D21, D22, T, WK, WKM1, WKP1, Z
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ICAMAX
      REAL               SLAMCH
      EXTERNAL           LSAME, ICAMAX, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           CSCAL, CSWAP, CSYR, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT, AIMAG, REAL
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( Z ) = ABS( REAL( Z ) ) + ABS( AIMAG( Z ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
```

```
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CSYTF2_ROOK', -INFO )
         RETURN
      END IF
*
*     Initialize ALPHA for use in choosing pivot block size.
*
      ALPHA = ( ONE+SQRT( SEVTEN ) ) / EIGHT
*
*     Compute machine safe minimum
*
      SFMIN = SLAMCH( 'S' )
*
      IF( UPPER ) THEN
*
*        Factorize A as U*D*U**T using the upper triangle of A
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        1 or 2
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop
*
         IF( K.LT.1 )
     $      GO TO 70
         KSTEP = 1
         P = K
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = CABS1( A( K, K ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.GT.1 ) THEN
            IMAX = ICAMAX( K-1, A( 1, K ), 1 )
            COLMAX = CABS1( A( IMAX, K ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( (MAX( ABSAKK, COLMAX ).EQ.ZERO) ) THEN
*
*           Column K is zero or underflow: set INFO and continue
*
```

```
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
         ELSE
*
            Test for interchange
*
            Equivalent to testing for (used to handle NaN and Inf)
            ABSAKK.GE.ALPHA*COLMAX
*
            IF( .NOT.( ABSAKK.LT.ALPHA*COLMAX ) ) THEN
*
               no interchange,
               use 1-by-1 pivot block
*
               KP = K
            ELSE
*
               DONE = .FALSE.
*
               Loop until pivot found
*
   12          CONTINUE
*
                  Begin pivot search loop body
*
                  JMAX is the column-index of the largest off-diagonal
                  element in row IMAX, and ROWMAX is its absolute value.
                  Determine both ROWMAX and JMAX.
*
                  IF( IMAX.NE.K ) THEN
                     JMAX = IMAX + ICAMAX( K-IMAX, A( IMAX, IMAX+1 ),
     $                                     LDA )
                     ROWMAX = CABS1( A( IMAX, JMAX ) )
                  ELSE
                     ROWMAX = ZERO
                  END IF
*
                  IF( IMAX.GT.1 ) THEN
                     ITEMP = ICAMAX( IMAX-1, A( 1, IMAX ), 1 )
                     STEMP = CABS1( A( ITEMP, IMAX ) )
                     IF( STEMP.GT.ROWMAX ) THEN
                        ROWMAX = STEMP
                        JMAX = ITEMP
                     END IF
                  END IF
*
                  Equivalent to testing for (used to handle NaN and Inf)
                  CABS1( A( IMAX, IMAX ) ).GE.ALPHA*ROWMAX
*
```

```
                 IF( .NOT.( CABS1(A( IMAX, IMAX )).LT.ALPHA*ROWMAX ) )
     $           THEN
*
*                   interchange rows and columns K and IMAX,
*                   use 1-by-1 pivot block
*
                    KP = IMAX
                    DONE = .TRUE.
*
*                Equivalent to testing for ROWMAX .EQ. COLMAX,
*                used to handle NaN and Inf
*
                 ELSE IF( ( P.EQ.JMAX ).OR.( ROWMAX.LE.COLMAX ) ) THEN
*
*                   interchange rows and columns K+1 and IMAX,
*                   use 2-by-2 pivot block
*
                    KP = IMAX
                    KSTEP = 2
                    DONE = .TRUE.
                 ELSE
*
*                   Pivot NOT found, set variables and repeat
*
                    P = IMAX
                    COLMAX = ROWMAX
                    IMAX = JMAX
                 END IF
*
*                End pivot search loop body
*
              IF( .NOT. DONE ) GOTO 12
*
           END IF
*
*          Swap TWO rows and TWO columns
*
*          First swap
*
           IF( ( KSTEP.EQ.2 ) .AND. ( P.NE.K ) ) THEN
*
*             Interchange rows and column K and P in the leading
*             submatrix A(1:k,1:k) if we have a 2-by-2 pivot
*
              IF( P.GT.1 )
     $           CALL CSWAP( P-1, A( 1, K ), 1, A( 1, P ), 1 )
              IF( P.LT.(K-1) )
     $           CALL CSWAP( K-P-1, A( P+1, K ), 1, A( P, P+1 ),
     $                    LDA )
              T = A( K, K )
```

```
               A( K, K ) = A( P, P )
               A( P, P ) = T
            END IF
*
*           Second swap
*
            KK = K - KSTEP + 1
            IF( KP.NE.KK ) THEN
*
*              Interchange rows and columns KK and KP in the leading
*              submatrix A(1:k,1:k)
*
               IF( KP.GT.1 )
     $            CALL CSWAP( KP-1, A( 1, KK ), 1, A( 1, KP ), 1 )
               IF( ( KK.GT.1 ) .AND. ( KP.LT.(KK-1) ) )
     $            CALL CSWAP( KK-KP-1, A( KP+1, KK ), 1, A( KP, KP+1 ),
     $                      LDA )
               T = A( KK, KK )
               A( KK, KK ) = A( KP, KP )
               A( KP, KP ) = T
               IF( KSTEP.EQ.2 ) THEN
                  T = A( K-1, K )
                  A( K-1, K ) = A( KP, K )
                  A( KP, K ) = T
               END IF
            END IF
*
*           Update the leading submatrix
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column k now holds
*
*              W(k) = U(k)*D(k)
*
*              where U(k) is the k-th column of U
*
               IF( K.GT.1 ) THEN
*
*                 Perform a rank-1 update of A(1:k-1,1:k-1) and
*                 store U(k) in column k
*
                  IF( CABS1( A( K, K ) ).GE.SFMIN ) THEN
*
*                    Perform a rank-1 update of A(1:k-1,1:k-1) as
*                    A := A - U(k)*D(k)*U(k)**T
*                       = A - W(k)*1/D(k)*W(k)**T
*
                     D11 = CONE / A( K, K )
                     CALL CSYR( UPLO, K-1, -D11, A( 1, K ), 1, A, LDA )
```

```
*
*                    Store U(k) in column k
*
                     CALL CSCAL( K-1, D11, A( 1, K ), 1 )
                  ELSE
*
*                    Store L(k) in column K
*
                     D11 = A( K, K )
                     DO 16 II = 1, K - 1
                        A( II, K ) = A( II, K ) / D11
   16                CONTINUE
*
*                    Perform a rank-1 update of A(k+1:n,k+1:n) as
*                    A := A - U(k)*D(k)*U(k)**T
*                       = A - W(k)*(1/D(k))*W(k)**T
*                       = A - (W(k)/D(k))*(D(k))*(W(k)/D(K))**T
*
                     CALL CSYR( UPLO, K-1, -D11, A( 1, K ), 1, A, LDA )
                  END IF
               END IF
*
            ELSE
*
               2-by-2 pivot block D(k): columns k and k-1 now hold
*
               ( W(k-1) W(k) ) = ( U(k-1) U(k) )*D(k)
*
               where U(k) and U(k-1) are the k-th and (k-1)-th columns
               of U
*
               Perform a rank-2 update of A(1:k-2,1:k-2) as
*
               A := A - ( U(k-1) U(k) )*D(k)*( U(k-1) U(k) )**T
                  = A - ( ( A(k-1)A(k) )*inv(D(k)) ) * ( A(k-1)A(k) )**T
*
               and store L(k) and L(k+1) in columns k and k+1
*
               IF( K.GT.2 ) THEN
*
                  D12 = A( K-1, K )
                  D22 = A( K-1, K-1 ) / D12
                  D11 = A( K, K ) / D12
                  T = CONE / ( D11*D22-CONE )
*
                  DO 30 J = K - 2, 1, -1
*
                     WKM1 = T*( D11*A( J, K-1 )-A( J, K ) )
                     WK = T*( D22*A( J, K )-A( J, K-1 ) )
*
```

```
                  DO 20 I = J, 1, -1
                     A( I, J ) = A( I, J ) - (A( I, K ) / D12 )*WK -
     $                           ( A( I, K-1 ) / D12 )*WKM1
   20             CONTINUE
*
*                 Store U(k) and U(k-1) in cols k and k-1 for row J
*
                  A( J, K ) = WK / D12
                  A( J, K-1 ) = WKM1 / D12
*
   30           CONTINUE
*
              END IF
*
           END IF
        END IF
*
*       Store details of the interchanges in IPIV
*
        IF( KSTEP.EQ.1 ) THEN
           IPIV( K ) = KP
        ELSE
           IPIV( K ) = -P
           IPIV( K-1 ) = -KP
        END IF
*
*       Decrease K and return to the start of the main loop
*
        K = K - KSTEP
        GO TO 10
*
      ELSE
*
*       Factorize A as L*D*L**T using the lower triangle of A
*
*       K is the main loop index, increasing from 1 to N in steps of
*       1 or 2
*
        K = 1
   40   CONTINUE
*
*       If K > N, exit from loop
*
        IF( K.GT.N )
     $     GO TO 70
        KSTEP = 1
        P = K
*
*       Determine rows and columns to be interchanged and whether
*       a 1-by-1 or 2-by-2 pivot block will be used
```

```
*
         ABSAKK = CABS1( A( K, K ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.LT.N ) THEN
            IMAX = K + ICAMAX( N-K, A( K+1, K ), 1 )
            COLMAX = CABS1( A( IMAX, K ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( ( MAX( ABSAKK, COLMAX ).EQ.ZERO ) ) THEN
*
*           Column K is zero or underflow: set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
         ELSE
*
*           Test for interchange
*
*           Equivalent to testing for (used to handle NaN and Inf)
*           ABSAKK.GE.ALPHA*COLMAX
*
            IF( .NOT.( ABSAKK.LT.ALPHA*COLMAX ) ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
            ELSE
*
               DONE = .FALSE.
*
*              Loop until pivot found
*
   42          CONTINUE
*
*                 Begin pivot search loop body
*
*                 JMAX is the column-index of the largest off-diagonal
*                 element in row IMAX, and ROWMAX is its absolute value.
*                 Determine both ROWMAX and JMAX.
*
                  IF( IMAX.NE.K ) THEN
                     JMAX = K - 1 + ICAMAX( IMAX-K, A( IMAX, K ), LDA )
                     ROWMAX = CABS1( A( IMAX, JMAX ) )
```

```
                 ELSE
                    ROWMAX = ZERO
                 END IF
*
                 IF( IMAX.LT.N ) THEN
                    ITEMP = IMAX + ICAMAX( N-IMAX, A( IMAX+1, IMAX ),
     $                                     1 )
                    STEMP = CABS1( A( ITEMP, IMAX ) )
                    IF( STEMP.GT.ROWMAX ) THEN
                       ROWMAX = STEMP
                       JMAX = ITEMP
                    END IF
                 END IF
*
*                Equivalent to testing for (used to handle NaN and Inf)
*                CABS1( A( IMAX, IMAX ) ).GE.ALPHA*ROWMAX
*
                 IF( .NOT.( CABS1(A( IMAX, IMAX )).LT.ALPHA*ROWMAX ) )
     $           THEN
*
*                   interchange rows and columns K and IMAX,
*                   use 1-by-1 pivot block
*
                    KP = IMAX
                    DONE = .TRUE.
*
*                Equivalent to testing for ROWMAX .EQ. COLMAX,
*                used to handle NaN and Inf
*
                 ELSE IF( ( P.EQ.JMAX ).OR.( ROWMAX.LE.COLMAX ) ) THEN
*
*                   interchange rows and columns K+1 and IMAX,
*                   use 2-by-2 pivot block
*
                    KP = IMAX
                    KSTEP = 2
                    DONE = .TRUE.
                 ELSE
*
*                   Pivot NOT found, set variables and repeat
*
                    P = IMAX
                    COLMAX = ROWMAX
                    IMAX = JMAX
                 END IF
*
*                End pivot search loop body
*
              IF( .NOT. DONE ) GOTO 42
*
```

```
            END IF
*
*           Swap TWO rows and TWO columns
*
*           First swap
*
            IF( ( KSTEP.EQ.2 ) .AND. ( P.NE.K ) ) THEN
*
*              Interchange rows and column K and P in the trailing
*              submatrix A(k:n,k:n) if we have a 2-by-2 pivot
*
               IF( P.LT.N )
     $            CALL CSWAP( N-P, A( P+1, K ), 1, A( P+1, P ), 1 )
               IF( P.GT.(K+1) )
     $            CALL CSWAP( P-K-1, A( K+1, K ), 1, A( P, K+1 ), LDA )
               T = A( K, K )
               A( K, K ) = A( P, P )
               A( P, P ) = T
            END IF
*
*           Second swap
*
            KK = K + KSTEP - 1
            IF( KP.NE.KK ) THEN
*
*              Interchange rows and columns KK and KP in the trailing
*              submatrix A(k:n,k:n)
*
               IF( KP.LT.N )
     $            CALL CSWAP( N-KP, A( KP+1, KK ), 1, A( KP+1, KP ), 1 )
               IF( ( KK.LT.N ) .AND. ( KP.GT.(KK+1) ) )
     $            CALL CSWAP( KP-KK-1, A( KK+1, KK ), 1, A( KP, KK+1 ),
     $                     LDA )
               T = A( KK, KK )
               A( KK, KK ) = A( KP, KP )
               A( KP, KP ) = T
               IF( KSTEP.EQ.2 ) THEN
                  T = A( K+1, K )
                  A( K+1, K ) = A( KP, K )
                  A( KP, K ) = T
               END IF
            END IF
*
*           Update the trailing submatrix
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column k now holds
*
*              W(k) = L(k)*D(k)
```

```
*
*             where L(k) is the k-th column of L
*
              IF( K.LT.N ) THEN
*
*             Perform a rank-1 update of A(k+1:n,k+1:n) and
*             store L(k) in column k
*
                IF( CABS1( A( K, K ) ).GE.SFMIN ) THEN
*
*                  Perform a rank-1 update of A(k+1:n,k+1:n) as
*                  A := A - L(k)*D(k)*L(k)**T
*                     = A - W(k)*(1/D(k))*W(k)**T
*
                   D11 = CONE / A( K, K )
                   CALL CSYR( UPLO, N-K, -D11, A( K+1, K ), 1,
     $                        A( K+1, K+1 ), LDA )
*
*                  Store L(k) in column k
*
                   CALL CSCAL( N-K, D11, A( K+1, K ), 1 )
                ELSE
*
*                  Store L(k) in column k
*
                   D11 = A( K, K )
                   DO 46 II = K + 1, N
                      A( II, K ) = A( II, K ) / D11
   46              CONTINUE
*
*                  Perform a rank-1 update of A(k+1:n,k+1:n) as
*                  A := A - L(k)*D(k)*L(k)**T
*                     = A - W(k)*(1/D(k))*W(k)**T
*                     = A - (W(k)/D(k))*(D(k))*(W(k)/D(K))**T
*
                   CALL CSYR( UPLO, N-K, -D11, A( K+1, K ), 1,
     $                        A( K+1, K+1 ), LDA )
                END IF
              END IF
*
            ELSE
*
              2-by-2 pivot block D(k): columns k and k+1 now hold
*
              ( W(k) W(k+1) ) = ( L(k) L(k+1) )*D(k)
*
              where L(k) and L(k+1) are the k-th and (k+1)-th columns
              of L
*
*
```

```
*              Perform a rank-2 update of A(k+2:n,k+2:n) as
*
*              A := A - ( L(k) L(k+1) ) * D(k) * ( L(k) L(k+1) )**T
*                 = A - ( ( A(k)A(k+1) )*inv(D(k) ) * ( A(k)A(k+1) )**T
*
*              and store L(k) and L(k+1) in columns k and k+1
*
               IF( K.LT.N-1 ) THEN
*
                  D21 = A( K+1, K )
                  D11 = A( K+1, K+1 ) / D21
                  D22 = A( K, K ) / D21
                  T = CONE / ( D11*D22-CONE )
*
                  DO 60 J = K + 2, N
*
*                    Compute  D21 * ( W(k)W(k+1) ) * inv(D(k)) for row J
*
                     WK = T*( D11*A( J, K )-A( J, K+1 ) )
                     WKP1 = T*( D22*A( J, K+1 )-A( J, K ) )
*
*                    Perform a rank-2 update of A(k+2:n,k+2:n)
*
                     DO 50 I = J, N
                        A( I, J ) = A( I, J ) - ( A( I, K ) / D21 )*WK -
     $                              ( A( I, K+1 ) / D21 )*WKP1
   50                CONTINUE
*
*                    Store L(k) and L(k+1) in cols k and k+1 for row J
*
                     A( J, K ) = WK / D21
                     A( J, K+1 ) = WKP1 / D21
*
   60             CONTINUE
*
               END IF
*
            END IF
         END IF
*
*        Store details of the interchanges in IPIV
*
         IF( KSTEP.EQ.1 ) THEN
            IPIV( K ) = KP
         ELSE
            IPIV( K ) = -P
            IPIV( K+1 ) = -KP
         END IF
*
*        Increase K and return to the start of the main loop
```

```
*
         K = K + KSTEP
         GO TO 40
*
      END IF
*
   70 CONTINUE
*
      RETURN
*
*     End of CSYTF2_ROOK
*
      END
```

60.15.4 lisp code

— csytf2_rook.lisp —

```
(let* ((zero 0.0)
       (one 1.0)
       (eight 8.0)
       (sevten 17.0)
       (cone (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 8.0 8.0) eight)
           (type (single-float 17.0 17.0) sevten)
           (type (f2cl-lib:complex8) cone)
           (ignorable zero one eight sevten cone))
  (defun csytf2_rook (uplo n a lda ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (labels ((cabs1 (z)
                 (+ (abs (f2cl-lib:freal z)) (abs (f2cl-lib:aimag z)))))
        (declare
         (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
          cabs1))
        (prog ((d11 #C(0.0 0.0)) (d12 #C(0.0 0.0)) (d21 #C(0.0 0.0))
               (d22 #C(0.0 0.0)) (t$ #C(0.0 0.0)) (wk #C(0.0 0.0))
               (wkm1 #C(0.0 0.0)) (wkp1 #C(0.0 0.0)) (z #C(0.0 0.0))
```

```
     (absakk 0.0) (alpha 0.0) (colmax 0.0) (rowmax 0.0) (stemp 0.0)
     (sfmin 0.0) (i 0) (imax 0) (j 0) (jmax 0) (itemp 0) (k 0) (kk 0)
     (kp 0) (kstep 0) (p 0) (ii 0) (upper nil) (done nil))
(declare (type (f2cl-lib:complex8) d11 d12 d21 d22 t$ wk wkm1 wkp1 z)
         (type (single-float) absakk alpha colmax rowmax stemp sfmin)
         (type (f2cl-lib:integer4) i imax j jmax itemp k kk kp kstep
          p ii)
         (type f2cl-lib:logical upper done))
(setf info 0)
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4)))
(cond
 ((/= info 0) (xerbla "CSYTF2_ROOK" (f2cl-lib:int-sub info))
  (go end_label)))
(setf alpha (/ (+ one (f2cl-lib:fsqrt sevten)) eight))
(setf sfmin (slamch "S"))
(cond
 (upper
  (tagbody
    (setf k n)
   label10
    (if (< k 1)
        (go label70))
    (setf kstep 1)
    (setf p k)
    (setf absakk
            (cabs1
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)))
    (cond
     ((> k 1)
      (setf imax
              (icamax (f2cl-lib:int-sub k 1)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
```

```
                                    ((1 lda) (1 *)) a-%offset%)
            1))
  (setf colmax
          (cabs1
           (f2cl-lib:fref a-%data% (imax k) ((1 lda) (1 *))
                          a-%offset%))))
 (t (setf colmax zero)))
(cond
 ((= (max absakk colmax) zero)
  (if (= info 0)
      (setf info k))
  (setf kp k))
 (t
  (cond ((not (< absakk (* alpha colmax))) (setf kp k))
        (t
         (tagbody
           (setf done f2cl-lib:%false%)
          label12
           (cond
            ((/= imax k)
             (setf jmax
                     (f2cl-lib:int-add imax
                                       (multiple-value-bind
                                           (ret-val var-0 var-1
                                            var-2)
                                           (icamax
                                            (f2cl-lib:int-sub k
                                                              imax)
                                            (f2cl-lib:array-slice
                                             a-%data%
                                             f2cl-lib:complex8
                                             (imax
                                              (f2cl-lib:int-add
                                               imax 1))
                                             ((1 lda) (1 *))
                                             a-%offset%)
                                            lda)
                                         (declare
                                          (ignore var-0 var-1))
                                         (when var-2
                                           (setf lda var-2))
                                         ret-val)))
             (setf rowmax
                     (cabs1
                      (f2cl-lib:fref a-%data% (imax jmax)
                                     ((1 lda) (1 *))
                                     a-%offset%))))
            (t (setf rowmax zero)))
           (cond
            ((> imax 1)
```

```
                (setf itemp
                        (icamax (f2cl-lib:int-sub imax 1)
                         (f2cl-lib:array-slice a-%data%
                                               f2cl-lib:complex8
                                               (1 imax)
                                               ((1 lda) (1 *))
                                               a-%offset%)
                         1))
                (setf stemp
                        (cabs1
                         (f2cl-lib:fref a-%data% (itemp imax)
                                        ((1 lda) (1 *))
                                        a-%offset%)))
                (cond
                 ((> stemp rowmax) (setf rowmax stemp)
                  (setf jmax itemp)))))
              (cond
               ((not
                 (<
                  (cabs1
                   (f2cl-lib:fref a (imax imax) ((1 lda) (1 *))))
                  (* alpha rowmax)))
                (setf kp imax) (setf done f2cl-lib:%true%))
               ((or (= p jmax) (<= rowmax colmax)) (setf kp imax)
                (setf kstep 2) (setf done f2cl-lib:%true%))
               (t (setf p imax) (setf colmax rowmax)
                (setf imax jmax)))
              (if (not done)
                  (go label12)))))
(cond
 ((and (= kstep 2) (/= p k))
  (if (> p 1)
      (cswap (f2cl-lib:int-sub p 1)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 p)
                             ((1 lda) (1 *)) a-%offset%)
       1))
  (if (< p (f2cl-lib:int-sub k 1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (cswap (f2cl-lib:int-sub k p 1)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 ((+ p 1) k) ((1 lda) (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 (p (f2cl-lib:int-add p 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
```

```
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf lda var-4))))
  (setf t$
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                       a-%offset%)
          (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                       a-%offset%)
          t$)))
(setf kk (f2cl-lib:int-add (f2cl-lib:int-sub k kstep) 1))
(cond
 ((/= kp kk)
  (if (> kp 1)
      (cswap (f2cl-lib:int-sub kp 1)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 kk)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 kp)
                             ((1 lda) (1 *)) a-%offset%)
       1))
  (if (and (> kk 1) (< kp (f2cl-lib:int-sub kk 1)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (cswap (f2cl-lib:int-sub kk kp 1)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 ((+ kp 1) kk) ((1 lda) (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 (kp (f2cl-lib:int-add kp 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf lda var-4))))
  (setf t$
          (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                       a-%offset%)
          (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                       a-%offset%)
          t$)
  (cond
   ((= kstep 2)
    (setf t$
            (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
```

```
                                ((1 lda) (1 *)) a-%offset%))
            (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                                ((1 lda) (1 *)) a-%offset%)
                    (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                                a-%offset%))
            (setf (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                                a-%offset%)
                    t$)))))
        (cond
         ((= kstep 1)
          (cond
           ((> k 1)
            (cond
             ((>= (cabs1 (f2cl-lib:fref a (k k) ((1 lda) (1 *))))
                  sfmin)
              (setf d11
                      (/ cone
                         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                        a-%offset%)))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (csyr uplo (f2cl-lib:int-sub k 1) (- d11)
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                         (1 k) ((1 lda) (1 *))
                                         a-%offset%)
                   1 a lda)
                (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
                (setf uplo var-0))
              (multiple-value-bind (var-0 var-1 var-2 var-3)
                  (cscal (f2cl-lib:int-sub k 1) d11
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                         (1 k) ((1 lda) (1 *))
                                         a-%offset%)
                   1)
                (declare (ignore var-0 var-2 var-3))
                (when var-1 (setf d11 var-1))))
             (t
              (setf d11
                      (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                     a-%offset%))
              (f2cl-lib:fdo (ii 1 (f2cl-lib:int-add ii 1))
                            ((> ii
                                (f2cl-lib:int-add k
                                                  (f2cl-lib:int-sub
                                                   1)))
                             nil)
                (tagbody
                  (setf (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
                                       a-%offset%)
                        (/
```

```
                    (f2cl-lib:fref a-%data% (ii k)
                                   ((1 lda) (1 *)) a-%offset%)
                    d11))
         label16))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (csyr uplo (f2cl-lib:int-sub k 1) (- d11)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 (1 k) ((1 lda) (1 *))
                                 a-%offset%)
           1 a lda)
        (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
        (setf uplo var-0)))))))
(t
 (cond
  ((> k 2)
   (setf d12
           (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                          ((1 lda) (1 *)) a-%offset%))
   (setf d22
           (/
            (f2cl-lib:fref a-%data%
                           ((f2cl-lib:int-sub k 1)
                            (f2cl-lib:int-sub k 1))
                           ((1 lda) (1 *)) a-%offset%)
            d12))
   (setf d11
           (/
            (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                           a-%offset%)
            d12))
   (setf t$ (/ cone (- (* d11 d22) cone)))
   (f2cl-lib:fdo (j (f2cl-lib:int-add k (f2cl-lib:int-sub 2))
                  (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                 ((> j 1) nil)
     (tagbody
       (setf wkm1
               (* t$
                  (-
                   (* d11
                      (f2cl-lib:fref a-%data%
                                     (j
                                      (f2cl-lib:int-sub k 1))
                                     ((1 lda) (1 *))
                                     a-%offset%))
                   (f2cl-lib:fref a-%data% (j k)
                                  ((1 lda) (1 *))
                                  a-%offset%))))
       (setf wk
               (* t$
```

```
                          (-
                           (* d22
                              (f2cl-lib:fref a-%data% (j k)
                                             ((1 lda) (1 *))
                                             a-%offset%))
                           (f2cl-lib:fref a-%data%
                                          (j (f2cl-lib:int-sub k 1))
                                          ((1 lda) (1 *))
                                          a-%offset%))))
                (f2cl-lib:fdo (i j
                               (f2cl-lib:int-add i
                                                 (f2cl-lib:int-sub 1)))
                              ((> i 1) nil)
                  (tagbody
                    (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                          (-
                           (f2cl-lib:fref a-%data% (i j)
                                          ((1 lda) (1 *)) a-%offset%)
                           (*
                            (/
                             (f2cl-lib:fref a-%data% (i k)
                                            ((1 lda) (1 *))
                                            a-%offset%)
                             d12)
                            wk)
                           (*
                            (/
                             (f2cl-lib:fref a-%data%
                                            (i
                                             (f2cl-lib:int-sub k 1))
                                            ((1 lda) (1 *))
                                            a-%offset%)
                             d12)
                            wkm1)))
                    label20))
                (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                     a-%offset%)
                      (/ wk d12))
                (setf (f2cl-lib:fref a-%data%
                                     (j (f2cl-lib:int-sub k 1))
                                     ((1 lda) (1 *)) a-%offset%)
                      (/ wkm1 d12))
              label30))))))))
        (cond
         ((= kstep 1)
          (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
                kp))
         (t
          (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
```

```
                (f2cl-lib:int-sub p))
        (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                             ((1 *)) ipiv-%offset%)
                (f2cl-lib:int-sub kp))))
      (setf k (f2cl-lib:int-sub k kstep))
      (go label10)))
    (t
     (tagbody
       (setf k 1)
      label40
       (if (> k n)
           (go label70))
       (setf kstep 1)
       (setf p k)
       (setf absakk
               (cabs1
                (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                               a-%offset%)))
       (cond
        ((< k n)
         (setf imax
                 (f2cl-lib:int-add k
                                   (icamax (f2cl-lib:int-sub n k)
                                    (f2cl-lib:array-slice a-%data%
                                                          f2cl-lib:complex8
                                                          ((+ k 1) k)
                                                          ((1 lda)
                                                           (1 *))
                                                          a-%offset%)
                                    1)))
         (setf colmax
                 (cabs1
                  (f2cl-lib:fref a-%data% (imax k) ((1 lda) (1 *))
                                 a-%offset%))))
        (t (setf colmax zero)))
       (cond
        ((= (max absakk colmax) zero)
         (if (= info 0)
             (setf info k))
         (setf kp k))
        (t
         (cond ((not (< absakk (* alpha colmax))) (setf kp k))
               (t
                (tagbody
                  (setf done f2cl-lib:%false%)
                 label42
                  (cond
                   ((/= imax k)
                    (setf jmax
                            (f2cl-lib:int-add (f2cl-lib:int-sub k 1)
```

```
                                (multiple-value-bind
                                    (ret-val var-0 var-1
                                     var-2)
                                    (icamax
                                     (f2cl-lib:int-sub
                                      imax k)
                                     (f2cl-lib:array-slice
                                      a-%data%
                                      f2cl-lib:complex8
                                      (imax k)
                                      ((1 lda) (1 *))
                                      a-%offset%)
                                     lda)
                                  (declare
                                   (ignore var-0 var-1))
                                  (when var-2
                                    (setf lda var-2))
                                  ret-val)))
  (setf rowmax
          (cabs1
           (f2cl-lib:fref a-%data% (imax jmax)
                          ((1 lda) (1 *))
                          a-%offset%))))
 (t (setf rowmax zero)))
(cond
 ((< imax n)
  (setf itemp
          (f2cl-lib:int-add imax
                            (icamax
                             (f2cl-lib:int-sub n imax)
                             (f2cl-lib:array-slice
                              a-%data%
                              f2cl-lib:complex8
                              ((+ imax 1) imax)
                              ((1 lda) (1 *))
                              a-%offset%)
                             1)))
  (setf stemp
          (cabs1
           (f2cl-lib:fref a-%data% (itemp imax)
                          ((1 lda) (1 *))
                          a-%offset%)))
  (cond
   ((> stemp rowmax) (setf rowmax stemp)
    (setf jmax itemp)))))
(cond
 ((not
   (<
    (cabs1
     (f2cl-lib:fref a (imax imax) ((1 lda) (1 *))))
```

```
              (* alpha rowmax)))
            (setf kp imax) (setf done f2cl-lib:%true%))
           ((or (= p jmax) (<= rowmax colmax)) (setf kp imax)
            (setf kstep 2) (setf done f2cl-lib:%true%))
           (t (setf p imax) (setf colmax rowmax)
            (setf imax jmax)))
          (if (not done)
              (go label42)))))
(cond
 ((and (= kstep 2) (/= p k))
  (if (< p n)
      (cswap (f2cl-lib:int-sub n p)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ p 1) k) ((1 lda) (1 *))
                             a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ p 1) p) ((1 lda) (1 *))
                             a-%offset%)
       1))
  (if (> p (f2cl-lib:int-add k 1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (cswap (f2cl-lib:int-sub p k 1)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 ((+ k 1) k) ((1 lda) (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 (p (f2cl-lib:int-add k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf lda var-4))))
  (setf t$
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                       a-%offset%)
          (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (p p) ((1 lda) (1 *))
                       a-%offset%)
          t$)))
(setf kk (f2cl-lib:int-sub (f2cl-lib:int-add k kstep) 1))
(cond
 ((/= kp kk)
  (if (< kp n)
      (cswap (f2cl-lib:int-sub n kp)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ kp 1) kk) ((1 lda) (1 *))
```

```
                                a-%offset%)
          1
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                ((+ kp 1) kp) ((1 lda) (1 *))
                                a-%offset%)
          1))
    (if (and (< kk n) (> kp (f2cl-lib:int-add kk 1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (cswap (f2cl-lib:int-sub kp kk 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   ((+ kk 1) kk) ((1 lda) (1 *))
                                   a-%offset%)
             1
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   (kp (f2cl-lib:int-add kk 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
           (declare (ignore var-0 var-1 var-2 var-3))
           (when var-4 (setf lda var-4))))
    (setf t$
            (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kk kk) ((1 lda) (1 *))
                         a-%offset%)
            (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%)
            t$)
    (cond
     ((= kstep 2)
      (setf t$
              (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                             ((1 lda) (1 *)) a-%offset%))
      (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                           ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                             a-%offset%))
      (setf (f2cl-lib:fref a-%data% (kp k) ((1 lda) (1 *))
                           a-%offset%)
              t$)))))
  (cond
   ((= kstep 1)
    (cond
     ((< k n)
      (cond
       ((>= (cabs1 (f2cl-lib:fref a (k k) ((1 lda) (1 *))))
            sfmin)
        (setf d11
                (/ cone
```

```
            (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                           a-%offset%)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (csyr uplo (f2cl-lib:int-sub n k) (- d11)
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                            ((+ k 1) k) ((1 lda) (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                            ((+ k 1)
                             (f2cl-lib:int-add k 1))
                            ((1 lda) (1 *)) a-%offset%)
      lda)
   (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
   (setf uplo var-0))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (cscal (f2cl-lib:int-sub n k) d11
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                            ((+ k 1) k) ((1 lda) (1 *))
                            a-%offset%)
      1)
   (declare (ignore var-0 var-2 var-3))
   (when var-1 (setf d11 var-1))))
(t
 (setf d11
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%))
 (f2cl-lib:fdo (ii (f2cl-lib:int-add k 1)
                (f2cl-lib:int-add ii 1))
               ((> ii n) nil)
   (tagbody
     (setf (f2cl-lib:fref a-%data% (ii k) ((1 lda) (1 *))
                          a-%offset%)
             (/
              (f2cl-lib:fref a-%data% (ii k)
                             ((1 lda) (1 *)) a-%offset%)
              d11))
    label46))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (csyr uplo (f2cl-lib:int-sub n k) (- d11)
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                            ((+ k 1) k) ((1 lda) (1 *))
                            a-%offset%)
      1
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                            ((+ k 1)
                             (f2cl-lib:int-add k 1))
                            ((1 lda) (1 *)) a-%offset%)
```

```
            lda)
        (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
        (setf uplo var-0)))))))
(t
 (cond
  ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
   (setf d21
           (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                          ((1 lda) (1 *)) a-%offset%))
   (setf d11
           (/
            (f2cl-lib:fref a-%data%
                           ((f2cl-lib:int-add k 1)
                            (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
            d21))
   (setf d22
           (/
            (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                           a-%offset%)
            d21))
   (setf t$ (/ cone (- (* d11 d22) cone)))
   (f2cl-lib:fdo (j (f2cl-lib:int-add k 2)
                  (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf wk
               (* t$
                  (-
                   (* d11
                      (f2cl-lib:fref a-%data% (j k)
                                     ((1 lda) (1 *))
                                     a-%offset%))
                   (f2cl-lib:fref a-%data%
                                  (j (f2cl-lib:int-add k 1))
                                  ((1 lda) (1 *))
                                  a-%offset%))))
       (setf wkp1
               (* t$
                  (-
                   (* d22
                      (f2cl-lib:fref a-%data%
                                     (j
                                      (f2cl-lib:int-add k 1))
                                     ((1 lda) (1 *))
                                     a-%offset%))
                   (f2cl-lib:fref a-%data% (j k)
                                  ((1 lda) (1 *))
                                  a-%offset%))))
       (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
```

```
                                  ((> i n) nil)
                          (tagbody
                            (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                                 a-%offset%)
                                  (-
                                   (f2cl-lib:fref a-%data% (i j)
                                                  ((1 lda) (1 *)) a-%offset%)
                                   (*
                                    (/
                                     (f2cl-lib:fref a-%data% (i k)
                                                    ((1 lda) (1 *))
                                                    a-%offset%)
                                     d21)
                                    wk)
                                   (*
                                    (/
                                     (f2cl-lib:fref a-%data%
                                                    (i
                                                     (f2cl-lib:int-add k 1))
                                                    ((1 lda) (1 *))
                                                    a-%offset%)
                                     d21)
                                    wkp1)))
                             label50))
                        (setf (f2cl-lib:fref a-%data% (j k) ((1 lda) (1 *))
                                             a-%offset%)
                              (/ wk d21))
                        (setf (f2cl-lib:fref a-%data%
                                             (j (f2cl-lib:int-add k 1))
                                             ((1 lda) (1 *)) a-%offset%)
                              (/ wkp1 d21))
                       label60))))))))
                (cond
                 ((= kstep 1)
                  (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
                        kp))
                 (t
                  (setf (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)
                        (f2cl-lib:int-sub p))
                  (setf (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
                                       ((1 *)) ipiv-%offset%)
                        (f2cl-lib:int-sub kp))))
                (setf k (f2cl-lib:int-add k kstep))
                (go label40))))
            label70
             (go end_label)
            end_label
             (return (values uplo nil nil lda nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
```

```
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csytf2_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::csyr))))
```

———-

60.16 csytrf

60.16.1 Axiom unit tests

— csytrf.input —

```
)set break resume
)sys rm -f csytrf.output
)spool csytrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.16.2 Axiom help page

— csytrf.help —

```
====================================================================
csytrf examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csytrf (UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO)
           CSYTRF

Function/Subroutine Documentation
   subroutine csytrf (character UPLO, integer N, complex, dimension( lda, * )
       A, integer LDA, integer, dimension( * ) IPIV, complex, dimension( * )
       WORK, integer LWORK, integer INFO)
       CSYTRF

Purpose:
   CSYTRF computes the factorization of a complex symmetric matrix A
   using the Bunch-Kaufman diagonal pivoting method.  The form of the
   factorization is

   A = U*D*U**T  or  A = L*D*L**T

   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, and D is symmetric and block diagonal with
   with 1-by-1 and 2-by-2 diagonal blocks.

   This is the blocked version of the algorithm, calling Level 3 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   N-by-N upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading N-by-N lower triangular part of A contains the lower
```

```
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, the block diagonal matrix D and the multipliers used
  to obtain the factor U or L (see below for further details).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D.
  If IPIV(k) > 0, then rows and columns k and IPIV(k) were
  interchanged and D(k,k) is a 1-by-1 diagonal block.
  If UPLO = 'U' and IPIV(k) = IPIV(k-1) < 0, then rows and
  columns k-1 and -IPIV(k) were interchanged and D(k-1:k,k-1:k)
  is a 2-by-2 diagonal block.  If UPLO = 'L' and IPIV(k) =
  IPIV(k+1) < 0, then rows and columns k+1 and -IPIV(k) were
  interchanged and D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of WORK.  LWORK >=1.  For best performance
  LWORK >= N*NB, where NB is the block size returned by ILAENV.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, D(i,i) is exactly zero.  The factorization
     has been completed, but the block diagonal matrix D is
     exactly singular, and division by zero will occur if it
     is used to solve a system of equations.

Further Details:

  If UPLO = 'U', then A = U*D*U**T, where
     U = P(n)*U(n)* ... *P(k)U(k)* ...,
  i.e., U is a product of terms P(k)*U(k), where k decreases from n to
  1 in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
```

```
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and U(k) is a unit upper triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                       (   I    v    0   )   k-s
             U(k) =    (   0    I    0   )   s
                       (   0    0    I   )   n-k
                          k-s   s   n-k

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(1:k-1,k).
   If s = 2, the upper triangle of D(k) overwrites A(k-1,k-1), A(k-1,k),
   and A(k,k), and v overwrites A(1:k-2,k-1:k).

   If UPLO = 'L', then A = L*D*L**T, where
      L = P(1)*L(1)* ... *P(k)*L(k)* ...,
   i.e., L is a product of terms P(k)*L(k), where k increases from 1 to
   n in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and L(k) is a unit lower triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                       (   I    0     0   )  k-1
             L(k) =    (   0    I     0   )  s
                       (   0    v     I   )  n-k-s+1
                          k-1   s  n-k-s+1

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(k+1:n,k).
   If s = 2, the lower triangle of D(k) overwrites A(k,k), A(k+1,k),
   and A(k+1,k+1), and v overwrites A(k+2:n,k:k+1).
```

———-

60.16.3 fortran code

— **csytrf.f** —

```
*  =====================================================================
      SUBROUTINE CSYTRF( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LWORK, N
```

```
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY, UPPER
      INTEGER            IINFO, IWS, J, K, KB, LDWORK, LWKOPT, NB, NBMIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLASYF, CSYTF2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
*
      IF( INFO.EQ.0 ) THEN
*
*        Determine the block size
*
         NB = ILAENV( 1, 'CSYTRF', UPLO, N, -1, -1, -1 )
         LWKOPT = N*NB
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
```

```
         CALL XERBLA( 'CSYTRF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
      NBMIN = 2
      LDWORK = N
      IF( NB.GT.1 .AND. NB.LT.N ) THEN
         IWS = LDWORK*NB
         IF( LWORK.LT.IWS ) THEN
            NB = MAX( LWORK / LDWORK, 1 )
            NBMIN = MAX( 2, ILAENV( 2, 'CSYTRF', UPLO, N, -1, -1, -1 ) )
         END IF
      ELSE
         IWS = 1
      END IF
      IF( NB.LT.NBMIN )
     $   NB = N
*
      IF( UPPER ) THEN
*
*        Factorize A as U*D*U**T using the upper triangle of A
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        KB, where KB is the number of columns factorized by CLASYF;
*        KB is either NB or NB-1, or K for the last block
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop
*
         IF( K.LT.1 )
     $      GO TO 40
*
         IF( K.GT.NB ) THEN
*
*           Factorize columns k-kb+1:k of A and use blocked code to
*           update columns 1:k-kb
*
            CALL CLASYF( UPLO, K, NB, KB, A, LDA, IPIV, WORK, N, IINFO )
         ELSE
*
*           Use unblocked code to factorize columns 1:k of A
*
            CALL CSYTF2( UPLO, K, A, LDA, IPIV, IINFO )
            KB = K
         END IF
*
```

```
*        Set INFO on the first occurrence of a zero pivot
*
         IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO
*
*        Decrease K and return to the start of the main loop
*
         K = K - KB
         GO TO 10
*
      ELSE
*
*        Factorize A as L*D*L**T using the lower triangle of A
*
*        K is the main loop index, increasing from 1 to N in steps of
*        KB, where KB is the number of columns factorized by CLASYF;
*        KB is either NB or NB-1, or N-K+1 for the last block
*
         K = 1
   20    CONTINUE
*
*        If K > N, exit from loop
*
         IF( K.GT.N )
     $      GO TO 40
*
         IF( K.LE.N-NB ) THEN
*
*           Factorize columns k:k+kb-1 of A and use blocked code to
*           update columns k+kb:n
*
            CALL CLASYF( UPLO, N-K+1, NB, KB, A( K, K ), LDA, IPIV( K ),
     $                   WORK, N, IINFO )
         ELSE
*
*           Use unblocked code to factorize columns k:n of A
*
            CALL CSYTF2( UPLO, N-K+1, A( K, K ), LDA, IPIV( K ), IINFO )
            KB = N - K + 1
         END IF
*
*        Set INFO on the first occurrence of a zero pivot
*
         IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO + K - 1
*
*        Adjust IPIV
*
         DO 30 J = K, K + KB - 1
            IF( IPIV( J ).GT.0 ) THEN
```

```
               IPIV( J ) = IPIV( J ) + K - 1
            ELSE
               IPIV( J ) = IPIV( J ) - K + 1
            END IF
   30    CONTINUE
*
*        Increase K and return to the start of the main loop
*
         K = K + KB
         GO TO 20
*
      END IF
*
   40 CONTINUE
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of CSYTRF
*
      END
```

60.16.4 lisp code

— csytrf.lisp —

```
(defun csytrf (uplo n a lda ipiv work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex8 (*)) work a)
           (type (f2cl-lib:integer4) info lwork lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((iinfo 0) (iws 0) (j 0) (k 0) (kb 0) (ldwork 0) (lwkopt 0) (nb 0)
           (nbmin 0) (lquery nil) (upper nil))
      (declare (type f2cl-lib:logical upper lquery)
               (type (f2cl-lib:integer4) nbmin nb lwkopt ldwork kb k j iws
                iinfo))
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
```

```
          ret-val))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4))
 ((and (< lwork 1) (not lquery)) (setf info -7)))
(cond
 ((= info 0)
  (setf nb
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (ilaenv 1 "CSYTRF" uplo n -1 -1 -1)
            (declare (ignore var-0 var-1 var-4 var-5 var-6))
            (when var-2 (setf uplo var-2))
            (when var-3 (setf n var-3))
            ret-val))
  (setf lwkopt (f2cl-lib:int-mul n nb))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwkopt 'f2cl-lib:complex8))))
(cond
 ((/= info 0) (xerbla "CSYTRF" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(setf nbmin 2)
(setf ldwork n)
(cond
 ((and (> nb 1) (< nb n)) (setf iws (f2cl-lib:int-mul ldwork nb))
  (cond
   ((< lwork iws)
    (setf nb (max (the f2cl-lib:integer4 (truncate lwork ldwork)) 1))
    (setf nbmin
            (max (the f2cl-lib:integer4 2)
                 (the f2cl-lib:integer4
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
                          (ilaenv 2 "CSYTRF" uplo n -1 -1 -1)
                        (declare (ignore var-0 var-1 var-4 var-5 var-6))
                        (when var-2 (setf uplo var-2))
                        (when var-3 (setf n var-3))
                        ret-val)))))))
 (t (setf iws 1)))
```

```
(if (< nb nbmin)
    (setf nb n))
(cond
 (upper
  (tagbody
    (setf k n)
   label10
    (if (< k 1)
        (go label40))
    (cond
     ((> k nb)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (clasyf uplo k nb kb a lda ipiv work n iinfo)
        (declare (ignore var-1 var-2 var-4 var-6 var-7))
        (setf uplo var-0)
        (setf kb var-3)
        (setf lda var-5)
        (setf n var-8)
        (setf iinfo var-9)))
     (t
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (csytf2 uplo k a lda ipiv iinfo)
        (declare (ignore var-1 var-2 var-4))
        (setf uplo var-0)
        (setf lda var-3)
        (setf iinfo var-5))
      (setf kb k)))
    (if (and (= info 0) (> iinfo 0))
        (setf info iinfo))
    (setf k (f2cl-lib:int-sub k kb))
    (go label10)))
 (t
  (tagbody
    (setf k 1)
   label20
    (if (> k n)
        (go label40))
    (cond
     ((<= k (f2cl-lib:int-add n (f2cl-lib:int-sub nb)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (clasyf uplo (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1) nb kb
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (k k)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (k)
                                 ((1 *)) ipiv-%offset%)
           work n iinfo)
        (declare (ignore var-1 var-2 var-4 var-6 var-7))
```

```
              (setf uplo var-0)
              (setf kb var-3)
              (setf lda var-5)
              (setf n var-8)
              (setf iinfo var-9)))
            (t
             (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                 (csytf2 uplo (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
                  (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (k k)
                                        ((1 lda) (1 *)) a-%offset%)
                  lda
                  (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (k)
                                        ((1 *)) ipiv-%offset%)
                  iinfo)
              (declare (ignore var-1 var-2 var-4))
              (setf uplo var-0)
              (setf lda var-3)
              (setf iinfo var-5))
             (setf kb (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1))))
           (if (and (= info 0) (> iinfo 0))
              (setf info (f2cl-lib:int-sub (f2cl-lib:int-add iinfo k) 1)))
           (f2cl-lib:fdo (j k (f2cl-lib:int-add j 1))
                         ((> j (f2cl-lib:int-add k kb (f2cl-lib:int-sub 1))) nil)
             (tagbody
              (cond
               ((> (f2cl-lib:fref ipiv (j) ((1 *))) 0)
                (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                        (f2cl-lib:int-sub
                         (f2cl-lib:int-add
                          (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                          k)
                         1)))
               (t
                (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                        (f2cl-lib:int-add
                         (f2cl-lib:int-sub
                          (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                          k)
                         1))))
              label30))
           (setf k (f2cl-lib:int-add k kb))
           (go label20))))
      label40
       (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
               (coerce lwkopt 'f2cl-lib:complex8))
       (go end_label)
      end_label
       (return (values uplo n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
```

```
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csytrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::csytf2
                                              fortran-to-lisp::clasyf))))
```

———-

60.17 csytrf_rook

60.17.1 Axiom unit tests

— **csytrfrook.input** —

```
)set break resume
)sys rm -f csytrfrook.output
)spool csytrfrook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.17.2 Axiom help page

— csytrf_rook.help —

```
=====================================================================
csytrf_rook examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csytrf_rook (UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO)
           CSYTRF_ROOK

Function/Subroutine Documentation
   subroutine csytrf_rook (character UPLO, integer N, complex, dimension( lda,
       * ) A, integer LDA, integer, dimension( * ) IPIV, complex, dimension( *
       ) WORK, integer LWORK, integer INFO)
       CSYTRF_ROOK

Purpose:
   CSYTRF_ROOK computes the factorization of a complex symmetric matrix A
   using the bounded Bunch-Kaufman ("rook") diagonal pivoting method.
   The form of the factorization is

   A = U*D*U**T  or  A = L*D*L**T

   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, and D is symmetric and block diagonal with
   1-by-1 and 2-by-2 diagonal blocks.

   This is the blocked version of the algorithm, calling Level 3 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
```

```
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, the block diagonal matrix D and the multipliers used
  to obtain the factor U or L (see below for further details).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D.

  If UPLO = 'U':
    If IPIV(k) > 0, then rows and columns k and IPIV(k)
    were interchanged and D(k,k) is a 1-by-1 diagonal block.

    If IPIV(k) < 0 and IPIV(k-1) < 0, then rows and
    columns k and -IPIV(k) were interchanged and rows and
    columns k-1 and -IPIV(k-1) were inerchaged,
    D(k-1:k,k-1:k) is a 2-by-2 diagonal block.

  If UPLO = 'L':
    If IPIV(k) > 0, then rows and columns k and IPIV(k)
    were interchanged and D(k,k) is a 1-by-1 diagonal block.

    If IPIV(k) < 0 and IPIV(k+1) < 0, then rows and
    columns k and -IPIV(k) were interchanged and rows and
    columns k+1 and -IPIV(k+1) were inerchaged,
    D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK)).
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of WORK.  LWORK >=1.  For best performance
  LWORK >= N*NB, where NB is the block size returned by ILAENV.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.
```

```
 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, D(i,i) is exactly zero.  The factorization
      has been completed, but the block diagonal matrix D is
      exactly singular, and division by zero will occur if it
      is used to solve a system of equations.

Further Details:

   If UPLO = 'U', then A = U*D*U**T, where
      U = P(n)*U(n)* ... *P(k)U(k)* ...,
   i.e., U is a product of terms P(k)*U(k), where k decreases from n to
   1 in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and U(k) is a unit upper triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                        (   I    v    0   )   k-s
                U(k) =  (   0    I    0   )   s
                        (   0    0    I   )   n-k
                           k-s   s   n-k

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(1:k-1,k).
   If s = 2, the upper triangle of D(k) overwrites A(k-1,k-1), A(k-1,k),
   and A(k,k), and v overwrites A(1:k-2,k-1:k).

   If UPLO = 'L', then A = L*D*L**T, where
      L = P(1)*L(1)* ... *P(k)*L(k)* ...,
   i.e., L is a product of terms P(k)*L(k), where k increases from 1 to
   n in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and L(k) is a unit lower triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                        (   I    0     0   )  k-1
                L(k) =  (   0    I     0   )  s
                        (   0    v     I   )  n-k-s+1
                           k-1   s  n-k-s+1

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(k+1:n,k).
   If s = 2, the lower triangle of D(k) overwrites A(k,k), A(k+1,k),
   and A(k+1,k+1), and v overwrites A(k+2:n,k:k+1).
```

60.17.3 fortran code

— csytrf_rook.f —

```
*  =====================================================================
      SUBROUTINE CSYTRF_ROOK( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY, UPPER
      INTEGER            IINFO, IWS, J, K, KB, LDWORK, LWKOPT, NB, NBMIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLASYF_ROOK, CSYTF2_ROOK, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
```

```
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
*
      IF( INFO.EQ.0 ) THEN
*
*        Determine the block size
*
         NB = ILAENV( 1, 'CSYTRF_ROOK', UPLO, N, -1, -1, -1 )
         LWKOPT = N*NB
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CSYTRF_ROOK', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
      NBMIN = 2
      LDWORK = N
      IF( NB.GT.1 .AND. NB.LT.N ) THEN
         IWS = LDWORK*NB
         IF( LWORK.LT.IWS ) THEN
            NB = MAX( LWORK / LDWORK, 1 )
            NBMIN = MAX( 2, ILAENV( 2, 'CSYTRF_ROOK',
     $                              UPLO, N, -1, -1, -1 ) )
         END IF
      ELSE
         IWS = 1
      END IF
      IF( NB.LT.NBMIN )
     $   NB = N
*
      IF( UPPER ) THEN
*
*        Factorize A as U*D*U**T using the upper triangle of A
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        KB, where KB is the number of columns factorized by CLASYF_ROOK;
*        KB is either NB or NB-1, or K for the last block
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop
*
```

```
         IF( K.LT.1 )
     $      GO TO 40
*
         IF( K.GT.NB ) THEN
*
*           Factorize columns k-kb+1:k of A and use blocked code to
*           update columns 1:k-kb
*
            CALL CLASYF_ROOK( UPLO, K, NB, KB, A, LDA,
     $                        IPIV, WORK, LDWORK, IINFO )
         ELSE
*
*           Use unblocked code to factorize columns 1:k of A
*
            CALL CSYTF2_ROOK( UPLO, K, A, LDA, IPIV, IINFO )
            KB = K
         END IF
*
*        Set INFO on the first occurrence of a zero pivot
*
         IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO
*
*        No need to adjust IPIV
*
*        Decrease K and return to the start of the main loop
*
         K = K - KB
         GO TO 10
*
      ELSE
*
*        Factorize A as L*D*L**T using the lower triangle of A
*
*        K is the main loop index, increasing from 1 to N in steps of
*        KB, where KB is the number of columns factorized by CLASYF_ROOK;
*        KB is either NB or NB-1, or N-K+1 for the last block
*
         K = 1
   20    CONTINUE
*
*        If K > N, exit from loop
*
         IF( K.GT.N )
     $      GO TO 40
*
         IF( K.LE.N-NB ) THEN
*
*           Factorize columns k:k+kb-1 of A and use blocked code to
*           update columns k+kb:n
```

```
*
           CALL CLASYF_ROOK( UPLO, N-K+1, NB, KB, A( K, K ), LDA,
     $                       IPIV( K ), WORK, LDWORK, IINFO )
        ELSE
*
*          Use unblocked code to factorize columns k:n of A
*
           CALL CSYTF2_ROOK( UPLO, N-K+1, A( K, K ), LDA, IPIV( K ),
     $                  IINFO )
           KB = N - K + 1
        END IF
*
*       Set INFO on the first occurrence of a zero pivot
*
        IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $     INFO = IINFO + K - 1
*
*       Adjust IPIV
*
        DO 30 J = K, K + KB - 1
           IF( IPIV( J ).GT.0 ) THEN
              IPIV( J ) = IPIV( J ) + K - 1
           ELSE
              IPIV( J ) = IPIV( J ) - K + 1
           END IF
   30   CONTINUE
*
*       Increase K and return to the start of the main loop
*
        K = K + KB
        GO TO 20
*
      END IF
*
   40 CONTINUE
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of CSYTRF_ROOK
*
      END
```

———-

60.17.4 lisp code

— csytrf_rook.lisp —

```
(defun csytrf_rook (uplo n a lda ipiv work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex8 (*)) work a)
           (type (f2cl-lib:integer4) info lwork lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((iinfo 0) (iws 0) (j 0) (k 0) (kb 0) (ldwork 0) (lwkopt 0) (nb 0)
           (nbmin 0) (lquery nil) (upper nil))
      (declare (type f2cl-lib:logical upper lquery)
               (type (f2cl-lib:integer4) nbmin nb lwkopt ldwork kb k j iws
                iinfo))
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond
       ((and (not upper)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -4))
       ((and (< lwork 1) (not lquery)) (setf info -7)))
      (cond
       ((= info 0)
        (setf nb
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (ilaenv 1 "CSYTRF_ROOK" uplo n -1 -1 -1)
                  (declare (ignore var-0 var-1 var-4 var-5 var-6))
                  (when var-2 (setf uplo var-2))
                  (when var-3 (setf n var-3))
                  ret-val))
        (setf lwkopt (f2cl-lib:int-mul n nb))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce lwkopt 'f2cl-lib:complex8))))
      (cond
```

```
 ((/= info 0) (xerbla "CSYTRF_ROOK" (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(setf nbmin 2)
(setf ldwork n)
(cond
 ((and (> nb 1) (< nb n)) (setf iws (f2cl-lib:int-mul ldwork nb))
  (cond
   ((< lwork iws)
    (setf nb (max (the f2cl-lib:integer4 (truncate lwork ldwork)) 1))
    (setf nbmin
            (max (the f2cl-lib:integer4 2)
                 (the f2cl-lib:integer4
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
                          (ilaenv 2 "CSYTRF_ROOK" uplo n -1 -1 -1)
                        (declare (ignore var-0 var-1 var-4 var-5 var-6))
                        (when var-2 (setf uplo var-2))
                        (when var-3 (setf n var-3))
                        ret-val)))))))
 (t (setf iws 1)))
(if (< nb nbmin)
    (setf nb n))
(cond
 (upper
  (tagbody
    (setf k n)
   label10
    (if (< k 1)
        (go label40))
    (cond
     ((> k nb)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (clasyf_rook uplo k nb kb a lda ipiv work ldwork iinfo)
        (declare (ignore var-1 var-2 var-4 var-6 var-7))
        (setf uplo var-0)
        (setf kb var-3)
        (setf lda var-5)
        (setf ldwork var-8)
        (setf iinfo var-9)))
     (t
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (csytf2_rook uplo k a lda ipiv iinfo)
        (declare (ignore var-1 var-2 var-4))
        (setf uplo var-0)
        (setf lda var-3)
        (setf iinfo var-5))
      (setf kb k)))
```

```
      (if (and (= info 0) (> iinfo 0))
          (setf info iinfo))
      (setf k (f2cl-lib:int-sub k kb))
      (go label10)))
    (t
     (tagbody
      (setf k 1)
     label20
      (if (> k n)
          (go label40))
      (cond
       ((<= k (f2cl-lib:int-add n (f2cl-lib:int-sub nb)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
            (clasyf_rook uplo (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
             nb kb
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (k k)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (k)
                                   ((1 *)) ipiv-%offset%)
             work ldwork iinfo)
          (declare (ignore var-1 var-2 var-4 var-6 var-7))
          (setf uplo var-0)
          (setf kb var-3)
          (setf lda var-5)
          (setf ldwork var-8)
          (setf iinfo var-9)))
       (t
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (csytf2_rook uplo (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (k k)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (k)
                                   ((1 *)) ipiv-%offset%)
             iinfo)
          (declare (ignore var-1 var-2 var-4))
          (setf uplo var-0)
          (setf lda var-3)
          (setf iinfo var-5))
        (setf kb (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1))))
      (if (and (= info 0) (> iinfo 0))
          (setf info (f2cl-lib:int-sub (f2cl-lib:int-add iinfo k) 1)))
      (f2cl-lib:fdo (j k (f2cl-lib:int-add j 1))
                    ((> j (f2cl-lib:int-add k kb (f2cl-lib:int-sub 1))) nil)
        (tagbody
          (cond
           ((> (f2cl-lib:fref ipiv (j) ((1 *))) 0)
            (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
```

```
                        (f2cl-lib:int-sub
                         (f2cl-lib:int-add
                          (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                          k)
                         1)))
                (t
                 (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                        (f2cl-lib:int-add
                         (f2cl-lib:int-sub
                          (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                          k)
                         1))))
            label30))
          (setf k (f2cl-lib:int-add k kb))
          (go label20))))
     label40
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce lwkopt 'f2cl-lib:complex8))
      (go end_label)
     end_label
      (return (values uplo n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csytrf_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::csytf2_rook
                                              fortran-to-lisp::clasyf_rook))))
```

———-

60.18 csytri

60.18.1 Axiom unit tests

— **csytri.input** —

```
)set break resume
)sys rm -f csytri.output
)spool csytri.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.18.2 Axiom help page

— **csytri.help** —

```
======================================================================
csytri examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csytri (UPLO, N, A, LDA, IPIV, WORK, INFO)
           CSYTRI

Function/Subroutine Documentation
   subroutine csytri (character UPLO, integer N, complex, dimension( lda, * )
       A, integer LDA, integer, dimension( * ) IPIV, complex, dimension( * )
       WORK, integer INFO)
       CSYTRI

Purpose:
```

```
  CSYTRI computes the inverse of a complex symmetric indefinite matrix
  A using the factorization A = U*D*U**T or A = L*D*L**T computed by
  CSYTRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the block diagonal matrix D and the multipliers
   used to obtain the factor U or L as computed by CSYTRF.

   On exit, if INFO = 0, the (symmetric) inverse of the original
   matrix.  If UPLO = 'U', the upper triangular part of the
   inverse is formed and the part of A below the diagonal is not
   referenced; if UPLO = 'L' the lower triangular part of the
   inverse is formed and the part of A above the diagonal is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by CSYTRF.

 WORK
   WORK is COMPLEX array, dimension (2*N)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value
   > 0: if INFO = i, D(i,i) = 0; the matrix is singular and its
        inverse could not be computed.
```

60.18.3 fortran code

— csytri.f —

```
*  =====================================================================
      SUBROUTINE CSYTRI( UPLO, N, A, LDA, IPIV, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ONE, ZERO
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ),
     $                   ZERO = ( 0.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            K, KP, KSTEP
      COMPLEX            AK, AKKP1, AKP1, D, T, TEMP
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      COMPLEX            CDOTU
      EXTERNAL           LSAME, CDOTU
*     ..
*     .. External Subroutines ..
      EXTERNAL           CCOPY, CSWAP, CSYMV, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
```

```
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CSYTRI', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
         DO 10 INFO = N, 1, -1
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
   10    CONTINUE
      ELSE
*
*        Lower triangular storage: examine D from top to bottom.
*
         DO 20 INFO = 1, N
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
   20    CONTINUE
      END IF
      INFO = 0
*
      IF( UPPER ) THEN
*
*        Compute inv(A) from the factorization A = U*D*U**T.
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = 1
   30    CONTINUE
*
*        If K > N, exit from loop.
```

```
*
         IF( K.GT.N )
     $      GO TO 40
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Invert the diagonal block.
*
            A( K, K ) = ONE / A( K, K )
*
*           Compute column K of the inverse.
*
            IF( K.GT.1 ) THEN
               CALL CCOPY( K-1, A( 1, K ), 1, WORK, 1 )
               CALL CSYMV( UPLO, K-1, -ONE, A, LDA, WORK, 1, ZERO,
     $                     A( 1, K ), 1 )
               A( K, K ) = A( K, K ) - CDOTU( K-1, WORK, 1, A( 1, K ),
     $                     1 )
            END IF
            KSTEP = 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Invert the diagonal block.
*
            T = A( K, K+1 )
            AK = A( K, K ) / T
            AKP1 = A( K+1, K+1 ) / T
            AKKP1 = A( K, K+1 ) / T
            D = T*( AK*AKP1-ONE )
            A( K, K ) = AKP1 / D
            A( K+1, K+1 ) = AK / D
            A( K, K+1 ) = -AKKP1 / D
*
*           Compute columns K and K+1 of the inverse.
*
            IF( K.GT.1 ) THEN
               CALL CCOPY( K-1, A( 1, K ), 1, WORK, 1 )
               CALL CSYMV( UPLO, K-1, -ONE, A, LDA, WORK, 1, ZERO,
     $                     A( 1, K ), 1 )
               A( K, K ) = A( K, K ) - CDOTU( K-1, WORK, 1, A( 1, K ),
     $                     1 )
               A( K, K+1 ) = A( K, K+1 ) -
     $                       CDOTU( K-1, A( 1, K ), 1, A( 1, K+1 ), 1 )
               CALL CCOPY( K-1, A( 1, K+1 ), 1, WORK, 1 )
               CALL CSYMV( UPLO, K-1, -ONE, A, LDA, WORK, 1, ZERO,
     $                     A( 1, K+1 ), 1 )
```

```
               A( K+1, K+1 ) = A( K+1, K+1 ) -
     $                         CDOTU( K-1, WORK, 1, A( 1, K+1 ), 1 )
            END IF
            KSTEP = 2
         END IF
*
         KP = ABS( IPIV( K ) )
         IF( KP.NE.K ) THEN
*
*           Interchange rows and columns K and KP in the leading
*           submatrix A(1:k+1,1:k+1)
*
            CALL CSWAP( KP-1, A( 1, K ), 1, A( 1, KP ), 1 )
            CALL CSWAP( K-KP-1, A( KP+1, K ), 1, A( KP, KP+1 ), LDA )
            TEMP = A( K, K )
            A( K, K ) = A( KP, KP )
            A( KP, KP ) = TEMP
            IF( KSTEP.EQ.2 ) THEN
               TEMP = A( K, K+1 )
               A( K, K+1 ) = A( KP, K+1 )
               A( KP, K+1 ) = TEMP
            END IF
         END IF
*
         K = K + KSTEP
         GO TO 30
   40    CONTINUE
*
      ELSE
*
         Compute inv(A) from the factorization A = L*D*L**T.
*
         K is the main loop index, increasing from 1 to N in steps of
         1 or 2, depending on the size of the diagonal blocks.
*
         K = N
   50    CONTINUE
*
         If K < 1, exit from loop.
*
         IF( K.LT.1 )
     $      GO TO 60
*
         IF( IPIV( K ).GT.0 ) THEN
*
            1 x 1 diagonal block
*
            Invert the diagonal block.
*
            A( K, K ) = ONE / A( K, K )
```

```
*
*          Compute column K of the inverse.
*
           IF( K.LT.N ) THEN
              CALL CCOPY( N-K, A( K+1, K ), 1, WORK, 1 )
              CALL CSYMV( UPLO, N-K, -ONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                    ZERO, A( K+1, K ), 1 )
              A( K, K ) = A( K, K ) - CDOTU( N-K, WORK, 1, A( K+1, K ),
     $                    1 )
           END IF
           KSTEP = 1
        ELSE
*
*          2 x 2 diagonal block
*
*          Invert the diagonal block.
*
           T = A( K, K-1 )
           AK = A( K-1, K-1 ) / T
           AKP1 = A( K, K ) / T
           AKKP1 = A( K, K-1 ) / T
           D = T*( AK*AKP1-ONE )
           A( K-1, K-1 ) = AKP1 / D
           A( K, K ) = AK / D
           A( K, K-1 ) = -AKKP1 / D
*
*          Compute columns K-1 and K of the inverse.
*
           IF( K.LT.N ) THEN
              CALL CCOPY( N-K, A( K+1, K ), 1, WORK, 1 )
              CALL CSYMV( UPLO, N-K, -ONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                    ZERO, A( K+1, K ), 1 )
              A( K, K ) = A( K, K ) - CDOTU( N-K, WORK, 1, A( K+1, K ),
     $                    1 )
              A( K, K-1 ) = A( K, K-1 ) -
     $                      CDOTU( N-K, A( K+1, K ), 1, A( K+1, K-1 ),
     $                      1 )
              CALL CCOPY( N-K, A( K+1, K-1 ), 1, WORK, 1 )
              CALL CSYMV( UPLO, N-K, -ONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                    ZERO, A( K+1, K-1 ), 1 )
              A( K-1, K-1 ) = A( K-1, K-1 ) -
     $                        CDOTU( N-K, WORK, 1, A( K+1, K-1 ), 1 )
           END IF
           KSTEP = 2
        END IF
*
        KP = ABS( IPIV( K ) )
        IF( KP.NE.K ) THEN
*
*          Interchange rows and columns K and KP in the trailing
```

```
*           submatrix A(k-1:n,k-1:n)
*
            IF( KP.LT.N )
     $         CALL CSWAP( N-KP, A( KP+1, K ), 1, A( KP+1, KP ), 1 )
            CALL CSWAP( KP-K-1, A( K+1, K ), 1, A( KP, K+1 ), LDA )
            TEMP = A( K, K )
            A( K, K ) = A( KP, KP )
            A( KP, KP ) = TEMP
            IF( KSTEP.EQ.2 ) THEN
               TEMP = A( K, K-1 )
               A( K, K-1 ) = A( KP, K-1 )
               A( KP, K-1 ) = TEMP
            END IF
         END IF
*
         K = K - KSTEP
         GO TO 50
   60    CONTINUE
      END IF
*
      RETURN
*
*     End of CSYTRI
*
      END
```

———-

60.18.4 lisp code

— **csytri.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)) (zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (ignorable one zero))
  (defun csytri (uplo n a lda ipiv work info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((ak #C(0.0 0.0)) (akkp1 #C(0.0 0.0)) (akp1 #C(0.0 0.0))
             (d #C(0.0 0.0)) (t$ #C(0.0 0.0)) (temp #C(0.0 0.0)) (k 0) (kp 0)
```

```
     (kstep 0) (upper nil))
(declare (type (f2cl-lib:complex8) ak akkp1 akp1 d t$ temp)
         (type (f2cl-lib:integer4) k kp kstep)
         (type f2cl-lib:logical upper))
(setf info 0)
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4)))
(cond
 ((/= info 0) (xerbla "CSYTRI" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(cond
 (upper
  (f2cl-lib:fdo (info n (f2cl-lib:int-add info (f2cl-lib:int-sub 1)))
                ((> info 1) nil)
    (tagbody
      (if (and
           (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
              0)
           (=
            (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                           a-%offset%)
            zero))
          (go end_label))
     label10)))
 (t
  (f2cl-lib:fdo (info 1 (f2cl-lib:int-add info 1))
                ((> info n) nil)
    (tagbody
      (if (and
           (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
              0)
           (=
```

```
                (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                               a-%offset%)
                zero))
              (go end_label))
          label20))))
     (setf info 0)
     (cond
      (upper
       (tagbody
         (setf k 1)
        label30
         (if (> k n)
             (go label40))
         (cond
          ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
           (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                   (/ one
                      (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                     a-%offset%)))
           (cond
            ((> k 1)
             (ccopy (f2cl-lib:int-sub k 1)
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                                    ((1 lda) (1 *)) a-%offset%)
              1 work 1)
             (multiple-value-bind
                 (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                  var-9)
                 (csymv uplo (f2cl-lib:int-sub k 1) (- one) a lda work 1
                  zero
                  (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                                        ((1 lda) (1 *)) a-%offset%)
                  1)
               (declare
                (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9))
               (setf uplo var-0))
             (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                     (-
                      (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                     a-%offset%)
                      (cdotu (f2cl-lib:int-sub k 1) work 1
                       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                             (1 k) ((1 lda) (1 *))
                                             a-%offset%)
                       1)))))
           (setf kstep 1))
          (t
           (setf t$
                   (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
```

```
                        ((1 lda) (1 *)) a-%offset%))
(setf ak
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         t$))
(setf akp1
        (/
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-add k 1)
                         (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
(setf akkp1
        (/
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
(setf d (* t$ (- (* ak akp1) one)))
(setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (/ akp1 d))
(setf (f2cl-lib:fref a-%data%
                     ((f2cl-lib:int-add k 1)
                      (f2cl-lib:int-add k 1))
                     ((1 lda) (1 *)) a-%offset%)
        (/ ak d))
(setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                     ((1 lda) (1 *)) a-%offset%)
        (/ (- akkp1) d))
(cond
 ((> k 1)
  (ccopy (f2cl-lib:int-sub k 1)
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                         ((1 lda) (1 *)) a-%offset%)
   1 work 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (csymv uplo (f2cl-lib:int-sub k 1) (- one) a lda work 1
       zero
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1)
    (declare
     (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9))
    (setf uplo var-0))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
          (-
           (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
```

```
                          a-%offset%)
          (cdotu (f2cl-lib:int-sub k 1) work 1
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 (1 k) ((1 lda) (1 *))
                                 a-%offset%)
           1)))
(setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                     ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
         (cdotu (f2cl-lib:int-sub k 1)
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                (1 k) ((1 lda) (1 *))
                                a-%offset%)
          1
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                (1 (f2cl-lib:int-add k 1))
                                ((1 lda) (1 *)) a-%offset%)
          1)))
(ccopy (f2cl-lib:int-sub k 1)
 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                       (1 (f2cl-lib:int-add k 1))
                       ((1 lda) (1 *)) a-%offset%)
 1 work 1)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (csymv uplo (f2cl-lib:int-sub k 1) (- one) a lda work 1
     zero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                           (1 (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
    var-9))
  (setf uplo var-0))
(setf (f2cl-lib:fref a-%data%
                     ((f2cl-lib:int-add k 1)
                      (f2cl-lib:int-add k 1))
                     ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-add k 1)
                         (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
         (cdotu (f2cl-lib:int-sub k 1) work 1
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                (1 (f2cl-lib:int-add k 1))
```

```
                                        ((1 lda) (1 *)) a-%offset%)
                 1)))))
      (setf kstep 2)))
    (setf kp
            (abs
             (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
    (cond
     ((/= kp k)
      (cswap (f2cl-lib:int-sub kp 1)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 kp)
                             ((1 lda) (1 *)) a-%offset%)
       1)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (cswap (f2cl-lib:int-sub k kp 1)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 ((+ kp 1) k) ((1 lda) (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 (kp (f2cl-lib:int-add kp 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf lda var-4)))
      (setf temp
              (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                             a-%offset%))
      (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                             a-%offset%))
      (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *)) a-%offset%)
              temp)
      (cond
       ((= kstep 2)
        (setf temp
                (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%))
        (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%))
        (setf (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
                temp)))))
    (setf k (f2cl-lib:int-add k kstep))
    (go label30)
   label40))
```

```
(t
 (tagbody
   (setf k n)
  label50
   (if (< k 1)
       (go label60))
   (cond
    ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
     (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
             (/ one
                (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                               a-%offset%)))
     (cond
      ((< k n)
       (ccopy (f2cl-lib:int-sub n k)
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 ((+ k 1) k)
                              ((1 lda) (1 *)) a-%offset%)
        1 work 1)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
            var-9)
           (csymv uplo (f2cl-lib:int-sub n k) (- one)
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                  ((+ k 1) (f2cl-lib:int-add k 1))
                                  ((1 lda) (1 *)) a-%offset%)
            lda work 1 zero
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                  ((+ k 1) k) ((1 lda) (1 *))
                                  a-%offset%)
            1)
         (declare
          (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9))
         (setf uplo var-0))
       (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
               (-
                (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                               a-%offset%)
                (cdotu (f2cl-lib:int-sub n k) work 1
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                       ((+ k 1) k) ((1 lda) (1 *))
                                       a-%offset%)
                 1)))))
     (setf kstep 1))
    (t
     (setf t$
             (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                            ((1 lda) (1 *)) a-%offset%))
     (setf ak
             (/
```

```
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-sub k 1)
                         (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
(setf akp1
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         t$))
(setf akkp1
        (/
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
(setf d (* t$ (- (* ak akp1) one)))
(setf (f2cl-lib:fref a-%data%
                     ((f2cl-lib:int-sub k 1)
                      (f2cl-lib:int-sub k 1))
                     ((1 lda) (1 *)) a-%offset%)
        (/ akp1 d))
(setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (/ ak d))
(setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                     ((1 lda) (1 *)) a-%offset%)
        (/ (- akkp1) d))
(cond
 ((< k n)
  (ccopy (f2cl-lib:int-sub n k)
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 ((+ k 1) k)
                         ((1 lda) (1 *)) a-%offset%)
   1 work 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (csymv uplo (f2cl-lib:int-sub n k) (- one)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ k 1) (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda work 1 zero
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ k 1) k) ((1 lda) (1 *))
                             a-%offset%)
       1)
    (declare
     (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9))
    (setf uplo var-0))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (-
```

```
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%)
          (cdotu (f2cl-lib:int-sub n k) work 1
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 ((+ k 1) k) ((1 lda) (1 *))
                                 a-%offset%)
           1)))
(setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (-
          (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                         ((1 lda) (1 *)) a-%offset%)
          (cdotu (f2cl-lib:int-sub n k)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 ((+ k 1) k) ((1 lda) (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 ((+ k 1)
                                  (f2cl-lib:int-sub k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           1)))
(ccopy (f2cl-lib:int-sub n k)
 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                       ((+ k 1) (f2cl-lib:int-sub k 1))
                       ((1 lda) (1 *)) a-%offset%)
 1 work 1)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (csymv uplo (f2cl-lib:int-sub n k) (- one)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                           ((+ k 1) (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda work 1 zero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                           ((+ k 1) (f2cl-lib:int-sub k 1))
                           ((1 lda) (1 *)) a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
    var-9))
  (setf uplo var-0))
(setf (f2cl-lib:fref a-%data%
                      ((f2cl-lib:int-sub k 1)
                       (f2cl-lib:int-sub k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (-
          (f2cl-lib:fref a-%data%
                         ((f2cl-lib:int-sub k 1)
```

```
                              (f2cl-lib:int-sub k 1))
                             ((1 lda) (1 *)) a-%offset%)
             (cdotu (f2cl-lib:int-sub n k) work 1
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                    ((+ k 1)
                                     (f2cl-lib:int-sub k 1))
                                    ((1 lda) (1 *)) a-%offset%)
              1)))))
  (setf kstep 2)))
(setf kp
        (abs
         (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
(cond
 ((/= kp k)
  (if (< kp n)
      (cswap (f2cl-lib:int-sub n kp)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ kp 1) k) ((1 lda) (1 *))
                             a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ kp 1) kp) ((1 lda) (1 *))
                             a-%offset%)
       1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (cswap (f2cl-lib:int-sub kp k 1)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 ((+ k 1) k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             (kp (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (setf temp
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
          (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *)) a-%offset%)
          temp)
  (cond
   ((= kstep 2)
    (setf temp
            (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                           ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                          ((1 lda) (1 *)) a-%offset%)
```

```
                        (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-sub k 1))
                                       ((1 lda) (1 *)) a-%offset%))
                (setf (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-sub k 1))
                                     ((1 lda) (1 *)) a-%offset%)
                        temp)))))
            (setf k (f2cl-lib:int-sub k kstep))
            (go label50)
           label60)))
        (go end_label)
       end_label
        (return (values uplo nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csytri
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::csymv))))
```

———-

60.19 csytri2

60.19.1 Axiom unit tests

— **csytri2.input** —

```
)set break resume
)sys rm -f csytri2.output
)spool csytri2.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.19.2 Axiom help page

— **csytri2.help** —

```
==================================================================
csytri2 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csytri2 (UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO)
           CSYTRI2

Function/Subroutine Documentation
   subroutine csytri2 (character UPLO, integer N, complex, dimension( lda, * )
       A, integer LDA, integer, dimension( * ) IPIV, complex, dimension( * )
       WORK, integer LWORK, integer INFO)
       CSYTRI2

Purpose:
   CSYTRI2 computes the inverse of a COMPLEX symmetric indefinite matrix
   A using the factorization A = U*D*U**T or A = L*D*L**T computed by
   CSYTRF. CSYTRI2 sets the LEADING DIMENSION of the workspace
   before calling CSYTRI2X that actually computes the inverse.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
```

```
  N is INTEGER
  The order of the matrix A.  N >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the NB diagonal matrix D and the multipliers
  used to obtain the factor U or L as computed by CSYTRF.

  On exit, if INFO = 0, the (symmetric) inverse of the original
  matrix.  If UPLO = 'U', the upper triangular part of the
  inverse is formed and the part of A below the diagonal is not
  referenced; if UPLO = 'L' the lower triangular part of the
  inverse is formed and the part of A above the diagonal is
  not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the NB structure of D
  as determined by CSYTRF.

WORK
  WORK is COMPLEX array, dimension (N+NB+1)*(NB+3)

LWORK
  LWORK is INTEGER
  The dimension of the array WORK.
  WORK is size >= (N+NB+1)*(NB+3)
  If LDWORK = -1, then a workspace query is assumed; the routine
  calculates:
  - the optimal size of the WORK array, returns
  this value as the first entry of the WORK array,
  - and no error message related to LDWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = i, D(i,i) = 0; the matrix is singular and its
       inverse could not be computed.
```

———-

60.19.3 fortran code

— csytri2.f —

```
*  =====================================================================
      SUBROUTINE CSYTRI2( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            UPPER, LQUERY
      INTEGER            MINSIZE, NBMAX
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           CSYTRI, CSYTRI2X
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      LQUERY = ( LWORK.EQ.-1 )
*     Get blocksize
      NBMAX = ILAENV( 1, 'CSYTRF', UPLO, N, -1, -1, -1 )
      IF ( NBMAX .GE. N ) THEN
         MINSIZE = N
      ELSE
         MINSIZE = (N+NBMAX+1)*(NBMAX+3)
      END IF
```

```
*
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF (LWORK .LT. MINSIZE .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
*
*     Quick return if possible
*
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CSYTRI2', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         WORK(1)=MINSIZE
         RETURN
      END IF
      IF( N.EQ.0 )
     $   RETURN

      IF( NBMAX .GE. N ) THEN
         CALL CSYTRI( UPLO, N, A, LDA, IPIV, WORK, INFO )
      ELSE
         CALL CSYTRI2X( UPLO, N, A, LDA, IPIV, WORK, NBMAX, INFO )
      END IF
      RETURN
*
*     End of CSYTRI2
*
      END
```

———-

60.19.4 lisp code

— csytri2.lisp —

```
(defun csytri2 (uplo n a lda ipiv work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex8 (*)) work a)
           (type (f2cl-lib:integer4) info lwork lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
```

```
     (a f2cl-lib:complex8 a-%data% a-%offset%)
     (work f2cl-lib:complex8 work-%data% work-%offset%)
     (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
   (prog ((minsize 0) (nbmax 0) (upper nil) (lquery nil))
     (declare (type f2cl-lib:logical lquery upper)
              (type (f2cl-lib:integer4) nbmax minsize))
     (setf info 0)
     (setf upper
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame uplo "U")
               (declare (ignore var-1))
               (when var-0 (setf uplo var-0))
               ret-val))
     (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
     (setf nbmax
             (multiple-value-bind
                 (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                 (ilaenv 1 "CSYTRF" uplo n -1 -1 -1)
               (declare (ignore var-0 var-1 var-4 var-5 var-6))
               (when var-2 (setf uplo var-2))
               (when var-3 (setf n var-3))
               ret-val))
     (cond ((>= nbmax n) (setf minsize n))
           (t
            (setf minsize
                    (f2cl-lib:int-mul (f2cl-lib:int-add n nbmax 1)
                                      (f2cl-lib:int-add nbmax 3)))))
     (cond
      ((and (not upper)
            (not
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame uplo "L")
               (declare (ignore var-1))
               (when var-0 (setf uplo var-0))
               ret-val)))
       (setf info -1))
      ((< n 0) (setf info -2))
      ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -4))
      ((and (< lwork minsize) (not lquery)) (setf info -7)))
     (cond
      ((/= info 0) (xerbla "CSYTRI2" (f2cl-lib:int-sub info)) (go end_label))
      (lquery
       (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
               (coerce minsize 'f2cl-lib:complex8))
       (go end_label)))
     (if (= n 0)
         (go end_label))
     (cond
      ((>= nbmax n)
```

```
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (csytri uplo n a lda ipiv work info)
          (declare (ignore var-2 var-4 var-5))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-6 (setf info var-6))))
       (t
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (csytri2x uplo n a lda ipiv work nbmax info)
          (declare (ignore var-2 var-4 var-5))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-6 (setf nbmax var-6))
          (when var-7 (setf info var-7)))))
      (go end_label)
     end_label
      (return (values uplo n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csytri2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

60.20 csytri2x

60.20.1 Axiom unit tests

— csytri2x.input —

```
)set break resume
)sys rm -f csytri2x.output
)spool csytri2x.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.20.2 Axiom help page

— csytri2x.help —

```
======================================================================
csytri2x examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csytri2x (UPLO, N, A, LDA, IPIV, WORK, NB, INFO)
           CSYTRI2X

Function/Subroutine Documentation
   subroutine csytri2x (character UPLO, integer N, complex, dimension( lda, *
       ) A, integer LDA, integer, dimension( * ) IPIV, complex, dimension(
       n+nb+1,* ) WORK, integer NB, integer INFO)
       CSYTRI2X

Purpose:
   CSYTRI2X computes the inverse of a real symmetric indefinite matrix
   A using the factorization A = U*D*U**T or A = L*D*L**T computed by CSYTRF.

Parameters:
```

```
UPLO
  UPLO is CHARACTER*1
  Specifies whether the details of the factorization are stored
  as an upper or lower triangular matrix.
  = 'U':  Upper triangular, form is A = U*D*U**T;
  = 'L':  Lower triangular, form is A = L*D*L**T.

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the NNB diagonal matrix D and the multipliers
  used to obtain the factor U or L as computed by CSYTRF.

  On exit, if INFO = 0, the (symmetric) inverse of the original
  matrix.  If UPLO = 'U', the upper triangular part of the
  inverse is formed and the part of A below the diagonal is not
  referenced; if UPLO = 'L' the lower triangular part of the
  inverse is formed and the part of A above the diagonal is
  not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the NNB structure of D
  as determined by CSYTRF.

WORK
  WORK is COMPLEX array, dimension (N+NNB+1,NNB+3)

NB
  NB is INTEGER
  Block size

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = i, D(i,i) = 0; the matrix is singular and its
       inverse could not be computed.
```

60.20.3 fortran code

— csytri2x.f —

```
*  =====================================================================
      SUBROUTINE CSYTRI2X( UPLO, N, A, LDA, IPIV, WORK, NB, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N, NB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), WORK( N+NB+1,* )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX              ONE, ZERO
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ),
     $                   ZERO = ( 0.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, IINFO, IP, K, CUT, NNB
      INTEGER            COUNT
      INTEGER            J, U11, INVD

      COMPLEX   AK, AKKP1, AKP1, D, T
      COMPLEX   U01_I_J, U01_IP1_J
      COMPLEX   U11_I_J, U11_IP1_J
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CSYCONV, XERBLA, CTRTRI
      EXTERNAL           CGEMM, CTRMM, CSYSWAPR
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
```

```
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
*
*     Quick return if possible
*
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CSYTRI2X', -INFO )
         RETURN
      END IF
      IF( N.EQ.0 )
     $   RETURN
*
*     Convert A
*     Workspace got Non-diag elements of D
*
      CALL CSYCONV( UPLO, 'C', N, A, LDA, IPIV, WORK, IINFO )
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
         DO INFO = N, 1, -1
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
         END DO
      ELSE
*
*        Lower triangular storage: examine D from top to bottom.
*
         DO INFO = 1, N
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
         END DO
      END IF
      INFO = 0
*
```

```
*  Splitting Workspace
*     U01 is a block (N,NB+1)
*     The first element of U01 is in WORK(1,1)
*     U11 is a block (NB+1,NB+1)
*     The first element of U11 is in WORK(N+1,1)
      U11 = N
*     INVD is a block (N,2)
*     The first element of INVD is in WORK(1,INVD)
      INVD = NB+2

      IF( UPPER ) THEN
*
*        invA = P * inv(U**T)*inv(D)*inv(U)*P**T.
*
        CALL CTRTRI( UPLO, 'U', N, A, LDA, INFO )
*
*       inv(D) and inv(D)*inv(U)
*
        K=1
        DO WHILE ( K .LE. N )
         IF( IPIV( K ).GT.0 ) THEN
*           1 x 1 diagonal NNB
             WORK(K,INVD) = ONE /  A( K, K )
             WORK(K,INVD+1) = 0
            K=K+1
         ELSE
*           2 x 2 diagonal NNB
             T = WORK(K+1,1)
             AK = A( K, K ) / T
             AKP1 = A( K+1, K+1 ) / T
             AKKP1 = WORK(K+1,1)  / T
             D = T*( AK*AKP1-ONE )
             WORK(K,INVD) = AKP1 / D
             WORK(K+1,INVD+1) = AK / D
             WORK(K,INVD+1) = -AKKP1 / D
             WORK(K+1,INVD) = -AKKP1 / D
            K=K+2
         END IF
        END DO
*
*       inv(U**T) = (inv(U))**T
*
*       inv(U**T)*inv(D)*inv(U)
*
        CUT=N
        DO WHILE (CUT .GT. 0)
           NNB=NB
           IF (CUT .LE. NNB) THEN
              NNB=CUT
           ELSE
```

```
              COUNT = 0
*             count negative elements,
              DO I=CUT+1-NNB,CUT
                  IF (IPIV(I) .LT. 0) COUNT=COUNT+1
              END DO
*             need a even number for a clear cut
              IF (MOD(COUNT,2) .EQ. 1) NNB=NNB+1
           END IF

           CUT=CUT-NNB
*
*          U01 Block
*
           DO I=1,CUT
             DO J=1,NNB
              WORK(I,J)=A(I,CUT+J)
             END DO
           END DO
*
*          U11 Block
*
           DO I=1,NNB
             WORK(U11+I,I)=ONE
             DO J=1,I-1
                WORK(U11+I,J)=ZERO
             END DO
             DO J=I+1,NNB
                WORK(U11+I,J)=A(CUT+I,CUT+J)
             END DO
           END DO
*
*          invD*U01
*
           I=1
           DO WHILE (I .LE. CUT)
             IF (IPIV(I) > 0) THEN
                DO J=1,NNB
                    WORK(I,J)=WORK(I,INVD)*WORK(I,J)
                END DO
                I=I+1
             ELSE
                DO J=1,NNB
                   U01_I_J = WORK(I,J)
                   U01_IP1_J = WORK(I+1,J)
                   WORK(I,J)=WORK(I,INVD)*U01_I_J+
     $                     WORK(I,INVD+1)*U01_IP1_J
                   WORK(I+1,J)=WORK(I+1,INVD)*U01_I_J+
     $                     WORK(I+1,INVD+1)*U01_IP1_J
                END DO
                I=I+2
```

```
            END IF
          END DO
*
*        invD1*U11
*
          I=1
          DO WHILE (I .LE. NNB)
            IF (IPIV(CUT+I) > 0) THEN
               DO J=I,NNB
                   WORK(U11+I,J)=WORK(CUT+I,INVD)*WORK(U11+I,J)
               END DO
               I=I+1
            ELSE
               DO J=I,NNB
                  U11_I_J = WORK(U11+I,J)
                  U11_IP1_J = WORK(U11+I+1,J)
               WORK(U11+I,J)=WORK(CUT+I,INVD)*WORK(U11+I,J) +
     $                     WORK(CUT+I,INVD+1)*WORK(U11+I+1,J)
               WORK(U11+I+1,J)=WORK(CUT+I+1,INVD)*U11_I_J+
     $                     WORK(CUT+I+1,INVD+1)*U11_IP1_J
               END DO
               I=I+2
            END IF
          END DO
*
*       U11**T*invD1*U11->U11
*
        CALL CTRMM('L','U','T','U',NNB, NNB,
     $             ONE,A(CUT+1,CUT+1),LDA,WORK(U11+1,1),N+NB+1)
*
         DO I=1,NNB
            DO J=I,NNB
              A(CUT+I,CUT+J)=WORK(U11+I,J)
            END DO
         END DO
*
*          U01**T*invD*U01->A(CUT+I,CUT+J)
*
         CALL CGEMM('T','N',NNB,NNB,CUT,ONE,A(1,CUT+1),LDA,
     $              WORK,N+NB+1, ZERO, WORK(U11+1,1), N+NB+1)
*
*        U11 =  U11**T*invD1*U11 + U01**T*invD*U01
*
         DO I=1,NNB
            DO J=I,NNB
              A(CUT+I,CUT+J)=A(CUT+I,CUT+J)+WORK(U11+I,J)
            END DO
         END DO
*
*        U01 =  U00**T*invD0*U01
```

```
*
         CALL CTRMM('L',UPLO,'T','U',CUT, NNB,
     $             ONE,A,LDA,WORK,N+NB+1)

*
*        Update U01
*
         DO I=1,CUT
           DO J=1,NNB
            A(I,CUT+J)=WORK(I,J)
           END DO
         END DO
*
*      Next Block
*
       END DO
*
*        Apply PERMUTATIONS P and P**T: P * inv(U**T)*inv(D)*inv(U) *P**T
*
            I=1
            DO WHILE ( I .LE. N )
               IF( IPIV(I) .GT. 0 ) THEN
                  IP=IPIV(I)
                 IF (I .LT. IP) CALL CSYSWAPR( UPLO, N, A, LDA, I ,IP )
                 IF (I .GT. IP) CALL CSYSWAPR( UPLO, N, A, LDA, IP ,I )
               ELSE
                 IP=-IPIV(I)
                 I=I+1
                 IF ( (I-1) .LT. IP)
     $                  CALL CSYSWAPR( UPLO, N, A, LDA, I-1 ,IP )
                 IF ( (I-1) .GT. IP)
     $                  CALL CSYSWAPR( UPLO, N, A, LDA, IP ,I-1 )
              ENDIF
               I=I+1
            END DO
      ELSE
*
*        LOWER...
*
*        invA = P * inv(U**T)*inv(D)*inv(U)*P**T.
*
         CALL CTRTRI( UPLO, 'U', N, A, LDA, INFO )
*
*       inv(D) and inv(D)*inv(U)
*
        K=N
        DO WHILE ( K .GE. 1 )
         IF( IPIV( K ).GT.0 ) THEN
*           1 x 1 diagonal NNB
             WORK(K,INVD) = ONE /  A( K, K )
```

```
            WORK(K,INVD+1) = 0
           K=K-1
         ELSE
*           2 x 2 diagonal NNB
            T = WORK(K-1,1)
            AK = A( K-1, K-1 ) / T
            AKP1 = A( K, K ) / T
            AKKP1 = WORK(K-1,1) / T
            D = T*( AK*AKP1-ONE )
            WORK(K-1,INVD) = AKP1 / D
            WORK(K,INVD) = AK / D
            WORK(K,INVD+1) = -AKKP1 / D
            WORK(K-1,INVD+1) = -AKKP1 / D
           K=K-2
         END IF
        END DO
*
*       inv(U**T) = (inv(U))**T
*
*       inv(U**T)*inv(D)*inv(U)
*
        CUT=0
        DO WHILE (CUT .LT. N)
           NNB=NB
           IF (CUT + NNB .GE. N) THEN
              NNB=N-CUT
           ELSE
              COUNT = 0
*             count negative elements,
              DO I=CUT+1,CUT+NNB
                  IF (IPIV(I) .LT. 0) COUNT=COUNT+1
              END DO
*             need a even number for a clear cut
              IF (MOD(COUNT,2) .EQ. 1) NNB=NNB+1
           END IF
*      L21 Block
           DO I=1,N-CUT-NNB
             DO J=1,NNB
              WORK(I,J)=A(CUT+NNB+I,CUT+J)
             END DO
           END DO
*     L11 Block
           DO I=1,NNB
             WORK(U11+I,I)=ONE
             DO J=I+1,NNB
                WORK(U11+I,J)=ZERO
             END DO
             DO J=1,I-1
                WORK(U11+I,J)=A(CUT+I,CUT+J)
             END DO
```

```
          END DO
*
*         invD*L21
*
          I=N-CUT-NNB
          DO WHILE (I .GE. 1)
            IF (IPIV(CUT+NNB+I) > 0) THEN
               DO J=1,NNB
                   WORK(I,J)=WORK(CUT+NNB+I,INVD)*WORK(I,J)
               END DO
               I=I-1
            ELSE
               DO J=1,NNB
                  U01_I_J = WORK(I,J)
                  U01_IP1_J = WORK(I-1,J)
                  WORK(I,J)=WORK(CUT+NNB+I,INVD)*U01_I_J+
     $                       WORK(CUT+NNB+I,INVD+1)*U01_IP1_J
                  WORK(I-1,J)=WORK(CUT+NNB+I-1,INVD+1)*U01_I_J+
     $                       WORK(CUT+NNB+I-1,INVD)*U01_IP1_J
               END DO
               I=I-2
            END IF
          END DO
*
*       invD1*L11
*
          I=NNB
          DO WHILE (I .GE. 1)
            IF (IPIV(CUT+I) > 0) THEN
               DO J=1,NNB
                   WORK(U11+I,J)=WORK(CUT+I,INVD)*WORK(U11+I,J)
               END DO
               I=I-1
            ELSE
               DO J=1,NNB
                  U11_I_J = WORK(U11+I,J)
                  U11_IP1_J = WORK(U11+I-1,J)
               WORK(U11+I,J)=WORK(CUT+I,INVD)*WORK(U11+I,J) +
     $                     WORK(CUT+I,INVD+1)*U11_IP1_J
               WORK(U11+I-1,J)=WORK(CUT+I-1,INVD+1)*U11_I_J+
     $                     WORK(CUT+I-1,INVD)*U11_IP1_J
               END DO
               I=I-2
            END IF
          END DO
*
*      L11**T*invD1*L11->L11
*
       CALL CTRMM('L',UPLO,'T','U',NNB, NNB,
     $            ONE,A(CUT+1,CUT+1),LDA,WORK(U11+1,1),N+NB+1)
```

```
*
         DO I=1,NNB
            DO J=1,I
              A(CUT+I,CUT+J)=WORK(U11+I,J)
            END DO
         END DO
*
        IF ( (CUT+NNB) .LT. N ) THEN
*
*          L21**T*invD2*L21->A(CUT+I,CUT+J)
*
         CALL CGEMM('T','N',NNB,NNB,N-NNB-CUT,ONE,A(CUT+NNB+1,CUT+1)
     $             ,LDA,WORK,N+NB+1, ZERO, WORK(U11+1,1), N+NB+1)

*
*        L11 =  L11**T*invD1*L11 + U01**T*invD*U01
*
         DO I=1,NNB
            DO J=1,I
              A(CUT+I,CUT+J)=A(CUT+I,CUT+J)+WORK(U11+I,J)
            END DO
         END DO
*
*        L01 =  L22**T*invD2*L21
*
         CALL CTRMM('L',UPLO,'T','U', N-NNB-CUT, NNB,
     $             ONE,A(CUT+NNB+1,CUT+NNB+1),LDA,WORK,N+NB+1)

*      Update L21
         DO I=1,N-CUT-NNB
           DO J=1,NNB
              A(CUT+NNB+I,CUT+J)=WORK(I,J)
           END DO
         END DO
       ELSE
*
*        L11 =  L11**T*invD1*L11
*
         DO I=1,NNB
            DO J=1,I
              A(CUT+I,CUT+J)=WORK(U11+I,J)
            END DO
         END DO
       END IF
*
*      Next Block
*
           CUT=CUT+NNB
       END DO
*
```

```
*        Apply PERMUTATIONS P and P**T: P * inv(U**T)*inv(D)*inv(U) *P**T
*
            I=N
            DO WHILE ( I .GE. 1 )
               IF( IPIV(I) .GT. 0 ) THEN
                  IP=IPIV(I)
                 IF (I .LT. IP) CALL CSYSWAPR( UPLO, N, A, LDA, I ,IP  )
                 IF (I .GT. IP) CALL CSYSWAPR( UPLO, N, A, LDA, IP ,I )
               ELSE
                 IP=-IPIV(I)
                 IF ( I .LT. IP) CALL CSYSWAPR( UPLO, N, A, LDA, I ,IP )
                 IF ( I .GT. IP) CALL CSYSWAPR( UPLO, N, A, LDA, IP ,I )
                 I=I-1
               ENDIF
               I=I-1
            END DO
      END IF
*
      RETURN
*
*     End of CSYTRI2X
*
      END
```

———-

60.20.4 lisp code

— csytri2x.lisp —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)) (zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (ignorable one zero))
  (defun csytri2x (uplo n a lda ipiv work nb info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) work a)
             (type (f2cl-lib:integer4) info nb lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((u11_i_j #C(0.0 0.0)) (u11_ip1_j #C(0.0 0.0))
             (u01_i_j #C(0.0 0.0)) (u01_ip1_j #C(0.0 0.0)) (ak #C(0.0 0.0))
             (akkp1 #C(0.0 0.0)) (akp1 #C(0.0 0.0)) (d #C(0.0 0.0))
```

```
     (t$ #C(0.0 0.0)) (j 0) (u11 0) (invd 0) (count$ 0) (i 0) (iinfo 0)
     (ip 0) (k 0) (cut 0) (nnb 0) (upper nil))
(declare
 (type (f2cl-lib:complex8) u11_i_j u11_ip1_j u01_i_j u01_ip1_j ak akkp1
  akp1 d t$)
 (type (f2cl-lib:integer4) j u11 invd count$ i iinfo ip k cut nnb)
 (type f2cl-lib:logical upper))
(setf info 0)
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4)))
(cond
 ((/= info 0) (xerbla "CSYTRI2X" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (csyconv uplo "C" n a lda ipiv work iinfo)
  (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
  (setf uplo var-0)
  (setf iinfo var-7))
(cond
 (upper
  (f2cl-lib:fdo (info n (f2cl-lib:int-add info (f2cl-lib:int-sub 1)))
                ((> info 1) nil)
    (tagbody
      (if (and
           (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
              0)
           (=
            (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                           a-%offset%)
            zero))
          (go end_label))
     label100000)))
```

```
 (t
  (f2cl-lib:fdo (info 1 (f2cl-lib:int-add info 1))
                ((> info n) nil)
    (tagbody
      (if (and
           (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
              0)
           (=
            (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                           a-%offset%)
            zero))
          (go end_label))
     label100001))))
(setf info 0)
(setf u11 n)
(setf invd (f2cl-lib:int-add nb 2))
(cond
 (upper
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (ctrtri uplo "U" n a lda info)
      (declare (ignore var-1 var-3))
      (when var-0 (setf uplo var-0))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-5 (setf info var-5)))
    (setf k 1)
   label100002
    (if (not (<= k n))
        (go label100003))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf (f2cl-lib:fref work-%data% (k invd)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
              (/ one
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%)))
      (setf (f2cl-lib:fref work-%data% (k (f2cl-lib:int-add invd 1))
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
              (coerce 0 'f2cl-lib:complex8))
      (setf k (f2cl-lib:int-add k 1)))
     (t
      (setf t$
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1) 1)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%))
      (setf ak
              (/
```

```
            (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                           a-%offset%)
            t$))
    (setf akp1
           (/
            (f2cl-lib:fref a-%data%
                           ((f2cl-lib:int-add k 1)
                            (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
            t$))
    (setf akkp1
           (/
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1) 1)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
            t$))
    (setf d (* t$ (- (* ak akp1) one)))
    (setf (f2cl-lib:fref work-%data% (k invd)
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
           (/ akp1 d))
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add k 1)
                          (f2cl-lib:int-add invd 1))
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
           (/ ak d))
    (setf (f2cl-lib:fref work-%data% (k (f2cl-lib:int-add invd 1))
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
           (/ (- akkp1) d))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1) invd)
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
           (/ (- akkp1) d))
    (setf k (f2cl-lib:int-add k 2))))
  (go label100002)
 label100003
  (setf cut n)
 label100004
  (if (not (> cut 0))
      (go label100005))
  (setf nnb nb)
  (cond ((<= cut nnb) (setf nnb cut))
        (t (setf count$ 0)
         (f2cl-lib:fdo (i
                        (f2cl-lib:int-add cut 1
                                          (f2cl-lib:int-sub nnb))
                        (f2cl-lib:int-add i 1))
                       ((> i cut) nil)
```

```
          (tagbody
            (if (<
                 (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                                ipiv-%offset%)
                 0)
                (setf count$ (f2cl-lib:int-add count$ 1)))
           label100006))
        (if (= (mod count$ 2) 1)
            (setf nnb (f2cl-lib:int-add nnb 1)))))
(setf cut (f2cl-lib:int-sub cut nnb))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i cut) nil)
  (tagbody
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
                (f2cl-lib:fref a-%data%
                               (i (f2cl-lib:int-add cut j))
                               ((1 lda) (1 *)) a-%offset%))
       label100008))
   label100007))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nnb) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) i)
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
            one)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add u11 i) j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
                zero)
       label100010))
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add u11 i) j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
                (f2cl-lib:fref a-%data%
                               ((f2cl-lib:int-add cut i)
```

```
                                  (f2cl-lib:int-add cut j))
                                 ((1 lda) (1 *)) a-%offset%))
         label100011))
     label100009))
  (setf i 1)
 label100012
  (if (not (<= i cut))
      (go label100013))
  (cond
   ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
                (*
                 (f2cl-lib:fref work-%data% (i invd)
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%)
                 (f2cl-lib:fref work-%data% (i j)
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%)))
       label100014))
    (setf i (f2cl-lib:int-add i 1)))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf u01_i_j
                (f2cl-lib:fref work-%data% (i j)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%))
        (setf u01_ip1_j
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1) j)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%))
        (setf (f2cl-lib:fref work-%data% (i j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
                (+
                 (*
                  (f2cl-lib:fref work-%data% (i invd)
                                 ((1 (f2cl-lib:int-add n nb 1))
                                  (1 *))
                                 work-%offset%)
                  u01_i_j)
                 (*
                  (f2cl-lib:fref work-%data%
                                 (i (f2cl-lib:int-add invd 1))
```

```
                                    ((1 (f2cl-lib:int-add n nb 1))
                                     (1 *))
                                    work-%offset%)
                     u01_ip1_j)))
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1) j)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)
                  (+
                   (*
                    (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-add i 1) invd)
                                   ((1 (f2cl-lib:int-add n nb 1))
                                    (1 *))
                                   work-%offset%)
                    u01_i_j)
                   (*
                    (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-add i 1)
                                    (f2cl-lib:int-add invd 1))
                                   ((1 (f2cl-lib:int-add n nb 1))
                                    (1 *))
                                   work-%offset%)
                    u01_ip1_j)))
        label100015))
      (setf i (f2cl-lib:int-add i 2))))
    (go label100012)
   label100013
    (setf i 1)
   label100016
    (if (not (<= i nnb))
        (go label100017))
    (cond
     ((> (f2cl-lib:fref ipiv ((f2cl-lib:int-add cut i)) ((1 *))) 0)
      (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                    ((> j nnb) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) j)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)
                  (*
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add cut i) invd)
                                  ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                  work-%offset%)
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add u11 i) j)
                                  ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                  work-%offset%)))
        label100018))
      (setf i (f2cl-lib:int-add i 1)))
```

```
(t
 (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
               ((> j nnb) nil)
   (tagbody
     (setf u11_i_j
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add u11 i) j)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%))
     (setf u11_ip1_j
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add u11 i 1) j)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%))
     (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) j)
                          ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                          work-%offset%)
             (+
              (*
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add cut i) invd)
                              ((1 (f2cl-lib:int-add n nb 1))
                               (1 *))
                              work-%offset%)
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add u11 i) j)
                              ((1 (f2cl-lib:int-add n nb 1))
                               (1 *))
                              work-%offset%))
              (*
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add cut i)
                               (f2cl-lib:int-add invd 1))
                              ((1 (f2cl-lib:int-add n nb 1))
                               (1 *))
                              work-%offset%)
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add u11 i 1) j)
                              ((1 (f2cl-lib:int-add n nb 1))
                               (1 *))
                              work-%offset%))))
     (setf (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add u11 i 1) j)
                          ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                          work-%offset%)
             (+
              (*
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add cut i 1) invd)
                              ((1 (f2cl-lib:int-add n nb 1))
```

```
                                    (1 *))
                                   work-%offset%)
                    u11_i_j)
                   (*
                    (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-add cut i 1)
                                    (f2cl-lib:int-add invd 1))
                                   ((1 (f2cl-lib:int-add n nb 1))
                                    (1 *))
                                   work-%offset%)
                    u11_ip1_j)))
        label100019))
      (setf i (f2cl-lib:int-add i 2))))
    (go label100016)
   label100017
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (ctrmm "L" "U" "T" "U" nnb nnb one
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               ((+ cut 1) (f2cl-lib:int-add cut 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                               ((+ u11 1) 1)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)
         (f2cl-lib:int-add n nb 1))
      (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9 var-10))
      (when var-4 (setf nnb var-4))
      (when var-5 (setf nnb var-5))
      (when var-6 (setf one var-6))
      (when var-8 (setf lda var-8)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i nnb) nil)
      (tagbody
        (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                      ((> j nnb) nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add cut i)
                                  (f2cl-lib:int-add cut j))
                                 ((1 lda) (1 *)) a-%offset%)
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add u11 i) j)
                                 ((1 (f2cl-lib:int-add n nb 1))
                                  (1 *))
                                 work-%offset%))
          label100021))
      label100020))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (cgemm "T" "N" nnb nnb cut one
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                           (1 (f2cl-lib:int-add cut 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda work (f2cl-lib:int-add n nb 1) zero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                           ((+ u11 1) 1)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
     (f2cl-lib:int-add n nb 1))
  (declare (ignore var-0 var-1 var-6 var-8 var-9 var-11 var-12))
  (when var-2 (setf nnb var-2))
  (when var-3 (setf nnb var-3))
  (when var-4 (setf cut var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf zero var-10)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nnb) nil)
  (tagbody
    (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data%
                             ((f2cl-lib:int-add cut i)
                              (f2cl-lib:int-add cut j))
                             ((1 lda) (1 *)) a-%offset%)
                (+
                 (f2cl-lib:fref a-%data%
                                ((f2cl-lib:int-add cut i)
                                 (f2cl-lib:int-add cut j))
                                ((1 lda) (1 *)) a-%offset%)
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add u11 i) j)
                                ((1 (f2cl-lib:int-add n nb 1))
                                 (1 *))
                                work-%offset%)))
       label100023))
   label100022))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ctrmm "L" uplo "T" "U" cut nnb one a lda work
     (f2cl-lib:int-add n nb 1))
  (declare (ignore var-0 var-2 var-3 var-7 var-9 var-10))
  (when var-1 (setf uplo var-1))
  (when var-4 (setf cut var-4))
```

```
    (when var-5 (setf nnb var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i cut) nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nnb) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add cut j))
                                ((1 lda) (1 *)) a-%offset%)
                  (f2cl-lib:fref work-%data% (i j)
                                  ((1 (f2cl-lib:int-add n nb 1))
                                   (1 *))
                                  work-%offset%))
         label100025))
     label100024))
  (go label100004)
 label100005
  (setf i 1)
 label100026
  (if (not (<= i n))
      (go label100027))
  (cond
   ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
    (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
    (if (< i ip)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (csyswapr uplo n a lda i ip)
          (declare (ignore var-2))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-4 (setf i var-4))
          (when var-5 (setf ip var-5))))
    (if (> i ip)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (csyswapr uplo n a lda ip i)
          (declare (ignore var-2))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-4 (setf ip var-4))
          (when var-5 (setf i var-5)))))
   (t
    (setf ip
            (f2cl-lib:int-sub
             (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)))
    (setf i (f2cl-lib:int-add i 1))
    (if (< (f2cl-lib:int-sub i 1) ip)
```

```
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
              (csyswapr uplo n a lda (f2cl-lib:int-sub i 1) ip)
            (declare (ignore var-2 var-4))
            (when var-0 (setf uplo var-0))
            (when var-1 (setf n var-1))
            (when var-3 (setf lda var-3))
            (when var-5 (setf ip var-5))))
      (if (> (f2cl-lib:int-sub i 1) ip)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
              (csyswapr uplo n a lda ip (f2cl-lib:int-sub i 1))
            (declare (ignore var-2 var-5))
            (when var-0 (setf uplo var-0))
            (when var-1 (setf n var-1))
            (when var-3 (setf lda var-3))
            (when var-4 (setf ip var-4))))))
    (setf i (f2cl-lib:int-add i 1))
    (go label100026)
   label100027))
 (t
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (ctrtri uplo "U" n a lda info)
      (declare (ignore var-1 var-3))
      (when var-0 (setf uplo var-0))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-5 (setf info var-5)))
    (setf k n)
   label100028
    (if (not (>= k 1))
        (go label100029))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf (f2cl-lib:fref work-%data% (k invd)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
            (/ one
               (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                              a-%offset%)))
      (setf (f2cl-lib:fref work-%data% (k (f2cl-lib:int-add invd 1))
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
            (coerce 0 'f2cl-lib:complex8))
      (setf k (f2cl-lib:int-sub k 1)))
     (t
      (setf t$
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1) 1)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%))
      (setf ak
```

```
            (/
             (f2cl-lib:fref a-%data%
                            ((f2cl-lib:int-sub k 1)
                             (f2cl-lib:int-sub k 1))
                            ((1 lda) (1 *)) a-%offset%)
             t$))
    (setf akp1
            (/
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)
             t$))
    (setf akkp1
            (/
             (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1) 1)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%)
             t$))
    (setf d (* t$ (- (* ak akp1) one)))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1) invd)
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
            (/ akp1 d))
    (setf (f2cl-lib:fref work-%data% (k invd)
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
            (/ ak d))
    (setf (f2cl-lib:fref work-%data% (k (f2cl-lib:int-add invd 1))
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
            (/ (- akkp1) d))
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-sub k 1)
                          (f2cl-lib:int-add invd 1))
                         ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                         work-%offset%)
            (/ (- akkp1) d))
    (setf k (f2cl-lib:int-sub k 2))))
  (go label100028)
label100029
  (setf cut 0)
label100030
  (if (not (< cut n))
      (go label100031))
  (setf nnb nb)
  (cond
   ((>= (f2cl-lib:int-add cut nnb) n)
    (setf nnb (f2cl-lib:int-sub n cut)))
   (t (setf count$ 0)
    (f2cl-lib:fdo (i (f2cl-lib:int-add cut 1) (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add cut nnb)) nil)
```

```
      (tagbody
        (if (< (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
               0)
            (setf count$ (f2cl-lib:int-add count$ 1)))
       label100032))
    (if (= (mod count$ 2) 1)
        (setf nnb (f2cl-lib:int-add nnb 1)))))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i
                    (f2cl-lib:int-add n (f2cl-lib:int-sub cut)
                                      (f2cl-lib:int-sub nnb)))
                 nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nnb) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i j)
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%)
                  (f2cl-lib:fref a-%data%
                                  ((f2cl-lib:int-add cut nnb i)
                                   (f2cl-lib:int-add cut j))
                                  ((1 lda) (1 *)) a-%offset%))
         label100034))
     label100033))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i nnb) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) i)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
            one)
      (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                    ((> j nnb) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add u11 i) j)
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%)
                  zero)
         label100036))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add u11 i) j)
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%)
                  (f2cl-lib:fref a-%data%
```

```
                                  ((f2cl-lib:int-add cut i)
                                   (f2cl-lib:int-add cut j))
                                  ((1 lda) (1 *)) a-%offset%))
        label100037))
    label100035))
 (setf i (f2cl-lib:int-sub n cut nnb))
label100038
 (if (not (>= i 1))
     (go label100039))
 (cond
  ((> (f2cl-lib:fref ipiv ((f2cl-lib:int-add cut nnb i)) ((1 *))) 0)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j nnb) nil)
     (tagbody
       (setf (f2cl-lib:fref work-%data% (i j)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%)
               (*
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add cut nnb i) invd)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)
                (f2cl-lib:fref work-%data% (i j)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)))
      label100040))
   (setf i (f2cl-lib:int-sub i 1)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j nnb) nil)
     (tagbody
       (setf u01_i_j
               (f2cl-lib:fref work-%data% (i j)
                              ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                              work-%offset%))
       (setf u01_ip1_j
               (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub i 1) j)
                              ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                              work-%offset%))
       (setf (f2cl-lib:fref work-%data% (i j)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%)
               (+
                (*
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add cut nnb i) invd)
                                ((1 (f2cl-lib:int-add n nb 1))
                                 (1 *))
                                work-%offset%)
                 u01_i_j)
```

```
                    (*
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add cut nnb i)
                                     (f2cl-lib:int-add invd 1))
                                    ((1 (f2cl-lib:int-add n nb 1))
                                     (1 *))
                                    work-%offset%)
                     u01_ip1_j)))
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub i 1) j)
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%)
                  (+
                   (*
                    (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-sub
                                     (f2cl-lib:int-add cut nnb i) 1)
                                    (f2cl-lib:int-add invd 1))
                                   ((1 (f2cl-lib:int-add n nb 1))
                                    (1 *))
                                   work-%offset%)
                    u01_i_j)
                   (*
                    (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-sub
                                     (f2cl-lib:int-add cut nnb i) 1)
                                    invd)
                                   ((1 (f2cl-lib:int-add n nb 1))
                                    (1 *))
                                   work-%offset%)
                    u01_ip1_j)))
        label100041))
     (setf i (f2cl-lib:int-sub i 2))))
   (go label100038)
  label100039
   (setf i nnb)
  label100042
   (if (not (>= i 1))
       (go label100043))
   (cond
    ((> (f2cl-lib:fref ipiv ((f2cl-lib:int-add cut i)) ((1 *))) 0)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j nnb) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) j)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)
                  (*
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add cut i) invd)
                                  ((1 (f2cl-lib:int-add n nb 1)) (1 *))
```

```
                                 work-%offset%)
                 (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add u11 i) j)
                                 ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                 work-%offset%)))
    label100044))
 (setf i (f2cl-lib:int-sub i 1)))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j nnb) nil)
   (tagbody
     (setf u11_i_j
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add u11 i) j)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%))
     (setf u11_ip1_j
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add u11 i) 1)
                             j)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%))
     (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) j)
                          ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                          work-%offset%)
             (+
              (*
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add cut i) invd)
                              ((1 (f2cl-lib:int-add n nb 1))
                               (1 *))
                              work-%offset%)
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add u11 i) j)
                              ((1 (f2cl-lib:int-add n nb 1))
                               (1 *))
                              work-%offset%))
              (*
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add cut i)
                               (f2cl-lib:int-add invd 1))
                              ((1 (f2cl-lib:int-add n nb 1))
                               (1 *))
                              work-%offset%)
               u11_ip1_j)))
     (setf (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add u11 i) 1)
                           j)
```

```
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%)
                        (+
                         (*
                          (f2cl-lib:fref work-%data%
                                         ((f2cl-lib:int-sub
                                           (f2cl-lib:int-add cut i) 1)
                                          (f2cl-lib:int-add invd 1))
                                         ((1 (f2cl-lib:int-add n nb 1))
                                          (1 *))
                                         work-%offset%)
                          u11_i_j)
                         (*
                          (f2cl-lib:fref work-%data%
                                         ((f2cl-lib:int-sub
                                           (f2cl-lib:int-add cut i) 1)
                                          invd)
                                         ((1 (f2cl-lib:int-add n nb 1))
                                          (1 *))
                                         work-%offset%)
                          u11_ip1_j)))
                 label100045))
            (setf i (f2cl-lib:int-sub i 2))))
          (go label100042)
        label100043
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10)
              (ctrmm "L" uplo "T" "U" nnb nnb one
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                     ((+ cut 1) (f2cl-lib:int-add cut 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                     ((+ u11 1) 1)
                                     ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                     work-%offset%)
               (f2cl-lib:int-add n nb 1))
            (declare (ignore var-0 var-2 var-3 var-7 var-9 var-10))
            (when var-1 (setf uplo var-1))
            (when var-4 (setf nnb var-4))
            (when var-5 (setf nnb var-5))
            (when var-6 (setf one var-6))
            (when var-8 (setf lda var-8)))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i nnb) nil)
            (tagbody
              (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                            ((> j i) nil)
                (tagbody
```

```
          (setf (f2cl-lib:fref a-%data%
                               ((f2cl-lib:int-add cut i)
                                (f2cl-lib:int-add cut j))
                               ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add u11 i) j)
                                 ((1 (f2cl-lib:int-add n nb 1))
                                  (1 *))
                                 work-%offset%))
        label100047))
    label100046))
  (cond
   ((< (f2cl-lib:int-add cut nnb) n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (cgemm "T" "N" nnb nnb (f2cl-lib:int-sub n nnb cut) one
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               ((+ cut nnb 1)
                                (f2cl-lib:int-add cut 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda work (f2cl-lib:int-add n nb 1) zero
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                               ((+ u11 1) 1)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)
         (f2cl-lib:int-add n nb 1))
      (declare
       (ignore var-0 var-1 var-4 var-6 var-8 var-9 var-11 var-12))
      (when var-2 (setf nnb var-2))
      (when var-3 (setf nnb var-3))
      (when var-5 (setf one var-5))
      (when var-7 (setf lda var-7))
      (when var-10 (setf zero var-10)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i nnb) nil)
      (tagbody
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j i) nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add cut i)
                                  (f2cl-lib:int-add cut j))
                                 ((1 lda) (1 *)) a-%offset%)
                  (+
                   (f2cl-lib:fref a-%data%
                                  ((f2cl-lib:int-add cut i)
                                   (f2cl-lib:int-add cut j))
                                  ((1 lda) (1 *)) a-%offset%)
                   (f2cl-lib:fref work-%data%
```

```
                                  ((f2cl-lib:int-add u11 i) j)
                                  ((1 (f2cl-lib:int-add n nb 1))
                                   (1 *))
                                  work-%offset%)))
        label100049))
    label100048))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (ctrmm "L" uplo "T" "U" (f2cl-lib:int-sub n nnb cut) nnb one
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                            ((+ cut nnb 1)
                             (f2cl-lib:int-add cut nnb 1))
                            ((1 lda) (1 *)) a-%offset%)
      lda work (f2cl-lib:int-add n nb 1))
   (declare (ignore var-0 var-2 var-3 var-4 var-7 var-9 var-10))
   (when var-1 (setf uplo var-1))
   (when var-5 (setf nnb var-5))
   (when var-6 (setf one var-6))
   (when var-8 (setf lda var-8)))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i
                   (f2cl-lib:int-add n (f2cl-lib:int-sub cut)
                                     (f2cl-lib:int-sub nnb)))
                nil)
   (tagbody
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j nnb) nil)
       (tagbody
         (setf (f2cl-lib:fref a-%data%
                              ((f2cl-lib:int-add cut nnb i)
                               (f2cl-lib:int-add cut j))
                              ((1 lda) (1 *)) a-%offset%)
                 (f2cl-lib:fref work-%data% (i j)
                                ((1 (f2cl-lib:int-add n nb 1))
                                 (1 *))
                                work-%offset%))
        label100051))
    label100050)))
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i nnb) nil)
   (tagbody
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j i) nil)
       (tagbody
         (setf (f2cl-lib:fref a-%data%
                              ((f2cl-lib:int-add cut i)
                               (f2cl-lib:int-add cut j))
                              ((1 lda) (1 *)) a-%offset%)
```

```
                    (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-add u11 i) j)
                                   ((1 (f2cl-lib:int-add n nb 1))
                                    (1 *))
                                   work-%offset%))
          label100053))
      label100052))))
 (setf cut (f2cl-lib:int-add cut nnb))
 (go label100030)
label100031
 (setf i n)
label100054
 (if (not (>= i 1))
     (go label100055))
 (cond
  ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
   (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
   (if (< i ip)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
           (csyswapr uplo n a lda i ip)
         (declare (ignore var-2))
         (when var-0 (setf uplo var-0))
         (when var-1 (setf n var-1))
         (when var-3 (setf lda var-3))
         (when var-4 (setf i var-4))
         (when var-5 (setf ip var-5))))
   (if (> i ip)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
           (csyswapr uplo n a lda ip i)
         (declare (ignore var-2))
         (when var-0 (setf uplo var-0))
         (when var-1 (setf n var-1))
         (when var-3 (setf lda var-3))
         (when var-4 (setf ip var-4))
         (when var-5 (setf i var-5)))))
  (t
   (setf ip
           (f2cl-lib:int-sub
            (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)))
   (if (< i ip)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
           (csyswapr uplo n a lda i ip)
         (declare (ignore var-2))
         (when var-0 (setf uplo var-0))
         (when var-1 (setf n var-1))
         (when var-3 (setf lda var-3))
         (when var-4 (setf i var-4))
         (when var-5 (setf ip var-5))))
   (if (> i ip)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
```

```
                    (csyswapr uplo n a lda ip i)
                  (declare (ignore var-2))
                  (when var-0 (setf uplo var-0))
                  (when var-1 (setf n var-1))
                  (when var-3 (setf lda var-3))
                  (when var-4 (setf ip var-4))
                  (when var-5 (setf i var-5))))
            (setf i (f2cl-lib:int-sub i 1))))
          (setf i (f2cl-lib:int-sub i 1))
          (go label100054)
         label100055)))
       (go end_label)
      end_label
       (return (values uplo n nil lda nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csytri2x
                 fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((string)
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::integer4
                                             (*))
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4))
                                          :return-values
                                          '(fortran-to-lisp::uplo
                                            fortran-to-lisp::n nil
                                            fortran-to-lisp::lda nil nil nil
                                            fortran-to-lisp::info)
                                          :calls
                                          '(fortran-to-lisp::csyconv))))
```

———-

60.21 csytri_rook

60.21.1 Axiom unit tests

— csytrirook.input —

```
)set break resume
)sys rm -f csytrirook.output
)spool csytrirook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.21.2 Axiom help page

— csytri_rook.help —

```
====================================================================
csytri_rook examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csytri_rook (UPLO, N, A, LDA, IPIV, WORK, INFO)
           CSYTRI_ROOK

Function/Subroutine Documentation
   subroutine csytri_rook (character UPLO, integer N, complex, dimension( lda,
       * ) A, integer LDA, integer, dimension( * ) IPIV, complex, dimension( *
       ) WORK, integer INFO)
       CSYTRI_ROOK

Purpose:
            CSYTRI_ROOK computes the inverse of a complex symmetric
            matrix A using the factorization A = U*D*U**T or A = L*D*L**T
            computed by CSYTRF_ROOK.
```

```
Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the block diagonal matrix D and the multipliers
   used to obtain the factor U or L as computed by CSYTRF_ROOK.

   On exit, if INFO = 0, the (symmetric) inverse of the original
   matrix.  If UPLO = 'U', the upper triangular part of the
   inverse is formed and the part of A below the diagonal is not
   referenced; if UPLO = 'L' the lower triangular part of the
   inverse is formed and the part of A above the diagonal is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by CSYTRF_ROOK.

 WORK
   WORK is COMPLEX array, dimension (N)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value
   > 0: if INFO = i, D(i,i) = 0; the matrix is singular and its
        inverse could not be computed.
```

60.21.3 fortran code

— csytri_rook.f —

```
*  =====================================================================
      SUBROUTINE CSYTRI_ROOK( UPLO, N, A, LDA, IPIV, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            CONE, CZERO
      PARAMETER          ( CONE = ( 1.0E+0, 0.0E+0 ),
     $                   CZERO = ( 0.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            K, KP, KSTEP
      COMPLEX            AK, AKKP1, AKP1, D, T, TEMP
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      COMPLEX            CDOTU
      EXTERNAL           LSAME, CDOTU
*     ..
*     .. External Subroutines ..
      EXTERNAL           CCOPY, CSWAP, CSYMV, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
```

```
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CSYTRI_ROOK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
         DO 10 INFO = N, 1, -1
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.CZERO )
     $         RETURN
   10    CONTINUE
      ELSE
*
*        Lower triangular storage: examine D from top to bottom.
*
         DO 20 INFO = 1, N
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.CZERO )
     $         RETURN
   20    CONTINUE
      END IF
      INFO = 0
*
      IF( UPPER ) THEN
*
*        Compute inv(A) from the factorization A = U*D*U**T.
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = 1
   30    CONTINUE
*
*        If K > N, exit from loop.
```

```
*
         IF( K.GT.N )
     $      GO TO 40
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Invert the diagonal block.
*
            A( K, K ) = CONE / A( K, K )
*
*           Compute column K of the inverse.
*
            IF( K.GT.1 ) THEN
               CALL CCOPY( K-1, A( 1, K ), 1, WORK, 1 )
               CALL CSYMV( UPLO, K-1, -CONE, A, LDA, WORK, 1, CZERO,
     $                     A( 1, K ), 1 )
               A( K, K ) = A( K, K ) - CDOTU( K-1, WORK, 1, A( 1, K ),
     $                     1 )
            END IF
            KSTEP = 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Invert the diagonal block.
*
            T = A( K, K+1 )
            AK = A( K, K ) / T
            AKP1 = A( K+1, K+1 ) / T
            AKKP1 = A( K, K+1 ) / T
            D = T*( AK*AKP1-CONE )
            A( K, K ) = AKP1 / D
            A( K+1, K+1 ) = AK / D
            A( K, K+1 ) = -AKKP1 / D
*
*           Compute columns K and K+1 of the inverse.
*
            IF( K.GT.1 ) THEN
               CALL CCOPY( K-1, A( 1, K ), 1, WORK, 1 )
               CALL CSYMV( UPLO, K-1, -CONE, A, LDA, WORK, 1, CZERO,
     $                     A( 1, K ), 1 )
               A( K, K ) = A( K, K ) - CDOTU( K-1, WORK, 1, A( 1, K ),
     $                     1 )
               A( K, K+1 ) = A( K, K+1 ) -
     $                       CDOTU( K-1, A( 1, K ), 1, A( 1, K+1 ), 1 )
               CALL CCOPY( K-1, A( 1, K+1 ), 1, WORK, 1 )
               CALL CSYMV( UPLO, K-1, -CONE, A, LDA, WORK, 1, CZERO,
     $                     A( 1, K+1 ), 1 )
```

```
               A( K+1, K+1 ) = A( K+1, K+1 ) -
     $                         CDOTU( K-1, WORK, 1, A( 1, K+1 ), 1 )
            END IF
            KSTEP = 2
         END IF
*
         IF( KSTEP.EQ.1 ) THEN
*
*           Interchange rows and columns K and IPIV(K) in the leading
*           submatrix A(1:k+1,1:k+1)
*
            KP = IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.GT.1 )
     $             CALL CSWAP( KP-1, A( 1, K ), 1, A( 1, KP ), 1 )
               CALL CSWAP( K-KP-1, A( KP+1, K ), 1, A( KP, KP+1 ), LDA )
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
            END IF
         ELSE
*
*           Interchange rows and columns K and K+1 with -IPIV(K) and
*           -IPIV(K+1)in the leading submatrix A(1:k+1,1:k+1)
*
            KP = -IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.GT.1 )
     $            CALL CSWAP( KP-1, A( 1, K ), 1, A( 1, KP ), 1 )
               CALL CSWAP( K-KP-1, A( KP+1, K ), 1, A( KP, KP+1 ), LDA )
*
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
               TEMP = A( K, K+1 )
               A( K, K+1 ) = A( KP, K+1 )
               A( KP, K+1 ) = TEMP
            END IF
*
            K = K + 1
            KP = -IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.GT.1 )
     $            CALL CSWAP( KP-1, A( 1, K ), 1, A( 1, KP ), 1 )
               CALL CSWAP( K-KP-1, A( KP+1, K ), 1, A( KP, KP+1 ), LDA )
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
            END IF
         END IF
```

```
*
         K = K + 1
         GO TO 30
   40    CONTINUE
*
      ELSE
*
*        Compute inv(A) from the factorization A = L*D*L**T.
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = N
   50    CONTINUE
*
*        If K < 1, exit from loop.
*
         IF( K.LT.1 )
     $      GO TO 60
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Invert the diagonal block.
*
            A( K, K ) = CONE / A( K, K )
*
*           Compute column K of the inverse.
*
            IF( K.LT.N ) THEN
               CALL CCOPY( N-K, A( K+1, K ), 1, WORK, 1 )
               CALL CSYMV( UPLO, N-K,-CONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                     CZERO, A( K+1, K ), 1 )
               A( K, K ) = A( K, K ) - CDOTU( N-K, WORK, 1, A( K+1, K ),
     $                     1 )
            END IF
            KSTEP = 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Invert the diagonal block.
*
            T = A( K, K-1 )
            AK = A( K-1, K-1 ) / T
            AKP1 = A( K, K ) / T
            AKKP1 = A( K, K-1 ) / T
            D = T*( AK*AKP1-CONE )
            A( K-1, K-1 ) = AKP1 / D
```

```
            A( K, K ) = AK / D
            A( K, K-1 ) = -AKKP1 / D
*
*           Compute columns K-1 and K of the inverse.
*
            IF( K.LT.N ) THEN
               CALL CCOPY( N-K, A( K+1, K ), 1, WORK, 1 )
               CALL CSYMV( UPLO, N-K,-CONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                     CZERO, A( K+1, K ), 1 )
               A( K, K ) = A( K, K ) - CDOTU( N-K, WORK, 1, A( K+1, K ),
     $                     1 )
               A( K, K-1 ) = A( K, K-1 ) -
     $                       CDOTU( N-K, A( K+1, K ), 1, A( K+1, K-1 ),
     $                       1 )
               CALL CCOPY( N-K, A( K+1, K-1 ), 1, WORK, 1 )
               CALL CSYMV( UPLO, N-K,-CONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                     CZERO, A( K+1, K-1 ), 1 )
               A( K-1, K-1 ) = A( K-1, K-1 ) -
     $                         CDOTU( N-K, WORK, 1, A( K+1, K-1 ), 1 )
            END IF
            KSTEP = 2
         END IF
*
         IF( KSTEP.EQ.1 ) THEN
*
*           Interchange rows and columns K and IPIV(K) in the trailing
*           submatrix A(k-1:n,k-1:n)
*
            KP = IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.LT.N )
     $            CALL CSWAP( N-KP, A( KP+1, K ), 1, A( KP+1, KP ), 1 )
               CALL CSWAP( KP-K-1, A( K+1, K ), 1, A( KP, K+1 ), LDA )
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
            END IF
         ELSE
*
*           Interchange rows and columns K and K-1 with -IPIV(K) and
*           -IPIV(K-1) in the trailing submatrix A(k-1:n,k-1:n)
*
            KP = -IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.LT.N )
     $            CALL CSWAP( N-KP, A( KP+1, K ), 1, A( KP+1, KP ), 1 )
               CALL CSWAP( KP-K-1, A( K+1, K ), 1, A( KP, K+1 ), LDA )
*
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
```

```
               A( KP, KP ) = TEMP
               TEMP = A( K, K-1 )
               A( K, K-1 ) = A( KP, K-1 )
               A( KP, K-1 ) = TEMP
            END IF
*
            K = K - 1
            KP = -IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.LT.N )
     $            CALL CSWAP( N-KP, A( KP+1, K ), 1, A( KP+1, KP ), 1 )
               CALL CSWAP( KP-K-1, A( K+1, K ), 1, A( KP, K+1 ), LDA )
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
            END IF
         END IF
*
         K = K - 1
         GO TO 50
   60    CONTINUE
      END IF
*
      RETURN
*
*     End of CSYTRI_ROOK
*
      END
```

60.21.4 lisp code

— csytri_rook.lisp —

```
(let* ((cone (f2cl-lib:cmplx 1.0 0.0)) (czero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) cone)
           (type (f2cl-lib:complex8) czero)
           (ignorable cone czero))
  (defun csytri_rook (uplo n a lda ipiv work info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
```

```
   (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
(prog ((ak #C(0.0 0.0)) (akkp1 #C(0.0 0.0)) (akp1 #C(0.0 0.0))
       (d #C(0.0 0.0)) (t$ #C(0.0 0.0)) (temp #C(0.0 0.0)) (k 0) (kp 0)
       (kstep 0) (upper nil))
  (declare (type (f2cl-lib:complex8) ak akkp1 akp1 d t$ temp)
           (type (f2cl-lib:integer4) k kp kstep)
           (type f2cl-lib:logical upper))
  (setf info 0)
  (setf upper
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
  (cond
   ((and (not upper)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4)))
  (cond
   ((/= info 0) (xerbla "CSYTRI_ROOK" (f2cl-lib:int-sub info))
    (go end_label)))
  (if (= n 0)
      (go end_label))
  (cond
   (upper
    (f2cl-lib:fdo (info n (f2cl-lib:int-add info (f2cl-lib:int-sub 1)))
                  ((> info 1) nil)
      (tagbody
        (if (and
             (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
                0)
             (=
              (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                             a-%offset%)
              czero))
            (go end_label))
      label10)))
   (t
    (f2cl-lib:fdo (info 1 (f2cl-lib:int-add info 1))
                  ((> info n) nil)
      (tagbody
        (if (and
```

```
              (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
                 0)
              (=
               (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                              a-%offset%)
               czero))
             (go end_label))
     label20))))
(setf info 0)
(cond
 (upper
  (tagbody
    (setf k 1)
   label30
    (if (> k n)
        (go label40))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
              (/ cone
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%)))
      (cond
       ((> k 1)
        (ccopy (f2cl-lib:int-sub k 1)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                               ((1 lda) (1 *)) a-%offset%)
         1 work 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9)
            (csymv uplo (f2cl-lib:int-sub k 1) (- cone) a lda work 1
             czero
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare
           (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
            var-9))
          (setf uplo var-0))
        (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                (-
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%)
                 (cdotu (f2cl-lib:int-sub k 1) work 1
                  (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                        (1 k) ((1 lda) (1 *))
                                        a-%offset%)
                  1)))))
      (setf kstep 1))
```

```
(t
 (setf t$
        (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                       ((1 lda) (1 *)) a-%offset%))
 (setf ak
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         t$))
 (setf akp1
        (/
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-add k 1)
                         (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
 (setf akkp1
        (/
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
 (setf d (* t$ (- (* ak akp1) cone)))
 (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (/ akp1 d))
 (setf (f2cl-lib:fref a-%data%
                      ((f2cl-lib:int-add k 1)
                       (f2cl-lib:int-add k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (/ ak d))
 (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (/ (- akkp1) d))
 (cond
  ((> k 1)
   (ccopy (f2cl-lib:int-sub k 1)
    (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                          ((1 lda) (1 *)) a-%offset%)
    1 work 1)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9)
       (csymv uplo (f2cl-lib:int-sub k 1) (- cone) a lda work 1
        czero
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                              ((1 lda) (1 *)) a-%offset%)
        1)
     (declare
      (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9))
     (setf uplo var-0))
```

```
(setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         (cdotu (f2cl-lib:int-sub k 1) work 1
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                (1 k) ((1 lda) (1 *))
                                a-%offset%)
          1)))
(setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
         (cdotu (f2cl-lib:int-sub k 1)
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                (1 k) ((1 lda) (1 *))
                                a-%offset%)
          1
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                (1 (f2cl-lib:int-add k 1))
                                ((1 lda) (1 *)) a-%offset%)
          1)))
(ccopy (f2cl-lib:int-sub k 1)
 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                       (1 (f2cl-lib:int-add k 1))
                       ((1 lda) (1 *)) a-%offset%)
 1 work 1)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (csymv uplo (f2cl-lib:int-sub k 1) (- cone) a lda work 1
     czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                           (1 (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
    var-9))
  (setf uplo var-0))
(setf (f2cl-lib:fref a-%data%
                      ((f2cl-lib:int-add k 1)
                       (f2cl-lib:int-add k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-add k 1)
                         (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
```

```
                (cdotu (f2cl-lib:int-sub k 1) work 1
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                       (1 (f2cl-lib:int-add k 1))
                                       ((1 lda) (1 *)) a-%offset%)
                 1)))))
  (setf kstep 2)))
(cond
 ((= kstep 1)
  (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
  (cond
   ((/= kp k)
    (if (> kp 1)
        (cswap (f2cl-lib:int-sub kp 1)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 kp)
                               ((1 lda) (1 *)) a-%offset%)
         1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (cswap (f2cl-lib:int-sub k kp 1)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               ((+ kp 1) k) ((1 lda) (1 *))
                               a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               (kp (f2cl-lib:int-add kp 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-0 var-1 var-2 var-3))
      (when var-4 (setf lda var-4)))
    (setf temp
            (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%)
            temp))))
 (t
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
  (cond
   ((/= kp k)
    (if (> kp 1)
        (cswap (f2cl-lib:int-sub kp 1)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                               ((1 lda) (1 *)) a-%offset%)
```

```
        1
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 kp)
                              ((1 lda) (1 *)) a-%offset%)
        1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (cswap (f2cl-lib:int-sub k kp 1)
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                              ((+ kp 1) k) ((1 lda) (1 *))
                              a-%offset%)
        1
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                              (kp (f2cl-lib:int-add kp 1))
                              ((1 lda) (1 *)) a-%offset%)
        lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (setf temp
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
          (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                       a-%offset%)
          temp)
  (setf temp
          (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                         ((1 lda) (1 *)) a-%offset%))
  (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                       ((1 lda) (1 *)) a-%offset%)
          (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-add k 1))
                         ((1 lda) (1 *)) a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-add k 1))
                       ((1 lda) (1 *)) a-%offset%)
          temp)))
(setf k (f2cl-lib:int-add k 1))
(setf kp
        (f2cl-lib:int-sub
         (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
(cond
 ((/= kp k)
  (if (> kp 1)
      (cswap (f2cl-lib:int-sub kp 1)
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                              ((1 lda) (1 *)) a-%offset%)
        1
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 kp)
                              ((1 lda) (1 *)) a-%offset%)
        1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
            (cswap (f2cl-lib:int-sub k kp 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   ((+ kp 1) k) ((1 lda) (1 *))
                                   a-%offset%)
             1
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   (kp (f2cl-lib:int-add kp 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
           (declare (ignore var-0 var-1 var-2 var-3))
           (when var-4 (setf lda var-4)))
         (setf temp
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%))
         (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                 (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                                a-%offset%))
         (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                              a-%offset%)
                 temp)))))
    (setf k (f2cl-lib:int-add k 1))
    (go label30)
   label40))
 (t
  (tagbody
    (setf k n)
   label50
    (if (< k 1)
        (go label60))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
              (/ cone
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%)))
      (cond
       ((< k n)
        (ccopy (f2cl-lib:int-sub n k)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 ((+ k 1) k)
                               ((1 lda) (1 *)) a-%offset%)
         1 work 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9)
            (csymv uplo (f2cl-lib:int-sub n k) (- cone)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   ((+ k 1) (f2cl-lib:int-add k 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda work 1 czero
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
```

```
                                ((+ k 1) k) ((1 lda) (1 *))
                                a-%offset%)
        1)
      (declare
       (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9))
      (setf uplo var-0))
    (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
            (-
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)
             (cdotu (f2cl-lib:int-sub n k) work 1
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                    ((+ k 1) k) ((1 lda) (1 *))
                                    a-%offset%)
              1)))))
  (setf kstep 1))
 (t
  (setf t$
          (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                         ((1 lda) (1 *)) a-%offset%))
  (setf ak
          (/
           (f2cl-lib:fref a-%data%
                          ((f2cl-lib:int-sub k 1)
                           (f2cl-lib:int-sub k 1))
                          ((1 lda) (1 *)) a-%offset%)
           t$))
  (setf akp1
          (/
           (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                          a-%offset%)
           t$))
  (setf akkp1
          (/
           (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                          ((1 lda) (1 *)) a-%offset%)
           t$))
  (setf d (* t$ (- (* ak akp1) cone)))
  (setf (f2cl-lib:fref a-%data%
                       ((f2cl-lib:int-sub k 1)
                        (f2cl-lib:int-sub k 1))
                       ((1 lda) (1 *)) a-%offset%)
          (/ akp1 d))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
          (/ ak d))
  (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                       ((1 lda) (1 *)) a-%offset%)
          (/ (- akkp1) d))
  (cond
```

```
((< k n)
 (ccopy (f2cl-lib:int-sub n k)
  (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 ((+ k 1) k)
                        ((1 lda) (1 *)) a-%offset%)
  1 work 1)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9)
     (csymv uplo (f2cl-lib:int-sub n k) (- cone)
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                            ((+ k 1) (f2cl-lib:int-add k 1))
                            ((1 lda) (1 *)) a-%offset%)
      lda work 1 czero
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                            ((+ k 1) k) ((1 lda) (1 *))
                            a-%offset%)
      1)
   (declare
    (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9))
   (setf uplo var-0))
 (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
         (-
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%)
          (cdotu (f2cl-lib:int-sub n k) work 1
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 ((+ k 1) k) ((1 lda) (1 *))
                                 a-%offset%)
           1)))
 (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                       ((1 lda) (1 *)) a-%offset%)
         (-
          (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                         ((1 lda) (1 *)) a-%offset%)
          (cdotu (f2cl-lib:int-sub n k)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 ((+ k 1) k) ((1 lda) (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                 ((+ k 1)
                                  (f2cl-lib:int-sub k 1))
                                 ((1 lda) (1 *)) a-%offset%)
           1)))
 (ccopy (f2cl-lib:int-sub n k)
  (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                        ((+ k 1) (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
  1 work 1)
```

```
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9)
        (csymv uplo (f2cl-lib:int-sub n k) (- cone)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               ((+ k 1) (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda work 1 czero
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               ((+ k 1) (f2cl-lib:int-sub k 1))
                               ((1 lda) (1 *)) a-%offset%)
         1)
      (declare
       (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9))
      (setf uplo var-0))
    (setf (f2cl-lib:fref a-%data%
                         ((f2cl-lib:int-sub k 1)
                          (f2cl-lib:int-sub k 1))
                         ((1 lda) (1 *)) a-%offset%)
            (-
             (f2cl-lib:fref a-%data%
                            ((f2cl-lib:int-sub k 1)
                             (f2cl-lib:int-sub k 1))
                            ((1 lda) (1 *)) a-%offset%)
             (cdotu (f2cl-lib:int-sub n k) work 1
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                    ((+ k 1)
                                     (f2cl-lib:int-sub k 1))
                                    ((1 lda) (1 *)) a-%offset%)
              1)))))
  (setf kstep 2)))
(cond
 ((= kstep 1)
  (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
  (cond
   ((/= kp k)
    (if (< kp n)
        (cswap (f2cl-lib:int-sub n kp)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               ((+ kp 1) k) ((1 lda) (1 *))
                               a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               ((+ kp 1) kp) ((1 lda) (1 *))
                               a-%offset%)
         1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (cswap (f2cl-lib:int-sub kp k 1)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
```

```
                                ((+ k 1) k) ((1 lda) (1 *))
                                a-%offset%)
          1
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                (kp (f2cl-lib:int-add k 1))
                                ((1 lda) (1 *)) a-%offset%)
          lda)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf lda var-4)))
      (setf temp
              (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                             a-%offset%))
      (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                             a-%offset%))
      (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                           a-%offset%)
              temp))))
  (t
   (setf kp
           (f2cl-lib:int-sub
            (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
   (cond
    ((/= kp k)
     (if (< kp n)
         (cswap (f2cl-lib:int-sub n kp)
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                ((+ kp 1) k) ((1 lda) (1 *))
                                a-%offset%)
          1
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                ((+ kp 1) kp) ((1 lda) (1 *))
                                a-%offset%)
          1))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (cswap (f2cl-lib:int-sub kp k 1)
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                ((+ k 1) k) ((1 lda) (1 *))
                                a-%offset%)
          1
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                (kp (f2cl-lib:int-add k 1))
                                ((1 lda) (1 *)) a-%offset%)
          lda)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf lda var-4)))
     (setf temp
              (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                             a-%offset%))
     (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
```

```
            (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%)
            temp)
    (setf temp
            (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                           ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                         ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-sub k 1))
                           ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-sub k 1))
                         ((1 lda) (1 *)) a-%offset%)
            temp)))
  (setf k (f2cl-lib:int-sub k 1))
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
  (cond
   ((/= kp k)
    (if (< kp n)
        (cswap (f2cl-lib:int-sub n kp)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               ((+ kp 1) k) ((1 lda) (1 *))
                               a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               ((+ kp 1) kp) ((1 lda) (1 *))
                               a-%offset%)
         1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (cswap (f2cl-lib:int-sub kp k 1)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               ((+ k 1) k) ((1 lda) (1 *))
                               a-%offset%)
         1
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               (kp (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-0 var-1 var-2 var-3))
      (when var-4 (setf lda var-4)))
    (setf temp
            (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
```

```
                                    a-%offset%)
                        temp)))))
            (setf k (f2cl-lib:int-sub k 1))
            (go label50)
           label60)))
        (go end_label)
       end_label
        (return (values uplo nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csytri_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::csymv))))
```

———-

60.22 csytrs

60.22.1 Axiom unit tests

— csytrs.input —

```
)set break resume
)sys rm -f csytrs.output
)spool csytrs.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

60.22.2 Axiom help page

— **csytrs.help** —

```
==================================================================
csytrs examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csytrs (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO)
           CSYTRS

Function/Subroutine Documentation
   subroutine csytrs (character UPLO, integer N, integer NRHS, complex,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       complex, dimension( ldb, * ) B, integer LDB, integer INFO)
       CSYTRS

Purpose:
   CSYTRS solves a system of linear equations A*X = B with a complex
   symmetric matrix A using the factorization A = U*D*U**T or
   A = L*D*L**T computed by CSYTRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
```

```
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrix B.  NRHS >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  The block diagonal matrix D and the multipliers used to
  obtain the factor U or L as computed by CSYTRF.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by CSYTRF.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the right hand side matrix B.
  On exit, the solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

60.22.3 fortran code

— **csytrs.f** —

```
*  =====================================================================
      SUBROUTINE CSYTRS( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
```

```
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ONE
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            J, K, KP
      COMPLEX            AK, AKM1, AKM1K, BK, BKM1, DENOM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEMV, CGERU, CSCAL, CSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CSYTRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
```

```
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*        Solve A*X = B, where A = U*D*U**T.
*
*        First solve U*D*X = B, overwriting B with X.
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop.
*
         IF( K.LT.1 )
     $      GO TO 30
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
*           Multiply by inv(U(K)), where U(K) is the transformation
*           stored in column K of A.
*
            CALL CGERU( K-1, NRHS, -ONE, A( 1, K ), 1, B( K, 1 ), LDB,
     $                  B( 1, 1 ), LDB )
*
*           Multiply by the inverse of the diagonal block.
*
            CALL CSCAL( NRHS, ONE / A( K, K ), B( K, 1 ), LDB )
            K = K - 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Interchange rows K-1 and -IPIV(K).
*
            KP = -IPIV( K )
            IF( KP.NE.K-1 )
     $         CALL CSWAP( NRHS, B( K-1, 1 ), LDB, B( KP, 1 ), LDB )
```

```
*
*          Multiply by inv(U(K)), where U(K) is the transformation
*          stored in columns K-1 and K of A.
*
           CALL CGERU( K-2, NRHS, -ONE, A( 1, K ), 1, B( K, 1 ), LDB,
     $                 B( 1, 1 ), LDB )
           CALL CGERU( K-2, NRHS, -ONE, A( 1, K-1 ), 1, B( K-1, 1 ),
     $                 LDB, B( 1, 1 ), LDB )
*
*          Multiply by the inverse of the diagonal block.
*
           AKM1K = A( K-1, K )
           AKM1 = A( K-1, K-1 ) / AKM1K
           AK = A( K, K ) / AKM1K
           DENOM = AKM1*AK - ONE
           DO 20 J = 1, NRHS
              BKM1 = B( K-1, J ) / AKM1K
              BK = B( K, J ) / AKM1K
              B( K-1, J ) = ( AK*BKM1-BK ) / DENOM
              B( K, J ) = ( AKM1*BK-BKM1 ) / DENOM
   20      CONTINUE
           K = K - 2
        END IF
*
        GO TO 10
   30   CONTINUE
*
*       Next solve U**T *X = B, overwriting B with X.
*
*       K is the main loop index, increasing from 1 to N in steps of
*       1 or 2, depending on the size of the diagonal blocks.
*
        K = 1
   40   CONTINUE
*
*       If K > N, exit from loop.
*
        IF( K.GT.N )
     $     GO TO 50
*
        IF( IPIV( K ).GT.0 ) THEN
*
*          1 x 1 diagonal block
*
*          Multiply by inv(U**T(K)), where U(K) is the transformation
*          stored in column K of A.
*
           CALL CGEMV( 'Transpose', K-1, NRHS, -ONE, B, LDB, A( 1, K ),
     $                 1, ONE, B( K, 1 ), LDB )
*
```

```
*          Interchange rows K and IPIV(K).
*
           KP = IPIV( K )
           IF( KP.NE.K )
     $        CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
           K = K + 1
        ELSE
*
*          2 x 2 diagonal block
*
*          Multiply by inv(U**T(K+1)), where U(K+1) is the transformation
*          stored in columns K and K+1 of A.
*
           CALL CGEMV( 'Transpose', K-1, NRHS, -ONE, B, LDB, A( 1, K ),
     $                 1, ONE, B( K, 1 ), LDB )
           CALL CGEMV( 'Transpose', K-1, NRHS, -ONE, B, LDB,
     $                 A( 1, K+1 ), 1, ONE, B( K+1, 1 ), LDB )
*
*          Interchange rows K and -IPIV(K).
*
           KP = -IPIV( K )
           IF( KP.NE.K )
     $        CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
           K = K + 2
        END IF
*
        GO TO 40
   50   CONTINUE
*
      ELSE
*
*       Solve A*X = B, where A = L*D*L**T.
*
*       First solve L*D*X = B, overwriting B with X.
*
*       K is the main loop index, increasing from 1 to N in steps of
*       1 or 2, depending on the size of the diagonal blocks.
*
        K = 1
   60   CONTINUE
*
*       If K > N, exit from loop.
*
        IF( K.GT.N )
     $     GO TO 80
*
        IF( IPIV( K ).GT.0 ) THEN
*
*          1 x 1 diagonal block
*
```

```
*          Interchange rows K and IPIV(K).
*
           KP = IPIV( K )
           IF( KP.NE.K )
     $        CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
*          Multiply by inv(L(K)), where L(K) is the transformation
*          stored in column K of A.
*
           IF( K.LT.N )
     $        CALL CGERU( N-K, NRHS, -ONE, A( K+1, K ), 1, B( K, 1 ),
     $                    LDB, B( K+1, 1 ), LDB )
*
*          Multiply by the inverse of the diagonal block.
*
           CALL CSCAL( NRHS, ONE / A( K, K ), B( K, 1 ), LDB )
           K = K + 1
        ELSE
*
*          2 x 2 diagonal block
*
*          Interchange rows K+1 and -IPIV(K).
*
           KP = -IPIV( K )
           IF( KP.NE.K+1 )
     $        CALL CSWAP( NRHS, B( K+1, 1 ), LDB, B( KP, 1 ), LDB )
*
*          Multiply by inv(L(K)), where L(K) is the transformation
*          stored in columns K and K+1 of A.
*
           IF( K.LT.N-1 ) THEN
              CALL CGERU( N-K-1, NRHS, -ONE, A( K+2, K ), 1, B( K, 1 ),
     $                    LDB, B( K+2, 1 ), LDB )
              CALL CGERU( N-K-1, NRHS, -ONE, A( K+2, K+1 ), 1,
     $                    B( K+1, 1 ), LDB, B( K+2, 1 ), LDB )
           END IF
*
*          Multiply by the inverse of the diagonal block.
*
           AKM1K = A( K+1, K )
           AKM1 = A( K, K ) / AKM1K
           AK = A( K+1, K+1 ) / AKM1K
           DENOM = AKM1*AK - ONE
           DO 70 J = 1, NRHS
              BKM1 = B( K, J ) / AKM1K
              BK = B( K+1, J ) / AKM1K
              B( K, J ) = ( AK*BKM1-BK ) / DENOM
              B( K+1, J ) = ( AKM1*BK-BKM1 ) / DENOM
   70      CONTINUE
           K = K + 2
```

```
         END IF
*
         GO TO 60
   80    CONTINUE
*
*        Next solve L**T *X = B, overwriting B with X.
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = N
   90    CONTINUE
*
*        If K < 1, exit from loop.
*
         IF( K.LT.1 )
     $      GO TO 100
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Multiply by inv(L**T(K)), where L(K) is the transformation
*           stored in column K of A.
*
            IF( K.LT.N )
     $         CALL CGEMV( 'Transpose', N-K, NRHS, -ONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K ), 1, ONE, B( K, 1 ), LDB )
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K = K - 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Multiply by inv(L**T(K-1)), where L(K-1) is the transformation
*           stored in columns K-1 and K of A.
*
            IF( K.LT.N ) THEN
               CALL CGEMV( 'Transpose', N-K, NRHS, -ONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K ), 1, ONE, B( K, 1 ), LDB )
               CALL CGEMV( 'Transpose', N-K, NRHS, -ONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K-1 ), 1, ONE, B( K-1, 1 ),
     $                     LDB )
            END IF
*
```

```
*           Interchange rows K and -IPIV(K).
*
            KP = -IPIV( K )
            IF( KP.NE.K )
     $         CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K = K - 2
         END IF
*
         GO TO 90
  100    CONTINUE
      END IF
*
      RETURN
*
*     End of CSYTRS
*
      END
```

———-

60.22.4 lisp code

— csytrs.lisp —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (ignorable one))
  (defun csytrs (uplo n nrhs a lda ipiv b ldb$ info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) b a)
             (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((ak #C(0.0 0.0)) (akm1 #C(0.0 0.0)) (akm1k #C(0.0 0.0))
             (bk #C(0.0 0.0)) (bkm1 #C(0.0 0.0)) (denom #C(0.0 0.0)) (j 0)
             (k 0) (kp 0) (upper nil))
        (declare (type (f2cl-lib:complex8) ak akm1 akm1k bk bkm1 denom)
                 (type (f2cl-lib:integer4) j k kp)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
```

```
          (when var-0 (setf uplo var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8)))
(cond
 ((/= info 0) (xerbla "CSYTRS" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= n 0) (= nrhs 0))
    (go end_label))
(cond
 (upper
  (tagbody
    (setf k n)
   label10
    (if (< k 1)
        (go label30))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (cswap nrhs
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (cgeru (f2cl-lib:int-sub k 1) nrhs (- one)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
```

```
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
    (when var-1 (setf nrhs var-1))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldb$ var-8)))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (cscal nrhs
       (/ one
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-1 var-2))
    (when var-0 (setf nrhs var-0))
    (when var-3 (setf ldb$ var-3)))
  (setf k (f2cl-lib:int-sub k 1)))
 (t
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
  (if (/= kp (f2cl-lib:int-sub k 1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (cswap nrhs
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                 ((+ k (f2cl-lib:int-sub 1)) 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-3))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (cgeru (f2cl-lib:int-sub k 2) nrhs (- one)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 1)
```

```
                          ((1 ldb$) (1 *)) b-%offset%)
     ldb$)
  (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
  (when var-1 (setf nrhs var-1))
  (when var-6 (setf ldb$ var-6))
  (when var-8 (setf ldb$ var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cgeru (f2cl-lib:int-sub k 2) nrhs (- one)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                          (1 (f2cl-lib:int-sub k 1))
                          ((1 lda) (1 *)) a-%offset%)
     1
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                          ((+ k (f2cl-lib:int-sub 1)) 1)
                          ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 1)
                          ((1 ldb$) (1 *)) b-%offset%)
     ldb$)
  (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
  (when var-1 (setf nrhs var-1))
  (when var-6 (setf ldb$ var-6))
  (when var-8 (setf ldb$ var-8)))
(setf akm1k
        (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                       ((1 lda) (1 *)) a-%offset%))
(setf akm1
        (/
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-sub k 1)
                         (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         akm1k))
(setf ak
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         akm1k))
(setf denom (- (* akm1 ak) one))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf bkm1
            (/
             (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1) j)
                            ((1 ldb$) (1 *)) b-%offset%)
             akm1k))
    (setf bk
            (/
```

```
                 (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                b-%offset%)
                 akm1k))
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)
                (/ (- (* ak bkm1) bk) denom))
        (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                             b-%offset%)
                (/ (- (* akm1 bk) bkm1) denom))
      label20))
    (setf k (f2cl-lib:int-sub k 2))))
  (go label10)
 label30
  (setf k 1)
 label40
  (if (> k n)
      (go label50))
  (cond
   ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (cgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- one) b ldb$
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                               ((1 lda) (1 *)) a-%offset%)
         1 one
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
      (when var-2 (setf nrhs var-2))
      (when var-5 (setf ldb$ var-5))
      (when var-8 (setf one var-8))
      (when var-10 (setf ldb$ var-10)))
    (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
    (if (/= kp k)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (cswap nrhs
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$)
          (declare (ignore var-1 var-3))
          (when var-0 (setf nrhs var-0))
          (when var-2 (setf ldb$ var-2))
          (when var-4 (setf ldb$ var-4))))
    (setf k (f2cl-lib:int-add k 1)))
   (t
```

```
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (cgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- one) b ldb$
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                               ((1 lda) (1 *)) a-%offset%)
         1 one
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
      (when var-2 (setf nrhs var-2))
      (when var-5 (setf ldb$ var-5))
      (when var-8 (setf one var-8))
      (when var-10 (setf ldb$ var-10)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (cgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- one) b ldb$
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               (1 (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%)
         1 one
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 ((+ k 1) 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
      (when var-2 (setf nrhs var-2))
      (when var-5 (setf ldb$ var-5))
      (when var-8 (setf one var-8))
      (when var-10 (setf ldb$ var-10)))
    (setf kp
            (f2cl-lib:int-sub
             (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
    (if (/= kp k)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (cswap nrhs
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$)
          (declare (ignore var-1 var-3))
          (when var-0 (setf nrhs var-0))
          (when var-2 (setf ldb$ var-2))
          (when var-4 (setf ldb$ var-4))))
    (setf k (f2cl-lib:int-add k 2))))
  (go label40)
 label50))
```

```
(t
 (tagbody
   (setf k 1)
  label60
   (if (> k n)
       (go label80))
   (cond
    ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
     (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
     (if (/= kp k)
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (cswap nrhs
              (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                    ((1 ldb$) (1 *)) b-%offset%)
              ldb$
              (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                    ((1 ldb$) (1 *)) b-%offset%)
              ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
     (if (< k n)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
             (cgeru (f2cl-lib:int-sub n k) nrhs (- one)
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                    ((+ k 1) k) ((1 lda) (1 *))
                                    a-%offset%)
              1
              (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                    ((1 ldb$) (1 *)) b-%offset%)
              ldb$
              (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                    ((+ k 1) 1) ((1 ldb$) (1 *))
                                    b-%offset%)
              ldb$)
            (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
            (when var-1 (setf nrhs var-1))
            (when var-6 (setf ldb$ var-6))
            (when var-8 (setf ldb$ var-8))))
     (multiple-value-bind (var-0 var-1 var-2 var-3)
         (cscal nrhs
          (/ one
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%))
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
       (declare (ignore var-1 var-2))
```

```
    (when var-0 (setf nrhs var-0))
    (when var-3 (setf ldb$ var-3)))
  (setf k (f2cl-lib:int-add k 1)))
 (t
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
  (if (/= kp (f2cl-lib:int-add k 1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (cswap nrhs
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                 ((+ k 1) 1) ((1 ldb$) (1 *))
                                 b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-3))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
  (cond
   ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (cgeru (f2cl-lib:int-sub n k 1) nrhs (- one)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               ((+ k 2) k) ((1 lda) (1 *))
                               a-%offset%)
         1
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                               ((+ k 2) 1) ((1 ldb$) (1 *))
                               b-%offset%)
         ldb$)
      (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
      (when var-1 (setf nrhs var-1))
      (when var-6 (setf ldb$ var-6))
      (when var-8 (setf ldb$ var-8)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (cgeru (f2cl-lib:int-sub n k 1) nrhs (- one)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               ((+ k 2) (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                               ((+ k 1) 1) ((1 ldb$) (1 *))
```

```
                                b-%offset%)
            ldb$
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                ((+ k 2) 1) ((1 ldb$) (1 *))
                                b-%offset%)
            ldb$)
        (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
        (when var-1 (setf nrhs var-1))
        (when var-6 (setf ldb$ var-6))
        (when var-8 (setf ldb$ var-8)))))
    (setf akm1k
            (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                           ((1 lda) (1 *)) a-%offset%))
    (setf akm1
            (/
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)
             akm1k))
    (setf ak
            (/
             (f2cl-lib:fref a-%data%
                            ((f2cl-lib:int-add k 1)
                             (f2cl-lib:int-add k 1))
                            ((1 lda) (1 *)) a-%offset%)
             akm1k))
    (setf denom (- (* akm1 ak) one))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (setf bkm1
                (/
                 (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                b-%offset%)
                 akm1k))
        (setf bk
                (/
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                                ((1 ldb$) (1 *)) b-%offset%)
                 akm1k))
        (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                             b-%offset%)
                (/ (- (* ak bkm1) bk) denom))
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)
                (/ (- (* akm1 bk) bkm1) denom))
       label70))
    (setf k (f2cl-lib:int-add k 2))))
  (go label60)
 label80
  (setf k n)
```

```
label90
 (if (< k 1)
     (go label100))
 (cond
  ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
   (if (< k n)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
            var-9 var-10)
           (cgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- one)
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                  ((+ k 1) 1) ((1 ldb$) (1 *))
                                  b-%offset%)
            ldb$
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                  ((+ k 1) k) ((1 lda) (1 *))
                                  a-%offset%)
            1 one
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
         (declare
          (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
         (when var-2 (setf nrhs var-2))
         (when var-5 (setf ldb$ var-5))
         (when var-8 (setf one var-8))
         (when var-10 (setf ldb$ var-10))))
   (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
   (if (/= kp k)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (cswap nrhs
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
         (declare (ignore var-1 var-3))
         (when var-0 (setf nrhs var-0))
         (when var-2 (setf ldb$ var-2))
         (when var-4 (setf ldb$ var-4))))
   (setf k (f2cl-lib:int-sub k 1)))
  (t
   (cond
    ((< k n)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
          var-9 var-10)
         (cgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- one)
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
```

```
                                ((+ k 1) 1) ((1 ldb$) (1 *))
                                b-%offset%)
                ldb$
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                ((+ k 1) k) ((1 lda) (1 *))
                                a-%offset%)
                1 one
                (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
                ldb$)
            (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
            (when var-2 (setf nrhs var-2))
            (when var-5 (setf ldb$ var-5))
            (when var-8 (setf one var-8))
            (when var-10 (setf ldb$ var-10)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10)
              (cgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- one)
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                ((+ k 1) 1) ((1 ldb$) (1 *))
                                b-%offset%)
               ldb$
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                ((+ k 1) (f2cl-lib:int-sub k 1))
                                ((1 lda) (1 *)) a-%offset%)
               1 one
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                ((+ k (f2cl-lib:int-sub 1)) 1)
                                ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
            (when var-2 (setf nrhs var-2))
            (when var-5 (setf ldb$ var-5))
            (when var-8 (setf one var-8))
            (when var-10 (setf ldb$ var-10)))))
        (setf kp
                (f2cl-lib:int-sub
                 (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
        (if (/= kp k)
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (cswap nrhs
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
                 ldb$
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
                 ldb$)
              (declare (ignore var-1 var-3))
              (when var-0 (setf nrhs var-0))
```

```
                    (when var-2 (setf ldb$ var-2))
                    (when var-4 (setf ldb$ var-4))))
              (setf k (f2cl-lib:int-sub k 2))))
            (go label90)
           label100)))
        (go end_label)
       end_label
        (return (values uplo nil nrhs nil nil nil nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csytrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil
                                              fortran-to-lisp::nrhs nil nil nil
                                              nil fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

60.23 csytrs2

60.23.1 Axiom unit tests

— csytrs2.input —

```
)set break resume
)sys rm -f csytrs2.output
)spool csytrs2.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.23.2 Axiom help page

— **csytrs2.help** —

```
==================================================================
csytrs2 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csytrs2 (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK, INFO)
           CSYTRS2

Function/Subroutine Documentation
   subroutine csytrs2 (character UPLO, integer N, integer NRHS, complex,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       complex, dimension( ldb, * ) B, integer LDB, complex, dimension( * )
       WORK, integer INFO)
       CSYTRS2

Purpose:
   CSYTRS2 solves a system of linear equations A*X = B with a COMPLEX
   symmetric matrix A using the factorization A = U*D*U**T or
   A = L*D*L**T computed by CSYTRF and converted by CSYCONV.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
```

```
  N is INTEGER
  The order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrix B.  NRHS >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  The block diagonal matrix D and the multipliers used to
  obtain the factor U or L as computed by CSYTRF.
  Note that A is input / output. This might be counter-intuitive,
  and one may think that A is input only. A is input / output. This
  is because, at the start of the subroutine, we permute A in a
  "better" form and then we permute A back to its original form at the end.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by CSYTRF.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the right hand side matrix B.
  On exit, the solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

WORK
  WORK is COMPLEX array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

60.23.3 fortran code

— csytrs2.f —

```
*  =====================================================================
      SUBROUTINE CSYTRS2( UPLO, N, NRHS, A, LDA, IPIV, B, LDB,
     $                    WORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ONE
      PARAMETER          ( ONE = (1.0E+0,0.0E+0) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, IINFO, J, K, KP
      COMPLEX            AK, AKM1, AKM1K, BK, BKM1, DENOM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CSCAL, CSYCONV, CSWAP, CTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
```

```
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CSYTRS2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
*     Convert A
*
      CALL CSYCONV( UPLO, 'C', N, A, LDA, IPIV, WORK, IINFO )
*
      IF( UPPER ) THEN
*
*        Solve A*X = B, where A = U*D*U**T.
*
*       P**T * B
        K=N
        DO WHILE ( K .GE. 1 )
         IF( IPIV( K ).GT.0 ) THEN
*           1 x 1 diagonal block
*           Interchange rows K and IPIV(K).
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K=K-1
         ELSE
*           2 x 2 diagonal block
*           Interchange rows K-1 and -IPIV(K).
            KP = -IPIV( K )
            IF( KP.EQ.-IPIV( K-1 ) )
     $         CALL CSWAP( NRHS, B( K-1, 1 ), LDB, B( KP, 1 ), LDB )
            K=K-2
         END IF
        END DO
*
*  Compute (U \P**T * B) -> B    [ (U \P**T * B) ]
*
        CALL CTRSM('L','U','N','U',N,NRHS,ONE,A,LDA,B,LDB)
```

```
*
*  Compute D \ B -> B   [ D \ (U \P**T * B) ]
*
         I=N
         DO WHILE ( I .GE. 1 )
            IF( IPIV(I) .GT. 0 ) THEN
              CALL CSCAL( NRHS, ONE / A( I, I ), B( I, 1 ), LDB )
            ELSEIF ( I .GT. 1) THEN
               IF ( IPIV(I-1) .EQ. IPIV(I) ) THEN
                  AKM1K = WORK(I)
                  AKM1 = A( I-1, I-1 ) / AKM1K
                  AK = A( I, I ) / AKM1K
                  DENOM = AKM1*AK - ONE
                  DO 15 J = 1, NRHS
                     BKM1 = B( I-1, J ) / AKM1K
                     BK = B( I, J ) / AKM1K
                     B( I-1, J ) = ( AK*BKM1-BK ) / DENOM
                     B( I, J ) = ( AKM1*BK-BKM1 ) / DENOM
 15             CONTINUE
               I = I - 1
               ENDIF
            ENDIF
            I = I - 1
         END DO
*
*      Compute (U**T \ B) -> B   [ U**T \ (D \ (U \P**T * B) ) ]
*
         CALL CTRSM('L','U','T','U',N,NRHS,ONE,A,LDA,B,LDB)
*
*       P * B  [ P * (U**T \ (D \ (U \P**T * B) )) ]
*
        K=1
        DO WHILE ( K .LE. N )
         IF( IPIV( K ).GT.0 ) THEN
*           1 x 1 diagonal block
*           Interchange rows K and IPIV(K).
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K=K+1
         ELSE
*           2 x 2 diagonal block
*           Interchange rows K-1 and -IPIV(K).
            KP = -IPIV( K )
            IF( K .LT. N .AND. KP.EQ.-IPIV( K+1 ) )
     $         CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K=K+2
         ENDIF
        END DO
*
```

```
      ELSE
*
*        Solve A*X = B, where A = L*D*L**T.
*
*       P**T * B
        K=1
        DO WHILE ( K .LE. N )
         IF( IPIV( K ).GT.0 ) THEN
*           1 x 1 diagonal block
*           Interchange rows K and IPIV(K).
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K=K+1
         ELSE
*           2 x 2 diagonal block
*           Interchange rows K and -IPIV(K+1).
            KP = -IPIV( K+1 )
            IF( KP.EQ.-IPIV( K ) )
     $         CALL CSWAP( NRHS, B( K+1, 1 ), LDB, B( KP, 1 ), LDB )
            K=K+2
         ENDIF
        END DO
*
*  Compute (L \P**T * B) -> B    [ (L \P**T * B) ]
*
        CALL CTRSM('L','L','N','U',N,NRHS,ONE,A,LDA,B,LDB)
*
*  Compute D \ B -> B   [ D \ (L \P**T * B) ]
*
         I=1
         DO WHILE ( I .LE. N )
            IF( IPIV(I) .GT. 0 ) THEN
              CALL CSCAL( NRHS, ONE / A( I, I ), B( I, 1 ), LDB )
            ELSE
                  AKM1K = WORK(I)
                  AKM1 = A( I, I ) / AKM1K
                  AK = A( I+1, I+1 ) / AKM1K
                  DENOM = AKM1*AK - ONE
                  DO 25 J = 1, NRHS
                     BKM1 = B( I, J ) / AKM1K
                     BK = B( I+1, J ) / AKM1K
                     B( I, J ) = ( AK*BKM1-BK ) / DENOM
                     B( I+1, J ) = ( AKM1*BK-BKM1 ) / DENOM
 25               CONTINUE
                  I = I + 1
            ENDIF
            I = I + 1
         END DO
*
```

```
*  Compute (L**T \ B) -> B   [ L**T \ (D \ (L \P**T * B) ) ]
*
        CALL CTRSM('L','L','T','U',N,NRHS,ONE,A,LDA,B,LDB)
*
*       P * B  [ P * (L**T \ (D \ (L \P**T * B) )) ]
*
        K=N
        DO WHILE ( K .GE. 1 )
         IF( IPIV( K ).GT.0 ) THEN
*           1 x 1 diagonal block
*           Interchange rows K and IPIV(K).
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K=K-1
         ELSE
*           2 x 2 diagonal block
*           Interchange rows K-1 and -IPIV(K).
            KP = -IPIV( K )
            IF( K.GT.1 .AND. KP.EQ.-IPIV( K-1 ) )
     $         CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K=K-2
         ENDIF
        END DO
*
      END IF
*
*     Revert A
*
      CALL CSYCONV( UPLO, 'R', N, A, LDA, IPIV, WORK, IINFO )
*
      RETURN
*
*     End of CSYTRS2
*
      END
```

———-

60.23.4 lisp code

— **csytrs2.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (ignorable one))
  (defun csytrs2 (uplo n nrhs a lda ipiv b ldb$ work info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
```

```
          (type (array f2cl-lib:complex8 (*)) work b a)
          (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
          (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (b f2cl-lib:complex8 b-%data% b-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((ak #C(0.0 0.0)) (akm1 #C(0.0 0.0)) (akm1k #C(0.0 0.0))
          (bk #C(0.0 0.0)) (bkm1 #C(0.0 0.0)) (denom #C(0.0 0.0)) (i 0)
          (iinfo 0) (j 0) (k 0) (kp 0) (upper nil))
      (declare (type (f2cl-lib:complex8) ak akm1 akm1k bk bkm1 denom)
               (type (f2cl-lib:integer4) i iinfo j k kp)
               (type f2cl-lib:logical upper))
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (cond
       ((and (not upper)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -5))
       ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -8)))
      (cond
       ((/= info 0) (xerbla "CSYTRS2" (f2cl-lib:int-sub info))
        (go end_label)))
      (if (or (= n 0) (= nrhs 0))
          (go end_label))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (csyconv uplo "C" n a lda ipiv work iinfo)
        (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
        (setf uplo var-0)
        (setf iinfo var-7))
      (cond
       (upper
        (tagbody
          (setf k n)
```

```
label100000
 (if (not (>= k 1))
     (go label100001))
 (cond
  ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
   (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
   (if (/= kp k)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (cswap nrhs
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
          (declare (ignore var-1 var-3))
          (when var-0 (setf nrhs var-0))
          (when var-2 (setf ldb$ var-2))
          (when var-4 (setf ldb$ var-4))))
   (setf k (f2cl-lib:int-sub k 1)))
  (t
   (setf kp
           (f2cl-lib:int-sub
            (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
   (if (= kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                          ((1 *)) ipiv-%offset%)))
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (cswap nrhs
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                  ((+ k (f2cl-lib:int-sub 1)) 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
          (declare (ignore var-1 var-3))
          (when var-0 (setf nrhs var-0))
          (when var-2 (setf ldb$ var-2))
          (when var-4 (setf ldb$ var-4))))
   (setf k (f2cl-lib:int-sub k 2))))
 (go label100000)
label100001
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (ctrsm "L" "U" "N" "U" n nrhs one a lda b ldb$)
   (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
   (when var-4 (setf n var-4))
```

```
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10)))
  (setf i n)
 label100002
  (if (not (>= i 1))
      (go label100003))
  (cond
   ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (cscal nrhs
         (/ one
            (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                           a-%offset%))
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (i 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-1 var-2))
      (when var-0 (setf nrhs var-0))
      (when var-3 (setf ldb$ var-3))))
   ((> i 1)
    (cond
     ((=
       (f2cl-lib:fref ipiv
                      ((f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((1 *)))
       (f2cl-lib:fref ipiv (i) ((1 *))))
      (setf akm1k
              (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
      (setf akm1
              (/
               (f2cl-lib:fref a-%data%
                              ((f2cl-lib:int-sub i 1)
                               (f2cl-lib:int-sub i 1))
                              ((1 lda) (1 *)) a-%offset%)
               akm1k))
      (setf ak
              (/
               (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                              a-%offset%)
               akm1k))
      (setf denom (- (* akm1 ak) one))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nrhs) nil)
        (tagbody
          (setf bkm1
                  (/
                   (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                  ((1 ldb$) (1 *)) b-%offset%)
```

```
                    akm1k))
            (setf bk
                   (/
                    (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)
                    akm1k))
            (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                   (/ (- (* ak bkm1) bk) denom))
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                   (/ (- (* akm1 bk) bkm1) denom))
          label15))
        (setf i (f2cl-lib:int-sub i 1))))))
  (setf i (f2cl-lib:int-sub i 1))
  (go label100002)
 label100003
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ctrsm "L" "U" "T" "U" n nrhs one a lda b ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10)))
  (setf k 1)
 label100004
  (if (not (<= k n))
      (go label100005))
  (cond
   ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
    (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
    (if (/= kp k)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (cswap nrhs
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$)
          (declare (ignore var-1 var-3))
          (when var-0 (setf nrhs var-0))
          (when var-2 (setf ldb$ var-2))
          (when var-4 (setf ldb$ var-4))))
    (setf k (f2cl-lib:int-add k 1)))
   (t
    (setf kp
```

```
                (f2cl-lib:int-sub
                 (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
      (if (and (< k n)
                (= kp
                   (f2cl-lib:int-sub
                    (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
                                   ((1 *)) ipiv-%offset%))))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (cswap nrhs
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-add k 2))))
    (go label100004)
   label100005))
 (t
  (tagbody
    (setf k 1)
   label100006
    (if (not (<= k n))
        (go label100007))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (cswap nrhs
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-add k 1)))
     (t
      (setf kp
              (f2cl-lib:int-sub
               (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
                              ((1 *)) ipiv-%offset%)))
```

```
    (if (= kp
           (f2cl-lib:int-sub
            (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (cswap nrhs
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                   ((+ k 1) 1) ((1 ldb$) (1 *))
                                   b-%offset%)
             ldb$
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$)
          (declare (ignore var-1 var-3))
          (when var-0 (setf nrhs var-0))
          (when var-2 (setf ldb$ var-2))
          (when var-4 (setf ldb$ var-4))))
    (setf k (f2cl-lib:int-add k 2))))
  (go label100006)
 label100007
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ctrsm "L" "L" "N" "U" n nrhs one a lda b ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10)))
  (setf i 1)
 label100008
  (if (not (<= i n))
      (go label100009))
  (cond
   ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (cscal nrhs
         (/ one
            (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                           a-%offset%))
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (i 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-1 var-2))
      (when var-0 (setf nrhs var-0))
      (when var-3 (setf ldb$ var-3))))
   (t
    (setf akm1k
            (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
    (setf akm1
```

```
            (/
             (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                            a-%offset%)
             akm1k))
    (setf ak
            (/
             (f2cl-lib:fref a-%data%
                            ((f2cl-lib:int-add i 1)
                             (f2cl-lib:int-add i 1))
                            ((1 lda) (1 *)) a-%offset%)
             akm1k))
    (setf denom (- (* akm1 ak) one))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (setf bkm1
                (/
                 (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                 akm1k))
        (setf bk
                (/
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                ((1 ldb$) (1 *)) b-%offset%)
                 akm1k))
        (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)
                (/ (- (* ak bkm1) bk) denom))
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)
                (/ (- (* akm1 bk) bkm1) denom))
       label25))
    (setf i (f2cl-lib:int-add i 1))))
  (setf i (f2cl-lib:int-add i 1))
  (go label100008)
 label100009
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ctrsm "L" "L" "T" "U" n nrhs one a lda b ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10)))
  (setf k n)
 label100010
  (if (not (>= k 1))
      (go label100011))
```

```
      (cond
       ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
        (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
        (if (/= kp k)
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (cswap nrhs
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$)
              (declare (ignore var-1 var-3))
              (when var-0 (setf nrhs var-0))
              (when var-2 (setf ldb$ var-2))
              (when var-4 (setf ldb$ var-4))))
        (setf k (f2cl-lib:int-sub k 1)))
       (t
        (setf kp
                (f2cl-lib:int-sub
                 (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
        (if (and (> k 1)
                 (= kp
                    (f2cl-lib:int-sub
                     (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                                    ((1 *)) ipiv-%offset%))))
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (cswap nrhs
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$)
              (declare (ignore var-1 var-3))
              (when var-0 (setf nrhs var-0))
              (when var-2 (setf ldb$ var-2))
              (when var-4 (setf ldb$ var-4))))
        (setf k (f2cl-lib:int-sub k 2))))
      (go label100010)
     label100011)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (csyconv uplo "R" n a lda ipiv work iinfo)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
    (setf uplo var-0)
    (setf iinfo var-7))
  (go end_label)
 end_label
  (return (values uplo n nrhs nil lda nil nil ldb$ nil info))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csytrs2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::ldb$ nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::csyconv))))
```

60.24 csytrs_rook

60.24.1 Axiom unit tests

— csytrsrook.input —

```
)set break resume
)sys rm -f csytrsrook.output
)spool csytrsrook.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

60.24.2 Axiom help page

— **csytrs_rook.help** —

```
=====================================================================
csytrs_rook examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csytrs_rook (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO)
           CSYTRS_ROOK

Function/Subroutine Documentation
   subroutine csytrs_rook (character UPLO, integer N, integer NRHS, complex,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       complex, dimension( ldb, * ) B, integer LDB, integer INFO)
       CSYTRS_ROOK

Purpose:
   CSYTRS_ROOK solves a system of linear equations A*X = B with
   a complex symmetric matrix A using the factorization A = U*D*U**T or
   A = L*D*L**T computed by CSYTRF_ROOK.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
```

```
  The number of right hand sides, i.e., the number of columns
  of the matrix B.  NRHS >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  The block diagonal matrix D and the multipliers used to
  obtain the factor U or L as computed by CSYTRF_ROOK.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by CSYTRF_ROOK.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the right hand side matrix B.
  On exit, the solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

60.24.3 fortran code

— csytrs_rook.f —

```
*  =====================================================================
      SUBROUTINE CSYTRS_ROOK( UPLO, N, NRHS, A, LDA, IPIV, B, LDB,
     $                        INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
```

```
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            CONE
      PARAMETER          ( CONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            J, K, KP
      COMPLEX            AK, AKM1, AKM1K, BK, BKM1, DENOM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEMV, CGERU, CSCAL, CSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CSYTRS_ROOK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
```

```
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*        Solve A*X = B, where A = U*D*U**T.
*
*        First solve U*D*X = B, overwriting B with X.
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop.
*
         IF( K.LT.1 )
     $      GO TO 30
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
*           Multiply by inv(U(K)), where U(K) is the transformation
*           stored in column K of A.
*
            CALL CGERU( K-1, NRHS, -CONE, A( 1, K ), 1, B( K, 1 ), LDB,
     $                  B( 1, 1 ), LDB )
*
*           Multiply by the inverse of the diagonal block.
*
            CALL CSCAL( NRHS, CONE / A( K, K ), B( K, 1 ), LDB )
            K = K - 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Interchange rows K and -IPIV(K) THEN K-1 and -IPIV(K-1)
*
            KP = -IPIV( K )
            IF( KP.NE.K )
     $         CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
```

```
*
            KP = -IPIV( K-1 )
            IF( KP.NE.K-1 )
     $         CALL CSWAP( NRHS, B( K-1, 1 ), LDB, B( KP, 1 ), LDB )
*
*           Multiply by inv(U(K)), where U(K) is the transformation
*           stored in columns K-1 and K of A.
*
            IF( K.GT.2 ) THEN
               CALL CGERU( K-2, NRHS,-CONE, A( 1, K ), 1, B( K, 1 ),
     $                     LDB, B( 1, 1 ), LDB )
               CALL CGERU( K-2, NRHS,-CONE, A( 1, K-1 ), 1, B( K-1, 1 ),
     $                     LDB, B( 1, 1 ), LDB )
            END IF
*
*           Multiply by the inverse of the diagonal block.
*
            AKM1K = A( K-1, K )
            AKM1 = A( K-1, K-1 ) / AKM1K
            AK = A( K, K ) / AKM1K
            DENOM = AKM1*AK - CONE
            DO 20 J = 1, NRHS
               BKM1 = B( K-1, J ) / AKM1K
               BK = B( K, J ) / AKM1K
               B( K-1, J ) = ( AK*BKM1-BK ) / DENOM
               B( K, J ) = ( AKM1*BK-BKM1 ) / DENOM
   20       CONTINUE
            K = K - 2
         END IF
*
         GO TO 10
   30    CONTINUE
*
*        Next solve U**T *X = B, overwriting B with X.
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = 1
   40    CONTINUE
*
*        If K > N, exit from loop.
*
         IF( K.GT.N )
     $      GO TO 50
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
```

```
*          Multiply by inv(U**T(K)), where U(K) is the transformation
*          stored in column K of A.
*
           IF( K.GT.1 )
     $        CALL CGEMV( 'Transpose', K-1, NRHS, -CONE, B,
     $                    LDB, A( 1, K ), 1, CONE, B( K, 1 ), LDB )
*
*          Interchange rows K and IPIV(K).
*
           KP = IPIV( K )
           IF( KP.NE.K )
     $        CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
           K = K + 1
        ELSE
*
*          2 x 2 diagonal block
*
*          Multiply by inv(U**T(K+1)), where U(K+1) is the transformation
*          stored in columns K and K+1 of A.
*
           IF( K.GT.1 ) THEN
              CALL CGEMV( 'Transpose', K-1, NRHS, -CONE, B,
     $                    LDB, A( 1, K ), 1, CONE, B( K, 1 ), LDB )
              CALL CGEMV( 'Transpose', K-1, NRHS, -CONE, B,
     $                    LDB, A( 1, K+1 ), 1, CONE, B( K+1, 1 ), LDB )
           END IF
*
*          Interchange rows K and -IPIV(K) THEN K+1 and -IPIV(K+1).
*
           KP = -IPIV( K )
           IF( KP.NE.K )
     $        CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
           KP = -IPIV( K+1 )
           IF( KP.NE.K+1 )
     $        CALL CSWAP( NRHS, B( K+1, 1 ), LDB, B( KP, 1 ), LDB )
*
           K = K + 2
        END IF
*
        GO TO 40
   50   CONTINUE
*
      ELSE
*
*       Solve A*X = B, where A = L*D*L**T.
*
*       First solve L*D*X = B, overwriting B with X.
*
*       K is the main loop index, increasing from 1 to N in steps of
```

```
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = 1
   60    CONTINUE
*
*        If K > N, exit from loop.
*
         IF( K.GT.N )
     $      GO TO 80
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
*           Multiply by inv(L(K)), where L(K) is the transformation
*           stored in column K of A.
*
            IF( K.LT.N )
     $         CALL CGERU( N-K, NRHS, -CONE, A( K+1, K ), 1, B( K, 1 ),
     $                     LDB, B( K+1, 1 ), LDB )
*
*           Multiply by the inverse of the diagonal block.
*
            CALL CSCAL( NRHS, CONE / A( K, K ), B( K, 1 ), LDB )
            K = K + 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Interchange rows K and -IPIV(K) THEN K+1 and -IPIV(K+1)
*
            KP = -IPIV( K )
            IF( KP.NE.K )
     $         CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
            KP = -IPIV( K+1 )
            IF( KP.NE.K+1 )
     $         CALL CSWAP( NRHS, B( K+1, 1 ), LDB, B( KP, 1 ), LDB )
*
*           Multiply by inv(L(K)), where L(K) is the transformation
*           stored in columns K and K+1 of A.
*
            IF( K.LT.N-1 ) THEN
               CALL CGERU( N-K-1, NRHS,-CONE, A( K+2, K ), 1, B( K, 1 ),
```

```
     $                   LDB, B( K+2, 1 ), LDB )
            CALL CGERU( N-K-1, NRHS,-CONE, A( K+2, K+1 ), 1,
     $                   B( K+1, 1 ), LDB, B( K+2, 1 ), LDB )
         END IF
*
*        Multiply by the inverse of the diagonal block.
*
         AKM1K = A( K+1, K )
         AKM1 = A( K, K ) / AKM1K
         AK = A( K+1, K+1 ) / AKM1K
         DENOM = AKM1*AK - CONE
         DO 70 J = 1, NRHS
            BKM1 = B( K, J ) / AKM1K
            BK = B( K+1, J ) / AKM1K
            B( K, J ) = ( AK*BKM1-BK ) / DENOM
            B( K+1, J ) = ( AKM1*BK-BKM1 ) / DENOM
   70    CONTINUE
         K = K + 2
      END IF
*
      GO TO 60
   80 CONTINUE
*
*     Next solve L**T *X = B, overwriting B with X.
*
*     K is the main loop index, decreasing from N to 1 in steps of
*     1 or 2, depending on the size of the diagonal blocks.
*
      K = N
   90 CONTINUE
*
*     If K < 1, exit from loop.
*
      IF( K.LT.1 )
     $   GO TO 100
*
      IF( IPIV( K ).GT.0 ) THEN
*
*        1 x 1 diagonal block
*
*        Multiply by inv(L**T(K)), where L(K) is the transformation
*        stored in column K of A.
*
         IF( K.LT.N )
     $      CALL CGEMV( 'Transpose', N-K, NRHS, -CONE, B( K+1, 1 ),
     $                  LDB, A( K+1, K ), 1, CONE, B( K, 1 ), LDB )
*
*        Interchange rows K and IPIV(K).
*
         KP = IPIV( K )
```

```
            IF( KP.NE.K )
     $         CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K = K - 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Multiply by inv(L**T(K-1)), where L(K-1) is the transformation
*           stored in columns K-1 and K of A.
*
            IF( K.LT.N ) THEN
               CALL CGEMV( 'Transpose', N-K, NRHS, -CONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K ), 1, CONE, B( K, 1 ), LDB )
               CALL CGEMV( 'Transpose', N-K, NRHS, -CONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K-1 ), 1, CONE, B( K-1, 1 ),
     $                     LDB )
            END IF
*
*           Interchange rows K and -IPIV(K) THEN K-1 and -IPIV(K-1)
*
            KP = -IPIV( K )
            IF( KP.NE.K )
     $         CALL CSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
            KP = -IPIV( K-1 )
            IF( KP.NE.K-1 )
     $         CALL CSWAP( NRHS, B( K-1, 1 ), LDB, B( KP, 1 ), LDB )
*
            K = K - 2
         END IF
*
         GO TO 90
  100    CONTINUE
      END IF
*
      RETURN
*
*     End of CSYTRS_ROOK
*
      END
```

———-

60.24.4 lisp code

— csytrs_rook.lisp —

```
(let* ((cone (f2cl-lib:cmplx 1.0 0.0)))
```

```
(declare (type (f2cl-lib:complex8) cone)
         (ignorable cone))
(defun csytrs_rook (uplo n nrhs a lda ipiv b ldb$ info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex8 (*)) b a)
           (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (b f2cl-lib:complex8 b-%data% b-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((ak #C(0.0 0.0)) (akm1 #C(0.0 0.0)) (akm1k #C(0.0 0.0))
           (bk #C(0.0 0.0)) (bkm1 #C(0.0 0.0)) (denom #C(0.0 0.0)) (j 0)
           (k 0) (kp 0) (upper nil))
      (declare (type (f2cl-lib:complex8) ak akm1 akm1k bk bkm1 denom)
               (type (f2cl-lib:integer4) j k kp)
               (type f2cl-lib:logical upper))
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (cond
       ((and (not upper)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -5))
       ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -8)))
      (cond
       ((/= info 0) (xerbla "CSYTRS_ROOK" (f2cl-lib:int-sub info))
        (go end_label)))
      (if (or (= n 0) (= nrhs 0))
          (go end_label))
      (cond
       (upper
        (tagbody
          (setf k n)
         label10
          (if (< k 1)
```

```
    (go label30))
(cond
 ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
  (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
  (if (/= kp k)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (cswap nrhs
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-3))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (cgeru (f2cl-lib:int-sub k 1) nrhs (- cone)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
    (when var-1 (setf nrhs var-1))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldb$ var-8)))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (cscal nrhs
       (/ cone
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-1 var-2))
    (when var-0 (setf nrhs var-0))
    (when var-3 (setf ldb$ var-3)))
  (setf k (f2cl-lib:int-sub k 1)))
 (t
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
  (if (/= kp k)
```

```
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (cswap nrhs
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-1 var-3))
      (when var-0 (setf nrhs var-0))
      (when var-2 (setf ldb$ var-2))
      (when var-4 (setf ldb$ var-4))))
(setf kp
        (f2cl-lib:int-sub
         (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                        ((1 *)) ipiv-%offset%)))
(if (/= kp (f2cl-lib:int-sub k 1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (cswap nrhs
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                               ((+ k (f2cl-lib:int-sub 1)) 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-1 var-3))
      (when var-0 (setf nrhs var-0))
      (when var-2 (setf ldb$ var-2))
      (when var-4 (setf ldb$ var-4))))
(cond
 ((> k 2)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (cgeru (f2cl-lib:int-sub k 2) nrhs (- cone)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
    (when var-1 (setf nrhs var-1))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldb$ var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
```

```
      (cgeru (f2cl-lib:int-sub k 2) nrhs (- cone)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             (1 (f2cl-lib:int-sub k 1))
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                             ((+ k (f2cl-lib:int-sub 1)) 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
    (when var-1 (setf nrhs var-1))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldb$ var-8)))))
(setf akm1k
        (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                       ((1 lda) (1 *)) a-%offset%))
(setf akm1
        (/
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-sub k 1)
                         (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         akm1k))
(setf ak
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         akm1k))
(setf denom (- (* akm1 ak) cone))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf bkm1
            (/
             (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1) j)
                            ((1 ldb$) (1 *)) b-%offset%)
             akm1k))
    (setf bk
            (/
             (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                            b-%offset%)
             akm1k))
    (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1) j)
                         ((1 ldb$) (1 *)) b-%offset%)
            (/ (- (* ak bkm1) bk) denom))
    (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                         b-%offset%)
```

```
                (/ (- (* akm1 bk) bkm1) denom))
      label20))
    (setf k (f2cl-lib:int-sub k 2))))
  (go label10)
 label30
  (setf k 1)
 label40
  (if (> k n)
      (go label50))
  (cond
   ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
    (if (> k 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10)
            (cgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- cone) b
             ldb$
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                                   ((1 lda) (1 *)) a-%offset%)
             1 cone
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$)
          (declare
           (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
          (when var-2 (setf nrhs var-2))
          (when var-5 (setf ldb$ var-5))
          (when var-8 (setf cone var-8))
          (when var-10 (setf ldb$ var-10))))
    (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
    (if (/= kp k)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (cswap nrhs
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$)
          (declare (ignore var-1 var-3))
          (when var-0 (setf nrhs var-0))
          (when var-2 (setf ldb$ var-2))
          (when var-4 (setf ldb$ var-4))))
    (setf k (f2cl-lib:int-add k 1)))
   (t
    (cond
     ((> k 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
```

```
        (cgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- cone) b
         ldb$
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 k)
                               ((1 lda) (1 *)) a-%offset%)
         1 cone
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
      (when var-2 (setf nrhs var-2))
      (when var-5 (setf ldb$ var-5))
      (when var-8 (setf cone var-8))
      (when var-10 (setf ldb$ var-10)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10)
        (cgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- cone) b
         ldb$
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                               (1 (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%)
         1 cone
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                               ((+ k 1) 1) ((1 ldb$) (1 *))
                               b-%offset%)
         ldb$)
      (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
      (when var-2 (setf nrhs var-2))
      (when var-5 (setf ldb$ var-5))
      (when var-8 (setf cone var-8))
      (when var-10 (setf ldb$ var-10)))))
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
  (if (/= kp k)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (cswap nrhs
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-3))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
```

```
                              ((1 *)) ipiv-%offset%)))
      (if (/= kp (f2cl-lib:int-add k 1))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (cswap nrhs
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                     ((+ k 1) 1) ((1 ldb$) (1 *))
                                     b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-add k 2))))
    (go label40)
   label50))
 (t
  (tagbody
    (setf k 1)
   label60
    (if (> k n)
        (go label80))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (cswap nrhs
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (if (< k n)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (cgeru (f2cl-lib:int-sub n k) nrhs (- cone)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                     ((+ k 1) k) ((1 lda) (1 *))
                                     a-%offset%)
               1
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
```

```
          ldb$
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                ((+ k 1) 1) ((1 ldb$) (1 *))
                                b-%offset%)
          ldb$)
       (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
       (when var-1 (setf nrhs var-1))
       (when var-6 (setf ldb$ var-6))
       (when var-8 (setf ldb$ var-8))))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (cscal nrhs
      (/ cone
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%))
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
   (declare (ignore var-1 var-2))
   (when var-0 (setf nrhs var-0))
   (when var-3 (setf ldb$ var-3)))
 (setf k (f2cl-lib:int-add k 1)))
(t
 (setf kp
         (f2cl-lib:int-sub
          (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
 (if (/= kp k)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (cswap nrhs
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
       (declare (ignore var-1 var-3))
       (when var-0 (setf nrhs var-0))
       (when var-2 (setf ldb$ var-2))
       (when var-4 (setf ldb$ var-4))))
 (setf kp
         (f2cl-lib:int-sub
          (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
                         ((1 *)) ipiv-%offset%)))
 (if (/= kp (f2cl-lib:int-add k 1))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (cswap nrhs
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                ((+ k 1) 1) ((1 ldb$) (1 *))
                                b-%offset%)
          ldb$
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
```

```
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
      (declare (ignore var-1 var-3))
      (when var-0 (setf nrhs var-0))
      (when var-2 (setf ldb$ var-2))
      (when var-4 (setf ldb$ var-4))))
(cond
 ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (cgeru (f2cl-lib:int-sub n k 1) nrhs (- cone)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ k 2) k) ((1 lda) (1 *))
                             a-%offset%)
       1
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                             ((+ k 2) 1) ((1 ldb$) (1 *))
                             b-%offset%)
       ldb$)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
    (when var-1 (setf nrhs var-1))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldb$ var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (cgeru (f2cl-lib:int-sub n k 1) nrhs (- cone)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ k 2) (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                             ((+ k 1) 1) ((1 ldb$) (1 *))
                             b-%offset%)
       ldb$
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                             ((+ k 2) 1) ((1 ldb$) (1 *))
                             b-%offset%)
       ldb$)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
    (when var-1 (setf nrhs var-1))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldb$ var-8)))))
(setf akm1k
        (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                       ((1 lda) (1 *)) a-%offset%))
(setf akm1
        (/
```

```
                (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                               a-%offset%)
                akm1k))
        (setf ak
               (/
                (f2cl-lib:fref a-%data%
                               ((f2cl-lib:int-add k 1)
                                (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%)
                akm1k))
        (setf denom (- (* akm1 ak) cone))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j nrhs) nil)
          (tagbody
            (setf bkm1
                    (/
                     (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                    b-%offset%)
                     akm1k))
            (setf bk
                    (/
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                                    ((1 ldb$) (1 *)) b-%offset%)
                     akm1k))
            (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                 b-%offset%)
                    (/ (- (* ak bkm1) bk) denom))
            (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                    (/ (- (* akm1 bk) bkm1) denom))
           label70))
        (setf k (f2cl-lib:int-add k 2))))
      (go label60)
     label80
      (setf k n)
     label90
      (if (< k 1)
          (go label100))
      (cond
       ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
        (if (< k n)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10)
                (cgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- cone)
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                       ((+ k 1) 1) ((1 ldb$) (1 *))
                                       b-%offset%)
                 ldb$
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
```

```
                                ((+ k 1) k) ((1 lda) (1 *))
                                a-%offset%)
          1 cone
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
       (declare
        (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
       (when var-2 (setf nrhs var-2))
       (when var-5 (setf ldb$ var-5))
       (when var-8 (setf cone var-8))
       (when var-10 (setf ldb$ var-10))))
 (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
 (if (/= kp k)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (cswap nrhs
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
       (declare (ignore var-1 var-3))
       (when var-0 (setf nrhs var-0))
       (when var-2 (setf ldb$ var-2))
       (when var-4 (setf ldb$ var-4))))
 (setf k (f2cl-lib:int-sub k 1)))
(t
 (cond
  ((< k n)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (cgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- cone)
        (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                              ((+ k 1) 1) ((1 ldb$) (1 *))
                              b-%offset%)
        ldb$
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                              ((+ k 1) k) ((1 lda) (1 *))
                              a-%offset%)
        1 cone
        (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$)
     (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
     (when var-2 (setf nrhs var-2))
     (when var-5 (setf ldb$ var-5))
     (when var-8 (setf cone var-8))
     (when var-10 (setf ldb$ var-10)))
```

```
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (cgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- cone)
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                             ((+ k 1) 1) ((1 ldb$) (1 *))
                             b-%offset%)
       ldb$
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ k 1) (f2cl-lib:int-sub k 1))
                             ((1 lda) (1 *)) a-%offset%)
       1 cone
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                             ((+ k (f2cl-lib:int-sub 1)) 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
    (when var-2 (setf nrhs var-2))
    (when var-5 (setf ldb$ var-5))
    (when var-8 (setf cone var-8))
    (when var-10 (setf ldb$ var-10)))))
(setf kp
        (f2cl-lib:int-sub
         (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
(if (/= kp k)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (cswap nrhs
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (k 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-1 var-3))
      (when var-0 (setf nrhs var-0))
      (when var-2 (setf ldb$ var-2))
      (when var-4 (setf ldb$ var-4))))
(setf kp
        (f2cl-lib:int-sub
         (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                        ((1 *)) ipiv-%offset%)))
(if (/= kp (f2cl-lib:int-sub k 1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (cswap nrhs
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                               ((+ k (f2cl-lib:int-sub 1)) 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (kp 1)
                               ((1 ldb$) (1 *)) b-%offset%)
```

```
                      ldb$)
                    (declare (ignore var-1 var-3))
                    (when var-0 (setf nrhs var-0))
                    (when var-2 (setf ldb$ var-2))
                    (when var-4 (setf ldb$ var-4))))
              (setf k (f2cl-lib:int-sub k 2))))
            (go label90)
           label100)))
        (go end_label)
       end_label
        (return (values uplo nil nrhs nil nil nil nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csytrs_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil
                                              fortran-to-lisp::nrhs nil nil nil
                                              nil fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

60.25 ctgsyl

60.25.1 Axiom unit tests

— ctgsyl.input —

```
)set break resume
)sys rm -f ctgsyl.output
)spool ctgsyl.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.25.2 Axiom help page

— **ctgsyl.help** —

```
==========================================================================
ctgsyl examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ctgsyl (TRANS, IJOB, M, N, A, LDA, B, LDB, C, LDC, D, LDD,
           E, LDE, F, LDF, SCALE, DIF, WORK, LWORK, IWORK, INFO)
           CTGSYL

Function/Subroutine Documentation
   subroutine ctgsyl (character TRANS, integer IJOB, integer M, integer N,
       complex, dimension( lda, * ) A, integer LDA, complex, dimension( ldb, *
       ) B, integer LDB, complex, dimension( ldc, * ) C, integer LDC, complex,
       dimension( ldd, * ) D, integer LDD, complex, dimension( lde, * ) E,
       integer LDE, complex, dimension( ldf, * ) F, integer LDF, real SCALE,
       real DIF, complex, dimension( * ) WORK, integer LWORK, integer,
       dimension( * ) IWORK, integer INFO)
       CTGSYL

Purpose:
   CTGSYL solves the generalized Sylvester equation:

                   A * R - L * B = scale * C            (1)
                   D * R - L * E = scale * F

   where R and L are unknown m-by-n matrices, (A, D), (B, E) and
   (C, F) are given matrix pairs of size m-by-m, n-by-n and m-by-n,
```

```
  respectively, with complex entries. A, B, D and E are upper
  triangular (i.e., (A,D) and (B,E) in generalized Schur form).

  The solution (R, L) overwrites (C, F). 0 <= SCALE <= 1
  is an output scaling factor chosen to avoid overflow.

  In matrix notation (1) is equivalent to solve Zx = scale*b, where Z
  is defined as

             Z = [ kron(In, A)  -kron(B**H, Im) ]        (2)
                 [ kron(In, D)  -kron(E**H, Im) ],

  Here Ix is the identity matrix of size x and X**H is the conjugate
  transpose of X. Kron(X, Y) is the Kronecker product between the
  matrices X and Y.

  If TRANS = 'C', y in the conjugate transposed system Z**H *y = scale*b
  is solved for, which is equivalent to solve for R and L in

                  A**H * R + D**H * L = scale * C           (3)
                  R * B**H + L * E**H = scale * -F

  This case (TRANS = 'C') is used to compute an one-norm-based estimate
  of Dif[(A,D), (B,E)], the separation between the matrix pairs (A,D)
  and (B,E), using CLACON.

  If IJOB >= 1, CTGSYL computes a Frobenius norm-based estimate of
  Dif[(A,D),(B,E)]. That is, the reciprocal of a lower bound on the
  reciprocal of the smallest singular value of Z.

  This is a level-3 BLAS algorithm.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   = 'N': solve the generalized sylvester equation (1).
   = 'C': solve the "conjugate transposed" system (3).

 IJOB
   IJOB is INTEGER
   Specifies what kind of functionality to be performed.
   =0: solve (1) only.
   =1: The functionality of 0 and 3.
   =2: The functionality of 0 and 4.
   =3: Only an estimate of Dif[(A,D), (B,E)] is computed.
       (look ahead strategy is used).
   =4: Only an estimate of Dif[(A,D), (B,E)] is computed.
       (CGECON on sub-systems is used).
   Not referenced if TRANS = 'C'.
```

```
M
  M is INTEGER
  The order of the matrices A and D, and the row dimension of
  the matrices C, F, R and L.

N
  N is INTEGER
  The order of the matrices B and E, and the column dimension
  of the matrices C, F, R and L.

A
  A is COMPLEX array, dimension (LDA, M)
  The upper triangular matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A. LDA >= max(1, M).

B
  B is COMPLEX array, dimension (LDB, N)
  The upper triangular matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= max(1, N).

C
  C is COMPLEX array, dimension (LDC, N)
  On entry, C contains the right-hand-side of the first matrix
  equation in (1) or (3).
  On exit, if IJOB = 0, 1 or 2, C has been overwritten by
  the solution R. If IJOB = 3 or 4 and TRANS = 'N', C holds R,
  the solution achieved during the computation of the Dif-estimate.

LDC
  LDC is INTEGER
  The leading dimension of the array C. LDC >= max(1, M).

D
  D is COMPLEX array, dimension (LDD, M)
  The upper triangular matrix D.

LDD
  LDD is INTEGER
  The leading dimension of the array D. LDD >= max(1, M).

E
  E is COMPLEX array, dimension (LDE, N)
  The upper triangular matrix E.
```

```
LDE
  LDE is INTEGER
  The leading dimension of the array E. LDE >= max(1, N).

F
  F is COMPLEX array, dimension (LDF, N)
  On entry, F contains the right-hand-side of the second matrix
  equation in (1) or (3).
  On exit, if IJOB = 0, 1 or 2, F has been overwritten by
  the solution L. If IJOB = 3 or 4 and TRANS = 'N', F holds L,
  the solution achieved during the computation of the Dif-estimate.

LDF
  LDF is INTEGER
  The leading dimension of the array F. LDF >= max(1, M).

DIF
  DIF is REAL
  On exit DIF is the reciprocal of a lower bound of the
  reciprocal of the Dif-function, i.e. DIF is an upper bound of
  Dif[(A,D), (B,E)] = sigma-min(Z), where Z as in (2).
  IF IJOB = 0 or TRANS = 'C', DIF is not referenced.

SCALE
  SCALE is REAL
  On exit SCALE is the scaling factor in (1) or (3).
  If 0 < SCALE < 1, C and F hold the solutions R and L, resp.,
  to a slightly perturbed system but the input matrices A, B,
  D and E have not been changed. If SCALE = 0, R and L will
  hold the solutions to the homogenious system with C = F = 0.

WORK
  WORK is COMPLEX array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The dimension of the array WORK. LWORK > = 1.
  If IJOB = 1 or 2 and TRANS = 'N', LWORK >= max(1,2*M*N).

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

IWORK
  IWORK is INTEGER array, dimension (M+N+2)

INFO
```

```
INFO is INTEGER
=0: successful exit
<0: If INFO = -i, the i-th argument had an illegal value.
>0: (A, D) and (B, E) have common or very close
    eigenvalues.
```

60.25.3 fortran code

— ctgsyl.f —

```
*  =====================================================================
      SUBROUTINE CTGSYL( TRANS, IJOB, M, N, A, LDA, B, LDB, C, LDC, D,
     $                   LDD, E, LDE, F, LDF, SCALE, DIF, WORK, LWORK,
     $                   IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            IJOB, INFO, LDA, LDB, LDC, LDD, LDE, LDF,
     $                   LWORK, M, N
      REAL               DIF, SCALE
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      COMPLEX            A( LDA, * ), B( LDB, * ), C( LDC, * ),
     $                   D( LDD, * ), E( LDE, * ), F( LDF, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*  Replaced various illegal calls to CCOPY by calls to CLASET.
*  Sven Hammarling, 1/5/02.
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      COMPLEX            CZERO
      PARAMETER          ( CZERO = (0.0E+0, 0.0E+0) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, NOTRAN
```

```
      INTEGER            I, IE, IFUNC, IROUND, IS, ISOLVE, J, JE, JS, K,
     $                   LINFO, LWMIN, MB, NB, P, PQ, Q
      REAL               DSCALE, DSUM, SCALE2, SCALOC
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEMM, CLACPY, CLASET, CSCAL, CTGSY2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          CMPLX, MAX, REAL, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode and test input parameters
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
*
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( NOTRAN ) THEN
         IF( ( IJOB.LT.0 ) .OR. ( IJOB.GT.4 ) ) THEN
            INFO = -2
         END IF
      END IF
      IF( INFO.EQ.0 ) THEN
         IF( M.LE.0 ) THEN
            INFO = -3
         ELSE IF( N.LE.0 ) THEN
            INFO = -4
         ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
            INFO = -6
         ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
            INFO = -8
         ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
            INFO = -10
         ELSE IF( LDD.LT.MAX( 1, M ) ) THEN
            INFO = -12
         ELSE IF( LDE.LT.MAX( 1, N ) ) THEN
            INFO = -14
         ELSE IF( LDF.LT.MAX( 1, M ) ) THEN
            INFO = -16
         END IF
      END IF
*
```

```
      IF( INFO.EQ.0 ) THEN
         IF( NOTRAN ) THEN
            IF( IJOB.EQ.1 .OR. IJOB.EQ.2 ) THEN
               LWMIN = MAX( 1, 2*M*N )
            ELSE
               LWMIN = 1
            END IF
         ELSE
            LWMIN = 1
         END IF
         WORK( 1 ) = LWMIN
*
         IF( LWORK.LT.LWMIN .AND. .NOT.LQUERY ) THEN
            INFO = -20
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CTGSYL', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         SCALE = 1
         IF( NOTRAN ) THEN
            IF( IJOB.NE.0 ) THEN
               DIF = 0
            END IF
         END IF
         RETURN
      END IF
*
*     Determine  optimal block sizes MB and NB
*
      MB = ILAENV( 2, 'CTGSYL', TRANS, M, N, -1, -1 )
      NB = ILAENV( 5, 'CTGSYL', TRANS, M, N, -1, -1 )
*
      ISOLVE = 1
      IFUNC = 0
      IF( NOTRAN ) THEN
         IF( IJOB.GE.3 ) THEN
            IFUNC = IJOB - 2
            CALL CLASET( 'F', M, N, CZERO, CZERO, C, LDC )
            CALL CLASET( 'F', M, N, CZERO, CZERO, F, LDF )
         ELSE IF( IJOB.GE.1 .AND. NOTRAN ) THEN
            ISOLVE = 2
```

```
         END IF
      END IF
*
      IF( ( MB.LE.1 .AND. NB.LE.1 ) .OR. ( MB.GE.M .AND. NB.GE.N ) )
     $     THEN
*
*        Use unblocked Level 2 solver
*
         DO 30 IROUND = 1, ISOLVE
*
            SCALE = ONE
            DSCALE = ZERO
            DSUM = ONE
            PQ = M*N
            CALL CTGSY2( TRANS, IFUNC, M, N, A, LDA, B, LDB, C, LDC, D,
     $                   LDD, E, LDE, F, LDF, SCALE, DSUM, DSCALE,
     $                   INFO )
            IF( DSCALE.NE.ZERO ) THEN
               IF( IJOB.EQ.1 .OR. IJOB.EQ.3 ) THEN
                  DIF = SQRT( REAL( 2*M*N ) ) / ( DSCALE*SQRT( DSUM ) )
               ELSE
                  DIF = SQRT( REAL( PQ ) ) / ( DSCALE*SQRT( DSUM ) )
               END IF
            END IF
            IF( ISOLVE.EQ.2 .AND. IROUND.EQ.1 ) THEN
               IF( NOTRAN ) THEN
                  IFUNC = IJOB
               END IF
               SCALE2 = SCALE
               CALL CLACPY( 'F', M, N, C, LDC, WORK, M )
               CALL CLACPY( 'F', M, N, F, LDF, WORK( M*N+1 ), M )
               CALL CLASET( 'F', M, N, CZERO, CZERO, C, LDC )
               CALL CLASET( 'F', M, N, CZERO, CZERO, F, LDF )
            ELSE IF( ISOLVE.EQ.2 .AND. IROUND.EQ.2 ) THEN
               CALL CLACPY( 'F', M, N, WORK, M, C, LDC )
               CALL CLACPY( 'F', M, N, WORK( M*N+1 ), M, F, LDF )
               SCALE = SCALE2
            END IF
   30    CONTINUE
*
         RETURN
*
      END IF
*
*     Determine block structure of A
*
      P = 0
      I = 1
   40 CONTINUE
      IF( I.GT.M )
```

```
     $   GO TO 50
      P = P + 1
      IWORK( P ) = I
      I = I + MB
      IF( I.GE.M )
     $   GO TO 50
      GO TO 40
   50 CONTINUE
      IWORK( P+1 ) = M + 1
      IF( IWORK( P ).EQ.IWORK( P+1 ) )
     $   P = P - 1
*
*     Determine block structure of B
*
      Q = P + 1
      J = 1
   60 CONTINUE
      IF( J.GT.N )
     $   GO TO 70
*
      Q = Q + 1
      IWORK( Q ) = J
      J = J + NB
      IF( J.GE.N )
     $   GO TO 70
      GO TO 60
*
   70 CONTINUE
      IWORK( Q+1 ) = N + 1
      IF( IWORK( Q ).EQ.IWORK( Q+1 ) )
     $   Q = Q - 1
*
      IF( NOTRAN ) THEN
         DO 150 IROUND = 1, ISOLVE
*
*           Solve (I, J) - subsystem
*               A(I, I) * R(I, J) - L(I, J) * B(J, J) = C(I, J)
*               D(I, I) * R(I, J) - L(I, J) * E(J, J) = F(I, J)
*           for I = P, P - 1, ..., 1; J = 1, 2, ..., Q
*
            PQ = 0
            SCALE = ONE
            DSCALE = ZERO
            DSUM = ONE
            DO 130 J = P + 2, Q
               JS = IWORK( J )
               JE = IWORK( J+1 ) - 1
               NB = JE - JS + 1
               DO 120 I = P, 1, -1
                  IS = IWORK( I )
```

```
                 IE = IWORK( I+1 ) - 1
                 MB = IE - IS + 1
                 CALL CTGSY2( TRANS, IFUNC, MB, NB, A( IS, IS ), LDA,
     $                        B( JS, JS ), LDB, C( IS, JS ), LDC,
     $                        D( IS, IS ), LDD, E( JS, JS ), LDE,
     $                        F( IS, JS ), LDF, SCALOC, DSUM, DSCALE,
     $                        LINFO )
                 IF( LINFO.GT.0 )
     $              INFO = LINFO
                 PQ = PQ + MB*NB
                 IF( SCALOC.NE.ONE ) THEN
                    DO 80 K = 1, JS - 1
                       CALL CSCAL( M, CMPLX( SCALOC, ZERO ), C( 1, K ),
     $                             1 )
                       CALL CSCAL( M, CMPLX( SCALOC, ZERO ), F( 1, K ),
     $                             1 )
   80               CONTINUE
                    DO 90 K = JS, JE
                       CALL CSCAL( IS-1, CMPLX( SCALOC, ZERO ),
     $                             C( 1, K ), 1 )
                       CALL CSCAL( IS-1, CMPLX( SCALOC, ZERO ),
     $                             F( 1, K ), 1 )
   90               CONTINUE
                    DO 100 K = JS, JE
                       CALL CSCAL( M-IE, CMPLX( SCALOC, ZERO ),
     $                             C( IE+1, K ), 1 )
                       CALL CSCAL( M-IE, CMPLX( SCALOC, ZERO ),
     $                             F( IE+1, K ), 1 )
  100               CONTINUE
                    DO 110 K = JE + 1, N
                       CALL CSCAL( M, CMPLX( SCALOC, ZERO ), C( 1, K ),
     $                             1 )
                       CALL CSCAL( M, CMPLX( SCALOC, ZERO ), F( 1, K ),
     $                             1 )
  110               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
*
*                Substitute R(I,J) and L(I,J) into remaining equation.
*
                 IF( I.GT.1 ) THEN
                    CALL CGEMM( 'N', 'N', IS-1, NB, MB,
     $                          CMPLX( -ONE, ZERO ), A( 1, IS ), LDA,
     $                          C( IS, JS ), LDC, CMPLX( ONE, ZERO ),
     $                          C( 1, JS ), LDC )
                    CALL CGEMM( 'N', 'N', IS-1, NB, MB,
     $                          CMPLX( -ONE, ZERO ), D( 1, IS ), LDD,
     $                          C( IS, JS ), LDC, CMPLX( ONE, ZERO ),
     $                          F( 1, JS ), LDF )
                 END IF
```

```
                IF( J.LT.Q ) THEN
                   CALL CGEMM( 'N', 'N', MB, N-JE, NB,
     $                         CMPLX( ONE, ZERO ), F( IS, JS ), LDF,
     $                         B( JS, JE+1 ), LDB, CMPLX( ONE, ZERO ),
     $                         C( IS, JE+1 ), LDC )
                   CALL CGEMM( 'N', 'N', MB, N-JE, NB,
     $                         CMPLX( ONE, ZERO ), F( IS, JS ), LDF,
     $                         E( JS, JE+1 ), LDE, CMPLX( ONE, ZERO ),
     $                         F( IS, JE+1 ), LDF )
                END IF
 120         CONTINUE
 130      CONTINUE
          IF( DSCALE.NE.ZERO ) THEN
             IF( IJOB.EQ.1 .OR. IJOB.EQ.3 ) THEN
                DIF = SQRT( REAL( 2*M*N ) ) / ( DSCALE*SQRT( DSUM ) )
             ELSE
                DIF = SQRT( REAL( PQ ) ) / ( DSCALE*SQRT( DSUM ) )
             END IF
          END IF
          IF( ISOLVE.EQ.2 .AND. IROUND.EQ.1 ) THEN
             IF( NOTRAN ) THEN
                IFUNC = IJOB
             END IF
             SCALE2 = SCALE
             CALL CLACPY( 'F', M, N, C, LDC, WORK, M )
             CALL CLACPY( 'F', M, N, F, LDF, WORK( M*N+1 ), M )
             CALL CLASET( 'F', M, N, CZERO, CZERO, C, LDC )
             CALL CLASET( 'F', M, N, CZERO, CZERO, F, LDF )
          ELSE IF( ISOLVE.EQ.2 .AND. IROUND.EQ.2 ) THEN
             CALL CLACPY( 'F', M, N, WORK, M, C, LDC )
             CALL CLACPY( 'F', M, N, WORK( M*N+1 ), M, F, LDF )
             SCALE = SCALE2
          END IF
 150   CONTINUE
       ELSE
*
*        Solve transposed (I, J)-subsystem
*            A(I, I)**H * R(I, J) + D(I, I)**H * L(I, J) = C(I, J)
*            R(I, J) * B(J, J)  + L(I, J) * E(J, J) = -F(I, J)
*        for I = 1,2,..., P; J = Q, Q-1,..., 1
*
         SCALE = ONE
         DO 210 I = 1, P
            IS = IWORK( I )
            IE = IWORK( I+1 ) - 1
            MB = IE - IS + 1
            DO 200 J = Q, P + 2, -1
               JS = IWORK( J )
               JE = IWORK( J+1 ) - 1
               NB = JE - JS + 1
```

```
               CALL CTGSY2( TRANS, IFUNC, MB, NB, A( IS, IS ), LDA,
     $                      B( JS, JS ), LDB, C( IS, JS ), LDC,
     $                      D( IS, IS ), LDD, E( JS, JS ), LDE,
     $                      F( IS, JS ), LDF, SCALOC, DSUM, DSCALE,
     $                      LINFO )
               IF( LINFO.GT.0 )
     $            INFO = LINFO
               IF( SCALOC.NE.ONE ) THEN
                  DO 160 K = 1, JS - 1
                     CALL CSCAL( M, CMPLX( SCALOC, ZERO ), C( 1, K ),
     $                           1 )
                     CALL CSCAL( M, CMPLX( SCALOC, ZERO ), F( 1, K ),
     $                           1 )
  160             CONTINUE
                  DO 170 K = JS, JE
                     CALL CSCAL( IS-1, CMPLX( SCALOC, ZERO ), C( 1, K ),
     $                           1 )
                     CALL CSCAL( IS-1, CMPLX( SCALOC, ZERO ), F( 1, K ),
     $                           1 )
  170             CONTINUE
                  DO 180 K = JS, JE
                     CALL CSCAL( M-IE, CMPLX( SCALOC, ZERO ),
     $                           C( IE+1, K ), 1 )
                     CALL CSCAL( M-IE, CMPLX( SCALOC, ZERO ),
     $                           F( IE+1, K ), 1 )
  180             CONTINUE
                  DO 190 K = JE + 1, N
                     CALL CSCAL( M, CMPLX( SCALOC, ZERO ), C( 1, K ),
     $                           1 )
                     CALL CSCAL( M, CMPLX( SCALOC, ZERO ), F( 1, K ),
     $                           1 )
  190             CONTINUE
                  SCALE = SCALE*SCALOC
               END IF
*
*              Substitute R(I,J) and L(I,J) into remaining equation.
*
               IF( J.GT.P+2 ) THEN
                  CALL CGEMM( 'N', 'C', MB, JS-1, NB,
     $                        CMPLX( ONE, ZERO ), C( IS, JS ), LDC,
     $                        B( 1, JS ), LDB, CMPLX( ONE, ZERO ),
     $                        F( IS, 1 ), LDF )
                  CALL CGEMM( 'N', 'C', MB, JS-1, NB,
     $                        CMPLX( ONE, ZERO ), F( IS, JS ), LDF,
     $                        E( 1, JS ), LDE, CMPLX( ONE, ZERO ),
     $                        F( IS, 1 ), LDF )
               END IF
               IF( I.LT.P ) THEN
                  CALL CGEMM( 'C', 'N', M-IE, NB, MB,
     $                        CMPLX( -ONE, ZERO ), A( IS, IE+1 ), LDA,
```

```
     $                       C( IS, JS ), LDC, CMPLX( ONE, ZERO ),
     $                       C( IE+1, JS ), LDC )
                  CALL CGEMM( 'C', 'N', M-IE, NB, MB,
     $                       CMPLX( -ONE, ZERO ), D( IS, IE+1 ), LDD,
     $                       F( IS, JS ), LDF, CMPLX( ONE, ZERO ),
     $                       C( IE+1, JS ), LDC )
               END IF
  200       CONTINUE
  210    CONTINUE
      END IF
*
      WORK( 1 ) = LWMIN
*
      RETURN
*
*     End of CTGSYL
*
      END
```

———-

60.25.4 lisp code

— ctgsyl.lisp —

```
(let* ((zero 0.0) (one 1.0) (czero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (f2cl-lib:complex8) czero)
           (ignorable zero one czero))
  (defun ctgsyl
         (trans ijob m n a lda b ldb$ c ldc d ldd e lde f ldf scale dif work
          lwork iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (single-float) dif scale)
             (type (array f2cl-lib:complex8 (*)) work f e d c b a)
             (type (f2cl-lib:integer4) info lwork ldf lde ldd ldc ldb$ lda n m
              ijob)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (c f2cl-lib:complex8 c-%data% c-%offset%)
         (d f2cl-lib:complex8 d-%data% d-%offset%)
         (e f2cl-lib:complex8 e-%data% e-%offset%)
         (f f2cl-lib:complex8 f-%data% f-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
```

```
    (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((dscale 0.0) (dsum 0.0) (scale2 0.0) (scaloc 0.0) (i 0) (ie 0)
         (ifunc 0) (iround 0) (is 0) (isolve 0) (j 0) (je 0) (js 0) (k 0)
         (linfo 0) (lwmin 0) (mb 0) (nb 0) (p 0) (pq 0) (q 0) (lquery nil)
         (notran nil))
    (declare (type (single-float) dscale dsum scale2 scaloc)
             (type (f2cl-lib:integer4) i ie ifunc iround is isolve j je js
              k linfo lwmin mb nb p pq q)
             (type f2cl-lib:logical lquery notran))
    (setf info 0)
    (setf notran
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "N")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val))
    (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
    (cond
     ((and (not notran)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "C")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val)))
      (setf info -1))
     (notran (cond ((or (< ijob 0) (> ijob 4)) (setf info -2)))))
    (cond
     ((= info 0)
      (cond ((<= m 0) (setf info -3)) ((<= n 0) (setf info -4))
            ((< lda
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -6))
            ((< ldb$
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -8))
            ((< ldc
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -10))
            ((< ldd
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -12))
            ((< lde
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -14))
            ((< ldf
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -16)))))
    (cond
     ((= info 0)
```

```
  (cond
   (notran
    (cond
     ((or (= ijob 1) (= ijob 2))
      (setf lwmin
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 (f2cl-lib:int-mul 2 m n)))))
     (t (setf lwmin 1))))
   (t (setf lwmin 1)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwmin 'f2cl-lib:complex8))
  (cond ((and (< lwork lwmin) (not lquery)) (setf info -20)))))
(cond
 ((/= info 0) (xerbla "CTGSYL" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond
 ((or (= m 0) (= n 0))
  (setf scale (coerce (the f2cl-lib:integer4 1) 'single-float))
  (cond
   (notran
    (cond
     ((/= ijob 0)
      (setf dif (coerce (the f2cl-lib:integer4 0) 'single-float))))))
  (go end_label)))
(setf mb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 2 "CTGSYL" trans m n -1 -1)
          (declare (ignore var-0 var-1 var-5 var-6))
          (when var-2 (setf trans var-2))
          (when var-3 (setf m var-3))
          (when var-4 (setf n var-4))
          ret-val))
(setf nb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 5 "CTGSYL" trans m n -1 -1)
          (declare (ignore var-0 var-1 var-5 var-6))
          (when var-2 (setf trans var-2))
          (when var-3 (setf m var-3))
          (when var-4 (setf n var-4))
          ret-val))
(setf isolve 1)
(setf ifunc 0)
(cond
 (notran
  (cond
   ((>= ijob 3) (setf ifunc (f2cl-lib:int-sub ijob 2))
    (claset "F" m n czero czero c ldc)
    (claset "F" m n czero czero f ldf))
```

```
    ((and (>= ijob 1) notran) (setf isolve 2)))))
  (cond
   ((or (and (<= mb 1) (<= nb 1)) (and (>= mb m) (>= nb n)))
    (f2cl-lib:fdo (iround 1 (f2cl-lib:int-add iround 1))
                  ((> iround isolve) nil)
      (tagbody
        (setf scale one)
        (setf dscale zero)
        (setf dsum one)
        (setf pq (f2cl-lib:int-mul m n))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17
             var-18 var-19)
            (ctgsy2 trans ifunc m n a lda b ldb$ c ldc d ldd e lde f ldf
             scale dsum dscale info)
          (declare
           (ignore var-1 var-3 var-4 var-5 var-6 var-8 var-10 var-11
            var-12 var-14))
          (setf trans var-0)
          (setf m var-2)
          (setf ldb$ var-7)
          (setf ldc var-9)
          (setf lde var-13)
          (setf ldf var-15)
          (setf scale var-16)
          (setf dsum var-17)
          (setf dscale var-18)
          (setf info var-19))
        (cond
         ((/= dscale zero)
          (cond
           ((or (= ijob 1) (= ijob 3))
            (setf dif
                    (/
                     (f2cl-lib:fsqrt
                      (f2cl-lib:freal (f2cl-lib:int-mul 2 m n)))
                     (* dscale (f2cl-lib:fsqrt dsum)))))
           (t
            (setf dif
                    (/ (f2cl-lib:fsqrt (f2cl-lib:freal pq))
                       (* dscale (f2cl-lib:fsqrt dsum))))))))
        (cond
         ((and (= isolve 2) (= iround 1))
          (cond (notran (setf ifunc ijob))) (setf scale2 scale)
          (clacpy "F" m n c ldc work m)
          (clacpy "F" m n f ldf
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                 ((+ (f2cl-lib:int-mul m n) 1)) ((1 *))
                                 work-%offset%)
```

```
          m)
         (claset "F" m n czero czero c ldc)
         (claset "F" m n czero czero f ldf))
        ((and (= isolve 2) (= iround 2)) (clacpy "F" m n work m c ldc)
         (clacpy "F" m n
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                ((+ (f2cl-lib:int-mul m n) 1)) ((1 *))
                                work-%offset%)
          m f ldf)
         (setf scale scale2)))
      label30))
   (go end_label)))
 (setf p 0)
 (setf i 1)
label40
 (if (> i m)
     (go label50))
 (setf p (f2cl-lib:int-add p 1))
 (setf (f2cl-lib:fref iwork-%data% (p) ((1 *)) iwork-%offset%) i)
 (setf i (f2cl-lib:int-add i mb))
 (if (>= i m)
     (go label50))
 (go label40)
label50
 (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add p 1)) ((1 *))
                      iwork-%offset%)
         (f2cl-lib:int-add m 1))
 (if (= (f2cl-lib:fref iwork-%data% (p) ((1 *)) iwork-%offset%)
        (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add p 1)) ((1 *))
                       iwork-%offset%))
     (setf p (f2cl-lib:int-sub p 1)))
 (setf q (f2cl-lib:int-add p 1))
 (setf j 1)
label60
 (if (> j n)
     (go label70))
 (setf q (f2cl-lib:int-add q 1))
 (setf (f2cl-lib:fref iwork-%data% (q) ((1 *)) iwork-%offset%) j)
 (setf j (f2cl-lib:int-add j nb))
 (if (>= j n)
     (go label70))
 (go label60)
label70
 (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add q 1)) ((1 *))
                      iwork-%offset%)
         (f2cl-lib:int-add n 1))
 (if (= (f2cl-lib:fref iwork-%data% (q) ((1 *)) iwork-%offset%)
        (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add q 1)) ((1 *))
                       iwork-%offset%))
     (setf q (f2cl-lib:int-sub q 1)))
```

```
(cond
 (notran
  (f2cl-lib:fdo (iround 1 (f2cl-lib:int-add iround 1))
                ((> iround isolve) nil)
    (tagbody
      (setf pq 0)
      (setf scale one)
      (setf dscale zero)
      (setf dsum one)
      (f2cl-lib:fdo (j (f2cl-lib:int-add p 2) (f2cl-lib:int-add j 1))
                    ((> j q) nil)
        (tagbody
          (setf js
                  (f2cl-lib:fref iwork-%data% (j) ((1 *))
                                 iwork-%offset%))
          (setf je
                  (f2cl-lib:int-sub
                   (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add j 1))
                                  ((1 *)) iwork-%offset%)
                   1))
          (setf nb (f2cl-lib:int-add (f2cl-lib:int-sub je js) 1))
          (f2cl-lib:fdo (i p (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                        ((> i 1) nil)
            (tagbody
              (setf is
                      (f2cl-lib:fref iwork-%data% (i) ((1 *))
                                     iwork-%offset%))
              (setf ie
                      (f2cl-lib:int-sub
                       (f2cl-lib:fref iwork-%data%
                                      ((f2cl-lib:int-add i 1)) ((1 *))
                                      iwork-%offset%)
                       1))
              (setf mb (f2cl-lib:int-add (f2cl-lib:int-sub ie is) 1))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
                   var-8 var-9 var-10 var-11 var-12 var-13 var-14
                   var-15 var-16 var-17 var-18 var-19)
                  (ctgsy2 trans ifunc mb nb
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                         (is is) ((1 lda) (1 *))
                                         a-%offset%)
                   lda
                   (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                         (js js) ((1 ldb$) (1 *))
                                         b-%offset%)
                   ldb$
                   (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                         (is js) ((1 ldc) (1 *))
                                         c-%offset%)
```

```
      ldc
      (f2cl-lib:array-slice d-%data% f2cl-lib:complex8
                            (is is) ((1 ldd) (1 *))
                            d-%offset%)
      ldd
      (f2cl-lib:array-slice e-%data% f2cl-lib:complex8
                            (js js) ((1 lde) (1 *))
                            e-%offset%)
      lde
      (f2cl-lib:array-slice f-%data% f2cl-lib:complex8
                            (is js) ((1 ldf) (1 *))
                            f-%offset%)
      ldf scaloc dsum dscale linfo)
  (declare
   (ignore var-1 var-3 var-4 var-5 var-6 var-8 var-10
    var-11 var-12 var-14))
  (setf trans var-0)
  (setf mb var-2)
  (setf ldb$ var-7)
  (setf ldc var-9)
  (setf lde var-13)
  (setf ldf var-15)
  (setf scaloc var-16)
  (setf dsum var-17)
  (setf dscale var-18)
  (setf linfo var-19))
(if (> linfo 0)
    (setf info linfo))
(setf pq (f2cl-lib:int-add pq (f2cl-lib:int-mul mb nb)))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k
                    (f2cl-lib:int-add js
                                      (f2cl-lib:int-sub
                                       1)))
                 nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (cscal m (f2cl-lib:cmplx scaloc zero)
           (f2cl-lib:array-slice c-%data%
                                 f2cl-lib:complex8 (1 k)
                                 ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf m var-0)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (cscal m (f2cl-lib:cmplx scaloc zero)
           (f2cl-lib:array-slice f-%data%
```

```
                                f2cl-lib:complex8 (1 k)
                                ((1 ldf) (1 *))
                                f-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf m var-0)))
   label80))
(f2cl-lib:fdo (k js (f2cl-lib:int-add k 1))
              ((> k je) nil)
  (tagbody
    (cscal (f2cl-lib:int-sub is 1)
     (f2cl-lib:cmplx scaloc zero)
     (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                           (1 k) ((1 ldc) (1 *))
                           c-%offset%)
     1)
    (cscal (f2cl-lib:int-sub is 1)
     (f2cl-lib:cmplx scaloc zero)
     (f2cl-lib:array-slice f-%data% f2cl-lib:complex8
                           (1 k) ((1 ldf) (1 *))
                           f-%offset%)
     1)
   label90))
(f2cl-lib:fdo (k js (f2cl-lib:int-add k 1))
              ((> k je) nil)
  (tagbody
    (cscal (f2cl-lib:int-sub m ie)
     (f2cl-lib:cmplx scaloc zero)
     (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                           ((+ ie 1) k) ((1 ldc) (1 *))
                           c-%offset%)
     1)
    (cscal (f2cl-lib:int-sub m ie)
     (f2cl-lib:cmplx scaloc zero)
     (f2cl-lib:array-slice f-%data% f2cl-lib:complex8
                           ((+ ie 1) k) ((1 ldf) (1 *))
                           f-%offset%)
     1)
   label100))
(f2cl-lib:fdo (k (f2cl-lib:int-add je 1)
               (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (cscal m (f2cl-lib:cmplx scaloc zero)
         (f2cl-lib:array-slice c-%data%
                                f2cl-lib:complex8 (1 k)
                                ((1 ldc) (1 *))
                                c-%offset%)
         1)
```

```
          (declare (ignore var-1 var-2 var-3))
          (when var-0 (setf m var-0)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (cscal m (f2cl-lib:cmplx scaloc zero)
             (f2cl-lib:array-slice f-%data%
                                   f2cl-lib:complex8 (1 k)
                                   ((1 ldf) (1 *))
                                   f-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3))
          (when var-0 (setf m var-0)))
      label110))
  (setf scale (* scale scaloc))))
(cond
 ((> i 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8 var-9 var-10 var-11 var-12)
      (cgemm "N" "N" (f2cl-lib:int-sub is 1) nb mb
       (f2cl-lib:cmplx (- one) zero)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             (1 is) ((1 lda) (1 *))
                             a-%offset%)
       lda
       (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                             (is js) ((1 ldc) (1 *))
                             c-%offset%)
       ldc (f2cl-lib:cmplx one zero)
       (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                             (1 js) ((1 ldc) (1 *))
                             c-%offset%)
       ldc)
    (declare
     (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-10
      var-11))
    (when var-3 (setf nb var-3))
    (when var-4 (setf mb var-4))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldc var-9))
    (when var-12 (setf ldc var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8 var-9 var-10 var-11 var-12)
      (cgemm "N" "N" (f2cl-lib:int-sub is 1) nb mb
       (f2cl-lib:cmplx (- one) zero)
       (f2cl-lib:array-slice d-%data% f2cl-lib:complex8
                             (1 is) ((1 ldd) (1 *))
                             d-%offset%)
       ldd
       (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
```

```
                                (is js) ((1 ldc) (1 *))
                                c-%offset%)
          ldc (f2cl-lib:cmplx one zero)
          (f2cl-lib:array-slice f-%data% f2cl-lib:complex8
                                (1 js) ((1 ldf) (1 *))
                                f-%offset%)
          ldf)
        (declare
         (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-10
          var-11))
        (when var-3 (setf nb var-3))
        (when var-4 (setf mb var-4))
        (when var-7 (setf ldd var-7))
        (when var-9 (setf ldc var-9))
        (when var-12 (setf ldf var-12)))))
    (cond
     ((< j q)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
           var-8 var-9 var-10 var-11 var-12)
          (cgemm "N" "N" mb (f2cl-lib:int-sub n je) nb
           (f2cl-lib:cmplx one zero)
           (f2cl-lib:array-slice f-%data% f2cl-lib:complex8
                                 (is js) ((1 ldf) (1 *))
                                 f-%offset%)
           ldf
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                 (js (f2cl-lib:int-add je 1))
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$ (f2cl-lib:cmplx one zero)
           (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                 (is (f2cl-lib:int-add je 1))
                                 ((1 ldc) (1 *)) c-%offset%)
           ldc)
        (declare
         (ignore var-0 var-1 var-3 var-5 var-6 var-8 var-10
          var-11))
        (when var-2 (setf mb var-2))
        (when var-4 (setf nb var-4))
        (when var-7 (setf ldf var-7))
        (when var-9 (setf ldb$ var-9))
        (when var-12 (setf ldc var-12)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
           var-8 var-9 var-10 var-11 var-12)
          (cgemm "N" "N" mb (f2cl-lib:int-sub n je) nb
           (f2cl-lib:cmplx one zero)
           (f2cl-lib:array-slice f-%data% f2cl-lib:complex8
                                 (is js) ((1 ldf) (1 *))
                                 f-%offset%)
```

```
                ldf
                (f2cl-lib:array-slice e-%data% f2cl-lib:complex8
                                      (js (f2cl-lib:int-add je 1))
                                      ((1 lde) (1 *)) e-%offset%)
                lde (f2cl-lib:cmplx one zero)
                (f2cl-lib:array-slice f-%data% f2cl-lib:complex8
                                      (is (f2cl-lib:int-add je 1))
                                      ((1 ldf) (1 *)) f-%offset%)
                ldf)
              (declare
               (ignore var-0 var-1 var-3 var-5 var-6 var-8 var-10
                var-11))
              (when var-2 (setf mb var-2))
              (when var-4 (setf nb var-4))
              (when var-7 (setf ldf var-7))
              (when var-9 (setf lde var-9))
              (when var-12 (setf ldf var-12)))))
        label120))
   label130))
(cond
 ((/= dscale zero)
  (cond
   ((or (= ijob 1) (= ijob 3))
    (setf dif
            (/
             (f2cl-lib:fsqrt
              (f2cl-lib:freal (f2cl-lib:int-mul 2 m n)))
             (* dscale (f2cl-lib:fsqrt dsum)))))
   (t
    (setf dif
            (/ (f2cl-lib:fsqrt (f2cl-lib:freal pq))
               (* dscale (f2cl-lib:fsqrt dsum))))))))
(cond
 ((and (= isolve 2) (= iround 1))
  (cond (notran (setf ifunc ijob))) (setf scale2 scale)
  (clacpy "F" m n c ldc work m)
  (clacpy "F" m n f ldf
   (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                         ((+ (f2cl-lib:int-mul m n) 1)) ((1 *))
                         work-%offset%)
   m)
  (claset "F" m n czero czero c ldc)
  (claset "F" m n czero czero f ldf))
 ((and (= isolve 2) (= iround 2)) (clacpy "F" m n work m c ldc)
  (clacpy "F" m n
   (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                         ((+ (f2cl-lib:int-mul m n) 1)) ((1 *))
                         work-%offset%)
   m f ldf)
  (setf scale scale2)))
```

```
    label150)))
 (t (setf scale one)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i p) nil)
    (tagbody
      (setf is (f2cl-lib:fref iwork-%data% (i) ((1 *)) iwork-%offset%))
      (setf ie
              (f2cl-lib:int-sub
               (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add i 1))
                              ((1 *)) iwork-%offset%)
               1))
      (setf mb (f2cl-lib:int-add (f2cl-lib:int-sub ie is) 1))
      (f2cl-lib:fdo (j q (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                    ((> j (f2cl-lib:int-add p 2)) nil)
        (tagbody
          (setf js
                  (f2cl-lib:fref iwork-%data% (j) ((1 *))
                                 iwork-%offset%))
          (setf je
                  (f2cl-lib:int-sub
                   (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add j 1))
                                  ((1 *)) iwork-%offset%)
                   1))
          (setf nb (f2cl-lib:int-add (f2cl-lib:int-sub je js) 1))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
               var-17 var-18 var-19)
              (ctgsy2 trans ifunc mb nb
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (is is)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (js js)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice c-%data% f2cl-lib:complex8 (is js)
                                     ((1 ldc) (1 *)) c-%offset%)
               ldc
               (f2cl-lib:array-slice d-%data% f2cl-lib:complex8 (is is)
                                     ((1 ldd) (1 *)) d-%offset%)
               ldd
               (f2cl-lib:array-slice e-%data% f2cl-lib:complex8 (js js)
                                     ((1 lde) (1 *)) e-%offset%)
               lde
               (f2cl-lib:array-slice f-%data% f2cl-lib:complex8 (is js)
                                     ((1 ldf) (1 *)) f-%offset%)
               ldf scaloc dsum dscale linfo)
            (declare
             (ignore var-1 var-3 var-4 var-5 var-6 var-8 var-10 var-11
              var-12 var-14))
```

```
  (setf trans var-0)
  (setf mb var-2)
  (setf ldb$ var-7)
  (setf ldc var-9)
  (setf lde var-13)
  (setf ldf var-15)
  (setf scaloc var-16)
  (setf dsum var-17)
  (setf dscale var-18)
  (setf linfo var-19))
(if (> linfo 0)
    (setf info linfo))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k
                    (f2cl-lib:int-add js
                                      (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (cscal m (f2cl-lib:cmplx scaloc zero)
           (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                 (1 k) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf m var-0)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (cscal m (f2cl-lib:cmplx scaloc zero)
           (f2cl-lib:array-slice f-%data% f2cl-lib:complex8
                                 (1 k) ((1 ldf) (1 *))
                                 f-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf m var-0)))
     label160))
  (f2cl-lib:fdo (k js (f2cl-lib:int-add k 1))
                ((> k je) nil)
    (tagbody
      (cscal (f2cl-lib:int-sub is 1)
       (f2cl-lib:cmplx scaloc zero)
       (f2cl-lib:array-slice c-%data% f2cl-lib:complex8 (1 k)
                             ((1 ldc) (1 *)) c-%offset%)
       1)
      (cscal (f2cl-lib:int-sub is 1)
       (f2cl-lib:cmplx scaloc zero)
       (f2cl-lib:array-slice f-%data% f2cl-lib:complex8 (1 k)
                             ((1 ldf) (1 *)) f-%offset%)
       1)
```

```
      label170))
    (f2cl-lib:fdo (k js (f2cl-lib:int-add k 1))
                  ((> k je) nil)
      (tagbody
        (cscal (f2cl-lib:int-sub m ie)
         (f2cl-lib:cmplx scaloc zero)
         (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                               ((+ ie 1) k) ((1 ldc) (1 *))
                               c-%offset%)
         1)
        (cscal (f2cl-lib:int-sub m ie)
         (f2cl-lib:cmplx scaloc zero)
         (f2cl-lib:array-slice f-%data% f2cl-lib:complex8
                               ((+ ie 1) k) ((1 ldf) (1 *))
                               f-%offset%)
         1)
      label180))
    (f2cl-lib:fdo (k (f2cl-lib:int-add je 1)
                   (f2cl-lib:int-add k 1))
                  ((> k n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (cscal m (f2cl-lib:cmplx scaloc zero)
             (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                   (1 k) ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3))
          (when var-0 (setf m var-0)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (cscal m (f2cl-lib:cmplx scaloc zero)
             (f2cl-lib:array-slice f-%data% f2cl-lib:complex8
                                   (1 k) ((1 ldf) (1 *))
                                   f-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3))
          (when var-0 (setf m var-0)))
      label190))
    (setf scale (* scale scaloc))))
  (cond
   ((> j (f2cl-lib:int-add p 2))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (cgemm "N" "C" mb (f2cl-lib:int-sub js 1) nb
         (f2cl-lib:cmplx one zero)
         (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                               (is js) ((1 ldc) (1 *))
                               c-%offset%)
         ldc
```

```
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                  (1 js) ((1 ldb$) (1 *))
                                  b-%offset%)
            ldb$ (f2cl-lib:cmplx one zero)
            (f2cl-lib:array-slice f-%data% f2cl-lib:complex8
                                  (is 1) ((1 ldf) (1 *))
                                  f-%offset%)
            ldf)
          (declare
           (ignore var-0 var-1 var-3 var-5 var-6 var-8 var-10
            var-11))
          (when var-2 (setf mb var-2))
          (when var-4 (setf nb var-4))
          (when var-7 (setf ldc var-7))
          (when var-9 (setf ldb$ var-9))
          (when var-12 (setf ldf var-12)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12)
            (cgemm "N" "C" mb (f2cl-lib:int-sub js 1) nb
             (f2cl-lib:cmplx one zero)
             (f2cl-lib:array-slice f-%data% f2cl-lib:complex8
                                   (is js) ((1 ldf) (1 *))
                                   f-%offset%)
             ldf
             (f2cl-lib:array-slice e-%data% f2cl-lib:complex8
                                   (1 js) ((1 lde) (1 *))
                                   e-%offset%)
             lde (f2cl-lib:cmplx one zero)
             (f2cl-lib:array-slice f-%data% f2cl-lib:complex8
                                   (is 1) ((1 ldf) (1 *))
                                   f-%offset%)
             ldf)
          (declare
           (ignore var-0 var-1 var-3 var-5 var-6 var-8 var-10
            var-11))
          (when var-2 (setf mb var-2))
          (when var-4 (setf nb var-4))
          (when var-7 (setf ldf var-7))
          (when var-9 (setf lde var-9))
          (when var-12 (setf ldf var-12)))))
      (cond
       ((< i p)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12)
            (cgemm "C" "N" (f2cl-lib:int-sub m ie) nb mb
             (f2cl-lib:cmplx (- one) zero)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                   (is (f2cl-lib:int-add ie 1))
```

```
                                        ((1 lda) (1 *)) a-%offset%)
                      lda
                      (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                        (is js) ((1 ldc) (1 *))
                                        c-%offset%)
                      ldc (f2cl-lib:cmplx one zero)
                      (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                        ((+ ie 1) js) ((1 ldc) (1 *))
                                        c-%offset%)
                      ldc)
                    (declare
                     (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-10
                      var-11))
                    (when var-3 (setf nb var-3))
                    (when var-4 (setf mb var-4))
                    (when var-7 (setf lda var-7))
                    (when var-9 (setf ldc var-9))
                    (when var-12 (setf ldc var-12)))
                  (multiple-value-bind
                      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                       var-9 var-10 var-11 var-12)
                      (cgemm "C" "N" (f2cl-lib:int-sub m ie) nb mb
                       (f2cl-lib:cmplx (- one) zero)
                       (f2cl-lib:array-slice d-%data% f2cl-lib:complex8
                                        (is (f2cl-lib:int-add ie 1))
                                        ((1 ldd) (1 *)) d-%offset%)
                       ldd
                       (f2cl-lib:array-slice f-%data% f2cl-lib:complex8
                                        (is js) ((1 ldf) (1 *))
                                        f-%offset%)
                       ldf (f2cl-lib:cmplx one zero)
                       (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                        ((+ ie 1) js) ((1 ldc) (1 *))
                                        c-%offset%)
                       ldc)
                    (declare
                     (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-10
                      var-11))
                    (when var-3 (setf nb var-3))
                    (when var-4 (setf mb var-4))
                    (when var-7 (setf ldd var-7))
                    (when var-9 (setf ldf var-9))
                    (when var-12 (setf ldc var-12)))))
             label200))
         label210))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce lwmin 'f2cl-lib:complex8))
  (go end_label)
 end_label
  (return
```

```
        (values trans nil m n nil lda nil ldb$ nil ldc nil ldd nil lde nil ldf
                scale dif nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ctgsyl
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (single-float) (single-float)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::integer4
                                              (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(fortran-to-lisp::trans nil
                                             fortran-to-lisp::m
                                             fortran-to-lisp::n nil
                                             fortran-to-lisp::lda nil
                                             fortran-to-lisp::ldb$ nil
                                             fortran-to-lisp::ldc nil
                                             fortran-to-lisp::ldd nil
                                             fortran-to-lisp::lde nil
                                             fortran-to-lisp::ldf
                                             fortran-to-lisp::scale
                                             fortran-to-lisp::dif nil nil nil
```

```
                                                  fortran-to-lisp::info)
                                                :calls
                                                '(fortran-to-lisp::clacpy
                                                  fortran-to-lisp::ctgsy2
                                                  fortran-to-lisp::claset))))
```

———-

60.26 ctrsyl

60.26.1 Axiom unit tests

— **ctrsyl.input** —

```
)set break resume
)sys rm -f ctrsyl.output
)spool ctrsyl.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

60.26.2 Axiom help page

— **ctrsyl.help** —

```
======================================================================
ctrsyl examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ctrsyl (TRANA, TRANB, ISGN, M, N, A, LDA, B, LDB, C, LDC,
           SCALE, INFO)
```

```
          CTRSYL

Function/Subroutine Documentation
   subroutine ctrsyl (character TRANA, character TRANB, integer ISGN, integer
       M, integer N, complex, dimension( lda, * ) A, integer LDA, complex,
       dimension( ldb, * ) B, integer LDB, complex, dimension( ldc, * ) C,
       integer LDC, real SCALE, integer INFO)
       CTRSYL

Purpose:
   CTRSYL solves the complex Sylvester matrix equation:

   op(A)*X + X*op(B) = scale*C or
   op(A)*X - X*op(B) = scale*C,

   where op(A) = A or A**H, and A and B are both upper triangular. A is
   M-by-M and B is N-by-N; the right hand side C and the solution X are
   M-by-N; and scale is an output scale factor, set <= 1 to avoid
   overflow in X.

Parameters:

 TRANA
   TRANA is CHARACTER*1
   Specifies the option op(A):
   = 'N': op(A) = A    (No transpose)
   = 'C': op(A) = A**H (Conjugate transpose)

 TRANB
   TRANB is CHARACTER*1
   Specifies the option op(B):
   = 'N': op(B) = B    (No transpose)
   = 'C': op(B) = B**H (Conjugate transpose)

 ISGN
   ISGN is INTEGER
   Specifies the sign in the equation:
   = +1: solve op(A)*X + X*op(B) = scale*C
   = -1: solve op(A)*X - X*op(B) = scale*C

 M
   M is INTEGER
   The order of the matrix A, and the number of rows in the
   matrices X and C. M >= 0.

 N
   N is INTEGER
   The order of the matrix B, and the number of columns in the
   matrices X and C. N >= 0.
```

```
A
  A is COMPLEX array, dimension (LDA,M)
  The upper triangular matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A. LDA >= max(1,M).

B
  B is COMPLEX array, dimension (LDB,N)
  The upper triangular matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= max(1,N).

C
  C is COMPLEX array, dimension (LDC,N)
  On entry, the M-by-N right hand side matrix C.
  On exit, C is overwritten by the solution matrix X.

LDC
  LDC is INTEGER
  The leading dimension of the array C. LDC >= max(1,M)

SCALE
  SCALE is REAL
  The scale factor, scale, set <= 1 to avoid overflow in X.

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  = 1: A and B have common or very close eigenvalues; perturbed
       values were used to solve the equation (but the matrices
       A and B are unchanged).
```

60.26.3 fortran code

— ctrsyl.f —

```
*  =====================================================================
      SUBROUTINE CTRSYL( TRANA, TRANB, ISGN, M, N, A, LDA, B, LDB, C,
     $                   LDC, SCALE, INFO )
*
```

```
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANA, TRANB
      INTEGER            INFO, ISGN, LDA, LDB, LDC, M, N
      REAL               SCALE
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), B( LDB, * ), C( LDC, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRNA, NOTRNB
      INTEGER            J, K, L
      REAL               BIGNUM, DA11, DB, EPS, SCALOC, SGN, SMIN,
     $                   SMLNUM
      COMPLEX            A11, SUML, SUMR, VEC, X11
*     ..
*     .. Local Arrays ..
      REAL               DUM( 1 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               CLANGE, SLAMCH
      COMPLEX            CDOTC, CDOTU, CLADIV
      EXTERNAL           LSAME, CLANGE, SLAMCH, CDOTC, CDOTU, CLADIV
*     ..
*     .. External Subroutines ..
      EXTERNAL           CSSCAL, SLABAD, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, CMPLX, CONJG, MAX, MIN, REAL
*     ..
*     .. Executable Statements ..
*
*     Decode and Test input parameters
*
      NOTRNA = LSAME( TRANA, 'N' )
      NOTRNB = LSAME( TRANB, 'N' )
*
      INFO = 0
```

```
      IF( .NOT.NOTRNA .AND. .NOT.LSAME( TRANA, 'C' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRNB .AND. .NOT.LSAME( TRANB, 'C' ) ) THEN
         INFO = -2
      ELSE IF( ISGN.NE.1 .AND. ISGN.NE.-1 ) THEN
         INFO = -3
      ELSE IF( M.LT.0 ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
         INFO = -11
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CTRSYL', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      SCALE = ONE
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Set constants to control overflow
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL SLABAD( SMLNUM, BIGNUM )
      SMLNUM = SMLNUM*REAL( M*N ) / EPS
      BIGNUM = ONE / SMLNUM
      SMIN = MAX( SMLNUM, EPS*CLANGE( 'M', M, M, A, LDA, DUM ),
     $       EPS*CLANGE( 'M', N, N, B, LDB, DUM ) )
      SGN = ISGN
*
      IF( NOTRNA .AND. NOTRNB ) THEN
*
*        Solve    A*X + ISGN*X*B = scale*C.
*
*        The (K,L)th block of X is determined starting from
*        bottom-left corner column by column by
*
*            A(K,K)*X(K,L) + ISGN*X(K,L)*B(L,L) = C(K,L) - R(K,L)
*
*        Where
```

```
*                   M                        L-1
*         R(K,L) = SUM [A(K,I)*X(I,L)] +ISGN*SUM [X(K,J)*B(J,L)].
*                  I=K+1                      J=1
*
        DO 30 L = 1, N
           DO 20 K = M, 1, -1
*
              SUML = CDOTU( M-K, A( K, MIN( K+1, M ) ), LDA,
     $                C( MIN( K+1, M ), L ), 1 )
              SUMR = CDOTU( L-1, C( K, 1 ), LDC, B( 1, L ), 1 )
              VEC = C( K, L ) - ( SUML+SGN*SUMR )
*
              SCALOC = ONE
              A11 = A( K, K ) + SGN*B( L, L )
              DA11 = ABS( REAL( A11 ) ) + ABS( AIMAG( A11 ) )
              IF( DA11.LE.SMIN ) THEN
                 A11 = SMIN
                 DA11 = SMIN
                 INFO = 1
              END IF
              DB = ABS( REAL( VEC ) ) + ABS( AIMAG( VEC ) )
              IF( DA11.LT.ONE .AND. DB.GT.ONE ) THEN
                 IF( DB.GT.BIGNUM*DA11 )
     $              SCALOC = ONE / DB
              END IF
              X11 = CLADIV( VEC*CMPLX( SCALOC ), A11 )
*
              IF( SCALOC.NE.ONE ) THEN
                 DO 10 J = 1, N
                    CALL CSSCAL( M, SCALOC, C( 1, J ), 1 )
   10            CONTINUE
                 SCALE = SCALE*SCALOC
              END IF
              C( K, L ) = X11
*
   20       CONTINUE
   30    CONTINUE
*
      ELSE IF( .NOT.NOTRNA .AND. NOTRNB ) THEN
*
*        Solve    A**H *X + ISGN*X*B = scale*C.
*
*        The (K,L)th block of X is determined starting from
*        upper-left corner column by column by
*
*            A**H(K,K)*X(K,L) + ISGN*X(K,L)*B(L,L) = C(K,L) - R(K,L)
*
*        Where
*                   K-1                           L-1
*         R(K,L) = SUM [A**H(I,K)*X(I,L)] + ISGN*SUM [X(K,J)*B(J,L)]
```

```
*                  I=1                           J=1
*
         DO 60 L = 1, N
            DO 50 K = 1, M
*
               SUML = CDOTC( K-1, A( 1, K ), 1, C( 1, L ), 1 )
               SUMR = CDOTU( L-1, C( K, 1 ), LDC, B( 1, L ), 1 )
               VEC = C( K, L ) - ( SUML+SGN*SUMR )
*
               SCALOC = ONE
               A11 = CONJG( A( K, K ) ) + SGN*B( L, L )
               DA11 = ABS( REAL( A11 ) ) + ABS( AIMAG( A11 ) )
               IF( DA11.LE.SMIN ) THEN
                  A11 = SMIN
                  DA11 = SMIN
                  INFO = 1
               END IF
               DB = ABS( REAL( VEC ) ) + ABS( AIMAG( VEC ) )
               IF( DA11.LT.ONE .AND. DB.GT.ONE ) THEN
                  IF( DB.GT.BIGNUM*DA11 )
     $               SCALOC = ONE / DB
               END IF
*
               X11 = CLADIV( VEC*CMPLX( SCALOC ), A11 )
*
               IF( SCALOC.NE.ONE ) THEN
                  DO 40 J = 1, N
                     CALL CSSCAL( M, SCALOC, C( 1, J ), 1 )
   40             CONTINUE
                  SCALE = SCALE*SCALOC
               END IF
               C( K, L ) = X11
*
   50       CONTINUE
   60    CONTINUE
*
      ELSE IF( .NOT.NOTRNA .AND. .NOT.NOTRNB ) THEN
*
*        Solve    A**H*X + ISGN*X*B**H = C.
*
*        The (K,L)th block of X is determined starting from
*        upper-right corner column by column by
*
*            A**H(K,K)*X(K,L) + ISGN*X(K,L)*B**H(L,L) = C(K,L) - R(K,L)
*
*        Where
*                    K-1
*           R(K,L) = SUM [A**H(I,K)*X(I,L)] +
*                    I=1
*                           N
```

```
*                    ISGN*SUM [X(K,J)*B**H(L,J)].
*                         J=L+1
*
         DO 90 L = N, 1, -1
            DO 80 K = 1, M
*
               SUML = CDOTC( K-1, A( 1, K ), 1, C( 1, L ), 1 )
               SUMR = CDOTC( N-L, C( K, MIN( L+1, N ) ), LDC,
     $                B( L, MIN( L+1, N ) ), LDB )
               VEC = C( K, L ) - ( SUML+SGN*CONJG( SUMR ) )
*
               SCALOC = ONE
               A11 = CONJG( A( K, K )+SGN*B( L, L ) )
               DA11 = ABS( REAL( A11 ) ) + ABS( AIMAG( A11 ) )
               IF( DA11.LE.SMIN ) THEN
                  A11 = SMIN
                  DA11 = SMIN
                  INFO = 1
               END IF
               DB = ABS( REAL( VEC ) ) + ABS( AIMAG( VEC ) )
               IF( DA11.LT.ONE .AND. DB.GT.ONE ) THEN
                  IF( DB.GT.BIGNUM*DA11 )
     $               SCALOC = ONE / DB
               END IF
*
               X11 = CLADIV( VEC*CMPLX( SCALOC ), A11 )
*
               IF( SCALOC.NE.ONE ) THEN
                  DO 70 J = 1, N
                     CALL CSSCAL( M, SCALOC, C( 1, J ), 1 )
   70             CONTINUE
                  SCALE = SCALE*SCALOC
               END IF
               C( K, L ) = X11
*
   80       CONTINUE
   90    CONTINUE
*
      ELSE IF( NOTRNA .AND. .NOT.NOTRNB ) THEN
*
*        Solve    A*X + ISGN*X*B**H = C.
*
*        The (K,L)th block of X is determined starting from
*        bottom-left corner column by column by
*
*           A(K,K)*X(K,L) + ISGN*X(K,L)*B**H(L,L) = C(K,L) - R(K,L)
*
*        Where
*                    M                          N
*          R(K,L) = SUM [A(K,I)*X(I,L)] + ISGN*SUM [X(K,J)*B**H(L,J)]
```

```
*                 I=K+1                       J=L+1
*
         DO 120 L = N, 1, -1
            DO 110 K = M, 1, -1
*
               SUML = CDOTU( M-K, A( K, MIN( K+1, M ) ), LDA,
     $                C( MIN( K+1, M ), L ), 1 )
               SUMR = CDOTC( N-L, C( K, MIN( L+1, N ) ), LDC,
     $                B( L, MIN( L+1, N ) ), LDB )
               VEC = C( K, L ) - ( SUML+SGN*CONJG( SUMR ) )
*
               SCALOC = ONE
               A11 = A( K, K ) + SGN*CONJG( B( L, L ) )
               DA11 = ABS( REAL( A11 ) ) + ABS( AIMAG( A11 ) )
               IF( DA11.LE.SMIN ) THEN
                  A11 = SMIN
                  DA11 = SMIN
                  INFO = 1
               END IF
               DB = ABS( REAL( VEC ) ) + ABS( AIMAG( VEC ) )
               IF( DA11.LT.ONE .AND. DB.GT.ONE ) THEN
                  IF( DB.GT.BIGNUM*DA11 )
     $               SCALOC = ONE / DB
               END IF
*
               X11 = CLADIV( VEC*CMPLX( SCALOC ), A11 )
*
               IF( SCALOC.NE.ONE ) THEN
                  DO 100 J = 1, N
                     CALL CSSCAL( M, SCALOC, C( 1, J ), 1 )
  100             CONTINUE
                  SCALE = SCALE*SCALOC
               END IF
               C( K, L ) = X11
*
  110       CONTINUE
  120    CONTINUE
*
      END IF
*
      RETURN
*
*     End of CTRSYL
*
      END
```

———-

60.26.4 lisp code

— ctrsyl.lisp —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun ctrsyl (trana tranb isgn m n a lda b ldb$ c ldc scale info)
    (declare (type (single-float) scale)
             (type (array f2cl-lib:complex8 (*)) c b a)
             (type (f2cl-lib:integer4) info ldc ldb$ lda n m isgn)
             (type (string *) tranb trana))
    (f2cl-lib:with-multi-array-data
        ((trana character trana-%data% trana-%offset%)
         (tranb character tranb-%data% tranb-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (c f2cl-lib:complex8 c-%data% c-%offset%))
      (prog ((dum (make-array 1 :element-type 'single-float)) (a11 #C(0.0 0.0))
             (suml #C(0.0 0.0)) (sumr #C(0.0 0.0)) (vec #C(0.0 0.0))
             (x11 #C(0.0 0.0)) (bignum 0.0) (da11 0.0) (db 0.0) (eps 0.0)
             (scaloc 0.0) (sgn 0.0) (smin 0.0) (smlnum 0.0) (j 0) (k 0) (l 0)
             (notrna nil) (notrnb nil))
        (declare (type (array single-float (1)) dum)
                 (type (f2cl-lib:complex8) a11 suml sumr vec x11)
                 (type (single-float) bignum da11 db eps scaloc sgn smin
                  smlnum)
                 (type (f2cl-lib:integer4) j k l)
                 (type f2cl-lib:logical notrna notrnb))
        (setf notrna
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame trana "N")
                  (declare (ignore var-1))
                  (when var-0 (setf trana var-0))
                  ret-val))
        (setf notrnb
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame tranb "N")
                  (declare (ignore var-1))
                  (when var-0 (setf tranb var-0))
                  ret-val))
        (setf info 0)
        (cond
         ((and (not notrna)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame trana "C")
                  (declare (ignore var-1))
                  (when var-0 (setf trana var-0))
```

```
          ret-val)))
  (setf info -1))
 ((and (not notrnb)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame tranb "C")
          (declare (ignore var-1))
          (when var-0 (setf tranb var-0))
          ret-val)))
  (setf info -2))
 ((and (/= isgn 1) (/= isgn (f2cl-lib:int-sub 1))) (setf info -3))
 ((< m 0) (setf info -4)) ((< n 0) (setf info -5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9))
 ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -11)))
(cond
 ((/= info 0) (xerbla "CTRSYL" (f2cl-lib:int-sub info))
  (go end_label)))
(setf scale one)
(if (or (= m 0) (= n 0))
    (go end_label))
(setf eps (slamch "P"))
(setf smlnum (slamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (slabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum
        (/ (* smlnum (f2cl-lib:freal (f2cl-lib:int-mul m n))) eps))
(setf bignum (/ one smlnum))
(setf smin
        (max smlnum
             (* eps
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                    (clange "M" m m a lda dum)
                  (declare (ignore var-0 var-2 var-3 var-4 var-5))
                  (setf m var-1)
                  ret-val))
             (* eps
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                    (clange "M" n n b ldb$ dum)
                  (declare (ignore var-0 var-2 var-3 var-4 var-5))
                  (setf n var-1)
```

```
                  ret-val))))
(setf sgn (coerce (the f2cl-lib:integer4 isgn) 'single-float))
(cond
 ((and notrna notrnb)
  (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                ((> l n) nil)
    (tagbody
      (f2cl-lib:fdo (k m (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                    ((> k 1) nil)
        (tagbody
          (setf suml
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4)
                      (cdotu (f2cl-lib:int-sub m k)
                       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                             (k
                                              (min
                                               (the f2cl-lib:integer4
                                                    (f2cl-lib:int-add k
                                                                      1))
                                               (the f2cl-lib:integer4
                                                    m)))
                                             ((1 lda) (1 *))
                                             a-%offset%)
                       lda
                       (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                             ((min
                                               (f2cl-lib:int-add k 1)
                                               m)
                                              l)
                                             ((1 ldc) (1 *))
                                             c-%offset%)
                       1)
                    (declare (ignore var-0 var-1 var-3 var-4))
                    (when var-2 (setf lda var-2))
                    ret-val))
          (setf sumr
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4)
                      (cdotu (f2cl-lib:int-sub l 1)
                       (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                             (k 1) ((1 ldc) (1 *))
                                             c-%offset%)
                       ldc
                       (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                             (1 l) ((1 ldb$) (1 *))
                                             b-%offset%)
                       1)
                    (declare (ignore var-0 var-1 var-3 var-4))
                    (when var-2 (setf ldc var-2))
```

```
                 ret-val))
        (setf vec
                (-
                 (f2cl-lib:fref c-%data% (k l) ((1 ldc) (1 *))
                                c-%offset%)
                 (+ suml (* sgn sumr))))
        (setf scaloc one)
        (setf a11
                (+
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%)
                 (* sgn
                    (f2cl-lib:fref b-%data% (l l) ((1 ldb$) (1 *))
                                   b-%offset%))))
        (setf da11
                (+ (abs (f2cl-lib:freal a11))
                   (abs (f2cl-lib:aimag a11))))
        (cond
         ((<= da11 smin) (setf a11 (coerce smin 'f2cl-lib:complex8))
          (setf da11 smin) (setf info 1)))
        (setf db
                (+ (abs (f2cl-lib:freal vec))
                   (abs (f2cl-lib:aimag vec))))
        (cond
         ((and (< da11 one) (> db one))
          (if (> db (* bignum da11))
              (setf scaloc (/ one db)))))
        (setf x11 (cladiv (* vec (f2cl-lib:cmplx scaloc)) a11))
        (cond
         ((/= scaloc one)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (multiple-value-bind (var-0 var-1 var-2 var-3)
                  (csscal m scaloc
                   (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                         (1 j) ((1 ldc) (1 *))
                                         c-%offset%)
                   1)
                (declare (ignore var-2 var-3))
                (when var-0 (setf m var-0))
                (when var-1 (setf scaloc var-1)))
             label10))
          (setf scale (* scale scaloc))))
        (setf (f2cl-lib:fref c-%data% (k l) ((1 ldc) (1 *))
                             c-%offset%)
                x11)
      label20))
    label30)))
((and (not notrna) notrnb)
```

```
(f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
              ((> l n) nil)
  (tagbody
    (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                  ((> k m) nil)
      (tagbody
        (setf suml
                (cdotc (f2cl-lib:int-sub k 1)
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                       (1 k) ((1 lda) (1 *))
                                       a-%offset%)
                 1
                 (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                       (1 l) ((1 ldc) (1 *))
                                       c-%offset%)
                 1))
        (setf sumr
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4)
                    (cdotu (f2cl-lib:int-sub l 1)
                     (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                           (k 1) ((1 ldc) (1 *))
                                           c-%offset%)
                     ldc
                     (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                           (1 l) ((1 ldb$) (1 *))
                                           b-%offset%)
                     1)
                  (declare (ignore var-0 var-1 var-3 var-4))
                  (when var-2 (setf ldc var-2))
                  ret-val))
        (setf vec
                (-
                 (f2cl-lib:fref c-%data% (k l) ((1 ldc) (1 *))
                                c-%offset%)
                 (+ suml (* sgn sumr))))
        (setf scaloc one)
        (setf a11
                (+
                 (f2cl-lib:conjg
                  (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                 a-%offset%))
                 (* sgn
                    (f2cl-lib:fref b-%data% (l l) ((1 ldb$) (1 *))
                                   b-%offset%))))
        (setf da11
                (+ (abs (f2cl-lib:freal a11))
                   (abs (f2cl-lib:aimag a11))))
        (cond
         ((<= da11 smin) (setf a11 (coerce smin 'f2cl-lib:complex8))
```

```
              (setf da11 smin) (setf info 1)))
            (setf db
                    (+ (abs (f2cl-lib:freal vec))
                       (abs (f2cl-lib:aimag vec))))
            (cond
             ((and (< da11 one) (> db one))
              (if (> db (* bignum da11))
                  (setf scaloc (/ one db)))))
            (setf x11 (cladiv (* vec (f2cl-lib:cmplx scaloc)) a11))
            (cond
             ((/= scaloc one)
              (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                            ((> j n) nil)
                (tagbody
                  (multiple-value-bind (var-0 var-1 var-2 var-3)
                      (csscal m scaloc
                       (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                             (1 j) ((1 ldc) (1 *))
                                             c-%offset%)
                       1)
                    (declare (ignore var-2 var-3))
                    (when var-0 (setf m var-0))
                    (when var-1 (setf scaloc var-1)))
                 label40))
              (setf scale (* scale scaloc))))
            (setf (f2cl-lib:fref c-%data% (k l) ((1 ldc) (1 *))
                                 c-%offset%)
                    x11)
          label50))
      label60)))
  ((and (not notrna) (not notrnb))
   (f2cl-lib:fdo (l n (f2cl-lib:int-add l (f2cl-lib:int-sub 1)))
                 ((> l 1) nil)
     (tagbody
       (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                     ((> k m) nil)
         (tagbody
           (setf suml
                   (cdotc (f2cl-lib:int-sub k 1)
                    (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                          (1 k) ((1 lda) (1 *))
                                          a-%offset%)
                    1
                    (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                          (1 l) ((1 ldc) (1 *))
                                          c-%offset%)
                    1))
           (setf sumr
                   (multiple-value-bind
                       (ret-val var-0 var-1 var-2 var-3 var-4)
```

```
                (cdotc (f2cl-lib:int-sub n l)
                 (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                       (k
                                        (min
                                         (the f2cl-lib:integer4
                                              (f2cl-lib:int-add l
                                                                1))
                                         (the f2cl-lib:integer4
                                              n)))
                                       ((1 ldc) (1 *))
                                       c-%offset%)
                 ldc
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                       (l
                                        (min
                                         (the f2cl-lib:integer4
                                              (f2cl-lib:int-add l
                                                                1))
                                         (the f2cl-lib:integer4
                                              n)))
                                       ((1 ldb$) (1 *))
                                       b-%offset%)
                 ldb$)
                (declare (ignore var-0 var-1 var-3))
                (when var-2 (setf ldc var-2))
                (when var-4 (setf ldb$ var-4))
                ret-val))
(setf vec
        (-
         (f2cl-lib:fref c-%data% (k l) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn (f2cl-lib:conjg sumr)))))
(setf scaloc one)
(setf a11
        (f2cl-lib:conjg
         (+
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%)
          (* sgn
             (f2cl-lib:fref b-%data% (l l) ((1 ldb$) (1 *))
                            b-%offset%)))))
(setf da11
        (+ (abs (f2cl-lib:freal a11))
           (abs (f2cl-lib:aimag a11))))
(cond
 ((<= da11 smin) (setf a11 (coerce smin 'f2cl-lib:complex8))
  (setf da11 smin) (setf info 1)))
(setf db
        (+ (abs (f2cl-lib:freal vec))
           (abs (f2cl-lib:aimag vec))))
```

```
            (cond
             ((and (< da11 one) (> db one))
              (if (> db (* bignum da11))
                  (setf scaloc (/ one db)))))
            (setf x11 (cladiv (* vec (f2cl-lib:cmplx scaloc)) a11))
            (cond
             ((/= scaloc one)
              (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                            ((> j n) nil)
                (tagbody
                  (multiple-value-bind (var-0 var-1 var-2 var-3)
                      (csscal m scaloc
                       (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                             (1 j) ((1 ldc) (1 *))
                                             c-%offset%)
                       1)
                    (declare (ignore var-2 var-3))
                    (when var-0 (setf m var-0))
                    (when var-1 (setf scaloc var-1)))
                 label70))
              (setf scale (* scale scaloc))))
            (setf (f2cl-lib:fref c-%data% (k l) ((1 ldc) (1 *))
                                 c-%offset%)
                    x11)
          label80))
      label90)))
((and notrna (not notrnb))
 (f2cl-lib:fdo (l n (f2cl-lib:int-add l (f2cl-lib:int-sub 1)))
               ((> l 1) nil)
   (tagbody
     (f2cl-lib:fdo (k m (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                   ((> k 1) nil)
       (tagbody
         (setf suml
                 (multiple-value-bind
                     (ret-val var-0 var-1 var-2 var-3 var-4)
                     (cdotu (f2cl-lib:int-sub m k)
                      (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                            (k
                                             (min
                                              (the f2cl-lib:integer4
                                                   (f2cl-lib:int-add k
                                                                     1))
                                              (the f2cl-lib:integer4
                                                   m)))
                                            ((1 lda) (1 *))
                                            a-%offset%)
                      lda
                      (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                            ((min
```

```
                                        (f2cl-lib:int-add k 1)
                                        m)
                                       l)
                                      ((1 ldc) (1 *))
                                      c-%offset%)
                  1)
              (declare (ignore var-0 var-1 var-3 var-4))
              (when var-2 (setf lda var-2))
              ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (cdotc (f2cl-lib:int-sub n l)
             (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                   (k
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add l
                                                            1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldc) (1 *))
                                   c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                   (l
                                    (min
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add l
                                                            1))
                                     (the f2cl-lib:integer4
                                          n)))
                                   ((1 ldb$) (1 *))
                                   b-%offset%)
             ldb$)
              (declare (ignore var-0 var-1 var-3))
              (when var-2 (setf ldc var-2))
              (when var-4 (setf ldb$ var-4))
              ret-val))
(setf vec
        (-
         (f2cl-lib:fref c-%data% (k l) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn (f2cl-lib:conjg sumr)))))
(setf scaloc one)
(setf a11
        (+
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         (* sgn
```

```
                              (f2cl-lib:conjg
                               (f2cl-lib:fref b-%data% (l l) ((1 ldb$) (1 *))
                                              b-%offset%)))))
                    (setf da11
                            (+ (abs (f2cl-lib:freal a11))
                               (abs (f2cl-lib:aimag a11))))
                    (cond
                     ((<= da11 smin) (setf a11 (coerce smin 'f2cl-lib:complex8))
                      (setf da11 smin) (setf info 1)))
                    (setf db
                            (+ (abs (f2cl-lib:freal vec))
                               (abs (f2cl-lib:aimag vec))))
                    (cond
                     ((and (< da11 one) (> db one))
                      (if (> db (* bignum da11))
                          (setf scaloc (/ one db)))))
                    (setf x11 (cladiv (* vec (f2cl-lib:cmplx scaloc)) a11))
                    (cond
                     ((/= scaloc one)
                      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                                    ((> j n) nil)
                        (tagbody
                          (multiple-value-bind (var-0 var-1 var-2 var-3)
                              (csscal m scaloc
                               (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                                     (1 j) ((1 ldc) (1 *))
                                                     c-%offset%)
                               1)
                            (declare (ignore var-2 var-3))
                            (when var-0 (setf m var-0))
                            (when var-1 (setf scaloc var-1)))
                         label100))
                      (setf scale (* scale scaloc))))
                    (setf (f2cl-lib:fref c-%data% (k l) ((1 ldc) (1 *))
                                         c-%offset%)
                            x11)
                  label110))
              label120))))
         (go end_label)
        end_label
         (return
          (values trana tranb nil m n nil lda nil ldb$ nil ldc scale info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ctrsyl
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
```

```
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (single-float)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::trana
  fortran-to-lisp::tranb nil
  fortran-to-lisp::m
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldc
  fortran-to-lisp::scale
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::cladiv
  fortran-to-lisp::clange))))
```

———-

Chapter 61

LAPACK - Symmetric Matrix, Comp. Routines, Complex16

61.1 zla_syamv

61.1.1 Axiom unit tests

— zlasyamv.input —

```
)set break resume
)sys rm -f zlasyamv.output
)spool zlasyamv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.1.2 Axiom help page

— zla_syamv.help —

```
==========================================================================
zla_syamv examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zla_syamv (UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)
           ZLA_SYAMV computes a matrix-vector product using a symmetric
           indefinite matrix to calculate error bounds.

Function/Subroutine Documentation
   subroutine zla_syamv (integer UPLO, integer N, double precision ALPHA,
       complex*16, dimension( lda, * ) A, integer LDA, complex*16, dimension(
       * ) X, integer INCX, double precision BETA, double precision,
       dimension( * ) Y, integer INCY)
       ZLA_SYAMV computes a matrix-vector product using a symmetric indefinite
       matrix to calculate error bounds.

Purpose:
   ZLA_SYAMV  performs the matrix-vector operation

   y := alpha*abs(A)*abs(x) + beta*abs(y),

   where alpha and beta are scalars, x and y are vectors and A is an
   n by n symmetric matrix.

   This function is primarily used in calculating error bounds.
   To protect against underflow during evaluation, components in
   the resulting vector are perturbed away from zero by (N+1)
   times the underflow threshold.  To prevent unnecessarily large
   errors for block-structure embedded in general matrices,
   "symbolically" zero components are not perturbed.  A zero
   entry is considered "symbolic" if all multiplications involved
   in computing that entry have at least one zero multiplicand.

Parameters:

 UPLO
   UPLO is INTEGER
   On entry, UPLO specifies whether the upper or lower
   triangular part of the array A is to be referenced as follows:

   UPLO = BLAS_UPPER   Only the upper triangular part of A is to be referenced.
   UPLO = BLAS_LOWER   Only the lower triangular part of A is to be referenced.

   Unchanged on exit.

 N
   N is INTEGER
```

```
  On entry, N specifies the number of columns of the matrix A.
  N must be at least zero.
  Unchanged on exit.

ALPHA
  ALPHA is DOUBLE PRECISION .
  On entry, ALPHA specifies the scalar alpha.
  Unchanged on exit.

A
  A is COMPLEX*16 array, DIMENSION ( LDA, n ).
  Before entry, the leading m by n part of the array A must
  contain the matrix of coefficients.
  Unchanged on exit.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, n ).
  Unchanged on exit.

X
  X is COMPLEX*16 array, DIMENSION at least
  ( 1 + ( n - 1 )*abs( INCX ) )
  Before entry, the incremented array X must contain the vector x.
  Unchanged on exit.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of
  X. INCX must not be zero.
  Unchanged on exit.

BETA
  BETA is DOUBLE PRECISION .
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.
  Unchanged on exit.

Y
  Y is DOUBLE PRECISION array, dimension
  ( 1 + ( n - 1 )*abs( INCY ) )
  Before entry with BETA non-zero, the incremented array Y
  must contain the vector y. On exit, Y is overwritten by the
  updated vector y.

INCY
  INCY is INTEGER
  On entry, INCY specifies the increment for the elements of
  Y. INCY must not be zero.
```

```
Unchanged on exit.
```

———-

61.1.3 fortran code

— **zla_syamv.f** —

```
*  ===================================================================
      SUBROUTINE ZLA_SYAMV( UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y,
     $                      INCY )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   ALPHA, BETA
      INTEGER            INCX, INCY, LDA, N
      INTEGER            UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), X( * )
      DOUBLE PRECISION   Y( * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            SYMB_ZERO
      DOUBLE PRECISION   TEMP, SAFE1
      INTEGER            I, INFO, IY, J, JX, KX, KY
      COMPLEX*16         ZDUM
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, DLAMCH
      DOUBLE PRECISION   DLAMCH
*     ..
*     .. External Functions ..
      EXTERNAL           ILAUPLO
      INTEGER            ILAUPLO
*     ..
```

```
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, ABS, SIGN, REAL, DIMAG
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( DBLE ( ZDUM ) ) + ABS( DIMAG ( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF     ( UPLO.NE.ILAUPLO( 'U' ) .AND.
     $         UPLO.NE.ILAUPLO( 'L' ) )THEN
         INFO = 1
      ELSE IF( N.LT.0 )THEN
         INFO = 2
      ELSE IF( LDA.LT.MAX( 1, N ) )THEN
         INFO = 5
      ELSE IF( INCX.EQ.0 )THEN
         INFO = 7
      ELSE IF( INCY.EQ.0 )THEN
         INFO = 10
      END IF
      IF( INFO.NE.0 )THEN
         CALL XERBLA( 'DSYMV ', INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( ( N.EQ.0 ).OR.( ( ALPHA.EQ.ZERO ).AND.( BETA.EQ.ONE ) ) )
     $   RETURN
*
*     Set up the start points in  X  and  Y.
*
      IF( INCX.GT.0 )THEN
         KX = 1
      ELSE
         KX = 1 - ( N - 1 )*INCX
      END IF
      IF( INCY.GT.0 )THEN
         KY = 1
      ELSE
         KY = 1 - ( N - 1 )*INCY
      END IF
*
*     Set SAFE1 essentially to be the underflow threshold times the
```

```
*     number of additions in each row.
*
      SAFE1 = DLAMCH( 'Safe minimum' )
      SAFE1 = (N+1)*SAFE1
*
*     Form  y := alpha*abs(A)*abs(x) + beta*abs(y).
*
*     The O(N^2) SYMB_ZERO tests could be replaced by O(N) queries to
*     the inexact flag.  Still doesn't help change the iteration order
*     to per-column.
*
      IY = KY
      IF ( INCX.EQ.1 ) THEN
         IF ( UPLO .EQ. ILAUPLO( 'U' ) ) THEN
            DO I = 1, N
               IF ( BETA .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
                  Y( IY ) = 0.0D+0
               ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
               ELSE
                  SYMB_ZERO = .FALSE.
                  Y( IY ) = BETA * ABS( Y( IY ) )
               END IF
               IF ( ALPHA .NE. ZERO ) THEN
                  DO J = 1, I
                     TEMP = CABS1( A( J, I ) )
                     SYMB_ZERO = SYMB_ZERO .AND.
     $                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                     Y( IY ) = Y( IY ) + ALPHA*CABS1( X( J ) )*TEMP
                  END DO
                  DO J = I+1, N
                     TEMP = CABS1( A( I, J ) )
                     SYMB_ZERO = SYMB_ZERO .AND.
     $                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                     Y( IY ) = Y( IY ) + ALPHA*CABS1( X( J ) )*TEMP
                  END DO
               END IF

               IF ( .NOT.SYMB_ZERO )
     $              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

               IY = IY + INCY
            END DO
         ELSE
            DO I = 1, N
               IF ( BETA .EQ. ZERO ) THEN
                  SYMB_ZERO = .TRUE.
```

```
            Y( IY ) = 0.0D+0
         ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
            SYMB_ZERO = .TRUE.
         ELSE
            SYMB_ZERO = .FALSE.
            Y( IY ) = BETA * ABS( Y( IY ) )
         END IF
         IF ( ALPHA .NE. ZERO ) THEN
            DO J = 1, I
               TEMP = CABS1( A( I, J ) )
               SYMB_ZERO = SYMB_ZERO .AND.
$                   ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

               Y( IY ) = Y( IY ) + ALPHA*CABS1( X( J ) )*TEMP
            END DO
            DO J = I+1, N
               TEMP = CABS1( A( J, I ) )
               SYMB_ZERO = SYMB_ZERO .AND.
$                   ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

               Y( IY ) = Y( IY ) + ALPHA*CABS1( X( J ) )*TEMP
            END DO
         END IF

         IF ( .NOT.SYMB_ZERO )
$              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

         IY = IY + INCY
      END DO
   END IF
 ELSE
   IF ( UPLO .EQ. ILAUPLO( 'U' ) ) THEN
      DO I = 1, N
         IF ( BETA .EQ. ZERO ) THEN
            SYMB_ZERO = .TRUE.
            Y( IY ) = 0.0D+0
         ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
            SYMB_ZERO = .TRUE.
         ELSE
            SYMB_ZERO = .FALSE.
            Y( IY ) = BETA * ABS( Y( IY ) )
         END IF
         JX = KX
         IF ( ALPHA .NE. ZERO ) THEN
            DO J = 1, I
               TEMP = CABS1( A( J, I ) )
               SYMB_ZERO = SYMB_ZERO .AND.
$                   ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

               Y( IY ) = Y( IY ) + ALPHA*CABS1( X( JX ) )*TEMP
```

```
                JX = JX + INCX
             END DO
             DO J = I+1, N
                TEMP = CABS1( A( I, J ) )
                SYMB_ZERO = SYMB_ZERO .AND.
$                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                Y( IY ) = Y( IY ) + ALPHA*CABS1( X( JX ) )*TEMP
                JX = JX + INCX
             END DO
          END IF

          IF ( .NOT.SYMB_ZERO )
$              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )

          IY = IY + INCY
       END DO
    ELSE
       DO I = 1, N
          IF ( BETA .EQ. ZERO ) THEN
             SYMB_ZERO = .TRUE.
             Y( IY ) = 0.0D+0
          ELSE IF ( Y( IY ) .EQ. ZERO ) THEN
             SYMB_ZERO = .TRUE.
          ELSE
             SYMB_ZERO = .FALSE.
             Y( IY ) = BETA * ABS( Y( IY ) )
          END IF
          JX = KX
          IF ( ALPHA .NE. ZERO ) THEN
             DO J = 1, I
                TEMP = CABS1( A( I, J ) )
                SYMB_ZERO = SYMB_ZERO .AND.
$                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                Y( IY ) = Y( IY ) + ALPHA*CABS1( X( JX ) )*TEMP
                JX = JX + INCX
             END DO
             DO J = I+1, N
                TEMP = CABS1( A( J, I ) )
                SYMB_ZERO = SYMB_ZERO .AND.
$                    ( X( J ) .EQ. ZERO .OR. TEMP .EQ. ZERO )

                Y( IY ) = Y( IY ) + ALPHA*CABS1( X( JX ) )*TEMP
                JX = JX + INCX
             END DO
          END IF

          IF ( .NOT.SYMB_ZERO )
$              Y( IY ) = Y( IY ) + SIGN( SAFE1, Y( IY ) )
```

```
               IY = IY + INCY
            END DO
         END IF

      END IF
*
      RETURN
*
*     End of ZLA_SYAMV
*
      END
```

———-

61.1.4 lisp code

— **zla_syamv.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun zla_syamv (uplo n alpha a lda x incx beta y incy)
    (declare (type (array double-float (*)) y)
             (type (array f2cl-lib:complex16 (*)) x a)
             (type (double-float) beta alpha)
             (type (f2cl-lib:integer4) incy incx lda n uplo))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (x f2cl-lib:complex16 x-%data% x-%offset%)
         (y double-float y-%data% y-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
          cabs1))
        (prog ((zdum #C(0.0d0 0.0d0)) (i 0) (info 0) (iy 0) (j 0) (jx 0) (kx 0)
               (ky 0) (temp 0.0d0) (safe1 0.0d0) (symb_zero nil))
          (declare (type (f2cl-lib:complex16) zdum)
                   (type (f2cl-lib:integer4) i info iy j jx kx ky)
                   (type (double-float) temp safe1)
                   (type f2cl-lib:logical symb_zero))
          (setf info 0)
          (cond
           ((and (/= uplo (ilauplo "U")) (/= uplo (ilauplo "L")))
            (setf info 1))
           ((< n 0) (setf info 2))
```

```
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info 5))
 ((= incx 0) (setf info 7)) ((= incy 0) (setf info 10)))
(cond ((/= info 0) (xerbla "DSYMV " info) (go end_label)))
(if (or (= n 0) (and (= alpha zero) (= beta one)))
    (go end_label))
(cond ((> incx 0) (setf kx 1))
      (t
       (setf kx
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incx)))))
(cond ((> incy 0) (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incy)))))
(setf safe1 (dlamch "Safe minimum"))
(setf safe1 (* (f2cl-lib:int-add n 1) safe1))
(setf iy ky)
(cond
 ((= incx 1)
  (cond
   ((= uplo (ilauplo "U"))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (cond
         ((= beta zero) (setf symb_zero f2cl-lib:%true%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  0.0d0))
         ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
          (setf symb_zero f2cl-lib:%true%))
         (t (setf symb_zero f2cl-lib:%false%)
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (* beta
                     (abs
                      (f2cl-lib:fref y-%data% (iy) ((1 *))
                                     y-%offset%))))))
        (cond
         ((/= alpha zero)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j i) nil)
            (tagbody
              (setf temp
                      (cabs1
                       (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                      a-%offset%)))
              (setf symb_zero
```

```
                (and symb_zero
                     (or
                      (=
                       (f2cl-lib:fref x-%data% (j) ((1 *))
                                      x-%offset%)
                       zero)
                      (= temp zero))))
        (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                (+
                 (f2cl-lib:fref y-%data% (iy) ((1 *))
                                y-%offset%)
                 (* alpha
                    (cabs1
                     (f2cl-lib:fref x-%data% (j) ((1 *))
                                    x-%offset%))
                    temp)))
      label100001))
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                   (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp
                (cabs1
                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)))
        (setf symb_zero
                (and symb_zero
                     (or
                      (=
                       (f2cl-lib:fref x-%data% (j) ((1 *))
                                      x-%offset%)
                       zero)
                      (= temp zero))))
        (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                (+
                 (f2cl-lib:fref y-%data% (iy) ((1 *))
                                y-%offset%)
                 (* alpha
                    (cabs1
                     (f2cl-lib:fref x-%data% (j) ((1 *))
                                    x-%offset%))
                    temp)))
      label100002))))
  (if (not symb_zero)
      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
              (+
               (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
               (f2cl-lib:sign safe1
                              (f2cl-lib:fref y-%data% (iy)
                                             ((1 *))
```

```
                                                    y-%offset%)))))
      (setf iy (f2cl-lib:int-add iy incy))
     label100000)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((= beta zero) (setf symb_zero f2cl-lib:%true%)
        (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                0.0d0))
       ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
        (setf symb_zero f2cl-lib:%true%))
       (t (setf symb_zero f2cl-lib:%false%)
        (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                (* beta
                   (abs
                    (f2cl-lib:fref y-%data% (iy) ((1 *))
                                   y-%offset%))))))
      (cond
       ((/= alpha zero)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j i) nil)
          (tagbody
            (setf temp
                    (cabs1
                     (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%)))
            (setf symb_zero
                    (and symb_zero
                         (or
                          (=
                           (f2cl-lib:fref x-%data% (j) ((1 *))
                                          x-%offset%)
                           zero)
                          (= temp zero))))
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+
                     (f2cl-lib:fref y-%data% (iy) ((1 *))
                                    y-%offset%)
                     (* alpha
                        (cabs1
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%))
                        temp)))
           label100004))
        (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                       (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
```

```
                (setf temp
                        (cabs1
                         (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                        a-%offset%)))
                (setf symb_zero
                        (and symb_zero
                             (or
                              (=
                               (f2cl-lib:fref x-%data% (j) ((1 *))
                                              x-%offset%)
                               zero)
                              (= temp zero))))
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data% (iy) ((1 *))
                                        y-%offset%)
                         (* alpha
                            (cabs1
                             (f2cl-lib:fref x-%data% (j) ((1 *))
                                            x-%offset%))
                            temp)))
             label100005))))
        (if (not symb_zero)
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+
                     (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (f2cl-lib:sign safe1
                                    (f2cl-lib:fref y-%data% (iy)
                                                   ((1 *))
                                                   y-%offset%)))))
        (setf iy (f2cl-lib:int-add iy incy))
       label100003)))))
(t
 (cond
  ((= uplo (ilauplo "U"))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (cond
        ((= beta zero) (setf symb_zero f2cl-lib:%true%)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 0.0d0))
        ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
         (setf symb_zero f2cl-lib:%true%))
        (t (setf symb_zero f2cl-lib:%false%)
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (* beta
                    (abs
                     (f2cl-lib:fref y-%data% (iy) ((1 *))
                                    y-%offset%))))))
```

```
(setf jx kx)
(cond
 ((/= alpha zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j i) nil)
    (tagbody
      (setf temp
              (cabs1
               (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                              a-%offset%)))
      (setf symb_zero
              (and symb_zero
                   (or
                    (=
                     (f2cl-lib:fref x-%data% (j) ((1 *))
                                    x-%offset%)
                     zero)
                    (= temp zero))))
      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
              (+
               (f2cl-lib:fref y-%data% (iy) ((1 *))
                              y-%offset%)
               (* alpha
                  (cabs1
                   (f2cl-lib:fref x-%data% (jx) ((1 *))
                                  x-%offset%))
                  temp)))
      (setf jx (f2cl-lib:int-add jx incx))
     label100007))
  (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf temp
              (cabs1
               (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                              a-%offset%)))
      (setf symb_zero
              (and symb_zero
                   (or
                    (=
                     (f2cl-lib:fref x-%data% (j) ((1 *))
                                    x-%offset%)
                     zero)
                    (= temp zero))))
      (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
              (+
               (f2cl-lib:fref y-%data% (iy) ((1 *))
                              y-%offset%)
               (* alpha
```

```
                              (cabs1
                               (f2cl-lib:fref x-%data% (jx) ((1 *))
                                              x-%offset%))
                              temp)))
                 (setf jx (f2cl-lib:int-add jx incx))
                label100008))))
          (if (not symb_zero)
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (+
                       (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (f2cl-lib:sign safe1
                                      (f2cl-lib:fref y-%data% (iy)
                                                     ((1 *))
                                                     y-%offset%)))))
          (setf iy (f2cl-lib:int-add iy incy))
         label100006)))
      (t
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (cond
            ((= beta zero) (setf symb_zero f2cl-lib:%true%)
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     0.0d0))
            ((= (f2cl-lib:fref y (iy) ((1 *))) zero)
             (setf symb_zero f2cl-lib:%true%))
            (t (setf symb_zero f2cl-lib:%false%)
             (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                     (* beta
                        (abs
                         (f2cl-lib:fref y-%data% (iy) ((1 *))
                                        y-%offset%))))))
           (setf jx kx)
           (cond
            ((/= alpha zero)
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j i) nil)
               (tagbody
                 (setf temp
                         (cabs1
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)))
                 (setf symb_zero
                         (and symb_zero
                              (or
                               (=
                                (f2cl-lib:fref x-%data% (j) ((1 *))
                                               x-%offset%)
                                zero)
                               (= temp zero))))
```

```
                    (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                            (+
                             (f2cl-lib:fref y-%data% (iy) ((1 *))
                                            y-%offset%)
                             (* alpha
                                (cabs1
                                 (f2cl-lib:fref x-%data% (jx) ((1 *))
                                                x-%offset%))
                                temp)))
                    (setf jx (f2cl-lib:int-add jx incx))
                   label100010))
                (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                               (f2cl-lib:int-add j 1))
                              ((> j n) nil)
                  (tagbody
                    (setf temp
                            (cabs1
                             (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                            a-%offset%)))
                    (setf symb_zero
                            (and symb_zero
                                 (or
                                  (=
                                   (f2cl-lib:fref x-%data% (j) ((1 *))
                                                  x-%offset%)
                                   zero)
                                  (= temp zero))))
                    (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                            (+
                             (f2cl-lib:fref y-%data% (iy) ((1 *))
                                            y-%offset%)
                             (* alpha
                                (cabs1
                                 (f2cl-lib:fref x-%data% (jx) ((1 *))
                                                x-%offset%))
                                temp)))
                    (setf jx (f2cl-lib:int-add jx incx))
                   label100011))))
            (if (not symb_zero)
                (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                        (+
                         (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (f2cl-lib:sign safe1
                                        (f2cl-lib:fref y-%data% (iy)
                                                       ((1 *))
                                                       y-%offset%)))))
            (setf iy (f2cl-lib:int-add iy incy))
           label100009))))))
  (go end_label)
 end_label
```

```
          (return (values nil nil nil nil nil nil nil nil nil nil)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zla_syamv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (double-float)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
                                              nil nil)
                                            :calls
                                            '(fortran-to-lisp::xerbla
                                              fortran-to-lisp::ilauplo))))
```

———-

61.2 zla_syrcond_c

61.2.1 Axiom unit tests

— zlasyrcondc.input —

```
)set break resume
)sys rm -f zlasyrcondc.output
)spool zlasyrcondc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.2.2 Axiom help page

— zla_syrcond_c.help —

```
==========================================================================
zla_syrcond_c examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function zla_syrcond_c (UPLO, N, A, LDA, AF, LDAF,
           IPIV, C, CAPPLY, INFO, WORK, RWORK)
           ZLA_SYRCOND_C computes the infinity norm condition number of
           op(A)*inv(diag(c)) for symmetric indefinite matrices.

Function/Subroutine Documentation
   double precision function zla_syrcond_c (character UPLO, integer N,
       complex*16, dimension( lda, * ) A, integer LDA, complex*16, dimension(
       ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV, double
       precision, dimension( * ) C, logical CAPPLY, integer INFO, complex*16,
       dimension( * ) WORK, double precision, dimension( * ) RWORK)
       ZLA_SYRCOND_C computes the infinity norm condition number of
       op(A)*inv(diag(c)) for symmetric indefinite matrices.

Purpose:
   ZLA_SYRCOND_C Computes the infinity norm condition number of
   op(A) * inv(diag(C)) where C is a DOUBLE PRECISION vector.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the N-by-N matrix A
```

```
LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX*16 array, dimension (LDAF,N)
  The block diagonal matrix D and the multipliers used to
  obtain the factor U or L as computed by ZSYTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by ZSYTRF.

C
  C is DOUBLE PRECISION array, dimension (N)
  The vector C in the formula op(A) * inv(diag(C)).

CAPPLY
  CAPPLY is LOGICAL
  If .TRUE. then access the vector C in the formula above.

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  i > 0:  The ith argument is invalid.

WORK
  WORK is COMPLEX*16 array, dimension (2*N).
  Workspace.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (N).
  Workspace.
```

———-

61.2.3 fortran code

— zla_syrcond_c.f —

```
*  =====================================================================
```

```
      DOUBLE PRECISION FUNCTION ZLA_SYRCOND_C( UPLO, N, A, LDA, AF,
     $                                         LDAF, IPIV, C, CAPPLY,
     $                                         INFO, WORK, RWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      LOGICAL            CAPPLY
      INTEGER            N, LDA, LDAF, INFO
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), WORK( * )
      DOUBLE PRECISION   C( * ), RWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            KASE
      DOUBLE PRECISION   AINVNM, ANORM, TMP
      INTEGER            I, J
      LOGICAL            UP, UPPER
      COMPLEX*16         ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZLACN2, ZSYTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
```

```
      ZLA_SYRCOND_C = 0.0D+0
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZLA_SYRCOND_C', -INFO )
         RETURN
      END IF
      UP = .FALSE.
      IF ( LSAME( UPLO, 'U' ) ) UP = .TRUE.
*
*     Compute norm of op(A)*op2(C).
*
      ANORM = 0.0D+0
      IF ( UP ) THEN
         DO I = 1, N
            TMP = 0.0D+0
            IF ( CAPPLY ) THEN
               DO J = 1, I
                  TMP = TMP + CABS1( A( J, I ) ) / C( J )
               END DO
               DO J = I+1, N
                  TMP = TMP + CABS1( A( I, J ) ) / C( J )
               END DO
            ELSE
               DO J = 1, I
                  TMP = TMP + CABS1( A( J, I ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + CABS1( A( I, J ) )
               END DO
            END IF
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      ELSE
         DO I = 1, N
            TMP = 0.0D+0
            IF ( CAPPLY ) THEN
               DO J = 1, I
                  TMP = TMP + CABS1( A( I, J ) ) / C( J )
```

```
            END DO
            DO J = I+1, N
               TMP = TMP + CABS1( A( J, I ) ) / C( J )
            END DO
         ELSE
            DO J = 1, I
               TMP = TMP + CABS1( A( I, J ) )
            END DO
            DO J = I+1, N
               TMP = TMP + CABS1( A( J, I ) )
            END DO
         END IF
         RWORK( I ) = TMP
         ANORM = MAX( ANORM, TMP )
      END DO
   END IF
*
*     Quick return if possible.
*
   IF( N.EQ.0 ) THEN
      ZLA_SYRCOND_C = 1.0D+0
      RETURN
   ELSE IF( ANORM .EQ. 0.0D+0 ) THEN
      RETURN
   END IF
*
*     Estimate the norm of inv(op(A)).
*
   AINVNM = 0.0D+0
*
   KASE = 0
10 CONTINUE
   CALL ZLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
   IF( KASE.NE.0 ) THEN
      IF( KASE.EQ.2 ) THEN
*
*        Multiply by R.
*
         DO I = 1, N
            WORK( I ) = WORK( I ) * RWORK( I )
         END DO
*
         IF ( UP ) THEN
            CALL ZSYTRS( 'U', N, 1, AF, LDAF, IPIV,
     $         WORK, N, INFO )
         ELSE
            CALL ZSYTRS( 'L', N, 1, AF, LDAF, IPIV,
     $         WORK, N, INFO )
         ENDIF
*
```

```
*          Multiply by inv(C).
*
           IF ( CAPPLY ) THEN
              DO I = 1, N
                 WORK( I ) = WORK( I ) * C( I )
              END DO
           END IF
        ELSE
*
*          Multiply by inv(C**T).
*
           IF ( CAPPLY ) THEN
              DO I = 1, N
                 WORK( I ) = WORK( I ) * C( I )
              END DO
           END IF
*
           IF ( UP ) THEN
              CALL ZSYTRS( 'U', N, 1, AF, LDAF, IPIV,
     $           WORK, N, INFO )
           ELSE
              CALL ZSYTRS( 'L', N, 1, AF, LDAF, IPIV,
     $           WORK, N, INFO )
           END IF
*
*          Multiply by R.
*
           DO I = 1, N
              WORK( I ) = WORK( I ) * RWORK( I )
           END DO
        END IF
        GO TO 10
     END IF
*
*    Compute the estimate of the reciprocal condition number.
*
     IF( AINVNM .NE. 0.0D+0 )
     $   ZLA_SYRCOND_C = 1.0D+0 / AINVNM
*
     RETURN
*
     END
```

———-

61.2.4 lisp code

— zla_syrcond_c.lisp —

```
(defun zla_syrcond_c (uplo n a lda af ldaf ipiv c capply info work rwork)
  (declare (type f2cl-lib:logical capply)
           (type (array double-float (*)) rwork c)
           (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex16 (*)) work af a)
           (type (f2cl-lib:integer4) info ldaf lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (af f2cl-lib:complex16 af-%data% af-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (c double-float c-%data% c-%offset%)
       (rwork double-float rwork-%data% rwork-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
        cabs1))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (zdum #C(0.0d0 0.0d0)) (up nil) (upper nil) (i 0) (j 0)
             (ainvnm 0.0d0) (anorm 0.0d0) (tmp 0.0d0) (kase 0)
             (zla_syrcond_c 0.0d0))
        (declare (type (double-float) zla_syrcond_c tmp anorm ainvnm)
                 (type (f2cl-lib:integer4) kase j i)
                 (type f2cl-lib:logical upper up)
                 (type (f2cl-lib:complex16) zdum)
                 (type (array f2cl-lib:integer4 (3)) isave))
        (setf zla_syrcond_c 0.0d0)
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -4))
         ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
```

```
  (setf info -6)))
(cond
 ((/= info 0) (xerbla "ZLA_SYRCOND_C" (f2cl-lib:int-sub info))
  (go end_label)))
(setf up f2cl-lib:%false%)
(if (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)
    (setf up f2cl-lib:%true%))
(setf anorm 0.0d0)
(cond
 (up
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf tmp 0.0d0)
      (cond
       (capply
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j i) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (/
                        (cabs1
                         (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                        a-%offset%))
                        (f2cl-lib:fref c-%data% (j) ((1 *))
                                       c-%offset%))))
           label100001))
        (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (/
                        (cabs1
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%))
                        (f2cl-lib:fref c-%data% (j) ((1 *))
                                       c-%offset%))))
           label100002)))
       (t
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j i) nil)
          (tagbody
            (setf tmp
                    (+ tmp
```

```
                          (cabs1
                           (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                          a-%offset%))))
                label100003))
            (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf tmp
                        (+ tmp
                           (cabs1
                            (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                           a-%offset%))))
                label100004))))
          (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                  tmp)
          (setf anorm (max anorm tmp))
         label100000)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf tmp 0.0d0)
          (cond
           (capply
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j i) nil)
              (tagbody
                (setf tmp
                        (+ tmp
                           (/
                            (cabs1
                             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                            a-%offset%))
                            (f2cl-lib:fref c-%data% (j) ((1 *))
                                           c-%offset%))))
                label100006))
            (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf tmp
                        (+ tmp
                           (/
                            (cabs1
                             (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                            a-%offset%))
                            (f2cl-lib:fref c-%data% (j) ((1 *))
                                           c-%offset%))))
                label100007)))
           (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                        ((> j i) nil)
              (tagbody
                (setf tmp
                        (+ tmp
                           (cabs1
                            (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                           a-%offset%))))
               label100008))
            (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                        ((> j n) nil)
              (tagbody
                (setf tmp
                        (+ tmp
                           (cabs1
                            (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                           a-%offset%))))
               label100009))))
          (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                  tmp)
          (setf anorm (max anorm tmp))
         label100005))))
  (cond ((= n 0) (setf zla_syrcond_c 1.0d0) (go end_label))
        ((= anorm 0.0d0) (go end_label)))
  (setf ainvnm 0.0d0)
  (setf kase 0)
 label10
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (zlacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 ((+ n 1))
                             ((1 *)) work-%offset%)
       work ainvnm kase isave)
    (declare (ignore var-1 var-2 var-5))
    (setf n var-0)
    (setf ainvnm var-3)
    (setf kase var-4))
  (cond
   ((/= kase 0)
    (cond
     ((= kase 2)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (*
                   (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                   (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                  rwork-%offset%)))
         label100010))
      (cond
       (up
```

```
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (zsytrs "U" n 1 af ldaf ipiv work n info)
      (declare (ignore var-0 var-2 var-3 var-5 var-6))
      (when var-1 (setf n var-1))
      (when var-4 (setf ldaf var-4))
      (when var-7 (setf n var-7))
      (when var-8 (setf info var-8))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (zsytrs "L" n 1 af ldaf ipiv work n info)
      (declare (ignore var-0 var-2 var-3 var-5 var-6))
      (when var-1 (setf n var-1))
      (when var-4 (setf ldaf var-4))
      (when var-7 (setf n var-7))
      (when var-8 (setf info var-8)))))
  (cond
   (capply
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (*
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
       label100011)))))
 (t
  (cond
   (capply
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (*
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
       label100012))))
  (cond
   (up
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (zsytrs "U" n 1 af ldaf ipiv work n info)
      (declare (ignore var-0 var-2 var-3 var-5 var-6))
      (when var-1 (setf n var-1))
      (when var-4 (setf ldaf var-4))
      (when var-7 (setf n var-7))
      (when var-8 (setf info var-8))))
```

```
             (t
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                  (zsytrs "L" n 1 af ldaf ipiv work n info)
                (declare (ignore var-0 var-2 var-3 var-5 var-6))
                (when var-1 (setf n var-1))
                (when var-4 (setf ldaf var-4))
                (when var-7 (setf n var-7))
                (when var-8 (setf info var-8)))))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                        (*
                         (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                         (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                        rwork-%offset%)))
               label100013))))
          (go label10)))
        (if (/= ainvnm 0.0d0)
            (setf zla_syrcond_c (/ 1.0d0 ainvnm)))
        (go end_label)
       end_label
        (return
         (values zla_syrcond_c uplo n nil nil nil ldaf nil nil nil info nil
                 nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zla_syrcond_c
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              fortran-to-lisp::logical
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array double-float (*)))
                                            :return-values
```

```
                                        '(fortran-to-lisp::uplo
                                          fortran-to-lisp::n nil nil nil
                                          fortran-to-lisp::ldaf nil nil nil
                                          fortran-to-lisp::info nil nil)
                                        :calls
                                        '(fortran-to-lisp::zlacn2
                                          fortran-to-lisp::xerbla))))
```

———-

61.3 zla_syrcond_x

61.3.1 Axiom unit tests

— **zlasyrcondx.input** —

```
)set break resume
)sys rm -f zlasyrcondx.output
)spool zlasyrcondx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.3.2 Axiom help page

— **zla_syrcond_x.help** —

```
==================================================================
zla_syrcond_x examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
```

```
      double precision function zla_syrcond_x (UPLO, N, A, LDA, AF, LDAF,
          IPIV, X, INFO, WORK, RWORK)
          ZLA_SYRCOND_X computes the infinity norm condition number of
          op(A)*diag(x) for symmetric indefinite matrices.

Function/Subroutine Documentation
   double precision function zla_syrcond_x (character UPLO, integer N,
       complex*16, dimension( lda, * ) A, integer LDA, complex*16, dimension(
       ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV, complex*16,
       dimension( * ) X, integer INFO, complex*16, dimension( * ) WORK, double
       precision, dimension( * ) RWORK)
       ZLA_SYRCOND_X computes the infinity norm condition number of
       op(A)*diag(x) for symmetric indefinite matrices.

Purpose:
   ZLA_SYRCOND_X Computes the infinity norm condition number of
   op(A) * diag(X) where X is a COMPLEX*16 vector.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is COMPLEX*16 array, dimension (LDAF,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by ZSYTRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by ZSYTRF.
```

```
 X
   X is COMPLEX*16 array, dimension (N)
   The vector X in the formula op(A) * diag(X).

 INFO
   INFO is INTEGER
   = 0:  Successful exit.
   i > 0:  The ith argument is invalid.

 WORK
   WORK is COMPLEX*16 array, dimension (2*N).
   Workspace.

 RWORK
   RWORK is DOUBLE PRECISION array, dimension (N).
   Workspace.
```

61.3.3 fortran code

— zla_syrcond_x.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION ZLA_SYRCOND_X( UPLO, N, A, LDA, AF,
     $                                         LDAF, IPIV, X, INFO,
     $                                         WORK, RWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            N, LDA, LDAF, INFO
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), WORK( * ), X( * )
      DOUBLE PRECISION   RWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
```

```
      INTEGER            KASE
      DOUBLE PRECISION   AINVNM, ANORM, TMP
      INTEGER            I, J
      LOGICAL            UP, UPPER
      COMPLEX*16         ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZLACN2, ZSYTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      ZLA_SYRCOND_X = 0.0D+0
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZLA_SYRCOND_X', -INFO )
         RETURN
      END IF
      UP = .FALSE.
      IF ( LSAME( UPLO, 'U' ) ) UP = .TRUE.
*
*     Compute norm of op(A)*op2(C).
*
      ANORM = 0.0D+0
```

```
      IF ( UP ) THEN
         DO I = 1, N
            TMP = 0.0D+0
            DO J = 1, I
               TMP = TMP + CABS1( A( J, I ) * X( J ) )
            END DO
            DO J = I+1, N
               TMP = TMP + CABS1( A( I, J ) * X( J ) )
            END DO
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      ELSE
         DO I = 1, N
            TMP = 0.0D+0
            DO J = 1, I
               TMP = TMP + CABS1( A( I, J ) * X( J ) )
            END DO
            DO J = I+1, N
               TMP = TMP + CABS1( A( J, I ) * X( J ) )
            END DO
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 ) THEN
         ZLA_SYRCOND_X = 1.0D+0
         RETURN
      ELSE IF( ANORM .EQ. 0.0D+0 ) THEN
         RETURN
      END IF
*
*     Estimate the norm of inv(op(A)).
*
      AINVNM = 0.0D+0
*
      KASE = 0
   10 CONTINUE
      CALL ZLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
```

```
*
            IF ( UP ) THEN
               CALL ZSYTRS( 'U', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ELSE
               CALL ZSYTRS( 'L', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ENDIF
*
*           Multiply by inv(X).
*
            DO I = 1, N
               WORK( I ) = WORK( I ) / X( I )
            END DO
         ELSE
*
*           Multiply by inv(X**T).
*
            DO I = 1, N
               WORK( I ) = WORK( I ) / X( I )
            END DO
*
            IF ( UP ) THEN
               CALL ZSYTRS( 'U', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            ELSE
               CALL ZSYTRS( 'L', N, 1, AF, LDAF, IPIV,
     $            WORK, N, INFO )
            END IF
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM .NE. 0.0D+0 )
     $   ZLA_SYRCOND_X = 1.0D+0 / AINVNM
*
      RETURN
*
      END
```

———

61.3.4 lisp code

— zla_syrcond_x.lisp —

```
(defun zla_syrcond_x (uplo n a lda af ldaf ipiv x info work rwork)
  (declare (type (array double-float (*)) rwork)
           (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex16 (*)) work x af a)
           (type (f2cl-lib:integer4) info ldaf lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (af f2cl-lib:complex16 af-%data% af-%offset%)
       (x f2cl-lib:complex16 x-%data% x-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (rwork double-float rwork-%data% rwork-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
        cabs1))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (zdum #C(0.0d0 0.0d0)) (up nil) (upper nil) (i 0) (j 0)
             (ainvnm 0.0d0) (anorm 0.0d0) (tmp 0.0d0) (kase 0)
             (zla_syrcond_x 0.0d0))
        (declare (type (double-float) zla_syrcond_x tmp anorm ainvnm)
                 (type (f2cl-lib:integer4) kase j i)
                 (type f2cl-lib:logical upper up)
                 (type (f2cl-lib:complex16) zdum)
                 (type (array f2cl-lib:integer4 (3)) isave))
        (setf zla_syrcond_x 0.0d0)
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
```

```
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6)))
(cond
 ((/= info 0) (xerbla "ZLA_SYRCOND_X" (f2cl-lib:int-sub info))
  (go end_label)))
(setf up f2cl-lib:%false%)
(if (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)
    (setf up f2cl-lib:%true%))
(setf anorm 0.0d0)
(cond
 (up
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf tmp 0.0d0)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j i) nil)
        (tagbody
          (setf tmp
                  (+ tmp
                     (cabs1
                      (*
                       (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                      a-%offset%)
                       (f2cl-lib:fref x-%data% (j) ((1 *))
                                      x-%offset%)))))
         label100001))
      (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf tmp
                  (+ tmp
                     (cabs1
                      (*
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                       (f2cl-lib:fref x-%data% (j) ((1 *))
                                      x-%offset%)))))
         label100002))
      (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
              tmp)
      (setf anorm (max anorm tmp))
     label100000)))
```

```
    (t
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf tmp 0.0d0)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j i) nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (cabs1
                         (*
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref x-%data% (j) ((1 *))
                                         x-%offset%)))))
            label100004))
         (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (cabs1
                         (*
                          (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref x-%data% (j) ((1 *))
                                         x-%offset%)))))
            label100005))
         (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                 tmp)
         (setf anorm (max anorm tmp))
        label100003))))
  (cond ((= n 0) (setf zla_syrcond_x 1.0d0) (go end_label))
        ((= anorm 0.0d0) (go end_label)))
  (setf ainvnm 0.0d0)
  (setf kase 0)
 label10
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (zlacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 ((+ n 1))
                             ((1 *)) work-%offset%)
       work ainvnm kase isave)
    (declare (ignore var-1 var-2 var-5))
    (setf n var-0)
    (setf ainvnm var-3)
    (setf kase var-4))
  (cond
   ((/= kase 0)
    (cond
```

```
((= kase 2)
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
             (*
              (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (f2cl-lib:fref rwork-%data% (i) ((1 *))
                             rwork-%offset%)))
    label100006))
 (cond
  (up
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (zsytrs "U" n 1 af ldaf ipiv work n info)
     (declare (ignore var-0 var-2 var-3 var-5 var-6))
     (when var-1 (setf n var-1))
     (when var-4 (setf ldaf var-4))
     (when var-7 (setf n var-7))
     (when var-8 (setf info var-8))))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (zsytrs "L" n 1 af ldaf ipiv work n info)
     (declare (ignore var-0 var-2 var-3 var-5 var-6))
     (when var-1 (setf n var-1))
     (when var-4 (setf ldaf var-4))
     (when var-7 (setf n var-7))
     (when var-8 (setf info var-8)))))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
             (/
              (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)))
    label100007)))
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
             (/
              (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)))
    label100008))
 (cond
  (up
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
```

```
                (zsytrs "U" n 1 af ldaf ipiv work n info)
              (declare (ignore var-0 var-2 var-3 var-5 var-6))
              (when var-1 (setf n var-1))
              (when var-4 (setf ldaf var-4))
              (when var-7 (setf n var-7))
              (when var-8 (setf info var-8))))
            (t
             (multiple-value-bind
                 (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                 (zsytrs "L" n 1 af ldaf ipiv work n info)
              (declare (ignore var-0 var-2 var-3 var-5 var-6))
              (when var-1 (setf n var-1))
              (when var-4 (setf ldaf var-4))
              (when var-7 (setf n var-7))
              (when var-8 (setf info var-8)))))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
             (tagbody
              (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                      (*
                       (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                       (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                      rwork-%offset%)))
               label100009))))
         (go label10)))
       (if (/= ainvnm 0.0d0)
           (setf zla_syrcond_x (/ 1.0d0 ainvnm)))
       (go end_label)
      end_label
       (return
        (values zla_syrcond_x uplo n nil nil nil ldaf nil nil info nil
                nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zla_syrcond_x
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::integer4
                                              (*))
                                             (array fortran-to-lisp::complex16
```

```
    (*))
   (fortran-to-lisp::integer4)
   (array fortran-to-lisp::complex16
    (*))
   (array double-float (*)))
 :return-values
 '(fortran-to-lisp::uplo
   fortran-to-lisp::n nil nil nil
   fortran-to-lisp::ldaf nil nil
   fortran-to-lisp::info nil nil)
 :calls
 '(fortran-to-lisp::zlacn2
   fortran-to-lisp::xerbla))))
```

———-

61.4 zla_syrfsx_extended

61.4.1 Axiom unit tests

— **zlasyrfsxextended.input** —

```
)set break resume
)sys rm -f zlasyrfsxextended.output
)spool zlasyrfsxextended.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.4.2 Axiom help page [Demm05]

— **zla_syrfsx_extended.help** —

```
==========================================================================
zla_syrfsx_extended examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zla_syrfsx_extended (PREC_TYPE, UPLO, N, NRHS, A, LDA, AF,
           LDAF, IPIV, COLEQU, C, B, LDB, Y, LDY, BERR_OUT, N_NORMS,
           ERR_BNDS_NORM, ERR_BNDS_COMP, RES, AYB, DY, Y_TAIL, RCOND, ITHRESH,
           RTHRESH, DZ_UB, IGNORE_CWISE, INFO)
           ZLA_SYRFSX_EXTENDED improves the computed solution to a system of
           linear equations for symmetric indefinite matrices by performing
           extra-precise iterative refinement and provides error bounds and
           backward error estimates for the solution.

Function/Subroutine Documentation
   subroutine zla_syrfsx_extended (integer PREC_TYPE, character UPLO, integer
       N, integer NRHS, complex*16, dimension( lda, * ) A, integer LDA,
       complex*16, dimension( ldaf, * ) AF, integer LDAF, integer, dimension(
       * ) IPIV, logical COLEQU, double precision, dimension( * ) C,
       complex*16, dimension( ldb, * ) B, integer LDB, complex*16, dimension(
       ldy, * ) Y, integer LDY, double precision, dimension( * ) BERR_OUT,
       integer N_NORMS, double precision, dimension( nrhs, * ) ERR_BNDS_NORM,
       double precision, dimension( nrhs, * ) ERR_BNDS_COMP, complex*16,
       dimension( * ) RES, double precision, dimension( * ) AYB, complex*16,
       dimension( * ) DY, complex*16, dimension( * ) Y_TAIL, double precision
       RCOND, integer ITHRESH, double precision RTHRESH, double precision
       DZ_UB, logical IGNORE_CWISE, integer INFO)
       ZLA_SYRFSX_EXTENDED improves the computed solution to a system of
       linear equations for symmetric indefinite matrices by performing extra-
       precise iterative refinement and provides error bounds and backward
       error estimates for the solution.

Purpose:
   ZLA_SYRFSX_EXTENDED improves the computed solution to a system of
   linear equations by performing extra-precise iterative refinement
   and provides error bounds and backward error estimates for the solution.
   This subroutine is called by ZSYRFSX to perform iterative refinement.
   In addition to normwise error bound, the code provides maximum
   componentwise error bound if possible. See comments for ERR_BNDS_NORM
   and ERR_BNDS_COMP for details of the error bounds. Note that this
   subroutine is only resonsible for setting the second fields of
   ERR_BNDS_NORM and ERR_BNDS_COMP.

Parameters:

 PREC_TYPE
   PREC_TYPE is INTEGER
   Specifies the intermediate precision to be used in refinement.
   The value is defined by ILAPREC(P) where P is a CHARACTER and
```

```
  P    = 'S':  Single
       = 'D':  Double
       = 'I':  Indigenous
       = 'X', 'E':  Extra

UPLO
  UPLO is CHARACTER*1
  = 'U':  Upper triangle of A is stored;
  = 'L':  Lower triangle of A is stored.

N
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right-hand-sides, i.e., the number of columns of the
  matrix B.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the N-by-N matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX*16 array, dimension (LDAF,N)
  The block diagonal matrix D and the multipliers used to
  obtain the factor U or L as computed by ZSYTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by ZSYTRF.

COLEQU
  COLEQU is LOGICAL
  If .TRUE. then column equilibration was done to A before calling
  this routine. This is needed to compute the solution and error
  bounds correctly.

C
  C is DOUBLE PRECISION array, dimension (N)
  The column scale factors for A. If COLEQU = .FALSE., C
```

```
  is not accessed. If C is input, each element of C should be a power
  of the radix to ensure a reliable solution and error estimates.
  Scaling by powers of the radix does not cause rounding errors unless
  the result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  The right-hand-side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

Y
  Y is COMPLEX*16 array, dimension (LDY,NRHS)
  On entry, the solution matrix X, as computed by ZSYTRS.
  On exit, the improved solution matrix Y.

LDY
  LDY is INTEGER
  The leading dimension of the array Y.  LDY >= max(1,N).

BERR_OUT
  BERR_OUT is DOUBLE PRECISION array, dimension (NRHS)
  On exit, BERR_OUT(j) contains the componentwise relative backward
  error for right-hand-side j from the formula
  max(i) ( abs(RES(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
  where abs(Z) is the componentwise absolute value of the matrix
  or vector Z. This is computed by ZLA_LIN_BERR.

N_NORMS
  N_NORMS is INTEGER
  Determines which error bounds to return (see ERR_BNDS_NORM
  and ERR_BNDS_COMP).
  If N_NORMS >= 1 return normwise error bounds.
  If N_NORMS >= 2 return componentwise error bounds.

ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
           max_j (abs(XTRUE(j,i) - X(j,i)))
            ------------------------------
                   max_j abs(X(j,i))
```

```
  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
    reciprocal condition number is less than the threshold
    sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
    almost certainly within a factor of 10 of the true error
    so long as the next entry is greater than the threshold
    sqrt(n) * slamch('Epsilon'). This error bound should only
    be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
    reciprocal condition number.  Compared with the threshold
    sqrt(n) * slamch('Epsilon') to determine if the error
    estimate is "guaranteed". These reciprocal condition
    numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
    appropriately scaled matrix Z.
    Let Z = S*A, where S scales each row by a power of the
    radix so all absolute row sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

ERR_BNDS_COMP
  ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                 abs(XTRUE(j,i) - X(j,i))
            max_j ----------------------
                        abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.
```

```
  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

RES
  RES is COMPLEX*16 array, dimension (N)
  Workspace to hold the intermediate residual.

AYB
  AYB is DOUBLE PRECISION array, dimension (N)
  Workspace.

DY
  DY is COMPLEX*16 array, dimension (N)
  Workspace to hold the intermediate solution.

Y_TAIL
  Y_TAIL is COMPLEX*16 array, dimension (N)
  Workspace to hold the trailing bits of the intermediate solution.

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
```

```
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

ITHRESH
  ITHRESH is INTEGER
  The maximum number of residual computations allowed for
  refinement. The default is 10. For 'aggressive' set to 100 to
  permit convergence using approximate factorizations or
  factorizations other than LU. If the factorization uses a
  technique other than Gaussian elimination, the guarantees in
  ERR_BNDS_NORM and ERR_BNDS_COMP may no longer be trustworthy.

RTHRESH
  RTHRESH is DOUBLE PRECISION
  Determines when to stop refinement if the error estimate stops
  decreasing. Refinement will stop when the next solution no longer
  satisfies norm(dx_{i+1}) < RTHRESH * norm(dx_i) where norm(Z) is
  the infinity norm of Z. RTHRESH satisfies 0 < RTHRESH <= 1. The
  default value is 0.5. For 'aggressive' set to 0.9 to permit
  convergence on extremely ill-conditioned matrices. See LAWN 165
  for more details.

DZ_UB
  DZ_UB is DOUBLE PRECISION
  Determines when to start considering componentwise convergence.
  Componentwise convergence is only considered after each component
  of the solution Y is stable, which we definte as the relative
  change in each component being less than DZ_UB. The default value
  is 0.25, requiring the first bit to be stable. See LAWN 165 for
  more details.

IGNORE_CWISE
  IGNORE_CWISE is LOGICAL
  If .TRUE. then ignore componentwise convergence. Default value
  is .FALSE..

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  < 0:  if INFO = -i, the ith argument to ZLA_HERFSX_EXTENDED had an illegal
  value
```

———-

61.4.3 fortran code

— zla_syrfsx_extended.f —

```
*  =====================================================================
      SUBROUTINE ZLA_SYRFSX_EXTENDED( PREC_TYPE, UPLO, N, NRHS, A, LDA,
     $                                AF, LDAF, IPIV, COLEQU, C, B, LDB,
     $                                Y, LDY, BERR_OUT, N_NORMS,
     $                                ERR_BNDS_NORM, ERR_BNDS_COMP, RES,
     $                                AYB, DY, Y_TAIL, RCOND, ITHRESH,
     $                                RTHRESH, DZ_UB, IGNORE_CWISE,
     $                                INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDAF, LDB, LDY, N, NRHS, PREC_TYPE,
     $                   N_NORMS, ITHRESH
      CHARACTER          UPLO
      LOGICAL            COLEQU, IGNORE_CWISE
      DOUBLE PRECISION   RTHRESH, DZ_UB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   Y( LDY, * ), RES( * ), DY( * ), Y_TAIL( * )
      DOUBLE PRECISION   C( * ), AYB( * ), RCOND, BERR_OUT( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            UPLO2, CNT, I, J, X_STATE, Z_STATE,
     $                   Y_PREC_STATE
      DOUBLE PRECISION   YK, DYK, YMIN, NORMY, NORMX, NORMDX, DXRAT,
     $                   DZRAT, PREVNORMDX, PREV_DZ_Z, DXRATMAX,
     $                   DZRATMAX, DX_X, DZ_Z, FINAL_DX_X, FINAL_DZ_Z,
     $                   EPS, HUGEVAL, INCR_THRESH
      LOGICAL            INCR_PREC, UPPER
      COMPLEX*16         ZDUM
*     ..
*     .. Parameters ..
      INTEGER            UNSTABLE_STATE, WORKING_STATE, CONV_STATE,
     $                   NOPROG_STATE, BASE_RESIDUAL, EXTRA_RESIDUAL,
```

```
     $                   EXTRA_Y
      PARAMETER          ( UNSTABLE_STATE = 0, WORKING_STATE = 1,
     $                   CONV_STATE = 2, NOPROG_STATE = 3 )
      PARAMETER          ( BASE_RESIDUAL = 0, EXTRA_RESIDUAL = 1,
     $                   EXTRA_Y = 2 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           ILAUPLO
      INTEGER            ILAUPLO
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZAXPY, ZCOPY, ZSYTRS, ZSYMV, BLAS_ZSYMV_X,
     $                   BLAS_ZSYMV2_X, ZLA_SYAMV, ZLA_WWADDW,
     $                   ZLA_LIN_BERR
      DOUBLE PRECISION   DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, REAL, DIMAG, MAX, MIN
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
```

```
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -13
      ELSE IF( LDY.LT.MAX( 1, N ) ) THEN
         INFO = -15
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZLA_HERFSX_EXTENDED', -INFO )
         RETURN
      END IF
      EPS = DLAMCH( 'Epsilon' )
      HUGEVAL = DLAMCH( 'Overflow' )
*     Force HUGEVAL to Inf
      HUGEVAL = HUGEVAL * HUGEVAL
*     Using HUGEVAL may lead to spurious underflows.
      INCR_THRESH = DBLE( N ) * EPS

      IF ( LSAME ( UPLO, 'L' ) ) THEN
         UPLO2 = ILAUPLO( 'L' )
      ELSE
         UPLO2 = ILAUPLO( 'U' )
      ENDIF

      DO J = 1, NRHS
         Y_PREC_STATE = EXTRA_RESIDUAL
         IF ( Y_PREC_STATE .EQ. EXTRA_Y ) THEN
            DO I = 1, N
               Y_TAIL( I ) = 0.0D+0
            END DO
         END IF

         DXRAT = 0.0D+0
         DXRATMAX = 0.0D+0
         DZRAT = 0.0D+0
         DZRATMAX = 0.0D+0
         FINAL_DX_X = HUGEVAL
         FINAL_DZ_Z = HUGEVAL
         PREVNORMDX = HUGEVAL
         PREV_DZ_Z = HUGEVAL
         DZ_Z = HUGEVAL
         DX_X = HUGEVAL

         X_STATE = WORKING_STATE
         Z_STATE = UNSTABLE_STATE
```

```
         INCR_PREC = .FALSE.

         DO CNT = 1, ITHRESH
*
*         Compute residual RES = B_s - op(A_s) * Y,
*             op(A) = A, A**T, or A**H depending on TRANS (and type).
*
           CALL ZCOPY( N, B( 1, J ), 1, RES, 1 )
           IF ( Y_PREC_STATE .EQ. BASE_RESIDUAL ) THEN
              CALL ZSYMV( UPLO, N, DCMPLX(-1.0D+0), A, LDA, Y(1,J), 1,
     $             DCMPLX(1.0D+0), RES, 1 )
           ELSE IF ( Y_PREC_STATE .EQ. EXTRA_RESIDUAL ) THEN
              CALL BLAS_ZSYMV_X( UPLO2, N, DCMPLX(-1.0D+0), A, LDA,
     $             Y( 1, J ), 1, DCMPLX(1.0D+0), RES, 1, PREC_TYPE )
           ELSE
              CALL BLAS_ZSYMV2_X(UPLO2, N, DCMPLX(-1.0D+0), A, LDA,
     $             Y(1, J), Y_TAIL, 1, DCMPLX(1.0D+0), RES, 1,
     $     PREC_TYPE)
           END IF

!         XXX: RES is no longer needed.
           CALL ZCOPY( N, RES, 1, DY, 1 )
           CALL ZSYTRS( UPLO, N, 1, AF, LDAF, IPIV, DY, N, INFO )
*
*         Calculate relative changes DX_X, DZ_Z and ratios DXRAT, DZRAT.
*
           NORMX = 0.0D+0
           NORMY = 0.0D+0
           NORMDX = 0.0D+0
           DZ_Z = 0.0D+0
           YMIN = HUGEVAL

           DO I = 1, N
              YK = CABS1( Y( I, J ) )
              DYK = CABS1( DY( I ) )

              IF ( YK .NE. 0.0D+0 ) THEN
                 DZ_Z = MAX( DZ_Z, DYK / YK )
              ELSE IF ( DYK .NE. 0.0D+0 ) THEN
                 DZ_Z = HUGEVAL
              END IF

              YMIN = MIN( YMIN, YK )

              NORMY = MAX( NORMY, YK )

              IF ( COLEQU ) THEN
                 NORMX = MAX( NORMX, YK * C( I ) )
                 NORMDX = MAX( NORMDX, DYK * C( I ) )
              ELSE
```

```
                 NORMX = NORMY
                 NORMDX = MAX( NORMDX, DYK )
              END IF
           END DO

           IF ( NORMX .NE. 0.0D+0 ) THEN
              DX_X = NORMDX / NORMX
           ELSE IF ( NORMDX .EQ. 0.0D+0 ) THEN
              DX_X = 0.0D+0
           ELSE
              DX_X = HUGEVAL
           END IF

           DXRAT = NORMDX / PREVNORMDX
           DZRAT = DZ_Z / PREV_DZ_Z
*
*        Check termination criteria.
*
           IF ( YMIN*RCOND .LT. INCR_THRESH*NORMY
     $           .AND. Y_PREC_STATE .LT. EXTRA_Y )
     $           INCR_PREC = .TRUE.

           IF ( X_STATE .EQ. NOPROG_STATE .AND. DXRAT .LE. RTHRESH )
     $           X_STATE = WORKING_STATE
           IF ( X_STATE .EQ. WORKING_STATE ) THEN
              IF ( DX_X .LE. EPS ) THEN
                 X_STATE = CONV_STATE
              ELSE IF ( DXRAT .GT. RTHRESH ) THEN
                 IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                    INCR_PREC = .TRUE.
                 ELSE
                    X_STATE = NOPROG_STATE
                 END IF
              ELSE
                 IF (DXRAT .GT. DXRATMAX) DXRATMAX = DXRAT
              END IF
              IF ( X_STATE .GT. WORKING_STATE ) FINAL_DX_X = DX_X
           END IF

           IF ( Z_STATE .EQ. UNSTABLE_STATE .AND. DZ_Z .LE. DZ_UB )
     $           Z_STATE = WORKING_STATE
           IF ( Z_STATE .EQ. NOPROG_STATE .AND. DZRAT .LE. RTHRESH )
     $           Z_STATE = WORKING_STATE
           IF ( Z_STATE .EQ. WORKING_STATE ) THEN
              IF ( DZ_Z .LE. EPS ) THEN
                 Z_STATE = CONV_STATE
              ELSE IF ( DZ_Z .GT. DZ_UB ) THEN
                 Z_STATE = UNSTABLE_STATE
                 DZRATMAX = 0.0D+0
                 FINAL_DZ_Z = HUGEVAL
```

```
               ELSE IF ( DZRAT .GT. RTHRESH ) THEN
                  IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                     INCR_PREC = .TRUE.
                  ELSE
                     Z_STATE = NOPROG_STATE
                  END IF
               ELSE
                  IF ( DZRAT .GT. DZRATMAX ) DZRATMAX = DZRAT
               END IF
               IF ( Z_STATE .GT. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
            END IF

            IF ( X_STATE.NE.WORKING_STATE.AND.
     $           ( IGNORE_CWISE.OR.Z_STATE.NE.WORKING_STATE ) )
     $          GOTO 666

            IF ( INCR_PREC ) THEN
               INCR_PREC = .FALSE.
               Y_PREC_STATE = Y_PREC_STATE + 1
               DO I = 1, N
                  Y_TAIL( I ) = 0.0D+0
               END DO
            END IF

            PREVNORMDX = NORMDX
            PREV_DZ_Z = DZ_Z
*
*           Update soluton.
*
            IF ( Y_PREC_STATE .LT. EXTRA_Y ) THEN
               CALL ZAXPY( N, DCMPLX(1.0D+0), DY, 1, Y(1,J), 1 )
            ELSE
               CALL ZLA_WWADDW( N, Y(1,J), Y_TAIL, DY )
            END IF

         END DO
*        Target of "IF (Z_STOP .AND. X_STOP)".  Sun's f77 won't EXIT.
 666     CONTINUE
*
*     Set final_* when cnt hits ithresh.
*
         IF ( X_STATE .EQ. WORKING_STATE ) FINAL_DX_X = DX_X
         IF ( Z_STATE .EQ. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
*
*     Compute error bounds.
*
         IF ( N_NORMS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) =
     $           FINAL_DX_X / (1 - DXRATMAX)
         END IF
```

```
         IF ( N_NORMS .GE. 2 ) THEN
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) =
     $           FINAL_DZ_Z / (1 - DZRATMAX)
         END IF
*
*     Compute componentwise relative backward error from formula
*         max(i) ( abs(R(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
*     where abs(Z) is the componentwise absolute value of the matrix
*     or vector Z.
*
*        Compute residual RES = B_s - op(A_s) * Y,
*            op(A) = A, A**T, or A**H depending on TRANS (and type).
*
         CALL ZCOPY( N, B( 1, J ), 1, RES, 1 )
         CALL ZSYMV( UPLO, N, DCMPLX(-1.0D+0), A, LDA, Y(1,J), 1,
     $        DCMPLX(1.0D+0), RES, 1 )

         DO I = 1, N
            AYB( I ) = CABS1( B( I, J ) )
         END DO
*
*     Compute abs(op(A_s))*abs(Y) + abs(B_s).
*
         CALL ZLA_SYAMV ( UPLO2, N, 1.0D+0,
     $        A, LDA, Y(1, J), 1, 1.0D+0, AYB, 1 )

         CALL ZLA_LIN_BERR ( N, N, 1, RES, AYB, BERR_OUT( J ) )
*
*     End of loop for each RHS.
*
      END DO
*
      RETURN
      END
```

———-

61.4.4 lisp code

— zla_syrfsx_extended.lisp —

```
(let* ((unstable_state 0)
       (working_state 1)
       (conv_state 2)
       (noprog_state 3)
       (base_residual 0)
       (extra_residual 1)
       (extra_y 2)
```

```
      (la_linrx_err_i 2))
 (declare (type (f2cl-lib:integer4 0 0) unstable_state)
          (type (f2cl-lib:integer4 1 1) working_state)
          (type (f2cl-lib:integer4 2 2) conv_state)
          (type (f2cl-lib:integer4 3 3) noprog_state)
          (type (f2cl-lib:integer4 0 0) base_residual)
          (type (f2cl-lib:integer4 1 1) extra_residual)
          (type (f2cl-lib:integer4 2 2) extra_y)
          (type (f2cl-lib:integer4 2 2) la_linrx_err_i)
          (ignorable unstable_state working_state conv_state noprog_state
           base_residual extra_residual extra_y la_linrx_err_i))
 (defun zla_syrfsx_extended
        (prec_type uplo n nrhs a lda af ldaf ipiv colequ c b ldb$ y ldy
         berr_out n_norms err_bnds_norm err_bnds_comp res ayb dy y_tail rcond
         ithresh rthresh dz_ub ignore_cwise info)
   (declare (type (double-float) dz_ub rthresh rcond)
            (type (array double-float (*)) ayb err_bnds_comp err_bnds_norm
             berr_out c)
            (type f2cl-lib:logical ignore_cwise colequ)
            (type (array f2cl-lib:integer4 (*)) ipiv)
            (type (array f2cl-lib:complex16 (*)) y_tail dy res y b af a)
            (type (string *) uplo)
            (type (f2cl-lib:integer4) info ithresh n_norms ldy ldb$ ldaf lda
             nrhs n prec_type))
   (f2cl-lib:with-multi-array-data
       ((uplo character uplo-%data% uplo-%offset%)
        (a f2cl-lib:complex16 a-%data% a-%offset%)
        (af f2cl-lib:complex16 af-%data% af-%offset%)
        (b f2cl-lib:complex16 b-%data% b-%offset%)
        (y f2cl-lib:complex16 y-%data% y-%offset%)
        (res f2cl-lib:complex16 res-%data% res-%offset%)
        (dy f2cl-lib:complex16 dy-%data% dy-%offset%)
        (y_tail f2cl-lib:complex16 y_tail-%data% y_tail-%offset%)
        (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
        (c double-float c-%data% c-%offset%)
        (berr_out double-float berr_out-%data% berr_out-%offset%)
        (err_bnds_norm double-float err_bnds_norm-%data%
         err_bnds_norm-%offset%)
        (err_bnds_comp double-float err_bnds_comp-%data%
         err_bnds_comp-%offset%)
        (ayb double-float ayb-%data% ayb-%offset%))
     (labels ((cabs1 (zdum)
                (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
       (declare
        (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
         cabs1))
       (prog ((zdum #C(0.0d0 0.0d0)) (incr_prec nil) (upper nil) (yk 0.0d0)
              (dyk 0.0d0) (ymin 0.0d0) (normy 0.0d0) (normx 0.0d0)
              (normdx 0.0d0) (dxrat 0.0d0) (dzrat 0.0d0) (prevnormdx 0.0d0)
              (prev_dz_z 0.0d0) (dxratmax 0.0d0) (dzratmax 0.0d0) (dx_x 0.0d0)
```

```
     (dz_z 0.0d0) (final_dx_x 0.0d0) (final_dz_z 0.0d0) (eps 0.0d0)
     (hugeval 0.0d0) (incr_thresh 0.0d0) (uplo2 0) (cnt 0) (i 0)
     (j 0) (x_state 0) (z_state 0) (y_prec_state 0))
(declare (type (f2cl-lib:complex16) zdum)
         (type f2cl-lib:logical incr_prec upper)
         (type (double-float) yk dyk ymin normy normx normdx dxrat
          dzrat prevnormdx prev_dz_z dxratmax dzratmax dx_x dz_z
          final_dx_x final_dz_z eps hugeval incr_thresh)
         (type (f2cl-lib:integer4) uplo2 cnt i j x_state z_state
          y_prec_state))
(setf info 0)
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -13))
 ((< ldy (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -15)))
(cond
 ((/= info 0) (xerbla "ZLA_HERFSX_EXTENDED" (f2cl-lib:int-sub info))
  (go end_label)))
(setf eps (dlamch "Epsilon"))
(setf hugeval (dlamch "Overflow"))
(setf hugeval (* hugeval hugeval))
(setf incr_thresh (* (f2cl-lib:dble n) eps))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame uplo "L")
    (declare (ignore var-1))
    (when var-0 (setf uplo var-0))
    ret-val)
  (setf uplo2 (ilauplo "L")))
 (t (setf uplo2 (ilauplo "U"))))
```

```
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf y_prec_state extra_residual)
    (cond
     ((= y_prec_state extra_y)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                               y_tail-%offset%)
                (coerce 0.0d0 'f2cl-lib:complex16))
         label100001))))
    (setf dxrat 0.0d0)
    (setf dxratmax 0.0d0)
    (setf dzrat 0.0d0)
    (setf dzratmax 0.0d0)
    (setf final_dx_x hugeval)
    (setf final_dz_z hugeval)
    (setf prevnormdx hugeval)
    (setf prev_dz_z hugeval)
    (setf dz_z hugeval)
    (setf dx_x hugeval)
    (setf x_state working_state)
    (setf z_state unstable_state)
    (setf incr_prec f2cl-lib:%false%)
    (f2cl-lib:fdo (cnt 1 (f2cl-lib:int-add cnt 1))
                  ((> cnt ithresh) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (zcopy n
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 j)
                                   ((1 ldb$) (1 *)) b-%offset%)
             1 res 1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf n var-0)))
        (cond
         ((= y_prec_state base_residual)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9)
              (zsymv uplo n (f2cl-lib:dcmplx -1.0d0) a lda
               (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                                     (1 j) ((1 ldy) (1 *))
                                     y-%offset%)
               1 (f2cl-lib:dcmplx 1.0d0) res 1)
            (declare
             (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
            (when var-0 (setf uplo var-0))
            (when var-1 (setf n var-1))
```

```
    (when var-4 (setf lda var-4))))
 ((= y_prec_state extra_residual)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (blas_zsymv_x uplo2 n (f2cl-lib:dcmplx -1.0d0) a lda
       (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                             (1 j) ((1 ldy) (1 *))
                             y-%offset%)
       1 (f2cl-lib:dcmplx 1.0d0) res 1 prec_type)
    (declare
     (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
    (when var-0 (setf uplo2 var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf lda var-4))
    (when var-10 (setf prec_type var-10))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11)
      (blas_zsymv2_x uplo2 n (f2cl-lib:dcmplx -1.0d0) a lda
       (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                             (1 j) ((1 ldy) (1 *))
                             y-%offset%)
       y_tail 1 (f2cl-lib:dcmplx 1.0d0) res 1 prec_type)
    (declare
     (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9
      var-10))
    (when var-0 (setf uplo2 var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf lda var-4))
    (when var-11 (setf prec_type var-11)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (zcopy n res 1 dy 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zsytrs uplo n 1 af ldaf ipiv dy n info)
  (declare (ignore var-2 var-3 var-5 var-6))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-4 (setf ldaf var-4))
  (when var-7 (setf n var-7))
  (when var-8 (setf info var-8)))
(setf normx 0.0d0)
(setf normy 0.0d0)
(setf normdx 0.0d0)
(setf dz_z 0.0d0)
(setf ymin hugeval)
```

```
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf yk
            (cabs1
             (f2cl-lib:fref y-%data% (i j) ((1 ldy) (1 *))
                            y-%offset%)))
    (setf dyk
            (cabs1
             (f2cl-lib:fref dy-%data% (i) ((1 *))
                            dy-%offset%)))
    (cond ((/= yk 0.0d0) (setf dz_z (max dz_z (/ dyk yk))))
          ((/= dyk 0.0d0) (setf dz_z hugeval)))
    (setf ymin (min ymin yk))
    (setf normy (max normy yk))
    (cond
     (colequ
      (setf normx
              (max normx
                   (* yk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%))))
      (setf normdx
              (max normdx
                   (* dyk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%)))))
     (t (setf normx normy) (setf normdx (max normdx dyk))))
   label100003))
(cond ((/= normx 0.0d0) (setf dx_x (/ normdx normx)))
      ((= normdx 0.0d0) (setf dx_x 0.0d0))
      (t (setf dx_x hugeval)))
(setf dxrat (/ normdx prevnormdx))
(setf dzrat (/ dz_z prev_dz_z))
(if (and (< (* ymin rcond) (* incr_thresh normy))
         (< y_prec_state extra_y))
    (setf incr_prec f2cl-lib:%true%))
(if (and (= x_state noprog_state) (<= dxrat rthresh))
    (setf x_state working_state))
(cond
 ((= x_state working_state)
  (cond ((<= dx_x eps) (setf x_state conv_state))
        ((> dxrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf x_state noprog_state))))
        (t
         (if (> dxrat dxratmax)
             (setf dxratmax dxrat))))
```

```
  (if (> x_state working_state)
      (setf final_dx_x dx_x))))
(if (and (= z_state unstable_state) (<= dz_z dz_ub))
    (setf z_state working_state))
(if (and (= z_state noprog_state) (<= dzrat rthresh))
    (setf z_state working_state))
(cond
 ((= z_state working_state)
  (cond ((<= dz_z eps) (setf z_state conv_state))
        ((> dz_z dz_ub) (setf z_state unstable_state)
         (setf dzratmax 0.0d0) (setf final_dz_z hugeval))
        ((> dzrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf z_state noprog_state))))
        (t
         (if (> dzrat dzratmax)
             (setf dzratmax dzrat))))
  (if (> z_state working_state)
      (setf final_dz_z dz_z))))
(if (and (/= x_state working_state)
         (or ignore_cwise (/= z_state working_state)))
    (go label666))
(cond
 (incr_prec (setf incr_prec f2cl-lib:%false%)
  (setf y_prec_state (f2cl-lib:int-add y_prec_state 1))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                           y_tail-%offset%)
              (coerce 0.0d0 'f2cl-lib:complex16))
     label100004))))
(setf prevnormdx normdx)
(setf prev_dz_z dz_z)
(cond
 ((< y_prec_state extra_y)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (zaxpy n (f2cl-lib:dcmplx 1.0d0) dy 1
       (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                             (1 j) ((1 ldy) (1 *))
                             y-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (zla_wwaddw n
       (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
```

```
                                    (1 j) ((1 ldy) (1 *))
                                    y-%offset%)
            y_tail dy)
         (declare (ignore var-1 var-2 var-3))
         (when var-0 (setf n var-0)))))
    label100002))
label666
 (if (= x_state working_state)
     (setf final_dx_x dx_x))
 (if (= z_state working_state)
     (setf final_dz_z dz_z))
 (cond
  ((>= n_norms 1)
   (setf (f2cl-lib:fref err_bnds_norm-%data% (j la_linrx_err_i)
                        ((1 nrhs) (1 *)) err_bnds_norm-%offset%)
          (/ final_dx_x (- 1 dxratmax)))))
 (cond
  ((>= n_norms 2)
   (setf (f2cl-lib:fref err_bnds_comp-%data% (j la_linrx_err_i)
                        ((1 nrhs) (1 *)) err_bnds_comp-%offset%)
          (/ final_dz_z (- 1 dzratmax)))))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (zcopy n
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 j)
                            ((1 ldb$) (1 *)) b-%offset%)
      1 res 1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf n var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
     (zsymv uplo n (f2cl-lib:dcmplx -1.0d0) a lda
      (f2cl-lib:array-slice y-%data% f2cl-lib:complex16 (1 j)
                            ((1 ldy) (1 *)) y-%offset%)
      1 (f2cl-lib:dcmplx 1.0d0) res 1)
   (declare (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
   (when var-0 (setf uplo var-0))
   (when var-1 (setf n var-1))
   (when var-4 (setf lda var-4)))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf (f2cl-lib:fref ayb-%data% (i) ((1 *)) ayb-%offset%)
             (cabs1
              (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)))
    label100005))
 (zla_syamv uplo2 n 1.0d0 a lda
  (f2cl-lib:array-slice y-%data% f2cl-lib:complex16 (1 j)
                        ((1 ldy) (1 *)) y-%offset%)
  1 1.0d0 ayb 1)
```

```
              (zla_lin_berr n n 1 res ayb
               (f2cl-lib:array-slice berr_out-%data% double-float (j) ((1 *))
                                     berr_out-%offset%))
            label100000))
          (go end_label)
         end_label
          (return
           (values prec_type uplo n nil nil lda nil ldaf nil nil nil nil nil
                   nil nil nil nil nil nil nil nil nil nil nil nil nil nil nil
                   info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zla_syrfsx_extended
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              fortran-to-lisp::logical
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (double-float)
                                              (fortran-to-lisp::integer4)
```

```
  (double-float) (double-float)
  fortran-to-lisp::logical
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::prec_type
  fortran-to-lisp::uplo
  fortran-to-lisp::n nil nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldaf nil nil nil
  nil nil nil nil nil nil nil nil
  nil nil nil nil nil nil nil nil
  nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zla_lin_berr
  fortran-to-lisp::zla_syamv
  fortran-to-lisp::ilauplo
  fortran-to-lisp::xerbla))))
```

———-

61.5 zla_syrpvgrw

61.5.1 Axiom unit tests

— **zlasyrpvgrw.input** —

```
)set break resume
)sys rm -f zlasyrpvgrw.output
)spool zlasyrpvgrw.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.5.2 Axiom help page

— **zla_syrpvgrw.help** —

```
==================================================================
```

```
zla_syrpvgrw examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function zla_syrpvgrw (UPLO, N, INFO, A, LDA, AF,
           LDAF, IPIV, WORK)
           ZLA_SYRPVGRW computes the reciprocal pivot growth factor
           norm(A)/norm(U) for a symmetric indefinite matrix.

Function/Subroutine Documentation
   double precision function zla_syrpvgrw (character*1 UPLO, integer N,
       integer INFO, complex*16, dimension( lda, * ) A, integer LDA,
       complex*16, dimension( ldaf, * ) AF, integer LDAF, integer, dimension(
       * ) IPIV, double precision, dimension( * ) WORK)
       ZLA_SYRPVGRW computes the reciprocal pivot growth factor
       norm(A)/norm(U) for a symmetric indefinite matrix.

Purpose:
   ZLA_SYRPVGRW computes the reciprocal pivot growth factor
   norm(A)/norm(U). The "max absolute element" norm is used. If this is
   much less than 1, the stability of the LU factorization of the
   (equilibrated) matrix A could be poor. This also means that the
   solution X, estimated condition numbers, and error bounds could be
   unreliable.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 INFO
   INFO is INTEGER
   The value of INFO returned from ZSYTRF, .i.e., the pivot in
   column INFO is exactly 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the N-by-N matrix A.
```

```
 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is COMPLEX*16 array, dimension (LDAF,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by ZSYTRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by ZSYTRF.

 WORK
   WORK is DOUBLE PRECISION array, dimension (2*N)
```

61.5.3 fortran code

— zla_syrpvgrw.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION ZLA_SYRPVGRW( UPLO, N, INFO, A, LDA, AF,
     $                                        LDAF, IPIV, WORK )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER*1        UPLO
      INTEGER            N, INFO, LDA, LDAF
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), AF( LDAF, * )
      DOUBLE PRECISION   WORK( * )
      INTEGER            IPIV( * )
*     ..
*
```

```
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            NCOLS, I, J, K, KP
      DOUBLE PRECISION   AMAX, UMAX, RPVGRW, TMP
      LOGICAL            UPPER
      COMPLEX*16         ZDUM
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, REAL, DIMAG, MAX, MIN
*     ..
*     .. External Subroutines ..
      EXTERNAL           LSAME, ZLASET
      LOGICAL            LSAME
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( DBLE ( ZDUM ) ) + ABS( DIMAG ( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      UPPER = LSAME( 'Upper', UPLO )
      IF ( INFO.EQ.0 ) THEN
         IF ( UPPER ) THEN
            NCOLS = 1
         ELSE
            NCOLS = N
         END IF
      ELSE
         NCOLS = INFO
      END IF

      RPVGRW = 1.0D+0
      DO I = 1, 2*N
         WORK( I ) = 0.0D+0
      END DO
*
*     Find the max magnitude entry of each column of A.  Compute the max
*     for all N columns so we can apply the pivot permutation while
*     looping below.  Assume a full factorization is the common case.
*
      IF ( UPPER ) THEN
         DO J = 1, N
            DO I = 1, J
               WORK( N+I ) = MAX( CABS1( A( I, J ) ), WORK( N+I ) )
               WORK( N+J ) = MAX( CABS1( A( I, J ) ), WORK( N+J ) )
            END DO
         END DO
```

```
      ELSE
         DO J = 1, N
            DO I = J, N
               WORK( N+I ) = MAX( CABS1( A( I, J ) ), WORK( N+I ) )
               WORK( N+J ) = MAX( CABS1( A( I, J ) ), WORK( N+J ) )
            END DO
         END DO
      END IF
*
*     Now find the max magnitude entry of each column of U or L.  Also
*     permute the magnitudes of A above so they're in the same order as
*     the factor.
*
*     The iteration orders and permutations were copied from zsytrs.
*     Calls to SSWAP would be severe overkill.
*
      IF ( UPPER ) THEN
         K = N
         DO WHILE ( K .LT. NCOLS .AND. K.GT.0 )
            IF ( IPIV( K ).GT.0 ) THEN
!              1x1 pivot
               KP = IPIV( K )
               IF ( KP .NE. K ) THEN
                  TMP = WORK( N+K )
                  WORK( N+K ) = WORK( N+KP )
                  WORK( N+KP ) = TMP
               END IF
               DO I = 1, K
                  WORK( K ) = MAX( CABS1( AF( I, K ) ), WORK( K ) )
               END DO
               K = K - 1
            ELSE
!              2x2 pivot
               KP = -IPIV( K )
               TMP = WORK( N+K-1 )
               WORK( N+K-1 ) = WORK( N+KP )
               WORK( N+KP ) = TMP
               DO I = 1, K-1
                  WORK( K ) = MAX( CABS1( AF( I, K ) ), WORK( K ) )
                  WORK( K-1 ) =
     $                 MAX( CABS1( AF( I, K-1 ) ), WORK( K-1 ) )
               END DO
               WORK( K ) = MAX( CABS1( AF( K, K ) ), WORK( K ) )
               K = K - 2
            END IF
         END DO
         K = NCOLS
         DO WHILE ( K .LE. N )
            IF ( IPIV( K ).GT.0 ) THEN
               KP = IPIV( K )
```

```
               IF ( KP .NE. K ) THEN
                  TMP = WORK( N+K )
                  WORK( N+K ) = WORK( N+KP )
                  WORK( N+KP ) = TMP
               END IF
               K = K + 1
            ELSE
               KP = -IPIV( K )
               TMP = WORK( N+K )
               WORK( N+K ) = WORK( N+KP )
               WORK( N+KP ) = TMP
               K = K + 2
            END IF
         END DO
      ELSE
         K = 1
         DO WHILE ( K .LE. NCOLS )
            IF ( IPIV( K ).GT.0 ) THEN
!              1x1 pivot
               KP = IPIV( K )
               IF ( KP .NE. K ) THEN
                  TMP = WORK( N+K )
                  WORK( N+K ) = WORK( N+KP )
                  WORK( N+KP ) = TMP
               END IF
               DO I = K, N
                  WORK( K ) = MAX( CABS1( AF( I, K ) ), WORK( K ) )
               END DO
               K = K + 1
            ELSE
!              2x2 pivot
               KP = -IPIV( K )
               TMP = WORK( N+K+1 )
               WORK( N+K+1 ) = WORK( N+KP )
               WORK( N+KP ) = TMP
               DO I = K+1, N
                  WORK( K ) = MAX( CABS1( AF( I, K ) ), WORK( K ) )
                  WORK( K+1 ) =
     $                 MAX( CABS1( AF( I, K+1 ) ), WORK( K+1 ) )
               END DO
               WORK( K ) = MAX( CABS1( AF( K, K ) ), WORK( K ) )
               K = K + 2
            END IF
         END DO
         K = NCOLS
         DO WHILE ( K .GE. 1 )
            IF ( IPIV( K ).GT.0 ) THEN
               KP = IPIV( K )
               IF ( KP .NE. K ) THEN
                  TMP = WORK( N+K )
```

```
                WORK( N+K ) = WORK( N+KP )
                WORK( N+KP ) = TMP
             END IF
             K = K - 1
          ELSE
             KP = -IPIV( K )
             TMP = WORK( N+K )
             WORK( N+K ) = WORK( N+KP )
             WORK( N+KP ) = TMP
             K = K - 2
          ENDIF
       END DO
    END IF
*
*     Compute the *inverse* of the max element growth factor.  Dividing
*     by zero would imply the largest entry of the factor's column is
*     zero.  Than can happen when either the column of A is zero or
*     massive pivots made the factor underflow to zero.  Neither counts
*     as growth in itself, so simply ignore terms with zero
*     denominators.
*
    IF ( UPPER ) THEN
       DO I = NCOLS, N
          UMAX = WORK( I )
          AMAX = WORK( N+I )
          IF ( UMAX /= 0.0D+0 ) THEN
             RPVGRW = MIN( AMAX / UMAX, RPVGRW )
          END IF
       END DO
    ELSE
       DO I = 1, NCOLS
          UMAX = WORK( I )
          AMAX = WORK( N+I )
          IF ( UMAX /= 0.0D+0 ) THEN
             RPVGRW = MIN( AMAX / UMAX, RPVGRW )
          END IF
       END DO
    END IF

    ZLA_SYRPVGRW = RPVGRW
    END
```

———-

61.5.4 lisp code

— zla_syrpvgrw.lisp —

```
(defun zla_syrpvgrw (uplo n info a lda af ldaf ipiv work)
  (declare (type (array double-float (*)) work)
           (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex16 (*)) af a)
           (type (f2cl-lib:integer4) ldaf lda info n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (af f2cl-lib:complex16 af-%data% af-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (work double-float work-%data% work-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
        cabs1))
      (prog ((zdum #C(0.0d0 0.0d0)) (upper nil) (amax 0.0d0) (umax 0.0d0)
             (rpvgrw 0.0d0) (tmp 0.0d0) (ncols 0) (i 0) (j 0) (k 0) (kp 0)
             (zla_syrpvgrw 0.0d0))
        (declare (type (f2cl-lib:integer4) kp k j i ncols)
                 (type (double-float) zla_syrpvgrw tmp rpvgrw umax amax)
                 (type f2cl-lib:logical upper)
                 (type (f2cl-lib:complex16) zdum))
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame "Upper" uplo)
                  (declare (ignore var-0))
                  (when var-1 (setf uplo var-1))
                  ret-val))
        (cond ((= info 0) (cond (upper (setf ncols 1)) (t (setf ncols n))))
              (t (setf ncols info)))
        (setf rpvgrw 1.0d0)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-mul 2 n)) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) 0.0d0)
           label100000))
        (cond
         (upper
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i j) nil)
                (tagbody
                  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                       ((1 *)) work-%offset%)
                        (max
                         (cabs1
```

```
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))
                   (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                  ((1 *)) work-%offset%)))
            (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j))
                                 ((1 *)) work-%offset%)
                  (max
                   (cabs1
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))
                   (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j))
                                  ((1 *)) work-%offset%)))
          label100002))
      label100001)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                ((1 *)) work-%offset%)
                 (max
                  (cabs1
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%))
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                 ((1 *)) work-%offset%)))
           (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j))
                                ((1 *)) work-%offset%)
                 (max
                  (cabs1
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%))
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n j))
                                 ((1 *)) work-%offset%)))
           label100004))
       label100003))))
 (cond
  (upper
   (tagbody
     (setf k n)
    label100005
     (if (not (and (< k ncols) (> k 0)))
         (go label100006))
     (cond
      ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
       (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
       (cond
```

```
 ((/= kp k)
  (setf tmp
          (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                         ((1 *)) work-%offset%))
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                       ((1 *)) work-%offset%)
          (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                         ((1 *)) work-%offset%))
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                       ((1 *)) work-%offset%)
          tmp)))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i k) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
            (max
             (cabs1
              (f2cl-lib:fref af-%data% (i k) ((1 ldaf) (1 *))
                             af-%offset%))
             (f2cl-lib:fref work-%data% (k) ((1 *))
                            work-%offset%)))
   label100007))
 (setf k (f2cl-lib:int-sub k 1)))
(t
 (setf kp
        (f2cl-lib:int-sub
         (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
 (setf tmp
        (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-sub (f2cl-lib:int-add n k)
                                          1))
                       ((1 *)) work-%offset%))
 (setf (f2cl-lib:fref work-%data%
                      ((f2cl-lib:int-sub (f2cl-lib:int-add n k)
                                         1))
                      ((1 *)) work-%offset%)
        (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                       ((1 *)) work-%offset%))
 (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                      ((1 *)) work-%offset%)
        tmp)
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
            (max
             (cabs1
              (f2cl-lib:fref af-%data% (i k) ((1 ldaf) (1 *))
                             af-%offset%))
```

```
                    (f2cl-lib:fref work-%data% (k) ((1 *))
                                   work-%offset%)))
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1))
                               ((1 *)) work-%offset%)
                 (max
                  (cabs1
                   (f2cl-lib:fref af-%data% (i (f2cl-lib:int-sub k 1))
                                  ((1 ldaf) (1 *)) af-%offset%))
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1))
                                 ((1 *)) work-%offset%)))
        label100008))
    (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
            (max
             (cabs1
              (f2cl-lib:fref af-%data% (k k) ((1 ldaf) (1 *))
                             af-%offset%))
             (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)))
    (setf k (f2cl-lib:int-sub k 2))))
  (go label100005)
 label100006
  (setf k ncols)
 label100009
  (if (not (<= k n))
      (go label100010))
  (cond
   ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
    (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
    (cond
     ((/= kp k)
      (setf tmp
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                             ((1 *)) work-%offset%))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                           ((1 *)) work-%offset%)
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                             ((1 *)) work-%offset%))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                           ((1 *)) work-%offset%)
              tmp)))
    (setf k (f2cl-lib:int-add k 1)))
   (t
    (setf kp
            (f2cl-lib:int-sub
             (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
    (setf tmp
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                           ((1 *)) work-%offset%))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                         work-%offset%)
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
```

```
                                ((1 *)) work-%offset%))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                              ((1 *)) work-%offset%)
                tmp)
        (setf k (f2cl-lib:int-add k 2))))
      (go label100009)
     label100010))
    (t
     (tagbody
       (setf k 1)
      label100011
       (if (not (<= k ncols))
           (go label100012))
       (cond
        ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
         (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
         (cond
          ((/= kp k)
           (setf tmp
                   (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                                  ((1 *)) work-%offset%))
           (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                                ((1 *)) work-%offset%)
                   (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                                  ((1 *)) work-%offset%))
           (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                                ((1 *)) work-%offset%)
                   tmp)))
         (f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                     (max
                      (cabs1
                       (f2cl-lib:fref af-%data% (i k) ((1 ldaf) (1 *))
                                      af-%offset%))
                      (f2cl-lib:fref work-%data% (k) ((1 *))
                                     work-%offset%)))
            label100013))
         (setf k (f2cl-lib:int-add k 1)))
        (t
         (setf kp
                 (f2cl-lib:int-sub
                  (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
         (setf tmp
                 (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k 1))
                                ((1 *)) work-%offset%))
         (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k 1))
                              ((1 *)) work-%offset%)
                 (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
```

```
                        ((1 *)) work-%offset%))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                       ((1 *)) work-%offset%)
            tmp)
    (f2cl-lib:fdo (i (f2cl-lib:int-add k 1) (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                (max
                 (cabs1
                  (f2cl-lib:fref af-%data% (i k) ((1 ldaf) (1 *))
                                 af-%offset%))
                 (f2cl-lib:fref work-%data% (k) ((1 *))
                                work-%offset%)))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1))
                             ((1 *)) work-%offset%)
                (max
                 (cabs1
                  (f2cl-lib:fref af-%data% (i (f2cl-lib:int-add k 1))
                                 ((1 ldaf) (1 *)) af-%offset%))
                 (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1))
                                ((1 *)) work-%offset%)))
       label100014))
    (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
            (max
             (cabs1
              (f2cl-lib:fref af-%data% (k k) ((1 ldaf) (1 *))
                             af-%offset%))
             (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)))
    (setf k (f2cl-lib:int-add k 2))))
  (go label100011)
 label100012
  (setf k ncols)
 label100015
  (if (not (>= k 1))
      (go label100016))
  (cond
   ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
    (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
    (cond
     ((/= kp k)
      (setf tmp
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                             ((1 *)) work-%offset%))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                           ((1 *)) work-%offset%)
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                             ((1 *)) work-%offset%))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                           ((1 *)) work-%offset%)
```

```
                    tmp)))
            (setf k (f2cl-lib:int-sub k 1)))
           (t
            (setf kp
                    (f2cl-lib:int-sub
                     (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
            (setf tmp
                    (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k))
                                   ((1 *)) work-%offset%))
            (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n k)) ((1 *))
                                 work-%offset%)
                    (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                                   ((1 *)) work-%offset%))
            (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n kp))
                                 ((1 *)) work-%offset%)
                    tmp)
            (setf k (f2cl-lib:int-sub k 2))))
          (go label100015)
         label100016)))
       (cond
        (upper
         (f2cl-lib:fdo (i ncols (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
            (setf umax (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
            (setf amax
                    (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                   ((1 *)) work-%offset%))
            (cond
             ((f2cl-lib:f2cl/ umax (= 0.0d0))
              (setf rpvgrw (min (/ amax umax) rpvgrw))))
           label100017)))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i ncols) nil)
           (tagbody
            (setf umax (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
            (setf amax
                    (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                   ((1 *)) work-%offset%))
            (cond
             ((f2cl-lib:f2cl/ umax (= 0.0d0))
              (setf rpvgrw (min (/ amax umax) rpvgrw))))
           label100018))))
       (setf zla_syrpvgrw rpvgrw)
      end_label
       (return (values zla_syrpvgrw uplo nil nil nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
```

```
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zla_syrpvgrw
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*)))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil nil
                                              nil nil nil nil nil)
                                            :calls 'nil)))
```

———-

61.6 zlasyf

61.6.1 Axiom unit tests

— **zlasyf.input** —

```
)set break resume
)sys rm -f zlasyf.output
)spool zlasyf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.6.2 Axiom help page

— zlasyf.help —

```
====================================================================
zlasyf examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zlasyf (UPLO, N, NB, KB, A, LDA, IPIV, W, LDW, INFO)
           ZLASYF computes a partial factorization of a complex symmetric
           matrix using the Bunch-Kaufman diagonal pivoting method.

Function/Subroutine Documentation
   subroutine zlasyf (character UPLO, integer N, integer NB, integer KB,
       complex*16, dimension( lda, * ) A, integer LDA, integer, dimension( * )
       IPIV, complex*16, dimension( ldw, * ) W, integer LDW, integer INFO)
       ZLASYF computes a partial factorization of a complex symmetric matrix
       using the Bunch-Kaufman diagonal pivoting method.

Purpose:
   ZLASYF computes a partial factorization of a complex symmetric matrix
   A using the Bunch-Kaufman diagonal pivoting method. The partial
   factorization has the form:

       A  =  ( I  U12 ) ( A11  0  ) (  I       0    )  if UPLO = 'U', or:
             ( 0  U22 ) (  0   D  ) ( U12**T U22**T )

       A  =  ( L11  0 ) ( D    0  ) ( L11**T L21**T )  if UPLO = 'L'
             ( L21  I ) ( 0   A22 ) (  0       I    )

   where the order of D is at most NB. The actual order is returned in
   the argument KB, and is either NB or NB-1, or N if N <= NB.
   Note that U**T denotes the transpose of U.

   ZLASYF is an auxiliary routine called by ZSYTRF. It uses blocked code
   (calling Level 3 BLAS) to update the submatrix A11 (if UPLO = 'U') or
   A22 (if UPLO = 'L').

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
```

```
  symmetric matrix A is stored:
  = 'U':  Upper triangular
  = 'L':  Lower triangular

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

NB
  NB is INTEGER
  The maximum number of columns of the matrix A that should be
  factored.  NB should be at least 2 to allow for 2-by-2 pivot blocks.

KB
  KB is INTEGER
  The number of columns of A that were actually factored.
  KB is either NB-1 or NB, or N if N <= NB.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  n-by-n upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading n-by-n lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.
  On exit, A contains details of the partial factorization.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D.

  If UPLO = 'U':
    Only the last KB elements of IPIV are set.

    If IPIV(k) > 0, then rows and columns k and IPIV(k) were
    interchanged and D(k,k) is a 1-by-1 diagonal block.

    If IPIV(k) = IPIV(k-1) < 0, then rows and columns
    k-1 and -IPIV(k) were interchanged and D(k-1:k,k-1:k)
    is a 2-by-2 diagonal block.

  If UPLO = 'L':
    Only the first KB elements of IPIV are set.
```

```
     If IPIV(k) > 0, then rows and columns k and IPIV(k) were
     interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) = IPIV(k+1) < 0, then rows and columns
     k+1 and -IPIV(k) were interchanged and D(k:k+1,k:k+1)
     is a 2-by-2 diagonal block.

 W
   W is COMPLEX*16 array, dimension (LDW,NB)

 LDW
   LDW is INTEGER
   The leading dimension of the array W.  LDW >= max(1,N).

 INFO
   INFO is INTEGER
   = 0: successful exit
   > 0: if INFO = k, D(k,k) is exactly zero.  The factorization
        has been completed, but the block diagonal matrix D is
        exactly singular.
```

61.6.3 fortran code

— zlasyf.f —

```
*  =====================================================================
      SUBROUTINE ZLASYF( UPLO, N, NB, KB, A, LDA, IPIV, W, LDW, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, KB, LDA, LDW, N, NB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), W( LDW, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
```

```
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   EIGHT, SEVTEN
      PARAMETER          ( EIGHT = 8.0D+0, SEVTEN = 17.0D+0 )
      COMPLEX*16         CONE
      PARAMETER          ( CONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            IMAX, J, JB, JJ, JMAX, JP, K, KK, KKW, KP,
     $                   KSTEP, KW
      DOUBLE PRECISION   ABSAKK, ALPHA, COLMAX, ROWMAX
      COMPLEX*16         D11, D21, D22, R1, T, Z
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IZAMAX
      EXTERNAL           LSAME, IZAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZCOPY, ZGEMM, ZGEMV, ZSCAL, ZSWAP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, MAX, MIN, SQRT
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( Z ) = ABS( DBLE( Z ) ) + ABS( DIMAG( Z ) )
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
*     Initialize ALPHA for use in choosing pivot block size.
*
      ALPHA = ( ONE+SQRT( SEVTEN ) ) / EIGHT
*
      IF( LSAME( UPLO, 'U' ) ) THEN
*
*        Factorize the trailing columns of A using the upper triangle
*        of A and working backwards, and compute the matrix W = U12*D
*        for use in updating A11
*
*        K is the main loop index, decreasing from N in steps of 1 or 2
*
*        KW is the column of W which corresponds to column K of A
*
         K = N
   10    CONTINUE
```

```
         KW = NB + K - N
*
*        Exit from loop
*
         IF( ( K.LE.N-NB+1 .AND. NB.LT.N ) .OR. K.LT.1 )
     $      GO TO 30
*
*        Copy column K of A to column KW of W and update it
*
         CALL ZCOPY( K, A( 1, K ), 1, W( 1, KW ), 1 )
         IF( K.LT.N )
     $      CALL ZGEMV( 'No transpose', K, N-K, -CONE, A( 1, K+1 ), LDA,
     $                  W( K, KW+1 ), LDW, CONE, W( 1, KW ), 1 )
*
         KSTEP = 1
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = CABS1( W( K, KW ) )
*
*        IMAX is the row-index of the largest off-diagonal element in

*
         IF( K.GT.1 ) THEN
            IMAX = IZAMAX( K-1, W( 1, KW ), 1 )
            COLMAX = CABS1( W( IMAX, KW ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( MAX( ABSAKK, COLMAX ).EQ.ZERO ) THEN
*
*           Column K is zero or underflow: set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
         ELSE
            IF( ABSAKK.GE.ALPHA*COLMAX ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
            ELSE
*
*              Copy column IMAX to column KW-1 of W and update it
*
               CALL ZCOPY( IMAX, A( 1, IMAX ), 1, W( 1, KW-1 ), 1 )
               CALL ZCOPY( K-IMAX, A( IMAX, IMAX+1 ), LDA,
```

```
     $                    W( IMAX+1, KW-1 ), 1 )
               IF( K.LT.N )
     $            CALL ZGEMV( 'No transpose', K, N-K, -CONE,
     $                        A( 1, K+1 ), LDA, W( IMAX, KW+1 ), LDW,
     $                        CONE, W( 1, KW-1 ), 1 )
*
*              JMAX is the column-index of the largest off-diagonal
*              element in row IMAX, and ROWMAX is its absolute value
*
               JMAX = IMAX + IZAMAX( K-IMAX, W( IMAX+1, KW-1 ), 1 )
               ROWMAX = CABS1( W( JMAX, KW-1 ) )
               IF( IMAX.GT.1 ) THEN
                  JMAX = IZAMAX( IMAX-1, W( 1, KW-1 ), 1 )
                  ROWMAX = MAX( ROWMAX, CABS1( W( JMAX, KW-1 ) ) )
               END IF
*
               IF( ABSAKK.GE.ALPHA*COLMAX*( COLMAX / ROWMAX ) ) THEN
*
*                 no interchange, use 1-by-1 pivot block
*
                  KP = K
               ELSE IF( CABS1( W( IMAX, KW-1 ) ).GE.ALPHA*ROWMAX ) THEN
*
*                 interchange rows and columns K and IMAX, use 1-by-1
*                 pivot block
*
                  KP = IMAX
*
*                 copy column KW-1 of W to column KW of W
*
                  CALL ZCOPY( K, W( 1, KW-1 ), 1, W( 1, KW ), 1 )
               ELSE
*
*                 interchange rows and columns K-1 and IMAX, use 2-by-2
*                 pivot block
*
                  KP = IMAX
                  KSTEP = 2
               END IF
            END IF
*
*           ============================================================
*
*           KK is the column of A where pivoting step stopped
*
            KK = K - KSTEP + 1
*
*           KKW is the column of W which corresponds to column KK of A
*
            KKW = NB + KK - N
```

```
*
*          Interchange rows and columns KP and KK.
*          Updated column KP is already stored in column KKW of W.
*
           IF( KP.NE.KK ) THEN
*
*             Copy non-updated column KK to column KP of submatrix A
*             at step K. No need to copy element into column K
*             (or K and K-1 for 2-by-2 pivot) of A, since these columns
*             will be later overwritten.
*
              A( KP, KP ) = A( KK, KK )
              CALL ZCOPY( KK-1-KP, A( KP+1, KK ), 1, A( KP, KP+1 ),
     $                    LDA )
              IF( KP.GT.1 )
     $           CALL ZCOPY( KP-1, A( 1, KK ), 1, A( 1, KP ), 1 )
*
*             Interchange rows KK and KP in last K+1 to N columns of A
*             (columns K (or K and K-1 for 2-by-2 pivot) of A will be
*             later overwritten). Interchange rows KK and KP
*             in last KKW to NB columns of W.
*
              IF( K.LT.N )
     $           CALL ZSWAP( N-K, A( KK, K+1 ), LDA, A( KP, K+1 ),
     $                       LDA )
              CALL ZSWAP( N-KK+1, W( KK, KKW ), LDW, W( KP, KKW ),
     $                    LDW )
           END IF
*
           IF( KSTEP.EQ.1 ) THEN
*
*             1-by-1 pivot block D(k): column kw of W now holds
*
*             W(kw) = U(k)*D(k),
*
*             where U(k) is the k-th column of U
*
*             Store subdiag. elements of column U(k)
*             and 1-by-1 block D(k) in column k of A.
*             NOTE: Diagonal element U(k,k) is a UNIT element
*             and not stored.
*                A(k,k) := D(k,k) = W(k,kw)
*                A(1:k-1,k) := U(1:k-1,k) = W(1:k-1,kw)/D(k,k)
*
              CALL ZCOPY( K, W( 1, KW ), 1, A( 1, K ), 1 )
              R1 = CONE / A( K, K )
              CALL ZSCAL( K-1, R1, A( 1, K ), 1 )
*
           ELSE
*
```

```
*              2-by-2 pivot block D(k): columns kw and kw-1 of W now hold
*
*              ( W(kw-1) W(kw) ) = ( U(k-1) U(k) )*D(k)
*
*              where U(k) and U(k-1) are the k-th and (k-1)-th columns
*              of U
*
*              Store U(1:k-2,k-1) and U(1:k-2,k) and 2-by-2
*              block D(k-1:k,k-1:k) in columns k-1 and k of A.
*              NOTE: 2-by-2 diagonal block U(k-1:k,k-1:k) is a UNIT
*              block and not stored.
*                 A(k-1:k,k-1:k) := D(k-1:k,k-1:k) = W(k-1:k,kw-1:kw)
*                 A(1:k-2,k-1:k) := U(1:k-2,k:k-1:k) =
*                 = W(1:k-2,kw-1:kw) * ( D(k-1:k,k-1:k)**(-1) )
*
               IF( K.GT.2 ) THEN
*
*                 Compose the columns of the inverse of 2-by-2 pivot
*                 block D in the following way to reduce the number
*                 of FLOPS when we myltiply panel ( W(kw-1) W(kw) ) by
*                 this inverse
*
*                 D**(-1) = ( d11 d21 )**(-1) =
*                           ( d21 d22 )
*
*                 = 1/(d11*d22-d21**2) * ( ( d22 ) (-d21 ) ) =
*                                        ( (-d21 ) ( d11 ) )
*
*                 = 1/d21 * 1/((d11/d21)*(d22/d21)-1) *
*
*                   * ( ( d22/d21 ) (      -1 ) ) =
*                     ( (      -1 ) ( d11/d21 ) )
*
*                 = 1/d21 * 1/(D22*D11-1) * ( ( D11 ) (  -1 ) ) =
*                                           ( ( -1  ) ( D22 ) )
*
*                 = 1/d21 * T * ( ( D11 ) (  -1 ) )
*                               ( (  -1 ) ( D22 ) )
*
*                 = D21 * ( ( D11 ) (  -1 ) )
*                         ( (  -1 ) ( D22 ) )
*
                  D21 = W( K-1, KW )
                  D11 = W( K, KW ) / D21
                  D22 = W( K-1, KW-1 ) / D21
                  T = CONE / ( D11*D22-CONE )
                  D21 = T / D21
*
*                 Update elements in columns A(k-1) and A(k) as
*                 dot products of rows of ( W(kw-1) W(kw) ) and columns
```

```
*                of D**(-1)
*
                 DO 20 J = 1, K - 2
                    A( J, K-1 ) = D21*( D11*W( J, KW-1 )-W( J, KW ) )
                    A( J, K ) = D21*( D22*W( J, KW )-W( J, KW-1 ) )
   20            CONTINUE
              END IF
*
*             Copy D(k) to A
*
              A( K-1, K-1 ) = W( K-1, KW-1 )
              A( K-1, K ) = W( K-1, KW )
              A( K, K ) = W( K, KW )
*
           END IF
*
        END IF
*
*       Store details of the interchanges in IPIV
*
        IF( KSTEP.EQ.1 ) THEN
           IPIV( K ) = KP
        ELSE
           IPIV( K ) = -KP
           IPIV( K-1 ) = -KP
        END IF
*
*       Decrease K and return to the start of the main loop
*
        K = K - KSTEP
        GO TO 10
*
   30   CONTINUE
*
*       Update the upper triangle of A11 (= A(1:k,1:k)) as
*
*       A11 := A11 - U12*D*U12**T = A11 - U12*W**T
*
*       computing blocks of NB columns at a time
*
        DO 50 J = ( ( K-1 ) / NB )*NB + 1, 1, -NB
           JB = MIN( NB, K-J+1 )
*
*          Update the upper triangle of the diagonal block
*
           DO 40 JJ = J, J + JB - 1
              CALL ZGEMV( 'No transpose', JJ-J+1, N-K, -CONE,
     $                    A( J, K+1 ), LDA, W( JJ, KW+1 ), LDW, CONE,
     $                    A( J, JJ ), 1 )
   40      CONTINUE
```

```
*
*          Update the rectangular superdiagonal block
*
           CALL ZGEMM( 'No transpose', 'Transpose', J-1, JB, N-K,
     $                 -CONE, A( 1, K+1 ), LDA, W( J, KW+1 ), LDW,
     $                 CONE, A( 1, J ), LDA )
   50    CONTINUE
*
*        Put U12 in standard form by partially undoing the interchanges
*        in columns k+1:n looping backwards from k+1 to n
*
         J = K + 1
   60    CONTINUE
*
*          Undo the interchanges (if any) of rows JJ and JP at each
*          step J
*
*          (Here, J is a diagonal index)
           JJ = J
           JP = IPIV( J )
           IF( JP.LT.0 ) THEN
              JP = -JP
*             (Here, J is a diagonal index)
              J = J + 1
           END IF
*          (NOTE: Here, J is used to determine row length. Length N-J+1
*          of the rows to swap back doesn't include diagonal element)
           J = J + 1
           IF( JP.NE.JJ .AND. J.LE.N )
     $        CALL ZSWAP( N-J+1, A( JP, J ), LDA, A( JJ, J ), LDA )
         IF( J.LT.N )
     $      GO TO 60
*
*        Set KB to the number of columns factorized
*
         KB = N - K
*
      ELSE
*
*        Factorize the leading columns of A using the lower triangle
*        of A and working forwards, and compute the matrix W = L21*D
*        for use in updating A22
*
*        K is the main loop index, increasing from 1 in steps of 1 or 2
*
         K = 1
   70    CONTINUE
*
*        Exit from loop
*
```

```
         IF( ( K.GE.NB .AND. NB.LT.N ) .OR. K.GT.N )
     $      GO TO 90
*
*        Copy column K of A to column K of W and update it
*
         CALL ZCOPY( N-K+1, A( K, K ), 1, W( K, K ), 1 )
         CALL ZGEMV( 'No transpose', N-K+1, K-1, -CONE, A( K, 1 ), LDA,
     $               W( K, 1 ), LDW, CONE, W( K, K ), 1 )
*
         KSTEP = 1
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = CABS1( W( K, K ) )
*
*        IMAX is the row-index of the largest off-diagonal element in

*
         IF( K.LT.N ) THEN
            IMAX = K + IZAMAX( N-K, W( K+1, K ), 1 )
            COLMAX = CABS1( W( IMAX, K ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( MAX( ABSAKK, COLMAX ).EQ.ZERO ) THEN
*
*           Column K is zero or underflow: set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
         ELSE
            IF( ABSAKK.GE.ALPHA*COLMAX ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
            ELSE
*
*              Copy column IMAX to column K+1 of W and update it
*
               CALL ZCOPY( IMAX-K, A( IMAX, K ), LDA, W( K, K+1 ), 1 )
               CALL ZCOPY( N-IMAX+1, A( IMAX, IMAX ), 1, W( IMAX, K+1 ),
     $                     1 )
               CALL ZGEMV( 'No transpose', N-K+1, K-1, -CONE, A( K, 1 ),
     $                     LDA, W( IMAX, 1 ), LDW, CONE, W( K, K+1 ),
     $                     1 )
*
```

```
*              JMAX is the column-index of the largest off-diagonal
*              element in row IMAX, and ROWMAX is its absolute value
*
               JMAX = K - 1 + IZAMAX( IMAX-K, W( K, K+1 ), 1 )
               ROWMAX = CABS1( W( JMAX, K+1 ) )
               IF( IMAX.LT.N ) THEN
                  JMAX = IMAX + IZAMAX( N-IMAX, W( IMAX+1, K+1 ), 1 )
                  ROWMAX = MAX( ROWMAX, CABS1( W( JMAX, K+1 ) ) )
               END IF
*
               IF( ABSAKK.GE.ALPHA*COLMAX*( COLMAX / ROWMAX ) ) THEN
*
*                 no interchange, use 1-by-1 pivot block
*
                  KP = K
               ELSE IF( CABS1( W( IMAX, K+1 ) ).GE.ALPHA*ROWMAX ) THEN
*
*                 interchange rows and columns K and IMAX, use 1-by-1
*                 pivot block
*
                  KP = IMAX
*
*                 copy column K+1 of W to column K of W
*
                  CALL ZCOPY( N-K+1, W( K, K+1 ), 1, W( K, K ), 1 )
               ELSE
*
*                 interchange rows and columns K+1 and IMAX, use 2-by-2
*                 pivot block
*
                  KP = IMAX
                  KSTEP = 2
               END IF
            END IF
*
*           ============================================================
*
*           KK is the column of A where pivoting step stopped
*
            KK = K + KSTEP - 1
*
*           Interchange rows and columns KP and KK.
*           Updated column KP is already stored in column KK of W.
*
            IF( KP.NE.KK ) THEN
*
*              Copy non-updated column KK to column KP of submatrix A
*              at step K. No need to copy element into column K
*              (or K and K+1 for 2-by-2 pivot) of A, since these columns
*              will be later overwritten.
```

```
*
               A( KP, KP ) = A( KK, KK )
               CALL ZCOPY( KP-KK-1, A( KK+1, KK ), 1, A( KP, KK+1 ),
     $                     LDA )
               IF( KP.LT.N )
     $            CALL ZCOPY( N-KP, A( KP+1, KK ), 1, A( KP+1, KP ), 1 )
*
*              Interchange rows KK and KP in first K-1 columns of A
*              (columns K (or K and K+1 for 2-by-2 pivot) of A will be
*              later overwritten). Interchange rows KK and KP
*              in first KK columns of W.
*
               IF( K.GT.1 )
     $            CALL ZSWAP( K-1, A( KK, 1 ), LDA, A( KP, 1 ), LDA )
               CALL ZSWAP( KK, W( KK, 1 ), LDW, W( KP, 1 ), LDW )
            END IF
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column k of W now holds
*
*              W(k) = L(k)*D(k),
*
*              where L(k) is the k-th column of L
*
*              Store subdiag. elements of column L(k)
*              and 1-by-1 block D(k) in column k of A.
*              (NOTE: Diagonal element L(k,k) is a UNIT element
*              and not stored)
*                 A(k,k) := D(k,k) = W(k,k)
*                 A(k+1:N,k) := L(k+1:N,k) = W(k+1:N,k)/D(k,k)
*
               CALL ZCOPY( N-K+1, W( K, K ), 1, A( K, K ), 1 )
               IF( K.LT.N ) THEN
                  R1 = CONE / A( K, K )
                  CALL ZSCAL( N-K, R1, A( K+1, K ), 1 )
               END IF
*
            ELSE
*
*              2-by-2 pivot block D(k): columns k and k+1 of W now hold
*
*              ( W(k) W(k+1) ) = ( L(k) L(k+1) )*D(k)
*
*              where L(k) and L(k+1) are the k-th and (k+1)-th columns
*              of L
*
*              Store L(k+2:N,k) and L(k+2:N,k+1) and 2-by-2
*              block D(k:k+1,k:k+1) in columns k and k+1 of A.
*              (NOTE: 2-by-2 diagonal block L(k:k+1,k:k+1) is a UNIT
```

```
*              block and not stored)
*                 A(k:k+1,k:k+1) := D(k:k+1,k:k+1) = W(k:k+1,k:k+1)
*                 A(k+2:N,k:k+1) := L(k+2:N,k:k+1) =
*                 = W(k+2:N,k:k+1) * ( D(k:k+1,k:k+1)**(-1) )
*
               IF( K.LT.N-1 ) THEN
*
*                 Compose the columns of the inverse of 2-by-2 pivot
*                 block D in the following way to reduce the number
*                 of FLOPS when we myltiply panel ( W(k) W(k+1) ) by
*                 this inverse
*
*                 D**(-1) = ( d11 d21 )**(-1) =
*                           ( d21 d22 )
*
*                 = 1/(d11*d22-d21**2) * ( ( d22 ) (-d21 ) ) =
*                                        ( (-d21 ) ( d11 ) )
*
*                 = 1/d21 * 1/((d11/d21)*(d22/d21)-1) *
*
*                   * ( ( d22/d21 ) (      -1 ) ) =
*                     ( (      -1 ) ( d11/d21 ) )
*
*                 = 1/d21 * 1/(D22*D11-1) * ( ( D11 ) (  -1 ) ) =
*                                           ( ( -1  ) ( D22 ) )
*
*                 = 1/d21 * T * ( ( D11 ) (  -1 ) )
*                               ( (  -1 ) ( D22 ) )
*
*                 = D21 * ( ( D11 ) (  -1 ) )
*                         ( (  -1 ) ( D22 ) )
*
                  D21 = W( K+1, K )
                  D11 = W( K+1, K+1 ) / D21
                  D22 = W( K, K ) / D21
                  T = CONE / ( D11*D22-CONE )
                  D21 = T / D21
*
*                 Update elements in columns A(k) and A(k+1) as
*                 dot products of rows of ( W(k) W(k+1) ) and columns
*                 of D**(-1)
*
                  DO 80 J = K + 2, N
                     A( J, K ) = D21*( D11*W( J, K )-W( J, K+1 ) )
                     A( J, K+1 ) = D21*( D22*W( J, K+1 )-W( J, K ) )
   80             CONTINUE
               END IF
*
*              Copy D(k) to A
*
```

```
               A( K, K ) = W( K, K )
               A( K+1, K ) = W( K+1, K )
               A( K+1, K+1 ) = W( K+1, K+1 )
*
            END IF
*
         END IF
*
*        Store details of the interchanges in IPIV
*
         IF( KSTEP.EQ.1 ) THEN
            IPIV( K ) = KP
         ELSE
            IPIV( K ) = -KP
            IPIV( K+1 ) = -KP
         END IF
*
*        Increase K and return to the start of the main loop
*
         K = K + KSTEP
         GO TO 70
*
   90    CONTINUE
*
*        Update the lower triangle of A22 (= A(k:n,k:n)) as
*
*        A22 := A22 - L21*D*L21**T = A22 - L21*W**T
*
*        computing blocks of NB columns at a time
*
         DO 110 J = K, N, NB
            JB = MIN( NB, N-J+1 )
*
*           Update the lower triangle of the diagonal block
*
            DO 100 JJ = J, J + JB - 1
               CALL ZGEMV( 'No transpose', J+JB-JJ, K-1, -CONE,
     $                     A( JJ, 1 ), LDA, W( JJ, 1 ), LDW, CONE,
     $                     A( JJ, JJ ), 1 )
  100       CONTINUE
*
*           Update the rectangular subdiagonal block
*
            IF( J+JB.LE.N )
     $         CALL ZGEMM( 'No transpose', 'Transpose', N-J-JB+1, JB,
     $                     K-1, -CONE, A( J+JB, 1 ), LDA, W( J, 1 ),
     $                     LDW, CONE, A( J+JB, J ), LDA )
  110    CONTINUE
*
*        Put L21 in standard form by partially undoing the interchanges
```

```
*        of rows in columns 1:k-1 looping backwards from k-1 to 1
*
         J = K - 1
  120    CONTINUE
*
*           Undo the interchanges (if any) of rows JJ and JP at each
*           step J
*
*           (Here, J is a diagonal index)
            JJ = J
            JP = IPIV( J )
            IF( JP.LT.0 ) THEN
               JP = -JP
*              (Here, J is a diagonal index)
               J = J - 1
            END IF
*           (NOTE: Here, J is used to determine row length. Length J
*           of the rows to swap back doesn't include diagonal element)
            J = J - 1
            IF( JP.NE.JJ .AND. J.GE.1 )
     $         CALL ZSWAP( J, A( JP, 1 ), LDA, A( JJ, 1 ), LDA )
         IF( J.GT.1 )
     $      GO TO 120
*
*        Set KB to the number of columns factorized
*
         KB = K - 1
*
      END IF
      RETURN
*
*     End of ZLASYF
*
      END
```

———-

61.6.4 lisp code

— **zlasyf.lisp** —

———-

61.7 zlasyf_rook

61.7.1 Axiom unit tests

— **zlasyfrook.input** —

```
)set break resume
)sys rm -f zlasyfrook.output
)spool zlasyfrook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.7.2 Axiom help page

— **zlasyf_rook.help** —

```
======================================================================
zlasyf_rook examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zlasyf_rook (UPLO, N, NB, KB, A, LDA, IPIV, W, LDW, INFO)
           ZLASYF_ROOK computes a partial factorization of a complex symmetric
           matrix using the bounded Bunch-Kaufman ('rook') diagonal pivoting
           method.

Function/Subroutine Documentation
   subroutine zlasyf_rook (character UPLO, integer N, integer NB, integer KB,
       complex*16, dimension( lda, * ) A, integer LDA, integer, dimension( * )
       IPIV, complex*16, dimension( ldw, * ) W, integer LDW, integer INFO)
       ZLASYF_ROOK computes a partial factorization of a complex symmetric
       matrix using the bounded Bunch-Kaufman ('rook') diagonal pivoting
       method.

Purpose:
```

```
  ZLASYF_ROOK computes a partial factorization of a complex symmetric
  matrix A using the bounded Bunch-Kaufman ("rook") diagonal
  pivoting method. The partial factorization has the form:

      A  =  ( I  U12 ) ( A11  0  ) (  I       0    )  if UPLO = 'U', or:
            ( 0  U22 ) (  0   D  ) ( U12**T U22**T )

      A  =  ( L11  0 ) (  D   0  ) ( L11**T L21**T )  if UPLO = 'L'
            ( L21  I ) (  0  A22 ) (  0       I    )

  where the order of D is at most NB. The actual order is returned in
  the argument KB, and is either NB or NB-1, or N if N <= NB.

  ZLASYF_ROOK is an auxiliary routine called by ZSYTRF_ROOK. It uses
  blocked code (calling Level 3 BLAS) to update the submatrix
  A11 (if UPLO = 'U') or A22 (if UPLO = 'L').

Parameters:

 UPLO
  UPLO is CHARACTER*1
  Specifies whether the upper or lower triangular part of the
  symmetric matrix A is stored:
  = 'U':  Upper triangular
  = 'L':  Lower triangular

 N
  N is INTEGER
  The order of the matrix A.  N >= 0.

 NB
  NB is INTEGER
  The maximum number of columns of the matrix A that should be
  factored.  NB should be at least 2 to allow for 2-by-2 pivot
  blocks.

 KB
  KB is INTEGER
  The number of columns of A that were actually factored.
  KB is either NB-1 or NB, or N if N <= NB.

 A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  n-by-n upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading n-by-n lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.
```

```
  On exit, A contains details of the partial factorization.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D.

  If UPLO = 'U':
    Only the last KB elements of IPIV are set.

    If IPIV(k) > 0, then rows and columns k and IPIV(k) were
    interchanged and D(k,k) is a 1-by-1 diagonal block.

    If IPIV(k) < 0 and IPIV(k-1) < 0, then rows and
    columns k and -IPIV(k) were interchanged and rows and
    columns k-1 and -IPIV(k-1) were inerchaged,
    D(k-1:k,k-1:k) is a 2-by-2 diagonal block.

  If UPLO = 'L':
    Only the first KB elements of IPIV are set.

    If IPIV(k) > 0, then rows and columns k and IPIV(k)
    were interchanged and D(k,k) is a 1-by-1 diagonal block.

    If IPIV(k) < 0 and IPIV(k+1) < 0, then rows and
    columns k and -IPIV(k) were interchanged and rows and
    columns k+1 and -IPIV(k+1) were inerchaged,
    D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

W
  W is COMPLEX*16 array, dimension (LDW,NB)

LDW
  LDW is INTEGER
  The leading dimension of the array W.  LDW >= max(1,N).

INFO
  INFO is INTEGER
  = 0: successful exit
  > 0: if INFO = k, D(k,k) is exactly zero.  The factorization
       has been completed, but the block diagonal matrix D is
       exactly singular.
```

61.7.3 fortran code

— zlasyf_rook.f —

```
*  ===================================================================
      SUBROUTINE ZLASYF_ROOK( UPLO, N, NB, KB, A, LDA, IPIV, W, LDW,
     $                        INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, KB, LDA, LDW, N, NB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), W( LDW, * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   EIGHT, SEVTEN
      PARAMETER          ( EIGHT = 8.0D+0, SEVTEN = 17.0D+0 )
      COMPLEX*16         CONE, CZERO
      PARAMETER          ( CONE = ( 1.0D+0, 0.0D+0 ),
     $                   CZERO = ( 0.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            DONE
      INTEGER            IMAX, ITEMP, J, JB, JJ, JMAX, JP1, JP2, K, KK,
     $                   KW, KKW, KP, KSTEP, P, II
      DOUBLE PRECISION   ABSAKK, ALPHA, COLMAX, ROWMAX, DTEMP, SFMIN
      COMPLEX*16         D11, D12, D21, D22, R1, T, Z
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IZAMAX
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, IZAMAX, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZCOPY, ZGEMM, ZGEMV, ZSCAL, ZSWAP
*     ..
```

```
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT, DIMAG, DBLE
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( Z ) = ABS( DBLE( Z ) ) + ABS( DIMAG( Z ) )
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
*     Initialize ALPHA for use in choosing pivot block size.
*
      ALPHA = ( ONE+SQRT( SEVTEN ) ) / EIGHT
*
*     Compute machine safe minimum
*
      SFMIN = DLAMCH( 'S' )
*
      IF( LSAME( UPLO, 'U' ) ) THEN
*
*        Factorize the trailing columns of A using the upper triangle
*        of A and working backwards, and compute the matrix W = U12*D
*        for use in updating A11
*
*        K is the main loop index, decreasing from N in steps of 1 or 2
*
         K = N
   10    CONTINUE
*
*        KW is the column of W which corresponds to column K of A
*
         KW = NB + K - N
*
*        Exit from loop
*
         IF( ( K.LE.N-NB+1 .AND. NB.LT.N ) .OR. K.LT.1 )
     $      GO TO 30
*
         KSTEP = 1
         P = K
*
*        Copy column K of A to column KW of W and update it
*
         CALL ZCOPY( K, A( 1, K ), 1, W( 1, KW ), 1 )
         IF( K.LT.N )
     $      CALL ZGEMV( 'No transpose', K, N-K, -CONE, A( 1, K+1 ),
     $                  LDA, W( K, KW+1 ), LDW, CONE, W( 1, KW ), 1 )
```

```
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = CABS1( W( K, KW ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.GT.1 ) THEN
            IMAX = IZAMAX( K-1, W( 1, KW ), 1 )
            COLMAX = CABS1( W( IMAX, KW ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( MAX( ABSAKK, COLMAX ).EQ.ZERO ) THEN
*
*           Column K is zero or underflow: set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
            CALL ZCOPY( K, W( 1, KW ), 1, A( 1, K ), 1 )
         ELSE
*
*           ============================================================
*
*           Test for interchange
*
*           Equivalent to testing for ABSAKK.GE.ALPHA*COLMAX
*           (used to handle NaN and Inf)
*
            IF( .NOT.( ABSAKK.LT.ALPHA*COLMAX ) ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
*
            ELSE
*
               DONE = .FALSE.
*
*              Loop until pivot found
*
   12          CONTINUE
*
*                 Begin pivot search loop body
*
```

```
*
*                 Copy column IMAX to column KW-1 of W and update it
*
                  CALL ZCOPY( IMAX, A( 1, IMAX ), 1, W( 1, KW-1 ), 1 )
                  CALL ZCOPY( K-IMAX, A( IMAX, IMAX+1 ), LDA,
     $                        W( IMAX+1, KW-1 ), 1 )
*
                  IF( K.LT.N )
     $               CALL ZGEMV( 'No transpose', K, N-K, -CONE,
     $                           A( 1, K+1 ), LDA, W( IMAX, KW+1 ), LDW,
     $                           CONE, W( 1, KW-1 ), 1 )
*
*                 JMAX is the column-index of the largest off-diagonal
*                 element in row IMAX, and ROWMAX is its absolute value.
*                 Determine both ROWMAX and JMAX.
*
                  IF( IMAX.NE.K ) THEN
                     JMAX = IMAX + IZAMAX( K-IMAX, W( IMAX+1, KW-1 ),
     $                                     1 )
                     ROWMAX = CABS1( W( JMAX, KW-1 ) )
                  ELSE
                     ROWMAX = ZERO
                  END IF
*
                  IF( IMAX.GT.1 ) THEN
                     ITEMP = IZAMAX( IMAX-1, W( 1, KW-1 ), 1 )
                     DTEMP = CABS1( W( ITEMP, KW-1 ) )
                     IF( DTEMP.GT.ROWMAX ) THEN
                        ROWMAX = DTEMP
                        JMAX = ITEMP
                     END IF
                  END IF
*
*                 Equivalent to testing for
*                 CABS1( W( IMAX, KW-1 ) ).GE.ALPHA*ROWMAX
*                 (used to handle NaN and Inf)
*
                  IF( .NOT.(CABS1( W( IMAX, KW-1 ) ).LT.ALPHA*ROWMAX ) )
     $            THEN
*
*                    interchange rows and columns K and IMAX,
*                    use 1-by-1 pivot block
*
                     KP = IMAX
*
*                    copy column KW-1 of W to column KW of W
*
                     CALL ZCOPY( K, W( 1, KW-1 ), 1, W( 1, KW ), 1 )
*
                     DONE = .TRUE.
```

```
*
*                 Equivalent to testing for ROWMAX.EQ.COLMAX,
*                 (used to handle NaN and Inf)
*
                  ELSE IF( ( P.EQ.JMAX ) .OR. ( ROWMAX.LE.COLMAX ) )
     $            THEN
*
*                    interchange rows and columns K-1 and IMAX,
*                    use 2-by-2 pivot block
*
                     KP = IMAX
                     KSTEP = 2
                     DONE = .TRUE.
                  ELSE
*
*                    Pivot not found: set params and repeat
*
                     P = IMAX
                     COLMAX = ROWMAX
                     IMAX = JMAX
*
*                    Copy updated JMAXth (next IMAXth) column to Kth of W
*
                     CALL ZCOPY( K, W( 1, KW-1 ), 1, W( 1, KW ), 1 )
*
                  END IF
*
*                 End pivot search loop body
*
               IF( .NOT. DONE ) GOTO 12
*
            END IF
*
*           ============================================================
*
            KK = K - KSTEP + 1
*
*           KKW is the column of W which corresponds to column KK of A
*
            KKW = NB + KK - N
*
            IF( ( KSTEP.EQ.2 ) .AND. ( P.NE.K ) ) THEN
*
               Copy non-updated column K to column P
*
               CALL ZCOPY( K-P, A( P+1, K ), 1, A( P, P+1 ), LDA )
               CALL ZCOPY( P, A( 1, K ), 1, A( 1, P ), 1 )
*
               Interchange rows K and P in last N-K+1 columns of A
*              and last N-K+2 columns of W
```

```
*
               CALL ZSWAP( N-K+1, A( K, K ), LDA, A( P, K ), LDA )
               CALL ZSWAP( N-KK+1, W( K, KKW ), LDW, W( P, KKW ), LDW )
            END IF
*
*           Updated column KP is already stored in column KKW of W
*
            IF( KP.NE.KK ) THEN
*
*              Copy non-updated column KK to column KP
*
               A( KP, K ) = A( KK, K )
               CALL ZCOPY( K-1-KP, A( KP+1, KK ), 1, A( KP, KP+1 ),
     $                     LDA )
               CALL ZCOPY( KP, A( 1, KK ), 1, A( 1, KP ), 1 )
*
*              Interchange rows KK and KP in last N-KK+1 columns
*              of A and W
*
               CALL ZSWAP( N-KK+1, A( KK, KK ), LDA, A( KP, KK ), LDA )
               CALL ZSWAP( N-KK+1, W( KK, KKW ), LDW, W( KP, KKW ),
     $                     LDW )
            END IF
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column KW of W now holds
*
*              W(k) = U(k)*D(k)
*
*              where U(k) is the k-th column of U
*
*              Store U(k) in column k of A
*
               CALL ZCOPY( K, W( 1, KW ), 1, A( 1, K ), 1 )
               IF( K.GT.1 ) THEN
                  IF( CABS1( A( K, K ) ).GE.SFMIN ) THEN
                     R1 = CONE / A( K, K )
                     CALL ZSCAL( K-1, R1, A( 1, K ), 1 )
                  ELSE IF( A( K, K ).NE.CZERO ) THEN
                     DO 14 II = 1, K - 1
                        A( II, K ) = A( II, K ) / A( K, K )
   14                CONTINUE
                  END IF
               END IF
*
            ELSE
*
*              2-by-2 pivot block D(k): columns KW and KW-1 of W now
*              hold
```

```
*
*             ( W(k-1) W(k) ) = ( U(k-1) U(k) )*D(k)
*
*             where U(k) and U(k-1) are the k-th and (k-1)-th columns
*             of U
*
              IF( K.GT.2 ) THEN
*
*                Store U(k) and U(k-1) in columns k and k-1 of A
*
                 D12 = W( K-1, KW )
                 D11 = W( K, KW ) / D12
                 D22 = W( K-1, KW-1 ) / D12
                 T = CONE / ( D11*D22-CONE )
                 DO 20 J = 1, K - 2
                    A( J, K-1 ) = T*( (D11*W( J, KW-1 )-W( J, KW ) ) /
     $                              D12 )
                    A( J, K ) = T*( ( D22*W( J, KW )-W( J, KW-1 ) ) /
     $                          D12 )
   20            CONTINUE
              END IF
*
*             Copy D(k) to A
*
              A( K-1, K-1 ) = W( K-1, KW-1 )
              A( K-1, K ) = W( K-1, KW )
              A( K, K ) = W( K, KW )
           END IF
        END IF
*
*       Store details of the interchanges in IPIV
*
        IF( KSTEP.EQ.1 ) THEN
           IPIV( K ) = KP
        ELSE
           IPIV( K ) = -P
           IPIV( K-1 ) = -KP
        END IF
*
*       Decrease K and return to the start of the main loop
*
        K = K - KSTEP
        GO TO 10
*
   30   CONTINUE
*
*       Update the upper triangle of A11 (= A(1:k,1:k)) as
*
*       A11 := A11 - U12*D*U12**T = A11 - U12*W**T
*
```

```
*        computing blocks of NB columns at a time
*
         DO 50 J = ( ( K-1 ) / NB )*NB + 1, 1, -NB
            JB = MIN( NB, K-J+1 )
*
*           Update the upper triangle of the diagonal block
*
            DO 40 JJ = J, J + JB - 1
               CALL ZGEMV( 'No transpose', JJ-J+1, N-K, -CONE,
     $                     A( J, K+1 ), LDA, W( JJ, KW+1 ), LDW, CONE,
     $                     A( J, JJ ), 1 )
   40       CONTINUE
*
*           Update the rectangular superdiagonal block
*
            IF( J.GE.2 )
     $         CALL ZGEMM( 'No transpose', 'Transpose', J-1, JB,
     $                  N-K, -CONE, A( 1, K+1 ), LDA, W( J, KW+1 ), LDW,
     $                  CONE, A( 1, J ), LDA )
   50    CONTINUE
*
*        Put U12 in standard form by partially undoing the interchanges
*        in columns k+1:n
*
         J = K + 1
   60    CONTINUE
*
            KSTEP = 1
            JP1 = 1
            JJ = J
            JP2 = IPIV( J )
            IF( JP2.LT.0 ) THEN
               JP2 = -JP2
               J = J + 1
               JP1 = -IPIV( J )
               KSTEP = 2
            END IF
*
            J = J + 1
            IF( JP2.NE.JJ .AND. J.LE.N )
     $         CALL ZSWAP( N-J+1, A( JP2, J ), LDA, A( JJ, J ), LDA )
            JJ = J - 1
            IF( JP1.NE.JJ .AND. KSTEP.EQ.2 )
     $         CALL ZSWAP( N-J+1, A( JP1, J ), LDA, A( JJ, J ), LDA )
         IF( J.LE.N )
     $      GO TO 60
*
*        Set KB to the number of columns factorized
*
         KB = N - K
```

```
*
      ELSE
*
*        Factorize the leading columns of A using the lower triangle
*        of A and working forwards, and compute the matrix W = L21*D
*        for use in updating A22
*
*        K is the main loop index, increasing from 1 in steps of 1 or 2
*
         K = 1
   70    CONTINUE
*
*        Exit from loop
*
         IF( ( K.GE.NB .AND. NB.LT.N ) .OR. K.GT.N )
     $      GO TO 90
*
         KSTEP = 1
         P = K
*
*        Copy column K of A to column K of W and update it
*
         CALL ZCOPY( N-K+1, A( K, K ), 1, W( K, K ), 1 )
         IF( K.GT.1 )
     $      CALL ZGEMV( 'No transpose', N-K+1, K-1, -CONE, A( K, 1 ),
     $                  LDA, W( K, 1 ), LDW, CONE, W( K, K ), 1 )
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = CABS1( W( K, K ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.LT.N ) THEN
            IMAX = K + IZAMAX( N-K, W( K+1, K ), 1 )
            COLMAX = CABS1( W( IMAX, K ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( MAX( ABSAKK, COLMAX ).EQ.ZERO ) THEN
*
*           Column K is zero or underflow: set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
```

```
            CALL ZCOPY( N-K+1, W( K, K ), 1, A( K, K ), 1 )
         ELSE
*
*           ============================================================
*
*           Test for interchange
*
*           Equivalent to testing for ABSAKK.GE.ALPHA*COLMAX
*           (used to handle NaN and Inf)
*
            IF( .NOT.( ABSAKK.LT.ALPHA*COLMAX ) ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
*
            ELSE
*
               DONE = .FALSE.
*
*              Loop until pivot found
*
   72          CONTINUE
*
*                 Begin pivot search loop body
*
*
*                 Copy column IMAX to column K+1 of W and update it
*
                  CALL ZCOPY( IMAX-K, A( IMAX, K ), LDA, W( K, K+1 ), 1)
                  CALL ZCOPY( N-IMAX+1, A( IMAX, IMAX ), 1,
     $                        W( IMAX, K+1 ), 1 )
                  IF( K.GT.1 )
     $               CALL ZGEMV( 'No transpose', N-K+1, K-1, -CONE,
     $                           A( K, 1 ), LDA, W( IMAX, 1 ), LDW,
     $                           CONE, W( K, K+1 ), 1 )
*
*                 JMAX is the column-index of the largest off-diagonal
*                 element in row IMAX, and ROWMAX is its absolute value.
*                 Determine both ROWMAX and JMAX.
*
                  IF( IMAX.NE.K ) THEN
                     JMAX = K - 1 + IZAMAX( IMAX-K, W( K, K+1 ), 1 )
                     ROWMAX = CABS1( W( JMAX, K+1 ) )
                  ELSE
                     ROWMAX = ZERO
                  END IF
*
                  IF( IMAX.LT.N ) THEN
                     ITEMP = IMAX + IZAMAX( N-IMAX, W( IMAX+1, K+1 ), 1)
```

```
                  DTEMP = CABS1( W( ITEMP, K+1 ) )
                  IF( DTEMP.GT.ROWMAX ) THEN
                     ROWMAX = DTEMP
                     JMAX = ITEMP
                  END IF
               END IF
*
*              Equivalent to testing for
*              CABS1( W( IMAX, K+1 ) ).GE.ALPHA*ROWMAX
*              (used to handle NaN and Inf)
*
               IF( .NOT.( CABS1( W( IMAX, K+1 ) ).LT.ALPHA*ROWMAX ) )
     $         THEN
*
*                 interchange rows and columns K and IMAX,
*                 use 1-by-1 pivot block
*
                  KP = IMAX
*
*                 copy column K+1 of W to column K of W
*
                  CALL ZCOPY( N-K+1, W( K, K+1 ), 1, W( K, K ), 1 )
*
                  DONE = .TRUE.
*
*              Equivalent to testing for ROWMAX.EQ.COLMAX,
*              (used to handle NaN and Inf)
*
               ELSE IF( ( P.EQ.JMAX ) .OR. ( ROWMAX.LE.COLMAX ) )
     $         THEN
*
*                 interchange rows and columns K+1 and IMAX,
*                 use 2-by-2 pivot block
*
                  KP = IMAX
                  KSTEP = 2
                  DONE = .TRUE.
               ELSE
*
*                 Pivot not found: set params and repeat
*
                  P = IMAX
                  COLMAX = ROWMAX
                  IMAX = JMAX
*
*                 Copy updated JMAXth (next IMAXth) column to Kth of W
*
                  CALL ZCOPY( N-K+1, W( K, K+1 ), 1, W( K, K ), 1 )
*
               END IF
```

```
*
*                End pivot search loop body
*
               IF( .NOT. DONE ) GOTO 72
*
            END IF
*
*           ============================================================
*
            KK = K + KSTEP - 1
*
            IF( ( KSTEP.EQ.2 ) .AND. ( P.NE.K ) ) THEN
*
*              Copy non-updated column K to column P
*
               CALL ZCOPY( P-K, A( K, K ), 1, A( P, K ), LDA )
               CALL ZCOPY( N-P+1, A( P, K ), 1, A( P, P ), 1 )
*
*              Interchange rows K and P in first K columns of A
*              and first K+1 columns of W
*
               CALL ZSWAP( K, A( K, 1 ), LDA, A( P, 1 ), LDA )
               CALL ZSWAP( KK, W( K, 1 ), LDW, W( P, 1 ), LDW )
            END IF
*
*           Updated column KP is already stored in column KK of W
*
            IF( KP.NE.KK ) THEN
*
*              Copy non-updated column KK to column KP
*
               A( KP, K ) = A( KK, K )
               CALL ZCOPY( KP-K-1, A( K+1, KK ), 1, A( KP, K+1 ), LDA )
               CALL ZCOPY( N-KP+1, A( KP, KK ), 1, A( KP, KP ), 1 )
*
*              Interchange rows KK and KP in first KK columns of A and W
*
               CALL ZSWAP( KK, A( KK, 1 ), LDA, A( KP, 1 ), LDA )
               CALL ZSWAP( KK, W( KK, 1 ), LDW, W( KP, 1 ), LDW )
            END IF
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column k of W now holds
*
*              W(k) = L(k)*D(k)
*
*              where L(k) is the k-th column of L
*
*              Store L(k) in column k of A
```

```
*
               CALL ZCOPY( N-K+1, W( K, K ), 1, A( K, K ), 1 )
               IF( K.LT.N ) THEN
                  IF( CABS1( A( K, K ) ).GE.SFMIN ) THEN
                     R1 = CONE / A( K, K )
                     CALL ZSCAL( N-K, R1, A( K+1, K ), 1 )
                  ELSE IF( A( K, K ).NE.CZERO ) THEN
                     DO 74 II = K + 1, N
                        A( II, K ) = A( II, K ) / A( K, K )
   74                CONTINUE
                  END IF
               END IF
*
            ELSE
*
*              2-by-2 pivot block D(k): columns k and k+1 of W now hold
*
*              ( W(k) W(k+1) ) = ( L(k) L(k+1) )*D(k)
*
*              where L(k) and L(k+1) are the k-th and (k+1)-th columns
*              of L
*
               IF( K.LT.N-1 ) THEN
*
*                 Store L(k) and L(k+1) in columns k and k+1 of A
*
                  D21 = W( K+1, K )
                  D11 = W( K+1, K+1 ) / D21
                  D22 = W( K, K ) / D21
                  T = CONE / ( D11*D22-CONE )
                  DO 80 J = K + 2, N
                     A( J, K ) = T*( ( D11*W( J, K )-W( J, K+1 ) ) /
     $                          D21 )
                     A( J, K+1 ) = T*( ( D22*W( J, K+1 )-W( J, K ) ) /
     $                            D21 )
   80             CONTINUE
               END IF
*
*              Copy D(k) to A
*
               A( K, K ) = W( K, K )
               A( K+1, K ) = W( K+1, K )
               A( K+1, K+1 ) = W( K+1, K+1 )
            END IF
         END IF
*
*        Store details of the interchanges in IPIV
*
         IF( KSTEP.EQ.1 ) THEN
            IPIV( K ) = KP
```

```
         ELSE
            IPIV( K ) = -P
            IPIV( K+1 ) = -KP
         END IF
*
*        Increase K and return to the start of the main loop
*
         K = K + KSTEP
         GO TO 70
*
   90    CONTINUE
*
*        Update the lower triangle of A22 (= A(k:n,k:n)) as
*
*        A22 := A22 - L21*D*L21**T = A22 - L21*W**T
*
*        computing blocks of NB columns at a time
*
         DO 110 J = K, N, NB
            JB = MIN( NB, N-J+1 )
*
*           Update the lower triangle of the diagonal block
*
            DO 100 JJ = J, J + JB - 1
               CALL ZGEMV( 'No transpose', J+JB-JJ, K-1, -CONE,
     $                     A( JJ, 1 ), LDA, W( JJ, 1 ), LDW, CONE,
     $                     A( JJ, JJ ), 1 )
  100       CONTINUE
*
*           Update the rectangular subdiagonal block
*
            IF( J+JB.LE.N )
     $         CALL ZGEMM( 'No transpose', 'Transpose', N-J-JB+1, JB,
     $                     K-1, -CONE, A( J+JB, 1 ), LDA, W( J, 1 ), LDW,
     $                     CONE, A( J+JB, J ), LDA )
  110    CONTINUE
*
*        Put L21 in standard form by partially undoing the interchanges
*        in columns 1:k-1
*
         J = K - 1
  120    CONTINUE
*
            KSTEP = 1
            JP1 = 1
            JJ = J
            JP2 = IPIV( J )
            IF( JP2.LT.0 ) THEN
               JP2 = -JP2
               J = J - 1
```

```
               JP1 = -IPIV( J )
               KSTEP = 2
            END IF
*
            J = J - 1
            IF( JP2.NE.JJ .AND. J.GE.1 )
     $         CALL ZSWAP( J, A( JP2, 1 ), LDA, A( JJ, 1 ), LDA )
            JJ = J + 1
            IF( JP1.NE.JJ .AND. KSTEP.EQ.2 )
     $         CALL ZSWAP( J, A( JP1, 1 ), LDA, A( JJ, 1 ), LDA )
         IF( J.GE.1 )
     $      GO TO 120
*
*        Set KB to the number of columns factorized
*
         KB = K - 1
*
      END IF
      RETURN
*
*     End of ZLASYF_ROOK
*
      END
```

———-

61.7.4 lisp code

— zlasyf_rook.lisp —

———-

61.8 zsycon

61.8.1 Axiom unit tests

— zsycon.input —

```
)set break resume
)sys rm -f zsycon.output
)spool zsycon.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.8.2 Axiom help page

— **zsycon.help** —

```
=====================================================================
zsycon examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zsycon (UPLO, N, A, LDA, IPIV, ANORM, RCOND, WORK, INFO)
           ZSYCON

Function/Subroutine Documentation
   subroutine zsycon (character UPLO, integer N, complex*16, dimension( lda, *
       ) A, integer LDA, integer, dimension( * ) IPIV, double precision ANORM,
       double precision RCOND, complex*16, dimension( * ) WORK, integer INFO)
       ZSYCON

Purpose:
   ZSYCON estimates the reciprocal of the condition number (in the
   1-norm) of a complex symmetric matrix A using the factorization
   A = U*D*U**T or A = L*D*L**T computed by ZSYTRF.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.
```

```
N
  N is INTEGER
  The order of the matrix A.  N >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  The block diagonal matrix D and the multipliers used to
  obtain the factor U or L as computed by ZSYTRF.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by ZSYTRF.

ANORM
  ANORM is DOUBLE PRECISION
  The 1-norm of the original matrix A.

RCOND
  RCOND is DOUBLE PRECISION
  The reciprocal of the condition number of the matrix A,
  computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is an
  estimate of the 1-norm of inv(A) computed in this routine.

WORK
  WORK is COMPLEX*16 array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

61.8.3 fortran code

— **zsycon.f** —

```
*  =====================================================================
      SUBROUTINE ZSYCON( UPLO, N, A, LDA, IPIV, ANORM, RCOND, WORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
```

```
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
      DOUBLE PRECISION   ANORM, RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, KASE
      DOUBLE PRECISION   AINVNM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLACN2, ZSYTRS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
```

```
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYCON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.LE.ZERO ) THEN
         RETURN
      END IF
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
         DO 10 I = N, 1, -1
            IF( IPIV( I ).GT.0 .AND. A( I, I ).EQ.ZERO )
     $         RETURN
   10    CONTINUE
      ELSE
*
*        Lower triangular storage: examine D from top to bottom.
*
         DO 20 I = 1, N
            IF( IPIV( I ).GT.0 .AND. A( I, I ).EQ.ZERO )
     $         RETURN
   20    CONTINUE
      END IF
*
*     Estimate the 1-norm of the inverse.
*
      KASE = 0
   30 CONTINUE
      CALL ZLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
*
*        Multiply by inv(L*D*L**T) or inv(U*D*U**T).
*
         CALL ZSYTRS( UPLO, N, 1, A, LDA, IPIV, WORK, N, INFO )
         GO TO 30
      END IF
```

```
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
      RETURN
*
*     End of ZSYCON
*
      END
```

———-

61.8.4 lisp code

— **zsycon.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun zsycon (uplo n a lda ipiv anorm rcond work info)
    (declare (type (double-float) rcond anorm)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (ainvnm 0.0d0) (i 0) (kase 0) (upper nil))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (double-float) ainvnm)
                 (type (f2cl-lib:integer4) i kase)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
```

```
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame uplo "L")
           (declare (ignore var-1))
           (when var-0 (setf uplo var-0))
           ret-val)))
   (setf info -1))
  ((< n 0) (setf info -2))
  ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
   (setf info -4))
  ((< anorm zero) (setf info -6)))
 (cond
  ((/= info 0) (xerbla "ZSYCON" (f2cl-lib:int-sub info))
   (go end_label)))
 (setf rcond zero)
 (cond ((= n 0) (setf rcond one) (go end_label))
       ((<= anorm zero) (go end_label)))
 (cond
  (upper
   (f2cl-lib:fdo (i n (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                 ((> i 1) nil)
     (tagbody
       (if (and
            (> (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) 0)
            (= (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
               zero))
           (go end_label))
      label10)))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (if (and
            (> (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) 0)
            (= (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
               zero))
           (go end_label))
      label20))))
 (setf kase 0)
label30
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
     (zlacn2 n
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 ((+ n 1))
                            ((1 *)) work-%offset%)
      work ainvnm kase isave)
   (declare (ignore var-1 var-2 var-5))
   (setf n var-0)
   (setf ainvnm var-3)
   (setf kase var-4))
 (cond
```

```
        ((/= kase 0)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
             (zsytrs uplo n 1 a lda ipiv work n info)
           (declare (ignore var-2 var-3 var-5 var-6))
           (when var-0 (setf uplo var-0))
           (when var-1 (setf n var-1))
           (when var-4 (setf lda var-4))
           (when var-7 (setf n var-7))
           (when var-8 (setf info var-8)))
         (go label30)))
       (if (/= ainvnm zero)
           (setf rcond (/ (/ one ainvnm) anorm)))
       (go end_label)
      end_label
       (return (values uplo n nil lda nil nil rcond nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsycon
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::integer4
                                              (*))
                                             (double-float) (double-float)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(fortran-to-lisp::uplo
                                             fortran-to-lisp::n nil
                                             fortran-to-lisp::lda nil nil
                                             fortran-to-lisp::rcond nil
                                             fortran-to-lisp::info)
                                           :calls
                                           '(fortran-to-lisp::zlacn2
                                             fortran-to-lisp::xerbla))))
```

———-

61.9 zsycon_rook

61.9.1 Axiom unit tests

— **zsyconrook.input** —

```
)set break resume
)sys rm -f zsyconrook.output
)spool zsyconrook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.9.2 Axiom help page

— **zsycon_rook.help** —

```
====================================================================
zsycon_rook examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zsycon_rook (UPLO, N, A, LDA, IPIV, ANORM, RCOND, WORK,
           INFO)
           ZSYCON_ROOK

Function/Subroutine Documentation
   subroutine zsycon_rook (character UPLO, integer N, complex*16, dimension(
       lda, * ) A, integer LDA, integer, dimension( * ) IPIV, double precision
       ANORM, double precision RCOND, complex*16, dimension( * ) WORK, integer
       INFO)
       ZSYCON_ROOK

Purpose:
   ZSYCON_ROOK estimates the reciprocal of the condition number (in the
   1-norm) of a complex symmetric matrix A using the factorization
```

```
   A = U*D*U**T or A = L*D*L**T computed by ZSYTRF_ROOK.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by ZSYTRF_ROOK.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by ZSYTRF_ROOK.

 ANORM
   ANORM is DOUBLE PRECISION
   The 1-norm of the original matrix A.

 RCOND
   RCOND is DOUBLE PRECISION
   The reciprocal of the condition number of the matrix A,
   computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is an
   estimate of the 1-norm of inv(A) computed in this routine.

 WORK
   WORK is COMPLEX*16 array, dimension (2*N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

61.9.3 fortran code

— **zsycon_rook.f** —

```
*  =====================================================================
      SUBROUTINE ZSYCON_ROOK( UPLO, N, A, LDA, IPIV, ANORM, RCOND, WORK,
     $                        INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
      DOUBLE PRECISION   ANORM, RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
      COMPLEX*16            CZERO
      PARAMETER          ( CZERO = ( 0.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, KASE
      DOUBLE PRECISION   AINVNM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZLACN2, ZSYTRS_ROOK, XERBLA
*     ..
```

```
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYCON_ROOK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.LE.ZERO ) THEN
         RETURN
      END IF
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
         DO 10 I = N, 1, -1
            IF( IPIV( I ).GT.0 .AND. A( I, I ).EQ.CZERO )
     $         RETURN
   10    CONTINUE
      ELSE
*
*        Lower triangular storage: examine D from top to bottom.
*
         DO 20 I = 1, N
            IF( IPIV( I ).GT.0 .AND. A( I, I ).EQ.CZERO )
     $         RETURN
```

```
   20    CONTINUE
      END IF
*
*     Estimate the 1-norm of the inverse.
*
      KASE = 0
   30 CONTINUE
      CALL ZLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
*
*        Multiply by inv(L*D*L**T) or inv(U*D*U**T).
*
         CALL ZSYTRS_ROOK( UPLO, N, 1, A, LDA, IPIV, WORK, N, INFO )
         GO TO 30
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
      RETURN
*
*     End of ZSYCON_ROOK
*
      END
```

61.9.4 lisp code

— zsycon_rook.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0) (czero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (f2cl-lib:complex16) czero)
           (ignorable one zero czero))
  (defun zsycon_rook (uplo n a lda ipiv anorm rcond work info)
    (declare (type (double-float) rcond anorm)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
```

```
   (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
(prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
       (ainvnm 0.0d0) (i 0) (kase 0) (upper nil))
  (declare (type (array f2cl-lib:integer4 (3)) isave)
           (type (double-float) ainvnm)
           (type (f2cl-lib:integer4) i kase)
           (type f2cl-lib:logical upper))
  (setf info 0)
  (setf upper
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
  (cond
   ((and (not upper)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4))
   ((< anorm zero) (setf info -6)))
  (cond
   ((/= info 0) (xerbla "ZSYCON_ROOK" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf rcond zero)
  (cond ((= n 0) (setf rcond one) (go end_label))
        ((<= anorm zero) (go end_label)))
  (cond
   (upper
    (f2cl-lib:fdo (i n (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
      (tagbody
        (if (and
             (> (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) 0)
             (= (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                czero))
            (go end_label))
      label10)))
   (t
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (if (and
             (> (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) 0)
```

```
                  (= (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                     czero))
                 (go end_label))
             label20))))
        (setf kase 0)
       label30
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (zlacn2 n
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 ((+ n 1))
                                   ((1 *)) work-%offset%)
             work ainvnm kase isave)
          (declare (ignore var-1 var-2 var-5))
          (setf n var-0)
          (setf ainvnm var-3)
          (setf kase var-4))
        (cond
         ((/= kase 0)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (zsytrs_rook uplo n 1 a lda ipiv work n info)
            (declare (ignore var-2 var-3 var-5 var-6))
            (when var-0 (setf uplo var-0))
            (when var-1 (setf n var-1))
            (when var-4 (setf lda var-4))
            (when var-7 (setf n var-7))
            (when var-8 (setf info var-8)))
          (go label30)))
        (if (/= ainvnm zero)
            (setf rcond (/ (/ one ainvnm) anorm)))
        (go end_label)
       end_label
        (return (values uplo n nil lda nil nil rcond nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsycon_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (double-float) (double-float)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4))
```

```
:return-values
'(fortran-to-lisp::uplo
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil nil
  fortran-to-lisp::rcond nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zlacn2
  fortran-to-lisp::xerbla))))
```

———-

61.10 zsyconv

61.10.1 Axiom unit tests

— **zsyconv.input** —

```
)set break resume
)sys rm -f zsyconv.output
)spool zsyconv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.10.2 Axiom help page

— **zsyconv.help** —

```
==================================================================
zsyconv examples
==================================================================

==================================================================
Man Page Details
==================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine zsyconv (UPLO, WAY, N, A, LDA, IPIV, E, INFO)
           ZSYCONV

Function/Subroutine Documentation
   subroutine zsyconv (character UPLO, character WAY, integer N, complex*16,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       complex*16, dimension( * ) E, integer INFO)
       ZSYCONV

Purpose:
   ZSYCONV converts A given by ZHETRF into L and D or vice-versa.
   Get nondiagonal elements of D (returned in workspace) and
   apply or reverse permutation done in TRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 WAY
   WAY is CHARACTER*1
   = 'C': Convert
   = 'R': Revert

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by ZSYTRF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by ZSYTRF.

 E
   E is COMPLEX*16 array, dimension (N)
```

```
  E stores the supdiagonal/subdiagonal of the symmetric 1-by-1
  or 2-by-2 block diagonal matrix D in LDLT.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

61.10.3 fortran code

— zsyconv.f —

```
*  =====================================================================
      SUBROUTINE ZSYCONV( UPLO, WAY, N, A, LDA, IPIV, E, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO, WAY
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ZERO
      PARAMETER          ( ZERO = (0.0D+0,0.0D+0) )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     .. Local Scalars ..
      LOGICAL            UPPER, CONVERT
      INTEGER            I, IP, J
      COMPLEX*16         TEMP
```

```
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      CONVERT = LSAME( WAY, 'C' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.CONVERT .AND. .NOT.LSAME( WAY, 'R' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5

      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYCONV', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*        A is UPPER
*
         IF ( CONVERT ) THEN
*
*           Convert A (A is upper)
*
*           Convert VALUE
*
            I=N
            E(1)=ZERO
            DO WHILE ( I .GT. 1 )
               IF( IPIV(I) .LT. 0 ) THEN
                  E(I)=A(I-1,I)
                  E(I-1)=ZERO
                  A(I-1,I)=ZERO
                  I=I-1
               ELSE
                  E(I)=ZERO
               ENDIF
               I=I-1
            END DO
*
```

```
*           Convert PERMUTATIONS
*
            I=N
            DO WHILE ( I .GE. 1 )
               IF( IPIV(I) .GT. 0) THEN
                  IP=IPIV(I)
                  IF( I .LT. N) THEN
                     DO 12 J= I+1,N
                       TEMP=A(IP,J)
                       A(IP,J)=A(I,J)
                       A(I,J)=TEMP
 12                  CONTINUE
                  ENDIF
               ELSE
                  IP=-IPIV(I)
                  IF( I .LT. N) THEN
                     DO 13 J= I+1,N
                        TEMP=A(IP,J)
                        A(IP,J)=A(I-1,J)
                        A(I-1,J)=TEMP
 13                  CONTINUE
                  ENDIF
                  I=I-1
               ENDIF
               I=I-1
            END DO
*
         ELSE
*
*           Revert A (A is upper)
*
*           Revert PERMUTATIONS
*
            I=1
            DO WHILE ( I .LE. N )
               IF( IPIV(I) .GT. 0 ) THEN
                  IP=IPIV(I)
                  IF( I .LT. N) THEN
                  DO J= I+1,N
                    TEMP=A(IP,J)
                    A(IP,J)=A(I,J)
                    A(I,J)=TEMP
                  END DO
                  ENDIF
               ELSE
                 IP=-IPIV(I)
                 I=I+1
                 IF( I .LT. N) THEN
                    DO J= I+1,N
                       TEMP=A(IP,J)
```

```
                     A(IP,J)=A(I-1,J)
                     A(I-1,J)=TEMP
                  END DO
               ENDIF
            ENDIF
            I=I+1
         END DO
*
*        Revert VALUE
*
         I=N
         DO WHILE ( I .GT. 1 )
            IF( IPIV(I) .LT. 0 ) THEN
               A(I-1,I)=E(I)
               I=I-1
            ENDIF
            I=I-1
         END DO
      END IF
*
   ELSE
*
*     A is LOWER
*
      IF ( CONVERT ) THEN
*
*        Convert A (A is lower)
*
*        Convert VALUE
*
         I=1
         E(N)=ZERO
         DO WHILE ( I .LE. N )
            IF( I.LT.N .AND. IPIV(I) .LT. 0 ) THEN
               E(I)=A(I+1,I)
               E(I+1)=ZERO
               A(I+1,I)=ZERO
               I=I+1
            ELSE
               E(I)=ZERO
            ENDIF
            I=I+1
         END DO
*
*        Convert PERMUTATIONS
*
         I=1
         DO WHILE ( I .LE. N )
            IF( IPIV(I) .GT. 0 ) THEN
               IP=IPIV(I)
```

```
                 IF (I .GT. 1) THEN
                    DO 22 J= 1,I-1
                       TEMP=A(IP,J)
                       A(IP,J)=A(I,J)
                       A(I,J)=TEMP
 22                 CONTINUE
                 ENDIF
              ELSE
                 IP=-IPIV(I)
                 IF (I .GT. 1) THEN
                    DO 23 J= 1,I-1
                       TEMP=A(IP,J)
                       A(IP,J)=A(I+1,J)
                       A(I+1,J)=TEMP
 23                 CONTINUE
                 ENDIF
                 I=I+1
              ENDIF
              I=I+1
           END DO
*
        ELSE
*
*          Revert A (A is lower)
*
*          Revert PERMUTATIONS
*
           I=N
           DO WHILE ( I .GE. 1 )
              IF( IPIV(I) .GT. 0 ) THEN
                 IP=IPIV(I)
                 IF (I .GT. 1) THEN
                    DO J= 1,I-1
                       TEMP=A(I,J)
                       A(I,J)=A(IP,J)
                       A(IP,J)=TEMP
                    END DO
                 ENDIF
              ELSE
                 IP=-IPIV(I)
                 I=I-1
                 IF (I .GT. 1) THEN
                    DO J= 1,I-1
                       TEMP=A(I+1,J)
                       A(I+1,J)=A(IP,J)
                       A(IP,J)=TEMP
                    END DO
                 ENDIF
              ENDIF
              I=I-1
```

```
            END DO
*
*           Revert VALUE
*
            I=1
            DO WHILE ( I .LE. N-1 )
               IF( IPIV(I) .LT. 0 ) THEN
                  A(I+1,I)=E(I)
                  I=I+1
               ENDIF
               I=I+1
            END DO
         END IF
      END IF
*
      RETURN
*
*     End of ZSYCONV
*
      END
```

———-

61.10.4 lisp code

— **zsyconv.lisp** —

```
(let* ((zero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) zero)
           (ignorable zero))
  (defun zsyconv (uplo way n a lda ipiv e info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) e a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) way uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (way character way-%data% way-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (e f2cl-lib:complex16 e-%data% e-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((temp #C(0.0d0 0.0d0)) (i 0) (ip 0) (j 0) (upper nil)
             (convert nil))
        (declare (type (f2cl-lib:complex16) temp)
                 (type (f2cl-lib:integer4) i ip j)
                 (type f2cl-lib:logical upper convert))
        (setf info 0)
        (setf upper
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(setf convert
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame way "C")
          (declare (ignore var-1))
          (when var-0 (setf way var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((and (not convert)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame way "R")
          (declare (ignore var-1))
          (when var-0 (setf way var-0))
          ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5)))
(cond
 ((/= info 0) (xerbla "ZSYCONV" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(cond
 (upper
  (cond
   (convert
    (tagbody
      (setf i n)
      (setf (f2cl-lib:fref e-%data% (1) ((1 *)) e-%offset%) zero)
     label100000
      (if (not (> i 1))
          (go label100001))
      (cond
       ((< (f2cl-lib:fref ipiv (i) ((1 *))) 0)
        (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) i)
```

```
                          ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1)) ((1 *))
                         e-%offset%)
            zero)
    (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) i)
                         ((1 lda) (1 *)) a-%offset%)
            zero)
    (setf i (f2cl-lib:int-sub i 1)))
   (t (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) zero)))
  (setf i (f2cl-lib:int-sub i 1))
  (go label100000)
 label100001
  (setf i n)
 label100002
  (if (not (>= i 1))
      (go label100003))
  (cond
   ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
    (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
    (cond
     ((< i n)
      (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                     (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf temp
                  (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                 a-%offset%))
          (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                               a-%offset%)
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%))
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%)
                  temp)
         label12)))))
   (t
    (setf ip
            (f2cl-lib:int-sub
             (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                            ipiv-%offset%)))
    (cond
     ((< i n)
      (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                     (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf temp
                  (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                 a-%offset%))
```

```
                (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                      a-%offset%)
                        (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-sub i 1) j)
                                        ((1 lda) (1 *)) a-%offset%))
                (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) j)
                                      ((1 lda) (1 *)) a-%offset%)
                        temp)
              label13))))
          (setf i (f2cl-lib:int-sub i 1))))
        (setf i (f2cl-lib:int-sub i 1))
        (go label100002)
       label100003))
     (t
      (tagbody
        (setf i 1)
       label100004
        (if (not (<= i n))
            (go label100005))
        (cond
         ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
          (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
          (cond
           ((< i n)
            (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                              (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf temp
                        (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                       a-%offset%))
                (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                      a-%offset%)
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                        temp)
              label100006)))))
         (t
          (setf ip
                  (f2cl-lib:int-sub
                   (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                                  ipiv-%offset%)))
          (setf i (f2cl-lib:int-add i 1))
          (cond
           ((< i n)
            (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                              (f2cl-lib:int-add j 1))
                          ((> j n) nil)
```

```
            (tagbody
              (setf temp
                      (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                   a-%offset%)
                      (f2cl-lib:fref a-%data%
                                     ((f2cl-lib:int-sub i 1) j)
                                     ((1 lda) (1 *)) a-%offset%))
              (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) j)
                                   ((1 lda) (1 *)) a-%offset%)
                      temp)
             label100007))))))
      (setf i (f2cl-lib:int-add i 1))
      (go label100004)
    label100005
      (setf i n)
    label100008
      (if (not (> i 1))
          (go label100009))
      (cond
       ((< (f2cl-lib:fref ipiv (i) ((1 *))) 0)
        (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) i)
                             ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))
        (setf i (f2cl-lib:int-sub i 1))))
      (setf i (f2cl-lib:int-sub i 1))
      (go label100008)
    label100009))))
(t
 (cond
  (convert
   (tagbody
     (setf i 1)
     (setf (f2cl-lib:fref e-%data% (n) ((1 *)) e-%offset%) zero)
    label100010
     (if (not (<= i n))
         (go label100011))
     (cond
      ((and (< i n) (< (f2cl-lib:fref ipiv (i) ((1 *))) 0))
       (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
               (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                               ((1 lda) (1 *)) a-%offset%))
       (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                            e-%offset%)
               zero)
       (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                            ((1 lda) (1 *)) a-%offset%)
               zero)
       (setf i (f2cl-lib:int-add i 1)))
```

```
   (t (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) zero)))
  (setf i (f2cl-lib:int-add i 1))
  (go label100010)
 label100011
  (setf i 1)
 label100012
  (if (not (<= i n))
      (go label100013))
  (cond
   ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
    (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
    (cond
     ((> i 1)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j
                        (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf temp
                  (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                 a-%offset%))
          (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                               a-%offset%)
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%))
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%)
                  temp)
         label22)))))
   (t
    (setf ip
            (f2cl-lib:int-sub
             (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                            ipiv-%offset%)))
    (cond
     ((> i 1)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j
                        (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf temp
                  (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                 a-%offset%))
          (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                               a-%offset%)
                  (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add i 1) j)
                                 ((1 lda) (1 *)) a-%offset%))
          (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) j)
```

```
                                  ((1 lda) (1 *)) a-%offset%)
                     temp)
            label23))))
        (setf i (f2cl-lib:int-add i 1))))
      (setf i (f2cl-lib:int-add i 1))
      (go label100012)
     label100013))
   (t
    (tagbody
      (setf i n)
     label100014
      (if (not (>= i 1))
          (go label100015))
      (cond
       ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
        (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
        (cond
         ((> i 1)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j
                            (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf temp
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%)
                      (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                    a-%offset%)
                      temp)
            label100016)))))
       (t
        (setf ip
                (f2cl-lib:int-sub
                 (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                                ipiv-%offset%)))
        (setf i (f2cl-lib:int-sub i 1))
        (cond
         ((> i 1)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j
                            (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf temp
                      (f2cl-lib:fref a-%data%
                                     ((f2cl-lib:int-add i 1) j)
```

```
                                        ((1 lda) (1 *)) a-%offset%))
                    (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) j)
                                        ((1 lda) (1 *)) a-%offset%)
                            (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                        a-%offset%))
                    (setf (f2cl-lib:fref a-%data% (ip j) ((1 lda) (1 *))
                                        a-%offset%)
                            temp)
                  label100017))))))
             (setf i (f2cl-lib:int-sub i 1))
             (go label100014)
            label100015
             (setf i 1)
            label100018
             (if (not (<= i (f2cl-lib:int-sub n 1)))
                 (go label100019))
             (cond
              ((< (f2cl-lib:fref ipiv (i) ((1 *))) 0)
               (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                                   ((1 lda) (1 *)) a-%offset%)
                       (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))
               (setf i (f2cl-lib:int-add i 1))))
             (setf i (f2cl-lib:int-add i 1))
             (go label100018)
            label100019)))))
        (go end_label)
       end_label
        (return (values uplo way nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsyconv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::way nil nil nil
                                              nil nil fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

61.11 zsyequb

61.11.1 Axiom unit tests

— **zsyequb.input** —

```
)set break resume
)sys rm -f zsyequb.output
)spool zsyequb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.11.2 Axiom help page

— **zsyequb.help** —

```
======================================================================
zsyequb examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zsyequb (UPLO, N, A, LDA, S, SCOND, AMAX, WORK, INFO)
           ZSYEQUB

Function/Subroutine Documentation
   subroutine zsyequb (character UPLO, integer N, complex*16, dimension( lda,
       * ) A, integer LDA, double precision, dimension( * ) S, double
       precision SCOND, double precision AMAX, complex*16, dimension( * )
```

```
      WORK, integer INFO)
      ZSYEQUB

Purpose:
   ZSYEQUB computes row and column scalings intended to equilibrate a
   symmetric matrix A and reduce its condition number
   (with respect to the two-norm).  S contains the scale factors,
   S(i) = 1/sqrt(A(i,i)), chosen so that the scaled matrix B with
   elements B(i,j) = S(i)*A(i,j)*S(j) has ones on the diagonal.  This
   choice of S puts the condition number of B within a factor N of the
   smallest possible condition number over all possible diagonal scalings.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   The N-by-N symmetric matrix whose scaling
   factors are to be computed.  Only the diagonal elements of A
   are referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 S
   S is DOUBLE PRECISION array, dimension (N)
   If INFO = 0, S contains the scale factors for A.

 SCOND
   SCOND is DOUBLE PRECISION
   If INFO = 0, S contains the ratio of the smallest S(i) to
   the largest S(i).  If SCOND >= 0.1 and AMAX is neither too
   large nor too small, it is not worth scaling by S.

 AMAX
   AMAX is DOUBLE PRECISION
   Absolute value of largest matrix element.  If AMAX is very
   close to overflow or very close to underflow, the matrix
   should be scaled.
```

```
 WORK
   WORK is COMPLEX*16 array, dimension (3*N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, the i-th diagonal element is nonpositive.
```

———-

61.11.3 fortran code

— **zsyequb.f** —

```
*  =====================================================================
      SUBROUTINE ZSYEQUB( UPLO, N, A, LDA, S, SCOND, AMAX, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, N
      DOUBLE PRECISION   AMAX, SCOND
      CHARACTER          UPLO
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), WORK( * )
      DOUBLE PRECISION   S( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D0, ZERO = 0.0D0 )
      INTEGER            MAX_ITER
      PARAMETER          ( MAX_ITER = 100 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, ITER
      DOUBLE PRECISION   AVG, STD, TOL, C0, C1, C2, T, U, SI, D, BASE,
     $                   SMIN, SMAX, SMLNUM, BIGNUM, SCALE, SUMSQ
      LOGICAL            UP
```

```
      COMPLEX*16         ZDUM
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      LOGICAL            LSAME
      EXTERNAL           DLAMCH, LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZLASSQ
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, INT, LOG, MAX, MIN, SQRT
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     Statement Function Definitions
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF ( .NOT. ( LSAME( UPLO, 'U' ) .OR. LSAME( UPLO, 'L' ) ) ) THEN
        INFO = -1
      ELSE IF ( N .LT. 0 ) THEN
        INFO = -2
      ELSE IF ( LDA .LT. MAX( 1, N ) ) THEN
        INFO = -4
      END IF
      IF ( INFO .NE. 0 ) THEN
        CALL XERBLA( 'ZSYEQUB', -INFO )
        RETURN
      END IF

      UP = LSAME( UPLO, 'U' )
      AMAX = ZERO
*
*     Quick return if possible.
*
      IF ( N .EQ. 0 ) THEN
        SCOND = ONE
        RETURN
      END IF

      DO I = 1, N
        S( I ) = ZERO
      END DO
```

```
      AMAX = ZERO
      IF ( UP ) THEN
         DO J = 1, N
            DO I = 1, J-1
               S( I ) = MAX( S( I ), CABS1( A( I, J ) ) )
               S( J ) = MAX( S( J ), CABS1( A( I, J ) ) )
               AMAX = MAX( AMAX, CABS1( A( I, J ) ) )
            END DO
            S( J ) = MAX( S( J ), CABS1( A( J, J) ) )
            AMAX = MAX( AMAX, CABS1( A( J, J ) ) )
         END DO
      ELSE
         DO J = 1, N
            S( J ) = MAX( S( J ), CABS1( A( J, J ) ) )
            AMAX = MAX( AMAX, CABS1( A( J, J ) ) )
            DO I = J+1, N
               S( I ) = MAX( S( I ), CABS1( A( I, J ) ) )
               S( J ) = MAX( S( J ), CABS1 (A( I, J ) ) )
               AMAX = MAX( AMAX, CABS1( A( I, J ) ) )
            END DO
         END DO
      END IF
      DO J = 1, N
         S( J ) = 1.0D+0 / S( J )
      END DO

      TOL = ONE / SQRT( 2.0D0 * N )

      DO ITER = 1, MAX_ITER
         SCALE = 0.0D+0
         SUMSQ = 0.0D+0
*       beta = |A|s
        DO I = 1, N
           WORK( I ) = ZERO
        END DO
        IF ( UP ) THEN
           DO J = 1, N
              DO I = 1, J-1
                 T = CABS1( A( I, J ) )
                 WORK( I ) = WORK( I ) + CABS1( A( I, J ) ) * S( J )
                 WORK( J ) = WORK( J ) + CABS1( A( I, J ) ) * S( I )
              END DO
              WORK( J ) = WORK( J ) + CABS1( A( J, J ) ) * S( J )
           END DO
        ELSE
           DO J = 1, N
              WORK( J ) = WORK( J ) + CABS1( A( J, J ) ) * S( J )
              DO I = J+1, N
                 T = CABS1( A( I, J ) )
                 WORK( I ) = WORK( I ) + CABS1( A( I, J ) ) * S( J )
```

```
                WORK( J ) = WORK( J ) + CABS1( A( I, J ) ) * S( I )
              END DO
           END DO
        END IF

*       avg = s^T beta / n
        AVG = 0.0D+0
        DO I = 1, N
          AVG = AVG + S( I )*WORK( I )
        END DO
        AVG = AVG / N

        STD = 0.0D+0
        DO I = N+1, 2*N
           WORK( I ) = S( I-N ) * WORK( I-N ) - AVG
        END DO
        CALL ZLASSQ( N, WORK( N+1 ), 1, SCALE, SUMSQ )
        STD = SCALE * SQRT( SUMSQ / N )

        IF ( STD .LT. TOL * AVG ) GOTO 999

        DO I = 1, N
          T = CABS1( A( I, I ) )
          SI = S( I )
          C2 = ( N-1 ) * T
          C1 = ( N-2 ) * ( WORK( I ) - T*SI )
          C0 = -(T*SI)*SI + 2*WORK( I )*SI - N*AVG
          D = C1*C1 - 4*C0*C2

          IF ( D .LE. 0 ) THEN
            INFO = -1
            RETURN
          END IF
          SI = -2*C0 / ( C1 + SQRT( D ) )

          D = SI - S( I )
          U = ZERO
          IF ( UP ) THEN
            DO J = 1, I
              T = CABS1( A( J, I ) )
              U = U + S( J )*T
              WORK( J ) = WORK( J ) + D*T
            END DO
            DO J = I+1,N
              T = CABS1( A( I, J ) )
              U = U + S( J )*T
              WORK( J ) = WORK( J ) + D*T
            END DO
          ELSE
            DO J = 1, I
```

```
              T = CABS1( A( I, J ) )
              U = U + S( J )*T
              WORK( J ) = WORK( J ) + D*T
            END DO
            DO J = I+1,N
              T = CABS1( A( J, I ) )
              U = U + S( J )*T
              WORK( J ) = WORK( J ) + D*T
            END DO
          END IF
          AVG = AVG + ( U + WORK( I ) ) * D / N
          S( I ) = SI
        END DO
      END DO

 999  CONTINUE

      SMLNUM = DLAMCH( 'SAFEMIN' )
      BIGNUM = ONE / SMLNUM
      SMIN = BIGNUM
      SMAX = ZERO
      T = ONE / SQRT( AVG )
      BASE = DLAMCH( 'B' )
      U = ONE / LOG( BASE )
      DO I = 1, N
        S( I ) = BASE ** INT( U * LOG( S( I ) * T ) )
        SMIN = MIN( SMIN, S( I ) )
        SMAX = MAX( SMAX, S( I ) )
      END DO
      SCOND = MAX( SMIN, SMLNUM ) / MIN( SMAX, BIGNUM )
*
      END
```

———-

61.11.4 lisp code

— zsyequb.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0) (max_iter 100))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (f2cl-lib:integer4 100 100) max_iter)
           (ignorable one zero max_iter))
  (defun zsyequb (uplo n a lda s scond amax work info)
    (declare (type (double-float) amax scond)
             (type (array double-float (*)) s)
             (type (array f2cl-lib:complex16 (*)) work a)
```

```
          (type (f2cl-lib:integer4) info lda n)
          (type (string *) uplo))
(f2cl-lib:with-multi-array-data
    ((uplo character uplo-%data% uplo-%offset%)
     (a f2cl-lib:complex16 a-%data% a-%offset%)
     (work f2cl-lib:complex16 work-%data% work-%offset%)
     (s double-float s-%data% s-%offset%))
  (labels ((cabs1 (zdum)
             (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
    (declare
     (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
      cabs1))
    (prog ((zdum #C(0.0d0 0.0d0)) (up nil) (avg 0.0d0) (std 0.0d0)
           (tol 0.0d0) (c0 0.0d0) (c1 0.0d0) (c2 0.0d0) (t$ 0.0d0)
           (u 0.0d0) (si 0.0d0) (d 0.0d0) (base 0.0d0) (smin 0.0d0)
           (smax 0.0d0) (smlnum 0.0d0) (bignum 0.0d0) (scale 0.0d0)
           (sumsq 0.0d0) (i 0) (j 0) (iter 0))
      (declare (type (f2cl-lib:complex16) zdum)
               (type f2cl-lib:logical up)
               (type (double-float) avg std tol c0 c1 c2 t$ u si d base
                smin smax smlnum bignum scale sumsq)
               (type (f2cl-lib:integer4) i j iter))
      (setf info 0)
      (cond
       ((not
         (or
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -4)))
      (cond
       ((/= info 0) (xerbla "ZSYEQUB" (f2cl-lib:int-sub info))
        (go end_label)))
      (setf up
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (setf amax zero)
```

```
(cond ((= n 0) (setf scond one) (go end_label)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%) zero)
   label100000))
(setf amax zero)
(cond
 (up
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                  (max
                   (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                   (cabs1
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))))
          (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                  (max
                   (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                   (cabs1
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))))
          (setf amax
                  (max amax
                       (cabs1
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))))
        label100002))
      (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
              (max (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                   (cabs1
                    (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                   a-%offset%))))
      (setf amax
              (max amax
                   (cabs1
                    (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                   a-%offset%))))
    label100001)))
 (t
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
```

```
                (max (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                     (cabs1
                      (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%))))
        (setf amax
                (max amax
                     (cabs1
                      (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%))))
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                    (max
                     (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                     (cabs1
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))))
            (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                    (max
                     (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                     (cabs1
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))))
            (setf amax
                    (max amax
                         (cabs1
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%))))
          label100004))
      label100003))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
            (/ 1.0d0
               (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
   label100005))
(setf tol (/ one (f2cl-lib:fsqrt (* 2.0d0 n))))
(f2cl-lib:fdo (iter 1 (f2cl-lib:int-add iter 1))
              ((> iter max_iter) nil)
  (tagbody
    (setf scale 0.0d0)
    (setf sumsq 0.0d0)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (coerce zero 'f2cl-lib:complex16))
       label100007))
```

```
(cond
 (up
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (f2cl-lib:int-add j
                                          (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf t$
                  (cabs1
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)))
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                  (+
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                   (*
                    (cabs1
                     (f2cl-lib:fref a-%data% (i j)
                                    ((1 lda) (1 *)) a-%offset%))
                    (f2cl-lib:fref s-%data% (j) ((1 *))
                                   s-%offset%))))
          (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                               work-%offset%)
                  (+
                   (f2cl-lib:fref work-%data% (j) ((1 *))
                                  work-%offset%)
                   (*
                    (cabs1
                     (f2cl-lib:fref a-%data% (i j)
                                    ((1 lda) (1 *)) a-%offset%))
                    (f2cl-lib:fref s-%data% (i) ((1 *))
                                   s-%offset%))))
        label100009))
      (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
              (+
               (f2cl-lib:fref work-%data% (j) ((1 *))
                              work-%offset%)
               (*
                (cabs1
                 (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                a-%offset%))
                (f2cl-lib:fref s-%data% (j) ((1 *))
                               s-%offset%))))
    label100008)))
 (t
```

```
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
            (+
             (f2cl-lib:fref work-%data% (j) ((1 *))
                            work-%offset%)
             (*
              (cabs1
               (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                              a-%offset%))
              (f2cl-lib:fref s-%data% (j) ((1 *))
                             s-%offset%))))
    (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                   (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf t$
                (cabs1
                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)))
        (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                             work-%offset%)
                (+
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 (*
                  (cabs1
                   (f2cl-lib:fref a-%data% (i j)
                                  ((1 lda) (1 *)) a-%offset%))
                  (f2cl-lib:fref s-%data% (j) ((1 *))
                                 s-%offset%))))
        (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                             work-%offset%)
                (+
                 (f2cl-lib:fref work-%data% (j) ((1 *))
                                work-%offset%)
                 (*
                  (cabs1
                   (f2cl-lib:fref a-%data% (i j)
                                  ((1 lda) (1 *)) a-%offset%))
                  (f2cl-lib:fref s-%data% (i) ((1 *))
                                 s-%offset%))))
        label100011))
    label100010))))
(setf avg 0.0d0)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf avg
```

```
            (coerce
             (realpart
              (+ avg
                 (*
                  (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%))))
             'double-float))
   label100012))
(setf avg (/ avg n))
(setf std 0.0d0)
(f2cl-lib:fdo (i (f2cl-lib:int-add n 1) (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-mul 2 n)) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (-
             (*
              (f2cl-lib:fref s-%data% ((f2cl-lib:int-sub i n))
                             ((1 *)) s-%offset%)
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub i n))
                             ((1 *)) work-%offset%))
             avg))
   label100013))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (zlassq n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ n 1)) ((1 *)) work-%offset%)
     1 scale sumsq)
  (declare (ignore var-0 var-1 var-2))
  (setf scale var-3)
  (setf sumsq var-4))
(setf std (* scale (f2cl-lib:fsqrt (/ sumsq n))))
(if (< std (* tol avg))
    (go label999))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf t$
            (cabs1
             (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                            a-%offset%)))
    (setf si (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%))
    (setf c2 (* (f2cl-lib:int-sub n 1) t$))
    (setf c1
            (coerce
             (realpart
              (* (f2cl-lib:int-sub n 2)
                 (-
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)
```

```
                (* t$ si))))
           'double-float))
(setf c0
        (coerce
         (realpart
          (-
           (+ (* (- (* t$ si)) si)
              (* 2
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 si))
           (* n avg)))
         'double-float))
(setf d (+ (* c1 c1) (* -4 c0 c2)))
(cond ((<= d 0) (setf info -1) (go end_label)))
(setf si (/ (* -2 c0) (+ c1 (f2cl-lib:fsqrt d))))
(setf d
        (- si
           (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
(setf u zero)
(cond
 (up
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j i) nil)
    (tagbody
      (setf t$
              (cabs1
               (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                              a-%offset%)))
      (setf u
              (+ u
                 (*
                  (f2cl-lib:fref s-%data% (j) ((1 *))
                                 s-%offset%)
                  t$)))
      (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                           work-%offset%)
              (+
               (f2cl-lib:fref work-%data% (j) ((1 *))
                              work-%offset%)
               (* d t$)))
     label100015))
  (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf t$
              (cabs1
               (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                              a-%offset%)))
```

```
      (setf u
              (+ u
                 (*
                  (f2cl-lib:fref s-%data% (j) ((1 *))
                                 s-%offset%)
                  t$)))
      (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                           work-%offset%)
              (+
               (f2cl-lib:fref work-%data% (j) ((1 *))
                              work-%offset%)
               (* d t$)))
    label100016)))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j i) nil)
   (tagbody
     (setf t$
              (cabs1
               (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                              a-%offset%)))
     (setf u
              (+ u
                 (*
                  (f2cl-lib:fref s-%data% (j) ((1 *))
                                 s-%offset%)
                  t$)))
     (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                          work-%offset%)
              (+
               (f2cl-lib:fref work-%data% (j) ((1 *))
                              work-%offset%)
               (* d t$)))
    label100017))
 (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf t$
              (cabs1
               (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                              a-%offset%)))
     (setf u
              (+ u
                 (*
                  (f2cl-lib:fref s-%data% (j) ((1 *))
                                 s-%offset%)
                  t$)))
     (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                          work-%offset%)
```

```
                        (+
                         (f2cl-lib:fref work-%data% (j) ((1 *))
                                        work-%offset%)
                         (* d t$)))
                label100018))))
            (setf avg
                    (coerce
                     (realpart
                      (+ avg
                         (/
                          (*
                           (+ u
                              (f2cl-lib:fref work-%data% (i) ((1 *))
                                             work-%offset%))
                           d)
                          n)))
                     'double-float))
            (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%) si)
           label100014))
      label100006))
  label999
   (setf smlnum (dlamch "SAFEMIN"))
   (setf bignum (/ one smlnum))
   (setf smin bignum)
   (setf smax zero)
   (setf t$ (/ one (f2cl-lib:fsqrt avg)))
   (setf base (dlamch "B"))
   (setf u (/ one (f2cl-lib:flog base)))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
               (expt base
                     (f2cl-lib:int
                      (* u
                         (f2cl-lib:flog
                          (*
                           (f2cl-lib:fref s-%data% (i) ((1 *))
                                          s-%offset%)
                           t$))))))
       (setf smin
               (min smin
                    (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
       (setf smax
               (max smax
                    (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
      label100019))
   (setf scond (/ (max smin smlnum) (min smax bignum)))
  end_label
   (return (values uplo nil nil nil nil scond amax nil info)))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsyequb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (double-float) (double-float)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil nil
                                              nil fortran-to-lisp::scond
                                              fortran-to-lisp::amax nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::zlassq
                                              fortran-to-lisp::xerbla))))
```

———-

61.12 zsyrfs

61.12.1 Axiom unit tests

— **zsyrfs.input** —

```
)set break resume
)sys rm -f zsyrfs.output
)spool zsyrfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.12.2 Axiom help page

— zsyrfs.help —

```
=====================================================================
zsyrfs examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zsyrfs (UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB, X,
           LDX, FERR, BERR, WORK, RWORK, INFO)
           ZSYRFS

Function/Subroutine Documentation
   subroutine zsyrfs (character UPLO, integer N, integer NRHS, complex*16,
       dimension( lda, * ) A, integer LDA, complex*16, dimension( ldaf, * )
       AF, integer LDAF, integer, dimension( * ) IPIV, complex*16, dimension(
       ldb, * ) B, integer LDB, complex*16, dimension( ldx, * ) X, integer
       LDX, double precision, dimension( * ) FERR, double precision,
       dimension( * ) BERR, complex*16, dimension( * ) WORK, double precision,
       dimension( * ) RWORK, integer INFO)
       ZSYRFS

Purpose:
   ZSYRFS improves the computed solution to a system of linear
   equations when the coefficient matrix is symmetric indefinite, and
   provides error bounds and backward error estimates for the solution.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.
```

```
A
  A is COMPLEX*16 array, dimension (LDA,N)
  The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
  upper triangular part of A contains the upper triangular part
  of the matrix A, and the strictly lower triangular part of A
  is not referenced.  If UPLO = 'L', the leading N-by-N lower
  triangular part of A contains the lower triangular part of
  the matrix A, and the strictly upper triangular part of A is
  not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX*16 array, dimension (LDAF,N)
  The factored form of the matrix A.  AF contains the block
  diagonal matrix D and the multipliers used to obtain the
  factor U or L from the factorization A = U*D*U**T or
  A = L*D*L**T as computed by ZSYTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by ZSYTRF.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by ZSYTRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

FERR
  FERR is DOUBLE PRECISION array, dimension (NRHS)
  The estimated forward error bound for each solution vector
```

```
   X(j) (the j-th column of the solution matrix X).
   If XTRUE is the true solution corresponding to X(j), FERR(j)
   is an estimated upper bound for the magnitude of the largest
   element in (X(j) - XTRUE) divided by the magnitude of the
   largest element in X(j).  The estimate is as reliable as
   the estimate for RCOND, and is almost always a slight
   overestimate of the true error.

 BERR
   BERR is DOUBLE PRECISION array, dimension (NRHS)
   The componentwise relative backward error of each solution
   vector X(j) (i.e., the smallest relative change in
   any element of A or B that makes X(j) an exact solution).

 WORK
   WORK is COMPLEX*16 array, dimension (2*N)

 RWORK
   RWORK is DOUBLE PRECISION array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

   ITMAX is the maximum number of steps of iterative refinement.
```

———-

61.12.3 fortran code

— **zsyrfs.f** —

```
*  =====================================================================
      SUBROUTINE ZSYRFS( UPLO, N, NRHS, A, LDA, AF, LDAF, IPIV, B, LDB,
     $                   X, LDX, FERR, BERR, WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
```

```
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   BERR( * ), FERR( * ), RWORK( * )
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
      COMPLEX*16         ONE
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ) )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D+0 )
      DOUBLE PRECISION   THREE
      PARAMETER          ( THREE = 3.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            COUNT, I, J, K, KASE, NZ
      DOUBLE PRECISION   EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN, XK
      COMPLEX*16         ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZAXPY, ZCOPY, ZLACN2, ZSYMV, ZSYTRS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, MAX
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
```

```
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -10
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -12
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYRFS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = N + 1
      EPS = DLAMCH( 'Epsilon' )
      SAFMIN = DLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 140 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
```

```
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - A * X
*
         CALL ZCOPY( N, B( 1, J ), 1, WORK, 1 )
         CALL ZSYMV( UPLO, N, -ONE, A, LDA, X( 1, J ), 1, ONE, WORK, 1 )
*
*        Compute componentwise relative backward error from formula
*
*        max(i) ( abs(R(i)) / ( abs(A)*abs(X) + abs(B) )(i) )
*
*        where abs(Z) is the componentwise absolute value of the matrix
*        or vector Z.  If the i-th component of the denominator is less
*        than SAFE2, then SAFE1 is added to the i-th components of the
*        numerator and denominator before dividing.
*
         DO 30 I = 1, N
            RWORK( I ) = CABS1( B( I, J ) )
   30    CONTINUE
*
*        Compute abs(A)*abs(X) + abs(B).
*
         IF( UPPER ) THEN
            DO 50 K = 1, N
               S = ZERO
               XK = CABS1( X( K, J ) )
               DO 40 I = 1, K - 1
                  RWORK( I ) = RWORK( I ) + CABS1( A( I, K ) )*XK
                  S = S + CABS1( A( I, K ) )*CABS1( X( I, J ) )
   40          CONTINUE
               RWORK( K ) = RWORK( K ) + CABS1( A( K, K ) )*XK + S
   50       CONTINUE
         ELSE
            DO 70 K = 1, N
               S = ZERO
               XK = CABS1( X( K, J ) )
               RWORK( K ) = RWORK( K ) + CABS1( A( K, K ) )*XK
               DO 60 I = K + 1, N
                  RWORK( I ) = RWORK( I ) + CABS1( A( I, K ) )*XK
                  S = S + CABS1( A( I, K ) )*CABS1( X( I, J ) )
   60          CONTINUE
               RWORK( K ) = RWORK( K ) + S
   70       CONTINUE
         END IF
         S = ZERO
         DO 80 I = 1, N
            IF( RWORK( I ).GT.SAFE2 ) THEN
               S = MAX( S, CABS1( WORK( I ) ) / RWORK( I ) )
            ELSE
               S = MAX( S, ( CABS1( WORK( I ) )+SAFE1 ) /
```

```
     $             ( RWORK( I )+SAFE1 ) )
            END IF
   80    CONTINUE
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
*           1) The residual BERR(J) is larger than machine epsilon, and
*           2) BERR(J) decreased by at least a factor of 2 during the
*              last iteration, and
*           3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
*
*           Update solution and try again.
*
            CALL ZSYTRS( UPLO, N, 1, AF, LDAF, IPIV, WORK, N, INFO )
            CALL ZAXPY( N, ONE, WORK, 1, X( 1, J ), 1 )
            LSTRES = BERR( J )
            COUNT = COUNT + 1
            GO TO 20
         END IF
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(A))*
*           ( abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
*          inv(A) is the inverse of A
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(A)*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(A)*abs(X) + abs(B) is less than SAFE2.
*
*        Use ZLACN2 to estimate the infinity-norm of the matrix
*           inv(A) * diag(W),
*        where W = abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) )))
*
         DO 90 I = 1, N
            IF( RWORK( I ).GT.SAFE2 ) THEN
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I )
            ELSE
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I ) +
```

```
     $                     SAFE1
            END IF
   90    CONTINUE
*
         KASE = 0
  100    CONTINUE
         CALL ZLACN2( N, WORK( N+1 ), WORK, FERR( J ), KASE, ISAVE )
         IF( KASE.NE.0 ) THEN
            IF( KASE.EQ.1 ) THEN
*
*              Multiply by diag(W)*inv(A**T).
*
               CALL ZSYTRS( UPLO, N, 1, AF, LDAF, IPIV, WORK, N, INFO )
               DO 110 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
  110          CONTINUE
            ELSE IF( KASE.EQ.2 ) THEN
*
*              Multiply by inv(A)*diag(W).
*
               DO 120 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
  120          CONTINUE
               CALL ZSYTRS( UPLO, N, 1, AF, LDAF, IPIV, WORK, N, INFO )
            END IF
            GO TO 100
         END IF
*
*        Normalize error.
*
         LSTRES = ZERO
         DO 130 I = 1, N
            LSTRES = MAX( LSTRES, CABS1( X( I, J ) ) )
  130    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  140 CONTINUE
*
      RETURN
*
*     End of ZSYRFS
*
      END
```

———-

61.12.4 lisp code

— zsyrfs.lisp —

```
(let* ((itmax 5)
       (zero 0.0d0)
       (one (f2cl-lib:cmplx 1.0d0 0.0d0))
       (two 2.0d0)
       (three 3.0d0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (f2cl-lib:complex16) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 3.0d0 3.0d0) three)
           (ignorable itmax zero one two three))
  (defun zsyrfs
         (uplo n nrhs a lda af ldaf ipiv b ldb$ x ldx ferr berr work rwork
          info)
    (declare (type (array double-float (*)) rwork berr ferr)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) work x b af a)
             (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (af f2cl-lib:complex16 af-%data% af-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (x f2cl-lib:complex16 x-%data% x-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (ferr double-float ferr-%data% ferr-%offset%)
         (berr double-float berr-%data% berr-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
          cabs1))
        (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
               (zdum #C(0.0d0 0.0d0)) (eps 0.0d0) (lstres 0.0d0) (s 0.0d0)
               (safe1 0.0d0) (safe2 0.0d0) (safmin 0.0d0) (xk 0.0d0) (count$ 0)
               (i 0) (j 0) (k 0) (kase 0) (nz 0) (upper nil))
          (declare (type (array f2cl-lib:integer4 (3)) isave)
                   (type (f2cl-lib:complex16) zdum)
                   (type (double-float) eps lstres s safe1 safe2 safmin xk)
                   (type (f2cl-lib:integer4) count$ i j k kase nz)
                   (type f2cl-lib:logical upper))
          (setf info 0)
```

```
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -10))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -12)))
(cond
 ((/= info 0) (xerbla "ZSYRFS" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
              zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%)
              zero)
     label10))
  (go end_label)))
(setf nz (f2cl-lib:int-add n 1))
(setf eps (dlamch "Epsilon"))
(setf safmin (dlamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
    (zcopy n
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 j)
                           ((1 ldb$) (1 *)) b-%offset%)
     1 work 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (zsymv uplo n (- one) a lda
     (f2cl-lib:array-slice x-%data% f2cl-lib:complex16 (1 j)
                           ((1 ldx) (1 *)) x-%offset%)
     1 one work 1)
  (declare
   (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
    var-9))
  (setf uplo var-0))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
            (cabs1
             (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                            b-%offset%)))
   label30))
(cond
 (upper
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (setf s zero)
      (setf xk
              (cabs1
               (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                              x-%offset%)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (f2cl-lib:int-add k
                                          (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                               rwork-%offset%)
                (+
                 (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                rwork-%offset%)
                 (*
                  (cabs1
                   (f2cl-lib:fref a-%data% (i k)
                                  ((1 lda) (1 *)) a-%offset%))
                  xk)))
```

```
            (setf s
                    (+ s
                       (*
                        (cabs1
                         (f2cl-lib:fref a-%data% (i k)
                                        ((1 lda) (1 *))
                                        a-%offset%))
                        (cabs1
                         (f2cl-lib:fref x-%data% (i j)
                                        ((1 ldx) (1 *))
                                        x-%offset%)))))
          label40))
      (setf (f2cl-lib:fref rwork-%data% (k) ((1 *))
                           rwork-%offset%)
            (+
             (f2cl-lib:fref rwork-%data% (k) ((1 *))
                            rwork-%offset%)
             (*
              (cabs1
               (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                              a-%offset%))
              xk)
             s))
    label50)))
(t
 (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
               ((> k n) nil)
   (tagbody
     (setf s zero)
     (setf xk
             (cabs1
              (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                             x-%offset%)))
     (setf (f2cl-lib:fref rwork-%data% (k) ((1 *))
                          rwork-%offset%)
             (+
              (f2cl-lib:fref rwork-%data% (k) ((1 *))
                             rwork-%offset%)
              (*
               (cabs1
                (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                               a-%offset%))
               xk)))
     (f2cl-lib:fdo (i (f2cl-lib:int-add k 1)
                    (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                              rwork-%offset%)
               (+
```

```
                        (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                       rwork-%offset%)
                        (*
                         (cabs1
                          (f2cl-lib:fref a-%data% (i k)
                                         ((1 lda) (1 *)) a-%offset%))
                         xk)))
                (setf s
                       (+ s
                          (*
                           (cabs1
                            (f2cl-lib:fref a-%data% (i k)
                                           ((1 lda) (1 *))
                                           a-%offset%))
                           (cabs1
                            (f2cl-lib:fref x-%data% (i j)
                                           ((1 ldx) (1 *))
                                           x-%offset%)))))
            label60))
        (setf (f2cl-lib:fref rwork-%data% (k) ((1 *))
                             rwork-%offset%)
              (+
               (f2cl-lib:fref rwork-%data% (k) ((1 *))
                              rwork-%offset%)
               s))
      label70))))
(setf s zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
      (setf s
              (max s
                   (/
                    (cabs1
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%))
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%)))))
     (t
      (setf s
              (max s
                   (/
                    (+
                     (cabs1
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%))
                     safe1)
                    (+
```

```
                        (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                       rwork-%offset%)
                        safe1))))))
   label80))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zsytrs uplo n 1 af ldaf ipiv work n info)
    (declare (ignore var-2 var-3 var-5 var-6))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf ldaf var-4))
    (when var-7 (setf n var-7))
    (when var-8 (setf info var-8)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (zaxpy n one work 1
       (f2cl-lib:array-slice x-%data% f2cl-lib:complex16 (1 j)
                             ((1 ldx) (1 *)) x-%offset%)
       1)
    (declare (ignore var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))
    (when var-1 (setf one var-1)))
  (setf lstres
          (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
  (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
      (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                           rwork-%offset%)
              (+
               (cabs1
                (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%))
               (* nz eps
                  (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                 rwork-%offset%)))))
     (t
      (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                           rwork-%offset%)
              (+
               (cabs1
                (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%))
```

```
                    (* nz eps
                       (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                      rwork-%offset%))
                    safe1))))
     label90))
  (setf kase 0)
 label100
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (zlacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             ((+ n 1)) ((1 *)) work-%offset%)
       work (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
       kase isave)
    (declare (ignore var-1 var-2 var-5))
    (setf n var-0)
    (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
            var-3)
    (setf kase var-4))
  (cond
   ((/= kase 0)
    (cond
     ((= kase 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (zsytrs uplo n 1 af ldaf ipiv work n info)
        (declare (ignore var-2 var-3 var-5 var-6))
        (when var-0 (setf uplo var-0))
        (when var-1 (setf n var-1))
        (when var-4 (setf ldaf var-4))
        (when var-7 (setf n var-7))
        (when var-8 (setf info var-8)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                (*
                 (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                rwork-%offset%)
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)))
         label110)))
     ((= kase 2)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                (*
                 (f2cl-lib:fref rwork-%data% (i) ((1 *))
```

```
                                          rwork-%offset%)
                              (f2cl-lib:fref work-%data% (i) ((1 *))
                                          work-%offset%)))
                     label120))
                  (multiple-value-bind
                      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                      (zsytrs uplo n 1 af ldaf ipiv work n info)
                    (declare (ignore var-2 var-3 var-5 var-6))
                    (when var-0 (setf uplo var-0))
                    (when var-1 (setf n var-1))
                    (when var-4 (setf ldaf var-4))
                    (when var-7 (setf n var-7))
                    (when var-8 (setf info var-8)))))
                (go label100)))
              (setf lstres zero)
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i n) nil)
                (tagbody
                  (setf lstres
                          (max lstres
                               (cabs1
                                (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                               x-%offset%))))
                 label130))
              (if (/= lstres zero)
                  (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                          (/
                           (f2cl-lib:fref ferr-%data% (j) ((1 *))
                                          ferr-%offset%)
                           lstres)))
            label140))
          (go end_label)
         end_label
          (return
           (values uplo n nil nil nil nil ldaf nil nil nil nil nil nil nil nil
                   nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsyrfs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
```

```
  (*))
 (fortran-to-lisp::integer4)
 (array fortran-to-lisp::integer4
  (*))
 (array fortran-to-lisp::complex16
  (*))
 (fortran-to-lisp::integer4)
 (array fortran-to-lisp::complex16
  (*))
 (fortran-to-lisp::integer4)
 (array double-float (*))
 (array double-float (*))
 (array fortran-to-lisp::complex16
  (*))
 (array double-float (*))
 (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::uplo
  fortran-to-lisp::n nil nil nil
  nil fortran-to-lisp::ldaf nil nil
  nil nil nil nil nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zlacn2
  fortran-to-lisp::zsymv
  fortran-to-lisp::xerbla))))
```

61.13 zsyrfsx

61.13.1 Axiom unit tests

— zsyrfsx.input —

```
)set break resume
)sys rm -f zsyrfsx.output
)spool zsyrfsx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.13.2 Axiom help page [Demm05]

— zsyrfsx.help —

```
==========================================================================
zsyrfsx examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zsyrfsx (UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF, IPIV, S, B,
           LDB, X, LDX, RCOND, BERR, N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP,
           NPARAMS, PARAMS, WORK, RWORK, INFO)
           ZSYRFSX

Function/Subroutine Documentation
   subroutine zsyrfsx (character UPLO, character EQUED, integer N, integer
       NRHS, complex*16, dimension( lda, * ) A, integer LDA, complex*16,
       dimension( ldaf, * ) AF, integer LDAF, integer, dimension( * ) IPIV,
       double precision, dimension( * ) S, complex*16, dimension( ldb, * ) B,
       integer LDB, complex*16, dimension( ldx, * ) X, integer LDX, double
       precision RCOND, double precision, dimension( * ) BERR, integer
       N_ERR_BNDS, double precision, dimension( nrhs, * ) ERR_BNDS_NORM,
       double precision, dimension( nrhs, * ) ERR_BNDS_COMP, integer NPARAMS,
       double precision, dimension( * ) PARAMS, complex*16, dimension( * )
       WORK, double precision, dimension( * ) RWORK, integer INFO)
       ZSYRFSX

Purpose:
   ZSYRFSX improves the computed solution to a system of linear
   equations when the coefficient matrix is symmetric indefinite, and
   provides error bounds and backward error estimates for the
   solution.  In addition to normwise error bound, the code provides
   maximum componentwise error bound if possible.  See comments for
   ERR_BNDS_NORM and ERR_BNDS_COMP for details of the error bounds.

   The original system of linear equations may have been equilibrated
   before calling this routine, as described by arguments EQUED and S
   below. In this case, the solution and error bounds returned are
   for the original unequilibrated system.

   Some optional parameters are bundled in the PARAMS array.  These
```

```
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done to A
   before calling this routine. This is needed to compute
   the solution and error bounds correctly.
   = 'N':  No equilibration
   = 'Y':  Both row and column equilibration, i.e., A has been
           replaced by diag(S) * A * diag(S).
           The right hand side B has been changed accordingly.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
   upper triangular part of A contains the upper triangular
   part of the matrix A, and the strictly lower triangular
   part of A is not referenced.  If UPLO = 'L', the leading
   N-by-N lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is COMPLEX*16 array, dimension (LDAF,N)
   The factored form of the matrix A.  AF contains the block
   diagonal matrix D and the multipliers used to obtain the
   factor U or L from the factorization A = U*D*U**T or A =
```

```
  L*D*L**T as computed by DSYTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D
  as determined by DSYTRF.

S
  S is DOUBLE PRECISION array, dimension (N)
  The scale factors for A.  If EQUED = 'Y', A is multiplied on
  the left and right by diag(S).  S is an input argument if FACT =
  'F'; otherwise, S is an output argument.  If FACT = 'F' and EQUED
  = 'Y', each element of S must be positive.  If S is output, each
  element of S is a power of the radix. If S is input, each element
  of S should be a power of the radix to ensure a reliable solution
  and error estimates. Scaling by powers of the radix does not cause
  rounding errors unless the result underflows or overflows.
  Rounding errors during scaling lead to refining with a matrix that
  is not equivalent to the input matrix, producing error estimates
  that may not be reliable.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by DGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.
```

```
BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
            max_j (abs(XTRUE(j,i) - X(j,i)))
             ------------------------------
                    max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
    reciprocal condition number is less than the threshold
    sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
    almost certainly within a factor of 10 of the true error
    so long as the next entry is greater than the threshold
    sqrt(n) * dlamch('Epsilon'). This error bound should only
    be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
    reciprocal condition number.  Compared with the threshold
    sqrt(n) * dlamch('Epsilon') to determine if the error
    estimate is "guaranteed". These reciprocal condition
    numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
```

```
    appropriately scaled matrix Z.
    Let Z = S*A, where S scales each row by a power of the
    radix so all absolute row sums of Z are approximately 1.

ERR_BNDS_COMP
  ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                abs(XTRUE(j,i) - X(j,i))
           max_j ----------------------
                        abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
    reciprocal condition number is less than the threshold
    sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
    almost certainly within a factor of 10 of the true error
    so long as the next entry is greater than the threshold
    sqrt(n) * dlamch('Epsilon'). This error bound should only
    be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
    reciprocal condition number.  Compared with the threshold
    sqrt(n) * dlamch('Epsilon') to determine if the error
    estimate is "guaranteed". These reciprocal condition
    numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
    appropriately scaled matrix Z.
    Let Z = S*(A*diag(x)), where x is the solution for the
    current right-hand side and S scales each row of
    A*diag(x) by a power of the radix so all absolute row
    sums of Z are approximately 1.
```

```
NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is DOUBLE PRECISION array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0D+0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
  (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is COMPLEX*16 array, dimension (2*N)

RWORK
  RWORK is DOUBLE PRECISION array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is
  guaranteed.
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
     has been completed, but the factor U is exactly singular, so
     the solution and error bounds could not be computed. RCOND = 0
```

```
      is returned.
   = N+J: The solution corresponding to the Jth right-hand side is
      not guaranteed. The solutions corresponding to other right-
      hand sides K with K > J may not be guaranteed as well, but
      only the first such right-hand side is reported. If a small
      componentwise error is not requested (PARAMS(3) = 0.0) then
      the Jth right-hand side is the first with a normwise error
      bound that is not guaranteed (the smallest J such
      that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
      the Jth right-hand side is the first with either a normwise or
      componentwise error bound that is not guaranteed (the smallest
      J such that either ERR_BNDS_NORM(J,1) = 0.0 or
      ERR_BNDS_COMP(J,1) = 0.0). See the definition of
      ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
      about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

———-

61.13.3 fortran code

— **zsyrfsx.f** —

```
*  ===================================================================
      SUBROUTINE ZSYRFSX( UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF, IPIV,
     $                    S, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS,
     $                    ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $                    WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO, EQUED
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX, * ), WORK( * )
      DOUBLE PRECISION   S( * ), PARAMS( * ), BERR( * ), RWORK( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
```

```
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   ITREF_DEFAULT, ITHRESH_DEFAULT
      DOUBLE PRECISION   COMPONENTWISE_DEFAULT, RTHRESH_DEFAULT
      DOUBLE PRECISION   DZTHRESH_DEFAULT
      PARAMETER          ( ITREF_DEFAULT = 1.0D+0 )
      PARAMETER          ( ITHRESH_DEFAULT = 10.0D+0 )
      PARAMETER          ( COMPONENTWISE_DEFAULT = 1.0D+0 )
      PARAMETER          ( RTHRESH_DEFAULT = 0.5D+0 )
      PARAMETER          ( DZTHRESH_DEFAULT = 0.25D+0 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. Local Scalars ..
      CHARACTER(1)       NORM
      LOGICAL            RCEQU
      INTEGER            J, PREC_TYPE, REF_TYPE
      INTEGER            N_NORMS
      DOUBLE PRECISION   ANORM, RCOND_TMP
      DOUBLE PRECISION   ILLRCOND_THRESH, ERR_LBND, CWISE_WRONG
      LOGICAL            IGNORE_CWISE
      INTEGER            ITHRESH
      DOUBLE PRECISION   RTHRESH, UNSTABLE_THRESH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZSYCON, ZLA_SYRFSX_EXTENDED
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT, TRANSFER
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, BLAS_FPINFO_X, ILATRANS, ILAPREC
      EXTERNAL           DLAMCH, ZLANSY, ZLA_SYRCOND_X, ZLA_SYRCOND_C
      DOUBLE PRECISION   DLAMCH, ZLANSY, ZLA_SYRCOND_X, ZLA_SYRCOND_C
      LOGICAL            LSAME
      INTEGER            BLAS_FPINFO_X
      INTEGER            ILATRANS, ILAPREC
*     ..
*     .. Executable Statements ..
```

```
*
*     Check the input parameters.
*
      INFO = 0
      REF_TYPE = INT( ITREF_DEFAULT )
      IF ( NPARAMS .GE. LA_LINRX_ITREF_I ) THEN
         IF ( PARAMS( LA_LINRX_ITREF_I ) .LT. 0.0D+0 ) THEN
            PARAMS( LA_LINRX_ITREF_I ) = ITREF_DEFAULT
         ELSE
            REF_TYPE = PARAMS( LA_LINRX_ITREF_I )
         END IF
      END IF
*
*     Set default parameters.
*
      ILLRCOND_THRESH = DBLE( N ) * DLAMCH( 'Epsilon' )
      ITHRESH = INT( ITHRESH_DEFAULT )
      RTHRESH = RTHRESH_DEFAULT
      UNSTABLE_THRESH = DZTHRESH_DEFAULT
      IGNORE_CWISE = COMPONENTWISE_DEFAULT .EQ. 0.0D+0
*
      IF ( NPARAMS.GE.LA_LINRX_ITHRESH_I ) THEN
         IF ( PARAMS( LA_LINRX_ITHRESH_I ).LT.0.0D+0 ) THEN
            PARAMS( LA_LINRX_ITHRESH_I ) = ITHRESH
         ELSE
            ITHRESH = INT( PARAMS( LA_LINRX_ITHRESH_I ) )
         END IF
      END IF
      IF ( NPARAMS.GE.LA_LINRX_CWISE_I ) THEN
         IF ( PARAMS( LA_LINRX_CWISE_I ).LT.0.0D+0 ) THEN
            IF ( IGNORE_CWISE ) THEN
               PARAMS( LA_LINRX_CWISE_I ) = 0.0D+0
            ELSE
               PARAMS( LA_LINRX_CWISE_I ) = 1.0D+0
            END IF
         ELSE
            IGNORE_CWISE = PARAMS( LA_LINRX_CWISE_I ) .EQ. 0.0D+0
         END IF
      END IF
      IF ( REF_TYPE .EQ. 0 .OR. N_ERR_BNDS .EQ. 0 ) THEN
         N_NORMS = 0
      ELSE IF ( IGNORE_CWISE ) THEN
         N_NORMS = 1
      ELSE
         N_NORMS = 2
      END IF
*
      RCEQU = LSAME( EQUED, 'Y' )
*
*     Test input parameters.
```

```
*
      IF ( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
        INFO = -1
      ELSE IF( .NOT.RCEQU .AND. .NOT.LSAME( EQUED, 'N' ) ) THEN
        INFO = -2
      ELSE IF( N.LT.0 ) THEN
        INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
        INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
        INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
        INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
        INFO = -12
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
        INFO = -14
      END IF
      IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'ZSYRFSX', -INFO )
        RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         RCOND = 1.0D+0
         DO J = 1, NRHS
            BERR( J ) = 0.0D+0
            IF ( N_ERR_BNDS .GE. 1 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
            IF ( N_ERR_BNDS .GE. 2 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 0.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 0.0D+0
            END IF
            IF ( N_ERR_BNDS .GE. 3 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 1.0D+0
            END IF
         END DO
         RETURN
      END IF
*
*     Default to failure.
*
      RCOND = 0.0D+0
      DO J = 1, NRHS
         BERR( J ) = 1.0D+0
```

```
         IF ( N_ERR_BNDS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
         END IF
         IF ( N_ERR_BNDS .GE. 2 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
         END IF
         IF ( N_ERR_BNDS .GE. 3 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 0.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 0.0D+0
         END IF
      END DO
*
*     Compute the norm of A and the reciprocal of the condition
*     number of A.
*
      NORM = 'I'
      ANORM = ZLANSY( NORM, UPLO, N, A, LDA, RWORK )
      CALL ZSYCON( UPLO, N, AF, LDAF, IPIV, ANORM, RCOND, WORK,
     $     INFO )
*
*     Perform refinement on each right-hand side
*
      IF ( REF_TYPE .NE. 0 ) THEN

         PREC_TYPE = ILAPREC( 'E' )

         CALL ZLA_SYRFSX_EXTENDED( PREC_TYPE, UPLO,  N,
     $        NRHS, A, LDA, AF, LDAF, IPIV, RCEQU, S, B,
     $        LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM, ERR_BNDS_COMP,
     $        WORK, RWORK, WORK(N+1),
     $        TRANSFER (RWORK(1:2*N), (/ (ZERO, ZERO) /), N), RCOND,
     $        ITHRESH, RTHRESH, UNSTABLE_THRESH, IGNORE_CWISE,
     $        INFO )
      END IF

      ERR_LBND = MAX( 10.0D+0, SQRT( DBLE( N ) ) ) * DLAMCH( 'Epsilon' )
      IF (N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 1) THEN
*
*     Compute scaled normwise condition number cond(A*C).
*
         IF ( RCEQU ) THEN
            RCOND_TMP = ZLA_SYRCOND_C( UPLO, N, A, LDA, AF, LDAF, IPIV,
     $           S, .TRUE., INFO, WORK, RWORK )
         ELSE
            RCOND_TMP = ZLA_SYRCOND_C( UPLO, N, A, LDA, AF, LDAF, IPIV,
     $           S, .FALSE., INFO, WORK, RWORK )
         END IF
         DO J = 1, NRHS
```

```
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .GT. 1.0D+0 )
     $           ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 0.0D+0
               IF ( INFO .LE. N ) INFO = N + J
            ELSE IF ( ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .LT. ERR_LBND )
     $              THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF
         END DO
      END IF

      IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 2 ) THEN
*
*     Compute componentwise condition number cond(A*diag(Y(:,J))) for
*     each right-hand side using the current solution as an estimate of
*     the true solution.  If the componentwise error estimate is too
*     large, then the solution is a lousy estimate of truth and the
*     estimated RCOND may be too optimistic.  To avoid misleading users,
*     the inverse condition number is set to 0.0 when the estimated
*     cwise error is at least CWISE_WRONG.
*
         CWISE_WRONG = SQRT( DLAMCH( 'Epsilon' ) )
         DO J = 1, NRHS
            IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .LT. CWISE_WRONG )
     $     THEN
               RCOND_TMP = ZLA_SYRCOND_X( UPLO, N, A, LDA, AF, LDAF,
     $         IPIV, X(1,J), INFO, WORK, RWORK )
            ELSE
               RCOND_TMP = 0.0D+0
            END IF
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
```

```
     $           .AND. ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .GT. 1.0D+0 )
     $           ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0

*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 0.0D+0
               IF (.NOT. IGNORE_CWISE
     $              .AND. INFO.LT.N + J ) INFO = N + J
            ELSE IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I )
     $              .LT. ERR_LBND ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF

         END DO
      END IF
*
      RETURN
*
*     End of ZSYRFSX
*
      END
```

———-

61.13.4 lisp code

— **zsyrfsx.lisp** —

———-

61.14 zsytf2

61.14.1 Axiom unit tests

— zsytf2.input —

```
)set break resume
)sys rm -f zsytf2.output
)spool zsytf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.14.2 Axiom help page

— zsytf2.help —

```
======================================================================
zsytf2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zsytf2 (UPLO, N, A, LDA, IPIV, INFO)
           ZSYTF2 computes the factorization of a real symmetric indefinite
           matrix, using the diagonal pivoting method (unblocked algorithm).

Function/Subroutine Documentation
   subroutine zsytf2 (character UPLO, integer N, complex*16, dimension( lda, *
       ) A, integer LDA, integer, dimension( * ) IPIV, integer INFO)
       ZSYTF2 computes the factorization of a real symmetric indefinite
       matrix, using the diagonal pivoting method (unblocked algorithm).

Purpose:
   ZSYTF2 computes the factorization of a complex symmetric matrix A
   using the Bunch-Kaufman diagonal pivoting method:
```

```
   A = U*D*U**T  or  A = L*D*L**T

   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, U**T is the transpose of U, and D is symmetric and
   block diagonal with 1-by-1 and 2-by-2 diagonal blocks.

   This is the unblocked version of the algorithm, calling Level 2 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is stored:
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   n-by-n upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading n-by-n lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

   On exit, the block diagonal matrix D and the multipliers used
   to obtain the factor U or L (see below for further details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D.

   If UPLO = 'U':
     If IPIV(k) > 0, then rows and columns k and IPIV(k) were
     interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) = IPIV(k-1) < 0, then rows and columns
     k-1 and -IPIV(k) were interchanged and D(k-1:k,k-1:k)
     is a 2-by-2 diagonal block.
```

```
   If UPLO = 'L':
     If IPIV(k) > 0, then rows and columns k and IPIV(k) were
     interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) = IPIV(k+1) < 0, then rows and columns
     k+1 and -IPIV(k) were interchanged and D(k:k+1,k:k+1)
     is a 2-by-2 diagonal block.

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -k, the k-th argument had an illegal value
   > 0: if INFO = k, D(k,k) is exactly zero.  The factorization
      has been completed, but the block diagonal matrix D is
      exactly singular, and division by zero will occur if it
      is used to solve a system of equations.

Further Details:

   If UPLO = 'U', then A = U*D*U**T, where
      U = P(n)*U(n)* ... *P(k)U(k)* ...,
   i.e., U is a product of terms P(k)*U(k), where k decreases from n to
   1 in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and U(k) is a unit upper triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                       (   I    v    0   )   k-s
              U(k) =   (   0    I    0   )   s
                       (   0    0    I   )   n-k
                          k-s   s   n-k

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(1:k-1,k).
   If s = 2, the upper triangle of D(k) overwrites A(k-1,k-1), A(k-1,k),
   and A(k,k), and v overwrites A(1:k-2,k-1:k).

   If UPLO = 'L', then A = L*D*L**T, where
      L = P(1)*L(1)* ... *P(k)*L(k)* ...,
   i.e., L is a product of terms P(k)*L(k), where k increases from 1 to
   n in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and L(k) is a unit lower triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                       (   I    0     0   )  k-1
              L(k) =   (   0    I     0   )  s
                       (   0    v     I   )  n-k-s+1
                          k-1   s  n-k-s+1

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(k+1:n,k).
```

```
If s = 2, the lower triangle of D(k) overwrites A(k,k), A(k+1,k),
and A(k+1,k+1), and v overwrites A(k+2:n,k:k+1).
```

———-

61.14.3 fortran code

— **zsytf2.f** —

```
*  =====================================================================
      SUBROUTINE ZSYTF2( UPLO, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   EIGHT, SEVTEN
      PARAMETER          ( EIGHT = 8.0D+0, SEVTEN = 17.0D+0 )
      COMPLEX*16         CONE
      PARAMETER          ( CONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, IMAX, J, JMAX, K, KK, KP, KSTEP
      DOUBLE PRECISION   ABSAKK, ALPHA, COLMAX, ROWMAX
      COMPLEX*16         D11, D12, D21, D22, R1, T, WK, WKM1, WKP1, Z
*     ..
*     .. External Functions ..
      LOGICAL            DISNAN, LSAME
      INTEGER            IZAMAX
      EXTERNAL           DISNAN, LSAME, IZAMAX
*     ..
```

```
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZSCAL, ZSWAP, ZSYR
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, MAX, SQRT
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( Z ) = ABS( DBLE( Z ) ) + ABS( DIMAG( Z ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYTF2', -INFO )
         RETURN
      END IF
*
*     Initialize ALPHA for use in choosing pivot block size.
*
      ALPHA = ( ONE+SQRT( SEVTEN ) ) / EIGHT
*
      IF( UPPER ) THEN
*
*        Factorize A as U*D*U**T using the upper triangle of A
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        1 or 2
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop
*
         IF( K.LT.1 )
     $      GO TO 70
         KSTEP = 1
*
```

```
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = CABS1( A( K, K ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.GT.1 ) THEN
            IMAX = IZAMAX( K-1, A( 1, K ), 1 )
            COLMAX = CABS1( A( IMAX, K ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( MAX( ABSAKK, COLMAX ).EQ.ZERO .OR. DISNAN(ABSAKK) ) THEN
*
*           Column K is zero or underflow, or contains a NaN:
*           set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
         ELSE
            IF( ABSAKK.GE.ALPHA*COLMAX ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
            ELSE
*
*              JMAX is the column-index of the largest off-diagonal
*              element in row IMAX, and ROWMAX is its absolute value
*
               JMAX = IMAX + IZAMAX( K-IMAX, A( IMAX, IMAX+1 ), LDA )
               ROWMAX = CABS1( A( IMAX, JMAX ) )
               IF( IMAX.GT.1 ) THEN
                  JMAX = IZAMAX( IMAX-1, A( 1, IMAX ), 1 )
                  ROWMAX = MAX( ROWMAX, CABS1( A( JMAX, IMAX ) ) )
               END IF
*
               IF( ABSAKK.GE.ALPHA*COLMAX*( COLMAX / ROWMAX ) ) THEN
*
*                 no interchange, use 1-by-1 pivot block
*
                  KP = K
               ELSE IF( CABS1( A( IMAX, IMAX ) ).GE.ALPHA*ROWMAX ) THEN
*
*                 interchange rows and columns K and IMAX, use 1-by-1
```

```
*                 pivot block
*
                  KP = IMAX
               ELSE
*
*                 interchange rows and columns K-1 and IMAX, use 2-by-2
*                 pivot block
*
                  KP = IMAX
                  KSTEP = 2
               END IF
            END IF
*
            KK = K - KSTEP + 1
            IF( KP.NE.KK ) THEN
*
*              Interchange rows and columns KK and KP in the leading
*              submatrix A(1:k,1:k)
*
               CALL ZSWAP( KP-1, A( 1, KK ), 1, A( 1, KP ), 1 )
               CALL ZSWAP( KK-KP-1, A( KP+1, KK ), 1, A( KP, KP+1 ),
     $                     LDA )
               T = A( KK, KK )
               A( KK, KK ) = A( KP, KP )
               A( KP, KP ) = T
               IF( KSTEP.EQ.2 ) THEN
                  T = A( K-1, K )
                  A( K-1, K ) = A( KP, K )
                  A( KP, K ) = T
               END IF
            END IF
*
*           Update the leading submatrix
*
            IF( KSTEP.EQ.1 ) THEN
*
*              1-by-1 pivot block D(k): column k now holds
*
*              W(k) = U(k)*D(k)
*
*              where U(k) is the k-th column of U
*
*              Perform a rank-1 update of A(1:k-1,1:k-1) as
*
*              A := A - U(k)*D(k)*U(k)**T = A - W(k)*1/D(k)*W(k)**T
*
               R1 = CONE / A( K, K )
               CALL ZSYR( UPLO, K-1, -R1, A( 1, K ), 1, A, LDA )
*
*              Store U(k) in column k
```

```
*
               CALL ZSCAL( K-1, R1, A( 1, K ), 1 )
            ELSE
*
*              2-by-2 pivot block D(k): columns k and k-1 now hold
*
*              ( W(k-1) W(k) ) = ( U(k-1) U(k) )*D(k)
*
*              where U(k) and U(k-1) are the k-th and (k-1)-th columns
*              of U
*
*              Perform a rank-2 update of A(1:k-2,1:k-2) as
*
*              A := A - ( U(k-1) U(k) )*D(k)*( U(k-1) U(k) )**T
*                 = A - ( W(k-1) W(k) )*inv(D(k))*( W(k-1) W(k) )**T
*
               IF( K.GT.2 ) THEN
*
                  D12 = A( K-1, K )
                  D22 = A( K-1, K-1 ) / D12
                  D11 = A( K, K ) / D12
                  T = CONE / ( D11*D22-CONE )
                  D12 = T / D12
*
                  DO 30 J = K - 2, 1, -1
                     WKM1 = D12*( D11*A( J, K-1 )-A( J, K ) )
                     WK = D12*( D22*A( J, K )-A( J, K-1 ) )
                     DO 20 I = J, 1, -1
                        A( I, J ) = A( I, J ) - A( I, K )*WK -
     $                              A( I, K-1 )*WKM1
   20                CONTINUE
                     A( J, K ) = WK
                     A( J, K-1 ) = WKM1
   30             CONTINUE
*
               END IF
*
            END IF
         END IF
*
*        Store details of the interchanges in IPIV
*
         IF( KSTEP.EQ.1 ) THEN
            IPIV( K ) = KP
         ELSE
            IPIV( K ) = -KP
            IPIV( K-1 ) = -KP
         END IF
*
*        Decrease K and return to the start of the main loop
```

```
*
         K = K - KSTEP
         GO TO 10
*
      ELSE
*
*        Factorize A as L*D*L**T using the lower triangle of A
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2
*
         K = 1
   40    CONTINUE
*
*        If K > N, exit from loop
*
         IF( K.GT.N )
     $      GO TO 70
         KSTEP = 1
*
*        Determine rows and columns to be interchanged and whether
*        a 1-by-1 or 2-by-2 pivot block will be used
*
         ABSAKK = CABS1( A( K, K ) )
*
*        IMAX is the row-index of the largest off-diagonal element in
*        column K, and COLMAX is its absolute value.
*        Determine both COLMAX and IMAX.
*
         IF( K.LT.N ) THEN
            IMAX = K + IZAMAX( N-K, A( K+1, K ), 1 )
            COLMAX = CABS1( A( IMAX, K ) )
         ELSE
            COLMAX = ZERO
         END IF
*
         IF( MAX( ABSAKK, COLMAX ).EQ.ZERO .OR. DISNAN(ABSAKK) ) THEN
*
*           Column K is zero or underflow, or contains a NaN:
*           set INFO and continue
*
            IF( INFO.EQ.0 )
     $         INFO = K
            KP = K
         ELSE
            IF( ABSAKK.GE.ALPHA*COLMAX ) THEN
*
*              no interchange, use 1-by-1 pivot block
*
               KP = K
```

```
            ELSE
*
*              JMAX is the column-index of the largest off-diagonal
*              element in row IMAX, and ROWMAX is its absolute value
*
               JMAX = K - 1 + IZAMAX( IMAX-K, A( IMAX, K ), LDA )
               ROWMAX = CABS1( A( IMAX, JMAX ) )
               IF( IMAX.LT.N ) THEN
                  JMAX = IMAX + IZAMAX( N-IMAX, A( IMAX+1, IMAX ), 1 )
                  ROWMAX = MAX( ROWMAX, CABS1( A( JMAX, IMAX ) ) )
               END IF
*
               IF( ABSAKK.GE.ALPHA*COLMAX*( COLMAX / ROWMAX ) ) THEN
*
*                 no interchange, use 1-by-1 pivot block
*
                  KP = K
               ELSE IF( CABS1( A( IMAX, IMAX ) ).GE.ALPHA*ROWMAX ) THEN
*
*                 interchange rows and columns K and IMAX, use 1-by-1
*                 pivot block
*
                  KP = IMAX
               ELSE
*
*                 interchange rows and columns K+1 and IMAX, use 2-by-2
*                 pivot block
*
                  KP = IMAX
                  KSTEP = 2
               END IF
            END IF
*
            KK = K + KSTEP - 1
            IF( KP.NE.KK ) THEN
*
*              Interchange rows and columns KK and KP in the trailing
*              submatrix A(k:n,k:n)
*
               IF( KP.LT.N )
     $            CALL ZSWAP( N-KP, A( KP+1, KK ), 1, A( KP+1, KP ), 1 )
               CALL ZSWAP( KP-KK-1, A( KK+1, KK ), 1, A( KP, KK+1 ),
     $                     LDA )
               T = A( KK, KK )
               A( KK, KK ) = A( KP, KP )
               A( KP, KP ) = T
               IF( KSTEP.EQ.2 ) THEN
                  T = A( K+1, K )
                  A( K+1, K ) = A( KP, K )
                  A( KP, K ) = T
```

```
            END IF
         END IF
*
*        Update the trailing submatrix
*
         IF( KSTEP.EQ.1 ) THEN
*
*           1-by-1 pivot block D(k): column k now holds
*
*           W(k) = L(k)*D(k)
*
*           where L(k) is the k-th column of L
*
            IF( K.LT.N ) THEN
*
*              Perform a rank-1 update of A(k+1:n,k+1:n) as
*
*              A := A - L(k)*D(k)*L(k)**T = A - W(k)*(1/D(k))*W(k)**T
*
               R1 = CONE / A( K, K )
               CALL ZSYR( UPLO, N-K, -R1, A( K+1, K ), 1,
     $                    A( K+1, K+1 ), LDA )
*
*              Store L(k) in column K
*
               CALL ZSCAL( N-K, R1, A( K+1, K ), 1 )
            END IF
         ELSE
*
*           2-by-2 pivot block D(k)
*
            IF( K.LT.N-1 ) THEN
*
*              Perform a rank-2 update of A(k+2:n,k+2:n) as
*
*              A := A - ( L(k) L(k+1) )*D(k)*( L(k) L(k+1) )**T
*                 = A - ( W(k) W(k+1) )*inv(D(k))*( W(k) W(k+1) )**T
*
*              where L(k) and L(k+1) are the k-th and (k+1)-th
*              columns of L
*
               D21 = A( K+1, K )
               D11 = A( K+1, K+1 ) / D21
               D22 = A( K, K ) / D21
               T = CONE / ( D11*D22-CONE )
               D21 = T / D21
*
               DO 60 J = K + 2, N
                  WK = D21*( D11*A( J, K )-A( J, K+1 ) )
                  WKP1 = D21*( D22*A( J, K+1 )-A( J, K ) )
```

```
                  DO 50 I = J, N
                     A( I, J ) = A( I, J ) - A( I, K )*WK -
     $                           A( I, K+1 )*WKP1
   50             CONTINUE
                  A( J, K ) = WK
                  A( J, K+1 ) = WKP1
   60           CONTINUE
             END IF
          END IF
       END IF
*
*      Store details of the interchanges in IPIV
*
       IF( KSTEP.EQ.1 ) THEN
          IPIV( K ) = KP
       ELSE
          IPIV( K ) = -KP
          IPIV( K+1 ) = -KP
       END IF
*
*      Increase K and return to the start of the main loop
*
       K = K + KSTEP
       GO TO 40
*
     END IF
*
   70 CONTINUE
     RETURN
*
*    End of ZSYTF2
*
     END
```

———-

61.14.4 lisp code

— **zsytf2.lisp** —

———-

61.15 zsytf2_rook

61.15.1 Axiom unit tests

— zsytf2rook.input —

```
)set break resume
)sys rm -f zsytf2rook.output
)spool zsytf2rook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.15.2 Axiom help page

— zsytf2_rook.help —

```
======================================================================
zsytf2_rook examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zsytf2_rook (UPLO, N, A, LDA, IPIV, INFO)
           ZSYTF2_ROOK computes the factorization of a complex symmetric
           indefinite matrix using the bounded Bunch-Kaufman ('rook') diagonal
           pivoting method (unblocked algorithm).

Function/Subroutine Documentation
   subroutine zsytf2_rook (character UPLO, integer N, complex*16, dimension(
       lda, * ) A, integer LDA, integer, dimension( * ) IPIV, integer INFO)
       ZSYTF2_ROOK computes the factorization of a complex symmetric
       indefinite matrix using the bounded Bunch-Kaufman ('rook') diagonal
       pivoting method (unblocked algorithm).

Purpose:
   ZSYTF2_ROOK computes the factorization of a complex symmetric matrix A
```

```
  using the bounded Bunch-Kaufman ("rook") diagonal pivoting method:

  A = U*D*U**T  or  A = L*D*L**T

  where U (or L) is a product of permutation and unit upper (lower)
  triangular matrices, U**T is the transpose of U, and D is symmetric and
  block diagonal with 1-by-1 and 2-by-2 diagonal blocks.

  This is the unblocked version of the algorithm, calling Level 2 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is stored:
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   n-by-n upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading n-by-n lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

   On exit, the block diagonal matrix D and the multipliers used
   to obtain the factor U or L (see below for further details).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D.

   If UPLO = 'U':
     If IPIV(k) > 0, then rows and columns k and IPIV(k)
     were interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) < 0 and IPIV(k-1) < 0, then rows and
     columns k and -IPIV(k) were interchanged and rows and
```

```
     columns k-1 and -IPIV(k-1) were inerchaged,
     D(k-1:k,k-1:k) is a 2-by-2 diagonal block.

   If UPLO = 'L':
     If IPIV(k) > 0, then rows and columns k and IPIV(k)
     were interchanged and D(k,k) is a 1-by-1 diagonal block.

     If IPIV(k) < 0 and IPIV(k+1) < 0, then rows and
     columns k and -IPIV(k) were interchanged and rows and
     columns k+1 and -IPIV(k+1) were inerchaged,
     D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -k, the k-th argument had an illegal value
   > 0: if INFO = k, D(k,k) is exactly zero.  The factorization
      has been completed, but the block diagonal matrix D is
      exactly singular, and division by zero will occur if it
      is used to solve a system of equations.

Further Details:

   If UPLO = 'U', then A = U*D*U**T, where
     U = P(n)*U(n)* ... *P(k)U(k)* ...,
   i.e., U is a product of terms P(k)*U(k), where k decreases from n to
   1 in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and U(k) is a unit upper triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                        (   I    v    0   )   k-s
                U(k) =  (   0    I    0   )   s
                        (   0    0    I   )   n-k
                           k-s   s   n-k

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(1:k-1,k).
   If s = 2, the upper triangle of D(k) overwrites A(k-1,k-1), A(k-1,k),
   and A(k,k), and v overwrites A(1:k-2,k-1:k).

   If UPLO = 'L', then A = L*D*L**T, where
      L = P(1)*L(1)* ... *P(k)*L(k)* ...,
   i.e., L is a product of terms P(k)*L(k), where k increases from 1 to
   n in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and L(k) is a unit lower triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                        (   I    0     0   )  k-1
                L(k) =  (   0    I     0   )  s
```

```
                    (   0    v     I   )  n-k-s+1
                       k-1   s  n-k-s+1

If s = 1, D(k) overwrites A(k,k), and v overwrites A(k+1:n,k).
If s = 2, the lower triangle of D(k) overwrites A(k,k), A(k+1,k),
and A(k+1,k+1), and v overwrites A(k+2:n,k:k+1).
```

61.15.3 fortran code

— zsytf2_rook.f —

```
*  =====================================================================
      SUBROUTINE ZSYTRF_ROOK( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY, UPPER
      INTEGER            IINFO, IWS, J, K, KB, LDWORK, LWKOPT, NB, NBMIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZLASYF_ROOK, ZSYTF2_ROOK, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
```

```
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
*
      IF( INFO.EQ.0 ) THEN
*
*        Determine the block size
*
         NB = ILAENV( 1, 'ZSYTRF_ROOK', UPLO, N, -1, -1, -1 )
         LWKOPT = N*NB
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYTRF_ROOK', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
      NBMIN = 2
      LDWORK = N
      IF( NB.GT.1 .AND. NB.LT.N ) THEN
         IWS = LDWORK*NB
         IF( LWORK.LT.IWS ) THEN
            NB = MAX( LWORK / LDWORK, 1 )
            NBMIN = MAX( 2, ILAENV( 2, 'ZSYTRF_ROOK',
     $                              UPLO, N, -1, -1, -1 ) )
         END IF
      ELSE
         IWS = 1
      END IF
      IF( NB.LT.NBMIN )
     $   NB = N
*
      IF( UPPER ) THEN
*
```

```
*        Factorize A as U*D*U**T using the upper triangle of A
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        KB, where KB is the number of columns factorized by ZLASYF_ROOK;
*        KB is either NB or NB-1, or K for the last block
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop
*
         IF( K.LT.1 )
     $      GO TO 40
*
         IF( K.GT.NB ) THEN
*
*           Factorize columns k-kb+1:k of A and use blocked code to
*           update columns 1:k-kb
*
            CALL ZLASYF_ROOK( UPLO, K, NB, KB, A, LDA,
     $                        IPIV, WORK, LDWORK, IINFO )
         ELSE
*
*           Use unblocked code to factorize columns 1:k of A
*
            CALL ZSYTF2_ROOK( UPLO, K, A, LDA, IPIV, IINFO )
            KB = K
         END IF
*
*        Set INFO on the first occurrence of a zero pivot
*
         IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO
*
*        No need to adjust IPIV
*
*        Decrease K and return to the start of the main loop
*
         K = K - KB
         GO TO 10
*
      ELSE
*
*        Factorize A as L*D*L**T using the lower triangle of A
*
*        K is the main loop index, increasing from 1 to N in steps of
*        KB, where KB is the number of columns factorized by ZLASYF_ROOK;
*        KB is either NB or NB-1, or N-K+1 for the last block
*
         K = 1
```

```
   20    CONTINUE
*
*        If K > N, exit from loop
*
         IF( K.GT.N )
     $      GO TO 40
*
         IF( K.LE.N-NB ) THEN
*
*           Factorize columns k:k+kb-1 of A and use blocked code to
*           update columns k+kb:n
*
            CALL ZLASYF_ROOK( UPLO, N-K+1, NB, KB, A( K, K ), LDA,
     $                        IPIV( K ), WORK, LDWORK, IINFO )
         ELSE
*
*           Use unblocked code to factorize columns k:n of A
*
            CALL ZSYTF2_ROOK( UPLO, N-K+1, A( K, K ), LDA, IPIV( K ),
     $                    IINFO )
            KB = N - K + 1
         END IF
*
*        Set INFO on the first occurrence of a zero pivot
*
         IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO + K - 1
*
*        Adjust IPIV
*
         DO 30 J = K, K + KB - 1
            IF( IPIV( J ).GT.0 ) THEN
               IPIV( J ) = IPIV( J ) + K - 1
            ELSE
               IPIV( J ) = IPIV( J ) - K + 1
            END IF
   30    CONTINUE
*
*        Increase K and return to the start of the main loop
*
         K = K + KB
         GO TO 20
*
      END IF
*
   40 CONTINUE
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of ZSYTRF_ROOK
```

```
*
      END
```

———-

61.15.4 lisp code

— **zsytf2_rook.lisp** —

———-

61.16 zsytrf

61.16.1 Axiom unit tests

— **zsytrf.input** —

```
)set break resume
)sys rm -f zsytrf.output
)spool zsytrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.16.2 Axiom help page

— **zsytrf.help** —

```
======================================================================
zsytrf examples
======================================================================

======================================================================
Man Page Details
======================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine zsytrf (UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO)
           ZSYTRF

Function/Subroutine Documentation
   subroutine zsytrf (character UPLO, integer N, complex*16, dimension( lda, *
       ) A, integer LDA, integer, dimension( * ) IPIV, complex*16, dimension(
       * ) WORK, integer LWORK, integer INFO)
       ZSYTRF

Purpose:
   ZSYTRF computes the factorization of a complex symmetric matrix A
   using the Bunch-Kaufman diagonal pivoting method.  The form of the
   factorization is

   A = U*D*U**T  or  A = L*D*L**T

   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, and D is symmetric and block diagonal with
   with 1-by-1 and 2-by-2 diagonal blocks.

   This is the blocked version of the algorithm, calling Level 3 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   N-by-N upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading N-by-N lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

   On exit, the block diagonal matrix D and the multipliers used
   to obtain the factor U or L (see below for further details).

 LDA
```

```
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D.
  If IPIV(k) > 0, then rows and columns k and IPIV(k) were
  interchanged and D(k,k) is a 1-by-1 diagonal block.
  If UPLO = 'U' and IPIV(k) = IPIV(k-1) < 0, then rows and
  columns k-1 and -IPIV(k) were interchanged and D(k-1:k,k-1:k)
  is a 2-by-2 diagonal block.  If UPLO = 'L' and IPIV(k) =
  IPIV(k+1) < 0, then rows and columns k+1 and -IPIV(k) were
  interchanged and D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of WORK.  LWORK >=1.  For best performance
  LWORK >= N*NB, where NB is the block size returned by ILAENV.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, D(i,i) is exactly zero.  The factorization
     has been completed, but the block diagonal matrix D is
     exactly singular, and division by zero will occur if it
     is used to solve a system of equations.

Further Details:

  If UPLO = 'U', then A = U*D*U**T, where
     U = P(n)*U(n)* ... *P(k)U(k)* ...,
  i.e., U is a product of terms P(k)*U(k), where k decreases from n to
  1 in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
  and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
  defined by IPIV(k), and U(k) is a unit upper triangular matrix, such
  that if the diagonal block D(k) is of order s (s = 1 or 2), then

                     (   I    v    0   )   k-s
             U(k) =  (   0    I    0   )   s
                     (   0    0    I   )   n-k
```

```
                        k-s   s   n-k

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(1:k-1,k).
   If s = 2, the upper triangle of D(k) overwrites A(k-1,k-1), A(k-1,k),
   and A(k,k), and v overwrites A(1:k-2,k-1:k).

   If UPLO = 'L', then A = L*D*L**T, where
      L = P(1)*L(1)* ... *P(k)*L(k)* ...,
   i.e., L is a product of terms P(k)*L(k), where k increases from 1 to
   n in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and L(k) is a unit lower triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                       (   I    0     0   )  k-1
             L(k) =    (   0    I     0   )  s
                       (   0    v     I   )  n-k-s+1
                          k-1   s  n-k-s+1

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(k+1:n,k).
   If s = 2, the lower triangle of D(k) overwrites A(k,k), A(k+1,k),
   and A(k+1,k+1), and v overwrites A(k+2:n,k:k+1).
```

———-

61.16.3 fortran code

— **zsytrf.f** —

```
*  =====================================================================
      SUBROUTINE ZSYTRF( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
```

```
*
*     .. Local Scalars ..
      LOGICAL            LQUERY, UPPER
      INTEGER            IINFO, IWS, J, K, KB, LDWORK, LWKOPT, NB, NBMIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLASYF, ZSYTF2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
*
      IF( INFO.EQ.0 ) THEN
*
*        Determine the block size
*
         NB = ILAENV( 1, 'ZSYTRF', UPLO, N, -1, -1, -1 )
         LWKOPT = N*NB
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYTRF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
      NBMIN = 2
```

```
      LDWORK = N
      IF( NB.GT.1 .AND. NB.LT.N ) THEN
         IWS = LDWORK*NB
         IF( LWORK.LT.IWS ) THEN
            NB = MAX( LWORK / LDWORK, 1 )
            NBMIN = MAX( 2, ILAENV( 2, 'ZSYTRF', UPLO, N, -1, -1, -1 ) )
         END IF
      ELSE
         IWS = 1
      END IF
      IF( NB.LT.NBMIN )
     $   NB = N
*
      IF( UPPER ) THEN
*
*        Factorize A as U*D*U**T using the upper triangle of A
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        KB, where KB is the number of columns factorized by ZLASYF;
*        KB is either NB or NB-1, or K for the last block
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop
*
         IF( K.LT.1 )
     $      GO TO 40
*
         IF( K.GT.NB ) THEN
*
*           Factorize columns k-kb+1:k of A and use blocked code to
*           update columns 1:k-kb
*
            CALL ZLASYF( UPLO, K, NB, KB, A, LDA, IPIV, WORK, N, IINFO )
         ELSE
*
*           Use unblocked code to factorize columns 1:k of A
*
            CALL ZSYTF2( UPLO, K, A, LDA, IPIV, IINFO )
            KB = K
         END IF
*
*        Set INFO on the first occurrence of a zero pivot
*
         IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO
*
*        Decrease K and return to the start of the main loop
*
```

```
         K = K - KB
         GO TO 10
*
      ELSE
*
*        Factorize A as L*D*L**T using the lower triangle of A
*
*        K is the main loop index, increasing from 1 to N in steps of
*        KB, where KB is the number of columns factorized by ZLASYF;
*        KB is either NB or NB-1, or N-K+1 for the last block
*
         K = 1
   20    CONTINUE
*
*        If K > N, exit from loop
*
         IF( K.GT.N )
     $      GO TO 40
*
         IF( K.LE.N-NB ) THEN
*
*           Factorize columns k:k+kb-1 of A and use blocked code to
*           update columns k+kb:n
*
            CALL ZLASYF( UPLO, N-K+1, NB, KB, A( K, K ), LDA, IPIV( K ),
     $                   WORK, N, IINFO )
         ELSE
*
*           Use unblocked code to factorize columns k:n of A
*
            CALL ZSYTF2( UPLO, N-K+1, A( K, K ), LDA, IPIV( K ), IINFO )
            KB = N - K + 1
         END IF
*
*        Set INFO on the first occurrence of a zero pivot
*
         IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO + K - 1
*
*        Adjust IPIV
*
         DO 30 J = K, K + KB - 1
            IF( IPIV( J ).GT.0 ) THEN
               IPIV( J ) = IPIV( J ) + K - 1
            ELSE
               IPIV( J ) = IPIV( J ) - K + 1
            END IF
   30    CONTINUE
*
*        Increase K and return to the start of the main loop
```

```
*
         K = K + KB
         GO TO 20
*
      END IF
*
   40 CONTINUE
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of ZSYTRF
*
      END
```

———-

61.16.4 lisp code

— **zsytrf.lisp** —

```
(defun zsytrf (uplo n a lda ipiv work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex16 (*)) work a)
           (type (f2cl-lib:integer4) info lwork lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((iinfo 0) (iws 0) (j 0) (k 0) (kb 0) (ldwork 0) (lwkopt 0) (nb 0)
           (nbmin 0) (lquery nil) (upper nil))
      (declare (type f2cl-lib:logical upper lquery)
               (type (f2cl-lib:integer4) nbmin nb lwkopt ldwork kb k j iws
                iinfo))
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond
       ((and (not upper)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
```

```
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4))
 ((and (< lwork 1) (not lquery)) (setf info -7)))
(cond
 ((= info 0)
  (setf nb
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (ilaenv 1 "ZSYTRF" uplo n -1 -1 -1)
            (declare (ignore var-0 var-1 var-4 var-5 var-6))
            (setf uplo var-2)
            (setf n var-3)
            ret-val))
  (setf lwkopt (f2cl-lib:int-mul n nb))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwkopt 'f2cl-lib:complex16))))
(cond
 ((/= info 0) (xerbla "ZSYTRF" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(setf nbmin 2)
(setf ldwork n)
(cond
 ((and (> nb 1) (< nb n)) (setf iws (f2cl-lib:int-mul ldwork nb))
  (cond
   ((< lwork iws)
    (setf nb (max (the f2cl-lib:integer4 (truncate lwork ldwork)) 1))
    (setf nbmin
            (max (the f2cl-lib:integer4 2)
                 (the f2cl-lib:integer4
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
                          (ilaenv 2 "ZSYTRF" uplo n -1 -1 -1)
                        (declare (ignore var-0 var-1 var-4 var-5 var-6))
                        (setf uplo var-2)
                        (setf n var-3)
                        ret-val)))))))
 (t (setf iws 1)))
(if (< nb nbmin)
    (setf nb n))
(cond
 (upper
  (tagbody
    (setf k n)
   label10
```

```
    (if (< k 1)
        (go label40))
    (cond
     ((> k nb)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (zlasyf uplo k nb kb a lda ipiv work n iinfo)
        (declare (ignore var-4 var-6 var-7))
        (when var-0 (setf uplo var-0))
        (when var-1 (setf k var-1))
        (when var-2 (setf nb var-2))
        (when var-3 (setf kb var-3))
        (when var-5 (setf lda var-5))
        (when var-8 (setf n var-8))
        (when var-9 (setf iinfo var-9))))
     (t
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (zsytf2 uplo k a lda ipiv iinfo)
        (declare (ignore var-2 var-4))
        (when var-0 (setf uplo var-0))
        (when var-1 (setf k var-1))
        (when var-3 (setf lda var-3))
        (when var-5 (setf iinfo var-5)))
      (setf kb k)))
    (if (and (= info 0) (> iinfo 0))
        (setf info iinfo))
    (setf k (f2cl-lib:int-sub k kb))
    (go label10)))
  (t
   (tagbody
    (setf k 1)
   label20
    (if (> k n)
        (go label40))
    (cond
     ((<= k (f2cl-lib:int-add n (f2cl-lib:int-sub nb)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (zlasyf uplo (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1) nb kb
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (k k)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (k)
                                 ((1 *)) ipiv-%offset%)
           work n iinfo)
        (declare (ignore var-1 var-4 var-6 var-7))
        (when var-0 (setf uplo var-0))
        (when var-2 (setf nb var-2))
        (when var-3 (setf kb var-3))
        (when var-5 (setf lda var-5))
```

```
              (when var-8 (setf n var-8))
              (when var-9 (setf iinfo var-9))))
            (t
             (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                 (zsytf2 uplo (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
                  (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (k k)
                                        ((1 lda) (1 *)) a-%offset%)
                  lda
                  (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (k)
                                        ((1 *)) ipiv-%offset%)
                  iinfo)
              (declare (ignore var-1 var-2 var-4))
              (when var-0 (setf uplo var-0))
              (when var-3 (setf lda var-3))
              (when var-5 (setf iinfo var-5)))
             (setf kb (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1))))
           (if (and (= info 0) (> iinfo 0))
               (setf info (f2cl-lib:int-sub (f2cl-lib:int-add iinfo k) 1)))
           (f2cl-lib:fdo (j k (f2cl-lib:int-add j 1))
                         ((> j (f2cl-lib:int-add k kb (f2cl-lib:int-sub 1))) nil)
             (tagbody
               (cond
                ((> (f2cl-lib:fref ipiv (j) ((1 *))) 0)
                 (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                         (f2cl-lib:int-sub
                          (f2cl-lib:int-add
                           (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                           k)
                          1)))
                (t
                 (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                         (f2cl-lib:int-add
                          (f2cl-lib:int-sub
                           (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                           k)
                          1))))
              label30))
           (setf k (f2cl-lib:int-add k kb))
           (go label20))))
     label40
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce lwkopt 'f2cl-lib:complex16))
      (go end_label)
     end_label
      (return (values uplo n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsytrf
```

```
        fortran-to-lisp::*f2cl-function-info*)
    (fortran-to-lisp::make-f2cl-finfo :arg-types
                                      '((string)
                                        (fortran-to-lisp::integer4)
                                        (array fortran-to-lisp::complex16
                                         (*))
                                        (fortran-to-lisp::integer4)
                                        (array fortran-to-lisp::integer4
                                         (*))
                                        (array fortran-to-lisp::complex16
                                         (*))
                                        (fortran-to-lisp::integer4)
                                        (fortran-to-lisp::integer4))
                                      :return-values
                                      '(fortran-to-lisp::uplo
                                        fortran-to-lisp::n nil
                                        fortran-to-lisp::lda nil nil nil
                                        fortran-to-lisp::info)
                                      :calls
                                      '(fortran-to-lisp::xerbla
                                        fortran-to-lisp::ilaenv))))
```

———-

61.17 zsytrf_rook

61.17.1 Axiom unit tests

— **zsytrfrook.input** —

```
)set break resume
)sys rm -f zsytrfrook.output
)spool zsytrfrook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.17.2 Axiom help page

— zsytrf_rook.help —

```
====================================================================
zsytrf_rook examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zsytrf_rook (UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO)
           ZSYTRF_ROOK

Function/Subroutine Documentation
   subroutine zsytrf_rook (character UPLO, integer N, complex*16, dimension(
       lda, * ) A, integer LDA, integer, dimension( * ) IPIV, complex*16,
       dimension( * ) WORK, integer LWORK, integer INFO)
       ZSYTRF_ROOK

Purpose:
   ZSYTRF_ROOK computes the factorization of a complex symmetric matrix A
   using the bounded Bunch-Kaufman ("rook") diagonal pivoting method.
   The form of the factorization is

   A = U*D*U**T  or  A = L*D*L**T

   where U (or L) is a product of permutation and unit upper (lower)
   triangular matrices, and D is symmetric and block diagonal with
   1-by-1 and 2-by-2 diagonal blocks.

   This is the blocked version of the algorithm, calling Level 3 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
```

```
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, the block diagonal matrix D and the multipliers used
  to obtain the factor U or L (see below for further details).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

IPIV
  IPIV is INTEGER array, dimension (N)
  Details of the interchanges and the block structure of D.

  If UPLO = 'U':
    If IPIV(k) > 0, then rows and columns k and IPIV(k)
    were interchanged and D(k,k) is a 1-by-1 diagonal block.

    If IPIV(k) < 0 and IPIV(k-1) < 0, then rows and
    columns k and -IPIV(k) were interchanged and rows and
    columns k-1 and -IPIV(k-1) were inerchaged,
    D(k-1:k,k-1:k) is a 2-by-2 diagonal block.

  If UPLO = 'L':
    If IPIV(k) > 0, then rows and columns k and IPIV(k)
    were interchanged and D(k,k) is a 1-by-1 diagonal block.

    If IPIV(k) < 0 and IPIV(k+1) < 0, then rows and
    columns k and -IPIV(k) were interchanged and rows and
    columns k+1 and -IPIV(k+1) were inerchaged,
    D(k:k+1,k:k+1) is a 2-by-2 diagonal block.

WORK
  WORK is COMPLEX*16 array, dimension (MAX(1,LWORK)).
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK
  LWORK is INTEGER
  The length of WORK.  LWORK >=1.  For best performance
  LWORK >= N*NB, where NB is the block size returned by ILAENV.

  If LWORK = -1, then a workspace query is assumed; the routine
  only calculates the optimal size of the WORK array, returns
  this value as the first entry of the WORK array, and no error
  message related to LWORK is issued by XERBLA.
```

```
 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, D(i,i) is exactly zero.  The factorization
      has been completed, but the block diagonal matrix D is
      exactly singular, and division by zero will occur if it
      is used to solve a system of equations.

Further Details:

   If UPLO = 'U', then A = U*D*U**T, where
      U = P(n)*U(n)* ... *P(k)U(k)* ...,
   i.e., U is a product of terms P(k)*U(k), where k decreases from n to
   1 in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and U(k) is a unit upper triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                       (   I    v    0   )   k-s
             U(k) =    (   0    I    0   )   s
                       (   0    0    I   )   n-k
                          k-s   s   n-k

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(1:k-1,k).
   If s = 2, the upper triangle of D(k) overwrites A(k-1,k-1), A(k-1,k),
   and A(k,k), and v overwrites A(1:k-2,k-1:k).

   If UPLO = 'L', then A = L*D*L**T, where
      L = P(1)*L(1)* ... *P(k)*L(k)* ...,
   i.e., L is a product of terms P(k)*L(k), where k increases from 1 to
   n in steps of 1 or 2, and D is a block diagonal matrix with 1-by-1
   and 2-by-2 diagonal blocks D(k).  P(k) is a permutation matrix as
   defined by IPIV(k), and L(k) is a unit lower triangular matrix, such
   that if the diagonal block D(k) is of order s (s = 1 or 2), then

                       (   I    0     0   )  k-1
             L(k) =    (   0    I     0   )  s
                       (   0    v     I   )  n-k-s+1
                          k-1   s  n-k-s+1

   If s = 1, D(k) overwrites A(k,k), and v overwrites A(k+1:n,k).
   If s = 2, the lower triangle of D(k) overwrites A(k,k), A(k+1,k),
   and A(k+1,k+1), and v overwrites A(k+2:n,k:k+1).
```

61.17.3 fortran code

— zsytrf_rook.f —

```
*  =====================================================================
      SUBROUTINE ZSYTRF_ROOK( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY, UPPER
      INTEGER            IINFO, IWS, J, K, KB, LDWORK, LWKOPT, NB, NBMIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZLASYF_ROOK, ZSYTF2_ROOK, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
```

```
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.1 .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
*
      IF( INFO.EQ.0 ) THEN
*
*        Determine the block size
*
         NB = ILAENV( 1, 'ZSYTRF_ROOK', UPLO, N, -1, -1, -1 )
         LWKOPT = N*NB
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYTRF_ROOK', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
      NBMIN = 2
      LDWORK = N
      IF( NB.GT.1 .AND. NB.LT.N ) THEN
         IWS = LDWORK*NB
         IF( LWORK.LT.IWS ) THEN
            NB = MAX( LWORK / LDWORK, 1 )
            NBMIN = MAX( 2, ILAENV( 2, 'ZSYTRF_ROOK',
     $                              UPLO, N, -1, -1, -1 ) )
         END IF
      ELSE
         IWS = 1
      END IF
      IF( NB.LT.NBMIN )
     $   NB = N
*
      IF( UPPER ) THEN
*
*        Factorize A as U*D*U**T using the upper triangle of A
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        KB, where KB is the number of columns factorized by ZLASYF_ROOK;
*        KB is either NB or NB-1, or K for the last block
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop
*
```

```
         IF( K.LT.1 )
     $      GO TO 40
*
         IF( K.GT.NB ) THEN
*
*           Factorize columns k-kb+1:k of A and use blocked code to
*           update columns 1:k-kb
*
            CALL ZLASYF_ROOK( UPLO, K, NB, KB, A, LDA,
     $                        IPIV, WORK, LDWORK, IINFO )
         ELSE
*
*           Use unblocked code to factorize columns 1:k of A
*
            CALL ZSYTF2_ROOK( UPLO, K, A, LDA, IPIV, IINFO )
            KB = K
         END IF
*
*        Set INFO on the first occurrence of a zero pivot
*
         IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO
*
*        No need to adjust IPIV
*
*        Decrease K and return to the start of the main loop
*
         K = K - KB
         GO TO 10
*
      ELSE
*
*        Factorize A as L*D*L**T using the lower triangle of A
*
*        K is the main loop index, increasing from 1 to N in steps of
*        KB, where KB is the number of columns factorized by ZLASYF_ROOK;
*        KB is either NB or NB-1, or N-K+1 for the last block
*
         K = 1
   20    CONTINUE
*
*        If K > N, exit from loop
*
         IF( K.GT.N )
     $      GO TO 40
*
         IF( K.LE.N-NB ) THEN
*
*           Factorize columns k:k+kb-1 of A and use blocked code to
*           update columns k+kb:n
```

```
*
            CALL ZLASYF_ROOK( UPLO, N-K+1, NB, KB, A( K, K ), LDA,
     $                        IPIV( K ), WORK, LDWORK, IINFO )
         ELSE
*
*           Use unblocked code to factorize columns k:n of A
*
            CALL ZSYTF2_ROOK( UPLO, N-K+1, A( K, K ), LDA, IPIV( K ),
     $                   IINFO )
            KB = N - K + 1
         END IF
*
*        Set INFO on the first occurrence of a zero pivot
*
         IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $      INFO = IINFO + K - 1
*
*        Adjust IPIV
*
         DO 30 J = K, K + KB - 1
            IF( IPIV( J ).GT.0 ) THEN
               IPIV( J ) = IPIV( J ) + K - 1
            ELSE
               IPIV( J ) = IPIV( J ) - K + 1
            END IF
   30    CONTINUE
*
*        Increase K and return to the start of the main loop
*
         K = K + KB
         GO TO 20
*
      END IF
*
   40 CONTINUE
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of ZSYTRF_ROOK
*
      END
```

———-

61.17.4 lisp code

— zsytrf_rook.lisp —

```
(defun zsytrf_rook (uplo n a lda ipiv work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex16 (*)) work a)
           (type (f2cl-lib:integer4) info lwork lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((iinfo 0) (iws 0) (j 0) (k 0) (kb 0) (ldwork 0) (lwkopt 0) (nb 0)
           (nbmin 0) (lquery nil) (upper nil))
      (declare (type f2cl-lib:logical upper lquery)
               (type (f2cl-lib:integer4) nbmin nb lwkopt ldwork kb k j iws
                iinfo))
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond
       ((and (not upper)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -4))
       ((and (< lwork 1) (not lquery)) (setf info -7)))
      (cond
       ((= info 0)
        (setf nb
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (ilaenv 1 "ZSYTRF_ROOK" uplo n -1 -1 -1)
                  (declare (ignore var-0 var-1 var-4 var-5 var-6))
                  (setf uplo var-2)
                  (setf n var-3)
                  ret-val))
        (setf lwkopt (f2cl-lib:int-mul n nb))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce lwkopt 'f2cl-lib:complex16))))
      (cond
```

```
 ((/= info 0) (xerbla "ZSYTRF_ROOK" (f2cl-lib:int-sub info))
  (go end_label))
 (lquery (go end_label)))
(setf nbmin 2)
(setf ldwork n)
(cond
 ((and (> nb 1) (< nb n)) (setf iws (f2cl-lib:int-mul ldwork nb))
  (cond
   ((< lwork iws)
    (setf nb (max (the f2cl-lib:integer4 (truncate lwork ldwork)) 1))
    (setf nbmin
            (max (the f2cl-lib:integer4 2)
                 (the f2cl-lib:integer4
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
                          (ilaenv 2 "ZSYTRF_ROOK" uplo n -1 -1 -1)
                        (declare (ignore var-0 var-1 var-4 var-5 var-6))
                        (setf uplo var-2)
                        (setf n var-3)
                        ret-val)))))))
 (t (setf iws 1)))
(if (< nb nbmin)
    (setf nb n))
(cond
 (upper
  (tagbody
    (setf k n)
   label10
    (if (< k 1)
        (go label40))
    (cond
     ((> k nb)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (zlasyf_rook uplo k nb kb a lda ipiv work ldwork iinfo)
        (declare (ignore var-4 var-6 var-7))
        (when var-0 (setf uplo var-0))
        (when var-1 (setf k var-1))
        (when var-2 (setf nb var-2))
        (when var-3 (setf kb var-3))
        (when var-5 (setf lda var-5))
        (when var-8 (setf ldwork var-8))
        (when var-9 (setf iinfo var-9))))
     (t
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (zsytf2_rook uplo k a lda ipiv iinfo)
        (declare (ignore var-2 var-4))
        (when var-0 (setf uplo var-0))
        (when var-1 (setf k var-1))
```

```
            (when var-3 (setf lda var-3))
            (when var-5 (setf iinfo var-5)))
          (setf kb k)))
        (if (and (= info 0) (> iinfo 0))
            (setf info iinfo))
        (setf k (f2cl-lib:int-sub k kb))
        (go label10)))
      (t
       (tagbody
        (setf k 1)
       label20
        (if (> k n)
            (go label40))
        (cond
         ((<= k (f2cl-lib:int-add n (f2cl-lib:int-sub nb)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (zlasyf_rook uplo (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
               nb kb
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (k k)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (k)
                                     ((1 *)) ipiv-%offset%)
               work ldwork iinfo)
            (declare (ignore var-1 var-4 var-6 var-7))
            (when var-0 (setf uplo var-0))
            (when var-2 (setf nb var-2))
            (when var-3 (setf kb var-3))
            (when var-5 (setf lda var-5))
            (when var-8 (setf ldwork var-8))
            (when var-9 (setf iinfo var-9))))
         (t
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
              (zsytf2_rook uplo (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (k k)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (k)
                                     ((1 *)) ipiv-%offset%)
               iinfo)
            (declare (ignore var-1 var-2 var-4))
            (when var-0 (setf uplo var-0))
            (when var-3 (setf lda var-3))
            (when var-5 (setf iinfo var-5)))
          (setf kb (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1))))
        (if (and (= info 0) (> iinfo 0))
            (setf info (f2cl-lib:int-sub (f2cl-lib:int-add iinfo k) 1)))
        (f2cl-lib:fdo (j k (f2cl-lib:int-add j 1))
                      ((> j (f2cl-lib:int-add k kb (f2cl-lib:int-sub 1))) nil)
```

```
            (tagbody
              (cond
               ((> (f2cl-lib:fref ipiv (j) ((1 *))) 0)
                (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                        (f2cl-lib:int-sub
                         (f2cl-lib:int-add
                          (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                          k)
                         1)))
               (t
                (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                        (f2cl-lib:int-add
                         (f2cl-lib:int-sub
                          (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%)
                          k)
                         1))))
             label30))
          (setf k (f2cl-lib:int-add k kb))
          (go label20))))
     label40
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce lwkopt 'f2cl-lib:complex16))
      (go end_label)
     end_label
      (return (values uplo n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsytrf_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::xerbla
```

```
                                        fortran-to-lisp::ilaenv))))
```

———-

61.18 zsytri

61.18.1 Axiom unit tests

— **zsytri.input** —

```
)set break resume
)sys rm -f zsytri.output
)spool zsytri.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.18.2 Axiom help page

— **zsytri.help** —

```
==================================================================
zsytri examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zsytri (UPLO, N, A, LDA, IPIV, WORK, INFO)
           ZSYTRI

Function/Subroutine Documentation
   subroutine zsytri (character UPLO, integer N, complex*16, dimension( lda, *
       ) A, integer LDA, integer, dimension( * ) IPIV, complex*16, dimension(
```

```
      * ) WORK, integer INFO)
      ZSYTRI

Purpose:
   ZSYTRI computes the inverse of a complex symmetric indefinite matrix
   A using the factorization A = U*D*U**T or A = L*D*L**T computed by ZSYTRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the block diagonal matrix D and the multipliers
   used to obtain the factor U or L as computed by ZSYTRF.

   On exit, if INFO = 0, the (symmetric) inverse of the original
   matrix.  If UPLO = 'U', the upper triangular part of the
   inverse is formed and the part of A below the diagonal is not
   referenced; if UPLO = 'L' the lower triangular part of the
   inverse is formed and the part of A above the diagonal is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by ZSYTRF.

 WORK
   WORK is COMPLEX*16 array, dimension (2*N)

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value
   > 0: if INFO = i, D(i,i) = 0; the matrix is singular and its
        inverse could not be computed.
```

———-

61.18.3 fortran code

— **zsytri.f** —

```
*  =====================================================================
      SUBROUTINE ZSYTRI( UPLO, N, A, LDA, IPIV, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE, ZERO
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ),
     $                   ZERO = ( 0.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            K, KP, KSTEP
      COMPLEX*16         AK, AKKP1, AKP1, D, T, TEMP
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      COMPLEX*16         ZDOTU
      EXTERNAL           LSAME, ZDOTU
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZCOPY, ZSWAP, ZSYMV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
```

```
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYTRI', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
         DO 10 INFO = N, 1, -1
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
   10    CONTINUE
      ELSE
*
*        Lower triangular storage: examine D from top to bottom.
*
         DO 20 INFO = 1, N
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
   20    CONTINUE
      END IF
      INFO = 0
*
      IF( UPPER ) THEN
*
*        Compute inv(A) from the factorization A = U*D*U**T.
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
```

```
*
         K = 1
   30    CONTINUE
*
*        If K > N, exit from loop.
*
         IF( K.GT.N )
     $      GO TO 40
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Invert the diagonal block.
*
            A( K, K ) = ONE / A( K, K )
*
*           Compute column K of the inverse.
*
            IF( K.GT.1 ) THEN
               CALL ZCOPY( K-1, A( 1, K ), 1, WORK, 1 )
               CALL ZSYMV( UPLO, K-1, -ONE, A, LDA, WORK, 1, ZERO,
     $                     A( 1, K ), 1 )
               A( K, K ) = A( K, K ) - ZDOTU( K-1, WORK, 1, A( 1, K ),
     $                     1 )
            END IF
            KSTEP = 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Invert the diagonal block.
*
            T = A( K, K+1 )
            AK = A( K, K ) / T
            AKP1 = A( K+1, K+1 ) / T
            AKKP1 = A( K, K+1 ) / T
            D = T*( AK*AKP1-ONE )
            A( K, K ) = AKP1 / D
            A( K+1, K+1 ) = AK / D
            A( K, K+1 ) = -AKKP1 / D
*
*           Compute columns K and K+1 of the inverse.
*
            IF( K.GT.1 ) THEN
               CALL ZCOPY( K-1, A( 1, K ), 1, WORK, 1 )
               CALL ZSYMV( UPLO, K-1, -ONE, A, LDA, WORK, 1, ZERO,
     $                     A( 1, K ), 1 )
               A( K, K ) = A( K, K ) - ZDOTU( K-1, WORK, 1, A( 1, K ),
     $                     1 )
```

```
               A( K, K+1 ) = A( K, K+1 ) -
     $                       ZDOTU( K-1, A( 1, K ), 1, A( 1, K+1 ), 1 )
               CALL ZCOPY( K-1, A( 1, K+1 ), 1, WORK, 1 )
               CALL ZSYMV( UPLO, K-1, -ONE, A, LDA, WORK, 1, ZERO,
     $                     A( 1, K+1 ), 1 )
               A( K+1, K+1 ) = A( K+1, K+1 ) -
     $                         ZDOTU( K-1, WORK, 1, A( 1, K+1 ), 1 )
            END IF
            KSTEP = 2
         END IF
*
         KP = ABS( IPIV( K ) )
         IF( KP.NE.K ) THEN
*
*           Interchange rows and columns K and KP in the leading
*           submatrix A(1:k+1,1:k+1)
*
            CALL ZSWAP( KP-1, A( 1, K ), 1, A( 1, KP ), 1 )
            CALL ZSWAP( K-KP-1, A( KP+1, K ), 1, A( KP, KP+1 ), LDA )
            TEMP = A( K, K )
            A( K, K ) = A( KP, KP )
            A( KP, KP ) = TEMP
            IF( KSTEP.EQ.2 ) THEN
               TEMP = A( K, K+1 )
               A( K, K+1 ) = A( KP, K+1 )
               A( KP, K+1 ) = TEMP
            END IF
         END IF
*
         K = K + KSTEP
         GO TO 30
   40    CONTINUE
*
      ELSE
*
*        Compute inv(A) from the factorization A = L*D*L**T.
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = N
   50    CONTINUE
*
*        If K < 1, exit from loop.
*
         IF( K.LT.1 )
     $      GO TO 60
*
         IF( IPIV( K ).GT.0 ) THEN
*
```

```
*          1 x 1 diagonal block
*
*          Invert the diagonal block.
*
           A( K, K ) = ONE / A( K, K )
*
*          Compute column K of the inverse.
*
           IF( K.LT.N ) THEN
              CALL ZCOPY( N-K, A( K+1, K ), 1, WORK, 1 )
              CALL ZSYMV( UPLO, N-K, -ONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                    ZERO, A( K+1, K ), 1 )
              A( K, K ) = A( K, K ) - ZDOTU( N-K, WORK, 1, A( K+1, K ),
     $                    1 )
           END IF
           KSTEP = 1
        ELSE
*
*          2 x 2 diagonal block
*
*          Invert the diagonal block.
*
           T = A( K, K-1 )
           AK = A( K-1, K-1 ) / T
           AKP1 = A( K, K ) / T
           AKKP1 = A( K, K-1 ) / T
           D = T*( AK*AKP1-ONE )
           A( K-1, K-1 ) = AKP1 / D
           A( K, K ) = AK / D
           A( K, K-1 ) = -AKKP1 / D
*
*          Compute columns K-1 and K of the inverse.
*
           IF( K.LT.N ) THEN
              CALL ZCOPY( N-K, A( K+1, K ), 1, WORK, 1 )
              CALL ZSYMV( UPLO, N-K, -ONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                    ZERO, A( K+1, K ), 1 )
              A( K, K ) = A( K, K ) - ZDOTU( N-K, WORK, 1, A( K+1, K ),
     $                    1 )
              A( K, K-1 ) = A( K, K-1 ) -
     $                      ZDOTU( N-K, A( K+1, K ), 1, A( K+1, K-1 ),
     $                      1 )
              CALL ZCOPY( N-K, A( K+1, K-1 ), 1, WORK, 1 )
              CALL ZSYMV( UPLO, N-K, -ONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                    ZERO, A( K+1, K-1 ), 1 )
              A( K-1, K-1 ) = A( K-1, K-1 ) -
     $                        ZDOTU( N-K, WORK, 1, A( K+1, K-1 ), 1 )
           END IF
           KSTEP = 2
        END IF
```

```
*
         KP = ABS( IPIV( K ) )
         IF( KP.NE.K ) THEN
*
*           Interchange rows and columns K and KP in the trailing
*           submatrix A(k-1:n,k-1:n)
*
            IF( KP.LT.N )
     $         CALL ZSWAP( N-KP, A( KP+1, K ), 1, A( KP+1, KP ), 1 )
            CALL ZSWAP( KP-K-1, A( K+1, K ), 1, A( KP, K+1 ), LDA )
            TEMP = A( K, K )
            A( K, K ) = A( KP, KP )
            A( KP, KP ) = TEMP
            IF( KSTEP.EQ.2 ) THEN
               TEMP = A( K, K-1 )
               A( K, K-1 ) = A( KP, K-1 )
               A( KP, K-1 ) = TEMP
            END IF
         END IF
*
         K = K - KSTEP
         GO TO 50
   60    CONTINUE
      END IF
*
      RETURN
*
*     End of ZSYTRI
*
      END
```

———-

61.18.4 lisp code

— **zsytri.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)) (zero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (type (f2cl-lib:complex16) zero)
           (ignorable one zero))
  (defun zsytri (uplo n a lda ipiv work info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
```

```
    (a f2cl-lib:complex16 a-%data% a-%offset%)
    (work f2cl-lib:complex16 work-%data% work-%offset%)
    (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
  (prog ((ak #C(0.0d0 0.0d0)) (akkp1 #C(0.0d0 0.0d0))
         (akp1 #C(0.0d0 0.0d0)) (d #C(0.0d0 0.0d0)) (t$ #C(0.0d0 0.0d0))
         (temp #C(0.0d0 0.0d0)) (k 0) (kp 0) (kstep 0) (upper nil))
    (declare (type (f2cl-lib:complex16) ak akkp1 akp1 d t$ temp)
             (type (f2cl-lib:integer4) k kp kstep)
             (type f2cl-lib:logical upper))
    (setf info 0)
    (setf upper
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
    (cond
     ((and (not upper)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
      (setf info -1))
     ((< n 0) (setf info -2))
     ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -4)))
    (cond
     ((/= info 0) (xerbla "ZSYTRI" (f2cl-lib:int-sub info))
      (go end_label)))
    (if (= n 0)
        (go end_label))
    (cond
     (upper
      (f2cl-lib:fdo (info n (f2cl-lib:int-add info (f2cl-lib:int-sub 1)))
                    ((> info 1) nil)
        (tagbody
          (if (and
               (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
                  0)
               (=
                (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                               a-%offset%)
                zero))
              (go end_label))
          label10)))
     (t
      (f2cl-lib:fdo (info 1 (f2cl-lib:int-add info 1))
                    ((> info n) nil)
```

```
      (tagbody
        (if (and
             (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
                0)
             (=
              (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                             a-%offset%)
              zero))
            (go end_label))
       label20))))
  (setf info 0)
  (cond
   (upper
    (tagbody
      (setf k 1)
     label30
      (if (> k n)
          (go label40))
      (cond
       ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
        (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                (/ one
                   (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                  a-%offset%)))
        (cond
         ((> k 1)
          (zcopy (f2cl-lib:int-sub k 1)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1 work 1)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9)
              (zsymv uplo (f2cl-lib:int-sub k 1) (- one) a lda work 1
               zero
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                                     ((1 lda) (1 *)) a-%offset%)
               1)
            (declare
             (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9))
            (setf uplo var-0))
          (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                  (-
                   (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                  a-%offset%)
                   (zdotu (f2cl-lib:int-sub k 1) work 1
                    (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                          (1 k) ((1 lda) (1 *))
                                          a-%offset%)
```

```
            1)))))
 (setf kstep 1))
(t
 (setf t$
        (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                       ((1 lda) (1 *)) a-%offset%))
 (setf ak
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         t$))
 (setf akp1
        (/
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-add k 1)
                         (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
 (setf akkp1
        (/
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
 (setf d (* t$ (- (* ak akp1) one)))
 (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (/ akp1 d))
 (setf (f2cl-lib:fref a-%data%
                      ((f2cl-lib:int-add k 1)
                       (f2cl-lib:int-add k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (/ ak d))
 (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (/ (- akkp1) d))
 (cond
  ((> k 1)
   (zcopy (f2cl-lib:int-sub k 1)
    (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                          ((1 lda) (1 *)) a-%offset%)
    1 work 1)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9)
       (zsymv uplo (f2cl-lib:int-sub k 1) (- one) a lda work 1
        zero
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                              ((1 lda) (1 *)) a-%offset%)
        1)
     (declare
      (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
    var-9))
  (setf uplo var-0))
(setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         (zdotu (f2cl-lib:int-sub k 1) work 1
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                (1 k) ((1 lda) (1 *))
                                a-%offset%)
          1)))
(setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
         (zdotu (f2cl-lib:int-sub k 1)
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                (1 k) ((1 lda) (1 *))
                                a-%offset%)
          1
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                (1 (f2cl-lib:int-add k 1))
                                ((1 lda) (1 *)) a-%offset%)
          1)))
(zcopy (f2cl-lib:int-sub k 1)
 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                       (1 (f2cl-lib:int-add k 1))
                       ((1 lda) (1 *)) a-%offset%)
 1 work 1)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zsymv uplo (f2cl-lib:int-sub k 1) (- one) a lda work 1
     zero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                           (1 (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
    var-9))
  (setf uplo var-0))
(setf (f2cl-lib:fref a-%data%
                      ((f2cl-lib:int-add k 1)
                       (f2cl-lib:int-add k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-add k 1)
```

```
                                (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%)
                (zdotu (f2cl-lib:int-sub k 1) work 1
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                       (1 (f2cl-lib:int-add k 1))
                                       ((1 lda) (1 *)) a-%offset%)
                 1)))))
  (setf kstep 2)))
(setf kp
        (abs
         (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
(cond
 ((/= kp k)
  (zswap (f2cl-lib:int-sub kp 1)
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                         ((1 lda) (1 *)) a-%offset%)
   1
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 kp)
                         ((1 lda) (1 *)) a-%offset%)
   1)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (zswap (f2cl-lib:int-sub k kp 1)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             ((+ kp 1) k) ((1 lda) (1 *))
                             a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             (kp (f2cl-lib:int-add kp 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (setf temp
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
          (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *)) a-%offset%)
          temp)
  (cond
   ((= kstep 2)
    (setf temp
            (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                          ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-add k 1))
```

```
                                ((1 lda) (1 *)) a-%offset%)
                 temp)))))
      (setf k (f2cl-lib:int-add k kstep))
      (go label30)
     label40))
    (t
     (tagbody
      (setf k n)
     label50
      (if (< k 1)
          (go label60))
      (cond
       ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
        (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                (/ one
                   (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                  a-%offset%)))
        (cond
         ((< k n)
          (zcopy (f2cl-lib:int-sub n k)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 ((+ k 1) k)
                                 ((1 lda) (1 *)) a-%offset%)
           1 work 1)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9)
              (zsymv uplo (f2cl-lib:int-sub n k) (- one)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                     ((+ k 1) (f2cl-lib:int-add k 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda work 1 zero
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                     ((+ k 1) k) ((1 lda) (1 *))
                                     a-%offset%)
               1)
            (declare
             (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9))
            (setf uplo var-0))
          (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                  (-
                   (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                  a-%offset%)
                   (zdotu (f2cl-lib:int-sub n k) work 1
                    (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                          ((+ k 1) k) ((1 lda) (1 *))
                                          a-%offset%)
                    1)))))
        (setf kstep 1))
       (t
```

```
(setf t$
        (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                       ((1 lda) (1 *)) a-%offset%))
(setf ak
        (/
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-sub k 1)
                         (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
(setf akp1
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         t$))
(setf akkp1
        (/
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
(setf d (* t$ (- (* ak akp1) one)))
(setf (f2cl-lib:fref a-%data%
                     ((f2cl-lib:int-sub k 1)
                      (f2cl-lib:int-sub k 1))
                     ((1 lda) (1 *)) a-%offset%)
        (/ akp1 d))
(setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (/ ak d))
(setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                     ((1 lda) (1 *)) a-%offset%)
        (/ (- akkp1) d))
(cond
 ((< k n)
  (zcopy (f2cl-lib:int-sub n k)
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 ((+ k 1) k)
                         ((1 lda) (1 *)) a-%offset%)
   1 work 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (zsymv uplo (f2cl-lib:int-sub n k) (- one)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             ((+ k 1) (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda work 1 zero
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             ((+ k 1) k) ((1 lda) (1 *))
                             a-%offset%)
       1)
    (declare
```

```
   (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
    var-9))
  (setf uplo var-0))
(setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         (zdotu (f2cl-lib:int-sub n k) work 1
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                ((+ k 1) k) ((1 lda) (1 *))
                                a-%offset%)
          1)))
(setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         (zdotu (f2cl-lib:int-sub n k)
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                ((+ k 1) k) ((1 lda) (1 *))
                                a-%offset%)
          1
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                ((+ k 1)
                                 (f2cl-lib:int-sub k 1))
                                ((1 lda) (1 *)) a-%offset%)
          1)))
(zcopy (f2cl-lib:int-sub n k)
 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                       ((+ k 1) (f2cl-lib:int-sub k 1))
                       ((1 lda) (1 *)) a-%offset%)
 1 work 1)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zsymv uplo (f2cl-lib:int-sub n k) (- one)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                           ((+ k 1) (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda work 1 zero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                           ((+ k 1) (f2cl-lib:int-sub k 1))
                           ((1 lda) (1 *)) a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
    var-9))
  (setf uplo var-0))
(setf (f2cl-lib:fref a-%data%
                      ((f2cl-lib:int-sub k 1)
```

```
                              (f2cl-lib:int-sub k 1))
                             ((1 lda) (1 *)) a-%offset%)
                 (-
                  (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-sub k 1)
                                  (f2cl-lib:int-sub k 1))
                                 ((1 lda) (1 *)) a-%offset%)
                  (zdotu (f2cl-lib:int-sub n k) work 1
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                         ((+ k 1)
                                          (f2cl-lib:int-sub k 1))
                                         ((1 lda) (1 *)) a-%offset%)
                   1)))))
  (setf kstep 2)))
(setf kp
        (abs
         (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
(cond
 ((/= kp k)
  (if (< kp n)
      (zswap (f2cl-lib:int-sub n kp)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             ((+ kp 1) k) ((1 lda) (1 *))
                             a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             ((+ kp 1) kp) ((1 lda) (1 *))
                             a-%offset%)
       1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (zswap (f2cl-lib:int-sub kp k 1)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             ((+ k 1) k) ((1 lda) (1 *))
                             a-%offset%)
       1
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             (kp (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf lda var-4)))
  (setf temp
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
          (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *)) a-%offset%)
          temp)
  (cond
```

```
              ((= kstep 2)
               (setf temp
                       (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                                       ((1 lda) (1 *)) a-%offset%))
               (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                                     ((1 lda) (1 *)) a-%offset%)
                       (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-sub k 1))
                                       ((1 lda) (1 *)) a-%offset%))
               (setf (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-sub k 1))
                                     ((1 lda) (1 *)) a-%offset%)
                       temp)))))
            (setf k (f2cl-lib:int-sub k kstep))
            (go label50)
           label60)))
        (go end_label)
       end_label
        (return (values uplo nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsytri
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::zsymv
                                              fortran-to-lisp::xerbla))))
```

———-

61.19 zsytri2

61.19.1 Axiom unit tests

— zsytri2.input —

```
)set break resume
)sys rm -f zsytri2.output
)spool zsytri2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.19.2 Axiom help page

— zsytri2.help —

```
======================================================================
zsytri2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zsytri2 (UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO)
           ZSYTRI2

Function/Subroutine Documentation
   subroutine zsytri2 (character UPLO, integer N, complex*16, dimension( lda,
       * ) A, integer LDA, integer, dimension( * ) IPIV, complex*16,
       dimension( * ) WORK, integer LWORK, integer INFO)
       ZSYTRI2

Purpose:
   ZSYTRI2 computes the inverse of a COMPLEX*16 symmetric indefinite matrix
   A using the factorization A = U*D*U**T or A = L*D*L**T computed by
   ZSYTRF. ZSYTRI2 sets the LEADING DIMENSION of the workspace
   before calling ZSYTRI2X that actually computes the inverse.
```

```
Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the NB diagonal matrix D and the multipliers
   used to obtain the factor U or L as computed by ZSYTRF.

   On exit, if INFO = 0, the (symmetric) inverse of the original
   matrix.  If UPLO = 'U', the upper triangular part of the
   inverse is formed and the part of A below the diagonal is not
   referenced; if UPLO = 'L' the lower triangular part of the
   inverse is formed and the part of A above the diagonal is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the NB structure of D
   as determined by ZSYTRF.

 WORK
   WORK is COMPLEX*16 array, dimension (N+NB+1)*(NB+3)

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK.
   WORK is size >= (N+NB+1)*(NB+3)
   If LDWORK = -1, then a workspace query is assumed; the routine
   calculates:
   - the optimal size of the WORK array, returns
   this value as the first entry of the WORK array,
   - and no error message related to LDWORK is issued by XERBLA.

 INFO
   INFO is INTEGER
```

```
= 0: successful exit
< 0: if INFO = -i, the i-th argument had an illegal value
> 0: if INFO = i, D(i,i) = 0; the matrix is singular and its
     inverse could not be computed.
```

61.19.3 fortran code

— **zsytri2.f** —

```
*  =====================================================================
      SUBROUTINE ZSYTRI2( UPLO, N, A, LDA, IPIV, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            UPPER, LQUERY
      INTEGER            MINSIZE, NBMAX
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZSYTRI, ZSYTRI2X
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
```

```
      UPPER = LSAME( UPLO, 'U' )
      LQUERY = ( LWORK.EQ.-1 )
*     Get blocksize
      NBMAX = ILAENV( 1, 'ZSYTRF', UPLO, N, -1, -1, -1 )
      IF ( NBMAX .GE. N ) THEN
         MINSIZE = N
      ELSE
         MINSIZE = (N+NBMAX+1)*(NBMAX+3)
      END IF
*
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF (LWORK .LT. MINSIZE .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
*
*     Quick return if possible
*
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYTRI2', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         WORK(1)=MINSIZE
         RETURN
      END IF
      IF( N.EQ.0 )
     $   RETURN

      IF( NBMAX .GE. N ) THEN
         CALL ZSYTRI( UPLO, N, A, LDA, IPIV, WORK, INFO )
      ELSE
         CALL ZSYTRI2X( UPLO, N, A, LDA, IPIV, WORK, NBMAX, INFO )
      END IF
      RETURN
*
*     End of ZSYTRI2
*
      END
```

———-

61.19.4 lisp code

— zsytri2.lisp —

```
(defun zsytri2 (uplo n a lda ipiv work lwork info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex16 (*)) work a)
           (type (f2cl-lib:integer4) info lwork lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((minsize 0) (nbmax 0) (upper nil) (lquery nil))
      (declare (type f2cl-lib:logical lquery upper)
               (type (f2cl-lib:integer4) nbmax minsize))
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (setf nbmax
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 1 "ZSYTRF" uplo n -1 -1 -1)
                (declare (ignore var-0 var-1 var-4 var-5 var-6))
                (setf uplo var-2)
                (setf n var-3)
                ret-val))
      (cond ((>= nbmax n) (setf minsize n))
            (t
             (setf minsize
                     (f2cl-lib:int-mul (f2cl-lib:int-add n nbmax 1)
                                       (f2cl-lib:int-add nbmax 3)))))
      (cond
       ((and (not upper)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
```

```
        (setf info -4))
       ((and (< lwork minsize) (not lquery)) (setf info -7)))
      (cond
       ((/= info 0) (xerbla "ZSYTRI2" (f2cl-lib:int-sub info)) (go end_label))
       (lquery
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce minsize 'f2cl-lib:complex16))
        (go end_label)))
      (if (= n 0)
          (go end_label))
      (cond
       ((>= nbmax n)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zsytri uplo n a lda ipiv work info)
          (declare (ignore var-2 var-4 var-5))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-6 (setf info var-6))))
       (t
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (zsytri2x uplo n a lda ipiv work nbmax info)
          (declare (ignore var-2 var-4 var-5))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-3 (setf lda var-3))
          (when var-6 (setf nbmax var-6))
          (when var-7 (setf info var-7)))))
      (go end_label)
     end_label
      (return (values uplo n nil lda nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsytri2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
```

```
                                        '(fortran-to-lisp::uplo
                                          fortran-to-lisp::n nil
                                          fortran-to-lisp::lda nil nil nil
                                          fortran-to-lisp::info)
                                        :calls
                                        '(fortran-to-lisp::xerbla
                                          fortran-to-lisp::ilaenv))))
```

———-

61.20 zsytri2x

61.20.1 Axiom unit tests

— zsytri2x.input —

```
)set break resume
)sys rm -f zsytri2x.output
)spool zsytri2x.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.20.2 Axiom help page

— zsytri2x.help —

```
======================================================================
zsytri2x examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
```

```
      subroutine zsytri2x (UPLO, N, A, LDA, IPIV, WORK, NB, INFO)
          ZSYTRI2X

Function/Subroutine Documentation
   subroutine zsytri2x (character UPLO, integer N, complex*16, dimension( lda,
       * ) A, integer LDA, integer, dimension( * ) IPIV, complex*16,
       dimension( n+nb+1,* ) WORK, integer NB, integer INFO)
       ZSYTRI2X

Purpose:
   ZSYTRI2X computes the inverse of a complex symmetric indefinite matrix
   A using the factorization A = U*D*U**T or A = L*D*L**T computed by ZSYTRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the NNB diagonal matrix D and the multipliers
   used to obtain the factor U or L as computed by ZSYTRF.

   On exit, if INFO = 0, the (symmetric) inverse of the original
   matrix.  If UPLO = 'U', the upper triangular part of the
   inverse is formed and the part of A below the diagonal is not
   referenced; if UPLO = 'L' the lower triangular part of the
   inverse is formed and the part of A above the diagonal is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the NNB structure of D
   as determined by ZSYTRF.

 WORK
   WORK is COMPLEX*16 array, dimension (N+NNB+1,NNB+3)
```

```
 NB
   NB is INTEGER
   Block size

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value
   > 0: if INFO = i, D(i,i) = 0; the matrix is singular and its
        inverse could not be computed.
```

———-

61.20.3 fortran code

— **zsytri2x.f** —

```
*  =====================================================================
      SUBROUTINE ZSYTRI2X( UPLO, N, A, LDA, IPIV, WORK, NB, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N, NB
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), WORK( N+NB+1,* )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE, ZERO
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ),
     $                   ZERO = ( 0.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, IINFO, IP, K, CUT, NNB
      INTEGER            COUNT
      INTEGER            J, U11, INVD
```

```
      COMPLEX*16         AK, AKKP1, AKP1, D, T
      COMPLEX*16         U01_I_J, U01_IP1_J
      COMPLEX*16         U11_I_J, U11_IP1_J
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZSYCONV, XERBLA, ZTRTRI
      EXTERNAL           ZGEMM, ZTRMM, ZSYSWAPR
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
*
*     Quick return if possible
*
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYTRI2X', -INFO )
         RETURN
      END IF
      IF( N.EQ.0 )
     $   RETURN
*
*     Convert A
*     Workspace got Non-diag elements of D
*
      CALL ZSYCONV( UPLO, 'C', N, A, LDA, IPIV, WORK, IINFO )
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
```

```
         DO INFO = N, 1, -1
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
         END DO
      ELSE
*
*        Lower triangular storage: examine D from top to bottom.
*
         DO INFO = 1, N
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.ZERO )
     $         RETURN
         END DO
      END IF
      INFO = 0
*
*  Splitting Workspace
*     U01 is a block (N,NB+1)
*     The first element of U01 is in WORK(1,1)
*     U11 is a block (NB+1,NB+1)
*     The first element of U11 is in WORK(N+1,1)
      U11 = N
*     INVD is a block (N,2)
*     The first element of INVD is in WORK(1,INVD)
      INVD = NB+2

      IF( UPPER ) THEN
*
*        invA = P * inv(U**T)*inv(D)*inv(U)*P**T.
*
        CALL ZTRTRI( UPLO, 'U', N, A, LDA, INFO )
*
*       inv(D) and inv(D)*inv(U)
*
        K=1
        DO WHILE ( K .LE. N )
         IF( IPIV( K ).GT.0 ) THEN
*           1 x 1 diagonal NNB
             WORK(K,INVD) = 1/  A( K, K )
             WORK(K,INVD+1) = 0
            K=K+1
         ELSE
*           2 x 2 diagonal NNB
             T = WORK(K+1,1)
             AK = A( K, K ) / T
             AKP1 = A( K+1, K+1 ) / T
             AKKP1 = WORK(K+1,1)  / T
             D = T*( AK*AKP1-ONE )
             WORK(K,INVD) = AKP1 / D
             WORK(K+1,INVD+1) = AK / D
             WORK(K,INVD+1) = -AKKP1 / D
```

```
             WORK(K+1,INVD) = -AKKP1 / D
            K=K+2
         END IF
        END DO
*
*       inv(U**T) = (inv(U))**T
*
*       inv(U**T)*inv(D)*inv(U)
*
        CUT=N
        DO WHILE (CUT .GT. 0)
           NNB=NB
           IF (CUT .LE. NNB) THEN
              NNB=CUT
           ELSE
              COUNT = 0
*             count negative elements,
              DO I=CUT+1-NNB,CUT
                  IF (IPIV(I) .LT. 0) COUNT=COUNT+1
              END DO
*             need a even number for a clear cut
              IF (MOD(COUNT,2) .EQ. 1) NNB=NNB+1
           END IF

           CUT=CUT-NNB
*
*          U01 Block
*
           DO I=1,CUT
             DO J=1,NNB
              WORK(I,J)=A(I,CUT+J)
             END DO
           END DO
*
*          U11 Block
*
           DO I=1,NNB
             WORK(U11+I,I)=ONE
             DO J=1,I-1
                WORK(U11+I,J)=ZERO
             END DO
             DO J=I+1,NNB
                WORK(U11+I,J)=A(CUT+I,CUT+J)
             END DO
           END DO
*
*          invD*U01
*
           I=1
           DO WHILE (I .LE. CUT)
```

```
            IF (IPIV(I) > 0) THEN
               DO J=1,NNB
                   WORK(I,J)=WORK(I,INVD)*WORK(I,J)
               END DO
               I=I+1
            ELSE
               DO J=1,NNB
                  U01_I_J = WORK(I,J)
                  U01_IP1_J = WORK(I+1,J)
                  WORK(I,J)=WORK(I,INVD)*U01_I_J+
     $                     WORK(I,INVD+1)*U01_IP1_J
                  WORK(I+1,J)=WORK(I+1,INVD)*U01_I_J+
     $                     WORK(I+1,INVD+1)*U01_IP1_J
               END DO
               I=I+2
            END IF
          END DO
*
*       invD1*U11
*
          I=1
          DO WHILE (I .LE. NNB)
            IF (IPIV(CUT+I) > 0) THEN
               DO J=I,NNB
                   WORK(U11+I,J)=WORK(CUT+I,INVD)*WORK(U11+I,J)
               END DO
               I=I+1
            ELSE
               DO J=I,NNB
                  U11_I_J = WORK(U11+I,J)
                  U11_IP1_J = WORK(U11+I+1,J)
               WORK(U11+I,J)=WORK(CUT+I,INVD)*WORK(U11+I,J) +
     $                     WORK(CUT+I,INVD+1)*WORK(U11+I+1,J)
               WORK(U11+I+1,J)=WORK(CUT+I+1,INVD)*U11_I_J+
     $                     WORK(CUT+I+1,INVD+1)*U11_IP1_J
               END DO
               I=I+2
            END IF
          END DO
*
*      U11**T*invD1*U11->U11
*
       CALL ZTRMM('L','U','T','U',NNB, NNB,
     $            ONE,A(CUT+1,CUT+1),LDA,WORK(U11+1,1),N+NB+1)
*
        DO I=1,NNB
           DO J=I,NNB
             A(CUT+I,CUT+J)=WORK(U11+I,J)
           END DO
        END DO
```

```
*
*          U01**T*invD*U01->A(CUT+I,CUT+J)
*
         CALL ZGEMM('T','N',NNB,NNB,CUT,ONE,A(1,CUT+1),LDA,
     $              WORK,N+NB+1, ZERO, WORK(U11+1,1), N+NB+1)
*
*        U11 =  U11**T*invD1*U11 + U01**T*invD*U01
*
         DO I=1,NNB
            DO J=I,NNB
              A(CUT+I,CUT+J)=A(CUT+I,CUT+J)+WORK(U11+I,J)
            END DO
         END DO
*
*        U01 =  U00**T*invD0*U01
*
         CALL ZTRMM('L',UPLO,'T','U',CUT, NNB,
     $             ONE,A,LDA,WORK,N+NB+1)

*
*        Update U01
*
         DO I=1,CUT
           DO J=1,NNB
            A(I,CUT+J)=WORK(I,J)
           END DO
         END DO
*
*      Next Block
*
       END DO
*
*        Apply PERMUTATIONS P and P**T: P * inv(U**T)*inv(D)*inv(U) *P**T
*
            I=1
            DO WHILE ( I .LE. N )
               IF( IPIV(I) .GT. 0 ) THEN
                  IP=IPIV(I)
                 IF (I .LT. IP) CALL ZSYSWAPR( UPLO, N, A, LDA, I ,IP )
                 IF (I .GT. IP) CALL ZSYSWAPR( UPLO, N, A, LDA, IP ,I )
               ELSE
                 IP=-IPIV(I)
                 I=I+1
                 IF ( (I-1) .LT. IP)
     $                 CALL ZSYSWAPR( UPLO, N, A, LDA, I-1 ,IP )
                 IF ( (I-1) .GT. IP)
     $                 CALL ZSYSWAPR( UPLO, N, A, LDA, IP ,I-1 )
              ENDIF
               I=I+1
            END DO
```

```
      ELSE
*
*        LOWER...
*
*        invA = P * inv(U**T)*inv(D)*inv(U)*P**T.
*
         CALL ZTRTRI( UPLO, 'U', N, A, LDA, INFO )
*
*       inv(D) and inv(D)*inv(U)
*
        K=N
        DO WHILE ( K .GE. 1 )
         IF( IPIV( K ).GT.0 ) THEN
*           1 x 1 diagonal NNB
             WORK(K,INVD) = 1/  A( K, K )
             WORK(K,INVD+1) = 0
            K=K-1
         ELSE
*           2 x 2 diagonal NNB
             T = WORK(K-1,1)
             AK = A( K-1, K-1 ) / T
             AKP1 = A( K, K ) / T
             AKKP1 = WORK(K-1,1) / T
             D = T*( AK*AKP1-ONE )
             WORK(K-1,INVD) = AKP1 / D
             WORK(K,INVD) = AK / D
             WORK(K,INVD+1) = -AKKP1 / D
             WORK(K-1,INVD+1) = -AKKP1 / D
            K=K-2
         END IF
        END DO
*
*       inv(U**T) = (inv(U))**T
*
*       inv(U**T)*inv(D)*inv(U)
*
        CUT=0
        DO WHILE (CUT .LT. N)
           NNB=NB
           IF (CUT + NNB .GE. N) THEN
              NNB=N-CUT
           ELSE
              COUNT = 0
*             count negative elements,
              DO I=CUT+1,CUT+NNB
                  IF (IPIV(I) .LT. 0) COUNT=COUNT+1
              END DO
*             need a even number for a clear cut
              IF (MOD(COUNT,2) .EQ. 1) NNB=NNB+1
           END IF
```

```
*      L21 Block
           DO I=1,N-CUT-NNB
             DO J=1,NNB
              WORK(I,J)=A(CUT+NNB+I,CUT+J)
             END DO
           END DO
*     L11 Block
           DO I=1,NNB
             WORK(U11+I,I)=ONE
             DO J=I+1,NNB
                WORK(U11+I,J)=ZERO
             END DO
             DO J=1,I-1
                WORK(U11+I,J)=A(CUT+I,CUT+J)
             END DO
           END DO
*
*          invD*L21
*
           I=N-CUT-NNB
           DO WHILE (I .GE. 1)
             IF (IPIV(CUT+NNB+I) > 0) THEN
                DO J=1,NNB
                    WORK(I,J)=WORK(CUT+NNB+I,INVD)*WORK(I,J)
                END DO
                I=I-1
             ELSE
                DO J=1,NNB
                   U01_I_J = WORK(I,J)
                   U01_IP1_J = WORK(I-1,J)
                   WORK(I,J)=WORK(CUT+NNB+I,INVD)*U01_I_J+
     $                       WORK(CUT+NNB+I,INVD+1)*U01_IP1_J
                   WORK(I-1,J)=WORK(CUT+NNB+I-1,INVD+1)*U01_I_J+
     $                       WORK(CUT+NNB+I-1,INVD)*U01_IP1_J
                END DO
                I=I-2
             END IF
           END DO
*
*        invD1*L11
*
           I=NNB
           DO WHILE (I .GE. 1)
             IF (IPIV(CUT+I) > 0) THEN
                DO J=1,NNB
                    WORK(U11+I,J)=WORK(CUT+I,INVD)*WORK(U11+I,J)
                END DO
                I=I-1
             ELSE
                DO J=1,NNB
```

```
                 U11_I_J = WORK(U11+I,J)
                 U11_IP1_J = WORK(U11+I-1,J)
              WORK(U11+I,J)=WORK(CUT+I,INVD)*WORK(U11+I,J) +
     $                     WORK(CUT+I,INVD+1)*U11_IP1_J
              WORK(U11+I-1,J)=WORK(CUT+I-1,INVD+1)*U11_I_J+
     $                     WORK(CUT+I-1,INVD)*U11_IP1_J
              END DO
              I=I-2
           END IF
         END DO
*
*       L11**T*invD1*L11->L11
*
        CALL ZTRMM('L',UPLO,'T','U',NNB, NNB,
     $             ONE,A(CUT+1,CUT+1),LDA,WORK(U11+1,1),N+NB+1)
*
         DO I=1,NNB
            DO J=1,I
              A(CUT+I,CUT+J)=WORK(U11+I,J)
            END DO
         END DO
*

        IF ( (CUT+NNB) .LT. N ) THEN
*
*         L21**T*invD2*L21->A(CUT+I,CUT+J)
*
         CALL ZGEMM('T','N',NNB,NNB,N-NNB-CUT,ONE,A(CUT+NNB+1,CUT+1)
     $             ,LDA,WORK,N+NB+1, ZERO, WORK(U11+1,1), N+NB+1)

*
*        L11 =  L11**T*invD1*L11 + U01**T*invD*U01
*
         DO I=1,NNB
            DO J=1,I
              A(CUT+I,CUT+J)=A(CUT+I,CUT+J)+WORK(U11+I,J)
            END DO
         END DO
*
*        U01 =  L22**T*invD2*L21
*
         CALL ZTRMM('L',UPLO,'T','U', N-NNB-CUT, NNB,
     $             ONE,A(CUT+NNB+1,CUT+NNB+1),LDA,WORK,N+NB+1)

*      Update L21
         DO I=1,N-CUT-NNB
           DO J=1,NNB
              A(CUT+NNB+I,CUT+J)=WORK(I,J)
           END DO
         END DO
```

```
      ELSE
*
*       L11 =  L11**T*invD1*L11
*
        DO I=1,NNB
           DO J=1,I
             A(CUT+I,CUT+J)=WORK(U11+I,J)
           END DO
        END DO
      END IF
*
*     Next Block
*
          CUT=CUT+NNB
      END DO
*
*       Apply PERMUTATIONS P and P**T: P * inv(U**T)*inv(D)*inv(U) *P**T
*
           I=N
           DO WHILE ( I .GE. 1 )
              IF( IPIV(I) .GT. 0 ) THEN
                 IP=IPIV(I)
                IF (I .LT. IP) CALL ZSYSWAPR( UPLO, N, A, LDA, I ,IP  )
                IF (I .GT. IP) CALL ZSYSWAPR( UPLO, N, A, LDA, IP ,I )
              ELSE
                IP=-IPIV(I)
                IF ( I .LT. IP) CALL ZSYSWAPR( UPLO, N, A, LDA, I ,IP )
                IF ( I .GT. IP) CALL ZSYSWAPR( UPLO, N, A, LDA, IP ,I )
                I=I-1
              ENDIF
              I=I-1
           END DO
     END IF
*
     RETURN
*
*    End of ZSYTRI2X
*
     END
```

———-

61.20.4 lisp code

— zsytri2x.lisp —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)) (zero (f2cl-lib:cmplx 0.0d0 0.0d0)))
```

```
(declare (type (f2cl-lib:complex16) one)
         (type (f2cl-lib:complex16) zero)
         (ignorable one zero))
(defun zsytri2x (uplo n a lda ipiv work nb info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex16 (*)) work a)
           (type (f2cl-lib:integer4) info nb lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((u11_i_j #C(0.0d0 0.0d0)) (u11_ip1_j #C(0.0d0 0.0d0))
           (u01_i_j #C(0.0d0 0.0d0)) (u01_ip1_j #C(0.0d0 0.0d0))
           (ak #C(0.0d0 0.0d0)) (akkp1 #C(0.0d0 0.0d0))
           (akp1 #C(0.0d0 0.0d0)) (d #C(0.0d0 0.0d0)) (t$ #C(0.0d0 0.0d0))
           (j 0) (u11 0) (invd 0) (count$ 0) (i 0) (iinfo 0) (ip 0) (k 0)
           (cut 0) (nnb 0) (upper nil))
      (declare
       (type (f2cl-lib:complex16) u11_i_j u11_ip1_j u01_i_j u01_ip1_j ak
        akkp1 akp1 d t$)
       (type (f2cl-lib:integer4) j u11 invd count$ i iinfo ip k cut nnb)
       (type f2cl-lib:logical upper))
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (cond
       ((and (not upper)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -4)))
      (cond
       ((/= info 0) (xerbla "ZSYTRI2X" (f2cl-lib:int-sub info))
        (go end_label)))
      (if (= n 0)
          (go end_label))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (zsyconv uplo "C" n a lda ipiv work iinfo)
```

```
  (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
  (setf uplo var-0)
  (setf iinfo var-7))
(cond
 (upper
  (f2cl-lib:fdo (info n (f2cl-lib:int-add info (f2cl-lib:int-sub 1)))
                ((> info 1) nil)
    (tagbody
      (if (and
           (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
              0)
           (=
            (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                           a-%offset%)
            zero))
          (go end_label))
     label100000)))
 (t
  (f2cl-lib:fdo (info 1 (f2cl-lib:int-add info 1))
                ((> info n) nil)
    (tagbody
      (if (and
           (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
              0)
           (=
            (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                           a-%offset%)
            zero))
          (go end_label))
     label100001))))
(setf info 0)
(setf u11 n)
(setf invd (f2cl-lib:int-add nb 2))
(cond
 (upper
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (ztrtri uplo "U" n a lda info)
      (declare (ignore var-1 var-3))
      (when var-0 (setf uplo var-0))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-5 (setf info var-5)))
    (setf k 1)
   label100002
    (if (not (<= k n))
        (go label100003))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf (f2cl-lib:fref work-%data% (k invd)
```

```
                        ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                        work-%offset%)
                (/ 1
                   (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                  a-%offset%)))
 (setf (f2cl-lib:fref work-%data% (k (f2cl-lib:int-add invd 1))
                        ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                        work-%offset%)
                (coerce 0 'f2cl-lib:complex16))
 (setf k (f2cl-lib:int-add k 1)))
(t
 (setf t$
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1) 1)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%))
 (setf ak
                (/
                 (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                a-%offset%)
                 t$))
 (setf akp1
                (/
                 (f2cl-lib:fref a-%data%
                                ((f2cl-lib:int-add k 1)
                                 (f2cl-lib:int-add k 1))
                                ((1 lda) (1 *)) a-%offset%)
                 t$))
 (setf akkp1
                (/
                 (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1) 1)
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%)
                 t$))
 (setf d (* t$ (- (* ak akp1) one)))
 (setf (f2cl-lib:fref work-%data% (k invd)
                        ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                        work-%offset%)
                (/ akp1 d))
 (setf (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add k 1)
                         (f2cl-lib:int-add invd 1))
                        ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                        work-%offset%)
                (/ ak d))
 (setf (f2cl-lib:fref work-%data% (k (f2cl-lib:int-add invd 1))
                        ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                        work-%offset%)
                (/ (- akkp1) d))
 (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k 1) invd)
                        ((1 (f2cl-lib:int-add n nb 1)) (1 *))
```

```
                          work-%offset%)
            (/ (- akkp1) d))
    (setf k (f2cl-lib:int-add k 2))))
  (go label100002)
 label100003
  (setf cut n)
 label100004
  (if (not (> cut 0))
      (go label100005))
  (setf nnb nb)
  (cond ((<= cut nnb) (setf nnb cut))
        (t (setf count$ 0)
         (f2cl-lib:fdo (i
                        (f2cl-lib:int-add cut 1
                                          (f2cl-lib:int-sub nnb))
                        (f2cl-lib:int-add i 1))
                       ((> i cut) nil)
           (tagbody
             (if (<
                  (f2cl-lib:fref ipiv-%data% (i) ((1 *))
                                 ipiv-%offset%)
                  0)
                 (setf count$ (f2cl-lib:int-add count$ 1)))
            label100006))
         (if (= (mod count$ 2) 1)
             (setf nnb (f2cl-lib:int-add nnb 1)))))
  (setf cut (f2cl-lib:int-sub cut nnb))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i cut) nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nnb) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i j)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)
                (f2cl-lib:fref a-%data%
                               (i (f2cl-lib:int-add cut j))
                               ((1 lda) (1 *)) a-%offset%))
         label100008))
     label100007))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i nnb) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) i)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
            one)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
```

```
                    nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add u11 i) j)
                                 ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                 work-%offset%)
                  zero)
           label100010))
        (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                      ((> j nnb) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add u11 i) j)
                                 ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                 work-%offset%)
                  (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add cut i)
                                  (f2cl-lib:int-add cut j))
                                 ((1 lda) (1 *)) a-%offset%))
           label100011))
      label100009))
  (setf i 1)
 label100012
  (if (not (<= i cut))
      (go label100013))
  (cond
   ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
              (*
               (f2cl-lib:fref work-%data% (i invd)
                              ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                              work-%offset%)
               (f2cl-lib:fref work-%data% (i j)
                              ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                              work-%offset%)))
       label100014))
    (setf i (f2cl-lib:int-add i 1)))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf u01_i_j
              (f2cl-lib:fref work-%data% (i j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%))
```

```
            (setf u01_ip1_j
                    (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1) j)
                                   ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                   work-%offset%))
            (setf (f2cl-lib:fref work-%data% (i j)
                                 ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                 work-%offset%)
                    (+
                     (*
                      (f2cl-lib:fref work-%data% (i invd)
                                     ((1 (f2cl-lib:int-add n nb 1))
                                      (1 *))
                                     work-%offset%)
                      u01_i_j)
                     (*
                      (f2cl-lib:fref work-%data%
                                     (i (f2cl-lib:int-add invd 1))
                                     ((1 (f2cl-lib:int-add n nb 1))
                                      (1 *))
                                     work-%offset%)
                      u01_ip1_j)))
            (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1) j)
                                 ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                 work-%offset%)
                    (+
                     (*
                      (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-add i 1) invd)
                                     ((1 (f2cl-lib:int-add n nb 1))
                                      (1 *))
                                     work-%offset%)
                      u01_i_j)
                     (*
                      (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-add i 1)
                                      (f2cl-lib:int-add invd 1))
                                     ((1 (f2cl-lib:int-add n nb 1))
                                      (1 *))
                                     work-%offset%)
                      u01_ip1_j)))
          label100015))
      (setf i (f2cl-lib:int-add i 2))))
    (go label100012)
   label100013
    (setf i 1)
   label100016
    (if (not (<= i nnb))
        (go label100017))
    (cond
     ((> (f2cl-lib:fref ipiv ((f2cl-lib:int-add cut i)) ((1 *))) 0)
```

```
 (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
               ((> j nnb) nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) j)
                          ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                          work-%offset%)
             (*
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add cut i) invd)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add u11 i) j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)))
    label100018))
 (setf i (f2cl-lib:int-add i 1)))
(t
 (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
               ((> j nnb) nil)
   (tagbody
     (setf u11_i_j
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add u11 i) j)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%))
     (setf u11_ip1_j
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add u11 i 1) j)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%))
     (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) j)
                          ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                          work-%offset%)
             (+
              (*
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add cut i) invd)
                              ((1 (f2cl-lib:int-add n nb 1))
                               (1 *))
                              work-%offset%)
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add u11 i) j)
                              ((1 (f2cl-lib:int-add n nb 1))
                               (1 *))
                              work-%offset%))
              (*
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add cut i)
                               (f2cl-lib:int-add invd 1))
```

```
                                    ((1 (f2cl-lib:int-add n nb 1))
                                     (1 *))
                                    work-%offset%)
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add u11 i 1) j)
                                    ((1 (f2cl-lib:int-add n nb 1))
                                     (1 *))
                                    work-%offset%))))
            (setf (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add u11 i 1) j)
                                 ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                 work-%offset%)
                  (+
                   (*
                    (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-add cut i 1) invd)
                                   ((1 (f2cl-lib:int-add n nb 1))
                                    (1 *))
                                   work-%offset%)
                    u11_i_j)
                   (*
                    (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-add cut i 1)
                                    (f2cl-lib:int-add invd 1))
                                   ((1 (f2cl-lib:int-add n nb 1))
                                    (1 *))
                                   work-%offset%)
                    u11_ip1_j)))
          label100019))
      (setf i (f2cl-lib:int-add i 2))))
    (go label100016)
  label100017
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (ztrmm "L" "U" "T" "U" nnb nnb one
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               ((+ cut 1) (f2cl-lib:int-add cut 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               ((+ u11 1) 1)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)
         (f2cl-lib:int-add n nb 1))
      (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9 var-10))
      (when var-4 (setf nnb var-4))
      (when var-5 (setf nnb var-5))
      (when var-6 (setf one var-6))
      (when var-8 (setf lda var-8)))
```

```
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nnb) nil)
  (tagbody
    (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data%
                             ((f2cl-lib:int-add cut i)
                              (f2cl-lib:int-add cut j))
                             ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add u11 i) j)
                               ((1 (f2cl-lib:int-add n nb 1))
                                (1 *))
                               work-%offset%))
       label100021))
   label100020))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (zgemm "T" "N" nnb nnb cut one
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                           (1 (f2cl-lib:int-add cut 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda work (f2cl-lib:int-add n nb 1) zero
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ u11 1) 1)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
     (f2cl-lib:int-add n nb 1))
  (declare (ignore var-0 var-1 var-6 var-8 var-9 var-11 var-12))
  (when var-2 (setf nnb var-2))
  (when var-3 (setf nnb var-3))
  (when var-4 (setf cut var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf zero var-10)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nnb) nil)
  (tagbody
    (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data%
                             ((f2cl-lib:int-add cut i)
                              (f2cl-lib:int-add cut j))
                             ((1 lda) (1 *)) a-%offset%)
                (+
                 (f2cl-lib:fref a-%data%
                                ((f2cl-lib:int-add cut i)
```

```
                                    (f2cl-lib:int-add cut j))
                                   ((1 lda) (1 *)) a-%offset%)
                     (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-add u11 i) j)
                                   ((1 (f2cl-lib:int-add n nb 1))
                                    (1 *))
                                   work-%offset%)))
        label100023))
    label100022))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (ztrmm "L" uplo "T" "U" cut nnb one a lda work
      (f2cl-lib:int-add n nb 1))
   (declare (ignore var-0 var-2 var-3 var-7 var-9 var-10))
   (when var-1 (setf uplo var-1))
   (when var-4 (setf cut var-4))
   (when var-5 (setf nnb var-5))
   (when var-6 (setf one var-6))
   (when var-8 (setf lda var-8)))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i cut) nil)
   (tagbody
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j nnb) nil)
       (tagbody
         (setf (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add cut j))
                              ((1 lda) (1 *)) a-%offset%)
                 (f2cl-lib:fref work-%data% (i j)
                                ((1 (f2cl-lib:int-add n nb 1))
                                 (1 *))
                                work-%offset%))
        label100025))
    label100024))
 (go label100004)
label100005
 (setf i 1)
label100026
 (if (not (<= i n))
     (go label100027))
 (cond
  ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
   (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
   (if (< i ip)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
           (zsyswapr uplo n a lda i ip)
         (declare (ignore var-2))
         (when var-0 (setf uplo var-0))
         (when var-1 (setf n var-1))
         (when var-3 (setf lda var-3))
```

```
                (when var-4 (setf i var-4))
                (when var-5 (setf ip var-5))))
          (if (> i ip)
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                  (zsyswapr uplo n a lda ip i)
                (declare (ignore var-2))
                (when var-0 (setf uplo var-0))
                (when var-1 (setf n var-1))
                (when var-3 (setf lda var-3))
                (when var-4 (setf ip var-4))
                (when var-5 (setf i var-5)))))
         (t
          (setf ip
                  (f2cl-lib:int-sub
                   (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)))
          (setf i (f2cl-lib:int-add i 1))
          (if (< (f2cl-lib:int-sub i 1) ip)
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                  (zsyswapr uplo n a lda (f2cl-lib:int-sub i 1) ip)
                (declare (ignore var-2 var-4))
                (when var-0 (setf uplo var-0))
                (when var-1 (setf n var-1))
                (when var-3 (setf lda var-3))
                (when var-5 (setf ip var-5))))
          (if (> (f2cl-lib:int-sub i 1) ip)
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                  (zsyswapr uplo n a lda ip (f2cl-lib:int-sub i 1))
                (declare (ignore var-2 var-5))
                (when var-0 (setf uplo var-0))
                (when var-1 (setf n var-1))
                (when var-3 (setf lda var-3))
                (when var-4 (setf ip var-4))))))
       (setf i (f2cl-lib:int-add i 1))
       (go label100026)
      label100027))
    (t
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
           (ztrtri uplo "U" n a lda info)
         (declare (ignore var-1 var-3))
         (when var-0 (setf uplo var-0))
         (when var-2 (setf n var-2))
         (when var-4 (setf lda var-4))
         (when var-5 (setf info var-5)))
       (setf k n)
      label100028
       (if (not (>= k 1))
           (go label100029))
       (cond
        ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
```

```
 (setf (f2cl-lib:fref work-%data% (k invd)
                      ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                      work-%offset%)
        (/ 1
           (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                          a-%offset%)))
 (setf (f2cl-lib:fref work-%data% (k (f2cl-lib:int-add invd 1))
                      ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                      work-%offset%)
        (coerce 0 'f2cl-lib:complex16))
 (setf k (f2cl-lib:int-sub k 1)))
(t
 (setf t$
        (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1) 1)
                       ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                       work-%offset%))
 (setf ak
        (/
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-sub k 1)
                         (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         t$))
 (setf akp1
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         t$))
 (setf akkp1
        (/
         (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1) 1)
                        ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                        work-%offset%)
         t$))
 (setf d (* t$ (- (* ak akp1) one)))
 (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1) invd)
                      ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                      work-%offset%)
        (/ akp1 d))
 (setf (f2cl-lib:fref work-%data% (k invd)
                      ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                      work-%offset%)
        (/ ak d))
 (setf (f2cl-lib:fref work-%data% (k (f2cl-lib:int-add invd 1))
                      ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                      work-%offset%)
        (/ (- akkp1) d))
 (setf (f2cl-lib:fref work-%data%
                      ((f2cl-lib:int-sub k 1)
                       (f2cl-lib:int-add invd 1))
```

```
                          ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                          work-%offset%)
            (/ (- akkp1) d))
    (setf k (f2cl-lib:int-sub k 2))))
  (go label100028)
 label100029
  (setf cut 0)
 label100030
  (if (not (< cut n))
      (go label100031))
  (setf nnb nb)
  (cond
   ((>= (f2cl-lib:int-add cut nnb) n)
    (setf nnb (f2cl-lib:int-sub n cut)))
   (t (setf count$ 0)
    (f2cl-lib:fdo (i (f2cl-lib:int-add cut 1) (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add cut nnb)) nil)
      (tagbody
        (if (< (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
               0)
            (setf count$ (f2cl-lib:int-add count$ 1)))
       label100032))
    (if (= (mod count$ 2) 1)
        (setf nnb (f2cl-lib:int-add nnb 1)))))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i
                    (f2cl-lib:int-add n (f2cl-lib:int-sub cut)
                                      (f2cl-lib:int-sub nnb)))
                 nil)
    (tagbody
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nnb) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i j)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)
                  (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add cut nnb i)
                                  (f2cl-lib:int-add cut j))
                                 ((1 lda) (1 *)) a-%offset%))
        label100034))
     label100033))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i nnb) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) i)
                           ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                           work-%offset%)
              one)
      (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
```

```
                    ((> j nnb) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add u11 i) j)
                                 ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                 work-%offset%)
                  zero)
           label100036))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add u11 i) j)
                                 ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                 work-%offset%)
                  (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add cut i)
                                  (f2cl-lib:int-add cut j))
                                 ((1 lda) (1 *)) a-%offset%))
           label100037))
       label100035))
    (setf i (f2cl-lib:int-sub n cut nnb))
   label100038
    (if (not (>= i 1))
        (go label100039))
    (cond
     ((> (f2cl-lib:fref ipiv ((f2cl-lib:int-add cut nnb i)) ((1 *))) 0)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nnb) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i j)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)
                (*
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add cut nnb i) invd)
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%)
                 (f2cl-lib:fref work-%data% (i j)
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%)))
         label100040))
      (setf i (f2cl-lib:int-sub i 1)))
     (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nnb) nil)
        (tagbody
          (setf u01_i_j
                (f2cl-lib:fref work-%data% (i j)
```

```
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%))
        (setf u01_ip1_j
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub i 1) j)
                                ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                                work-%offset%))
        (setf (f2cl-lib:fref work-%data% (i j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
                (+
                 (*
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add cut nnb i) invd)
                                 ((1 (f2cl-lib:int-add n nb 1))
                                  (1 *))
                                 work-%offset%)
                  u01_i_j)
                 (*
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add cut nnb i)
                                  (f2cl-lib:int-add invd 1))
                                 ((1 (f2cl-lib:int-add n nb 1))
                                  (1 *))
                                 work-%offset%)
                  u01_ip1_j)))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub i 1) j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
                (+
                 (*
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-add cut nnb i) 1)
                                  (f2cl-lib:int-add invd 1))
                                 ((1 (f2cl-lib:int-add n nb 1))
                                  (1 *))
                                 work-%offset%)
                  u01_i_j)
                 (*
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-add cut nnb i) 1)
                                  invd)
                                 ((1 (f2cl-lib:int-add n nb 1))
                                  (1 *))
                                 work-%offset%)
                  u01_ip1_j)))
      label100041))
  (setf i (f2cl-lib:int-sub i 2))))
(go label100038)
```

```
label100039
 (setf i nnb)
label100042
 (if (not (>= i 1))
     (go label100043))
 (cond
  ((> (f2cl-lib:fref ipiv ((f2cl-lib:int-add cut i)) ((1 *))) 0)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j nnb) nil)
     (tagbody
       (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) j)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%)
             (*
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add cut i) invd)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add u11 i) j)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)))
      label100044))
   (setf i (f2cl-lib:int-sub i 1)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j nnb) nil)
     (tagbody
       (setf u11_i_j
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-add u11 i) j)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%))
       (setf u11_ip1_j
             (f2cl-lib:fref work-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add u11 i) 1)
                             j)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%))
       (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add u11 i) j)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%)
             (+
              (*
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add cut i) invd)
                              ((1 (f2cl-lib:int-add n nb 1))
                               (1 *))
                              work-%offset%)
```

```
                        (f2cl-lib:fref work-%data%
                                       ((f2cl-lib:int-add u11 i) j)
                                       ((1 (f2cl-lib:int-add n nb 1))
                                        (1 *))
                                       work-%offset%))
                       (*
                        (f2cl-lib:fref work-%data%
                                       ((f2cl-lib:int-add cut i)
                                        (f2cl-lib:int-add invd 1))
                                       ((1 (f2cl-lib:int-add n nb 1))
                                        (1 *))
                                       work-%offset%)
                        u11_ip1_j)))
          (setf (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add u11 i) 1)
                                j)
                               ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                               work-%offset%)
                 (+
                  (*
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-sub
                                    (f2cl-lib:int-add cut i) 1)
                                   (f2cl-lib:int-add invd 1))
                                  ((1 (f2cl-lib:int-add n nb 1))
                                   (1 *))
                                  work-%offset%)
                   u11_i_j)
                  (*
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-sub
                                    (f2cl-lib:int-add cut i) 1)
                                   invd)
                                  ((1 (f2cl-lib:int-add n nb 1))
                                   (1 *))
                                  work-%offset%)
                   u11_ip1_j)))
        label100045))
      (setf i (f2cl-lib:int-sub i 2))))
    (go label100042)
   label100043
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (ztrmm "L" uplo "T" "U" nnb nnb one
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               ((+ cut 1) (f2cl-lib:int-add cut 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda
```

```
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                            ((+ u11 1) 1)
                            ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                            work-%offset%)
      (f2cl-lib:int-add n nb 1))
  (declare (ignore var-0 var-2 var-3 var-7 var-9 var-10))
  (when var-1 (setf uplo var-1))
  (when var-4 (setf nnb var-4))
  (when var-5 (setf nnb var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nnb) nil)
  (tagbody
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j i) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data%
                             ((f2cl-lib:int-add cut i)
                              (f2cl-lib:int-add cut j))
                             ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add u11 i) j)
                              ((1 (f2cl-lib:int-add n nb 1))
                               (1 *))
                              work-%offset%))
      label100047))
   label100046))
(cond
 ((< (f2cl-lib:int-add cut nnb) n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (zgemm "T" "N" nnb nnb (f2cl-lib:int-sub n nnb cut) one
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             ((+ cut nnb 1)
                              (f2cl-lib:int-add cut 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda work (f2cl-lib:int-add n nb 1) zero
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             ((+ u11 1) 1)
                             ((1 (f2cl-lib:int-add n nb 1)) (1 *))
                             work-%offset%)
       (f2cl-lib:int-add n nb 1))
    (declare
     (ignore var-0 var-1 var-4 var-6 var-8 var-9 var-11 var-12))
    (when var-2 (setf nnb var-2))
    (when var-3 (setf nnb var-3))
    (when var-5 (setf one var-5))
    (when var-7 (setf lda var-7))
```

```
  (when var-10 (setf zero var-10)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nnb) nil)
  (tagbody
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j i) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data%
                             ((f2cl-lib:int-add cut i)
                              (f2cl-lib:int-add cut j))
                             ((1 lda) (1 *)) a-%offset%)
                (+
                 (f2cl-lib:fref a-%data%
                                ((f2cl-lib:int-add cut i)
                                 (f2cl-lib:int-add cut j))
                                ((1 lda) (1 *)) a-%offset%)
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add u11 i) j)
                                ((1 (f2cl-lib:int-add n nb 1))
                                 (1 *))
                                work-%offset%)))
       label100049))
   label100048))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "L" uplo "T" "U" (f2cl-lib:int-sub n nnb cut) nnb one
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                           ((+ cut nnb 1)
                            (f2cl-lib:int-add cut nnb 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda work (f2cl-lib:int-add n nb 1))
  (declare (ignore var-0 var-2 var-3 var-4 var-7 var-9 var-10))
  (when var-1 (setf uplo var-1))
  (when var-5 (setf nnb var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i
                  (f2cl-lib:int-add n (f2cl-lib:int-sub cut)
                                    (f2cl-lib:int-sub nnb)))
               nil)
  (tagbody
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nnb) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data%
                             ((f2cl-lib:int-add cut nnb i)
                              (f2cl-lib:int-add cut j))
                             ((1 lda) (1 *)) a-%offset%)
```

```
                    (f2cl-lib:fref work-%data% (i j)
                                   ((1 (f2cl-lib:int-add n nb 1))
                                    (1 *))
                                   work-%offset%))
          label100051))
      label100050)))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i nnb) nil)
     (tagbody
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j i) nil)
         (tagbody
           (setf (f2cl-lib:fref a-%data%
                                ((f2cl-lib:int-add cut i)
                                 (f2cl-lib:int-add cut j))
                                ((1 lda) (1 *)) a-%offset%)
                    (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-add u11 i) j)
                                   ((1 (f2cl-lib:int-add n nb 1))
                                    (1 *))
                                   work-%offset%))
          label100053))
      label100052))))
 (setf cut (f2cl-lib:int-add cut nnb))
 (go label100030)
label100031
 (setf i n)
label100054
 (if (not (>= i 1))
     (go label100055))
 (cond
  ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
   (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
   (if (< i ip)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
           (zsyswapr uplo n a lda i ip)
         (declare (ignore var-2))
         (when var-0 (setf uplo var-0))
         (when var-1 (setf n var-1))
         (when var-3 (setf lda var-3))
         (when var-4 (setf i var-4))
         (when var-5 (setf ip var-5))))
   (if (> i ip)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
           (zsyswapr uplo n a lda ip i)
         (declare (ignore var-2))
         (when var-0 (setf uplo var-0))
         (when var-1 (setf n var-1))
         (when var-3 (setf lda var-3))
```

```
                  (when var-4 (setf ip var-4))
                  (when var-5 (setf i var-5)))))
            (t
             (setf ip
                     (f2cl-lib:int-sub
                      (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)))
             (if (< i ip)
                 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                     (zsyswapr uplo n a lda i ip)
                   (declare (ignore var-2))
                   (when var-0 (setf uplo var-0))
                   (when var-1 (setf n var-1))
                   (when var-3 (setf lda var-3))
                   (when var-4 (setf i var-4))
                   (when var-5 (setf ip var-5))))
             (if (> i ip)
                 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                     (zsyswapr uplo n a lda ip i)
                   (declare (ignore var-2))
                   (when var-0 (setf uplo var-0))
                   (when var-1 (setf n var-1))
                   (when var-3 (setf lda var-3))
                   (when var-4 (setf ip var-4))
                   (when var-5 (setf i var-5))))
             (setf i (f2cl-lib:int-sub i 1))))
           (setf i (f2cl-lib:int-sub i 1))
           (go label100054)
          label100055)))
       (go end_label)
      end_label
       (return (values uplo n nil lda nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsytri2x
                 fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((string)
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex16
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::integer4
                                             (*))
                                            (array fortran-to-lisp::complex16
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4))
                                          :return-values
```

```
                                        '(fortran-to-lisp::uplo
                                          fortran-to-lisp::n nil
                                          fortran-to-lisp::lda nil nil nil
                                          fortran-to-lisp::info)
                                        :calls
                                        '(fortran-to-lisp::zsyconv
                                          fortran-to-lisp::xerbla))))
```

———-

61.21 zsytri_rook

61.21.1 Axiom unit tests

— **zsytrirook.input** —

```
)set break resume
)sys rm -f zsytrirook.output
)spool zsytrirook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.21.2 Axiom help page

— **zsytri_rook.help** —

```
==================================================================
zsytri_rook examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
```

```
       subroutine zsytri_rook (UPLO, N, A, LDA, IPIV, WORK, INFO)
           ZSYTRI_ROOK

Function/Subroutine Documentation
   subroutine zsytri_rook (character UPLO, integer N, complex*16, dimension(
       lda, * ) A, integer LDA, integer, dimension( * ) IPIV, complex*16,
       dimension( * ) WORK, integer INFO)
       ZSYTRI_ROOK

Purpose:
   ZSYTRI_ROOK computes the inverse of a complex symmetric
   matrix A using the factorization A = U*D*U**T or A = L*D*L**T
   computed by ZSYTRF_ROOK.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the block diagonal matrix D and the multipliers
   used to obtain the factor U or L as computed by ZSYTRF_ROOK.

   On exit, if INFO = 0, the (symmetric) inverse of the original
   matrix.  If UPLO = 'U', the upper triangular part of the
   inverse is formed and the part of A below the diagonal is not
   referenced; if UPLO = 'L' the lower triangular part of the
   inverse is formed and the part of A above the diagonal is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by ZSYTRF_ROOK.

 WORK
   WORK is COMPLEX*16 array, dimension (N)
```

```
 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -i, the i-th argument had an illegal value
   > 0: if INFO = i, D(i,i) = 0; the matrix is singular and its
        inverse could not be computed.
```

61.21.3 fortran code

— zsytri_rook.f —

```
*  =====================================================================
      SUBROUTINE ZSYTRI_ROOK( UPLO, N, A, LDA, IPIV, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         CONE, CZERO
      PARAMETER          ( CONE = ( 1.0D+0, 0.0D+0 ),
     $                   CZERO = ( 0.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            K, KP, KSTEP
      COMPLEX*16         AK, AKKP1, AKP1, D, T, TEMP
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      COMPLEX*16         ZDOTU
      EXTERNAL           LSAME, ZDOTU
```

```
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZCOPY, ZSWAP, ZSYMV, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYTRI_ROOK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Check that the diagonal matrix D is nonsingular.
*
      IF( UPPER ) THEN
*
*        Upper triangular storage: examine D from bottom to top
*
         DO 10 INFO = N, 1, -1
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.CZERO )
     $         RETURN
   10    CONTINUE
      ELSE
*
*        Lower triangular storage: examine D from top to bottom.
*
         DO 20 INFO = 1, N
            IF( IPIV( INFO ).GT.0 .AND. A( INFO, INFO ).EQ.CZERO )
     $         RETURN
   20    CONTINUE
      END IF
      INFO = 0
```

```
*
      IF( UPPER ) THEN
*
*        Compute inv(A) from the factorization A = U*D*U**T.
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = 1
   30    CONTINUE
*
*        If K > N, exit from loop.
*
         IF( K.GT.N )
     $      GO TO 40
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Invert the diagonal block.
*
            A( K, K ) = CONE / A( K, K )
*
*           Compute column K of the inverse.
*
            IF( K.GT.1 ) THEN
               CALL ZCOPY( K-1, A( 1, K ), 1, WORK, 1 )
               CALL ZSYMV( UPLO, K-1, -CONE, A, LDA, WORK, 1, CZERO,
     $                     A( 1, K ), 1 )
               A( K, K ) = A( K, K ) - ZDOTU( K-1, WORK, 1, A( 1, K ),
     $                     1 )
            END IF
            KSTEP = 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Invert the diagonal block.
*
            T = A( K, K+1 )
            AK = A( K, K ) / T
            AKP1 = A( K+1, K+1 ) / T
            AKKP1 = A( K, K+1 ) / T
            D = T*( AK*AKP1-CONE )
            A( K, K ) = AKP1 / D
            A( K+1, K+1 ) = AK / D
            A( K, K+1 ) = -AKKP1 / D
*
*           Compute columns K and K+1 of the inverse.
```

```
*
            IF( K.GT.1 ) THEN
               CALL ZCOPY( K-1, A( 1, K ), 1, WORK, 1 )
               CALL ZSYMV( UPLO, K-1, -CONE, A, LDA, WORK, 1, CZERO,
     $                     A( 1, K ), 1 )
               A( K, K ) = A( K, K ) - ZDOTU( K-1, WORK, 1, A( 1, K ),
     $                     1 )
               A( K, K+1 ) = A( K, K+1 ) -
     $                       ZDOTU( K-1, A( 1, K ), 1, A( 1, K+1 ), 1 )
               CALL ZCOPY( K-1, A( 1, K+1 ), 1, WORK, 1 )
               CALL ZSYMV( UPLO, K-1, -CONE, A, LDA, WORK, 1, CZERO,
     $                     A( 1, K+1 ), 1 )
               A( K+1, K+1 ) = A( K+1, K+1 ) -
     $                         ZDOTU( K-1, WORK, 1, A( 1, K+1 ), 1 )
            END IF
            KSTEP = 2
         END IF
*
         IF( KSTEP.EQ.1 ) THEN
*
*           Interchange rows and columns K and IPIV(K) in the leading
*           submatrix A(1:k+1,1:k+1)
*
            KP = IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.GT.1 )
     $             CALL ZSWAP( KP-1, A( 1, K ), 1, A( 1, KP ), 1 )
               CALL ZSWAP( K-KP-1, A( KP+1, K ), 1, A( KP, KP+1 ), LDA )
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
            END IF
         ELSE
*
*           Interchange rows and columns K and K+1 with -IPIV(K) and
*           -IPIV(K+1)in the leading submatrix A(1:k+1,1:k+1)
*
            KP = -IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.GT.1 )
     $            CALL ZSWAP( KP-1, A( 1, K ), 1, A( 1, KP ), 1 )
               CALL ZSWAP( K-KP-1, A( KP+1, K ), 1, A( KP, KP+1 ), LDA )
*
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
               TEMP = A( K, K+1 )
               A( K, K+1 ) = A( KP, K+1 )
               A( KP, K+1 ) = TEMP
            END IF
```

```
*
            K = K + 1
            KP = -IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.GT.1 )
     $            CALL ZSWAP( KP-1, A( 1, K ), 1, A( 1, KP ), 1 )
               CALL ZSWAP( K-KP-1, A( KP+1, K ), 1, A( KP, KP+1 ), LDA )
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
            END IF
         END IF
*
         K = K + 1
         GO TO 30
   40    CONTINUE
*
      ELSE
*
*        Compute inv(A) from the factorization A = L*D*L**T.
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = N
   50    CONTINUE
*
*        If K < 1, exit from loop.
*
         IF( K.LT.1 )
     $      GO TO 60
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Invert the diagonal block.
*
            A( K, K ) = CONE / A( K, K )
*
*           Compute column K of the inverse.
*
            IF( K.LT.N ) THEN
               CALL ZCOPY( N-K, A( K+1, K ), 1, WORK, 1 )
               CALL ZSYMV( UPLO, N-K,-CONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                     CZERO, A( K+1, K ), 1 )
               A( K, K ) = A( K, K ) - ZDOTU( N-K, WORK, 1, A( K+1, K ),
     $                     1 )
            END IF
            KSTEP = 1
```

```
         ELSE
*
*           2 x 2 diagonal block
*
*           Invert the diagonal block.
*
            T = A( K, K-1 )
            AK = A( K-1, K-1 ) / T
            AKP1 = A( K, K ) / T
            AKKP1 = A( K, K-1 ) / T
            D = T*( AK*AKP1-CONE )
            A( K-1, K-1 ) = AKP1 / D
            A( K, K ) = AK / D
            A( K, K-1 ) = -AKKP1 / D
*
*           Compute columns K-1 and K of the inverse.
*
            IF( K.LT.N ) THEN
               CALL ZCOPY( N-K, A( K+1, K ), 1, WORK, 1 )
               CALL ZSYMV( UPLO, N-K,-CONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                     CZERO, A( K+1, K ), 1 )
               A( K, K ) = A( K, K ) - ZDOTU( N-K, WORK, 1, A( K+1, K ),
     $                     1 )
               A( K, K-1 ) = A( K, K-1 ) -
     $                       ZDOTU( N-K, A( K+1, K ), 1, A( K+1, K-1 ),
     $                       1 )
               CALL ZCOPY( N-K, A( K+1, K-1 ), 1, WORK, 1 )
               CALL ZSYMV( UPLO, N-K,-CONE, A( K+1, K+1 ), LDA, WORK, 1,
     $                     CZERO, A( K+1, K-1 ), 1 )
               A( K-1, K-1 ) = A( K-1, K-1 ) -
     $                         ZDOTU( N-K, WORK, 1, A( K+1, K-1 ), 1 )
            END IF
            KSTEP = 2
         END IF
*
         IF( KSTEP.EQ.1 ) THEN
*
*           Interchange rows and columns K and IPIV(K) in the trailing
*           submatrix A(k-1:n,k-1:n)
*
            KP = IPIV( K )
            IF( KP.NE.K ) THEN
               IF( KP.LT.N )
     $            CALL ZSWAP( N-KP, A( KP+1, K ), 1, A( KP+1, KP ), 1 )
               CALL ZSWAP( KP-K-1, A( K+1, K ), 1, A( KP, K+1 ), LDA )
               TEMP = A( K, K )
               A( K, K ) = A( KP, KP )
               A( KP, KP ) = TEMP
            END IF
         ELSE
```

```
*
*          Interchange rows and columns K and K-1 with -IPIV(K) and
*          -IPIV(K-1) in the trailing submatrix A(k-1:n,k-1:n)
*
           KP = -IPIV( K )
           IF( KP.NE.K ) THEN
              IF( KP.LT.N )
     $           CALL ZSWAP( N-KP, A( KP+1, K ), 1, A( KP+1, KP ), 1 )
              CALL ZSWAP( KP-K-1, A( K+1, K ), 1, A( KP, K+1 ), LDA )
*
              TEMP = A( K, K )
              A( K, K ) = A( KP, KP )
              A( KP, KP ) = TEMP
              TEMP = A( K, K-1 )
              A( K, K-1 ) = A( KP, K-1 )
              A( KP, K-1 ) = TEMP
           END IF
*
           K = K - 1
           KP = -IPIV( K )
           IF( KP.NE.K ) THEN
              IF( KP.LT.N )
     $           CALL ZSWAP( N-KP, A( KP+1, K ), 1, A( KP+1, KP ), 1 )
              CALL ZSWAP( KP-K-1, A( K+1, K ), 1, A( KP, K+1 ), LDA )
              TEMP = A( K, K )
              A( K, K ) = A( KP, KP )
              A( KP, KP ) = TEMP
           END IF
        END IF
*
        K = K - 1
        GO TO 50
   60   CONTINUE
      END IF
*
      RETURN
*
*     End of ZSYTRI_ROOK
*
      END
```

———-

61.21.4 lisp code

— zsytri_rook.lisp —

```
(let* ((cone (f2cl-lib:cmplx 1.0d0 0.0d0)) (czero (f2cl-lib:cmplx 0.0d0 0.0d0)))
```

```
(declare (type (f2cl-lib:complex16) cone)
         (type (f2cl-lib:complex16) czero)
         (ignorable cone czero))
(defun zsytri_rook (uplo n a lda ipiv work info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex16 (*)) work a)
           (type (f2cl-lib:integer4) info lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((ak #C(0.0d0 0.0d0)) (akkp1 #C(0.0d0 0.0d0))
           (akp1 #C(0.0d0 0.0d0)) (d #C(0.0d0 0.0d0)) (t$ #C(0.0d0 0.0d0))
           (temp #C(0.0d0 0.0d0)) (k 0) (kp 0) (kstep 0) (upper nil))
      (declare (type (f2cl-lib:complex16) ak akkp1 akp1 d t$ temp)
               (type (f2cl-lib:integer4) k kp kstep)
               (type f2cl-lib:logical upper))
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (cond
       ((and (not upper)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -4)))
      (cond
       ((/= info 0) (xerbla "ZSYTRI_ROOK" (f2cl-lib:int-sub info))
        (go end_label)))
      (if (= n 0)
          (go end_label))
      (cond
       (upper
        (f2cl-lib:fdo (info n (f2cl-lib:int-add info (f2cl-lib:int-sub 1)))
                      ((> info 1) nil)
          (tagbody
            (if (and
                 (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
```

```
                0)
              (=
               (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                              a-%offset%)
               czero))
             (go end_label))
         label10)))
     (t
      (f2cl-lib:fdo (info 1 (f2cl-lib:int-add info 1))
                    ((> info n) nil)
        (tagbody
          (if (and
               (> (f2cl-lib:fref ipiv-%data% (info) ((1 *)) ipiv-%offset%)
                  0)
               (=
                (f2cl-lib:fref a-%data% (info info) ((1 lda) (1 *))
                               a-%offset%)
                czero))
             (go end_label))
         label20))))
    (setf info 0)
    (cond
     (upper
      (tagbody
        (setf k 1)
       label30
        (if (> k n)
            (go label40))
        (cond
         ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
          (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                  (/ cone
                     (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                    a-%offset%)))
          (cond
           ((> k 1)
            (zcopy (f2cl-lib:int-sub k 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                                   ((1 lda) (1 *)) a-%offset%)
             1 work 1)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9)
                (zsymv uplo (f2cl-lib:int-sub k 1) (- cone) a lda work 1
                 czero
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                                       ((1 lda) (1 *)) a-%offset%)
                 1)
              (declare
               (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
        var-9))
      (setf uplo var-0))
    (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
            (-
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)
             (zdotu (f2cl-lib:int-sub k 1) work 1
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                    (1 k) ((1 lda) (1 *))
                                    a-%offset%)
              1)))))
  (setf kstep 1))
 (t
  (setf t$
          (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                         ((1 lda) (1 *)) a-%offset%))
  (setf ak
          (/
           (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                          a-%offset%)
           t$))
  (setf akp1
          (/
           (f2cl-lib:fref a-%data%
                          ((f2cl-lib:int-add k 1)
                           (f2cl-lib:int-add k 1))
                          ((1 lda) (1 *)) a-%offset%)
           t$))
  (setf akkp1
          (/
           (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                          ((1 lda) (1 *)) a-%offset%)
           t$))
  (setf d (* t$ (- (* ak akp1) cone)))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
          (/ akp1 d))
  (setf (f2cl-lib:fref a-%data%
                       ((f2cl-lib:int-add k 1)
                        (f2cl-lib:int-add k 1))
                       ((1 lda) (1 *)) a-%offset%)
          (/ ak d))
  (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                       ((1 lda) (1 *)) a-%offset%)
          (/ (- akkp1) d))
  (cond
   ((> k 1)
    (zcopy (f2cl-lib:int-sub k 1)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                           ((1 lda) (1 *)) a-%offset%)
     1 work 1)
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zsymv uplo (f2cl-lib:int-sub k 1) (- cone) a lda work 1
     czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                           ((1 lda) (1 *)) a-%offset%)
     1)
  (declare
   (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
    var-9))
  (setf uplo var-0))
(setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         (zdotu (f2cl-lib:int-sub k 1) work 1
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                (1 k) ((1 lda) (1 *))
                                a-%offset%)
          1)))
(setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                      ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
         (zdotu (f2cl-lib:int-sub k 1)
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                (1 k) ((1 lda) (1 *))
                                a-%offset%)
          1
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                (1 (f2cl-lib:int-add k 1))
                                ((1 lda) (1 *)) a-%offset%)
          1)))
(zcopy (f2cl-lib:int-sub k 1)
 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                       (1 (f2cl-lib:int-add k 1))
                       ((1 lda) (1 *)) a-%offset%)
 1 work 1)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (zsymv uplo (f2cl-lib:int-sub k 1) (- cone) a lda work 1
     czero
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                           (1 (f2cl-lib:int-add k 1))
                           ((1 lda) (1 *)) a-%offset%)
     1)
  (declare
```

```
          (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9))
         (setf uplo var-0))
       (setf (f2cl-lib:fref a-%data%
                            ((f2cl-lib:int-add k 1)
                             (f2cl-lib:int-add k 1))
                            ((1 lda) (1 *)) a-%offset%)
               (-
                (f2cl-lib:fref a-%data%
                               ((f2cl-lib:int-add k 1)
                                (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%)
                (zdotu (f2cl-lib:int-sub k 1) work 1
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                       (1 (f2cl-lib:int-add k 1))
                                       ((1 lda) (1 *)) a-%offset%)
                 1)))))
     (setf kstep 2)))
   (cond
    ((= kstep 1)
     (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
     (cond
      ((/= kp k)
       (if (> kp 1)
           (zswap (f2cl-lib:int-sub kp 1)
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                                  ((1 lda) (1 *)) a-%offset%)
            1
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 kp)
                                  ((1 lda) (1 *)) a-%offset%)
            1))
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (zswap (f2cl-lib:int-sub k kp 1)
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                  ((+ kp 1) k) ((1 lda) (1 *))
                                  a-%offset%)
            1
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                  (kp (f2cl-lib:int-add kp 1))
                                  ((1 lda) (1 *)) a-%offset%)
            lda)
         (declare (ignore var-0 var-1 var-2 var-3))
         (when var-4 (setf lda var-4)))
       (setf temp
               (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                              a-%offset%))
       (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
               (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                              a-%offset%))
       (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
```

```
                        a-%offset%)
          temp))))
(t
 (setf kp
         (f2cl-lib:int-sub
          (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
 (cond
  ((/= kp k)
   (if (> kp 1)
       (zswap (f2cl-lib:int-sub kp 1)
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                              ((1 lda) (1 *)) a-%offset%)
        1
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 kp)
                              ((1 lda) (1 *)) a-%offset%)
        1))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
       (zswap (f2cl-lib:int-sub k kp 1)
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                              ((+ kp 1) k) ((1 lda) (1 *))
                              a-%offset%)
        1
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                              (kp (f2cl-lib:int-add kp 1))
                              ((1 lda) (1 *)) a-%offset%)
        lda)
     (declare (ignore var-0 var-1 var-2 var-3))
     (when var-4 (setf lda var-4)))
   (setf temp
           (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                          a-%offset%))
   (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
           (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                          a-%offset%))
   (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                        a-%offset%)
           temp)
   (setf temp
           (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                          ((1 lda) (1 *)) a-%offset%))
   (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
           (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-add k 1))
                          ((1 lda) (1 *)) a-%offset%))
   (setf (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
           temp)))
 (setf k (f2cl-lib:int-add k 1))
 (setf kp
         (f2cl-lib:int-sub
```

```
                (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
        (cond
         ((/= kp k)
          (if (> kp 1)
              (zswap (f2cl-lib:int-sub kp 1)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                                     ((1 lda) (1 *)) a-%offset%)
               1
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 kp)
                                     ((1 lda) (1 *)) a-%offset%)
               1))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (zswap (f2cl-lib:int-sub k kp 1)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                     ((+ kp 1) k) ((1 lda) (1 *))
                                     a-%offset%)
               1
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                     (kp (f2cl-lib:int-add kp 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda)
            (declare (ignore var-0 var-1 var-2 var-3))
            (when var-4 (setf lda var-4)))
          (setf temp
                  (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                 a-%offset%))
          (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                  (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                                 a-%offset%))
          (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                               a-%offset%)
                  temp)))))
     (setf k (f2cl-lib:int-add k 1))
     (go label30)
    label40))
  (t
   (tagbody
     (setf k n)
    label50
     (if (< k 1)
         (go label60))
     (cond
      ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
       (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
               (/ cone
                  (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                 a-%offset%)))
       (cond
        ((< k n)
         (zcopy (f2cl-lib:int-sub n k)
```

```
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 ((+ k 1) k)
                           ((1 lda) (1 *)) a-%offset%)
     1 work 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9)
        (zsymv uplo (f2cl-lib:int-sub n k) (- cone)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               ((+ k 1) (f2cl-lib:int-add k 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda work 1 czero
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               ((+ k 1) k) ((1 lda) (1 *))
                               a-%offset%)
         1)
      (declare
       (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9))
      (setf uplo var-0))
    (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
            (-
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)
             (zdotu (f2cl-lib:int-sub n k) work 1
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                    ((+ k 1) k) ((1 lda) (1 *))
                                    a-%offset%)
              1)))))
  (setf kstep 1))
 (t
  (setf t$
          (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                         ((1 lda) (1 *)) a-%offset%))
  (setf ak
          (/
           (f2cl-lib:fref a-%data%
                          ((f2cl-lib:int-sub k 1)
                           (f2cl-lib:int-sub k 1))
                          ((1 lda) (1 *)) a-%offset%)
           t$))
  (setf akp1
          (/
           (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                          a-%offset%)
           t$))
  (setf akkp1
          (/
           (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                          ((1 lda) (1 *)) a-%offset%)
           t$))
```

```
(setf d (* t$ (- (* ak akp1) cone)))
(setf (f2cl-lib:fref a-%data%
                     ((f2cl-lib:int-sub k 1)
                      (f2cl-lib:int-sub k 1))
                     ((1 lda) (1 *)) a-%offset%)
        (/ akp1 d))
(setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (/ ak d))
(setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                     ((1 lda) (1 *)) a-%offset%)
        (/ (- akkp1) d))
(cond
 ((< k n)
  (zcopy (f2cl-lib:int-sub n k)
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 ((+ k 1) k)
                         ((1 lda) (1 *)) a-%offset%)
   1 work 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (zsymv uplo (f2cl-lib:int-sub n k) (- cone)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             ((+ k 1) (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda work 1 czero
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             ((+ k 1) k) ((1 lda) (1 *))
                             a-%offset%)
       1)
    (declare
     (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9))
    (setf uplo var-0))
  (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         (zdotu (f2cl-lib:int-sub n k) work 1
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                ((+ k 1) k) ((1 lda) (1 *))
                                a-%offset%)
          1)))
  (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                       ((1 lda) (1 *)) a-%offset%)
        (-
         (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         (zdotu (f2cl-lib:int-sub n k)
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                ((+ k 1) k) ((1 lda) (1 *))
```

```
                                        a-%offset%)
                  1
                  (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                        ((+ k 1)
                                         (f2cl-lib:int-sub k 1))
                                        ((1 lda) (1 *)) a-%offset%)
                  1)))
        (zcopy (f2cl-lib:int-sub n k)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               ((+ k 1) (f2cl-lib:int-sub k 1))
                               ((1 lda) (1 *)) a-%offset%)
         1 work 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9)
            (zsymv uplo (f2cl-lib:int-sub n k) (- cone)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   ((+ k 1) (f2cl-lib:int-add k 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda work 1 czero
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   ((+ k 1) (f2cl-lib:int-sub k 1))
                                   ((1 lda) (1 *)) a-%offset%)
             1)
          (declare
           (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
            var-9))
          (setf uplo var-0))
        (setf (f2cl-lib:fref a-%data%
                             ((f2cl-lib:int-sub k 1)
                              (f2cl-lib:int-sub k 1))
                             ((1 lda) (1 *)) a-%offset%)
              (-
               (f2cl-lib:fref a-%data%
                              ((f2cl-lib:int-sub k 1)
                               (f2cl-lib:int-sub k 1))
                              ((1 lda) (1 *)) a-%offset%)
               (zdotu (f2cl-lib:int-sub n k) work 1
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                      ((+ k 1)
                                       (f2cl-lib:int-sub k 1))
                                      ((1 lda) (1 *)) a-%offset%)
                1)))))
      (setf kstep 2)))
    (cond
     ((= kstep 1)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (cond
       ((/= kp k)
        (if (< kp n)
```

```
            (zswap (f2cl-lib:int-sub n kp)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   ((+ kp 1) k) ((1 lda) (1 *))
                                   a-%offset%)
             1
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   ((+ kp 1) kp) ((1 lda) (1 *))
                                   a-%offset%)
             1))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (zswap (f2cl-lib:int-sub kp k 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   ((+ k 1) k) ((1 lda) (1 *))
                                   a-%offset%)
             1
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   (kp (f2cl-lib:int-add k 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-1 var-2 var-3))
          (when var-4 (setf lda var-4)))
        (setf temp
                (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                               a-%offset%))
        (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                               a-%offset%))
        (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                             a-%offset%)
                temp))))
    (t
     (setf kp
             (f2cl-lib:int-sub
              (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
     (cond
      ((/= kp k)
       (if (< kp n)
            (zswap (f2cl-lib:int-sub n kp)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   ((+ kp 1) k) ((1 lda) (1 *))
                                   a-%offset%)
             1
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   ((+ kp 1) kp) ((1 lda) (1 *))
                                   a-%offset%)
             1))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (zswap (f2cl-lib:int-sub kp k 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   ((+ k 1) k) ((1 lda) (1 *))
```

```
                                a-%offset%)
        1
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                (kp (f2cl-lib:int-add k 1))
                                ((1 lda) (1 *)) a-%offset%)
        lda)
      (declare (ignore var-0 var-1 var-2 var-3))
      (when var-4 (setf lda var-4)))
    (setf temp
            (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                           a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                         a-%offset%)
            temp)
    (setf temp
            (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                           ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% (k (f2cl-lib:int-sub k 1))
                         ((1 lda) (1 *)) a-%offset%)
            (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-sub k 1))
                           ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% (kp (f2cl-lib:int-sub k 1))
                         ((1 lda) (1 *)) a-%offset%)
            temp)))
  (setf k (f2cl-lib:int-sub k 1))
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
  (cond
   ((/= kp k)
    (if (< kp n)
        (zswap (f2cl-lib:int-sub n kp)
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                ((+ kp 1) k) ((1 lda) (1 *))
                                a-%offset%)
        1
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                ((+ kp 1) kp) ((1 lda) (1 *))
                                a-%offset%)
        1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zswap (f2cl-lib:int-sub kp k 1)
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                ((+ k 1) k) ((1 lda) (1 *))
                                a-%offset%)
        1
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
```

```
                                        (kp (f2cl-lib:int-add k 1))
                                        ((1 lda) (1 *)) a-%offset%)
                    lda)
                  (declare (ignore var-0 var-1 var-2 var-3))
                  (when var-4 (setf lda var-4)))
                (setf temp
                        (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                       a-%offset%))
                (setf (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *)) a-%offset%)
                        (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                                       a-%offset%))
                (setf (f2cl-lib:fref a-%data% (kp kp) ((1 lda) (1 *))
                                     a-%offset%)
                        temp)))))
            (setf k (f2cl-lib:int-sub k 1))
            (go label50)
           label60)))
        (go end_label)
       end_label
        (return (values uplo nil nil lda nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsytri_rook
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::zsymv
                                              fortran-to-lisp::xerbla))))
```

———-

61.22 zsytrs

61.22.1 Axiom unit tests

— zsytrs.input —

```
)set break resume
)sys rm -f zsytrs.output
)spool zsytrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.22.2 Axiom help page

— zsytrs.help —

```
======================================================================
zsytrs examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zsytrs (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO)
           ZSYTRS

Function/Subroutine Documentation
   subroutine zsytrs (character UPLO, integer N, integer NRHS, complex*16,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       complex*16, dimension( ldb, * ) B, integer LDB, integer INFO)
       ZSYTRS

Purpose:
   ZSYTRS solves a system of linear equations A*X = B with a complex
   symmetric matrix A using the factorization A = U*D*U**T or
   A = L*D*L**T computed by ZSYTRF.
```

```
Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by ZSYTRF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by ZSYTRF.

 B
   B is COMPLEX*16 array, dimension (LDB,NRHS)
   On entry, the right hand side matrix B.
   On exit, the solution matrix X.

 LDB
   LDB is INTEGER
   The leading dimension of the array B.  LDB >= max(1,N).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

61.22.3 fortran code

— zsytrs.f —

```
*  =====================================================================
      SUBROUTINE ZSYTRS( UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            J, K, KP
      COMPLEX*16         AK, AKM1, AKM1K, BK, BKM1, DENOM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGEMV, ZGERU, ZSCAL, ZSWAP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
```

```
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYTRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*        Solve A*X = B, where A = U*D*U**T.
*
*        First solve U*D*X = B, overwriting B with X.
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop.
*
         IF( K.LT.1 )
     $      GO TO 30
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
*           Multiply by inv(U(K)), where U(K) is the transformation
*           stored in column K of A.
*
            CALL ZGERU( K-1, NRHS, -ONE, A( 1, K ), 1, B( K, 1 ), LDB,
     $                  B( 1, 1 ), LDB )
```

```
*
*          Multiply by the inverse of the diagonal block.
*
           CALL ZSCAL( NRHS, ONE / A( K, K ), B( K, 1 ), LDB )
           K = K - 1
        ELSE
*
*          2 x 2 diagonal block
*
*          Interchange rows K-1 and -IPIV(K).
*
           KP = -IPIV( K )
           IF( KP.NE.K-1 )
     $        CALL ZSWAP( NRHS, B( K-1, 1 ), LDB, B( KP, 1 ), LDB )
*
*          Multiply by inv(U(K)), where U(K) is the transformation
*          stored in columns K-1 and K of A.
*
           CALL ZGERU( K-2, NRHS, -ONE, A( 1, K ), 1, B( K, 1 ), LDB,
     $                 B( 1, 1 ), LDB )
           CALL ZGERU( K-2, NRHS, -ONE, A( 1, K-1 ), 1, B( K-1, 1 ),
     $                 LDB, B( 1, 1 ), LDB )
*
*          Multiply by the inverse of the diagonal block.
*
           AKM1K = A( K-1, K )
           AKM1 = A( K-1, K-1 ) / AKM1K
           AK = A( K, K ) / AKM1K
           DENOM = AKM1*AK - ONE
           DO 20 J = 1, NRHS
              BKM1 = B( K-1, J ) / AKM1K
              BK = B( K, J ) / AKM1K
              B( K-1, J ) = ( AK*BKM1-BK ) / DENOM
              B( K, J ) = ( AKM1*BK-BKM1 ) / DENOM
   20      CONTINUE
           K = K - 2
        END IF
*
        GO TO 10
   30   CONTINUE
*
*       Next solve U**T *X = B, overwriting B with X.
*
*       K is the main loop index, increasing from 1 to N in steps of
*       1 or 2, depending on the size of the diagonal blocks.
*
        K = 1
   40   CONTINUE
*
*       If K > N, exit from loop.
```

```
*
         IF( K.GT.N )
     $      GO TO 50
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Multiply by inv(U**T(K)), where U(K) is the transformation
*           stored in column K of A.
*
            CALL ZGEMV( 'Transpose', K-1, NRHS, -ONE, B, LDB, A( 1, K ),
     $                  1, ONE, B( K, 1 ), LDB )
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K = K + 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Multiply by inv(U**T(K+1)), where U(K+1) is the transformation
*           stored in columns K and K+1 of A.
*
            CALL ZGEMV( 'Transpose', K-1, NRHS, -ONE, B, LDB, A( 1, K ),
     $                  1, ONE, B( K, 1 ), LDB )
            CALL ZGEMV( 'Transpose', K-1, NRHS, -ONE, B, LDB,
     $                  A( 1, K+1 ), 1, ONE, B( K+1, 1 ), LDB )
*
*           Interchange rows K and -IPIV(K).
*
            KP = -IPIV( K )
            IF( KP.NE.K )
     $         CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K = K + 2
         END IF
*
         GO TO 40
   50    CONTINUE
*
      ELSE
*
*        Solve A*X = B, where A = L*D*L**T.
*
*        First solve L*D*X = B, overwriting B with X.
*
*        K is the main loop index, increasing from 1 to N in steps of
```

```
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = 1
   60    CONTINUE
*
*        If K > N, exit from loop.
*
         IF( K.GT.N )
     $      GO TO 80
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
*           Multiply by inv(L(K)), where L(K) is the transformation
*           stored in column K of A.
*
            IF( K.LT.N )
     $         CALL ZGERU( N-K, NRHS, -ONE, A( K+1, K ), 1, B( K, 1 ),
     $                     LDB, B( K+1, 1 ), LDB )
*
*           Multiply by the inverse of the diagonal block.
*
            CALL ZSCAL( NRHS, ONE / A( K, K ), B( K, 1 ), LDB )
            K = K + 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Interchange rows K+1 and -IPIV(K).
*
            KP = -IPIV( K )
            IF( KP.NE.K+1 )
     $         CALL ZSWAP( NRHS, B( K+1, 1 ), LDB, B( KP, 1 ), LDB )
*
*           Multiply by inv(L(K)), where L(K) is the transformation
*           stored in columns K and K+1 of A.
*
            IF( K.LT.N-1 ) THEN
               CALL ZGERU( N-K-1, NRHS, -ONE, A( K+2, K ), 1, B( K, 1 ),
     $                     LDB, B( K+2, 1 ), LDB )
               CALL ZGERU( N-K-1, NRHS, -ONE, A( K+2, K+1 ), 1,
     $                     B( K+1, 1 ), LDB, B( K+2, 1 ), LDB )
            END IF
```

```
*
*          Multiply by the inverse of the diagonal block.
*
           AKM1K = A( K+1, K )
           AKM1 = A( K, K ) / AKM1K
           AK = A( K+1, K+1 ) / AKM1K
           DENOM = AKM1*AK - ONE
           DO 70 J = 1, NRHS
              BKM1 = B( K, J ) / AKM1K
              BK = B( K+1, J ) / AKM1K
              B( K, J ) = ( AK*BKM1-BK ) / DENOM
              B( K+1, J ) = ( AKM1*BK-BKM1 ) / DENOM
   70      CONTINUE
           K = K + 2
        END IF
*
        GO TO 60
   80   CONTINUE
*
*       Next solve L**T *X = B, overwriting B with X.
*
*       K is the main loop index, decreasing from N to 1 in steps of
*       1 or 2, depending on the size of the diagonal blocks.
*
        K = N
   90   CONTINUE
*
*       If K < 1, exit from loop.
*
        IF( K.LT.1 )
     $     GO TO 100
*
        IF( IPIV( K ).GT.0 ) THEN
*
*          1 x 1 diagonal block
*
*          Multiply by inv(L**T(K)), where L(K) is the transformation
*          stored in column K of A.
*
           IF( K.LT.N )
     $        CALL ZGEMV( 'Transpose', N-K, NRHS, -ONE, B( K+1, 1 ),
     $                    LDB, A( K+1, K ), 1, ONE, B( K, 1 ), LDB )
*
*          Interchange rows K and IPIV(K).
*
           KP = IPIV( K )
           IF( KP.NE.K )
     $        CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
           K = K - 1
        ELSE
```

```
*
*          2 x 2 diagonal block
*
*          Multiply by inv(L**T(K-1)), where L(K-1) is the transformation
*          stored in columns K-1 and K of A.
*
           IF( K.LT.N ) THEN
              CALL ZGEMV( 'Transpose', N-K, NRHS, -ONE, B( K+1, 1 ),
     $                    LDB, A( K+1, K ), 1, ONE, B( K, 1 ), LDB )
              CALL ZGEMV( 'Transpose', N-K, NRHS, -ONE, B( K+1, 1 ),
     $                    LDB, A( K+1, K-1 ), 1, ONE, B( K-1, 1 ),
     $                    LDB )
           END IF
*
*          Interchange rows K and -IPIV(K).
*
           KP = -IPIV( K )
           IF( KP.NE.K )
     $        CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
           K = K - 2
        END IF
*
        GO TO 90
  100   CONTINUE
      END IF
*
      RETURN
*
*     End of ZSYTRS
*
      END
```

———-

61.22.4 lisp code

— **zsytrs.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (ignorable one))
  (defun zsytrs (uplo n nrhs a lda ipiv b ldb$ info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) b a)
             (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
```

```
    (a f2cl-lib:complex16 a-%data% a-%offset%)
    (b f2cl-lib:complex16 b-%data% b-%offset%)
    (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
  (prog ((ak #C(0.0d0 0.0d0)) (akm1 #C(0.0d0 0.0d0))
         (akm1k #C(0.0d0 0.0d0)) (bk #C(0.0d0 0.0d0))
         (bkm1 #C(0.0d0 0.0d0)) (denom #C(0.0d0 0.0d0)) (j 0) (k 0) (kp 0)
         (upper nil))
    (declare (type (f2cl-lib:complex16) ak akm1 akm1k bk bkm1 denom)
             (type (f2cl-lib:integer4) j k kp)
             (type f2cl-lib:logical upper))
    (setf info 0)
    (setf upper
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
    (cond
     ((and (not upper)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
      (setf info -1))
     ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
     ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -5))
     ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -8)))
    (cond
     ((/= info 0) (xerbla "ZSYTRS" (f2cl-lib:int-sub info))
      (go end_label)))
    (if (or (= n 0) (= nrhs 0))
        (go end_label))
    (cond
     (upper
      (tagbody
        (setf k n)
       label10
        (if (< k 1)
            (go label30))
        (cond
         ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
          (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
          (if (/= kp k)
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (zswap nrhs
                   (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
```

```
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
       (declare (ignore var-1 var-3))
       (when var-0 (setf nrhs var-0))
       (when var-2 (setf ldb$ var-2))
       (when var-4 (setf ldb$ var-4))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (zgeru (f2cl-lib:int-sub k 1) nrhs (- one)
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                            ((1 lda) (1 *)) a-%offset%)
      1
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
   (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
   (when var-1 (setf nrhs var-1))
   (when var-6 (setf ldb$ var-6))
   (when var-8 (setf ldb$ var-8)))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (zscal nrhs
      (/ one
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%))
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
   (declare (ignore var-1 var-2))
   (when var-0 (setf nrhs var-0))
   (when var-3 (setf ldb$ var-3)))
 (setf k (f2cl-lib:int-sub k 1)))
(t
 (setf kp
         (f2cl-lib:int-sub
          (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
 (if (/= kp (f2cl-lib:int-sub k 1))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (zswap nrhs
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                ((+ k (f2cl-lib:int-sub 1)) 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                ((1 ldb$) (1 *)) b-%offset%)
```

```
          ldb$)
       (declare (ignore var-1 var-3))
       (when var-0 (setf nrhs var-0))
       (when var-2 (setf ldb$ var-2))
       (when var-4 (setf ldb$ var-4))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zgeru (f2cl-lib:int-sub k 2) nrhs (- one)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                           ((1 lda) (1 *)) a-%offset%)
     1
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$)
  (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
  (when var-1 (setf nrhs var-1))
  (when var-6 (setf ldb$ var-6))
  (when var-8 (setf ldb$ var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zgeru (f2cl-lib:int-sub k 2) nrhs (- one)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                           (1 (f2cl-lib:int-sub k 1))
                           ((1 lda) (1 *)) a-%offset%)
     1
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                           ((+ k (f2cl-lib:int-sub 1)) 1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 1)
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$)
  (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
  (when var-1 (setf nrhs var-1))
  (when var-6 (setf ldb$ var-6))
  (when var-8 (setf ldb$ var-8)))
(setf akm1k
        (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                       ((1 lda) (1 *)) a-%offset%))
(setf akm1
        (/
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-sub k 1)
                         (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         akm1k))
(setf ak
```

```
            (/
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)
             akm1k))
    (setf denom (- (* akm1 ak) one))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (setf bkm1
                (/
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1) j)
                                ((1 ldb$) (1 *)) b-%offset%)
                 akm1k))
        (setf bk
                (/
                 (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                b-%offset%)
                 akm1k))
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)
                (/ (- (* ak bkm1) bk) denom))
        (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                             b-%offset%)
                (/ (- (* akm1 bk) bkm1) denom))
       label20))
    (setf k (f2cl-lib:int-sub k 2))))
  (go label10)
 label30
  (setf k 1)
 label40
  (if (> k n)
      (go label50))
  (cond
   ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (zgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- one) b ldb$
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                               ((1 lda) (1 *)) a-%offset%)
         1 one
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
      (when var-2 (setf nrhs var-2))
      (when var-5 (setf ldb$ var-5))
      (when var-8 (setf one var-8))
      (when var-10 (setf ldb$ var-10)))
    (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
```

```
 (if (/= kp k)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (zswap nrhs
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
       (declare (ignore var-1 var-3))
       (when var-0 (setf nrhs var-0))
       (when var-2 (setf ldb$ var-2))
       (when var-4 (setf ldb$ var-4))))
 (setf k (f2cl-lib:int-add k 1)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (zgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- one) b ldb$
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                            ((1 lda) (1 *)) a-%offset%)
      1 one
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
   (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
   (when var-2 (setf nrhs var-2))
   (when var-5 (setf ldb$ var-5))
   (when var-8 (setf one var-8))
   (when var-10 (setf ldb$ var-10)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (zgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- one) b ldb$
      (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                            (1 (f2cl-lib:int-add k 1))
                            ((1 lda) (1 *)) a-%offset%)
      1 one
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                            ((+ k 1) 1) ((1 ldb$) (1 *))
                            b-%offset%)
      ldb$)
   (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
   (when var-2 (setf nrhs var-2))
   (when var-5 (setf ldb$ var-5))
   (when var-8 (setf one var-8))
   (when var-10 (setf ldb$ var-10)))
 (setf kp
         (f2cl-lib:int-sub
          (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
```

```
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (zswap nrhs
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-add k 2))))
    (go label40)
   label50))
 (t
  (tagbody
    (setf k 1)
   label60
    (if (> k n)
        (go label80))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (zswap nrhs
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (if (< k n)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (zgeru (f2cl-lib:int-sub n k) nrhs (- one)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                     ((+ k 1) k) ((1 lda) (1 *))
                                     a-%offset%)
               1
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
```

```
                                  ((+ k 1) 1) ((1 ldb$) (1 *))
                                  b-%offset%)
            ldb$)
          (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
          (when var-1 (setf nrhs var-1))
          (when var-6 (setf ldb$ var-6))
          (when var-8 (setf ldb$ var-8))))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (zscal nrhs
      (/ one
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%))
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
   (declare (ignore var-1 var-2))
   (when var-0 (setf nrhs var-0))
   (when var-3 (setf ldb$ var-3)))
 (setf k (f2cl-lib:int-add k 1)))
(t
 (setf kp
         (f2cl-lib:int-sub
          (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
 (if (/= kp (f2cl-lib:int-add k 1))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (zswap nrhs
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                ((+ k 1) 1) ((1 ldb$) (1 *))
                                b-%offset%)
          ldb$
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
       (declare (ignore var-1 var-3))
       (when var-0 (setf nrhs var-0))
       (when var-2 (setf ldb$ var-2))
       (when var-4 (setf ldb$ var-4))))
 (cond
  ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (zgeru (f2cl-lib:int-sub n k 1) nrhs (- one)
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                              ((+ k 2) k) ((1 lda) (1 *))
                              a-%offset%)
        1
        (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$
        (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
```

```
                                ((+ k 2) 1) ((1 ldb$) (1 *))
                                b-%offset%)
        ldb$)
      (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
      (when var-1 (setf nrhs var-1))
      (when var-6 (setf ldb$ var-6))
      (when var-8 (setf ldb$ var-8)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (zgeru (f2cl-lib:int-sub n k 1) nrhs (- one)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                ((+ k 2) (f2cl-lib:int-add k 1))
                                ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                ((+ k 1) 1) ((1 ldb$) (1 *))
                                b-%offset%)
         ldb$
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                ((+ k 2) 1) ((1 ldb$) (1 *))
                                b-%offset%)
         ldb$)
      (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
      (when var-1 (setf nrhs var-1))
      (when var-6 (setf ldb$ var-6))
      (when var-8 (setf ldb$ var-8)))))
(setf akm1k
        (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                       ((1 lda) (1 *)) a-%offset%))
(setf akm1
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         akm1k))
(setf ak
        (/
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-add k 1)
                         (f2cl-lib:int-add k 1))
                        ((1 lda) (1 *)) a-%offset%)
         akm1k))
(setf denom (- (* akm1 ak) one))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf bkm1
            (/
             (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                            b-%offset%)
             akm1k))
```

```
        (setf bk
                (/
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                                ((1 ldb$) (1 *)) b-%offset%)
                 akm1k))
        (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                             b-%offset%)
                (/ (- (* ak bkm1) bk) denom))
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)
                (/ (- (* akm1 bk) bkm1) denom))
      label70))
    (setf k (f2cl-lib:int-add k 2))))
  (go label60)
 label80
  (setf k n)
 label90
  (if (< k 1)
      (go label100))
  (cond
   ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
    (if (< k n)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10)
            (zgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- one)
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                   ((+ k 1) 1) ((1 ldb$) (1 *))
                                   b-%offset%)
             ldb$
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   ((+ k 1) k) ((1 lda) (1 *))
                                   a-%offset%)
             1 one
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$)
          (declare
           (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
          (when var-2 (setf nrhs var-2))
          (when var-5 (setf ldb$ var-5))
          (when var-8 (setf one var-8))
          (when var-10 (setf ldb$ var-10))))
    (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
    (if (/= kp k)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (zswap nrhs
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$
```

```
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
        (declare (ignore var-1 var-3))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
 (setf k (f2cl-lib:int-sub k 1)))
(t
 (cond
  ((< k n)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (zgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- one)
        (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                              ((+ k 1) 1) ((1 ldb$) (1 *))
                              b-%offset%)
        ldb$
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                              ((+ k 1) k) ((1 lda) (1 *))
                              a-%offset%)
        1 one
        (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$)
     (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
     (when var-2 (setf nrhs var-2))
     (when var-5 (setf ldb$ var-5))
     (when var-8 (setf one var-8))
     (when var-10 (setf ldb$ var-10)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (zgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- one)
        (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                              ((+ k 1) 1) ((1 ldb$) (1 *))
                              b-%offset%)
        ldb$
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                              ((+ k 1) (f2cl-lib:int-sub k 1))
                              ((1 lda) (1 *)) a-%offset%)
        1 one
        (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                              ((+ k (f2cl-lib:int-sub 1)) 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$)
     (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
     (when var-2 (setf nrhs var-2))
     (when var-5 (setf ldb$ var-5))
```

```
                (when var-8 (setf one var-8))
                (when var-10 (setf ldb$ var-10)))))
            (setf kp
                    (f2cl-lib:int-sub
                     (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
            (if (/= kp k)
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (zswap nrhs
                     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                           ((1 ldb$) (1 *)) b-%offset%)
                     ldb$
                     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                           ((1 ldb$) (1 *)) b-%offset%)
                     ldb$)
                  (declare (ignore var-1 var-3))
                  (when var-0 (setf nrhs var-0))
                  (when var-2 (setf ldb$ var-2))
                  (when var-4 (setf ldb$ var-4))))
            (setf k (f2cl-lib:int-sub k 2))))
          (go label90)
         label100)))
       (go end_label)
      end_label
       (return (values uplo nil nrhs nil nil nil nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsytrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil
                                              fortran-to-lisp::nrhs nil nil nil
                                              nil fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

61.23 zsytrs2

61.23.1 Axiom unit tests

— **zsytrs2.input** —

```
)set break resume
)sys rm -f zsytrs2.output
)spool zsytrs2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.23.2 Axiom help page

— **zsytrs2.help** —

```
====================================================================
zsytrs2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zsytrs2 (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, WORK, INFO)
           ZSYTRS2

Function/Subroutine Documentation
   subroutine zsytrs2 (character UPLO, integer N, integer NRHS, complex*16,
       dimension( lda, * ) A, integer LDA, integer, dimension( * ) IPIV,
       complex*16, dimension( ldb, * ) B, integer LDB, complex*16, dimension(
       * ) WORK, integer INFO)
```

```
      ZSYTRS2

Purpose:
   ZSYTRS2 solves a system of linear equations A*X = B with a real
   symmetric matrix A using the factorization A = U*D*U**T or
   A = L*D*L**T computed by ZSYTRF and converted by ZSYCONV.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by ZSYTRF.
   Note that A is input / output. This might be counter-intuitive,
   and one may think that A is input only. A is input / output. This
   is because, at the start of the subroutine, we permute A in a
   "better" form and then we permute A back to its original form at the end.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by ZSYTRF.

 B
   B is COMPLEX*16 array, dimension (LDB,NRHS)
   On entry, the right hand side matrix B.
   On exit, the solution matrix X.

 LDB
   LDB is INTEGER
```

```
  The leading dimension of the array B.  LDB >= max(1,N).

 WORK
  WORK is REAL array, dimension (N)

 INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

61.23.3 fortran code

— **zsytrs2.f** —

```
*  =====================================================================
      SUBROUTINE ZSYTRS2( UPLO, N, NRHS, A, LDA, IPIV, B, LDB,
     $                    WORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16       A( LDA, * ), B( LDB, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE
      PARAMETER          ( ONE = (1.0D+0,0.0D+0) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I, IINFO, J, K, KP
      COMPLEX*16         AK, AKM1, AKM1K, BK, BKM1, DENOM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
```

```
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZSCAL, ZSYCONV, ZSWAP, ZTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYTRS2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
*     Convert A
*
      CALL ZSYCONV( UPLO, 'C', N, A, LDA, IPIV, WORK, IINFO )
*
      IF( UPPER ) THEN
*
*        Solve A*X = B, where A = U*D*U**T.
*
*       P**T * B
        K=N
        DO WHILE ( K .GE. 1 )
         IF( IPIV( K ).GT.0 ) THEN
*           1 x 1 diagonal block
*           Interchange rows K and IPIV(K).
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
```

```
           K=K-1
        ELSE
*          2 x 2 diagonal block
*          Interchange rows K-1 and -IPIV(K).
           KP = -IPIV( K )
           IF( KP.EQ.-IPIV( K-1 ) )
     $        CALL ZSWAP( NRHS, B( K-1, 1 ), LDB, B( KP, 1 ), LDB )
           K=K-2
        END IF
       END DO
*
*  Compute (U \P**T * B) -> B    [ (U \P**T * B) ]
*
       CALL ZTRSM('L','U','N','U',N,NRHS,ONE,A,LDA,B,LDB)
*
*  Compute D \ B -> B   [ D \ (U \P**T * B) ]
*
        I=N
        DO WHILE ( I .GE. 1 )
           IF( IPIV(I) .GT. 0 ) THEN
             CALL ZSCAL( NRHS, ONE / A( I, I ), B( I, 1 ), LDB )
           ELSEIF ( I .GT. 1) THEN
              IF ( IPIV(I-1) .EQ. IPIV(I) ) THEN
                 AKM1K = WORK(I)
                 AKM1 = A( I-1, I-1 ) / AKM1K
                 AK = A( I, I ) / AKM1K
                 DENOM = AKM1*AK - ONE
                 DO 15 J = 1, NRHS
                    BKM1 = B( I-1, J ) / AKM1K
                    BK = B( I, J ) / AKM1K
                    B( I-1, J ) = ( AK*BKM1-BK ) / DENOM
                    B( I, J ) = ( AKM1*BK-BKM1 ) / DENOM
 15               CONTINUE
              I = I - 1
              ENDIF
           ENDIF
           I = I - 1
        END DO
*
*      Compute (U**T \ B) -> B   [ U**T \ (D \ (U \P**T * B) ) ]
*
        CALL ZTRSM('L','U','T','U',N,NRHS,ONE,A,LDA,B,LDB)
*
*       P * B  [ P * (U**T \ (D \ (U \P**T * B) )) ]
*
       K=1
       DO WHILE ( K .LE. N )
        IF( IPIV( K ).GT.0 ) THEN
*          1 x 1 diagonal block
*          Interchange rows K and IPIV(K).
```

```
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K=K+1
         ELSE
*           2 x 2 diagonal block
*           Interchange rows K-1 and -IPIV(K).
            KP = -IPIV( K )
            IF( K .LT. N .AND. KP.EQ.-IPIV( K+1 ) )
     $         CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K=K+2
         ENDIF
        END DO
*
      ELSE
*
*        Solve A*X = B, where A = L*D*L**T.
*
*       P**T * B
        K=1
        DO WHILE ( K .LE. N )
         IF( IPIV( K ).GT.0 ) THEN
*           1 x 1 diagonal block
*           Interchange rows K and IPIV(K).
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K=K+1
         ELSE
*           2 x 2 diagonal block
*           Interchange rows K and -IPIV(K+1).
            KP = -IPIV( K+1 )
            IF( KP.EQ.-IPIV( K ) )
     $         CALL ZSWAP( NRHS, B( K+1, 1 ), LDB, B( KP, 1 ), LDB )
            K=K+2
         ENDIF
        END DO
*
*  Compute (L \P**T * B) -> B    [ (L \P**T * B) ]
*
        CALL ZTRSM('L','L','N','U',N,NRHS,ONE,A,LDA,B,LDB)
*
*  Compute D \ B -> B   [ D \ (L \P**T * B) ]
*
         I=1
         DO WHILE ( I .LE. N )
            IF( IPIV(I) .GT. 0 ) THEN
              CALL ZSCAL( NRHS, ONE / A( I, I ), B( I, 1 ), LDB )
            ELSE
                  AKM1K = WORK(I)
```

```
                 AKM1 = A( I, I ) / AKM1K
                 AK = A( I+1, I+1 ) / AKM1K
                 DENOM = AKM1*AK - ONE
                 DO 25 J = 1, NRHS
                    BKM1 = B( I, J ) / AKM1K
                    BK = B( I+1, J ) / AKM1K
                    B( I, J ) = ( AK*BKM1-BK ) / DENOM
                    B( I+1, J ) = ( AKM1*BK-BKM1 ) / DENOM
 25              CONTINUE
                 I = I + 1
            ENDIF
            I = I + 1
         END DO
*
*  Compute (L**T \ B) -> B   [ L**T \ (D \ (L \P**T * B) ) ]
*
        CALL ZTRSM('L','L','T','U',N,NRHS,ONE,A,LDA,B,LDB)
*
*       P * B  [ P * (L**T \ (D \ (L \P**T * B) )) ]
*
        K=N
        DO WHILE ( K .GE. 1 )
         IF( IPIV( K ).GT.0 ) THEN
*           1 x 1 diagonal block
*           Interchange rows K and IPIV(K).
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K=K-1
         ELSE
*           2 x 2 diagonal block
*           Interchange rows K-1 and -IPIV(K).
            KP = -IPIV( K )
            IF( K.GT.1 .AND. KP.EQ.-IPIV( K-1 ) )
     $         CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K=K-2
         ENDIF
        END DO
*
      END IF
*
*     Revert A
*
      CALL ZSYCONV( UPLO, 'R', N, A, LDA, IPIV, WORK, IINFO )
*
      RETURN
*
*     End of ZSYTRS2
*
      END
```

———-

61.23.4 lisp code

— **zsytrs2.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (ignorable one))
  (defun zsytrs2 (uplo n nrhs a lda ipiv b ldb$ work info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) work b a)
             (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((ak #C(0.0d0 0.0d0)) (akm1 #C(0.0d0 0.0d0))
             (akm1k #C(0.0d0 0.0d0)) (bk #C(0.0d0 0.0d0))
             (bkm1 #C(0.0d0 0.0d0)) (denom #C(0.0d0 0.0d0)) (i 0) (iinfo 0)
             (j 0) (k 0) (kp 0) (upper nil))
        (declare (type (f2cl-lib:complex16) ak akm1 akm1k bk bkm1 denom)
                 (type (f2cl-lib:integer4) i iinfo j k kp)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -5))
         ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -8)))
```

```
(cond
 ((/= info 0) (xerbla "ZSYTRS2" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= n 0) (= nrhs 0))
    (go end_label))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zsyconv uplo "C" n a lda ipiv work iinfo)
  (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
  (setf uplo var-0)
  (setf iinfo var-7))
(cond
 (upper
  (tagbody
    (setf k n)
   label100000
    (if (not (>= k 1))
        (go label100001))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (zswap nrhs
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-sub k 1)))
     (t
      (setf kp
              (f2cl-lib:int-sub
               (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
      (if (= kp
             (f2cl-lib:int-sub
              (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                             ((1 *)) ipiv-%offset%)))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (zswap nrhs
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                     ((+ k (f2cl-lib:int-sub 1)) 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
```

```
            ldb$)
          (declare (ignore var-1 var-3))
          (when var-0 (setf nrhs var-0))
          (when var-2 (setf ldb$ var-2))
          (when var-4 (setf ldb$ var-4))))
    (setf k (f2cl-lib:int-sub k 2))))
  (go label100000)
 label100001
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ztrsm "L" "U" "N" "U" n nrhs one a lda b ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10)))
  (setf i n)
 label100002
  (if (not (>= i 1))
      (go label100003))
  (cond
   ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zscal nrhs
         (/ one
            (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                           a-%offset%))
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (i 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-1 var-2))
      (when var-0 (setf nrhs var-0))
      (when var-3 (setf ldb$ var-3))))
   ((> i 1)
    (cond
     ((=
       (f2cl-lib:fref ipiv
                      ((f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((1 *)))
       (f2cl-lib:fref ipiv (i) ((1 *))))
      (setf akm1k
              (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
      (setf akm1
              (/
               (f2cl-lib:fref a-%data%
                              ((f2cl-lib:int-sub i 1)
                               (f2cl-lib:int-sub i 1))
                              ((1 lda) (1 *)) a-%offset%)
```

```
                akm1k))
        (setf ak
                (/
                 (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                a-%offset%)
                 akm1k))
        (setf denom (- (* akm1 ak) one))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j nrhs) nil)
          (tagbody
            (setf bkm1
                    (/
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                    ((1 ldb$) (1 *)) b-%offset%)
                     akm1k))
            (setf bk
                    (/
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)
                     akm1k))
            (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                    (/ (- (* ak bkm1) bk) denom))
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                    (/ (- (* akm1 bk) bkm1) denom))
           label15))
        (setf i (f2cl-lib:int-sub i 1))))))
    (setf i (f2cl-lib:int-sub i 1))
    (go label100002)
   label100003
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (ztrsm "L" "U" "T" "U" n nrhs one a lda b ldb$)
      (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
      (when var-4 (setf n var-4))
      (when var-5 (setf nrhs var-5))
      (when var-6 (setf one var-6))
      (when var-8 (setf lda var-8))
      (when var-10 (setf ldb$ var-10)))
    (setf k 1)
   label100004
    (if (not (<= k n))
        (go label100005))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
                (zswap nrhs
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$)
               (declare (ignore var-1 var-3))
               (when var-0 (setf nrhs var-0))
               (when var-2 (setf ldb$ var-2))
               (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-add k 1)))
     (t
      (setf kp
              (f2cl-lib:int-sub
               (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
      (if (and (< k n)
               (= kp
                  (f2cl-lib:int-sub
                   (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
                                  ((1 *)) ipiv-%offset%))))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (zswap nrhs
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                       ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                       ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
             (declare (ignore var-1 var-3))
             (when var-0 (setf nrhs var-0))
             (when var-2 (setf ldb$ var-2))
             (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-add k 2))))
    (go label100004)
   label100005))
 (t
  (tagbody
    (setf k 1)
   label100006
    (if (not (<= k n))
        (go label100007))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (zswap nrhs
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                       ((1 ldb$) (1 *)) b-%offset%)
```

```
                ldb$
                (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                      ((1 ldb$) (1 *)) b-%offset%)
                ldb$)
              (declare (ignore var-1 var-3))
              (when var-0 (setf nrhs var-0))
              (when var-2 (setf ldb$ var-2))
              (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-add k 1)))
     (t
      (setf kp
              (f2cl-lib:int-sub
               (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
                              ((1 *)) ipiv-%offset%)))
      (if (= kp
             (f2cl-lib:int-sub
              (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (zswap nrhs
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                     ((+ k 1) 1) ((1 ldb$) (1 *))
                                     b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
              (declare (ignore var-1 var-3))
              (when var-0 (setf nrhs var-0))
              (when var-2 (setf ldb$ var-2))
              (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-add k 2))))
    (go label100006)
   label100007
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (ztrsm "L" "L" "N" "U" n nrhs one a lda b ldb$)
      (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
      (when var-4 (setf n var-4))
      (when var-5 (setf nrhs var-5))
      (when var-6 (setf one var-6))
      (when var-8 (setf lda var-8))
      (when var-10 (setf ldb$ var-10)))
    (setf i 1)
   label100008
    (if (not (<= i n))
        (go label100009))
    (cond
     ((> (f2cl-lib:fref ipiv (i) ((1 *))) 0)
      (multiple-value-bind (var-0 var-1 var-2 var-3)
```

```
          (zscal nrhs
           (/ one
              (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                             a-%offset%))
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (i 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-2))
        (when var-0 (setf nrhs var-0))
        (when var-3 (setf ldb$ var-3))))
      (t
       (setf akm1k
               (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
       (setf akm1
               (/
                (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                               a-%offset%)
                akm1k))
       (setf ak
               (/
                (f2cl-lib:fref a-%data%
                               ((f2cl-lib:int-add i 1)
                                (f2cl-lib:int-add i 1))
                               ((1 lda) (1 *)) a-%offset%)
                akm1k))
       (setf denom (- (* akm1 ak) one))
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j nrhs) nil)
         (tagbody
           (setf bkm1
                   (/
                    (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)
                    akm1k))
           (setf bk
                   (/
                    (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                   ((1 ldb$) (1 *)) b-%offset%)
                    akm1k))
           (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                   (/ (- (* ak bkm1) bk) denom))
           (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                ((1 ldb$) (1 *)) b-%offset%)
                   (/ (- (* akm1 bk) bkm1) denom))
          label25))
       (setf i (f2cl-lib:int-add i 1))))
     (setf i (f2cl-lib:int-add i 1))
     (go label100008)
    label100009
```

```
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (ztrsm "L" "L" "T" "U" n nrhs one a lda b ldb$)
   (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
   (when var-4 (setf n var-4))
   (when var-5 (setf nrhs var-5))
   (when var-6 (setf one var-6))
   (when var-8 (setf lda var-8))
   (when var-10 (setf ldb$ var-10)))
 (setf k n)
label100010
 (if (not (>= k 1))
     (go label100011))
 (cond
  ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
   (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
   (if (/= kp k)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (zswap nrhs
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
          (declare (ignore var-1 var-3))
          (when var-0 (setf nrhs var-0))
          (when var-2 (setf ldb$ var-2))
          (when var-4 (setf ldb$ var-4))))
   (setf k (f2cl-lib:int-sub k 1)))
  (t
   (setf kp
           (f2cl-lib:int-sub
            (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
   (if (and (> k 1)
            (= kp
               (f2cl-lib:int-sub
                (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                               ((1 *)) ipiv-%offset%))))
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (zswap nrhs
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
          (declare (ignore var-1 var-3))
          (when var-0 (setf nrhs var-0))
```

```
                    (when var-2 (setf ldb$ var-2))
                    (when var-4 (setf ldb$ var-4))))
              (setf k (f2cl-lib:int-sub k 2))))
            (go label100010)
           label100011)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (zsyconv uplo "R" n a lda ipiv work iinfo)
          (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
          (setf uplo var-0)
          (setf iinfo var-7))
        (go end_label)
       end_label
        (return (values uplo n nrhs nil lda nil nil ldb$ nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsytrs2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil nil
                                              fortran-to-lisp::ldb$ nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::zsyconv
                                              fortran-to-lisp::xerbla))))
```

61.24 zsytrs_rook

61.24.1 Axiom unit tests

— zsytrsrook.input —

```
)set break resume
)sys rm -f zsytrsrook.output
)spool zsytrsrook.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.24.2 Axiom help page

— zsytrs_rook.help —

```
==================================================================
zsytrs_rook examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zsytrs_rook (UPLO, N, NRHS, A, LDA, IPIV, B, LDB, INFO)
           ZSYTRS_ROOK

Function/Subroutine Documentation
   subroutine zsytrs_rook (character UPLO, integer N, integer NRHS,
       complex*16, dimension( lda, * ) A, integer LDA, integer, dimension( * )
       IPIV, complex*16, dimension( ldb, * ) B, integer LDB, integer INFO)
       ZSYTRS_ROOK

Purpose:
   ZSYTRS_ROOK solves a system of linear equations A*X = B with
   a complex symmetric matrix A using the factorization A = U*D*U**T or
   A = L*D*L**T computed by ZSYTRF_ROOK.
```

```
Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   The block diagonal matrix D and the multipliers used to
   obtain the factor U or L as computed by ZSYTRF_ROOK.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 IPIV
   IPIV is INTEGER array, dimension (N)
   Details of the interchanges and the block structure of D
   as determined by ZSYTRF_ROOK.

 B
   B is COMPLEX*16 array, dimension (LDB,NRHS)
   On entry, the right hand side matrix B.
   On exit, the solution matrix X.

 LDB
   LDB is INTEGER
   The leading dimension of the array B.  LDB >= max(1,N).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

61.24.3 fortran code

— zsytrs_rook.f —

```
*  =====================================================================
      SUBROUTINE ZSYTRS_ROOK( UPLO, N, NRHS, A, LDA, IPIV, B, LDB,
     $                        INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         CONE
      PARAMETER          ( CONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            J, K, KP
      COMPLEX*16         AK, AKM1, AKM1K, BK, BKM1, DENOM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZGEMV, ZGERU, ZSCAL, ZSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
```

```
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYTRS_ROOK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*        Solve A*X = B, where A = U*D*U**T.
*
*        First solve U*D*X = B, overwriting B with X.
*
*        K is the main loop index, decreasing from N to 1 in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = N
   10    CONTINUE
*
*        If K < 1, exit from loop.
*
         IF( K.LT.1 )
     $      GO TO 30
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
*           Multiply by inv(U(K)), where U(K) is the transformation
*           stored in column K of A.
*
            CALL ZGERU( K-1, NRHS, -CONE, A( 1, K ), 1, B( K, 1 ), LDB,
```

```
     $                B( 1, 1 ), LDB )
*
*          Multiply by the inverse of the diagonal block.
*
           CALL ZSCAL( NRHS, CONE / A( K, K ), B( K, 1 ), LDB )
           K = K - 1
        ELSE
*
*          2 x 2 diagonal block
*
*          Interchange rows K and -IPIV(K) THEN K-1 and -IPIV(K-1)
*
           KP = -IPIV( K )
           IF( KP.NE.K )
     $        CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
           KP = -IPIV( K-1 )
           IF( KP.NE.K-1 )
     $        CALL ZSWAP( NRHS, B( K-1, 1 ), LDB, B( KP, 1 ), LDB )
*
*          Multiply by inv(U(K)), where U(K) is the transformation
*          stored in columns K-1 and K of A.
*
           IF( K.GT.2 ) THEN
              CALL ZGERU( K-2, NRHS,-CONE, A( 1, K ), 1, B( K, 1 ),
     $                    LDB, B( 1, 1 ), LDB )
              CALL ZGERU( K-2, NRHS,-CONE, A( 1, K-1 ), 1, B( K-1, 1 ),
     $                    LDB, B( 1, 1 ), LDB )
           END IF
*
*          Multiply by the inverse of the diagonal block.
*
           AKM1K = A( K-1, K )
           AKM1 = A( K-1, K-1 ) / AKM1K
           AK = A( K, K ) / AKM1K
           DENOM = AKM1*AK - CONE
           DO 20 J = 1, NRHS
              BKM1 = B( K-1, J ) / AKM1K
              BK = B( K, J ) / AKM1K
              B( K-1, J ) = ( AK*BKM1-BK ) / DENOM
              B( K, J ) = ( AKM1*BK-BKM1 ) / DENOM
   20      CONTINUE
           K = K - 2
        END IF
*
        GO TO 10
   30   CONTINUE
*
*       Next solve U**T *X = B, overwriting B with X.
*
```

```
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = 1
   40    CONTINUE
*
*        If K > N, exit from loop.
*
         IF( K.GT.N )
     $      GO TO 50
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Multiply by inv(U**T(K)), where U(K) is the transformation
*           stored in column K of A.
*
            IF( K.GT.1 )
     $         CALL ZGEMV( 'Transpose', K-1, NRHS, -CONE, B,
     $                     LDB, A( 1, K ), 1, CONE, B( K, 1 ), LDB )
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K = K + 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Multiply by inv(U**T(K+1)), where U(K+1) is the transformation
*           stored in columns K and K+1 of A.
*
            IF( K.GT.1 ) THEN
               CALL ZGEMV( 'Transpose', K-1, NRHS, -CONE, B,
     $                     LDB, A( 1, K ), 1, CONE, B( K, 1 ), LDB )
               CALL ZGEMV( 'Transpose', K-1, NRHS, -CONE, B,
     $                     LDB, A( 1, K+1 ), 1, CONE, B( K+1, 1 ), LDB )
            END IF
*
*           Interchange rows K and -IPIV(K) THEN K+1 and -IPIV(K+1).
*
            KP = -IPIV( K )
            IF( KP.NE.K )
     $         CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
            KP = -IPIV( K+1 )
            IF( KP.NE.K+1 )
```

```
     $         CALL ZSWAP( NRHS, B( K+1, 1 ), LDB, B( KP, 1 ), LDB )
*
            K = K + 2
         END IF
*
         GO TO 40
   50    CONTINUE
*
      ELSE
*
*        Solve A*X = B, where A = L*D*L**T.
*
*        First solve L*D*X = B, overwriting B with X.
*
*        K is the main loop index, increasing from 1 to N in steps of
*        1 or 2, depending on the size of the diagonal blocks.
*
         K = 1
   60    CONTINUE
*
*        If K > N, exit from loop.
*
         IF( K.GT.N )
     $      GO TO 80
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
*           Multiply by inv(L(K)), where L(K) is the transformation
*           stored in column K of A.
*
            IF( K.LT.N )
     $         CALL ZGERU( N-K, NRHS, -CONE, A( K+1, K ), 1, B( K, 1 ),
     $                     LDB, B( K+1, 1 ), LDB )
*
*           Multiply by the inverse of the diagonal block.
*
            CALL ZSCAL( NRHS, CONE / A( K, K ), B( K, 1 ), LDB )
            K = K + 1
         ELSE
*
*           2 x 2 diagonal block
*
```

```
*          Interchange rows K and -IPIV(K) THEN K+1 and -IPIV(K+1)
*
           KP = -IPIV( K )
           IF( KP.NE.K )
     $        CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
           KP = -IPIV( K+1 )
           IF( KP.NE.K+1 )
     $        CALL ZSWAP( NRHS, B( K+1, 1 ), LDB, B( KP, 1 ), LDB )
*
*          Multiply by inv(L(K)), where L(K) is the transformation
*          stored in columns K and K+1 of A.
*
           IF( K.LT.N-1 ) THEN
              CALL ZGERU( N-K-1, NRHS,-CONE, A( K+2, K ), 1, B( K, 1 ),
     $                    LDB, B( K+2, 1 ), LDB )
              CALL ZGERU( N-K-1, NRHS,-CONE, A( K+2, K+1 ), 1,
     $                    B( K+1, 1 ), LDB, B( K+2, 1 ), LDB )
           END IF
*
*          Multiply by the inverse of the diagonal block.
*
           AKM1K = A( K+1, K )
           AKM1 = A( K, K ) / AKM1K
           AK = A( K+1, K+1 ) / AKM1K
           DENOM = AKM1*AK - CONE
           DO 70 J = 1, NRHS
              BKM1 = B( K, J ) / AKM1K
              BK = B( K+1, J ) / AKM1K
              B( K, J ) = ( AK*BKM1-BK ) / DENOM
              B( K+1, J ) = ( AKM1*BK-BKM1 ) / DENOM
   70      CONTINUE
           K = K + 2
        END IF
*
        GO TO 60
   80   CONTINUE
*
*       Next solve L**T *X = B, overwriting B with X.
*
*       K is the main loop index, decreasing from N to 1 in steps of
*       1 or 2, depending on the size of the diagonal blocks.
*
        K = N
   90   CONTINUE
*
*       If K < 1, exit from loop.
*
        IF( K.LT.1 )
     $     GO TO 100
```

```
*
         IF( IPIV( K ).GT.0 ) THEN
*
*           1 x 1 diagonal block
*
*           Multiply by inv(L**T(K)), where L(K) is the transformation
*           stored in column K of A.
*
            IF( K.LT.N )
     $         CALL ZGEMV( 'Transpose', N-K, NRHS, -CONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K ), 1, CONE, B( K, 1 ), LDB )
*
*           Interchange rows K and IPIV(K).
*
            KP = IPIV( K )
            IF( KP.NE.K )
     $         CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
            K = K - 1
         ELSE
*
*           2 x 2 diagonal block
*
*           Multiply by inv(L**T(K-1)), where L(K-1) is the transformation
*           stored in columns K-1 and K of A.
*
            IF( K.LT.N ) THEN
               CALL ZGEMV( 'Transpose', N-K, NRHS, -CONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K ), 1, CONE, B( K, 1 ), LDB )
               CALL ZGEMV( 'Transpose', N-K, NRHS, -CONE, B( K+1, 1 ),
     $                     LDB, A( K+1, K-1 ), 1, CONE, B( K-1, 1 ),
     $                     LDB )
            END IF
*
*           Interchange rows K and -IPIV(K) THEN K-1 and -IPIV(K-1)
*
            KP = -IPIV( K )
            IF( KP.NE.K )
     $         CALL ZSWAP( NRHS, B( K, 1 ), LDB, B( KP, 1 ), LDB )
*
            KP = -IPIV( K-1 )
            IF( KP.NE.K-1 )
     $         CALL ZSWAP( NRHS, B( K-1, 1 ), LDB, B( KP, 1 ), LDB )
*
            K = K - 2
         END IF
*
         GO TO 90
  100    CONTINUE
      END IF
*
```

```
      RETURN
*
*     End of ZSYTRS_ROOK
*
      END
```

———-

61.24.4 lisp code

— **zsytrs_rook.lisp** —

```
(let* ((cone (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) cone)
           (ignorable cone))
  (defun zsytrs_rook (uplo n nrhs a lda ipiv b ldb$ info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) b a)
             (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((ak #C(0.0d0 0.0d0)) (akm1 #C(0.0d0 0.0d0))
             (akm1k #C(0.0d0 0.0d0)) (bk #C(0.0d0 0.0d0))
             (bkm1 #C(0.0d0 0.0d0)) (denom #C(0.0d0 0.0d0)) (j 0) (k 0) (kp 0)
             (upper nil))
        (declare (type (f2cl-lib:complex16) ak akm1 akm1k bk bkm1 denom)
                 (type (f2cl-lib:integer4) j k kp)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
```

```
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8)))
(cond
 ((/= info 0) (xerbla "ZSYTRS_ROOK" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= n 0) (= nrhs 0))
    (go end_label))
(cond
 (upper
  (tagbody
    (setf k n)
   label10
    (if (< k 1)
        (go label30))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (zswap nrhs
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (zgeru (f2cl-lib:int-sub k 1) nrhs (- cone)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
        (when var-1 (setf nrhs var-1))
        (when var-6 (setf ldb$ var-6))
        (when var-8 (setf ldb$ var-8)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
```

```
      (zscal nrhs
       (/ cone
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-1 var-2))
    (when var-0 (setf nrhs var-0))
    (when var-3 (setf ldb$ var-3)))
  (setf k (f2cl-lib:int-sub k 1)))
 (t
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
  (if (/= kp k)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (zswap nrhs
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-3))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
  (setf kp
          (f2cl-lib:int-sub
           (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                          ((1 *)) ipiv-%offset%)))
  (if (/= kp (f2cl-lib:int-sub k 1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (zswap nrhs
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                 ((+ k (f2cl-lib:int-sub 1)) 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$)
        (declare (ignore var-1 var-3))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf ldb$ var-4))))
  (cond
   ((> k 2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
```

```
      (zgeru (f2cl-lib:int-sub k 2) nrhs (- cone)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
    (when var-1 (setf nrhs var-1))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldb$ var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zgeru (f2cl-lib:int-sub k 2) nrhs (- cone)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             (1 (f2cl-lib:int-sub k 1))
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                             ((+ k (f2cl-lib:int-sub 1)) 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
    (when var-1 (setf nrhs var-1))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldb$ var-8)))))
(setf akm1k
        (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub k 1) k)
                       ((1 lda) (1 *)) a-%offset%))
(setf akm1
        (/
         (f2cl-lib:fref a-%data%
                        ((f2cl-lib:int-sub k 1)
                         (f2cl-lib:int-sub k 1))
                        ((1 lda) (1 *)) a-%offset%)
         akm1k))
(setf ak
        (/
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         akm1k))
(setf denom (- (* akm1 ak) cone))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
```

```
      (tagbody
        (setf bkm1
                (/
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1) j)
                                ((1 ldb$) (1 *)) b-%offset%)
                 akm1k))
        (setf bk
                (/
                 (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                b-%offset%)
                 akm1k))
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)
                (/ (- (* ak bkm1) bk) denom))
        (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                             b-%offset%)
                (/ (- (* akm1 bk) bkm1) denom))
       label20))
    (setf k (f2cl-lib:int-sub k 2))))
  (go label10)
 label30
  (setf k 1)
 label40
  (if (> k n)
      (go label50))
  (cond
   ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
    (if (> k 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10)
            (zgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- cone) b
             ldb$
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                                   ((1 lda) (1 *)) a-%offset%)
             1 cone
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                   ((1 ldb$) (1 *)) b-%offset%)
             ldb$)
          (declare
           (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
          (when var-2 (setf nrhs var-2))
          (when var-5 (setf ldb$ var-5))
          (when var-8 (setf cone var-8))
          (when var-10 (setf ldb$ var-10))))
    (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
    (if (/= kp k)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (zswap nrhs
             (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
```

```
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
       (declare (ignore var-1 var-3))
       (when var-0 (setf nrhs var-0))
       (when var-2 (setf ldb$ var-2))
       (when var-4 (setf ldb$ var-4))))
 (setf k (f2cl-lib:int-add k 1)))
(t
 (cond
  ((> k 1)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (zgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- cone) b
        ldb$
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 k)
                              ((1 lda) (1 *)) a-%offset%)
        1 cone
        (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$)
     (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
     (when var-2 (setf nrhs var-2))
     (when var-5 (setf ldb$ var-5))
     (when var-8 (setf cone var-8))
     (when var-10 (setf ldb$ var-10)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (zgemv "Transpose" (f2cl-lib:int-sub k 1) nrhs (- cone) b
        ldb$
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                              (1 (f2cl-lib:int-add k 1))
                              ((1 lda) (1 *)) a-%offset%)
        1 cone
        (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                              ((+ k 1) 1) ((1 ldb$) (1 *))
                              b-%offset%)
        ldb$)
     (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
     (when var-2 (setf nrhs var-2))
     (when var-5 (setf ldb$ var-5))
     (when var-8 (setf cone var-8))
     (when var-10 (setf ldb$ var-10)))))
 (setf kp
         (f2cl-lib:int-sub
          (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
```

```
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (zswap nrhs
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (setf kp
              (f2cl-lib:int-sub
               (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
                              ((1 *)) ipiv-%offset%)))
      (if (/= kp (f2cl-lib:int-add k 1))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (zswap nrhs
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                     ((+ k 1) 1) ((1 ldb$) (1 *))
                                     b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$)
            (declare (ignore var-1 var-3))
            (when var-0 (setf nrhs var-0))
            (when var-2 (setf ldb$ var-2))
            (when var-4 (setf ldb$ var-4))))
      (setf k (f2cl-lib:int-add k 2))))
    (go label40)
   label50))
 (t
  (tagbody
    (setf k 1)
   label60
    (if (> k n)
        (go label80))
    (cond
     ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
      (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
      (if (/= kp k)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (zswap nrhs
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
               ldb$
               (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
```

```
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
      (declare (ignore var-1 var-3))
      (when var-0 (setf nrhs var-0))
      (when var-2 (setf ldb$ var-2))
      (when var-4 (setf ldb$ var-4))))
 (if (< k n)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (zgeru (f2cl-lib:int-sub n k) nrhs (- cone)
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                ((+ k 1) k) ((1 lda) (1 *))
                                a-%offset%)
          1
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                ((+ k 1) 1) ((1 ldb$) (1 *))
                                b-%offset%)
          ldb$)
      (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
      (when var-1 (setf nrhs var-1))
      (when var-6 (setf ldb$ var-6))
      (when var-8 (setf ldb$ var-8))))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (zscal nrhs
      (/ cone
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%))
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                            ((1 ldb$) (1 *)) b-%offset%)
      ldb$)
   (declare (ignore var-1 var-2))
   (when var-0 (setf nrhs var-0))
   (when var-3 (setf ldb$ var-3)))
 (setf k (f2cl-lib:int-add k 1)))
(t
 (setf kp
         (f2cl-lib:int-sub
          (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
 (if (/= kp k)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (zswap nrhs
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$)
```

```
      (declare (ignore var-1 var-3))
      (when var-0 (setf nrhs var-0))
      (when var-2 (setf ldb$ var-2))
      (when var-4 (setf ldb$ var-4))))
(setf kp
        (f2cl-lib:int-sub
         (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-add k 1))
                        ((1 *)) ipiv-%offset%)))
(if (/= kp (f2cl-lib:int-add k 1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zswap nrhs
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                               ((+ k 1) 1) ((1 ldb$) (1 *))
                               b-%offset%)
         ldb$
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-1 var-3))
      (when var-0 (setf nrhs var-0))
      (when var-2 (setf ldb$ var-2))
      (when var-4 (setf ldb$ var-4))))
(cond
 ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zgeru (f2cl-lib:int-sub n k 1) nrhs (- cone)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             ((+ k 2) k) ((1 lda) (1 *))
                             a-%offset%)
       1
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                             ((+ k 2) 1) ((1 ldb$) (1 *))
                             b-%offset%)
       ldb$)
    (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
    (when var-1 (setf nrhs var-1))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldb$ var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zgeru (f2cl-lib:int-sub n k 1) nrhs (- cone)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             ((+ k 2) (f2cl-lib:int-add k 1))
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
```

```
                                ((+ k 1) 1) ((1 ldb$) (1 *))
                                b-%offset%)
            ldb$
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                ((+ k 2) 1) ((1 ldb$) (1 *))
                                b-%offset%)
            ldb$)
        (declare (ignore var-0 var-2 var-3 var-4 var-5 var-7))
        (when var-1 (setf nrhs var-1))
        (when var-6 (setf ldb$ var-6))
        (when var-8 (setf ldb$ var-8)))))
    (setf akm1k
            (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k 1) k)
                           ((1 lda) (1 *)) a-%offset%))
    (setf akm1
            (/
             (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                            a-%offset%)
             akm1k))
    (setf ak
            (/
             (f2cl-lib:fref a-%data%
                            ((f2cl-lib:int-add k 1)
                             (f2cl-lib:int-add k 1))
                            ((1 lda) (1 *)) a-%offset%)
             akm1k))
    (setf denom (- (* akm1 ak) cone))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (setf bkm1
                (/
                 (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                b-%offset%)
                 akm1k))
        (setf bk
                (/
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                                ((1 ldb$) (1 *)) b-%offset%)
                 akm1k))
        (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                             b-%offset%)
                (/ (- (* ak bkm1) bk) denom))
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)
                (/ (- (* akm1 bk) bkm1) denom))
       label70))
    (setf k (f2cl-lib:int-add k 2))))
  (go label60)
 label80
```

```
 (setf k n)
label90
 (if (< k 1)
     (go label100))
 (cond
  ((> (f2cl-lib:fref ipiv (k) ((1 *))) 0)
   (if (< k n)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
            var-9 var-10)
           (zgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- cone)
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                  ((+ k 1) 1) ((1 ldb$) (1 *))
                                  b-%offset%)
            ldb$
            (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                  ((+ k 1) k) ((1 lda) (1 *))
                                  a-%offset%)
            1 cone
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
          (declare
           (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
          (when var-2 (setf nrhs var-2))
          (when var-5 (setf ldb$ var-5))
          (when var-8 (setf cone var-8))
          (when var-10 (setf ldb$ var-10))))
   (setf kp (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%))
   (if (/= kp k)
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (zswap nrhs
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$
            (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$)
          (declare (ignore var-1 var-3))
          (when var-0 (setf nrhs var-0))
          (when var-2 (setf ldb$ var-2))
          (when var-4 (setf ldb$ var-4))))
   (setf k (f2cl-lib:int-sub k 1)))
  (t
   (cond
    ((< k n)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
          var-9 var-10)
         (zgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- cone)
```

```
        (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                              ((+ k 1) 1) ((1 ldb$) (1 *))
                              b-%offset%)
        ldb$
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                              ((+ k 1) k) ((1 lda) (1 *))
                              a-%offset%)
        1 cone
        (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                              ((1 ldb$) (1 *)) b-%offset%)
        ldb$)
    (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
    (when var-2 (setf nrhs var-2))
    (when var-5 (setf ldb$ var-5))
    (when var-8 (setf cone var-8))
    (when var-10 (setf ldb$ var-10)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (zgemv "Transpose" (f2cl-lib:int-sub n k) nrhs (- cone)
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                             ((+ k 1) 1) ((1 ldb$) (1 *))
                             b-%offset%)
       ldb$
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             ((+ k 1) (f2cl-lib:int-sub k 1))
                             ((1 lda) (1 *)) a-%offset%)
       1 cone
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                             ((+ k (f2cl-lib:int-sub 1)) 1)
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$)
    (declare (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9))
    (when var-2 (setf nrhs var-2))
    (when var-5 (setf ldb$ var-5))
    (when var-8 (setf cone var-8))
    (when var-10 (setf ldb$ var-10)))))
(setf kp
        (f2cl-lib:int-sub
         (f2cl-lib:fref ipiv-%data% (k) ((1 *)) ipiv-%offset%)))
(if (/= kp k)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zswap nrhs
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (k 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$)
      (declare (ignore var-1 var-3))
```

```
                    (when var-0 (setf nrhs var-0))
                    (when var-2 (setf ldb$ var-2))
                    (when var-4 (setf ldb$ var-4))))
            (setf kp
                    (f2cl-lib:int-sub
                     (f2cl-lib:fref ipiv-%data% ((f2cl-lib:int-sub k 1))
                                    ((1 *)) ipiv-%offset%)))
            (if (/= kp (f2cl-lib:int-sub k 1))
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (zswap nrhs
                     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                           ((+ k (f2cl-lib:int-sub 1)) 1)
                                           ((1 ldb$) (1 *)) b-%offset%)
                     ldb$
                     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (kp 1)
                                           ((1 ldb$) (1 *)) b-%offset%)
                     ldb$)
                    (declare (ignore var-1 var-3))
                    (when var-0 (setf nrhs var-0))
                    (when var-2 (setf ldb$ var-2))
                    (when var-4 (setf ldb$ var-4))))
            (setf k (f2cl-lib:int-sub k 2))))
          (go label90)
         label100)))
       (go end_label)
      end_label
       (return (values uplo nil nrhs nil nil nil nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsytrs_rook
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::integer4
                                              (*))
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(fortran-to-lisp::uplo nil
                                             fortran-to-lisp::nrhs nil nil nil
                                             nil fortran-to-lisp::ldb$
```

```
                                        fortran-to-lisp::info)
                                      :calls '(fortran-to-lisp::xerbla))))
```

———-

61.25 ztgsyl

61.25.1 Axiom unit tests

— **ztgsyl.input** —

```
)set break resume
)sys rm -f ztgsyl.output
)spool ztgsyl.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.25.2 Axiom help page

— **ztgsyl.help** —

```
======================================================================
ztgsyl examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ztgsyl (TRANS, IJOB, M, N, A, LDA, B, LDB, C, LDC, D, LDD,
           E, LDE, F, LDF, SCALE, DIF, WORK, LWORK, IWORK, INFO)
           ZTGSYL

Function/Subroutine Documentation
```

```
   subroutine ztgsyl (character TRANS, integer IJOB, integer M, integer N,
       complex*16, dimension( lda, * ) A, integer LDA, complex*16, dimension(
       ldb, * ) B, integer LDB, complex*16, dimension( ldc, * ) C, integer
       LDC, complex*16, dimension( ldd, * ) D, integer LDD, complex*16,
       dimension( lde, * ) E, integer LDE, complex*16, dimension( ldf, * ) F,
       integer LDF, double precision SCALE, double precision DIF, complex*16,
       dimension( * ) WORK, integer LWORK, integer, dimension( * ) IWORK,
       integer INFO)
       ZTGSYL

Purpose:
   ZTGSYL solves the generalized Sylvester equation:

                   A * R - L * B = scale * C            (1)
                   D * R - L * E = scale * F

   where R and L are unknown m-by-n matrices, (A, D), (B, E) and
   (C, F) are given matrix pairs of size m-by-m, n-by-n and m-by-n,
   respectively, with complex entries. A, B, D and E are upper
   triangular (i.e., (A,D) and (B,E) in generalized Schur form).

   The solution (R, L) overwrites (C, F). 0 <= SCALE <= 1
   is an output scaling factor chosen to avoid overflow.

   In matrix notation (1) is equivalent to solve Zx = scale*b, where Z
   is defined as

              Z = [ kron(In, A)  -kron(B**H, Im) ]        (2)
                  [ kron(In, D)  -kron(E**H, Im) ],

   Here Ix is the identity matrix of size x and X**H is the conjugate
   transpose of X. Kron(X, Y) is the Kronecker product between the
   matrices X and Y.

   If TRANS = 'C', y in the conjugate transposed system Z**H *y = scale*b
   is solved for, which is equivalent to solve for R and L in

                   A**H * R + D**H * L = scale * C           (3)
                   R * B**H + L * E**H = scale * -F

   This case (TRANS = 'C') is used to compute an one-norm-based estimate
   of Dif[(A,D), (B,E)], the separation between the matrix pairs (A,D)
   and (B,E), using ZLACON.

   If IJOB >= 1, ZTGSYL computes a Frobenius norm-based estimate of
   Dif[(A,D),(B,E)]. That is, the reciprocal of a lower bound on the
   reciprocal of the smallest singular value of Z.

   This is a level-3 BLAS algorithm.
```

```
Parameters:

 TRANS
   TRANS is CHARACTER*1
   = 'N': solve the generalized sylvester equation (1).
   = 'C': solve the "conjugate transposed" system (3).

 IJOB
   IJOB is INTEGER
   Specifies what kind of functionality to be performed.
   =0: solve (1) only.
   =1: The functionality of 0 and 3.
   =2: The functionality of 0 and 4.
   =3: Only an estimate of Dif[(A,D), (B,E)] is computed.
       (look ahead strategy is used).
   =4: Only an estimate of Dif[(A,D), (B,E)] is computed.
       (ZGECON on sub-systems is used).
   Not referenced if TRANS = 'C'.

 M
   M is INTEGER
   The order of the matrices A and D, and the row dimension of
   the matrices C, F, R and L.

 N
   N is INTEGER
   The order of the matrices B and E, and the column dimension
   of the matrices C, F, R and L.

 A
   A is COMPLEX*16 array, dimension (LDA, M)
   The upper triangular matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A. LDA >= max(1, M).

 B
   B is COMPLEX*16 array, dimension (LDB, N)
   The upper triangular matrix B.

 LDB
   LDB is INTEGER
   The leading dimension of the array B. LDB >= max(1, N).

 C
   C is COMPLEX*16 array, dimension (LDC, N)
   On entry, C contains the right-hand-side of the first matrix
   equation in (1) or (3).
   On exit, if IJOB = 0, 1 or 2, C has been overwritten by
```

```
  the solution R. If IJOB = 3 or 4 and TRANS = 'N', C holds R,
  the solution achieved during the computation of the Dif-estimate.

LDC
  LDC is INTEGER
  The leading dimension of the array C. LDC >= max(1, M).

D
  D is COMPLEX*16 array, dimension (LDD, M)
  The upper triangular matrix D.

LDD
  LDD is INTEGER
  The leading dimension of the array D. LDD >= max(1, M).

E
  E is COMPLEX*16 array, dimension (LDE, N)
  The upper triangular matrix E.

LDE
  LDE is INTEGER
  The leading dimension of the array E. LDE >= max(1, N).

F
  F is COMPLEX*16 array, dimension (LDF, N)
  On entry, F contains the right-hand-side of the second matrix
  equation in (1) or (3).
  On exit, if IJOB = 0, 1 or 2, F has been overwritten by
  the solution L. If IJOB = 3 or 4 and TRANS = 'N', F holds L,
  the solution achieved during the computation of the Dif-estimate.

LDF
  LDF is INTEGER
  The leading dimension of the array F. LDF >= max(1, M).

DIF
  DIF is DOUBLE PRECISION
  On exit DIF is the reciprocal of a lower bound of the
  reciprocal of the Dif-function, i.e. DIF is an upper bound of
  Dif[(A,D), (B,E)] = sigma-min(Z), where Z as in (2).
  IF IJOB = 0 or TRANS = 'C', DIF is not referenced.

SCALE
  SCALE is DOUBLE PRECISION
  On exit SCALE is the scaling factor in (1) or (3).
  If 0 < SCALE < 1, C and F hold the solutions R and L, resp.,
  to a slightly perturbed system but the input matrices A, B,
  D and E have not been changed. If SCALE = 0, R and L will
  hold the solutions to the homogenious system with C = F = 0.
```

```
 WORK
   WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
   On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK
   LWORK is INTEGER
   The dimension of the array WORK. LWORK > = 1.
   If IJOB = 1 or 2 and TRANS = 'N', LWORK >= max(1,2*M*N).

   If LWORK = -1, then a workspace query is assumed; the routine
   only calculates the optimal size of the WORK array, returns
   this value as the first entry of the WORK array, and no error
   message related to LWORK is issued by XERBLA.

 IWORK
   IWORK is INTEGER array, dimension (M+N+2)

 INFO
   INFO is INTEGER
   =0: successful exit
   <0: If INFO = -i, the i-th argument had an illegal value.
   >0: (A, D) and (B, E) have common or very close eigenvalues.
```

———-

61.25.3 fortran code

— ztgsyl.f —

```
*  =====================================================================
      SUBROUTINE ZTGSYL( TRANS, IJOB, M, N, A, LDA, B, LDB, C, LDC, D,
     $                   LDD, E, LDE, F, LDF, SCALE, DIF, WORK, LWORK,
     $                   IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            IJOB, INFO, LDA, LDB, LDC, LDD, LDE, LDF,
     $                   LWORK, M, N
      DOUBLE PRECISION   DIF, SCALE
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
```

```
      COMPLEX*16         A( LDA, * ), B( LDB, * ), C( LDC, * ),
     $                   D( LDD, * ), E( LDE, * ), F( LDF, * ),
     $                   WORK( * )
*     ..
*
*  ====================================================================
*  Replaced various illegal calls to CCOPY by calls to CLASET.
*  Sven Hammarling, 1/5/02.
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      COMPLEX*16         CZERO
      PARAMETER          ( CZERO = (0.0D+0, 0.0D+0) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, NOTRAN
      INTEGER            I, IE, IFUNC, IROUND, IS, ISOLVE, J, JE, JS, K,
     $                   LINFO, LWMIN, MB, NB, P, PQ, Q
      DOUBLE PRECISION   DSCALE, DSUM, SCALE2, SCALOC
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGEMM, ZLACPY, ZLASET, ZSCAL, ZTGSY2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, DCMPLX, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode and test input parameters
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
*
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( NOTRAN ) THEN
         IF( ( IJOB.LT.0 ) .OR. ( IJOB.GT.4 ) ) THEN
            INFO = -2
         END IF
      END IF
      IF( INFO.EQ.0 ) THEN
         IF( M.LE.0 ) THEN
            INFO = -3
```

```
         ELSE IF( N.LE.0 ) THEN
            INFO = -4
         ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
            INFO = -6
         ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
            INFO = -8
         ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
            INFO = -10
         ELSE IF( LDD.LT.MAX( 1, M ) ) THEN
            INFO = -12
         ELSE IF( LDE.LT.MAX( 1, N ) ) THEN
            INFO = -14
         ELSE IF( LDF.LT.MAX( 1, M ) ) THEN
            INFO = -16
         END IF
      END IF
*
      IF( INFO.EQ.0 ) THEN
         IF( NOTRAN ) THEN
            IF( IJOB.EQ.1 .OR. IJOB.EQ.2 ) THEN
               LWMIN = MAX( 1, 2*M*N )
            ELSE
               LWMIN = 1
            END IF
         ELSE
            LWMIN = 1
         END IF
         WORK( 1 ) = LWMIN
*
         IF( LWORK.LT.LWMIN .AND. .NOT.LQUERY ) THEN
            INFO = -20
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZTGSYL', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         SCALE = 1
         IF( NOTRAN ) THEN
            IF( IJOB.NE.0 ) THEN
               DIF = 0
            END IF
         END IF
```

```
         RETURN
      END IF
*
*     Determine  optimal block sizes MB and NB
*
      MB = ILAENV( 2, 'ZTGSYL', TRANS, M, N, -1, -1 )
      NB = ILAENV( 5, 'ZTGSYL', TRANS, M, N, -1, -1 )
*
      ISOLVE = 1
      IFUNC = 0
      IF( NOTRAN ) THEN
         IF( IJOB.GE.3 ) THEN
            IFUNC = IJOB - 2
            CALL ZLASET( 'F', M, N, CZERO, CZERO, C, LDC )
            CALL ZLASET( 'F', M, N, CZERO, CZERO, F, LDF )
         ELSE IF( IJOB.GE.1 .AND. NOTRAN ) THEN
            ISOLVE = 2
         END IF
      END IF
*
      IF( ( MB.LE.1 .AND. NB.LE.1 ) .OR. ( MB.GE.M .AND. NB.GE.N ) )
     $     THEN
*
*        Use unblocked Level 2 solver
*
         DO 30 IROUND = 1, ISOLVE
*
            SCALE = ONE
            DSCALE = ZERO
            DSUM = ONE
            PQ = M*N
            CALL ZTGSY2( TRANS, IFUNC, M, N, A, LDA, B, LDB, C, LDC, D,
     $                   LDD, E, LDE, F, LDF, SCALE, DSUM, DSCALE,
     $                   INFO )
            IF( DSCALE.NE.ZERO ) THEN
               IF( IJOB.EQ.1 .OR. IJOB.EQ.3 ) THEN
                  DIF = SQRT( DBLE( 2*M*N ) ) / ( DSCALE*SQRT( DSUM ) )
               ELSE
                  DIF = SQRT( DBLE( PQ ) ) / ( DSCALE*SQRT( DSUM ) )
               END IF
            END IF
            IF( ISOLVE.EQ.2 .AND. IROUND.EQ.1 ) THEN
               IF( NOTRAN ) THEN
                  IFUNC = IJOB
               END IF
               SCALE2 = SCALE
               CALL ZLACPY( 'F', M, N, C, LDC, WORK, M )
               CALL ZLACPY( 'F', M, N, F, LDF, WORK( M*N+1 ), M )
               CALL ZLASET( 'F', M, N, CZERO, CZERO, C, LDC )
               CALL ZLASET( 'F', M, N, CZERO, CZERO, F, LDF )
```

```
            ELSE IF( ISOLVE.EQ.2 .AND. IROUND.EQ.2 ) THEN
               CALL ZLACPY( 'F', M, N, WORK, M, C, LDC )
               CALL ZLACPY( 'F', M, N, WORK( M*N+1 ), M, F, LDF )
               SCALE = SCALE2
            END IF
   30    CONTINUE
*
         RETURN
*
      END IF
*
*     Determine block structure of A
*
      P = 0
      I = 1
   40 CONTINUE
      IF( I.GT.M )
     $   GO TO 50
      P = P + 1
      IWORK( P ) = I
      I = I + MB
      IF( I.GE.M )
     $   GO TO 50
      GO TO 40
   50 CONTINUE
      IWORK( P+1 ) = M + 1
      IF( IWORK( P ).EQ.IWORK( P+1 ) )
     $   P = P - 1
*
*     Determine block structure of B
*
      Q = P + 1
      J = 1
   60 CONTINUE
      IF( J.GT.N )
     $   GO TO 70
*
      Q = Q + 1
      IWORK( Q ) = J
      J = J + NB
      IF( J.GE.N )
     $   GO TO 70
      GO TO 60
*
   70 CONTINUE
      IWORK( Q+1 ) = N + 1
      IF( IWORK( Q ).EQ.IWORK( Q+1 ) )
     $   Q = Q - 1
*
      IF( NOTRAN ) THEN
```

```
      DO 150 IROUND = 1, ISOLVE
*
*        Solve (I, J) - subsystem
*            A(I, I) * R(I, J) - L(I, J) * B(J, J) = C(I, J)
*            D(I, I) * R(I, J) - L(I, J) * E(J, J) = F(I, J)
*        for I = P, P - 1, ..., 1; J = 1, 2, ..., Q
*
         PQ = 0
         SCALE = ONE
         DSCALE = ZERO
         DSUM = ONE
         DO 130 J = P + 2, Q
            JS = IWORK( J )
            JE = IWORK( J+1 ) - 1
            NB = JE - JS + 1
            DO 120 I = P, 1, -1
               IS = IWORK( I )
               IE = IWORK( I+1 ) - 1
               MB = IE - IS + 1
               CALL ZTGSY2( TRANS, IFUNC, MB, NB, A( IS, IS ), LDA,
     $                      B( JS, JS ), LDB, C( IS, JS ), LDC,
     $                      D( IS, IS ), LDD, E( JS, JS ), LDE,
     $                      F( IS, JS ), LDF, SCALOC, DSUM, DSCALE,
     $                      LINFO )
               IF( LINFO.GT.0 )
     $            INFO = LINFO
               PQ = PQ + MB*NB
               IF( SCALOC.NE.ONE ) THEN
                  DO 80 K = 1, JS - 1
                     CALL ZSCAL( M, DCMPLX( SCALOC, ZERO ),
     $                           C( 1, K ), 1 )
                     CALL ZSCAL( M, DCMPLX( SCALOC, ZERO ),
     $                           F( 1, K ), 1 )
   80             CONTINUE
                  DO 90 K = JS, JE
                     CALL ZSCAL( IS-1, DCMPLX( SCALOC, ZERO ),
     $                           C( 1, K ), 1 )
                     CALL ZSCAL( IS-1, DCMPLX( SCALOC, ZERO ),
     $                           F( 1, K ), 1 )
   90             CONTINUE
                  DO 100 K = JS, JE
                     CALL ZSCAL( M-IE, DCMPLX( SCALOC, ZERO ),
     $                           C( IE+1, K ), 1 )
                     CALL ZSCAL( M-IE, DCMPLX( SCALOC, ZERO ),
     $                           F( IE+1, K ), 1 )
  100             CONTINUE
                  DO 110 K = JE + 1, N
                     CALL ZSCAL( M, DCMPLX( SCALOC, ZERO ),
     $                           C( 1, K ), 1 )
                     CALL ZSCAL( M, DCMPLX( SCALOC, ZERO ),
```

```
     $                              F( 1, K ), 1 )
  110                CONTINUE
                     SCALE = SCALE*SCALOC
                  END IF
*
*                 Substitute R(I,J) and L(I,J) into remaining equation.
*
                  IF( I.GT.1 ) THEN
                     CALL ZGEMM( 'N', 'N', IS-1, NB, MB,
     $                           DCMPLX( -ONE, ZERO ), A( 1, IS ), LDA,
     $                           C( IS, JS ), LDC, DCMPLX( ONE, ZERO ),
     $                           C( 1, JS ), LDC )
                     CALL ZGEMM( 'N', 'N', IS-1, NB, MB,
     $                           DCMPLX( -ONE, ZERO ), D( 1, IS ), LDD,
     $                           C( IS, JS ), LDC, DCMPLX( ONE, ZERO ),
     $                           F( 1, JS ), LDF )
                  END IF
                  IF( J.LT.Q ) THEN
                     CALL ZGEMM( 'N', 'N', MB, N-JE, NB,
     $                           DCMPLX( ONE, ZERO ), F( IS, JS ), LDF,
     $                           B( JS, JE+1 ), LDB,
     $                           DCMPLX( ONE, ZERO ), C( IS, JE+1 ),
     $                           LDC )
                     CALL ZGEMM( 'N', 'N', MB, N-JE, NB,
     $                           DCMPLX( ONE, ZERO ), F( IS, JS ), LDF,
     $                           E( JS, JE+1 ), LDE,
     $                           DCMPLX( ONE, ZERO ), F( IS, JE+1 ),
     $                           LDF )
                  END IF
  120          CONTINUE
  130       CONTINUE
            IF( DSCALE.NE.ZERO ) THEN
               IF( IJOB.EQ.1 .OR. IJOB.EQ.3 ) THEN
                  DIF = SQRT( DBLE( 2*M*N ) ) / ( DSCALE*SQRT( DSUM ) )
               ELSE
                  DIF = SQRT( DBLE( PQ ) ) / ( DSCALE*SQRT( DSUM ) )
               END IF
            END IF
            IF( ISOLVE.EQ.2 .AND. IROUND.EQ.1 ) THEN
               IF( NOTRAN ) THEN
                  IFUNC = IJOB
               END IF
               SCALE2 = SCALE
               CALL ZLACPY( 'F', M, N, C, LDC, WORK, M )
               CALL ZLACPY( 'F', M, N, F, LDF, WORK( M*N+1 ), M )
               CALL ZLASET( 'F', M, N, CZERO, CZERO, C, LDC )
               CALL ZLASET( 'F', M, N, CZERO, CZERO, F, LDF )
            ELSE IF( ISOLVE.EQ.2 .AND. IROUND.EQ.2 ) THEN
               CALL ZLACPY( 'F', M, N, WORK, M, C, LDC )
               CALL ZLACPY( 'F', M, N, WORK( M*N+1 ), M, F, LDF )
```

```
            SCALE = SCALE2
         END IF
  150    CONTINUE
      ELSE
*
*        Solve transposed (I, J)-subsystem
*            A(I, I)**H * R(I, J) + D(I, I)**H * L(I, J) = C(I, J)
*            R(I, J) * B(J, J)  + L(I, J) * E(J, J) = -F(I, J)
*        for I = 1,2,..., P; J = Q, Q-1,..., 1
*
         SCALE = ONE
         DO 210 I = 1, P
            IS = IWORK( I )
            IE = IWORK( I+1 ) - 1
            MB = IE - IS + 1
            DO 200 J = Q, P + 2, -1
               JS = IWORK( J )
               JE = IWORK( J+1 ) - 1
               NB = JE - JS + 1
               CALL ZTGSY2( TRANS, IFUNC, MB, NB, A( IS, IS ), LDA,
     $                      B( JS, JS ), LDB, C( IS, JS ), LDC,
     $                      D( IS, IS ), LDD, E( JS, JS ), LDE,
     $                      F( IS, JS ), LDF, SCALOC, DSUM, DSCALE,
     $                      LINFO )
               IF( LINFO.GT.0 )
     $            INFO = LINFO
               IF( SCALOC.NE.ONE ) THEN
                  DO 160 K = 1, JS - 1
                     CALL ZSCAL( M, DCMPLX( SCALOC, ZERO ), C( 1, K ),
     $                           1 )
                     CALL ZSCAL( M, DCMPLX( SCALOC, ZERO ), F( 1, K ),
     $                           1 )
  160             CONTINUE
                  DO 170 K = JS, JE
                     CALL ZSCAL( IS-1, DCMPLX( SCALOC, ZERO ),
     $                           C( 1, K ), 1 )
                     CALL ZSCAL( IS-1, DCMPLX( SCALOC, ZERO ),
     $                           F( 1, K ), 1 )
  170             CONTINUE
                  DO 180 K = JS, JE
                     CALL ZSCAL( M-IE, DCMPLX( SCALOC, ZERO ),
     $                           C( IE+1, K ), 1 )
                     CALL ZSCAL( M-IE, DCMPLX( SCALOC, ZERO ),
     $                           F( IE+1, K ), 1 )
  180             CONTINUE
                  DO 190 K = JE + 1, N
                     CALL ZSCAL( M, DCMPLX( SCALOC, ZERO ), C( 1, K ),
     $                           1 )
                     CALL ZSCAL( M, DCMPLX( SCALOC, ZERO ), F( 1, K ),
     $                           1 )
```

```
  190            CONTINUE
                 SCALE = SCALE*SCALOC
              END IF
*
*             Substitute R(I,J) and L(I,J) into remaining equation.
*
              IF( J.GT.P+2 ) THEN
                 CALL ZGEMM( 'N', 'C', MB, JS-1, NB,
     $                       DCMPLX( ONE, ZERO ), C( IS, JS ), LDC,
     $                       B( 1, JS ), LDB, DCMPLX( ONE, ZERO ),
     $                       F( IS, 1 ), LDF )
                 CALL ZGEMM( 'N', 'C', MB, JS-1, NB,
     $                       DCMPLX( ONE, ZERO ), F( IS, JS ), LDF,
     $                       E( 1, JS ), LDE, DCMPLX( ONE, ZERO ),
     $                       F( IS, 1 ), LDF )
              END IF
              IF( I.LT.P ) THEN
                 CALL ZGEMM( 'C', 'N', M-IE, NB, MB,
     $                       DCMPLX( -ONE, ZERO ), A( IS, IE+1 ), LDA,
     $                       C( IS, JS ), LDC, DCMPLX( ONE, ZERO ),
     $                       C( IE+1, JS ), LDC )
                 CALL ZGEMM( 'C', 'N', M-IE, NB, MB,
     $                       DCMPLX( -ONE, ZERO ), D( IS, IE+1 ), LDD,
     $                       F( IS, JS ), LDF, DCMPLX( ONE, ZERO ),
     $                       C( IE+1, JS ), LDC )
              END IF
  200       CONTINUE
  210    CONTINUE
      END IF
*
      WORK( 1 ) = LWMIN
*
      RETURN
*
*     End of ZTGSYL
*
      END
```

———-

61.25.4 lisp code

— **ztgsyl.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0) (czero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (f2cl-lib:complex16) czero)
```

```
          (ignorable zero one czero))
 (defun ztgsyl
        (trans ijob m n a lda b ldb$ c ldc d ldd e lde f ldf scale dif work
         lwork iwork info)
   (declare (type (array f2cl-lib:integer4 (*)) iwork)
            (type (double-float) dif scale)
            (type (array f2cl-lib:complex16 (*)) work f e d c b a)
            (type (f2cl-lib:integer4) info lwork ldf lde ldd ldc ldb$ lda n m
             ijob)
            (type (string *) trans))
   (f2cl-lib:with-multi-array-data
       ((trans character trans-%data% trans-%offset%)
        (a f2cl-lib:complex16 a-%data% a-%offset%)
        (b f2cl-lib:complex16 b-%data% b-%offset%)
        (c f2cl-lib:complex16 c-%data% c-%offset%)
        (d f2cl-lib:complex16 d-%data% d-%offset%)
        (e f2cl-lib:complex16 e-%data% e-%offset%)
        (f f2cl-lib:complex16 f-%data% f-%offset%)
        (work f2cl-lib:complex16 work-%data% work-%offset%)
        (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
     (prog ((dscale 0.0d0) (dsum 0.0d0) (scale2 0.0d0) (scaloc 0.0d0) (i 0)
            (ie 0) (ifunc 0) (iround 0) (is 0) (isolve 0) (j 0) (je 0) (js 0)
            (k 0) (linfo 0) (lwmin 0) (mb 0) (nb 0) (p 0) (pq 0) (q 0)
            (lquery nil) (notran nil))
       (declare (type (double-float) dscale dsum scale2 scaloc)
                (type (f2cl-lib:integer4) i ie ifunc iround is isolve j je js
                 k linfo lwmin mb nb p pq q)
                (type f2cl-lib:logical lquery notran))
       (setf info 0)
       (setf notran
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame trans "N")
                 (declare (ignore var-1))
                 (when var-0 (setf trans var-0))
                 ret-val))
       (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
       (cond
        ((and (not notran)
              (not
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame trans "C")
                 (declare (ignore var-1))
                 (when var-0 (setf trans var-0))
                 ret-val)))
         (setf info -1))
        (notran (cond ((or (< ijob 0) (> ijob 4)) (setf info -2)))))
       (cond
        ((= info 0)
         (cond ((<= m 0) (setf info -3)) ((<= n 0) (setf info -4))
               ((< lda
```

```
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -6))
        ((< ldb$
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -8))
        ((< ldc
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -10))
        ((< ldd
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -12))
        ((< lde
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -14))
        ((< ldf
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -16)))))
(cond
 ((= info 0)
  (cond
   (notran
    (cond
     ((or (= ijob 1) (= ijob 2))
      (setf lwmin
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 (f2cl-lib:int-mul 2 m n)))))
     (t (setf lwmin 1))))
   (t (setf lwmin 1)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwmin 'f2cl-lib:complex16))
  (cond ((and (< lwork lwmin) (not lquery)) (setf info -20)))))
(cond
 ((/= info 0) (xerbla "ZTGSYL" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond
 ((or (= m 0) (= n 0))
  (setf scale (coerce (the f2cl-lib:integer4 1) 'double-float))
  (cond
   (notran
    (cond
     ((/= ijob 0)
      (setf dif (coerce (the f2cl-lib:integer4 0) 'double-float))))))
  (go end_label)))
(setf mb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 2 "ZTGSYL" trans m n -1 -1)
          (declare (ignore var-0 var-1 var-5 var-6))
          (setf trans var-2)
          (setf m var-3)
```

```
          (setf n var-4)
          ret-val))
(setf nb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 5 "ZTGSYL" trans m n -1 -1)
          (declare (ignore var-0 var-1 var-5 var-6))
          (setf trans var-2)
          (setf m var-3)
          (setf n var-4)
          ret-val))
(setf isolve 1)
(setf ifunc 0)
(cond
 (notran
  (cond
   ((>= ijob 3) (setf ifunc (f2cl-lib:int-sub ijob 2))
    (zlaset "F" m n czero czero c ldc)
    (zlaset "F" m n czero czero f ldf))
   ((and (>= ijob 1) notran) (setf isolve 2)))))
(cond
 ((or (and (<= mb 1) (<= nb 1)) (and (>= mb m) (>= nb n)))
  (f2cl-lib:fdo (iround 1 (f2cl-lib:int-add iround 1))
                ((> iround isolve) nil)
    (tagbody
      (setf scale one)
      (setf dscale zero)
      (setf dsum one)
      (setf pq (f2cl-lib:int-mul m n))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17
           var-18 var-19)
          (ztgsy2 trans ifunc m n a lda b ldb$ c ldc d ldd e lde f ldf
           scale dsum dscale info)
        (declare
         (ignore var-1 var-3 var-4 var-5 var-6 var-8 var-10 var-11
          var-12 var-14))
        (setf trans var-0)
        (setf m var-2)
        (setf ldb$ var-7)
        (setf ldc var-9)
        (setf lde var-13)
        (setf ldf var-15)
        (setf scale var-16)
        (setf dsum var-17)
        (setf dscale var-18)
        (setf info var-19))
      (cond
       ((/= dscale zero)
```

```
          (cond
           ((or (= ijob 1) (= ijob 3))
            (setf dif
                    (/
                     (f2cl-lib:fsqrt
                      (f2cl-lib:dble (f2cl-lib:int-mul 2 m n)))
                     (* dscale (f2cl-lib:fsqrt dsum)))))
           (t
            (setf dif
                    (/ (f2cl-lib:fsqrt (f2cl-lib:dble pq))
                       (* dscale (f2cl-lib:fsqrt dsum))))))))
        (cond
         ((and (= isolve 2) (= iround 1))
          (cond (notran (setf ifunc ijob))) (setf scale2 scale)
          (zlacpy "F" m n c ldc work m)
          (zlacpy "F" m n f ldf
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                 ((+ (f2cl-lib:int-mul m n) 1)) ((1 *))
                                 work-%offset%)
           m)
          (zlaset "F" m n czero czero c ldc)
          (zlaset "F" m n czero czero f ldf))
         ((and (= isolve 2) (= iround 2)) (zlacpy "F" m n work m c ldc)
          (zlacpy "F" m n
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                 ((+ (f2cl-lib:int-mul m n) 1)) ((1 *))
                                 work-%offset%)
           m f ldf)
          (setf scale scale2)))
      label30))
    (go end_label)))
  (setf p 0)
  (setf i 1)
 label40
  (if (> i m)
      (go label50))
  (setf p (f2cl-lib:int-add p 1))
  (setf (f2cl-lib:fref iwork-%data% (p) ((1 *)) iwork-%offset%) i)
  (setf i (f2cl-lib:int-add i mb))
  (if (>= i m)
      (go label50))
  (go label40)
 label50
  (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add p 1)) ((1 *))
                       iwork-%offset%)
          (f2cl-lib:int-add m 1))
  (if (= (f2cl-lib:fref iwork-%data% (p) ((1 *)) iwork-%offset%)
         (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add p 1)) ((1 *))
                        iwork-%offset%))
      (setf p (f2cl-lib:int-sub p 1)))
```

```
 (setf q (f2cl-lib:int-add p 1))
 (setf j 1)
label60
 (if (> j n)
     (go label70))
 (setf q (f2cl-lib:int-add q 1))
 (setf (f2cl-lib:fref iwork-%data% (q) ((1 *)) iwork-%offset%) j)
 (setf j (f2cl-lib:int-add j nb))
 (if (>= j n)
     (go label70))
 (go label60)
label70
 (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add q 1)) ((1 *))
                      iwork-%offset%)
        (f2cl-lib:int-add n 1))
 (if (= (f2cl-lib:fref iwork-%data% (q) ((1 *)) iwork-%offset%)
        (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add q 1)) ((1 *))
                       iwork-%offset%))
     (setf q (f2cl-lib:int-sub q 1)))
 (cond
  (notran
   (f2cl-lib:fdo (iround 1 (f2cl-lib:int-add iround 1))
                 ((> iround isolve) nil)
     (tagbody
       (setf pq 0)
       (setf scale one)
       (setf dscale zero)
       (setf dsum one)
       (f2cl-lib:fdo (j (f2cl-lib:int-add p 2) (f2cl-lib:int-add j 1))
                     ((> j q) nil)
         (tagbody
           (setf js
                   (f2cl-lib:fref iwork-%data% (j) ((1 *))
                                  iwork-%offset%))
           (setf je
                   (f2cl-lib:int-sub
                    (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add j 1))
                                   ((1 *)) iwork-%offset%)
                    1))
           (setf nb (f2cl-lib:int-add (f2cl-lib:int-sub je js) 1))
           (f2cl-lib:fdo (i p (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                         ((> i 1) nil)
             (tagbody
               (setf is
                       (f2cl-lib:fref iwork-%data% (i) ((1 *))
                                      iwork-%offset%))
               (setf ie
                       (f2cl-lib:int-sub
                        (f2cl-lib:fref iwork-%data%
                                       ((f2cl-lib:int-add i 1)) ((1 *))
```

```
                        iwork-%offset%)
         1))
(setf mb (f2cl-lib:int-add (f2cl-lib:int-sub ie is) 1))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
     var-8 var-9 var-10 var-11 var-12 var-13 var-14
     var-15 var-16 var-17 var-18 var-19)
    (ztgsy2 trans ifunc mb nb
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                           (is is) ((1 lda) (1 *))
                           a-%offset%)
     lda
     (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                           (js js) ((1 ldb$) (1 *))
                           b-%offset%)
     ldb$
     (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                           (is js) ((1 ldc) (1 *))
                           c-%offset%)
     ldc
     (f2cl-lib:array-slice d-%data% f2cl-lib:complex16
                           (is is) ((1 ldd) (1 *))
                           d-%offset%)
     ldd
     (f2cl-lib:array-slice e-%data% f2cl-lib:complex16
                           (js js) ((1 lde) (1 *))
                           e-%offset%)
     lde
     (f2cl-lib:array-slice f-%data% f2cl-lib:complex16
                           (is js) ((1 ldf) (1 *))
                           f-%offset%)
     ldf scaloc dsum dscale linfo)
  (declare
   (ignore var-1 var-3 var-4 var-5 var-6 var-8 var-10
    var-11 var-12 var-14))
  (setf trans var-0)
  (setf mb var-2)
  (setf ldb$ var-7)
  (setf ldc var-9)
  (setf lde var-13)
  (setf ldf var-15)
  (setf scaloc var-16)
  (setf dsum var-17)
  (setf dscale var-18)
  (setf linfo var-19))
(if (> linfo 0)
    (setf info linfo))
(setf pq (f2cl-lib:int-add pq (f2cl-lib:int-mul mb nb)))
(cond
 ((/= scaloc one)
```

```
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k
                  (f2cl-lib:int-add js
                                    (f2cl-lib:int-sub
                                     1)))
               nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zscal m (f2cl-lib:dcmplx scaloc zero)
         (f2cl-lib:array-slice c-%data%
                               f2cl-lib:complex16 (1 k)
                               ((1 ldc) (1 *))
                               c-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf m var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zscal m (f2cl-lib:dcmplx scaloc zero)
         (f2cl-lib:array-slice f-%data%
                               f2cl-lib:complex16 (1 k)
                               ((1 ldf) (1 *))
                               f-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf m var-0)))
   label80))
(f2cl-lib:fdo (k js (f2cl-lib:int-add k 1))
              ((> k je) nil)
  (tagbody
    (zscal (f2cl-lib:int-sub is 1)
     (f2cl-lib:dcmplx scaloc zero)
     (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                           (1 k) ((1 ldc) (1 *))
                           c-%offset%)
     1)
    (zscal (f2cl-lib:int-sub is 1)
     (f2cl-lib:dcmplx scaloc zero)
     (f2cl-lib:array-slice f-%data% f2cl-lib:complex16
                           (1 k) ((1 ldf) (1 *))
                           f-%offset%)
     1)
   label90))
(f2cl-lib:fdo (k js (f2cl-lib:int-add k 1))
              ((> k je) nil)
  (tagbody
    (zscal (f2cl-lib:int-sub m ie)
     (f2cl-lib:dcmplx scaloc zero)
     (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                           ((+ ie 1) k) ((1 ldc) (1 *))
                           c-%offset%)
```

```
          1)
         (zscal (f2cl-lib:int-sub m ie)
          (f2cl-lib:dcmplx scaloc zero)
          (f2cl-lib:array-slice f-%data% f2cl-lib:complex16
                                ((+ ie 1) k) ((1 ldf) (1 *))
                                f-%offset%)
          1)
        label100))
      (f2cl-lib:fdo (k (f2cl-lib:int-add je 1)
                       (f2cl-lib:int-add k 1))
                      ((> k n) nil)
        (tagbody
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (zscal m (f2cl-lib:dcmplx scaloc zero)
               (f2cl-lib:array-slice c-%data%
                                     f2cl-lib:complex16 (1 k)
                                     ((1 ldc) (1 *))
                                     c-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3))
            (when var-0 (setf m var-0)))
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (zscal m (f2cl-lib:dcmplx scaloc zero)
               (f2cl-lib:array-slice f-%data%
                                     f2cl-lib:complex16 (1 k)
                                     ((1 ldf) (1 *))
                                     f-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3))
            (when var-0 (setf m var-0)))
        label110))
      (setf scale (* scale scaloc))))
    (cond
     ((> i 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
           var-8 var-9 var-10 var-11 var-12)
          (zgemm "N" "N" (f2cl-lib:int-sub is 1) nb mb
           (f2cl-lib:dcmplx (- one) zero)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                 (1 is) ((1 lda) (1 *))
                                 a-%offset%)
           lda
           (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                 (is js) ((1 ldc) (1 *))
                                 c-%offset%)
           ldc (f2cl-lib:dcmplx one zero)
           (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                 (1 js) ((1 ldc) (1 *))
                                 c-%offset%)
```

```
          ldc)
        (declare
         (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-10
          var-11))
        (when var-3 (setf nb var-3))
        (when var-4 (setf mb var-4))
        (when var-7 (setf lda var-7))
        (when var-9 (setf ldc var-9))
        (when var-12 (setf ldc var-12)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
           var-8 var-9 var-10 var-11 var-12)
          (zgemm "N" "N" (f2cl-lib:int-sub is 1) nb mb
           (f2cl-lib:dcmplx (- one) zero)
           (f2cl-lib:array-slice d-%data% f2cl-lib:complex16
                                 (1 is) ((1 ldd) (1 *))
                                 d-%offset%)
           ldd
           (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                 (is js) ((1 ldc) (1 *))
                                 c-%offset%)
           ldc (f2cl-lib:dcmplx one zero)
           (f2cl-lib:array-slice f-%data% f2cl-lib:complex16
                                 (1 js) ((1 ldf) (1 *))
                                 f-%offset%)
           ldf)
        (declare
         (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-10
          var-11))
        (when var-3 (setf nb var-3))
        (when var-4 (setf mb var-4))
        (when var-7 (setf ldd var-7))
        (when var-9 (setf ldc var-9))
        (when var-12 (setf ldf var-12)))))
    (cond
     ((< j q)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
           var-8 var-9 var-10 var-11 var-12)
          (zgemm "N" "N" mb (f2cl-lib:int-sub n je) nb
           (f2cl-lib:dcmplx one zero)
           (f2cl-lib:array-slice f-%data% f2cl-lib:complex16
                                 (is js) ((1 ldf) (1 *))
                                 f-%offset%)
           ldf
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                 (js (f2cl-lib:int-add je 1))
                                 ((1 ldb$) (1 *)) b-%offset%)
           ldb$ (f2cl-lib:dcmplx one zero)
           (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
```

```
                                        (is (f2cl-lib:int-add je 1))
                                        ((1 ldc) (1 *)) c-%offset%)
                    ldc)
                  (declare
                   (ignore var-0 var-1 var-3 var-5 var-6 var-8 var-10
                    var-11))
                  (when var-2 (setf mb var-2))
                  (when var-4 (setf nb var-4))
                  (when var-7 (setf ldf var-7))
                  (when var-9 (setf ldb$ var-9))
                  (when var-12 (setf ldc var-12)))
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
                     var-8 var-9 var-10 var-11 var-12)
                    (zgemm "N" "N" mb (f2cl-lib:int-sub n je) nb
                     (f2cl-lib:dcmplx one zero)
                     (f2cl-lib:array-slice f-%data% f2cl-lib:complex16
                                        (is js) ((1 ldf) (1 *))
                                        f-%offset%)
                     ldf
                     (f2cl-lib:array-slice e-%data% f2cl-lib:complex16
                                        (js (f2cl-lib:int-add je 1))
                                        ((1 lde) (1 *)) e-%offset%)
                     lde (f2cl-lib:dcmplx one zero)
                     (f2cl-lib:array-slice f-%data% f2cl-lib:complex16
                                        (is (f2cl-lib:int-add je 1))
                                        ((1 ldf) (1 *)) f-%offset%)
                     ldf)
                  (declare
                   (ignore var-0 var-1 var-3 var-5 var-6 var-8 var-10
                    var-11))
                  (when var-2 (setf mb var-2))
                  (when var-4 (setf nb var-4))
                  (when var-7 (setf ldf var-7))
                  (when var-9 (setf lde var-9))
                  (when var-12 (setf ldf var-12)))))
            label120))
        label130))
      (cond
       ((/= dscale zero)
        (cond
         ((or (= ijob 1) (= ijob 3))
          (setf dif
                  (/
                   (f2cl-lib:fsqrt
                    (f2cl-lib:dble (f2cl-lib:int-mul 2 m n)))
                   (* dscale (f2cl-lib:fsqrt dsum)))))
         (t
          (setf dif
                  (/ (f2cl-lib:fsqrt (f2cl-lib:dble pq))
```

```
                    (* dscale (f2cl-lib:fsqrt dsum))))))))
          (cond
           ((and (= isolve 2) (= iround 1))
            (cond (notran (setf ifunc ijob))) (setf scale2 scale)
            (zlacpy "F" m n c ldc work m)
            (zlacpy "F" m n f ldf
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                   ((+ (f2cl-lib:int-mul m n) 1)) ((1 *))
                                   work-%offset%)
             m)
            (zlaset "F" m n czero czero c ldc)
            (zlaset "F" m n czero czero f ldf))
           ((and (= isolve 2) (= iround 2)) (zlacpy "F" m n work m c ldc)
            (zlacpy "F" m n
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                   ((+ (f2cl-lib:int-mul m n) 1)) ((1 *))
                                   work-%offset%)
             m f ldf)
            (setf scale scale2)))
        label150)))
    (t (setf scale one)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i p) nil)
       (tagbody
         (setf is (f2cl-lib:fref iwork-%data% (i) ((1 *)) iwork-%offset%))
         (setf ie
                 (f2cl-lib:int-sub
                  (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) iwork-%offset%)
                  1))
         (setf mb (f2cl-lib:int-add (f2cl-lib:int-sub ie is) 1))
         (f2cl-lib:fdo (j q (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((> j (f2cl-lib:int-add p 2)) nil)
           (tagbody
             (setf js
                     (f2cl-lib:fref iwork-%data% (j) ((1 *))
                                    iwork-%offset%))
             (setf je
                     (f2cl-lib:int-sub
                      (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add j 1))
                                     ((1 *)) iwork-%offset%)
                      1))
             (setf nb (f2cl-lib:int-add (f2cl-lib:int-sub je js) 1))
             (multiple-value-bind
                 (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                  var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
                  var-17 var-18 var-19)
                 (ztgsy2 trans ifunc mb nb
                  (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                        (is is) ((1 lda) (1 *))
```

```
                              a-%offset%)
        lda
        (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                              (js js) ((1 ldb$) (1 *))
                              b-%offset%)
        ldb$
        (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                              (is js) ((1 ldc) (1 *))
                              c-%offset%)
        ldc
        (f2cl-lib:array-slice d-%data% f2cl-lib:complex16
                              (is is) ((1 ldd) (1 *))
                              d-%offset%)
        ldd
        (f2cl-lib:array-slice e-%data% f2cl-lib:complex16
                              (js js) ((1 lde) (1 *))
                              e-%offset%)
        lde
        (f2cl-lib:array-slice f-%data% f2cl-lib:complex16
                              (is js) ((1 ldf) (1 *))
                              f-%offset%)
        ldf scaloc dsum dscale linfo)
    (declare
     (ignore var-1 var-3 var-4 var-5 var-6 var-8 var-10 var-11
      var-12 var-14))
    (setf trans var-0)
    (setf mb var-2)
    (setf ldb$ var-7)
    (setf ldc var-9)
    (setf lde var-13)
    (setf ldf var-15)
    (setf scaloc var-16)
    (setf dsum var-17)
    (setf dscale var-18)
    (setf linfo var-19))
  (if (> linfo 0)
      (setf info linfo))
  (cond
   ((/= scaloc one)
    (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                  ((> k
                      (f2cl-lib:int-add js
                                        (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (zscal m (f2cl-lib:dcmplx scaloc zero)
             (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                   (1 k) ((1 ldc) (1 *))
                                   c-%offset%)
```

```
          1)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf m var-0)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (zscal m (f2cl-lib:dcmplx scaloc zero)
           (f2cl-lib:array-slice f-%data% f2cl-lib:complex16
                                 (1 k) ((1 ldf) (1 *))
                                 f-%offset%)
          1)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf m var-0)))
    label160))
(f2cl-lib:fdo (k js (f2cl-lib:int-add k 1))
              ((> k je) nil)
  (tagbody
    (zscal (f2cl-lib:int-sub is 1)
     (f2cl-lib:dcmplx scaloc zero)
     (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                           (1 k) ((1 ldc) (1 *))
                           c-%offset%)
     1)
    (zscal (f2cl-lib:int-sub is 1)
     (f2cl-lib:dcmplx scaloc zero)
     (f2cl-lib:array-slice f-%data% f2cl-lib:complex16
                           (1 k) ((1 ldf) (1 *))
                           f-%offset%)
     1)
    label170))
(f2cl-lib:fdo (k js (f2cl-lib:int-add k 1))
              ((> k je) nil)
  (tagbody
    (zscal (f2cl-lib:int-sub m ie)
     (f2cl-lib:dcmplx scaloc zero)
     (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                           ((+ ie 1) k) ((1 ldc) (1 *))
                           c-%offset%)
     1)
    (zscal (f2cl-lib:int-sub m ie)
     (f2cl-lib:dcmplx scaloc zero)
     (f2cl-lib:array-slice f-%data% f2cl-lib:complex16
                           ((+ ie 1) k) ((1 ldf) (1 *))
                           f-%offset%)
     1)
    label180))
(f2cl-lib:fdo (k (f2cl-lib:int-add je 1)
               (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zscal m (f2cl-lib:dcmplx scaloc zero)
```

```
            (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                  (1 k) ((1 ldc) (1 *))
                                  c-%offset%)
            1)
          (declare (ignore var-1 var-2 var-3))
          (when var-0 (setf m var-0)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (zscal m (f2cl-lib:dcmplx scaloc zero)
            (f2cl-lib:array-slice f-%data% f2cl-lib:complex16
                                  (1 k) ((1 ldf) (1 *))
                                  f-%offset%)
            1)
          (declare (ignore var-1 var-2 var-3))
          (when var-0 (setf m var-0)))
      label190))
  (setf scale (* scale scaloc))))
(cond
 ((> j (f2cl-lib:int-add p 2))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "N" "C" mb (f2cl-lib:int-sub js 1) nb
       (f2cl-lib:dcmplx one zero)
       (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                             (is js) ((1 ldc) (1 *))
                             c-%offset%)
       ldc
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                             (1 js) ((1 ldb$) (1 *))
                             b-%offset%)
       ldb$ (f2cl-lib:dcmplx one zero)
       (f2cl-lib:array-slice f-%data% f2cl-lib:complex16
                             (is 1) ((1 ldf) (1 *))
                             f-%offset%)
       ldf)
    (declare
     (ignore var-0 var-1 var-3 var-5 var-6 var-8 var-10
      var-11))
    (when var-2 (setf mb var-2))
    (when var-4 (setf nb var-4))
    (when var-7 (setf ldc var-7))
    (when var-9 (setf ldb$ var-9))
    (when var-12 (setf ldf var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "N" "C" mb (f2cl-lib:int-sub js 1) nb
       (f2cl-lib:dcmplx one zero)
       (f2cl-lib:array-slice f-%data% f2cl-lib:complex16
                             (is js) ((1 ldf) (1 *))
```

```
                                f-%offset%)
                ldf
                (f2cl-lib:array-slice e-%data% f2cl-lib:complex16
                                      (1 js) ((1 lde) (1 *))
                                      e-%offset%)
                lde (f2cl-lib:dcmplx one zero)
                (f2cl-lib:array-slice f-%data% f2cl-lib:complex16
                                      (is 1) ((1 ldf) (1 *))
                                      f-%offset%)
                ldf)
            (declare
             (ignore var-0 var-1 var-3 var-5 var-6 var-8 var-10
              var-11))
            (when var-2 (setf mb var-2))
            (when var-4 (setf nb var-4))
            (when var-7 (setf ldf var-7))
            (when var-9 (setf lde var-9))
            (when var-12 (setf ldf var-12)))))
        (cond
         ((< i p)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10 var-11 var-12)
              (zgemm "C" "N" (f2cl-lib:int-sub m ie) nb mb
               (f2cl-lib:dcmplx (- one) zero)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                     (is (f2cl-lib:int-add ie 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                     (is js) ((1 ldc) (1 *))
                                     c-%offset%)
               ldc (f2cl-lib:dcmplx one zero)
               (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                     ((+ ie 1) js) ((1 ldc) (1 *))
                                     c-%offset%)
               ldc)
            (declare
             (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-10
              var-11))
            (when var-3 (setf nb var-3))
            (when var-4 (setf mb var-4))
            (when var-7 (setf lda var-7))
            (when var-9 (setf ldc var-9))
            (when var-12 (setf ldc var-12)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10 var-11 var-12)
              (zgemm "C" "N" (f2cl-lib:int-sub m ie) nb mb
               (f2cl-lib:dcmplx (- one) zero)
```

```
                        (f2cl-lib:array-slice d-%data% f2cl-lib:complex16
                                              (is (f2cl-lib:int-add ie 1))
                                              ((1 ldd) (1 *)) d-%offset%)
                        ldd
                        (f2cl-lib:array-slice f-%data% f2cl-lib:complex16
                                              (is js) ((1 ldf) (1 *))
                                              f-%offset%)
                        ldf (f2cl-lib:dcmplx one zero)
                        (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                              ((+ ie 1) js) ((1 ldc) (1 *))
                                              c-%offset%)
                        ldc)
                      (declare
                       (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-10
                        var-11))
                      (when var-3 (setf nb var-3))
                      (when var-4 (setf mb var-4))
                      (when var-7 (setf ldd var-7))
                      (when var-9 (setf ldf var-9))
                      (when var-12 (setf ldc var-12)))))
                 label200))
             label210))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce lwmin 'f2cl-lib:complex16))
        (go end_label)
       end_label
        (return
         (values trans nil m n nil lda nil ldb$ nil ldc nil ldd nil lde nil ldf
                 scale dif nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ztgsyl
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
```

```
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (double-float) (double-float)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::trans nil
  fortran-to-lisp::m
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldc nil
  fortran-to-lisp::ldd nil
  fortran-to-lisp::lde nil
  fortran-to-lisp::ldf
  fortran-to-lisp::scale
  fortran-to-lisp::dif nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zlacpy
  fortran-to-lisp::ztgsy2
  fortran-to-lisp::zlaset
  fortran-to-lisp::ilaenv
  fortran-to-lisp::xerbla))))
```

———-

61.26 ztrsyl

61.26.1 Axiom unit tests

— ztrsyl.input —

```
)set break resume
)sys rm -f ztrsyl.output
```

```
)spool ztrsyl.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

61.26.2 Axiom help page

— **ztrsyl.help** —

```
=====================================================================
ztrsyl examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ztrsyl (TRANA, TRANB, ISGN, M, N, A, LDA, B, LDB, C, LDC,
           SCALE, INFO)
           ZTRSYL

Function/Subroutine Documentation
   subroutine ztrsyl (character TRANA, character TRANB, integer ISGN, integer
       M, integer N, complex*16, dimension( lda, * ) A, integer LDA,
       complex*16, dimension( ldb, * ) B, integer LDB, complex*16, dimension(
       ldc, * ) C, integer LDC, double precision SCALE, integer INFO)
       ZTRSYL

Purpose:
   ZTRSYL solves the complex Sylvester matrix equation:

   op(A)*X + X*op(B) = scale*C or
   op(A)*X - X*op(B) = scale*C,

   where op(A) = A or A**H, and A and B are both upper triangular. A is
   M-by-M and B is N-by-N; the right hand side C and the solution X are
   M-by-N; and scale is an output scale factor, set <= 1 to avoid
   overflow in X.

Parameters:
```

```
TRANA
  TRANA is CHARACTER*1
  Specifies the option op(A):
  = 'N': op(A) = A    (No transpose)
  = 'C': op(A) = A**H (Conjugate transpose)

TRANB
  TRANB is CHARACTER*1
  Specifies the option op(B):
  = 'N': op(B) = B    (No transpose)
  = 'C': op(B) = B**H (Conjugate transpose)

ISGN
  ISGN is INTEGER
  Specifies the sign in the equation:
  = +1: solve op(A)*X + X*op(B) = scale*C
  = -1: solve op(A)*X - X*op(B) = scale*C

M
  M is INTEGER
  The order of the matrix A, and the number of rows in the
  matrices X and C. M >= 0.

N
  N is INTEGER
  The order of the matrix B, and the number of columns in the
  matrices X and C. N >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,M)
  The upper triangular matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A. LDA >= max(1,M).

B
  B is COMPLEX*16 array, dimension (LDB,N)
  The upper triangular matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B. LDB >= max(1,N).

C
  C is COMPLEX*16 array, dimension (LDC,N)
  On entry, the M-by-N right hand side matrix C.
  On exit, C is overwritten by the solution matrix X.

LDC
```

```
  LDC is INTEGER
  The leading dimension of the array C. LDC >= max(1,M)

 SCALE
  SCALE is DOUBLE PRECISION
  The scale factor, scale, set <= 1 to avoid overflow in X.

 INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  = 1: A and B have common or very close eigenvalues; perturbed
       values were used to solve the equation (but the matrices
       A and B are unchanged).
```

———-

61.26.3 fortran code

— **ztrsyl.f** —

```
*  =====================================================================
      SUBROUTINE ZTRSYL( TRANA, TRANB, ISGN, M, N, A, LDA, B, LDB, C,
     $                   LDC, SCALE, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANA, TRANB
      INTEGER            INFO, ISGN, LDA, LDB, LDC, M, N
      DOUBLE PRECISION   SCALE
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), B( LDB, * ), C( LDC, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRNA, NOTRNB
```

```
      INTEGER            J, K, L
      DOUBLE PRECISION   BIGNUM, DA11, DB, EPS, SCALOC, SGN, SMIN,
     $                   SMLNUM
      COMPLEX*16         A11, SUML, SUMR, VEC, X11
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, ZLANGE
      COMPLEX*16         ZDOTC, ZDOTU, ZLADIV
      EXTERNAL           LSAME, DLAMCH, ZLANGE, ZDOTC, ZDOTU, ZLADIV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, XERBLA, ZDSCAL
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DCMPLX, DCONJG, DIMAG, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Decode and Test input parameters
*
      NOTRNA = LSAME( TRANA, 'N' )
      NOTRNB = LSAME( TRANB, 'N' )
*
      INFO = 0
      IF( .NOT.NOTRNA .AND. .NOT.LSAME( TRANA, 'C' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRNB .AND. .NOT.LSAME( TRANB, 'C' ) ) THEN
         INFO = -2
      ELSE IF( ISGN.NE.1 .AND. ISGN.NE.-1 ) THEN
         INFO = -3
      ELSE IF( M.LT.0 ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
         INFO = -11
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZTRSYL', -INFO )
         RETURN
      END IF
*
```

```
*     Quick return if possible
*
      SCALE = ONE
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Set constants to control overflow
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SMLNUM*DBLE( M*N ) / EPS
      BIGNUM = ONE / SMLNUM
      SMIN = MAX( SMLNUM, EPS*ZLANGE( 'M', M, M, A, LDA, DUM ),
     $       EPS*ZLANGE( 'M', N, N, B, LDB, DUM ) )
      SGN = ISGN
*
      IF( NOTRNA .AND. NOTRNB ) THEN
*
*        Solve    A*X + ISGN*X*B = scale*C.
*
*        The (K,L)th block of X is determined starting from
*        bottom-left corner column by column by
*
*            A(K,K)*X(K,L) + ISGN*X(K,L)*B(L,L) = C(K,L) - R(K,L)
*
*        Where
*                    M                        L-1
*          R(K,L) = SUM [A(K,I)*X(I,L)] +ISGN*SUM [X(K,J)*B(J,L)].
*                  I=K+1                      J=1
*
         DO 30 L = 1, N
            DO 20 K = M, 1, -1
*
               SUML = ZDOTU( M-K, A( K, MIN( K+1, M ) ), LDA,
     $                C( MIN( K+1, M ), L ), 1 )
               SUMR = ZDOTU( L-1, C( K, 1 ), LDC, B( 1, L ), 1 )
               VEC = C( K, L ) - ( SUML+SGN*SUMR )
*
               SCALOC = ONE
               A11 = A( K, K ) + SGN*B( L, L )
               DA11 = ABS( DBLE( A11 ) ) + ABS( DIMAG( A11 ) )
               IF( DA11.LE.SMIN ) THEN
                  A11 = SMIN
                  DA11 = SMIN
                  INFO = 1
               END IF
               DB = ABS( DBLE( VEC ) ) + ABS( DIMAG( VEC ) )
               IF( DA11.LT.ONE .AND. DB.GT.ONE ) THEN
```

```
                IF( DB.GT.BIGNUM*DA11 )
     $             SCALOC = ONE / DB
             END IF
             X11 = ZLADIV( VEC*DCMPLX( SCALOC ), A11 )
*
             IF( SCALOC.NE.ONE ) THEN
                DO 10 J = 1, N
                   CALL ZDSCAL( M, SCALOC, C( 1, J ), 1 )
   10           CONTINUE
                SCALE = SCALE*SCALOC
             END IF
             C( K, L ) = X11
*
   20      CONTINUE
   30   CONTINUE
*
      ELSE IF( .NOT.NOTRNA .AND. NOTRNB ) THEN
*
*       Solve    A**H *X + ISGN*X*B = scale*C.
*
*       The (K,L)th block of X is determined starting from
*       upper-left corner column by column by
*
*           A**H(K,K)*X(K,L) + ISGN*X(K,L)*B(L,L) = C(K,L) - R(K,L)
*
*       Where
*                  K-1                           L-1
*         R(K,L) = SUM [A**H(I,K)*X(I,L)] + ISGN*SUM [X(K,J)*B(J,L)]
*                  I=1                           J=1
*
        DO 60 L = 1, N
           DO 50 K = 1, M
*
             SUML = ZDOTC( K-1, A( 1, K ), 1, C( 1, L ), 1 )
             SUMR = ZDOTU( L-1, C( K, 1 ), LDC, B( 1, L ), 1 )
             VEC = C( K, L ) - ( SUML+SGN*SUMR )
*
             SCALOC = ONE
             A11 = DCONJG( A( K, K ) ) + SGN*B( L, L )
             DA11 = ABS( DBLE( A11 ) ) + ABS( DIMAG( A11 ) )
             IF( DA11.LE.SMIN ) THEN
                A11 = SMIN
                DA11 = SMIN
                INFO = 1
             END IF
             DB = ABS( DBLE( VEC ) ) + ABS( DIMAG( VEC ) )
             IF( DA11.LT.ONE .AND. DB.GT.ONE ) THEN
                IF( DB.GT.BIGNUM*DA11 )
     $             SCALOC = ONE / DB
             END IF
```

```
*
              X11 = ZLADIV( VEC*DCMPLX( SCALOC ), A11 )
*
              IF( SCALOC.NE.ONE ) THEN
                 DO 40 J = 1, N
                    CALL ZDSCAL( M, SCALOC, C( 1, J ), 1 )
   40            CONTINUE
                 SCALE = SCALE*SCALOC
              END IF
              C( K, L ) = X11
*
   50       CONTINUE
   60    CONTINUE
*
      ELSE IF( .NOT.NOTRNA .AND. .NOT.NOTRNB ) THEN
*
*        Solve    A**H*X + ISGN*X*B**H = C.
*
*        The (K,L)th block of X is determined starting from
*        upper-right corner column by column by
*
*            A**H(K,K)*X(K,L) + ISGN*X(K,L)*B**H(L,L) = C(K,L) - R(K,L)
*
*        Where
*                    K-1
*           R(K,L) = SUM [A**H(I,K)*X(I,L)] +
*                    I=1
*                           N
*                     ISGN*SUM [X(K,J)*B**H(L,J)].
*                          J=L+1
*
         DO 90 L = N, 1, -1
            DO 80 K = 1, M
*
              SUML = ZDOTC( K-1, A( 1, K ), 1, C( 1, L ), 1 )
              SUMR = ZDOTC( N-L, C( K, MIN( L+1, N ) ), LDC,
     $                B( L, MIN( L+1, N ) ), LDB )
              VEC = C( K, L ) - ( SUML+SGN*DCONJG( SUMR ) )
*
              SCALOC = ONE
              A11 = DCONJG( A( K, K )+SGN*B( L, L ) )
              DA11 = ABS( DBLE( A11 ) ) + ABS( DIMAG( A11 ) )
              IF( DA11.LE.SMIN ) THEN
                 A11 = SMIN
                 DA11 = SMIN
                 INFO = 1
              END IF
              DB = ABS( DBLE( VEC ) ) + ABS( DIMAG( VEC ) )
              IF( DA11.LT.ONE .AND. DB.GT.ONE ) THEN
                 IF( DB.GT.BIGNUM*DA11 )
```

```
     $               SCALOC = ONE / DB
               END IF
*
               X11 = ZLADIV( VEC*DCMPLX( SCALOC ), A11 )
*
               IF( SCALOC.NE.ONE ) THEN
                  DO 70 J = 1, N
                     CALL ZDSCAL( M, SCALOC, C( 1, J ), 1 )
   70             CONTINUE
                  SCALE = SCALE*SCALOC
               END IF
               C( K, L ) = X11
*
   80       CONTINUE
   90    CONTINUE
*
      ELSE IF( NOTRNA .AND. .NOT.NOTRNB ) THEN
*
*        Solve    A*X + ISGN*X*B**H = C.
*
*        The (K,L)th block of X is determined starting from
*        bottom-left corner column by column by
*
*           A(K,K)*X(K,L) + ISGN*X(K,L)*B**H(L,L) = C(K,L) - R(K,L)
*
*        Where
*                    M                          N
*          R(K,L) = SUM [A(K,I)*X(I,L)] + ISGN*SUM [X(K,J)*B**H(L,J)]
*                  I=K+1                      J=L+1
*
         DO 120 L = N, 1, -1
            DO 110 K = M, 1, -1
*
               SUML = ZDOTU( M-K, A( K, MIN( K+1, M ) ), LDA,
     $                C( MIN( K+1, M ), L ), 1 )
               SUMR = ZDOTC( N-L, C( K, MIN( L+1, N ) ), LDC,
     $                B( L, MIN( L+1, N ) ), LDB )
               VEC = C( K, L ) - ( SUML+SGN*DCONJG( SUMR ) )
*
               SCALOC = ONE
               A11 = A( K, K ) + SGN*DCONJG( B( L, L ) )
               DA11 = ABS( DBLE( A11 ) ) + ABS( DIMAG( A11 ) )
               IF( DA11.LE.SMIN ) THEN
                  A11 = SMIN
                  DA11 = SMIN
                  INFO = 1
               END IF
               DB = ABS( DBLE( VEC ) ) + ABS( DIMAG( VEC ) )
               IF( DA11.LT.ONE .AND. DB.GT.ONE ) THEN
                  IF( DB.GT.BIGNUM*DA11 )
```

```
     $               SCALOC = ONE / DB
               END IF
*
               X11 = ZLADIV( VEC*DCMPLX( SCALOC ), A11 )
*
               IF( SCALOC.NE.ONE ) THEN
                  DO 100 J = 1, N
                     CALL ZDSCAL( M, SCALOC, C( 1, J ), 1 )
  100             CONTINUE
                  SCALE = SCALE*SCALOC
               END IF
               C( K, L ) = X11
*
  110       CONTINUE
  120    CONTINUE
*
      END IF
*
      RETURN
*
*     End of ZTRSYL
*
      END
```

———-

61.26.4 lisp code

— ztrsyl.lisp —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun ztrsyl (trana tranb isgn m n a lda b ldb$ c ldc scale info)
    (declare (type (double-float) scale)
             (type (array f2cl-lib:complex16 (*)) c b a)
             (type (f2cl-lib:integer4) info ldc ldb$ lda n m isgn)
             (type (string *) tranb trana))
    (f2cl-lib:with-multi-array-data
        ((trana character trana-%data% trana-%offset%)
         (tranb character tranb-%data% tranb-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (c f2cl-lib:complex16 c-%data% c-%offset%))
      (prog ((dum (make-array 1 :element-type 'double-float))
             (a11 #C(0.0d0 0.0d0)) (suml #C(0.0d0 0.0d0))
             (sumr #C(0.0d0 0.0d0)) (vec #C(0.0d0 0.0d0)) (x11 #C(0.0d0 0.0d0))
             (bignum 0.0d0) (da11 0.0d0) (db 0.0d0) (eps 0.0d0) (scaloc 0.0d0)
```

```
     (sgn 0.0d0) (smin 0.0d0) (smlnum 0.0d0) (j 0) (k 0) (l 0)
     (notrna nil) (notrnb nil))
(declare (type (array double-float (1)) dum)
         (type (f2cl-lib:complex16) a11 suml sumr vec x11)
         (type (double-float) bignum da11 db eps scaloc sgn smin
          smlnum)
         (type (f2cl-lib:integer4) j k l)
         (type f2cl-lib:logical notrna notrnb))
(setf notrna
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trana "N")
          (declare (ignore var-1))
          (when var-0 (setf trana var-0))
          ret-val))
(setf notrnb
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame tranb "N")
          (declare (ignore var-1))
          (when var-0 (setf tranb var-0))
          ret-val))
(setf info 0)
(cond
 ((and (not notrna)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trana "C")
          (declare (ignore var-1))
          (when var-0 (setf trana var-0))
          ret-val)))
  (setf info -1))
 ((and (not notrnb)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame tranb "C")
          (declare (ignore var-1))
          (when var-0 (setf tranb var-0))
          ret-val)))
  (setf info -2))
 ((and (/= isgn 1) (/= isgn (f2cl-lib:int-sub 1))) (setf info -3))
 ((< m 0) (setf info -4)) ((< n 0) (setf info -5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9))
 ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -11)))
(cond
 ((/= info 0) (xerbla "ZTRSYL" (f2cl-lib:int-sub info))
  (go end_label)))
(setf scale one)
```

```
(if (or (= m 0) (= n 0))
    (go end_label))
(setf eps (dlamch "P"))
(setf smlnum (dlamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (* smlnum (f2cl-lib:dble (f2cl-lib:int-mul m n))) eps))
(setf bignum (/ one smlnum))
(setf smin
        (max smlnum
             (* eps
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                    (zlange "M" m m a lda dum)
                  (declare (ignore var-0 var-2 var-3 var-4 var-5))
                  (setf m var-1)
                  ret-val))
             (* eps
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                    (zlange "M" n n b ldb$ dum)
                  (declare (ignore var-0 var-2 var-3 var-4 var-5))
                  (setf n var-1)
                  ret-val))))
(setf sgn (coerce (the f2cl-lib:integer4 isgn) 'double-float))
(cond
 ((and notrna notrnb)
  (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                ((> l n) nil)
    (tagbody
      (f2cl-lib:fdo (k m (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                    ((> k 1) nil)
        (tagbody
          (setf suml
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4)
                      (zdotu (f2cl-lib:int-sub m k)
                       (f2cl-lib:array-slice a-%data%
                                             f2cl-lib:complex16
                                             (k
                                              (min
                                               (the f2cl-lib:integer4
                                                    (f2cl-lib:int-add k
                                                                      1))
                                               (the f2cl-lib:integer4
                                                    m)))
```

```
                                        ((1 lda) (1 *))
                                        a-%offset%)
                    lda
                    (f2cl-lib:array-slice c-%data%
                                        f2cl-lib:complex16
                                        ((min
                                          (f2cl-lib:int-add k 1)
                                          m)
                                         l)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                    1)
                  (declare (ignore var-0 var-1 var-3 var-4))
                  (when var-2 (setf lda var-2))
                  ret-val))
(setf sumr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (zdotu (f2cl-lib:int-sub l 1)
             (f2cl-lib:array-slice c-%data%
                                        f2cl-lib:complex16 (k 1)
                                        ((1 ldc) (1 *))
                                        c-%offset%)
             ldc
             (f2cl-lib:array-slice b-%data%
                                        f2cl-lib:complex16 (1 l)
                                        ((1 ldb$) (1 *))
                                        b-%offset%)
             1)
          (declare (ignore var-0 var-1 var-3 var-4))
          (when var-2 (setf ldc var-2))
          ret-val))
(setf vec
        (-
         (f2cl-lib:fref c-%data% (k l) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf scaloc one)
(setf a11
        (+
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         (* sgn
            (f2cl-lib:fref b-%data% (l l) ((1 ldb$) (1 *))
                           b-%offset%))))
(setf da11
        (+ (abs (f2cl-lib:dble a11))
           (abs (f2cl-lib:dimag a11))))
(cond
 ((<= da11 smin) (setf a11 (coerce smin 'f2cl-lib:complex16))
```

```
                (setf da11 smin) (setf info 1)))
              (setf db
                      (+ (abs (f2cl-lib:dble vec))
                         (abs (f2cl-lib:dimag vec))))
              (cond
               ((and (< da11 one) (> db one))
                (if (> db (* bignum da11))
                    (setf scaloc (/ one db)))))
              (setf x11 (zladiv (* vec (f2cl-lib:dcmplx scaloc)) a11))
              (cond
               ((/= scaloc one)
                (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                              ((> j n) nil)
                  (tagbody
                    (multiple-value-bind (var-0 var-1 var-2 var-3)
                        (zdscal m scaloc
                         (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                               (1 j) ((1 ldc) (1 *))
                                               c-%offset%)
                         1)
                      (declare (ignore var-2 var-3))
                      (when var-0 (setf m var-0))
                      (when var-1 (setf scaloc var-1)))
                  label10))
                (setf scale (* scale scaloc))))
              (setf (f2cl-lib:fref c-%data% (k l) ((1 ldc) (1 *))
                                    c-%offset%)
                      x11)
            label20))
        label30)))
    ((and (not notrna) notrnb)
     (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                   ((> l n) nil)
       (tagbody
         (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                       ((> k m) nil)
           (tagbody
             (setf suml
                     (zdotc (f2cl-lib:int-sub k 1)
                      (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                            (1 k) ((1 lda) (1 *))
                                            a-%offset%)
                      1
                      (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                            (1 l) ((1 ldc) (1 *))
                                            c-%offset%)
                      1))
             (setf sumr
                     (multiple-value-bind
                         (ret-val var-0 var-1 var-2 var-3 var-4)
```

```
                (zdotu (f2cl-lib:int-sub l 1)
                 (f2cl-lib:array-slice c-%data%
                                       f2cl-lib:complex16 (k 1)
                                       ((1 ldc) (1 *))
                                       c-%offset%)
                 ldc
                 (f2cl-lib:array-slice b-%data%
                                       f2cl-lib:complex16 (1 l)
                                       ((1 ldb$) (1 *))
                                       b-%offset%)
                 1)
              (declare (ignore var-0 var-1 var-3 var-4))
              (when var-2 (setf ldc var-2))
              ret-val))
(setf vec
        (-
         (f2cl-lib:fref c-%data% (k l) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn sumr))))
(setf scaloc one)
(setf a11
        (+
         (f2cl-lib:dconjg
          (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                         a-%offset%))
         (* sgn
            (f2cl-lib:fref b-%data% (l l) ((1 ldb$) (1 *))
                           b-%offset%))))
(setf da11
        (+ (abs (f2cl-lib:dble a11))
           (abs (f2cl-lib:dimag a11))))
(cond
 ((<= da11 smin) (setf a11 (coerce smin 'f2cl-lib:complex16))
  (setf da11 smin) (setf info 1)))
(setf db
        (+ (abs (f2cl-lib:dble vec))
           (abs (f2cl-lib:dimag vec))))
(cond
 ((and (< da11 one) (> db one))
  (if (> db (* bignum da11))
      (setf scaloc (/ one db)))))
(setf x11 (zladiv (* vec (f2cl-lib:dcmplx scaloc)) a11))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (zdscal m scaloc
           (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
```

```
                                          (1 j) ((1 ldc) (1 *))
                                          c-%offset%)
                      1)
                  (declare (ignore var-2 var-3))
                  (when var-0 (setf m var-0))
                  (when var-1 (setf scaloc var-1)))
              label40))
            (setf scale (* scale scaloc))))
          (setf (f2cl-lib:fref c-%data% (k l) ((1 ldc) (1 *))
                                c-%offset%)
                  x11)
        label50))
    label60)))
((and (not notrna) (not notrnb))
 (f2cl-lib:fdo (l n (f2cl-lib:int-add l (f2cl-lib:int-sub 1)))
               ((> l 1) nil)
   (tagbody
     (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                   ((> k m) nil)
       (tagbody
         (setf suml
                 (zdotc (f2cl-lib:int-sub k 1)
                  (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                        (1 k) ((1 lda) (1 *))
                                        a-%offset%)
                  1
                  (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                        (1 l) ((1 ldc) (1 *))
                                        c-%offset%)
                  1))
         (setf sumr
                 (multiple-value-bind
                     (ret-val var-0 var-1 var-2 var-3 var-4)
                     (zdotc (f2cl-lib:int-sub n l)
                      (f2cl-lib:array-slice c-%data%
                                            f2cl-lib:complex16
                                            (k
                                             (min
                                              (the f2cl-lib:integer4
                                                   (f2cl-lib:int-add l
                                                                     1))
                                              (the f2cl-lib:integer4
                                                   n)))
                                            ((1 ldc) (1 *))
                                            c-%offset%)
                      ldc
                      (f2cl-lib:array-slice b-%data%
                                            f2cl-lib:complex16
                                            (l
                                             (min
```

```
                                        (the f2cl-lib:integer4
                                             (f2cl-lib:int-add l
                                                               1))
                                        (the f2cl-lib:integer4
                                             n)))
                                     ((1 ldb$) (1 *))
                                     b-%offset%)
                ldb$)
             (declare (ignore var-0 var-1 var-3))
             (when var-2 (setf ldc var-2))
             (when var-4 (setf ldb$ var-4))
             ret-val))
(setf vec
        (-
         (f2cl-lib:fref c-%data% (k l) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn (f2cl-lib:dconjg sumr)))))
(setf scaloc one)
(setf a11
        (coerce
         (f2cl-lib:dconjg
          (+
           (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                          a-%offset%)
           (* sgn
              (f2cl-lib:fref b-%data% (l l) ((1 ldb$) (1 *))
                             b-%offset%))))
         'f2cl-lib:complex16))
(setf da11
        (+ (abs (f2cl-lib:dble a11))
           (abs (f2cl-lib:dimag a11))))
(cond
 ((<= da11 smin) (setf a11 (coerce smin 'f2cl-lib:complex16))
  (setf da11 smin) (setf info 1)))
(setf db
        (+ (abs (f2cl-lib:dble vec))
           (abs (f2cl-lib:dimag vec))))
(cond
 ((and (< da11 one) (> db one))
  (if (> db (* bignum da11))
      (setf scaloc (/ one db)))))
(setf x11 (zladiv (* vec (f2cl-lib:dcmplx scaloc)) a11))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (zdscal m scaloc
           (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
```

```
                                          (1 j) ((1 ldc) (1 *))
                                          c-%offset%)
                      1)
                  (declare (ignore var-2 var-3))
                  (when var-0 (setf m var-0))
                  (when var-1 (setf scaloc var-1)))
              label70))
            (setf scale (* scale scaloc))))
          (setf (f2cl-lib:fref c-%data% (k l) ((1 ldc) (1 *))
                                c-%offset%)
                  x11)
        label80))
    label90)))
((and notrna (not notrnb))
 (f2cl-lib:fdo (l n (f2cl-lib:int-add l (f2cl-lib:int-sub 1)))
               ((> l 1) nil)
   (tagbody
     (f2cl-lib:fdo (k m (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                   ((> k 1) nil)
       (tagbody
         (setf suml
                 (multiple-value-bind
                     (ret-val var-0 var-1 var-2 var-3 var-4)
                     (zdotu (f2cl-lib:int-sub m k)
                      (f2cl-lib:array-slice a-%data%
                                            f2cl-lib:complex16
                                            (k
                                             (min
                                              (the f2cl-lib:integer4
                                                   (f2cl-lib:int-add k
                                                                     1))
                                              (the f2cl-lib:integer4
                                                   m)))
                                            ((1 lda) (1 *))
                                            a-%offset%)
                      lda
                      (f2cl-lib:array-slice c-%data%
                                            f2cl-lib:complex16
                                            ((min
                                              (f2cl-lib:int-add k 1)
                                              m)
                                             l)
                                            ((1 ldc) (1 *))
                                            c-%offset%)
                      1)
                   (declare (ignore var-0 var-1 var-3 var-4))
                   (when var-2 (setf lda var-2))
                   ret-val))
         (setf sumr
                 (multiple-value-bind
```

```
                (ret-val var-0 var-1 var-2 var-3 var-4)
                (zdotc (f2cl-lib:int-sub n l)
                 (f2cl-lib:array-slice c-%data%
                                       f2cl-lib:complex16
                                       (k
                                        (min
                                         (the f2cl-lib:integer4
                                              (f2cl-lib:int-add l
                                                                1))
                                         (the f2cl-lib:integer4
                                              n)))
                                       ((1 ldc) (1 *))
                                       c-%offset%)
                 ldc
                 (f2cl-lib:array-slice b-%data%
                                       f2cl-lib:complex16
                                       (l
                                        (min
                                         (the f2cl-lib:integer4
                                              (f2cl-lib:int-add l
                                                                1))
                                         (the f2cl-lib:integer4
                                              n)))
                                       ((1 ldb$) (1 *))
                                       b-%offset%)
                 ldb$)
               (declare (ignore var-0 var-1 var-3))
               (when var-2 (setf ldc var-2))
               (when var-4 (setf ldb$ var-4))
               ret-val))
(setf vec
        (-
         (f2cl-lib:fref c-%data% (k l) ((1 ldc) (1 *))
                        c-%offset%)
         (+ suml (* sgn (f2cl-lib:dconjg sumr)))))
(setf scaloc one)
(setf a11
        (+
         (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                        a-%offset%)
         (* sgn
            (f2cl-lib:dconjg
             (f2cl-lib:fref b-%data% (l l) ((1 ldb$) (1 *))
                            b-%offset%)))))
(setf da11
        (+ (abs (f2cl-lib:dble a11))
           (abs (f2cl-lib:dimag a11))))
(cond
 ((<= da11 smin) (setf a11 (coerce smin 'f2cl-lib:complex16))
  (setf da11 smin) (setf info 1)))
```

```
                  (setf db
                          (+ (abs (f2cl-lib:dble vec))
                             (abs (f2cl-lib:dimag vec))))
                  (cond
                   ((and (< da11 one) (> db one))
                    (if (> db (* bignum da11))
                        (setf scaloc (/ one db)))))
                  (setf x11 (zladiv (* vec (f2cl-lib:dcmplx scaloc)) a11))
                  (cond
                   ((/= scaloc one)
                    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                                  ((> j n) nil)
                      (tagbody
                        (multiple-value-bind (var-0 var-1 var-2 var-3)
                            (zdscal m scaloc
                             (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                                   (1 j) ((1 ldc) (1 *))
                                                   c-%offset%)
                             1)
                          (declare (ignore var-2 var-3))
                          (when var-0 (setf m var-0))
                          (when var-1 (setf scaloc var-1)))
                       label100))
                    (setf scale (* scale scaloc))))
                  (setf (f2cl-lib:fref c-%data% (k l) ((1 ldc) (1 *))
                                       c-%offset%)
                          x11)
                label110))
            label120))))
        (go end_label)
       end_label
        (return
         (values trana tranb nil m n nil lda nil ldb$ nil ldc scale info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ztrsyl
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
```

```
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (double-float)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::trana
  fortran-to-lisp::tranb nil
  fortran-to-lisp::m
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldc
  fortran-to-lisp::scale
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zladiv
  fortran-to-lisp::zlange
  fortran-to-lisp::xerbla))))
```

———-

Chapter 62

LAPACK - Symmetric Matrix, Auxilliary Routines, Double

62.1 dlansy

62.1.1 Axiom unit tests

— **dlansy.input** —

```
)set break resume
)sys rm -f dlansy.output
)spool dlansy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

62.1.2 Axiom help page

— **dlansy.help** —

```
======================================================================
dlansy examples
======================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function dlansy (NORM, UPLO, N, A, LDA, WORK)
           DLANSY returns the value of the 1-norm, or the Frobenius norm, or
           the infinity norm, or the element of largest absolute value of a
           real symmetric matrix.

Function/Subroutine Documentation
   double precision function dlansy (character NORM, character UPLO, integer
       N, double precision, dimension( lda, * ) A, integer LDA, double
       precision, dimension( * ) WORK)
       DLANSY returns the value of the 1-norm, or the Frobenius norm, or the
       infinity norm, or the element of largest absolute value of a real
       symmetric matrix.

Purpose:
   DLANSY  returns the value of the one norm,  or the Frobenius norm, or
   the  infinity norm,  or the  element of  largest absolute value  of a
   real symmetric matrix A.

 Returns:

   DLANSY

       DLANSY = ( max(abs(A(i,j))), NORM = 'M' or 'm'
                (
                ( norm1(A),         NORM = '1', 'O' or 'o'
                (
                ( normI(A),         NORM = 'I' or 'i'
                (
                 ( normF(A),         NORM = 'F', 'f', 'E' or 'e'

   where  norm1  denotes the  one norm of a matrix (maximum column sum),
   normI  denotes the  infinity norm  of a matrix  (maximum row sum) and
   normF  denotes the  Frobenius norm of a matrix (square root of sum of
   squares).  Note that  max(abs(A(i,j)))  is not a consistent matrix norm.

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies the value to be returned in DLANSY as described above.

 UPLO
   UPLO is CHARACTER*1
```

```
  Specifies whether the upper or lower triangular part of the
  symmetric matrix A is to be referenced.
  = 'U':  Upper triangular part of A is referenced
  = 'L':  Lower triangular part of A is referenced

N
  N is INTEGER
  The order of the matrix A.  N >= 0.  When N = 0, DLANSY is set to zero.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  The symmetric matrix A.  If UPLO = 'U', the leading n by n
  upper triangular part of A contains the upper triangular part
  of the matrix A, and the strictly lower triangular part of A
  is not referenced.  If UPLO = 'L', the leading n by n lower
  triangular part of A contains the lower triangular part of
  the matrix A, and the strictly upper triangular part of A is
  not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(N,1).

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK)),
  where LWORK >= N when NORM = 'I' or '1' or 'O'; otherwise,
  WORK is not referenced.
```

62.1.3 fortran code

— dlansy.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION DLANSY( NORM, UPLO, N, A, LDA, WORK )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          NORM, UPLO
      INTEGER            LDA, N
*     ..
*     .. Array Arguments ..
```

```
      DOUBLE PRECISION   A( LDA, * ), WORK( * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   ABSA, SCALE, SUM, VALUE
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASSQ
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, DISNAN
      EXTERNAL           LSAME, DISNAN
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SQRT
*     ..
*     .. Executable Statements ..
*
      IF( N.EQ.0 ) THEN
         VALUE = ZERO
      ELSE IF( LSAME( NORM, 'M' ) ) THEN
*
*        Find max(abs(A(i,j))).
*
         VALUE = ZERO
         IF( LSAME( UPLO, 'U' ) ) THEN
            DO 20 J = 1, N
               DO 10 I = 1, J
                  SUM = ABS( A( I, J ) )
                  IF( VALUE .LT. SUM .OR. DISNAN( SUM ) ) VALUE = SUM
   10          CONTINUE
   20       CONTINUE
         ELSE
            DO 40 J = 1, N
               DO 30 I = J, N
                  SUM = ABS( A( I, J ) )
                  IF( VALUE .LT. SUM .OR. DISNAN( SUM ) ) VALUE = SUM
   30          CONTINUE
   40       CONTINUE
         END IF
      ELSE IF( ( LSAME( NORM, 'I' ) ) .OR. ( LSAME( NORM, 'O' ) ) .OR.
     $         ( NORM.EQ.'1' ) ) THEN
*
```

```
*        Find normI(A) ( = norm1(A), since A is symmetric).
*
         VALUE = ZERO
         IF( LSAME( UPLO, 'U' ) ) THEN
            DO 60 J = 1, N
               SUM = ZERO
               DO 50 I = 1, J - 1
                  ABSA = ABS( A( I, J ) )
                  SUM = SUM + ABSA
                  WORK( I ) = WORK( I ) + ABSA
   50          CONTINUE
               WORK( J ) = SUM + ABS( A( J, J ) )
   60       CONTINUE
            DO 70 I = 1, N
               SUM = WORK( I )
               IF( VALUE .LT. SUM .OR. DISNAN( SUM ) ) VALUE = SUM
   70       CONTINUE
         ELSE
            DO 80 I = 1, N
               WORK( I ) = ZERO
   80       CONTINUE
            DO 100 J = 1, N
               SUM = WORK( J ) + ABS( A( J, J ) )
               DO 90 I = J + 1, N
                  ABSA = ABS( A( I, J ) )
                  SUM = SUM + ABSA
                  WORK( I ) = WORK( I ) + ABSA
   90          CONTINUE
               IF( VALUE .LT. SUM .OR. DISNAN( SUM ) ) VALUE = SUM
  100       CONTINUE
         END IF
      ELSE IF( ( LSAME( NORM, 'F' ) ) .OR. ( LSAME( NORM, 'E' ) ) ) THEN
*
*        Find normF(A).
*
         SCALE = ZERO
         SUM = ONE
         IF( LSAME( UPLO, 'U' ) ) THEN
            DO 110 J = 2, N
               CALL DLASSQ( J-1, A( 1, J ), 1, SCALE, SUM )
  110       CONTINUE
         ELSE
            DO 120 J = 1, N - 1
               CALL DLASSQ( N-J, A( J+1, J ), 1, SCALE, SUM )
  120       CONTINUE
         END IF
         SUM = 2*SUM
         CALL DLASSQ( N, A, LDA+1, SCALE, SUM )
         VALUE = SCALE*SQRT( SUM )
      END IF
```

```
*
      DLANSY = VALUE
      RETURN
*
*     End of DLANSY
*
      END
```

———-

62.1.4 lisp code

— **dlansy.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dlansy (norm uplo n a lda work)
    (declare (type (array double-float (*)) work a)
             (type (f2cl-lib:integer4) lda n)
             (type (string *) uplo norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((absa 0.0d0) (scale 0.0d0) (sum 0.0d0) (value 0.0d0) (i 0) (j 0)
             (dlansy 0.0d0))
        (declare (type (f2cl-lib:integer4) i j)
                 (type (double-float) absa scale sum value dlansy))
        (cond ((= n 0) (setf value zero))
              ((multiple-value-bind (ret-val var-0 var-1)
                   (lsame norm "M")
                 (declare (ignore var-1))
                 (when var-0 (setf norm var-0))
                 ret-val)
               (setf value zero)
               (cond
                ((multiple-value-bind (ret-val var-0 var-1)
                     (lsame uplo "U")
                   (declare (ignore var-1))
                   (when var-0 (setf uplo var-0))
                   ret-val)
                 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                               ((> j n) nil)
                   (tagbody
                     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                        ((> i j) nil)
                (tagbody
                  (setf sum
                          (abs
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%)))
                  (if (or (< value sum)
                          (multiple-value-bind (ret-val var-0)
                              (disnan sum)
                            (declare (ignore))
                            (setf sum var-0)
                            ret-val))
                      (setf value sum))
                 label10))
             label20)))
        (t
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                           ((> i n) nil)
               (tagbody
                 (setf sum
                         (abs
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)))
                 (if (or (< value sum)
                         (multiple-value-bind (ret-val var-0)
                             (disnan sum)
                           (declare (ignore))
                           (setf sum var-0)
                           ret-val))
                     (setf value sum))
                label30))
             label40)))))
      ((or
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame norm "I")
          (declare (ignore var-1))
          (when var-0 (setf norm var-0))
          ret-val)
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame norm "O")
          (declare (ignore var-1))
          (when var-0 (setf norm var-0))
          ret-val)
        (f2cl-lib:fstring-= norm "1"))
       (setf value zero)
       (cond
        ((multiple-value-bind (ret-val var-0 var-1)
```

```
      (lsame uplo "U")
    (declare (ignore var-1))
    (when var-0 (setf uplo var-0))
    ret-val)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf sum zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (f2cl-lib:int-add j
                                          (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf absa
                  (abs
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)))
          (setf sum (+ sum absa))
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                  (+
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                   absa))
         label50))
      (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                           work-%offset%)
              (+ sum
                 (abs
                  (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                 a-%offset%))))
    label60))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf sum
              (f2cl-lib:fref work-%data% (i) ((1 *))
                             work-%offset%))
      (if (or (< value sum)
              (multiple-value-bind (ret-val var-0)
                  (disnan sum)
                (declare (ignore))
                (setf sum var-0)
                ret-val))
          (setf value sum))
    label70)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
```

```
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                zero)
         label80))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf sum
                  (+
                   (f2cl-lib:fref work-%data% (j) ((1 *))
                                  work-%offset%)
                   (abs
                    (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                   a-%offset%))))
          (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                           (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf absa
                      (abs
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)))
              (setf sum (+ sum absa))
              (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)
                      (+
                       (f2cl-lib:fref work-%data% (i) ((1 *))
                                      work-%offset%)
                       absa))
             label90))
          (if (or (< value sum)
                  (multiple-value-bind (ret-val var-0)
                      (disnan sum)
                    (declare (ignore))
                    (setf sum var-0)
                    ret-val))
              (setf value sum))
         label100)))))
    ((or
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame norm "F")
        (declare (ignore var-1))
        (when var-0 (setf norm var-0))
        ret-val)
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame norm "E")
        (declare (ignore var-1))
        (when var-0 (setf norm var-0))
        ret-val))
```

```
                (setf scale zero) (setf sum one)
                (cond
                 ((multiple-value-bind (ret-val var-0 var-1)
                      (lsame uplo "U")
                    (declare (ignore var-1))
                    (when var-0 (setf uplo var-0))
                    ret-val)
                  (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                                ((> j n) nil)
                    (tagbody
                      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                          (dlassq (f2cl-lib:int-sub j 1)
                           (f2cl-lib:array-slice a-%data% double-float (1 j)
                                                 ((1 lda) (1 *)) a-%offset%)
                           1 scale sum)
                        (declare (ignore var-0 var-1 var-2))
                        (when var-3 (setf scale var-3))
                        (when var-4 (setf sum var-4)))
                     label110)))
                 (t
                  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                                ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                                 nil)
                    (tagbody
                      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                          (dlassq (f2cl-lib:int-sub n j)
                           (f2cl-lib:array-slice a-%data% double-float
                                                 ((+ j 1) j) ((1 lda) (1 *))
                                                 a-%offset%)
                           1 scale sum)
                        (declare (ignore var-0 var-1 var-2))
                        (when var-3 (setf scale var-3))
                        (when var-4 (setf sum var-4)))
                     label120))))
                (setf sum (* 2 sum))
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (dlassq n a (f2cl-lib:int-add lda 1) scale sum)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf n var-0))
                  (when var-3 (setf scale var-3))
                  (when var-4 (setf sum var-4)))
                (setf value (* scale (f2cl-lib:fsqrt sum)))))
        (setf dlansy value)
        (go end_label)
       end_label
        (return (values dlansy norm uplo n nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
```

```
(setf (gethash 'fortran-to-lisp::dlansy
               fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((string) (string)
                                            (fortran-to-lisp::integer4)
                                            (array double-float (*))
                                            (fortran-to-lisp::integer4)
                                            (array double-float (*)))
                                          :return-values
                                          '(fortran-to-lisp::norm
                                            fortran-to-lisp::uplo
                                            fortran-to-lisp::n nil nil nil)
                                          :calls '(fortran-to-lisp::disnan))))
```

———-

62.2 dlaqsy

62.2.1 Axiom unit tests

— **dlaqsy.input** —

```
)set break resume
)sys rm -f dlaqsy.output
)spool dlaqsy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

62.2.2 Axiom help page

— **dlaqsy.help** —

```
======================================================================
dlaqsy examples
======================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlaqsy (UPLO, N, A, LDA, S, SCOND, AMAX, EQUED)
           DLAQSY scales a symmetric/Hermitian matrix, using scaling factors
           computed by spoequ.

Function/Subroutine Documentation
   subroutine dlaqsy (character UPLO, integer N, double precision, dimension(
       lda, * ) A, integer LDA, double precision, dimension( * ) S, double
       precision SCOND, double precision AMAX, character EQUED)
       DLAQSY scales a symmetric/Hermitian matrix, using scaling factors
       computed by spoequ.

Purpose:
   DLAQSY equilibrates a symmetric matrix A using the scaling factors
   in the vector S.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is stored.
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   n by n upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading n by n lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

   On exit, if EQUED = 'Y', the equilibrated matrix:
   diag(S) * A * diag(S).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(N,1).
```

```
 S
   S is DOUBLE PRECISION array, dimension (N)
   The scale factors for A.

 SCOND
   SCOND is DOUBLE PRECISION
   Ratio of the smallest S(i) to the largest S(i).

 AMAX
   AMAX is DOUBLE PRECISION
   Absolute value of largest matrix entry.

 EQUED
   EQUED is CHARACTER*1
   Specifies whether or not equilibration was done.
   = 'N':  No equilibration.
   = 'Y':  Equilibration was done, i.e., A has been replaced by
   diag(S) * A * diag(S).

Internal Parameters:

   THRESH is a threshold value used to decide if scaling should be done
   based on the ratio of the scaling factors.  If SCOND < THRESH,
   scaling is done.

   LARGE and SMALL are threshold values used to decide if scaling should
   be done based on the absolute size of the largest matrix element.
   If AMAX > LARGE or AMAX < SMALL, scaling is done.
```

———-

62.2.3 fortran code

— dlaqsy.f —

```
*  =====================================================================
      SUBROUTINE DLAQSY( UPLO, N, A, LDA, S, SCOND, AMAX, EQUED )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, UPLO
      INTEGER            LDA, N
```

```
      DOUBLE PRECISION   AMAX, SCOND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), S( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, THRESH
      PARAMETER          ( ONE = 1.0D+0, THRESH = 0.1D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   CJ, LARGE, SMALL
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( N.LE.0 ) THEN
         EQUED = 'N'
         RETURN
      END IF
*
*     Initialize LARGE and SMALL.
*
      SMALL = DLAMCH( 'Safe minimum' ) / DLAMCH( 'Precision' )
      LARGE = ONE / SMALL
*
      IF( SCOND.GE.THRESH .AND. AMAX.GE.SMALL .AND. AMAX.LE.LARGE ) THEN
*
*        No equilibration
*
         EQUED = 'N'
      ELSE
*
*        Replace A by diag(S) * A * diag(S).
*
         IF( LSAME( UPLO, 'U' ) ) THEN
*
*           Upper triangle of A is stored.
*
            DO 20 J = 1, N
               CJ = S( J )
```

```
               DO 10 I = 1, J
                  A( I, J ) = CJ*S( I )*A( I, J )
   10          CONTINUE
   20       CONTINUE
         ELSE
*
*           Lower triangle of A is stored.
*
            DO 40 J = 1, N
               CJ = S( J )
               DO 30 I = J, N
                  A( I, J ) = CJ*S( I )*A( I, J )
   30          CONTINUE
   40       CONTINUE
         END IF
         EQUED = 'Y'
      END IF
*
      RETURN
*
*     End of DLAQSY
*
      END
```

———-

62.2.4 lisp code

— dlaqsy.lisp —

```
(let* ((one 1.0d0) (thresh 0.1d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.1d0 0.1d0) thresh)
           (ignorable one thresh))
  (defun dlaqsy (uplo n a lda s scond amax equed)
    (declare (type (double-float) amax scond)
             (type (array double-float (*)) s a)
             (type (f2cl-lib:integer4) lda n)
             (type (string *) equed uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (equed character equed-%data% equed-%offset%)
         (a double-float a-%data% a-%offset%)
         (s double-float s-%data% s-%offset%))
      (prog ((cj 0.0d0) (large 0.0d0) (small 0.0d0) (i 0) (j 0))
        (declare (type (double-float) cj large small)
                 (type (f2cl-lib:integer4) i j))
        (cond
```

```
  ((<= n 0) (f2cl-lib:f2cl-set-string equed "N" (string 1))
   (go end_label)))
 (setf small (/ (dlamch "Safe minimum") (dlamch "Precision")))
 (setf large (/ one small))
 (cond
  ((and (>= scond thresh) (>= amax small) (<= amax large))
   (f2cl-lib:f2cl-set-string equed "N" (string 1)))
  (t
   (cond
    ((multiple-value-bind (ret-val var-0 var-1)
         (lsame uplo "U")
       (declare (ignore var-1))
       (when var-0 (setf uplo var-0))
       ret-val)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf cj (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i j) nil)
           (tagbody
             (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                   (* cj
                      (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%)))
             label10))
         label20)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf cj (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
         (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                   (* cj
                      (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%)))
             label30))
         label40))))
   (f2cl-lib:f2cl-set-string equed "Y" (string 1))))
 (go end_label)
end_label
 (return (values uplo nil nil nil nil nil nil equed))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlaqsy
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (double-float) (double-float)
                                              (string))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil nil
                                              nil nil nil
                                              fortran-to-lisp::equed)
                                            :calls 'nil)))
```

———-

62.3 dlasy2

62.3.1 Axiom unit tests

— **dlasy2.input** —

```
)set break resume
)sys rm -f dlasy2.output
)spool dlasy2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

62.3.2 Axiom help page

— **dlasy2.help** —

```
====================================================================
dlasy2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlasy2 (LTRANL, LTRANR, ISGN, N1, N2, TL, LDTL, TR, LDTR, B,
           LDB, SCALE, X, LDX, XNORM, INFO)
           DLASY2 solves the Sylvester matrix equation where the matrices are
           of order 1 or 2.

Function/Subroutine Documentation
   subroutine dlasy2 (logical LTRANL, logical LTRANR, integer ISGN, integer
       N1, integer N2, double precision, dimension( ldtl, * ) TL, integer
       LDTL, double precision, dimension( ldtr, * ) TR, integer LDTR, double
       precision, dimension( ldb, * ) B, integer LDB, double precision SCALE,
       double precision, dimension( ldx, * ) X, integer LDX, double precision
       XNORM, integer INFO)
       DLASY2 solves the Sylvester matrix equation where the matrices are of
       order 1 or 2.

Purpose:
   DLASY2 solves for the N1 by N2 matrix X, 1 <= N1,N2 <= 2, in

   op(TL)*X + ISGN*X*op(TR) = SCALE*B,

   where TL is N1 by N1, TR is N2 by N2, B is N1 by N2, and ISGN = 1 or -1.
   op(T) = T or T**T, where T**T denotes the transpose of T.

Parameters:

 LTRANL
   LTRANL is LOGICAL
   On entry, LTRANL specifies the op(TL):
   = .FALSE., op(TL) = TL,
   = .TRUE., op(TL) = TL**T.

 LTRANR
   LTRANR is LOGICAL
   On entry, LTRANR specifies the op(TR):
   = .FALSE., op(TR) = TR,
   = .TRUE., op(TR) = TR**T.

 ISGN
   ISGN is INTEGER
   On entry, ISGN specifies the sign of the equation
```

```
  as described before. ISGN may only be 1 or -1.

N1
  N1 is INTEGER
  On entry, N1 specifies the order of matrix TL.
  N1 may only be 0, 1 or 2.

N2
  N2 is INTEGER
  On entry, N2 specifies the order of matrix TR.
  N2 may only be 0, 1 or 2.

TL
  TL is DOUBLE PRECISION array, dimension (LDTL,2)
  On entry, TL contains an N1 by N1 matrix.

LDTL
  LDTL is INTEGER
  The leading dimension of the matrix TL. LDTL >= max(1,N1).

TR
  TR is DOUBLE PRECISION array, dimension (LDTR,2)
  On entry, TR contains an N2 by N2 matrix.

LDTR
  LDTR is INTEGER
  The leading dimension of the matrix TR. LDTR >= max(1,N2).

B
  B is DOUBLE PRECISION array, dimension (LDB,2)
  On entry, the N1 by N2 matrix B contains the right-hand
  side of the equation.

LDB
  LDB is INTEGER
  The leading dimension of the matrix B. LDB >= max(1,N1).

SCALE
  SCALE is DOUBLE PRECISION
  On exit, SCALE contains the scale factor. SCALE is chosen
  less than or equal to 1 to prevent the solution overflowing.

X
  X is DOUBLE PRECISION array, dimension (LDX,2)
  On exit, X contains the N1 by N2 solution.

LDX
  LDX is INTEGER
  The leading dimension of the matrix X. LDX >= max(1,N1).
```

```
 XNORM
   XNORM is DOUBLE PRECISION
   On exit, XNORM is the infinity-norm of the solution.

 INFO
   INFO is INTEGER
   On exit, INFO is set to
   0: successful exit.
   1: TL and TR have too close eigenvalues, so TL or
      TR is perturbed to get a nonsingular equation.
   NOTE: In the interests of speed, this routine does not
   check the inputs for errors.
```

———-

62.3.3 fortran code

— dlasy2.f —

```
*  =====================================================================
      SUBROUTINE DLASY2( LTRANL, LTRANR, ISGN, N1, N2, TL, LDTL, TR,
     $                   LDTR, B, LDB, SCALE, X, LDX, XNORM, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      LOGICAL            LTRANL, LTRANR
      INTEGER            INFO, ISGN, LDB, LDTL, LDTR, LDX, N1, N2
      DOUBLE PRECISION   SCALE, XNORM
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   B( LDB, * ), TL( LDTL, * ), TR( LDTR, * ),
     $                   X( LDX, * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   TWO, HALF, EIGHT
      PARAMETER          ( TWO = 2.0D+0, HALF = 0.5D+0, EIGHT = 8.0D+0 )
*     ..
*     .. Local Scalars ..
```

```
      LOGICAL            BSWAP, XSWAP
      INTEGER            I, IP, IPIV, IPSV, J, JP, JPSV, K
      DOUBLE PRECISION   BET, EPS, GAM, L21, SGN, SMIN, SMLNUM, TAU1,
     $                   TEMP, U11, U12, U22, XMAX
*     ..
*     .. Local Arrays ..
      LOGICAL            BSWPIV( 4 ), XSWPIV( 4 )
      INTEGER            JPIV( 4 ), LOCL21( 4 ), LOCU12( 4 ),
     $                   LOCU22( 4 )
      DOUBLE PRECISION   BTMP( 4 ), T16( 4, 4 ), TMP( 4 ), X2( 2 )
*     ..
*     .. External Functions ..
      INTEGER            IDAMAX
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           IDAMAX, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DSWAP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Data statements ..
      DATA               LOCU12 / 3, 4, 1, 2 / , LOCL21 / 2, 1, 4, 3 / ,
     $                   LOCU22 / 4, 3, 2, 1 /
      DATA               XSWPIV / .FALSE., .FALSE., .TRUE., .TRUE. /
      DATA               BSWPIV / .FALSE., .TRUE., .FALSE., .TRUE. /
*     ..
*     .. Executable Statements ..
*
*     Do not check the input parameters for errors
*
      INFO = 0
*
*     Quick return if possible
*
      IF( N1.EQ.0 .OR. N2.EQ.0 )
     $   RETURN
*
*     Set constants to control overflow
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' ) / EPS
      SGN = ISGN
*
      K = N1 + N1 + N2 - 2
      GO TO ( 10, 20, 30, 50 )K
*
*     1 by 1: TL11*X + SGN*X*TR11 = B11
*
```

```
   10 CONTINUE
      TAU1 = TL( 1, 1 ) + SGN*TR( 1, 1 )
      BET = ABS( TAU1 )
      IF( BET.LE.SMLNUM ) THEN
         TAU1 = SMLNUM
         BET = SMLNUM
         INFO = 1
      END IF
*
      SCALE = ONE
      GAM = ABS( B( 1, 1 ) )
      IF( SMLNUM*GAM.GT.BET )
     $   SCALE = ONE / GAM
*
      X( 1, 1 ) = ( B( 1, 1 )*SCALE ) / TAU1
      XNORM = ABS( X( 1, 1 ) )
      RETURN
*
*     1 by 2:
*     TL11*[X11 X12] + ISGN*[X11 X12]*op[TR11 TR12]  = [B11 B12]
*                                       [TR21 TR22]
*
   20 CONTINUE
*
      SMIN = MAX( EPS*MAX( ABS( TL( 1, 1 ) ), ABS( TR( 1, 1 ) ),
     $       ABS( TR( 1, 2 ) ), ABS( TR( 2, 1 ) ), ABS( TR( 2, 2 ) ) ),
     $       SMLNUM )
      TMP( 1 ) = TL( 1, 1 ) + SGN*TR( 1, 1 )
      TMP( 4 ) = TL( 1, 1 ) + SGN*TR( 2, 2 )
      IF( LTRANR ) THEN
         TMP( 2 ) = SGN*TR( 2, 1 )
         TMP( 3 ) = SGN*TR( 1, 2 )
      ELSE
         TMP( 2 ) = SGN*TR( 1, 2 )
         TMP( 3 ) = SGN*TR( 2, 1 )
      END IF
      BTMP( 1 ) = B( 1, 1 )
      BTMP( 2 ) = B( 1, 2 )
      GO TO 40
*
*     2 by 1:
*          op[TL11 TL12]*[X11] + ISGN* [X11]*TR11  = [B11]
*            [TL21 TL22] [X21]         [X21]         [B21]
*
   30 CONTINUE
      SMIN = MAX( EPS*MAX( ABS( TR( 1, 1 ) ), ABS( TL( 1, 1 ) ),
     $       ABS( TL( 1, 2 ) ), ABS( TL( 2, 1 ) ), ABS( TL( 2, 2 ) ) ),
     $       SMLNUM )
      TMP( 1 ) = TL( 1, 1 ) + SGN*TR( 1, 1 )
      TMP( 4 ) = TL( 2, 2 ) + SGN*TR( 1, 1 )
```

```
      IF( LTRANL ) THEN
         TMP( 2 ) = TL( 1, 2 )
         TMP( 3 ) = TL( 2, 1 )
      ELSE
         TMP( 2 ) = TL( 2, 1 )
         TMP( 3 ) = TL( 1, 2 )
      END IF
      BTMP( 1 ) = B( 1, 1 )
      BTMP( 2 ) = B( 2, 1 )
   40 CONTINUE
*
*     Solve 2 by 2 system using complete pivoting.
*     Set pivots less than SMIN to SMIN.
*
      IPIV = IDAMAX( 4, TMP, 1 )
      U11 = TMP( IPIV )
      IF( ABS( U11 ).LE.SMIN ) THEN
         INFO = 1
         U11 = SMIN
      END IF
      U12 = TMP( LOCU12( IPIV ) )
      L21 = TMP( LOCL21( IPIV ) ) / U11
      U22 = TMP( LOCU22( IPIV ) ) - U12*L21
      XSWAP = XSWPIV( IPIV )
      BSWAP = BSWPIV( IPIV )
      IF( ABS( U22 ).LE.SMIN ) THEN
         INFO = 1
         U22 = SMIN
      END IF
      IF( BSWAP ) THEN
         TEMP = BTMP( 2 )
         BTMP( 2 ) = BTMP( 1 ) - L21*TEMP
         BTMP( 1 ) = TEMP
      ELSE
         BTMP( 2 ) = BTMP( 2 ) - L21*BTMP( 1 )
      END IF
      SCALE = ONE
      IF( ( TWO*SMLNUM )*ABS( BTMP( 2 ) ).GT.ABS( U22 ) .OR.
     $    ( TWO*SMLNUM )*ABS( BTMP( 1 ) ).GT.ABS( U11 ) ) THEN
         SCALE = HALF / MAX( ABS( BTMP( 1 ) ), ABS( BTMP( 2 ) ) )
         BTMP( 1 ) = BTMP( 1 )*SCALE
         BTMP( 2 ) = BTMP( 2 )*SCALE
      END IF
      X2( 2 ) = BTMP( 2 ) / U22
      X2( 1 ) = BTMP( 1 ) / U11 - ( U12 / U11 )*X2( 2 )
      IF( XSWAP ) THEN
         TEMP = X2( 2 )
         X2( 2 ) = X2( 1 )
         X2( 1 ) = TEMP
      END IF
```

```
      X( 1, 1 ) = X2( 1 )
      IF( N1.EQ.1 ) THEN
         X( 1, 2 ) = X2( 2 )
         XNORM = ABS( X( 1, 1 ) ) + ABS( X( 1, 2 ) )
      ELSE
         X( 2, 1 ) = X2( 2 )
         XNORM = MAX( ABS( X( 1, 1 ) ), ABS( X( 2, 1 ) ) )
      END IF
      RETURN
*
*     2 by 2:
*     op[TL11 TL12]*[X11 X12] +ISGN* [X11 X12]*op[TR11 TR12] = [B11 B12]
*       [TL21 TL22] [X21 X22]        [X21 X22]   [TR21 TR22]   [B21 B22]
*
*     Solve equivalent 4 by 4 system using complete pivoting.
*     Set pivots less than SMIN to SMIN.
*
   50 CONTINUE
      SMIN = MAX( ABS( TR( 1, 1 ) ), ABS( TR( 1, 2 ) ),
     $       ABS( TR( 2, 1 ) ), ABS( TR( 2, 2 ) ) )
      SMIN = MAX( SMIN, ABS( TL( 1, 1 ) ), ABS( TL( 1, 2 ) ),
     $       ABS( TL( 2, 1 ) ), ABS( TL( 2, 2 ) ) )
      SMIN = MAX( EPS*SMIN, SMLNUM )
      BTMP( 1 ) = ZERO
      CALL DCOPY( 16, BTMP, 0, T16, 1 )
      T16( 1, 1 ) = TL( 1, 1 ) + SGN*TR( 1, 1 )
      T16( 2, 2 ) = TL( 2, 2 ) + SGN*TR( 1, 1 )
      T16( 3, 3 ) = TL( 1, 1 ) + SGN*TR( 2, 2 )
      T16( 4, 4 ) = TL( 2, 2 ) + SGN*TR( 2, 2 )
      IF( LTRANL ) THEN
         T16( 1, 2 ) = TL( 2, 1 )
         T16( 2, 1 ) = TL( 1, 2 )
         T16( 3, 4 ) = TL( 2, 1 )
         T16( 4, 3 ) = TL( 1, 2 )
      ELSE
         T16( 1, 2 ) = TL( 1, 2 )
         T16( 2, 1 ) = TL( 2, 1 )
         T16( 3, 4 ) = TL( 1, 2 )
         T16( 4, 3 ) = TL( 2, 1 )
      END IF
      IF( LTRANR ) THEN
         T16( 1, 3 ) = SGN*TR( 1, 2 )
         T16( 2, 4 ) = SGN*TR( 1, 2 )
         T16( 3, 1 ) = SGN*TR( 2, 1 )
         T16( 4, 2 ) = SGN*TR( 2, 1 )
      ELSE
         T16( 1, 3 ) = SGN*TR( 2, 1 )
         T16( 2, 4 ) = SGN*TR( 2, 1 )
         T16( 3, 1 ) = SGN*TR( 1, 2 )
         T16( 4, 2 ) = SGN*TR( 1, 2 )
```

```
      END IF
      BTMP( 1 ) = B( 1, 1 )
      BTMP( 2 ) = B( 2, 1 )
      BTMP( 3 ) = B( 1, 2 )
      BTMP( 4 ) = B( 2, 2 )
*
*     Perform elimination
*
      DO 100 I = 1, 3
         XMAX = ZERO
         DO 70 IP = I, 4
            DO 60 JP = I, 4
               IF( ABS( T16( IP, JP ) ).GE.XMAX ) THEN
                  XMAX = ABS( T16( IP, JP ) )
                  IPSV = IP
                  JPSV = JP
               END IF
   60       CONTINUE
   70    CONTINUE
         IF( IPSV.NE.I ) THEN
            CALL DSWAP( 4, T16( IPSV, 1 ), 4, T16( I, 1 ), 4 )
            TEMP = BTMP( I )
            BTMP( I ) = BTMP( IPSV )
            BTMP( IPSV ) = TEMP
         END IF
         IF( JPSV.NE.I )
     $      CALL DSWAP( 4, T16( 1, JPSV ), 1, T16( 1, I ), 1 )
         JPIV( I ) = JPSV
         IF( ABS( T16( I, I ) ).LT.SMIN ) THEN
            INFO = 1
            T16( I, I ) = SMIN
         END IF
         DO 90 J = I + 1, 4
            T16( J, I ) = T16( J, I ) / T16( I, I )
            BTMP( J ) = BTMP( J ) - T16( J, I )*BTMP( I )
            DO 80 K = I + 1, 4
               T16( J, K ) = T16( J, K ) - T16( J, I )*T16( I, K )
   80       CONTINUE
   90    CONTINUE
  100 CONTINUE
      IF( ABS( T16( 4, 4 ) ).LT.SMIN )
     $   T16( 4, 4 ) = SMIN
      SCALE = ONE
      IF( ( EIGHT*SMLNUM )*ABS( BTMP( 1 ) ).GT.ABS( T16( 1, 1 ) ) .OR.
     $    ( EIGHT*SMLNUM )*ABS( BTMP( 2 ) ).GT.ABS( T16( 2, 2 ) ) .OR.
     $    ( EIGHT*SMLNUM )*ABS( BTMP( 3 ) ).GT.ABS( T16( 3, 3 ) ) .OR.
     $    ( EIGHT*SMLNUM )*ABS( BTMP( 4 ) ).GT.ABS( T16( 4, 4 ) ) ) THEN
         SCALE = ( ONE / EIGHT ) / MAX( ABS( BTMP( 1 ) ),
     $           ABS( BTMP( 2 ) ), ABS( BTMP( 3 ) ), ABS( BTMP( 4 ) ) )
         BTMP( 1 ) = BTMP( 1 )*SCALE
```

```
         BTMP( 2 ) = BTMP( 2 )*SCALE
         BTMP( 3 ) = BTMP( 3 )*SCALE
         BTMP( 4 ) = BTMP( 4 )*SCALE
      END IF
      DO 120 I = 1, 4
         K = 5 - I
         TEMP = ONE / T16( K, K )
         TMP( K ) = BTMP( K )*TEMP
         DO 110 J = K + 1, 4
            TMP( K ) = TMP( K ) - ( TEMP*T16( K, J ) )*TMP( J )
  110    CONTINUE
  120 CONTINUE
      DO 130 I = 1, 3
         IF( JPIV( 4-I ).NE.4-I ) THEN
            TEMP = TMP( 4-I )
            TMP( 4-I ) = TMP( JPIV( 4-I ) )
            TMP( JPIV( 4-I ) ) = TEMP
         END IF
  130 CONTINUE
      X( 1, 1 ) = TMP( 1 )
      X( 2, 1 ) = TMP( 2 )
      X( 1, 2 ) = TMP( 3 )
      X( 2, 2 ) = TMP( 4 )
      XNORM = MAX( ABS( TMP( 1 ) )+ABS( TMP( 3 ) ),
     $        ABS( TMP( 2 ) )+ABS( TMP( 4 ) ) )
      RETURN
*
*     End of DLASY2
*
      END
```

———-

62.3.4 lisp code

— **dlasy2.lisp** —

———-

62.4 dsyswapr

62.4.1 Axiom unit tests

— **dsyswapr.input** —

```
)set break resume
)sys rm -f dsyswapr.output
)spool dsyswapr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

62.4.2 Axiom help page

— **dsyswapr.help** —

```
======================================================================
dsyswapr examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsyswapr (UPLO, N, A, LDA, I1, I2)
           DSYSWAPR applies an elementary permutation on the rows and columns
           of a symmetric matrix.

Function/Subroutine Documentation
   subroutine dsyswapr (character UPLO, integer N, double precision,
       dimension( lda, n ) A, integer LDA, integer I1, integer I2)
       DSYSWAPR applies an elementary permutation on the rows and columns of a
       symmetric matrix.

Purpose:
   DSYSWAPR applies an elementary permutation on the rows and the columns of
   a symmetric matrix.
```

```
Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the NB diagonal matrix D and the multipliers
   used to obtain the factor U or L as computed by DSYTRF.

   On exit, if INFO = 0, the (symmetric) inverse of the original
   matrix.  If UPLO = 'U', the upper triangular part of the
   inverse is formed and the part of A below the diagonal is not
   referenced; if UPLO = 'L' the lower triangular part of the
   inverse is formed and the part of A above the diagonal is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 I1
   I1 is INTEGER
   Index of the first row to swap

 I2
   I2 is INTEGER
   Index of the second row to swap
```

———-

62.4.3 fortran code

— dsyswapr.f —

```
*  =====================================================================
      SUBROUTINE DSYSWAPR( UPLO, N, A, LDA, I1, I2)
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
```

```
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER        UPLO
      INTEGER          I1, I2, LDA, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION A( LDA, N )
*
*  =====================================================================
*
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I
      DOUBLE PRECISION   TMP
*
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL         DSWAP
*     ..
*     .. Executable Statements ..
*
      UPPER = LSAME( UPLO, 'U' )
      IF (UPPER) THEN
*
*         UPPER
*         first swap
*          - swap column I1 and I2 from I1 to I1-1
         CALL DSWAP( I1-1, A(1,I1), 1, A(1,I2), 1 )
*
*          second swap :
*          - swap A(I1,I1) and A(I2,I2)
*          - swap row I1 from I1+1 to I2-1 with col I2 from I1+1 to I2-1
         TMP=A(I1,I1)
         A(I1,I1)=A(I2,I2)
         A(I2,I2)=TMP
*
         DO I=1,I2-I1-1
            TMP=A(I1,I1+I)
            A(I1,I1+I)=A(I1+I,I2)
            A(I1+I,I2)=TMP
         END DO
*
*          third swap
```

```
*          - swap row I1 and I2 from I2+1 to N
         DO I=I2+1,N
            TMP=A(I1,I)
            A(I1,I)=A(I2,I)
            A(I2,I)=TMP
         END DO
*
        ELSE
*
*         LOWER
*         first swap
*          - swap row I1 and I2 from I1 to I1-1
         CALL DSWAP( I1-1, A(I1,1), LDA, A(I2,1), LDA )
*
*         second swap :
*          - swap A(I1,I1) and A(I2,I2)
*          - swap col I1 from I1+1 to I2-1 with row I2 from I1+1 to I2-1
          TMP=A(I1,I1)
          A(I1,I1)=A(I2,I2)
          A(I2,I2)=TMP
*
          DO I=1,I2-I1-1
             TMP=A(I1+I,I1)
             A(I1+I,I1)=A(I2,I1+I)
             A(I2,I1+I)=TMP
          END DO
*
*         third swap
*          - swap col I1 and I2 from I2+1 to N
          DO I=I2+1,N
             TMP=A(I,I1)
             A(I,I1)=A(I,I2)
             A(I,I2)=TMP
          END DO
*
      ENDIF
      END SUBROUTINE DSYSWAPR
```

———-

62.4.4 lisp code

— dsyswapr.lisp —

———-

62.5 dtgsy2

62.5.1 Axiom unit tests

— dtgsy2.input —

```
)set break resume
)sys rm -f dtgsy2.output
)spool dtgsy2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

62.5.2 Axiom help page

— dtgsy2.help —

```
====================================================================
dtgsy2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dtgsy2 (TRANS, IJOB, M, N, A, LDA, B, LDB, C, LDC, D, LDD,
           E, LDE, F, LDF, SCALE, RDSUM, RDSCAL, IWORK, PQ, INFO)
           DTGSY2 solves the generalized Sylvester equation (unblocked
           algorithm).

Function/Subroutine Documentation
   subroutine dtgsy2 (character TRANS, integer IJOB, integer M, integer N,
       double precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldb, * ) B, integer LDB, double precision, dimension( ldc, *
       ) C, integer LDC, double precision, dimension( ldd, * ) D, integer LDD,
       double precision, dimension( lde, * ) E, integer LDE, double precision,
       dimension( ldf, * ) F, integer LDF, double precision SCALE, double
       precision RDSUM, double precision RDSCAL, integer, dimension( * )
       IWORK, integer PQ, integer INFO)
```

```
      DTGSY2 solves the generalized Sylvester equation (unblocked algorithm).

Purpose:
   DTGSY2 solves the generalized Sylvester equation:

   A * R - L * B = scale * C                (1)
   D * R - L * E = scale * F,

   using Level 1 and 2 BLAS. where R and L are unknown M-by-N matrices,
   (A, D), (B, E) and (C, F) are given matrix pairs of size M-by-M,
   N-by-N and M-by-N, respectively, with real entries. (A, D) and (B, E)
   must be in generalized Schur canonical form, i.e. A, B are upper
   quasi triangular and D, E are upper triangular. The solution (R, L)
   overwrites (C, F). 0 <= SCALE <= 1 is an output scaling factor
   chosen to avoid overflow.

   In matrix notation solving equation (1) corresponds to solve
   Z*x = scale*b, where Z is defined as

      Z = [ kron(In, A)  -kron(B**T, Im) ]             (2)
          [ kron(In, D)  -kron(E**T, Im) ],

   Ik is the identity matrix of size k and X**T is the transpose of X.
   kron(X, Y) is the Kronecker product between the matrices X and Y.
   In the process of solving (1), we solve a number of such systems
   where Dim(In), Dim(In) = 1 or 2.

   If TRANS = 'T', solve the transposed system Z**T*y = scale*b for y,
   which is equivalent to solve for R and L in

      A**T * R  + D**T * L   = scale * C           (3)
      R  * B**T + L  * E**T  = scale * -F

   This case is used to compute an estimate of Dif[(A, D), (B, E)] =
   sigma_min(Z) using reverse communicaton with DLACON.

   DTGSY2 also (IJOB >= 1) contributes to the computation in DTGSYL
   of an upper bound on the separation between to matrix pairs. Then
   the input (A, D), (B, E) are sub-pencils of the matrix pair in
   DTGSYL. See DTGSYL for details.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   = 'N', solve the generalized Sylvester equation (1).
   = 'T': solve the 'transposed' system (3).

 IJOB
   IJOB is INTEGER
```

```
  Specifies what kind of functionality to be performed.
  = 0: solve (1) only.
  = 1: A contribution from this subsystem to a Frobenius
       norm-based estimate of the separation between two matrix
       pairs is computed. (look ahead strategy is used).
  = 2: A contribution from this subsystem to a Frobenius
       norm-based estimate of the separation between two matrix
       pairs is computed. (DGECON on sub-systems is used.)
  Not referenced if TRANS = 'T'.

M
  M is INTEGER
  On entry, M specifies the order of A and D, and the row
  dimension of C, F, R and L.

N
  N is INTEGER
  On entry, N specifies the order of B and E, and the column
  dimension of C, F, R and L.

A
  A is DOUBLE PRECISION array, dimension (LDA, M)
  On entry, A contains an upper quasi triangular matrix.

LDA
  LDA is INTEGER
  The leading dimension of the matrix A. LDA >= max(1, M).

B
  B is DOUBLE PRECISION array, dimension (LDB, N)
  On entry, B contains an upper quasi triangular matrix.

LDB
  LDB is INTEGER
  The leading dimension of the matrix B. LDB >= max(1, N).

C
  C is DOUBLE PRECISION array, dimension (LDC, N)
  On entry, C contains the right-hand-side of the first matrix
  equation in (1).
  On exit, if IJOB = 0, C has been overwritten by the
  solution R.

LDC
  LDC is INTEGER
  The leading dimension of the matrix C. LDC >= max(1, M).

D
  D is DOUBLE PRECISION array, dimension (LDD, M)
  On entry, D contains an upper triangular matrix.
```

```
LDD
  LDD is INTEGER
  The leading dimension of the matrix D. LDD >= max(1, M).

E
  E is DOUBLE PRECISION array, dimension (LDE, N)
  On entry, E contains an upper triangular matrix.

LDE
  LDE is INTEGER
  The leading dimension of the matrix E. LDE >= max(1, N).

F
  F is DOUBLE PRECISION array, dimension (LDF, N)
  On entry, F contains the right-hand-side of the second matrix
  equation in (1).
  On exit, if IJOB = 0, F has been overwritten by the
  solution L.

LDF
  LDF is INTEGER
  The leading dimension of the matrix F. LDF >= max(1, M).

SCALE
  SCALE is DOUBLE PRECISION
  On exit, 0 <= SCALE <= 1. If 0 < SCALE < 1, the solutions
  R and L (C and F on entry) will hold the solutions to a
  slightly perturbed system but the input matrices A, B, D and
  E have not been changed. If SCALE = 0, R and L will hold the
  solutions to the homogeneous system with C = F = 0. Normally, SCALE = 1.

RDSUM
  RDSUM is DOUBLE PRECISION
  On entry, the sum of squares of computed contributions to
  the Dif-estimate under computation by DTGSYL, where the
  scaling factor RDSCAL (see below) has been factored out.
  On exit, the corresponding sum of squares updated with the
  contributions from the current sub-system.
  If TRANS = 'T' RDSUM is not touched.
  NOTE: RDSUM only makes sense when DTGSY2 is called by DTGSYL.

RDSCAL
  RDSCAL is DOUBLE PRECISION
  On entry, scaling factor used to prevent overflow in RDSUM.
  On exit, RDSCAL is updated w.r.t. the current contributions
  in RDSUM.
  If TRANS = 'T', RDSCAL is not touched.
  NOTE: RDSCAL only makes sense when DTGSY2 is called by DTGSYL.
```

```
 IWORK
   IWORK is INTEGER array, dimension (M+N+2)

 PQ
   PQ is INTEGER
   On exit, the number of subsystems (of size 2-by-2, 4-by-4 and
   8-by-8) solved by this routine.

 INFO
   INFO is INTEGER
   On exit, if INFO is set to
   =0: Successful exit
   <0: If INFO = -i, the i-th argument had an illegal value.
   >0: The matrix pairs (A, D) and (B, E) have common or very
       close eigenvalues.
```

62.5.3 fortran code

— dtgsy2.f —

```
*  =====================================================================
      SUBROUTINE DTGSY2( TRANS, IJOB, M, N, A, LDA, B, LDB, C, LDC, D,
     $                   LDD, E, LDE, F, LDF, SCALE, RDSUM, RDSCAL,
     $                   IWORK, PQ, INFO )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            IJOB, INFO, LDA, LDB, LDC, LDD, LDE, LDF, M, N,
     $                   PQ
      DOUBLE PRECISION   RDSCAL, RDSUM, SCALE
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * ), C( LDC, * ),
     $                   D( LDD, * ), E( LDE, * ), F( LDF, * )
*     ..
*
*  =====================================================================
*  Replaced various illegal calls to DCOPY by calls to DLASET.
*  Sven Hammarling, 27/5/02.
```

```
*
*     .. Parameters ..
      INTEGER            LDZ
      PARAMETER          ( LDZ = 8 )
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      INTEGER            I, IE, IERR, II, IS, ISP1, J, JE, JJ, JS, JSP1,
     $                   K, MB, NB, P, Q, ZDIM
      DOUBLE PRECISION   ALPHA, SCALOC
*     ..
*     .. Local Arrays ..
      INTEGER            IPIV( LDZ ), JPIV( LDZ )
      DOUBLE PRECISION   RHS( LDZ ), Z( LDZ, LDZ )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DCOPY, DGEMM, DGEMV, DGER, DGESC2,
     $                   DGETC2, DLASET, DLATDF, DSCAL, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Decode and test input parameters
*
      INFO = 0
      IERR = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) ) THEN
         INFO = -1
      ELSE IF( NOTRAN ) THEN
         IF( ( IJOB.LT.0 ) .OR. ( IJOB.GT.2 ) ) THEN
            INFO = -2
         END IF
      END IF
      IF( INFO.EQ.0 ) THEN
         IF( M.LE.0 ) THEN
            INFO = -3
         ELSE IF( N.LE.0 ) THEN
            INFO = -4
         ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
            INFO = -6
         ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
```

```
            INFO = -8
         ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
            INFO = -10
         ELSE IF( LDD.LT.MAX( 1, M ) ) THEN
            INFO = -12
         ELSE IF( LDE.LT.MAX( 1, N ) ) THEN
            INFO = -14
         ELSE IF( LDF.LT.MAX( 1, M ) ) THEN
            INFO = -16
         END IF
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DTGSY2', -INFO )
         RETURN
      END IF
*
*     Determine block structure of A
*
      PQ = 0
      P = 0
      I = 1
   10 CONTINUE
      IF( I.GT.M )
     $   GO TO 20
      P = P + 1
      IWORK( P ) = I
      IF( I.EQ.M )
     $   GO TO 20
      IF( A( I+1, I ).NE.ZERO ) THEN
         I = I + 2
      ELSE
         I = I + 1
      END IF
      GO TO 10
   20 CONTINUE
      IWORK( P+1 ) = M + 1
*
*     Determine block structure of B
*
      Q = P + 1
      J = 1
   30 CONTINUE
      IF( J.GT.N )
     $   GO TO 40
      Q = Q + 1
      IWORK( Q ) = J
      IF( J.EQ.N )
     $   GO TO 40
      IF( B( J+1, J ).NE.ZERO ) THEN
         J = J + 2
```

```
      ELSE
         J = J + 1
      END IF
      GO TO 30
   40 CONTINUE
      IWORK( Q+1 ) = N + 1
      PQ = P*( Q-P-1 )
*
      IF( NOTRAN ) THEN
*
*        Solve (I, J) - subsystem
*           A(I, I) * R(I, J) - L(I, J) * B(J, J) = C(I, J)
*           D(I, I) * R(I, J) - L(I, J) * E(J, J) = F(I, J)
*        for I = P, P - 1, ..., 1; J = 1, 2, ..., Q
*
         SCALE = ONE
         SCALOC = ONE
         DO 120 J = P + 2, Q
            JS = IWORK( J )
            JSP1 = JS + 1
            JE = IWORK( J+1 ) - 1
            NB = JE - JS + 1
            DO 110 I = P, 1, -1
*
               IS = IWORK( I )
               ISP1 = IS + 1
               IE = IWORK( I+1 ) - 1
               MB = IE - IS + 1
               ZDIM = MB*NB*2
*
               IF( ( MB.EQ.1 ) .AND. ( NB.EQ.1 ) ) THEN
*
*                 Build a 2-by-2 system Z * x = RHS
*
                  Z( 1, 1 ) = A( IS, IS )
                  Z( 2, 1 ) = D( IS, IS )
                  Z( 1, 2 ) = -B( JS, JS )
                  Z( 2, 2 ) = -E( JS, JS )
*
*                 Set up right hand side(s)
*
                  RHS( 1 ) = C( IS, JS )
                  RHS( 2 ) = F( IS, JS )
*
*                 Solve Z * x = RHS
*
                  CALL DGETC2( ZDIM, Z, LDZ, IPIV, JPIV, IERR )
                  IF( IERR.GT.0 )
     $               INFO = IERR
*
```

```
                IF( IJOB.EQ.0 ) THEN
                   CALL DGESC2( ZDIM, Z, LDZ, RHS, IPIV, JPIV,
     $                          SCALOC )
                   IF( SCALOC.NE.ONE ) THEN
                      DO 50 K = 1, N
                         CALL DSCAL( M, SCALOC, C( 1, K ), 1 )
                         CALL DSCAL( M, SCALOC, F( 1, K ), 1 )
   50                 CONTINUE
                      SCALE = SCALE*SCALOC
                   END IF
                ELSE
                   CALL DLATDF( IJOB, ZDIM, Z, LDZ, RHS, RDSUM,
     $                          RDSCAL, IPIV, JPIV )
                END IF
*
*               Unpack solution vector(s)
*
                C( IS, JS ) = RHS( 1 )
                F( IS, JS ) = RHS( 2 )
*
*               Substitute R(I, J) and L(I, J) into remaining
*               equation.
*
                IF( I.GT.1 ) THEN
                   ALPHA = -RHS( 1 )
                   CALL DAXPY( IS-1, ALPHA, A( 1, IS ), 1, C( 1, JS ),
     $                         1 )
                   CALL DAXPY( IS-1, ALPHA, D( 1, IS ), 1, F( 1, JS ),
     $                         1 )
                END IF
                IF( J.LT.Q ) THEN
                   CALL DAXPY( N-JE, RHS( 2 ), B( JS, JE+1 ), LDB,
     $                         C( IS, JE+1 ), LDC )
                   CALL DAXPY( N-JE, RHS( 2 ), E( JS, JE+1 ), LDE,
     $                         F( IS, JE+1 ), LDF )
                END IF
*
             ELSE IF( ( MB.EQ.1 ) .AND. ( NB.EQ.2 ) ) THEN
*
*               Build a 4-by-4 system Z * x = RHS
*
                Z( 1, 1 ) = A( IS, IS )
                Z( 2, 1 ) = ZERO
                Z( 3, 1 ) = D( IS, IS )
                Z( 4, 1 ) = ZERO
*
                Z( 1, 2 ) = ZERO
                Z( 2, 2 ) = A( IS, IS )
                Z( 3, 2 ) = ZERO
                Z( 4, 2 ) = D( IS, IS )
```

```
*
                 Z( 1, 3 ) = -B( JS, JS )
                 Z( 2, 3 ) = -B( JS, JSP1 )
                 Z( 3, 3 ) = -E( JS, JS )
                 Z( 4, 3 ) = -E( JS, JSP1 )
*
                 Z( 1, 4 ) = -B( JSP1, JS )
                 Z( 2, 4 ) = -B( JSP1, JSP1 )
                 Z( 3, 4 ) = ZERO
                 Z( 4, 4 ) = -E( JSP1, JSP1 )
*
*                Set up right hand side(s)
*
                 RHS( 1 ) = C( IS, JS )
                 RHS( 2 ) = C( IS, JSP1 )
                 RHS( 3 ) = F( IS, JS )
                 RHS( 4 ) = F( IS, JSP1 )
*
*                Solve Z * x = RHS
*
                 CALL DGETC2( ZDIM, Z, LDZ, IPIV, JPIV, IERR )
                 IF( IERR.GT.0 )
     $              INFO = IERR
*
                 IF( IJOB.EQ.0 ) THEN
                    CALL DGESC2( ZDIM, Z, LDZ, RHS, IPIV, JPIV,
     $                           SCALOC )
                    IF( SCALOC.NE.ONE ) THEN
                       DO 60 K = 1, N
                          CALL DSCAL( M, SCALOC, C( 1, K ), 1 )
                          CALL DSCAL( M, SCALOC, F( 1, K ), 1 )
   60                  CONTINUE
                       SCALE = SCALE*SCALOC
                    END IF
                 ELSE
                    CALL DLATDF( IJOB, ZDIM, Z, LDZ, RHS, RDSUM,
     $                           RDSCAL, IPIV, JPIV )
                 END IF
*
*                Unpack solution vector(s)
*
                 C( IS, JS ) = RHS( 1 )
                 C( IS, JSP1 ) = RHS( 2 )
                 F( IS, JS ) = RHS( 3 )
                 F( IS, JSP1 ) = RHS( 4 )
*
*                Substitute R(I, J) and L(I, J) into remaining
*                equation.
*
                 IF( I.GT.1 ) THEN
```

```
                 CALL DGER( IS-1, NB, -ONE, A( 1, IS ), 1, RHS( 1 ),
     $                     1, C( 1, JS ), LDC )
                 CALL DGER( IS-1, NB, -ONE, D( 1, IS ), 1, RHS( 1 ),
     $                     1, F( 1, JS ), LDF )
              END IF
              IF( J.LT.Q ) THEN
                 CALL DAXPY( N-JE, RHS( 3 ), B( JS, JE+1 ), LDB,
     $                       C( IS, JE+1 ), LDC )
                 CALL DAXPY( N-JE, RHS( 3 ), E( JS, JE+1 ), LDE,
     $                       F( IS, JE+1 ), LDF )
                 CALL DAXPY( N-JE, RHS( 4 ), B( JSP1, JE+1 ), LDB,
     $                       C( IS, JE+1 ), LDC )
                 CALL DAXPY( N-JE, RHS( 4 ), E( JSP1, JE+1 ), LDE,
     $                       F( IS, JE+1 ), LDF )
              END IF
*
           ELSE IF( ( MB.EQ.2 ) .AND. ( NB.EQ.1 ) ) THEN
*
*             Build a 4-by-4 system Z * x = RHS
*
              Z( 1, 1 ) = A( IS, IS )
              Z( 2, 1 ) = A( ISP1, IS )
              Z( 3, 1 ) = D( IS, IS )
              Z( 4, 1 ) = ZERO
*
              Z( 1, 2 ) = A( IS, ISP1 )
              Z( 2, 2 ) = A( ISP1, ISP1 )
              Z( 3, 2 ) = D( IS, ISP1 )
              Z( 4, 2 ) = D( ISP1, ISP1 )
*
              Z( 1, 3 ) = -B( JS, JS )
              Z( 2, 3 ) = ZERO
              Z( 3, 3 ) = -E( JS, JS )
              Z( 4, 3 ) = ZERO
*
              Z( 1, 4 ) = ZERO
              Z( 2, 4 ) = -B( JS, JS )
              Z( 3, 4 ) = ZERO
              Z( 4, 4 ) = -E( JS, JS )
*
*             Set up right hand side(s)
*
              RHS( 1 ) = C( IS, JS )
              RHS( 2 ) = C( ISP1, JS )
              RHS( 3 ) = F( IS, JS )
              RHS( 4 ) = F( ISP1, JS )
*
*             Solve Z * x = RHS
*
              CALL DGETC2( ZDIM, Z, LDZ, IPIV, JPIV, IERR )
```

```
                 IF( IERR.GT.0 )
     $              INFO = IERR
                 IF( IJOB.EQ.0 ) THEN
                    CALL DGESC2( ZDIM, Z, LDZ, RHS, IPIV, JPIV,
     $                           SCALOC )
                    IF( SCALOC.NE.ONE ) THEN
                       DO 70 K = 1, N
                          CALL DSCAL( M, SCALOC, C( 1, K ), 1 )
                          CALL DSCAL( M, SCALOC, F( 1, K ), 1 )
   70                  CONTINUE
                       SCALE = SCALE*SCALOC
                    END IF
                 ELSE
                    CALL DLATDF( IJOB, ZDIM, Z, LDZ, RHS, RDSUM,
     $                           RDSCAL, IPIV, JPIV )
                 END IF
*
*                Unpack solution vector(s)
*
                 C( IS, JS ) = RHS( 1 )
                 C( ISP1, JS ) = RHS( 2 )
                 F( IS, JS ) = RHS( 3 )
                 F( ISP1, JS ) = RHS( 4 )
*
*                Substitute R(I, J) and L(I, J) into remaining
*                equation.
*
                 IF( I.GT.1 ) THEN
                    CALL DGEMV( 'N', IS-1, MB, -ONE, A( 1, IS ), LDA,
     $                          RHS( 1 ), 1, ONE, C( 1, JS ), 1 )
                    CALL DGEMV( 'N', IS-1, MB, -ONE, D( 1, IS ), LDD,
     $                          RHS( 1 ), 1, ONE, F( 1, JS ), 1 )
                 END IF
                 IF( J.LT.Q ) THEN
                    CALL DGER( MB, N-JE, ONE, RHS( 3 ), 1,
     $                         B( JS, JE+1 ), LDB, C( IS, JE+1 ), LDC )
                    CALL DGER( MB, N-JE, ONE, RHS( 3 ), 1,
     $                         E( JS, JE+1 ), LDE, F( IS, JE+1 ), LDF )
                 END IF
*
              ELSE IF( ( MB.EQ.2 ) .AND. ( NB.EQ.2 ) ) THEN
*
*                Build an 8-by-8 system Z * x = RHS
*
                 CALL DLASET( 'F', LDZ, LDZ, ZERO, ZERO, Z, LDZ )
*
                 Z( 1, 1 ) = A( IS, IS )
                 Z( 2, 1 ) = A( ISP1, IS )
                 Z( 5, 1 ) = D( IS, IS )
*
```

```
                 Z( 1, 2 ) = A( IS, ISP1 )
                 Z( 2, 2 ) = A( ISP1, ISP1 )
                 Z( 5, 2 ) = D( IS, ISP1 )
                 Z( 6, 2 ) = D( ISP1, ISP1 )
*
                 Z( 3, 3 ) = A( IS, IS )
                 Z( 4, 3 ) = A( ISP1, IS )
                 Z( 7, 3 ) = D( IS, IS )
*
                 Z( 3, 4 ) = A( IS, ISP1 )
                 Z( 4, 4 ) = A( ISP1, ISP1 )
                 Z( 7, 4 ) = D( IS, ISP1 )
                 Z( 8, 4 ) = D( ISP1, ISP1 )
*
                 Z( 1, 5 ) = -B( JS, JS )
                 Z( 3, 5 ) = -B( JS, JSP1 )
                 Z( 5, 5 ) = -E( JS, JS )
                 Z( 7, 5 ) = -E( JS, JSP1 )
*
                 Z( 2, 6 ) = -B( JS, JS )
                 Z( 4, 6 ) = -B( JS, JSP1 )
                 Z( 6, 6 ) = -E( JS, JS )
                 Z( 8, 6 ) = -E( JS, JSP1 )
*
                 Z( 1, 7 ) = -B( JSP1, JS )
                 Z( 3, 7 ) = -B( JSP1, JSP1 )
                 Z( 7, 7 ) = -E( JSP1, JSP1 )
*
                 Z( 2, 8 ) = -B( JSP1, JS )
                 Z( 4, 8 ) = -B( JSP1, JSP1 )
                 Z( 8, 8 ) = -E( JSP1, JSP1 )
*
*                Set up right hand side(s)
*
                 K = 1
                 II = MB*NB + 1
                 DO 80 JJ = 0, NB - 1
                    CALL DCOPY( MB, C( IS, JS+JJ ), 1, RHS( K ), 1 )
                    CALL DCOPY( MB, F( IS, JS+JJ ), 1, RHS( II ), 1 )
                    K = K + MB
                    II = II + MB
   80            CONTINUE
*
*                Solve Z * x = RHS
*
                 CALL DGETC2( ZDIM, Z, LDZ, IPIV, JPIV, IERR )
                 IF( IERR.GT.0 )
     $              INFO = IERR
                 IF( IJOB.EQ.0 ) THEN
                    CALL DGESC2( ZDIM, Z, LDZ, RHS, IPIV, JPIV,
```

```
     $                            SCALOC )
                     IF( SCALOC.NE.ONE ) THEN
                        DO 90 K = 1, N
                           CALL DSCAL( M, SCALOC, C( 1, K ), 1 )
                           CALL DSCAL( M, SCALOC, F( 1, K ), 1 )
   90                   CONTINUE
                        SCALE = SCALE*SCALOC
                     END IF
                  ELSE
                     CALL DLATDF( IJOB, ZDIM, Z, LDZ, RHS, RDSUM,
     $                            RDSCAL, IPIV, JPIV )
                  END IF
*
*                 Unpack solution vector(s)
*
                  K = 1
                  II = MB*NB + 1
                  DO 100 JJ = 0, NB - 1
                     CALL DCOPY( MB, RHS( K ), 1, C( IS, JS+JJ ), 1 )
                     CALL DCOPY( MB, RHS( II ), 1, F( IS, JS+JJ ), 1 )
                     K = K + MB
                     II = II + MB
  100             CONTINUE
*
*                 Substitute R(I, J) and L(I, J) into remaining
*                 equation.
*
                  IF( I.GT.1 ) THEN
                     CALL DGEMM( 'N', 'N', IS-1, NB, MB, -ONE,
     $                           A( 1, IS ), LDA, RHS( 1 ), MB, ONE,
     $                           C( 1, JS ), LDC )
                     CALL DGEMM( 'N', 'N', IS-1, NB, MB, -ONE,
     $                           D( 1, IS ), LDD, RHS( 1 ), MB, ONE,
     $                           F( 1, JS ), LDF )
                  END IF
                  IF( J.LT.Q ) THEN
                     K = MB*NB + 1
                     CALL DGEMM( 'N', 'N', MB, N-JE, NB, ONE, RHS( K ),
     $                           MB, B( JS, JE+1 ), LDB, ONE,
     $                           C( IS, JE+1 ), LDC )
                     CALL DGEMM( 'N', 'N', MB, N-JE, NB, ONE, RHS( K ),
     $                           MB, E( JS, JE+1 ), LDE, ONE,
     $                           F( IS, JE+1 ), LDF )
                  END IF
*
               END IF
*
  110       CONTINUE
  120    CONTINUE
      ELSE
```

```
*
*       Solve (I, J) - subsystem
*            A(I, I)**T * R(I, J) + D(I, I)**T * L(J, J)  =  C(I, J)
*            R(I, I)  * B(J, J) + L(I, J)  * E(J, J)  = -F(I, J)
*       for I = 1, 2, ..., P, J = Q, Q - 1, ..., 1
*
        SCALE = ONE
        SCALOC = ONE
        DO 200 I = 1, P
*
           IS = IWORK( I )
           ISP1 = IS + 1
           IE = IWORK ( I+1 ) - 1
           MB = IE - IS + 1
           DO 190 J = Q, P + 2, -1
*
              JS = IWORK( J )
              JSP1 = JS + 1
              JE = IWORK( J+1 ) - 1
              NB = JE - JS + 1
              ZDIM = MB*NB*2
              IF( ( MB.EQ.1 ) .AND. ( NB.EQ.1 ) ) THEN
*
*                Build a 2-by-2 system Z**T * x = RHS
*
                 Z( 1, 1 ) = A( IS, IS )
                 Z( 2, 1 ) = -B( JS, JS )
                 Z( 1, 2 ) = D( IS, IS )
                 Z( 2, 2 ) = -E( JS, JS )
*
*                Set up right hand side(s)
*
                 RHS( 1 ) = C( IS, JS )
                 RHS( 2 ) = F( IS, JS )
*
*                Solve Z**T * x = RHS
*
                 CALL DGETC2( ZDIM, Z, LDZ, IPIV, JPIV, IERR )
                 IF( IERR.GT.0 )
     $              INFO = IERR
*
                 CALL DGESC2( ZDIM, Z, LDZ, RHS, IPIV, JPIV, SCALOC )
                 IF( SCALOC.NE.ONE ) THEN
                    DO 130 K = 1, N
                       CALL DSCAL( M, SCALOC, C( 1, K ), 1 )
                       CALL DSCAL( M, SCALOC, F( 1, K ), 1 )
  130               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
*
```

```
*                 Unpack solution vector(s)
*
                  C( IS, JS ) = RHS( 1 )
                  F( IS, JS ) = RHS( 2 )
*
*                 Substitute R(I, J) and L(I, J) into remaining
*                 equation.
*
                  IF( J.GT.P+2 ) THEN
                     ALPHA = RHS( 1 )
                     CALL DAXPY( JS-1, ALPHA, B( 1, JS ), 1, F( IS, 1 ),
     $                           LDF )
                     ALPHA = RHS( 2 )
                     CALL DAXPY( JS-1, ALPHA, E( 1, JS ), 1, F( IS, 1 ),
     $                           LDF )
                  END IF
                  IF( I.LT.P ) THEN
                     ALPHA = -RHS( 1 )
                     CALL DAXPY( M-IE, ALPHA, A( IS, IE+1 ), LDA,
     $                           C( IE+1, JS ), 1 )
                     ALPHA = -RHS( 2 )
                     CALL DAXPY( M-IE, ALPHA, D( IS, IE+1 ), LDD,
     $                           C( IE+1, JS ), 1 )
                  END IF
*
               ELSE IF( ( MB.EQ.1 ) .AND. ( NB.EQ.2 ) ) THEN
*
*                 Build a 4-by-4 system Z**T * x = RHS
*
                  Z( 1, 1 ) = A( IS, IS )
                  Z( 2, 1 ) = ZERO
                  Z( 3, 1 ) = -B( JS, JS )
                  Z( 4, 1 ) = -B( JSP1, JS )
*
                  Z( 1, 2 ) = ZERO
                  Z( 2, 2 ) = A( IS, IS )
                  Z( 3, 2 ) = -B( JS, JSP1 )
                  Z( 4, 2 ) = -B( JSP1, JSP1 )
*
                  Z( 1, 3 ) = D( IS, IS )
                  Z( 2, 3 ) = ZERO
                  Z( 3, 3 ) = -E( JS, JS )
                  Z( 4, 3 ) = ZERO
*
                  Z( 1, 4 ) = ZERO
                  Z( 2, 4 ) = D( IS, IS )
                  Z( 3, 4 ) = -E( JS, JSP1 )
                  Z( 4, 4 ) = -E( JSP1, JSP1 )
*
*                 Set up right hand side(s)
```

```
*
                 RHS( 1 ) = C( IS, JS )
                 RHS( 2 ) = C( IS, JSP1 )
                 RHS( 3 ) = F( IS, JS )
                 RHS( 4 ) = F( IS, JSP1 )
*
*                Solve Z**T * x = RHS
*
                 CALL DGETC2( ZDIM, Z, LDZ, IPIV, JPIV, IERR )
                 IF( IERR.GT.0 )
     $              INFO = IERR
                 CALL DGESC2( ZDIM, Z, LDZ, RHS, IPIV, JPIV, SCALOC )
                 IF( SCALOC.NE.ONE ) THEN
                    DO 140 K = 1, N
                       CALL DSCAL( M, SCALOC, C( 1, K ), 1 )
                       CALL DSCAL( M, SCALOC, F( 1, K ), 1 )
  140               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
*
*                Unpack solution vector(s)
*
                 C( IS, JS ) = RHS( 1 )
                 C( IS, JSP1 ) = RHS( 2 )
                 F( IS, JS ) = RHS( 3 )
                 F( IS, JSP1 ) = RHS( 4 )
*
*                Substitute R(I, J) and L(I, J) into remaining
*                equation.
*
                 IF( J.GT.P+2 ) THEN
                    CALL DAXPY( JS-1, RHS( 1 ), B( 1, JS ), 1,
     $                          F( IS, 1 ), LDF )
                    CALL DAXPY( JS-1, RHS( 2 ), B( 1, JSP1 ), 1,
     $                          F( IS, 1 ), LDF )
                    CALL DAXPY( JS-1, RHS( 3 ), E( 1, JS ), 1,
     $                          F( IS, 1 ), LDF )
                    CALL DAXPY( JS-1, RHS( 4 ), E( 1, JSP1 ), 1,
     $                          F( IS, 1 ), LDF )
                 END IF
                 IF( I.LT.P ) THEN
                    CALL DGER( M-IE, NB, -ONE, A( IS, IE+1 ), LDA,
     $                         RHS( 1 ), 1, C( IE+1, JS ), LDC )
                    CALL DGER( M-IE, NB, -ONE, D( IS, IE+1 ), LDD,
     $                         RHS( 3 ), 1, C( IE+1, JS ), LDC )
                 END IF
*
              ELSE IF( ( MB.EQ.2 ) .AND. ( NB.EQ.1 ) ) THEN
*
*                Build a 4-by-4 system Z**T * x = RHS
```

```
*
                 Z( 1, 1 ) = A( IS, IS )
                 Z( 2, 1 ) = A( IS, ISP1 )
                 Z( 3, 1 ) = -B( JS, JS )
                 Z( 4, 1 ) = ZERO
*
                 Z( 1, 2 ) = A( ISP1, IS )
                 Z( 2, 2 ) = A( ISP1, ISP1 )
                 Z( 3, 2 ) = ZERO
                 Z( 4, 2 ) = -B( JS, JS )
*
                 Z( 1, 3 ) = D( IS, IS )
                 Z( 2, 3 ) = D( IS, ISP1 )
                 Z( 3, 3 ) = -E( JS, JS )
                 Z( 4, 3 ) = ZERO
*
                 Z( 1, 4 ) = ZERO
                 Z( 2, 4 ) = D( ISP1, ISP1 )
                 Z( 3, 4 ) = ZERO
                 Z( 4, 4 ) = -E( JS, JS )
*
*                Set up right hand side(s)
*
                 RHS( 1 ) = C( IS, JS )
                 RHS( 2 ) = C( ISP1, JS )
                 RHS( 3 ) = F( IS, JS )
                 RHS( 4 ) = F( ISP1, JS )
*
*                Solve Z**T * x = RHS
*
                 CALL DGETC2( ZDIM, Z, LDZ, IPIV, JPIV, IERR )
                 IF( IERR.GT.0 )
     $              INFO = IERR
*
                 CALL DGESC2( ZDIM, Z, LDZ, RHS, IPIV, JPIV, SCALOC )
                 IF( SCALOC.NE.ONE ) THEN
                    DO 150 K = 1, N
                       CALL DSCAL( M, SCALOC, C( 1, K ), 1 )
                       CALL DSCAL( M, SCALOC, F( 1, K ), 1 )
  150               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
*
*                Unpack solution vector(s)
*
                 C( IS, JS ) = RHS( 1 )
                 C( ISP1, JS ) = RHS( 2 )
                 F( IS, JS ) = RHS( 3 )
                 F( ISP1, JS ) = RHS( 4 )
*
```

```
*                 Substitute R(I, J) and L(I, J) into remaining
*                 equation.
*
                  IF( J.GT.P+2 ) THEN
                     CALL DGER( MB, JS-1, ONE, RHS( 1 ), 1, B( 1, JS ),
     $                          1, F( IS, 1 ), LDF )
                     CALL DGER( MB, JS-1, ONE, RHS( 3 ), 1, E( 1, JS ),
     $                          1, F( IS, 1 ), LDF )
                  END IF
                  IF( I.LT.P ) THEN
                     CALL DGEMV( 'T', MB, M-IE, -ONE, A( IS, IE+1 ),
     $                           LDA, RHS( 1 ), 1, ONE, C( IE+1, JS ),
     $                           1 )
                     CALL DGEMV( 'T', MB, M-IE, -ONE, D( IS, IE+1 ),
     $                           LDD, RHS( 3 ), 1, ONE, C( IE+1, JS ),
     $                           1 )
                  END IF
*
               ELSE IF( ( MB.EQ.2 ) .AND. ( NB.EQ.2 ) ) THEN
*
*                 Build an 8-by-8 system Z**T * x = RHS
*
                  CALL DLASET( 'F', LDZ, LDZ, ZERO, ZERO, Z, LDZ )
*
                  Z( 1, 1 ) = A( IS, IS )
                  Z( 2, 1 ) = A( IS, ISP1 )
                  Z( 5, 1 ) = -B( JS, JS )
                  Z( 7, 1 ) = -B( JSP1, JS )
*
                  Z( 1, 2 ) = A( ISP1, IS )
                  Z( 2, 2 ) = A( ISP1, ISP1 )
                  Z( 6, 2 ) = -B( JS, JS )
                  Z( 8, 2 ) = -B( JSP1, JS )
*
                  Z( 3, 3 ) = A( IS, IS )
                  Z( 4, 3 ) = A( IS, ISP1 )
                  Z( 5, 3 ) = -B( JS, JSP1 )
                  Z( 7, 3 ) = -B( JSP1, JSP1 )
*
                  Z( 3, 4 ) = A( ISP1, IS )
                  Z( 4, 4 ) = A( ISP1, ISP1 )
                  Z( 6, 4 ) = -B( JS, JSP1 )
                  Z( 8, 4 ) = -B( JSP1, JSP1 )
*
                  Z( 1, 5 ) = D( IS, IS )
                  Z( 2, 5 ) = D( IS, ISP1 )
                  Z( 5, 5 ) = -E( JS, JS )
*
                  Z( 2, 6 ) = D( ISP1, ISP1 )
                  Z( 6, 6 ) = -E( JS, JS )
```

```
*
                 Z( 3, 7 ) = D( IS, IS )
                 Z( 4, 7 ) = D( IS, ISP1 )
                 Z( 5, 7 ) = -E( JS, JSP1 )
                 Z( 7, 7 ) = -E( JSP1, JSP1 )
*
                 Z( 4, 8 ) = D( ISP1, ISP1 )
                 Z( 6, 8 ) = -E( JS, JSP1 )
                 Z( 8, 8 ) = -E( JSP1, JSP1 )
*
*                Set up right hand side(s)
*
                 K = 1
                 II = MB*NB + 1
                 DO 160 JJ = 0, NB - 1
                    CALL DCOPY( MB, C( IS, JS+JJ ), 1, RHS( K ), 1 )
                    CALL DCOPY( MB, F( IS, JS+JJ ), 1, RHS( II ), 1 )
                    K = K + MB
                    II = II + MB
  160            CONTINUE
*
*
*                Solve Z**T * x = RHS
*
                 CALL DGETC2( ZDIM, Z, LDZ, IPIV, JPIV, IERR )
                 IF( IERR.GT.0 )
     $              INFO = IERR
*
                 CALL DGESC2( ZDIM, Z, LDZ, RHS, IPIV, JPIV, SCALOC )
                 IF( SCALOC.NE.ONE ) THEN
                    DO 170 K = 1, N
                       CALL DSCAL( M, SCALOC, C( 1, K ), 1 )
                       CALL DSCAL( M, SCALOC, F( 1, K ), 1 )
  170               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
*
*                Unpack solution vector(s)
*
                 K = 1
                 II = MB*NB + 1
                 DO 180 JJ = 0, NB - 1
                    CALL DCOPY( MB, RHS( K ), 1, C( IS, JS+JJ ), 1 )
                    CALL DCOPY( MB, RHS( II ), 1, F( IS, JS+JJ ), 1 )
                    K = K + MB
                    II = II + MB
  180            CONTINUE
*
*                Substitute R(I, J) and L(I, J) into remaining
*                equation.
```

```
*
                  IF( J.GT.P+2 ) THEN
                     CALL DGEMM( 'N', 'T', MB, JS-1, NB, ONE,
     $                           C( IS, JS ), LDC, B( 1, JS ), LDB, ONE,
     $                           F( IS, 1 ), LDF )
                     CALL DGEMM( 'N', 'T', MB, JS-1, NB, ONE,
     $                           F( IS, JS ), LDF, E( 1, JS ), LDE, ONE,
     $                           F( IS, 1 ), LDF )
                  END IF
                  IF( I.LT.P ) THEN
                     CALL DGEMM( 'T', 'N', M-IE, NB, MB, -ONE,
     $                           A( IS, IE+1 ), LDA, C( IS, JS ), LDC,
     $                           ONE, C( IE+1, JS ), LDC )
                     CALL DGEMM( 'T', 'N', M-IE, NB, MB, -ONE,
     $                           D( IS, IE+1 ), LDD, F( IS, JS ), LDF,
     $                           ONE, C( IE+1, JS ), LDC )
                  END IF
*
               END IF
*
  190       CONTINUE
  200    CONTINUE
*
      END IF
      RETURN
*
*     End of DTGSY2
*
      END
```

———-

62.5.4 lisp code

— **dtgsy2.lisp** —

```
(let* ((ldz 8) (zero 0.0d0) (one 1.0d0))
  (declare (type (f2cl-lib:integer4 8 8) ldz)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable ldz zero one))
  (defun dtgsy2
         (trans ijob m n a lda b ldb$ c ldc d ldd e lde f ldf scale rdsum
          rdscal iwork pq info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (double-float) rdscal rdsum scale)
             (type (array double-float (*)) f e d c b a)
             (type (f2cl-lib:integer4) info pq ldf lde ldd ldc ldb$ lda n m
```

```
          ijob)
         (type (string *) trans))
(f2cl-lib:with-multi-array-data
    ((trans character trans-%data% trans-%offset%)
     (a double-float a-%data% a-%offset%)
     (b double-float b-%data% b-%offset%)
     (c double-float c-%data% c-%offset%)
     (d double-float d-%data% d-%offset%)
     (e double-float e-%data% e-%offset%)
     (f double-float f-%data% f-%offset%)
     (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((rhs (make-array ldz :element-type 'double-float))
         (z
          (make-array (the fixnum (reduce #'* (list ldz ldz)))
                      :element-type 'double-float))
         (ipiv (make-array ldz :element-type 'f2cl-lib:integer4))
         (jpiv (make-array ldz :element-type 'f2cl-lib:integer4))
         (alpha 0.0d0) (scaloc 0.0d0) (i 0) (ie 0) (ierr 0) (ii 0) (is 0)
         (isp1 0) (j 0) (je 0) (jj 0) (js 0) (jsp1 0) (k 0) (mb 0) (nb 0)
         (p 0) (q 0) (zdim 0) (notran nil))
    (declare (type (array double-float (*)) rhs z)
             (type (array f2cl-lib:integer4 (*)) ipiv jpiv)
             (type (double-float) alpha scaloc)
             (type (f2cl-lib:integer4) i ie ierr ii is isp1 j je jj js jsp1
              k mb nb p q zdim)
             (type f2cl-lib:logical notran))
    (setf info 0)
    (setf ierr 0)
    (setf notran
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "N")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val))
    (cond
     ((and (not notran)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "T")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val)))
      (setf info -1))
     (notran (cond ((or (< ijob 0) (> ijob 2)) (setf info -2)))))
    (cond
     ((= info 0)
      (cond ((<= m 0) (setf info -3)) ((<= n 0) (setf info -4))
            ((< lda
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -6))
```

```
          ((< ldb$
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
           (setf info -8))
          ((< ldc
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
           (setf info -10))
          ((< ldd
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
           (setf info -12))
          ((< lde
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
           (setf info -14))
          ((< ldf
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
           (setf info -16)))))
 (cond
  ((/= info 0) (xerbla "DTGSY2" (f2cl-lib:int-sub info))
   (go end_label)))
 (setf pq 0)
 (setf p 0)
 (setf i 1)
label10
 (if (> i m)
     (go label20))
 (setf p (f2cl-lib:int-add p 1))
 (setf (f2cl-lib:fref iwork-%data% (p) ((1 *)) iwork-%offset%) i)
 (if (= i m)
     (go label20))
 (cond
  ((/= (f2cl-lib:fref a ((f2cl-lib:int-add i 1) i) ((1 lda) (1 *)))
       zero)
   (setf i (f2cl-lib:int-add i 2)))
  (t (setf i (f2cl-lib:int-add i 1))))
 (go label10)
label20
 (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add p 1)) ((1 *))
                      iwork-%offset%)
         (f2cl-lib:int-add m 1))
 (setf q (f2cl-lib:int-add p 1))
 (setf j 1)
label30
 (if (> j n)
     (go label40))
 (setf q (f2cl-lib:int-add q 1))
 (setf (f2cl-lib:fref iwork-%data% (q) ((1 *)) iwork-%offset%) j)
 (if (= j n)
     (go label40))
 (cond
  ((/= (f2cl-lib:fref b ((f2cl-lib:int-add j 1) j) ((1 ldb$) (1 *)))
       zero)
```

```
    (setf j (f2cl-lib:int-add j 2)))
   (t (setf j (f2cl-lib:int-add j 1))))
  (go label30)
 label40
  (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add q 1)) ((1 *))
                       iwork-%offset%)
        (f2cl-lib:int-add n 1))
  (setf pq (f2cl-lib:int-mul p (f2cl-lib:int-sub q p 1)))
  (cond
   (notran (setf scale one) (setf scaloc one)
    (f2cl-lib:fdo (j (f2cl-lib:int-add p 2) (f2cl-lib:int-add j 1))
                  ((> j q) nil)
      (tagbody
        (setf js (f2cl-lib:fref iwork-%data% (j) ((1 *)) iwork-%offset%))
        (setf jsp1 (f2cl-lib:int-add js 1))
        (setf je
                (f2cl-lib:int-sub
                 (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add j 1))
                                ((1 *)) iwork-%offset%)
                 1))
        (setf nb (f2cl-lib:int-add (f2cl-lib:int-sub je js) 1))
        (f2cl-lib:fdo (i p (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (setf is
                    (f2cl-lib:fref iwork-%data% (i) ((1 *))
                                   iwork-%offset%))
            (setf isp1 (f2cl-lib:int-add is 1))
            (setf ie
                    (f2cl-lib:int-sub
                     (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add i 1))
                                    ((1 *)) iwork-%offset%)
                     1))
            (setf mb (f2cl-lib:int-add (f2cl-lib:int-sub ie is) 1))
            (setf zdim (f2cl-lib:int-mul mb nb 2))
            (cond
             ((and (= mb 1) (= nb 1))
              (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
                      (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz)))
                      (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                                     d-%offset%))
              (setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz)))
                      (-
                       (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                                      b-%offset%)))
              (setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
                      (-
                       (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
```

```
                        e-%offset%)))
(setf (f2cl-lib:fref rhs (1) ((1 ldz)))
        (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                       c-%offset%))
(setf (f2cl-lib:fref rhs (2) ((1 ldz)))
        (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                       f-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (dgetc2 zdim z ldz ipiv jpiv ierr)
  (declare (ignore var-1 var-3 var-4))
  (when var-0 (setf zdim var-0))
  (when var-2 (setf ldz var-2))
  (when var-5 (setf ierr var-5)))
(if (> ierr 0)
    (setf info ierr))
(cond
 ((= ijob 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dgesc2 zdim z ldz rhs ipiv jpiv scaloc)
    (declare (ignore var-1 var-3 var-4 var-5))
    (when var-0 (setf zdim var-0))
    (when var-2 (setf ldz var-2))
    (when var-6 (setf scaloc var-6)))
  (cond
   ((/= scaloc one)
    (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                  ((> k n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal m scaloc
             (f2cl-lib:array-slice c-%data% double-float
                                   (1 k) ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal m scaloc
             (f2cl-lib:array-slice f-%data% double-float
                                   (1 k) ((1 ldf) (1 *))
                                   f-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
       label50))
    (setf scale (* scale scaloc)))))
 (t
```

```
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8)
      (dlatdf ijob zdim z ldz rhs rdsum rdscal ipiv jpiv)
    (declare (ignore var-2 var-4 var-7 var-8))
    (when var-0 (setf ijob var-0))
    (when var-1 (setf zdim var-1))
    (when var-3 (setf ldz var-3))
    (when var-5 (setf rdsum var-5))
    (when var-6 (setf rdscal var-6)))))
(setf (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (1) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (2) ((1 ldz))))
(cond
 ((> i 1)
  (setf alpha (- (f2cl-lib:fref rhs (1) ((1 ldz)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5)
      (daxpy (f2cl-lib:int-sub is 1) alpha
       (f2cl-lib:array-slice a-%data% double-float (1 is)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice c-%data% double-float (1 js)
                             ((1 ldc) (1 *)) c-%offset%)
       1)
    (declare (ignore var-0 var-2 var-3 var-4 var-5))
    (when var-1 (setf alpha var-1)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5)
      (daxpy (f2cl-lib:int-sub is 1) alpha
       (f2cl-lib:array-slice d-%data% double-float (1 is)
                             ((1 ldd) (1 *)) d-%offset%)
       1
       (f2cl-lib:array-slice f-%data% double-float (1 js)
                             ((1 ldf) (1 *)) f-%offset%)
       1)
    (declare (ignore var-0 var-2 var-3 var-4 var-5))
    (when var-1 (setf alpha var-1)))))
(cond
 ((< j q)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5)
      (daxpy (f2cl-lib:int-sub n je)
       (f2cl-lib:array-slice rhs double-float (2)
                             ((1 ldz)))
       (f2cl-lib:array-slice b-%data% double-float
                             (js (f2cl-lib:int-add je 1))
```

```
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$
          (f2cl-lib:array-slice c-%data% double-float
                                (is (f2cl-lib:int-add je 1))
                                ((1 ldc) (1 *)) c-%offset%)
          ldc)
      (declare (ignore var-0 var-1 var-2 var-4))
      (when var-3 (setf ldb$ var-3))
      (when var-5 (setf ldc var-5)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5)
        (daxpy (f2cl-lib:int-sub n je)
         (f2cl-lib:array-slice rhs double-float (2)
                                ((1 ldz)))
         (f2cl-lib:array-slice e-%data% double-float
                                (js (f2cl-lib:int-add je 1))
                                ((1 lde) (1 *)) e-%offset%)
         lde
         (f2cl-lib:array-slice f-%data% double-float
                                (is (f2cl-lib:int-add je 1))
                                ((1 ldf) (1 *)) f-%offset%)
         ldf)
      (declare (ignore var-0 var-1 var-2 var-4))
      (when var-3 (setf lde var-3))
      (when var-5 (setf ldf var-5))))))
((and (= mb 1) (= nb 2))
 (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                        a-%offset%))
 (setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz))) zero)
 (setf (f2cl-lib:fref z (3 1) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                        d-%offset%))
 (setf (f2cl-lib:fref z (4 1) ((1 ldz) (1 ldz))) zero)
 (setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz))) zero)
 (setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                        a-%offset%))
 (setf (f2cl-lib:fref z (3 2) ((1 ldz) (1 ldz))) zero)
 (setf (f2cl-lib:fref z (4 2) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                        d-%offset%))
 (setf (f2cl-lib:fref z (1 3) ((1 ldz) (1 ldz)))
         (-
          (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                         b-%offset%)))
 (setf (f2cl-lib:fref z (2 3) ((1 ldz) (1 ldz)))
         (-
          (f2cl-lib:fref b-%data% (js jsp1) ((1 ldb$) (1 *))
                         b-%offset%)))
```

```
(setf (f2cl-lib:fref z (3 3) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (4 3) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js jsp1) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (1 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (jsp1 js) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (2 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (jsp1 jsp1)
                        ((1 ldb$) (1 *)) b-%offset%)))
(setf (f2cl-lib:fref z (3 4) ((1 ldz) (1 ldz))) zero)
(setf (f2cl-lib:fref z (4 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (jsp1 jsp1)
                        ((1 lde) (1 *)) e-%offset%)))
(setf (f2cl-lib:fref rhs (1) ((1 ldz)))
        (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                       c-%offset%))
(setf (f2cl-lib:fref rhs (2) ((1 ldz)))
        (f2cl-lib:fref c-%data% (is jsp1) ((1 ldc) (1 *))
                       c-%offset%))
(setf (f2cl-lib:fref rhs (3) ((1 ldz)))
        (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                       f-%offset%))
(setf (f2cl-lib:fref rhs (4) ((1 ldz)))
        (f2cl-lib:fref f-%data% (is jsp1) ((1 ldf) (1 *))
                       f-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (dgetc2 zdim z ldz ipiv jpiv ierr)
  (declare (ignore var-1 var-3 var-4))
  (when var-0 (setf zdim var-0))
  (when var-2 (setf ldz var-2))
  (when var-5 (setf ierr var-5)))
(if (> ierr 0)
    (setf info ierr))
(cond
 ((= ijob 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dgesc2 zdim z ldz rhs ipiv jpiv scaloc)
    (declare (ignore var-1 var-3 var-4 var-5))
    (when var-0 (setf zdim var-0))
    (when var-2 (setf ldz var-2))
    (when var-6 (setf scaloc var-6)))
```

```
  (cond
   ((/= scaloc one)
    (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                  ((> k n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal m scaloc
             (f2cl-lib:array-slice c-%data% double-float
                                   (1 k) ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal m scaloc
             (f2cl-lib:array-slice f-%data% double-float
                                   (1 k) ((1 ldf) (1 *))
                                   f-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
       label60))
    (setf scale (* scale scaloc)))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8)
      (dlatdf ijob zdim z ldz rhs rdsum rdscal ipiv jpiv)
    (declare (ignore var-2 var-4 var-7 var-8))
    (when var-0 (setf ijob var-0))
    (when var-1 (setf zdim var-1))
    (when var-3 (setf ldz var-3))
    (when var-5 (setf rdsum var-5))
    (when var-6 (setf rdscal var-6)))))
(setf (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (1) ((1 ldz))))
(setf (f2cl-lib:fref c-%data% (is jsp1) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (2) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (3) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (is jsp1) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (4) ((1 ldz))))
(cond
 ((> i 1)
```

```
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8)
      (dger (f2cl-lib:int-sub is 1) nb (- one)
       (f2cl-lib:array-slice a-%data% double-float (1 is)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice rhs double-float (1)
                             ((1 ldz)))
       1
       (f2cl-lib:array-slice c-%data% double-float (1 js)
                             ((1 ldc) (1 *)) c-%offset%)
       ldc)
    (declare
     (ignore var-0 var-2 var-3 var-4 var-5 var-6 var-7))
    (when var-1 (setf nb var-1))
    (when var-8 (setf ldc var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8)
      (dger (f2cl-lib:int-sub is 1) nb (- one)
       (f2cl-lib:array-slice d-%data% double-float (1 is)
                             ((1 ldd) (1 *)) d-%offset%)
       1
       (f2cl-lib:array-slice rhs double-float (1)
                             ((1 ldz)))
       1
       (f2cl-lib:array-slice f-%data% double-float (1 js)
                             ((1 ldf) (1 *)) f-%offset%)
       ldf)
    (declare
     (ignore var-0 var-2 var-3 var-4 var-5 var-6 var-7))
    (when var-1 (setf nb var-1))
    (when var-8 (setf ldf var-8)))))
(cond
 ((< j q)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5)
      (daxpy (f2cl-lib:int-sub n je)
       (f2cl-lib:array-slice rhs double-float (3)
                             ((1 ldz)))
       (f2cl-lib:array-slice b-%data% double-float
                             (js (f2cl-lib:int-add je 1))
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice c-%data% double-float
                             (is (f2cl-lib:int-add je 1))
                             ((1 ldc) (1 *)) c-%offset%)
       ldc)
    (declare (ignore var-0 var-1 var-2 var-4))
```

```
  (when var-3 (setf ldb$ var-3))
  (when var-5 (setf ldc var-5)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5)
    (daxpy (f2cl-lib:int-sub n je)
     (f2cl-lib:array-slice rhs double-float (3)
                           ((1 ldz)))
     (f2cl-lib:array-slice e-%data% double-float
                           (js (f2cl-lib:int-add je 1))
                           ((1 lde) (1 *)) e-%offset%)
     lde
     (f2cl-lib:array-slice f-%data% double-float
                           (is (f2cl-lib:int-add je 1))
                           ((1 ldf) (1 *)) f-%offset%)
     ldf)
  (declare (ignore var-0 var-1 var-2 var-4))
  (when var-3 (setf lde var-3))
  (when var-5 (setf ldf var-5)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5)
    (daxpy (f2cl-lib:int-sub n je)
     (f2cl-lib:array-slice rhs double-float (4)
                           ((1 ldz)))
     (f2cl-lib:array-slice b-%data% double-float
                           (jsp1 (f2cl-lib:int-add je 1))
                           ((1 ldb$) (1 *)) b-%offset%)
     ldb$
     (f2cl-lib:array-slice c-%data% double-float
                           (is (f2cl-lib:int-add je 1))
                           ((1 ldc) (1 *)) c-%offset%)
     ldc)
  (declare (ignore var-0 var-1 var-2 var-4))
  (when var-3 (setf ldb$ var-3))
  (when var-5 (setf ldc var-5)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5)
    (daxpy (f2cl-lib:int-sub n je)
     (f2cl-lib:array-slice rhs double-float (4)
                           ((1 ldz)))
     (f2cl-lib:array-slice e-%data% double-float
                           (jsp1 (f2cl-lib:int-add je 1))
                           ((1 lde) (1 *)) e-%offset%)
     lde
     (f2cl-lib:array-slice f-%data% double-float
                           (is (f2cl-lib:int-add je 1))
                           ((1 ldf) (1 *)) f-%offset%)
     ldf)
  (declare (ignore var-0 var-1 var-2 var-4))
  (when var-3 (setf lde var-3))
  (when var-5 (setf ldf var-5))))))
```

```
((and (= mb 2) (= nb 1))
 (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                       a-%offset%))
 (setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (isp1 is) ((1 lda) (1 *))
                       a-%offset%))
 (setf (f2cl-lib:fref z (3 1) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                       d-%offset%))
 (setf (f2cl-lib:fref z (4 1) ((1 ldz) (1 ldz))) zero)
 (setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (is isp1) ((1 lda) (1 *))
                       a-%offset%))
 (setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (isp1 isp1) ((1 lda) (1 *))
                       a-%offset%))
 (setf (f2cl-lib:fref z (3 2) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is isp1) ((1 ldd) (1 *))
                       d-%offset%))
 (setf (f2cl-lib:fref z (4 2) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (isp1 isp1) ((1 ldd) (1 *))
                       d-%offset%))
 (setf (f2cl-lib:fref z (1 3) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                        b-%offset%)))
 (setf (f2cl-lib:fref z (2 3) ((1 ldz) (1 ldz))) zero)
 (setf (f2cl-lib:fref z (3 3) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                        e-%offset%)))
 (setf (f2cl-lib:fref z (4 3) ((1 ldz) (1 ldz))) zero)
 (setf (f2cl-lib:fref z (1 4) ((1 ldz) (1 ldz))) zero)
 (setf (f2cl-lib:fref z (2 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                        b-%offset%)))
 (setf (f2cl-lib:fref z (3 4) ((1 ldz) (1 ldz))) zero)
 (setf (f2cl-lib:fref z (4 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                        e-%offset%)))
 (setf (f2cl-lib:fref rhs (1) ((1 ldz)))
        (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                       c-%offset%))
 (setf (f2cl-lib:fref rhs (2) ((1 ldz)))
        (f2cl-lib:fref c-%data% (isp1 js) ((1 ldc) (1 *))
                       c-%offset%))
 (setf (f2cl-lib:fref rhs (3) ((1 ldz)))
```

```
        (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                       f-%offset%))
(setf (f2cl-lib:fref rhs (4) ((1 ldz)))
        (f2cl-lib:fref f-%data% (isp1 js) ((1 ldf) (1 *))
                       f-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (dgetc2 zdim z ldz ipiv jpiv ierr)
  (declare (ignore var-1 var-3 var-4))
  (when var-0 (setf zdim var-0))
  (when var-2 (setf ldz var-2))
  (when var-5 (setf ierr var-5)))
(if (> ierr 0)
    (setf info ierr))
(cond
 ((= ijob 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dgesc2 zdim z ldz rhs ipiv jpiv scaloc)
    (declare (ignore var-1 var-3 var-4 var-5))
    (when var-0 (setf zdim var-0))
    (when var-2 (setf ldz var-2))
    (when var-6 (setf scaloc var-6)))
  (cond
   ((/= scaloc one)
    (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                  ((> k n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal m scaloc
             (f2cl-lib:array-slice c-%data% double-float
                                   (1 k) ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal m scaloc
             (f2cl-lib:array-slice f-%data% double-float
                                   (1 k) ((1 ldf) (1 *))
                                   f-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
       label70))
    (setf scale (* scale scaloc)))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
```

```
       var-8)
      (dlatdf ijob zdim z ldz rhs rdsum rdscal ipiv jpiv)
    (declare (ignore var-2 var-4 var-7 var-8))
    (when var-0 (setf ijob var-0))
    (when var-1 (setf zdim var-1))
    (when var-3 (setf ldz var-3))
    (when var-5 (setf rdsum var-5))
    (when var-6 (setf rdscal var-6)))))
(setf (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (1) ((1 ldz))))
(setf (f2cl-lib:fref c-%data% (isp1 js) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (2) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (3) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (isp1 js) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (4) ((1 ldz))))
(cond
 ((> i 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8 var-9 var-10)
      (dgemv "N" (f2cl-lib:int-sub is 1) mb (- one)
       (f2cl-lib:array-slice a-%data% double-float (1 is)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice rhs double-float (1)
                             ((1 ldz)))
       1 one
       (f2cl-lib:array-slice c-%data% double-float (1 js)
                             ((1 ldc) (1 *)) c-%offset%)
       1)
    (declare
     (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9
      var-10))
    (when var-2 (setf mb var-2))
    (when var-5 (setf lda var-5))
    (when var-8 (setf one var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8 var-9 var-10)
      (dgemv "N" (f2cl-lib:int-sub is 1) mb (- one)
       (f2cl-lib:array-slice d-%data% double-float (1 is)
                             ((1 ldd) (1 *)) d-%offset%)
       ldd
       (f2cl-lib:array-slice rhs double-float (1)
                             ((1 ldz)))
```

```
        1 one
        (f2cl-lib:array-slice f-%data% double-float (1 js)
                              ((1 ldf) (1 *)) f-%offset%)
        1)
    (declare
     (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9
      var-10))
    (when var-2 (setf mb var-2))
    (when var-5 (setf ldd var-5))
    (when var-8 (setf one var-8)))))
(cond
 ((< j q)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8)
      (dger mb (f2cl-lib:int-sub n je) one
       (f2cl-lib:array-slice rhs double-float (3)
                             ((1 ldz)))
       1
       (f2cl-lib:array-slice b-%data% double-float
                             (js (f2cl-lib:int-add je 1))
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice c-%data% double-float
                             (is (f2cl-lib:int-add je 1))
                             ((1 ldc) (1 *)) c-%offset%)
       ldc)
    (declare (ignore var-1 var-3 var-4 var-5 var-7))
    (when var-0 (setf mb var-0))
    (when var-2 (setf one var-2))
    (when var-6 (setf ldb$ var-6))
    (when var-8 (setf ldc var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8)
      (dger mb (f2cl-lib:int-sub n je) one
       (f2cl-lib:array-slice rhs double-float (3)
                             ((1 ldz)))
       1
       (f2cl-lib:array-slice e-%data% double-float
                             (js (f2cl-lib:int-add je 1))
                             ((1 lde) (1 *)) e-%offset%)
       lde
       (f2cl-lib:array-slice f-%data% double-float
                             (is (f2cl-lib:int-add je 1))
                             ((1 ldf) (1 *)) f-%offset%)
       ldf)
    (declare (ignore var-1 var-3 var-4 var-5 var-7))
    (when var-0 (setf mb var-0))
    (when var-2 (setf one var-2))
```

```
      (when var-6 (setf lde var-6))
      (when var-8 (setf ldf var-8))))))
  ((and (= mb 2) (= nb 2))
   (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "F" ldz ldz zero zero z ldz)
    (declare (ignore var-0 var-5))
    (when var-1 (setf ldz var-1))
    (when var-2 (setf ldz var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldz var-6)))
   (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                        a-%offset%))
   (setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (isp1 is) ((1 lda) (1 *))
                        a-%offset%))
   (setf (f2cl-lib:fref z (5 1) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                        d-%offset%))
   (setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (is isp1) ((1 lda) (1 *))
                        a-%offset%))
   (setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (isp1 isp1) ((1 lda) (1 *))
                        a-%offset%))
   (setf (f2cl-lib:fref z (5 2) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref d-%data% (is isp1) ((1 ldd) (1 *))
                        d-%offset%))
   (setf (f2cl-lib:fref z (6 2) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref d-%data% (isp1 isp1) ((1 ldd) (1 *))
                        d-%offset%))
   (setf (f2cl-lib:fref z (3 3) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                        a-%offset%))
   (setf (f2cl-lib:fref z (4 3) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (isp1 is) ((1 lda) (1 *))
                        a-%offset%))
   (setf (f2cl-lib:fref z (7 3) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                        d-%offset%))
   (setf (f2cl-lib:fref z (3 4) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (is isp1) ((1 lda) (1 *))
                        a-%offset%))
   (setf (f2cl-lib:fref z (4 4) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (isp1 isp1) ((1 lda) (1 *))
                        a-%offset%))
   (setf (f2cl-lib:fref z (7 4) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref d-%data% (is isp1) ((1 ldd) (1 *))
```

```
                        d-%offset%))
(setf (f2cl-lib:fref z (8 4) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (isp1 isp1) ((1 ldd) (1 *))
                        d-%offset%))
(setf (f2cl-lib:fref z (1 5) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (3 5) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (js jsp1) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (5 5) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (7 5) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js jsp1) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (2 6) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (4 6) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (js jsp1) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (6 6) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (8 6) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js jsp1) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (1 7) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (jsp1 js) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (3 7) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (jsp1 jsp1)
                        ((1 ldb$) (1 *)) b-%offset%)))
(setf (f2cl-lib:fref z (7 7) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (jsp1 jsp1)
                        ((1 lde) (1 *)) e-%offset%)))
(setf (f2cl-lib:fref z (2 8) ((1 ldz) (1 ldz)))
        (-
```

```
         (f2cl-lib:fref b-%data% (jsp1 js) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (4 8) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (jsp1 jsp1)
                        ((1 ldb$) (1 *)) b-%offset%)))
(setf (f2cl-lib:fref z (8 8) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (jsp1 jsp1)
                        ((1 lde) (1 *)) e-%offset%)))
(setf k 1)
(setf ii (f2cl-lib:int-add (f2cl-lib:int-mul mb nb) 1))
(f2cl-lib:fdo (jj 0 (f2cl-lib:int-add jj 1))
              ((> jj
                  (f2cl-lib:int-add nb
                                    (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy mb
         (f2cl-lib:array-slice c-%data% double-float
                               (is
                                (f2cl-lib:int-add js jj))
                               ((1 ldc) (1 *)) c-%offset%)
         1
         (f2cl-lib:array-slice rhs double-float (k)
                               ((1 ldz)))
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf mb var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy mb
         (f2cl-lib:array-slice f-%data% double-float
                               (is
                                (f2cl-lib:int-add js jj))
                               ((1 ldf) (1 *)) f-%offset%)
         1
         (f2cl-lib:array-slice rhs double-float (ii)
                               ((1 ldz)))
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf mb var-0)))
    (setf k (f2cl-lib:int-add k mb))
    (setf ii (f2cl-lib:int-add ii mb))
   label80))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (dgetc2 zdim z ldz ipiv jpiv ierr)
  (declare (ignore var-1 var-3 var-4))
  (when var-0 (setf zdim var-0))
  (when var-2 (setf ldz var-2))
```

```
  (when var-5 (setf ierr var-5)))
(if (> ierr 0)
    (setf info ierr))
(cond
 ((= ijob 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dgesc2 zdim z ldz rhs ipiv jpiv scaloc)
    (declare (ignore var-1 var-3 var-4 var-5))
    (when var-0 (setf zdim var-0))
    (when var-2 (setf ldz var-2))
    (when var-6 (setf scaloc var-6)))
  (cond
   ((/= scaloc one)
    (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                  ((> k n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal m scaloc
             (f2cl-lib:array-slice c-%data% double-float
                                   (1 k) ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal m scaloc
             (f2cl-lib:array-slice f-%data% double-float
                                   (1 k) ((1 ldf) (1 *))
                                   f-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
       label90))
    (setf scale (* scale scaloc)))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8)
      (dlatdf ijob zdim z ldz rhs rdsum rdscal ipiv jpiv)
    (declare (ignore var-2 var-4 var-7 var-8))
    (when var-0 (setf ijob var-0))
    (when var-1 (setf zdim var-1))
    (when var-3 (setf ldz var-3))
    (when var-5 (setf rdsum var-5))
    (when var-6 (setf rdscal var-6)))))
(setf k 1)
(setf ii (f2cl-lib:int-add (f2cl-lib:int-mul mb nb) 1))
```

```
(f2cl-lib:fdo (jj 0 (f2cl-lib:int-add jj 1))
              ((> jj
                  (f2cl-lib:int-add nb
                                    (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy mb
         (f2cl-lib:array-slice rhs double-float (k)
                               ((1 ldz)))
         1
         (f2cl-lib:array-slice c-%data% double-float
                               (is
                                (f2cl-lib:int-add js jj))
                               ((1 ldc) (1 *)) c-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf mb var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy mb
         (f2cl-lib:array-slice rhs double-float (ii)
                               ((1 ldz)))
         1
         (f2cl-lib:array-slice f-%data% double-float
                               (is
                                (f2cl-lib:int-add js jj))
                               ((1 ldf) (1 *)) f-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf mb var-0)))
    (setf k (f2cl-lib:int-add k mb))
    (setf ii (f2cl-lib:int-add ii mb))
   label100))
(cond
 ((> i 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8 var-9 var-10 var-11 var-12)
      (dgemm "N" "N" (f2cl-lib:int-sub is 1) nb mb (- one)
       (f2cl-lib:array-slice a-%data% double-float (1 is)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice rhs double-float (1)
                             ((1 ldz)))
       mb one
       (f2cl-lib:array-slice c-%data% double-float (1 js)
                             ((1 ldc) (1 *)) c-%offset%)
       ldc)
    (declare
     (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
```

```
      (when var-3 (setf nb var-3))
      (when var-4 (setf mb var-4))
      (when var-7 (setf lda var-7))
      (when var-9 (setf mb var-9))
      (when var-10 (setf one var-10))
      (when var-12 (setf ldc var-12)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
         var-8 var-9 var-10 var-11 var-12)
        (dgemm "N" "N" (f2cl-lib:int-sub is 1) nb mb (- one)
         (f2cl-lib:array-slice d-%data% double-float (1 is)
                               ((1 ldd) (1 *)) d-%offset%)
         ldd
         (f2cl-lib:array-slice rhs double-float (1)
                               ((1 ldz)))
         mb one
         (f2cl-lib:array-slice f-%data% double-float (1 js)
                               ((1 ldf) (1 *)) f-%offset%)
         ldf)
      (declare
       (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
      (when var-3 (setf nb var-3))
      (when var-4 (setf mb var-4))
      (when var-7 (setf ldd var-7))
      (when var-9 (setf mb var-9))
      (when var-10 (setf one var-10))
      (when var-12 (setf ldf var-12)))))
  (cond
   ((< j q)
    (setf k (f2cl-lib:int-add (f2cl-lib:int-mul mb nb) 1))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
         var-8 var-9 var-10 var-11 var-12)
        (dgemm "N" "N" mb (f2cl-lib:int-sub n je) nb one
         (f2cl-lib:array-slice rhs double-float (k)
                               ((1 ldz)))
         mb
         (f2cl-lib:array-slice b-%data% double-float
                               (js (f2cl-lib:int-add je 1))
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$ one
         (f2cl-lib:array-slice c-%data% double-float
                               (is (f2cl-lib:int-add je 1))
                               ((1 ldc) (1 *)) c-%offset%)
         ldc)
      (declare (ignore var-0 var-1 var-3 var-6 var-8 var-11))
      (when var-2 (setf mb var-2))
      (when var-4 (setf nb var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf mb var-7))
```

```
                    (when var-9 (setf ldb$ var-9))
                    (when var-10 (setf one var-10))
                    (when var-12 (setf ldc var-12)))
                  (multiple-value-bind
                      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
                       var-8 var-9 var-10 var-11 var-12)
                      (dgemm "N" "N" mb (f2cl-lib:int-sub n je) nb one
                       (f2cl-lib:array-slice rhs double-float (k)
                                             ((1 ldz)))
                       mb
                       (f2cl-lib:array-slice e-%data% double-float
                                             (js (f2cl-lib:int-add je 1))
                                             ((1 lde) (1 *)) e-%offset%)
                       lde one
                       (f2cl-lib:array-slice f-%data% double-float
                                             (is (f2cl-lib:int-add je 1))
                                             ((1 ldf) (1 *)) f-%offset%)
                       ldf)
                    (declare (ignore var-0 var-1 var-3 var-6 var-8 var-11))
                    (when var-2 (setf mb var-2))
                    (when var-4 (setf nb var-4))
                    (when var-5 (setf one var-5))
                    (when var-7 (setf mb var-7))
                    (when var-9 (setf lde var-9))
                    (when var-10 (setf one var-10))
                    (when var-12 (setf ldf var-12)))))))
            label110))
        label120)))
    (t (setf scale one) (setf scaloc one)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i p) nil)
       (tagbody
         (setf is (f2cl-lib:fref iwork-%data% (i) ((1 *)) iwork-%offset%))
         (setf isp1 (f2cl-lib:int-add is 1))
         (setf ie
                 (f2cl-lib:int-sub
                  (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) iwork-%offset%)
                  1))
         (setf mb (f2cl-lib:int-add (f2cl-lib:int-sub ie is) 1))
         (f2cl-lib:fdo (j q (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((> j (f2cl-lib:int-add p 2)) nil)
           (tagbody
             (setf js
                     (f2cl-lib:fref iwork-%data% (j) ((1 *))
                                    iwork-%offset%))
             (setf jsp1 (f2cl-lib:int-add js 1))
             (setf je
                     (f2cl-lib:int-sub
                      (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add j 1))
```

```
                        ((1 *)) iwork-%offset%)
        1))
(setf nb (f2cl-lib:int-add (f2cl-lib:int-sub je js) 1))
(setf zdim (f2cl-lib:int-mul mb nb 2))
(cond
 ((and (= mb 1) (= nb 1))
  (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
          (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                         a-%offset%))
  (setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz)))
          (-
           (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                          b-%offset%)))
  (setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz)))
          (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                         d-%offset%))
  (setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
          (-
           (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                          e-%offset%)))
  (setf (f2cl-lib:fref rhs (1) ((1 ldz)))
          (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                         c-%offset%))
  (setf (f2cl-lib:fref rhs (2) ((1 ldz)))
          (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                         f-%offset%))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (dgetc2 zdim z ldz ipiv jpiv ierr)
    (declare (ignore var-1 var-3 var-4))
    (when var-0 (setf zdim var-0))
    (when var-2 (setf ldz var-2))
    (when var-5 (setf ierr var-5)))
  (if (> ierr 0)
      (setf info ierr))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dgesc2 zdim z ldz rhs ipiv jpiv scaloc)
    (declare (ignore var-1 var-3 var-4 var-5))
    (when var-0 (setf zdim var-0))
    (when var-2 (setf ldz var-2))
    (when var-6 (setf scaloc var-6)))
  (cond
   ((/= scaloc one)
    (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                  ((> k n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal m scaloc
             (f2cl-lib:array-slice c-%data% double-float
                                   (1 k) ((1 ldc) (1 *))
```

```
                                  c-%offset%)
            1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal m scaloc
             (f2cl-lib:array-slice f-%data% double-float
                                  (1 k) ((1 ldf) (1 *))
                                  f-%offset%)
            1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
      label130))
  (setf scale (* scale scaloc))))
(setf (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (1) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (2) ((1 ldz))))
(cond
 ((> j (f2cl-lib:int-add p 2))
  (setf alpha (f2cl-lib:fref rhs (1) ((1 ldz))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5)
      (daxpy (f2cl-lib:int-sub js 1) alpha
       (f2cl-lib:array-slice b-%data% double-float (1 js)
                             ((1 ldb$) (1 *)) b-%offset%)
       1
       (f2cl-lib:array-slice f-%data% double-float (is 1)
                             ((1 ldf) (1 *)) f-%offset%)
       ldf)
    (declare (ignore var-0 var-2 var-3 var-4))
    (when var-1 (setf alpha var-1))
    (when var-5 (setf ldf var-5)))
  (setf alpha (f2cl-lib:fref rhs (2) ((1 ldz))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5)
      (daxpy (f2cl-lib:int-sub js 1) alpha
       (f2cl-lib:array-slice e-%data% double-float (1 js)
                             ((1 lde) (1 *)) e-%offset%)
       1
       (f2cl-lib:array-slice f-%data% double-float (is 1)
                             ((1 ldf) (1 *)) f-%offset%)
       ldf)
    (declare (ignore var-0 var-2 var-3 var-4))
    (when var-1 (setf alpha var-1))
    (when var-5 (setf ldf var-5)))))
```

```
(cond
 ((< i p)
  (setf alpha (- (f2cl-lib:fref rhs (1) ((1 ldz)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5)
      (daxpy (f2cl-lib:int-sub m ie) alpha
       (f2cl-lib:array-slice a-%data% double-float
                             (is (f2cl-lib:int-add ie 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice c-%data% double-float
                             ((+ ie 1) js) ((1 ldc) (1 *))
                             c-%offset%)
       1)
    (declare (ignore var-0 var-2 var-4 var-5))
    (when var-1 (setf alpha var-1))
    (when var-3 (setf lda var-3)))
  (setf alpha (- (f2cl-lib:fref rhs (2) ((1 ldz)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5)
      (daxpy (f2cl-lib:int-sub m ie) alpha
       (f2cl-lib:array-slice d-%data% double-float
                             (is (f2cl-lib:int-add ie 1))
                             ((1 ldd) (1 *)) d-%offset%)
       ldd
       (f2cl-lib:array-slice c-%data% double-float
                             ((+ ie 1) js) ((1 ldc) (1 *))
                             c-%offset%)
       1)
    (declare (ignore var-0 var-2 var-4 var-5))
    (when var-1 (setf alpha var-1))
    (when var-3 (setf ldd var-3))))))
((and (= mb 1) (= nb 2))
 (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                       a-%offset%))
 (setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz))) zero)
 (setf (f2cl-lib:fref z (3 1) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                        b-%offset%)))
 (setf (f2cl-lib:fref z (4 1) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (jsp1 js) ((1 ldb$) (1 *))
                        b-%offset%)))
 (setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz))) zero)
 (setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                       a-%offset%))
 (setf (f2cl-lib:fref z (3 2) ((1 ldz) (1 ldz)))
```

```
        (-
         (f2cl-lib:fref b-%data% (js jsp1) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (4 2) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (jsp1 jsp1)
                        ((1 ldb$) (1 *)) b-%offset%)))
(setf (f2cl-lib:fref z (1 3) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                       d-%offset%))
(setf (f2cl-lib:fref z (2 3) ((1 ldz) (1 ldz))) zero)
(setf (f2cl-lib:fref z (3 3) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (4 3) ((1 ldz) (1 ldz))) zero)
(setf (f2cl-lib:fref z (1 4) ((1 ldz) (1 ldz))) zero)
(setf (f2cl-lib:fref z (2 4) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                       d-%offset%))
(setf (f2cl-lib:fref z (3 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js jsp1) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (4 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (jsp1 jsp1)
                        ((1 lde) (1 *)) e-%offset%)))
(setf (f2cl-lib:fref rhs (1) ((1 ldz)))
        (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                       c-%offset%))
(setf (f2cl-lib:fref rhs (2) ((1 ldz)))
        (f2cl-lib:fref c-%data% (is jsp1) ((1 ldc) (1 *))
                       c-%offset%))
(setf (f2cl-lib:fref rhs (3) ((1 ldz)))
        (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                       f-%offset%))
(setf (f2cl-lib:fref rhs (4) ((1 ldz)))
        (f2cl-lib:fref f-%data% (is jsp1) ((1 ldf) (1 *))
                       f-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (dgetc2 zdim z ldz ipiv jpiv ierr)
  (declare (ignore var-1 var-3 var-4))
  (when var-0 (setf zdim var-0))
  (when var-2 (setf ldz var-2))
  (when var-5 (setf ierr var-5)))
(if (> ierr 0)
    (setf info ierr))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
    (dgesc2 zdim z ldz rhs ipiv jpiv scaloc)
  (declare (ignore var-1 var-3 var-4 var-5))
  (when var-0 (setf zdim var-0))
  (when var-2 (setf ldz var-2))
  (when var-6 (setf scaloc var-6)))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal m scaloc
           (f2cl-lib:array-slice c-%data% double-float
                                 (1 k) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal m scaloc
           (f2cl-lib:array-slice f-%data% double-float
                                 (1 k) ((1 ldf) (1 *))
                                 f-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
     label140))
  (setf scale (* scale scaloc))))
(setf (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (1) ((1 ldz))))
(setf (f2cl-lib:fref c-%data% (is jsp1) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (2) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (3) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (is jsp1) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (4) ((1 ldz))))
(cond
 ((> j (f2cl-lib:int-add p 2))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5)
      (daxpy (f2cl-lib:int-sub js 1)
       (f2cl-lib:array-slice rhs double-float (1)
                             ((1 ldz)))
       (f2cl-lib:array-slice b-%data% double-float (1 js)
```

```
                                ((1 ldb$) (1 *)) b-%offset%)
                 1
                 (f2cl-lib:array-slice f-%data% double-float (is 1)
                                ((1 ldf) (1 *)) f-%offset%)
                 ldf)
            (declare (ignore var-0 var-1 var-2 var-3 var-4))
            (when var-5 (setf ldf var-5)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5)
              (daxpy (f2cl-lib:int-sub js 1)
                (f2cl-lib:array-slice rhs double-float (2)
                                ((1 ldz)))
                (f2cl-lib:array-slice b-%data% double-float (1 jsp1)
                                ((1 ldb$) (1 *)) b-%offset%)
                1
                (f2cl-lib:array-slice f-%data% double-float (is 1)
                                ((1 ldf) (1 *)) f-%offset%)
                ldf)
            (declare (ignore var-0 var-1 var-2 var-3 var-4))
            (when var-5 (setf ldf var-5)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5)
              (daxpy (f2cl-lib:int-sub js 1)
                (f2cl-lib:array-slice rhs double-float (3)
                                ((1 ldz)))
                (f2cl-lib:array-slice e-%data% double-float (1 js)
                                ((1 lde) (1 *)) e-%offset%)
                1
                (f2cl-lib:array-slice f-%data% double-float (is 1)
                                ((1 ldf) (1 *)) f-%offset%)
                ldf)
            (declare (ignore var-0 var-1 var-2 var-3 var-4))
            (when var-5 (setf ldf var-5)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5)
              (daxpy (f2cl-lib:int-sub js 1)
                (f2cl-lib:array-slice rhs double-float (4)
                                ((1 ldz)))
                (f2cl-lib:array-slice e-%data% double-float (1 jsp1)
                                ((1 lde) (1 *)) e-%offset%)
                1
                (f2cl-lib:array-slice f-%data% double-float (is 1)
                                ((1 ldf) (1 *)) f-%offset%)
                ldf)
            (declare (ignore var-0 var-1 var-2 var-3 var-4))
            (when var-5 (setf ldf var-5)))))
        (cond
         ((< i p)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
```

```
          var-8)
         (dger (f2cl-lib:int-sub m ie) nb (- one)
          (f2cl-lib:array-slice a-%data% double-float
                                (is (f2cl-lib:int-add ie 1))
                                ((1 lda) (1 *)) a-%offset%)
          lda
          (f2cl-lib:array-slice rhs double-float (1)
                                ((1 ldz)))
          1
          (f2cl-lib:array-slice c-%data% double-float
                                ((+ ie 1) js) ((1 ldc) (1 *))
                                c-%offset%)
          ldc)
       (declare (ignore var-0 var-2 var-3 var-5 var-6 var-7))
       (when var-1 (setf nb var-1))
       (when var-4 (setf lda var-4))
       (when var-8 (setf ldc var-8)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
          var-8)
         (dger (f2cl-lib:int-sub m ie) nb (- one)
          (f2cl-lib:array-slice d-%data% double-float
                                (is (f2cl-lib:int-add ie 1))
                                ((1 ldd) (1 *)) d-%offset%)
          ldd
          (f2cl-lib:array-slice rhs double-float (3)
                                ((1 ldz)))
          1
          (f2cl-lib:array-slice c-%data% double-float
                                ((+ ie 1) js) ((1 ldc) (1 *))
                                c-%offset%)
          ldc)
       (declare (ignore var-0 var-2 var-3 var-5 var-6 var-7))
       (when var-1 (setf nb var-1))
       (when var-4 (setf ldd var-4))
       (when var-8 (setf ldc var-8))))))
  ((and (= mb 2) (= nb 1))
   (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
          (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                         a-%offset%))
   (setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz)))
          (f2cl-lib:fref a-%data% (is isp1) ((1 lda) (1 *))
                         a-%offset%))
   (setf (f2cl-lib:fref z (3 1) ((1 ldz) (1 ldz)))
          (-
           (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                          b-%offset%)))
   (setf (f2cl-lib:fref z (4 1) ((1 ldz) (1 ldz))) zero)
   (setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz)))
          (f2cl-lib:fref a-%data% (isp1 is) ((1 lda) (1 *))
```

```
                        a-%offset%))
(setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (isp1 isp1) ((1 lda) (1 *))
                        a-%offset%))
(setf (f2cl-lib:fref z (3 2) ((1 ldz) (1 ldz))) zero)
(setf (f2cl-lib:fref z (4 2) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                         b-%offset%)))
(setf (f2cl-lib:fref z (1 3) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                        d-%offset%))
(setf (f2cl-lib:fref z (2 3) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is isp1) ((1 ldd) (1 *))
                        d-%offset%))
(setf (f2cl-lib:fref z (3 3) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                         e-%offset%)))
(setf (f2cl-lib:fref z (4 3) ((1 ldz) (1 ldz))) zero)
(setf (f2cl-lib:fref z (1 4) ((1 ldz) (1 ldz))) zero)
(setf (f2cl-lib:fref z (2 4) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (isp1 isp1) ((1 ldd) (1 *))
                        d-%offset%))
(setf (f2cl-lib:fref z (3 4) ((1 ldz) (1 ldz))) zero)
(setf (f2cl-lib:fref z (4 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                         e-%offset%)))
(setf (f2cl-lib:fref rhs (1) ((1 ldz)))
        (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                        c-%offset%))
(setf (f2cl-lib:fref rhs (2) ((1 ldz)))
        (f2cl-lib:fref c-%data% (isp1 js) ((1 ldc) (1 *))
                        c-%offset%))
(setf (f2cl-lib:fref rhs (3) ((1 ldz)))
        (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                        f-%offset%))
(setf (f2cl-lib:fref rhs (4) ((1 ldz)))
        (f2cl-lib:fref f-%data% (isp1 js) ((1 ldf) (1 *))
                        f-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (dgetc2 zdim z ldz ipiv jpiv ierr)
  (declare (ignore var-1 var-3 var-4))
  (when var-0 (setf zdim var-0))
  (when var-2 (setf ldz var-2))
  (when var-5 (setf ierr var-5)))
(if (> ierr 0)
    (setf info ierr))
(multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dgesc2 zdim z ldz rhs ipiv jpiv scaloc)
  (declare (ignore var-1 var-3 var-4 var-5))
  (when var-0 (setf zdim var-0))
  (when var-2 (setf ldz var-2))
  (when var-6 (setf scaloc var-6)))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal m scaloc
           (f2cl-lib:array-slice c-%data% double-float
                                 (1 k) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal m scaloc
           (f2cl-lib:array-slice f-%data% double-float
                                 (1 k) ((1 ldf) (1 *))
                                 f-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
     label150))
  (setf scale (* scale scaloc))))
(setf (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (1) ((1 ldz))))
(setf (f2cl-lib:fref c-%data% (isp1 js) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (2) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (3) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (isp1 js) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (4) ((1 ldz))))
(cond
 ((> j (f2cl-lib:int-add p 2))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8)
      (dger mb (f2cl-lib:int-sub js 1) one
       (f2cl-lib:array-slice rhs double-float (1)
```

```
                                ((1 ldz)))
                1
                (f2cl-lib:array-slice b-%data% double-float (1 js)
                                ((1 ldb$) (1 *)) b-%offset%)
                1
                (f2cl-lib:array-slice f-%data% double-float (is 1)
                                ((1 ldf) (1 *)) f-%offset%)
                ldf)
            (declare (ignore var-1 var-3 var-4 var-5 var-6 var-7))
            (when var-0 (setf mb var-0))
            (when var-2 (setf one var-2))
            (when var-8 (setf ldf var-8)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
               var-8)
              (dger mb (f2cl-lib:int-sub js 1) one
               (f2cl-lib:array-slice rhs double-float (3)
                                ((1 ldz)))
               1
               (f2cl-lib:array-slice e-%data% double-float (1 js)
                                ((1 lde) (1 *)) e-%offset%)
               1
               (f2cl-lib:array-slice f-%data% double-float (is 1)
                                ((1 ldf) (1 *)) f-%offset%)
               ldf)
            (declare (ignore var-1 var-3 var-4 var-5 var-6 var-7))
            (when var-0 (setf mb var-0))
            (when var-2 (setf one var-2))
            (when var-8 (setf ldf var-8)))))
        (cond
         ((< i p)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
               var-8 var-9 var-10)
              (dgemv "T" mb (f2cl-lib:int-sub m ie) (- one)
               (f2cl-lib:array-slice a-%data% double-float
                                (is (f2cl-lib:int-add ie 1))
                                ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice rhs double-float (1)
                                ((1 ldz)))
               1 one
               (f2cl-lib:array-slice c-%data% double-float
                                ((+ ie 1) js) ((1 ldc) (1 *))
                                c-%offset%)
               1)
            (declare
             (ignore var-0 var-2 var-3 var-4 var-6 var-7 var-9
              var-10))
            (when var-1 (setf mb var-1))
```

```
      (when var-5 (setf lda var-5))
      (when var-8 (setf one var-8)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
         var-8 var-9 var-10)
        (dgemv "T" mb (f2cl-lib:int-sub m ie) (- one)
         (f2cl-lib:array-slice d-%data% double-float
                               (is (f2cl-lib:int-add ie 1))
                               ((1 ldd) (1 *)) d-%offset%)
         ldd
         (f2cl-lib:array-slice rhs double-float (3)
                               ((1 ldz)))
         1 one
         (f2cl-lib:array-slice c-%data% double-float
                               ((+ ie 1) js) ((1 ldc) (1 *))
                               c-%offset%)
         1)
      (declare
       (ignore var-0 var-2 var-3 var-4 var-6 var-7 var-9
        var-10))
      (when var-1 (setf mb var-1))
      (when var-5 (setf ldd var-5))
      (when var-8 (setf one var-8))))))
((and (= mb 2) (= nb 2))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlaset "F" ldz ldz zero zero z ldz)
   (declare (ignore var-0 var-5))
   (when var-1 (setf ldz var-1))
   (when var-2 (setf ldz var-2))
   (when var-3 (setf zero var-3))
   (when var-4 (setf zero var-4))
   (when var-6 (setf ldz var-6)))
 (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                        a-%offset%))
 (setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (is isp1) ((1 lda) (1 *))
                        a-%offset%))
 (setf (f2cl-lib:fref z (5 1) ((1 ldz) (1 ldz)))
         (-
          (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                         b-%offset%)))
 (setf (f2cl-lib:fref z (7 1) ((1 ldz) (1 ldz)))
         (-
          (f2cl-lib:fref b-%data% (jsp1 js) ((1 ldb$) (1 *))
                         b-%offset%)))
 (setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (isp1 is) ((1 lda) (1 *))
                        a-%offset%))
```

```
(setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (isp1 isp1) ((1 lda) (1 *))
                       a-%offset%))
(setf (f2cl-lib:fref z (6 2) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (8 2) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (jsp1 js) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (3 3) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                       a-%offset%))
(setf (f2cl-lib:fref z (4 3) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (is isp1) ((1 lda) (1 *))
                       a-%offset%))
(setf (f2cl-lib:fref z (5 3) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (js jsp1) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (7 3) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (jsp1 jsp1)
                        ((1 ldb$) (1 *)) b-%offset%)))
(setf (f2cl-lib:fref z (3 4) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (isp1 is) ((1 lda) (1 *))
                       a-%offset%))
(setf (f2cl-lib:fref z (4 4) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (isp1 isp1) ((1 lda) (1 *))
                       a-%offset%))
(setf (f2cl-lib:fref z (6 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (js jsp1) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (8 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (jsp1 jsp1)
                        ((1 ldb$) (1 *)) b-%offset%)))
(setf (f2cl-lib:fref z (1 5) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                       d-%offset%))
(setf (f2cl-lib:fref z (2 5) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is isp1) ((1 ldd) (1 *))
                       d-%offset%))
(setf (f2cl-lib:fref z (5 5) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (2 6) ((1 ldz) (1 ldz)))
```

```
        (f2cl-lib:fref d-%data% (isp1 isp1) ((1 ldd) (1 *))
                       d-%offset%))
(setf (f2cl-lib:fref z (6 6) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (3 7) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                       d-%offset%))
(setf (f2cl-lib:fref z (4 7) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is isp1) ((1 ldd) (1 *))
                       d-%offset%))
(setf (f2cl-lib:fref z (5 7) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js jsp1) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (7 7) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (jsp1 jsp1)
                        ((1 lde) (1 *)) e-%offset%)))
(setf (f2cl-lib:fref z (4 8) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (isp1 isp1) ((1 ldd) (1 *))
                       d-%offset%))
(setf (f2cl-lib:fref z (6 8) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js jsp1) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (8 8) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (jsp1 jsp1)
                        ((1 lde) (1 *)) e-%offset%)))
(setf k 1)
(setf ii (f2cl-lib:int-add (f2cl-lib:int-mul mb nb) 1))
(f2cl-lib:fdo (jj 0 (f2cl-lib:int-add jj 1))
              ((> jj
                  (f2cl-lib:int-add nb
                                    (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy mb
         (f2cl-lib:array-slice c-%data% double-float
                               (is
                                (f2cl-lib:int-add js jj))
                               ((1 ldc) (1 *)) c-%offset%)
         1
         (f2cl-lib:array-slice rhs double-float (k)
                               ((1 ldz)))
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
```

```
      (when var-0 (setf mb var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy mb
         (f2cl-lib:array-slice f-%data% double-float
                               (is
                                (f2cl-lib:int-add js jj))
                               ((1 ldf) (1 *)) f-%offset%)
         1
         (f2cl-lib:array-slice rhs double-float (ii)
                               ((1 ldz)))
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf mb var-0)))
    (setf k (f2cl-lib:int-add k mb))
    (setf ii (f2cl-lib:int-add ii mb))
   label160))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (dgetc2 zdim z ldz ipiv jpiv ierr)
  (declare (ignore var-1 var-3 var-4))
  (when var-0 (setf zdim var-0))
  (when var-2 (setf ldz var-2))
  (when var-5 (setf ierr var-5)))
(if (> ierr 0)
    (setf info ierr))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dgesc2 zdim z ldz rhs ipiv jpiv scaloc)
  (declare (ignore var-1 var-3 var-4 var-5))
  (when var-0 (setf zdim var-0))
  (when var-2 (setf ldz var-2))
  (when var-6 (setf scaloc var-6)))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal m scaloc
           (f2cl-lib:array-slice c-%data% double-float
                                 (1 k) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal m scaloc
           (f2cl-lib:array-slice f-%data% double-float
                                 (1 k) ((1 ldf) (1 *))
                                 f-%offset%)
```

```
          1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
     label170))
  (setf scale (* scale scaloc))))
(setf k 1)
(setf ii (f2cl-lib:int-add (f2cl-lib:int-mul mb nb) 1))
(f2cl-lib:fdo (jj 0 (f2cl-lib:int-add jj 1))
              ((> jj
                  (f2cl-lib:int-add nb
                                    (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy mb
         (f2cl-lib:array-slice rhs double-float (k)
                               ((1 ldz)))
         1
         (f2cl-lib:array-slice c-%data% double-float
                               (is
                                (f2cl-lib:int-add js jj))
                               ((1 ldc) (1 *)) c-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf mb var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy mb
         (f2cl-lib:array-slice rhs double-float (ii)
                               ((1 ldz)))
         1
         (f2cl-lib:array-slice f-%data% double-float
                               (is
                                (f2cl-lib:int-add js jj))
                               ((1 ldf) (1 *)) f-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf mb var-0)))
    (setf k (f2cl-lib:int-add k mb))
    (setf ii (f2cl-lib:int-add ii mb))
   label180))
(cond
 ((> j (f2cl-lib:int-add p 2))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8 var-9 var-10 var-11 var-12)
      (dgemm "N" "T" mb (f2cl-lib:int-sub js 1) nb one
       (f2cl-lib:array-slice c-%data% double-float (is js)
                             ((1 ldc) (1 *)) c-%offset%)
       ldc
```

```
          (f2cl-lib:array-slice b-%data% double-float (1 js)
                                ((1 ldb$) (1 *)) b-%offset%)
          ldb$ one
          (f2cl-lib:array-slice f-%data% double-float (is 1)
                                ((1 ldf) (1 *)) f-%offset%)
          ldf)
        (declare (ignore var-0 var-1 var-3 var-6 var-8 var-11))
        (when var-2 (setf mb var-2))
        (when var-4 (setf nb var-4))
        (when var-5 (setf one var-5))
        (when var-7 (setf ldc var-7))
        (when var-9 (setf ldb$ var-9))
        (when var-10 (setf one var-10))
        (when var-12 (setf ldf var-12)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
           var-8 var-9 var-10 var-11 var-12)
          (dgemm "N" "T" mb (f2cl-lib:int-sub js 1) nb one
           (f2cl-lib:array-slice f-%data% double-float (is js)
                                 ((1 ldf) (1 *)) f-%offset%)
           ldf
           (f2cl-lib:array-slice e-%data% double-float (1 js)
                                 ((1 lde) (1 *)) e-%offset%)
           lde one
           (f2cl-lib:array-slice f-%data% double-float (is 1)
                                 ((1 ldf) (1 *)) f-%offset%)
           ldf)
        (declare (ignore var-0 var-1 var-3 var-6 var-8 var-11))
        (when var-2 (setf mb var-2))
        (when var-4 (setf nb var-4))
        (when var-5 (setf one var-5))
        (when var-7 (setf ldf var-7))
        (when var-9 (setf lde var-9))
        (when var-10 (setf one var-10))
        (when var-12 (setf ldf var-12)))))
    (cond
     ((< i p)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
           var-8 var-9 var-10 var-11 var-12)
          (dgemm "T" "N" (f2cl-lib:int-sub m ie) nb mb (- one)
           (f2cl-lib:array-slice a-%data% double-float
                                 (is (f2cl-lib:int-add ie 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice c-%data% double-float (is js)
                                 ((1 ldc) (1 *)) c-%offset%)
           ldc one
           (f2cl-lib:array-slice c-%data% double-float
                                 ((+ ie 1) js) ((1 ldc) (1 *))
```

```
                                                c-%offset%)
                            ldc)
                          (declare
                           (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
                          (when var-3 (setf nb var-3))
                          (when var-4 (setf mb var-4))
                          (when var-7 (setf lda var-7))
                          (when var-9 (setf ldc var-9))
                          (when var-10 (setf one var-10))
                          (when var-12 (setf ldc var-12)))
                        (multiple-value-bind
                            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
                             var-8 var-9 var-10 var-11 var-12)
                            (dgemm "T" "N" (f2cl-lib:int-sub m ie) nb mb (- one)
                             (f2cl-lib:array-slice d-%data% double-float
                                                   (is (f2cl-lib:int-add ie 1))
                                                   ((1 ldd) (1 *)) d-%offset%)
                             ldd
                             (f2cl-lib:array-slice f-%data% double-float (is js)
                                                   ((1 ldf) (1 *)) f-%offset%)
                             ldf one
                             (f2cl-lib:array-slice c-%data% double-float
                                                   ((+ ie 1) js) ((1 ldc) (1 *))
                                                   c-%offset%)
                             ldc)
                          (declare
                           (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
                          (when var-3 (setf nb var-3))
                          (when var-4 (setf mb var-4))
                          (when var-7 (setf ldd var-7))
                          (when var-9 (setf ldf var-9))
                          (when var-10 (setf one var-10))
                          (when var-12 (setf ldc var-12)))))))
                  label190))
              label200))))
        (go end_label)
       end_label
        (return
         (values trans ijob m nil nil lda nil ldb$ nil ldc nil ldd nil lde nil
                 ldf scale rdsum rdscal nil pq info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dtgsy2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
```

```
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (double-float) (double-float)
  (double-float)
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::trans
  fortran-to-lisp::ijob
  fortran-to-lisp::m nil nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldc nil
  fortran-to-lisp::ldd nil
  fortran-to-lisp::lde nil
  fortran-to-lisp::ldf
  fortran-to-lisp::scale
  fortran-to-lisp::rdsum
  fortran-to-lisp::rdscal nil
  fortran-to-lisp::pq
  fortran-to-lisp::info)
:calls 'nil)))
```

Chapter 63

LAPACK - Symmetric Matrix, Auxilliary Routines, Real

63.1 slansy

63.1.1 Axiom unit tests

— slansy.input —

```
)set break resume
)sys rm -f slansy.output
)spool slansy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

63.1.2 Axiom help page

— slansy.help —

```
==========================================================================
slansy examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       real function slansy (NORM, UPLO, N, A, LDA, WORK)
           SLANSY returns the value of the 1-norm, or the Frobenius norm, or
           the infinity norm, or the element of largest absolute value of a
           real symmetric matrix.

Function/Subroutine Documentation
   real function slansy (character NORM, character UPLO, integer N, real,
       dimension( lda, * ) A, integer LDA, real, dimension( * ) WORK)
       SLANSY returns the value of the 1-norm, or the Frobenius norm, or the
       infinity norm, or the element of largest absolute value of a real
       symmetric matrix.

Purpose:
   SLANSY  returns the value of the one norm,  or the Frobenius norm, or
   the  infinity norm,  or the  element of  largest absolute value  of a
   real symmetric matrix A.

 Returns:

   SLANSY

       SLANSY = ( max(abs(A(i,j))), NORM = 'M' or 'm'
                (
                ( norm1(A),         NORM = '1', 'O' or 'o'
                (
                ( normI(A),         NORM = 'I' or 'i'
                (
                ( normF(A),         NORM = 'F', 'f', 'E' or 'e'

   where  norm1  denotes the  one norm of a matrix (maximum column sum),
   normI  denotes the  infinity norm  of a matrix  (maximum row sum) and
   normF  denotes the  Frobenius norm of a matrix (square root of sum of
   squares).  Note that  max(abs(A(i,j)))  is not a consistent matrix norm.

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies the value to be returned in SLANSY as described above.

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
```

```
  symmetric matrix A is to be referenced.
  = 'U':  Upper triangular part of A is referenced
  = 'L':  Lower triangular part of A is referenced

N
  N is INTEGER
  The order of the matrix A.  N >= 0.  When N = 0, SLANSY is set to zero.

A
  A is REAL array, dimension (LDA,N)
  The symmetric matrix A.  If UPLO = 'U', the leading n by n
  upper triangular part of A contains the upper triangular part
  of the matrix A, and the strictly lower triangular part of A
  is not referenced.  If UPLO = 'L', the leading n by n lower
  triangular part of A contains the lower triangular part of
  the matrix A, and the strictly upper triangular part of A is
  not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(N,1).

WORK
  WORK is REAL array, dimension (MAX(1,LWORK)),
  where LWORK >= N when NORM = 'I' or '1' or 'O'; otherwise,
  WORK is not referenced.
```

———-

63.1.3 fortran code

— slansy.f —

```
*  =====================================================================
      REAL             FUNCTION SLANSY( NORM, UPLO, N, A, LDA, WORK )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          NORM, UPLO
      INTEGER            LDA, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), WORK( * )
```

```
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               ABSA, SCALE, SUM, VALUE
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLASSQ
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, SISNAN
      EXTERNAL           LSAME, SISNAN
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SQRT
*     ..
*     .. Executable Statements ..
*
      IF( N.EQ.0 ) THEN
         VALUE = ZERO
      ELSE IF( LSAME( NORM, 'M' ) ) THEN
*
*        Find max(abs(A(i,j))).
*
         VALUE = ZERO
         IF( LSAME( UPLO, 'U' ) ) THEN
            DO 20 J = 1, N
               DO 10 I = 1, J
                  SUM = ABS( A( I, J ) )
                  IF( VALUE .LT. SUM .OR. SISNAN( SUM ) ) VALUE = SUM
   10          CONTINUE
   20       CONTINUE
         ELSE
            DO 40 J = 1, N
               DO 30 I = J, N
                  SUM = ABS( A( I, J ) )
                  IF( VALUE .LT. SUM .OR. SISNAN( SUM ) ) VALUE = SUM
   30          CONTINUE
   40       CONTINUE
         END IF
      ELSE IF( ( LSAME( NORM, 'I' ) ) .OR. ( LSAME( NORM, 'O' ) ) .OR.
     $         ( NORM.EQ.'1' ) ) THEN
*
*        Find normI(A) ( = norm1(A), since A is symmetric).
```

```
*
         VALUE = ZERO
         IF( LSAME( UPLO, 'U' ) ) THEN
            DO 60 J = 1, N
               SUM = ZERO
               DO 50 I = 1, J - 1
                  ABSA = ABS( A( I, J ) )
                  SUM = SUM + ABSA
                  WORK( I ) = WORK( I ) + ABSA
   50          CONTINUE
               WORK( J ) = SUM + ABS( A( J, J ) )
   60       CONTINUE
            DO 70 I = 1, N
               SUM = WORK( I )
               IF( VALUE .LT. SUM .OR. SISNAN( SUM ) ) VALUE = SUM
   70       CONTINUE
         ELSE
            DO 80 I = 1, N
               WORK( I ) = ZERO
   80       CONTINUE
            DO 100 J = 1, N
               SUM = WORK( J ) + ABS( A( J, J ) )
               DO 90 I = J + 1, N
                  ABSA = ABS( A( I, J ) )
                  SUM = SUM + ABSA
                  WORK( I ) = WORK( I ) + ABSA
   90          CONTINUE
               IF( VALUE .LT. SUM .OR. SISNAN( SUM ) ) VALUE = SUM
  100       CONTINUE
         END IF
      ELSE IF( ( LSAME( NORM, 'F' ) ) .OR. ( LSAME( NORM, 'E' ) ) ) THEN
*
*        Find normF(A).
*
         SCALE = ZERO
         SUM = ONE
         IF( LSAME( UPLO, 'U' ) ) THEN
            DO 110 J = 2, N
               CALL SLASSQ( J-1, A( 1, J ), 1, SCALE, SUM )
  110       CONTINUE
         ELSE
            DO 120 J = 1, N - 1
               CALL SLASSQ( N-J, A( J+1, J ), 1, SCALE, SUM )
  120       CONTINUE
         END IF
         SUM = 2*SUM
         CALL SLASSQ( N, A, LDA+1, SCALE, SUM )
         VALUE = SCALE*SQRT( SUM )
      END IF
*
```

```
      SLANSY = VALUE
      RETURN
*
*     End of SLANSY
*
      END
```

———-

63.1.4 lisp code

— **slansy.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun slansy (norm uplo n a lda work)
    (declare (type (array single-float (*)) work a)
             (type (f2cl-lib:integer4) lda n)
             (type (string *) uplo norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((absa 0.0) (scale 0.0) (sum 0.0) (value 0.0) (i 0) (j 0)
             (slansy 0.0))
        (declare (type (f2cl-lib:integer4) i j)
                 (type (single-float) absa scale sum value slansy))
        (cond ((= n 0) (setf value zero))
              ((multiple-value-bind (ret-val var-0 var-1)
                   (lsame norm "M")
                 (declare (ignore var-1))
                 (when var-0 (setf norm var-0))
                 ret-val)
               (setf value zero)
               (cond
                ((multiple-value-bind (ret-val var-0 var-1)
                     (lsame uplo "U")
                   (declare (ignore var-1))
                   (when var-0 (setf uplo var-0))
                   ret-val)
                 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                               ((> j n) nil)
                   (tagbody
                     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                   ((> i j) nil)
```

```
            (tagbody
              (setf sum
                      (abs
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)))
              (if (or (< value sum) (sisnan sum))
                  (setf value sum))
             label10))
         label20)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf sum
                     (abs
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%)))
             (if (or (< value sum) (sisnan sum))
                 (setf value sum))
            label30))
         label40)))))
  ((or
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame norm "I")
      (declare (ignore var-1))
      (when var-0 (setf norm var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame norm "O")
      (declare (ignore var-1))
      (when var-0 (setf norm var-0))
      ret-val)
    (f2cl-lib:fstring-= norm "1"))
   (setf value zero)
   (cond
    ((multiple-value-bind (ret-val var-0 var-1)
         (lsame uplo "U")
       (declare (ignore var-1))
       (when var-0 (setf uplo var-0))
       ret-val)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf sum zero)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i
                           (f2cl-lib:int-add j
```

```
                                                (f2cl-lib:int-sub 1)))
                          nil)
                (tagbody
                  (setf absa
                          (abs
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%)))
                  (setf sum (+ sum absa))
                  (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                       work-%offset%)
                          (+
                           (f2cl-lib:fref work-%data% (i) ((1 *))
                                          work-%offset%)
                           absa))
                 label50))
              (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                                   work-%offset%)
                    (+ sum
                       (abs
                        (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                       a-%offset%))))
            label60))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf sum
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%))
              (if (or (< value sum) (sisnan sum))
                  (setf value sum))
            label70)))
        (t
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)
                    zero)
            label80))
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf sum
                    (+
                     (f2cl-lib:fref work-%data% (j) ((1 *))
                                    work-%offset%)
                     (abs
                      (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%))))
              (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
```

```
                        (f2cl-lib:int-add i 1))
                       ((> i n) nil)
            (tagbody
              (setf absa
                      (abs
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)))
              (setf sum (+ sum absa))
              (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)
                      (+
                       (f2cl-lib:fref work-%data% (i) ((1 *))
                                      work-%offset%)
                       absa))
             label90))
          (if (or (< value sum) (sisnan sum))
              (setf value sum))
        label100)))))
  ((or
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame norm "F")
      (declare (ignore var-1))
      (when var-0 (setf norm var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame norm "E")
      (declare (ignore var-1))
      (when var-0 (setf norm var-0))
      ret-val))
   (setf scale zero) (setf sum one)
   (cond
    ((multiple-value-bind (ret-val var-0 var-1)
         (lsame uplo "U")
       (declare (ignore var-1))
       (when var-0 (setf uplo var-0))
       ret-val)
     (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (slassq (f2cl-lib:int-sub j 1)
              (f2cl-lib:array-slice a-%data% single-float (1 j)
                                    ((1 lda) (1 *)) a-%offset%)
              1 scale sum)
           (declare (ignore var-0 var-1 var-2))
           (when var-3 (setf scale var-3))
           (when var-4 (setf sum var-4)))
       label110)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                              ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                               nil)
                    (tagbody
                      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                          (slassq (f2cl-lib:int-sub n j)
                           (f2cl-lib:array-slice a-%data% single-float
                                                 ((+ j 1) j) ((1 lda) (1 *))
                                                 a-%offset%)
                           1 scale sum)
                        (declare (ignore var-0 var-1 var-2))
                        (when var-3 (setf scale var-3))
                        (when var-4 (setf sum var-4)))
                     label120))))
                (setf sum (* 2 sum))
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (slassq n a (f2cl-lib:int-add lda 1) scale sum)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf n var-0))
                  (when var-3 (setf scale var-3))
                  (when var-4 (setf sum var-4)))
                (setf value (* scale (f2cl-lib:fsqrt sum)))))
        (setf slansy value)
        (go end_label)
       end_label
        (return (values slansy norm uplo n nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slansy
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*)))
                                            :return-values
                                            '(fortran-to-lisp::norm
                                              fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil nil nil)
                                            :calls '(fortran-to-lisp::sisnan))))
```

———-

63.2 slaqsy

63.2.1 Axiom unit tests

— **slaqsy.input** —

```
)set break resume
)sys rm -f slaqsy.output
)spool slaqsy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

63.2.2 Axiom help page

— **slaqsy.help** —

```
======================================================================
slaqsy examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slaqsy (UPLO, N, A, LDA, S, SCOND, AMAX, EQUED)
           SLAQSY scales a symmetric/Hermitian matrix, using scaling factors
           computed by spoequ.

Function/Subroutine Documentation
   subroutine slaqsy (character UPLO, integer N, real, dimension( lda, * ) A,
       integer LDA, real, dimension( * ) S, real SCOND, real AMAX, character
       EQUED)
       SLAQSY scales a symmetric/Hermitian matrix, using scaling factors
       computed by spoequ.

Purpose:
   SLAQSY equilibrates a symmetric matrix A using the scaling factors
   in the vector S.
```

```
Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is stored.
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   n by n upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading n by n lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

   On exit, if EQUED = 'Y', the equilibrated matrix:
   diag(S) * A * diag(S).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(N,1).

 S
   S is REAL array, dimension (N)
   The scale factors for A.

 SCOND
   SCOND is REAL
   Ratio of the smallest S(i) to the largest S(i).

 AMAX
   AMAX is REAL
   Absolute value of largest matrix entry.

 EQUED
   EQUED is CHARACTER*1
   Specifies whether or not equilibration was done.
   = 'N':  No equilibration.
   = 'Y':  Equilibration was done, i.e., A has been replaced by
           diag(S) * A * diag(S).
```

```
Internal Parameters:

   THRESH is a threshold value used to decide if scaling should be done
   based on the ratio of the scaling factors.  If SCOND < THRESH,
   scaling is done.

   LARGE and SMALL are threshold values used to decide if scaling should
   be done based on the absolute size of the largest matrix element.
   If AMAX > LARGE or AMAX < SMALL, scaling is done.
```

———-

63.2.3 fortran code

— **slaqsy.f** —

```
*  =====================================================================
      SUBROUTINE SLAQSY( UPLO, N, A, LDA, S, SCOND, AMAX, EQUED )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, UPLO
      INTEGER            LDA, N
      REAL               AMAX, SCOND
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), S( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, THRESH
      PARAMETER          ( ONE = 1.0E+0, THRESH = 0.1E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               CJ, LARGE, SMALL
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH
      EXTERNAL           LSAME, SLAMCH
```

```
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( N.LE.0 ) THEN
         EQUED = 'N'
         RETURN
      END IF
*
*     Initialize LARGE and SMALL.
*
      SMALL = SLAMCH( 'Safe minimum' ) / SLAMCH( 'Precision' )
      LARGE = ONE / SMALL
*
      IF( SCOND.GE.THRESH .AND. AMAX.GE.SMALL .AND. AMAX.LE.LARGE ) THEN
*
*        No equilibration
*
         EQUED = 'N'
      ELSE
*
*        Replace A by diag(S) * A * diag(S).
*
         IF( LSAME( UPLO, 'U' ) ) THEN
*
*           Upper triangle of A is stored.
*
            DO 20 J = 1, N
               CJ = S( J )
               DO 10 I = 1, J
                  A( I, J ) = CJ*S( I )*A( I, J )
   10          CONTINUE
   20       CONTINUE
         ELSE
*
*           Lower triangle of A is stored.
*
            DO 40 J = 1, N
               CJ = S( J )
               DO 30 I = J, N
                  A( I, J ) = CJ*S( I )*A( I, J )
   30          CONTINUE
   40       CONTINUE
         END IF
         EQUED = 'Y'
      END IF
*
      RETURN
*
```

```
*     End of SLAQSY
*
      END
```

———-

63.2.4 lisp code

— **slaqsy.lisp** —

```
(let* ((one 1.0) (thresh 0.1))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.1 0.1) thresh)
           (ignorable one thresh))
  (defun slaqsy (uplo n a lda s scond amax equed)
    (declare (type (single-float) amax scond)
             (type (array single-float (*)) s a)
             (type (f2cl-lib:integer4) lda n)
             (type (string *) equed uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (equed character equed-%data% equed-%offset%)
         (a single-float a-%data% a-%offset%)
         (s single-float s-%data% s-%offset%))
      (prog ((cj 0.0) (large 0.0) (small 0.0) (i 0) (j 0))
        (declare (type (single-float) cj large small)
                 (type (f2cl-lib:integer4) i j))
        (cond
         ((<= n 0) (f2cl-lib:f2cl-set-string equed "N" (string 1))
          (go end_label)))
        (setf small (/ (slamch "Safe minimum") (slamch "Precision")))
        (setf large (/ one small))
        (cond
         ((and (>= scond thresh) (>= amax small) (<= amax large))
          (f2cl-lib:f2cl-set-string equed "N" (string 1)))
         (t
          (cond
           ((multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf cj (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
                (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                              ((> i j) nil)
```

```
                  (tagbody
                    (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                            (* cj
                               (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                               (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                              a-%offset%)))
                   label10))
               label20)))
            (t
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (setf cj (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
                 (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                               ((> i n) nil)
                   (tagbody
                     (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%)
                             (* cj
                                (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                               a-%offset%)))
                   label30))
               label40))))
          (f2cl-lib:f2cl-set-string equed "Y" (string 1))))
        (go end_label)
       end_label
        (return (values uplo nil nil nil nil nil nil equed))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slaqsy
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (string))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil nil
                                              nil nil nil
                                              fortran-to-lisp::equed)
                                            :calls 'nil)))
```

———-

63.3 slasy2

63.3.1 Axiom unit tests

— **slasy2.input** —

```
)set break resume
)sys rm -f slasy2.output
)spool slasy2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

63.3.2 Axiom help page

— **slasy2.help** —

```
====================================================================
slasy2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slasy2 (LTRANL, LTRANR, ISGN, N1, N2, TL, LDTL, TR, LDTR, B,
           LDB, SCALE, X, LDX, XNORM, INFO)
           SLASY2 solves the Sylvester matrix equation where the matrices are
           of order 1 or 2.

Function/Subroutine Documentation
   subroutine slasy2 (logical LTRANL, logical LTRANR, integer ISGN, integer
       N1, integer N2, real, dimension( ldtl, * ) TL, integer LDTL, real,
       dimension( ldtr, * ) TR, integer LDTR, real, dimension( ldb, * ) B,
       integer LDB, real SCALE, real, dimension( ldx, * ) X, integer LDX, real
```

```
       XNORM, integer INFO)
       SLASY2 solves the Sylvester matrix equation where the matrices are of
       order 1 or 2.

Purpose:
   SLASY2 solves for the N1 by N2 matrix X, 1 <= N1,N2 <= 2, in

   op(TL)*X + ISGN*X*op(TR) = SCALE*B,

   where TL is N1 by N1, TR is N2 by N2, B is N1 by N2, and ISGN = 1 or -1
   op(T) = T or T**T, where T**T denotes the transpose of T.

Parameters:

 LTRANL
   LTRANL is LOGICAL
   On entry, LTRANL specifies the op(TL):
   = .FALSE., op(TL) = TL,
   = .TRUE., op(TL) = TL**T.

 LTRANR
   LTRANR is LOGICAL
   On entry, LTRANR specifies the op(TR):
   = .FALSE., op(TR) = TR,
   = .TRUE., op(TR) = TR**T.

 ISGN
   ISGN is INTEGER
   On entry, ISGN specifies the sign of the equation
   as described before. ISGN may only be 1 or -1.

 N1
   N1 is INTEGER
   On entry, N1 specifies the order of matrix TL.
   N1 may only be 0, 1 or 2.

 N2
   N2 is INTEGER
   On entry, N2 specifies the order of matrix TR.
   N2 may only be 0, 1 or 2.

 TL
   TL is REAL array, dimension (LDTL,2)
   On entry, TL contains an N1 by N1 matrix.

 LDTL
   LDTL is INTEGER
   The leading dimension of the matrix TL. LDTL >= max(1,N1).

 TR
```

```
  TR is REAL array, dimension (LDTR,2)
  On entry, TR contains an N2 by N2 matrix.

LDTR
  LDTR is INTEGER
  The leading dimension of the matrix TR. LDTR >= max(1,N2).

B
  B is REAL array, dimension (LDB,2)
  On entry, the N1 by N2 matrix B contains the right-hand
  side of the equation.

LDB
  LDB is INTEGER
  The leading dimension of the matrix B. LDB >= max(1,N1).

SCALE
  SCALE is REAL
  On exit, SCALE contains the scale factor. SCALE is chosen
  less than or equal to 1 to prevent the solution overflowing.

X
  X is REAL array, dimension (LDX,2)
  On exit, X contains the N1 by N2 solution.

LDX
  LDX is INTEGER
  The leading dimension of the matrix X. LDX >= max(1,N1).

XNORM
  XNORM is REAL
  On exit, XNORM is the infinity-norm of the solution.

INFO
  INFO is INTEGER
  On exit, INFO is set to
    0: successful exit.
    1: TL and TR have too close eigenvalues, so TL or
       TR is perturbed to get a nonsingular equation.
  NOTE: In the interests of speed, this routine does not
  check the inputs for errors.
```

———-

63.3.3 fortran code

— slasy2.f —

```
*  =====================================================================
      SUBROUTINE SLASY2( LTRANL, LTRANR, ISGN, N1, N2, TL, LDTL, TR,
     $                   LDTR, B, LDB, SCALE, X, LDX, XNORM, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      LOGICAL            LTRANL, LTRANR
      INTEGER            INFO, ISGN, LDB, LDTL, LDTR, LDX, N1, N2
      REAL               SCALE, XNORM
*     ..
*     .. Array Arguments ..
      REAL               B( LDB, * ), TL( LDTL, * ), TR( LDTR, * ),
     $                   X( LDX, * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               TWO, HALF, EIGHT
      PARAMETER          ( TWO = 2.0E+0, HALF = 0.5E+0, EIGHT = 8.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            BSWAP, XSWAP
      INTEGER            I, IP, IPIV, IPSV, J, JP, JPSV, K
      REAL               BET, EPS, GAM, L21, SGN, SMIN, SMLNUM, TAU1,
     $                   TEMP, U11, U12, U22, XMAX
*     ..
*     .. Local Arrays ..
      LOGICAL            BSWPIV( 4 ), XSWPIV( 4 )
      INTEGER            JPIV( 4 ), LOCL21( 4 ), LOCU12( 4 ),
     $                   LOCU22( 4 )
      REAL               BTMP( 4 ), T16( 4, 4 ), TMP( 4 ), X2( 2 )
*     ..
*     .. External Functions ..
      INTEGER            ISAMAX
      REAL               SLAMCH
      EXTERNAL           ISAMAX, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SSWAP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
```

```
*     .. Data statements ..
      DATA               LOCU12 / 3, 4, 1, 2 / , LOCL21 / 2, 1, 4, 3 / ,
     $                   LOCU22 / 4, 3, 2, 1 /
      DATA               XSWPIV / .FALSE., .FALSE., .TRUE., .TRUE. /
      DATA               BSWPIV / .FALSE., .TRUE., .FALSE., .TRUE. /
*     ..
*     .. Executable Statements ..
*
*     Do not check the input parameters for errors
*
      INFO = 0
*
*     Quick return if possible
*
      IF( N1.EQ.0 .OR. N2.EQ.0 )
     $   RETURN
*
*     Set constants to control overflow
*
      EPS = SLAMCH( 'P' )
      SMLNUM = SLAMCH( 'S' ) / EPS
      SGN = ISGN
*
      K = N1 + N1 + N2 - 2
      GO TO ( 10, 20, 30, 50 )K
*
*     1 by 1: TL11*X + SGN*X*TR11 = B11
*
   10 CONTINUE
      TAU1 = TL( 1, 1 ) + SGN*TR( 1, 1 )
      BET = ABS( TAU1 )
      IF( BET.LE.SMLNUM ) THEN
         TAU1 = SMLNUM
         BET = SMLNUM
         INFO = 1
      END IF
*
      SCALE = ONE
      GAM = ABS( B( 1, 1 ) )
      IF( SMLNUM*GAM.GT.BET )
     $   SCALE = ONE / GAM
*
      X( 1, 1 ) = ( B( 1, 1 )*SCALE ) / TAU1
      XNORM = ABS( X( 1, 1 ) )
      RETURN
*
*     1 by 2:
*     TL11*[X11 X12] + ISGN*[X11 X12]*op[TR11 TR12]  = [B11 B12]
*                                       [TR21 TR22]
*
```

```
   20 CONTINUE
*
      SMIN = MAX( EPS*MAX( ABS( TL( 1, 1 ) ), ABS( TR( 1, 1 ) ),
     $       ABS( TR( 1, 2 ) ), ABS( TR( 2, 1 ) ), ABS( TR( 2, 2 ) ) ),
     $       SMLNUM )
      TMP( 1 ) = TL( 1, 1 ) + SGN*TR( 1, 1 )
      TMP( 4 ) = TL( 1, 1 ) + SGN*TR( 2, 2 )
      IF( LTRANR ) THEN
         TMP( 2 ) = SGN*TR( 2, 1 )
         TMP( 3 ) = SGN*TR( 1, 2 )
      ELSE
         TMP( 2 ) = SGN*TR( 1, 2 )
         TMP( 3 ) = SGN*TR( 2, 1 )
      END IF
      BTMP( 1 ) = B( 1, 1 )
      BTMP( 2 ) = B( 1, 2 )
      GO TO 40
*
*     2 by 1:
*          op[TL11 TL12]*[X11] + ISGN* [X11]*TR11  = [B11]
*            [TL21 TL22] [X21]         [X21]         [B21]
*
   30 CONTINUE
      SMIN = MAX( EPS*MAX( ABS( TR( 1, 1 ) ), ABS( TL( 1, 1 ) ),
     $       ABS( TL( 1, 2 ) ), ABS( TL( 2, 1 ) ), ABS( TL( 2, 2 ) ) ),
     $       SMLNUM )
      TMP( 1 ) = TL( 1, 1 ) + SGN*TR( 1, 1 )
      TMP( 4 ) = TL( 2, 2 ) + SGN*TR( 1, 1 )
      IF( LTRANL ) THEN
         TMP( 2 ) = TL( 1, 2 )
         TMP( 3 ) = TL( 2, 1 )
      ELSE
         TMP( 2 ) = TL( 2, 1 )
         TMP( 3 ) = TL( 1, 2 )
      END IF
      BTMP( 1 ) = B( 1, 1 )
      BTMP( 2 ) = B( 2, 1 )
   40 CONTINUE
*
*     Solve 2 by 2 system using complete pivoting.
*     Set pivots less than SMIN to SMIN.
*
      IPIV = ISAMAX( 4, TMP, 1 )
      U11 = TMP( IPIV )
      IF( ABS( U11 ).LE.SMIN ) THEN
         INFO = 1
         U11 = SMIN
      END IF
      U12 = TMP( LOCU12( IPIV ) )
      L21 = TMP( LOCL21( IPIV ) ) / U11
```

```
      U22 = TMP( LOCU22( IPIV ) ) - U12*L21
      XSWAP = XSWPIV( IPIV )
      BSWAP = BSWPIV( IPIV )
      IF( ABS( U22 ).LE.SMIN ) THEN
         INFO = 1
         U22 = SMIN
      END IF
      IF( BSWAP ) THEN
         TEMP = BTMP( 2 )
         BTMP( 2 ) = BTMP( 1 ) - L21*TEMP
         BTMP( 1 ) = TEMP
      ELSE
         BTMP( 2 ) = BTMP( 2 ) - L21*BTMP( 1 )
      END IF
      SCALE = ONE
      IF( ( TWO*SMLNUM )*ABS( BTMP( 2 ) ).GT.ABS( U22 ) .OR.
     $    ( TWO*SMLNUM )*ABS( BTMP( 1 ) ).GT.ABS( U11 ) ) THEN
         SCALE = HALF / MAX( ABS( BTMP( 1 ) ), ABS( BTMP( 2 ) ) )
         BTMP( 1 ) = BTMP( 1 )*SCALE
         BTMP( 2 ) = BTMP( 2 )*SCALE
      END IF
      X2( 2 ) = BTMP( 2 ) / U22
      X2( 1 ) = BTMP( 1 ) / U11 - ( U12 / U11 )*X2( 2 )
      IF( XSWAP ) THEN
         TEMP = X2( 2 )
         X2( 2 ) = X2( 1 )
         X2( 1 ) = TEMP
      END IF
      X( 1, 1 ) = X2( 1 )
      IF( N1.EQ.1 ) THEN
         X( 1, 2 ) = X2( 2 )
         XNORM = ABS( X( 1, 1 ) ) + ABS( X( 1, 2 ) )
      ELSE
         X( 2, 1 ) = X2( 2 )
         XNORM = MAX( ABS( X( 1, 1 ) ), ABS( X( 2, 1 ) ) )
      END IF
      RETURN
*
*     2 by 2:
*     op[TL11 TL12]*[X11 X12] +ISGN* [X11 X12]*op[TR11 TR12] = [B11 B12]
*       [TL21 TL22] [X21 X22]        [X21 X22]   [TR21 TR22]   [B21 B22]
*
*     Solve equivalent 4 by 4 system using complete pivoting.
*     Set pivots less than SMIN to SMIN.
*
   50 CONTINUE
      SMIN = MAX( ABS( TR( 1, 1 ) ), ABS( TR( 1, 2 ) ),
     $       ABS( TR( 2, 1 ) ), ABS( TR( 2, 2 ) ) )
      SMIN = MAX( SMIN, ABS( TL( 1, 1 ) ), ABS( TL( 1, 2 ) ),
     $       ABS( TL( 2, 1 ) ), ABS( TL( 2, 2 ) ) )
```

```
      SMIN = MAX( EPS*SMIN, SMLNUM )
      BTMP( 1 ) = ZERO
      CALL SCOPY( 16, BTMP, 0, T16, 1 )
      T16( 1, 1 ) = TL( 1, 1 ) + SGN*TR( 1, 1 )
      T16( 2, 2 ) = TL( 2, 2 ) + SGN*TR( 1, 1 )
      T16( 3, 3 ) = TL( 1, 1 ) + SGN*TR( 2, 2 )
      T16( 4, 4 ) = TL( 2, 2 ) + SGN*TR( 2, 2 )
      IF( LTRANL ) THEN
         T16( 1, 2 ) = TL( 2, 1 )
         T16( 2, 1 ) = TL( 1, 2 )
         T16( 3, 4 ) = TL( 2, 1 )
         T16( 4, 3 ) = TL( 1, 2 )
      ELSE
         T16( 1, 2 ) = TL( 1, 2 )
         T16( 2, 1 ) = TL( 2, 1 )
         T16( 3, 4 ) = TL( 1, 2 )
         T16( 4, 3 ) = TL( 2, 1 )
      END IF
      IF( LTRANR ) THEN
         T16( 1, 3 ) = SGN*TR( 1, 2 )
         T16( 2, 4 ) = SGN*TR( 1, 2 )
         T16( 3, 1 ) = SGN*TR( 2, 1 )
         T16( 4, 2 ) = SGN*TR( 2, 1 )
      ELSE
         T16( 1, 3 ) = SGN*TR( 2, 1 )
         T16( 2, 4 ) = SGN*TR( 2, 1 )
         T16( 3, 1 ) = SGN*TR( 1, 2 )
         T16( 4, 2 ) = SGN*TR( 1, 2 )
      END IF
      BTMP( 1 ) = B( 1, 1 )
      BTMP( 2 ) = B( 2, 1 )
      BTMP( 3 ) = B( 1, 2 )
      BTMP( 4 ) = B( 2, 2 )
*
*     Perform elimination
*
      DO 100 I = 1, 3
         XMAX = ZERO
         DO 70 IP = I, 4
            DO 60 JP = I, 4
               IF( ABS( T16( IP, JP ) ).GE.XMAX ) THEN
                  XMAX = ABS( T16( IP, JP ) )
                  IPSV = IP
                  JPSV = JP
               END IF
   60       CONTINUE
   70    CONTINUE
         IF( IPSV.NE.I ) THEN
            CALL SSWAP( 4, T16( IPSV, 1 ), 4, T16( I, 1 ), 4 )
            TEMP = BTMP( I )
```

```
         BTMP( I ) = BTMP( IPSV )
         BTMP( IPSV ) = TEMP
      END IF
      IF( JPSV.NE.I )
   $     CALL SSWAP( 4, T16( 1, JPSV ), 1, T16( 1, I ), 1 )
      JPIV( I ) = JPSV
      IF( ABS( T16( I, I ) ).LT.SMIN ) THEN
         INFO = 1
         T16( I, I ) = SMIN
      END IF
      DO 90 J = I + 1, 4
         T16( J, I ) = T16( J, I ) / T16( I, I )
         BTMP( J ) = BTMP( J ) - T16( J, I )*BTMP( I )
         DO 80 K = I + 1, 4
            T16( J, K ) = T16( J, K ) - T16( J, I )*T16( I, K )
 80      CONTINUE
 90   CONTINUE
100 CONTINUE
    IF( ABS( T16( 4, 4 ) ).LT.SMIN )
   $   T16( 4, 4 ) = SMIN
    SCALE = ONE
    IF( ( EIGHT*SMLNUM )*ABS( BTMP( 1 ) ).GT.ABS( T16( 1, 1 ) ) .OR.
   $    ( EIGHT*SMLNUM )*ABS( BTMP( 2 ) ).GT.ABS( T16( 2, 2 ) ) .OR.
   $    ( EIGHT*SMLNUM )*ABS( BTMP( 3 ) ).GT.ABS( T16( 3, 3 ) ) .OR.
   $    ( EIGHT*SMLNUM )*ABS( BTMP( 4 ) ).GT.ABS( T16( 4, 4 ) ) ) THEN
      SCALE = ( ONE / EIGHT ) / MAX( ABS( BTMP( 1 ) ),
   $           ABS( BTMP( 2 ) ), ABS( BTMP( 3 ) ), ABS( BTMP( 4 ) ) )
      BTMP( 1 ) = BTMP( 1 )*SCALE
      BTMP( 2 ) = BTMP( 2 )*SCALE
      BTMP( 3 ) = BTMP( 3 )*SCALE
      BTMP( 4 ) = BTMP( 4 )*SCALE
    END IF
    DO 120 I = 1, 4
      K = 5 - I
      TEMP = ONE / T16( K, K )
      TMP( K ) = BTMP( K )*TEMP
      DO 110 J = K + 1, 4
         TMP( K ) = TMP( K ) - ( TEMP*T16( K, J ) )*TMP( J )
110   CONTINUE
120 CONTINUE
    DO 130 I = 1, 3
      IF( JPIV( 4-I ).NE.4-I ) THEN
         TEMP = TMP( 4-I )
         TMP( 4-I ) = TMP( JPIV( 4-I ) )
         TMP( JPIV( 4-I ) ) = TEMP
      END IF
130 CONTINUE
    X( 1, 1 ) = TMP( 1 )
    X( 2, 1 ) = TMP( 2 )
    X( 1, 2 ) = TMP( 3 )
```

```
      X( 2, 2 ) = TMP( 4 )
      XNORM = MAX( ABS( TMP( 1 ) )+ABS( TMP( 3 ) ),
     $        ABS( TMP( 2 ) )+ABS( TMP( 4 ) ) )
      RETURN
*
*     End of SLASY2
*
      END
```

———-

63.3.4 lisp code

— **slasy2.lisp** —

```
(let* ((zero 0.0) (one 1.0) (two 2.0) (half 0.5) (eight 8.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 0.5 0.5) half)
           (type (single-float 8.0 8.0) eight)
           (ignorable zero one two half eight))
  (let ((bswpiv
         (make-array 4 :element-type 't :initial-contents '(nil t nil t)))
        (xswpiv
         (make-array 4 :element-type 't :initial-contents '(nil nil t t)))
        (locu22
         (make-array 4 :element-type 'f2cl-lib:integer4 :initial-contents
                     '(4 3 2 1)))
        (locl21
         (make-array 4 :element-type 'f2cl-lib:integer4 :initial-contents
                     '(2 1 4 3)))
        (locu12
         (make-array 4 :element-type 'f2cl-lib:integer4 :initial-contents
                     '(3 4 1 2))))
    (declare (type (array f2cl-lib:logical (4)) bswpiv xswpiv)
             (type (array f2cl-lib:integer4 (4)) locu22 locl21 locu12))
    (defun slasy2
           (ltranl ltranr isgn n1 n2 tl ldtl tr ldtr b ldb$ scale x ldx xnorm
            info)
      (declare (type (single-float) xnorm scale)
               (type (array single-float (*)) x b tr tl)
               (type (f2cl-lib:integer4) info ldx ldb$ ldtr ldtl n2 n1 isgn)
               (type f2cl-lib:logical ltranr ltranl))
      (f2cl-lib:with-multi-array-data
          ((tl single-float tl-%data% tl-%offset%)
           (tr single-float tr-%data% tr-%offset%)
           (b single-float b-%data% b-%offset%)
```

```
   (x single-float x-%data% x-%offset%))
 (prog ((btmp (make-array 4 :element-type 'single-float))
        (t16 (make-array 16 :element-type 'single-float))
        (tmp (make-array 4 :element-type 'single-float))
        (x2 (make-array 2 :element-type 'single-float))
        (jpiv (make-array 4 :element-type 'f2cl-lib:integer4)) (bet 0.0)
        (eps 0.0) (gam 0.0) (l21 0.0) (sgn 0.0) (smin 0.0) (smlnum 0.0)
        (tau1 0.0) (temp 0.0) (u11 0.0) (u12 0.0) (u22 0.0) (xmax 0.0)
        (i 0) (ip 0) (ipiv 0) (ipsv 0) (j 0) (jp 0) (jpsv 0) (k 0)
        (bswap nil) (xswap nil))
  (declare (type (array single-float (16)) t16)
           (type (array single-float (4)) btmp tmp)
           (type (array single-float (2)) x2)
           (type (array f2cl-lib:integer4 (4)) jpiv)
           (type (single-float) bet eps gam l21 sgn smin smlnum tau1
            temp u11 u12 u22 xmax)
           (type (f2cl-lib:integer4) i ip ipiv ipsv j jp jpsv k)
           (type f2cl-lib:logical bswap xswap))
  (setf info 0)
  (if (or (= n1 0) (= n2 0))
      (go end_label))
  (setf eps (slamch "P"))
  (setf smlnum (/ (slamch "S") eps))
  (setf sgn (coerce (the f2cl-lib:integer4 isgn) 'single-float))
  (setf k (f2cl-lib:int-sub (f2cl-lib:int-add n1 n1 n2) 2))
  (f2cl-lib:computed-goto (label10 label20 label30 label50) k)
 label10
  (setf tau1
          (+
           (f2cl-lib:fref tl-%data% (1 1) ((1 ldtl) (1 *)) tl-%offset%)
           (* sgn
              (f2cl-lib:fref tr-%data% (1 1) ((1 ldtr) (1 *))
                             tr-%offset%))))
  (setf bet (abs tau1))
  (cond
   ((<= bet smlnum) (setf tau1 smlnum) (setf bet smlnum)
    (setf info 1)))
  (setf scale one)
  (setf gam
          (abs
           (f2cl-lib:fref b-%data% (1 1) ((1 ldb$) (1 *)) b-%offset%)))
  (if (> (* smlnum gam) bet)
      (setf scale (/ one gam)))
  (setf (f2cl-lib:fref x-%data% (1 1) ((1 ldx) (1 *)) x-%offset%)
          (/
           (*
            (f2cl-lib:fref b-%data% (1 1) ((1 ldb$) (1 *)) b-%offset%)
            scale)
           tau1))
  (setf xnorm
```

```
        (abs
         (f2cl-lib:fref x-%data% (1 1) ((1 ldx) (1 *)) x-%offset%)))
 (go end_label)
label20
 (setf smin
        (max
         (* eps
            (max
             (abs
              (f2cl-lib:fref tl-%data% (1 1) ((1 ldtl) (1 *))
                             tl-%offset%))
             (abs
              (f2cl-lib:fref tr-%data% (1 1) ((1 ldtr) (1 *))
                             tr-%offset%))
             (abs
              (f2cl-lib:fref tr-%data% (1 2) ((1 ldtr) (1 *))
                             tr-%offset%))
             (abs
              (f2cl-lib:fref tr-%data% (2 1) ((1 ldtr) (1 *))
                             tr-%offset%))
             (abs
              (f2cl-lib:fref tr-%data% (2 2) ((1 ldtr) (1 *))
                             tr-%offset%))))
         smlnum))
 (setf (f2cl-lib:fref tmp (1) ((1 4)))
        (+
         (f2cl-lib:fref tl-%data% (1 1) ((1 ldtl) (1 *)) tl-%offset%)
         (* sgn
            (f2cl-lib:fref tr-%data% (1 1) ((1 ldtr) (1 *))
                           tr-%offset%))))
 (setf (f2cl-lib:fref tmp (4) ((1 4)))
        (+
         (f2cl-lib:fref tl-%data% (1 1) ((1 ldtl) (1 *)) tl-%offset%)
         (* sgn
            (f2cl-lib:fref tr-%data% (2 2) ((1 ldtr) (1 *))
                           tr-%offset%))))
 (cond
  (ltranr
   (setf (f2cl-lib:fref tmp (2) ((1 4)))
          (* sgn
             (f2cl-lib:fref tr-%data% (2 1) ((1 ldtr) (1 *))
                            tr-%offset%)))
   (setf (f2cl-lib:fref tmp (3) ((1 4)))
          (* sgn
             (f2cl-lib:fref tr-%data% (1 2) ((1 ldtr) (1 *))
                            tr-%offset%))))
  (t
   (setf (f2cl-lib:fref tmp (2) ((1 4)))
          (* sgn
             (f2cl-lib:fref tr-%data% (1 2) ((1 ldtr) (1 *))
```

```
                              tr-%offset%)))
    (setf (f2cl-lib:fref tmp (3) ((1 4)))
            (* sgn
               (f2cl-lib:fref tr-%data% (2 1) ((1 ldtr) (1 *))
                              tr-%offset%)))))
  (setf (f2cl-lib:fref btmp (1) ((1 4)))
          (f2cl-lib:fref b-%data% (1 1) ((1 ldb$) (1 *)) b-%offset%))
  (setf (f2cl-lib:fref btmp (2) ((1 4)))
          (f2cl-lib:fref b-%data% (1 2) ((1 ldb$) (1 *)) b-%offset%))
  (go label40)
 label30
  (setf smin
          (max
           (* eps
              (max
               (abs
                (f2cl-lib:fref tr-%data% (1 1) ((1 ldtr) (1 *))
                               tr-%offset%))
               (abs
                (f2cl-lib:fref tl-%data% (1 1) ((1 ldtl) (1 *))
                               tl-%offset%))
               (abs
                (f2cl-lib:fref tl-%data% (1 2) ((1 ldtl) (1 *))
                               tl-%offset%))
               (abs
                (f2cl-lib:fref tl-%data% (2 1) ((1 ldtl) (1 *))
                               tl-%offset%))
               (abs
                (f2cl-lib:fref tl-%data% (2 2) ((1 ldtl) (1 *))
                               tl-%offset%))))
           smlnum))
  (setf (f2cl-lib:fref tmp (1) ((1 4)))
          (+
           (f2cl-lib:fref tl-%data% (1 1) ((1 ldtl) (1 *)) tl-%offset%)
           (* sgn
              (f2cl-lib:fref tr-%data% (1 1) ((1 ldtr) (1 *))
                             tr-%offset%))))
  (setf (f2cl-lib:fref tmp (4) ((1 4)))
          (+
           (f2cl-lib:fref tl-%data% (2 2) ((1 ldtl) (1 *)) tl-%offset%)
           (* sgn
              (f2cl-lib:fref tr-%data% (1 1) ((1 ldtr) (1 *))
                             tr-%offset%))))
  (cond
   (ltranl
    (setf (f2cl-lib:fref tmp (2) ((1 4)))
            (f2cl-lib:fref tl-%data% (1 2) ((1 ldtl) (1 *))
                           tl-%offset%))
    (setf (f2cl-lib:fref tmp (3) ((1 4)))
            (f2cl-lib:fref tl-%data% (2 1) ((1 ldtl) (1 *))
```

```
                          tl-%offset%)))
  (t
   (setf (f2cl-lib:fref tmp (2) ((1 4)))
           (f2cl-lib:fref tl-%data% (2 1) ((1 ldtl) (1 *))
                          tl-%offset%))
   (setf (f2cl-lib:fref tmp (3) ((1 4)))
           (f2cl-lib:fref tl-%data% (1 2) ((1 ldtl) (1 *))
                          tl-%offset%))))
 (setf (f2cl-lib:fref btmp (1) ((1 4)))
         (f2cl-lib:fref b-%data% (1 1) ((1 ldb$) (1 *)) b-%offset%))
 (setf (f2cl-lib:fref btmp (2) ((1 4)))
         (f2cl-lib:fref b-%data% (2 1) ((1 ldb$) (1 *)) b-%offset%))
label40
 (setf ipiv (isamax 4 tmp 1))
 (setf u11 (f2cl-lib:fref tmp (ipiv) ((1 4))))
 (cond ((<= (abs u11) smin) (setf info 1) (setf u11 smin)))
 (setf u12
         (f2cl-lib:fref tmp ((f2cl-lib:fref locu12 (ipiv) ((1 4))))
                        ((1 4))))
 (setf l21
         (/
          (f2cl-lib:fref tmp ((f2cl-lib:fref locl21 (ipiv) ((1 4))))
                         ((1 4)))
          u11))
 (setf u22
         (-
          (f2cl-lib:fref tmp ((f2cl-lib:fref locu22 (ipiv) ((1 4))))
                         ((1 4)))
          (* u12 l21)))
 (setf xswap (f2cl-lib:fref xswpiv (ipiv) ((1 4))))
 (setf bswap (f2cl-lib:fref bswpiv (ipiv) ((1 4))))
 (cond ((<= (abs u22) smin) (setf info 1) (setf u22 smin)))
 (cond
  (bswap (setf temp (f2cl-lib:fref btmp (2) ((1 4))))
   (setf (f2cl-lib:fref btmp (2) ((1 4)))
           (- (f2cl-lib:fref btmp (1) ((1 4))) (* l21 temp)))
   (setf (f2cl-lib:fref btmp (1) ((1 4))) temp))
  (t
   (setf (f2cl-lib:fref btmp (2) ((1 4)))
           (- (f2cl-lib:fref btmp (2) ((1 4)))
              (* l21 (f2cl-lib:fref btmp (1) ((1 4)))))))
 (setf scale one)
 (cond
  ((or
    (> (* two smlnum (abs (f2cl-lib:fref btmp (2) ((1 4)))))
       (abs u22))
    (> (* two smlnum (abs (f2cl-lib:fref btmp (1) ((1 4)))))
       (abs u11)))
   (setf scale
           (/ half
```

```
                (max (abs (f2cl-lib:fref btmp (1) ((1 4))))
                     (abs (f2cl-lib:fref btmp (2) ((1 4)))))))
    (setf (f2cl-lib:fref btmp (1) ((1 4)))
            (* (f2cl-lib:fref btmp (1) ((1 4))) scale))
    (setf (f2cl-lib:fref btmp (2) ((1 4)))
            (* (f2cl-lib:fref btmp (2) ((1 4))) scale))))
  (setf (f2cl-lib:fref x2 (2) ((1 2)))
          (/ (f2cl-lib:fref btmp (2) ((1 4))) u22))
  (setf (f2cl-lib:fref x2 (1) ((1 2)))
          (- (/ (f2cl-lib:fref btmp (1) ((1 4))) u11)
             (* (/ u12 u11) (f2cl-lib:fref x2 (2) ((1 2))))))
  (cond
   (xswap (setf temp (f2cl-lib:fref x2 (2) ((1 2))))
    (setf (f2cl-lib:fref x2 (2) ((1 2)))
            (f2cl-lib:fref x2 (1) ((1 2))))
    (setf (f2cl-lib:fref x2 (1) ((1 2))) temp)))
  (setf (f2cl-lib:fref x-%data% (1 1) ((1 ldx) (1 *)) x-%offset%)
          (f2cl-lib:fref x2 (1) ((1 2))))
  (cond
   ((= n1 1)
    (setf (f2cl-lib:fref x-%data% (1 2) ((1 ldx) (1 *)) x-%offset%)
            (f2cl-lib:fref x2 (2) ((1 2))))
    (setf xnorm
            (+
             (abs
              (f2cl-lib:fref x-%data% (1 1) ((1 ldx) (1 *))
                             x-%offset%))
             (abs
              (f2cl-lib:fref x-%data% (1 2) ((1 ldx) (1 *))
                             x-%offset%)))))
   (t
    (setf (f2cl-lib:fref x-%data% (2 1) ((1 ldx) (1 *)) x-%offset%)
            (f2cl-lib:fref x2 (2) ((1 2))))
    (setf xnorm
            (max
             (abs
              (f2cl-lib:fref x-%data% (1 1) ((1 ldx) (1 *))
                             x-%offset%))
             (abs
              (f2cl-lib:fref x-%data% (2 1) ((1 ldx) (1 *))
                             x-%offset%))))))
  (go end_label)
 label50
  (setf smin
          (max
           (abs
            (f2cl-lib:fref tr-%data% (1 1) ((1 ldtr) (1 *))
                           tr-%offset%))
           (abs
            (f2cl-lib:fref tr-%data% (1 2) ((1 ldtr) (1 *))
```

```
                          tr-%offset%))
          (abs
           (f2cl-lib:fref tr-%data% (2 1) ((1 ldtr) (1 *))
                          tr-%offset%))
          (abs
           (f2cl-lib:fref tr-%data% (2 2) ((1 ldtr) (1 *))
                          tr-%offset%))))
(setf smin
        (max smin
             (abs
              (f2cl-lib:fref tl-%data% (1 1) ((1 ldtl) (1 *))
                             tl-%offset%))
             (abs
              (f2cl-lib:fref tl-%data% (1 2) ((1 ldtl) (1 *))
                             tl-%offset%))
             (abs
              (f2cl-lib:fref tl-%data% (2 1) ((1 ldtl) (1 *))
                             tl-%offset%))
             (abs
              (f2cl-lib:fref tl-%data% (2 2) ((1 ldtl) (1 *))
                             tl-%offset%))))
(setf smin (max (* eps smin) smlnum))
(setf (f2cl-lib:fref btmp (1) ((1 4))) zero)
(scopy 16 btmp 0 t16 1)
(setf (f2cl-lib:fref t16 (1 1) ((1 4) (1 4)))
        (+
         (f2cl-lib:fref tl-%data% (1 1) ((1 ldtl) (1 *)) tl-%offset%)
         (* sgn
            (f2cl-lib:fref tr-%data% (1 1) ((1 ldtr) (1 *))
                           tr-%offset%))))
(setf (f2cl-lib:fref t16 (2 2) ((1 4) (1 4)))
        (+
         (f2cl-lib:fref tl-%data% (2 2) ((1 ldtl) (1 *)) tl-%offset%)
         (* sgn
            (f2cl-lib:fref tr-%data% (1 1) ((1 ldtr) (1 *))
                           tr-%offset%))))
(setf (f2cl-lib:fref t16 (3 3) ((1 4) (1 4)))
        (+
         (f2cl-lib:fref tl-%data% (1 1) ((1 ldtl) (1 *)) tl-%offset%)
         (* sgn
            (f2cl-lib:fref tr-%data% (2 2) ((1 ldtr) (1 *))
                           tr-%offset%))))
(setf (f2cl-lib:fref t16 (4 4) ((1 4) (1 4)))
        (+
         (f2cl-lib:fref tl-%data% (2 2) ((1 ldtl) (1 *)) tl-%offset%)
         (* sgn
            (f2cl-lib:fref tr-%data% (2 2) ((1 ldtr) (1 *))
                           tr-%offset%))))
(cond
 (ltranl
```

```
    (setf (f2cl-lib:fref t16 (1 2) ((1 4) (1 4)))
            (f2cl-lib:fref tl-%data% (2 1) ((1 ldtl) (1 *))
                           tl-%offset%))
    (setf (f2cl-lib:fref t16 (2 1) ((1 4) (1 4)))
            (f2cl-lib:fref tl-%data% (1 2) ((1 ldtl) (1 *))
                           tl-%offset%))
    (setf (f2cl-lib:fref t16 (3 4) ((1 4) (1 4)))
            (f2cl-lib:fref tl-%data% (2 1) ((1 ldtl) (1 *))
                           tl-%offset%))
    (setf (f2cl-lib:fref t16 (4 3) ((1 4) (1 4)))
            (f2cl-lib:fref tl-%data% (1 2) ((1 ldtl) (1 *))
                           tl-%offset%)))
   (t
    (setf (f2cl-lib:fref t16 (1 2) ((1 4) (1 4)))
            (f2cl-lib:fref tl-%data% (1 2) ((1 ldtl) (1 *))
                           tl-%offset%))
    (setf (f2cl-lib:fref t16 (2 1) ((1 4) (1 4)))
            (f2cl-lib:fref tl-%data% (2 1) ((1 ldtl) (1 *))
                           tl-%offset%))
    (setf (f2cl-lib:fref t16 (3 4) ((1 4) (1 4)))
            (f2cl-lib:fref tl-%data% (1 2) ((1 ldtl) (1 *))
                           tl-%offset%))
    (setf (f2cl-lib:fref t16 (4 3) ((1 4) (1 4)))
            (f2cl-lib:fref tl-%data% (2 1) ((1 ldtl) (1 *))
                           tl-%offset%))))
  (cond
   (ltranr
    (setf (f2cl-lib:fref t16 (1 3) ((1 4) (1 4)))
            (* sgn
               (f2cl-lib:fref tr-%data% (1 2) ((1 ldtr) (1 *))
                              tr-%offset%)))
    (setf (f2cl-lib:fref t16 (2 4) ((1 4) (1 4)))
            (* sgn
               (f2cl-lib:fref tr-%data% (1 2) ((1 ldtr) (1 *))
                              tr-%offset%)))
    (setf (f2cl-lib:fref t16 (3 1) ((1 4) (1 4)))
            (* sgn
               (f2cl-lib:fref tr-%data% (2 1) ((1 ldtr) (1 *))
                              tr-%offset%)))
    (setf (f2cl-lib:fref t16 (4 2) ((1 4) (1 4)))
            (* sgn
               (f2cl-lib:fref tr-%data% (2 1) ((1 ldtr) (1 *))
                              tr-%offset%))))
   (t
    (setf (f2cl-lib:fref t16 (1 3) ((1 4) (1 4)))
            (* sgn
               (f2cl-lib:fref tr-%data% (2 1) ((1 ldtr) (1 *))
                              tr-%offset%)))
    (setf (f2cl-lib:fref t16 (2 4) ((1 4) (1 4)))
            (* sgn
```

```
                (f2cl-lib:fref tr-%data% (2 1) ((1 ldtr) (1 *))
                               tr-%offset%)))
  (setf (f2cl-lib:fref t16 (3 1) ((1 4) (1 4)))
          (* sgn
             (f2cl-lib:fref tr-%data% (1 2) ((1 ldtr) (1 *))
                               tr-%offset%)))
  (setf (f2cl-lib:fref t16 (4 2) ((1 4) (1 4)))
          (* sgn
             (f2cl-lib:fref tr-%data% (1 2) ((1 ldtr) (1 *))
                               tr-%offset%)))))
(setf (f2cl-lib:fref btmp (1) ((1 4)))
        (f2cl-lib:fref b-%data% (1 1) ((1 ldb$) (1 *)) b-%offset%))
(setf (f2cl-lib:fref btmp (2) ((1 4)))
        (f2cl-lib:fref b-%data% (2 1) ((1 ldb$) (1 *)) b-%offset%))
(setf (f2cl-lib:fref btmp (3) ((1 4)))
        (f2cl-lib:fref b-%data% (1 2) ((1 ldb$) (1 *)) b-%offset%))
(setf (f2cl-lib:fref btmp (4) ((1 4)))
        (f2cl-lib:fref b-%data% (2 2) ((1 ldb$) (1 *)) b-%offset%))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i 3) nil)
  (tagbody
    (setf xmax zero)
    (f2cl-lib:fdo (ip i (f2cl-lib:int-add ip 1))
                  ((> ip 4) nil)
      (tagbody
        (f2cl-lib:fdo (jp i (f2cl-lib:int-add jp 1))
                      ((> jp 4) nil)
          (tagbody
            (cond
             ((>= (abs (f2cl-lib:fref t16 (ip jp) ((1 4) (1 4))))
                  xmax)
              (setf xmax
                      (abs
                       (f2cl-lib:fref t16 (ip jp) ((1 4) (1 4)))))
              (setf ipsv ip) (setf jpsv jp)))
            label60))
       label70))
    (cond
     ((/= ipsv i)
      (sswap 4
       (f2cl-lib:array-slice t16 single-float (ipsv 1) ((1 4) (1 4)))
       4 (f2cl-lib:array-slice t16 single-float (i 1) ((1 4) (1 4)))
       4)
      (setf temp (f2cl-lib:fref btmp (i) ((1 4))))
      (setf (f2cl-lib:fref btmp (i) ((1 4)))
              (f2cl-lib:fref btmp (ipsv) ((1 4))))
      (setf (f2cl-lib:fref btmp (ipsv) ((1 4))) temp)))
    (if (/= jpsv i)
        (sswap 4
         (f2cl-lib:array-slice t16 single-float (1 jpsv)
```

```
                                ((1 4) (1 4)))
            1
            (f2cl-lib:array-slice t16 single-float (1 i) ((1 4) (1 4)))
            1))
      (setf (f2cl-lib:fref jpiv (i) ((1 4))) jpsv)
      (cond
       ((< (abs (f2cl-lib:fref t16 (i i) ((1 4) (1 4)))) smin)
        (setf info 1)
        (setf (f2cl-lib:fref t16 (i i) ((1 4) (1 4))) smin)))
      (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                    ((> j 4) nil)
        (tagbody
          (setf (f2cl-lib:fref t16 (j i) ((1 4) (1 4)))
                  (/ (f2cl-lib:fref t16 (j i) ((1 4) (1 4)))
                     (f2cl-lib:fref t16 (i i) ((1 4) (1 4)))))
          (setf (f2cl-lib:fref btmp (j) ((1 4)))
                  (- (f2cl-lib:fref btmp (j) ((1 4)))
                     (* (f2cl-lib:fref t16 (j i) ((1 4) (1 4)))
                        (f2cl-lib:fref btmp (i) ((1 4))))))
          (f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
                         (f2cl-lib:int-add k 1))
                        ((> k 4) nil)
            (tagbody
              (setf (f2cl-lib:fref t16 (j k) ((1 4) (1 4)))
                      (- (f2cl-lib:fref t16 (j k) ((1 4) (1 4)))
                         (* (f2cl-lib:fref t16 (j i) ((1 4) (1 4)))
                            (f2cl-lib:fref t16 (i k) ((1 4) (1 4))))))
             label80))
         label90))
     label100))
  (if (< (abs (f2cl-lib:fref t16 (4 4) ((1 4) (1 4)))) smin)
      (setf (f2cl-lib:fref t16 (4 4) ((1 4) (1 4))) smin))
  (setf scale one)
  (cond
   ((or
     (> (* eight smlnum (abs (f2cl-lib:fref btmp (1) ((1 4)))))
        (abs (f2cl-lib:fref t16 (1 1) ((1 4) (1 4)))))
     (> (* eight smlnum (abs (f2cl-lib:fref btmp (2) ((1 4)))))
        (abs (f2cl-lib:fref t16 (2 2) ((1 4) (1 4)))))
     (> (* eight smlnum (abs (f2cl-lib:fref btmp (3) ((1 4)))))
        (abs (f2cl-lib:fref t16 (3 3) ((1 4) (1 4)))))
     (> (* eight smlnum (abs (f2cl-lib:fref btmp (4) ((1 4)))))
        (abs (f2cl-lib:fref t16 (4 4) ((1 4) (1 4))))))
    (setf scale
            (/ (/ one eight)
               (max (abs (f2cl-lib:fref btmp (1) ((1 4))))
                    (abs (f2cl-lib:fref btmp (2) ((1 4))))
                    (abs (f2cl-lib:fref btmp (3) ((1 4))))
                    (abs (f2cl-lib:fref btmp (4) ((1 4)))))))
    (setf (f2cl-lib:fref btmp (1) ((1 4)))
```

```
          (* (f2cl-lib:fref btmp (1) ((1 4))) scale))
  (setf (f2cl-lib:fref btmp (2) ((1 4)))
          (* (f2cl-lib:fref btmp (2) ((1 4))) scale))
  (setf (f2cl-lib:fref btmp (3) ((1 4)))
          (* (f2cl-lib:fref btmp (3) ((1 4))) scale))
  (setf (f2cl-lib:fref btmp (4) ((1 4)))
          (* (f2cl-lib:fref btmp (4) ((1 4))) scale))))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i 4) nil)
  (tagbody
    (setf k (f2cl-lib:int-sub 5 i))
    (setf temp (/ one (f2cl-lib:fref t16 (k k) ((1 4) (1 4)))))
    (setf (f2cl-lib:fref tmp (k) ((1 4)))
            (* (f2cl-lib:fref btmp (k) ((1 4))) temp))
    (f2cl-lib:fdo (j (f2cl-lib:int-add k 1) (f2cl-lib:int-add j 1))
                  ((> j 4) nil)
      (tagbody
        (setf (f2cl-lib:fref tmp (k) ((1 4)))
                (- (f2cl-lib:fref tmp (k) ((1 4)))
                   (* temp (f2cl-lib:fref t16 (k j) ((1 4) (1 4)))
                      (f2cl-lib:fref tmp (j) ((1 4))))))
       label110))
   label120))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i 3) nil)
  (tagbody
    (cond
     ((/=
       (f2cl-lib:fref jpiv
                      ((f2cl-lib:int-add 4 (f2cl-lib:int-sub i)))
                      ((1 4)))
       (f2cl-lib:int-add 4 (f2cl-lib:int-sub i)))
      (setf temp
              (f2cl-lib:fref tmp ((f2cl-lib:int-sub 4 i)) ((1 4))))
      (setf (f2cl-lib:fref tmp ((f2cl-lib:int-sub 4 i)) ((1 4)))
              (f2cl-lib:fref tmp
                             ((f2cl-lib:fref jpiv
                                             ((f2cl-lib:int-sub 4 i))
                                             ((1 4))))
                             ((1 4))))
      (setf (f2cl-lib:fref tmp
                           ((f2cl-lib:fref jpiv
                                           ((f2cl-lib:int-sub 4 i))
                                           ((1 4))))
                           ((1 4)))
              temp)))
   label130))
(setf (f2cl-lib:fref x-%data% (1 1) ((1 ldx) (1 *)) x-%offset%)
        (f2cl-lib:fref tmp (1) ((1 4))))
(setf (f2cl-lib:fref x-%data% (2 1) ((1 ldx) (1 *)) x-%offset%)
```

```
                (f2cl-lib:fref tmp (2) ((1 4))))
          (setf (f2cl-lib:fref x-%data% (1 2) ((1 ldx) (1 *)) x-%offset%)
                (f2cl-lib:fref tmp (3) ((1 4))))
          (setf (f2cl-lib:fref x-%data% (2 2) ((1 ldx) (1 *)) x-%offset%)
                (f2cl-lib:fref tmp (4) ((1 4))))
          (setf xnorm
                (max
                 (+ (abs (f2cl-lib:fref tmp (1) ((1 4))))
                    (abs (f2cl-lib:fref tmp (3) ((1 4)))))
                 (+ (abs (f2cl-lib:fref tmp (2) ((1 4))))
                    (abs (f2cl-lib:fref tmp (4) ((1 4)))))))
          (go end_label)
         end_label
          (return
           (values nil nil nil nil nil nil nil nil nil nil nil scale nil nil
                   xnorm info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slasy2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '(fortran-to-lisp::logical
                                              fortran-to-lisp::logical
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
                                              nil nil nil
                                              fortran-to-lisp::scale nil nil
                                              fortran-to-lisp::xnorm
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

63.4 ssyswapr

63.4.1 Axiom unit tests

— **ssyswapr.input** —

```
)set break resume
)sys rm -f ssyswapr.output
)spool ssyswapr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

63.4.2 Axiom help page

— **ssyswapr.help** —

```
======================================================================
ssyswapr examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ssyswapr (UPLO, N, A, LDA, I1, I2)
           SSYSWAPR applies an elementary permutation on the rows and columns
           of a symmetric matrix.

Function/Subroutine Documentation
   subroutine ssyswapr (character UPLO, integer N, real, dimension( lda, n )
       A, integer LDA, integer I1, integer I2)
       SSYSWAPR applies an elementary permutation on the rows and columns of a
       symmetric matrix.

Purpose:
   SSYSWAPR applies an elementary permutation on the rows and the columns of
   a symmetric matrix.
```

```
Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the NB diagonal matrix D and the multipliers
   used to obtain the factor U or L as computed by SSYTRF.

   On exit, if INFO = 0, the (symmetric) inverse of the original
   matrix.  If UPLO = 'U', the upper triangular part of the
   inverse is formed and the part of A below the diagonal is not
   referenced; if UPLO = 'L' the lower triangular part of the
   inverse is formed and the part of A above the diagonal is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 I1
   I1 is INTEGER
   Index of the first row to swap

 I2
   I2 is INTEGER
   Index of the second row to swap
```

63.4.3 fortran code

— ssyswapr.f —

```
*  =====================================================================
      SUBROUTINE SSYSWAPR( UPLO, N, A, LDA, I1, I2)
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
```

```
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER        UPLO
      INTEGER          I1, I2, LDA, N
*     ..
*     .. Array Arguments ..
      REAL             A( LDA, N )
*
*  =====================================================================
*
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I
      REAL               TMP
*
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SSWAP
*     ..
*     .. Executable Statements ..
*
      UPPER = LSAME( UPLO, 'U' )
      IF (UPPER) THEN
*
*         UPPER
*         first swap
*          - swap column I1 and I2 from I1 to I1-1
         CALL SSWAP( I1-1, A(1,I1), 1, A(1,I2), 1 )
*
*          second swap :
*          - swap A(I1,I1) and A(I2,I2)
*          - swap row I1 from I1+1 to I2-1 with col I2 from I1+1 to I2-1
         TMP=A(I1,I1)
         A(I1,I1)=A(I2,I2)
         A(I2,I2)=TMP
*
         DO I=1,I2-I1-1
            TMP=A(I1,I1+I)
            A(I1,I1+I)=A(I1+I,I2)
            A(I1+I,I2)=TMP
         END DO
*
*          third swap
```

```
*          - swap row I1 and I2 from I2+1 to N
         DO I=I2+1,N
            TMP=A(I1,I)
            A(I1,I)=A(I2,I)
            A(I2,I)=TMP
         END DO
*
        ELSE
*
*         LOWER
*         first swap
*          - swap row I1 and I2 from I1 to I1-1
         CALL SSWAP( I1-1, A(I1,1), LDA, A(I2,1), LDA )
*
*         second swap :
*          - swap A(I1,I1) and A(I2,I2)
*          - swap col I1 from I1+1 to I2-1 with row I2 from I1+1 to I2-1
          TMP=A(I1,I1)
          A(I1,I1)=A(I2,I2)
          A(I2,I2)=TMP
*
          DO I=1,I2-I1-1
             TMP=A(I1+I,I1)
             A(I1+I,I1)=A(I2,I1+I)
             A(I2,I1+I)=TMP
          END DO
*
*         third swap
*          - swap col I1 and I2 from I2+1 to N
          DO I=I2+1,N
             TMP=A(I,I1)
             A(I,I1)=A(I,I2)
             A(I,I2)=TMP
          END DO
*
      ENDIF
      END SUBROUTINE SSYSWAPR
```

———-

63.4.4 lisp code

— **ssyswapr.lisp** —

———-

63.5 stgsy2

63.5.1 Axiom unit tests

— stgsy2.input —

```
)set break resume
)sys rm -f stgsy2.output
)spool stgsy2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

63.5.2 Axiom help page

— stgsy2.help —

```
======================================================================
stgsy2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine stgsy2 (TRANS, IJOB, M, N, A, LDA, B, LDB, C, LDC, D, LDD,
           E, LDE, F, LDF, SCALE, RDSUM, RDSCAL, IWORK, PQ, INFO)
           STGSY2 solves the generalized Sylvester equation (unblocked
           algorithm).

Function/Subroutine Documentation
   subroutine stgsy2 (character TRANS, integer IJOB, integer M, integer N,
       real, dimension( lda, * ) A, integer LDA, real, dimension( ldb, * ) B,
       integer LDB, real, dimension( ldc, * ) C, integer LDC, real, dimension(
       ldd, * ) D, integer LDD, real, dimension( lde, * ) E, integer LDE,
       real, dimension( ldf, * ) F, integer LDF, real SCALE, real RDSUM, real
       RDSCAL, integer, dimension( * ) IWORK, integer PQ, integer INFO)
       STGSY2 solves the generalized Sylvester equation (unblocked algorithm).
```

```
Purpose:
   STGSY2 solves the generalized Sylvester equation:

        A * R - L * B = scale * C                (1)
        D * R - L * E = scale * F,

   using Level 1 and 2 BLAS. where R and L are unknown M-by-N matrices,
   (A, D), (B, E) and (C, F) are given matrix pairs of size M-by-M,
   N-by-N and M-by-N, respectively, with real entries. (A, D) and (B, E)
   must be in generalized Schur canonical form, i.e. A, B are upper
   quasi triangular and D, E are upper triangular. The solution (R, L)
   overwrites (C, F). 0 <= SCALE <= 1 is an output scaling factor
   chosen to avoid overflow.

   In matrix notation solving equation (1) corresponds to solve
   Z*x = scale*b, where Z is defined as

        Z = [ kron(In, A)  -kron(B**T, Im) ]             (2)
            [ kron(In, D)  -kron(E**T, Im) ],

   Ik is the identity matrix of size k and X**T is the transpose of X.
   kron(X, Y) is the Kronecker product between the matrices X and Y.
   In the process of solving (1), we solve a number of such systems
   where Dim(In), Dim(In) = 1 or 2.

   If TRANS = 'T', solve the transposed system Z**T*y = scale*b for y,
   which is equivalent to solve for R and L in

        A**T * R  + D**T * L   = scale * C           (3)
        R  * B**T + L  * E**T  = scale * -F

   This case is used to compute an estimate of Dif[(A, D), (B, E)] =
   sigma_min(Z) using reverse communicaton with SLACON.

   STGSY2 also (IJOB >= 1) contributes to the computation in STGSYL
   of an upper bound on the separation between to matrix pairs. Then
   the input (A, D), (B, E) are sub-pencils of the matrix pair in
   STGSYL. See STGSYL for details.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   = 'N', solve the generalized Sylvester equation (1).
   = 'T': solve the 'transposed' system (3).

 IJOB
   IJOB is INTEGER
   Specifies what kind of functionality to be performed.
   = 0: solve (1) only.
```

```
  = 1: A contribution from this subsystem to a Frobenius
     norm-based estimate of the separation between two matrix
     pairs is computed. (look ahead strategy is used).
  = 2: A contribution from this subsystem to a Frobenius
     norm-based estimate of the separation between two matrix
    pairs is computed. (SGECON on sub-systems is used.)
  Not referenced if TRANS = 'T'.

M
  M is INTEGER
  On entry, M specifies the order of A and D, and the row
  dimension of C, F, R and L.

N
  N is INTEGER
  On entry, N specifies the order of B and E, and the column
  dimension of C, F, R and L.

A
  A is REAL array, dimension (LDA, M)
  On entry, A contains an upper quasi triangular matrix.

LDA
  LDA is INTEGER
  The leading dimension of the matrix A. LDA >= max(1, M).

B
  B is REAL array, dimension (LDB, N)
  On entry, B contains an upper quasi triangular matrix.

LDB
  LDB is INTEGER
  The leading dimension of the matrix B. LDB >= max(1, N).

C
  C is REAL array, dimension (LDC, N)
  On entry, C contains the right-hand-side of the first matrix
  equation in (1).
  On exit, if IJOB = 0, C has been overwritten by the
  solution R.

LDC
  LDC is INTEGER
  The leading dimension of the matrix C. LDC >= max(1, M).

D
  D is REAL array, dimension (LDD, M)
  On entry, D contains an upper triangular matrix.

LDD
```

```
  LDD is INTEGER
  The leading dimension of the matrix D. LDD >= max(1, M).

E
  E is REAL array, dimension (LDE, N)
  On entry, E contains an upper triangular matrix.

LDE
  LDE is INTEGER
  The leading dimension of the matrix E. LDE >= max(1, N).

F
  F is REAL array, dimension (LDF, N)
  On entry, F contains the right-hand-side of the second matrix
  equation in (1).
  On exit, if IJOB = 0, F has been overwritten by the solution L.

LDF
  LDF is INTEGER
  The leading dimension of the matrix F. LDF >= max(1, M).

SCALE
  SCALE is REAL
  On exit, 0 <= SCALE <= 1. If 0 < SCALE < 1, the solutions
  R and L (C and F on entry) will hold the solutions to a
  slightly perturbed system but the input matrices A, B, D and
  E have not been changed. If SCALE = 0, R and L will hold the
  solutions to the homogeneous system with C = F = 0. Normally, SCALE = 1.

RDSUM
  RDSUM is REAL
  On entry, the sum of squares of computed contributions to
  the Dif-estimate under computation by STGSYL, where the
  scaling factor RDSCAL (see below) has been factored out.
  On exit, the corresponding sum of squares updated with the
  contributions from the current sub-system.
  If TRANS = 'T' RDSUM is not touched.
  NOTE: RDSUM only makes sense when STGSY2 is called by STGSYL.

RDSCAL
  RDSCAL is REAL
  On entry, scaling factor used to prevent overflow in RDSUM.
  On exit, RDSCAL is updated w.r.t. the current contributions
  in RDSUM.
  If TRANS = 'T', RDSCAL is not touched.
  NOTE: RDSCAL only makes sense when STGSY2 is called by STGSYL.

IWORK
  IWORK is INTEGER array, dimension (M+N+2)
```

```
 PQ
   PQ is INTEGER
   On exit, the number of subsystems (of size 2-by-2, 4-by-4 and
   8-by-8) solved by this routine.

 INFO
   INFO is INTEGER
   On exit, if INFO is set to
   =0: Successful exit
   <0: If INFO = -i, the i-th argument had an illegal value.
   >0: The matrix pairs (A, D) and (B, E) have common or very
       close eigenvalues.
```

———-

63.5.3 fortran code

— **stgsy2.f** —

```
*  =====================================================================
      SUBROUTINE STGSY2( TRANS, IJOB, M, N, A, LDA, B, LDB, C, LDC, D,
     $                   LDD, E, LDE, F, LDF, SCALE, RDSUM, RDSCAL,
     $                   IWORK, PQ, INFO )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            IJOB, INFO, LDA, LDB, LDC, LDD, LDE, LDF, M, N,
     $                   PQ
      REAL               RDSCAL, RDSUM, SCALE
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      REAL               A( LDA, * ), B( LDB, * ), C( LDC, * ),
     $                   D( LDD, * ), E( LDE, * ), F( LDF, * )
*     ..
*
*  =====================================================================
*  Replaced various illegal calls to SCOPY by calls to SLASET.
*  Sven Hammarling, 27/5/02.
*
*     .. Parameters ..
      INTEGER            LDZ
```

```
      PARAMETER          ( LDZ = 8 )
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      INTEGER            I, IE, IERR, II, IS, ISP1, J, JE, JJ, JS, JSP1,
     $                   K, MB, NB, P, Q, ZDIM
      REAL               ALPHA, SCALOC
*     ..
*     .. Local Arrays ..
      INTEGER            IPIV( LDZ ), JPIV( LDZ )
      REAL               RHS( LDZ ), Z( LDZ, LDZ )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SAXPY, SCOPY, SGEMM, SGEMV, SGER, SGESC2,
     $                   SGETC2, SSCAL, SLASET, SLATDF, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Decode and test input parameters
*
      INFO = 0
      IERR = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) ) THEN
         INFO = -1
      ELSE IF( NOTRAN ) THEN
         IF( ( IJOB.LT.0 ) .OR. ( IJOB.GT.2 ) ) THEN
            INFO = -2
         END IF
      END IF
      IF( INFO.EQ.0 ) THEN
         IF( M.LE.0 ) THEN
            INFO = -3
         ELSE IF( N.LE.0 ) THEN
            INFO = -4
         ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
            INFO = -6
         ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
            INFO = -8
         ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
            INFO = -10
```

```
         ELSE IF( LDD.LT.MAX( 1, M ) ) THEN
            INFO = -12
         ELSE IF( LDE.LT.MAX( 1, N ) ) THEN
            INFO = -14
         ELSE IF( LDF.LT.MAX( 1, M ) ) THEN
            INFO = -16
         END IF
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'STGSY2', -INFO )
         RETURN
      END IF
*
*     Determine block structure of A
*
      PQ = 0
      P = 0
      I = 1
   10 CONTINUE
      IF( I.GT.M )
     $   GO TO 20
      P = P + 1
      IWORK( P ) = I
      IF( I.EQ.M )
     $   GO TO 20
      IF( A( I+1, I ).NE.ZERO ) THEN
         I = I + 2
      ELSE
         I = I + 1
      END IF
      GO TO 10
   20 CONTINUE
      IWORK( P+1 ) = M + 1
*
*     Determine block structure of B
*
      Q = P + 1
      J = 1
   30 CONTINUE
      IF( J.GT.N )
     $   GO TO 40
      Q = Q + 1
      IWORK( Q ) = J
      IF( J.EQ.N )
     $   GO TO 40
      IF( B( J+1, J ).NE.ZERO ) THEN
         J = J + 2
      ELSE
         J = J + 1
      END IF
```

```
      GO TO 30
   40 CONTINUE
      IWORK( Q+1 ) = N + 1
      PQ = P*( Q-P-1 )
*
      IF( NOTRAN ) THEN
*
*        Solve (I, J) - subsystem
*           A(I, I) * R(I, J) - L(I, J) * B(J, J) = C(I, J)
*           D(I, I) * R(I, J) - L(I, J) * E(J, J) = F(I, J)
*        for I = P, P - 1, ..., 1; J = 1, 2, ..., Q
*
         SCALE = ONE
         SCALOC = ONE
         DO 120 J = P + 2, Q
            JS = IWORK( J )
            JSP1 = JS + 1
            JE = IWORK( J+1 ) - 1
            NB = JE - JS + 1
            DO 110 I = P, 1, -1
*
               IS = IWORK( I )
               ISP1 = IS + 1
               IE = IWORK( I+1 ) - 1
               MB = IE - IS + 1
               ZDIM = MB*NB*2
*
               IF( ( MB.EQ.1 ) .AND. ( NB.EQ.1 ) ) THEN
*
*                Build a 2-by-2 system Z * x = RHS
*
                 Z( 1, 1 ) = A( IS, IS )
                 Z( 2, 1 ) = D( IS, IS )
                 Z( 1, 2 ) = -B( JS, JS )
                 Z( 2, 2 ) = -E( JS, JS )
*
*                Set up right hand side(s)
*
                 RHS( 1 ) = C( IS, JS )
                 RHS( 2 ) = F( IS, JS )
*
*                Solve Z * x = RHS
*
                 CALL SGETC2( ZDIM, Z, LDZ, IPIV, JPIV, IERR )
                 IF( IERR.GT.0 )
     $              INFO = IERR
*
                 IF( IJOB.EQ.0 ) THEN
                    CALL SGESC2( ZDIM, Z, LDZ, RHS, IPIV, JPIV,
     $                           SCALOC )
```

```
                  IF( SCALOC.NE.ONE ) THEN
                     DO 50 K = 1, N
                        CALL SSCAL( M, SCALOC, C( 1, K ), 1 )
                        CALL SSCAL( M, SCALOC, F( 1, K ), 1 )
   50                CONTINUE
                     SCALE = SCALE*SCALOC
                  END IF
               ELSE
                  CALL SLATDF( IJOB, ZDIM, Z, LDZ, RHS, RDSUM,
     $                          RDSCAL, IPIV, JPIV )
               END IF
*
*              Unpack solution vector(s)
*
               C( IS, JS ) = RHS( 1 )
               F( IS, JS ) = RHS( 2 )
*
*              Substitute R(I, J) and L(I, J) into remaining
*              equation.
*
               IF( I.GT.1 ) THEN
                  ALPHA = -RHS( 1 )
                  CALL SAXPY( IS-1, ALPHA, A( 1, IS ), 1, C( 1, JS ),
     $                        1 )
                  CALL SAXPY( IS-1, ALPHA, D( 1, IS ), 1, F( 1, JS ),
     $                        1 )
               END IF
               IF( J.LT.Q ) THEN
                  CALL SAXPY( N-JE, RHS( 2 ), B( JS, JE+1 ), LDB,
     $                        C( IS, JE+1 ), LDC )
                  CALL SAXPY( N-JE, RHS( 2 ), E( JS, JE+1 ), LDE,
     $                        F( IS, JE+1 ), LDF )
               END IF
*
            ELSE IF( ( MB.EQ.1 ) .AND. ( NB.EQ.2 ) ) THEN
*
*              Build a 4-by-4 system Z * x = RHS
*
               Z( 1, 1 ) = A( IS, IS )
               Z( 2, 1 ) = ZERO
               Z( 3, 1 ) = D( IS, IS )
               Z( 4, 1 ) = ZERO
*
               Z( 1, 2 ) = ZERO
               Z( 2, 2 ) = A( IS, IS )
               Z( 3, 2 ) = ZERO
               Z( 4, 2 ) = D( IS, IS )
*
               Z( 1, 3 ) = -B( JS, JS )
               Z( 2, 3 ) = -B( JS, JSP1 )
```

```
                 Z( 3, 3 ) = -E( JS, JS )
                 Z( 4, 3 ) = -E( JS, JSP1 )
*
                 Z( 1, 4 ) = -B( JSP1, JS )
                 Z( 2, 4 ) = -B( JSP1, JSP1 )
                 Z( 3, 4 ) = ZERO
                 Z( 4, 4 ) = -E( JSP1, JSP1 )
*
*                Set up right hand side(s)
*
                 RHS( 1 ) = C( IS, JS )
                 RHS( 2 ) = C( IS, JSP1 )
                 RHS( 3 ) = F( IS, JS )
                 RHS( 4 ) = F( IS, JSP1 )
*
*                Solve Z * x = RHS
*
                 CALL SGETC2( ZDIM, Z, LDZ, IPIV, JPIV, IERR )
                 IF( IERR.GT.0 )
     $              INFO = IERR
*
                 IF( IJOB.EQ.0 ) THEN
                    CALL SGESC2( ZDIM, Z, LDZ, RHS, IPIV, JPIV,
     $                           SCALOC )
                    IF( SCALOC.NE.ONE ) THEN
                       DO 60 K = 1, N
                          CALL SSCAL( M, SCALOC, C( 1, K ), 1 )
                          CALL SSCAL( M, SCALOC, F( 1, K ), 1 )
   60                  CONTINUE
                       SCALE = SCALE*SCALOC
                    END IF
                 ELSE
                    CALL SLATDF( IJOB, ZDIM, Z, LDZ, RHS, RDSUM,
     $                           RDSCAL, IPIV, JPIV )
                 END IF
*
*                Unpack solution vector(s)
*
                 C( IS, JS ) = RHS( 1 )
                 C( IS, JSP1 ) = RHS( 2 )
                 F( IS, JS ) = RHS( 3 )
                 F( IS, JSP1 ) = RHS( 4 )
*
*                Substitute R(I, J) and L(I, J) into remaining
*                equation.
*
                 IF( I.GT.1 ) THEN
                    CALL SGER( IS-1, NB, -ONE, A( 1, IS ), 1, RHS( 1 ),
     $                         1, C( 1, JS ), LDC )
                    CALL SGER( IS-1, NB, -ONE, D( 1, IS ), 1, RHS( 1 ),
```

```
     $                         1, F( 1, JS ), LDF )
                 END IF
                 IF( J.LT.Q ) THEN
                    CALL SAXPY( N-JE, RHS( 3 ), B( JS, JE+1 ), LDB,
     $                          C( IS, JE+1 ), LDC )
                    CALL SAXPY( N-JE, RHS( 3 ), E( JS, JE+1 ), LDE,
     $                          F( IS, JE+1 ), LDF )
                    CALL SAXPY( N-JE, RHS( 4 ), B( JSP1, JE+1 ), LDB,
     $                          C( IS, JE+1 ), LDC )
                    CALL SAXPY( N-JE, RHS( 4 ), E( JSP1, JE+1 ), LDE,
     $                          F( IS, JE+1 ), LDF )
                 END IF
*
              ELSE IF( ( MB.EQ.2 ) .AND. ( NB.EQ.1 ) ) THEN
*
*                Build a 4-by-4 system Z * x = RHS
*
                 Z( 1, 1 ) = A( IS, IS )
                 Z( 2, 1 ) = A( ISP1, IS )
                 Z( 3, 1 ) = D( IS, IS )
                 Z( 4, 1 ) = ZERO
*
                 Z( 1, 2 ) = A( IS, ISP1 )
                 Z( 2, 2 ) = A( ISP1, ISP1 )
                 Z( 3, 2 ) = D( IS, ISP1 )
                 Z( 4, 2 ) = D( ISP1, ISP1 )
*
                 Z( 1, 3 ) = -B( JS, JS )
                 Z( 2, 3 ) = ZERO
                 Z( 3, 3 ) = -E( JS, JS )
                 Z( 4, 3 ) = ZERO
*
                 Z( 1, 4 ) = ZERO
                 Z( 2, 4 ) = -B( JS, JS )
                 Z( 3, 4 ) = ZERO
                 Z( 4, 4 ) = -E( JS, JS )
*
*                Set up right hand side(s)
*
                 RHS( 1 ) = C( IS, JS )
                 RHS( 2 ) = C( ISP1, JS )
                 RHS( 3 ) = F( IS, JS )
                 RHS( 4 ) = F( ISP1, JS )
*
*                Solve Z * x = RHS
*
                 CALL SGETC2( ZDIM, Z, LDZ, IPIV, JPIV, IERR )
                 IF( IERR.GT.0 )
     $              INFO = IERR
                 IF( IJOB.EQ.0 ) THEN
```

```
                 CALL SGESC2( ZDIM, Z, LDZ, RHS, IPIV, JPIV,
     $                        SCALOC )
                 IF( SCALOC.NE.ONE ) THEN
                    DO 70 K = 1, N
                       CALL SSCAL( M, SCALOC, C( 1, K ), 1 )
                       CALL SSCAL( M, SCALOC, F( 1, K ), 1 )
   70               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
              ELSE
                 CALL SLATDF( IJOB, ZDIM, Z, LDZ, RHS, RDSUM,
     $                        RDSCAL, IPIV, JPIV )
              END IF
*
*             Unpack solution vector(s)
*
              C( IS, JS ) = RHS( 1 )
              C( ISP1, JS ) = RHS( 2 )
              F( IS, JS ) = RHS( 3 )
              F( ISP1, JS ) = RHS( 4 )
*
*             Substitute R(I, J) and L(I, J) into remaining
*             equation.
*
              IF( I.GT.1 ) THEN
                 CALL SGEMV( 'N', IS-1, MB, -ONE, A( 1, IS ), LDA,
     $                       RHS( 1 ), 1, ONE, C( 1, JS ), 1 )
                 CALL SGEMV( 'N', IS-1, MB, -ONE, D( 1, IS ), LDD,
     $                       RHS( 1 ), 1, ONE, F( 1, JS ), 1 )
              END IF
              IF( J.LT.Q ) THEN
                 CALL SGER( MB, N-JE, ONE, RHS( 3 ), 1,
     $                      B( JS, JE+1 ), LDB, C( IS, JE+1 ), LDC )
                 CALL SGER( MB, N-JE, ONE, RHS( 3 ), 1,
     $                      E( JS, JE+1 ), LDE, F( IS, JE+1 ), LDF )
              END IF
*
           ELSE IF( ( MB.EQ.2 ) .AND. ( NB.EQ.2 ) ) THEN
*
*             Build an 8-by-8 system Z * x = RHS
*
              CALL SLASET( 'F', LDZ, LDZ, ZERO, ZERO, Z, LDZ )
*
              Z( 1, 1 ) = A( IS, IS )
              Z( 2, 1 ) = A( ISP1, IS )
              Z( 5, 1 ) = D( IS, IS )
*
              Z( 1, 2 ) = A( IS, ISP1 )
              Z( 2, 2 ) = A( ISP1, ISP1 )
              Z( 5, 2 ) = D( IS, ISP1 )
```

```
                 Z( 6, 2 ) = D( ISP1, ISP1 )
*
                 Z( 3, 3 ) = A( IS, IS )
                 Z( 4, 3 ) = A( ISP1, IS )
                 Z( 7, 3 ) = D( IS, IS )
*
                 Z( 3, 4 ) = A( IS, ISP1 )
                 Z( 4, 4 ) = A( ISP1, ISP1 )
                 Z( 7, 4 ) = D( IS, ISP1 )
                 Z( 8, 4 ) = D( ISP1, ISP1 )
*
                 Z( 1, 5 ) = -B( JS, JS )
                 Z( 3, 5 ) = -B( JS, JSP1 )
                 Z( 5, 5 ) = -E( JS, JS )
                 Z( 7, 5 ) = -E( JS, JSP1 )
*
                 Z( 2, 6 ) = -B( JS, JS )
                 Z( 4, 6 ) = -B( JS, JSP1 )
                 Z( 6, 6 ) = -E( JS, JS )
                 Z( 8, 6 ) = -E( JS, JSP1 )
*
                 Z( 1, 7 ) = -B( JSP1, JS )
                 Z( 3, 7 ) = -B( JSP1, JSP1 )
                 Z( 7, 7 ) = -E( JSP1, JSP1 )
*
                 Z( 2, 8 ) = -B( JSP1, JS )
                 Z( 4, 8 ) = -B( JSP1, JSP1 )
                 Z( 8, 8 ) = -E( JSP1, JSP1 )
*
*                Set up right hand side(s)
*
                 K = 1
                 II = MB*NB + 1
                 DO 80 JJ = 0, NB - 1
                    CALL SCOPY( MB, C( IS, JS+JJ ), 1, RHS( K ), 1 )
                    CALL SCOPY( MB, F( IS, JS+JJ ), 1, RHS( II ), 1 )
                    K = K + MB
                    II = II + MB
   80            CONTINUE
*
*                Solve Z * x = RHS
*
                 CALL SGETC2( ZDIM, Z, LDZ, IPIV, JPIV, IERR )
                 IF( IERR.GT.0 )
     $              INFO = IERR
                 IF( IJOB.EQ.0 ) THEN
                    CALL SGESC2( ZDIM, Z, LDZ, RHS, IPIV, JPIV,
     $                           SCALOC )
                    IF( SCALOC.NE.ONE ) THEN
                       DO 90 K = 1, N
```

```
                        CALL SSCAL( M, SCALOC, C( 1, K ), 1 )
                        CALL SSCAL( M, SCALOC, F( 1, K ), 1 )
   90                 CONTINUE
                      SCALE = SCALE*SCALOC
                   END IF
                ELSE
                   CALL SLATDF( IJOB, ZDIM, Z, LDZ, RHS, RDSUM,
     $                          RDSCAL, IPIV, JPIV )
                END IF
*
*               Unpack solution vector(s)
*
                K = 1
                II = MB*NB + 1
                DO 100 JJ = 0, NB - 1
                   CALL SCOPY( MB, RHS( K ), 1, C( IS, JS+JJ ), 1 )
                   CALL SCOPY( MB, RHS( II ), 1, F( IS, JS+JJ ), 1 )
                   K = K + MB
                   II = II + MB
  100           CONTINUE
*
*               Substitute R(I, J) and L(I, J) into remaining
*               equation.
*
                IF( I.GT.1 ) THEN
                   CALL SGEMM( 'N', 'N', IS-1, NB, MB, -ONE,
     $                         A( 1, IS ), LDA, RHS( 1 ), MB, ONE,
     $                         C( 1, JS ), LDC )
                   CALL SGEMM( 'N', 'N', IS-1, NB, MB, -ONE,
     $                         D( 1, IS ), LDD, RHS( 1 ), MB, ONE,
     $                         F( 1, JS ), LDF )
                END IF
                IF( J.LT.Q ) THEN
                   K = MB*NB + 1
                   CALL SGEMM( 'N', 'N', MB, N-JE, NB, ONE, RHS( K ),
     $                         MB, B( JS, JE+1 ), LDB, ONE,
     $                         C( IS, JE+1 ), LDC )
                   CALL SGEMM( 'N', 'N', MB, N-JE, NB, ONE, RHS( K ),
     $                         MB, E( JS, JE+1 ), LDE, ONE,
     $                         F( IS, JE+1 ), LDF )
                END IF
*
             END IF
*
  110       CONTINUE
  120    CONTINUE
      ELSE
*
*       Solve (I, J) - subsystem
*            A(I, I)**T * R(I, J) + D(I, I)**T * L(J, J)  =  C(I, J)
```

```
*            R(I, I)  * B(J, J) + L(I, J)  * E(J, J)  = -F(I, J)
*       for I = 1, 2, ..., P, J = Q, Q - 1, ..., 1
*
        SCALE = ONE
        SCALOC = ONE
        DO 200 I = 1, P
*
           IS = IWORK( I )
           ISP1 = IS + 1
           IE = IWORK( I+1 ) - 1
           MB = IE - IS + 1
           DO 190 J = Q, P + 2, -1
*
              JS = IWORK( J )
              JSP1 = JS + 1
              JE = IWORK( J+1 ) - 1
              NB = JE - JS + 1
              ZDIM = MB*NB*2
              IF( ( MB.EQ.1 ) .AND. ( NB.EQ.1 ) ) THEN
*
*                Build a 2-by-2 system Z**T * x = RHS
*
                 Z( 1, 1 ) = A( IS, IS )
                 Z( 2, 1 ) = -B( JS, JS )
                 Z( 1, 2 ) = D( IS, IS )
                 Z( 2, 2 ) = -E( JS, JS )
*
*                Set up right hand side(s)
*
                 RHS( 1 ) = C( IS, JS )
                 RHS( 2 ) = F( IS, JS )
*
*                Solve Z**T * x = RHS
*
                 CALL SGETC2( ZDIM, Z, LDZ, IPIV, JPIV, IERR )
                 IF( IERR.GT.0 )
     $              INFO = IERR
*
                 CALL SGESC2( ZDIM, Z, LDZ, RHS, IPIV, JPIV, SCALOC )
                 IF( SCALOC.NE.ONE ) THEN
                    DO 130 K = 1, N
                       CALL SSCAL( M, SCALOC, C( 1, K ), 1 )
                       CALL SSCAL( M, SCALOC, F( 1, K ), 1 )
  130               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
*
*                Unpack solution vector(s)
*
                 C( IS, JS ) = RHS( 1 )
```

```
                 F( IS, JS ) = RHS( 2 )
*
*                Substitute R(I, J) and L(I, J) into remaining
*                equation.
*
                 IF( J.GT.P+2 ) THEN
                    ALPHA = RHS( 1 )
                    CALL SAXPY( JS-1, ALPHA, B( 1, JS ), 1, F( IS, 1 ),
     $                          LDF )
                    ALPHA = RHS( 2 )
                    CALL SAXPY( JS-1, ALPHA, E( 1, JS ), 1, F( IS, 1 ),
     $                          LDF )
                 END IF
                 IF( I.LT.P ) THEN
                    ALPHA = -RHS( 1 )
                    CALL SAXPY( M-IE, ALPHA, A( IS, IE+1 ), LDA,
     $                          C( IE+1, JS ), 1 )
                    ALPHA = -RHS( 2 )
                    CALL SAXPY( M-IE, ALPHA, D( IS, IE+1 ), LDD,
     $                          C( IE+1, JS ), 1 )
                 END IF
*
              ELSE IF( ( MB.EQ.1 ) .AND. ( NB.EQ.2 ) ) THEN
*
*                Build a 4-by-4 system Z**T * x = RHS
*
                 Z( 1, 1 ) = A( IS, IS )
                 Z( 2, 1 ) = ZERO
                 Z( 3, 1 ) = -B( JS, JS )
                 Z( 4, 1 ) = -B( JSP1, JS )
*
                 Z( 1, 2 ) = ZERO
                 Z( 2, 2 ) = A( IS, IS )
                 Z( 3, 2 ) = -B( JS, JSP1 )
                 Z( 4, 2 ) = -B( JSP1, JSP1 )
*
                 Z( 1, 3 ) = D( IS, IS )
                 Z( 2, 3 ) = ZERO
                 Z( 3, 3 ) = -E( JS, JS )
                 Z( 4, 3 ) = ZERO
*
                 Z( 1, 4 ) = ZERO
                 Z( 2, 4 ) = D( IS, IS )
                 Z( 3, 4 ) = -E( JS, JSP1 )
                 Z( 4, 4 ) = -E( JSP1, JSP1 )
*
*                Set up right hand side(s)
*
                 RHS( 1 ) = C( IS, JS )
                 RHS( 2 ) = C( IS, JSP1 )
```

```
                 RHS( 3 ) = F( IS, JS )
                 RHS( 4 ) = F( IS, JSP1 )
*
*                Solve Z**T * x = RHS
*
                 CALL SGETC2( ZDIM, Z, LDZ, IPIV, JPIV, IERR )
                 IF( IERR.GT.0 )
     $              INFO = IERR
                 CALL SGESC2( ZDIM, Z, LDZ, RHS, IPIV, JPIV, SCALOC )
                 IF( SCALOC.NE.ONE ) THEN
                    DO 140 K = 1, N
                       CALL SSCAL( M, SCALOC, C( 1, K ), 1 )
                       CALL SSCAL( M, SCALOC, F( 1, K ), 1 )
  140               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
*
*                Unpack solution vector(s)
*
                 C( IS, JS ) = RHS( 1 )
                 C( IS, JSP1 ) = RHS( 2 )
                 F( IS, JS ) = RHS( 3 )
                 F( IS, JSP1 ) = RHS( 4 )
*
*                Substitute R(I, J) and L(I, J) into remaining
*                equation.
*
                 IF( J.GT.P+2 ) THEN
                    CALL SAXPY( JS-1, RHS( 1 ), B( 1, JS ), 1,
     $                          F( IS, 1 ), LDF )
                    CALL SAXPY( JS-1, RHS( 2 ), B( 1, JSP1 ), 1,
     $                          F( IS, 1 ), LDF )
                    CALL SAXPY( JS-1, RHS( 3 ), E( 1, JS ), 1,
     $                          F( IS, 1 ), LDF )
                    CALL SAXPY( JS-1, RHS( 4 ), E( 1, JSP1 ), 1,
     $                          F( IS, 1 ), LDF )
                 END IF
                 IF( I.LT.P ) THEN
                    CALL SGER( M-IE, NB, -ONE, A( IS, IE+1 ), LDA,
     $                         RHS( 1 ), 1, C( IE+1, JS ), LDC )
                    CALL SGER( M-IE, NB, -ONE, D( IS, IE+1 ), LDD,
     $                         RHS( 3 ), 1, C( IE+1, JS ), LDC )
                 END IF
*
              ELSE IF( ( MB.EQ.2 ) .AND. ( NB.EQ.1 ) ) THEN
*
*                Build a 4-by-4 system Z**T * x = RHS
*
                 Z( 1, 1 ) = A( IS, IS )
                 Z( 2, 1 ) = A( IS, ISP1 )
```

```
                 Z( 3, 1 ) = -B( JS, JS )
                 Z( 4, 1 ) = ZERO
*
                 Z( 1, 2 ) = A( ISP1, IS )
                 Z( 2, 2 ) = A( ISP1, ISP1 )
                 Z( 3, 2 ) = ZERO
                 Z( 4, 2 ) = -B( JS, JS )
*
                 Z( 1, 3 ) = D( IS, IS )
                 Z( 2, 3 ) = D( IS, ISP1 )
                 Z( 3, 3 ) = -E( JS, JS )
                 Z( 4, 3 ) = ZERO
*
                 Z( 1, 4 ) = ZERO
                 Z( 2, 4 ) = D( ISP1, ISP1 )
                 Z( 3, 4 ) = ZERO
                 Z( 4, 4 ) = -E( JS, JS )
*
*                Set up right hand side(s)
*
                 RHS( 1 ) = C( IS, JS )
                 RHS( 2 ) = C( ISP1, JS )
                 RHS( 3 ) = F( IS, JS )
                 RHS( 4 ) = F( ISP1, JS )
*
*                Solve Z**T * x = RHS
*
                 CALL SGETC2( ZDIM, Z, LDZ, IPIV, JPIV, IERR )
                 IF( IERR.GT.0 )
     $              INFO = IERR
*
                 CALL SGESC2( ZDIM, Z, LDZ, RHS, IPIV, JPIV, SCALOC )
                 IF( SCALOC.NE.ONE ) THEN
                    DO 150 K = 1, N
                       CALL SSCAL( M, SCALOC, C( 1, K ), 1 )
                       CALL SSCAL( M, SCALOC, F( 1, K ), 1 )
  150               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
*
*                Unpack solution vector(s)
*
                 C( IS, JS ) = RHS( 1 )
                 C( ISP1, JS ) = RHS( 2 )
                 F( IS, JS ) = RHS( 3 )
                 F( ISP1, JS ) = RHS( 4 )
*
*                Substitute R(I, J) and L(I, J) into remaining
*                equation.
*
```

```
                 IF( J.GT.P+2 ) THEN
                    CALL SGER( MB, JS-1, ONE, RHS( 1 ), 1, B( 1, JS ),
     $                         1, F( IS, 1 ), LDF )
                    CALL SGER( MB, JS-1, ONE, RHS( 3 ), 1, E( 1, JS ),
     $                         1, F( IS, 1 ), LDF )
                 END IF
                 IF( I.LT.P ) THEN
                    CALL SGEMV( 'T', MB, M-IE, -ONE, A( IS, IE+1 ),
     $                          LDA, RHS( 1 ), 1, ONE, C( IE+1, JS ),
     $                          1 )
                    CALL SGEMV( 'T', MB, M-IE, -ONE, D( IS, IE+1 ),
     $                          LDD, RHS( 3 ), 1, ONE, C( IE+1, JS ),
     $                          1 )
                 END IF
*
              ELSE IF( ( MB.EQ.2 ) .AND. ( NB.EQ.2 ) ) THEN
*
*                Build an 8-by-8 system Z**T * x = RHS
*
                 CALL SLASET( 'F', LDZ, LDZ, ZERO, ZERO, Z, LDZ )
*
                 Z( 1, 1 ) = A( IS, IS )
                 Z( 2, 1 ) = A( IS, ISP1 )
                 Z( 5, 1 ) = -B( JS, JS )
                 Z( 7, 1 ) = -B( JSP1, JS )
*
                 Z( 1, 2 ) = A( ISP1, IS )
                 Z( 2, 2 ) = A( ISP1, ISP1 )
                 Z( 6, 2 ) = -B( JS, JS )
                 Z( 8, 2 ) = -B( JSP1, JS )
*
                 Z( 3, 3 ) = A( IS, IS )
                 Z( 4, 3 ) = A( IS, ISP1 )
                 Z( 5, 3 ) = -B( JS, JSP1 )
                 Z( 7, 3 ) = -B( JSP1, JSP1 )
*
                 Z( 3, 4 ) = A( ISP1, IS )
                 Z( 4, 4 ) = A( ISP1, ISP1 )
                 Z( 6, 4 ) = -B( JS, JSP1 )
                 Z( 8, 4 ) = -B( JSP1, JSP1 )
*
                 Z( 1, 5 ) = D( IS, IS )
                 Z( 2, 5 ) = D( IS, ISP1 )
                 Z( 5, 5 ) = -E( JS, JS )
*
                 Z( 2, 6 ) = D( ISP1, ISP1 )
                 Z( 6, 6 ) = -E( JS, JS )
*
                 Z( 3, 7 ) = D( IS, IS )
                 Z( 4, 7 ) = D( IS, ISP1 )
```

```
                 Z( 5, 7 ) = -E( JS, JSP1 )
                 Z( 7, 7 ) = -E( JSP1, JSP1 )
*
                 Z( 4, 8 ) = D( ISP1, ISP1 )
                 Z( 6, 8 ) = -E( JS, JSP1 )
                 Z( 8, 8 ) = -E( JSP1, JSP1 )
*
*                Set up right hand side(s)
*
                 K = 1
                 II = MB*NB + 1
                 DO 160 JJ = 0, NB - 1
                    CALL SCOPY( MB, C( IS, JS+JJ ), 1, RHS( K ), 1 )
                    CALL SCOPY( MB, F( IS, JS+JJ ), 1, RHS( II ), 1 )
                    K = K + MB
                    II = II + MB
  160            CONTINUE
*
*
*                Solve Z**T * x = RHS
*
                 CALL SGETC2( ZDIM, Z, LDZ, IPIV, JPIV, IERR )
                 IF( IERR.GT.0 )
     $              INFO = IERR
*
                 CALL SGESC2( ZDIM, Z, LDZ, RHS, IPIV, JPIV, SCALOC )
                 IF( SCALOC.NE.ONE ) THEN
                    DO 170 K = 1, N
                       CALL SSCAL( M, SCALOC, C( 1, K ), 1 )
                       CALL SSCAL( M, SCALOC, F( 1, K ), 1 )
  170               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
*
*                Unpack solution vector(s)
*
                 K = 1
                 II = MB*NB + 1
                 DO 180 JJ = 0, NB - 1
                    CALL SCOPY( MB, RHS( K ), 1, C( IS, JS+JJ ), 1 )
                    CALL SCOPY( MB, RHS( II ), 1, F( IS, JS+JJ ), 1 )
                    K = K + MB
                    II = II + MB
  180            CONTINUE
*
*                Substitute R(I, J) and L(I, J) into remaining
*                equation.
*
                 IF( J.GT.P+2 ) THEN
                    CALL SGEMM( 'N', 'T', MB, JS-1, NB, ONE,
```

```
     $                          C( IS, JS ), LDC, B( 1, JS ), LDB, ONE,
     $                          F( IS, 1 ), LDF )
                     CALL SGEMM( 'N', 'T', MB, JS-1, NB, ONE,
     $                          F( IS, JS ), LDF, E( 1, JS ), LDE, ONE,
     $                          F( IS, 1 ), LDF )
                  END IF
                  IF( I.LT.P ) THEN
                     CALL SGEMM( 'T', 'N', M-IE, NB, MB, -ONE,
     $                          A( IS, IE+1 ), LDA, C( IS, JS ), LDC,
     $                          ONE, C( IE+1, JS ), LDC )
                     CALL SGEMM( 'T', 'N', M-IE, NB, MB, -ONE,
     $                          D( IS, IE+1 ), LDD, F( IS, JS ), LDF,
     $                          ONE, C( IE+1, JS ), LDC )
                  END IF
*
               END IF
*
  190       CONTINUE
  200    CONTINUE
*
      END IF
      RETURN
*
*     End of STGSY2
*
      END
```

———-

63.5.4 lisp code

— stgsy2.lisp —

```
(let* ((ldz 8) (zero 0.0) (one 1.0))
  (declare (type (f2cl-lib:integer4 8 8) ldz)
           (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable ldz zero one))
  (defun stgsy2
         (trans ijob m n a lda b ldb$ c ldc d ldd e lde f ldf scale rdsum
          rdscal iwork pq info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (single-float) rdscal rdsum scale)
             (type (array single-float (*)) f e d c b a)
             (type (f2cl-lib:integer4) info pq ldf lde ldd ldc ldb$ lda n m
              ijob)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
```

```
  ((trans character trans-%data% trans-%offset%)
   (a single-float a-%data% a-%offset%)
   (b single-float b-%data% b-%offset%)
   (c single-float c-%data% c-%offset%)
   (d single-float d-%data% d-%offset%)
   (e single-float e-%data% e-%offset%)
   (f single-float f-%data% f-%offset%)
   (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
(prog ((rhs (make-array ldz :element-type 'single-float))
       (z
        (make-array (the fixnum (reduce #'* (list ldz ldz)))
                    :element-type 'single-float))
       (ipiv (make-array ldz :element-type 'f2cl-lib:integer4))
       (jpiv (make-array ldz :element-type 'f2cl-lib:integer4))
       (alpha 0.0) (scaloc 0.0) (i 0) (ie 0) (ierr 0) (ii 0) (is 0)
       (isp1 0) (j 0) (je 0) (jj 0) (js 0) (jsp1 0) (k 0) (mb 0) (nb 0)
       (p 0) (q 0) (zdim 0) (notran nil))
  (declare (type (array single-float (*)) rhs z)
           (type (array f2cl-lib:integer4 (*)) ipiv jpiv)
           (type (single-float) alpha scaloc)
           (type (f2cl-lib:integer4) i ie ierr ii is isp1 j je jj js jsp1
            k mb nb p q zdim)
           (type f2cl-lib:logical notran))
  (setf info 0)
  (setf ierr 0)
  (setf notran
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame trans "N")
            (declare (ignore var-1))
            (when var-0 (setf trans var-0))
            ret-val))
  (cond
   ((and (not notran)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame trans "T")
            (declare (ignore var-1))
            (when var-0 (setf trans var-0))
            ret-val)))
    (setf info -1))
   (notran (cond ((or (< ijob 0) (> ijob 2)) (setf info -2)))))
  (cond
   ((= info 0)
    (cond ((<= m 0) (setf info -3)) ((<= n 0) (setf info -4))
          ((< lda
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
           (setf info -6))
          ((< ldb$
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
           (setf info -8))
```

```
            ((< ldc
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -10))
            ((< ldd
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -12))
            ((< lde
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -14))
            ((< ldf
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -16)))))
  (cond
   ((/= info 0) (xerbla "STGSY2" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf pq 0)
  (setf p 0)
  (setf i 1)
 label10
  (if (> i m)
      (go label20))
  (setf p (f2cl-lib:int-add p 1))
  (setf (f2cl-lib:fref iwork-%data% (p) ((1 *)) iwork-%offset%) i)
  (if (= i m)
      (go label20))
  (cond
   ((/= (f2cl-lib:fref a ((f2cl-lib:int-add i 1) i) ((1 lda) (1 *)))
        zero)
    (setf i (f2cl-lib:int-add i 2)))
   (t (setf i (f2cl-lib:int-add i 1))))
  (go label10)
 label20
  (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add p 1)) ((1 *))
                       iwork-%offset%)
          (f2cl-lib:int-add m 1))
  (setf q (f2cl-lib:int-add p 1))
  (setf j 1)
 label30
  (if (> j n)
      (go label40))
  (setf q (f2cl-lib:int-add q 1))
  (setf (f2cl-lib:fref iwork-%data% (q) ((1 *)) iwork-%offset%) j)
  (if (= j n)
      (go label40))
  (cond
   ((/= (f2cl-lib:fref b ((f2cl-lib:int-add j 1) j) ((1 ldb$) (1 *)))
        zero)
    (setf j (f2cl-lib:int-add j 2)))
   (t (setf j (f2cl-lib:int-add j 1))))
  (go label30)
```

```
label40
 (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add q 1)) ((1 *))
                      iwork-%offset%)
        (f2cl-lib:int-add n 1))
 (setf pq (f2cl-lib:int-mul p (f2cl-lib:int-sub q p 1)))
 (cond
  (notran (setf scale one) (setf scaloc one)
   (f2cl-lib:fdo (j (f2cl-lib:int-add p 2) (f2cl-lib:int-add j 1))
                 ((> j q) nil)
     (tagbody
       (setf js (f2cl-lib:fref iwork-%data% (j) ((1 *)) iwork-%offset%))
       (setf jsp1 (f2cl-lib:int-add js 1))
       (setf je
               (f2cl-lib:int-sub
                (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add j 1))
                               ((1 *)) iwork-%offset%)
                1))
       (setf nb (f2cl-lib:int-add (f2cl-lib:int-sub je js) 1))
       (f2cl-lib:fdo (i p (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     ((> i 1) nil)
         (tagbody
           (setf is
                   (f2cl-lib:fref iwork-%data% (i) ((1 *))
                                  iwork-%offset%))
           (setf isp1 (f2cl-lib:int-add is 1))
           (setf ie
                   (f2cl-lib:int-sub
                    (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add i 1))
                                   ((1 *)) iwork-%offset%)
                    1))
           (setf mb (f2cl-lib:int-add (f2cl-lib:int-sub ie is) 1))
           (setf zdim (f2cl-lib:int-mul mb nb 2))
           (cond
            ((and (= mb 1) (= nb 1))
             (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
                     (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                                    a-%offset%))
             (setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz)))
                     (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                                    d-%offset%))
             (setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz)))
                     (-
                      (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                                     b-%offset%)))
             (setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
                     (-
                      (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                                     e-%offset%)))
             (setf (f2cl-lib:fref rhs (1) ((1 ldz)))
                     (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
```

```
                        c-%offset%))
(setf (f2cl-lib:fref rhs (2) ((1 ldz)))
        (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                        f-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (sgetc2 zdim z ldz ipiv jpiv ierr)
  (declare (ignore var-1 var-3 var-4))
  (setf zdim var-0)
  (setf ldz var-2)
  (setf ierr var-5))
(if (> ierr 0)
    (setf info ierr))
(cond
 ((= ijob 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (sgesc2 zdim z ldz rhs ipiv jpiv scaloc)
    (declare (ignore var-1 var-3 var-4 var-5))
    (setf zdim var-0)
    (setf ldz var-2)
    (setf scaloc var-6))
  (cond
   ((/= scaloc one)
    (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                  ((> k n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal m scaloc
             (f2cl-lib:array-slice c-%data% single-float
                                   (1 k) ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal m scaloc
             (f2cl-lib:array-slice f-%data% single-float
                                   (1 k) ((1 ldf) (1 *))
                                   f-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
       label50))
    (setf scale (* scale scaloc)))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8)
```

```
      (slatdf ijob zdim z ldz rhs rdsum rdscal ipiv jpiv)
    (declare (ignore var-0 var-2 var-4 var-7 var-8))
    (setf zdim var-1)
    (setf ldz var-3)
    (setf rdsum var-5)
    (setf rdscal var-6))))
(setf (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (1) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (2) ((1 ldz))))
(cond
 ((> i 1)
  (setf alpha (- (f2cl-lib:fref rhs (1) ((1 ldz)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5)
      (saxpy (f2cl-lib:int-sub is 1) alpha
       (f2cl-lib:array-slice a-%data% single-float (1 is)
                             ((1 lda) (1 *)) a-%offset%)
       1
       (f2cl-lib:array-slice c-%data% single-float (1 js)
                             ((1 ldc) (1 *)) c-%offset%)
       1)
    (declare (ignore var-0 var-2 var-3 var-4 var-5))
    (when var-1 (setf alpha var-1)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5)
      (saxpy (f2cl-lib:int-sub is 1) alpha
       (f2cl-lib:array-slice d-%data% single-float (1 is)
                             ((1 ldd) (1 *)) d-%offset%)
       1
       (f2cl-lib:array-slice f-%data% single-float (1 js)
                             ((1 ldf) (1 *)) f-%offset%)
       1)
    (declare (ignore var-0 var-2 var-3 var-4 var-5))
    (when var-1 (setf alpha var-1)))))
(cond
 ((< j q)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5)
      (saxpy (f2cl-lib:int-sub n je)
       (f2cl-lib:array-slice rhs single-float (2)
                             ((1 ldz)))
       (f2cl-lib:array-slice b-%data% single-float
                             (js (f2cl-lib:int-add je 1))
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice c-%data% single-float
                             (is (f2cl-lib:int-add je 1))
```

```
                                ((1 ldc) (1 *)) c-%offset%)
        ldc)
      (declare (ignore var-0 var-1 var-2 var-4))
      (when var-3 (setf ldb$ var-3))
      (when var-5 (setf ldc var-5)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5)
        (saxpy (f2cl-lib:int-sub n je)
         (f2cl-lib:array-slice rhs single-float (2)
                                ((1 ldz)))
         (f2cl-lib:array-slice e-%data% single-float
                                (js (f2cl-lib:int-add je 1))
                                ((1 lde) (1 *)) e-%offset%)
         lde
         (f2cl-lib:array-slice f-%data% single-float
                                (is (f2cl-lib:int-add je 1))
                                ((1 ldf) (1 *)) f-%offset%)
         ldf)
      (declare (ignore var-0 var-1 var-2 var-4))
      (when var-3 (setf lde var-3))
      (when var-5 (setf ldf var-5))))))
((and (= mb 1) (= nb 2))
 (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                        a-%offset%))
 (setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz))) zero)
 (setf (f2cl-lib:fref z (3 1) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                        d-%offset%))
 (setf (f2cl-lib:fref z (4 1) ((1 ldz) (1 ldz))) zero)
 (setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz))) zero)
 (setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                        a-%offset%))
 (setf (f2cl-lib:fref z (3 2) ((1 ldz) (1 ldz))) zero)
 (setf (f2cl-lib:fref z (4 2) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                        d-%offset%))
 (setf (f2cl-lib:fref z (1 3) ((1 ldz) (1 ldz)))
         (-
          (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                         b-%offset%)))
 (setf (f2cl-lib:fref z (2 3) ((1 ldz) (1 ldz)))
         (-
          (f2cl-lib:fref b-%data% (js jsp1) ((1 ldb$) (1 *))
                         b-%offset%)))
 (setf (f2cl-lib:fref z (3 3) ((1 ldz) (1 ldz)))
         (-
          (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                         e-%offset%)))
```

```
(setf (f2cl-lib:fref z (4 3) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js jsp1) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (1 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (jsp1 js) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (2 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (jsp1 jsp1)
                        ((1 ldb$) (1 *)) b-%offset%)))
(setf (f2cl-lib:fref z (3 4) ((1 ldz) (1 ldz))) zero)
(setf (f2cl-lib:fref z (4 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (jsp1 jsp1)
                        ((1 lde) (1 *)) e-%offset%)))
(setf (f2cl-lib:fref rhs (1) ((1 ldz)))
        (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                       c-%offset%))
(setf (f2cl-lib:fref rhs (2) ((1 ldz)))
        (f2cl-lib:fref c-%data% (is jsp1) ((1 ldc) (1 *))
                       c-%offset%))
(setf (f2cl-lib:fref rhs (3) ((1 ldz)))
        (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                       f-%offset%))
(setf (f2cl-lib:fref rhs (4) ((1 ldz)))
        (f2cl-lib:fref f-%data% (is jsp1) ((1 ldf) (1 *))
                       f-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (sgetc2 zdim z ldz ipiv jpiv ierr)
  (declare (ignore var-1 var-3 var-4))
  (setf zdim var-0)
  (setf ldz var-2)
  (setf ierr var-5))
(if (> ierr 0)
    (setf info ierr))
(cond
 ((= ijob 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (sgesc2 zdim z ldz rhs ipiv jpiv scaloc)
    (declare (ignore var-1 var-3 var-4 var-5))
    (setf zdim var-0)
    (setf ldz var-2)
    (setf scaloc var-6))
  (cond
   ((/= scaloc one)
    (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                  ((> k n) nil)
```

```
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal m scaloc
             (f2cl-lib:array-slice c-%data% single-float
                                   (1 k) ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal m scaloc
             (f2cl-lib:array-slice f-%data% single-float
                                   (1 k) ((1 ldf) (1 *))
                                   f-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
       label60))
    (setf scale (* scale scaloc)))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8)
      (slatdf ijob zdim z ldz rhs rdsum rdscal ipiv jpiv)
    (declare (ignore var-0 var-2 var-4 var-7 var-8))
    (setf zdim var-1)
    (setf ldz var-3)
    (setf rdsum var-5)
    (setf rdscal var-6))))
(setf (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (1) ((1 ldz))))
(setf (f2cl-lib:fref c-%data% (is jsp1) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (2) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (3) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (is jsp1) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (4) ((1 ldz))))
(cond
 ((> i 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8)
      (sger (f2cl-lib:int-sub is 1) nb (- one)
       (f2cl-lib:array-slice a-%data% single-float (1 is)
```

```
                                ((1 lda) (1 *)) a-%offset%)
        1
        (f2cl-lib:array-slice rhs single-float (1)
                                ((1 ldz)))
        1
        (f2cl-lib:array-slice c-%data% single-float (1 js)
                                ((1 ldc) (1 *)) c-%offset%)
        ldc)
    (declare
     (ignore var-0 var-2 var-3 var-4 var-5 var-6 var-7))
    (when var-1 (setf nb var-1))
    (when var-8 (setf ldc var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8)
      (sger (f2cl-lib:int-sub is 1) nb (- one)
       (f2cl-lib:array-slice d-%data% single-float (1 is)
                                ((1 ldd) (1 *)) d-%offset%)
       1
       (f2cl-lib:array-slice rhs single-float (1)
                                ((1 ldz)))
       1
       (f2cl-lib:array-slice f-%data% single-float (1 js)
                                ((1 ldf) (1 *)) f-%offset%)
       ldf)
    (declare
     (ignore var-0 var-2 var-3 var-4 var-5 var-6 var-7))
    (when var-1 (setf nb var-1))
    (when var-8 (setf ldf var-8)))))
(cond
 ((< j q)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5)
      (saxpy (f2cl-lib:int-sub n je)
       (f2cl-lib:array-slice rhs single-float (3)
                                ((1 ldz)))
       (f2cl-lib:array-slice b-%data% single-float
                                (js (f2cl-lib:int-add je 1))
                                ((1 ldb$) (1 *)) b-%offset%)
       ldb$
       (f2cl-lib:array-slice c-%data% single-float
                                (is (f2cl-lib:int-add je 1))
                                ((1 ldc) (1 *)) c-%offset%)
       ldc)
    (declare (ignore var-0 var-1 var-2 var-4))
    (when var-3 (setf ldb$ var-3))
    (when var-5 (setf ldc var-5)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5)
      (saxpy (f2cl-lib:int-sub n je)
```

```
            (f2cl-lib:array-slice rhs single-float (3)
                                  ((1 ldz)))
            (f2cl-lib:array-slice e-%data% single-float
                                  (js (f2cl-lib:int-add je 1))
                                  ((1 lde) (1 *)) e-%offset%)
            lde
            (f2cl-lib:array-slice f-%data% single-float
                                  (is (f2cl-lib:int-add je 1))
                                  ((1 ldf) (1 *)) f-%offset%)
            ldf)
          (declare (ignore var-0 var-1 var-2 var-4))
          (when var-3 (setf lde var-3))
          (when var-5 (setf ldf var-5)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5)
            (saxpy (f2cl-lib:int-sub n je)
            (f2cl-lib:array-slice rhs single-float (4)
                                  ((1 ldz)))
            (f2cl-lib:array-slice b-%data% single-float
                                  (jsp1 (f2cl-lib:int-add je 1))
                                  ((1 ldb$) (1 *)) b-%offset%)
            ldb$
            (f2cl-lib:array-slice c-%data% single-float
                                  (is (f2cl-lib:int-add je 1))
                                  ((1 ldc) (1 *)) c-%offset%)
            ldc)
          (declare (ignore var-0 var-1 var-2 var-4))
          (when var-3 (setf ldb$ var-3))
          (when var-5 (setf ldc var-5)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5)
            (saxpy (f2cl-lib:int-sub n je)
            (f2cl-lib:array-slice rhs single-float (4)
                                  ((1 ldz)))
            (f2cl-lib:array-slice e-%data% single-float
                                  (jsp1 (f2cl-lib:int-add je 1))
                                  ((1 lde) (1 *)) e-%offset%)
            lde
            (f2cl-lib:array-slice f-%data% single-float
                                  (is (f2cl-lib:int-add je 1))
                                  ((1 ldf) (1 *)) f-%offset%)
            ldf)
          (declare (ignore var-0 var-1 var-2 var-4))
          (when var-3 (setf lde var-3))
          (when var-5 (setf ldf var-5))))))
    ((and (= mb 2) (= nb 1))
     (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
             (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                            a-%offset%))
     (setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz)))
```

```
        (f2cl-lib:fref a-%data% (isp1 is) ((1 lda) (1 *))
                       a-%offset%))
(setf (f2cl-lib:fref z (3 1) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                       d-%offset%))
(setf (f2cl-lib:fref z (4 1) ((1 ldz) (1 ldz))) zero)
(setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (is isp1) ((1 lda) (1 *))
                       a-%offset%))
(setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (isp1 isp1) ((1 lda) (1 *))
                       a-%offset%))
(setf (f2cl-lib:fref z (3 2) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is isp1) ((1 ldd) (1 *))
                       d-%offset%))
(setf (f2cl-lib:fref z (4 2) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (isp1 isp1) ((1 ldd) (1 *))
                       d-%offset%))
(setf (f2cl-lib:fref z (1 3) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (2 3) ((1 ldz) (1 ldz))) zero)
(setf (f2cl-lib:fref z (3 3) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (4 3) ((1 ldz) (1 ldz))) zero)
(setf (f2cl-lib:fref z (1 4) ((1 ldz) (1 ldz))) zero)
(setf (f2cl-lib:fref z (2 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (3 4) ((1 ldz) (1 ldz))) zero)
(setf (f2cl-lib:fref z (4 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref rhs (1) ((1 ldz)))
        (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                       c-%offset%))
(setf (f2cl-lib:fref rhs (2) ((1 ldz)))
        (f2cl-lib:fref c-%data% (isp1 js) ((1 ldc) (1 *))
                       c-%offset%))
(setf (f2cl-lib:fref rhs (3) ((1 ldz)))
        (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                       f-%offset%))
(setf (f2cl-lib:fref rhs (4) ((1 ldz)))
        (f2cl-lib:fref f-%data% (isp1 js) ((1 ldf) (1 *))
                       f-%offset%))
```

```
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (sgetc2 zdim z ldz ipiv jpiv ierr)
  (declare (ignore var-1 var-3 var-4))
  (setf zdim var-0)
  (setf ldz var-2)
  (setf ierr var-5))
(if (> ierr 0)
    (setf info ierr))
(cond
 ((= ijob 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (sgesc2 zdim z ldz rhs ipiv jpiv scaloc)
    (declare (ignore var-1 var-3 var-4 var-5))
    (setf zdim var-0)
    (setf ldz var-2)
    (setf scaloc var-6))
  (cond
   ((/= scaloc one)
    (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                  ((> k n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal m scaloc
             (f2cl-lib:array-slice c-%data% single-float
                                   (1 k) ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal m scaloc
             (f2cl-lib:array-slice f-%data% single-float
                                   (1 k) ((1 ldf) (1 *))
                                   f-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
       label70))
    (setf scale (* scale scaloc)))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8)
      (slatdf ijob zdim z ldz rhs rdsum rdscal ipiv jpiv)
    (declare (ignore var-0 var-2 var-4 var-7 var-8))
    (setf zdim var-1)
    (setf ldz var-3)
```

```
    (setf rdsum var-5)
    (setf rdscal var-6))))
(setf (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (1) ((1 ldz))))
(setf (f2cl-lib:fref c-%data% (isp1 js) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (2) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (3) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (isp1 js) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (4) ((1 ldz))))
(cond
 ((> i 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8 var-9 var-10)
      (sgemv "N" (f2cl-lib:int-sub is 1) mb (- one)
       (f2cl-lib:array-slice a-%data% single-float (1 is)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice rhs single-float (1)
                             ((1 ldz)))
       1 one
       (f2cl-lib:array-slice c-%data% single-float (1 js)
                             ((1 ldc) (1 *)) c-%offset%)
       1)
    (declare
     (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9
      var-10))
    (when var-2 (setf mb var-2))
    (when var-5 (setf lda var-5))
    (when var-8 (setf one var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8 var-9 var-10)
      (sgemv "N" (f2cl-lib:int-sub is 1) mb (- one)
       (f2cl-lib:array-slice d-%data% single-float (1 is)
                             ((1 ldd) (1 *)) d-%offset%)
       ldd
       (f2cl-lib:array-slice rhs single-float (1)
                             ((1 ldz)))
       1 one
       (f2cl-lib:array-slice f-%data% single-float (1 js)
                             ((1 ldf) (1 *)) f-%offset%)
       1)
    (declare
     (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9
```

```
        var-10))
      (when var-2 (setf mb var-2))
      (when var-5 (setf ldd var-5))
      (when var-8 (setf one var-8)))))
  (cond
   ((< j q)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
         var-8)
        (sger mb (f2cl-lib:int-sub n je) one
         (f2cl-lib:array-slice rhs single-float (3)
                               ((1 ldz)))
         1
         (f2cl-lib:array-slice b-%data% single-float
                               (js (f2cl-lib:int-add je 1))
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$
         (f2cl-lib:array-slice c-%data% single-float
                               (is (f2cl-lib:int-add je 1))
                               ((1 ldc) (1 *)) c-%offset%)
         ldc)
      (declare (ignore var-1 var-3 var-4 var-5 var-7))
      (when var-0 (setf mb var-0))
      (when var-2 (setf one var-2))
      (when var-6 (setf ldb$ var-6))
      (when var-8 (setf ldc var-8)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
         var-8)
        (sger mb (f2cl-lib:int-sub n je) one
         (f2cl-lib:array-slice rhs single-float (3)
                               ((1 ldz)))
         1
         (f2cl-lib:array-slice e-%data% single-float
                               (js (f2cl-lib:int-add je 1))
                               ((1 lde) (1 *)) e-%offset%)
         lde
         (f2cl-lib:array-slice f-%data% single-float
                               (is (f2cl-lib:int-add je 1))
                               ((1 ldf) (1 *)) f-%offset%)
         ldf)
      (declare (ignore var-1 var-3 var-4 var-5 var-7))
      (when var-0 (setf mb var-0))
      (when var-2 (setf one var-2))
      (when var-6 (setf lde var-6))
      (when var-8 (setf ldf var-8))))))
 ((and (= mb 2) (= nb 2))
  (slaset "F" ldz ldz zero zero z ldz)
  (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
          (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
```

```
                        a-%offset%))
(setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (isp1 is) ((1 lda) (1 *))
                        a-%offset%))
(setf (f2cl-lib:fref z (5 1) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                        d-%offset%))
(setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (is isp1) ((1 lda) (1 *))
                        a-%offset%))
(setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (isp1 isp1) ((1 lda) (1 *))
                        a-%offset%))
(setf (f2cl-lib:fref z (5 2) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is isp1) ((1 ldd) (1 *))
                        d-%offset%))
(setf (f2cl-lib:fref z (6 2) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (isp1 isp1) ((1 ldd) (1 *))
                        d-%offset%))
(setf (f2cl-lib:fref z (3 3) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                        a-%offset%))
(setf (f2cl-lib:fref z (4 3) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (isp1 is) ((1 lda) (1 *))
                        a-%offset%))
(setf (f2cl-lib:fref z (7 3) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                        d-%offset%))
(setf (f2cl-lib:fref z (3 4) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (is isp1) ((1 lda) (1 *))
                        a-%offset%))
(setf (f2cl-lib:fref z (4 4) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (isp1 isp1) ((1 lda) (1 *))
                        a-%offset%))
(setf (f2cl-lib:fref z (7 4) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is isp1) ((1 ldd) (1 *))
                        d-%offset%))
(setf (f2cl-lib:fref z (8 4) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (isp1 isp1) ((1 ldd) (1 *))
                        d-%offset%))
(setf (f2cl-lib:fref z (1 5) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                         b-%offset%)))
(setf (f2cl-lib:fref z (3 5) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (js jsp1) ((1 ldb$) (1 *))
                         b-%offset%)))
(setf (f2cl-lib:fref z (5 5) ((1 ldz) (1 ldz)))
        (-
```

```
         (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (7 5) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js jsp1) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (2 6) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (4 6) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (js jsp1) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (6 6) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (8 6) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js jsp1) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (1 7) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (jsp1 js) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (3 7) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (jsp1 jsp1)
                        ((1 ldb$) (1 *)) b-%offset%)))
(setf (f2cl-lib:fref z (7 7) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (jsp1 jsp1)
                        ((1 lde) (1 *)) e-%offset%)))
(setf (f2cl-lib:fref z (2 8) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (jsp1 js) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (4 8) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (jsp1 jsp1)
                        ((1 ldb$) (1 *)) b-%offset%)))
(setf (f2cl-lib:fref z (8 8) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (jsp1 jsp1)
                        ((1 lde) (1 *)) e-%offset%)))
(setf k 1)
(setf ii (f2cl-lib:int-add (f2cl-lib:int-mul mb nb) 1))
(f2cl-lib:fdo (jj 0 (f2cl-lib:int-add jj 1))
              ((> jj
```

```
                    (f2cl-lib:int-add nb
                                      (f2cl-lib:int-sub 1)))
                nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy mb
         (f2cl-lib:array-slice c-%data% single-float
                               (is
                                (f2cl-lib:int-add js jj))
                               ((1 ldc) (1 *)) c-%offset%)
         1
         (f2cl-lib:array-slice rhs single-float (k)
                               ((1 ldz)))
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf mb var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy mb
         (f2cl-lib:array-slice f-%data% single-float
                               (is
                                (f2cl-lib:int-add js jj))
                               ((1 ldf) (1 *)) f-%offset%)
         1
         (f2cl-lib:array-slice rhs single-float (ii)
                               ((1 ldz)))
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf mb var-0)))
    (setf k (f2cl-lib:int-add k mb))
    (setf ii (f2cl-lib:int-add ii mb))
   label80))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (sgetc2 zdim z ldz ipiv jpiv ierr)
  (declare (ignore var-1 var-3 var-4))
  (setf zdim var-0)
  (setf ldz var-2)
  (setf ierr var-5))
(if (> ierr 0)
    (setf info ierr))
(cond
 ((= ijob 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (sgesc2 zdim z ldz rhs ipiv jpiv scaloc)
    (declare (ignore var-1 var-3 var-4 var-5))
    (setf zdim var-0)
    (setf ldz var-2)
    (setf scaloc var-6))
  (cond
   ((/= scaloc one)
```

```
    (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                  ((> k n) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal m scaloc
             (f2cl-lib:array-slice c-%data% single-float
                                   (1 k) ((1 ldc) (1 *))
                                   c-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal m scaloc
             (f2cl-lib:array-slice f-%data% single-float
                                   (1 k) ((1 ldf) (1 *))
                                   f-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
       label90))
    (setf scale (* scale scaloc)))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8)
      (slatdf ijob zdim z ldz rhs rdsum rdscal ipiv jpiv)
    (declare (ignore var-0 var-2 var-4 var-7 var-8))
    (setf zdim var-1)
    (setf ldz var-3)
    (setf rdsum var-5)
    (setf rdscal var-6))))
(setf k 1)
(setf ii (f2cl-lib:int-add (f2cl-lib:int-mul mb nb) 1))
(f2cl-lib:fdo (jj 0 (f2cl-lib:int-add jj 1))
              ((> jj
                  (f2cl-lib:int-add nb
                                    (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy mb
         (f2cl-lib:array-slice rhs single-float (k)
                               ((1 ldz)))
         1
         (f2cl-lib:array-slice c-%data% single-float
                               (is
                                (f2cl-lib:int-add js jj))
                               ((1 ldc) (1 *)) c-%offset%)
```

```
          1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf mb var-0)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (scopy mb
           (f2cl-lib:array-slice rhs single-float (ii)
                                 ((1 ldz)))
           1
           (f2cl-lib:array-slice f-%data% single-float
                                 (is
                                  (f2cl-lib:int-add js jj))
                                 ((1 ldf) (1 *)) f-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf mb var-0)))
      (setf k (f2cl-lib:int-add k mb))
      (setf ii (f2cl-lib:int-add ii mb))
    label100))
  (cond
   ((> i 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
         var-8 var-9 var-10 var-11 var-12)
        (sgemm "N" "N" (f2cl-lib:int-sub is 1) nb mb (- one)
         (f2cl-lib:array-slice a-%data% single-float (1 is)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice rhs single-float (1)
                               ((1 ldz)))
         mb one
         (f2cl-lib:array-slice c-%data% single-float (1 js)
                               ((1 ldc) (1 *)) c-%offset%)
         ldc)
      (declare
       (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
      (when var-3 (setf nb var-3))
      (when var-4 (setf mb var-4))
      (when var-7 (setf lda var-7))
      (when var-9 (setf mb var-9))
      (when var-10 (setf one var-10))
      (when var-12 (setf ldc var-12)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
         var-8 var-9 var-10 var-11 var-12)
        (sgemm "N" "N" (f2cl-lib:int-sub is 1) nb mb (- one)
         (f2cl-lib:array-slice d-%data% single-float (1 is)
                               ((1 ldd) (1 *)) d-%offset%)
         ldd
         (f2cl-lib:array-slice rhs single-float (1)
                               ((1 ldz)))
```

```
        mb one
        (f2cl-lib:array-slice f-%data% single-float (1 js)
                              ((1 ldf) (1 *)) f-%offset%)
        ldf)
    (declare
     (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
    (when var-3 (setf nb var-3))
    (when var-4 (setf mb var-4))
    (when var-7 (setf ldd var-7))
    (when var-9 (setf mb var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldf var-12)))))
(cond
 ((< j q)
  (setf k (f2cl-lib:int-add (f2cl-lib:int-mul mb nb) 1))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8 var-9 var-10 var-11 var-12)
      (sgemm "N" "N" mb (f2cl-lib:int-sub n je) nb one
       (f2cl-lib:array-slice rhs single-float (k)
                             ((1 ldz)))
       mb
       (f2cl-lib:array-slice b-%data% single-float
                             (js (f2cl-lib:int-add je 1))
                             ((1 ldb$) (1 *)) b-%offset%)
       ldb$ one
       (f2cl-lib:array-slice c-%data% single-float
                             (is (f2cl-lib:int-add je 1))
                             ((1 ldc) (1 *)) c-%offset%)
       ldc)
    (declare (ignore var-0 var-1 var-3 var-6 var-8 var-11))
    (when var-2 (setf mb var-2))
    (when var-4 (setf nb var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf mb var-7))
    (when var-9 (setf ldb$ var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldc var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8 var-9 var-10 var-11 var-12)
      (sgemm "N" "N" mb (f2cl-lib:int-sub n je) nb one
       (f2cl-lib:array-slice rhs single-float (k)
                             ((1 ldz)))
       mb
       (f2cl-lib:array-slice e-%data% single-float
                             (js (f2cl-lib:int-add je 1))
                             ((1 lde) (1 *)) e-%offset%)
       lde one
       (f2cl-lib:array-slice f-%data% single-float
```

```
                                          (is (f2cl-lib:int-add je 1))
                                          ((1 ldf) (1 *)) f-%offset%)
                     ldf)
                  (declare (ignore var-0 var-1 var-3 var-6 var-8 var-11))
                  (when var-2 (setf mb var-2))
                  (when var-4 (setf nb var-4))
                  (when var-5 (setf one var-5))
                  (when var-7 (setf mb var-7))
                  (when var-9 (setf lde var-9))
                  (when var-10 (setf one var-10))
                  (when var-12 (setf ldf var-12)))))))
          label110))
     label120)))
  (t (setf scale one) (setf scaloc one)
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i p) nil)
     (tagbody
       (setf is (f2cl-lib:fref iwork-%data% (i) ((1 *)) iwork-%offset%))
       (setf isp1 (f2cl-lib:int-add is 1))
       (setf ie
               (f2cl-lib:int-sub
                (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add i 1))
                               ((1 *)) iwork-%offset%)
                1))
       (setf mb (f2cl-lib:int-add (f2cl-lib:int-sub ie is) 1))
       (f2cl-lib:fdo (j q (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     ((> j (f2cl-lib:int-add p 2)) nil)
         (tagbody
           (setf js
                   (f2cl-lib:fref iwork-%data% (j) ((1 *))
                                  iwork-%offset%))
           (setf jsp1 (f2cl-lib:int-add js 1))
           (setf je
                   (f2cl-lib:int-sub
                    (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add j 1))
                                   ((1 *)) iwork-%offset%)
                    1))
           (setf nb (f2cl-lib:int-add (f2cl-lib:int-sub je js) 1))
           (setf zdim (f2cl-lib:int-mul mb nb 2))
           (cond
            ((and (= mb 1) (= nb 1))
             (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
                     (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                                    a-%offset%))
             (setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz)))
                     (-
                      (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                                     b-%offset%)))
             (setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz)))
                     (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
```

```
                        d-%offset%))
(setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                         e-%offset%)))
(setf (f2cl-lib:fref rhs (1) ((1 ldz)))
        (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                        c-%offset%))
(setf (f2cl-lib:fref rhs (2) ((1 ldz)))
        (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                        f-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (sgetc2 zdim z ldz ipiv jpiv ierr)
  (declare (ignore var-1 var-3 var-4))
  (setf zdim var-0)
  (setf ldz var-2)
  (setf ierr var-5))
(if (> ierr 0)
    (setf info ierr))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (sgesc2 zdim z ldz rhs ipiv jpiv scaloc)
  (declare (ignore var-1 var-3 var-4 var-5))
  (setf zdim var-0)
  (setf ldz var-2)
  (setf scaloc var-6))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal m scaloc
           (f2cl-lib:array-slice c-%data% single-float
                                 (1 k) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal m scaloc
           (f2cl-lib:array-slice f-%data% single-float
                                 (1 k) ((1 ldf) (1 *))
                                 f-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
     label130))
```

```
  (setf scale (* scale scaloc))))
(setf (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (1) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (2) ((1 ldz))))
(cond
 ((> j (f2cl-lib:int-add p 2))
  (setf alpha (f2cl-lib:fref rhs (1) ((1 ldz))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5)
      (saxpy (f2cl-lib:int-sub js 1) alpha
       (f2cl-lib:array-slice b-%data% single-float (1 js)
                             ((1 ldb$) (1 *)) b-%offset%)
       1
       (f2cl-lib:array-slice f-%data% single-float (is 1)
                             ((1 ldf) (1 *)) f-%offset%)
       ldf)
    (declare (ignore var-0 var-2 var-3 var-4))
    (when var-1 (setf alpha var-1))
    (when var-5 (setf ldf var-5)))
  (setf alpha (f2cl-lib:fref rhs (2) ((1 ldz))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5)
      (saxpy (f2cl-lib:int-sub js 1) alpha
       (f2cl-lib:array-slice e-%data% single-float (1 js)
                             ((1 lde) (1 *)) e-%offset%)
       1
       (f2cl-lib:array-slice f-%data% single-float (is 1)
                             ((1 ldf) (1 *)) f-%offset%)
       ldf)
    (declare (ignore var-0 var-2 var-3 var-4))
    (when var-1 (setf alpha var-1))
    (when var-5 (setf ldf var-5)))))
(cond
 ((< i p)
  (setf alpha (- (f2cl-lib:fref rhs (1) ((1 ldz)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5)
      (saxpy (f2cl-lib:int-sub m ie) alpha
       (f2cl-lib:array-slice a-%data% single-float
                             (is (f2cl-lib:int-add ie 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice c-%data% single-float
                             ((+ ie 1) js) ((1 ldc) (1 *))
                             c-%offset%)
       1)
    (declare (ignore var-0 var-2 var-4 var-5))
```

```
      (when var-1 (setf alpha var-1))
      (when var-3 (setf lda var-3)))
    (setf alpha (- (f2cl-lib:fref rhs (2) ((1 ldz)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5)
        (saxpy (f2cl-lib:int-sub m ie) alpha
         (f2cl-lib:array-slice d-%data% single-float
                               (is (f2cl-lib:int-add ie 1))
                               ((1 ldd) (1 *)) d-%offset%)
         ldd
         (f2cl-lib:array-slice c-%data% single-float
                               ((+ ie 1) js) ((1 ldc) (1 *))
                               c-%offset%)
         1)
      (declare (ignore var-0 var-2 var-4 var-5))
      (when var-1 (setf alpha var-1))
      (when var-3 (setf ldd var-3))))))
((and (= mb 1) (= nb 2))
 (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                        a-%offset%))
 (setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz))) zero)
 (setf (f2cl-lib:fref z (3 1) ((1 ldz) (1 ldz)))
         (-
          (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                         b-%offset%)))
 (setf (f2cl-lib:fref z (4 1) ((1 ldz) (1 ldz)))
         (-
          (f2cl-lib:fref b-%data% (jsp1 js) ((1 ldb$) (1 *))
                         b-%offset%)))
 (setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz))) zero)
 (setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                        a-%offset%))
 (setf (f2cl-lib:fref z (3 2) ((1 ldz) (1 ldz)))
         (-
          (f2cl-lib:fref b-%data% (js jsp1) ((1 ldb$) (1 *))
                         b-%offset%)))
 (setf (f2cl-lib:fref z (4 2) ((1 ldz) (1 ldz)))
         (-
          (f2cl-lib:fref b-%data% (jsp1 jsp1)
                         ((1 ldb$) (1 *)) b-%offset%)))
 (setf (f2cl-lib:fref z (1 3) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                        d-%offset%))
 (setf (f2cl-lib:fref z (2 3) ((1 ldz) (1 ldz))) zero)
 (setf (f2cl-lib:fref z (3 3) ((1 ldz) (1 ldz)))
         (-
          (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                         e-%offset%)))
```

```
(setf (f2cl-lib:fref z (4 3) ((1 ldz) (1 ldz))) zero)
(setf (f2cl-lib:fref z (1 4) ((1 ldz) (1 ldz))) zero)
(setf (f2cl-lib:fref z (2 4) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                       d-%offset%))
(setf (f2cl-lib:fref z (3 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js jsp1) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (4 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (jsp1 jsp1)
                        ((1 lde) (1 *)) e-%offset%)))
(setf (f2cl-lib:fref rhs (1) ((1 ldz)))
        (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                       c-%offset%))
(setf (f2cl-lib:fref rhs (2) ((1 ldz)))
        (f2cl-lib:fref c-%data% (is jsp1) ((1 ldc) (1 *))
                       c-%offset%))
(setf (f2cl-lib:fref rhs (3) ((1 ldz)))
        (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                       f-%offset%))
(setf (f2cl-lib:fref rhs (4) ((1 ldz)))
        (f2cl-lib:fref f-%data% (is jsp1) ((1 ldf) (1 *))
                       f-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (sgetc2 zdim z ldz ipiv jpiv ierr)
  (declare (ignore var-1 var-3 var-4))
  (setf zdim var-0)
  (setf ldz var-2)
  (setf ierr var-5))
(if (> ierr 0)
    (setf info ierr))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (sgesc2 zdim z ldz rhs ipiv jpiv scaloc)
  (declare (ignore var-1 var-3 var-4 var-5))
  (setf zdim var-0)
  (setf ldz var-2)
  (setf scaloc var-6))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal m scaloc
           (f2cl-lib:array-slice c-%data% single-float
                                 (1 k) ((1 ldc) (1 *))
                                 c-%offset%)
```

```
          1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal m scaloc
           (f2cl-lib:array-slice f-%data% single-float
                                 (1 k) ((1 ldf) (1 *))
                                 f-%offset%)
          1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
     label140))
  (setf scale (* scale scaloc))))
(setf (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (1) ((1 ldz))))
(setf (f2cl-lib:fref c-%data% (is jsp1) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (2) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (3) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (is jsp1) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (4) ((1 ldz))))
(cond
 ((> j (f2cl-lib:int-add p 2))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5)
      (saxpy (f2cl-lib:int-sub js 1)
       (f2cl-lib:array-slice rhs single-float (1)
                             ((1 ldz)))
       (f2cl-lib:array-slice b-%data% single-float (1 js)
                             ((1 ldb$) (1 *)) b-%offset%)
       1
       (f2cl-lib:array-slice f-%data% single-float (is 1)
                             ((1 ldf) (1 *)) f-%offset%)
       ldf)
    (declare (ignore var-0 var-1 var-2 var-3 var-4))
    (when var-5 (setf ldf var-5)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5)
      (saxpy (f2cl-lib:int-sub js 1)
       (f2cl-lib:array-slice rhs single-float (2)
                             ((1 ldz)))
       (f2cl-lib:array-slice b-%data% single-float (1 jsp1)
                             ((1 ldb$) (1 *)) b-%offset%)
       1
```

```
          (f2cl-lib:array-slice f-%data% single-float (is 1)
                                ((1 ldf) (1 *)) f-%offset%)
          ldf)
        (declare (ignore var-0 var-1 var-2 var-3 var-4))
        (when var-5 (setf ldf var-5)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5)
          (saxpy (f2cl-lib:int-sub js 1)
           (f2cl-lib:array-slice rhs single-float (3)
                                 ((1 ldz)))
           (f2cl-lib:array-slice e-%data% single-float (1 js)
                                 ((1 lde) (1 *)) e-%offset%)
           1
           (f2cl-lib:array-slice f-%data% single-float (is 1)
                                 ((1 ldf) (1 *)) f-%offset%)
           ldf)
        (declare (ignore var-0 var-1 var-2 var-3 var-4))
        (when var-5 (setf ldf var-5)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5)
          (saxpy (f2cl-lib:int-sub js 1)
           (f2cl-lib:array-slice rhs single-float (4)
                                 ((1 ldz)))
           (f2cl-lib:array-slice e-%data% single-float (1 jsp1)
                                 ((1 lde) (1 *)) e-%offset%)
           1
           (f2cl-lib:array-slice f-%data% single-float (is 1)
                                 ((1 ldf) (1 *)) f-%offset%)
           ldf)
        (declare (ignore var-0 var-1 var-2 var-3 var-4))
        (when var-5 (setf ldf var-5)))))
    (cond
     ((< i p)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
           var-8)
          (sger (f2cl-lib:int-sub m ie) nb (- one)
           (f2cl-lib:array-slice a-%data% single-float
                                 (is (f2cl-lib:int-add ie 1))
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice rhs single-float (1)
                                 ((1 ldz)))
           1
           (f2cl-lib:array-slice c-%data% single-float
                                 ((+ ie 1) js) ((1 ldc) (1 *))
                                 c-%offset%)
           ldc)
        (declare (ignore var-0 var-2 var-3 var-5 var-6 var-7))
        (when var-1 (setf nb var-1))
```

```
        (when var-4 (setf lda var-4))
        (when var-8 (setf ldc var-8)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
           var-8)
          (sger (f2cl-lib:int-sub m ie) nb (- one)
           (f2cl-lib:array-slice d-%data% single-float
                                 (is (f2cl-lib:int-add ie 1))
                                 ((1 ldd) (1 *)) d-%offset%)
           ldd
           (f2cl-lib:array-slice rhs single-float (3)
                                 ((1 ldz)))
           1
           (f2cl-lib:array-slice c-%data% single-float
                                 ((+ ie 1) js) ((1 ldc) (1 *))
                                 c-%offset%)
           ldc)
        (declare (ignore var-0 var-2 var-3 var-5 var-6 var-7))
        (when var-1 (setf nb var-1))
        (when var-4 (setf ldd var-4))
        (when var-8 (setf ldc var-8))))))
    ((and (= mb 2) (= nb 1))
     (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
             (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                            a-%offset%))
     (setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz)))
             (f2cl-lib:fref a-%data% (is isp1) ((1 lda) (1 *))
                            a-%offset%))
     (setf (f2cl-lib:fref z (3 1) ((1 ldz) (1 ldz)))
             (-
              (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                             b-%offset%)))
     (setf (f2cl-lib:fref z (4 1) ((1 ldz) (1 ldz))) zero)
     (setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz)))
             (f2cl-lib:fref a-%data% (isp1 is) ((1 lda) (1 *))
                            a-%offset%))
     (setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
             (f2cl-lib:fref a-%data% (isp1 isp1) ((1 lda) (1 *))
                            a-%offset%))
     (setf (f2cl-lib:fref z (3 2) ((1 ldz) (1 ldz))) zero)
     (setf (f2cl-lib:fref z (4 2) ((1 ldz) (1 ldz)))
             (-
              (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                             b-%offset%)))
     (setf (f2cl-lib:fref z (1 3) ((1 ldz) (1 ldz)))
             (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                            d-%offset%))
     (setf (f2cl-lib:fref z (2 3) ((1 ldz) (1 ldz)))
             (f2cl-lib:fref d-%data% (is isp1) ((1 ldd) (1 *))
                            d-%offset%))
```

```
(setf (f2cl-lib:fref z (3 3) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (4 3) ((1 ldz) (1 ldz))) zero)
(setf (f2cl-lib:fref z (1 4) ((1 ldz) (1 ldz))) zero)
(setf (f2cl-lib:fref z (2 4) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (isp1 isp1) ((1 ldd) (1 *))
                       d-%offset%))
(setf (f2cl-lib:fref z (3 4) ((1 ldz) (1 ldz))) zero)
(setf (f2cl-lib:fref z (4 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref rhs (1) ((1 ldz)))
        (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                       c-%offset%))
(setf (f2cl-lib:fref rhs (2) ((1 ldz)))
        (f2cl-lib:fref c-%data% (isp1 js) ((1 ldc) (1 *))
                       c-%offset%))
(setf (f2cl-lib:fref rhs (3) ((1 ldz)))
        (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                       f-%offset%))
(setf (f2cl-lib:fref rhs (4) ((1 ldz)))
        (f2cl-lib:fref f-%data% (isp1 js) ((1 ldf) (1 *))
                       f-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (sgetc2 zdim z ldz ipiv jpiv ierr)
  (declare (ignore var-1 var-3 var-4))
  (setf zdim var-0)
  (setf ldz var-2)
  (setf ierr var-5))
(if (> ierr 0)
    (setf info ierr))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (sgesc2 zdim z ldz rhs ipiv jpiv scaloc)
  (declare (ignore var-1 var-3 var-4 var-5))
  (setf zdim var-0)
  (setf ldz var-2)
  (setf scaloc var-6))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal m scaloc
           (f2cl-lib:array-slice c-%data% single-float
                                 (1 k) ((1 ldc) (1 *))
```

```
                                  c-%offset%)
            1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (sscal m scaloc
             (f2cl-lib:array-slice f-%data% single-float
                                   (1 k) ((1 ldf) (1 *))
                                   f-%offset%)
            1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf scaloc var-1)))
      label150))
  (setf scale (* scale scaloc))))
(setf (f2cl-lib:fref c-%data% (is js) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (1) ((1 ldz))))
(setf (f2cl-lib:fref c-%data% (isp1 js) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (2) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (is js) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (3) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (isp1 js) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (4) ((1 ldz))))
(cond
 ((> j (f2cl-lib:int-add p 2))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8)
      (sger mb (f2cl-lib:int-sub js 1) one
       (f2cl-lib:array-slice rhs single-float (1)
                             ((1 ldz)))
       1
       (f2cl-lib:array-slice b-%data% single-float (1 js)
                             ((1 ldb$) (1 *)) b-%offset%)
       1
       (f2cl-lib:array-slice f-%data% single-float (is 1)
                             ((1 ldf) (1 *)) f-%offset%)
       ldf)
    (declare (ignore var-1 var-3 var-4 var-5 var-6 var-7))
    (when var-0 (setf mb var-0))
    (when var-2 (setf one var-2))
    (when var-8 (setf ldf var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8)
```

```
        (sger mb (f2cl-lib:int-sub js 1) one
         (f2cl-lib:array-slice rhs single-float (3)
                               ((1 ldz)))
         1
         (f2cl-lib:array-slice e-%data% single-float (1 js)
                               ((1 lde) (1 *)) e-%offset%)
         1
         (f2cl-lib:array-slice f-%data% single-float (is 1)
                               ((1 ldf) (1 *)) f-%offset%)
         ldf)
      (declare (ignore var-1 var-3 var-4 var-5 var-6 var-7))
      (when var-0 (setf mb var-0))
      (when var-2 (setf one var-2))
      (when var-8 (setf ldf var-8)))))
  (cond
   ((< i p)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
         var-8 var-9 var-10)
        (sgemv "T" mb (f2cl-lib:int-sub m ie) (- one)
         (f2cl-lib:array-slice a-%data% single-float
                               (is (f2cl-lib:int-add ie 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice rhs single-float (1)
                               ((1 ldz)))
         1 one
         (f2cl-lib:array-slice c-%data% single-float
                               ((+ ie 1) js) ((1 ldc) (1 *))
                               c-%offset%)
         1)
      (declare
       (ignore var-0 var-2 var-3 var-4 var-6 var-7 var-9
        var-10))
      (when var-1 (setf mb var-1))
      (when var-5 (setf lda var-5))
      (when var-8 (setf one var-8)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
         var-8 var-9 var-10)
        (sgemv "T" mb (f2cl-lib:int-sub m ie) (- one)
         (f2cl-lib:array-slice d-%data% single-float
                               (is (f2cl-lib:int-add ie 1))
                               ((1 ldd) (1 *)) d-%offset%)
         ldd
         (f2cl-lib:array-slice rhs single-float (3)
                               ((1 ldz)))
         1 one
         (f2cl-lib:array-slice c-%data% single-float
                               ((+ ie 1) js) ((1 ldc) (1 *))
```

```
                                c-%offset%)
        1)
     (declare
      (ignore var-0 var-2 var-3 var-4 var-6 var-7 var-9
       var-10))
     (when var-1 (setf mb var-1))
     (when var-5 (setf ldd var-5))
     (when var-8 (setf one var-8))))))
((and (= mb 2) (= nb 2))
 (slaset "F" ldz ldz zero zero z ldz)
 (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                        a-%offset%))
 (setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (is isp1) ((1 lda) (1 *))
                        a-%offset%))
 (setf (f2cl-lib:fref z (5 1) ((1 ldz) (1 ldz)))
         (-
          (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                         b-%offset%)))
 (setf (f2cl-lib:fref z (7 1) ((1 ldz) (1 ldz)))
         (-
          (f2cl-lib:fref b-%data% (jsp1 js) ((1 ldb$) (1 *))
                         b-%offset%)))
 (setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (isp1 is) ((1 lda) (1 *))
                        a-%offset%))
 (setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (isp1 isp1) ((1 lda) (1 *))
                        a-%offset%))
 (setf (f2cl-lib:fref z (6 2) ((1 ldz) (1 ldz)))
         (-
          (f2cl-lib:fref b-%data% (js js) ((1 ldb$) (1 *))
                         b-%offset%)))
 (setf (f2cl-lib:fref z (8 2) ((1 ldz) (1 ldz)))
         (-
          (f2cl-lib:fref b-%data% (jsp1 js) ((1 ldb$) (1 *))
                         b-%offset%)))
 (setf (f2cl-lib:fref z (3 3) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (is is) ((1 lda) (1 *))
                        a-%offset%))
 (setf (f2cl-lib:fref z (4 3) ((1 ldz) (1 ldz)))
         (f2cl-lib:fref a-%data% (is isp1) ((1 lda) (1 *))
                        a-%offset%))
 (setf (f2cl-lib:fref z (5 3) ((1 ldz) (1 ldz)))
         (-
          (f2cl-lib:fref b-%data% (js jsp1) ((1 ldb$) (1 *))
                         b-%offset%)))
 (setf (f2cl-lib:fref z (7 3) ((1 ldz) (1 ldz)))
         (-
```

```
         (f2cl-lib:fref b-%data% (jsp1 jsp1)
                        ((1 ldb$) (1 *)) b-%offset%)))
(setf (f2cl-lib:fref z (3 4) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (isp1 is) ((1 lda) (1 *))
                       a-%offset%))
(setf (f2cl-lib:fref z (4 4) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref a-%data% (isp1 isp1) ((1 lda) (1 *))
                       a-%offset%))
(setf (f2cl-lib:fref z (6 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (js jsp1) ((1 ldb$) (1 *))
                        b-%offset%)))
(setf (f2cl-lib:fref z (8 4) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref b-%data% (jsp1 jsp1)
                        ((1 ldb$) (1 *)) b-%offset%)))
(setf (f2cl-lib:fref z (1 5) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                       d-%offset%))
(setf (f2cl-lib:fref z (2 5) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is isp1) ((1 ldd) (1 *))
                       d-%offset%))
(setf (f2cl-lib:fref z (5 5) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (2 6) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (isp1 isp1) ((1 ldd) (1 *))
                       d-%offset%))
(setf (f2cl-lib:fref z (6 6) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js js) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (3 7) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is is) ((1 ldd) (1 *))
                       d-%offset%))
(setf (f2cl-lib:fref z (4 7) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (is isp1) ((1 ldd) (1 *))
                       d-%offset%))
(setf (f2cl-lib:fref z (5 7) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js jsp1) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (7 7) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (jsp1 jsp1)
                        ((1 lde) (1 *)) e-%offset%)))
(setf (f2cl-lib:fref z (4 8) ((1 ldz) (1 ldz)))
        (f2cl-lib:fref d-%data% (isp1 isp1) ((1 ldd) (1 *))
                       d-%offset%))
```

```
(setf (f2cl-lib:fref z (6 8) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (js jsp1) ((1 lde) (1 *))
                        e-%offset%)))
(setf (f2cl-lib:fref z (8 8) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:fref e-%data% (jsp1 jsp1)
                        ((1 lde) (1 *)) e-%offset%)))
(setf k 1)
(setf ii (f2cl-lib:int-add (f2cl-lib:int-mul mb nb) 1))
(f2cl-lib:fdo (jj 0 (f2cl-lib:int-add jj 1))
              ((> jj
                  (f2cl-lib:int-add nb
                                    (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy mb
         (f2cl-lib:array-slice c-%data% single-float
                               (is
                                (f2cl-lib:int-add js jj))
                               ((1 ldc) (1 *)) c-%offset%)
         1
         (f2cl-lib:array-slice rhs single-float (k)
                               ((1 ldz)))
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf mb var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy mb
         (f2cl-lib:array-slice f-%data% single-float
                               (is
                                (f2cl-lib:int-add js jj))
                               ((1 ldf) (1 *)) f-%offset%)
         1
         (f2cl-lib:array-slice rhs single-float (ii)
                               ((1 ldz)))
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf mb var-0)))
    (setf k (f2cl-lib:int-add k mb))
    (setf ii (f2cl-lib:int-add ii mb))
   label160))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (sgetc2 zdim z ldz ipiv jpiv ierr)
  (declare (ignore var-1 var-3 var-4))
  (setf zdim var-0)
  (setf ldz var-2)
  (setf ierr var-5))
(if (> ierr 0)
```

```
    (setf info ierr))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (sgesc2 zdim z ldz rhs ipiv jpiv scaloc)
  (declare (ignore var-1 var-3 var-4 var-5))
  (setf zdim var-0)
  (setf ldz var-2)
  (setf scaloc var-6))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal m scaloc
           (f2cl-lib:array-slice c-%data% single-float
                                 (1 k) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (sscal m scaloc
           (f2cl-lib:array-slice f-%data% single-float
                                 (1 k) ((1 ldf) (1 *))
                                 f-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf m var-0))
        (when var-1 (setf scaloc var-1)))
     label170))
  (setf scale (* scale scaloc))))
(setf k 1)
(setf ii (f2cl-lib:int-add (f2cl-lib:int-mul mb nb) 1))
(f2cl-lib:fdo (jj 0 (f2cl-lib:int-add jj 1))
              ((> jj
                  (f2cl-lib:int-add nb
                                    (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy mb
         (f2cl-lib:array-slice rhs single-float (k)
                               ((1 ldz)))
         1
         (f2cl-lib:array-slice c-%data% single-float
                               (is
                                (f2cl-lib:int-add js jj))
                               ((1 ldc) (1 *)) c-%offset%)
```

```
          1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf mb var-0)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (scopy mb
           (f2cl-lib:array-slice rhs single-float (ii)
                                 ((1 ldz)))
           1
           (f2cl-lib:array-slice f-%data% single-float
                                 (is
                                  (f2cl-lib:int-add js jj))
                                 ((1 ldf) (1 *)) f-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf mb var-0)))
      (setf k (f2cl-lib:int-add k mb))
      (setf ii (f2cl-lib:int-add ii mb))
    label180))
  (cond
   ((> j (f2cl-lib:int-add p 2))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
         var-8 var-9 var-10 var-11 var-12)
        (sgemm "N" "T" mb (f2cl-lib:int-sub js 1) nb one
         (f2cl-lib:array-slice c-%data% single-float (is js)
                               ((1 ldc) (1 *)) c-%offset%)
         ldc
         (f2cl-lib:array-slice b-%data% single-float (1 js)
                               ((1 ldb$) (1 *)) b-%offset%)
         ldb$ one
         (f2cl-lib:array-slice f-%data% single-float (is 1)
                               ((1 ldf) (1 *)) f-%offset%)
         ldf)
      (declare (ignore var-0 var-1 var-3 var-6 var-8 var-11))
      (when var-2 (setf mb var-2))
      (when var-4 (setf nb var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf ldc var-7))
      (when var-9 (setf ldb$ var-9))
      (when var-10 (setf one var-10))
      (when var-12 (setf ldf var-12)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
         var-8 var-9 var-10 var-11 var-12)
        (sgemm "N" "T" mb (f2cl-lib:int-sub js 1) nb one
         (f2cl-lib:array-slice f-%data% single-float (is js)
                               ((1 ldf) (1 *)) f-%offset%)
         ldf
         (f2cl-lib:array-slice e-%data% single-float (1 js)
                               ((1 lde) (1 *)) e-%offset%)
```

```
        lde one
        (f2cl-lib:array-slice f-%data% single-float (is 1)
                              ((1 ldf) (1 *)) f-%offset%)
        ldf)
    (declare (ignore var-0 var-1 var-3 var-6 var-8 var-11))
    (when var-2 (setf mb var-2))
    (when var-4 (setf nb var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldf var-7))
    (when var-9 (setf lde var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldf var-12)))))
(cond
 ((< i p)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8 var-9 var-10 var-11 var-12)
      (sgemm "T" "N" (f2cl-lib:int-sub m ie) nb mb (- one)
       (f2cl-lib:array-slice a-%data% single-float
                             (is (f2cl-lib:int-add ie 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice c-%data% single-float (is js)
                             ((1 ldc) (1 *)) c-%offset%)
       ldc one
       (f2cl-lib:array-slice c-%data% single-float
                             ((+ ie 1) js) ((1 ldc) (1 *))
                             c-%offset%)
       ldc)
    (declare
     (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
    (when var-3 (setf nb var-3))
    (when var-4 (setf mb var-4))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldc var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldc var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8 var-9 var-10 var-11 var-12)
      (sgemm "T" "N" (f2cl-lib:int-sub m ie) nb mb (- one)
       (f2cl-lib:array-slice d-%data% single-float
                             (is (f2cl-lib:int-add ie 1))
                             ((1 ldd) (1 *)) d-%offset%)
       ldd
       (f2cl-lib:array-slice f-%data% single-float (is js)
                             ((1 ldf) (1 *)) f-%offset%)
       ldf one
       (f2cl-lib:array-slice c-%data% single-float
                             ((+ ie 1) js) ((1 ldc) (1 *))
```

```
                                                c-%offset%)
                          ldc)
                        (declare
                         (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
                        (when var-3 (setf nb var-3))
                        (when var-4 (setf mb var-4))
                        (when var-7 (setf ldd var-7))
                        (when var-9 (setf ldf var-9))
                        (when var-10 (setf one var-10))
                        (when var-12 (setf ldc var-12)))))))
                label190))
            label200))))
        (go end_label)
       end_label
        (return
         (values trans nil m nil nil lda nil ldb$ nil ldc nil ldd nil lde nil
                 ldf scale rdsum rdscal nil pq info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::stgsy2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (single-float)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::trans nil
                                              fortran-to-lisp::m nil nil
                                              fortran-to-lisp::lda nil
```

```
  fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldc nil
  fortran-to-lisp::ldd nil
  fortran-to-lisp::lde nil
  fortran-to-lisp::ldf
  fortran-to-lisp::scale
  fortran-to-lisp::rdsum
  fortran-to-lisp::rdscal nil
  fortran-to-lisp::pq
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::slaset
  fortran-to-lisp::slatdf
  fortran-to-lisp::sgesc2
  fortran-to-lisp::sgetc2))))
```

———-

Chapter 64

LAPACK - Symmetric Matrix, Auxilliary Routines, Complex

64.1 claesy

64.1.1 Axiom unit tests

— **claesy.input** —

```
)set break resume
)sys rm -f claesy.output
)spool claesy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

64.1.2 Axiom help page

— **claesy.help** —

```
==========================================================================
claesy examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine claesy (A, B, C, RT1, RT2, EVSCAL, CS1, SN1)
           CLAESY computes the eigenvalues and eigenvectors of a 2-by-2
           complex symmetric matrix.

Function/Subroutine Documentation
   subroutine claesy (complex A, complex B, complex C, complex RT1, complex
       RT2, complex EVSCAL, complex CS1, complex SN1)
       CLAESY computes the eigenvalues and eigenvectors of a 2-by-2 complex
       symmetric matrix.

Purpose:
   CLAESY computes the eigendecomposition of a 2-by-2 symmetric matrix
        ( ( A, B );( B, C ) )
   provided the norm of the matrix of eigenvectors is larger than
   some threshold value.

   RT1 is the eigenvalue of larger absolute value, and RT2 of
   smaller absolute value.  If the eigenvectors are computed, then
   on return ( CS1, SN1 ) is the unit eigenvector for RT1, hence

         [  CS1     SN1   ] . [ A  B ] . [ CS1    -SN1   ] = [ RT1  0  ]
         [ -SN1     CS1   ]   [ B  C ]   [ SN1     CS1   ]   [  0  RT2 ]

Parameters:

 A
   A is COMPLEX
   The ( 1, 1 ) element of input matrix.

 B
   B is COMPLEX
   The ( 1, 2 ) element of input matrix.  The ( 2, 1 ) element
   is also given by B, since the 2-by-2 matrix is symmetric.

 C
   C is COMPLEX
   The ( 2, 2 ) element of input matrix.

 RT1
   RT1 is COMPLEX
   The eigenvalue of larger modulus.

 RT2
```

```
  RT2 is COMPLEX
  The eigenvalue of smaller modulus.

EVSCAL
  EVSCAL is COMPLEX
  The complex value by which the eigenvector matrix was scaled
  to make it orthonormal.  If EVSCAL is zero, the eigenvectors
  were not computed.  This means one of two things:  the 2-by-2
  matrix could not be diagonalized, or the norm of the matrix
  of eigenvectors before scaling was larger than the threshold
  value THRESH (set below).

CS1
  CS1 is COMPLEX

SN1
  SN1 is COMPLEX
  If EVSCAL .NE. 0,  ( CS1, SN1 ) is the unit right eigenvector
  for RT1.
```

64.1.3 fortran code

— claesy.f —

```
*  =====================================================================
      SUBROUTINE CLAESY( A, B, C, RT1, RT2, EVSCAL, CS1, SN1 )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      COMPLEX            A, B, C, CS1, EVSCAL, RT1, RT2, SN1
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E0 )
      REAL               ONE
      PARAMETER          ( ONE = 1.0E0 )
      COMPLEX            CONE
      PARAMETER          ( CONE = ( 1.0E0, 0.0E0 ) )
```

```
      REAL               HALF
      PARAMETER          ( HALF = 0.5E0 )
      REAL               THRESH
      PARAMETER          ( THRESH = 0.1E0 )
*     ..
*     .. Local Scalars ..
      REAL               BABS, EVNORM, TABS, Z
      COMPLEX            S, T, TMP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*
*     Special case:  The matrix is actually diagonal.
*     To avoid divide by zero later, we treat this case separately.
*
      IF( ABS( B ).EQ.ZERO ) THEN
         RT1 = A
         RT2 = C
         IF( ABS( RT1 ).LT.ABS( RT2 ) ) THEN
            TMP = RT1
            RT1 = RT2
            RT2 = TMP
            CS1 = ZERO
            SN1 = ONE
         ELSE
            CS1 = ONE
            SN1 = ZERO
         END IF
      ELSE
*
*        Compute the eigenvalues and eigenvectors.
*        The characteristic equation is
*           lambda **2 - (A+C) lambda + (A*C - B*B)
*        and we solve it using the quadratic formula.
*
         S = ( A+C )*HALF
         T = ( A-C )*HALF
*
*        Take the square root carefully to avoid over/under flow.
*
         BABS = ABS( B )
         TABS = ABS( T )
         Z = MAX( BABS, TABS )
         IF( Z.GT.ZERO )
     $      T = Z*SQRT( ( T / Z )**2+( B / Z )**2 )
*
*        Compute the two eigenvalues.  RT1 and RT2 are exchanged
```

```
*        if necessary so that RT1 will have the greater magnitude.
*
         RT1 = S + T
         RT2 = S - T
         IF( ABS( RT1 ).LT.ABS( RT2 ) ) THEN
            TMP = RT1
            RT1 = RT2
            RT2 = TMP
         END IF
*
*        Choose CS1 = 1 and SN1 to satisfy the first equation, then
*        scale the components of this eigenvector so that the matrix
*        of eigenvectors X satisfies  X * X**T = I .  (No scaling is
*        done if the norm of the eigenvalue matrix is less than THRESH.)
*
         SN1 = ( RT1-A ) / B
         TABS = ABS( SN1 )
         IF( TABS.GT.ONE ) THEN
            T = TABS*SQRT( ( ONE / TABS )**2+( SN1 / TABS )**2 )
         ELSE
            T = SQRT( CONE+SN1*SN1 )
         END IF
         EVNORM = ABS( T )
         IF( EVNORM.GE.THRESH ) THEN
            EVSCAL = CONE / T
            CS1 = EVSCAL
            SN1 = SN1*EVSCAL
         ELSE
            EVSCAL = ZERO
         END IF
      END IF
      RETURN
*
*     End of CLAESY
*
      END
```

———-

64.1.4 lisp code

— **claesy.lisp** —

```
(let* ((zero 0.0)
       (one 1.0)
       (cone (f2cl-lib:cmplx 1.0 0.0))
       (half 0.5)
       (thresh 0.1))
```

```
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (f2cl-lib:complex8) cone)
           (type (single-float 0.5 0.5) half)
           (type (single-float 0.1 0.1) thresh)
           (ignorable zero one cone half thresh))
  (defun claesy (a b c rt1 rt2 evscal cs1 sn1)
    (declare (type (f2cl-lib:complex8) sn1 cs1 evscal rt2 rt1 c b a))
    (prog ((s #C(0.0 0.0)) (t$ #C(0.0 0.0)) (tmp #C(0.0 0.0)) (babs 0.0)
           (evnorm 0.0) (tabs 0.0) (z 0.0))
      (declare (type (f2cl-lib:complex8) s t$ tmp)
               (type (single-float) babs evnorm tabs z))
      (cond
       ((= (abs b) zero) (setf rt1 a) (setf rt2 c)
        (cond
         ((< (abs rt1) (abs rt2)) (setf tmp rt1) (setf rt1 rt2) (setf rt2 tmp)
          (setf cs1 (coerce zero 'f2cl-lib:complex8))
          (setf sn1 (coerce one 'f2cl-lib:complex8)))
         (t (setf cs1 (coerce one 'f2cl-lib:complex8))
          (setf sn1 (coerce zero 'f2cl-lib:complex8)))))
       (t (setf s (* (+ a c) half)) (setf t$ (* (- a c) half))
        (setf babs (abs b)) (setf tabs (abs t$)) (setf z (max babs tabs))
        (if (> z zero)
            (setf t$
                    (* z
                       (f2cl-lib:fsqrt
                        (+ (expt (/ t$ z) 2) (expt (/ b z) 2))))))
        (setf rt1 (+ s t$)) (setf rt2 (- s t$))
        (cond
         ((< (abs rt1) (abs rt2)) (setf tmp rt1) (setf rt1 rt2)
          (setf rt2 tmp)))
        (setf sn1 (/ (- rt1 a) b)) (setf tabs (abs sn1))
        (cond
         ((> tabs one)
          (setf t$
                  (* tabs
                     (f2cl-lib:fsqrt
                      (+ (expt (/ one tabs) 2) (expt (/ sn1 tabs) 2))))))
         (t (setf t$ (f2cl-lib:fsqrt (+ cone (* sn1 sn1))))))
        (setf evnorm (abs t$))
        (cond
         ((>= evnorm thresh) (setf evscal (/ cone t$)) (setf cs1 evscal)
          (setf sn1 (* sn1 evscal)))
         (t (setf evscal (coerce zero 'f2cl-lib:complex8))))))
      (go end_label)
     end_label
      (return (values nil nil nil rt1 rt2 evscal cs1 sn1)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
```

```
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::claesy
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::complex8)
                                              (fortran-to-lisp::complex8)
                                              (fortran-to-lisp::complex8)
                                              (fortran-to-lisp::complex8)
                                              (fortran-to-lisp::complex8)
                                              (fortran-to-lisp::complex8)
                                              (fortran-to-lisp::complex8)
                                              (fortran-to-lisp::complex8))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::rt1
                                              fortran-to-lisp::rt2
                                              fortran-to-lisp::evscal
                                              fortran-to-lisp::cs1
                                              fortran-to-lisp::sn1)
                                            :calls 'nil)))
```

———-

64.2 clansy

64.2.1 Axiom unit tests

— **clansy.input** —

```
)set break resume
)sys rm -f clansy.output
)spool clansy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

64.2.2 Axiom help page

— **clansy.help** —

```
====================================================================
clansy examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       real function clansy (NORM, UPLO, N, A, LDA, WORK)
           CLANSY returns the value of the 1-norm, or the Frobenius norm, or
           the infinity norm, or the element of largest absolute value of a
           complex symmetric matrix.

Function/Subroutine Documentation
   real function clansy (character NORM, character UPLO, integer N, complex,
       dimension( lda, * ) A, integer LDA, real, dimension( * ) WORK)
       CLANSY returns the value of the 1-norm, or the Frobenius norm, or the
       infinity norm, or the element of largest absolute value of a complex
       symmetric matrix.

Purpose:
   CLANSY  returns the value of the one norm,  or the Frobenius norm, or
   the  infinity norm,  or the  element of  largest absolute value  of a
   complex symmetric matrix A.

 Returns:

   CLANSY

        CLANSY = ( max(abs(A(i,j))), NORM = 'M' or 'm'
                 (
                 ( norm1(A),         NORM = '1', 'O' or 'o'
                 (
                 ( normI(A),         NORM = 'I' or 'i'
                 (
                 ( normF(A),         NORM = 'F', 'f', 'E' or 'e'

   where  norm1  denotes the  one norm of a matrix (maximum column sum),
   normI  denotes the  infinity norm  of a matrix  (maximum row sum) and
   normF  denotes the  Frobenius norm of a matrix (square root of sum of
   squares).  Note that  max(abs(A(i,j)))  is not a consistent matrix norm.

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies the value to be returned in CLANSY as described above.
```

```
 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is to be referenced.
   = 'U':  Upper triangular part of A is referenced
   = 'L':  Lower triangular part of A is referenced

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.  When N = 0, CLANSY is set to zero.

 A
   A is COMPLEX array, dimension (LDA,N)
   The symmetric matrix A.  If UPLO = 'U', the leading n by n
   upper triangular part of A contains the upper triangular part
   of the matrix A, and the strictly lower triangular part of A
   is not referenced.  If UPLO = 'L', the leading n by n lower
   triangular part of A contains the lower triangular part of
   the matrix A, and the strictly upper triangular part of A is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(N,1).

 WORK
   WORK is REAL array, dimension (MAX(1,LWORK)),
   where LWORK >= N when NORM = 'I' or '1' or 'O'; otherwise,
   WORK is not referenced.
```

———-

64.2.3 fortran code

— clansy.f —

```
*  =====================================================================
      REAL             FUNCTION CLANSY( NORM, UPLO, N, A, LDA, WORK )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          NORM, UPLO
      INTEGER            LDA, N
```

```
*     ..
*     .. Array Arguments ..
      REAL               WORK( * )
      COMPLEX            A( LDA, * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               ABSA, SCALE, SUM, VALUE
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, SISNAN
      EXTERNAL           LSAME, SISNAN
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLASSQ
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SQRT
*     ..
*     .. Executable Statements ..
*
      IF( N.EQ.0 ) THEN
         VALUE = ZERO
      ELSE IF( LSAME( NORM, 'M' ) ) THEN
*
*        Find max(abs(A(i,j))).
*
         VALUE = ZERO
         IF( LSAME( UPLO, 'U' ) ) THEN
            DO 20 J = 1, N
               DO 10 I = 1, J
                  SUM = ABS( A( I, J ) )
                  IF( VALUE .LT. SUM .OR. SISNAN( SUM ) ) VALUE = SUM
   10          CONTINUE
   20       CONTINUE
         ELSE
            DO 40 J = 1, N
               DO 30 I = J, N
                  SUM = ABS( A( I, J ) )
                  IF( VALUE .LT. SUM .OR. SISNAN( SUM ) ) VALUE = SUM
   30          CONTINUE
   40       CONTINUE
         END IF
```

```
      ELSE IF( ( LSAME( NORM, 'I' ) ) .OR. ( LSAME( NORM, 'O' ) ) .OR.
     $         ( NORM.EQ.'1' ) ) THEN
*
*        Find normI(A) ( = norm1(A), since A is symmetric).
*
         VALUE = ZERO
         IF( LSAME( UPLO, 'U' ) ) THEN
            DO 60 J = 1, N
               SUM = ZERO
               DO 50 I = 1, J - 1
                  ABSA = ABS( A( I, J ) )
                  SUM = SUM + ABSA
                  WORK( I ) = WORK( I ) + ABSA
   50          CONTINUE
               WORK( J ) = SUM + ABS( A( J, J ) )
   60       CONTINUE
            DO 70 I = 1, N
               SUM = WORK( I )
               IF( VALUE .LT. SUM .OR. SISNAN( SUM ) ) VALUE = SUM
   70       CONTINUE
         ELSE
            DO 80 I = 1, N
               WORK( I ) = ZERO
   80       CONTINUE
            DO 100 J = 1, N
               SUM = WORK( J ) + ABS( A( J, J ) )
               DO 90 I = J + 1, N
                  ABSA = ABS( A( I, J ) )
                  SUM = SUM + ABSA
                  WORK( I ) = WORK( I ) + ABSA
   90          CONTINUE
               IF( VALUE .LT. SUM .OR. SISNAN( SUM ) ) VALUE = SUM
  100       CONTINUE
         END IF
      ELSE IF( ( LSAME( NORM, 'F' ) ) .OR. ( LSAME( NORM, 'E' ) ) ) THEN
*
*        Find normF(A).
*
         SCALE = ZERO
         SUM = ONE
         IF( LSAME( UPLO, 'U' ) ) THEN
            DO 110 J = 2, N
               CALL CLASSQ( J-1, A( 1, J ), 1, SCALE, SUM )
  110       CONTINUE
         ELSE
            DO 120 J = 1, N - 1
               CALL CLASSQ( N-J, A( J+1, J ), 1, SCALE, SUM )
  120       CONTINUE
         END IF
         SUM = 2*SUM
```

```
         CALL CLASSQ( N, A, LDA+1, SCALE, SUM )
         VALUE = SCALE*SQRT( SUM )
      END IF
*
      CLANSY = VALUE
      RETURN
*
*     End of CLANSY
*
      END
```

———-

64.2.4 lisp code

— **clansy.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun clansy (norm uplo n a lda work)
    (declare (type (array single-float (*)) work)
             (type (array f2cl-lib:complex8 (*)) a)
             (type (f2cl-lib:integer4) lda n)
             (type (string *) uplo norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((absa 0.0) (scale 0.0) (sum 0.0) (value 0.0) (i 0) (j 0)
             (clansy 0.0))
        (declare (type (f2cl-lib:integer4) i j)
                 (type (single-float) absa scale sum value clansy))
        (cond ((= n 0) (setf value zero))
              ((multiple-value-bind (ret-val var-0 var-1)
                   (lsame norm "M")
                 (declare (ignore var-1))
                 (when var-0 (setf norm var-0))
                 ret-val)
               (setf value zero)
               (cond
                ((multiple-value-bind (ret-val var-0 var-1)
                     (lsame uplo "U")
                   (declare (ignore var-1))
                   (when var-0 (setf uplo var-0))
                   ret-val)
```

```
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf sum
                      (abs
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)))
              (if (or (< value sum)
                      (multiple-value-bind (ret-val var-0)
                          (sisnan sum)
                        (declare (ignore))
                        (when var-0 (setf sum var-0))
                        ret-val))
                  (setf value sum))
             label10))
         label20)))
    (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf sum
                      (abs
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)))
              (if (or (< value sum)
                      (multiple-value-bind (ret-val var-0)
                          (sisnan sum)
                        (declare (ignore))
                        (when var-0 (setf sum var-0))
                        ret-val))
                  (setf value sum))
             label30))
         label40)))))
  ((or
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame norm "I")
      (declare (ignore var-1))
      (when var-0 (setf norm var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame norm "O")
      (declare (ignore var-1))
      (when var-0 (setf norm var-0))
      ret-val)
```

```
 (f2cl-lib:fstring-= norm "1"))
(setf value zero)
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame uplo "U")
    (declare (ignore var-1))
    (when var-0 (setf uplo var-0))
    ret-val)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf sum zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (f2cl-lib:int-add j
                                          (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf absa
                  (abs
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)))
          (setf sum (+ sum absa))
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                  (+
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                   absa))
         label50))
      (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                           work-%offset%)
              (+ sum
                 (abs
                  (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                 a-%offset%))))
    label60))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf sum
              (f2cl-lib:fref work-%data% (i) ((1 *))
                             work-%offset%))
      (if (or (< value sum)
              (multiple-value-bind (ret-val var-0)
                  (sisnan sum)
                (declare (ignore))
                (when var-0 (setf sum var-0))
                ret-val))
          (setf value sum))
```

```
        label70)))
    (t
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                              work-%offset%)
               zero)
        label80))
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf sum
                 (+
                  (f2cl-lib:fref work-%data% (j) ((1 *))
                                 work-%offset%)
                  (abs
                   (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                  a-%offset%))))
         (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                          (f2cl-lib:int-add i 1))
                        ((> i n) nil)
           (tagbody
             (setf absa
                     (abs
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%)))
             (setf sum (+ sum absa))
             (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                     (+
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%)
                      absa))
            label90))
         (if (or (< value sum)
                 (multiple-value-bind (ret-val var-0)
                     (sisnan sum)
                   (declare (ignore))
                   (when var-0 (setf sum var-0))
                   ret-val))
             (setf value sum))
        label100)))))
  ((or
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame norm "F")
      (declare (ignore var-1))
      (when var-0 (setf norm var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
```

```
                (lsame norm "E")
              (declare (ignore var-1))
              (when var-0 (setf norm var-0))
              ret-val))
          (setf scale zero) (setf sum one)
          (cond
           ((multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
             (declare (ignore var-1))
             (when var-0 (setf uplo var-0))
             ret-val)
            (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
             (tagbody
               (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                   (classq (f2cl-lib:int-sub j 1)
                    (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                          (1 j) ((1 lda) (1 *))
                                          a-%offset%)
                    1 scale sum)
                 (declare (ignore var-0 var-1 var-2))
                 (when var-3 (setf scale var-3))
                 (when var-4 (setf sum var-4)))
              label110)))
           (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                           nil)
             (tagbody
               (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                   (classq (f2cl-lib:int-sub n j)
                    (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                          ((+ j 1) j) ((1 lda) (1 *))
                                          a-%offset%)
                    1 scale sum)
                 (declare (ignore var-0 var-1 var-2))
                 (when var-3 (setf scale var-3))
                 (when var-4 (setf sum var-4)))
              label120))))
          (setf sum (* 2 sum))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (classq n a (f2cl-lib:int-add lda 1) scale sum)
            (declare (ignore var-1 var-2))
            (when var-0 (setf n var-0))
            (when var-3 (setf scale var-3))
            (when var-4 (setf sum var-4)))
          (setf value (* scale (f2cl-lib:fsqrt sum)))))
 (setf clansy value)
 (go end_label)
end_label
```

```
        (return (values clansy norm uplo n nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::clansy
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*)))
                                            :return-values
                                            '(fortran-to-lisp::norm
                                              fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil nil nil)
                                            :calls 'nil)))
```

———-

64.3 claqsy

64.3.1 Axiom unit tests

— **claqsy.input** —

```
)set break resume
)sys rm -f claqsy.output
)spool claqsy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

64.3.2 Axiom help page

— **claqsy.help** —

```
====================================================================
claqsy examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine claqsy (UPLO, N, A, LDA, S, SCOND, AMAX, EQUED)
           CLAQSY scales a symmetric/Hermitian matrix, using scaling factors
           computed by spoequ.

Function/Subroutine Documentation
   subroutine claqsy (character UPLO, integer N, complex, dimension( lda, * )
       A, integer LDA, real, dimension( * ) S, real SCOND, real AMAX,
       character EQUED)
       CLAQSY scales a symmetric/Hermitian matrix, using scaling factors
       computed by spoequ.

Purpose:
   CLAQSY equilibrates a symmetric matrix A using the scaling factors
   in the vector S.

 Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is stored.
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   n by n upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading n by n lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

   On exit, if EQUED = 'Y', the equilibrated matrix:
   diag(S) * A * diag(S).
```

```
 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(N,1).

 S
   S is REAL array, dimension (N)
   The scale factors for A.

 SCOND
   SCOND is REAL
   Ratio of the smallest S(i) to the largest S(i).

 AMAX
   AMAX is REAL
   Absolute value of largest matrix entry.

 EQUED
   EQUED is CHARACTER*1
   Specifies whether or not equilibration was done.
   = 'N':  No equilibration.
   = 'Y':  Equilibration was done, i.e., A has been replaced by
           diag(S) * A * diag(S).

Internal Parameters:

   THRESH is a threshold value used to decide if scaling should be done
   based on the ratio of the scaling factors.  If SCOND < THRESH,
   scaling is done.

   LARGE and SMALL are threshold values used to decide if scaling should
   be done based on the absolute size of the largest matrix element.
   If AMAX > LARGE or AMAX < SMALL, scaling is done.
```

———-

64.3.3 fortran code

— claqsy.f —

```
*  ===================================================================
      SUBROUTINE CLAQSY( UPLO, N, A, LDA, S, SCOND, AMAX, EQUED )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
```

```
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, UPLO
      INTEGER            LDA, N
      REAL               AMAX, SCOND
*     ..
*     .. Array Arguments ..
      REAL               S( * )
      COMPLEX            A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, THRESH
      PARAMETER          ( ONE = 1.0E+0, THRESH = 0.1E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               CJ, LARGE, SMALL
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH
      EXTERNAL           LSAME, SLAMCH
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( N.LE.0 ) THEN
         EQUED = 'N'
         RETURN
      END IF
*
*     Initialize LARGE and SMALL.
*
      SMALL = SLAMCH( 'Safe minimum' ) / SLAMCH( 'Precision' )
      LARGE = ONE / SMALL
*
      IF( SCOND.GE.THRESH .AND. AMAX.GE.SMALL .AND. AMAX.LE.LARGE ) THEN
*
*        No equilibration
*
         EQUED = 'N'
      ELSE
*
*        Replace A by diag(S) * A * diag(S).
*
         IF( LSAME( UPLO, 'U' ) ) THEN
```

```
*
*          Upper triangle of A is stored.
*
           DO 20 J = 1, N
              CJ = S( J )
              DO 10 I = 1, J
                 A( I, J ) = CJ*S( I )*A( I, J )
   10         CONTINUE
   20      CONTINUE
         ELSE
*
*          Lower triangle of A is stored.
*
           DO 40 J = 1, N
              CJ = S( J )
              DO 30 I = J, N
                 A( I, J ) = CJ*S( I )*A( I, J )
   30         CONTINUE
   40      CONTINUE
         END IF
         EQUED = 'Y'
      END IF
*
      RETURN
*
*     End of CLAQSY
*
      END
```

———-

64.3.4 lisp code

— **claqsy.lisp** —

```
(let* ((one 1.0) (thresh 0.1))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.1 0.1) thresh)
           (ignorable one thresh))
  (defun claqsy (uplo n a lda s scond amax equed)
    (declare (type (single-float) amax scond)
             (type (array single-float (*)) s)
             (type (array f2cl-lib:complex8 (*)) a)
             (type (f2cl-lib:integer4) lda n)
             (type (string *) equed uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (equed character equed-%data% equed-%offset%)
```

```
    (a f2cl-lib:complex8 a-%data% a-%offset%)
    (s single-float s-%data% s-%offset%))
  (prog ((cj 0.0) (large 0.0) (small 0.0) (i 0) (j 0))
    (declare (type (single-float) cj large small)
             (type (f2cl-lib:integer4) i j))
    (cond
     ((<= n 0) (f2cl-lib:f2cl-set-string equed "N" (string 1))
      (go end_label)))
    (setf small (/ (slamch "Safe minimum") (slamch "Precision")))
    (setf large (/ one small))
    (cond
     ((and (>= scond thresh) (>= amax small) (<= amax large))
      (f2cl-lib:f2cl-set-string equed "N" (string 1)))
     (t
      (cond
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf cj (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i j) nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%)
                      (* cj
                         (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)))
               label10))
           label20)))
       (t
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf cj (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
            (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%)
                      (* cj
                         (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)))
               label30))
```

```
              label40))))
          (f2cl-lib:f2cl-set-string equed "Y" (string 1))))
        (go end_label)
       end_label
        (return (values uplo nil nil nil nil nil nil equed))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::claqsy
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (string))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil nil
                                              nil nil nil
                                              fortran-to-lisp::equed)
                                            :calls 'nil)))
```

———-

64.4 csymv

64.4.1 Axiom unit tests

— **csymv.input** —

```
)set break resume
)sys rm -f csymv.output
)spool csymv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

64.4.2 Axiom help page

— csymv.help —

```
==================================================================
csymv examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csymv (UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)
           CSYMV computes a matrix-vector product for a complex symmetric
           matrix.

Function/Subroutine Documentation
   subroutine csymv (character UPLO, integer N, complex ALPHA, complex,
       dimension( lda, * ) A, integer LDA, complex, dimension( * ) X, integer
       INCX, complex BETA, complex, dimension( * ) Y, integer INCY)
       CSYMV computes a matrix-vector product for a complex symmetric matrix.

Purpose:
   CSYMV  performs the matrix-vector  operation

   y := alpha*A*x + beta*y,

   where alpha and beta are scalars, x and y are n element vectors and
   A is an n by n symmetric matrix.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the upper or lower
   triangular part of the array A is to be referenced as
   follows:

   UPLO = 'U' or 'u'   Only the upper triangular part of A is to be referenced.
   UPLO = 'L' or 'l'   Only the lower triangular part of A is to be referenced.

   Unchanged on exit.

 N
   N is INTEGER
   On entry, N specifies the order of the matrix A.
   N must be at least zero.
```

```
  Unchanged on exit.

ALPHA
  ALPHA is COMPLEX
  On entry, ALPHA specifies the scalar alpha.
  Unchanged on exit.

A
  A is COMPLEX array, dimension ( LDA, N )
  Before entry, with  UPLO = 'U' or 'u', the leading n by n
  upper triangular part of the array A must contain the upper
  triangular part of the symmetric matrix and the strictly
  lower triangular part of A is not referenced.
  Before entry, with UPLO = 'L' or 'l', the leading n by n
  lower triangular part of the array A must contain the lower
  triangular part of the symmetric matrix and the strictly
  upper triangular part of A is not referenced.
  Unchanged on exit.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, N ).
  Unchanged on exit.

X
  X is COMPLEX array, dimension at least ( 1 + ( N - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the N-element vector x.
  Unchanged on exit.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of
  X. INCX must not be zero.
  Unchanged on exit.

BETA
  BETA is COMPLEX
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.
  Unchanged on exit.

Y
  Y is COMPLEX array, dimension at least
  ( 1 + ( N - 1 )*abs( INCY ) ).
  Before entry, the incremented array Y must contain the n
  element vector y. On exit, Y is overwritten by the updated vector y.

INCY
  INCY is INTEGER
```

```
On entry, INCY specifies the increment for the elements of
Y. INCY must not be zero.
Unchanged on exit.
```

———-

64.4.3 fortran code

— **csymv.f** —

```
*  =====================================================================
      SUBROUTINE CSYMV( UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INCX, INCY, LDA, N
      COMPLEX            ALPHA, BETA
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), X( * ), Y( * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      COMPLEX            ONE
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ) )
      COMPLEX            ZERO
      PARAMETER          ( ZERO = ( 0.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, INFO, IX, IY, J, JX, JY, KX, KY
      COMPLEX            TEMP1, TEMP2
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = 1
      ELSE IF( N.LT.0 ) THEN
         INFO = 2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = 5
      ELSE IF( INCX.EQ.0 ) THEN
         INFO = 7
      ELSE IF( INCY.EQ.0 ) THEN
         INFO = 10
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CSYMV ', INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( ( N.EQ.0 ) .OR. ( ( ALPHA.EQ.ZERO ) .AND. ( BETA.EQ.ONE ) ) )
     $   RETURN
*
*     Set up the start points in  X  and  Y.
*
      IF( INCX.GT.0 ) THEN
         KX = 1
      ELSE
         KX = 1 - ( N-1 )*INCX
      END IF
      IF( INCY.GT.0 ) THEN
         KY = 1
      ELSE
         KY = 1 - ( N-1 )*INCY
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the triangular part
*     of A.
*
*     First form  y := beta*y.
*
      IF( BETA.NE.ONE ) THEN
         IF( INCY.EQ.1 ) THEN
            IF( BETA.EQ.ZERO ) THEN
```

```
            DO 10 I = 1, N
               Y( I ) = ZERO
   10       CONTINUE
         ELSE
            DO 20 I = 1, N
               Y( I ) = BETA*Y( I )
   20       CONTINUE
         END IF
      ELSE
         IY = KY
         IF( BETA.EQ.ZERO ) THEN
            DO 30 I = 1, N
               Y( IY ) = ZERO
               IY = IY + INCY
   30       CONTINUE
         ELSE
            DO 40 I = 1, N
               Y( IY ) = BETA*Y( IY )
               IY = IY + INCY
   40       CONTINUE
         END IF
      END IF
   END IF
   IF( ALPHA.EQ.ZERO )
  $   RETURN
   IF( LSAME( UPLO, 'U' ) ) THEN
*
*        Form  y  when A is stored in upper triangle.
*
      IF( ( INCX.EQ.1 ) .AND. ( INCY.EQ.1 ) ) THEN
         DO 60 J = 1, N
            TEMP1 = ALPHA*X( J )
            TEMP2 = ZERO
            DO 50 I = 1, J - 1
               Y( I ) = Y( I ) + TEMP1*A( I, J )
               TEMP2 = TEMP2 + A( I, J )*X( I )
   50       CONTINUE
            Y( J ) = Y( J ) + TEMP1*A( J, J ) + ALPHA*TEMP2
   60    CONTINUE
      ELSE
         JX = KX
         JY = KY
         DO 80 J = 1, N
            TEMP1 = ALPHA*X( JX )
            TEMP2 = ZERO
            IX = KX
            IY = KY
            DO 70 I = 1, J - 1
               Y( IY ) = Y( IY ) + TEMP1*A( I, J )
               TEMP2 = TEMP2 + A( I, J )*X( IX )
```

```
                 IX = IX + INCX
                 IY = IY + INCY
   70         CONTINUE
              Y( JY ) = Y( JY ) + TEMP1*A( J, J ) + ALPHA*TEMP2
              JX = JX + INCX
              JY = JY + INCY
   80       CONTINUE
         END IF
      ELSE
*
*        Form  y  when A is stored in lower triangle.
*
         IF( ( INCX.EQ.1 ) .AND. ( INCY.EQ.1 ) ) THEN
            DO 100 J = 1, N
               TEMP1 = ALPHA*X( J )
               TEMP2 = ZERO
               Y( J ) = Y( J ) + TEMP1*A( J, J )
               DO 90 I = J + 1, N
                  Y( I ) = Y( I ) + TEMP1*A( I, J )
                  TEMP2 = TEMP2 + A( I, J )*X( I )
   90          CONTINUE
               Y( J ) = Y( J ) + ALPHA*TEMP2
  100       CONTINUE
         ELSE
            JX = KX
            JY = KY
            DO 120 J = 1, N
               TEMP1 = ALPHA*X( JX )
               TEMP2 = ZERO
               Y( JY ) = Y( JY ) + TEMP1*A( J, J )
               IX = JX
               IY = JY
               DO 110 I = J + 1, N
                  IX = IX + INCX
                  IY = IY + INCY
                  Y( IY ) = Y( IY ) + TEMP1*A( I, J )
                  TEMP2 = TEMP2 + A( I, J )*X( IX )
  110          CONTINUE
               Y( JY ) = Y( JY ) + ALPHA*TEMP2
               JX = JX + INCX
               JY = JY + INCY
  120       CONTINUE
         END IF
      END IF
*
      RETURN
*
*     End of CSYMV
*
      END
```

———-

64.4.4 lisp code

— **csymv.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)) (zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (type (f2cl-lib:complex8) zero)
           (ignorable one zero))
  (defun csymv (uplo n alpha a lda x incx beta y incy)
    (declare (type (array f2cl-lib:complex8 (*)) y x a)
             (type (f2cl-lib:complex8) beta alpha)
             (type (f2cl-lib:integer4) incy incx lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%)
         (y f2cl-lib:complex8 y-%data% y-%offset%))
      (prog ((temp1 #C(0.0 0.0)) (temp2 #C(0.0 0.0)) (i 0) (info 0) (ix 0)
             (iy 0) (j 0) (jx 0) (jy 0) (kx 0) (ky 0))
        (declare (type (f2cl-lib:complex8) temp1 temp2)
                 (type (f2cl-lib:integer4) i info ix iy j jx jy kx ky))
        (setf info 0)
        (cond
         ((and
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
          (setf info 1))
         ((< n 0) (setf info 2))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info 5))
         ((= incx 0) (setf info 7)) ((= incy 0) (setf info 10)))
        (cond
         ((/= info 0)
          (multiple-value-bind (var-0 var-1)
              (xerbla "CSYMV " info)
```

```
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= n 0) (and (= alpha zero) (= beta one)))
    (go end_label))
(cond ((> incx 0) (setf kx 1))
      (t
       (setf kx
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incx)))))
(cond ((> incy 0) (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incy)))))
(cond
 ((/= beta one)
  (cond
   ((= incy 1)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%) zero)
         label10)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                  (* beta
                     (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)))
         label20)))))
   (t (setf iy ky)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) zero)
          (setf iy (f2cl-lib:int-add iy incy))
         label30)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
```

```
                    (* beta
                       (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)))
          (setf iy (f2cl-lib:int-add iy incy))
         label40))))))))
(if (= alpha zero)
    (go end_label))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame uplo "U")
    (declare (ignore var-1))
    (when var-0 (setf uplo var-0))
    ret-val)
  (cond
   ((and (= incx 1) (= incy 1))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp1
                (* alpha
                   (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
        (setf temp2 zero)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                       (* temp1
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%))))
            (setf temp2
                    (+ temp2
                       (*
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%)
                        (f2cl-lib:fref x-%data% (i) ((1 *))
                                       x-%offset%))))
           label50))
        (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                   (* temp1
                      (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%))
                   (* alpha temp2)))
       label60)))
   (t (setf jx kx) (setf jy ky)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp1
```

```
                (* alpha
                   (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
        (setf temp2 zero)
        (setf ix kx)
        (setf iy ky)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (* temp1
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%))))
            (setf temp2
                    (+ temp2
                       (*
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%)
                        (f2cl-lib:fref x-%data% (ix) ((1 *))
                                       x-%offset%))))
            (setf ix (f2cl-lib:int-add ix incx))
            (setf iy (f2cl-lib:int-add iy incy))
           label70))
        (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                   (* temp1
                      (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%))
                   (* alpha temp2)))
        (setf jx (f2cl-lib:int-add jx incx))
        (setf jy (f2cl-lib:int-add jy incy))
       label80)))))
(t
 (cond
  ((and (= incx 1) (= incy 1))
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf temp1
               (* alpha
                  (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
       (setf temp2 zero)
       (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
               (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                  (* temp1
                     (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                    a-%offset%))))
       (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                     ((> i n) nil)
```

```
            (tagbody
              (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                      (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                         (* temp1
                            (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                           a-%offset%))))
              (setf temp2
                      (+ temp2
                         (*
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref x-%data% (i) ((1 *))
                                         x-%offset%))))
             label90))
          (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                  (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                     (* alpha temp2)))
        label100)))
      (t (setf jx kx) (setf jy ky)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (setf temp1
                   (* alpha
                      (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
           (setf temp2 zero)
           (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                   (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                      (* temp1
                         (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                        a-%offset%))))
           (setf ix jx)
           (setf iy jy)
           (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf ix (f2cl-lib:int-add ix incx))
               (setf iy (f2cl-lib:int-add iy incy))
               (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                       (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                          (* temp1
                             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                            a-%offset%))))
               (setf temp2
                       (+ temp2
                          (*
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%)
                           (f2cl-lib:fref x-%data% (ix) ((1 *))
                                          x-%offset%))))
```

```
                 label110))
               (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                       (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                          (* alpha temp2)))
               (setf jx (f2cl-lib:int-add jx incx))
               (setf jy (f2cl-lib:int-add jy incy))
              label120))))))
        (go end_label)
       end_label
        (return (values uplo nil nil nil nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csymv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::complex8)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::complex8)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil nil
                                              nil nil nil nil nil nil)
                                            :calls 'nil)))
```

———-

64.5 csyr

64.5.1 Axiom unit tests

— csyr.input —

```
)set break resume
)sys rm -f csyr.output
)spool csyr.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

64.5.2 Axiom help page

— **csyr.help** —

```
====================================================================
csyr examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csyr (UPLO, N, ALPHA, X, INCX, A, LDA)
           CSYR performs the symmetric rank-1 update of a complex symmetric
           matrix.

Function/Subroutine Documentation
   subroutine csyr (character UPLO, integer N, complex ALPHA, complex,
       dimension( * ) X, integer INCX, complex, dimension( lda, * ) A, integer
       LDA)
       CSYR performs the symmetric rank-1 update of a complex symmetric
       matrix.

Purpose:
   CSYR   performs the symmetric rank 1 operation

   A := alpha*x*x**H + A,

   where alpha is a complex scalar, x is an n element vector and A is an
   n by n symmetric matrix.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the upper or lower
   triangular part of the array A is to be referenced as
```

```
  follows:

  UPLO = 'U' or 'u'   Only the upper triangular part of A is to be referenced.
  UPLO = 'L' or 'l'   Only the lower triangular part of A is to be referenced.

  Unchanged on exit.

N
  N is INTEGER
  On entry, N specifies the order of the matrix A.
  N must be at least zero.
  Unchanged on exit.

ALPHA
  ALPHA is COMPLEX
  On entry, ALPHA specifies the scalar alpha.
  Unchanged on exit.

X
  X is COMPLEX array, dimension at least ( 1 + ( N - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the N-
  element vector x.
  Unchanged on exit.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of
  X. INCX must not be zero.
  Unchanged on exit.

A
  A is COMPLEX array, dimension ( LDA, N )
  Before entry, with  UPLO = 'U' or 'u', the leading n by n
  upper triangular part of the array A must contain the upper
  triangular part of the symmetric matrix and the strictly
  lower triangular part of A is not referenced. On exit, the
  upper triangular part of the array A is overwritten by the
  upper triangular part of the updated matrix.
  Before entry, with UPLO = 'L' or 'l', the leading n by n
  lower triangular part of the array A must contain the lower
  triangular part of the symmetric matrix and the strictly
  upper triangular part of A is not referenced. On exit, the
  lower triangular part of the array A is overwritten by the
  lower triangular part of the updated matrix.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, N ).
  Unchanged on exit.
```

———-

64.5.3 fortran code

— **csyr.f** —

```
*  ==================================================================
      SUBROUTINE CSYR( UPLO, N, ALPHA, X, INCX, A, LDA )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INCX, LDA, N
      COMPLEX            ALPHA
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), X( * )
*     ..
*
* ===================================================================
*
*     .. Parameters ..
      COMPLEX            ZERO
      PARAMETER          ( ZERO = ( 0.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, INFO, IX, J, JX, KX
      COMPLEX            TEMP
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
```

```
*
      INFO = 0
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = 1
      ELSE IF( N.LT.0 ) THEN
         INFO = 2
      ELSE IF( INCX.EQ.0 ) THEN
         INFO = 5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = 7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CSYR  ', INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( ( N.EQ.0 ) .OR. ( ALPHA.EQ.ZERO ) )
     $   RETURN
*
*     Set the start point in X if the increment is not unity.
*
      IF( INCX.LE.0 ) THEN
         KX = 1 - ( N-1 )*INCX
      ELSE IF( INCX.NE.1 ) THEN
         KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the triangular part
*     of A.
*
      IF( LSAME( UPLO, 'U' ) ) THEN
*
*        Form  A  when A is stored in upper triangle.
*
         IF( INCX.EQ.1 ) THEN
            DO 20 J = 1, N
               IF( X( J ).NE.ZERO ) THEN
                  TEMP = ALPHA*X( J )
                  DO 10 I = 1, J
                     A( I, J ) = A( I, J ) + X( I )*TEMP
   10             CONTINUE
               END IF
   20       CONTINUE
         ELSE
            JX = KX
            DO 40 J = 1, N
               IF( X( JX ).NE.ZERO ) THEN
```

```
                TEMP = ALPHA*X( JX )
                IX = KX
                DO 30 I = 1, J
                   A( I, J ) = A( I, J ) + X( IX )*TEMP
                   IX = IX + INCX
   30           CONTINUE
             END IF
             JX = JX + INCX
   40       CONTINUE
         END IF
      ELSE
*
*        Form  A  when A is stored in lower triangle.
*
         IF( INCX.EQ.1 ) THEN
            DO 60 J = 1, N
               IF( X( J ).NE.ZERO ) THEN
                  TEMP = ALPHA*X( J )
                  DO 50 I = J, N
                     A( I, J ) = A( I, J ) + X( I )*TEMP
   50             CONTINUE
               END IF
   60       CONTINUE
         ELSE
            JX = KX
            DO 80 J = 1, N
               IF( X( JX ).NE.ZERO ) THEN
                  TEMP = ALPHA*X( JX )
                  IX = JX
                  DO 70 I = J, N
                     A( I, J ) = A( I, J ) + X( IX )*TEMP
                     IX = IX + INCX
   70             CONTINUE
               END IF
               JX = JX + INCX
   80       CONTINUE
         END IF
      END IF
*
      RETURN
*
*     End of CSYR
*
      END
```

———-

64.5.4 lisp code

— csyr.lisp —

```
(let* ((zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) zero)
           (ignorable zero))
  (defun csyr (uplo n alpha x incx a lda)
    (declare (type (array f2cl-lib:complex8 (*)) a x)
             (type (f2cl-lib:complex8) alpha)
             (type (f2cl-lib:integer4) lda incx n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%))
      (prog ((temp #C(0.0 0.0)) (i 0) (info 0) (ix 0) (j 0) (jx 0) (kx 0))
        (declare (type (f2cl-lib:complex8) temp)
                 (type (f2cl-lib:integer4) i info ix j jx kx))
        (setf info 0)
        (cond
         ((and
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
          (setf info 1))
         ((< n 0) (setf info 2)) ((= incx 0) (setf info 5))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info 7)))
        (cond
         ((/= info 0)
          (multiple-value-bind (var-0 var-1)
              (xerbla "CSYR  " info)
            (declare (ignore var-0))
            (when var-1 (setf info var-1)))
          (go end_label)))
        (if (or (= n 0) (= alpha zero))
            (go end_label))
        (cond
         ((<= incx 0)
```

```
  (setf kx
          (f2cl-lib:int-sub 1
                            (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                              incx))))
 ((/= incx 1) (setf kx 1)))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame uplo "U")
    (declare (ignore var-1))
    (when var-0 (setf uplo var-0))
    ret-val)
  (cond
   ((= incx 1)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
         ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
          (setf temp
                  (* alpha
                     (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%)
                      (+
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                       (*
                        (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                        temp)))
             label10))))
      label20)))
   (t (setf jx kx)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
         ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
          (setf temp
                  (* alpha
                     (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
          (setf ix kx)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%)
                      (+
```

```
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%)
                        (*
                         (f2cl-lib:fref x-%data% (ix) ((1 *))
                                        x-%offset%)
                         temp)))
                (setf ix (f2cl-lib:int-add ix incx))
               label30))))
          (setf jx (f2cl-lib:int-add jx incx))
         label40)))))
    (t
     (cond
      ((= incx 1)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (cond
            ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
             (setf temp
                     (* alpha
                        (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
             (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                           ((> i n) nil)
               (tagbody
                 (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                       (+
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%)
                        (*
                         (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                         temp)))
               label50))))
         label60)))
      (t (setf jx kx)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (cond
            ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
             (setf temp
                     (* alpha
                        (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
             (setf ix jx)
             (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                           ((> i n) nil)
               (tagbody
                 (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                       (+
```

```
                              (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                             a-%offset%)
                              (*
                               (f2cl-lib:fref x-%data% (ix) ((1 *))
                                              x-%offset%)
                               temp)))
                       (setf ix (f2cl-lib:int-add ix incx))
                      label70))))
                 (setf jx (f2cl-lib:int-add jx incx))
                label80))))))
        (go end_label)
       end_label
        (return (values uplo nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::csyr fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::complex8)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil nil
                                              nil nil nil)
                                            :calls 'nil)))
```

———-

64.6 csyswapr

64.6.1 Axiom unit tests

— csyswapr.input —

```
)set break resume
)sys rm -f csyswapr.output
)spool csyswapr.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

64.6.2 Axiom help page

— csyswapr.help —

```
=====================================================================
csyswapr examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine csyswapr (UPLO, N, A, LDA, I1, I2)
           CSYSWAPR

Function/Subroutine Documentation
   subroutine csyswapr (character UPLO, integer N, complex, dimension( lda, n
       ) A, integer LDA, integer I1, integer I2)
       CSYSWAPR

Purpose:
   CSYSWAPR applies an elementary permutation on the rows and the columns of
   a symmetric matrix.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
```

```
  A is COMPLEX array, dimension (LDA,N)
  On entry, the NB diagonal matrix D and the multipliers
  used to obtain the factor U or L as computed by CSYTRF.

  On exit, if INFO = 0, the (symmetric) inverse of the original
  matrix.  If UPLO = 'U', the upper triangular part of the
  inverse is formed and the part of A below the diagonal is not
  referenced; if UPLO = 'L' the lower triangular part of the
  inverse is formed and the part of A above the diagonal is
  not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

I1
  I1 is INTEGER
  Index of the first row to swap

I2
  I2 is INTEGER
  Index of the second row to swap
```

———-

64.6.3 fortran code

— **csyswapr.f** —

```
*  =====================================================================
      SUBROUTINE CSYSWAPR( UPLO, N, A, LDA, I1, I2)
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER        UPLO
      INTEGER          I1, I2, LDA, N
*     ..
*     .. Array Arguments ..
      COMPLEX          A( LDA, N )
*
*  =====================================================================
*
*     ..
```

```
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I
      COMPLEX            TMP
*
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CSWAP
*     ..
*     .. Executable Statements ..
*
      UPPER = LSAME( UPLO, 'U' )
      IF (UPPER) THEN
*
*         UPPER
*         first swap
*          - swap column I1 and I2 from I1 to I1-1
         CALL CSWAP( I1-1, A(1,I1), 1, A(1,I2), 1 )
*
*          second swap :
*          - swap A(I1,I1) and A(I2,I2)
*          - swap row I1 from I1+1 to I2-1 with col I2 from I1+1 to I2-1
         TMP=A(I1,I1)
         A(I1,I1)=A(I2,I2)
         A(I2,I2)=TMP
*
         DO I=1,I2-I1-1
            TMP=A(I1,I1+I)
            A(I1,I1+I)=A(I1+I,I2)
            A(I1+I,I2)=TMP
         END DO
*
*          third swap
*          - swap row I1 and I2 from I2+1 to N
         DO I=I2+1,N
            TMP=A(I1,I)
            A(I1,I)=A(I2,I)
            A(I2,I)=TMP
         END DO
*
        ELSE
*
*         LOWER
*         first swap
*          - swap row I1 and I2 from I1 to I1-1
         CALL CSWAP ( I1-1, A(I1,1), LDA, A(I2,1), LDA )
*
```

```
*         second swap :
*          - swap A(I1,I1) and A(I2,I2)
*          - swap col I1 from I1+1 to I2-1 with row I2 from I1+1 to I2-1
          TMP=A(I1,I1)
          A(I1,I1)=A(I2,I2)
          A(I2,I2)=TMP
*
          DO I=1,I2-I1-1
             TMP=A(I1+I,I1)
             A(I1+I,I1)=A(I2,I1+I)
             A(I2,I1+I)=TMP
          END DO
*
*         third swap
*          - swap col I1 and I2 from I2+1 to N
          DO I=I2+1,N
             TMP=A(I,I1)
             A(I,I1)=A(I,I2)
             A(I,I2)=TMP
          END DO
*
      ENDIF
      END SUBROUTINE CSYSWAPR
```

———-

64.6.4 lisp code

— **csyswapr.lisp** —

———-

64.7 ctgsy2

64.7.1 Axiom unit tests

— **ctgsy2.input** —

```
)set break resume
)sys rm -f ctgsy2.output
)spool ctgsy2.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

64.7.2 Axiom help page

— ctgsy2.help —

```
=====================================================================
ctgsy2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ctgsy2 (TRANS, IJOB, M, N, A, LDA, B, LDB, C, LDC, D, LDD,
           E, LDE, F, LDF, SCALE, RDSUM, RDSCAL, INFO)
           CTGSY2 solves the generalized Sylvester equation (unblocked
           algorithm).

Function/Subroutine Documentation
   subroutine ctgsy2 (character TRANS, integer IJOB, integer M, integer N,
       complex, dimension( lda, * ) A, integer LDA, complex, dimension( ldb, *
       ) B, integer LDB, complex, dimension( ldc, * ) C, integer LDC, complex,
       dimension( ldd, * ) D, integer LDD, complex, dimension( lde, * ) E,
       integer LDE, complex, dimension( ldf, * ) F, integer LDF, real SCALE,
       real RDSUM, real RDSCAL, integer INFO)
       CTGSY2 solves the generalized Sylvester equation (unblocked algorithm).

Purpose:
   CTGSY2 solves the generalized Sylvester equation

               A * R - L * B = scale *  C               (1)
               D * R - L * E = scale * F

   using Level 1 and 2 BLAS, where R and L are unknown M-by-N matrices,
   (A, D), (B, E) and (C, F) are given matrix pairs of size M-by-M,
   N-by-N and M-by-N, respectively. A, B, D and E are upper triangular
   (i.e., (A,D) and (B,E) in generalized Schur form).
```

```
  The solution (R, L) overwrites (C, F). 0 <= SCALE <= 1 is an output
  scaling factor chosen to avoid overflow.

  In matrix notation solving equation (1) corresponds to solve
  Zx = scale * b, where Z is defined as

             Z = [ kron(In, A)  -kron(B**H, Im) ]             (2)
                 [ kron(In, D)  -kron(E**H, Im) ],

  Ik is the identity matrix of size k and X**H is the transpose of X.
  kron(X, Y) is the Kronecker product between the matrices X and Y.

  If TRANS = 'C', y in the conjugate transposed system Z**H*y = scale*b
  is solved for, which is equivalent to solve for R and L in

                A**H * R  + D**H * L   = scale * C           (3)
                R  * B**H + L  * E**H  = scale * -F

  This case is used to compute an estimate of Dif[(A, D), (B, E)] =
  = sigma_min(Z) using reverse communicaton with CLACON.

  CTGSY2 also (IJOB >= 1) contributes to the computation in CTGSYL
  of an upper bound on the separation between to matrix pairs. Then
  the input (A, D), (B, E) are sub-pencils of two matrix pairs in CTGSYL.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   = 'N', solve the generalized Sylvester equation (1).
   = 'T': solve the 'transposed' system (3).

 IJOB
   IJOB is INTEGER
   Specifies what kind of functionality to be performed.
   =0: solve (1) only.
   =1: A contribution from this subsystem to a Frobenius
       norm-based estimate of the separation between two matrix
       pairs is computed. (look ahead strategy is used).
   =2: A contribution from this subsystem to a Frobenius
       norm-based estimate of the separation between two matrix
       pairs is computed. (SGECON on sub-systems is used.)
   Not referenced if TRANS = 'T'.

 M
   M is INTEGER
   On entry, M specifies the order of A and D, and the row
   dimension of C, F, R and L.

 N
```

```
  N is INTEGER
  On entry, N specifies the order of B and E, and the column
  dimension of C, F, R and L.

A
  A is COMPLEX array, dimension (LDA, M)
  On entry, A contains an upper triangular matrix.

LDA
  LDA is INTEGER
  The leading dimension of the matrix A. LDA >= max(1, M).

B
  B is COMPLEX array, dimension (LDB, N)
  On entry, B contains an upper triangular matrix.

LDB
  LDB is INTEGER
  The leading dimension of the matrix B. LDB >= max(1, N).

C
  C is COMPLEX array, dimension (LDC, N)
  On entry, C contains the right-hand-side of the first matrix
  equation in (1).
  On exit, if IJOB = 0, C has been overwritten by the solution
  R.

LDC
  LDC is INTEGER
  The leading dimension of the matrix C. LDC >= max(1, M).

D
  D is COMPLEX array, dimension (LDD, M)
  On entry, D contains an upper triangular matrix.

LDD
  LDD is INTEGER
  The leading dimension of the matrix D. LDD >= max(1, M).

E
  E is COMPLEX array, dimension (LDE, N)
  On entry, E contains an upper triangular matrix.

LDE
  LDE is INTEGER
  The leading dimension of the matrix E. LDE >= max(1, N).

F
  F is COMPLEX array, dimension (LDF, N)
  On entry, F contains the right-hand-side of the second matrix
```

```
  equation in (1).
  On exit, if IJOB = 0, F has been overwritten by the solution L.

LDF
  LDF is INTEGER
  The leading dimension of the matrix F. LDF >= max(1, M).

SCALE
  SCALE is REAL
  On exit, 0 <= SCALE <= 1. If 0 < SCALE < 1, the solutions
  R and L (C and F on entry) will hold the solutions to a
  slightly perturbed system but the input matrices A, B, D and
  E have not been changed. If SCALE = 0, R and L will hold the
  solutions to the homogeneous system with C = F = 0.
  Normally, SCALE = 1.

RDSUM
  RDSUM is REAL
  On entry, the sum of squares of computed contributions to
  the Dif-estimate under computation by CTGSYL, where the
  scaling factor RDSCAL (see below) has been factored out.
  On exit, the corresponding sum of squares updated with the
  contributions from the current sub-system.
  If TRANS = 'T' RDSUM is not touched.
  NOTE: RDSUM only makes sense when CTGSY2 is called by CTGSYL.

RDSCAL
  RDSCAL is REAL
  On entry, scaling factor used to prevent overflow in RDSUM.
  On exit, RDSCAL is updated w.r.t. the current contributions
  in RDSUM.
  If TRANS = 'T', RDSCAL is not touched.
  NOTE: RDSCAL only makes sense when CTGSY2 is called by CTGSYL.

INFO
  INFO is INTEGER
  On exit, if INFO is set to
    =0: Successful exit
    <0: If INFO = -i, input argument number i is illegal.
    >0: The matrix pairs (A, D) and (B, E) have common or very
         close eigenvalues.
```

———-

64.7.3 fortran code

— ctgsy2.f —

```
*  =====================================================================
      SUBROUTINE CTGSY2( TRANS, IJOB, M, N, A, LDA, B, LDB, C, LDC, D,
     $                   LDD, E, LDE, F, LDF, SCALE, RDSUM, RDSCAL,
     $                   INFO )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            IJOB, INFO, LDA, LDB, LDC, LDD, LDE, LDF, M, N
      REAL               RDSCAL, RDSUM, SCALE
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), B( LDB, * ), C( LDC, * ),
     $                   D( LDD, * ), E( LDE, * ), F( LDF, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      INTEGER            LDZ
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0, LDZ = 2 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      INTEGER            I, IERR, J, K
      REAL               SCALOC
      COMPLEX            ALPHA
*     ..
*     .. Local Arrays ..
      INTEGER            IPIV( LDZ ), JPIV( LDZ )
      COMPLEX            RHS( LDZ ), Z( LDZ, LDZ )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CAXPY, CGESC2, CGETC2, CSCAL, CLATDF, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          CMPLX, CONJG, MAX
*     ..
*     .. Executable Statements ..
*
*     Decode and test input parameters
```

```
*
      INFO = 0
      IERR = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( NOTRAN ) THEN
         IF( ( IJOB.LT.0 ) .OR. ( IJOB.GT.2 ) ) THEN
            INFO = -2
         END IF
      END IF
      IF( INFO.EQ.0 ) THEN
         IF( M.LE.0 ) THEN
            INFO = -3
         ELSE IF( N.LE.0 ) THEN
            INFO = -4
         ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
            INFO = -6
         ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
            INFO = -8
         ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
            INFO = -10
         ELSE IF( LDD.LT.MAX( 1, M ) ) THEN
            INFO = -12
         ELSE IF( LDE.LT.MAX( 1, N ) ) THEN
            INFO = -14
         ELSE IF( LDF.LT.MAX( 1, M ) ) THEN
            INFO = -16
         END IF
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CTGSY2', -INFO )
         RETURN
      END IF
*
      IF( NOTRAN ) THEN
*
*        Solve (I, J) - system
*           A(I, I) * R(I, J) - L(I, J) * B(J, J) = C(I, J)
*           D(I, I) * R(I, J) - L(I, J) * E(J, J) = F(I, J)
*        for I = M, M - 1, ..., 1; J = 1, 2, ..., N
*
         SCALE = ONE
         SCALOC = ONE
         DO 30 J = 1, N
            DO 20 I = M, 1, -1
*
*              Build 2 by 2 system
*
               Z( 1, 1 ) = A( I, I )
```

```
               Z( 2, 1 ) = D( I, I )
               Z( 1, 2 ) = -B( J, J )
               Z( 2, 2 ) = -E( J, J )
*
*              Set up right hand side(s)
*
               RHS( 1 ) = C( I, J )
               RHS( 2 ) = F( I, J )
*
*              Solve Z * x = RHS
*
               CALL CGETC2( LDZ, Z, LDZ, IPIV, JPIV, IERR )
               IF( IERR.GT.0 )
     $            INFO = IERR
               IF( IJOB.EQ.0 ) THEN
                  CALL CGESC2( LDZ, Z, LDZ, RHS, IPIV, JPIV, SCALOC )
                  IF( SCALOC.NE.ONE ) THEN
                     DO 10 K = 1, N
                        CALL CSCAL( M, CMPLX( SCALOC, ZERO ), C( 1, K ),
     $                              1 )
                        CALL CSCAL( M, CMPLX( SCALOC, ZERO ), F( 1, K ),
     $                              1 )
   10                CONTINUE
                     SCALE = SCALE*SCALOC
                  END IF
               ELSE
                  CALL CLATDF( IJOB, LDZ, Z, LDZ, RHS, RDSUM, RDSCAL,
     $                         IPIV, JPIV )
               END IF
*
*              Unpack solution vector(s)
*
               C( I, J ) = RHS( 1 )
               F( I, J ) = RHS( 2 )
*
*              Substitute R(I, J) and L(I, J) into remaining equation.
*
               IF( I.GT.1 ) THEN
                  ALPHA = -RHS( 1 )
                  CALL CAXPY( I-1, ALPHA, A( 1, I ), 1, C( 1, J ), 1 )
                  CALL CAXPY( I-1, ALPHA, D( 1, I ), 1, F( 1, J ), 1 )
               END IF
               IF( J.LT.N ) THEN
                  CALL CAXPY( N-J, RHS( 2 ), B( J, J+1 ), LDB,
     $                        C( I, J+1 ), LDC )
                  CALL CAXPY( N-J, RHS( 2 ), E( J, J+1 ), LDE,
     $                        F( I, J+1 ), LDF )
               END IF
*
   20       CONTINUE
```

```
   30    CONTINUE
      ELSE
*
*        Solve transposed (I, J) - system:
*           A(I, I)**H * R(I, J) + D(I, I)**H * L(J, J) = C(I, J)
*           R(I, I) * B(J, J) + L(I, J) * E(J, J)   = -F(I, J)
*        for I = 1, 2, ..., M, J = N, N - 1, ..., 1
*
         SCALE = ONE
         SCALOC = ONE
         DO 80 I = 1, M
            DO 70 J = N, 1, -1
*
*              Build 2 by 2 system Z**H
*
               Z( 1, 1 ) = CONJG( A( I, I ) )
               Z( 2, 1 ) = -CONJG( B( J, J ) )
               Z( 1, 2 ) = CONJG( D( I, I ) )
               Z( 2, 2 ) = -CONJG( E( J, J ) )
*
*
*              Set up right hand side(s)
*
               RHS( 1 ) = C( I, J )
               RHS( 2 ) = F( I, J )
*
*              Solve Z**H * x = RHS
*
               CALL CGETC2( LDZ, Z, LDZ, IPIV, JPIV, IERR )
               IF( IERR.GT.0 )
     $            INFO = IERR
               CALL CGESC2( LDZ, Z, LDZ, RHS, IPIV, JPIV, SCALOC )
               IF( SCALOC.NE.ONE ) THEN
                  DO 40 K = 1, N
                     CALL CSCAL( M, CMPLX( SCALOC, ZERO ), C( 1, K ),
     $                           1 )
                     CALL CSCAL( M, CMPLX( SCALOC, ZERO ), F( 1, K ),
     $                           1 )
   40             CONTINUE
                  SCALE = SCALE*SCALOC
               END IF
*
*              Unpack solution vector(s)
*
               C( I, J ) = RHS( 1 )
               F( I, J ) = RHS( 2 )
*
*              Substitute R(I, J) and L(I, J) into remaining equation.
*
               DO 50 K = 1, J - 1
```

```
                F( I, K ) = F( I, K ) + RHS( 1 )*CONJG( B( K, J ) ) +
     $                      RHS( 2 )*CONJG( E( K, J ) )
   50          CONTINUE
               DO 60 K = I + 1, M
                C( K, J ) = C( K, J ) - CONJG( A( I, K ) )*RHS( 1 ) -
     $                      CONJG( D( I, K ) )*RHS( 2 )
   60          CONTINUE
*
   70       CONTINUE
   80    CONTINUE
      END IF
      RETURN
*
*     End of CTGSY2
*
      END
```

———-

64.7.4 lisp code

— **ctgsy2.lisp** —

```
(let* ((zero 0.0) (one 1.0) (ldz 2))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (f2cl-lib:integer4 2 2) ldz)
           (ignorable zero one ldz))
  (defun ctgsy2
         (trans ijob m n a lda b ldb$ c ldc d ldd e lde f ldf scale rdsum
          rdscal info)
    (declare (type (single-float) rdscal rdsum scale)
             (type (array f2cl-lib:complex8 (*)) f e d c b a)
             (type (f2cl-lib:integer4) info ldf lde ldd ldc ldb$ lda n m ijob)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (c f2cl-lib:complex8 c-%data% c-%offset%)
         (d f2cl-lib:complex8 d-%data% d-%offset%)
         (e f2cl-lib:complex8 e-%data% e-%offset%)
         (f f2cl-lib:complex8 f-%data% f-%offset%))
      (prog ((rhs (make-array ldz :element-type 'f2cl-lib:complex8))
             (z
              (make-array (the fixnum (reduce #'* (list ldz ldz)))
                          :element-type 'f2cl-lib:complex8))
             (ipiv (make-array ldz :element-type 'f2cl-lib:integer4))
```

```
     (jpiv (make-array ldz :element-type 'f2cl-lib:integer4))
     (alpha #C(0.0 0.0)) (scaloc 0.0) (i 0) (ierr 0) (j 0) (k 0)
     (notran nil))
(declare (type (array f2cl-lib:complex8 (*)) rhs z)
         (type (array f2cl-lib:integer4 (*)) ipiv jpiv)
         (type (f2cl-lib:complex8) alpha)
         (type (single-float) scaloc)
         (type (f2cl-lib:integer4) i ierr j k)
         (type f2cl-lib:logical notran))
(setf info 0)
(setf ierr 0)
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -1))
 (notran (cond ((or (< ijob 0) (> ijob 2)) (setf info -2)))))
(cond
 ((= info 0)
  (cond ((<= m 0) (setf info -3)) ((<= n 0) (setf info -4))
        ((< lda
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -6))
        ((< ldb$
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -8))
        ((< ldc
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -10))
        ((< ldd
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -12))
        ((< lde
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -14))
        ((< ldf
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -16)))))
(cond
 ((/= info 0) (xerbla "CTGSY2" (f2cl-lib:int-sub info))
```

```
  (go end_label)))
 (cond
  (notran (setf scale one) (setf scaloc one)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     ((> i 1) nil)
         (tagbody
           (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
                   (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                  a-%offset%))
           (setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz)))
                   (f2cl-lib:fref d-%data% (i i) ((1 ldd) (1 *))
                                  d-%offset%))
           (setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz)))
                   (-
                    (f2cl-lib:fref b-%data% (j j) ((1 ldb$) (1 *))
                                   b-%offset%)))
           (setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
                   (-
                    (f2cl-lib:fref e-%data% (j j) ((1 lde) (1 *))
                                   e-%offset%)))
           (setf (f2cl-lib:fref rhs (1) ((1 ldz)))
                   (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                  c-%offset%))
           (setf (f2cl-lib:fref rhs (2) ((1 ldz)))
                   (f2cl-lib:fref f-%data% (i j) ((1 ldf) (1 *))
                                  f-%offset%))
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
               (cgetc2 ldz z ldz ipiv jpiv ierr)
             (declare (ignore var-1 var-3 var-4))
             (setf ldz var-0)
             (setf ldz var-2)
             (setf ierr var-5))
           (if (> ierr 0)
               (setf info ierr))
           (cond
            ((= ijob 0)
             (multiple-value-bind
                 (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                 (cgesc2 ldz z ldz rhs ipiv jpiv scaloc)
               (declare (ignore var-1 var-3 var-4 var-5))
               (setf ldz var-0)
               (setf ldz var-2)
               (setf scaloc var-6))
             (cond
              ((/= scaloc one)
               (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                             ((> k n) nil)
```

```
        (tagbody
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (cscal m (f2cl-lib:cmplx scaloc zero)
               (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                     (1 k) ((1 ldc) (1 *))
                                     c-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3))
            (when var-0 (setf m var-0)))
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (cscal m (f2cl-lib:cmplx scaloc zero)
               (f2cl-lib:array-slice f-%data% f2cl-lib:complex8
                                     (1 k) ((1 ldf) (1 *))
                                     f-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3))
            (when var-0 (setf m var-0)))
         label10))
      (setf scale (* scale scaloc)))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clatdf ijob ldz z ldz rhs rdsum rdscal ipiv jpiv)
      (declare (ignore var-0 var-2 var-4 var-7 var-8))
      (setf ldz var-1)
      (setf ldz var-3)
      (setf rdsum var-5)
      (setf rdscal var-6))))
  (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                       c-%offset%)
          (f2cl-lib:fref rhs (1) ((1 ldz))))
  (setf (f2cl-lib:fref f-%data% (i j) ((1 ldf) (1 *))
                       f-%offset%)
          (f2cl-lib:fref rhs (2) ((1 ldz))))
  (cond
   ((> i 1) (setf alpha (- (f2cl-lib:fref rhs (1) ((1 ldz)))))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (caxpy (f2cl-lib:int-sub i 1) alpha
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 i)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice c-%data% f2cl-lib:complex8 (1 j)
                               ((1 ldc) (1 *)) c-%offset%)
         1)
      (declare (ignore var-0 var-2 var-3 var-4 var-5))
      (when var-1 (setf alpha var-1)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (caxpy (f2cl-lib:int-sub i 1) alpha
         (f2cl-lib:array-slice d-%data% f2cl-lib:complex8 (1 i)
                               ((1 ldd) (1 *)) d-%offset%)
```

```
                    1
                    (f2cl-lib:array-slice f-%data% f2cl-lib:complex8 (1 j)
                                          ((1 ldf) (1 *)) f-%offset%)
                    1)
                (declare (ignore var-0 var-2 var-3 var-4 var-5))
                (when var-1 (setf alpha var-1)))))
            (cond
             ((< j n)
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                  (caxpy (f2cl-lib:int-sub n j)
                    (f2cl-lib:array-slice rhs f2cl-lib:complex8 (2)
                                          ((1 ldz)))
                    (f2cl-lib:array-slice b-%data% f2cl-lib:complex8
                                          (j (f2cl-lib:int-add j 1))
                                          ((1 ldb$) (1 *)) b-%offset%)
                    ldb$
                    (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                          (i (f2cl-lib:int-add j 1))
                                          ((1 ldc) (1 *)) c-%offset%)
                    ldc)
                (declare (ignore var-0 var-1 var-2 var-4))
                (when var-3 (setf ldb$ var-3))
                (when var-5 (setf ldc var-5)))
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                  (caxpy (f2cl-lib:int-sub n j)
                    (f2cl-lib:array-slice rhs f2cl-lib:complex8 (2)
                                          ((1 ldz)))
                    (f2cl-lib:array-slice e-%data% f2cl-lib:complex8
                                          (j (f2cl-lib:int-add j 1))
                                          ((1 lde) (1 *)) e-%offset%)
                    lde
                    (f2cl-lib:array-slice f-%data% f2cl-lib:complex8
                                          (i (f2cl-lib:int-add j 1))
                                          ((1 ldf) (1 *)) f-%offset%)
                    ldf)
                (declare (ignore var-0 var-1 var-2 var-4))
                (when var-3 (setf lde var-3))
                (when var-5 (setf ldf var-5)))))
          label20))
     label30)))
  (t (setf scale one) (setf scaloc one)
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i m) nil)
     (tagbody
       (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     ((> j 1) nil)
         (tagbody
           (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
                   (f2cl-lib:conjg
                    (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
```

```
                        a-%offset%)))
(setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:conjg
          (f2cl-lib:fref b-%data% (j j) ((1 ldb$) (1 *))
                         b-%offset%))))
(setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz)))
        (f2cl-lib:conjg
         (f2cl-lib:fref d-%data% (i i) ((1 ldd) (1 *))
                        d-%offset%)))
(setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
        (-
         (f2cl-lib:conjg
          (f2cl-lib:fref e-%data% (j j) ((1 lde) (1 *))
                         e-%offset%))))
(setf (f2cl-lib:fref rhs (1) ((1 ldz)))
        (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                       c-%offset%))
(setf (f2cl-lib:fref rhs (2) ((1 ldz)))
        (f2cl-lib:fref f-%data% (i j) ((1 ldf) (1 *))
                       f-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (cgetc2 ldz z ldz ipiv jpiv ierr)
  (declare (ignore var-1 var-3 var-4))
  (setf ldz var-0)
  (setf ldz var-2)
  (setf ierr var-5))
(if (> ierr 0)
    (setf info ierr))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (cgesc2 ldz z ldz rhs ipiv jpiv scaloc)
  (declare (ignore var-1 var-3 var-4 var-5))
  (setf ldz var-0)
  (setf ldz var-2)
  (setf scaloc var-6))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (cscal m (f2cl-lib:cmplx scaloc zero)
           (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                 (1 k) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf m var-0)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
```

```
          (cscal m (f2cl-lib:cmplx scaloc zero)
           (f2cl-lib:array-slice f-%data% f2cl-lib:complex8
                                 (1 k) ((1 ldf) (1 *))
                                 f-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf m var-0)))
     label40))
  (setf scale (* scale scaloc))))
(setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (1) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (i j) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (2) ((1 ldz))))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k
                  (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref f-%data% (i k) ((1 ldf) (1 *))
                         f-%offset%)
            (+
             (f2cl-lib:fref f-%data% (i k) ((1 ldf) (1 *))
                            f-%offset%)
             (* (f2cl-lib:fref rhs (1) ((1 ldz)))
                (f2cl-lib:conjg
                 (f2cl-lib:fref b-%data% (k j)
                                ((1 ldb$) (1 *))
                                b-%offset%)))
             (* (f2cl-lib:fref rhs (2) ((1 ldz)))
                (f2cl-lib:conjg
                 (f2cl-lib:fref e-%data% (k j)
                                ((1 lde) (1 *))
                                e-%offset%)))))
   label50))
(f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
               (f2cl-lib:int-add k 1))
              ((> k m) nil)
  (tagbody
    (setf (f2cl-lib:fref c-%data% (k j) ((1 ldc) (1 *))
                         c-%offset%)
            (-
             (f2cl-lib:fref c-%data% (k j) ((1 ldc) (1 *))
                            c-%offset%)
             (*
              (f2cl-lib:conjg
               (f2cl-lib:fref a-%data% (i k) ((1 lda) (1 *))
                              a-%offset%))
              (f2cl-lib:fref rhs (1) ((1 ldz))))
```

```
                              (*
                               (f2cl-lib:conjg
                                (f2cl-lib:fref d-%data% (i k) ((1 ldd) (1 *))
                                               d-%offset%))
                               (f2cl-lib:fref rhs (2) ((1 ldz))))))
                    label60))
                label70))
            label80))))
        (go end_label)
       end_label
        (return
         (values trans nil m nil nil nil nil ldb$ nil ldc nil nil nil lde nil
                 ldf scale rdsum rdscal info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ctgsy2
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4)
                                             (single-float) (single-float)
                                             (single-float)
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(fortran-to-lisp::trans nil
                                             fortran-to-lisp::m nil nil nil
                                             nil fortran-to-lisp::ldb$ nil
                                             fortran-to-lisp::ldc nil nil nil
```

```
  fortran-to-lisp::lde nil
  fortran-to-lisp::ldf
  fortran-to-lisp::scale
  fortran-to-lisp::rdsum
  fortran-to-lisp::rdscal
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::clatdf
  fortran-to-lisp::cgesc2
  fortran-to-lisp::cgetc2))))
```

———-

Chapter 65

LAPACK - Symmetric Matrix, Auxilliary Routines, Complex16

65.1 zlaesy

65.1.1 Axiom unit tests

— **zlaesy.input** —

```
)set break resume
)sys rm -f zlaesy.output
)spool zlaesy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

65.1.2 Axiom help page

— **zlaesy.help** —

```
==========================================================================
zlaesy examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zlaesy (A, B, C, RT1, RT2, EVSCAL, CS1, SN1)
           ZLAESY computes the eigenvalues and eigenvectors of a 2-by-2
           complex symmetric matrix.

Function/Subroutine Documentation
   subroutine zlaesy (complex*16 A, complex*16 B, complex*16 C, complex*16
       RT1, complex*16 RT2, complex*16 EVSCAL, complex*16 CS1, complex*16 SN1)
       ZLAESY computes the eigenvalues and eigenvectors of a 2-by-2 complex
       symmetric matrix.

Purpose:
   ZLAESY computes the eigendecomposition of a 2-by-2 symmetric matrix
      ( ( A, B );( B, C ) )
   provided the norm of the matrix of eigenvectors is larger than
   some threshold value.

   RT1 is the eigenvalue of larger absolute value, and RT2 of
   smaller absolute value.  If the eigenvectors are computed, then
   on return ( CS1, SN1 ) is the unit eigenvector for RT1, hence

   [  CS1     SN1   ] . [ A  B ] . [ CS1    -SN1   ] = [ RT1  0  ]
   [ -SN1     CS1   ]   [ B  C ]   [ SN1     CS1   ]   [  0  RT2 ]

Parameters:

 A
   A is COMPLEX*16
   The ( 1, 1 ) element of input matrix.

 B
   B is COMPLEX*16
   The ( 1, 2 ) element of input matrix.  The ( 2, 1 ) element
   is also given by B, since the 2-by-2 matrix is symmetric.

 C
   C is COMPLEX*16
   The ( 2, 2 ) element of input matrix.

 RT1
   RT1 is COMPLEX*16
   The eigenvalue of larger modulus.

 RT2
```

```
  RT2 is COMPLEX*16
  The eigenvalue of smaller modulus.

EVSCAL
  EVSCAL is COMPLEX*16
  The complex value by which the eigenvector matrix was scaled
  to make it orthonormal.  If EVSCAL is zero, the eigenvectors
  were not computed.  This means one of two things:  the 2-by-2
  matrix could not be diagonalized, or the norm of the matrix
  of eigenvectors before scaling was larger than the threshold
  value THRESH (set below).

CS1
  CS1 is COMPLEX*16

SN1
  SN1 is COMPLEX*16
  If EVSCAL .NE. 0,  ( CS1, SN1 ) is the unit right eigenvector for RT1.
```

65.1.3 fortran code

— zlaesy.f —

```
*  ===================================================================
      SUBROUTINE ZLAESY( A, B, C, RT1, RT2, EVSCAL, CS1, SN1 )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      COMPLEX*16         A, B, C, CS1, EVSCAL, RT1, RT2, SN1
*     ..
*
* ====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D0 )
      COMPLEX*16         CONE
      PARAMETER          ( CONE = ( 1.0D0, 0.0D0 ) )
      DOUBLE PRECISION   HALF
```

```
      PARAMETER          ( HALF = 0.5D0 )
      DOUBLE PRECISION   THRESH
      PARAMETER          ( THRESH = 0.1D0 )
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION   BABS, EVNORM, TABS, Z
      COMPLEX*16         S, T, TMP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*
*     Special case:  The matrix is actually diagonal.
*     To avoid divide by zero later, we treat this case separately.
*
      IF( ABS( B ).EQ.ZERO ) THEN
         RT1 = A
         RT2 = C
         IF( ABS( RT1 ).LT.ABS( RT2 ) ) THEN
            TMP = RT1
            RT1 = RT2
            RT2 = TMP
            CS1 = ZERO
            SN1 = ONE
         ELSE
            CS1 = ONE
            SN1 = ZERO
         END IF
      ELSE
*
*        Compute the eigenvalues and eigenvectors.
*        The characteristic equation is
*           lambda **2 - (A+C) lambda + (A*C - B*B)
*        and we solve it using the quadratic formula.
*
         S = ( A+C )*HALF
         T = ( A-C )*HALF
*
*        Take the square root carefully to avoid over/under flow.
*
         BABS = ABS( B )
         TABS = ABS( T )
         Z = MAX( BABS, TABS )
         IF( Z.GT.ZERO )
     $      T = Z*SQRT( ( T / Z )**2+( B / Z )**2 )
*
*        Compute the two eigenvalues.  RT1 and RT2 are exchanged
*        if necessary so that RT1 will have the greater magnitude.
```

```
*
         RT1 = S + T
         RT2 = S - T
         IF( ABS( RT1 ).LT.ABS( RT2 ) ) THEN
            TMP = RT1
            RT1 = RT2
            RT2 = TMP
         END IF
*
*        Choose CS1 = 1 and SN1 to satisfy the first equation, then
*        scale the components of this eigenvector so that the matrix
*        of eigenvectors X satisfies  X * X**T = I .  (No scaling is
*        done if the norm of the eigenvalue matrix is less than THRESH.)
*
         SN1 = ( RT1-A ) / B
         TABS = ABS( SN1 )
         IF( TABS.GT.ONE ) THEN
            T = TABS*SQRT( ( ONE / TABS )**2+( SN1 / TABS )**2 )
         ELSE
            T = SQRT( CONE+SN1*SN1 )
         END IF
         EVNORM = ABS( T )
         IF( EVNORM.GE.THRESH ) THEN
            EVSCAL = CONE / T
            CS1 = EVSCAL
            SN1 = SN1*EVSCAL
         ELSE
            EVSCAL = ZERO
         END IF
      END IF
      RETURN
*
*     End of ZLAESY
*
      END
```

———-

65.1.4 lisp code

— **zlaesy.lisp** —

```
(let* ((zero 0.0d0)
       (one 1.0d0)
       (cone (f2cl-lib:cmplx 1.0d0 0.0d0))
       (half 0.5d0)
       (thresh 0.1d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
```

```
          (type (double-float 1.0d0 1.0d0) one)
          (type (f2cl-lib:complex16) cone)
          (type (double-float 0.5d0 0.5d0) half)
          (type (double-float 0.1d0 0.1d0) thresh)
          (ignorable zero one cone half thresh))
  (defun zlaesy (a b c rt1 rt2 evscal cs1 sn1)
    (declare (type (f2cl-lib:complex16) sn1 cs1 evscal rt2 rt1 c b a))
    (prog ((s #C(0.0d0 0.0d0)) (t$ #C(0.0d0 0.0d0)) (tmp #C(0.0d0 0.0d0))
           (babs 0.0d0) (evnorm 0.0d0) (tabs 0.0d0) (z 0.0d0))
      (declare (type (f2cl-lib:complex16) s t$ tmp)
               (type (double-float) babs evnorm tabs z))
      (cond
       ((= (abs b) zero) (setf rt1 a) (setf rt2 c)
        (cond
         ((< (abs rt1) (abs rt2)) (setf tmp rt1) (setf rt1 rt2) (setf rt2 tmp)
          (setf cs1 (coerce zero 'f2cl-lib:complex16))
          (setf sn1 (coerce one 'f2cl-lib:complex16)))
         (t (setf cs1 (coerce one 'f2cl-lib:complex16))
          (setf sn1 (coerce zero 'f2cl-lib:complex16)))))
       (t (setf s (* (+ a c) half)) (setf t$ (* (- a c) half))
        (setf babs (abs b)) (setf tabs (abs t$)) (setf z (max babs tabs))
        (if (> z zero)
            (setf t$
                    (* z
                       (f2cl-lib:fsqrt
                        (+ (expt (/ t$ z) 2) (expt (/ b z) 2))))))
        (setf rt1 (+ s t$)) (setf rt2 (- s t$))
        (cond
         ((< (abs rt1) (abs rt2)) (setf tmp rt1) (setf rt1 rt2)
          (setf rt2 tmp)))
        (setf sn1 (/ (- rt1 a) b)) (setf tabs (abs sn1))
        (cond
         ((> tabs one)
          (setf t$
                  (* tabs
                     (f2cl-lib:fsqrt
                      (+ (expt (/ one tabs) 2) (expt (/ sn1 tabs) 2))))))
         (t (setf t$ (f2cl-lib:fsqrt (+ cone (* sn1 sn1))))))
        (setf evnorm (abs t$))
        (cond
         ((>= evnorm thresh) (setf evscal (/ cone t$)) (setf cs1 evscal)
          (setf sn1 (* sn1 evscal)))
         (t (setf evscal (coerce zero 'f2cl-lib:complex16))))))
      (go end_label)
     end_label
      (return (values nil nil nil rt1 rt2 evscal cs1 sn1)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
```

```
(setf (gethash 'fortran-to-lisp::zlaesy
               fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((fortran-to-lisp::complex16)
                                            (fortran-to-lisp::complex16)
                                            (fortran-to-lisp::complex16)
                                            (fortran-to-lisp::complex16)
                                            (fortran-to-lisp::complex16)
                                            (fortran-to-lisp::complex16)
                                            (fortran-to-lisp::complex16)
                                            (fortran-to-lisp::complex16))
                                          :return-values
                                          '(nil nil nil fortran-to-lisp::rt1
                                            fortran-to-lisp::rt2
                                            fortran-to-lisp::evscal
                                            fortran-to-lisp::cs1
                                            fortran-to-lisp::sn1)
                                          :calls 'nil)))
```

———-

65.2 zlansy

65.2.1 Axiom unit tests

— **zlansy.input** —

```
)set break resume
)sys rm -f zlansy.output
)spool zlansy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

65.2.2 Axiom help page

— **zlansy.help** —

```
====================================================================
zlansy examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function zlansy (NORM, UPLO, N, A, LDA, WORK)
           ZLANSY returns the value of the 1-norm, or the Frobenius norm, or
           the infinity norm, or the element of largest absolute value of a
           complex symmetric matrix.

Function/Subroutine Documentation
   double precision function zlansy (character NORM, character UPLO, integer
       N, complex*16, dimension( lda, * ) A, integer LDA, double precision,
       dimension( * ) WORK)
       ZLANSY returns the value of the 1-norm, or the Frobenius norm, or the
       infinity norm, or the element of largest absolute value of a complex
       symmetric matrix.

Purpose:
   ZLANSY  returns the value of the one norm,  or the Frobenius norm, or
   the  infinity norm,  or the  element of  largest absolute value  of a
   complex symmetric matrix A.

 Returns:

   ZLANSY

       ZLANSY = ( max(abs(A(i,j))), NORM = 'M' or 'm'
                (
                ( norm1(A),         NORM = '1', 'O' or 'o'
                (
                ( normI(A),         NORM = 'I' or 'i'
                (
                ( normF(A),         NORM = 'F', 'f', 'E' or 'e'

   where  norm1  denotes the  one norm of a matrix (maximum column sum),
   normI  denotes the  infinity norm  of a matrix  (maximum row sum) and
   normF  denotes the  Frobenius norm of a matrix (square root of sum of
   squares).  Note that  max(abs(A(i,j)))  is not a consistent matrix norm.

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies the value to be returned in ZLANSY as described above.
```

```
UPLO
  UPLO is CHARACTER*1
  Specifies whether the upper or lower triangular part of the
  symmetric matrix A is to be referenced.
  = 'U':  Upper triangular part of A is referenced
  = 'L':  Lower triangular part of A is referenced

N
  N is INTEGER
  The order of the matrix A.  N >= 0.  When N = 0, ZLANSY is set to zero.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  The symmetric matrix A.  If UPLO = 'U', the leading n by n
  upper triangular part of A contains the upper triangular part
  of the matrix A, and the strictly lower triangular part of A
  is not referenced.  If UPLO = 'L', the leading n by n lower
  triangular part of A contains the lower triangular part of
  the matrix A, and the strictly upper triangular part of A is
  not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(N,1).

WORK
  WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK)),
  where LWORK >= N when NORM = 'I' or '1' or 'O'; otherwise,
  WORK is not referenced.
```

65.2.3 fortran code

— zlansy.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION ZLANSY( NORM, UPLO, N, A, LDA, WORK )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          NORM, UPLO
```

```
      INTEGER            LDA, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   WORK( * )
      COMPLEX*16         A( LDA, * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   ABSA, SCALE, SUM, VALUE
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, DISNAN
      EXTERNAL           LSAME, DISNAN
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZLASSQ
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SQRT
*     ..
*     .. Executable Statements ..
*
      IF( N.EQ.0 ) THEN
         VALUE = ZERO
      ELSE IF( LSAME( NORM, 'M' ) ) THEN
*
*        Find max(abs(A(i,j))).
*
         VALUE = ZERO
         IF( LSAME( UPLO, 'U' ) ) THEN
            DO 20 J = 1, N
               DO 10 I = 1, J
                  SUM = ABS( A( I, J ) )
                  IF( VALUE .LT. SUM .OR. DISNAN( SUM ) ) VALUE = SUM
   10          CONTINUE
   20       CONTINUE
         ELSE
            DO 40 J = 1, N
               DO 30 I = J, N
                  SUM = ABS( A( I, J ) )
                  IF( VALUE .LT. SUM .OR. DISNAN( SUM ) ) VALUE = SUM
   30          CONTINUE
   40       CONTINUE
```

```
         END IF
      ELSE IF( ( LSAME( NORM, 'I' ) ) .OR. ( LSAME( NORM, 'O' ) ) .OR.
     $         ( NORM.EQ.'1' ) ) THEN
*
*        Find normI(A) ( = norm1(A), since A is symmetric).
*
         VALUE = ZERO
         IF( LSAME( UPLO, 'U' ) ) THEN
            DO 60 J = 1, N
               SUM = ZERO
               DO 50 I = 1, J - 1
                  ABSA = ABS( A( I, J ) )
                  SUM = SUM + ABSA
                  WORK( I ) = WORK( I ) + ABSA
   50          CONTINUE
               WORK( J ) = SUM + ABS( A( J, J ) )
   60       CONTINUE
            DO 70 I = 1, N
               SUM = WORK( I )
               IF( VALUE .LT. SUM .OR. DISNAN( SUM ) ) VALUE = SUM
   70       CONTINUE
         ELSE
            DO 80 I = 1, N
               WORK( I ) = ZERO
   80       CONTINUE
            DO 100 J = 1, N
               SUM = WORK( J ) + ABS( A( J, J ) )
               DO 90 I = J + 1, N
                  ABSA = ABS( A( I, J ) )
                  SUM = SUM + ABSA
                  WORK( I ) = WORK( I ) + ABSA
   90          CONTINUE
               IF( VALUE .LT. SUM .OR. DISNAN( SUM ) ) VALUE = SUM
  100       CONTINUE
         END IF
      ELSE IF( ( LSAME( NORM, 'F' ) ) .OR. ( LSAME( NORM, 'E' ) ) ) THEN
*
*        Find normF(A).
*
         SCALE = ZERO
         SUM = ONE
         IF( LSAME( UPLO, 'U' ) ) THEN
            DO 110 J = 2, N
               CALL ZLASSQ( J-1, A( 1, J ), 1, SCALE, SUM )
  110       CONTINUE
         ELSE
            DO 120 J = 1, N - 1
               CALL ZLASSQ( N-J, A( J+1, J ), 1, SCALE, SUM )
  120       CONTINUE
         END IF
```

```
         SUM = 2*SUM
         CALL ZLASSQ( N, A, LDA+1, SCALE, SUM )
         VALUE = SCALE*SQRT( SUM )
      END IF
*
      ZLANSY = VALUE
      RETURN
*
*     End of ZLANSY
*
      END
```

———-

65.2.4 lisp code

— zlansy.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun zlansy (norm uplo n a lda work)
    (declare (type (array double-float (*)) work)
             (type (array f2cl-lib:complex16 (*)) a)
             (type (f2cl-lib:integer4) lda n)
             (type (string *) uplo norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((absa 0.0d0) (scale 0.0d0) (sum 0.0d0) (value 0.0d0) (i 0) (j 0)
             (zlansy 0.0d0))
        (declare (type (f2cl-lib:integer4) i j)
                 (type (double-float) absa scale sum value zlansy))
        (cond ((= n 0) (setf value zero))
              ((multiple-value-bind (ret-val var-0 var-1)
                   (lsame norm "M")
                 (declare (ignore var-1))
                 (when var-0 (setf norm var-0))
                 ret-val)
               (setf value zero)
               (cond
                ((multiple-value-bind (ret-val var-0 var-1)
                     (lsame uplo "U")
                   (declare (ignore var-1))
                   (when var-0 (setf uplo var-0))
```

```
        ret-val)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf sum
                      (abs
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)))
              (if (or (< value sum)
                      (multiple-value-bind (ret-val var-0)
                          (disnan sum)
                        (declare (ignore))
                        (when var-0 (setf sum var-0))
                        ret-val))
                  (setf value sum))
             label10))
         label20)))
    (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf sum
                      (abs
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)))
              (if (or (< value sum)
                      (multiple-value-bind (ret-val var-0)
                          (disnan sum)
                        (declare (ignore))
                        (when var-0 (setf sum var-0))
                        ret-val))
                  (setf value sum))
             label30))
         label40)))))
  ((or
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame norm "I")
      (declare (ignore var-1))
      (when var-0 (setf norm var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame norm "O")
      (declare (ignore var-1))
      (when var-0 (setf norm var-0))
```

```
    ret-val)
  (f2cl-lib:fstring-= norm "1"))
 (setf value zero)
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame uplo "U")
     (declare (ignore var-1))
     (when var-0 (setf uplo var-0))
     ret-val)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf sum zero)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i
                         (f2cl-lib:int-add j
                                           (f2cl-lib:int-sub 1)))
                      nil)
         (tagbody
           (setf absa
                   (abs
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%)))
           (setf sum (+ sum absa))
           (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                   (+
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)
                    absa))
          label50))
       (setf (f2cl-lib:fref work-%data% (j) ((1 *))
                            work-%offset%)
               (+ sum
                  (abs
                   (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                  a-%offset%))))
     label60))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf sum
               (f2cl-lib:fref work-%data% (i) ((1 *))
                              work-%offset%))
       (if (or (< value sum)
               (multiple-value-bind (ret-val var-0)
                   (disnan sum)
                 (declare (ignore))
                 (when var-0 (setf sum var-0))
                 ret-val))
```

```
            (setf value sum))
        label70)))
    (t
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                              work-%offset%)
               zero)
        label80))
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (setf sum
                 (+
                  (f2cl-lib:fref work-%data% (j) ((1 *))
                                 work-%offset%)
                  (abs
                   (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                  a-%offset%))))
         (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                          (f2cl-lib:int-add i 1))
                        ((> i n) nil)
           (tagbody
             (setf absa
                     (abs
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%)))
             (setf sum (+ sum absa))
             (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                     (+
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%)
                      absa))
            label90))
         (if (or (< value sum)
                 (multiple-value-bind (ret-val var-0)
                     (disnan sum)
                   (declare (ignore))
                   (when var-0 (setf sum var-0))
                   ret-val))
             (setf value sum))
        label100)))))
((or
  (multiple-value-bind (ret-val var-0 var-1)
      (lsame norm "F")
    (declare (ignore var-1))
    (when var-0 (setf norm var-0))
    ret-val)
```

```
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame norm "E")
              (declare (ignore var-1))
              (when var-0 (setf norm var-0))
              ret-val))
           (setf scale zero) (setf sum one)
           (cond
            ((multiple-value-bind (ret-val var-0 var-1)
                 (lsame uplo "U")
               (declare (ignore var-1))
               (when var-0 (setf uplo var-0))
               ret-val)
             (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                           ((> j n) nil)
               (tagbody
                 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                     (zlassq (f2cl-lib:int-sub j 1)
                      (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                            (1 j) ((1 lda) (1 *))
                                            a-%offset%)
                      1 scale sum)
                   (declare (ignore var-0 var-1 var-2))
                   (when var-3 (setf scale var-3))
                   (when var-4 (setf sum var-4)))
                label110)))
            (t
             (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                           ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                            nil)
               (tagbody
                 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                     (zlassq (f2cl-lib:int-sub n j)
                      (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                            ((+ j 1) j) ((1 lda) (1 *))
                                            a-%offset%)
                      1 scale sum)
                   (declare (ignore var-0 var-1 var-2))
                   (when var-3 (setf scale var-3))
                   (when var-4 (setf sum var-4)))
                label120))))
           (setf sum (* 2 sum))
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
               (zlassq n a (f2cl-lib:int-add lda 1) scale sum)
             (declare (ignore var-1 var-2))
             (when var-0 (setf n var-0))
             (when var-3 (setf scale var-3))
             (when var-4 (setf sum var-4)))
           (setf value (* scale (f2cl-lib:fsqrt sum)))))
       (setf zlansy value)
       (go end_label)
```

```
       end_label
        (return (values zlansy norm uplo n nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zlansy
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*)))
                                            :return-values
                                            '(fortran-to-lisp::norm
                                              fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil nil nil)
                                            :calls 'nil)))
```

———-

65.3 zlaqsy

65.3.1 Axiom unit tests

— **zlaqsy.input** —

```
)set break resume
)sys rm -f zlaqsy.output
)spool zlaqsy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

65.3.2 Axiom help page

— zlaqsy.help —

```
=====================================================================
zlaqsy examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zlaqsy (UPLO, N, A, LDA, S, SCOND, AMAX, EQUED)
           ZLAQSY scales a symmetric/Hermitian matrix, using scaling factors
           computed by spoequ.

Function/Subroutine Documentation
   subroutine zlaqsy (character UPLO, integer N, complex*16, dimension( lda, *
       ) A, integer LDA, double precision, dimension( * ) S, double precision
       SCOND, double precision AMAX, character EQUED)
       ZLAQSY scales a symmetric/Hermitian matrix, using scaling factors
       computed by spoequ.

Purpose:
   ZLAQSY equilibrates a symmetric matrix A using the scaling factors
   in the vector S.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is stored.
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   n by n upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading n by n lower triangular part of A contains the lower
```

```
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if EQUED = 'Y', the equilibrated matrix:
  diag(S) * A * diag(S).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(N,1).

S
  S is DOUBLE PRECISION array, dimension (N)
  The scale factors for A.

SCOND
  SCOND is DOUBLE PRECISION
  Ratio of the smallest S(i) to the largest S(i).

AMAX
  AMAX is DOUBLE PRECISION
  Absolute value of largest matrix entry.

EQUED
  EQUED is CHARACTER*1
  Specifies whether or not equilibration was done.
  = 'N':  No equilibration.
  = 'Y':  Equilibration was done, i.e., A has been replaced by
          diag(S) * A * diag(S).

Internal Parameters:

  THRESH is a threshold value used to decide if scaling should be done
  based on the ratio of the scaling factors.  If SCOND < THRESH,
  scaling is done.

  LARGE and SMALL are threshold values used to decide if scaling should
  be done based on the absolute size of the largest matrix element.
  If AMAX > LARGE or AMAX < SMALL, scaling is done.
```

———-

65.3.3 fortran code

— zlaqsy.f —

```
*  =====================================================================
      SUBROUTINE ZLAQSY( UPLO, N, A, LDA, S, SCOND, AMAX, EQUED )
```

```
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, UPLO
      INTEGER            LDA, N
      DOUBLE PRECISION   AMAX, SCOND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   S( * )
      COMPLEX*16         A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, THRESH
      PARAMETER          ( ONE = 1.0D+0, THRESH = 0.1D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   CJ, LARGE, SMALL
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( N.LE.0 ) THEN
         EQUED = 'N'
         RETURN
      END IF
*
*     Initialize LARGE and SMALL.
*
      SMALL = DLAMCH( 'Safe minimum' ) / DLAMCH( 'Precision' )
      LARGE = ONE / SMALL
*
      IF( SCOND.GE.THRESH .AND. AMAX.GE.SMALL .AND. AMAX.LE.LARGE ) THEN
*
*        No equilibration
*
         EQUED = 'N'
```

```
      ELSE
*
*        Replace A by diag(S) * A * diag(S).
*
         IF( LSAME( UPLO, 'U' ) ) THEN
*
*           Upper triangle of A is stored.
*
            DO 20 J = 1, N
               CJ = S( J )
               DO 10 I = 1, J
                  A( I, J ) = CJ*S( I )*A( I, J )
   10          CONTINUE
   20       CONTINUE
         ELSE
*
*           Lower triangle of A is stored.
*
            DO 40 J = 1, N
               CJ = S( J )
               DO 30 I = J, N
                  A( I, J ) = CJ*S( I )*A( I, J )
   30          CONTINUE
   40       CONTINUE
         END IF
         EQUED = 'Y'
      END IF
*
      RETURN
*
*     End of ZLAQSY
*
      END
```

———-

65.3.4 lisp code

— **zlaqsy.lisp** —

```
(let* ((one 1.0d0) (thresh 0.1d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.1d0 0.1d0) thresh)
           (ignorable one thresh))
  (defun zlaqsy (uplo n a lda s scond amax equed)
    (declare (type (double-float) amax scond)
             (type (array double-float (*)) s)
             (type (array f2cl-lib:complex16 (*)) a)
```

```
          (type (f2cl-lib:integer4) lda n)
          (type (string *) equed uplo))
(f2cl-lib:with-multi-array-data
    ((uplo character uplo-%data% uplo-%offset%)
     (equed character equed-%data% equed-%offset%)
     (a f2cl-lib:complex16 a-%data% a-%offset%)
     (s double-float s-%data% s-%offset%))
  (prog ((cj 0.0d0) (large 0.0d0) (small 0.0d0) (i 0) (j 0))
    (declare (type (double-float) cj large small)
             (type (f2cl-lib:integer4) i j))
    (cond
     ((<= n 0) (f2cl-lib:f2cl-set-string equed "N" (string 1))
      (go end_label)))
    (setf small (/ (dlamch "Safe minimum") (dlamch "Precision")))
    (setf large (/ one small))
    (cond
     ((and (>= scond thresh) (>= amax small) (<= amax large))
      (f2cl-lib:f2cl-set-string equed "N" (string 1)))
     (t
      (cond
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf cj (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i j) nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                      (* cj
                         (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)))
               label10))
           label20)))
       (t
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf cj (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
            (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
```

```
                            (* cj
                               (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                               (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                              a-%offset%)))
                    label30))
                label40))))
          (f2cl-lib:f2cl-set-string equed "Y" (string 1))))
        (go end_label)
       end_label
        (return (values uplo nil nil nil nil nil nil equed))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zlaqsy
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (double-float) (double-float)
                                              (string))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil nil
                                              nil nil nil
                                              fortran-to-lisp::equed)
                                            :calls 'nil)))
```

———-

65.4 zsymv

65.4.1 Axiom unit tests

— zsymv.input —

```
)set break resume
)sys rm -f zsymv.output
)spool zsymv.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

65.4.2 Axiom help page

— **zsymv.help** —

```
==================================================================
zsymv examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zsymv (UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY)
           ZSYMV computes a matrix-vector product for a complex symmetric
           matrix.

Function/Subroutine Documentation
   subroutine zsymv (character UPLO, integer N, complex*16 ALPHA, complex*16,
       dimension( lda, * ) A, integer LDA, complex*16, dimension( * ) X,
       integer INCX, complex*16 BETA, complex*16, dimension( * ) Y, integer
       INCY)
       ZSYMV computes a matrix-vector product for a complex symmetric matrix.

Purpose:
   ZSYMV  performs the matrix-vector  operation

   y := alpha*A*x + beta*y,

   where alpha and beta are scalars, x and y are n element vectors and
   A is an n by n symmetric matrix.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the upper or lower
   triangular part of the array A is to be referenced as
   follows:

   UPLO = 'U' or 'u'   Only the upper triangular part of A is to be referenced.
```

```
  UPLO = 'L' or 'l'   Only the lower triangular part of A is to be referenced.

  Unchanged on exit.

N
  N is INTEGER
  On entry, N specifies the order of the matrix A.
  N must be at least zero.
  Unchanged on exit.

ALPHA
  ALPHA is COMPLEX*16
  On entry, ALPHA specifies the scalar alpha.
  Unchanged on exit.

A
  A is COMPLEX*16 array, dimension ( LDA, N )
  Before entry, with  UPLO = 'U' or 'u', the leading n by n
  upper triangular part of the array A must contain the upper
  triangular part of the symmetric matrix and the strictly
  lower triangular part of A is not referenced.
  Before entry, with UPLO = 'L' or 'l', the leading n by n
  lower triangular part of the array A must contain the lower
  triangular part of the symmetric matrix and the strictly
  upper triangular part of A is not referenced.
  Unchanged on exit.

LDA
  LDA is INTEGER
  On entry, LDA specifies the first dimension of A as declared
  in the calling (sub) program. LDA must be at least max( 1, N ).
  Unchanged on exit.

X
  X is COMPLEX*16 array, dimension at least
  ( 1 + ( N - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the N-element vector x.
                     Unchanged on exit.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of
  X. INCX must not be zero.
  Unchanged on exit.

BETA
  BETA is COMPLEX*16
  On entry, BETA specifies the scalar beta. When BETA is
  supplied as zero then Y need not be set on input.
  Unchanged on exit.
```

```
 Y
   Y is COMPLEX*16 array, dimension at least
   ( 1 + ( N - 1 )*abs( INCY ) ).
   Before entry, the incremented array Y must contain the n
   element vector y. On exit, Y is overwritten by the updated vector y.

 INCY
   INCY is INTEGER
   On entry, INCY specifies the increment for the elements of
   Y. INCY must not be zero.
   Unchanged on exit.
```

———-

65.4.3 fortran code

— **zsymv.f** —

```
*  =====================================================================
      SUBROUTINE ZSYMV( UPLO, N, ALPHA, A, LDA, X, INCX, BETA, Y, INCY )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INCX, INCY, LDA, N
      COMPLEX*16         ALPHA, BETA
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), X( * ), Y( * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ) )
      COMPLEX*16         ZERO
      PARAMETER          ( ZERO = ( 0.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, INFO, IX, IY, J, JX, JY, KX, KY
      COMPLEX*16         TEMP1, TEMP2
```

```
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = 1
      ELSE IF( N.LT.0 ) THEN
         INFO = 2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = 5
      ELSE IF( INCX.EQ.0 ) THEN
         INFO = 7
      ELSE IF( INCY.EQ.0 ) THEN
         INFO = 10
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYMV ', INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( ( N.EQ.0 ) .OR. ( ( ALPHA.EQ.ZERO ) .AND. ( BETA.EQ.ONE ) ) )
     $   RETURN
*
*     Set up the start points in  X  and  Y.
*
      IF( INCX.GT.0 ) THEN
         KX = 1
      ELSE
         KX = 1 - ( N-1 )*INCX
      END IF
      IF( INCY.GT.0 ) THEN
         KY = 1
      ELSE
         KY = 1 - ( N-1 )*INCY
      END IF
*
```

```
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the triangular part
*     of A.
*
*     First form  y := beta*y.
*
      IF( BETA.NE.ONE ) THEN
         IF( INCY.EQ.1 ) THEN
            IF( BETA.EQ.ZERO ) THEN
               DO 10 I = 1, N
                  Y( I ) = ZERO
   10          CONTINUE
            ELSE
               DO 20 I = 1, N
                  Y( I ) = BETA*Y( I )
   20          CONTINUE
            END IF
         ELSE
            IY = KY
            IF( BETA.EQ.ZERO ) THEN
               DO 30 I = 1, N
                  Y( IY ) = ZERO
                  IY = IY + INCY
   30          CONTINUE
            ELSE
               DO 40 I = 1, N
                  Y( IY ) = BETA*Y( IY )
                  IY = IY + INCY
   40          CONTINUE
            END IF
         END IF
      END IF
      IF( ALPHA.EQ.ZERO )
     $   RETURN
      IF( LSAME( UPLO, 'U' ) ) THEN
*
*        Form  y  when A is stored in upper triangle.
*
         IF( ( INCX.EQ.1 ) .AND. ( INCY.EQ.1 ) ) THEN
            DO 60 J = 1, N
               TEMP1 = ALPHA*X( J )
               TEMP2 = ZERO
               DO 50 I = 1, J - 1
                  Y( I ) = Y( I ) + TEMP1*A( I, J )
                  TEMP2 = TEMP2 + A( I, J )*X( I )
   50          CONTINUE
               Y( J ) = Y( J ) + TEMP1*A( J, J ) + ALPHA*TEMP2
   60       CONTINUE
         ELSE
            JX = KX
```

```
            JY = KY
            DO 80 J = 1, N
               TEMP1 = ALPHA*X( JX )
               TEMP2 = ZERO
               IX = KX
               IY = KY
               DO 70 I = 1, J - 1
                  Y( IY ) = Y( IY ) + TEMP1*A( I, J )
                  TEMP2 = TEMP2 + A( I, J )*X( IX )
                  IX = IX + INCX
                  IY = IY + INCY
   70          CONTINUE
               Y( JY ) = Y( JY ) + TEMP1*A( J, J ) + ALPHA*TEMP2
               JX = JX + INCX
               JY = JY + INCY
   80       CONTINUE
         END IF
      ELSE
*
*        Form  y  when A is stored in lower triangle.
*
         IF( ( INCX.EQ.1 ) .AND. ( INCY.EQ.1 ) ) THEN
            DO 100 J = 1, N
               TEMP1 = ALPHA*X( J )
               TEMP2 = ZERO
               Y( J ) = Y( J ) + TEMP1*A( J, J )
               DO 90 I = J + 1, N
                  Y( I ) = Y( I ) + TEMP1*A( I, J )
                  TEMP2 = TEMP2 + A( I, J )*X( I )
   90          CONTINUE
               Y( J ) = Y( J ) + ALPHA*TEMP2
  100       CONTINUE
         ELSE
            JX = KX
            JY = KY
            DO 120 J = 1, N
               TEMP1 = ALPHA*X( JX )
               TEMP2 = ZERO
               Y( JY ) = Y( JY ) + TEMP1*A( J, J )
               IX = JX
               IY = JY
               DO 110 I = J + 1, N
                  IX = IX + INCX
                  IY = IY + INCY
                  Y( IY ) = Y( IY ) + TEMP1*A( I, J )
                  TEMP2 = TEMP2 + A( I, J )*X( IX )
  110          CONTINUE
               Y( JY ) = Y( JY ) + ALPHA*TEMP2
               JX = JX + INCX
               JY = JY + INCY
```

```
  120       CONTINUE
         END IF
      END IF
*
      RETURN
*
*     End of ZSYMV
*
      END
```

———-

65.4.4 lisp code

— **zsymv.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)) (zero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (type (f2cl-lib:complex16) zero)
           (ignorable one zero))
  (defun zsymv (uplo n alpha a lda x incx beta y incy)
    (declare (type (array f2cl-lib:complex16 (*)) y x a)
             (type (f2cl-lib:complex16) beta alpha)
             (type (f2cl-lib:integer4) incy incx lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (x f2cl-lib:complex16 x-%data% x-%offset%)
         (y f2cl-lib:complex16 y-%data% y-%offset%))
      (prog ((temp1 #C(0.0d0 0.0d0)) (temp2 #C(0.0d0 0.0d0)) (i 0) (info 0)
             (ix 0) (iy 0) (j 0) (jx 0) (jy 0) (kx 0) (ky 0))
        (declare (type (f2cl-lib:complex16) temp1 temp2)
                 (type (f2cl-lib:integer4) i info ix iy j jx jy kx ky))
        (setf info 0)
        (cond
         ((and
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
```

```
      ret-val)))
  (setf info 1))
 ((< n 0) (setf info 2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info 5))
 ((= incx 0) (setf info 7)) ((= incy 0) (setf info 10)))
(cond ((/= info 0) (xerbla "ZSYMV " info) (go end_label)))
(if (or (= n 0) (and (= alpha zero) (= beta one)))
    (go end_label))
(cond ((> incx 0) (setf kx 1))
      (t
       (setf kx
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incx)))))
(cond ((> incy 0) (setf ky 1))
      (t
       (setf ky
               (f2cl-lib:int-sub 1
                                 (f2cl-lib:int-mul
                                  (f2cl-lib:int-sub n 1) incy)))))
(cond
 ((/= beta one)
  (cond
   ((= incy 1)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%) zero)
         label10)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                  (* beta
                     (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)))
         label20)))))
   (t (setf iy ky)
    (cond
     ((= beta zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%) zero)
          (setf iy (f2cl-lib:int-add iy incy))
         label30)))
     (t
```

```
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                  (* beta
                     (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)))
          (setf iy (f2cl-lib:int-add iy incy))
         label40))))))))
(if (= alpha zero)
    (go end_label))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame uplo "U")
    (declare (ignore var-1))
    (when var-0 (setf uplo var-0))
    ret-val)
  (cond
   ((and (= incx 1) (= incy 1))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf temp1
                (* alpha
                   (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
        (setf temp2 zero)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                    (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                       (* temp1
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%))))
            (setf temp2
                    (+ temp2
                       (*
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%)
                        (f2cl-lib:fref x-%data% (i) ((1 *))
                                       x-%offset%))))
           label50))
        (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                   (* temp1
                      (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                     a-%offset%))
                   (* alpha temp2)))
       label60)))
   (t (setf jx kx) (setf jy ky)
```

```
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf temp1
                  (* alpha
                     (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
          (setf temp2 zero)
          (setf ix kx)
          (setf iy ky)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                      (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                         (* temp1
                            (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                           a-%offset%))))
              (setf temp2
                      (+ temp2
                         (*
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref x-%data% (ix) ((1 *))
                                         x-%offset%))))
              (setf ix (f2cl-lib:int-add ix incx))
              (setf iy (f2cl-lib:int-add iy incy))
             label70))
          (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                  (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                     (* temp1
                        (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                       a-%offset%))
                     (* alpha temp2)))
          (setf jx (f2cl-lib:int-add jx incx))
          (setf jy (f2cl-lib:int-add jy incy))
         label80)))))
    (t
     (cond
      ((and (= incx 1) (= incy 1))
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (setf temp1
                   (* alpha
                      (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
           (setf temp2 zero)
           (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                   (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                      (* temp1
```

```
                    (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                   a-%offset%))))
      (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                  (+ (f2cl-lib:fref y-%data% (i) ((1 *)) y-%offset%)
                     (* temp1
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))))
          (setf temp2
                  (+ temp2
                     (*
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%)
                      (f2cl-lib:fref x-%data% (i) ((1 *))
                                     x-%offset%))))
         label90))
      (setf (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
              (+ (f2cl-lib:fref y-%data% (j) ((1 *)) y-%offset%)
                 (* alpha temp2)))
    label100)))
(t (setf jx kx) (setf jy ky)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf temp1
             (* alpha
                (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
     (setf temp2 zero)
     (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
             (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                (* temp1
                   (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                  a-%offset%))))
     (setf ix jx)
     (setf iy jy)
     (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf ix (f2cl-lib:int-add ix incx))
         (setf iy (f2cl-lib:int-add iy incy))
         (setf (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                 (+ (f2cl-lib:fref y-%data% (iy) ((1 *)) y-%offset%)
                    (* temp1
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%))))
         (setf temp2
                 (+ temp2
                    (*
```

```
                                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                               a-%offset%)
                                (f2cl-lib:fref x-%data% (ix) ((1 *))
                                               x-%offset%))))
                   label110))
                (setf (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                        (+ (f2cl-lib:fref y-%data% (jy) ((1 *)) y-%offset%)
                           (* alpha temp2)))
                (setf jx (f2cl-lib:int-add jx incx))
                (setf jy (f2cl-lib:int-add jy incy))
               label120))))))
        (go end_label)
       end_label
        (return (values uplo nil nil nil nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsymv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::complex16)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::complex16)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil nil
                                              nil nil nil nil nil nil)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

65.5 zsyr

65.5.1 Axiom unit tests

— zsyr.input —

```
)set break resume
)sys rm -f zsyr.output
)spool zsyr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

65.5.2 Axiom help page

— zsyr.help —

```
==================================================================
zsyr examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zsyr (UPLO, N, ALPHA, X, INCX, A, LDA)
           ZSYR performs the symmetric rank-1 update of a complex symmetric
           matrix.

Function/Subroutine Documentation
   subroutine zsyr (character UPLO, integer N, complex*16 ALPHA, complex*16,
       dimension( * ) X, integer INCX, complex*16, dimension( lda, * ) A,
       integer LDA)
       ZSYR performs the symmetric rank-1 update of a complex symmetric
       matrix.

Purpose:
   ZSYR   performs the symmetric rank 1 operation

   A := alpha*x*x**H + A,

   where alpha is a complex scalar, x is an n element vector and A is an
   n by n symmetric matrix.

Parameters:

 UPLO
```

```
  UPLO is CHARACTER*1
  On entry, UPLO specifies whether the upper or lower
  triangular part of the array A is to be referenced as
  follows:

  UPLO = 'U' or 'u'   Only the upper triangular part of A is to be referenced.
  UPLO = 'L' or 'l'   Only the lower triangular part of A is to be referenced.

  Unchanged on exit.

N
  N is INTEGER
  On entry, N specifies the order of the matrix A.
  N must be at least zero.
  Unchanged on exit.

ALPHA
  ALPHA is COMPLEX*16
  On entry, ALPHA specifies the scalar alpha.
  Unchanged on exit.

X
  X is COMPLEX*16 array, dimension at least
  ( 1 + ( N - 1 )*abs( INCX ) ).
  Before entry, the incremented array X must contain the N-element vector x.
  Unchanged on exit.

INCX
  INCX is INTEGER
  On entry, INCX specifies the increment for the elements of
  X. INCX must not be zero.
  Unchanged on exit.

A
  A is COMPLEX*16 array, dimension ( LDA, N )
  Before entry, with  UPLO = 'U' or 'u', the leading n by n
  upper triangular part of the array A must contain the upper
  triangular part of the symmetric matrix and the strictly
  lower triangular part of A is not referenced. On exit, the
  upper triangular part of the array A is overwritten by the
  upper triangular part of the updated matrix.
  Before entry, with UPLO = 'L' or 'l', the leading n by n
  lower triangular part of the array A must contain the lower
  triangular part of the symmetric matrix and the strictly
  upper triangular part of A is not referenced. On exit, the
  lower triangular part of the array A is overwritten by the
  lower triangular part of the updated matrix.

LDA
  LDA is INTEGER
```

```
On entry, LDA specifies the first dimension of A as declared
in the calling (sub) program. LDA must be at least max( 1, N ).
Unchanged on exit.
```

———-

65.5.3 fortran code

— **zsyr.f** —

```
*  =====================================================================
      SUBROUTINE ZSYR( UPLO, N, ALPHA, X, INCX, A, LDA )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INCX, LDA, N
      COMPLEX*16         ALPHA
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), X( * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ZERO
      PARAMETER          ( ZERO = ( 0.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, INFO, IX, J, JX, KX
      COMPLEX*16         TEMP
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
```

```
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = 1
      ELSE IF( N.LT.0 ) THEN
         INFO = 2
      ELSE IF( INCX.EQ.0 ) THEN
         INFO = 5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = 7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZSYR  ', INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( ( N.EQ.0 ) .OR. ( ALPHA.EQ.ZERO ) )
     $   RETURN
*
*     Set the start point in X if the increment is not unity.
*
      IF( INCX.LE.0 ) THEN
         KX = 1 - ( N-1 )*INCX
      ELSE IF( INCX.NE.1 ) THEN
         KX = 1
      END IF
*
*     Start the operations. In this version the elements of A are
*     accessed sequentially with one pass through the triangular part
*     of A.
*
      IF( LSAME( UPLO, 'U' ) ) THEN
*
*        Form  A  when A is stored in upper triangle.
*
         IF( INCX.EQ.1 ) THEN
            DO 20 J = 1, N
               IF( X( J ).NE.ZERO ) THEN
                  TEMP = ALPHA*X( J )
                  DO 10 I = 1, J
                     A( I, J ) = A( I, J ) + X( I )*TEMP
   10             CONTINUE
               END IF
   20       CONTINUE
         ELSE
```

```
         JX = KX
         DO 40 J = 1, N
            IF( X( JX ).NE.ZERO ) THEN
               TEMP = ALPHA*X( JX )
               IX = KX
               DO 30 I = 1, J
                  A( I, J ) = A( I, J ) + X( IX )*TEMP
                  IX = IX + INCX
   30          CONTINUE
            END IF
            JX = JX + INCX
   40    CONTINUE
      END IF
   ELSE
*
*     Form  A  when A is stored in lower triangle.
*
      IF( INCX.EQ.1 ) THEN
         DO 60 J = 1, N
            IF( X( J ).NE.ZERO ) THEN
               TEMP = ALPHA*X( J )
               DO 50 I = J, N
                  A( I, J ) = A( I, J ) + X( I )*TEMP
   50          CONTINUE
            END IF
   60    CONTINUE
      ELSE
         JX = KX
         DO 80 J = 1, N
            IF( X( JX ).NE.ZERO ) THEN
               TEMP = ALPHA*X( JX )
               IX = JX
               DO 70 I = J, N
                  A( I, J ) = A( I, J ) + X( IX )*TEMP
                  IX = IX + INCX
   70          CONTINUE
            END IF
            JX = JX + INCX
   80    CONTINUE
      END IF
   END IF
*
   RETURN
*
*  End of ZSYR
*
   END
```

———-

65.5.4 lisp code

— zsyr.lisp —

```
(let* ((zero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) zero)
           (ignorable zero))
  (defun zsyr (uplo n alpha x incx a lda)
    (declare (type (array f2cl-lib:complex16 (*)) a x)
             (type (f2cl-lib:complex16) alpha)
             (type (f2cl-lib:integer4) lda incx n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (x f2cl-lib:complex16 x-%data% x-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%))
      (prog ((temp #C(0.0d0 0.0d0)) (i 0) (info 0) (ix 0) (j 0) (jx 0) (kx 0))
        (declare (type (f2cl-lib:complex16) temp)
                 (type (f2cl-lib:integer4) i info ix j jx kx))
        (setf info 0)
        (cond
         ((and
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
          (setf info 1))
         ((< n 0) (setf info 2)) ((= incx 0) (setf info 5))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info 7)))
        (cond ((/= info 0) (xerbla "ZSYR  " info) (go end_label)))
        (if (or (= n 0) (= alpha zero))
            (go end_label))
        (cond
         ((<= incx 0)
          (setf kx
                  (f2cl-lib:int-sub 1
                                    (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                      incx))))
         ((/= incx 1) (setf kx 1)))
        (cond
```

```
((multiple-value-bind (ret-val var-0 var-1)
     (lsame uplo "U")
   (declare (ignore var-1))
   (when var-0 (setf uplo var-0))
   ret-val)
 (cond
  ((= incx 1)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
         (setf temp
                 (* alpha
                    (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i j) nil)
           (tagbody
             (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                     (+
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%)
                      (*
                       (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                       temp)))
           label10))))
     label20)))
  (t (setf jx kx)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (cond
        ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
         (setf temp
                 (* alpha
                    (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
         (setf ix kx)
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i j) nil)
           (tagbody
             (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                     (+
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%)
                      (*
                       (f2cl-lib:fref x-%data% (ix) ((1 *))
                                      x-%offset%)
                       temp)))
```

```
                (setf ix (f2cl-lib:int-add ix incx))
               label30))))
          (setf jx (f2cl-lib:int-add jx incx))
         label40)))))
    (t
     (cond
      ((= incx 1)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (cond
            ((/= (f2cl-lib:fref x (j) ((1 *))) zero)
             (setf temp
                     (* alpha
                        (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
             (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                           ((> i n) nil)
               (tagbody
                 (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                       (+
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%)
                        (*
                         (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                         temp)))
               label50))))
         label60)))
      (t (setf jx kx)
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (cond
            ((/= (f2cl-lib:fref x (jx) ((1 *))) zero)
             (setf temp
                     (* alpha
                        (f2cl-lib:fref x-%data% (jx) ((1 *)) x-%offset%)))
             (setf ix jx)
             (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                           ((> i n) nil)
               (tagbody
                 (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                       (+
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%)
                        (*
                         (f2cl-lib:fref x-%data% (ix) ((1 *))
                                        x-%offset%)
                         temp)))
```

```
                      (setf ix (f2cl-lib:int-add ix incx))
                     label70))))
                (setf jx (f2cl-lib:int-add jx incx))
               label80))))))
        (go end_label)
       end_label
        (return (values uplo nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zsyr fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::complex16)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil nil
                                              nil nil nil)
                                            :calls '(fortran-to-lisp::xerbla))))
```

65.6 zsyswapr

65.6.1 Axiom unit tests

— zsyswapr.input —

```
)set break resume
)sys rm -f zsyswapr.output
)spool zsyswapr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

65.6.2 Axiom help page

— zsyswapr.help —

```
=====================================================================
zsyswapr examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zsyswapr (UPLO, N, A, LDA, I1, I2)
           ZSYSWAPR

Function/Subroutine Documentation
   subroutine zsyswapr (character UPLO, integer N, complex*16, dimension( lda,
       n ) A, integer LDA, integer I1, integer I2)
       ZSYSWAPR

Purpose:
   ZSYSWAPR applies an elementary permutation on the rows and the columns of
   a symmetric matrix.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the details of the factorization are stored
   as an upper or lower triangular matrix.
   = 'U':  Upper triangular, form is A = U*D*U**T;
   = 'L':  Lower triangular, form is A = L*D*L**T.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the NB diagonal matrix D and the multipliers
   used to obtain the factor U or L as computed by ZSYTRF.

   On exit, if INFO = 0, the (symmetric) inverse of the original
   matrix.  If UPLO = 'U', the upper triangular part of the
```

```
  inverse is formed and the part of A below the diagonal is not
  referenced; if UPLO = 'L' the lower triangular part of the
  inverse is formed and the part of A above the diagonal is
  not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

I1
  I1 is INTEGER
  Index of the first row to swap

I2
  I2 is INTEGER
  Index of the second row to swap
```

———-

65.6.3 fortran code

— zsyswapr.f —

```
*  =====================================================================
      SUBROUTINE ZSYSWAPR( UPLO, N, A, LDA, I1, I2)
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER        UPLO
      INTEGER          I1, I2, LDA, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16       A( LDA, N )
*
*  =====================================================================
*
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            I
      COMPLEX*16         TMP
*
*     .. External Functions ..
```

```
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZSWAP
*     ..
*     .. Executable Statements ..
*
      UPPER = LSAME( UPLO, 'U' )
      IF (UPPER) THEN
*
*         UPPER
*         first swap
*          - swap column I1 and I2 from I1 to I1-1
         CALL ZSWAP( I1-1, A(1,I1), 1, A(1,I2), 1 )
*
*          second swap :
*          - swap A(I1,I1) and A(I2,I2)
*          - swap row I1 from I1+1 to I2-1 with col I2 from I1+1 to I2-1
         TMP=A(I1,I1)
         A(I1,I1)=A(I2,I2)
         A(I2,I2)=TMP
*
         DO I=1,I2-I1-1
            TMP=A(I1,I1+I)
            A(I1,I1+I)=A(I1+I,I2)
            A(I1+I,I2)=TMP
         END DO
*
*          third swap
*          - swap row I1 and I2 from I2+1 to N
         DO I=I2+1,N
            TMP=A(I1,I)
            A(I1,I)=A(I2,I)
            A(I2,I)=TMP
         END DO
*
        ELSE
*
*         LOWER
*         first swap
*          - swap row I1 and I2 from I1 to I1-1
         CALL ZSWAP( I1-1, A(I1,1), LDA, A(I2,1), LDA )
*
*         second swap :
*          - swap A(I1,I1) and A(I2,I2)
*          - swap col I1 from I1+1 to I2-1 with row I2 from I1+1 to I2-1
          TMP=A(I1,I1)
          A(I1,I1)=A(I2,I2)
          A(I2,I2)=TMP
```

```
*
         DO I=1,I2-I1-1
            TMP=A(I1+I,I1)
            A(I1+I,I1)=A(I2,I1+I)
            A(I2,I1+I)=TMP
         END DO
*
*        third swap
*         - swap col I1 and I2 from I2+1 to N
         DO I=I2+1,N
            TMP=A(I,I1)
            A(I,I1)=A(I,I2)
            A(I,I2)=TMP
         END DO
*
      ENDIF
      END SUBROUTINE ZSYSWAPR
```

———-

65.6.4 lisp code

— **zsyswapr.lisp** —

———-

65.7 ztgsy2

65.7.1 Axiom unit tests

— **ztgsy2.input** —

```
)set break resume
)sys rm -f ztgsy2.output
)spool ztgsy2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

65.7.2 Axiom help page

— ztgsy2.help —

```
==================================================================
ztgsy2 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine ztgsy2 (TRANS, IJOB, M, N, A, LDA, B, LDB, C, LDC, D, LDD,
           E, LDE, F, LDF, SCALE, RDSUM, RDSCAL, INFO)
           ZTGSY2 solves the generalized Sylvester equation (unblocked
           algorithm).

Function/Subroutine Documentation
   subroutine ztgsy2 (character TRANS, integer IJOB, integer M, integer N,
       complex*16, dimension( lda, * ) A, integer LDA, complex*16, dimension(
       ldb, * ) B, integer LDB, complex*16, dimension( ldc, * ) C, integer
       LDC, complex*16, dimension( ldd, * ) D, integer LDD, complex*16,
       dimension( lde, * ) E, integer LDE, complex*16, dimension( ldf, * ) F,
       integer LDF, double precision SCALE, double precision RDSUM, double
       precision RDSCAL, integer INFO)
       ZTGSY2 solves the generalized Sylvester equation (unblocked algorithm).

Purpose:
   ZTGSY2 solves the generalized Sylvester equation

   A * R - L * B = scale * C               (1)
   D * R - L * E = scale * F

   using Level 1 and 2 BLAS, where R and L are unknown M-by-N matrices,
   (A, D), (B, E) and (C, F) are given matrix pairs of size M-by-M,
   N-by-N and M-by-N, respectively. A, B, D and E are upper triangular
   (i.e., (A,D) and (B,E) in generalized Schur form).

   The solution (R, L) overwrites (C, F). 0 <= SCALE <= 1 is an output
   scaling factor chosen to avoid overflow.

   In matrix notation solving equation (1) corresponds to solve
   Zx = scale * b, where Z is defined as
```

```
  Z = [ kron(In, A)  -kron(B**H, Im) ]             (2)
      [ kron(In, D)  -kron(E**H, Im) ],

  Ik is the identity matrix of size k and X**H is the conjuguate
  transpose of X.  kron(X, Y) is the Kronecker product
  between the matrices X and Y.

  If TRANS = 'C', y in the conjugate transposed system Z**H*y = scale*b
  is solved for, which is equivalent to solve for R and L in

  A**H * R  + D**H * L   = scale * C           (3)
  R  * B**H + L  * E**H  = scale * -F

  This case is used to compute an estimate of Dif[(A, D), (B, E)] =
  = sigma_min(Z) using reverse communicaton with ZLACON.

  ZTGSY2 also (IJOB >= 1) contributes to the computation in ZTGSYL
  of an upper bound on the separation between to matrix pairs. Then
  the input (A, D), (B, E) are sub-pencils of two matrix pairs in ZTGSYL.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   = 'N', solve the generalized Sylvester equation (1).
   = 'T': solve the 'transposed' system (3).

 IJOB
   IJOB is INTEGER
   Specifies what kind of functionality to be performed.
   =0: solve (1) only.
   =1: A contribution from this subsystem to a Frobenius
       norm-based estimate of the separation between two matrix
       pairs is computed. (look ahead strategy is used).
   =2: A contribution from this subsystem to a Frobenius
       norm-based estimate of the separation between two matrix
       pairs is computed. (DGECON on sub-systems is used.)
   Not referenced if TRANS = 'T'.

 M
   M is INTEGER
   On entry, M specifies the order of A and D, and the row
   dimension of C, F, R and L.

 N
   N is INTEGER
   On entry, N specifies the order of B and E, and the column
   dimension of C, F, R and L.

 A
```

```
  A is COMPLEX*16 array, dimension (LDA, M)
  On entry, A contains an upper triangular matrix.

LDA
  LDA is INTEGER
  The leading dimension of the matrix A. LDA >= max(1, M).

B
  B is COMPLEX*16 array, dimension (LDB, N)
  On entry, B contains an upper triangular matrix.

LDB
  LDB is INTEGER
  The leading dimension of the matrix B. LDB >= max(1, N).

C
  C is COMPLEX*16 array, dimension (LDC, N)
  On entry, C contains the right-hand-side of the first matrix
  equation in (1).
  On exit, if IJOB = 0, C has been overwritten by the solution R.

LDC
  LDC is INTEGER
  The leading dimension of the matrix C. LDC >= max(1, M).

D
  D is COMPLEX*16 array, dimension (LDD, M)
  On entry, D contains an upper triangular matrix.

LDD
  LDD is INTEGER
  The leading dimension of the matrix D. LDD >= max(1, M).

E
  E is COMPLEX*16 array, dimension (LDE, N)
  On entry, E contains an upper triangular matrix.

LDE
  LDE is INTEGER
  The leading dimension of the matrix E. LDE >= max(1, N).

F
  F is COMPLEX*16 array, dimension (LDF, N)
  On entry, F contains the right-hand-side of the second matrix
  equation in (1).
  On exit, if IJOB = 0, F has been overwritten by the solution L.

LDF
  LDF is INTEGER
  The leading dimension of the matrix F. LDF >= max(1, M).
```

```
 SCALE
   SCALE is DOUBLE PRECISION
   On exit, 0 <= SCALE <= 1. If 0 < SCALE < 1, the solutions
   R and L (C and F on entry) will hold the solutions to a
   slightly perturbed system but the input matrices A, B, D and
   E have not been changed. If SCALE = 0, R and L will hold the
   solutions to the homogeneous system with C = F = 0.
   Normally, SCALE = 1.

 RDSUM
   RDSUM is DOUBLE PRECISION
   On entry, the sum of squares of computed contributions to
   the Dif-estimate under computation by ZTGSYL, where the
   scaling factor RDSCAL (see below) has been factored out.
   On exit, the corresponding sum of squares updated with the
   contributions from the current sub-system.
   If TRANS = 'T' RDSUM is not touched.
   NOTE: RDSUM only makes sense when ZTGSY2 is called by ZTGSYL.

 RDSCAL
   RDSCAL is DOUBLE PRECISION
   On entry, scaling factor used to prevent overflow in RDSUM.
   On exit, RDSCAL is updated w.r.t. the current contributions
   in RDSUM.
   If TRANS = 'T', RDSCAL is not touched.
   NOTE: RDSCAL only makes sense when ZTGSY2 is called by ZTGSYL.

 INFO
   INFO is INTEGER
   On exit, if INFO is set to
   =0: Successful exit
   <0: If INFO = -i, input argument number i is illegal.
   >0: The matrix pairs (A, D) and (B, E) have common or very
       close eigenvalues.
```

———-

65.7.3 fortran code

— **ztgsy2.f** —

```
*  =====================================================================
      SUBROUTINE ZTGSY2( TRANS, IJOB, M, N, A, LDA, B, LDB, C, LDC, D,
     $                   LDD, E, LDE, F, LDF, SCALE, RDSUM, RDSCAL,
     $                   INFO )
*
```

```
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            IJOB, INFO, LDA, LDB, LDC, LDD, LDE, LDF, M, N
      DOUBLE PRECISION   RDSCAL, RDSUM, SCALE
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), B( LDB, * ), C( LDC, * ),
     $                   D( LDD, * ), E( LDE, * ), F( LDF, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      INTEGER            LDZ
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0, LDZ = 2 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      INTEGER            I, IERR, J, K
      DOUBLE PRECISION   SCALOC
      COMPLEX*16         ALPHA
*     ..
*     .. Local Arrays ..
      INTEGER            IPIV( LDZ ), JPIV( LDZ )
      COMPLEX*16         RHS( LDZ ), Z( LDZ, LDZ )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZAXPY, ZGESC2, ZGETC2, ZLATDF, ZSCAL
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DCMPLX, DCONJG, MAX
*     ..
*     .. Executable Statements ..
*
*     Decode and test input parameters
*
      INFO = 0
      IERR = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'C' ) ) THEN
```

```
         INFO = -1
      ELSE IF( NOTRAN ) THEN
         IF( ( IJOB.LT.0 ) .OR. ( IJOB.GT.2 ) ) THEN
            INFO = -2
         END IF
      END IF
      IF( INFO.EQ.0 ) THEN
         IF( M.LE.0 ) THEN
            INFO = -3
         ELSE IF( N.LE.0 ) THEN
            INFO = -4
         ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
            INFO = -6
         ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
            INFO = -8
         ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
            INFO = -10
         ELSE IF( LDD.LT.MAX( 1, M ) ) THEN
            INFO = -12
         ELSE IF( LDE.LT.MAX( 1, N ) ) THEN
            INFO = -14
         ELSE IF( LDF.LT.MAX( 1, M ) ) THEN
            INFO = -16
         END IF
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZTGSY2', -INFO )
         RETURN
      END IF
*
      IF( NOTRAN ) THEN
*
*        Solve (I, J) - system
*           A(I, I) * R(I, J) - L(I, J) * B(J, J) = C(I, J)
*           D(I, I) * R(I, J) - L(I, J) * E(J, J) = F(I, J)
*        for I = M, M - 1, ..., 1; J = 1, 2, ..., N
*
         SCALE = ONE
         SCALOC = ONE
         DO 30 J = 1, N
            DO 20 I = M, 1, -1
*
*              Build 2 by 2 system
*
               Z( 1, 1 ) = A( I, I )
               Z( 2, 1 ) = D( I, I )
               Z( 1, 2 ) = -B( J, J )
               Z( 2, 2 ) = -E( J, J )
*
*              Set up right hand side(s)
```

```
*
              RHS( 1 ) = C( I, J )
              RHS( 2 ) = F( I, J )
*
*             Solve Z * x = RHS
*
              CALL ZGETC2( LDZ, Z, LDZ, IPIV, JPIV, IERR )
              IF( IERR.GT.0 )
     $           INFO = IERR
              IF( IJOB.EQ.0 ) THEN
                 CALL ZGESC2( LDZ, Z, LDZ, RHS, IPIV, JPIV, SCALOC )
                 IF( SCALOC.NE.ONE ) THEN
                    DO 10 K = 1, N
                       CALL ZSCAL( M, DCMPLX( SCALOC, ZERO ),
     $                             C( 1, K ), 1 )
                       CALL ZSCAL( M, DCMPLX( SCALOC, ZERO ),
     $                             F( 1, K ), 1 )
   10               CONTINUE
                    SCALE = SCALE*SCALOC
                 END IF
              ELSE
                 CALL ZLATDF( IJOB, LDZ, Z, LDZ, RHS, RDSUM, RDSCAL,
     $                        IPIV, JPIV )
              END IF
*
*             Unpack solution vector(s)
*
              C( I, J ) = RHS( 1 )
              F( I, J ) = RHS( 2 )
*
*             Substitute R(I, J) and L(I, J) into remaining equation.
*
              IF( I.GT.1 ) THEN
                 ALPHA = -RHS( 1 )
                 CALL ZAXPY( I-1, ALPHA, A( 1, I ), 1, C( 1, J ), 1 )
                 CALL ZAXPY( I-1, ALPHA, D( 1, I ), 1, F( 1, J ), 1 )
              END IF
              IF( J.LT.N ) THEN
                 CALL ZAXPY( N-J, RHS( 2 ), B( J, J+1 ), LDB,
     $                       C( I, J+1 ), LDC )
                 CALL ZAXPY( N-J, RHS( 2 ), E( J, J+1 ), LDE,
     $                       F( I, J+1 ), LDF )
              END IF
*
   20       CONTINUE
   30    CONTINUE
      ELSE
*
*        Solve transposed (I, J) - system:
*           A(I, I)**H * R(I, J) + D(I, I)**H * L(J, J) = C(I, J)
```

```
*          R(I, I) * B(J, J) + L(I, J) * E(J, J)   = -F(I, J)
*       for I = 1, 2, ..., M, J = N, N - 1, ..., 1
*
        SCALE = ONE
        SCALOC = ONE
        DO 80 I = 1, M
           DO 70 J = N, 1, -1
*
*             Build 2 by 2 system Z**H
*
              Z( 1, 1 ) = DCONJG( A( I, I ) )
              Z( 2, 1 ) = -DCONJG( B( J, J ) )
              Z( 1, 2 ) = DCONJG( D( I, I ) )
              Z( 2, 2 ) = -DCONJG( E( J, J ) )
*
*
*             Set up right hand side(s)
*
              RHS( 1 ) = C( I, J )
              RHS( 2 ) = F( I, J )
*
*             Solve Z**H * x = RHS
*
              CALL ZGETC2( LDZ, Z, LDZ, IPIV, JPIV, IERR )
              IF( IERR.GT.0 )
     $           INFO = IERR
              CALL ZGESC2( LDZ, Z, LDZ, RHS, IPIV, JPIV, SCALOC )
              IF( SCALOC.NE.ONE ) THEN
                 DO 40 K = 1, N
                    CALL ZSCAL( M, DCMPLX( SCALOC, ZERO ), C( 1, K ),
     $                          1 )
                    CALL ZSCAL( M, DCMPLX( SCALOC, ZERO ), F( 1, K ),
     $                          1 )
   40            CONTINUE
                 SCALE = SCALE*SCALOC
              END IF
*
*             Unpack solution vector(s)
*
              C( I, J ) = RHS( 1 )
              F( I, J ) = RHS( 2 )
*
*             Substitute R(I, J) and L(I, J) into remaining equation.
*
              DO 50 K = 1, J - 1
                 F( I, K ) = F( I, K ) + RHS( 1 )*DCONJG( B( K, J ) ) +
     $                       RHS( 2 )*DCONJG( E( K, J ) )
   50         CONTINUE
              DO 60 K = I + 1, M
                 C( K, J ) = C( K, J ) - DCONJG( A( I, K ) )*RHS( 1 ) -
```

```
     $                        DCONJG( D( I, K ) )*RHS( 2 )
   60          CONTINUE
*
   70       CONTINUE
   80    CONTINUE
      END IF
      RETURN
*
*     End of ZTGSY2
*
      END
```

———-

65.7.4 lisp code

— **ztgsy2.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0) (ldz 2))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (f2cl-lib:integer4 2 2) ldz)
           (ignorable zero one ldz))
  (defun ztgsy2
         (trans ijob m n a lda b ldb$ c ldc d ldd e lde f ldf scale rdsum
          rdscal info)
    (declare (type (double-float) rdscal rdsum scale)
             (type (array f2cl-lib:complex16 (*)) f e d c b a)
             (type (f2cl-lib:integer4) info ldf lde ldd ldc ldb$ lda n m ijob)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (c f2cl-lib:complex16 c-%data% c-%offset%)
         (d f2cl-lib:complex16 d-%data% d-%offset%)
         (e f2cl-lib:complex16 e-%data% e-%offset%)
         (f f2cl-lib:complex16 f-%data% f-%offset%))
      (prog ((rhs (make-array ldz :element-type 'f2cl-lib:complex16))
             (z
              (make-array (the fixnum (reduce #'* (list ldz ldz)))
                          :element-type 'f2cl-lib:complex16))
             (ipiv (make-array ldz :element-type 'f2cl-lib:integer4))
             (jpiv (make-array ldz :element-type 'f2cl-lib:integer4))
             (alpha #C(0.0d0 0.0d0)) (scaloc 0.0d0) (i 0) (ierr 0) (j 0) (k 0)
             (notran nil))
        (declare (type (array f2cl-lib:complex16 (*)) rhs z)
                 (type (array f2cl-lib:integer4 (*)) ipiv jpiv)
```

```
         (type (f2cl-lib:complex16) alpha)
         (type (double-float) scaloc)
         (type (f2cl-lib:integer4) i ierr j k)
         (type f2cl-lib:logical notran))
(setf info 0)
(setf ierr 0)
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -1))
 (notran (cond ((or (< ijob 0) (> ijob 2)) (setf info -2)))))
(cond
 ((= info 0)
  (cond ((<= m 0) (setf info -3)) ((<= n 0) (setf info -4))
        ((< lda
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -6))
        ((< ldb$
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -8))
        ((< ldc
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -10))
        ((< ldd
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -12))
        ((< lde
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -14))
        ((< ldf
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
         (setf info -16)))))
(cond
 ((/= info 0) (xerbla "ZTGSY2" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (notran (setf scale one) (setf scaloc one)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
```

```
(tagbody
  (f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                ((> i 1) nil)
    (tagbody
      (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
              (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                              a-%offset%))
      (setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz)))
              (f2cl-lib:fref d-%data% (i i) ((1 ldd) (1 *))
                              d-%offset%))
      (setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz)))
              (-
               (f2cl-lib:fref b-%data% (j j) ((1 ldb$) (1 *))
                               b-%offset%)))
      (setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
              (-
               (f2cl-lib:fref e-%data% (j j) ((1 lde) (1 *))
                               e-%offset%)))
      (setf (f2cl-lib:fref rhs (1) ((1 ldz)))
              (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                              c-%offset%))
      (setf (f2cl-lib:fref rhs (2) ((1 ldz)))
              (f2cl-lib:fref f-%data% (i j) ((1 ldf) (1 *))
                              f-%offset%))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (zgetc2 ldz z ldz ipiv jpiv ierr)
        (declare (ignore var-1 var-3 var-4))
        (setf ldz var-0)
        (setf ldz var-2)
        (setf ierr var-5))
      (if (> ierr 0)
          (setf info ierr))
      (cond
       ((= ijob 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zgesc2 ldz z ldz rhs ipiv jpiv scaloc)
          (declare (ignore var-1 var-3 var-4 var-5))
          (setf ldz var-0)
          (setf ldz var-2)
          (setf scaloc var-6))
        (cond
         ((/= scaloc one)
          (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                        ((> k n) nil)
            (tagbody
              (multiple-value-bind (var-0 var-1 var-2 var-3)
                  (zscal m (f2cl-lib:dcmplx scaloc zero)
                   (f2cl-lib:array-slice c-%data%
                                         f2cl-lib:complex16 (1 k)
```

```
                                        ((1 ldc) (1 *))
                                        c-%offset%)
                 1)
              (declare (ignore var-1 var-2 var-3))
              (when var-0 (setf m var-0)))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (zscal m (f2cl-lib:dcmplx scaloc zero)
                 (f2cl-lib:array-slice f-%data%
                                        f2cl-lib:complex16 (1 k)
                                        ((1 ldf) (1 *))
                                        f-%offset%)
                 1)
              (declare (ignore var-1 var-2 var-3))
              (when var-0 (setf m var-0)))
           label10))
        (setf scale (* scale scaloc)))))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (zlatdf ijob ldz z ldz rhs rdsum rdscal ipiv jpiv)
        (declare (ignore var-0 var-2 var-4 var-7 var-8))
        (setf ldz var-1)
        (setf ldz var-3)
        (setf rdsum var-5)
        (setf rdscal var-6))))
    (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                         c-%offset%)
            (f2cl-lib:fref rhs (1) ((1 ldz))))
    (setf (f2cl-lib:fref f-%data% (i j) ((1 ldf) (1 *))
                         f-%offset%)
            (f2cl-lib:fref rhs (2) ((1 ldz))))
    (cond
     ((> i 1) (setf alpha (- (f2cl-lib:fref rhs (1) ((1 ldz)))))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (zaxpy (f2cl-lib:int-sub i 1) alpha
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                 (1 i) ((1 lda) (1 *))
                                 a-%offset%)
           1
           (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                 (1 j) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-0 var-2 var-3 var-4 var-5))
        (when var-1 (setf alpha var-1)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (zaxpy (f2cl-lib:int-sub i 1) alpha
           (f2cl-lib:array-slice d-%data% f2cl-lib:complex16
                                 (1 i) ((1 ldd) (1 *))
                                 d-%offset%)
```

```
                    1
                    (f2cl-lib:array-slice f-%data% f2cl-lib:complex16
                                          (1 j) ((1 ldf) (1 *))
                                          f-%offset%)
                    1)
                (declare (ignore var-0 var-2 var-3 var-4 var-5))
                (when var-1 (setf alpha var-1)))))
            (cond
             ((< j n)
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                  (zaxpy (f2cl-lib:int-sub n j)
                    (f2cl-lib:array-slice rhs f2cl-lib:complex16 (2)
                                          ((1 ldz)))
                    (f2cl-lib:array-slice b-%data% f2cl-lib:complex16
                                          (j (f2cl-lib:int-add j 1))
                                          ((1 ldb$) (1 *)) b-%offset%)
                    ldb$
                    (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                          (i (f2cl-lib:int-add j 1))
                                          ((1 ldc) (1 *)) c-%offset%)
                    ldc)
                (declare (ignore var-0 var-1 var-2 var-4))
                (when var-3 (setf ldb$ var-3))
                (when var-5 (setf ldc var-5)))
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                  (zaxpy (f2cl-lib:int-sub n j)
                    (f2cl-lib:array-slice rhs f2cl-lib:complex16 (2)
                                          ((1 ldz)))
                    (f2cl-lib:array-slice e-%data% f2cl-lib:complex16
                                          (j (f2cl-lib:int-add j 1))
                                          ((1 lde) (1 *)) e-%offset%)
                    lde
                    (f2cl-lib:array-slice f-%data% f2cl-lib:complex16
                                          (i (f2cl-lib:int-add j 1))
                                          ((1 ldf) (1 *)) f-%offset%)
                    ldf)
                (declare (ignore var-0 var-1 var-2 var-4))
                (when var-3 (setf lde var-3))
                (when var-5 (setf ldf var-5)))))
          label20))
     label30)))
  (t (setf scale one) (setf scaloc one)
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i m) nil)
     (tagbody
       (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                     ((> j 1) nil)
         (tagbody
           (setf (f2cl-lib:fref z (1 1) ((1 ldz) (1 ldz)))
                   (coerce
```

```
          (f2cl-lib:dconjg
           (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                          a-%offset%))
          'f2cl-lib:complex16))
(setf (f2cl-lib:fref z (2 1) ((1 ldz) (1 ldz)))
        (coerce
         (-
          (f2cl-lib:dconjg
           (f2cl-lib:fref b-%data% (j j) ((1 ldb$) (1 *))
                          b-%offset%)))
         'f2cl-lib:complex16))
(setf (f2cl-lib:fref z (1 2) ((1 ldz) (1 ldz)))
        (coerce
         (f2cl-lib:dconjg
          (f2cl-lib:fref d-%data% (i i) ((1 ldd) (1 *))
                         d-%offset%))
         'f2cl-lib:complex16))
(setf (f2cl-lib:fref z (2 2) ((1 ldz) (1 ldz)))
        (coerce
         (-
          (f2cl-lib:dconjg
           (f2cl-lib:fref e-%data% (j j) ((1 lde) (1 *))
                          e-%offset%)))
         'f2cl-lib:complex16))
(setf (f2cl-lib:fref rhs (1) ((1 ldz)))
        (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                       c-%offset%))
(setf (f2cl-lib:fref rhs (2) ((1 ldz)))
        (f2cl-lib:fref f-%data% (i j) ((1 ldf) (1 *))
                       f-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (zgetc2 ldz z ldz ipiv jpiv ierr)
  (declare (ignore var-1 var-3 var-4))
  (setf ldz var-0)
  (setf ldz var-2)
  (setf ierr var-5))
(if (> ierr 0)
    (setf info ierr))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zgesc2 ldz z ldz rhs ipiv jpiv scaloc)
  (declare (ignore var-1 var-3 var-4 var-5))
  (setf ldz var-0)
  (setf ldz var-2)
  (setf scaloc var-6))
(cond
 ((/= scaloc one)
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
```

```
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (zscal m (f2cl-lib:dcmplx scaloc zero)
           (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                 (1 k) ((1 ldc) (1 *))
                                 c-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf m var-0)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (zscal m (f2cl-lib:dcmplx scaloc zero)
           (f2cl-lib:array-slice f-%data% f2cl-lib:complex16
                                 (1 k) ((1 ldf) (1 *))
                                 f-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3))
        (when var-0 (setf m var-0)))
     label40))
  (setf scale (* scale scaloc))))
(setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                     c-%offset%)
        (f2cl-lib:fref rhs (1) ((1 ldz))))
(setf (f2cl-lib:fref f-%data% (i j) ((1 ldf) (1 *))
                     f-%offset%)
        (f2cl-lib:fref rhs (2) ((1 ldz))))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k
                  (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref f-%data% (i k) ((1 ldf) (1 *))
                         f-%offset%)
          (+
           (f2cl-lib:fref f-%data% (i k) ((1 ldf) (1 *))
                          f-%offset%)
           (* (f2cl-lib:fref rhs (1) ((1 ldz)))
              (f2cl-lib:dconjg
               (f2cl-lib:fref b-%data% (k j)
                              ((1 ldb$) (1 *))
                              b-%offset%)))
           (* (f2cl-lib:fref rhs (2) ((1 ldz)))
              (f2cl-lib:dconjg
               (f2cl-lib:fref e-%data% (k j)
                              ((1 lde) (1 *))
                              e-%offset%)))))
   label50))
(f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
               (f2cl-lib:int-add k 1))
              ((> k m) nil)
  (tagbody
    (setf (f2cl-lib:fref c-%data% (k j) ((1 ldc) (1 *))
```

```
                                        c-%offset%)
                              (-
                               (f2cl-lib:fref c-%data% (k j) ((1 ldc) (1 *))
                                              c-%offset%)
                               (*
                                (f2cl-lib:dconjg
                                 (f2cl-lib:fref a-%data% (i k) ((1 lda) (1 *))
                                                a-%offset%))
                                (f2cl-lib:fref rhs (1) ((1 ldz))))
                               (*
                                (f2cl-lib:dconjg
                                 (f2cl-lib:fref d-%data% (i k) ((1 ldd) (1 *))
                                                d-%offset%))
                                (f2cl-lib:fref rhs (2) ((1 ldz))))))
                      label60))
                  label70))
              label80))))
        (go end_label)
       end_label
        (return
         (values trans nil m nil nil nil nil ldb$ nil ldc nil nil nil lde nil
                 ldf scale rdsum rdscal info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ztgsy2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
```

```
  (fortran-to-lisp::integer4)
  (double-float) (double-float)
  (double-float)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::trans nil
  fortran-to-lisp::m nil nil nil
  nil fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldc nil nil nil
  fortran-to-lisp::lde nil
  fortran-to-lisp::ldf
  fortran-to-lisp::scale
  fortran-to-lisp::rdsum
  fortran-to-lisp::rdscal
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zlatdf
  fortran-to-lisp::zgesc2
  fortran-to-lisp::zgetc2
  fortran-to-lisp::xerbla))))
```

Chapter 66

LAPACK - Positive Definite Matrix, Linear Solve, Double

66.1 dposv

66.1.1 Axiom unit tests

— dposv.input —

```
)set break resume
)sys rm -f dposv.output
)spool dposv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

66.1.2 Axiom help page

— dposv.help —

```
========================================================================
dposv examples
========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dposv (UPLO, N, NRHS, A, LDA, B, LDB, INFO)
            DPOSV computes the solution to system of linear equations A * X =
           B for PO matrices

Function/Subroutine Documentation
   subroutine dposv (character UPLO, integer N, integer NRHS, double
       precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldb, * ) B, integer LDB, integer INFO)
        DPOSV computes the solution to system of linear equations A * X = B
       for PO matrices

Purpose:
   DPOSV computes the solution to a real system of linear equations
      A * X = B,
   where A is an N-by-N symmetric positive definite matrix and X and B
   are N-by-NRHS matrices.

   The Cholesky decomposition is used to factor A as
      A = U**T* U,  if UPLO = 'U', or
      A = L * L**T,  if UPLO = 'L',
   where U is an upper triangular matrix and L is a lower triangular
   matrix.  The factored form of A is then used to solve the system of
   equations A * X = B.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the
   matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
```

```
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if INFO = 0, the factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, the leading minor of order i of A is not
        positive definite, so the factorization could not be
        completed, and the solution has not been computed.
```

———-

66.1.3 fortran code

— dposv.f —

```
*  ===================================================================
      SUBROUTINE DPOSV( UPLO, N, NRHS, A, LDA, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
```

```
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DPOTRF, DPOTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DPOSV ', -INFO )
         RETURN
      END IF
*
*     Compute the Cholesky factorization A = U**T*U or A = L*L**T.
*
      CALL DPOTRF( UPLO, N, A, LDA, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
         CALL DPOTRS( UPLO, N, NRHS, A, LDA, B, LDB, INFO )
*
      END IF
```

```
      RETURN
*
*     End of DPOSV
*
      END
```

———-

66.1.4 lisp code

— **dposv.lisp** —

```
(defun dposv (uplo n nrhs a lda b ldb$ info)
  (declare (type (array double-float (*)) b a)
           (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a double-float a-%data% a-%offset%)
       (b double-float b-%data% b-%offset%))
    (prog ()
      (declare)
      (setf info 0)
      (cond
       ((and
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -5))
       ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -7)))
      (cond
       ((/= info 0) (xerbla "DPOSV " (f2cl-lib:int-sub info)) (go end_label)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dpotrf uplo n a lda info)
        (declare (ignore var-2))
```

```
        (when var-0 (setf uplo var-0))
        (when var-1 (setf n var-1))
        (when var-3 (setf lda var-3))
        (when var-4 (setf info var-4)))
      (cond
       ((= info 0)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (dpotrs uplo n nrhs a lda b ldb$ info)
          (declare (ignore var-3 var-5))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf lda var-4))
          (when var-6 (setf ldb$ var-6))
          (when var-7 (setf info var-7)))))
      (go end_label)
     end_label
      (return (values uplo n nrhs nil lda nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dposv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

66.2 dposvx

66.2.1 Axiom unit tests

— **dposvx.input** —

```
)set break resume
)sys rm -f dposvx.output
)spool dposvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

66.2.2 Axiom help page

— **dposvx.help** —

```
====================================================================
dposvx examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dposvx (FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, EQUED, S, B,
           LDB, X, LDX, RCOND, FERR, BERR, WORK, IWORK, INFO)
            DPOSVX computes the solution to system of linear equations A * X =
           B for PO matrices

Function/Subroutine Documentation
   subroutine dposvx (character FACT, character UPLO, integer N, integer NRHS,
       double precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldaf, * ) AF, integer LDAF, character EQUED, double
       precision, dimension( * ) S, double precision, dimension( ldb, * ) B,
       integer LDB, double precision, dimension( ldx, * ) X, integer LDX,
       double precision RCOND, double precision, dimension( * ) FERR, double
       precision, dimension( * ) BERR, double precision, dimension( * ) WORK,
       integer, dimension( * ) IWORK, integer INFO)
```

```
       DPOSVX computes the solution to system of linear equations A * X = B
      for PO matrices

Purpose:
   DPOSVX uses the Cholesky factorization A = U**T*U or A = L*L**T to
   compute the solution to a real system of linear equations
      A * X = B,
   where A is an N-by-N symmetric positive definite matrix and X and B
   are N-by-NRHS matrices.

   Error bounds on the solution and a condition estimate are also provided.

Description:

   The following steps are performed:

   1. If FACT = 'E', real scaling factors are computed to equilibrate
      the system:
         diag(S) * A * diag(S) * inv(diag(S)) * X = diag(S) * B
      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(S)*A*diag(S) and B by diag(S)*B.

   2. If FACT = 'N' or 'E', the Cholesky decomposition is used to
      factor the matrix A (after equilibration if FACT = 'E') as
         A = U**T* U,  if UPLO = 'U', or
         A = L * L**T,  if UPLO = 'L',
      where U is an upper triangular matrix and L is a lower triangular
      matrix.

   3. If the leading i-by-i principal minor is not positive definite,
      then the routine returns with INFO = i. Otherwise, the factored
      form of A is used to estimate the condition number of the matrix A.
      If the reciprocal of the condition number is less than machine
      precision, INFO = N+1 is returned as a warning, but the routine
      still goes on to solve for X and compute error bounds as
      described below.

   4. The system of equations is solved for X using the factored form of A.

   5. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(S) so that it solves the original system before equilibration.

Parameters:

 FACT
   FACT is CHARACTER*1
```

```
  Specifies whether or not the factored form of the matrix A is
  supplied on entry, and if not, whether the matrix A should be
  equilibrated before it is factored.
  = 'F':  On entry, AF contains the factored form of A.
          If EQUED = 'Y', the matrix A has been equilibrated
          with scaling factors given by S.  A and AF will not be modified.
  = 'N':  The matrix A will be copied to AF and factored.
  = 'E':  The matrix A will be equilibrated if necessary, then
          copied to AF and factored.

UPLO
  UPLO is CHARACTER*1
  = 'U':  Upper triangle of A is stored;
  = 'L':  Lower triangle of A is stored.

N
  N is INTEGER
  The number of linear equations, i.e., the order of the
  matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the symmetric matrix A, except if FACT = 'F' and
  EQUED = 'Y', then A must contain the equilibrated matrix
  diag(S)*A*diag(S).  If UPLO = 'U', the leading
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.  A is not modified if
  FACT = 'F' or 'N', or if FACT = 'E' and EQUED = 'N' on exit.

  On exit, if FACT = 'E' and EQUED = 'Y', A is overwritten by
  diag(S)*A*diag(S).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is DOUBLE PRECISION array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the triangular factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T, in the same storage
```

```
  format as A.  If EQUED .ne. 'N', then AF is the factored form
  of the equilibrated matrix diag(S)*A*diag(S).

  If FACT = 'N', then AF is an output argument and on exit
  returns the triangular factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T of the original
  matrix A.

  If FACT = 'E', then AF is an output argument and on exit
  returns the triangular factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T of the equilibrated
  matrix A (see the description of A for the form of the
  equilibrated matrix).

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'Y':  Equilibration was done, i.e., A has been replaced by
          diag(S) * A * diag(S).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

S
  S is DOUBLE PRECISION array, dimension (N)
  The scale factors for A; not accessed if EQUED = 'N'.  S is
  an input argument if FACT = 'F'; otherwise, S is an output
  argument.  If FACT = 'F' and EQUED = 'Y', each element of S
  must be positive.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if EQUED = 'N', B is not modified; if EQUED = 'Y',
  B is overwritten by diag(S) * B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X to
  the original system of equations.  Note that if EQUED = 'Y',
  A and B are modified on exit, and the solution to the
  equilibrated system is inv(diag(S))*X.
```

```
LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  The estimate of the reciprocal condition number of the matrix
  A after equilibration (if done).  If RCOND is less than the
  machine precision (in particular, if RCOND = 0), the matrix
  is singular to working precision.  This condition is
  indicated by a return code of INFO > 0.

FERR
  FERR is DOUBLE PRECISION array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is DOUBLE PRECISION array, dimension (3*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = i, and i is
  <= N:  the leading minor of order i of A is
       not positive definite, so the factorization
       could not be completed, and the solution has not
       been computed. RCOND = 0 is returned.
  = N+1: U is nonsingular, but RCOND is less than machine
       precision, meaning that the matrix is singular
       to working precision.  Nevertheless, the
       solution and error bounds are computed because
       there are a number of situations where the
```

```
        computed solution can be more accurate than the
        value of RCOND would suggest.
```

———-

66.2.3 fortran code

— **dposvx.f** —

```
*  =====================================================================
      SUBROUTINE DPOSVX( FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, EQUED,
     $                   S, B, LDB, X, LDX, RCOND, FERR, BERR, WORK,
     $                   IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   BERR( * ), FERR( * ), S( * ), WORK( * ),
     $                   X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            EQUIL, NOFACT, RCEQU
      INTEGER            I, INFEQU, J
      DOUBLE PRECISION   AMAX, ANORM, BIGNUM, SCOND, SMAX, SMIN, SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, DLANSY
      EXTERNAL           LSAME, DLAMCH, DLANSY
*     ..
```

```
*     .. External Subroutines ..
      EXTERNAL           DLACPY, DLAQSY, DPOCON, DPOEQU, DPORFS, DPOTRF,
     $                   DPOTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         RCEQU = .FALSE.
      ELSE
         RCEQU = LSAME( EQUED, 'Y' )
         SMLNUM = DLAMCH( 'Safe minimum' )
         BIGNUM = ONE / SMLNUM
      END IF
*
*     Test the input parameters.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.LSAME( FACT, 'F' ) )
     $     THEN
         INFO = -1
      ELSE IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) )
     $          THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $         ( RCEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -9
      ELSE
         IF( RCEQU ) THEN
            SMIN = BIGNUM
            SMAX = ZERO
            DO 10 J = 1, N
               SMIN = MIN( SMIN, S( J ) )
               SMAX = MAX( SMAX, S( J ) )
   10       CONTINUE
            IF( SMIN.LE.ZERO ) THEN
               INFO = -10
```

```
            ELSE IF( N.GT.0 ) THEN
               SCOND = MAX( SMIN, SMLNUM ) / MIN( SMAX, BIGNUM )
            ELSE
               SCOND = ONE
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -12
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -14
            END IF
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DPOSVX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*        Compute row and column scalings to equilibrate the matrix A.
*
         CALL DPOEQU( N, A, LDA, S, SCOND, AMAX, INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*           Equilibrate the matrix.
*
            CALL DLAQSY( UPLO, N, A, LDA, S, SCOND, AMAX, EQUED )
            RCEQU = LSAME( EQUED, 'Y' )
         END IF
      END IF
*
*     Scale the right hand side.
*
      IF( RCEQU ) THEN
         DO 30 J = 1, NRHS
            DO 20 I = 1, N
               B( I, J ) = S( I )*B( I, J )
   20       CONTINUE
   30    CONTINUE
      END IF
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the Cholesky factorization A = U**T *U or A = L*L**T.
*
         CALL DLACPY( UPLO, N, N, A, LDA, AF, LDAF )
         CALL DPOTRF( UPLO, N, AF, LDAF, INFO )
```

```
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 )THEN
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A.
*
      ANORM = DLANSY( '1', UPLO, N, A, LDA, WORK )
*
*     Compute the reciprocal of the condition number of A.
*
      CALL DPOCON( UPLO, N, AF, LDAF, ANORM, RCOND, WORK, IWORK, INFO )
*
*     Compute the solution matrix X.
*
      CALL DLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL DPOTRS( UPLO, N, NRHS, AF, LDAF, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL DPORFS( UPLO, N, NRHS, A, LDA, AF, LDAF, B, LDB, X, LDX,
     $             FERR, BERR, WORK, IWORK, INFO )
*
*     Transform the solution matrix X to a solution of the original
*     system.
*
      IF( RCEQU ) THEN
         DO 50 J = 1, NRHS
            DO 40 I = 1, N
               X( I, J ) = S( I )*X( I, J )
   40       CONTINUE
   50    CONTINUE
         DO 60 J = 1, NRHS
            FERR( J ) = FERR( J ) / SCOND
   60    CONTINUE
      END IF
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.DLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      RETURN
*
*     End of DPOSVX
```

```
*
      END
```

———-

66.2.4 lisp code

— **dposvx.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dposvx
         (fact uplo n nrhs a lda af ldaf equed s b ldb$ x ldx rcond ferr berr
          work iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (double-float) rcond)
             (type (array double-float (*)) work berr ferr x b s af a)
             (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
             (type (string *) equed uplo fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (equed character equed-%data% equed-%offset%)
         (a double-float a-%data% a-%offset%)
         (af double-float af-%data% af-%offset%)
         (s double-float s-%data% s-%offset%)
         (b double-float b-%data% b-%offset%)
         (x double-float x-%data% x-%offset%)
         (ferr double-float ferr-%data% ferr-%offset%)
         (berr double-float berr-%data% berr-%offset%)
         (work double-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((amax 0.0d0) (anorm 0.0d0) (bignum 0.0d0) (scond 0.0d0)
             (smax 0.0d0) (smin 0.0d0) (smlnum 0.0d0) (i 0) (infequ 0) (j 0)
             (equil nil) (nofact nil) (rcequ nil))
        (declare (type (double-float) amax anorm bignum scond smax smin smlnum)
                 (type (f2cl-lib:integer4) i infequ j)
                 (type f2cl-lib:logical equil nofact rcequ))
        (setf info 0)
        (setf nofact
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame fact "N")
                  (declare (ignore var-1))
                  (when var-0 (setf fact var-0))
                  ret-val))
        (setf equil
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "E")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val))
(cond
 ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
  (setf rcequ f2cl-lib:%false%))
 (t
  (setf rcequ
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame equed "Y")
            (declare (ignore var-1))
            (when var-0 (setf equed var-0))
            ret-val))
  (setf smlnum (dlamch "Safe minimum")) (setf bignum (/ one smlnum))))
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame fact "F")
     (declare (ignore var-1))
     (when var-0 (setf fact var-0))
     ret-val)
```

```
    (not
     (or rcequ
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame equed "N")
           (declare (ignore var-1))
           (when var-0 (setf equed var-0))
           ret-val))))
  (setf info -9))
 (t
  (cond
   (rcequ (setf smin bignum) (setf smax zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf smin
                (min smin
                     (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
        (setf smax
                (max smax
                     (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
       label10))
    (cond ((<= smin zero) (setf info -10))
          ((> n 0)
           (setf scond (/ (max smin smlnum) (min smax bignum))))
          (t (setf scond one)))))
  (cond
   ((= info 0)
    (cond
     ((< ldb$
         (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -12))
     ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -14)))))))
(cond
 ((/= info 0) (xerbla "DPOSVX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (equil
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dpoequ n a lda s scond amax infequ)
    (declare (ignore var-0 var-1 var-2 var-3))
    (setf scond var-4)
    (setf amax var-5)
    (setf infequ var-6))
  (cond
   ((= infequ 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (dlaqsy uplo n a lda s scond amax equed)
      (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
```

```
      (setf uplo var-0)
      (setf equed var-7))
    (setf rcequ
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame equed "Y")
              (declare (ignore var-1))
              (when var-0 (setf equed var-0))
              ret-val))))))
(cond
 (rcequ
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                  (* (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)))
         label20))
     label30))))
(cond
 ((or nofact equil)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy uplo n n a lda af ldaf)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
    (setf uplo var-0))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dpotrf uplo n af ldaf info)
    (declare (ignore var-2))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf ldaf var-3))
    (when var-4 (setf info var-4)))
  (cond ((> info 0) (setf rcond zero) (go end_label)))))
(setf anorm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlansy "1" uplo n a lda work)
          (declare (ignore var-0 var-3 var-4 var-5))
          (setf uplo var-1)
          (setf n var-2)
          ret-val))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dpocon uplo n af ldaf anorm rcond work iwork info)
  (declare (ignore var-2 var-4 var-6 var-7))
  (setf uplo var-0)
```

```
  (setf n var-1)
  (setf ldaf var-3)
  (setf rcond var-5)
  (setf info var-8))
(dlacpy "Full" n nrhs b ldb$ x ldx)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dpotrs uplo n nrhs af ldaf x ldx info)
  (declare (ignore var-3 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldaf var-4))
  (when var-6 (setf ldx var-6))
  (when var-7 (setf info var-7)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15)
    (dporfs uplo n nrhs a lda af ldaf b ldb$ x ldx ferr berr work iwork
     info)
  (declare
   (ignore var-2 var-3 var-5 var-7 var-8 var-9 var-10 var-11 var-12
    var-13 var-14))
  (setf uplo var-0)
  (setf n var-1)
  (setf lda var-4)
  (setf ldaf var-6)
  (setf info var-15))
(cond
 (rcequ
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                               x-%offset%)
                  (* (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                     (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                    x-%offset%)))
         label40))
     label50))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
              (/ (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                 scond))
     label60))))
(if (< rcond (dlamch "Epsilon"))
```

```
            (setf info (f2cl-lib:int-add n 1)))
        (go end_label)
       end_label
        (return
         (values fact uplo n nrhs nil lda nil ldaf equed nil nil nil nil ldx
                 rcond nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dposvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (string) (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::fact
                                              fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldaf
                                              fortran-to-lisp::equed nil nil
                                              nil nil fortran-to-lisp::ldx
                                              fortran-to-lisp::rcond nil nil
                                              nil nil fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::dporfs
                                              fortran-to-lisp::dpocon
                                              fortran-to-lisp::dlansy
                                              fortran-to-lisp::dlacpy
                                              fortran-to-lisp::dlaqsy
                                              fortran-to-lisp::dpoequ))))
```

———-

66.3 dposvxx

66.3.1 Axiom unit tests

— **dposvxx.input** —

```
)set break resume
)sys rm -f dposvxx.output
)spool dposvxx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

66.3.2 Axiom help page [Demm05]

— **dposvxx.help** —

```
======================================================================
dposvxx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dposvxx (FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, EQUED, S, B,
           LDB, X, LDX, RCOND, RPVGRW, BERR, N_ERR_BNDS, ERR_BNDS_NORM,
           ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, IWORK, INFO)
            DPOSVXX computes the solution to system of linear equations A * X
           = B for PO matrices

Function/Subroutine Documentation
```

```
   subroutine dposvxx (character FACT, character UPLO, integer N, integer
       NRHS, double precision, dimension( lda, * ) A, integer LDA, double
       precision, dimension( ldaf, * ) AF, integer LDAF, character EQUED,
       double precision, dimension( * ) S, double precision, dimension( ldb, *
       ) B, integer LDB, double precision, dimension( ldx, * ) X, integer LDX,
       double precision RCOND, double precision RPVGRW, double precision,
       dimension( * ) BERR, integer N_ERR_BNDS, double precision, dimension(
       nrhs, * ) ERR_BNDS_NORM, double precision, dimension( nrhs, * )
       ERR_BNDS_COMP, integer NPARAMS, double precision, dimension( * )
       PARAMS, double precision, dimension( * ) WORK, integer, dimension( * )
       IWORK, integer INFO)
        DPOSVXX computes the solution to system of linear equations A * X = B
       for PO matrices

Purpose:
   DPOSVXX uses the Cholesky factorization A = U**T*U or A = L*L**T
   to compute the solution to a double precision system of linear equations
   A * X = B, where A is an N-by-N symmetric positive definite matrix
   and X and B are N-by-NRHS matrices.

   If requested, both normwise and maximum componentwise error bounds
   are returned. DPOSVXX will return a solution with a tiny
   guaranteed error (O(eps) where eps is the working machine
   precision) unless the matrix is very ill-conditioned, in which
   case a warning is returned. Relevant condition numbers also are
   calculated and returned.

   DPOSVXX accepts user-provided factorizations and equilibration
   factors; see the definitions of the FACT and EQUED options.
   Solving with refinement and using a factorization from a previous
   DPOSVXX call will also produce a solution with either O(eps)
   errors or warnings, but we cannot make that claim for general
   user-provided factorizations and equilibration factors if they
   differ from what DPOSVXX would itself produce.

Description:

   The following steps are performed:

   1. If FACT = 'E', double precision scaling factors are computed to
      equilibrate the system:

      diag(S)*A*diag(S)     *inv(diag(S))*X = diag(S)*B

      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(S)*A*diag(S) and B by diag(S)*B.

   2. If FACT = 'N' or 'E', the Cholesky decomposition is used to
      factor the matrix A (after equilibration if FACT = 'E') as
```

```
         A = U**T* U,  if UPLO = 'U', or
         A = L * L**T,  if UPLO = 'L',
      where U is an upper triangular matrix and L is a lower triangular
      matrix.

   3. If the leading i-by-i principal minor is not positive definite,
      then the routine returns with INFO = i. Otherwise, the factored
      form of A is used to estimate the condition number of the matrix
      A (see argument RCOND).  If the reciprocal of the condition number
      is less than machine precision, the routine still goes on to solve
      for X and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. By default (unless PARAMS(LA_LINRX_ITREF_I) is set to zero),
      the routine will use iterative refinement to try to get a small
      error and error bounds.  Refinement calculates the residual to at
      least twice the working precision.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(S) so that it solves the original system before equilibration.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AF contains the factored form of A.
      If EQUED is not 'N', the matrix A has been
      equilibrated with scaling factors given by S.
      A and AF are not modified.
   = 'N':  The matrix A will be copied to AF and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
      copied to AF and factored.

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
```

```
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  On entry, the symmetric matrix A, except if FACT = 'F' and EQUED =
  'Y', then A must contain the equilibrated matrix
  diag(S)*A*diag(S).  If UPLO = 'U', the leading N-by-N upper
  triangular part of A contains the upper triangular part of the
  matrix A, and the strictly lower triangular part of A is not
  referenced.  If UPLO = 'L', the leading N-by-N lower triangular
  part of A contains the lower triangular part of the matrix A, and
  the strictly upper triangular part of A is not referenced.  A is
  not modified if FACT = 'F' or 'N', or if FACT = 'E' and EQUED =
  'N' on exit.

  On exit, if FACT = 'E' and EQUED = 'Y', A is overwritten by
  diag(S)*A*diag(S).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is DOUBLE PRECISION array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the triangular factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T, in the same storage
  format as A.  If EQUED .ne. 'N', then AF is the factored
  form of the equilibrated matrix diag(S)*A*diag(S).

  If FACT = 'N', then AF is an output argument and on exit
  returns the triangular factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T of the original
  matrix A.

  If FACT = 'E', then AF is an output argument and on exit
  returns the triangular factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T of the equilibrated
  matrix A (see the description of A for the form of the
  equilibrated matrix).

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).
```

```
EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'Y':  Both row and column equilibration, i.e., A has been
          replaced by diag(S) * A * diag(S).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

S
  S is DOUBLE PRECISION array, dimension (N)
  The row scale factors for A.  If EQUED = 'Y', A is multiplied on
  the left and right by diag(S).  S is an input argument if FACT =
  'F'; otherwise, S is an output argument.  If FACT = 'F' and EQUED
  = 'Y', each element of S must be positive.  If S is output, each
  element of S is a power of the radix. If S is input, each element
  of S should be a power of the radix to ensure a reliable solution
  and error estimates. Scaling by powers of the radix does not cause
  rounding errors unless the result underflows or overflows.
  Rounding errors during scaling lead to refining with a matrix that
  is not equivalent to the input matrix, producing error estimates
  that may not be reliable.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if EQUED = 'Y', B is overwritten by diag(S)*B;

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  If INFO = 0, the N-by-NRHS solution matrix X to the original
  system of equations.  Note that A and B are modified on exit if
  EQUED .ne. 'N', and the solution to the equilibrated system is
  inv(diag(S))*X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
```

```
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

RPVGRW
  RPVGRW is DOUBLE PRECISION
  Reciprocal pivot growth.  On exit, this contains the reciprocal
  pivot growth factor norm(A)/norm(U). The "max absolute element"
  norm is used.  If this is much less than 1, then the stability of
  the LU factorization of the (equilibrated) matrix A could be poor.
  This also means that the solution X, estimated condition numbers,
  and error bounds could be unreliable. If factorization fails with
  0<INFO<=N, then this contains the reciprocal pivot growth factor
  for the leading INFO columns of A.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
          max_j (abs(XTRUE(j,i) - X(j,i)))
           ------------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
```

```
      reciprocal condition number is less than the threshold
      sqrt(n) * dlamch('Epsilon').

   err = 2 "Guaranteed" error bound: The estimated forward error,
      almost certainly within a factor of 10 of the true error
      so long as the next entry is greater than the threshold
      sqrt(n) * dlamch('Epsilon'). This error bound should only
      be trusted if the previous boolean is true.

   err = 3  Reciprocal condition number: Estimated normwise
      reciprocal condition number.  Compared with the threshold
      sqrt(n) * dlamch('Epsilon') to determine if the error
      estimate is "guaranteed". These reciprocal condition
      numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
      appropriately scaled matrix Z.
      Let Z = S*A, where S scales each row by a power of the
      radix so all absolute row sums of Z are approximately 1.

 ERR_BNDS_COMP
   ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
   For each right-hand side, this array contains information about
   various error bounds and condition numbers corresponding to the
   componentwise relative error, which is defined as follows:

   Componentwise relative error in the ith solution vector:
                   abs(XTRUE(j,i) - X(j,i))
             max_j ----------------------
                         abs(X(j,i))

   The array is indexed by the right-hand side i (on which the
   componentwise relative error depends), and the type of error
   information as described below. There currently are up to three
   pieces of information returned for each right-hand side. If
   componentwise accuracy is not requested (PARAMS(3) = 0.0), then
   ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
   the first (:,N_ERR_BNDS) entries are returned.

   The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
   right-hand side.

   The second index in ERR_BNDS_COMP(:,err) contains the following
   three fields:

   err = 1 "Trust/don't trust" boolean. Trust the answer if the
      reciprocal condition number is less than the threshold
      sqrt(n) * dlamch('Epsilon').

   err = 2 "Guaranteed" error bound: The estimated forward error,
      almost certainly within a factor of 10 of the true error
      so long as the next entry is greater than the threshold
```

```
     sqrt(n) * dlamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * dlamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is DOUBLE PRECISION array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0D+0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the extra-precise refinement algorithm.
  (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is DOUBLE PRECISION array, dimension (4*N)
```

```
 IWORK
   IWORK is INTEGER array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  Successful exit. The solution to every right-hand side is
         guaranteed.
   < 0:  If INFO = -i, the i-th argument had an illegal value
   > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
       has been completed, but the factor U is exactly singular, so
       the solution and error bounds could not be computed. RCOND = 0
       is returned.
   = N+J: The solution corresponding to the Jth right-hand side is
       not guaranteed. The solutions corresponding to other right-
       hand sides K with K > J may not be guaranteed as well, but
       only the first such right-hand side is reported. If a small
       componentwise error is not requested (PARAMS(3) = 0.0) then
       the Jth right-hand side is the first with a normwise error
       bound that is not guaranteed (the smallest J such
       that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
       the Jth right-hand side is the first with either a normwise or
       componentwise error bound that is not guaranteed (the smallest
       J such that either ERR_BNDS_NORM(J,1) = 0.0 or
       ERR_BNDS_COMP(J,1) = 0.0). See the definition of
       ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
       about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

———-

66.3.3 fortran code

— dposvxx.f —

```
*  ==================================================================
      SUBROUTINE DPOSVXX( FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, EQUED,
     $                    S, B, LDB, X, LDX, RCOND, RPVGRW, BERR,
     $                    N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP,
     $                    NPARAMS, PARAMS, WORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, UPLO
```

```
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      DOUBLE PRECISION   RCOND, RPVGRW
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX, * ), WORK( * )
      DOUBLE PRECISION   S( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
*     ..
*     .. Local Scalars ..
      LOGICAL            EQUIL, NOFACT, RCEQU
      INTEGER            INFEQU, J
      DOUBLE PRECISION   AMAX, BIGNUM, SMIN, SMAX,
     $                   SCOND, SMLNUM
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, DLAMCH, DLA_PORPVGRW
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, DLA_PORPVGRW
*     ..
*     .. External Subroutines ..
      EXTERNAL           DPOEQUB, DPOTRF, DPOTRS, DLACPY, DLAQSY,
     $                   XERBLA, DLASCL2, DPORFSX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
```

```
      SMLNUM = DLAMCH( 'Safe minimum' )
      BIGNUM = ONE / SMLNUM
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         RCEQU = .FALSE.
      ELSE
         RCEQU = LSAME( EQUED, 'Y' )
      ENDIF
*
*     Default is failure.  If an input parameter is wrong or
*     factorization fails, make everything look horrible.  Only the
*     pivot growth is set here, the rest is initialized in DPORFSX.
*
      RPVGRW = ZERO
*
*     Test the input parameters.  PARAMS is not tested until DPORFSX.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.
     $     LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.LSAME( UPLO, 'U' ) .AND.
     $         .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $        ( RCEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -9
      ELSE
         IF ( RCEQU ) THEN
            SMIN = BIGNUM
            SMAX = ZERO
            DO 10 J = 1, N
               SMIN = MIN( SMIN, S( J ) )
               SMAX = MAX( SMAX, S( J ) )
 10         CONTINUE
            IF( SMIN.LE.ZERO ) THEN
               INFO = -10
            ELSE IF( N.GT.0 ) THEN
               SCOND = MAX( SMIN, SMLNUM ) / MIN( SMAX, BIGNUM )
            ELSE
               SCOND = ONE
            END IF
         END IF
```

```
         IF( INFO.EQ.0 ) THEN
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -12
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -14
            END IF
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DPOSVXX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*     Compute row and column scalings to equilibrate the matrix A.
*
         CALL DPOEQUB( N, A, LDA, S, SCOND, AMAX, INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*     Equilibrate the matrix.
*
            CALL DLAQSY( UPLO, N, A, LDA, S, SCOND, AMAX, EQUED )
            RCEQU = LSAME( EQUED, 'Y' )
         END IF
      END IF
*
*     Scale the right-hand side.
*
      IF( RCEQU ) CALL DLASCL2( N, NRHS, S, B, LDB )
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the Cholesky factorization of A.
*
         CALL DLACPY( UPLO, N, N, A, LDA, AF, LDAF )
         CALL DPOTRF( UPLO, N, AF, LDAF, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.NE.0 ) THEN
*
*           Pivot in column INFO is exactly 0
*           Compute the reciprocal pivot growth factor of the
*           leading rank-deficient INFO columns of A.
*
            RPVGRW = DLA_PORPVGRW( UPLO, INFO, A, LDA, AF, LDAF, WORK )
            RETURN
         ENDIF
```

```
      END IF
*
*     Compute the reciprocal growth factor RPVGRW.
*
      RPVGRW = DLA_PORPVGRW( UPLO, N, A, LDA, AF, LDAF, WORK )
*
*     Compute the solution matrix X.
*
      CALL DLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL DPOTRS( UPLO, N, NRHS, AF, LDAF, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL DPORFSX( UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF,
     $     S, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS, ERR_BNDS_NORM,
     $     ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, IWORK, INFO )

*
*     Scale solutions.
*
      IF ( RCEQU ) THEN
         CALL DLASCL2 ( N, NRHS, S, X, LDX )
      END IF
*
      RETURN
*
*     End of DPOSVXX
*
      END
```

———-

66.3.4 lisp code

— **dposvxx.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dposvxx
         (fact uplo n nrhs a lda af ldaf equed s b ldb$ x ldx rcond rpvgrw berr
          n_err_bnds err_bnds_norm err_bnds_comp nparams params work iwork
          info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (double-float) rpvgrw rcond)
             (type (array double-float (*)) work params err_bnds_comp
```

```
            err_bnds_norm berr x b s af a)
           (type (f2cl-lib:integer4) info nparams n_err_bnds ldx ldb$ ldaf
            lda nrhs n)
           (type (string *) equed uplo fact))
  (f2cl-lib:with-multi-array-data
      ((fact character fact-%data% fact-%offset%)
       (uplo character uplo-%data% uplo-%offset%)
       (equed character equed-%data% equed-%offset%)
       (a double-float a-%data% a-%offset%)
       (af double-float af-%data% af-%offset%)
       (s double-float s-%data% s-%offset%)
       (b double-float b-%data% b-%offset%)
       (x double-float x-%data% x-%offset%)
       (berr double-float berr-%data% berr-%offset%)
       (err_bnds_norm double-float err_bnds_norm-%data%
        err_bnds_norm-%offset%)
       (err_bnds_comp double-float err_bnds_comp-%data%
        err_bnds_comp-%offset%)
       (params double-float params-%data% params-%offset%)
       (work double-float work-%data% work-%offset%)
       (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
    (prog ((amax 0.0d0) (bignum 0.0d0) (smin 0.0d0) (smax 0.0d0)
           (scond 0.0d0) (smlnum 0.0d0) (infequ 0) (j 0) (equil nil)
           (nofact nil) (rcequ nil))
      (declare (type (double-float) amax bignum smin smax scond smlnum)
               (type (f2cl-lib:integer4) infequ j)
               (type f2cl-lib:logical equil nofact rcequ))
      (setf info 0)
      (setf nofact
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame fact "N")
                (declare (ignore var-1))
                (when var-0 (setf fact var-0))
                ret-val))
      (setf equil
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame fact "E")
                (declare (ignore var-1))
                (when var-0 (setf fact var-0))
                ret-val))
      (setf smlnum (dlamch "Safe minimum"))
      (setf bignum (/ one smlnum))
      (cond
       ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
        (setf rcequ f2cl-lib:%false%))
       (t
        (setf rcequ
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame equed "Y")
                  (declare (ignore var-1))
```

```
            (when var-0 (setf equed var-0))
            ret-val))))
(setf rpvgrw zero)
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame fact "F")
     (declare (ignore var-1))
     (when var-0 (setf fact var-0))
     ret-val)
   (not
    (or rcequ
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame equed "N")
          (declare (ignore var-1))
          (when var-0 (setf equed var-0))
          ret-val))))
  (setf info -9))
 (t
  (cond
   (rcequ (setf smin bignum) (setf smax zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
```

```
      (tagbody
        (setf smin
                (min smin
                     (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
        (setf smax
                (max smax
                     (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
       label10))
    (cond ((<= smin zero) (setf info -10))
          ((> n 0)
           (setf scond (/ (max smin smlnum) (min smax bignum))))
          (t (setf scond one)))))
  (cond
   ((= info 0)
    (cond
     ((< ldb$
         (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -12))
     ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -14)))))))
(cond
 ((/= info 0) (xerbla "DPOSVXX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (equil
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dpoequb n a lda s scond amax infequ)
    (declare (ignore var-0 var-1 var-2 var-3))
    (setf scond var-4)
    (setf amax var-5)
    (setf infequ var-6))
  (cond
   ((= infequ 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (dlaqsy uplo n a lda s scond amax equed)
      (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
      (setf uplo var-0)
      (setf equed var-7))
    (setf rcequ
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame equed "Y")
              (declare (ignore var-1))
              (when var-0 (setf equed var-0))
              ret-val))))))
(if rcequ
    (dlascl2 n nrhs s b ldb$))
(cond
 ((or nofact equil)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
      (dlacpy uplo n n a lda af ldaf)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
    (setf uplo var-0))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dpotrf uplo n af ldaf info)
    (declare (ignore var-2))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf ldaf var-3))
    (when var-4 (setf info var-4)))
  (cond
   ((/= info 0)
    (setf rpvgrw
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (dla_porpvgrw uplo info a lda af ldaf work)
              (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
              (setf uplo var-0)
              ret-val))
    (go end_label)))))
(setf rpvgrw
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dla_porpvgrw uplo n a lda af ldaf work)
          (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
          (setf uplo var-0)
          ret-val))
(dlacpy "Full" n nrhs b ldb$ x ldx)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dpotrs uplo n nrhs af ldaf x ldx info)
  (declare (ignore var-3 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldaf var-4))
  (when var-6 (setf ldx var-6))
  (when var-7 (setf info var-7)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19
     var-20 var-21 var-22)
    (dporfsx uplo equed n nrhs a lda af ldaf s b ldb$ x ldx rcond berr
     n_err_bnds err_bnds_norm err_bnds_comp nparams params work iwork
     info)
  (declare
   (ignore var-4 var-6 var-8 var-9 var-11 var-14 var-16 var-17 var-19
    var-20 var-21))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf equed var-1))
  (when var-2 (setf n var-2))
```

```
          (when var-3 (setf nrhs var-3))
          (when var-5 (setf lda var-5))
          (when var-7 (setf ldaf var-7))
          (when var-10 (setf ldb$ var-10))
          (when var-12 (setf ldx var-12))
          (when var-13 (setf rcond var-13))
          (when var-15 (setf n_err_bnds var-15))
          (when var-18 (setf nparams var-18))
          (when var-22 (setf info var-22)))
        (cond (rcequ (dlascl2 n nrhs s x ldx)))
        (go end_label)
       end_label
        (return
         (values fact uplo n nrhs nil lda nil ldaf equed nil nil ldb$ nil ldx
                 rcond rpvgrw nil n_err_bnds nil nil nparams nil nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dposvxx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (string) (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::fact
                                              fortran-to-lisp::uplo
                                              fortran-to-lisp::n
```

```
                                                  fortran-to-lisp::nrhs nil
                                                  fortran-to-lisp::lda nil
                                                  fortran-to-lisp::ldaf
                                                  fortran-to-lisp::equed nil nil
                                                  fortran-to-lisp::ldb$ nil
                                                  fortran-to-lisp::ldx
                                                  fortran-to-lisp::rcond
                                                  fortran-to-lisp::rpvgrw nil
                                                  fortran-to-lisp::n_err_bnds nil
                                                  nil fortran-to-lisp::nparams nil
                                                  nil nil fortran-to-lisp::info)
                                                :calls
                                                '(fortran-to-lisp::dla_porpvgrw
                                                  fortran-to-lisp::dlacpy
                                                  fortran-to-lisp::dlascl2
                                                  fortran-to-lisp::dlaqsy
                                                  fortran-to-lisp::dpoequb))))
```

———-

66.4 dsposv

66.4.1 Axiom unit tests

— **dsposv.input** —

```
)set break resume
)sys rm -f dsposv.output
)spool dsposv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

66.4.2 Axiom help page

— **dsposv.help** —

```
==================================================================
```

```
dsposv examples
===================================================================

===================================================================
Man Page Details
===================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dsposv (UPLO, N, NRHS, A, LDA, B, LDB, X, LDX, WORK, SWORK,
           ITER, INFO)
            DSPOSV computes the solution to system of linear equations A * X =
           B for PO matrices

Function/Subroutine Documentation
   subroutine dsposv (character UPLO, integer N, integer NRHS, double
       precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldb, * ) B, integer LDB, double precision, dimension( ldx, *
       ) X, integer LDX, double precision, dimension( n, * ) WORK, real,
       dimension( * ) SWORK, integer ITER, integer INFO)
        DSPOSV computes the solution to system of linear equations A * X = B
       for PO matrices

Purpose:
   DSPOSV computes the solution to a real system of linear equations
   A * X = B,
   where A is an N-by-N symmetric positive definite matrix and X and B
   are N-by-NRHS matrices.

   DSPOSV first attempts to factorize the matrix in SINGLE PRECISION
   and use this factorization within an iterative refinement procedure
   to produce a solution with DOUBLE PRECISION normwise backward error
   quality (see below). If the approach fails the method switches to a
   DOUBLE PRECISION factorization and solve.

   The iterative refinement is not going to be a winning strategy if
   the ratio SINGLE PRECISION performance over DOUBLE PRECISION
   performance is too small. A reasonable strategy should take the
   number of right-hand sides and the size of the matrix into account.
   This might be done with a call to ILAENV in the future. Up to now, we
   always try iterative refinement.

   The iterative refinement process is stopped if
      ITER > ITERMAX
   or for all the RHS we have:
      RNRM < SQRT(N)*XNRM*ANRM*EPS*BWDMAX
   where
      o ITER is the number of the current iteration in the iterative
        refinement process
      o RNRM is the infinity-norm of the residual
```

```
      o XNRM is the infinity-norm of the solution
      o ANRM is the infinity-operator-norm of the matrix A
      o EPS is the machine epsilon returned by DLAMCH('Epsilon')
   The value ITERMAX and BWDMAX are fixed to 30 and 1.0D+00 respectively.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is DOUBLE PRECISION array,
   dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   N-by-N upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading N-by-N lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.
   On exit, if iterative refinement has been successfully used
   (INFO.EQ.0 and ITER.GE.0, see description below), then A is
   unchanged, if double precision factorization has been used
   (INFO.EQ.0 and ITER.LT.0, see description below), then the
   array A contains the factor U or L from the Cholesky
   factorization A = U**T*U or A = L*L**T.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 B
   B is DOUBLE PRECISION array, dimension (LDB,NRHS)
   The N-by-NRHS right hand side matrix B.

 LDB
   LDB is INTEGER
   The leading dimension of the array B.  LDB >= max(1,N).
```

```
X
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  If INFO = 0, the N-by-NRHS solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

WORK
  WORK is DOUBLE PRECISION array, dimension (N,NRHS)
  This array is used to hold the residual vectors.

SWORK
  SWORK is REAL array, dimension (N*(N+NRHS))
  This array is used to use the single precision matrix and the
  right-hand sides or solutions in single precision.

ITER
  ITER is INTEGER
  < 0: iterative refinement has failed, double precision
  factorization has been performed
  -1 : the routine fell back to full precision for
       implementation- or machine-specific reasons
  -2 : narrowing the precision induced an overflow,
       the routine fell back to full precision
  -3 : failure of SPOTRF
  -31: stop the iterative refinement after the 30th iterations
  > 0: iterative refinement has been sucessfully used.
       Returns the number of iterations

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, the leading minor of order i of (DOUBLE
        PRECISION) A is not positive definite, so the
        factorization could not be completed, and the solution
        has not been computed.
```

———-

66.4.3 fortran code

— dsposv.f —

```
*  ====================================================================
      SUBROUTINE DSPOSV( UPLO, N, NRHS, A, LDA, B, LDB, X, LDX, WORK,
```

```
     $                   SWORK, ITER, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, ITER, LDA, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      REAL               SWORK( * )
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * ), WORK( N, * ),
     $                   X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      LOGICAL            DOITREF
      PARAMETER          ( DOITREF = .TRUE. )
*
      INTEGER            ITERMAX
      PARAMETER          ( ITERMAX = 30 )
*
      DOUBLE PRECISION   BWDMAX
      PARAMETER          ( BWDMAX = 1.0E+00 )
*
      DOUBLE PRECISION   NEGONE, ONE
      PARAMETER          ( NEGONE = -1.0D+0, ONE = 1.0D+0 )
*
*     .. Local Scalars ..
      INTEGER            I, IITER, PTSA, PTSX
      DOUBLE PRECISION   ANRM, CTE, EPS, RNRM, XNRM
*
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DSYMM, DLACPY, DLAT2S, DLAG2S, SLAG2D,
     $                   SPOTRF, SPOTRS, XERBLA
*     ..
*     .. External Functions ..
      INTEGER            IDAMAX
      DOUBLE PRECISION   DLAMCH, DLANSY
      LOGICAL            LSAME
      EXTERNAL           IDAMAX, DLAMCH, DLANSY, LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, MAX, SQRT
*     ..
*     .. Executable Statements ..
```

```
*
      INFO = 0
      ITER = 0
*
*     Test the input parameters.
*
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DSPOSV', -INFO )
         RETURN
      END IF
*
*     Quick return if (N.EQ.0).
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Skip single precision iterative refinement if a priori slower
*     than double precision factorization.
*
      IF( .NOT.DOITREF ) THEN
         ITER = -1
         GO TO 40
      END IF
*
*     Compute some constants.
*
      ANRM = DLANSY( 'I', UPLO, N, A, LDA, WORK )
      EPS = DLAMCH( 'Epsilon' )
      CTE = ANRM*EPS*SQRT( DBLE( N ) )*BWDMAX
*
*     Set the indices PTSA, PTSX for referencing SA and SX in SWORK.
*
      PTSA = 1
      PTSX = PTSA + N*N
*
*     Convert B from double precision to single precision and store the
*     result in SX.
```

```
*
      CALL DLAG2S( N, NRHS, B, LDB, SWORK( PTSX ), N, INFO )
*
      IF( INFO.NE.0 ) THEN
         ITER = -2
         GO TO 40
      END IF
*
*     Convert A from double precision to single precision and store the
*     result in SA.
*
      CALL DLAT2S( UPLO, N, A, LDA, SWORK( PTSA ), N, INFO )
*
      IF( INFO.NE.0 ) THEN
         ITER = -2
         GO TO 40
      END IF
*
*     Compute the Cholesky factorization of SA.
*
      CALL SPOTRF( UPLO, N, SWORK( PTSA ), N, INFO )
*
      IF( INFO.NE.0 ) THEN
         ITER = -3
         GO TO 40
      END IF
*
*     Solve the system SA*SX = SB.
*
      CALL SPOTRS( UPLO, N, NRHS, SWORK( PTSA ), N, SWORK( PTSX ), N,
     $             INFO )
*
*     Convert SX back to double precision
*
      CALL SLAG2D( N, NRHS, SWORK( PTSX ), N, X, LDX, INFO )
*
*     Compute R = B - AX (R is WORK).
*
      CALL DLACPY( 'All', N, NRHS, B, LDB, WORK, N )
*
      CALL DSYMM( 'Left', UPLO, N, NRHS, NEGONE, A, LDA, X, LDX, ONE,
     $            WORK, N )
*
*     Check whether the NRHS normwise backward errors satisfy the
*     stopping criterion. If yes, set ITER=0 and return.
*
      DO I = 1, NRHS
         XNRM = ABS( X( IDAMAX( N, X( 1, I ), 1 ), I ) )
         RNRM = ABS( WORK( IDAMAX( N, WORK( 1, I ), 1 ), I ) )
         IF( RNRM.GT.XNRM*CTE )
```

```
     $      GO TO 10
      END DO
*
*     If we are here, the NRHS normwise backward errors satisfy the
*     stopping criterion. We are good to exit.
*
      ITER = 0
      RETURN
*
   10 CONTINUE
*
      DO 30 IITER = 1, ITERMAX
*
*        Convert R (in WORK) from double precision to single precision
*        and store the result in SX.
*
         CALL DLAG2S( N, NRHS, WORK, N, SWORK( PTSX ), N, INFO )
*
         IF( INFO.NE.0 ) THEN
            ITER = -2
            GO TO 40
         END IF
*
*        Solve the system SA*SX = SR.
*
         CALL SPOTRS( UPLO, N, NRHS, SWORK( PTSA ), N, SWORK( PTSX ), N,
     $                INFO )
*
*        Convert SX back to double precision and update the current
*        iterate.
*
         CALL SLAG2D( N, NRHS, SWORK( PTSX ), N, WORK, N, INFO )
*
         DO I = 1, NRHS
            CALL DAXPY( N, ONE, WORK( 1, I ), 1, X( 1, I ), 1 )
         END DO
*
*        Compute R = B - AX (R is WORK).
*
         CALL DLACPY( 'All', N, NRHS, B, LDB, WORK, N )
*
         CALL DSYMM( 'L', UPLO, N, NRHS, NEGONE, A, LDA, X, LDX, ONE,
     $               WORK, N )
*
*        Check whether the NRHS normwise backward errors satisfy the
*        stopping criterion. If yes, set ITER=IITER>0 and return.
*
         DO I = 1, NRHS
            XNRM = ABS( X( IDAMAX( N, X( 1, I ), 1 ), I ) )
            RNRM = ABS( WORK( IDAMAX( N, WORK( 1, I ), 1 ), I ) )
```

```
            IF( RNRM.GT.XNRM*CTE )
     $         GO TO 20
         END DO
*
*        If we are here, the NRHS normwise backward errors satisfy the
*        stopping criterion, we are good to exit.
*
         ITER = IITER
*
         RETURN
*
   20    CONTINUE
*
   30 CONTINUE
*
*     If we are at this place of the code, this is because we have
*     performed ITER=ITERMAX iterations and never satisified the
*     stopping criterion, set up the ITER flag accordingly and follow
*     up on double precision routine.
*
      ITER = -ITERMAX - 1
*
   40 CONTINUE
*
*     Single-precision iterative refinement failed to converge to a
*     satisfactory solution, so we resort to double precision.
*
      CALL DPOTRF( UPLO, N, A, LDA, INFO )
*
      IF( INFO.NE.0 )
     $   RETURN
*
      CALL DLACPY( 'All', N, NRHS, B, LDB, X, LDX )
      CALL DPOTRS( UPLO, N, NRHS, A, LDA, X, LDX, INFO )
*
      RETURN
*
*     End of DSPOSV.
*
      END
```

———-

66.4.4 lisp code

— dsposv.lisp —

```
(let* ((doitref f2cl-lib:%true%)
```

```
      (itermax 30)
      (bwdmax (coerce 1.0 'double-float))
      (negone (- 1.0d0))
      (one 1.0d0))
 (declare (type f2cl-lib:logical doitref)
          (type (f2cl-lib:integer4 30 30) itermax)
          (type (double-float) bwdmax)
          (type (double-float) negone)
          (type (double-float 1.0d0 1.0d0) one)
          (ignorable doitref itermax bwdmax negone one))
 (defun dsposv (uplo n nrhs a lda b ldb$ x ldx work swork iter info)
   (declare (type (array single-float (*)) swork)
            (type (array double-float (*)) work x b a)
            (type (f2cl-lib:integer4) info iter ldx ldb$ lda nrhs n)
            (type (string *) uplo))
   (f2cl-lib:with-multi-array-data
       ((uplo character uplo-%data% uplo-%offset%)
        (a double-float a-%data% a-%offset%)
        (b double-float b-%data% b-%offset%)
        (x double-float x-%data% x-%offset%)
        (work double-float work-%data% work-%offset%)
        (swork single-float swork-%data% swork-%offset%))
     (prog ((anrm 0.0d0) (cte 0.0d0) (eps 0.0d0) (rnrm 0.0d0) (xnrm 0.0d0)
            (i 0) (iiter 0) (ptsa 0) (ptsx 0))
       (declare (type (double-float) anrm cte eps rnrm xnrm)
                (type (f2cl-lib:integer4) i iiter ptsa ptsx))
       (setf info 0)
       (setf iter 0)
       (cond
        ((and
          (not
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame uplo "U")
             (declare (ignore var-1))
             (when var-0 (setf uplo var-0))
             ret-val))
          (not
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame uplo "L")
             (declare (ignore var-1))
             (when var-0 (setf uplo var-0))
             ret-val)))
         (setf info -1))
        ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
        ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -5))
        ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -7))
        ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -9)))
```

```
(cond
 ((/= info 0) (xerbla "DSPOSV" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(cond ((not doitref) (setf iter -1) (go label40)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlansy "I" uplo n a lda work)
          (declare (ignore var-0 var-3 var-4 var-5))
          (setf uplo var-1)
          (setf n var-2)
          ret-val))
(setf eps (dlamch "Epsilon"))
(setf cte (* anrm eps (f2cl-lib:fsqrt (f2cl-lib:dble n)) bwdmax))
(setf ptsa 1)
(setf ptsx (f2cl-lib:int-add ptsa (f2cl-lib:int-mul n n)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlag2s n nrhs b ldb$
     (f2cl-lib:array-slice swork-%data% single-float (ptsx) ((1 *))
                           swork-%offset%)
     n info)
  (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5))
  (setf info var-6))
(cond ((/= info 0) (setf iter -2) (go label40)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlat2s uplo n a lda
     (f2cl-lib:array-slice swork-%data% single-float (ptsa) ((1 *))
                           swork-%offset%)
     n info)
  (declare (ignore var-1 var-2 var-3 var-4 var-5))
  (setf uplo var-0)
  (setf info var-6))
(cond ((/= info 0) (setf iter -2) (go label40)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (spotrf uplo n
     (f2cl-lib:array-slice swork-%data% single-float (ptsa) ((1 *))
                           swork-%offset%)
     n info)
  (declare (ignore var-2))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf n var-3))
  (when var-4 (setf info var-4)))
(cond ((/= info 0) (setf iter -3) (go label40)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (spotrs uplo n nrhs
     (f2cl-lib:array-slice swork-%data% single-float (ptsa) ((1 *))
                           swork-%offset%)
```

```
      n
      (f2cl-lib:array-slice swork-%data% single-float (ptsx) ((1 *))
                            swork-%offset%)
      n info)
  (declare (ignore var-3 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf n var-6))
  (when var-7 (setf info var-7)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slag2d n nrhs
     (f2cl-lib:array-slice swork-%data% single-float (ptsx) ((1 *))
                           swork-%offset%)
     n x ldx info)
  (declare (ignore var-2 var-4))
  (when var-0 (setf n var-0))
  (when var-1 (setf nrhs var-1))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldx var-5))
  (when var-6 (setf info var-6)))
(dlacpy "All" n nrhs b ldb$ work n)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11)
    (dsymm "Left" uplo n nrhs negone a lda x ldx one work n)
  (declare (ignore var-0 var-5 var-7 var-10))
  (when var-1 (setf uplo var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf nrhs var-3))
  (when var-4 (setf negone var-4))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldx var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf n var-11)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nrhs) nil)
  (tagbody
    (setf xnrm
            (abs
             (f2cl-lib:fref x-%data%
                            ((multiple-value-bind
                                 (ret-val var-0 var-1 var-2)
                                 (idamax n
                                  (f2cl-lib:array-slice x double-float
                                                        (1 i)
                                                        ((1 ldx)
                                                         (1 *)))
                                  1)
```

```
                                  (declare (ignore var-1 var-2))
                                  (when var-0 (setf n var-0))
                                  ret-val)
                                i)
                               ((1 ldx) (1 *)) x-%offset%)))
        (setf rnrm
                (abs
                 (f2cl-lib:fref work-%data%
                               ((multiple-value-bind
                                    (ret-val var-0 var-1 var-2)
                                    (idamax n
                                     (f2cl-lib:array-slice work
                                                           double-float
                                                           (1 i)
                                                           ((1 n) (1 *)))
                                     1)
                                  (declare (ignore var-1 var-2))
                                  (when var-0 (setf n var-0))
                                  ret-val)
                                i)
                               ((1 n) (1 *)) work-%offset%)))
        (if (> rnrm (* xnrm cte))
            (go label10))
       label100000))
     (setf iter 0)
     (go end_label)
    label10
     (f2cl-lib:fdo (iiter 1 (f2cl-lib:int-add iiter 1))
                   ((> iiter itermax) nil)
       (tagbody
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
             (dlag2s n nrhs work n
              (f2cl-lib:array-slice swork-%data% single-float (ptsx) ((1 *))
                                    swork-%offset%)
              n info)
           (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5))
           (setf info var-6))
         (cond ((/= info 0) (setf iter -2) (go label40)))
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
             (spotrs uplo n nrhs
              (f2cl-lib:array-slice swork-%data% single-float (ptsa) ((1 *))
                                    swork-%offset%)
              n
              (f2cl-lib:array-slice swork-%data% single-float (ptsx) ((1 *))
                                    swork-%offset%)
              n info)
           (declare (ignore var-3 var-5))
           (when var-0 (setf uplo var-0))
           (when var-1 (setf n var-1))
```

```
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf n var-6))
  (when var-7 (setf info var-7)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slag2d n nrhs
     (f2cl-lib:array-slice swork-%data% single-float (ptsx) ((1 *))
                           swork-%offset%)
     n work n info)
  (declare (ignore var-2 var-4))
  (when var-0 (setf n var-0))
  (when var-1 (setf nrhs var-1))
  (when var-3 (setf n var-3))
  (when var-5 (setf n var-5))
  (when var-6 (setf info var-6)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nrhs) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (daxpy n one
         (f2cl-lib:array-slice work-%data% double-float (1 i)
                               ((1 n) (1 *)) work-%offset%)
         1
         (f2cl-lib:array-slice x-%data% double-float (1 i)
                               ((1 ldx) (1 *)) x-%offset%)
         1)
      (declare (ignore var-2 var-3 var-4 var-5))
      (when var-0 (setf n var-0))
      (when var-1 (setf one var-1)))
   label100001))
(dlacpy "All" n nrhs b ldb$ work n)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (dsymm "L" uplo n nrhs negone a lda x ldx one work n)
  (declare (ignore var-0 var-5 var-7 var-10))
  (when var-1 (setf uplo var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf nrhs var-3))
  (when var-4 (setf negone var-4))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldx var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf n var-11)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nrhs) nil)
  (tagbody
    (setf xnrm
            (abs
             (f2cl-lib:fref x-%data%
```

```
                                    ((multiple-value-bind
                                         (ret-val var-0 var-1 var-2)
                                         (idamax n
                                          (f2cl-lib:array-slice x
                                                                double-float
                                                                (1 i)
                                                                ((1 ldx)
                                                                 (1 *)))
                                          1)
                                       (declare (ignore var-1 var-2))
                                       (when var-0 (setf n var-0))
                                       ret-val)
                                     i)
                                    ((1 ldx) (1 *)) x-%offset%)))
            (setf rnrm
                    (abs
                     (f2cl-lib:fref work-%data%
                                    ((multiple-value-bind
                                         (ret-val var-0 var-1 var-2)
                                         (idamax n
                                          (f2cl-lib:array-slice work
                                                                double-float
                                                                (1 i)
                                                                ((1 n)
                                                                 (1 *)))
                                          1)
                                       (declare (ignore var-1 var-2))
                                       (when var-0 (setf n var-0))
                                       ret-val)
                                     i)
                                    ((1 n) (1 *)) work-%offset%)))
            (if (> rnrm (* xnrm cte))
                (go label20))
           label100002))
        (setf iter iiter)
        (go end_label)
       label20
       label30))
    (setf iter (f2cl-lib:int-sub -1 itermax))
   label40
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dpotrf uplo n a lda info)
      (declare (ignore var-2))
      (setf uplo var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf info var-4))
    (if (/= info 0)
        (go end_label))
    (dlacpy "All" n nrhs b ldb$ x ldx)
```

```
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (dpotrs uplo n nrhs a lda x ldx info)
          (declare (ignore var-3 var-5))
          (setf uplo var-0)
          (setf n var-1)
          (setf nrhs var-2)
          (setf lda var-4)
          (setf ldx var-6)
          (setf info var-7))
        (go end_label)
       end_label
        (return
         (values uplo n nrhs nil lda nil nil nil ldx nil nil iter info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dsposv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil nil nil
                                              fortran-to-lisp::ldx nil nil
                                              fortran-to-lisp::iter
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::dpotrs
                                              fortran-to-lisp::dpotrf
                                              fortran-to-lisp::dlacpy
                                              fortran-to-lisp::dlat2s
                                              fortran-to-lisp::dlag2s
                                              fortran-to-lisp::dlansy))))
```

Chapter 67

LAPACK - Positive Definite Matrix, Linear Solve, Real

67.1 sposv

67.1.1 Axiom unit tests

— sposv.input —

```
)set break resume
)sys rm -f sposv.output
)spool sposv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

67.1.2 Axiom help page

— sposv.help —

```
==========================================================================
sposv examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sposv (UPLO, N, NRHS, A, LDA, B, LDB, INFO)
            SPOSV computes the solution to system of linear equations A * X =
           B for PO matrices

Function/Subroutine Documentation
   subroutine sposv (character UPLO, integer N, integer NRHS, real, dimension(
       lda, * ) A, integer LDA, real, dimension( ldb, * ) B, integer LDB,
       integer INFO)
        SPOSV computes the solution to system of linear equations A * X = B
       for PO matrices

Purpose:
   SPOSV computes the solution to a real system of linear equations
      A * X = B,
   where A is an N-by-N symmetric positive definite matrix and X and B
   are N-by-NRHS matrices.

   The Cholesky decomposition is used to factor A as
      A = U**T* U,  if UPLO = 'U', or
      A = L * L**T,  if UPLO = 'L',
   where U is an upper triangular matrix and L is a lower triangular
   matrix.  The factored form of A is then used to solve the system of
   equations A * X = B.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
```

```
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if INFO = 0, the factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

B
  B is REAL array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, the leading minor of order i of A is not
        positive definite, so the factorization could not be
        completed, and the solution has not been computed.
```

———-

67.1.3 fortran code

— sposv.f —

```
*  ===================================================================
      SUBROUTINE SPOSV( UPLO, N, NRHS, A, LDA, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
```

```
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SPOTRF, SPOTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SPOSV ', -INFO )
         RETURN
      END IF
*
*     Compute the Cholesky factorization A = U**T*U or A = L*L**T.
*
      CALL SPOTRF( UPLO, N, A, LDA, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
         CALL SPOTRS( UPLO, N, NRHS, A, LDA, B, LDB, INFO )
*
      END IF
      RETURN
```

```
*
*     End of SPOSV
*
      END
```

———-

67.1.4 lisp code

— **sposv.lisp** —

```
(defun sposv (uplo n nrhs a lda b ldb$ info)
  (declare (type (array single-float (*)) b a)
           (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a single-float a-%data% a-%offset%)
       (b single-float b-%data% b-%offset%))
    (prog ()
      (declare)
      (setf info 0)
      (cond
       ((and
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -5))
       ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -7)))
      (cond
       ((/= info 0) (xerbla "SPOSV " (f2cl-lib:int-sub info)) (go end_label)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (spotrf uplo n a lda info)
        (declare (ignore var-2))
        (when var-0 (setf uplo var-0))
```

```
        (when var-1 (setf n var-1))
        (when var-3 (setf lda var-3))
        (when var-4 (setf info var-4)))
      (cond
       ((= info 0)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (spotrs uplo n nrhs a lda b ldb$ info)
          (declare (ignore var-3 var-5))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf lda var-4))
          (when var-6 (setf ldb$ var-6))
          (when var-7 (setf info var-7)))))
      (go end_label)
     end_label
      (return (values uplo n nrhs nil lda nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sposv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

67.2 sposvx

67.2.1 Axiom unit tests

— **sposvx.input** —

```
)set break resume
)sys rm -f sposvx.output
)spool sposvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

67.2.2 Axiom help page

— **sposvx.help** —

```
==================================================================
sposvx examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sposvx (FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, EQUED, S, B,
           LDB, X, LDX, RCOND, FERR, BERR, WORK, IWORK, INFO)
            SPOSVX computes the solution to system of linear equations A * X =
           B for PO matrices

Function/Subroutine Documentation
   subroutine sposvx (character FACT, character UPLO, integer N, integer NRHS,
       real, dimension( lda, * ) A, integer LDA, real, dimension( ldaf, * )
       AF, integer LDAF, character EQUED, real, dimension( * ) S, real,
       dimension( ldb, * ) B, integer LDB, real, dimension( ldx, * ) X,
       integer LDX, real RCOND, real, dimension( * ) FERR, real, dimension( *
       ) BERR, real, dimension( * ) WORK, integer, dimension( * ) IWORK,
       integer INFO)
        SPOSVX computes the solution to system of linear equations A * X = B
```

```
       for PO matrices

Purpose:
   SPOSVX uses the Cholesky factorization A = U**T*U or A = L*L**T to
   compute the solution to a real system of linear equations
      A * X = B,
   where A is an N-by-N symmetric positive definite matrix and X and B
   are N-by-NRHS matrices.

   Error bounds on the solution and a condition estimate are also provided.

Description:

   The following steps are performed:

   1. If FACT = 'E', real scaling factors are computed to equilibrate
      the system:
         diag(S) * A * diag(S) * inv(diag(S)) * X = diag(S) * B
      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(S)*A*diag(S) and B by diag(S)*B.

   2. If FACT = 'N' or 'E', the Cholesky decomposition is used to
      factor the matrix A (after equilibration if FACT = 'E') as
         A = U**T* U,  if UPLO = 'U', or
         A = L * L**T,  if UPLO = 'L',
      where U is an upper triangular matrix and L is a lower triangular
      matrix.

   3. If the leading i-by-i principal minor is not positive definite,
      then the routine returns with INFO = i. Otherwise, the factored
      form of A is used to estimate the condition number of the matrix
      A.  If the reciprocal of the condition number is less than machine
      precision, INFO = N+1 is returned as a warning, but the routine
      still goes on to solve for X and compute error bounds as
      described below.

   4. The system of equations is solved for X using the factored form of A.

   5. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(S) so that it solves the original system before equilibration.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
```

```
  supplied on entry, and if not, whether the matrix A should be
  equilibrated before it is factored.
  = 'F':  On entry, AF contains the factored form of A.
     If EQUED = 'Y', the matrix A has been equilibrated
     with scaling factors given by S.  A and AF will not
     be modified.
  = 'N':  The matrix A will be copied to AF and factored.
  = 'E':  The matrix A will be equilibrated if necessary, then
     copied to AF and factored.

UPLO
  UPLO is CHARACTER*1
  = 'U':  Upper triangle of A is stored;
  = 'L':  Lower triangle of A is stored.

N
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is REAL array, dimension (LDA,N)
  On entry, the symmetric matrix A, except if FACT = 'F' and
  EQUED = 'Y', then A must contain the equilibrated matrix
  diag(S)*A*diag(S).  If UPLO = 'U', the leading
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.  A is not modified if
  FACT = 'F' or 'N', or if FACT = 'E' and EQUED = 'N' on exit.

  On exit, if FACT = 'E' and EQUED = 'Y', A is overwritten by
  diag(S)*A*diag(S).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is REAL array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the triangular factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T, in the same storage
  format as A.  If EQUED .ne. 'N', then AF is the factored form
```

```
  of the equilibrated matrix diag(S)*A*diag(S).

  If FACT = 'N', then AF is an output argument and on exit
  returns the triangular factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T of the original
  matrix A.

  If FACT = 'E', then AF is an output argument and on exit
  returns the triangular factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T of the equilibrated
  matrix A (see the description of A for the form of the
  equilibrated matrix).

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'Y':  Equilibration was done, i.e., A has been replaced by
     diag(S) * A * diag(S).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

S
  S is REAL array, dimension (N)
  The scale factors for A; not accessed if EQUED = 'N'.  S is
  an input argument if FACT = 'F'; otherwise, S is an output
  argument.  If FACT = 'F' and EQUED = 'Y', each element of S
  must be positive.

B
  B is REAL array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if EQUED = 'N', B is not modified; if EQUED = 'Y',
  B is overwritten by diag(S) * B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is REAL array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X to
  the original system of equations.  Note that if EQUED = 'Y',
  A and B are modified on exit, and the solution to the
  equilibrated system is inv(diag(S))*X.
```

```
LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  The estimate of the reciprocal condition number of the matrix
  A after equilibration (if done).  If RCOND is less than the
  machine precision (in particular, if RCOND = 0), the matrix
  is singular to working precision.  This condition is
  indicated by a return code of INFO > 0.

FERR
  FERR is REAL array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is REAL array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is REAL array, dimension (3*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = i, and i is
    <= N:  the leading minor of order i of A is
       not positive definite, so the factorization
       could not be completed, and the solution has not
       been computed. RCOND = 0 is returned.
    = N+1: U is nonsingular, but RCOND is less than machine
       precision, meaning that the matrix is singular
       to working precision.  Nevertheless, the
       solution and error bounds are computed because
       there are a number of situations where the
       computed solution can be more accurate than the
```

```
        value of RCOND would suggest.
```

———-

67.2.3 fortran code

— **sposvx.f** —

```
*  =====================================================================
      SUBROUTINE SPOSVX( FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, EQUED,
     $                   S, B, LDB, X, LDX, RCOND, FERR, BERR, WORK,
     $                   IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      REAL               A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   BERR( * ), FERR( * ), S( * ), WORK( * ),
     $                   X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            EQUIL, NOFACT, RCEQU
      INTEGER            I, INFEQU, J
      REAL               AMAX, ANORM, BIGNUM, SCOND, SMAX, SMIN, SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH, SLANSY
      EXTERNAL           LSAME, SLAMCH, SLANSY
*     ..
*     .. External Subroutines ..
```

```
      EXTERNAL           SLACPY, SLAQSY, SPOCON, SPOEQU, SPORFS, SPOTRF,
     $                   SPOTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         RCEQU = .FALSE.
      ELSE
         RCEQU = LSAME( EQUED, 'Y' )
         SMLNUM = SLAMCH( 'Safe minimum' )
         BIGNUM = ONE / SMLNUM
      END IF
*
*     Test the input parameters.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.LSAME( FACT, 'F' ) )
     $     THEN
         INFO = -1
      ELSE IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) )
     $          THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $         ( RCEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -9
      ELSE
         IF( RCEQU ) THEN
            SMIN = BIGNUM
            SMAX = ZERO
            DO 10 J = 1, N
               SMIN = MIN( SMIN, S( J ) )
               SMAX = MAX( SMAX, S( J ) )
   10       CONTINUE
            IF( SMIN.LE.ZERO ) THEN
               INFO = -10
            ELSE IF( N.GT.0 ) THEN
```

```
            SCOND = MAX( SMIN, SMLNUM ) / MIN( SMAX, BIGNUM )
         ELSE
            SCOND = ONE
         END IF
      END IF
      IF( INFO.EQ.0 ) THEN
         IF( LDB.LT.MAX( 1, N ) ) THEN
            INFO = -12
         ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
            INFO = -14
         END IF
      END IF
   END IF
*
   IF( INFO.NE.0 ) THEN
      CALL XERBLA( 'SPOSVX', -INFO )
      RETURN
   END IF
*
   IF( EQUIL ) THEN
*
*     Compute row and column scalings to equilibrate the matrix A.
*
      CALL SPOEQU( N, A, LDA, S, SCOND, AMAX, INFEQU )
      IF( INFEQU.EQ.0 ) THEN
*
*        Equilibrate the matrix.
*
         CALL SLAQSY( UPLO, N, A, LDA, S, SCOND, AMAX, EQUED )
         RCEQU = LSAME( EQUED, 'Y' )
      END IF
   END IF
*
*  Scale the right hand side.
*
   IF( RCEQU ) THEN
      DO 30 J = 1, NRHS
         DO 20 I = 1, N
            B( I, J ) = S( I )*B( I, J )
20       CONTINUE
30    CONTINUE
   END IF
*
   IF( NOFACT .OR. EQUIL ) THEN
*
*     Compute the Cholesky factorization A = U**T *U or A = L*L**T.
*
      CALL SLACPY( UPLO, N, N, A, LDA, AF, LDAF )
      CALL SPOTRF( UPLO, N, AF, LDAF, INFO )
*
```

```
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 )THEN
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A.
*
      ANORM = SLANSY( '1', UPLO, N, A, LDA, WORK )
*
*     Compute the reciprocal of the condition number of A.
*
      CALL SPOCON( UPLO, N, AF, LDAF, ANORM, RCOND, WORK, IWORK, INFO )
*
*     Compute the solution matrix X.
*
      CALL SLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL SPOTRS( UPLO, N, NRHS, AF, LDAF, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL SPORFS( UPLO, N, NRHS, A, LDA, AF, LDAF, B, LDB, X, LDX,
     $             FERR, BERR, WORK, IWORK, INFO )
*
*     Transform the solution matrix X to a solution of the original
*     system.
*
      IF( RCEQU ) THEN
         DO 50 J = 1, NRHS
            DO 40 I = 1, N
               X( I, J ) = S( I )*X( I, J )
   40       CONTINUE
   50    CONTINUE
         DO 60 J = 1, NRHS
            FERR( J ) = FERR( J ) / SCOND
   60    CONTINUE
      END IF
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.SLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      RETURN
*
*     End of SPOSVX
*
```

```
      END
```

———-

67.2.4 lisp code

— **sposvx.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sposvx
         (fact uplo n nrhs a lda af ldaf equed s b ldb$ x ldx rcond ferr berr
          work iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (single-float) rcond)
             (type (array single-float (*)) work berr ferr x b s af a)
             (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
             (type (string *) equed uplo fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (equed character equed-%data% equed-%offset%)
         (a single-float a-%data% a-%offset%)
         (af single-float af-%data% af-%offset%)
         (s single-float s-%data% s-%offset%)
         (b single-float b-%data% b-%offset%)
         (x single-float x-%data% x-%offset%)
         (ferr single-float ferr-%data% ferr-%offset%)
         (berr single-float berr-%data% berr-%offset%)
         (work single-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((amax 0.0) (anorm 0.0) (bignum 0.0) (scond 0.0) (smax 0.0)
             (smin 0.0) (smlnum 0.0) (i 0) (infequ 0) (j 0) (equil nil)
             (nofact nil) (rcequ nil))
        (declare (type (single-float) amax anorm bignum scond smax smin smlnum)
                 (type (f2cl-lib:integer4) i infequ j)
                 (type f2cl-lib:logical equil nofact rcequ))
        (setf info 0)
        (setf nofact
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame fact "N")
                  (declare (ignore var-1))
                  (when var-0 (setf fact var-0))
                  ret-val))
        (setf equil
                (multiple-value-bind (ret-val var-0 var-1)
```

```
            (lsame fact "E")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val))
(cond
 ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
  (setf rcequ f2cl-lib:%false%))
 (t
  (setf rcequ
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame equed "Y")
            (declare (ignore var-1))
            (when var-0 (setf equed var-0))
            ret-val))
  (setf smlnum (slamch "Safe minimum")) (setf bignum (/ one smlnum))))
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame fact "F")
     (declare (ignore var-1))
     (when var-0 (setf fact var-0))
     ret-val)
   (not
```

```
      (or rcequ
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame equed "N")
            (declare (ignore var-1))
            (when var-0 (setf equed var-0))
            ret-val))))
    (setf info -9))
   (t
    (cond
     (rcequ (setf smin bignum) (setf smax zero)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf smin
                  (min smin
                       (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
          (setf smax
                  (max smax
                       (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
         label10))
      (cond ((<= smin zero) (setf info -10))
            ((> n 0)
             (setf scond (/ (max smin smlnum) (min smax bignum))))
            (t (setf scond one)))))
    (cond
     ((= info 0)
      (cond
       ((< ldb$
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -12))
       ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -14)))))))
  (cond
   ((/= info 0) (xerbla "SPOSVX" (f2cl-lib:int-sub info))
    (go end_label)))
  (cond
   (equil
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (spoequ n a lda s scond amax infequ)
      (declare (ignore var-0 var-1 var-2 var-3))
      (setf scond var-4)
      (setf amax var-5)
      (setf infequ var-6))
    (cond
     ((= infequ 0)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (slaqsy uplo n a lda s scond amax equed)
        (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
        (setf uplo var-0)
```

```
      (setf equed var-7))
    (setf rcequ
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame equed "Y")
              (declare (ignore var-1))
              (when var-0 (setf equed var-0))
              ret-val))))))
(cond
 (rcequ
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                  (* (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)))
         label20))
     label30))))
(cond
 ((or nofact equil)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy uplo n n a lda af ldaf)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
    (setf uplo var-0))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (spotrf uplo n af ldaf info)
    (declare (ignore var-2))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf ldaf var-3))
    (when var-4 (setf info var-4)))
  (cond ((> info 0) (setf rcond zero) (go end_label)))))
(setf anorm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (slansy "1" uplo n a lda work)
          (declare (ignore var-0 var-3 var-4 var-5))
          (setf uplo var-1)
          (setf n var-2)
          ret-val))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (spocon uplo n af ldaf anorm rcond work iwork info)
  (declare (ignore var-2 var-4 var-6 var-7))
  (setf uplo var-0)
  (setf n var-1)
```

```
  (setf ldaf var-3)
  (setf rcond var-5)
  (setf info var-8))
(slacpy "Full" n nrhs b ldb$ x ldx)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (spotrs uplo n nrhs af ldaf x ldx info)
  (declare (ignore var-3 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldaf var-4))
  (when var-6 (setf ldx var-6))
  (when var-7 (setf info var-7)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15)
    (sporfs uplo n nrhs a lda af ldaf b ldb$ x ldx ferr berr work iwork
     info)
  (declare
   (ignore var-2 var-3 var-5 var-7 var-8 var-9 var-10 var-11 var-12
    var-13 var-14))
  (setf uplo var-0)
  (setf n var-1)
  (setf lda var-4)
  (setf ldaf var-6)
  (setf info var-15))
(cond
 (rcequ
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                               x-%offset%)
                  (* (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                     (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                    x-%offset%)))
         label40))
     label50))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
              (/ (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                 scond))
     label60))))
(if (< rcond (slamch "Epsilon"))
    (setf info (f2cl-lib:int-add n 1)))
```

```
        (go end_label)
       end_label
        (return
         (values fact uplo n nrhs nil lda nil ldaf equed nil nil nil nil ldx
                 rcond nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sposvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (string) (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::fact
                                              fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldaf
                                              fortran-to-lisp::equed nil nil
                                              nil nil fortran-to-lisp::ldx
                                              fortran-to-lisp::rcond nil nil
                                              nil nil fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::sporfs
                                              fortran-to-lisp::spocon
                                              fortran-to-lisp::slansy
                                              fortran-to-lisp::slacpy
                                              fortran-to-lisp::slaqsy
                                              fortran-to-lisp::spoequ))))
```

———-

67.3 sposvxx

67.3.1 Axiom unit tests

— **sposvxx.input** —

```
)set break resume
)sys rm -f sposvxx.output
)spool sposvxx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

67.3.2 Axiom help page [Demm05]

— **sposvxx.help** —

```
====================================================================
sposvxx examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sposvxx (FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, EQUED, S, B,
           LDB, X, LDX, RCOND, RPVGRW, BERR, N_ERR_BNDS, ERR_BNDS_NORM,
           ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, IWORK, INFO)
            SPOSVXX computes the solution to system of linear equations A * X
           = B for PO matrices

Function/Subroutine Documentation
   subroutine sposvxx (character FACT, character UPLO, integer N, integer
```

```
     NRHS, real, dimension( lda, * ) A, integer LDA, real, dimension( ldaf,
     * ) AF, integer LDAF, character EQUED, real, dimension( * ) S, real,
     dimension( ldb, * ) B, integer LDB, real, dimension( ldx, * ) X,
     integer LDX, real RCOND, real RPVGRW, real, dimension( * ) BERR,
     integer N_ERR_BNDS, real, dimension( nrhs, * ) ERR_BNDS_NORM, real,
     dimension( nrhs, * ) ERR_BNDS_COMP, integer NPARAMS, real, dimension( *
     ) PARAMS, real, dimension( * ) WORK, integer, dimension( * ) IWORK,
     integer INFO)
      SPOSVXX computes the solution to system of linear equations A * X = B
     for PO matrices

Purpose:
   SPOSVXX uses the Cholesky factorization A = U**T*U or A = L*L**T
   to compute the solution to a real system of linear equations
   A * X = B, where A is an N-by-N symmetric positive definite matrix
   and X and B are N-by-NRHS matrices.

   If requested, both normwise and maximum componentwise error bounds
   are returned. SPOSVXX will return a solution with a tiny
   guaranteed error (O(eps) where eps is the working machine
   precision) unless the matrix is very ill-conditioned, in which
   case a warning is returned. Relevant condition numbers also are
   calculated and returned.

   SPOSVXX accepts user-provided factorizations and equilibration
   factors; see the definitions of the FACT and EQUED options.
   Solving with refinement and using a factorization from a previous
   SPOSVXX call will also produce a solution with either O(eps)
   errors or warnings, but we cannot make that claim for general
   user-provided factorizations and equilibration factors if they
   differ from what SPOSVXX would itself produce.

Description:

   The following steps are performed:

   1. If FACT = 'E', real scaling factors are computed to equilibrate
      the system:

      diag(S)*A*diag(S)     *inv(diag(S))*X = diag(S)*B

      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(S)*A*diag(S) and B by diag(S)*B.

   2. If FACT = 'N' or 'E', the Cholesky decomposition is used to
      factor the matrix A (after equilibration if FACT = 'E') as
         A = U**T* U,  if UPLO = 'U', or
         A = L * L**T,  if UPLO = 'L',
      where U is an upper triangular matrix and L is a lower triangular
```

```
      matrix.

   3. If the leading i-by-i principal minor is not positive definite,
      then the routine returns with INFO = i. Otherwise, the factored
      form of A is used to estimate the condition number of the matrix
      A (see argument RCOND).  If the reciprocal of the condition number
      is less than machine precision, the routine still goes on to solve
      for X and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. By default (unless PARAMS(LA_LINRX_ITREF_I) is set to zero),
      the routine will use iterative refinement to try to get a small
      error and error bounds.  Refinement calculates the residual to at
      least twice the working precision.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(S) so that it solves the original system before equilibration.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AF contains the factored form of A.
      If EQUED is not 'N', the matrix A has been
      equilibrated with scaling factors given by S.
      A and AF are not modified.
   = 'N':  The matrix A will be copied to AF and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
      copied to AF and factored.

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
```

```
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is REAL array, dimension (LDA,N)
  On entry, the symmetric matrix A, except if FACT = 'F' and EQUED =
  'Y', then A must contain the equilibrated matrix
  diag(S)*A*diag(S).  If UPLO = 'U', the leading N-by-N upper
  triangular part of A contains the upper triangular part of the
  matrix A, and the strictly lower triangular part of A is not
  referenced.  If UPLO = 'L', the leading N-by-N lower triangular
  part of A contains the lower triangular part of the matrix A, and
  the strictly upper triangular part of A is not referenced.  A is
  not modified if FACT = 'F' or 'N', or if FACT = 'E' and EQUED =
  'N' on exit.

  On exit, if FACT = 'E' and EQUED = 'Y', A is overwritten by
  diag(S)*A*diag(S).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is REAL array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the triangular factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T, in the same storage
  format as A.  If EQUED .ne. 'N', then AF is the factored
  form of the equilibrated matrix diag(S)*A*diag(S).

  If FACT = 'N', then AF is an output argument and on exit
  returns the triangular factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T of the original
  matrix A.

  If FACT = 'E', then AF is an output argument and on exit
  returns the triangular factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T of the equilibrated
  matrix A (see the description of A for the form of the
  equilibrated matrix).

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
```

```
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'Y':  Both row and column equilibration, i.e., A has been
          replaced by diag(S) * A * diag(S).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

S
  S is REAL array, dimension (N)
  The row scale factors for A.  If EQUED = 'Y', A is multiplied on
  the left and right by diag(S).  S is an input argument if FACT =
  'F'; otherwise, S is an output argument.  If FACT = 'F' and EQUED
  = 'Y', each element of S must be positive.  If S is output, each
  element of S is a power of the radix. If S is input, each element
  of S should be a power of the radix to ensure a reliable solution
  and error estimates. Scaling by powers of the radix does not cause
  rounding errors unless the result underflows or overflows.
  Rounding errors during scaling lead to refining with a matrix that
  is not equivalent to the input matrix, producing error estimates
  that may not be reliable.

B
  B is REAL array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if EQUED = 'Y', B is overwritten by diag(S)*B;

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is REAL array, dimension (LDX,NRHS)
  If INFO = 0, the N-by-NRHS solution matrix X to the original
  system of equations.  Note that A and B are modified on exit if
  EQUED .ne. 'N', and the solution to the equilibrated system is
  inv(diag(S))*X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.
```

```
RPVGRW
  RPVGRW is REAL
  Reciprocal pivot growth.  On exit, this contains the reciprocal
  pivot growth factor norm(A)/norm(U). The "max absolute element"
  norm is used.  If this is much less than 1, then the stability of
  the LU factorization of the (equilibrated) matrix A could be poor.
  This also means that the solution X, estimated condition numbers,
  and error bounds could be unreliable. If factorization fails with
  0<INFO<=N, then this contains the reciprocal pivot growth factor
  for the leading INFO columns of A.

BERR
  BERR is REAL array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
            max_j (abs(XTRUE(j,i) - X(j,i)))
             ------------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').
```

```
  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.

ERR_BNDS_COMP
  ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                 abs(XTRUE(j,i) - X(j,i))
           max_j ----------------------
                       abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.
```

```
  err = 3  Reciprocal condition number: Estimated componentwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is REAL array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
  (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is REAL array, dimension (4*N)
```

```
 IWORK
   IWORK is INTEGER array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  Successful exit. The solution to every right-hand side is
         guaranteed.
   < 0:  If INFO = -i, the i-th argument had an illegal value
   > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
      has been completed, but the factor U is exactly singular, so
      the solution and error bounds could not be computed. RCOND = 0
      is returned.
   = N+J: The solution corresponding to the Jth right-hand side is
      not guaranteed. The solutions corresponding to other right-
      hand sides K with K > J may not be guaranteed as well, but
      only the first such right-hand side is reported. If a small
      componentwise error is not requested (PARAMS(3) = 0.0) then
      the Jth right-hand side is the first with a normwise error
      bound that is not guaranteed (the smallest J such
      that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
      the Jth right-hand side is the first with either a normwise or
      componentwise error bound that is not guaranteed (the smallest
      J such that either ERR_BNDS_NORM(J,1) = 0.0 or
      ERR_BNDS_COMP(J,1) = 0.0). See the definition of
      ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
      about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

———-

67.3.3 fortran code

— **sposvxx.f** —

```
*  ==================================================================
      SUBROUTINE SPOSVXX( FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, EQUED,
     $                    S, B, LDB, X, LDX, RCOND, RPVGRW, BERR,
     $                    N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP,
     $                    NPARAMS, PARAMS, WORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
```

```
     $                   N_ERR_BNDS
      REAL               RCOND, RPVGRW
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      REAL               A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX, * ), WORK( * )
      REAL               S( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
*     ..
*     .. Local Scalars ..
      LOGICAL            EQUIL, NOFACT, RCEQU
      INTEGER            INFEQU, J
      REAL               AMAX, BIGNUM, SMIN, SMAX,
     $                   SCOND, SMLNUM
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, SLAMCH, SLA_PORPVGRW
      LOGICAL            LSAME
      REAL               SLAMCH, SLA_PORPVGRW
*     ..
*     .. External Subroutines ..
      EXTERNAL           SPOEQUB, SPOTRF, SPOTRS, SLACPY, SLAQSY,
     $                   XERBLA, SLASCL2, SPORFSX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      SMLNUM = SLAMCH( 'Safe minimum' )
```

```
      BIGNUM = ONE / SMLNUM
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         RCEQU = .FALSE.
      ELSE
         RCEQU = LSAME( EQUED, 'Y' )
      ENDIF
*
*     Default is failure.  If an input parameter is wrong or
*     factorization fails, make everything look horrible.  Only the
*     pivot growth is set here, the rest is initialized in SPORFSX.
*
      RPVGRW = ZERO
*
*     Test the input parameters.  PARAMS is not tested until SPORFSX.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.
     $     LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.LSAME( UPLO, 'U' ) .AND.
     $         .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $        ( RCEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -9
      ELSE
         IF ( RCEQU ) THEN
            SMIN = BIGNUM
            SMAX = ZERO
            DO 10 J = 1, N
               SMIN = MIN( SMIN, S( J ) )
               SMAX = MAX( SMAX, S( J ) )
 10         CONTINUE
            IF( SMIN.LE.ZERO ) THEN
               INFO = -10
            ELSE IF( N.GT.0 ) THEN
               SCOND = MAX( SMIN, SMLNUM ) / MIN( SMAX, BIGNUM )
            ELSE
               SCOND = ONE
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
```

```
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -12
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -14
            END IF
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SPOSVXX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*     Compute row and column scalings to equilibrate the matrix A.
*
         CALL SPOEQUB( N, A, LDA, S, SCOND, AMAX, INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*     Equilibrate the matrix.
*
            CALL SLAQSY( UPLO, N, A, LDA, S, SCOND, AMAX, EQUED )
            RCEQU = LSAME( EQUED, 'Y' )
         END IF
      END IF
*
*     Scale the right-hand side.
*
      IF( RCEQU ) CALL SLASCL2( N, NRHS, S, B, LDB )
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the Cholesky factorization of A.
*
         CALL SLACPY( UPLO, N, N, A, LDA, AF, LDAF )
         CALL SPOTRF( UPLO, N, AF, LDAF, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.NE.0 ) THEN
*
*           Pivot in column INFO is exactly 0
*           Compute the reciprocal pivot growth factor of the
*           leading rank-deficient INFO columns of A.
*
            RPVGRW = SLA_PORPVGRW( UPLO, INFO, A, LDA, AF, LDAF, WORK )
            RETURN
         ENDIF
      END IF
```

```
*
*     Compute the reciprocal growth factor RPVGRW.
*
      RPVGRW = SLA_PORPVGRW( UPLO, N, A, LDA, AF, LDAF, WORK )
*
*     Compute the solution matrix X.
*
      CALL SLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL SPOTRS( UPLO, N, NRHS, AF, LDAF, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL SPORFSX( UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF,
     $     S, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS, ERR_BNDS_NORM,
     $     ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, IWORK, INFO )

*
*     Scale solutions.
*
      IF ( RCEQU ) THEN
         CALL SLASCL2 ( N, NRHS, S, X, LDX )
      END IF
*
      RETURN
*
*     End of SPOSVXX
*
      END
```

———-

67.3.4 lisp code

— **sposvxx.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun sposvxx
         (fact uplo n nrhs a lda af ldaf equed s b ldb$ x ldx rcond rpvgrw berr
          n_err_bnds err_bnds_norm err_bnds_comp nparams params work iwork
          info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (single-float) rpvgrw rcond)
             (type (array single-float (*)) work params err_bnds_comp
              err_bnds_norm berr x b s af a)
```

```
          (type (f2cl-lib:integer4) info nparams n_err_bnds ldx ldb$ ldaf
           lda nrhs n)
          (type (string *) equed uplo fact))
  (f2cl-lib:with-multi-array-data
      ((fact character fact-%data% fact-%offset%)
       (uplo character uplo-%data% uplo-%offset%)
       (equed character equed-%data% equed-%offset%)
       (a single-float a-%data% a-%offset%)
       (af single-float af-%data% af-%offset%)
       (s single-float s-%data% s-%offset%)
       (b single-float b-%data% b-%offset%)
       (x single-float x-%data% x-%offset%)
       (berr single-float berr-%data% berr-%offset%)
       (err_bnds_norm single-float err_bnds_norm-%data%
        err_bnds_norm-%offset%)
       (err_bnds_comp single-float err_bnds_comp-%data%
        err_bnds_comp-%offset%)
       (params single-float params-%data% params-%offset%)
       (work single-float work-%data% work-%offset%)
       (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
    (prog ((amax 0.0) (bignum 0.0) (smin 0.0) (smax 0.0) (scond 0.0)
          (smlnum 0.0) (infequ 0) (j 0) (equil nil) (nofact nil)
          (rcequ nil))
      (declare (type (single-float) amax bignum smin smax scond smlnum)
               (type (f2cl-lib:integer4) infequ j)
               (type f2cl-lib:logical equil nofact rcequ))
      (setf info 0)
      (setf nofact
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame fact "N")
                (declare (ignore var-1))
                (when var-0 (setf fact var-0))
                ret-val))
      (setf equil
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame fact "E")
                (declare (ignore var-1))
                (when var-0 (setf fact var-0))
                ret-val))
      (setf smlnum (slamch "Safe minimum"))
      (setf bignum (/ one smlnum))
      (cond
       ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
        (setf rcequ f2cl-lib:%false%))
       (t
        (setf rcequ
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame equed "Y")
                  (declare (ignore var-1))
                  (when var-0 (setf equed var-0))
```

```
            ret-val))))
(setf rpvgrw zero)
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame fact "F")
     (declare (ignore var-1))
     (when var-0 (setf fact var-0))
     ret-val)
   (not
    (or rcequ
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame equed "N")
          (declare (ignore var-1))
          (when var-0 (setf equed var-0))
          ret-val))))
  (setf info -9))
 (t
  (cond
   (rcequ (setf smin bignum) (setf smax zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
```

```
        (setf smin
                (min smin
                     (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
        (setf smax
                (max smax
                     (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
       label10))
    (cond ((<= smin zero) (setf info -10))
          ((> n 0)
           (setf scond (/ (max smin smlnum) (min smax bignum))))
          (t (setf scond one)))))
  (cond
   ((= info 0)
    (cond
     ((< ldb$
         (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -12))
     ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -14)))))))
(cond
 ((/= info 0) (xerbla "SPOSVXX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (equil
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (spoequb n a lda s scond amax infequ)
    (declare (ignore var-0 var-1 var-2 var-3))
    (setf scond var-4)
    (setf amax var-5)
    (setf infequ var-6))
  (cond
   ((= infequ 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (slaqsy uplo n a lda s scond amax equed)
      (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
      (setf uplo var-0)
      (setf equed var-7))
    (setf rcequ
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame equed "Y")
              (declare (ignore var-1))
              (when var-0 (setf equed var-0))
              ret-val))))))
(if rcequ
    (slascl2 n nrhs s b ldb$))
(cond
 ((or nofact equil)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacpy uplo n n a lda af ldaf)
```

```
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
    (setf uplo var-0))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (spotrf uplo n af ldaf info)
    (declare (ignore var-2))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf ldaf var-3))
    (when var-4 (setf info var-4)))
  (cond
   ((/= info 0)
    (setf rpvgrw
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (sla_porpvgrw uplo info a lda af ldaf work)
              (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
              (setf uplo var-0)
              ret-val))
    (go end_label)))))
(setf rpvgrw
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (sla_porpvgrw uplo n a lda af ldaf work)
          (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
          (setf uplo var-0)
          ret-val))
(slacpy "Full" n nrhs b ldb$ x ldx)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (spotrs uplo n nrhs af ldaf x ldx info)
  (declare (ignore var-3 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldaf var-4))
  (when var-6 (setf ldx var-6))
  (when var-7 (setf info var-7)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19
     var-20 var-21 var-22)
    (sporfsx uplo equed n nrhs a lda af ldaf s b ldb$ x ldx rcond berr
     n_err_bnds err_bnds_norm err_bnds_comp nparams params work iwork
     info)
  (declare
   (ignore var-4 var-6 var-8 var-9 var-11 var-14 var-16 var-17 var-19
    var-20 var-21))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf equed var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf nrhs var-3))
```

```
          (when var-5 (setf lda var-5))
          (when var-7 (setf ldaf var-7))
          (when var-10 (setf ldb$ var-10))
          (when var-12 (setf ldx var-12))
          (when var-13 (setf rcond var-13))
          (when var-15 (setf n_err_bnds var-15))
          (when var-18 (setf nparams var-18))
          (when var-22 (setf info var-22)))
        (cond (rcequ (slascl2 n nrhs s x ldx)))
        (go end_label)
       end_label
        (return
         (values fact uplo n nrhs nil lda nil ldaf equed nil nil ldb$ nil ldx
                 rcond rpvgrw nil n_err_bnds nil nil nparams nil nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sposvxx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (string) (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::fact
                                              fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
```

```
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldaf
  fortran-to-lisp::equed nil nil
  fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond
  fortran-to-lisp::rpvgrw nil
  fortran-to-lisp::n_err_bnds nil
  nil fortran-to-lisp::nparams nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::sla_porpvgrw
  fortran-to-lisp::slacpy
  fortran-to-lisp::slascl2
  fortran-to-lisp::slaqsy
  fortran-to-lisp::spoequb))))
```

———-

Chapter 68

LAPACK - Positive Definite Matrix, Linear Solve, Complex

68.1 cposv

68.1.1 Axiom unit tests

— cposv.input —

```
)set break resume
)sys rm -f cposv.output
)spool cposv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

68.1.2 Axiom help page

— cposv.help —

```
========================================================================
cposv examples
========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cposv (UPLO, N, NRHS, A, LDA, B, LDB, INFO)
            CPOSV computes the solution to system of linear equations A * X =
           B for PO matrices

Function/Subroutine Documentation
   subroutine cposv (character UPLO, integer N, integer NRHS, complex,
       dimension( lda, * ) A, integer LDA, complex, dimension( ldb, * ) B,
       integer LDB, integer INFO)
        CPOSV computes the solution to system of linear equations A * X = B
       for PO matrices

Purpose:
   CPOSV computes the solution to a complex system of linear equations
      A * X = B,
   where A is an N-by-N Hermitian positive definite matrix and X and B
   are N-by-NRHS matrices.

   The Cholesky decomposition is used to factor A as
      A = U**H* U,  if UPLO = 'U', or
      A = L * L**H,  if UPLO = 'L',
   where U is an upper triangular matrix and  L is a lower triangular
   matrix.  The factored form of A is then used to solve the system of
   equations A * X = B.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the Hermitian matrix A.  If UPLO = 'U', the leading
```

```
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if INFO = 0, the factor U or L from the Cholesky
  factorization A = U**H*U or A = L*L**H.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, the leading minor of order i of A is not
        positive definite, so the factorization could not be
        completed, and the solution has not been computed.
```

———-

68.1.3 fortran code

— cposv.f —

```
*  ===================================================================
      SUBROUTINE CPOSV( UPLO, N, NRHS, A, LDA, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
```

```
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CPOTRF, CPOTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CPOSV ', -INFO )
         RETURN
      END IF
*
*     Compute the Cholesky factorization A = U**H*U or A = L*L**H.
*
      CALL CPOTRF( UPLO, N, A, LDA, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
         CALL CPOTRS( UPLO, N, NRHS, A, LDA, B, LDB, INFO )
*
      END IF
      RETURN
```

```
*
*     End of CPOSV
*
      END
```

———-

68.1.4 lisp code

— **cposv.lisp** —

```
(defun cposv (uplo n nrhs a lda b ldb$ info)
  (declare (type (array f2cl-lib:complex8 (*)) b a)
           (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (b f2cl-lib:complex8 b-%data% b-%offset%))
    (prog ()
      (declare)
      (setf info 0)
      (cond
       ((and
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -5))
       ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -7)))
      (cond
       ((/= info 0) (xerbla "CPOSV " (f2cl-lib:int-sub info)) (go end_label)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (cpotrf uplo n a lda info)
        (declare (ignore var-2))
        (when var-0 (setf uplo var-0))
```

```
        (when var-1 (setf n var-1))
        (when var-3 (setf lda var-3))
        (when var-4 (setf info var-4)))
      (cond
       ((= info 0)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (cpotrs uplo n nrhs a lda b ldb$ info)
          (declare (ignore var-3 var-5))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf lda var-4))
          (when var-6 (setf ldb$ var-6))
          (when var-7 (setf info var-7)))))
      (go end_label)
     end_label
      (return (values uplo n nrhs nil lda nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cposv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

68.2 cposvx

68.2.1 Axiom unit tests

— **cposvx.input** —

```
)set break resume
)sys rm -f cposvx.output
)spool cposvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

68.2.2 Axiom help page

— **cposvx.help** —

```
======================================================================
cposvx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cposvx (FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, EQUED, S, B,
           LDB, X, LDX, RCOND, FERR, BERR, WORK, RWORK, INFO)
            CPOSVX computes the solution to system of linear equations A * X =
           B for PO matrices

Function/Subroutine Documentation
   subroutine cposvx (character FACT, character UPLO, integer N, integer NRHS,
       complex, dimension( lda, * ) A, integer LDA, complex, dimension( ldaf,
       * ) AF, integer LDAF, character EQUED, real, dimension( * ) S, complex,
       dimension( ldb, * ) B, integer LDB, complex, dimension( ldx, * ) X,
       integer LDX, real RCOND, real, dimension( * ) FERR, real, dimension( *
       ) BERR, complex, dimension( * ) WORK, real, dimension( * ) RWORK,
       integer INFO)
        CPOSVX computes the solution to system of linear equations A * X = B
```

```
       for PO matrices

Purpose:
   CPOSVX uses the Cholesky factorization A = U**H*U or A = L*L**H to
   compute the solution to a complex system of linear equations
      A * X = B,
   where A is an N-by-N Hermitian positive definite matrix and X and B
   are N-by-NRHS matrices.

   Error bounds on the solution and a condition estimate are also
   provided.

Description:

   The following steps are performed:

   1. If FACT = 'E', real scaling factors are computed to equilibrate
      the system:
         diag(S) * A * diag(S) * inv(diag(S)) * X = diag(S) * B
      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(S)*A*diag(S) and B by diag(S)*B.

   2. If FACT = 'N' or 'E', the Cholesky decomposition is used to
      factor the matrix A (after equilibration if FACT = 'E') as
         A = U**H* U,  if UPLO = 'U', or
         A = L * L**H,  if UPLO = 'L',
      where U is an upper triangular matrix and L is a lower triangular
      matrix.

   3. If the leading i-by-i principal minor is not positive definite,
      then the routine returns with INFO = i. Otherwise, the factored
      form of A is used to estimate the condition number of the matrix
      A.  If the reciprocal of the condition number is less than machine
      precision, INFO = N+1 is returned as a warning, but the routine
      still goes on to solve for X and compute error bounds as
      described below.

   4. The system of equations is solved for X using the factored form of A.

   5. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(S) so that it solves the original system before
      equilibration.

Parameters:

 FACT
```

```
  FACT is CHARACTER*1
  Specifies whether or not the factored form of the matrix A is
  supplied on entry, and if not, whether the matrix A should be
  equilibrated before it is factored.
  = 'F':  On entry, AF contains the factored form of A.
     If EQUED = 'Y', the matrix A has been equilibrated
     with scaling factors given by S.  A and AF will not
     be modified.
  = 'N':  The matrix A will be copied to AF and factored.
  = 'E':  The matrix A will be equilibrated if necessary, then
     copied to AF and factored.

UPLO
  UPLO is CHARACTER*1
  = 'U':  Upper triangle of A is stored;
  = 'L':  Lower triangle of A is stored.

N
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the Hermitian matrix A, except if FACT = 'F' and
  EQUED = 'Y', then A must contain the equilibrated matrix
  diag(S)*A*diag(S).  If UPLO = 'U', the leading
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.  A is not modified if
  FACT = 'F' or 'N', or if FACT = 'E' and EQUED = 'N' on exit.

  On exit, if FACT = 'E' and EQUED = 'Y', A is overwritten by
  diag(S)*A*diag(S).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the triangular factor U or L from the Cholesky
```

```
  factorization A = U**H*U or A = L*L**H, in the same storage
  format as A.  If EQUED .ne. 'N', then AF is the factored form
  of the equilibrated matrix diag(S)*A*diag(S).

  If FACT = 'N', then AF is an output argument and on exit
  returns the triangular factor U or L from the Cholesky
  factorization A = U**H*U or A = L*L**H of the original
  matrix A.

  If FACT = 'E', then AF is an output argument and on exit
  returns the triangular factor U or L from the Cholesky
  factorization A = U**H*U or A = L*L**H of the equilibrated
  matrix A (see the description of A for the form of the
  equilibrated matrix).

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'Y':  Equilibration was done, i.e., A has been replaced by
          diag(S) * A * diag(S).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

S
  S is REAL array, dimension (N)
  The scale factors for A; not accessed if EQUED = 'N'.  S is
  an input argument if FACT = 'F'; otherwise, S is an output
  argument.  If FACT = 'F' and EQUED = 'Y', each element of S
  must be positive.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS righthand side matrix B.
  On exit, if EQUED = 'N', B is not modified; if EQUED = 'Y',
  B is overwritten by diag(S) * B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X to
  the original system of equations.  Note that if EQUED = 'Y',
  A and B are modified on exit, and the solution to the
```

```
  equilibrated system is inv(diag(S))*X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  The estimate of the reciprocal condition number of the matrix
  A after equilibration (if done).  If RCOND is less than the
  machine precision (in particular, if RCOND = 0), the matrix
  is singular to working precision.  This condition is
  indicated by a return code of INFO > 0.

FERR
  FERR is REAL array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is REAL array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is COMPLEX array, dimension (2*N)

RWORK
  RWORK is REAL array, dimension (N)

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = i, and i is
  <= N:  the leading minor of order i of A is
     not positive definite, so the factorization
     could not be completed, and the solution has not
     been computed. RCOND = 0 is returned.
  = N+1: U is nonsingular, but RCOND is less than machine
     precision, meaning that the matrix is singular
     to working precision.  Nevertheless, the
     solution and error bounds are computed because
```

```
      there are a number of situations where the
      computed solution can be more accurate than the
      value of RCOND would suggest.
```

———-

68.2.3 fortran code

— **cposvx.f** —

```
*  =====================================================================
      SUBROUTINE CPOSVX( FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, EQUED,
     $                   S, B, LDB, X, LDX, RCOND, FERR, BERR, WORK,
     $                   RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      REAL               BERR( * ), FERR( * ), RWORK( * ), S( * )
      COMPLEX            A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            EQUIL, NOFACT, RCEQU
      INTEGER            I, INFEQU, J
      REAL               AMAX, ANORM, BIGNUM, SCOND, SMAX, SMIN, SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               CLANHE, SLAMCH
      EXTERNAL           LSAME, CLANHE, SLAMCH
*     ..
```

```
*     .. External Subroutines ..
      EXTERNAL           CLACPY, CLAQHE, CPOCON, CPOEQU, CPORFS, CPOTRF,
     $                   CPOTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         RCEQU = .FALSE.
      ELSE
         RCEQU = LSAME( EQUED, 'Y' )
         SMLNUM = SLAMCH( 'Safe minimum' )
         BIGNUM = ONE / SMLNUM
      END IF
*
*     Test the input parameters.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.LSAME( FACT, 'F' ) )
     $     THEN
         INFO = -1
      ELSE IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) )
     $          THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $         ( RCEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -9
      ELSE
         IF( RCEQU ) THEN
            SMIN = BIGNUM
            SMAX = ZERO
            DO 10 J = 1, N
               SMIN = MIN( SMIN, S( J ) )
               SMAX = MAX( SMAX, S( J ) )
   10       CONTINUE
            IF( SMIN.LE.ZERO ) THEN
               INFO = -10
```

```
            ELSE IF( N.GT.0 ) THEN
               SCOND = MAX( SMIN, SMLNUM ) / MIN( SMAX, BIGNUM )
            ELSE
               SCOND = ONE
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -12
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -14
            END IF
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CPOSVX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*        Compute row and column scalings to equilibrate the matrix A.
*
         CALL CPOEQU( N, A, LDA, S, SCOND, AMAX, INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*           Equilibrate the matrix.
*
            CALL CLAQHE( UPLO, N, A, LDA, S, SCOND, AMAX, EQUED )
            RCEQU = LSAME( EQUED, 'Y' )
         END IF
      END IF
*
*     Scale the right hand side.
*
      IF( RCEQU ) THEN
         DO 30 J = 1, NRHS
            DO 20 I = 1, N
               B( I, J ) = S( I )*B( I, J )
   20       CONTINUE
   30    CONTINUE
      END IF
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the Cholesky factorization A = U**H *U or A = L*L**H.
*
         CALL CLACPY( UPLO, N, N, A, LDA, AF, LDAF )
         CALL CPOTRF( UPLO, N, AF, LDAF, INFO )
```

```
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 )THEN
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A.
*
      ANORM = CLANHE( '1', UPLO, N, A, LDA, RWORK )
*
*     Compute the reciprocal of the condition number of A.
*
      CALL CPOCON( UPLO, N, AF, LDAF, ANORM, RCOND, WORK, RWORK, INFO )
*
*     Compute the solution matrix X.
*
      CALL CLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL CPOTRS( UPLO, N, NRHS, AF, LDAF, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL CPORFS( UPLO, N, NRHS, A, LDA, AF, LDAF, B, LDB, X, LDX,
     $             FERR, BERR, WORK, RWORK, INFO )
*
*     Transform the solution matrix X to a solution of the original
*     system.
*
      IF( RCEQU ) THEN
         DO 50 J = 1, NRHS
            DO 40 I = 1, N
               X( I, J ) = S( I )*X( I, J )
   40       CONTINUE
   50    CONTINUE
         DO 60 J = 1, NRHS
            FERR( J ) = FERR( J ) / SCOND
   60    CONTINUE
      END IF
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.SLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      RETURN
*
*     End of CPOSVX
```

```
*
      END
```

———-

68.2.4 lisp code

— cposvx.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun cposvx
         (fact uplo n nrhs a lda af ldaf equed s b ldb$ x ldx rcond ferr berr
          work rwork info)
    (declare (type (single-float) rcond)
             (type (array single-float (*)) rwork berr ferr s)
             (type (array f2cl-lib:complex8 (*)) work x b af a)
             (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
             (type (string *) equed uplo fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (equed character equed-%data% equed-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (af f2cl-lib:complex8 af-%data% af-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (s single-float s-%data% s-%offset%)
         (ferr single-float ferr-%data% ferr-%offset%)
         (berr single-float berr-%data% berr-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%))
      (prog ((amax 0.0) (anorm 0.0) (bignum 0.0) (scond 0.0) (smax 0.0)
             (smin 0.0) (smlnum 0.0) (i 0) (infequ 0) (j 0) (equil nil)
             (nofact nil) (rcequ nil))
        (declare (type (single-float) amax anorm bignum scond smax smin smlnum)
                 (type (f2cl-lib:integer4) i infequ j)
                 (type f2cl-lib:logical equil nofact rcequ))
        (setf info 0)
        (setf nofact
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame fact "N")
                  (declare (ignore var-1))
                  (when var-0 (setf fact var-0))
                  ret-val))
        (setf equil
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "E")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val))
(cond
 ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
  (setf rcequ f2cl-lib:%false%))
 (t
  (setf rcequ
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame equed "Y")
            (declare (ignore var-1))
            (when var-0 (setf equed var-0))
            ret-val))
  (setf smlnum (slamch "Safe minimum")) (setf bignum (/ one smlnum))))
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame fact "F")
     (declare (ignore var-1))
     (when var-0 (setf fact var-0))
     ret-val)
```

```
    (not
     (or rcequ
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame equed "N")
           (declare (ignore var-1))
           (when var-0 (setf equed var-0))
           ret-val))))
  (setf info -9))
 (t
  (cond
   (rcequ (setf smin bignum) (setf smax zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf smin
                (min smin
                     (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
        (setf smax
                (max smax
                     (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
       label10))
    (cond ((<= smin zero) (setf info -10))
          ((> n 0)
           (setf scond (/ (max smin smlnum) (min smax bignum))))
          (t (setf scond one)))))
  (cond
   ((= info 0)
    (cond
     ((< ldb$
         (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -12))
     ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -14)))))))
(cond
 ((/= info 0) (xerbla "CPOSVX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (equil
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (cpoequ n a lda s scond amax infequ)
    (declare (ignore var-0 var-1 var-2 var-3))
    (setf scond var-4)
    (setf amax var-5)
    (setf infequ var-6))
  (cond
   ((= infequ 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (claqhe uplo n a lda s scond amax equed)
      (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
```

```
      (setf uplo var-0)
      (setf equed var-7))
    (setf rcequ
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame equed "Y")
              (declare (ignore var-1))
              (when var-0 (setf equed var-0))
              ret-val))))))
(cond
 (rcequ
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                  (* (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)))
         label20))
     label30))))
(cond
 ((or nofact equil)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy uplo n n a lda af ldaf)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
    (setf uplo var-0))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (cpotrf uplo n af ldaf info)
    (declare (ignore var-2))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf ldaf var-3))
    (when var-4 (setf info var-4)))
  (cond ((> info 0) (setf rcond zero) (go end_label)))))
(setf anorm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (clanhe "1" uplo n a lda rwork)
          (declare (ignore var-0 var-2 var-3 var-4 var-5))
          (setf uplo var-1)
          ret-val))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (cpocon uplo n af ldaf anorm rcond work rwork info)
  (declare (ignore var-2 var-4 var-6 var-7))
  (setf uplo var-0)
  (setf n var-1)
```

```
  (setf ldaf var-3)
  (setf rcond var-5)
  (setf info var-8))
(clacpy "Full" n nrhs b ldb$ x ldx)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cpotrs uplo n nrhs af ldaf x ldx info)
  (declare (ignore var-3 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldaf var-4))
  (when var-6 (setf ldx var-6))
  (when var-7 (setf info var-7)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15)
    (cporfs uplo n nrhs a lda af ldaf b ldb$ x ldx ferr berr work rwork
     info)
  (declare
   (ignore var-2 var-3 var-5 var-7 var-8 var-9 var-10 var-11 var-12
    var-13 var-14))
  (setf uplo var-0)
  (setf n var-1)
  (setf lda var-4)
  (setf ldaf var-6)
  (setf info var-15))
(cond
 (rcequ
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                               x-%offset%)
                  (* (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                     (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                    x-%offset%)))
         label40))
     label50))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
              (/ (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                 scond))
     label60))))
(if (< rcond (slamch "Epsilon"))
    (setf info (f2cl-lib:int-add n 1)))
```

```
        (go end_label)
       end_label
        (return
         (values fact uplo n nrhs nil lda nil ldaf equed nil nil nil nil ldx
                 rcond nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cposvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (string) (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::fact
                                              fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldaf
                                              fortran-to-lisp::equed nil nil
                                              nil nil fortran-to-lisp::ldx
                                              fortran-to-lisp::rcond nil nil
                                              nil nil fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::cporfs
                                              fortran-to-lisp::cpocon
                                              fortran-to-lisp::clanhe
```

```
                                        fortran-to-lisp::clacpy
                                        fortran-to-lisp::claqhe
                                        fortran-to-lisp::cpoequ))))
```

———-

68.3 cposvxx

68.3.1 Axiom unit tests

— **cposvxx.input** —

```
)set break resume
)sys rm -f cposvxx.output
)spool cposvxx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

68.3.2 Axiom help page [Demm05]

— **cposvxx.help** —

```
======================================================================
cposvxx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cposvxx (FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, EQUED, S, B,
           LDB, X, LDX, RCOND, RPVGRW, BERR, N_ERR_BNDS, ERR_BNDS_NORM,
           ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, RWORK, INFO)
            CPOSVXX computes the solution to system of linear equations A * X
```

```
          = B for PO matrices

Function/Subroutine Documentation
   subroutine cposvxx (character FACT, character UPLO, integer N, integer
       NRHS, complex, dimension( lda, * ) A, integer LDA, complex, dimension(
       ldaf, * ) AF, integer LDAF, character EQUED, real, dimension( * ) S,
       complex, dimension( ldb, * ) B, integer LDB, complex, dimension( ldx, *
       ) X, integer LDX, real RCOND, real RPVGRW, real, dimension( * ) BERR,
       integer N_ERR_BNDS, real, dimension( nrhs, * ) ERR_BNDS_NORM, real,
       dimension( nrhs, * ) ERR_BNDS_COMP, integer NPARAMS, real, dimension( *
       ) PARAMS, complex, dimension( * ) WORK, real, dimension( * ) RWORK,
       integer INFO)
        CPOSVXX computes the solution to system of linear equations A * X = B
       for PO matrices

Purpose:
   CPOSVXX uses the Cholesky factorization A = U**T*U or A = L*L**T
   to compute the solution to a complex system of linear equations
   A * X = B, where A is an N-by-N symmetric positive definite matrix
   and X and B are N-by-NRHS matrices.

   If requested, both normwise and maximum componentwise error bounds
   are returned. CPOSVXX will return a solution with a tiny
   guaranteed error (O(eps) where eps is the working machine
   precision) unless the matrix is very ill-conditioned, in which
   case a warning is returned. Relevant condition numbers also are
   calculated and returned.

   CPOSVXX accepts user-provided factorizations and equilibration
   factors; see the definitions of the FACT and EQUED options.
   Solving with refinement and using a factorization from a previous
   CPOSVXX call will also produce a solution with either O(eps)
   errors or warnings, but we cannot make that claim for general
   user-provided factorizations and equilibration factors if they
   differ from what CPOSVXX would itself produce.

Description:

   The following steps are performed:

   1. If FACT = 'E', real scaling factors are computed to equilibrate
      the system:

         diag(S)*A*diag(S)     *inv(diag(S))*X = diag(S)*B

      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(S)*A*diag(S) and B by diag(S)*B.

   2. If FACT = 'N' or 'E', the Cholesky decomposition is used to
```

```
      factor the matrix A (after equilibration if FACT = 'E') as
         A = U**T* U,  if UPLO = 'U', or
         A = L * L**T,  if UPLO = 'L',
      where U is an upper triangular matrix and L is a lower triangular
      matrix.

   3. If the leading i-by-i principal minor is not positive definite,
      then the routine returns with INFO = i. Otherwise, the factored
      form of A is used to estimate the condition number of the matrix
      A (see argument RCOND).  If the reciprocal of the condition number
      is less than machine precision, the routine still goes on to solve
      for X and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. By default (unless PARAMS(LA_LINRX_ITREF_I) is set to zero),
      the routine will use iterative refinement to try to get a small
      error and error bounds.  Refinement calculates the residual to at
      least twice the working precision.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(S) so that it solves the original system before equilibration.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AF contains the factored form of A.
      If EQUED is not 'N', the matrix A has been
      equilibrated with scaling factors given by S.
      A and AF are not modified.
   = 'N':  The matrix A will be copied to AF and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
      copied to AF and factored.

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
```

```
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the symmetric matrix A, except if FACT = 'F' and EQUED =
  'Y', then A must contain the equilibrated matrix
  diag(S)*A*diag(S).  If UPLO = 'U', the leading N-by-N upper
  triangular part of A contains the upper triangular part of the
  matrix A, and the strictly lower triangular part of A is not
  referenced.  If UPLO = 'L', the leading N-by-N lower triangular
  part of A contains the lower triangular part of the matrix A, and
  the strictly upper triangular part of A is not referenced.  A is
  not modified if FACT = 'F' or 'N', or if FACT = 'E' and EQUED =
  'N' on exit.

  On exit, if FACT = 'E' and EQUED = 'Y', A is overwritten by
  diag(S)*A*diag(S).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the triangular factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T, in the same storage
  format as A.  If EQUED .ne. 'N', then AF is the factored
  form of the equilibrated matrix diag(S)*A*diag(S).

  If FACT = 'N', then AF is an output argument and on exit
  returns the triangular factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T of the original
  matrix A.

  If FACT = 'E', then AF is an output argument and on exit
  returns the triangular factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T of the equilibrated
  matrix A (see the description of A for the form of the
  equilibrated matrix).

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).
```

```
EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'Y':  Both row and column equilibration, i.e., A has been
          replaced by diag(S) * A * diag(S).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

S
  S is REAL array, dimension (N)
  The row scale factors for A.  If EQUED = 'Y', A is multiplied on
  the left and right by diag(S).  S is an input argument if FACT =
  'F'; otherwise, S is an output argument.  If FACT = 'F' and EQUED
  = 'Y', each element of S must be positive.  If S is output, each
  element of S is a power of the radix. If S is input, each element
  of S should be a power of the radix to ensure a reliable solution
  and error estimates. Scaling by powers of the radix does not cause
  rounding errors unless the result underflows or overflows.
  Rounding errors during scaling lead to refining with a matrix that
  is not equivalent to the input matrix, producing error estimates
  that may not be reliable.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if EQUED = 'Y', B is overwritten by diag(S)*B;

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  If INFO = 0, the N-by-NRHS solution matrix X to the original
  system of equations.  Note that A and B are modified on exit if
  EQUED .ne. 'N', and the solution to the equilibrated system is
  inv(diag(S))*X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
```

```
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

RPVGRW
  RPVGRW is REAL
  Reciprocal pivot growth.  On exit, this contains the reciprocal
  pivot growth factor norm(A)/norm(U). The "max absolute element"
  norm is used.  If this is much less than 1, then the stability of
  the LU factorization of the (equilibrated) matrix A could be poor.
  This also means that the solution X, estimated condition numbers,
  and error bounds could be unreliable. If factorization fails with
  0<INFO<=N, then this contains the reciprocal pivot growth factor
  for the leading INFO columns of A.

BERR
  BERR is REAL array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
         max_j (abs(XTRUE(j,i) - X(j,i)))
          ------------------------------
                max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:
```

```
  err = 1 "Trust/don't trust" boolean. Trust the answer if the
    reciprocal condition number is less than the threshold
    sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
    almost certainly within a factor of 10 of the true error
    so long as the next entry is greater than the threshold
    sqrt(n) * slamch('Epsilon'). This error bound should only
    be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
    reciprocal condition number.  Compared with the threshold
    sqrt(n) * slamch('Epsilon') to determine if the error
    estimate is "guaranteed". These reciprocal condition
    numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
    appropriately scaled matrix Z.
    Let Z = S*A, where S scales each row by a power of the
    radix so all absolute row sums of Z are approximately 1.

ERR_BNDS_COMP
  ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                 abs(XTRUE(j,i) - X(j,i))
           max_j ----------------------
                       abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
```

```
      so long as the next entry is greater than the threshold
      sqrt(n) * slamch('Epsilon'). This error bound should only
      be trusted if the previous boolean is true.

   err = 3  Reciprocal condition number: Estimated componentwise
      reciprocal condition number.  Compared with the threshold
      sqrt(n) * slamch('Epsilon') to determine if the error
      estimate is "guaranteed". These reciprocal condition
      numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
      appropriately scaled matrix Z.
      Let Z = S*(A*diag(x)), where x is the solution for the
      current right-hand side and S scales each row of
      A*diag(x) by a power of the radix so all absolute row
      sums of Z are approximately 1.

 NPARAMS
   NPARAMS is INTEGER
   Specifies the number of parameters set in PARAMS.  If .LE. 0, the
   PARAMS array is never referenced and default values are used.

 PARAMS
   PARAMS is REAL array, dimension NPARAMS
   Specifies algorithm parameters.  If an entry is .LT. 0.0, then
   that entry will be filled with default value used for that
   parameter.  Only positions up to NPARAMS are accessed; defaults
   are used for higher-numbered parameters.

   PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
   refinement or not.
   Default: 1.0
   = 0.0 : No refinement is performed, and no error bounds are computed.
   = 1.0 : Use the double-precision refinement algorithm,
           possibly with doubled-single computations if the
           compilation environment does not support DOUBLE PRECISION.
   (other values are reserved for future use)

   PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
   computations allowed for refinement.
   Default: 10
   Aggressive: Set to 100 to permit convergence using approximate
               factorizations or factorizations other than LU. If
               the factorization uses a technique other than
               Gaussian elimination, the guarantees in
               err_bnds_norm and err_bnds_comp may no longer be trustworthy.

   PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
   will attempt to find a solution with small componentwise
   relative error in the double-precision algorithm.  Positive
   is true, 0.0 is false.
   Default: 1.0 (attempt componentwise convergence)
```

```
 WORK
   WORK is COMPLEX array, dimension (2*N)

 RWORK
   RWORK is REAL array, dimension (2*N)

 INFO
   INFO is INTEGER
   = 0:  Successful exit. The solution to every right-hand side is
   guaranteed.
   < 0:  If INFO = -i, the i-th argument had an illegal value
   > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
      has been completed, but the factor U is exactly singular, so
      the solution and error bounds could not be computed. RCOND = 0
      is returned.
   = N+J: The solution corresponding to the Jth right-hand side is
      not guaranteed. The solutions corresponding to other right-
      hand sides K with K > J may not be guaranteed as well, but
      only the first such right-hand side is reported. If a small
      componentwise error is not requested (PARAMS(3) = 0.0) then
      the Jth right-hand side is the first with a normwise error
      bound that is not guaranteed (the smallest J such
      that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
      the Jth right-hand side is the first with either a normwise or
      componentwise error bound that is not guaranteed (the smallest
      J such that either ERR_BNDS_NORM(J,1) = 0.0 or
      ERR_BNDS_COMP(J,1) = 0.0). See the definition of
      ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
      about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

———-

68.3.3 fortran code

— cposvxx.f —

```
*  =====================================================================
      SUBROUTINE CPOSVXX( FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, EQUED,
     $                    S, B, LDB, X, LDX, RCOND, RPVGRW, BERR,
     $                    N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP,
     $                    NPARAMS, PARAMS, WORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
```

```
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      REAL               RCOND, RPVGRW
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   WORK( * ), X( LDX, * )
      REAL               S( * ), PARAMS( * ), BERR( * ), RWORK( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
*     ..
*     .. Local Scalars ..
      LOGICAL            EQUIL, NOFACT, RCEQU
      INTEGER            INFEQU, J
      REAL               AMAX, BIGNUM, SMIN, SMAX, SCOND, SMLNUM
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, SLAMCH, CLA_PORPVGRW
      LOGICAL            LSAME
      REAL               SLAMCH, CLA_PORPVGRW
*     ..
*     .. External Subroutines ..
      EXTERNAL           CPOCON, CPOEQUB, CPOTRF, CPOTRS, CLACPY,
     $                   CLAQHE, XERBLA, CLASCL2, CPORFSX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
```

```
      EQUIL = LSAME( FACT, 'E' )
      SMLNUM = SLAMCH( 'Safe minimum' )
      BIGNUM = ONE / SMLNUM
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         RCEQU = .FALSE.
      ELSE
         RCEQU = LSAME( EQUED, 'Y' )
      ENDIF
*
*     Default is failure.  If an input parameter is wrong or
*     factorization fails, make everything look horrible.  Only the
*     pivot growth is set here, the rest is initialized in CPORFSX.
*
      RPVGRW = ZERO
*
*     Test the input parameters.  PARAMS is not tested until CPORFSX.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.
     $     LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.LSAME( UPLO, 'U' ) .AND.
     $         .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $        ( RCEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -9
      ELSE
         IF ( RCEQU ) THEN
            SMIN = BIGNUM
            SMAX = ZERO
            DO 10 J = 1, N
               SMIN = MIN( SMIN, S( J ) )
               SMAX = MAX( SMAX, S( J ) )
 10         CONTINUE
            IF( SMIN.LE.ZERO ) THEN
               INFO = -10
            ELSE IF( N.GT.0 ) THEN
               SCOND = MAX( SMIN, SMLNUM ) / MIN( SMAX, BIGNUM )
            ELSE
               SCOND = ONE
            END IF
```

```
         END IF
         IF( INFO.EQ.0 ) THEN
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -12
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -14
            END IF
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CPOSVXX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*     Compute row and column scalings to equilibrate the matrix A.
*
         CALL CPOEQUB( N, A, LDA, S, SCOND, AMAX, INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*     Equilibrate the matrix.
*
            CALL CLAQHE( UPLO, N, A, LDA, S, SCOND, AMAX, EQUED )
            RCEQU = LSAME( EQUED, 'Y' )
         END IF
      END IF
*
*     Scale the right-hand side.
*
      IF( RCEQU ) CALL CLASCL2( N, NRHS, S, B, LDB )
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the Cholesky factorization of A.
*
         CALL CLACPY( UPLO, N, N, A, LDA, AF, LDAF )
         CALL CPOTRF( UPLO, N, AF, LDAF, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 ) THEN
*
*           Pivot in column INFO is exactly 0
*           Compute the reciprocal pivot growth factor of the
*           leading rank-deficient INFO columns of A.
*
            RPVGRW = CLA_PORPVGRW( UPLO, N, A, LDA, AF, LDAF, RWORK )
            RETURN
```

```
         END IF
      END IF
*
*     Compute the reciprocal pivot growth factor RPVGRW.
*
      RPVGRW = CLA_PORPVGRW( UPLO, N, A, LDA, AF, LDAF, RWORK )
*
*     Compute the solution matrix X.
*
      CALL CLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL CPOTRS( UPLO, N, NRHS, AF, LDAF, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL CPORFSX( UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF,
     $     S, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS, ERR_BNDS_NORM,
     $     ERR_BNDS_COMP,  NPARAMS, PARAMS, WORK, RWORK, INFO )

*
*     Scale solutions.
*
      IF ( RCEQU ) THEN
         CALL CLASCL2( N, NRHS, S, X, LDX )
      END IF
*
      RETURN
*
*     End of CPOSVXX
*
      END
```

———-

68.3.4 lisp code

— **cposvxx.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun cposvxx
         (fact uplo n nrhs a lda af ldaf equed s b ldb$ x ldx rcond rpvgrw berr
          n_err_bnds err_bnds_norm err_bnds_comp nparams params work rwork
          info)
    (declare (type (single-float) rpvgrw rcond)
             (type (array single-float (*)) rwork params err_bnds_comp
```

```
            err_bnds_norm berr s)
           (type (array f2cl-lib:complex8 (*)) work x b af a)
           (type (f2cl-lib:integer4) info nparams n_err_bnds ldx ldb$ ldaf
            lda nrhs n)
           (type (string *) equed uplo fact))
  (f2cl-lib:with-multi-array-data
      ((fact character fact-%data% fact-%offset%)
       (uplo character uplo-%data% uplo-%offset%)
       (equed character equed-%data% equed-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (af f2cl-lib:complex8 af-%data% af-%offset%)
       (b f2cl-lib:complex8 b-%data% b-%offset%)
       (x f2cl-lib:complex8 x-%data% x-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%)
       (s single-float s-%data% s-%offset%)
       (berr single-float berr-%data% berr-%offset%)
       (err_bnds_norm single-float err_bnds_norm-%data%
        err_bnds_norm-%offset%)
       (err_bnds_comp single-float err_bnds_comp-%data%
        err_bnds_comp-%offset%)
       (params single-float params-%data% params-%offset%)
       (rwork single-float rwork-%data% rwork-%offset%))
    (prog ((amax 0.0) (bignum 0.0) (smin 0.0) (smax 0.0) (scond 0.0)
           (smlnum 0.0) (infequ 0) (j 0) (equil nil) (nofact nil)
           (rcequ nil))
      (declare (type (single-float) amax bignum smin smax scond smlnum)
               (type (f2cl-lib:integer4) infequ j)
               (type f2cl-lib:logical equil nofact rcequ))
      (setf info 0)
      (setf nofact
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame fact "N")
                (declare (ignore var-1))
                (when var-0 (setf fact var-0))
                ret-val))
      (setf equil
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame fact "E")
                (declare (ignore var-1))
                (when var-0 (setf fact var-0))
                ret-val))
      (setf smlnum (slamch "Safe minimum"))
      (setf bignum (/ one smlnum))
      (cond
       ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
        (setf rcequ f2cl-lib:%false%))
       (t
        (setf rcequ
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame equed "Y")
```

```
            (declare (ignore var-1))
            (when var-0 (setf equed var-0))
            ret-val))))
(setf rpvgrw zero)
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame fact "F")
     (declare (ignore var-1))
     (when var-0 (setf fact var-0))
     ret-val)
   (not
    (or rcequ
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame equed "N")
          (declare (ignore var-1))
          (when var-0 (setf equed var-0))
          ret-val))))
  (setf info -9))
 (t
  (cond
   (rcequ (setf smin bignum) (setf smax zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                    ((> j n) nil)
          (tagbody
            (setf smin
                    (min smin
                         (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
            (setf smax
                    (max smax
                         (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
           label10))
        (cond ((<= smin zero) (setf info -10))
              ((> n 0)
               (setf scond (/ (max smin smlnum) (min smax bignum))))
              (t (setf scond one)))))
      (cond
       ((= info 0)
        (cond
         ((< ldb$
             (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -12))
         ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -14)))))))
    (cond
     ((/= info 0) (xerbla "CPOSVXX" (f2cl-lib:int-sub info))
      (go end_label)))
    (cond
     (equil
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (cpoequb n a lda s scond amax infequ)
        (declare (ignore var-0 var-1 var-2 var-3))
        (setf scond var-4)
        (setf amax var-5)
        (setf infequ var-6))
      (cond
       ((= infequ 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (claqhe uplo n a lda s scond amax equed)
          (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
          (setf uplo var-0)
          (setf equed var-7))
        (setf rcequ
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame equed "Y")
                  (declare (ignore var-1))
                  (when var-0 (setf equed var-0))
                  ret-val))))))
    (if rcequ
        (clascl2 n nrhs s b ldb$))
    (cond
     ((or nofact equil)
```

```
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (clacpy uplo n n a lda af ldaf)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
    (setf uplo var-0))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (cpotrf uplo n af ldaf info)
    (declare (ignore var-2))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf ldaf var-3))
    (when var-4 (setf info var-4)))
  (cond
   ((> info 0)
    (setf rpvgrw
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (cla_porpvgrw uplo n a lda af ldaf rwork)
              (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
              (setf uplo var-0)
              ret-val))
    (go end_label)))))
(setf rpvgrw
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (cla_porpvgrw uplo n a lda af ldaf rwork)
          (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
          (setf uplo var-0)
          ret-val))
(clacpy "Full" n nrhs b ldb$ x ldx)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cpotrs uplo n nrhs af ldaf x ldx info)
  (declare (ignore var-3 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldaf var-4))
  (when var-6 (setf ldx var-6))
  (when var-7 (setf info var-7)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19
     var-20 var-21 var-22)
    (cporfsx uplo equed n nrhs a lda af ldaf s b ldb$ x ldx rcond berr
     n_err_bnds err_bnds_norm err_bnds_comp nparams params work rwork
     info)
  (declare
   (ignore var-4 var-6 var-8 var-9 var-11 var-14 var-16 var-17 var-19
    var-20 var-21))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf equed var-1))
```

```
          (when var-2 (setf n var-2))
          (when var-3 (setf nrhs var-3))
          (when var-5 (setf lda var-5))
          (when var-7 (setf ldaf var-7))
          (when var-10 (setf ldb$ var-10))
          (when var-12 (setf ldx var-12))
          (when var-13 (setf rcond var-13))
          (when var-15 (setf n_err_bnds var-15))
          (when var-18 (setf nparams var-18))
          (when var-22 (setf info var-22)))
        (cond (rcequ (clascl2 n nrhs s x ldx)))
        (go end_label)
       end_label
        (return
         (values fact uplo n nrhs nil lda nil ldaf equed nil nil ldb$ nil ldx
                 rcond rpvgrw nil n_err_bnds nil nil nparams nil nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cposvxx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (string) (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array single-float (*))
```

```
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::fact
  fortran-to-lisp::uplo
  fortran-to-lisp::n
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldaf
  fortran-to-lisp::equed nil nil
  fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond
  fortran-to-lisp::rpvgrw nil
  fortran-to-lisp::n_err_bnds nil
  nil fortran-to-lisp::nparams nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::cla_porpvgrw
  fortran-to-lisp::clacpy
  fortran-to-lisp::clascl2
  fortran-to-lisp::claqhe
  fortran-to-lisp::cpoequb))))
```

Chapter 69

LAPACK - Positive Definite Matrix, Linear Solve, Complex16

69.1 zcposv

69.1.1 Axiom unit tests

— **zcposv.input** —

```
)set break resume
)sys rm -f zcposv.output
)spool zcposv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

69.1.2 Axiom help page

— **zcposv.help** —

```
==================================================================
```

```
zcposv examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zcposv (UPLO, N, NRHS, A, LDA, B, LDB, X, LDX, WORK, SWORK,
           RWORK, ITER, INFO)
            ZCPOSV computes the solution to system of linear equations A * X =
           B for PO matrices

Function/Subroutine Documentation
   subroutine zcposv (character UPLO, integer N, integer NRHS, complex*16,
       dimension( lda, * ) A, integer LDA, complex*16, dimension( ldb, * ) B,
       integer LDB, complex*16, dimension( ldx, * ) X, integer LDX,
       complex*16, dimension( n, * ) WORK, complex, dimension( * ) SWORK,
       double precision, dimension( * ) RWORK, integer ITER, integer INFO)
        ZCPOSV computes the solution to system of linear equations A * X = B
       for PO matrices

Purpose:
   ZCPOSV computes the solution to a complex system of linear equations
      A * X = B,
   where A is an N-by-N Hermitian positive definite matrix and X and B
   are N-by-NRHS matrices.

   ZCPOSV first attempts to factorize the matrix in COMPLEX and use this
   factorization within an iterative refinement procedure to produce a
   solution with COMPLEX*16 normwise backward error quality (see below).
   If the approach fails the method switches to a COMPLEX*16
   factorization and solve.

   The iterative refinement is not going to be a winning strategy if
   the ratio COMPLEX performance over COMPLEX*16 performance is too
   small. A reasonable strategy should take the number of right-hand
   sides and the size of the matrix into account. This might be done
   with a call to ILAENV in the future. Up to now, we always try
   iterative refinement.

   The iterative refinement process is stopped if
      ITER > ITERMAX
   or for all the RHS we have:
      RNRM < SQRT(N)*XNRM*ANRM*EPS*BWDMAX
   where
      o ITER is the number of the current iteration in the iterative
        refinement process
      o RNRM is the infinity-norm of the residual
```

```
      o XNRM is the infinity-norm of the solution
      o ANRM is the infinity-operator-norm of the matrix A
      o EPS is the machine epsilon returned by DLAMCH('Epsilon')
  The value ITERMAX and BWDMAX are fixed to 30 and 1.0D+00 respectively.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is COMPLEX*16 array,
   dimension (LDA,N)
   On entry, the Hermitian matrix A. If UPLO = 'U', the leading
   N-by-N upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading N-by-N lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

   Note that the imaginary parts of the diagonal
   elements need not be set and are assumed to be zero.

   On exit, if iterative refinement has been successfully used
   (INFO.EQ.0 and ITER.GE.0, see description below), then A is
   unchanged, if double precision factorization has been used
   (INFO.EQ.0 and ITER.LT.0, see description below), then the
   array A contains the factor U or L from the Cholesky
   factorization A = U**H*U or A = L*L**H.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 B
   B is COMPLEX*16 array, dimension (LDB,NRHS)
   The N-by-NRHS right hand side matrix B.
```

```
LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  If INFO = 0, the N-by-NRHS solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

WORK
  WORK is COMPLEX*16 array, dimension (N*NRHS)
  This array is used to hold the residual vectors.

SWORK
  SWORK is COMPLEX array, dimension (N*(N+NRHS))
  This array is used to use the single precision matrix and the
  right-hand sides or solutions in single precision.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (N)

ITER
  ITER is INTEGER
  < 0: iterative refinement has failed, COMPLEX*16
       factorization has been performed
  -1 : the routine fell back to full precision for
       implementation- or machine-specific reasons
  -2 : narrowing the precision induced an overflow,
       the routine fell back to full precision
  -3 : failure of CPOTRF
  -31: stop the iterative refinement after the 30th iterations
  > 0: iterative refinement has been sucessfully used.
       Returns the number of iterations

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, the leading minor of order i of
     (COMPLEX*16) A is not positive definite, so the
     factorization could not be completed, and the solution
     has not been computed.
```

———-

69.1.3 fortran code

— zcposv.f —

```
*  =====================================================================
      SUBROUTINE ZCPOSV( UPLO, N, NRHS, A, LDA, B, LDB, X, LDX, WORK,
     $                   SWORK, RWORK, ITER, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, ITER, LDA, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   RWORK( * )
      COMPLEX            SWORK( * )
      COMPLEX*16         A( LDA, * ), B( LDB, * ), WORK( N, * ),
     $                   X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      LOGICAL            DOITREF
      PARAMETER          ( DOITREF = .TRUE. )
*
      INTEGER            ITERMAX
      PARAMETER          ( ITERMAX = 30 )
*
      DOUBLE PRECISION   BWDMAX
      PARAMETER          ( BWDMAX = 1.0E+00 )
*
      COMPLEX*16         NEGONE, ONE
      PARAMETER          ( NEGONE = ( -1.0D+00, 0.0D+00 ),
     $                   ONE = ( 1.0D+00, 0.0D+00 ) )
*
*     .. Local Scalars ..
      INTEGER            I, IITER, PTSA, PTSX
      DOUBLE PRECISION   ANRM, CTE, EPS, RNRM, XNRM
      COMPLEX*16         ZDUM
*
*     .. External Subroutines ..
      EXTERNAL           ZAXPY, ZHEMM, ZLACPY, ZLAT2C, ZLAG2C, CLAG2Z,
     $                   CPOTRF, CPOTRS, XERBLA
*     ..
```

```
*     .. External Functions ..
      INTEGER            IZAMAX
      DOUBLE PRECISION   DLAMCH, ZLANHE
      LOGICAL            LSAME
      EXTERNAL           IZAMAX, DLAMCH, ZLANHE, LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, MAX, SQRT
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      ITER = 0
*
*     Test the input parameters.
*
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZCPOSV', -INFO )
         RETURN
      END IF
*
*     Quick return if (N.EQ.0).
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Skip single precision iterative refinement if a priori slower
*     than double precision factorization.
*
      IF( .NOT.DOITREF ) THEN
         ITER = -1
         GO TO 40
```

```
      END IF
*
*     Compute some constants.
*
      ANRM = ZLANHE( 'I', UPLO, N, A, LDA, RWORK )
      EPS = DLAMCH( 'Epsilon' )
      CTE = ANRM*EPS*SQRT( DBLE( N ) )*BWDMAX
*
*     Set the indices PTSA, PTSX for referencing SA and SX in SWORK.
*
      PTSA = 1
      PTSX = PTSA + N*N
*
*     Convert B from double precision to single precision and store the
*     result in SX.
*
      CALL ZLAG2C( N, NRHS, B, LDB, SWORK( PTSX ), N, INFO )
*
      IF( INFO.NE.0 ) THEN
         ITER = -2
         GO TO 40
      END IF
*
*     Convert A from double precision to single precision and store the
*     result in SA.
*
      CALL ZLAT2C( UPLO, N, A, LDA, SWORK( PTSA ), N, INFO )
*
      IF( INFO.NE.0 ) THEN
         ITER = -2
         GO TO 40
      END IF
*
*     Compute the Cholesky factorization of SA.
*
      CALL CPOTRF( UPLO, N, SWORK( PTSA ), N, INFO )
*
      IF( INFO.NE.0 ) THEN
         ITER = -3
         GO TO 40
      END IF
*
*     Solve the system SA*SX = SB.
*
      CALL CPOTRS( UPLO, N, NRHS, SWORK( PTSA ), N, SWORK( PTSX ), N,
     $             INFO )
*
*     Convert SX back to COMPLEX*16
*
      CALL CLAG2Z( N, NRHS, SWORK( PTSX ), N, X, LDX, INFO )
```

```
*
*     Compute R = B - AX (R is WORK).
*
      CALL ZLACPY( 'All', N, NRHS, B, LDB, WORK, N )
*
      CALL ZHEMM( 'Left', UPLO, N, NRHS, NEGONE, A, LDA, X, LDX, ONE,
     $            WORK, N )
*
*     Check whether the NRHS normwise backward errors satisfy the
*     stopping criterion. If yes, set ITER=0 and return.
*
      DO I = 1, NRHS
         XNRM = CABS1( X( IZAMAX( N, X( 1, I ), 1 ), I ) )
         RNRM = CABS1( WORK( IZAMAX( N, WORK( 1, I ), 1 ), I ) )
         IF( RNRM.GT.XNRM*CTE )
     $      GO TO 10
      END DO
*
*     If we are here, the NRHS normwise backward errors satisfy the
*     stopping criterion. We are good to exit.
*
      ITER = 0
      RETURN
*
   10 CONTINUE
*
      DO 30 IITER = 1, ITERMAX
*
*        Convert R (in WORK) from double precision to single precision
*        and store the result in SX.
*
         CALL ZLAG2C( N, NRHS, WORK, N, SWORK( PTSX ), N, INFO )
*
         IF( INFO.NE.0 ) THEN
            ITER = -2
            GO TO 40
         END IF
*
*        Solve the system SA*SX = SR.
*
         CALL CPOTRS( UPLO, N, NRHS, SWORK( PTSA ), N, SWORK( PTSX ), N,
     $                INFO )
*
*        Convert SX back to double precision and update the current
*        iterate.
*
         CALL CLAG2Z( N, NRHS, SWORK( PTSX ), N, WORK, N, INFO )
*
         DO I = 1, NRHS
            CALL ZAXPY( N, ONE, WORK( 1, I ), 1, X( 1, I ), 1 )
```

```
         END DO
*
*        Compute R = B - AX (R is WORK).
*
         CALL ZLACPY( 'All', N, NRHS, B, LDB, WORK, N )
*
         CALL ZHEMM( 'L', UPLO, N, NRHS, NEGONE, A, LDA, X, LDX, ONE,
     $               WORK, N )
*
*        Check whether the NRHS normwise backward errors satisfy the
*        stopping criterion. If yes, set ITER=IITER>0 and return.
*
         DO I = 1, NRHS
            XNRM = CABS1( X( IZAMAX( N, X( 1, I ), 1 ), I ) )
            RNRM = CABS1( WORK( IZAMAX( N, WORK( 1, I ), 1 ), I ) )
            IF( RNRM.GT.XNRM*CTE )
     $         GO TO 20
         END DO
*
*        If we are here, the NRHS normwise backward errors satisfy the
*        stopping criterion, we are good to exit.
*
         ITER = IITER
*
         RETURN
*
   20    CONTINUE
*
   30 CONTINUE
*
*     If we are at this place of the code, this is because we have
*     performed ITER=ITERMAX iterations and never satisified the
*     stopping criterion, set up the ITER flag accordingly and follow
*     up on double precision routine.
*
      ITER = -ITERMAX - 1
*
   40 CONTINUE
*
*     Single-precision iterative refinement failed to converge to a
*     satisfactory solution, so we resort to double precision.
*
      CALL ZPOTRF( UPLO, N, A, LDA, INFO )
*
      IF( INFO.NE.0 )
     $   RETURN
*
      CALL ZLACPY( 'All', N, NRHS, B, LDB, X, LDX )
      CALL ZPOTRS( UPLO, N, NRHS, A, LDA, X, LDX, INFO )
*
```

```
      RETURN
*
*     End of ZCPOSV.
*
      END
```

———-

69.1.4 lisp code

— **zcposv.lisp** —

```
(let* ((doitref f2cl-lib:%true%)
       (itermax 30)
       (bwdmax (coerce 1.0 'double-float))
       (negone (f2cl-lib:cmplx (- 1.0d0) 0.0d0))
       (one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type f2cl-lib:logical doitref)
           (type (f2cl-lib:integer4 30 30) itermax)
           (type (double-float) bwdmax)
           (type (f2cl-lib:complex16) negone)
           (type (f2cl-lib:complex16) one)
           (ignorable doitref itermax bwdmax negone one))
  (defun zcposv (uplo n nrhs a lda b ldb$ x ldx work swork rwork iter info)
    (declare (type (array double-float (*)) rwork)
             (type (array f2cl-lib:complex8 (*)) swork)
             (type (array f2cl-lib:complex16 (*)) work x b a)
             (type (f2cl-lib:integer4) info iter ldx ldb$ lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (x f2cl-lib:complex16 x-%data% x-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (swork f2cl-lib:complex8 swork-%data% swork-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
          cabs1))
        (prog ((zdum #C(0.0d0 0.0d0)) (anrm 0.0d0) (cte 0.0d0) (eps 0.0d0)
               (rnrm 0.0d0) (xnrm 0.0d0) (i 0) (iiter 0) (ptsa 0) (ptsx 0))
          (declare (type (f2cl-lib:complex16) zdum)
                   (type (double-float) anrm cte eps rnrm xnrm)
                   (type (f2cl-lib:integer4) i iiter ptsa ptsx))
          (setf info 0)
```

```
(setf iter 0)
(cond
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9)))
(cond
 ((/= info 0) (xerbla "ZCPOSV" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(cond ((not doitref) (setf iter -1) (go label40)))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlanhe "I" uplo n a lda rwork)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf uplo var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf eps (dlamch "Epsilon"))
(setf cte (* anrm eps (f2cl-lib:fsqrt (f2cl-lib:dble n)) bwdmax))
(setf ptsa 1)
(setf ptsx (f2cl-lib:int-add ptsa (f2cl-lib:int-mul n n)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlag2c n nrhs b ldb$
     (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsx)
                           ((1 *)) swork-%offset%)
     n info)
  (declare (ignore var-2 var-4))
  (when var-0 (setf n var-0))
  (when var-1 (setf nrhs var-1))
```

```
  (when var-3 (setf ldb$ var-3))
  (when var-5 (setf n var-5))
  (when var-6 (setf info var-6)))
(cond ((/= info 0) (setf iter -2) (go label40)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlat2c uplo n a lda
     (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsa)
                           ((1 *)) swork-%offset%)
     n info)
  (declare (ignore var-2 var-4))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf lda var-3))
  (when var-5 (setf n var-5))
  (when var-6 (setf info var-6)))
(cond ((/= info 0) (setf iter -2) (go label40)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (cpotrf uplo n
     (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsa)
                           ((1 *)) swork-%offset%)
     n info)
  (declare (ignore var-2))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf n var-3))
  (when var-4 (setf info var-4)))
(cond ((/= info 0) (setf iter -3) (go label40)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cpotrs uplo n nrhs
     (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsa)
                           ((1 *)) swork-%offset%)
     n
     (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsx)
                           ((1 *)) swork-%offset%)
     n info)
  (declare (ignore var-3 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf n var-4))
  (when var-6 (setf n var-6))
  (when var-7 (setf info var-7)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clag2z n nrhs
     (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsx)
                           ((1 *)) swork-%offset%)
     n x ldx info)
  (declare (ignore var-2 var-4))
  (when var-0 (setf n var-0))
```

```
  (when var-1 (setf nrhs var-1))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldx var-5))
  (when var-6 (setf info var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "All" n nrhs b ldb$ work n)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf n var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (zhemm "Left" uplo n nrhs negone a lda x ldx one work n)
  (declare (ignore var-0 var-5 var-7 var-10))
  (when var-1 (setf uplo var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf nrhs var-3))
  (when var-4 (setf negone var-4))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldx var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf n var-11)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nrhs) nil)
  (tagbody
    (setf xnrm
            (cabs1
             (f2cl-lib:fref x-%data%
                            ((multiple-value-bind
                                 (ret-val var-0 var-1 var-2)
                                 (izamax n
                                  (f2cl-lib:array-slice x
                                                        f2cl-lib:complex16
                                                        (1 i)
                                                        ((1 ldx)
                                                         (1 *)))
                                  1)
                                (declare (ignore var-1 var-2))
                                (when var-0 (setf n var-0))
                                ret-val)
                             i)
                            ((1 ldx) (1 *)) x-%offset%)))
    (setf rnrm
            (cabs1
             (f2cl-lib:fref work-%data%
                            ((multiple-value-bind
                                 (ret-val var-0 var-1 var-2)
                                 (izamax n
```

```
                                        (f2cl-lib:array-slice work
                                                              f2cl-lib:complex16
                                                              (1 i)
                                                              ((1 n)
                                                               (1 *)))
                                        1)
                                      (declare (ignore var-1 var-2))
                                      (when var-0 (setf n var-0))
                                      ret-val)
                                    i)
                                   ((1 n) (1 *)) work-%offset%)))
        (if (> rnrm (* xnrm cte))
            (go label10))
       label100000))
    (setf iter 0)
    (go end_label)
   label10
    (f2cl-lib:fdo (iiter 1 (f2cl-lib:int-add iiter 1))
                  ((> iiter itermax) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zlag2c n nrhs work n
             (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsx)
                                   ((1 *)) swork-%offset%)
             n info)
          (declare (ignore var-2 var-4))
          (when var-0 (setf n var-0))
          (when var-1 (setf nrhs var-1))
          (when var-3 (setf n var-3))
          (when var-5 (setf n var-5))
          (when var-6 (setf info var-6)))
        (cond ((/= info 0) (setf iter -2) (go label40)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (cpotrs uplo n nrhs
             (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsa)
                                   ((1 *)) swork-%offset%)
             n
             (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsx)
                                   ((1 *)) swork-%offset%)
             n info)
          (declare (ignore var-3 var-5))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf n var-4))
          (when var-6 (setf n var-6))
          (when var-7 (setf info var-7)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (clag2z n nrhs
```

```
     (f2cl-lib:array-slice swork-%data% f2cl-lib:complex8 (ptsx)
                           ((1 *)) swork-%offset%)
     n work n info)
  (declare (ignore var-2 var-4))
  (when var-0 (setf n var-0))
  (when var-1 (setf nrhs var-1))
  (when var-3 (setf n var-3))
  (when var-5 (setf n var-5))
  (when var-6 (setf info var-6)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nrhs) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (zaxpy n one
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (1 i) ((1 n) (1 *)) work-%offset%)
         1
         (f2cl-lib:array-slice x-%data% f2cl-lib:complex16 (1 i)
                               ((1 ldx) (1 *)) x-%offset%)
         1)
      (declare (ignore var-2 var-3 var-4 var-5))
      (when var-0 (setf n var-0))
      (when var-1 (setf one var-1)))
   label100001))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "All" n nrhs b ldb$ work n)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf n var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (zhemm "L" uplo n nrhs negone a lda x ldx one work n)
  (declare (ignore var-0 var-5 var-7 var-10))
  (when var-1 (setf uplo var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf nrhs var-3))
  (when var-4 (setf negone var-4))
  (when var-6 (setf lda var-6))
  (when var-8 (setf ldx var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf n var-11)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nrhs) nil)
  (tagbody
    (setf xnrm
            (cabs1
             (f2cl-lib:fref x-%data%
```

```
                                    ((multiple-value-bind
                                         (ret-val var-0 var-1 var-2)
                                         (izamax n
                                          (f2cl-lib:array-slice x
                                                                f2cl-lib:complex16
                                                                (1 i)
                                                                ((1 ldx)
                                                                 (1 *)))
                                          1)
                                       (declare (ignore var-1 var-2))
                                       (when var-0 (setf n var-0))
                                       ret-val)
                                     i)
                                    ((1 ldx) (1 *)) x-%offset%)))
                 (setf rnrm
                         (cabs1
                          (f2cl-lib:fref work-%data%
                                    ((multiple-value-bind
                                         (ret-val var-0 var-1 var-2)
                                         (izamax n
                                          (f2cl-lib:array-slice work
                                                                f2cl-lib:complex16
                                                                (1 i)
                                                                ((1 n)
                                                                 (1 *)))
                                          1)
                                       (declare (ignore var-1 var-2))
                                       (when var-0 (setf n var-0))
                                       ret-val)
                                     i)
                                    ((1 n) (1 *)) work-%offset%)))
                 (if (> rnrm (* xnrm cte))
                     (go label20))
                label100002))
             (setf iter iiter)
             (go end_label)
            label20
            label30))
         (setf iter (f2cl-lib:int-sub -1 itermax))
        label40
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (zpotrf uplo n a lda info)
           (declare (ignore var-2))
           (when var-0 (setf uplo var-0))
           (when var-1 (setf n var-1))
           (when var-3 (setf lda var-3))
           (when var-4 (setf info var-4)))
         (if (/= info 0)
             (go end_label))
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
              (zlacpy "All" n nrhs b ldb$ x ldx)
            (declare (ignore var-0 var-3 var-5))
            (when var-1 (setf n var-1))
            (when var-2 (setf nrhs var-2))
            (when var-4 (setf ldb$ var-4))
            (when var-6 (setf ldx var-6)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
              (zpotrs uplo n nrhs a lda x ldx info)
            (declare (ignore var-3 var-5))
            (when var-0 (setf uplo var-0))
            (when var-1 (setf n var-1))
            (when var-2 (setf nrhs var-2))
            (when var-4 (setf lda var-4))
            (when var-6 (setf ldx var-6))
            (when var-7 (setf info var-7)))
          (go end_label)
         end_label
          (return
           (values uplo n nrhs nil lda nil ldb$ nil ldx nil nil nil iter
                   info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zcposv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
```

```
                                                  fortran-to-lisp::nrhs nil
                                                  fortran-to-lisp::lda nil
                                                  fortran-to-lisp::ldb$ nil
                                                  fortran-to-lisp::ldx nil nil nil
                                                  fortran-to-lisp::iter
                                                  fortran-to-lisp::info)
                                                :calls '(fortran-to-lisp::xerbla))))
```

———-

69.2 zposv

69.2.1 Axiom unit tests

— **zposv.input** —

```
)set break resume
)sys rm -f zposv.output
)spool zposv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

69.2.2 Axiom help page

— **zposv.help** —

```
======================================================================
zposv examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
```

```
      subroutine zposv (UPLO, N, NRHS, A, LDA, B, LDB, INFO)
           ZPOSV computes the solution to system of linear equations A * X =
          B for PO matrices

Function/Subroutine Documentation
   subroutine zposv (character UPLO, integer N, integer NRHS, complex*16,
       dimension( lda, * ) A, integer LDA, complex*16, dimension( ldb, * ) B,
       integer LDB, integer INFO)
        ZPOSV computes the solution to system of linear equations A * X = B
       for PO matrices

Purpose:
   ZPOSV computes the solution to a complex system of linear equations
      A * X = B,
   where A is an N-by-N Hermitian positive definite matrix and X and B
   are N-by-NRHS matrices.

   The Cholesky decomposition is used to factor A as
      A = U**H* U,  if UPLO = 'U', or
      A = L * L**H,  if UPLO = 'L',
   where U is an upper triangular matrix and  L is a lower triangular
   matrix.  The factored form of A is then used to solve the system of
   equations A * X = B.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the Hermitian matrix A.  If UPLO = 'U', the leading
   N-by-N upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading N-by-N lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.
```

```
  On exit, if INFO = 0, the factor U or L from the Cholesky
  factorization A = U**H *U or A = L*L**H.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, the leading minor of order i of A is not
        positive definite, so the factorization could not be
        completed, and the solution has not been computed.
```

———-

69.2.3 fortran code

— **zposv.f** —

```
*  =====================================================================
      SUBROUTINE ZPOSV( UPLO, N, NRHS, A, LDA, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
```

```
*
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZPOTRF, ZPOTRS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZPOSV ', -INFO )
         RETURN
      END IF
*
*     Compute the Cholesky factorization A = U**H *U or A = L*L**H.
*
      CALL ZPOTRF( UPLO, N, A, LDA, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
         CALL ZPOTRS( UPLO, N, NRHS, A, LDA, B, LDB, INFO )
*
      END IF
      RETURN
*
*     End of ZPOSV
*
      END
```

69.2.4 lisp code

— zposv.lisp —

```
(defun zposv (uplo n nrhs a lda b ldb$ info)
  (declare (type (array f2cl-lib:complex16 (*)) b a)
           (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (b f2cl-lib:complex16 b-%data% b-%offset%))
    (prog ()
      (declare)
      (setf info 0)
      (cond
       ((and
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -5))
       ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -7)))
      (cond
       ((/= info 0) (xerbla "ZPOSV " (f2cl-lib:int-sub info)) (go end_label)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (zpotrf uplo n a lda info)
        (declare (ignore var-2))
        (when var-0 (setf uplo var-0))
        (when var-1 (setf n var-1))
        (when var-3 (setf lda var-3))
        (when var-4 (setf info var-4)))
      (cond
       ((= info 0)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (zpotrs uplo n nrhs a lda b ldb$ info)
          (declare (ignore var-3 var-5))
```

```
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf lda var-4))
          (when var-6 (setf ldb$ var-6))
          (when var-7 (setf info var-7)))))
      (go end_label)
     end_label
      (return (values uplo n nrhs nil lda nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zposv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

69.3 zposvx

69.3.1 Axiom unit tests

— zposvx.input —

```
)set break resume
)sys rm -f zposvx.output
)spool zposvx.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

69.3.2 Axiom help page

— **zposvx.help** —

```
==================================================================
zposvx examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zposvx (FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, EQUED, S, B,
           LDB, X, LDX, RCOND, FERR, BERR, WORK, RWORK, INFO)
            ZPOSVX computes the solution to system of linear equations A * X =
           B for PO matrices

Function/Subroutine Documentation
   subroutine zposvx (character FACT, character UPLO, integer N, integer NRHS,
       complex*16, dimension( lda, * ) A, integer LDA, complex*16, dimension(
       ldaf, * ) AF, integer LDAF, character EQUED, double precision,
       dimension( * ) S, complex*16, dimension( ldb, * ) B, integer LDB,
       complex*16, dimension( ldx, * ) X, integer LDX, double precision RCOND,
       double precision, dimension( * ) FERR, double precision, dimension( * )
       BERR, complex*16, dimension( * ) WORK, double precision, dimension( * )
       RWORK, integer INFO)
        ZPOSVX computes the solution to system of linear equations A * X = B
       for PO matrices

Purpose:
   ZPOSVX uses the Cholesky factorization A = U**H*U or A = L*L**H to
   compute the solution to a complex system of linear equations
      A * X = B,
   where A is an N-by-N Hermitian positive definite matrix and X and B
   are N-by-NRHS matrices.

   Error bounds on the solution and a condition estimate are also provided.
```

```
Description:

   The following steps are performed:

   1. If FACT = 'E', real scaling factors are computed to equilibrate
      the system:
         diag(S) * A * diag(S) * inv(diag(S)) * X = diag(S) * B
      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(S)*A*diag(S) and B by diag(S)*B.

   2. If FACT = 'N' or 'E', the Cholesky decomposition is used to
      factor the matrix A (after equilibration if FACT = 'E') as
         A = U**H* U,  if UPLO = 'U', or
         A = L * L**H,  if UPLO = 'L',
      where U is an upper triangular matrix and L is a lower triangular
      matrix.

   3. If the leading i-by-i principal minor is not positive definite,
      then the routine returns with INFO = i. Otherwise, the factored
      form of A is used to estimate the condition number of the matrix
      A.  If the reciprocal of the condition number is less than machine
      precision, INFO = N+1 is returned as a warning, but the routine
      still goes on to solve for X and compute error bounds as
      described below.

   4. The system of equations is solved for X using the factored form of A.

   5. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

   6. If equilibration was used, the matrix X is premultiplied by
     diag(S) so that it solves the original system before equilibration.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AF contains the factored form of A.
      If EQUED = 'Y', the matrix A has been equilibrated
      with scaling factors given by S.  A and AF will not
      be modified.
   = 'N':  The matrix A will be copied to AF and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
      copied to AF and factored.
```

```
UPLO
  UPLO is CHARACTER*1
  = 'U':  Upper triangle of A is stored;
  = 'L':  Lower triangle of A is stored.

N
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the Hermitian matrix A, except if FACT = 'F' and
  EQUED = 'Y', then A must contain the equilibrated matrix
  diag(S)*A*diag(S).  If UPLO = 'U', the leading
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.  A is not modified if
  FACT = 'F' or 'N', or if FACT = 'E' and EQUED = 'N' on exit.

  On exit, if FACT = 'E' and EQUED = 'Y', A is overwritten by
  diag(S)*A*diag(S).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX*16 array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the triangular factor U or L from the Cholesky
  factorization A = U**H *U or A = L*L**H, in the same storage
  format as A.  If EQUED .ne. 'N', then AF is the factored form
  of the equilibrated matrix diag(S)*A*diag(S).

  If FACT = 'N', then AF is an output argument and on exit
  returns the triangular factor U or L from the Cholesky
  factorization A = U**H *U or A = L*L**H of the original
  matrix A.

  If FACT = 'E', then AF is an output argument and on exit
  returns the triangular factor U or L from the Cholesky
  factorization A = U**H *U or A = L*L**H of the equilibrated
```

```
  matrix A (see the description of A for the form of the
  equilibrated matrix).

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'Y':  Equilibration was done, i.e., A has been replaced by
          diag(S) * A * diag(S).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
  output argument.

S
  S is DOUBLE PRECISION array, dimension (N)
  The scale factors for A; not accessed if EQUED = 'N'.  S is
  an input argument if FACT = 'F'; otherwise, S is an output
  argument.  If FACT = 'F' and EQUED = 'Y', each element of S
  must be positive.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS righthand side matrix B.
  On exit, if EQUED = 'N', B is not modified; if EQUED = 'Y',
  B is overwritten by diag(S) * B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X to
  the original system of equations.  Note that if EQUED = 'Y',
  A and B are modified on exit, and the solution to the
  equilibrated system is inv(diag(S))*X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  The estimate of the reciprocal condition number of the matrix
  A after equilibration (if done).  If RCOND is less than the
  machine precision (in particular, if RCOND = 0), the matrix
  is singular to working precision.  This condition is
```

```
  indicated by a return code of INFO > 0.

FERR
  FERR is DOUBLE PRECISION array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is COMPLEX*16 array, dimension (2*N)

RWORK
  RWORK is DOUBLE PRECISION array, dimension (N)

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = i, and i is
  <= N:  the leading minor of order i of A is
     not positive definite, so the factorization
     could not be completed, and the solution has not
     been computed. RCOND = 0 is returned.
  = N+1: U is nonsingular, but RCOND is less than machine
     precision, meaning that the matrix is singular
     to working precision.  Nevertheless, the
     solution and error bounds are computed because
     there are a number of situations where the
     computed solution can be more accurate than the
     value of RCOND would suggest.
```

———-

69.3.3 fortran code

— zposvx.f —

```
*  =====================================================================
      SUBROUTINE ZPOSVX( FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, EQUED,
     $                   S, B, LDB, X, LDX, RCOND, FERR, BERR, WORK,
     $                   RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   BERR( * ), FERR( * ), RWORK( * ), S( * )
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            EQUIL, NOFACT, RCEQU
      INTEGER            I, INFEQU, J
      DOUBLE PRECISION   AMAX, ANORM, BIGNUM, SCOND, SMAX, SMIN, SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, ZLANHE
      EXTERNAL           LSAME, DLAMCH, ZLANHE
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLACPY, ZLAQHE, ZPOCON, ZPOEQU, ZPORFS,
     $                   ZPOTRF, ZPOTRS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      IF( NOFACT .OR. EQUIL ) THEN
```

```
         EQUED = 'N'
         RCEQU = .FALSE.
      ELSE
         RCEQU = LSAME( EQUED, 'Y' )
         SMLNUM = DLAMCH( 'Safe minimum' )
         BIGNUM = ONE / SMLNUM
      END IF
*
*     Test the input parameters.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.LSAME( FACT, 'F' ) )
     $     THEN
         INFO = -1
      ELSE IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) )
     $          THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $         ( RCEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -9
      ELSE
         IF( RCEQU ) THEN
            SMIN = BIGNUM
            SMAX = ZERO
            DO 10 J = 1, N
               SMIN = MIN( SMIN, S( J ) )
               SMAX = MAX( SMAX, S( J ) )
   10       CONTINUE
            IF( SMIN.LE.ZERO ) THEN
               INFO = -10
            ELSE IF( N.GT.0 ) THEN
               SCOND = MAX( SMIN, SMLNUM ) / MIN( SMAX, BIGNUM )
            ELSE
               SCOND = ONE
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -12
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -14
            END IF
         END IF
```

```
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZPOSVX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*        Compute row and column scalings to equilibrate the matrix A.
*
         CALL ZPOEQU( N, A, LDA, S, SCOND, AMAX, INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*           Equilibrate the matrix.
*
            CALL ZLAQHE( UPLO, N, A, LDA, S, SCOND, AMAX, EQUED )
            RCEQU = LSAME( EQUED, 'Y' )
         END IF
      END IF
*
*     Scale the right hand side.
*
      IF( RCEQU ) THEN
         DO 30 J = 1, NRHS
            DO 20 I = 1, N
               B( I, J ) = S( I )*B( I, J )
   20       CONTINUE
   30    CONTINUE
      END IF
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the Cholesky factorization A = U**H *U or A = L*L**H.
*
         CALL ZLACPY( UPLO, N, N, A, LDA, AF, LDAF )
         CALL ZPOTRF( UPLO, N, AF, LDAF, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 )THEN
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A.
*
      ANORM = ZLANHE( '1', UPLO, N, A, LDA, RWORK )
*
```

```
*     Compute the reciprocal of the condition number of A.
*
      CALL ZPOCON( UPLO, N, AF, LDAF, ANORM, RCOND, WORK, RWORK, INFO )
*
*     Compute the solution matrix X.
*
      CALL ZLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL ZPOTRS( UPLO, N, NRHS, AF, LDAF, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL ZPORFS( UPLO, N, NRHS, A, LDA, AF, LDAF, B, LDB, X, LDX,
     $             FERR, BERR, WORK, RWORK, INFO )
*
*     Transform the solution matrix X to a solution of the original
*     system.
*
      IF( RCEQU ) THEN
         DO 50 J = 1, NRHS
            DO 40 I = 1, N
               X( I, J ) = S( I )*X( I, J )
   40       CONTINUE
   50    CONTINUE
         DO 60 J = 1, NRHS
            FERR( J ) = FERR( J ) / SCOND
   60    CONTINUE
      END IF
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.DLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      RETURN
*
*     End of ZPOSVX
*
      END
```

———-

69.3.4 lisp code

— zposvx.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
```

```
          (type (double-float 1.0d0 1.0d0) one)
          (ignorable zero one))
(defun zposvx
       (fact uplo n nrhs a lda af ldaf equed s b ldb$ x ldx rcond ferr berr
        work rwork info)
  (declare (type (double-float) rcond)
           (type (array double-float (*)) rwork berr ferr s)
           (type (array f2cl-lib:complex16 (*)) work x b af a)
           (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
           (type (string *) equed uplo fact))
  (f2cl-lib:with-multi-array-data
      ((fact character fact-%data% fact-%offset%)
       (uplo character uplo-%data% uplo-%offset%)
       (equed character equed-%data% equed-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (af f2cl-lib:complex16 af-%data% af-%offset%)
       (b f2cl-lib:complex16 b-%data% b-%offset%)
       (x f2cl-lib:complex16 x-%data% x-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%)
       (s double-float s-%data% s-%offset%)
       (ferr double-float ferr-%data% ferr-%offset%)
       (berr double-float berr-%data% berr-%offset%)
       (rwork double-float rwork-%data% rwork-%offset%))
    (prog ((amax 0.0d0) (anorm 0.0d0) (bignum 0.0d0) (scond 0.0d0)
           (smax 0.0d0) (smin 0.0d0) (smlnum 0.0d0) (i 0) (infequ 0) (j 0)
           (equil nil) (nofact nil) (rcequ nil))
      (declare (type (double-float) amax anorm bignum scond smax smin smlnum)
               (type (f2cl-lib:integer4) i infequ j)
               (type f2cl-lib:logical equil nofact rcequ))
      (setf info 0)
      (setf nofact
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame fact "N")
                (declare (ignore var-1))
                (when var-0 (setf fact var-0))
                ret-val))
      (setf equil
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame fact "E")
                (declare (ignore var-1))
                (when var-0 (setf fact var-0))
                ret-val))
      (cond
       ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
        (setf rcequ f2cl-lib:%false%))
       (t
        (setf rcequ
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame equed "Y")
                  (declare (ignore var-1))
```

```
            (when var-0 (setf equed var-0))
            ret-val))
  (setf smlnum (dlamch "Safe minimum")) (setf bignum (/ one smlnum))))
(cond
 ((and (not nofact) (not equil)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -8))
 ((and
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame fact "F")
     (declare (ignore var-1))
     (when var-0 (setf fact var-0))
     ret-val)
   (not
    (or rcequ
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame equed "N")
          (declare (ignore var-1))
          (when var-0 (setf equed var-0))
          ret-val))))
  (setf info -9))
 (t
  (cond
   (rcequ (setf smin bignum) (setf smax zero)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
```

```
        (tagbody
          (setf smin
                  (min smin
                       (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
          (setf smax
                  (max smax
                       (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
         label10))
      (cond ((<= smin zero) (setf info -10))
            ((> n 0)
             (setf scond (/ (max smin smlnum) (min smax bignum))))
            (t (setf scond one)))))
    (cond
     ((= info 0)
      (cond
       ((< ldb$
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -12))
       ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -14)))))))
  (cond
   ((/= info 0) (xerbla "ZPOSVX" (f2cl-lib:int-sub info))
    (go end_label)))
  (cond
   (equil
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zpoequ n a lda s scond amax infequ)
      (declare (ignore var-0 var-1 var-2 var-3))
      (setf scond var-4)
      (setf amax var-5)
      (setf infequ var-6))
    (cond
     ((= infequ 0)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (zlaqhe uplo n a lda s scond amax equed)
        (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
        (setf uplo var-0)
        (setf equed var-7))
      (setf rcequ
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame equed "Y")
                (declare (ignore var-1))
                (when var-0 (setf equed var-0))
                ret-val))))))
  (cond
   (rcequ
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
```

```
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                  (* (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)))
         label20))
     label30))))
(cond
 ((or nofact equil)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy uplo n n a lda af ldaf)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
    (setf uplo var-0))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (zpotrf uplo n af ldaf info)
    (declare (ignore var-2))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf ldaf var-3))
    (when var-4 (setf info var-4)))
  (cond ((> info 0) (setf rcond zero) (go end_label)))))
(setf anorm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlanhe "1" uplo n a lda rwork)
          (declare (ignore var-0 var-2 var-3 var-4 var-5))
          (setf uplo var-1)
          ret-val))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zpocon uplo n af ldaf anorm rcond work rwork info)
  (declare (ignore var-2 var-4 var-6 var-7))
  (setf uplo var-0)
  (setf n var-1)
  (setf ldaf var-3)
  (setf rcond var-5)
  (setf info var-8))
(zlacpy "Full" n nrhs b ldb$ x ldx)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zpotrs uplo n nrhs af ldaf x ldx info)
  (declare (ignore var-3 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldaf var-4))
  (when var-6 (setf ldx var-6))
  (when var-7 (setf info var-7)))
```

```
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
             var-11 var-12 var-13 var-14 var-15)
            (zporfs uplo n nrhs a lda af ldaf b ldb$ x ldx ferr berr work rwork
             info)
          (declare
           (ignore var-2 var-3 var-5 var-7 var-8 var-9 var-10 var-11 var-12
            var-13 var-14))
          (setf uplo var-0)
          (setf n var-1)
          (setf lda var-4)
          (setf ldaf var-6)
          (setf info var-15))
        (cond
         (rcequ
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j nrhs) nil)
            (tagbody
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i n) nil)
                (tagbody
                  (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                       x-%offset%)
                          (* (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                             (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                            x-%offset%)))
                 label40))
             label50))
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j nrhs) nil)
            (tagbody
              (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                      (/ (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                         scond))
             label60))))
        (if (< rcond (dlamch "Epsilon"))
            (setf info (f2cl-lib:int-add n 1)))
        (go end_label)
       end_label
        (return
         (values fact uplo n nrhs nil lda nil ldaf equed nil nil nil nil ldx
                 rcond nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zposvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
```

```
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (string) (array double-float (*))
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (double-float)
  (array double-float (*))
  (array double-float (*))
  (array fortran-to-lisp::complex16
   (*))
  (array double-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::fact
  fortran-to-lisp::uplo
  fortran-to-lisp::n
  fortran-to-lisp::nrhs nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldaf
  fortran-to-lisp::equed nil nil
  nil nil fortran-to-lisp::ldx
  fortran-to-lisp::rcond nil nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zporfs
  fortran-to-lisp::zpocon
  fortran-to-lisp::zlanhe
  fortran-to-lisp::zlacpy
  fortran-to-lisp::zlaqhe
  fortran-to-lisp::zpoequ
  fortran-to-lisp::xerbla))))
```

69.4 zposvxx

69.4.1 Axiom unit tests

— **zposvxx.input** —

```
)set break resume
)sys rm -f zposvxx.output
)spool zposvxx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

69.4.2 Axiom help page [Demm05]

— **zposvxx.help** —

```
==================================================================
zposvxx examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zposvxx (FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, EQUED, S, B,
           LDB, X, LDX, RCOND, RPVGRW, BERR, N_ERR_BNDS, ERR_BNDS_NORM,
           ERR_BNDS_COMP, NPARAMS, PARAMS, WORK, RWORK, INFO)
            ZPOSVXX computes the solution to system of linear equations A * X
           = B for PO matrices

Function/Subroutine Documentation
   subroutine zposvxx (character FACT, character UPLO, integer N, integer
       NRHS, complex*16, dimension( lda, * ) A, integer LDA, complex*16,
       dimension( ldaf, * ) AF, integer LDAF, character EQUED, double
       precision, dimension( * ) S, complex*16, dimension( ldb, * ) B, integer
       LDB, complex*16, dimension( ldx, * ) X, integer LDX, double precision
       RCOND, double precision RPVGRW, double precision, dimension( * ) BERR,
       integer N_ERR_BNDS, double precision, dimension( nrhs, * )
```

```
     ERR_BNDS_NORM, double precision, dimension( nrhs, * ) ERR_BNDS_COMP,
     integer NPARAMS, double precision, dimension( * ) PARAMS, complex*16,
     dimension( * ) WORK, double precision, dimension( * ) RWORK, integer
     INFO)
      ZPOSVXX computes the solution to system of linear equations A * X = B
     for PO matrices

Purpose:
   ZPOSVXX uses the Cholesky factorization A = U**T*U or A = L*L**T
   to compute the solution to a complex*16 system of linear equations
   A * X = B, where A is an N-by-N symmetric positive definite matrix
   and X and B are N-by-NRHS matrices.

   If requested, both normwise and maximum componentwise error bounds
   are returned. ZPOSVXX will return a solution with a tiny
   guaranteed error (O(eps) where eps is the working machine
   precision) unless the matrix is very ill-conditioned, in which
   case a warning is returned. Relevant condition numbers also are
   calculated and returned.

   ZPOSVXX accepts user-provided factorizations and equilibration
   factors; see the definitions of the FACT and EQUED options.
   Solving with refinement and using a factorization from a previous
   ZPOSVXX call will also produce a solution with either O(eps)
   errors or warnings, but we cannot make that claim for general
   user-provided factorizations and equilibration factors if they
   differ from what ZPOSVXX would itself produce.

Description:

   The following steps are performed:

   1. If FACT = 'E', double precision scaling factors are computed to
      equilibrate the system:

      diag(S)*A*diag(S)     *inv(diag(S))*X = diag(S)*B

      Whether or not the system will be equilibrated depends on the
      scaling of the matrix A, but if equilibration is used, A is
      overwritten by diag(S)*A*diag(S) and B by diag(S)*B.

   2. If FACT = 'N' or 'E', the Cholesky decomposition is used to
      factor the matrix A (after equilibration if FACT = 'E') as
         A = U**T* U,  if UPLO = 'U', or
         A = L * L**T,  if UPLO = 'L',
      where U is an upper triangular matrix and L is a lower triangular
      matrix.

   3. If the leading i-by-i principal minor is not positive definite,
      then the routine returns with INFO = i. Otherwise, the factored
```

```
      form of A is used to estimate the condition number of the matrix
      A (see argument RCOND).  If the reciprocal of the condition number
      is less than machine precision, the routine still goes on to solve
      for X and compute error bounds as described below.

   4. The system of equations is solved for X using the factored form of A.

   5. By default (unless PARAMS(LA_LINRX_ITREF_I) is set to zero),
      the routine will use iterative refinement to try to get a small
      error and error bounds.  Refinement calculates the residual to at
      least twice the working precision.

   6. If equilibration was used, the matrix X is premultiplied by
      diag(S) so that it solves the original system before equilibration.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix A is
   supplied on entry, and if not, whether the matrix A should be
   equilibrated before it is factored.
   = 'F':  On entry, AF contains the factored form of A.
      If EQUED is not 'N', the matrix A has been
      equilibrated with scaling factors given by S.
      A and AF are not modified.
   = 'N':  The matrix A will be copied to AF and factored.
   = 'E':  The matrix A will be equilibrated if necessary, then
      copied to AF and factored.

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.
```

```
A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the symmetric matrix A, except if FACT = 'F' and EQUED =
  'Y', then A must contain the equilibrated matrix
  diag(S)*A*diag(S).  If UPLO = 'U', the leading N-by-N upper
  triangular part of A contains the upper triangular part of the
  matrix A, and the strictly lower triangular part of A is not
  referenced.  If UPLO = 'L', the leading N-by-N lower triangular
  part of A contains the lower triangular part of the matrix A, and
  the strictly upper triangular part of A is not referenced.  A is
  not modified if FACT = 'F' or 'N', or if FACT = 'E' and EQUED =
  'N' on exit.

  On exit, if FACT = 'E' and EQUED = 'Y', A is overwritten by
  diag(S)*A*diag(S).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX*16 array, dimension (LDAF,N)
  If FACT = 'F', then AF is an input argument and on entry
  contains the triangular factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T, in the same storage
  format as A.  If EQUED .ne. 'N', then AF is the factored
  form of the equilibrated matrix diag(S)*A*diag(S).

  If FACT = 'N', then AF is an output argument and on exit
  returns the triangular factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T of the original
  matrix A.

  If FACT = 'E', then AF is an output argument and on exit
  returns the triangular factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T of the equilibrated
  matrix A (see the description of A for the form of the
  equilibrated matrix).

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

EQUED
  EQUED is CHARACTER*1
  Specifies the form of equilibration that was done.
  = 'N':  No equilibration (always true if FACT = 'N').
  = 'Y':  Both row and column equilibration, i.e., A has been
          replaced by diag(S) * A * diag(S).
  EQUED is an input argument if FACT = 'F'; otherwise, it is an
```

```
  output argument.

S
  S is DOUBLE PRECISION array, dimension (N)
  The row scale factors for A.  If EQUED = 'Y', A is multiplied on
  the left and right by diag(S).  S is an input argument if FACT =
  'F'; otherwise, S is an output argument.  If FACT = 'F' and EQUED
  = 'Y', each element of S must be positive.  If S is output, each
  element of S is a power of the radix. If S is input, each element
  of S should be a power of the radix to ensure a reliable solution
  and error estimates. Scaling by powers of the radix does not cause
  rounding errors unless the result underflows or overflows.
  Rounding errors during scaling lead to refining with a matrix that
  is not equivalent to the input matrix, producing error estimates
  that may not be reliable.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit,
  if EQUED = 'N', B is not modified;
  if EQUED = 'Y', B is overwritten by diag(S)*B;

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  If INFO = 0, the N-by-NRHS solution matrix X to the original
  system of equations.  Note that A and B are modified on exit if
  EQUED .ne. 'N', and the solution to the equilibrated system is
  inv(diag(S))*X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

RPVGRW
  RPVGRW is DOUBLE PRECISION
  Reciprocal pivot growth.  On exit, this contains the reciprocal
```

```
  pivot growth factor norm(A)/norm(U). The "max absolute element"
  norm is used.  If this is much less than 1, then the stability of
  the LU factorization of the (equilibrated) matrix A could be poor.
  This also means that the solution X, estimated condition numbers,
  and error bounds could be unreliable. If factorization fails with
  0<INFO<=N, then this contains the reciprocal pivot growth factor
  for the leading INFO columns of A.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
          max_j (abs(XTRUE(j,i) - X(j,i)))
           ------------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * dlamch('Epsilon'). This error bound should only
```

```
      be trusted if the previous boolean is true.

   err = 3  Reciprocal condition number: Estimated normwise
      reciprocal condition number.  Compared with the threshold
      sqrt(n) * dlamch('Epsilon') to determine if the error
      estimate is "guaranteed". These reciprocal condition
      numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
      appropriately scaled matrix Z.
      Let Z = S*A, where S scales each row by a power of the
      radix so all absolute row sums of Z are approximately 1.

 ERR_BNDS_COMP
   ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
   For each right-hand side, this array contains information about
   various error bounds and condition numbers corresponding to the
   componentwise relative error, which is defined as follows:

   Componentwise relative error in the ith solution vector:
               abs(XTRUE(j,i) - X(j,i))
          max_j ----------------------
                        abs(X(j,i))

   The array is indexed by the right-hand side i (on which the
   componentwise relative error depends), and the type of error
   information as described below. There currently are up to three
   pieces of information returned for each right-hand side. If
   componentwise accuracy is not requested (PARAMS(3) = 0.0), then
   ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
   the first (:,N_ERR_BNDS) entries are returned.

   The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
   right-hand side.

   The second index in ERR_BNDS_COMP(:,err) contains the following
   three fields:

   err = 1 "Trust/don't trust" boolean. Trust the answer if the
      reciprocal condition number is less than the threshold
      sqrt(n) * dlamch('Epsilon').

   err = 2 "Guaranteed" error bound: The estimated forward error,
      almost certainly within a factor of 10 of the true error
      so long as the next entry is greater than the threshold
      sqrt(n) * dlamch('Epsilon'). This error bound should only
      be trusted if the previous boolean is true.

   err = 3  Reciprocal condition number: Estimated componentwise
      reciprocal condition number.  Compared with the threshold
      sqrt(n) * dlamch('Epsilon') to determine if the error
      estimate is "guaranteed". These reciprocal condition
```

```
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is DOUBLE PRECISION array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0D+0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the extra-precise refinement algorithm.
  (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is COMPLEX*16 array, dimension (2*N)

RWORK
  RWORK is DOUBLE PRECISION array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is
```

```
      guaranteed.
   < 0:  If INFO = -i, the i-th argument had an illegal value
   > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
      has been completed, but the factor U is exactly singular, so
      the solution and error bounds could not be computed. RCOND = 0
      is returned.
   = N+J: The solution corresponding to the Jth right-hand side is
      not guaranteed. The solutions corresponding to other right-
      hand sides K with K > J may not be guaranteed as well, but
      only the first such right-hand side is reported. If a small
      componentwise error is not requested (PARAMS(3) = 0.0) then
      the Jth right-hand side is the first with a normwise error
      bound that is not guaranteed (the smallest J such
      that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
      the Jth right-hand side is the first with either a normwise or
      componentwise error bound that is not guaranteed (the smallest
      J such that either ERR_BNDS_NORM(J,1) = 0.0 or
      ERR_BNDS_COMP(J,1) = 0.0). See the definition of
      ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
      about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

———-

69.4.3 fortran code

— zposvxx.f —

```
*  ==================================================================
      SUBROUTINE ZPOSVXX( FACT, UPLO, N, NRHS, A, LDA, AF, LDAF, EQUED,
     $                    S, B, LDB, X, LDX, RCOND, RPVGRW, BERR,
     $                    N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP,
     $                    NPARAMS, PARAMS, WORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          EQUED, FACT, UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      DOUBLE PRECISION   RCOND, RPVGRW
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   WORK( * ), X( LDX, * )
```

```
      DOUBLE PRECISION   S( * ), PARAMS( * ), BERR( * ), RWORK( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  =================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
*     ..
*     .. Local Scalars ..
      LOGICAL            EQUIL, NOFACT, RCEQU
      INTEGER            INFEQU, J
      DOUBLE PRECISION   AMAX, BIGNUM, SMIN, SMAX, SCOND, SMLNUM
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, DLAMCH, ZLA_PORPVGRW
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, ZLA_PORPVGRW
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZPOCON, ZPOEQUB, ZPOTRF, ZPOTRS, ZLACPY,
     $                   ZLAQHE, XERBLA, ZLASCL2, ZPORFSX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      EQUIL = LSAME( FACT, 'E' )
      SMLNUM = DLAMCH( 'Safe minimum' )
      BIGNUM = ONE / SMLNUM
      IF( NOFACT .OR. EQUIL ) THEN
         EQUED = 'N'
         RCEQU = .FALSE.
      ELSE
         RCEQU = LSAME( EQUED, 'Y' )
      ENDIF
*
```

```
*     Default is failure.  If an input parameter is wrong or
*     factorization fails, make everything look horrible.  Only the
*     pivot growth is set here, the rest is initialized in ZPORFSX.
*
      RPVGRW = ZERO
*
*     Test the input parameters.  PARAMS is not tested until ZPORFSX.
*
      IF( .NOT.NOFACT .AND. .NOT.EQUIL .AND. .NOT.
     $     LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.LSAME( UPLO, 'U' ) .AND.
     $         .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LSAME( FACT, 'F' ) .AND. .NOT.
     $        ( RCEQU .OR. LSAME( EQUED, 'N' ) ) ) THEN
         INFO = -9
      ELSE
         IF ( RCEQU ) THEN
            SMIN = BIGNUM
            SMAX = ZERO
            DO 10 J = 1, N
               SMIN = MIN( SMIN, S( J ) )
               SMAX = MAX( SMAX, S( J ) )
 10         CONTINUE
            IF( SMIN.LE.ZERO ) THEN
               INFO = -10
            ELSE IF( N.GT.0 ) THEN
               SCOND = MAX( SMIN, SMLNUM ) / MIN( SMAX, BIGNUM )
            ELSE
               SCOND = ONE
            END IF
         END IF
         IF( INFO.EQ.0 ) THEN
            IF( LDB.LT.MAX( 1, N ) ) THEN
               INFO = -12
            ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
               INFO = -14
            END IF
         END IF
      END IF
*
```

```
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZPOSVXX', -INFO )
         RETURN
      END IF
*
      IF( EQUIL ) THEN
*
*     Compute row and column scalings to equilibrate the matrix A.
*
         CALL ZPOEQUB( N, A, LDA, S, SCOND, AMAX, INFEQU )
         IF( INFEQU.EQ.0 ) THEN
*
*     Equilibrate the matrix.
*
            CALL ZLAQHE( UPLO, N, A, LDA, S, SCOND, AMAX, EQUED )
            RCEQU = LSAME( EQUED, 'Y' )
         END IF
      END IF
*
*     Scale the right-hand side.
*
      IF( RCEQU ) CALL ZLASCL2( N, NRHS, S, B, LDB )
*
      IF( NOFACT .OR. EQUIL ) THEN
*
*        Compute the Cholesky factorization of A.
*
         CALL ZLACPY( UPLO, N, N, A, LDA, AF, LDAF )
         CALL ZPOTRF( UPLO, N, AF, LDAF, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 ) THEN
*
*           Pivot in column INFO is exactly 0
*           Compute the reciprocal pivot growth factor of the
*           leading rank-deficient INFO columns of A.
*
            RPVGRW = ZLA_PORPVGRW( UPLO, N, A, LDA, AF, LDAF, RWORK )
            RETURN
         END IF
      END IF
*
*     Compute the reciprocal pivot growth factor RPVGRW.
*
      RPVGRW = ZLA_PORPVGRW( UPLO, N, A, LDA, AF, LDAF, RWORK )
*
*     Compute the solution matrix X.
*
      CALL ZLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
```

```
      CALL ZPOTRS( UPLO, N, NRHS, AF, LDAF, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solution and
*     compute error bounds and backward error estimates for it.
*
      CALL ZPORFSX( UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF,
     $     S, B, LDB, X, LDX, RCOND, BERR, N_ERR_BNDS, ERR_BNDS_NORM,
     $     ERR_BNDS_COMP,  NPARAMS, PARAMS, WORK, RWORK, INFO )

*
*     Scale solutions.
*
      IF ( RCEQU ) THEN
         CALL ZLASCL2( N, NRHS, S, X, LDX )
      END IF
*
      RETURN
*
*     End of ZPOSVXX
*
      END
```

———-

69.4.4 lisp code

— zposvxx.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun zposvxx
         (fact uplo n nrhs a lda af ldaf equed s b ldb$ x ldx rcond rpvgrw berr
          n_err_bnds err_bnds_norm err_bnds_comp nparams params work rwork
          info)
    (declare (type (double-float) rpvgrw rcond)
             (type (array double-float (*)) rwork params err_bnds_comp
              err_bnds_norm berr s)
             (type (array f2cl-lib:complex16 (*)) work x b af a)
             (type (f2cl-lib:integer4) info nparams n_err_bnds ldx ldb$ ldaf
              lda nrhs n)
             (type (string *) equed uplo fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (uplo character uplo-%data% uplo-%offset%)
         (equed character equed-%data% equed-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
```

```
    (af f2cl-lib:complex16 af-%data% af-%offset%)
    (b f2cl-lib:complex16 b-%data% b-%offset%)
    (x f2cl-lib:complex16 x-%data% x-%offset%)
    (work f2cl-lib:complex16 work-%data% work-%offset%)
    (s double-float s-%data% s-%offset%)
    (berr double-float berr-%data% berr-%offset%)
    (err_bnds_norm double-float err_bnds_norm-%data%
     err_bnds_norm-%offset%)
    (err_bnds_comp double-float err_bnds_comp-%data%
     err_bnds_comp-%offset%)
    (params double-float params-%data% params-%offset%)
    (rwork double-float rwork-%data% rwork-%offset%))
  (prog ((amax 0.0d0) (bignum 0.0d0) (smin 0.0d0) (smax 0.0d0)
         (scond 0.0d0) (smlnum 0.0d0) (infequ 0) (j 0) (equil nil)
         (nofact nil) (rcequ nil))
    (declare (type (double-float) amax bignum smin smax scond smlnum)
             (type (f2cl-lib:integer4) infequ j)
             (type f2cl-lib:logical equil nofact rcequ))
    (setf info 0)
    (setf nofact
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "N")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val))
    (setf equil
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "E")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val))
    (setf smlnum (dlamch "Safe minimum"))
    (setf bignum (/ one smlnum))
    (cond
     ((or nofact equil) (f2cl-lib:f2cl-set-string equed "N" (string 1))
      (setf rcequ f2cl-lib:%false%))
     (t
      (setf rcequ
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame equed "Y")
                (declare (ignore var-1))
                (when var-0 (setf equed var-0))
                ret-val))))
    (setf rpvgrw zero)
    (cond
     ((and (not nofact) (not equil)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "F")
              (declare (ignore var-1))
```

```
          (when var-0 (setf fact var-0))
          ret-val)))
 (setf info -1))
((and
  (not
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame uplo "U")
     (declare (ignore var-1))
     (when var-0 (setf uplo var-0))
     ret-val))
  (not
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame uplo "L")
     (declare (ignore var-1))
     (when var-0 (setf uplo var-0))
     ret-val)))
 (setf info -2))
((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
 (setf info -6))
((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
 (setf info -8))
((and
  (multiple-value-bind (ret-val var-0 var-1)
      (lsame fact "F")
    (declare (ignore var-1))
    (when var-0 (setf fact var-0))
    ret-val)
  (not
   (or rcequ
       (multiple-value-bind (ret-val var-0 var-1)
           (lsame equed "N")
         (declare (ignore var-1))
         (when var-0 (setf equed var-0))
         ret-val))))
 (setf info -9))
(t
 (cond
  (rcequ (setf smin bignum) (setf smax zero)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf smin
               (min smin
                    (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
       (setf smax
               (max smax
                    (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)))
      label10))
   (cond ((<= smin zero) (setf info -10))
```

```
          ((> n 0)
           (setf scond (/ (max smin smlnum) (min smax bignum))))
          (t (setf scond one)))))
  (cond
   ((= info 0)
    (cond
     ((< ldb$
         (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -12))
     ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -14)))))))
(cond
 ((/= info 0) (xerbla "ZPOSVXX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (equil
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zpoequb n a lda s scond amax infequ)
    (declare (ignore var-0 var-1 var-2 var-3))
    (setf scond var-4)
    (setf amax var-5)
    (setf infequ var-6))
  (cond
   ((= infequ 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (zlaqhe uplo n a lda s scond amax equed)
      (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
      (setf uplo var-0)
      (setf equed var-7))
    (setf rcequ
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame equed "Y")
              (declare (ignore var-1))
              (when var-0 (setf equed var-0))
              ret-val))))))
(if rcequ
    (zlascl2 n nrhs s b ldb$))
(cond
 ((or nofact equil)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy uplo n n a lda af ldaf)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
    (setf uplo var-0))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (zpotrf uplo n af ldaf info)
    (declare (ignore var-2))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf ldaf var-3))
```

```
    (when var-4 (setf info var-4)))
  (cond
   ((> info 0)
    (setf rpvgrw
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (zla_porpvgrw uplo n a lda af ldaf rwork)
              (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
              (setf uplo var-0)
              ret-val))
    (go end_label)))))
(setf rpvgrw
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zla_porpvgrw uplo n a lda af ldaf rwork)
          (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6))
          (setf uplo var-0)
          ret-val))
(zlacpy "Full" n nrhs b ldb$ x ldx)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zpotrs uplo n nrhs af ldaf x ldx info)
  (declare (ignore var-3 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldaf var-4))
  (when var-6 (setf ldx var-6))
  (when var-7 (setf info var-7)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19
     var-20 var-21 var-22)
    (zporfsx uplo equed n nrhs a lda af ldaf s b ldb$ x ldx rcond berr
     n_err_bnds err_bnds_norm err_bnds_comp nparams params work rwork
     info)
  (declare
   (ignore var-4 var-6 var-8 var-9 var-11 var-14 var-16 var-17 var-19
    var-20 var-21))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf equed var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf nrhs var-3))
  (when var-5 (setf lda var-5))
  (when var-7 (setf ldaf var-7))
  (when var-10 (setf ldb$ var-10))
  (when var-12 (setf ldx var-12))
  (when var-13 (setf rcond var-13))
  (when var-15 (setf n_err_bnds var-15))
  (when var-18 (setf nparams var-18))
  (when var-22 (setf info var-22)))
```

```
        (cond (rcequ (zlascl2 n nrhs s x ldx)))
        (go end_label)
       end_label
        (return
         (values fact uplo n nrhs nil lda nil ldaf equed nil nil ldb$ nil ldx
                 rcond rpvgrw nil n_err_bnds nil nil nparams nil nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zposvxx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (string) (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::fact
                                              fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldaf
                                              fortran-to-lisp::equed nil nil
                                              fortran-to-lisp::ldb$ nil
```

```
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond
  fortran-to-lisp::rpvgrw nil
  fortran-to-lisp::n_err_bnds nil
  nil fortran-to-lisp::nparams nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zla_porpvgrw
  fortran-to-lisp::zlacpy
  fortran-to-lisp::zlascl2
  fortran-to-lisp::zlaqhe
  fortran-to-lisp::zpoequb
  fortran-to-lisp::xerbla))))
```

Chapter 70

LAPACK - Positive Definite Matrix, Comp. Routines, Double

70.1 dla_porcond

70.1.1 Axiom unit tests

— **dlaporcond.input** —

```
)set break resume
)sys rm -f dlaporcond.output
)spool dlaporcond.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

70.1.2 Axiom help page

— **dla_porcond.help** —

```
==========================================================================
```

```
dla_porcond examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function dla_porcond (UPLO, N, A, LDA, AF, LDAF,
           CMODE, C, INFO, WORK, IWORK)
           DLA_PORCOND estimates the Skeel condition number for a symmetric
           positive-definite matrix.

Function/Subroutine Documentation
   double precision function dla_porcond (character UPLO, integer N, double
       precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldaf, * ) AF, integer LDAF, integer CMODE, double precision,
       dimension( * ) C, integer INFO, double precision, dimension( * ) WORK,
       integer, dimension( * ) IWORK)
       DLA_PORCOND estimates the Skeel condition number for a symmetric
       positive-definite matrix.

Purpose:
   DLA_PORCOND Estimates the Skeel condition number of  op(A) * op2(C)
   where op2 is determined by CMODE as follows
   CMODE =  1    op2(C) = C
   CMODE =  0    op2(C) = I
   CMODE = -1    op2(C) = inv(C)
   The Skeel condition number  cond(A) = norminf( |inv(A)||A| )
   is computed by computing scaling factors R such that
   diag(R)*A*op2(C) is row equilibrated and computing the standard
   infinity-norm condition number.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
```

```
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is DOUBLE PRECISION array, dimension (LDAF,N)
  The triangular factor U or L from the Cholesky factorization
  A = U**T*U or A = L*L**T, as computed by DPOTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

CMODE
  CMODE is INTEGER
  Determines op2(C) in the formula op(A) * op2(C) as follows:
  CMODE =  1    op2(C) = C
  CMODE =  0    op2(C) = I
  CMODE = -1    op2(C) = inv(C)

C
  C is DOUBLE PRECISION array, dimension (N)
  The vector C in the formula op(A) * op2(C).

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  i > 0:  The ith argument is invalid.

WORK
  WORK is DOUBLE PRECISION array, dimension (3*N).
  Workspace.

IWORK
  IWORK is INTEGER array, dimension (N).
  Workspace.
```

———-

70.1.3 fortran code

— dla_porcond.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION DLA_PORCOND( UPLO, N, A, LDA, AF, LDAF,
     $                                       CMODE, C, INFO, WORK,
     $                                       IWORK )
*
```

```
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            N, LDA, LDAF, INFO, CMODE
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * ), WORK( * ),
     $                   C( * )
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            KASE, I, J
      DOUBLE PRECISION   AINVNM, TMP
      LOGICAL            UP
*     ..
*     .. Array Arguments ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX
      EXTERNAL           LSAME, IDAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLACN2, DPOTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
      DLA_PORCOND = 0.0D+0
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -2
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLA_PORCOND', -INFO )
         RETURN
      END IF

      IF( N.EQ.0 ) THEN
```

```
         DLA_PORCOND = 1.0D+0
         RETURN
      END IF
      UP = .FALSE.
      IF ( LSAME( UPLO, 'U' ) ) UP = .TRUE.
*
*     Compute the equilibration matrix R such that
*     inv(R)*A*C has unit 1-norm.
*
      IF ( UP ) THEN
         DO I = 1, N
            TMP = 0.0D+0
            IF ( CMODE .EQ. 1 ) THEN
               DO J = 1, I
                  TMP = TMP + ABS( A( J, I ) * C( J ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + ABS( A( I, J ) * C( J ) )
               END DO
            ELSE IF ( CMODE .EQ. 0 ) THEN
               DO J = 1, I
                  TMP = TMP + ABS( A( J, I ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + ABS( A( I, J ) )
               END DO
            ELSE
               DO J = 1, I
                  TMP = TMP + ABS( A( J ,I ) / C( J ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + ABS( A( I, J ) / C( J ) )
               END DO
            END IF
            WORK( 2*N+I ) = TMP
         END DO
      ELSE
         DO I = 1, N
            TMP = 0.0D+0
            IF ( CMODE .EQ. 1 ) THEN
               DO J = 1, I
                  TMP = TMP + ABS( A( I, J ) * C( J ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + ABS( A( J, I ) * C( J ) )
               END DO
            ELSE IF ( CMODE .EQ. 0 ) THEN
               DO J = 1, I
                  TMP = TMP + ABS( A( I, J ) )
               END DO
```

```
            DO J = I+1, N
               TMP = TMP + ABS( A( J, I ) )
            END DO
         ELSE
            DO J = 1, I
               TMP = TMP + ABS( A( I, J ) / C( J ) )
            END DO
            DO J = I+1, N
               TMP = TMP + ABS( A( J, I ) / C( J ) )
            END DO
         END IF
         WORK( 2*N+I ) = TMP
      END DO
   ENDIF
*
*     Estimate the norm of inv(op(A)).
*
   AINVNM = 0.0D+0

   KASE = 0
10 CONTINUE
   CALL DLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
   IF( KASE.NE.0 ) THEN
      IF( KASE.EQ.2 ) THEN
*
*        Multiply by R.
*
         DO I = 1, N
            WORK( I ) = WORK( I ) * WORK( 2*N+I )
         END DO

         IF (UP) THEN
            CALL DPOTRS( 'Upper', N, 1, AF, LDAF, WORK, N, INFO )
         ELSE
            CALL DPOTRS( 'Lower', N, 1, AF, LDAF, WORK, N, INFO )
         ENDIF
*
*        Multiply by inv(C).
*
         IF ( CMODE .EQ. 1 ) THEN
            DO I = 1, N
               WORK( I ) = WORK( I ) / C( I )
            END DO
         ELSE IF ( CMODE .EQ. -1 ) THEN
            DO I = 1, N
               WORK( I ) = WORK( I ) * C( I )
            END DO
         END IF
      ELSE
*
```

```
*           Multiply by inv(C**T).
*
            IF ( CMODE .EQ. 1 ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) / C( I )
               END DO
            ELSE IF ( CMODE .EQ. -1 ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF

            IF ( UP ) THEN
               CALL DPOTRS( 'Upper', N, 1, AF, LDAF, WORK, N, INFO )
            ELSE
               CALL DPOTRS( 'Lower', N, 1, AF, LDAF, WORK, N, INFO )
            ENDIF
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * WORK( 2*N+I )
            END DO
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM .NE. 0.0D+0 )
     $   DLA_PORCOND = ( 1.0D+0 / AINVNM )
*
      RETURN
*
      END
```

———-

70.1.4 lisp code

— dla_porcond.lisp —

```
(defun dla_porcond (uplo n a lda af ldaf cmode c info work iwork)
  (declare (type (array f2cl-lib:integer4 (*)) iwork)
           (type (array double-float (*)) work c af a)
           (type (f2cl-lib:integer4) info cmode ldaf lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
```

```
    ((uplo character uplo-%data% uplo-%offset%)
     (a double-float a-%data% a-%offset%)
     (af double-float af-%data% af-%offset%)
     (c double-float c-%data% c-%offset%)
     (work double-float work-%data% work-%offset%)
     (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4)) (up nil)
         (ainvnm 0.0d0) (tmp 0.0d0) (kase 0) (i 0) (j 0) (dla_porcond 0.0d0))
    (declare (type (f2cl-lib:integer4) j i kase)
             (type (double-float) dla_porcond tmp ainvnm)
             (type f2cl-lib:logical up)
             (type (array f2cl-lib:integer4 (3)) isave))
    (setf dla_porcond 0.0d0)
    (setf info 0)
    (cond ((< n 0) (setf info -2)))
    (cond
     ((/= info 0) (xerbla "DLA_PORCOND" (f2cl-lib:int-sub info))
      (go end_label)))
    (cond ((= n 0) (setf dla_porcond 1.0d0) (go end_label)))
    (setf up f2cl-lib:%false%)
    (if (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)
        (setf up f2cl-lib:%true%))
    (cond
     (up
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf tmp 0.0d0)
          (cond
           ((= cmode 1)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j i) nil)
              (tagbody
                (setf tmp
                        (+ tmp
                           (abs
                            (*
                             (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                            a-%offset%)
                             (f2cl-lib:fref c-%data% (j) ((1 *))
                                            c-%offset%)))))
               label100001))
            (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf tmp
```

```
                (+ tmp
                   (abs
                    (*
                     (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%)
                     (f2cl-lib:fref c-%data% (j) ((1 *))
                                    c-%offset%)))))
    label100002)))
((= cmode 0)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j i) nil)
   (tagbody
     (setf tmp
                (+ tmp
                   (abs
                    (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                   a-%offset%))))
    label100003))
 (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf tmp
                (+ tmp
                   (abs
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))))
    label100004)))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j i) nil)
   (tagbody
     (setf tmp
                (+ tmp
                   (abs
                    (/
                     (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                    a-%offset%)
                     (f2cl-lib:fref c-%data% (j) ((1 *))
                                    c-%offset%)))))
    label100005))
 (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf tmp
                (+ tmp
                   (abs
                    (/
                     (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%)
                     (f2cl-lib:fref c-%data% (j) ((1 *))
```

```
                                        c-%offset%)))))
            label100006))))
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) i))
                             ((1 *)) work-%offset%)
              tmp)
      label100000)))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf tmp 0.0d0)
       (cond
        ((= cmode 1)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j i) nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (abs
                         (*
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref c-%data% (j) ((1 *))
                                         c-%offset%)))))
            label100008))
         (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (abs
                         (*
                          (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref c-%data% (j) ((1 *))
                                         c-%offset%)))))
            label100009)))
        ((= cmode 0)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j i) nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (abs
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%))))
            label100010))
         (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                       ((> j n) nil)
```

```
            (tagbody
              (setf tmp
                      (+ tmp
                         (abs
                          (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                         a-%offset%))))
             label100011)))
          (t
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j i) nil)
            (tagbody
              (setf tmp
                      (+ tmp
                         (abs
                          (/
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%)
                           (f2cl-lib:fref c-%data% (j) ((1 *))
                                          c-%offset%)))))
             label100012))
           (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                         ((> j n) nil)
            (tagbody
              (setf tmp
                      (+ tmp
                         (abs
                          (/
                           (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                          a-%offset%)
                           (f2cl-lib:fref c-%data% (j) ((1 *))
                                          c-%offset%)))))
             label100013))))
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) i))
                             ((1 *)) work-%offset%)
              tmp)
      label100007))))
  (setf ainvnm 0.0d0)
  (setf kase 0)
 label10
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacn2 n
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1)) ((1 *))
                             work-%offset%)
       work iwork ainvnm kase isave)
    (declare (ignore var-1 var-2 var-3 var-6))
    (setf n var-0)
    (setf ainvnm var-4)
    (setf kase var-5))
  (cond
```

```
((/= kase 0)
 (cond
  ((= kase 2)
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add
                                   (f2cl-lib:int-mul 2 n) i))
                                 ((1 *)) work-%offset%)))
      label100014))
   (cond
    (up
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (dpotrs "Upper" n 1 af ldaf work n info)
       (declare (ignore var-0 var-2 var-3 var-5))
       (when var-1 (setf n var-1))
       (when var-4 (setf ldaf var-4))
       (when var-6 (setf n var-6))
       (when var-7 (setf info var-7))))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (dpotrs "Lower" n 1 af ldaf work n info)
       (declare (ignore var-0 var-2 var-3 var-5))
       (when var-1 (setf n var-1))
       (when var-4 (setf ldaf var-4))
       (when var-6 (setf n var-6))
       (when var-7 (setf info var-7)))))
   (cond
    ((= cmode 1)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (/
                  (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
      label100015)))
    ((= cmode (f2cl-lib:int-sub 1))
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (*
                  (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
```

```
        label100016)))))
    (t
     (cond
      ((= cmode 1)
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (/
                  (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
         label100017)))
      ((= cmode (f2cl-lib:int-sub 1))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (*
                  (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
         label100018))))
     (cond
      (up
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
           (dpotrs "Upper" n 1 af ldaf work n info)
         (declare (ignore var-0 var-2 var-3 var-5))
         (when var-1 (setf n var-1))
         (when var-4 (setf ldaf var-4))
         (when var-6 (setf n var-6))
         (when var-7 (setf info var-7))))
      (t
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
           (dpotrs "Lower" n 1 af ldaf work n info)
         (declare (ignore var-0 var-2 var-3 var-5))
         (when var-1 (setf n var-1))
         (when var-4 (setf ldaf var-4))
         (when var-6 (setf n var-6))
         (when var-7 (setf info var-7)))))
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                    (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-add
                                     (f2cl-lib:int-mul 2 n) i))
                                   ((1 *)) work-%offset%)))
        label100019))))
```

```
        (go label10)))
      (if (/= ainvnm 0.0d0)
          (setf dla_porcond (/ 1.0d0 ainvnm)))
      (go end_label)
     end_label
      (return
       (values dla_porcond uplo n nil nil nil ldaf nil nil info nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dla_porcond
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*)))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::ldaf nil nil
                                              fortran-to-lisp::info nil nil)
                                            :calls '(fortran-to-lisp::dlacn2))))
```

———-

70.2 dla_porfsx_extended

70.2.1 Axiom unit tests

— dlaporfsxextended.input —

```
)set break resume
)sys rm -f dlaporfsxextended.output
)spool dlaporfsxextended.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

70.2.2 Axiom help page [Demm05]

— dla_porfsx_extended.help —

```
==========================================================================
dla_porfsx_extended examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dla_porfsx_extended (PREC_TYPE, UPLO, N, NRHS, A, LDA, AF,
           LDAF, COLEQU, C, B, LDB, Y, LDY, BERR_OUT, N_NORMS, ERR_BNDS_NORM,
           ERR_BNDS_COMP, RES, AYB, DY, Y_TAIL, RCOND, ITHRESH, RTHRESH,
           DZ_UB, IGNORE_CWISE, INFO)
           DLA_PORFSX_EXTENDED improves the computed solution to a system of
           linear equations for symmetric or Hermitian positive-definite
           matrices by performing extra-precise iterative refinement and
           provides error bounds and backward error estimates for the
           solution.

Function/Subroutine Documentation
   subroutine dla_porfsx_extended (integer PREC_TYPE, character UPLO, integer
       N, integer NRHS, double precision, dimension( lda, * ) A, integer LDA,
       double precision, dimension( ldaf, * ) AF, integer LDAF, logical
       COLEQU, double precision, dimension( * ) C, double precision,
       dimension( ldb, * ) B, integer LDB, double precision, dimension( ldy, *
       ) Y, integer LDY, double precision, dimension( * ) BERR_OUT, integer
       N_NORMS, double precision, dimension( nrhs, * ) ERR_BNDS_NORM, double
       precision, dimension( nrhs, * ) ERR_BNDS_COMP, double precision,
       dimension( * ) RES, double precision, dimension(*) AYB, double
       precision, dimension( * ) DY, double precision, dimension( * ) Y_TAIL,
       double precision RCOND, integer ITHRESH, double precision RTHRESH,
       double precision DZ_UB, logical IGNORE_CWISE, integer INFO)
       DLA_PORFSX_EXTENDED improves the computed solution to a system of
       linear equations for symmetric or Hermitian positive-definite matrices
       by performing extra-precise iterative refinement and provides error
```

```
       bounds and backward error estimates for the solution.

Purpose:
   DLA_PORFSX_EXTENDED improves the computed solution to a system of
   linear equations by performing extra-precise iterative refinement
   and provides error bounds and backward error estimates for the solution.
   This subroutine is called by DPORFSX to perform iterative refinement.
   In addition to normwise error bound, the code provides maximum
   componentwise error bound if possible. See comments for ERR_BNDS_NORM
   and ERR_BNDS_COMP for details of the error bounds. Note that this
   subroutine is only resonsible for setting the second fields of
   ERR_BNDS_NORM and ERR_BNDS_COMP.

Parameters:

   PREC_TYPE
   PREC_TYPE is INTEGER
   Specifies the intermediate precision to be used in refinement.
   The value is defined by ILAPREC(P) where P is a CHARACTER and
   P    = 'S':  Single
        = 'D':  Double
        = 'I':  Indigenous
        = 'X', 'E':  Extra

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right-hand-sides, i.e., the number of columns of the
   matrix B.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is DOUBLE PRECISION array, dimension (LDAF,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**T*U or A = L*L**T, as computed by DPOTRF.
```

```
LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

COLEQU
  COLEQU is LOGICAL
  If .TRUE. then column equilibration was done to A before calling
  this routine. This is needed to compute the solution and error
  bounds correctly.

C
  C is DOUBLE PRECISION array, dimension (N)
  The column scale factors for A. If COLEQU = .FALSE., C
  is not accessed. If C is input, each element of C should be a power
  of the radix to ensure a reliable solution and error estimates.
  Scaling by powers of the radix does not cause rounding errors unless
  the result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  The right-hand-side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

Y
  Y is DOUBLE PRECISION array, dimension (LDY,NRHS)
  On entry, the solution matrix X, as computed by DPOTRS.
  On exit, the improved solution matrix Y.

LDY
  LDY is INTEGER
  The leading dimension of the array Y.  LDY >= max(1,N).

BERR_OUT
  BERR_OUT is DOUBLE PRECISION array, dimension (NRHS)
  On exit, BERR_OUT(j) contains the componentwise relative backward
  error for right-hand-side j from the formula
  max(i) ( abs(RES(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
  where abs(Z) is the componentwise absolute value of the matrix
  or vector Z. This is computed by DLA_LIN_BERR.

N_NORMS
  N_NORMS is INTEGER
  Determines which error bounds to return (see ERR_BNDS_NORM
  and ERR_BNDS_COMP).
```

```
  If N_NORMS >= 1 return normwise error bounds.
  If N_NORMS >= 2 return componentwise error bounds.

ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
           max_j (abs(XTRUE(j,i) - X(j,i)))
            ------------------------------
                    max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
    reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

ERR_BNDS_COMP
  ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:
```

```
  Componentwise relative error in the ith solution vector:
                     abs(XTRUE(j,i) - X(j,i))
                max_j ----------------------
                            abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
    reciprocal condition number is less than the threshold
    sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
    almost certainly within a factor of 10 of the true error
    so long as the next entry is greater than the threshold
    sqrt(n) * slamch('Epsilon'). This error bound should only
    be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
    reciprocal condition number.  Compared with the threshold
    sqrt(n) * slamch('Epsilon') to determine if the error
    estimate is "guaranteed". These reciprocal condition
    numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
    appropriately scaled matrix Z.
    Let Z = S*(A*diag(x)), where x is the solution for the
    current right-hand side and S scales each row of
    A*diag(x) by a power of the radix so all absolute row
    sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

RES
  RES is DOUBLE PRECISION array, dimension (N)
  Workspace to hold the intermediate residual.

AYB
  AYB is DOUBLE PRECISION array, dimension (N)
  Workspace. This can be the same workspace passed for Y_TAIL.
```

```
DY
  DY is DOUBLE PRECISION array, dimension (N)
  Workspace to hold the intermediate solution.

Y_TAIL
  Y_TAIL is DOUBLE PRECISION array, dimension (N)
  Workspace to hold the trailing bits of the intermediate solution.

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

ITHRESH
  ITHRESH is INTEGER
  The maximum number of residual computations allowed for
  refinement. The default is 10. For 'aggressive' set to 100 to
  permit convergence using approximate factorizations or
  factorizations other than LU. If the factorization uses a
  technique other than Gaussian elimination, the guarantees in
  ERR_BNDS_NORM and ERR_BNDS_COMP may no longer be trustworthy.

RTHRESH
  RTHRESH is DOUBLE PRECISION
  Determines when to stop refinement if the error estimate stops
  decreasing. Refinement will stop when the next solution no longer
  satisfies norm(dx_{i+1}) < RTHRESH * norm(dx_i) where norm(Z) is
  the infinity norm of Z. RTHRESH satisfies 0 < RTHRESH <= 1. The
  default value is 0.5. For 'aggressive' set to 0.9 to permit
  convergence on extremely ill-conditioned matrices. See LAWN 165
  for more details.

DZ_UB
  DZ_UB is DOUBLE PRECISION
  Determines when to start considering componentwise convergence.
  Componentwise convergence is only considered after each component
  of the solution Y is stable, which we definte as the relative
  change in each component being less than DZ_UB. The default value
  is 0.25, requiring the first bit to be stable. See LAWN 165 for
  more details.

IGNORE_CWISE
  IGNORE_CWISE is LOGICAL
  If .TRUE. then ignore componentwise convergence. Default value
  is .FALSE..
```

```
 INFO
   INFO is INTEGER
   = 0:  Successful exit.
   < 0:  if INFO = -i, the ith argument to DPOTRS had an illegal value
```

———-

70.2.3 fortran code

— dla_porfsx_extended.f —

```
*  =====================================================================
      SUBROUTINE DLA_PORFSX_EXTENDED( PREC_TYPE, UPLO, N, NRHS, A, LDA,
     $                                AF, LDAF, COLEQU, C, B, LDB, Y,
     $                                LDY, BERR_OUT, N_NORMS,
     $                                ERR_BNDS_NORM, ERR_BNDS_COMP, RES,
     $                                AYB, DY, Y_TAIL, RCOND, ITHRESH,
     $                                RTHRESH, DZ_UB, IGNORE_CWISE,
     $                                INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDAF, LDB, LDY, N, NRHS, PREC_TYPE,
     $                   N_NORMS, ITHRESH
      CHARACTER          UPLO
      LOGICAL            COLEQU, IGNORE_CWISE
      DOUBLE PRECISION   RTHRESH, DZ_UB
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   Y( LDY, * ), RES( * ), DY( * ), Y_TAIL( * )
      DOUBLE PRECISION   C( * ), AYB(*), RCOND, BERR_OUT( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            UPLO2, CNT, I, J, X_STATE, Z_STATE
      DOUBLE PRECISION   YK, DYK, YMIN, NORMY, NORMX, NORMDX, DXRAT,
     $                   DZRAT, PREVNORMDX, PREV_DZ_Z, DXRATMAX,
```

```
     $                   DZRATMAX, DX_X, DZ_Z, FINAL_DX_X, FINAL_DZ_Z,
     $                   EPS, HUGEVAL, INCR_THRESH
      LOGICAL            INCR_PREC
*     ..
*     .. Parameters ..
      INTEGER           UNSTABLE_STATE, WORKING_STATE, CONV_STATE,
     $                  NOPROG_STATE, Y_PREC_STATE, BASE_RESIDUAL,
     $                  EXTRA_RESIDUAL, EXTRA_Y
      PARAMETER         ( UNSTABLE_STATE = 0, WORKING_STATE = 1,
     $                  CONV_STATE = 2, NOPROG_STATE = 3 )
      PARAMETER         ( BASE_RESIDUAL = 0, EXTRA_RESIDUAL = 1,
     $                  EXTRA_Y = 2 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           ILAUPLO
      INTEGER            ILAUPLO
*     ..
*     .. External Subroutines ..
      EXTERNAL          DAXPY, DCOPY, DPOTRS, DSYMV, BLAS_DSYMV_X,
     $                  BLAS_DSYMV2_X, DLA_SYAMV, DLA_WWADDW,
     $                  DLA_LIN_BERR
      DOUBLE PRECISION   DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC         ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      IF (INFO.NE.0) RETURN
      EPS = DLAMCH( 'Epsilon' )
      HUGEVAL = DLAMCH( 'Overflow' )
*     Force HUGEVAL to Inf
```

```
      HUGEVAL = HUGEVAL * HUGEVAL
*     Using HUGEVAL may lead to spurious underflows.
      INCR_THRESH = DBLE( N ) * EPS

      IF ( LSAME ( UPLO, 'L' ) ) THEN
         UPLO2 = ILAUPLO( 'L' )
      ELSE
         UPLO2 = ILAUPLO( 'U' )
      ENDIF

      DO J = 1, NRHS
         Y_PREC_STATE = EXTRA_RESIDUAL
         IF ( Y_PREC_STATE .EQ. EXTRA_Y ) THEN
            DO I = 1, N
               Y_TAIL( I ) = 0.0D+0
            END DO
         END IF

         DXRAT = 0.0D+0
         DXRATMAX = 0.0D+0
         DZRAT = 0.0D+0
         DZRATMAX = 0.0D+0
         FINAL_DX_X = HUGEVAL
         FINAL_DZ_Z = HUGEVAL
         PREVNORMDX = HUGEVAL
         PREV_DZ_Z = HUGEVAL
         DZ_Z = HUGEVAL
         DX_X = HUGEVAL

         X_STATE = WORKING_STATE
         Z_STATE = UNSTABLE_STATE
         INCR_PREC = .FALSE.

         DO CNT = 1, ITHRESH
*
*         Compute residual RES = B_s - op(A_s) * Y,
*             op(A) = A, A**T, or A**H depending on TRANS (and type).
*
            CALL DCOPY( N, B( 1, J ), 1, RES, 1 )
            IF ( Y_PREC_STATE .EQ. BASE_RESIDUAL ) THEN
               CALL DSYMV( UPLO, N, -1.0D+0, A, LDA, Y(1,J), 1,
     $              1.0D+0, RES, 1 )
            ELSE IF ( Y_PREC_STATE .EQ. EXTRA_RESIDUAL ) THEN
               CALL BLAS_DSYMV_X( UPLO2, N, -1.0D+0, A, LDA,
     $              Y( 1, J ), 1, 1.0D+0, RES, 1, PREC_TYPE )
            ELSE
               CALL BLAS_DSYMV2_X(UPLO2, N, -1.0D+0, A, LDA,
     $              Y(1, J), Y_TAIL, 1, 1.0D+0, RES, 1, PREC_TYPE)
            END IF
```

```
!         XXX: RES is no longer needed.
            CALL DCOPY( N, RES, 1, DY, 1 )
            CALL DPOTRS( UPLO, N, 1, AF, LDAF, DY, N, INFO )
*
*         Calculate relative changes DX_X, DZ_Z and ratios DXRAT, DZRAT.
*
            NORMX = 0.0D+0
            NORMY = 0.0D+0
            NORMDX = 0.0D+0
            DZ_Z = 0.0D+0
            YMIN = HUGEVAL

            DO I = 1, N
               YK = ABS( Y( I, J ) )
               DYK = ABS( DY( I ) )

               IF ( YK .NE. 0.0D+0 ) THEN
                  DZ_Z = MAX( DZ_Z, DYK / YK )
               ELSE IF ( DYK .NE. 0.0D+0 ) THEN
                  DZ_Z = HUGEVAL
               END IF

               YMIN = MIN( YMIN, YK )

               NORMY = MAX( NORMY, YK )

               IF ( COLEQU ) THEN
                  NORMX = MAX( NORMX, YK * C( I ) )
                  NORMDX = MAX( NORMDX, DYK * C( I ) )
               ELSE
                  NORMX = NORMY
                  NORMDX = MAX( NORMDX, DYK )
               END IF
            END DO

            IF ( NORMX .NE. 0.0D+0 ) THEN
               DX_X = NORMDX / NORMX
            ELSE IF ( NORMDX .EQ. 0.0D+0 ) THEN
               DX_X = 0.0D+0
            ELSE
               DX_X = HUGEVAL
            END IF

            DXRAT = NORMDX / PREVNORMDX
            DZRAT = DZ_Z / PREV_DZ_Z
*
*         Check termination criteria.
*
            IF ( YMIN*RCOND .LT. INCR_THRESH*NORMY
     $           .AND. Y_PREC_STATE .LT. EXTRA_Y )
```

```
$           INCR_PREC = .TRUE.

      IF ( X_STATE .EQ. NOPROG_STATE .AND. DXRAT .LE. RTHRESH )
$           X_STATE = WORKING_STATE
      IF ( X_STATE .EQ. WORKING_STATE ) THEN
         IF ( DX_X .LE. EPS ) THEN
            X_STATE = CONV_STATE
         ELSE IF ( DXRAT .GT. RTHRESH ) THEN
            IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
               INCR_PREC = .TRUE.
            ELSE
               X_STATE = NOPROG_STATE
            END IF
         ELSE
            IF ( DXRAT .GT. DXRATMAX ) DXRATMAX = DXRAT
         END IF
         IF ( X_STATE .GT. WORKING_STATE ) FINAL_DX_X = DX_X
      END IF

      IF ( Z_STATE .EQ. UNSTABLE_STATE .AND. DZ_Z .LE. DZ_UB )
$           Z_STATE = WORKING_STATE
      IF ( Z_STATE .EQ. NOPROG_STATE .AND. DZRAT .LE. RTHRESH )
$           Z_STATE = WORKING_STATE
      IF ( Z_STATE .EQ. WORKING_STATE ) THEN
         IF ( DZ_Z .LE. EPS ) THEN
            Z_STATE = CONV_STATE
         ELSE IF ( DZ_Z .GT. DZ_UB ) THEN
            Z_STATE = UNSTABLE_STATE
            DZRATMAX = 0.0D+0
            FINAL_DZ_Z = HUGEVAL
         ELSE IF ( DZRAT .GT. RTHRESH ) THEN
            IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
               INCR_PREC = .TRUE.
            ELSE
               Z_STATE = NOPROG_STATE
            END IF
         ELSE
            IF ( DZRAT .GT. DZRATMAX ) DZRATMAX = DZRAT
         END IF
         IF ( Z_STATE .GT. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
      END IF

      IF ( X_STATE.NE.WORKING_STATE.AND.
$           ( IGNORE_CWISE.OR.Z_STATE.NE.WORKING_STATE ) )
$           GOTO 666

      IF ( INCR_PREC ) THEN
         INCR_PREC = .FALSE.
         Y_PREC_STATE = Y_PREC_STATE + 1
         DO I = 1, N
```

```
                 Y_TAIL( I ) = 0.0D+0
              END DO
           END IF

           PREVNORMDX = NORMDX
           PREV_DZ_Z = DZ_Z
*
*          Update soluton.
*
           IF (Y_PREC_STATE .LT. EXTRA_Y) THEN
              CALL DAXPY( N, 1.0D+0, DY, 1, Y(1,J), 1 )
           ELSE
              CALL DLA_WWADDW( N, Y( 1, J ), Y_TAIL, DY )
           END IF

        END DO
*       Target of "IF (Z_STOP .AND. X_STOP)".  Sun's f77 won't EXIT.
 666    CONTINUE
*
*     Set final_* when cnt hits ithresh.
*
        IF ( X_STATE .EQ. WORKING_STATE ) FINAL_DX_X = DX_X
        IF ( Z_STATE .EQ. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
*
*     Compute error bounds.
*
        IF ( N_NORMS .GE. 1 ) THEN
           ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) =
     $           FINAL_DX_X / (1 - DXRATMAX)
        END IF
        IF ( N_NORMS .GE. 2 ) THEN
           ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) =
     $           FINAL_DZ_Z / (1 - DZRATMAX)
        END IF
*
*     Compute componentwise relative backward error from formula
*         max(i) ( abs(R(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
*     where abs(Z) is the componentwise absolute value of the matrix
*     or vector Z.
*
*       Compute residual RES = B_s - op(A_s) * Y,
*           op(A) = A, A**T, or A**H depending on TRANS (and type).
*
        CALL DCOPY( N, B( 1, J ), 1, RES, 1 )
        CALL DSYMV( UPLO, N, -1.0D+0, A, LDA, Y(1,J), 1, 1.0D+0, RES,
     $     1 )

        DO I = 1, N
           AYB( I ) = ABS( B( I, J ) )
        END DO
```

```
*
*     Compute abs(op(A_s))*abs(Y) + abs(B_s).
*
         CALL DLA_SYAMV( UPLO2, N, 1.0D+0,
     $        A, LDA, Y(1, J), 1, 1.0D+0, AYB, 1 )

         CALL DLA_LIN_BERR( N, N, 1, RES, AYB, BERR_OUT( J ) )
*
*     End of loop for each RHS.
*
      END DO
*
      RETURN
      END
```

———-

70.2.4 lisp code

— **dla_porfsx_extended.lisp** —

```
(let* ((unstable_state 0)
       (working_state 1)
       (conv_state 2)
       (noprog_state 3)
       (base_residual 0)
       (extra_residual 1)
       (extra_y 2)
       (la_linrx_err_i 2))
  (declare (type (f2cl-lib:integer4 0 0) unstable_state)
           (type (f2cl-lib:integer4 1 1) working_state)
           (type (f2cl-lib:integer4 2 2) conv_state)
           (type (f2cl-lib:integer4 3 3) noprog_state)
           (type (f2cl-lib:integer4 0 0) base_residual)
           (type (f2cl-lib:integer4 1 1) extra_residual)
           (type (f2cl-lib:integer4 2 2) extra_y)
           (type (f2cl-lib:integer4 2 2) la_linrx_err_i)
           (ignorable unstable_state working_state conv_state noprog_state
            base_residual extra_residual extra_y la_linrx_err_i))
  (defun dla_porfsx_extended
         (prec_type uplo n nrhs a lda af ldaf colequ c b ldb$ y ldy berr_out
          n_norms err_bnds_norm err_bnds_comp res ayb dy y_tail rcond ithresh
          rthresh dz_ub ignore_cwise info)
    (declare (type (double-float) dz_ub rthresh rcond)
             (type f2cl-lib:logical ignore_cwise colequ)
             (type (array double-float (*)) y_tail dy ayb res err_bnds_comp
              err_bnds_norm berr_out y b c af a)
             (type (string *) uplo)
```

```
          (type (f2cl-lib:integer4) info ithresh n_norms ldy ldb$ ldaf lda
           nrhs n prec_type))
(f2cl-lib:with-multi-array-data
    ((uplo character uplo-%data% uplo-%offset%)
     (a double-float a-%data% a-%offset%)
     (af double-float af-%data% af-%offset%)
     (c double-float c-%data% c-%offset%)
     (b double-float b-%data% b-%offset%)
     (y double-float y-%data% y-%offset%)
     (berr_out double-float berr_out-%data% berr_out-%offset%)
     (err_bnds_norm double-float err_bnds_norm-%data%
      err_bnds_norm-%offset%)
     (err_bnds_comp double-float err_bnds_comp-%data%
      err_bnds_comp-%offset%)
     (res double-float res-%data% res-%offset%)
     (ayb double-float ayb-%data% ayb-%offset%)
     (dy double-float dy-%data% dy-%offset%)
     (y_tail double-float y_tail-%data% y_tail-%offset%))
  (prog ((y_prec_state 0) (incr_prec nil) (yk 0.0d0) (dyk 0.0d0)
         (ymin 0.0d0) (normy 0.0d0) (normx 0.0d0) (normdx 0.0d0)
         (dxrat 0.0d0) (dzrat 0.0d0) (prevnormdx 0.0d0) (prev_dz_z 0.0d0)
         (dxratmax 0.0d0) (dzratmax 0.0d0) (dx_x 0.0d0) (dz_z 0.0d0)
         (final_dx_x 0.0d0) (final_dz_z 0.0d0) (eps 0.0d0) (hugeval 0.0d0)
         (incr_thresh 0.0d0) (uplo2 0) (cnt 0) (i 0) (j 0) (x_state 0)
         (z_state 0))
    (declare (type f2cl-lib:logical incr_prec)
             (type (double-float) yk dyk ymin normy normx normdx dxrat
              dzrat prevnormdx prev_dz_z dxratmax dzratmax dx_x dz_z
              final_dx_x final_dz_z eps hugeval incr_thresh)
             (type (f2cl-lib:integer4) y_prec_state uplo2 cnt i j x_state
              z_state))
    (if (/= info 0)
        (go end_label))
    (setf eps (dlamch "Epsilon"))
    (setf hugeval (dlamch "Overflow"))
    (setf hugeval (* hugeval hugeval))
    (setf incr_thresh (* (f2cl-lib:dble n) eps))
    (cond
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame uplo "L")
        (declare (ignore var-1))
        (when var-0 (setf uplo var-0))
        ret-val)
      (setf uplo2 (ilauplo "L")))
     (t (setf uplo2 (ilauplo "U"))))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (setf y_prec_state extra_residual)
        (cond
```

```
 ((= y_prec_state extra_y)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                           y_tail-%offset%)
            0.0d0)
     label100001))))
(setf dxrat 0.0d0)
(setf dxratmax 0.0d0)
(setf dzrat 0.0d0)
(setf dzratmax 0.0d0)
(setf final_dx_x hugeval)
(setf final_dz_z hugeval)
(setf prevnormdx hugeval)
(setf prev_dz_z hugeval)
(setf dz_z hugeval)
(setf dx_x hugeval)
(setf x_state working_state)
(setf z_state unstable_state)
(setf incr_prec f2cl-lib:%false%)
(f2cl-lib:fdo (cnt 1 (f2cl-lib:int-add cnt 1))
              ((> cnt ithresh) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy n
         (f2cl-lib:array-slice b-%data% double-float (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1 res 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (cond
     ((= y_prec_state base_residual)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9)
          (dsymv uplo n -1.0d0 a lda
           (f2cl-lib:array-slice y-%data% double-float (1 j)
                                 ((1 ldy) (1 *)) y-%offset%)
           1 1.0d0 res 1)
        (declare
         (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
        (when var-0 (setf uplo var-0))
        (when var-1 (setf n var-1))
        (when var-4 (setf lda var-4))))
     ((= y_prec_state extra_residual)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (blas_dsymv_x uplo2 n -1.0d0 a lda
```

```
        (f2cl-lib:array-slice y-%data% double-float (1 j)
                              ((1 ldy) (1 *)) y-%offset%)
        1 1.0d0 res 1 prec_type)
    (declare
     (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
    (when var-0 (setf uplo2 var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf lda var-4))
    (when var-10 (setf prec_type var-10))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11)
      (blas_dsymv2_x uplo2 n -1.0d0 a lda
       (f2cl-lib:array-slice y-%data% double-float (1 j)
                             ((1 ldy) (1 *)) y-%offset%)
       y_tail 1 1.0d0 res 1 prec_type)
    (declare
     (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9 var-10))
    (when var-0 (setf uplo2 var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf lda var-4))
    (when var-11 (setf prec_type var-11)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dcopy n res 1 dy 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dpotrs uplo n 1 af ldaf dy n info)
  (declare (ignore var-2 var-3 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-4 (setf ldaf var-4))
  (when var-6 (setf n var-6))
  (when var-7 (setf info var-7)))
(setf normx 0.0d0)
(setf normy 0.0d0)
(setf normdx 0.0d0)
(setf dz_z 0.0d0)
(setf ymin hugeval)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf yk
            (abs
             (f2cl-lib:fref y-%data% (i j) ((1 ldy) (1 *))
                            y-%offset%)))
    (setf dyk
            (abs
```

```
                (f2cl-lib:fref dy-%data% (i) ((1 *))
                               dy-%offset%)))
    (cond ((/= yk 0.0d0) (setf dz_z (max dz_z (/ dyk yk))))
          ((/= dyk 0.0d0) (setf dz_z hugeval)))
    (setf ymin (min ymin yk))
    (setf normy (max normy yk))
    (cond
     (colequ
      (setf normx
              (max normx
                   (* yk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%))))
      (setf normdx
              (max normdx
                   (* dyk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%)))))
     (t (setf normx normy) (setf normdx (max normdx dyk))))
   label100003))
(cond ((/= normx 0.0d0) (setf dx_x (/ normdx normx)))
      ((= normdx 0.0d0) (setf dx_x 0.0d0))
      (t (setf dx_x hugeval)))
(setf dxrat (/ normdx prevnormdx))
(setf dzrat (/ dz_z prev_dz_z))
(if (and (< (* ymin rcond) (* incr_thresh normy))
         (< y_prec_state extra_y))
    (setf incr_prec f2cl-lib:%true%))
(if (and (= x_state noprog_state) (<= dxrat rthresh))
    (setf x_state working_state))
(cond
 ((= x_state working_state)
  (cond ((<= dx_x eps) (setf x_state conv_state))
        ((> dxrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf x_state noprog_state))))
        (t
         (if (> dxrat dxratmax)
             (setf dxratmax dxrat))))
  (if (> x_state working_state)
      (setf final_dx_x dx_x))))
(if (and (= z_state unstable_state) (<= dz_z dz_ub))
    (setf z_state working_state))
(if (and (= z_state noprog_state) (<= dzrat rthresh))
    (setf z_state working_state))
(cond
 ((= z_state working_state)
  (cond ((<= dz_z eps) (setf z_state conv_state))
```

```
                  ((> dz_z dz_ub) (setf z_state unstable_state)
                   (setf dzratmax 0.0d0) (setf final_dz_z hugeval))
                  ((> dzrat rthresh)
                   (cond
                    ((/= y_prec_state extra_y)
                     (setf incr_prec f2cl-lib:%true%))
                    (t (setf z_state noprog_state))))
                  (t
                   (if (> dzrat dzratmax)
                       (setf dzratmax dzrat))))
          (if (> z_state working_state)
              (setf final_dz_z dz_z))))
      (if (and (/= x_state working_state)
               (or ignore_cwise (/= z_state working_state)))
          (go label666))
      (cond
       (incr_prec (setf incr_prec f2cl-lib:%false%)
        (setf y_prec_state (f2cl-lib:int-add y_prec_state 1))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                                 y_tail-%offset%)
                  0.0d0)
           label100004))))
      (setf prevnormdx normdx)
      (setf prev_dz_z dz_z)
      (cond
       ((< y_prec_state extra_y)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (daxpy n 1.0d0 dy 1
             (f2cl-lib:array-slice y-%data% double-float (1 j)
                                   ((1 ldy) (1 *)) y-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-4 var-5))
          (when var-0 (setf n var-0))))
       (t
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dla_wwaddw n
             (f2cl-lib:array-slice y-%data% double-float (1 j)
                                   ((1 ldy) (1 *)) y-%offset%)
             y_tail dy)
          (declare (ignore var-1 var-2 var-3))
          (when var-0 (setf n var-0)))))
    label100002))
label666
 (if (= x_state working_state)
     (setf final_dx_x dx_x))
 (if (= z_state working_state)
     (setf final_dz_z dz_z))
```

```
      (cond
       ((>= n_norms 1)
        (setf (f2cl-lib:fref err_bnds_norm-%data% (j la_linrx_err_i)
                             ((1 nrhs) (1 *)) err_bnds_norm-%offset%)
              (/ final_dx_x (- 1 dxratmax)))))
      (cond
       ((>= n_norms 2)
        (setf (f2cl-lib:fref err_bnds_comp-%data% (j la_linrx_err_i)
                             ((1 nrhs) (1 *)) err_bnds_comp-%offset%)
              (/ final_dz_z (- 1 dzratmax)))))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy n
           (f2cl-lib:array-slice b-%data% double-float (1 j)
                                 ((1 ldb$) (1 *)) b-%offset%)
           1 res 1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf n var-0)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (dsymv uplo n -1.0d0 a lda
           (f2cl-lib:array-slice y-%data% double-float (1 j)
                                 ((1 ldy) (1 *)) y-%offset%)
           1 1.0d0 res 1)
        (declare (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
        (when var-0 (setf uplo var-0))
        (when var-1 (setf n var-1))
        (when var-4 (setf lda var-4)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref ayb-%data% (i) ((1 *)) ayb-%offset%)
                  (abs
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)))
         label100005))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (dla_syamv uplo2 n 1.0d0 a lda
           (f2cl-lib:array-slice y-%data% double-float (1 j)
                                 ((1 ldy) (1 *)) y-%offset%)
           1 1.0d0 ayb 1)
        (declare (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
        (when var-0 (setf uplo2 var-0))
        (when var-1 (setf n var-1))
        (when var-4 (setf lda var-4)))
      (dla_lin_berr n n 1 res ayb
       (f2cl-lib:array-slice berr_out-%data% double-float (j) ((1 *))
                             berr_out-%offset%))
    label100000))
  (go end_label)
```

```
      end_label
       (return
        (values prec_type uplo n nil nil lda nil ldaf nil nil nil nil nil nil
                nil nil nil nil nil nil nil nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dla_porfsx_extended
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (string)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4)
                                             fortran-to-lisp::logical
                                             (array double-float (*))
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (array double-float (*))
                                             (array double-float (*))
                                             (array double-float (*))
                                             (array double-float (*))
                                             (array double-float (*))
                                             (double-float)
                                             (fortran-to-lisp::integer4)
                                             (double-float) (double-float)
                                             fortran-to-lisp::logical
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(fortran-to-lisp::prec_type
                                             fortran-to-lisp::uplo
                                             fortran-to-lisp::n nil nil
                                             fortran-to-lisp::lda nil
                                             fortran-to-lisp::ldaf nil nil nil
                                             nil nil nil nil nil nil nil nil
                                             nil nil nil nil nil nil nil nil
                                             fortran-to-lisp::info)
                                           :calls
                                           '(fortran-to-lisp::dla_lin_berr))))
```

———-

70.3 dla_porpvgrw

70.3.1 Axiom unit tests

— **dlaporpvgrw.input** —

```
)set break resume
)sys rm -f dlaporpvgrw.output
)spool dlaporpvgrw.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

70.3.2 Axiom help page

— **dla_porpvgrw.help** —

```
====================================================================
dla_porpvgrw examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function dla_porpvgrw (UPLO, NCOLS, A, LDA, AF, LDAF,
           WORK)
           DLA_PORPVGRW computes the reciprocal pivot growth factor
           norm(A)/norm(U) for a symmetric or Hermitian positive-definite
           matrix.

Function/Subroutine Documentation
   double precision function dla_porpvgrw (character*1 UPLO, integer NCOLS,
```

```
      double precision, dimension( lda, * ) A, integer LDA, double precision,
      dimension( ldaf, * ) AF, integer LDAF, double precision, dimension( * )
      WORK)
      DLA_PORPVGRW computes the reciprocal pivot growth factor
      norm(A)/norm(U) for a symmetric or Hermitian positive-definite matrix.

Purpose:
   DLA_PORPVGRW computes the reciprocal pivot growth factor
   norm(A)/norm(U). The "max absolute element" norm is used. If this is
   much less than 1, the stability of the LU factorization of the
   (equilibrated) matrix A could be poor. This also means that the
   solution X, estimated condition numbers, and error bounds could be
   unreliable.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 NCOLS
   NCOLS is INTEGER
   The number of columns of the matrix A. NCOLS >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is DOUBLE PRECISION array, dimension (LDAF,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**T*U or A = L*L**T, as computed by DPOTRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 WORK
   WORK is DOUBLE PRECISION array, dimension (2*N)
```

70.3.3 fortran code

— dla_porpvgrw.f —

```
*  ====================================================================
      DOUBLE PRECISION FUNCTION DLA_PORPVGRW( UPLO, NCOLS, A, LDA, AF,
     $                                        LDAF, WORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER*1        UPLO
      INTEGER            NCOLS, LDA, LDAF
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * ), WORK( * )
*     ..
*
*  ====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   AMAX, UMAX, RPVGRW
      LOGICAL            UPPER
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, DLASET
      LOGICAL            LSAME
*     ..
*     .. Executable Statements ..
*
      UPPER = LSAME( 'Upper', UPLO )
*
*     DPOTRF will have factored only the NCOLSxNCOLS leading minor, so
*     we restrict the growth search to that minor and use only the first
*     2*NCOLS workspace entries.
*
      RPVGRW = 1.0D+0
      DO I = 1, 2*NCOLS
         WORK( I ) = 0.0D+0
      END DO
*
*     Find the max magnitude entry of each column.
```

```
*
      IF ( UPPER ) THEN
         DO J = 1, NCOLS
            DO I = 1, J
               WORK( NCOLS+J ) =
     $              MAX( ABS( A( I, J ) ), WORK( NCOLS+J ) )
            END DO
         END DO
      ELSE
         DO J = 1, NCOLS
            DO I = J, NCOLS
               WORK( NCOLS+J ) =
     $              MAX( ABS( A( I, J ) ), WORK( NCOLS+J ) )
            END DO
         END DO
      END IF
*
*     Now find the max magnitude entry of each column of the factor in
*     AF.  No pivoting, so no permutations.
*
      IF ( LSAME( 'Upper', UPLO ) ) THEN
         DO J = 1, NCOLS
            DO I = 1, J
               WORK( J ) = MAX( ABS( AF( I, J ) ), WORK( J ) )
            END DO
         END DO
      ELSE
         DO J = 1, NCOLS
            DO I = J, NCOLS
               WORK( J ) = MAX( ABS( AF( I, J ) ), WORK( J ) )
            END DO
         END DO
      END IF
*
*     Compute the *inverse* of the max element growth factor.  Dividing
*     by zero would imply the largest entry of the factor's column is
*     zero.  Than can happen when either the column of A is zero or
*     massive pivots made the factor underflow to zero.  Neither counts
*     as growth in itself, so simply ignore terms with zero
*     denominators.
*
      IF ( LSAME( 'Upper', UPLO ) ) THEN
         DO I = 1, NCOLS
            UMAX = WORK( I )
            AMAX = WORK( NCOLS+I )
            IF ( UMAX /= 0.0D+0 ) THEN
               RPVGRW = MIN( AMAX / UMAX, RPVGRW )
            END IF
         END DO
      ELSE
```

```
         DO I = 1, NCOLS
            UMAX = WORK( I )
            AMAX = WORK( NCOLS+I )
            IF ( UMAX /= 0.0D+0 ) THEN
               RPVGRW = MIN( AMAX / UMAX, RPVGRW )
            END IF
         END DO
      END IF

      DLA_PORPVGRW = RPVGRW
      END
```

———-

70.3.4 lisp code

— dla_porpvgrw.lisp —

```
(defun dla_porpvgrw (uplo ncols a lda af ldaf work)
  (declare (type (array double-float (*)) work af a)
           (type (f2cl-lib:integer4) ldaf lda ncols)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a double-float a-%data% a-%offset%)
       (af double-float af-%data% af-%offset%)
       (work double-float work-%data% work-%offset%))
    (prog ((upper nil) (amax 0.0d0) (umax 0.0d0) (rpvgrw 0.0d0) (i 0) (j 0)
           (dla_porpvgrw 0.0d0))
      (declare (type (f2cl-lib:integer4) j i)
               (type (double-float) dla_porpvgrw rpvgrw umax amax)
               (type f2cl-lib:logical upper))
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame "Upper" uplo)
                (declare (ignore var-0))
                (when var-1 (setf uplo var-1))
                ret-val))
      (setf rpvgrw 1.0d0)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i (f2cl-lib:int-mul 2 ncols)) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) 0.0d0)
         label100000))
      (cond
       (upper
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j ncols) nil)
```

```
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i j) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ncols j))
                                 ((1 *)) work-%offset%)
                  (max
                   (abs
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add ncols j)) ((1 *))
                                  work-%offset%)))
           label100002))
       label100001)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j ncols) nil)
      (tagbody
        (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                      ((> i ncols) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ncols j))
                                 ((1 *)) work-%offset%)
                  (max
                   (abs
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add ncols j)) ((1 *))
                                  work-%offset%)))
           label100004))
       label100003))))
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame "Upper" uplo)
     (declare (ignore var-0))
     (when var-1 (setf uplo var-1))
     ret-val)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j ncols) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i j) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                  (max
                   (abs
                    (f2cl-lib:fref af-%data% (i j) ((1 ldaf) (1 *))
                                   af-%offset%))
```

```
                    (f2cl-lib:fref work-%data% (j) ((1 *))
                                   work-%offset%)))
          label100006))
      label100005)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j ncols) nil)
     (tagbody
       (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                     ((> i ncols) nil)
         (tagbody
           (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                   (max
                    (abs
                     (f2cl-lib:fref af-%data% (i j) ((1 ldaf) (1 *))
                                    af-%offset%))
                    (f2cl-lib:fref work-%data% (j) ((1 *))
                                   work-%offset%)))
          label100008))
      label100007))))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame "Upper" uplo)
    (declare (ignore var-0))
    (when var-1 (setf uplo var-1))
    ret-val)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ncols) nil)
    (tagbody
      (setf umax (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
      (setf amax
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ncols i))
                             ((1 *)) work-%offset%))
      (cond
       ((f2cl-lib:f2cl/ umax (= 0.0d0))
        (setf rpvgrw (min (/ amax umax) rpvgrw))))
     label100009)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i ncols) nil)
    (tagbody
      (setf umax (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
      (setf amax
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ncols i))
                             ((1 *)) work-%offset%))
      (cond
       ((f2cl-lib:f2cl/ umax (= 0.0d0))
        (setf rpvgrw (min (/ amax umax) rpvgrw))))
     label100010))))
(setf dla_porpvgrw rpvgrw)
```

```
     end_label
      (return (values dla_porpvgrw uplo nil nil nil nil nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dla_porpvgrw
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*)))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil nil
                                              nil nil nil)
                                            :calls 'nil)))
```

———-

70.4 dpocon

70.4.1 Axiom unit tests

— **dpocon.input** —

```
)set break resume
)sys rm -f dpocon.output
)spool dpocon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

70.4.2 Axiom help page

— dpocon.help —

```
====================================================================
dpocon examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dpocon (UPLO, N, A, LDA, ANORM, RCOND, WORK, IWORK, INFO)
           DPOCON

Function/Subroutine Documentation
   subroutine dpocon (character UPLO, integer N, double precision, dimension(
       lda, * ) A, integer LDA, double precision ANORM, double precision
       RCOND, double precision, dimension( * ) WORK, integer, dimension( * )
       IWORK, integer INFO)
       DPOCON

Purpose:
   DPOCON estimates the reciprocal of the condition number (in the
   1-norm) of a real symmetric positive definite matrix using the
   Cholesky factorization A = U**T*U or A = L*L**T computed by DPOTRF.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**T*U or A = L*L**T, as computed by DPOTRF.

 LDA
```

```
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

ANORM
  ANORM is DOUBLE PRECISION
  The 1-norm (or infinity-norm) of the symmetric matrix A.

RCOND
  RCOND is DOUBLE PRECISION
  The reciprocal of the condition number of the matrix A,
  computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is an
  estimate of the 1-norm of inv(A) computed in this routine.

WORK
  WORK is DOUBLE PRECISION array, dimension (3*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

70.4.3 fortran code

— dpocon.f —

```
*  =====================================================================
      SUBROUTINE DPOCON( UPLO, N, A, LDA, ANORM, RCOND, WORK, IWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
      DOUBLE PRECISION   ANORM, RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), WORK( * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      CHARACTER          NORMIN
      INTEGER            IX, KASE
      DOUBLE PRECISION   AINVNM, SCALE, SCALEL, SCALEU, SMLNUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, IDAMAX, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLACN2, DLATRS, DRSCL, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -5
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DPOCON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
```

```
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
      SMLNUM = DLAMCH( 'Safe minimum' )
*
*     Estimate the 1-norm of inv(A).
*
      KASE = 0
      NORMIN = 'N'
   10 CONTINUE
      CALL DLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( UPPER ) THEN
*
*           Multiply by inv(U**T).
*
            CALL DLATRS( 'Upper', 'Transpose', 'Non-unit', NORMIN, N, A,
     $                   LDA, WORK, SCALEL, WORK( 2*N+1 ), INFO )
            NORMIN = 'Y'
*
*           Multiply by inv(U).
*
            CALL DLATRS( 'Upper', 'No transpose', 'Non-unit', NORMIN, N,
     $                   A, LDA, WORK, SCALEU, WORK( 2*N+1 ), INFO )
         ELSE
*
*           Multiply by inv(L).
*
            CALL DLATRS( 'Lower', 'No transpose', 'Non-unit', NORMIN, N,
     $                   A, LDA, WORK, SCALEL, WORK( 2*N+1 ), INFO )
            NORMIN = 'Y'
*
*           Multiply by inv(L**T).
*
            CALL DLATRS( 'Lower', 'Transpose', 'Non-unit', NORMIN, N, A,
     $                   LDA, WORK, SCALEU, WORK( 2*N+1 ), INFO )
         END IF
*
*        Multiply by 1/SCALE if doing so will not cause overflow.
*
         SCALE = SCALEL*SCALEU
         IF( SCALE.NE.ONE ) THEN
            IX = IDAMAX( N, WORK, 1 )
            IF( SCALE.LT.ABS( WORK( IX ) )*SMLNUM .OR. SCALE.EQ.ZERO )
```

```
     $         GO TO 20
            CALL DRSCL( N, SCALE, WORK, 1 )
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
   20 CONTINUE
      RETURN
*
*     End of DPOCON
*
      END
```

———-

70.4.4 lisp code

— dpocon.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dpocon (uplo n a lda anorm rcond work iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (double-float) rcond anorm)
             (type (array double-float (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (work double-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (ainvnm 0.0d0) (scale 0.0d0) (scalel 0.0d0) (scaleu 0.0d0)
             (smlnum 0.0d0) (ix 0) (kase 0)
             (normin
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (upper nil))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (double-float) ainvnm scale scalel scaleu smlnum)
                 (type (f2cl-lib:integer4) ix kase)
```

```
          (type (string 1) normin)
          (type f2cl-lib:logical upper))
 (setf info 0)
 (setf upper
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame uplo "U")
           (declare (ignore var-1))
           (when var-0 (setf uplo var-0))
           ret-val))
 (cond
  ((and (not upper)
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame uplo "L")
           (declare (ignore var-1))
           (when var-0 (setf uplo var-0))
           ret-val)))
   (setf info -1))
  ((< n 0) (setf info -2))
  ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
   (setf info -4))
  ((< anorm zero) (setf info -5)))
 (cond
  ((/= info 0) (xerbla "DPOCON" (f2cl-lib:int-sub info))
   (go end_label)))
 (setf rcond zero)
 (cond ((= n 0) (setf rcond one) (go end_label))
       ((= anorm zero) (go end_label)))
 (setf smlnum (dlamch "Safe minimum"))
 (setf kase 0)
 (f2cl-lib:f2cl-set-string normin "N" (string 1))
label10
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlacn2 n
      (f2cl-lib:array-slice work-%data% double-float ((+ n 1)) ((1 *))
                            work-%offset%)
      work iwork ainvnm kase isave)
   (declare (ignore var-1 var-2 var-3 var-6))
   (setf n var-0)
   (setf ainvnm var-4)
   (setf kase var-5))
 (cond
  ((/= kase 0)
   (cond
    (upper
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (dlatrs "Upper" "Transpose" "Non-unit" normin n a lda work
          scalel
```

```
        (f2cl-lib:array-slice work-%data% double-float
                              ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                              work-%offset%)
        info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
      (setf normin var-3)
      (setf n var-4)
      (setf lda var-6)
      (setf scalel var-8)
      (setf info var-10))
    (f2cl-lib:f2cl-set-string normin "Y" (string 1))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (dlatrs "Upper" "No transpose" "Non-unit" normin n a lda work
         scaleu
         (f2cl-lib:array-slice work-%data% double-float
                               ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                               work-%offset%)
         info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
      (setf normin var-3)
      (setf n var-4)
      (setf lda var-6)
      (setf scaleu var-8)
      (setf info var-10)))
  (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (dlatrs "Lower" "No transpose" "Non-unit" normin n a lda work
         scalel
         (f2cl-lib:array-slice work-%data% double-float
                               ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                               work-%offset%)
         info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
      (setf normin var-3)
      (setf n var-4)
      (setf lda var-6)
      (setf scalel var-8)
      (setf info var-10))
    (f2cl-lib:f2cl-set-string normin "Y" (string 1))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (dlatrs "Lower" "Transpose" "Non-unit" normin n a lda work
         scaleu
         (f2cl-lib:array-slice work-%data% double-float
                               ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
```

```
                                    work-%offset%)
                 info)
              (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
              (setf normin var-3)
              (setf n var-4)
              (setf lda var-6)
              (setf scaleu var-8)
              (setf info var-10))))
          (setf scale (* scalel scaleu))
          (cond
           ((/= scale one)
            (setf ix
                    (multiple-value-bind (ret-val var-0 var-1 var-2)
                        (idamax n work 1)
                      (declare (ignore var-1 var-2))
                      (when var-0 (setf n var-0))
                      ret-val))
            (if (or
                 (< scale
                    (*
                     (abs
                      (f2cl-lib:fref work-%data% (ix) ((1 *)) work-%offset%))
                     smlnum))
                 (= scale zero))
                (go label20))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (drscl n scale work 1)
              (declare (ignore var-2 var-3))
              (when var-0 (setf n var-0))
              (when var-1 (setf scale var-1)))))
          (go label10)))
         (if (/= ainvnm zero)
             (setf rcond (/ (/ one ainvnm) anorm)))
       label20
        (go end_label)
       end_label
        (return (values uplo n nil lda nil rcond nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dpocon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              (array double-float (*))
```

```
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::uplo
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::rcond nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::dlatrs
  fortran-to-lisp::dlacn2))))
```

———-

70.5 dpoequ

70.5.1 Axiom unit tests

— **dpoequ.input** —

```
)set break resume
)sys rm -f dpoequ.output
)spool dpoequ.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

70.5.2 Axiom help page

— **dpoequ.help** —

```
==================================================================
dpoequ examples
==================================================================

==================================================================
```

```
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dpoequ (N, A, LDA, S, SCOND, AMAX, INFO)
           DPOEQU

Function/Subroutine Documentation
   subroutine dpoequ (integer N, double precision, dimension( lda, * ) A,
       integer LDA, double precision, dimension( * ) S, double precision
       SCOND, double precision AMAX, integer INFO)
       DPOEQU

Purpose:
   DPOEQU computes row and column scalings intended to equilibrate a
   symmetric positive definite matrix A and reduce its condition number
   (with respect to the two-norm).  S contains the scale factors,
   S(i) = 1/sqrt(A(i,i)), chosen so that the scaled matrix B with
   elements B(i,j) = S(i)*A(i,j)*S(j) has ones on the diagonal.  This
   choice of S puts the condition number of B within a factor N of the
   smallest possible condition number over all possible diagonal scalings.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   The N-by-N symmetric positive definite matrix whose scaling
   factors are to be computed.  Only the diagonal elements of A
   are referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 S
   S is DOUBLE PRECISION array, dimension (N)
   If INFO = 0, S contains the scale factors for A.

 SCOND
   SCOND is DOUBLE PRECISION
   If INFO = 0, S contains the ratio of the smallest S(i) to
   the largest S(i).  If SCOND >= 0.1 and AMAX is neither too
   large nor too small, it is not worth scaling by S.

 AMAX
```

```
  AMAX is DOUBLE PRECISION
  Absolute value of largest matrix element.  If AMAX is very
  close to overflow or very close to underflow, the matrix
  should be scaled.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, the i-th diagonal element is nonpositive.
```

———-

70.5.3 fortran code

— dpoequ.f —

```
*  =====================================================================
      SUBROUTINE DPOEQU( N, A, LDA, S, SCOND, AMAX, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, N
      DOUBLE PRECISION   AMAX, SCOND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), S( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      DOUBLE PRECISION   SMIN
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -3
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DPOEQU', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SCOND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Find the minimum and maximum diagonal elements.
*
      S( 1 ) = A( 1, 1 )
      SMIN = S( 1 )
      AMAX = S( 1 )
      DO 10 I = 2, N
         S( I ) = A( I, I )
         SMIN = MIN( SMIN, S( I ) )
         AMAX = MAX( AMAX, S( I ) )
   10 CONTINUE
*
      IF( SMIN.LE.ZERO ) THEN
*
*        Find the first non-positive diagonal element and return.
*
         DO 20 I = 1, N
            IF( S( I ).LE.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   20    CONTINUE
      ELSE
*
*        Set the scale factors to the reciprocals
*        of the diagonal elements.
```

```
*
         DO 30 I = 1, N
            S( I ) = ONE / SQRT( S( I ) )
   30    CONTINUE
*
*        Compute SCOND = min(S(I)) / max(S(I))
*
         SCOND = SQRT( SMIN ) / SQRT( AMAX )
      END IF
      RETURN
*
*     End of DPOEQU
*
      END
```

———-

70.5.4 lisp code

— **dpoequ.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dpoequ (n a lda s scond amax info)
    (declare (type (double-float) amax scond)
             (type (array double-float (*)) s a)
             (type (f2cl-lib:integer4) info lda n))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (s double-float s-%data% s-%offset%))
      (prog ((smin 0.0d0) (i 0))
        (declare (type (double-float) smin)
                 (type (f2cl-lib:integer4) i))
        (setf info 0)
        (cond ((< n 0) (setf info -1))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -3)))
        (cond
         ((/= info 0) (xerbla "DPOEQU" (f2cl-lib:int-sub info))
          (go end_label)))
        (cond ((= n 0) (setf scond one) (setf amax zero) (go end_label)))
        (setf (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%)
                (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%))
        (setf smin (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
        (setf amax (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
```

```
        (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                    (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%))
            (setf smin
                    (min smin (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
            (setf amax
                    (max amax (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
           label10))
        (cond
         ((<= smin zero)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (cond
               ((<= (f2cl-lib:fref s (i) ((1 *))) zero) (setf info i)
                (go end_label)))
             label20)))
         (t
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                      (/ one
                         (f2cl-lib:fsqrt
                          (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%))))
             label30))
          (setf scond (/ (f2cl-lib:fsqrt smin) (f2cl-lib:fsqrt amax)))))
        (go end_label)
       end_label
        (return (values nil nil nil nil scond amax info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dpoequ
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (double-float) (double-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil
                                              fortran-to-lisp::scond
                                              fortran-to-lisp::amax
                                              fortran-to-lisp::info)
```

```
                                        :calls 'nil)))
```

———-

70.6 dpoequb

70.6.1 Axiom unit tests

— **dpoequb.input** —

```
)set break resume
)sys rm -f dpoequb.output
)spool dpoequb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

70.6.2 Axiom help page

— **dpoequb.help** —

```
======================================================================
dpoequb examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dpoequb (N, A, LDA, S, SCOND, AMAX, INFO)
           DPOEQUB

Function/Subroutine Documentation
   subroutine dpoequb (integer N, double precision, dimension( lda, * ) A,
       integer LDA, double precision, dimension( * ) S, double precision
```

```
      SCOND, double precision AMAX, integer INFO)
      DPOEQUB

Purpose:
   DPOEQU computes row and column scalings intended to equilibrate a
   symmetric positive definite matrix A and reduce its condition number
   (with respect to the two-norm).  S contains the scale factors,
   S(i) = 1/sqrt(A(i,i)), chosen so that the scaled matrix B with
   elements B(i,j) = S(i)*A(i,j)*S(j) has ones on the diagonal.  This
   choice of S puts the condition number of B within a factor N of the
   smallest possible condition number over all possible diagonal scalings.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   The N-by-N symmetric positive definite matrix whose scaling
   factors are to be computed.  Only the diagonal elements of A
   are referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 S
   S is DOUBLE PRECISION array, dimension (N)
   If INFO = 0, S contains the scale factors for A.

 SCOND
   SCOND is DOUBLE PRECISION
   If INFO = 0, S contains the ratio of the smallest S(i) to
   the largest S(i).  If SCOND >= 0.1 and AMAX is neither too
   large nor too small, it is not worth scaling by S.

 AMAX
   AMAX is DOUBLE PRECISION
   Absolute value of largest matrix element.  If AMAX is very
   close to overflow or very close to underflow, the matrix
   should be scaled.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, the i-th diagonal element is nonpositive.
```

———-

70.6.3 fortran code

— **dpoequb.f** —

```
*  =====================================================================
      SUBROUTINE DPOEQUB( N, A, LDA, S, SCOND, AMAX, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, N
      DOUBLE PRECISION   AMAX, SCOND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), S( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      DOUBLE PRECISION   SMIN, BASE, TMP
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT, LOG, INT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
*     Positive definite only performs 1 pass of equilibration.
*
```

```
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -3
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DPOEQUB', -INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 ) THEN
         SCOND = ONE
         AMAX = ZERO
         RETURN
      END IF

      BASE = DLAMCH( 'B' )
      TMP = -0.5D+0 / LOG ( BASE )
*
*     Find the minimum and maximum diagonal elements.
*
      S( 1 ) = A( 1, 1 )
      SMIN = S( 1 )
      AMAX = S( 1 )
      DO 10 I = 2, N
         S( I ) = A( I, I )
         SMIN = MIN( SMIN, S( I ) )
         AMAX = MAX( AMAX, S( I ) )
   10 CONTINUE
*
      IF( SMIN.LE.ZERO ) THEN
*
*        Find the first non-positive diagonal element and return.
*
         DO 20 I = 1, N
            IF( S( I ).LE.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   20    CONTINUE
      ELSE
*
*        Set the scale factors to the reciprocals
*        of the diagonal elements.
*
         DO 30 I = 1, N
            S( I ) = BASE ** INT( TMP * LOG( S( I ) ) )
```

```
   30    CONTINUE
*
*        Compute SCOND = min(S(I)) / max(S(I)).
*
         SCOND = SQRT( SMIN ) / SQRT( AMAX )
      END IF
*
      RETURN
*
*     End of DPOEQUB
*
      END
```

———-

70.6.4 lisp code

— **dpoequb.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dpoequb (n a lda s scond amax info)
    (declare (type (double-float) amax scond)
             (type (array double-float (*)) s a)
             (type (f2cl-lib:integer4) info lda n))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (s double-float s-%data% s-%offset%))
      (prog ((smin 0.0d0) (base 0.0d0) (tmp 0.0d0) (i 0))
        (declare (type (double-float) smin base tmp)
                 (type (f2cl-lib:integer4) i))
        (setf info 0)
        (cond ((< n 0) (setf info -1))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -3)))
        (cond
         ((/= info 0) (xerbla "DPOEQUB" (f2cl-lib:int-sub info))
          (go end_label)))
        (cond ((= n 0) (setf scond one) (setf amax zero) (go end_label)))
        (setf base (dlamch "B"))
        (setf tmp (/ -0.5d0 (f2cl-lib:flog base)))
        (setf (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%)
                (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%))
        (setf smin (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
        (setf amax (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
```

```
        (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                    (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%))
            (setf smin
                    (min smin (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
            (setf amax
                    (max amax (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
           label10))
        (cond
         ((<= smin zero)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (cond
               ((<= (f2cl-lib:fref s (i) ((1 *))) zero) (setf info i)
                (go end_label)))
             label20)))
         (t
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                      (expt base
                            (f2cl-lib:int
                             (* tmp
                                (f2cl-lib:flog
                                 (f2cl-lib:fref s-%data% (i) ((1 *))
                                                s-%offset%))))))
             label30))
          (setf scond (/ (f2cl-lib:fsqrt smin) (f2cl-lib:fsqrt amax)))))
        (go end_label)
       end_label
        (return (values nil nil nil nil scond amax info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dpoequb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (double-float) (double-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil
```

```
                                          fortran-to-lisp::scond
                                          fortran-to-lisp::amax
                                          fortran-to-lisp::info)
                                        :calls 'nil)))
```

———-

70.7 dporfs

70.7.1 Axiom unit tests

— **dporfs.input** —

```
)set break resume
)sys rm -f dporfs.output
)spool dporfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

70.7.2 Axiom help page

— **dporfs.help** —

```
====================================================================
dporfs examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dporfs (UPLO, N, NRHS, A, LDA, AF, LDAF, B, LDB, X, LDX,
           FERR, BERR, WORK, IWORK, INFO)
           DPORFS
```

```
Function/Subroutine Documentation
   subroutine dporfs (character UPLO, integer N, integer NRHS, double
       precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldaf, * ) AF, integer LDAF, double precision, dimension(
       ldb, * ) B, integer LDB, double precision, dimension( ldx, * ) X,
       integer LDX, double precision, dimension( * ) FERR, double precision,
       dimension( * ) BERR, double precision, dimension( * ) WORK, integer,
       dimension( * ) IWORK, integer INFO)
       DPORFS

Purpose:
   DPORFS improves the computed solution to a system of linear
   equations when the coefficient matrix is symmetric positive definite,
   and provides error bounds and backward error estimates for the
   solution.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
   upper triangular part of A contains the upper triangular part
   of the matrix A, and the strictly lower triangular part of A
   is not referenced.  If UPLO = 'L', the leading N-by-N lower
   triangular part of A contains the lower triangular part of
   the matrix A, and the strictly upper triangular part of A is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is DOUBLE PRECISION array, dimension (LDAF,N)
   The triangular factor U or L from the Cholesky factorization
```

```
  A = U**T*U or A = L*L**T, as computed by DPOTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by DPOTRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

FERR
  FERR is DOUBLE PRECISION array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is DOUBLE PRECISION array, dimension (3*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

```
Internal Parameters:

   ITMAX is the maximum number of steps of iterative refinement.
```

———-

70.7.3 fortran code

— **dporfs.f** —

```
*  =====================================================================
      SUBROUTINE DPORFS( UPLO, N, NRHS, A, LDA, AF, LDAF, B, LDB, X,
     $                   LDX, FERR, BERR, WORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   BERR( * ), FERR( * ), WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D+0 )
      DOUBLE PRECISION   THREE
      PARAMETER          ( THREE = 3.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            COUNT, I, J, K, KASE, NZ
```

```
      DOUBLE PRECISION   EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN, XK
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DCOPY, DLACN2, DPOTRS, DSYMV, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -11
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DPORFS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
```

```
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = N + 1
      EPS = DLAMCH( 'Epsilon' )
      SAFMIN = DLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 140 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - A * X
*
         CALL DCOPY( N, B( 1, J ), 1, WORK( N+1 ), 1 )
         CALL DSYMV( UPLO, N, -ONE, A, LDA, X( 1, J ), 1, ONE,
     $               WORK( N+1 ), 1 )
*
*        Compute componentwise relative backward error from formula
*
*        max(i) ( abs(R(i)) / ( abs(A)*abs(X) + abs(B) )(i) )
*
*        where abs(Z) is the componentwise absolute value of the matrix
*        or vector Z.  If the i-th component of the denominator is less
*        than SAFE2, then SAFE1 is added to the i-th components of the
*        numerator and denominator before dividing.
*
         DO 30 I = 1, N
            WORK( I ) = ABS( B( I, J ) )
   30    CONTINUE
*
*        Compute abs(A)*abs(X) + abs(B).
*
         IF( UPPER ) THEN
            DO 50 K = 1, N
               S = ZERO
               XK = ABS( X( K, J ) )
               DO 40 I = 1, K - 1
                  WORK( I ) = WORK( I ) + ABS( A( I, K ) )*XK
                  S = S + ABS( A( I, K ) )*ABS( X( I, J ) )
   40          CONTINUE
               WORK( K ) = WORK( K ) + ABS( A( K, K ) )*XK + S
```

```
   50       CONTINUE
         ELSE
            DO 70 K = 1, N
               S = ZERO
               XK = ABS( X( K, J ) )
               WORK( K ) = WORK( K ) + ABS( A( K, K ) )*XK
               DO 60 I = K + 1, N
                  WORK( I ) = WORK( I ) + ABS( A( I, K ) )*XK
                  S = S + ABS( A( I, K ) )*ABS( X( I, J ) )
   60          CONTINUE
               WORK( K ) = WORK( K ) + S
   70       CONTINUE
         END IF
         S = ZERO
         DO 80 I = 1, N
            IF( WORK( I ).GT.SAFE2 ) THEN
               S = MAX( S, ABS( WORK( N+I ) ) / WORK( I ) )
            ELSE
               S = MAX( S, ( ABS( WORK( N+I ) )+SAFE1 ) /
     $             ( WORK( I )+SAFE1 ) )
            END IF
   80    CONTINUE
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
*           1) The residual BERR(J) is larger than machine epsilon, and
*           2) BERR(J) decreased by at least a factor of 2 during the
*              last iteration, and
*           3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
*
*           Update solution and try again.
*
            CALL DPOTRS( UPLO, N, 1, AF, LDAF, WORK( N+1 ), N, INFO )
            CALL DAXPY( N, ONE, WORK( N+1 ), 1, X( 1, J ), 1 )
            LSTRES = BERR( J )
            COUNT = COUNT + 1
            GO TO 20
         END IF
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(A))*
*           ( abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
```

```
*          inv(A) is the inverse of A
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(A)*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(A)*abs(X) + abs(B) is less than SAFE2.
*
*        Use DLACN2 to estimate the infinity-norm of the matrix
*           inv(A) * diag(W),
*        where W = abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) )))
*
         DO 90 I = 1, N
            IF( WORK( I ).GT.SAFE2 ) THEN
               WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I )
            ELSE
               WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I ) + SAFE1
            END IF
   90    CONTINUE
*
         KASE = 0
  100    CONTINUE
         CALL DLACN2( N, WORK( 2*N+1 ), WORK( N+1 ), IWORK, FERR( J ),
     $                KASE, ISAVE )
         IF( KASE.NE.0 ) THEN
            IF( KASE.EQ.1 ) THEN
*
               Multiply by diag(W)*inv(A**T).
*
               CALL DPOTRS( UPLO, N, 1, AF, LDAF, WORK( N+1 ), N, INFO )
               DO 110 I = 1, N
                  WORK( N+I ) = WORK( I )*WORK( N+I )
  110          CONTINUE
            ELSE IF( KASE.EQ.2 ) THEN
*
               Multiply by inv(A)*diag(W).
*
               DO 120 I = 1, N
                  WORK( N+I ) = WORK( I )*WORK( N+I )
  120          CONTINUE
               CALL DPOTRS( UPLO, N, 1, AF, LDAF, WORK( N+1 ), N, INFO )
            END IF
            GO TO 100
         END IF
*
*        Normalize error.
*
         LSTRES = ZERO
```

```
         DO 130 I = 1, N
            LSTRES = MAX( LSTRES, ABS( X( I, J ) ) )
  130    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  140 CONTINUE
*
      RETURN
*
*     End of DPORFS
*
      END
```

———-

70.7.4 lisp code

— **dporfs.lisp** —

```
(let* ((itmax 5) (zero 0.0d0) (one 1.0d0) (two 2.0d0) (three 3.0d0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 3.0d0 3.0d0) three)
           (ignorable itmax zero one two three))
  (defun dporfs
         (uplo n nrhs a lda af ldaf b ldb$ x ldx ferr berr work iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (array double-float (*)) work berr ferr x b af a)
             (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (af double-float af-%data% af-%offset%)
         (b double-float b-%data% b-%offset%)
         (x double-float x-%data% x-%offset%)
         (ferr double-float ferr-%data% ferr-%offset%)
         (berr double-float berr-%data% berr-%offset%)
         (work double-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (eps 0.0d0) (lstres 0.0d0) (s 0.0d0) (safe1 0.0d0) (safe2 0.0d0)
             (safmin 0.0d0) (xk 0.0d0) (count$ 0) (i 0) (j 0) (k 0) (kase 0)
             (nz 0) (upper nil))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
```

```
         (type (double-float) eps lstres s safe1 safe2 safmin xk)
         (type (f2cl-lib:integer4) count$ i j k kase nz)
         (type f2cl-lib:logical upper))
(setf info 0)
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -11)))
(cond
 ((/= info 0) (xerbla "DPORFS" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%) zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) zero)
     label10))
  (go end_label)))
(setf nz (f2cl-lib:int-add n 1))
(setf eps (dlamch "Epsilon"))
(setf safmin (dlamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
```

```
label20
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (dcopy n
      (f2cl-lib:array-slice b-%data% double-float (1 j)
                            ((1 ldb$) (1 *)) b-%offset%)
      1
      (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                            ((1 *)) work-%offset%)
      1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf n var-0)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
     (dsymv uplo n (- one) a lda
      (f2cl-lib:array-slice x-%data% double-float (1 j)
                            ((1 ldx) (1 *)) x-%offset%)
      1 one
      (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                            ((1 *)) work-%offset%)
      1)
   (declare (ignore var-2 var-3 var-5 var-6 var-8 var-9))
   (when var-0 (setf uplo var-0))
   (when var-1 (setf n var-1))
   (when var-4 (setf lda var-4))
   (when var-7 (setf one var-7)))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
             (abs
              (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)))
    label30))
 (cond
  (upper
   (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                 ((> k n) nil)
     (tagbody
       (setf s zero)
       (setf xk
               (abs
                (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                               x-%offset%)))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i
                         (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                      nil)
         (tagbody
           (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
```

```
                        (+
                         (f2cl-lib:fref work-%data% (i) ((1 *))
                                        work-%offset%)
                         (*
                          (abs
                           (f2cl-lib:fref a-%data% (i k) ((1 lda) (1 *))
                                          a-%offset%))
                          xk)))
                (setf s
                        (+ s
                           (*
                            (abs
                             (f2cl-lib:fref a-%data% (i k)
                                            ((1 lda) (1 *)) a-%offset%))
                            (abs
                             (f2cl-lib:fref x-%data% (i j)
                                            ((1 ldx) (1 *))
                                            x-%offset%)))))
               label40))
            (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                    (+
                     (f2cl-lib:fref work-%data% (k) ((1 *))
                                    work-%offset%)
                     (*
                      (abs
                       (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                      a-%offset%))
                      xk)
                     s))
          label50)))
      (t
       (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                     ((> k n) nil)
         (tagbody
           (setf s zero)
           (setf xk
                   (abs
                    (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                                   x-%offset%)))
           (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                   (+
                    (f2cl-lib:fref work-%data% (k) ((1 *))
                                   work-%offset%)
                    (*
                     (abs
                      (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                     a-%offset%))
                     xk)))
           (f2cl-lib:fdo (i (f2cl-lib:int-add k 1)
                            (f2cl-lib:int-add i 1))
```

```
                    ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)
                    (+
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)
                     (*
                      (abs
                       (f2cl-lib:fref a-%data% (i k) ((1 lda) (1 *))
                                      a-%offset%))
                      xk)))
              (setf s
                    (+ s
                       (*
                        (abs
                         (f2cl-lib:fref a-%data% (i k)
                                        ((1 lda) (1 *)) a-%offset%))
                        (abs
                         (f2cl-lib:fref x-%data% (i j)
                                        ((1 ldx) (1 *))
                                        x-%offset%)))))
            label60))
        (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
              (+
               (f2cl-lib:fref work-%data% (k) ((1 *))
                              work-%offset%)
               s))
      label70))))
  (setf s zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
        (setf s
              (max s
                   (/
                    (abs
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add n i))
                                    ((1 *)) work-%offset%))
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)))))
       (t
        (setf s
              (max s
                   (/
                    (+
                     (abs
```

```
                        (f2cl-lib:fref work-%data%
                                       ((f2cl-lib:int-add n i))
                                       ((1 *)) work-%offset%))
                       safe1)
                      (+
                       (f2cl-lib:fref work-%data% (i) ((1 *))
                                      work-%offset%)
                       safe1))))))
   label80))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dpotrs uplo n 1 af ldaf
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 *)) work-%offset%)
       n info)
    (declare (ignore var-2 var-3 var-5))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf ldaf var-4))
    (when var-6 (setf n var-6))
    (when var-7 (setf info var-7)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (daxpy n one
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 *)) work-%offset%)
       1
       (f2cl-lib:array-slice x-%data% double-float (1 j)
                             ((1 ldx) (1 *)) x-%offset%)
       1)
    (declare (ignore var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))
    (when var-1 (setf one var-1)))
  (setf lstres
          (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
  (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (+
               (abs
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%))
```

```
                    (* nz eps
                       (f2cl-lib:fref work-%data% (i) ((1 *))
                                      work-%offset%)))))
         (t
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (+
                   (abs
                    (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                   ((1 *)) work-%offset%))
                   (* nz eps
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%))
                   safe1))))
      label90))
   (setf kase 0)
  label100
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlacn2 n
        (f2cl-lib:array-slice work-%data% double-float
                              ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                              ((1 *)) work-%offset%)
        iwork (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
        kase isave)
     (declare (ignore var-1 var-2 var-3 var-6))
     (setf n var-0)
     (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
             var-4)
     (setf kase var-5))
   (cond
    ((/= kase 0)
     (cond
      ((= kase 1)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
           (dpotrs uplo n 1 af ldaf
            (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                                  ((1 *)) work-%offset%)
            n info)
         (declare (ignore var-2 var-3 var-5))
         (when var-0 (setf uplo var-0))
         (when var-1 (setf n var-1))
         (when var-4 (setf ldaf var-4))
         (when var-6 (setf n var-6))
         (when var-7 (setf info var-7)))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
```

```
                                  ((1 *)) work-%offset%)
                         (*
                          (f2cl-lib:fref work-%data% (i) ((1 *))
                                         work-%offset%)
                          (f2cl-lib:fref work-%data%
                                         ((f2cl-lib:int-add n i)) ((1 *))
                                         work-%offset%)))
              label110)))
          ((= kase 2)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                  ((1 *)) work-%offset%)
                         (*
                          (f2cl-lib:fref work-%data% (i) ((1 *))
                                         work-%offset%)
                          (f2cl-lib:fref work-%data%
                                         ((f2cl-lib:int-add n i)) ((1 *))
                                         work-%offset%)))
              label120))
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
               (dpotrs uplo n 1 af ldaf
                (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                                      ((1 *)) work-%offset%)
                n info)
             (declare (ignore var-2 var-3 var-5))
             (when var-0 (setf uplo var-0))
             (when var-1 (setf n var-1))
             (when var-4 (setf ldaf var-4))
             (when var-6 (setf n var-6))
             (when var-7 (setf info var-7)))))
        (go label100)))
      (setf lstres zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf lstres
                  (max lstres
                       (abs
                        (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                       x-%offset%))))
         label130))
      (if (/= lstres zero)
          (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                  (/
                   (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                   lstres)))
     label140))
```

```
        (go end_label)
       end_label
        (return
         (values uplo n nil nil lda nil ldaf nil nil nil nil nil nil nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dporfs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldaf nil nil nil
                                              nil nil nil nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::dlacn2))))
```

———-

70.8 dporfsx

70.8.1 Axiom unit tests

— dporfsx.input —

```
)set break resume
)sys rm -f dporfsx.output
)spool dporfsx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

70.8.2 Axiom help page citeDemm05

— dporfsx.help —

```
======================================================================
dporfsx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dporfsx (UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF, S, B, LDB,
           X, LDX, RCOND, BERR, N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP,
           NPARAMS, PARAMS, WORK, IWORK, INFO)
           DPORFSX

Function/Subroutine Documentation
   subroutine dporfsx (character UPLO, character EQUED, integer N, integer
       NRHS, double precision, dimension( lda, * ) A, integer LDA, double
       precision, dimension( ldaf, * ) AF, integer LDAF, double precision,
       dimension( * ) S, double precision, dimension( ldb, * ) B, integer LDB,
       double precision, dimension( ldx, * ) X, integer LDX, double precision
       RCOND, double precision, dimension( * ) BERR, integer N_ERR_BNDS,
       double precision, dimension( nrhs, * ) ERR_BNDS_NORM, double precision,
       dimension( nrhs, * ) ERR_BNDS_COMP, integer NPARAMS, double precision,
       dimension( * ) PARAMS, double precision, dimension( * ) WORK, integer,
       dimension( * ) IWORK, integer INFO)
       DPORFSX

Purpose:
   DPORFSX improves the computed solution to a system of linear
   equations when the coefficient matrix is symmetric positive
   definite, and provides error bounds and backward error estimates
```

```
   for the solution.  In addition to normwise error bound, the code
   provides maximum componentwise error bound if possible.  See
   comments for ERR_BNDS_NORM and ERR_BNDS_COMP for details of the
   error bounds.

   The original system of linear equations may have been equilibrated
   before calling this routine, as described by arguments EQUED and S
   below. In this case, the solution and error bounds returned are
   for the original unequilibrated system.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done to A
   before calling this routine. This is needed to compute
   the solution and error bounds correctly.
   = 'N':  No equilibration
   = 'Y':  Both row and column equilibration, i.e., A has been
           replaced by diag(S) * A * diag(S).
           The right hand side B has been changed accordingly.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
   upper triangular part of A contains the upper triangular part
   of the matrix A, and the strictly lower triangular part of A
   is not referenced.  If UPLO = 'L', the leading N-by-N lower
   triangular part of A contains the lower triangular part of
   the matrix A, and the strictly upper triangular part of A is
```

```
  not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is DOUBLE PRECISION array, dimension (LDAF,N)
  The triangular factor U or L from the Cholesky factorization
  A = U**T*U or A = L*L**T, as computed by DPOTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

S
  S is DOUBLE PRECISION array, dimension (N)
  The row scale factors for A.  If EQUED = 'Y', A is multiplied on
  the left and right by diag(S).  S is an input argument if FACT =
  'F'; otherwise, S is an output argument.  If FACT = 'F' and EQUED
  = 'Y', each element of S must be positive.  If S is output, each
  element of S is a power of the radix. If S is input, each element
  of S should be a power of the radix to ensure a reliable solution
  and error estimates. Scaling by powers of the radix does not cause
  rounding errors unless the result underflows or overflows.
  Rounding errors during scaling lead to refining with a matrix that
  is not equivalent to the input matrix, producing error estimates
  that may not be reliable.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by DGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
```

```
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
            max_j (abs(XTRUE(j,i) - X(j,i)))
             ------------------------------
                   max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * dlamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
```

```
      reciprocal condition number.  Compared with the threshold
      sqrt(n) * dlamch('Epsilon') to determine if the error
      estimate is "guaranteed". These reciprocal condition
      numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
      appropriately scaled matrix Z.
      Let Z = S*A, where S scales each row by a power of the
      radix so all absolute row sums of Z are approximately 1.

ERR_BNDS_COMP
  ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                  abs(XTRUE(j,i) - X(j,i))
            max_j ----------------------
                        abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * dlamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * dlamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
```

```
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is DOUBLE PRECISION array, dimension (NPARAMS)
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0D+0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
  (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is DOUBLE PRECISION array, dimension (4*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is
        guaranteed.
```

```
   < 0:  If INFO = -i, the i-th argument had an illegal value
   > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
      has been completed, but the factor U is exactly singular, so
      the solution and error bounds could not be computed. RCOND = 0
      is returned.
   = N+J: The solution corresponding to the Jth right-hand side is
      not guaranteed. The solutions corresponding to other right-
      hand sides K with K > J may not be guaranteed as well, but
      only the first such right-hand side is reported. If a small
      componentwise error is not requested (PARAMS(3) = 0.0) then
      the Jth right-hand side is the first with a normwise error
      bound that is not guaranteed (the smallest J such
      that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
      the Jth right-hand side is the first with either a normwise or
      componentwise error bound that is not guaranteed (the smallest
      J such that either ERR_BNDS_NORM(J,1) = 0.0 or
      ERR_BNDS_COMP(J,1) = 0.0). See the definition of
      ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
      about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

70.8.3 fortran code

— dporfsx.f —

```
*  =====================================================================
      SUBROUTINE DPORFSX( UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF, S, B,
     $                    LDB, X, LDX, RCOND, BERR, N_ERR_BNDS,
     $                    ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $                    WORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO, EQUED
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX, * ), WORK( * )
```

```
      DOUBLE PRECISION   S( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   ITREF_DEFAULT, ITHRESH_DEFAULT
      DOUBLE PRECISION   COMPONENTWISE_DEFAULT, RTHRESH_DEFAULT
      DOUBLE PRECISION   DZTHRESH_DEFAULT
      PARAMETER          ( ITREF_DEFAULT = 1.0D+0 )
      PARAMETER          ( ITHRESH_DEFAULT = 10.0D+0 )
      PARAMETER          ( COMPONENTWISE_DEFAULT = 1.0D+0 )
      PARAMETER          ( RTHRESH_DEFAULT = 0.5D+0 )
      PARAMETER          ( DZTHRESH_DEFAULT = 0.25D+0 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. Local Scalars ..
      CHARACTER(1)       NORM
      LOGICAL            RCEQU
      INTEGER            J, PREC_TYPE, REF_TYPE
      INTEGER            N_NORMS
      DOUBLE PRECISION   ANORM, RCOND_TMP
      DOUBLE PRECISION   ILLRCOND_THRESH, ERR_LBND, CWISE_WRONG
      LOGICAL            IGNORE_CWISE
      INTEGER            ITHRESH
      DOUBLE PRECISION   RTHRESH, UNSTABLE_THRESH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, DPOCON, DLA_PORFSX_EXTENDED
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, BLAS_FPINFO_X, ILATRANS, ILAPREC
      EXTERNAL           DLAMCH, DLANSY, DLA_PORCOND
      DOUBLE PRECISION   DLAMCH, DLANSY, DLA_PORCOND
      LOGICAL            LSAME
```

```
      INTEGER            BLAS_FPINFO_X
      INTEGER            ILATRANS, ILAPREC
*     ..
*     .. Executable Statements ..
*
*     Check the input parameters.
*
      INFO = 0
      REF_TYPE = INT( ITREF_DEFAULT )
      IF ( NPARAMS .GE. LA_LINRX_ITREF_I ) THEN
         IF ( PARAMS( LA_LINRX_ITREF_I ) .LT. 0.0D+0 ) THEN
            PARAMS( LA_LINRX_ITREF_I ) = ITREF_DEFAULT
         ELSE
            REF_TYPE = PARAMS( LA_LINRX_ITREF_I )
         END IF
      END IF
*
*     Set default parameters.
*
      ILLRCOND_THRESH = DBLE( N ) * DLAMCH( 'Epsilon' )
      ITHRESH = INT( ITHRESH_DEFAULT )
      RTHRESH = RTHRESH_DEFAULT
      UNSTABLE_THRESH = DZTHRESH_DEFAULT
      IGNORE_CWISE = COMPONENTWISE_DEFAULT .EQ. 0.0D+0
*
      IF ( NPARAMS.GE.LA_LINRX_ITHRESH_I ) THEN
         IF ( PARAMS( LA_LINRX_ITHRESH_I ).LT.0.0D+0 ) THEN
            PARAMS( LA_LINRX_ITHRESH_I ) = ITHRESH
         ELSE
            ITHRESH = INT( PARAMS( LA_LINRX_ITHRESH_I ) )
         END IF
      END IF
      IF ( NPARAMS.GE.LA_LINRX_CWISE_I ) THEN
         IF ( PARAMS( LA_LINRX_CWISE_I ).LT.0.0D+0 ) THEN
            IF ( IGNORE_CWISE ) THEN
               PARAMS( LA_LINRX_CWISE_I ) = 0.0D+0
            ELSE
               PARAMS( LA_LINRX_CWISE_I ) = 1.0D+0
            END IF
         ELSE
            IGNORE_CWISE = PARAMS( LA_LINRX_CWISE_I ) .EQ. 0.0D+0
         END IF
      END IF
      IF ( REF_TYPE .EQ. 0 .OR. N_ERR_BNDS .EQ. 0 ) THEN
         N_NORMS = 0
      ELSE IF ( IGNORE_CWISE ) THEN
         N_NORMS = 1
      ELSE
         N_NORMS = 2
      END IF
```

```
*
      RCEQU = LSAME( EQUED, 'Y' )
*
*     Test input parameters.
*
      IF (.NOT.LSAME(UPLO, 'U') .AND. .NOT.LSAME(UPLO, 'L')) THEN
        INFO = -1
      ELSE IF( .NOT.RCEQU .AND. .NOT.LSAME( EQUED, 'N' ) ) THEN
        INFO = -2
      ELSE IF( N.LT.0 ) THEN
        INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
        INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
        INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
        INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
        INFO = -11
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
        INFO = -13
      END IF
      IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'DPORFSX', -INFO )
        RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         RCOND = 1.0D+0
         DO J = 1, NRHS
            BERR( J ) = 0.0D+0
            IF ( N_ERR_BNDS .GE. 1 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
            IF ( N_ERR_BNDS .GE. 2 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 0.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 0.0D+0
            END IF
            IF ( N_ERR_BNDS .GE. 3 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 1.0D+0
            END IF
         END DO
         RETURN
      END IF
*
*     Default to failure.
```

```
*
      RCOND = 0.0D+0
      DO J = 1, NRHS
         BERR( J ) = 1.0D+0
         IF ( N_ERR_BNDS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
         END IF
         IF ( N_ERR_BNDS .GE. 2 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
         END IF
         IF ( N_ERR_BNDS .GE. 3 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 0.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 0.0D+0
         END IF
      END DO
*
*     Compute the norm of A and the reciprocal of the condition
*     number of A.
*
      NORM = 'I'
      ANORM = DLANSY( NORM, UPLO, N, A, LDA, WORK )
      CALL DPOCON( UPLO, N, AF, LDAF, ANORM, RCOND, WORK,
     $     IWORK, INFO )
*
*     Perform refinement on each right-hand side
*
      IF ( REF_TYPE .NE. 0 ) THEN

         PREC_TYPE = ILAPREC( 'E' )

         CALL DLA_PORFSX_EXTENDED( PREC_TYPE, UPLO,  N,
     $        NRHS, A, LDA, AF, LDAF, RCEQU, S, B,
     $        LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM, ERR_BNDS_COMP,
     $        WORK( N+1 ), WORK( 1 ), WORK( 2*N+1 ), WORK( 1 ), RCOND,
     $        ITHRESH, RTHRESH, UNSTABLE_THRESH, IGNORE_CWISE,
     $        INFO )
      END IF

      ERR_LBND = MAX( 10.0D+0, SQRT( DBLE( N ) ) ) * DLAMCH( 'Epsilon' )
      IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 1 ) THEN
*
*     Compute scaled normwise condition number cond(A*C).
*
         IF ( RCEQU ) THEN
            RCOND_TMP = DLA_PORCOND( UPLO, N, A, LDA, AF, LDAF,
     $           -1, S, INFO, WORK, IWORK )
         ELSE
            RCOND_TMP = DLA_PORCOND( UPLO, N, A, LDA, AF, LDAF,
```

```
     $           0, S, INFO, WORK, IWORK )
         END IF
         DO J = 1, NRHS
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .GT. 1.0D+0 )
     $           ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 0.0D+0
               IF ( INFO .LE. N ) INFO = N + J
            ELSE IF ( ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .LT. ERR_LBND )
     $              THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
*
*     Save the condition number.
*
            IF (N_ERR_BNDS .GE. LA_LINRX_RCOND_I) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF
         END DO
      END IF

      IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 2 ) THEN
*
*     Compute componentwise condition number cond(A*diag(Y(:,J))) for
*     each right-hand side using the current solution as an estimate of
*     the true solution.  If the componentwise error estimate is too
*     large, then the solution is a lousy estimate of truth and the
*     estimated RCOND may be too optimistic.  To avoid misleading users,
*     the inverse condition number is set to 0.0 when the estimated
*     cwise error is at least CWISE_WRONG.
*
         CWISE_WRONG = SQRT( DLAMCH( 'Epsilon' ) )
         DO J = 1, NRHS
            IF (ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .LT. CWISE_WRONG )
     $     THEN
               RCOND_TMP = DLA_PORCOND( UPLO, N, A, LDA, AF, LDAF, 1,
     $              X( 1, J ), INFO, WORK, IWORK )
            ELSE
               RCOND_TMP = 0.0D+0
            END IF
*
```

```
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .GT. 1.0D+0 )
     $           ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 0.0D+0
               IF ( PARAMS( LA_LINRX_CWISE_I ) .EQ. 1.0D+0
     $              .AND. INFO.LT.N + J ) INFO = N + J
            ELSE IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I )
     $              .LT. ERR_LBND ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF

         END DO
      END IF
*
      RETURN
*
*     End of DPORFSX
*
      END
```

———-

70.8.4 lisp code

— dporfsx.lisp —

———-

70.9 dpotf2

70.9.1 Axiom unit tests

— dpotf2.input —

```
)set break resume
)sys rm -f dpotf2.output
)spool dpotf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

70.9.2 Axiom help page

— dpotf2.help —

```
======================================================================
dpotf2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dpotf2 (UPLO, N, A, LDA, INFO)
           DPOTF2 computes the Cholesky factorization of a symmetric/Hermitian
           positive definite matrix (unblocked algorithm).

Function/Subroutine Documentation
   subroutine dpotf2 (character UPLO, integer N, double precision, dimension(
       lda, * ) A, integer LDA, integer INFO)
       DPOTF2 computes the Cholesky factorization of a symmetric/Hermitian
       positive definite matrix (unblocked algorithm).

Purpose:
   DPOTF2 computes the Cholesky factorization of a real symmetric
   positive definite matrix A.
```

```
   The factorization has the form
      A = U**T * U ,  if UPLO = 'U', or
      A = L  * L**T,  if UPLO = 'L',
   where U is an upper triangular matrix and L is lower triangular.

   This is the unblocked version of the algorithm, calling Level 2 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is stored.
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   n by n upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading n by n lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

   On exit, if INFO = 0, the factor U or L from the Cholesky
   factorization A = U**T *U  or A = L*L**T.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -k, the k-th argument had an illegal value
   > 0: if INFO = k, the leading minor of order k is not
        positive definite, and the factorization could not be completed.
```

———-

70.9.3 fortran code

— dpotf2.f —

```
*  =====================================================================
      SUBROUTINE DPOTF2( UPLO, N, A, LDA, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            J
      DOUBLE PRECISION   AJJ
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, DISNAN
      DOUBLE PRECISION   DDOT
      EXTERNAL           LSAME, DDOT, DISNAN
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEMV, DSCAL, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
```

```
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DPOTF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*        Compute the Cholesky factorization A = U**T *U.
*
         DO 10 J = 1, N
*
*           Compute U(J,J) and test for non-positive-definiteness.
*
            AJJ = A( J, J ) - DDOT( J-1, A( 1, J ), 1, A( 1, J ), 1 )
            IF( AJJ.LE.ZERO.OR.DISNAN( AJJ ) ) THEN
               A( J, J ) = AJJ
               GO TO 30
            END IF
            AJJ = SQRT( AJJ )
            A( J, J ) = AJJ
*
*           Compute elements J+1:N of row J.
*
            IF( J.LT.N ) THEN
               CALL DGEMV( 'Transpose', J-1, N-J, -ONE, A( 1, J+1 ),
     $                     LDA, A( 1, J ), 1, ONE, A( J, J+1 ), LDA )
               CALL DSCAL( N-J, ONE / AJJ, A( J, J+1 ), LDA )
            END IF
   10    CONTINUE
      ELSE
*
*        Compute the Cholesky factorization A = L*L**T.
*
         DO 20 J = 1, N
*
*           Compute L(J,J) and test for non-positive-definiteness.
*
            AJJ = A( J, J ) - DDOT( J-1, A( J, 1 ), LDA, A( J, 1 ),
     $            LDA )
```

```
            IF( AJJ.LE.ZERO.OR.DISNAN( AJJ ) ) THEN
               A( J, J ) = AJJ
               GO TO 30
            END IF
            AJJ = SQRT( AJJ )
            A( J, J ) = AJJ
*
*           Compute elements J+1:N of column J.
*
            IF( J.LT.N ) THEN
               CALL DGEMV( 'No transpose', N-J, J-1, -ONE, A( J+1, 1 ),
     $                     LDA, A( J, 1 ), LDA, ONE, A( J+1, J ), 1 )
               CALL DSCAL( N-J, ONE / AJJ, A( J+1, J ), 1 )
            END IF
   20    CONTINUE
      END IF
      GO TO 40
*
   30 CONTINUE
      INFO = J
*
   40 CONTINUE
      RETURN
*
*     End of DPOTF2
*
      END
```

———-

70.9.4 lisp code

— dpotf2.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dpotf2 (uplo n a lda info)
    (declare (type (array double-float (*)) a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%))
      (prog ((ajj 0.0d0) (j 0) (upper nil))
        (declare (type (double-float) ajj)
                 (type (f2cl-lib:integer4) j)
```

```
         (type f2cl-lib:logical upper))
(setf info 0)
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4)))
(cond
 ((/= info 0) (xerbla "DPOTF2" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(cond
 (upper
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf ajj
              (-
               (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                              a-%offset%)
               (ddot (f2cl-lib:int-sub j 1)
                (f2cl-lib:array-slice a-%data% double-float (1 j)
                                      ((1 lda) (1 *)) a-%offset%)
                1
                (f2cl-lib:array-slice a-%data% double-float (1 j)
                                      ((1 lda) (1 *)) a-%offset%)
                1)))
      (cond
       ((or (<= ajj zero)
            (multiple-value-bind (ret-val var-0)
                (disnan ajj)
              (declare (ignore))
              (setf ajj var-0)
              ret-val))
        (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
              ajj)
```

```
        (go label30)))
      (setf ajj (f2cl-lib:fsqrt ajj))
      (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
              ajj)
      (cond
       ((< j n)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10)
            (dgemv "Transpose" (f2cl-lib:int-sub j 1)
             (f2cl-lib:int-sub n j) (- one)
             (f2cl-lib:array-slice a-%data% double-float
                                   (1 (f2cl-lib:int-add j 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice a-%data% double-float (1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             1 one
             (f2cl-lib:array-slice a-%data% double-float
                                   (j (f2cl-lib:int-add j 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare
           (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7 var-9))
          (when var-5 (setf lda var-5))
          (when var-8 (setf one var-8))
          (when var-10 (setf lda var-10)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal (f2cl-lib:int-sub n j) (/ one ajj)
             (f2cl-lib:array-slice a-%data% double-float
                                   (j (f2cl-lib:int-add j 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-1 var-2))
          (when var-3 (setf lda var-3)))))
    label10)))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf ajj
             (-
              (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                             a-%offset%)
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4)
                  (ddot (f2cl-lib:int-sub j 1)
                   (f2cl-lib:array-slice a-%data% double-float (j 1)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda
```

```
                    (f2cl-lib:array-slice a-%data% double-float (j 1)
                                          ((1 lda) (1 *)) a-%offset%)
                    lda)
                 (declare (ignore var-0 var-1 var-3))
                 (when var-2 (setf lda var-2))
                 (when var-4 (setf lda var-4))
                 ret-val)))
        (cond
         ((or (<= ajj zero)
              (multiple-value-bind (ret-val var-0)
                  (disnan ajj)
                (declare (ignore))
                (setf ajj var-0)
                ret-val))
          (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
                  ajj)
          (go label30)))
        (setf ajj (f2cl-lib:fsqrt ajj))
        (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
                ajj)
        (cond
         ((< j n)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10)
              (dgemv "No transpose" (f2cl-lib:int-sub n j)
               (f2cl-lib:int-sub j 1) (- one)
               (f2cl-lib:array-slice a-%data% double-float ((+ j 1) 1)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice a-%data% double-float (j 1)
                                     ((1 lda) (1 *)) a-%offset%)
               lda one
               (f2cl-lib:array-slice a-%data% double-float ((+ j 1) j)
                                     ((1 lda) (1 *)) a-%offset%)
               1)
            (declare
             (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9 var-10))
            (when var-5 (setf lda var-5))
            (when var-7 (setf lda var-7))
            (when var-8 (setf one var-8)))
          (dscal (f2cl-lib:int-sub n j) (/ one ajj)
           (f2cl-lib:array-slice a-%data% double-float ((+ j 1) j)
                                 ((1 lda) (1 *)) a-%offset%)
           1)))
      label20))))
 (go label40)
label30
 (setf info j)
label40
```

```
        (go end_label)
       end_label
        (return (values uplo nil nil lda info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dpotf2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::lda
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::disnan))))
```

———-

70.10 dpotrf

70.10.1 Axiom unit tests

— dpotrf.input —

```
)set break resume
)sys rm -f dpotrf.output
)spool dpotrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

70.10.2 Axiom help page

— dpotrf.help —

```
=====================================================================
dpotrf examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dpotrf (UPLO, N, A, LDA, INFO)
           DPOTRF

Function/Subroutine Documentation
   subroutine dpotrf (character UPLO, integer N, double precision, dimension(
       lda, * ) A, integer LDA, integer INFO)
       DPOTRF

Purpose:
   DPOTRF computes the Cholesky factorization of a real symmetric
   positive definite matrix A.

   The factorization has the form
      A = U**T * U,  if UPLO = 'U', or
      A = L  * L**T,  if UPLO = 'L',
   where U is an upper triangular matrix and L is lower triangular.

   This is the block version of the algorithm, calling Level 3 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   N-by-N upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
```

```
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if INFO = 0, the factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, the leading minor of order i is not
        positive definite, and the factorization could not be completed.
```

70.10.3 fortran code

— dpotrf.f —

```
*  ==================================================================
      SUBROUTINE DPOTRF( UPLO, N, A, LDA, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * )
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
```

```
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            J, JB, NB
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEMM, DPOTRF2, DSYRK, DTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DPOTRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Determine the block size for this environment.
*
      NB = ILAENV( 1, 'DPOTRF', UPLO, N, -1, -1, -1 )
      IF( NB.LE.1 .OR. NB.GE.N ) THEN
*
*        Use unblocked code.
*
         CALL DPOTRF2( UPLO, N, A, LDA, INFO )
      ELSE
*
*        Use blocked code.
*
```

```
      IF( UPPER ) THEN
*
*        Compute the Cholesky factorization A = U**T*U.
*
         DO 10 J = 1, N, NB
*
*           Update and factorize the current diagonal block and test
*           for non-positive-definiteness.
*
            JB = MIN( NB, N-J+1 )
            CALL DSYRK( 'Upper', 'Transpose', JB, J-1, -ONE,
     $                  A( 1, J ), LDA, ONE, A( J, J ), LDA )
            CALL DPOTRF2( 'Upper', JB, A( J, J ), LDA, INFO )
            IF( INFO.NE.0 )
     $         GO TO 30
            IF( J+JB.LE.N ) THEN
*
*              Compute the current block row.
*
               CALL DGEMM( 'Transpose', 'No transpose', JB, N-J-JB+1,
     $                     J-1, -ONE, A( 1, J ), LDA, A( 1, J+JB ),
     $                     LDA, ONE, A( J, J+JB ), LDA )
               CALL DTRSM( 'Left', 'Upper', 'Transpose', 'Non-unit',
     $                     JB, N-J-JB+1, ONE, A( J, J ), LDA,
     $                     A( J, J+JB ), LDA )
            END IF
   10    CONTINUE
*
      ELSE
*
*        Compute the Cholesky factorization A = L*L**T.
*
         DO 20 J = 1, N, NB
*
*           Update and factorize the current diagonal block and test
*           for non-positive-definiteness.
*
            JB = MIN( NB, N-J+1 )
            CALL DSYRK( 'Lower', 'No transpose', JB, J-1, -ONE,
     $                  A( J, 1 ), LDA, ONE, A( J, J ), LDA )
            CALL DPOTRF2( 'Lower', JB, A( J, J ), LDA, INFO )
            IF( INFO.NE.0 )
     $         GO TO 30
            IF( J+JB.LE.N ) THEN
*
*              Compute the current block column.
*
               CALL DGEMM( 'No transpose', 'Transpose', N-J-JB+1, JB,
     $                     J-1, -ONE, A( J+JB, 1 ), LDA, A( J, 1 ),
     $                     LDA, ONE, A( J+JB, J ), LDA )
```

```
                 CALL DTRSM( 'Right', 'Lower', 'Transpose', 'Non-unit',
     $                       N-J-JB+1, JB, ONE, A( J, J ), LDA,
     $                       A( J+JB, J ), LDA )
              END IF
   20       CONTINUE
         END IF
      END IF
      GO TO 40
*
   30 CONTINUE
      INFO = INFO + J - 1
*
   40 CONTINUE
      RETURN
*
*     End of DPOTRF
*
      END
```

———-

70.10.4 lisp code

— dpotrf.lisp —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dpotrf (uplo n a lda info)
    (declare (type (array double-float (*)) a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%))
      (prog ((j 0) (jb 0) (nb 0) (upper nil))
        (declare (type (f2cl-lib:integer4) j jb nb)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4)))
(cond
 ((/= info 0) (xerbla "DPOTRF" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(setf nb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 1 "DPOTRF" uplo n -1 -1 -1)
          (declare (ignore var-0 var-1 var-4 var-5 var-6))
          (when var-2 (setf uplo var-2))
          (when var-3 (setf n var-3))
          ret-val))
(cond
 ((or (<= nb 1) (>= nb n))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dpotrf2 uplo n a lda info)
    (declare (ignore var-2))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf lda var-3))
    (when var-4 (setf info var-4))))
 (t
  (cond
   (upper
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                  ((> j n) nil)
      (tagbody
        (setf jb
                (min (the f2cl-lib:integer4 nb)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-sub n j)
                                            1))))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9)
            (dsyrk "Upper" "Transpose" jb (f2cl-lib:int-sub j 1)
             (- one)
             (f2cl-lib:array-slice a-%data% double-float (1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             lda one
```

```
      (f2cl-lib:array-slice a-%data% double-float (j j)
                            ((1 lda) (1 *)) a-%offset%)
      lda)
   (declare (ignore var-0 var-1 var-3 var-4 var-5 var-8))
   (when var-2 (setf jb var-2))
   (when var-6 (setf lda var-6))
   (when var-7 (setf one var-7))
   (when var-9 (setf lda var-9)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (dpotrf2 "Upper" jb
      (f2cl-lib:array-slice a-%data% double-float (j j)
                            ((1 lda) (1 *)) a-%offset%)
      lda info)
   (declare (ignore var-0 var-2))
   (when var-1 (setf jb var-1))
   (when var-3 (setf lda var-3))
   (when var-4 (setf info var-4)))
 (if (/= info 0)
     (go label30))
 (cond
  ((<= (f2cl-lib:int-add j jb) n)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10 var-11 var-12)
       (dgemm "Transpose" "No transpose" jb
        (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1)
        (f2cl-lib:int-sub j 1) (- one)
        (f2cl-lib:array-slice a-%data% double-float (1 j)
                              ((1 lda) (1 *)) a-%offset%)
        lda
        (f2cl-lib:array-slice a-%data% double-float
                              (1 (f2cl-lib:int-add j jb))
                              ((1 lda) (1 *)) a-%offset%)
        lda one
        (f2cl-lib:array-slice a-%data% double-float
                              (j (f2cl-lib:int-add j jb))
                              ((1 lda) (1 *)) a-%offset%)
        lda)
     (declare
      (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-8 var-11))
     (when var-2 (setf jb var-2))
     (when var-7 (setf lda var-7))
     (when var-9 (setf lda var-9))
     (when var-10 (setf one var-10))
     (when var-12 (setf lda var-12)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (dtrsm "Left" "Upper" "Transpose" "Non-unit" jb
        (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) one
```

```
                (f2cl-lib:array-slice a-%data% double-float (j j)
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice a-%data% double-float
                                      (j (f2cl-lib:int-add j jb))
                                      ((1 lda) (1 *)) a-%offset%)
                lda)
            (declare
             (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9))
            (when var-4 (setf jb var-4))
            (when var-6 (setf one var-6))
            (when var-8 (setf lda var-8))
            (when var-10 (setf lda var-10)))))
      label10)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                 ((> j n) nil)
     (tagbody
       (setf jb
               (min (the f2cl-lib:integer4 nb)
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add (f2cl-lib:int-sub n j)
                                           1))))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
            var-9)
           (dsyrk "Lower" "No transpose" jb (f2cl-lib:int-sub j 1)
            (- one)
            (f2cl-lib:array-slice a-%data% double-float (j 1)
                                  ((1 lda) (1 *)) a-%offset%)
            lda one
            (f2cl-lib:array-slice a-%data% double-float (j j)
                                  ((1 lda) (1 *)) a-%offset%)
            lda)
         (declare (ignore var-0 var-1 var-3 var-4 var-5 var-8))
         (when var-2 (setf jb var-2))
         (when var-6 (setf lda var-6))
         (when var-7 (setf one var-7))
         (when var-9 (setf lda var-9)))
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (dpotrf2 "Lower" jb
            (f2cl-lib:array-slice a-%data% double-float (j j)
                                  ((1 lda) (1 *)) a-%offset%)
            lda info)
         (declare (ignore var-0 var-2))
         (when var-1 (setf jb var-1))
         (when var-3 (setf lda var-3))
         (when var-4 (setf info var-4)))
       (if (/= info 0)
           (go label30))
```

```
            (cond
             ((<= (f2cl-lib:int-add j jb) n)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                   var-9 var-10 var-11 var-12)
                  (dgemm "No transpose" "Transpose"
                   (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) jb
                   (f2cl-lib:int-sub j 1) (- one)
                   (f2cl-lib:array-slice a-%data% double-float ((+ j jb) 1)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda
                   (f2cl-lib:array-slice a-%data% double-float (j 1)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda one
                   (f2cl-lib:array-slice a-%data% double-float ((+ j jb) j)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda)
                (declare
                 (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-8 var-11))
                (when var-3 (setf jb var-3))
                (when var-7 (setf lda var-7))
                (when var-9 (setf lda var-9))
                (when var-10 (setf one var-10))
                (when var-12 (setf lda var-12)))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                   var-9 var-10)
                  (dtrsm "Right" "Lower" "Transpose" "Non-unit"
                   (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) jb one
                   (f2cl-lib:array-slice a-%data% double-float (j j)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda
                   (f2cl-lib:array-slice a-%data% double-float ((+ j jb) j)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda)
                (declare
                 (ignore var-0 var-1 var-2 var-3 var-4 var-7 var-9))
                (when var-5 (setf jb var-5))
                (when var-6 (setf one var-6))
                (when var-8 (setf lda var-8))
                (when var-10 (setf lda var-10)))))
        label20))))))
 (go label40)
label30
 (setf info (f2cl-lib:int-sub (f2cl-lib:int-add info j) 1))
label40
 (go end_label)
end_label
 (return (values uplo n nil lda info))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dpotrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

70.11 dpotrf2

70.11.1 Axiom unit tests

— **dpotrf2.input** —

```
)set break resume
)sys rm -f dpotrf2.output
)spool dpotrf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

70.11.2 Axiom help page

— **dpotrf2.help** —

```
=====================================================================
```

```
dpotrf2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       recursive subroutine dpotrf2 (UPLO, N, A, LDA, INFO)
           DPOTRF2

Function/Subroutine Documentation
   recursive subroutine dpotrf2 (character UPLO, integer N, double precision,
       dimension( lda, * ) A, integer LDA, integer INFO)
       DPOTRF2

Purpose:
   DPOTRF2 computes the Cholesky factorization of a real symmetric
   positive definite matrix A using the recursive algorithm.

   The factorization has the form
      A = U**T * U,  if UPLO = 'U', or
      A = L  * L**T,  if UPLO = 'L',
   where U is an upper triangular matrix and L is lower triangular.

   This is the recursive version of the algorithm. It divides
   the matrix into four submatrices:

       [  A11 | A12  ]  where A11 is n1 by n1 and A22 is n2 by n2
   A = [ -----|----- ]  with n1 = n/2
       [  A21 | A22  ]       n2 = n-n1

   The subroutine calls itself to factor A11. Update and scale A21
   or A12, update A22 then calls itself to factor A22.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
```

```
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if INFO = 0, the factor U or L from the Cholesky
  factorization A = U**T*U or A = L*L**T.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, the leading minor of order i is not
        positive definite, and the factorization could not be completed.
```

———-

70.11.3 fortran code

— dpotrf2.f —

```
*  =====================================================================
      RECURSIVE SUBROUTINE DPOTRF2( UPLO, N, A, LDA, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
```

```
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            N1, N2, IINFO
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, DISNAN
      EXTERNAL           LSAME, DISNAN
*     ..
*     .. External Subroutines ..
      EXTERNAL           DSYRK, DTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DPOTRF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     N=1 case
*
      IF( N.EQ.1 ) THEN
*
*        Test for non-positive-definiteness
*
         IF( A( 1, 1 ).LE.ZERO.OR.DISNAN( A( 1, 1 ) ) ) THEN
            INFO = 1
            RETURN
         END IF
*
```

```
*        Factor
*
         A( 1, 1 ) = SQRT( A( 1, 1 ) )
*
*     Use recursive code
*
      ELSE
         N1 = N/2
         N2 = N-N1
*
*        Factor A11
*
         CALL DPOTRF2( UPLO, N1, A( 1, 1 ), LDA, IINFO )
         IF ( IINFO.NE.0 ) THEN
            INFO = IINFO
            RETURN
         END IF
*
*        Compute the Cholesky factorization A = U**T*U
*
         IF( UPPER ) THEN
*
*           Update and scale A12
*
            CALL DTRSM( 'L', 'U', 'T', 'N', N1, N2, ONE,
     $                  A( 1, 1 ), LDA, A( 1, N1+1 ), LDA )
*
*           Update and factor A22
*
            CALL DSYRK( UPLO, 'T', N2, N1, -ONE, A( 1, N1+1 ), LDA,
     $                  ONE, A( N1+1, N1+1 ), LDA )
            CALL DPOTRF2( UPLO, N2, A( N1+1, N1+1 ), LDA, IINFO )
            IF ( IINFO.NE.0 ) THEN
               INFO = IINFO + N1
               RETURN
            END IF
*
*        Compute the Cholesky factorization A = L*L**T
*
         ELSE
*
*           Update and scale A21
*
            CALL DTRSM( 'R', 'L', 'T', 'N', N2, N1, ONE,
     $                  A( 1, 1 ), LDA, A( N1+1, 1 ), LDA )
*
*           Update and factor A22
*
            CALL DSYRK( UPLO, 'N', N2, N1, -ONE, A( N1+1, 1 ), LDA,
     $                  ONE, A( N1+1, N1+1 ), LDA )
```

```
            CALL DPOTRF2( UPLO, N2, A( N1+1, N1+1 ), LDA, IINFO )
            IF ( IINFO.NE.0 ) THEN
               INFO = IINFO + N1
               RETURN
            END IF
         END IF
      END IF
      RETURN
*
*     End of DPOTRF2
*
      END
```

———-

70.11.4 lisp code

— dpotrf2.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun *main* ()
    (prog ((n1 0) (n2 0) (iinfo 0) (upper nil)
           (uplo
            (make-array '(1) :element-type 'character :initial-element #\ ))
           (info 0) (lda 0)
           (a
            (make-array (the fixnum (reduce #'* (list lda *))) :element-type
                        'double-float))
           (n 0) (subroutine 0.0))
      (declare (type (single-float) subroutine)
               (type (array double-float (*)) a)
               (type (f2cl-lib:integer4) n lda info n1 n2 iinfo)
               (type f2cl-lib:logical upper)
               (type (string 1) uplo))
      '"****NOT TRANSLATED: (RECURSIVE SUBROUTINE DPOTRF2 (UPLO |,| N |,| A |,| LDA |,| INFO))"
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (cond
       ((and (not upper)
             (not
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4)))
(cond
 ((/= info 0) (xerbla "DPOTRF2" (f2cl-lib:int-sub info)) (go end_label)))
(if (= n 0)
    (go end_label))
(cond
 ((= n 1)
  (cond
   ((or (<= (f2cl-lib:fref a (1 1) ((1 lda) (1 *))) zero)
        (multiple-value-bind (ret-val var-0)
            (disnan (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))
          (declare (ignore))
          (setf (f2cl-lib:fref a (1 1) ((1 lda) (1 *))) var-0)
          ret-val))
    (setf info 1) (go end_label)))
  (setf (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))
          (f2cl-lib:fsqrt (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))))
 (t (setf n1 (the f2cl-lib:integer4 (truncate n 2)))
  (setf n2 (f2cl-lib:int-sub n n1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dpotrf2 uplo n1
       (f2cl-lib:array-slice a double-float (1 1) ((1 lda) (1 *))) lda
       iinfo)
    (declare (ignore var-2))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n1 var-1))
    (when var-3 (setf lda var-3))
    (when var-4 (setf iinfo var-4)))
  (cond ((/= iinfo 0) (setf info iinfo) (go end_label)))
  (cond
   (upper
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (dtrsm "L" "U" "T" "N" n1 n2 one
         (f2cl-lib:array-slice a double-float (1 1) ((1 lda) (1 *))) lda
         (f2cl-lib:array-slice a double-float (1 (f2cl-lib:int-add n1 1))
                               ((1 lda) (1 *)))
         lda)
      (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
      (when var-4 (setf n1 var-4))
      (when var-5 (setf n2 var-5))
```

```
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf lda var-10)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dsyrk uplo "T" n2 n1 (- one)
       (f2cl-lib:array-slice a double-float (1 (f2cl-lib:int-add n1 1))
                             ((1 lda) (1 *)))
       lda one
       (f2cl-lib:array-slice a double-float
                             ((+ n1 1) (f2cl-lib:int-add n1 1))
                             ((1 lda) (1 *)))
       lda)
    (declare (ignore var-1 var-4 var-5 var-8))
    (when var-0 (setf uplo var-0))
    (when var-2 (setf n2 var-2))
    (when var-3 (setf n1 var-3))
    (when var-6 (setf lda var-6))
    (when var-7 (setf one var-7))
    (when var-9 (setf lda var-9)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dpotrf2 uplo n2
       (f2cl-lib:array-slice a double-float
                             ((+ n1 1) (f2cl-lib:int-add n1 1))
                             ((1 lda) (1 *)))
       lda iinfo)
    (declare (ignore var-2))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n2 var-1))
    (when var-3 (setf lda var-3))
    (when var-4 (setf iinfo var-4)))
  (cond
   ((/= iinfo 0) (setf info (f2cl-lib:int-add iinfo n1))
    (go end_label))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dtrsm "R" "L" "T" "N" n2 n1 one
       (f2cl-lib:array-slice a double-float (1 1) ((1 lda) (1 *))) lda
       (f2cl-lib:array-slice a double-float ((+ n1 1) 1)
                             ((1 lda) (1 *)))
       lda)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n2 var-4))
    (when var-5 (setf n1 var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf lda var-10)))
  (multiple-value-bind
```

```
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (dsyrk uplo "N" n2 n1 (- one)
               (f2cl-lib:array-slice a double-float ((+ n1 1) 1)
                                     ((1 lda) (1 *)))
               lda one
               (f2cl-lib:array-slice a double-float
                                     ((+ n1 1) (f2cl-lib:int-add n1 1))
                                     ((1 lda) (1 *)))
               lda)
            (declare (ignore var-1 var-4 var-5 var-8))
            (when var-0 (setf uplo var-0))
            (when var-2 (setf n2 var-2))
            (when var-3 (setf n1 var-3))
            (when var-6 (setf lda var-6))
            (when var-7 (setf one var-7))
            (when var-9 (setf lda var-9)))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dpotrf2 uplo n2
               (f2cl-lib:array-slice a double-float
                                     ((+ n1 1) (f2cl-lib:int-add n1 1))
                                     ((1 lda) (1 *)))
               lda iinfo)
            (declare (ignore var-2))
            (when var-0 (setf uplo var-0))
            (when var-1 (setf n2 var-1))
            (when var-3 (setf lda var-3))
            (when var-4 (setf iinfo var-4)))
          (cond
           ((/= iinfo 0) (setf info (f2cl-lib:int-add iinfo n1))
            (go end_label)))))))
      (go end_label)
     end_label
      (return nil))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::*main*
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types 'nil :return-values 'nil
                                            :calls 'nil)))
```

———-

70.12 dpotri

70.12.1 Axiom unit tests

— dpotri.input —

```
)set break resume
)sys rm -f dpotri.output
)spool dpotri.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

70.12.2 Axiom help page

— dpotri.help —

```
====================================================================
dpotri examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dpotri (UPLO, N, A, LDA, INFO)
           DPOTRI

Function/Subroutine Documentation
   subroutine dpotri (character UPLO, integer N, double precision, dimension(
       lda, * ) A, integer LDA, integer INFO)
       DPOTRI

Purpose:
   DPOTRI computes the inverse of a real symmetric positive definite
   matrix A using the Cholesky factorization A = U**T*U or A = L*L**T
   computed by DPOTRF.

Parameters:
```

```
 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the triangular factor U or L from the Cholesky
   factorization A = U**T*U or A = L*L**T, as computed by DPOTRF.
   On exit, the upper or lower triangle of the (symmetric)
   inverse of A, overwriting the input factor U or L.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, the (i,i) element of the factor U or L is
         zero, and the inverse could not be computed.
```

———-

70.12.3 fortran code

— **dpotri.f** —

```
*  =====================================================================
      SUBROUTINE DPOTRI( UPLO, N, A, LDA, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
```

```
      DOUBLE PRECISION   A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLAUUM, DTRTRI, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DPOTRI', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Invert the triangular Cholesky factor U or L.
*
      CALL DTRTRI( UPLO, 'Non-unit', N, A, LDA, INFO )
      IF( INFO.GT.0 )
     $   RETURN
*
*     Form inv(U) * inv(U)**T or inv(L)**T * inv(L).
*
      CALL DLAUUM( UPLO, N, A, LDA, INFO )
*
      RETURN
*
*     End of DPOTRI
```

```
*
      END
```

———-

70.12.4 lisp code

— **dpotri.lisp** —

```
(defun dpotri (uplo n a lda info)
  (declare (type (array double-float (*)) a)
           (type (f2cl-lib:integer4) info lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a double-float a-%data% a-%offset%))
    (prog ()
      (declare)
      (setf info 0)
      (cond
       ((and
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -4)))
      (cond
       ((/= info 0) (xerbla "DPOTRI" (f2cl-lib:int-sub info)) (go end_label)))
      (if (= n 0)
          (go end_label))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (dtrtri uplo "Non-unit" n a lda info)
        (declare (ignore var-1 var-3))
        (when var-0 (setf uplo var-0))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-5 (setf info var-5)))
```

```
      (if (> info 0)
          (go end_label))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dlauum uplo n a lda info)
        (declare (ignore var-2))
        (setf uplo var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf info var-4))
      (go end_label)
     end_label
      (return (values uplo n nil lda info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dpotri
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::dlauum))))
```

———-

70.13 dpotrs

70.13.1 Axiom unit tests

— **dpotrs.input** —

```
)set break resume
)sys rm -f dpotrs.output
)spool dpotrs.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

70.13.2 Axiom help page

— **dpotrs.help** —

```
=====================================================================
dpotrs examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dpotrs (UPLO, N, NRHS, A, LDA, B, LDB, INFO)
           DPOTRS

Function/Subroutine Documentation
   subroutine dpotrs (character UPLO, integer N, integer NRHS, double
       precision, dimension( lda, * ) A, integer LDA, double precision,
       dimension( ldb, * ) B, integer LDB, integer INFO)
       DPOTRS

Purpose:
   DPOTRS solves a system of linear equations A*X = B with a symmetric
   positive definite matrix A using the Cholesky factorization
   A = U**T*U or A = L*L**T computed by DPOTRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
```

```
  of the matrix B.  NRHS >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  The triangular factor U or L from the Cholesky factorization
  A = U**T*U or A = L*L**T, as computed by DPOTRF.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the right hand side matrix B.
  On exit, the solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

70.13.3 fortran code

— **dpotrs.f** —

```
*  =====================================================================
      SUBROUTINE DPOTRS( UPLO, N, NRHS, A, LDA, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * )
*     ..
*
```

```
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DPOTRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*        Solve A*X = B where A = U**T *U.
*
```

```
*        Solve U**T *X = B, overwriting B with X.
*
         CALL DTRSM( 'Left', 'Upper', 'Transpose', 'Non-unit', N, NRHS,
     $               ONE, A, LDA, B, LDB )
*
*        Solve U*X = B, overwriting B with X.
*
         CALL DTRSM( 'Left', 'Upper', 'No transpose', 'Non-unit', N,
     $               NRHS, ONE, A, LDA, B, LDB )
      ELSE
*
*        Solve A*X = B where A = L*L**T.
*
*        Solve L*X = B, overwriting B with X.
*
         CALL DTRSM( 'Left', 'Lower', 'No transpose', 'Non-unit', N,
     $               NRHS, ONE, A, LDA, B, LDB )
*
*        Solve L**T *X = B, overwriting B with X.
*
         CALL DTRSM( 'Left', 'Lower', 'Transpose', 'Non-unit', N, NRHS,
     $               ONE, A, LDA, B, LDB )
      END IF
*
      RETURN
*
*     End of DPOTRS
*
      END
```

———-

70.13.4 lisp code

— **dpotrs.lisp** —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dpotrs (uplo n nrhs a lda b ldb$ info)
    (declare (type (array double-float (*)) b a)
             (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%))
      (prog ((upper nil))
```

```
(declare (type f2cl-lib:logical upper))
(setf info 0)
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7)))
(cond
 ((/= info 0) (xerbla "DPOTRS" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= n 0) (= nrhs 0))
    (go end_label))
(cond
 (upper
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dtrsm "Left" "Upper" "Transpose" "Non-unit" n nrhs one a lda b
       ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dtrsm "Left" "Upper" "No transpose" "Non-unit" n nrhs one a lda
       b ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
```

```
            (when var-10 (setf ldb$ var-10))))
          (t
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                var-10)
               (dtrsm "Left" "Lower" "No transpose" "Non-unit" n nrhs one a lda
                b ldb$)
             (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
             (when var-4 (setf n var-4))
             (when var-5 (setf nrhs var-5))
             (when var-6 (setf one var-6))
             (when var-8 (setf lda var-8))
             (when var-10 (setf ldb$ var-10)))
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                var-10)
               (dtrsm "Left" "Lower" "Transpose" "Non-unit" n nrhs one a lda b
                ldb$)
             (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
             (when var-4 (setf n var-4))
             (when var-5 (setf nrhs var-5))
             (when var-6 (setf one var-6))
             (when var-8 (setf lda var-8))
             (when var-10 (setf ldb$ var-10)))))
         (go end_label)
        end_label
         (return (values uplo n nrhs nil lda nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dpotrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

Chapter 71

LAPACK - Positive Definite Matrix, Comp. Routines, Real

71.1 sla_porcond

71.1.1 Axiom unit tests

— slaporcond.input —

```
)set break resume
)sys rm -f slaporcond.output
)spool slaporcond.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

71.1.2 Axiom help page

— sla_porcond.help —

```
======================================================================
sla_porcond examples
======================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       real function sla_porcond (UPLO, N, A, LDA, AF, LDAF, CMODE, C, INFO,
           WORK, IWORK)
           SLA_PORCOND estimates the Skeel condition number for a symmetric
           positive-definite matrix.

Function/Subroutine Documentation
   real function sla_porcond (character UPLO, integer N, real, dimension( lda,
       * ) A, integer LDA, real, dimension( ldaf, * ) AF, integer LDAF,
       integer CMODE, real, dimension( * ) C, integer INFO, real, dimension( *
       ) WORK, integer, dimension( * ) IWORK)
       SLA_PORCOND estimates the Skeel condition number for a symmetric
       positive-definite matrix.

Purpose:
   SLA_PORCOND Estimates the Skeel condition number of  op(A) * op2(C)
   where op2 is determined by CMODE as follows
   CMODE =  1    op2(C) = C
   CMODE =  0    op2(C) = I
   CMODE = -1    op2(C) = inv(C)
   The Skeel condition number  cond(A) = norminf( |inv(A)||A| )
   is computed by computing scaling factors R such that
   diag(R)*A*op2(C) is row equilibrated and computing the standard
   infinity-norm condition number.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).
```

```
AF
  AF is REAL array, dimension (LDAF,N)
  The triangular factor U or L from the Cholesky factorization
  A = U**T*U or A = L*L**T, as computed by SPOTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

CMODE
  CMODE is INTEGER
  Determines op2(C) in the formula op(A) * op2(C) as follows:
  CMODE =  1    op2(C) = C
  CMODE =  0    op2(C) = I
  CMODE = -1    op2(C) = inv(C)

C
  C is REAL array, dimension (N)
  The vector C in the formula op(A) * op2(C).

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  i > 0:  The ith argument is invalid.

WORK
  WORK is REAL array, dimension (3*N).
  Workspace.

IWORK
  IWORK is INTEGER array, dimension (N).
  Workspace.
```

71.1.3 fortran code

— sla_porcond.f —

```
*  =====================================================================
      REAL FUNCTION SLA_PORCOND( UPLO, N, A, LDA, AF, LDAF, CMODE, C,
     $                           INFO, WORK, IWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
```

```
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            N, LDA, LDAF, INFO, CMODE
      REAL               A( LDA, * ), AF( LDAF, * ), WORK( * ),
     $                   C( * )
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            KASE, I, J
      REAL               AINVNM, TMP
      LOGICAL            UP
*     ..
*     .. Array Arguments ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ISAMAX
      EXTERNAL           LSAME, ISAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLACN2, SPOTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
      SLA_PORCOND = 0.0
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -2
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SLA_PORCOND', -INFO )
         RETURN
      END IF

      IF( N.EQ.0 ) THEN
         SLA_PORCOND = 1.0
         RETURN
      END IF
      UP = .FALSE.
```

```
      IF ( LSAME( UPLO, 'U' ) ) UP = .TRUE.
*
*     Compute the equilibration matrix R such that
*     inv(R)*A*C has unit 1-norm.
*
      IF ( UP ) THEN
         DO I = 1, N
            TMP = 0.0
            IF ( CMODE .EQ. 1 ) THEN
               DO J = 1, I
                  TMP = TMP + ABS( A( J, I ) * C( J ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + ABS( A( I, J ) * C( J ) )
               END DO
            ELSE IF ( CMODE .EQ. 0 ) THEN
               DO J = 1, I
                  TMP = TMP + ABS( A( J, I ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + ABS( A( I, J ) )
               END DO
            ELSE
               DO J = 1, I
                  TMP = TMP + ABS( A( J ,I ) / C( J ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + ABS( A( I, J ) / C( J ) )
               END DO
            END IF
            WORK( 2*N+I ) = TMP
         END DO
      ELSE
         DO I = 1, N
            TMP = 0.0
            IF ( CMODE .EQ. 1 ) THEN
               DO J = 1, I
                  TMP = TMP + ABS( A( I, J ) * C( J ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + ABS( A( J, I ) * C( J ) )
               END DO
            ELSE IF ( CMODE .EQ. 0 ) THEN
               DO J = 1, I
                  TMP = TMP + ABS( A( I, J ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + ABS( A( J, I ) )
               END DO
            ELSE
```

```
            DO J = 1, I
               TMP = TMP + ABS( A( I, J ) / C( J ) )
            END DO
            DO J = I+1, N
               TMP = TMP + ABS( A( J, I ) / C( J ) )
            END DO
         END IF
         WORK( 2*N+I ) = TMP
      END DO
   ENDIF
*
*     Estimate the norm of inv(op(A)).
*
   AINVNM = 0.0

   KASE = 0
10 CONTINUE
   CALL SLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
   IF( KASE.NE.0 ) THEN
      IF( KASE.EQ.2 ) THEN
*
*        Multiply by R.
*
         DO I = 1, N
            WORK( I ) = WORK( I ) * WORK( 2*N+I )
         END DO

         IF (UP) THEN
            CALL SPOTRS( 'Upper', N, 1, AF, LDAF, WORK, N, INFO )
         ELSE
            CALL SPOTRS( 'Lower', N, 1, AF, LDAF, WORK, N, INFO )
         ENDIF
*
*        Multiply by inv(C).
*
         IF ( CMODE .EQ. 1 ) THEN
            DO I = 1, N
               WORK( I ) = WORK( I ) / C( I )
            END DO
         ELSE IF ( CMODE .EQ. -1 ) THEN
            DO I = 1, N
               WORK( I ) = WORK( I ) * C( I )
            END DO
         END IF
      ELSE
*
*        Multiply by inv(C**T).
*
         IF ( CMODE .EQ. 1 ) THEN
            DO I = 1, N
```

```
                 WORK( I ) = WORK( I ) / C( I )
              END DO
           ELSE IF ( CMODE .EQ. -1 ) THEN
              DO I = 1, N
                 WORK( I ) = WORK( I ) * C( I )
              END DO
           END IF

           IF ( UP ) THEN
              CALL SPOTRS( 'Upper', N, 1, AF, LDAF, WORK, N, INFO )
           ELSE
              CALL SPOTRS( 'Lower', N, 1, AF, LDAF, WORK, N, INFO )
           ENDIF
*
*          Multiply by R.
*
           DO I = 1, N
              WORK( I ) = WORK( I ) * WORK( 2*N+I )
           END DO
        END IF
        GO TO 10
     END IF
*
*    Compute the estimate of the reciprocal condition number.
*
     IF( AINVNM .NE. 0.0 )
    $   SLA_PORCOND = ( 1.0 / AINVNM )
*
     RETURN
*
     END
```

71.1.4 lisp code

— sla_porcond.lisp —

```
(defun sla_porcond (uplo n a lda af ldaf cmode c info work iwork)
  (declare (type (array f2cl-lib:integer4 (*)) iwork)
           (type (array single-float (*)) work c af a)
           (type (f2cl-lib:integer4) info cmode ldaf lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a single-float a-%data% a-%offset%)
       (af single-float af-%data% af-%offset%)
       (c single-float c-%data% c-%offset%)
```

```
    (work single-float work-%data% work-%offset%)
    (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4)) (up nil)
         (ainvnm 0.0) (tmp 0.0) (kase 0) (i 0) (j 0) (sla_porcond 0.0))
    (declare (type (f2cl-lib:integer4) j i kase)
             (type (single-float) sla_porcond tmp ainvnm)
             (type f2cl-lib:logical up)
             (type (array f2cl-lib:integer4 (3)) isave))
    (setf sla_porcond 0.0)
    (setf info 0)
    (cond ((< n 0) (setf info -2)))
    (cond
     ((/= info 0) (xerbla "SLA_PORCOND" (f2cl-lib:int-sub info))
      (go end_label)))
    (cond ((= n 0) (setf sla_porcond 1.0) (go end_label)))
    (setf up f2cl-lib:%false%)
    (if (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)
        (setf up f2cl-lib:%true%))
    (cond
     (up
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf tmp 0.0)
          (cond
           ((= cmode 1)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j i) nil)
              (tagbody
                (setf tmp
                        (+ tmp
                           (abs
                            (*
                             (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                            a-%offset%)
                             (f2cl-lib:fref c-%data% (j) ((1 *))
                                            c-%offset%)))))
               label100001))
            (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf tmp
                        (+ tmp
                           (abs
                            (*
                             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
```

```
                                    a-%offset%)
                      (f2cl-lib:fref c-%data% (j) ((1 *))
                                    c-%offset%)))))
      label100002)))
  ((= cmode 0)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j i) nil)
     (tagbody
       (setf tmp
               (+ tmp
                  (abs
                   (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                  a-%offset%))))
      label100003))
   (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf tmp
               (+ tmp
                  (abs
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%))))
      label100004)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j i) nil)
     (tagbody
       (setf tmp
               (+ tmp
                  (abs
                   (/
                    (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                   a-%offset%)
                    (f2cl-lib:fref c-%data% (j) ((1 *))
                                   c-%offset%)))))
      label100005))
   (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf tmp
               (+ tmp
                  (abs
                   (/
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%)
                    (f2cl-lib:fref c-%data% (j) ((1 *))
                                   c-%offset%)))))
      label100006))))
(setf (f2cl-lib:fref work-%data%
                     ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) i))
```

```
                              ((1 *)) work-%offset%)
                tmp)
     label100000)))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf tmp 0.0)
       (cond
        ((= cmode 1)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j i) nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (abs
                         (*
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref c-%data% (j) ((1 *))
                                         c-%offset%)))))
             label100008))
         (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (abs
                         (*
                          (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref c-%data% (j) ((1 *))
                                         c-%offset%)))))
             label100009)))
        ((= cmode 0)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j i) nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (abs
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%))))
             label100010))
         (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (abs
```

```
                          (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                         a-%offset%))))
              label100011)))
           (t
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j i) nil)
              (tagbody
                (setf tmp
                        (+ tmp
                           (abs
                            (/
                             (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                            a-%offset%)
                             (f2cl-lib:fref c-%data% (j) ((1 *))
                                            c-%offset%)))))
              label100012))
            (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf tmp
                        (+ tmp
                           (abs
                            (/
                             (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                            a-%offset%)
                             (f2cl-lib:fref c-%data% (j) ((1 *))
                                            c-%offset%)))))
              label100013))))
         (setf (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) i))
                              ((1 *)) work-%offset%)
                tmp)
       label100007))))
  (setf ainvnm 0.0)
  (setf kase 0)
 label10
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacn2 n
       (f2cl-lib:array-slice work-%data% single-float ((+ n 1)) ((1 *))
                             work-%offset%)
       work iwork ainvnm kase isave)
    (declare (ignore var-1 var-2 var-3 var-6))
    (setf n var-0)
    (setf ainvnm var-4)
    (setf kase var-5))
  (cond
   ((/= kase 0)
    (cond
     ((= kase 2)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add
                                  (f2cl-lib:int-mul 2 n) i))
                                ((1 *)) work-%offset%)))
     label100014))
  (cond
   (up
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (spotrs "Upper" n 1 af ldaf work n info)
      (declare (ignore var-0 var-2 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-4 (setf ldaf var-4))
      (when var-6 (setf n var-6))
      (when var-7 (setf info var-7))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (spotrs "Lower" n 1 af ldaf work n info)
      (declare (ignore var-0 var-2 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-4 (setf ldaf var-4))
      (when var-6 (setf n var-6))
      (when var-7 (setf info var-7)))))
  (cond
   ((= cmode 1)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (/
                 (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
       label100015)))
   ((= cmode (f2cl-lib:int-sub 1))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (*
                 (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
       label100016)))))
 (t
  (cond
   ((= cmode 1)
```

```
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (/
                   (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                   (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
         label100017)))
     ((= cmode (f2cl-lib:int-sub 1))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (*
                   (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                   (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
         label100018))))
    (cond
     (up
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (spotrs "Upper" n 1 af ldaf work n info)
        (declare (ignore var-0 var-2 var-3 var-5))
        (when var-1 (setf n var-1))
        (when var-4 (setf ldaf var-4))
        (when var-6 (setf n var-6))
        (when var-7 (setf info var-7))))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (spotrs "Lower" n 1 af ldaf work n info)
        (declare (ignore var-0 var-2 var-3 var-5))
        (when var-1 (setf n var-1))
        (when var-4 (setf ldaf var-4))
        (when var-6 (setf n var-6))
        (when var-7 (setf info var-7)))))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add
                                    (f2cl-lib:int-mul 2 n) i))
                                  ((1 *)) work-%offset%)))
       label100019))))
  (go label10)))
(if (/= ainvnm 0.0)
    (setf sla_porcond (/ 1.0 ainvnm)))
(go end_label)
```

```
     end_label
      (return
       (values sla_porcond uplo n nil nil nil ldaf nil nil info nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sla_porcond
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*)))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::ldaf nil nil
                                              fortran-to-lisp::info nil nil)
                                            :calls '(fortran-to-lisp::slacn2))))
```

———-

71.2 sla_porfsx_extended

71.2.1 Axiom unit tests

— slaporfsxextended.input —

```
)set break resume
)sys rm -f slaporfsxextended.output
)spool slaporfsxextended.output
)set message test on
)set message auto off
)clear all

)spool
```

```
)lisp (bye)
```

———-

71.2.2 Axiom help page [Demm05]

— **sla_porfsx_extended.help** —

```
=====================================================================
sla_porfsx_extended examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sla_porfsx_extended (PREC_TYPE, UPLO, N, NRHS, A, LDA, AF,
           LDAF, COLEQU, C, B, LDB, Y, LDY, BERR_OUT, N_NORMS, ERR_BNDS_NORM,
           ERR_BNDS_COMP, RES, AYB, DY, Y_TAIL, RCOND, ITHRESH, RTHRESH,
           DZ_UB, IGNORE_CWISE, INFO)
           SLA_PORFSX_EXTENDED improves the computed solution to a system of
           linear equations for symmetric or Hermitian positive-definite
           matrices by performing extra-precise iterative refinement and
           provides error bounds and backward error estimates for the
           solution.

Function/Subroutine Documentation
   subroutine sla_porfsx_extended (integer PREC_TYPE, character UPLO, integer
       N, integer NRHS, real, dimension( lda, * ) A, integer LDA, real,
       dimension( ldaf, * ) AF, integer LDAF, logical COLEQU, real, dimension(
       * ) C, real, dimension( ldb, * ) B, integer LDB, real, dimension( ldy,
       * ) Y, integer LDY, real, dimension( * ) BERR_OUT, integer N_NORMS,
       real, dimension( nrhs, * ) ERR_BNDS_NORM, real, dimension( nrhs, * )
       ERR_BNDS_COMP, real, dimension( * ) RES, real, dimension(*) AYB, real,
       dimension( * ) DY, real, dimension( * ) Y_TAIL, real RCOND, integer
       ITHRESH, real RTHRESH, real DZ_UB, logical IGNORE_CWISE, integer INFO)
       SLA_PORFSX_EXTENDED improves the computed solution to a system of
       linear equations for symmetric or Hermitian positive-definite matrices
       by performing extra-precise iterative refinement and provides error
       bounds and backward error estimates for the solution.

Purpose:
   SLA_PORFSX_EXTENDED improves the computed solution to a system of
   linear equations by performing extra-precise iterative refinement
   and provides error bounds and backward error estimates for the solution.
   This subroutine is called by SPORFSX to perform iterative refinement.
```

```
   In addition to normwise error bound, the code provides maximum
   componentwise error bound if possible. See comments for ERR_BNDS_NORM
   and ERR_BNDS_COMP for details of the error bounds. Note that this
   subroutine is only resonsible for setting the second fields of
   ERR_BNDS_NORM and ERR_BNDS_COMP.

Parameters:

 PREC_TYPE
   PREC_TYPE is INTEGER
   Specifies the intermediate precision to be used in refinement.
   The value is defined by ILAPREC(P) where P is a CHARACTER and
   P    = 'S':  Single
        = 'D':  Double
        = 'I':  Indigenous
        = 'X', 'E':  Extra

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right-hand-sides, i.e., the number of columns of the
   matrix B.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is REAL array, dimension (LDAF,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**T*U or A = L*L**T, as computed by SPOTRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 COLEQU
   COLEQU is LOGICAL
```

```
  If .TRUE. then column equilibration was done to A before calling
  this routine. This is needed to compute the solution and error
  bounds correctly.

C
  C is REAL array, dimension (N)
  The column scale factors for A. If COLEQU = .FALSE., C
  is not accessed. If C is input, each element of C should be a power
  of the radix to ensure a reliable solution and error estimates.
  Scaling by powers of the radix does not cause rounding errors unless
  the result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is REAL array, dimension (LDB,NRHS)
  The right-hand-side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

Y
  Y is REAL array, dimension (LDY,NRHS)
  On entry, the solution matrix X, as computed by SPOTRS.
  On exit, the improved solution matrix Y.

LDY
  LDY is INTEGER
  The leading dimension of the array Y.  LDY >= max(1,N).

BERR_OUT
  BERR_OUT is REAL array, dimension (NRHS)
  On exit, BERR_OUT(j) contains the componentwise relative backward
  error for right-hand-side j from the formula
  max(i) ( abs(RES(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
  where abs(Z) is the componentwise absolute value of the matrix
  or vector Z. This is computed by SLA_LIN_BERR.

N_NORMS
  N_NORMS is INTEGER
  Determines which error bounds to return (see ERR_BNDS_NORM
  and ERR_BNDS_COMP).
  If N_NORMS >= 1 return normwise error bounds.
  If N_NORMS >= 2 return componentwise error bounds.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
```

```
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
           max_j (abs(XTRUE(j,i) - X(j,i)))
            ------------------------------
                    max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

ERR_BNDS_COMP
  ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                 abs(XTRUE(j,i) - X(j,i))
           max_j ----------------------
                       abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
```

```
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

RES
  RES is REAL array, dimension (N)
  Workspace to hold the intermediate residual.

AYB
  AYB is REAL array, dimension (N)
  Workspace. This can be the same workspace passed for Y_TAIL.

DY
  DY is REAL array, dimension (N)
  Workspace to hold the intermediate solution.

Y_TAIL
  Y_TAIL is REAL array, dimension (N)
```

```
  Workspace to hold the trailing bits of the intermediate solution.

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

ITHRESH
  ITHRESH is INTEGER
  The maximum number of residual computations allowed for
  refinement. The default is 10. For 'aggressive' set to 100 to
  permit convergence using approximate factorizations or
  factorizations other than LU. If the factorization uses a
  technique other than Gaussian elimination, the guarantees in
  ERR_BNDS_NORM and ERR_BNDS_COMP may no longer be trustworthy.

RTHRESH
  RTHRESH is REAL
  Determines when to stop refinement if the error estimate stops
  decreasing. Refinement will stop when the next solution no longer
  satisfies norm(dx_{i+1}) < RTHRESH * norm(dx_i) where norm(Z) is
  the infinity norm of Z. RTHRESH satisfies 0 < RTHRESH <= 1. The
  default value is 0.5. For 'aggressive' set to 0.9 to permit
  convergence on extremely ill-conditioned matrices. See LAWN 165
  for more details.

DZ_UB
  DZ_UB is REAL
  Determines when to start considering componentwise convergence.
  Componentwise convergence is only considered after each component
  of the solution Y is stable, which we definte as the relative
  change in each component being less than DZ_UB. The default value
  is 0.25, requiring the first bit to be stable. See LAWN 165 for
  more details.

IGNORE_CWISE
  IGNORE_CWISE is LOGICAL
  If .TRUE. then ignore componentwise convergence. Default value
  is .FALSE..

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  < 0:  if INFO = -i, the ith argument to SPOTRS had an illegal value
```

———-

71.2.3 fortran code

— sla_porfsx_extended.f —

```
*  =====================================================================
      SUBROUTINE SLA_PORFSX_EXTENDED( PREC_TYPE, UPLO, N, NRHS, A, LDA,
     $                                AF, LDAF, COLEQU, C, B, LDB, Y,
     $                                LDY, BERR_OUT, N_NORMS,
     $                                ERR_BNDS_NORM, ERR_BNDS_COMP, RES,
     $                                AYB, DY, Y_TAIL, RCOND, ITHRESH,
     $                                RTHRESH, DZ_UB, IGNORE_CWISE,
     $                                INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDAF, LDB, LDY, N, NRHS, PREC_TYPE,
     $                   N_NORMS, ITHRESH
      CHARACTER          UPLO
      LOGICAL            COLEQU, IGNORE_CWISE
      REAL               RTHRESH, DZ_UB
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   Y( LDY, * ), RES( * ), DY( * ), Y_TAIL( * )
      REAL               C( * ), AYB(*), RCOND, BERR_OUT( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            UPLO2, CNT, I, J, X_STATE, Z_STATE
      REAL               YK, DYK, YMIN, NORMY, NORMX, NORMDX, DXRAT,
     $                   DZRAT, PREVNORMDX, PREV_DZ_Z, DXRATMAX,
     $                   DZRATMAX, DX_X, DZ_Z, FINAL_DX_X, FINAL_DZ_Z,
     $                   EPS, HUGEVAL, INCR_THRESH
      LOGICAL            INCR_PREC
*     ..
*     .. Parameters ..
      INTEGER           UNSTABLE_STATE, WORKING_STATE, CONV_STATE,
     $                  NOPROG_STATE, Y_PREC_STATE, BASE_RESIDUAL,
```

```
     $                  EXTRA_RESIDUAL, EXTRA_Y
      PARAMETER         ( UNSTABLE_STATE = 0, WORKING_STATE = 1,
     $                  CONV_STATE = 2, NOPROG_STATE = 3 )
      PARAMETER         ( BASE_RESIDUAL = 0, EXTRA_RESIDUAL = 1,
     $                  EXTRA_Y = 2 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           ILAUPLO
      INTEGER            ILAUPLO
*     ..
*     .. External Subroutines ..
      EXTERNAL          SAXPY, SCOPY, SPOTRS, SSYMV, BLAS_SSYMV_X,
     $                  BLAS_SSYMV2_X, SLA_SYAMV, SLA_WWADDW,
     $                  SLA_LIN_BERR
      REAL              SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC         ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      IF (INFO.NE.0) RETURN
      EPS = SLAMCH( 'Epsilon' )
      HUGEVAL = SLAMCH( 'Overflow' )
*     Force HUGEVAL to Inf
      HUGEVAL = HUGEVAL * HUGEVAL
*     Using HUGEVAL may lead to spurious underflows.
      INCR_THRESH = REAL( N ) * EPS

      IF ( LSAME ( UPLO, 'L' ) ) THEN
         UPLO2 = ILAUPLO( 'L' )
      ELSE
```

```
         UPLO2 = ILAUPLO( 'U' )
      ENDIF

      DO J = 1, NRHS
         Y_PREC_STATE = EXTRA_RESIDUAL
         IF ( Y_PREC_STATE .EQ. EXTRA_Y ) THEN
            DO I = 1, N
               Y_TAIL( I ) = 0.0
            END DO
         END IF

         DXRAT = 0.0
         DXRATMAX = 0.0
         DZRAT = 0.0
         DZRATMAX = 0.0
         FINAL_DX_X = HUGEVAL
         FINAL_DZ_Z = HUGEVAL
         PREVNORMDX = HUGEVAL
         PREV_DZ_Z = HUGEVAL
         DZ_Z = HUGEVAL
         DX_X = HUGEVAL

         X_STATE = WORKING_STATE
         Z_STATE = UNSTABLE_STATE
         INCR_PREC = .FALSE.

         DO CNT = 1, ITHRESH
*
*         Compute residual RES = B_s - op(A_s) * Y,
*             op(A) = A, A**T, or A**H depending on TRANS (and type).
*
            CALL SCOPY( N, B( 1, J ), 1, RES, 1 )
            IF ( Y_PREC_STATE .EQ. BASE_RESIDUAL ) THEN
               CALL SSYMV( UPLO, N, -1.0, A, LDA, Y(1,J), 1,
     $              1.0, RES, 1 )
            ELSE IF ( Y_PREC_STATE .EQ. EXTRA_RESIDUAL ) THEN
               CALL BLAS_SSYMV_X( UPLO2, N, -1.0, A, LDA,
     $              Y( 1, J ), 1, 1.0, RES, 1, PREC_TYPE )
            ELSE
               CALL BLAS_SSYMV2_X(UPLO2, N, -1.0, A, LDA,
     $              Y(1, J), Y_TAIL, 1, 1.0, RES, 1, PREC_TYPE)
            END IF

!         XXX: RES is no longer needed.
            CALL SCOPY( N, RES, 1, DY, 1 )
            CALL SPOTRS( UPLO, N, 1, AF, LDAF, DY, N, INFO )
*
*         Calculate relative changes DX_X, DZ_Z and ratios DXRAT, DZRAT.
*
            NORMX = 0.0
```

```
            NORMY = 0.0
            NORMDX = 0.0
            DZ_Z = 0.0
            YMIN = HUGEVAL

            DO I = 1, N
               YK = ABS( Y( I, J ) )
               DYK = ABS( DY( I ) )

               IF ( YK .NE. 0.0 ) THEN
                  DZ_Z = MAX( DZ_Z, DYK / YK )
               ELSE IF ( DYK .NE. 0.0 ) THEN
                  DZ_Z = HUGEVAL
               END IF

               YMIN = MIN( YMIN, YK )

               NORMY = MAX( NORMY, YK )

               IF ( COLEQU ) THEN
                  NORMX = MAX( NORMX, YK * C( I ) )
                  NORMDX = MAX( NORMDX, DYK * C( I ) )
               ELSE
                  NORMX = NORMY
                  NORMDX = MAX( NORMDX, DYK )
               END IF
            END DO

            IF ( NORMX .NE. 0.0 ) THEN
               DX_X = NORMDX / NORMX
            ELSE IF ( NORMDX .EQ. 0.0 ) THEN
               DX_X = 0.0
            ELSE
               DX_X = HUGEVAL
            END IF

            DXRAT = NORMDX / PREVNORMDX
            DZRAT = DZ_Z / PREV_DZ_Z
*
*         Check termination criteria.
*
            IF ( YMIN*RCOND .LT. INCR_THRESH*NORMY
     $           .AND. Y_PREC_STATE .LT. EXTRA_Y )
     $           INCR_PREC = .TRUE.

            IF ( X_STATE .EQ. NOPROG_STATE .AND. DXRAT .LE. RTHRESH )
     $           X_STATE = WORKING_STATE
            IF ( X_STATE .EQ. WORKING_STATE ) THEN
               IF ( DX_X .LE. EPS ) THEN
                  X_STATE = CONV_STATE
```

```
            ELSE IF ( DXRAT .GT. RTHRESH ) THEN
               IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                  INCR_PREC = .TRUE.
               ELSE
                  X_STATE = NOPROG_STATE
               END IF
            ELSE
               IF ( DXRAT .GT. DXRATMAX ) DXRATMAX = DXRAT
            END IF
            IF ( X_STATE .GT. WORKING_STATE ) FINAL_DX_X = DX_X
         END IF

         IF ( Z_STATE .EQ. UNSTABLE_STATE .AND. DZ_Z .LE. DZ_UB )
     $        Z_STATE = WORKING_STATE
         IF ( Z_STATE .EQ. NOPROG_STATE .AND. DZRAT .LE. RTHRESH )
     $        Z_STATE = WORKING_STATE
         IF ( Z_STATE .EQ. WORKING_STATE ) THEN
            IF ( DZ_Z .LE. EPS ) THEN
               Z_STATE = CONV_STATE
            ELSE IF ( DZ_Z .GT. DZ_UB ) THEN
               Z_STATE = UNSTABLE_STATE
               DZRATMAX = 0.0
               FINAL_DZ_Z = HUGEVAL
            ELSE IF ( DZRAT .GT. RTHRESH ) THEN
               IF ( Y_PREC_STATE .NE. EXTRA_Y ) THEN
                  INCR_PREC = .TRUE.
               ELSE
                  Z_STATE = NOPROG_STATE
               END IF
            ELSE
               IF ( DZRAT .GT. DZRATMAX ) DZRATMAX = DZRAT
            END IF
            IF ( Z_STATE .GT. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
         END IF

         IF ( X_STATE.NE.WORKING_STATE.AND.
     $        ( IGNORE_CWISE.OR.Z_STATE.NE.WORKING_STATE ) )
     $        GOTO 666

         IF ( INCR_PREC ) THEN
            INCR_PREC = .FALSE.
            Y_PREC_STATE = Y_PREC_STATE + 1
            DO I = 1, N
               Y_TAIL( I ) = 0.0
            END DO
         END IF

         PREVNORMDX = NORMDX
         PREV_DZ_Z = DZ_Z
*
```

```
*          Update soluton.
*
           IF (Y_PREC_STATE .LT. EXTRA_Y) THEN
              CALL SAXPY( N, 1.0, DY, 1, Y(1,J), 1 )
           ELSE
              CALL SLA_WWADDW( N, Y( 1, J ), Y_TAIL, DY )
           END IF

        END DO
*       Target of "IF (Z_STOP .AND. X_STOP)".  Sun's f77 won't EXIT.
 666    CONTINUE
*
*     Set final_* when cnt hits ithresh.
*
        IF ( X_STATE .EQ. WORKING_STATE ) FINAL_DX_X = DX_X
        IF ( Z_STATE .EQ. WORKING_STATE ) FINAL_DZ_Z = DZ_Z
*
*     Compute error bounds.
*
        IF ( N_NORMS .GE. 1 ) THEN
           ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) =
     $           FINAL_DX_X / (1 - DXRATMAX)
        END IF
        IF ( N_NORMS .GE. 2 ) THEN
           ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) =
     $           FINAL_DZ_Z / (1 - DZRATMAX)
        END IF
*
*     Compute componentwise relative backward error from formula
*         max(i) ( abs(R(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
*     where abs(Z) is the componentwise absolute value of the matrix
*     or vector Z.
*
*       Compute residual RES = B_s - op(A_s) * Y,
*           op(A) = A, A**T, or A**H depending on TRANS (and type).
*
        CALL SCOPY( N, B( 1, J ), 1, RES, 1 )
        CALL SSYMV( UPLO, N, -1.0, A, LDA, Y(1,J), 1, 1.0, RES, 1 )

        DO I = 1, N
           AYB( I ) = ABS( B( I, J ) )
        END DO
*
*     Compute abs(op(A_s))*abs(Y) + abs(B_s).
*
        CALL SLA_SYAMV( UPLO2, N, 1.0,
     $        A, LDA, Y(1, J), 1, 1.0, AYB, 1 )

        CALL SLA_LIN_BERR( N, N, 1, RES, AYB, BERR_OUT( J ) )
*
```

```
*     End of loop for each RHS.
*
      END DO
*
      RETURN
      END
```

———-

71.2.4 lisp code

— **sla_porfsx_extended.lisp** —

```
(let* ((unstable_state 0)
       (working_state 1)
       (conv_state 2)
       (noprog_state 3)
       (base_residual 0)
       (extra_residual 1)
       (extra_y 2)
       (la_linrx_err_i 2))
  (declare (type (f2cl-lib:integer4 0 0) unstable_state)
           (type (f2cl-lib:integer4 1 1) working_state)
           (type (f2cl-lib:integer4 2 2) conv_state)
           (type (f2cl-lib:integer4 3 3) noprog_state)
           (type (f2cl-lib:integer4 0 0) base_residual)
           (type (f2cl-lib:integer4 1 1) extra_residual)
           (type (f2cl-lib:integer4 2 2) extra_y)
           (type (f2cl-lib:integer4 2 2) la_linrx_err_i)
           (ignorable unstable_state working_state conv_state noprog_state
            base_residual extra_residual extra_y la_linrx_err_i))
  (defun sla_porfsx_extended
         (prec_type uplo n nrhs a lda af ldaf colequ c b ldb$ y ldy berr_out
          n_norms err_bnds_norm err_bnds_comp res ayb dy y_tail rcond ithresh
          rthresh dz_ub ignore_cwise info)
    (declare (type (single-float) dz_ub rthresh rcond)
             (type f2cl-lib:logical ignore_cwise colequ)
             (type (array single-float (*)) y_tail dy ayb res err_bnds_comp
              err_bnds_norm berr_out y b c af a)
             (type (string *) uplo)
             (type (f2cl-lib:integer4) info ithresh n_norms ldy ldb$ ldaf lda
              nrhs n prec_type))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (af single-float af-%data% af-%offset%)
         (c single-float c-%data% c-%offset%)
         (b single-float b-%data% b-%offset%)
```

```
    (y single-float y-%data% y-%offset%)
    (berr_out single-float berr_out-%data% berr_out-%offset%)
    (err_bnds_norm single-float err_bnds_norm-%data%
     err_bnds_norm-%offset%)
    (err_bnds_comp single-float err_bnds_comp-%data%
     err_bnds_comp-%offset%)
    (res single-float res-%data% res-%offset%)
    (ayb single-float ayb-%data% ayb-%offset%)
    (dy single-float dy-%data% dy-%offset%)
    (y_tail single-float y_tail-%data% y_tail-%offset%))
  (prog ((y_prec_state 0) (incr_prec nil) (yk 0.0) (dyk 0.0) (ymin 0.0)
         (normy 0.0) (normx 0.0) (normdx 0.0) (dxrat 0.0) (dzrat 0.0)
         (prevnormdx 0.0) (prev_dz_z 0.0) (dxratmax 0.0) (dzratmax 0.0)
         (dx_x 0.0) (dz_z 0.0) (final_dx_x 0.0) (final_dz_z 0.0) (eps 0.0)
         (hugeval 0.0) (incr_thresh 0.0) (uplo2 0) (cnt 0) (i 0) (j 0)
         (x_state 0) (z_state 0))
    (declare (type f2cl-lib:logical incr_prec)
             (type (single-float) yk dyk ymin normy normx normdx dxrat
              dzrat prevnormdx prev_dz_z dxratmax dzratmax dx_x dz_z
              final_dx_x final_dz_z eps hugeval incr_thresh)
             (type (f2cl-lib:integer4) y_prec_state uplo2 cnt i j x_state
              z_state))
    (if (/= info 0)
        (go end_label))
    (setf eps (slamch "Epsilon"))
    (setf hugeval (slamch "Overflow"))
    (setf hugeval (* hugeval hugeval))
    (setf incr_thresh (* (f2cl-lib:freal n) eps))
    (cond
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame uplo "L")
        (declare (ignore var-1))
        (when var-0 (setf uplo var-0))
        ret-val)
      (setf uplo2 (ilauplo "L")))
     (t (setf uplo2 (ilauplo "U"))))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (setf y_prec_state extra_residual)
        (cond
         ((= y_prec_state extra_y)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                                   y_tail-%offset%)
                    0.0)
             label100001))))
        (setf dxrat 0.0)
```

```
(setf dxratmax 0.0)
(setf dzrat 0.0)
(setf dzratmax 0.0)
(setf final_dx_x hugeval)
(setf final_dz_z hugeval)
(setf prevnormdx hugeval)
(setf prev_dz_z hugeval)
(setf dz_z hugeval)
(setf dx_x hugeval)
(setf x_state working_state)
(setf z_state unstable_state)
(setf incr_prec f2cl-lib:%false%)
(f2cl-lib:fdo (cnt 1 (f2cl-lib:int-add cnt 1))
              ((> cnt ithresh) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy n
         (f2cl-lib:array-slice b-%data% single-float (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1 res 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (cond
     ((= y_prec_state base_residual)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9)
          (ssymv uplo n -1.0 a lda
           (f2cl-lib:array-slice y-%data% single-float (1 j)
                                 ((1 ldy) (1 *)) y-%offset%)
           1 1.0 res 1)
        (declare
         (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
        (when var-0 (setf uplo var-0))
        (when var-1 (setf n var-1))
        (when var-4 (setf lda var-4))))
     ((= y_prec_state extra_residual)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (blas_ssymv_x uplo2 n -1.0 a lda
           (f2cl-lib:array-slice y-%data% single-float (1 j)
                                 ((1 ldy) (1 *)) y-%offset%)
           1 1.0 res 1 prec_type)
        (declare
         (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
        (when var-0 (setf uplo2 var-0))
        (when var-1 (setf n var-1))
        (when var-4 (setf lda var-4))
        (when var-10 (setf prec_type var-10))))
```

```
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11)
      (blas_ssymv2_x uplo2 n -1.0 a lda
       (f2cl-lib:array-slice y-%data% single-float (1 j)
                             ((1 ldy) (1 *)) y-%offset%)
       y_tail 1 1.0 res 1 prec_type)
    (declare
     (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9 var-10))
    (when var-0 (setf uplo2 var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf lda var-4))
    (when var-11 (setf prec_type var-11)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (scopy n res 1 dy 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (spotrs uplo n 1 af ldaf dy n info)
  (declare (ignore var-2 var-3 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-4 (setf ldaf var-4))
  (when var-6 (setf n var-6))
  (when var-7 (setf info var-7)))
(setf normx 0.0)
(setf normy 0.0)
(setf normdx 0.0)
(setf dz_z 0.0)
(setf ymin hugeval)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf yk
            (abs
             (f2cl-lib:fref y-%data% (i j) ((1 ldy) (1 *))
                            y-%offset%)))
    (setf dyk
            (abs
             (f2cl-lib:fref dy-%data% (i) ((1 *))
                            dy-%offset%)))
    (cond ((/= yk 0.0) (setf dz_z (max dz_z (/ dyk yk))))
          ((/= dyk 0.0) (setf dz_z hugeval)))
    (setf ymin (min ymin yk))
    (setf normy (max normy yk))
    (cond
     (colequ
      (setf normx
```

```
                (max normx
                     (* yk
                        (f2cl-lib:fref c-%data% (i) ((1 *))
                                       c-%offset%))))
        (setf normdx
                (max normdx
                     (* dyk
                        (f2cl-lib:fref c-%data% (i) ((1 *))
                                       c-%offset%)))))
      (t (setf normx normy) (setf normdx (max normdx dyk))))
   label100003))
(cond ((/= normx 0.0) (setf dx_x (/ normdx normx)))
      ((= normdx 0.0) (setf dx_x 0.0)) (t (setf dx_x hugeval)))
(setf dxrat (/ normdx prevnormdx))
(setf dzrat (/ dz_z prev_dz_z))
(if (and (< (* ymin rcond) (* incr_thresh normy))
         (< y_prec_state extra_y))
    (setf incr_prec f2cl-lib:%true%))
(if (and (= x_state noprog_state) (<= dxrat rthresh))
    (setf x_state working_state))
(cond
 ((= x_state working_state)
  (cond ((<= dx_x eps) (setf x_state conv_state))
        ((> dxrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf x_state noprog_state))))
        (t
         (if (> dxrat dxratmax)
             (setf dxratmax dxrat))))
  (if (> x_state working_state)
      (setf final_dx_x dx_x))))
(if (and (= z_state unstable_state) (<= dz_z dz_ub))
    (setf z_state working_state))
(if (and (= z_state noprog_state) (<= dzrat rthresh))
    (setf z_state working_state))
(cond
 ((= z_state working_state)
  (cond ((<= dz_z eps) (setf z_state conv_state))
        ((> dz_z dz_ub) (setf z_state unstable_state)
         (setf dzratmax 0.0) (setf final_dz_z hugeval))
        ((> dzrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf z_state noprog_state))))
        (t
         (if (> dzrat dzratmax)
             (setf dzratmax dzrat))))
```

```
          (if (> z_state working_state)
              (setf final_dz_z dz_z))))
        (if (and (/= x_state working_state)
                 (or ignore_cwise (/= z_state working_state)))
            (go label666))
        (cond
         (incr_prec (setf incr_prec f2cl-lib:%false%)
          (setf y_prec_state (f2cl-lib:int-add y_prec_state 1))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                                   y_tail-%offset%)
                    0.0)
             label100004))))
        (setf prevnormdx normdx)
        (setf prev_dz_z dz_z)
        (cond
         ((< y_prec_state extra_y)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
              (saxpy n 1.0 dy 1
               (f2cl-lib:array-slice y-%data% single-float (1 j)
                                     ((1 ldy) (1 *)) y-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3 var-4 var-5))
            (when var-0 (setf n var-0))))
         (t
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (sla_wwaddw n
               (f2cl-lib:array-slice y-%data% single-float (1 j)
                                     ((1 ldy) (1 *)) y-%offset%)
               y_tail dy)
            (declare (ignore var-1 var-2 var-3))
            (when var-0 (setf n var-0)))))
      label100002))
  label666
   (if (= x_state working_state)
       (setf final_dx_x dx_x))
   (if (= z_state working_state)
       (setf final_dz_z dz_z))
   (cond
    ((>= n_norms 1)
     (setf (f2cl-lib:fref err_bnds_norm-%data% (j la_linrx_err_i)
                          ((1 nrhs) (1 *)) err_bnds_norm-%offset%)
           (/ final_dx_x (- 1 dxratmax)))))
   (cond
    ((>= n_norms 2)
     (setf (f2cl-lib:fref err_bnds_comp-%data% (j la_linrx_err_i)
                          ((1 nrhs) (1 *)) err_bnds_comp-%offset%)
           (/ final_dz_z (- 1 dzratmax)))))
```

```
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (scopy n
                 (f2cl-lib:array-slice b-%data% single-float (1 j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 1 res 1)
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf n var-0)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                (ssymv uplo n -1.0 a lda
                 (f2cl-lib:array-slice y-%data% single-float (1 j)
                                       ((1 ldy) (1 *)) y-%offset%)
                 1 1.0 res 1)
              (declare (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
              (when var-0 (setf uplo var-0))
              (when var-1 (setf n var-1))
              (when var-4 (setf lda var-4)))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref ayb-%data% (i) ((1 *)) ayb-%offset%)
                        (abs
                         (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                        b-%offset%)))
               label100005))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                (sla_syamv uplo2 n 1.0 a lda
                 (f2cl-lib:array-slice y-%data% single-float (1 j)
                                       ((1 ldy) (1 *)) y-%offset%)
                 1 1.0 ayb 1)
              (declare (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
              (when var-0 (setf uplo2 var-0))
              (when var-1 (setf n var-1))
              (when var-4 (setf lda var-4)))
            (sla_lin_berr n n 1 res ayb
             (f2cl-lib:array-slice berr_out-%data% single-float (j) ((1 *))
                                   berr_out-%offset%))
          label100000))
        (go end_label)
      end_label
        (return
         (values prec_type uplo n nil nil lda nil ldaf nil nil nil nil nil nil
                 nil nil nil nil nil nil nil nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sla_porfsx_extended
                 fortran-to-lisp::*f2cl-function-info*)
```

```
(fortran-to-lisp::make-f2cl-finfo :arg-types
                                  '((fortran-to-lisp::integer4)
                                    (string)
                                    (fortran-to-lisp::integer4)
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    fortran-to-lisp::logical
                                    (array single-float (*))
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (fortran-to-lisp::integer4)
                                    (array single-float (*))
                                    (array single-float (*))
                                    (array single-float (*))
                                    (array single-float (*))
                                    (array single-float (*))
                                    (array single-float (*))
                                    (single-float)
                                    (fortran-to-lisp::integer4)
                                    (single-float) (single-float)
                                    fortran-to-lisp::logical
                                    (fortran-to-lisp::integer4))
                                  :return-values
                                  '(fortran-to-lisp::prec_type
                                    fortran-to-lisp::uplo
                                    fortran-to-lisp::n nil nil
                                    fortran-to-lisp::lda nil
                                    fortran-to-lisp::ldaf nil nil nil
                                    nil nil nil nil nil nil nil nil
                                    nil nil nil nil nil nil nil nil
                                    fortran-to-lisp::info)
                                  :calls
                                  '(fortran-to-lisp::sla_lin_berr
                                    fortran-to-lisp::ilauplo))))
```

71.3 sla_porpvgrw

71.3.1 Axiom unit tests

— **slaporpvgrw.input** —

```
)set break resume
)sys rm -f slaporpvgrw.output
)spool slaporpvgrw.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

71.3.2 Axiom help page

— **sla_porpvgrw.help** —

```
====================================================================
sla_porpvgrw examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       real function sla_porpvgrw (UPLO, NCOLS, A, LDA, AF, LDAF, WORK)
           SLA_PORPVGRW computes the reciprocal pivot growth factor
           norm(A)/norm(U) for a symmetric or Hermitian positive-definite
           matrix.

Function/Subroutine Documentation
   real function sla_porpvgrw (character*1 UPLO, integer NCOLS, real,
       dimension( lda, * ) A, integer LDA, real, dimension( ldaf, * ) AF,
       integer LDAF, real, dimension( * ) WORK)
       SLA_PORPVGRW computes the reciprocal pivot growth factor
       norm(A)/norm(U) for a symmetric or Hermitian positive-definite matrix.

Purpose:
   SLA_PORPVGRW computes the reciprocal pivot growth factor
```

```
   norm(A)/norm(U). The "max absolute element" norm is used. If this is
   much less than 1, the stability of the LU factorization of the
   (equilibrated) matrix A could be poor. This also means that the
   solution X, estimated condition numbers, and error bounds could be
   unreliable.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 NCOLS
   NCOLS is INTEGER
   The number of columns of the matrix A. NCOLS >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is REAL array, dimension (LDAF,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**T*U or A = L*L**T, as computed by SPOTRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 WORK
   WORK is REAL array, dimension (2*N)
```

———-

71.3.3 fortran code

— sla_porpvgrw.f —

```
*  =====================================================================
      REAL FUNCTION SLA_PORPVGRW( UPLO, NCOLS, A, LDA, AF, LDAF, WORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
```

```
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER*1        UPLO
      INTEGER            NCOLS, LDA, LDAF
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), AF( LDAF, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               AMAX, UMAX, RPVGRW
      LOGICAL            UPPER
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, SLASET
      LOGICAL            LSAME
*     ..
*     .. Executable Statements ..
*
      UPPER = LSAME( 'Upper', UPLO )
*
*     SPOTRF will have factored only the NCOLSxNCOLS leading minor, so
*     we restrict the growth search to that minor and use only the first
*     2*NCOLS workspace entries.
*
      RPVGRW = 1.0
      DO I = 1, 2*NCOLS
         WORK( I ) = 0.0
      END DO
*
*     Find the max magnitude entry of each column.
*
      IF ( UPPER ) THEN
         DO J = 1, NCOLS
            DO I = 1, J
               WORK( NCOLS+J ) =
     $              MAX( ABS( A( I, J ) ), WORK( NCOLS+J ) )
            END DO
         END DO
      ELSE
         DO J = 1, NCOLS
```

```
            DO I = J, NCOLS
               WORK( NCOLS+J ) =
     $              MAX( ABS( A( I, J ) ), WORK( NCOLS+J ) )
            END DO
         END DO
      END IF
*
*     Now find the max magnitude entry of each column of the factor in
*     AF.  No pivoting, so no permutations.
*
      IF ( LSAME( 'Upper', UPLO ) ) THEN
         DO J = 1, NCOLS
            DO I = 1, J
               WORK( J ) = MAX( ABS( AF( I, J ) ), WORK( J ) )
            END DO
         END DO
      ELSE
         DO J = 1, NCOLS
            DO I = J, NCOLS
               WORK( J ) = MAX( ABS( AF( I, J ) ), WORK( J ) )
            END DO
         END DO
      END IF
*
*     Compute the *inverse* of the max element growth factor.  Dividing
*     by zero would imply the largest entry of the factor's column is
*     zero.  Than can happen when either the column of A is zero or
*     massive pivots made the factor underflow to zero.  Neither counts
*     as growth in itself, so simply ignore terms with zero
*     denominators.
*
      IF ( LSAME( 'Upper', UPLO ) ) THEN
         DO I = 1, NCOLS
            UMAX = WORK( I )
            AMAX = WORK( NCOLS+I )
            IF ( UMAX /= 0.0 ) THEN
               RPVGRW = MIN( AMAX / UMAX, RPVGRW )
            END IF
         END DO
      ELSE
         DO I = 1, NCOLS
            UMAX = WORK( I )
            AMAX = WORK( NCOLS+I )
            IF ( UMAX /= 0.0 ) THEN
               RPVGRW = MIN( AMAX / UMAX, RPVGRW )
            END IF
         END DO
      END IF

      SLA_PORPVGRW = RPVGRW
```

```
      END
```

———-

71.3.4 lisp code

— **sla_porpvgrw.lisp** —

```
(defun sla_porpvgrw (uplo ncols a lda af ldaf work)
  (declare (type (array single-float (*)) work af a)
           (type (f2cl-lib:integer4) ldaf lda ncols)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a single-float a-%data% a-%offset%)
       (af single-float af-%data% af-%offset%)
       (work single-float work-%data% work-%offset%))
    (prog ((upper nil) (amax 0.0) (umax 0.0) (rpvgrw 0.0) (i 0) (j 0)
           (sla_porpvgrw 0.0))
      (declare (type (f2cl-lib:integer4) j i)
               (type (single-float) sla_porpvgrw rpvgrw umax amax)
               (type f2cl-lib:logical upper))
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame "Upper" uplo)
                (declare (ignore var-0))
                (when var-1 (setf uplo var-1))
                ret-val))
      (setf rpvgrw 1.0)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i (f2cl-lib:int-mul 2 ncols)) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) 0.0)
         label100000))
      (cond
       (upper
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j ncols) nil)
          (tagbody
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i j) nil)
              (tagbody
                (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ncols j))
                                     ((1 *)) work-%offset%)
                      (max
                       (abs
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))
```

```
                    (f2cl-lib:fref work-%data%
                                   ((f2cl-lib:int-add ncols j)) ((1 *))
                                   work-%offset%)))
            label100002))
        label100001)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j ncols) nil)
       (tagbody
         (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                       ((> i ncols) nil)
           (tagbody
             (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ncols j))
                                  ((1 *)) work-%offset%)
                     (max
                      (abs
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%))
                      (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-add ncols j)) ((1 *))
                                     work-%offset%)))
            label100004))
        label100003))))
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame "Upper" uplo)
      (declare (ignore var-0))
      (when var-1 (setf uplo var-1))
      ret-val)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j ncols) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i j) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                    (max
                     (abs
                      (f2cl-lib:fref af-%data% (i j) ((1 ldaf) (1 *))
                                     af-%offset%))
                     (f2cl-lib:fref work-%data% (j) ((1 *))
                                    work-%offset%)))
            label100006))
        label100005)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j ncols) nil)
       (tagbody
         (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                       ((> i ncols) nil)
```

```
              (tagbody
                (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                        (max
                         (abs
                          (f2cl-lib:fref af-%data% (i j) ((1 ldaf) (1 *))
                                         af-%offset%))
                         (f2cl-lib:fref work-%data% (j) ((1 *))
                                        work-%offset%)))
               label100008))
           label100007))))
      (cond
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame "Upper" uplo)
          (declare (ignore var-0))
          (when var-1 (setf uplo var-1))
          ret-val)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i ncols) nil)
          (tagbody
            (setf umax (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
            (setf amax
                    (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ncols i))
                                   ((1 *)) work-%offset%))
            (cond
             ((f2cl-lib:f2cl/ umax (= 0.0))
              (setf rpvgrw (min (/ amax umax) rpvgrw))))
           label100009)))
       (t
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i ncols) nil)
          (tagbody
            (setf umax (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
            (setf amax
                    (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ncols i))
                                   ((1 *)) work-%offset%))
            (cond
             ((f2cl-lib:f2cl/ umax (= 0.0))
              (setf rpvgrw (min (/ amax umax) rpvgrw))))
           label100010))))
      (setf sla_porpvgrw rpvgrw)
     end_label
      (return (values sla_porpvgrw uplo nil nil nil nil nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sla_porpvgrw
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
```

```
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*)))
:return-values
'(fortran-to-lisp::uplo nil nil nil
  nil nil nil)
:calls 'nil)))
```

———-

71.4 spocon

71.4.1 Axiom unit tests

— **spocon.input** —

```
)set break resume
)sys rm -f spocon.output
)spool spocon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

71.4.2 Axiom help page

— **spocon.help** —

```
==========================================================================
spocon examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine spocon (UPLO, N, A, LDA, ANORM, RCOND, WORK, IWORK, INFO)
           SPOCON

Function/Subroutine Documentation
   subroutine spocon (character UPLO, integer N, real, dimension( lda, * ) A,
       integer LDA, real ANORM, real RCOND, real, dimension( * ) WORK,
       integer, dimension( * ) IWORK, integer INFO)
       SPOCON

Purpose:
   SPOCON estimates the reciprocal of the condition number (in the
   1-norm) of a real symmetric positive definite matrix using the
   Cholesky factorization A = U**T*U or A = L*L**T computed by SPOTRF.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**T*U or A = L*L**T, as computed by SPOTRF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 ANORM
   ANORM is REAL
   The 1-norm (or infinity-norm) of the symmetric matrix A.

 RCOND
   RCOND is REAL
   The reciprocal of the condition number of the matrix A,
   computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is an
   estimate of the 1-norm of inv(A) computed in this routine.
```

```
 WORK
   WORK is REAL array, dimension (3*N)

 IWORK
   IWORK is INTEGER array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

71.4.3 fortran code

— **spocon.f** —

```
*  =====================================================================
      SUBROUTINE SPOCON( UPLO, N, A, LDA, ANORM, RCOND, WORK, IWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
      REAL               ANORM, RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      REAL               A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      CHARACTER          NORMIN
      INTEGER            IX, KASE
      REAL               AINVNM, SCALE, SCALEL, SCALEU, SMLNUM
```

```
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ISAMAX
      REAL               SLAMCH
      EXTERNAL           LSAME, ISAMAX, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLACN2, SLATRS, SRSCL, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -5
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SPOCON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
      SMLNUM = SLAMCH( 'Safe minimum' )
*
*     Estimate the 1-norm of inv(A).
*
```

```
      KASE = 0
      NORMIN = 'N'
   10 CONTINUE
      CALL SLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( UPPER ) THEN
*
*           Multiply by inv(U**T).
*
            CALL SLATRS( 'Upper', 'Transpose', 'Non-unit', NORMIN, N, A,
     $                   LDA, WORK, SCALEL, WORK( 2*N+1 ), INFO )
            NORMIN = 'Y'
*
*           Multiply by inv(U).
*
            CALL SLATRS( 'Upper', 'No transpose', 'Non-unit', NORMIN, N,
     $                   A, LDA, WORK, SCALEU, WORK( 2*N+1 ), INFO )
         ELSE
*
*           Multiply by inv(L).
*
            CALL SLATRS( 'Lower', 'No transpose', 'Non-unit', NORMIN, N,
     $                   A, LDA, WORK, SCALEL, WORK( 2*N+1 ), INFO )
            NORMIN = 'Y'
*
*           Multiply by inv(L**T).
*
            CALL SLATRS( 'Lower', 'Transpose', 'Non-unit', NORMIN, N, A,
     $                   LDA, WORK, SCALEU, WORK( 2*N+1 ), INFO )
         END IF
*
*        Multiply by 1/SCALE if doing so will not cause overflow.
*
         SCALE = SCALEL*SCALEU
         IF( SCALE.NE.ONE ) THEN
            IX = ISAMAX( N, WORK, 1 )
            IF( SCALE.LT.ABS( WORK( IX ) )*SMLNUM .OR. SCALE.EQ.ZERO )
     $         GO TO 20
            CALL SRSCL( N, SCALE, WORK, 1 )
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
   20 CONTINUE
      RETURN
```

```
*
*     End of SPOCON
*
      END
```

———-

71.4.4 lisp code

— **spocon.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun spocon (uplo n a lda anorm rcond work iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (single-float) rcond anorm)
             (type (array single-float (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (work single-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (ainvnm 0.0) (scale 0.0) (scalel 0.0) (scaleu 0.0) (smlnum 0.0)
             (ix 0) (kase 0)
             (normin
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (upper nil))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (single-float) ainvnm scale scalel scaleu smlnum)
                 (type (f2cl-lib:integer4) ix kase)
                 (type (string 1) normin)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
```

```
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4))
   ((< anorm zero) (setf info -5)))
  (cond
   ((/= info 0) (xerbla "SPOCON" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf rcond zero)
  (cond ((= n 0) (setf rcond one) (go end_label))
        ((= anorm zero) (go end_label)))
  (setf smlnum (slamch "Safe minimum"))
  (setf kase 0)
  (f2cl-lib:f2cl-set-string normin "N" (string 1))
 label10
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacn2 n
       (f2cl-lib:array-slice work-%data% single-float ((+ n 1)) ((1 *))
                             work-%offset%)
       work iwork ainvnm kase isave)
    (declare (ignore var-1 var-2 var-3 var-6))
    (setf n var-0)
    (setf ainvnm var-4)
    (setf kase var-5))
  (cond
   ((/= kase 0)
    (cond
     (upper
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (slatrs "Upper" "Transpose" "Non-unit" normin n a lda work
           scalel
           (f2cl-lib:array-slice work-%data% single-float
                                 ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                                 work-%offset%)
           info)
        (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
        (setf normin var-3)
        (setf n var-4)
        (setf lda var-6)
        (setf scalel var-8)
        (setf info var-10))
      (f2cl-lib:f2cl-set-string normin "Y" (string 1))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
        var-10)
       (slatrs "Upper" "No transpose" "Non-unit" normin n a lda work
        scaleu
        (f2cl-lib:array-slice work-%data% single-float
                              ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                              work-%offset%)
        info)
     (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
     (setf normin var-3)
     (setf n var-4)
     (setf lda var-6)
     (setf scaleu var-8)
     (setf info var-10)))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (slatrs "Lower" "No transpose" "Non-unit" normin n a lda work
        scalel
        (f2cl-lib:array-slice work-%data% single-float
                              ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                              work-%offset%)
        info)
     (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
     (setf normin var-3)
     (setf n var-4)
     (setf lda var-6)
     (setf scalel var-8)
     (setf info var-10))
   (f2cl-lib:f2cl-set-string normin "Y" (string 1))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (slatrs "Lower" "Transpose" "Non-unit" normin n a lda work
        scaleu
        (f2cl-lib:array-slice work-%data% single-float
                              ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                              work-%offset%)
        info)
     (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
     (setf normin var-3)
     (setf n var-4)
     (setf lda var-6)
     (setf scaleu var-8)
     (setf info var-10))))
 (setf scale (* scalel scaleu))
 (cond
  ((/= scale one)
   (setf ix
         (multiple-value-bind (ret-val var-0 var-1 var-2)
```

```
                        (isamax n work 1)
                      (declare (ignore var-1 var-2))
                      (when var-0 (setf n var-0))
                      ret-val))
            (if (or
                 (< scale
                    (*
                     (abs
                      (f2cl-lib:fref work-%data% (ix) ((1 *)) work-%offset%))
                     smlnum))
                 (= scale zero))
                (go label20))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (srscl n scale work 1)
              (declare (ignore var-2 var-3))
              (when var-0 (setf n var-0))
              (when var-1 (setf scale var-1)))))
          (go label10)))
        (if (/= ainvnm zero)
            (setf rcond (/ (/ one ainvnm) anorm)))
       label20
        (go end_label)
       end_label
        (return (values uplo n nil lda nil rcond nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::spocon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (array single-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::rcond nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::slatrs
                                              fortran-to-lisp::slacn2))))
```

———-

71.5 spoequ

71.5.1 Axiom unit tests

— **spoequ.input** —

```
)set break resume
)sys rm -f spoequ.output
)spool spoequ.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

71.5.2 Axiom help page

— **spoequ.help** —

```
======================================================================
spoequ examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine spoequ (N, A, LDA, S, SCOND, AMAX, INFO)
           SPOEQU

Function/Subroutine Documentation
   subroutine spoequ (integer N, real, dimension( lda, * ) A, integer LDA,
       real, dimension( * ) S, real SCOND, real AMAX, integer INFO)
       SPOEQU
```

```
Purpose:
   SPOEQU computes row and column scalings intended to equilibrate a
   symmetric positive definite matrix A and reduce its condition number
   (with respect to the two-norm).  S contains the scale factors,
   S(i) = 1/sqrt(A(i,i)), chosen so that the scaled matrix B with
   elements B(i,j) = S(i)*A(i,j)*S(j) has ones on the diagonal.  This
   choice of S puts the condition number of B within a factor N of the
   smallest possible condition number over all possible diagonal scalings.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   The N-by-N symmetric positive definite matrix whose scaling
   factors are to be computed.  Only the diagonal elements of A
   are referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 S
   S is REAL array, dimension (N)
   If INFO = 0, S contains the scale factors for A.

 SCOND
   SCOND is REAL
   If INFO = 0, S contains the ratio of the smallest S(i) to
   the largest S(i).  If SCOND >= 0.1 and AMAX is neither too
   large nor too small, it is not worth scaling by S.

 AMAX
   AMAX is REAL
   Absolute value of largest matrix element.  If AMAX is very
   close to overflow or very close to underflow, the matrix
   should be scaled.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, the i-th diagonal element is nonpositive.
```

71.5.3 fortran code

— spoequ.f —

```
*  =====================================================================
      SUBROUTINE SPOEQU( N, A, LDA, S, SCOND, AMAX, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, N
      REAL               AMAX, SCOND
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), S( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      REAL               SMIN
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -3
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SPOEQU', -INFO )
         RETURN
```

```
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SCOND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Find the minimum and maximum diagonal elements.
*
      S( 1 ) = A( 1, 1 )
      SMIN = S( 1 )
      AMAX = S( 1 )
      DO 10 I = 2, N
         S( I ) = A( I, I )
         SMIN = MIN( SMIN, S( I ) )
         AMAX = MAX( AMAX, S( I ) )
   10 CONTINUE
*
      IF( SMIN.LE.ZERO ) THEN
*
*        Find the first non-positive diagonal element and return.
*
         DO 20 I = 1, N
            IF( S( I ).LE.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   20    CONTINUE
      ELSE
*
*        Set the scale factors to the reciprocals
*        of the diagonal elements.
*
         DO 30 I = 1, N
            S( I ) = ONE / SQRT( S( I ) )
   30    CONTINUE
*
*        Compute SCOND = min(S(I)) / max(S(I))
*
         SCOND = SQRT( SMIN ) / SQRT( AMAX )
      END IF
      RETURN
*
*     End of SPOEQU
*
      END
```

———-

71.5.4 lisp code

— **spoequ.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun spoequ (n a lda s scond amax info)
    (declare (type (single-float) amax scond)
             (type (array single-float (*)) s a)
             (type (f2cl-lib:integer4) info lda n))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (s single-float s-%data% s-%offset%))
      (prog ((smin 0.0) (i 0))
        (declare (type (single-float) smin)
                 (type (f2cl-lib:integer4) i))
        (setf info 0)
        (cond ((< n 0) (setf info -1))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -3)))
        (cond
         ((/= info 0) (xerbla "SPOEQU" (f2cl-lib:int-sub info))
          (go end_label)))
        (cond ((= n 0) (setf scond one) (setf amax zero) (go end_label)))
        (setf (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%)
                (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%))
        (setf smin (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
        (setf amax (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
        (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                    (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%))
            (setf smin
                    (min smin (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
            (setf amax
                    (max amax (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
           label10))
        (cond
         ((<= smin zero)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
```

```
              (cond
               ((<= (f2cl-lib:fref s (i) ((1 *))) zero) (setf info i)
                (go end_label)))
             label20)))
          (t
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                       (/ one
                          (f2cl-lib:fsqrt
                           (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%))))
             label30))
           (setf scond (/ (f2cl-lib:fsqrt smin) (f2cl-lib:fsqrt amax)))))
         (go end_label)
        end_label
         (return (values nil nil nil nil scond amax info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::spoequ
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil
                                              fortran-to-lisp::scond
                                              fortran-to-lisp::amax
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

71.6 spoequb

71.6.1 Axiom unit tests

— spoequb.input —

```
)set break resume
)sys rm -f spoequb.output
)spool spoequb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

71.6.2 Axiom help page

— **spoequb.help** —

```
==================================================================
spoequb examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine spoequb (N, A, LDA, S, SCOND, AMAX, INFO)
           SPOEQUB

Function/Subroutine Documentation
   subroutine spoequb (integer N, real, dimension( lda, * ) A, integer LDA,
       real, dimension( * ) S, real SCOND, real AMAX, integer INFO)
       SPOEQUB

Purpose:
   SPOEQU computes row and column scalings intended to equilibrate a
   symmetric positive definite matrix A and reduce its condition number
   (with respect to the two-norm).  S contains the scale factors,
   S(i) = 1/sqrt(A(i,i)), chosen so that the scaled matrix B with
   elements B(i,j) = S(i)*A(i,j)*S(j) has ones on the diagonal.  This
   choice of S puts the condition number of B within a factor N of the
   smallest possible condition number over all possible diagonal scalings.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.
```

```
A
  A is REAL array, dimension (LDA,N)
  The N-by-N symmetric positive definite matrix whose scaling
  factors are to be computed.  Only the diagonal elements of A
  are referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

S
  S is REAL array, dimension (N)
  If INFO = 0, S contains the scale factors for A.

SCOND
  SCOND is REAL
  If INFO = 0, S contains the ratio of the smallest S(i) to
  the largest S(i).  If SCOND >= 0.1 and AMAX is neither too
  large nor too small, it is not worth scaling by S.

AMAX
  AMAX is REAL
  Absolute value of largest matrix element.  If AMAX is very
  close to overflow or very close to underflow, the matrix
  should be scaled.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, the i-th diagonal element is nonpositive.
```

71.6.3 fortran code

— spoequb.f —

```
*  =====================================================================
      SUBROUTINE SPOEQUB( N, A, LDA, S, SCOND, AMAX, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
```

```
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, N
      REAL               AMAX, SCOND
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), S( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      REAL               SMIN, BASE, TMP
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT, LOG, INT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
*     Positive definite only performs 1 pass of equilibration.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -3
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SPOEQUB', -INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 ) THEN
         SCOND = ONE
         AMAX = ZERO
```

```
         RETURN
      END IF

      BASE = SLAMCH( 'B' )
      TMP = -0.5 / LOG ( BASE )
*
*     Find the minimum and maximum diagonal elements.
*
      S( 1 ) = A( 1, 1 )
      SMIN = S( 1 )
      AMAX = S( 1 )
      DO 10 I = 2, N
         S( I ) = A( I, I )
         SMIN = MIN( SMIN, S( I ) )
         AMAX = MAX( AMAX, S( I ) )
   10 CONTINUE
*
      IF( SMIN.LE.ZERO ) THEN
*
*        Find the first non-positive diagonal element and return.
*
         DO 20 I = 1, N
            IF( S( I ).LE.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   20    CONTINUE
      ELSE
*
*        Set the scale factors to the reciprocals
*        of the diagonal elements.
*
         DO 30 I = 1, N
            S( I ) = BASE ** INT( TMP * LOG( S( I ) ) )
   30    CONTINUE
*
*        Compute SCOND = min(S(I)) / max(S(I)).
*
         SCOND = SQRT( SMIN ) / SQRT( AMAX )
      END IF
*
      RETURN
*
*     End of SPOEQUB
*
      END
```

———-

71.6.4 lisp code

— spoequb.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun spoequb (n a lda s scond amax info)
    (declare (type (single-float) amax scond)
             (type (array single-float (*)) s a)
             (type (f2cl-lib:integer4) info lda n))
    (f2cl-lib:with-multi-array-data
        ((a single-float a-%data% a-%offset%)
         (s single-float s-%data% s-%offset%))
      (prog ((smin 0.0) (base 0.0) (tmp 0.0) (i 0))
        (declare (type (single-float) smin base tmp)
                 (type (f2cl-lib:integer4) i))
        (setf info 0)
        (cond ((< n 0) (setf info -1))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -3)))
        (cond
         ((/= info 0) (xerbla "SPOEQUB" (f2cl-lib:int-sub info))
          (go end_label)))
        (cond ((= n 0) (setf scond one) (setf amax zero) (go end_label)))
        (setf base (slamch "B"))
        (setf tmp (/ -0.5 (f2cl-lib:flog base)))
        (setf (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%)
                (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%))
        (setf smin (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
        (setf amax (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
        (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                    (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%))
            (setf smin
                    (min smin (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
            (setf amax
                    (max amax (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
           label10))
        (cond
         ((<= smin zero)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (cond
```

```
              ((<= (f2cl-lib:fref s (i) ((1 *))) zero) (setf info i)
               (go end_label)))
            label20)))
         (t
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                      (expt base
                            (f2cl-lib:int
                             (* tmp
                                (f2cl-lib:flog
                                 (f2cl-lib:fref s-%data% (i) ((1 *))
                                                s-%offset%))))))
            label30))
          (setf scond (/ (f2cl-lib:fsqrt smin) (f2cl-lib:fsqrt amax)))))
        (go end_label)
       end_label
        (return (values nil nil nil nil scond amax info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::spoequb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil
                                              fortran-to-lisp::scond
                                              fortran-to-lisp::amax
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

71.7 sporfs

71.7.1 Axiom unit tests

— sporfs.input —

```
)set break resume
)sys rm -f sporfs.output
)spool sporfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

71.7.2 Axiom help page

— sporfs.help —

```
==================================================================
sporfs examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sporfs (UPLO, N, NRHS, A, LDA, AF, LDAF, B, LDB, X, LDX,
           FERR, BERR, WORK, IWORK, INFO)
           SPORFS

Function/Subroutine Documentation
   subroutine sporfs (character UPLO, integer N, integer NRHS, real,
       dimension( lda, * ) A, integer LDA, real, dimension( ldaf, * ) AF,
       integer LDAF, real, dimension( ldb, * ) B, integer LDB, real,
       dimension( ldx, * ) X, integer LDX, real, dimension( * ) FERR, real,
       dimension( * ) BERR, real, dimension( * ) WORK, integer, dimension( * )
       IWORK, integer INFO)
       SPORFS

Purpose:
```

```
  SPORFS improves the computed solution to a system of linear
  equations when the coefficient matrix is symmetric positive definite,
  and provides error bounds and backward error estimates for the solution.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 A
   A is REAL array, dimension (LDA,N)
   The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
   upper triangular part of A contains the upper triangular part
   of the matrix A, and the strictly lower triangular part of A
   is not referenced.  If UPLO = 'L', the leading N-by-N lower
   triangular part of A contains the lower triangular part of
   the matrix A, and the strictly upper triangular part of A is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is REAL array, dimension (LDAF,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**T*U or A = L*L**T, as computed by SPOTRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 B
   B is REAL array, dimension (LDB,NRHS)
   The right hand side matrix B.

 LDB
   LDB is INTEGER
   The leading dimension of the array B.  LDB >= max(1,N).
```

```
 X
   X is REAL array, dimension (LDX,NRHS)
   On entry, the solution matrix X, as computed by SPOTRS.
   On exit, the improved solution matrix X.

 LDX
   LDX is INTEGER
   The leading dimension of the array X.  LDX >= max(1,N).

 FERR
   FERR is REAL array, dimension (NRHS)
   The estimated forward error bound for each solution vector
   X(j) (the j-th column of the solution matrix X).
   If XTRUE is the true solution corresponding to X(j), FERR(j)
   is an estimated upper bound for the magnitude of the largest
   element in (X(j) - XTRUE) divided by the magnitude of the
   largest element in X(j).  The estimate is as reliable as
   the estimate for RCOND, and is almost always a slight
   overestimate of the true error.

 BERR
   BERR is REAL array, dimension (NRHS)
   The componentwise relative backward error of each solution
   vector X(j) (i.e., the smallest relative change in
   any element of A or B that makes X(j) an exact solution).

 WORK
   WORK is REAL array, dimension (3*N)

 IWORK
   IWORK is INTEGER array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

   ITMAX is the maximum number of steps of iterative refinement.
```

———-

71.7.3 fortran code

— sporfs.f —

```
*  =====================================================================
      SUBROUTINE SPORFS( UPLO, N, NRHS, A, LDA, AF, LDAF, B, LDB, X,
     $                   LDX, FERR, BERR, WORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      REAL               A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   BERR( * ), FERR( * ), WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
      REAL               TWO
      PARAMETER          ( TWO = 2.0E+0 )
      REAL               THREE
      PARAMETER          ( THREE = 3.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            COUNT, I, J, K, KASE, NZ
      REAL               EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN, XK
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SAXPY, SCOPY, SLACN2, SPOTRS, SSYMV, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH
```

```
      EXTERNAL           LSAME, SLAMCH
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -11
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SPORFS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = N + 1
      EPS = SLAMCH( 'Epsilon' )
      SAFMIN = SLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 140 J = 1, NRHS
*
```

```
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - A * X
*
         CALL SCOPY( N, B( 1, J ), 1, WORK( N+1 ), 1 )
         CALL SSYMV( UPLO, N, -ONE, A, LDA, X( 1, J ), 1, ONE,
     $               WORK( N+1 ), 1 )
*
*        Compute componentwise relative backward error from formula
*
*        max(i) ( abs(R(i)) / ( abs(A)*abs(X) + abs(B) )(i) )
*
*        where abs(Z) is the componentwise absolute value of the matrix
*        or vector Z.  If the i-th component of the denominator is less
*        than SAFE2, then SAFE1 is added to the i-th components of the
*        numerator and denominator before dividing.
*
         DO 30 I = 1, N
            WORK( I ) = ABS( B( I, J ) )
   30    CONTINUE
*
*        Compute abs(A)*abs(X) + abs(B).
*
         IF( UPPER ) THEN
            DO 50 K = 1, N
               S = ZERO
               XK = ABS( X( K, J ) )
               DO 40 I = 1, K - 1
                  WORK( I ) = WORK( I ) + ABS( A( I, K ) )*XK
                  S = S + ABS( A( I, K ) )*ABS( X( I, J ) )
   40          CONTINUE
               WORK( K ) = WORK( K ) + ABS( A( K, K ) )*XK + S
   50       CONTINUE
         ELSE
            DO 70 K = 1, N
               S = ZERO
               XK = ABS( X( K, J ) )
               WORK( K ) = WORK( K ) + ABS( A( K, K ) )*XK
               DO 60 I = K + 1, N
                  WORK( I ) = WORK( I ) + ABS( A( I, K ) )*XK
                  S = S + ABS( A( I, K ) )*ABS( X( I, J ) )
   60          CONTINUE
               WORK( K ) = WORK( K ) + S
   70       CONTINUE
         END IF
         S = ZERO
```

```
         DO 80 I = 1, N
            IF( WORK( I ).GT.SAFE2 ) THEN
               S = MAX( S, ABS( WORK( N+I ) ) / WORK( I ) )
            ELSE
               S = MAX( S, ( ABS( WORK( N+I ) )+SAFE1 ) /
     $             ( WORK( I )+SAFE1 ) )
            END IF
   80    CONTINUE
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
*           1) The residual BERR(J) is larger than machine epsilon, and
*           2) BERR(J) decreased by at least a factor of 2 during the
*              last iteration, and
*           3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
*
*           Update solution and try again.
*
            CALL SPOTRS( UPLO, N, 1, AF, LDAF, WORK( N+1 ), N, INFO )
            CALL SAXPY( N, ONE, WORK( N+1 ), 1, X( 1, J ), 1 )
            LSTRES = BERR( J )
            COUNT = COUNT + 1
            GO TO 20
         END IF
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(A))*
*           ( abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
*          inv(A) is the inverse of A
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(A)*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(A)*abs(X) + abs(B) is less than SAFE2.
*
*        Use SLACN2 to estimate the infinity-norm of the matrix
*           inv(A) * diag(W),
*        where W = abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) )))
*
```

```
         DO 90 I = 1, N
            IF( WORK( I ).GT.SAFE2 ) THEN
               WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I )
            ELSE
               WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I ) + SAFE1
            END IF
   90    CONTINUE
*
         KASE = 0
  100    CONTINUE
         CALL SLACN2( N, WORK( 2*N+1 ), WORK( N+1 ), IWORK, FERR( J ),
     $                KASE, ISAVE )
         IF( KASE.NE.0 ) THEN
            IF( KASE.EQ.1 ) THEN
*
*              Multiply by diag(W)*inv(A**T).
*
               CALL SPOTRS( UPLO, N, 1, AF, LDAF, WORK( N+1 ), N, INFO )
               DO 110 I = 1, N
                  WORK( N+I ) = WORK( I )*WORK( N+I )
  110          CONTINUE
            ELSE IF( KASE.EQ.2 ) THEN
*
*              Multiply by inv(A)*diag(W).
*
               DO 120 I = 1, N
                  WORK( N+I ) = WORK( I )*WORK( N+I )
  120          CONTINUE
               CALL SPOTRS( UPLO, N, 1, AF, LDAF, WORK( N+1 ), N, INFO )
            END IF
            GO TO 100
         END IF
*
*        Normalize error.
*
         LSTRES = ZERO
         DO 130 I = 1, N
            LSTRES = MAX( LSTRES, ABS( X( I, J ) ) )
  130    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  140 CONTINUE
*
      RETURN
*
*     End of SPORFS
*
      END
```

———-

71.7.4 lisp code

— sporfs.lisp —

```
(let* ((itmax 5) (zero 0.0) (one 1.0) (two 2.0) (three 3.0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 3.0 3.0) three)
           (ignorable itmax zero one two three))
  (defun sporfs
         (uplo n nrhs a lda af ldaf b ldb$ x ldx ferr berr work iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (array single-float (*)) work berr ferr x b af a)
             (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (af single-float af-%data% af-%offset%)
         (b single-float b-%data% b-%offset%)
         (x single-float x-%data% x-%offset%)
         (ferr single-float ferr-%data% ferr-%offset%)
         (berr single-float berr-%data% berr-%offset%)
         (work single-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4)) (eps 0.0)
             (lstres 0.0) (s 0.0) (safe1 0.0) (safe2 0.0) (safmin 0.0) (xk 0.0)
             (count$ 0) (i 0) (j 0) (k 0) (kase 0) (nz 0) (upper nil))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (single-float) eps lstres s safe1 safe2 safmin xk)
                 (type (f2cl-lib:integer4) count$ i j k kase nz)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
```

```
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -11)))
(cond
 ((/= info 0) (xerbla "SPORFS" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%) zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) zero)
     label10))
  (go end_label)))
(setf nz (f2cl-lib:int-add n 1))
(setf eps (slamch "Epsilon"))
(setf safmin (slamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy n
         (f2cl-lib:array-slice b-%data% single-float (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1
         (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                               ((1 *)) work-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (ssymv uplo n (- one) a lda
         (f2cl-lib:array-slice x-%data% single-float (1 j)
```

```
                              ((1 ldx) (1 *)) x-%offset%)
        1 one
        (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                              ((1 *)) work-%offset%)
        1)
    (declare (ignore var-2 var-3 var-5 var-6 var-8 var-9))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf lda var-4))
    (when var-7 (setf one var-7)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (abs
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)))
     label30))
  (cond
   (upper
    (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                  ((> k n) nil)
      (tagbody
        (setf s zero)
        (setf xk
                (abs
                 (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                                x-%offset%)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i
                          (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)
                    (+
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)
                     (*
                      (abs
                       (f2cl-lib:fref a-%data% (i k) ((1 lda) (1 *))
                                      a-%offset%))
                      xk)))
            (setf s
                    (+ s
                       (*
                        (abs
                         (f2cl-lib:fref a-%data% (i k)
                                        ((1 lda) (1 *)) a-%offset%))
                        (abs
```

```
                              (f2cl-lib:fref x-%data% (i j)
                                             ((1 ldx) (1 *))
                                             x-%offset%)))))
                 label40))
            (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                  (+
                   (f2cl-lib:fref work-%data% (k) ((1 *))
                                  work-%offset%)
                   (*
                    (abs
                     (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                    a-%offset%))
                    xk)
                   s))
          label50)))
      (t
       (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                     ((> k n) nil)
         (tagbody
           (setf s zero)
           (setf xk
                 (abs
                  (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                                 x-%offset%)))
           (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                 (+
                  (f2cl-lib:fref work-%data% (k) ((1 *))
                                 work-%offset%)
                  (*
                   (abs
                    (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                   a-%offset%))
                   xk)))
           (f2cl-lib:fdo (i (f2cl-lib:int-add k 1)
                            (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)
                     (+
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%)
                      (*
                       (abs
                        (f2cl-lib:fref a-%data% (i k) ((1 lda) (1 *))
                                       a-%offset%))
                       xk)))
               (setf s
                     (+ s
                        (*
```

```
                        (abs
                         (f2cl-lib:fref a-%data% (i k)
                                        ((1 lda) (1 *)) a-%offset%))
                        (abs
                         (f2cl-lib:fref x-%data% (i j)
                                        ((1 ldx) (1 *))
                                        x-%offset%)))))
          label60))
      (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
              (+
               (f2cl-lib:fref work-%data% (k) ((1 *))
                              work-%offset%)
               s))
     label70))))
(setf s zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
      (setf s
              (max s
                   (/
                    (abs
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add n i))
                                    ((1 *)) work-%offset%))
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)))))
     (t
      (setf s
              (max s
                   (/
                    (+
                     (abs
                      (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-add n i))
                                     ((1 *)) work-%offset%))
                     safe1)
                    (+
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)
                     safe1))))))
   label80))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
  (multiple-value-bind
```

```
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (spotrs uplo n 1 af ldaf
         (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                               ((1 *)) work-%offset%)
         n info)
      (declare (ignore var-2 var-3 var-5))
      (when var-0 (setf uplo var-0))
      (when var-1 (setf n var-1))
      (when var-4 (setf ldaf var-4))
      (when var-6 (setf n var-6))
      (when var-7 (setf info var-7)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (saxpy n one
         (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                               ((1 *)) work-%offset%)
         1
         (f2cl-lib:array-slice x-%data% single-float (1 j)
                               ((1 ldx) (1 *)) x-%offset%)
         1)
      (declare (ignore var-2 var-3 var-4 var-5))
      (when var-0 (setf n var-0))
      (when var-1 (setf one var-1)))
    (setf lstres
            (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
    (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (+
                 (abs
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                 ((1 *)) work-%offset%))
                 (* nz eps
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)))))
       (t
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (+
                 (abs
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                 ((1 *)) work-%offset%))
                 (* nz eps
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%))
                 safe1))))
     label90))
  (setf kase 0)
```

```
label100
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (slacn2 n
      (f2cl-lib:array-slice work-%data% single-float
                            ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                            ((1 *)) work-%offset%)
      iwork (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
      kase isave)
   (declare (ignore var-1 var-2 var-3 var-6))
   (setf n var-0)
   (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
           var-4)
   (setf kase var-5))
 (cond
  ((/= kase 0)
   (cond
    ((= kase 1)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (spotrs uplo n 1 af ldaf
          (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                                ((1 *)) work-%offset%)
          n info)
       (declare (ignore var-2 var-3 var-5))
       (when var-0 (setf uplo var-0))
       (when var-1 (setf n var-1))
       (when var-4 (setf ldaf var-4))
       (when var-6 (setf n var-6))
       (when var-7 (setf info var-7)))
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                              ((1 *)) work-%offset%)
                 (*
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add n i)) ((1 *))
                                 work-%offset%)))
        label110)))
    ((= kase 2)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                              ((1 *)) work-%offset%)
                 (*
```

```
                            (f2cl-lib:fref work-%data% (i) ((1 *))
                                           work-%offset%)
                            (f2cl-lib:fref work-%data%
                                           ((f2cl-lib:int-add n i)) ((1 *))
                                           work-%offset%)))
                   label120))
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
                    (spotrs uplo n 1 af ldaf
                     (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                                           ((1 *)) work-%offset%)
                     n info)
                  (declare (ignore var-2 var-3 var-5))
                  (when var-0 (setf uplo var-0))
                  (when var-1 (setf n var-1))
                  (when var-4 (setf ldaf var-4))
                  (when var-6 (setf n var-6))
                  (when var-7 (setf info var-7)))))
              (go label100)))
            (setf lstres zero)
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf lstres
                        (max lstres
                             (abs
                              (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                             x-%offset%))))
               label130))
            (if (/= lstres zero)
                (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                        (/
                         (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                         lstres)))
           label140))
         (go end_label)
        end_label
         (return
          (values uplo n nil nil lda nil ldaf nil nil nil nil nil nil nil nil
                  info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sporfs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
```

```
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::uplo
  fortran-to-lisp::n nil nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldaf nil nil nil
  nil nil nil nil nil
  fortran-to-lisp::info)
:calls '(fortran-to-lisp::slacn2))))
```

———-

71.8 sporfsx

71.8.1 Axiom unit tests

— sporfsx.input —

```
)set break resume
)sys rm -f sporfsx.output
)spool sporfsx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

71.8.2 Axiom help page [Demm05]

— sporfsx.help —

```
====================================================================
sporfsx examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sporfsx (UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF, S, B, LDB,
           X, LDX, RCOND, BERR, N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP,
           NPARAMS, PARAMS, WORK, IWORK, INFO)
           SPORFSX

Function/Subroutine Documentation
   subroutine sporfsx (character UPLO, character EQUED, integer N, integer
       NRHS, real, dimension( lda, * ) A, integer LDA, real, dimension( ldaf,
       * ) AF, integer LDAF, real, dimension( * ) S, real, dimension( ldb, * )
       B, integer LDB, real, dimension( ldx, * ) X, integer LDX, real RCOND,
       real, dimension( * ) BERR, integer N_ERR_BNDS, real, dimension( nrhs, *
       ) ERR_BNDS_NORM, real, dimension( nrhs, * ) ERR_BNDS_COMP, integer
       NPARAMS, real, dimension( * ) PARAMS, real, dimension( * ) WORK,
       integer, dimension( * ) IWORK, integer INFO)
       SPORFSX

Purpose:
   SPORFSX improves the computed solution to a system of linear
   equations when the coefficient matrix is symmetric positive
   definite, and provides error bounds and backward error estimates
   for the solution.  In addition to normwise error bound, the code
   provides maximum componentwise error bound if possible.  See
   comments for ERR_BNDS_NORM and ERR_BNDS_COMP for details of the
   error bounds.

   The original system of linear equations may have been equilibrated
   before calling this routine, as described by arguments EQUED and S
   below. In this case, the solution and error bounds returned are
   for the original unequilibrated system.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.
```

```
Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done to A
   before calling this routine. This is needed to compute
   the solution and error bounds correctly.
   = 'N':  No equilibration
   = 'Y':  Both row and column equilibration, i.e., A has been
           replaced by diag(S) * A * diag(S).
           The right hand side B has been changed accordingly.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 A
   A is REAL array, dimension (LDA,N)
   The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
   upper triangular part of A contains the upper triangular part
   of the matrix A, and the strictly lower triangular part of A
   is not referenced.  If UPLO = 'L', the leading N-by-N lower
   triangular part of A contains the lower triangular part of
   the matrix A, and the strictly upper triangular part of A is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is REAL array, dimension (LDAF,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**T*U or A = L*L**T, as computed by SPOTRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).
```

```
S
  S is REAL array, dimension (N)
  The row scale factors for A.  If EQUED = 'Y', A is multiplied on
  the left and right by diag(S).  S is an input argument if FACT =
  'F'; otherwise, S is an output argument.  If FACT = 'F' and EQUED
  = 'Y', each element of S must be positive.  If S is output, each
  element of S is a power of the radix. If S is input, each element
  of S should be a power of the radix to ensure a reliable solution
  and error estimates. Scaling by powers of the radix does not cause
  rounding errors unless the result underflows or overflows.
  Rounding errors during scaling lead to refining with a matrix that
  is not equivalent to the input matrix, producing error estimates
  that may not be reliable.

B
  B is REAL array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is REAL array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by SGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

BERR
  BERR is REAL array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
```

```
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
           max_j (abs(XTRUE(j,i) - X(j,i)))
            ------------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * slamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*A, where S scales each row by a power of the
     radix so all absolute row sums of Z are approximately 1.

ERR_BNDS_COMP
  ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
```

```
              abs(XTRUE(j,i) - X(j,i))
         max_j ----------------------
                     abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
    reciprocal condition number is less than the threshold
    sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
    almost certainly within a factor of 10 of the true error
    so long as the next entry is greater than the threshold
    sqrt(n) * slamch('Epsilon'). This error bound should only
    be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
    reciprocal condition number.  Compared with the threshold
    sqrt(n) * slamch('Epsilon') to determine if the error
    estimate is "guaranteed". These reciprocal condition
    numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
    appropriately scaled matrix Z.
    Let Z = S*(A*diag(x)), where x is the solution for the
    current right-hand side and S scales each row of
    A*diag(x) by a power of the radix so all absolute row
    sums of Z are approximately 1.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is REAL array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.
```

```
  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
  (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is REAL array, dimension (4*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is
  guaranteed.
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
     has been completed, but the factor U is exactly singular, so
     the solution and error bounds could not be computed. RCOND = 0
     is returned.
  = N+J: The solution corresponding to the Jth right-hand side is
     not guaranteed. The solutions corresponding to other right-
     hand sides K with K > J may not be guaranteed as well, but
     only the first such right-hand side is reported. If a small
     componentwise error is not requested (PARAMS(3) = 0.0) then
     the Jth right-hand side is the first with a normwise error
     bound that is not guaranteed (the smallest J such
     that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
     the Jth right-hand side is the first with either a normwise or
     componentwise error bound that is not guaranteed (the smallest
```

```
      J such that either ERR_BNDS_NORM(J,1) = 0.0 or
      ERR_BNDS_COMP(J,1) = 0.0). See the definition of
      ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
      about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

———-

71.8.3 fortran code

— **sporfsx.f** —

```
*  ====================================================================
      SUBROUTINE SPORFSX( UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF, S, B,
     $                    LDB, X, LDX, RCOND, BERR, N_ERR_BNDS,
     $                    ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $                    WORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO, EQUED
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      REAL               A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX, * ), WORK( * )
      REAL               S( * ), PARAMS( * ), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               ITREF_DEFAULT, ITHRESH_DEFAULT,
     $                   COMPONENTWISE_DEFAULT
      REAL               RTHRESH_DEFAULT, DZTHRESH_DEFAULT
      PARAMETER          ( ITREF_DEFAULT = 1.0 )
      PARAMETER          ( ITHRESH_DEFAULT = 10.0 )
```

```
      PARAMETER          ( COMPONENTWISE_DEFAULT = 1.0 )
      PARAMETER          ( RTHRESH_DEFAULT = 0.5 )
      PARAMETER          ( DZTHRESH_DEFAULT = 0.25 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. Local Scalars ..
      CHARACTER(1)       NORM
      LOGICAL            RCEQU
      INTEGER            J, PREC_TYPE, REF_TYPE
      INTEGER            N_NORMS
      REAL               ANORM, RCOND_TMP
      REAL               ILLRCOND_THRESH, ERR_LBND, CWISE_WRONG
      LOGICAL            IGNORE_CWISE
      INTEGER            ITHRESH
      REAL               RTHRESH, UNSTABLE_THRESH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, SPOCON, SLA_PORFSX_EXTENDED
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, BLAS_FPINFO_X, ILATRANS, ILAPREC
      EXTERNAL           SLAMCH, SLANSY, SLA_PORCOND
      REAL               SLAMCH, SLANSY, SLA_PORCOND
      LOGICAL            LSAME
      INTEGER            BLAS_FPINFO_X
      INTEGER            ILATRANS, ILAPREC
*     ..
*     .. Executable Statements ..
*
*     Check the input parameters.
*
      INFO = 0
      REF_TYPE = INT( ITREF_DEFAULT )
      IF ( NPARAMS .GE. LA_LINRX_ITREF_I ) THEN
         IF ( PARAMS( LA_LINRX_ITREF_I ) .LT. 0.0 ) THEN
            PARAMS( LA_LINRX_ITREF_I ) = ITREF_DEFAULT
         ELSE
            REF_TYPE = PARAMS( LA_LINRX_ITREF_I )
         END IF
```

```
      END IF
*
*     Set default parameters.
*
      ILLRCOND_THRESH = REAL( N ) * SLAMCH( 'Epsilon' )
      ITHRESH = INT( ITHRESH_DEFAULT )
      RTHRESH = RTHRESH_DEFAULT
      UNSTABLE_THRESH = DZTHRESH_DEFAULT
      IGNORE_CWISE = COMPONENTWISE_DEFAULT .EQ. 0.0
*
      IF ( NPARAMS.GE.LA_LINRX_ITHRESH_I ) THEN
         IF ( PARAMS( LA_LINRX_ITHRESH_I ).LT.0.0 ) THEN
            PARAMS( LA_LINRX_ITHRESH_I ) = ITHRESH
         ELSE
            ITHRESH = INT( PARAMS( LA_LINRX_ITHRESH_I ) )
         END IF
      END IF
      IF ( NPARAMS.GE.LA_LINRX_CWISE_I ) THEN
         IF ( PARAMS( LA_LINRX_CWISE_I ).LT.0.0 ) THEN
            IF ( IGNORE_CWISE ) THEN
               PARAMS( LA_LINRX_CWISE_I ) = 0.0
            ELSE
               PARAMS( LA_LINRX_CWISE_I ) = 1.0
            END IF
         ELSE
            IGNORE_CWISE = PARAMS( LA_LINRX_CWISE_I ) .EQ. 0.0
         END IF
      END IF
      IF ( REF_TYPE .EQ. 0 .OR. N_ERR_BNDS .EQ. 0 ) THEN
         N_NORMS = 0
      ELSE IF ( IGNORE_CWISE ) THEN
         N_NORMS = 1
      ELSE
         N_NORMS = 2
      END IF
*
      RCEQU = LSAME( EQUED, 'Y' )
*
*     Test input parameters.
*
      IF (.NOT.LSAME(UPLO, 'U') .AND. .NOT.LSAME(UPLO, 'L')) THEN
        INFO = -1
      ELSE IF( .NOT.RCEQU .AND. .NOT.LSAME( EQUED, 'N' ) ) THEN
        INFO = -2
      ELSE IF( N.LT.0 ) THEN
        INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
        INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
        INFO = -6
```

```
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
        INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
        INFO = -11
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
        INFO = -13
      END IF
      IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'SPORFSX', -INFO )
        RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         RCOND = 1.0
         DO J = 1, NRHS
            BERR( J ) = 0.0
            IF ( N_ERR_BNDS .GE. 1 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
            IF ( N_ERR_BNDS .GE. 2 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 0.0
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 0.0
            END IF
            IF ( N_ERR_BNDS .GE. 3 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 1.0
            END IF
         END DO
         RETURN
      END IF
*
*     Default to failure.
*
      RCOND = 0.0
      DO J = 1, NRHS
         BERR( J ) = 1.0
         IF ( N_ERR_BNDS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
            ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
         END IF
         IF ( N_ERR_BNDS .GE. 2 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
         END IF
         IF ( N_ERR_BNDS .GE. 3 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 0.0
            ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 0.0
```

```
         END IF
      END DO
*
*     Compute the norm of A and the reciprocal of the condition
*     number of A.
*
      NORM = 'I'
      ANORM = SLANSY( NORM, UPLO, N, A, LDA, WORK )
      CALL SPOCON( UPLO, N, AF, LDAF, ANORM, RCOND, WORK,
     $     IWORK, INFO )
*
*     Perform refinement on each right-hand side
*
      IF ( REF_TYPE .NE. 0 ) THEN

         PREC_TYPE = ILAPREC( 'D' )

         CALL SLA_PORFSX_EXTENDED( PREC_TYPE, UPLO,  N,
     $        NRHS, A, LDA, AF, LDAF, RCEQU, S, B,
     $        LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM, ERR_BNDS_COMP,
     $        WORK( N+1 ), WORK( 1 ), WORK( 2*N+1 ), WORK( 1 ), RCOND,
     $        ITHRESH, RTHRESH, UNSTABLE_THRESH, IGNORE_CWISE,
     $        INFO )
      END IF

      ERR_LBND = MAX( 10.0, SQRT( REAL( N ) ) ) * SLAMCH( 'Epsilon' )
      IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 1 ) THEN
*
*     Compute scaled normwise condition number cond(A*C).
*
         IF ( RCEQU ) THEN
            RCOND_TMP = SLA_PORCOND( UPLO, N, A, LDA, AF, LDAF,
     $           -1, S, INFO, WORK, IWORK )
         ELSE
            RCOND_TMP = SLA_PORCOND( UPLO, N, A, LDA, AF, LDAF,
     $           0, S, INFO, WORK, IWORK )
         END IF
         DO J = 1, NRHS
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .GT. 1.0 )
     $           ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 0.0
```

```
            IF ( INFO .LE. N ) INFO = N + J
         ELSE IF ( ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .LT. ERR_LBND )
     $              THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = ERR_LBND
            ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
         END IF
*
*     Save the condition number.
*
         IF (N_ERR_BNDS .GE. LA_LINRX_RCOND_I) THEN
            ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = RCOND_TMP
         END IF
      END DO
   END IF

   IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 2 ) THEN
*
*     Compute componentwise condition number cond(A*diag(Y(:,J))) for
*     each right-hand side using the current solution as an estimate of
*     the true solution.  If the componentwise error estimate is too
*     large, then the solution is a lousy estimate of truth and the
*     estimated RCOND may be too optimistic.  To avoid misleading users,
*     the inverse condition number is set to 0.0 when the estimated
*     cwise error is at least CWISE_WRONG.
*
      CWISE_WRONG = SQRT( SLAMCH( 'Epsilon' ) )
      DO J = 1, NRHS
         IF (ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .LT. CWISE_WRONG )
     $     THEN
            RCOND_TMP = SLA_PORCOND( UPLO, N, A, LDA, AF, LDAF, 1,
     $              X( 1, J ), INFO, WORK, IWORK )
         ELSE
            RCOND_TMP = 0.0
         END IF
*
*     Cap the error at 1.0.
*
         IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .GT. 1.0 )
     $           ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
*
*     Threshold the error (see LAWN).
*
         IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
            ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 0.0
            IF ( PARAMS( LA_LINRX_CWISE_I ) .EQ. 1.0
     $              .AND. INFO.LT.N + J ) INFO = N + J
         ELSE IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I )
     $              .LT. ERR_LBND ) THEN
```

```
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF

         END DO
      END IF
*
      RETURN
*
*     End of SPORFSX
*
      END
```

———-

71.8.4 lisp code

— **sporfsx.lisp** —

———-

71.9 spotf2

71.9.1 Axiom unit tests

— **spotf2.input** —

```
)set break resume
)sys rm -f spotf2.output
)spool spotf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

71.9.2 Axiom help page

— spotf2.help —

```
=====================================================================
spotf2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine spotf2 (UPLO, N, A, LDA, INFO)
           SPOTF2 computes the Cholesky factorization of a symmetric/Hermitian
           positive definite matrix (unblocked algorithm).

Function/Subroutine Documentation
   subroutine spotf2 (character UPLO, integer N, real, dimension( lda, * ) A,
       integer LDA, integer INFO)
       SPOTF2 computes the Cholesky factorization of a symmetric/Hermitian
       positive definite matrix (unblocked algorithm).

Purpose:
   SPOTF2 computes the Cholesky factorization of a real symmetric
   positive definite matrix A.

   The factorization has the form
     A = U**T * U ,  if UPLO = 'U', or
     A = L  * L**T,  if UPLO = 'L',
   where U is an upper triangular matrix and L is lower triangular.

   This is the unblocked version of the algorithm, calling Level 2 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   symmetric matrix A is stored.
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
```

```
  The order of the matrix A.  N >= 0.

A
  A is REAL array, dimension (LDA,N)
  On entry, the symmetric matrix A.  If UPLO = 'U', the leading
  n by n upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading n by n lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if INFO = 0, the factor U or L from the Cholesky
  factorization A = U**T *U  or A = L*L**T.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -k, the k-th argument had an illegal value
  > 0: if INFO = k, the leading minor of order k is not
       positive definite, and the factorization could not be completed.
```

———-

71.9.3 fortran code

— **spotf2.f** —

```
*  =====================================================================
      SUBROUTINE SPOTF2( UPLO, N, A, LDA, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * )
*     ..
```

```
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            J
      REAL               AJJ
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, SISNAN
      REAL               SDOT
      EXTERNAL           LSAME, SDOT, SISNAN
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEMV, SSCAL, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SPOTF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*        Compute the Cholesky factorization A = U**T *U.
*
```

```
         DO 10 J = 1, N
*
*           Compute U(J,J) and test for non-positive-definiteness.
*
            AJJ = A( J, J ) - SDOT( J-1, A( 1, J ), 1, A( 1, J ), 1 )
            IF( AJJ.LE.ZERO.OR.SISNAN( AJJ ) ) THEN
               A( J, J ) = AJJ
               GO TO 30
            END IF
            AJJ = SQRT( AJJ )
            A( J, J ) = AJJ
*
*           Compute elements J+1:N of row J.
*
            IF( J.LT.N ) THEN
               CALL SGEMV( 'Transpose', J-1, N-J, -ONE, A( 1, J+1 ),
     $                     LDA, A( 1, J ), 1, ONE, A( J, J+1 ), LDA )
               CALL SSCAL( N-J, ONE / AJJ, A( J, J+1 ), LDA )
            END IF
   10    CONTINUE
      ELSE
*
*        Compute the Cholesky factorization A = L*L**T.
*
         DO 20 J = 1, N
*
*           Compute L(J,J) and test for non-positive-definiteness.
*
            AJJ = A( J, J ) - SDOT( J-1, A( J, 1 ), LDA, A( J, 1 ),
     $            LDA )
            IF( AJJ.LE.ZERO.OR.SISNAN( AJJ ) ) THEN
               A( J, J ) = AJJ
               GO TO 30
            END IF
            AJJ = SQRT( AJJ )
            A( J, J ) = AJJ
*
*           Compute elements J+1:N of column J.
*
            IF( J.LT.N ) THEN
               CALL SGEMV( 'No transpose', N-J, J-1, -ONE, A( J+1, 1 ),
     $                     LDA, A( J, 1 ), LDA, ONE, A( J+1, J ), 1 )
               CALL SSCAL( N-J, ONE / AJJ, A( J+1, J ), 1 )
            END IF
   20    CONTINUE
      END IF
      GO TO 40
*
   30 CONTINUE
      INFO = J
```

```
*
   40 CONTINUE
      RETURN
*
*     End of SPOTF2
*
      END
```

———-

71.9.4 lisp code

— **spotf2.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun spotf2 (uplo n a lda info)
    (declare (type (array single-float (*)) a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%))
      (prog ((ajj 0.0) (j 0) (upper nil))
        (declare (type (single-float) ajj)
                 (type (f2cl-lib:integer4) j)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -4)))
```

```
(cond
 ((/= info 0) (xerbla "SPOTF2" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(cond
 (upper
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf ajj
              (-
               (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                              a-%offset%)
               (sdot (f2cl-lib:int-sub j 1)
                (f2cl-lib:array-slice a-%data% single-float (1 j)
                                      ((1 lda) (1 *)) a-%offset%)
                1
                (f2cl-lib:array-slice a-%data% single-float (1 j)
                                      ((1 lda) (1 *)) a-%offset%)
                1)))
      (cond
       ((or (<= ajj zero) (sisnan ajj))
        (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
                ajj)
        (go label30)))
      (setf ajj (f2cl-lib:fsqrt ajj))
      (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
              ajj)
      (cond
       ((< j n)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10)
            (sgemv "Transpose" (f2cl-lib:int-sub j 1)
             (f2cl-lib:int-sub n j) (- one)
             (f2cl-lib:array-slice a-%data% single-float
                                   (1 (f2cl-lib:int-add j 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice a-%data% single-float (1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             1 one
             (f2cl-lib:array-slice a-%data% single-float
                                   (j (f2cl-lib:int-add j 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare
           (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7 var-9))
          (when var-5 (setf lda var-5))
```

```
            (when var-8 (setf one var-8))
            (when var-10 (setf lda var-10)))
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (sscal (f2cl-lib:int-sub n j) (/ one ajj)
               (f2cl-lib:array-slice a-%data% single-float
                                     (j (f2cl-lib:int-add j 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda)
            (declare (ignore var-0 var-1 var-2))
            (when var-3 (setf lda var-3)))))
      label10)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf ajj
               (-
                (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                               a-%offset%)
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4)
                    (sdot (f2cl-lib:int-sub j 1)
                     (f2cl-lib:array-slice a-%data% single-float (j 1)
                                           ((1 lda) (1 *)) a-%offset%)
                     lda
                     (f2cl-lib:array-slice a-%data% single-float (j 1)
                                           ((1 lda) (1 *)) a-%offset%)
                     lda)
                  (declare (ignore var-0 var-1 var-3))
                  (when var-2 (setf lda var-2))
                  (when var-4 (setf lda var-4))
                  ret-val)))
       (cond
        ((or (<= ajj zero) (sisnan ajj))
         (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
                 ajj)
         (go label30)))
       (setf ajj (f2cl-lib:fsqrt ajj))
       (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
               ajj)
       (cond
        ((< j n)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9 var-10)
             (sgemv "No transpose" (f2cl-lib:int-sub n j)
              (f2cl-lib:int-sub j 1) (- one)
              (f2cl-lib:array-slice a-%data% single-float ((+ j 1) 1)
                                    ((1 lda) (1 *)) a-%offset%)
              lda
```

```
                    (f2cl-lib:array-slice a-%data% single-float (j 1)
                                          ((1 lda) (1 *)) a-%offset%)
                    lda one
                    (f2cl-lib:array-slice a-%data% single-float ((+ j 1) j)
                                          ((1 lda) (1 *)) a-%offset%)
                    1)
                  (declare
                   (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9 var-10))
                  (when var-5 (setf lda var-5))
                  (when var-7 (setf lda var-7))
                  (when var-8 (setf one var-8)))
                 (sscal (f2cl-lib:int-sub n j) (/ one ajj)
                  (f2cl-lib:array-slice a-%data% single-float ((+ j 1) j)
                                        ((1 lda) (1 *)) a-%offset%)
                  1)))
               label20))))
        (go label40)
       label30
        (setf info j)
       label40
        (go end_label)
       end_label
        (return (values uplo nil nil lda info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::spotf2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::lda
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::sisnan))))
```

———-

71.10 spotrf

71.10.1 Axiom unit tests

— spotrf.input —

```
)set break resume
)sys rm -f spotrf.output
)spool spotrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

71.10.2 Axiom help page

— spotrf.help —

```
======================================================================
spotrf examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine spotrf (UPLO, N, A, LDA, INFO)
           SPOTRF

Function/Subroutine Documentation
   subroutine spotrf (character UPLO, integer N, real, dimension( lda, * ) A,
       integer LDA, integer INFO)
       SPOTRF

Purpose:
   SPOTRF computes the Cholesky factorization of a real symmetric
   positive definite matrix A.

   The factorization has the form
      A = U**T * U,  if UPLO = 'U', or
```

```
      A = L  * L**T,  if UPLO = 'L',
   where U is an upper triangular matrix and L is lower triangular.

   This is the block version of the algorithm, calling Level 3 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   N-by-N upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading N-by-N lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

   On exit, if INFO = 0, the factor U or L from the Cholesky
   factorization A = U**T*U or A = L*L**T.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, the leading minor of order i is not
         positive definite, and the factorization could not be completed.
```

71.10.3 fortran code

— spotrf.f —

```
*  =====================================================================
```

```
      SUBROUTINE SPOTRF( UPLO, N, A, LDA, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            J, JB, NB
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEMM, SPOTRF2, SSYRK, STRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
```

```
         CALL XERBLA( 'SPOTRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Determine the block size for this environment.
*
      NB = ILAENV( 1, 'SPOTRF', UPLO, N, -1, -1, -1 )
      IF( NB.LE.1 .OR. NB.GE.N ) THEN
*
*        Use unblocked code.
*
         CALL SPOTRF2( UPLO, N, A, LDA, INFO )
      ELSE
*
*        Use blocked code.
*
         IF( UPPER ) THEN
*
*           Compute the Cholesky factorization A = U**T*U.
*
            DO 10 J = 1, N, NB
*
*              Update and factorize the current diagonal block and test
*              for non-positive-definiteness.
*
               JB = MIN( NB, N-J+1 )
               CALL SSYRK( 'Upper', 'Transpose', JB, J-1, -ONE,
     $                     A( 1, J ), LDA, ONE, A( J, J ), LDA )
               CALL SPOTRF2( 'Upper', JB, A( J, J ), LDA, INFO )
               IF( INFO.NE.0 )
     $            GO TO 30
               IF( J+JB.LE.N ) THEN
*
                  Compute the current block row.
*
                  CALL SGEMM( 'Transpose', 'No transpose', JB, N-J-JB+1,
     $                        J-1, -ONE, A( 1, J ), LDA, A( 1, J+JB ),
     $                        LDA, ONE, A( J, J+JB ), LDA )
                  CALL STRSM( 'Left', 'Upper', 'Transpose', 'Non-unit',
     $                        JB, N-J-JB+1, ONE, A( J, J ), LDA,
     $                        A( J, J+JB ), LDA )
               END IF
   10       CONTINUE
*
         ELSE
```

```
*
*          Compute the Cholesky factorization A = L*L**T.
*
           DO 20 J = 1, N, NB
*
*             Update and factorize the current diagonal block and test
*             for non-positive-definiteness.
*
              JB = MIN( NB, N-J+1 )
              CALL SSYRK( 'Lower', 'No transpose', JB, J-1, -ONE,
     $                    A( J, 1 ), LDA, ONE, A( J, J ), LDA )
              CALL SPOTRF2( 'Lower', JB, A( J, J ), LDA, INFO )
              IF( INFO.NE.0 )
     $           GO TO 30
              IF( J+JB.LE.N ) THEN
*
*                Compute the current block column.
*
                 CALL SGEMM( 'No transpose', 'Transpose', N-J-JB+1, JB,
     $                       J-1, -ONE, A( J+JB, 1 ), LDA, A( J, 1 ),
     $                       LDA, ONE, A( J+JB, J ), LDA )
                 CALL STRSM( 'Right', 'Lower', 'Transpose', 'Non-unit',
     $                       N-J-JB+1, JB, ONE, A( J, J ), LDA,
     $                       A( J+JB, J ), LDA )
              END IF
   20      CONTINUE
        END IF
      END IF
      GO TO 40
*
   30 CONTINUE
      INFO = INFO + J - 1
*
   40 CONTINUE
      RETURN
*
*     End of SPOTRF
*
      END
```

———-

71.10.4 lisp code

— spotrf.lisp —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
```

```
          (ignorable one))
 (defun spotrf (uplo n a lda info)
   (declare (type (array single-float (*)) a)
            (type (f2cl-lib:integer4) info lda n)
            (type (string *) uplo))
   (f2cl-lib:with-multi-array-data
       ((uplo character uplo-%data% uplo-%offset%)
        (a single-float a-%data% a-%offset%))
     (prog ((j 0) (jb 0) (nb 0) (upper nil))
       (declare (type (f2cl-lib:integer4) j jb nb)
                (type f2cl-lib:logical upper))
       (setf info 0)
       (setf upper
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame uplo "U")
                 (declare (ignore var-1))
                 (when var-0 (setf uplo var-0))
                 ret-val))
       (cond
        ((and (not upper)
              (not
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame uplo "L")
                 (declare (ignore var-1))
                 (when var-0 (setf uplo var-0))
                 ret-val)))
         (setf info -1))
        ((< n 0) (setf info -2))
        ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (setf info -4)))
       (cond
        ((/= info 0) (xerbla "SPOTRF" (f2cl-lib:int-sub info))
         (go end_label)))
       (if (= n 0)
           (go end_label))
       (setf nb
               (multiple-value-bind
                   (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                   (ilaenv 1 "SPOTRF" uplo n -1 -1 -1)
                 (declare (ignore var-0 var-1 var-4 var-5 var-6))
                 (setf uplo var-2)
                 (setf n var-3)
                 ret-val))
       (cond
        ((or (<= nb 1) (>= nb n))
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (spotrf2 uplo n a lda info)
           (declare (ignore var-2))
           (when var-0 (setf uplo var-0))
           (when var-1 (setf n var-1))
```

```
    (when var-3 (setf lda var-3))
    (when var-4 (setf info var-4))))
  (t
   (cond
    (upper
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                   ((> j n) nil)
       (tagbody
         (setf jb
                 (min (the f2cl-lib:integer4 nb)
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add (f2cl-lib:int-sub n j)
                                             1))))
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9)
             (ssyrk "Upper" "Transpose" jb (f2cl-lib:int-sub j 1)
              (- one)
              (f2cl-lib:array-slice a-%data% single-float (1 j)
                                    ((1 lda) (1 *)) a-%offset%)
              lda one
              (f2cl-lib:array-slice a-%data% single-float (j j)
                                    ((1 lda) (1 *)) a-%offset%)
              lda)
           (declare (ignore var-0 var-1 var-3 var-4 var-5 var-8))
           (when var-2 (setf jb var-2))
           (when var-6 (setf lda var-6))
           (when var-7 (setf one var-7))
           (when var-9 (setf lda var-9)))
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (spotrf2 "Upper" jb
              (f2cl-lib:array-slice a-%data% single-float (j j)
                                    ((1 lda) (1 *)) a-%offset%)
              lda info)
           (declare (ignore var-0 var-2))
           (when var-1 (setf jb var-1))
           (when var-3 (setf lda var-3))
           (when var-4 (setf info var-4)))
         (if (/= info 0)
             (go label30))
         (cond
          ((<= (f2cl-lib:int-add j jb) n)
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                var-9 var-10 var-11 var-12)
               (sgemm "Transpose" "No transpose" jb
                (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1)
                (f2cl-lib:int-sub j 1) (- one)
                (f2cl-lib:array-slice a-%data% single-float (1 j)
                                      ((1 lda) (1 *)) a-%offset%)
```

```
                lda
                (f2cl-lib:array-slice a-%data% single-float
                                      (1 (f2cl-lib:int-add j jb))
                                      ((1 lda) (1 *)) a-%offset%)
                lda one
                (f2cl-lib:array-slice a-%data% single-float
                                      (j (f2cl-lib:int-add j jb))
                                      ((1 lda) (1 *)) a-%offset%)
                lda)
              (declare
               (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-8 var-11))
              (when var-2 (setf jb var-2))
              (when var-7 (setf lda var-7))
              (when var-9 (setf lda var-9))
              (when var-10 (setf one var-10))
              (when var-12 (setf lda var-12)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10)
                (strsm "Left" "Upper" "Transpose" "Non-unit" jb
                 (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) one
                 (f2cl-lib:array-slice a-%data% single-float (j j)
                                      ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice a-%data% single-float
                                      (j (f2cl-lib:int-add j jb))
                                      ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9))
              (when var-4 (setf jb var-4))
              (when var-6 (setf one var-6))
              (when var-8 (setf lda var-8))
              (when var-10 (setf lda var-10)))))
        label10)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                   ((> j n) nil)
       (tagbody
         (setf jb
                 (min (the f2cl-lib:integer4 nb)
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add (f2cl-lib:int-sub n j)
                                             1))))
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9)
             (ssyrk "Lower" "No transpose" jb (f2cl-lib:int-sub j 1)
              (- one)
              (f2cl-lib:array-slice a-%data% single-float (j 1)
```

```
                              ((1 lda) (1 *)) a-%offset%)
      lda one
      (f2cl-lib:array-slice a-%data% single-float (j j)
                              ((1 lda) (1 *)) a-%offset%)
      lda)
  (declare (ignore var-0 var-1 var-3 var-4 var-5 var-8))
  (when var-2 (setf jb var-2))
  (when var-6 (setf lda var-6))
  (when var-7 (setf one var-7))
  (when var-9 (setf lda var-9)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (spotrf2 "Lower" jb
     (f2cl-lib:array-slice a-%data% single-float (j j)
                              ((1 lda) (1 *)) a-%offset%)
     lda info)
  (declare (ignore var-0 var-2))
  (when var-1 (setf jb var-1))
  (when var-3 (setf lda var-3))
  (when var-4 (setf info var-4)))
(if (/= info 0)
    (go label30))
(cond
 ((<= (f2cl-lib:int-add j jb) n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (sgemm "No transpose" "Transpose"
       (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) jb
       (f2cl-lib:int-sub j 1) (- one)
       (f2cl-lib:array-slice a-%data% single-float ((+ j jb) 1)
                               ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice a-%data% single-float (j 1)
                               ((1 lda) (1 *)) a-%offset%)
       lda one
       (f2cl-lib:array-slice a-%data% single-float ((+ j jb) j)
                               ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare
     (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-8 var-11))
    (when var-3 (setf jb var-3))
    (when var-7 (setf lda var-7))
    (when var-9 (setf lda var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf lda var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (strsm "Right" "Lower" "Transpose" "Non-unit"
       (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) jb one
```

```
                      (f2cl-lib:array-slice a-%data% single-float (j j)
                                            ((1 lda) (1 *)) a-%offset%)
                      lda
                      (f2cl-lib:array-slice a-%data% single-float ((+ j jb) j)
                                            ((1 lda) (1 *)) a-%offset%)
                      lda)
                    (declare
                     (ignore var-0 var-1 var-2 var-3 var-4 var-7 var-9))
                    (when var-5 (setf jb var-5))
                    (when var-6 (setf one var-6))
                    (when var-8 (setf lda var-8))
                    (when var-10 (setf lda var-10)))))
              label20))))))
        (go label40)
       label30
        (setf info (f2cl-lib:int-sub (f2cl-lib:int-add info j) 1))
       label40
        (go end_label)
       end_label
        (return (values uplo n nil lda info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::spotrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::ilaenv))))
```

71.11 spotrf2

71.11.1 Axiom unit tests

— spotrf2.input —

```
)set break resume
)sys rm -f spotrf2.output
)spool spotrf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

71.11.2 Axiom help page

— spotrf2.help —

```
====================================================================
spotrf2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       recursive subroutine spotrf2 (UPLO, N, A, LDA, INFO)
           SPOTRF2

Function/Subroutine Documentation
   recursive subroutine spotrf2 (character UPLO, integer N, real, dimension(
       lda, * ) A, integer LDA, integer INFO)
       SPOTRF2

Purpose:
   SPOTRF2 computes the Cholesky factorization of a real symmetric
   positive definite matrix A using the recursive algorithm.

   The factorization has the form
      A = U**T * U,  if UPLO = 'U', or
```

```
      A = L  * L**T,  if UPLO = 'L',
   where U is an upper triangular matrix and L is lower triangular.

   This is the recursive version of the algorithm. It divides
   the matrix into four submatrices:

          [  A11 | A12  ]  where A11 is n1 by n1 and A22 is n2 by n2
      A = [ -----|----- ]  with n1 = n/2
          [  A21 | A22  ]       n2 = n-n1

   The subroutine calls itself to factor A11. Update and scale A21
   or A12, update A22 then call itself to factor A22.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   N-by-N upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading N-by-N lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

   On exit, if INFO = 0, the factor U or L from the Cholesky
   factorization A = U**T*U or A = L*L**T.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, the leading minor of order i is not
         positive definite, and the factorization could not be completed.
```

———-

71.11.3 fortran code

— spotrf2.f —

```
*  ===================================================================
      RECURSIVE SUBROUTINE SPOTRF2( UPLO, N, A, LDA, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO=0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            N1, N2, IINFO
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, SISNAN
      EXTERNAL           LSAME, SISNAN
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGEMM, SSYRK, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
```

```
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SPOTRF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     N=1 case
*
      IF( N.EQ.1 ) THEN
*
*        Test for non-positive-definiteness
*
         IF( A( 1, 1 ).LE.ZERO.OR.SISNAN( A( 1, 1 ) ) ) THEN
            INFO = 1
            RETURN
         END IF
*
*        Factor
*
         A( 1, 1 ) = SQRT( A( 1, 1 ) )
*
*     Use recursive code
*
      ELSE
         N1 = N/2
         N2 = N-N1
*
*        Factor A11
*
         CALL SPOTRF2( UPLO, N1, A( 1, 1 ), LDA, IINFO )
         IF ( IINFO.NE.0 ) THEN
            INFO = IINFO
            RETURN
         END IF
*
*        Compute the Cholesky factorization A = U**T*U
*
         IF( UPPER ) THEN
*
*           Update and scale A12
*
            CALL STRSM( 'L', 'U', 'T', 'N', N1, N2, ONE,
```

```
     $                 A( 1, 1 ), LDA, A( 1, N1+1 ), LDA )
*
*          Update and factor A22
*
           CALL SSYRK( UPLO, 'T', N2, N1, -ONE, A( 1, N1+1 ), LDA,
     $                 ONE, A( N1+1, N1+1 ), LDA )
           CALL SPOTRF2( UPLO, N2, A( N1+1, N1+1 ), LDA, IINFO )
           IF ( IINFO.NE.0 ) THEN
              INFO = IINFO + N1
              RETURN
           END IF
*
        Compute the Cholesky factorization A = L*L**T
*
        ELSE
*
           Update and scale A21
*
           CALL STRSM( 'R', 'L', 'T', 'N', N2, N1, ONE,
     $                 A( 1, 1 ), LDA, A( N1+1, 1 ), LDA )
*
           Update and factor A22
*
           CALL SSYRK( UPLO, 'N', N2, N1, -ONE, A( N1+1, 1 ), LDA,
     $                 ONE, A( N1+1, N1+1 ), LDA )
           CALL SPOTRF2( UPLO, N2, A( N1+1, N1+1 ), LDA, IINFO )
           IF ( IINFO.NE.0 ) THEN
              INFO = IINFO + N1
              RETURN
           END IF
        END IF
     END IF
     RETURN
*
     End of SPOTRF2
*
     END
```

———-

71.11.4 lisp code

— spotrf2.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
```

```
(defun *main* ()
  (prog ((n1 0) (n2 0) (iinfo 0) (upper nil)
         (uplo
          (make-array '(1) :element-type 'character :initial-element #\ ))
         (info 0) (lda 0)
         (a
          (make-array (the fixnum (reduce #'* (list lda *))) :element-type
                      'single-float))
         (n 0) (subroutine 0.0))
    (declare (type (single-float) subroutine)
             (type (array single-float (*)) a)
             (type (f2cl-lib:integer4) n lda info n1 n2 iinfo)
             (type f2cl-lib:logical upper)
             (type (string 1) uplo))
    '"****NOT TRANSLATED: (RECURSIVE SUBROUTINE SPOTRF2 (UPLO |,| N |,| A |,| LDA |,| INFO))"
    (setf info 0)
    (setf upper
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
    (cond
     ((and (not upper)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
      (setf info -1))
     ((< n 0) (setf info -2))
     ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -4)))
    (cond
     ((/= info 0) (xerbla "SPOTRF2" (f2cl-lib:int-sub info)) (go end_label)))
    (if (= n 0)
        (go end_label))
    (cond
     ((= n 1)
      (cond
       ((or (<= (f2cl-lib:fref a (1 1) ((1 lda) (1 *))) zero)
            (sisnan (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))))
        (setf info 1) (go end_label)))
      (setf (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))
              (f2cl-lib:fsqrt (f2cl-lib:fref a (1 1) ((1 lda) (1 *))))))
     (t (setf n1 (the f2cl-lib:integer4 (truncate n 2)))
      (setf n2 (f2cl-lib:int-sub n n1))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (spotrf2 uplo n1
```

```
      (f2cl-lib:array-slice a single-float (1 1) ((1 lda) (1 *))) lda
      iinfo)
  (declare (ignore var-2))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n1 var-1))
  (when var-3 (setf lda var-3))
  (when var-4 (setf iinfo var-4)))
(cond ((/= iinfo 0) (setf info iinfo) (go end_label)))
(cond
 (upper
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (strsm "L" "U" "T" "N" n1 n2 one
       (f2cl-lib:array-slice a single-float (1 1) ((1 lda) (1 *))) lda
       (f2cl-lib:array-slice a single-float (1 (f2cl-lib:int-add n1 1))
                             ((1 lda) (1 *)))
       lda)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n1 var-4))
    (when var-5 (setf n2 var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf lda var-10)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (ssyrk uplo "T" n2 n1 (- one)
       (f2cl-lib:array-slice a single-float (1 (f2cl-lib:int-add n1 1))
                             ((1 lda) (1 *)))
       lda one
       (f2cl-lib:array-slice a single-float
                             ((+ n1 1) (f2cl-lib:int-add n1 1))
                             ((1 lda) (1 *)))
       lda)
    (declare (ignore var-1 var-4 var-5 var-8))
    (when var-0 (setf uplo var-0))
    (when var-2 (setf n2 var-2))
    (when var-3 (setf n1 var-3))
    (when var-6 (setf lda var-6))
    (when var-7 (setf one var-7))
    (when var-9 (setf lda var-9)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (spotrf2 uplo n2
       (f2cl-lib:array-slice a single-float
                             ((+ n1 1) (f2cl-lib:int-add n1 1))
                             ((1 lda) (1 *)))
       lda iinfo)
    (declare (ignore var-2))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n2 var-1))
```

```
    (when var-3 (setf lda var-3))
    (when var-4 (setf iinfo var-4)))
  (cond
   ((/= iinfo 0) (setf info (f2cl-lib:int-add iinfo n1))
    (go end_label))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (strsm "R" "L" "T" "N" n2 n1 one
       (f2cl-lib:array-slice a single-float (1 1) ((1 lda) (1 *))) lda
       (f2cl-lib:array-slice a single-float ((+ n1 1) 1)
                             ((1 lda) (1 *)))
       lda)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n2 var-4))
    (when var-5 (setf n1 var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf lda var-10)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (ssyrk uplo "N" n2 n1 (- one)
       (f2cl-lib:array-slice a single-float ((+ n1 1) 1)
                             ((1 lda) (1 *)))
       lda one
       (f2cl-lib:array-slice a single-float
                             ((+ n1 1) (f2cl-lib:int-add n1 1))
                             ((1 lda) (1 *)))
       lda)
    (declare (ignore var-1 var-4 var-5 var-8))
    (when var-0 (setf uplo var-0))
    (when var-2 (setf n2 var-2))
    (when var-3 (setf n1 var-3))
    (when var-6 (setf lda var-6))
    (when var-7 (setf one var-7))
    (when var-9 (setf lda var-9)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (spotrf2 uplo n2
       (f2cl-lib:array-slice a single-float
                             ((+ n1 1) (f2cl-lib:int-add n1 1))
                             ((1 lda) (1 *)))
       lda iinfo)
    (declare (ignore var-2))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n2 var-1))
    (when var-3 (setf lda var-3))
    (when var-4 (setf iinfo var-4)))
  (cond
   ((/= iinfo 0) (setf info (f2cl-lib:int-add iinfo n1))
```

```
            (go end_label)))))))
      (go end_label)
     end_label
      (return nil))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::*main*
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types 'nil :return-values 'nil
                                            :calls 'nil)))
```

———-

71.12 spotri

71.12.1 Axiom unit tests

— **spotri.input** —

```
)set break resume
)sys rm -f spotri.output
)spool spotri.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

71.12.2 Axiom help page

— **spotri.help** —

```
====================================================================
spotri examples
====================================================================

====================================================================
```

```
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine spotri (UPLO, N, A, LDA, INFO)
           SPOTRI

Function/Subroutine Documentation
   subroutine spotri (character UPLO, integer N, real, dimension( lda, * ) A,
       integer LDA, integer INFO)
       SPOTRI

Purpose:
   SPOTRI computes the inverse of a real symmetric positive definite
   matrix A using the Cholesky factorization A = U**T*U or A = L*L**T
   computed by SPOTRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   On entry, the triangular factor U or L from the Cholesky
   factorization A = U**T*U or A = L*L**T, as computed by SPOTRF.
   On exit, the upper or lower triangle of the (symmetric)
   inverse of A, overwriting the input factor U or L.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, the (i,i) element of the factor U or L is
         zero, and the inverse could not be computed.
```

71.12.3 fortran code

— spotri.f —

```
*  =====================================================================
      SUBROUTINE SPOTRI( UPLO, N, A, LDA, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLAUUM, STRTRI, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SPOTRI', -INFO )
         RETURN
      END IF
*
```

```
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Invert the triangular Cholesky factor U or L.
*
      CALL STRTRI( UPLO, 'Non-unit', N, A, LDA, INFO )
      IF( INFO.GT.0 )
     $   RETURN
*
*     Form inv(U) * inv(U)**T or inv(L)**T * inv(L).
*
      CALL SLAUUM( UPLO, N, A, LDA, INFO )
*
      RETURN
*
*     End of SPOTRI
*
      END
```

71.12.4 lisp code

— spotri.lisp —

```
(defun spotri (uplo n a lda info)
  (declare (type (array single-float (*)) a)
           (type (f2cl-lib:integer4) info lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a single-float a-%data% a-%offset%))
    (prog ()
      (declare)
      (setf info 0)
      (cond
       ((and
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
```

```
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -4)))
      (cond
       ((/= info 0) (xerbla "SPOTRI" (f2cl-lib:int-sub info)) (go end_label)))
      (if (= n 0)
          (go end_label))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (strtri uplo "Non-unit" n a lda info)
        (declare (ignore var-1 var-3))
        (when var-0 (setf uplo var-0))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-5 (setf info var-5)))
      (if (> info 0)
          (go end_label))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (slauum uplo n a lda info)
        (declare (ignore var-2))
        (setf uplo var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf info var-4))
      (go end_label)
     end_label
      (return (values uplo n nil lda info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::spotri
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::slauum))))
```

———-

71.13 spotrs

71.13.1 Axiom unit tests

— **spotrs.input** —

```
)set break resume
)sys rm -f spotrs.output
)spool spotrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

71.13.2 Axiom help page

— **spotrs.help** —

```
====================================================================
spotrs examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine spotrs (UPLO, N, NRHS, A, LDA, B, LDB, INFO)
           SPOTRS

Function/Subroutine Documentation
   subroutine spotrs (character UPLO, integer N, integer NRHS, real,
       dimension( lda, * ) A, integer LDA, real, dimension( ldb, * ) B,
       integer LDB, integer INFO)
       SPOTRS

Purpose:
```

```
  SPOTRS solves a system of linear equations A*X = B with a symmetric
  positive definite matrix A using the Cholesky factorization
  A = U**T*U or A = L*L**T computed by SPOTRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is REAL array, dimension (LDA,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**T*U or A = L*L**T, as computed by SPOTRF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 B
   B is REAL array, dimension (LDB,NRHS)
   On entry, the right hand side matrix B.
   On exit, the solution matrix X.

 LDB
   LDB is INTEGER
   The leading dimension of the array B.  LDB >= max(1,N).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

71.13.3 fortran code

— spotrs.f —

```
*  =====================================================================
      SUBROUTINE SPOTRS( UPLO, N, NRHS, A, LDA, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           STRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
```

```
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SPOTRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*        Solve A*X = B where A = U**T *U.
*
*        Solve U**T *X = B, overwriting B with X.
*
         CALL STRSM( 'Left', 'Upper', 'Transpose', 'Non-unit', N, NRHS,
     $               ONE, A, LDA, B, LDB )
*
*        Solve U*X = B, overwriting B with X.
*
         CALL STRSM( 'Left', 'Upper', 'No transpose', 'Non-unit', N,
     $               NRHS, ONE, A, LDA, B, LDB )
      ELSE
*
*        Solve A*X = B where A = L*L**T.
*
*        Solve L*X = B, overwriting B with X.
*
         CALL STRSM( 'Left', 'Lower', 'No transpose', 'Non-unit', N,
     $               NRHS, ONE, A, LDA, B, LDB )
*
*        Solve L**T *X = B, overwriting B with X.
*
         CALL STRSM( 'Left', 'Lower', 'Transpose', 'Non-unit', N, NRHS,
     $               ONE, A, LDA, B, LDB )
      END IF
*
      RETURN
*
*     End of SPOTRS
*
      END
```

———-

71.13.4 lisp code

— spotrs.lisp —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun spotrs (uplo n nrhs a lda b ldb$ info)
    (declare (type (array single-float (*)) b a)
             (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a single-float a-%data% a-%offset%)
         (b single-float b-%data% b-%offset%))
      (prog ((upper nil))
        (declare (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -5))
         ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -7)))
        (cond
         ((/= info 0) (xerbla "SPOTRS" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (or (= n 0) (= nrhs 0))
            (go end_label))
        (cond
         (upper
          (multiple-value-bind
```

```
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10)
              (strsm "Left" "Upper" "Transpose" "Non-unit" n nrhs one a lda b
               ldb$)
            (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
            (when var-4 (setf n var-4))
            (when var-5 (setf nrhs var-5))
            (when var-6 (setf one var-6))
            (when var-8 (setf lda var-8))
            (when var-10 (setf ldb$ var-10)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10)
              (strsm "Left" "Upper" "No transpose" "Non-unit" n nrhs one a lda
               b ldb$)
            (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
            (when var-4 (setf n var-4))
            (when var-5 (setf nrhs var-5))
            (when var-6 (setf one var-6))
            (when var-8 (setf lda var-8))
            (when var-10 (setf ldb$ var-10))))
         (t
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10)
              (strsm "Left" "Lower" "No transpose" "Non-unit" n nrhs one a lda
               b ldb$)
            (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
            (when var-4 (setf n var-4))
            (when var-5 (setf nrhs var-5))
            (when var-6 (setf one var-6))
            (when var-8 (setf lda var-8))
            (when var-10 (setf ldb$ var-10)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10)
              (strsm "Left" "Lower" "Transpose" "Non-unit" n nrhs one a lda b
               ldb$)
            (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
            (when var-4 (setf n var-4))
            (when var-5 (setf nrhs var-5))
            (when var-6 (setf one var-6))
            (when var-8 (setf lda var-8))
            (when var-10 (setf ldb$ var-10)))))
        (go end_label)
       end_label
        (return (values uplo n nrhs nil lda nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
```

```
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::spotrs
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((string)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(fortran-to-lisp::uplo
                                             fortran-to-lisp::n
                                             fortran-to-lisp::nrhs nil
                                             fortran-to-lisp::lda nil
                                             fortran-to-lisp::ldb$
                                             fortran-to-lisp::info)
                                           :calls 'nil)))
```

———-

Chapter 72

LAPACK - Positive Definite Matrix, Comp. Routines, Complex

72.1 cla_porcond_c

72.1.1 Axiom unit tests

— **claporcondc.input** —

```
)set break resume
)sys rm -f claporcondc.output
)spool claporcondc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

72.1.2 Axiom help page

— **cla_porcond_c.help** —

```
==========================================================================
```

```
cla_porcond_c examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       real function cla_porcond_c (UPLO, N, A, LDA, AF, LDAF, C, CAPPLY,
           INFO, WORK, RWORK)
           CLA_PORCOND_C computes the infinity norm condition number of
           op(A)*inv(diag(c)) for Hermitian positive-definite matrices.

Function/Subroutine Documentation
   real function cla_porcond_c (character UPLO, integer N, complex, dimension(
       lda, * ) A, integer LDA, complex, dimension( ldaf, * ) AF, integer
       LDAF, real, dimension( * ) C, logical CAPPLY, integer INFO, complex,
       dimension( * ) WORK, real, dimension( * ) RWORK)
       CLA_PORCOND_C computes the infinity norm condition number of
       op(A)*inv(diag(c)) for Hermitian positive-definite matrices.

Purpose:
   CLA_PORCOND_C Computes the infinity norm condition number of
   op(A) * inv(diag(C)) where C is a DOUBLE PRECISION vector

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the N-by-N matrix A

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is COMPLEX array, dimension (LDAF,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**H*U or A = L*L**H, as computed by CPOTRF.
```

```
 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 C
   C is REAL array, dimension (N)
   The vector C in the formula op(A) * inv(diag(C)).

 CAPPLY
   CAPPLY is LOGICAL
   If .TRUE. then access the vector C in the formula above.

 INFO
   INFO is INTEGER
   = 0:  Successful exit.
   i > 0:  The ith argument is invalid.

 WORK
   WORK is COMPLEX array, dimension (2*N).
   Workspace.

 RWORK
   RWORK is REAL array, dimension (N).
   Workspace.
```

———-

72.1.3 fortran code

— cla_porcond_c.f —

```
*  =====================================================================
      REAL FUNCTION CLA_PORCOND_C( UPLO, N, A, LDA, AF, LDAF, C, CAPPLY,
     $                             INFO, WORK, RWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      LOGICAL            CAPPLY
      INTEGER            N, LDA, LDAF, INFO
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), AF( LDAF, * ), WORK( * )
```

```
      REAL               C( * ), RWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            KASE
      REAL               AINVNM, ANORM, TMP
      INTEGER            I, J
      LOGICAL            UP, UPPER
      COMPLEX            ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLACN2, CPOTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, REAL, AIMAG
*     ..
*     .. Statement Functions ..
      REAL CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      CLA_PORCOND_C = 0.0E+0
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CLA_PORCOND_C', -INFO )
         RETURN
      END IF
```

```
      UP = .FALSE.
      IF ( LSAME( UPLO, 'U' ) ) UP = .TRUE.
*
*     Compute norm of op(A)*op2(C).
*
      ANORM = 0.0E+0
      IF ( UP ) THEN
         DO I = 1, N
            TMP = 0.0E+0
            IF ( CAPPLY ) THEN
               DO J = 1, I
                  TMP = TMP + CABS1( A( J, I ) ) / C( J )
               END DO
               DO J = I+1, N
                  TMP = TMP + CABS1( A( I, J ) ) / C( J )
               END DO
            ELSE
               DO J = 1, I
                  TMP = TMP + CABS1( A( J, I ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + CABS1( A( I, J ) )
               END DO
            END IF
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      ELSE
         DO I = 1, N
            TMP = 0.0E+0
            IF ( CAPPLY ) THEN
               DO J = 1, I
                  TMP = TMP + CABS1( A( I, J ) ) / C( J )
               END DO
               DO J = I+1, N
                  TMP = TMP + CABS1( A( J, I ) ) / C( J )
               END DO
            ELSE
               DO J = 1, I
                  TMP = TMP + CABS1( A( I, J ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + CABS1( A( J, I ) )
               END DO
            END IF
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      END IF
*
```

```
*     Quick return if possible.
*
      IF( N.EQ.0 ) THEN
         CLA_PORCOND_C = 1.0E+0
         RETURN
      ELSE IF( ANORM .EQ. 0.0E+0 ) THEN
         RETURN
      END IF
*
*     Estimate the norm of inv(op(A)).
*
      AINVNM = 0.0E+0
*
      KASE = 0
   10 CONTINUE
      CALL CLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
*
            IF ( UP ) THEN
               CALL CPOTRS( 'U', N, 1, AF, LDAF,
     $            WORK, N, INFO )
            ELSE
               CALL CPOTRS( 'L', N, 1, AF, LDAF,
     $            WORK, N, INFO )
            ENDIF
*
*           Multiply by inv(C).
*
            IF ( CAPPLY ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF
         ELSE
*
*           Multiply by inv(C**H).
*
            IF ( CAPPLY ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF
*
```

```
            IF ( UP ) THEN
               CALL CPOTRS( 'U', N, 1, AF, LDAF,
     $            WORK, N, INFO )
            ELSE
               CALL CPOTRS( 'L', N, 1, AF, LDAF,
     $            WORK, N, INFO )
            END IF
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM .NE. 0.0E+0 )
     $   CLA_PORCOND_C = 1.0E+0 / AINVNM
*
      RETURN
*
      END
```

———-

72.1.4 lisp code

— cla_porcond_c.lisp —

```
(defun cla_porcond_c (uplo n a lda af ldaf c capply info work rwork)
  (declare (type f2cl-lib:logical capply)
           (type (array single-float (*)) rwork c)
           (type (array f2cl-lib:complex8 (*)) work af a)
           (type (f2cl-lib:integer4) info ldaf lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (af f2cl-lib:complex8 af-%data% af-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%)
       (c single-float c-%data% c-%offset%)
       (rwork single-float rwork-%data% rwork-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
      (declare
```

```
 (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
  cabs1))
(prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
       (zdum #C(0.0 0.0)) (up nil) (upper nil) (i 0) (j 0) (ainvnm 0.0)
       (anorm 0.0) (tmp 0.0) (kase 0) (cla_porcond_c 0.0))
  (declare (type (single-float) cla_porcond_c tmp anorm ainvnm)
           (type (f2cl-lib:integer4) kase j i)
           (type f2cl-lib:logical upper up)
           (type (f2cl-lib:complex8) zdum)
           (type (array f2cl-lib:integer4 (3)) isave))
  (setf cla_porcond_c 0.0)
  (setf info 0)
  (setf upper
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
  (cond
   ((and (not upper)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4))
   ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -6)))
  (cond
   ((/= info 0) (xerbla "CLA_PORCOND_C" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf up f2cl-lib:%false%)
  (if (multiple-value-bind (ret-val var-0 var-1)
          (lsame uplo "U")
        (declare (ignore var-1))
        (when var-0 (setf uplo var-0))
        ret-val)
      (setf up f2cl-lib:%true%))
  (setf anorm 0.0)
  (cond
   (up
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf tmp 0.0)
        (cond
```

```
          (capply
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j i) nil)
             (tagbody
               (setf tmp
                       (+ tmp
                          (/
                           (cabs1
                            (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                           a-%offset%))
                           (f2cl-lib:fref c-%data% (j) ((1 *))
                                          c-%offset%))))
              label100001))
           (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (setf tmp
                       (+ tmp
                          (/
                           (cabs1
                            (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                           a-%offset%))
                           (f2cl-lib:fref c-%data% (j) ((1 *))
                                          c-%offset%))))
              label100002)))
          (t
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j i) nil)
             (tagbody
               (setf tmp
                       (+ tmp
                          (cabs1
                           (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                          a-%offset%))))
              label100003))
           (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (setf tmp
                       (+ tmp
                          (cabs1
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%))))
              label100004))))
         (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                 tmp)
         (setf anorm (max anorm tmp))
        label100000)))
      (t
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
            ((> i n) nil)
(tagbody
  (setf tmp 0.0)
  (cond
   (capply
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j i) nil)
      (tagbody
        (setf tmp
                (+ tmp
                   (/
                    (cabs1
                     (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%))
                    (f2cl-lib:fref c-%data% (j) ((1 *))
                                   c-%offset%))))
       label100006))
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf tmp
                (+ tmp
                   (/
                    (cabs1
                     (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                    a-%offset%))
                    (f2cl-lib:fref c-%data% (j) ((1 *))
                                   c-%offset%))))
       label100007)))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j i) nil)
      (tagbody
        (setf tmp
                (+ tmp
                   (cabs1
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))))
       label100008))
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf tmp
                (+ tmp
                   (cabs1
                    (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                   a-%offset%))))
       label100009))))
  (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
        tmp)
```

```
        (setf anorm (max anorm tmp))
       label100005))))
  (cond ((= n 0) (setf cla_porcond_c 1.0) (go end_label))
        ((= anorm 0.0) (go end_label)))
  (setf ainvnm 0.0)
  (setf kase 0)
 label10
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (clacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ n 1))
                             ((1 *)) work-%offset%)
       work ainvnm kase isave)
    (declare (ignore var-1 var-2 var-5))
    (setf n var-0)
    (setf ainvnm var-3)
    (setf kase var-4))
  (cond
   ((/= kase 0)
    (cond
     ((= kase 2)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (*
                   (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                   (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                  rwork-%offset%)))
         label100010))
      (cond
       (up
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (cpotrs "U" n 1 af ldaf work n info)
          (declare (ignore var-0 var-2 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-4 (setf ldaf var-4))
          (when var-6 (setf n var-6))
          (when var-7 (setf info var-7))))
       (t
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (cpotrs "L" n 1 af ldaf work n info)
          (declare (ignore var-0 var-2 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-4 (setf ldaf var-4))
          (when var-6 (setf n var-6))
          (when var-7 (setf info var-7)))))
      (cond
       (capply
```

```
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (*
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                   (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
         label100011)))))
 (t
  (cond
   (capply
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (*
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
         label100012))))
  (cond
   (up
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (cpotrs "U" n 1 af ldaf work n info)
      (declare (ignore var-0 var-2 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-4 (setf ldaf var-4))
      (when var-6 (setf n var-6))
      (when var-7 (setf info var-7))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (cpotrs "L" n 1 af ldaf work n info)
      (declare (ignore var-0 var-2 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-4 (setf ldaf var-4))
      (when var-6 (setf n var-6))
      (when var-7 (setf info var-7)))))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (*
               (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               (f2cl-lib:fref rwork-%data% (i) ((1 *))
                              rwork-%offset%)))
     label100013))))
(go label10)))
```

```
        (if (/= ainvnm 0.0)
            (setf cla_porcond_c (/ 1.0 ainvnm)))
        (go end_label)
       end_label
        (return
         (values cla_porcond_c uplo n nil nil nil ldaf nil nil info nil
                 nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cla_porcond_c
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              fortran-to-lisp::logical
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array single-float (*)))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::ldaf nil nil
                                              fortran-to-lisp::info nil nil)
                                            :calls '(fortran-to-lisp::clacn2))))
```

———-

72.2 cla_porcond_x

72.2.1 Axiom unit tests

— claporcondx.input —

```
)set break resume
)sys rm -f claporcondx.output
```

```
)spool claporcondx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

72.2.2 Axiom help page

— cla_porcond_x.help —

```
=====================================================================
cla_porcond_x examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       real function cla_porcond_x (UPLO, N, A, LDA, AF, LDAF, X, INFO, WORK,
           RWORK)
           CLA_PORCOND_X computes the infinity norm condition number of
           op(A)*diag(x) for Hermitian positive-definite matrices.

Function/Subroutine Documentation
   real function cla_porcond_x (character UPLO, integer N, complex, dimension(
       lda, * ) A, integer LDA, complex, dimension( ldaf, * ) AF, integer
       LDAF, complex, dimension( * ) X, integer INFO, complex, dimension( * )
       WORK, real, dimension( * ) RWORK)
       CLA_PORCOND_X computes the infinity norm condition number of
       op(A)*diag(x) for Hermitian positive-definite matrices.

Purpose:
   CLA_PORCOND_X Computes the infinity norm condition number of
   op(A) * diag(X) where X is a COMPLEX vector.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.
```

```
N
  N is INTEGER
  The number of linear equations, i.e., the order of the matrix A.  N >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the N-by-N matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX array, dimension (LDAF,N)
  The triangular factor U or L from the Cholesky factorization
  A = U**H*U or A = L*L**H, as computed by CPOTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

X
  X is COMPLEX array, dimension (N)
  The vector X in the formula op(A) * diag(X).

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  i > 0:  The ith argument is invalid.

WORK
  WORK is COMPLEX array, dimension (2*N).
  Workspace.

RWORK
  RWORK is REAL array, dimension (N).
  Workspace.
```

———-

72.2.3 fortran code

— cla_porcond_x.f —

```
*  =====================================================================
      REAL FUNCTION CLA_PORCOND_X( UPLO, N, A, LDA, AF, LDAF, X, INFO,
     $                             WORK, RWORK )
```

```
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            N, LDA, LDAF, INFO
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), AF( LDAF, * ), WORK( * ), X( * )
      REAL               RWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            KASE, I, J
      REAL               AINVNM, ANORM, TMP
      LOGICAL            UP, UPPER
      COMPLEX            ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLACN2, CPOTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, REAL, AIMAG
*     ..
*     .. Statement Functions ..
      REAL CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      CLA_PORCOND_X = 0.0E+0
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
```

```
      ELSE IF ( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CLA_PORCOND_X', -INFO )
         RETURN
      END IF
      UP = .FALSE.
      IF ( LSAME( UPLO, 'U' ) ) UP = .TRUE.
*
*     Compute norm of op(A)*op2(C).
*
      ANORM = 0.0
      IF ( UP ) THEN
         DO I = 1, N
            TMP = 0.0E+0
            DO J = 1, I
               TMP = TMP + CABS1( A( J, I ) * X( J ) )
            END DO
            DO J = I+1, N
               TMP = TMP + CABS1( A( I, J ) * X( J ) )
            END DO
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      ELSE
         DO I = 1, N
            TMP = 0.0E+0
            DO J = 1, I
               TMP = TMP + CABS1( A( I, J ) * X( J ) )
            END DO
            DO J = I+1, N
               TMP = TMP + CABS1( A( J, I ) * X( J ) )
            END DO
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 ) THEN
         CLA_PORCOND_X = 1.0E+0
         RETURN
      ELSE IF( ANORM .EQ. 0.0E+0 ) THEN
         RETURN
```

```
      END IF
*
*     Estimate the norm of inv(op(A)).
*
      AINVNM = 0.0E+0
*
      KASE = 0
   10 CONTINUE
      CALL CLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
*
            IF ( UP ) THEN
               CALL CPOTRS( 'U', N, 1, AF, LDAF,
     $            WORK, N, INFO )
            ELSE
               CALL CPOTRS( 'L', N, 1, AF, LDAF,
     $            WORK, N, INFO )
            ENDIF
*
*           Multiply by inv(X).
*
            DO I = 1, N
               WORK( I ) = WORK( I ) / X( I )
            END DO
         ELSE
*
*           Multiply by inv(X**H).
*
            DO I = 1, N
               WORK( I ) = WORK( I ) / X( I )
            END DO
*
            IF ( UP ) THEN
               CALL CPOTRS( 'U', N, 1, AF, LDAF,
     $            WORK, N, INFO )
            ELSE
               CALL CPOTRS( 'L', N, 1, AF, LDAF,
     $            WORK, N, INFO )
            END IF
*
*           Multiply by R.
*
            DO I = 1, N
```

```
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM .NE. 0.0E+0 )
     $   CLA_PORCOND_X = 1.0E+0 / AINVNM
*
      RETURN
*
      END
```

———-

72.2.4 lisp code

— cla_porcond_x.lisp —

```
(defun cla_porcond_x (uplo n a lda af ldaf x info work rwork)
  (declare (type (array single-float (*)) rwork)
           (type (array f2cl-lib:complex8 (*)) work x af a)
           (type (f2cl-lib:integer4) info ldaf lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (af f2cl-lib:complex8 af-%data% af-%offset%)
       (x f2cl-lib:complex8 x-%data% x-%offset%)
       (work f2cl-lib:complex8 work-%data% work-%offset%)
       (rwork single-float rwork-%data% rwork-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
        cabs1))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (zdum #C(0.0 0.0)) (up nil) (upper nil) (ainvnm 0.0) (anorm 0.0)
             (tmp 0.0) (kase 0) (i 0) (j 0) (cla_porcond_x 0.0))
        (declare (type (f2cl-lib:integer4) j i kase)
                 (type (single-float) cla_porcond_x tmp anorm ainvnm)
                 (type f2cl-lib:logical upper up)
                 (type (f2cl-lib:complex8) zdum)
                 (type (array f2cl-lib:integer4 (3)) isave))
        (setf cla_porcond_x 0.0)
        (setf info 0)
```

```
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -6)))
(cond
 ((/= info 0) (xerbla "CLA_PORCOND_X" (f2cl-lib:int-sub info))
  (go end_label)))
(setf up f2cl-lib:%false%)
(if (multiple-value-bind (ret-val var-0 var-1)
        (lsame uplo "U")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)
    (setf up f2cl-lib:%true%))
(setf anorm 0.0)
(cond
 (up
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf tmp 0.0)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j i) nil)
        (tagbody
          (setf tmp
                  (+ tmp
                     (cabs1
                      (*
                       (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                      a-%offset%)
                       (f2cl-lib:fref x-%data% (j) ((1 *))
                                      x-%offset%)))))
         label100001))
      (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
```

```
                    ((> j n) nil)
            (tagbody
              (setf tmp
                      (+ tmp
                         (cabs1
                          (*
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%)
                           (f2cl-lib:fref x-%data% (j) ((1 *))
                                          x-%offset%)))))
             label100002))
          (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                  tmp)
          (setf anorm (max anorm tmp))
         label100000)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf tmp 0.0)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j i) nil)
            (tagbody
              (setf tmp
                      (+ tmp
                         (cabs1
                          (*
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%)
                           (f2cl-lib:fref x-%data% (j) ((1 *))
                                          x-%offset%)))))
             label100004))
          (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf tmp
                      (+ tmp
                         (cabs1
                          (*
                           (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                          a-%offset%)
                           (f2cl-lib:fref x-%data% (j) ((1 *))
                                          x-%offset%)))))
             label100005))
          (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                  tmp)
          (setf anorm (max anorm tmp))
         label100003))))
    (cond ((= n 0) (setf cla_porcond_x 1.0) (go end_label))
          ((= anorm 0.0) (go end_label)))
```

```
 (setf ainvnm 0.0)
 (setf kase 0)
label10
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
     (clacn2 n
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ n 1))
                            ((1 *)) work-%offset%)
      work ainvnm kase isave)
   (declare (ignore var-1 var-2 var-5))
   (setf n var-0)
   (setf ainvnm var-3)
   (setf kase var-4))
 (cond
  ((/= kase 0)
   (cond
    ((= kase 2)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (*
                  (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                 rwork-%offset%)))
        label100006))
     (cond
      (up
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
           (cpotrs "U" n 1 af ldaf work n info)
         (declare (ignore var-0 var-2 var-3 var-5))
         (when var-1 (setf n var-1))
         (when var-4 (setf ldaf var-4))
         (when var-6 (setf n var-6))
         (when var-7 (setf info var-7))))
      (t
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
           (cpotrs "L" n 1 af ldaf work n info)
         (declare (ignore var-0 var-2 var-3 var-5))
         (when var-1 (setf n var-1))
         (when var-4 (setf ldaf var-4))
         (when var-6 (setf n var-6))
         (when var-7 (setf info var-7)))))
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (/
                  (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
```

```
                    (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)))
            label100007)))
          (t
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                       (/
                        (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                        (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)))
            label100008))
           (cond
            (up
             (multiple-value-bind
                 (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
                 (cpotrs "U" n 1 af ldaf work n info)
               (declare (ignore var-0 var-2 var-3 var-5))
               (when var-1 (setf n var-1))
               (when var-4 (setf ldaf var-4))
               (when var-6 (setf n var-6))
               (when var-7 (setf info var-7))))
            (t
             (multiple-value-bind
                 (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
                 (cpotrs "L" n 1 af ldaf work n info)
               (declare (ignore var-0 var-2 var-3 var-5))
               (when var-1 (setf n var-1))
               (when var-4 (setf ldaf var-4))
               (when var-6 (setf n var-6))
               (when var-7 (setf info var-7)))))
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                       (*
                        (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                        (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                       rwork-%offset%)))
            label100009))))
         (go label10)))
       (if (/= ainvnm 0.0)
           (setf cla_porcond_x (/ 1.0 ainvnm)))
       (go end_label)
      end_label
       (return
        (values cla_porcond_x uplo n nil nil nil ldaf nil info nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
```

```
(setf (gethash 'fortran-to-lisp::cla_porcond_x
               fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((string)
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (array single-float (*)))
                                          :return-values
                                          '(fortran-to-lisp::uplo
                                            fortran-to-lisp::n nil nil nil
                                            fortran-to-lisp::ldaf nil
                                            fortran-to-lisp::info nil nil)
                                          :calls '(fortran-to-lisp::clacn2))))
```

———-

72.3 cla_porfsx_extended

72.3.1 Axiom unit tests

— **claporfsxextended.input** —

```
)set break resume
)sys rm -f claporfsxextended.output
)spool claporfsxextended.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

72.3.2 Axiom help page [Demm05]

— cla_porfsx_extended.help —

```
==================================================================
cla_porfsx_extended examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cla_porfsx_extended (PREC_TYPE, UPLO, N, NRHS, A, LDA, AF,
           LDAF, COLEQU, C, B, LDB, Y, LDY, BERR_OUT, N_NORMS, ERR_BNDS_NORM,
           ERR_BNDS_COMP, RES, AYB, DY, Y_TAIL, RCOND, ITHRESH, RTHRESH,
           DZ_UB, IGNORE_CWISE, INFO)
           CLA_PORFSX_EXTENDED improves the computed solution to a system of
           linear equations for symmetric or Hermitian positive-definite
           matrices by performing extra-precise iterative refinement and
           provides error bounds and backward error estimates for the
           solution.

Function/Subroutine Documentation
   subroutine cla_porfsx_extended (integer PREC_TYPE, character UPLO, integer
       N, integer NRHS, complex, dimension( lda, * ) A, integer LDA, complex,
       dimension( ldaf, * ) AF, integer LDAF, logical COLEQU, real, dimension(
       * ) C, complex, dimension( ldb, * ) B, integer LDB, complex, dimension(
       ldy, * ) Y, integer LDY, real, dimension( * ) BERR_OUT, integer
       N_NORMS, real, dimension( nrhs, * ) ERR_BNDS_NORM, real, dimension(
       nrhs, * ) ERR_BNDS_COMP, complex, dimension( * ) RES, real, dimension(
       * ) AYB, complex, dimension( * ) DY, complex, dimension( * ) Y_TAIL,
       real RCOND, integer ITHRESH, real RTHRESH, real DZ_UB, logical
       IGNORE_CWISE, integer INFO)
       CLA_PORFSX_EXTENDED improves the computed solution to a system of
       linear equations for symmetric or Hermitian positive-definite matrices
       by performing extra-precise iterative refinement and provides error
       bounds and backward error estimates for the solution.

Purpose:
   CLA_PORFSX_EXTENDED improves the computed solution to a system of
   linear equations by performing extra-precise iterative refinement
   and provides error bounds and backward error estimates for the solution.
   This subroutine is called by CPORFSX to perform iterative refinement.
   In addition to normwise error bound, the code provides maximum
   componentwise error bound if possible. See comments for ERR_BNDS_NORM
   and ERR_BNDS_COMP for details of the error bounds. Note that this
   subroutine is only resonsible for setting the second fields of
```

```
   ERR_BNDS_NORM and ERR_BNDS_COMP.

Parameters:

 PREC_TYPE
   PREC_TYPE is INTEGER
   Specifies the intermediate precision to be used in refinement.
   The value is defined by ILAPREC(P) where P is a CHARACTER and
   P    = 'S':  Single
        = 'D':  Double
        = 'I':  Indigenous
        = 'X', 'E':  Extra

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right-hand-sides, i.e., the number of columns of the
   matrix B.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is COMPLEX array, dimension (LDAF,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**T*U or A = L*L**T, as computed by CPOTRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 COLEQU
   COLEQU is LOGICAL
   If .TRUE. then column equilibration was done to A before calling
   this routine. This is needed to compute the solution and error
   bounds correctly.
```

```
C
  C is REAL array, dimension (N)
  The column scale factors for A. If COLEQU = .FALSE., C
  is not accessed. If C is input, each element of C should be a power
  of the radix to ensure a reliable solution and error estimates.
  Scaling by powers of the radix does not cause rounding errors unless
  the result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  The right-hand-side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

Y
  Y is COMPLEX array, dimension (LDY,NRHS)
  On entry, the solution matrix X, as computed by CPOTRS.
  On exit, the improved solution matrix Y.

LDY
  LDY is INTEGER
  The leading dimension of the array Y.  LDY >= max(1,N).

BERR_OUT
  BERR_OUT is REAL array, dimension (NRHS)
  On exit, BERR_OUT(j) contains the componentwise relative backward
  error for right-hand-side j from the formula
  max(i) ( abs(RES(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
  where abs(Z) is the componentwise absolute value of the matrix
  or vector Z. This is computed by CLA_LIN_BERR.

N_NORMS
  N_NORMS is INTEGER
  Determines which error bounds to return (see ERR_BNDS_NORM
  and ERR_BNDS_COMP).
  If N_NORMS >= 1 return normwise error bounds.
  If N_NORMS >= 2 return componentwise error bounds.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
          max_j (abs(XTRUE(j,i) - X(j,i)))
```

```
          ------------------------------
                 max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
    reciprocal condition number is less than the threshold
    sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
    almost certainly within a factor of 10 of the true error
    so long as the next entry is greater than the threshold
    sqrt(n) * slamch('Epsilon'). This error bound should only
    be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
    reciprocal condition number.  Compared with the threshold
    sqrt(n) * slamch('Epsilon') to determine if the error
    estimate is "guaranteed". These reciprocal condition
    numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
    appropriately scaled matrix Z.
    Let Z = S*A, where S scales each row by a power of the
    radix so all absolute row sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.


ERR_BNDS_COMP
  ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                  abs(XTRUE(j,i) - X(j,i))
            max_j ----------------------
                        abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
```

```
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
    reciprocal condition number is less than the threshold
    sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
    almost certainly within a factor of 10 of the true error
    so long as the next entry is greater than the threshold
    sqrt(n) * slamch('Epsilon'). This error bound should only
    be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
    reciprocal condition number.  Compared with the threshold
    sqrt(n) * slamch('Epsilon') to determine if the error
    estimate is "guaranteed". These reciprocal condition
    numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
    appropriately scaled matrix Z.
    Let Z = S*(A*diag(x)), where x is the solution for the
    current right-hand side and S scales each row of
    A*diag(x) by a power of the radix so all absolute row
    sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

RES
  RES is COMPLEX array, dimension (N)
  Workspace to hold the intermediate residual.

AYB
  AYB is REAL array, dimension (N)
  Workspace.

DY
  DY is COMPLEX array, dimension (N)
  Workspace to hold the intermediate solution.

Y_TAIL
  Y_TAIL is COMPLEX array, dimension (N)
  Workspace to hold the trailing bits of the intermediate solution.

RCOND
```

```
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

ITHRESH
  ITHRESH is INTEGER
  The maximum number of residual computations allowed for
  refinement. The default is 10. For 'aggressive' set to 100 to
  permit convergence using approximate factorizations or
  factorizations other than LU. If the factorization uses a
  technique other than Gaussian elimination, the guarantees in
  ERR_BNDS_NORM and ERR_BNDS_COMP may no longer be trustworthy.

RTHRESH
  RTHRESH is REAL
  Determines when to stop refinement if the error estimate stops
  decreasing. Refinement will stop when the next solution no longer
  satisfies norm(dx_{i+1}) < RTHRESH * norm(dx_i) where norm(Z) is
  the infinity norm of Z. RTHRESH satisfies 0 < RTHRESH <= 1. The
  default value is 0.5. For 'aggressive' set to 0.9 to permit
  convergence on extremely ill-conditioned matrices. See LAWN 165
  for more details.

DZ_UB
  DZ_UB is REAL
  Determines when to start considering componentwise convergence.
  Componentwise convergence is only considered after each component
  of the solution Y is stable, which we definte as the relative
  change in each component being less than DZ_UB. The default value
  is 0.25, requiring the first bit to be stable. See LAWN 165 for
  more details.

IGNORE_CWISE
  IGNORE_CWISE is LOGICAL
  If .TRUE. then ignore componentwise convergence. Default value
  is .FALSE..

INFO
  INFO is INTEGER
  = 0:  Successful exit.
  < 0:  if INFO = -i, the ith argument to CPOTRS had an illegal value
```

———-

72.3.3 fortran code

— cla_porfsx_extended.f —

```
*  ==================================================================
      SUBROUTINE CLA_PORFSX_EXTENDED( PREC_TYPE, UPLO, N, NRHS, A, LDA,
     $                                AF, LDAF, COLEQU, C, B, LDB, Y,
     $                                LDY, BERR_OUT, N_NORMS,
     $                                ERR_BNDS_NORM, ERR_BNDS_COMP, RES,
     $                                AYB, DY, Y_TAIL, RCOND, ITHRESH,
     $                                RTHRESH, DZ_UB, IGNORE_CWISE,
     $                                INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDAF, LDB, LDY, N, NRHS, PREC_TYPE,
     $                   N_NORMS, ITHRESH
      CHARACTER          UPLO
      LOGICAL            COLEQU, IGNORE_CWISE
      REAL               RTHRESH, DZ_UB
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   Y( LDY, * ), RES( * ), DY( * ), Y_TAIL( * )
      REAL               C( * ), AYB( * ), RCOND, BERR_OUT( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  ==================================================================
*
*     .. Local Scalars ..
      INTEGER            UPLO2, CNT, I, J, X_STATE, Z_STATE,
     $                   Y_PREC_STATE
      REAL               YK, DYK, YMIN, NORMY, NORMX, NORMDX, DXRAT,
     $                   DZRAT, PREVNORMDX, PREV_DZ_Z, DXRATMAX,
     $                   DZRATMAX, DX_X, DZ_Z, FINAL_DX_X, FINAL_DZ_Z,
     $                   EPS, HUGEVAL, INCR_THRESH
      LOGICAL            INCR_PREC
      COMPLEX            ZDUM
*     ..
*     .. Parameters ..
      INTEGER            UNSTABLE_STATE, WORKING_STATE, CONV_STATE,
     $                   NOPROG_STATE, BASE_RESIDUAL, EXTRA_RESIDUAL,
     $                   EXTRA_Y
```

```
      PARAMETER          ( UNSTABLE_STATE = 0, WORKING_STATE = 1,
     $                   CONV_STATE = 2, NOPROG_STATE = 3 )
      PARAMETER          ( BASE_RESIDUAL = 0, EXTRA_RESIDUAL = 1,
     $                   EXTRA_Y = 2 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           ILAUPLO
      INTEGER            ILAUPLO
*     ..
*     .. External Subroutines ..
      EXTERNAL           CAXPY, CCOPY, CPOTRS, CHEMV, BLAS_CHEMV_X,
     $                   BLAS_CHEMV2_X, CLA_HEAMV, CLA_WWADDW,
     $                   CLA_LIN_BERR, SLAMCH
      REAL               SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, REAL, AIMAG, MAX, MIN
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      IF (INFO.NE.0) RETURN
      EPS = SLAMCH( 'Epsilon' )
      HUGEVAL = SLAMCH( 'Overflow' )
*     Force HUGEVAL to Inf
      HUGEVAL = HUGEVAL * HUGEVAL
*     Using HUGEVAL may lead to spurious underflows.
```

```
      INCR_THRESH = REAL(N) * EPS

      IF (LSAME (UPLO, 'L')) THEN
         UPLO2 = ILAUPLO( 'L' )
      ELSE
         UPLO2 = ILAUPLO( 'U' )
      ENDIF

      DO J = 1, NRHS
         Y_PREC_STATE = EXTRA_RESIDUAL
         IF (Y_PREC_STATE .EQ. EXTRA_Y) THEN
            DO I = 1, N
               Y_TAIL( I ) = 0.0
            END DO
         END IF

         DXRAT = 0.0
         DXRATMAX = 0.0
         DZRAT = 0.0
         DZRATMAX = 0.0
         FINAL_DX_X = HUGEVAL
         FINAL_DZ_Z = HUGEVAL
         PREVNORMDX = HUGEVAL
         PREV_DZ_Z = HUGEVAL
         DZ_Z = HUGEVAL
         DX_X = HUGEVAL

         X_STATE = WORKING_STATE
         Z_STATE = UNSTABLE_STATE
         INCR_PREC = .FALSE.

         DO CNT = 1, ITHRESH
*
*         Compute residual RES = B_s - op(A_s) * Y,
*             op(A) = A, A**T, or A**H depending on TRANS (and type).
*
            CALL CCOPY( N, B( 1, J ), 1, RES, 1 )
            IF (Y_PREC_STATE .EQ. BASE_RESIDUAL) THEN
               CALL CHEMV(UPLO, N, CMPLX(-1.0), A, LDA, Y(1,J), 1,
     $              CMPLX(1.0), RES, 1)
            ELSE IF (Y_PREC_STATE .EQ. EXTRA_RESIDUAL) THEN
               CALL BLAS_CHEMV_X(UPLO2, N, CMPLX(-1.0), A, LDA,
     $              Y( 1, J ), 1, CMPLX(1.0), RES, 1, PREC_TYPE)
            ELSE
               CALL BLAS_CHEMV2_X(UPLO2, N, CMPLX(-1.0), A, LDA,
     $              Y(1, J), Y_TAIL, 1, CMPLX(1.0), RES, 1, PREC_TYPE)
            END IF

!         XXX: RES is no longer needed.
            CALL CCOPY( N, RES, 1, DY, 1 )
```

```
            CALL CPOTRS( UPLO, N, 1, AF, LDAF, DY, N, INFO)
*
*         Calculate relative changes DX_X, DZ_Z and ratios DXRAT, DZRAT.
*
            NORMX = 0.0
            NORMY = 0.0
            NORMDX = 0.0
            DZ_Z = 0.0
            YMIN = HUGEVAL

            DO I = 1, N
               YK = CABS1(Y(I, J))
               DYK = CABS1(DY(I))

               IF (YK .NE. 0.0) THEN
                  DZ_Z = MAX( DZ_Z, DYK / YK )
               ELSE IF (DYK .NE. 0.0) THEN
                  DZ_Z = HUGEVAL
               END IF

               YMIN = MIN( YMIN, YK )

               NORMY = MAX( NORMY, YK )

               IF ( COLEQU ) THEN
                  NORMX = MAX(NORMX, YK * C(I))
                  NORMDX = MAX(NORMDX, DYK * C(I))
               ELSE
                  NORMX = NORMY
                  NORMDX = MAX(NORMDX, DYK)
               END IF
            END DO

            IF (NORMX .NE. 0.0) THEN
               DX_X = NORMDX / NORMX
            ELSE IF (NORMDX .EQ. 0.0) THEN
               DX_X = 0.0
            ELSE
               DX_X = HUGEVAL
            END IF

            DXRAT = NORMDX / PREVNORMDX
            DZRAT = DZ_Z / PREV_DZ_Z
*
*         Check termination criteria.
*
            IF (YMIN*RCOND .LT. INCR_THRESH*NORMY
     $           .AND. Y_PREC_STATE .LT. EXTRA_Y)
     $           INCR_PREC = .TRUE.
```

```
      IF (X_STATE .EQ. NOPROG_STATE .AND. DXRAT .LE. RTHRESH)
     $     X_STATE = WORKING_STATE
      IF (X_STATE .EQ. WORKING_STATE) THEN
         IF (DX_X .LE. EPS) THEN
            X_STATE = CONV_STATE
         ELSE IF (DXRAT .GT. RTHRESH) THEN
            IF (Y_PREC_STATE .NE. EXTRA_Y) THEN
               INCR_PREC = .TRUE.
            ELSE
               X_STATE = NOPROG_STATE
            END IF
         ELSE
            IF (DXRAT .GT. DXRATMAX) DXRATMAX = DXRAT
         END IF
         IF (X_STATE .GT. WORKING_STATE) FINAL_DX_X = DX_X
      END IF

      IF (Z_STATE .EQ. UNSTABLE_STATE .AND. DZ_Z .LE. DZ_UB)
     $     Z_STATE = WORKING_STATE
      IF (Z_STATE .EQ. NOPROG_STATE .AND. DZRAT .LE. RTHRESH)
     $     Z_STATE = WORKING_STATE
      IF (Z_STATE .EQ. WORKING_STATE) THEN
         IF (DZ_Z .LE. EPS) THEN
            Z_STATE = CONV_STATE
         ELSE IF (DZ_Z .GT. DZ_UB) THEN
            Z_STATE = UNSTABLE_STATE
            DZRATMAX = 0.0
            FINAL_DZ_Z = HUGEVAL
         ELSE IF (DZRAT .GT. RTHRESH) THEN
            IF (Y_PREC_STATE .NE. EXTRA_Y) THEN
               INCR_PREC = .TRUE.
            ELSE
               Z_STATE = NOPROG_STATE
            END IF
         ELSE
            IF (DZRAT .GT. DZRATMAX) DZRATMAX = DZRAT
         END IF
         IF (Z_STATE .GT. WORKING_STATE) FINAL_DZ_Z = DZ_Z
      END IF

      IF ( X_STATE.NE.WORKING_STATE.AND.
     $     (IGNORE_CWISE.OR.Z_STATE.NE.WORKING_STATE) )
     $     GOTO 666

      IF (INCR_PREC) THEN
         INCR_PREC = .FALSE.
         Y_PREC_STATE = Y_PREC_STATE + 1
         DO I = 1, N
            Y_TAIL( I ) = 0.0
         END DO
```

```
            END IF

            PREVNORMDX = NORMDX
            PREV_DZ_Z = DZ_Z
*
*           Update soluton.
*
            IF (Y_PREC_STATE .LT. EXTRA_Y) THEN
               CALL CAXPY( N, CMPLX(1.0), DY, 1, Y(1,J), 1 )
            ELSE
               CALL CLA_WWADDW(N, Y(1,J), Y_TAIL, DY)
            END IF

         END DO
*        Target of "IF (Z_STOP .AND. X_STOP)".  Sun's f77 won't EXIT.
 666     CONTINUE
*
*     Set final_* when cnt hits ithresh.
*
         IF (X_STATE .EQ. WORKING_STATE) FINAL_DX_X = DX_X
         IF (Z_STATE .EQ. WORKING_STATE) FINAL_DZ_Z = DZ_Z
*
*     Compute error bounds.
*
         IF (N_NORMS .GE. 1) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) =
     $           FINAL_DX_X / (1 - DXRATMAX)
         END IF
         IF (N_NORMS .GE. 2) THEN
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) =
     $           FINAL_DZ_Z / (1 - DZRATMAX)
         END IF
*
*     Compute componentwise relative backward error from formula
*         max(i) ( abs(R(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
*     where abs(Z) is the componentwise absolute value of the matrix
*     or vector Z.
*
*        Compute residual RES = B_s - op(A_s) * Y,
*            op(A) = A, A**T, or A**H depending on TRANS (and type).
*
         CALL CCOPY( N, B( 1, J ), 1, RES, 1 )
         CALL CHEMV(UPLO, N, CMPLX(-1.0), A, LDA, Y(1,J), 1, CMPLX(1.0),
     $        RES, 1)

         DO I = 1, N
            AYB( I ) = CABS1( B( I, J ) )
         END DO
*
*     Compute abs(op(A_s))*abs(Y) + abs(B_s).
```

```
*
         CALL CLA_HEAMV (UPLO2, N, 1.0,
     $        A, LDA, Y(1, J), 1, 1.0, AYB, 1)

         CALL CLA_LIN_BERR (N, N, 1, RES, AYB, BERR_OUT(J))
*
*     End of loop for each RHS.
*
      END DO
*
      RETURN
      END
```

———-

72.3.4 lisp code

— **cla_porfsx_extended.lisp** —

```
(let* ((unstable_state 0)
       (working_state 1)
       (conv_state 2)
       (noprog_state 3)
       (base_residual 0)
       (extra_residual 1)
       (extra_y 2)
       (la_linrx_err_i 2))
  (declare (type (f2cl-lib:integer4 0 0) unstable_state)
           (type (f2cl-lib:integer4 1 1) working_state)
           (type (f2cl-lib:integer4 2 2) conv_state)
           (type (f2cl-lib:integer4 3 3) noprog_state)
           (type (f2cl-lib:integer4 0 0) base_residual)
           (type (f2cl-lib:integer4 1 1) extra_residual)
           (type (f2cl-lib:integer4 2 2) extra_y)
           (type (f2cl-lib:integer4 2 2) la_linrx_err_i)
           (ignorable unstable_state working_state conv_state noprog_state
            base_residual extra_residual extra_y la_linrx_err_i))
  (defun cla_porfsx_extended
         (prec_type uplo n nrhs a lda af ldaf colequ c b ldb$ y ldy berr_out
          n_norms err_bnds_norm err_bnds_comp res ayb dy y_tail rcond ithresh
          rthresh dz_ub ignore_cwise info)
    (declare (type (single-float) dz_ub rthresh rcond)
             (type (array single-float (*)) ayb err_bnds_comp err_bnds_norm
              berr_out c)
             (type f2cl-lib:logical ignore_cwise colequ)
             (type (array f2cl-lib:complex8 (*)) y_tail dy res y b af a)
             (type (string *) uplo)
             (type (f2cl-lib:integer4) info ithresh n_norms ldy ldb$ ldaf lda
```

```
          nrhs n prec_type))
(f2cl-lib:with-multi-array-data
    ((uplo character uplo-%data% uplo-%offset%)
     (a f2cl-lib:complex8 a-%data% a-%offset%)
     (af f2cl-lib:complex8 af-%data% af-%offset%)
     (b f2cl-lib:complex8 b-%data% b-%offset%)
     (y f2cl-lib:complex8 y-%data% y-%offset%)
     (res f2cl-lib:complex8 res-%data% res-%offset%)
     (dy f2cl-lib:complex8 dy-%data% dy-%offset%)
     (y_tail f2cl-lib:complex8 y_tail-%data% y_tail-%offset%)
     (c single-float c-%data% c-%offset%)
     (berr_out single-float berr_out-%data% berr_out-%offset%)
     (err_bnds_norm single-float err_bnds_norm-%data%
      err_bnds_norm-%offset%)
     (err_bnds_comp single-float err_bnds_comp-%data%
      err_bnds_comp-%offset%)
     (ayb single-float ayb-%data% ayb-%offset%))
  (labels ((cabs1 (zdum)
             (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
    (declare
     (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
      cabs1))
    (prog ((zdum #C(0.0 0.0)) (incr_prec nil) (yk 0.0) (dyk 0.0) (ymin 0.0)
           (normy 0.0) (normx 0.0) (normdx 0.0) (dxrat 0.0) (dzrat 0.0)
           (prevnormdx 0.0) (prev_dz_z 0.0) (dxratmax 0.0) (dzratmax 0.0)
           (dx_x 0.0) (dz_z 0.0) (final_dx_x 0.0) (final_dz_z 0.0)
           (eps 0.0) (hugeval 0.0) (incr_thresh 0.0) (uplo2 0) (cnt 0)
           (i 0) (j 0) (x_state 0) (z_state 0) (y_prec_state 0))
      (declare (type (f2cl-lib:complex8) zdum)
               (type f2cl-lib:logical incr_prec)
               (type (single-float) yk dyk ymin normy normx normdx dxrat
                dzrat prevnormdx prev_dz_z dxratmax dzratmax dx_x dz_z
                final_dx_x final_dz_z eps hugeval incr_thresh)
               (type (f2cl-lib:integer4) uplo2 cnt i j x_state z_state
                y_prec_state))
      (if (/= info 0)
          (go end_label))
      (setf eps (slamch "Epsilon"))
      (setf hugeval (slamch "Overflow"))
      (setf hugeval (* hugeval hugeval))
      (setf incr_thresh (* (f2cl-lib:freal n) eps))
      (cond
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)
        (setf uplo2 (ilauplo "L")))
       (t (setf uplo2 (ilauplo "U"))))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
            ((> j nrhs) nil)
(tagbody
  (setf y_prec_state extra_residual)
  (cond
   ((= y_prec_state extra_y)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                             y_tail-%offset%)
              (coerce 0.0 'f2cl-lib:complex8))
      label100001))))
  (setf dxrat 0.0)
  (setf dxratmax 0.0)
  (setf dzrat 0.0)
  (setf dzratmax 0.0)
  (setf final_dx_x hugeval)
  (setf final_dz_z hugeval)
  (setf prevnormdx hugeval)
  (setf prev_dz_z hugeval)
  (setf dz_z hugeval)
  (setf dx_x hugeval)
  (setf x_state working_state)
  (setf z_state unstable_state)
  (setf incr_prec f2cl-lib:%false%)
  (f2cl-lib:fdo (cnt 1 (f2cl-lib:int-add cnt 1))
                ((> cnt ithresh) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (ccopy n
           (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 j)
                                 ((1 ldb$) (1 *)) b-%offset%)
           1 res 1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf n var-0)))
      (cond
       ((= y_prec_state base_residual)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9)
            (chemv uplo n (f2cl-lib:cmplx -1.0) a lda
             (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                                   ((1 ldy) (1 *)) y-%offset%)
             1 (f2cl-lib:cmplx 1.0) res 1)
          (declare
           (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
          (when var-0 (setf uplo var-0))
          (when var-1 (setf n var-1))
          (when var-4 (setf lda var-4))))
       ((= y_prec_state extra_residual)
```

```
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (blas_chemv_x uplo2 n (f2cl-lib:cmplx -1.0) a lda
       (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                             ((1 ldy) (1 *)) y-%offset%)
       1 (f2cl-lib:cmplx 1.0) res 1 prec_type)
    (declare
     (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
    (when var-0 (setf uplo2 var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf lda var-4))
    (when var-10 (setf prec_type var-10))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11)
      (blas_chemv2_x uplo2 n (f2cl-lib:cmplx -1.0) a lda
       (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                             ((1 ldy) (1 *)) y-%offset%)
       y_tail 1 (f2cl-lib:cmplx 1.0) res 1 prec_type)
    (declare
     (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9
      var-10))
    (when var-0 (setf uplo2 var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf lda var-4))
    (when var-11 (setf prec_type var-11)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (ccopy n res 1 dy 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cpotrs uplo n 1 af ldaf dy n info)
  (declare (ignore var-2 var-3 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-4 (setf ldaf var-4))
  (when var-6 (setf n var-6))
  (when var-7 (setf info var-7)))
(setf normx 0.0)
(setf normy 0.0)
(setf normdx 0.0)
(setf dz_z 0.0)
(setf ymin hugeval)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf yk
```

```
                (cabs1
                 (f2cl-lib:fref y-%data% (i j) ((1 ldy) (1 *))
                                y-%offset%)))
        (setf dyk
                (cabs1
                 (f2cl-lib:fref dy-%data% (i) ((1 *))
                                dy-%offset%)))
        (cond ((/= yk 0.0) (setf dz_z (max dz_z (/ dyk yk))))
              ((/= dyk 0.0) (setf dz_z hugeval)))
        (setf ymin (min ymin yk))
        (setf normy (max normy yk))
        (cond
         (colequ
          (setf normx
                  (max normx
                       (* yk
                          (f2cl-lib:fref c-%data% (i) ((1 *))
                                         c-%offset%))))
          (setf normdx
                  (max normdx
                       (* dyk
                          (f2cl-lib:fref c-%data% (i) ((1 *))
                                         c-%offset%)))))
         (t (setf normx normy) (setf normdx (max normdx dyk))))
       label100003))
    (cond ((/= normx 0.0) (setf dx_x (/ normdx normx)))
          ((= normdx 0.0) (setf dx_x 0.0))
          (t (setf dx_x hugeval)))
    (setf dxrat (/ normdx prevnormdx))
    (setf dzrat (/ dz_z prev_dz_z))
    (if (and (< (* ymin rcond) (* incr_thresh normy))
             (< y_prec_state extra_y))
        (setf incr_prec f2cl-lib:%true%))
    (if (and (= x_state noprog_state) (<= dxrat rthresh))
        (setf x_state working_state))
    (cond
     ((= x_state working_state)
      (cond ((<= dx_x eps) (setf x_state conv_state))
            ((> dxrat rthresh)
             (cond
              ((/= y_prec_state extra_y)
               (setf incr_prec f2cl-lib:%true%))
              (t (setf x_state noprog_state))))
            (t
             (if (> dxrat dxratmax)
                 (setf dxratmax dxrat))))
      (if (> x_state working_state)
          (setf final_dx_x dx_x))))
    (if (and (= z_state unstable_state) (<= dz_z dz_ub))
        (setf z_state working_state))
```

```
  (if (and (= z_state noprog_state) (<= dzrat rthresh))
      (setf z_state working_state))
  (cond
   ((= z_state working_state)
    (cond ((<= dz_z eps) (setf z_state conv_state))
          ((> dz_z dz_ub) (setf z_state unstable_state)
           (setf dzratmax 0.0) (setf final_dz_z hugeval))
          ((> dzrat rthresh)
           (cond
            (((/= y_prec_state extra_y)
             (setf incr_prec f2cl-lib:%true%))
            (t (setf z_state noprog_state))))
          (t
           (if (> dzrat dzratmax)
               (setf dzratmax dzrat))))
    (if (> z_state working_state)
        (setf final_dz_z dz_z))))
  (if (and (/= x_state working_state)
           (or ignore_cwise (/= z_state working_state)))
      (go label666))
  (cond
   (incr_prec (setf incr_prec f2cl-lib:%false%)
    (setf y_prec_state (f2cl-lib:int-add y_prec_state 1))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                             y_tail-%offset%)
                (coerce 0.0 'f2cl-lib:complex8))
       label100004))))
  (setf prevnormdx normdx)
  (setf prev_dz_z dz_z)
  (cond
   ((< y_prec_state extra_y)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (caxpy n (f2cl-lib:cmplx 1.0) dy 1
         (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                               ((1 ldy) (1 *)) y-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4 var-5))
      (when var-0 (setf n var-0))))
   (t
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (cla_wwaddw n
         (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                               ((1 ldy) (1 *)) y-%offset%)
         y_tail dy)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf n var-0)))))
label100002))
```

```
    label666
     (if (= x_state working_state)
         (setf final_dx_x dx_x))
     (if (= z_state working_state)
         (setf final_dz_z dz_z))
     (cond
      ((>= n_norms 1)
       (setf (f2cl-lib:fref err_bnds_norm-%data% (j la_linrx_err_i)
                            ((1 nrhs) (1 *)) err_bnds_norm-%offset%)
             (/ final_dx_x (- 1 dxratmax)))))
     (cond
      ((>= n_norms 2)
       (setf (f2cl-lib:fref err_bnds_comp-%data% (j la_linrx_err_i)
                            ((1 nrhs) (1 *)) err_bnds_comp-%offset%)
             (/ final_dz_z (- 1 dzratmax)))))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (ccopy n
          (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 j)
                                ((1 ldb$) (1 *)) b-%offset%)
          1 res 1)
       (declare (ignore var-1 var-2 var-3 var-4))
       (when var-0 (setf n var-0)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (chemv uplo n (f2cl-lib:cmplx -1.0) a lda
          (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                                ((1 ldy) (1 *)) y-%offset%)
          1 (f2cl-lib:cmplx 1.0) res 1)
       (declare (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
       (when var-0 (setf uplo var-0))
       (when var-1 (setf n var-1))
       (when var-4 (setf lda var-4)))
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref ayb-%data% (i) ((1 *)) ayb-%offset%)
                 (cabs1
                  (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)))
        label100005))
     (cla_heamv uplo2 n 1.0 a lda
      (f2cl-lib:array-slice y-%data% f2cl-lib:complex8 (1 j)
                            ((1 ldy) (1 *)) y-%offset%)
      1 1.0 ayb 1)
     (cla_lin_berr n n 1 res ayb
      (f2cl-lib:array-slice berr_out-%data% single-float (j) ((1 *))
                            berr_out-%offset%))
    label100000))
 (go end_label)
end_label
```

```
          (return
           (values prec_type uplo n nil nil lda nil ldaf nil nil nil nil nil
                   nil nil nil nil nil nil nil nil nil nil nil nil nil nil
                   info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cla_porfsx_extended
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              fortran-to-lisp::logical
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (single-float)
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              fortran-to-lisp::logical
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::prec_type
                                              fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil nil
                                              fortran-to-lisp::lda nil
```

```
                                        fortran-to-lisp::ldaf nil nil nil
                                        nil nil nil nil nil nil nil nil
                                        nil nil nil nil nil nil nil nil
                                        fortran-to-lisp::info)
                                      :calls
                                      '(fortran-to-lisp::cla_lin_berr
                                        fortran-to-lisp::cla_heamv))))
```

———-

72.4 cla_porpvgrw

72.4.1 Axiom unit tests

— **claporpvgrw.input** —

```
)set break resume
)sys rm -f claporpvgrw.output
)spool claporpvgrw.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

72.4.2 Axiom help page

— **cla_porpvgrw.help** —

```
====================================================================
cla_porpvgrw examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
```

```
      real function cla_porpvgrw (UPLO, NCOLS, A, LDA, AF, LDAF, WORK)
          CLA_PORPVGRW computes the reciprocal pivot growth factor
          norm(A)/norm(U) for a symmetric or Hermitian positive-definite
          matrix.

Function/Subroutine Documentation
   real function cla_porpvgrw (character*1 UPLO, integer NCOLS, complex,
       dimension( lda, * ) A, integer LDA, complex, dimension( ldaf, * ) AF,
       integer LDAF, real, dimension( * ) WORK)
       CLA_PORPVGRW computes the reciprocal pivot growth factor
       norm(A)/norm(U) for a symmetric or Hermitian positive-definite matrix.

Purpose:
   CLA_PORPVGRW computes the reciprocal pivot growth factor
   norm(A)/norm(U). The "max absolute element" norm is used. If this is
   much less than 1, the stability of the LU factorization of the
   (equilibrated) matrix A could be poor. This also means that the
   solution X, estimated condition numbers, and error bounds could be
   unreliable.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 NCOLS
   NCOLS is INTEGER
   The number of columns of the matrix A. NCOLS >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is COMPLEX array, dimension (LDAF,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**T*U or A = L*L**T, as computed by CPOTRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 WORK
   WORK is COMPLEX array, dimension (2*N)
```

———-

72.4.3 fortran code

— **cla_porpvgrw.f** —

```
*  =====================================================================
      REAL FUNCTION CLA_PORPVGRW( UPLO, NCOLS, A, LDA, AF, LDAF, WORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER*1        UPLO
      INTEGER            NCOLS, LDA, LDAF
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), AF( LDAF, * )
      REAL               WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               AMAX, UMAX, RPVGRW
      LOGICAL            UPPER
      COMPLEX            ZDUM
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, CLASET
      LOGICAL            LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, REAL, AIMAG
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
      UPPER = LSAME( 'Upper', UPLO )
```

```
*
*     SPOTRF will have factored only the NCOLSxNCOLS leading minor, so
*     we restrict the growth search to that minor and use only the first
*     2*NCOLS workspace entries.
*
      RPVGRW = 1.0
      DO I = 1, 2*NCOLS
         WORK( I ) = 0.0
      END DO
*
*     Find the max magnitude entry of each column.
*
      IF ( UPPER ) THEN
         DO J = 1, NCOLS
            DO I = 1, J
               WORK( NCOLS+J ) =
     $              MAX( CABS1( A( I, J ) ), WORK( NCOLS+J ) )
            END DO
         END DO
      ELSE
         DO J = 1, NCOLS
            DO I = J, NCOLS
               WORK( NCOLS+J ) =
     $              MAX( CABS1( A( I, J ) ), WORK( NCOLS+J ) )
            END DO
         END DO
      END IF
*
*     Now find the max magnitude entry of each column of the factor in
*     AF.  No pivoting, so no permutations.
*
      IF ( LSAME( 'Upper', UPLO ) ) THEN
         DO J = 1, NCOLS
            DO I = 1, J
               WORK( J ) = MAX( CABS1( AF( I, J ) ), WORK( J ) )
            END DO
         END DO
      ELSE
         DO J = 1, NCOLS
            DO I = J, NCOLS
               WORK( J ) = MAX( CABS1( AF( I, J ) ), WORK( J ) )
            END DO
         END DO
      END IF
*
*     Compute the *inverse* of the max element growth factor.  Dividing
*     by zero would imply the largest entry of the factor's column is
*     zero.  Than can happen when either the column of A is zero or
*     massive pivots made the factor underflow to zero.  Neither counts
*     as growth in itself, so simply ignore terms with zero
```

```
*     denominators.
*
      IF ( LSAME( 'Upper', UPLO ) ) THEN
         DO I = 1, NCOLS
            UMAX = WORK( I )
            AMAX = WORK( NCOLS+I )
            IF ( UMAX /= 0.0 ) THEN
               RPVGRW = MIN( AMAX / UMAX, RPVGRW )
            END IF
         END DO
      ELSE
         DO I = 1, NCOLS
            UMAX = WORK( I )
            AMAX = WORK( NCOLS+I )
            IF ( UMAX /= 0.0 ) THEN
               RPVGRW = MIN( AMAX / UMAX, RPVGRW )
            END IF
         END DO
      END IF

      CLA_PORPVGRW = RPVGRW
      END
```

———-

72.4.4 lisp code

— cla_porpvgrw.lisp —

```
(defun cla_porpvgrw (uplo ncols a lda af ldaf work)
  (declare (type (array single-float (*)) work)
           (type (array f2cl-lib:complex8 (*)) af a)
           (type (f2cl-lib:integer4) ldaf lda ncols)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%)
       (af f2cl-lib:complex8 af-%data% af-%offset%)
       (work single-float work-%data% work-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
        cabs1))
      (prog ((zdum #C(0.0 0.0)) (upper nil) (amax 0.0) (umax 0.0) (rpvgrw 0.0)
             (i 0) (j 0) (cla_porpvgrw 0.0))
        (declare (type (f2cl-lib:integer4) j i)
                 (type (single-float) cla_porpvgrw rpvgrw umax amax)
```

```
         (type f2cl-lib:logical upper)
         (type (f2cl-lib:complex8) zdum))
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame "Upper" uplo)
          (declare (ignore var-0))
          (when var-1 (setf uplo var-1))
          ret-val))
(setf rpvgrw 1.0)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-mul 2 ncols)) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) 0.0)
   label100000))
(cond
 (upper
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j ncols) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i j) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ncols j))
                                ((1 *)) work-%offset%)
                  (max
                   (cabs1
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add ncols j)) ((1 *))
                                  work-%offset%)))
         label100002))
     label100001)))
 (t
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j ncols) nil)
    (tagbody
      (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                    ((> i ncols) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ncols j))
                                ((1 *)) work-%offset%)
                  (max
                   (cabs1
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add ncols j)) ((1 *))
                                  work-%offset%)))
         label100004))
```

```
      label100003))))
    (cond
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame "Upper" uplo)
        (declare (ignore var-0))
        (when var-1 (setf uplo var-1))
        ret-val)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j ncols) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                      (max
                       (cabs1
                        (f2cl-lib:fref af-%data% (i j) ((1 ldaf) (1 *))
                                       af-%offset%))
                       (f2cl-lib:fref work-%data% (j) ((1 *))
                                      work-%offset%)))
            label100006))
      label100005)))
     (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j ncols) nil)
        (tagbody
          (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                        ((> i ncols) nil)
            (tagbody
              (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                      (max
                       (cabs1
                        (f2cl-lib:fref af-%data% (i j) ((1 ldaf) (1 *))
                                       af-%offset%))
                       (f2cl-lib:fref work-%data% (j) ((1 *))
                                      work-%offset%)))
            label100008))
      label100007))))
    (cond
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame "Upper" uplo)
        (declare (ignore var-0))
        (when var-1 (setf uplo var-1))
        ret-val)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i ncols) nil)
        (tagbody
          (setf umax (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
          (setf amax
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ncols i))
```

```
                                    ((1 *)) work-%offset%))
              (cond
               ((f2cl-lib:f2cl/ umax (= 0.0))
                (setf rpvgrw (min (/ amax umax) rpvgrw))))
             label100009)))
         (t
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i ncols) nil)
            (tagbody
              (setf umax (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
              (setf amax
                      (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ncols i))
                                    ((1 *)) work-%offset%))
              (cond
               ((f2cl-lib:f2cl/ umax (= 0.0))
                (setf rpvgrw (min (/ amax umax) rpvgrw))))
             label100010))))
        (setf cla_porpvgrw rpvgrw)
       end_label
        (return (values cla_porpvgrw uplo nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cla_porpvgrw
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*)))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil nil
                                              nil nil nil)
                                            :calls 'nil)))
```

———-

72.5 cpocon

72.5.1 Axiom unit tests

— **cpocon.input** —

```
)set break resume
)sys rm -f cpocon.output
)spool cpocon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

72.5.2 Axiom help page

— **cpocon.help** —

```
==================================================================
cpocon examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cpocon (UPLO, N, A, LDA, ANORM, RCOND, WORK, RWORK, INFO)
           CPOCON

Function/Subroutine Documentation
   subroutine cpocon (character UPLO, integer N, complex, dimension( lda, * )
       A, integer LDA, real ANORM, real RCOND, complex, dimension( * ) WORK,
       real, dimension( * ) RWORK, integer INFO)
       CPOCON

Purpose:
   CPOCON estimates the reciprocal of the condition number (in the
   1-norm) of a complex Hermitian positive definite matrix using the
   Cholesky factorization A = U**H*U or A = L*L**H computed by CPOTRF.
```

```
   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**H*U or A = L*L**H, as computed by CPOTRF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 ANORM
   ANORM is REAL
   The 1-norm (or infinity-norm) of the Hermitian matrix A.

 RCOND
   RCOND is REAL
   The reciprocal of the condition number of the matrix A,
   computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is an
   estimate of the 1-norm of inv(A) computed in this routine.

 WORK
   WORK is COMPLEX array, dimension (2*N)

 RWORK
   RWORK is REAL array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

72.5.3 fortran code

— cpocon.f —

```
*  =====================================================================
      SUBROUTINE CPOCON( UPLO, N, A, LDA, ANORM, RCOND, WORK, RWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
      REAL               ANORM, RCOND
*     ..
*     .. Array Arguments ..
      REAL               RWORK( * )
      COMPLEX            A( LDA, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      CHARACTER          NORMIN
      INTEGER            IX, KASE
      REAL               AINVNM, SCALE, SCALEL, SCALEU, SMLNUM
      COMPLEX            ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ICAMAX
      REAL               SLAMCH
      EXTERNAL           LSAME, ICAMAX, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLACN2, CLATRS, CSRSCL, XERBLA
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          ABS, AIMAG, MAX, REAL
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -5
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CPOCON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
      SMLNUM = SLAMCH( 'Safe minimum' )
*
*     Estimate the 1-norm of inv(A).
*
      KASE = 0
      NORMIN = 'N'
   10 CONTINUE
      CALL CLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( UPPER ) THEN
*
*           Multiply by inv(U**H).
```

```
*
            CALL CLATRS( 'Upper', 'Conjugate transpose', 'Non-unit',
     $                   NORMIN, N, A, LDA, WORK, SCALEL, RWORK, INFO )
            NORMIN = 'Y'
*
*           Multiply by inv(U).
*
            CALL CLATRS( 'Upper', 'No transpose', 'Non-unit', NORMIN, N,
     $                   A, LDA, WORK, SCALEU, RWORK, INFO )
         ELSE
*
*           Multiply by inv(L).
*
            CALL CLATRS( 'Lower', 'No transpose', 'Non-unit', NORMIN, N,
     $                   A, LDA, WORK, SCALEL, RWORK, INFO )
            NORMIN = 'Y'
*
*           Multiply by inv(L**H).
*
            CALL CLATRS( 'Lower', 'Conjugate transpose', 'Non-unit',
     $                   NORMIN, N, A, LDA, WORK, SCALEU, RWORK, INFO )
         END IF
*
*        Multiply by 1/SCALE if doing so will not cause overflow.
*
         SCALE = SCALEL*SCALEU
         IF( SCALE.NE.ONE ) THEN
            IX = ICAMAX( N, WORK, 1 )
            IF( SCALE.LT.CABS1( WORK( IX ) )*SMLNUM .OR. SCALE.EQ.ZERO )
     $         GO TO 20
            CALL CSRSCL( N, SCALE, WORK, 1 )
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
   20 CONTINUE
      RETURN
*
*     End of CPOCON
*
      END
```

———-

72.5.4 lisp code

— cpocon.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun cpocon (uplo n a lda anorm rcond work rwork info)
    (declare (type (array single-float (*)) rwork)
             (type (single-float) rcond anorm)
             (type (array f2cl-lib:complex8 (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
          cabs1))
        (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
               (zdum #C(0.0 0.0)) (ainvnm 0.0) (scale 0.0) (scalel 0.0)
               (scaleu 0.0) (smlnum 0.0) (ix 0) (kase 0)
               (normin
                (make-array '(1) :element-type 'character :initial-element
                            #\ ))
               (upper nil))
          (declare (type (array f2cl-lib:integer4 (3)) isave)
                   (type (f2cl-lib:complex8) zdum)
                   (type (single-float) ainvnm scale scalel scaleu smlnum)
                   (type (f2cl-lib:integer4) ix kase)
                   (type (string 1) normin)
                   (type f2cl-lib:logical upper))
          (setf info 0)
          (setf upper
                  (multiple-value-bind (ret-val var-0 var-1)
                      (lsame uplo "U")
                    (declare (ignore var-1))
                    (when var-0 (setf uplo var-0))
                    ret-val))
          (cond
           ((and (not upper)
                 (not
                  (multiple-value-bind (ret-val var-0 var-1)
                      (lsame uplo "L")
```

```
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4))
   ((< anorm zero) (setf info -5)))
  (cond
   ((/= info 0) (xerbla "CPOCON" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf rcond zero)
  (cond ((= n 0) (setf rcond one) (go end_label))
        ((= anorm zero) (go end_label)))
  (setf smlnum (slamch "Safe minimum"))
  (setf kase 0)
  (f2cl-lib:f2cl-set-string normin "N" (string 1))
 label10
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (clacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ n 1))
                             ((1 *)) work-%offset%)
       work ainvnm kase isave)
    (declare (ignore var-1 var-2 var-5))
    (setf n var-0)
    (setf ainvnm var-3)
    (setf kase var-4))
  (cond
   ((/= kase 0)
    (cond
     (upper
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (clatrs "Upper" "Conjugate transpose" "Non-unit" normin n a
           lda work scalel rwork info)
        (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
        (setf normin var-3)
        (setf n var-4)
        (setf lda var-6)
        (setf scalel var-8)
        (setf info var-10))
      (f2cl-lib:f2cl-set-string normin "Y" (string 1))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (clatrs "Upper" "No transpose" "Non-unit" normin n a lda work
           scaleu rwork info)
        (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
        (setf normin var-3)
```

```
      (setf n var-4)
      (setf lda var-6)
      (setf scaleu var-8)
      (setf info var-10)))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (clatrs "Lower" "No transpose" "Non-unit" normin n a lda work
          scalel rwork info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
      (setf normin var-3)
      (setf n var-4)
      (setf lda var-6)
      (setf scalel var-8)
      (setf info var-10))
     (f2cl-lib:f2cl-set-string normin "Y" (string 1))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (clatrs "Lower" "Conjugate transpose" "Non-unit" normin n a
          lda work scaleu rwork info)
      (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
      (setf normin var-3)
      (setf n var-4)
      (setf lda var-6)
      (setf scaleu var-8)
      (setf info var-10))))
  (setf scale (* scalel scaleu))
  (cond
   ((/= scale one)
    (setf ix
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (icamax n work 1)
              (declare (ignore var-1 var-2))
              (when var-0 (setf n var-0))
              ret-val))
    (if (or
         (< scale
            (*
             (cabs1
              (f2cl-lib:fref work-%data% (ix) ((1 *)) work-%offset%))
             smlnum))
         (= scale zero))
        (go label20))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (csrscl n scale work 1)
      (declare (ignore var-2 var-3))
      (when var-0 (setf n var-0))
      (when var-1 (setf scale var-1)))))
```

```
            (go label10)))
          (if (/= ainvnm zero)
              (setf rcond (/ (/ one ainvnm) anorm)))
         label20
          (go end_label)
         end_label
          (return (values uplo n nil lda nil rcond nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cpocon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::rcond nil nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::clatrs
                                              fortran-to-lisp::clacn2))))
```

———-

72.6 cpoequ

72.6.1 Axiom unit tests

— cpoequ.input —

```
)set break resume
)sys rm -f cpoequ.output
)spool cpoequ.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

72.6.2 Axiom help page

— **cpoequ.help** —

```
=====================================================================
cpoequ examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cpoequ (N, A, LDA, S, SCOND, AMAX, INFO)
           CPOEQU

Function/Subroutine Documentation
   subroutine cpoequ (integer N, complex, dimension( lda, * ) A, integer LDA,
       real, dimension( * ) S, real SCOND, real AMAX, integer INFO)
       CPOEQU

Purpose:
   CPOEQU computes row and column scalings intended to equilibrate a
   Hermitian positive definite matrix A and reduce its condition number
   (with respect to the two-norm).  S contains the scale factors,
   S(i) = 1/sqrt(A(i,i)), chosen so that the scaled matrix B with
   elements B(i,j) = S(i)*A(i,j)*S(j) has ones on the diagonal.  This
   choice of S puts the condition number of B within a factor N of the
   smallest possible condition number over all possible diagonal scalings.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
```

```
  The N-by-N Hermitian positive definite matrix whose scaling
  factors are to be computed.  Only the diagonal elements of A
  are referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

S
  S is REAL array, dimension (N)
  If INFO = 0, S contains the scale factors for A.

SCOND
  SCOND is REAL
  If INFO = 0, S contains the ratio of the smallest S(i) to
  the largest S(i).  If SCOND >= 0.1 and AMAX is neither too
  large nor too small, it is not worth scaling by S.

AMAX
  AMAX is REAL
  Absolute value of largest matrix element.  If AMAX is very
  close to overflow or very close to underflow, the matrix
  should be scaled.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, the i-th diagonal element is nonpositive.
```

———-

72.6.3 fortran code

— cpoequ.f —

```
*  =====================================================================
      SUBROUTINE CPOEQU( N, A, LDA, S, SCOND, AMAX, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, N
      REAL               AMAX, SCOND
```

```
*     ..
*     .. Array Arguments ..
      REAL               S( * )
      COMPLEX            A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      REAL               SMIN
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, REAL, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -3
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CPOEQU', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SCOND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Find the minimum and maximum diagonal elements.
*
      S( 1 ) = REAL( A( 1, 1 ) )
      SMIN = S( 1 )
      AMAX = S( 1 )
```

```
      DO 10 I = 2, N
         S( I ) = REAL( A( I, I ) )
         SMIN = MIN( SMIN, S( I ) )
         AMAX = MAX( AMAX, S( I ) )
   10 CONTINUE
*
      IF( SMIN.LE.ZERO ) THEN
*
*        Find the first non-positive diagonal element and return.
*
         DO 20 I = 1, N
            IF( S( I ).LE.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   20    CONTINUE
      ELSE
*
*        Set the scale factors to the reciprocals
*        of the diagonal elements.
*
         DO 30 I = 1, N
            S( I ) = ONE / SQRT( S( I ) )
   30    CONTINUE
*
*        Compute SCOND = min(S(I)) / max(S(I))
*
         SCOND = SQRT( SMIN ) / SQRT( AMAX )
      END IF
      RETURN
*
*     End of CPOEQU
*
      END
```

———-

72.6.4 lisp code

— cpoequ.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun cpoequ (n a lda s scond amax info)
    (declare (type (single-float) amax scond)
             (type (array single-float (*)) s)
```

```
          (type (array f2cl-lib:complex8 (*)) a)
          (type (f2cl-lib:integer4) info lda n))
(f2cl-lib:with-multi-array-data
    ((a f2cl-lib:complex8 a-%data% a-%offset%)
     (s single-float s-%data% s-%offset%))
  (prog ((smin 0.0) (i 0))
    (declare (type (single-float) smin)
             (type (f2cl-lib:integer4) i))
    (setf info 0)
    (cond ((< n 0) (setf info -1))
          ((< lda
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
           (setf info -3)))
    (cond
     ((/= info 0) (xerbla "CPOEQU" (f2cl-lib:int-sub info))
      (go end_label)))
    (cond ((= n 0) (setf scond one) (setf amax zero) (go end_label)))
    (setf (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%)
            (f2cl-lib:freal
             (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)))
    (setf smin (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
    (setf amax (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
    (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                (f2cl-lib:freal
                 (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                a-%offset%)))
        (setf smin
                (min smin (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
        (setf amax
                (max amax (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
       label10))
    (cond
     ((<= smin zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (cond
           ((<= (f2cl-lib:fref s (i) ((1 *))) zero) (setf info i)
            (go end_label)))
       label20)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                  (/ one
                     (f2cl-lib:fsqrt
```

```
                          (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%))))
             label30))
          (setf scond (/ (f2cl-lib:fsqrt smin) (f2cl-lib:fsqrt amax)))))
        (go end_label)
       end_label
        (return (values nil nil nil nil scond amax info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cpoequ
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil
                                              fortran-to-lisp::scond
                                              fortran-to-lisp::amax
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

72.7 cpoequb

72.7.1 Axiom unit tests

— cpoequb.input —

```
)set break resume
)sys rm -f cpoequb.output
)spool cpoequb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

72.7.2 Axiom help page

— cpoequb.help —

```
=====================================================================
cpoequb examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cpoequb (N, A, LDA, S, SCOND, AMAX, INFO)
           CPOEQUB

Function/Subroutine Documentation
   subroutine cpoequb (integer N, complex, dimension( lda, * ) A, integer LDA,
       real, dimension( * ) S, real SCOND, real AMAX, integer INFO)
       CPOEQUB

Purpose:
   CPOEQUB computes row and column scalings intended to equilibrate a
   symmetric positive definite matrix A and reduce its condition number
   (with respect to the two-norm).  S contains the scale factors,
   S(i) = 1/sqrt(A(i,i)), chosen so that the scaled matrix B with
   elements B(i,j) = S(i)*A(i,j)*S(j) has ones on the diagonal.  This
   choice of S puts the condition number of B within a factor N of the
   smallest possible condition number over all possible diagonal scalings.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   The N-by-N symmetric positive definite matrix whose scaling
   factors are to be computed.  Only the diagonal elements of A
   are referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).
```

```
 S
   S is REAL array, dimension (N)
   If INFO = 0, S contains the scale factors for A.

 SCOND
   SCOND is REAL
   If INFO = 0, S contains the ratio of the smallest S(i) to
   the largest S(i).  If SCOND >= 0.1 and AMAX is neither too
   large nor too small, it is not worth scaling by S.

 AMAX
   AMAX is REAL
   Absolute value of largest matrix element.  If AMAX is very
   close to overflow or very close to underflow, the matrix should be scaled.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, the i-th diagonal element is nonpositive.
```

72.7.3 fortran code

— cpoequb.f —

```
*  =====================================================================
      SUBROUTINE CPOEQUB( N, A, LDA, S, SCOND, AMAX, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, N
      REAL               AMAX, SCOND
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * )
      REAL               S( * )
*     ..
*
*  =====================================================================
*
```

```
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      REAL               SMIN, BASE, TMP
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT, LOG, INT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
*     Positive definite only performs 1 pass of equilibration.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -3
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CPOEQUB', -INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 ) THEN
         SCOND = ONE
         AMAX = ZERO
         RETURN
      END IF

      BASE = SLAMCH( 'B' )
      TMP = -0.5 / LOG ( BASE )
*
*     Find the minimum and maximum diagonal elements.
*
      S( 1 ) = A( 1, 1 )
      SMIN = S( 1 )
```

```
      AMAX = S( 1 )
      DO 10 I = 2, N
         S( I ) = A( I, I )
         SMIN = MIN( SMIN, S( I ) )
         AMAX = MAX( AMAX, S( I ) )
   10 CONTINUE
*
      IF( SMIN.LE.ZERO ) THEN
*
*        Find the first non-positive diagonal element and return.
*
         DO 20 I = 1, N
            IF( S( I ).LE.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   20    CONTINUE
      ELSE
*
*        Set the scale factors to the reciprocals
*        of the diagonal elements.
*
         DO 30 I = 1, N
            S( I ) = BASE ** INT( TMP * LOG( S( I ) ) )
   30    CONTINUE
*
*        Compute SCOND = min(S(I)) / max(S(I)).
*
         SCOND = SQRT( SMIN ) / SQRT( AMAX )
      END IF
*
      RETURN
*
*     End of CPOEQUB
*
      END
```

———-

72.7.4 lisp code

— cpoequb.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun cpoequb (n a lda s scond amax info)
```

```
(declare (type (single-float) amax scond)
         (type (array single-float (*)) s)
         (type (array f2cl-lib:complex8 (*)) a)
         (type (f2cl-lib:integer4) info lda n))
(f2cl-lib:with-multi-array-data
    ((a f2cl-lib:complex8 a-%data% a-%offset%)
     (s single-float s-%data% s-%offset%))
  (prog ((smin 0.0) (base 0.0) (tmp 0.0) (i 0))
    (declare (type (single-float) smin base tmp)
             (type (f2cl-lib:integer4) i))
    (setf info 0)
    (cond ((< n 0) (setf info -1))
          ((< lda
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
           (setf info -3)))
    (cond
     ((/= info 0) (xerbla "CPOEQUB" (f2cl-lib:int-sub info))
      (go end_label)))
    (cond ((= n 0) (setf scond one) (setf amax zero) (go end_label)))
    (setf base (slamch "B"))
    (setf tmp (/ -0.5 (f2cl-lib:flog base)))
    (setf (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%)
            (coerce
             (realpart
              (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%))
             'single-float))
    (setf smin (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
    (setf amax (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
    (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                (coerce
                 (realpart
                  (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                 a-%offset%))
                 'single-float))
        (setf smin
                (min smin (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
        (setf amax
                (max amax (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
       label10))
    (cond
     ((<= smin zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (cond
           ((<= (f2cl-lib:fref s (i) ((1 *))) zero) (setf info i)
            (go end_label)))
```

```
            label20)))
         (t
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                      (expt base
                            (f2cl-lib:int
                             (* tmp
                                (f2cl-lib:flog
                                 (f2cl-lib:fref s-%data% (i) ((1 *))
                                                s-%offset%))))))
            label30))
          (setf scond (/ (f2cl-lib:fsqrt smin) (f2cl-lib:fsqrt amax)))))
        (go end_label)
       end_label
        (return (values nil nil nil nil scond amax info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cpoequb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil
                                              fortran-to-lisp::scond
                                              fortran-to-lisp::amax
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

72.8 cporfs

72.8.1 Axiom unit tests

— cporfs.input —

```
)set break resume
)sys rm -f cporfs.output
)spool cporfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

72.8.2 Axiom help page

— **cporfs.help** —

```
====================================================================
cporfs examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cporfs (UPLO, N, NRHS, A, LDA, AF, LDAF, B, LDB, X, LDX,
           FERR, BERR, WORK, RWORK, INFO)
           CPORFS

Function/Subroutine Documentation
   subroutine cporfs (character UPLO, integer N, integer NRHS, complex,
       dimension( lda, * ) A, integer LDA, complex, dimension( ldaf, * ) AF,
       integer LDAF, complex, dimension( ldb, * ) B, integer LDB, complex,
       dimension( ldx, * ) X, integer LDX, real, dimension( * ) FERR, real,
       dimension( * ) BERR, complex, dimension( * ) WORK, real, dimension( * )
       RWORK, integer INFO)
       CPORFS

Purpose:
   CPORFS improves the computed solution to a system of linear
   equations when the coefficient matrix is Hermitian positive definite,
   and provides error bounds and backward error estimates for the solution.

Parameters:

 UPLO
   UPLO is CHARACTER*1
```

```
  = 'U':  Upper triangle of A is stored;
  = 'L':  Lower triangle of A is stored.

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  The Hermitian matrix A.  If UPLO = 'U', the leading N-by-N
  upper triangular part of A contains the upper triangular part
  of the matrix A, and the strictly lower triangular part of A
  is not referenced.  If UPLO = 'L', the leading N-by-N lower
  triangular part of A contains the lower triangular part of
  the matrix A, and the strictly upper triangular part of A is
  not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX array, dimension (LDAF,N)
  The triangular factor U or L from the Cholesky factorization
  A = U**H*U or A = L*L**H, as computed by CPOTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

B
  B is COMPLEX array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by CPOTRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
```

```
  The leading dimension of the array X.  LDX >= max(1,N).

 FERR
  FERR is REAL array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

 BERR
  BERR is REAL array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

 WORK
  WORK is COMPLEX array, dimension (2*N)

 RWORK
  RWORK is REAL array, dimension (N)

 INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

  ITMAX is the maximum number of steps of iterative refinement.
```

———-

72.8.3 fortran code

— **cporfs.f** —

```
*  ===================================================================
      SUBROUTINE CPORFS( UPLO, N, NRHS, A, LDA, AF, LDAF, B, LDB, X,
     $                   LDX, FERR, BERR, WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
```

```
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      REAL               BERR( * ), FERR( * ), RWORK( * )
      COMPLEX            A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
      COMPLEX            ONE
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ) )
      REAL               TWO
      PARAMETER          ( TWO = 2.0E+0 )
      REAL               THREE
      PARAMETER          ( THREE = 3.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            COUNT, I, J, K, KASE, NZ
      REAL               EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN, XK
      COMPLEX            ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           CAXPY, CCOPY, CHEMV, CLACN2, CPOTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, MAX, REAL
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH
      EXTERNAL           LSAME, SLAMCH
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
```

```
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -11
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CPORFS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = N + 1
      EPS = SLAMCH( 'Epsilon' )
      SAFMIN = SLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 140 J = 1, NRHS
*
```

```
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - A * X
*
         CALL CCOPY( N, B( 1, J ), 1, WORK, 1 )
         CALL CHEMV( UPLO, N, -ONE, A, LDA, X( 1, J ), 1, ONE, WORK, 1 )
*
*        Compute componentwise relative backward error from formula
*
*        max(i) ( abs(R(i)) / ( abs(A)*abs(X) + abs(B) )(i) )
*
*        where abs(Z) is the componentwise absolute value of the matrix
*        or vector Z.  If the i-th component of the denominator is less
*        than SAFE2, then SAFE1 is added to the i-th components of the
*        numerator and denominator before dividing.
*
         DO 30 I = 1, N
            RWORK( I ) = CABS1( B( I, J ) )
   30    CONTINUE
*
*        Compute abs(A)*abs(X) + abs(B).
*
         IF( UPPER ) THEN
            DO 50 K = 1, N
               S = ZERO
               XK = CABS1( X( K, J ) )
               DO 40 I = 1, K - 1
                  RWORK( I ) = RWORK( I ) + CABS1( A( I, K ) )*XK
                  S = S + CABS1( A( I, K ) )*CABS1( X( I, J ) )
   40          CONTINUE
               RWORK( K ) = RWORK( K ) + ABS( REAL( A( K, K ) ) )*XK + S
   50       CONTINUE
         ELSE
            DO 70 K = 1, N
               S = ZERO
               XK = CABS1( X( K, J ) )
               RWORK( K ) = RWORK( K ) + ABS( REAL( A( K, K ) ) )*XK
               DO 60 I = K + 1, N
                  RWORK( I ) = RWORK( I ) + CABS1( A( I, K ) )*XK
                  S = S + CABS1( A( I, K ) )*CABS1( X( I, J ) )
   60          CONTINUE
               RWORK( K ) = RWORK( K ) + S
   70       CONTINUE
         END IF
         S = ZERO
         DO 80 I = 1, N
```

```
            IF( RWORK( I ).GT.SAFE2 ) THEN
               S = MAX( S, CABS1( WORK( I ) ) / RWORK( I ) )
            ELSE
               S = MAX( S, ( CABS1( WORK( I ) )+SAFE1 ) /
     $             ( RWORK( I )+SAFE1 ) )
            END IF
   80    CONTINUE
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
*           1) The residual BERR(J) is larger than machine epsilon, and
*           2) BERR(J) decreased by at least a factor of 2 during the
*              last iteration, and
*           3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
*
*           Update solution and try again.
*
            CALL CPOTRS( UPLO, N, 1, AF, LDAF, WORK, N, INFO )
            CALL CAXPY( N, ONE, WORK, 1, X( 1, J ), 1 )
            LSTRES = BERR( J )
            COUNT = COUNT + 1
            GO TO 20
         END IF
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(A))*
*           ( abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
*          inv(A) is the inverse of A
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(A)*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(A)*abs(X) + abs(B) is less than SAFE2.
*
*        Use CLACN2 to estimate the infinity-norm of the matrix
*           inv(A) * diag(W),
*        where W = abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) )))
*
         DO 90 I = 1, N
```

```
            IF( RWORK( I ).GT.SAFE2 ) THEN
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I )
            ELSE
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I ) +
     $                      SAFE1
            END IF
   90    CONTINUE
*
         KASE = 0
  100    CONTINUE
         CALL CLACN2( N, WORK( N+1 ), WORK, FERR( J ), KASE, ISAVE )
         IF( KASE.NE.0 ) THEN
            IF( KASE.EQ.1 ) THEN
*
*              Multiply by diag(W)*inv(A**H).
*
               CALL CPOTRS( UPLO, N, 1, AF, LDAF, WORK, N, INFO )
               DO 110 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
  110          CONTINUE
            ELSE IF( KASE.EQ.2 ) THEN
*
*              Multiply by inv(A)*diag(W).
*
               DO 120 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
  120          CONTINUE
               CALL CPOTRS( UPLO, N, 1, AF, LDAF, WORK, N, INFO )
            END IF
            GO TO 100
         END IF
*
*        Normalize error.
*
         LSTRES = ZERO
         DO 130 I = 1, N
            LSTRES = MAX( LSTRES, CABS1( X( I, J ) ) )
  130    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  140 CONTINUE
*
      RETURN
*
*     End of CPORFS
*
      END
```

———-

72.8.4 lisp code

— cporfs.lisp —

```
(let* ((itmax 5)
       (zero 0.0)
       (one (f2cl-lib:cmplx 1.0 0.0))
       (two 2.0)
       (three 3.0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (single-float 0.0 0.0) zero)
           (type (f2cl-lib:complex8) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 3.0 3.0) three)
           (ignorable itmax zero one two three))
  (defun cporfs
         (uplo n nrhs a lda af ldaf b ldb$ x ldx ferr berr work rwork info)
    (declare (type (array single-float (*)) rwork berr ferr)
             (type (array f2cl-lib:complex8 (*)) work x b af a)
             (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (af f2cl-lib:complex8 af-%data% af-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (ferr single-float ferr-%data% ferr-%offset%)
         (berr single-float berr-%data% berr-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
          cabs1))
        (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
               (zdum #C(0.0 0.0)) (eps 0.0) (lstres 0.0) (s 0.0) (safe1 0.0)
               (safe2 0.0) (safmin 0.0) (xk 0.0) (count$ 0) (i 0) (j 0) (k 0)
               (kase 0) (nz 0) (upper nil))
          (declare (type (array f2cl-lib:integer4 (3)) isave)
                   (type (f2cl-lib:complex8) zdum)
                   (type (single-float) eps lstres s safe1 safe2 safmin xk)
                   (type (f2cl-lib:integer4) count$ i j k kase nz)
                   (type f2cl-lib:logical upper))
          (setf info 0)
          (setf upper
                 (multiple-value-bind (ret-val var-0 var-1)
                     (lsame uplo "U")
```

```
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -11)))
(cond
 ((/= info 0) (xerbla "CPORFS" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
              zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%)
              zero)
     label10))
  (go end_label)))
(setf nz (f2cl-lib:int-add n 1))
(setf eps (slamch "Epsilon"))
(setf safmin (slamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (ccopy n
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
```

```
     1 work 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (chemv uplo n (- one) a lda
     (f2cl-lib:array-slice x-%data% f2cl-lib:complex8 (1 j)
                           ((1 ldx) (1 *)) x-%offset%)
     1 one work 1)
  (declare (ignore var-2 var-3 var-5 var-6 var-8 var-9))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-4 (setf lda var-4))
  (when var-7 (setf one var-7)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
            (cabs1
             (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                            b-%offset%)))
   label30))
(cond
 (upper
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (setf s zero)
      (setf xk
              (cabs1
               (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                              x-%offset%)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (f2cl-lib:int-add k
                                          (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                               rwork-%offset%)
                  (+
                   (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                  rwork-%offset%)
                   (*
                    (cabs1
                     (f2cl-lib:fref a-%data% (i k)
                                    ((1 lda) (1 *)) a-%offset%))
                    xk)))
          (setf s
                  (+ s
```

```
                        (*
                         (cabs1
                          (f2cl-lib:fref a-%data% (i k)
                                         ((1 lda) (1 *))
                                         a-%offset%))
                         (cabs1
                          (f2cl-lib:fref x-%data% (i j)
                                         ((1 ldx) (1 *))
                                         x-%offset%)))))
          label40))
      (setf (f2cl-lib:fref rwork-%data% (k) ((1 *))
                           rwork-%offset%)
            (+
             (f2cl-lib:fref rwork-%data% (k) ((1 *))
                            rwork-%offset%)
             (*
              (abs
               (f2cl-lib:freal
                (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                               a-%offset%)))
              xk)
             s))
    label50)))
(t
 (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
               ((> k n) nil)
   (tagbody
     (setf s zero)
     (setf xk
            (cabs1
             (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                            x-%offset%)))
     (setf (f2cl-lib:fref rwork-%data% (k) ((1 *))
                          rwork-%offset%)
            (+
             (f2cl-lib:fref rwork-%data% (k) ((1 *))
                            rwork-%offset%)
             (*
              (abs
               (f2cl-lib:freal
                (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                               a-%offset%)))
              xk)))
     (f2cl-lib:fdo (i (f2cl-lib:int-add k 1)
                    (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                              rwork-%offset%)
               (+
```

```
                        (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                       rwork-%offset%)
                        (*
                         (cabs1
                          (f2cl-lib:fref a-%data% (i k)
                                         ((1 lda) (1 *)) a-%offset%))
                         xk)))
                (setf s
                        (+ s
                           (*
                            (cabs1
                             (f2cl-lib:fref a-%data% (i k)
                                            ((1 lda) (1 *))
                                            a-%offset%))
                            (cabs1
                             (f2cl-lib:fref x-%data% (i j)
                                            ((1 ldx) (1 *))
                                            x-%offset%)))))
           label60))
      (setf (f2cl-lib:fref rwork-%data% (k) ((1 *))
                           rwork-%offset%)
            (+
             (f2cl-lib:fref rwork-%data% (k) ((1 *))
                            rwork-%offset%)
             s))
     label70))))
(setf s zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
      (setf s
              (max s
                   (/
                    (cabs1
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%))
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%)))))
     (t
      (setf s
              (max s
                   (/
                    (+
                     (cabs1
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%))
                     safe1)
                    (+
```

```
                        (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                       rwork-%offset%)
                        safe1))))))
   label80))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (cpotrs uplo n 1 af ldaf work n info)
    (declare (ignore var-2 var-3 var-5))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf ldaf var-4))
    (when var-6 (setf n var-6))
    (when var-7 (setf info var-7)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (caxpy n one work 1
       (f2cl-lib:array-slice x-%data% f2cl-lib:complex8 (1 j)
                             ((1 ldx) (1 *)) x-%offset%)
       1)
    (declare (ignore var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))
    (when var-1 (setf one var-1)))
  (setf lstres
          (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
  (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
      (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                           rwork-%offset%)
              (+
               (cabs1
                (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%))
               (* nz eps
                  (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                 rwork-%offset%)))))
     (t
      (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                           rwork-%offset%)
              (+
               (cabs1
                (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%))
```

```
                    (* nz eps
                       (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                      rwork-%offset%))
                    safe1))))
     label90))
  (setf kase 0)
 label100
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (clacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             ((+ n 1)) ((1 *)) work-%offset%)
       work (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
       kase isave)
    (declare (ignore var-1 var-2 var-5))
    (setf n var-0)
    (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
            var-3)
    (setf kase var-4))
  (cond
   ((/= kase 0)
    (cond
     ((= kase 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (cpotrs uplo n 1 af ldaf work n info)
        (declare (ignore var-2 var-3 var-5))
        (when var-0 (setf uplo var-0))
        (when var-1 (setf n var-1))
        (when var-4 (setf ldaf var-4))
        (when var-6 (setf n var-6))
        (when var-7 (setf info var-7)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                (*
                 (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                rwork-%offset%)
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)))
         label110)))
     ((= kase 2)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                (*
                 (f2cl-lib:fref rwork-%data% (i) ((1 *))
```

```
                                          rwork-%offset%)
                              (f2cl-lib:fref work-%data% (i) ((1 *))
                                          work-%offset%)))
                     label120))
                  (multiple-value-bind
                      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
                      (cpotrs uplo n 1 af ldaf work n info)
                    (declare (ignore var-2 var-3 var-5))
                    (when var-0 (setf uplo var-0))
                    (when var-1 (setf n var-1))
                    (when var-4 (setf ldaf var-4))
                    (when var-6 (setf n var-6))
                    (when var-7 (setf info var-7)))))
                (go label100)))
              (setf lstres zero)
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i n) nil)
                (tagbody
                  (setf lstres
                          (max lstres
                               (cabs1
                                (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                               x-%offset%))))
                 label130))
              (if (/= lstres zero)
                  (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                          (/
                           (f2cl-lib:fref ferr-%data% (j) ((1 *))
                                          ferr-%offset%)
                           lstres)))
            label140))
          (go end_label)
         end_label
          (return
           (values uplo n nil nil lda nil ldaf nil nil nil nil nil nil nil nil
                   info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cporfs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
```

```
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::complex8
   (*))
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::uplo
  fortran-to-lisp::n nil nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldaf nil nil nil
  nil nil nil nil nil
  fortran-to-lisp::info)
:calls '(fortran-to-lisp::clacn2))))
```

———-

72.9 cporfsx

72.9.1 Axiom unit tests

— **cporfsx.input** —

```
)set break resume
)sys rm -f cporfsx.output
)spool cporfsx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

72.9.2 Axiom help page [Demm05]

— cporfsx.help —

```
==================================================================
cporfsx examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cporfsx (UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF, S, B, LDB,
           X, LDX, RCOND, BERR, N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP,
           NPARAMS, PARAMS, WORK, RWORK, INFO)
           CPORFSX

Function/Subroutine Documentation
   subroutine cporfsx (character UPLO, character EQUED, integer N, integer
       NRHS, complex, dimension( lda, * ) A, integer LDA, complex, dimension(
       ldaf, * ) AF, integer LDAF, real, dimension( * ) S, complex, dimension(
       ldb, * ) B, integer LDB, complex, dimension( ldx, * ) X, integer LDX,
       real RCOND, real, dimension( * ) BERR, integer N_ERR_BNDS, real,
       dimension( nrhs, * ) ERR_BNDS_NORM, real, dimension( nrhs, * )
       ERR_BNDS_COMP, integer NPARAMS, real, dimension(*) PARAMS, complex,
       dimension( * ) WORK, real, dimension( * ) RWORK, integer INFO)
       CPORFSX

Purpose:
   CPORFSX improves the computed solution to a system of linear
   equations when the coefficient matrix is symmetric positive
   definite, and provides error bounds and backward error estimates
   for the solution.  In addition to normwise error bound, the code
   provides maximum componentwise error bound if possible.  See
   comments for ERR_BNDS_NORM and ERR_BNDS_COMP for details of the
   error bounds.

   The original system of linear equations may have been equilibrated
   before calling this routine, as described by arguments EQUED and S
   below. In this case, the solution and error bounds returned are
   for the original unequilibrated system.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.
```

```
Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done to A
   before calling this routine. This is needed to compute
   the solution and error bounds correctly.
   = 'N':  No equilibration
   = 'Y':  Both row and column equilibration, i.e., A has been
           replaced by diag(S) * A * diag(S).
           The right hand side B has been changed accordingly.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
   upper triangular part of A contains the upper triangular part
   of the matrix A, and the strictly lower triangular part of A
   is not referenced.  If UPLO = 'L', the leading N-by-N lower
   triangular part of A contains the lower triangular part of
   the matrix A, and the strictly upper triangular part of A is
   not referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is COMPLEX array, dimension (LDAF,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**T*U or A = L*L**T, as computed by SPOTRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).
```

```
S
  S is REAL array, dimension (N)
  The row scale factors for A.  If EQUED = 'Y', A is multiplied on
  the left and right by diag(S).  S is an input argument if FACT =
  'F'; otherwise, S is an output argument.  If FACT = 'F' and EQUED
  = 'Y', each element of S must be positive.  If S is output, each
  element of S is a power of the radix. If S is input, each element
  of S should be a power of the radix to ensure a reliable solution
  and error estimates. Scaling by powers of the radix does not cause
  rounding errors unless the result underflows or overflows.
  Rounding errors during scaling lead to refining with a matrix that
  is not equivalent to the input matrix, producing error estimates
  that may not be reliable.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by SGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

BERR
  BERR is REAL array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
```

```
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
           max_j (abs(XTRUE(j,i) - X(j,i)))
            ------------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * slamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
    reciprocal condition number.  Compared with the threshold
    sqrt(n) * slamch('Epsilon') to determine if the error
    estimate is "guaranteed". These reciprocal condition
    numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
    appropriately scaled matrix Z.
    Let Z = S*A, where S scales each row by a power of the
    radix so all absolute row sums of Z are approximately 1.

ERR_BNDS_COMP
  ERR_BNDS_COMP is REAL array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
```

```
                 abs(XTRUE(j,i) - X(j,i))
            max_j ----------------------
                       abs(X(j,i))
```

The array is indexed by the right-hand side i (on which the componentwise relative error depends), and the type of error information as described below. There currently are up to three pieces of information returned for each right-hand side. If componentwise accuracy is not requested (PARAMS(3) = 0.0), then ERR_BNDS_COMP is not accessed. If N_ERR_BNDS .LT. 3, then at most the first (:,N_ERR_BNDS) entries are returned.

The first index in ERR_BNDS_COMP(i,:) corresponds to the ith right-hand side.

The second index in ERR_BNDS_COMP(:,err) contains the following three fields:

err = 1 "Trust/don't trust" boolean. Trust the answer if the reciprocal condition number is less than the threshold sqrt(n) * slamch('Epsilon').

err = 2 "Guaranteed" error bound: The estimated forward error, almost certainly within a factor of 10 of the true error so long as the next entry is greater than the threshold sqrt(n) * slamch('Epsilon'). This error bound should only be trusted if the previous boolean is true.

err = 3 Reciprocal condition number: Estimated componentwise reciprocal condition number. Compared with the threshold sqrt(n) * slamch('Epsilon') to determine if the error estimate is "guaranteed". These reciprocal condition numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some appropriately scaled matrix Z.
Let Z = S*(A*diag(x)), where x is the solution for the current right-hand side and S scales each row of A*diag(x) by a power of the radix so all absolute row sums of Z are approximately 1.

NPARAMS
NPARAMS is INTEGER
Specifies the number of parameters set in PARAMS. If .LE. 0, the PARAMS array is never referenced and default values are used.

PARAMS
PARAMS is REAL array, dimension NPARAMS
Specifies algorithm parameters. If an entry is .LT. 0.0, then that entry will be filled with default value used for that parameter. Only positions up to NPARAMS are accessed; defaults are used for higher-numbered parameters.

```
  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
  (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is COMPLEX array, dimension (2*N)

RWORK
  RWORK is REAL array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is
  guaranteed.
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
      has been completed, but the factor U is exactly singular, so
      the solution and error bounds could not be computed. RCOND = 0
      is returned.
  = N+J: The solution corresponding to the Jth right-hand side is
      not guaranteed. The solutions corresponding to other right-
      hand sides K with K > J may not be guaranteed as well, but
      only the first such right-hand side is reported. If a small
      componentwise error is not requested (PARAMS(3) = 0.0) then
      the Jth right-hand side is the first with a normwise error
      bound that is not guaranteed (the smallest J such
      that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
      the Jth right-hand side is the first with either a normwise or
      componentwise error bound that is not guaranteed (the smallest
```

```
      J such that either ERR_BNDS_NORM(J,1) = 0.0 or
      ERR_BNDS_COMP(J,1) = 0.0). See the definition of
      ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
      about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

———-

72.9.3 fortran code

— **cporfsx.f** —

```
*  ====================================================================
      SUBROUTINE CPORFSX( UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF, S, B,
     $                    LDB, X, LDX, RCOND, BERR, N_ERR_BNDS,
     $                    ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $                    WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO, EQUED
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX, * ), WORK( * )
      REAL               RWORK( * ), S( * ), PARAMS(*), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               ITREF_DEFAULT, ITHRESH_DEFAULT,
     $                   COMPONENTWISE_DEFAULT
      REAL               RTHRESH_DEFAULT, DZTHRESH_DEFAULT
      PARAMETER          ( ITREF_DEFAULT = 1.0 )
      PARAMETER          ( ITHRESH_DEFAULT = 10.0 )
      PARAMETER          ( COMPONENTWISE_DEFAULT = 1.0 )
```

```
      PARAMETER          ( RTHRESH_DEFAULT = 0.5 )
      PARAMETER          ( DZTHRESH_DEFAULT = 0.25 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. Local Scalars ..
      CHARACTER(1)       NORM
      LOGICAL            RCEQU
      INTEGER            J, PREC_TYPE, REF_TYPE
      INTEGER            N_NORMS
      REAL               ANORM, RCOND_TMP
      REAL               ILLRCOND_THRESH, ERR_LBND, CWISE_WRONG
      LOGICAL            IGNORE_CWISE
      INTEGER            ITHRESH
      REAL               RTHRESH, UNSTABLE_THRESH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, CPOCON, CLA_PORFSX_EXTENDED
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT, TRANSFER
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, BLAS_FPINFO_X, ILATRANS, ILAPREC
      EXTERNAL           SLAMCH, CLANHE, CLA_PORCOND_X, CLA_PORCOND_C
      REAL               SLAMCH, CLANHE, CLA_PORCOND_X, CLA_PORCOND_C
      LOGICAL            LSAME
      INTEGER            BLAS_FPINFO_X
      INTEGER            ILATRANS, ILAPREC
*     ..
*     .. Executable Statements ..
*
*     Check the input parameters.
*
      INFO = 0
      REF_TYPE = INT( ITREF_DEFAULT )
      IF ( NPARAMS .GE. LA_LINRX_ITREF_I ) THEN
         IF ( PARAMS( LA_LINRX_ITREF_I ) .LT. 0.0 ) THEN
            PARAMS( LA_LINRX_ITREF_I ) = ITREF_DEFAULT
         ELSE
            REF_TYPE = PARAMS( LA_LINRX_ITREF_I )
         END IF
      END IF
```

```
*
*     Set default parameters.
*
      ILLRCOND_THRESH = REAL( N ) * SLAMCH( 'Epsilon' )
      ITHRESH = INT( ITHRESH_DEFAULT )
      RTHRESH = RTHRESH_DEFAULT
      UNSTABLE_THRESH = DZTHRESH_DEFAULT
      IGNORE_CWISE = COMPONENTWISE_DEFAULT .EQ. 0.0
*
      IF ( NPARAMS.GE.LA_LINRX_ITHRESH_I ) THEN
         IF ( PARAMS(LA_LINRX_ITHRESH_I ).LT.0.0 ) THEN
            PARAMS( LA_LINRX_ITHRESH_I ) = ITHRESH
         ELSE
            ITHRESH = INT( PARAMS( LA_LINRX_ITHRESH_I ) )
         END IF
      END IF
      IF ( NPARAMS.GE.LA_LINRX_CWISE_I ) THEN
         IF ( PARAMS(LA_LINRX_CWISE_I ).LT.0.0 ) THEN
            IF ( IGNORE_CWISE ) THEN
               PARAMS( LA_LINRX_CWISE_I ) = 0.0
            ELSE
               PARAMS( LA_LINRX_CWISE_I ) = 1.0
            END IF
         ELSE
            IGNORE_CWISE = PARAMS( LA_LINRX_CWISE_I ) .EQ. 0.0
         END IF
      END IF
      IF ( REF_TYPE .EQ. 0 .OR. N_ERR_BNDS .EQ. 0 ) THEN
         N_NORMS = 0
      ELSE IF ( IGNORE_CWISE ) THEN
         N_NORMS = 1
      ELSE
         N_NORMS = 2
      END IF
*
      RCEQU = LSAME( EQUED, 'Y' )
*
*     Test input parameters.
*
      IF (.NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
        INFO = -1
      ELSE IF( .NOT.RCEQU .AND. .NOT.LSAME( EQUED, 'N' ) ) THEN
        INFO = -2
      ELSE IF( N.LT.0 ) THEN
        INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
        INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
        INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
```

```
        INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
        INFO = -11
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
        INFO = -13
      END IF
      IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'CPORFSX', -INFO )
        RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         RCOND = 1.0
         DO J = 1, NRHS
            BERR( J ) = 0.0
            IF ( N_ERR_BNDS .GE. 1 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
            END IF
            IF ( N_ERR_BNDS .GE. 2 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 0.0
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 0.0
            END IF
            IF ( N_ERR_BNDS .GE. 3 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 1.0
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 1.0
            END IF
         END DO
         RETURN
      END IF
*
*     Default to failure.
*
      RCOND = 0.0
      DO J = 1, NRHS
         BERR( J ) = 1.0
         IF ( N_ERR_BNDS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
            ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
         END IF
         IF ( N_ERR_BNDS .GE. 2 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
         END IF
         IF ( N_ERR_BNDS .GE. 3 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 0.0
            ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 0.0
         END IF
```

```
      END DO
*
*     Compute the norm of A and the reciprocal of the condition
*     number of A.
*
      NORM = 'I'
      ANORM = CLANHE( NORM, UPLO, N, A, LDA, RWORK )
      CALL CPOCON( UPLO, N, AF, LDAF, ANORM, RCOND, WORK, RWORK,
     $     INFO )
*
*     Perform refinement on each right-hand side
*
      IF ( REF_TYPE .NE. 0 ) THEN

         PREC_TYPE = ILAPREC( 'D' )

         CALL CLA_PORFSX_EXTENDED( PREC_TYPE, UPLO, N,
     $        NRHS, A, LDA, AF, LDAF, RCEQU, S, B,
     $        LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM, ERR_BNDS_COMP,
     $        WORK, RWORK, WORK(N+1),
     $        TRANSFER (RWORK(1:2*N), (/ (ZERO, ZERO) /), N), RCOND,
     $        ITHRESH, RTHRESH, UNSTABLE_THRESH, IGNORE_CWISE,
     $        INFO )
      END IF

      ERR_LBND = MAX( 10.0, SQRT( REAL( N ) ) ) * SLAMCH( 'Epsilon' )
      IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 1 ) THEN
*
*     Compute scaled normwise condition number cond(A*C).
*
         IF ( RCEQU ) THEN
            RCOND_TMP = CLA_PORCOND_C( UPLO, N, A, LDA, AF, LDAF,
     $           S, .TRUE., INFO, WORK, RWORK )
         ELSE
            RCOND_TMP = CLA_PORCOND_C( UPLO, N, A, LDA, AF, LDAF,
     $           S, .FALSE., INFO, WORK, RWORK )
         END IF
         DO J = 1, NRHS
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .GT. 1.0 )
     $           ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 0.0
```

```
            IF ( INFO .LE. N ) INFO = N + J
         ELSE IF ( ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .LT. ERR_LBND )
     $     THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = ERR_LBND
            ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0
         END IF
*
*     Save the condition number.
*
         IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = RCOND_TMP
         END IF

      END DO
     END IF

     IF (N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 2) THEN
*
*     Compute componentwise condition number cond(A*diag(Y(:,J))) for
*     each right-hand side using the current solution as an estimate of
*     the true solution.  If the componentwise error estimate is too
*     large, then the solution is a lousy estimate of truth and the
*     estimated RCOND may be too optimistic.  To avoid misleading users,
*     the inverse condition number is set to 0.0 when the estimated
*     cwise error is at least CWISE_WRONG.
*
      CWISE_WRONG = SQRT( SLAMCH( 'Epsilon' ) )
      DO J = 1, NRHS
         IF (ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .LT. CWISE_WRONG )
     $     THEN
            RCOND_TMP = CLA_PORCOND_X( UPLO, N, A, LDA, AF, LDAF,
     $         X(1,J), INFO, WORK, RWORK )
         ELSE
            RCOND_TMP = 0.0
         END IF
*
*     Cap the error at 1.0.
*
         IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .GT. 1.0 )
     $           ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
*
*     Threshold the error (see LAWN).
*
         IF (RCOND_TMP .LT. ILLRCOND_THRESH) THEN
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0
            ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 0.0
            IF ( PARAMS( LA_LINRX_CWISE_I ) .EQ. 1.0
     $              .AND. INFO.LT.N + J ) INFO = N + J
         ELSE IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I )
```

```
     $              .LT. ERR_LBND ) THEN
                ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = ERR_LBND
                ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0
             END IF
*
*     Save the condition number.
*
             IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
                ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = RCOND_TMP
             END IF

          END DO
      END IF
*
      RETURN
*
*     End of CPORFSX
*
      END
```

———-

72.9.4 lisp code

— **cporfsx.lisp** —

———-

72.10 cpotf2

72.10.1 Axiom unit tests

— **cpotf2.input** —

```
)set break resume
)sys rm -f cpotf2.output
)spool cpotf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

72.10.2 Axiom help page

— cpotf2.help —

```
=====================================================================
cpotf2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cpotf2 (UPLO, N, A, LDA, INFO)
           CPOTF2 computes the Cholesky factorization of a symmetric/Hermitian
           positive definite matrix (unblocked algorithm).

Function/Subroutine Documentation
   subroutine cpotf2 (character UPLO, integer N, complex, dimension( lda, * )
       A, integer LDA, integer INFO)
       CPOTF2 computes the Cholesky factorization of a symmetric/Hermitian
       positive definite matrix (unblocked algorithm).

Purpose:
   CPOTF2 computes the Cholesky factorization of a complex Hermitian
   positive definite matrix A.

   The factorization has the form
      A = U**H * U ,  if UPLO = 'U', or
      A = L  * L**H,  if UPLO = 'L',
   where U is an upper triangular matrix and L is lower triangular.

   This is the unblocked version of the algorithm, calling Level 2 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   Hermitian matrix A is stored.
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
```

```
  The order of the matrix A.  N >= 0.

A
  A is COMPLEX array, dimension (LDA,N)
  On entry, the Hermitian matrix A.  If UPLO = 'U', the leading
  n by n upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading n by n lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if INFO = 0, the factor U or L from the Cholesky
  factorization A = U**H *U  or A = L*L**H.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -k, the k-th argument had an illegal value
  > 0: if INFO = k, the leading minor of order k is not
       positive definite, and the factorization could not be completed.
```

———-

72.10.3 fortran code

— **cpotf2.f** —

```
*  =====================================================================
      SUBROUTINE CPOTF2( UPLO, N, A, LDA, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * )
*     ..
```

```
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
      COMPLEX            CONE
      PARAMETER          ( CONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            J
      REAL               AJJ
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, SISNAN
      COMPLEX            CDOTC
      EXTERNAL           LSAME, CDOTC, SISNAN
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEMV, CLACGV, CSSCAL, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, REAL, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CPOTF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
```

```
*        Compute the Cholesky factorization A = U**H *U.
*
         DO 10 J = 1, N
*
*           Compute U(J,J) and test for non-positive-definiteness.
*
            AJJ = REAL( A( J, J ) ) - CDOTC( J-1, A( 1, J ), 1,
     $            A( 1, J ), 1 )
            IF( AJJ.LE.ZERO.OR.SISNAN( AJJ ) ) THEN
               A( J, J ) = AJJ
               GO TO 30
            END IF
            AJJ = SQRT( AJJ )
            A( J, J ) = AJJ
*
*           Compute elements J+1:N of row J.
*
            IF( J.LT.N ) THEN
               CALL CLACGV( J-1, A( 1, J ), 1 )
               CALL CGEMV( 'Transpose', J-1, N-J, -CONE, A( 1, J+1 ),
     $                     LDA, A( 1, J ), 1, CONE, A( J, J+1 ), LDA )
               CALL CLACGV( J-1, A( 1, J ), 1 )
               CALL CSSCAL( N-J, ONE / AJJ, A( J, J+1 ), LDA )
            END IF
   10    CONTINUE
      ELSE
*
*        Compute the Cholesky factorization A = L*L**H.
*
         DO 20 J = 1, N
*
*           Compute L(J,J) and test for non-positive-definiteness.
*
            AJJ = REAL( A( J, J ) ) - CDOTC( J-1, A( J, 1 ), LDA,
     $            A( J, 1 ), LDA )
            IF( AJJ.LE.ZERO.OR.SISNAN( AJJ ) ) THEN
               A( J, J ) = AJJ
               GO TO 30
            END IF
            AJJ = SQRT( AJJ )
            A( J, J ) = AJJ
*
*           Compute elements J+1:N of column J.
*
            IF( J.LT.N ) THEN
               CALL CLACGV( J-1, A( J, 1 ), LDA )
               CALL CGEMV( 'No transpose', N-J, J-1, -CONE, A( J+1, 1 ),
     $                     LDA, A( J, 1 ), LDA, CONE, A( J+1, J ), 1 )
               CALL CLACGV( J-1, A( J, 1 ), LDA )
               CALL CSSCAL( N-J, ONE / AJJ, A( J+1, J ), 1 )
```

```
            END IF
   20    CONTINUE
      END IF
      GO TO 40
*
   30 CONTINUE
      INFO = J
*
   40 CONTINUE
      RETURN
*
*     End of CPOTF2
*
      END
```

———-

72.10.4 lisp code

— cpotf2.lisp —

```
(let* ((one 1.0) (zero 0.0) (cone (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (type (f2cl-lib:complex8) cone)
           (ignorable one zero cone))
  (defun cpotf2 (uplo n a lda info)
    (declare (type (array f2cl-lib:complex8 (*)) a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%))
      (prog ((ajj 0.0) (j 0) (upper nil))
        (declare (type (single-float) ajj)
                 (type (f2cl-lib:integer4) j)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
```

```
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4)))
(cond
 ((/= info 0) (xerbla "CPOTF2" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(cond
 (upper
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf ajj
              (coerce
               (realpart
                (-
                 (f2cl-lib:freal
                  (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                 a-%offset%))
                 (cdotc (f2cl-lib:int-sub j 1)
                  (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                        (1 j) ((1 lda) (1 *))
                                        a-%offset%)
                  1
                  (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                        (1 j) ((1 lda) (1 *))
                                        a-%offset%)
                  1)))
               'single-float))
      (cond
       ((or (<= ajj zero)
            (multiple-value-bind (ret-val var-0)
                (sisnan ajj)
              (declare (ignore))
              (when var-0 (setf ajj var-0))
              ret-val))
        (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
                (coerce ajj 'f2cl-lib:complex8))
        (go label30)))
      (setf ajj (f2cl-lib:fsqrt ajj))
      (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
              (coerce ajj 'f2cl-lib:complex8))
      (cond
       ((< j n)
```

```
          (clacgv (f2cl-lib:int-sub j 1)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 j)
                                 ((1 lda) (1 *)) a-%offset%)
           1)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10)
              (cgemv "Transpose" (f2cl-lib:int-sub j 1)
               (f2cl-lib:int-sub n j) (- cone)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                     (1 (f2cl-lib:int-add j 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 j)
                                     ((1 lda) (1 *)) a-%offset%)
               1 cone
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                     (j (f2cl-lib:int-add j 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda)
            (declare
             (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7 var-9))
            (when var-5 (setf lda var-5))
            (when var-8 (setf cone var-8))
            (when var-10 (setf lda var-10)))
          (clacgv (f2cl-lib:int-sub j 1)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 j)
                                 ((1 lda) (1 *)) a-%offset%)
           1)
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (csscal (f2cl-lib:int-sub n j) (/ one ajj)
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                     (j (f2cl-lib:int-add j 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda)
            (declare (ignore var-0 var-1 var-2))
            (when var-3 (setf lda var-3)))))
      label10)))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf ajj
               (coerce
                (realpart
                 (-
                  (f2cl-lib:freal
                   (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                  a-%offset%))
                  (multiple-value-bind
```

```
                (ret-val var-0 var-1 var-2 var-3 var-4)
                (cdotc (f2cl-lib:int-sub j 1)
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                       (j 1) ((1 lda) (1 *))
                                       a-%offset%)
                 lda
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                       (j 1) ((1 lda) (1 *))
                                       a-%offset%)
                 lda)
              (declare (ignore var-0 var-1 var-3))
              (when var-2 (setf lda var-2))
              (when var-4 (setf lda var-4))
              ret-val)))
         'single-float))
(cond
 ((or (<= ajj zero)
      (multiple-value-bind (ret-val var-0)
          (sisnan ajj)
        (declare (ignore))
        (when var-0 (setf ajj var-0))
        ret-val))
  (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
          (coerce ajj 'f2cl-lib:complex8))
  (go label30)))
(setf ajj (f2cl-lib:fsqrt ajj))
(setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
        (coerce ajj 'f2cl-lib:complex8))
(cond
 ((< j n)
  (clacgv (f2cl-lib:int-sub j 1)
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (j 1)
                         ((1 lda) (1 *)) a-%offset%)
   lda)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (cgemv "No transpose" (f2cl-lib:int-sub n j)
       (f2cl-lib:int-sub j 1) (- cone)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ j 1) 1) ((1 lda) (1 *))
                             a-%offset%)
       lda
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (j 1)
                             ((1 lda) (1 *)) a-%offset%)
       lda cone
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ j 1) j) ((1 lda) (1 *))
                             a-%offset%)
       1)
```

```
                  (declare
                   (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9 var-10))
                  (when var-5 (setf lda var-5))
                  (when var-7 (setf lda var-7))
                  (when var-8 (setf cone var-8)))
                (clacgv (f2cl-lib:int-sub j 1)
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (j 1)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
                (csscal (f2cl-lib:int-sub n j) (/ one ajj)
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 ((+ j 1) j)
                                       ((1 lda) (1 *)) a-%offset%)
                 1)))
              label20))))
         (go label40)
        label30
         (setf info j)
        label40
         (go end_label)
        end_label
         (return (values uplo nil nil lda info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cpotf2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::lda
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::clacgv))))
```

———-

72.11 cpotrf

72.11.1 Axiom unit tests

— cpotrf.input —

```
)set break resume
)sys rm -f cpotrf.output
)spool cpotrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

72.11.2 Axiom help page

— cpotrf.help —

```
======================================================================
cpotrf examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cpotrf (UPLO, N, A, LDA, INFO)
           CPOTRF

Function/Subroutine Documentation
   subroutine cpotrf (character UPLO, integer N, complex, dimension( lda, * )
       A, integer LDA, integer INFO)
       CPOTRF

Purpose:
   CPOTRF computes the Cholesky factorization of a complex Hermitian
   positive definite matrix A.

   The factorization has the form
      A = U**H * U,  if UPLO = 'U', or
```

```
      A = L  * L**H,  if UPLO = 'L',
   where U is an upper triangular matrix and L is lower triangular.

   This is the block version of the algorithm, calling Level 3 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the Hermitian matrix A.  If UPLO = 'U', the leading
   N-by-N upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading N-by-N lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

   On exit, if INFO = 0, the factor U or L from the Cholesky
   factorization A = U**H*U or A = L*L**H.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, the leading minor of order i is not
         positive definite, and the factorization could not be completed.
```

———-

72.11.3 fortran code

— cpotrf.f —

```
*  =====================================================================
```

```
      SUBROUTINE CPOTRF( UPLO, N, A, LDA, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      COMPLEX            CONE
      PARAMETER          ( ONE = 1.0E+0, CONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            J, JB, NB
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGEMM, CHERK, CPOTRF2, CTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
```

```
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CPOTRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Determine the block size for this environment.
*
      NB = ILAENV( 1, 'CPOTRF', UPLO, N, -1, -1, -1 )
      IF( NB.LE.1 .OR. NB.GE.N ) THEN
*
*        Use unblocked code.
*
         CALL CPOTRF2( UPLO, N, A, LDA, INFO )
      ELSE
*
*        Use blocked code.
*
         IF( UPPER ) THEN
*
*           Compute the Cholesky factorization A = U**H *U.
*
            DO 10 J = 1, N, NB
*
*              Update and factorize the current diagonal block and test
*              for non-positive-definiteness.
*
               JB = MIN( NB, N-J+1 )
               CALL CHERK( 'Upper', 'Conjugate transpose', JB, J-1,
     $                     -ONE, A( 1, J ), LDA, ONE, A( J, J ), LDA )
               CALL CPOTRF2( 'Upper', JB, A( J, J ), LDA, INFO )
               IF( INFO.NE.0 )
     $            GO TO 30
               IF( J+JB.LE.N ) THEN
*
*                 Compute the current block row.
*
                  CALL CGEMM( 'Conjugate transpose', 'No transpose', JB,
     $                        N-J-JB+1, J-1, -CONE, A( 1, J ), LDA,
     $                        A( 1, J+JB ), LDA, CONE, A( J, J+JB ),
     $                        LDA )
                  CALL CTRSM( 'Left', 'Upper', 'Conjugate transpose',
     $                        'Non-unit', JB, N-J-JB+1, CONE, A( J, J ),
     $                        LDA, A( J, J+JB ), LDA )
               END IF
   10       CONTINUE
```

```
*
         ELSE
*
*           Compute the Cholesky factorization A = L*L**H.
*
            DO 20 J = 1, N, NB
*
*              Update and factorize the current diagonal block and test
*              for non-positive-definiteness.
*
               JB = MIN( NB, N-J+1 )
               CALL CHERK( 'Lower', 'No transpose', JB, J-1, -ONE,
     $                     A( J, 1 ), LDA, ONE, A( J, J ), LDA )
               CALL CPOTRF2( 'Lower', JB, A( J, J ), LDA, INFO )
               IF( INFO.NE.0 )
     $            GO TO 30
               IF( J+JB.LE.N ) THEN
*
*                 Compute the current block column.
*
                  CALL CGEMM( 'No transpose', 'Conjugate transpose',
     $                        N-J-JB+1, JB, J-1, -CONE, A( J+JB, 1 ),
     $                        LDA, A( J, 1 ), LDA, CONE, A( J+JB, J ),
     $                        LDA )
                  CALL CTRSM( 'Right', 'Lower', 'Conjugate transpose',
     $                        'Non-unit', N-J-JB+1, JB, CONE, A( J, J ),
     $                        LDA, A( J+JB, J ), LDA )
               END IF
   20       CONTINUE
         END IF
      END IF
      GO TO 40
*
   30 CONTINUE
      INFO = INFO + J - 1
*
   40 CONTINUE
      RETURN
*
*     End of CPOTRF
*
      END
```

———-

72.11.4 lisp code

— cpotrf.lisp —

```
(let* ((one 1.0) (cone (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (single-float 1.0 1.0) one)
           (type (f2cl-lib:complex8) cone)
           (ignorable one cone))
  (defun cpotrf (uplo n a lda info)
    (declare (type (array f2cl-lib:complex8 (*)) a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%))
      (prog ((j 0) (jb 0) (nb 0) (upper nil))
        (declare (type (f2cl-lib:integer4) j jb nb)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -4)))
        (cond
         ((/= info 0) (xerbla "CPOTRF" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (= n 0)
            (go end_label))
        (setf nb
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (ilaenv 1 "CPOTRF" uplo n -1 -1 -1)
                  (declare (ignore var-0 var-1 var-4 var-5 var-6))
                  (when var-2 (setf uplo var-2))
                  (when var-3 (setf n var-3))
                  ret-val))
        (cond
         ((or (<= nb 1) (>= nb n))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (cpotrf2 uplo n a lda info)
```

```
    (declare (ignore var-2))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf lda var-3))
    (when var-4 (setf info var-4))))
  (t
   (cond
    (upper
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                   ((> j n) nil)
       (tagbody
         (setf jb
                 (min (the f2cl-lib:integer4 nb)
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add (f2cl-lib:int-sub n j)
                                             1))))
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9)
             (cherk "Upper" "Conjugate transpose" jb
              (f2cl-lib:int-sub j 1) (- one)
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 j)
                                    ((1 lda) (1 *)) a-%offset%)
              lda one
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (j j)
                                    ((1 lda) (1 *)) a-%offset%)
              lda)
           (declare (ignore var-0 var-1 var-3 var-4 var-5 var-8))
           (when var-2 (setf jb var-2))
           (when var-6 (setf lda var-6))
           (when var-7 (setf one var-7))
           (when var-9 (setf lda var-9)))
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (cpotrf2 "Upper" jb
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (j j)
                                    ((1 lda) (1 *)) a-%offset%)
              lda info)
           (declare (ignore var-0 var-2))
           (when var-1 (setf jb var-1))
           (when var-3 (setf lda var-3))
           (when var-4 (setf info var-4)))
         (if (/= info 0)
             (go label30))
         (cond
          ((<= (f2cl-lib:int-add j jb) n)
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                var-9 var-10 var-11 var-12)
               (cgemm "Conjugate transpose" "No transpose" jb
                (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1)
```

```
                (f2cl-lib:int-sub j 1) (- cone)
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (1 j)
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                      (1 (f2cl-lib:int-add j jb))
                                      ((1 lda) (1 *)) a-%offset%)
                lda cone
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                      (j (f2cl-lib:int-add j jb))
                                      ((1 lda) (1 *)) a-%offset%)
                lda)
              (declare
               (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-8 var-11))
              (when var-2 (setf jb var-2))
              (when var-7 (setf lda var-7))
              (when var-9 (setf lda var-9))
              (when var-10 (setf cone var-10))
              (when var-12 (setf lda var-12)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10)
                (ctrsm "Left" "Upper" "Conjugate transpose" "Non-unit" jb
                 (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) cone
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (j j)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                       (j (f2cl-lib:int-add j jb))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9))
              (when var-4 (setf jb var-4))
              (when var-6 (setf cone var-6))
              (when var-8 (setf lda var-8))
              (when var-10 (setf lda var-10)))))
        label10)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                   ((> j n) nil)
       (tagbody
         (setf jb
                 (min (the f2cl-lib:integer4 nb)
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add (f2cl-lib:int-sub n j)
                                             1))))
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9)
```

```
    (cherk "Lower" "No transpose" jb (f2cl-lib:int-sub j 1)
     (- one)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (j 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda one
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (j j)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-3 var-4 var-5 var-8))
  (when var-2 (setf jb var-2))
  (when var-6 (setf lda var-6))
  (when var-7 (setf one var-7))
  (when var-9 (setf lda var-9)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (cpotrf2 "Lower" jb
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (j j)
                           ((1 lda) (1 *)) a-%offset%)
     lda info)
  (declare (ignore var-0 var-2))
  (when var-1 (setf jb var-1))
  (when var-3 (setf lda var-3))
  (when var-4 (setf info var-4)))
(if (/= info 0)
    (go label30))
(cond
 ((<= (f2cl-lib:int-add j jb) n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (cgemm "No transpose" "Conjugate transpose"
       (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) jb
       (f2cl-lib:int-sub j 1) (- cone)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ j jb) 1) ((1 lda) (1 *))
                             a-%offset%)
       lda
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (j 1)
                             ((1 lda) (1 *)) a-%offset%)
       lda cone
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                             ((+ j jb) j) ((1 lda) (1 *))
                             a-%offset%)
       lda)
    (declare
     (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-8 var-11))
    (when var-3 (setf jb var-3))
    (when var-7 (setf lda var-7))
    (when var-9 (setf lda var-9))
    (when var-10 (setf cone var-10))
    (when var-12 (setf lda var-12)))
```

```
                  (multiple-value-bind
                      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                       var-9 var-10)
                      (ctrsm "Right" "Lower" "Conjugate transpose" "Non-unit"
                       (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) jb cone
                       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8 (j j)
                                             ((1 lda) (1 *)) a-%offset%)
                       lda
                       (f2cl-lib:array-slice a-%data% f2cl-lib:complex8
                                             ((+ j jb) j) ((1 lda) (1 *))
                                             a-%offset%)
                       lda)
                    (declare
                     (ignore var-0 var-1 var-2 var-3 var-4 var-7 var-9))
                    (when var-5 (setf jb var-5))
                    (when var-6 (setf cone var-6))
                    (when var-8 (setf lda var-8))
                    (when var-10 (setf lda var-10)))))
                label20))))))
        (go label40)
       label30
        (setf info (f2cl-lib:int-sub (f2cl-lib:int-add info j) 1))
       label40
        (go end_label)
       end_label
        (return (values uplo n nil lda info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cpotrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

72.12 cpotrf2

72.12.1 Axiom unit tests

— cpotrf2.input —

```
)set break resume
)sys rm -f cpotrf2.output
)spool cpotrf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

72.12.2 Axiom help page

— cpotrf2.help —

```
======================================================================
cpotrf2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       recursive subroutine cpotrf2 (UPLO, N, A, LDA, INFO)
           CPOTRF2

Function/Subroutine Documentation
   recursive subroutine cpotrf2 (character UPLO, integer N, complex,
       dimension( lda, * ) A, integer LDA, integer INFO)
       CPOTRF2

Purpose:
   CPOTRF2 computes the Cholesky factorization of a real symmetric
   positive definite matrix A using the recursive algorithm.

   The factorization has the form
      A = U**H * U,  if UPLO = 'U', or
```

```
      A = L  * L**H,  if UPLO = 'L',
   where U is an upper triangular matrix and L is lower triangular.

   This is the recursive version of the algorithm. It divides
   the matrix into four submatrices:

          [  A11 | A12  ]  where A11 is n1 by n1 and A22 is n2 by n2
      A = [ -----|----- ]  with n1 = n/2
          [  A21 | A22  ]       n2 = n-n1

   The subroutine calls itself to factor A11. Update and scale A21
   or A12, update A22 then calls itself to factor A22.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   N-by-N upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading N-by-N lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

   On exit, if INFO = 0, the factor U or L from the Cholesky
   factorization A = U**H*U or A = L*L**H.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, the leading minor of order i is not
         positive definite, and the factorization could not be completed.
```

———-

72.12.3 fortran code

— cpotrf2.f —

```
*  =====================================================================
      RECURSIVE SUBROUTINE CPOTRF2( UPLO, N, A, LDA, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
      COMPLEX            CONE
      PARAMETER          ( CONE = (1.0E+0, 0.0E+0) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            N1, N2, IINFO
      REAL               AJJ
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, SISNAN
      EXTERNAL           LSAME, SISNAN
*     ..
*     .. External Subroutines ..
      EXTERNAL           CHERK, CTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, REAL, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
```

```
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CPOTRF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     N=1 case
*
      IF( N.EQ.1 ) THEN
*
*        Test for non-positive-definiteness
*
         AJJ = REAL( A( 1, 1 ) )
         IF( AJJ.LE.ZERO.OR.SISNAN( AJJ ) ) THEN
            INFO = 1
            RETURN
         END IF
*
*        Factor
*
         A( 1, 1 ) = SQRT( AJJ )
*
*     Use recursive code
*
      ELSE
         N1 = N/2
         N2 = N-N1
*
*        Factor A11
*
         CALL CPOTRF2( UPLO, N1, A( 1, 1 ), LDA, IINFO )
         IF ( IINFO.NE.0 ) THEN
            INFO = IINFO
            RETURN
         END IF
*
*        Compute the Cholesky factorization A = U**H*U
*
         IF( UPPER ) THEN
```

```
*
*          Update and scale A12
*
           CALL CTRSM( 'L', 'U', 'C', 'N', N1, N2, CONE,
     $                 A( 1, 1 ), LDA, A( 1, N1+1 ), LDA )
*
*          Update and factor A22
*
           CALL CHERK( UPLO, 'C', N2, N1, -ONE, A( 1, N1+1 ), LDA,
     $                 ONE, A( N1+1, N1+1 ), LDA )
*
           CALL CPOTRF2( UPLO, N2, A( N1+1, N1+1 ), LDA, IINFO )
*
           IF ( IINFO.NE.0 ) THEN
              INFO = IINFO + N1
              RETURN
           END IF
*
        Compute the Cholesky factorization A = L*L**H
*
        ELSE
*
*          Update and scale A21
*
           CALL CTRSM( 'R', 'L', 'C', 'N', N2, N1, CONE,
     $                 A( 1, 1 ), LDA, A( N1+1, 1 ), LDA )
*
*          Update and factor A22
*
           CALL CHERK( UPLO, 'N', N2, N1, -ONE, A( N1+1, 1 ), LDA,
     $                 ONE, A( N1+1, N1+1 ), LDA )
*
           CALL CPOTRF2( UPLO, N2, A( N1+1, N1+1 ), LDA, IINFO )
*
           IF ( IINFO.NE.0 ) THEN
              INFO = IINFO + N1
              RETURN
           END IF
*
        END IF
     END IF
     RETURN
*
*    End of CPOTRF2
*
     END
```

———-

72.12.4 lisp code

— cpotrf2.lisp —

```
(let* ((one 1.0) (zero 0.0) (cone (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (type (f2cl-lib:complex8) cone)
           (ignorable one zero cone))
  (defun *main* ()
    (prog ((ajj 0.0) (n1 0) (n2 0) (iinfo 0) (upper nil)
           (uplo
            (make-array '(1) :element-type 'character :initial-element #\ ))
           (info 0) (lda 0)
           (a
            (make-array (the fixnum (reduce #'* (list lda *))) :element-type
                        'f2cl-lib:complex8))
           (n 0) (subroutine 0.0))
      (declare (type (array f2cl-lib:complex8 (*)) a)
               (type (single-float) subroutine ajj)
               (type (f2cl-lib:integer4) n lda info n1 n2 iinfo)
               (type f2cl-lib:logical upper)
               (type (string 1) uplo))
      '"****NOT TRANSLATED: (RECURSIVE SUBROUTINE CPOTRF2 (UPLO |,| N |,| A |,| LDA |,| INFO))"
      (setf info 0)
      (setf upper
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "U")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val))
      (cond
       ((and (not upper)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame uplo "L")
                (declare (ignore var-1))
                (when var-0 (setf uplo var-0))
                ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -4)))
      (cond
       ((/= info 0) (xerbla "CPOTRF2" (f2cl-lib:int-sub info)) (go end_label)))
      (if (= n 0)
          (go end_label))
      (cond
       ((= n 1)
```

```
 (setf ajj (f2cl-lib:freal (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))))
 (cond
  ((or (<= ajj zero)
       (multiple-value-bind (ret-val var-0)
           (sisnan ajj)
         (declare (ignore))
         (when var-0 (setf ajj var-0))
         ret-val))
   (setf info 1) (go end_label)))
 (setf (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))
         (coerce (f2cl-lib:fsqrt ajj) 'f2cl-lib:complex8)))
(t (setf n1 (the f2cl-lib:integer4 (truncate n 2)))
 (setf n2 (f2cl-lib:int-sub n n1))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (cpotrf2 uplo n1
      (f2cl-lib:array-slice a f2cl-lib:complex8 (1 1) ((1 lda) (1 *)))
      lda iinfo)
   (declare (ignore var-2))
   (when var-0 (setf uplo var-0))
   (when var-1 (setf n1 var-1))
   (when var-3 (setf lda var-3))
   (when var-4 (setf iinfo var-4)))
 (cond ((/= iinfo 0) (setf info iinfo) (go end_label)))
 (cond
  (upper
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (ctrsm "L" "U" "C" "N" n1 n2 cone
        (f2cl-lib:array-slice a f2cl-lib:complex8 (1 1) ((1 lda) (1 *)))
        lda
        (f2cl-lib:array-slice a f2cl-lib:complex8
                              (1 (f2cl-lib:int-add n1 1))
                              ((1 lda) (1 *)))
        lda)
     (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
     (when var-4 (setf n1 var-4))
     (when var-5 (setf n2 var-5))
     (when var-6 (setf cone var-6))
     (when var-8 (setf lda var-8))
     (when var-10 (setf lda var-10)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (cherk uplo "C" n2 n1 (- one)
        (f2cl-lib:array-slice a f2cl-lib:complex8
                              (1 (f2cl-lib:int-add n1 1))
                              ((1 lda) (1 *)))
        lda one
        (f2cl-lib:array-slice a f2cl-lib:complex8
                              ((+ n1 1) (f2cl-lib:int-add n1 1))
```

```
                              ((1 lda) (1 *)))
        lda)
      (declare (ignore var-1 var-4 var-5 var-8))
      (when var-0 (setf uplo var-0))
      (when var-2 (setf n2 var-2))
      (when var-3 (setf n1 var-3))
      (when var-6 (setf lda var-6))
      (when var-7 (setf one var-7))
      (when var-9 (setf lda var-9)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (cpotrf2 uplo n2
         (f2cl-lib:array-slice a f2cl-lib:complex8
                              ((+ n1 1) (f2cl-lib:int-add n1 1))
                              ((1 lda) (1 *)))
         lda iinfo)
      (declare (ignore var-2))
      (when var-0 (setf uplo var-0))
      (when var-1 (setf n2 var-1))
      (when var-3 (setf lda var-3))
      (when var-4 (setf iinfo var-4)))
    (cond
     ((/= iinfo 0) (setf info (f2cl-lib:int-add iinfo n1))
      (go end_label))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (ctrsm "R" "L" "C" "N" n2 n1 cone
         (f2cl-lib:array-slice a f2cl-lib:complex8 (1 1) ((1 lda) (1 *)))
         lda
         (f2cl-lib:array-slice a f2cl-lib:complex8 ((+ n1 1) 1)
                              ((1 lda) (1 *)))
         lda)
      (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
      (when var-4 (setf n2 var-4))
      (when var-5 (setf n1 var-5))
      (when var-6 (setf cone var-6))
      (when var-8 (setf lda var-8))
      (when var-10 (setf lda var-10)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (cherk uplo "N" n2 n1 (- one)
         (f2cl-lib:array-slice a f2cl-lib:complex8 ((+ n1 1) 1)
                              ((1 lda) (1 *)))
         lda one
         (f2cl-lib:array-slice a f2cl-lib:complex8
                              ((+ n1 1) (f2cl-lib:int-add n1 1))
                              ((1 lda) (1 *)))
         lda)
      (declare (ignore var-1 var-4 var-5 var-8))
```

```
            (when var-0 (setf uplo var-0))
            (when var-2 (setf n2 var-2))
            (when var-3 (setf n1 var-3))
            (when var-6 (setf lda var-6))
            (when var-7 (setf one var-7))
            (when var-9 (setf lda var-9)))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (cpotrf2 uplo n2
               (f2cl-lib:array-slice a f2cl-lib:complex8
                                     ((+ n1 1) (f2cl-lib:int-add n1 1))
                                     ((1 lda) (1 *)))
               lda iinfo)
            (declare (ignore var-2))
            (when var-0 (setf uplo var-0))
            (when var-1 (setf n2 var-1))
            (when var-3 (setf lda var-3))
            (when var-4 (setf iinfo var-4)))
          (cond
           ((/= iinfo 0) (setf info (f2cl-lib:int-add iinfo n1))
            (go end_label)))))))
      (go end_label)
     end_label
      (return nil))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::*main*
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types 'nil :return-values 'nil
                                            :calls 'nil)))
```

———-

72.13 cpotri

72.13.1 Axiom unit tests

— cpotri.input —

```
)set break resume
)sys rm -f cpotri.output
)spool cpotri.output
)set message test on
)set message auto off
```

```
)clear all

)spool
)lisp (bye)
```

———-

72.13.2 Axiom help page

— **cpotri.help** —

```
=====================================================================
cpotri examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cpotri (UPLO, N, A, LDA, INFO)
           CPOTRI

Function/Subroutine Documentation
   subroutine cpotri (character UPLO, integer N, complex, dimension( lda, * )
       A, integer LDA, integer INFO)
       CPOTRI

Purpose:
   CPOTRI computes the inverse of a complex Hermitian positive definite
   matrix A using the Cholesky factorization A = U**H*U or A = L*L**H
   computed by CPOTRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   On entry, the triangular factor U or L from the Cholesky
```

```
  factorization A = U**H*U or A = L*L**H, as computed by CPOTRF.
  On exit, the upper or lower triangle of the (Hermitian)
  inverse of A, overwriting the input factor U or L.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, the (i,i) element of the factor U or L is
        zero, and the inverse could not be computed.
```

———-

72.13.3 fortran code

— **cpotri.f** —

```
*  =====================================================================
      SUBROUTINE CPOTRI( UPLO, N, A, LDA, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLAUUM, CTRTRI, XERBLA
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CPOTRI', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Invert the triangular Cholesky factor U or L.
*
      CALL CTRTRI( UPLO, 'Non-unit', N, A, LDA, INFO )
      IF( INFO.GT.0 )
     $   RETURN
*
*     Form inv(U) * inv(U)**H or inv(L)**H * inv(L).
*
      CALL CLAUUM( UPLO, N, A, LDA, INFO )
*
      RETURN
*
*     End of CPOTRI
*
      END
```

———-

72.13.4 lisp code

— cpotri.lisp —

```
(defun cpotri (uplo n a lda info)
  (declare (type (array f2cl-lib:complex8 (*)) a)
```

```
          (type (f2cl-lib:integer4) info lda n)
          (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex8 a-%data% a-%offset%))
    (prog ()
      (declare)
      (setf info 0)
      (cond
       ((and
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -4)))
      (cond
       ((/= info 0) (xerbla "CPOTRI" (f2cl-lib:int-sub info)) (go end_label)))
      (if (= n 0)
          (go end_label))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (ctrtri uplo "Non-unit" n a lda info)
        (declare (ignore var-1 var-3))
        (when var-0 (setf uplo var-0))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-5 (setf info var-5)))
      (if (> info 0)
          (go end_label))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (clauum uplo n a lda info)
        (declare (ignore var-2))
        (setf uplo var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf info var-4))
      (go end_label)
     end_label
      (return (values uplo n nil lda info)))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cpotri
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::clauum))))
```

———-

72.14 cpotrs

72.14.1 Axiom unit tests

— cpotrs.input —

```
)set break resume
)sys rm -f cpotrs.output
)spool cpotrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

72.14.2 Axiom help page

— cpotrs.help —

```
====================================================================
cpotrs examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cpotrs (UPLO, N, NRHS, A, LDA, B, LDB, INFO)
           CPOTRS

Function/Subroutine Documentation
   subroutine cpotrs (character UPLO, integer N, integer NRHS, complex,
       dimension( lda, * ) A, integer LDA, complex, dimension( ldb, * ) B,
       integer LDB, integer INFO)
       CPOTRS

Purpose:
   CPOTRS solves a system of linear equations A*X = B with a Hermitian
   positive definite matrix A using the Cholesky factorization
   A = U**H*U or A = L*L**H computed by CPOTRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is COMPLEX array, dimension (LDA,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**H*U or A = L*L**H, as computed by CPOTRF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 B
```

```
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the right hand side matrix B.
  On exit, the solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

72.14.3 fortran code

— **cpotrs.f** —

```
*  =====================================================================
      SUBROUTINE CPOTRS( UPLO, N, NRHS, A, LDA, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      COMPLEX            A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            ONE
      PARAMETER          ( ONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
```

```
*     ..
*     .. External Subroutines ..
      EXTERNAL           CTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CPOTRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*        Solve A*X = B where A = U**H *U.
*
*        Solve U**H *X = B, overwriting B with X.
*
         CALL CTRSM( 'Left', 'Upper', 'Conjugate transpose', 'Non-unit',
     $               N, NRHS, ONE, A, LDA, B, LDB )
*
*        Solve U*X = B, overwriting B with X.
*
         CALL CTRSM( 'Left', 'Upper', 'No transpose', 'Non-unit', N,
     $               NRHS, ONE, A, LDA, B, LDB )
      ELSE
*
*        Solve A*X = B where A = L*L**H.
```

```
*
*        Solve L*X = B, overwriting B with X.
*
         CALL CTRSM( 'Left', 'Lower', 'No transpose', 'Non-unit', N,
     $               NRHS, ONE, A, LDA, B, LDB )
*
*        Solve L**H *X = B, overwriting B with X.
*
         CALL CTRSM( 'Left', 'Lower', 'Conjugate transpose', 'Non-unit',
     $               N, NRHS, ONE, A, LDA, B, LDB )
      END IF
*
      RETURN
*
*     End of CPOTRS
*
      END
```

———-

72.14.4 lisp code

— **cpotrs.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:complex8) one)
           (ignorable one))
  (defun cpotrs (uplo n nrhs a lda b ldb$ info)
    (declare (type (array f2cl-lib:complex8 (*)) b a)
             (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex8 a-%data% a-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%))
      (prog ((upper nil))
        (declare (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
```

```
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7)))
(cond
 ((/= info 0) (xerbla "CPOTRS" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= n 0) (= nrhs 0))
    (go end_label))
(cond
 (upper
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ctrsm "Left" "Upper" "Conjugate transpose" "Non-unit" n nrhs one
       a lda b ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ctrsm "Left" "Upper" "No transpose" "Non-unit" n nrhs one a lda
       b ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ctrsm "Left" "Lower" "No transpose" "Non-unit" n nrhs one a lda
       b ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
```

```
            (when var-10 (setf ldb$ var-10)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10)
              (ctrsm "Left" "Lower" "Conjugate transpose" "Non-unit" n nrhs one
               a lda b ldb$)
            (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
            (when var-4 (setf n var-4))
            (when var-5 (setf nrhs var-5))
            (when var-6 (setf one var-6))
            (when var-8 (setf lda var-8))
            (when var-10 (setf ldb$ var-10)))))
        (go end_label)
       end_label
        (return (values uplo n nrhs nil lda nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cpotrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

Chapter 73

LAPACK - Positive Definite Matrix, Comp. Routines, Complex16

73.1 zla_porcond_c

73.1.1 Axiom unit tests

— **zlaporcondc.input** —

```
)set break resume
)sys rm -f zlaporcondc.output
)spool zlaporcondc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

73.1.2 Axiom help page

— **zla_porcond_c.help** —

```
==========================================================================
```

```
zla_porcond_c examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function zla_porcond_c (UPLO, N, A, LDA, AF, LDAF, C,
           CAPPLY, INFO, WORK, RWORK)
           ZLA_PORCOND_C computes the infinity norm condition number of
           op(A)*inv(diag(c)) for Hermitian positive-definite matrices.

Function/Subroutine Documentation
   double precision function zla_porcond_c (character UPLO, integer N,
       complex*16, dimension( lda, * ) A, integer LDA, complex*16, dimension(
       ldaf, * ) AF, integer LDAF, double precision, dimension( * ) C, logical
       CAPPLY, integer INFO, complex*16, dimension( * ) WORK, double
       precision, dimension( * ) RWORK)
       ZLA_PORCOND_C computes the infinity norm condition number of
       op(A)*inv(diag(c)) for Hermitian positive-definite matrices.

Purpose:
   ZLA_PORCOND_C Computes the infinity norm condition number of
   op(A) * inv(diag(C)) where C is a DOUBLE PRECISION vector

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the N-by-N matrix A

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is COMPLEX*16 array, dimension (LDAF,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**H*U or A = L*L**H, as computed by ZPOTRF.
```

```
 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 C
   C is DOUBLE PRECISION array, dimension (N)
   The vector C in the formula op(A) * inv(diag(C)).

 CAPPLY
   CAPPLY is LOGICAL
   If .TRUE. then access the vector C in the formula above.

 INFO
   INFO is INTEGER
   = 0:  Successful exit.
   i > 0:  The ith argument is invalid.

 WORK
   WORK is COMPLEX*16 array, dimension (2*N).
   Workspace.

 RWORK
   RWORK is DOUBLE PRECISION array, dimension (N).
   Workspace.
```

73.1.3 fortran code

— zla_porcond_c.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION ZLA_PORCOND_C( UPLO, N, A, LDA, AF,
     $                                         LDAF, C, CAPPLY, INFO,
     $                                         WORK, RWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      LOGICAL            CAPPLY
      INTEGER            N, LDA, LDAF, INFO
*     ..
```

```
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), WORK( * )
      DOUBLE PRECISION   C( * ), RWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            KASE
      DOUBLE PRECISION   AINVNM, ANORM, TMP
      INTEGER            I, J
      LOGICAL            UP, UPPER
      COMPLEX*16         ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZLACN2, ZPOTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, REAL, DIMAG
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      ZLA_PORCOND_C = 0.0D+0
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZLA_PORCOND_C', -INFO )
```

```
         RETURN
      END IF
      UP = .FALSE.
      IF ( LSAME( UPLO, 'U' ) ) UP = .TRUE.
*
*     Compute norm of op(A)*op2(C).
*
      ANORM = 0.0D+0
      IF ( UP ) THEN
         DO I = 1, N
            TMP = 0.0D+0
            IF ( CAPPLY ) THEN
               DO J = 1, I
                  TMP = TMP + CABS1( A( J, I ) ) / C( J )
               END DO
               DO J = I+1, N
                  TMP = TMP + CABS1( A( I, J ) ) / C( J )
               END DO
            ELSE
               DO J = 1, I
                  TMP = TMP + CABS1( A( J, I ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + CABS1( A( I, J ) )
               END DO
            END IF
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      ELSE
         DO I = 1, N
            TMP = 0.0D+0
            IF ( CAPPLY ) THEN
               DO J = 1, I
                  TMP = TMP + CABS1( A( I, J ) ) / C( J )
               END DO
               DO J = I+1, N
                  TMP = TMP + CABS1( A( J, I ) ) / C( J )
               END DO
            ELSE
               DO J = 1, I
                  TMP = TMP + CABS1( A( I, J ) )
               END DO
               DO J = I+1, N
                  TMP = TMP + CABS1( A( J, I ) )
               END DO
            END IF
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
```

```
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 ) THEN
         ZLA_PORCOND_C = 1.0D+0
         RETURN
      ELSE IF( ANORM .EQ. 0.0D+0 ) THEN
         RETURN
      END IF
*
*     Estimate the norm of inv(op(A)).
*
      AINVNM = 0.0D+0
*
      KASE = 0
   10 CONTINUE
      CALL ZLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
*
            IF ( UP ) THEN
               CALL ZPOTRS( 'U', N, 1, AF, LDAF,
     $            WORK, N, INFO )
            ELSE
               CALL ZPOTRS( 'L', N, 1, AF, LDAF,
     $            WORK, N, INFO )
            ENDIF
*
*           Multiply by inv(C).
*
            IF ( CAPPLY ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
            END IF
         ELSE
*
*           Multiply by inv(C**H).
*
            IF ( CAPPLY ) THEN
               DO I = 1, N
                  WORK( I ) = WORK( I ) * C( I )
               END DO
```

```
            END IF
*
            IF ( UP ) THEN
               CALL ZPOTRS( 'U', N, 1, AF, LDAF,
     $            WORK, N, INFO )
            ELSE
               CALL ZPOTRS( 'L', N, 1, AF, LDAF,
     $            WORK, N, INFO )
            END IF
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM .NE. 0.0D+0 )
     $   ZLA_PORCOND_C = 1.0D+0 / AINVNM
*
      RETURN
*
      END
```

———-

73.1.4 lisp code

— zla_porcond_c.lisp —

```
(defun zla_porcond_c (uplo n a lda af ldaf c capply info work rwork)
  (declare (type f2cl-lib:logical capply)
           (type (array double-float (*)) rwork c)
           (type (array f2cl-lib:complex16 (*)) work af a)
           (type (f2cl-lib:integer4) info ldaf lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (af f2cl-lib:complex16 af-%data% af-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%)
       (c double-float c-%data% c-%offset%)
       (rwork double-float rwork-%data% rwork-%offset%))
    (labels ((cabs1 (zdum)
```

```
          (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
(declare
 (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
  cabs1))
(prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
       (zdum #C(0.0d0 0.0d0)) (up nil) (upper nil) (i 0) (j 0)
       (ainvnm 0.0d0) (anorm 0.0d0) (tmp 0.0d0) (kase 0)
       (zla_porcond_c 0.0d0))
  (declare (type (double-float) zla_porcond_c tmp anorm ainvnm)
           (type (f2cl-lib:integer4) kase j i)
           (type f2cl-lib:logical upper up)
           (type (f2cl-lib:complex16) zdum)
           (type (array f2cl-lib:integer4 (3)) isave))
  (setf zla_porcond_c 0.0d0)
  (setf info 0)
  (setf upper
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
  (cond
   ((and (not upper)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4))
   ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -6)))
  (cond
   ((/= info 0) (xerbla "ZLA_PORCOND_C" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf up f2cl-lib:%false%)
  (if (multiple-value-bind (ret-val var-0 var-1)
          (lsame uplo "U")
        (declare (ignore var-1))
        (when var-0 (setf uplo var-0))
        ret-val)
      (setf up f2cl-lib:%true%))
  (setf anorm 0.0d0)
  (cond
   (up
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
```

```
(tagbody
  (setf tmp 0.0d0)
  (cond
   (capply
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j i) nil)
      (tagbody
        (setf tmp
                (+ tmp
                   (/
                    (cabs1
                     (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                    a-%offset%))
                    (f2cl-lib:fref c-%data% (j) ((1 *))
                                   c-%offset%))))
       label100001))
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf tmp
                (+ tmp
                   (/
                    (cabs1
                     (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%))
                    (f2cl-lib:fref c-%data% (j) ((1 *))
                                   c-%offset%))))
       label100002)))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j i) nil)
      (tagbody
        (setf tmp
                (+ tmp
                   (cabs1
                    (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                   a-%offset%))))
       label100003))
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf tmp
                (+ tmp
                   (cabs1
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))))
       label100004))))
  (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
        tmp)
  (setf anorm (max anorm tmp))
```

```
    label100000)))
(t
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf tmp 0.0d0)
     (cond
      (capply
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j i) nil)
         (tagbody
           (setf tmp
                   (+ tmp
                      (/
                       (cabs1
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))
                       (f2cl-lib:fref c-%data% (j) ((1 *))
                                      c-%offset%))))
           label100006))
       (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (setf tmp
                   (+ tmp
                      (/
                       (cabs1
                        (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                       a-%offset%))
                       (f2cl-lib:fref c-%data% (j) ((1 *))
                                      c-%offset%))))
           label100007)))
      (t
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j i) nil)
         (tagbody
           (setf tmp
                   (+ tmp
                      (cabs1
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%))))
           label100008))
       (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (setf tmp
                   (+ tmp
                      (cabs1
                       (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                      a-%offset%))))
```

```
            label100009))))
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                tmp)
        (setf anorm (max anorm tmp))
       label100005))))
  (cond ((= n 0) (setf zla_porcond_c 1.0d0) (go end_label))
        ((= anorm 0.0d0) (go end_label)))
  (setf ainvnm 0.0d0)
  (setf kase 0)
 label10
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (zlacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 ((+ n 1))
                             ((1 *)) work-%offset%)
       work ainvnm kase isave)
    (declare (ignore var-1 var-2 var-5))
    (setf n var-0)
    (setf ainvnm var-3)
    (setf kase var-4))
  (cond
   ((/= kase 0)
    (cond
     ((= kase 2)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (*
                   (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                   (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                  rwork-%offset%)))
         label100010))
      (cond
       (up
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (zpotrs "U" n 1 af ldaf work n info)
          (declare (ignore var-0 var-2 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-4 (setf ldaf var-4))
          (when var-6 (setf n var-6))
          (when var-7 (setf info var-7))))
       (t
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (zpotrs "L" n 1 af ldaf work n info)
          (declare (ignore var-0 var-2 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-4 (setf ldaf var-4))
          (when var-6 (setf n var-6))
```

```
      (when var-7 (setf info var-7)))))
  (cond
   (capply
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (*
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
       label100011)))))
 (t
  (cond
   (capply
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (*
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 (f2cl-lib:fref c-%data% (i) ((1 *)) c-%offset%)))
       label100012))))
  (cond
   (up
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (zpotrs "U" n 1 af ldaf work n info)
      (declare (ignore var-0 var-2 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-4 (setf ldaf var-4))
      (when var-6 (setf n var-6))
      (when var-7 (setf info var-7))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (zpotrs "L" n 1 af ldaf work n info)
      (declare (ignore var-0 var-2 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-4 (setf ldaf var-4))
      (when var-6 (setf n var-6))
      (when var-7 (setf info var-7)))))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (*
               (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               (f2cl-lib:fref rwork-%data% (i) ((1 *))
```

```
                                        rwork-%offset%)))
              label100013))))
          (go label10)))
        (if (/= ainvnm 0.0d0)
            (setf zla_porcond_c (/ 1.0d0 ainvnm)))
        (go end_label)
       end_label
        (return
         (values zla_porcond_c uplo n nil nil nil ldaf nil nil info nil
                 nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zla_porcond_c
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              fortran-to-lisp::logical
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array double-float (*)))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::ldaf nil nil
                                              fortran-to-lisp::info nil nil)
                                            :calls
                                            '(fortran-to-lisp::zlacn2
                                              fortran-to-lisp::xerbla))))
```

73.2 zla_porcond_x

73.2.1 Axiom unit tests

— zlaporcondx.input —

```
)set break resume
)sys rm -f zlaporcondx.output
)spool zlaporcondx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

73.2.2 Axiom help page

— zla_porcond_x.help —

```
======================================================================
zla_porcond_x examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function zla_porcond_x (UPLO, N, A, LDA, AF, LDAF, X,
           INFO, WORK, RWORK)
           ZLA_PORCOND_X computes the infinity norm condition number of
           op(A)*diag(x) for Hermitian positive-definite matrices.

Function/Subroutine Documentation
   double precision function zla_porcond_x (character UPLO, integer N,
       complex*16, dimension( lda, * ) A, integer LDA, complex*16, dimension(
       ldaf, * ) AF, integer LDAF, complex*16, dimension( * ) X, integer INFO,
       complex*16, dimension( * ) WORK, double precision, dimension( * )
       RWORK)
       ZLA_PORCOND_X computes the infinity norm condition number of
       op(A)*diag(x) for Hermitian positive-definite matrices.
```

```
Purpose:
   ZLA_PORCOND_X Computes the infinity norm condition number of
   op(A) * diag(X) where X is a COMPLEX*16 vector.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is COMPLEX*16 array, dimension (LDAF,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**H*U or A = L*L**H, as computed by ZPOTRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 X
   X is COMPLEX*16 array, dimension (N)
   The vector X in the formula op(A) * diag(X).

 INFO
   INFO is INTEGER
   = 0:  Successful exit.
   i > 0:  The ith argument is invalid.

 WORK
   WORK is COMPLEX*16 array, dimension (2*N).
   Workspace.

 RWORK
   RWORK is DOUBLE PRECISION array, dimension (N).
   Workspace.
```

———-

73.2.3 fortran code

— zla_porcond_x.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION ZLA_PORCOND_X( UPLO, N, A, LDA, AF,
     $                                         LDAF, X, INFO, WORK,
     $                                         RWORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            N, LDA, LDAF, INFO
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), WORK( * ), X( * )
      DOUBLE PRECISION   RWORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            KASE, I, J
      DOUBLE PRECISION   AINVNM, ANORM, TMP
      LOGICAL            UP, UPPER
      COMPLEX*16         ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZLACN2, ZPOTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, REAL, DIMAG
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION CABS1
```

```
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      ZLA_PORCOND_X = 0.0D+0
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF ( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZLA_PORCOND_X', -INFO )
         RETURN
      END IF
      UP = .FALSE.
      IF ( LSAME( UPLO, 'U' ) ) UP = .TRUE.
*
*     Compute norm of op(A)*op2(C).
*
      ANORM = 0.0D+0
      IF ( UP ) THEN
         DO I = 1, N
            TMP = 0.0D+0
            DO J = 1, I
               TMP = TMP + CABS1( A( J, I ) * X( J ) )
            END DO
            DO J = I+1, N
               TMP = TMP + CABS1( A( I, J ) * X( J ) )
            END DO
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      ELSE
         DO I = 1, N
            TMP = 0.0D+0
            DO J = 1, I
               TMP = TMP + CABS1( A( I, J ) * X( J ) )
            END DO
            DO J = I+1, N
               TMP = TMP + CABS1( A( J, I ) * X( J ) )
            END DO
```

```
            RWORK( I ) = TMP
            ANORM = MAX( ANORM, TMP )
         END DO
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 ) THEN
         ZLA_PORCOND_X = 1.0D+0
         RETURN
      ELSE IF( ANORM .EQ. 0.0D+0 ) THEN
         RETURN
      END IF
*
*     Estimate the norm of inv(op(A)).
*
      AINVNM = 0.0D+0
*
      KASE = 0
   10 CONTINUE
      CALL ZLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.2 ) THEN
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
*
            IF ( UP ) THEN
               CALL ZPOTRS( 'U', N, 1, AF, LDAF,
     $            WORK, N, INFO )
            ELSE
               CALL ZPOTRS( 'L', N, 1, AF, LDAF,
     $            WORK, N, INFO )
            ENDIF
*
*           Multiply by inv(X).
*
            DO I = 1, N
               WORK( I ) = WORK( I ) / X( I )
            END DO
         ELSE
*
*           Multiply by inv(X**H).
*
            DO I = 1, N
               WORK( I ) = WORK( I ) / X( I )
            END DO
```

```
*
            IF ( UP ) THEN
               CALL ZPOTRS( 'U', N, 1, AF, LDAF,
     $            WORK, N, INFO )
            ELSE
               CALL ZPOTRS( 'L', N, 1, AF, LDAF,
     $            WORK, N, INFO )
            END IF
*
*           Multiply by R.
*
            DO I = 1, N
               WORK( I ) = WORK( I ) * RWORK( I )
            END DO
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM .NE. 0.0D+0 )
     $   ZLA_PORCOND_X = 1.0D+0 / AINVNM
*
      RETURN
*
      END
```

73.2.4 lisp code

— zla_porcond_x.lisp —

```
(defun zla_porcond_x (uplo n a lda af ldaf x info work rwork)
  (declare (type (array double-float (*)) rwork)
           (type (array f2cl-lib:complex16 (*)) work x af a)
           (type (f2cl-lib:integer4) info ldaf lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (af f2cl-lib:complex16 af-%data% af-%offset%)
       (x f2cl-lib:complex16 x-%data% x-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%)
       (rwork double-float rwork-%data% rwork-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
      (declare
```

```
 (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
  cabs1))
(prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
       (zdum #C(0.0d0 0.0d0)) (up nil) (upper nil) (ainvnm 0.0d0)
       (anorm 0.0d0) (tmp 0.0d0) (kase 0) (i 0) (j 0)
       (zla_porcond_x 0.0d0))
  (declare (type (f2cl-lib:integer4) j i kase)
           (type (double-float) zla_porcond_x tmp anorm ainvnm)
           (type f2cl-lib:logical upper up)
           (type (f2cl-lib:complex16) zdum)
           (type (array f2cl-lib:integer4 (3)) isave))
  (setf zla_porcond_x 0.0d0)
  (setf info 0)
  (setf upper
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
  (cond
   ((and (not upper)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4))
   ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -6)))
  (cond
   ((/= info 0) (xerbla "ZLA_PORCOND_X" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf up f2cl-lib:%false%)
  (if (multiple-value-bind (ret-val var-0 var-1)
          (lsame uplo "U")
        (declare (ignore var-1))
        (when var-0 (setf uplo var-0))
        ret-val)
      (setf up f2cl-lib:%true%))
  (setf anorm 0.0d0)
  (cond
   (up
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf tmp 0.0d0)
```

```
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j i) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (cabs1
                        (*
                         (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                        a-%offset%)
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%)))))
           label100001))
        (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (setf tmp
                    (+ tmp
                       (cabs1
                        (*
                         (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)
                         (f2cl-lib:fref x-%data% (j) ((1 *))
                                        x-%offset%)))))
           label100002))
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
              tmp)
        (setf anorm (max anorm tmp))
       label100000)))
    (t
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf tmp 0.0d0)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j i) nil)
           (tagbody
             (setf tmp
                     (+ tmp
                        (cabs1
                         (*
                          (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref x-%data% (j) ((1 *))
                                         x-%offset%)))))
            label100004))
         (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf tmp
                     (+ tmp
```

```
                        (cabs1
                         (*
                          (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                         a-%offset%)
                          (f2cl-lib:fref x-%data% (j) ((1 *))
                                         x-%offset%)))))
           label100005))
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                tmp)
        (setf anorm (max anorm tmp))
       label100003))))
  (cond ((= n 0) (setf zla_porcond_x 1.0d0) (go end_label))
        ((= anorm 0.0d0) (go end_label)))
  (setf ainvnm 0.0d0)
  (setf kase 0)
 label10
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (zlacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 ((+ n 1))
                             ((1 *)) work-%offset%)
       work ainvnm kase isave)
    (declare (ignore var-1 var-2 var-5))
    (setf n var-0)
    (setf ainvnm var-3)
    (setf kase var-4))
  (cond
   ((/= kase 0)
    (cond
     ((= kase 2)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (*
                   (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                   (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                  rwork-%offset%)))
         label100006))
      (cond
       (up
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (zpotrs "U" n 1 af ldaf work n info)
          (declare (ignore var-0 var-2 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-4 (setf ldaf var-4))
          (when var-6 (setf n var-6))
          (when var-7 (setf info var-7))))
       (t
        (multiple-value-bind
```

```
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (zpotrs "L" n 1 af ldaf work n info)
      (declare (ignore var-0 var-2 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-4 (setf ldaf var-4))
      (when var-6 (setf n var-6))
      (when var-7 (setf info var-7)))))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (/
               (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)))
     label100007)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (/
               (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)))
     label100008))
  (cond
   (up
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (zpotrs "U" n 1 af ldaf work n info)
      (declare (ignore var-0 var-2 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-4 (setf ldaf var-4))
      (when var-6 (setf n var-6))
      (when var-7 (setf info var-7))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (zpotrs "L" n 1 af ldaf work n info)
      (declare (ignore var-0 var-2 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-4 (setf ldaf var-4))
      (when var-6 (setf n var-6))
      (when var-7 (setf info var-7)))))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (*
               (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               (f2cl-lib:fref rwork-%data% (i) ((1 *))
```

```
                                        rwork-%offset%)))
              label100009))))
          (go label10)))
        (if (/= ainvnm 0.0d0)
            (setf zla_porcond_x (/ 1.0d0 ainvnm)))
        (go end_label)
       end_label
        (return
         (values zla_porcond_x uplo n nil nil nil ldaf nil info nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zla_porcond_x
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array double-float (*)))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::ldaf nil
                                              fortran-to-lisp::info nil nil)
                                            :calls
                                            '(fortran-to-lisp::zlacn2
                                              fortran-to-lisp::xerbla))))
```

73.3 zla_porfsx_extended

73.3.1 Axiom unit tests

— zlaporfsxextended.input —

```
)set break resume
)sys rm -f zlaporfsxextended.output
)spool zlaporfsxextended.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

73.3.2 Axiom help page [Demm05]

— zla_porfsx_extended.help —

```
==================================================================
zla_porfsx_extended examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zla_porfsx_extended (PREC_TYPE, UPLO, N, NRHS, A, LDA, AF,
           LDAF, COLEQU, C, B, LDB, Y, LDY, BERR_OUT, N_NORMS, ERR_BNDS_NORM,
           ERR_BNDS_COMP, RES, AYB, DY, Y_TAIL, RCOND, ITHRESH, RTHRESH,
           DZ_UB, IGNORE_CWISE, INFO)
           ZLA_PORFSX_EXTENDED improves the computed solution to a system of
           linear equations for symmetric or Hermitian positive-definite
           matrices by performing extra-precise iterative refinement and
           provides error bounds and backward error estimates for the
           solution.

Function/Subroutine Documentation
   subroutine zla_porfsx_extended (integer PREC_TYPE, character UPLO, integer
       N, integer NRHS, complex*16, dimension( lda, * ) A, integer LDA,
       complex*16, dimension( ldaf, * ) AF, integer LDAF, logical COLEQU,
```

```
      double precision, dimension( * ) C, complex*16, dimension( ldb, * ) B,
      integer LDB, complex*16, dimension( ldy, * ) Y, integer LDY, double
      precision, dimension( * ) BERR_OUT, integer N_NORMS, double precision,
      dimension( nrhs, * ) ERR_BNDS_NORM, double precision, dimension( nrhs,
      * ) ERR_BNDS_COMP, complex*16, dimension( * ) RES, double precision,
      dimension( * ) AYB, complex*16, dimension( * ) DY, complex*16,
      dimension( * ) Y_TAIL, double precision RCOND, integer ITHRESH, double
      precision RTHRESH, double precision DZ_UB, logical IGNORE_CWISE,
      integer INFO)
      ZLA_PORFSX_EXTENDED improves the computed solution to a system of
      linear equations for symmetric or Hermitian positive-definite matrices
      by performing extra-precise iterative refinement and provides error
      bounds and backward error estimates for the solution.

Purpose:
   ZLA_PORFSX_EXTENDED improves the computed solution to a system of
   linear equations by performing extra-precise iterative refinement
   and provides error bounds and backward error estimates for the solution.
   This subroutine is called by ZPORFSX to perform iterative refinement.
   In addition to normwise error bound, the code provides maximum
   componentwise error bound if possible. See comments for ERR_BNDS_NORM
   and ERR_BNDS_COMP for details of the error bounds. Note that this
   subroutine is only resonsible for setting the second fields of
   ERR_BNDS_NORM and ERR_BNDS_COMP.

Parameters:

 PREC_TYPE
   PREC_TYPE is INTEGER
   Specifies the intermediate precision to be used in refinement.
   The value is defined by ILAPREC(P) where P is a CHARACTER and
   P    = 'S':  Single
        = 'D':  Double
        = 'I':  Indigenous
        = 'X', 'E':  Extra

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The number of linear equations, i.e., the order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right-hand-sides, i.e., the number of columns of the
   matrix B.
```

```
A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the N-by-N matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX*16 array, dimension (LDAF,N)
  The triangular factor U or L from the Cholesky factorization
  A = U**T*U or A = L*L**T, as computed by ZPOTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

COLEQU
  COLEQU is LOGICAL
  If .TRUE. then column equilibration was done to A before calling
  this routine. This is needed to compute the solution and error
  bounds correctly.

C
  C is DOUBLE PRECISION array, dimension (N)
  The column scale factors for A. If COLEQU = .FALSE., C
  is not accessed. If C is input, each element of C should be a power
  of the radix to ensure a reliable solution and error estimates.
  Scaling by powers of the radix does not cause rounding errors unless
  the result underflows or overflows. Rounding errors during scaling
  lead to refining with a matrix that is not equivalent to the
  input matrix, producing error estimates that may not be reliable.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  The right-hand-side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

Y
  Y is COMPLEX*16 array, dimension (LDY,NRHS)
  On entry, the solution matrix X, as computed by ZPOTRS.
  On exit, the improved solution matrix Y.

LDY
  LDY is INTEGER
  The leading dimension of the array Y.  LDY >= max(1,N).
```

```
BERR_OUT
  BERR_OUT is DOUBLE PRECISION array, dimension (NRHS)
  On exit, BERR_OUT(j) contains the componentwise relative backward
  error for right-hand-side j from the formula
  max(i) ( abs(RES(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
  where abs(Z) is the componentwise absolute value of the matrix
  or vector Z. This is computed by ZLA_LIN_BERR.

N_NORMS
  N_NORMS is INTEGER
  Determines which error bounds to return (see ERR_BNDS_NORM
  and ERR_BNDS_COMP).
  If N_NORMS >= 1 return normwise error bounds.
  If N_NORMS >= 2 return componentwise error bounds.

ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
           max_j (abs(XTRUE(j,i) - X(j,i)))
            ------------------------------
                  max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
    reciprocal condition number is less than the threshold
    sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
    almost certainly within a factor of 10 of the true error
    so long as the next entry is greater than the threshold
    sqrt(n) * slamch('Epsilon'). This error bound should only
    be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
    reciprocal condition number.  Compared with the threshold
    sqrt(n) * slamch('Epsilon') to determine if the error
    estimate is "guaranteed". These reciprocal condition
```

```
    numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
    appropriately scaled matrix Z.
    Let Z = S*A, where S scales each row by a power of the
    radix so all absolute row sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

ERR_BNDS_COMP
  ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                 abs(XTRUE(j,i) - X(j,i))
            max_j ----------------------
                       abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.

  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
    reciprocal condition number is less than the threshold
    sqrt(n) * slamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
    almost certainly within a factor of 10 of the true error
    so long as the next entry is greater than the threshold
    sqrt(n) * slamch('Epsilon'). This error bound should only
    be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
    reciprocal condition number.  Compared with the threshold
    sqrt(n) * slamch('Epsilon') to determine if the error
    estimate is "guaranteed". These reciprocal condition
    numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
    appropriately scaled matrix Z.
    Let Z = S*(A*diag(x)), where x is the solution for the
    current right-hand side and S scales each row of
```

```
    A*diag(x) by a power of the radix so all absolute row
    sums of Z are approximately 1.

  This subroutine is only responsible for setting the second field above.

RES
  RES is COMPLEX*16 array, dimension (N)
  Workspace to hold the intermediate residual.

AYB
  AYB is DOUBLE PRECISION array, dimension (N)
  Workspace.

DY
  DY is COMPLEX*16 PRECISION array, dimension (N)
  Workspace to hold the intermediate solution.

Y_TAIL
  Y_TAIL is COMPLEX*16 array, dimension (N)
  Workspace to hold the trailing bits of the intermediate solution.

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

ITHRESH
  ITHRESH is INTEGER
  The maximum number of residual computations allowed for
  refinement. The default is 10. For 'aggressive' set to 100 to
  permit convergence using approximate factorizations or
  factorizations other than LU. If the factorization uses a
  technique other than Gaussian elimination, the guarantees in
  ERR_BNDS_NORM and ERR_BNDS_COMP may no longer be trustworthy.

RTHRESH
  RTHRESH is DOUBLE PRECISION
  Determines when to stop refinement if the error estimate stops
  decreasing. Refinement will stop when the next solution no longer
  satisfies norm(dx_{i+1}) < RTHRESH * norm(dx_i) where norm(Z) is
  the infinity norm of Z. RTHRESH satisfies 0 < RTHRESH <= 1. The
  default value is 0.5. For 'aggressive' set to 0.9 to permit
  convergence on extremely ill-conditioned matrices. See LAWN 165
  for more details.

DZ_UB
```

```
  DZ_UB is DOUBLE PRECISION
  Determines when to start considering componentwise convergence.
  Componentwise convergence is only considered after each component
  of the solution Y is stable, which we definte as the relative
  change in each component being less than DZ_UB. The default value
  is 0.25, requiring the first bit to be stable. See LAWN 165 for
  more details.

 IGNORE_CWISE
  IGNORE_CWISE is LOGICAL
  If .TRUE. then ignore componentwise convergence. Default value
  is .FALSE..

 INFO
  INFO is INTEGER
  = 0:  Successful exit.
  < 0:  if INFO = -i, the ith argument to ZPOTRS had an illegal value
```

———-

73.3.3 fortran code

— zla_porfsx_extended.f —

```
*  =====================================================================
      SUBROUTINE ZLA_PORFSX_EXTENDED( PREC_TYPE, UPLO, N, NRHS, A, LDA,
     $                                AF, LDAF, COLEQU, C, B, LDB, Y,
     $                                LDY, BERR_OUT, N_NORMS,
     $                                ERR_BNDS_NORM, ERR_BNDS_COMP, RES,
     $                                AYB, DY, Y_TAIL, RCOND, ITHRESH,
     $                                RTHRESH, DZ_UB, IGNORE_CWISE,
     $                                INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDAF, LDB, LDY, N, NRHS, PREC_TYPE,
     $                   N_NORMS, ITHRESH
      CHARACTER          UPLO
      LOGICAL            COLEQU, IGNORE_CWISE
      DOUBLE PRECISION   RTHRESH, DZ_UB
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
```

```
     $                   Y( LDY, * ), RES( * ), DY( * ), Y_TAIL( * )
      DOUBLE PRECISION   C( * ), AYB( * ), RCOND, BERR_OUT( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            UPLO2, CNT, I, J, X_STATE, Z_STATE,
     $                   Y_PREC_STATE
      DOUBLE PRECISION   YK, DYK, YMIN, NORMY, NORMX, NORMDX, DXRAT,
     $                   DZRAT, PREVNORMDX, PREV_DZ_Z, DXRATMAX,
     $                   DZRATMAX, DX_X, DZ_Z, FINAL_DX_X, FINAL_DZ_Z,
     $                   EPS, HUGEVAL, INCR_THRESH
      LOGICAL            INCR_PREC
      COMPLEX*16         ZDUM
*     ..
*     .. Parameters ..
      INTEGER            UNSTABLE_STATE, WORKING_STATE, CONV_STATE,
     $                   NOPROG_STATE, BASE_RESIDUAL, EXTRA_RESIDUAL,
     $                   EXTRA_Y
      PARAMETER          ( UNSTABLE_STATE = 0, WORKING_STATE = 1,
     $                   CONV_STATE = 2, NOPROG_STATE = 3 )
      PARAMETER          ( BASE_RESIDUAL = 0, EXTRA_RESIDUAL = 1,
     $                   EXTRA_Y = 2 )
      INTEGER            FINAL_NRM_ERR_I, FINAL_CMP_ERR_I, BERR_I
      INTEGER            RCOND_I, NRM_RCOND_I, NRM_ERR_I, CMP_RCOND_I
      INTEGER            CMP_ERR_I, PIV_GROWTH_I
      PARAMETER          ( FINAL_NRM_ERR_I = 1, FINAL_CMP_ERR_I = 2,
     $                   BERR_I = 3 )
      PARAMETER          ( RCOND_I = 4, NRM_RCOND_I = 5, NRM_ERR_I = 6 )
      PARAMETER          ( CMP_RCOND_I = 7, CMP_ERR_I = 8,
     $                   PIV_GROWTH_I = 9 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           ILAUPLO
      INTEGER            ILAUPLO
*     ..
*     .. External Subroutines ..
```

```
      EXTERNAL           ZAXPY, ZCOPY, ZPOTRS, ZHEMV, BLAS_ZHEMV_X,
     $                   BLAS_ZHEMV2_X, ZLA_HEAMV, ZLA_WWADDW,
     $                   ZLA_LIN_BERR, DLAMCH
      DOUBLE PRECISION   DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, MAX, MIN
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      IF (INFO.NE.0) RETURN
      EPS = DLAMCH( 'Epsilon' )
      HUGEVAL = DLAMCH( 'Overflow' )
*     Force HUGEVAL to Inf
      HUGEVAL = HUGEVAL * HUGEVAL
*     Using HUGEVAL may lead to spurious underflows.
      INCR_THRESH = DBLE(N) * EPS

      IF (LSAME (UPLO, 'L')) THEN
         UPLO2 = ILAUPLO( 'L' )
      ELSE
         UPLO2 = ILAUPLO( 'U' )
      ENDIF

      DO J = 1, NRHS
         Y_PREC_STATE = EXTRA_RESIDUAL
         IF (Y_PREC_STATE .EQ. EXTRA_Y) THEN
            DO I = 1, N
               Y_TAIL( I ) = 0.0D+0
            END DO
         END IF

         DXRAT = 0.0D+0
         DXRATMAX = 0.0D+0
         DZRAT = 0.0D+0
         DZRATMAX = 0.0D+0
         FINAL_DX_X = HUGEVAL
         FINAL_DZ_Z = HUGEVAL
         PREVNORMDX = HUGEVAL
         PREV_DZ_Z = HUGEVAL
         DZ_Z = HUGEVAL
         DX_X = HUGEVAL

         X_STATE = WORKING_STATE
```

```
         Z_STATE = UNSTABLE_STATE
         INCR_PREC = .FALSE.

         DO CNT = 1, ITHRESH
*
*         Compute residual RES = B_s - op(A_s) * Y,
*             op(A) = A, A**T, or A**H depending on TRANS (and type).
*
           CALL ZCOPY( N, B( 1, J ), 1, RES, 1 )
           IF (Y_PREC_STATE .EQ. BASE_RESIDUAL) THEN
              CALL ZHEMV(UPLO, N, DCMPLX(-1.0D+0), A, LDA, Y(1,J), 1,
     $             DCMPLX(1.0D+0), RES, 1)
           ELSE IF (Y_PREC_STATE .EQ. EXTRA_RESIDUAL) THEN
              CALL BLAS_ZHEMV_X(UPLO2, N, DCMPLX(-1.0D+0), A, LDA,
     $             Y( 1, J ), 1, DCMPLX(1.0D+0), RES, 1, PREC_TYPE)
           ELSE
              CALL BLAS_ZHEMV2_X(UPLO2, N, DCMPLX(-1.0D+0), A, LDA,
     $             Y(1, J), Y_TAIL, 1, DCMPLX(1.0D+0), RES, 1,
     $      PREC_TYPE)
           END IF

!         XXX: RES is no longer needed.
           CALL ZCOPY( N, RES, 1, DY, 1 )
           CALL ZPOTRS( UPLO, N, 1, AF, LDAF, DY, N, INFO)
*
*         Calculate relative changes DX_X, DZ_Z and ratios DXRAT, DZRAT.
*
           NORMX = 0.0D+0
           NORMY = 0.0D+0
           NORMDX = 0.0D+0
           DZ_Z = 0.0D+0
           YMIN = HUGEVAL

           DO I = 1, N
              YK = CABS1(Y(I, J))
              DYK = CABS1(DY(I))

              IF (YK .NE. 0.0D+0) THEN
                 DZ_Z = MAX( DZ_Z, DYK / YK )
              ELSE IF (DYK .NE. 0.0D+0) THEN
                 DZ_Z = HUGEVAL
              END IF

              YMIN = MIN( YMIN, YK )

              NORMY = MAX( NORMY, YK )

              IF ( COLEQU ) THEN
                 NORMX = MAX(NORMX, YK * C(I))
                 NORMDX = MAX(NORMDX, DYK * C(I))
```

```
               ELSE
                  NORMX = NORMY
                  NORMDX = MAX(NORMDX, DYK)
               END IF
            END DO

            IF (NORMX .NE. 0.0D+0) THEN
               DX_X = NORMDX / NORMX
            ELSE IF (NORMDX .EQ. 0.0D+0) THEN
               DX_X = 0.0D+0
            ELSE
               DX_X = HUGEVAL
            END IF

            DXRAT = NORMDX / PREVNORMDX
            DZRAT = DZ_Z / PREV_DZ_Z
*
*         Check termination criteria.
*
            IF (YMIN*RCOND .LT. INCR_THRESH*NORMY
     $           .AND. Y_PREC_STATE .LT. EXTRA_Y)
     $           INCR_PREC = .TRUE.

            IF (X_STATE .EQ. NOPROG_STATE .AND. DXRAT .LE. RTHRESH)
     $           X_STATE = WORKING_STATE
            IF (X_STATE .EQ. WORKING_STATE) THEN
               IF (DX_X .LE. EPS) THEN
                  X_STATE = CONV_STATE
               ELSE IF (DXRAT .GT. RTHRESH) THEN
                  IF (Y_PREC_STATE .NE. EXTRA_Y) THEN
                     INCR_PREC = .TRUE.
                  ELSE
                     X_STATE = NOPROG_STATE
                  END IF
               ELSE
                  IF (DXRAT .GT. DXRATMAX) DXRATMAX = DXRAT
               END IF
               IF (X_STATE .GT. WORKING_STATE) FINAL_DX_X = DX_X
            END IF

            IF (Z_STATE .EQ. UNSTABLE_STATE .AND. DZ_Z .LE. DZ_UB)
     $           Z_STATE = WORKING_STATE
            IF (Z_STATE .EQ. NOPROG_STATE .AND. DZRAT .LE. RTHRESH)
     $           Z_STATE = WORKING_STATE
            IF (Z_STATE .EQ. WORKING_STATE) THEN
               IF (DZ_Z .LE. EPS) THEN
                  Z_STATE = CONV_STATE
               ELSE IF (DZ_Z .GT. DZ_UB) THEN
                  Z_STATE = UNSTABLE_STATE
                  DZRATMAX = 0.0D+0
```

```
                 FINAL_DZ_Z = HUGEVAL
              ELSE IF (DZRAT .GT. RTHRESH) THEN
                 IF (Y_PREC_STATE .NE. EXTRA_Y) THEN
                    INCR_PREC = .TRUE.
                 ELSE
                    Z_STATE = NOPROG_STATE
                 END IF
              ELSE
                 IF (DZRAT .GT. DZRATMAX) DZRATMAX = DZRAT
              END IF
              IF (Z_STATE .GT. WORKING_STATE) FINAL_DZ_Z = DZ_Z
           END IF

           IF ( X_STATE.NE.WORKING_STATE.AND.
     $           (IGNORE_CWISE.OR.Z_STATE.NE.WORKING_STATE) )
     $           GOTO 666

           IF (INCR_PREC) THEN
              INCR_PREC = .FALSE.
              Y_PREC_STATE = Y_PREC_STATE + 1
              DO I = 1, N
                 Y_TAIL( I ) = 0.0D+0
              END DO
           END IF

           PREVNORMDX = NORMDX
           PREV_DZ_Z = DZ_Z
*
*          Update soluton.
*
           IF (Y_PREC_STATE .LT. EXTRA_Y) THEN
              CALL ZAXPY( N, DCMPLX(1.0D+0), DY, 1, Y(1,J), 1 )
           ELSE
              CALL ZLA_WWADDW(N, Y(1,J), Y_TAIL, DY)
           END IF

        END DO
*       Target of "IF (Z_STOP .AND. X_STOP)".  Sun's f77 won't EXIT.
 666    CONTINUE
*
*     Set final_* when cnt hits ithresh.
*
        IF (X_STATE .EQ. WORKING_STATE) FINAL_DX_X = DX_X
        IF (Z_STATE .EQ. WORKING_STATE) FINAL_DZ_Z = DZ_Z
*
*     Compute error bounds.
*
        IF (N_NORMS .GE. 1) THEN
           ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) =
     $           FINAL_DX_X / (1 - DXRATMAX)
```

```
         END IF
         IF (N_NORMS .GE. 2) THEN
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) =
     $           FINAL_DZ_Z / (1 - DZRATMAX)
         END IF
*
*     Compute componentwise relative backward error from formula
*         max(i) ( abs(R(i)) / ( abs(op(A_s))*abs(Y) + abs(B_s) )(i) )
*     where abs(Z) is the componentwise absolute value of the matrix
*     or vector Z.
*
*        Compute residual RES = B_s - op(A_s) * Y,
*            op(A) = A, A**T, or A**H depending on TRANS (and type).
*
         CALL ZCOPY( N, B( 1, J ), 1, RES, 1 )
         CALL ZHEMV(UPLO, N, DCMPLX(-1.0D+0), A, LDA, Y(1,J), 1,
     $        DCMPLX(1.0D+0), RES, 1)

         DO I = 1, N
            AYB( I ) = CABS1( B( I, J ) )
         END DO
*
*     Compute abs(op(A_s))*abs(Y) + abs(B_s).
*
         CALL ZLA_HEAMV (UPLO2, N, 1.0D+0,
     $        A, LDA, Y(1, J), 1, 1.0D+0, AYB, 1)

         CALL ZLA_LIN_BERR (N, N, 1, RES, AYB, BERR_OUT(J))
*
*     End of loop for each RHS.
*
      END DO
*
      RETURN
      END
```

———-

73.3.4 lisp code

— zla_porfsx_extended.lisp —

```
(let* ((unstable_state 0)
       (working_state 1)
       (conv_state 2)
       (noprog_state 3)
       (base_residual 0)
       (extra_residual 1)
```

```
      (extra_y 2)
      (la_linrx_err_i 2))
 (declare (type (f2cl-lib:integer4 0 0) unstable_state)
          (type (f2cl-lib:integer4 1 1) working_state)
          (type (f2cl-lib:integer4 2 2) conv_state)
          (type (f2cl-lib:integer4 3 3) noprog_state)
          (type (f2cl-lib:integer4 0 0) base_residual)
          (type (f2cl-lib:integer4 1 1) extra_residual)
          (type (f2cl-lib:integer4 2 2) extra_y)
          (type (f2cl-lib:integer4 2 2) la_linrx_err_i)
          (ignorable unstable_state working_state conv_state noprog_state
           base_residual extra_residual extra_y la_linrx_err_i))
 (defun zla_porfsx_extended
        (prec_type uplo n nrhs a lda af ldaf colequ c b ldb$ y ldy berr_out
         n_norms err_bnds_norm err_bnds_comp res ayb dy y_tail rcond ithresh
         rthresh dz_ub ignore_cwise info)
   (declare (type (double-float) dz_ub rthresh rcond)
            (type (array double-float (*)) ayb err_bnds_comp err_bnds_norm
             berr_out c)
            (type f2cl-lib:logical ignore_cwise colequ)
            (type (array f2cl-lib:complex16 (*)) y_tail dy res y b af a)
            (type (string *) uplo)
            (type (f2cl-lib:integer4) info ithresh n_norms ldy ldb$ ldaf lda
             nrhs n prec_type))
   (f2cl-lib:with-multi-array-data
       ((uplo character uplo-%data% uplo-%offset%)
        (a f2cl-lib:complex16 a-%data% a-%offset%)
        (af f2cl-lib:complex16 af-%data% af-%offset%)
        (b f2cl-lib:complex16 b-%data% b-%offset%)
        (y f2cl-lib:complex16 y-%data% y-%offset%)
        (res f2cl-lib:complex16 res-%data% res-%offset%)
        (dy f2cl-lib:complex16 dy-%data% dy-%offset%)
        (y_tail f2cl-lib:complex16 y_tail-%data% y_tail-%offset%)
        (c double-float c-%data% c-%offset%)
        (berr_out double-float berr_out-%data% berr_out-%offset%)
        (err_bnds_norm double-float err_bnds_norm-%data%
         err_bnds_norm-%offset%)
        (err_bnds_comp double-float err_bnds_comp-%data%
         err_bnds_comp-%offset%)
        (ayb double-float ayb-%data% ayb-%offset%))
     (labels ((cabs1 (zdum)
                (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
       (declare
        (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
         cabs1))
       (prog ((zdum #C(0.0d0 0.0d0)) (incr_prec nil) (yk 0.0d0) (dyk 0.0d0)
              (ymin 0.0d0) (normy 0.0d0) (normx 0.0d0) (normdx 0.0d0)
              (dxrat 0.0d0) (dzrat 0.0d0) (prevnormdx 0.0d0) (prev_dz_z 0.0d0)
              (dxratmax 0.0d0) (dzratmax 0.0d0) (dx_x 0.0d0) (dz_z 0.0d0)
              (final_dx_x 0.0d0) (final_dz_z 0.0d0) (eps 0.0d0)
```

```
     (hugeval 0.0d0) (incr_thresh 0.0d0) (uplo2 0) (cnt 0) (i 0)
     (j 0) (x_state 0) (z_state 0) (y_prec_state 0))
(declare (type (f2cl-lib:complex16) zdum)
         (type f2cl-lib:logical incr_prec)
         (type (double-float) yk dyk ymin normy normx normdx dxrat
          dzrat prevnormdx prev_dz_z dxratmax dzratmax dx_x dz_z
          final_dx_x final_dz_z eps hugeval incr_thresh)
         (type (f2cl-lib:integer4) uplo2 cnt i j x_state z_state
          y_prec_state))
(if (/= info 0)
    (go end_label))
(setf eps (dlamch "Epsilon"))
(setf hugeval (dlamch "Overflow"))
(setf hugeval (* hugeval hugeval))
(setf incr_thresh (* (f2cl-lib:dble n) eps))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame uplo "L")
    (declare (ignore var-1))
    (when var-0 (setf uplo var-0))
    ret-val)
  (setf uplo2 (ilauplo "L")))
 (t (setf uplo2 (ilauplo "U"))))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf y_prec_state extra_residual)
    (cond
     ((= y_prec_state extra_y)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                               y_tail-%offset%)
                (coerce 0.0d0 'f2cl-lib:complex16))
         label100001))))
    (setf dxrat 0.0d0)
    (setf dxratmax 0.0d0)
    (setf dzrat 0.0d0)
    (setf dzratmax 0.0d0)
    (setf final_dx_x hugeval)
    (setf final_dz_z hugeval)
    (setf prevnormdx hugeval)
    (setf prev_dz_z hugeval)
    (setf dz_z hugeval)
    (setf dx_x hugeval)
    (setf x_state working_state)
    (setf z_state unstable_state)
    (setf incr_prec f2cl-lib:%false%)
    (f2cl-lib:fdo (cnt 1 (f2cl-lib:int-add cnt 1))
```

```
            ((> cnt ithresh) nil)
(tagbody
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (zcopy n
       (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 j)
                             ((1 ldb$) (1 *)) b-%offset%)
       1 res 1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf n var-0)))
  (cond
   ((= y_prec_state base_residual)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9)
        (zhemv uplo n (f2cl-lib:dcmplx -1.0d0) a lda
         (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                               (1 j) ((1 ldy) (1 *))
                               y-%offset%)
         1 (f2cl-lib:dcmplx 1.0d0) res 1)
      (declare
       (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
      (when var-0 (setf uplo var-0))
      (when var-1 (setf n var-1))
      (when var-4 (setf lda var-4))))
   ((= y_prec_state extra_residual)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10)
        (blas_zhemv_x uplo2 n (f2cl-lib:dcmplx -1.0d0) a lda
         (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                               (1 j) ((1 ldy) (1 *))
                               y-%offset%)
         1 (f2cl-lib:dcmplx 1.0d0) res 1 prec_type)
      (declare
       (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
      (when var-0 (setf uplo2 var-0))
      (when var-1 (setf n var-1))
      (when var-4 (setf lda var-4))
      (when var-10 (setf prec_type var-10))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11)
        (blas_zhemv2_x uplo2 n (f2cl-lib:dcmplx -1.0d0) a lda
         (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                               (1 j) ((1 ldy) (1 *))
                               y-%offset%)
         y_tail 1 (f2cl-lib:dcmplx 1.0d0) res 1 prec_type)
      (declare
       (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9
```

```
      var-10))
    (when var-0 (setf uplo2 var-0))
    (when var-1 (setf n var-1))
    (when var-4 (setf lda var-4))
    (when var-11 (setf prec_type var-11)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (zcopy n res 1 dy 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zpotrs uplo n 1 af ldaf dy n info)
  (declare (ignore var-2 var-3 var-5))
  (when var-0 (setf uplo var-0))
  (when var-1 (setf n var-1))
  (when var-4 (setf ldaf var-4))
  (when var-6 (setf n var-6))
  (when var-7 (setf info var-7)))
(setf normx 0.0d0)
(setf normy 0.0d0)
(setf normdx 0.0d0)
(setf dz_z 0.0d0)
(setf ymin hugeval)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf yk
            (cabs1
             (f2cl-lib:fref y-%data% (i j) ((1 ldy) (1 *))
                            y-%offset%)))
    (setf dyk
            (cabs1
             (f2cl-lib:fref dy-%data% (i) ((1 *))
                            dy-%offset%)))
    (cond ((/= yk 0.0d0) (setf dz_z (max dz_z (/ dyk yk))))
          ((/= dyk 0.0d0) (setf dz_z hugeval)))
    (setf ymin (min ymin yk))
    (setf normy (max normy yk))
    (cond
     (colequ
      (setf normx
              (max normx
                   (* yk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%))))
      (setf normdx
              (max normdx
                   (* dyk
                      (f2cl-lib:fref c-%data% (i) ((1 *))
                                     c-%offset%)))))
```

```
     (t (setf normx normy) (setf normdx (max normdx dyk))))
   label100003))
(cond ((/= normx 0.0d0) (setf dx_x (/ normdx normx)))
      ((= normdx 0.0d0) (setf dx_x 0.0d0))
      (t (setf dx_x hugeval)))
(setf dxrat (/ normdx prevnormdx))
(setf dzrat (/ dz_z prev_dz_z))
(if (and (< (* ymin rcond) (* incr_thresh normy))
         (< y_prec_state extra_y))
    (setf incr_prec f2cl-lib:%true%))
(if (and (= x_state noprog_state) (<= dxrat rthresh))
    (setf x_state working_state))
(cond
 ((= x_state working_state)
  (cond ((<= dx_x eps) (setf x_state conv_state))
        ((> dxrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf x_state noprog_state))))
        (t
         (if (> dxrat dxratmax)
             (setf dxratmax dxrat))))
  (if (> x_state working_state)
      (setf final_dx_x dx_x))))
(if (and (= z_state unstable_state) (<= dz_z dz_ub))
    (setf z_state working_state))
(if (and (= z_state noprog_state) (<= dzrat rthresh))
    (setf z_state working_state))
(cond
 ((= z_state working_state)
  (cond ((<= dz_z eps) (setf z_state conv_state))
        ((> dz_z dz_ub) (setf z_state unstable_state)
         (setf dzratmax 0.0d0) (setf final_dz_z hugeval))
        ((> dzrat rthresh)
         (cond
          ((/= y_prec_state extra_y)
           (setf incr_prec f2cl-lib:%true%))
          (t (setf z_state noprog_state))))
        (t
         (if (> dzrat dzratmax)
             (setf dzratmax dzrat))))
  (if (> z_state working_state)
      (setf final_dz_z dz_z))))
(if (and (/= x_state working_state)
         (or ignore_cwise (/= z_state working_state)))
    (go label666))
(cond
 (incr_prec (setf incr_prec f2cl-lib:%false%)
  (setf y_prec_state (f2cl-lib:int-add y_prec_state 1))
```

```
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref y_tail-%data% (i) ((1 *))
                                 y_tail-%offset%)
                  (coerce 0.0d0 'f2cl-lib:complex16))
           label100004))))
      (setf prevnormdx normdx)
      (setf prev_dz_z dz_z)
      (cond
       ((< y_prec_state extra_y)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (zaxpy n (f2cl-lib:dcmplx 1.0d0) dy 1
             (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                                   (1 j) ((1 ldy) (1 *))
                                   y-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-4 var-5))
          (when var-0 (setf n var-0))))
       (t
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (zla_wwaddw n
             (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                                   (1 j) ((1 ldy) (1 *))
                                   y-%offset%)
             y_tail dy)
          (declare (ignore var-1 var-2 var-3))
          (when var-0 (setf n var-0)))))
    label100002))
label666
 (if (= x_state working_state)
     (setf final_dx_x dx_x))
 (if (= z_state working_state)
     (setf final_dz_z dz_z))
 (cond
  ((>= n_norms 1)
   (setf (f2cl-lib:fref err_bnds_norm-%data% (j la_linrx_err_i)
                        ((1 nrhs) (1 *)) err_bnds_norm-%offset%)
         (/ final_dx_x (- 1 dxratmax)))))
 (cond
  ((>= n_norms 2)
   (setf (f2cl-lib:fref err_bnds_comp-%data% (j la_linrx_err_i)
                        ((1 nrhs) (1 *)) err_bnds_comp-%offset%)
         (/ final_dz_z (- 1 dzratmax)))))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (zcopy n
      (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 j)
                            ((1 ldb$) (1 *)) b-%offset%)
      1 res 1)
   (declare (ignore var-1 var-2 var-3 var-4))
```

```
                (when var-0 (setf n var-0)))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (zhemv uplo n (f2cl-lib:dcmplx -1.0d0) a lda
                   (f2cl-lib:array-slice y-%data% f2cl-lib:complex16 (1 j)
                                         ((1 ldy) (1 *)) y-%offset%)
                   1 (f2cl-lib:dcmplx 1.0d0) res 1)
                (declare (ignore var-2 var-3 var-5 var-6 var-7 var-8 var-9))
                (when var-0 (setf uplo var-0))
                (when var-1 (setf n var-1))
                (when var-4 (setf lda var-4)))
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i n) nil)
                (tagbody
                  (setf (f2cl-lib:fref ayb-%data% (i) ((1 *)) ayb-%offset%)
                          (cabs1
                           (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                          b-%offset%)))
                 label100005))
              (zla_heamv uplo2 n 1.0d0 a lda
               (f2cl-lib:array-slice y-%data% f2cl-lib:complex16 (1 j)
                                     ((1 ldy) (1 *)) y-%offset%)
               1 1.0d0 ayb 1)
              (zla_lin_berr n n 1 res ayb
               (f2cl-lib:array-slice berr_out-%data% double-float (j) ((1 *))
                                     berr_out-%offset%))
            label100000))
          (go end_label)
        end_label
          (return
           (values prec_type uplo n nil nil lda nil ldaf nil nil nil nil nil
                   nil nil nil nil nil nil nil nil nil nil nil nil nil nil
                   info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zla_porfsx_extended
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
```

```
  fortran-to-lisp::logical
  (array double-float (*))
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (array fortran-to-lisp::complex16
   (*))
  (array double-float (*))
  (array fortran-to-lisp::complex16
   (*))
  (array fortran-to-lisp::complex16
   (*))
  (double-float)
  (fortran-to-lisp::integer4)
  (double-float) (double-float)
  fortran-to-lisp::logical
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::prec_type
  fortran-to-lisp::uplo
  fortran-to-lisp::n nil nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::ldaf nil nil nil
  nil nil nil nil nil nil nil nil
  nil nil nil nil nil nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zla_lin_berr
  fortran-to-lisp::zla_heamv
  fortran-to-lisp::ilauplo))))
```

———-

73.4 zla_porpvgrw

73.4.1 Axiom unit tests

— zlaporpvgrw.input —

```
)set break resume
)sys rm -f zlaporpvgrw.output
)spool zlaporpvgrw.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

73.4.2 Axiom help page

— zla_porpvgrw.help —

```
======================================================================
zla_porpvgrw examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function zla_porpvgrw (UPLO, NCOLS, A, LDA, AF, LDAF,
           WORK)
           ZLA_PORPVGRW computes the reciprocal pivot growth factor
           norm(A)/norm(U) for a symmetric or Hermitian positive-definite
           matrix.

Function/Subroutine Documentation
   double precision function zla_porpvgrw (character*1 UPLO, integer NCOLS,
       complex*16, dimension( lda, * ) A, integer LDA, complex*16, dimension(
       ldaf, * ) AF, integer LDAF, double precision, dimension( * ) WORK)
       ZLA_PORPVGRW computes the reciprocal pivot growth factor
       norm(A)/norm(U) for a symmetric or Hermitian positive-definite matrix.

Purpose:
   ZLA_PORPVGRW computes the reciprocal pivot growth factor
   norm(A)/norm(U). The "max absolute element" norm is used. If this is
   much less than 1, the stability of the LU factorization of the
   (equilibrated) matrix A could be poor. This also means that the
   solution X, estimated condition numbers, and error bounds could be
   unreliable.

Parameters:
```

```
 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 NCOLS
   NCOLS is INTEGER
   The number of columns of the matrix A. NCOLS >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the N-by-N matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 AF
   AF is COMPLEX*16 array, dimension (LDAF,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**T*U or A = L*L**T, as computed by ZPOTRF.

 LDAF
   LDAF is INTEGER
   The leading dimension of the array AF.  LDAF >= max(1,N).

 WORK
   WORK is COMPLEX*16 array, dimension (2*N)
```

———-

73.4.3 fortran code

— zla_porpvgrw.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION ZLA_PORPVGRW( UPLO, NCOLS, A, LDA, AF,
     $                                        LDAF, WORK )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER*1        UPLO
```

```
      INTEGER            NCOLS, LDA, LDAF
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), AF( LDAF, * )
      DOUBLE PRECISION   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   AMAX, UMAX, RPVGRW
      LOGICAL            UPPER
      COMPLEX*16         ZDUM
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, ZLASET
      LOGICAL            LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, REAL, DIMAG
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function Definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
      UPPER = LSAME( 'Upper', UPLO )
*
*     DPOTRF will have factored only the NCOLSxNCOLS leading minor, so
*     we restrict the growth search to that minor and use only the first
*     2*NCOLS workspace entries.
*
      RPVGRW = 1.0D+0
      DO I = 1, 2*NCOLS
         WORK( I ) = 0.0D+0
      END DO
*
*     Find the max magnitude entry of each column.
*
      IF ( UPPER ) THEN
         DO J = 1, NCOLS
            DO I = 1, J
               WORK( NCOLS+J ) =
     $              MAX( CABS1( A( I, J ) ), WORK( NCOLS+J ) )
            END DO
         END DO
      ELSE
```

```
         DO J = 1, NCOLS
            DO I = J, NCOLS
               WORK( NCOLS+J ) =
     $              MAX( CABS1( A( I, J ) ), WORK( NCOLS+J ) )
            END DO
         END DO
      END IF
*
*     Now find the max magnitude entry of each column of the factor in
*     AF.  No pivoting, so no permutations.
*
      IF ( LSAME( 'Upper', UPLO ) ) THEN
         DO J = 1, NCOLS
            DO I = 1, J
               WORK( J ) = MAX( CABS1( AF( I, J ) ), WORK( J ) )
            END DO
         END DO
      ELSE
         DO J = 1, NCOLS
            DO I = J, NCOLS
               WORK( J ) = MAX( CABS1( AF( I, J ) ), WORK( J ) )
            END DO
         END DO
      END IF
*
*     Compute the *inverse* of the max element growth factor.  Dividing
*     by zero would imply the largest entry of the factor's column is
*     zero.  Than can happen when either the column of A is zero or
*     massive pivots made the factor underflow to zero.  Neither counts
*     as growth in itself, so simply ignore terms with zero
*     denominators.
*
      IF ( LSAME( 'Upper', UPLO ) ) THEN
         DO I = 1, NCOLS
            UMAX = WORK( I )
            AMAX = WORK( NCOLS+I )
            IF ( UMAX /= 0.0D+0 ) THEN
               RPVGRW = MIN( AMAX / UMAX, RPVGRW )
            END IF
         END DO
      ELSE
         DO I = 1, NCOLS
            UMAX = WORK( I )
            AMAX = WORK( NCOLS+I )
            IF ( UMAX /= 0.0D+0 ) THEN
               RPVGRW = MIN( AMAX / UMAX, RPVGRW )
            END IF
         END DO
      END IF
```

```
      ZLA_PORPVGRW = RPVGRW
      END
```

———-

73.4.4 lisp code

— **zla_porpvgrw.lisp** —

```
(defun zla_porpvgrw (uplo ncols a lda af ldaf work)
  (declare (type (array double-float (*)) work)
           (type (array f2cl-lib:complex16 (*)) af a)
           (type (f2cl-lib:integer4) ldaf lda ncols)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (af f2cl-lib:complex16 af-%data% af-%offset%)
       (work double-float work-%data% work-%offset%))
    (labels ((cabs1 (zdum)
               (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
      (declare
       (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
        cabs1))
      (prog ((zdum #C(0.0d0 0.0d0)) (upper nil) (amax 0.0d0) (umax 0.0d0)
             (rpvgrw 0.0d0) (i 0) (j 0) (zla_porpvgrw 0.0d0))
        (declare (type (f2cl-lib:integer4) j i)
                 (type (double-float) zla_porpvgrw rpvgrw umax amax)
                 (type f2cl-lib:logical upper)
                 (type (f2cl-lib:complex16) zdum))
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame "Upper" uplo)
                  (declare (ignore var-0))
                  (when var-1 (setf uplo var-1))
                  ret-val))
        (setf rpvgrw 1.0d0)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-mul 2 ncols)) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) 0.0d0)
           label100000))
        (cond
         (upper
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j ncols) nil)
            (tagbody
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                    ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ncols j))
                                   ((1 *)) work-%offset%)
                    (max
                     (cabs1
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add ncols j)) ((1 *))
                                    work-%offset%)))
             label100002))
         label100001)))
     (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j ncols) nil)
        (tagbody
          (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                        ((> i ncols) nil)
            (tagbody
              (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ncols j))
                                   ((1 *)) work-%offset%)
                    (max
                     (cabs1
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add ncols j)) ((1 *))
                                    work-%offset%)))
             label100004))
         label100003))))
    (cond
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame "Upper" uplo)
        (declare (ignore var-0))
        (when var-1 (setf uplo var-1))
        ret-val)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j ncols) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i j) nil)
            (tagbody
              (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                    (max
                     (cabs1
                      (f2cl-lib:fref af-%data% (i j) ((1 ldaf) (1 *))
                                     af-%offset%))
                     (f2cl-lib:fref work-%data% (j) ((1 *))
                                    work-%offset%)))
```

```
            label100006))
        label100005)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j ncols) nil)
       (tagbody
         (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                       ((> i ncols) nil)
           (tagbody
             (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                     (max
                      (cabs1
                       (f2cl-lib:fref af-%data% (i j) ((1 ldaf) (1 *))
                                      af-%offset%))
                      (f2cl-lib:fref work-%data% (j) ((1 *))
                                     work-%offset%)))
            label100008))
        label100007))))
   (cond
    ((multiple-value-bind (ret-val var-0 var-1)
         (lsame "Upper" uplo)
       (declare (ignore var-0))
       (when var-1 (setf uplo var-1))
       ret-val)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i ncols) nil)
       (tagbody
         (setf umax (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
         (setf amax
                 (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ncols i))
                                ((1 *)) work-%offset%))
         (cond
          ((f2cl-lib:f2cl/ umax (= 0.0d0))
           (setf rpvgrw (min (/ amax umax) rpvgrw))))
        label100009)))
    (t
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i ncols) nil)
       (tagbody
         (setf umax (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
         (setf amax
                 (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ncols i))
                                ((1 *)) work-%offset%))
         (cond
          ((f2cl-lib:f2cl/ umax (= 0.0d0))
           (setf rpvgrw (min (/ amax umax) rpvgrw))))
        label100010))))
   (setf zla_porpvgrw rpvgrw)
  end_label
   (return (values zla_porpvgrw uplo nil nil nil nil nil nil))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zla_porpvgrw
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*)))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil nil
                                              nil nil nil)
                                            :calls 'nil)))
```

———-

73.5 zpocon

73.5.1 Axiom unit tests

— **zpocon.input** —

```
)set break resume
)sys rm -f zpocon.output
)spool zpocon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

73.5.2 Axiom help page

— zpocon.help —

```
====================================================================
zpocon examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zpocon (UPLO, N, A, LDA, ANORM, RCOND, WORK, RWORK, INFO)
           ZPOCON

Function/Subroutine Documentation
   subroutine zpocon (character UPLO, integer N, complex*16, dimension( lda, *
       ) A, integer LDA, double precision ANORM, double precision RCOND,
       complex*16, dimension( * ) WORK, double precision, dimension( * )
       RWORK, integer INFO)
       ZPOCON

Purpose:
   ZPOCON estimates the reciprocal of the condition number (in the
   1-norm) of a complex Hermitian positive definite matrix using the
   Cholesky factorization A = U**H*U or A = L*L**H computed by ZPOTRF.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**H*U or A = L*L**H, as computed by ZPOTRF.

 LDA
```

```
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

ANORM
  ANORM is DOUBLE PRECISION
  The 1-norm (or infinity-norm) of the Hermitian matrix A.

RCOND
  RCOND is DOUBLE PRECISION
  The reciprocal of the condition number of the matrix A,
  computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is an
  estimate of the 1-norm of inv(A) computed in this routine.

WORK
  WORK is COMPLEX*16 array, dimension (2*N)

RWORK
  RWORK is DOUBLE PRECISION array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

73.5.3 fortran code

— zpocon.f —

```
*  ===================================================================
      SUBROUTINE ZPOCON( UPLO, N, A, LDA, ANORM, RCOND, WORK, RWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
      DOUBLE PRECISION   ANORM, RCOND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   RWORK( * )
      COMPLEX*16         A( LDA, * ), WORK( * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      CHARACTER          NORMIN
      INTEGER            IX, KASE
      DOUBLE PRECISION   AINVNM, SCALE, SCALEL, SCALEU, SMLNUM
      COMPLEX*16         ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IZAMAX
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, IZAMAX, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZDRSCL, ZLACN2, ZLATRS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, MAX
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -5
```

```
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZPOCON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
      SMLNUM = DLAMCH( 'Safe minimum' )
*
*     Estimate the 1-norm of inv(A).
*
      KASE = 0
      NORMIN = 'N'
   10 CONTINUE
      CALL ZLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( UPPER ) THEN
*
*           Multiply by inv(U**H).
*
            CALL ZLATRS( 'Upper', 'Conjugate transpose', 'Non-unit',
     $                   NORMIN, N, A, LDA, WORK, SCALEL, RWORK, INFO )
            NORMIN = 'Y'
*
*           Multiply by inv(U).
*
            CALL ZLATRS( 'Upper', 'No transpose', 'Non-unit', NORMIN, N,
     $                   A, LDA, WORK, SCALEU, RWORK, INFO )
         ELSE
*
*           Multiply by inv(L).
*
            CALL ZLATRS( 'Lower', 'No transpose', 'Non-unit', NORMIN, N,
     $                   A, LDA, WORK, SCALEL, RWORK, INFO )
            NORMIN = 'Y'
*
*           Multiply by inv(L**H).
*
            CALL ZLATRS( 'Lower', 'Conjugate transpose', 'Non-unit',
     $                   NORMIN, N, A, LDA, WORK, SCALEU, RWORK, INFO )
         END IF
```

```
*
*        Multiply by 1/SCALE if doing so will not cause overflow.
*
         SCALE = SCALEL*SCALEU
         IF( SCALE.NE.ONE ) THEN
            IX = IZAMAX( N, WORK, 1 )
            IF( SCALE.LT.CABS1( WORK( IX ) )*SMLNUM .OR. SCALE.EQ.ZERO )
     $         GO TO 20
            CALL ZDRSCL( N, SCALE, WORK, 1 )
         END IF
         GO TO 10
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
   20 CONTINUE
      RETURN
*
*     End of ZPOCON
*
      END
```

———-

73.5.4 lisp code

— **zpocon.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun zpocon (uplo n a lda anorm rcond work rwork info)
    (declare (type (array double-float (*)) rwork)
             (type (double-float) rcond anorm)
             (type (array f2cl-lib:complex16 (*)) work a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
```

```
(declare
 (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
  cabs1))
(prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
       (zdum #C(0.0d0 0.0d0)) (ainvnm 0.0d0) (scale 0.0d0)
       (scalel 0.0d0) (scaleu 0.0d0) (smlnum 0.0d0) (ix 0) (kase 0)
       (normin
        (make-array '(1) :element-type 'character :initial-element
                    #\ ))
       (upper nil))
  (declare (type (array f2cl-lib:integer4 (3)) isave)
           (type (f2cl-lib:complex16) zdum)
           (type (double-float) ainvnm scale scalel scaleu smlnum)
           (type (f2cl-lib:integer4) ix kase)
           (type (string 1) normin)
           (type f2cl-lib:logical upper))
  (setf info 0)
  (setf upper
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
  (cond
   ((and (not upper)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4))
   ((< anorm zero) (setf info -5)))
  (cond
   ((/= info 0) (xerbla "ZPOCON" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf rcond zero)
  (cond ((= n 0) (setf rcond one) (go end_label))
        ((= anorm zero) (go end_label)))
  (setf smlnum (dlamch "Safe minimum"))
  (setf kase 0)
  (f2cl-lib:f2cl-set-string normin "N" (string 1))
 label10
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (zlacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 ((+ n 1))
                             ((1 *)) work-%offset%)
```

```
      work ainvnm kase isave)
    (declare (ignore var-1 var-2 var-5))
    (setf n var-0)
    (setf ainvnm var-3)
    (setf kase var-4))
  (cond
   ((/= kase 0)
    (cond
     (upper
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (zlatrs "Upper" "Conjugate transpose" "Non-unit" normin n a
           lda work scalel rwork info)
        (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
        (setf normin var-3)
        (setf n var-4)
        (setf lda var-6)
        (setf scalel var-8)
        (setf info var-10))
      (f2cl-lib:f2cl-set-string normin "Y" (string 1))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (zlatrs "Upper" "No transpose" "Non-unit" normin n a lda work
           scaleu rwork info)
        (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
        (setf normin var-3)
        (setf n var-4)
        (setf lda var-6)
        (setf scaleu var-8)
        (setf info var-10)))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (zlatrs "Lower" "No transpose" "Non-unit" normin n a lda work
           scalel rwork info)
        (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
        (setf normin var-3)
        (setf n var-4)
        (setf lda var-6)
        (setf scalel var-8)
        (setf info var-10))
      (f2cl-lib:f2cl-set-string normin "Y" (string 1))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (zlatrs "Lower" "Conjugate transpose" "Non-unit" normin n a
           lda work scaleu rwork info)
```

```
                (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9))
                (setf normin var-3)
                (setf n var-4)
                (setf lda var-6)
                (setf scaleu var-8)
                (setf info var-10))))
            (setf scale (* scalel scaleu))
            (cond
             ((/= scale one)
              (setf ix
                      (multiple-value-bind (ret-val var-0 var-1 var-2)
                          (izamax n work 1)
                        (declare (ignore var-1 var-2))
                        (when var-0 (setf n var-0))
                        ret-val))
              (if (or
                   (< scale
                      (*
                       (cabs1
                        (f2cl-lib:fref work-%data% (ix) ((1 *)) work-%offset%))
                       smlnum))
                   (= scale zero))
                  (go label20))
              (multiple-value-bind (var-0 var-1 var-2 var-3)
                  (zdrscl n scale work 1)
                (declare (ignore var-1 var-2 var-3))
                (setf n var-0))))
            (go label10)))
          (if (/= ainvnm zero)
              (setf rcond (/ (/ one ainvnm) anorm)))
         label20
          (go end_label)
         end_label
          (return (values uplo n nil lda nil rcond nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zpocon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array double-float (*))
```

```
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::uplo
  fortran-to-lisp::n nil
  fortran-to-lisp::lda nil
  fortran-to-lisp::rcond nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zdrscl
  fortran-to-lisp::zlatrs
  fortran-to-lisp::zlacn2
  fortran-to-lisp::xerbla))))
```

———-

73.6 zpoequ

73.6.1 Axiom unit tests

— **zpoequ.input** —

```
)set break resume
)sys rm -f zpoequ.output
)spool zpoequ.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

73.6.2 Axiom help page

— **zpoequ.help** —

```
==================================================================
zpoequ examples
==================================================================

==================================================================
```

```
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zpoequ (N, A, LDA, S, SCOND, AMAX, INFO)
           ZPOEQU

Function/Subroutine Documentation
   subroutine zpoequ (integer N, complex*16, dimension( lda, * ) A, integer
       LDA, double precision, dimension( * ) S, double precision SCOND, double
       precision AMAX, integer INFO)
       ZPOEQU

Purpose:
   ZPOEQU computes row and column scalings intended to equilibrate a
   Hermitian positive definite matrix A and reduce its condition number
   (with respect to the two-norm).  S contains the scale factors,
   S(i) = 1/sqrt(A(i,i)), chosen so that the scaled matrix B with
   elements B(i,j) = S(i)*A(i,j)*S(j) has ones on the diagonal.  This
   choice of S puts the condition number of B within a factor N of the
   smallest possible condition number over all possible diagonal scalings.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   The N-by-N Hermitian positive definite matrix whose scaling
   factors are to be computed.  Only the diagonal elements of A
   are referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 S
   S is DOUBLE PRECISION array, dimension (N)
   If INFO = 0, S contains the scale factors for A.

 SCOND
   SCOND is DOUBLE PRECISION
   If INFO = 0, S contains the ratio of the smallest S(i) to
   the largest S(i).  If SCOND >= 0.1 and AMAX is neither too
   large nor too small, it is not worth scaling by S.

 AMAX
```

```
   AMAX is DOUBLE PRECISION
   Absolute value of largest matrix element.  If AMAX is very
   close to overflow or very close to underflow, the matrix should be scaled.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, the i-th diagonal element is nonpositive.
```

73.6.3 fortran code

— zpoequ.f —

```
*  =====================================================================
      SUBROUTINE ZPOEQU( N, A, LDA, S, SCOND, AMAX, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, N
      DOUBLE PRECISION   AMAX, SCOND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   S( * )
      COMPLEX*16         A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      DOUBLE PRECISION   SMIN
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          DBLE, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -3
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZPOEQU', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         SCOND = ONE
         AMAX = ZERO
         RETURN
      END IF
*
*     Find the minimum and maximum diagonal elements.
*
      S( 1 ) = DBLE( A( 1, 1 ) )
      SMIN = S( 1 )
      AMAX = S( 1 )
      DO 10 I = 2, N
         S( I ) = DBLE( A( I, I ) )
         SMIN = MIN( SMIN, S( I ) )
         AMAX = MAX( AMAX, S( I ) )
   10 CONTINUE
*
      IF( SMIN.LE.ZERO ) THEN
*
*        Find the first non-positive diagonal element and return.
*
         DO 20 I = 1, N
            IF( S( I ).LE.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   20    CONTINUE
      ELSE
*
*        Set the scale factors to the reciprocals
*        of the diagonal elements.
```

```
*
         DO 30 I = 1, N
            S( I ) = ONE / SQRT( S( I ) )
   30    CONTINUE
*
*        Compute SCOND = min(S(I)) / max(S(I))
*
         SCOND = SQRT( SMIN ) / SQRT( AMAX )
      END IF
      RETURN
*
*     End of ZPOEQU
*
      END
```

———-

73.6.4 lisp code

— **zpoequ.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun zpoequ (n a lda s scond amax info)
    (declare (type (double-float) amax scond)
             (type (array double-float (*)) s)
             (type (array f2cl-lib:complex16 (*)) a)
             (type (f2cl-lib:integer4) info lda n))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (s double-float s-%data% s-%offset%))
      (prog ((smin 0.0d0) (i 0))
        (declare (type (double-float) smin)
                 (type (f2cl-lib:integer4) i))
        (setf info 0)
        (cond ((< n 0) (setf info -1))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -3)))
        (cond
         ((/= info 0) (xerbla "ZPOEQU" (f2cl-lib:int-sub info))
          (go end_label)))
        (cond ((= n 0) (setf scond one) (setf amax zero) (go end_label)))
        (setf (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%)
                (f2cl-lib:dble
                 (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)))
```

```
        (setf smin (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
        (setf amax (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
        (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                    (f2cl-lib:dble
                     (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                    a-%offset%)))
            (setf smin
                    (min smin (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
            (setf amax
                    (max amax (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
           label10))
        (cond
         ((<= smin zero)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (cond
               ((<= (f2cl-lib:fref s (i) ((1 *))) zero) (setf info i)
                (go end_label)))
             label20)))
         (t
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                      (/ one
                         (f2cl-lib:fsqrt
                          (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%))))
             label30))
          (setf scond (/ (f2cl-lib:fsqrt smin) (f2cl-lib:fsqrt amax)))))
        (go end_label)
       end_label
        (return (values nil nil nil nil scond amax info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zpoequ
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (double-float) (double-float)
                                              (fortran-to-lisp::integer4))
```

```
                                        :return-values
                                        '(nil nil nil nil
                                          fortran-to-lisp::scond
                                          fortran-to-lisp::amax
                                          fortran-to-lisp::info)
                                        :calls '(fortran-to-lisp::xerbla))))
```

———-

73.7 zpoequb

73.7.1 Axiom unit tests

— **zpoequb.input** —

```
)set break resume
)sys rm -f zpoequb.output
)spool zpoequb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

73.7.2 Axiom help page

— **zpoequb.help** —

```
======================================================================
zpoequb examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zpoequb (N, A, LDA, S, SCOND, AMAX, INFO)
```

```
          ZPOEQUB

Function/Subroutine Documentation
   subroutine zpoequb (integer N, complex*16, dimension( lda, * ) A, integer
       LDA, double precision, dimension( * ) S, double precision SCOND, double
       precision AMAX, integer INFO)
       ZPOEQUB

Purpose:
   ZPOEQUB computes row and column scalings intended to equilibrate a
   symmetric positive definite matrix A and reduce its condition number
   (with respect to the two-norm).  S contains the scale factors,
   S(i) = 1/sqrt(A(i,i)), chosen so that the scaled matrix B with
   elements B(i,j) = S(i)*A(i,j)*S(j) has ones on the diagonal.  This
   choice of S puts the condition number of B within a factor N of the
   smallest possible condition number over all possible diagonal scalings.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   The N-by-N symmetric positive definite matrix whose scaling
   factors are to be computed.  Only the diagonal elements of A
   are referenced.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 S
   S is DOUBLE PRECISION array, dimension (N)
   If INFO = 0, S contains the scale factors for A.

 SCOND
   SCOND is DOUBLE PRECISION
   If INFO = 0, S contains the ratio of the smallest S(i) to
   the largest S(i).  If SCOND >= 0.1 and AMAX is neither too
   large nor too small, it is not worth scaling by S.

 AMAX
   AMAX is DOUBLE PRECISION
   Absolute value of largest matrix element.  If AMAX is very
   close to overflow or very close to underflow, the matrix should be scaled.

 INFO
   INFO is INTEGER
```

```
= 0:  successful exit
< 0:  if INFO = -i, the i-th argument had an illegal value
> 0:  if INFO = i, the i-th diagonal element is nonpositive.
```

73.7.3 fortran code

— zpoequb.f —

```
*  =====================================================================
      SUBROUTINE ZPOEQUB( N, A, LDA, S, SCOND, AMAX, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, N
      DOUBLE PRECISION   AMAX, SCOND
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * )
      DOUBLE PRECISION   S( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      DOUBLE PRECISION   SMIN, BASE, TMP
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT, LOG, INT, REAL, DIMAG
*     ..
```

```
*     .. Executable Statements ..
*
*     Test the input parameters.
*
*     Positive definite only performs 1 pass of equilibration.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -3
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZPOEQUB', -INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 ) THEN
         SCOND = ONE
         AMAX = ZERO
         RETURN
      END IF

      BASE = DLAMCH( 'B' )
      TMP = -0.5D+0 / LOG ( BASE )
*
*     Find the minimum and maximum diagonal elements.
*
      S( 1 ) = A( 1, 1 )
      SMIN = S( 1 )
      AMAX = S( 1 )
      DO 10 I = 2, N
         S( I ) = A( I, I )
         SMIN = MIN( SMIN, S( I ) )
         AMAX = MAX( AMAX, S( I ) )
   10 CONTINUE
*
      IF( SMIN.LE.ZERO ) THEN
*
*        Find the first non-positive diagonal element and return.
*
         DO 20 I = 1, N
            IF( S( I ).LE.ZERO ) THEN
               INFO = I
               RETURN
            END IF
   20    CONTINUE
      ELSE
```

```
*
*        Set the scale factors to the reciprocals
*        of the diagonal elements.
*
         DO 30 I = 1, N
            S( I ) = BASE ** INT( TMP * LOG( S( I ) ) )
   30    CONTINUE
*
*        Compute SCOND = min(S(I)) / max(S(I)).
*
         SCOND = SQRT( SMIN ) / SQRT( AMAX )
      END IF
*
      RETURN
*
*     End of ZPOEQUB
*
      END
```

73.7.4 lisp code

— zpoequb.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun zpoequb (n a lda s scond amax info)
    (declare (type (double-float) amax scond)
             (type (array double-float (*)) s)
             (type (array f2cl-lib:complex16 (*)) a)
             (type (f2cl-lib:integer4) info lda n))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (s double-float s-%data% s-%offset%))
      (prog ((smin 0.0d0) (base 0.0d0) (tmp 0.0d0) (i 0))
        (declare (type (double-float) smin base tmp)
                 (type (f2cl-lib:integer4) i))
        (setf info 0)
        (cond ((< n 0) (setf info -1))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -3)))
        (cond
         ((/= info 0) (xerbla "ZPOEQUB" (f2cl-lib:int-sub info))
          (go end_label)))
```

```
  (cond ((= n 0) (setf scond one) (setf amax zero) (go end_label)))
  (setf base (dlamch "B"))
  (setf tmp (/ -0.5d0 (f2cl-lib:flog base)))
  (setf (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%)
          (coerce
           (realpart
            (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%))
           'double-float))
  (setf smin (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
  (setf amax (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%))
  (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
              (coerce
               (realpart
                (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                               a-%offset%))
               'double-float))
      (setf smin
              (min smin (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
      (setf amax
              (max amax (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)))
     label10))
  (cond
   ((<= smin zero)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (cond
         ((<= (f2cl-lib:fref s (i) ((1 *))) zero) (setf info i)
          (go end_label)))
       label20)))
   (t
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref s-%data% (i) ((1 *)) s-%offset%)
                (expt base
                      (f2cl-lib:int
                       (* tmp
                          (f2cl-lib:flog
                           (f2cl-lib:fref s-%data% (i) ((1 *))
                                          s-%offset%))))))
       label30))
    (setf scond (/ (f2cl-lib:fsqrt smin) (f2cl-lib:fsqrt amax)))))
  (go end_label)
 end_label
  (return (values nil nil nil nil scond amax info))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zpoequb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (double-float) (double-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil
                                              fortran-to-lisp::scond
                                              fortran-to-lisp::amax
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

73.8 zporfs

73.8.1 Axiom unit tests

— zporfs.input —

```
)set break resume
)sys rm -f zporfs.output
)spool zporfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

73.8.2 Axiom help page

— zporfs.help —

```
====================================================================
zporfs examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zporfs (UPLO, N, NRHS, A, LDA, AF, LDAF, B, LDB, X, LDX,
           FERR, BERR, WORK, RWORK, INFO)
           ZPORFS

Function/Subroutine Documentation
   subroutine zporfs (character UPLO, integer N, integer NRHS, complex*16,
       dimension( lda, * ) A, integer LDA, complex*16, dimension( ldaf, * )
       AF, integer LDAF, complex*16, dimension( ldb, * ) B, integer LDB,
       complex*16, dimension( ldx, * ) X, integer LDX, double precision,
       dimension( * ) FERR, double precision, dimension( * ) BERR, complex*16,
       dimension( * ) WORK, double precision, dimension( * ) RWORK, integer
       INFO)
       ZPORFS

Purpose:
   ZPORFS improves the computed solution to a system of linear
   equations when the coefficient matrix is Hermitian positive definite,
   and provides error bounds and backward error estimates for the solution.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   The Hermitian matrix A.  If UPLO = 'U', the leading N-by-N
   upper triangular part of A contains the upper triangular part
   of the matrix A, and the strictly lower triangular part of A
```

```
  is not referenced.  If UPLO = 'L', the leading N-by-N lower
  triangular part of A contains the lower triangular part of
  the matrix A, and the strictly upper triangular part of A is
  not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX*16 array, dimension (LDAF,N)
  The triangular factor U or L from the Cholesky factorization
  A = U**H*U or A = L*L**H, as computed by ZPOTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by ZPOTRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

FERR
  FERR is DOUBLE PRECISION array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
```

```
   any element of A or B that makes X(j) an exact solution).

 WORK
   WORK is COMPLEX*16 array, dimension (2*N)

 RWORK
   RWORK is DOUBLE PRECISION array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

   ITMAX is the maximum number of steps of iterative refinement.
```

73.8.3 fortran code

— zporfs.f —

```
*  ====================================================================
      SUBROUTINE ZPORFS( UPLO, N, NRHS, A, LDA, AF, LDAF, B, LDB, X,
     $                   LDX, FERR, BERR, WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   BERR( * ), FERR( * ), RWORK( * )
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
```

```
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
      COMPLEX*16         ONE
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ) )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D+0 )
      DOUBLE PRECISION   THREE
      PARAMETER          ( THREE = 3.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            COUNT, I, J, K, KASE, NZ
      DOUBLE PRECISION   EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN, XK
      COMPLEX*16         ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZAXPY, ZCOPY, ZHEMV, ZLACN2, ZPOTRS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, MAX
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
```

```
         INFO = -7
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -11
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZPORFS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = N + 1
      EPS = DLAMCH( 'Epsilon' )
      SAFMIN = DLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 140 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - A * X
*
         CALL ZCOPY( N, B( 1, J ), 1, WORK, 1 )
         CALL ZHEMV( UPLO, N, -ONE, A, LDA, X( 1, J ), 1, ONE, WORK, 1 )
*
*        Compute componentwise relative backward error from formula
*
*        max(i) ( abs(R(i)) / ( abs(A)*abs(X) + abs(B) )(i) )
*
*        where abs(Z) is the componentwise absolute value of the matrix
*        or vector Z.  If the i-th component of the denominator is less
```

```
*        than SAFE2, then SAFE1 is added to the i-th components of the
*        numerator and denominator before dividing.
*
         DO 30 I = 1, N
            RWORK( I ) = CABS1( B( I, J ) )
   30    CONTINUE
*
*        Compute abs(A)*abs(X) + abs(B).
*
         IF( UPPER ) THEN
            DO 50 K = 1, N
               S = ZERO
               XK = CABS1( X( K, J ) )
               DO 40 I = 1, K - 1
                  RWORK( I ) = RWORK( I ) + CABS1( A( I, K ) )*XK
                  S = S + CABS1( A( I, K ) )*CABS1( X( I, J ) )
   40          CONTINUE
               RWORK( K ) = RWORK( K ) + ABS( DBLE( A( K, K ) ) )*XK + S
   50       CONTINUE
         ELSE
            DO 70 K = 1, N
               S = ZERO
               XK = CABS1( X( K, J ) )
               RWORK( K ) = RWORK( K ) + ABS( DBLE( A( K, K ) ) )*XK
               DO 60 I = K + 1, N
                  RWORK( I ) = RWORK( I ) + CABS1( A( I, K ) )*XK
                  S = S + CABS1( A( I, K ) )*CABS1( X( I, J ) )
   60          CONTINUE
               RWORK( K ) = RWORK( K ) + S
   70       CONTINUE
         END IF
         S = ZERO
         DO 80 I = 1, N
            IF( RWORK( I ).GT.SAFE2 ) THEN
               S = MAX( S, CABS1( WORK( I ) ) / RWORK( I ) )
            ELSE
               S = MAX( S, ( CABS1( WORK( I ) )+SAFE1 ) /
     $             ( RWORK( I )+SAFE1 ) )
            END IF
   80    CONTINUE
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
*           1) The residual BERR(J) is larger than machine epsilon, and
*           2) BERR(J) decreased by at least a factor of 2 during the
*              last iteration, and
*           3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
```

```
*
*          Update solution and try again.
*
           CALL ZPOTRS( UPLO, N, 1, AF, LDAF, WORK, N, INFO )
           CALL ZAXPY( N, ONE, WORK, 1, X( 1, J ), 1 )
           LSTRES = BERR( J )
           COUNT = COUNT + 1
           GO TO 20
        END IF
*
*       Bound error from formula
*
*       norm(X - XTRUE) / norm(X) .le. FERR =
*       norm( abs(inv(A))*
*          ( abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) ))) / norm(X)
*
*       where
*         norm(Z) is the magnitude of the largest component of Z
*         inv(A) is the inverse of A
*         abs(Z) is the componentwise absolute value of the matrix or
*            vector Z
*         NZ is the maximum number of nonzeros in any row of A, plus 1
*         EPS is machine epsilon
*
*       The i-th component of abs(R)+NZ*EPS*(abs(A)*abs(X)+abs(B))
*       is incremented by SAFE1 if the i-th component of
*       abs(A)*abs(X) + abs(B) is less than SAFE2.
*
*       Use ZLACN2 to estimate the infinity-norm of the matrix
*          inv(A) * diag(W),
*       where W = abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) )))
*
        DO 90 I = 1, N
           IF( RWORK( I ).GT.SAFE2 ) THEN
              RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I )
           ELSE
              RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I ) +
     $                     SAFE1
           END IF
   90   CONTINUE
*
        KASE = 0
  100   CONTINUE
        CALL ZLACN2( N, WORK( N+1 ), WORK, FERR( J ), KASE, ISAVE )
        IF( KASE.NE.0 ) THEN
           IF( KASE.EQ.1 ) THEN
*
              Multiply by diag(W)*inv(A**H).
*
              CALL ZPOTRS( UPLO, N, 1, AF, LDAF, WORK, N, INFO )
```

```
               DO 110 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
  110          CONTINUE
            ELSE IF( KASE.EQ.2 ) THEN
*
*              Multiply by inv(A)*diag(W).
*
               DO 120 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
  120          CONTINUE
               CALL ZPOTRS( UPLO, N, 1, AF, LDAF, WORK, N, INFO )
            END IF
            GO TO 100
         END IF
*
*        Normalize error.
*
         LSTRES = ZERO
         DO 130 I = 1, N
            LSTRES = MAX( LSTRES, CABS1( X( I, J ) ) )
  130    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  140 CONTINUE
*
      RETURN
*
*     End of ZPORFS
*
      END
```

———-

73.8.4 lisp code

— **zporfs.lisp** —

```
(let* ((itmax 5)
       (zero 0.0d0)
       (one (f2cl-lib:cmplx 1.0d0 0.0d0))
       (two 2.0d0)
       (three 3.0d0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (f2cl-lib:complex16) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 3.0d0 3.0d0) three)
```

```
          (ignorable itmax zero one two three))
 (defun zporfs
        (uplo n nrhs a lda af ldaf b ldb$ x ldx ferr berr work rwork info)
   (declare (type (array double-float (*)) rwork berr ferr)
            (type (array f2cl-lib:complex16 (*)) work x b af a)
            (type (f2cl-lib:integer4) info ldx ldb$ ldaf lda nrhs n)
            (type (string *) uplo))
   (f2cl-lib:with-multi-array-data
       ((uplo character uplo-%data% uplo-%offset%)
        (a f2cl-lib:complex16 a-%data% a-%offset%)
        (af f2cl-lib:complex16 af-%data% af-%offset%)
        (b f2cl-lib:complex16 b-%data% b-%offset%)
        (x f2cl-lib:complex16 x-%data% x-%offset%)
        (work f2cl-lib:complex16 work-%data% work-%offset%)
        (ferr double-float ferr-%data% ferr-%offset%)
        (berr double-float berr-%data% berr-%offset%)
        (rwork double-float rwork-%data% rwork-%offset%))
     (labels ((cabs1 (zdum)
                (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
       (declare
        (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
         cabs1))
       (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
              (zdum #C(0.0d0 0.0d0)) (eps 0.0d0) (lstres 0.0d0) (s 0.0d0)
              (safe1 0.0d0) (safe2 0.0d0) (safmin 0.0d0) (xk 0.0d0) (count$ 0)
              (i 0) (j 0) (k 0) (kase 0) (nz 0) (upper nil))
         (declare (type (array f2cl-lib:integer4 (3)) isave)
                  (type (f2cl-lib:complex16) zdum)
                  (type (double-float) eps lstres s safe1 safe2 safmin xk)
                  (type (f2cl-lib:integer4) count$ i j k kase nz)
                  (type f2cl-lib:logical upper))
         (setf info 0)
         (setf upper
                 (multiple-value-bind (ret-val var-0 var-1)
                     (lsame uplo "U")
                   (declare (ignore var-1))
                   (when var-0 (setf uplo var-0))
                   ret-val))
         (cond
          ((and (not upper)
                (not
                 (multiple-value-bind (ret-val var-0 var-1)
                     (lsame uplo "L")
                   (declare (ignore var-1))
                   (when var-0 (setf uplo var-0))
                   ret-val)))
           (setf info -1))
          ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
          ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
           (setf info -5))
```

```
 ((< ldaf (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -11)))
(cond
 ((/= info 0) (xerbla "ZPORFS" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
              zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%)
              zero)
     label10))
  (go end_label)))
(setf nz (f2cl-lib:int-add n 1))
(setf eps (dlamch "Epsilon"))
(setf safmin (dlamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zcopy n
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1 work 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (zhemv uplo n (- one) a lda
         (f2cl-lib:array-slice x-%data% f2cl-lib:complex16 (1 j)
                               ((1 ldx) (1 *)) x-%offset%)
         1 one work 1)
      (declare (ignore var-2 var-3 var-5 var-6 var-8 var-9))
      (when var-0 (setf uplo var-0))
      (when var-1 (setf n var-1))
      (when var-4 (setf lda var-4))
      (when var-7 (setf one var-7)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                (cabs1
                 (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)))
       label30))
    (cond
     (upper
      (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                    ((> k n) nil)
        (tagbody
          (setf s zero)
          (setf xk
                  (cabs1
                   (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                                  x-%offset%)))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i
                            (f2cl-lib:int-add k
                                              (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%)
                    (+
                     (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                    rwork-%offset%)
                     (*
                      (cabs1
                       (f2cl-lib:fref a-%data% (i k)
                                      ((1 lda) (1 *)) a-%offset%))
                      xk)))
              (setf s
                    (+ s
                       (*
                        (cabs1
                         (f2cl-lib:fref a-%data% (i k)
                                        ((1 lda) (1 *))
                                        a-%offset%))
                        (cabs1
                         (f2cl-lib:fref x-%data% (i j)
                                        ((1 ldx) (1 *))
                                        x-%offset%)))))
            label40))
          (setf (f2cl-lib:fref rwork-%data% (k) ((1 *))
                               rwork-%offset%)
                  (+
                   (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                  rwork-%offset%)
```

```
                    (*
                     (abs
                      (f2cl-lib:dble
                       (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                      a-%offset%)))
                     xk)
                    s))
        label50)))
    (t
     (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                   ((> k n) nil)
       (tagbody
         (setf s zero)
         (setf xk
                 (cabs1
                  (f2cl-lib:fref x-%data% (k j) ((1 ldx) (1 *))
                                 x-%offset%)))
         (setf (f2cl-lib:fref rwork-%data% (k) ((1 *))
                              rwork-%offset%)
                 (+
                  (f2cl-lib:fref rwork-%data% (k) ((1 *))
                                 rwork-%offset%)
                  (*
                   (abs
                    (f2cl-lib:dble
                     (f2cl-lib:fref a-%data% (k k) ((1 lda) (1 *))
                                    a-%offset%)))
                   xk)))
         (f2cl-lib:fdo (i (f2cl-lib:int-add k 1)
                          (f2cl-lib:int-add i 1))
                        ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                  rwork-%offset%)
                     (+
                      (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                     rwork-%offset%)
                      (*
                       (cabs1
                        (f2cl-lib:fref a-%data% (i k)
                                       ((1 lda) (1 *)) a-%offset%))
                       xk)))
             (setf s
                     (+ s
                        (*
                         (cabs1
                          (f2cl-lib:fref a-%data% (i k)
                                         ((1 lda) (1 *))
                                         a-%offset%))
                         (cabs1
```

```
                        (f2cl-lib:fref x-%data% (i j)
                                       ((1 ldx) (1 *))
                                       x-%offset%)))))
         label60))
      (setf (f2cl-lib:fref rwork-%data% (k) ((1 *))
                           rwork-%offset%)
            (+
             (f2cl-lib:fref rwork-%data% (k) ((1 *))
                            rwork-%offset%)
             s))
     label70))))
(setf s zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
      (setf s
              (max s
                   (/
                    (cabs1
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%))
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%)))))
     (t
      (setf s
              (max s
                   (/
                    (+
                     (cabs1
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%))
                     safe1)
                    (+
                     (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                    rwork-%offset%)
                     safe1))))))
   label80))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zpotrs uplo n 1 af ldaf work n info)
    (declare (ignore var-2 var-3 var-5))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
```

```
      (when var-4 (setf ldaf var-4))
      (when var-6 (setf n var-6))
      (when var-7 (setf info var-7)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (zaxpy n one work 1
         (f2cl-lib:array-slice x-%data% f2cl-lib:complex16 (1 j)
                               ((1 ldx) (1 *)) x-%offset%)
         1)
      (declare (ignore var-2 var-3 var-4 var-5))
      (when var-0 (setf n var-0))
      (when var-1 (setf one var-1)))
    (setf lstres
            (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
    (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                             rwork-%offset%)
                (+
                 (cabs1
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%))
                 (* nz eps
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%)))))
       (t
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                             rwork-%offset%)
                (+
                 (cabs1
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%))
                 (* nz eps
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%))
                 safe1))))
     label90))
  (setf kase 0)
 label100
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (zlacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             ((+ n 1)) ((1 *)) work-%offset%)
       work (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
       kase isave)
    (declare (ignore var-1 var-2 var-5))
    (setf n var-0)
```

```
  (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
          var-3)
  (setf kase var-4))
(cond
 ((/= kase 0)
  (cond
   ((= kase 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (zpotrs uplo n 1 af ldaf work n info)
      (declare (ignore var-2 var-3 var-5))
      (when var-0 (setf uplo var-0))
      (when var-1 (setf n var-1))
      (when var-4 (setf ldaf var-4))
      (when var-6 (setf n var-6))
      (when var-7 (setf info var-7)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                             work-%offset%)
                (*
                 (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                rwork-%offset%)
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)))
       label110)))
   ((= kase 2)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                             work-%offset%)
                (*
                 (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                rwork-%offset%)
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)))
       label120))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (zpotrs uplo n 1 af ldaf work n info)
      (declare (ignore var-2 var-3 var-5))
      (when var-0 (setf uplo var-0))
      (when var-1 (setf n var-1))
      (when var-4 (setf ldaf var-4))
      (when var-6 (setf n var-6))
      (when var-7 (setf info var-7)))))
  (go label100)))
(setf lstres zero)
```

```
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i n) nil)
                (tagbody
                  (setf lstres
                          (max lstres
                               (cabs1
                                (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                               x-%offset%))))
                 label130))
              (if (/= lstres zero)
                  (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                          (/
                           (f2cl-lib:fref ferr-%data% (j) ((1 *))
                                          ferr-%offset%)
                           lstres)))
             label140))
          (go end_label)
         end_label
          (return
           (values uplo n nil nil lda nil ldaf nil nil nil nil nil nil nil nil
                   info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zporfs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
```

```
                                        '(fortran-to-lisp::uplo
                                          fortran-to-lisp::n nil nil
                                          fortran-to-lisp::lda nil
                                          fortran-to-lisp::ldaf nil nil nil
                                          nil nil nil nil nil
                                          fortran-to-lisp::info)
                                        :calls
                                        '(fortran-to-lisp::zlacn2
                                          fortran-to-lisp::xerbla))))
```

———-

73.9 zporfsx

73.9.1 Axiom unit tests

— **zporfsx.input** —

```
)set break resume
)sys rm -f zporfsx.output
)spool zporfsx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

73.9.2 Axiom help page [Demm05]

— **zporfsx.help** —

```
==========================================================================
zporfsx examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine zporfsx (UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF, S, B, LDB,
           X, LDX, RCOND, BERR, N_ERR_BNDS, ERR_BNDS_NORM, ERR_BNDS_COMP,
           NPARAMS, PARAMS, WORK, RWORK, INFO)
           ZPORFSX

Function/Subroutine Documentation
   subroutine zporfsx (character UPLO, character EQUED, integer N, integer
       NRHS, complex*16, dimension( lda, * ) A, integer LDA, complex*16,
       dimension( ldaf, * ) AF, integer LDAF, double precision, dimension( * )
       S, complex*16, dimension( ldb, * ) B, integer LDB, complex*16,
       dimension( ldx, * ) X, integer LDX, double precision RCOND, double
       precision, dimension( * ) BERR, integer N_ERR_BNDS, double precision,
       dimension( nrhs, * ) ERR_BNDS_NORM, double precision, dimension( nrhs,
       * ) ERR_BNDS_COMP, integer NPARAMS, double precision, dimension(*)
       PARAMS, complex*16, dimension( * ) WORK, double precision, dimension( *
       ) RWORK, integer INFO)
       ZPORFSX

Purpose:
   ZPORFSX improves the computed solution to a system of linear
   equations when the coefficient matrix is symmetric positive
   definite, and provides error bounds and backward error estimates
   for the solution.  In addition to normwise error bound, the code
   provides maximum componentwise error bound if possible.  See
   comments for ERR_BNDS_NORM and ERR_BNDS_COMP for details of the
   error bounds.

   The original system of linear equations may have been equilibrated
   before calling this routine, as described by arguments EQUED and S
   below. In this case, the solution and error bounds returned are
   for the original unequilibrated system.

   Some optional parameters are bundled in the PARAMS array.  These
   settings determine how refinement is performed, but often the
   defaults are acceptable.  If the defaults are acceptable, users
   can pass NPARAMS = 0 which prevents the source code from accessing
   the PARAMS argument.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 EQUED
   EQUED is CHARACTER*1
   Specifies the form of equilibration that was done to A
```

```
  before calling this routine. This is needed to compute
  the solution and error bounds correctly.
  = 'N':  No equilibration
  = 'Y':  Both row and column equilibration, i.e., A has been
          replaced by diag(S) * A * diag(S).
          The right hand side B has been changed accordingly.

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  The symmetric matrix A.  If UPLO = 'U', the leading N-by-N
  upper triangular part of A contains the upper triangular part
  of the matrix A, and the strictly lower triangular part of A
  is not referenced.  If UPLO = 'L', the leading N-by-N lower
  triangular part of A contains the lower triangular part of
  the matrix A, and the strictly upper triangular part of A is
  not referenced.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

AF
  AF is COMPLEX*16 array, dimension (LDAF,N)
  The triangular factor U or L from the Cholesky factorization
  A = U**T*U or A = L*L**T, as computed by DPOTRF.

LDAF
  LDAF is INTEGER
  The leading dimension of the array AF.  LDAF >= max(1,N).

S
  S is DOUBLE PRECISION array, dimension (N)
  The row scale factors for A.  If EQUED = 'Y', A is multiplied on
  the left and right by diag(S).  S is an input argument if FACT =
  'F'; otherwise, S is an output argument.  If FACT = 'F' and EQUED
  = 'Y', each element of S must be positive.  If S is output, each
  element of S is a power of the radix. If S is input, each element
  of S should be a power of the radix to ensure a reliable solution
  and error estimates. Scaling by powers of the radix does not cause
  rounding errors unless the result underflows or overflows.
  Rounding errors during scaling lead to refining with a matrix that
```

```
  is not equivalent to the input matrix, producing error estimates
  that may not be reliable.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by DGETRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  Reciprocal scaled condition number.  This is an estimate of the
  reciprocal Skeel condition number of the matrix A after
  equilibration (if done).  If this is less than the machine
  precision (in particular, if it is zero), the matrix is singular
  to working precision.  Note that the error may still be small even
  if this number is very small and the matrix appears ill-conditioned.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  Componentwise relative backward error.  This is the
  componentwise relative backward error of each solution vector X(j)
  (i.e., the smallest relative change in any element of A or B that
  makes X(j) an exact solution).

N_ERR_BNDS
  N_ERR_BNDS is INTEGER
  Number of error bounds to return for each right hand side
  and each type (normwise or componentwise).  See ERR_BNDS_NORM and
  ERR_BNDS_COMP below.

ERR_BNDS_NORM
  ERR_BNDS_NORM is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  normwise relative error, which is defined as follows:

  Normwise relative error in the ith solution vector:
           max_j (abs(XTRUE(j,i) - X(j,i)))
```

```
            -----------------------------
                   max_j abs(X(j,i))

  The array is indexed by the type of error information as described
  below. There currently are up to three pieces of information
  returned.

  The first index in ERR_BNDS_NORM(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_NORM(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
    reciprocal condition number is less than the threshold
    sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
    almost certainly within a factor of 10 of the true error
    so long as the next entry is greater than the threshold
    sqrt(n) * dlamch('Epsilon'). This error bound should only
    be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated normwise
    reciprocal condition number.  Compared with the threshold
    sqrt(n) * dlamch('Epsilon') to determine if the error
    estimate is "guaranteed". These reciprocal condition
    numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
    appropriately scaled matrix Z.
    Let Z = S*A, where S scales each row by a power of the
    radix so all absolute row sums of Z are approximately 1.

ERR_BNDS_COMP
  ERR_BNDS_COMP is DOUBLE PRECISION array, dimension (NRHS, N_ERR_BNDS)
  For each right-hand side, this array contains information about
  various error bounds and condition numbers corresponding to the
  componentwise relative error, which is defined as follows:

  Componentwise relative error in the ith solution vector:
                 abs(XTRUE(j,i) - X(j,i))
           max_j ----------------------
                        abs(X(j,i))

  The array is indexed by the right-hand side i (on which the
  componentwise relative error depends), and the type of error
  information as described below. There currently are up to three
  pieces of information returned for each right-hand side. If
  componentwise accuracy is not requested (PARAMS(3) = 0.0), then
  ERR_BNDS_COMP is not accessed.  If N_ERR_BNDS .LT. 3, then at most
  the first (:,N_ERR_BNDS) entries are returned.
```

```
  The first index in ERR_BNDS_COMP(i,:) corresponds to the ith
  right-hand side.

  The second index in ERR_BNDS_COMP(:,err) contains the following
  three fields:

  err = 1 "Trust/don't trust" boolean. Trust the answer if the
     reciprocal condition number is less than the threshold
     sqrt(n) * dlamch('Epsilon').

  err = 2 "Guaranteed" error bound: The estimated forward error,
     almost certainly within a factor of 10 of the true error
     so long as the next entry is greater than the threshold
     sqrt(n) * dlamch('Epsilon'). This error bound should only
     be trusted if the previous boolean is true.

  err = 3  Reciprocal condition number: Estimated componentwise
     reciprocal condition number.  Compared with the threshold
     sqrt(n) * dlamch('Epsilon') to determine if the error
     estimate is "guaranteed". These reciprocal condition
     numbers are 1 / (norm(Z^{-1},inf) * norm(Z,inf)) for some
     appropriately scaled matrix Z.
     Let Z = S*(A*diag(x)), where x is the solution for the
     current right-hand side and S scales each row of
     A*diag(x) by a power of the radix so all absolute row
     sums of Z are approximately 1.

NPARAMS
  NPARAMS is INTEGER
  Specifies the number of parameters set in PARAMS.  If .LE. 0, the
  PARAMS array is never referenced and default values are used.

PARAMS
  PARAMS is DOUBLE PRECISION array, dimension NPARAMS
  Specifies algorithm parameters.  If an entry is .LT. 0.0, then
  that entry will be filled with default value used for that
  parameter.  Only positions up to NPARAMS are accessed; defaults
  are used for higher-numbered parameters.

  PARAMS(LA_LINRX_ITREF_I = 1) : Whether to perform iterative
  refinement or not.
  Default: 1.0D+0
  = 0.0 : No refinement is performed, and no error bounds are computed.
  = 1.0 : Use the double-precision refinement algorithm,
          possibly with doubled-single computations if the
          compilation environment does not support DOUBLE PRECISION.
  (other values are reserved for future use)

  PARAMS(LA_LINRX_ITHRESH_I = 2) : Maximum number of residual
```

```
  computations allowed for refinement.
  Default: 10
  Aggressive: Set to 100 to permit convergence using approximate
              factorizations or factorizations other than LU. If
              the factorization uses a technique other than
              Gaussian elimination, the guarantees in
              err_bnds_norm and err_bnds_comp may no longer be trustworthy.

  PARAMS(LA_LINRX_CWISE_I = 3) : Flag determining if the code
  will attempt to find a solution with small componentwise
  relative error in the double-precision algorithm.  Positive
  is true, 0.0 is false.
  Default: 1.0 (attempt componentwise convergence)

WORK
  WORK is COMPLEX*16 array, dimension (2*N)

RWORK
  RWORK is DOUBLE PRECISION array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  Successful exit. The solution to every right-hand side is
     guaranteed.
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0 and <= N:  U(INFO,INFO) is exactly zero.  The factorization
     has been completed, but the factor U is exactly singular, so
     the solution and error bounds could not be computed. RCOND = 0
     is returned.
  = N+J: The solution corresponding to the Jth right-hand side is
     not guaranteed. The solutions corresponding to other right-
     hand sides K with K > J may not be guaranteed as well, but
     only the first such right-hand side is reported. If a small
     componentwise error is not requested (PARAMS(3) = 0.0) then
     the Jth right-hand side is the first with a normwise error
     bound that is not guaranteed (the smallest J such
     that ERR_BNDS_NORM(J,1) = 0.0). By default (PARAMS(3) = 1.0)
     the Jth right-hand side is the first with either a normwise or
     componentwise error bound that is not guaranteed (the smallest
     J such that either ERR_BNDS_NORM(J,1) = 0.0 or
     ERR_BNDS_COMP(J,1) = 0.0). See the definition of
     ERR_BNDS_NORM(:,1) and ERR_BNDS_COMP(:,1). To get information
     about all of the right-hand sides check ERR_BNDS_NORM or ERR_BNDS_COMP.
```

73.9.3 fortran code

— zporfsx.f —

```
*  ==================================================================
      SUBROUTINE ZPORFSX( UPLO, EQUED, N, NRHS, A, LDA, AF, LDAF, S, B,
     $                    LDB, X, LDX, RCOND, BERR, N_ERR_BNDS,
     $                    ERR_BNDS_NORM, ERR_BNDS_COMP, NPARAMS, PARAMS,
     $                    WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO, EQUED
      INTEGER            INFO, LDA, LDAF, LDB, LDX, N, NRHS, NPARAMS,
     $                   N_ERR_BNDS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), AF( LDAF, * ), B( LDB, * ),
     $                   X( LDX, * ), WORK( * )
      DOUBLE PRECISION   RWORK( * ), S( * ), PARAMS(*), BERR( * ),
     $                   ERR_BNDS_NORM( NRHS, * ),
     $                   ERR_BNDS_COMP( NRHS, * )
*     ..
*
*  ==================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   ITREF_DEFAULT, ITHRESH_DEFAULT
      DOUBLE PRECISION   COMPONENTWISE_DEFAULT, RTHRESH_DEFAULT
      DOUBLE PRECISION   DZTHRESH_DEFAULT
      PARAMETER          ( ITREF_DEFAULT = 1.0D+0 )
      PARAMETER          ( ITHRESH_DEFAULT = 10.0D+0 )
      PARAMETER          ( COMPONENTWISE_DEFAULT = 1.0D+0 )
      PARAMETER          ( RTHRESH_DEFAULT = 0.5D+0 )
      PARAMETER          ( DZTHRESH_DEFAULT = 0.25D+0 )
      INTEGER            LA_LINRX_ITREF_I, LA_LINRX_ITHRESH_I,
     $                   LA_LINRX_CWISE_I
      PARAMETER          ( LA_LINRX_ITREF_I = 1,
     $                   LA_LINRX_ITHRESH_I = 2 )
      PARAMETER          ( LA_LINRX_CWISE_I = 3 )
      INTEGER            LA_LINRX_TRUST_I, LA_LINRX_ERR_I,
     $                   LA_LINRX_RCOND_I
```

```
      PARAMETER          ( LA_LINRX_TRUST_I = 1, LA_LINRX_ERR_I = 2 )
      PARAMETER          ( LA_LINRX_RCOND_I = 3 )
*     ..
*     .. Local Scalars ..
      CHARACTER(1)       NORM
      LOGICAL            RCEQU
      INTEGER            J, PREC_TYPE, REF_TYPE
      INTEGER            N_NORMS
      DOUBLE PRECISION   ANORM, RCOND_TMP
      DOUBLE PRECISION   ILLRCOND_THRESH, ERR_LBND, CWISE_WRONG
      LOGICAL            IGNORE_CWISE
      INTEGER            ITHRESH
      DOUBLE PRECISION   RTHRESH, UNSTABLE_THRESH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZPOCON, ZLA_PORFSX_EXTENDED
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT, TRANSFER
*     ..
*     .. External Functions ..
      EXTERNAL           LSAME, BLAS_FPINFO_X, ILATRANS, ILAPREC
      EXTERNAL           DLAMCH, ZLANHE, ZLA_PORCOND_X, ZLA_PORCOND_C
      DOUBLE PRECISION   DLAMCH, ZLANHE, ZLA_PORCOND_X, ZLA_PORCOND_C
      LOGICAL            LSAME
      INTEGER            BLAS_FPINFO_X
      INTEGER            ILATRANS, ILAPREC
*     ..
*     .. Executable Statements ..
*
*     Check the input parameters.
*
      INFO = 0
      REF_TYPE = INT( ITREF_DEFAULT )
      IF ( NPARAMS .GE. LA_LINRX_ITREF_I ) THEN
         IF ( PARAMS( LA_LINRX_ITREF_I ) .LT. 0.0D+0 ) THEN
            PARAMS( LA_LINRX_ITREF_I ) = ITREF_DEFAULT
         ELSE
            REF_TYPE = PARAMS( LA_LINRX_ITREF_I )
         END IF
      END IF
*
*     Set default parameters.
*
      ILLRCOND_THRESH = DBLE( N ) * DLAMCH( 'Epsilon' )
      ITHRESH = INT( ITHRESH_DEFAULT )
      RTHRESH = RTHRESH_DEFAULT
      UNSTABLE_THRESH = DZTHRESH_DEFAULT
      IGNORE_CWISE = COMPONENTWISE_DEFAULT .EQ. 0.0D+0
*
```

```
      IF ( NPARAMS.GE.LA_LINRX_ITHRESH_I ) THEN
         IF ( PARAMS(LA_LINRX_ITHRESH_I ).LT.0.0D+0 ) THEN
            PARAMS( LA_LINRX_ITHRESH_I ) = ITHRESH
         ELSE
            ITHRESH = INT( PARAMS( LA_LINRX_ITHRESH_I ) )
         END IF
      END IF
      IF ( NPARAMS.GE.LA_LINRX_CWISE_I ) THEN
         IF ( PARAMS(LA_LINRX_CWISE_I ).LT.0.0D+0 ) THEN
            IF ( IGNORE_CWISE ) THEN
               PARAMS( LA_LINRX_CWISE_I ) = 0.0D+0
            ELSE
               PARAMS( LA_LINRX_CWISE_I ) = 1.0D+0
            END IF
         ELSE
            IGNORE_CWISE = PARAMS( LA_LINRX_CWISE_I ) .EQ. 0.0D+0
         END IF
      END IF
      IF ( REF_TYPE .EQ. 0 .OR. N_ERR_BNDS .EQ. 0 ) THEN
         N_NORMS = 0
      ELSE IF ( IGNORE_CWISE ) THEN
         N_NORMS = 1
      ELSE
         N_NORMS = 2
      END IF
*
      RCEQU = LSAME( EQUED, 'Y' )
*
*     Test input parameters.
*
      IF (.NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
        INFO = -1
      ELSE IF( .NOT.RCEQU .AND. .NOT.LSAME( EQUED, 'N' ) ) THEN
        INFO = -2
      ELSE IF( N.LT.0 ) THEN
        INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
        INFO = -4
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
        INFO = -6
      ELSE IF( LDAF.LT.MAX( 1, N ) ) THEN
        INFO = -8
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
        INFO = -11
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
        INFO = -13
      END IF
      IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'ZPORFSX', -INFO )
        RETURN
```

```
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         RCOND = 1.0D+0
         DO J = 1, NRHS
            BERR( J ) = 0.0D+0
            IF ( N_ERR_BNDS .GE. 1 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
            IF ( N_ERR_BNDS .GE. 2 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 0.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 0.0D+0
            END IF
            IF ( N_ERR_BNDS .GE. 3 ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 1.0D+0
            END IF
         END DO
         RETURN
      END IF
*
*     Default to failure.
*
      RCOND = 0.0D+0
      DO J = 1, NRHS
         BERR( J ) = 1.0D+0
         IF ( N_ERR_BNDS .GE. 1 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
         END IF
         IF ( N_ERR_BNDS .GE. 2 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
         END IF
         IF ( N_ERR_BNDS .GE. 3 ) THEN
            ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = 0.0D+0
            ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = 0.0D+0
         END IF
      END DO
*
*     Compute the norm of A and the reciprocal of the condition
*     number of A.
*
      NORM = 'I'
      ANORM = ZLANHE( NORM, UPLO, N, A, LDA, RWORK )
      CALL ZPOCON( UPLO, N, AF, LDAF, ANORM, RCOND, WORK, RWORK,
     $     INFO )
```

```
*
*     Perform refinement on each right-hand side
*
      IF ( REF_TYPE .NE. 0 ) THEN

         PREC_TYPE = ILAPREC( 'E' )

         CALL ZLA_PORFSX_EXTENDED( PREC_TYPE, UPLO, N,
     $        NRHS, A, LDA, AF, LDAF, RCEQU, S, B,
     $        LDB, X, LDX, BERR, N_NORMS, ERR_BNDS_NORM, ERR_BNDS_COMP,
     $        WORK, RWORK, WORK(N+1),
     $        TRANSFER (RWORK(1:2*N), (/ (ZERO, ZERO) /), N), RCOND,
     $        ITHRESH, RTHRESH, UNSTABLE_THRESH, IGNORE_CWISE,
     $        INFO )
      END IF

      ERR_LBND = MAX( 10.0D+0, SQRT( DBLE( N ) ) ) * DLAMCH( 'Epsilon' )
      IF ( N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 1 ) THEN
*
*     Compute scaled normwise condition number cond(A*C).
*
         IF ( RCEQU ) THEN
            RCOND_TMP = ZLA_PORCOND_C( UPLO, N, A, LDA, AF, LDAF,
     $           S, .TRUE., INFO, WORK, RWORK )
         ELSE
            RCOND_TMP = ZLA_PORCOND_C( UPLO, N, A, LDA, AF, LDAF,
     $           S, .FALSE., INFO, WORK, RWORK )
         END IF
         DO J = 1, NRHS
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .GT. 1.0D+0 )
     $           ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
*
*     Threshold the error (see LAWN).
*
            IF ( RCOND_TMP .LT. ILLRCOND_THRESH ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = 1.0D+0
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 0.0D+0
               IF ( INFO .LE. N ) INFO = N + J
            ELSE IF ( ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) .LT. ERR_LBND )
     $     THEN
               ERR_BNDS_NORM( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_NORM( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
*
*     Save the condition number.
*
```

```
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_NORM( J, LA_LINRX_RCOND_I ) = RCOND_TMP
            END IF

         END DO
      END IF

      IF (N_ERR_BNDS .GE. 1 .AND. N_NORMS .GE. 2) THEN
*
*     Compute componentwise condition number cond(A*diag(Y(:,J))) for
*     each right-hand side using the current solution as an estimate of
*     the true solution.  If the componentwise error estimate is too
*     large, then the solution is a lousy estimate of truth and the
*     estimated RCOND may be too optimistic.  To avoid misleading users,
*     the inverse condition number is set to 0.0 when the estimated
*     cwise error is at least CWISE_WRONG.
*
         CWISE_WRONG = SQRT( DLAMCH( 'Epsilon' ) )
         DO J = 1, NRHS
            IF (ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .LT. CWISE_WRONG )
     $     THEN
               RCOND_TMP = ZLA_PORCOND_X( UPLO, N, A, LDA, AF, LDAF,
     $         X(1,J), INFO, WORK, RWORK )
            ELSE
               RCOND_TMP = 0.0D+0
            END IF
*
*     Cap the error at 1.0.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_ERR_I
     $           .AND. ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) .GT. 1.0D+0 )
     $           ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
*
*     Threshold the error (see LAWN).
*
            IF (RCOND_TMP .LT. ILLRCOND_THRESH) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = 1.0D+0
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 0.0D+0
               IF ( PARAMS( LA_LINRX_CWISE_I ) .EQ. 1.0D+0
     $              .AND. INFO.LT.N + J ) INFO = N + J
            ELSE IF ( ERR_BNDS_COMP( J, LA_LINRX_ERR_I )
     $              .LT. ERR_LBND ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_ERR_I ) = ERR_LBND
               ERR_BNDS_COMP( J, LA_LINRX_TRUST_I ) = 1.0D+0
            END IF
*
*     Save the condition number.
*
            IF ( N_ERR_BNDS .GE. LA_LINRX_RCOND_I ) THEN
               ERR_BNDS_COMP( J, LA_LINRX_RCOND_I ) = RCOND_TMP
```

```
            END IF

         END DO
      END IF
*
      RETURN
*
*     End of ZPORFSX
*
      END
```

———-

73.9.4 lisp code

— **zporfsx.lisp** —

———-

73.10 zpotf2

73.10.1 Axiom unit tests

— **zpotf2.input** —

```
)set break resume
)sys rm -f zpotf2.output
)spool zpotf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

73.10.2 Axiom help page

— **zpotf2.help** —

```
====================================================================
zpotf2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zpotf2 (UPLO, N, A, LDA, INFO)
           ZPOTF2 computes the Cholesky factorization of a symmetric/Hermitian
           positive definite matrix (unblocked algorithm).

Function/Subroutine Documentation
   subroutine zpotf2 (character UPLO, integer N, complex*16, dimension( lda, *
       ) A, integer LDA, integer INFO)
       ZPOTF2 computes the Cholesky factorization of a symmetric/Hermitian
       positive definite matrix (unblocked algorithm).

Purpose:
   ZPOTF2 computes the Cholesky factorization of a complex Hermitian
   positive definite matrix A.

   The factorization has the form
      A = U**H * U ,  if UPLO = 'U', or
      A = L  * L**H,  if UPLO = 'L',
   where U is an upper triangular matrix and L is lower triangular.

   This is the unblocked version of the algorithm, calling Level 2 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the upper or lower triangular part of the
   Hermitian matrix A is stored.
   = 'U':  Upper triangular
   = 'L':  Lower triangular

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the Hermitian matrix A.  If UPLO = 'U', the leading
   n by n upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
```

```
   leading n by n lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

   On exit, if INFO = 0, the factor U or L from the Cholesky
   factorization A = U**H *U  or A = L*L**H.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -k, the k-th argument had an illegal value
   > 0: if INFO = k, the leading minor of order k is not
        positive definite, and the factorization could not be completed.
```

73.10.3 fortran code

— zpotf2.f —

```
*  =====================================================================
      SUBROUTINE ZPOTF2( UPLO, N, A, LDA, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
      COMPLEX*16         CONE
      PARAMETER          ( CONE = ( 1.0D+0, 0.0D+0 ) )
```

```
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            J
      DOUBLE PRECISION   AJJ
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, DISNAN
      COMPLEX*16         ZDOTC
      EXTERNAL           LSAME, ZDOTC, DISNAN
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZDSCAL, ZGEMV, ZLACGV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZPOTF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*        Compute the Cholesky factorization A = U**H *U.
*
         DO 10 J = 1, N
*
*           Compute U(J,J) and test for non-positive-definiteness.
*
            AJJ = DBLE( A( J, J ) ) - ZDOTC( J-1, A( 1, J ), 1,
     $            A( 1, J ), 1 )
```

```
            IF( AJJ.LE.ZERO.OR.DISNAN( AJJ ) ) THEN
               A( J, J ) = AJJ
               GO TO 30
            END IF
            AJJ = SQRT( AJJ )
            A( J, J ) = AJJ
*
*           Compute elements J+1:N of row J.
*
            IF( J.LT.N ) THEN
               CALL ZLACGV( J-1, A( 1, J ), 1 )
               CALL ZGEMV( 'Transpose', J-1, N-J, -CONE, A( 1, J+1 ),
     $                     LDA, A( 1, J ), 1, CONE, A( J, J+1 ), LDA )
               CALL ZLACGV( J-1, A( 1, J ), 1 )
               CALL ZDSCAL( N-J, ONE / AJJ, A( J, J+1 ), LDA )
            END IF
   10    CONTINUE
      ELSE
*
*        Compute the Cholesky factorization A = L*L**H.
*
         DO 20 J = 1, N
*
*           Compute L(J,J) and test for non-positive-definiteness.
*
            AJJ = DBLE( A( J, J ) ) - ZDOTC( J-1, A( J, 1 ), LDA,
     $            A( J, 1 ), LDA )
            IF( AJJ.LE.ZERO.OR.DISNAN( AJJ ) ) THEN
               A( J, J ) = AJJ
               GO TO 30
            END IF
            AJJ = SQRT( AJJ )
            A( J, J ) = AJJ
*
*           Compute elements J+1:N of column J.
*
            IF( J.LT.N ) THEN
               CALL ZLACGV( J-1, A( J, 1 ), LDA )
               CALL ZGEMV( 'No transpose', N-J, J-1, -CONE, A( J+1, 1 ),
     $                     LDA, A( J, 1 ), LDA, CONE, A( J+1, J ), 1 )
               CALL ZLACGV( J-1, A( J, 1 ), LDA )
               CALL ZDSCAL( N-J, ONE / AJJ, A( J+1, J ), 1 )
            END IF
   20    CONTINUE
      END IF
      GO TO 40
*
   30 CONTINUE
      INFO = J
*
```

```
   40 CONTINUE
      RETURN
*
*     End of ZPOTF2
*
      END
```

———-

73.10.4 lisp code

— zpotf2.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0) (cone (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (f2cl-lib:complex16) cone)
           (ignorable one zero cone))
  (defun zpotf2 (uplo n a lda info)
    (declare (type (array f2cl-lib:complex16 (*)) a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%))
      (prog ((ajj 0.0d0) (j 0) (upper nil))
        (declare (type (double-float) ajj)
                 (type (f2cl-lib:integer4) j)
                 (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
          (setf info -4)))
```

```
(cond
 ((/= info 0) (xerbla "ZPOTF2" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(cond
 (upper
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf ajj
              (coerce
               (realpart
                (-
                 (f2cl-lib:dble
                  (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                 a-%offset%))
                 (zdotc (f2cl-lib:int-sub j 1)
                  (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                        (1 j) ((1 lda) (1 *))
                                        a-%offset%)
                  1
                  (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                        (1 j) ((1 lda) (1 *))
                                        a-%offset%)
                  1)))
               'double-float))
      (cond
       ((or (<= ajj zero)
            (multiple-value-bind (ret-val var-0)
                (disnan ajj)
              (declare (ignore))
              (when var-0 (setf ajj var-0))
              ret-val))
        (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
                (coerce ajj 'f2cl-lib:complex16))
        (go label30)))
      (setf ajj (f2cl-lib:fsqrt ajj))
      (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
              (coerce ajj 'f2cl-lib:complex16))
      (cond
       ((< j n)
        (zlacgv (f2cl-lib:int-sub j 1)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 j)
                               ((1 lda) (1 *)) a-%offset%)
         1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10)
            (zgemv "Transpose" (f2cl-lib:int-sub j 1)
```

```
                (f2cl-lib:int-sub n j) (- cone)
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                      (1 (f2cl-lib:int-add j 1))
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 j)
                                      ((1 lda) (1 *)) a-%offset%)
                1 cone
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                      (j (f2cl-lib:int-add j 1))
                                      ((1 lda) (1 *)) a-%offset%)
                lda)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7 var-9))
              (when var-5 (setf lda var-5))
              (when var-8 (setf cone var-8))
              (when var-10 (setf lda var-10)))
            (zlacgv (f2cl-lib:int-sub j 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 j)
                                   ((1 lda) (1 *)) a-%offset%)
             1)
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (zdscal (f2cl-lib:int-sub n j) (/ one ajj)
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                       (j (f2cl-lib:int-add j 1))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare (ignore var-0 var-1 var-2))
              (when var-3 (setf lda var-3)))))
        label10)))
      (t
       (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                     ((> j n) nil)
         (tagbody
           (setf ajj
                 (coerce
                  (realpart
                   (-
                    (f2cl-lib:dble
                     (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                    a-%offset%))
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4)
                        (zdotc (f2cl-lib:int-sub j 1)
                         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                               (j 1) ((1 lda) (1 *))
                                               a-%offset%)
                         lda
                         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                               (j 1) ((1 lda) (1 *))
```

```
                                        a-%offset%)
                lda)
              (declare (ignore var-0 var-1 var-3))
              (when var-2 (setf lda var-2))
              (when var-4 (setf lda var-4))
              ret-val)))
         'double-float))
(cond
 ((or (<= ajj zero)
      (multiple-value-bind (ret-val var-0)
          (disnan ajj)
        (declare (ignore))
        (when var-0 (setf ajj var-0))
        ret-val))
  (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
          (coerce ajj 'f2cl-lib:complex16))
  (go label30)))
(setf ajj (f2cl-lib:fsqrt ajj))
(setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
        (coerce ajj 'f2cl-lib:complex16))
(cond
 ((< j n)
  (zlacgv (f2cl-lib:int-sub j 1)
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (j 1)
                         ((1 lda) (1 *)) a-%offset%)
   lda)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (zgemv "No transpose" (f2cl-lib:int-sub n j)
       (f2cl-lib:int-sub j 1) (- cone)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             ((+ j 1) 1) ((1 lda) (1 *))
                             a-%offset%)
       lda
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (j 1)
                             ((1 lda) (1 *)) a-%offset%)
       lda cone
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             ((+ j 1) j) ((1 lda) (1 *))
                             a-%offset%)
       1)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9 var-10))
    (when var-5 (setf lda var-5))
    (when var-7 (setf lda var-7))
    (when var-8 (setf cone var-8)))
  (zlacgv (f2cl-lib:int-sub j 1)
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (j 1)
                         ((1 lda) (1 *)) a-%offset%)
```

```
                 lda)
                (zdscal (f2cl-lib:int-sub n j) (/ one ajj)
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 ((+ j 1) j)
                                       ((1 lda) (1 *)) a-%offset%)
                 1)))
             label20))))
        (go label40)
       label30
        (setf info j)
       label40
        (go end_label)
       end_label
        (return (values uplo nil nil lda info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zpotf2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil
                                              fortran-to-lisp::lda
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::zlacgv
                                              fortran-to-lisp::xerbla))))
```

———-

73.11 zpotrf

73.11.1 Axiom unit tests

— **zpotrf.input** —

```
)set break resume
)sys rm -f zpotrf.output
)spool zpotrf.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

73.11.2 Axiom help page

— **zpotrf.help** —

```
==================================================================
zpotrf examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zpotrf (UPLO, N, A, LDA, INFO)
           ZPOTRF

Function/Subroutine Documentation
   subroutine zpotrf (character UPLO, integer N, complex*16, dimension( lda, *
       ) A, integer LDA, integer INFO)
       ZPOTRF

Purpose:
   ZPOTRF computes the Cholesky factorization of a complex Hermitian
   positive definite matrix A.

   The factorization has the form
      A = U**H * U,  if UPLO = 'U', or
      A = L  * L**H,  if UPLO = 'L',
   where U is an upper triangular matrix and L is lower triangular.

   This is the block version of the algorithm, calling Level 3 BLAS.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.
```

```
N
  N is INTEGER
  The order of the matrix A.  N >= 0.

A
  A is COMPLEX*16 array, dimension (LDA,N)
  On entry, the Hermitian matrix A.  If UPLO = 'U', the leading
  N-by-N upper triangular part of A contains the upper
  triangular part of the matrix A, and the strictly lower
  triangular part of A is not referenced.  If UPLO = 'L', the
  leading N-by-N lower triangular part of A contains the lower
  triangular part of the matrix A, and the strictly upper
  triangular part of A is not referenced.

  On exit, if INFO = 0, the factor U or L from the Cholesky
  factorization A = U**H *U or A = L*L**H.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, the leading minor of order i is not
        positive definite, and the factorization could not be completed.
```

———-

73.11.3 fortran code

— zpotrf.f —

```
*  =====================================================================
      SUBROUTINE ZPOTRF( UPLO, N, A, LDA, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
```

```
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      COMPLEX*16         CONE
      PARAMETER          ( ONE = 1.0D+0, CONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            J, JB, NB
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGEMM, ZHERK, ZPOTRF2, ZTRSM
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZPOTRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Determine the block size for this environment.
```

```
*
      NB = ILAENV( 1, 'ZPOTRF', UPLO, N, -1, -1, -1 )
      IF( NB.LE.1 .OR. NB.GE.N ) THEN
*
*        Use unblocked code.
*
         CALL ZPOTRF2( UPLO, N, A, LDA, INFO )
      ELSE
*
*        Use blocked code.
*
         IF( UPPER ) THEN
*
*           Compute the Cholesky factorization A = U**H *U.
*
            DO 10 J = 1, N, NB
*
*              Update and factorize the current diagonal block and test
*              for non-positive-definiteness.
*
               JB = MIN( NB, N-J+1 )
               CALL ZHERK( 'Upper', 'Conjugate transpose', JB, J-1,
     $                     -ONE, A( 1, J ), LDA, ONE, A( J, J ), LDA )
               CALL ZPOTRF2( 'Upper', JB, A( J, J ), LDA, INFO )
               IF( INFO.NE.0 )
     $            GO TO 30
               IF( J+JB.LE.N ) THEN
*
*                 Compute the current block row.
*
                  CALL ZGEMM( 'Conjugate transpose', 'No transpose', JB,
     $                        N-J-JB+1, J-1, -CONE, A( 1, J ), LDA,
     $                        A( 1, J+JB ), LDA, CONE, A( J, J+JB ),
     $                        LDA )
                  CALL ZTRSM( 'Left', 'Upper', 'Conjugate transpose',
     $                        'Non-unit', JB, N-J-JB+1, CONE, A( J, J ),
     $                        LDA, A( J, J+JB ), LDA )
               END IF
   10       CONTINUE
*
         ELSE
*
*           Compute the Cholesky factorization A = L*L**H.
*
            DO 20 J = 1, N, NB
*
*              Update and factorize the current diagonal block and test
*              for non-positive-definiteness.
*
               JB = MIN( NB, N-J+1 )
```

```
               CALL ZHERK( 'Lower', 'No transpose', JB, J-1, -ONE,
     $                     A( J, 1 ), LDA, ONE, A( J, J ), LDA )
               CALL ZPOTRF2( 'Lower', JB, A( J, J ), LDA, INFO )
               IF( INFO.NE.0 )
     $            GO TO 30
               IF( J+JB.LE.N ) THEN
*
*                 Compute the current block column.
*
                  CALL ZGEMM( 'No transpose', 'Conjugate transpose',
     $                        N-J-JB+1, JB, J-1, -CONE, A( J+JB, 1 ),
     $                        LDA, A( J, 1 ), LDA, CONE, A( J+JB, J ),
     $                        LDA )
                  CALL ZTRSM( 'Right', 'Lower', 'Conjugate transpose',
     $                        'Non-unit', N-J-JB+1, JB, CONE, A( J, J ),
     $                        LDA, A( J+JB, J ), LDA )
               END IF
   20       CONTINUE
         END IF
      END IF
      GO TO 40
*
   30 CONTINUE
      INFO = INFO + J - 1
*
   40 CONTINUE
      RETURN
*
*     End of ZPOTRF
*
      END
```

———-

73.11.4 lisp code

— zpotrf.lisp —

```
(let* ((one 1.0d0) (cone (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (f2cl-lib:complex16) cone)
           (ignorable one cone))
  (defun zpotrf (uplo n a lda info)
    (declare (type (array f2cl-lib:complex16 (*)) a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
```

```
    (a f2cl-lib:complex16 a-%data% a-%offset%))
  (prog ((j 0) (jb 0) (nb 0) (upper nil))
    (declare (type (f2cl-lib:integer4) j jb nb)
             (type f2cl-lib:logical upper))
    (setf info 0)
    (setf upper
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val))
    (cond
     ((and (not upper)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)))
      (setf info -1))
     ((< n 0) (setf info -2))
     ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -4)))
    (cond
     ((/= info 0) (xerbla "ZPOTRF" (f2cl-lib:int-sub info))
      (go end_label)))
    (if (= n 0)
        (go end_label))
    (setf nb
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 1 "ZPOTRF" uplo n -1 -1 -1)
              (declare (ignore var-0 var-1 var-4 var-5 var-6))
              (setf uplo var-2)
              (setf n var-3)
              ret-val))
    (cond
     ((or (<= nb 1) (>= nb n))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (zpotrf2 uplo n a lda info)
        (declare (ignore var-2))
        (when var-0 (setf uplo var-0))
        (when var-1 (setf n var-1))
        (when var-3 (setf lda var-3))
        (when var-4 (setf info var-4))))
     (t
      (cond
       (upper
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                      ((> j n) nil)
```

```
(tagbody
  (setf jb
          (min (the f2cl-lib:integer4 nb)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub n j)
                                      1))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (zherk "Upper" "Conjugate transpose" jb
       (f2cl-lib:int-sub j 1) (- one)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 j)
                             ((1 lda) (1 *)) a-%offset%)
       lda one
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (j j)
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-3 var-4 var-5 var-8))
    (when var-2 (setf jb var-2))
    (when var-6 (setf lda var-6))
    (when var-7 (setf one var-7))
    (when var-9 (setf lda var-9)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (zpotrf2 "Upper" jb
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (j j)
                             ((1 lda) (1 *)) a-%offset%)
       lda info)
    (declare (ignore var-0 var-2))
    (when var-1 (setf jb var-1))
    (when var-3 (setf lda var-3))
    (when var-4 (setf info var-4)))
  (if (/= info 0)
      (go label30))
  (cond
   ((<= (f2cl-lib:int-add j jb) n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (zgemm "Conjugate transpose" "No transpose" jb
         (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1)
         (f2cl-lib:int-sub j 1) (- cone)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 j)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               (1 (f2cl-lib:int-add j jb))
                               ((1 lda) (1 *)) a-%offset%)
         lda cone
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               (j (f2cl-lib:int-add j jb))
```

```
                                    ((1 lda) (1 *)) a-%offset%)
                lda)
              (declare
               (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-8 var-11))
              (when var-2 (setf jb var-2))
              (when var-7 (setf lda var-7))
              (when var-9 (setf lda var-9))
              (when var-10 (setf cone var-10))
              (when var-12 (setf lda var-12)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10)
                (ztrsm "Left" "Upper" "Conjugate transpose" "Non-unit" jb
                 (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) cone
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (j j)
                                    ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                    (j (f2cl-lib:int-add j jb))
                                    ((1 lda) (1 *)) a-%offset%)
                lda)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9))
              (when var-4 (setf jb var-4))
              (when var-6 (setf cone var-6))
              (when var-8 (setf lda var-8))
              (when var-10 (setf lda var-10)))))
        label10)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                   ((> j n) nil)
       (tagbody
         (setf jb
                 (min (the f2cl-lib:integer4 nb)
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add (f2cl-lib:int-sub n j)
                                             1))))
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
              var-9)
             (zherk "Lower" "No transpose" jb (f2cl-lib:int-sub j 1)
              (- one)
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (j 1)
                                    ((1 lda) (1 *)) a-%offset%)
              lda one
              (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (j j)
                                    ((1 lda) (1 *)) a-%offset%)
              lda)
           (declare (ignore var-0 var-1 var-3 var-4 var-5 var-8))
           (when var-2 (setf jb var-2))
```

```
  (when var-6 (setf lda var-6))
  (when var-7 (setf one var-7))
  (when var-9 (setf lda var-9)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (zpotrf2 "Lower" jb
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (j j)
                           ((1 lda) (1 *)) a-%offset%)
     lda info)
  (declare (ignore var-0 var-2))
  (when var-1 (setf jb var-1))
  (when var-3 (setf lda var-3))
  (when var-4 (setf info var-4)))
(if (/= info 0)
    (go label30))
(cond
 ((<= (f2cl-lib:int-add j jb) n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "No transpose" "Conjugate transpose"
       (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) jb
       (f2cl-lib:int-sub j 1) (- cone)
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             ((+ j jb) 1) ((1 lda) (1 *))
                             a-%offset%)
       lda
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (j 1)
                             ((1 lda) (1 *)) a-%offset%)
       lda cone
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             ((+ j jb) j) ((1 lda) (1 *))
                             a-%offset%)
       lda)
    (declare
     (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-8 var-11))
    (when var-3 (setf jb var-3))
    (when var-7 (setf lda var-7))
    (when var-9 (setf lda var-9))
    (when var-10 (setf cone var-10))
    (when var-12 (setf lda var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (ztrsm "Right" "Lower" "Conjugate transpose" "Non-unit"
       (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) jb cone
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (j j)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                             ((+ j jb) j) ((1 lda) (1 *))
```

```
                                          a-%offset%)
                      lda)
                    (declare
                     (ignore var-0 var-1 var-2 var-3 var-4 var-7 var-9))
                    (when var-5 (setf jb var-5))
                    (when var-6 (setf cone var-6))
                    (when var-8 (setf lda var-8))
                    (when var-10 (setf lda var-10)))))
               label20))))))
        (go label40)
       label30
        (setf info (f2cl-lib:int-sub (f2cl-lib:int-add info j) 1))
       label40
        (go end_label)
       end_label
        (return (values uplo n nil lda info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zpotrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil
                                              fortran-to-lisp::lda
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::ilaenv
                                              fortran-to-lisp::xerbla))))
```

———-

73.12 zpotrf2

73.12.1 Axiom unit tests

— zpotrf2.input —

```
)set break resume
)sys rm -f zpotrf2.output
)spool zpotrf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

73.12.2 Axiom help page

— **zpotrf2.help** —

```
======================================================================
zpotrf2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       recursive subroutine zpotrf2 (UPLO, N, A, LDA, INFO)
           ZPOTRF2

Function/Subroutine Documentation
   recursive subroutine zpotrf2 (character UPLO, integer N, complex*16,
       dimension( lda, * ) A, integer LDA, integer INFO)
       ZPOTRF2

Purpose:
   ZPOTRF2 computes the Cholesky factorization of a real symmetric
   positive definite matrix A using the recursive algorithm.

   The factorization has the form
      A = U**H * U,  if UPLO = 'U', or
      A = L  * L**H,  if UPLO = 'L',
   where U is an upper triangular matrix and L is lower triangular.

   This is the recursive version of the algorithm. It divides
   the matrix into four submatrices:

           [  A11 | A12  ]  where A11 is n1 by n1 and A22 is n2 by n2
      A = [ -----|----- ]  with n1 = n/2
```

```
          [  A21 | A22  ]       n2 = n-n1

   The subroutine calls itself to factor A11. Update and scale A21
   or A12, update A22 then call itself to factor A22.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the symmetric matrix A.  If UPLO = 'U', the leading
   N-by-N upper triangular part of A contains the upper
   triangular part of the matrix A, and the strictly lower
   triangular part of A is not referenced.  If UPLO = 'L', the
   leading N-by-N lower triangular part of A contains the lower
   triangular part of the matrix A, and the strictly upper
   triangular part of A is not referenced.

   On exit, if INFO = 0, the factor U or L from the Cholesky
   factorization A = U**H*U or A = L*L**H.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, the leading minor of order i is not
         positive definite, and the factorization could not be completed.
```

73.12.3 fortran code

— zpotrf2.f —

```
*  =====================================================================
```

```
      RECURSIVE SUBROUTINE ZPOTRF2( UPLO, N, A, LDA, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
      COMPLEX*16         CONE
      PARAMETER          ( CONE = (1.0D+0, 0.0D+0) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            N1, N2, IINFO
      DOUBLE PRECISION   AJJ
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, DISNAN
      EXTERNAL           LSAME, DISNAN
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZHERK, ZTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, DBLE, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
```

```
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZPOTRF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     N=1 case
*
      IF( N.EQ.1 ) THEN
*
*        Test for non-positive-definiteness
*
         AJJ = DBLE( A( 1, 1 ) )
         IF( AJJ.LE.ZERO.OR.DISNAN( AJJ ) ) THEN
            INFO = 1
            RETURN
         END IF
*
*        Factor
*
         A( 1, 1 ) = SQRT( AJJ )
*
*     Use recursive code
*
      ELSE
         N1 = N/2
         N2 = N-N1
*
*        Factor A11
*
         CALL ZPOTRF2( UPLO, N1, A( 1, 1 ), LDA, IINFO )
         IF ( IINFO.NE.0 ) THEN
            INFO = IINFO
            RETURN
         END IF
*
*        Compute the Cholesky factorization A = U**H*U
*
         IF( UPPER ) THEN
*
*           Update and scale A12
*
            CALL ZTRSM( 'L', 'U', 'C', 'N', N1, N2, CONE,
     $                  A( 1, 1 ), LDA, A( 1, N1+1 ), LDA )
*
```

```
*          Update and factor A22
*
           CALL ZHERK( UPLO, 'C', N2, N1, -ONE, A( 1, N1+1 ), LDA,
     $                 ONE, A( N1+1, N1+1 ), LDA )
           CALL ZPOTRF2( UPLO, N2, A( N1+1, N1+1 ), LDA, IINFO )
           IF ( IINFO.NE.0 ) THEN
              INFO = IINFO + N1
              RETURN
           END IF
*
*       Compute the Cholesky factorization A = L*L**H
*
        ELSE
*
*          Update and scale A21
*
           CALL ZTRSM( 'R', 'L', 'C', 'N', N2, N1, CONE,
     $                 A( 1, 1 ), LDA, A( N1+1, 1 ), LDA )
*
*          Update and factor A22
*
           CALL ZHERK( UPLO, 'N', N2, N1, -ONE, A( N1+1, 1 ), LDA,
     $                 ONE, A( N1+1, N1+1 ), LDA )
           CALL ZPOTRF2( UPLO, N2, A( N1+1, N1+1 ), LDA, IINFO )
           IF ( IINFO.NE.0 ) THEN
              INFO = IINFO + N1
              RETURN
           END IF
        END IF
     END IF
     RETURN
*
*    End of ZPOTRF2
*
     END
```

———-

73.12.4 lisp code

— zpotrf2.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0) (cone (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (f2cl-lib:complex16) cone)
           (ignorable one zero cone))
  (defun *main* ()
```

```
(prog ((ajj 0.0d0) (n1 0) (n2 0) (iinfo 0) (upper nil)
       (uplo
        (make-array '(1) :element-type 'character :initial-element #\ ))
       (info 0) (lda 0)
       (a
        (make-array (the fixnum (reduce #'* (list lda *))) :element-type
                    'f2cl-lib:complex16))
       (n 0) (subroutine 0.0))
  (declare (type (single-float) subroutine)
           (type (array f2cl-lib:complex16 (*)) a)
           (type (double-float) ajj)
           (type (f2cl-lib:integer4) n lda info n1 n2 iinfo)
           (type f2cl-lib:logical upper)
           (type (string 1) uplo))
  '"****NOT TRANSLATED: (RECURSIVE SUBROUTINE ZPOTRF2 (UPLO |,| N |,| A |,| LDA |,| INFO))"
  (setf info 0)
  (setf upper
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
  (cond
   ((and (not upper)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4)))
  (cond
   ((/= info 0) (xerbla "ZPOTRF2" (f2cl-lib:int-sub info)) (go end_label)))
  (if (= n 0)
      (go end_label))
  (cond
   ((= n 1)
    (setf ajj (f2cl-lib:dble (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))))
    (cond
     ((or (<= ajj zero)
          (multiple-value-bind (ret-val var-0)
              (disnan ajj)
            (declare (ignore))
            (when var-0 (setf ajj var-0))
            ret-val))
      (setf info 1) (go end_label)))
    (setf (f2cl-lib:fref a (1 1) ((1 lda) (1 *)))
```

```
          (coerce (f2cl-lib:fsqrt ajj) 'f2cl-lib:complex16)))
(t (setf n1 (the f2cl-lib:integer4 (truncate n 2)))
 (setf n2 (f2cl-lib:int-sub n n1))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (zpotrf2 uplo n1
      (f2cl-lib:array-slice a f2cl-lib:complex16 (1 1) ((1 lda) (1 *)))
      lda iinfo)
   (declare (ignore var-2))
   (when var-0 (setf uplo var-0))
   (when var-1 (setf n1 var-1))
   (when var-3 (setf lda var-3))
   (when var-4 (setf iinfo var-4)))
 (cond ((/= iinfo 0) (setf info iinfo) (go end_label)))
 (cond
  (upper
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (ztrsm "L" "U" "C" "N" n1 n2 cone
        (f2cl-lib:array-slice a f2cl-lib:complex16 (1 1)
                              ((1 lda) (1 *)))
        lda
        (f2cl-lib:array-slice a f2cl-lib:complex16
                              (1 (f2cl-lib:int-add n1 1))
                              ((1 lda) (1 *)))
        lda)
     (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
     (when var-4 (setf n1 var-4))
     (when var-5 (setf n2 var-5))
     (when var-6 (setf cone var-6))
     (when var-8 (setf lda var-8))
     (when var-10 (setf lda var-10)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (zherk uplo "C" n2 n1 (- one)
        (f2cl-lib:array-slice a f2cl-lib:complex16
                              (1 (f2cl-lib:int-add n1 1))
                              ((1 lda) (1 *)))
        lda one
        (f2cl-lib:array-slice a f2cl-lib:complex16
                              ((+ n1 1) (f2cl-lib:int-add n1 1))
                              ((1 lda) (1 *)))
        lda)
     (declare (ignore var-1 var-4 var-5 var-8))
     (when var-0 (setf uplo var-0))
     (when var-2 (setf n2 var-2))
     (when var-3 (setf n1 var-3))
     (when var-6 (setf lda var-6))
     (when var-7 (setf one var-7))
     (when var-9 (setf lda var-9)))
```

```
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (zpotrf2 uplo n2
      (f2cl-lib:array-slice a f2cl-lib:complex16
                            ((+ n1 1) (f2cl-lib:int-add n1 1))
                            ((1 lda) (1 *)))
      lda iinfo)
   (declare (ignore var-2))
   (when var-0 (setf uplo var-0))
   (when var-1 (setf n2 var-1))
   (when var-3 (setf lda var-3))
   (when var-4 (setf iinfo var-4)))
 (cond
  ((/= iinfo 0) (setf info (f2cl-lib:int-add iinfo n1))
   (go end_label))))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (ztrsm "R" "L" "C" "N" n2 n1 cone
      (f2cl-lib:array-slice a f2cl-lib:complex16 (1 1)
                            ((1 lda) (1 *)))
      lda
      (f2cl-lib:array-slice a f2cl-lib:complex16 ((+ n1 1) 1)
                            ((1 lda) (1 *)))
      lda)
   (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
   (when var-4 (setf n2 var-4))
   (when var-5 (setf n1 var-5))
   (when var-6 (setf cone var-6))
   (when var-8 (setf lda var-8))
   (when var-10 (setf lda var-10)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
     (zherk uplo "N" n2 n1 (- one)
      (f2cl-lib:array-slice a f2cl-lib:complex16 ((+ n1 1) 1)
                            ((1 lda) (1 *)))
      lda one
      (f2cl-lib:array-slice a f2cl-lib:complex16
                            ((+ n1 1) (f2cl-lib:int-add n1 1))
                            ((1 lda) (1 *)))
      lda)
   (declare (ignore var-1 var-4 var-5 var-8))
   (when var-0 (setf uplo var-0))
   (when var-2 (setf n2 var-2))
   (when var-3 (setf n1 var-3))
   (when var-6 (setf lda var-6))
   (when var-7 (setf one var-7))
   (when var-9 (setf lda var-9)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (zpotrf2 uplo n2
```

```
              (f2cl-lib:array-slice a f2cl-lib:complex16
                                    ((+ n1 1) (f2cl-lib:int-add n1 1))
                                    ((1 lda) (1 *)))
              lda iinfo)
            (declare (ignore var-2))
            (when var-0 (setf uplo var-0))
            (when var-1 (setf n2 var-1))
            (when var-3 (setf lda var-3))
            (when var-4 (setf iinfo var-4)))
          (cond
           ((/= iinfo 0) (setf info (f2cl-lib:int-add iinfo n1))
            (go end_label)))))))
      (go end_label)
     end_label
      (return nil))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::*main*
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types 'nil :return-values 'nil
                                            :calls 'nil)))
```

———-

73.13 zpotri

73.13.1 Axiom unit tests

— **zpotri.input** —

```
)set break resume
)sys rm -f zpotri.output
)spool zpotri.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

73.13.2 Axiom help page

— zpotri.help —

```
====================================================================
zpotri examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zpotri (UPLO, N, A, LDA, INFO)
           ZPOTRI

Function/Subroutine Documentation
   subroutine zpotri (character UPLO, integer N, complex*16, dimension( lda, *
       ) A, integer LDA, integer INFO)
       ZPOTRI

Purpose:
   ZPOTRI computes the inverse of a complex Hermitian positive definite
   matrix A using the Cholesky factorization A = U**H*U or A = L*L**H
   computed by ZPOTRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   On entry, the triangular factor U or L from the Cholesky
   factorization A = U**H*U or A = L*L**H, as computed by ZPOTRF.
   On exit, the upper or lower triangle of the (Hermitian)
   inverse of A, overwriting the input factor U or L.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).
```

```
 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, the (i,i) element of the factor U or L is
         zero, and the inverse could not be computed.
```

73.13.3 fortran code

— zpotri.f —

```
*  =====================================================================
      SUBROUTINE ZPOTRI( UPLO, N, A, LDA, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLAUUM, ZTRTRI
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
```

```
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZPOTRI', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Invert the triangular Cholesky factor U or L.
*
      CALL ZTRTRI( UPLO, 'Non-unit', N, A, LDA, INFO )
      IF( INFO.GT.0 )
     $   RETURN
*
*     Form inv(U) * inv(U)**H or inv(L)**H * inv(L).
*
      CALL ZLAUUM( UPLO, N, A, LDA, INFO )
*
      RETURN
*
*     End of ZPOTRI
*
      END
```

73.13.4 lisp code

— zpotri.lisp —

```
(defun zpotri (uplo n a lda info)
  (declare (type (array f2cl-lib:complex16 (*)) a)
           (type (f2cl-lib:integer4) info lda n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%))
    (prog ()
      (declare)
      (setf info 0)
```

```
      (cond
       ((and
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -4)))
      (cond
       ((/= info 0) (xerbla "ZPOTRI" (f2cl-lib:int-sub info)) (go end_label)))
      (if (= n 0)
          (go end_label))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (ztrtri uplo "Non-unit" n a lda info)
        (declare (ignore var-1 var-3))
        (when var-0 (setf uplo var-0))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-5 (setf info var-5)))
      (if (> info 0)
          (go end_label))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (zlauum uplo n a lda info)
        (declare (ignore var-2))
        (setf uplo var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf info var-4))
      (go end_label)
     end_label
      (return (values uplo n nil lda info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zpotri
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
```

```
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::uplo
  fortran-to-lisp::n nil
  fortran-to-lisp::lda
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zlauum
  fortran-to-lisp::xerbla))))
```

———-

73.14 zpotrs

73.14.1 Axiom unit tests

— zpotrs.input —

```
)set break resume
)sys rm -f zpotrs.output
)spool zpotrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

73.14.2 Axiom help page

— zpotrs.help —

```
==================================================================
zpotrs examples
==================================================================

==================================================================
```

```
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zpotrs (UPLO, N, NRHS, A, LDA, B, LDB, INFO)
           ZPOTRS

Function/Subroutine Documentation
   subroutine zpotrs (character UPLO, integer N, integer NRHS, complex*16,
       dimension( lda, * ) A, integer LDA, complex*16, dimension( ldb, * ) B,
       integer LDB, integer INFO)
       ZPOTRS

Purpose:
   ZPOTRS solves a system of linear equations A*X = B with a Hermitian
   positive definite matrix A using the Cholesky factorization
   A = U**H * U or A = L * L**H computed by ZPOTRF.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  Upper triangle of A is stored;
   = 'L':  Lower triangle of A is stored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 A
   A is COMPLEX*16 array, dimension (LDA,N)
   The triangular factor U or L from the Cholesky factorization
   A = U**H * U or A = L * L**H, as computed by ZPOTRF.

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,N).

 B
   B is COMPLEX*16 array, dimension (LDB,NRHS)
   On entry, the right hand side matrix B.
   On exit, the solution matrix X.

 LDB
```

```
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

73.14.3 fortran code

— zpotrs.f —

```
*  =====================================================================
      SUBROUTINE ZPOTRS( UPLO, N, NRHS, A, LDA, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZTRSM
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZPOTRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( UPPER ) THEN
*
*        Solve A*X = B where A = U**H *U.
*
*        Solve U**H *X = B, overwriting B with X.
*
         CALL ZTRSM( 'Left', 'Upper', 'Conjugate transpose', 'Non-unit',
     $               N, NRHS, ONE, A, LDA, B, LDB )
*
*        Solve U*X = B, overwriting B with X.
*
         CALL ZTRSM( 'Left', 'Upper', 'No transpose', 'Non-unit', N,
     $               NRHS, ONE, A, LDA, B, LDB )
      ELSE
*
*        Solve A*X = B where A = L*L**H.
*
*        Solve L*X = B, overwriting B with X.
*
         CALL ZTRSM( 'Left', 'Lower', 'No transpose', 'Non-unit', N,
     $               NRHS, ONE, A, LDA, B, LDB )
```

```
*
*        Solve L**H *X = B, overwriting B with X.
*
         CALL ZTRSM( 'Left', 'Lower', 'Conjugate transpose', 'Non-unit',
     $               N, NRHS, ONE, A, LDA, B, LDB )
      END IF
*
      RETURN
*
*     End of ZPOTRS
*
      END
```

———-

73.14.4 lisp code

— **zpotrs.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (ignorable one))
  (defun zpotrs (uplo n nrhs a lda b ldb$ info)
    (declare (type (array f2cl-lib:complex16 (*)) b a)
             (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%))
      (prog ((upper nil))
        (declare (type f2cl-lib:logical upper))
        (setf info 0)
        (setf upper
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "U")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and (not upper)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val)))
          (setf info -1))
```

```
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7)))
(cond
 ((/= info 0) (xerbla "ZPOTRS" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= n 0) (= nrhs 0))
    (go end_label))
(cond
 (upper
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ztrsm "Left" "Upper" "Conjugate transpose" "Non-unit" n nrhs one
       a lda b ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ztrsm "Left" "Upper" "No transpose" "Non-unit" n nrhs one a lda
       b ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ztrsm "Left" "Lower" "No transpose" "Non-unit" n nrhs one a lda
       b ldb$)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf nrhs var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldb$ var-10)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ztrsm "Left" "Lower" "Conjugate transpose" "Non-unit" n nrhs one
```

```
              a lda b ldb$)
            (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
            (when var-4 (setf n var-4))
            (when var-5 (setf nrhs var-5))
            (when var-6 (setf one var-6))
            (when var-8 (setf lda var-8))
            (when var-10 (setf ldb$ var-10)))))
        (go end_label)
       end_label
        (return (values uplo n nrhs nil lda nil ldb$ info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zpotrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil
                                              fortran-to-lisp::lda nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

Chapter 74

LAPACK - General Tridiagonal Matrix, Linear Solve, Double

74.1 dgtsv

74.1.1 Axiom unit tests

— dgtsv.input —

```
)set break resume
)sys rm -f dgtsv.output
)spool dgtsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

74.1.2 Axiom help page

— dgtsv.help —

```
========================================================================
dgtsv examples
========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgtsv (N, NRHS, DL, D, DU, B, LDB, INFO)
            DGTSV computes the solution to system of linear equations A * X =
           B for GT matrices

Function/Subroutine Documentation
   subroutine dgtsv (integer N, integer NRHS, double precision, dimension( * )
       DL, double precision, dimension( * ) D, double precision, dimension( *
       ) DU, double precision, dimension( ldb, * ) B, integer LDB, integer
       INFO)
        DGTSV computes the solution to system of linear equations A * X = B
       for GT matrices

Purpose:
   DGTSV  solves the equation

   A*X = B,

   where A is an n by n tridiagonal matrix, by Gaussian elimination with
   partial pivoting.

   Note that the equation  A**T*X = B  may be solved by interchanging the
   order of the arguments DU and DL.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 DL
   DL is DOUBLE PRECISION array, dimension (N-1)
   On entry, DL must contain the (n-1) sub-diagonal elements of A.

 On exit, DL is overwritten by the (n-2) elements of the

   the LU factorization of A, in DL(1), ..., DL(n-2).

 D
```

```
  D is DOUBLE PRECISION array, dimension (N)
  On entry, D must contain the diagonal elements of A.

  On exit, D is overwritten by the n diagonal elements of U.

DU
  DU is DOUBLE PRECISION array, dimension (N-1)
  On entry, DU must contain the (n-1) super-diagonal elements of A.

  On exit, DU is overwritten by the (n-1) elements of the first
  super-diagonal of U.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the N by NRHS matrix of right hand side matrix B.
  On exit, if INFO = 0, the N by NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = i, U(i,i) is exactly zero, and the solution
       has not been computed.  The factorization has not been
       completed unless i = N.
```

———-

74.1.3 fortran code

— dgtsv.f —

```
*  =====================================================================
      SUBROUTINE DGTSV( N, NRHS, DL, D, DU, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
```

```
      DOUBLE PRECISION   B( LDB, * ), D( * ), DL( * ), DU( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   FACT, TEMP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGTSV ', -INFO )
         RETURN
      END IF
*
      IF( N.EQ.0 )
     $   RETURN
*
      IF( NRHS.EQ.1 ) THEN
         DO 10 I = 1, N - 2
            IF( ABS( D( I ) ).GE.ABS( DL( I ) ) ) THEN
*
*              No row interchange required
*
               IF( D( I ).NE.ZERO ) THEN
                  FACT = DL( I ) / D( I )
                  D( I+1 ) = D( I+1 ) - FACT*DU( I )
                  B( I+1, 1 ) = B( I+1, 1 ) - FACT*B( I, 1 )
               ELSE
                  INFO = I
                  RETURN
```

```
            END IF
            DL( I ) = ZERO
         ELSE
*
*           Interchange rows I and I+1
*
            FACT = D( I ) / DL( I )
            D( I ) = DL( I )
            TEMP = D( I+1 )
            D( I+1 ) = DU( I ) - FACT*TEMP
            DL( I ) = DU( I+1 )
            DU( I+1 ) = -FACT*DL( I )
            DU( I ) = TEMP
            TEMP = B( I, 1 )
            B( I, 1 ) = B( I+1, 1 )
            B( I+1, 1 ) = TEMP - FACT*B( I+1, 1 )
         END IF
   10    CONTINUE
         IF( N.GT.1 ) THEN
            I = N - 1
            IF( ABS( D( I ) ).GE.ABS( DL( I ) ) ) THEN
               IF( D( I ).NE.ZERO ) THEN
                  FACT = DL( I ) / D( I )
                  D( I+1 ) = D( I+1 ) - FACT*DU( I )
                  B( I+1, 1 ) = B( I+1, 1 ) - FACT*B( I, 1 )
               ELSE
                  INFO = I
                  RETURN
               END IF
            ELSE
               FACT = D( I ) / DL( I )
               D( I ) = DL( I )
               TEMP = D( I+1 )
               D( I+1 ) = DU( I ) - FACT*TEMP
               DU( I ) = TEMP
               TEMP = B( I, 1 )
               B( I, 1 ) = B( I+1, 1 )
               B( I+1, 1 ) = TEMP - FACT*B( I+1, 1 )
            END IF
         END IF
         IF( D( N ).EQ.ZERO ) THEN
            INFO = N
            RETURN
         END IF
      ELSE
         DO 40 I = 1, N - 2
            IF( ABS( D( I ) ).GE.ABS( DL( I ) ) ) THEN
*
*           No row interchange required
*
```

```
            IF( D( I ).NE.ZERO ) THEN
               FACT = DL( I ) / D( I )
               D( I+1 ) = D( I+1 ) - FACT*DU( I )
               DO 20 J = 1, NRHS
                  B( I+1, J ) = B( I+1, J ) - FACT*B( I, J )
   20          CONTINUE
            ELSE
               INFO = I
               RETURN
            END IF
            DL( I ) = ZERO
         ELSE
*
*           Interchange rows I and I+1
*
            FACT = D( I ) / DL( I )
            D( I ) = DL( I )
            TEMP = D( I+1 )
            D( I+1 ) = DU( I ) - FACT*TEMP
            DL( I ) = DU( I+1 )
            DU( I+1 ) = -FACT*DL( I )
            DU( I ) = TEMP
            DO 30 J = 1, NRHS
               TEMP = B( I, J )
               B( I, J ) = B( I+1, J )
               B( I+1, J ) = TEMP - FACT*B( I+1, J )
   30       CONTINUE
         END IF
   40    CONTINUE
         IF( N.GT.1 ) THEN
            I = N - 1
            IF( ABS( D( I ) ).GE.ABS( DL( I ) ) ) THEN
               IF( D( I ).NE.ZERO ) THEN
                  FACT = DL( I ) / D( I )
                  D( I+1 ) = D( I+1 ) - FACT*DU( I )
                  DO 50 J = 1, NRHS
                     B( I+1, J ) = B( I+1, J ) - FACT*B( I, J )
   50             CONTINUE
               ELSE
                  INFO = I
                  RETURN
               END IF
            ELSE
               FACT = D( I ) / DL( I )
               D( I ) = DL( I )
               TEMP = D( I+1 )
               D( I+1 ) = DU( I ) - FACT*TEMP
               DU( I ) = TEMP
               DO 60 J = 1, NRHS
                  TEMP = B( I, J )
```

```
                B( I, J ) = B( I+1, J )
                B( I+1, J ) = TEMP - FACT*B( I+1, J )
   60         CONTINUE
            END IF
         END IF
         IF( D( N ).EQ.ZERO ) THEN
            INFO = N
            RETURN
         END IF
      END IF
*
*     Back solve with the matrix U from the factorization.
*
      IF( NRHS.LE.2 ) THEN
         J = 1
   70    CONTINUE
         B( N, J ) = B( N, J ) / D( N )
         IF( N.GT.1 )
     $      B( N-1, J ) = ( B( N-1, J )-DU( N-1 )*B( N, J ) ) / D( N-1 )
         DO 80 I = N - 2, 1, -1
            B( I, J ) = ( B( I, J )-DU( I )*B( I+1, J )-DL( I )*
     $                  B( I+2, J ) ) / D( I )
   80    CONTINUE
         IF( J.LT.NRHS ) THEN
            J = J + 1
            GO TO 70
         END IF
      ELSE
         DO 100 J = 1, NRHS
            B( N, J ) = B( N, J ) / D( N )
            IF( N.GT.1 )
     $         B( N-1, J ) = ( B( N-1, J )-DU( N-1 )*B( N, J ) ) /
     $                       D( N-1 )
            DO 90 I = N - 2, 1, -1
               B( I, J ) = ( B( I, J )-DU( I )*B( I+1, J )-DL( I )*
     $                     B( I+2, J ) ) / D( I )
   90       CONTINUE
  100    CONTINUE
      END IF
*
      RETURN
*
*     End of DGTSV
*
      END
```

———-

74.1.4 lisp code

— dgtsv.lisp —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun dgtsv (n nrhs dl d du b ldb$ info)
    (declare (type (array double-float (*)) b du d dl)
             (type (f2cl-lib:integer4) info ldb$ nrhs n))
    (f2cl-lib:with-multi-array-data
        ((dl double-float dl-%data% dl-%offset%)
         (d double-float d-%data% d-%offset%)
         (du double-float du-%data% du-%offset%)
         (b double-float b-%data% b-%offset%))
      (prog ((fact 0.0d0) (temp 0.0d0) (i 0) (j 0))
        (declare (type (double-float) fact temp)
                 (type (f2cl-lib:integer4) i j))
        (setf info 0)
        (cond ((< n 0) (setf info -1)) ((< nrhs 0) (setf info -2))
              ((< ldb$
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -7)))
        (cond
         ((/= info 0) (xerbla "DGTSV " (f2cl-lib:int-sub info))
          (go end_label)))
        (if (= n 0)
            (go end_label))
        (cond
         ((= nrhs 1)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))) nil)
            (tagbody
              (cond
               ((>= (abs (f2cl-lib:fref d (i) ((1 *))))
                    (abs (f2cl-lib:fref dl (i) ((1 *)))))
                (cond
                 ((/= (f2cl-lib:fref d (i) ((1 *))) zero)
                  (setf fact
                          (/ (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                             (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
                  (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                        ((1 *)) d-%offset%)
                          (-
                           (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                          ((1 *)) d-%offset%)
                           (* fact
                              (f2cl-lib:fref du-%data% (i) ((1 *))
                                             du-%offset%))))
```

```
            (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
                    (-
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
                     (* fact
                        (f2cl-lib:fref b-%data% (i 1) ((1 ldb$) (1 *))
                                       b-%offset%)))))
          (t (setf info i) (go end_label)))
         (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) zero))
        (t
         (setf fact
                 (/ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                    (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
         (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                 (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
         (setf temp
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                  ((1 *)) d-%offset%))
         (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                               d-%offset%)
                 (- (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                    (* fact temp)))
         (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                 (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1))
                                  ((1 *)) du-%offset%))
         (setf (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                               du-%offset%)
                 (* (- fact)
                    (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
         (setf (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%) temp)
         (setf temp
                 (f2cl-lib:fref b-%data% (i 1) ((1 ldb$) (1 *))
                                  b-%offset%))
         (setf (f2cl-lib:fref b-%data% (i 1) ((1 ldb$) (1 *))
                               b-%offset%)
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) 1)
                                  ((1 ldb$) (1 *)) b-%offset%))
         (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) 1)
                               ((1 ldb$) (1 *)) b-%offset%)
                 (- temp
                    (* fact
                       (f2cl-lib:fref b-%data%
                                      ((f2cl-lib:int-add i 1) 1)
                                      ((1 ldb$) (1 *)) b-%offset%))))))
   label10))
(cond
 ((> n 1) (setf i (f2cl-lib:int-sub n 1))
  (cond
   ((>= (abs (f2cl-lib:fref d (i) ((1 *))))
```

```
            (abs (f2cl-lib:fref dl (i) ((1 *)))))
        (cond
         ((/= (f2cl-lib:fref d (i) ((1 *))) zero)
          (setf fact
                  (/ (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                     (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
          (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                d-%offset%)
                  (-
                   (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                  ((1 *)) d-%offset%)
                   (* fact
                      (f2cl-lib:fref du-%data% (i) ((1 *))
                                     du-%offset%))))
          (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) 1)
                                ((1 ldb$) (1 *)) b-%offset%)
                  (-
                   (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
                   (* fact
                      (f2cl-lib:fref b-%data% (i 1) ((1 ldb$) (1 *))
                                     b-%offset%)))))
         (t (setf info i) (go end_label))))
       (t
        (setf fact
                (/ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                   (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
        (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
        (setf temp
                (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                d-%offset%))
        (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                               d-%offset%)
                (- (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                   (* fact temp)))
        (setf (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%) temp)
        (setf temp
                (f2cl-lib:fref b-%data% (i 1) ((1 ldb$) (1 *))
                                b-%offset%))
        (setf (f2cl-lib:fref b-%data% (i 1) ((1 ldb$) (1 *)) b-%offset%)
                (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) 1)
                                ((1 ldb$) (1 *)) b-%offset%))
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) 1)
                               ((1 ldb$) (1 *)) b-%offset%)
                (- temp
                   (* fact
                      (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) 1)
                                     ((1 ldb$) (1 *)) b-%offset%))))))))
    (cond
```

```
    ((= (f2cl-lib:fref d (n) ((1 *))) zero) (setf info n)
     (go end_label))))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))) nil)
     (tagbody
       (cond
        ((>= (abs (f2cl-lib:fref d (i) ((1 *))))
             (abs (f2cl-lib:fref dl (i) ((1 *)))))
         (cond
          ((/= (f2cl-lib:fref d (i) ((1 *))) zero)
           (setf fact
                   (/ (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                      (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
           (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) d-%offset%)
                   (-
                    (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                   ((1 *)) d-%offset%)
                    (* fact
                       (f2cl-lib:fref du-%data% (i) ((1 *))
                                      du-%offset%))))
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j nrhs) nil)
             (tagbody
               (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                     ((1 ldb$) (1 *)) b-%offset%)
                     (-
                      (f2cl-lib:fref b-%data%
                                     ((f2cl-lib:int-add i 1) j)
                                     ((1 ldb$) (1 *)) b-%offset%)
                      (* fact
                         (f2cl-lib:fref b-%data% (i j)
                                        ((1 ldb$) (1 *))
                                        b-%offset%))))
              label20)))
          (t (setf info i) (go end_label)))
         (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) zero))
        (t
         (setf fact
                 (/ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                    (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
         (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                 (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
         (setf temp
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) d-%offset%))
         (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                               d-%offset%)
                 (- (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
```

```
                    (* fact temp)))
            (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                    (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1))
                                   ((1 *)) du-%offset%))
            (setf (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                 du-%offset%)
                    (* (- fact)
                       (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
            (setf (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%) temp)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j nrhs) nil)
              (tagbody
                (setf temp
                        (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%))
                (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                      b-%offset%)
                        (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                       ((1 ldb$) (1 *)) b-%offset%))
                (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                      ((1 ldb$) (1 *)) b-%offset%)
                        (- temp
                           (* fact
                              (f2cl-lib:fref b-%data%
                                             ((f2cl-lib:int-add i 1) j)
                                             ((1 ldb$) (1 *))
                                             b-%offset%))))
             label30))))
   label40))
(cond
 ((> n 1) (setf i (f2cl-lib:int-sub n 1))
  (cond
   ((>= (abs (f2cl-lib:fref d (i) ((1 *))))
        (abs (f2cl-lib:fref dl (i) ((1 *)))))
    (cond
     ((/= (f2cl-lib:fref d (i) ((1 *))) zero)
      (setf fact
              (/ (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                 (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
      (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                           d-%offset%)
              (-
               (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                              ((1 *)) d-%offset%)
               (* fact
                  (f2cl-lib:fref du-%data% (i) ((1 *))
                                 du-%offset%))))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nrhs) nil)
        (tagbody
```

```
                (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                      ((1 ldb$) (1 *)) b-%offset%)
                        (-
                         (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                        ((1 ldb$) (1 *)) b-%offset%)
                         (* fact
                            (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                           b-%offset%))))
             label50)))
          (t (setf info i) (go end_label))))
        (t
         (setf fact
                 (/ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                    (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
         (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                 (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
         (setf temp
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                d-%offset%))
         (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                              d-%offset%)
                 (- (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                    (* fact temp)))
         (setf (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%) temp)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j nrhs) nil)
           (tagbody
             (setf temp
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%))
             (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                    ((1 ldb$) (1 *)) b-%offset%))
             (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                  ((1 ldb$) (1 *)) b-%offset%)
                     (- temp
                        (* fact
                           (f2cl-lib:fref b-%data%
                                          ((f2cl-lib:int-add i 1) j)
                                          ((1 ldb$) (1 *)) b-%offset%))))
           label60))))))
  (cond
   ((= (f2cl-lib:fref d (n) ((1 *))) zero) (setf info n)
    (go end_label)))))
(cond
 ((<= nrhs 2)
  (tagbody
    (setf j 1)
   label70
```

```
(setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
        (/
         (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
         (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
(if (> n 1)
    (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                          ((1 ldb$) (1 *)) b-%offset%)
            (/
             (-
              (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                              ((1 ldb$) (1 *)) b-%offset%)
              (*
               (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub n 1))
                               ((1 *)) du-%offset%)
               (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                               b-%offset%)))
             (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                             ((1 *)) d-%offset%))))
(f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))
               (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                          b-%offset%)
            (/
             (-
              (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)
              (* (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%))
              (* (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 2) j)
                                 ((1 ldb$) (1 *)) b-%offset%)))
             (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
   label80))
(cond ((< j nrhs) (setf j (f2cl-lib:int-add j 1)) (go label70)))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j nrhs) nil)
  (tagbody
    (setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
            (/
             (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                             b-%offset%)
             (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
    (if (> n 1)
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                              ((1 ldb$) (1 *)) b-%offset%)
                (/
```

```
                          (-
                           (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                                          ((1 ldb$) (1 *)) b-%offset%)
                           (*
                            (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub n 1))
                                           ((1 *)) du-%offset%)
                            (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                                           b-%offset%)))
                          (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                         ((1 *)) d-%offset%))))
              (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))
                             (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                            ((> i 1) nil)
                (tagbody
                  (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)
                        (/
                         (-
                          (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                         b-%offset%)
                          (*
                           (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                           (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                          ((1 ldb$) (1 *)) b-%offset%))
                          (*
                           (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                           (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 2) j)
                                          ((1 ldb$) (1 *)) b-%offset%)))
                         (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
                 label90))
             label100))))
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgtsv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil
```

```
                                        fortran-to-lisp::info)
                                      :calls 'nil)))
```

———-

74.2 dgtsvx

74.2.1 Axiom unit tests

— **dgtsvx.input** —

```
)set break resume
)sys rm -f dgtsvx.output
)spool dgtsvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

74.2.2 Axiom help page

— **dgtsvx.help** —

```
======================================================================
dgtsvx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgtsvx (FACT, TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF, DU2,
           IPIV, B, LDB, X, LDX, RCOND, FERR, BERR, WORK, IWORK, INFO)
            DGTSVX computes the solution to system of linear equations A * X =
           B for GT matrices
```

```
Function/Subroutine Documentation
   subroutine dgtsvx (character FACT, character TRANS, integer N, integer
       NRHS, double precision, dimension( * ) DL, double precision, dimension(
       * ) D, double precision, dimension( * ) DU, double precision,
       dimension( * ) DLF, double precision, dimension( * ) DF, double
       precision, dimension( * ) DUF, double precision, dimension( * ) DU2,
       integer, dimension( * ) IPIV, double precision, dimension( ldb, * ) B,
       integer LDB, double precision, dimension( ldx, * ) X, integer LDX,
       double precision RCOND, double precision, dimension( * ) FERR, double
       precision, dimension( * ) BERR, double precision, dimension( * ) WORK,
       integer, dimension( * ) IWORK, integer INFO)
        DGTSVX computes the solution to system of linear equations A * X = B
       for GT matrices

Purpose:
   DGTSVX uses the LU factorization to compute the solution to a real
   system of linear equations A * X = B or A**T * X = B,
   where A is a tridiagonal matrix of order N and X and B are N-by-NRHS
   matrices.

   Error bounds on the solution and a condition estimate are also
   provided.

Description:

   The following steps are performed:

   1. If FACT = 'N', the LU decomposition is used to factor the matrix A
      as A = L * U, where L is a product of permutation and unit lower
      bidiagonal matrices and U is upper triangular with nonzeros in
      only the main diagonal and first two superdiagonals.

   2. If some U(i,i)=0, so that U is exactly singular, then the routine
      returns with INFO = i. Otherwise, the factored form of A is used
      to estimate the condition number of the matrix A.  If the
      reciprocal of the condition number is less than machine precision,
      INFO = N+1 is returned as a warning, but the routine still goes on
      to solve for X and compute error bounds as described below.

   3. The system of equations is solved for X using the factored form of A.

   4. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of A has been
   supplied on entry.
```

```
  = 'F':  DLF, DF, DUF, DU2, and IPIV contain the factored
     form of A; DL, D, DU, DLF, DF, DUF, DU2 and IPIV
     will not be modified.
  = 'N':  The matrix will be copied to DLF, DF, and DUF
     and factored.

TRANS
  TRANS is CHARACTER*1
  Specifies the form of the system of equations:
  = 'N':  A * X = B     (No transpose)
  = 'T':  A**T * X = B  (Transpose)
  = 'C':  A**H * X = B  (Conjugate transpose = Transpose)

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrix B.  NRHS >= 0.

DL
  DL is DOUBLE PRECISION array, dimension (N-1)
  The (n-1) subdiagonal elements of A.

D
  D is DOUBLE PRECISION array, dimension (N)
  The n diagonal elements of A.

DU
  DU is DOUBLE PRECISION array, dimension (N-1)
  The (n-1) superdiagonal elements of A.

DLF
  DLF is DOUBLE PRECISION array, dimension (N-1)
  If FACT = 'F', then DLF is an input argument and on entry
  contains the (n-1) multipliers that define the matrix L from
  the LU factorization of A as computed by DGTTRF.

  If FACT = 'N', then DLF is an output argument and on exit
  contains the (n-1) multipliers that define the matrix L from
  the LU factorization of A.

DF
  DF is DOUBLE PRECISION array, dimension (N)
  If FACT = 'F', then DF is an input argument and on entry
  contains the n diagonal elements of the upper triangular
  matrix U from the LU factorization of A.
```

```
  If FACT = 'N', then DF is an output argument and on exit
  contains the n diagonal elements of the upper triangular
  matrix U from the LU factorization of A.

DUF
  DUF is DOUBLE PRECISION array, dimension (N-1)
  If FACT = 'F', then DUF is an input argument and on entry
  contains the (n-1) elements of the first superdiagonal of U.

  If FACT = 'N', then DUF is an output argument and on exit
  contains the (n-1) elements of the first superdiagonal of U.

DU2
  DU2 is DOUBLE PRECISION array, dimension (N-2)
  If FACT = 'F', then DU2 is an input argument and on entry
  contains the (n-2) elements of the second superdiagonal of U.

  If FACT = 'N', then DU2 is an output argument and on exit
  contains the (n-2) elements of the second superdiagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains the pivot indices from the LU factorization of A as
  computed by DGTTRF.

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the LU factorization of A;
  row i of the matrix was interchanged with row IPIV(i).
  IPIV(i) will always be either i or i+1; IPIV(i) = i indicates
  a row interchange was not required.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  The N-by-NRHS right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
```

```
  The estimate of the reciprocal condition number of the matrix
  A.  If RCOND is less than the machine precision (in
  particular, if RCOND = 0), the matrix is singular to working
  precision.  This condition is indicated by a return code of INFO > 0.

FERR
  FERR is DOUBLE PRECISION array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is DOUBLE PRECISION array, dimension (3*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, and i is
  <= N:  U(i,i) is exactly zero.  The factorization
     has not been completed unless i = N, but the
     factor U is exactly singular, so the solution
     and error bounds could not be computed.
     RCOND = 0 is returned.
  = N+1: U is nonsingular, but RCOND is less than machine
     precision, meaning that the matrix is singular
     to working precision.  Nevertheless, the
     solution and error bounds are computed because
    there are a number of situations where the
    computed solution can be more accurate than the
    value of RCOND would suggest.
```

———-

74.2.3 fortran code

— dgtsvx.f —

```
*  =====================================================================
      SUBROUTINE DGTSVX( FACT, TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF,
     $                   DU2, IPIV, B, LDB, X, LDX, RCOND, FERR, BERR,
     $                   WORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          FACT, TRANS
      INTEGER            INFO, LDB, LDX, N, NRHS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      DOUBLE PRECISION   B( LDB, * ), BERR( * ), D( * ), DF( * ),
     $                   DL( * ), DLF( * ), DU( * ), DU2( * ), DUF( * ),
     $                   FERR( * ), WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOFACT, NOTRAN
      CHARACTER          NORM
      DOUBLE PRECISION   ANORM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, DLANGT
      EXTERNAL           LSAME, DLAMCH, DLANGT
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DGTCON, DGTRFS, DGTTRF, DGTTRS, DLACPY,
     $                   XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
```

```
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOFACT .AND. .NOT.LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $         LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -14
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -16
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGTSVX', -INFO )
         RETURN
      END IF
*
      IF( NOFACT ) THEN
*
*        Compute the LU factorization of A.
*
         CALL DCOPY( N, D, 1, DF, 1 )
         IF( N.GT.1 ) THEN
            CALL DCOPY( N-1, DL, 1, DLF, 1 )
            CALL DCOPY( N-1, DU, 1, DUF, 1 )
         END IF
         CALL DGTTRF( N, DLF, DF, DUF, DU2, IPIV, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 )THEN
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A.
*
      IF( NOTRAN ) THEN
         NORM = '1'
      ELSE
         NORM = 'I'
      END IF
```

```
      ANORM = DLANGT( NORM, N, DL, D, DU )
*
*     Compute the reciprocal of the condition number of A.
*
      CALL DGTCON( NORM, N, DLF, DF, DUF, DU2, IPIV, ANORM, RCOND, WORK,
     $             IWORK, INFO )
*
*     Compute the solution vectors X.
*
      CALL DLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL DGTTRS( TRANS, N, NRHS, DLF, DF, DUF, DU2, IPIV, X, LDX,
     $             INFO )
*
*     Use iterative refinement to improve the computed solutions and
*     compute error bounds and backward error estimates for them.
*
      CALL DGTRFS( TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF, DU2, IPIV,
     $             B, LDB, X, LDX, FERR, BERR, WORK, IWORK, INFO )
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.DLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      RETURN
*
*     End of DGTSVX
*
      END
```

———-

74.2.4 lisp code

— **dgtsvx.lisp** —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun dgtsvx
         (fact trans n nrhs dl d du dlf df duf du2 ipiv b ldb$ x ldx rcond ferr
          berr work iwork info)
    (declare (type (double-float) rcond)
             (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array double-float (*)) work berr ferr x b du2 duf df dlf
              du d dl)
             (type (f2cl-lib:integer4) info ldx ldb$ nrhs n)
             (type (string *) trans fact))
```

```
(f2cl-lib:with-multi-array-data
    ((fact character fact-%data% fact-%offset%)
     (trans character trans-%data% trans-%offset%)
     (dl double-float dl-%data% dl-%offset%)
     (d double-float d-%data% d-%offset%)
     (du double-float du-%data% du-%offset%)
     (dlf double-float dlf-%data% dlf-%offset%)
     (df double-float df-%data% df-%offset%)
     (duf double-float duf-%data% duf-%offset%)
     (du2 double-float du2-%data% du2-%offset%)
     (b double-float b-%data% b-%offset%)
     (x double-float x-%data% x-%offset%)
     (ferr double-float ferr-%data% ferr-%offset%)
     (berr double-float berr-%data% berr-%offset%)
     (work double-float work-%data% work-%offset%)
     (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
     (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((anorm 0.0d0)
         (norm
          (make-array '(1) :element-type 'character :initial-element #\ ))
         (nofact nil) (notran nil))
    (declare (type (double-float) anorm)
             (type (string 1) norm)
             (type f2cl-lib:logical nofact notran))
    (setf info 0)
    (setf nofact
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "N")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val))
    (setf notran
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "N")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val))
    (cond
     ((and (not nofact)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "F")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val)))
      (setf info -1))
     ((and (not notran)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "T")
```

```
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -14))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -16)))
(cond
 ((/= info 0) (xerbla "DGTSVX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (nofact
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy n d 1 df 1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf n var-0)))
  (cond
   ((> n 1) (dcopy (f2cl-lib:int-sub n 1) dl 1 dlf 1)
    (dcopy (f2cl-lib:int-sub n 1) du 1 duf 1)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dgttrf n dlf df duf du2 ipiv info)
    (declare (ignore var-1 var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))
    (when var-6 (setf info var-6)))
  (cond ((> info 0) (setf rcond zero) (go end_label)))))
(cond (notran (f2cl-lib:f2cl-set-string norm "1" (string 1)))
      (t (f2cl-lib:f2cl-set-string norm "I" (string 1))))
(setf anorm
        (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
            (dlangt norm n dl d du)
          (declare (ignore var-2 var-3 var-4))
          (when var-0 (setf norm var-0))
          (when var-1 (setf n var-1))
          ret-val))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11)
    (dgtcon norm n dlf df duf du2 ipiv anorm rcond work iwork info)
  (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-9 var-10))
  (setf norm var-0)
  (setf n var-1)
  (setf rcond var-8)
```

```
          (setf info var-11))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dlacpy "Full" n nrhs b ldb$ x ldx)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldb$ var-4))
          (when var-6 (setf ldx var-6)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10)
            (dgttrs trans n nrhs dlf df duf du2 ipiv x ldx info)
          (declare (ignore var-3 var-4 var-5 var-6 var-7 var-8))
          (when var-0 (setf trans var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-9 (setf ldx var-9))
          (when var-10 (setf info var-10)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
             var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19)
            (dgtrfs trans n nrhs dl d du dlf df duf du2 ipiv b ldb$ x ldx ferr
             berr work iwork info)
          (declare
           (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
            var-11 var-12 var-13 var-15 var-16 var-17 var-18))
          (setf trans var-0)
          (setf n var-1)
          (setf ldx var-14)
          (setf info var-19))
        (if (< rcond (dlamch "Epsilon"))
            (setf info (f2cl-lib:int-add n 1)))
        (go end_label)
       end_label
        (return
         (values fact trans n nrhs nil nil nil nil nil nil nil nil nil ldb$ nil
                 ldx rcond nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgtsvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
```

```
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (double-float)
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::fact
  fortran-to-lisp::trans
  fortran-to-lisp::n
  fortran-to-lisp::nrhs nil nil nil
  nil nil nil nil nil nil
  fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond nil nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::dgtrfs
  fortran-to-lisp::dgtcon))))
```

———-

Chapter 75

LAPACK - General Tridiagonal Matrix, Linear Solve, Real

75.1 sgtsv

75.1.1 Axiom unit tests

— sgtsv.input —

```
)set break resume
)sys rm -f sgtsv.output
)spool sgtsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

75.1.2 Axiom help page

— sgtsv.help —

```
==========================================================================
sgtsv examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgtsv (N, NRHS, DL, D, DU, B, LDB, INFO)
            SGTSV computes the solution to system of linear equations A * X =
           B for GT matrices

Function/Subroutine Documentation
   subroutine sgtsv (integer N, integer NRHS, real, dimension( * ) DL, real,
       dimension( * ) D, real, dimension( * ) DU, real, dimension( ldb, * ) B,
       integer LDB, integer INFO)
        SGTSV computes the solution to system of linear equations A * X = B
       for GT matrices

Purpose:
   SGTSV  solves the equation

   A*X = B,

   where A is an n by n tridiagonal matrix, by Gaussian elimination with
   partial pivoting.

   Note that the equation  A**T*X = B  may be solved by interchanging the
   order of the arguments DU and DL.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 DL
   DL is REAL array, dimension (N-1)
   On entry, DL must contain the (n-1) sub-diagonal elements of A.

   On exit, DL is overwritten by the (n-2) elements of the
   second super-diagonal of the upper triangular matrix U from
   the LU factorization of A, in DL(1), ..., DL(n-2).

 D
   D is REAL array, dimension (N)
```

```
  On entry, D must contain the diagonal elements of A.

  On exit, D is overwritten by the n diagonal elements of U.

DU
  DU is REAL array, dimension (N-1)
  On entry, DU must contain the (n-1) super-diagonal elements of A.

  On exit, DU is overwritten by the (n-1) elements of the first
  super-diagonal of U.

B
  B is REAL array, dimension (LDB,NRHS)
  On entry, the N by NRHS matrix of right hand side matrix B.
  On exit, if INFO = 0, the N by NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -i, the i-th argument had an illegal value
  > 0: if INFO = i, U(i,i) is exactly zero, and the solution
       has not been computed.  The factorization has not been
       completed unless i = N.
```

———-

75.1.3 fortran code

— **sgtsv.f** —

```
*  =====================================================================
      SUBROUTINE SGTSV( N, NRHS, DL, D, DU, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      REAL               B( LDB, * ), D( * ), DL( * ), DU( * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      REAL               FACT, TEMP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGTSV ', -INFO )
         RETURN
      END IF
*
      IF( N.EQ.0 )
     $   RETURN
*
      IF( NRHS.EQ.1 ) THEN
         DO 10 I = 1, N - 2
            IF( ABS( D( I ) ).GE.ABS( DL( I ) ) ) THEN
*
*              No row interchange required
*
               IF( D( I ).NE.ZERO ) THEN
                  FACT = DL( I ) / D( I )
                  D( I+1 ) = D( I+1 ) - FACT*DU( I )
                  B( I+1, 1 ) = B( I+1, 1 ) - FACT*B( I, 1 )
               ELSE
                  INFO = I
                  RETURN
               END IF
```

```
            DL( I ) = ZERO
         ELSE
*
*           Interchange rows I and I+1
*
            FACT = D( I ) / DL( I )
            D( I ) = DL( I )
            TEMP = D( I+1 )
            D( I+1 ) = DU( I ) - FACT*TEMP
            DL( I ) = DU( I+1 )
            DU( I+1 ) = -FACT*DL( I )
            DU( I ) = TEMP
            TEMP = B( I, 1 )
            B( I, 1 ) = B( I+1, 1 )
            B( I+1, 1 ) = TEMP - FACT*B( I+1, 1 )
         END IF
   10    CONTINUE
         IF( N.GT.1 ) THEN
            I = N - 1
            IF( ABS( D( I ) ).GE.ABS( DL( I ) ) ) THEN
               IF( D( I ).NE.ZERO ) THEN
                  FACT = DL( I ) / D( I )
                  D( I+1 ) = D( I+1 ) - FACT*DU( I )
                  B( I+1, 1 ) = B( I+1, 1 ) - FACT*B( I, 1 )
               ELSE
                  INFO = I
                  RETURN
               END IF
            ELSE
               FACT = D( I ) / DL( I )
               D( I ) = DL( I )
               TEMP = D( I+1 )
               D( I+1 ) = DU( I ) - FACT*TEMP
               DU( I ) = TEMP
               TEMP = B( I, 1 )
               B( I, 1 ) = B( I+1, 1 )
               B( I+1, 1 ) = TEMP - FACT*B( I+1, 1 )
            END IF
         END IF
         IF( D( N ).EQ.ZERO ) THEN
            INFO = N
            RETURN
         END IF
      ELSE
         DO 40 I = 1, N - 2
            IF( ABS( D( I ) ).GE.ABS( DL( I ) ) ) THEN
*
*           No row interchange required
*
               IF( D( I ).NE.ZERO ) THEN
```

```
                 FACT = DL( I ) / D( I )
                 D( I+1 ) = D( I+1 ) - FACT*DU( I )
                 DO 20 J = 1, NRHS
                    B( I+1, J ) = B( I+1, J ) - FACT*B( I, J )
   20            CONTINUE
              ELSE
                 INFO = I
                 RETURN
              END IF
              DL( I ) = ZERO
           ELSE
*
*             Interchange rows I and I+1
*
              FACT = D( I ) / DL( I )
              D( I ) = DL( I )
              TEMP = D( I+1 )
              D( I+1 ) = DU( I ) - FACT*TEMP
              DL( I ) = DU( I+1 )
              DU( I+1 ) = -FACT*DL( I )
              DU( I ) = TEMP
              DO 30 J = 1, NRHS
                 TEMP = B( I, J )
                 B( I, J ) = B( I+1, J )
                 B( I+1, J ) = TEMP - FACT*B( I+1, J )
   30         CONTINUE
           END IF
   40    CONTINUE
         IF( N.GT.1 ) THEN
           I = N - 1
           IF( ABS( D( I ) ).GE.ABS( DL( I ) ) ) THEN
              IF( D( I ).NE.ZERO ) THEN
                 FACT = DL( I ) / D( I )
                 D( I+1 ) = D( I+1 ) - FACT*DU( I )
                 DO 50 J = 1, NRHS
                    B( I+1, J ) = B( I+1, J ) - FACT*B( I, J )
   50            CONTINUE
              ELSE
                 INFO = I
                 RETURN
              END IF
           ELSE
              FACT = D( I ) / DL( I )
              D( I ) = DL( I )
              TEMP = D( I+1 )
              D( I+1 ) = DU( I ) - FACT*TEMP
              DU( I ) = TEMP
              DO 60 J = 1, NRHS
                 TEMP = B( I, J )
                 B( I, J ) = B( I+1, J )
```

```
                B( I+1, J ) = TEMP - FACT*B( I+1, J )
   60         CONTINUE
            END IF
         END IF
         IF( D( N ).EQ.ZERO ) THEN
            INFO = N
            RETURN
         END IF
      END IF
*
*     Back solve with the matrix U from the factorization.
*
      IF( NRHS.LE.2 ) THEN
         J = 1
   70    CONTINUE
         B( N, J ) = B( N, J ) / D( N )
         IF( N.GT.1 )
     $      B( N-1, J ) = ( B( N-1, J )-DU( N-1 )*B( N, J ) ) / D( N-1 )
         DO 80 I = N - 2, 1, -1
            B( I, J ) = ( B( I, J )-DU( I )*B( I+1, J )-DL( I )*
     $                  B( I+2, J ) ) / D( I )
   80    CONTINUE
         IF( J.LT.NRHS ) THEN
            J = J + 1
            GO TO 70
         END IF
      ELSE
         DO 100 J = 1, NRHS
            B( N, J ) = B( N, J ) / D( N )
            IF( N.GT.1 )
     $         B( N-1, J ) = ( B( N-1, J )-DU( N-1 )*B( N, J ) ) /
     $                       D( N-1 )
            DO 90 I = N - 2, 1, -1
               B( I, J ) = ( B( I, J )-DU( I )*B( I+1, J )-DL( I )*
     $                     B( I+2, J ) ) / D( I )
   90       CONTINUE
  100    CONTINUE
      END IF
*
      RETURN
*
*     End of SGTSV
*
      END
```

———-

75.1.4 lisp code

— sgtsv.lisp —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun sgtsv (n nrhs dl d du b ldb$ info)
    (declare (type (array single-float (*)) b du d dl)
             (type (f2cl-lib:integer4) info ldb$ nrhs n))
    (f2cl-lib:with-multi-array-data
        ((dl single-float dl-%data% dl-%offset%)
         (d single-float d-%data% d-%offset%)
         (du single-float du-%data% du-%offset%)
         (b single-float b-%data% b-%offset%))
      (prog ((fact 0.0) (temp 0.0) (i 0) (j 0))
        (declare (type (single-float) fact temp)
                 (type (f2cl-lib:integer4) i j))
        (setf info 0)
        (cond ((< n 0) (setf info -1)) ((< nrhs 0) (setf info -2))
              ((< ldb$
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -7)))
        (cond
         ((/= info 0) (xerbla "SGTSV " (f2cl-lib:int-sub info))
          (go end_label)))
        (if (= n 0)
            (go end_label))
        (cond
         ((= nrhs 1)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))) nil)
            (tagbody
              (cond
               ((>= (abs (f2cl-lib:fref d (i) ((1 *))))
                    (abs (f2cl-lib:fref dl (i) ((1 *)))))
                (cond
                 ((/= (f2cl-lib:fref d (i) ((1 *))) zero)
                  (setf fact
                          (/ (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                             (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
                  (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                        ((1 *)) d-%offset%)
                          (-
                           (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                          ((1 *)) d-%offset%)
                           (* fact
                              (f2cl-lib:fref du-%data% (i) ((1 *))
                                             du-%offset%))))
```

```
            (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
                    (-
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) 1)
                                     ((1 ldb$) (1 *)) b-%offset%)
                     (* fact
                        (f2cl-lib:fref b-%data% (i 1) ((1 ldb$) (1 *))
                                       b-%offset%)))))
           (t (setf info i) (go end_label)))
          (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) zero))
         (t
          (setf fact
                  (/ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                     (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
          (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                  (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
          (setf temp
                  (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                  ((1 *)) d-%offset%))
          (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                d-%offset%)
                  (- (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                     (* fact temp)))
          (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                  (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1))
                                  ((1 *)) du-%offset%))
          (setf (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                du-%offset%)
                  (* (- fact)
                     (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
          (setf (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%) temp)
          (setf temp
                  (f2cl-lib:fref b-%data% (i 1) ((1 ldb$) (1 *))
                                  b-%offset%))
          (setf (f2cl-lib:fref b-%data% (i 1) ((1 ldb$) (1 *))
                                b-%offset%)
                  (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) 1)
                                  ((1 ldb$) (1 *)) b-%offset%))
          (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) 1)
                                ((1 ldb$) (1 *)) b-%offset%)
                  (- temp
                     (* fact
                        (f2cl-lib:fref b-%data%
                                       ((f2cl-lib:int-add i 1) 1)
                                       ((1 ldb$) (1 *)) b-%offset%))))))
   label10))
(cond
 ((> n 1) (setf i (f2cl-lib:int-sub n 1))
  (cond
   ((>= (abs (f2cl-lib:fref d (i) ((1 *))))
```

```
            (abs (f2cl-lib:fref dl (i) ((1 *)))))
        (cond
         ((/= (f2cl-lib:fref d (i) ((1 *))) zero)
          (setf fact
                  (/ (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                     (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
          (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                d-%offset%)
                  (-
                   (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                  ((1 *)) d-%offset%)
                   (* fact
                      (f2cl-lib:fref du-%data% (i) ((1 *))
                                     du-%offset%))))
          (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) 1)
                                ((1 ldb$) (1 *)) b-%offset%)
                  (-
                   (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) 1)
                                  ((1 ldb$) (1 *)) b-%offset%)
                   (* fact
                      (f2cl-lib:fref b-%data% (i 1) ((1 ldb$) (1 *))
                                     b-%offset%)))))
         (t (setf info i) (go end_label))))
       (t
        (setf fact
                (/ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                   (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
        (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
        (setf temp
                (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                d-%offset%))
        (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                               d-%offset%)
                (- (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                   (* fact temp)))
        (setf (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%) temp)
        (setf temp
                (f2cl-lib:fref b-%data% (i 1) ((1 ldb$) (1 *))
                                b-%offset%))
        (setf (f2cl-lib:fref b-%data% (i 1) ((1 ldb$) (1 *)) b-%offset%)
                (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) 1)
                                ((1 ldb$) (1 *)) b-%offset%))
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) 1)
                               ((1 ldb$) (1 *)) b-%offset%)
                (- temp
                   (* fact
                      (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) 1)
                                     ((1 ldb$) (1 *)) b-%offset%))))))))
    (cond
```

```
      ((= (f2cl-lib:fref d (n) ((1 *))) zero) (setf info n)
       (go end_label))))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))) nil)
        (tagbody
          (cond
           ((>= (abs (f2cl-lib:fref d (i) ((1 *))))
                (abs (f2cl-lib:fref dl (i) ((1 *)))))
            (cond
             ((/= (f2cl-lib:fref d (i) ((1 *))) zero)
              (setf fact
                      (/ (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                         (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
              (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                    ((1 *)) d-%offset%)
                      (-
                       (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                       ((1 *)) d-%offset%)
                       (* fact
                          (f2cl-lib:fref du-%data% (i) ((1 *))
                                         du-%offset%))))
              (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                            ((> j nrhs) nil)
                (tagbody
                  (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                        ((1 ldb$) (1 *)) b-%offset%)
                          (-
                           (f2cl-lib:fref b-%data%
                                          ((f2cl-lib:int-add i 1) j)
                                          ((1 ldb$) (1 *)) b-%offset%)
                           (* fact
                              (f2cl-lib:fref b-%data% (i j)
                                             ((1 ldb$) (1 *))
                                             b-%offset%))))
                 label20)))
             (t (setf info i) (go end_label)))
            (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) zero))
           (t
            (setf fact
                    (/ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                       (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
            (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                    (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
            (setf temp
                    (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                    ((1 *)) d-%offset%))
            (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                 d-%offset%)
                    (- (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
```

```
                    (* fact temp)))
          (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                  (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) du-%offset%))
          (setf (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                               du-%offset%)
                  (* (- fact)
                     (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
          (setf (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%) temp)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j nrhs) nil)
            (tagbody
              (setf temp
                      (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                     b-%offset%))
              (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)
                      (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                     ((1 ldb$) (1 *)) b-%offset%))
              (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                    ((1 ldb$) (1 *)) b-%offset%)
                      (- temp
                         (* fact
                            (f2cl-lib:fref b-%data%
                                           ((f2cl-lib:int-add i 1) j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%))))
           label30))))
   label40))
(cond
 ((> n 1) (setf i (f2cl-lib:int-sub n 1))
  (cond
   ((>= (abs (f2cl-lib:fref d (i) ((1 *))))
        (abs (f2cl-lib:fref dl (i) ((1 *)))))
    (cond
     ((/= (f2cl-lib:fref d (i) ((1 *))) zero)
      (setf fact
              (/ (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                 (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
      (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                           d-%offset%)
              (-
               (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                              ((1 *)) d-%offset%)
               (* fact
                  (f2cl-lib:fref du-%data% (i) ((1 *))
                                 du-%offset%))))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nrhs) nil)
        (tagbody
```

```
                (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                      ((1 ldb$) (1 *)) b-%offset%)
                        (-
                         (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                        ((1 ldb$) (1 *)) b-%offset%)
                         (* fact
                            (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                           b-%offset%))))
             label50)))
          (t (setf info i) (go end_label))))
        (t
         (setf fact
                 (/ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                    (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
         (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                 (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
         (setf temp
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                d-%offset%))
         (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                               d-%offset%)
                 (- (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                    (* fact temp)))
         (setf (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%) temp)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j nrhs) nil)
           (tagbody
             (setf temp
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%))
             (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                    ((1 ldb$) (1 *)) b-%offset%))
             (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                  ((1 ldb$) (1 *)) b-%offset%)
                     (- temp
                        (* fact
                           (f2cl-lib:fref b-%data%
                                          ((f2cl-lib:int-add i 1) j)
                                          ((1 ldb$) (1 *)) b-%offset%))))
           label60))))))
  (cond
   ((= (f2cl-lib:fref d (n) ((1 *))) zero) (setf info n)
    (go end_label)))))
(cond
 ((<= nrhs 2)
  (tagbody
    (setf j 1)
   label70
```

```
   (setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
           (/
            (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
            (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
   (if (> n 1)
       (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)
                 (/
                  (-
                   (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                                   ((1 ldb$) (1 *)) b-%offset%)
                   (*
                    (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub n 1))
                                    ((1 *)) du-%offset%)
                    (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                                    b-%offset%)))
                  (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                  ((1 *)) d-%offset%))))
   (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))
                    (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                   ((> i 1) nil)
     (tagbody
       (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)
                 (/
                  (-
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)
                   (* (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                      (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                      ((1 ldb$) (1 *)) b-%offset%))
                   (* (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                      (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 2) j)
                                      ((1 ldb$) (1 *)) b-%offset%)))
                  (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
      label80))
   (cond ((< j nrhs) (setf j (f2cl-lib:int-add j 1)) (go label70)))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j nrhs) nil)
   (tagbody
     (setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
             (/
              (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                              b-%offset%)
              (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
     (if (> n 1)
         (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                               ((1 ldb$) (1 *)) b-%offset%)
                 (/
```

```
                          (-
                           (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                                          ((1 ldb$) (1 *)) b-%offset%)
                           (*
                            (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub n 1))
                                           ((1 *)) du-%offset%)
                            (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                                           b-%offset%)))
                          (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                         ((1 *)) d-%offset%))))
             (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))
                            (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                           ((> i 1) nil)
               (tagbody
                 (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                      b-%offset%)
                        (/
                         (-
                          (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                         b-%offset%)
                          (*
                           (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                           (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                          ((1 ldb$) (1 *)) b-%offset%))
                          (*
                           (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                           (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 2) j)
                                          ((1 ldb$) (1 *)) b-%offset%)))
                         (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
                 label90))
            label100))))
       (go end_label)
      end_label
       (return (values nil nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgtsv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil
```

```
                                        fortran-to-lisp::info)
                                      :calls 'nil)))
```

———-

75.2 sgtsvx

75.2.1 Axiom unit tests

— **sgtsvx.input** —

```
)set break resume
)sys rm -f sgtsvx.output
)spool sgtsvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

75.2.2 Axiom help page

— **sgtsvx.help** —

```
======================================================================
sgtsvx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgtsvx (FACT, TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF, DU2,
           IPIV, B, LDB, X, LDX, RCOND, FERR, BERR, WORK, IWORK, INFO)
            SGTSVX computes the solution to system of linear equations A * X =
           B for GT matrices
```

```
Function/Subroutine Documentation
   subroutine sgtsvx (character FACT, character TRANS, integer N, integer
       NRHS, real, dimension( * ) DL, real, dimension( * ) D, real, dimension(
       * ) DU, real, dimension( * ) DLF, real, dimension( * ) DF, real,
       dimension( * ) DUF, real, dimension( * ) DU2, integer, dimension( * )
       IPIV, real, dimension( ldb, * ) B, integer LDB, real, dimension( ldx, *
       ) X, integer LDX, real RCOND, real, dimension( * ) FERR, real,
       dimension( * ) BERR, real, dimension( * ) WORK, integer, dimension( * )
       IWORK, integer INFO)
        SGTSVX computes the solution to system of linear equations A * X = B
       for GT matrices

Purpose:
   SGTSVX uses the LU factorization to compute the solution to a real
   system of linear equations A * X = B or A**T * X = B,
   where A is a tridiagonal matrix of order N and X and B are N-by-NRHS
   matrices.

   Error bounds on the solution and a condition estimate are also
   provided.

Description:

   The following steps are performed:

   1. If FACT = 'N', the LU decomposition is used to factor the matrix A
      as A = L * U, where L is a product of permutation and unit lower
      bidiagonal matrices and U is upper triangular with nonzeros in
      only the main diagonal and first two superdiagonals.

   2. If some U(i,i)=0, so that U is exactly singular, then the routine
      returns with INFO = i. Otherwise, the factored form of A is used
      to estimate the condition number of the matrix A.  If the
      reciprocal of the condition number is less than machine precision,
      INFO = N+1 is returned as a warning, but the routine still goes on
      to solve for X and compute error bounds as described below.

   3. The system of equations is solved for X using the factored form of A.

   4. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of A has been
   supplied on entry.
   = 'F':  DLF, DF, DUF, DU2, and IPIV contain the factored
      form of A; DL, D, DU, DLF, DF, DUF, DU2 and IPIV
```

```
     will not be modified.
  = 'N':  The matrix will be copied to DLF, DF, and DUF
     and factored.

TRANS
  TRANS is CHARACTER*1
  Specifies the form of the system of equations:
  = 'N':  A * X = B     (No transpose)
  = 'T':  A**T * X = B  (Transpose)
  = 'C':  A**H * X = B  (Conjugate transpose = Transpose)

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrix B.  NRHS >= 0.

DL
  DL is REAL array, dimension (N-1)
  The (n-1) subdiagonal elements of A.

D
  D is REAL array, dimension (N)
  The n diagonal elements of A.

DU
  DU is REAL array, dimension (N-1)
  The (n-1) superdiagonal elements of A.

DLF
  DLF is REAL array, dimension (N-1)
  If FACT = 'F', then DLF is an input argument and on entry
  contains the (n-1) multipliers that define the matrix L from
  the LU factorization of A as computed by SGTTRF.

  If FACT = 'N', then DLF is an output argument and on exit
  contains the (n-1) multipliers that define the matrix L from
  the LU factorization of A.

DF
  DF is REAL array, dimension (N)
  If FACT = 'F', then DF is an input argument and on entry
  contains the n diagonal elements of the upper triangular
  matrix U from the LU factorization of A.

  If FACT = 'N', then DF is an output argument and on exit
  contains the n diagonal elements of the upper triangular
```

```
  matrix U from the LU factorization of A.

DUF
  DUF is REAL array, dimension (N-1)
  If FACT = 'F', then DUF is an input argument and on entry
  contains the (n-1) elements of the first superdiagonal of U.

  If FACT = 'N', then DUF is an output argument and on exit
  contains the (n-1) elements of the first superdiagonal of U.

DU2
  DU2 is REAL array, dimension (N-2)
  If FACT = 'F', then DU2 is an input argument and on entry
  contains the (n-2) elements of the second superdiagonal of U.

  If FACT = 'N', then DU2 is an output argument and on exit
  contains the (n-2) elements of the second superdiagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains the pivot indices from the LU factorization of A as
  computed by SGTTRF.

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the LU factorization of A;
  row i of the matrix was interchanged with row IPIV(i).
  IPIV(i) will always be either i or i+1; IPIV(i) = i indicates
  a row interchange was not required.

B
  B is REAL array, dimension (LDB,NRHS)
  The N-by-NRHS right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is REAL array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  The estimate of the reciprocal condition number of the matrix
  A.  If RCOND is less than the machine precision (in
```

```
  particular, if RCOND = 0), the matrix is singular to working
  precision.  This condition is indicated by a return code of INFO > 0.

FERR
  FERR is REAL array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is REAL array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is REAL array, dimension (3*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, and i is
  <= N:  U(i,i) is exactly zero.  The factorization
     has not been completed unless i = N, but the
     factor U is exactly singular, so the solution
     and error bounds could not be computed.
     RCOND = 0 is returned.
  = N+1: U is nonsingular, but RCOND is less than machine
     precision, meaning that the matrix is singular
     to working precision.  Nevertheless, the
     solution and error bounds are computed because
     there are a number of situations where the
     computed solution can be more accurate than the
     value of RCOND would suggest.
```

———-

75.2.3 fortran code

— sgtsvx.f —

```
*  =====================================================================
      SUBROUTINE SGTSVX( FACT, TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF,
     $                   DU2, IPIV, B, LDB, X, LDX, RCOND, FERR, BERR,
     $                   WORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          FACT, TRANS
      INTEGER            INFO, LDB, LDX, N, NRHS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      REAL               B( LDB, * ), BERR( * ), D( * ), DF( * ),
     $                   DL( * ), DLF( * ), DU( * ), DU2( * ), DUF( * ),
     $                   FERR( * ), WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOFACT, NOTRAN
      CHARACTER          NORM
      REAL               ANORM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH, SLANGT
      EXTERNAL           LSAME, SLAMCH, SLANGT
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SGTCON, SGTRFS, SGTTRF, SGTTRS, SLACPY,
     $                   XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
```

```
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOFACT .AND. .NOT.LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $         LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -14
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -16
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGTSVX', -INFO )
         RETURN
      END IF
*
      IF( NOFACT ) THEN
*
*        Compute the LU factorization of A.
*
         CALL SCOPY( N, D, 1, DF, 1 )
         IF( N.GT.1 ) THEN
            CALL SCOPY( N-1, DL, 1, DLF, 1 )
            CALL SCOPY( N-1, DU, 1, DUF, 1 )
         END IF
         CALL SGTTRF( N, DLF, DF, DUF, DU2, IPIV, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 )THEN
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A.
*
      IF( NOTRAN ) THEN
         NORM = '1'
      ELSE
         NORM = 'I'
      END IF
```

```
      ANORM = SLANGT( NORM, N, DL, D, DU )
*
*     Compute the reciprocal of the condition number of A.
*
      CALL SGTCON( NORM, N, DLF, DF, DUF, DU2, IPIV, ANORM, RCOND, WORK,
     $             IWORK, INFO )
*
*     Compute the solution vectors X.
*
      CALL SLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL SGTTRS( TRANS, N, NRHS, DLF, DF, DUF, DU2, IPIV, X, LDX,
     $             INFO )
*
*     Use iterative refinement to improve the computed solutions and
*     compute error bounds and backward error estimates for them.
*
      CALL SGTRFS( TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF, DU2, IPIV,
     $             B, LDB, X, LDX, FERR, BERR, WORK, IWORK, INFO )
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.SLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      RETURN
*
*     End of SGTSVX
*
      END
```

———-

75.2.4 lisp code

— **sgtsvx.lisp** —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun sgtsvx
         (fact trans n nrhs dl d du dlf df duf du2 ipiv b ldb$ x ldx rcond ferr
          berr work iwork info)
    (declare (type (single-float) rcond)
             (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array single-float (*)) work berr ferr x b du2 duf df dlf
              du d dl)
             (type (f2cl-lib:integer4) info ldx ldb$ nrhs n)
             (type (string *) trans fact))
```

```
(f2cl-lib:with-multi-array-data
    ((fact character fact-%data% fact-%offset%)
     (trans character trans-%data% trans-%offset%)
     (dl single-float dl-%data% dl-%offset%)
     (d single-float d-%data% d-%offset%)
     (du single-float du-%data% du-%offset%)
     (dlf single-float dlf-%data% dlf-%offset%)
     (df single-float df-%data% df-%offset%)
     (duf single-float duf-%data% duf-%offset%)
     (du2 single-float du2-%data% du2-%offset%)
     (b single-float b-%data% b-%offset%)
     (x single-float x-%data% x-%offset%)
     (ferr single-float ferr-%data% ferr-%offset%)
     (berr single-float berr-%data% berr-%offset%)
     (work single-float work-%data% work-%offset%)
     (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
     (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((anorm 0.0)
         (norm
          (make-array '(1) :element-type 'character :initial-element #\ ))
         (nofact nil) (notran nil))
    (declare (type (single-float) anorm)
             (type (string 1) norm)
             (type f2cl-lib:logical nofact notran))
    (setf info 0)
    (setf nofact
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "N")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val))
    (setf notran
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "N")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val))
    (cond
     ((and (not nofact)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "F")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val)))
      (setf info -1))
     ((and (not notran)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "T")
```

```
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -14))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -16)))
(cond
 ((/= info 0) (xerbla "SGTSVX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (nofact
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (scopy n d 1 df 1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf n var-0)))
  (cond
   ((> n 1) (scopy (f2cl-lib:int-sub n 1) dl 1 dlf 1)
    (scopy (f2cl-lib:int-sub n 1) du 1 duf 1)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (sgttrf n dlf df duf du2 ipiv info)
    (declare (ignore var-1 var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))
    (when var-6 (setf info var-6)))
  (cond ((> info 0) (setf rcond zero) (go end_label)))))
(cond (notran (f2cl-lib:f2cl-set-string norm "1" (string 1)))
      (t (f2cl-lib:f2cl-set-string norm "I" (string 1))))
(setf anorm
        (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
            (slangt norm n dl d du)
          (declare (ignore var-2 var-3 var-4))
          (when var-0 (setf norm var-0))
          (when var-1 (setf n var-1))
          ret-val))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11)
    (sgtcon norm n dlf df duf du2 ipiv anorm rcond work iwork info)
  (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-9 var-10))
  (setf norm var-0)
  (setf n var-1)
  (setf rcond var-8)
```

```
          (setf info var-11))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (slacpy "Full" n nrhs b ldb$ x ldx)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldb$ var-4))
          (when var-6 (setf ldx var-6)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10)
            (sgttrs trans n nrhs dlf df duf du2 ipiv x ldx info)
          (declare (ignore var-3 var-4 var-5 var-6 var-7 var-8))
          (when var-0 (setf trans var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-9 (setf ldx var-9))
          (when var-10 (setf info var-10)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
             var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19)
            (sgtrfs trans n nrhs dl d du dlf df duf du2 ipiv b ldb$ x ldx ferr
             berr work iwork info)
          (declare
           (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
            var-11 var-12 var-13 var-15 var-16 var-17 var-18))
          (setf trans var-0)
          (setf n var-1)
          (setf ldx var-14)
          (setf info var-19))
        (if (< rcond (slamch "Epsilon"))
            (setf info (f2cl-lib:int-add n 1)))
        (go end_label)
       end_label
        (return
         (values fact trans n nrhs nil nil nil nil nil nil nil nil nil ldb$ nil
                 ldx rcond nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgtsvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
```

```
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (single-float)
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::fact
  fortran-to-lisp::trans
  fortran-to-lisp::n
  fortran-to-lisp::nrhs nil nil nil
  nil nil nil nil nil nil
  fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond nil nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::sgtrfs
  fortran-to-lisp::sgtcon))))
```

———-

Chapter 76

LAPACK - General Tridiagonal Matrix, Linear Solve, Complex

76.1 cgtsv

76.1.1 Axiom unit tests

— cgtsv.input —

```
)set break resume
)sys rm -f cgtsv.output
)spool cgtsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

76.1.2 Axiom help page

— cgtsv.help —

```
==========================================================================
cgtsv examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgtsv (N, NRHS, DL, D, DU, B, LDB, INFO)
            CGTSV computes the solution to system of linear equations A * X =
           B for GT matrices

Function/Subroutine Documentation
   subroutine cgtsv (integer N, integer NRHS, complex, dimension( * ) DL,
       complex, dimension( * ) D, complex, dimension( * ) DU, complex,
       dimension( ldb, * ) B, integer LDB, integer INFO)
        CGTSV computes the solution to system of linear equations A * X = B
       for GT matrices

Purpose:
   CGTSV  solves the equation

     A*X = B,

   where A is an N-by-N tridiagonal matrix, by Gaussian elimination with
   partial pivoting.

   Note that the equation  A**T *X = B  may be solved by interchanging the
   order of the arguments DU and DL.

 Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 DL
   DL is COMPLEX array, dimension (N-1)
   On entry, DL must contain the (n-1) subdiagonal elements of A.
   On exit, DL is overwritten by the (n-2) elements of the
   second superdiagonal of the upper triangular matrix U from
   the LU factorization of A, in DL(1), ..., DL(n-2).

 D
   D is COMPLEX array, dimension (N)
   On entry, D must contain the diagonal elements of A.
```

```
  On exit, D is overwritten by the n diagonal elements of U.

DU
  DU is COMPLEX array, dimension (N-1)
  On entry, DU must contain the (n-1) superdiagonal elements of A.
  On exit, DU is overwritten by the (n-1) elements of the first
  superdiagonal of U.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, U(i,i) is exactly zero, and the solution
        has not been computed.  The factorization has not been
        completed unless i = N.
```

———-

76.1.3 fortran code

— cgtsv.f —

```
*  =====================================================================
      SUBROUTINE CGTSV( N, NRHS, DL, D, DU, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      COMPLEX            B( LDB, * ), D( * ), DL( * ), DU( * )
*     ..
*
*  =====================================================================
```

```
*
*     .. Parameters ..
      COMPLEX            ZERO
      PARAMETER          ( ZERO = ( 0.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            J, K
      COMPLEX            MULT, TEMP, ZDUM
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, MAX, REAL
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGTSV ', -INFO )
         RETURN
      END IF
*
      IF( N.EQ.0 )
     $   RETURN
*
      DO 30 K = 1, N - 1
         IF( DL( K ).EQ.ZERO ) THEN
*
*           Subdiagonal is zero, no elimination is required.
*
            IF( D( K ).EQ.ZERO ) THEN
*
*              Diagonal is zero: set INFO = K and return; a unique
*              solution can not be found.
*
               INFO = K
```

```
               RETURN
            END IF
         ELSE IF( CABS1( D( K ) ).GE.CABS1( DL( K ) ) ) THEN
*
*           No row interchange required
*
            MULT = DL( K ) / D( K )
            D( K+1 ) = D( K+1 ) - MULT*DU( K )
            DO 10 J = 1, NRHS
               B( K+1, J ) = B( K+1, J ) - MULT*B( K, J )
   10       CONTINUE
            IF( K.LT.( N-1 ) )
     $         DL( K ) = ZERO
         ELSE
*
*           Interchange rows K and K+1
*
            MULT = D( K ) / DL( K )
            D( K ) = DL( K )
            TEMP = D( K+1 )
            D( K+1 ) = DU( K ) - MULT*TEMP
            IF( K.LT.( N-1 ) ) THEN
               DL( K ) = DU( K+1 )
               DU( K+1 ) = -MULT*DL( K )
            END IF
            DU( K ) = TEMP
            DO 20 J = 1, NRHS
               TEMP = B( K, J )
               B( K, J ) = B( K+1, J )
               B( K+1, J ) = TEMP - MULT*B( K+1, J )
   20       CONTINUE
         END IF
   30 CONTINUE
      IF( D( N ).EQ.ZERO ) THEN
         INFO = N
         RETURN
      END IF
*
*     Back solve with the matrix U from the factorization.
*
      DO 50 J = 1, NRHS
         B( N, J ) = B( N, J ) / D( N )
         IF( N.GT.1 )
     $      B( N-1, J ) = ( B( N-1, J )-DU( N-1 )*B( N, J ) ) / D( N-1 )
         DO 40 K = N - 2, 1, -1
            B( K, J ) = ( B( K, J )-DU( K )*B( K+1, J )-DL( K )*
     $                  B( K+2, J ) ) / D( K )
   40    CONTINUE
   50 CONTINUE
*
```

```
      RETURN
*
*     End of CGTSV
*
      END
```

———-

76.1.4 lisp code

— **cgtsv.lisp** —

```
(let* ((zero (f2cl-lib:cmplx 0.0 0.0)))
  (declare (type (f2cl-lib:complex8) zero)
           (ignorable zero))
  (defun cgtsv (n nrhs dl d du b ldb$ info)
    (declare (type (array f2cl-lib:complex8 (*)) b du d dl)
             (type (f2cl-lib:integer4) info ldb$ nrhs n))
    (f2cl-lib:with-multi-array-data
        ((dl f2cl-lib:complex8 dl-%data% dl-%offset%)
         (d f2cl-lib:complex8 d-%data% d-%offset%)
         (du f2cl-lib:complex8 du-%data% du-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
          cabs1))
        (prog ((mult #C(0.0 0.0)) (temp #C(0.0 0.0)) (zdum #C(0.0 0.0)) (j 0)
               (k 0))
          (declare (type (f2cl-lib:complex8) mult temp zdum)
                   (type (f2cl-lib:integer4) j k))
          (setf info 0)
          (cond ((< n 0) (setf info -1)) ((< nrhs 0) (setf info -2))
                ((< ldb$
                    (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
                 (setf info -7)))
          (cond
           ((/= info 0) (xerbla "CGTSV " (f2cl-lib:int-sub info))
            (go end_label)))
          (if (= n 0)
              (go end_label))
          (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                        ((> k (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
            (tagbody
              (cond
               ((= (f2cl-lib:fref dl (k) ((1 *))) zero)
                (cond
```

```
  ((= (f2cl-lib:fref d (k) ((1 *))) zero) (setf info k)
   (go end_label))))
((>= (cabs1 (f2cl-lib:fref d (k) ((1 *))))
     (cabs1 (f2cl-lib:fref dl (k) ((1 *)))))
 (setf mult
         (/ (f2cl-lib:fref dl-%data% (k) ((1 *)) dl-%offset%)
            (f2cl-lib:fref d-%data% (k) ((1 *)) d-%offset%)))
 (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add k 1)) ((1 *))
                       d-%offset%)
         (-
          (f2cl-lib:fref d-%data% ((f2cl-lib:int-add k 1))
                          ((1 *)) d-%offset%)
          (* mult
             (f2cl-lib:fref du-%data% (k) ((1 *))
                             du-%offset%))))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j nrhs) nil)
   (tagbody
     (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                           ((1 ldb$) (1 *)) b-%offset%)
             (-
              (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                              ((1 ldb$) (1 *)) b-%offset%)
              (* mult
                 (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                 b-%offset%))))
    label10))
 (if (< k (f2cl-lib:int-sub n 1))
     (setf (f2cl-lib:fref dl-%data% (k) ((1 *)) dl-%offset%)
             zero)))
(t
 (setf mult
         (/ (f2cl-lib:fref d-%data% (k) ((1 *)) d-%offset%)
            (f2cl-lib:fref dl-%data% (k) ((1 *)) dl-%offset%)))
 (setf (f2cl-lib:fref d-%data% (k) ((1 *)) d-%offset%)
         (f2cl-lib:fref dl-%data% (k) ((1 *)) dl-%offset%))
 (setf temp
         (f2cl-lib:fref d-%data% ((f2cl-lib:int-add k 1))
                         ((1 *)) d-%offset%))
 (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add k 1)) ((1 *))
                       d-%offset%)
         (- (f2cl-lib:fref du-%data% (k) ((1 *)) du-%offset%)
            (* mult temp)))
 (cond
  ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
   (setf (f2cl-lib:fref dl-%data% (k) ((1 *)) dl-%offset%)
           (f2cl-lib:fref du-%data% ((f2cl-lib:int-add k 1))
                           ((1 *)) du-%offset%))
   (setf (f2cl-lib:fref du-%data% ((f2cl-lib:int-add k 1))
                         ((1 *)) du-%offset%)
```

```
                    (* (- mult)
                       (f2cl-lib:fref dl-%data% (k) ((1 *))
                                      dl-%offset%)))))
          (setf (f2cl-lib:fref du-%data% (k) ((1 *)) du-%offset%) temp)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j nrhs) nil)
            (tagbody
              (setf temp
                      (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                     b-%offset%))
              (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                    b-%offset%)
                      (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                                      ((1 ldb$) (1 *)) b-%offset%))
              (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                                    ((1 ldb$) (1 *)) b-%offset%)
                      (- temp
                         (* mult
                            (f2cl-lib:fref b-%data%
                                           ((f2cl-lib:int-add k 1) j)
                                           ((1 ldb$) (1 *))
                                           b-%offset%))))
           label20))))
     label30))
  (cond
   ((= (f2cl-lib:fref d (n) ((1 *))) zero) (setf info n)
    (go end_label)))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
              (/
               (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                              b-%offset%)
               (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
      (if (> n 1)
          (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                                ((1 ldb$) (1 *)) b-%offset%)
                  (/
                   (-
                    (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                                    ((1 ldb$) (1 *)) b-%offset%)
                    (*
                     (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub n 1))
                                     ((1 *)) du-%offset%)
                     (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                                     b-%offset%)))
                   (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                  ((1 *)) d-%offset%))))
      (f2cl-lib:fdo (k (f2cl-lib:int-add n (f2cl-lib:int-sub 2))
```

```
                          (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                         ((> k 1) nil)
                (tagbody
                  (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                       b-%offset%)
                        (/
                         (-
                          (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                         b-%offset%)
                          (*
                           (f2cl-lib:fref du-%data% (k) ((1 *)) du-%offset%)
                           (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                                          ((1 ldb$) (1 *)) b-%offset%))
                          (*
                           (f2cl-lib:fref dl-%data% (k) ((1 *)) dl-%offset%)
                           (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 2) j)
                                          ((1 ldb$) (1 *)) b-%offset%)))
                         (f2cl-lib:fref d-%data% (k) ((1 *)) d-%offset%)))
                 label40))
              label50))
          (go end_label)
         end_label
          (return (values nil nil nil nil nil nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgtsv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

76.2 cgtsvx

76.2.1 Axiom unit tests

— **cgtsvx.input** —

```
)set break resume
)sys rm -f cgtsvx.output
)spool cgtsvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

76.2.2 Axiom help page

— **cgtsvx.help** —

```
======================================================================
cgtsvx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgtsvx (FACT, TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF, DU2,
           IPIV, B, LDB, X, LDX, RCOND, FERR, BERR, WORK, RWORK, INFO)
            CGTSVX computes the solution to system of linear equations A * X =
           B for GT matrices

Function/Subroutine Documentation
   subroutine cgtsvx (character FACT, character TRANS, integer N, integer
       NRHS, complex, dimension( * ) DL, complex, dimension( * ) D, complex,
       dimension( * ) DU, complex, dimension( * ) DLF, complex, dimension( * )
       DF, complex, dimension( * ) DUF, complex, dimension( * ) DU2, integer,
       dimension( * ) IPIV, complex, dimension( ldb, * ) B, integer LDB,
       complex, dimension( ldx, * ) X, integer LDX, real RCOND, real,
       dimension( * ) FERR, real, dimension( * ) BERR, complex, dimension( * )
       WORK, real, dimension( * ) RWORK, integer INFO)
```

```
       CGTSVX computes the solution to system of linear equations A * X = B
      for GT matrices

Purpose:
   CGTSVX uses the LU factorization to compute the solution to a complex
   system of linear equations A * X = B, A**T * X = B, or A**H * X = B,
   where A is a tridiagonal matrix of order N and X and B are N-by-NRHS
   matrices.

   Error bounds on the solution and a condition estimate are also
   provided.

Description:
   The following steps are performed:

   1. If FACT = 'N', the LU decomposition is used to factor the matrix A
      as A = L * U, where L is a product of permutation and unit lower
      bidiagonal matrices and U is upper triangular with nonzeros in
      only the main diagonal and first two superdiagonals.

   2. If some U(i,i)=0, so that U is exactly singular, then the routine
      returns with INFO = i. Otherwise, the factored form of A is used
      to estimate the condition number of the matrix A.  If the
      reciprocal of the condition number is less than machine precision,
      INFO = N+1 is returned as a warning, but the routine still goes on
      to solve for X and compute error bounds as described below.

   3. The system of equations is solved for X using the factored form of A.

   4. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of A has been
   supplied on entry.
   = 'F':  DLF, DF, DUF, DU2, and IPIV contain the factored form
           of A; DL, D, DU, DLF, DF, DUF, DU2 and IPIV will not
           be modified.
   = 'N':  The matrix will be copied to DLF, DF, and DUF and factored.

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose)
```

```
N
  N is INTEGER
  The order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrix B.  NRHS >= 0.

DL
  DL is COMPLEX array, dimension (N-1)
  The (n-1) subdiagonal elements of A.

D
  D is COMPLEX array, dimension (N)
  The n diagonal elements of A.

DU
  DU is COMPLEX array, dimension (N-1)
  The (n-1) superdiagonal elements of A.

DLF
  DLF is COMPLEX array, dimension (N-1)
  If FACT = 'F', then DLF is an input argument and on entry
  contains the (n-1) multipliers that define the matrix L from
  the LU factorization of A as computed by CGTTRF.

  If FACT = 'N', then DLF is an output argument and on exit
  contains the (n-1) multipliers that define the matrix L from
  the LU factorization of A.

DF
  DF is COMPLEX array, dimension (N)
  If FACT = 'F', then DF is an input argument and on entry
  contains the n diagonal elements of the upper triangular
  matrix U from the LU factorization of A.

  If FACT = 'N', then DF is an output argument and on exit
  contains the n diagonal elements of the upper triangular
  matrix U from the LU factorization of A.

DUF
  DUF is COMPLEX array, dimension (N-1)
  If FACT = 'F', then DUF is an input argument and on entry
  contains the (n-1) elements of the first superdiagonal of U.

  If FACT = 'N', then DUF is an output argument and on exit
  contains the (n-1) elements of the first superdiagonal of U.

DU2
```

```
  DU2 is COMPLEX array, dimension (N-2)
  If FACT = 'F', then DU2 is an input argument and on entry
  contains the (n-2) elements of the second superdiagonal of U.

  If FACT = 'N', then DU2 is an output argument and on exit
  contains the (n-2) elements of the second superdiagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains the pivot indices from the LU factorization of A as
  computed by CGTTRF.

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the LU factorization of A;
  row i of the matrix was interchanged with row IPIV(i).
  IPIV(i) will always be either i or i+1; IPIV(i) = i indicates
  a row interchange was not required.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  The N-by-NRHS right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  The estimate of the reciprocal condition number of the matrix
  A.  If RCOND is less than the machine precision (in
  particular, if RCOND = 0), the matrix is singular to working
  precision.  This condition is indicated by a return code of INFO > 0.

FERR
  FERR is REAL array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
```

```
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is REAL array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is COMPLEX array, dimension (2*N)

RWORK
  RWORK is REAL array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, and i is
     <= N:  U(i,i) is exactly zero.  The factorization
            has not been completed unless i = N, but the
            factor U is exactly singular, so the solution
            and error bounds could not be computed.
            RCOND = 0 is returned.
     = N+1: U is nonsingular, but RCOND is less than machine
            precision, meaning that the matrix is singular
            to working precision.  Nevertheless, the
            solution and error bounds are computed because
            there are a number of situations where the
            computed solution can be more accurate than the
            value of RCOND would suggest.
```

———-

76.2.3 fortran code

— cgtsvx.f —

```
*  =====================================================================
      SUBROUTINE CGTSVX( FACT, TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF,
     $                   DU2, IPIV, B, LDB, X, LDX, RCOND, FERR, BERR,
     $                   WORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
```

```
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          FACT, TRANS
      INTEGER            INFO, LDB, LDX, N, NRHS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               BERR( * ), FERR( * ), RWORK( * )
      COMPLEX            B( LDB, * ), D( * ), DF( * ), DL( * ),
     $                   DLF( * ), DU( * ), DU2( * ), DUF( * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOFACT, NOTRAN
      CHARACTER          NORM
      REAL               ANORM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               CLANGT, SLAMCH
      EXTERNAL           LSAME, CLANGT, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           CCOPY, CGTCON, CGTRFS, CGTTRF, CGTTRS, CLACPY,
     $                   XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOFACT .AND. .NOT.LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $         LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
```

```
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -14
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -16
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGTSVX', -INFO )
         RETURN
      END IF
*
      IF( NOFACT ) THEN
*
*        Compute the LU factorization of A.
*
         CALL CCOPY( N, D, 1, DF, 1 )
         IF( N.GT.1 ) THEN
            CALL CCOPY( N-1, DL, 1, DLF, 1 )
            CALL CCOPY( N-1, DU, 1, DUF, 1 )
         END IF
         CALL CGTTRF( N, DLF, DF, DUF, DU2, IPIV, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 )THEN
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A.
*
      IF( NOTRAN ) THEN
         NORM = '1'
      ELSE
         NORM = 'I'
      END IF
      ANORM = CLANGT( NORM, N, DL, D, DU )
*
*     Compute the reciprocal of the condition number of A.
*
      CALL CGTCON( NORM, N, DLF, DF, DUF, DU2, IPIV, ANORM, RCOND, WORK,
     $             INFO )
*
*     Compute the solution vectors X.
*
      CALL CLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL CGTTRS( TRANS, N, NRHS, DLF, DF, DUF, DU2, IPIV, X, LDX,
     $             INFO )
```

```
*
*     Use iterative refinement to improve the computed solutions and
*     compute error bounds and backward error estimates for them.
*
      CALL CGTRFS( TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF, DU2, IPIV,
     $             B, LDB, X, LDX, FERR, BERR, WORK, RWORK, INFO )
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.SLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      RETURN
*
*     End of CGTSVX
*
      END
```

76.2.4 lisp code

— cgtsvx.lisp —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun cgtsvx
         (fact trans n nrhs dl d du dlf df duf du2 ipiv b ldb$ x ldx rcond ferr
          berr work rwork info)
    (declare (type (array single-float (*)) rwork berr ferr)
             (type (single-float) rcond)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) work x b du2 duf df dlf du d
              dl)
             (type (f2cl-lib:integer4) info ldx ldb$ nrhs n)
             (type (string *) trans fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (trans character trans-%data% trans-%offset%)
         (dl f2cl-lib:complex8 dl-%data% dl-%offset%)
         (d f2cl-lib:complex8 d-%data% d-%offset%)
         (du f2cl-lib:complex8 du-%data% du-%offset%)
         (dlf f2cl-lib:complex8 dlf-%data% dlf-%offset%)
         (df f2cl-lib:complex8 df-%data% df-%offset%)
         (duf f2cl-lib:complex8 duf-%data% duf-%offset%)
         (du2 f2cl-lib:complex8 du2-%data% du2-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
```

```
    (x f2cl-lib:complex8 x-%data% x-%offset%)
    (work f2cl-lib:complex8 work-%data% work-%offset%)
    (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
    (ferr single-float ferr-%data% ferr-%offset%)
    (berr single-float berr-%data% berr-%offset%)
    (rwork single-float rwork-%data% rwork-%offset%))
  (prog ((anorm 0.0)
         (norm
          (make-array '(1) :element-type 'character :initial-element #\ ))
         (nofact nil) (notran nil))
    (declare (type (single-float) anorm)
             (type (string 1) norm)
             (type f2cl-lib:logical nofact notran))
    (setf info 0)
    (setf nofact
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "N")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val))
    (setf notran
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "N")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val))
    (cond
     ((and (not nofact)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "F")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val)))
      (setf info -1))
     ((and (not notran)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "T")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "C")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val)))
      (setf info -2))
     ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
```

```
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -14))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -16)))
(cond
 ((/= info 0) (xerbla "CGTSVX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (nofact
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (ccopy n d 1 df 1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf n var-0)))
  (cond
   ((> n 1) (ccopy (f2cl-lib:int-sub n 1) dl 1 dlf 1)
    (ccopy (f2cl-lib:int-sub n 1) du 1 duf 1)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (cgttrf n dlf df duf du2 ipiv info)
    (declare (ignore var-1 var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))
    (when var-6 (setf info var-6)))
  (cond ((> info 0) (setf rcond zero) (go end_label)))))
(cond (notran (f2cl-lib:f2cl-set-string norm "1" (string 1)))
      (t (f2cl-lib:f2cl-set-string norm "I" (string 1))))
(setf anorm
        (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
            (clangt norm n dl d du)
          (declare (ignore var-2 var-3 var-4))
          (when var-0 (setf norm var-0))
          (when var-1 (setf n var-1))
          ret-val))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (cgtcon norm n dlf df duf du2 ipiv anorm rcond work info)
  (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-9))
  (setf norm var-0)
  (setf n var-1)
  (setf rcond var-8)
  (setf info var-10))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (clacpy "Full" n nrhs b ldb$ x ldx)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf nrhs var-2))
  (when var-4 (setf ldb$ var-4))
  (when var-6 (setf ldx var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
```

```
            (cgttrs trans n nrhs dlf df duf du2 ipiv x ldx info)
          (declare (ignore var-3 var-4 var-5 var-6 var-7 var-8))
          (when var-0 (setf trans var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-9 (setf ldx var-9))
          (when var-10 (setf info var-10)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
             var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19)
            (cgtrfs trans n nrhs dl d du dlf df duf du2 ipiv b ldb$ x ldx ferr
             berr work rwork info)
          (declare
           (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
            var-11 var-12 var-13 var-15 var-16 var-17 var-18))
          (setf trans var-0)
          (setf n var-1)
          (setf ldx var-14)
          (setf info var-19))
        (if (< rcond (slamch "Epsilon"))
            (setf info (f2cl-lib:int-add n 1)))
        (go end_label)
       end_label
        (return
         (values fact trans n nrhs nil nil nil nil nil nil nil nil nil ldb$ nil
                 ldx rcond nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgtsvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
```

```
  (array fortran-to-lisp::integer4
   (*))
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (single-float)
  (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::complex8
   (*))
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::fact
  fortran-to-lisp::trans
  fortran-to-lisp::n
  fortran-to-lisp::nrhs nil nil nil
  nil nil nil nil nil nil
  fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond nil nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::cgtrfs
  fortran-to-lisp::cgtcon))))
```

Chapter 77

LAPACK - General Tridiagonal Matrix, Linear Solve, Complex16

77.1 zgtsv

77.1.1 Axiom unit tests

— zgtsv.input —

```
)set break resume
)sys rm -f zgtsv.output
)spool zgtsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

77.1.2 Axiom help page

— zgtsv.help —

```
==========================================================================
```

```
zgtsv examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgtsv (N, NRHS, DL, D, DU, B, LDB, INFO)
            ZGTSV computes the solution to system of linear equations A * X =
           B for GT matrices

Function/Subroutine Documentation
   subroutine zgtsv (integer N, integer NRHS, complex*16, dimension( * ) DL,
       complex*16, dimension( * ) D, complex*16, dimension( * ) DU,
       complex*16, dimension( ldb, * ) B, integer LDB, integer INFO)
        ZGTSV computes the solution to system of linear equations A * X = B
       for GT matrices

Purpose:
   ZGTSV  solves the equation

   A*X = B,

   where A is an N-by-N tridiagonal matrix, by Gaussian elimination with
   partial pivoting.

   Note that the equation  A**T *X = B  may be solved by interchanging the
   order of the arguments DU and DL.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 DL
   DL is COMPLEX*16 array, dimension (N-1)
   On entry, DL must contain the (n-1) subdiagonal elements of A.
   On exit, DL is overwritten by the (n-2) elements of the
   second superdiagonal of the upper triangular matrix U from
   the LU factorization of A, in DL(1), ..., DL(n-2).

 D
```

```
  D is COMPLEX*16 array, dimension (N)
  On entry, D must contain the diagonal elements of A.
  On exit, D is overwritten by the n diagonal elements of U.

DU
  DU is COMPLEX*16 array, dimension (N-1)
  On entry, DU must contain the (n-1) superdiagonal elements of A.
  On exit, DU is overwritten by the (n-1) elements of the first
  superdiagonal of U.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, U(i,i) is exactly zero, and the solution
        has not been computed.  The factorization has not been
        completed unless i = N.
```

———-

77.1.3 fortran code

— **zgtsv.f** —

```
*  =====================================================================
      SUBROUTINE ZGTSV( N, NRHS, DL, D, DU, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      COMPLEX*16         B( LDB, * ), D( * ), DL( * ), DU( * )
*     ..
```

```
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ZERO
      PARAMETER          ( ZERO = ( 0.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            J, K
      COMPLEX*16         MULT, TEMP, ZDUM
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, MAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGTSV ', -INFO )
         RETURN
      END IF
*
      IF( N.EQ.0 )
     $   RETURN
*
      DO 30 K = 1, N - 1
         IF( DL( K ).EQ.ZERO ) THEN
*
*           Subdiagonal is zero, no elimination is required.
*
            IF( D( K ).EQ.ZERO ) THEN
*
*              Diagonal is zero: set INFO = K and return; a unique
*              solution can not be found.
```

```
*
              INFO = K
              RETURN
           END IF
        ELSE IF( CABS1( D( K ) ).GE.CABS1( DL( K ) ) ) THEN
*
*          No row interchange required
*
           MULT = DL( K ) / D( K )
           D( K+1 ) = D( K+1 ) - MULT*DU( K )
           DO 10 J = 1, NRHS
              B( K+1, J ) = B( K+1, J ) - MULT*B( K, J )
   10      CONTINUE
           IF( K.LT.( N-1 ) )
     $        DL( K ) = ZERO
        ELSE
*
*          Interchange rows K and K+1
*
           MULT = D( K ) / DL( K )
           D( K ) = DL( K )
           TEMP = D( K+1 )
           D( K+1 ) = DU( K ) - MULT*TEMP
           IF( K.LT.( N-1 ) ) THEN
              DL( K ) = DU( K+1 )
              DU( K+1 ) = -MULT*DL( K )
           END IF
           DU( K ) = TEMP
           DO 20 J = 1, NRHS
              TEMP = B( K, J )
              B( K, J ) = B( K+1, J )
              B( K+1, J ) = TEMP - MULT*B( K+1, J )
   20      CONTINUE
        END IF
   30 CONTINUE
      IF( D( N ).EQ.ZERO ) THEN
         INFO = N
         RETURN
      END IF
*
*     Back solve with the matrix U from the factorization.
*
      DO 50 J = 1, NRHS
         B( N, J ) = B( N, J ) / D( N )
         IF( N.GT.1 )
     $      B( N-1, J ) = ( B( N-1, J )-DU( N-1 )*B( N, J ) ) / D( N-1 )
         DO 40 K = N - 2, 1, -1
            B( K, J ) = ( B( K, J )-DU( K )*B( K+1, J )-DL( K )*
     $                  B( K+2, J ) ) / D( K )
   40    CONTINUE
```

```
   50 CONTINUE
*
      RETURN
*
*     End of ZGTSV
*
      END
```

———-

77.1.4 lisp code

— **zgtsv.lisp** —

```
(let* ((zero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) zero)
           (ignorable zero))
  (defun zgtsv (n nrhs dl d du b ldb$ info)
    (declare (type (array f2cl-lib:complex16 (*)) b du d dl)
             (type (f2cl-lib:integer4) info ldb$ nrhs n))
    (f2cl-lib:with-multi-array-data
        ((dl f2cl-lib:complex16 dl-%data% dl-%offset%)
         (d f2cl-lib:complex16 d-%data% d-%offset%)
         (du f2cl-lib:complex16 du-%data% du-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
          cabs1))
        (prog ((mult #C(0.0d0 0.0d0)) (temp #C(0.0d0 0.0d0))
               (zdum #C(0.0d0 0.0d0)) (j 0) (k 0))
          (declare (type (f2cl-lib:complex16) mult temp zdum)
                   (type (f2cl-lib:integer4) j k))
          (setf info 0)
          (cond ((< n 0) (setf info -1)) ((< nrhs 0) (setf info -2))
                ((< ldb$
                    (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
                 (setf info -7)))
          (cond
           ((/= info 0) (xerbla "ZGTSV " (f2cl-lib:int-sub info))
            (go end_label)))
          (if (= n 0)
              (go end_label))
          (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                        ((> k (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
            (tagbody
              (cond
```

```
((= (f2cl-lib:fref dl (k) ((1 *))) zero)
 (cond
  ((= (f2cl-lib:fref d (k) ((1 *))) zero) (setf info k)
   (go end_label))))
((>= (cabs1 (f2cl-lib:fref d (k) ((1 *))))
     (cabs1 (f2cl-lib:fref dl (k) ((1 *)))))
 (setf mult
         (/ (f2cl-lib:fref dl-%data% (k) ((1 *)) dl-%offset%)
            (f2cl-lib:fref d-%data% (k) ((1 *)) d-%offset%)))
 (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add k 1)) ((1 *))
                       d-%offset%)
         (-
          (f2cl-lib:fref d-%data% ((f2cl-lib:int-add k 1))
                         ((1 *)) d-%offset%)
          (* mult
             (f2cl-lib:fref du-%data% (k) ((1 *))
                            du-%offset%))))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j nrhs) nil)
   (tagbody
     (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                           ((1 ldb$) (1 *)) b-%offset%)
             (-
              (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)
              (* mult
                 (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                b-%offset%))))
    label10))
 (if (< k (f2cl-lib:int-sub n 1))
     (setf (f2cl-lib:fref dl-%data% (k) ((1 *)) dl-%offset%)
             zero)))
(t
 (setf mult
         (/ (f2cl-lib:fref d-%data% (k) ((1 *)) d-%offset%)
            (f2cl-lib:fref dl-%data% (k) ((1 *)) dl-%offset%)))
 (setf (f2cl-lib:fref d-%data% (k) ((1 *)) d-%offset%)
         (f2cl-lib:fref dl-%data% (k) ((1 *)) dl-%offset%))
 (setf temp
         (f2cl-lib:fref d-%data% ((f2cl-lib:int-add k 1))
                        ((1 *)) d-%offset%))
 (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add k 1)) ((1 *))
                       d-%offset%)
         (- (f2cl-lib:fref du-%data% (k) ((1 *)) du-%offset%)
            (* mult temp)))
 (cond
  ((< k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
   (setf (f2cl-lib:fref dl-%data% (k) ((1 *)) dl-%offset%)
           (f2cl-lib:fref du-%data% ((f2cl-lib:int-add k 1))
                          ((1 *)) du-%offset%))
```

```
          (setf (f2cl-lib:fref du-%data% ((f2cl-lib:int-add k 1))
                               ((1 *)) du-%offset%)
                  (* (- mult)
                     (f2cl-lib:fref dl-%data% (k) ((1 *))
                                    dl-%offset%)))))
        (setf (f2cl-lib:fref du-%data% (k) ((1 *)) du-%offset%) temp)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j nrhs) nil)
          (tagbody
            (setf temp
                    (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                   b-%offset%))
            (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                 b-%offset%)
                    (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                                   ((1 ldb$) (1 *)) b-%offset%))
            (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                    (- temp
                       (* mult
                          (f2cl-lib:fref b-%data%
                                         ((f2cl-lib:int-add k 1) j)
                                         ((1 ldb$) (1 *))
                                         b-%offset%))))
          label20))))
   label30))
(cond
 ((= (f2cl-lib:fref d (n) ((1 *))) zero) (setf info n)
  (go end_label)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
            (/
             (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                            b-%offset%)
             (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
    (if (> n 1)
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)
                (/
                 (-
                  (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                  (*
                   (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub n 1))
                                  ((1 *)) du-%offset%)
                   (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                                  b-%offset%)))
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
```

```
                                        ((1 *)) d-%offset%))))
              (f2cl-lib:fdo (k (f2cl-lib:int-add n (f2cl-lib:int-sub 2))
                             (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                            ((> k 1) nil)
                (tagbody
                  (setf (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                       b-%offset%)
                        (/
                         (-
                          (f2cl-lib:fref b-%data% (k j) ((1 ldb$) (1 *))
                                         b-%offset%)
                          (*
                           (f2cl-lib:fref du-%data% (k) ((1 *)) du-%offset%)
                           (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 1) j)
                                          ((1 ldb$) (1 *)) b-%offset%))
                          (*
                           (f2cl-lib:fref dl-%data% (k) ((1 *)) dl-%offset%)
                           (f2cl-lib:fref b-%data% ((f2cl-lib:int-add k 2) j)
                                          ((1 ldb$) (1 *)) b-%offset%)))
                         (f2cl-lib:fref d-%data% (k) ((1 *)) d-%offset%)))
                 label40))
             label50))
          (go end_label)
         end_label
          (return (values nil nil nil nil nil nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgtsv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

77.2 zgtsvx

77.2.1 Axiom unit tests

— zgtsvx.input —

```
)set break resume
)sys rm -f zgtsvx.output
)spool zgtsvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

77.2.2 Axiom help page

— zgtsvx.help —

```
======================================================================
zgtsvx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgtsvx (FACT, TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF, DU2,
           IPIV, B, LDB, X, LDX, RCOND, FERR, BERR, WORK, RWORK, INFO)
            ZGTSVX computes the solution to system of linear equations A * X =
           B for GT matrices

Function/Subroutine Documentation
   subroutine zgtsvx (character FACT, character TRANS, integer N, integer
       NRHS, complex*16, dimension( * ) DL, complex*16, dimension( * ) D,
       complex*16, dimension( * ) DU, complex*16, dimension( * ) DLF,
       complex*16, dimension( * ) DF, complex*16, dimension( * ) DUF,
```

```
      complex*16, dimension( * ) DU2, integer, dimension( * ) IPIV,
      complex*16, dimension( ldb, * ) B, integer LDB, complex*16, dimension(
      ldx, * ) X, integer LDX, double precision RCOND, double precision,
      dimension( * ) FERR, double precision, dimension( * ) BERR, complex*16,
      dimension( * ) WORK, double precision, dimension( * ) RWORK, integer
      INFO)
       ZGTSVX computes the solution to system of linear equations A * X = B
      for GT matrices

Purpose:
   ZGTSVX uses the LU factorization to compute the solution to a complex
   system of linear equations A * X = B, A**T * X = B, or A**H * X = B,
   where A is a tridiagonal matrix of order N and X and B are N-by-NRHS
   matrices.

   Error bounds on the solution and a condition estimate are also
   provided.

Description:

   The following steps are performed:

   1. If FACT = 'N', the LU decomposition is used to factor the matrix A
      as A = L * U, where L is a product of permutation and unit lower
      bidiagonal matrices and U is upper triangular with nonzeros in
      only the main diagonal and first two superdiagonals.

   2. If some U(i,i)=0, so that U is exactly singular, then the routine
      returns with INFO = i. Otherwise, the factored form of A is used
      to estimate the condition number of the matrix A.  If the
      reciprocal of the condition number is less than machine precision,
      INFO = N+1 is returned as a warning, but the routine still goes on
      to solve for X and compute error bounds as described below.

   3. The system of equations is solved for X using the factored form of A.

   4. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of A has been
   supplied on entry.
   = 'F':  DLF, DF, DUF, DU2, and IPIV contain the factored form
      of A; DL, D, DU, DLF, DF, DUF, DU2 and IPIV will not
      be modified.
   = 'N':  The matrix will be copied to DLF, DF, and DUF
      and factored.
```

```
TRANS
  TRANS is CHARACTER*1
  Specifies the form of the system of equations:
  = 'N':  A * X = B     (No transpose)
  = 'T':  A**T * X = B  (Transpose)
  = 'C':  A**H * X = B  (Conjugate transpose)

N
  N is INTEGER
  The order of the matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrix B.  NRHS >= 0.

DL
  DL is COMPLEX*16 array, dimension (N-1)
  The (n-1) subdiagonal elements of A.

D
  D is COMPLEX*16 array, dimension (N)
  The n diagonal elements of A.

DU
  DU is COMPLEX*16 array, dimension (N-1)
  The (n-1) superdiagonal elements of A.

DLF
  DLF is COMPLEX*16 array, dimension (N-1)
  If FACT = 'F', then DLF is an input argument and on entry
  contains the (n-1) multipliers that define the matrix L from
  the LU factorization of A as computed by ZGTTRF.

  If FACT = 'N', then DLF is an output argument and on exit
  contains the (n-1) multipliers that define the matrix L from
  the LU factorization of A.

DF
  DF is COMPLEX*16 array, dimension (N)
  If FACT = 'F', then DF is an input argument and on entry
  contains the n diagonal elements of the upper triangular
  matrix U from the LU factorization of A.

  If FACT = 'N', then DF is an output argument and on exit
  contains the n diagonal elements of the upper triangular
  matrix U from the LU factorization of A.

DUF
```

```
  DUF is COMPLEX*16 array, dimension (N-1)
  If FACT = 'F', then DUF is an input argument and on entry
  contains the (n-1) elements of the first superdiagonal of U.

  If FACT = 'N', then DUF is an output argument and on exit
  contains the (n-1) elements of the first superdiagonal of U.

DU2
  DU2 is COMPLEX*16 array, dimension (N-2)
  If FACT = 'F', then DU2 is an input argument and on entry
  contains the (n-2) elements of the second superdiagonal of U.

  If FACT = 'N', then DU2 is an output argument and on exit
  contains the (n-2) elements of the second superdiagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  If FACT = 'F', then IPIV is an input argument and on entry
  contains the pivot indices from the LU factorization of A as
  computed by ZGTTRF.

  If FACT = 'N', then IPIV is an output argument and on exit
  contains the pivot indices from the LU factorization of A;
  row i of the matrix was interchanged with row IPIV(i).
  IPIV(i) will always be either i or i+1; IPIV(i) = i indicates
  a row interchange was not required.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  The N-by-NRHS right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  The estimate of the reciprocal condition number of the matrix
  A.  If RCOND is less than the machine precision (in
  particular, if RCOND = 0), the matrix is singular to working
  precision.  This condition is indicated by a return code of INFO > 0.
```

```
FERR
  FERR is DOUBLE PRECISION array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is COMPLEX*16 array, dimension (2*N)

RWORK
  RWORK is DOUBLE PRECISION array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, and i is
  <= N:  U(i,i) is exactly zero.  The factorization
     has not been completed unless i = N, but the
     factor U is exactly singular, so the solution
     and error bounds could not be computed.
     RCOND = 0 is returned.
  = N+1: U is nonsingular, but RCOND is less than machine
     precision, meaning that the matrix is singular
     to working precision.  Nevertheless, the
     solution and error bounds are computed because
     there are a number of situations where the
     computed solution can be more accurate than the
     value of RCOND would suggest.
```

———-

77.2.3 fortran code

— zgtsvx.f —

```
*  =====================================================================
      SUBROUTINE ZGTSVX( FACT, TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF,
     $                   DU2, IPIV, B, LDB, X, LDX, RCOND, FERR, BERR,
     $                   WORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          FACT, TRANS
      INTEGER            INFO, LDB, LDX, N, NRHS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   BERR( * ), FERR( * ), RWORK( * )
      COMPLEX*16         B( LDB, * ), D( * ), DF( * ), DL( * ),
     $                   DLF( * ), DU( * ), DU2( * ), DUF( * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOFACT, NOTRAN
      CHARACTER          NORM
      DOUBLE PRECISION   ANORM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, ZLANGT
      EXTERNAL           LSAME, DLAMCH, ZLANGT
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZCOPY, ZGTCON, ZGTRFS, ZGTTRF, ZGTTRS,
     $                   ZLACPY
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
```

```
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOFACT .AND. .NOT.LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $         LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -14
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -16
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGTSVX', -INFO )
         RETURN
      END IF
*
      IF( NOFACT ) THEN
*
*        Compute the LU factorization of A.
*
         CALL ZCOPY( N, D, 1, DF, 1 )
         IF( N.GT.1 ) THEN
            CALL ZCOPY( N-1, DL, 1, DLF, 1 )
            CALL ZCOPY( N-1, DU, 1, DUF, 1 )
         END IF
         CALL ZGTTRF( N, DLF, DF, DUF, DU2, IPIV, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 )THEN
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A.
*
      IF( NOTRAN ) THEN
         NORM = '1'
      ELSE
         NORM = 'I'
      END IF
      ANORM = ZLANGT( NORM, N, DL, D, DU )
*
*     Compute the reciprocal of the condition number of A.
*
```

```
      CALL ZGTCON( NORM, N, DLF, DF, DUF, DU2, IPIV, ANORM, RCOND, WORK,
     $             INFO )
*
*     Compute the solution vectors X.
*
      CALL ZLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL ZGTTRS( TRANS, N, NRHS, DLF, DF, DUF, DU2, IPIV, X, LDX,
     $             INFO )
*
*     Use iterative refinement to improve the computed solutions and
*     compute error bounds and backward error estimates for them.
*
      CALL ZGTRFS( TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF, DU2, IPIV,
     $             B, LDB, X, LDX, FERR, BERR, WORK, RWORK, INFO )
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.DLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      RETURN
*
*     End of ZGTSVX
*
      END
```

———-

77.2.4 lisp code

— zgtsvx.lisp —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun zgtsvx
         (fact trans n nrhs dl d du dlf df duf du2 ipiv b ldb$ x ldx rcond ferr
          berr work rwork info)
    (declare (type (array double-float (*)) rwork berr ferr)
             (type (double-float) rcond)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) work x b du2 duf df dlf du d
              dl)
             (type (f2cl-lib:integer4) info ldx ldb$ nrhs n)
             (type (string *) trans fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (trans character trans-%data% trans-%offset%)
```

```
    (dl f2cl-lib:complex16 dl-%data% dl-%offset%)
    (d f2cl-lib:complex16 d-%data% d-%offset%)
    (du f2cl-lib:complex16 du-%data% du-%offset%)
    (dlf f2cl-lib:complex16 dlf-%data% dlf-%offset%)
    (df f2cl-lib:complex16 df-%data% df-%offset%)
    (duf f2cl-lib:complex16 duf-%data% duf-%offset%)
    (du2 f2cl-lib:complex16 du2-%data% du2-%offset%)
    (b f2cl-lib:complex16 b-%data% b-%offset%)
    (x f2cl-lib:complex16 x-%data% x-%offset%)
    (work f2cl-lib:complex16 work-%data% work-%offset%)
    (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
    (ferr double-float ferr-%data% ferr-%offset%)
    (berr double-float berr-%data% berr-%offset%)
    (rwork double-float rwork-%data% rwork-%offset%))
  (prog ((anorm 0.0d0)
         (norm
          (make-array '(1) :element-type 'character :initial-element #\ ))
         (nofact nil) (notran nil))
    (declare (type (double-float) anorm)
             (type (string 1) norm)
             (type f2cl-lib:logical nofact notran))
    (setf info 0)
    (setf nofact
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "N")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val))
    (setf notran
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "N")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val))
    (cond
     ((and (not nofact)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame fact "F")
              (declare (ignore var-1))
              (when var-0 (setf fact var-0))
              ret-val)))
      (setf info -1))
     ((and (not notran)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "T")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val))
```

```
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame trans "C")
           (declare (ignore var-1))
           (when var-0 (setf trans var-0))
           ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3)) ((< nrhs 0) (setf info -4))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -14))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -16)))
(cond
 ((/= info 0) (xerbla "ZGTSVX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (nofact
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (zcopy n d 1 df 1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf n var-0)))
  (cond
   ((> n 1) (zcopy (f2cl-lib:int-sub n 1) dl 1 dlf 1)
    (zcopy (f2cl-lib:int-sub n 1) du 1 duf 1)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zgttrf n dlf df duf du2 ipiv info)
    (declare (ignore var-1 var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))
    (when var-6 (setf info var-6)))
  (cond ((> info 0) (setf rcond zero) (go end_label)))))
(cond (notran (f2cl-lib:f2cl-set-string norm "1" (string 1)))
      (t (f2cl-lib:f2cl-set-string norm "I" (string 1))))
(setf anorm
        (multiple-value-bind (ret-val var-0 var-1 var-2 var-3 var-4)
            (zlangt norm n dl d du)
          (declare (ignore var-2 var-3 var-4))
          (when var-0 (setf norm var-0))
          (when var-1 (setf n var-1))
          ret-val))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgtcon norm n dlf df duf du2 ipiv anorm rcond work info)
  (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-9))
  (setf norm var-0)
  (setf n var-1)
  (setf rcond var-8)
  (setf info var-10))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlacpy "Full" n nrhs b ldb$ x ldx)
```

```
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-4 (setf ldb$ var-4))
          (when var-6 (setf ldx var-6)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10)
            (zgttrs trans n nrhs dlf df duf du2 ipiv x ldx info)
          (declare (ignore var-3 var-4 var-5 var-6 var-7 var-8))
          (when var-0 (setf trans var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-9 (setf ldx var-9))
          (when var-10 (setf info var-10)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
             var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19)
            (zgtrfs trans n nrhs dl d du dlf df duf du2 ipiv b ldb$ x ldx ferr
             berr work rwork info)
          (declare
           (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
            var-11 var-12 var-13 var-15 var-16 var-17 var-18))
          (setf trans var-0)
          (setf n var-1)
          (setf ldx var-14)
          (setf info var-19))
        (if (< rcond (dlamch "Epsilon"))
            (setf info (f2cl-lib:int-add n 1)))
        (go end_label)
       end_label
        (return
         (values fact trans n nrhs nil nil nil nil nil nil nil nil nil ldb$ nil
                 ldx rcond nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgtsvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
```

```
  (array fortran-to-lisp::complex16
   (*))
  (array fortran-to-lisp::complex16
   (*))
  (array fortran-to-lisp::complex16
   (*))
  (array fortran-to-lisp::complex16
   (*))
  (array fortran-to-lisp::integer4
   (*))
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (double-float)
  (array double-float (*))
  (array double-float (*))
  (array fortran-to-lisp::complex16
   (*))
  (array double-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::fact
  fortran-to-lisp::trans
  fortran-to-lisp::n
  fortran-to-lisp::nrhs nil nil nil
  nil nil nil nil nil nil
  fortran-to-lisp::ldb$ nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond nil nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zgtrfs
  fortran-to-lisp::zgtcon
  fortran-to-lisp::xerbla))))
```

Chapter 78

LAPACK - General Tridiagonal Matrix, Comp. Routines, Double

78.1 dgtcon

78.1.1 Axiom unit tests

— **dgtcon.input** —

```
)set break resume
)sys rm -f dgtcon.output
)spool dgtcon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

78.1.2 Axiom help page

— **dgtcon.help** —

```
==========================================================================
```

```
dgtcon examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgtcon (NORM, N, DL, D, DU, DU2, IPIV, ANORM, RCOND, WORK,
           IWORK, INFO)
           DGTCON

Function/Subroutine Documentation
   subroutine dgtcon (character NORM, integer N, double precision, dimension(
       * ) DL, double precision, dimension( * ) D, double precision,
       dimension( * ) DU, double precision, dimension( * ) DU2, integer,
       dimension( * ) IPIV, double precision ANORM, double precision RCOND,
       double precision, dimension( * ) WORK, integer, dimension( * ) IWORK,
       integer INFO)
       DGTCON

Purpose:
   DGTCON estimates the reciprocal of the condition number of a real
   tridiagonal matrix A using the LU factorization as computed by
   DGTTRF.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies whether the 1-norm condition number or the
   infinity-norm condition number is required:
   = '1' or 'O':  1-norm;
   = 'I':         Infinity-norm.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 DL
   DL is DOUBLE PRECISION array, dimension (N-1)
   The (n-1) multipliers that define the matrix L from the
   LU factorization of A as computed by DGTTRF.

 D
   D is DOUBLE PRECISION array, dimension (N)
```

```
  The n diagonal elements of the upper triangular matrix U from
  the LU factorization of A.

DU
  DU is DOUBLE PRECISION array, dimension (N-1)
  The (n-1) elements of the first superdiagonal of U.

DU2
  DU2 is DOUBLE PRECISION array, dimension (N-2)
  The (n-2) elements of the second superdiagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= n, row i of the matrix was
  interchanged with row IPIV(i).  IPIV(i) will always be either
  i or i+1; IPIV(i) = i indicates a row interchange was not required.

ANORM
  ANORM is DOUBLE PRECISION
  If NORM = '1' or 'O', the 1-norm of the original matrix A.
  If NORM = 'I', the infinity-norm of the original matrix A.

RCOND
  RCOND is DOUBLE PRECISION
  The reciprocal of the condition number of the matrix A,
  computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is an
  estimate of the 1-norm of inv(A) computed in this routine.

WORK
  WORK is DOUBLE PRECISION array, dimension (2*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

78.1.3 fortran code

— dgtcon.f —

```
*  =====================================================================
      SUBROUTINE DGTCON( NORM, N, DL, D, DU, DU2, IPIV, ANORM, RCOND,
```

```
     $                   WORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            INFO, N
      DOUBLE PRECISION   ANORM, RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      DOUBLE PRECISION   D( * ), DL( * ), DU( * ), DU2( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ONENRM
      INTEGER            I, KASE, KASE1
      DOUBLE PRECISION   AINVNM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGTTRS, DLACN2, XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      ONENRM = NORM.EQ.'1' .OR. LSAME( NORM, 'O' )
      IF( .NOT.ONENRM .AND. .NOT.LSAME( NORM, 'I' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -8
```

```
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGTCON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
*     Check that D(1:N) is non-zero.
*
      DO 10 I = 1, N
         IF( D( I ).EQ.ZERO )
     $      RETURN
   10 CONTINUE
*
      AINVNM = ZERO
      IF( ONENRM ) THEN
         KASE1 = 1
      ELSE
         KASE1 = 2
      END IF
      KASE = 0
   20 CONTINUE
      CALL DLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.KASE1 ) THEN
*
*           Multiply by inv(U)*inv(L).
*
            CALL DGTTRS( 'No transpose', N, 1, DL, D, DU, DU2, IPIV,
     $                   WORK, N, INFO )
         ELSE
*
*           Multiply by inv(L**T)*inv(U**T).
*
            CALL DGTTRS( 'Transpose', N, 1, DL, D, DU, DU2, IPIV, WORK,
     $                   N, INFO )
         END IF
         GO TO 20
      END IF
*
*     Compute the estimate of the reciprocal condition number.
```

```
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
      RETURN
*
*     End of DGTCON
*
      END
```

———-

78.1.4 lisp code

— **dgtcon.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dgtcon (norm n dl d du du2 ipiv anorm rcond work iwork info)
    (declare (type (double-float) rcond anorm)
             (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array double-float (*)) work du2 du d dl)
             (type (f2cl-lib:integer4) info n)
             (type (string *) norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (dl double-float dl-%data% dl-%offset%)
         (d double-float d-%data% d-%offset%)
         (du double-float du-%data% du-%offset%)
         (du2 double-float du2-%data% du2-%offset%)
         (work double-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (ainvnm 0.0d0) (i 0) (kase 0) (kase1 0) (onenrm nil))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (double-float) ainvnm)
                 (type (f2cl-lib:integer4) i kase kase1)
                 (type f2cl-lib:logical onenrm))
        (setf info 0)
        (setf onenrm
                (or (f2cl-lib:fstring-= norm "1")
                    (multiple-value-bind (ret-val var-0 var-1)
                        (lsame norm "O")
                      (declare (ignore var-1))
                      (when var-0 (setf norm var-0))
```

```
                ret-val)))
  (cond
   ((and (not onenrm)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame norm "I")
            (declare (ignore var-1))
            (when var-0 (setf norm var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2)) ((< anorm zero) (setf info -8)))
  (cond
   ((/= info 0) (xerbla "DGTCON" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf rcond zero)
  (cond ((= n 0) (setf rcond one) (go end_label))
        ((= anorm zero) (go end_label)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (if (= (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) zero)
          (go end_label))
     label10))
  (setf ainvnm zero)
  (cond (onenrm (setf kase1 1)) (t (setf kase1 2)))
  (setf kase 0)
 label20
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacn2 n
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1)) ((1 *))
                             work-%offset%)
       work iwork ainvnm kase isave)
    (declare (ignore var-1 var-2 var-3 var-6))
    (when var-0 (setf n var-0))
    (when var-4 (setf ainvnm var-4))
    (when var-5 (setf kase var-5)))
  (cond
   ((/= kase 0)
    (cond
     ((= kase kase1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (dgttrs "No transpose" n 1 dl d du du2 ipiv work n info)
        (declare
         (ignore var-0 var-2 var-3 var-4 var-5 var-6 var-7 var-8))
        (when var-1 (setf n var-1))
        (when var-9 (setf n var-9))
        (when var-10 (setf info var-10))))
     (t
```

```
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                 var-10)
                (dgttrs "Transpose" n 1 dl d du du2 ipiv work n info)
              (declare
               (ignore var-0 var-2 var-3 var-4 var-5 var-6 var-7 var-8))
              (when var-1 (setf n var-1))
              (when var-9 (setf n var-9))
              (when var-10 (setf info var-10)))))
          (go label20)))
        (if (/= ainvnm zero)
            (setf rcond (/ (/ one ainvnm) anorm)))
        (go end_label)
       end_label
        (return (values norm n nil nil nil nil nil nil rcond nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgtcon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (double-float) (double-float)
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::norm
                                              fortran-to-lisp::n nil nil nil
                                              nil nil nil
                                              fortran-to-lisp::rcond nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

78.2 dgtrfs

78.2.1 Axiom unit tests

— dgtrfs.input —

```
)set break resume
)sys rm -f dgtrfs.output
)spool dgtrfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

78.2.2 Axiom help page

— dgtrfs.help —

```
======================================================================
dgtrfs examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgtrfs (TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF, DU2, IPIV,
           B, LDB, X, LDX, FERR, BERR, WORK, IWORK, INFO)
           DGTRFS

Function/Subroutine Documentation
   subroutine dgtrfs (character TRANS, integer N, integer NRHS, double
       precision, dimension( * ) DL, double precision, dimension( * ) D,
       double precision, dimension( * ) DU, double precision, dimension( * )
       DLF, double precision, dimension( * ) DF, double precision, dimension(
       * ) DUF, double precision, dimension( * ) DU2, integer, dimension( * )
       IPIV, double precision, dimension( ldb, * ) B, integer LDB, double
       precision, dimension( ldx, * ) X, integer LDX, double precision,
       dimension( * ) FERR, double precision, dimension( * ) BERR, double
       precision, dimension( * ) WORK, integer, dimension( * ) IWORK, integer
```

```
       INFO)
       DGTRFS

Purpose:
   DGTRFS improves the computed solution to a system of linear
   equations when the coefficient matrix is tridiagonal, and provides
   error bounds and backward error estimates for the solution.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose = Transpose)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 DL
   DL is DOUBLE PRECISION array, dimension (N-1)
   The (n-1) subdiagonal elements of A.

 D
   D is DOUBLE PRECISION array, dimension (N)
   The diagonal elements of A.

 DU
   DU is DOUBLE PRECISION array, dimension (N-1)
   The (n-1) superdiagonal elements of A.

 DLF
   DLF is DOUBLE PRECISION array, dimension (N-1)
   The (n-1) multipliers that define the matrix L from the
   LU factorization of A as computed by DGTTRF.

 DF
   DF is DOUBLE PRECISION array, dimension (N)
   The n diagonal elements of the upper triangular matrix U from
   the LU factorization of A.

 DUF
   DUF is DOUBLE PRECISION array, dimension (N-1)
```

```
  The (n-1) elements of the first superdiagonal of U.

DU2
  DU2 is DOUBLE PRECISION array, dimension (N-2)
  The (n-2) elements of the second superdiagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= n, row i of the matrix was
  interchanged with row IPIV(i).  IPIV(i) will always be either
  i or i+1; IPIV(i) = i indicates a row interchange was not required.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by DGTTRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

FERR
  FERR is DOUBLE PRECISION array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is DOUBLE PRECISION array, dimension (3*N)

IWORK
```

```
   IWORK is INTEGER array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

   ITMAX is the maximum number of steps of iterative refinement.
```

———-

78.2.3 fortran code

— **dgtrfs.f** —

```
*  =====================================================================
      SUBROUTINE DGTRFS( TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF, DU2,
     $                   IPIV, B, LDB, X, LDX, FERR, BERR, WORK, IWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      DOUBLE PRECISION   B( LDB, * ), BERR( * ), D( * ), DF( * ),
     $                   DL( * ), DLF( * ), DU( * ), DU2( * ), DUF( * ),
     $                   FERR( * ), WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D+0 )
```

```
      DOUBLE PRECISION   THREE
      PARAMETER          ( THREE = 3.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      CHARACTER          TRANSN, TRANST
      INTEGER            COUNT, I, J, KASE, NZ
      DOUBLE PRECISION   EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DCOPY, DGTTRS, DLACN2, DLAGTM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -13
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -15
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGTRFS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
```

```
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
      IF( NOTRAN ) THEN
         TRANSN = 'N'
         TRANST = 'T'
      ELSE
         TRANSN = 'T'
         TRANST = 'N'
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = 4
      EPS = DLAMCH( 'Epsilon' )
      SAFMIN = DLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 110 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - op(A) * X,
*        where op(A) = A, A**T, or A**H, depending on TRANS.
*
         CALL DCOPY( N, B( 1, J ), 1, WORK( N+1 ), 1 )
         CALL DLAGTM( TRANS, N, 1, -ONE, DL, D, DU, X( 1, J ), LDX, ONE,
     $                WORK( N+1 ), N )
*
*        Compute abs(op(A))*abs(x) + abs(b) for use in the backward
*        error bound.
*
         IF( NOTRAN ) THEN
            IF( N.EQ.1 ) THEN
               WORK( 1 ) = ABS( B( 1, J ) ) + ABS( D( 1 )*X( 1, J ) )
            ELSE
               WORK( 1 ) = ABS( B( 1, J ) ) + ABS( D( 1 )*X( 1, J ) ) +
     $                     ABS( DU( 1 )*X( 2, J ) )
               DO 30 I = 2, N - 1
                  WORK( I ) = ABS( B( I, J ) ) +
```

```
     $                       ABS( DL( I-1 )*X( I-1, J ) ) +
     $                       ABS( D( I )*X( I, J ) ) +
     $                       ABS( DU( I )*X( I+1, J ) )
   30          CONTINUE
               WORK( N ) = ABS( B( N, J ) ) +
     $                     ABS( DL( N-1 )*X( N-1, J ) ) +
     $                     ABS( D( N )*X( N, J ) )
            END IF
         ELSE
            IF( N.EQ.1 ) THEN
               WORK( 1 ) = ABS( B( 1, J ) ) + ABS( D( 1 )*X( 1, J ) )
            ELSE
               WORK( 1 ) = ABS( B( 1, J ) ) + ABS( D( 1 )*X( 1, J ) ) +
     $                     ABS( DL( 1 )*X( 2, J ) )
               DO 40 I = 2, N - 1
                  WORK( I ) = ABS( B( I, J ) ) +
     $                       ABS( DU( I-1 )*X( I-1, J ) ) +
     $                       ABS( D( I )*X( I, J ) ) +
     $                       ABS( DL( I )*X( I+1, J ) )
   40          CONTINUE
               WORK( N ) = ABS( B( N, J ) ) +
     $                     ABS( DU( N-1 )*X( N-1, J ) ) +
     $                     ABS( D( N )*X( N, J ) )
            END IF
         END IF
*
*        Compute componentwise relative backward error from formula
*
*        max(i) ( abs(R(i)) / ( abs(op(A))*abs(X) + abs(B) )(i) )
*
*        where abs(Z) is the componentwise absolute value of the matrix
*        or vector Z.  If the i-th component of the denominator is less
*        than SAFE2, then SAFE1 is added to the i-th components of the
*        numerator and denominator before dividing.
*
         S = ZERO
         DO 50 I = 1, N
            IF( WORK( I ).GT.SAFE2 ) THEN
               S = MAX( S, ABS( WORK( N+I ) ) / WORK( I ) )
            ELSE
               S = MAX( S, ( ABS( WORK( N+I ) )+SAFE1 ) /
     $             ( WORK( I )+SAFE1 ) )
            END IF
   50    CONTINUE
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
*           1) The residual BERR(J) is larger than machine epsilon, and
*           2) BERR(J) decreased by at least a factor of 2 during the
*              last iteration, and
```

```
*          3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
*
*          Update solution and try again.
*
           CALL DGTTRS( TRANS, N, 1, DLF, DF, DUF, DU2, IPIV,
     $                  WORK( N+1 ), N, INFO )
           CALL DAXPY( N, ONE, WORK( N+1 ), 1, X( 1, J ), 1 )
           LSTRES = BERR( J )
           COUNT = COUNT + 1
           GO TO 20
         END IF
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(op(A)))*
*           ( abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
*          inv(op(A)) is the inverse of op(A)
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(op(A))*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(op(A))*abs(X) + abs(B) is less than SAFE2.
*
*        Use DLACN2 to estimate the infinity-norm of the matrix
*           inv(op(A)) * diag(W),
*        where W = abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) )))
*
         DO 60 I = 1, N
            IF( WORK( I ).GT.SAFE2 ) THEN
               WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I )
            ELSE
               WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I ) + SAFE1
            END IF
   60    CONTINUE
*
         KASE = 0
   70    CONTINUE
         CALL DLACN2( N, WORK( 2*N+1 ), WORK( N+1 ), IWORK, FERR( J ),
     $                KASE, ISAVE )
         IF( KASE.NE.0 ) THEN
```

```
            IF( KASE.EQ.1 ) THEN
*
*              Multiply by diag(W)*inv(op(A)**T).
*
               CALL DGTTRS( TRANST, N, 1, DLF, DF, DUF, DU2, IPIV,
     $                      WORK( N+1 ), N, INFO )
               DO 80 I = 1, N
                  WORK( N+I ) = WORK( I )*WORK( N+I )
   80          CONTINUE
            ELSE
*
*              Multiply by inv(op(A))*diag(W).
*
               DO 90 I = 1, N
                  WORK( N+I ) = WORK( I )*WORK( N+I )
   90          CONTINUE
               CALL DGTTRS( TRANSN, N, 1, DLF, DF, DUF, DU2, IPIV,
     $                      WORK( N+1 ), N, INFO )
            END IF
            GO TO 70
         END IF
*
*        Normalize error.
*
         LSTRES = ZERO
         DO 100 I = 1, N
            LSTRES = MAX( LSTRES, ABS( X( I, J ) ) )
  100    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  110 CONTINUE
*
      RETURN
*
*     End of DGTRFS
*
      END
```

———-

78.2.4 lisp code

— **dgtrfs.lisp** —

```
(let* ((itmax 5) (zero 0.0d0) (one 1.0d0) (two 2.0d0) (three 3.0d0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (double-float 0.0d0 0.0d0) zero)
```

```
          (type (double-float 1.0d0 1.0d0) one)
          (type (double-float 2.0d0 2.0d0) two)
          (type (double-float 3.0d0 3.0d0) three)
          (ignorable itmax zero one two three))
(defun dgtrfs
       (trans n nrhs dl d du dlf df duf du2 ipiv b ldb$ x ldx ferr berr work
        iwork info)
  (declare (type (array f2cl-lib:integer4 (*)) iwork ipiv)
           (type (array double-float (*)) work berr ferr x b du2 duf df dlf
            du d dl)
           (type (f2cl-lib:integer4) info ldx ldb$ nrhs n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (dl double-float dl-%data% dl-%offset%)
       (d double-float d-%data% d-%offset%)
       (du double-float du-%data% du-%offset%)
       (dlf double-float dlf-%data% dlf-%offset%)
       (df double-float df-%data% df-%offset%)
       (duf double-float duf-%data% duf-%offset%)
       (du2 double-float du2-%data% du2-%offset%)
       (b double-float b-%data% b-%offset%)
       (x double-float x-%data% x-%offset%)
       (ferr double-float ferr-%data% ferr-%offset%)
       (berr double-float berr-%data% berr-%offset%)
       (work double-float work-%data% work-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
       (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
    (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
           (eps 0.0d0) (lstres 0.0d0) (s 0.0d0) (safe1 0.0d0) (safe2 0.0d0)
           (safmin 0.0d0) (count$ 0) (i 0) (j 0) (kase 0) (nz 0)
           (transn
            (make-array '(1) :element-type 'character :initial-element #\ ))
           (transt
            (make-array '(1) :element-type 'character :initial-element #\ ))
           (notran nil))
      (declare (type (array f2cl-lib:integer4 (3)) isave)
               (type (double-float) eps lstres s safe1 safe2 safmin)
               (type (f2cl-lib:integer4) count$ i j kase nz)
               (type (string 1) transn transt)
               (type f2cl-lib:logical notran))
      (setf info 0)
      (setf notran
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "N")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val))
      (cond
       ((and (not notran)
```

```
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame trans "T")
           (declare (ignore var-1))
           (when var-0 (setf trans var-0))
           ret-val))
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame trans "C")
           (declare (ignore var-1))
           (when var-0 (setf trans var-0))
           ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -13))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -15)))
(cond
 ((/= info 0) (xerbla "DGTRFS" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%) zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) zero)
     label10))
  (go end_label)))
(cond
 (notran (f2cl-lib:f2cl-set-string transn "N" (string 1))
  (f2cl-lib:f2cl-set-string transt "T" (string 1)))
 (t (f2cl-lib:f2cl-set-string transn "T" (string 1))
  (f2cl-lib:f2cl-set-string transt "N" (string 1))))
(setf nz 4)
(setf eps (dlamch "Epsilon"))
(setf safmin (dlamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy n
         (f2cl-lib:array-slice b-%data% double-float (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
```

```
     1
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                           ((1 *)) work-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (dlagtm trans n 1 (- one) dl d du
     (f2cl-lib:array-slice x-%data% double-float (1 j)
                           ((1 ldx) (1 *)) x-%offset%)
     ldx one
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                           ((1 *)) work-%offset%)
     n)
  (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-10))
  (when var-0 (setf trans var-0))
  (when var-1 (setf n var-1))
  (when var-8 (setf ldx var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf n var-11)))
(cond
 (notran
  (cond
   ((= n 1)
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (+
             (abs
              (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                             b-%offset%))
             (abs
              (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
                 (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                                x-%offset%))))))
   (t
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (+
             (abs
              (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                             b-%offset%))
             (abs
              (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
                 (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                                x-%offset%)))
             (abs
              (* (f2cl-lib:fref du-%data% (1) ((1 *)) du-%offset%)
                 (f2cl-lib:fref x-%data% (2 j) ((1 ldx) (1 *))
                                x-%offset%)))))
    (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
```

```
                    ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                     nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (+
                   (abs
                    (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%))
                   (abs
                    (*
                     (f2cl-lib:fref dl-%data%
                                    ((f2cl-lib:int-sub i 1)) ((1 *))
                                    dl-%offset%)
                     (f2cl-lib:fref x-%data%
                                    ((f2cl-lib:int-sub i 1) j)
                                    ((1 ldx) (1 *)) x-%offset%)))
                   (abs
                    (*
                     (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                     (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                    x-%offset%)))
                   (abs
                    (*
                     (f2cl-lib:fref du-%data% (i) ((1 *))
                                    du-%offset%)
                     (f2cl-lib:fref x-%data%
                                    ((f2cl-lib:int-add i 1) j)
                                    ((1 ldx) (1 *)) x-%offset%)))))
           label30))
        (setf (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
              (+
               (abs
                (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                               b-%offset%))
               (abs
                (*
                 (f2cl-lib:fref dl-%data% ((f2cl-lib:int-sub n 1))
                                ((1 *)) dl-%offset%)
                 (f2cl-lib:fref x-%data% ((f2cl-lib:int-sub n 1) j)
                                ((1 ldx) (1 *)) x-%offset%)))
               (abs
                (* (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                   (f2cl-lib:fref x-%data% (n j) ((1 ldx) (1 *))
                                  x-%offset%))))))))
      (t
       (cond
        ((= n 1)
         (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
               (+
                (abs
```

```
                (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                               b-%offset%))
               (abs
                (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
                   (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                                  x-%offset%))))))
(t
 (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
         (+
          (abs
           (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                          b-%offset%))
          (abs
           (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
              (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                             x-%offset%)))
          (abs
           (* (f2cl-lib:fref dl-%data% (1) ((1 *)) dl-%offset%)
              (f2cl-lib:fref x-%data% (2 j) ((1 ldx) (1 *))
                             x-%offset%)))))
 (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
               ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
             (+
              (abs
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%))
              (abs
               (*
                (f2cl-lib:fref du-%data%
                               ((f2cl-lib:int-sub i 1)) ((1 *))
                               du-%offset%)
                (f2cl-lib:fref x-%data%
                               ((f2cl-lib:int-sub i 1) j)
                               ((1 ldx) (1 *)) x-%offset%)))
              (abs
               (*
                (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                               x-%offset%)))
              (abs
               (*
                (f2cl-lib:fref dl-%data% (i) ((1 *))
                               dl-%offset%)
                (f2cl-lib:fref x-%data%
                               ((f2cl-lib:int-add i 1) j)
                               ((1 ldx) (1 *)) x-%offset%)))))
    label40))
```

```
    (setf (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
            (+
             (abs
              (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                             b-%offset%))
             (abs
              (*
               (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub n 1))
                              ((1 *)) du-%offset%)
               (f2cl-lib:fref x-%data% ((f2cl-lib:int-sub n 1) j)
                              ((1 ldx) (1 *)) x-%offset%)))
             (abs
              (* (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                 (f2cl-lib:fref x-%data% (n j) ((1 ldx) (1 *))
                                x-%offset%)))))))))
(setf s zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
      (setf s
              (max s
                   (/
                    (abs
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add n i))
                                    ((1 *)) work-%offset%))
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)))))
     (t
      (setf s
              (max s
                   (/
                    (+
                     (abs
                      (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-add n i))
                                     ((1 *)) work-%offset%))
                     safe1)
                    (+
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)
                     safe1))))))
   label50))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
```

```
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dgttrs trans n 1 dlf df duf du2 ipiv
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 *)) work-%offset%)
       n info)
    (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-8))
    (when var-0 (setf trans var-0))
    (when var-1 (setf n var-1))
    (when var-9 (setf n var-9))
    (when var-10 (setf info var-10)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (daxpy n one
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 *)) work-%offset%)
       1
       (f2cl-lib:array-slice x-%data% double-float (1 j)
                             ((1 ldx) (1 *)) x-%offset%)
       1)
    (declare (ignore var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))
    (when var-1 (setf one var-1)))
  (setf lstres
          (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
  (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (+
               (abs
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%))
               (* nz eps
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)))))
     (t
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (+
               (abs
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%))
               (* nz eps
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%))
               safe1))))
   label60))
```

```
 (setf kase 0)
label70
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlacn2 n
      (f2cl-lib:array-slice work-%data% double-float
                            ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                            ((1 *)) work-%offset%)
      iwork
      (f2cl-lib:array-slice ferr-%data% double-float (j) ((1 *))
                            ferr-%offset%)
      kase isave)
   (declare (ignore var-1 var-2 var-3 var-4 var-6))
   (when var-0 (setf n var-0))
   (when var-5 (setf kase var-5)))
 (cond
  ((/= kase 0)
   (cond
    ((= kase 1)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
          var-9 var-10)
         (dgttrs transt n 1 dlf df duf du2 ipiv
          (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                                ((1 *)) work-%offset%)
          n info)
       (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-8))
       (when var-0 (setf transt var-0))
       (when var-1 (setf n var-1))
       (when var-9 (setf n var-9))
       (when var-10 (setf info var-10)))
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                              ((1 *)) work-%offset%)
                (*
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add n i)) ((1 *))
                                work-%offset%)))
        label80)))
    (t
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                              ((1 *)) work-%offset%)
```

```
                          (*
                           (f2cl-lib:fref work-%data% (i) ((1 *))
                                          work-%offset%)
                           (f2cl-lib:fref work-%data%
                                          ((f2cl-lib:int-add n i)) ((1 *))
                                          work-%offset%)))
                   label90))
               (multiple-value-bind
                   (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                    var-9 var-10)
                   (dgttrs transn n 1 dlf df duf du2 ipiv
                    (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                                          ((1 *)) work-%offset%)
                    n info)
                 (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-8))
                 (when var-0 (setf transn var-0))
                 (when var-1 (setf n var-1))
                 (when var-9 (setf n var-9))
                 (when var-10 (setf info var-10)))))
             (go label70)))
           (setf lstres zero)
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf lstres
                       (max lstres
                            (abs
                             (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                            x-%offset%))))
              label100))
           (if (/= lstres zero)
               (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                       (/
                        (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                        lstres)))
          label110))
        (go end_label)
      end_label
        (return
         (values trans n nil nil nil nil nil nil nil nil nil nil nil ldx
                 nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgtrfs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
```

```
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::trans
  fortran-to-lisp::n nil nil nil
  nil nil nil nil nil nil nil nil
  nil fortran-to-lisp::ldx nil nil
  nil nil fortran-to-lisp::info)
:calls 'nil)))
```

———-

78.3 dgttrf

78.3.1 Axiom unit tests

— dgttrf.input —

```
)set break resume
)sys rm -f dgttrf.output
)spool dgttrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

78.3.2 Axiom help page

— dgttrf.help —

```
=====================================================================
dgttrf examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgttrf (N, DL, D, DU, DU2, IPIV, INFO)
           DGTTRF

Function/Subroutine Documentation
   subroutine dgttrf (integer N, double precision, dimension( * ) DL, double
       precision, dimension( * ) D, double precision, dimension( * ) DU,
       double precision, dimension( * ) DU2, integer, dimension( * ) IPIV,
       integer INFO)
       DGTTRF

Purpose:
   DGTTRF computes an LU factorization of a real tridiagonal matrix A
   using elimination with partial pivoting and row interchanges.

   The factorization has the form
      A = L * U
   where L is a product of permutation and unit lower bidiagonal
   matrices and U is upper triangular with nonzeros in only the main
   diagonal and first two superdiagonals.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.

 DL
   DL is DOUBLE PRECISION array, dimension (N-1)
   On entry, DL must contain the (n-1) sub-diagonal elements of A.

   On exit, DL is overwritten by the (n-1) multipliers that
   define the matrix L from the LU factorization of A.
```

```
D
  D is DOUBLE PRECISION array, dimension (N)
  On entry, D must contain the diagonal elements of A.

  On exit, D is overwritten by the n diagonal elements of the
  upper triangular matrix U from the LU factorization of A.

DU
  DU is DOUBLE PRECISION array, dimension (N-1)
  On entry, DU must contain the (n-1) super-diagonal elements of A.

  On exit, DU is overwritten by the (n-1) elements of the first
  super-diagonal of U.

DU2
  DU2 is DOUBLE PRECISION array, dimension (N-2)
  On exit, DU2 is overwritten by the (n-2) elements of the
  second super-diagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= n, row i of the matrix was
  interchanged with row IPIV(i).  IPIV(i) will always be either
  i or i+1; IPIV(i) = i indicates a row interchange was not required.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -k, the k-th argument had an illegal value
  > 0:  if INFO = k, U(k,k) is exactly zero. The factorization
        has been completed, but the factor U is exactly
        singular, and division by zero will occur if it is used
        to solve a system of equations.
```

———-

78.3.3 fortran code

— dgttrf.f —

```
*  =====================================================================
      SUBROUTINE DGTTRF( N, DL, D, DU, DU2, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
```

```
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   D( * ), DL( * ), DU( * ), DU2( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      DOUBLE PRECISION   FACT, TEMP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
         CALL XERBLA( 'DGTTRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Initialize IPIV(i) = i and DU2(I) = 0
*
      DO 10 I = 1, N
         IPIV( I ) = I
   10 CONTINUE
      DO 20 I = 1, N - 2
         DU2( I ) = ZERO
   20 CONTINUE
*
      DO 30 I = 1, N - 2
```

```
         IF( ABS( D( I ) ).GE.ABS( DL( I ) ) ) THEN
*
*           No row interchange required, eliminate DL(I)
*
            IF( D( I ).NE.ZERO ) THEN
               FACT = DL( I ) / D( I )
               DL( I ) = FACT
               D( I+1 ) = D( I+1 ) - FACT*DU( I )
            END IF
         ELSE
*
*           Interchange rows I and I+1, eliminate DL(I)
*
            FACT = D( I ) / DL( I )
            D( I ) = DL( I )
            DL( I ) = FACT
            TEMP = DU( I )
            DU( I ) = D( I+1 )
            D( I+1 ) = TEMP - FACT*D( I+1 )
            DU2( I ) = DU( I+1 )
            DU( I+1 ) = -FACT*DU( I+1 )
            IPIV( I ) = I + 1
         END IF
   30 CONTINUE
      IF( N.GT.1 ) THEN
         I = N - 1
         IF( ABS( D( I ) ).GE.ABS( DL( I ) ) ) THEN
            IF( D( I ).NE.ZERO ) THEN
               FACT = DL( I ) / D( I )
               DL( I ) = FACT
               D( I+1 ) = D( I+1 ) - FACT*DU( I )
            END IF
         ELSE
            FACT = D( I ) / DL( I )
            D( I ) = DL( I )
            DL( I ) = FACT
            TEMP = DU( I )
            DU( I ) = D( I+1 )
            D( I+1 ) = TEMP - FACT*D( I+1 )
            IPIV( I ) = I + 1
         END IF
      END IF
*
*     Check for a zero on the diagonal of U.
*
      DO 40 I = 1, N
         IF( D( I ).EQ.ZERO ) THEN
            INFO = I
            GO TO 50
         END IF
```

```
   40 CONTINUE
   50 CONTINUE
*
      RETURN
*
*     End of DGTTRF
*
      END
```

78.3.4 lisp code

— **dgttrf.lisp** —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun dgttrf (n dl d du du2 ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array double-float (*)) du2 du d dl)
             (type (f2cl-lib:integer4) info n))
    (f2cl-lib:with-multi-array-data
        ((dl double-float dl-%data% dl-%offset%)
         (d double-float d-%data% d-%offset%)
         (du double-float du-%data% du-%offset%)
         (du2 double-float du2-%data% du2-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((fact 0.0d0) (temp 0.0d0) (i 0))
        (declare (type (double-float) fact temp)
                 (type (f2cl-lib:integer4) i))
        (setf info 0)
        (cond
         ((< n 0) (setf info -1) (xerbla "DGTTRF" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (= n 0)
            (go end_label))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) i)
           label10))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))) nil)
          (tagbody
            (setf (f2cl-lib:fref du2-%data% (i) ((1 *)) du2-%offset%) zero)
           label20))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
              ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))) nil)
  (tagbody
    (cond
     ((>= (abs (f2cl-lib:fref d (i) ((1 *))))
          (abs (f2cl-lib:fref dl (i) ((1 *)))))
      (cond
       ((/= (f2cl-lib:fref d (i) ((1 *))) zero)
        (setf fact
                (/ (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                   (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
        (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) fact)
        (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                              d-%offset%)
                (-
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) d-%offset%)
                 (* fact
                    (f2cl-lib:fref du-%data% (i) ((1 *))
                                    du-%offset%)))))))
     (t
      (setf fact
              (/ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                 (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
      (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
              (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
      (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) fact)
      (setf temp (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%))
      (setf (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
              (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                              d-%offset%))
      (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                            d-%offset%)
              (- temp
                 (* fact
                    (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                    ((1 *)) d-%offset%))))
      (setf (f2cl-lib:fref du2-%data% (i) ((1 *)) du2-%offset%)
              (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                              du-%offset%))
      (setf (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                            du-%offset%)
              (* (- fact)
                 (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) du-%offset%)))
      (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
              (f2cl-lib:int-add i 1))))
   label30))
 (cond
  ((> n 1) (setf i (f2cl-lib:int-sub n 1))
   (cond
```

```
          ((>= (abs (f2cl-lib:fref d (i) ((1 *))))
               (abs (f2cl-lib:fref dl (i) ((1 *)))))
           (cond
            ((/= (f2cl-lib:fref d (i) ((1 *))) zero)
             (setf fact
                     (/ (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                        (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
             (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) fact)
             (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                   d-%offset%)
                     (-
                      (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                     d-%offset%)
                      (* fact
                         (f2cl-lib:fref du-%data% (i) ((1 *))
                                        du-%offset%)))))))
          (t
           (setf fact
                   (/ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                      (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
           (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                   (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
           (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) fact)
           (setf temp (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%))
           (setf (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                   (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                   d-%offset%))
           (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                 d-%offset%)
                   (- temp
                      (* fact
                         (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                        ((1 *)) d-%offset%))))
           (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
                   (f2cl-lib:int-add i 1))))))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (cond
             ((= (f2cl-lib:fref d (i) ((1 *))) zero) (setf info i)
              (go label50)))
           label40))
       label50
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
```

```
(setf (gethash 'fortran-to-lisp::dgttrf
               fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((fortran-to-lisp::integer4)
                                            (array double-float (*))
                                            (array double-float (*))
                                            (array double-float (*))
                                            (array double-float (*))
                                            (array fortran-to-lisp::integer4
                                             (*))
                                            (fortran-to-lisp::integer4))
                                          :return-values
                                          '(nil nil nil nil nil nil
                                            fortran-to-lisp::info)
                                          :calls 'nil)))
```

———-

78.4 dgttrs

78.4.1 Axiom unit tests

— **dgttrs.input** —

```
)set break resume
)sys rm -f dgttrs.output
)spool dgttrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

78.4.2 Axiom help page

— **dgttrs.help** —

```
==================================================================
dgttrs examples
```

```
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgttrs (TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB, INFO)
           DGTTRS

Function/Subroutine Documentation
   subroutine dgttrs (character TRANS, integer N, integer NRHS, double
       precision, dimension( * ) DL, double precision, dimension( * ) D,
       double precision, dimension( * ) DU, double precision, dimension( * )
       DU2, integer, dimension( * ) IPIV, double precision, dimension( ldb, *
       ) B, integer LDB, integer INFO)
       DGTTRS

Purpose:
   DGTTRS solves one of the systems of equations
      A*X = B  or  A**T*X = B,
   with a tridiagonal matrix A using the LU factorization computed
   by DGTTRF.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations.
   = 'N':  A * X = B  (No transpose)
   = 'T':  A**T* X = B  (Transpose)
   = 'C':  A**T* X = B  (Conjugate transpose = Transpose)

 N
   N is INTEGER
   The order of the matrix A.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 DL
   DL is DOUBLE PRECISION array, dimension (N-1)
   The (n-1) multipliers that define the matrix L from the
   LU factorization of A.

 D
   D is DOUBLE PRECISION array, dimension (N)
```

```
  The n diagonal elements of the upper triangular matrix U from
  the LU factorization of A.

DU
  DU is DOUBLE PRECISION array, dimension (N-1)
  The (n-1) elements of the first super-diagonal of U.

DU2
  DU2 is DOUBLE PRECISION array, dimension (N-2)
  The (n-2) elements of the second super-diagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= n, row i of the matrix was
  interchanged with row IPIV(i).  IPIV(i) will always be either
  i or i+1; IPIV(i) = i indicates a row interchange was not required.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the matrix of right hand side vectors B.
  On exit, B is overwritten by the solution vectors X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

78.4.3 fortran code

— dgttrs.f —

```
*  =====================================================================
      SUBROUTINE DGTTRS( TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
```

```
      CHARACTER          TRANS
      INTEGER            INFO, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   B( LDB, * ), D( * ), DL( * ), DU( * ), DU2( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      INTEGER            ITRANS, J, JB, NB
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGTTS2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOTRAN = ( TRANS.EQ.'N' .OR. TRANS.EQ.'n' )
      IF( .NOT.NOTRAN .AND. .NOT.( TRANS.EQ.'T' .OR. TRANS.EQ.
     $    't' ) .AND. .NOT.( TRANS.EQ.'C' .OR. TRANS.EQ.'c' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDB.LT.MAX( N, 1 ) ) THEN
         INFO = -10
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGTTRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
*     Decode TRANS
*
```

```
      IF( NOTRAN ) THEN
         ITRANS = 0
      ELSE
         ITRANS = 1
      END IF
*
*     Determine the number of right-hand sides to solve at a time.
*
      IF( NRHS.EQ.1 ) THEN
         NB = 1
      ELSE
         NB = MAX( 1, ILAENV( 1, 'DGTTRS', TRANS, N, NRHS, -1, -1 ) )
      END IF
*
      IF( NB.GE.NRHS ) THEN
         CALL DGTTS2( ITRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB )
      ELSE
         DO 10 J = 1, NRHS, NB
            JB = MIN( NRHS-J+1, NB )
            CALL DGTTS2( ITRANS, N, JB, DL, D, DU, DU2, IPIV, B( 1, J ),
     $                   LDB )
   10    CONTINUE
      END IF
*
*     End of DGTTRS
*
      END
```

———-

78.4.4 lisp code

— **dgttrs.lisp** —

```
(defun dgttrs (trans n nrhs dl d du du2 ipiv b ldb$ info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array double-float (*)) b du2 du d dl)
           (type (f2cl-lib:integer4) info ldb$ nrhs n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (dl double-float dl-%data% dl-%offset%)
       (d double-float d-%data% d-%offset%)
       (du double-float du-%data% du-%offset%)
       (du2 double-float du2-%data% du2-%offset%)
       (b double-float b-%data% b-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((itrans 0) (j 0) (jb 0) (nb 0) (notran nil))
```

```
(declare (type f2cl-lib:logical notran)
         (type (f2cl-lib:integer4) nb jb j itrans))
(setf info 0)
(setf notran
        (or (f2cl-lib:fstring-= trans "N")
            (f2cl-lib:fstring-= trans "n")))
(cond
 ((and (not notran)
       (not
        (or (f2cl-lib:fstring-= trans "T")
            (f2cl-lib:fstring-= trans "t")))
       (not
        (or (f2cl-lib:fstring-= trans "C")
            (f2cl-lib:fstring-= trans "c"))))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< ldb$ (max (the f2cl-lib:integer4 n) (the f2cl-lib:integer4 1)))
  (setf info -10)))
(cond
 ((/= info 0) (xerbla "DGTTRS" (f2cl-lib:int-sub info)) (go end_label)))
(if (or (= n 0) (= nrhs 0))
    (go end_label))
(cond (notran (setf itrans 0)) (t (setf itrans 1)))
(cond ((= nrhs 1) (setf nb 1))
      (t
       (setf nb
               (max (the f2cl-lib:integer4 1)
                    (the f2cl-lib:integer4
                         (multiple-value-bind
                             (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                              var-6)
                             (ilaenv 1 "DGTTRS" trans n nrhs -1 -1)
                           (declare (ignore var-0 var-1 var-5 var-6))
                           (when var-2 (setf trans var-2))
                           (when var-3 (setf n var-3))
                           (when var-4 (setf nrhs var-4))
                           ret-val))))))
(cond
 ((>= nb nrhs)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dgtts2 itrans n nrhs dl d du du2 ipiv b ldb$)
    (declare (ignore var-3 var-4 var-5 var-6 var-7 var-8))
    (when var-0 (setf itrans var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf nrhs var-2))
    (when var-9 (setf ldb$ var-9))))
 (t
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                ((> j nrhs) nil)
```

```
          (tagbody
            (setf jb
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-sub nrhs j) 1))
                     (the f2cl-lib:integer4 nb)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                (dgtts2 itrans n jb dl d du du2 ipiv
                 (f2cl-lib:array-slice b-%data% double-float (1 j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$)
              (declare (ignore var-3 var-4 var-5 var-6 var-7 var-8))
              (when var-0 (setf itrans var-0))
              (when var-1 (setf n var-1))
              (when var-2 (setf jb var-2))
              (when var-9 (setf ldb$ var-9)))
           label10))))
     end_label
      (return (values trans n nrhs nil nil nil nil nil nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgttrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil nil nil
                                              nil nil nil fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

78.5 dgtts2

78.5.1 Axiom unit tests

— **dgtts2.input** —

```
)set break resume
)sys rm -f dgtts2.output
)spool dgtts2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

78.5.2 Axiom help page

— **dgtts2.help** —

```
======================================================================
dgtts2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dgtts2 (ITRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB)
           DGTTS2 solves a system of linear equations with a tridiagonal
           matrix using the LU factorization computed by sgttrf.

Function/Subroutine Documentation
   subroutine dgtts2 (integer ITRANS, integer N, integer NRHS, double
       precision, dimension( * ) DL, double precision, dimension( * ) D,
       double precision, dimension( * ) DU, double precision, dimension( * )
       DU2, integer, dimension( * ) IPIV, double precision, dimension( ldb, *
       ) B, integer LDB)
       DGTTS2 solves a system of linear equations with a tridiagonal matrix
       using the LU factorization computed by sgttrf.

Purpose:
```

```
  DGTTS2 solves one of the systems of equations
     A*X = B  or  A**T*X = B,
  with a tridiagonal matrix A using the LU factorization computed by DGTTRF.

Parameters:

 ITRANS
   ITRANS is INTEGER
   Specifies the form of the system of equations.
   = 0:  A * X = B  (No transpose)
   = 1:  A**T* X = B  (Transpose)
   = 2:  A**T* X = B  (Conjugate transpose = Transpose)

 N
   N is INTEGER
   The order of the matrix A.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 DL
   DL is DOUBLE PRECISION array, dimension (N-1)
   The (n-1) multipliers that define the matrix L from the
   LU factorization of A.

 D
   D is DOUBLE PRECISION array, dimension (N)
   The n diagonal elements of the upper triangular matrix U from
   the LU factorization of A.

 DU
   DU is DOUBLE PRECISION array, dimension (N-1)
   The (n-1) elements of the first super-diagonal of U.

 DU2
   DU2 is DOUBLE PRECISION array, dimension (N-2)
   The (n-2) elements of the second super-diagonal of U.

 IPIV
   IPIV is INTEGER array, dimension (N)
   The pivot indices; for 1 <= i <= n, row i of the matrix was
   interchanged with row IPIV(i).  IPIV(i) will always be either
   i or i+1; IPIV(i) = i indicates a row interchange was not required.

 B
   B is DOUBLE PRECISION array, dimension (LDB,NRHS)
   On entry, the matrix of right hand side vectors B.
   On exit, B is overwritten by the solution vectors X.
```

```
LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).
```

———-

78.5.3 fortran code

— **dgtts2.f** —

```
*  =====================================================================
      SUBROUTINE DGTTS2( ITRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            ITRANS, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   B( LDB, * ), D( * ), DL( * ), DU( * ), DU2( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, IP, J
      DOUBLE PRECISION   TEMP
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( ITRANS.EQ.0 ) THEN
*
*        Solve A*X = B using the LU factorization of A,
*        overwriting each right hand side vector with its solution.
*
         IF( NRHS.LE.1 ) THEN
            J = 1
```

```
   10       CONTINUE
*
*           Solve L*x = b.
*
            DO 20 I = 1, N - 1
               IP = IPIV( I )
               TEMP = B( I+1-IP+I, J ) - DL( I )*B( IP, J )
               B( I, J ) = B( IP, J )
               B( I+1, J ) = TEMP
   20       CONTINUE
*
*           Solve U*x = b.
*
            B( N, J ) = B( N, J ) / D( N )
            IF( N.GT.1 )
     $         B( N-1, J ) = ( B( N-1, J )-DU( N-1 )*B( N, J ) ) /
     $                       D( N-1 )
            DO 30 I = N - 2, 1, -1
               B( I, J ) = ( B( I, J )-DU( I )*B( I+1, J )-DU2( I )*
     $                     B( I+2, J ) ) / D( I )
   30       CONTINUE
            IF( J.LT.NRHS ) THEN
               J = J + 1
               GO TO 10
            END IF
         ELSE
            DO 60 J = 1, NRHS
*
*              Solve L*x = b.
*
               DO 40 I = 1, N - 1
                  IF( IPIV( I ).EQ.I ) THEN
                     B( I+1, J ) = B( I+1, J ) - DL( I )*B( I, J )
                  ELSE
                     TEMP = B( I, J )
                     B( I, J ) = B( I+1, J )
                     B( I+1, J ) = TEMP - DL( I )*B( I, J )
                  END IF
   40          CONTINUE
*
*              Solve U*x = b.
*
               B( N, J ) = B( N, J ) / D( N )
               IF( N.GT.1 )
     $            B( N-1, J ) = ( B( N-1, J )-DU( N-1 )*B( N, J ) ) /
     $                          D( N-1 )
               DO 50 I = N - 2, 1, -1
                  B( I, J ) = ( B( I, J )-DU( I )*B( I+1, J )-DU2( I )*
     $                        B( I+2, J ) ) / D( I )
   50          CONTINUE
```

```
   60       CONTINUE
         END IF
      ELSE
*
*        Solve A**T * X = B.
*
         IF( NRHS.LE.1 ) THEN
*
*           Solve U**T*x = b.
*
            J = 1
   70       CONTINUE
            B( 1, J ) = B( 1, J ) / D( 1 )
            IF( N.GT.1 )
     $         B( 2, J ) = ( B( 2, J )-DU( 1 )*B( 1, J ) ) / D( 2 )
            DO 80 I = 3, N
               B( I, J ) = ( B( I, J )-DU( I-1 )*B( I-1, J )-DU2( I-2 )*
     $                     B( I-2, J ) ) / D( I )
   80       CONTINUE
*
*           Solve L**T*x = b.
*
            DO 90 I = N - 1, 1, -1
               IP = IPIV( I )
               TEMP = B( I, J ) - DL( I )*B( I+1, J )
               B( I, J ) = B( IP, J )
               B( IP, J ) = TEMP
   90       CONTINUE
            IF( J.LT.NRHS ) THEN
               J = J + 1
               GO TO 70
            END IF
*
         ELSE
            DO 120 J = 1, NRHS
*
*              Solve U**T*x = b.
*
               B( 1, J ) = B( 1, J ) / D( 1 )
               IF( N.GT.1 )
     $            B( 2, J ) = ( B( 2, J )-DU( 1 )*B( 1, J ) ) / D( 2 )
               DO 100 I = 3, N
                  B( I, J ) = ( B( I, J )-DU( I-1 )*B( I-1, J )-
     $                        DU2( I-2 )*B( I-2, J ) ) / D( I )
  100          CONTINUE
               DO 110 I = N - 1, 1, -1
                  IF( IPIV( I ).EQ.I ) THEN
                     B( I, J ) = B( I, J ) - DL( I )*B( I+1, J )
                  ELSE
                     TEMP = B( I+1, J )
```

```
                    B( I+1, J ) = B( I, J ) - DL( I )*TEMP
                    B( I, J ) = TEMP
                 END IF
  110          CONTINUE
  120       CONTINUE
         END IF
      END IF
*
*     End of DGTTS2
*
      END
```

———-

78.5.4 lisp code

— dgtts2.lisp —

```
(defun dgtts2 (itrans n nrhs dl d du du2 ipiv b ldb$)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array double-float (*)) b du2 du d dl)
           (type (f2cl-lib:integer4) ldb$ nrhs n itrans))
  (f2cl-lib:with-multi-array-data
      ((dl double-float dl-%data% dl-%offset%)
       (d double-float d-%data% d-%offset%)
       (du double-float du-%data% du-%offset%)
       (du2 double-float du2-%data% du2-%offset%)
       (b double-float b-%data% b-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((temp 0.0d0) (i 0) (ip 0) (j 0))
      (declare (type (f2cl-lib:integer4) j ip i)
               (type (double-float) temp))
      (if (or (= n 0) (= nrhs 0))
          (go end_label))
      (cond
       ((= itrans 0)
        (cond
         ((<= nrhs 1)
          (tagbody
            (setf j 1)
           label10
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
              (tagbody
                (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
                (setf temp
                        (-
                         (f2cl-lib:fref b-%data%
```

```
                                ((f2cl-lib:int-add
                                  (f2cl-lib:int-sub
                                   (f2cl-lib:int-add i 1) ip)
                                  i)
                                 j)
                                ((1 ldb$) (1 *)) b-%offset%)
                 (* (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                    (f2cl-lib:fref b-%data% (ip j) ((1 ldb$) (1 *))
                                   b-%offset%))))
    (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                         b-%offset%)
            (f2cl-lib:fref b-%data% (ip j) ((1 ldb$) (1 *))
                           b-%offset%))
    (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                         ((1 ldb$) (1 *)) b-%offset%)
            temp)
   label20))
(setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
        (/
         (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
         (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
(if (> n 1)
    (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                         ((1 ldb$) (1 *)) b-%offset%)
            (/
             (-
              (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)
              (*
               (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub n 1))
                              ((1 *)) du-%offset%)
               (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                              b-%offset%)))
             (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                            ((1 *)) d-%offset%))))
(f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))
               (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                         b-%offset%)
            (/
             (-
              (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)
              (* (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                ((1 ldb$) (1 *)) b-%offset%))
              (*
               (f2cl-lib:fref du2-%data% (i) ((1 *)) du2-%offset%)
```

```
                    (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 2) j)
                                   ((1 ldb$) (1 *)) b-%offset%)))
                 (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
      label30))
   (cond ((< j nrhs) (setf j (f2cl-lib:int-add j 1)) (go label10)))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j nrhs) nil)
   (tagbody
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                    nil)
       (tagbody
         (cond
          ((= (f2cl-lib:fref ipiv (i) ((1 *))) i)
           (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                ((1 ldb$) (1 *)) b-%offset%)
                   (-
                    (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                   ((1 ldb$) (1 *)) b-%offset%)
                    (*
                     (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)))))
          (t
           (setf temp
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%))
           (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                   (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                  ((1 ldb$) (1 *)) b-%offset%))
           (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                ((1 ldb$) (1 *)) b-%offset%)
                   (- temp
                      (*
                       (f2cl-lib:fref dl-%data% (i) ((1 *))
                                      dl-%offset%)
                       (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                      b-%offset%))))))
         label40))
     (setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
             (/
              (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                             b-%offset%)
              (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
     (if (> n 1)
         (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                               ((1 ldb$) (1 *)) b-%offset%)
                 (/
```

```
                    (-
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                                    ((1 ldb$) (1 *)) b-%offset%)
                     (*
                      (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub n 1))
                                     ((1 *)) du-%offset%)
                      (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                                     b-%offset%)))
                    (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                   ((1 *)) d-%offset%))))
        (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))
                       (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (/
                   (-
                    (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)
                    (*
                     (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                    ((1 ldb$) (1 *)) b-%offset%))
                    (*
                     (f2cl-lib:fref du2-%data% (i) ((1 *))
                                    du2-%offset%)
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 2) j)
                                    ((1 ldb$) (1 *)) b-%offset%)))
                   (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
           label50))
      label60)))))
(t
 (cond
  ((<= nrhs 1)
   (tagbody
     (setf j 1)
    label70
     (setf (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *)) b-%offset%)
           (/
            (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *)) b-%offset%)
            (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)))
     (if (> n 1)
         (setf (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                              b-%offset%)
               (/
                (-
                 (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                                b-%offset%)
                 (* (f2cl-lib:fref du-%data% (1) ((1 *)) du-%offset%)
```

```
                    (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                                   b-%offset%)))
                (f2cl-lib:fref d-%data% (2) ((1 *)) d-%offset%))))
    (f2cl-lib:fdo (i 3 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)
              (/
               (-
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)
                (*
                 (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub i 1))
                                ((1 *)) du-%offset%)
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                ((1 ldb$) (1 *)) b-%offset%))
                (*
                 (f2cl-lib:fref du2-%data% ((f2cl-lib:int-sub i 2))
                                ((1 *)) du2-%offset%)
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 2) j)
                                ((1 ldb$) (1 *)) b-%offset%)))
               (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
       label80))
    (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                  (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
      (tagbody
        (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
        (setf temp
              (-
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)
               (* (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                  (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%))))
        (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)
              (f2cl-lib:fref b-%data% (ip j) ((1 ldb$) (1 *))
                             b-%offset%))
        (setf (f2cl-lib:fref b-%data% (ip j) ((1 ldb$) (1 *))
                             b-%offset%)
              temp)
       label90))
    (cond ((< j nrhs) (setf j (f2cl-lib:int-add j 1)) (go label70)))))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *)) b-%offset%)
```

```
        (/
         (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                        b-%offset%)
         (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)))
(if (> n 1)
    (setf (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                         b-%offset%)
            (/
             (-
              (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                             b-%offset%)
              (*
               (f2cl-lib:fref du-%data% (1) ((1 *)) du-%offset%)
               (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                              b-%offset%)))
             (f2cl-lib:fref d-%data% (2) ((1 *)) d-%offset%))))
(f2cl-lib:fdo (i 3 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                         b-%offset%)
            (/
             (-
              (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)
              (*
               (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub i 1))
                              ((1 *)) du-%offset%)
               (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                              ((1 ldb$) (1 *)) b-%offset%))
              (*
               (f2cl-lib:fref du2-%data% ((f2cl-lib:int-sub i 2))
                              ((1 *)) du2-%offset%)
               (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 2) j)
                              ((1 ldb$) (1 *)) b-%offset%)))
             (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
   label100))
(f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (cond
     ((= (f2cl-lib:fref ipiv (i) ((1 *))) i)
      (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                           b-%offset%)
              (-
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)
               (*
                (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
```

```
                              (f2cl-lib:fref b-%data%
                                             ((f2cl-lib:int-add i 1) j)
                                             ((1 ldb$) (1 *)) b-%offset%)))))
                     (t
                      (setf temp
                              (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                             ((1 ldb$) (1 *)) b-%offset%))
                      (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                           ((1 ldb$) (1 *)) b-%offset%)
                              (-
                               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                              b-%offset%)
                               (*
                                (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                                temp)))
                      (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                           b-%offset%)
                              temp)))
                 label110))
             label120))))))
     end_label
      (return (values nil nil nil nil nil nil nil nil nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dgtts2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
                                              nil nil)
                                            :calls 'nil)))
```

———-

Chapter 79

LAPACK - General Tridiagonal Matrix, Comp. Routines, Real

79.1 sgtcon

79.1.1 Axiom unit tests

— sgtcon.input —

```
)set break resume
)sys rm -f sgtcon.output
)spool sgtcon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

79.1.2 Axiom help page

— sgtcon.help —

```
======================================================================
sgtcon examples
======================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgtcon (NORM, N, DL, D, DU, DU2, IPIV, ANORM, RCOND, WORK,
           IWORK, INFO)
           SGTCON

Function/Subroutine Documentation
   subroutine sgtcon (character NORM, integer N, real, dimension( * ) DL,
       real, dimension( * ) D, real, dimension( * ) DU, real, dimension( * )
       DU2, integer, dimension( * ) IPIV, real ANORM, real RCOND, real,
       dimension( * ) WORK, integer, dimension( * ) IWORK, integer INFO)
       SGTCON

Purpose:
   SGTCON estimates the reciprocal of the condition number of a real
   tridiagonal matrix A using the LU factorization as computed by
   SGTTRF.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies whether the 1-norm condition number or the
   infinity-norm condition number is required:
   = '1' or 'O':  1-norm;
   = 'I':         Infinity-norm.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 DL
   DL is REAL array, dimension (N-1)
   The (n-1) multipliers that define the matrix L from the
   LU factorization of A as computed by SGTTRF.

 D
   D is REAL array, dimension (N)
   The n diagonal elements of the upper triangular matrix U from
   the LU factorization of A.

 DU
```

```
  DU is REAL array, dimension (N-1)
  The (n-1) elements of the first superdiagonal of U.

DU2
  DU2 is REAL array, dimension (N-2)
  The (n-2) elements of the second superdiagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= n, row i of the matrix was
  interchanged with row IPIV(i).  IPIV(i) will always be either
  i or i+1; IPIV(i) = i indicates a row interchange was not required.

ANORM
  ANORM is REAL
  If NORM = '1' or 'O', the 1-norm of the original matrix A.
  If NORM = 'I', the infinity-norm of the original matrix A.

RCOND
  RCOND is REAL
  The reciprocal of the condition number of the matrix A,
  computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is an
  estimate of the 1-norm of inv(A) computed in this routine.

WORK
  WORK is REAL array, dimension (2*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

79.1.3 fortran code

— **sgtcon.f** —

```
*  =====================================================================
      SUBROUTINE SGTCON( NORM, N, DL, D, DU, DU2, IPIV, ANORM, RCOND,
     $                   WORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
```

```
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            INFO, N
      REAL               ANORM, RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      REAL               D( * ), DL( * ), DU( * ), DU2( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ONENRM
      INTEGER            I, KASE, KASE1
      REAL               AINVNM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGTTRS, SLACN2, XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      ONENRM = NORM.EQ.'1' .OR. LSAME( NORM, 'O' )
      IF( .NOT.ONENRM .AND. .NOT.LSAME( NORM, 'I' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGTCON', -INFO )
         RETURN
```

```
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
*     Check that D(1:N) is non-zero.
*
      DO 10 I = 1, N
         IF( D( I ).EQ.ZERO )
     $      RETURN
   10 CONTINUE
*
      AINVNM = ZERO
      IF( ONENRM ) THEN
         KASE1 = 1
      ELSE
         KASE1 = 2
      END IF
      KASE = 0
   20 CONTINUE
      CALL SLACN2( N, WORK( N+1 ), WORK, IWORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.KASE1 ) THEN
*
*           Multiply by inv(U)*inv(L).
*
            CALL SGTTRS( 'No transpose', N, 1, DL, D, DU, DU2, IPIV,
     $                   WORK, N, INFO )
         ELSE
*
*           Multiply by inv(L**T)*inv(U**T).
*
            CALL SGTTRS( 'Transpose', N, 1, DL, D, DU, DU2, IPIV, WORK,
     $                   N, INFO )
         END IF
         GO TO 20
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
```

```
      RETURN
*
*     End of SGTCON
*
      END
```

———-

79.1.4 lisp code

— **sgtcon.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun sgtcon (norm n dl d du du2 ipiv anorm rcond work iwork info)
    (declare (type (single-float) rcond anorm)
             (type (array f2cl-lib:integer4 (*)) iwork ipiv)
             (type (array single-float (*)) work du2 du d dl)
             (type (f2cl-lib:integer4) info n)
             (type (string *) norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (dl single-float dl-%data% dl-%offset%)
         (d single-float d-%data% d-%offset%)
         (du single-float du-%data% du-%offset%)
         (du2 single-float du2-%data% du2-%offset%)
         (work single-float work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (ainvnm 0.0) (i 0) (kase 0) (kase1 0) (onenrm nil))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (single-float) ainvnm)
                 (type (f2cl-lib:integer4) i kase kase1)
                 (type f2cl-lib:logical onenrm))
        (setf info 0)
        (setf onenrm
                (or (f2cl-lib:fstring-= norm "1")
                    (multiple-value-bind (ret-val var-0 var-1)
                        (lsame norm "O")
                      (declare (ignore var-1))
                      (when var-0 (setf norm var-0))
                      ret-val)))
        (cond
         ((and (not onenrm)
               (not
```

```
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame norm "I")
            (declare (ignore var-1))
            (when var-0 (setf norm var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2)) ((< anorm zero) (setf info -8)))
  (cond
   ((/= info 0) (xerbla "SGTCON" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf rcond zero)
  (cond ((= n 0) (setf rcond one) (go end_label))
        ((= anorm zero) (go end_label)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (if (= (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) zero)
          (go end_label))
     label10))
  (setf ainvnm zero)
  (cond (onenrm (setf kase1 1)) (t (setf kase1 2)))
  (setf kase 0)
 label20
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slacn2 n
       (f2cl-lib:array-slice work-%data% single-float ((+ n 1)) ((1 *))
                             work-%offset%)
       work iwork ainvnm kase isave)
    (declare (ignore var-1 var-2 var-3 var-6))
    (when var-0 (setf n var-0))
    (when var-4 (setf ainvnm var-4))
    (when var-5 (setf kase var-5)))
  (cond
   ((/= kase 0)
    (cond
     ((= kase kase1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (sgttrs "No transpose" n 1 dl d du du2 ipiv work n info)
        (declare
         (ignore var-0 var-2 var-3 var-4 var-5 var-6 var-7 var-8))
        (when var-1 (setf n var-1))
        (when var-9 (setf n var-9))
        (when var-10 (setf info var-10))))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (sgttrs "Transpose" n 1 dl d du du2 ipiv work n info)
```

```
              (declare
               (ignore var-0 var-2 var-3 var-4 var-5 var-6 var-7 var-8))
              (when var-1 (setf n var-1))
              (when var-9 (setf n var-9))
              (when var-10 (setf info var-10)))))
          (go label20)))
        (if (/= ainvnm zero)
            (setf rcond (/ (/ one ainvnm) anorm)))
        (go end_label)
       end_label
        (return (values norm n nil nil nil nil nil nil rcond nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgtcon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (single-float) (single-float)
                                              (array single-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::norm
                                              fortran-to-lisp::n nil nil nil
                                              nil nil nil
                                              fortran-to-lisp::rcond nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

79.2 sgtrfs

79.2.1 Axiom unit tests

— sgtrfs.input —

```
)set break resume
)sys rm -f sgtrfs.output
)spool sgtrfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

79.2.2 Axiom help page

— sgtrfs.help —

```
======================================================================
sgtrfs examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgtrfs (TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF, DU2, IPIV,
           B, LDB, X, LDX, FERR, BERR, WORK, IWORK, INFO)
           SGTRFS

Function/Subroutine Documentation
   subroutine sgtrfs (character TRANS, integer N, integer NRHS, real,
       dimension( * ) DL, real, dimension( * ) D, real, dimension( * ) DU,
       real, dimension( * ) DLF, real, dimension( * ) DF, real, dimension( * )
       DUF, real, dimension( * ) DU2, integer, dimension( * ) IPIV, real,
       dimension( ldb, * ) B, integer LDB, real, dimension( ldx, * ) X,
       integer LDX, real, dimension( * ) FERR, real, dimension( * ) BERR,
       real, dimension( * ) WORK, integer, dimension( * ) IWORK, integer INFO)
       SGTRFS
```

```
Purpose:
   SGTRFS improves the computed solution to a system of linear
   equations when the coefficient matrix is tridiagonal, and provides
   error bounds and backward error estimates for the solution.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose = Transpose)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 DL
   DL is REAL array, dimension (N-1)
   The (n-1) subdiagonal elements of A.

 D
   D is REAL array, dimension (N)
   The diagonal elements of A.

 DU
   DU is REAL array, dimension (N-1)
   The (n-1) superdiagonal elements of A.

 DLF
   DLF is REAL array, dimension (N-1)
   The (n-1) multipliers that define the matrix L from the
   LU factorization of A as computed by SGTTRF.

 DF
   DF is REAL array, dimension (N)
   The n diagonal elements of the upper triangular matrix U from
   the LU factorization of A.

 DUF
   DUF is REAL array, dimension (N-1)
   The (n-1) elements of the first superdiagonal of U.

 DU2
```

```
  DU2 is REAL array, dimension (N-2)
  The (n-2) elements of the second superdiagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= n, row i of the matrix was
  interchanged with row IPIV(i).  IPIV(i) will always be either
  i or i+1; IPIV(i) = i indicates a row interchange was not required.

B
  B is REAL array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is REAL array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by SGTTRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

FERR
  FERR is REAL array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is REAL array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is REAL array, dimension (3*N)

IWORK
  IWORK is INTEGER array, dimension (N)

INFO
```

```
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

   ITMAX is the maximum number of steps of iterative refinement.
```

79.2.3 fortran code

— sgtrfs.f —

```
*  =====================================================================
      SUBROUTINE SGTRFS( TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF, DU2,
     $                   IPIV, B, LDB, X, LDX, FERR, BERR, WORK, IWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * ), IWORK( * )
      REAL               B( LDB, * ), BERR( * ), D( * ), DF( * ),
     $                   DL( * ), DLF( * ), DU( * ), DU2( * ), DUF( * ),
     $                   FERR( * ), WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               TWO
      PARAMETER          ( TWO = 2.0E+0 )
      REAL               THREE
      PARAMETER          ( THREE = 3.0E+0 )
*     ..
```

```
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      CHARACTER          TRANSN, TRANST
      INTEGER            COUNT, I, J, KASE, NZ
      REAL               EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SAXPY, SCOPY, SGTTRS, SLACN2, SLAGTM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH
      EXTERNAL           LSAME, SLAMCH
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -13
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -15
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGTRFS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
```

```
         RETURN
      END IF
*
      IF( NOTRAN ) THEN
         TRANSN = 'N'
         TRANST = 'T'
      ELSE
         TRANSN = 'T'
         TRANST = 'N'
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = 4
      EPS = SLAMCH( 'Epsilon' )
      SAFMIN = SLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 110 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - op(A) * X,
*        where op(A) = A, A**T, or A**H, depending on TRANS.
*
         CALL SCOPY( N, B( 1, J ), 1, WORK( N+1 ), 1 )
         CALL SLAGTM( TRANS, N, 1, -ONE, DL, D, DU, X( 1, J ), LDX, ONE,
     $                WORK( N+1 ), N )
*
*        Compute abs(op(A))*abs(x) + abs(b) for use in the backward
*        error bound.
*
         IF( NOTRAN ) THEN
            IF( N.EQ.1 ) THEN
               WORK( 1 ) = ABS( B( 1, J ) ) + ABS( D( 1 )*X( 1, J ) )
            ELSE
               WORK( 1 ) = ABS( B( 1, J ) ) + ABS( D( 1 )*X( 1, J ) ) +
     $                     ABS( DU( 1 )*X( 2, J ) )
               DO 30 I = 2, N - 1
                  WORK( I ) = ABS( B( I, J ) ) +
     $                        ABS( DL( I-1 )*X( I-1, J ) ) +
     $                        ABS( D( I )*X( I, J ) ) +
     $                        ABS( DU( I )*X( I+1, J ) )
```

```
   30         CONTINUE
              WORK( N ) = ABS( B( N, J ) ) +
     $                    ABS( DL( N-1 )*X( N-1, J ) ) +
     $                    ABS( D( N )*X( N, J ) )
           END IF
        ELSE
           IF( N.EQ.1 ) THEN
              WORK( 1 ) = ABS( B( 1, J ) ) + ABS( D( 1 )*X( 1, J ) )
           ELSE
              WORK( 1 ) = ABS( B( 1, J ) ) + ABS( D( 1 )*X( 1, J ) ) +
     $                    ABS( DL( 1 )*X( 2, J ) )
              DO 40 I = 2, N - 1
                 WORK( I ) = ABS( B( I, J ) ) +
     $                       ABS( DU( I-1 )*X( I-1, J ) ) +
     $                       ABS( D( I )*X( I, J ) ) +
     $                       ABS( DL( I )*X( I+1, J ) )
   40         CONTINUE
              WORK( N ) = ABS( B( N, J ) ) +
     $                    ABS( DU( N-1 )*X( N-1, J ) ) +
     $                    ABS( D( N )*X( N, J ) )
           END IF
        END IF
*
*       Compute componentwise relative backward error from formula
*
*       max(i) ( abs(R(i)) / ( abs(op(A))*abs(X) + abs(B) )(i) )
*
*       where abs(Z) is the componentwise absolute value of the matrix
*       or vector Z.  If the i-th component of the denominator is less
*       than SAFE2, then SAFE1 is added to the i-th components of the
*       numerator and denominator before dividing.
*
        S = ZERO
        DO 50 I = 1, N
           IF( WORK( I ).GT.SAFE2 ) THEN
              S = MAX( S, ABS( WORK( N+I ) ) / WORK( I ) )
           ELSE
              S = MAX( S, ( ABS( WORK( N+I ) )+SAFE1 ) /
     $            ( WORK( I )+SAFE1 ) )
           END IF
   50   CONTINUE
        BERR( J ) = S
*
*       Test stopping criterion. Continue iterating if
*          1) The residual BERR(J) is larger than machine epsilon, and
*          2) BERR(J) decreased by at least a factor of 2 during the
*             last iteration, and
*          3) At most ITMAX iterations tried.
*
        IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
```

```
     $       COUNT.LE.ITMAX ) THEN
*
*          Update solution and try again.
*
           CALL SGTTRS( TRANS, N, 1, DLF, DF, DUF, DU2, IPIV,
     $                  WORK( N+1 ), N, INFO )
           CALL SAXPY( N, ONE, WORK( N+1 ), 1, X( 1, J ), 1 )
           LSTRES = BERR( J )
           COUNT = COUNT + 1
           GO TO 20
        END IF
*
*       Bound error from formula
*
*       norm(X - XTRUE) / norm(X) .le. FERR =
*       norm( abs(inv(op(A)))*
*          ( abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) ))) / norm(X)
*
*       where
*         norm(Z) is the magnitude of the largest component of Z
*         inv(op(A)) is the inverse of op(A)
*         abs(Z) is the componentwise absolute value of the matrix or
*            vector Z
*         NZ is the maximum number of nonzeros in any row of A, plus 1
*         EPS is machine epsilon
*
*       The i-th component of abs(R)+NZ*EPS*(abs(op(A))*abs(X)+abs(B))
*       is incremented by SAFE1 if the i-th component of
*       abs(op(A))*abs(X) + abs(B) is less than SAFE2.
*
*       Use SLACN2 to estimate the infinity-norm of the matrix
*          inv(op(A)) * diag(W),
*       where W = abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) )))
*
        DO 60 I = 1, N
           IF( WORK( I ).GT.SAFE2 ) THEN
              WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I )
           ELSE
              WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I ) + SAFE1
           END IF
   60   CONTINUE
*
        KASE = 0
   70   CONTINUE
        CALL SLACN2( N, WORK( 2*N+1 ), WORK( N+1 ), IWORK, FERR( J ),
     $               KASE, ISAVE )
        IF( KASE.NE.0 ) THEN
           IF( KASE.EQ.1 ) THEN
*
*             Multiply by diag(W)*inv(op(A)**T).
```

```
*
               CALL SGTTRS( TRANST, N, 1, DLF, DF, DUF, DU2, IPIV,
     $                      WORK( N+1 ), N, INFO )
               DO 80 I = 1, N
                  WORK( N+I ) = WORK( I )*WORK( N+I )
   80          CONTINUE
            ELSE
*
*              Multiply by inv(op(A))*diag(W).
*
               DO 90 I = 1, N
                  WORK( N+I ) = WORK( I )*WORK( N+I )
   90          CONTINUE
               CALL SGTTRS( TRANSN, N, 1, DLF, DF, DUF, DU2, IPIV,
     $                      WORK( N+1 ), N, INFO )
            END IF
            GO TO 70
         END IF
*
*        Normalize error.
*
         LSTRES = ZERO
         DO 100 I = 1, N
            LSTRES = MAX( LSTRES, ABS( X( I, J ) ) )
  100    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  110 CONTINUE
*
      RETURN
*
*     End of SGTRFS
*
      END
```

———-

79.2.4 lisp code

— **sgtrfs.lisp** —

```
(let* ((itmax 5) (zero 0.0) (one 1.0) (two 2.0) (three 3.0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 3.0 3.0) three)
```

```
          (ignorable itmax zero one two three))
 (defun sgtrfs
        (trans n nrhs dl d du dlf df duf du2 ipiv b ldb$ x ldx ferr berr work
         iwork info)
   (declare (type (array f2cl-lib:integer4 (*)) iwork ipiv)
            (type (array single-float (*)) work berr ferr x b du2 duf df dlf
             du d dl)
            (type (f2cl-lib:integer4) info ldx ldb$ nrhs n)
            (type (string *) trans))
   (f2cl-lib:with-multi-array-data
       ((trans character trans-%data% trans-%offset%)
        (dl single-float dl-%data% dl-%offset%)
        (d single-float d-%data% d-%offset%)
        (du single-float du-%data% du-%offset%)
        (dlf single-float dlf-%data% dlf-%offset%)
        (df single-float df-%data% df-%offset%)
        (duf single-float duf-%data% duf-%offset%)
        (du2 single-float du2-%data% du2-%offset%)
        (b single-float b-%data% b-%offset%)
        (x single-float x-%data% x-%offset%)
        (ferr single-float ferr-%data% ferr-%offset%)
        (berr single-float berr-%data% berr-%offset%)
        (work single-float work-%data% work-%offset%)
        (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
        (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
     (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4)) (eps 0.0)
            (lstres 0.0) (s 0.0) (safe1 0.0) (safe2 0.0) (safmin 0.0)
            (count$ 0) (i 0) (j 0) (kase 0) (nz 0)
            (transn
             (make-array '(1) :element-type 'character :initial-element #\ ))
            (transt
             (make-array '(1) :element-type 'character :initial-element #\ ))
            (notran nil))
       (declare (type (array f2cl-lib:integer4 (3)) isave)
                (type (single-float) eps lstres s safe1 safe2 safmin)
                (type (f2cl-lib:integer4) count$ i j kase nz)
                (type (string 1) transn transt)
                (type f2cl-lib:logical notran))
       (setf info 0)
       (setf notran
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame trans "N")
                 (declare (ignore var-1))
                 (when var-0 (setf trans var-0))
                 ret-val))
       (cond
        ((and (not notran)
              (not
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame trans "T")
```

```
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -13))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -15)))
(cond
 ((/= info 0) (xerbla "SGTRFS" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%) zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) zero)
     label10))
  (go end_label)))
(cond
 (notran (f2cl-lib:f2cl-set-string transn "N" (string 1))
  (f2cl-lib:f2cl-set-string transt "T" (string 1)))
 (t (f2cl-lib:f2cl-set-string transn "T" (string 1))
  (f2cl-lib:f2cl-set-string transt "N" (string 1))))
(setf nz 4)
(setf eps (slamch "Epsilon"))
(setf safmin (slamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy n
         (f2cl-lib:array-slice b-%data% single-float (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1
         (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                               ((1 *)) work-%offset%)
```

```
     1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11)
    (slagtm trans n 1 (- one) dl d du
     (f2cl-lib:array-slice x-%data% single-float (1 j)
                           ((1 ldx) (1 *)) x-%offset%)
     ldx one
     (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                           ((1 *)) work-%offset%)
     n)
  (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-10))
  (when var-0 (setf trans var-0))
  (when var-1 (setf n var-1))
  (when var-8 (setf ldx var-8))
  (when var-9 (setf one var-9))
  (when var-11 (setf n var-11)))
(cond
 (notran
  (cond
   ((= n 1)
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (+
             (abs
              (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                             b-%offset%))
             (abs
              (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
                 (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                                x-%offset%))))))
   (t
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (+
             (abs
              (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                             b-%offset%))
             (abs
              (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
                 (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                                x-%offset%)))
             (abs
              (* (f2cl-lib:fref du-%data% (1) ((1 *)) du-%offset%)
                 (f2cl-lib:fref x-%data% (2 j) ((1 ldx) (1 *))
                                x-%offset%)))))
    (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
```

```
            (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                    (+
                     (abs
                      (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                     b-%offset%))
                     (abs
                      (*
                       (f2cl-lib:fref dl-%data%
                                      ((f2cl-lib:int-sub i 1)) ((1 *))
                                      dl-%offset%)
                       (f2cl-lib:fref x-%data%
                                      ((f2cl-lib:int-sub i 1) j)
                                      ((1 ldx) (1 *)) x-%offset%)))
                     (abs
                      (*
                       (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                       (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                      x-%offset%)))
                     (abs
                      (*
                       (f2cl-lib:fref du-%data% (i) ((1 *))
                                      du-%offset%)
                       (f2cl-lib:fref x-%data%
                                      ((f2cl-lib:int-add i 1) j)
                                      ((1 ldx) (1 *)) x-%offset%)))))
          label30))
      (setf (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
              (+
               (abs
                (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                               b-%offset%))
               (abs
                (*
                 (f2cl-lib:fref dl-%data% ((f2cl-lib:int-sub n 1))
                                ((1 *)) dl-%offset%)
                 (f2cl-lib:fref x-%data% ((f2cl-lib:int-sub n 1) j)
                                ((1 ldx) (1 *)) x-%offset%)))
               (abs
                (* (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                   (f2cl-lib:fref x-%data% (n j) ((1 ldx) (1 *))
                                  x-%offset%))))))))
    (t
     (cond
      ((= n 1)
       (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
               (+
                (abs
                 (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                                b-%offset%))
                (abs
```

```
                (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
                   (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                                  x-%offset%))))))
(t
 (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
         (+
          (abs
           (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                          b-%offset%))
          (abs
           (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
              (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                             x-%offset%)))
          (abs
           (* (f2cl-lib:fref dl-%data% (1) ((1 *)) dl-%offset%)
              (f2cl-lib:fref x-%data% (2 j) ((1 ldx) (1 *))
                             x-%offset%)))))
 (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
               ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
             (+
              (abs
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%))
              (abs
               (*
                (f2cl-lib:fref du-%data%
                               ((f2cl-lib:int-sub i 1)) ((1 *))
                               du-%offset%)
                (f2cl-lib:fref x-%data%
                               ((f2cl-lib:int-sub i 1) j)
                               ((1 ldx) (1 *)) x-%offset%)))
              (abs
               (*
                (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                               x-%offset%)))
              (abs
               (*
                (f2cl-lib:fref dl-%data% (i) ((1 *))
                               dl-%offset%)
                (f2cl-lib:fref x-%data%
                               ((f2cl-lib:int-add i 1) j)
                               ((1 ldx) (1 *)) x-%offset%)))))
   label40))
 (setf (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
         (+
          (abs
```

```
                (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                               b-%offset%))
               (abs
                (*
                 (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub n 1))
                                ((1 *)) du-%offset%)
                 (f2cl-lib:fref x-%data% ((f2cl-lib:int-sub n 1) j)
                                ((1 ldx) (1 *)) x-%offset%)))
               (abs
                (* (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                   (f2cl-lib:fref x-%data% (n j) ((1 ldx) (1 *))
                                  x-%offset%)))))))))
(setf s zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
      (setf s
              (max s
                   (/
                    (abs
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add n i))
                                    ((1 *)) work-%offset%))
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)))))
     (t
      (setf s
              (max s
                   (/
                    (+
                     (abs
                      (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-add n i))
                                     ((1 *)) work-%offset%))
                     safe1)
                    (+
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)
                     safe1))))))
   label50))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
```

```
        (sgttrs trans n 1 dlf df duf du2 ipiv
         (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                               ((1 *)) work-%offset%)
         n info)
      (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-8))
      (when var-0 (setf trans var-0))
      (when var-1 (setf n var-1))
      (when var-9 (setf n var-9))
      (when var-10 (setf info var-10)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (saxpy n one
         (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                               ((1 *)) work-%offset%)
         1
         (f2cl-lib:array-slice x-%data% single-float (1 j)
                               ((1 ldx) (1 *)) x-%offset%)
         1)
      (declare (ignore var-2 var-3 var-4 var-5))
      (when var-0 (setf n var-0))
      (when var-1 (setf one var-1)))
    (setf lstres
            (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
    (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (+
                 (abs
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                 ((1 *)) work-%offset%))
                 (* nz eps
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)))))
       (t
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (+
                 (abs
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                 ((1 *)) work-%offset%))
                 (* nz eps
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%))
                 safe1))))
     label60))
  (setf kase 0)
 label70
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
      (slacn2 n
       (f2cl-lib:array-slice work-%data% single-float
                             ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                             ((1 *)) work-%offset%)
       iwork
       (f2cl-lib:array-slice ferr-%data% single-float (j) ((1 *))
                             ferr-%offset%)
       kase isave)
    (declare (ignore var-1 var-2 var-3 var-4 var-6))
    (when var-0 (setf n var-0))
    (when var-5 (setf kase var-5)))
  (cond
   ((/= kase 0)
    (cond
     ((= kase 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (sgttrs transt n 1 dlf df duf du2 ipiv
           (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                                 ((1 *)) work-%offset%)
           n info)
        (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-8))
        (when var-0 (setf transt var-0))
        (when var-1 (setf n var-1))
        (when var-9 (setf n var-9))
        (when var-10 (setf info var-10)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%)
                (*
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add n i)) ((1 *))
                                work-%offset%)))
         label80)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%)
                (*
                 (f2cl-lib:fref work-%data% (i) ((1 *))
                                work-%offset%)
```

```
                            (f2cl-lib:fref work-%data%
                                           ((f2cl-lib:int-add n i)) ((1 *))
                                           work-%offset%)))
                   label90))
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                     var-9 var-10)
                    (sgttrs transn n 1 dlf df duf du2 ipiv
                     (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                                           ((1 *)) work-%offset%)
                     n info)
                  (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-8))
                  (when var-0 (setf transn var-0))
                  (when var-1 (setf n var-1))
                  (when var-9 (setf n var-9))
                  (when var-10 (setf info var-10)))))
              (go label70)))
            (setf lstres zero)
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf lstres
                        (max lstres
                             (abs
                              (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                             x-%offset%))))
               label100))
            (if (/= lstres zero)
                (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                        (/
                         (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                         lstres)))
           label110))
        (go end_label)
       end_label
        (return
         (values trans n nil nil nil nil nil nil nil nil nil nil nil nil ldx
                 nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgtrfs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
```

```
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::trans
  fortran-to-lisp::n nil nil nil
  nil nil nil nil nil nil nil nil
  nil fortran-to-lisp::ldx nil nil
  nil nil fortran-to-lisp::info)
:calls 'nil)))
```

———-

79.3 sgttrf

79.3.1 Axiom unit tests

— sgttrf.input —

```
)set break resume
)sys rm -f sgttrf.output
)spool sgttrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

79.3.2 Axiom help page

— sgttrf.help —

```
=====================================================================
sgttrf examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgttrf (N, DL, D, DU, DU2, IPIV, INFO)
           SGTTRF

Function/Subroutine Documentation
   subroutine sgttrf (integer N, real, dimension( * ) DL, real, dimension( * )
       D, real, dimension( * ) DU, real, dimension( * ) DU2, integer,
       dimension( * ) IPIV, integer INFO)
       SGTTRF

Purpose:
   SGTTRF computes an LU factorization of a real tridiagonal matrix A
   using elimination with partial pivoting and row interchanges.

   The factorization has the form
      A = L * U
   where L is a product of permutation and unit lower bidiagonal
   matrices and U is upper triangular with nonzeros in only the main
   diagonal and first two superdiagonals.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.

 DL
   DL is REAL array, dimension (N-1)
   On entry, DL must contain the (n-1) sub-diagonal elements of A.

   On exit, DL is overwritten by the (n-1) multipliers that
   define the matrix L from the LU factorization of A.

 D
   D is REAL array, dimension (N)
   On entry, D must contain the diagonal elements of A.
```

```
  On exit, D is overwritten by the n diagonal elements of the
  upper triangular matrix U from the LU factorization of A.

DU
  DU is REAL array, dimension (N-1)
  On entry, DU must contain the (n-1) super-diagonal elements of A.

  On exit, DU is overwritten by the (n-1) elements of the first
  super-diagonal of U.

DU2
  DU2 is REAL array, dimension (N-2)
  On exit, DU2 is overwritten by the (n-2) elements of the
  second super-diagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= n, row i of the matrix was
  interchanged with row IPIV(i).  IPIV(i) will always be either
  i or i+1; IPIV(i) = i indicates a row interchange was not required.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -k, the k-th argument had an illegal value
  > 0:  if INFO = k, U(k,k) is exactly zero. The factorization
     has been completed, but the factor U is exactly
     singular, and division by zero will occur if it is used
     to solve a system of equations.
```

79.3.3 fortran code

— sgttrf.f —

```
*  =====================================================================
      SUBROUTINE SGTTRF( N, DL, D, DU, DU2, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, N
```

```
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               D( * ), DL( * ), DU( * ), DU2( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      REAL               FACT, TEMP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
         CALL XERBLA( 'SGTTRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Initialize IPIV(i) = i and DU2(I) = 0
*
      DO 10 I = 1, N
         IPIV( I ) = I
   10 CONTINUE
      DO 20 I = 1, N - 2
         DU2( I ) = ZERO
   20 CONTINUE
*
      DO 30 I = 1, N - 2
         IF( ABS( D( I ) ).GE.ABS( DL( I ) ) ) THEN
*
*           No row interchange required, eliminate DL(I)
*
```

```
            IF( D( I ).NE.ZERO ) THEN
               FACT = DL( I ) / D( I )
               DL( I ) = FACT
               D( I+1 ) = D( I+1 ) - FACT*DU( I )
            END IF
         ELSE
*
*           Interchange rows I and I+1, eliminate DL(I)
*
            FACT = D( I ) / DL( I )
            D( I ) = DL( I )
            DL( I ) = FACT
            TEMP = DU( I )
            DU( I ) = D( I+1 )
            D( I+1 ) = TEMP - FACT*D( I+1 )
            DU2( I ) = DU( I+1 )
            DU( I+1 ) = -FACT*DU( I+1 )
            IPIV( I ) = I + 1
         END IF
   30 CONTINUE
      IF( N.GT.1 ) THEN
         I = N - 1
         IF( ABS( D( I ) ).GE.ABS( DL( I ) ) ) THEN
            IF( D( I ).NE.ZERO ) THEN
               FACT = DL( I ) / D( I )
               DL( I ) = FACT
               D( I+1 ) = D( I+1 ) - FACT*DU( I )
            END IF
         ELSE
            FACT = D( I ) / DL( I )
            D( I ) = DL( I )
            DL( I ) = FACT
            TEMP = DU( I )
            DU( I ) = D( I+1 )
            D( I+1 ) = TEMP - FACT*D( I+1 )
            IPIV( I ) = I + 1
         END IF
      END IF
*
*     Check for a zero on the diagonal of U.
*
      DO 40 I = 1, N
         IF( D( I ).EQ.ZERO ) THEN
            INFO = I
            GO TO 50
         END IF
   40 CONTINUE
   50 CONTINUE
*
      RETURN
```

```
*
*     End of SGTTRF
*
      END
```

———-

79.3.4 lisp code

— **sgttrf.lisp** —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun sgttrf (n dl d du du2 ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array single-float (*)) du2 du d dl)
             (type (f2cl-lib:integer4) info n))
    (f2cl-lib:with-multi-array-data
        ((dl single-float dl-%data% dl-%offset%)
         (d single-float d-%data% d-%offset%)
         (du single-float du-%data% du-%offset%)
         (du2 single-float du2-%data% du2-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((fact 0.0) (temp 0.0) (i 0))
        (declare (type (single-float) fact temp)
                 (type (f2cl-lib:integer4) i))
        (setf info 0)
        (cond
         ((< n 0) (setf info -1) (xerbla "SGTTRF" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (= n 0)
            (go end_label))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) i)
           label10))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))) nil)
          (tagbody
            (setf (f2cl-lib:fref du2-%data% (i) ((1 *)) du2-%offset%) zero)
           label20))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))) nil)
          (tagbody
            (cond
             ((>= (abs (f2cl-lib:fref d (i) ((1 *))))
```

```
            (abs (f2cl-lib:fref dl (i) ((1 *)))))
        (cond
         ((/= (f2cl-lib:fref d (i) ((1 *))) zero)
          (setf fact
                  (/ (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                     (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
          (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) fact)
          (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                d-%offset%)
                  (-
                   (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                  ((1 *)) d-%offset%)
                   (* fact
                      (f2cl-lib:fref du-%data% (i) ((1 *))
                                     du-%offset%)))))))
       (t
        (setf fact
                (/ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                   (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
        (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
        (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) fact)
        (setf temp (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%))
        (setf (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                d-%offset%))
        (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                              d-%offset%)
                (- temp
                   (* fact
                      (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                     ((1 *)) d-%offset%))))
        (setf (f2cl-lib:fref du2-%data% (i) ((1 *)) du2-%offset%)
                (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                du-%offset%))
        (setf (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                              du-%offset%)
                (* (- fact)
                   (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1))
                                  ((1 *)) du-%offset%)))
        (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
                (f2cl-lib:int-add i 1))))
    label30))
  (cond
   ((> n 1) (setf i (f2cl-lib:int-sub n 1))
    (cond
     ((>= (abs (f2cl-lib:fref d (i) ((1 *))))
          (abs (f2cl-lib:fref dl (i) ((1 *)))))
      (cond
       ((/= (f2cl-lib:fref d (i) ((1 *))) zero)
```

```
              (setf fact
                      (/ (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                         (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
              (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) fact)
              (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                   d-%offset%)
                      (-
                       (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                      d-%offset%)
                       (* fact
                          (f2cl-lib:fref du-%data% (i) ((1 *))
                                         du-%offset%)))))))
            (t
             (setf fact
                     (/ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                        (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
             (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                     (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
             (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) fact)
             (setf temp (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%))
             (setf (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                     (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                    d-%offset%))
             (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                  d-%offset%)
                     (- temp
                        (* fact
                           (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                          ((1 *)) d-%offset%))))
             (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
                     (f2cl-lib:int-add i 1))))))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (cond
               ((= (f2cl-lib:fref d (i) ((1 *))) zero) (setf info i)
                (go label50)))
             label40))
         label50
          (go end_label)
         end_label
          (return (values nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgttrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
```

```
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil nil nil nil
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

79.4 sgttrs

79.4.1 Axiom unit tests

— **sgttrs.input** —

```
)set break resume
)sys rm -f sgttrs.output
)spool sgttrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

79.4.2 Axiom help page

— **sgttrs.help** —

```
======================================================================
sgttrs examples
======================================================================

======================================================================
Man Page Details
```

```
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgttrs (TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB, INFO)
           SGTTRS

Function/Subroutine Documentation
   subroutine sgttrs (character TRANS, integer N, integer NRHS, real,
       dimension( * ) DL, real, dimension( * ) D, real, dimension( * ) DU,
       real, dimension( * ) DU2, integer, dimension( * ) IPIV, real,
       dimension( ldb, * ) B, integer LDB, integer INFO)
       SGTTRS

Purpose:
   SGTTRS solves one of the systems of equations
      A*X = B  or  A**T*X = B,
   with a tridiagonal matrix A using the LU factorization computed by SGTTRF.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations.
   = 'N':  A * X = B  (No transpose)
   = 'T':  A**T* X = B  (Transpose)
   = 'C':  A**T* X = B  (Conjugate transpose = Transpose)

 N
   N is INTEGER
   The order of the matrix A.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 DL
   DL is REAL array, dimension (N-1)
   The (n-1) multipliers that define the matrix L from the
   LU factorization of A.

 D
   D is REAL array, dimension (N)
   The n diagonal elements of the upper triangular matrix U from
   the LU factorization of A.

 DU
   DU is REAL array, dimension (N-1)
   The (n-1) elements of the first super-diagonal of U.
```

```
 DU2
   DU2 is REAL array, dimension (N-2)
   The (n-2) elements of the second super-diagonal of U.

 IPIV
   IPIV is INTEGER array, dimension (N)
   The pivot indices; for 1 <= i <= n, row i of the matrix was
   interchanged with row IPIV(i).  IPIV(i) will always be either
   i or i+1; IPIV(i) = i indicates a row interchange was not required.

 B
   B is REAL array, dimension (LDB,NRHS)
   On entry, the matrix of right hand side vectors B.
   On exit, B is overwritten by the solution vectors X.

 LDB
   LDB is INTEGER
   The leading dimension of the array B.  LDB >= max(1,N).

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

79.4.3 fortran code

— **sgttrs.f** —

```
*  =====================================================================
      SUBROUTINE SGTTRS( TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               B( LDB, * ), D( * ), DL( * ), DU( * ), DU2( * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      INTEGER            ITRANS, J, JB, NB
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           SGTTS2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOTRAN = ( TRANS.EQ.'N' .OR. TRANS.EQ.'n' )
      IF( .NOT.NOTRAN .AND. .NOT.( TRANS.EQ.'T' .OR. TRANS.EQ.
     $    't' ) .AND. .NOT.( TRANS.EQ.'C' .OR. TRANS.EQ.'c' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDB.LT.MAX( N, 1 ) ) THEN
         INFO = -10
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SGTTRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
*     Decode TRANS
*
      IF( NOTRAN ) THEN
         ITRANS = 0
      ELSE
         ITRANS = 1
      END IF
*
```

```
*     Determine the number of right-hand sides to solve at a time.
*
      IF( NRHS.EQ.1 ) THEN
         NB = 1
      ELSE
         NB = MAX( 1, ILAENV( 1, 'SGTTRS', TRANS, N, NRHS, -1, -1 ) )
      END IF
*
      IF( NB.GE.NRHS ) THEN
         CALL SGTTS2( ITRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB )
      ELSE
         DO 10 J = 1, NRHS, NB
            JB = MIN( NRHS-J+1, NB )
            CALL SGTTS2( ITRANS, N, JB, DL, D, DU, DU2, IPIV, B( 1, J ),
     $                   LDB )
   10    CONTINUE
      END IF
*
*     End of SGTTRS
*
      END
```

———-

79.4.4 lisp code

— **sgttrs.lisp** —

```
(defun sgttrs (trans n nrhs dl d du du2 ipiv b ldb$ info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array single-float (*)) b du2 du d dl)
           (type (f2cl-lib:integer4) info ldb$ nrhs n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (dl single-float dl-%data% dl-%offset%)
       (d single-float d-%data% d-%offset%)
       (du single-float du-%data% du-%offset%)
       (du2 single-float du2-%data% du2-%offset%)
       (b single-float b-%data% b-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((itrans 0) (j 0) (jb 0) (nb 0) (notran nil))
      (declare (type f2cl-lib:logical notran)
               (type (f2cl-lib:integer4) nb jb j itrans))
      (setf info 0)
      (setf notran
              (or (f2cl-lib:fstring-= trans "N")
                  (f2cl-lib:fstring-= trans "n")))
```

```
(cond
 ((and (not notran)
       (not
        (or (f2cl-lib:fstring-= trans "T")
            (f2cl-lib:fstring-= trans "t")))
       (not
        (or (f2cl-lib:fstring-= trans "C")
            (f2cl-lib:fstring-= trans "c"))))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< ldb$ (max (the f2cl-lib:integer4 n) (the f2cl-lib:integer4 1)))
  (setf info -10)))
(cond
 ((/= info 0) (xerbla "SGTTRS" (f2cl-lib:int-sub info)) (go end_label)))
(if (or (= n 0) (= nrhs 0))
    (go end_label))
(cond (notran (setf itrans 0)) (t (setf itrans 1)))
(cond ((= nrhs 1) (setf nb 1))
      (t
       (setf nb
               (max (the f2cl-lib:integer4 1)
                    (the f2cl-lib:integer4
                         (multiple-value-bind
                             (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                              var-6)
                             (ilaenv 1 "SGTTRS" trans n nrhs -1 -1)
                           (declare (ignore var-0 var-1 var-5 var-6))
                           (setf trans var-2)
                           (setf n var-3)
                           (setf nrhs var-4)
                           ret-val))))))
(cond
 ((>= nb nrhs)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (sgtts2 itrans n nrhs dl d du du2 ipiv b ldb$)
    (declare (ignore var-3 var-4 var-5 var-6 var-7 var-8))
    (when var-0 (setf itrans var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf nrhs var-2))
    (when var-9 (setf ldb$ var-9))))
 (t
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                ((> j nrhs) nil)
    (tagbody
      (setf jb
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub nrhs j) 1))
               (the f2cl-lib:integer4 nb)))
```

```
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                (sgtts2 itrans n jb dl d du du2 ipiv
                 (f2cl-lib:array-slice b-%data% single-float (1 j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$)
              (declare (ignore var-3 var-4 var-5 var-6 var-7 var-8))
              (when var-0 (setf itrans var-0))
              (when var-1 (setf n var-1))
              (when var-2 (setf jb var-2))
              (when var-9 (setf ldb$ var-9)))
           label10))))
     end_label
      (return (values trans n nrhs nil nil nil nil nil nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgttrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::trans
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil nil nil
                                              nil nil nil fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::ilaenv))))
```

———-

79.5 sgtts2

79.5.1 Axiom unit tests

— sgtts2.input —

```
)set break resume
)sys rm -f sgtts2.output
)spool sgtts2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

79.5.2 Axiom help page

— sgtts2.help —

```
======================================================================
sgtts2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sgtts2 (ITRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB)
           SGTTS2 solves a system of linear equations with a tridiagonal
           matrix using the LU factorization computed by sgttrf.

Function/Subroutine Documentation
   subroutine sgtts2 (integer ITRANS, integer N, integer NRHS, real,
       dimension( * ) DL, real, dimension( * ) D, real, dimension( * ) DU,
       real, dimension( * ) DU2, integer, dimension( * ) IPIV, real,
       dimension( ldb, * ) B, integer LDB)
       SGTTS2 solves a system of linear equations with a tridiagonal matrix
       using the LU factorization computed by sgttrf.

Purpose:
   SGTTS2 solves one of the systems of equations
```

```
      A*X = B  or  A**T*X = B,
  with a tridiagonal matrix A using the LU factorization computed by SGTTRF.

Parameters:

 ITRANS
   ITRANS is INTEGER
   Specifies the form of the system of equations.
   = 0:  A * X = B  (No transpose)
   = 1:  A**T* X = B  (Transpose)
   = 2:  A**T* X = B  (Conjugate transpose = Transpose)

 N
   N is INTEGER
   The order of the matrix A.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 DL
   DL is REAL array, dimension (N-1)
   The (n-1) multipliers that define the matrix L from the
   LU factorization of A.

 D
   D is REAL array, dimension (N)
   The n diagonal elements of the upper triangular matrix U from
   the LU factorization of A.

 DU
   DU is REAL array, dimension (N-1)
   The (n-1) elements of the first super-diagonal of U.

 DU2
   DU2 is REAL array, dimension (N-2)
   The (n-2) elements of the second super-diagonal of U.

 IPIV
   IPIV is INTEGER array, dimension (N)
   The pivot indices; for 1 <= i <= n, row i of the matrix was
   interchanged with row IPIV(i).  IPIV(i) will always be either
   i or i+1; IPIV(i) = i indicates a row interchange was not required.

 B
   B is REAL array, dimension (LDB,NRHS)
   On entry, the matrix of right hand side vectors B.
   On exit, B is overwritten by the solution vectors X.
```

```
LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).
```

———-

79.5.3 fortran code

— **sgtts2.f** —

```
*  =====================================================================
      SUBROUTINE SGTTS2( ITRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            ITRANS, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               B( LDB, * ), D( * ), DL( * ), DU( * ), DU2( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, IP, J
      REAL               TEMP
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( ITRANS.EQ.0 ) THEN
*
*        Solve A*X = B using the LU factorization of A,
*        overwriting each right hand side vector with its solution.
*
         IF( NRHS.LE.1 ) THEN
            J = 1
   10       CONTINUE
```

```
*
*          Solve L*x = b.
*
           DO 20 I = 1, N - 1
              IP = IPIV( I )
              TEMP = B( I+1-IP+I, J ) - DL( I )*B( IP, J )
              B( I, J ) = B( IP, J )
              B( I+1, J ) = TEMP
   20      CONTINUE
*
*          Solve U*x = b.
*
           B( N, J ) = B( N, J ) / D( N )
           IF( N.GT.1 )
     $        B( N-1, J ) = ( B( N-1, J )-DU( N-1 )*B( N, J ) ) /
     $                      D( N-1 )
           DO 30 I = N - 2, 1, -1
              B( I, J ) = ( B( I, J )-DU( I )*B( I+1, J )-DU2( I )*
     $                    B( I+2, J ) ) / D( I )
   30      CONTINUE
           IF( J.LT.NRHS ) THEN
              J = J + 1
              GO TO 10
           END IF
        ELSE
           DO 60 J = 1, NRHS
*
*             Solve L*x = b.
*
              DO 40 I = 1, N - 1
                 IF( IPIV( I ).EQ.I ) THEN
                    B( I+1, J ) = B( I+1, J ) - DL( I )*B( I, J )
                 ELSE
                    TEMP = B( I, J )
                    B( I, J ) = B( I+1, J )
                    B( I+1, J ) = TEMP - DL( I )*B( I, J )
                 END IF
   40         CONTINUE
*
*             Solve U*x = b.
*
              B( N, J ) = B( N, J ) / D( N )
              IF( N.GT.1 )
     $           B( N-1, J ) = ( B( N-1, J )-DU( N-1 )*B( N, J ) ) /
     $                         D( N-1 )
              DO 50 I = N - 2, 1, -1
                 B( I, J ) = ( B( I, J )-DU( I )*B( I+1, J )-DU2( I )*
     $                       B( I+2, J ) ) / D( I )
   50         CONTINUE
   60      CONTINUE
```

```
         END IF
      ELSE
*
*        Solve A**T * X = B.
*
         IF( NRHS.LE.1 ) THEN
*
*           Solve U**T*x = b.
*
            J = 1
   70       CONTINUE
            B( 1, J ) = B( 1, J ) / D( 1 )
            IF( N.GT.1 )
     $         B( 2, J ) = ( B( 2, J )-DU( 1 )*B( 1, J ) ) / D( 2 )
            DO 80 I = 3, N
               B( I, J ) = ( B( I, J )-DU( I-1 )*B( I-1, J )-DU2( I-2 )*
     $                     B( I-2, J ) ) / D( I )
   80       CONTINUE
*
*           Solve L**T*x = b.
*
            DO 90 I = N - 1, 1, -1
               IP = IPIV( I )
               TEMP = B( I, J ) - DL( I )*B( I+1, J )
               B( I, J ) = B( IP, J )
               B( IP, J ) = TEMP
   90       CONTINUE
            IF( J.LT.NRHS ) THEN
               J = J + 1
               GO TO 70
            END IF
*
         ELSE
            DO 120 J = 1, NRHS
*
*              Solve U**T*x = b.
*
               B( 1, J ) = B( 1, J ) / D( 1 )
               IF( N.GT.1 )
     $            B( 2, J ) = ( B( 2, J )-DU( 1 )*B( 1, J ) ) / D( 2 )
               DO 100 I = 3, N
                  B( I, J ) = ( B( I, J )-DU( I-1 )*B( I-1, J )-
     $                        DU2( I-2 )*B( I-2, J ) ) / D( I )
  100          CONTINUE
               DO 110 I = N - 1, 1, -1
                  IF( IPIV( I ).EQ.I ) THEN
                     B( I, J ) = B( I, J ) - DL( I )*B( I+1, J )
                  ELSE
                     TEMP = B( I+1, J )
                     B( I+1, J ) = B( I, J ) - DL( I )*TEMP
```

```
                  B( I, J ) = TEMP
               END IF
  110        CONTINUE
  120      CONTINUE
         END IF
      END IF
*
*     End of SGTTS2
*
      END
```

———-

79.5.4 lisp code

— **sgtts2.lisp** —

```
(defun sgtts2 (itrans n nrhs dl d du du2 ipiv b ldb$)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array single-float (*)) b du2 du d dl)
           (type (f2cl-lib:integer4) ldb$ nrhs n itrans))
  (f2cl-lib:with-multi-array-data
      ((dl single-float dl-%data% dl-%offset%)
       (d single-float d-%data% d-%offset%)
       (du single-float du-%data% du-%offset%)
       (du2 single-float du2-%data% du2-%offset%)
       (b single-float b-%data% b-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((temp 0.0) (i 0) (ip 0) (j 0))
      (declare (type (f2cl-lib:integer4) j ip i)
               (type (single-float) temp))
      (if (or (= n 0) (= nrhs 0))
          (go end_label))
      (cond
       ((= itrans 0)
        (cond
         ((<= nrhs 1)
          (tagbody
            (setf j 1)
           label10
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
              (tagbody
                (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
                (setf temp
                        (-
                         (f2cl-lib:fref b-%data%
                                        ((f2cl-lib:int-add
```

```
                                (f2cl-lib:int-sub
                                 (f2cl-lib:int-add i 1) ip)
                                i)
                               j)
                              ((1 ldb$) (1 *)) b-%offset%)
                 (* (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                    (f2cl-lib:fref b-%data% (ip j) ((1 ldb$) (1 *))
                                   b-%offset%))))
    (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                         b-%offset%)
            (f2cl-lib:fref b-%data% (ip j) ((1 ldb$) (1 *))
                           b-%offset%))
    (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                         ((1 ldb$) (1 *)) b-%offset%)
            temp)
   label20))
(setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
        (/
         (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
         (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
(if (> n 1)
    (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                         ((1 ldb$) (1 *)) b-%offset%)
            (/
             (-
              (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)
              (*
               (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub n 1))
                              ((1 *)) du-%offset%)
               (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                              b-%offset%)))
             (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                            ((1 *)) d-%offset%))))
(f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))
               (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                         b-%offset%)
            (/
             (-
              (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)
              (* (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                ((1 ldb$) (1 *)) b-%offset%))
              (*
               (f2cl-lib:fref du2-%data% (i) ((1 *)) du2-%offset%)
               (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 2) j)
```

```
                                  ((1 ldb$) (1 *)) b-%offset%)))
                 (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
      label30))
   (cond ((< j nrhs) (setf j (f2cl-lib:int-add j 1)) (go label10)))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j nrhs) nil)
   (tagbody
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                    nil)
       (tagbody
         (cond
          ((= (f2cl-lib:fref ipiv (i) ((1 *))) i)
           (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                   (-
                    (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                    ((1 ldb$) (1 *)) b-%offset%)
                    (*
                     (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                     b-%offset%)))))
          (t
           (setf temp
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%))
           (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                   (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                   ((1 ldb$) (1 *)) b-%offset%))
           (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                   (- temp
                      (*
                       (f2cl-lib:fref dl-%data% (i) ((1 *))
                                      dl-%offset%)
                       (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                      b-%offset%))))))
       label40))
     (setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
             (/
              (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                             b-%offset%)
              (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
     (if (> n 1)
         (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                               ((1 ldb$) (1 *)) b-%offset%)
                 (/
                  (-
```

```
                        (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                        (*
                         (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub n 1))
                                        ((1 *)) du-%offset%)
                         (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                                        b-%offset%)))
                       (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                      ((1 *)) d-%offset%))))
        (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))
                       (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (/
                   (-
                    (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)
                    (*
                     (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                    ((1 ldb$) (1 *)) b-%offset%))
                    (*
                     (f2cl-lib:fref du2-%data% (i) ((1 *))
                                    du2-%offset%)
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 2) j)
                                    ((1 ldb$) (1 *)) b-%offset%)))
                   (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
           label50))
      label60)))))
(t
 (cond
  ((<= nrhs 1)
   (tagbody
     (setf j 1)
    label70
     (setf (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *)) b-%offset%)
             (/
              (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *)) b-%offset%)
              (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)))
     (if (> n 1)
         (setf (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                              b-%offset%)
                 (/
                  (-
                   (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                                  b-%offset%)
                   (* (f2cl-lib:fref du-%data% (1) ((1 *)) du-%offset%)
                      (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
```

```
                                    b-%offset%)))
                 (f2cl-lib:fref d-%data% (2) ((1 *)) d-%offset%))))
    (f2cl-lib:fdo (i 3 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)
              (/
               (-
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)
                (*
                 (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub i 1))
                                ((1 *)) du-%offset%)
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                ((1 ldb$) (1 *)) b-%offset%))
                (*
                 (f2cl-lib:fref du2-%data% ((f2cl-lib:int-sub i 2))
                                ((1 *)) du2-%offset%)
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 2) j)
                                ((1 ldb$) (1 *)) b-%offset%)))
               (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
       label80))
    (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
      (tagbody
        (setf ip (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%))
        (setf temp
              (-
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)
               (* (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                  (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%))))
        (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)
              (f2cl-lib:fref b-%data% (ip j) ((1 ldb$) (1 *))
                             b-%offset%))
        (setf (f2cl-lib:fref b-%data% (ip j) ((1 ldb$) (1 *))
                             b-%offset%)
              temp)
       label90))
    (cond ((< j nrhs) (setf j (f2cl-lib:int-add j 1)) (go label70)))))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *)) b-%offset%)
            (/
```

```
          (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                         b-%offset%)
          (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)))
(if (> n 1)
    (setf (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                         b-%offset%)
            (/
             (-
              (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                             b-%offset%)
              (*
               (f2cl-lib:fref du-%data% (1) ((1 *)) du-%offset%)
               (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                              b-%offset%)))
             (f2cl-lib:fref d-%data% (2) ((1 *)) d-%offset%))))
(f2cl-lib:fdo (i 3 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                         b-%offset%)
            (/
             (-
              (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)
              (*
               (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub i 1))
                              ((1 *)) du-%offset%)
               (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                              ((1 ldb$) (1 *)) b-%offset%))
              (*
               (f2cl-lib:fref du2-%data% ((f2cl-lib:int-sub i 2))
                              ((1 *)) du2-%offset%)
               (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 2) j)
                              ((1 ldb$) (1 *)) b-%offset%)))
             (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
   label100))
(f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (cond
     ((= (f2cl-lib:fref ipiv (i) ((1 *))) i)
      (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                           b-%offset%)
              (-
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)
               (*
                (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                (f2cl-lib:fref b-%data%
```

```
                                        ((f2cl-lib:int-add i 1) j)
                                        ((1 ldb$) (1 *)) b-%offset%)))))
                  (t
                   (setf temp
                           (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                          ((1 ldb$) (1 *)) b-%offset%))
                   (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                        ((1 ldb$) (1 *)) b-%offset%)
                           (-
                            (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                           b-%offset%)
                            (*
                             (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                             temp)))
                   (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                        b-%offset%)
                           temp)))
                label110))
            label120))))))
     end_label
      (return (values nil nil nil nil nil nil nil nil nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sgtts2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
                                              nil nil)
                                            :calls 'nil)))
```

Chapter 80

LAPACK - General Tridiagonal Matrix, Comp. Routines, Complex

80.1 cgtcon

80.1.1 Axiom unit tests

— **cgtcon.input** —

```
)set break resume
)sys rm -f cgtcon.output
)spool cgtcon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

80.1.2 Axiom help page

— **cgtcon.help** —

```
==================================================================
```

```
cgtcon examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgtcon (NORM, N, DL, D, DU, DU2, IPIV, ANORM, RCOND, WORK,
           INFO)
           CGTCON

Function/Subroutine Documentation
   subroutine cgtcon (character NORM, integer N, complex, dimension( * ) DL,
       complex, dimension( * ) D, complex, dimension( * ) DU, complex,
       dimension( * ) DU2, integer, dimension( * ) IPIV, real ANORM, real
       RCOND, complex, dimension( * ) WORK, integer INFO)
       CGTCON

Purpose:
   CGTCON estimates the reciprocal of the condition number of a complex
   tridiagonal matrix A using the LU factorization as computed by
   CGTTRF.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies whether the 1-norm condition number or the
   infinity-norm condition number is required:
   = '1' or 'O':  1-norm;
   = 'I':         Infinity-norm.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 DL
   DL is COMPLEX array, dimension (N-1)
   The (n-1) multipliers that define the matrix L from the
   LU factorization of A as computed by CGTTRF.

 D
   D is COMPLEX array, dimension (N)
   The n diagonal elements of the upper triangular matrix U from
   the LU factorization of A.
```

```
DU
  DU is COMPLEX array, dimension (N-1)
  The (n-1) elements of the first superdiagonal of U.

DU2
  DU2 is COMPLEX array, dimension (N-2)
  The (n-2) elements of the second superdiagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= n, row i of the matrix was
  interchanged with row IPIV(i).  IPIV(i) will always be either
  i or i+1; IPIV(i) = i indicates a row interchange was not required.

ANORM
  ANORM is REAL
  If NORM = '1' or 'O', the 1-norm of the original matrix A.
  If NORM = 'I', the infinity-norm of the original matrix A.

RCOND
  RCOND is REAL
  The reciprocal of the condition number of the matrix A,
  computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is an
  estimate of the 1-norm of inv(A) computed in this routine.

WORK
  WORK is COMPLEX array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

80.1.3 fortran code

— cgtcon.f —

```
*  ====================================================================
      SUBROUTINE CGTCON( NORM, N, DL, D, DU, DU2, IPIV, ANORM, RCOND,
     $                   WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
```

```
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            INFO, N
      REAL               ANORM, RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            D( * ), DL( * ), DU( * ), DU2( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ONENRM
      INTEGER            I, KASE, KASE1
      REAL               AINVNM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGTTRS, CLACN2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          CMPLX
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      ONENRM = NORM.EQ.'1' .OR. LSAME( NORM, 'O' )
      IF( .NOT.ONENRM .AND. .NOT.LSAME( NORM, 'I' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
```

```
         CALL XERBLA( 'CGTCON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
*     Check that D(1:N) is non-zero.
*
      DO 10 I = 1, N
         IF( D( I ).EQ.CMPLX( ZERO ) )
     $      RETURN
   10 CONTINUE
*
      AINVNM = ZERO
      IF( ONENRM ) THEN
         KASE1 = 1
      ELSE
         KASE1 = 2
      END IF
      KASE = 0
   20 CONTINUE
      CALL CLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.KASE1 ) THEN
*
*           Multiply by inv(U)*inv(L).
*
            CALL CGTTRS( 'No transpose', N, 1, DL, D, DU, DU2, IPIV,
     $                   WORK, N, INFO )
         ELSE
*
*           Multiply by inv(L**H)*inv(U**H).
*
            CALL CGTTRS( 'Conjugate transpose', N, 1, DL, D, DU, DU2,
     $                   IPIV, WORK, N, INFO )
         END IF
         GO TO 20
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
```

```
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
      RETURN
*
*     End of CGTCON
*
      END
```

80.1.4 lisp code

— **cgtcon.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun cgtcon (norm n dl d du du2 ipiv anorm rcond work info)
    (declare (type (single-float) rcond anorm)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) work du2 du d dl)
             (type (f2cl-lib:integer4) info n)
             (type (string *) norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (dl f2cl-lib:complex8 dl-%data% dl-%offset%)
         (d f2cl-lib:complex8 d-%data% d-%offset%)
         (du f2cl-lib:complex8 du-%data% du-%offset%)
         (du2 f2cl-lib:complex8 du2-%data% du2-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (ainvnm 0.0) (i 0) (kase 0) (kase1 0) (onenrm nil))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (single-float) ainvnm)
                 (type (f2cl-lib:integer4) i kase kase1)
                 (type f2cl-lib:logical onenrm))
        (setf info 0)
        (setf onenrm
                (or (f2cl-lib:fstring-= norm "1")
                    (multiple-value-bind (ret-val var-0 var-1)
                        (lsame norm "O")
                      (declare (ignore var-1))
                      (when var-0 (setf norm var-0))
                      ret-val)))
        (cond
         ((and (not onenrm)
```

```
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame norm "I")
           (declare (ignore var-1))
           (when var-0 (setf norm var-0))
           ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2)) ((< anorm zero) (setf info -8)))
  (cond
   ((/= info 0) (xerbla "CGTCON" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf rcond zero)
  (cond ((= n 0) (setf rcond one) (go end_label))
        ((= anorm zero) (go end_label)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (if (= (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
             (f2cl-lib:cmplx zero))
          (go end_label))
    label10))
  (setf ainvnm zero)
  (cond (onenrm (setf kase1 1)) (t (setf kase1 2)))
  (setf kase 0)
 label20
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (clacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8 ((+ n 1))
                             ((1 *)) work-%offset%)
       work ainvnm kase isave)
    (declare (ignore var-1 var-2 var-5))
    (when var-0 (setf n var-0))
    (when var-3 (setf ainvnm var-3))
    (when var-4 (setf kase var-4)))
  (cond
   ((/= kase 0)
    (cond
     ((= kase kase1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (cgttrs "No transpose" n 1 dl d du du2 ipiv work n info)
        (declare
         (ignore var-0 var-2 var-3 var-4 var-5 var-6 var-7 var-8))
        (when var-1 (setf n var-1))
        (when var-9 (setf n var-9))
        (when var-10 (setf info var-10))))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
                var-10)
               (cgttrs "Conjugate transpose" n 1 dl d du du2 ipiv work n info)
             (declare
              (ignore var-0 var-2 var-3 var-4 var-5 var-6 var-7 var-8))
             (when var-1 (setf n var-1))
             (when var-9 (setf n var-9))
             (when var-10 (setf info var-10)))))
          (go label20)))
        (if (/= ainvnm zero)
            (setf rcond (/ (/ one ainvnm) anorm)))
        (go end_label)
       end_label
        (return (values norm n nil nil nil nil nil nil rcond nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgtcon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (single-float) (single-float)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::norm
                                              fortran-to-lisp::n nil nil nil
                                              nil nil nil
                                              fortran-to-lisp::rcond nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

80.2 cgtrfs

80.2.1 Axiom unit tests

— cgtrfs.input —

```
)set break resume
)sys rm -f cgtrfs.output
)spool cgtrfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

80.2.2 Axiom help page

— cgtrfs.help —

```
=====================================================================
cgtrfs examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgtrfs (TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF, DU2, IPIV,
           B, LDB, X, LDX, FERR, BERR, WORK, RWORK, INFO)
           CGTRFS

Function/Subroutine Documentation
   subroutine cgtrfs (character TRANS, integer N, integer NRHS, complex,
       dimension( * ) DL, complex, dimension( * ) D, complex, dimension( * )
       DU, complex, dimension( * ) DLF, complex, dimension( * ) DF, complex,
       dimension( * ) DUF, complex, dimension( * ) DU2, integer, dimension( *
       ) IPIV, complex, dimension( ldb, * ) B, integer LDB, complex,
       dimension( ldx, * ) X, integer LDX, real, dimension( * ) FERR, real,
       dimension( * ) BERR, complex, dimension( * ) WORK, real, dimension( * )
       RWORK, integer INFO)
       CGTRFS
```

```
Purpose:
   CGTRFS improves the computed solution to a system of linear
   equations when the coefficient matrix is tridiagonal, and provides
   error bounds and backward error estimates for the solution.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 DL
   DL is COMPLEX array, dimension (N-1)
   The (n-1) subdiagonal elements of A.

 D
   D is COMPLEX array, dimension (N)
   The diagonal elements of A.

 DU
   DU is COMPLEX array, dimension (N-1)
   The (n-1) superdiagonal elements of A.

 DLF
   DLF is COMPLEX array, dimension (N-1)
   The (n-1) multipliers that define the matrix L from the
   LU factorization of A as computed by CGTTRF.

 DF
   DF is COMPLEX array, dimension (N)
   The n diagonal elements of the upper triangular matrix U from
   the LU factorization of A.

 DUF
   DUF is COMPLEX array, dimension (N-1)
   The (n-1) elements of the first superdiagonal of U.
```

```
DU2
  DU2 is COMPLEX array, dimension (N-2)
  The (n-2) elements of the second superdiagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= n, row i of the matrix was
  interchanged with row IPIV(i).  IPIV(i) will always be either
  i or i+1; IPIV(i) = i indicates a row interchange was not required.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by CGTTRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

FERR
  FERR is REAL array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is REAL array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is COMPLEX array, dimension (2*N)

RWORK
  RWORK is REAL array, dimension (N)
```

```
 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

 ITMAX is the maximum number of steps of iterative refinement.
```

80.2.3 fortran code

— cgtrfs.f —

```
*  ===================================================================
      SUBROUTINE CGTRFS( TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF, DU2,
     $                   IPIV, B, LDB, X, LDX, FERR, BERR, WORK, RWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      REAL               BERR( * ), FERR( * ), RWORK( * )
      COMPLEX            B( LDB, * ), D( * ), DF( * ), DL( * ),
     $                   DLF( * ), DU( * ), DU2( * ), DUF( * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
      REAL               TWO
      PARAMETER          ( TWO = 2.0E+0 )
      REAL               THREE
```

```
      PARAMETER          ( THREE = 3.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      CHARACTER          TRANSN, TRANST
      INTEGER            COUNT, I, J, KASE, NZ
      REAL               EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN
      COMPLEX            ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           CAXPY, CCOPY, CGTTRS, CLACN2, CLAGTM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, CMPLX, MAX, REAL
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH
      EXTERNAL           LSAME, SLAMCH
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -13
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -15
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGTRFS', -INFO )
         RETURN
```

```
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
      IF( NOTRAN ) THEN
         TRANSN = 'N'
         TRANST = 'C'
      ELSE
         TRANSN = 'C'
         TRANST = 'N'
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = 4
      EPS = SLAMCH( 'Epsilon' )
      SAFMIN = SLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 110 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - op(A) * X,
*        where op(A) = A, A**T, or A**H, depending on TRANS.
*
         CALL CCOPY( N, B( 1, J ), 1, WORK, 1 )
         CALL CLAGTM( TRANS, N, 1, -ONE, DL, D, DU, X( 1, J ), LDX, ONE,
     $                WORK, N )
*
*        Compute abs(op(A))*abs(x) + abs(b) for use in the backward
*        error bound.
*
         IF( NOTRAN ) THEN
            IF( N.EQ.1 ) THEN
```

```
            RWORK( 1 ) = CABS1( B( 1, J ) ) +
     $                   CABS1( D( 1 ) )*CABS1( X( 1, J ) )
         ELSE
            RWORK( 1 ) = CABS1( B( 1, J ) ) +
     $                   CABS1( D( 1 ) )*CABS1( X( 1, J ) ) +
     $                   CABS1( DU( 1 ) )*CABS1( X( 2, J ) )
            DO 30 I = 2, N - 1
               RWORK( I ) = CABS1( B( I, J ) ) +
     $                      CABS1( DL( I-1 ) )*CABS1( X( I-1, J ) ) +
     $                      CABS1( D( I ) )*CABS1( X( I, J ) ) +
     $                      CABS1( DU( I ) )*CABS1( X( I+1, J ) )
   30       CONTINUE
            RWORK( N ) = CABS1( B( N, J ) ) +
     $                   CABS1( DL( N-1 ) )*CABS1( X( N-1, J ) ) +
     $                   CABS1( D( N ) )*CABS1( X( N, J ) )
         END IF
      ELSE
         IF( N.EQ.1 ) THEN
            RWORK( 1 ) = CABS1( B( 1, J ) ) +
     $                   CABS1( D( 1 ) )*CABS1( X( 1, J ) )
         ELSE
            RWORK( 1 ) = CABS1( B( 1, J ) ) +
     $                   CABS1( D( 1 ) )*CABS1( X( 1, J ) ) +
     $                   CABS1( DL( 1 ) )*CABS1( X( 2, J ) )
            DO 40 I = 2, N - 1
               RWORK( I ) = CABS1( B( I, J ) ) +
     $                      CABS1( DU( I-1 ) )*CABS1( X( I-1, J ) ) +
     $                      CABS1( D( I ) )*CABS1( X( I, J ) ) +
     $                      CABS1( DL( I ) )*CABS1( X( I+1, J ) )
   40       CONTINUE
            RWORK( N ) = CABS1( B( N, J ) ) +
     $                   CABS1( DU( N-1 ) )*CABS1( X( N-1, J ) ) +
     $                   CABS1( D( N ) )*CABS1( X( N, J ) )
         END IF
      END IF
*
*     Compute componentwise relative backward error from formula
*
*     max(i) ( abs(R(i)) / ( abs(op(A))*abs(X) + abs(B) )(i) )
*
*     where abs(Z) is the componentwise absolute value of the matrix
*     or vector Z.  If the i-th component of the denominator is less
*     than SAFE2, then SAFE1 is added to the i-th components of the
*     numerator and denominator before dividing.
*
      S = ZERO
      DO 50 I = 1, N
         IF( RWORK( I ).GT.SAFE2 ) THEN
            S = MAX( S, CABS1( WORK( I ) ) / RWORK( I ) )
         ELSE
```

```
               S = MAX( S, ( CABS1( WORK( I ) )+SAFE1 ) /
     $             ( RWORK( I )+SAFE1 ) )
            END IF
   50    CONTINUE
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
*           1) The residual BERR(J) is larger than machine epsilon, and
*           2) BERR(J) decreased by at least a factor of 2 during the
*              last iteration, and
*           3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
*
*           Update solution and try again.
*
            CALL CGTTRS( TRANS, N, 1, DLF, DF, DUF, DU2, IPIV, WORK, N,
     $                   INFO )
            CALL CAXPY( N, CMPLX( ONE ), WORK, 1, X( 1, J ), 1 )
            LSTRES = BERR( J )
            COUNT = COUNT + 1
            GO TO 20
         END IF
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(op(A)))*
*           ( abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
*          inv(op(A)) is the inverse of op(A)
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(op(A))*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(op(A))*abs(X) + abs(B) is less than SAFE2.
*
*        Use CLACN2 to estimate the infinity-norm of the matrix
*           inv(op(A)) * diag(W),
*        where W = abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) )))
*
         DO 60 I = 1, N
            IF( RWORK( I ).GT.SAFE2 ) THEN
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I )
```

```
            ELSE
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I ) +
     $                      SAFE1
            END IF
   60    CONTINUE
*
         KASE = 0
   70    CONTINUE
         CALL CLACN2( N, WORK( N+1 ), WORK, FERR( J ), KASE, ISAVE )
         IF( KASE.NE.0 ) THEN
            IF( KASE.EQ.1 ) THEN
*
*              Multiply by diag(W)*inv(op(A)**H).
*
               CALL CGTTRS( TRANST, N, 1, DLF, DF, DUF, DU2, IPIV, WORK,
     $                      N, INFO )
               DO 80 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
   80          CONTINUE
            ELSE
*
*              Multiply by inv(op(A))*diag(W).
*
               DO 90 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
   90          CONTINUE
               CALL CGTTRS( TRANSN, N, 1, DLF, DF, DUF, DU2, IPIV, WORK,
     $                      N, INFO )
            END IF
            GO TO 70
         END IF
*
*        Normalize error.
*
         LSTRES = ZERO
         DO 100 I = 1, N
            LSTRES = MAX( LSTRES, CABS1( X( I, J ) ) )
  100    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  110 CONTINUE
*
      RETURN
*
*     End of CGTRFS
*
      END
```

———-

80.2.4 lisp code

— cgtrfs.lisp —

```
(let* ((itmax 5) (zero 0.0) (one 1.0) (two 2.0) (three 3.0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 3.0 3.0) three)
           (ignorable itmax zero one two three))
  (defun cgtrfs
         (trans n nrhs dl d du dlf df duf du2 ipiv b ldb$ x ldx ferr berr work
          rwork info)
    (declare (type (array single-float (*)) rwork berr ferr)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) work x b du2 duf df dlf du d
              dl)
             (type (f2cl-lib:integer4) info ldx ldb$ nrhs n)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (dl f2cl-lib:complex8 dl-%data% dl-%offset%)
         (d f2cl-lib:complex8 d-%data% d-%offset%)
         (du f2cl-lib:complex8 du-%data% du-%offset%)
         (dlf f2cl-lib:complex8 dlf-%data% dlf-%offset%)
         (df f2cl-lib:complex8 df-%data% df-%offset%)
         (duf f2cl-lib:complex8 duf-%data% duf-%offset%)
         (du2 f2cl-lib:complex8 du2-%data% du2-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (ferr single-float ferr-%data% ferr-%offset%)
         (berr single-float berr-%data% berr-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
          cabs1))
        (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
               (zdum #C(0.0 0.0)) (eps 0.0) (lstres 0.0) (s 0.0) (safe1 0.0)
               (safe2 0.0) (safmin 0.0) (count$ 0) (i 0) (j 0) (kase 0) (nz 0)
               (transn
                (make-array '(1) :element-type 'character :initial-element
                            #\ ))
               (transt
                (make-array '(1) :element-type 'character :initial-element
```

```
                  #\ ))
     (notran nil))
(declare (type (array f2cl-lib:integer4 (3)) isave)
         (type (f2cl-lib:complex8) zdum)
         (type (single-float) eps lstres s safe1 safe2 safmin)
         (type (f2cl-lib:integer4) count$ i j kase nz)
         (type (string 1) transn transt)
         (type f2cl-lib:logical notran))
(setf info 0)
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -13))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -15)))
(cond
 ((/= info 0) (xerbla "CGTRFS" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
              zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%)
              zero)
     label10))
  (go end_label)))
(cond
```

```
 (notran (f2cl-lib:f2cl-set-string transn "N" (string 1))
  (f2cl-lib:f2cl-set-string transt "C" (string 1)))
 (t (f2cl-lib:f2cl-set-string transn "C" (string 1))
  (f2cl-lib:f2cl-set-string transt "N" (string 1))))
(setf nz 4)
(setf eps (slamch "Epsilon"))
(setf safmin (slamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (ccopy n
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1 work 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11)
        (clagtm trans n 1 (- one) dl d du
         (f2cl-lib:array-slice x-%data% f2cl-lib:complex8 (1 j)
                               ((1 ldx) (1 *)) x-%offset%)
         ldx one work n)
      (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-10))
      (when var-0 (setf trans var-0))
      (when var-1 (setf n var-1))
      (when var-8 (setf ldx var-8))
      (when var-9 (setf one var-9))
      (when var-11 (setf n var-11)))
    (cond
     (notran
      (cond
       ((= n 1)
        (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
                (+
                 (cabs1
                  (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                                 b-%offset%))
                 (*
                  (cabs1
                   (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
                  (cabs1
                   (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                                  x-%offset%))))))
```

```
(t
 (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
         (+
          (cabs1
           (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                          b-%offset%))
          (*
           (cabs1
            (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
           (cabs1
            (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                           x-%offset%)))
          (*
           (cabs1
            (f2cl-lib:fref du-%data% (1) ((1 *)) du-%offset%))
           (cabs1
            (f2cl-lib:fref x-%data% (2 j) ((1 ldx) (1 *))
                           x-%offset%)))))
 (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
               ((> i
                   (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                          rwork-%offset%)
             (+
              (cabs1
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%))
              (*
               (cabs1
                (f2cl-lib:fref dl-%data%
                               ((f2cl-lib:int-sub i 1))
                               ((1 *)) dl-%offset%))
               (cabs1
                (f2cl-lib:fref x-%data%
                               ((f2cl-lib:int-sub i 1) j)
                               ((1 ldx) (1 *)) x-%offset%)))
              (*
               (cabs1
                (f2cl-lib:fref d-%data% (i) ((1 *))
                               d-%offset%))
               (cabs1
                (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                               x-%offset%)))
              (*
               (cabs1
                (f2cl-lib:fref du-%data% (i) ((1 *))
                               du-%offset%))
               (cabs1
```

```
                    (f2cl-lib:fref x-%data%
                                   ((f2cl-lib:int-add i 1) j)
                                   ((1 ldx) (1 *)) x-%offset%)))))
      label30))
    (setf (f2cl-lib:fref rwork-%data% (n) ((1 *)) rwork-%offset%)
          (+
           (cabs1
            (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                           b-%offset%))
           (*
            (cabs1
             (f2cl-lib:fref dl-%data% ((f2cl-lib:int-sub n 1))
                            ((1 *)) dl-%offset%))
            (cabs1
             (f2cl-lib:fref x-%data% ((f2cl-lib:int-sub n 1) j)
                            ((1 ldx) (1 *)) x-%offset%)))
           (*
            (cabs1
             (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%))
            (cabs1
             (f2cl-lib:fref x-%data% (n j) ((1 ldx) (1 *))
                            x-%offset%))))))))
  (t
   (cond
    ((= n 1)
     (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
           (+
            (cabs1
             (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                            b-%offset%))
            (*
             (cabs1
              (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
             (cabs1
              (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                             x-%offset%))))))
    (t
     (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
           (+
            (cabs1
             (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                            b-%offset%))
            (*
             (cabs1
              (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
             (cabs1
              (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                             x-%offset%)))
            (*
             (cabs1
```

```
            (f2cl-lib:fref dl-%data% (1) ((1 *)) dl-%offset%))
           (cabs1
            (f2cl-lib:fref x-%data% (2 j) ((1 ldx) (1 *))
                           x-%offset%)))))
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i
                  (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                         rwork-%offset%)
            (+
             (cabs1
              (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%))
             (*
              (cabs1
               (f2cl-lib:fref du-%data%
                              ((f2cl-lib:int-sub i 1))
                              ((1 *)) du-%offset%))
              (cabs1
               (f2cl-lib:fref x-%data%
                              ((f2cl-lib:int-sub i 1) j)
                              ((1 ldx) (1 *)) x-%offset%)))
             (*
              (cabs1
               (f2cl-lib:fref d-%data% (i) ((1 *))
                              d-%offset%))
              (cabs1
               (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                              x-%offset%)))
             (*
              (cabs1
               (f2cl-lib:fref dl-%data% (i) ((1 *))
                              dl-%offset%))
              (cabs1
               (f2cl-lib:fref x-%data%
                              ((f2cl-lib:int-add i 1) j)
                              ((1 ldx) (1 *)) x-%offset%)))))
   label40))
(setf (f2cl-lib:fref rwork-%data% (n) ((1 *)) rwork-%offset%)
        (+
         (cabs1
          (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                         b-%offset%))
         (*
          (cabs1
           (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub n 1))
                          ((1 *)) du-%offset%))
          (cabs1
```

```
                 (f2cl-lib:fref x-%data% ((f2cl-lib:int-sub n 1) j)
                                ((1 ldx) (1 *)) x-%offset%)))
               (*
                (cabs1
                 (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%))
                (cabs1
                 (f2cl-lib:fref x-%data% (n j) ((1 ldx) (1 *))
                                x-%offset%)))))))))
(setf s zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
      (setf s
              (max s
                   (/
                    (cabs1
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%))
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%)))))
     (t
      (setf s
              (max s
                   (/
                    (+
                     (cabs1
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%))
                     safe1)
                    (+
                     (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                    rwork-%offset%)
                     safe1))))))
   label50))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (cgttrs trans n 1 dlf df duf du2 ipiv work n info)
    (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-8))
    (when var-0 (setf trans var-0))
    (when var-1 (setf n var-1))
    (when var-9 (setf n var-9))
    (when var-10 (setf info var-10)))
```

```
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (caxpy n (f2cl-lib:cmplx one) work 1
         (f2cl-lib:array-slice x-%data% f2cl-lib:complex8 (1 j)
                               ((1 ldx) (1 *)) x-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4 var-5))
      (when var-0 (setf n var-0)))
    (setf lstres
            (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
    (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                             rwork-%offset%)
                (+
                 (cabs1
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%))
                 (* nz eps
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%)))))
       (t
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                             rwork-%offset%)
                (+
                 (cabs1
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%))
                 (* nz eps
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%))
                 safe1))))
     label60))
  (setf kase 0)
 label70
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (clacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             ((+ n 1)) ((1 *)) work-%offset%)
       work
       (f2cl-lib:array-slice ferr-%data% single-float (j) ((1 *))
                             ferr-%offset%)
       kase isave)
    (declare (ignore var-1 var-2 var-3 var-5))
    (when var-0 (setf n var-0))
    (when var-4 (setf kase var-4)))
  (cond
```

```
((/= kase 0)
 (cond
  ((= kase 1)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (cgttrs transt n 1 dlf df duf du2 ipiv work n info)
     (declare
      (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-8))
     (when var-0 (setf transt var-0))
     (when var-1 (setf n var-1))
     (when var-9 (setf n var-9))
     (when var-10 (setf info var-10)))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                            work-%offset%)
               (*
                (f2cl-lib:fref rwork-%data% (i) ((1 *))
                               rwork-%offset%)
                (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)))
      label80)))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                            work-%offset%)
               (*
                (f2cl-lib:fref rwork-%data% (i) ((1 *))
                               rwork-%offset%)
                (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)))
      label90))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (cgttrs transn n 1 dlf df duf du2 ipiv work n info)
     (declare
      (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-8))
     (when var-0 (setf transn var-0))
     (when var-1 (setf n var-1))
     (when var-9 (setf n var-9))
     (when var-10 (setf info var-10)))))
 (go label70)))
(setf lstres zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
```

```
                (tagbody
                  (setf lstres
                          (max lstres
                               (cabs1
                                (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                               x-%offset%))))
                 label100))
               (if (/= lstres zero)
                   (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                          (/
                           (f2cl-lib:fref ferr-%data% (j) ((1 *))
                                          ferr-%offset%)
                           lstres)))
             label110))
           (go end_label)
          end_label
           (return
            (values trans n nil nil nil nil nil nil nil nil nil nil nil ldx
                    nil nil nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgtrfs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
```

```
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::complex8
   (*))
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::trans
  fortran-to-lisp::n nil nil nil
  nil nil nil nil nil nil nil nil
  nil fortran-to-lisp::ldx nil nil
  nil nil fortran-to-lisp::info)
:calls 'nil)))
```

———-

80.3 cgttrf

80.3.1 Axiom unit tests

— **cgttrf.input** —

```
)set break resume
)sys rm -f cgttrf.output
)spool cgttrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

80.3.2 Axiom help page

— **cgttrf.help** —

```
==========================================================================
cgttrf examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgttrf (N, DL, D, DU, DU2, IPIV, INFO)
           CGTTRF

Function/Subroutine Documentation
   subroutine cgttrf (integer N, complex, dimension( * ) DL, complex,
       dimension( * ) D, complex, dimension( * ) DU, complex, dimension( * )
       DU2, integer, dimension( * ) IPIV, integer INFO)
       CGTTRF

Purpose:
   CGTTRF computes an LU factorization of a complex tridiagonal matrix A
   using elimination with partial pivoting and row interchanges.

   The factorization has the form
     A = L * U
   where L is a product of permutation and unit lower bidiagonal
   matrices and U is upper triangular with nonzeros in only the main
   diagonal and first two superdiagonals.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.

 DL
   DL is COMPLEX array, dimension (N-1)
   On entry, DL must contain the (n-1) sub-diagonal elements of A.

   On exit, DL is overwritten by the (n-1) multipliers that
   define the matrix L from the LU factorization of A.

 D
   D is COMPLEX array, dimension (N)
   On entry, D must contain the diagonal elements of A.

   On exit, D is overwritten by the n diagonal elements of the
   upper triangular matrix U from the LU factorization of A.

 DU
   DU is COMPLEX array, dimension (N-1)
   On entry, DU must contain the (n-1) super-diagonal elements of A.
```

```
  On exit, DU is overwritten by the (n-1) elements of the first
  super-diagonal of U.

DU2
  DU2 is COMPLEX array, dimension (N-2)
  On exit, DU2 is overwritten by the (n-2) elements of the
  second super-diagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= n, row i of the matrix was
  interchanged with row IPIV(i).  IPIV(i) will always be either
  i or i+1; IPIV(i) = i indicates a row interchange was not required.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -k, the k-th argument had an illegal value
  > 0:  if INFO = k, U(k,k) is exactly zero. The factorization
        has been completed, but the factor U is exactly
        singular, and division by zero will occur if it is used
        to solve a system of equations.
```

———-

80.3.3 fortran code

— cgttrf.f —

```
*  =====================================================================
      SUBROUTINE CGTTRF( N, DL, D, DU, DU2, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            D( * ), DL( * ), DU( * ), DU2( * )
*     ..
*
*  =====================================================================
*
```

```
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      COMPLEX            FACT, TEMP, ZDUM
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, REAL
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
         CALL XERBLA( 'CGTTRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Initialize IPIV(i) = i and DU2(i) = 0
*
      DO 10 I = 1, N
         IPIV( I ) = I
   10 CONTINUE
      DO 20 I = 1, N - 2
         DU2( I ) = ZERO
   20 CONTINUE
*
      DO 30 I = 1, N - 2
         IF( CABS1( D( I ) ).GE.CABS1( DL( I ) ) ) THEN
*
*           No row interchange required, eliminate DL(I)
*
            IF( CABS1( D( I ) ).NE.ZERO ) THEN
               FACT = DL( I ) / D( I )
```

```
            DL( I ) = FACT
            D( I+1 ) = D( I+1 ) - FACT*DU( I )
         END IF
      ELSE
*
*        Interchange rows I and I+1, eliminate DL(I)
*
         FACT = D( I ) / DL( I )
         D( I ) = DL( I )
         DL( I ) = FACT
         TEMP = DU( I )
         DU( I ) = D( I+1 )
         D( I+1 ) = TEMP - FACT*D( I+1 )
         DU2( I ) = DU( I+1 )
         DU( I+1 ) = -FACT*DU( I+1 )
         IPIV( I ) = I + 1
      END IF
   30 CONTINUE
      IF( N.GT.1 ) THEN
         I = N - 1
         IF( CABS1( D( I ) ).GE.CABS1( DL( I ) ) ) THEN
            IF( CABS1( D( I ) ).NE.ZERO ) THEN
               FACT = DL( I ) / D( I )
               DL( I ) = FACT
               D( I+1 ) = D( I+1 ) - FACT*DU( I )
            END IF
         ELSE
            FACT = D( I ) / DL( I )
            D( I ) = DL( I )
            DL( I ) = FACT
            TEMP = DU( I )
            DU( I ) = D( I+1 )
            D( I+1 ) = TEMP - FACT*D( I+1 )
            IPIV( I ) = I + 1
         END IF
      END IF
*
*     Check for a zero on the diagonal of U.
*
      DO 40 I = 1, N
         IF( CABS1( D( I ) ).EQ.ZERO ) THEN
            INFO = I
            GO TO 50
         END IF
   40 CONTINUE
   50 CONTINUE
*
      RETURN
*
*     End of CGTTRF
```

```
*
      END
```

———-

80.3.4 lisp code

— **cgttrf.lisp** —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun cgttrf (n dl d du du2 ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex8 (*)) du2 du d dl)
             (type (f2cl-lib:integer4) info n))
    (f2cl-lib:with-multi-array-data
        ((dl f2cl-lib:complex8 dl-%data% dl-%offset%)
         (d f2cl-lib:complex8 d-%data% d-%offset%)
         (du f2cl-lib:complex8 du-%data% du-%offset%)
         (du2 f2cl-lib:complex8 du2-%data% du2-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
          cabs1))
        (prog ((fact #C(0.0 0.0)) (temp #C(0.0 0.0)) (zdum #C(0.0 0.0)) (i 0))
          (declare (type (f2cl-lib:complex8) fact temp zdum)
                   (type (f2cl-lib:integer4) i))
          (setf info 0)
          (cond
           ((< n 0) (setf info -1) (xerbla "CGTTRF" (f2cl-lib:int-sub info))
            (go end_label)))
          (if (= n 0)
              (go end_label))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) i)
             label10))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))) nil)
            (tagbody
              (setf (f2cl-lib:fref du2-%data% (i) ((1 *)) du2-%offset%)
                      (coerce zero 'f2cl-lib:complex8))
             label20))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))) nil)
  (tagbody
    (cond
     ((>= (cabs1 (f2cl-lib:fref d (i) ((1 *))))
          (cabs1 (f2cl-lib:fref dl (i) ((1 *)))))
      (cond
       ((/= (cabs1 (f2cl-lib:fref d (i) ((1 *)))) zero)
        (setf fact
                (/ (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                   (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
        (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) fact)
        (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                              ((1 *)) d-%offset%)
                (-
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) d-%offset%)
                 (* fact
                    (f2cl-lib:fref du-%data% (i) ((1 *))
                                    du-%offset%)))))))
     (t
      (setf fact
              (/ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                 (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
      (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
              (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
      (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) fact)
      (setf temp (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%))
      (setf (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
              (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                              ((1 *)) d-%offset%))
      (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                            d-%offset%)
              (- temp
                 (* fact
                    (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                    ((1 *)) d-%offset%))))
      (setf (f2cl-lib:fref du2-%data% (i) ((1 *)) du2-%offset%)
              (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1))
                              ((1 *)) du-%offset%))
      (setf (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                            du-%offset%)
              (* (- fact)
                 (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) du-%offset%)))
      (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
              (f2cl-lib:int-add i 1))))
   label30))
(cond
 ((> n 1) (setf i (f2cl-lib:int-sub n 1))
  (cond
```

```
            ((>= (cabs1 (f2cl-lib:fref d (i) ((1 *))))
                 (cabs1 (f2cl-lib:fref dl (i) ((1 *)))))
             (cond
              ((/= (cabs1 (f2cl-lib:fref d (i) ((1 *)))) zero)
               (setf fact
                       (/ (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                          (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
               (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) fact)
               (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                     d-%offset%)
                       (-
                        (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                        ((1 *)) d-%offset%)
                        (* fact
                           (f2cl-lib:fref du-%data% (i) ((1 *))
                                           du-%offset%)))))))
            (t
             (setf fact
                     (/ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                        (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
             (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                     (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
             (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) fact)
             (setf temp (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%))
             (setf (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                     (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                     d-%offset%))
             (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                    d-%offset%)
                     (- temp
                        (* fact
                           (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                           ((1 *)) d-%offset%))))
             (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
                     (f2cl-lib:int-add i 1))))))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (cond
               ((= (cabs1 (f2cl-lib:fref d (i) ((1 *)))) zero) (setf info i)
                (go label50)))
             label40))
         label50
          (go end_label)
         end_label
          (return (values nil nil nil nil nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
```

```
(setf (gethash 'fortran-to-lisp::cgttrf
               fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((fortran-to-lisp::integer4)
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (array fortran-to-lisp::complex8
                                             (*))
                                            (array fortran-to-lisp::integer4
                                             (*))
                                            (fortran-to-lisp::integer4))
                                          :return-values
                                          '(nil nil nil nil nil nil
                                            fortran-to-lisp::info)
                                          :calls 'nil)))
```

———-

80.4 cgttrs

80.4.1 Axiom unit tests

— **cgttrs.input** —

```
)set break resume
)sys rm -f cgttrs.output
)spool cgttrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

80.4.2 Axiom help page

— **cgttrs.help** —

```
====================================================================
cgttrs examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgttrs (TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB, INFO)
           CGTTRS

Function/Subroutine Documentation
   subroutine cgttrs (character TRANS, integer N, integer NRHS, complex,
       dimension( * ) DL, complex, dimension( * ) D, complex, dimension( * )
       DU, complex, dimension( * ) DU2, integer, dimension( * ) IPIV, complex,
       dimension( ldb, * ) B, integer LDB, integer INFO)
       CGTTRS

Purpose:
   CGTTRS solves one of the systems of equations
      A * X = B,  A**T * X = B,  or  A**H * X = B,
   with a tridiagonal matrix A using the LU factorization computed
   by CGTTRF.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations.
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose)

 N
   N is INTEGER
   The order of the matrix A.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 DL
   DL is COMPLEX array, dimension (N-1)
   The (n-1) multipliers that define the matrix L from the
   LU factorization of A.

 D
```

```
  D is COMPLEX array, dimension (N)
  The n diagonal elements of the upper triangular matrix U from
  the LU factorization of A.

DU
  DU is COMPLEX array, dimension (N-1)
  The (n-1) elements of the first super-diagonal of U.

DU2
  DU2 is COMPLEX array, dimension (N-2)
  The (n-2) elements of the second super-diagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= n, row i of the matrix was
  interchanged with row IPIV(i).  IPIV(i) will always be either
  i or i+1; IPIV(i) = i indicates a row interchange was not required.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the matrix of right hand side vectors B.
  On exit, B is overwritten by the solution vectors X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -k, the k-th argument had an illegal value
```

———-

80.4.3 fortran code

— **cgttrs.f** —

```
*  =====================================================================
      SUBROUTINE CGTTRS( TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
```

```
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            B( LDB, * ), D( * ), DL( * ), DU( * ), DU2( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      INTEGER            ITRANS, J, JB, NB
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           CGTTS2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOTRAN = ( TRANS.EQ.'N' .OR. TRANS.EQ.'n' )
      IF( .NOT.NOTRAN .AND. .NOT.( TRANS.EQ.'T' .OR. TRANS.EQ.
     $    't' ) .AND. .NOT.( TRANS.EQ.'C' .OR. TRANS.EQ.'c' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDB.LT.MAX( N, 1 ) ) THEN
         INFO = -10
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGTTRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
*     Decode TRANS
```

```
*
      IF( NOTRAN ) THEN
         ITRANS = 0
      ELSE IF( TRANS.EQ.'T' .OR. TRANS.EQ.'t' ) THEN
         ITRANS = 1
      ELSE
         ITRANS = 2
      END IF
*
*     Determine the number of right-hand sides to solve at a time.
*
      IF( NRHS.EQ.1 ) THEN
         NB = 1
      ELSE
         NB = MAX( 1, ILAENV( 1, 'CGTTRS', TRANS, N, NRHS, -1, -1 ) )
      END IF
*
      IF( NB.GE.NRHS ) THEN
         CALL CGTTS2( ITRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB )
      ELSE
         DO 10 J = 1, NRHS, NB
            JB = MIN( NRHS-J+1, NB )
            CALL CGTTS2( ITRANS, N, JB, DL, D, DU, DU2, IPIV, B( 1, J ),
     $                   LDB )
   10    CONTINUE
      END IF
*
*     End of CGTTRS
*
      END
```

———-

80.4.4 lisp code

— **cgttrs.lisp** —

```
(defun cgttrs (trans n nrhs dl d du du2 ipiv b ldb$ info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex8 (*)) b du2 du d dl)
           (type (f2cl-lib:integer4) info ldb$ nrhs n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
       (dl f2cl-lib:complex8 dl-%data% dl-%offset%)
       (d f2cl-lib:complex8 d-%data% d-%offset%)
       (du f2cl-lib:complex8 du-%data% du-%offset%)
       (du2 f2cl-lib:complex8 du2-%data% du2-%offset%)
```

```
     (b f2cl-lib:complex8 b-%data% b-%offset%)
     (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
   (prog ((itrans 0) (j 0) (jb 0) (nb 0) (notran nil))
     (declare (type f2cl-lib:logical notran)
              (type (f2cl-lib:integer4) nb jb j itrans))
     (setf info 0)
     (setf notran
             (or (f2cl-lib:fstring-= trans "N")
                 (f2cl-lib:fstring-= trans "n")))
     (cond
      ((and (not notran)
            (not
             (or (f2cl-lib:fstring-= trans "T")
                 (f2cl-lib:fstring-= trans "t")))
            (not
             (or (f2cl-lib:fstring-= trans "C")
                 (f2cl-lib:fstring-= trans "c"))))
       (setf info -1))
      ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
      ((< ldb$ (max (the f2cl-lib:integer4 n) (the f2cl-lib:integer4 1)))
       (setf info -10)))
     (cond
      ((/= info 0) (xerbla "CGTTRS" (f2cl-lib:int-sub info)) (go end_label)))
     (if (or (= n 0) (= nrhs 0))
         (go end_label))
     (cond (notran (setf itrans 0))
           ((or (f2cl-lib:fstring-= trans "T") (f2cl-lib:fstring-= trans "t"))
            (setf itrans 1))
           (t (setf itrans 2)))
     (cond ((= nrhs 1) (setf nb 1))
           (t
            (setf nb
                    (max (the f2cl-lib:integer4 1)
                         (the f2cl-lib:integer4
                              (multiple-value-bind
                                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                                   var-6)
                                  (ilaenv 1 "CGTTRS" trans n nrhs -1 -1)
                                (declare (ignore var-0 var-1 var-5 var-6))
                                (when var-2 (setf trans var-2))
                                (when var-3 (setf n var-3))
                                (when var-4 (setf nrhs var-4))
                                ret-val))))))
     (cond
      ((>= nb nrhs)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
           (cgtts2 itrans n nrhs dl d du du2 ipiv b ldb$)
         (declare (ignore var-3 var-4 var-5 var-6 var-7 var-8))
         (when var-0 (setf itrans var-0))
```

```
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-9 (setf ldb$ var-9))))
       (t
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                      ((> j nrhs) nil)
          (tagbody
            (setf jb
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-sub nrhs j) 1))
                     (the f2cl-lib:integer4 nb)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                (cgtts2 itrans n jb dl d du du2 ipiv
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$)
              (declare (ignore var-3 var-4 var-5 var-6 var-7 var-8))
              (when var-0 (setf itrans var-0))
              (when var-1 (setf n var-1))
              (when var-2 (setf jb var-2))
              (when var-9 (setf ldb$ var-9)))
           label10))))
     end_label
      (return (values trans n nrhs nil nil nil nil nil nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgttrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
```

```
                                                    :return-values
                                                    '(fortran-to-lisp::trans
                                                      fortran-to-lisp::n
                                                      fortran-to-lisp::nrhs nil nil nil
                                                      nil nil nil fortran-to-lisp::ldb$
                                                      fortran-to-lisp::info)
                                                    :calls 'nil)))
```

———-

80.5 cgtts2

80.5.1 Axiom unit tests

— **cgtts2.input** —

```
)set break resume
)sys rm -f cgtts2.output
)spool cgtts2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

80.5.2 Axiom help page

— **cgtts2.help** —

```
====================================================================
cgtts2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
```

```
      subroutine cgtts2 (ITRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB)
          CGTTS2 solves a system of linear equations with a tridiagonal
          matrix using the LU factorization computed by sgttrf.

Function/Subroutine Documentation
   subroutine cgtts2 (integer ITRANS, integer N, integer NRHS, complex,
       dimension( * ) DL, complex, dimension( * ) D, complex, dimension( * )
       DU, complex, dimension( * ) DU2, integer, dimension( * ) IPIV, complex,
       dimension( ldb, * ) B, integer LDB)
       CGTTS2 solves a system of linear equations with a tridiagonal matrix
       using the LU factorization computed by sgttrf.

Purpose:
   CGTTS2 solves one of the systems of equations
      A * X = B,  A**T * X = B,  or  A**H * X = B,
   with a tridiagonal matrix A using the LU factorization computed
   by CGTTRF.

Parameters:

 ITRANS
   ITRANS is INTEGER
   Specifies the form of the system of equations.
   = 0:  A * X = B     (No transpose)
   = 1:  A**T * X = B  (Transpose)
   = 2:  A**H * X = B  (Conjugate transpose)

 N
   N is INTEGER
   The order of the matrix A.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 DL
   DL is COMPLEX array, dimension (N-1)
   The (n-1) multipliers that define the matrix L from the
   LU factorization of A.

 D
   D is COMPLEX array, dimension (N)
   The n diagonal elements of the upper triangular matrix U from
   the LU factorization of A.

 DU
   DU is COMPLEX array, dimension (N-1)
   The (n-1) elements of the first super-diagonal of U.
```

```
DU2
  DU2 is COMPLEX array, dimension (N-2)
  The (n-2) elements of the second super-diagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= n, row i of the matrix was
  interchanged with row IPIV(i).  IPIV(i) will always be either
  i or i+1; IPIV(i) = i indicates a row interchange was not required.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the matrix of right hand side vectors B.
  On exit, B is overwritten by the solution vectors X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).
```

80.5.3 fortran code

— cgtts2.f —

```
*  =====================================================================
      SUBROUTINE CGTTS2( ITRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            ITRANS, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX            B( LDB, * ), D( * ), DL( * ), DU( * ), DU2( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
      COMPLEX            TEMP
*     ..
```

```
*     .. Intrinsic Functions ..
      INTRINSIC          CONJG
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( ITRANS.EQ.0 ) THEN
*
*        Solve A*X = B using the LU factorization of A,
*        overwriting each right hand side vector with its solution.
*
         IF( NRHS.LE.1 ) THEN
            J = 1
   10       CONTINUE
*
*           Solve L*x = b.
*
            DO 20 I = 1, N - 1
               IF( IPIV( I ).EQ.I ) THEN
                  B( I+1, J ) = B( I+1, J ) - DL( I )*B( I, J )
               ELSE
                  TEMP = B( I, J )
                  B( I, J ) = B( I+1, J )
                  B( I+1, J ) = TEMP - DL( I )*B( I, J )
               END IF
   20       CONTINUE
*
*           Solve U*x = b.
*
            B( N, J ) = B( N, J ) / D( N )
            IF( N.GT.1 )
     $         B( N-1, J ) = ( B( N-1, J )-DU( N-1 )*B( N, J ) ) /
     $                       D( N-1 )
            DO 30 I = N - 2, 1, -1
               B( I, J ) = ( B( I, J )-DU( I )*B( I+1, J )-DU2( I )*
     $                     B( I+2, J ) ) / D( I )
   30       CONTINUE
            IF( J.LT.NRHS ) THEN
               J = J + 1
               GO TO 10
            END IF
         ELSE
            DO 60 J = 1, NRHS
*
*           Solve L*x = b.
*
```

```
               DO 40 I = 1, N - 1
                  IF( IPIV( I ).EQ.I ) THEN
                     B( I+1, J ) = B( I+1, J ) - DL( I )*B( I, J )
                  ELSE
                     TEMP = B( I, J )
                     B( I, J ) = B( I+1, J )
                     B( I+1, J ) = TEMP - DL( I )*B( I, J )
                  END IF
   40          CONTINUE
*
*          Solve U*x = b.
*
               B( N, J ) = B( N, J ) / D( N )
               IF( N.GT.1 )
     $            B( N-1, J ) = ( B( N-1, J )-DU( N-1 )*B( N, J ) ) /
     $                          D( N-1 )
               DO 50 I = N - 2, 1, -1
                  B( I, J ) = ( B( I, J )-DU( I )*B( I+1, J )-DU2( I )*
     $                        B( I+2, J ) ) / D( I )
   50          CONTINUE
   60       CONTINUE
         END IF
      ELSE IF( ITRANS.EQ.1 ) THEN
*
*        Solve A**T * X = B.
*
         IF( NRHS.LE.1 ) THEN
            J = 1
   70       CONTINUE
*
*           Solve U**T * x = b.
*
            B( 1, J ) = B( 1, J ) / D( 1 )
            IF( N.GT.1 )
     $         B( 2, J ) = ( B( 2, J )-DU( 1 )*B( 1, J ) ) / D( 2 )
            DO 80 I = 3, N
               B( I, J ) = ( B( I, J )-DU( I-1 )*B( I-1, J )-DU2( I-2 )*
     $                     B( I-2, J ) ) / D( I )
   80       CONTINUE
*
*           Solve L**T * x = b.
*
            DO 90 I = N - 1, 1, -1
               IF( IPIV( I ).EQ.I ) THEN
                  B( I, J ) = B( I, J ) - DL( I )*B( I+1, J )
               ELSE
                  TEMP = B( I+1, J )
                  B( I+1, J ) = B( I, J ) - DL( I )*TEMP
                  B( I, J ) = TEMP
               END IF
```

```
   90       CONTINUE
            IF( J.LT.NRHS ) THEN
               J = J + 1
               GO TO 70
            END IF
         ELSE
            DO 120 J = 1, NRHS
*
*           Solve U**T * x = b.
*
               B( 1, J ) = B( 1, J ) / D( 1 )
               IF( N.GT.1 )
     $            B( 2, J ) = ( B( 2, J )-DU( 1 )*B( 1, J ) ) / D( 2 )
               DO 100 I = 3, N
                  B( I, J ) = ( B( I, J )-DU( I-1 )*B( I-1, J )-
     $                        DU2( I-2 )*B( I-2, J ) ) / D( I )
  100          CONTINUE
*
*           Solve L**T * x = b.
*
               DO 110 I = N - 1, 1, -1
                  IF( IPIV( I ).EQ.I ) THEN
                     B( I, J ) = B( I, J ) - DL( I )*B( I+1, J )
                  ELSE
                     TEMP = B( I+1, J )
                     B( I+1, J ) = B( I, J ) - DL( I )*TEMP
                     B( I, J ) = TEMP
                  END IF
  110          CONTINUE
  120       CONTINUE
         END IF
      ELSE
*
*        Solve A**H * X = B.
*
         IF( NRHS.LE.1 ) THEN
            J = 1
  130       CONTINUE
*
*           Solve U**H * x = b.
*
            B( 1, J ) = B( 1, J ) / CONJG( D( 1 ) )
            IF( N.GT.1 )
     $         B( 2, J ) = ( B( 2, J )-CONJG( DU( 1 ) )*B( 1, J ) ) /
     $                     CONJG( D( 2 ) )
            DO 140 I = 3, N
               B( I, J ) = ( B( I, J )-CONJG( DU( I-1 ) )*B( I-1, J )-
     $                     CONJG( DU2( I-2 ) )*B( I-2, J ) ) /
     $                     CONJG( D( I ) )
  140       CONTINUE
```

```
*
*          Solve L**H * x = b.
*
           DO 150 I = N - 1, 1, -1
              IF( IPIV( I ).EQ.I ) THEN
                 B( I, J ) = B( I, J ) - CONJG( DL( I ) )*B( I+1, J )
              ELSE
                 TEMP = B( I+1, J )
                 B( I+1, J ) = B( I, J ) - CONJG( DL( I ) )*TEMP
                 B( I, J ) = TEMP
              END IF
  150      CONTINUE
           IF( J.LT.NRHS ) THEN
              J = J + 1
              GO TO 130
           END IF
        ELSE
           DO 180 J = 1, NRHS
*
*          Solve U**H * x = b.
*
              B( 1, J ) = B( 1, J ) / CONJG( D( 1 ) )
              IF( N.GT.1 )
     $           B( 2, J ) = ( B( 2, J )-CONJG( DU( 1 ) )*B( 1, J ) ) /
     $                       CONJG( D( 2 ) )
              DO 160 I = 3, N
                 B( I, J ) = ( B( I, J )-CONJG( DU( I-1 ) )*
     $                       B( I-1, J )-CONJG( DU2( I-2 ) )*
     $                       B( I-2, J ) ) / CONJG( D( I ) )
  160         CONTINUE
*
*          Solve L**H * x = b.
*
              DO 170 I = N - 1, 1, -1
                 IF( IPIV( I ).EQ.I ) THEN
                    B( I, J ) = B( I, J ) - CONJG( DL( I ) )*
     $                          B( I+1, J )
                 ELSE
                    TEMP = B( I+1, J )
                    B( I+1, J ) = B( I, J ) - CONJG( DL( I ) )*TEMP
                    B( I, J ) = TEMP
                 END IF
  170         CONTINUE
  180      CONTINUE
        END IF
     END IF
*
*    End of CGTTS2
*
     END
```

———-

80.5.4 lisp code

— **cgtts2.lisp** —

```
(defun cgtts2 (itrans n nrhs dl d du du2 ipiv b ldb$)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex8 (*)) b du2 du d dl)
           (type (f2cl-lib:integer4) ldb$ nrhs n itrans))
  (f2cl-lib:with-multi-array-data
      ((dl f2cl-lib:complex8 dl-%data% dl-%offset%)
       (d f2cl-lib:complex8 d-%data% d-%offset%)
       (du f2cl-lib:complex8 du-%data% du-%offset%)
       (du2 f2cl-lib:complex8 du2-%data% du2-%offset%)
       (b f2cl-lib:complex8 b-%data% b-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((temp #C(0.0 0.0)) (i 0) (j 0))
      (declare (type (f2cl-lib:integer4) j i)
               (type (f2cl-lib:complex8) temp))
      (if (or (= n 0) (= nrhs 0))
          (go end_label))
      (cond
       ((= itrans 0)
        (cond
         ((<= nrhs 1)
          (tagbody
            (setf j 1)
           label10
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
              (tagbody
                (cond
                 ((= (f2cl-lib:fref ipiv (i) ((1 *))) i)
                  (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                        (-
                         (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                        ((1 ldb$) (1 *)) b-%offset%)
                         (* (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                            (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                           b-%offset%)))))
                 (t
                  (setf temp
                        (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%))
                  (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)
```

```
                    (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                   ((1 ldb$) (1 *)) b-%offset%))
            (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                    (- temp
                       (*
                        (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                        (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%))))))
        label20))
    (setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
            (/
             (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
             (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
    (if (> n 1)
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                              ((1 ldb$) (1 *)) b-%offset%)
                (/
                 (-
                  (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                  (*
                   (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub n 1))
                                  ((1 *)) du-%offset%)
                   (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                                  b-%offset%)))
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                ((1 *)) d-%offset%))))
    (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))
                   (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
      (tagbody
        (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)
                (/
                 (-
                  (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (* (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                    ((1 ldb$) (1 *)) b-%offset%))
                  (*
                   (f2cl-lib:fref du2-%data% (i) ((1 *)) du2-%offset%)
                   (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 2) j)
                                  ((1 ldb$) (1 *)) b-%offset%)))
                 (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
        label30))
    (cond ((< j nrhs) (setf j (f2cl-lib:int-add j 1)) (go label10)))))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
            ((> j nrhs) nil)
(tagbody
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref ipiv (i) ((1 *))) i)
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                              ((1 ldb$) (1 *)) b-%offset%)
                (-
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                 (*
                  (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                  (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)))))
       (t
        (setf temp
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%))
        (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)
                (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                               ((1 ldb$) (1 *)) b-%offset%))
        (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                              ((1 ldb$) (1 *)) b-%offset%)
                (- temp
                   (*
                    (f2cl-lib:fref dl-%data% (i) ((1 *))
                                   dl-%offset%)
                    (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%))))))
    label40))
  (setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
          (/
           (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                          b-%offset%)
           (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
  (if (> n 1)
      (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                            ((1 ldb$) (1 *)) b-%offset%)
              (/
               (-
                (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                               ((1 ldb$) (1 *)) b-%offset%)
                (*
                 (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub n 1))
                                 ((1 *)) du-%offset%)
                 (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
```

```
                                        b-%offset%)))
                        (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                       ((1 *)) d-%offset%))))
        (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))
                       (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (/
                   (-
                    (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)
                    (*
                     (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                     ((1 ldb$) (1 *)) b-%offset%))
                    (*
                     (f2cl-lib:fref du2-%data% (i) ((1 *))
                                     du2-%offset%)
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 2) j)
                                     ((1 ldb$) (1 *)) b-%offset%)))
                   (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
          label50))
      label60)))))
((= itrans 1)
 (cond
  ((<= nrhs 1)
   (tagbody
     (setf j 1)
    label70
     (setf (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *)) b-%offset%)
             (/
              (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *)) b-%offset%)
              (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)))
     (if (> n 1)
         (setf (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                              b-%offset%)
               (/
                (-
                 (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                                b-%offset%)
                 (* (f2cl-lib:fref du-%data% (1) ((1 *)) du-%offset%)
                    (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                                    b-%offset%)))
                (f2cl-lib:fref d-%data% (2) ((1 *)) d-%offset%))))
     (f2cl-lib:fdo (i 3 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
```

```
                                b-%offset%)
                        (/
                         (-
                          (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                         b-%offset%)
                          (*
                           (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub i 1))
                                          ((1 *)) du-%offset%)
                           (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                          ((1 ldb$) (1 *)) b-%offset%))
                          (*
                           (f2cl-lib:fref du2-%data% ((f2cl-lib:int-sub i 2))
                                          ((1 *)) du2-%offset%)
                           (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 2) j)
                                          ((1 ldb$) (1 *)) b-%offset%)))
                         (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
        label80))
    (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
      (tagbody
        (cond
         ((= (f2cl-lib:fref ipiv (i) ((1 *))) i)
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)
                  (-
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)
                   (* (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                      (f2cl-lib:fref b-%data%
                                     ((f2cl-lib:int-add i 1) j)
                                     ((1 ldb$) (1 *)) b-%offset%)))))
         (t
          (setf temp
                  (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%))
          (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                               ((1 ldb$) (1 *)) b-%offset%)
                  (-
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)
                   (* (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                      temp)))
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)
                  temp)))
        label90))
    (cond ((< j nrhs) (setf j (f2cl-lib:int-add j 1)) (go label70)))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
            ((> j nrhs) nil)
(tagbody
  (setf (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *)) b-%offset%)
          (/
           (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                          b-%offset%)
           (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)))
  (if (> n 1)
      (setf (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                           b-%offset%)
            (/
             (-
              (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                             b-%offset%)
              (*
               (f2cl-lib:fref du-%data% (1) ((1 *)) du-%offset%)
               (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                              b-%offset%)))
             (f2cl-lib:fref d-%data% (2) ((1 *)) d-%offset%))))
  (f2cl-lib:fdo (i 3 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                           b-%offset%)
            (/
             (-
              (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)
              (*
               (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub i 1))
                              ((1 *)) du-%offset%)
               (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                              ((1 ldb$) (1 *)) b-%offset%))
              (*
               (f2cl-lib:fref du2-%data% ((f2cl-lib:int-sub i 2))
                              ((1 *)) du2-%offset%)
               (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 2) j)
                              ((1 ldb$) (1 *)) b-%offset%)))
             (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
     label100))
  (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                 (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                ((> i 1) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref ipiv (i) ((1 *))) i)
        (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)
              (-
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
```

```
                                        b-%offset%)
                          (*
                           (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                           (f2cl-lib:fref b-%data%
                                          ((f2cl-lib:int-add i 1) j)
                                          ((1 ldb$) (1 *)) b-%offset%)))))
                 (t
                  (setf temp
                          (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                         ((1 ldb$) (1 *)) b-%offset%))
                  (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                          (-
                           (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                          b-%offset%)
                           (*
                            (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                            temp)))
                  (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)
                          temp)))
             label110))
        label120)))))
(t
 (cond
  ((<= nrhs 1)
   (tagbody
     (setf j 1)
    label130
     (setf (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *)) b-%offset%)
             (/
              (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *)) b-%offset%)
              (f2cl-lib:conjg
               (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))))
     (if (> n 1)
         (setf (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                              b-%offset%)
                 (/
                  (-
                   (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                                  b-%offset%)
                   (*
                    (f2cl-lib:conjg
                     (f2cl-lib:fref du-%data% (1) ((1 *)) du-%offset%))
                    (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                                   b-%offset%)))
                  (f2cl-lib:conjg
                   (f2cl-lib:fref d-%data% (2) ((1 *)) d-%offset%)))))
     (f2cl-lib:fdo (i 3 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
```

```
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (/
                   (-
                    (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)
                    (*
                     (f2cl-lib:conjg
                      (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub i 1))
                                     ((1 *)) du-%offset%))
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                    ((1 ldb$) (1 *)) b-%offset%))
                    (*
                     (f2cl-lib:conjg
                      (f2cl-lib:fref du2-%data% ((f2cl-lib:int-sub i 2))
                                     ((1 *)) du2-%offset%))
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 2) j)
                                    ((1 ldb$) (1 *)) b-%offset%)))
                   (f2cl-lib:conjg
                    (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))))
           label140))
        (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                       (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (cond
             ((= (f2cl-lib:fref ipiv (i) ((1 *))) i)
              (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)
                    (-
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)
                     (*
                      (f2cl-lib:conjg
                       (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
                      (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                     ((1 ldb$) (1 *)) b-%offset%)))))
             (t
              (setf temp
                    (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                   ((1 ldb$) (1 *)) b-%offset%))
              (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                   ((1 ldb$) (1 *)) b-%offset%)
                    (-
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)
                     (*
                      (f2cl-lib:conjg
                       (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
```

```
                    temp)))
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  temp)))
      label150))
   (cond ((< j nrhs) (setf j (f2cl-lib:int-add j 1)) (go label130)))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j nrhs) nil)
   (tagbody
     (setf (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *)) b-%offset%)
             (/
              (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                             b-%offset%)
              (f2cl-lib:conjg
               (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))))
     (if (> n 1)
         (setf (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                              b-%offset%)
                 (/
                  (-
                   (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                                  b-%offset%)
                   (*
                    (f2cl-lib:conjg
                     (f2cl-lib:fref du-%data% (1) ((1 *))
                                    du-%offset%))
                    (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                                   b-%offset%)))
                  (f2cl-lib:conjg
                   (f2cl-lib:fref d-%data% (2) ((1 *)) d-%offset%)))))
     (f2cl-lib:fdo (i 3 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)
                 (/
                  (-
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)
                   (*
                    (f2cl-lib:conjg
                     (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub i 1))
                                    ((1 *)) du-%offset%))
                    (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                   ((1 ldb$) (1 *)) b-%offset%))
                   (*
                    (f2cl-lib:conjg
                     (f2cl-lib:fref du2-%data%
                                    ((f2cl-lib:int-sub i 2)) ((1 *))
```

```
                                          du2-%offset%))
                              (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 2) j)
                                          ((1 ldb$) (1 *)) b-%offset%)))
                            (f2cl-lib:conjg
                             (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))))
                 label160))
              (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                             (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                            ((> i 1) nil)
                (tagbody
                  (cond
                   ((= (f2cl-lib:fref ipiv (i) ((1 *))) i)
                    (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                         b-%offset%)
                            (-
                             (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                          b-%offset%)
                             (*
                              (f2cl-lib:conjg
                               (f2cl-lib:fref dl-%data% (i) ((1 *))
                                              dl-%offset%))
                              (f2cl-lib:fref b-%data%
                                              ((f2cl-lib:int-add i 1) j)
                                              ((1 ldb$) (1 *)) b-%offset%)))))
                   (t
                    (setf temp
                            (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                           ((1 ldb$) (1 *)) b-%offset%))
                    (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                         ((1 ldb$) (1 *)) b-%offset%)
                            (-
                             (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                          b-%offset%)
                             (*
                              (f2cl-lib:conjg
                               (f2cl-lib:fref dl-%data% (i) ((1 *))
                                              dl-%offset%))
                              temp)))
                    (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                         b-%offset%)
                            temp)))
                 label170))
            label180))))))
     end_label
      (return (values nil nil nil nil nil nil nil nil nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgtts2
```

```
      fortran-to-lisp::*f2cl-function-info*)
(fortran-to-lisp::make-f2cl-finfo :arg-types
                                  '((fortran-to-lisp::integer4)
                                    (fortran-to-lisp::integer4)
                                    (fortran-to-lisp::integer4)
                                    (array fortran-to-lisp::complex8
                                     (*))
                                    (array fortran-to-lisp::complex8
                                     (*))
                                    (array fortran-to-lisp::complex8
                                     (*))
                                    (array fortran-to-lisp::complex8
                                     (*))
                                    (array fortran-to-lisp::integer4
                                     (*))
                                    (array fortran-to-lisp::complex8
                                     (*))
                                    (fortran-to-lisp::integer4))
                                  :return-values
                                  '(nil nil nil nil nil nil nil nil
                                    nil nil)
                                  :calls 'nil)))
```

———-

Chapter 81

LAPACK - General Tridiagonal Matrix, Comp. Routines, Complex16

81.1 zgtcon

81.1.1 Axiom unit tests

— zgtcon.input —

```
)set break resume
)sys rm -f zgtcon.output
)spool zgtcon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

81.1.2 Axiom help page

— zgtcon.help —

```
==========================================================================
```

```
zgtcon examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgtcon (NORM, N, DL, D, DU, DU2, IPIV, ANORM, RCOND, WORK,
           INFO)
           ZGTCON

Function/Subroutine Documentation
   subroutine zgtcon (character NORM, integer N, complex*16, dimension( * )
       DL, complex*16, dimension( * ) D, complex*16, dimension( * ) DU,
       complex*16, dimension( * ) DU2, integer, dimension( * ) IPIV, double
       precision ANORM, double precision RCOND, complex*16, dimension( * )
       WORK, integer INFO)
       ZGTCON

Purpose:
   ZGTCON estimates the reciprocal of the condition number of a complex
   tridiagonal matrix A using the LU factorization as computed by ZGTTRF.

   An estimate is obtained for norm(inv(A)), and the reciprocal of the
   condition number is computed as RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies whether the 1-norm condition number or the
   infinity-norm condition number is required:
   = '1' or 'O':  1-norm;
   = 'I':         Infinity-norm.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 DL
   DL is COMPLEX*16 array, dimension (N-1)
   The (n-1) multipliers that define the matrix L from the
   LU factorization of A as computed by ZGTTRF.

 D
   D is COMPLEX*16 array, dimension (N)
   The n diagonal elements of the upper triangular matrix U from
   the LU factorization of A.
```

```
DU
  DU is COMPLEX*16 array, dimension (N-1)
  The (n-1) elements of the first superdiagonal of U.

DU2
  DU2 is COMPLEX*16 array, dimension (N-2)
  The (n-2) elements of the second superdiagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= n, row i of the matrix was
  interchanged with row IPIV(i).  IPIV(i) will always be either
  i or i+1; IPIV(i) = i indicates a row interchange was not required.

ANORM
  ANORM is DOUBLE PRECISION
  If NORM = '1' or 'O', the 1-norm of the original matrix A.
  If NORM = 'I', the infinity-norm of the original matrix A.

RCOND
  RCOND is DOUBLE PRECISION
  The reciprocal of the condition number of the matrix A,
  computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is an
  estimate of the 1-norm of inv(A) computed in this routine.

WORK
  WORK is COMPLEX*16 array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

81.1.3 fortran code

— zgtcon.f —

```
*  ====================================================================
      SUBROUTINE ZGTCON( NORM, N, DL, D, DU, DU2, IPIV, ANORM, RCOND,
     $                   WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
```

```
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            INFO, N
      DOUBLE PRECISION   ANORM, RCOND
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         D( * ), DL( * ), DU( * ), DU2( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ONENRM
      INTEGER            I, KASE, KASE1
      DOUBLE PRECISION   AINVNM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGTTRS, ZLACN2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DCMPLX
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      ONENRM = NORM.EQ.'1' .OR. LSAME( NORM, 'O' )
      IF( .NOT.ONENRM .AND. .NOT.LSAME( NORM, 'I' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
```

```
         CALL XERBLA( 'ZGTCON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
*     Check that D(1:N) is non-zero.
*
      DO 10 I = 1, N
         IF( D( I ).EQ.DCMPLX( ZERO ) )
     $      RETURN
   10 CONTINUE
*
      AINVNM = ZERO
      IF( ONENRM ) THEN
         KASE1 = 1
      ELSE
         KASE1 = 2
      END IF
      KASE = 0
   20 CONTINUE
      CALL ZLACN2( N, WORK( N+1 ), WORK, AINVNM, KASE, ISAVE )
      IF( KASE.NE.0 ) THEN
         IF( KASE.EQ.KASE1 ) THEN
*
*           Multiply by inv(U)*inv(L).
*
            CALL ZGTTRS( 'No transpose', N, 1, DL, D, DU, DU2, IPIV,
     $                   WORK, N, INFO )
         ELSE
*
*           Multiply by inv(L**H)*inv(U**H).
*
            CALL ZGTTRS( 'Conjugate transpose', N, 1, DL, D, DU, DU2,
     $                   IPIV, WORK, N, INFO )
         END IF
         GO TO 20
      END IF
*
*     Compute the estimate of the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
```

```
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
      RETURN
*
*     End of ZGTCON
*
      END
```

———-

81.1.4 lisp code

— **zgtcon.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun zgtcon (norm n dl d du du2 ipiv anorm rcond work info)
    (declare (type (double-float) rcond anorm)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) work du2 du d dl)
             (type (f2cl-lib:integer4) info n)
             (type (string *) norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (dl f2cl-lib:complex16 dl-%data% dl-%offset%)
         (d f2cl-lib:complex16 d-%data% d-%offset%)
         (du f2cl-lib:complex16 du-%data% du-%offset%)
         (du2 f2cl-lib:complex16 du2-%data% du2-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
             (ainvnm 0.0d0) (i 0) (kase 0) (kase1 0) (onenrm nil))
        (declare (type (array f2cl-lib:integer4 (3)) isave)
                 (type (double-float) ainvnm)
                 (type (f2cl-lib:integer4) i kase kase1)
                 (type f2cl-lib:logical onenrm))
        (setf info 0)
        (setf onenrm
                (or (f2cl-lib:fstring-= norm "1")
                    (multiple-value-bind (ret-val var-0 var-1)
                        (lsame norm "O")
                      (declare (ignore var-1))
                      (when var-0 (setf norm var-0))
                      ret-val)))
        (cond
         ((and (not onenrm)
```

```
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame norm "I")
           (declare (ignore var-1))
           (when var-0 (setf norm var-0))
           ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2)) ((< anorm zero) (setf info -8)))
  (cond
   ((/= info 0) (xerbla "ZGTCON" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf rcond zero)
  (cond ((= n 0) (setf rcond one) (go end_label))
        ((= anorm zero) (go end_label)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (if (= (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
             (f2cl-lib:dcmplx zero))
          (go end_label))
    label10))
  (setf ainvnm zero)
  (cond (onenrm (setf kase1 1)) (t (setf kase1 2)))
  (setf kase 0)
 label20
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (zlacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 ((+ n 1))
                             ((1 *)) work-%offset%)
       work ainvnm kase isave)
    (declare (ignore var-1 var-2 var-5))
    (when var-0 (setf n var-0))
    (when var-3 (setf ainvnm var-3))
    (when var-4 (setf kase var-4)))
  (cond
   ((/= kase 0)
    (cond
     ((= kase kase1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (zgttrs "No transpose" n 1 dl d du du2 ipiv work n info)
        (declare
         (ignore var-0 var-2 var-3 var-4 var-5 var-6 var-7 var-8))
        (when var-1 (setf n var-1))
        (when var-9 (setf n var-9))
        (when var-10 (setf info var-10))))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
                var-10)
               (zgttrs "Conjugate transpose" n 1 dl d du du2 ipiv work n info)
             (declare
              (ignore var-0 var-2 var-3 var-4 var-5 var-6 var-7 var-8))
             (when var-1 (setf n var-1))
             (when var-9 (setf n var-9))
             (when var-10 (setf info var-10)))))
          (go label20)))
        (if (/= ainvnm zero)
            (setf rcond (/ (/ one ainvnm) anorm)))
        (go end_label)
       end_label
        (return (values norm n nil nil nil nil nil nil rcond nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgtcon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (double-float) (double-float)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::norm
                                              fortran-to-lisp::n nil nil nil
                                              nil nil nil
                                              fortran-to-lisp::rcond nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

81.2 zgtrfs

81.2.1 Axiom unit tests

— zgtrfs.input —

```
)set break resume
)sys rm -f zgtrfs.output
)spool zgtrfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

81.2.2 Axiom help page

— zgtrfs.help —

```
======================================================================
zgtrfs examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgtrfs (TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF, DU2, IPIV,
           B, LDB, X, LDX, FERR, BERR, WORK, RWORK, INFO)
           ZGTRFS

Function/Subroutine Documentation
   subroutine zgtrfs (character TRANS, integer N, integer NRHS, complex*16,
       dimension( * ) DL, complex*16, dimension( * ) D, complex*16, dimension(
       * ) DU, complex*16, dimension( * ) DLF, complex*16, dimension( * ) DF,
       complex*16, dimension( * ) DUF, complex*16, dimension( * ) DU2,
       integer, dimension( * ) IPIV, complex*16, dimension( ldb, * ) B,
       integer LDB, complex*16, dimension( ldx, * ) X, integer LDX, double
       precision, dimension( * ) FERR, double precision, dimension( * ) BERR,
       complex*16, dimension( * ) WORK, double precision, dimension( * )
       RWORK, integer INFO)
```

```
       ZGTRFS

Purpose:
   ZGTRFS improves the computed solution to a system of linear
   equations when the coefficient matrix is tridiagonal, and provides
   error bounds and backward error estimates for the solution.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations:
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 DL
   DL is COMPLEX*16 array, dimension (N-1)
   The (n-1) subdiagonal elements of A.

 D
   D is COMPLEX*16 array, dimension (N)
   The diagonal elements of A.

 DU
   DU is COMPLEX*16 array, dimension (N-1)
   The (n-1) superdiagonal elements of A.

 DLF
   DLF is COMPLEX*16 array, dimension (N-1)
   The (n-1) multipliers that define the matrix L from the
   LU factorization of A as computed by ZGTTRF.

 DF
   DF is COMPLEX*16 array, dimension (N)
   The n diagonal elements of the upper triangular matrix U from
   the LU factorization of A.

 DUF
   DUF is COMPLEX*16 array, dimension (N-1)
   The (n-1) elements of the first superdiagonal of U.
```

```
DU2
  DU2 is COMPLEX*16 array, dimension (N-2)
  The (n-2) elements of the second superdiagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= n, row i of the matrix was
  interchanged with row IPIV(i).  IPIV(i) will always be either
  i or i+1; IPIV(i) = i indicates a row interchange was not required.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by ZGTTRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

FERR
  FERR is DOUBLE PRECISION array, dimension (NRHS)
  The estimated forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).  The estimate is as reliable as
  the estimate for RCOND, and is almost always a slight
  overestimate of the true error.

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is COMPLEX*16 array, dimension (2*N)

RWORK
  RWORK is DOUBLE PRECISION array, dimension (N)
```

```
 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

   ITMAX is the maximum number of steps of iterative refinement.
```

81.2.3 fortran code

— zgtrfs.f —

```
*  =====================================================================
      SUBROUTINE ZGTRFS( TRANS, N, NRHS, DL, D, DU, DLF, DF, DUF, DU2,
     $                   IPIV, B, LDB, X, LDX, FERR, BERR, WORK, RWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   BERR( * ), FERR( * ), RWORK( * )
      COMPLEX*16         B( LDB, * ), D( * ), DF( * ), DL( * ),
     $                   DLF( * ), DU( * ), DU2( * ), DUF( * ),
     $                   WORK( * ), X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D+0 )
```

```
      DOUBLE PRECISION   THREE
      PARAMETER          ( THREE = 3.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      CHARACTER          TRANSN, TRANST
      INTEGER            COUNT, I, J, KASE, NZ
      DOUBLE PRECISION   EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN
      COMPLEX*16         ZDUM
*     ..
*     .. Local Arrays ..
      INTEGER            ISAVE( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZAXPY, ZCOPY, ZGTTRS, ZLACN2, ZLAGTM
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DCMPLX, DIMAG, MAX
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -13
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -15
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGTRFS', -INFO )
```

```
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
      IF( NOTRAN ) THEN
         TRANSN = 'N'
         TRANST = 'C'
      ELSE
         TRANSN = 'C'
         TRANST = 'N'
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = 4
      EPS = DLAMCH( 'Epsilon' )
      SAFMIN = DLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 110 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - op(A) * X,
*        where op(A) = A, A**T, or A**H, depending on TRANS.
*
         CALL ZCOPY( N, B( 1, J ), 1, WORK, 1 )
         CALL ZLAGTM( TRANS, N, 1, -ONE, DL, D, DU, X( 1, J ), LDX, ONE,
     $                WORK, N )
*
*        Compute abs(op(A))*abs(x) + abs(b) for use in the backward
*        error bound.
*
         IF( NOTRAN ) THEN
```

```
            IF( N.EQ.1 ) THEN
               RWORK( 1 ) = CABS1( B( 1, J ) ) +
     $                      CABS1( D( 1 ) )*CABS1( X( 1, J ) )
            ELSE
               RWORK( 1 ) = CABS1( B( 1, J ) ) +
     $                      CABS1( D( 1 ) )*CABS1( X( 1, J ) ) +
     $                      CABS1( DU( 1 ) )*CABS1( X( 2, J ) )
               DO 30 I = 2, N - 1
                  RWORK( I ) = CABS1( B( I, J ) ) +
     $                         CABS1( DL( I-1 ) )*CABS1( X( I-1, J ) ) +
     $                         CABS1( D( I ) )*CABS1( X( I, J ) ) +
     $                         CABS1( DU( I ) )*CABS1( X( I+1, J ) )
   30          CONTINUE
               RWORK( N ) = CABS1( B( N, J ) ) +
     $                      CABS1( DL( N-1 ) )*CABS1( X( N-1, J ) ) +
     $                      CABS1( D( N ) )*CABS1( X( N, J ) )
            END IF
         ELSE
            IF( N.EQ.1 ) THEN
               RWORK( 1 ) = CABS1( B( 1, J ) ) +
     $                      CABS1( D( 1 ) )*CABS1( X( 1, J ) )
            ELSE
               RWORK( 1 ) = CABS1( B( 1, J ) ) +
     $                      CABS1( D( 1 ) )*CABS1( X( 1, J ) ) +
     $                      CABS1( DL( 1 ) )*CABS1( X( 2, J ) )
               DO 40 I = 2, N - 1
                  RWORK( I ) = CABS1( B( I, J ) ) +
     $                         CABS1( DU( I-1 ) )*CABS1( X( I-1, J ) ) +
     $                         CABS1( D( I ) )*CABS1( X( I, J ) ) +
     $                         CABS1( DL( I ) )*CABS1( X( I+1, J ) )
   40          CONTINUE
               RWORK( N ) = CABS1( B( N, J ) ) +
     $                      CABS1( DU( N-1 ) )*CABS1( X( N-1, J ) ) +
     $                      CABS1( D( N ) )*CABS1( X( N, J ) )
            END IF
         END IF
*
*        Compute componentwise relative backward error from formula
*
*        max(i) ( abs(R(i)) / ( abs(op(A))*abs(X) + abs(B) )(i) )
*
*        where abs(Z) is the componentwise absolute value of the matrix
*        or vector Z.  If the i-th component of the denominator is less
*        than SAFE2, then SAFE1 is added to the i-th components of the
*        numerator and denominator before dividing.
*
         S = ZERO
         DO 50 I = 1, N
            IF( RWORK( I ).GT.SAFE2 ) THEN
               S = MAX( S, CABS1( WORK( I ) ) / RWORK( I ) )
```

```
            ELSE
               S = MAX( S, ( CABS1( WORK( I ) )+SAFE1 ) /
     $             ( RWORK( I )+SAFE1 ) )
            END IF
   50    CONTINUE
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
*           1) The residual BERR(J) is larger than machine epsilon, and
*           2) BERR(J) decreased by at least a factor of 2 during the
*              last iteration, and
*           3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
*
*           Update solution and try again.
*
            CALL ZGTTRS( TRANS, N, 1, DLF, DF, DUF, DU2, IPIV, WORK, N,
     $                   INFO )
            CALL ZAXPY( N, DCMPLX( ONE ), WORK, 1, X( 1, J ), 1 )
            LSTRES = BERR( J )
            COUNT = COUNT + 1
            GO TO 20
         END IF
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(op(A)))*
*           ( abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
*          inv(op(A)) is the inverse of op(A)
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(op(A))*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(op(A))*abs(X) + abs(B) is less than SAFE2.
*
*        Use ZLACN2 to estimate the infinity-norm of the matrix
*           inv(op(A)) * diag(W),
*        where W = abs(R) + NZ*EPS*( abs(op(A))*abs(X)+abs(B) )))
*
         DO 60 I = 1, N
            IF( RWORK( I ).GT.SAFE2 ) THEN
```

```
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I )
            ELSE
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I ) +
     $                      SAFE1
            END IF
   60    CONTINUE
*
         KASE = 0
   70    CONTINUE
         CALL ZLACN2( N, WORK( N+1 ), WORK, FERR( J ), KASE, ISAVE )
         IF( KASE.NE.0 ) THEN
            IF( KASE.EQ.1 ) THEN
*
*              Multiply by diag(W)*inv(op(A)**H).
*
               CALL ZGTTRS( TRANST, N, 1, DLF, DF, DUF, DU2, IPIV, WORK,
     $                      N, INFO )
               DO 80 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
   80          CONTINUE
            ELSE
*
*              Multiply by inv(op(A))*diag(W).
*
               DO 90 I = 1, N
                  WORK( I ) = RWORK( I )*WORK( I )
   90          CONTINUE
               CALL ZGTTRS( TRANSN, N, 1, DLF, DF, DUF, DU2, IPIV, WORK,
     $                      N, INFO )
            END IF
            GO TO 70
         END IF
*
*        Normalize error.
*
         LSTRES = ZERO
         DO 100 I = 1, N
            LSTRES = MAX( LSTRES, CABS1( X( I, J ) ) )
  100    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  110 CONTINUE
*
      RETURN
*
*     End of ZGTRFS
*
      END
```

———-

81.2.4 lisp code

— **zgtrfs.lisp** —

```
(let* ((itmax 5) (zero 0.0d0) (one 1.0d0) (two 2.0d0) (three 3.0d0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 3.0d0 3.0d0) three)
           (ignorable itmax zero one two three))
  (defun zgtrfs
         (trans n nrhs dl d du dlf df duf du2 ipiv b ldb$ x ldx ferr berr work
          rwork info)
    (declare (type (array double-float (*)) rwork berr ferr)
             (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) work x b du2 duf df dlf du d
              dl)
             (type (f2cl-lib:integer4) info ldx ldb$ nrhs n)
             (type (string *) trans))
    (f2cl-lib:with-multi-array-data
        ((trans character trans-%data% trans-%offset%)
         (dl f2cl-lib:complex16 dl-%data% dl-%offset%)
         (d f2cl-lib:complex16 d-%data% d-%offset%)
         (du f2cl-lib:complex16 du-%data% du-%offset%)
         (dlf f2cl-lib:complex16 dlf-%data% dlf-%offset%)
         (df f2cl-lib:complex16 df-%data% df-%offset%)
         (duf f2cl-lib:complex16 duf-%data% duf-%offset%)
         (du2 f2cl-lib:complex16 du2-%data% du2-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (x f2cl-lib:complex16 x-%data% x-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%)
         (ferr double-float ferr-%data% ferr-%offset%)
         (berr double-float berr-%data% berr-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
          cabs1))
        (prog ((isave (make-array 3 :element-type 'f2cl-lib:integer4))
               (zdum #C(0.0d0 0.0d0)) (eps 0.0d0) (lstres 0.0d0) (s 0.0d0)
               (safe1 0.0d0) (safe2 0.0d0) (safmin 0.0d0) (count$ 0) (i 0)
               (j 0) (kase 0) (nz 0)
               (transn
```

```
      (make-array '(1) :element-type 'character :initial-element
                  #\ ))
     (transt
      (make-array '(1) :element-type 'character :initial-element
                  #\ ))
     (notran nil))
(declare (type (array f2cl-lib:integer4 (3)) isave)
         (type (f2cl-lib:complex16) zdum)
         (type (double-float) eps lstres s safe1 safe2 safmin)
         (type (f2cl-lib:integer4) count$ i j kase nz)
         (type (string 1) transn transt)
         (type f2cl-lib:logical notran))
(setf info 0)
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -13))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -15)))
(cond
 ((/= info 0) (xerbla "ZGTRFS" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
              zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%)
```

```
              zero)
     label10))
  (go end_label)))
(cond
 (notran (f2cl-lib:f2cl-set-string transn "N" (string 1))
  (f2cl-lib:f2cl-set-string transt "C" (string 1)))
 (t (f2cl-lib:f2cl-set-string transn "C" (string 1))
  (f2cl-lib:f2cl-set-string transt "N" (string 1))))
(setf nz 4)
(setf eps (dlamch "Epsilon"))
(setf safmin (dlamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zcopy n
         (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 j)
                               ((1 ldb$) (1 *)) b-%offset%)
         1 work 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11)
        (zlagtm trans n 1 (- one) dl d du
         (f2cl-lib:array-slice x-%data% f2cl-lib:complex16 (1 j)
                               ((1 ldx) (1 *)) x-%offset%)
         ldx one work n)
      (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-10))
      (when var-0 (setf trans var-0))
      (when var-1 (setf n var-1))
      (when var-8 (setf ldx var-8))
      (when var-9 (setf one var-9))
      (when var-11 (setf n var-11)))
    (cond
     (notran
      (cond
       ((= n 1)
        (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
                (+
                 (cabs1
                  (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                                 b-%offset%))
                 (*
                  (cabs1
```

```
                (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
               (cabs1
                (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                               x-%offset%))))))
(t
 (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
         (+
          (cabs1
           (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                          b-%offset%))
          (*
           (cabs1
            (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
           (cabs1
            (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                           x-%offset%)))
          (*
           (cabs1
            (f2cl-lib:fref du-%data% (1) ((1 *)) du-%offset%))
           (cabs1
            (f2cl-lib:fref x-%data% (2 j) ((1 ldx) (1 *))
                           x-%offset%)))))
 (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
               ((> i
                   (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                          rwork-%offset%)
             (+
              (cabs1
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%))
              (*
               (cabs1
                (f2cl-lib:fref dl-%data%
                               ((f2cl-lib:int-sub i 1))
                               ((1 *)) dl-%offset%))
               (cabs1
                (f2cl-lib:fref x-%data%
                               ((f2cl-lib:int-sub i 1) j)
                               ((1 ldx) (1 *)) x-%offset%)))
              (*
               (cabs1
                (f2cl-lib:fref d-%data% (i) ((1 *))
                               d-%offset%))
               (cabs1
                (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                               x-%offset%)))
              (*
```

```
                    (cabs1
                     (f2cl-lib:fref du-%data% (i) ((1 *))
                                    du-%offset%))
                    (cabs1
                     (f2cl-lib:fref x-%data%
                                    ((f2cl-lib:int-add i 1) j)
                                    ((1 ldx) (1 *)) x-%offset%)))))
       label30))
    (setf (f2cl-lib:fref rwork-%data% (n) ((1 *)) rwork-%offset%)
            (+
             (cabs1
              (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                             b-%offset%))
             (*
              (cabs1
               (f2cl-lib:fref dl-%data% ((f2cl-lib:int-sub n 1))
                              ((1 *)) dl-%offset%))
              (cabs1
               (f2cl-lib:fref x-%data% ((f2cl-lib:int-sub n 1) j)
                              ((1 ldx) (1 *)) x-%offset%)))
             (*
              (cabs1
               (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%))
              (cabs1
               (f2cl-lib:fref x-%data% (n j) ((1 ldx) (1 *))
                              x-%offset%))))))))
(t
 (cond
  ((= n 1)
   (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
           (+
            (cabs1
             (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                            b-%offset%))
            (*
             (cabs1
              (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
             (cabs1
              (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                             x-%offset%))))))
  (t
   (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
           (+
            (cabs1
             (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                            b-%offset%))
            (*
             (cabs1
              (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
             (cabs1
```

```
            (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                           x-%offset%)))
          (*
           (cabs1
            (f2cl-lib:fref dl-%data% (1) ((1 *)) dl-%offset%))
           (cabs1
            (f2cl-lib:fref x-%data% (2 j) ((1 ldx) (1 *))
                           x-%offset%)))))
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i
                  (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                         rwork-%offset%)
          (+
           (cabs1
            (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                           b-%offset%))
           (*
            (cabs1
             (f2cl-lib:fref du-%data%
                            ((f2cl-lib:int-sub i 1))
                            ((1 *)) du-%offset%))
            (cabs1
             (f2cl-lib:fref x-%data%
                            ((f2cl-lib:int-sub i 1) j)
                            ((1 ldx) (1 *)) x-%offset%)))
           (*
            (cabs1
             (f2cl-lib:fref d-%data% (i) ((1 *))
                            d-%offset%))
            (cabs1
             (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                            x-%offset%)))
           (*
            (cabs1
             (f2cl-lib:fref dl-%data% (i) ((1 *))
                            dl-%offset%))
            (cabs1
             (f2cl-lib:fref x-%data%
                            ((f2cl-lib:int-add i 1) j)
                            ((1 ldx) (1 *)) x-%offset%)))))
   label40))
(setf (f2cl-lib:fref rwork-%data% (n) ((1 *)) rwork-%offset%)
        (+
         (cabs1
          (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                         b-%offset%))
         (*
```

```
                (cabs1
                 (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub n 1))
                                ((1 *)) du-%offset%))
                (cabs1
                 (f2cl-lib:fref x-%data% ((f2cl-lib:int-sub n 1) j)
                                ((1 ldx) (1 *)) x-%offset%)))
               (*
                (cabs1
                 (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%))
                (cabs1
                 (f2cl-lib:fref x-%data% (n j) ((1 ldx) (1 *))
                                x-%offset%)))))))))
(setf s zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
      (setf s
              (max s
                   (/
                    (cabs1
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%))
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%)))))
     (t
      (setf s
              (max s
                   (/
                    (+
                     (cabs1
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%))
                     safe1)
                    (+
                     (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                    rwork-%offset%)
                     safe1))))))
   label50))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (zgttrs trans n 1 dlf df duf du2 ipiv work n info)
    (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-8))
```

```
      (when var-0 (setf trans var-0))
      (when var-1 (setf n var-1))
      (when var-9 (setf n var-9))
      (when var-10 (setf info var-10)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (zaxpy n (f2cl-lib:dcmplx one) work 1
         (f2cl-lib:array-slice x-%data% f2cl-lib:complex16 (1 j)
                               ((1 ldx) (1 *)) x-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4 var-5))
      (when var-0 (setf n var-0)))
    (setf lstres
            (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
    (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                             rwork-%offset%)
                (+
                 (cabs1
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%))
                 (* nz eps
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%)))))
       (t
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                             rwork-%offset%)
                (+
                 (cabs1
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%))
                 (* nz eps
                    (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                   rwork-%offset%))
                 safe1))))
     label60))
  (setf kase 0)
 label70
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (zlacn2 n
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             ((+ n 1)) ((1 *)) work-%offset%)
       work
       (f2cl-lib:array-slice ferr-%data% double-float (j) ((1 *))
                             ferr-%offset%)
       kase isave)
```

```
    (declare (ignore var-1 var-2 var-3 var-5))
    (when var-0 (setf n var-0))
    (when var-4 (setf kase var-4)))
  (cond
   ((/= kase 0)
    (cond
     ((= kase 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (zgttrs transt n 1 dlf df duf du2 ipiv work n info)
        (declare
         (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-8))
        (when var-0 (setf transt var-0))
        (when var-1 (setf n var-1))
        (when var-9 (setf n var-9))
        (when var-10 (setf info var-10)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                  (*
                   (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                  rwork-%offset%)
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)))
          label80)))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%)
                  (*
                   (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                  rwork-%offset%)
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)))
          label90))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10)
          (zgttrs transn n 1 dlf df duf du2 ipiv work n info)
        (declare
         (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-8))
        (when var-0 (setf transn var-0))
        (when var-1 (setf n var-1))
        (when var-9 (setf n var-9))
        (when var-10 (setf info var-10)))))
```

```
                (go label70)))
              (setf lstres zero)
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i n) nil)
                (tagbody
                  (setf lstres
                          (max lstres
                               (cabs1
                                (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                               x-%offset%))))
                 label100))
              (if (/= lstres zero)
                  (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                          (/
                           (f2cl-lib:fref ferr-%data% (j) ((1 *))
                                          ferr-%offset%)
                           lstres)))
            label110))
          (go end_label)
         end_label
          (return
           (values trans n nil nil nil nil nil nil nil nil nil nil nil nil ldx
                   nil nil nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgtrfs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
```

```
  (*))
 (fortran-to-lisp::integer4)
 (array fortran-to-lisp::complex16
  (*))
 (fortran-to-lisp::integer4)
 (array double-float (*))
 (array double-float (*))
 (array fortran-to-lisp::complex16
  (*))
 (array double-float (*))
 (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::trans
  fortran-to-lisp::n nil nil nil
  nil nil nil nil nil nil nil nil
  nil fortran-to-lisp::ldx nil nil
  nil nil fortran-to-lisp::info)
:calls '(fortran-to-lisp::xerbla))))
```

———-

81.3 zgttrf

81.3.1 Axiom unit tests

— zgttrf.input —

```
)set break resume
)sys rm -f zgttrf.output
)spool zgttrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

81.3.2 Axiom help page

— zgttrf.help —

```
====================================================================
zgttrf examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgttrf (N, DL, D, DU, DU2, IPIV, INFO)
           ZGTTRF

Function/Subroutine Documentation
   subroutine zgttrf (integer N, complex*16, dimension( * ) DL, complex*16,
       dimension( * ) D, complex*16, dimension( * ) DU, complex*16, dimension(
       * ) DU2, integer, dimension( * ) IPIV, integer INFO)
       ZGTTRF

Purpose:
   ZGTTRF computes an LU factorization of a complex tridiagonal matrix A
   using elimination with partial pivoting and row interchanges.

   The factorization has the form
      A = L * U
   where L is a product of permutation and unit lower bidiagonal
   matrices and U is upper triangular with nonzeros in only the main
   diagonal and first two superdiagonals.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.

 DL
   DL is COMPLEX*16 array, dimension (N-1)
   On entry, DL must contain the (n-1) sub-diagonal elements of A.

   On exit, DL is overwritten by the (n-1) multipliers that
   define the matrix L from the LU factorization of A.

 D
   D is COMPLEX*16 array, dimension (N)
   On entry, D must contain the diagonal elements of A.

   On exit, D is overwritten by the n diagonal elements of the
   upper triangular matrix U from the LU factorization of A.

 DU
```

```
  DU is COMPLEX*16 array, dimension (N-1)
  On entry, DU must contain the (n-1) super-diagonal elements of A.

  On exit, DU is overwritten by the (n-1) elements of the first
  super-diagonal of U.

DU2
  DU2 is COMPLEX*16 array, dimension (N-2)
  On exit, DU2 is overwritten by the (n-2) elements of the
  second super-diagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= n, row i of the matrix was
  interchanged with row IPIV(i).  IPIV(i) will always be either
  i or i+1; IPIV(i) = i indicates a row interchange was not
  required.

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -k, the k-th argument had an illegal value
  > 0:  if INFO = k, U(k,k) is exactly zero. The factorization
     has been completed, but the factor U is exactly
     singular, and division by zero will occur if it is used
     to solve a system of equations.
```

———-

81.3.3 fortran code

— **zgttrf.f** —

```
*  =====================================================================
      SUBROUTINE ZGTTRF( N, DL, D, DU, DU2, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         D( * ), DL( * ), DU( * ), DU2( * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      COMPLEX*16         FACT, TEMP, ZDUM
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
         CALL XERBLA( 'ZGTTRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Initialize IPIV(i) = i and DU2(i) = 0
*
      DO 10 I = 1, N
         IPIV( I ) = I
   10 CONTINUE
      DO 20 I = 1, N - 2
         DU2( I ) = ZERO
   20 CONTINUE
*
      DO 30 I = 1, N - 2
         IF( CABS1( D( I ) ).GE.CABS1( DL( I ) ) ) THEN
*
```

```
*          No row interchange required, eliminate DL(I)
*
           IF( CABS1( D( I ) ).NE.ZERO ) THEN
              FACT = DL( I ) / D( I )
              DL( I ) = FACT
              D( I+1 ) = D( I+1 ) - FACT*DU( I )
           END IF
        ELSE
*
*          Interchange rows I and I+1, eliminate DL(I)
*
           FACT = D( I ) / DL( I )
           D( I ) = DL( I )
           DL( I ) = FACT
           TEMP = DU( I )
           DU( I ) = D( I+1 )
           D( I+1 ) = TEMP - FACT*D( I+1 )
           DU2( I ) = DU( I+1 )
           DU( I+1 ) = -FACT*DU( I+1 )
           IPIV( I ) = I + 1
        END IF
   30 CONTINUE
      IF( N.GT.1 ) THEN
         I = N - 1
         IF( CABS1( D( I ) ).GE.CABS1( DL( I ) ) ) THEN
            IF( CABS1( D( I ) ).NE.ZERO ) THEN
               FACT = DL( I ) / D( I )
               DL( I ) = FACT
               D( I+1 ) = D( I+1 ) - FACT*DU( I )
            END IF
         ELSE
            FACT = D( I ) / DL( I )
            D( I ) = DL( I )
            DL( I ) = FACT
            TEMP = DU( I )
            DU( I ) = D( I+1 )
            D( I+1 ) = TEMP - FACT*D( I+1 )
            IPIV( I ) = I + 1
         END IF
      END IF
*
*     Check for a zero on the diagonal of U.
*
      DO 40 I = 1, N
         IF( CABS1( D( I ) ).EQ.ZERO ) THEN
            INFO = I
            GO TO 50
         END IF
   40 CONTINUE
   50 CONTINUE
```

```
*
      RETURN
*
*     End of ZGTTRF
*
      END
```

———-

81.3.4 lisp code

— **zgttrf.lisp** —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun zgttrf (n dl d du du2 ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array f2cl-lib:complex16 (*)) du2 du d dl)
             (type (f2cl-lib:integer4) info n))
    (f2cl-lib:with-multi-array-data
        ((dl f2cl-lib:complex16 dl-%data% dl-%offset%)
         (d f2cl-lib:complex16 d-%data% d-%offset%)
         (du f2cl-lib:complex16 du-%data% du-%offset%)
         (du2 f2cl-lib:complex16 du2-%data% du2-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
          cabs1))
        (prog ((fact #C(0.0d0 0.0d0)) (temp #C(0.0d0 0.0d0))
               (zdum #C(0.0d0 0.0d0)) (i 0))
          (declare (type (f2cl-lib:complex16) fact temp zdum)
                   (type (f2cl-lib:integer4) i))
          (setf info 0)
          (cond
           ((< n 0) (setf info -1) (xerbla "ZGTTRF" (f2cl-lib:int-sub info))
            (go end_label)))
          (if (= n 0)
              (go end_label))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%) i)
             label10))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))) nil)
```

```
  (tagbody
    (setf (f2cl-lib:fref du2-%data% (i) ((1 *)) du2-%offset%)
            (coerce zero 'f2cl-lib:complex16))
   label20))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))) nil)
  (tagbody
    (cond
     ((>= (cabs1 (f2cl-lib:fref d (i) ((1 *))))
          (cabs1 (f2cl-lib:fref dl (i) ((1 *)))))
      (cond
       ((/= (cabs1 (f2cl-lib:fref d (i) ((1 *)))) zero)
        (setf fact
                (/ (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                   (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
        (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) fact)
        (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                              ((1 *)) d-%offset%)
                (-
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) d-%offset%)
                 (* fact
                    (f2cl-lib:fref du-%data% (i) ((1 *))
                                   du-%offset%)))))))
     (t
      (setf fact
              (/ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                 (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
      (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
              (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
      (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) fact)
      (setf temp (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%))
      (setf (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
              (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                              ((1 *)) d-%offset%))
      (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                           d-%offset%)
              (- temp
                 (* fact
                    (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                   ((1 *)) d-%offset%))))
      (setf (f2cl-lib:fref du2-%data% (i) ((1 *)) du2-%offset%)
              (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1))
                              ((1 *)) du-%offset%))
      (setf (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                           du-%offset%)
              (* (- fact)
                 (f2cl-lib:fref du-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) du-%offset%)))
      (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
```

```
                (f2cl-lib:int-add i 1))))
    label30))
  (cond
   ((> n 1) (setf i (f2cl-lib:int-sub n 1))
    (cond
     ((>= (cabs1 (f2cl-lib:fref d (i) ((1 *))))
          (cabs1 (f2cl-lib:fref dl (i) ((1 *)))))
      (cond
       ((/= (cabs1 (f2cl-lib:fref d (i) ((1 *)))) zero)
        (setf fact
                (/ (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                   (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
        (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) fact)
        (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                              d-%offset%)
                (-
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) d-%offset%)
                 (* fact
                    (f2cl-lib:fref du-%data% (i) ((1 *))
                                   du-%offset%)))))))
     (t
      (setf fact
              (/ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                 (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)))
      (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
              (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
      (setf (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%) fact)
      (setf temp (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%))
      (setf (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
              (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                              d-%offset%))
      (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                            d-%offset%)
              (- temp
                 (* fact
                    (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                   ((1 *)) d-%offset%))))
      (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
              (f2cl-lib:int-add i 1))))))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((= (cabs1 (f2cl-lib:fref d (i) ((1 *)))) zero) (setf info i)
        (go label50)))
    label40))
 label50
  (go end_label)
 end_label
```

```
          (return (values nil nil nil nil nil nil info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgttrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

81.4 zgttrs

81.4.1 Axiom unit tests

— zgttrs.input —

```
)set break resume
)sys rm -f zgttrs.output
)spool zgttrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

81.4.2 Axiom help page

— zgttrs.help —

```
=====================================================================
zgttrs examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgttrs (TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB, INFO)
           ZGTTRS

Function/Subroutine Documentation
   subroutine zgttrs (character TRANS, integer N, integer NRHS, complex*16,
       dimension( * ) DL, complex*16, dimension( * ) D, complex*16, dimension(
       * ) DU, complex*16, dimension( * ) DU2, integer, dimension( * ) IPIV,
       complex*16, dimension( ldb, * ) B, integer LDB, integer INFO)
       ZGTTRS

Purpose:
   ZGTTRS solves one of the systems of equations
      A * X = B,  A**T * X = B,  or  A**H * X = B,
   with a tridiagonal matrix A using the LU factorization computed by ZGTTRF.

Parameters:

 TRANS
   TRANS is CHARACTER*1
   Specifies the form of the system of equations.
   = 'N':  A * X = B     (No transpose)
   = 'T':  A**T * X = B  (Transpose)
   = 'C':  A**H * X = B  (Conjugate transpose)

 N
   N is INTEGER
   The order of the matrix A.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 DL
   DL is COMPLEX*16 array, dimension (N-1)
```

```
  The (n-1) multipliers that define the matrix L from the
  LU factorization of A.

D
  D is COMPLEX*16 array, dimension (N)
  The n diagonal elements of the upper triangular matrix U from
  the LU factorization of A.

DU
  DU is COMPLEX*16 array, dimension (N-1)
  The (n-1) elements of the first super-diagonal of U.

DU2
  DU2 is COMPLEX*16 array, dimension (N-2)
  The (n-2) elements of the second super-diagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= n, row i of the matrix was
  interchanged with row IPIV(i).  IPIV(i) will always be either
  i or i+1; IPIV(i) = i indicates a row interchange was not required.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  On entry, the matrix of right hand side vectors B.
  On exit, B is overwritten by the solution vectors X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -k, the k-th argument had an illegal value
```

———-

81.4.3 fortran code

— **zgttrs.f** —

```
*  =====================================================================
      SUBROUTINE ZGTTRS( TRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
```

```
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         B( LDB, * ), D( * ), DL( * ), DU( * ), DU2( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      INTEGER            ITRANS, J, JB, NB
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZGTTS2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      NOTRAN = ( TRANS.EQ.'N' .OR. TRANS.EQ.'n' )
      IF( .NOT.NOTRAN .AND. .NOT.( TRANS.EQ.'T' .OR. TRANS.EQ.
     $    't' ) .AND. .NOT.( TRANS.EQ.'C' .OR. TRANS.EQ.'c' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDB.LT.MAX( N, 1 ) ) THEN
         INFO = -10
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGTTRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
```

```
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
*     Decode TRANS
*
      IF( NOTRAN ) THEN
         ITRANS = 0
      ELSE IF( TRANS.EQ.'T' .OR. TRANS.EQ.'t' ) THEN
         ITRANS = 1
      ELSE
         ITRANS = 2
      END IF
*
*     Determine the number of right-hand sides to solve at a time.
*
      IF( NRHS.EQ.1 ) THEN
         NB = 1
      ELSE
         NB = MAX( 1, ILAENV( 1, 'ZGTTRS', TRANS, N, NRHS, -1, -1 ) )
      END IF
*
      IF( NB.GE.NRHS ) THEN
         CALL ZGTTS2( ITRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB )
      ELSE
         DO 10 J = 1, NRHS, NB
            JB = MIN( NRHS-J+1, NB )
            CALL ZGTTS2( ITRANS, N, JB, DL, D, DU, DU2, IPIV, B( 1, J ),
     $                   LDB )
   10    CONTINUE
      END IF
*
*     End of ZGTTRS
*
      END
```

———-

81.4.4 lisp code

— zgttrs.lisp —

```
(defun zgttrs (trans n nrhs dl d du du2 ipiv b ldb$ info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex16 (*)) b du2 du d dl)
           (type (f2cl-lib:integer4) info ldb$ nrhs n)
           (type (string *) trans))
  (f2cl-lib:with-multi-array-data
      ((trans character trans-%data% trans-%offset%)
```

```
     (dl f2cl-lib:complex16 dl-%data% dl-%offset%)
     (d f2cl-lib:complex16 d-%data% d-%offset%)
     (du f2cl-lib:complex16 du-%data% du-%offset%)
     (du2 f2cl-lib:complex16 du2-%data% du2-%offset%)
     (b f2cl-lib:complex16 b-%data% b-%offset%)
     (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
   (prog ((itrans 0) (j 0) (jb 0) (nb 0) (notran nil))
     (declare (type f2cl-lib:logical notran)
              (type (f2cl-lib:integer4) nb jb j itrans))
     (setf info 0)
     (setf notran
             (or (f2cl-lib:fstring-= trans "N")
                 (f2cl-lib:fstring-= trans "n")))
     (cond
      ((and (not notran)
            (not
             (or (f2cl-lib:fstring-= trans "T")
                 (f2cl-lib:fstring-= trans "t")))
            (not
             (or (f2cl-lib:fstring-= trans "C")
                 (f2cl-lib:fstring-= trans "c"))))
       (setf info -1))
      ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
      ((< ldb$ (max (the f2cl-lib:integer4 n) (the f2cl-lib:integer4 1)))
       (setf info -10)))
     (cond
      ((/= info 0) (xerbla "ZGTTRS" (f2cl-lib:int-sub info)) (go end_label)))
     (if (or (= n 0) (= nrhs 0))
         (go end_label))
     (cond (notran (setf itrans 0))
           ((or (f2cl-lib:fstring-= trans "T") (f2cl-lib:fstring-= trans "t"))
            (setf itrans 1))
           (t (setf itrans 2)))
     (cond ((= nrhs 1) (setf nb 1))
           (t
            (setf nb
                    (max (the f2cl-lib:integer4 1)
                         (the f2cl-lib:integer4
                              (multiple-value-bind
                                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                                   var-6)
                                  (ilaenv 1 "ZGTTRS" trans n nrhs -1 -1)
                                (declare (ignore var-0 var-1 var-5 var-6))
                                (setf trans var-2)
                                (setf n var-3)
                                (setf nrhs var-4)
                                ret-val))))))
     (cond
      ((>= nb nrhs)
       (multiple-value-bind
```

```
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
            (zgtts2 itrans n nrhs dl d du du2 ipiv b ldb$)
          (declare (ignore var-3 var-4 var-5 var-6 var-7 var-8))
          (when var-0 (setf itrans var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-9 (setf ldb$ var-9))))
       (t
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                      ((> j nrhs) nil)
          (tagbody
            (setf jb
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-sub nrhs j) 1))
                     (the f2cl-lib:integer4 nb)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                (zgtts2 itrans n jb dl d du du2 ipiv
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$)
              (declare (ignore var-3 var-4 var-5 var-6 var-7 var-8))
              (when var-0 (setf itrans var-0))
              (when var-1 (setf n var-1))
              (when var-2 (setf jb var-2))
              (when var-9 (setf ldb$ var-9)))
           label10))))
     end_label
      (return (values trans n nrhs nil nil nil nil nil nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgttrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
```

```
  (array fortran-to-lisp::complex16
   (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::trans
  fortran-to-lisp::n
  fortran-to-lisp::nrhs nil nil nil
  nil nil nil fortran-to-lisp::ldb$
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::ilaenv
  fortran-to-lisp::xerbla))))
```

———-

81.5 zgtts2

81.5.1 Axiom unit tests

— **zgtts2.input** —

```
)set break resume
)sys rm -f zgtts2.output
)spool zgtts2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

81.5.2 Axiom help page

— **zgtts2.help** —

```
==========================================================================
zgtts2 examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zgtts2 (ITRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB)
           ZGTTS2 solves a system of linear equations with a tridiagonal
           matrix using the LU factorization computed by sgttrf.

Function/Subroutine Documentation
   subroutine zgtts2 (integer ITRANS, integer N, integer NRHS, complex*16,
       dimension( * ) DL, complex*16, dimension( * ) D, complex*16, dimension(
       * ) DU, complex*16, dimension( * ) DU2, integer, dimension( * ) IPIV,
       complex*16, dimension( ldb, * ) B, integer LDB)
       ZGTTS2 solves a system of linear equations with a tridiagonal matrix
       using the LU factorization computed by sgttrf.

Purpose:
   ZGTTS2 solves one of the systems of equations
      A * X = B,  A**T * X = B,  or  A**H * X = B,
   with a tridiagonal matrix A using the LU factorization computed by ZGTTRF.

Parameters:

 ITRANS
   ITRANS is INTEGER
   Specifies the form of the system of equations.
   = 0:  A * X = B     (No transpose)
   = 1:  A**T * X = B  (Transpose)
   = 2:  A**H * X = B  (Conjugate transpose)

 N
   N is INTEGER
   The order of the matrix A.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 DL
   DL is COMPLEX*16 array, dimension (N-1)
   The (n-1) multipliers that define the matrix L from the
   LU factorization of A.

 D
   D is COMPLEX*16 array, dimension (N)
   The n diagonal elements of the upper triangular matrix U from
   the LU factorization of A.
```

```
DU
  DU is COMPLEX*16 array, dimension (N-1)
  The (n-1) elements of the first super-diagonal of U.

DU2
  DU2 is COMPLEX*16 array, dimension (N-2)
  The (n-2) elements of the second super-diagonal of U.

IPIV
  IPIV is INTEGER array, dimension (N)
  The pivot indices; for 1 <= i <= n, row i of the matrix was
  interchanged with row IPIV(i).  IPIV(i) will always be either
  i or i+1; IPIV(i) = i indicates a row interchange was not required.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  On entry, the matrix of right hand side vectors B.
  On exit, B is overwritten by the solution vectors X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).
```

———-

81.5.3 fortran code

— **zgtts2.f** —

```
*  =====================================================================
      SUBROUTINE ZGTTS2( ITRANS, N, NRHS, DL, D, DU, DU2, IPIV, B, LDB )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            ITRANS, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      COMPLEX*16         B( LDB, * ), D( * ), DL( * ), DU( * ), DU2( * )
*     ..
*
*  =====================================================================
```

```
*
*     .. Local Scalars ..
      INTEGER            I, J
      COMPLEX*16         TEMP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DCONJG
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( ITRANS.EQ.0 ) THEN
*
*        Solve A*X = B using the LU factorization of A,
*        overwriting each right hand side vector with its solution.
*
         IF( NRHS.LE.1 ) THEN
            J = 1
   10       CONTINUE
*
*           Solve L*x = b.
*
            DO 20 I = 1, N - 1
               IF( IPIV( I ).EQ.I ) THEN
                  B( I+1, J ) = B( I+1, J ) - DL( I )*B( I, J )
               ELSE
                  TEMP = B( I, J )
                  B( I, J ) = B( I+1, J )
                  B( I+1, J ) = TEMP - DL( I )*B( I, J )
               END IF
   20       CONTINUE
*
*           Solve U*x = b.
*
            B( N, J ) = B( N, J ) / D( N )
            IF( N.GT.1 )
     $         B( N-1, J ) = ( B( N-1, J )-DU( N-1 )*B( N, J ) ) /
     $                       D( N-1 )
            DO 30 I = N - 2, 1, -1
               B( I, J ) = ( B( I, J )-DU( I )*B( I+1, J )-DU2( I )*
     $                     B( I+2, J ) ) / D( I )
   30       CONTINUE
            IF( J.LT.NRHS ) THEN
               J = J + 1
               GO TO 10
            END IF
```

```
         ELSE
            DO 60 J = 1, NRHS
*
*           Solve L*x = b.
*
               DO 40 I = 1, N - 1
                  IF( IPIV( I ).EQ.I ) THEN
                     B( I+1, J ) = B( I+1, J ) - DL( I )*B( I, J )
                  ELSE
                     TEMP = B( I, J )
                     B( I, J ) = B( I+1, J )
                     B( I+1, J ) = TEMP - DL( I )*B( I, J )
                  END IF
   40          CONTINUE
*
*           Solve U*x = b.
*
               B( N, J ) = B( N, J ) / D( N )
               IF( N.GT.1 )
     $            B( N-1, J ) = ( B( N-1, J )-DU( N-1 )*B( N, J ) ) /
     $                          D( N-1 )
               DO 50 I = N - 2, 1, -1
                  B( I, J ) = ( B( I, J )-DU( I )*B( I+1, J )-DU2( I )*
     $                        B( I+2, J ) ) / D( I )
   50          CONTINUE
   60       CONTINUE
         END IF
      ELSE IF( ITRANS.EQ.1 ) THEN
*
*        Solve A**T * X = B.
*
         IF( NRHS.LE.1 ) THEN
            J = 1
   70       CONTINUE
*
*           Solve U**T * x = b.
*
            B( 1, J ) = B( 1, J ) / D( 1 )
            IF( N.GT.1 )
     $         B( 2, J ) = ( B( 2, J )-DU( 1 )*B( 1, J ) ) / D( 2 )
            DO 80 I = 3, N
               B( I, J ) = ( B( I, J )-DU( I-1 )*B( I-1, J )-DU2( I-2 )*
     $                     B( I-2, J ) ) / D( I )
   80       CONTINUE
*
*           Solve L**T * x = b.
*
            DO 90 I = N - 1, 1, -1
               IF( IPIV( I ).EQ.I ) THEN
                  B( I, J ) = B( I, J ) - DL( I )*B( I+1, J )
```

```
               ELSE
                  TEMP = B( I+1, J )
                  B( I+1, J ) = B( I, J ) - DL( I )*TEMP
                  B( I, J ) = TEMP
               END IF
   90       CONTINUE
            IF( J.LT.NRHS ) THEN
               J = J + 1
               GO TO 70
            END IF
         ELSE
            DO 120 J = 1, NRHS
*
*           Solve U**T * x = b.
*
               B( 1, J ) = B( 1, J ) / D( 1 )
               IF( N.GT.1 )
     $            B( 2, J ) = ( B( 2, J )-DU( 1 )*B( 1, J ) ) / D( 2 )
               DO 100 I = 3, N
                  B( I, J ) = ( B( I, J )-DU( I-1 )*B( I-1, J )-
     $                        DU2( I-2 )*B( I-2, J ) ) / D( I )
  100          CONTINUE
*
*           Solve L**T * x = b.
*
               DO 110 I = N - 1, 1, -1
                  IF( IPIV( I ).EQ.I ) THEN
                     B( I, J ) = B( I, J ) - DL( I )*B( I+1, J )
                  ELSE
                     TEMP = B( I+1, J )
                     B( I+1, J ) = B( I, J ) - DL( I )*TEMP
                     B( I, J ) = TEMP
                  END IF
  110          CONTINUE
  120       CONTINUE
         END IF
      ELSE
*
*        Solve A**H * X = B.
*
         IF( NRHS.LE.1 ) THEN
            J = 1
  130       CONTINUE
*
*           Solve U**H * x = b.
*
            B( 1, J ) = B( 1, J ) / DCONJG( D( 1 ) )
            IF( N.GT.1 )
     $         B( 2, J ) = ( B( 2, J )-DCONJG( DU( 1 ) )*B( 1, J ) ) /
     $                     DCONJG( D( 2 ) )
```

```
            DO 140 I = 3, N
               B( I, J ) = ( B( I, J )-DCONJG( DU( I-1 ) )*B( I-1, J )-
     $                     DCONJG( DU2( I-2 ) )*B( I-2, J ) ) /
     $                     DCONJG( D( I ) )
  140       CONTINUE
*
*           Solve L**H * x = b.
*
            DO 150 I = N - 1, 1, -1
               IF( IPIV( I ).EQ.I ) THEN
                  B( I, J ) = B( I, J ) - DCONJG( DL( I ) )*B( I+1, J )
               ELSE
                  TEMP = B( I+1, J )
                  B( I+1, J ) = B( I, J ) - DCONJG( DL( I ) )*TEMP
                  B( I, J ) = TEMP
               END IF
  150       CONTINUE
            IF( J.LT.NRHS ) THEN
               J = J + 1
               GO TO 130
            END IF
         ELSE
            DO 180 J = 1, NRHS
*
*           Solve U**H * x = b.
*
               B( 1, J ) = B( 1, J ) / DCONJG( D( 1 ) )
               IF( N.GT.1 )
     $            B( 2, J ) = ( B( 2, J )-DCONJG( DU( 1 ) )*B( 1, J ) )
     $                        / DCONJG( D( 2 ) )
               DO 160 I = 3, N
                  B( I, J ) = ( B( I, J )-DCONJG( DU( I-1 ) )*
     $                       B( I-1, J )-DCONJG( DU2( I-2 ) )*
     $                       B( I-2, J ) ) / DCONJG( D( I ) )
  160          CONTINUE
*
*           Solve L**H * x = b.
*
               DO 170 I = N - 1, 1, -1
                  IF( IPIV( I ).EQ.I ) THEN
                     B( I, J ) = B( I, J ) - DCONJG( DL( I ) )*
     $                           B( I+1, J )
                  ELSE
                     TEMP = B( I+1, J )
                     B( I+1, J ) = B( I, J ) - DCONJG( DL( I ) )*TEMP
                     B( I, J ) = TEMP
                  END IF
  170          CONTINUE
  180       CONTINUE
         END IF
```

```
      END IF
*
*     End of ZGTTS2
*
      END
```

———-

81.5.4 lisp code

— **zgtts2.lisp** —

```
(defun zgtts2 (itrans n nrhs dl d du du2 ipiv b ldb$)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array f2cl-lib:complex16 (*)) b du2 du d dl)
           (type (f2cl-lib:integer4) ldb$ nrhs n itrans))
  (f2cl-lib:with-multi-array-data
      ((dl f2cl-lib:complex16 dl-%data% dl-%offset%)
       (d f2cl-lib:complex16 d-%data% d-%offset%)
       (du f2cl-lib:complex16 du-%data% du-%offset%)
       (du2 f2cl-lib:complex16 du2-%data% du2-%offset%)
       (b f2cl-lib:complex16 b-%data% b-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((temp #C(0.0d0 0.0d0)) (i 0) (j 0))
      (declare (type (f2cl-lib:integer4) j i)
               (type (f2cl-lib:complex16) temp))
      (if (or (= n 0) (= nrhs 0))
          (go end_label))
      (cond
       ((= itrans 0)
        (cond
         ((<= nrhs 1)
          (tagbody
            (setf j 1)
           label10
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
              (tagbody
                (cond
                 ((= (f2cl-lib:fref ipiv (i) ((1 *))) i)
                  (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                        (-
                         (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                        ((1 ldb$) (1 *)) b-%offset%)
                         (* (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                            (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                           b-%offset%)))))
```

```
      (t
       (setf temp
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%))
       (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                            b-%offset%)
               (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                              ((1 ldb$) (1 *)) b-%offset%))
       (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                            ((1 ldb$) (1 *)) b-%offset%)
               (- temp
                  (*
                   (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%))))))
   label20))
(setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
        (/
         (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
         (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
(if (> n 1)
    (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                          ((1 ldb$) (1 *)) b-%offset%)
            (/
             (-
              (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)
              (*
               (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub n 1))
                              ((1 *)) du-%offset%)
               (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                              b-%offset%)))
             (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                            ((1 *)) d-%offset%))))
(f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))
               (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                          b-%offset%)
            (/
             (-
              (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)
              (* (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                ((1 ldb$) (1 *)) b-%offset%))
              (*
               (f2cl-lib:fref du2-%data% (i) ((1 *)) du2-%offset%)
               (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 2) j)
```

```
                                    ((1 ldb$) (1 *)) b-%offset%)))
                  (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
      label30))
   (cond ((< j nrhs) (setf j (f2cl-lib:int-add j 1)) (go label10)))))
(t
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j nrhs) nil)
   (tagbody
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                    nil)
       (tagbody
         (cond
          ((= (f2cl-lib:fref ipiv (i) ((1 *))) i)
           (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                 (-
                  (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                  ((1 ldb$) (1 *)) b-%offset%)
                  (*
                   (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)))))
          (t
           (setf temp
                 (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%))
           (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                 (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%))
           (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                 (- temp
                    (*
                     (f2cl-lib:fref dl-%data% (i) ((1 *))
                                    dl-%offset%)
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%))))))
       label40))
     (setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
             (/
              (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                             b-%offset%)
              (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
     (if (> n 1)
         (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                               ((1 ldb$) (1 *)) b-%offset%)
               (/
                (-
```

```
                        (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub n 1) j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                        (*
                         (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub n 1))
                                        ((1 *)) du-%offset%)
                         (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                                        b-%offset%)))
                       (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                      ((1 *)) d-%offset%))))
        (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 2))
                         (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                        ((> i 1) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (/
                   (-
                    (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)
                    (*
                     (f2cl-lib:fref du-%data% (i) ((1 *)) du-%offset%)
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                    ((1 ldb$) (1 *)) b-%offset%))
                    (*
                     (f2cl-lib:fref du2-%data% (i) ((1 *))
                                    du2-%offset%)
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 2) j)
                                    ((1 ldb$) (1 *)) b-%offset%)))
                   (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
          label50))
      label60)))))
((= itrans 1)
 (cond
  ((<= nrhs 1)
   (tagbody
     (setf j 1)
    label70
     (setf (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *)) b-%offset%)
           (/
            (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *)) b-%offset%)
            (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)))
     (if (> n 1)
         (setf (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                              b-%offset%)
               (/
                (-
                 (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                                b-%offset%)
                 (* (f2cl-lib:fref du-%data% (1) ((1 *)) du-%offset%)
                    (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
```

```
                                b-%offset%)))
             (f2cl-lib:fref d-%data% (2) ((1 *)) d-%offset%))))
(f2cl-lib:fdo (i 3 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                         b-%offset%)
          (/
           (-
            (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                           b-%offset%)
            (*
             (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub i 1))
                            ((1 *)) du-%offset%)
             (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                            ((1 ldb$) (1 *)) b-%offset%))
            (*
             (f2cl-lib:fref du2-%data% ((f2cl-lib:int-sub i 2))
                            ((1 *)) du2-%offset%)
             (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 2) j)
                            ((1 ldb$) (1 *)) b-%offset%)))
           (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
   label80))
(f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (cond
     ((= (f2cl-lib:fref ipiv (i) ((1 *))) i)
      (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                           b-%offset%)
            (-
             (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                            b-%offset%)
             (* (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                (f2cl-lib:fref b-%data%
                               ((f2cl-lib:int-add i 1) j)
                               ((1 ldb$) (1 *)) b-%offset%)))))
     (t
      (setf temp
            (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                           ((1 ldb$) (1 *)) b-%offset%))
      (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                           ((1 ldb$) (1 *)) b-%offset%)
            (-
             (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                            b-%offset%)
             (* (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                temp)))
      (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
```

```
                                b-%offset%)
                    temp)))
          label90))
      (cond ((< j nrhs) (setf j (f2cl-lib:int-add j 1)) (go label70)))))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j nrhs) nil)
      (tagbody
        (setf (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *)) b-%offset%)
                (/
                 (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                                b-%offset%)
                 (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)))
        (if (> n 1)
            (setf (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                                  b-%offset%)
                  (/
                   (-
                    (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                                   b-%offset%)
                    (*
                     (f2cl-lib:fref du-%data% (1) ((1 *)) du-%offset%)
                     (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                                    b-%offset%)))
                   (f2cl-lib:fref d-%data% (2) ((1 *)) d-%offset%))))
        (f2cl-lib:fdo (i 3 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)
                  (/
                   (-
                    (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)
                    (*
                     (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub i 1))
                                    ((1 *)) du-%offset%)
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                    ((1 ldb$) (1 *)) b-%offset%))
                    (*
                     (f2cl-lib:fref du2-%data% ((f2cl-lib:int-sub i 2))
                                    ((1 *)) du2-%offset%)
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 2) j)
                                    ((1 ldb$) (1 *)) b-%offset%)))
                   (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
          label100))
        (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                       (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
```

```
                (cond
                 ((= (f2cl-lib:fref ipiv (i) ((1 *))) i)
                  (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)
                        (-
                         (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                        b-%offset%)
                         (*
                          (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                          (f2cl-lib:fref b-%data%
                                         ((f2cl-lib:int-add i 1) j)
                                         ((1 ldb$) (1 *)) b-%offset%)))))
                 (t
                  (setf temp
                        (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                       ((1 ldb$) (1 *)) b-%offset%))
                  (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                        (-
                         (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                        b-%offset%)
                         (*
                          (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%)
                          temp)))
                  (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)
                        temp)))
            label110))
       label120)))))
(t
 (cond
  ((<= nrhs 1)
   (tagbody
     (setf j 1)
    label130
     (setf (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *)) b-%offset%)
             (/
              (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *)) b-%offset%)
              (f2cl-lib:dconjg
               (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))))
     (if (> n 1)
         (setf (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                              b-%offset%)
                 (/
                  (-
                   (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                                  b-%offset%)
                   (*
                    (f2cl-lib:dconjg
                     (f2cl-lib:fref du-%data% (1) ((1 *)) du-%offset%))
```

```
                    (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                                   b-%offset%)))
                 (f2cl-lib:dconjg
                  (f2cl-lib:fref d-%data% (2) ((1 *)) d-%offset%)))))
(f2cl-lib:fdo (i 3 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                         b-%offset%)
          (/
           (-
            (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                           b-%offset%)
            (*
             (f2cl-lib:dconjg
              (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub i 1))
                             ((1 *)) du-%offset%))
             (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                            ((1 ldb$) (1 *)) b-%offset%))
            (*
             (f2cl-lib:dconjg
              (f2cl-lib:fref du2-%data% ((f2cl-lib:int-sub i 2))
                             ((1 *)) du2-%offset%))
             (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 2) j)
                            ((1 ldb$) (1 *)) b-%offset%)))
           (f2cl-lib:dconjg
            (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))))
   label140))
(f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (cond
     ((= (f2cl-lib:fref ipiv (i) ((1 *))) i)
      (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                           b-%offset%)
            (-
             (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                            b-%offset%)
             (*
              (f2cl-lib:dconjg
               (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
              (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                             ((1 ldb$) (1 *)) b-%offset%)))))
     (t
      (setf temp
            (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                           ((1 ldb$) (1 *)) b-%offset%))
      (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                           ((1 ldb$) (1 *)) b-%offset%)
```

```
                    (-
                     (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)
                     (*
                      (f2cl-lib:dconjg
                       (f2cl-lib:fref dl-%data% (i) ((1 *)) dl-%offset%))
                      temp)))
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  temp)))
        label150))
    (cond ((< j nrhs) (setf j (f2cl-lib:int-add j 1)) (go label130)))))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j nrhs) nil)
     (tagbody
       (setf (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *)) b-%offset%)
               (/
                (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                               b-%offset%)
                (f2cl-lib:dconjg
                 (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))))
       (if (> n 1)
           (setf (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                                b-%offset%)
                 (/
                  (-
                   (f2cl-lib:fref b-%data% (2 j) ((1 ldb$) (1 *))
                                  b-%offset%)
                   (*
                    (f2cl-lib:dconjg
                     (f2cl-lib:fref du-%data% (1) ((1 *))
                                    du-%offset%))
                    (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                                   b-%offset%)))
                  (f2cl-lib:dconjg
                   (f2cl-lib:fref d-%data% (2) ((1 *)) d-%offset%)))))
       (f2cl-lib:fdo (i 3 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                 (/
                  (-
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)
                   (*
                    (f2cl-lib:dconjg
                     (f2cl-lib:fref du-%data% ((f2cl-lib:int-sub i 1))
                                    ((1 *)) du-%offset%))
```

```
                          (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                         ((1 ldb$) (1 *)) b-%offset%))
                         (*
                          (f2cl-lib:dconjg
                           (f2cl-lib:fref du2-%data%
                                          ((f2cl-lib:int-sub i 2)) ((1 *))
                                          du2-%offset%))
                          (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 2) j)
                                         ((1 ldb$) (1 *)) b-%offset%)))
                        (f2cl-lib:dconjg
                         (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))))
              label160))
           (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                          (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                         ((> i 1) nil)
             (tagbody
               (cond
                ((= (f2cl-lib:fref ipiv (i) ((1 *))) i)
                 (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                      b-%offset%)
                         (-
                          (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                         b-%offset%)
                          (*
                           (f2cl-lib:dconjg
                            (f2cl-lib:fref dl-%data% (i) ((1 *))
                                           dl-%offset%))
                           (f2cl-lib:fref b-%data%
                                          ((f2cl-lib:int-add i 1) j)
                                          ((1 ldb$) (1 *)) b-%offset%)))))
                (t
                 (setf temp
                         (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                        ((1 ldb$) (1 *)) b-%offset%))
                 (setf (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                      ((1 ldb$) (1 *)) b-%offset%)
                         (-
                          (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                         b-%offset%)
                          (*
                           (f2cl-lib:dconjg
                            (f2cl-lib:fref dl-%data% (i) ((1 *))
                                           dl-%offset%))
                           temp)))
                 (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                      b-%offset%)
                         temp)))
              label170))
          label180))))))
end_label
```

```
      (return (values nil nil nil nil nil nil nil nil nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zgtts2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
                                              nil nil)
                                            :calls 'nil)))
```

Chapter 82

LAPACK - Positive Definite Tridiagonal Matrix, Linear Solve, Double

82.1 dptsv

82.1.1 Axiom unit tests

— **dptsv.input** —

```
)set break resume
)sys rm -f dptsv.output
)spool dptsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

82.1.2 Axiom help page

— **dptsv.help** —

```
==========================================================================
```

```
dptsv examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dptsv (N, NRHS, D, E, B, LDB, INFO)
            DPTSV computes the solution to system of linear equations A * X =
           B for PT matrices

Function/Subroutine Documentation
   subroutine dptsv (integer N, integer NRHS, double precision, dimension( * )
       D, double precision, dimension( * ) E, double precision, dimension(
       ldb, * ) B, integer LDB, integer INFO)
        DPTSV computes the solution to system of linear equations A * X = B
       for PT matrices

Purpose:
   DPTSV computes the solution to a real system of linear equations
   A*X = B, where A is an N-by-N symmetric positive definite tridiagonal
   matrix, and X and B are N-by-NRHS matrices.

   A is factored as A = L*D*L**T, and the factored form of A is then
   used to solve the system of equations.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 D
   D is DOUBLE PRECISION array, dimension (N)
   On entry, the n diagonal elements of the tridiagonal matrix
   A.  On exit, the n diagonal elements of the diagonal matrix
   D from the factorization A = L*D*L**T.

 E
   E is DOUBLE PRECISION array, dimension (N-1)
   On entry, the (n-1) subdiagonal elements of the tridiagonal
   matrix A.  On exit, the (n-1) subdiagonal elements of the
   unit bidiagonal factor L from the L*D*L**T factorization of
```

```
  A.  (E can also be regarded as the superdiagonal of the unit
  bidiagonal factor U from the U**T*D*U factorization of A.)

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, the leading minor of order i is not
        positive definite, and the solution has not been
        computed.  The factorization has not been completed unless i = N.
```

82.1.3 fortran code

— dptsv.f —

```
*  =====================================================================
      SUBROUTINE DPTSV( N, NRHS, D, E, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   B( LDB, * ), D( * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. External Subroutines ..
      EXTERNAL           DPTTRF, DPTTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DPTSV ', -INFO )
         RETURN
      END IF
*
*     Compute the L*D*L**T (or U**T*D*U) factorization of A.
*
      CALL DPTTRF( N, D, E, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
         CALL DPTTRS( N, NRHS, D, E, B, LDB, INFO )
      END IF
      RETURN
*
*     End of DPTSV
*
      END
```

———-

82.1.4 lisp code

— dptsv.lisp —

```
(defun dptsv (n nrhs d e b ldb$ info)
  (declare (type (array double-float (*)) b e d)
           (type (f2cl-lib:integer4) info ldb$ nrhs n))
  (f2cl-lib:with-multi-array-data
      ((d double-float d-%data% d-%offset%)
       (e double-float e-%data% e-%offset%)
       (b double-float b-%data% b-%offset%))
    (prog ()
```

```
      (declare)
      (setf info 0)
      (cond ((< n 0) (setf info -1)) ((< nrhs 0) (setf info -2))
            ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -6)))
      (cond
       ((/= info 0) (xerbla "DPTSV " (f2cl-lib:int-sub info)) (go end_label)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dpttrf n d e info)
        (declare (ignore var-1 var-2))
        (when var-0 (setf n var-0))
        (when var-3 (setf info var-3)))
      (cond
       ((= info 0)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dpttrs n nrhs d e b ldb$ info)
          (declare (ignore var-2 var-3 var-4))
          (when var-0 (setf n var-0))
          (when var-1 (setf nrhs var-1))
          (when var-5 (setf ldb$ var-5))
          (when var-6 (setf info var-6)))))
      (go end_label)
     end_label
      (return (values n nrhs nil nil nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dptsv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil nil nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

82.2 dptsvx

82.2.1 Axiom unit tests

— dptsvx.input —

```
)set break resume
)sys rm -f dptsvx.output
)spool dptsvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

82.2.2 Axiom help page

— dptsvx.help —

```
====================================================================
dptsvx examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dptsvx (FACT, N, NRHS, D, E, DF, EF, B, LDB, X, LDX, RCOND,
           FERR, BERR, WORK, INFO)
            DPTSVX computes the solution to system of linear equations A * X =
           B for PT matrices

Function/Subroutine Documentation
   subroutine dptsvx (character FACT, integer N, integer NRHS, double
       precision, dimension( * ) D, double precision, dimension( * ) E, double
       precision, dimension( * ) DF, double precision, dimension( * ) EF,
       double precision, dimension( ldb, * ) B, integer LDB, double precision,
       dimension( ldx, * ) X, integer LDX, double precision RCOND, double
       precision, dimension( * ) FERR, double precision, dimension( * ) BERR,
       double precision, dimension( * ) WORK, integer INFO)
        DPTSVX computes the solution to system of linear equations A * X = B
```

```
      for PT matrices

Purpose:
   DPTSVX uses the factorization A = L*D*L**T to compute the solution
   to a real system of linear equations A*X = B, where A is an N-by-N
   symmetric positive definite tridiagonal matrix and X and B are
   N-by-NRHS matrices.

   Error bounds on the solution and a condition estimate are also provided.

Description:

   The following steps are performed:

   1. If FACT = 'N', the matrix A is factored as A = L*D*L**T, where L
      is a unit lower bidiagonal matrix and D is diagonal.  The
      factorization can also be regarded as having the form
      A = U**T*D*U.

   2. If the leading i-by-i principal minor is not positive definite,
      then the routine returns with INFO = i. Otherwise, the factored
      form of A is used to estimate the condition number of the matrix
      A.  If the reciprocal of the condition number is less than machine
      precision, INFO = N+1 is returned as a warning, but the routine
      still goes on to solve for X and compute error bounds as
      described below.

   3. The system of equations is solved for X using the factored form of A.

   4. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of A has been
   supplied on entry.
   = 'F':  On entry, DF and EF contain the factored form of A.
      D, E, DF, and EF will not be modified.
   = 'N':  The matrix A will be copied to DF and EF and factored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.
```

```
D
  D is DOUBLE PRECISION array, dimension (N)
  The n diagonal elements of the tridiagonal matrix A.

E
  E is DOUBLE PRECISION array, dimension (N-1)
  The (n-1) subdiagonal elements of the tridiagonal matrix A.

DF
  DF is DOUBLE PRECISION array, dimension (N)
  If FACT = 'F', then DF is an input argument and on entry
  contains the n diagonal elements of the diagonal matrix D
  from the L*D*L**T factorization of A.
  If FACT = 'N', then DF is an output argument and on exit
  contains the n diagonal elements of the diagonal matrix D
  from the L*D*L**T factorization of A.

EF
  EF is DOUBLE PRECISION array, dimension (N-1)
  If FACT = 'F', then EF is an input argument and on entry
  contains the (n-1) subdiagonal elements of the unit
  bidiagonal factor L from the L*D*L**T factorization of A.
  If FACT = 'N', then EF is an output argument and on exit
  contains the (n-1) subdiagonal elements of the unit
  bidiagonal factor L from the L*D*L**T factorization of A.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  The N-by-NRHS right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is DOUBLE PRECISION array, dimension (LDX,NRHS)
  If INFO = 0 of INFO = N+1, the N-by-NRHS solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
  The reciprocal condition number of the matrix A.  If RCOND
  is less than the machine precision (in particular, if
  RCOND = 0), the matrix is singular to working precision.
  This condition is indicated by a return code of INFO > 0.
```

```
 FERR
   FERR is DOUBLE PRECISION array, dimension (NRHS)
   The forward error bound for each solution vector
   X(j) (the j-th column of the solution matrix X).
   If XTRUE is the true solution corresponding to X(j), FERR(j)
   is an estimated upper bound for the magnitude of the largest
   element in (X(j) - XTRUE) divided by the magnitude of the
   largest element in X(j).

 BERR
   BERR is DOUBLE PRECISION array, dimension (NRHS)
   The componentwise relative backward error of each solution
   vector X(j) (i.e., the smallest relative change in any
   element of A or B that makes X(j) an exact solution).

 WORK
   WORK is DOUBLE PRECISION array, dimension (2*N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
   > 0:  if INFO = i, and i is
   <= N:  the leading minor of order i of A is
      not positive definite, so the factorization
      could not be completed, and the solution has not
      been computed. RCOND = 0 is returned.
   = N+1: U is nonsingular, but RCOND is less than machine
      precision, meaning that the matrix is singular
      to working precision.  Nevertheless, the
      solution and error bounds are computed because
      there are a number of situations where the
      computed solution can be more accurate than the
      value of RCOND would suggest.
```

———-

82.2.3 fortran code

— **dptsvx.f** —

```
*  ====================================================================
      SUBROUTINE DPTSVX( FACT, N, NRHS, D, E, DF, EF, B, LDB, X, LDX,
     $                   RCOND, FERR, BERR, WORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
```

```
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          FACT
      INTEGER            INFO, LDB, LDX, N, NRHS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   B( LDB, * ), BERR( * ), D( * ), DF( * ),
     $                   E( * ), EF( * ), FERR( * ), WORK( * ),
     $                   X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOFACT
      DOUBLE PRECISION   ANORM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, DLANST
      EXTERNAL           LSAME, DLAMCH, DLANST
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DLACPY, DPTCON, DPTRFS, DPTTRF, DPTTRS,
     $                   XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      IF( .NOT.NOFACT .AND. .NOT.LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
```

```
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -11
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DPTSVX', -INFO )
         RETURN
      END IF
*
      IF( NOFACT ) THEN
*
*        Compute the L*D*L**T (or U**T*D*U) factorization of A.
*
         CALL DCOPY( N, D, 1, DF, 1 )
         IF( N.GT.1 )
     $      CALL DCOPY( N-1, E, 1, EF, 1 )
         CALL DPTTRF( N, DF, EF, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 )THEN
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A.
*
      ANORM = DLANST( '1', N, D, E )
*
*     Compute the reciprocal of the condition number of A.
*
      CALL DPTCON( N, DF, EF, ANORM, RCOND, WORK, INFO )
*
*     Compute the solution vectors X.
*
      CALL DLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL DPTTRS( N, NRHS, DF, EF, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solutions and
*     compute error bounds and backward error estimates for them.
*
      CALL DPTRFS( N, NRHS, D, E, DF, EF, B, LDB, X, LDX, FERR, BERR,
     $             WORK, INFO )
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.DLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      RETURN
```

```
*
*     End of DPTSVX
*
      END
```

———-

82.2.4 lisp code

— **dptsvx.lisp** —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun dptsvx (fact n nrhs d e df ef b ldb$ x ldx rcond ferr berr work info)
    (declare (type (double-float) rcond)
             (type (array double-float (*)) work berr ferr x b ef df e d)
             (type (f2cl-lib:integer4) info ldx ldb$ nrhs n)
             (type (string *) fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%)
         (df double-float df-%data% df-%offset%)
         (ef double-float ef-%data% ef-%offset%)
         (b double-float b-%data% b-%offset%)
         (x double-float x-%data% x-%offset%)
         (ferr double-float ferr-%data% ferr-%offset%)
         (berr double-float berr-%data% berr-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((anorm 0.0d0) (nofact nil))
        (declare (type (double-float) anorm)
                 (type f2cl-lib:logical nofact))
        (setf info 0)
        (setf nofact
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame fact "N")
                  (declare (ignore var-1))
                  (when var-0 (setf fact var-0))
                  ret-val))
        (cond
         ((and (not nofact)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame fact "F")
                  (declare (ignore var-1))
                  (when var-0 (setf fact var-0))
                  ret-val)))
```

```
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -11)))
(cond
 ((/= info 0) (xerbla "DPTSVX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (nofact
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy n d 1 df 1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf n var-0)))
  (if (> n 1)
      (dcopy (f2cl-lib:int-sub n 1) e 1 ef 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dpttrf n df ef info)
    (declare (ignore var-1 var-2))
    (when var-0 (setf n var-0))
    (when var-3 (setf info var-3)))
  (cond ((> info 0) (setf rcond zero) (go end_label)))))
(setf anorm
        (multiple-value-bind (ret-val var-0 var-1 var-2 var-3)
            (dlanst "1" n d e)
          (declare (ignore var-0 var-2 var-3))
          (setf n var-1)
          ret-val))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dptcon n df ef anorm rcond work info)
  (declare (ignore var-1 var-2 var-3 var-5))
  (setf n var-0)
  (setf rcond var-4)
  (setf info var-6))
(dlacpy "Full" n nrhs b ldb$ x ldx)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dpttrs n nrhs df ef x ldx info)
  (declare (ignore var-2 var-3 var-4))
  (when var-0 (setf n var-0))
  (when var-1 (setf nrhs var-1))
  (when var-5 (setf ldx var-5))
  (when var-6 (setf info var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13)
    (dptrfs n nrhs d e df ef b ldb$ x ldx ferr berr work info)
  (declare
   (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
    var-11 var-12))
```

```
          (setf n var-0)
          (setf info var-13))
        (if (< rcond (dlamch "Epsilon"))
            (setf info (f2cl-lib:int-add n 1)))
        (go end_label)
       end_label
        (return
         (values fact n nrhs nil nil nil nil nil nil nil ldx rcond nil nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dptsvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::fact
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil nil nil
                                              nil nil nil nil
                                              fortran-to-lisp::ldx
                                              fortran-to-lisp::rcond nil nil
                                              nil fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::dptrfs
                                              fortran-to-lisp::dlacpy
                                              fortran-to-lisp::dptcon
                                              fortran-to-lisp::dlanst))))
```

Chapter 83

LAPACK - Positive Definite Tridiagonal Matrix, Linear Solve, Real

83.1 sptsv

83.1.1 Axiom unit tests

— **sptsv.input** —

```
)set break resume
)sys rm -f sptsv.output
)spool sptsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

83.1.2 Axiom help page

— **sptsv.help** —

```
==========================================================================
```

```
sptsv examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sptsv (N, NRHS, D, E, B, LDB, INFO)
            SPTSV computes the solution to system of linear equations A * X =
           B for PT matrices

Function/Subroutine Documentation
   subroutine sptsv (integer N, integer NRHS, real, dimension( * ) D, real,
       dimension( * ) E, real, dimension( ldb, * ) B, integer LDB, integer
       INFO)
        SPTSV computes the solution to system of linear equations A * X = B
       for PT matrices

Purpose:
   SPTSV computes the solution to a real system of linear equations
   A*X = B, where A is an N-by-N symmetric positive definite tridiagonal
   matrix, and X and B are N-by-NRHS matrices.

   A is factored as A = L*D*L**T, and the factored form of A is then
   used to solve the system of equations.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 D
   D is REAL array, dimension (N)
   On entry, the n diagonal elements of the tridiagonal matrix
   A.  On exit, the n diagonal elements of the diagonal matrix
   D from the factorization A = L*D*L**T.

 E
   E is REAL array, dimension (N-1)
   On entry, the (n-1) subdiagonal elements of the tridiagonal
   matrix A.  On exit, the (n-1) subdiagonal elements of the
   unit bidiagonal factor L from the L*D*L**T factorization of
```

```
  A.  (E can also be regarded as the superdiagonal of the unit
  bidiagonal factor U from the U**T*D*U factorization of A.)

B
  B is REAL array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, the leading minor of order i is not
        positive definite, and the solution has not been
        computed.  The factorization has not been completed unless i = N.
```

83.1.3 fortran code

— sptsv.f —

```
*  =====================================================================
      SUBROUTINE SPTSV( N, NRHS, D, E, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      REAL               B( LDB, * ), D( * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. External Subroutines ..
      EXTERNAL           SPTTRF, SPTTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SPTSV ', -INFO )
         RETURN
      END IF
*
*     Compute the L*D*L**T (or U**T*D*U) factorization of A.
*
      CALL SPTTRF( N, D, E, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
         CALL SPTTRS( N, NRHS, D, E, B, LDB, INFO )
      END IF
      RETURN
*
*     End of SPTSV
*
      END
```

———-

83.1.4 lisp code

— sptsv.lisp —

```
(defun sptsv (n nrhs d e b ldb$ info)
  (declare (type (array single-float (*)) b e d)
           (type (f2cl-lib:integer4) info ldb$ nrhs n))
  (f2cl-lib:with-multi-array-data
      ((d single-float d-%data% d-%offset%)
       (e single-float e-%data% e-%offset%)
       (b single-float b-%data% b-%offset%))
    (prog ()
```

```
      (declare)
      (setf info 0)
      (cond ((< n 0) (setf info -1)) ((< nrhs 0) (setf info -2))
            ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -6)))
      (cond
       ((/= info 0) (xerbla "SPTSV " (f2cl-lib:int-sub info)) (go end_label)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (spttrf n d e info)
        (declare (ignore var-1 var-2))
        (when var-0 (setf n var-0))
        (when var-3 (setf info var-3)))
      (cond
       ((= info 0)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (spttrs n nrhs d e b ldb$ info)
          (declare (ignore var-2 var-3 var-4))
          (when var-0 (setf n var-0))
          (when var-1 (setf nrhs var-1))
          (when var-5 (setf ldb$ var-5))
          (when var-6 (setf info var-6)))))
      (go end_label)
     end_label
      (return (values n nrhs nil nil nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sptsv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil nil nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

83.2 sptsvx

83.2.1 Axiom unit tests

— **sptsvx.input** —

```
)set break resume
)sys rm -f sptsvx.output
)spool sptsvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

83.2.2 Axiom help page

— **sptsvx.help** —

```
====================================================================
sptsvx examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sptsvx (FACT, N, NRHS, D, E, DF, EF, B, LDB, X, LDX, RCOND,
           FERR, BERR, WORK, INFO)
            SPTSVX computes the solution to system of linear equations A * X =
           B for PT matrices

Function/Subroutine Documentation
   subroutine sptsvx (character FACT, integer N, integer NRHS, real,
       dimension( * ) D, real, dimension( * ) E, real, dimension( * ) DF,
       real, dimension( * ) EF, real, dimension( ldb, * ) B, integer LDB,
       real, dimension( ldx, * ) X, integer LDX, real RCOND, real, dimension(
       * ) FERR, real, dimension( * ) BERR, real, dimension( * ) WORK, integer
       INFO)
        SPTSVX computes the solution to system of linear equations A * X = B
       for PT matrices
```

```
Purpose:
   SPTSVX uses the factorization A = L*D*L**T to compute the solution
   to a real system of linear equations A*X = B, where A is an N-by-N
   symmetric positive definite tridiagonal matrix and X and B are
   N-by-NRHS matrices.

   Error bounds on the solution and a condition estimate are also provided.

Description:

   The following steps are performed:

   1. If FACT = 'N', the matrix A is factored as A = L*D*L**T, where L
      is a unit lower bidiagonal matrix and D is diagonal.  The
      factorization can also be regarded as having the form
      A = U**T*D*U.

   2. If the leading i-by-i principal minor is not positive definite,
      then the routine returns with INFO = i. Otherwise, the factored
      form of A is used to estimate the condition number of the matrix
      A.  If the reciprocal of the condition number is less than machine
      precision, INFO = N+1 is returned as a warning, but the routine
      still goes on to solve for X and compute error bounds as described below.

   3. The system of equations is solved for X using the factored form of A.

   4. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of A has been
   supplied on entry.
   = 'F':  On entry, DF and EF contain the factored form of A.
           D, E, DF, and EF will not be modified.
   = 'N':  The matrix A will be copied to DF and EF and factored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrices B and X.  NRHS >= 0.

 D
```

```
  D is REAL array, dimension (N)
  The n diagonal elements of the tridiagonal matrix A.

E
  E is REAL array, dimension (N-1)
  The (n-1) subdiagonal elements of the tridiagonal matrix A.

DF
  DF is REAL array, dimension (N)
  If FACT = 'F', then DF is an input argument and on entry
  contains the n diagonal elements of the diagonal matrix D
  from the L*D*L**T factorization of A.
  If FACT = 'N', then DF is an output argument and on exit
  contains the n diagonal elements of the diagonal matrix D
  from the L*D*L**T factorization of A.

EF
  EF is REAL array, dimension (N-1)
  If FACT = 'F', then EF is an input argument and on entry
  contains the (n-1) subdiagonal elements of the unit
  bidiagonal factor L from the L*D*L**T factorization of A.
  If FACT = 'N', then EF is an output argument and on exit
  contains the (n-1) subdiagonal elements of the unit
  bidiagonal factor L from the L*D*L**T factorization of A.

B
  B is REAL array, dimension (LDB,NRHS)
  The N-by-NRHS right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is REAL array, dimension (LDX,NRHS)
  If INFO = 0 of INFO = N+1, the N-by-NRHS solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  The reciprocal condition number of the matrix A.  If RCOND
  is less than the machine precision (in particular, if
  RCOND = 0), the matrix is singular to working precision.
  This condition is indicated by a return code of INFO > 0.

FERR
  FERR is REAL array, dimension (NRHS)
```

```
  The forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).

BERR
  BERR is REAL array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in any
  element of A or B that makes X(j) an exact solution).

WORK
  WORK is REAL array, dimension (2*N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, and i is
  <= N:  the leading minor of order i of A is
     not positive definite, so the factorization
     could not be completed, and the solution has not
     been computed. RCOND = 0 is returned.
  = N+1: U is nonsingular, but RCOND is less than machine
     precision, meaning that the matrix is singular
     to working precision.  Nevertheless, the
     solution and error bounds are computed because
     there are a number of situations where the
     computed solution can be more accurate than the
     value of RCOND would suggest.
```

———-

83.2.3 fortran code

— sptsvx.f —

```
*  ====================================================================
      SUBROUTINE SPTSVX( FACT, N, NRHS, D, E, DF, EF, B, LDB, X, LDX,
     $                   RCOND, FERR, BERR, WORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
```

```
*
*     .. Scalar Arguments ..
      CHARACTER          FACT
      INTEGER            INFO, LDB, LDX, N, NRHS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      REAL               B( LDB, * ), BERR( * ), D( * ), DF( * ),
     $                   E( * ), EF( * ), FERR( * ), WORK( * ),
     $                   X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOFACT
      REAL               ANORM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH, SLANST
      EXTERNAL           LSAME, SLAMCH, SLANST
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SLACPY, SPTCON, SPTRFS, SPTTRF, SPTTRS,
     $                   XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      IF( .NOT.NOFACT .AND. .NOT.LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -11
```

```
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SPTSVX', -INFO )
         RETURN
      END IF
*
      IF( NOFACT ) THEN
*
*        Compute the L*D*L**T (or U**T*D*U) factorization of A.
*
         CALL SCOPY( N, D, 1, DF, 1 )
         IF( N.GT.1 )
     $      CALL SCOPY( N-1, E, 1, EF, 1 )
         CALL SPTTRF( N, DF, EF, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 )THEN
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A.
*
      ANORM = SLANST( '1', N, D, E )
*
*     Compute the reciprocal of the condition number of A.
*
      CALL SPTCON( N, DF, EF, ANORM, RCOND, WORK, INFO )
*
*     Compute the solution vectors X.
*
      CALL SLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL SPTTRS( N, NRHS, DF, EF, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solutions and
*     compute error bounds and backward error estimates for them.
*
      CALL SPTRFS( N, NRHS, D, E, DF, EF, B, LDB, X, LDX, FERR, BERR,
     $             WORK, INFO )
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.SLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      RETURN
*
*     End of SPTSVX
```

```
*
      END
```

———-

83.2.4 lisp code

— sptsvx.lisp —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun sptsvx (fact n nrhs d e df ef b ldb$ x ldx rcond ferr berr work info)
    (declare (type (single-float) rcond)
             (type (array single-float (*)) work berr ferr x b ef df e d)
             (type (f2cl-lib:integer4) info ldx ldb$ nrhs n)
             (type (string *) fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%)
         (df single-float df-%data% df-%offset%)
         (ef single-float ef-%data% ef-%offset%)
         (b single-float b-%data% b-%offset%)
         (x single-float x-%data% x-%offset%)
         (ferr single-float ferr-%data% ferr-%offset%)
         (berr single-float berr-%data% berr-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((anorm 0.0) (nofact nil))
        (declare (type (single-float) anorm)
                 (type f2cl-lib:logical nofact))
        (setf info 0)
        (setf nofact
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame fact "N")
                  (declare (ignore var-1))
                  (when var-0 (setf fact var-0))
                  ret-val))
        (cond
         ((and (not nofact)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame fact "F")
                  (declare (ignore var-1))
                  (when var-0 (setf fact var-0))
                  ret-val)))
          (setf info -1))
         ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
```

```
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -11)))
(cond
 ((/= info 0) (xerbla "SPTSVX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (nofact
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (scopy n d 1 df 1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf n var-0)))
  (if (> n 1)
      (scopy (f2cl-lib:int-sub n 1) e 1 ef 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (spttrf n df ef info)
    (declare (ignore var-1 var-2))
    (when var-0 (setf n var-0))
    (when var-3 (setf info var-3)))
  (cond ((> info 0) (setf rcond zero) (go end_label)))))
(setf anorm
        (multiple-value-bind (ret-val var-0 var-1 var-2 var-3)
            (slanst "1" n d e)
          (declare (ignore var-0 var-2 var-3))
          (setf n var-1)
          ret-val))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (sptcon n df ef anorm rcond work info)
  (declare (ignore var-1 var-2 var-3 var-5))
  (setf n var-0)
  (setf rcond var-4)
  (setf info var-6))
(slacpy "Full" n nrhs b ldb$ x ldx)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (spttrs n nrhs df ef x ldx info)
  (declare (ignore var-2 var-3 var-4))
  (when var-0 (setf n var-0))
  (when var-1 (setf nrhs var-1))
  (when var-5 (setf ldx var-5))
  (when var-6 (setf info var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13)
    (sptrfs n nrhs d e df ef b ldb$ x ldx ferr berr work info)
  (declare
   (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
    var-11 var-12))
  (setf n var-0)
  (setf info var-13))
```

```
        (if (< rcond (slamch "Epsilon"))
            (setf info (f2cl-lib:int-add n 1)))
        (go end_label)
       end_label
        (return
         (values fact n nrhs nil nil nil nil nil nil nil ldx rcond nil nil nil
                 info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sptsvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::fact
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil nil nil
                                              nil nil nil nil
                                              fortran-to-lisp::ldx
                                              fortran-to-lisp::rcond nil nil
                                              nil fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::sptrfs
                                              fortran-to-lisp::slacpy
                                              fortran-to-lisp::sptcon
                                              fortran-to-lisp::slanst))))
```

Chapter 84

LAPACK - Positive Definite Tridiagonal Matrix, Linear Solve, Complex

84.1 cptsv

84.1.1 Axiom unit tests

— **cptsv.input** —

```
)set break resume
)sys rm -f cptsv.output
)spool cptsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

84.1.2 Axiom help page

— **cptsv.help** —

```
==========================================================================
```

```
cptsv examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cptsv (N, NRHS, D, E, B, LDB, INFO)
            CPTSV computes the solution to system of linear equations A * X =
           B for PT matrices

Function/Subroutine Documentation
   subroutine cptsv (integer N, integer NRHS, real, dimension( * ) D, complex,
       dimension( * ) E, complex, dimension( ldb, * ) B, integer LDB, integer
       INFO)
        CPTSV computes the solution to system of linear equations A * X = B
       for PT matrices

Purpose:
   CPTSV computes the solution to a complex system of linear equations
   A*X = B, where A is an N-by-N Hermitian positive definite tridiagonal
   matrix, and X and B are N-by-NRHS matrices.

   A is factored as A = L*D*L**H, and the factored form of A is then
   used to solve the system of equations.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 D
   D is REAL array, dimension (N)
   On entry, the n diagonal elements of the tridiagonal matrix
   A.  On exit, the n diagonal elements of the diagonal matrix
   D from the factorization A = L*D*L**H.

 E
   E is COMPLEX array, dimension (N-1)
   On entry, the (n-1) subdiagonal elements of the tridiagonal
   matrix A.  On exit, the (n-1) subdiagonal elements of the
   unit bidiagonal factor L from the L*D*L**H factorization of
```

```
  A.  E can also be regarded as the superdiagonal of the unit
  bidiagonal factor U from the U**H*D*U factorization of A.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, the leading minor of order i is not
        positive definite, and the solution has not been
        computed.  The factorization has not been completed unless i = N.
```

84.1.3 fortran code

— cptsv.f —

```
*  =====================================================================
      SUBROUTINE CPTSV( N, NRHS, D, E, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      REAL               D( * )
      COMPLEX            B( LDB, * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. External Subroutines ..
      EXTERNAL           CPTTRF, CPTTRS, XERBLA
*     ..
```

```
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CPTSV ', -INFO )
         RETURN
      END IF
*
*     Compute the L*D*L**H (or U**H*D*U) factorization of A.
*
      CALL CPTTRF( N, D, E, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
         CALL CPTTRS( 'Lower', N, NRHS, D, E, B, LDB, INFO )
      END IF
      RETURN
*
*     End of CPTSV
*
      END
```

———-

84.1.4 lisp code

— cptsv.lisp —

```
(defun cptsv (n nrhs d e b ldb$ info)
  (declare (type (array f2cl-lib:complex8 (*)) b e)
           (type (array single-float (*)) d)
           (type (f2cl-lib:integer4) info ldb$ nrhs n))
  (f2cl-lib:with-multi-array-data
      ((d single-float d-%data% d-%offset%)
       (e f2cl-lib:complex8 e-%data% e-%offset%)
```

```
       (b f2cl-lib:complex8 b-%data% b-%offset%))
    (prog ()
      (declare)
      (setf info 0)
      (cond ((< n 0) (setf info -1)) ((< nrhs 0) (setf info -2))
            ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -6)))
      (cond
       ((/= info 0) (xerbla "CPTSV " (f2cl-lib:int-sub info)) (go end_label)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (cpttrf n d e info)
        (declare (ignore var-1 var-2))
        (when var-0 (setf n var-0))
        (when var-3 (setf info var-3)))
      (cond
       ((= info 0)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (cpttrs "Lower" n nrhs d e b ldb$ info)
          (declare (ignore var-0 var-3 var-4 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-6 (setf ldb$ var-6))
          (when var-7 (setf info var-7)))))
      (go end_label)
     end_label
      (return (values n nrhs nil nil nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cptsv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil nil nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

84.2 cptsvx

84.2.1 Axiom unit tests

— cptsvx.input —

```
)set break resume
)sys rm -f cptsvx.output
)spool cptsvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

84.2.2 Axiom help page

— cptsvx.help —

```
======================================================================
cptsvx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cptsvx (FACT, N, NRHS, D, E, DF, EF, B, LDB, X, LDX, RCOND,
           FERR, BERR, WORK, RWORK, INFO)
            CPTSVX computes the solution to system of linear equations A * X =
           B for PT matrices

Function/Subroutine Documentation
   subroutine cptsvx (character FACT, integer N, integer NRHS, real,
       dimension( * ) D, complex, dimension( * ) E, real, dimension( * ) DF,
       complex, dimension( * ) EF, complex, dimension( ldb, * ) B, integer
       LDB, complex, dimension( ldx, * ) X, integer LDX, real RCOND, real,
```

```
      dimension( * ) FERR, real, dimension( * ) BERR, complex, dimension( * )
      WORK, real, dimension( * ) RWORK, integer INFO)
       CPTSVX computes the solution to system of linear equations A * X = B
      for PT matrices

Purpose:
   CPTSVX uses the factorization A = L*D*L**H to compute the solution
   to a complex system of linear equations A*X = B, where A is an
   N-by-N Hermitian positive definite tridiagonal matrix and X and B
   are N-by-NRHS matrices.

   Error bounds on the solution and a condition estimate are also  provided.

Description:

   The following steps are performed:

   1. If FACT = 'N', the matrix A is factored as A = L*D*L**H, where L
      is a unit lower bidiagonal matrix and D is diagonal.  The
      factorization can also be regarded as having the form
      A = U**H*D*U.

   2. If the leading i-by-i principal minor is not positive definite,
      then the routine returns with INFO = i. Otherwise, the factored
      form of A is used to estimate the condition number of the matrix
      A.  If the reciprocal of the condition number is less than machine
      precision, INFO = N+1 is returned as a warning, but the routine
      still goes on to solve for X and compute error bounds as
      described below.

   3. The system of equations is solved for X using the factored form of A.

   4. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix
   A is supplied on entry.
   = 'F':  On entry, DF and EF contain the factored form of A.
           D, E, DF, and EF will not be modified.
   = 'N':  The matrix A will be copied to DF and EF and factored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
```

```
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

D
  D is REAL array, dimension (N)
  The n diagonal elements of the tridiagonal matrix A.

E
  E is COMPLEX array, dimension (N-1)
  The (n-1) subdiagonal elements of the tridiagonal matrix A.

DF
  DF is REAL array, dimension (N)
  If FACT = 'F', then DF is an input argument and on entry
  contains the n diagonal elements of the diagonal matrix D
  from the L*D*L**H factorization of A.
  If FACT = 'N', then DF is an output argument and on exit
  contains the n diagonal elements of the diagonal matrix D
  from the L*D*L**H factorization of A.

EF
  EF is COMPLEX array, dimension (N-1)
  If FACT = 'F', then EF is an input argument and on entry
  contains the (n-1) subdiagonal elements of the unit
  bidiagonal factor L from the L*D*L**H factorization of A.
  If FACT = 'N', then EF is an output argument and on exit
  contains the (n-1) subdiagonal elements of the unit
  bidiagonal factor L from the L*D*L**H factorization of A.

B
  B is COMPLEX array, dimension (LDB,NRHS)
  The N-by-NRHS right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is REAL
  The reciprocal condition number of the matrix A.  If RCOND
  is less than the machine precision (in particular, if
```

```
  RCOND = 0), the matrix is singular to working precision.
  This condition is indicated by a return code of INFO > 0.

FERR
  FERR is REAL array, dimension (NRHS)
  The forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).

BERR
  BERR is REAL array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in any
  element of A or B that makes X(j) an exact solution).

WORK
  WORK is COMPLEX array, dimension (N)

RWORK
  RWORK is REAL array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, and i is
  <= N:  the leading minor of order i of A is
     not positive definite, so the factorization
     could not be completed, and the solution has not
     been computed. RCOND = 0 is returned.
  = N+1: U is nonsingular, but RCOND is less than machine
     precision, meaning that the matrix is singular
     to working precision.  Nevertheless, the
     solution and error bounds are computed because
     there are a number of situations where the
     computed solution can be more accurate than the
     value of RCOND would suggest.
```

———-

84.2.3 fortran code

— cptsvx.f —

```
*  =====================================================================
      SUBROUTINE CPTSVX( FACT, N, NRHS, D, E, DF, EF, B, LDB, X, LDX,
     $                   RCOND, FERR, BERR, WORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          FACT
      INTEGER            INFO, LDB, LDX, N, NRHS
      REAL               RCOND
*     ..
*     .. Array Arguments ..
      REAL               BERR( * ), D( * ), DF( * ), FERR( * ),
     $                   RWORK( * )
      COMPLEX            B( LDB, * ), E( * ), EF( * ), WORK( * ),
     $                   X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOFACT
      REAL               ANORM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               CLANHT, SLAMCH
      EXTERNAL           LSAME, CLANHT, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           CCOPY, CLACPY, CPTCON, CPTRFS, CPTTRF, CPTTRS,
     $                   SCOPY, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      IF( .NOT.NOFACT .AND. .NOT.LSAME( FACT, 'F' ) ) THEN
```

```
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -11
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CPTSVX', -INFO )
         RETURN
      END IF
*
      IF( NOFACT ) THEN
*
*        Compute the L*D*L**H (or U**H*D*U) factorization of A.
*
         CALL SCOPY( N, D, 1, DF, 1 )
         IF( N.GT.1 )
     $      CALL CCOPY( N-1, E, 1, EF, 1 )
         CALL CPTTRF( N, DF, EF, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 )THEN
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A.
*
      ANORM = CLANHT( '1', N, D, E )
*
*     Compute the reciprocal of the condition number of A.
*
      CALL CPTCON( N, DF, EF, ANORM, RCOND, RWORK, INFO )
*
*     Compute the solution vectors X.
*
      CALL CLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL CPTTRS( 'Lower', N, NRHS, DF, EF, X, LDX, INFO )
*
*     Use iterative refinement to improve the computed solutions and
*     compute error bounds and backward error estimates for them.
*
      CALL CPTRFS( 'Lower', N, NRHS, D, E, DF, EF, B, LDB, X, LDX, FERR,
     $             BERR, WORK, RWORK, INFO )
```

```
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.SLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      RETURN
*
*     End of CPTSVX
*
      END
```

———-

84.2.4 lisp code

— **cptsvx.lisp** —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun cptsvx
         (fact n nrhs d e df ef b ldb$ x ldx rcond ferr berr work rwork info)
    (declare (type (single-float) rcond)
             (type (array f2cl-lib:complex8 (*)) work x b ef e)
             (type (array single-float (*)) rwork berr ferr df d)
             (type (f2cl-lib:integer4) info ldx ldb$ nrhs n)
             (type (string *) fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (d single-float d-%data% d-%offset%)
         (df single-float df-%data% df-%offset%)
         (ferr single-float ferr-%data% ferr-%offset%)
         (berr single-float berr-%data% berr-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%)
         (e f2cl-lib:complex8 e-%data% e-%offset%)
         (ef f2cl-lib:complex8 ef-%data% ef-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%))
      (prog ((anorm 0.0) (nofact nil))
        (declare (type (single-float) anorm)
                 (type f2cl-lib:logical nofact))
        (setf info 0)
        (setf nofact
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame fact "N")
                  (declare (ignore var-1))
```

```
          (when var-0 (setf fact var-0))
          ret-val))
(cond
 ((and (not nofact)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -11)))
(cond
 ((/= info 0) (xerbla "CPTSVX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (nofact
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (scopy n d 1 df 1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf n var-0)))
  (if (> n 1)
      (ccopy (f2cl-lib:int-sub n 1) e 1 ef 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (cpttrf n df ef info)
    (declare (ignore var-1 var-2))
    (when var-0 (setf n var-0))
    (when var-3 (setf info var-3)))
  (cond ((> info 0) (setf rcond zero) (go end_label)))))
(setf anorm
        (multiple-value-bind (ret-val var-0 var-1 var-2 var-3)
            (clanht "1" n d e)
          (declare (ignore var-0 var-2 var-3))
          (setf n var-1)
          ret-val))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (cptcon n df ef anorm rcond rwork info)
  (declare (ignore var-1 var-2 var-3 var-5))
  (setf n var-0)
  (setf rcond var-4)
  (setf info var-6))
(clacpy "Full" n nrhs b ldb$ x ldx)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (cpttrs "Lower" n nrhs df ef x ldx info)
  (declare (ignore var-0 var-3 var-4 var-5))
  (when var-1 (setf n var-1))
```

```
          (when var-2 (setf nrhs var-2))
          (when var-6 (setf ldx var-6))
          (when var-7 (setf info var-7)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
             var-11 var-12 var-13 var-14 var-15)
            (cptrfs "Lower" n nrhs d e df ef b ldb$ x ldx ferr berr work rwork
             info)
          (declare
           (ignore var-0 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
            var-11 var-12 var-13 var-14))
          (setf n var-1)
          (setf info var-15))
        (if (< rcond (slamch "Epsilon"))
            (setf info (f2cl-lib:int-add n 1)))
        (go end_label)
       end_label
        (return
         (values fact n nrhs nil nil nil nil nil nil nil ldx rcond nil nil nil
                 nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cptsvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
```

```
'(fortran-to-lisp::fact
  fortran-to-lisp::n
  fortran-to-lisp::nrhs nil nil nil
  nil nil nil nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond nil nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::cptrfs
  fortran-to-lisp::clacpy
  fortran-to-lisp::cptcon
  fortran-to-lisp::clanht))))
```

———-

Chapter 85

LAPACK - Positive Definite Tridiagonal Matrix, Linear Solve, Complex16

85.1 zptsv

85.1.1 Axiom unit tests

— **zptsv.input** —

```
)set break resume
)sys rm -f zptsv.output
)spool zptsv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

85.1.2 Axiom help page

— **zptsv.help** —

```
==========================================================================
```

```
zptsv examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zptsv (N, NRHS, D, E, B, LDB, INFO)
            ZPTSV computes the solution to system of linear equations A * X =
           B for PT matrices

Function/Subroutine Documentation
   subroutine zptsv (integer N, integer NRHS, double precision, dimension( * )
       D, complex*16, dimension( * ) E, complex*16, dimension( ldb, * ) B,
       integer LDB, integer INFO)
        ZPTSV computes the solution to system of linear equations A * X = B
       for PT matrices

Purpose:
   ZPTSV computes the solution to a complex system of linear equations
   A*X = B, where A is an N-by-N Hermitian positive definite tridiagonal
   matrix, and X and B are N-by-NRHS matrices.

   A is factored as A = L*D*L**H, and the factored form of A is then
   used to solve the system of equations.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 D
   D is DOUBLE PRECISION array, dimension (N)
   On entry, the n diagonal elements of the tridiagonal matrix
   A.  On exit, the n diagonal elements of the diagonal matrix
   D from the factorization A = L*D*L**H.

 E
   E is COMPLEX*16 array, dimension (N-1)
   On entry, the (n-1) subdiagonal elements of the tridiagonal
   matrix A.  On exit, the (n-1) subdiagonal elements of the
   unit bidiagonal factor L from the L*D*L**H factorization of
```

```
  A.  E can also be regarded as the superdiagonal of the unit
  bidiagonal factor U from the U**H*D*U factorization of A.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  On entry, the N-by-NRHS right hand side matrix B.
  On exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, the leading minor of order i is not
     positive definite, and the solution has not been
     computed.  The factorization has not been completed unless i = N.
```

85.1.3 fortran code

— zptsv.f —

```
*  =====================================================================
      SUBROUTINE ZPTSV( N, NRHS, D, E, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * )
      COMPLEX*16         B( LDB, * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZPTTRF, ZPTTRS
*     ..
```

```
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZPTSV ', -INFO )
         RETURN
      END IF
*
*     Compute the L*D*L**H (or U**H*D*U) factorization of A.
*
      CALL ZPTTRF( N, D, E, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
         CALL ZPTTRS( 'Lower', N, NRHS, D, E, B, LDB, INFO )
      END IF
      RETURN
*
*     End of ZPTSV
*
      END
```

———-

85.1.4 lisp code

— zptsv.lisp —

```
(defun zptsv (n nrhs d e b ldb$ info)
  (declare (type (array f2cl-lib:complex16 (*)) b e)
           (type (array double-float (*)) d)
           (type (f2cl-lib:integer4) info ldb$ nrhs n))
  (f2cl-lib:with-multi-array-data
      ((d double-float d-%data% d-%offset%)
       (e f2cl-lib:complex16 e-%data% e-%offset%)
```

```
       (b f2cl-lib:complex16 b-%data% b-%offset%))
    (prog ()
      (declare)
      (setf info 0)
      (cond ((< n 0) (setf info -1)) ((< nrhs 0) (setf info -2))
            ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -6)))
      (cond
       ((/= info 0) (xerbla "ZPTSV " (f2cl-lib:int-sub info)) (go end_label)))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (zpttrf n d e info)
        (declare (ignore var-1 var-2))
        (when var-0 (setf n var-0))
        (when var-3 (setf info var-3)))
      (cond
       ((= info 0)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (zpttrs "Lower" n nrhs d e b ldb$ info)
          (declare (ignore var-0 var-3 var-4 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-6 (setf ldb$ var-6))
          (when var-7 (setf info var-7)))))
      (go end_label)
     end_label
      (return (values n nrhs nil nil nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zptsv fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil nil nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

85.2 zptsvx

85.2.1 Axiom unit tests

— zptsvx.input —

```
)set break resume
)sys rm -f zptsvx.output
)spool zptsvx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

85.2.2 Axiom help page

— zptsvx.help —

```
======================================================================
zptsvx examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zptsvx (FACT, N, NRHS, D, E, DF, EF, B, LDB, X, LDX, RCOND,
           FERR, BERR, WORK, RWORK, INFO)
            ZPTSVX computes the solution to system of linear equations A * X =
           B for PT matrices

Function/Subroutine Documentation
   subroutine zptsvx (character FACT, integer N, integer NRHS, double
       precision, dimension( * ) D, complex*16, dimension( * ) E, double
       precision, dimension( * ) DF, complex*16, dimension( * ) EF,
       complex*16, dimension( ldb, * ) B, integer LDB, complex*16, dimension(
```

```
      ldx, * ) X, integer LDX, double precision RCOND, double precision,
      dimension( * ) FERR, double precision, dimension( * ) BERR, complex*16,
      dimension( * ) WORK, double precision, dimension( * ) RWORK, integer
      INFO)
       ZPTSVX computes the solution to system of linear equations A * X = B
      for PT matrices

Purpose:
   ZPTSVX uses the factorization A = L*D*L**H to compute the solution
   to a complex system of linear equations A*X = B, where A is an
   N-by-N Hermitian positive definite tridiagonal matrix and X and B
   are N-by-NRHS matrices.

   Error bounds on the solution and a condition estimate are also provided.

Description:

   The following steps are performed:

   1. If FACT = 'N', the matrix A is factored as A = L*D*L**H, where L
      is a unit lower bidiagonal matrix and D is diagonal.  The
      factorization can also be regarded as having the form
      A = U**H*D*U.

   2. If the leading i-by-i principal minor is not positive definite,
      then the routine returns with INFO = i. Otherwise, the factored
      form of A is used to estimate the condition number of the matrix
      A.  If the reciprocal of the condition number is less than machine
      precision, INFO = N+1 is returned as a warning, but the routine
      still goes on to solve for X and compute error bounds as
      described below.

   3. The system of equations is solved for X using the factored form of A.

   4. Iterative refinement is applied to improve the computed solution
      matrix and calculate error bounds and backward error estimates for it.

Parameters:

 FACT
   FACT is CHARACTER*1
   Specifies whether or not the factored form of the matrix
   A is supplied on entry.
   = 'F':  On entry, DF and EF contain the factored form of A.
           D, E, DF, and EF will not be modified.
   = 'N':  The matrix A will be copied to DF and EF and factored.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.
```

```
NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrices B and X.  NRHS >= 0.

D
  D is DOUBLE PRECISION array, dimension (N)
  The n diagonal elements of the tridiagonal matrix A.

E
  E is COMPLEX*16 array, dimension (N-1)
  The (n-1) subdiagonal elements of the tridiagonal matrix A.

DF
  DF is DOUBLE PRECISION array, dimension (N)
  If FACT = 'F', then DF is an input argument and on entry
  contains the n diagonal elements of the diagonal matrix D
  from the L*D*L**H factorization of A.
  If FACT = 'N', then DF is an output argument and on exit
  contains the n diagonal elements of the diagonal matrix D
  from the L*D*L**H factorization of A.

EF
  EF is COMPLEX*16 array, dimension (N-1)
  If FACT = 'F', then EF is an input argument and on entry
  contains the (n-1) subdiagonal elements of the unit
  bidiagonal factor L from the L*D*L**H factorization of A.
  If FACT = 'N', then EF is an output argument and on exit
  contains the (n-1) subdiagonal elements of the unit
  bidiagonal factor L from the L*D*L**H factorization of A.

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  The N-by-NRHS right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  If INFO = 0 or INFO = N+1, the N-by-NRHS solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

RCOND
  RCOND is DOUBLE PRECISION
```

```
  The reciprocal condition number of the matrix A.  If RCOND
  is less than the machine precision (in particular, if
  RCOND = 0), the matrix is singular to working precision.
  This condition is indicated by a return code of INFO > 0.

FERR
  FERR is DOUBLE PRECISION array, dimension (NRHS)
  The forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in any
  element of A or B that makes X(j) an exact solution).

WORK
  WORK is COMPLEX*16 array, dimension (N)

RWORK
  RWORK is DOUBLE PRECISION array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  if INFO = i, and i is
  <= N:  the leading minor of order i of A is
     not positive definite, so the factorization
     could not be completed, and the solution has not
     been computed. RCOND = 0 is returned.
  = N+1: U is nonsingular, but RCOND is less than machine
     precision, meaning that the matrix is singular
     to working precision.  Nevertheless, the
     solution and error bounds are computed because
     there are a number of situations where the
     computed solution can be more accurate than the
     value of RCOND would suggest.
```

———-

85.2.3 fortran code

— zptsvx.f —

```
*  =====================================================================
      SUBROUTINE ZPTSVX( FACT, N, NRHS, D, E, DF, EF, B, LDB, X, LDX,
     $                   RCOND, FERR, BERR, WORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          FACT
      INTEGER            INFO, LDB, LDX, N, NRHS
      DOUBLE PRECISION   RCOND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   BERR( * ), D( * ), DF( * ), FERR( * ),
     $                   RWORK( * )
      COMPLEX*16         B( LDB, * ), E( * ), EF( * ), WORK( * ),
     $                   X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOFACT
      DOUBLE PRECISION   ANORM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH, ZLANHT
      EXTERNAL           LSAME, DLAMCH, ZLANHT
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, XERBLA, ZCOPY, ZLACPY, ZPTCON, ZPTRFS,
     $                   ZPTTRF, ZPTTRS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
```

```
*     Test the input parameters.
*
      INFO = 0
      NOFACT = LSAME( FACT, 'N' )
      IF( .NOT.NOFACT .AND. .NOT.LSAME( FACT, 'F' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -11
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZPTSVX', -INFO )
         RETURN
      END IF
*
      IF( NOFACT ) THEN
*
*        Compute the L*D*L**H (or U**H*D*U) factorization of A.
*
         CALL DCOPY( N, D, 1, DF, 1 )
         IF( N.GT.1 )
     $      CALL ZCOPY( N-1, E, 1, EF, 1 )
         CALL ZPTTRF( N, DF, EF, INFO )
*
*        Return if INFO is non-zero.
*
         IF( INFO.GT.0 )THEN
            RCOND = ZERO
            RETURN
         END IF
      END IF
*
*     Compute the norm of the matrix A.
*
      ANORM = ZLANHT( '1', N, D, E )
*
*     Compute the reciprocal of the condition number of A.
*
      CALL ZPTCON( N, DF, EF, ANORM, RCOND, RWORK, INFO )
*
*     Compute the solution vectors X.
*
      CALL ZLACPY( 'Full', N, NRHS, B, LDB, X, LDX )
      CALL ZPTTRS( 'Lower', N, NRHS, DF, EF, X, LDX, INFO )
*
```

```
*     Use iterative refinement to improve the computed solutions and
*     compute error bounds and backward error estimates for them.
*
      CALL ZPTRFS( 'Lower', N, NRHS, D, E, DF, EF, B, LDB, X, LDX, FERR,
     $             BERR, WORK, RWORK, INFO )
*
*     Set INFO = N+1 if the matrix is singular to working precision.
*
      IF( RCOND.LT.DLAMCH( 'Epsilon' ) )
     $   INFO = N + 1
*
      RETURN
*
*     End of ZPTSVX
*
      END
```

85.2.4 lisp code

— zptsvx.lisp —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun zptsvx
         (fact n nrhs d e df ef b ldb$ x ldx rcond ferr berr work rwork info)
    (declare (type (double-float) rcond)
             (type (array f2cl-lib:complex16 (*)) work x b ef e)
             (type (array double-float (*)) rwork berr ferr df d)
             (type (f2cl-lib:integer4) info ldx ldb$ nrhs n)
             (type (string *) fact))
    (f2cl-lib:with-multi-array-data
        ((fact character fact-%data% fact-%offset%)
         (d double-float d-%data% d-%offset%)
         (df double-float df-%data% df-%offset%)
         (ferr double-float ferr-%data% ferr-%offset%)
         (berr double-float berr-%data% berr-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%)
         (e f2cl-lib:complex16 e-%data% e-%offset%)
         (ef f2cl-lib:complex16 ef-%data% ef-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (x f2cl-lib:complex16 x-%data% x-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%))
      (prog ((anorm 0.0d0) (nofact nil))
        (declare (type (double-float) anorm)
                 (type f2cl-lib:logical nofact))
```

```
(setf info 0)
(setf nofact
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "N")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val))
(cond
 ((and (not nofact)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame fact "F")
          (declare (ignore var-1))
          (when var-0 (setf fact var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -11)))
(cond
 ((/= info 0) (xerbla "ZPTSVX" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 (nofact
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy n d 1 df 1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf n var-0)))
  (if (> n 1)
      (zcopy (f2cl-lib:int-sub n 1) e 1 ef 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (zpttrf n df ef info)
    (declare (ignore var-1 var-2))
    (when var-0 (setf n var-0))
    (when var-3 (setf info var-3)))
  (cond ((> info 0) (setf rcond zero) (go end_label)))))
(setf anorm
        (multiple-value-bind (ret-val var-0 var-1 var-2 var-3)
            (zlanht "1" n d e)
          (declare (ignore var-0 var-2 var-3))
          (setf n var-1)
          ret-val))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zptcon n df ef anorm rcond rwork info)
  (declare (ignore var-1 var-2 var-3 var-5))
  (setf n var-0)
  (setf rcond var-4)
  (setf info var-6))
```

```
        (zlacpy "Full" n nrhs b ldb$ x ldx)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (zpttrs "Lower" n nrhs df ef x ldx info)
          (declare (ignore var-0 var-3 var-4 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-6 (setf ldx var-6))
          (when var-7 (setf info var-7)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
             var-11 var-12 var-13 var-14 var-15)
            (zptrfs "Lower" n nrhs d e df ef b ldb$ x ldx ferr berr work rwork
             info)
          (declare
           (ignore var-0 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
            var-11 var-12 var-13 var-14))
          (setf n var-1)
          (setf info var-15))
        (if (< rcond (dlamch "Epsilon"))
            (setf info (f2cl-lib:int-add n 1)))
        (go end_label)
       end_label
        (return
         (values fact n nrhs nil nil nil nil nil nil nil ldx rcond nil nil nil
                 nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zptsvx
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (double-float)
                                              (array double-float (*))
                                              (array double-float (*))
```

```
  (array fortran-to-lisp::complex16
   (*))
  (array double-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::fact
  fortran-to-lisp::n
  fortran-to-lisp::nrhs nil nil nil
  nil nil nil nil
  fortran-to-lisp::ldx
  fortran-to-lisp::rcond nil nil
  nil nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::zptrfs
  fortran-to-lisp::zlacpy
  fortran-to-lisp::zptcon
  fortran-to-lisp::zlanht
  fortran-to-lisp::xerbla))))
```

———-

Chapter 86

LAPACK - Positive Definite Tridiagonal Matrix, Comp. Routines, Double

86.1 dptcon

86.1.1 Axiom unit tests

— **dptcon.input** —

```
)set break resume
)sys rm -f dptcon.output
)spool dptcon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

86.1.2 Axiom help page

— **dptcon.help** —

```
==================================================================
```

```
dptcon examples
===================================================================

===================================================================
Man Page Details
===================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dptcon (N, D, E, ANORM, RCOND, WORK, INFO)
           DPTCON

Function/Subroutine Documentation
   subroutine dptcon (integer N, double precision, dimension( * ) D, double
       precision, dimension( * ) E, double precision ANORM, double precision
       RCOND, double precision, dimension( * ) WORK, integer INFO)
       DPTCON

Purpose:
   DPTCON computes the reciprocal of the condition number (in the
   1-norm) of a real symmetric positive definite tridiagonal matrix
   using the factorization A = L*D*L**T or A = U**T*D*U computed by
   DPTTRF.

   Norm(inv(A)) is computed by a direct method, and the reciprocal of
   the condition number is computed as
      RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 D
   D is DOUBLE PRECISION array, dimension (N)
   The n diagonal elements of the diagonal matrix D from the
   factorization of A, as computed by DPTTRF.

 E
   E is DOUBLE PRECISION array, dimension (N-1)
   The (n-1) off-diagonal elements of the unit bidiagonal factor
   U or L from the factorization of A,  as computed by DPTTRF.

 ANORM
   ANORM is DOUBLE PRECISION
   The 1-norm of the original matrix A.

 RCOND
   RCOND is DOUBLE PRECISION
```

```
  The reciprocal of the condition number of the matrix A,
  computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is the
  1-norm of inv(A) computed in this routine.

WORK
  WORK is DOUBLE PRECISION array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

86.1.3 fortran code

— dptcon.f —

```
*  =====================================================================
      SUBROUTINE DPTCON( N, D, E, ANORM, RCOND, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, N
      DOUBLE PRECISION   ANORM, RCOND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), E( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IX
      DOUBLE PRECISION   AINVNM
*     ..
*     .. External Functions ..
      INTEGER            IDAMAX
      EXTERNAL           IDAMAX
```

```
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DPTCON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
*     Check that D(1:N) is positive.
*
      DO 10 I = 1, N
         IF( D( I ).LE.ZERO )
     $      RETURN
   10 CONTINUE
*
*     Solve M(A) * x = e, where M(A) = (m(i,j)) is given by
*
*        m(i,j) =  abs(A(i,j)), i = j,
*        m(i,j) = -abs(A(i,j)), i .ne. j,
*
*     and e = [ 1, 1, ..., 1 ]**T.  Note M(A) = M(L)*D*M(L)**T.
*
*     Solve M(L) * x = e.
*
      WORK( 1 ) = ONE
      DO 20 I = 2, N
```

```
         WORK( I ) = ONE + WORK( I-1 )*ABS( E( I-1 ) )
   20 CONTINUE
*
*     Solve D * M(L)**T * x = b.
*
      WORK( N ) = WORK( N ) / D( N )
      DO 30 I = N - 1, 1, -1
         WORK( I ) = WORK( I ) / D( I ) + WORK( I+1 )*ABS( E( I ) )
   30 CONTINUE
*
*     Compute AINVNM = max(x(i)), 1<=i<=n.
*
      IX = IDAMAX( N, WORK, 1 )
      AINVNM = ABS( WORK( IX ) )
*
*     Compute the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
      RETURN
*
*     End of DPTCON
*
      END
```

———-

86.1.4 lisp code

— dptcon.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dptcon (n d e anorm rcond work info)
    (declare (type (double-float) rcond anorm)
             (type (array double-float (*)) work e d)
             (type (f2cl-lib:integer4) info n))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((ainvnm 0.0d0) (i 0) (ix 0))
        (declare (type (double-float) ainvnm)
                 (type (f2cl-lib:integer4) i ix))
        (setf info 0)
```

```
(cond ((< n 0) (setf info -1)) ((< anorm zero) (setf info -4)))
(cond
 ((/= info 0) (xerbla "DPTCON" (f2cl-lib:int-sub info))
  (go end_label)))
(setf rcond zero)
(cond ((= n 0) (setf rcond one) (go end_label))
      ((= anorm zero) (go end_label)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (if (<= (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) zero)
        (go end_label))
   label10))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) one)
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (+ one
               (*
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub i 1))
                               ((1 *)) work-%offset%)
                (abs
                 (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                ((1 *)) e-%offset%)))))
   label20))
(setf (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
        (/ (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
           (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
(f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (+
             (/ (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
             (*
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1))
                             ((1 *)) work-%offset%)
              (abs (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))))
   label30))
(setf ix
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (idamax n work 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(setf ainvnm
        (abs (f2cl-lib:fref work-%data% (ix) ((1 *)) work-%offset%)))
```

```
        (if (/= ainvnm zero)
            (setf rcond (/ (/ one ainvnm) anorm)))
        (go end_label)
       end_label
        (return (values n nil nil nil rcond nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dptcon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (double-float) (double-float)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::rcond nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

86.2 dpteqr

86.2.1 Axiom unit tests

— **dpteqr.input** —

```
)set break resume
)sys rm -f dpteqr.output
)spool dpteqr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

86.2.2 Axiom help page

— dpteqr.help —

```
====================================================================
dpteqr examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dpteqr (COMPZ, N, D, E, Z, LDZ, WORK, INFO)
           DPTEQR

Function/Subroutine Documentation
   subroutine dpteqr (character COMPZ, integer N, double precision, dimension(
       * ) D, double precision, dimension( * ) E, double precision, dimension(
       ldz, * ) Z, integer LDZ, double precision, dimension( * ) WORK, integer
       INFO)
       DPTEQR

Purpose:
   DPTEQR computes all eigenvalues and, optionally, eigenvectors of a
   symmetric positive definite tridiagonal matrix by first factoring the
   matrix using DPTTRF, and then calling DBDSQR to compute the singular
   values of the bidiagonal factor.

   This routine computes the eigenvalues of the positive definite
   tridiagonal matrix to high relative accuracy.  This means that if the
   eigenvalues range over many orders of magnitude in size, then the
   small eigenvalues and corresponding eigenvectors will be computed
   more accurately than, for example, with the standard QR method.

   The eigenvectors of a full or band symmetric positive definite matrix
   can also be found if DSYTRD, DSPTRD, or DSBTRD has been used to
   reduce this matrix to tridiagonal form. (The reduction to tridiagonal
   form, however, may preclude the possibility of obtaining high
   relative accuracy in the small eigenvalues of the original matrix, if
   these eigenvalues range over many orders of magnitude.)

Parameters:

 COMPZ
   COMPZ is CHARACTER*1
   = 'N':  Compute eigenvalues only.
   = 'V':  Compute eigenvectors of original symmetric
```

```
     matrix also.  Array Z contains the orthogonal
     matrix used to reduce the original matrix to tridiagonal form.
  = 'I':  Compute eigenvectors of tridiagonal matrix also.

N
  N is INTEGER
  The order of the matrix.  N >= 0.

D
  D is DOUBLE PRECISION array, dimension (N)
  On entry, the n diagonal elements of the tridiagonal matrix.
  On normal exit, D contains the eigenvalues, in descending order.

E
  E is DOUBLE PRECISION array, dimension (N-1)
  On entry, the (n-1) subdiagonal elements of the tridiagonal matrix.
  On exit, E has been destroyed.

Z
  Z is DOUBLE PRECISION array, dimension (LDZ, N)
  On entry, if COMPZ = 'V', the orthogonal matrix used in the
  reduction to tridiagonal form.
  On exit, if COMPZ = 'V', the orthonormal eigenvectors of the
  original symmetric matrix;
  if COMPZ = 'I', the orthonormal eigenvectors of the tridiagonal matrix.
  If INFO > 0 on exit, Z contains the eigenvectors associated
  with only the stored eigenvalues.
  If  COMPZ = 'N', then Z is not referenced.

LDZ
  LDZ is INTEGER
  The leading dimension of the array Z.  LDZ >= 1, and if
  COMPZ = 'V' or 'I', LDZ >= max(1,N).

WORK
  WORK is DOUBLE PRECISION array, dimension (4*N)

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if INFO = i, and i is:
  <= N  the Cholesky factorization of the matrix could
     not be performed because the i-th principal minor
     was not positive definite.
  > N   the SVD algorithm failed to converge;
     if INFO = N+i, i off-diagonal elements of the
     bidiagonal factor did not converge to zero.
```

———-

86.2.3 fortran code

— dpteqr.f —

```
*  =====================================================================
      SUBROUTINE DPTEQR( COMPZ, N, D, E, Z, LDZ, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          COMPZ
      INTEGER            INFO, LDZ, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), E( * ), WORK( * ), Z( LDZ, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DBDSQR, DLASET, DPTTRF, XERBLA
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   C( 1, 1 ), VT( 1, 1 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, ICOMPZ, NRU
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
```

```
      INFO = 0
*
      IF( LSAME( COMPZ, 'N' ) ) THEN
         ICOMPZ = 0
      ELSE IF( LSAME( COMPZ, 'V' ) ) THEN
         ICOMPZ = 1
      ELSE IF( LSAME( COMPZ, 'I' ) ) THEN
         ICOMPZ = 2
      ELSE
         ICOMPZ = -1
      END IF
      IF( ICOMPZ.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( ( LDZ.LT.1 ) .OR. ( ICOMPZ.GT.0 .AND. LDZ.LT.MAX( 1,
     $         N ) ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DPTEQR', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
      IF( N.EQ.1 ) THEN
         IF( ICOMPZ.GT.0 )
     $      Z( 1, 1 ) = ONE
         RETURN
      END IF
      IF( ICOMPZ.EQ.2 )
     $   CALL DLASET( 'Full', N, N, ZERO, ONE, Z, LDZ )
*
*     Call DPTTRF to factor the matrix.
*
      CALL DPTTRF( N, D, E, INFO )
      IF( INFO.NE.0 )
     $   RETURN
      DO 10 I = 1, N
         D( I ) = SQRT( D( I ) )
   10 CONTINUE
      DO 20 I = 1, N - 1
         E( I ) = E( I )*D( I )
   20 CONTINUE
*
*     Call DBDSQR to compute the singular values/vectors of the
```

```
*     bidiagonal factor.
*
      IF( ICOMPZ.GT.0 ) THEN
         NRU = N
      ELSE
         NRU = 0
      END IF
      CALL DBDSQR( 'Lower', N, 0, NRU, 0, D, E, VT, 1, Z, LDZ, C, 1,
     $             WORK, INFO )
*
*     Square the singular values.
*
      IF( INFO.EQ.0 ) THEN
         DO 30 I = 1, N
            D( I ) = D( I )*D( I )
   30    CONTINUE
      ELSE
         INFO = N + INFO
      END IF
*
      RETURN
*
*     End of DPTEQR
*
      END
```

———-

86.2.4 lisp code

— **dpteqr.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dpteqr (compz n d e z ldz work info)
    (declare (type (array double-float (*)) work z e d)
             (type (f2cl-lib:integer4) info ldz n)
             (type (string *) compz))
    (f2cl-lib:with-multi-array-data
        ((compz character compz-%data% compz-%offset%)
         (d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%)
         (z double-float z-%data% z-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((i 0) (icompz 0) (nru 0)
             (c (make-array 1 :element-type 'double-float))
```

```
     (vt (make-array 1 :element-type 'double-float)))
(declare (type (f2cl-lib:integer4) i icompz nru)
         (type (array double-float (1)) c vt))
(setf info 0)
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "N")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf icompz 0))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "V")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf icompz 1))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "I")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf icompz 2))
 (t (setf icompz -1)))
(cond ((< icompz 0) (setf info -1)) ((< n 0) (setf info -2))
      ((or (< ldz 1)
           (and (> icompz 0)
                (< ldz
                   (max (the f2cl-lib:integer4 1)
                        (the f2cl-lib:integer4 n)))))
       (setf info -6)))
(cond
 ((/= info 0) (xerbla "DPTEQR" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(cond
 ((= n 1)
  (if (> icompz 0)
      (setf (f2cl-lib:fref z-%data% (1 1) ((1 ldz) (1 *)) z-%offset%)
              one))
  (go end_label)))
(if (= icompz 2)
    (dlaset "Full" n n zero one z ldz))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (dpttrf n d e info)
  (declare (ignore var-1 var-2))
  (when var-0 (setf n var-0))
  (when var-3 (setf info var-3)))
(if (/= info 0)
```

```
            (go end_label))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                    (f2cl-lib:fsqrt
                     (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
           label10))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
          (tagbody
            (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                    (* (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                       (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
           label20))
        (cond ((> icompz 0) (setf nru n)) (t (setf nru 0)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
             var-11 var-12 var-13 var-14)
            (dbdsqr "Lower" n 0 nru 0 d e vt 1 z ldz c 1 work info)
          (declare
           (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9 var-11
            var-12 var-13))
          (setf n var-1)
          (setf nru var-3)
          (setf ldz var-10)
          (setf info var-14))
        (cond
         ((= info 0)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                      (* (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                         (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
             label30)))
         (t (setf info (f2cl-lib:int-add n info))))
        (go end_label)
       end_label
        (return (values compz n nil nil nil ldz nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dpteqr
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
```

```
  (array double-float (*))
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::compz
  fortran-to-lisp::n nil nil nil
  fortran-to-lisp::ldz nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::dbdsqr
  fortran-to-lisp::dlaset))))
```

———-

86.3 dptrfs

86.3.1 Axiom unit tests

— **dptrfs.input** —

```
)set break resume
)sys rm -f dptrfs.output
)spool dptrfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

86.3.2 Axiom help page

— **dptrfs.help** —

```
==========================================================================
dptrfs examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dptrfs (N, NRHS, D, E, DF, EF, B, LDB, X, LDX, FERR, BERR,
           WORK, INFO)
           DPTRFS

Function/Subroutine Documentation
   subroutine dptrfs (integer N, integer NRHS, double precision, dimension( *
       ) D, double precision, dimension( * ) E, double precision, dimension( *
       ) DF, double precision, dimension( * ) EF, double precision, dimension(
       ldb, * ) B, integer LDB, double precision, dimension( ldx, * ) X,
       integer LDX, double precision, dimension( * ) FERR, double precision,
       dimension( * ) BERR, double precision, dimension( * ) WORK, integer
       INFO)
       DPTRFS

Purpose:
   DPTRFS improves the computed solution to a system of linear
   equations when the coefficient matrix is symmetric positive definite
   and tridiagonal, and provides error bounds and backward error
   estimates for the solution.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 D
   D is DOUBLE PRECISION array, dimension (N)
   The n diagonal elements of the tridiagonal matrix A.

 E
   E is DOUBLE PRECISION array, dimension (N-1)
   The (n-1) subdiagonal elements of the tridiagonal matrix A.

 DF
   DF is DOUBLE PRECISION array, dimension (N)
   The n diagonal elements of the diagonal matrix D from the
   factorization computed by DPTTRF.
```

```
 EF
   EF is DOUBLE PRECISION array, dimension (N-1)
   The (n-1) subdiagonal elements of the unit bidiagonal factor
   L from the factorization computed by DPTTRF.

 B
   B is DOUBLE PRECISION array, dimension (LDB,NRHS)
   The right hand side matrix B.

 LDB
   LDB is INTEGER
   The leading dimension of the array B.  LDB >= max(1,N).

 X
   X is DOUBLE PRECISION array, dimension (LDX,NRHS)
   On entry, the solution matrix X, as computed by DPTTRS.
   On exit, the improved solution matrix X.

 LDX
   LDX is INTEGER
   The leading dimension of the array X.  LDX >= max(1,N).

 FERR
   FERR is DOUBLE PRECISION array, dimension (NRHS)
   The forward error bound for each solution vector
   X(j) (the j-th column of the solution matrix X).
   If XTRUE is the true solution corresponding to X(j), FERR(j)
   is an estimated upper bound for the magnitude of the largest
   element in (X(j) - XTRUE) divided by the magnitude of the
   largest element in X(j).

 BERR
   BERR is DOUBLE PRECISION array, dimension (NRHS)
   The componentwise relative backward error of each solution
   vector X(j) (i.e., the smallest relative change in
   any element of A or B that makes X(j) an exact solution).

 WORK
   WORK is DOUBLE PRECISION array, dimension (2*N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

   ITMAX is the maximum number of steps of iterative refinement.
```

———-

86.3.3 fortran code

— dptrfs.f —

```
*  =====================================================================
      SUBROUTINE DPTRFS( N, NRHS, D, E, DF, EF, B, LDB, X, LDX, FERR,
     $                   BERR, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   B( LDB, * ), BERR( * ), D( * ), DF( * ),
     $                   E( * ), EF( * ), FERR( * ), WORK( * ),
     $                   X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D+0 )
      DOUBLE PRECISION   THREE
      PARAMETER          ( THREE = 3.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            COUNT, I, IX, J, NZ
      DOUBLE PRECISION   BI, CX, DX, EPS, EX, LSTRES, S, SAFE1, SAFE2,
     $                   SAFMIN
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DPTTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
```

```
*     ..
*     .. External Functions ..
      INTEGER            IDAMAX
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           IDAMAX, DLAMCH
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -10
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DPTRFS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = 4
      EPS = DLAMCH( 'Epsilon' )
      SAFMIN = DLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 90 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
```

```
*
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - A * X.  Also compute
*        abs(A)*abs(x) + abs(b) for use in the backward error bound.
*
         IF( N.EQ.1 ) THEN
            BI = B( 1, J )
            DX = D( 1 )*X( 1, J )
            WORK( N+1 ) = BI - DX
            WORK( 1 ) = ABS( BI ) + ABS( DX )
         ELSE
            BI = B( 1, J )
            DX = D( 1 )*X( 1, J )
            EX = E( 1 )*X( 2, J )
            WORK( N+1 ) = BI - DX - EX
            WORK( 1 ) = ABS( BI ) + ABS( DX ) + ABS( EX )
            DO 30 I = 2, N - 1
               BI = B( I, J )
               CX = E( I-1 )*X( I-1, J )
               DX = D( I )*X( I, J )
               EX = E( I )*X( I+1, J )
               WORK( N+I ) = BI - CX - DX - EX
               WORK( I ) = ABS( BI ) + ABS( CX ) + ABS( DX ) + ABS( EX )
   30       CONTINUE
            BI = B( N, J )
            CX = E( N-1 )*X( N-1, J )
            DX = D( N )*X( N, J )
            WORK( N+N ) = BI - CX - DX
            WORK( N ) = ABS( BI ) + ABS( CX ) + ABS( DX )
         END IF
*
*        Compute componentwise relative backward error from formula
*
*        max(i) ( abs(R(i)) / ( abs(A)*abs(X) + abs(B) )(i) )
*
*        where abs(Z) is the componentwise absolute value of the matrix
*        or vector Z.  If the i-th component of the denominator is less
*        than SAFE2, then SAFE1 is added to the i-th components of the
*        numerator and denominator before dividing.
*
         S = ZERO
         DO 40 I = 1, N
            IF( WORK( I ).GT.SAFE2 ) THEN
               S = MAX( S, ABS( WORK( N+I ) ) / WORK( I ) )
            ELSE
               S = MAX( S, ( ABS( WORK( N+I ) )+SAFE1 ) /
     $             ( WORK( I )+SAFE1 ) )
            END IF
   40    CONTINUE
```

```
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
*           1) The residual BERR(J) is larger than machine epsilon, and
*           2) BERR(J) decreased by at least a factor of 2 during the
*              last iteration, and
*           3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
*
*           Update solution and try again.
*
            CALL DPTTRS( N, 1, DF, EF, WORK( N+1 ), N, INFO )
            CALL DAXPY( N, ONE, WORK( N+1 ), 1, X( 1, J ), 1 )
            LSTRES = BERR( J )
            COUNT = COUNT + 1
            GO TO 20
         END IF
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(A))*
*           ( abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
*          inv(A) is the inverse of A
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(A)*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(A)*abs(X) + abs(B) is less than SAFE2.
*
         DO 50 I = 1, N
            IF( WORK( I ).GT.SAFE2 ) THEN
               WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I )
            ELSE
               WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I ) + SAFE1
            END IF
   50    CONTINUE
         IX = IDAMAX( N, WORK, 1 )
         FERR( J ) = WORK( IX )
*
*        Estimate the norm of inv(A).
*
```

```
*        Solve M(A) * x = e, where M(A) = (m(i,j)) is given by
*
*           m(i,j) =  abs(A(i,j)), i = j,
*           m(i,j) = -abs(A(i,j)), i .ne. j,
*
*        and e = [ 1, 1, ..., 1 ]**T.  Note M(A) = M(L)*D*M(L)**T.
*
*        Solve M(L) * x = e.
*
         WORK( 1 ) = ONE
         DO 60 I = 2, N
            WORK( I ) = ONE + WORK( I-1 )*ABS( EF( I-1 ) )
   60    CONTINUE
*
*        Solve D * M(L)**T * x = b.
*
         WORK( N ) = WORK( N ) / DF( N )
         DO 70 I = N - 1, 1, -1
            WORK( I ) = WORK( I ) / DF( I ) + WORK( I+1 )*ABS( EF( I ) )
   70    CONTINUE
*
*        Compute norm(inv(A)) = max(x(i)), 1<=i<=n.
*
         IX = IDAMAX( N, WORK, 1 )
         FERR( J ) = FERR( J )*ABS( WORK( IX ) )
*
*        Normalize error.
*
         LSTRES = ZERO
         DO 80 I = 1, N
            LSTRES = MAX( LSTRES, ABS( X( I, J ) ) )
   80    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
   90 CONTINUE
*
      RETURN
*
*     End of DPTRFS
*
      END
```

86.3.4 lisp code

— dptrfs.lisp —

```
(let* ((itmax 5) (zero 0.0d0) (one 1.0d0) (two 2.0d0) (three 3.0d0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 3.0d0 3.0d0) three)
           (ignorable itmax zero one two three))
  (defun dptrfs (n nrhs d e df ef b ldb$ x ldx ferr berr work info)
    (declare (type (array double-float (*)) work berr ferr x b ef df e d)
             (type (f2cl-lib:integer4) info ldx ldb$ nrhs n))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%)
         (df double-float df-%data% df-%offset%)
         (ef double-float ef-%data% ef-%offset%)
         (b double-float b-%data% b-%offset%)
         (x double-float x-%data% x-%offset%)
         (ferr double-float ferr-%data% ferr-%offset%)
         (berr double-float berr-%data% berr-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((bi 0.0d0) (cx 0.0d0) (dx 0.0d0) (eps 0.0d0) (ex 0.0d0)
             (lstres 0.0d0) (s 0.0d0) (safe1 0.0d0) (safe2 0.0d0)
             (safmin 0.0d0) (count$ 0) (i 0) (ix 0) (j 0) (nz 0))
        (declare
         (type (double-float) bi cx dx eps ex lstres s safe1 safe2 safmin)
         (type (f2cl-lib:integer4) count$ i ix j nz))
        (setf info 0)
        (cond ((< n 0) (setf info -1)) ((< nrhs 0) (setf info -2))
              ((< ldb$
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -8))
              ((< ldx
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -10)))
        (cond
         ((/= info 0) (xerbla "DPTRFS" (f2cl-lib:int-sub info))
          (go end_label)))
        (cond
         ((or (= n 0) (= nrhs 0))
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j nrhs) nil)
            (tagbody
              (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%) zero)
              (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) zero)
             label10))
          (go end_label)))
        (setf nz 4)
        (setf eps (dlamch "Epsilon"))
        (setf safmin (dlamch "Safe minimum"))
        (setf safe1 (* nz safmin))
```

```
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (cond
     ((= n 1)
      (setf bi
              (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                             b-%offset%))
      (setf dx
              (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
                 (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                                x-%offset%)))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 1)) ((1 *))
                           work-%offset%)
              (- bi dx))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (+ (abs bi) (abs dx))))
     (t
      (setf bi
              (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                             b-%offset%))
      (setf dx
              (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
                 (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                                x-%offset%)))
      (setf ex
              (* (f2cl-lib:fref e-%data% (1) ((1 *)) e-%offset%)
                 (f2cl-lib:fref x-%data% (2 j) ((1 ldx) (1 *))
                                x-%offset%)))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 1)) ((1 *))
                           work-%offset%)
              (- bi dx ex))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (+ (abs bi) (abs dx) (abs ex)))
      (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                    ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf bi
                  (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%))
          (setf cx
                  (*
                   (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                  ((1 *)) e-%offset%)
                   (f2cl-lib:fref x-%data% ((f2cl-lib:int-sub i 1) j)
```

```
                                ((1 ldx) (1 *)) x-%offset%)))
        (setf dx
                (* (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                   (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                  x-%offset%)))
        (setf ex
                (* (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                   (f2cl-lib:fref x-%data% ((f2cl-lib:int-add i 1) j)
                                  ((1 ldx) (1 *)) x-%offset%)))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                             ((1 *)) work-%offset%)
                (- bi cx dx ex))
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (+ (abs bi) (abs cx) (abs dx) (abs ex)))
      label30))
    (setf bi
            (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                           b-%offset%))
    (setf cx
            (*
             (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub n 1)) ((1 *))
                            e-%offset%)
             (f2cl-lib:fref x-%data% ((f2cl-lib:int-sub n 1) j)
                            ((1 ldx) (1 *)) x-%offset%)))
    (setf dx
            (* (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
               (f2cl-lib:fref x-%data% (n j) ((1 ldx) (1 *))
                              x-%offset%)))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n n)) ((1 *))
                         work-%offset%)
            (- bi cx dx))
    (setf (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
            (+ (abs bi) (abs cx) (abs dx)))))
  (setf s zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
        (setf s
                (max s
                     (/
                      (abs
                       (f2cl-lib:fref work-%data%
                                      ((f2cl-lib:int-add n i))
                                      ((1 *)) work-%offset%))
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%)))))
       (t
        (setf s
```

```
                (max s
                     (/
                      (+
                       (abs
                        (f2cl-lib:fref work-%data%
                                       ((f2cl-lib:int-add n i))
                                       ((1 *)) work-%offset%))
                       safe1)
                      (+
                       (f2cl-lib:fref work-%data% (i) ((1 *))
                                      work-%offset%)
                       safe1))))))
   label40))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dpttrs n 1 df ef
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 *)) work-%offset%)
       n info)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf n var-0))
    (when var-5 (setf n var-5))
    (when var-6 (setf info var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (daxpy n one
       (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                             ((1 *)) work-%offset%)
       1
       (f2cl-lib:array-slice x-%data% double-float (1 j)
                             ((1 ldx) (1 *)) x-%offset%)
       1)
    (declare (ignore var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))
    (when var-1 (setf one var-1)))
  (setf lstres
          (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
  (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (+
               (abs
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
```

```
                                  ((1 *)) work-%offset%))
                  (* nz eps
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)))))
      (t
       (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               (+
                (abs
                 (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                                ((1 *)) work-%offset%))
                (* nz eps
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%))
                safe1))))
   label50))
(setf ix
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (idamax n work 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
        (f2cl-lib:fref work-%data% (ix) ((1 *)) work-%offset%))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) one)
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (+ one
               (*
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub i 1))
                               ((1 *)) work-%offset%)
                (abs
                 (f2cl-lib:fref ef-%data% ((f2cl-lib:int-sub i 1))
                                ((1 *)) ef-%offset%)))))
   label60))
(setf (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
        (/ (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
           (f2cl-lib:fref df-%data% (n) ((1 *)) df-%offset%)))
(f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (+
             (/
              (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (f2cl-lib:fref df-%data% (i) ((1 *)) df-%offset%))
             (*
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1))
```

```
                                        ((1 *)) work-%offset%)
                          (abs
                           (f2cl-lib:fref ef-%data% (i) ((1 *))
                                          ef-%offset%)))))
               label70))
            (setf ix
                    (multiple-value-bind (ret-val var-0 var-1 var-2)
                        (idamax n work 1)
                      (declare (ignore var-1 var-2))
                      (when var-0 (setf n var-0))
                      ret-val))
            (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                    (* (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                       (abs
                        (f2cl-lib:fref work-%data% (ix) ((1 *))
                                       work-%offset%))))
            (setf lstres zero)
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf lstres
                        (max lstres
                             (abs
                              (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                             x-%offset%))))
               label80))
            (if (/= lstres zero)
                (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                        (/
                         (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                         lstres)))
           label90))
        (go end_label)
       end_label
        (return
         (values n nil nil nil nil nil nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dptrfs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
```

```
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::n nil nil nil
  nil nil nil nil nil nil nil nil
  nil fortran-to-lisp::info)
:calls 'nil)))
```

———-

86.4 dpttrf

86.4.1 Axiom unit tests

— **dpttrf.input** —

```
)set break resume
)sys rm -f dpttrf.output
)spool dpttrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

86.4.2 Axiom help page

— **dpttrf.help** —

```
==================================================================
dpttrf examples
==================================================================

==================================================================
```

```
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dpttrf (N, D, E, INFO)
           DPTTRF

Function/Subroutine Documentation
   subroutine dpttrf (integer N, double precision, dimension( * ) D, double
       precision, dimension( * ) E, integer INFO)
       DPTTRF

Purpose:
   DPTTRF computes the L*D*L**T factorization of a real symmetric
   positive definite tridiagonal matrix A.  The factorization may also
   be regarded as having the form A = U**T*D*U.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 D
   D is DOUBLE PRECISION array, dimension (N)
   On entry, the n diagonal elements of the tridiagonal matrix
   A.  On exit, the n diagonal elements of the diagonal matrix
   D from the L*D*L**T factorization of A.

 E
   E is DOUBLE PRECISION array, dimension (N-1)
   On entry, the (n-1) subdiagonal elements of the tridiagonal
   matrix A.  On exit, the (n-1) subdiagonal elements of the
   unit bidiagonal factor L from the L*D*L**T factorization of A.
   E can also be regarded as the superdiagonal of the unit
   bidiagonal factor U from the U**T*D*U factorization of A.

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -k, the k-th argument had an illegal value
   > 0: if INFO = k, the leading minor of order k is not
      positive definite; if k < N, the factorization could not
      be completed, while if k = N, the factorization was
      completed, but D(N) <= 0.
```

86.4.3 fortran code

— dpttrf.f —

```
*  =====================================================================
      SUBROUTINE DPTTRF( N, D, E, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, I4
      DOUBLE PRECISION   EI
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MOD
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
         CALL XERBLA( 'DPTTRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
```

```
     $   RETURN
*
*     Compute the L*D*L**T (or U**T*D*U) factorization of A.
*
      I4 = MOD( N-1, 4 )
      DO 10 I = 1, I4
         IF( D( I ).LE.ZERO ) THEN
            INFO = I
            GO TO 30
         END IF
         EI = E( I )
         E( I ) = EI / D( I )
         D( I+1 ) = D( I+1 ) - E( I )*EI
   10 CONTINUE
*
      DO 20 I = I4 + 1, N - 4, 4
*
*        Drop out of the loop if d(i) <= 0: the matrix is not positive
*        definite.
*
         IF( D( I ).LE.ZERO ) THEN
            INFO = I
            GO TO 30
         END IF
*
*        Solve for e(i) and d(i+1).
*
         EI = E( I )
         E( I ) = EI / D( I )
         D( I+1 ) = D( I+1 ) - E( I )*EI
*
         IF( D( I+1 ).LE.ZERO ) THEN
            INFO = I + 1
            GO TO 30
         END IF
*
*        Solve for e(i+1) and d(i+2).
*
         EI = E( I+1 )
         E( I+1 ) = EI / D( I+1 )
         D( I+2 ) = D( I+2 ) - E( I+1 )*EI
*
         IF( D( I+2 ).LE.ZERO ) THEN
            INFO = I + 2
            GO TO 30
         END IF
*
*        Solve for e(i+2) and d(i+3).
*
         EI = E( I+2 )
```

```
         E( I+2 ) = EI / D( I+2 )
         D( I+3 ) = D( I+3 ) - E( I+2 )*EI
*
         IF( D( I+3 ).LE.ZERO ) THEN
            INFO = I + 3
            GO TO 30
         END IF
*
*        Solve for e(i+3) and d(i+4).
*
         EI = E( I+3 )
         E( I+3 ) = EI / D( I+3 )
         D( I+4 ) = D( I+4 ) - E( I+3 )*EI
   20 CONTINUE
*
*     Check d(n) for positive definiteness.
*
      IF( D( N ).LE.ZERO )
     $   INFO = N
*
   30 CONTINUE
      RETURN
*
*     End of DPTTRF
*
      END
```

———-

86.4.4 lisp code

— **dpttrf.lisp** —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun dpttrf (n d e info)
    (declare (type (array double-float (*)) e d)
             (type (f2cl-lib:integer4) info n))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%))
      (prog ((ei 0.0d0) (i 0) (i4 0))
        (declare (type (double-float) ei)
                 (type (f2cl-lib:integer4) i i4))
        (setf info 0)
        (cond
         ((< n 0) (setf info -1) (xerbla "DPTTRF" (f2cl-lib:int-sub info))
```

```
  (go end_label)))
(if (= n 0)
    (go end_label))
(setf i4 (mod (f2cl-lib:int-sub n 1) 4))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i i4) nil)
  (tagbody
    (cond
     ((<= (f2cl-lib:fref d (i) ((1 *))) zero) (setf info i)
      (go label30)))
    (setf ei (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))
    (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
            (/ ei (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
    (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         d-%offset%)
            (-
             (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                            d-%offset%)
             (* (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) ei)))
   label10))
(f2cl-lib:fdo (i (f2cl-lib:int-add i4 1) (f2cl-lib:int-add i 4))
              ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 4))) nil)
  (tagbody
    (cond
     ((<= (f2cl-lib:fref d (i) ((1 *))) zero) (setf info i)
      (go label30)))
    (setf ei (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))
    (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
            (/ ei (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
    (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         d-%offset%)
            (-
             (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                            d-%offset%)
             (* (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) ei)))
    (cond
     ((<= (f2cl-lib:fref d ((f2cl-lib:int-add i 1)) ((1 *))) zero)
      (setf info (f2cl-lib:int-add i 1)) (go label30)))
    (setf ei
            (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                           e-%offset%))
    (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         e-%offset%)
            (/ ei
               (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                              d-%offset%)))
    (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                         d-%offset%)
            (-
             (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 2)) ((1 *))
```

```
                                d-%offset%)
                         (*
                          (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                         e-%offset%)
                          ei)))
                (cond
                 ((<= (f2cl-lib:fref d ((f2cl-lib:int-add i 2)) ((1 *))) zero)
                  (setf info (f2cl-lib:int-add i 2)) (go label30)))
                (setf ei
                        (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                                       e-%offset%))
                (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                                     e-%offset%)
                        (/ ei
                           (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                                           d-%offset%)))
                (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                                     d-%offset%)
                        (-
                         (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                                        d-%offset%)
                         (*
                          (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                                         e-%offset%)
                          ei)))
                (cond
                 ((<= (f2cl-lib:fref d ((f2cl-lib:int-add i 3)) ((1 *))) zero)
                  (setf info (f2cl-lib:int-add i 3)) (go label30)))
                (setf ei
                        (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                                       e-%offset%))
                (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                                     e-%offset%)
                        (/ ei
                           (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                                           d-%offset%)))
                (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 4)) ((1 *))
                                     d-%offset%)
                        (-
                         (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 4)) ((1 *))
                                        d-%offset%)
                         (*
                          (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                                         e-%offset%)
                          ei)))
            label20))
        (if (<= (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) zero)
            (setf info n))
        label30
        (go end_label)
```

```
       end_label
        (return (values nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dpttrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

86.5 dpttrs

86.5.1 Axiom unit tests

— **dpttrs.input** —

```
)set break resume
)sys rm -f dpttrs.output
)spool dpttrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

86.5.2 Axiom help page

— **dpttrs.help** —

```
==================================================================
```

```
dpttrs examples
===================================================================

===================================================================
Man Page Details
===================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dpttrs (N, NRHS, D, E, B, LDB, INFO)
           DPTTRS

Function/Subroutine Documentation
   subroutine dpttrs (integer N, integer NRHS, double precision, dimension( *
       ) D, double precision, dimension( * ) E, double precision, dimension(
       ldb, * ) B, integer LDB, integer INFO)
       DPTTRS

Purpose:
   DPTTRS solves a tridiagonal system of the form
      A * X = B
   using the L*D*L**T factorization of A computed by DPTTRF.  D is a
   diagonal matrix specified in the vector D, L is a unit bidiagonal
   matrix whose subdiagonal is specified in the vector E, and X and B
   are N by NRHS matrices.

Parameters:

 N
   N is INTEGER
   The order of the tridiagonal matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 D
   D is DOUBLE PRECISION array, dimension (N)
   The n diagonal elements of the diagonal matrix D from the
   L*D*L**T factorization of A.

 E
   E is DOUBLE PRECISION array, dimension (N-1)
   The (n-1) subdiagonal elements of the unit bidiagonal factor
   L from the L*D*L**T factorization of A.  E can also be regarded
   as the superdiagonal of the unit bidiagonal factor U from the
   factorization A = U**T*D*U.

 B
```

```
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the right hand side vectors B for the system of linear equations.
  On exit, the solution vectors, X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -k, the k-th argument had an illegal value
```

———-

86.5.3 fortran code

— **dpttrs.f** —

```
*  =====================================================================
      SUBROUTINE DPTTRS( N, NRHS, D, E, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   B( LDB, * ), D( * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            J, JB, NB
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DPTTS2, XERBLA
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DPTTRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
*     Determine the number of right-hand sides to solve at a time.
*
      IF( NRHS.EQ.1 ) THEN
         NB = 1
      ELSE
         NB = MAX( 1, ILAENV( 1, 'DPTTRS', ' ', N, NRHS, -1, -1 ) )
      END IF
*
      IF( NB.GE.NRHS ) THEN
         CALL DPTTS2( N, NRHS, D, E, B, LDB )
      ELSE
         DO 10 J = 1, NRHS, NB
            JB = MIN( NRHS-J+1, NB )
            CALL DPTTS2( N, JB, D, E, B( 1, J ), LDB )
   10    CONTINUE
      END IF
*
      RETURN
*
*     End of DPTTRS
*
      END
```

———-

86.5.4 lisp code

— dpttrs.lisp —

```
(defun dpttrs (n nrhs d e b ldb$ info)
  (declare (type (array double-float (*)) b e d)
           (type (f2cl-lib:integer4) info ldb$ nrhs n))
  (f2cl-lib:with-multi-array-data
      ((d double-float d-%data% d-%offset%)
       (e double-float e-%data% e-%offset%)
       (b double-float b-%data% b-%offset%))
    (prog ((j 0) (jb 0) (nb 0))
      (declare (type (f2cl-lib:integer4) nb jb j))
      (setf info 0)
      (cond ((< n 0) (setf info -1)) ((< nrhs 0) (setf info -2))
            ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -6)))
      (cond
       ((/= info 0) (xerbla "DPTTRS" (f2cl-lib:int-sub info)) (go end_label)))
      (if (or (= n 0) (= nrhs 0))
          (go end_label))
      (cond ((= nrhs 1) (setf nb 1))
            (t
             (setf nb
                     (max (the f2cl-lib:integer4 1)
                          (the f2cl-lib:integer4
                               (multiple-value-bind
                                   (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                                    var-6)
                                   (ilaenv 1 "DPTTRS" " " n nrhs -1 -1)
                                 (declare
                                  (ignore var-0 var-1 var-2 var-5 var-6))
                                 (when var-3 (setf n var-3))
                                 (when var-4 (setf nrhs var-4))
                                 ret-val))))))
      (cond
       ((>= nb nrhs)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (dptts2 n nrhs d e b ldb$)
          (declare (ignore var-2 var-3 var-4))
          (when var-0 (setf n var-0))
          (when var-1 (setf nrhs var-1))
          (when var-5 (setf ldb$ var-5))))
       (t
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                      ((> j nrhs) nil)
          (tagbody
            (setf jb
                    (min
```

```
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add (f2cl-lib:int-sub nrhs j) 1))
                    (the f2cl-lib:integer4 nb)))
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                (dptts2 n jb d e
                 (f2cl-lib:array-slice b-%data% double-float (1 j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$)
              (declare (ignore var-2 var-3 var-4))
              (when var-0 (setf n var-0))
              (when var-1 (setf jb var-1))
              (when var-5 (setf ldb$ var-5)))
           label10))))
      (go end_label)
     end_label
      (return (values n nrhs nil nil nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dpttrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil nil nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

86.6 dptts2

86.6.1 Axiom unit tests

— dptts2.input —

```
)set break resume
)sys rm -f dptts2.output
)spool dptts2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

86.6.2 Axiom help page

— dptts2.help —

```
==========================================================================
dptts2 examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dptts2 (N, NRHS, D, E, B, LDB)
           DPTTS2 solves a tridiagonal system of the form AX=B using the L D
           LH factorization computed by spttrf.

Function/Subroutine Documentation
   subroutine dptts2 (integer N, integer NRHS, double precision, dimension( *
       ) D, double precision, dimension( * ) E, double precision, dimension(
       ldb, * ) B, integer LDB)
       DPTTS2 solves a tridiagonal system of the form AX=B using the L D LH
       factorization computed by spttrf.

Purpose:
   DPTTS2 solves a tridiagonal system of the form
      A * X = B
   using the L*D*L**T factorization of A computed by DPTTRF.  D is a
   diagonal matrix specified in the vector D, L is a unit bidiagonal
   matrix whose subdiagonal is specified in the vector E, and X and B
   are N by NRHS matrices.

Parameters:

 N
```

```
  N is INTEGER
  The order of the tridiagonal matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrix B.  NRHS >= 0.

D
  D is DOUBLE PRECISION array, dimension (N)
  The n diagonal elements of the diagonal matrix D from the
  L*D*L**T factorization of A.

E
  E is DOUBLE PRECISION array, dimension (N-1)
  The (n-1) subdiagonal elements of the unit bidiagonal factor
  L from the L*D*L**T factorization of A.  E can also be regarded
  as the superdiagonal of the unit bidiagonal factor U from the
  factorization A = U**T*D*U.

B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the right hand side vectors B for the system of linear equations.
  On exit, the solution vectors, X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).
```

86.6.3 fortran code

— dptts2.f —

```
*  =====================================================================
      SUBROUTINE DPTTS2( N, NRHS, D, E, B, LDB )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            LDB, N, NRHS
*     ..
*     .. Array Arguments ..
```

```
      DOUBLE PRECISION   B( LDB, * ), D( * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
*     ..
*     .. External Subroutines ..
      EXTERNAL           DSCAL
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( N.LE.1 ) THEN
         IF( N.EQ.1 )
     $      CALL DSCAL( NRHS, 1.D0 / D( 1 ), B, LDB )
         RETURN
      END IF
*
*     Solve A * X = B using the factorization A = L*D*L**T,
*     overwriting each right hand side vector with its solution.
*
      DO 30 J = 1, NRHS
*
*          Solve L * x = b.
*
         DO 10 I = 2, N
            B( I, J ) = B( I, J ) - B( I-1, J )*E( I-1 )
   10    CONTINUE
*
*          Solve D * L**T * x = b.
*
         B( N, J ) = B( N, J ) / D( N )
         DO 20 I = N - 1, 1, -1
            B( I, J ) = B( I, J ) / D( I ) - B( I+1, J )*E( I )
   20    CONTINUE
   30 CONTINUE
*
      RETURN
*
*     End of DPTTS2
*
      END
```

———-

86.6.4 lisp code

— dptts2.lisp —

```
(defun dptts2 (n nrhs d e b ldb$)
  (declare (type (array double-float (*)) b e d)
           (type (f2cl-lib:integer4) ldb$ nrhs n))
  (f2cl-lib:with-multi-array-data
      ((d double-float d-%data% d-%offset%)
       (e double-float e-%data% e-%offset%)
       (b double-float b-%data% b-%offset%))
    (prog ((i 0) (j 0))
      (declare (type (f2cl-lib:integer4) j i))
      (cond
       ((<= n 1)
        (if (= n 1)
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (dscal nrhs
                 (/ 1.0d0 (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)) b
                 ldb$)
              (declare (ignore var-1 var-2))
              (when var-0 (setf nrhs var-0))
              (when var-3 (setf ldb$ var-3))))
        (go end_label)))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nrhs) nil)
        (tagbody
          (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *)) b-%offset%)
                      (-
                       (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                      b-%offset%)
                       (*
                        (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                        (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                       ((1 *)) e-%offset%))))
             label10))
          (setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
                  (/ (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
                     (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
          (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                           (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                        ((> i 1) nil)
            (tagbody
              (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *)) b-%offset%)
                      (-
```

```
                        (/
                         (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                        b-%offset%)
                         (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
                        (*
                         (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                        ((1 ldb$) (1 *)) b-%offset%)
                         (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))))
             label20))
         label30))
      (go end_label)
     end_label
      (return (values nil nrhs nil nil nil ldb$)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dptts2
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (array double-float (*))
                                             (array double-float (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(nil fortran-to-lisp::nrhs nil nil
                                             nil fortran-to-lisp::ldb$)
                                           :calls 'nil)))
```

———-

Chapter 87

LAPACK - Positive Definite Tridiagonal Matrix, Comp. Routines, Real

87.1 sptcon

87.1.1 Axiom unit tests

— **sptcon.input** —

```
)set break resume
)sys rm -f sptcon.output
)spool sptcon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

87.1.2 Axiom help page

— **sptcon.help** —

```
==========================================================================
```

```
sptcon examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sptcon (N, D, E, ANORM, RCOND, WORK, INFO)
           SPTCON

Function/Subroutine Documentation
   subroutine sptcon (integer N, real, dimension( * ) D, real, dimension( * )
       E, real ANORM, real RCOND, real, dimension( * ) WORK, integer INFO)
       SPTCON

Purpose:
   SPTCON computes the reciprocal of the condition number (in the
   1-norm) of a real symmetric positive definite tridiagonal matrix
   using the factorization A = L*D*L**T or A = U**T*D*U computed by
   SPTTRF.

   Norm(inv(A)) is computed by a direct method, and the reciprocal of
   the condition number is computed as
      RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 D
   D is REAL array, dimension (N)
   The n diagonal elements of the diagonal matrix D from the
   factorization of A, as computed by SPTTRF.

 E
   E is REAL array, dimension (N-1)
   The (n-1) off-diagonal elements of the unit bidiagonal factor
   U or L from the factorization of A,  as computed by SPTTRF.

 ANORM
   ANORM is REAL
   The 1-norm of the original matrix A.

 RCOND
   RCOND is REAL
   The reciprocal of the condition number of the matrix A,
```

```
   computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is the
   1-norm of inv(A) computed in this routine.

 WORK
   WORK is REAL array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

87.1.3 fortran code

— **sptcon.f** —

```
*  =====================================================================
      SUBROUTINE SPTCON( N, D, E, ANORM, RCOND, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, N
      REAL               ANORM, RCOND
*     ..
*     .. Array Arguments ..
      REAL               D( * ), E( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IX
      REAL               AINVNM
*     ..
*     .. External Functions ..
      INTEGER            ISAMAX
      EXTERNAL           ISAMAX
*     ..
```

```
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SPTCON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
*     Check that D(1:N) is positive.
*
      DO 10 I = 1, N
         IF( D( I ).LE.ZERO )
     $      RETURN
   10 CONTINUE
*
*     Solve M(A) * x = e, where M(A) = (m(i,j)) is given by
*
*        m(i,j) =  abs(A(i,j)), i = j,
*        m(i,j) = -abs(A(i,j)), i .ne. j,
*
*     and e = [ 1, 1, ..., 1 ]**T.  Note M(A) = M(L)*D*M(L)**T.
*
*     Solve M(L) * x = e.
*
      WORK( 1 ) = ONE
      DO 20 I = 2, N
         WORK( I ) = ONE + WORK( I-1 )*ABS( E( I-1 ) )
```

```
   20 CONTINUE
*
*     Solve D * M(L)**T * x = b.
*
      WORK( N ) = WORK( N ) / D( N )
      DO 30 I = N - 1, 1, -1
         WORK( I ) = WORK( I ) / D( I ) + WORK( I+1 )*ABS( E( I ) )
   30 CONTINUE
*
*     Compute AINVNM = max(x(i)), 1<=i<=n.
*
      IX = ISAMAX( N, WORK, 1 )
      AINVNM = ABS( WORK( IX ) )
*
*     Compute the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
      RETURN
*
*     End of SPTCON
*
      END
```

87.1.4 lisp code

— sptcon.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun sptcon (n d e anorm rcond work info)
    (declare (type (single-float) rcond anorm)
             (type (array single-float (*)) work e d)
             (type (f2cl-lib:integer4) info n))
    (f2cl-lib:with-multi-array-data
        ((d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((ainvnm 0.0) (i 0) (ix 0))
        (declare (type (single-float) ainvnm)
                 (type (f2cl-lib:integer4) i ix))
        (setf info 0)
        (cond ((< n 0) (setf info -1)) ((< anorm zero) (setf info -4)))
```

```
(cond
 ((/= info 0) (xerbla "SPTCON" (f2cl-lib:int-sub info))
  (go end_label)))
(setf rcond zero)
(cond ((= n 0) (setf rcond one) (go end_label))
      ((= anorm zero) (go end_label)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (if (<= (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) zero)
        (go end_label))
   label10))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) one)
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (+ one
               (*
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub i 1))
                               ((1 *)) work-%offset%)
                (abs
                 (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                ((1 *)) e-%offset%)))))
   label20))
(setf (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
        (/ (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
           (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
(f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (+
             (/ (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
             (*
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1))
                             ((1 *)) work-%offset%)
              (abs (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))))
   label30))
(setf ix
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (isamax n work 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(setf ainvnm
        (abs (f2cl-lib:fref work-%data% (ix) ((1 *)) work-%offset%)))
(if (/= ainvnm zero)
```

```
            (setf rcond (/ (/ one ainvnm) anorm)))
        (go end_label)
       end_label
        (return (values n nil nil nil rcond nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sptcon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::rcond nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

87.2 spteqr

87.2.1 Axiom unit tests

— **spteqr.input** —

```
)set break resume
)sys rm -f spteqr.output
)spool spteqr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

87.2.2 Axiom help page

— spteqr.help —

```
==================================================================
spteqr examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine spteqr (COMPZ, N, D, E, Z, LDZ, WORK, INFO)
           SPTEQR

Function/Subroutine Documentation
   subroutine spteqr (character COMPZ, integer N, real, dimension( * ) D,
       real, dimension( * ) E, real, dimension( ldz, * ) Z, integer LDZ, real,
       dimension( * ) WORK, integer INFO)
       SPTEQR

Purpose:
   SPTEQR computes all eigenvalues and, optionally, eigenvectors of a
   symmetric positive definite tridiagonal matrix by first factoring the
   matrix using SPTTRF, and then calling SBDSQR to compute the singular
   values of the bidiagonal factor.

   This routine computes the eigenvalues of the positive definite
   tridiagonal matrix to high relative accuracy.  This means that if the
   eigenvalues range over many orders of magnitude in size, then the
   small eigenvalues and corresponding eigenvectors will be computed
   more accurately than, for example, with the standard QR method.

   The eigenvectors of a full or band symmetric positive definite matrix
   can also be found if SSYTRD, SSPTRD, or SSBTRD has been used to
   reduce this matrix to tridiagonal form. (The reduction to tridiagonal
   form, however, may preclude the possibility of obtaining high
   relative accuracy in the small eigenvalues of the original matrix, if
   these eigenvalues range over many orders of magnitude.)

Parameters:

 COMPZ
   COMPZ is CHARACTER*1
   = 'N':  Compute eigenvalues only.
   = 'V':  Compute eigenvectors of original symmetric
      matrix also.  Array Z contains the orthogonal
```

```
     matrix used to reduce the original matrix to tridiagonal form.
  = 'I':  Compute eigenvectors of tridiagonal matrix also.

N
  N is INTEGER
  The order of the matrix.  N >= 0.

D
  D is REAL array, dimension (N)
  On entry, the n diagonal elements of the tridiagonal matrix.
  On normal exit, D contains the eigenvalues, in descending order.

E
  E is REAL array, dimension (N-1)
  On entry, the (n-1) subdiagonal elements of the tridiagonal matrix.
  On exit, E has been destroyed.

Z
  Z is REAL array, dimension (LDZ, N)
  On entry, if COMPZ = 'V', the orthogonal matrix used in the
  reduction to tridiagonal form.
  On exit, if COMPZ = 'V', the orthonormal eigenvectors of the
  original symmetric matrix;
  if COMPZ = 'I', the orthonormal eigenvectors of the tridiagonal matrix.
  If INFO > 0 on exit, Z contains the eigenvectors associated
  with only the stored eigenvalues.
  If  COMPZ = 'N', then Z is not referenced.

LDZ
  LDZ is INTEGER
  The leading dimension of the array Z.  LDZ >= 1, and if
  COMPZ = 'V' or 'I', LDZ >= max(1,N).

WORK
  WORK is REAL array, dimension (4*N)

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if INFO = i, and i is:
  <= N  the Cholesky factorization of the matrix could
     not be performed because the i-th principal minor
     was not positive definite.
  > N   the SVD algorithm failed to converge;
     if INFO = N+i, i off-diagonal elements of the
     bidiagonal factor did not converge to zero.
```

———-

87.2.3 fortran code

— spteqr.f —

```
*  =====================================================================
      SUBROUTINE SPTEQR( COMPZ, N, D, E, Z, LDZ, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          COMPZ
      INTEGER            INFO, LDZ, N
*     ..
*     .. Array Arguments ..
      REAL               D( * ), E( * ), WORK( * ), Z( LDZ, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           SBDSQR, SLASET, SPTTRF, XERBLA
*     ..
*     .. Local Arrays ..
      REAL               C( 1, 1 ), VT( 1, 1 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, ICOMPZ, NRU
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( LSAME( COMPZ, 'N' ) ) THEN
```

```
         ICOMPZ = 0
      ELSE IF( LSAME( COMPZ, 'V' ) ) THEN
         ICOMPZ = 1
      ELSE IF( LSAME( COMPZ, 'I' ) ) THEN
         ICOMPZ = 2
      ELSE
         ICOMPZ = -1
      END IF
      IF( ICOMPZ.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( ( LDZ.LT.1 ) .OR. ( ICOMPZ.GT.0 .AND. LDZ.LT.MAX( 1,
     $         N ) ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SPTEQR', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
      IF( N.EQ.1 ) THEN
         IF( ICOMPZ.GT.0 )
     $      Z( 1, 1 ) = ONE
         RETURN
      END IF
      IF( ICOMPZ.EQ.2 )
     $   CALL SLASET( 'Full', N, N, ZERO, ONE, Z, LDZ )
*
*     Call SPTTRF to factor the matrix.
*
      CALL SPTTRF( N, D, E, INFO )
      IF( INFO.NE.0 )
     $   RETURN
      DO 10 I = 1, N
         D( I ) = SQRT( D( I ) )
   10 CONTINUE
      DO 20 I = 1, N - 1
         E( I ) = E( I )*D( I )
   20 CONTINUE
*
*     Call SBDSQR to compute the singular values/vectors of the
*     bidiagonal factor.
*
      IF( ICOMPZ.GT.0 ) THEN
```

```
         NRU = N
      ELSE
         NRU = 0
      END IF
      CALL SBDSQR( 'Lower', N, 0, NRU, 0, D, E, VT, 1, Z, LDZ, C, 1,
     $             WORK, INFO )
*
*     Square the singular values.
*
      IF( INFO.EQ.0 ) THEN
         DO 30 I = 1, N
            D( I ) = D( I )*D( I )
   30    CONTINUE
      ELSE
         INFO = N + INFO
      END IF
*
      RETURN
*
*     End of SPTEQR
*
      END
```

———-

87.2.4 lisp code

— spteqr.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun spteqr (compz n d e z ldz work info)
    (declare (type (array single-float (*)) work z e d)
             (type (f2cl-lib:integer4) info ldz n)
             (type (string *) compz))
    (f2cl-lib:with-multi-array-data
        ((compz character compz-%data% compz-%offset%)
         (d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%)
         (z single-float z-%data% z-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((i 0) (icompz 0) (nru 0)
             (c (make-array 1 :element-type 'single-float))
             (vt (make-array 1 :element-type 'single-float)))
        (declare (type (f2cl-lib:integer4) i icompz nru)
                 (type (array single-float (1)) c vt))
```

```
(setf info 0)
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "N")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf icompz 0))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "V")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf icompz 1))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "I")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf icompz 2))
 (t (setf icompz -1)))
(cond ((< icompz 0) (setf info -1)) ((< n 0) (setf info -2))
      ((or (< ldz 1)
           (and (> icompz 0)
                (< ldz
                   (max (the f2cl-lib:integer4 1)
                        (the f2cl-lib:integer4 n)))))
       (setf info -6)))
(cond
 ((/= info 0) (xerbla "SPTEQR" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(cond
 ((= n 1)
  (if (> icompz 0)
      (setf (f2cl-lib:fref z-%data% (1 1) ((1 ldz) (1 *)) z-%offset%)
            one))
  (go end_label)))
(if (= icompz 2)
    (slaset "Full" n n zero one z ldz))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (spttrf n d e info)
  (declare (ignore var-1 var-2))
  (when var-0 (setf n var-0))
  (when var-3 (setf info var-3)))
(if (/= info 0)
    (go end_label))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
```

```
          (tagbody
            (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                    (f2cl-lib:fsqrt
                     (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
           label10))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
          (tagbody
            (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                    (* (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                       (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
           label20))
        (cond ((> icompz 0) (setf nru n)) (t (setf nru 0)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
             var-11 var-12 var-13 var-14)
            (sbdsqr "Lower" n 0 nru 0 d e vt 1 z ldz c 1 work info)
          (declare
           (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9 var-11
            var-12 var-13))
          (setf n var-1)
          (setf nru var-3)
          (setf ldz var-10)
          (setf info var-14))
        (cond
         ((= info 0)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                      (* (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                         (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
             label30)))
         (t (setf info (f2cl-lib:int-add n info))))
        (go end_label)
       end_label
        (return (values compz n nil nil nil ldz nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::spteqr
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
```

```
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::compz
  fortran-to-lisp::n nil nil nil
  fortran-to-lisp::ldz nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::sbdsqr
  fortran-to-lisp::slaset))))
```

———-

87.3 sptrfs

87.3.1 Axiom unit tests

— **sptrfs.input** —

```
)set break resume
)sys rm -f sptrfs.output
)spool sptrfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

87.3.2 Axiom help page

— **sptrfs.help** —

```
=====================================================================
sptrfs examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine sptrfs (N, NRHS, D, E, DF, EF, B, LDB, X, LDX, FERR, BERR,
           WORK, INFO)
           SPTRFS

Function/Subroutine Documentation
   subroutine sptrfs (integer N, integer NRHS, real, dimension( * ) D, real,
       dimension( * ) E, real, dimension( * ) DF, real, dimension( * ) EF,
       real, dimension( ldb, * ) B, integer LDB, real, dimension( ldx, * ) X,
       integer LDX, real, dimension( * ) FERR, real, dimension( * ) BERR,
       real, dimension( * ) WORK, integer INFO)
       SPTRFS

Purpose:
   SPTRFS improves the computed solution to a system of linear
   equations when the coefficient matrix is symmetric positive definite
   and tridiagonal, and provides error bounds and backward error
   estimates for the solution.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 D
   D is REAL array, dimension (N)
   The n diagonal elements of the tridiagonal matrix A.

 E
   E is REAL array, dimension (N-1)
   The (n-1) subdiagonal elements of the tridiagonal matrix A.

 DF
   DF is REAL array, dimension (N)
   The n diagonal elements of the diagonal matrix D from the
   factorization computed by SPTTRF.

 EF
   EF is REAL array, dimension (N-1)
   The (n-1) subdiagonal elements of the unit bidiagonal factor
   L from the factorization computed by SPTTRF.
```

```
 B
   B is REAL array, dimension (LDB,NRHS)
   The right hand side matrix B.

 LDB
   LDB is INTEGER
   The leading dimension of the array B.  LDB >= max(1,N).

 X
   X is REAL array, dimension (LDX,NRHS)
   On entry, the solution matrix X, as computed by SPTTRS.
   On exit, the improved solution matrix X.

 LDX
   LDX is INTEGER
   The leading dimension of the array X.  LDX >= max(1,N).

 FERR
   FERR is REAL array, dimension (NRHS)
   The forward error bound for each solution vector
   X(j) (the j-th column of the solution matrix X).
   If XTRUE is the true solution corresponding to X(j), FERR(j)
   is an estimated upper bound for the magnitude of the largest
   element in (X(j) - XTRUE) divided by the magnitude of the
   largest element in X(j).

 BERR
   BERR is REAL array, dimension (NRHS)
   The componentwise relative backward error of each solution
   vector X(j) (i.e., the smallest relative change in
   any element of A or B that makes X(j) an exact solution).

 WORK
   WORK is REAL array, dimension (2*N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

   ITMAX is the maximum number of steps of iterative refinement.
```

———-

87.3.3 fortran code

— sptrfs.f —

```
*  =====================================================================
      SUBROUTINE SPTRFS( N, NRHS, D, E, DF, EF, B, LDB, X, LDX, FERR,
     $                   BERR, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      REAL               B( LDB, * ), BERR( * ), D( * ), DF( * ),
     $                   E( * ), EF( * ), FERR( * ), WORK( * ),
     $                   X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
      REAL               TWO
      PARAMETER          ( TWO = 2.0E+0 )
      REAL               THREE
      PARAMETER          ( THREE = 3.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            COUNT, I, IX, J, NZ
      REAL               BI, CX, DX, EPS, EX, LSTRES, S, SAFE1, SAFE2,
     $                   SAFMIN
*     ..
*     .. External Subroutines ..
      EXTERNAL           SAXPY, SPTTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. External Functions ..
      INTEGER            ISAMAX
```

```
      REAL               SLAMCH
      EXTERNAL           ISAMAX, SLAMCH
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -10
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SPTRFS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = 4
      EPS = SLAMCH( 'Epsilon' )
      SAFMIN = SLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 90 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
*
```

```
*        Compute residual R = B - A * X.  Also compute
*        abs(A)*abs(x) + abs(b) for use in the backward error bound.
*
         IF( N.EQ.1 ) THEN
            BI = B( 1, J )
            DX = D( 1 )*X( 1, J )
            WORK( N+1 ) = BI - DX
            WORK( 1 ) = ABS( BI ) + ABS( DX )
         ELSE
            BI = B( 1, J )
            DX = D( 1 )*X( 1, J )
            EX = E( 1 )*X( 2, J )
            WORK( N+1 ) = BI - DX - EX
            WORK( 1 ) = ABS( BI ) + ABS( DX ) + ABS( EX )
            DO 30 I = 2, N - 1
               BI = B( I, J )
               CX = E( I-1 )*X( I-1, J )
               DX = D( I )*X( I, J )
               EX = E( I )*X( I+1, J )
               WORK( N+I ) = BI - CX - DX - EX
               WORK( I ) = ABS( BI ) + ABS( CX ) + ABS( DX ) + ABS( EX )
   30       CONTINUE
            BI = B( N, J )
            CX = E( N-1 )*X( N-1, J )
            DX = D( N )*X( N, J )
            WORK( N+N ) = BI - CX - DX
            WORK( N ) = ABS( BI ) + ABS( CX ) + ABS( DX )
         END IF
*
*        Compute componentwise relative backward error from formula
*
*        max(i) ( abs(R(i)) / ( abs(A)*abs(X) + abs(B) )(i) )
*
*        where abs(Z) is the componentwise absolute value of the matrix
*        or vector Z.  If the i-th component of the denominator is less
*        than SAFE2, then SAFE1 is added to the i-th components of the
*        numerator and denominator before dividing.
*
         S = ZERO
         DO 40 I = 1, N
            IF( WORK( I ).GT.SAFE2 ) THEN
               S = MAX( S, ABS( WORK( N+I ) ) / WORK( I ) )
            ELSE
               S = MAX( S, ( ABS( WORK( N+I ) )+SAFE1 ) /
     $             ( WORK( I )+SAFE1 ) )
            END IF
   40    CONTINUE
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
```

```
*          1) The residual BERR(J) is larger than machine epsilon, and
*          2) BERR(J) decreased by at least a factor of 2 during the
*             last iteration, and
*          3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
*
*          Update solution and try again.
*
           CALL SPTTRS( N, 1, DF, EF, WORK( N+1 ), N, INFO )
           CALL SAXPY( N, ONE, WORK( N+1 ), 1, X( 1, J ), 1 )
           LSTRES = BERR( J )
           COUNT = COUNT + 1
           GO TO 20
         END IF
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(A))*
*           ( abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
*          inv(A) is the inverse of A
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(A)*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(A)*abs(X) + abs(B) is less than SAFE2.
*
         DO 50 I = 1, N
           IF( WORK( I ).GT.SAFE2 ) THEN
              WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I )
           ELSE
              WORK( I ) = ABS( WORK( N+I ) ) + NZ*EPS*WORK( I ) + SAFE1
           END IF
   50    CONTINUE
         IX = ISAMAX( N, WORK, 1 )
         FERR( J ) = WORK( IX )
*
*        Estimate the norm of inv(A).
*
*        Solve M(A) * x = e, where M(A) = (m(i,j)) is given by
*
*          m(i,j) =  abs(A(i,j)), i = j,
```

```
*          m(i,j) = -abs(A(i,j)), i .ne. j,
*
*       and e = [ 1, 1, ..., 1 ]**T.  Note M(A) = M(L)*D*M(L)**T.
*
*       Solve M(L) * x = e.
*
        WORK( 1 ) = ONE
        DO 60 I = 2, N
           WORK( I ) = ONE + WORK( I-1 )*ABS( EF( I-1 ) )
   60   CONTINUE
*
*       Solve D * M(L)**T * x = b.
*
        WORK( N ) = WORK( N ) / DF( N )
        DO 70 I = N - 1, 1, -1
           WORK( I ) = WORK( I ) / DF( I ) + WORK( I+1 )*ABS( EF( I ) )
   70   CONTINUE
*
*       Compute norm(inv(A)) = max(x(i)), 1<=i<=n.
*
        IX = ISAMAX( N, WORK, 1 )
        FERR( J ) = FERR( J )*ABS( WORK( IX ) )
*
*       Normalize error.
*
        LSTRES = ZERO
        DO 80 I = 1, N
           LSTRES = MAX( LSTRES, ABS( X( I, J ) ) )
   80   CONTINUE
        IF( LSTRES.NE.ZERO )
     $     FERR( J ) = FERR( J ) / LSTRES
*
   90 CONTINUE
*
      RETURN
*
*     End of SPTRFS
*
      END
```

———-

87.3.4 lisp code

— sptrfs.lisp —

```
(let* ((itmax 5) (zero 0.0) (one 1.0) (two 2.0) (three 3.0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
```

```
          (type (single-float 0.0 0.0) zero)
          (type (single-float 1.0 1.0) one)
          (type (single-float 2.0 2.0) two)
          (type (single-float 3.0 3.0) three)
          (ignorable itmax zero one two three))
(defun sptrfs (n nrhs d e df ef b ldb$ x ldx ferr berr work info)
  (declare (type (array single-float (*)) work berr ferr x b ef df e d)
           (type (f2cl-lib:integer4) info ldx ldb$ nrhs n))
  (f2cl-lib:with-multi-array-data
      ((d single-float d-%data% d-%offset%)
       (e single-float e-%data% e-%offset%)
       (df single-float df-%data% df-%offset%)
       (ef single-float ef-%data% ef-%offset%)
       (b single-float b-%data% b-%offset%)
       (x single-float x-%data% x-%offset%)
       (ferr single-float ferr-%data% ferr-%offset%)
       (berr single-float berr-%data% berr-%offset%)
       (work single-float work-%data% work-%offset%))
    (prog ((bi 0.0) (cx 0.0) (dx 0.0) (eps 0.0) (ex 0.0) (lstres 0.0) (s 0.0)
           (safe1 0.0) (safe2 0.0) (safmin 0.0) (count$ 0) (i 0) (ix 0) (j 0)
           (nz 0))
      (declare
       (type (single-float) bi cx dx eps ex lstres s safe1 safe2 safmin)
       (type (f2cl-lib:integer4) count$ i ix j nz))
      (setf info 0)
      (cond ((< n 0) (setf info -1)) ((< nrhs 0) (setf info -2))
            ((< ldb$
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -8))
            ((< ldx
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -10)))
      (cond
       ((/= info 0) (xerbla "SPTRFS" (f2cl-lib:int-sub info))
        (go end_label)))
      (cond
       ((or (= n 0) (= nrhs 0))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j nrhs) nil)
          (tagbody
            (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%) zero)
            (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) zero)
           label10))
        (go end_label)))
      (setf nz 4)
      (setf eps (slamch "Epsilon"))
      (setf safmin (slamch "Safe minimum"))
      (setf safe1 (* nz safmin))
      (setf safe2 (/ safe1 eps))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
            ((> j nrhs) nil)
(tagbody
  (setf count$ 1)
  (setf lstres three)
 label20
  (cond
   ((= n 1)
    (setf bi
            (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                           b-%offset%))
    (setf dx
            (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
               (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                              x-%offset%)))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 1)) ((1 *))
                         work-%offset%)
            (- bi dx))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (+ (abs bi) (abs dx))))
   (t
    (setf bi
            (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                           b-%offset%))
    (setf dx
            (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
               (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                              x-%offset%)))
    (setf ex
            (* (f2cl-lib:fref e-%data% (1) ((1 *)) e-%offset%)
               (f2cl-lib:fref x-%data% (2 j) ((1 ldx) (1 *))
                              x-%offset%)))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 1)) ((1 *))
                         work-%offset%)
            (- bi dx ex))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (+ (abs bi) (abs dx) (abs ex)))
    (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf bi
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%))
        (setf cx
                (*
                 (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                ((1 *)) e-%offset%)
                 (f2cl-lib:fref x-%data% ((f2cl-lib:int-sub i 1) j)
                                ((1 ldx) (1 *)) x-%offset%)))
        (setf dx
```

```
                (* (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                   (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                  x-%offset%)))
        (setf ex
                (* (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                   (f2cl-lib:fref x-%data% ((f2cl-lib:int-add i 1) j)
                                  ((1 ldx) (1 *)) x-%offset%)))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                             ((1 *)) work-%offset%)
                (- bi cx dx ex))
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (+ (abs bi) (abs cx) (abs dx) (abs ex)))
       label30))
    (setf bi
            (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                           b-%offset%))
    (setf cx
            (*
             (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub n 1)) ((1 *))
                            e-%offset%)
             (f2cl-lib:fref x-%data% ((f2cl-lib:int-sub n 1) j)
                            ((1 ldx) (1 *)) x-%offset%)))
    (setf dx
            (* (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
               (f2cl-lib:fref x-%data% (n j) ((1 ldx) (1 *))
                              x-%offset%)))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n n)) ((1 *))
                         work-%offset%)
            (- bi cx dx))
    (setf (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
            (+ (abs bi) (abs cx) (abs dx)))))
  (setf s zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
        (setf s
                (max s
                     (/
                      (abs
                       (f2cl-lib:fref work-%data%
                                      ((f2cl-lib:int-add n i))
                                      ((1 *)) work-%offset%))
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%)))))
       (t
        (setf s
                (max s
                     (/
```

```
                    (+
                     (abs
                      (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-add n i))
                                     ((1 *)) work-%offset%))
                     safe1)
                    (+
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)
                     safe1))))))
   label40))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (spttrs n 1 df ef
       (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                             ((1 *)) work-%offset%)
       n info)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf n var-0))
    (when var-5 (setf n var-5))
    (when var-6 (setf info var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (saxpy n one
       (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                             ((1 *)) work-%offset%)
       1
       (f2cl-lib:array-slice x-%data% single-float (1 j)
                             ((1 ldx) (1 *)) x-%offset%)
       1)
    (declare (ignore var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0))
    (when var-1 (setf one var-1)))
  (setf lstres
          (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
  (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref work (i) ((1 *))) safe2)
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (+
               (abs
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%))
               (* nz eps
```

```
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)))))
     (t
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (+
               (abs
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%))
               (* nz eps
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%))
               safe1))))
   label50))
(setf ix
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (isamax n work 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
        (f2cl-lib:fref work-%data% (ix) ((1 *)) work-%offset%))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) one)
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (+ one
               (*
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub i 1))
                               ((1 *)) work-%offset%)
                (abs
                 (f2cl-lib:fref ef-%data% ((f2cl-lib:int-sub i 1))
                                ((1 *)) ef-%offset%)))))
   label60))
(setf (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
        (/ (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
           (f2cl-lib:fref df-%data% (n) ((1 *)) df-%offset%)))
(f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (+
             (/
              (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (f2cl-lib:fref df-%data% (i) ((1 *)) df-%offset%))
             (*
              (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1))
                             ((1 *)) work-%offset%)
              (abs
```

```
                        (f2cl-lib:fref ef-%data% (i) ((1 *))
                                       ef-%offset%)))))
            label70))
          (setf ix
                  (multiple-value-bind (ret-val var-0 var-1 var-2)
                      (isamax n work 1)
                    (declare (ignore var-1 var-2))
                    (when var-0 (setf n var-0))
                    ret-val))
          (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                  (* (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                     (abs
                      (f2cl-lib:fref work-%data% (ix) ((1 *))
                                     work-%offset%))))
          (setf lstres zero)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf lstres
                      (max lstres
                           (abs
                            (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                           x-%offset%))))
            label80))
          (if (/= lstres zero)
              (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                      (/
                       (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                       lstres)))
        label90))
      (go end_label)
     end_label
      (return
       (values n nil nil nil nil nil nil nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sptrfs
                 fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (array single-float (*))
                                            (array single-float (*))
                                            (array single-float (*))
                                            (array single-float (*))
                                            (array single-float (*))
                                            (fortran-to-lisp::integer4)
                                            (array single-float (*))
```

```
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::n nil nil nil
  nil nil nil nil nil nil nil nil
  nil fortran-to-lisp::info)
:calls 'nil)))
```

———-

87.4 spttrf

87.4.1 Axiom unit tests

— **spttrf.input** —

```
)set break resume
)sys rm -f spttrf.output
)spool spttrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

87.4.2 Axiom help page

— **spttrf.help** —

```
==========================================================================
spttrf examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine spttrf (N, D, E, INFO)
           SPTTRF

Function/Subroutine Documentation
   subroutine spttrf (integer N, real, dimension( * ) D, real, dimension( * )
       E, integer INFO)
       SPTTRF

Purpose:
   SPTTRF computes the L*D*L**T factorization of a real symmetric
   positive definite tridiagonal matrix A.  The factorization may also
   be regarded as having the form A = U**T*D*U.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 D
   D is REAL array, dimension (N)
   On entry, the n diagonal elements of the tridiagonal matrix
   A.  On exit, the n diagonal elements of the diagonal matrix
   D from the L*D*L**T factorization of A.

 E
   E is REAL array, dimension (N-1)
   On entry, the (n-1) subdiagonal elements of the tridiagonal
   matrix A.  On exit, the (n-1) subdiagonal elements of the
   unit bidiagonal factor L from the L*D*L**T factorization of A.
   E can also be regarded as the superdiagonal of the unit
   bidiagonal factor U from the U**T*D*U factorization of A.

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -k, the k-th argument had an illegal value
   > 0: if INFO = k, the leading minor of order k is not
      positive definite; if k < N, the factorization could not
      be completed, while if k = N, the factorization was
      completed, but D(N) <= 0.
```

87.4.3 fortran code

— spttrf.f —

```
*  =====================================================================
      SUBROUTINE SPTTRF( N, D, E, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, N
*     ..
*     .. Array Arguments ..
      REAL               D( * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, I4
      REAL               EI
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MOD
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
         CALL XERBLA( 'SPTTRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
```

```
     $   RETURN
*
*     Compute the L*D*L**T (or U**T*D*U) factorization of A.
*
      I4 = MOD( N-1, 4 )
      DO 10 I = 1, I4
         IF( D( I ).LE.ZERO ) THEN
            INFO = I
            GO TO 30
         END IF
         EI = E( I )
         E( I ) = EI / D( I )
         D( I+1 ) = D( I+1 ) - E( I )*EI
   10 CONTINUE
*
      DO 20 I = I4 + 1, N - 4, 4
*
*        Drop out of the loop if d(i) <= 0: the matrix is not positive
*        definite.
*
         IF( D( I ).LE.ZERO ) THEN
            INFO = I
            GO TO 30
         END IF
*
*        Solve for e(i) and d(i+1).
*
         EI = E( I )
         E( I ) = EI / D( I )
         D( I+1 ) = D( I+1 ) - E( I )*EI
*
         IF( D( I+1 ).LE.ZERO ) THEN
            INFO = I + 1
            GO TO 30
         END IF
*
*        Solve for e(i+1) and d(i+2).
*
         EI = E( I+1 )
         E( I+1 ) = EI / D( I+1 )
         D( I+2 ) = D( I+2 ) - E( I+1 )*EI
*
         IF( D( I+2 ).LE.ZERO ) THEN
            INFO = I + 2
            GO TO 30
         END IF
*
*        Solve for e(i+2) and d(i+3).
*
         EI = E( I+2 )
```

```
         E( I+2 ) = EI / D( I+2 )
         D( I+3 ) = D( I+3 ) - E( I+2 )*EI
*
         IF( D( I+3 ).LE.ZERO ) THEN
            INFO = I + 3
            GO TO 30
         END IF
*
*        Solve for e(i+3) and d(i+4).
*
         EI = E( I+3 )
         E( I+3 ) = EI / D( I+3 )
         D( I+4 ) = D( I+4 ) - E( I+3 )*EI
   20 CONTINUE
*
*     Check d(n) for positive definiteness.
*
      IF( D( N ).LE.ZERO )
     $   INFO = N
*
   30 CONTINUE
      RETURN
*
*     End of SPTTRF
*
      END
```

———-

87.4.4 lisp code

— **spttrf.lisp** —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun spttrf (n d e info)
    (declare (type (array single-float (*)) e d)
             (type (f2cl-lib:integer4) info n))
    (f2cl-lib:with-multi-array-data
        ((d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%))
      (prog ((ei 0.0) (i 0) (i4 0))
        (declare (type (single-float) ei)
                 (type (f2cl-lib:integer4) i i4))
        (setf info 0)
        (cond
         ((< n 0) (setf info -1) (xerbla "SPTTRF" (f2cl-lib:int-sub info))
```

```
  (go end_label)))
(if (= n 0)
    (go end_label))
(setf i4 (mod (f2cl-lib:int-sub n 1) 4))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i i4) nil)
  (tagbody
    (cond
     ((<= (f2cl-lib:fref d (i) ((1 *))) zero) (setf info i)
      (go label30)))
    (setf ei (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))
    (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
            (/ ei (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
    (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         d-%offset%)
            (-
             (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                            d-%offset%)
             (* (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) ei)))
   label10))
(f2cl-lib:fdo (i (f2cl-lib:int-add i4 1) (f2cl-lib:int-add i 4))
              ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 4))) nil)
  (tagbody
    (cond
     ((<= (f2cl-lib:fref d (i) ((1 *))) zero) (setf info i)
      (go label30)))
    (setf ei (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))
    (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
            (/ ei (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
    (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         d-%offset%)
            (-
             (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                            d-%offset%)
             (* (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) ei)))
    (cond
     ((<= (f2cl-lib:fref d ((f2cl-lib:int-add i 1)) ((1 *))) zero)
      (setf info (f2cl-lib:int-add i 1)) (go label30)))
    (setf ei
            (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                           e-%offset%))
    (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         e-%offset%)
            (/ ei
               (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                              d-%offset%)))
    (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                         d-%offset%)
            (-
             (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 2)) ((1 *))
```

```
                                d-%offset%)
                         (*
                          (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                         e-%offset%)
                          ei)))
                (cond
                 ((<= (f2cl-lib:fref d ((f2cl-lib:int-add i 2)) ((1 *))) zero)
                  (setf info (f2cl-lib:int-add i 2)) (go label30)))
                (setf ei
                        (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                                       e-%offset%))
                (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                                     e-%offset%)
                        (/ ei
                           (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                                           d-%offset%)))
                (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                                     d-%offset%)
                        (-
                         (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                                        d-%offset%)
                         (*
                          (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                                         e-%offset%)
                          ei)))
                (cond
                 ((<= (f2cl-lib:fref d ((f2cl-lib:int-add i 3)) ((1 *))) zero)
                  (setf info (f2cl-lib:int-add i 3)) (go label30)))
                (setf ei
                        (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                                       e-%offset%))
                (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                                     e-%offset%)
                        (/ ei
                           (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                                           d-%offset%)))
                (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 4)) ((1 *))
                                     d-%offset%)
                        (-
                         (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 4)) ((1 *))
                                        d-%offset%)
                         (*
                          (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                                         e-%offset%)
                          ei)))
            label20))
        (if (<= (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) zero)
            (setf info n))
        label30
        (go end_label)
```

```
       end_label
        (return (values nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::spttrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

87.5 spttrs

87.5.1 Axiom unit tests

— **spttrs.input** —

```
)set break resume
)sys rm -f spttrs.output
)spool spttrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

87.5.2 Axiom help page

— **spttrs.help** —

```
==================================================================
```

```
spttrs examples
===================================================================

===================================================================
Man Page Details
===================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine spttrs (N, NRHS, D, E, B, LDB, INFO)
           SPTTRS

Function/Subroutine Documentation
   subroutine spttrs (integer N, integer NRHS, real, dimension( * ) D, real,
       dimension( * ) E, real, dimension( ldb, * ) B, integer LDB, integer
       INFO)
       SPTTRS

Purpose:
   SPTTRS solves a tridiagonal system of the form
      A * X = B
   using the L*D*L**T factorization of A computed by SPTTRF.  D is a
   diagonal matrix specified in the vector D, L is a unit bidiagonal
   matrix whose subdiagonal is specified in the vector E, and X and B
   are N by NRHS matrices.

Parameters:

 N
   N is INTEGER
   The order of the tridiagonal matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 D
   D is REAL array, dimension (N)
   The n diagonal elements of the diagonal matrix D from the
   L*D*L**T factorization of A.

 E
   E is REAL array, dimension (N-1)
   The (n-1) subdiagonal elements of the unit bidiagonal factor
   L from the L*D*L**T factorization of A.  E can also be regarded
   as the superdiagonal of the unit bidiagonal factor U from the
   factorization A = U**T*D*U.

 B
```

```
  B is REAL array, dimension (LDB,NRHS)
  On entry, the right hand side vectors B for the system of linear equations.
  On exit, the solution vectors, X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -k, the k-th argument had an illegal value
```

———-

87.5.3 fortran code

— **spttrs.f** —

```
*  =====================================================================
      SUBROUTINE SPTTRS( N, NRHS, D, E, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      REAL               B( LDB, * ), D( * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            J, JB, NB
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           SPTTS2, XERBLA
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SPTTRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
*     Determine the number of right-hand sides to solve at a time.
*
      IF( NRHS.EQ.1 ) THEN
         NB = 1
      ELSE
         NB = MAX( 1, ILAENV( 1, 'SPTTRS', ' ', N, NRHS, -1, -1 ) )
      END IF
*
      IF( NB.GE.NRHS ) THEN
         CALL SPTTS2( N, NRHS, D, E, B, LDB )
      ELSE
         DO 10 J = 1, NRHS, NB
            JB = MIN( NRHS-J+1, NB )
            CALL SPTTS2( N, JB, D, E, B( 1, J ), LDB )
   10    CONTINUE
      END IF
*
      RETURN
*
*     End of SPTTRS
*
      END
```

———-

87.5.4 lisp code

— spttrs.lisp —

```
(defun spttrs (n nrhs d e b ldb$ info)
  (declare (type (array single-float (*)) b e d)
           (type (f2cl-lib:integer4) info ldb$ nrhs n))
  (f2cl-lib:with-multi-array-data
      ((d single-float d-%data% d-%offset%)
       (e single-float e-%data% e-%offset%)
       (b single-float b-%data% b-%offset%))
    (prog ((j 0) (jb 0) (nb 0))
      (declare (type (f2cl-lib:integer4) nb jb j))
      (setf info 0)
      (cond ((< n 0) (setf info -1)) ((< nrhs 0) (setf info -2))
            ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -6)))
      (cond
       ((/= info 0) (xerbla "SPTTRS" (f2cl-lib:int-sub info)) (go end_label)))
      (if (or (= n 0) (= nrhs 0))
          (go end_label))
      (cond ((= nrhs 1) (setf nb 1))
            (t
             (setf nb
                     (max (the f2cl-lib:integer4 1)
                          (the f2cl-lib:integer4
                               (multiple-value-bind
                                   (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                                    var-6)
                                   (ilaenv 1 "SPTTRS" " " n nrhs -1 -1)
                                 (declare
                                  (ignore var-0 var-1 var-2 var-5 var-6))
                                 (setf n var-3)
                                 (setf nrhs var-4)
                                 ret-val))))))
      (cond
       ((>= nb nrhs)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (sptts2 n nrhs d e b ldb$)
          (declare (ignore var-2 var-3 var-4))
          (when var-0 (setf n var-0))
          (when var-1 (setf nrhs var-1))
          (when var-5 (setf ldb$ var-5))))
       (t
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                      ((> j nrhs) nil)
          (tagbody
            (setf jb
                    (min
```

```
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add (f2cl-lib:int-sub nrhs j) 1))
                    (the f2cl-lib:integer4 nb)))
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
                (sptts2 n jb d e
                 (f2cl-lib:array-slice b-%data% single-float (1 j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$)
              (declare (ignore var-2 var-3 var-4))
              (when var-0 (setf n var-0))
              (when var-1 (setf jb var-1))
              (when var-5 (setf ldb$ var-5)))
           label10))))
      (go end_label)
     end_label
      (return (values n nrhs nil nil nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::spttrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil nil nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::ilaenv))))
```

———-

87.6 sptts2

87.6.1 Axiom unit tests

— sptts2.input —

```
)set break resume
)sys rm -f sptts2.output
)spool sptts2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

87.6.2 Axiom help page

— sptts2.help —

```
==================================================================
sptts2 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sptts2 (N, NRHS, D, E, B, LDB)
           SPTTS2 solves a tridiagonal system of the form AX=B using the L D
           LH factorization computed by spttrf.

Function/Subroutine Documentation
   subroutine sptts2 (integer N, integer NRHS, real, dimension( * ) D, real,
       dimension( * ) E, real, dimension( ldb, * ) B, integer LDB)
       SPTTS2 solves a tridiagonal system of the form AX=B using the L D LH
       factorization computed by spttrf.

Purpose:
   SPTTS2 solves a tridiagonal system of the form
      A * X = B
   using the L*D*L**T factorization of A computed by SPTTRF.  D is a
   diagonal matrix specified in the vector D, L is a unit bidiagonal
   matrix whose subdiagonal is specified in the vector E, and X and B
   are N by NRHS matrices.

Parameters:

 N
   N is INTEGER
```

```
  The order of the tridiagonal matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrix B.  NRHS >= 0.

D
  D is REAL array, dimension (N)
  The n diagonal elements of the diagonal matrix D from the
  L*D*L**T factorization of A.

E
  E is REAL array, dimension (N-1)
  The (n-1) subdiagonal elements of the unit bidiagonal factor
  L from the L*D*L**T factorization of A.  E can also be regarded
  as the superdiagonal of the unit bidiagonal factor U from the
  factorization A = U**T*D*U.

B
  B is REAL array, dimension (LDB,NRHS)
  On entry, the right hand side vectors B for the system of linear equations.
  On exit, the solution vectors, X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).
```

———-

87.6.3 fortran code

— **sptts2.f** —

```
*  =====================================================================
      SUBROUTINE SPTTS2( N, NRHS, D, E, B, LDB )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      REAL               B( LDB, * ), D( * ), E( * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
*     ..
*     .. External Subroutines ..
      EXTERNAL           SSCAL
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( N.LE.1 ) THEN
         IF( N.EQ.1 )
     $      CALL SSCAL( NRHS, 1. / D( 1 ), B, LDB )
         RETURN
      END IF
*
*     Solve A * X = B using the factorization A = L*D*L**T,
*     overwriting each right hand side vector with its solution.
*
      DO 30 J = 1, NRHS
*
*           Solve L * x = b.
*
         DO 10 I = 2, N
            B( I, J ) = B( I, J ) - B( I-1, J )*E( I-1 )
   10    CONTINUE
*
*           Solve D * L**T * x = b.
*
         B( N, J ) = B( N, J ) / D( N )
         DO 20 I = N - 1, 1, -1
            B( I, J ) = B( I, J ) / D( I ) - B( I+1, J )*E( I )
   20    CONTINUE
   30 CONTINUE
*
      RETURN
*
*     End of SPTTS2
*
      END
```

———-

87.6.4 lisp code

— sptts2.lisp —

```
(defun sptts2 (n nrhs d e b ldb$)
  (declare (type (array single-float (*)) b e d)
           (type (f2cl-lib:integer4) ldb$ nrhs n))
  (f2cl-lib:with-multi-array-data
      ((d single-float d-%data% d-%offset%)
       (e single-float e-%data% e-%offset%)
       (b single-float b-%data% b-%offset%))
    (prog ((i 0) (j 0))
      (declare (type (f2cl-lib:integer4) j i))
      (cond
       ((<= n 1)
        (if (= n 1)
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (sscal nrhs
                 (/ 1.0 (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)) b
                 ldb$)
              (declare (ignore var-1 var-2))
              (when var-0 (setf nrhs var-0))
              (when var-3 (setf ldb$ var-3))))
        (go end_label)))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nrhs) nil)
        (tagbody
          (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *)) b-%offset%)
                      (-
                       (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                      b-%offset%)
                       (*
                        (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                        (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                       ((1 *)) e-%offset%))))
             label10))
          (setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
                  (/ (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
                     (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
          (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                           (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                        ((> i 1) nil)
            (tagbody
              (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *)) b-%offset%)
                      (-
```

```
                        (/
                         (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                        b-%offset%)
                         (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
                        (*
                         (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                        ((1 ldb$) (1 *)) b-%offset%)
                         (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))))
             label20))
         label30))
      (go end_label)
     end_label
      (return (values nil nrhs nil nil nil ldb$)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sptts2
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (array single-float (*))
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(nil fortran-to-lisp::nrhs nil nil
                                             nil fortran-to-lisp::ldb$)
                                           :calls 'nil)))
```

———-

Chapter 88

LAPACK - Positive Definite Tridiagonal Matrix, Comp. Routines, Complex

88.1 cptcon

88.1.1 Axiom unit tests

— **cptcon.input** —

```
)set break resume
)sys rm -f cptcon.output
)spool cptcon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

88.1.2 Axiom help page

— **cptcon.help** —

```
==========================================================================
```

```
cptcon examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cptcon (N, D, E, ANORM, RCOND, RWORK, INFO)
           CPTCON

Function/Subroutine Documentation
   subroutine cptcon (integer N, real, dimension( * ) D, complex, dimension( *
       ) E, real ANORM, real RCOND, real, dimension( * ) RWORK, integer INFO)
       CPTCON

Purpose:
   CPTCON computes the reciprocal of the condition number (in the
   1-norm) of a complex Hermitian positive definite tridiagonal matrix
   using the factorization A = L*D*L**H or A = U**H*D*U computed by CPTTRF.

   Norm(inv(A)) is computed by a direct method, and the reciprocal of
   the condition number is computed as
      RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 D
   D is REAL array, dimension (N)
   The n diagonal elements of the diagonal matrix D from the
   factorization of A, as computed by CPTTRF.

 E
   E is COMPLEX array, dimension (N-1)
   The (n-1) off-diagonal elements of the unit bidiagonal factor
   U or L from the factorization of A, as computed by CPTTRF.

 ANORM
   ANORM is REAL
   The 1-norm of the original matrix A.

 RCOND
   RCOND is REAL
   The reciprocal of the condition number of the matrix A,
   computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is the
```

```
  1-norm of inv(A) computed in this routine.

RWORK
  RWORK is REAL array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

88.1.3 fortran code

— cptcon.f —

```
*  =====================================================================
      SUBROUTINE CPTCON( N, D, E, ANORM, RCOND, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, N
      REAL               ANORM, RCOND
*     ..
*     .. Array Arguments ..
      REAL               D( * ), RWORK( * )
      COMPLEX            E( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IX
      REAL               AINVNM
*     ..
*     .. External Functions ..
      INTEGER            ISAMAX
      EXTERNAL           ISAMAX
*     ..
```

```
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CPTCON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
*     Check that D(1:N) is positive.
*
      DO 10 I = 1, N
         IF( D( I ).LE.ZERO )
     $      RETURN
   10 CONTINUE
*
*     Solve M(A) * x = e, where M(A) = (m(i,j)) is given by
*
*        m(i,j) =  abs(A(i,j)), i = j,
*        m(i,j) = -abs(A(i,j)), i .ne. j,
*
*     and e = [ 1, 1, ..., 1 ]**T.  Note M(A) = M(L)*D*M(L)**H.
*
*     Solve M(L) * x = e.
*
      RWORK( 1 ) = ONE
      DO 20 I = 2, N
         RWORK( I ) = ONE + RWORK( I-1 )*ABS( E( I-1 ) )
```

```
   20 CONTINUE
*
*     Solve D * M(L)**H * x = b.
*
      RWORK( N ) = RWORK( N ) / D( N )
      DO 30 I = N - 1, 1, -1
         RWORK( I ) = RWORK( I ) / D( I ) + RWORK( I+1 )*ABS( E( I ) )
   30 CONTINUE
*
*     Compute AINVNM = max(x(i)), 1<=i<=n.
*
      IX = ISAMAX( N, RWORK, 1 )
      AINVNM = ABS( RWORK( IX ) )
*
*     Compute the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
      RETURN
*
*     End of CPTCON
*
      END
```

88.1.4 lisp code

— cptcon.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun cptcon (n d e anorm rcond rwork info)
    (declare (type (single-float) rcond anorm)
             (type (array f2cl-lib:complex8 (*)) e)
             (type (array single-float (*)) rwork d)
             (type (f2cl-lib:integer4) info n))
    (f2cl-lib:with-multi-array-data
        ((d single-float d-%data% d-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%)
         (e f2cl-lib:complex8 e-%data% e-%offset%))
      (prog ((ainvnm 0.0) (i 0) (ix 0))
        (declare (type (single-float) ainvnm)
                 (type (f2cl-lib:integer4) i ix))
        (setf info 0)
```

```
(cond ((< n 0) (setf info -1)) ((< anorm zero) (setf info -4)))
(cond
 ((/= info 0) (xerbla "CPTCON" (f2cl-lib:int-sub info))
  (go end_label)))
(setf rcond zero)
(cond ((= n 0) (setf rcond one) (go end_label))
      ((= anorm zero) (go end_label)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (if (<= (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) zero)
        (go end_label))
   label10))
(setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%) one)
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
            (+ one
               (*
                (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-sub i 1))
                               ((1 *)) rwork-%offset%)
                (abs
                 (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                ((1 *)) e-%offset%)))))
   label20))
(setf (f2cl-lib:fref rwork-%data% (n) ((1 *)) rwork-%offset%)
        (/ (f2cl-lib:fref rwork-%data% (n) ((1 *)) rwork-%offset%)
           (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
(f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
            (+
             (/ (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
             (*
              (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add i 1))
                             ((1 *)) rwork-%offset%)
              (abs (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))))
   label30))
(setf ix
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (isamax n rwork 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(setf ainvnm
        (abs (f2cl-lib:fref rwork-%data% (ix) ((1 *)) rwork-%offset%)))
```

```
        (if (/= ainvnm zero)
            (setf rcond (/ (/ one ainvnm) anorm)))
        (go end_label)
       end_label
        (return (values n nil nil nil rcond nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cptcon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (single-float) (single-float)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::rcond nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

88.2 cpteqr

88.2.1 Axiom unit tests

— **cpteqr.input** —

```
)set break resume
)sys rm -f cpteqr.output
)spool cpteqr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

88.2.2 Axiom help page

— cpteqr.help —

```
=====================================================================
cpteqr examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cpteqr (COMPZ, N, D, E, Z, LDZ, WORK, INFO)
           CPTEQR

Function/Subroutine Documentation
   subroutine cpteqr (character COMPZ, integer N, real, dimension( * ) D,
       real, dimension( * ) E, complex, dimension( ldz, * ) Z, integer LDZ,
       real, dimension( * ) WORK, integer INFO)
       CPTEQR

Purpose:
   CPTEQR computes all eigenvalues and, optionally, eigenvectors of a
   symmetric positive definite tridiagonal matrix by first factoring the
   matrix using SPTTRF and then calling CBDSQR to compute the singular
   values of the bidiagonal factor.

   This routine computes the eigenvalues of the positive definite
   tridiagonal matrix to high relative accuracy.  This means that if the
   eigenvalues range over many orders of magnitude in size, then the
   small eigenvalues and corresponding eigenvectors will be computed
   more accurately than, for example, with the standard QR method.

   The eigenvectors of a full or band positive definite Hermitian matrix
   can also be found if CHETRD, CHPTRD, or CHBTRD has been used to
   reduce this matrix to tridiagonal form.  (The reduction to
   tridiagonal form, however, may preclude the possibility of obtaining
   high relative accuracy in the small eigenvalues of the original
   matrix, if these eigenvalues range over many orders of magnitude.)

Parameters:

 COMPZ
   COMPZ is CHARACTER*1
   = 'N':  Compute eigenvalues only.
   = 'V':  Compute eigenvectors of original Hermitian
      matrix also.  Array Z contains the unitary matrix
```

```
     used to reduce the original matrix to tridiagonal form.
  = 'I':  Compute eigenvectors of tridiagonal matrix also.

N
  N is INTEGER
  The order of the matrix.  N >= 0.

D
  D is REAL array, dimension (N)
  On entry, the n diagonal elements of the tridiagonal matrix.
  On normal exit, D contains the eigenvalues, in descending order.

E
  E is REAL array, dimension (N-1)
  On entry, the (n-1) subdiagonal elements of the tridiagonal matrix.
  On exit, E has been destroyed.

Z
  Z is COMPLEX array, dimension (LDZ, N)
  On entry, if COMPZ = 'V', the unitary matrix used in the
  reduction to tridiagonal form.
  On exit, if COMPZ = 'V', the orthonormal eigenvectors of the
  original Hermitian matrix;
  if COMPZ = 'I', the orthonormal eigenvectors of the tridiagonal matrix.
  If INFO > 0 on exit, Z contains the eigenvectors associated
  with only the stored eigenvalues.
  If  COMPZ = 'N', then Z is not referenced.

LDZ
  LDZ is INTEGER
  The leading dimension of the array Z.  LDZ >= 1, and if
  COMPZ = 'V' or 'I', LDZ >= max(1,N).

WORK
  WORK is REAL array, dimension (4*N)

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if INFO = i, and i is:
  <= N  the Cholesky factorization of the matrix could
     not be performed because the i-th principal minor
     was not positive definite.
  > N   the SVD algorithm failed to converge;
     if INFO = N+i, i off-diagonal elements of the
     bidiagonal factor did not converge to zero.
```

———-

88.2.3 fortran code

— cpteqr.f —

```
*  ====================================================================
      SUBROUTINE CPTEQR( COMPZ, N, D, E, Z, LDZ, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          COMPZ
      INTEGER            INFO, LDZ, N
*     ..
*     .. Array Arguments ..
      REAL               D( * ), E( * ), WORK( * )
      COMPLEX            Z( LDZ, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX            CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0E+0, 0.0E+0 ),
     $                   CONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           CBDSQR, CLASET, SPTTRF, XERBLA
*     ..
*     .. Local Arrays ..
      COMPLEX            C( 1, 1 ), VT( 1, 1 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, ICOMPZ, NRU
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
```

```
*
      IF( LSAME( COMPZ, 'N' ) ) THEN
         ICOMPZ = 0
      ELSE IF( LSAME( COMPZ, 'V' ) ) THEN
         ICOMPZ = 1
      ELSE IF( LSAME( COMPZ, 'I' ) ) THEN
         ICOMPZ = 2
      ELSE
         ICOMPZ = -1
      END IF
      IF( ICOMPZ.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( ( LDZ.LT.1 ) .OR. ( ICOMPZ.GT.0 .AND. LDZ.LT.MAX( 1,
     $         N ) ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CPTEQR', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
      IF( N.EQ.1 ) THEN
         IF( ICOMPZ.GT.0 )
     $      Z( 1, 1 ) = CONE
         RETURN
      END IF
      IF( ICOMPZ.EQ.2 )
     $   CALL CLASET( 'Full', N, N, CZERO, CONE, Z, LDZ )
*
*     Call SPTTRF to factor the matrix.
*
      CALL SPTTRF( N, D, E, INFO )
      IF( INFO.NE.0 )
     $   RETURN
      DO 10 I = 1, N
         D( I ) = SQRT( D( I ) )
   10 CONTINUE
      DO 20 I = 1, N - 1
         E( I ) = E( I )*D( I )
   20 CONTINUE
*
*     Call CBDSQR to compute the singular values/vectors of the
*     bidiagonal factor.
```

```
*
      IF( ICOMPZ.GT.0 ) THEN
         NRU = N
      ELSE
         NRU = 0
      END IF
      CALL CBDSQR( 'Lower', N, 0, NRU, 0, D, E, VT, 1, Z, LDZ, C, 1,
     $             WORK, INFO )
*
*     Square the singular values.
*
      IF( INFO.EQ.0 ) THEN
         DO 30 I = 1, N
            D( I ) = D( I )*D( I )
   30    CONTINUE
      ELSE
         INFO = N + INFO
      END IF
*
      RETURN
*
*     End of CPTEQR
*
      END
```

———-

88.2.4 lisp code

— **cpteqr.lisp** —

```
(let* ((czero (f2cl-lib:cmplx 0.0 0.0)) (cone (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (f2cl-lib:complex8) czero)
           (type (f2cl-lib:complex8) cone)
           (ignorable czero cone))
  (defun cpteqr (compz n d e z ldz work info)
    (declare (type (array f2cl-lib:complex8 (*)) z)
             (type (array single-float (*)) work e d)
             (type (f2cl-lib:integer4) info ldz n)
             (type (string *) compz))
    (f2cl-lib:with-multi-array-data
        ((compz character compz-%data% compz-%offset%)
         (d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%)
         (work single-float work-%data% work-%offset%)
         (z f2cl-lib:complex8 z-%data% z-%offset%))
      (prog ((i 0) (icompz 0) (nru 0)
             (c (make-array 1 :element-type 'f2cl-lib:complex8))
```

```
     (vt (make-array 1 :element-type 'f2cl-lib:complex8)))
(declare (type (f2cl-lib:integer4) i icompz nru)
         (type (array f2cl-lib:complex8 (1)) c vt))
(setf info 0)
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "N")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf icompz 0))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "V")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf icompz 1))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame compz "I")
    (declare (ignore var-1))
    (when var-0 (setf compz var-0))
    ret-val)
  (setf icompz 2))
 (t (setf icompz -1)))
(cond ((< icompz 0) (setf info -1)) ((< n 0) (setf info -2))
      ((or (< ldz 1)
           (and (> icompz 0)
                (< ldz
                   (max (the f2cl-lib:integer4 1)
                        (the f2cl-lib:integer4 n)))))
       (setf info -6)))
(cond
 ((/= info 0) (xerbla "CPTEQR" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(cond
 ((= n 1)
  (if (> icompz 0)
      (setf (f2cl-lib:fref z-%data% (1 1) ((1 ldz) (1 *)) z-%offset%)
              cone))
  (go end_label)))
(if (= icompz 2)
    (claset "Full" n n czero cone z ldz))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (spttrf n d e info)
  (declare (ignore var-1 var-2))
  (when var-0 (setf n var-0))
  (when var-3 (setf info var-3)))
(if (/= info 0)
```

```
            (go end_label))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                    (f2cl-lib:fsqrt
                     (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
           label10))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
          (tagbody
            (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                    (* (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                       (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
           label20))
        (cond ((> icompz 0) (setf nru n)) (t (setf nru 0)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
             var-11 var-12 var-13 var-14)
            (cbdsqr "Lower" n 0 nru 0 d e vt 1 z ldz c 1 work info)
          (declare
           (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9 var-11
            var-12 var-13))
          (setf n var-1)
          (setf nru var-3)
          (setf ldz var-10)
          (setf info var-14))
        (cond
         ((= info 0)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                      (* (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                         (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
             label30)))
         (t (setf info (f2cl-lib:int-add n info))))
        (go end_label)
       end_label
        (return (values compz n nil nil nil ldz nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cpteqr
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
```

```
  (array single-float (*))
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::compz
  fortran-to-lisp::n nil nil nil
  fortran-to-lisp::ldz nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::cbdsqr
  fortran-to-lisp::claset))))
```

———-

88.3 cptrfs

88.3.1 Axiom unit tests

— **cptrfs.input** —

```
)set break resume
)sys rm -f cptrfs.output
)spool cptrfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

88.3.2 Axiom help page

— **cptrfs.help** —

```
==========================================================================
cptrfs examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cptrfs (UPLO, N, NRHS, D, E, DF, EF, B, LDB, X, LDX, FERR,
           BERR, WORK, RWORK, INFO)
           CPTRFS

Function/Subroutine Documentation
   subroutine cptrfs (character UPLO, integer N, integer NRHS, real,
       dimension( * ) D, complex, dimension( * ) E, real, dimension( * ) DF,
       complex, dimension( * ) EF, complex, dimension( ldb, * ) B, integer
       LDB, complex, dimension( ldx, * ) X, integer LDX, real, dimension( * )
       FERR, real, dimension( * ) BERR, complex, dimension( * ) WORK, real,
       dimension( * ) RWORK, integer INFO)
       CPTRFS

Purpose:
   CPTRFS improves the computed solution to a system of linear
   equations when the coefficient matrix is Hermitian positive definite
   and tridiagonal, and provides error bounds and backward error
   estimates for the solution.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the superdiagonal or the subdiagonal of the
   tridiagonal matrix A is stored and the form of the
   factorization:
   = 'U':  E is the superdiagonal of A, and A = U**H*D*U;
   = 'L':  E is the subdiagonal of A, and A = L*D*L**H.
   (The two forms are equivalent if A is real.)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 D
   D is REAL array, dimension (N)
   The n real diagonal elements of the tridiagonal matrix A.
```

```
E
  E is COMPLEX array, dimension (N-1)
  The (n-1) off-diagonal elements of the tridiagonal matrix A (see UPLO).

DF
  DF is REAL array, dimension (N)
  The n diagonal elements of the diagonal matrix D from
  the factorization computed by CPTTRF.

EF
  EF is COMPLEX array, dimension (N-1)
  The (n-1) off-diagonal elements of the unit bidiagonal
  factor U or L from the factorization computed by CPTTRF (see UPLO).

B
  B is COMPLEX array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by CPTTRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

FERR
  FERR is REAL array, dimension (NRHS)
  The forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).

BERR
  BERR is REAL array, dimension (NRHS)
  The componentwise relative backward error of each solution
  vector X(j) (i.e., the smallest relative change in
  any element of A or B that makes X(j) an exact solution).

WORK
  WORK is COMPLEX array, dimension (N)

RWORK
```

```
   RWORK is REAL array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

   ITMAX is the maximum number of steps of iterative refinement.
```

88.3.3 fortran code

— cptrfs.f —

```
*  =====================================================================
      SUBROUTINE CPTRFS( UPLO, N, NRHS, D, E, DF, EF, B, LDB, X, LDX,
     $                   FERR, BERR, WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      REAL               BERR( * ), D( * ), DF( * ), FERR( * ),
     $                   RWORK( * )
      COMPLEX            B( LDB, * ), E( * ), EF( * ), WORK( * ),
     $                   X( LDX, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
      REAL               TWO
```

```
      PARAMETER          ( TWO = 2.0E+0 )
      REAL               THREE
      PARAMETER          ( THREE = 3.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            COUNT, I, IX, J, NZ
      REAL               EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN
      COMPLEX            BI, CX, DX, EX, ZDUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ISAMAX
      REAL               SLAMCH
      EXTERNAL           LSAME, ISAMAX, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           CAXPY, CPTTRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, AIMAG, CMPLX, CONJG, MAX, REAL
*     ..
*     .. Statement Functions ..
      REAL               CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( REAL( ZDUM ) ) + ABS( AIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -11
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CPTRFS', -INFO )
         RETURN
      END IF
*
```

```
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = 4
      EPS = SLAMCH( 'Epsilon' )
      SAFMIN = SLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 100 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - A * X.  Also compute
*        abs(A)*abs(x) + abs(b) for use in the backward error bound.
*
         IF( UPPER ) THEN
            IF( N.EQ.1 ) THEN
               BI = B( 1, J )
               DX = D( 1 )*X( 1, J )
               WORK( 1 ) = BI - DX
               RWORK( 1 ) = CABS1( BI ) + CABS1( DX )
            ELSE
               BI = B( 1, J )
               DX = D( 1 )*X( 1, J )
               EX = E( 1 )*X( 2, J )
               WORK( 1 ) = BI - DX - EX
               RWORK( 1 ) = CABS1( BI ) + CABS1( DX ) +
     $                      CABS1( E( 1 ) )*CABS1( X( 2, J ) )
               DO 30 I = 2, N - 1
                  BI = B( I, J )
                  CX = CONJG( E( I-1 ) )*X( I-1, J )
                  DX = D( I )*X( I, J )
                  EX = E( I )*X( I+1, J )
                  WORK( I ) = BI - CX - DX - EX
```

```
                RWORK( I ) = CABS1( BI ) +
     $                       CABS1( E( I-1 ) )*CABS1( X( I-1, J ) ) +
     $                       CABS1( DX ) + CABS1( E( I ) )*
     $                       CABS1( X( I+1, J ) )
   30          CONTINUE
               BI = B( N, J )
               CX = CONJG( E( N-1 ) )*X( N-1, J )
               DX = D( N )*X( N, J )
               WORK( N ) = BI - CX - DX
               RWORK( N ) = CABS1( BI ) + CABS1( E( N-1 ) )*
     $                      CABS1( X( N-1, J ) ) + CABS1( DX )
            END IF
         ELSE
            IF( N.EQ.1 ) THEN
               BI = B( 1, J )
               DX = D( 1 )*X( 1, J )
               WORK( 1 ) = BI - DX
               RWORK( 1 ) = CABS1( BI ) + CABS1( DX )
            ELSE
               BI = B( 1, J )
               DX = D( 1 )*X( 1, J )
               EX = CONJG( E( 1 ) )*X( 2, J )
               WORK( 1 ) = BI - DX - EX
               RWORK( 1 ) = CABS1( BI ) + CABS1( DX ) +
     $                      CABS1( E( 1 ) )*CABS1( X( 2, J ) )
               DO 40 I = 2, N - 1
                  BI = B( I, J )
                  CX = E( I-1 )*X( I-1, J )
                  DX = D( I )*X( I, J )
                  EX = CONJG( E( I ) )*X( I+1, J )
                  WORK( I ) = BI - CX - DX - EX
                  RWORK( I ) = CABS1( BI ) +
     $                       CABS1( E( I-1 ) )*CABS1( X( I-1, J ) ) +
     $                       CABS1( DX ) + CABS1( E( I ) )*
     $                       CABS1( X( I+1, J ) )
   40          CONTINUE
               BI = B( N, J )
               CX = E( N-1 )*X( N-1, J )
               DX = D( N )*X( N, J )
               WORK( N ) = BI - CX - DX
               RWORK( N ) = CABS1( BI ) + CABS1( E( N-1 ) )*
     $                      CABS1( X( N-1, J ) ) + CABS1( DX )
            END IF
         END IF
*
*        Compute componentwise relative backward error from formula
*
*        max(i) ( abs(R(i)) / ( abs(A)*abs(X) + abs(B) )(i) )
*
*        where abs(Z) is the componentwise absolute value of the matrix
```

```
*        or vector Z.  If the i-th component of the denominator is less
*        than SAFE2, then SAFE1 is added to the i-th components of the
*        numerator and denominator before dividing.
*
         S = ZERO
         DO 50 I = 1, N
            IF( RWORK( I ).GT.SAFE2 ) THEN
               S = MAX( S, CABS1( WORK( I ) ) / RWORK( I ) )
            ELSE
               S = MAX( S, ( CABS1( WORK( I ) )+SAFE1 ) /
     $             ( RWORK( I )+SAFE1 ) )
            END IF
   50    CONTINUE
         BERR( J ) = S
*
*        Test stopping criterion. Continue iterating if
*           1) The residual BERR(J) is larger than machine epsilon, and
*           2) BERR(J) decreased by at least a factor of 2 during the
*              last iteration, and
*           3) At most ITMAX iterations tried.
*
         IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $       COUNT.LE.ITMAX ) THEN
*
*           Update solution and try again.
*
            CALL CPTTRS( UPLO, N, 1, DF, EF, WORK, N, INFO )
            CALL CAXPY( N, CMPLX( ONE ), WORK, 1, X( 1, J ), 1 )
            LSTRES = BERR( J )
            COUNT = COUNT + 1
            GO TO 20
         END IF
*
*        Bound error from formula
*
*        norm(X - XTRUE) / norm(X) .le. FERR =
*        norm( abs(inv(A))*
*           ( abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) ))) / norm(X)
*
*        where
*          norm(Z) is the magnitude of the largest component of Z
*          inv(A) is the inverse of A
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(A)*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(A)*abs(X) + abs(B) is less than SAFE2.
```

```
*
         DO 60 I = 1, N
            IF( RWORK( I ).GT.SAFE2 ) THEN
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I )
            ELSE
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I ) +
     $                      SAFE1
            END IF
   60    CONTINUE
         IX = ISAMAX( N, RWORK, 1 )
         FERR( J ) = RWORK( IX )
*
*        Estimate the norm of inv(A).
*
*        Solve M(A) * x = e, where M(A) = (m(i,j)) is given by
*
*           m(i,j) =  abs(A(i,j)), i = j,
*           m(i,j) = -abs(A(i,j)), i .ne. j,
*
*        and e = [ 1, 1, ..., 1 ]**T.  Note M(A) = M(L)*D*M(L)**H.
*
*        Solve M(L) * x = e.
*
         RWORK( 1 ) = ONE
         DO 70 I = 2, N
            RWORK( I ) = ONE + RWORK( I-1 )*ABS( EF( I-1 ) )
   70    CONTINUE
*
*        Solve D * M(L)**H * x = b.
*
         RWORK( N ) = RWORK( N ) / DF( N )
         DO 80 I = N - 1, 1, -1
            RWORK( I ) = RWORK( I ) / DF( I ) +
     $                   RWORK( I+1 )*ABS( EF( I ) )
   80    CONTINUE
*
*        Compute norm(inv(A)) = max(x(i)), 1<=i<=n.
*
         IX = ISAMAX( N, RWORK, 1 )
         FERR( J ) = FERR( J )*ABS( RWORK( IX ) )
*
*        Normalize error.
*
         LSTRES = ZERO
         DO 90 I = 1, N
            LSTRES = MAX( LSTRES, ABS( X( I, J ) ) )
   90    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
```

```
  100 CONTINUE
*
      RETURN
*
*     End of CPTRFS
*
      END
```

88.3.4 lisp code

— cptrfs.lisp —

```
(let* ((itmax 5) (zero 0.0) (one 1.0) (two 2.0) (three 3.0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 3.0 3.0) three)
           (ignorable itmax zero one two three))
  (defun cptrfs (uplo n nrhs d e df ef b ldb$ x ldx ferr berr work rwork info)
    (declare (type (array f2cl-lib:complex8 (*)) work x b ef e)
             (type (array single-float (*)) rwork berr ferr df d)
             (type (f2cl-lib:integer4) info ldx ldb$ nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (d single-float d-%data% d-%offset%)
         (df single-float df-%data% df-%offset%)
         (ferr single-float ferr-%data% ferr-%offset%)
         (berr single-float berr-%data% berr-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%)
         (e f2cl-lib:complex8 e-%data% e-%offset%)
         (ef f2cl-lib:complex8 ef-%data% ef-%offset%)
         (b f2cl-lib:complex8 b-%data% b-%offset%)
         (x f2cl-lib:complex8 x-%data% x-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:freal zdum)) (abs (f2cl-lib:aimag zdum)))))
        (declare
         (ftype (function (f2cl-lib:complex8) (values single-float &rest t))
          cabs1))
        (prog ((bi #C(0.0 0.0)) (cx #C(0.0 0.0)) (dx #C(0.0 0.0))
               (ex #C(0.0 0.0)) (zdum #C(0.0 0.0)) (eps 0.0) (lstres 0.0)
               (s 0.0) (safe1 0.0) (safe2 0.0) (safmin 0.0) (count$ 0) (i 0)
               (ix 0) (j 0) (nz 0) (upper nil))
          (declare (type (f2cl-lib:complex8) bi cx dx ex zdum)
```

```
         (type (single-float) eps lstres s safe1 safe2 safmin)
         (type (f2cl-lib:integer4) count$ i ix j nz)
         (type f2cl-lib:logical upper))
(setf info 0)
(setf upper
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val))
(cond
 ((and (not upper)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
 ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9))
 ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -11)))
(cond
 ((/= info 0) (xerbla "CPTRFS" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((or (= n 0) (= nrhs 0))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nrhs) nil)
    (tagbody
      (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
              zero)
      (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%)
              zero)
     label10))
  (go end_label)))
(setf nz 4)
(setf eps (slamch "Epsilon"))
(setf safmin (slamch "Safe minimum"))
(setf safe1 (* nz safmin))
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (cond
```

```
(upper
 (cond
  ((= n 1)
   (setf bi
           (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                          b-%offset%))
   (setf dx
           (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
              (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                             x-%offset%)))
   (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
           (- bi dx))
   (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
           (+ (cabs1 bi) (cabs1 dx))))
  (t
   (setf bi
           (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                          b-%offset%))
   (setf dx
           (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
              (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                             x-%offset%)))
   (setf ex
           (* (f2cl-lib:fref e-%data% (1) ((1 *)) e-%offset%)
              (f2cl-lib:fref x-%data% (2 j) ((1 ldx) (1 *))
                             x-%offset%)))
   (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
           (- bi dx ex))
   (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
           (+ (cabs1 bi) (cabs1 dx)
              (*
               (cabs1
                (f2cl-lib:fref e-%data% (1) ((1 *)) e-%offset%))
               (cabs1
                (f2cl-lib:fref x-%data% (2 j) ((1 ldx) (1 *))
                               x-%offset%)))))
   (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                 ((> i
                     (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                  nil)
     (tagbody
       (setf bi
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%))
       (setf cx
               (*
                (f2cl-lib:conjg
                 (f2cl-lib:fref e-%data%
                                ((f2cl-lib:int-sub i 1)) ((1 *))
                                e-%offset%))
```

```
                (f2cl-lib:fref x-%data%
                               ((f2cl-lib:int-sub i 1) j)
                               ((1 ldx) (1 *)) x-%offset%)))
        (setf dx
                (*
                 (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                 (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                               x-%offset%)))
        (setf ex
                (*
                 (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                 (f2cl-lib:fref x-%data%
                               ((f2cl-lib:int-add i 1) j)
                               ((1 ldx) (1 *)) x-%offset%)))
        (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                             work-%offset%)
                (- bi cx dx ex))
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                             rwork-%offset%)
                (+ (cabs1 bi)
                   (*
                    (cabs1
                     (f2cl-lib:fref e-%data%
                                    ((f2cl-lib:int-sub i 1))
                                    ((1 *)) e-%offset%))
                    (cabs1
                     (f2cl-lib:fref x-%data%
                                    ((f2cl-lib:int-sub i 1) j)
                                    ((1 ldx) (1 *)) x-%offset%)))
                   (cabs1 dx)
                   (*
                    (cabs1
                     (f2cl-lib:fref e-%data% (i) ((1 *))
                                    e-%offset%))
                    (cabs1
                     (f2cl-lib:fref x-%data%
                                    ((f2cl-lib:int-add i 1) j)
                                    ((1 ldx) (1 *))
                                    x-%offset%)))))
   label30))
(setf bi
        (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                       b-%offset%))
(setf cx
        (*
         (f2cl-lib:conjg
          (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub n 1))
                         ((1 *)) e-%offset%))
         (f2cl-lib:fref x-%data% ((f2cl-lib:int-sub n 1) j)
                        ((1 ldx) (1 *)) x-%offset%)))
```

```
    (setf dx
            (* (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
               (f2cl-lib:fref x-%data% (n j) ((1 ldx) (1 *))
                              x-%offset%)))
    (setf (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
            (- bi cx dx))
    (setf (f2cl-lib:fref rwork-%data% (n) ((1 *)) rwork-%offset%)
            (+ (cabs1 bi)
               (*
                (cabs1
                 (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub n 1))
                                ((1 *)) e-%offset%))
                (cabs1
                 (f2cl-lib:fref x-%data%
                                ((f2cl-lib:int-sub n 1) j)
                                ((1 ldx) (1 *)) x-%offset%)))
               (cabs1 dx))))))
  (t
   (cond
    ((= n 1)
     (setf bi
            (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                           b-%offset%))
     (setf dx
            (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
               (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                              x-%offset%)))
     (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (- bi dx))
     (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
            (+ (cabs1 bi) (cabs1 dx))))
    (t
     (setf bi
            (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                           b-%offset%))
     (setf dx
            (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
               (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                              x-%offset%)))
     (setf ex
            (*
             (f2cl-lib:conjg
              (f2cl-lib:fref e-%data% (1) ((1 *)) e-%offset%))
             (f2cl-lib:fref x-%data% (2 j) ((1 ldx) (1 *))
                            x-%offset%)))
     (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (- bi dx ex))
     (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
            (+ (cabs1 bi) (cabs1 dx)
               (*
```

```
                (cabs1
                 (f2cl-lib:fref e-%data% (1) ((1 *)) e-%offset%))
                (cabs1
                 (f2cl-lib:fref x-%data% (2 j) ((1 ldx) (1 *))
                                x-%offset%)))))
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i
                  (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf bi
            (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                           b-%offset%))
    (setf cx
            (*
             (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                            ((1 *)) e-%offset%)
             (f2cl-lib:fref x-%data%
                            ((f2cl-lib:int-sub i 1) j)
                            ((1 ldx) (1 *)) x-%offset%)))
    (setf dx
            (*
             (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
             (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                            x-%offset%)))
    (setf ex
            (*
             (f2cl-lib:conjg
              (f2cl-lib:fref e-%data% (i) ((1 *))
                             e-%offset%))
             (f2cl-lib:fref x-%data%
                            ((f2cl-lib:int-add i 1) j)
                            ((1 ldx) (1 *)) x-%offset%)))
    (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                         work-%offset%)
            (- bi cx dx ex))
    (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                         rwork-%offset%)
            (+ (cabs1 bi)
               (*
                (cabs1
                 (f2cl-lib:fref e-%data%
                                ((f2cl-lib:int-sub i 1))
                                ((1 *)) e-%offset%))
                (cabs1
                 (f2cl-lib:fref x-%data%
                                ((f2cl-lib:int-sub i 1) j)
                                ((1 ldx) (1 *)) x-%offset%)))
               (cabs1 dx)
               (*
```

```
                        (cabs1
                         (f2cl-lib:fref e-%data% (i) ((1 *))
                                        e-%offset%))
                        (cabs1
                         (f2cl-lib:fref x-%data%
                                        ((f2cl-lib:int-add i 1) j)
                                        ((1 ldx) (1 *))
                                        x-%offset%)))))
          label40))
      (setf bi
              (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                             b-%offset%))
      (setf cx
              (*
               (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub n 1))
                              ((1 *)) e-%offset%)
               (f2cl-lib:fref x-%data% ((f2cl-lib:int-sub n 1) j)
                              ((1 ldx) (1 *)) x-%offset%)))
      (setf dx
              (* (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                 (f2cl-lib:fref x-%data% (n j) ((1 ldx) (1 *))
                                x-%offset%)))
      (setf (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
              (- bi cx dx))
      (setf (f2cl-lib:fref rwork-%data% (n) ((1 *)) rwork-%offset%)
              (+ (cabs1 bi)
                 (*
                  (cabs1
                   (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub n 1))
                                  ((1 *)) e-%offset%))
                  (cabs1
                   (f2cl-lib:fref x-%data%
                                  ((f2cl-lib:int-sub n 1) j)
                                  ((1 ldx) (1 *)) x-%offset%)))
                 (cabs1 dx)))))))
  (setf s zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
        (setf s
                (max s
                     (/
                      (cabs1
                       (f2cl-lib:fref work-%data% (i) ((1 *))
                                      work-%offset%))
                      (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                     rwork-%offset%)))))
       (t
```

```
      (setf s
              (max s
                   (/
                    (+
                     (cabs1
                      (f2cl-lib:fref work-%data% (i) ((1 *))
                                     work-%offset%))
                     safe1)
                    (+
                     (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                    rwork-%offset%)
                     safe1))))))
   label50))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (cpttrs uplo n 1 df ef work n info)
    (declare (ignore var-2 var-3 var-4 var-5))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-6 (setf n var-6))
    (when var-7 (setf info var-7)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (caxpy n (f2cl-lib:cmplx one) work 1
       (f2cl-lib:array-slice x-%data% f2cl-lib:complex8 (1 j)
                             ((1 ldx) (1 *)) x-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0)))
  (setf lstres
          (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
  (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
      (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                           rwork-%offset%)
              (+
               (cabs1
                (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%))
               (* nz eps
                  (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                 rwork-%offset%)))))
```

```
     (t
      (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                           rwork-%offset%)
              (+
               (cabs1
                (f2cl-lib:fref work-%data% (i) ((1 *))
                               work-%offset%))
               (* nz eps
                  (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                 rwork-%offset%))
               safe1))))
   label60))
(setf ix
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (isamax n rwork 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
        (f2cl-lib:fref rwork-%data% (ix) ((1 *)) rwork-%offset%))
(setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
        one)
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
            (+ one
               (*
                (f2cl-lib:fref rwork-%data%
                               ((f2cl-lib:int-sub i 1)) ((1 *))
                               rwork-%offset%)
                (abs
                 (f2cl-lib:fref ef-%data%
                                ((f2cl-lib:int-sub i 1)) ((1 *))
                                ef-%offset%)))))
   label70))
(setf (f2cl-lib:fref rwork-%data% (n) ((1 *)) rwork-%offset%)
        (/
         (f2cl-lib:fref rwork-%data% (n) ((1 *)) rwork-%offset%)
         (f2cl-lib:fref df-%data% (n) ((1 *)) df-%offset%)))
(f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
            (+
             (/
              (f2cl-lib:fref rwork-%data% (i) ((1 *))
                             rwork-%offset%)
              (f2cl-lib:fref df-%data% (i) ((1 *)) df-%offset%))
```

```
                          (*
                           (f2cl-lib:fref rwork-%data%
                                          ((f2cl-lib:int-add i 1)) ((1 *))
                                          rwork-%offset%)
                           (abs
                            (f2cl-lib:fref ef-%data% (i) ((1 *))
                                           ef-%offset%)))))
                 label80))
               (setf ix
                       (multiple-value-bind (ret-val var-0 var-1 var-2)
                           (isamax n rwork 1)
                         (declare (ignore var-1 var-2))
                         (when var-0 (setf n var-0))
                         ret-val))
               (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                       (* (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                          (abs
                           (f2cl-lib:fref rwork-%data% (ix) ((1 *))
                                          rwork-%offset%))))
               (setf lstres zero)
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i n) nil)
                 (tagbody
                   (setf lstres
                           (max lstres
                                (abs
                                 (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                                x-%offset%))))
                 label90))
               (if (/= lstres zero)
                   (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                           (/
                            (f2cl-lib:fref ferr-%data% (j) ((1 *))
                                           ferr-%offset%)
                            lstres)))
             label100))
           (go end_label)
         end_label
           (return
            (values uplo n nil nil nil nil nil nil nil nil nil nil nil nil nil
                    info)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cptrfs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
```

```
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array fortran-to-lisp::complex8
   (*))
  (array single-float (*))
  (array fortran-to-lisp::complex8
   (*))
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::complex8
   (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::complex8
   (*))
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::uplo
  fortran-to-lisp::n nil nil nil
  nil nil nil nil nil nil nil nil
  nil nil fortran-to-lisp::info)
:calls 'nil)))
```

———-

88.4 cpttrf

88.4.1 Axiom unit tests

— **cpttrf.input** —

```
)set break resume
)sys rm -f cpttrf.output
)spool cpttrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

88.4.2 Axiom help page

— cpttrf.help —

```
=====================================================================
cpttrf examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cpttrf (N, D, E, INFO)
           CPTTRF

Function/Subroutine Documentation
   subroutine cpttrf (integer N, real, dimension( * ) D, complex, dimension( *
       ) E, integer INFO)
       CPTTRF

Purpose:
   CPTTRF computes the L*D*L**H factorization of a complex Hermitian
   positive definite tridiagonal matrix A.  The factorization may also
   be regarded as having the form A = U**H *D*U.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 D
   D is REAL array, dimension (N)
   On entry, the n diagonal elements of the tridiagonal matrix
   A.  On exit, the n diagonal elements of the diagonal matrix
   D from the L*D*L**H factorization of A.

 E
   E is COMPLEX array, dimension (N-1)
   On entry, the (n-1) subdiagonal elements of the tridiagonal
   matrix A.  On exit, the (n-1) subdiagonal elements of the
   unit bidiagonal factor L from the L*D*L**H factorization of A.
   E can also be regarded as the superdiagonal of the unit
   bidiagonal factor U from the U**H *D*U factorization of A.

 INFO
   INFO is INTEGER
```

```
   = 0: successful exit
   < 0: if INFO = -k, the k-th argument had an illegal value
   > 0: if INFO = k, the leading minor of order k is not
      positive definite; if k < N, the factorization could not
      be completed, while if k = N, the factorization was
      completed, but D(N) <= 0.
```

88.4.3 fortran code

— cpttrf.f —

```
*  =====================================================================
      SUBROUTINE CPTTRF( N, D, E, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, N
*     ..
*     .. Array Arguments ..
      REAL               D( * )
      COMPLEX            E( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, I4
      REAL               EII, EIR, F, G
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          AIMAG, CMPLX, MOD, REAL
*     ..
*     .. Executable Statements ..
*
```

```
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
         CALL XERBLA( 'CPTTRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Compute the L*D*L**H (or U**H *D*U) factorization of A.
*
      I4 = MOD( N-1, 4 )
      DO 10 I = 1, I4
         IF( D( I ).LE.ZERO ) THEN
            INFO = I
            GO TO 20
         END IF
         EIR = REAL( E( I ) )
         EII = AIMAG( E( I ) )
         F = EIR / D( I )
         G = EII / D( I )
         E( I ) = CMPLX( F, G )
         D( I+1 ) = D( I+1 ) - F*EIR - G*EII
   10 CONTINUE
*
      DO 110 I = I4+1, N - 4, 4
*
*        Drop out of the loop if d(i) <= 0: the matrix is not positive
*        definite.
*
         IF( D( I ).LE.ZERO ) THEN
            INFO = I
            GO TO 20
         END IF
*
*        Solve for e(i) and d(i+1).
*
         EIR = REAL( E( I ) )
         EII = AIMAG( E( I ) )
         F = EIR / D( I )
         G = EII / D( I )
         E( I ) = CMPLX( F, G )
         D( I+1 ) = D( I+1 ) - F*EIR - G*EII
*
         IF( D( I+1 ).LE.ZERO ) THEN
```

```
            INFO = I+1
            GO TO 20
         END IF
*
*        Solve for e(i+1) and d(i+2).
*
         EIR = REAL( E( I+1 ) )
         EII = AIMAG( E( I+1 ) )
         F = EIR / D( I+1 )
         G = EII / D( I+1 )
         E( I+1 ) = CMPLX( F, G )
         D( I+2 ) = D( I+2 ) - F*EIR - G*EII
*
         IF( D( I+2 ).LE.ZERO ) THEN
            INFO = I+2
            GO TO 20
         END IF
*
*        Solve for e(i+2) and d(i+3).
*
         EIR = REAL( E( I+2 ) )
         EII = AIMAG( E( I+2 ) )
         F = EIR / D( I+2 )
         G = EII / D( I+2 )
         E( I+2 ) = CMPLX( F, G )
         D( I+3 ) = D( I+3 ) - F*EIR - G*EII
*
         IF( D( I+3 ).LE.ZERO ) THEN
            INFO = I+3
            GO TO 20
         END IF
*
*        Solve for e(i+3) and d(i+4).
*
         EIR = REAL( E( I+3 ) )
         EII = AIMAG( E( I+3 ) )
         F = EIR / D( I+3 )
         G = EII / D( I+3 )
         E( I+3 ) = CMPLX( F, G )
         D( I+4 ) = D( I+4 ) - F*EIR - G*EII
  110 CONTINUE
*
*     Check d(n) for positive definiteness.
*
      IF( D( N ).LE.ZERO )
     $   INFO = N
*
   20 CONTINUE
      RETURN
*
```

```
*     End of CPTTRF
*
      END
```

———-

88.4.4 lisp code

— **cpttrf.lisp** —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun cpttrf (n d e info)
    (declare (type (array f2cl-lib:complex8 (*)) e)
             (type (array single-float (*)) d)
             (type (f2cl-lib:integer4) info n))
    (f2cl-lib:with-multi-array-data
        ((d single-float d-%data% d-%offset%)
         (e f2cl-lib:complex8 e-%data% e-%offset%))
      (prog ((eii 0.0) (eir 0.0) (f 0.0) (g 0.0) (i 0) (i4 0))
        (declare (type (single-float) eii eir f g)
                 (type (f2cl-lib:integer4) i i4))
        (setf info 0)
        (cond
         ((< n 0) (setf info -1) (xerbla "CPTTRF" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (= n 0)
            (go end_label))
        (setf i4 (mod (f2cl-lib:int-sub n 1) 4))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i i4) nil)
          (tagbody
            (cond
             ((<= (f2cl-lib:fref d (i) ((1 *))) zero) (setf info i)
              (go label20)))
            (setf eir
                    (f2cl-lib:freal
                     (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
            (setf eii
                    (f2cl-lib:aimag
                     (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
            (setf f (/ eir (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
            (setf g (/ eii (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
            (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                    (f2cl-lib:cmplx f g))
            (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                 d-%offset%)
```

```
                (-
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                d-%offset%)
                 (* f eir) (* g eii)))
   label10))
(f2cl-lib:fdo (i (f2cl-lib:int-add i4 1) (f2cl-lib:int-add i 4))
              ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 4))) nil)
  (tagbody
    (cond
     ((<= (f2cl-lib:fref d (i) ((1 *))) zero) (setf info i)
      (go label20)))
    (setf eir
            (f2cl-lib:freal
             (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
    (setf eii
            (f2cl-lib:aimag
             (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
    (setf f (/ eir (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
    (setf g (/ eii (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
    (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
            (f2cl-lib:cmplx f g))
    (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                          d-%offset%)
            (-
             (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                            d-%offset%)
             (* f eir) (* g eii)))
    (cond
     ((<= (f2cl-lib:fref d ((f2cl-lib:int-add i 1)) ((1 *))) zero)
      (setf info (f2cl-lib:int-add i 1)) (go label20)))
    (setf eir
            (f2cl-lib:freal
             (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                            e-%offset%)))
    (setf eii
            (f2cl-lib:aimag
             (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                            e-%offset%)))
    (setf f
            (/ eir
               (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                              d-%offset%)))
    (setf g
            (/ eii
               (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                              d-%offset%)))
    (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                          e-%offset%)
            (f2cl-lib:cmplx f g))
    (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 2)) ((1 *))
```

```
                    d-%offset%)
        (-
         (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                        d-%offset%)
         (* f eir) (* g eii)))
(cond
 ((<= (f2cl-lib:fref d ((f2cl-lib:int-add i 2)) ((1 *))) zero)
  (setf info (f2cl-lib:int-add i 2)) (go label20)))
(setf eir
        (f2cl-lib:freal
         (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                        e-%offset%)))
(setf eii
        (f2cl-lib:aimag
         (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                        e-%offset%)))
(setf f
        (/ eir
           (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                          d-%offset%)))
(setf g
        (/ eii
           (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                          d-%offset%)))
(setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                    e-%offset%)
        (f2cl-lib:cmplx f g))
(setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                    d-%offset%)
        (-
         (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                        d-%offset%)
         (* f eir) (* g eii)))
(cond
 ((<= (f2cl-lib:fref d ((f2cl-lib:int-add i 3)) ((1 *))) zero)
  (setf info (f2cl-lib:int-add i 3)) (go label20)))
(setf eir
        (f2cl-lib:freal
         (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                        e-%offset%)))
(setf eii
        (f2cl-lib:aimag
         (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                        e-%offset%)))
(setf f
        (/ eir
           (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                          d-%offset%)))
(setf g
        (/ eii
```

```
                      (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                                     d-%offset%)))
            (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                                 e-%offset%)
                  (f2cl-lib:cmplx f g))
            (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 4)) ((1 *))
                                 d-%offset%)
                  (-
                   (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 4)) ((1 *))
                                  d-%offset%)
                   (* f eir) (* g eii)))
          label110))
        (if (<= (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) zero)
            (setf info n))
      label20
       (go end_label)
      end_label
       (return (values nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cpttrf
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(nil nil nil
                                             fortran-to-lisp::info)
                                           :calls 'nil)))
```

———-

88.5 cpttrs

88.5.1 Axiom unit tests

— cpttrs.input —

```
)set break resume
)sys rm -f cpttrs.output
```

```
)spool cpttrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

88.5.2 Axiom help page

— **cpttrs.help** —

```
=====================================================================
cpttrs examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cpttrs (UPLO, N, NRHS, D, E, B, LDB, INFO)
           CPTTRS

Function/Subroutine Documentation
   subroutine cpttrs (character UPLO, integer N, integer NRHS, real,
       dimension( * ) D, complex, dimension( * ) E, complex, dimension( ldb, *
       ) B, integer LDB, integer INFO)
       CPTTRS

Purpose:
   CPTTRS solves a tridiagonal system of the form
      A * X = B
   using the factorization A = U**H*D*U or A = L*D*L**H computed by CPTTRF.
   D is a diagonal matrix specified in the vector D, U (or L) is a unit
   bidiagonal matrix whose superdiagonal (subdiagonal) is specified in
   the vector E, and X and B are N by NRHS matrices.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies the form of the factorization and whether the
   vector E is the superdiagonal of the upper bidiagonal factor
   U or the subdiagonal of the lower bidiagonal factor L.
```

```
  = 'U':  A = U**H*D*U, E is the superdiagonal of U
  = 'L':  A = L*D*L**H, E is the subdiagonal of L

N
  N is INTEGER
  The order of the tridiagonal matrix A.  N >= 0.

NRHS
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrix B.  NRHS >= 0.

D
  D is REAL array, dimension (N)
  The n diagonal elements of the diagonal matrix D from the
  factorization A = U**H*D*U or A = L*D*L**H.

E
  E is COMPLEX array, dimension (N-1)
  If UPLO = 'U', the (n-1) superdiagonal elements of the unit
  bidiagonal factor U from the factorization A = U**H*D*U.
  If UPLO = 'L', the (n-1) subdiagonal elements of the unit
  bidiagonal factor L from the factorization A = L*D*L**H.

B
  B is REAL array, dimension (LDB,NRHS)
  On entry, the right hand side vectors B for the system of linear equations.
  On exit, the solution vectors, X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

INFO
  INFO is INTEGER
  = 0: successful exit
  < 0: if INFO = -k, the k-th argument had an illegal value
```

———-

88.5.3 fortran code

— cpttrs.f —

```
*  ==================================================================
      SUBROUTINE CPTTRS( UPLO, N, NRHS, D, E, B, LDB, INFO )
*
```

```
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      REAL               D( * )
      COMPLEX            B( LDB, * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            IUPLO, J, JB, NB
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           CPTTS2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      UPPER = ( UPLO.EQ.'U' .OR. UPLO.EQ.'u' )
      IF( .NOT.UPPER .AND. .NOT.( UPLO.EQ.'L' .OR. UPLO.EQ.'l' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CPTTRS', -INFO )
         RETURN
      END IF
*
```

```
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
*     Determine the number of right-hand sides to solve at a time.
*
      IF( NRHS.EQ.1 ) THEN
         NB = 1
      ELSE
         NB = MAX( 1, ILAENV( 1, 'CPTTRS', UPLO, N, NRHS, -1, -1 ) )
      END IF
*
*     Decode UPLO
*
      IF( UPPER ) THEN
         IUPLO = 1
      ELSE
         IUPLO = 0
      END IF
*
      IF( NB.GE.NRHS ) THEN
         CALL CPTTS2( IUPLO, N, NRHS, D, E, B, LDB )
      ELSE
         DO 10 J = 1, NRHS, NB
            JB = MIN( NRHS-J+1, NB )
            CALL CPTTS2( IUPLO, N, JB, D, E, B( 1, J ), LDB )
   10    CONTINUE
      END IF
*
      RETURN
*
*     End of CPTTRS
*
      END
```

———-

88.5.4 lisp code

— cpttrs.lisp —

```
(defun cpttrs (uplo n nrhs d e b ldb$ info)
  (declare (type (array f2cl-lib:complex8 (*)) b e)
           (type (array single-float (*)) d)
           (type (f2cl-lib:integer4) info ldb$ nrhs n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
```

```
    ((uplo character uplo-%data% uplo-%offset%)
     (d single-float d-%data% d-%offset%)
     (e f2cl-lib:complex8 e-%data% e-%offset%)
     (b f2cl-lib:complex8 b-%data% b-%offset%))
  (prog ((iuplo 0) (j 0) (jb 0) (nb 0) (upper nil))
    (declare (type f2cl-lib:logical upper)
             (type (f2cl-lib:integer4) nb jb j iuplo))
    (setf info 0)
    (setf upper
            (or (f2cl-lib:fstring-= uplo "U") (f2cl-lib:fstring-= uplo "u")))
    (cond
     ((and (not upper)
           (not
            (or (f2cl-lib:fstring-= uplo "L")
                (f2cl-lib:fstring-= uplo "l"))))
      (setf info -1))
     ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
     ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -7)))
    (cond
     ((/= info 0) (xerbla "CPTTRS" (f2cl-lib:int-sub info)) (go end_label)))
    (if (or (= n 0) (= nrhs 0))
        (go end_label))
    (cond ((= nrhs 1) (setf nb 1))
          (t
           (setf nb
                   (max (the f2cl-lib:integer4 1)
                        (the f2cl-lib:integer4
                             (multiple-value-bind
                                 (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                                  var-6)
                                 (ilaenv 1 "CPTTRS" uplo n nrhs -1 -1)
                               (declare (ignore var-0 var-1 var-5 var-6))
                               (when var-2 (setf uplo var-2))
                               (when var-3 (setf n var-3))
                               (when var-4 (setf nrhs var-4))
                               ret-val))))))
    (cond (upper (setf iuplo 1)) (t (setf iuplo 0)))
    (cond
     ((>= nb nrhs)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (cptts2 iuplo n nrhs d e b ldb$)
        (declare (ignore var-3 var-4 var-5))
        (when var-0 (setf iuplo var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf nrhs var-2))
        (when var-6 (setf ldb$ var-6))))
     (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                    ((> j nrhs) nil)
```

```
          (tagbody
            (setf jb
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-sub nrhs j) 1))
                     (the f2cl-lib:integer4 nb)))
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (cptts2 iuplo n jb d e
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex8 (1 j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$)
              (declare (ignore var-3 var-4 var-5))
              (when var-0 (setf iuplo var-0))
              (when var-1 (setf n var-1))
              (when var-2 (setf jb var-2))
              (when var-6 (setf ldb$ var-6)))
           label10))))
      (go end_label)
     end_label
      (return (values uplo n nrhs nil nil nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cpttrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n
                                              fortran-to-lisp::nrhs nil nil nil
                                              fortran-to-lisp::ldb$
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

88.6 cptts2

88.6.1 Axiom unit tests

— cptts2.input —

```
)set break resume
)sys rm -f cptts2.output
)spool cptts2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

88.6.2 Axiom help page

— cptts2.help —

```
==================================================================
cptts2 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cptts2 (IUPLO, N, NRHS, D, E, B, LDB)
           CPTTS2 solves a tridiagonal system of the form AX=B using the L D
           LH factorization computed by spttrf.

Function/Subroutine Documentation
   subroutine cptts2 (integer IUPLO, integer N, integer NRHS, real, dimension(
       * ) D, complex, dimension( * ) E, complex, dimension( ldb, * ) B,
       integer LDB)
       CPTTS2 solves a tridiagonal system of the form AX=B using the L D LH
       factorization computed by spttrf.

Purpose:
   CPTTS2 solves a tridiagonal system of the form
     A * X = B
```

```
  using the factorization A = U**H*D*U or A = L*D*L**H computed by CPTTRF.
  D is a diagonal matrix specified in the vector D, U (or L) is a unit
  bidiagonal matrix whose superdiagonal (subdiagonal) is specified in
  the vector E, and X and B are N by NRHS matrices.

Parameters:

 IUPLO
   IUPLO is INTEGER
   Specifies the form of the factorization and whether the
   vector E is the superdiagonal of the upper bidiagonal factor
   U or the subdiagonal of the lower bidiagonal factor L.
   = 1:  A = U**H *D*U, E is the superdiagonal of U
   = 0:  A = L*D*L**H, E is the subdiagonal of L

 N
   N is INTEGER
   The order of the tridiagonal matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 D
   D is REAL array, dimension (N)
   The n diagonal elements of the diagonal matrix D from the
   factorization A = U**H *D*U or A = L*D*L**H.

 E
   E is COMPLEX array, dimension (N-1)
   If IUPLO = 1, the (n-1) superdiagonal elements of the unit
   bidiagonal factor U from the factorization A = U**H*D*U.
   If IUPLO = 0, the (n-1) subdiagonal elements of the unit
   bidiagonal factor L from the factorization A = L*D*L**H.

 B
   B is REAL array, dimension (LDB,NRHS)
   On entry, the right hand side vectors B for the system of linear equations.
   On exit, the solution vectors, X.

 LDB
   LDB is INTEGER
   The leading dimension of the array B.  LDB >= max(1,N).
```

———-

88.6.3 fortran code

— cptts2.f —

```
*  =====================================================================
      SUBROUTINE CPTTS2( IUPLO, N, NRHS, D, E, B, LDB )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            IUPLO, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      REAL               D( * )
      COMPLEX            B( LDB, * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
*     ..
*     .. External Subroutines ..
      EXTERNAL           CSSCAL
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          CONJG
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( N.LE.1 ) THEN
         IF( N.EQ.1 )
     $      CALL CSSCAL( NRHS, 1. / D( 1 ), B, LDB )
         RETURN
      END IF
*
      IF( IUPLO.EQ.1 ) THEN
*
*        Solve A * X = B using the factorization A = U**H *D*U,
*        overwriting each right hand side vector with its solution.
*
         IF( NRHS.LE.2 ) THEN
            J = 1
    5       CONTINUE
```

```
*
*          Solve U**H * x = b.
*
           DO 10 I = 2, N
              B( I, J ) = B( I, J ) - B( I-1, J )*CONJG( E( I-1 ) )
   10      CONTINUE
*
*          Solve D * U * x = b.
*
           DO 20 I = 1, N
              B( I, J ) = B( I, J ) / D( I )
   20      CONTINUE
           DO 30 I = N - 1, 1, -1
              B( I, J ) = B( I, J ) - B( I+1, J )*E( I )
   30      CONTINUE
           IF( J.LT.NRHS ) THEN
              J = J + 1
              GO TO 5
           END IF
        ELSE
           DO 60 J = 1, NRHS
*
*             Solve U**H * x = b.
*
              DO 40 I = 2, N
                 B( I, J ) = B( I, J ) - B( I-1, J )*CONJG( E( I-1 ) )
   40         CONTINUE
*
*             Solve D * U * x = b.
*
              B( N, J ) = B( N, J ) / D( N )
              DO 50 I = N - 1, 1, -1
                 B( I, J ) = B( I, J ) / D( I ) - B( I+1, J )*E( I )
   50         CONTINUE
   60      CONTINUE
        END IF
     ELSE
*
*       Solve A * X = B using the factorization A = L*D*L**H,
*       overwriting each right hand side vector with its solution.
*
        IF( NRHS.LE.2 ) THEN
           J = 1
   65      CONTINUE
*
*          Solve L * x = b.
*
           DO 70 I = 2, N
              B( I, J ) = B( I, J ) - B( I-1, J )*E( I-1 )
   70      CONTINUE
```

```
*
*          Solve D * L**H * x = b.
*
           DO 80 I = 1, N
              B( I, J ) = B( I, J ) / D( I )
   80      CONTINUE
           DO 90 I = N - 1, 1, -1
              B( I, J ) = B( I, J ) - B( I+1, J )*CONJG( E( I ) )
   90      CONTINUE
           IF( J.LT.NRHS ) THEN
              J = J + 1
              GO TO 65
           END IF
        ELSE
           DO 120 J = 1, NRHS
*
*             Solve L * x = b.
*
              DO 100 I = 2, N
                 B( I, J ) = B( I, J ) - B( I-1, J )*E( I-1 )
  100         CONTINUE
*
*             Solve D * L**H * x = b.
*
              B( N, J ) = B( N, J ) / D( N )
              DO 110 I = N - 1, 1, -1
                 B( I, J ) = B( I, J ) / D( I ) -
     $                       B( I+1, J )*CONJG( E( I ) )
  110         CONTINUE
  120      CONTINUE
        END IF
      END IF
*
      RETURN
*
*     End of CPTTS2
*
      END
```

———-

88.6.4 lisp code

— cptts2.lisp —

```
(defun cptts2 (iuplo n nrhs d e b ldb$)
  (declare (type (array f2cl-lib:complex8 (*)) b e)
           (type (array single-float (*)) d)
```

```
          (type (f2cl-lib:integer4) ldb$ nrhs n iuplo))
  (f2cl-lib:with-multi-array-data
      ((d single-float d-%data% d-%offset%)
       (e f2cl-lib:complex8 e-%data% e-%offset%)
       (b f2cl-lib:complex8 b-%data% b-%offset%))
    (prog ((i 0) (j 0))
      (declare (type (f2cl-lib:integer4) j i))
      (cond
       ((<= n 1)
        (if (= n 1)
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (csscal nrhs
                 (/ 1.0 (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)) b
                 ldb$)
              (declare (ignore var-1 var-2))
              (when var-0 (setf nrhs var-0))
              (when var-3 (setf ldb$ var-3))))
        (go end_label)))
      (cond
       ((= iuplo 1)
        (cond
         ((<= nrhs 2)
          (tagbody
            (setf j 1)
           label5
            (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                     b-%offset%)
                      (-
                       (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                      b-%offset%)
                       (*
                        (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                        (f2cl-lib:conjg
                         (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                        ((1 *)) e-%offset%)))))
               label10))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                     b-%offset%)
                      (/
                       (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                      b-%offset%)
                       (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
               label20))
```

```
      (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                     (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                    ((> i 1) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                (-
                 (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                 (*
                  (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                  (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))))
         label30))
      (cond ((< j nrhs) (setf j (f2cl-lib:int-add j 1)) (go label5)))))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j nrhs) nil)
      (tagbody
        (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)
                  (-
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)
                   (*
                    (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                   ((1 ldb$) (1 *)) b-%offset%)
                    (f2cl-lib:conjg
                     (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                    ((1 *)) e-%offset%)))))
           label40))
        (setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
              (/
               (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                              b-%offset%)
               (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
        (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                       (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)
                  (-
                   (/
                    (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                   b-%offset%)
                    (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
```

```
                        (*
                         (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                        ((1 ldb$) (1 *)) b-%offset%)
                         (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))))
            label50))
       label60)))))
(t
 (cond
  ((<= nrhs 2)
   (tagbody
     (setf j 1)
    label65
     (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)
                 (-
                  (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (*
                   (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                  ((1 ldb$) (1 *)) b-%offset%)
                   (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                  ((1 *)) e-%offset%))))
        label70))
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)
                 (/
                  (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
        label80))
     (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                      (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                   ((> i 1) nil)
       (tagbody
         (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)
                 (-
                  (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (*
                   (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                  ((1 ldb$) (1 *)) b-%offset%)
                   (f2cl-lib:conjg
                    (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))))
```

```
              label90))
            (cond ((< j nrhs) (setf j (f2cl-lib:int-add j 1)) (go label65)))))
          (t
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j nrhs) nil)
            (tagbody
              (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                            ((> i n) nil)
                (tagbody
                  (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)
                        (-
                         (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                        b-%offset%)
                         (*
                          (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                         ((1 ldb$) (1 *)) b-%offset%)
                          (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                         ((1 *)) e-%offset%))))
                label100))
              (setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
                    (/
                     (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                                    b-%offset%)
                     (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
              (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                               (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                            ((> i 1) nil)
                (tagbody
                  (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)
                        (-
                         (/
                          (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                         b-%offset%)
                          (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
                         (*
                          (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                         ((1 ldb$) (1 *)) b-%offset%)
                          (f2cl-lib:conjg
                           (f2cl-lib:fref e-%data% (i) ((1 *))
                                          e-%offset%)))))
                label110))
            label120))))))
      (go end_label)
     end_label
      (return (values nil nil nrhs nil nil nil ldb$)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
```

```
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cptts2
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (array fortran-to-lisp::complex8
                                              (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(nil nil fortran-to-lisp::nrhs nil
                                             nil nil fortran-to-lisp::ldb$)
                                           :calls 'nil)))
```

Chapter 89

LAPACK - Positive Definite Tridiagonal Matrix, Comp. Routines, Complex16

89.1 zptcon

89.1.1 Axiom unit tests

— zptcon.input —

```
)set break resume
)sys rm -f zptcon.output
)spool zptcon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

89.1.2 Axiom help page

— zptcon.help —

```
==========================================================================
```

```
zptcon examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zptcon (N, D, E, ANORM, RCOND, RWORK, INFO)
           ZPTCON

Function/Subroutine Documentation
   subroutine zptcon (integer N, double precision, dimension( * ) D,
       complex*16, dimension( * ) E, double precision ANORM, double precision
       RCOND, double precision, dimension( * ) RWORK, integer INFO)
       ZPTCON

Purpose:
   ZPTCON computes the reciprocal of the condition number (in the
   1-norm) of a complex Hermitian positive definite tridiagonal matrix
   using the factorization A = L*D*L**H or A = U**H*D*U computed by
   ZPTTRF.

   Norm(inv(A)) is computed by a direct method, and the reciprocal of
   the condition number is computed as
       RCOND = 1 / (ANORM * norm(inv(A))).

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 D
   D is DOUBLE PRECISION array, dimension (N)
   The n diagonal elements of the diagonal matrix D from the
   factorization of A, as computed by ZPTTRF.

 E
   E is COMPLEX*16 array, dimension (N-1)
   The (n-1) off-diagonal elements of the unit bidiagonal factor
   U or L from the factorization of A, as computed by ZPTTRF.

 ANORM
   ANORM is DOUBLE PRECISION
   The 1-norm of the original matrix A.

 RCOND
   RCOND is DOUBLE PRECISION
```

```
  The reciprocal of the condition number of the matrix A,
  computed as RCOND = 1/(ANORM * AINVNM), where AINVNM is the
  1-norm of inv(A) computed in this routine.

RWORK
  RWORK is DOUBLE PRECISION array, dimension (N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

89.1.3 fortran code

— **zptcon.f** —

```
*  =====================================================================
      SUBROUTINE ZPTCON( N, D, E, ANORM, RCOND, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, N
      DOUBLE PRECISION   ANORM, RCOND
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), RWORK( * )
      COMPLEX*16         E( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IX
      DOUBLE PRECISION   AINVNM
*     ..
*     .. External Functions ..
      INTEGER            IDAMAX
```

```
      EXTERNAL           IDAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ANORM.LT.ZERO ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZPTCON', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      RCOND = ZERO
      IF( N.EQ.0 ) THEN
         RCOND = ONE
         RETURN
      ELSE IF( ANORM.EQ.ZERO ) THEN
         RETURN
      END IF
*
*     Check that D(1:N) is positive.
*
      DO 10 I = 1, N
         IF( D( I ).LE.ZERO )
     $      RETURN
   10 CONTINUE
*
*     Solve M(A) * x = e, where M(A) = (m(i,j)) is given by
*
*        m(i,j) =  abs(A(i,j)), i = j,
*        m(i,j) = -abs(A(i,j)), i .ne. j,
*
*     and e = [ 1, 1, ..., 1 ]**T.  Note M(A) = M(L)*D*M(L)**H.
*
*     Solve M(L) * x = e.
*
      RWORK( 1 ) = ONE
```

```
      DO 20 I = 2, N
         RWORK( I ) = ONE + RWORK( I-1 )*ABS( E( I-1 ) )
   20 CONTINUE
*
*     Solve D * M(L)**H * x = b.
*
      RWORK( N ) = RWORK( N ) / D( N )
      DO 30 I = N - 1, 1, -1
         RWORK( I ) = RWORK( I ) / D( I ) + RWORK( I+1 )*ABS( E( I ) )
   30 CONTINUE
*
*     Compute AINVNM = max(x(i)), 1<=i<=n.
*
      IX = IDAMAX( N, RWORK, 1 )
      AINVNM = ABS( RWORK( IX ) )
*
*     Compute the reciprocal condition number.
*
      IF( AINVNM.NE.ZERO )
     $   RCOND = ( ONE / AINVNM ) / ANORM
*
      RETURN
*
*     End of ZPTCON
*
      END
```

———-

89.1.4 lisp code

— **zptcon.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun zptcon (n d e anorm rcond rwork info)
    (declare (type (double-float) rcond anorm)
             (type (array f2cl-lib:complex16 (*)) e)
             (type (array double-float (*)) rwork d)
             (type (f2cl-lib:integer4) info n))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%)
         (e f2cl-lib:complex16 e-%data% e-%offset%))
      (prog ((ainvnm 0.0d0) (i 0) (ix 0))
        (declare (type (double-float) ainvnm)
```

```
         (type (f2cl-lib:integer4) i ix))
(setf info 0)
(cond ((< n 0) (setf info -1)) ((< anorm zero) (setf info -4)))
(cond
 ((/= info 0) (xerbla "ZPTCON" (f2cl-lib:int-sub info))
  (go end_label)))
(setf rcond zero)
(cond ((= n 0) (setf rcond one) (go end_label))
      ((= anorm zero) (go end_label)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (if (<= (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) zero)
        (go end_label))
   label10))
(setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%) one)
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
            (+ one
               (*
                (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-sub i 1))
                               ((1 *)) rwork-%offset%)
                (abs
                 (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                ((1 *)) e-%offset%)))))
   label20))
(setf (f2cl-lib:fref rwork-%data% (n) ((1 *)) rwork-%offset%)
        (/ (f2cl-lib:fref rwork-%data% (n) ((1 *)) rwork-%offset%)
           (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
(f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
            (+
             (/ (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
             (*
              (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add i 1))
                             ((1 *)) rwork-%offset%)
              (abs (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))))
   label30))
(setf ix
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (idamax n rwork 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
```

```
        (setf ainvnm
                (abs (f2cl-lib:fref rwork-%data% (ix) ((1 *)) rwork-%offset%)))
        (if (/= ainvnm zero)
            (setf rcond (/ (/ one ainvnm) anorm)))
        (go end_label)
       end_label
        (return (values n nil nil nil rcond nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zptcon
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (double-float) (double-float)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n nil nil nil
                                              fortran-to-lisp::rcond nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

89.2 zpteqr

89.2.1 Axiom unit tests

— **zpteqr.input** —

```
)set break resume
)sys rm -f zpteqr.output
)spool zpteqr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

89.2.2 Axiom help page

— zpteqr.help —

```
=====================================================================
zpteqr examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zpteqr (COMPZ, N, D, E, Z, LDZ, WORK, INFO)
           ZPTEQR

Function/Subroutine Documentation
   subroutine zpteqr (character COMPZ, integer N, double precision, dimension(
       * ) D, double precision, dimension( * ) E, complex*16, dimension( ldz,
       * ) Z, integer LDZ, double precision, dimension( * ) WORK, integer
       INFO)
       ZPTEQR

Purpose:
   ZPTEQR computes all eigenvalues and, optionally, eigenvectors of a
   symmetric positive definite tridiagonal matrix by first factoring the
   matrix using DPTTRF and then calling ZBDSQR to compute the singular
   values of the bidiagonal factor.

   This routine computes the eigenvalues of the positive definite
   tridiagonal matrix to high relative accuracy.  This means that if the
   eigenvalues range over many orders of magnitude in size, then the
   small eigenvalues and corresponding eigenvectors will be computed
   more accurately than, for example, with the standard QR method.

   The eigenvectors of a full or band positive definite Hermitian matrix
   can also be found if ZHETRD, ZHPTRD, or ZHBTRD has been used to
   reduce this matrix to tridiagonal form.  (The reduction to
   tridiagonal form, however, may preclude the possibility of obtaining
   high relative accuracy in the small eigenvalues of the original
   matrix, if these eigenvalues range over many orders of magnitude.)

Parameters:

 COMPZ
```

```
  COMPZ is CHARACTER*1
  = 'N':  Compute eigenvalues only.
  = 'V':  Compute eigenvectors of original Hermitian
     matrix also.  Array Z contains the unitary matrix
     used to reduce the original matrix to tridiagonal form.
  = 'I':  Compute eigenvectors of tridiagonal matrix also.

N
  N is INTEGER
  The order of the matrix.  N >= 0.

D
  D is DOUBLE PRECISION array, dimension (N)
  On entry, the n diagonal elements of the tridiagonal matrix.
  On normal exit, D contains the eigenvalues, in descending order.

E
  E is DOUBLE PRECISION array, dimension (N-1)
  On entry, the (n-1) subdiagonal elements of the tridiagonal matrix.
  On exit, E has been destroyed.

Z
  Z is COMPLEX*16 array, dimension (LDZ, N)
  On entry, if COMPZ = 'V', the unitary matrix used in the
  reduction to tridiagonal form.
  On exit, if COMPZ = 'V', the orthonormal eigenvectors of the
  original Hermitian matrix;
  if COMPZ = 'I', the orthonormal eigenvectors of the tridiagonal matrix.
  If INFO > 0 on exit, Z contains the eigenvectors associated
  with only the stored eigenvalues.
  If  COMPZ = 'N', then Z is not referenced.

LDZ
  LDZ is INTEGER
  The leading dimension of the array Z.  LDZ >= 1, and if
  COMPZ = 'V' or 'I', LDZ >= max(1,N).

WORK
  WORK is DOUBLE PRECISION array, dimension (4*N)

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if INFO = i, and i is:
  <= N  the Cholesky factorization of the matrix could
     not be performed because the i-th principal minor
     was not positive definite.
  > N   the SVD algorithm failed to converge;
     if INFO = N+i, i off-diagonal elements of the
```

```
      bidiagonal factor did not converge to zero.
```

———-

89.2.3 fortran code

— **zpteqr.f** —

```
*  =====================================================================
      SUBROUTINE ZPTEQR( COMPZ, N, D, E, Z, LDZ, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          COMPZ
      INTEGER            INFO, LDZ, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), E( * ), WORK( * )
      COMPLEX*16         Z( LDZ, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0D+0, 0.0D+0 ),
     $                   CONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DPTTRF, XERBLA, ZBDSQR, ZLASET
*     ..
*     .. Local Arrays ..
      COMPLEX*16         C( 1, 1 ), VT( 1, 1 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, ICOMPZ, NRU
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
```

```
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( LSAME( COMPZ, 'N' ) ) THEN
         ICOMPZ = 0
      ELSE IF( LSAME( COMPZ, 'V' ) ) THEN
         ICOMPZ = 1
      ELSE IF( LSAME( COMPZ, 'I' ) ) THEN
         ICOMPZ = 2
      ELSE
         ICOMPZ = -1
      END IF
      IF( ICOMPZ.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( ( LDZ.LT.1 ) .OR. ( ICOMPZ.GT.0 .AND. LDZ.LT.MAX( 1,
     $         N ) ) ) THEN
         INFO = -6
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZPTEQR', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
      IF( N.EQ.1 ) THEN
         IF( ICOMPZ.GT.0 )
     $      Z( 1, 1 ) = CONE
         RETURN
      END IF
      IF( ICOMPZ.EQ.2 )
     $   CALL ZLASET( 'Full', N, N, CZERO, CONE, Z, LDZ )
*
*     Call DPTTRF to factor the matrix.
*
      CALL DPTTRF( N, D, E, INFO )
      IF( INFO.NE.0 )
     $   RETURN
      DO 10 I = 1, N
         D( I ) = SQRT( D( I ) )
   10 CONTINUE
```

```
      DO 20 I = 1, N - 1
         E( I ) = E( I )*D( I )
   20 CONTINUE
*
*     Call ZBDSQR to compute the singular values/vectors of the
*     bidiagonal factor.
*
      IF( ICOMPZ.GT.0 ) THEN
         NRU = N
      ELSE
         NRU = 0
      END IF
      CALL ZBDSQR( 'Lower', N, 0, NRU, 0, D, E, VT, 1, Z, LDZ, C, 1,
     $             WORK, INFO )
*
*     Square the singular values.
*
      IF( INFO.EQ.0 ) THEN
         DO 30 I = 1, N
            D( I ) = D( I )*D( I )
   30    CONTINUE
      ELSE
         INFO = N + INFO
      END IF
*
      RETURN
*
*     End of ZPTEQR
*
      END
```

———-

89.2.4 lisp code

— **zpteqr.lisp** —

```
(let* ((czero (f2cl-lib:cmplx 0.0d0 0.0d0)) (cone (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) czero)
           (type (f2cl-lib:complex16) cone)
           (ignorable czero cone))
  (defun zpteqr (compz n d e z ldz work info)
    (declare (type (array f2cl-lib:complex16 (*)) z)
             (type (array double-float (*)) work e d)
             (type (f2cl-lib:integer4) info ldz n)
             (type (string *) compz))
    (f2cl-lib:with-multi-array-data
        ((compz character compz-%data% compz-%offset%)
```

```
   (d double-float d-%data% d-%offset%)
   (e double-float e-%data% e-%offset%)
   (work double-float work-%data% work-%offset%)
   (z f2cl-lib:complex16 z-%data% z-%offset%))
(prog ((i 0) (icompz 0) (nru 0)
       (c (make-array 1 :element-type 'f2cl-lib:complex16))
       (vt (make-array 1 :element-type 'f2cl-lib:complex16)))
  (declare (type (f2cl-lib:integer4) i icompz nru)
           (type (array f2cl-lib:complex16 (1)) c vt))
  (setf info 0)
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame compz "N")
      (declare (ignore var-1))
      (when var-0 (setf compz var-0))
      ret-val)
    (setf icompz 0))
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame compz "V")
      (declare (ignore var-1))
      (when var-0 (setf compz var-0))
      ret-val)
    (setf icompz 1))
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame compz "I")
      (declare (ignore var-1))
      (when var-0 (setf compz var-0))
      ret-val)
    (setf icompz 2))
   (t (setf icompz -1)))
  (cond ((< icompz 0) (setf info -1)) ((< n 0) (setf info -2))
        ((or (< ldz 1)
             (and (> icompz 0)
                  (< ldz
                     (max (the f2cl-lib:integer4 1)
                          (the f2cl-lib:integer4 n)))))
         (setf info -6)))
  (cond
   ((/= info 0) (xerbla "ZPTEQR" (f2cl-lib:int-sub info))
    (go end_label)))
  (if (= n 0)
      (go end_label))
  (cond
   ((= n 1)
    (if (> icompz 0)
        (setf (f2cl-lib:fref z-%data% (1 1) ((1 ldz) (1 *)) z-%offset%)
                cone))
    (go end_label)))
  (if (= icompz 2)
      (zlaset "Full" n n czero cone z ldz))
```

```
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dpttrf n d e info)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          (when var-3 (setf info var-3)))
        (if (/= info 0)
            (go end_label))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                    (f2cl-lib:fsqrt
                     (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
           label10))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
          (tagbody
            (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                    (* (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                       (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
           label20))
        (cond ((> icompz 0) (setf nru n)) (t (setf nru 0)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
             var-11 var-12 var-13 var-14)
            (zbdsqr "Lower" n 0 nru 0 d e vt 1 z ldz c 1 work info)
          (declare
           (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9 var-11
            var-12 var-13))
          (setf n var-1)
          (setf nru var-3)
          (setf ldz var-10)
          (setf info var-14))
        (cond
         ((= info 0)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                      (* (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                         (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
             label30)))
         (t (setf info (f2cl-lib:int-add n info))))
        (go end_label)
       end_label
        (return (values compz n nil nil nil ldz nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
```

```
(setf (gethash 'fortran-to-lisp::zpteqr
               fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((string)
                                            (fortran-to-lisp::integer4)
                                            (array double-float (*))
                                            (array double-float (*))
                                            (array fortran-to-lisp::complex16
                                             (*))
                                            (fortran-to-lisp::integer4)
                                            (array double-float (*))
                                            (fortran-to-lisp::integer4))
                                          :return-values
                                          '(fortran-to-lisp::compz
                                            fortran-to-lisp::n nil nil nil
                                            fortran-to-lisp::ldz nil
                                            fortran-to-lisp::info)
                                          :calls
                                          '(fortran-to-lisp::zbdsqr
                                            fortran-to-lisp::zlaset
                                            fortran-to-lisp::xerbla))))
```

———-

89.3 zptrfs

89.3.1 Axiom unit tests

— **zptrfs.input** —

```
)set break resume
)sys rm -f zptrfs.output
)spool zptrfs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

89.3.2 Axiom help page

— zptrfs.help —

```
==================================================================
zptrfs examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zptrfs (UPLO, N, NRHS, D, E, DF, EF, B, LDB, X, LDX, FERR,
           BERR, WORK, RWORK, INFO)
           ZPTRFS

Function/Subroutine Documentation
   subroutine zptrfs (character UPLO, integer N, integer NRHS, double
       precision, dimension( * ) D, complex*16, dimension( * ) E, double
       precision, dimension( * ) DF, complex*16, dimension( * ) EF,
       complex*16, dimension( ldb, * ) B, integer LDB, complex*16, dimension(
       ldx, * ) X, integer LDX, double precision, dimension( * ) FERR, double
       precision, dimension( * ) BERR, complex*16, dimension( * ) WORK, double
       precision, dimension( * ) RWORK, integer INFO)
       ZPTRFS

Purpose:
   ZPTRFS improves the computed solution to a system of linear
   equations when the coefficient matrix is Hermitian positive definite
   and tridiagonal, and provides error bounds and backward error
   estimates for the solution.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies whether the superdiagonal or the subdiagonal of the
   tridiagonal matrix A is stored and the form of the factorization:
   = 'U':  E is the superdiagonal of A, and A = U**H*D*U;
   = 'L':  E is the subdiagonal of A, and A = L*D*L**H.
   (The two forms are equivalent if A is real.)

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 NRHS
```

```
  NRHS is INTEGER
  The number of right hand sides, i.e., the number of columns
  of the matrix B.  NRHS >= 0.

D
  D is DOUBLE PRECISION array, dimension (N)
  The n real diagonal elements of the tridiagonal matrix A.

E
  E is COMPLEX*16 array, dimension (N-1)
  The (n-1) off-diagonal elements of the tridiagonal matrix A (see UPLO).

DF
  DF is DOUBLE PRECISION array, dimension (N)
  The n diagonal elements of the diagonal matrix D from
  the factorization computed by ZPTTRF.

EF
  EF is COMPLEX*16 array, dimension (N-1)
  The (n-1) off-diagonal elements of the unit bidiagonal
  factor U or L from the factorization computed by ZPTTRF (see UPLO).

B
  B is COMPLEX*16 array, dimension (LDB,NRHS)
  The right hand side matrix B.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).

X
  X is COMPLEX*16 array, dimension (LDX,NRHS)
  On entry, the solution matrix X, as computed by ZPTTRS.
  On exit, the improved solution matrix X.

LDX
  LDX is INTEGER
  The leading dimension of the array X.  LDX >= max(1,N).

FERR
  FERR is DOUBLE PRECISION array, dimension (NRHS)
  The forward error bound for each solution vector
  X(j) (the j-th column of the solution matrix X).
  If XTRUE is the true solution corresponding to X(j), FERR(j)
  is an estimated upper bound for the magnitude of the largest
  element in (X(j) - XTRUE) divided by the magnitude of the
  largest element in X(j).

BERR
  BERR is DOUBLE PRECISION array, dimension (NRHS)
```

```
   The componentwise relative backward error of each solution
   vector X(j) (i.e., the smallest relative change in
   any element of A or B that makes X(j) an exact solution).

 WORK
   WORK is COMPLEX*16 array, dimension (N)

 RWORK
   RWORK is DOUBLE PRECISION array, dimension (N)

 INFO
   INFO is INTEGER
   = 0:  successful exit
   < 0:  if INFO = -i, the i-th argument had an illegal value

Internal Parameters:

   ITMAX is the maximum number of steps of iterative refinement.
```

89.3.3 fortran code

— zptrfs.f —

```
*  =====================================================================
      SUBROUTINE ZPTRFS( UPLO, N, NRHS, D, E, DF, EF, B, LDB, X, LDX,
     $                   FERR, BERR, WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDB, LDX, N, NRHS
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   BERR( * ), D( * ), DF( * ), FERR( * ),
     $                   RWORK( * )
      COMPLEX*16         B( LDB, * ), E( * ), EF( * ), WORK( * ),
     $                   X( LDX, * )
*     ..
*
*  =====================================================================
*
```

```
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D+0 )
      DOUBLE PRECISION   THREE
      PARAMETER          ( THREE = 3.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            COUNT, I, IX, J, NZ
      DOUBLE PRECISION   EPS, LSTRES, S, SAFE1, SAFE2, SAFMIN
      COMPLEX*16         BI, CX, DX, EX, ZDUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, IDAMAX, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZAXPY, ZPTTRS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DCMPLX, DCONJG, DIMAG, MAX
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -9
```

```
      ELSE IF( LDX.LT.MAX( 1, N ) ) THEN
         INFO = -11
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZPTRFS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 ) THEN
         DO 10 J = 1, NRHS
            FERR( J ) = ZERO
            BERR( J ) = ZERO
   10    CONTINUE
         RETURN
      END IF
*
*     NZ = maximum number of nonzero elements in each row of A, plus 1
*
      NZ = 4
      EPS = DLAMCH( 'Epsilon' )
      SAFMIN = DLAMCH( 'Safe minimum' )
      SAFE1 = NZ*SAFMIN
      SAFE2 = SAFE1 / EPS
*
*     Do for each right hand side
*
      DO 100 J = 1, NRHS
*
         COUNT = 1
         LSTRES = THREE
   20    CONTINUE
*
*        Loop until stopping criterion is satisfied.
*
*        Compute residual R = B - A * X.  Also compute
*        abs(A)*abs(x) + abs(b) for use in the backward error bound.
*
         IF( UPPER ) THEN
            IF( N.EQ.1 ) THEN
               BI = B( 1, J )
               DX = D( 1 )*X( 1, J )
               WORK( 1 ) = BI - DX
               RWORK( 1 ) = CABS1( BI ) + CABS1( DX )
            ELSE
               BI = B( 1, J )
               DX = D( 1 )*X( 1, J )
               EX = E( 1 )*X( 2, J )
               WORK( 1 ) = BI - DX - EX
```

```
            RWORK( 1 ) = CABS1( BI ) + CABS1( DX ) +
     $                   CABS1( E( 1 ) )*CABS1( X( 2, J ) )
            DO 30 I = 2, N - 1
               BI = B( I, J )
               CX = DCONJG( E( I-1 ) )*X( I-1, J )
               DX = D( I )*X( I, J )
               EX = E( I )*X( I+1, J )
               WORK( I ) = BI - CX - DX - EX
               RWORK( I ) = CABS1( BI ) +
     $                      CABS1( E( I-1 ) )*CABS1( X( I-1, J ) ) +
     $                      CABS1( DX ) + CABS1( E( I ) )*
     $                      CABS1( X( I+1, J ) )
   30       CONTINUE
            BI = B( N, J )
            CX = DCONJG( E( N-1 ) )*X( N-1, J )
            DX = D( N )*X( N, J )
            WORK( N ) = BI - CX - DX
            RWORK( N ) = CABS1( BI ) + CABS1( E( N-1 ) )*
     $                   CABS1( X( N-1, J ) ) + CABS1( DX )
         END IF
      ELSE
         IF( N.EQ.1 ) THEN
            BI = B( 1, J )
            DX = D( 1 )*X( 1, J )
            WORK( 1 ) = BI - DX
            RWORK( 1 ) = CABS1( BI ) + CABS1( DX )
         ELSE
            BI = B( 1, J )
            DX = D( 1 )*X( 1, J )
            EX = DCONJG( E( 1 ) )*X( 2, J )
            WORK( 1 ) = BI - DX - EX
            RWORK( 1 ) = CABS1( BI ) + CABS1( DX ) +
     $                   CABS1( E( 1 ) )*CABS1( X( 2, J ) )
            DO 40 I = 2, N - 1
               BI = B( I, J )
               CX = E( I-1 )*X( I-1, J )
               DX = D( I )*X( I, J )
               EX = DCONJG( E( I ) )*X( I+1, J )
               WORK( I ) = BI - CX - DX - EX
               RWORK( I ) = CABS1( BI ) +
     $                      CABS1( E( I-1 ) )*CABS1( X( I-1, J ) ) +
     $                      CABS1( DX ) + CABS1( E( I ) )*
     $                      CABS1( X( I+1, J ) )
   40       CONTINUE
            BI = B( N, J )
            CX = E( N-1 )*X( N-1, J )
            DX = D( N )*X( N, J )
            WORK( N ) = BI - CX - DX
            RWORK( N ) = CABS1( BI ) + CABS1( E( N-1 ) )*
     $                   CABS1( X( N-1, J ) ) + CABS1( DX )
```

```
         END IF
      END IF
*
*     Compute componentwise relative backward error from formula
*
*     max(i) ( abs(R(i)) / ( abs(A)*abs(X) + abs(B) )(i) )
*
*     where abs(Z) is the componentwise absolute value of the matrix
*     or vector Z.  If the i-th component of the denominator is less
*     than SAFE2, then SAFE1 is added to the i-th components of the
*     numerator and denominator before dividing.
*
      S = ZERO
      DO 50 I = 1, N
         IF( RWORK( I ).GT.SAFE2 ) THEN
            S = MAX( S, CABS1( WORK( I ) ) / RWORK( I ) )
         ELSE
            S = MAX( S, ( CABS1( WORK( I ) )+SAFE1 ) /
     $          ( RWORK( I )+SAFE1 ) )
         END IF
   50 CONTINUE
      BERR( J ) = S
*
*     Test stopping criterion. Continue iterating if
*        1) The residual BERR(J) is larger than machine epsilon, and
*        2) BERR(J) decreased by at least a factor of 2 during the
*           last iteration, and
*        3) At most ITMAX iterations tried.
*
      IF( BERR( J ).GT.EPS .AND. TWO*BERR( J ).LE.LSTRES .AND.
     $     COUNT.LE.ITMAX ) THEN
*
*        Update solution and try again.
*
         CALL ZPTTRS( UPLO, N, 1, DF, EF, WORK, N, INFO )
         CALL ZAXPY( N, DCMPLX( ONE ), WORK, 1, X( 1, J ), 1 )
         LSTRES = BERR( J )
         COUNT = COUNT + 1
         GO TO 20
      END IF
*
*     Bound error from formula
*
*     norm(X - XTRUE) / norm(X) .le. FERR =
*     norm( abs(inv(A))*
*        ( abs(R) + NZ*EPS*( abs(A)*abs(X)+abs(B) ))) / norm(X)
*
*     where
*       norm(Z) is the magnitude of the largest component of Z
*       inv(A) is the inverse of A
```

```
*          abs(Z) is the componentwise absolute value of the matrix or
*             vector Z
*          NZ is the maximum number of nonzeros in any row of A, plus 1
*          EPS is machine epsilon
*
*        The i-th component of abs(R)+NZ*EPS*(abs(A)*abs(X)+abs(B))
*        is incremented by SAFE1 if the i-th component of
*        abs(A)*abs(X) + abs(B) is less than SAFE2.
*
         DO 60 I = 1, N
            IF( RWORK( I ).GT.SAFE2 ) THEN
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I )
            ELSE
               RWORK( I ) = CABS1( WORK( I ) ) + NZ*EPS*RWORK( I ) +
     $                      SAFE1
            END IF
   60    CONTINUE
         IX = IDAMAX( N, RWORK, 1 )
         FERR( J ) = RWORK( IX )
*
*        Estimate the norm of inv(A).
*
*        Solve M(A) * x = e, where M(A) = (m(i,j)) is given by
*
*           m(i,j) =  abs(A(i,j)), i = j,
*           m(i,j) = -abs(A(i,j)), i .ne. j,
*
*        and e = [ 1, 1, ..., 1 ]**T.  Note M(A) = M(L)*D*M(L)**H.
*
*        Solve M(L) * x = e.
*
         RWORK( 1 ) = ONE
         DO 70 I = 2, N
            RWORK( I ) = ONE + RWORK( I-1 )*ABS( EF( I-1 ) )
   70    CONTINUE
*
*        Solve D * M(L)**H * x = b.
*
         RWORK( N ) = RWORK( N ) / DF( N )
         DO 80 I = N - 1, 1, -1
            RWORK( I ) = RWORK( I ) / DF( I ) +
     $                   RWORK( I+1 )*ABS( EF( I ) )
   80    CONTINUE
*
*        Compute norm(inv(A)) = max(x(i)), 1<=i<=n.
*
         IX = IDAMAX( N, RWORK, 1 )
         FERR( J ) = FERR( J )*ABS( RWORK( IX ) )
*
*        Normalize error.
```

```
*
         LSTRES = ZERO
         DO 90 I = 1, N
            LSTRES = MAX( LSTRES, ABS( X( I, J ) ) )
   90    CONTINUE
         IF( LSTRES.NE.ZERO )
     $      FERR( J ) = FERR( J ) / LSTRES
*
  100 CONTINUE
*
      RETURN
*
*     End of ZPTRFS
*
      END
```

———-

89.3.4 lisp code

— **zptrfs.lisp** —

```
(let* ((itmax 5) (zero 0.0d0) (one 1.0d0) (two 2.0d0) (three 3.0d0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 3.0d0 3.0d0) three)
           (ignorable itmax zero one two three))
  (defun zptrfs (uplo n nrhs d e df ef b ldb$ x ldx ferr berr work rwork info)
    (declare (type (array f2cl-lib:complex16 (*)) work x b ef e)
             (type (array double-float (*)) rwork berr ferr df d)
             (type (f2cl-lib:integer4) info ldx ldb$ nrhs n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (d double-float d-%data% d-%offset%)
         (df double-float df-%data% df-%offset%)
         (ferr double-float ferr-%data% ferr-%offset%)
         (berr double-float berr-%data% berr-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%)
         (e f2cl-lib:complex16 e-%data% e-%offset%)
         (ef f2cl-lib:complex16 ef-%data% ef-%offset%)
         (b f2cl-lib:complex16 b-%data% b-%offset%)
         (x f2cl-lib:complex16 x-%data% x-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum)))))
```

```
(declare
 (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
  cabs1))
(prog ((bi #C(0.0d0 0.0d0)) (cx #C(0.0d0 0.0d0)) (dx #C(0.0d0 0.0d0))
       (ex #C(0.0d0 0.0d0)) (zdum #C(0.0d0 0.0d0)) (eps 0.0d0)
       (lstres 0.0d0) (s 0.0d0) (safe1 0.0d0) (safe2 0.0d0)
       (safmin 0.0d0) (count$ 0) (i 0) (ix 0) (j 0) (nz 0) (upper nil))
  (declare (type (f2cl-lib:complex16) bi cx dx ex zdum)
           (type (double-float) eps lstres s safe1 safe2 safmin)
           (type (f2cl-lib:integer4) count$ i ix j nz)
           (type f2cl-lib:logical upper))
  (setf info 0)
  (setf upper
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
  (cond
   ((and (not upper)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
   ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -9))
   ((< ldx (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -11)))
  (cond
   ((/= info 0) (xerbla "ZPTRFS" (f2cl-lib:int-sub info))
    (go end_label)))
  (cond
   ((or (= n 0) (= nrhs 0))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nrhs) nil)
      (tagbody
        (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                zero)
        (setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%)
                zero)
      label10))
    (go end_label)))
  (setf nz 4)
  (setf eps (dlamch "Epsilon"))
  (setf safmin (dlamch "Safe minimum"))
  (setf safe1 (* nz safmin))
```

```
(setf safe2 (/ safe1 eps))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nrhs) nil)
  (tagbody
    (setf count$ 1)
    (setf lstres three)
   label20
    (cond
     (upper
      (cond
       ((= n 1)
        (setf bi
                (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                               b-%offset%))
        (setf dx
                (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
                   (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                                  x-%offset%)))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (- bi dx))
        (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
                (+ (cabs1 bi) (cabs1 dx))))
       (t
        (setf bi
                (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                               b-%offset%))
        (setf dx
                (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
                   (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                                  x-%offset%)))
        (setf ex
                (* (f2cl-lib:fref e-%data% (1) ((1 *)) e-%offset%)
                   (f2cl-lib:fref x-%data% (2 j) ((1 ldx) (1 *))
                                  x-%offset%)))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (- bi dx ex))
        (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
                (+ (cabs1 bi) (cabs1 dx)
                   (*
                    (cabs1
                     (f2cl-lib:fref e-%data% (1) ((1 *)) e-%offset%))
                    (cabs1
                     (f2cl-lib:fref x-%data% (2 j) ((1 ldx) (1 *))
                                    x-%offset%)))))
        (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                      ((> i
                          (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (setf bi
```

```
                (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%))
        (setf cx
                (*
                 (f2cl-lib:dconjg
                  (f2cl-lib:fref e-%data%
                                 ((f2cl-lib:int-sub i 1)) ((1 *))
                                 e-%offset%))
                 (f2cl-lib:fref x-%data%
                                ((f2cl-lib:int-sub i 1) j)
                                ((1 ldx) (1 *)) x-%offset%)))
        (setf dx
                (*
                 (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                 (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                x-%offset%)))
        (setf ex
                (*
                 (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                 (f2cl-lib:fref x-%data%
                                ((f2cl-lib:int-add i 1) j)
                                ((1 ldx) (1 *)) x-%offset%)))
        (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                             work-%offset%)
                (- bi cx dx ex))
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                             rwork-%offset%)
                (+ (cabs1 bi)
                   (*
                    (cabs1
                     (f2cl-lib:fref e-%data%
                                    ((f2cl-lib:int-sub i 1))
                                    ((1 *)) e-%offset%))
                    (cabs1
                     (f2cl-lib:fref x-%data%
                                    ((f2cl-lib:int-sub i 1) j)
                                    ((1 ldx) (1 *)) x-%offset%)))
                   (cabs1 dx)
                   (*
                    (cabs1
                     (f2cl-lib:fref e-%data% (i) ((1 *))
                                    e-%offset%))
                    (cabs1
                     (f2cl-lib:fref x-%data%
                                    ((f2cl-lib:int-add i 1) j)
                                    ((1 ldx) (1 *))
                                    x-%offset%)))))
   label30))
(setf bi
        (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
```

```
                          b-%offset%))
    (setf cx
            (*
             (f2cl-lib:dconjg
              (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub n 1))
                             ((1 *)) e-%offset%))
             (f2cl-lib:fref x-%data% ((f2cl-lib:int-sub n 1) j)
                            ((1 ldx) (1 *)) x-%offset%)))
    (setf dx
            (* (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
               (f2cl-lib:fref x-%data% (n j) ((1 ldx) (1 *))
                              x-%offset%)))
    (setf (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
            (- bi cx dx))
    (setf (f2cl-lib:fref rwork-%data% (n) ((1 *)) rwork-%offset%)
            (+ (cabs1 bi)
               (*
                (cabs1
                 (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub n 1))
                                ((1 *)) e-%offset%))
                (cabs1
                 (f2cl-lib:fref x-%data%
                                ((f2cl-lib:int-sub n 1) j)
                                ((1 ldx) (1 *)) x-%offset%)))
               (cabs1 dx))))))
(t
 (cond
  ((= n 1)
   (setf bi
           (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                          b-%offset%))
   (setf dx
           (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
              (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                             x-%offset%)))
   (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
           (- bi dx))
   (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
           (+ (cabs1 bi) (cabs1 dx))))
  (t
   (setf bi
           (f2cl-lib:fref b-%data% (1 j) ((1 ldb$) (1 *))
                          b-%offset%))
   (setf dx
           (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
              (f2cl-lib:fref x-%data% (1 j) ((1 ldx) (1 *))
                             x-%offset%)))
   (setf ex
           (*
            (f2cl-lib:dconjg
```

```
          (f2cl-lib:fref e-%data% (1) ((1 *)) e-%offset%))
         (f2cl-lib:fref x-%data% (2 j) ((1 ldx) (1 *))
                        x-%offset%)))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (- bi dx ex))
(setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
        (+ (cabs1 bi) (cabs1 dx)
           (*
            (cabs1
             (f2cl-lib:fref e-%data% (1) ((1 *)) e-%offset%))
            (cabs1
             (f2cl-lib:fref x-%data% (2 j) ((1 ldx) (1 *))
                            x-%offset%)))))
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i
                  (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf bi
            (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                           b-%offset%))
    (setf cx
            (*
             (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                            ((1 *)) e-%offset%)
             (f2cl-lib:fref x-%data%
                            ((f2cl-lib:int-sub i 1) j)
                            ((1 ldx) (1 *)) x-%offset%)))
    (setf dx
            (*
             (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
             (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                            x-%offset%)))
    (setf ex
            (*
             (f2cl-lib:dconjg
              (f2cl-lib:fref e-%data% (i) ((1 *))
                             e-%offset%))
             (f2cl-lib:fref x-%data%
                            ((f2cl-lib:int-add i 1) j)
                            ((1 ldx) (1 *)) x-%offset%)))
    (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                         work-%offset%)
            (- bi cx dx ex))
    (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                         rwork-%offset%)
            (+ (cabs1 bi)
               (*
                (cabs1
                 (f2cl-lib:fref e-%data%
```

```
                                        ((f2cl-lib:int-sub i 1))
                                        ((1 *)) e-%offset%))
                         (cabs1
                          (f2cl-lib:fref x-%data%
                                        ((f2cl-lib:int-sub i 1) j)
                                        ((1 ldx) (1 *)) x-%offset%)))
                        (cabs1 dx)
                        (*
                         (cabs1
                          (f2cl-lib:fref e-%data% (i) ((1 *))
                                        e-%offset%))
                         (cabs1
                          (f2cl-lib:fref x-%data%
                                        ((f2cl-lib:int-add i 1) j)
                                        ((1 ldx) (1 *))
                                        x-%offset%)))))
        label40))
    (setf bi
            (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                           b-%offset%))
    (setf cx
            (*
             (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub n 1))
                            ((1 *)) e-%offset%)
             (f2cl-lib:fref x-%data% ((f2cl-lib:int-sub n 1) j)
                            ((1 ldx) (1 *)) x-%offset%)))
    (setf dx
            (* (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
               (f2cl-lib:fref x-%data% (n j) ((1 ldx) (1 *))
                              x-%offset%)))
    (setf (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
            (- bi cx dx))
    (setf (f2cl-lib:fref rwork-%data% (n) ((1 *)) rwork-%offset%)
            (+ (cabs1 bi)
               (*
                (cabs1
                 (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub n 1))
                                ((1 *)) e-%offset%))
                (cabs1
                 (f2cl-lib:fref x-%data%
                                ((f2cl-lib:int-sub n 1) j)
                                ((1 ldx) (1 *)) x-%offset%)))
               (cabs1 dx)))))))
(setf s zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
      (setf s
```

```
                (max s
                     (/
                      (cabs1
                       (f2cl-lib:fref work-%data% (i) ((1 *))
                                      work-%offset%))
                      (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                     rwork-%offset%)))))
      (t
       (setf s
                (max s
                     (/
                      (+
                       (cabs1
                        (f2cl-lib:fref work-%data% (i) ((1 *))
                                       work-%offset%))
                       safe1)
                      (+
                       (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                      rwork-%offset%)
                       safe1))))))
   label50))
(setf (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%) s)
(cond
 ((and (> (f2cl-lib:fref berr (j) ((1 *))) eps)
       (<= (* two (f2cl-lib:fref berr (j) ((1 *)))) lstres)
       (<= count$ itmax))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (zpttrs uplo n 1 df ef work n info)
    (declare (ignore var-2 var-3 var-4 var-5))
    (when var-0 (setf uplo var-0))
    (when var-1 (setf n var-1))
    (when var-6 (setf n var-6))
    (when var-7 (setf info var-7)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
      (zaxpy n (f2cl-lib:dcmplx one) work 1
       (f2cl-lib:array-slice x-%data% f2cl-lib:complex16 (1 j)
                             ((1 ldx) (1 *)) x-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4 var-5))
    (when var-0 (setf n var-0)))
  (setf lstres
          (f2cl-lib:fref berr-%data% (j) ((1 *)) berr-%offset%))
  (setf count$ (f2cl-lib:int-add count$ 1)) (go label20)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:fref rwork (i) ((1 *))) safe2)
      (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
```

```
                              rwork-%offset%)
                 (+
                  (cabs1
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%))
                  (* nz eps
                     (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                    rwork-%offset%)))))
       (t
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *))
                              rwork-%offset%)
                 (+
                  (cabs1
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%))
                  (* nz eps
                     (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                    rwork-%offset%))
                  safe1))))
   label60))
(setf ix
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (idamax n rwork 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
        (f2cl-lib:fref rwork-%data% (ix) ((1 *)) rwork-%offset%))
(setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
        one)
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
            (+ one
               (*
                (f2cl-lib:fref rwork-%data%
                               ((f2cl-lib:int-sub i 1)) ((1 *))
                               rwork-%offset%)
                (abs
                 (f2cl-lib:fref ef-%data%
                                ((f2cl-lib:int-sub i 1)) ((1 *))
                                ef-%offset%)))))
   label70))
(setf (f2cl-lib:fref rwork-%data% (n) ((1 *)) rwork-%offset%)
        (/
         (f2cl-lib:fref rwork-%data% (n) ((1 *)) rwork-%offset%)
         (f2cl-lib:fref df-%data% (n) ((1 *)) df-%offset%)))
(f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
```

```
                    ((> i 1) nil)
          (tagbody
            (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%)
                    (+
                     (/
                      (f2cl-lib:fref rwork-%data% (i) ((1 *))
                                     rwork-%offset%)
                      (f2cl-lib:fref df-%data% (i) ((1 *)) df-%offset%))
                     (*
                      (f2cl-lib:fref rwork-%data%
                                     ((f2cl-lib:int-add i 1)) ((1 *))
                                     rwork-%offset%)
                      (abs
                       (f2cl-lib:fref ef-%data% (i) ((1 *))
                                      ef-%offset%)))))
           label80))
        (setf ix
                (multiple-value-bind (ret-val var-0 var-1 var-2)
                    (idamax n rwork 1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf n var-0))
                  ret-val))
        (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                (* (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                   (abs
                    (f2cl-lib:fref rwork-%data% (ix) ((1 *))
                                   rwork-%offset%))))
        (setf lstres zero)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf lstres
                    (max lstres
                         (abs
                          (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                         x-%offset%))))
           label90))
        (if (/= lstres zero)
            (setf (f2cl-lib:fref ferr-%data% (j) ((1 *)) ferr-%offset%)
                    (/
                     (f2cl-lib:fref ferr-%data% (j) ((1 *))
                                    ferr-%offset%)
                     lstres)))
      label100))
    (go end_label)
  end_label
    (return
     (values uplo n nil nil nil nil nil nil nil nil nil nil nil nil nil
             info)))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zptrfs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
                                              fortran-to-lisp::n nil nil nil
                                              nil nil nil nil nil nil nil nil
                                              nil nil fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

89.4 zpttrf

89.4.1 Axiom unit tests

— **zpttrf.input** —

```
)set break resume
)sys rm -f zpttrf.output
)spool zpttrf.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

89.4.2 Axiom help page

— **zpttrf.help** —

```
==========================================================================
zpttrf examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zpttrf (N, D, E, INFO)
           ZPTTRF

Function/Subroutine Documentation
   subroutine zpttrf (integer N, double precision, dimension( * ) D,
       complex*16, dimension( * ) E, integer INFO)
       ZPTTRF

Purpose:
   ZPTTRF computes the L*D*L**H factorization of a complex Hermitian
   positive definite tridiagonal matrix A.  The factorization may also
   be regarded as having the form A = U**H *D*U.

Parameters:

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.

 D
   D is DOUBLE PRECISION array, dimension (N)
   On entry, the n diagonal elements of the tridiagonal matrix
   A.  On exit, the n diagonal elements of the diagonal matrix
   D from the L*D*L**H factorization of A.
```

```
 E
   E is COMPLEX*16 array, dimension (N-1)
   On entry, the (n-1) subdiagonal elements of the tridiagonal
   matrix A.  On exit, the (n-1) subdiagonal elements of the
   unit bidiagonal factor L from the L*D*L**H factorization of A.
   E can also be regarded as the superdiagonal of the unit
   bidiagonal factor U from the U**H *D*U factorization of A.

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -k, the k-th argument had an illegal value
   > 0: if INFO = k, the leading minor of order k is not
      positive definite; if k < N, the factorization could not
      be completed, while if k = N, the factorization was
      completed, but D(N) <= 0.
```

———-

89.4.3 fortran code

— zpttrf.f —

```
*  =====================================================================
      SUBROUTINE ZPTTRF( N, D, E, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * )
      COMPLEX*16         E( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, I4
```

```
      DOUBLE PRECISION   EII, EIR, F, G
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, DCMPLX, DIMAG, MOD
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
         CALL XERBLA( 'ZPTTRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Compute the L*D*L**H (or U**H *D*U) factorization of A.
*
      I4 = MOD( N-1, 4 )
      DO 10 I = 1, I4
         IF( D( I ).LE.ZERO ) THEN
            INFO = I
            GO TO 30
         END IF
         EIR = DBLE( E( I ) )
         EII = DIMAG( E( I ) )
         F = EIR / D( I )
         G = EII / D( I )
         E( I ) = DCMPLX( F, G )
         D( I+1 ) = D( I+1 ) - F*EIR - G*EII
   10 CONTINUE
*
      DO 20 I = I4 + 1, N - 4, 4
*
*        Drop out of the loop if d(i) <= 0: the matrix is not positive
*        definite.
*
         IF( D( I ).LE.ZERO ) THEN
            INFO = I
            GO TO 30
         END IF
*
```

```
*        Solve for e(i) and d(i+1).
*
         EIR = DBLE( E( I ) )
         EII = DIMAG( E( I ) )
         F = EIR / D( I )
         G = EII / D( I )
         E( I ) = DCMPLX( F, G )
         D( I+1 ) = D( I+1 ) - F*EIR - G*EII
*
         IF( D( I+1 ).LE.ZERO ) THEN
            INFO = I + 1
            GO TO 30
         END IF
*
*        Solve for e(i+1) and d(i+2).
*
         EIR = DBLE( E( I+1 ) )
         EII = DIMAG( E( I+1 ) )
         F = EIR / D( I+1 )
         G = EII / D( I+1 )
         E( I+1 ) = DCMPLX( F, G )
         D( I+2 ) = D( I+2 ) - F*EIR - G*EII
*
         IF( D( I+2 ).LE.ZERO ) THEN
            INFO = I + 2
            GO TO 30
         END IF
*
*        Solve for e(i+2) and d(i+3).
*
         EIR = DBLE( E( I+2 ) )
         EII = DIMAG( E( I+2 ) )
         F = EIR / D( I+2 )
         G = EII / D( I+2 )
         E( I+2 ) = DCMPLX( F, G )
         D( I+3 ) = D( I+3 ) - F*EIR - G*EII
*
         IF( D( I+3 ).LE.ZERO ) THEN
            INFO = I + 3
            GO TO 30
         END IF
*
*        Solve for e(i+3) and d(i+4).
*
         EIR = DBLE( E( I+3 ) )
         EII = DIMAG( E( I+3 ) )
         F = EIR / D( I+3 )
         G = EII / D( I+3 )
         E( I+3 ) = DCMPLX( F, G )
         D( I+4 ) = D( I+4 ) - F*EIR - G*EII
```

```
   20 CONTINUE
*
*     Check d(n) for positive definiteness.
*
      IF( D( N ).LE.ZERO )
     $   INFO = N
*
   30 CONTINUE
      RETURN
*
*     End of ZPTTRF
*
      END
```

———-

89.4.4 lisp code

— **zpttrf.lisp** —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun zpttrf (n d e info)
    (declare (type (array f2cl-lib:complex16 (*)) e)
             (type (array double-float (*)) d)
             (type (f2cl-lib:integer4) info n))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (e f2cl-lib:complex16 e-%data% e-%offset%))
      (prog ((eii 0.0d0) (eir 0.0d0) (f 0.0d0) (g 0.0d0) (i 0) (i4 0))
        (declare (type (double-float) eii eir f g)
                 (type (f2cl-lib:integer4) i i4))
        (setf info 0)
        (cond
         ((< n 0) (setf info -1) (xerbla "ZPTTRF" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (= n 0)
            (go end_label))
        (setf i4 (mod (f2cl-lib:int-sub n 1) 4))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i i4) nil)
          (tagbody
            (cond
             ((<= (f2cl-lib:fref d (i) ((1 *))) zero) (setf info i)
              (go label30)))
            (setf eir
                    (f2cl-lib:dble
```

```
                 (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
        (setf eii
                (f2cl-lib:dimag
                 (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
        (setf f (/ eir (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
        (setf g (/ eii (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
        (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                (f2cl-lib:dcmplx f g))
        (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                              d-%offset%)
                (-
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                d-%offset%)
                 (* f eir) (* g eii)))
      label10))
    (f2cl-lib:fdo (i (f2cl-lib:int-add i4 1) (f2cl-lib:int-add i 4))
                  ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 4))) nil)
      (tagbody
        (cond
         ((<= (f2cl-lib:fref d (i) ((1 *))) zero) (setf info i)
          (go label30)))
        (setf eir
                (f2cl-lib:dble
                 (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
        (setf eii
                (f2cl-lib:dimag
                 (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
        (setf f (/ eir (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
        (setf g (/ eii (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
        (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                (f2cl-lib:dcmplx f g))
        (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                              d-%offset%)
                (-
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                d-%offset%)
                 (* f eir) (* g eii)))
        (cond
         ((<= (f2cl-lib:fref d ((f2cl-lib:int-add i 1)) ((1 *))) zero)
          (setf info (f2cl-lib:int-add i 1)) (go label30)))
        (setf eir
                (f2cl-lib:dble
                 (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                e-%offset%)))
        (setf eii
                (f2cl-lib:dimag
                 (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                                e-%offset%)))
        (setf f
                (/ eir
```

```
            (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                           d-%offset%)))
(setf g
        (/ eii
           (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                           d-%offset%)))
(setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                      e-%offset%)
        (f2cl-lib:dcmplx f g))
(setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                      d-%offset%)
        (-
         (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                        d-%offset%)
         (* f eir) (* g eii)))
(cond
 ((<= (f2cl-lib:fref d ((f2cl-lib:int-add i 2)) ((1 *))) zero)
  (setf info (f2cl-lib:int-add i 2)) (go label30)))
(setf eir
        (f2cl-lib:dble
         (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                        e-%offset%)))
(setf eii
        (f2cl-lib:dimag
         (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                        e-%offset%)))
(setf f
        (/ eir
           (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                           d-%offset%)))
(setf g
        (/ eii
           (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                           d-%offset%)))
(setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 2)) ((1 *))
                      e-%offset%)
        (f2cl-lib:dcmplx f g))
(setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                      d-%offset%)
        (-
         (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                        d-%offset%)
         (* f eir) (* g eii)))
(cond
 ((<= (f2cl-lib:fref d ((f2cl-lib:int-add i 3)) ((1 *))) zero)
  (setf info (f2cl-lib:int-add i 3)) (go label30)))
(setf eir
        (f2cl-lib:dble
         (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                        e-%offset%)))
```

```
            (setf eii
                    (f2cl-lib:dimag
                     (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                                    e-%offset%)))
            (setf f
                    (/ eir
                       (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                                      d-%offset%)))
            (setf g
                    (/ eii
                       (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                                      d-%offset%)))
            (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 3)) ((1 *))
                                 e-%offset%)
                    (f2cl-lib:dcmplx f g))
            (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 4)) ((1 *))
                                 d-%offset%)
                    (-
                     (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 4)) ((1 *))
                                    d-%offset%)
                     (* f eir) (* g eii)))
           label20))
         (if (<= (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) zero)
             (setf info n))
        label30
         (go end_label)
        end_label
         (return (values nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zpttrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil
                                              fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::xerbla))))
```

———-

89.5 zpttrs

89.5.1 Axiom unit tests

— **zpttrs.input** —

```
)set break resume
)sys rm -f zpttrs.output
)spool zpttrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

89.5.2 Axiom help page

— **zpttrs.help** —

```
======================================================================
zpttrs examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine zpttrs (UPLO, N, NRHS, D, E, B, LDB, INFO)
           ZPTTRS

Function/Subroutine Documentation
   subroutine zpttrs (character UPLO, integer N, integer NRHS, double
       precision, dimension( * ) D, complex*16, dimension( * ) E, complex*16,
       dimension( ldb, * ) B, integer LDB, integer INFO)
       ZPTTRS

Purpose:
   ZPTTRS solves a tridiagonal system of the form
      A * X = B
   using the factorization A = U**H *D* U or A = L*D*L**H computed by ZPTTRF.
   D is a diagonal matrix specified in the vector D, U (or L) is a unit
```

```
  bidiagonal matrix whose superdiagonal (subdiagonal) is specified in
  the vector E, and X and B are N by NRHS matrices.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies the form of the factorization and whether the
   vector E is the superdiagonal of the upper bidiagonal factor
   U or the subdiagonal of the lower bidiagonal factor L.
   = 'U':  A = U**H *D*U, E is the superdiagonal of U
   = 'L':  A = L*D*L**H, E is the subdiagonal of L

 N
   N is INTEGER
   The order of the tridiagonal matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 D
   D is DOUBLE PRECISION array, dimension (N)
   The n diagonal elements of the diagonal matrix D from the
   factorization A = U**H *D*U or A = L*D*L**H.

 E
   E is COMPLEX*16 array, dimension (N-1)
   If UPLO = 'U', the (n-1) superdiagonal elements of the unit
   bidiagonal factor U from the factorization A = U**H*D*U.
   If UPLO = 'L', the (n-1) subdiagonal elements of the unit
   bidiagonal factor L from the factorization A = L*D*L**H.

 B
   B is DOUBLE PRECISION array, dimension (LDB,NRHS)
   On entry, the right hand side vectors B for the system of linear equations.
   On exit, the solution vectors, X.

 LDB
   LDB is INTEGER
   The leading dimension of the array B.  LDB >= max(1,N).

 INFO
   INFO is INTEGER
   = 0: successful exit
   < 0: if INFO = -k, the k-th argument had an illegal value
```

89.5.3 fortran code

— zpttrs.f —

```
*  =====================================================================
      SUBROUTINE ZPTTRS( UPLO, N, NRHS, D, E, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * )
      COMPLEX*16         B( LDB, * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            UPPER
      INTEGER            IUPLO, J, JB, NB
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZPTTS2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments.
*
      INFO = 0
      UPPER = ( UPLO.EQ.'U' .OR. UPLO.EQ.'u' )
      IF( .NOT.UPPER .AND. .NOT.( UPLO.EQ.'L' .OR. UPLO.EQ.'l' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
```

```
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZPTTRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
*     Determine the number of right-hand sides to solve at a time.
*
      IF( NRHS.EQ.1 ) THEN
         NB = 1
      ELSE
         NB = MAX( 1, ILAENV( 1, 'ZPTTRS', UPLO, N, NRHS, -1, -1 ) )
      END IF
*
*     Decode UPLO
*
      IF( UPPER ) THEN
         IUPLO = 1
      ELSE
         IUPLO = 0
      END IF
*
      IF( NB.GE.NRHS ) THEN
         CALL ZPTTS2( IUPLO, N, NRHS, D, E, B, LDB )
      ELSE
         DO 10 J = 1, NRHS, NB
            JB = MIN( NRHS-J+1, NB )
            CALL ZPTTS2( IUPLO, N, JB, D, E, B( 1, J ), LDB )
   10    CONTINUE
      END IF
*
      RETURN
*
*     End of ZPTTRS
*
      END
```

———-

89.5.4 lisp code

— zpttrs.lisp —

```
(defun zpttrs (uplo n nrhs d e b ldb$ info)
  (declare (type (array f2cl-lib:complex16 (*)) b e)
           (type (array double-float (*)) d)
           (type (f2cl-lib:integer4) info ldb$ nrhs n)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (d double-float d-%data% d-%offset%)
       (e f2cl-lib:complex16 e-%data% e-%offset%)
       (b f2cl-lib:complex16 b-%data% b-%offset%))
    (prog ((iuplo 0) (j 0) (jb 0) (nb 0) (upper nil))
      (declare (type f2cl-lib:logical upper)
               (type (f2cl-lib:integer4) nb jb j iuplo))
      (setf info 0)
      (setf upper
              (or (f2cl-lib:fstring-= uplo "U") (f2cl-lib:fstring-= uplo "u")))
      (cond
       ((and (not upper)
             (not
              (or (f2cl-lib:fstring-= uplo "L")
                  (f2cl-lib:fstring-= uplo "l"))))
        (setf info -1))
       ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
       ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -7)))
      (cond
       ((/= info 0) (xerbla "ZPTTRS" (f2cl-lib:int-sub info)) (go end_label)))
      (if (or (= n 0) (= nrhs 0))
          (go end_label))
      (cond ((= nrhs 1) (setf nb 1))
            (t
             (setf nb
                     (max (the f2cl-lib:integer4 1)
                          (the f2cl-lib:integer4
                               (multiple-value-bind
                                   (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                                    var-6)
                                   (ilaenv 1 "ZPTTRS" uplo n nrhs -1 -1)
                                 (declare (ignore var-0 var-1 var-5 var-6))
                                 (setf uplo var-2)
                                 (setf n var-3)
                                 (setf nrhs var-4)
                                 ret-val))))))
      (cond (upper (setf iuplo 1)) (t (setf iuplo 0)))
      (cond
```

```
       ((>= nb nrhs)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (zptts2 iuplo n nrhs d e b ldb$)
          (declare (ignore var-3 var-4 var-5))
          (when var-0 (setf iuplo var-0))
          (when var-1 (setf n var-1))
          (when var-2 (setf nrhs var-2))
          (when var-6 (setf ldb$ var-6))))
       (t
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                      ((> j nrhs) nil)
          (tagbody
            (setf jb
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-sub nrhs j) 1))
                     (the f2cl-lib:integer4 nb)))
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (zptts2 iuplo n jb d e
                 (f2cl-lib:array-slice b-%data% f2cl-lib:complex16 (1 j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                 ldb$)
              (declare (ignore var-3 var-4 var-5))
              (when var-0 (setf iuplo var-0))
              (when var-1 (setf n var-1))
              (when var-2 (setf jb var-2))
              (when var-6 (setf ldb$ var-6)))
           label10))))
      (go end_label)
     end_label
      (return (values uplo n nrhs nil nil nil ldb$ info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zpttrs
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (array fortran-to-lisp::complex16
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo
```

```
                                                  fortran-to-lisp::n
                                                  fortran-to-lisp::nrhs nil nil nil
                                                  fortran-to-lisp::ldb$
                                                  fortran-to-lisp::info)
                                                :calls
                                                '(fortran-to-lisp::ilaenv
                                                  fortran-to-lisp::xerbla))))
```

———-

89.6 zptts2

89.6.1 Axiom unit tests

— **zptts2.input** —

```
)set break resume
)sys rm -f zptts2.output
)spool zptts2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

89.6.2 Axiom help page

— **zptts2.help** —

```
====================================================================
zptts2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
```

```
      subroutine zptts2 (IUPLO, N, NRHS, D, E, B, LDB)
          ZPTTS2 solves a tridiagonal system of the form AX=B using the L D
          LH factorization computed by spttrf.

Function/Subroutine Documentation
   subroutine zptts2 (integer IUPLO, integer N, integer NRHS, double
       precision, dimension( * ) D, complex*16, dimension( * ) E, complex*16,
       dimension( ldb, * ) B, integer LDB)
       ZPTTS2 solves a tridiagonal system of the form AX=B using the L D LH
       factorization computed by spttrf.

Purpose:
   ZPTTS2 solves a tridiagonal system of the form
      A * X = B
   using the factorization A = U**H *D*U or A = L*D*L**H computed by ZPTTRF.
   D is a diagonal matrix specified in the vector D, U (or L) is a unit
   bidiagonal matrix whose superdiagonal (subdiagonal) is specified in
   the vector E, and X and B are N by NRHS matrices.

Parameters:

 IUPLO
   IUPLO is INTEGER
   Specifies the form of the factorization and whether the
   vector E is the superdiagonal of the upper bidiagonal factor
   U or the subdiagonal of the lower bidiagonal factor L.
   = 1:  A = U**H *D*U, E is the superdiagonal of U
   = 0:  A = L*D*L**H, E is the subdiagonal of L

 N
   N is INTEGER
   The order of the tridiagonal matrix A.  N >= 0.

 NRHS
   NRHS is INTEGER
   The number of right hand sides, i.e., the number of columns
   of the matrix B.  NRHS >= 0.

 D
   D is DOUBLE PRECISION array, dimension (N)
   The n diagonal elements of the diagonal matrix D from the
   factorization A = U**H *D*U or A = L*D*L**H.

 E
   E is COMPLEX*16 array, dimension (N-1)
   If IUPLO = 1, the (n-1) superdiagonal elements of the unit
   bidiagonal factor U from the factorization A = U**H*D*U.
   If IUPLO = 0, the (n-1) subdiagonal elements of the unit
   bidiagonal factor L from the factorization A = L*D*L**H.
```

```
B
  B is DOUBLE PRECISION array, dimension (LDB,NRHS)
  On entry, the right hand side vectors B for the system of
  linear equations.
  On exit, the solution vectors, X.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,N).
```

———-

89.6.3 fortran code

— **zptts2.f** —

```
*  =====================================================================
      SUBROUTINE ZPTTS2( IUPLO, N, NRHS, D, E, B, LDB )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            IUPLO, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * )
      COMPLEX*16         B( LDB, * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZDSCAL
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DCONJG
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
```

```
      IF( N.LE.1 ) THEN
         IF( N.EQ.1 )
     $      CALL ZDSCAL( NRHS, 1.D0 / D( 1 ), B, LDB )
         RETURN
      END IF
*
      IF( IUPLO.EQ.1 ) THEN
*
*        Solve A * X = B using the factorization A = U**H *D*U,
*        overwriting each right hand side vector with its solution.
*
         IF( NRHS.LE.2 ) THEN
            J = 1
   10       CONTINUE
*
*           Solve U**H * x = b.
*
            DO 20 I = 2, N
               B( I, J ) = B( I, J ) - B( I-1, J )*DCONJG( E( I-1 ) )
   20       CONTINUE
*
*           Solve D * U * x = b.
*
            DO 30 I = 1, N
               B( I, J ) = B( I, J ) / D( I )
   30       CONTINUE
            DO 40 I = N - 1, 1, -1
               B( I, J ) = B( I, J ) - B( I+1, J )*E( I )
   40       CONTINUE
            IF( J.LT.NRHS ) THEN
               J = J + 1
               GO TO 10
            END IF
         ELSE
            DO 70 J = 1, NRHS
*
               Solve U**H * x = b.
*
               DO 50 I = 2, N
                  B( I, J ) = B( I, J ) - B( I-1, J )*DCONJG( E( I-1 ) )
   50          CONTINUE
*
               Solve D * U * x = b.
*
               B( N, J ) = B( N, J ) / D( N )
               DO 60 I = N - 1, 1, -1
                  B( I, J ) = B( I, J ) / D( I ) - B( I+1, J )*E( I )
   60          CONTINUE
   70       CONTINUE
         END IF
```

```
      ELSE
*
*        Solve A * X = B using the factorization A = L*D*L**H,
*        overwriting each right hand side vector with its solution.
*
         IF( NRHS.LE.2 ) THEN
            J = 1
   80       CONTINUE
*
*           Solve L * x = b.
*
            DO 90 I = 2, N
               B( I, J ) = B( I, J ) - B( I-1, J )*E( I-1 )
   90       CONTINUE
*
*           Solve D * L**H * x = b.
*
            DO 100 I = 1, N
               B( I, J ) = B( I, J ) / D( I )
  100       CONTINUE
            DO 110 I = N - 1, 1, -1
               B( I, J ) = B( I, J ) - B( I+1, J )*DCONJG( E( I ) )
  110       CONTINUE
            IF( J.LT.NRHS ) THEN
               J = J + 1
               GO TO 80
            END IF
         ELSE
            DO 140 J = 1, NRHS
*
*              Solve L * x = b.
*
               DO 120 I = 2, N
                  B( I, J ) = B( I, J ) - B( I-1, J )*E( I-1 )
  120          CONTINUE
*
*              Solve D * L**H * x = b.
*
               B( N, J ) = B( N, J ) / D( N )
               DO 130 I = N - 1, 1, -1
                  B( I, J ) = B( I, J ) / D( I ) -
     $                        B( I+1, J )*DCONJG( E( I ) )
  130          CONTINUE
  140       CONTINUE
         END IF
      END IF
*
      RETURN
*
*     End of ZPTTS2
```

```
*
      END
```

———-

89.6.4 lisp code

— **zptts2.lisp** —

```
(defun zptts2 (iuplo n nrhs d e b ldb$)
  (declare (type (array f2cl-lib:complex16 (*)) b e)
           (type (array double-float (*)) d)
           (type (f2cl-lib:integer4) ldb$ nrhs n iuplo))
  (f2cl-lib:with-multi-array-data
      ((d double-float d-%data% d-%offset%)
       (e f2cl-lib:complex16 e-%data% e-%offset%)
       (b f2cl-lib:complex16 b-%data% b-%offset%))
    (prog ((i 0) (j 0))
      (declare (type (f2cl-lib:integer4) j i))
      (cond
       ((<= n 1)
        (if (= n 1)
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (zdscal nrhs
                 (/ 1.0d0 (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)) b
                 ldb$)
              (declare (ignore var-1 var-2))
              (when var-0 (setf nrhs var-0))
              (when var-3 (setf ldb$ var-3))))
        (go end_label)))
      (cond
       ((= iuplo 1)
        (cond
         ((<= nrhs 2)
          (tagbody
            (setf j 1)
           label10
            (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                     b-%offset%)
                      (-
                       (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                      b-%offset%)
                       (*
                        (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                       ((1 ldb$) (1 *)) b-%offset%)
```

```
                    (f2cl-lib:dconjg
                     (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                    ((1 *)) e-%offset%)))))
       label20))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)
              (/
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)
               (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
       label30))
    (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
      (tagbody
        (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                             b-%offset%)
              (-
               (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                              b-%offset%)
               (*
                (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                               ((1 ldb$) (1 *)) b-%offset%)
                (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))))
       label40))
    (cond ((< j nrhs) (setf j (f2cl-lib:int-add j 1)) (go label10)))))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j nrhs) nil)
    (tagbody
      (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)
                (-
                 (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                 (*
                  (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                  (f2cl-lib:dconjg
                   (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                  ((1 *)) e-%offset%)))))
         label50))
      (setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
            (/
```

```
                    (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                                   b-%offset%)
                    (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
          (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                         (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                        ((> i 1) nil)
            (tagbody
              (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)
                      (-
                       (/
                        (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                       b-%offset%)
                        (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
                       (*
                        (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                       ((1 ldb$) (1 *)) b-%offset%)
                        (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))))
             label60))
        label70)))))
(t
 (cond
  ((<= nrhs 2)
   (tagbody
     (setf j 1)
    label80
     (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)
                 (-
                  (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (*
                   (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                  ((1 ldb$) (1 *)) b-%offset%)
                   (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                  ((1 *)) e-%offset%))))
        label90))
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                               b-%offset%)
                 (/
                  (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
        label100))
```

```
      (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                     (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                    ((> i 1) nil)
        (tagbody
          (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                (-
                 (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                b-%offset%)
                 (*
                  (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                 ((1 ldb$) (1 *)) b-%offset%)
                  (f2cl-lib:dconjg
                   (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))))
         label110))
      (cond ((< j nrhs) (setf j (f2cl-lib:int-add j 1)) (go label80)))))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j nrhs) nil)
      (tagbody
        (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)
                  (-
                   (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)
                   (*
                    (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub i 1) j)
                                    ((1 ldb$) (1 *)) b-%offset%)
                    (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                    ((1 *)) e-%offset%))))
           label120))
        (setf (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *)) b-%offset%)
              (/
               (f2cl-lib:fref b-%data% (n j) ((1 ldb$) (1 *))
                              b-%offset%)
               (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
        (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                       (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                  b-%offset%)
                  (-
                   (/
                    (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)
                    (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
```

```
                          (*
                           (f2cl-lib:fref b-%data% ((f2cl-lib:int-add i 1) j)
                                          ((1 ldb$) (1 *)) b-%offset%)
                           (f2cl-lib:dconjg
                            (f2cl-lib:fref e-%data% (i) ((1 *))
                                           e-%offset%)))))
                label130))
            label140))))))
      (go end_label)
     end_label
      (return (values nil nil nrhs nil nil nil ldb$)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::zptts2
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (array double-float (*))
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (array fortran-to-lisp::complex16
                                              (*))
                                             (fortran-to-lisp::integer4))
                                           :return-values
                                           '(nil nil fortran-to-lisp::nrhs nil
                                             nil nil fortran-to-lisp::ldb$)
                                           :calls 'nil)))
```

———-

Chapter 90

LAPACK - Other Auxilliary Routines

90.1 disnan

90.1.1 Axiom unit tests

— **disnan.input** —

```
)set break resume
)sys rm -f disnan.output
)spool disnan.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.1.2 Axiom help page

— **disnan.help** —

```
==========================================================================
disnan examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       logical function disnan (DIN)
           DISNAN tests input for NaN.

Function/Subroutine Documentation
   logical function disnan (double precision DIN)
       DISNAN tests input for NaN.

Purpose:
   DISNAN returns .TRUE. if its argument is NaN, and .FALSE.
   otherwise.  To be replaced by the Fortran 2003 intrinsic in the future.

Parameters:

 DIN
   DIN is DOUBLE PRECISION
   Input to test for NaN.
```

———-

90.1.3 fortran code

— **disnan.f** —

```
*  =====================================================================
      LOGICAL FUNCTION DISNAN( DIN )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   DIN
*     ..
*
*  =====================================================================
*
*  .. External Functions ..
      LOGICAL DLAISNAN
      EXTERNAL DLAISNAN
```

```
*  ..
*  .. Executable Statements ..
      DISNAN = DLAISNAN(DIN,DIN)
      RETURN
      END
```

———-

90.1.4 lisp code

— **disnan.lisp** —

———-

90.2 dlabad

90.2.1 Axiom unit tests

— **dlabad.input** —

```
)set break resume
)sys rm -f dlabad.output
)spool dlabad.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.2.2 Axiom help page

— **dlabad.help** —

```
====================================================================
dlabad examples
====================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlabad (SMALL, LARGE)
           DLABAD

Function/Subroutine Documentation
   subroutine dlabad (double precision SMALL, double precision LARGE)
       DLABAD

Purpose:
   DLABAD takes as input the values computed by DLAMCH for underflow and
   overflow, and returns the square root of each of these values if the
   log of LARGE is sufficiently large.  This subroutine is intended to
   identify machines with a large exponent range, such as the Crays, and
   redefine the underflow and overflow limits to be the square roots of
   the values computed by DLAMCH.  This subroutine is needed because
   DLAMCH does not compensate for poor arithmetic in the upper half of
   the exponent range, as is found on a Cray.

Parameters:

 SMALL
   SMALL is DOUBLE PRECISION
   On entry, the underflow threshold as computed by DLAMCH.
   On exit, if LOG10(LARGE) is sufficiently large, the square
   root of SMALL, otherwise unchanged.

 LARGE
   LARGE is DOUBLE PRECISION
   On entry, the overflow threshold as computed by DLAMCH.
   On exit, if LOG10(LARGE) is sufficiently large, the square
   root of LARGE, otherwise unchanged.
```

———-

90.2.3 fortran code

— dlabad.f —

```
*  =====================================================================
      SUBROUTINE DLABAD( SMALL, LARGE )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
```

```
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   LARGE, SMALL
*     ..
*
*  =====================================================================
*
*     .. Intrinsic Functions ..
      INTRINSIC          LOG10, SQRT
*     ..
*     .. Executable Statements ..
*
*     If it looks like we're on a Cray, take the square root of
*     SMALL and LARGE to avoid overflow and underflow problems.
*
      IF( LOG10( LARGE ).GT.2000.D0 ) THEN
         SMALL = SQRT( SMALL )
         LARGE = SQRT( LARGE )
      END IF
*
      RETURN
*
*     End of DLABAD
*
      END
```

———-

90.2.4 lisp code

— dlabad.lisp —

———-

90.3 dlacpy

90.3.1 Axiom unit tests

— dlacpy.input —

```
)set break resume
)sys rm -f dlacpy.output
)spool dlacpy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

90.3.2 Axiom help page

— dlacpy.help —

```
======================================================================
dlacpy examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlacpy (UPLO, M, N, A, LDA, B, LDB)
           DLACPY copies all or part of one two-dimensional array to another.

Function/Subroutine Documentation
   subroutine dlacpy (character UPLO, integer M, integer N, double precision,
       dimension( lda, * ) A, integer LDA, double precision, dimension( ldb, *
       ) B, integer LDB)
       DLACPY copies all or part of one two-dimensional array to another.

Purpose:
   DLACPY copies all or part of a two-dimensional matrix A to another matrix B.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies the part of the matrix A to be copied to B.
   = 'U':      Upper triangular part
   = 'L':      Lower triangular part
   Otherwise:  All of the matrix A

 M
```

```
  M is INTEGER
  The number of rows of the matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  The m by n matrix A.  If UPLO = 'U', only the upper triangle
  or trapezoid is accessed; if UPLO = 'L', only the lower
  triangle or trapezoid is accessed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

B
  B is DOUBLE PRECISION array, dimension (LDB,N)
  On exit, B = A in the locations specified by UPLO.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,M).
```

90.3.3 fortran code

— dlacpy.f —

```
*  =====================================================================
      SUBROUTINE DLACPY( UPLO, M, N, A, LDA, B, LDB )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            LDA, LDB, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * )
*     ..
*
```

```
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MIN
*     ..
*     .. Executable Statements ..
*
      IF( LSAME( UPLO, 'U' ) ) THEN
         DO 20 J = 1, N
            DO 10 I = 1, MIN( J, M )
               B( I, J ) = A( I, J )
   10       CONTINUE
   20    CONTINUE
      ELSE IF( LSAME( UPLO, 'L' ) ) THEN
         DO 40 J = 1, N
            DO 30 I = J, M
               B( I, J ) = A( I, J )
   30       CONTINUE
   40    CONTINUE
      ELSE
         DO 60 J = 1, N
            DO 50 I = 1, M
               B( I, J ) = A( I, J )
   50       CONTINUE
   60    CONTINUE
      END IF
      RETURN
*
*     End of DLACPY
*
      END
```

———-

90.3.4 lisp code

— **dlacpy.lisp** —

———-

90.4 dladiv

90.4.1 Axiom unit tests

— dladiv.input —

```
)set break resume
)sys rm -f dladiv.output
)spool dladiv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.4.2 Axiom help page

— dladiv.help —

```
======================================================================
dladiv examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dladiv (A, B, C, D, P, Q)
           DLADIV performs complex division in real arithmetic, avoiding
           unnecessary overflow.
       subroutine dladiv1 (A, B, C, D, P, Q)
       double precision function dladiv2 (A, B, C, D, R, T)

Function/Subroutine Documentation
   subroutine dladiv (double precision A, double precision B, double precision
       C, double precision D, double precision P, double precision Q)
       DLADIV performs complex division in real arithmetic, avoiding
       unnecessary overflow.

Purpose:
   DLADIV performs complex division in  real arithmetic
```

```
                    a + i*b
          p + i*q = ---------
                    c + i*d

   The algorithm is due to Michael Baudin and Robert L. Smith
   and can be found in the paper
   "A Robust Complex Division in Scilab"

Parameters:

 A
   A is DOUBLE PRECISION

 B
   B is DOUBLE PRECISION

 C
   C is DOUBLE PRECISION

 D
   D is DOUBLE PRECISION
   The scalars a, b, c, and d in the above expression.

 P
   P is DOUBLE PRECISION

 Q
   Q is DOUBLE PRECISION
   The scalars p and q in the above expression.

   subroutine dladiv1 (double precision A, double precision B, double
       precision C, double precision D, double precision P, double precision
       Q)
   double precision function dladiv2 (double precision A, double precision B,
       double precision C, double precision D, double precision R, double
       precision T)
```

———-

90.4.3 fortran code

— dladiv.f —

```
*  =====================================================================
      SUBROUTINE DLADIV( A, B, C, D, P, Q )
*
```

```
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     January 2013
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   A, B, C, D, P, Q
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   BS
      PARAMETER          ( BS = 2.0D0 )
      DOUBLE PRECISION   HALF
      PARAMETER          ( HALF = 0.5D0 )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D0 )
*
*     .. Local Scalars ..
      DOUBLE PRECISION   AA, BB, CC, DD, AB, CD, S, OV, UN, BE, EPS
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLADIV1
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
      AA = A
      BB = B
      CC = C
      DD = D
      AB = MAX( ABS(A), ABS(B) )
      CD = MAX( ABS(C), ABS(D) )
      S = 1.0D0

      OV = DLAMCH( 'Overflow threshold' )
      UN = DLAMCH( 'Safe minimum' )
      EPS = DLAMCH( 'Epsilon' )
      BE = BS / (EPS*EPS)

      IF( AB >= HALF*OV ) THEN
         AA = HALF * AA
         BB = HALF * BB
```

```
         S  = TWO * S
      END IF
      IF( CD >= HALF*OV ) THEN
         CC = HALF * CC
         DD = HALF * DD
         S  = HALF * S
      END IF
      IF( AB <= UN*BS/EPS ) THEN
         AA = AA * BE
         BB = BB * BE
         S  = S / BE
      END IF
      IF( CD <= UN*BS/EPS ) THEN
         CC = CC * BE
         DD = DD * BE
         S  = S * BE
      END IF
      IF( ABS( D ).LE.ABS( C ) ) THEN
         CALL DLADIV1(AA, BB, CC, DD, P, Q)
      ELSE
         CALL DLADIV1(BB, AA, DD, CC, P, Q)
         Q = -Q
      END IF
      P = P * S
      Q = Q * S
*
      RETURN
*
*     End of DLADIV
*
      END



      SUBROUTINE DLADIV1( A, B, C, D, P, Q )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     January 2013
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   A, B, C, D, P, Q
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D0 )
```

```
*
*     .. Local Scalars ..
      DOUBLE PRECISION   R, T
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLADIV2
      EXTERNAL           DLADIV2
*     ..
*     .. Executable Statements ..
*
      R = D / C
      T = ONE / (C + D * R)
      P = DLADIV2(A, B, C, D, R, T)
      A = -A
      Q = DLADIV2(B, A, C, D, R, T)
*
      RETURN
*
*     End of DLADIV1
*
      END

      DOUBLE PRECISION FUNCTION DLADIV2( A, B, C, D, R, T )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     January 2013
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   A, B, C, D, R, T
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
*
*     .. Local Scalars ..
      DOUBLE PRECISION   BR
*     ..
*     .. Executable Statements ..
*
      IF( R.NE.ZERO ) THEN
         BR = B * R
         IF( BR.NE.ZERO ) THEN
            DLADIV2 = (A + BR) * T
         ELSE
            DLADIV2 = A * T + (B * T) * R
```

```
         END IF
      ELSE
         DLADIV2 = (A + D * (B / C)) * T
      END IF
*
      RETURN
*
*     End of DLADIV12
*
      END
```

———-

90.4.4 lisp code

— dladiv.lisp —

———-

90.5 dlae2

90.5.1 Axiom unit tests

— dlae2.input —

```
)set break resume
)sys rm -f dlae2.output
)spool dlae2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.5.2 Axiom help page

— dlae2.help —

```
====================================================================
dlae2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlae2 (A, B, C, RT1, RT2)
           DLAE2 computes the eigenvalues of a 2-by-2 symmetric matrix.

Function/Subroutine Documentation
   subroutine dlae2 (double precision A, double precision B, double precision
       C, double precision RT1, double precision RT2)
       DLAE2 computes the eigenvalues of a 2-by-2 symmetric matrix.

Purpose:
   DLAE2  computes the eigenvalues of a 2-by-2 symmetric matrix
      [  A   B  ]
      [  B   C  ].
   On return, RT1 is the eigenvalue of larger absolute value, and RT2
   is the eigenvalue of smaller absolute value.

Parameters:

 A
   A is DOUBLE PRECISION
   The (1,1) element of the 2-by-2 matrix.

 B
   B is DOUBLE PRECISION
   The (1,2) and (2,1) elements of the 2-by-2 matrix.

 C
   C is DOUBLE PRECISION
   The (2,2) element of the 2-by-2 matrix.

 RT1
   RT1 is DOUBLE PRECISION
   The eigenvalue of larger absolute value.

 RT2
   RT2 is DOUBLE PRECISION
   The eigenvalue of smaller absolute value.

Further Details:

   RT1 is accurate to a few ulps barring over/underflow.
```

```
RT2 may be inaccurate if there is massive cancellation in the
determinant A*C-B*B; higher precision or correctly rounded or
correctly truncated arithmetic would be needed to compute RT2
accurately in all cases.

Overflow is possible only if RT1 is within a factor of 5 of overflow.
Underflow is harmless if the input data is 0 or exceeds
underflow_threshold / macheps.
```

———-

90.5.3 fortran code

— dlae2.f —

```
*  =====================================================================
      SUBROUTINE DLAE2( A, B, C, RT1, RT2 )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   A, B, C, RT1, RT2
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D0 )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D0 )
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
      DOUBLE PRECISION   HALF
      PARAMETER          ( HALF = 0.5D0 )
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION   AB, ACMN, ACMX, ADF, DF, RT, SM, TB
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SQRT
*     ..
*     .. Executable Statements ..
```

```
*
*     Compute the eigenvalues
*
      SM = A + C
      DF = A - C
      ADF = ABS( DF )
      TB = B + B
      AB = ABS( TB )
      IF( ABS( A ).GT.ABS( C ) ) THEN
         ACMX = A
         ACMN = C
      ELSE
         ACMX = C
         ACMN = A
      END IF
      IF( ADF.GT.AB ) THEN
         RT = ADF*SQRT( ONE+( AB / ADF )**2 )
      ELSE IF( ADF.LT.AB ) THEN
         RT = AB*SQRT( ONE+( ADF / AB )**2 )
      ELSE
*
*        Includes case AB=ADF=0
*
         RT = AB*SQRT( TWO )
      END IF
      IF( SM.LT.ZERO ) THEN
         RT1 = HALF*( SM-RT )
*
*        Order of execution important.
*        To get fully accurate smaller eigenvalue,
*        next line needs to be executed in higher precision.
*
         RT2 = ( ACMX / RT1 )*ACMN - ( B / RT1 )*B
      ELSE IF( SM.GT.ZERO ) THEN
         RT1 = HALF*( SM+RT )
*
*        Order of execution important.
*        To get fully accurate smaller eigenvalue,
*        next line needs to be executed in higher precision.
*
         RT2 = ( ACMX / RT1 )*ACMN - ( B / RT1 )*B
      ELSE
*
*        Includes case RT1 = RT2 = 0
*
         RT1 = HALF*RT
         RT2 = -HALF*RT
      END IF
      RETURN
*
```

```
*     End of DLAE2
*
      END
```

———-

90.5.4 lisp code

— dlae2.lisp —

```
(let* ((one 1.0d0) (two 2.0d0) (zero 0.0d0) (half 0.5d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 0.5d0 0.5d0) half)
           (ignorable one two zero half))
  (defun dlae2 (a b c rt1 rt2)
    (declare (type (double-float) rt2 rt1 c b a))
    (prog ((ab 0.0d0) (acmn 0.0d0) (acmx 0.0d0) (adf 0.0d0) (df 0.0d0)
           (rt 0.0d0) (sm 0.0d0) (tb 0.0d0))
      (declare (type (double-float) ab acmn acmx adf df rt sm tb))
      (setf sm (+ a c))
      (setf df (- a c))
      (setf adf (abs df))
      (setf tb (+ b b))
      (setf ab (abs tb))
      (cond ((> (abs a) (abs c)) (setf acmx a) (setf acmn c))
            (t (setf acmx c) (setf acmn a)))
      (cond
       ((> adf ab)
        (setf rt (* adf (f2cl-lib:fsqrt (+ one (expt (/ ab adf) 2))))))
       ((< adf ab)
        (setf rt (* ab (f2cl-lib:fsqrt (+ one (expt (/ adf ab) 2))))))
       (t (setf rt (* ab (f2cl-lib:fsqrt two)))))
      (cond
       ((< sm zero) (setf rt1 (* half (- sm rt)))
        (setf rt2 (- (* (/ acmx rt1) acmn) (* (/ b rt1) b))))
       ((> sm zero) (setf rt1 (* half (+ sm rt)))
        (setf rt2 (- (* (/ acmx rt1) acmn) (* (/ b rt1) b))))
       (t (setf rt1 (* half rt)) (setf rt2 (* (- half) rt))))
      (go end_label)
     end_label
      (return (values nil nil nil rt1 rt2)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlae2 fortran-to-lisp::*f2cl-function-info*)
```

```
(fortran-to-lisp::make-f2cl-finfo :arg-types
                                  '((double-float) (double-float)
                                    (double-float) (double-float)
                                    (double-float))
                                  :return-values
                                  '(nil nil nil fortran-to-lisp::rt1
                                    fortran-to-lisp::rt2)
                                  :calls 'nil)))
```

———-

90.6 dlaebz

90.6.1 Axiom unit tests

— **dlaebz.input** —

```
)set break resume
)sys rm -f dlaebz.output
)spool dlaebz.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.6.2 Axiom help page

— **dlaebz.help** —

```
==================================================================
dlaebz examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
```

```
   Functions/Subroutines
       subroutine dlaebz (IJOB, NITMAX, N, MMAX, MINP, NBMIN, ABSTOL, RELTOL,
           PIVMIN, D, E, E2, NVAL, AB, C, MOUT, NAB, WORK, IWORK, INFO)
           DLAEBZ computes the number of eigenvalues of a real symmetric
           tridiagonal matrix which are less than or equal to a given value,
           and performs other tasks required by the routine sstebz.

Function/Subroutine Documentation
   subroutine dlaebz (integer IJOB, integer NITMAX, integer N, integer MMAX,
       integer MINP, integer NBMIN, double precision ABSTOL, double precision
       RELTOL, double precision PIVMIN, double precision, dimension( * ) D,
       double precision, dimension( * ) E, double precision, dimension( * )
       E2, integer, dimension( * ) NVAL, double precision, dimension( mmax, *
       ) AB, double precision, dimension( * ) C, integer MOUT, integer,
       dimension( mmax, * ) NAB, double precision, dimension( * ) WORK,
       integer, dimension( * ) IWORK, integer INFO)
       DLAEBZ computes the number of eigenvalues of a real symmetric
       tridiagonal matrix which are less than or equal to a given value, and
       performs other tasks required by the routine sstebz.

Purpose:
   DLAEBZ contains the iteration loops which compute and use the
   function N(w), which is the count of eigenvalues of a symmetric
   tridiagonal matrix T less than or equal to its argument  w.  It
   performs a choice of two types of loops:

   IJOB=1, followed by
   IJOB=2: It takes as input a list of intervals and returns a list of
      sufficiently small intervals whose union contains the same
      eigenvalues as the union of the original intervals.
      The input intervals are (AB(j,1),AB(j,2)], j=1,...,MINP.
      The output interval (AB(j,1),AB(j,2)] will contain
      eigenvalues NAB(j,1)+1,...,NAB(j,2), where 1 <= j <= MOUT.

   IJOB=3: It performs a binary search in each input interval
      (AB(j,1),AB(j,2)] for a point  w(j)  such that
      N(w(j))=NVAL(j), and uses  C(j)  as the starting point of
      the search.  If such a w(j) is found, then on output
      AB(j,1)=AB(j,2)=w.  If no such w(j) is found, then on output
      (AB(j,1),AB(j,2)] will be a small interval containing the
      point where N(w) jumps through NVAL(j), unless that point
      lies outside the initial interval.

   Note that the intervals are in all cases half-open intervals,
   i.e., of the form  (a,b] , which includes  b  but not  a .

   To avoid underflow, the matrix should be scaled so that its largest
   element is no greater than  overflow**(1/2) * underflow**(1/4)
   in absolute value.  To assure the most accurate computation
   of small eigenvalues, the matrix should be scaled to be
```

```
  not much smaller than that, either.

  See W. Kahan "Accurate Eigenvalues of a Symmetric Tridiagonal
  Matrix", Report CS41, Computer Science Dept., Stanford
  University, July 21, 1966

  Note: the arguments are, in general, *not* checked for unreasonable
  values.

Parameters:

 IJOB
   IJOB is INTEGER
   Specifies what is to be done:
   = 1:  Compute NAB for the initial intervals.
   = 2:  Perform bisection iteration to find eigenvalues of T.
   = 3:  Perform bisection iteration to invert N(w), i.e.,
         to find a point which has a specified number of
         eigenvalues of T to its left.
   Other values will cause DLAEBZ to return with INFO=-1.

 NITMAX
   NITMAX is INTEGER
   The maximum number of "levels" of bisection to be
   performed, i.e., an interval of width W will not be made
   smaller than 2^(-NITMAX) * W.  If not all intervals
   have converged after NITMAX iterations, then INFO is set
   to the number of non-converged intervals.

 N
   N is INTEGER
   The dimension n of the tridiagonal matrix T.  It must be at least 1.

 MMAX
   MMAX is INTEGER
   The maximum number of intervals.  If more than MMAX intervals
   are generated, then DLAEBZ will quit with INFO=MMAX+1.

 MINP
   MINP is INTEGER
   The initial number of intervals.  It may not be greater than MMAX.

 NBMIN
   NBMIN is INTEGER
   The smallest number of intervals that should be processed
   using a vector loop.  If zero, then only the scalar loop will be used.

 ABSTOL
   ABSTOL is DOUBLE PRECISION
   The minimum (absolute) width of an interval.  When an
```

```
  interval is narrower than ABSTOL, or than RELTOL times the
  larger (in magnitude) endpoint, then it is considered to be
  sufficiently small, i.e., converged.  This must be at least zero.

RELTOL
  RELTOL is DOUBLE PRECISION
  The minimum relative width of an interval.  When an interval
  is narrower than ABSTOL, or than RELTOL times the larger (in
  magnitude) endpoint, then it is considered to be
  sufficiently small, i.e., converged.  Note: this should
  always be at least radix*machine epsilon.

PIVMIN
  PIVMIN is DOUBLE PRECISION
  The minimum absolute value of a "pivot" in the Sturm sequence loop.
  This must be at least  max |e(j)**2|*safe_min  and at
  least safe_min, where safe_min is at least
  the smallest number that can divide one without overflow.

D
  D is DOUBLE PRECISION array, dimension (N)
  The diagonal elements of the tridiagonal matrix T.

E
  E is DOUBLE PRECISION array, dimension (N)
  The offdiagonal elements of the tridiagonal matrix T in
  positions 1 through N-1.  E(N) is arbitrary.

E2
  E2 is DOUBLE PRECISION array, dimension (N)
  The squares of the offdiagonal elements of the tridiagonal
  matrix T.  E2(N) is ignored.

NVAL
  NVAL is INTEGER array, dimension (MINP)
  If IJOB=1 or 2, not referenced.
  If IJOB=3, the desired values of N(w).  The elements of NVAL
  will be reordered to correspond with the intervals in AB.
  Thus, NVAL(j) on output will not, in general be the same as
  NVAL(j) on input, but it will correspond with the interval
  (AB(j,1),AB(j,2)] on output.

AB
  AB is DOUBLE PRECISION array, dimension (MMAX,2)
  The endpoints of the intervals.  AB(j,1) is  a(j), the left
  endpoint of the j-th interval, and AB(j,2) is b(j), the
  right endpoint of the j-th interval.  The input intervals
  will, in general, be modified, split, and reordered by the calculation.

C
```

```
  C is DOUBLE PRECISION array, dimension (MMAX)
  If IJOB=1, ignored.
  If IJOB=2, workspace.
  If IJOB=3, then on input C(j) should be initialized to the
  first search point in the binary search.

MOUT
  MOUT is INTEGER
  If IJOB=1, the number of eigenvalues in the intervals.
  If IJOB=2 or 3, the number of intervals output.
  If IJOB=3, MOUT will equal MINP.

NAB
  NAB is INTEGER array, dimension (MMAX,2)
  If IJOB=1, then on output NAB(i,j) will be set to N(AB(i,j)).
  If IJOB=2, then on input, NAB(i,j) should be set.  It must
     satisfy the condition:
     N(AB(i,1)) <= NAB(i,1) <= NAB(i,2) <= N(AB(i,2)),
     which means that in interval i only eigenvalues
     NAB(i,1)+1,...,NAB(i,2) will be considered.  Usually,
     NAB(i,j)=N(AB(i,j)), from a previous call to DLAEBZ with IJOB=1.

  On output, NAB(i,j) will contain
  max(na(k),min(nb(k),N(AB(i,j)))), where k is the index of
  the input interval that the output interval
  (AB(j,1),AB(j,2)] came from, and na(k) and nb(k) are the
  the input values of NAB(k,1) and NAB(k,2).
  If IJOB=3, then on output, NAB(i,j) contains N(AB(i,j)),
  unless N(w) > NVAL(i) for all search points  w , in which
  case NAB(i,1) will not be modified, i.e., the output
  value will be the same as the input value (modulo
  reorderings -- see NVAL and AB), or unless N(w) < NVAL(i)
  for all search points  w , in which case NAB(i,2) will
  not be modified.  Normally, NAB should be set to some
  distinctive value(s) before DLAEBZ is called.

WORK
  WORK is DOUBLE PRECISION array, dimension (MMAX)
  Workspace.

IWORK
  IWORK is INTEGER array, dimension (MMAX)
  Workspace.

INFO
  INFO is INTEGER
  = 0:       All intervals converged.
  = 1--MMAX: The last INFO intervals did not converge.
  = MMAX+1:  More than MMAX intervals were generated.
```

```
Further Details:

   This routine is intended to be called only by other LAPACK
   routines, thus the interface is less user-friendly.  It is intended
   for two purposes:

   (a) finding eigenvalues.  In this case, DLAEBZ should have one or
       more initial intervals set up in AB, and DLAEBZ should be called
       with IJOB=1.  This sets up NAB, and also counts the eigenvalues.
       Intervals with no eigenvalues would usually be thrown out at
       this point.  Also, if not all the eigenvalues in an interval i
       are desired, NAB(i,1) can be increased or NAB(i,2) decreased.
       For example, set NAB(i,1)=NAB(i,2)-1 to get the largest
       eigenvalue.  DLAEBZ is then called with IJOB=2 and MMAX
       no smaller than the value of MOUT returned by the call with
       IJOB=1.  After this (IJOB=2) call, eigenvalues NAB(i,1)+1
       through NAB(i,2) are approximately AB(i,1) (or AB(i,2)) to the
       tolerance specified by ABSTOL and RELTOL.

   (b) finding an interval (a',b'] containing eigenvalues w(f),...,w(l).
       In this case, start with a Gershgorin interval  (a,b).  Set up
       AB to contain 2 search intervals, both initially (a,b).  One
       NVAL element should contain  f-1  and the other should contain l,
       while C should contain a and b, resp.  NAB(i,1) should be -1
       and NAB(i,2) should be N+1, to flag an error if the desired
       interval does not lie in (a,b).  DLAEBZ is then called with IJOB=3.
       On exit, if w(f-1) < w(f), then one of the intervals --
       j -- will have AB(j,1)=AB(j,2) and NAB(j,1)=NAB(j,2)=f-1, while
       if, to the specified tolerance, w(f-k)=...=w(f+r), k > 0 and r >= 0,
       then the interval will have  N(AB(j,1))=NAB(j,1)=f-k and
       N(AB(j,2))=NAB(j,2)=f+r.  The cases w(l) < w(l+1) and
       w(l-r)=...=w(l+k) are handled similarly.
```

———-

90.6.3 fortran code

— dlaebz.f —

```
*  =====================================================================
      SUBROUTINE DLAEBZ( IJOB, NITMAX, N, MMAX, MINP, NBMIN, ABSTOL,
     $                   RELTOL, PIVMIN, D, E, E2, NVAL, AB, C, MOUT,
     $                   NAB, WORK, IWORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
```

```
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            IJOB, INFO, MINP, MMAX, MOUT, N, NBMIN, NITMAX
      DOUBLE PRECISION   ABSTOL, PIVMIN, RELTOL
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * ), NAB( MMAX, * ), NVAL( * )
      DOUBLE PRECISION   AB( MMAX, * ), C( * ), D( * ), E( * ), E2( * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, TWO, HALF
      PARAMETER          ( ZERO = 0.0D0, TWO = 2.0D0,
     $                   HALF = 1.0D0 / TWO )
*     ..
*     .. Local Scalars ..
      INTEGER            ITMP1, ITMP2, J, JI, JIT, JP, KF, KFNEW, KL,
     $                   KLNEW
      DOUBLE PRECISION   TMP1, TMP2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Check for Errors
*
      INFO = 0
      IF( IJOB.LT.1 .OR. IJOB.GT.3 ) THEN
         INFO = -1
         RETURN
      END IF
*
*     Initialize NAB
*
      IF( IJOB.EQ.1 ) THEN
*
*        Compute the number of eigenvalues in the initial intervals.
*
         MOUT = 0
         DO 30 JI = 1, MINP
            DO 20 JP = 1, 2
               TMP1 = D( 1 ) - AB( JI, JP )
               IF( ABS( TMP1 ).LT.PIVMIN )
     $            TMP1 = -PIVMIN
               NAB( JI, JP ) = 0
```

```
               IF( TMP1.LE.ZERO )
     $            NAB( JI, JP ) = 1
*
               DO 10 J = 2, N
                  TMP1 = D( J ) - E2( J-1 ) / TMP1 - AB( JI, JP )
                  IF( ABS( TMP1 ).LT.PIVMIN )
     $               TMP1 = -PIVMIN
                  IF( TMP1.LE.ZERO )
     $               NAB( JI, JP ) = NAB( JI, JP ) + 1
   10          CONTINUE
   20       CONTINUE
            MOUT = MOUT + NAB( JI, 2 ) - NAB( JI, 1 )
   30    CONTINUE
         RETURN
      END IF
*
*     Initialize for loop
*
*     KF and KL have the following meaning:
*        Intervals 1,...,KF-1 have converged.
*        Intervals KF,...,KL  still need to be refined.
*
      KF = 1
      KL = MINP
*
*     If IJOB=2, initialize C.
*     If IJOB=3, use the user-supplied starting point.
*
      IF( IJOB.EQ.2 ) THEN
         DO 40 JI = 1, MINP
            C( JI ) = HALF*( AB( JI, 1 )+AB( JI, 2 ) )
   40    CONTINUE
      END IF
*
*     Iteration loop
*
      DO 130 JIT = 1, NITMAX
*
*        Loop over intervals
*
         IF( KL-KF+1.GE.NBMIN .AND. NBMIN.GT.0 ) THEN
*
*           Begin of Parallel Version of the loop
*
            DO 60 JI = KF, KL
*
*              Compute N(c), the number of eigenvalues less than c
*
               WORK( JI ) = D( 1 ) - C( JI )
               IWORK( JI ) = 0
```

```
               IF( WORK( JI ).LE.PIVMIN ) THEN
                  IWORK( JI ) = 1
                  WORK( JI ) = MIN( WORK( JI ), -PIVMIN )
               END IF
*
               DO 50 J = 2, N
                  WORK( JI ) = D( J ) - E2( J-1 ) / WORK( JI ) - C( JI )
                  IF( WORK( JI ).LE.PIVMIN ) THEN
                     IWORK( JI ) = IWORK( JI ) + 1
                     WORK( JI ) = MIN( WORK( JI ), -PIVMIN )
                  END IF
   50          CONTINUE
   60       CONTINUE
*
            IF( IJOB.LE.2 ) THEN
*
*              IJOB=2: Choose all intervals containing eigenvalues.
*
               KLNEW = KL
               DO 70 JI = KF, KL
*
*                 Insure that N(w) is monotone
*
                  IWORK( JI ) = MIN( NAB( JI, 2 ),
     $                           MAX( NAB( JI, 1 ), IWORK( JI ) ) )
*
*                 Update the Queue -- add intervals if both halves
*                 contain eigenvalues.
*
                  IF( IWORK( JI ).EQ.NAB( JI, 2 ) ) THEN
*
*                    No eigenvalue in the upper interval:
*                    just use the lower interval.
*
                     AB( JI, 2 ) = C( JI )
*
                  ELSE IF( IWORK( JI ).EQ.NAB( JI, 1 ) ) THEN
*
*                    No eigenvalue in the lower interval:
*                    just use the upper interval.
*
                     AB( JI, 1 ) = C( JI )
                  ELSE
                     KLNEW = KLNEW + 1
                     IF( KLNEW.LE.MMAX ) THEN
*
*                       Eigenvalue in both intervals -- add upper to
*                       queue.
*
                        AB( KLNEW, 2 ) = AB( JI, 2 )
```

```
                     NAB( KLNEW, 2 ) = NAB( JI, 2 )
                     AB( KLNEW, 1 ) = C( JI )
                     NAB( KLNEW, 1 ) = IWORK( JI )
                     AB( JI, 2 ) = C( JI )
                     NAB( JI, 2 ) = IWORK( JI )
                  ELSE
                     INFO = MMAX + 1
                  END IF
               END IF
   70          CONTINUE
               IF( INFO.NE.0 )
     $            RETURN
               KL = KLNEW
            ELSE
*
*              IJOB=3: Binary search.  Keep only the interval containing
*                      w   s.t. N(w) = NVAL
*
               DO 80 JI = KF, KL
                  IF( IWORK( JI ).LE.NVAL( JI ) ) THEN
                     AB( JI, 1 ) = C( JI )
                     NAB( JI, 1 ) = IWORK( JI )
                  END IF
                  IF( IWORK( JI ).GE.NVAL( JI ) ) THEN
                     AB( JI, 2 ) = C( JI )
                     NAB( JI, 2 ) = IWORK( JI )
                  END IF
   80          CONTINUE
            END IF
*
         ELSE
*
*           End of Parallel Version of the loop
*
*           Begin of Serial Version of the loop
*
            KLNEW = KL
            DO 100 JI = KF, KL
*
*              Compute N(w), the number of eigenvalues less than w
*
               TMP1 = C( JI )
               TMP2 = D( 1 ) - TMP1
               ITMP1 = 0
               IF( TMP2.LE.PIVMIN ) THEN
                  ITMP1 = 1
                  TMP2 = MIN( TMP2, -PIVMIN )
               END IF
*
               DO 90 J = 2, N
```

```
                 TMP2 = D( J ) - E2( J-1 ) / TMP2 - TMP1
                 IF( TMP2.LE.PIVMIN ) THEN
                    ITMP1 = ITMP1 + 1
                    TMP2 = MIN( TMP2, -PIVMIN )
                 END IF
   90         CONTINUE
*
              IF( IJOB.LE.2 ) THEN
*
*                IJOB=2: Choose all intervals containing eigenvalues.
*
*                Insure that N(w) is monotone
*
                 ITMP1 = MIN( NAB( JI, 2 ),
     $                   MAX( NAB( JI, 1 ), ITMP1 ) )
*
*                Update the Queue -- add intervals if both halves
*                contain eigenvalues.
*
                 IF( ITMP1.EQ.NAB( JI, 2 ) ) THEN
*
*                   No eigenvalue in the upper interval:
*                   just use the lower interval.
*
                    AB( JI, 2 ) = TMP1
*
                 ELSE IF( ITMP1.EQ.NAB( JI, 1 ) ) THEN
*
*                   No eigenvalue in the lower interval:
*                   just use the upper interval.
*
                    AB( JI, 1 ) = TMP1
                 ELSE IF( KLNEW.LT.MMAX ) THEN
*
*                   Eigenvalue in both intervals -- add upper to queue.
*
                    KLNEW = KLNEW + 1
                    AB( KLNEW, 2 ) = AB( JI, 2 )
                    NAB( KLNEW, 2 ) = NAB( JI, 2 )
                    AB( KLNEW, 1 ) = TMP1
                    NAB( KLNEW, 1 ) = ITMP1
                    AB( JI, 2 ) = TMP1
                    NAB( JI, 2 ) = ITMP1
                 ELSE
                    INFO = MMAX + 1
                    RETURN
                 END IF
              ELSE
*
*                IJOB=3: Binary search.  Keep only the interval
```

```
*                        containing  w  s.t. N(w) = NVAL
*
                 IF( ITMP1.LE.NVAL( JI ) ) THEN
                    AB( JI, 1 ) = TMP1
                    NAB( JI, 1 ) = ITMP1
                 END IF
                 IF( ITMP1.GE.NVAL( JI ) ) THEN
                    AB( JI, 2 ) = TMP1
                    NAB( JI, 2 ) = ITMP1
                 END IF
              END IF
  100       CONTINUE
            KL = KLNEW
*
         END IF
*
*        Check for convergence
*
         KFNEW = KF
         DO 110 JI = KF, KL
            TMP1 = ABS( AB( JI, 2 )-AB( JI, 1 ) )
            TMP2 = MAX( ABS( AB( JI, 2 ) ), ABS( AB( JI, 1 ) ) )
            IF( TMP1.LT.MAX( ABSTOL, PIVMIN, RELTOL*TMP2 ) .OR.
     $          NAB( JI, 1 ).GE.NAB( JI, 2 ) ) THEN
*
*             Converged -- Swap with position KFNEW,
*                          then increment KFNEW
*
              IF( JI.GT.KFNEW ) THEN
                 TMP1 = AB( JI, 1 )
                 TMP2 = AB( JI, 2 )
                 ITMP1 = NAB( JI, 1 )
                 ITMP2 = NAB( JI, 2 )
                 AB( JI, 1 ) = AB( KFNEW, 1 )
                 AB( JI, 2 ) = AB( KFNEW, 2 )
                 NAB( JI, 1 ) = NAB( KFNEW, 1 )
                 NAB( JI, 2 ) = NAB( KFNEW, 2 )
                 AB( KFNEW, 1 ) = TMP1
                 AB( KFNEW, 2 ) = TMP2
                 NAB( KFNEW, 1 ) = ITMP1
                 NAB( KFNEW, 2 ) = ITMP2
                 IF( IJOB.EQ.3 ) THEN
                    ITMP1 = NVAL( JI )
                    NVAL( JI ) = NVAL( KFNEW )
                    NVAL( KFNEW ) = ITMP1
                 END IF
              END IF
              KFNEW = KFNEW + 1
            END IF
  110    CONTINUE
```

```
         KF = KFNEW
*
*        Choose Midpoints
*
         DO 120 JI = KF, KL
            C( JI ) = HALF*( AB( JI, 1 )+AB( JI, 2 ) )
  120    CONTINUE
*
*        If no more intervals to refine, quit.
*
         IF( KF.GT.KL )
     $      GO TO 140
  130 CONTINUE
*
*     Converged
*
  140 CONTINUE
      INFO = MAX( KL+1-KF, 0 )
      MOUT = KL
*
      RETURN
*
*     End of DLAEBZ
*
      END
```

———-

90.6.4 lisp code

— dlaebz.lisp —

```
(let* ((zero 0.0d0) (two 2.0d0) (half (f2cl-lib:f2cl/ 1.0d0 two)))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float) half)
           (ignorable zero two half))
  (defun dlaebz
         (ijob nitmax n mmax minp nbmin abstol reltol pivmin d e e2 nval ab c
          mout nab work iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork nab nval)
             (type (array double-float (*)) work c ab e2 e d)
             (type (double-float) pivmin reltol abstol)
             (type (f2cl-lib:integer4) info mout nbmin minp mmax n nitmax
              ijob))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%)
```

```
    (e2 double-float e2-%data% e2-%offset%)
    (ab double-float ab-%data% ab-%offset%)
    (c double-float c-%data% c-%offset%)
    (work double-float work-%data% work-%offset%)
    (nval f2cl-lib:integer4 nval-%data% nval-%offset%)
    (nab f2cl-lib:integer4 nab-%data% nab-%offset%)
    (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((tmp1 0.0d0) (tmp2 0.0d0) (itmp1 0) (itmp2 0) (j 0) (ji 0) (jit 0)
         (jp 0) (kf 0) (kfnew 0) (kl 0) (klnew 0))
    (declare (type (double-float) tmp1 tmp2)
             (type (f2cl-lib:integer4) itmp1 itmp2 j ji jit jp kf kfnew kl
              klnew))
    (setf info 0)
    (cond ((or (< ijob 1) (> ijob 3)) (setf info -1) (go end_label)))
    (cond
     ((= ijob 1) (setf mout 0)
      (f2cl-lib:fdo (ji 1 (f2cl-lib:int-add ji 1))
                    ((> ji minp) nil)
        (tagbody
          (f2cl-lib:fdo (jp 1 (f2cl-lib:int-add jp 1))
                        ((> jp 2) nil)
            (tagbody
              (setf tmp1
                      (- (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
                         (f2cl-lib:fref ab-%data% (ji jp) ((1 mmax) (1 *))
                                        ab-%offset%)))
              (if (< (abs tmp1) pivmin)
                  (setf tmp1 (- pivmin)))
              (setf (f2cl-lib:fref nab-%data% (ji jp) ((1 mmax) (1 *))
                                   nab-%offset%)
                      0)
              (if (<= tmp1 zero)
                  (setf (f2cl-lib:fref nab-%data% (ji jp) ((1 mmax) (1 *))
                                       nab-%offset%)
                        1))
              (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                            ((> j n) nil)
                (tagbody
                  (setf tmp1
                          (-
                           (+
                            (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                            (/
                             (-
                              (f2cl-lib:fref e2-%data%
                                             ((f2cl-lib:int-sub j 1))
                                             ((1 *)) e2-%offset%))
                             tmp1))
                           (f2cl-lib:fref ab-%data% (ji jp)
                                          ((1 mmax) (1 *)) ab-%offset%)))
```

```
                (if (< (abs tmp1) pivmin)
                    (setf tmp1 (- pivmin)))
                (if (<= tmp1 zero)
                    (setf (f2cl-lib:fref nab-%data% (ji jp)
                                         ((1 mmax) (1 *)) nab-%offset%)
                            (f2cl-lib:int-add
                             (f2cl-lib:fref nab-%data% (ji jp)
                                            ((1 mmax) (1 *))
                                            nab-%offset%)
                             1)))
              label10))
          label20))
      (setf mout
              (f2cl-lib:int-sub
               (f2cl-lib:int-add mout
                                 (f2cl-lib:fref nab-%data% (ji 2)
                                                ((1 mmax) (1 *))
                                                nab-%offset%))
               (f2cl-lib:fref nab-%data% (ji 1) ((1 mmax) (1 *))
                              nab-%offset%)))
     label30))
  (go end_label)))
(setf kf 1)
(setf kl minp)
(cond
 ((= ijob 2)
  (f2cl-lib:fdo (ji 1 (f2cl-lib:int-add ji 1))
                ((> ji minp) nil)
    (tagbody
      (setf (f2cl-lib:fref c-%data% (ji) ((1 *)) c-%offset%)
              (* half
                 (+
                  (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
                                 ab-%offset%)
                  (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                                 ab-%offset%))))
     label40))))
(f2cl-lib:fdo (jit 1 (f2cl-lib:int-add jit 1))
              ((> jit nitmax) nil)
  (tagbody
    (cond
     ((and (>= (f2cl-lib:int-add kl (f2cl-lib:int-sub kf) 1) nbmin)
           (> nbmin 0))
      (f2cl-lib:fdo (ji kf (f2cl-lib:int-add ji 1))
                    ((> ji kl) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (ji) ((1 *)) work-%offset%)
                  (- (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
                     (f2cl-lib:fref c-%data% (ji) ((1 *)) c-%offset%)))
          (setf (f2cl-lib:fref iwork-%data% (ji) ((1 *))
```

```
                          iwork-%offset%)
                  0)
            (cond
             ((<= (f2cl-lib:fref work (ji) ((1 *))) pivmin)
              (setf (f2cl-lib:fref iwork-%data% (ji) ((1 *))
                                   iwork-%offset%)
                    1)
              (setf (f2cl-lib:fref work-%data% (ji) ((1 *))
                                   work-%offset%)
                    (min
                     (f2cl-lib:fref work-%data% (ji) ((1 *))
                                    work-%offset%)
                     (- pivmin)))))
            (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                          ((> j n) nil)
              (tagbody
                (setf (f2cl-lib:fref work-%data% (ji) ((1 *))
                                     work-%offset%)
                      (-
                       (+
                        (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                        (/
                         (-
                          (f2cl-lib:fref e2-%data%
                                         ((f2cl-lib:int-sub j 1))
                                         ((1 *)) e2-%offset%))
                         (f2cl-lib:fref work-%data% (ji) ((1 *))
                                        work-%offset%)))
                       (f2cl-lib:fref c-%data% (ji) ((1 *))
                                      c-%offset%)))
                (cond
                 ((<= (f2cl-lib:fref work (ji) ((1 *))) pivmin)
                  (setf (f2cl-lib:fref iwork-%data% (ji) ((1 *))
                                       iwork-%offset%)
                        (f2cl-lib:int-add
                         (f2cl-lib:fref iwork-%data% (ji) ((1 *))
                                        iwork-%offset%)
                         1))
                  (setf (f2cl-lib:fref work-%data% (ji) ((1 *))
                                       work-%offset%)
                        (min
                         (f2cl-lib:fref work-%data% (ji) ((1 *))
                                        work-%offset%)
                         (- pivmin)))))
               label50))
           label60))
          (cond
           ((<= ijob 2) (setf klnew kl)
            (f2cl-lib:fdo (ji kf (f2cl-lib:int-add ji 1))
                          ((> ji kl) nil)
```

```
(tagbody
  (setf (f2cl-lib:fref iwork-%data% (ji) ((1 *))
                       iwork-%offset%)
        (min
         (the f2cl-lib:integer4
              (f2cl-lib:fref nab-%data% (ji 2)
                             ((1 mmax) (1 *))
                             nab-%offset%))
         (the f2cl-lib:integer4
              (max
               (the f2cl-lib:integer4
                    (f2cl-lib:fref nab-%data% (ji 1)
                                   ((1 mmax) (1 *))
                                   nab-%offset%))
               (the f2cl-lib:integer4
                    (f2cl-lib:fref iwork-%data% (ji)
                                   ((1 *))
                                   iwork-%offset%))))))
  (cond
   ((= (f2cl-lib:fref iwork (ji) ((1 *)))
       (f2cl-lib:fref nab (ji 2) ((1 mmax) (1 *))))
    (setf (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                         ab-%offset%)
          (f2cl-lib:fref c-%data% (ji) ((1 *))
                         c-%offset%)))
   ((= (f2cl-lib:fref iwork (ji) ((1 *)))
       (f2cl-lib:fref nab (ji 1) ((1 mmax) (1 *))))
    (setf (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
                         ab-%offset%)
          (f2cl-lib:fref c-%data% (ji) ((1 *))
                         c-%offset%)))
   (t (setf klnew (f2cl-lib:int-add klnew 1))
    (cond
     ((<= klnew mmax)
      (setf (f2cl-lib:fref ab-%data% (klnew 2)
                           ((1 mmax) (1 *)) ab-%offset%)
            (f2cl-lib:fref ab-%data% (ji 2)
                           ((1 mmax) (1 *)) ab-%offset%))
      (setf (f2cl-lib:fref nab-%data% (klnew 2)
                           ((1 mmax) (1 *)) nab-%offset%)
            (f2cl-lib:fref nab-%data% (ji 2)
                           ((1 mmax) (1 *)) nab-%offset%))
      (setf (f2cl-lib:fref ab-%data% (klnew 1)
                           ((1 mmax) (1 *)) ab-%offset%)
            (f2cl-lib:fref c-%data% (ji) ((1 *))
                           c-%offset%))
      (setf (f2cl-lib:fref nab-%data% (klnew 1)
                           ((1 mmax) (1 *)) nab-%offset%)
            (f2cl-lib:fref iwork-%data% (ji) ((1 *))
                           iwork-%offset%))
```

```
            (setf (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                                 ab-%offset%)
                    (f2cl-lib:fref c-%data% (ji) ((1 *))
                                   c-%offset%))
            (setf (f2cl-lib:fref nab-%data% (ji 2) ((1 mmax) (1 *))
                                 nab-%offset%)
                    (f2cl-lib:fref iwork-%data% (ji) ((1 *))
                                   iwork-%offset%)))
           (t (setf info (f2cl-lib:int-add mmax 1))))))
       label70))
    (if (/= info 0)
        (go end_label))
    (setf kl klnew))
   (t
    (f2cl-lib:fdo (ji kf (f2cl-lib:int-add ji 1))
                  ((> ji kl) nil)
      (tagbody
        (cond
         ((<= (f2cl-lib:fref iwork (ji) ((1 *)))
              (f2cl-lib:fref nval (ji) ((1 *))))
          (setf (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
                               ab-%offset%)
                  (f2cl-lib:fref c-%data% (ji) ((1 *)) c-%offset%))
          (setf (f2cl-lib:fref nab-%data% (ji 1) ((1 mmax) (1 *))
                               nab-%offset%)
                  (f2cl-lib:fref iwork-%data% (ji) ((1 *))
                                 iwork-%offset%))))
        (cond
         ((>= (f2cl-lib:fref iwork (ji) ((1 *)))
              (f2cl-lib:fref nval (ji) ((1 *))))
          (setf (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                               ab-%offset%)
                  (f2cl-lib:fref c-%data% (ji) ((1 *)) c-%offset%))
          (setf (f2cl-lib:fref nab-%data% (ji 2) ((1 mmax) (1 *))
                               nab-%offset%)
                  (f2cl-lib:fref iwork-%data% (ji) ((1 *))
                                 iwork-%offset%))))
       label80)))))
(t (setf klnew kl)
 (f2cl-lib:fdo (ji kf (f2cl-lib:int-add ji 1))
               ((> ji kl) nil)
   (tagbody
     (setf tmp1 (f2cl-lib:fref c-%data% (ji) ((1 *)) c-%offset%))
     (setf tmp2
             (- (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
                tmp1))
     (setf itmp1 0)
     (cond
      ((<= tmp2 pivmin) (setf itmp1 1)
       (setf tmp2 (min tmp2 (- pivmin)))))
```

```
(f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf tmp2
            (-
             (+
              (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
              (/
               (-
                (f2cl-lib:fref e2-%data%
                               ((f2cl-lib:int-sub j 1))
                               ((1 *)) e2-%offset%))
               tmp2))
             tmp1))
    (cond
     ((<= tmp2 pivmin)
      (setf itmp1 (f2cl-lib:int-add itmp1 1))
      (setf tmp2 (min tmp2 (- pivmin)))))
   label90))
(cond
 ((<= ijob 2)
  (setf itmp1
          (min
           (the f2cl-lib:integer4
                (f2cl-lib:fref nab-%data% (ji 2)
                               ((1 mmax) (1 *))
                               nab-%offset%))
           (the f2cl-lib:integer4
                (max
                 (the f2cl-lib:integer4
                      (f2cl-lib:fref nab-%data% (ji 1)
                                     ((1 mmax) (1 *))
                                     nab-%offset%))
                 (the f2cl-lib:integer4 itmp1)))))
  (cond
   ((= itmp1 (f2cl-lib:fref nab (ji 2) ((1 mmax) (1 *))))
    (setf (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                         ab-%offset%)
          tmp1))
   ((= itmp1 (f2cl-lib:fref nab (ji 1) ((1 mmax) (1 *))))
    (setf (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
                         ab-%offset%)
          tmp1))
   ((< klnew mmax) (setf klnew (f2cl-lib:int-add klnew 1))
    (setf (f2cl-lib:fref ab-%data% (klnew 2) ((1 mmax) (1 *))
                         ab-%offset%)
          (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                           ab-%offset%))
    (setf (f2cl-lib:fref nab-%data% (klnew 2)
                         ((1 mmax) (1 *)) nab-%offset%)
```

```
                (f2cl-lib:fref nab-%data% (ji 2) ((1 mmax) (1 *))
                               nab-%offset%))
            (setf (f2cl-lib:fref ab-%data% (klnew 1) ((1 mmax) (1 *))
                                 ab-%offset%)
                  tmp1)
            (setf (f2cl-lib:fref nab-%data% (klnew 1)
                                 ((1 mmax) (1 *)) nab-%offset%)
                  itmp1)
            (setf (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                                 ab-%offset%)
                  tmp1)
            (setf (f2cl-lib:fref nab-%data% (ji 2) ((1 mmax) (1 *))
                                 nab-%offset%)
                  itmp1))
           (t (setf info (f2cl-lib:int-add mmax 1)) (go end_label))))
          (t
           (cond
            ((<= itmp1 (f2cl-lib:fref nval (ji) ((1 *))))
             (setf (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
                                  ab-%offset%)
                   tmp1)
             (setf (f2cl-lib:fref nab-%data% (ji 1) ((1 mmax) (1 *))
                                  nab-%offset%)
                   itmp1)))
           (cond
            ((>= itmp1 (f2cl-lib:fref nval (ji) ((1 *))))
             (setf (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                                  ab-%offset%)
                   tmp1)
             (setf (f2cl-lib:fref nab-%data% (ji 2) ((1 mmax) (1 *))
                                  nab-%offset%)
                   itmp1)))))
      label100))
  (setf kl klnew)))
(setf kfnew kf)
(f2cl-lib:fdo (ji kf (f2cl-lib:int-add ji 1))
              ((> ji kl) nil)
  (tagbody
    (setf tmp1
            (abs
             (-
              (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                             ab-%offset%)
              (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
                             ab-%offset%))))
    (setf tmp2
            (max
             (abs
              (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                             ab-%offset%))
```

```
          (abs
           (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
                          ab-%offset%))))
(cond
 ((or (< tmp1 (max abstol pivmin (* reltol tmp2)))
      (>= (f2cl-lib:fref nab (ji 1) ((1 mmax) (1 *)))
          (f2cl-lib:fref nab (ji 2) ((1 mmax) (1 *)))))
  (cond
   ((> ji kfnew)
    (setf tmp1
            (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
                           ab-%offset%))
    (setf tmp2
            (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                           ab-%offset%))
    (setf itmp1
            (f2cl-lib:fref nab-%data% (ji 1) ((1 mmax) (1 *))
                           nab-%offset%))
    (setf itmp2
            (f2cl-lib:fref nab-%data% (ji 2) ((1 mmax) (1 *))
                           nab-%offset%))
    (setf (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
                         ab-%offset%)
            (f2cl-lib:fref ab-%data% (kfnew 1) ((1 mmax) (1 *))
                           ab-%offset%))
    (setf (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                         ab-%offset%)
            (f2cl-lib:fref ab-%data% (kfnew 2) ((1 mmax) (1 *))
                           ab-%offset%))
    (setf (f2cl-lib:fref nab-%data% (ji 1) ((1 mmax) (1 *))
                         nab-%offset%)
            (f2cl-lib:fref nab-%data% (kfnew 1)
                           ((1 mmax) (1 *)) nab-%offset%))
    (setf (f2cl-lib:fref nab-%data% (ji 2) ((1 mmax) (1 *))
                         nab-%offset%)
            (f2cl-lib:fref nab-%data% (kfnew 2)
                           ((1 mmax) (1 *)) nab-%offset%))
    (setf (f2cl-lib:fref ab-%data% (kfnew 1) ((1 mmax) (1 *))
                         ab-%offset%)
            tmp1)
    (setf (f2cl-lib:fref ab-%data% (kfnew 2) ((1 mmax) (1 *))
                         ab-%offset%)
            tmp2)
    (setf (f2cl-lib:fref nab-%data% (kfnew 1) ((1 mmax) (1 *))
                         nab-%offset%)
            itmp1)
    (setf (f2cl-lib:fref nab-%data% (kfnew 2) ((1 mmax) (1 *))
                         nab-%offset%)
            itmp2)
    (cond
```

```
                    ((= ijob 3)
                     (setf itmp1
                             (f2cl-lib:fref nval-%data% (ji) ((1 *))
                                            nval-%offset%))
                     (setf (f2cl-lib:fref nval-%data% (ji) ((1 *))
                                          nval-%offset%)
                             (f2cl-lib:fref nval-%data% (kfnew) ((1 *))
                                            nval-%offset%))
                     (setf (f2cl-lib:fref nval-%data% (kfnew) ((1 *))
                                          nval-%offset%)
                             itmp1)))))
                  (setf kfnew (f2cl-lib:int-add kfnew 1))))
               label110))
            (setf kf kfnew)
            (f2cl-lib:fdo (ji kf (f2cl-lib:int-add ji 1))
                          ((> ji kl) nil)
              (tagbody
                (setf (f2cl-lib:fref c-%data% (ji) ((1 *)) c-%offset%)
                        (* half
                           (+
                            (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
                                           ab-%offset%)
                            (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                                           ab-%offset%))))
               label120))
            (if (> kf kl)
                (go label140))
           label130))
       label140
        (setf info
                (max
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-sub (f2cl-lib:int-add kl 1) kf))
                 (the f2cl-lib:integer4 0)))
        (setf mout kl)
        (go end_label)
       end_label
        (return
         (values nil nil nil nil nil nil nil nil nil nil nil nil nil nil nil
                 mout nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlaebz
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
```

```
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (double-float) (double-float)
  (double-float)
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (array double-float (*))
  (array double-float (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (array double-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil nil nil nil nil nil
  nil nil nil nil nil nil nil
  fortran-to-lisp::mout nil nil nil
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

90.7 dlaev2

90.7.1 Axiom unit tests

— dlaev2.input —

```
)set break resume
)sys rm -f dlaev2.output
)spool dlaev2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.7.2 Axiom help page

— dlaev2.help —

```
====================================================================
dlaev2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlaev2 (A, B, C, RT1, RT2, CS1, SN1)
           DLAEV2 computes the eigenvalues and eigenvectors of a 2-by-2
           symmetric/Hermitian matrix.

Function/Subroutine Documentation
   subroutine dlaev2 (double precision A, double precision B, double precision
       C, double precision RT1, double precision RT2, double precision CS1,
       double precision SN1)
       DLAEV2 computes the eigenvalues and eigenvectors of a 2-by-2
       symmetric/Hermitian matrix.

Purpose:
   DLAEV2 computes the eigendecomposition of a 2-by-2 symmetric matrix
      [  A   B  ]
      [  B   C  ].
   On return, RT1 is the eigenvalue of larger absolute value, RT2 is the
   eigenvalue of smaller absolute value, and (CS1,SN1) is the unit right
   eigenvector for RT1, giving the decomposition

      [ CS1  SN1 ] [  A   B  ] [ CS1 -SN1 ]  =  [ RT1  0  ]
      [-SN1  CS1 ] [  B   C  ] [ SN1  CS1 ]     [  0  RT2 ].

Parameters:

 A
   A is DOUBLE PRECISION
   The (1,1) element of the 2-by-2 matrix.

 B
   B is DOUBLE PRECISION
   The (1,2) element and the conjugate of the (2,1) element of
   the 2-by-2 matrix.

 C
   C is DOUBLE PRECISION
```

```
   The (2,2) element of the 2-by-2 matrix.

 RT1
   RT1 is DOUBLE PRECISION
   The eigenvalue of larger absolute value.

 RT2
   RT2 is DOUBLE PRECISION
   The eigenvalue of smaller absolute value.

 CS1
   CS1 is DOUBLE PRECISION

 SN1
   SN1 is DOUBLE PRECISION
   The vector (CS1, SN1) is a unit right eigenvector for RT1.

Further Details:
   RT1 is accurate to a few ulps barring over/underflow.

   RT2 may be inaccurate if there is massive cancellation in the
   determinant A*C-B*B; higher precision or correctly rounded or
   correctly truncated arithmetic would be needed to compute RT2
   accurately in all cases.

   CS1 and SN1 are accurate to a few ulps barring over/underflow.

   Overflow is possible only if RT1 is within a factor of 5 of overflow.
   Underflow is harmless if the input data is 0 or exceeds
   underflow_threshold / macheps.
```

———-

90.7.3 fortran code

— dlaev2.f —

```
*  =====================================================================
      SUBROUTINE DLAEV2( A, B, C, RT1, RT2, CS1, SN1 )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   A, B, C, CS1, RT1, RT2, SN1
```

```
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D0 )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D0 )
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
      DOUBLE PRECISION   HALF
      PARAMETER          ( HALF = 0.5D0 )
*     ..
*     .. Local Scalars ..
      INTEGER            SGN1, SGN2
      DOUBLE PRECISION   AB, ACMN, ACMX, ACS, ADF, CS, CT, DF, RT, SM,
     $                   TB, TN
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SQRT
*     ..
*     .. Executable Statements ..
*
*     Compute the eigenvalues
*
      SM = A + C
      DF = A - C
      ADF = ABS( DF )
      TB = B + B
      AB = ABS( TB )
      IF( ABS( A ).GT.ABS( C ) ) THEN
         ACMX = A
         ACMN = C
      ELSE
         ACMX = C
         ACMN = A
      END IF
      IF( ADF.GT.AB ) THEN
         RT = ADF*SQRT( ONE+( AB / ADF )**2 )
      ELSE IF( ADF.LT.AB ) THEN
         RT = AB*SQRT( ONE+( ADF / AB )**2 )
      ELSE
*
*        Includes case AB=ADF=0
*
         RT = AB*SQRT( TWO )
      END IF
      IF( SM.LT.ZERO ) THEN
         RT1 = HALF*( SM-RT )
```

```
         SGN1 = -1
*
*        Order of execution important.
*        To get fully accurate smaller eigenvalue,
*        next line needs to be executed in higher precision.
*
         RT2 = ( ACMX / RT1 )*ACMN - ( B / RT1 )*B
      ELSE IF( SM.GT.ZERO ) THEN
         RT1 = HALF*( SM+RT )
         SGN1 = 1
*
*        Order of execution important.
*        To get fully accurate smaller eigenvalue,
*        next line needs to be executed in higher precision.
*
         RT2 = ( ACMX / RT1 )*ACMN - ( B / RT1 )*B
      ELSE
*
*        Includes case RT1 = RT2 = 0
*
         RT1 = HALF*RT
         RT2 = -HALF*RT
         SGN1 = 1
      END IF
*
*     Compute the eigenvector
*
      IF( DF.GE.ZERO ) THEN
         CS = DF + RT
         SGN2 = 1
      ELSE
         CS = DF - RT
         SGN2 = -1
      END IF
      ACS = ABS( CS )
      IF( ACS.GT.AB ) THEN
         CT = -TB / CS
         SN1 = ONE / SQRT( ONE+CT*CT )
         CS1 = CT*SN1
      ELSE
         IF( AB.EQ.ZERO ) THEN
            CS1 = ONE
            SN1 = ZERO
         ELSE
            TN = -CS / TB
            CS1 = ONE / SQRT( ONE+TN*TN )
            SN1 = TN*CS1
         END IF
      END IF
      IF( SGN1.EQ.SGN2 ) THEN
```

```
         TN = CS1
         CS1 = -SN1
         SN1 = TN
      END IF
      RETURN
*
*     End of DLAEV2
*
      END
```

90.7.4 lisp code

— dlaev2.lisp —

```
(let* ((one 1.0d0) (two 2.0d0) (zero 0.0d0) (half 0.5d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 0.5d0 0.5d0) half)
           (ignorable one two zero half))
  (defun dlaev2 (a b c rt1 rt2 cs1 sn1)
    (declare (type (double-float) sn1 cs1 rt2 rt1 c b a))
    (prog ((ab 0.0d0) (acmn 0.0d0) (acmx 0.0d0) (acs 0.0d0) (adf 0.0d0)
           (cs 0.0d0) (ct 0.0d0) (df 0.0d0) (rt 0.0d0) (sm 0.0d0) (tb 0.0d0)
           (tn 0.0d0) (sgn1 0) (sgn2 0))
      (declare (type (double-float) ab acmn acmx acs adf cs ct df rt sm tb tn)
               (type (f2cl-lib:integer4) sgn1 sgn2))
      (setf sm (+ a c))
      (setf df (- a c))
      (setf adf (abs df))
      (setf tb (+ b b))
      (setf ab (abs tb))
      (cond ((> (abs a) (abs c)) (setf acmx a) (setf acmn c))
            (t (setf acmx c) (setf acmn a)))
      (cond
       ((> adf ab)
        (setf rt (* adf (f2cl-lib:fsqrt (+ one (expt (/ ab adf) 2))))))
       ((< adf ab)
        (setf rt (* ab (f2cl-lib:fsqrt (+ one (expt (/ adf ab) 2))))))
       (t (setf rt (* ab (f2cl-lib:fsqrt two)))))
      (cond
       ((< sm zero) (setf rt1 (* half (- sm rt))) (setf sgn1 -1)
        (setf rt2 (- (* (/ acmx rt1) acmn) (* (/ b rt1) b))))
       ((> sm zero) (setf rt1 (* half (+ sm rt))) (setf sgn1 1)
        (setf rt2 (- (* (/ acmx rt1) acmn) (* (/ b rt1) b))))
       (t (setf rt1 (* half rt)) (setf rt2 (* (- half) rt)) (setf sgn1 1)))
```

```
      (cond ((>= df zero) (setf cs (+ df rt)) (setf sgn2 1))
            (t (setf cs (- df rt)) (setf sgn2 -1)))
      (setf acs (abs cs))
      (cond
       ((> acs ab) (setf ct (/ (- tb) cs))
        (setf sn1 (/ one (f2cl-lib:fsqrt (+ one (* ct ct)))))
        (setf cs1 (* ct sn1)))
       (t
        (cond ((= ab zero) (setf cs1 one) (setf sn1 zero))
              (t (setf tn (/ (- cs) tb))
               (setf cs1 (/ one (f2cl-lib:fsqrt (+ one (* tn tn)))))
               (setf sn1 (* tn cs1))))))
      (cond ((= sgn1 sgn2) (setf tn cs1) (setf cs1 (- sn1)) (setf sn1 tn)))
      (go end_label)
     end_label
      (return (values nil nil nil rt1 rt2 cs1 sn1)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlaev2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((double-float) (double-float)
                                              (double-float) (double-float)
                                              (double-float) (double-float)
                                              (double-float))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::rt1
                                              fortran-to-lisp::rt2
                                              fortran-to-lisp::cs1
                                              fortran-to-lisp::sn1)
                                            :calls 'nil)))
```

———-

90.8 dlagts

90.8.1 Axiom unit tests

— **dlagts.input** —

```
)set break resume
)sys rm -f dlagts.output
)spool dlagts.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.8.2 Axiom help page

— dlagts.help —

```
====================================================================
dlagts examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlagts (JOB, N, A, B, C, D, IN, Y, TOL, INFO)
           DLAGTS solves the system of equations (T-I)x = y or (T-I)Tx =
           y,where T is a general tridiagonal matrix and  a scalar, using the
           LU factorization computed by slagtf.

Function/Subroutine Documentation
   subroutine dlagts (integer JOB, integer N, double precision, dimension( * )
       A, double precision, dimension( * ) B, double precision, dimension( * )
       C, double precision, dimension( * ) D, integer, dimension( * ) IN,
       double precision, dimension( * ) Y, double precision TOL, integer INFO)
       DLAGTS solves the system of equations (T-I)x = y or (T-I)Tx = y,where
       T is a general tridiagonal matrix and  a scalar, using the LU
       factorization computed by slagtf.

Purpose:
   DLAGTS may be used to solve one of the systems of equations

      (T - lambda*I)*x = y   or   (T - lambda*I)**T*x = y,

   where T is an n by n tridiagonal matrix, for x, following the
   factorization of (T - lambda*I) as

      (T - lambda*I) = P*L*U ,

   by routine DLAGTF. The choice of equation to be solved is
```

```
  controlled by the argument JOB, and in each case there is an option
  to perturb zero or very small diagonal elements of U, this option
  being intended for use in applications such as inverse iteration.

Parameters:

 JOB
   JOB is INTEGER
   Specifies the job to be performed by DLAGTS as follows:
   =  1: The equations  (T - lambda*I)x = y  are to be solved,
      but diagonal elements of U are not to be perturbed.
   = -1: The equations  (T - lambda*I)x = y  are to be solved
      and, if overflow would otherwise occur, the diagonal
      elements of U are to be perturbed. See argument TOL below.
   =  2: The equations  (T - lambda*I)**Tx = y  are to be solved,
      but diagonal elements of U are not to be perturbed.
   = -2: The equations  (T - lambda*I)**Tx = y  are to be solved
      and, if overflow would otherwise occur, the diagonal
      elements of U are to be perturbed. See argument TOL below.

 N
   N is INTEGER
   The order of the matrix T.

 A
   A is DOUBLE PRECISION array, dimension (N)
   On entry, A must contain the diagonal elements of U as
   returned from DLAGTF.

 B
   B is DOUBLE PRECISION array, dimension (N-1)
   On entry, B must contain the first super-diagonal elements of
   U as returned from DLAGTF.

 C
   C is DOUBLE PRECISION array, dimension (N-1)
   On entry, C must contain the sub-diagonal elements of L as
   returned from DLAGTF.

 D
   D is DOUBLE PRECISION array, dimension (N-2)
   On entry, D must contain the second super-diagonal elements
   of U as returned from DLAGTF.

 IN
   IN is INTEGER array, dimension (N)
   On entry, IN must contain details of the matrix P as returned from DLAGTF.

 Y
   Y is DOUBLE PRECISION array, dimension (N)
```

```
  On entry, the right hand side vector y.
  On exit, Y is overwritten by the solution vector x.

TOL
  TOL is DOUBLE PRECISION
  On entry, with  JOB .lt. 0, TOL should be the minimum
  perturbation to be made to very small diagonal elements of U.
  TOL should normally be chosen as about eps*norm(U), where eps
  is the relative machine precision, but if TOL is supplied as
  non-positive, then it is reset to eps*max( abs( u(i,j) ) ).
  If  JOB .gt. 0  then TOL is not referenced.

  On exit, TOL is changed as described above, only if TOL is
  non-positive on entry. Otherwise TOL is unchanged.

INFO
  INFO is INTEGER
  = 0   : successful exit
  .lt. 0: if INFO = -i, the i-th argument had an illegal value
  .gt. 0: overflow would occur when computing the INFO(th)
          element of the solution vector x. This can only occur
          when JOB is supplied as positive and either means
          that a diagonal element of U is very small, or that
          the elements of the right-hand side vector y are very large.
```

90.8.3 fortran code

— dlagts.f —

```
*  ===================================================================
      SUBROUTINE DLAGTS( JOB, N, A, B, C, D, IN, Y, TOL, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, JOB, N
      DOUBLE PRECISION   TOL
*     ..
*     .. Array Arguments ..
      INTEGER            IN( * )
      DOUBLE PRECISION   A( * ), B( * ), C( * ), D( * ), Y( * )
*     ..
```

```
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            K
      DOUBLE PRECISION   ABSAK, AK, BIGNUM, EPS, PERT, SFMIN, TEMP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SIGN
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      IF( ( ABS( JOB ).GT.2 ) .OR. ( JOB.EQ.0 ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLAGTS', -INFO )
         RETURN
      END IF
*
      IF( N.EQ.0 )
     $   RETURN
*
      EPS = DLAMCH( 'Epsilon' )
      SFMIN = DLAMCH( 'Safe minimum' )
      BIGNUM = ONE / SFMIN
*
      IF( JOB.LT.0 ) THEN
         IF( TOL.LE.ZERO ) THEN
            TOL = ABS( A( 1 ) )
            IF( N.GT.1 )
     $         TOL = MAX( TOL, ABS( A( 2 ) ), ABS( B( 1 ) ) )
            DO 10 K = 3, N
               TOL = MAX( TOL, ABS( A( K ) ), ABS( B( K-1 ) ),
     $               ABS( D( K-2 ) ) )
   10       CONTINUE
```

```
            TOL = TOL*EPS
            IF( TOL.EQ.ZERO )
     $         TOL = EPS
         END IF
      END IF
*
      IF( ABS( JOB ).EQ.1 ) THEN
         DO 20 K = 2, N
            IF( IN( K-1 ).EQ.0 ) THEN
               Y( K ) = Y( K ) - C( K-1 )*Y( K-1 )
            ELSE
               TEMP = Y( K-1 )
               Y( K-1 ) = Y( K )
               Y( K ) = TEMP - C( K-1 )*Y( K )
            END IF
   20    CONTINUE
         IF( JOB.EQ.1 ) THEN
            DO 30 K = N, 1, -1
               IF( K.LE.N-2 ) THEN
                  TEMP = Y( K ) - B( K )*Y( K+1 ) - D( K )*Y( K+2 )
               ELSE IF( K.EQ.N-1 ) THEN
                  TEMP = Y( K ) - B( K )*Y( K+1 )
               ELSE
                  TEMP = Y( K )
               END IF
               AK = A( K )
               ABSAK = ABS( AK )
               IF( ABSAK.LT.ONE ) THEN
                  IF( ABSAK.LT.SFMIN ) THEN
                     IF( ABSAK.EQ.ZERO .OR. ABS( TEMP )*SFMIN.GT.ABSAK )
     $                    THEN
                        INFO = K
                        RETURN
                     ELSE
                        TEMP = TEMP*BIGNUM
                        AK = AK*BIGNUM
                     END IF
                  ELSE IF( ABS( TEMP ).GT.ABSAK*BIGNUM ) THEN
                     INFO = K
                     RETURN
                  END IF
               END IF
               Y( K ) = TEMP / AK
   30       CONTINUE
         ELSE
            DO 50 K = N, 1, -1
               IF( K.LE.N-2 ) THEN
                  TEMP = Y( K ) - B( K )*Y( K+1 ) - D( K )*Y( K+2 )
               ELSE IF( K.EQ.N-1 ) THEN
                  TEMP = Y( K ) - B( K )*Y( K+1 )
```

```
            ELSE
               TEMP = Y( K )
            END IF
            AK = A( K )
            PERT = SIGN( TOL, AK )
   40       CONTINUE
            ABSAK = ABS( AK )
            IF( ABSAK.LT.ONE ) THEN
               IF( ABSAK.LT.SFMIN ) THEN
                  IF( ABSAK.EQ.ZERO .OR. ABS( TEMP )*SFMIN.GT.ABSAK )
     $                 THEN
                     AK = AK + PERT
                     PERT = 2*PERT
                     GO TO 40
                  ELSE
                     TEMP = TEMP*BIGNUM
                     AK = AK*BIGNUM
                  END IF
               ELSE IF( ABS( TEMP ).GT.ABSAK*BIGNUM ) THEN
                  AK = AK + PERT
                  PERT = 2*PERT
                  GO TO 40
               END IF
            END IF
            Y( K ) = TEMP / AK
   50    CONTINUE
      END IF
    ELSE
*
*     Come to here if  JOB = 2 or -2
*
      IF( JOB.EQ.2 ) THEN
         DO 60 K = 1, N
            IF( K.GE.3 ) THEN
               TEMP = Y( K ) - B( K-1 )*Y( K-1 ) - D( K-2 )*Y( K-2 )
            ELSE IF( K.EQ.2 ) THEN
               TEMP = Y( K ) - B( K-1 )*Y( K-1 )
            ELSE
               TEMP = Y( K )
            END IF
            AK = A( K )
            ABSAK = ABS( AK )
            IF( ABSAK.LT.ONE ) THEN
               IF( ABSAK.LT.SFMIN ) THEN
                  IF( ABSAK.EQ.ZERO .OR. ABS( TEMP )*SFMIN.GT.ABSAK )
     $                 THEN
                     INFO = K
                     RETURN
                  ELSE
                     TEMP = TEMP*BIGNUM
```

```
                  AK = AK*BIGNUM
               END IF
            ELSE IF( ABS( TEMP ).GT.ABSAK*BIGNUM ) THEN
               INFO = K
               RETURN
            END IF
         END IF
         Y( K ) = TEMP / AK
60    CONTINUE
      ELSE
         DO 80 K = 1, N
            IF( K.GE.3 ) THEN
               TEMP = Y( K ) - B( K-1 )*Y( K-1 ) - D( K-2 )*Y( K-2 )
            ELSE IF( K.EQ.2 ) THEN
               TEMP = Y( K ) - B( K-1 )*Y( K-1 )
            ELSE
               TEMP = Y( K )
            END IF
            AK = A( K )
            PERT = SIGN( TOL, AK )
70          CONTINUE
            ABSAK = ABS( AK )
            IF( ABSAK.LT.ONE ) THEN
               IF( ABSAK.LT.SFMIN ) THEN
                  IF( ABSAK.EQ.ZERO .OR. ABS( TEMP )*SFMIN.GT.ABSAK )
     $                 THEN
                     AK = AK + PERT
                     PERT = 2*PERT
                     GO TO 70
                  ELSE
                     TEMP = TEMP*BIGNUM
                     AK = AK*BIGNUM
                  END IF
               ELSE IF( ABS( TEMP ).GT.ABSAK*BIGNUM ) THEN
                  AK = AK + PERT
                  PERT = 2*PERT
                  GO TO 70
               END IF
            END IF
            Y( K ) = TEMP / AK
80    CONTINUE
      END IF
*
      DO 90 K = N, 2, -1
         IF( IN( K-1 ).EQ.0 ) THEN
            Y( K-1 ) = Y( K-1 ) - C( K-1 )*Y( K )
         ELSE
            TEMP = Y( K-1 )
            Y( K-1 ) = Y( K )
            Y( K ) = TEMP - C( K-1 )*Y( K )
```

```
            END IF
   90    CONTINUE
      END IF
*
*     End of DLAGTS
*
      END
```

———-

90.8.4 lisp code

— **dlagts.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dlagts (job n a b c d in y tol info)
    (declare (type (double-float) tol)
             (type (array f2cl-lib:integer4 (*)) in)
             (type (array double-float (*)) y d c b a)
             (type (f2cl-lib:integer4) info n job))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (b double-float b-%data% b-%offset%)
         (c double-float c-%data% c-%offset%)
         (d double-float d-%data% d-%offset%)
         (y double-float y-%data% y-%offset%)
         (in f2cl-lib:integer4 in-%data% in-%offset%))
      (prog ((absak 0.0d0) (ak 0.0d0) (bignum 0.0d0) (eps 0.0d0) (pert 0.0d0)
             (sfmin 0.0d0) (temp 0.0d0) (k 0))
        (declare (type (double-float) absak ak bignum eps pert sfmin temp)
                 (type (f2cl-lib:integer4) k))
        (setf info 0)
        (cond ((or (> (abs job) 2) (= job 0)) (setf info -1))
              ((< n 0) (setf info -2)))
        (cond
         ((/= info 0) (xerbla "DLAGTS" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (= n 0)
            (go end_label))
        (setf eps (dlamch "Epsilon"))
        (setf sfmin (dlamch "Safe minimum"))
        (setf bignum (/ one sfmin))
        (cond
         ((< job 0)
          (cond
```

```
      ((<= tol zero)
       (setf tol (abs (f2cl-lib:fref a-%data% (1) ((1 *)) a-%offset%)))
       (if (> n 1)
           (setf tol
                   (max tol
                        (abs
                         (f2cl-lib:fref a-%data% (2) ((1 *)) a-%offset%))
                        (abs
                         (f2cl-lib:fref b-%data% (1) ((1 *))
                                        b-%offset%)))))
       (f2cl-lib:fdo (k 3 (f2cl-lib:int-add k 1))
                     ((> k n) nil)
         (tagbody
           (setf tol
                   (max tol
                        (abs
                         (f2cl-lib:fref a-%data% (k) ((1 *)) a-%offset%))
                        (abs
                         (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1))
                                        ((1 *)) b-%offset%))
                        (abs
                         (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub k 2))
                                        ((1 *)) d-%offset%))))
          label10))
       (setf tol (* tol eps))
       (if (= tol zero)
           (setf tol eps))))))
    (cond
     ((= (abs job) 1)
      (f2cl-lib:fdo (k 2 (f2cl-lib:int-add k 1))
                    ((> k n) nil)
       (tagbody
         (cond
          ((=
            (f2cl-lib:fref in ((f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                           ((1 *)))
            0)
           (setf (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                   (- (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                      (*
                       (f2cl-lib:fref c-%data% ((f2cl-lib:int-sub k 1))
                                      ((1 *)) c-%offset%)
                       (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1))
                                      ((1 *)) y-%offset%)))))
          (t
           (setf temp
                   (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1))
                                  ((1 *)) y-%offset%))
           (setf (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                                y-%offset%)
```

```
                (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%))
        (setf (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                (- temp
                   (*
                    (f2cl-lib:fref c-%data% ((f2cl-lib:int-sub k 1))
                                   ((1 *)) c-%offset%)
                    (f2cl-lib:fref y-%data% (k) ((1 *))
                                   y-%offset%))))))
    label20))
  (cond
   ((= job 1)
    (f2cl-lib:fdo (k n (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                  ((> k 1) nil)
      (tagbody
        (cond
         ((<= k (f2cl-lib:int-add n (f2cl-lib:int-sub 2)))
          (setf temp
                  (- (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                     (* (f2cl-lib:fref b-%data% (k) ((1 *)) b-%offset%)
                        (f2cl-lib:fref y-%data%
                                       ((f2cl-lib:int-add k 1)) ((1 *))
                                       y-%offset%))
                     (* (f2cl-lib:fref d-%data% (k) ((1 *)) d-%offset%)
                        (f2cl-lib:fref y-%data%
                                       ((f2cl-lib:int-add k 2)) ((1 *))
                                       y-%offset%)))))
         ((= k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
          (setf temp
                  (- (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                     (* (f2cl-lib:fref b-%data% (k) ((1 *)) b-%offset%)
                        (f2cl-lib:fref y-%data%
                                       ((f2cl-lib:int-add k 1)) ((1 *))
                                       y-%offset%)))))
         (t
          (setf temp (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%))))
        (setf ak (f2cl-lib:fref a-%data% (k) ((1 *)) a-%offset%))
        (setf absak (abs ak))
        (cond
         ((< absak one)
          (cond
           ((< absak sfmin)
            (cond
             ((or (= absak zero) (> (* (abs temp) sfmin) absak))
              (setf info k) (go end_label))
             (t (setf temp (* temp bignum)) (setf ak (* ak bignum)))))
           ((> (abs temp) (* absak bignum)) (setf info k)
            (go end_label)))))
        (setf (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                (/ temp ak))
       label30)))
```

```
      (t
       (f2cl-lib:fdo (k n (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                     ((> k 1) nil)
         (tagbody
           (cond
            ((<= k (f2cl-lib:int-add n (f2cl-lib:int-sub 2)))
             (setf temp
                     (- (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                        (* (f2cl-lib:fref b-%data% (k) ((1 *)) b-%offset%)
                           (f2cl-lib:fref y-%data%
                                          ((f2cl-lib:int-add k 1)) ((1 *))
                                          y-%offset%))
                        (* (f2cl-lib:fref d-%data% (k) ((1 *)) d-%offset%)
                           (f2cl-lib:fref y-%data%
                                          ((f2cl-lib:int-add k 2)) ((1 *))
                                          y-%offset%)))))
            ((= k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
             (setf temp
                     (- (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                        (* (f2cl-lib:fref b-%data% (k) ((1 *)) b-%offset%)
                           (f2cl-lib:fref y-%data%
                                          ((f2cl-lib:int-add k 1)) ((1 *))
                                          y-%offset%)))))
            (t
             (setf temp (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%))))
           (setf ak (f2cl-lib:fref a-%data% (k) ((1 *)) a-%offset%))
           (setf pert (f2cl-lib:sign tol ak))
          label40
           (setf absak (abs ak))
           (cond
            ((< absak one)
             (cond
              ((< absak sfmin)
               (cond
                ((or (= absak zero) (> (* (abs temp) sfmin) absak))
                 (setf ak (+ ak pert)) (setf pert (* 2 pert))
                 (go label40))
                (t (setf temp (* temp bignum)) (setf ak (* ak bignum)))))
              ((> (abs temp) (* absak bignum)) (setf ak (+ ak pert))
               (setf pert (* 2 pert)) (go label40)))))
           (setf (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                 (/ temp ak))
          label50)))))
    (t
     (cond
      ((= job 2)
       (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                     ((> k n) nil)
         (tagbody
           (cond
```

```
        ((>= k 3)
         (setf temp
                 (- (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                    (*
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1))
                                    ((1 *)) b-%offset%)
                     (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1))
                                    ((1 *)) y-%offset%))
                    (*
                     (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub k 2))
                                    ((1 *)) d-%offset%)
                     (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 2))
                                    ((1 *)) y-%offset%)))))
        ((= k 2)
         (setf temp
                 (- (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                    (*
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1))
                                    ((1 *)) b-%offset%)
                     (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1))
                                    ((1 *)) y-%offset%)))))
        (t
         (setf temp (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%))))
       (setf ak (f2cl-lib:fref a-%data% (k) ((1 *)) a-%offset%))
       (setf absak (abs ak))
       (cond
        ((< absak one)
         (cond
          ((< absak sfmin)
           (cond
            ((or (= absak zero) (> (* (abs temp) sfmin) absak))
             (setf info k) (go end_label))
            (t (setf temp (* temp bignum)) (setf ak (* ak bignum)))))
          ((> (abs temp) (* absak bignum)) (setf info k)
           (go end_label)))))
       (setf (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
               (/ temp ak))
     label60)))
  (t
   (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                 ((> k n) nil)
     (tagbody
       (cond
        ((>= k 3)
         (setf temp
                 (- (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                    (*
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1))
                                    ((1 *)) b-%offset%)
                     (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1))
```

```
                                        ((1 *)) y-%offset%))
                              (*
                               (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub k 2))
                                              ((1 *)) d-%offset%)
                               (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 2))
                                              ((1 *)) y-%offset%)))))
                 ((= k 2)
                  (setf temp
                          (- (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                             (*
                              (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1))
                                              ((1 *)) b-%offset%)
                              (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1))
                                              ((1 *)) y-%offset%)))))
                 (t
                  (setf temp (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%))))
                 (setf ak (f2cl-lib:fref a-%data% (k) ((1 *)) a-%offset%))
                 (setf pert (f2cl-lib:sign tol ak))
                label70
                 (setf absak (abs ak))
                 (cond
                  ((< absak one)
                   (cond
                    ((< absak sfmin)
                     (cond
                      ((or (= absak zero) (> (* (abs temp) sfmin) absak))
                       (setf ak (+ ak pert)) (setf pert (* 2 pert))
                       (go label70))
                      (t (setf temp (* temp bignum)) (setf ak (* ak bignum)))))
                    ((> (abs temp) (* absak bignum)) (setf ak (+ ak pert))
                     (setf pert (* 2 pert)) (go label70)))))
                 (setf (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                         (/ temp ak))
                label80))))
            (f2cl-lib:fdo (k n (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                          ((> k 2) nil)
              (tagbody
                (cond
                 ((=
                   (f2cl-lib:fref in ((f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                                  ((1 *)))
                   0)
                 (setf (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                                      y-%offset%)
                         (-
                          (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1))
                                         ((1 *)) y-%offset%)
                          (*
                           (f2cl-lib:fref c-%data% ((f2cl-lib:int-sub k 1))
                                          ((1 *)) c-%offset%)
```

```
                        (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)))))
                (t
                 (setf temp
                         (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1))
                                        ((1 *)) y-%offset%))
                 (setf (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                                      y-%offset%)
                         (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%))
                 (setf (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                         (- temp
                            (*
                             (f2cl-lib:fref c-%data% ((f2cl-lib:int-sub k 1))
                                            ((1 *)) c-%offset%)
                             (f2cl-lib:fref y-%data% (k) ((1 *))
                                            y-%offset%))))))
              label90))))
        end_label
         (return (values nil nil nil nil nil nil nil nil tol info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlagts
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array double-float (*))
                                              (double-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
                                              fortran-to-lisp::tol
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

90.9 dlaisnan

90.9.1 Axiom unit tests

— dlaisnan.input —

```
)set break resume
)sys rm -f dlaisnan.output
)spool dlaisnan.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.9.2 Axiom help page

— dlaisnan.help —

```
======================================================================
dlaisnan examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       logical function dlaisnan (DIN1, DIN2)
           DLAISNAN tests input for NaN by comparing two arguments for
           inequality.

Function/Subroutine Documentation
   logical function dlaisnan (double precision DIN1, double precision DIN2)
       DLAISNAN tests input for NaN by comparing two arguments for inequality.

Purpose:
   This routine is not for general use.  It exists solely to avoid
   over-optimization in DISNAN.

   DLAISNAN checks for NaNs by comparing its two arguments for
   inequality.  NaN is the only floating-point value where NaN != NaN
```

```
   returns .TRUE.  To check for NaNs, pass the same variable as both
   arguments.

   A compiler must assume that the two arguments are
   not the same variable, and the test will not be optimized away.
   Interprocedural or whole-program optimization may delete this
   test.  The ISNAN functions will be replaced by the correct
   Fortran 03 intrinsic once the intrinsic is widely available.

Parameters:

 DIN1
   DIN1 is DOUBLE PRECISION

 DIN2
   DIN2 is DOUBLE PRECISION
   Two numbers to compare for inequality.
```

90.9.3 fortran code

— dlaisnan.f —

```
*  =====================================================================
      LOGICAL FUNCTION DLAISNAN( DIN1, DIN2 )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   DIN1, DIN2
*     ..
*
*  =====================================================================
*
*  .. Executable Statements ..
      DLAISNAN = (DIN1.NE.DIN2)
      RETURN
      END
```

90.9.4 lisp code

— dlaisnan.lisp —

———-

90.10 dlamc1

90.10.1 Axiom unit tests

— dlamc1.input —

```
)set break resume
)sys rm -f dlamc1.output
)spool dlamc1.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.10.2 Axiom help page

— dlamc1.help —

```
======================================================================
dlamc1 examples
======================================================================

======================================================================
Man Page Details
======================================================================

subroutine dlamc1(Integer BETA, Integer T, Logical RND, Logical IEEE1)

Purpose:
   DLAMC1 determines the machine parameters given by BETA, T, RND, and IEEE1
```

```
BETA
  The base of the machine.

T
  The number of (BETA) digits in the mantissa

RND
  Specifies whether proper rounding (RND=.TRUE.) or chopping (RND=.FALSE.)
  occurs in addition. This may not be a reliable guide to the way in
  which the machine performs its arithmetic.

IEEE1
  Specifies whether rounding appears to be done in the
  'round to nearest' style.
```

———-

90.10.3 fortran code

— dlamc1.f —

———-

90.10.4 lisp code

— dlamc1.lisp —

———-

90.11 dlamc2

90.11.1 Axiom unit tests

— dlamc2.input —

```
)set break resume
)sys rm -f dlamc2.output
```

```
)spool dlamc2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

90.11.2 Axiom help page

— dlamc2.help —

```
=====================================================================
dlamc2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

subroutine dlamc2(Integer BETA, Integer T, Logical RND,
                  Double Precision EPS, Integer EMIN,
                  Double Precision RMIN, Integer EMAX,
                  Double Precision RMAX)
Purpose:
   DLAMC2 determines the machine parameters specified in its argument list.

 BETA
   The base of the machine

 T
   The number of (BETA) digits in the mantissa.

 RND
   Specifies whether proper rounding (RND=.TRUE.) or chopping
   (RND=.FALSE.) occurs in addition. This may not be a reliable guide
   to the way in which the machine performs its arithmetic.

 EPS
   The smallest positive number such that
      fl(1.0-EPS) .LT 1.0,
   where fl denotes the computed value.

 EMIN
   The minimum exponent before (gradual) underflow occurs.
```

```
RMIN
  The smallest normalized number for the machine, given by
     BASE**(EMIN-1)
  where BASE is the floating point value of BETA

EMAX
  The maximum exponent before overflow occurs.

RMAX
  The largest positive number for the machine, given by
     BASE**EMAX*(1-EPS)
  where BASE is the floating point value of BETA
```

———-

90.11.3 fortran code

— dlamc2.f —

———-

90.11.4 lisp code

— dlamc2.lisp —

———-

90.12 dlamc3

90.12.1 Axiom unit tests

— dlamc3.input —

```
)set break resume
)sys rm -f dlamc3.output
)spool dlamc3.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.12.2 Axiom help page

— **dlamc3.help** —

```
==================================================================
dlamc3 examples
==================================================================

==================================================================
Man Page Details
==================================================================

double precision function dlamc3( double precision A, double precision B)

Purpose:
   DLAMC3 is intended to force A and B to be stored prior to doing the
   addition of A and B, for use in situations where optimizers might
   hold one of these in a register.

 A
   A is a Double Precsion

 B
   B is a Double Precision
```

———-

90.12.3 fortran code

— **dlamc3.f** —

———-

90.12.4 lisp code

— dlamc3.lisp —

———-

90.13 dlamch

90.13.1 Axiom unit tests

— dlamch.input —

```
)set break resume
)sys rm -f dlamch.output
)spool dlamch.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.13.2 Axiom help page

— dlamch.help —

```
=====================================================================
dlamch examples
=====================================================================

=====================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function dlamch (CMACH)

   double precision function dlamch (character CMACH)
       DLAMCH
```

```
Purpose:
   DLAMCH determines double precision machine parameters.

Parameters:

 CMACH
   Specifies the value to be returned by DLAMCH:
   = 'E' or 'e',   DLAMCH := eps
   = 'S' or 's ,   DLAMCH := sfmin
   = 'B' or 'b',   DLAMCH := base
   = 'P' or 'p',   DLAMCH := eps*base
   = 'N' or 'n',   DLAMCH := t
   = 'R' or 'r',   DLAMCH := rnd
   = 'M' or 'm',   DLAMCH := emin
   = 'U' or 'u',   DLAMCH := rmin
   = 'L' or 'l',   DLAMCH := emax
   = 'O' or 'o',   DLAMCH := rmax
   where
   eps   = relative machine precision
   sfmin = safe minimum, such that 1/sfmin does not overflow
   base  = base of the machine
   prec  = eps*base
   t     = number of (base) digits in the mantissa
   rnd   = 1.0 when rounding occurs in addition, 0.0 otherwise
   emin  = minimum exponent before (gradual) underflow
   rmin  = underflow threshold - base**(emin-1)
   emax  = largest exponent before overflow
   rmax  = overflow threshold  - (base**emax)*(1-eps)


Version 3.6.0                  Mon Nov 16 2015                    dlamch.f(3)
 Page Details
=====================================================================
```

———-

90.13.3 fortran code

— dlamch.f —

———-

90.13.4 lisp code

— dlamch.lisp —

———-

90.14 dlaneg

90.14.1 Axiom unit tests

— dlaneg.input —

```
)set break resume
)sys rm -f dlaneg.output
)spool dlaneg.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.14.2 Axiom help page

— dlaneg.help —

```
======================================================================
dlaneg examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       integer function dlaneg (N, D, LLD, SIGMA, PIVMIN, R)
           DLANEG computes the Sturm count.
```

```
Function/Subroutine Documentation
   integer function dlaneg (integer N, double precision, dimension( * ) D,
       double precision, dimension( * ) LLD, double precision SIGMA, double
       precision PIVMIN, integer R)
       DLANEG computes the Sturm count.

Purpose:
   DLANEG computes the Sturm count, the number of negative pivots
   encountered while factoring tridiagonal T - sigma I = L D L^T.
   This implementation works directly on the factors without forming
   the tridiagonal matrix T.  The Sturm count is also the number of
   eigenvalues of T less than sigma.

   This routine is called from DLARRB.

   The current routine does not use the PIVMIN parameter but rather
   requires IEEE-754 propagation of Infinities and NaNs.  This
   routine also has no input range restrictions but does require
   default exception handling such that x/0 produces Inf when x is
   non-zero, and Inf/Inf produces NaN.  For more information, see:

   Marques, Riedy, and Voemel, "Benefits of IEEE-754 Features in
   Modern Symmetric Tridiagonal Eigensolvers," SIAM Journal on
   Scientific Computing, v28, n5, 2006.  DOI 10.1137/050641624
   (Tech report version in LAWN 172 with the same title.)

Parameters:

 N
   N is INTEGER
   The order of the matrix.

 D
   D is DOUBLE PRECISION array, dimension (N)
   The N diagonal elements of the diagonal matrix D.

 LLD
   LLD is DOUBLE PRECISION array, dimension (N-1)
   The (N-1) elements L(i)*L(i)*D(i).

 SIGMA
   SIGMA is DOUBLE PRECISION
   Shift amount in T - sigma I = L D L^T.

 PIVMIN
   PIVMIN is DOUBLE PRECISION
   The minimum pivot in the Sturm sequence.  May be used
   when zero pivots are encountered on non-IEEE-754 architectures.

 R
```

```
R is INTEGER
The twist index for the twisted factorization that is used
for the negcount.
```

———-

90.14.3 fortran code

— **dlaneg.f** —

```
*  =====================================================================
      INTEGER FUNCTION DLANEG( N, D, LLD, SIGMA, PIVMIN, R )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            N, R
      DOUBLE PRECISION   PIVMIN, SIGMA
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), LLD( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER        ( ZERO = 0.0D0, ONE = 1.0D0 )
*     Some architectures propagate Infinities and NaNs very slowly, so
*     the code computes counts in BLKLEN chunks.  Then a NaN can
*     propagate at most BLKLEN columns before being detected.  This is
*     not a general tuning parameter; it needs only to be just large
*     enough that the overhead is tiny in common cases.
      INTEGER BLKLEN
      PARAMETER ( BLKLEN = 128 )
*     ..
*     .. Local Scalars ..
      INTEGER            BJ, J, NEG1, NEG2, NEGCNT
      DOUBLE PRECISION   BSAV, DMINUS, DPLUS, GAMMA, P, T, TMP
      LOGICAL SAWNAN
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MIN, MAX
*     ..
```

```
*     .. External Functions ..
      LOGICAL DISNAN
      EXTERNAL DISNAN
*     ..
*     .. Executable Statements ..

      NEGCNT = 0

*     I) upper part: L D L^T - SIGMA I = L+ D+ L+^T
      T = -SIGMA
      DO 210 BJ = 1, R-1, BLKLEN
         NEG1 = 0
         BSAV = T
         DO 21 J = BJ, MIN(BJ+BLKLEN-1, R-1)
            DPLUS = D( J ) + T
            IF( DPLUS.LT.ZERO ) NEG1 = NEG1 + 1
            TMP = T / DPLUS
            T = TMP * LLD( J ) - SIGMA
 21      CONTINUE
         SAWNAN = DISNAN( T )
*     Run a slower version of the above loop if a NaN is detected.
*     A NaN should occur only with a zero pivot after an infinite
*     pivot.  In that case, substituting 1 for T/DPLUS is the
*     correct limit.
         IF( SAWNAN ) THEN
            NEG1 = 0
            T = BSAV
            DO 22 J = BJ, MIN(BJ+BLKLEN-1, R-1)
               DPLUS = D( J ) + T
               IF( DPLUS.LT.ZERO ) NEG1 = NEG1 + 1
               TMP = T / DPLUS
               IF (DISNAN(TMP)) TMP = ONE
               T = TMP * LLD(J) - SIGMA
 22         CONTINUE
         END IF
         NEGCNT = NEGCNT + NEG1
 210  CONTINUE
*
*     II) lower part: L D L^T - SIGMA I = U- D- U-^T
      P = D( N ) - SIGMA
      DO 230 BJ = N-1, R, -BLKLEN
         NEG2 = 0
         BSAV = P
         DO 23 J = BJ, MAX(BJ-BLKLEN+1, R), -1
            DMINUS = LLD( J ) + P
            IF( DMINUS.LT.ZERO ) NEG2 = NEG2 + 1
            TMP = P / DMINUS
            P = TMP * D( J ) - SIGMA
 23      CONTINUE
         SAWNAN = DISNAN( P )
```

```
*     As above, run a slower version that substitutes 1 for Inf/Inf.
*
         IF( SAWNAN ) THEN
            NEG2 = 0
            P = BSAV
            DO 24 J = BJ, MAX(BJ-BLKLEN+1, R), -1
               DMINUS = LLD( J ) + P
               IF( DMINUS.LT.ZERO ) NEG2 = NEG2 + 1
               TMP = P / DMINUS
               IF (DISNAN(TMP)) TMP = ONE
               P = TMP * D(J) - SIGMA
 24         CONTINUE
         END IF
         NEGCNT = NEGCNT + NEG2
 230  CONTINUE
*
*     III) Twist index
*       T was shifted by SIGMA initially.
      GAMMA = (T + SIGMA) + P
      IF( GAMMA.LT.ZERO ) NEGCNT = NEGCNT+1

      DLANEG = NEGCNT
      END
```

———-

90.14.4 lisp code

— dlaneg.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0) (blklen 128))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (f2cl-lib:integer4 128 128) blklen)
           (ignorable zero one blklen))
  (defun dlaneg (n d lld sigma pivmin r)
    (declare (type (double-float) pivmin sigma)
             (type (array double-float (*)) lld d)
             (type (f2cl-lib:integer4) r n))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (lld double-float lld-%data% lld-%offset%))
      (prog ((sawnan nil) (bsav 0.0d0) (dminus 0.0d0) (dplus 0.0d0)
             (gamma 0.0d0) (p 0.0d0) (t$ 0.0d0) (tmp 0.0d0) (bj 0) (j 0)
             (neg1 0) (neg2 0) (negcnt 0) (dlaneg 0))
        (declare (type f2cl-lib:logical sawnan)
                 (type (double-float) bsav dminus dplus gamma p t$ tmp)
                 (type (f2cl-lib:integer4) bj j neg1 neg2 negcnt dlaneg))
```

```
(setf negcnt 0)
(setf t$ (- sigma))
(f2cl-lib:fdo (bj 1 (f2cl-lib:int-add bj blklen))
              ((> bj (f2cl-lib:int-add r (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (setf neg1 0)
    (setf bsav t$)
    (f2cl-lib:fdo (j bj (f2cl-lib:int-add j 1))
                  ((> j
                      (min
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add bj blklen
                                              (f2cl-lib:int-sub 1)))
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add r
                                              (f2cl-lib:int-sub 1)))))
                   nil)
      (tagbody
        (setf dplus
                (+ (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%) t$))
        (if (< dplus zero)
            (setf neg1 (f2cl-lib:int-add neg1 1)))
        (setf tmp (/ t$ dplus))
        (setf t$
                (-
                 (* tmp
                    (f2cl-lib:fref lld-%data% (j) ((1 *))
                                   lld-%offset%))
                 sigma))
       label21))
    (setf sawnan
            (multiple-value-bind (ret-val var-0)
                (disnan t$)
              (declare (ignore))
              (setf t$ var-0)
              ret-val))
    (cond
     (sawnan (setf neg1 0) (setf t$ bsav)
      (f2cl-lib:fdo (j bj (f2cl-lib:int-add j 1))
                    ((> j
                        (min
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add bj blklen
                                                (f2cl-lib:int-sub 1)))
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add r
                                                (f2cl-lib:int-sub
                                                 1)))))
                     nil)
        (tagbody
```

```
            (setf dplus
                    (+ (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                       t$))
            (if (< dplus zero)
                (setf neg1 (f2cl-lib:int-add neg1 1)))
            (setf tmp (/ t$ dplus))
            (if (multiple-value-bind (ret-val var-0)
                    (disnan tmp)
                  (declare (ignore))
                  (setf tmp var-0)
                  ret-val)
                (setf tmp one))
            (setf t$
                    (-
                     (* tmp
                        (f2cl-lib:fref lld-%data% (j) ((1 *))
                                       lld-%offset%))
                     sigma))
          label22))))
    (setf negcnt (f2cl-lib:int-add negcnt neg1))
   label210))
(setf p (- (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) sigma))
(f2cl-lib:fdo (bj (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add bj (f2cl-lib:int-sub blklen)))
              ((> bj r) nil)
  (tagbody
    (setf neg2 0)
    (setf bsav p)
    (f2cl-lib:fdo (j bj (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                  ((> j
                      (max
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add bj
                                              (f2cl-lib:int-sub blklen)
                                              1))
                       (the f2cl-lib:integer4 r)))
                   nil)
      (tagbody
        (setf dminus
                (+ (f2cl-lib:fref lld-%data% (j) ((1 *)) lld-%offset%)
                   p))
        (if (< dminus zero)
            (setf neg2 (f2cl-lib:int-add neg2 1)))
        (setf tmp (/ p dminus))
        (setf p
                (-
                 (* tmp
                    (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%))
                 sigma))
       label23))
```

```
            (setf sawnan
                    (multiple-value-bind (ret-val var-0)
                        (disnan p)
                      (declare (ignore))
                      (setf p var-0)
                      ret-val))
            (cond
             (sawnan (setf neg2 0) (setf p bsav)
              (f2cl-lib:fdo (j bj (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                            ((> j
                                (max
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add bj
                                                        (f2cl-lib:int-sub
                                                         blklen)
                                                        1))
                                 (the f2cl-lib:integer4 r)))
                             nil)
                (tagbody
                  (setf dminus
                          (+
                           (f2cl-lib:fref lld-%data% (j) ((1 *)) lld-%offset%)
                           p))
                  (if (< dminus zero)
                      (setf neg2 (f2cl-lib:int-add neg2 1)))
                  (setf tmp (/ p dminus))
                  (if (multiple-value-bind (ret-val var-0)
                          (disnan tmp)
                        (declare (ignore))
                        (setf tmp var-0)
                        ret-val)
                      (setf tmp one))
                  (setf p
                          (-
                           (* tmp
                              (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%))
                           sigma))
                 label24))))
            (setf negcnt (f2cl-lib:int-add negcnt neg2))
           label230))
         (setf gamma (+ t$ sigma p))
         (if (< gamma zero)
             (setf negcnt (f2cl-lib:int-add negcnt 1)))
         (setf dlaneg negcnt)
        end_label
         (return (values dlaneg nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
```

```
(setf (gethash 'fortran-to-lisp::dlaneg
               fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((fortran-to-lisp::integer4)
                                            (array double-float (*))
                                            (array double-float (*))
                                            (double-float) (double-float)
                                            (fortran-to-lisp::integer4))
                                          :return-values
                                          '(nil nil nil nil nil nil) :calls
                                          '(fortran-to-lisp::disnan))))
```

———-

90.15 dlanst

90.15.1 Axiom unit tests

— **dlanst.input** —

```
)set break resume
)sys rm -f dlanst.output
)spool dlanst.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.15.2 Axiom help page

— **dlanst.help** —

```
==========================================================================
dlanst examples
==========================================================================

==========================================================================
Man Page Details
```

```
====================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function dlanst (NORM, N, D, E)
           DLANST returns the value of the 1-norm, or the Frobenius norm, or
           the infinity norm, or the element of largest absolute value of a
           real symmetric tridiagonal matrix.

Function/Subroutine Documentation
   double precision function dlanst (character NORM, integer N, double
       precision, dimension( * ) D, double precision, dimension( * ) E)
       DLANST returns the value of the 1-norm, or the Frobenius norm, or the
       infinity norm, or the element of largest absolute value of a real
       symmetric tridiagonal matrix.

Purpose:
   DLANST  returns the value of the one norm,  or the Frobenius norm, or
   the  infinity norm,  or the  element of  largest absolute value  of a
   real symmetric tridiagonal matrix A.

 Returns:

   DLANST

        DLANST = ( max(abs(A(i,j))), NORM = 'M' or 'm'
                 (
                 ( norm1(A),         NORM = '1', 'O' or 'o'
                 (
                 ( normI(A),         NORM = 'I' or 'i'
                 (
                 ( normF(A),         NORM = 'F', 'f', 'E' or 'e'

   where  norm1  denotes the  one norm of a matrix (maximum column sum),
   normI  denotes the  infinity norm  of a matrix  (maximum row sum) and
   normF  denotes the  Frobenius norm of a matrix (square root of sum of
   squares).  Note that  max(abs(A(i,j)))  is not a consistent matrix norm.

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies the value to be returned in DLANST as described above.

 N
   N is INTEGER
   The order of the matrix A.  N >= 0.  When N = 0, DLANST is set to zero.

 D
   D is DOUBLE PRECISION array, dimension (N)
```

```
  The diagonal elements of A.

 E
  E is DOUBLE PRECISION array, dimension (N-1)
  The (n-1) sub-diagonal or super-diagonal elements of A.
```

90.15.3 fortran code

— dlanst.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION DLANST( NORM, N, D, E )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      DOUBLE PRECISION   ANORM, SCALE, SUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, DISNAN
      EXTERNAL           LSAME, DISNAN
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASSQ
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SQRT
```

```
*     ..
*     .. Executable Statements ..
*
      IF( N.LE.0 ) THEN
         ANORM = ZERO
      ELSE IF( LSAME( NORM, 'M' ) ) THEN
*
*        Find max(abs(A(i,j))).
*
         ANORM = ABS( D( N ) )
         DO 10 I = 1, N - 1
            SUM = ABS( D( I ) )
            IF( ANORM .LT. SUM .OR. DISNAN( SUM ) ) ANORM = SUM
            SUM = ABS( E( I ) )
            IF( ANORM .LT. SUM .OR. DISNAN( SUM ) ) ANORM = SUM
   10    CONTINUE
      ELSE IF( LSAME( NORM, 'O' ) .OR. NORM.EQ.'1' .OR.
     $         LSAME( NORM, 'I' ) ) THEN
*
*        Find norm1(A).
*
         IF( N.EQ.1 ) THEN
            ANORM = ABS( D( 1 ) )
         ELSE
            ANORM = ABS( D( 1 ) )+ABS( E( 1 ) )
            SUM = ABS( E( N-1 ) )+ABS( D( N ) )
            IF( ANORM .LT. SUM .OR. DISNAN( SUM ) ) ANORM = SUM
            DO 20 I = 2, N - 1
               SUM = ABS( D( I ) )+ABS( E( I ) )+ABS( E( I-1 ) )
               IF( ANORM .LT. SUM .OR. DISNAN( SUM ) ) ANORM = SUM
   20       CONTINUE
         END IF
      ELSE IF( ( LSAME( NORM, 'F' ) ) .OR. ( LSAME( NORM, 'E' ) ) ) THEN
*
*        Find normF(A).
*
         SCALE = ZERO
         SUM = ONE
         IF( N.GT.1 ) THEN
            CALL DLASSQ( N-1, E, 1, SCALE, SUM )
            SUM = 2*SUM
         END IF
         CALL DLASSQ( N, D, 1, SCALE, SUM )
         ANORM = SCALE*SQRT( SUM )
      END IF
*
      DLANST = ANORM
      RETURN
*
*     End of DLANST
```

```
*
     END
```

———-

90.15.4 lisp code

— **dlanst.lisp** —

———-

90.16 dlapy2

90.16.1 Axiom unit tests

— **dlapy2.input** —

```
)set break resume
)sys rm -f dlapy2.output
)spool dlapy2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.16.2 Axiom help page

— **dlapy2.help** —

```
==========================================================================
dlapy2 examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       double precision function dlapy2 (X, Y)
           DLAPY2 returns sqrt(x2+y2).

Function/Subroutine Documentation
   double precision function dlapy2 (double precision X, double precision Y)
       DLAPY2 returns sqrt(x2+y2).

Purpose:
   DLAPY2 returns sqrt(x**2+y**2), taking care not to cause unnecessary
   overflow.

Parameters:

 X
   X is DOUBLE PRECISION

 Y
   Y is DOUBLE PRECISION
   X and Y specify the values x and y.
```

90.16.3 fortran code

— dlapy2.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION DLAPY2( X, Y )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   X, Y
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
      DOUBLE PRECISION   ONE
```

```
      PARAMETER          ( ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION   W, XABS, YABS, Z
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      XABS = ABS( X )
      YABS = ABS( Y )
      W = MAX( XABS, YABS )
      Z = MIN( XABS, YABS )
      IF( Z.EQ.ZERO ) THEN
         DLAPY2 = W
      ELSE
         DLAPY2 = W*SQRT( ONE+( Z / W )**2 )
      END IF
      RETURN
*
*     End of DLAPY2
*
      END
```

———-

90.16.4 lisp code

— dlapy2.lisp —

———-

90.17 dlapy3

90.17.1 Axiom unit tests

— dlapy3.input —

```
)set break resume
)sys rm -f dlapy3.output
)spool dlapy3.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.17.2 Axiom help page

— dlapy3.help —

```
=====================================================================
dlapy3 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       double precision function dlapy3 (X, Y, Z)
           DLAPY3 returns sqrt(x2+y2+z2).

Function/Subroutine Documentation
   double precision function dlapy3 (double precision X, double precision Y,
       double precision Z)
       DLAPY3 returns sqrt(x2+y2+z2).

Purpose:
   DLAPY3 returns sqrt(x**2+y**2+z**2), taking care not to cause
   unnecessary overflow.

Parameters:

 X
   X is DOUBLE PRECISION

 Y
   Y is DOUBLE PRECISION

 Z
   Z is DOUBLE PRECISION
   X, Y and Z specify the values x, y and z.
```

———-

90.17.3 fortran code

— dlapy3.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION DLAPY3( X, Y, Z )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   X, Y, Z
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION   W, XABS, YABS, ZABS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
      XABS = ABS( X )
      YABS = ABS( Y )
      ZABS = ABS( Z )
      W = MAX( XABS, YABS, ZABS )
      IF( W.EQ.ZERO ) THEN
*     W can be zero for max(0,nan,0)
*     adding all three entries together will make sure
*     NaN will not disappear.
         DLAPY3 =  XABS + YABS + ZABS
      ELSE
         DLAPY3 = W*SQRT( ( XABS / W )**2+( YABS / W )**2+
     $            ( ZABS / W )**2 )
      END IF
      RETURN
*
*     End of DLAPY3
```

```
*
      END
```

———-

90.17.4 lisp code

— **dlapy3.lisp** —

———-

90.18 dlarnv

90.18.1 Axiom unit tests

— **dlarnv.input** —

```
)set break resume
)sys rm -f dlarnv.output
)spool dlarnv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.18.2 Axiom help page

— **dlarnv.help** —

```
======================================================================
dlarnv examples
======================================================================

======================================================================
Man Page Details
======================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine dlarnv (IDIST, ISEED, N, X)
           DLARNV returns a vector of random numbers from a uniform or normal
           distribution.

Function/Subroutine Documentation
   subroutine dlarnv (integer IDIST, integer, dimension( 4 ) ISEED, integer N,
       double precision, dimension( * ) X)
       DLARNV returns a vector of random numbers from a uniform or normal
       distribution.

Purpose:
   DLARNV returns a vector of n random real numbers from a uniform or
   normal distribution.

Parameters:

 IDIST
   IDIST is INTEGER
   Specifies the distribution of the random numbers:
   = 1:  uniform (0,1)
   = 2:  uniform (-1,1)
   = 3:  normal (0,1)

 ISEED
   ISEED is INTEGER array, dimension (4)
   On entry, the seed of the random number generator; the array
   elements must be between 0 and 4095, and ISEED(4) must be odd.
   On exit, the seed is updated.

 N
   N is INTEGER
   The number of random numbers to be generated.

 X
   X is DOUBLE PRECISION array, dimension (N)
   The generated random numbers.

Further Details:

   This routine calls the auxiliary routine DLARUV to generate random
   real numbers from a uniform (0,1) distribution, in batches of up to
   128 using vectorisable code. The Box-Muller method is used to
   transform numbers from a uniform to a normal distribution.
```

90.18.3 fortran code

— dlarnv.f —

```
*  =====================================================================
      SUBROUTINE DLARNV( IDIST, ISEED, N, X )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            IDIST, N
*     ..
*     .. Array Arguments ..
      INTEGER            ISEED( 4 )
      DOUBLE PRECISION   X( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, TWO
      PARAMETER          ( ONE = 1.0D+0, TWO = 2.0D+0 )
      INTEGER            LV
      PARAMETER          ( LV = 128 )
      DOUBLE PRECISION   TWOPI
      PARAMETER          ( TWOPI = 6.2831853071795864769252867663D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IL, IL2, IV
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   U( LV )
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          COS, LOG, MIN, SQRT
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARUV
*     ..
*     .. Executable Statements ..
*
      DO 40 IV = 1, N, LV / 2
         IL = MIN( LV / 2, N-IV+1 )
         IF( IDIST.EQ.3 ) THEN
            IL2 = 2*IL
         ELSE
```

```
            IL2 = IL
         END IF
*
*        Call DLARUV to generate IL2 numbers from a uniform (0,1)
*        distribution (IL2 <= LV)
*
         CALL DLARUV( ISEED, IL2, U )
*
         IF( IDIST.EQ.1 ) THEN
*
*           Copy generated numbers
*
            DO 10 I = 1, IL
               X( IV+I-1 ) = U( I )
   10       CONTINUE
         ELSE IF( IDIST.EQ.2 ) THEN
*
*           Convert generated numbers to uniform (-1,1) distribution
*
            DO 20 I = 1, IL
               X( IV+I-1 ) = TWO*U( I ) - ONE
   20       CONTINUE
         ELSE IF( IDIST.EQ.3 ) THEN
*
*           Convert generated numbers to normal (0,1) distribution
*
            DO 30 I = 1, IL
               X( IV+I-1 ) = SQRT( -TWO*LOG( U( 2*I-1 ) ) )*
     $                       COS( TWOPI*U( 2*I ) )
   30       CONTINUE
         END IF
   40 CONTINUE
      RETURN
*
*     End of DLARNV
*
      END
```

———-

90.18.4 lisp code

— dlarnv.lisp —

```
(let* ((one 1.0d0) (two 2.0d0) (lv 128) (twopi 6.283185307179586d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (f2cl-lib:integer4 128 128) lv)
```

```
          (type (double-float 6.283185307179586d0 6.283185307179586d0) twopi)
          (ignorable one two lv twopi))
 (defun dlarnv (idist iseed n x)
   (declare (type (array double-float (*)) x)
            (type (array f2cl-lib:integer4 (*)) iseed)
            (type (f2cl-lib:integer4) n idist))
   (f2cl-lib:with-multi-array-data
       ((iseed f2cl-lib:integer4 iseed-%data% iseed-%offset%)
        (x double-float x-%data% x-%offset%))
     (prog ((u (make-array lv :element-type 'double-float)) (i 0) (il 0)
            (il2 0) (iv 0))
       (declare (type (array double-float (*)) u)
                (type (f2cl-lib:integer4) i il il2 iv))
       (f2cl-lib:fdo (iv 1 (+ iv (the f2cl-lib:integer4 (truncate lv 2))))
                     ((> iv n) nil)
         (tagbody
           (setf il
                   (min (the f2cl-lib:integer4 (truncate lv 2))
                        (f2cl-lib:int-add (f2cl-lib:int-sub n iv) 1)))
           (cond ((= idist 3) (setf il2 (f2cl-lib:int-mul 2 il)))
                 (t (setf il2 il)))
           (multiple-value-bind (var-0 var-1 var-2)
               (dlaruv iseed il2 u)
             (declare (ignore var-0 var-2))
             (when var-1 (setf il2 var-1)))
           (cond
            ((= idist 1)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i il) nil)
               (tagbody
                 (setf (f2cl-lib:fref x-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add iv i) 1))
                                      ((1 *)) x-%offset%)
                       (f2cl-lib:fref u (i) ((1 lv))))
                label10)))
            ((= idist 2)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i il) nil)
               (tagbody
                 (setf (f2cl-lib:fref x-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add iv i) 1))
                                      ((1 *)) x-%offset%)
                       (- (* two (f2cl-lib:fref u (i) ((1 lv)))) one))
                label20)))
            ((= idist 3)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i il) nil)
               (tagbody
```

```
                 (setf (f2cl-lib:fref x-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add iv i) 1))
                                      ((1 *)) x-%offset%)
                         (*
                          (f2cl-lib:fsqrt
                           (* (- two)
                              (f2cl-lib:flog
                               (f2cl-lib:fref u
                                              ((f2cl-lib:int-sub
                                                (f2cl-lib:int-mul 2 i) 1))
                                              ((1 lv))))))
                          (cos
                           (* twopi
                              (f2cl-lib:fref u ((f2cl-lib:int-mul 2 i))
                                             ((1 lv)))))))
                 label30))))
           label40))
        (go end_label)
       end_label
        (return (values nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlarnv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*)))
                                            :return-values '(nil nil nil nil)
                                            :calls 'nil)))
```

———-

90.19 dlarra

90.19.1 Axiom unit tests

— dlarra.input —

```
)set break resume
```

```
)sys rm -f dlarra.output
)spool dlarra.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

90.19.2 Axiom help page

— dlarra.help —

```
=====================================================================
dlarra examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlarra (N, D, E, E2, SPLTOL, TNRM, NSPLIT, ISPLIT, INFO)
           DLARRA computes the splitting points with the specified threshold.

Function/Subroutine Documentation
   subroutine dlarra (integer N, double precision, dimension( * ) D, double
       precision, dimension( * ) E, double precision, dimension( * ) E2,
       double precision SPLTOL, double precision TNRM, integer NSPLIT,
       integer, dimension( * ) ISPLIT, integer INFO)
       DLARRA computes the splitting points with the specified threshold.

Purpose:
   Compute the splitting points with threshold SPLTOL.
   DLARRA sets any "small" off-diagonal elements to zero.

Parameters:

 N
   N is INTEGER
   The order of the matrix. N > 0.

 D
   D is DOUBLE PRECISION array, dimension (N)
   On entry, the N diagonal elements of the tridiagonal matrix T.
```

```
E
  E is DOUBLE PRECISION array, dimension (N)
  On entry, the first (N-1) entries contain the subdiagonal
  elements of the tridiagonal matrix T; E(N) need not be set.
  On exit, the entries E( ISPLIT( I ) ), 1 <= I <= NSPLIT,
  are set to zero, the other entries of E are untouched.

E2
  E2 is DOUBLE PRECISION array, dimension (N)
  On entry, the first (N-1) entries contain the SQUARES of the
  subdiagonal elements of the tridiagonal matrix T;
  E2(N) need not be set.
  On exit, the entries E2( ISPLIT( I ) ),
  1 <= I <= NSPLIT, have been set to zero

SPLTOL
  SPLTOL is DOUBLE PRECISION
  The threshold for splitting. Two criteria can be used:
  SPLTOL<0 : criterion based on absolute off-diagonal value
  SPLTOL>0 : criterion that preserves relative accuracy

TNRM
  TNRM is DOUBLE PRECISION
  The norm of the matrix.

NSPLIT
  NSPLIT is INTEGER
  The number of blocks T splits into. 1 <= NSPLIT <= N.

ISPLIT
  ISPLIT is INTEGER array, dimension (N)
  The splitting points, at which T breaks up into blocks.
  The first block consists of rows/columns 1 to ISPLIT(1),
  the second of rows/columns ISPLIT(1)+1 through ISPLIT(2),
  etc., and the NSPLIT-th consists of rows/columns
  ISPLIT(NSPLIT-1)+1 through ISPLIT(NSPLIT)=N.

INFO
  INFO is INTEGER
  = 0:  successful exit
```

———-

90.19.3 fortran code

— dlarra.f —

```
*  =====================================================================
      SUBROUTINE DLARRA( N, D, E, E2, SPLTOL, TNRM,
     $                   NSPLIT, ISPLIT, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, N, NSPLIT
      DOUBLE PRECISION    SPLTOL, TNRM
*     ..
*     .. Array Arguments ..
      INTEGER            ISPLIT( * )
      DOUBLE PRECISION   D( * ), E( * ), E2( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      DOUBLE PRECISION   EABS, TMP1

*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS
*     ..
*     .. Executable Statements ..
*
      INFO = 0

*     Compute splitting points
      NSPLIT = 1
      IF(SPLTOL.LT.ZERO) THEN
*        Criterion based on absolute off-diagonal value
         TMP1 = ABS(SPLTOL)* TNRM
         DO 9 I = 1, N-1
            EABS = ABS( E(I) )
            IF( EABS .LE. TMP1) THEN
               E(I) = ZERO
               E2(I) = ZERO
               ISPLIT( NSPLIT ) = I
               NSPLIT = NSPLIT + 1
            END IF
 9       CONTINUE
```

```
      ELSE
*        Criterion that guarantees relative accuracy
         DO 10 I = 1, N-1
            EABS = ABS( E(I) )
            IF( EABS .LE. SPLTOL * SQRT(ABS(D(I)))*SQRT(ABS(D(I+1))) )
     $      THEN
               E(I) = ZERO
               E2(I) = ZERO
               ISPLIT( NSPLIT ) = I
               NSPLIT = NSPLIT + 1
            END IF
 10      CONTINUE
      ENDIF
      ISPLIT( NSPLIT ) = N

      RETURN
*
*     End of DLARRA
*
      END
```

———-

90.19.4 lisp code

— dlarra.lisp —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun dlarra (n d e e2 spltol tnrm nsplit isplit info)
    (declare (type (array f2cl-lib:integer4 (*)) isplit)
             (type (double-float) tnrm spltol)
             (type (array double-float (*)) e2 e d)
             (type (f2cl-lib:integer4) info nsplit n))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%)
         (e2 double-float e2-%data% e2-%offset%)
         (isplit f2cl-lib:integer4 isplit-%data% isplit-%offset%))
      (prog ((eabs 0.0d0) (tmp1 0.0d0) (i 0))
        (declare (type (double-float) eabs tmp1)
                 (type (f2cl-lib:integer4) i))
        (setf info 0)
        (setf nsplit 1)
        (cond
         ((< spltol zero) (setf tmp1 (* (abs spltol) tnrm))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                        ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
            (tagbody
              (setf eabs (abs (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
              (cond
               ((<= eabs tmp1)
                (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) zero)
                (setf (f2cl-lib:fref e2-%data% (i) ((1 *)) e2-%offset%) zero)
                (setf (f2cl-lib:fref isplit-%data% (nsplit) ((1 *))
                                     isplit-%offset%)
                        i)
                (setf nsplit (f2cl-lib:int-add nsplit 1))))
             label9)))
          (t
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
            (tagbody
              (setf eabs (abs (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
              (cond
               ((<= eabs
                    (* spltol
                       (f2cl-lib:fsqrt (abs (f2cl-lib:fref d (i) ((1 *)))))
                       (f2cl-lib:fsqrt
                        (abs
                         (f2cl-lib:fref d ((f2cl-lib:int-add i 1)) ((1 *)))))))
                (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) zero)
                (setf (f2cl-lib:fref e2-%data% (i) ((1 *)) e2-%offset%) zero)
                (setf (f2cl-lib:fref isplit-%data% (nsplit) ((1 *))
                                     isplit-%offset%)
                        i)
                (setf nsplit (f2cl-lib:int-add nsplit 1))))
             label10))))
         (setf (f2cl-lib:fref isplit-%data% (nsplit) ((1 *)) isplit-%offset%) n)
         (go end_label)
        end_label
         (return (values nil nil nil nil nil nil nsplit nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlarra
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (double-float) (double-float)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
```

```
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil nil nil nil
  fortran-to-lisp::nsplit nil
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

90.20 dlarrb

90.20.1 Axiom unit tests

— **dlarrb.input** —

```
)set break resume
)sys rm -f dlarrb.output
)spool dlarrb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.20.2 Axiom help page

— **dlarrb.help** —

```
=====================================================================
dlarrb examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlarrb (N, D, LLD, IFIRST, ILAST, RTOL1, RTOL2, OFFSET, W,
```

```
          WGAP, WERR, WORK, IWORK, PIVMIN, SPDIAM, TWIST, INFO)
          DLARRB provides limited bisection to locate eigenvalues for more
          accuracy.

Function/Subroutine Documentation
   subroutine dlarrb (integer N, double precision, dimension( * ) D, double
       precision, dimension( * ) LLD, integer IFIRST, integer ILAST, double
       precision RTOL1, double precision RTOL2, integer OFFSET, double
       precision, dimension( * ) W, double precision, dimension( * ) WGAP,
       double precision, dimension( * ) WERR, double precision, dimension( * )
       WORK, integer, dimension( * ) IWORK, double precision PIVMIN, double
       precision SPDIAM, integer TWIST, integer INFO)
       DLARRB provides limited bisection to locate eigenvalues for more
       accuracy.

Purpose:
   Given the relatively robust representation(RRR) L D L^T, DLARRB
   does "limited" bisection to refine the eigenvalues of L D L^T,
   W( IFIRST-OFFSET ) through W( ILAST-OFFSET ), to more accuracy. Initial
   guesses for these eigenvalues are input in W, the corresponding estimate
   of the error in these guesses and their gaps are input in WERR
   and WGAP, respectively. During bisection, intervals
   [left, right] are maintained by storing their mid-points and
   semi-widths in the arrays W and WERR respectively.

Parameters:

 N
   N is INTEGER
   The order of the matrix.

 D
   D is DOUBLE PRECISION array, dimension (N)
   The N diagonal elements of the diagonal matrix D.

 LLD
   LLD is DOUBLE PRECISION array, dimension (N-1)
   The (N-1) elements L(i)*L(i)*D(i).

 IFIRST
   IFIRST is INTEGER
   The index of the first eigenvalue to be computed.

 ILAST
   ILAST is INTEGER
   The index of the last eigenvalue to be computed.

 RTOL1
   RTOL1 is DOUBLE PRECISION
```

```
RTOL2
  RTOL2 is DOUBLE PRECISION
  Tolerance for the convergence of the bisection intervals.
  An interval [LEFT,RIGHT] has converged if
  RIGHT-LEFT.LT.MAX( RTOL1*GAP, RTOL2*MAX(|LEFT|,|RIGHT|) )
  where GAP is the (estimated) distance to the nearest eigenvalue.

OFFSET
  OFFSET is INTEGER
  Offset for the arrays W, WGAP and WERR, i.e., the IFIRST-OFFSET
  through ILAST-OFFSET elements of these arrays are to be used.

W
  W is DOUBLE PRECISION array, dimension (N)
  On input, W( IFIRST-OFFSET ) through W( ILAST-OFFSET ) are
  estimates of the eigenvalues of L D L^T indexed IFIRST through ILAST.
  On output, these estimates are refined.

WGAP
  WGAP is DOUBLE PRECISION array, dimension (N-1)
  On input, the (estimated) gaps between consecutive
  eigenvalues of L D L^T, i.e., WGAP(I-OFFSET) is the gap between
  eigenvalues I and I+1. Note that if IFIRST.EQ.ILAST
  then WGAP(IFIRST-OFFSET) must be set to ZERO.
  On output, these gaps are refined.

WERR
  WERR is DOUBLE PRECISION array, dimension (N)
  On input, WERR( IFIRST-OFFSET ) through WERR( ILAST-OFFSET ) are
  the errors in the estimates of the corresponding elements in W.
  On output, these errors are refined.

WORK
  WORK is DOUBLE PRECISION array, dimension (2*N)
  Workspace.

IWORK
  IWORK is INTEGER array, dimension (2*N)
  Workspace.

PIVMIN
  PIVMIN is DOUBLE PRECISION
  The minimum pivot in the Sturm sequence.

SPDIAM
  SPDIAM is DOUBLE PRECISION
  The spectral diameter of the matrix.

TWIST
  TWIST is INTEGER
```

```
  The twist index for the twisted factorization that is used
  for the negcount.
  TWIST = N: Compute negcount from L D L^T - LAMBDA I = L+ D+ L+^T
  TWIST = 1: Compute negcount from L D L^T - LAMBDA I = U- D- U-^T
  TWIST = R: Compute negcount from L D L^T - LAMBDA I = N(r) D(r) N(r)

INFO
  INFO is INTEGER
  Error flag.
```

———-

90.20.3 fortran code

— dlarrb.f —

```
*  =====================================================================
      SUBROUTINE DLARRB( N, D, LLD, IFIRST, ILAST, RTOL1,
     $                   RTOL2, OFFSET, W, WGAP, WERR, WORK, IWORK,
     $                   PIVMIN, SPDIAM, TWIST, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            IFIRST, ILAST, INFO, N, OFFSET, TWIST
      DOUBLE PRECISION   PIVMIN, RTOL1, RTOL2, SPDIAM
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   D( * ), LLD( * ), W( * ),
     $                   WERR( * ), WGAP( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, TWO, HALF
      PARAMETER        ( ZERO = 0.0D0, TWO = 2.0D0,
     $                   HALF = 0.5D0 )
      INTEGER   MAXITR
*     ..
*     .. Local Scalars ..
      INTEGER            I, I1, II, IP, ITER, K, NEGCNT, NEXT, NINT,
     $                   OLNINT, PREV, R
```

```
      DOUBLE PRECISION   BACK, CVRGD, GAP, LEFT, LGAP, MID, MNWDTH,
     $                   RGAP, RIGHT, TMP, WIDTH
*     ..
*     .. External Functions ..
      INTEGER            DLANEG
      EXTERNAL           DLANEG
*
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
      MAXITR = INT( ( LOG( SPDIAM+PIVMIN )-LOG( PIVMIN ) ) /
     $           LOG( TWO ) ) + 2
      MNWDTH = TWO * PIVMIN
*
      R = TWIST
      IF((R.LT.1).OR.(R.GT.N)) R = N
*
*     Initialize unconverged intervals in [ WORK(2*I-1), WORK(2*I) ].
*     The Sturm Count, Count( WORK(2*I-1) ) is arranged to be I-1, while
*     Count( WORK(2*I) ) is stored in IWORK( 2*I ). The integer IWORK( 2*I-1 )
*     for an unconverged interval is set to the index of the next unconverged
*     interval, and is -1 or 0 for a converged interval. Thus a linked
*     list of unconverged intervals is set up.
*
      I1 = IFIRST
*     The number of unconverged intervals
      NINT = 0
*     The last unconverged interval found
      PREV = 0

      RGAP = WGAP( I1-OFFSET )
      DO 75 I = I1, ILAST
         K = 2*I
         II = I - OFFSET
         LEFT = W( II ) - WERR( II )
         RIGHT = W( II ) + WERR( II )
         LGAP = RGAP
         RGAP = WGAP( II )
         GAP = MIN( LGAP, RGAP )

*        Make sure that [LEFT,RIGHT] contains the desired eigenvalue
*        Compute negcount from dstqds facto L+D+L+^T = L D L^T - LEFT
*
*        Do while( NEGCNT(LEFT).GT.I-1 )
*
```

```
         BACK = WERR( II )
 20      CONTINUE
         NEGCNT = DLANEG( N, D, LLD, LEFT, PIVMIN, R )
         IF( NEGCNT.GT.I-1 ) THEN
            LEFT = LEFT - BACK
            BACK = TWO*BACK
            GO TO 20
         END IF
*
*        Do while( NEGCNT(RIGHT).LT.I )
*        Compute negcount from dstqds facto L+D+L+^T = L D L^T - RIGHT
*
         BACK = WERR( II )
 50      CONTINUE

         NEGCNT = DLANEG( N, D, LLD, RIGHT, PIVMIN, R )
          IF( NEGCNT.LT.I ) THEN
             RIGHT = RIGHT + BACK
             BACK = TWO*BACK
             GO TO 50
          END IF
         WIDTH = HALF*ABS( LEFT - RIGHT )
         TMP = MAX( ABS( LEFT ), ABS( RIGHT ) )
         CVRGD = MAX(RTOL1*GAP,RTOL2*TMP)
         IF( WIDTH.LE.CVRGD .OR. WIDTH.LE.MNWDTH ) THEN
*           This interval has already converged and does not need refinement.
*           (Note that the gaps might change through refining the
*            eigenvalues, however, they can only get bigger.)
*           Remove it from the list.
            IWORK( K-1 ) = -1
*           Make sure that I1 always points to the first unconverged interval
            IF((I.EQ.I1).AND.(I.LT.ILAST)) I1 = I + 1
            IF((PREV.GE.I1).AND.(I.LE.ILAST)) IWORK( 2*PREV-1 ) = I + 1
         ELSE
*           unconverged interval found
            PREV = I
            NINT = NINT + 1
            IWORK( K-1 ) = I + 1
            IWORK( K ) = NEGCNT
         END IF
         WORK( K-1 ) = LEFT
         WORK( K ) = RIGHT
 75   CONTINUE

*
*     Do while( NINT.GT.0 ), i.e. there are still unconverged intervals
*     and while (ITER.LT.MAXITR)
*
      ITER = 0
 80   CONTINUE
```

```
      PREV = I1 - 1
      I = I1
      OLNINT = NINT

      DO 100 IP = 1, OLNINT
         K = 2*I
         II = I - OFFSET
         RGAP = WGAP( II )
         LGAP = RGAP
         IF(II.GT.1) LGAP = WGAP( II-1 )
         GAP = MIN( LGAP, RGAP )
         NEXT = IWORK( K-1 )
         LEFT = WORK( K-1 )
         RIGHT = WORK( K )
         MID = HALF*( LEFT + RIGHT )

*        semiwidth of interval
         WIDTH = RIGHT - MID
         TMP = MAX( ABS( LEFT ), ABS( RIGHT ) )
         CVRGD = MAX(RTOL1*GAP,RTOL2*TMP)
         IF( ( WIDTH.LE.CVRGD ) .OR. ( WIDTH.LE.MNWDTH ).OR.
     $       ( ITER.EQ.MAXITR ) )THEN
*           reduce number of unconverged intervals
            NINT = NINT - 1
*           Mark interval as converged.
            IWORK( K-1 ) = 0
            IF( I1.EQ.I ) THEN
               I1 = NEXT
            ELSE
*              Prev holds the last unconverged interval previously examined
               IF(PREV.GE.I1) IWORK( 2*PREV-1 ) = NEXT
            END IF
            I = NEXT
            GO TO 100
         END IF
         PREV = I
*
*        Perform one bisection step
*
         NEGCNT = DLANEG( N, D, LLD, MID, PIVMIN, R )
         IF( NEGCNT.LE.I-1 ) THEN
            WORK( K-1 ) = MID
         ELSE
            WORK( K ) = MID
         END IF
         I = NEXT
 100  CONTINUE
      ITER = ITER + 1
*     do another loop if there are still unconverged intervals
*     However, in the last iteration, all intervals are accepted
```

```
*     since this is the best we can do.
      IF( ( NINT.GT.0 ).AND.(ITER.LE.MAXITR) ) GO TO 80
*
*
*     At this point, all the intervals have converged
      DO 110 I = IFIRST, ILAST
         K = 2*I
         II = I - OFFSET
*        All intervals marked by '0' have been refined.
         IF( IWORK( K-1 ).EQ.0 ) THEN
            W( II ) = HALF*( WORK( K-1 )+WORK( K ) )
            WERR( II ) = WORK( K ) - W( II )
         END IF
 110  CONTINUE
*
      DO 111 I = IFIRST+1, ILAST
         K = 2*I
         II = I - OFFSET
         WGAP( II-1 ) = MAX( ZERO,
     $                     W(II) - WERR (II) - W( II-1 ) - WERR( II-1 ))
 111  CONTINUE

      RETURN
*
*     End of DLARRB
*
      END
```

———-

90.20.4 lisp code

— dlarrb.lisp —

```
(let* ((zero 0.0d0) (two 2.0d0) (half 0.5d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 0.5d0 0.5d0) half)
           (ignorable zero two half))
  (defun dlarrb
         (n d lld ifirst ilast rtol1 rtol2 offset w wgap werr work iwork pivmin
          spdiam twist info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (double-float) spdiam pivmin rtol2 rtol1)
             (type (array double-float (*)) work werr wgap w lld d)
             (type (f2cl-lib:integer4) info twist offset ilast ifirst n))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
```

```
    (lld double-float lld-%data% lld-%offset%)
    (w double-float w-%data% w-%offset%)
    (wgap double-float wgap-%data% wgap-%offset%)
    (werr double-float werr-%data% werr-%offset%)
    (work double-float work-%data% work-%offset%)
    (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((back 0.0d0) (cvrgd 0.0d0) (gap 0.0d0) (left 0.0d0) (lgap 0.0d0)
         (mid 0.0d0) (mnwdth 0.0d0) (rgap 0.0d0) (right 0.0d0) (tmp 0.0d0)
         (width 0.0d0) (i 0) (i1 0) (ii 0) (ip 0) (iter 0) (k 0) (negcnt 0)
         (next 0) (f2cl-lib:nint 0) (olnint 0) (prev 0) (r 0) (maxitr 0))
    (declare
     (type (double-float) back cvrgd gap left lgap mid mnwdth rgap right
      tmp width)
     (type (f2cl-lib:integer4) i i1 ii ip iter k negcnt next f2cl-lib:nint
      olnint prev r maxitr))
    (setf info 0)
    (setf maxitr
            (f2cl-lib:int-add
             (f2cl-lib:int
              (/
               (- (f2cl-lib:flog (+ spdiam pivmin)) (f2cl-lib:flog pivmin))
               (f2cl-lib:flog two)))
             2))
    (setf mnwdth (* two pivmin))
    (setf r twist)
    (if (or (< r 1) (> r n))
        (setf r n))
    (setf i1 ifirst)
    (setf f2cl-lib:nint 0)
    (setf prev 0)
    (setf rgap
            (f2cl-lib:fref wgap-%data% ((f2cl-lib:int-sub i1 offset))
                           ((1 *)) wgap-%offset%))
    (f2cl-lib:fdo (i i1 (f2cl-lib:int-add i 1))
                  ((> i ilast) nil)
      (tagbody
        (setf k (f2cl-lib:int-mul 2 i))
        (setf ii (f2cl-lib:int-sub i offset))
        (setf left
                (- (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%)
                   (f2cl-lib:fref werr-%data% (ii) ((1 *)) werr-%offset%)))
        (setf right
                (+ (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%)
                   (f2cl-lib:fref werr-%data% (ii) ((1 *)) werr-%offset%)))
        (setf lgap rgap)
        (setf rgap (f2cl-lib:fref wgap-%data% (ii) ((1 *)) wgap-%offset%))
        (setf gap (min lgap rgap))
        (setf back (f2cl-lib:fref werr-%data% (ii) ((1 *)) werr-%offset%))
       label20
        (setf negcnt (dlaneg n d lld left pivmin r))
```

```
      (cond
       ((> negcnt (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
        (setf left (- left back)) (setf back (* two back)) (go label20)))
      (setf back (f2cl-lib:fref werr-%data% (ii) ((1 *)) werr-%offset%))
     label50
      (setf negcnt (dlaneg n d lld right pivmin r))
      (cond
       ((< negcnt i) (setf right (+ right back)) (setf back (* two back))
        (go label50)))
      (setf width (* half (abs (- left right))))
      (setf tmp (max (abs left) (abs right)))
      (setf cvrgd (max (* rtol1 gap) (* rtol2 tmp)))
      (cond
       ((or (<= width cvrgd) (<= width mnwdth))
        (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-sub k 1))
                             ((1 *)) iwork-%offset%)
              -1)
        (if (and (= i i1) (< i ilast))
            (setf i1 (f2cl-lib:int-add i 1)))
        (if (and (>= prev i1) (<= i ilast))
            (setf (f2cl-lib:fref iwork-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-mul 2 prev) 1))
                                 ((1 *)) iwork-%offset%)
                  (f2cl-lib:int-add i 1))))
       (t (setf prev i)
        (setf f2cl-lib:nint (f2cl-lib:int-add f2cl-lib:nint 1))
        (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-sub k 1))
                             ((1 *)) iwork-%offset%)
              (f2cl-lib:int-add i 1))
        (setf (f2cl-lib:fref iwork-%data% (k) ((1 *)) iwork-%offset%)
              negcnt)))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                           work-%offset%)
            left)
      (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%) right)
     label75))
  (setf iter 0)
 label80
  (setf prev (f2cl-lib:int-sub i1 1))
  (setf i i1)
  (setf olnint f2cl-lib:nint)
  (f2cl-lib:fdo (ip 1 (f2cl-lib:int-add ip 1))
                ((> ip olnint) nil)
    (tagbody
      (setf k (f2cl-lib:int-mul 2 i))
      (setf ii (f2cl-lib:int-sub i offset))
      (setf rgap (f2cl-lib:fref wgap-%data% (ii) ((1 *)) wgap-%offset%))
      (setf lgap rgap)
      (if (> ii 1)
```

```
        (setf lgap
                (f2cl-lib:fref wgap-%data% ((f2cl-lib:int-sub ii 1))
                               ((1 *)) wgap-%offset%)))
    (setf gap (min lgap rgap))
    (setf next
            (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-sub k 1))
                           ((1 *)) iwork-%offset%))
    (setf left
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                           work-%offset%))
    (setf right (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%))
    (setf mid (* half (+ left right)))
    (setf width (- right mid))
    (setf tmp (max (abs left) (abs right)))
    (setf cvrgd (max (* rtol1 gap) (* rtol2 tmp)))
    (cond
     ((or (<= width cvrgd) (<= width mnwdth) (= iter maxitr))
      (setf f2cl-lib:nint (f2cl-lib:int-sub f2cl-lib:nint 1))
      (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-sub k 1))
                           ((1 *)) iwork-%offset%)
              0)
      (cond ((= i1 i) (setf i1 next))
            (t
             (if (>= prev i1)
                 (setf (f2cl-lib:fref iwork-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-mul 2 prev) 1))
                                      ((1 *)) iwork-%offset%)
                       next))))
      (setf i next) (go label100)))
    (setf prev i)
    (setf negcnt (dlaneg n d lld mid pivmin r))
    (cond
     ((<= negcnt (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                           work-%offset%)
              mid))
     (t
      (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
              mid)))
    (setf i next)
   label100))
(setf iter (f2cl-lib:int-add iter 1))
(if (and (> f2cl-lib:nint 0) (<= iter maxitr))
    (go label80))
(f2cl-lib:fdo (i ifirst (f2cl-lib:int-add i 1))
              ((> i ilast) nil)
  (tagbody
    (setf k (f2cl-lib:int-mul 2 i))
    (setf ii (f2cl-lib:int-sub i offset))
```

```
            (cond
             ((=
               (f2cl-lib:fref iwork ((f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                              ((1 *)))
               0)
              (setf (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%)
                      (* half
                         (+
                          (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1))
                                         ((1 *)) work-%offset%)
                          (f2cl-lib:fref work-%data% (k) ((1 *))
                                         work-%offset%))))
              (setf (f2cl-lib:fref werr-%data% (ii) ((1 *)) werr-%offset%)
                      (- (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                         (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%)))))
           label110))
         (f2cl-lib:fdo (i (f2cl-lib:int-add ifirst 1) (f2cl-lib:int-add i 1))
                       ((> i ilast) nil)
           (tagbody
             (setf k (f2cl-lib:int-mul 2 i))
             (setf ii (f2cl-lib:int-sub i offset))
             (setf (f2cl-lib:fref wgap-%data% ((f2cl-lib:int-sub ii 1)) ((1 *))
                                  wgap-%offset%)
                     (max zero
                          (- (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%)
                             (f2cl-lib:fref werr-%data% (ii) ((1 *))
                                            werr-%offset%)
                             (f2cl-lib:fref w-%data% ((f2cl-lib:int-sub ii 1))
                                            ((1 *)) w-%offset%)
                             (f2cl-lib:fref werr-%data%
                                            ((f2cl-lib:int-sub ii 1)) ((1 *))
                                            werr-%offset%))))
           label111))
         (go end_label)
        end_label
         (return
          (values nil nil nil nil nil nil nil nil nil nil nil nil nil nil nil
                  nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlarrb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
```

```
  (double-float) (double-float)
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (double-float) (double-float)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil nil nil nil nil nil
  nil nil nil nil nil nil nil nil
  fortran-to-lisp::info)
:calls '(fortran-to-lisp::dlaneg))))
```

———-

90.21 dlarrc

90.21.1 Axiom unit tests

— dlarrc.input —

```
)set break resume
)sys rm -f dlarrc.output
)spool dlarrc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.21.2 Axiom help page

— dlarrc.help —

```
==================================================================
```

```
dlarrc examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlarrc (JOBT, N, VL, VU, D, E, PIVMIN, EIGCNT, LCNT, RCNT,
           INFO)
           DLARRC computes the number of eigenvalues of the symmetric
           tridiagonal matrix.

Function/Subroutine Documentation
   subroutine dlarrc (character JOBT, integer N, double precision VL, double
       precision VU, double precision, dimension( * ) D, double precision,
       dimension( * ) E, double precision PIVMIN, integer EIGCNT, integer
       LCNT, integer RCNT, integer INFO)
       DLARRC computes the number of eigenvalues of the symmetric tridiagonal
       matrix.

Purpose:
   Find the number of eigenvalues of the symmetric tridiagonal matrix T
   that are in the interval (VL,VU] if JOBT = 'T', and of L D L^T
   if JOBT = 'L'.

Parameters:

 JOBT
   JOBT is CHARACTER*1
   = 'T':  Compute Sturm count for matrix T.
   = 'L':  Compute Sturm count for matrix L D L^T.

 N
   N is INTEGER
   The order of the matrix. N > 0.

 VL
   VL is DOUBLE PRECISION

 VU
   VU is DOUBLE PRECISION
   The lower and upper bounds for the eigenvalues.

 D
   D is DOUBLE PRECISION array, dimension (N)
   JOBT = 'T': The N diagonal elements of the tridiagonal matrix T.
   JOBT = 'L': The N diagonal elements of the diagonal matrix D.
```

```
E
  E is DOUBLE PRECISION array, dimension (N)
  JOBT = 'T': The N-1 offdiagonal elements of the matrix T.
  JOBT = 'L': The N-1 offdiagonal elements of the matrix L.

PIVMIN
  PIVMIN is DOUBLE PRECISION
  The minimum pivot in the Sturm sequence for T.

EIGCNT
  EIGCNT is INTEGER
  The number of eigenvalues of the symmetric tridiagonal matrix T
  that are in the interval (VL,VU]

LCNT
  LCNT is INTEGER

RCNT
  RCNT is INTEGER
  The left and right negcounts of the interval.

INFO
  INFO is INTEGER
```

———-

90.21.3 fortran code

— dlarrc.f —

```
*  =====================================================================
      SUBROUTINE DLARRC( JOBT, N, VL, VU, D, E, PIVMIN,
     $                   EIGCNT, LCNT, RCNT, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          JOBT
      INTEGER            EIGCNT, INFO, LCNT, N, RCNT
      DOUBLE PRECISION   PIVMIN, VL, VU
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), E( * )
*     ..
```

```
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      LOGICAL            MATT
      DOUBLE PRECISION   LPIVOT, RPIVOT, SL, SU, TMP, TMP2

*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      LCNT = 0
      RCNT = 0
      EIGCNT = 0
      MATT = LSAME( JOBT, 'T' )


      IF (MATT) THEN
*        Sturm sequence count on T
         LPIVOT = D( 1 ) - VL
         RPIVOT = D( 1 ) - VU
         IF( LPIVOT.LE.ZERO ) THEN
            LCNT = LCNT + 1
         ENDIF
         IF( RPIVOT.LE.ZERO ) THEN
            RCNT = RCNT + 1
         ENDIF
         DO 10 I = 1, N-1
            TMP = E(I)**2
            LPIVOT = ( D( I+1 )-VL ) - TMP/LPIVOT
            RPIVOT = ( D( I+1 )-VU ) - TMP/RPIVOT
            IF( LPIVOT.LE.ZERO ) THEN
               LCNT = LCNT + 1
            ENDIF
            IF( RPIVOT.LE.ZERO ) THEN
               RCNT = RCNT + 1
            ENDIF
 10      CONTINUE
      ELSE
*        Sturm sequence count on L D L^T
         SL = -VL
```

```
         SU = -VU
         DO 20 I = 1, N - 1
            LPIVOT = D( I ) + SL
            RPIVOT = D( I ) + SU
            IF( LPIVOT.LE.ZERO ) THEN
               LCNT = LCNT + 1
            ENDIF
            IF( RPIVOT.LE.ZERO ) THEN
               RCNT = RCNT + 1
            ENDIF
            TMP = E(I) * D(I) * E(I)
*
            TMP2 = TMP / LPIVOT
            IF( TMP2.EQ.ZERO ) THEN
               SL =  TMP - VL
            ELSE
               SL = SL*TMP2 - VL
            END IF
*
            TMP2 = TMP / RPIVOT
            IF( TMP2.EQ.ZERO ) THEN
               SU =  TMP - VU
            ELSE
               SU = SU*TMP2 - VU
            END IF
 20      CONTINUE
         LPIVOT = D( N ) + SL
         RPIVOT = D( N ) + SU
         IF( LPIVOT.LE.ZERO ) THEN
            LCNT = LCNT + 1
         ENDIF
         IF( RPIVOT.LE.ZERO ) THEN
            RCNT = RCNT + 1
         ENDIF
      ENDIF
      EIGCNT = RCNT - LCNT

      RETURN
*
*     end of DLARRC
*
      END
```

———-

90.21.4 lisp code

— dlarrc.lisp —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun dlarrc (jobt n vl vu d e pivmin eigcnt lcnt rcnt info)
    (declare (type (array double-float (*)) e d)
             (type (double-float) pivmin vu vl)
             (type (f2cl-lib:integer4) info rcnt lcnt eigcnt n)
             (type (string *) jobt))
    (f2cl-lib:with-multi-array-data
        ((jobt character jobt-%data% jobt-%offset%)
         (d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%))
      (prog ((lpivot 0.0d0) (rpivot 0.0d0) (sl 0.0d0) (su 0.0d0) (tmp 0.0d0)
             (tmp2 0.0d0) (matt nil) (i 0))
        (declare (type (double-float) lpivot rpivot sl su tmp tmp2)
                 (type f2cl-lib:logical matt)
                 (type (f2cl-lib:integer4) i))
        (setf info 0)
        (setf lcnt 0)
        (setf rcnt 0)
        (setf eigcnt 0)
        (setf matt
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame jobt "T")
                  (declare (ignore var-1))
                  (when var-0 (setf jobt var-0))
                  ret-val))
        (cond
         (matt
          (setf lpivot (- (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%) vl))
          (setf rpivot (- (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%) vu))
          (cond ((<= lpivot zero) (setf lcnt (f2cl-lib:int-add lcnt 1))))
          (cond ((<= rpivot zero) (setf rcnt (f2cl-lib:int-add rcnt 1))))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
            (tagbody
              (setf tmp
                      (expt (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) 2))
              (setf lpivot
                      (+
                       (-
                        (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                       ((1 *)) d-%offset%)
                        vl)
                       (/ (- tmp) lpivot)))
              (setf rpivot
                      (+
                       (-
                        (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                       ((1 *)) d-%offset%)
```

```
                      vu)
                     (/ (- tmp) rpivot)))
             (cond ((<= lpivot zero) (setf lcnt (f2cl-lib:int-add lcnt 1))))
             (cond ((<= rpivot zero) (setf rcnt (f2cl-lib:int-add rcnt 1))))
            label10)))
         (t (setf sl (- vl)) (setf su (- vu))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
           (tagbody
             (setf lpivot
                     (+ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) sl))
             (setf rpivot
                     (+ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) su))
             (cond ((<= lpivot zero) (setf lcnt (f2cl-lib:int-add lcnt 1))))
             (cond ((<= rpivot zero) (setf rcnt (f2cl-lib:int-add rcnt 1))))
             (setf tmp
                     (* (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                        (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                        (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
             (setf tmp2 (/ tmp lpivot))
             (cond ((= tmp2 zero) (setf sl (- tmp vl)))
                   (t (setf sl (- (* sl tmp2) vl))))
             (setf tmp2 (/ tmp rpivot))
             (cond ((= tmp2 zero) (setf su (- tmp vu)))
                   (t (setf su (- (* su tmp2) vu))))
            label20))
          (setf lpivot (+ (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) sl))
          (setf rpivot (+ (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) su))
          (cond ((<= lpivot zero) (setf lcnt (f2cl-lib:int-add lcnt 1))))
          (cond ((<= rpivot zero) (setf rcnt (f2cl-lib:int-add rcnt 1))))))
        (setf eigcnt (f2cl-lib:int-sub rcnt lcnt))
        (go end_label)
       end_label
        (return (values jobt nil nil nil nil nil nil eigcnt lcnt rcnt info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlarrc
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (double-float)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
```

```
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::jobt nil nil nil
  nil nil nil
  fortran-to-lisp::eigcnt
  fortran-to-lisp::lcnt
  fortran-to-lisp::rcnt
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

90.22 dlarrd

90.22.1 Axiom unit tests

— **dlarrd.input** —

```
)set break resume
)sys rm -f dlarrd.output
)spool dlarrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.22.2 Axiom help page

— **dlarrd.help** —

```
====================================================================
dlarrd examples
====================================================================

====================================================================
Man Page Details
====================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine dlarrd (RANGE, ORDER, N, VL, VU, IL, IU, GERS, RELTOL, D, E,
           E2, PIVMIN, NSPLIT, ISPLIT, M, W, WERR, WL, WU, IBLOCK, INDEXW,
           WORK, IWORK, INFO)
           DLARRD computes the eigenvalues of a symmetric tridiagonal matrix
           to suitable accuracy.

Function/Subroutine Documentation
   subroutine dlarrd (character RANGE, character ORDER, integer N, double
       precision VL, double precision VU, integer IL, integer IU, double
       precision, dimension( * ) GERS, double precision RELTOL, double
       precision, dimension( * ) D, double precision, dimension( * ) E, double
       precision, dimension( * ) E2, double precision PIVMIN, integer NSPLIT,
       integer, dimension( * ) ISPLIT, integer M, double precision, dimension(
       * ) W, double precision, dimension( * ) WERR, double precision WL,
       double precision WU, integer, dimension( * ) IBLOCK, integer,
       dimension( * ) INDEXW, double precision, dimension( * ) WORK, integer,
       dimension( * ) IWORK, integer INFO)
       DLARRD computes the eigenvalues of a symmetric tridiagonal matrix to
       suitable accuracy.

Purpose:
   DLARRD computes the eigenvalues of a symmetric tridiagonal
   matrix T to suitable accuracy. This is an auxiliary code to be
   called from DSTEMR.
   The user may ask for all eigenvalues, all eigenvalues
   in the half-open interval (VL, VU], or the IL-th through IU-th
   eigenvalues.

   To avoid overflow, the matrix must be scaled so that its
   largest element is no greater than
   overflow**(1/2) * underflow**(1/4)
   in absolute value, and for greatest
   accuracy, it should not be much smaller than that.

Parameters:

 RANGE
   RANGE is CHARACTER*1
   = 'A': ("All")   all eigenvalues will be found.
   = 'V': ("Value") all eigenvalues in the half-open interval
          (VL, VU] will be found.
   = 'I': ("Index") the IL-th through IU-th eigenvalues (of the
          entire matrix) will be found.

 ORDER
   ORDER is CHARACTER*1
   = 'B': ("By Block") the eigenvalues will be grouped by
        split-off block (see IBLOCK, ISPLIT) and
```

```
        ordered from smallest to largest within the block.
  = 'E': ("Entire matrix")
        the eigenvalues for the entire matrix
        will be ordered from smallest to largest.

N
  N is INTEGER
  The order of the tridiagonal matrix T.  N >= 0.

VL
  VL is DOUBLE PRECISION

VU
  VU is DOUBLE PRECISION
  If RANGE='V', the lower and upper bounds of the interval to
  be searched for eigenvalues.  Eigenvalues less than or equal
  to VL, or greater than VU, will not be returned.  VL < VU.
  Not referenced if RANGE = 'A' or 'I'.

IL
  IL is INTEGER

IU
  IU is INTEGER
  If RANGE='I', the indices (in ascending order) of the
  smallest and largest eigenvalues to be returned.
  1 <= IL <= IU <= N, if N > 0; IL = 1 and IU = 0 if N = 0.
  Not referenced if RANGE = 'A' or 'V'.

GERS
  GERS is DOUBLE PRECISION array, dimension (2*N)
  The N Gerschgorin intervals (the i-th Gerschgorin interval
  is (GERS(2*i-1), GERS(2*i)).

RELTOL
  RELTOL is DOUBLE PRECISION
  The minimum relative width of an interval.  When an interval
  is narrower than RELTOL times the larger (in
  magnitude) endpoint, then it is considered to be
  sufficiently small, i.e., converged.  Note: this should
  always be at least radix*machine epsilon.

D
  D is DOUBLE PRECISION array, dimension (N)
  The n diagonal elements of the tridiagonal matrix T.

E
  E is DOUBLE PRECISION array, dimension (N-1)
  The (n-1) off-diagonal elements of the tridiagonal matrix T.
```

```
E2
  E2 is DOUBLE PRECISION array, dimension (N-1)
  The (n-1) squared off-diagonal elements of the tridiagonal matrix T.

PIVMIN
  PIVMIN is DOUBLE PRECISION
  The minimum pivot allowed in the Sturm sequence for T.

NSPLIT
  NSPLIT is INTEGER
  The number of diagonal blocks in the matrix T.
  1 <= NSPLIT <= N.

ISPLIT
  ISPLIT is INTEGER array, dimension (N)
  The splitting points, at which T breaks up into submatrices.
  The first submatrix consists of rows/columns 1 to ISPLIT(1),
  the second of rows/columns ISPLIT(1)+1 through ISPLIT(2),
  etc., and the NSPLIT-th consists of rows/columns
  ISPLIT(NSPLIT-1)+1 through ISPLIT(NSPLIT)=N.
  (Only the first NSPLIT elements will actually be used, but
  since the user cannot know a priori what value NSPLIT will
  have, N words must be reserved for ISPLIT.)

M
  M is INTEGER
  The actual number of eigenvalues found. 0 <= M <= N.
  (See also the description of INFO=2,3.)

W
  W is DOUBLE PRECISION array, dimension (N)
  On exit, the first M elements of W will contain the
  eigenvalue approximations. DLARRD computes an interval
  I_j = (a_j, b_j] that includes eigenvalue j. The eigenvalue
  approximation is given as the interval midpoint
  W(j)= ( a_j + b_j)/2. The corresponding error is bounded by
  WERR(j) = abs( a_j - b_j)/2

WERR
  WERR is DOUBLE PRECISION array, dimension (N)
  The error bound on the corresponding eigenvalue approximation in W.

WL
  WL is DOUBLE PRECISION

WU
  WU is DOUBLE PRECISION
  The interval (WL, WU] contains all the wanted eigenvalues.
  If RANGE='V', then WL=VL and WU=VU.
  If RANGE='A', then WL and WU are the global Gerschgorin bounds
```

```
        on the spectrum.
  If RANGE='I', then WL and WU are computed by DLAEBZ from the
        index range specified.

 IBLOCK
  IBLOCK is INTEGER array, dimension (N)
  At each row/column j where E(j) is zero or small, the
  matrix T is considered to split into a block diagonal
  matrix.  On exit, if INFO = 0, IBLOCK(i) specifies to which
  block (from 1 to the number of blocks) the eigenvalue W(i)
  belongs.  (DLARRD may use the remaining N-M elements as workspace.)

 INDEXW
  INDEXW is INTEGER array, dimension (N)
  The indices of the eigenvalues within each block (submatrix);
  for example, INDEXW(i)= j and IBLOCK(i)=k imply that the
  i-th eigenvalue W(i) is the j-th eigenvalue in block k.

 WORK
  WORK is DOUBLE PRECISION array, dimension (4*N)

 IWORK
  IWORK is INTEGER array, dimension (3*N)

 INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  some or all of the eigenvalues failed to converge or
        were not computed:
  =1 or 3: Bisection failed to converge for some
        eigenvalues; these eigenvalues are flagged by a
        negative block number.  The effect is that the
        eigenvalues may not be as accurate as the
        absolute and relative tolerances.  This is
        generally caused by unexpectedly inaccurate arithmetic.
  =2 or 3: RANGE='I' only: Not all of the eigenvalues
        IL:IU were found.
        Effect: M < IU+1-IL
        Cause:  non-monotonic arithmetic, causing the
           Sturm sequence to be non-monotonic.
        Cure:   recalculate, using RANGE='A', and pick
           out eigenvalues IL:IU.  In some cases,
           increasing the PARAMETER "FUDGE" may make things work.
  = 4:    RANGE='I', and the Gershgorin interval
        initially used was too small.  No eigenvalues were computed.
        Probable cause: your machine has sloppy floating-point arithmetic.
        Cure: Increase the PARAMETER "FUDGE", recompile, and try again.

Internal Parameters:
```

```
 FUDGE   DOUBLE PRECISION, default = 2
   A "fudge factor" to widen the Gershgorin intervals.  Ideally,
   a value of 1 should work, but on machines with sloppy
   arithmetic, this needs to be larger.  The default for
   publicly released versions should be large enough to handle
   the worst machine around.  Note that this has no effect
   on accuracy of the solution.
```

———-

90.22.3 fortran code

— dlarrd.f —

```
*  =====================================================================
      SUBROUTINE DLARRD( RANGE, ORDER, N, VL, VU, IL, IU, GERS,
     $                   RELTOL, D, E, E2, PIVMIN, NSPLIT, ISPLIT,
     $                   M, W, WERR, WL, WU, IBLOCK, INDEXW,
     $                   WORK, IWORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          ORDER, RANGE
      INTEGER            IL, INFO, IU, M, N, NSPLIT
      DOUBLE PRECISION    PIVMIN, RELTOL, VL, VU, WL, WU
*     ..
*     .. Array Arguments ..
      INTEGER            IBLOCK( * ), INDEXW( * ),
     $                   ISPLIT( * ), IWORK( * )
      DOUBLE PRECISION   D( * ), E( * ), E2( * ),
     $                   GERS( * ), W( * ), WERR( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE, TWO, HALF, FUDGE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0,
     $                     TWO = 2.0D0, HALF = ONE/TWO,
     $                     FUDGE = TWO )
      INTEGER   ALLRNG, VALRNG, INDRNG
      PARAMETER ( ALLRNG = 1, VALRNG = 2, INDRNG = 3 )
```

```
*     ..
*     .. Local Scalars ..
      LOGICAL            NCNVRG, TOOFEW
      INTEGER            I, IB, IBEGIN, IDISCL, IDISCU, IE, IEND, IINFO,
     $                   IM, IN, IOFF, IOUT, IRANGE, ITMAX, ITMP1,
     $                   ITMP2, IW, IWOFF, J, JBLK, JDISC, JE, JEE, NB,
     $                   NWL, NWU
      DOUBLE PRECISION   ATOLI, EPS, GL, GU, RTOLI, TMP1, TMP2,
     $                   TNORM, UFLOW, WKILL, WLU, WUL

*     ..
*     .. Local Arrays ..
      INTEGER            IDUMMA( 1 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, ILAENV, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLAEBZ
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, INT, LOG, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
*     Decode RANGE
*
      IF( LSAME( RANGE, 'A' ) ) THEN
         IRANGE = ALLRNG
      ELSE IF( LSAME( RANGE, 'V' ) ) THEN
         IRANGE = VALRNG
      ELSE IF( LSAME( RANGE, 'I' ) ) THEN
         IRANGE = INDRNG
      ELSE
         IRANGE = 0
      END IF
*
*     Check for Errors
*
      IF( IRANGE.LE.0 ) THEN
         INFO = -1
      ELSE IF( .NOT.(LSAME(ORDER,'B').OR.LSAME(ORDER,'E')) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
```

```
      ELSE IF( IRANGE.EQ.VALRNG ) THEN
         IF( VL.GE.VU )
     $      INFO = -5
      ELSE IF( IRANGE.EQ.INDRNG .AND.
     $        ( IL.LT.1 .OR. IL.GT.MAX( 1, N ) ) ) THEN
         INFO = -6
      ELSE IF( IRANGE.EQ.INDRNG .AND.
     $        ( IU.LT.MIN( N, IL ) .OR. IU.GT.N ) ) THEN
         INFO = -7
      END IF
*
      IF( INFO.NE.0 ) THEN
         RETURN
      END IF

*     Initialize error flags
      INFO = 0
      NCNVRG = .FALSE.
      TOOFEW = .FALSE.

*     Quick return if possible
      M = 0
      IF( N.EQ.0 ) RETURN

*     Simplification:
      IF( IRANGE.EQ.INDRNG .AND. IL.EQ.1 .AND. IU.EQ.N ) IRANGE = 1

*     Get machine constants
      EPS = DLAMCH( 'P' )
      UFLOW = DLAMCH( 'U' )


*     Special Case when N=1
*     Treat case of 1x1 matrix for quick return
      IF( N.EQ.1 ) THEN
         IF( (IRANGE.EQ.ALLRNG).OR.
     $       ((IRANGE.EQ.VALRNG).AND.(D(1).GT.VL).AND.(D(1).LE.VU)).OR.
     $       ((IRANGE.EQ.INDRNG).AND.(IL.EQ.1).AND.(IU.EQ.1)) ) THEN
            M = 1
            W(1) = D(1)
*           The computation error of the eigenvalue is zero
            WERR(1) = ZERO
            IBLOCK( 1 ) = 1
            INDEXW( 1 ) = 1
         ENDIF
         RETURN
      END IF

*     NB is the minimum vector length for vector bisection, or 0
*     if only scalar is to be done.
```

```
      NB = ILAENV( 1, 'DSTEBZ', ' ', N, -1, -1, -1 )
      IF( NB.LE.1 ) NB = 0

*     Find global spectral radius
      GL = D(1)
      GU = D(1)
      DO 5 I = 1,N
         GL =  MIN( GL, GERS( 2*I - 1))
         GU = MAX( GU, GERS(2*I) )
 5    CONTINUE
*     Compute global Gerschgorin bounds and spectral diameter
      TNORM = MAX( ABS( GL ), ABS( GU ) )
      GL = GL - FUDGE*TNORM*EPS*N - FUDGE*TWO*PIVMIN
      GU = GU + FUDGE*TNORM*EPS*N + FUDGE*TWO*PIVMIN
*     [JAN/28/2009] remove the line below since SPDIAM variable not use
*     SPDIAM = GU - GL
*     Input arguments for DLAEBZ:
*     The relative tolerance.  An interval (a,b] lies within
*     "relative tolerance" if  b-a < RELTOL*max(|a|,|b|),
      RTOLI = RELTOL
*     Set the absolute tolerance for interval convergence to zero to force
*     interval convergence based on relative size of the interval.
*     This is dangerous because intervals might not converge when RELTOL is
*     small. But at least a very small number should be selected so that for
*     strongly graded matrices, the code can get relatively accurate
*     eigenvalues.
      ATOLI = FUDGE*TWO*UFLOW + FUDGE*TWO*PIVMIN

      IF( IRANGE.EQ.INDRNG ) THEN

*        RANGE='I': Compute an interval containing eigenvalues
*        IL through IU. The initial interval [GL,GU] from the global
*        Gerschgorin bounds GL and GU is refined by DLAEBZ.
         ITMAX = INT( ( LOG( TNORM+PIVMIN )-LOG( PIVMIN ) ) /
     $           LOG( TWO ) ) + 2
         WORK( N+1 ) = GL
         WORK( N+2 ) = GL
         WORK( N+3 ) = GU
         WORK( N+4 ) = GU
         WORK( N+5 ) = GL
         WORK( N+6 ) = GU
         IWORK( 1 ) = -1
         IWORK( 2 ) = -1
         IWORK( 3 ) = N + 1
         IWORK( 4 ) = N + 1
         IWORK( 5 ) = IL - 1
         IWORK( 6 ) = IU
*
         CALL DLAEBZ( 3, ITMAX, N, 2, 2, NB, ATOLI, RTOLI, PIVMIN,
     $         D, E, E2, IWORK( 5 ), WORK( N+1 ), WORK( N+5 ), IOUT,
```

```
     $                IWORK, W, IBLOCK, IINFO )
         IF( IINFO .NE. 0 ) THEN
            INFO = IINFO
            RETURN
         END IF
*        On exit, output intervals may not be ordered by ascending negcount
         IF( IWORK( 6 ).EQ.IU ) THEN
            WL = WORK( N+1 )
            WLU = WORK( N+3 )
            NWL = IWORK( 1 )
            WU = WORK( N+4 )
            WUL = WORK( N+2 )
            NWU = IWORK( 4 )
         ELSE
            WL = WORK( N+2 )
            WLU = WORK( N+4 )
            NWL = IWORK( 2 )
            WU = WORK( N+3 )
            WUL = WORK( N+1 )
            NWU = IWORK( 3 )
         END IF
*        On exit, the interval [WL, WLU] contains a value with negcount NWL,
*        and [WUL, WU] contains a value with negcount NWU.
         IF( NWL.LT.0 .OR. NWL.GE.N .OR. NWU.LT.1 .OR. NWU.GT.N ) THEN
            INFO = 4
            RETURN
         END IF

      ELSEIF( IRANGE.EQ.VALRNG ) THEN
         WL = VL
         WU = VU

      ELSEIF( IRANGE.EQ.ALLRNG ) THEN
         WL = GL
         WU = GU
      ENDIF



*     Find Eigenvalues -- Loop Over blocks and recompute NWL and NWU.
*     NWL accumulates the number of eigenvalues .le. WL,
*     NWU accumulates the number of eigenvalues .le. WU
      M = 0
      IEND = 0
      INFO = 0
      NWL = 0
      NWU = 0
*
      DO 70 JBLK = 1, NSPLIT
         IOFF = IEND
```

```
         IBEGIN = IOFF + 1
         IEND = ISPLIT( JBLK )
         IN = IEND - IOFF
*
         IF( IN.EQ.1 ) THEN
*           1x1 block
            IF( WL.GE.D( IBEGIN )-PIVMIN )
     $         NWL = NWL + 1
            IF( WU.GE.D( IBEGIN )-PIVMIN )
     $         NWU = NWU + 1
            IF( IRANGE.EQ.ALLRNG .OR.
     $           ( WL.LT.D( IBEGIN )-PIVMIN
     $             .AND. WU.GE. D( IBEGIN )-PIVMIN ) ) THEN
               M = M + 1
               W( M ) = D( IBEGIN )
               WERR(M) = ZERO
*              The gap for a single block doesn't matter for the later
*              algorithm and is assigned an arbitrary large value
               IBLOCK( M ) = JBLK
               INDEXW( M ) = 1
            END IF

*        Disabled 2x2 case because of a failure on the following matrix
*        RANGE = 'I', IL = IU = 4
*          Original Tridiagonal, d = [
*           -0.150102010615740E+00
*           -0.849897989384260E+00
*           -0.128208148052635E-15
*            0.128257718286320E-15
*          ];
*          e = [
*           -0.357171383266986E+00
*           -0.180411241501588E-15
*           -0.175152352710251E-15
*          ];
*
*         ELSE IF( IN.EQ.2 ) THEN
**           2x2 block
*            DISC = SQRT( (HALF*(D(IBEGIN)-D(IEND)))**2 + E(IBEGIN)**2 )
*            TMP1 = HALF*(D(IBEGIN)+D(IEND))
*            L1 = TMP1 - DISC
*            IF( WL.GE. L1-PIVMIN )
*     $         NWL = NWL + 1
*            IF( WU.GE. L1-PIVMIN )
*     $         NWU = NWU + 1
*            IF( IRANGE.EQ.ALLRNG .OR. ( WL.LT.L1-PIVMIN .AND. WU.GE.
*     $          L1-PIVMIN ) ) THEN
*               M = M + 1
*               W( M ) = L1
**              The uncertainty of eigenvalues of a 2x2 matrix is very small
```

```
*              WERR( M ) = EPS * ABS( W( M ) ) * TWO
*              IBLOCK( M ) = JBLK
*              INDEXW( M ) = 1
*           ENDIF
*           L2 = TMP1 + DISC
*           IF( WL.GE. L2-PIVMIN )
*     $        NWL = NWL + 1
*           IF( WU.GE. L2-PIVMIN )
*     $        NWU = NWU + 1
*           IF( IRANGE.EQ.ALLRNG .OR. ( WL.LT.L2-PIVMIN .AND. WU.GE.
*     $         L2-PIVMIN ) ) THEN
*              M = M + 1
*              W( M ) = L2
**             The uncertainty of eigenvalues of a 2x2 matrix is very small
*              WERR( M ) = EPS * ABS( W( M ) ) * TWO
*              IBLOCK( M ) = JBLK
*              INDEXW( M ) = 2
*           ENDIF
         ELSE
*          General Case - block of size IN >= 2
*          Compute local Gerschgorin interval and use it as the initial
*          interval for DLAEBZ
           GU = D( IBEGIN )
           GL = D( IBEGIN )
           TMP1 = ZERO

           DO 40 J = IBEGIN, IEND
              GL =  MIN( GL, GERS( 2*J - 1))
              GU = MAX( GU, GERS(2*J) )
   40      CONTINUE
*          [JAN/28/2009]
*          change SPDIAM by TNORM in lines 2 and 3 thereafter
*          line 1: remove computation of SPDIAM (not useful anymore)
*          SPDIAM = GU - GL
*          GL = GL - FUDGE*SPDIAM*EPS*IN - FUDGE*PIVMIN
*          GU = GU + FUDGE*SPDIAM*EPS*IN + FUDGE*PIVMIN
           GL = GL - FUDGE*TNORM*EPS*IN - FUDGE*PIVMIN
           GU = GU + FUDGE*TNORM*EPS*IN + FUDGE*PIVMIN
*
           IF( IRANGE.GT.1 ) THEN
              IF( GU.LT.WL ) THEN
*                the local block contains none of the wanted eigenvalues
                 NWL = NWL + IN
                 NWU = NWU + IN
                 GO TO 70
              END IF
*             refine search interval if possible, only range (WL,WU] matters
              GL = MAX( GL, WL )
              GU = MIN( GU, WU )
              IF( GL.GE.GU )
```

```
     $            GO TO 70
          END IF

*         Find negcount of initial interval boundaries GL and GU
          WORK( N+1 ) = GL
          WORK( N+IN+1 ) = GU
          CALL DLAEBZ( 1, 0, IN, IN, 1, NB, ATOLI, RTOLI, PIVMIN,
     $                 D( IBEGIN ), E( IBEGIN ), E2( IBEGIN ),
     $                 IDUMMA, WORK( N+1 ), WORK( N+2*IN+1 ), IM,
     $                 IWORK, W( M+1 ), IBLOCK( M+1 ), IINFO )
          IF( IINFO .NE. 0 ) THEN
             INFO = IINFO
             RETURN
          END IF
*
          NWL = NWL + IWORK( 1 )
          NWU = NWU + IWORK( IN+1 )
          IWOFF = M - IWORK( 1 )

*         Compute Eigenvalues
          ITMAX = INT( ( LOG( GU-GL+PIVMIN )-LOG( PIVMIN ) ) /
     $            LOG( TWO ) ) + 2
          CALL DLAEBZ( 2, ITMAX, IN, IN, 1, NB, ATOLI, RTOLI, PIVMIN,
     $                 D( IBEGIN ), E( IBEGIN ), E2( IBEGIN ),
     $                 IDUMMA, WORK( N+1 ), WORK( N+2*IN+1 ), IOUT,
     $                 IWORK, W( M+1 ), IBLOCK( M+1 ), IINFO )
          IF( IINFO .NE. 0 ) THEN
             INFO = IINFO
             RETURN
          END IF
*
*         Copy eigenvalues into W and IBLOCK
*         Use -JBLK for block number for unconverged eigenvalues.
*         Loop over the number of output intervals from DLAEBZ
          DO 60 J = 1, IOUT
*            eigenvalue approximation is middle point of interval
             TMP1 = HALF*( WORK( J+N )+WORK( J+IN+N ) )
*            semi length of error interval
             TMP2 = HALF*ABS( WORK( J+N )-WORK( J+IN+N ) )
             IF( J.GT.IOUT-IINFO ) THEN
*               Flag non-convergence.
                NCNVRG = .TRUE.
                IB = -JBLK
             ELSE
                IB = JBLK
             END IF
             DO 50 JE = IWORK( J ) + 1 + IWOFF,
     $               IWORK( J+IN ) + IWOFF
                W( JE ) = TMP1
                WERR( JE ) = TMP2
```

```
                 INDEXW( JE ) = JE - IWOFF
                 IBLOCK( JE ) = IB
   50         CONTINUE
   60      CONTINUE
*
           M = M + IM
        END IF
   70 CONTINUE

*     If RANGE='I', then (WL,WU) contains eigenvalues NWL+1,...,NWU
*     If NWL+1 < IL or NWU > IU, discard extra eigenvalues.
      IF( IRANGE.EQ.INDRNG ) THEN
         IDISCL = IL - 1 - NWL
         IDISCU = NWU - IU
*
         IF( IDISCL.GT.0 ) THEN
            IM = 0
            DO 80 JE = 1, M
*              Remove some of the smallest eigenvalues from the left so that
*              at the end IDISCL =0. Move all eigenvalues up to the left.
               IF( W( JE ).LE.WLU .AND. IDISCL.GT.0 ) THEN
                  IDISCL = IDISCL - 1
               ELSE
                  IM = IM + 1
                  W( IM ) = W( JE )
                  WERR( IM ) = WERR( JE )
                  INDEXW( IM ) = INDEXW( JE )
                  IBLOCK( IM ) = IBLOCK( JE )
               END IF
 80         CONTINUE
            M = IM
         END IF
         IF( IDISCU.GT.0 ) THEN
*           Remove some of the largest eigenvalues from the right so that
*           at the end IDISCU =0. Move all eigenvalues up to the left.
            IM=M+1
            DO 81 JE = M, 1, -1
               IF( W( JE ).GE.WUL .AND. IDISCU.GT.0 ) THEN
                  IDISCU = IDISCU - 1
               ELSE
                  IM = IM - 1
                  W( IM ) = W( JE )
                  WERR( IM ) = WERR( JE )
                  INDEXW( IM ) = INDEXW( JE )
                  IBLOCK( IM ) = IBLOCK( JE )
               END IF
 81         CONTINUE
            JEE = 0
            DO 82 JE = IM, M
               JEE = JEE + 1
```

```
            W( JEE ) = W( JE )
            WERR( JEE ) = WERR( JE )
            INDEXW( JEE ) = INDEXW( JE )
            IBLOCK( JEE ) = IBLOCK( JE )
 82      CONTINUE
         M = M-IM+1
      END IF

      IF( IDISCL.GT.0 .OR. IDISCU.GT.0 ) THEN
*        Code to deal with effects of bad arithmetic. (If N(w) is
*        monotone non-decreasing, this should never happen.)
*        Some low eigenvalues to be discarded are not in (WL,WLU],
*        or high eigenvalues to be discarded are not in (WUL,WU]
*        so just kill off the smallest IDISCL/largest IDISCU
*        eigenvalues, by marking the corresponding IBLOCK = 0
         IF( IDISCL.GT.0 ) THEN
            WKILL = WU
            DO 100 JDISC = 1, IDISCL
               IW = 0
               DO 90 JE = 1, M
                  IF( IBLOCK( JE ).NE.0 .AND.
     $                 ( W( JE ).LT.WKILL .OR. IW.EQ.0 ) ) THEN
                     IW = JE
                     WKILL = W( JE )
                  END IF
 90            CONTINUE
               IBLOCK( IW ) = 0
 100        CONTINUE
         END IF
         IF( IDISCU.GT.0 ) THEN
            WKILL = WL
            DO 120 JDISC = 1, IDISCU
               IW = 0
               DO 110 JE = 1, M
                  IF( IBLOCK( JE ).NE.0 .AND.
     $                 ( W( JE ).GE.WKILL .OR. IW.EQ.0 ) ) THEN
                     IW = JE
                     WKILL = W( JE )
                  END IF
 110           CONTINUE
               IBLOCK( IW ) = 0
 120        CONTINUE
         END IF
*        Now erase all eigenvalues with IBLOCK set to zero
         IM = 0
         DO 130 JE = 1, M
            IF( IBLOCK( JE ).NE.0 ) THEN
               IM = IM + 1
               W( IM ) = W( JE )
               WERR( IM ) = WERR( JE )
```

```
                 INDEXW( IM ) = INDEXW( JE )
                 IBLOCK( IM ) = IBLOCK( JE )
              END IF
 130        CONTINUE
            M = IM
         END IF
         IF( IDISCL.LT.0 .OR. IDISCU.LT.0 ) THEN
            TOOFEW = .TRUE.
         END IF
      END IF
*
      IF(( IRANGE.EQ.ALLRNG .AND. M.NE.N ).OR.
     $   ( IRANGE.EQ.INDRNG .AND. M.NE.IU-IL+1 ) ) THEN
         TOOFEW = .TRUE.
      END IF

*     If ORDER='B', do nothing the eigenvalues are already sorted by
*        block.
*     If ORDER='E', sort the eigenvalues from smallest to largest

      IF( LSAME(ORDER,'E') .AND. NSPLIT.GT.1 ) THEN
         DO 150 JE = 1, M - 1
            IE = 0
            TMP1 = W( JE )
            DO 140 J = JE + 1, M
               IF( W( J ).LT.TMP1 ) THEN
                  IE = J
                  TMP1 = W( J )
               END IF
  140       CONTINUE
            IF( IE.NE.0 ) THEN
               TMP2 = WERR( IE )
               ITMP1 = IBLOCK( IE )
               ITMP2 = INDEXW( IE )
               W( IE ) = W( JE )
               WERR( IE ) = WERR( JE )
               IBLOCK( IE ) = IBLOCK( JE )
               INDEXW( IE ) = INDEXW( JE )
               W( JE ) = TMP1
               WERR( JE ) = TMP2
               IBLOCK( JE ) = ITMP1
               INDEXW( JE ) = ITMP2
            END IF
  150    CONTINUE
      END IF
*
      INFO = 0
      IF( NCNVRG )
     $   INFO = INFO + 1
      IF( TOOFEW )
```

```
     $   INFO = INFO + 2
      RETURN
*
*     End of DLARRD
*
      END
```

———-

90.22.4 lisp code

— dlarrd.lisp —

```
(let* ((zero 0.0d0)
       (one 1.0d0)
       (two 2.0d0)
       (half (f2cl-lib:f2cl/ one two))
       (fudge two)
       (allrng 1)
       (valrng 2)
       (indrng 3))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float) half)
           (type (double-float) fudge)
           (type (f2cl-lib:integer4 1 1) allrng)
           (type (f2cl-lib:integer4 2 2) valrng)
           (type (f2cl-lib:integer4 3 3) indrng)
           (ignorable zero one two half fudge allrng valrng indrng))
  (defun dlarrd
         (range order n vl vu il iu gers reltol d e e2 pivmin nsplit isplit m w
          werr wl wu iblock indexw work iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork indexw iblock isplit)
             (type (array double-float (*)) work werr w e2 e d gers)
             (type (double-float) wu wl pivmin reltol vu vl)
             (type (f2cl-lib:integer4) info m nsplit iu il n)
             (type (string *) order range))
    (f2cl-lib:with-multi-array-data
        ((range character range-%data% range-%offset%)
         (order character order-%data% order-%offset%)
         (gers double-float gers-%data% gers-%offset%)
         (d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%)
         (e2 double-float e2-%data% e2-%offset%)
         (w double-float w-%data% w-%offset%)
         (werr double-float werr-%data% werr-%offset%)
         (work double-float work-%data% work-%offset%)
```

```
    (isplit f2cl-lib:integer4 isplit-%data% isplit-%offset%)
    (iblock f2cl-lib:integer4 iblock-%data% iblock-%offset%)
    (indexw f2cl-lib:integer4 indexw-%data% indexw-%offset%)
    (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((idumma (make-array 1 :element-type 'f2cl-lib:integer4))
         (atoli 0.0d0) (eps 0.0d0) (gl 0.0d0) (gu 0.0d0) (rtoli 0.0d0)
         (tmp1 0.0d0) (tmp2 0.0d0) (tnorm 0.0d0) (uflow 0.0d0)
         (wkill 0.0d0) (wlu 0.0d0) (wul 0.0d0) (i 0) (ib 0) (ibegin 0)
         (idiscl 0) (idiscu 0) (ie 0) (iend 0) (iinfo 0) (im 0) (in 0)
         (ioff 0) (iout 0) (irange 0) (itmax 0) (itmp1 0) (itmp2 0) (iw 0)
         (iwoff 0) (j 0) (jblk 0) (jdisc 0) (je 0) (jee 0) (nb 0) (nwl 0)
         (nwu 0) (ncnvrg nil) (toofew nil))
    (declare (type (array f2cl-lib:integer4 (1)) idumma)
             (type (double-float) atoli eps gl gu rtoli tmp1 tmp2 tnorm
              uflow wkill wlu wul)
             (type (f2cl-lib:integer4) i ib ibegin idiscl idiscu ie iend
              iinfo im in ioff iout irange itmax itmp1 itmp2 iw iwoff j
              jblk jdisc je jee nb nwl nwu)
             (type f2cl-lib:logical ncnvrg toofew))
    (setf info 0)
    (cond
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame range "A")
        (declare (ignore var-1))
        (when var-0 (setf range var-0))
        ret-val)
      (setf irange allrng))
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame range "V")
        (declare (ignore var-1))
        (when var-0 (setf range var-0))
        ret-val)
      (setf irange valrng))
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame range "I")
        (declare (ignore var-1))
        (when var-0 (setf range var-0))
        ret-val)
      (setf irange indrng))
     (t (setf irange 0)))
    (cond ((<= irange 0) (setf info -1))
          ((not
            (or
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame order "B")
               (declare (ignore var-1))
               (when var-0 (setf order var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1)
                 (lsame order "E")
```

```
            (declare (ignore var-1))
            (when var-0 (setf order var-0))
            ret-val)))
        (setf info -2))
       ((< n 0) (setf info -3))
       ((= irange valrng)
        (if (>= vl vu)
            (setf info -5)))
       ((and (= irange indrng)
             (or (< il 1)
                 (> il
                    (max (the f2cl-lib:integer4 1)
                         (the f2cl-lib:integer4 n)))))
        (setf info -6))
       ((and (= irange indrng)
             (or
              (< iu
                 (min (the f2cl-lib:integer4 n)
                      (the f2cl-lib:integer4 il)))
              (> iu n)))
        (setf info -7)))
(cond ((/= info 0) (go end_label)))
(setf info 0)
(setf ncnvrg f2cl-lib:%false%)
(setf toofew f2cl-lib:%false%)
(setf m 0)
(if (= n 0)
    (go end_label))
(if (and (= irange indrng) (= il 1) (= iu n))
    (setf irange 1))
(setf eps (dlamch "P"))
(setf uflow (dlamch "U"))
(cond
 ((= n 1)
  (cond
   ((or (= irange allrng)
        (and (= irange valrng) (> (f2cl-lib:fref d (1) ((1 *))) vl)
             (<= (f2cl-lib:fref d (1) ((1 *))) vu))
        (and (= irange indrng) (= il 1) (= iu 1)))
    (setf m 1)
    (setf (f2cl-lib:fref w-%data% (1) ((1 *)) w-%offset%)
            (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
    (setf (f2cl-lib:fref werr-%data% (1) ((1 *)) werr-%offset%) zero)
    (setf (f2cl-lib:fref iblock-%data% (1) ((1 *)) iblock-%offset%) 1)
    (setf (f2cl-lib:fref indexw-%data% (1) ((1 *)) indexw-%offset%)
            1)))
  (go end_label)))
(setf nb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
            (ilaenv 1 "DSTEBZ" " " n -1 -1 -1)
          (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6))
          (when var-3 (setf n var-3))
          ret-val))
(if (<= nb 1)
    (setf nb 0))
(setf gl (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
(setf gu (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf gl
            (min gl
                 (f2cl-lib:fref gers-%data%
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-mul 2 i) 1))
                                ((1 *)) gers-%offset%)))
    (setf gu
            (max gu
                 (f2cl-lib:fref gers-%data% ((f2cl-lib:int-mul 2 i))
                                ((1 *)) gers-%offset%)))
   label5))
(setf tnorm (max (abs gl) (abs gu)))
(setf gl (+ gl (* (- fudge) tnorm eps n) (* (- fudge) two pivmin)))
(setf gu (+ gu (* fudge tnorm eps n) (* fudge two pivmin)))
(setf rtoli reltol)
(setf atoli (+ (* fudge two uflow) (* fudge two pivmin)))
(cond
 ((= irange indrng)
  (setf itmax
          (f2cl-lib:int-add
           (f2cl-lib:int
            (/
             (- (f2cl-lib:flog (+ tnorm pivmin))
                (f2cl-lib:flog pivmin))
             (f2cl-lib:flog two)))
           2))
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 1)) ((1 *))
                       work-%offset%)
          gl)
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 2)) ((1 *))
                       work-%offset%)
          gl)
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 3)) ((1 *))
                       work-%offset%)
          gu)
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 4)) ((1 *))
                       work-%offset%)
          gu)
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 5)) ((1 *))
```

```
                      work-%offset%)
        gl)
(setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 6)) ((1 *))
                      work-%offset%)
        gu)
(setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) -1)
(setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%) -1)
(setf (f2cl-lib:fref iwork-%data% (3) ((1 *)) iwork-%offset%)
        (f2cl-lib:int-add n 1))
(setf (f2cl-lib:fref iwork-%data% (4) ((1 *)) iwork-%offset%)
        (f2cl-lib:int-add n 1))
(setf (f2cl-lib:fref iwork-%data% (5) ((1 *)) iwork-%offset%)
        (f2cl-lib:int-sub il 1))
(setf (f2cl-lib:fref iwork-%data% (6) ((1 *)) iwork-%offset%) iu)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18
     var-19)
    (dlaebz 3 itmax n 2 2 nb atoli rtoli pivmin d e e2
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (5) ((1 *))
                           iwork-%offset%)
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1)) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float ((+ n 5)) ((1 *))
                           work-%offset%)
     iout iwork w iblock iinfo)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
    var-9 var-10 var-11 var-12 var-13 var-14 var-16 var-17 var-18))
  (setf iout var-15)
  (setf iinfo var-19))
(cond ((/= iinfo 0) (setf info iinfo) (go end_label)))
(cond
 ((= (f2cl-lib:fref iwork (6) ((1 *))) iu)
  (setf wl
          (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 1)) ((1 *))
                         work-%offset%))
  (setf wlu
          (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 3)) ((1 *))
                         work-%offset%))
  (setf nwl (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%))
  (setf wu
          (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 4)) ((1 *))
                         work-%offset%))
  (setf wul
          (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 2)) ((1 *))
                         work-%offset%))
  (setf nwu (f2cl-lib:fref iwork-%data% (4) ((1 *)) iwork-%offset%)))
 (t
  (setf wl
```

```
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 2)) ((1 *))
                           work-%offset%))
    (setf wlu
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 4)) ((1 *))
                           work-%offset%))
    (setf nwl (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%))
    (setf wu
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 3)) ((1 *))
                           work-%offset%))
    (setf wul
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 1)) ((1 *))
                           work-%offset%))
    (setf nwu
            (f2cl-lib:fref iwork-%data% (3) ((1 *)) iwork-%offset%))))
  (cond
   ((or (< nwl 0) (>= nwl n) (< nwu 1) (> nwu n)) (setf info 4)
    (go end_label))))
 ((= irange valrng) (setf wl vl) (setf wu vu))
 ((= irange allrng) (setf wl gl) (setf wu gu)))
(setf m 0)
(setf iend 0)
(setf info 0)
(setf nwl 0)
(setf nwu 0)
(f2cl-lib:fdo (jblk 1 (f2cl-lib:int-add jblk 1))
              ((> jblk nsplit) nil)
  (tagbody
    (setf ioff iend)
    (setf ibegin (f2cl-lib:int-add ioff 1))
    (setf iend
            (f2cl-lib:fref isplit-%data% (jblk) ((1 *))
                           isplit-%offset%))
    (setf in (f2cl-lib:int-sub iend ioff))
    (cond
     ((= in 1)
      (if (>= wl
              (- (f2cl-lib:fref d-%data% (ibegin) ((1 *)) d-%offset%)
                 pivmin))
          (setf nwl (f2cl-lib:int-add nwl 1)))
      (if (>= wu
              (- (f2cl-lib:fref d-%data% (ibegin) ((1 *)) d-%offset%)
                 pivmin))
          (setf nwu (f2cl-lib:int-add nwu 1)))
      (cond
       ((or (= irange allrng)
            (and
             (< wl (+ (f2cl-lib:fref d (ibegin) ((1 *))) (- pivmin)))
             (>= wu
                 (+ (f2cl-lib:fref d (ibegin) ((1 *))) (- pivmin)))))
        (setf m (f2cl-lib:int-add m 1))
```

```
    (setf (f2cl-lib:fref w-%data% (m) ((1 *)) w-%offset%)
            (f2cl-lib:fref d-%data% (ibegin) ((1 *)) d-%offset%))
    (setf (f2cl-lib:fref werr-%data% (m) ((1 *)) werr-%offset%)
            zero)
    (setf (f2cl-lib:fref iblock-%data% (m) ((1 *)) iblock-%offset%)
            jblk)
    (setf (f2cl-lib:fref indexw-%data% (m) ((1 *)) indexw-%offset%)
            1))))
  (t (setf gu (f2cl-lib:fref d-%data% (ibegin) ((1 *)) d-%offset%))
   (setf gl (f2cl-lib:fref d-%data% (ibegin) ((1 *)) d-%offset%))
   (setf tmp1 zero)
   (f2cl-lib:fdo (j ibegin (f2cl-lib:int-add j 1))
                 ((> j iend) nil)
    (tagbody
      (setf gl
              (min gl
                   (f2cl-lib:fref gers-%data%
                                  ((f2cl-lib:int-sub
                                    (f2cl-lib:int-mul 2 j) 1))
                                  ((1 *)) gers-%offset%)))
      (setf gu
              (max gu
                   (f2cl-lib:fref gers-%data%
                                  ((f2cl-lib:int-mul 2 j)) ((1 *))
                                  gers-%offset%)))
     label40))
   (setf gl (- (+ gl (* (- fudge) tnorm eps in)) (* fudge pivmin)))
   (setf gu (+ gu (* fudge tnorm eps in) (* fudge pivmin)))
   (cond
    ((> irange 1)
     (cond
      ((< gu wl) (setf nwl (f2cl-lib:int-add nwl in))
       (setf nwu (f2cl-lib:int-add nwu in)) (go label70)))
     (setf gl (max gl wl)) (setf gu (min gu wu))
     (if (>= gl gu)
         (go label70))))
   (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 1)) ((1 *))
                        work-%offset%)
          gl)
   (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n in 1))
                        ((1 *)) work-%offset%)
          gu)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17
        var-18 var-19)
       (dlaebz 1 0 in in 1 nb atoli rtoli pivmin
        (f2cl-lib:array-slice d-%data% double-float (ibegin) ((1 *))
                              d-%offset%)
        (f2cl-lib:array-slice e-%data% double-float (ibegin) ((1 *))
```

```
                              e-%offset%)
     (f2cl-lib:array-slice e2-%data% double-float (ibegin)
                              ((1 *)) e2-%offset%)
     idumma
     (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                              ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float
                              ((+ n (f2cl-lib:int-mul 2 in) 1))
                              ((1 *)) work-%offset%)
     im iwork
     (f2cl-lib:array-slice w-%data% double-float ((+ m 1))
                              ((1 *)) w-%offset%)
     (f2cl-lib:array-slice iblock-%data% f2cl-lib:integer4
                              ((+ m 1)) ((1 *)) iblock-%offset%)
     iinfo)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
    var-9 var-10 var-11 var-12 var-13 var-14 var-16 var-17
    var-18))
  (setf im var-15)
  (setf iinfo var-19))
(cond ((/= iinfo 0) (setf info iinfo) (go end_label)))
(setf nwl
        (f2cl-lib:int-add nwl
                              (f2cl-lib:fref iwork-%data% (1) ((1 *))
                                             iwork-%offset%)))
(setf nwu
        (f2cl-lib:int-add nwu
                              (f2cl-lib:fref iwork-%data%
                                             ((f2cl-lib:int-add in
                                                                1))
                                             ((1 *))
                                             iwork-%offset%)))
(setf iwoff
        (f2cl-lib:int-sub m
                              (f2cl-lib:fref iwork-%data% (1) ((1 *))
                                             iwork-%offset%)))
(setf itmax
        (f2cl-lib:int-add
         (f2cl-lib:int
          (/
           (- (f2cl-lib:flog (+ (- gu gl) pivmin))
              (f2cl-lib:flog pivmin))
           (f2cl-lib:flog two)))
         2))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17
     var-18 var-19)
    (dlaebz 2 itmax in in 1 nb atoli rtoli pivmin
```

```
      (f2cl-lib:array-slice d-%data% double-float (ibegin) ((1 *))
                            d-%offset%)
      (f2cl-lib:array-slice e-%data% double-float (ibegin) ((1 *))
                            e-%offset%)
      (f2cl-lib:array-slice e2-%data% double-float (ibegin)
                            ((1 *)) e2-%offset%)
      idumma
      (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float
                            ((+ n (f2cl-lib:int-mul 2 in) 1))
                            ((1 *)) work-%offset%)
      iout iwork
      (f2cl-lib:array-slice w-%data% double-float ((+ m 1))
                            ((1 *)) w-%offset%)
      (f2cl-lib:array-slice iblock-%data% f2cl-lib:integer4
                            ((+ m 1)) ((1 *)) iblock-%offset%)
      iinfo)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
    var-9 var-10 var-11 var-12 var-13 var-14 var-16 var-17
    var-18))
  (setf iout var-15)
  (setf iinfo var-19))
(cond ((/= iinfo 0) (setf info iinfo) (go end_label)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j iout) nil)
  (tagbody
    (setf tmp1
            (* half
               (+
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add j n)) ((1 *))
                               work-%offset%)
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-add j in n))
                               ((1 *)) work-%offset%))))
    (setf tmp2
            (* half
               (abs
                (-
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add j n)) ((1 *))
                                work-%offset%)
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add j in n))
                                ((1 *)) work-%offset%)))))
    (cond
     ((> j (f2cl-lib:int-add iout (f2cl-lib:int-sub iinfo)))
      (setf ncnvrg f2cl-lib:%true%)
```

```
              (setf ib (f2cl-lib:int-sub jblk)))
             (t (setf ib jblk)))
            (f2cl-lib:fdo (je
                           (f2cl-lib:int-add
                            (f2cl-lib:fref iwork-%data% (j) ((1 *))
                                           iwork-%offset%)
                            1 iwoff)
                           (f2cl-lib:int-add je 1))
                          ((> je
                              (f2cl-lib:int-add
                               (f2cl-lib:fref iwork
                                              ((f2cl-lib:int-add j in))
                                              ((1 *)))
                               iwoff))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref w-%data% (je) ((1 *)) w-%offset%)
                      tmp1)
                (setf (f2cl-lib:fref werr-%data% (je) ((1 *))
                                     werr-%offset%)
                      tmp2)
                (setf (f2cl-lib:fref indexw-%data% (je) ((1 *))
                                     indexw-%offset%)
                      (f2cl-lib:int-sub je iwoff))
                (setf (f2cl-lib:fref iblock-%data% (je) ((1 *))
                                     iblock-%offset%)
                      ib)
               label50))
           label60))
        (setf m (f2cl-lib:int-add m im))))
     label70))
  (cond
   ((= irange indrng) (setf idiscl (f2cl-lib:int-sub il 1 nwl))
    (setf idiscu (f2cl-lib:int-sub nwu iu))
    (cond
     ((> idiscl 0) (setf im 0)
      (f2cl-lib:fdo (je 1 (f2cl-lib:int-add je 1))
                    ((> je m) nil)
        (tagbody
          (cond
           ((and (<= (f2cl-lib:fref w (je) ((1 *))) wlu) (> idiscl 0))
            (setf idiscl (f2cl-lib:int-sub idiscl 1)))
           (t (setf im (f2cl-lib:int-add im 1))
            (setf (f2cl-lib:fref w-%data% (im) ((1 *)) w-%offset%)
                  (f2cl-lib:fref w-%data% (je) ((1 *)) w-%offset%))
            (setf (f2cl-lib:fref werr-%data% (im) ((1 *)) werr-%offset%)
                  (f2cl-lib:fref werr-%data% (je) ((1 *))
                                 werr-%offset%))
            (setf (f2cl-lib:fref indexw-%data% (im) ((1 *))
                                 indexw-%offset%)
```

```
                  (f2cl-lib:fref indexw-%data% (je) ((1 *))
                                 indexw-%offset%))
          (setf (f2cl-lib:fref iblock-%data% (im) ((1 *))
                               iblock-%offset%)
                  (f2cl-lib:fref iblock-%data% (je) ((1 *))
                                 iblock-%offset%))))
      label80))
    (setf m im)))
  (cond
   ((> idiscu 0) (setf im (f2cl-lib:int-add m 1))
    (f2cl-lib:fdo (je m (f2cl-lib:int-add je (f2cl-lib:int-sub 1)))
                  ((> je 1) nil)
      (tagbody
        (cond
         ((and (>= (f2cl-lib:fref w (je) ((1 *))) wul) (> idiscu 0))
          (setf idiscu (f2cl-lib:int-sub idiscu 1)))
         (t (setf im (f2cl-lib:int-sub im 1))
          (setf (f2cl-lib:fref w-%data% (im) ((1 *)) w-%offset%)
                  (f2cl-lib:fref w-%data% (je) ((1 *)) w-%offset%))
          (setf (f2cl-lib:fref werr-%data% (im) ((1 *)) werr-%offset%)
                  (f2cl-lib:fref werr-%data% (je) ((1 *))
                                 werr-%offset%))
          (setf (f2cl-lib:fref indexw-%data% (im) ((1 *))
                               indexw-%offset%)
                  (f2cl-lib:fref indexw-%data% (je) ((1 *))
                                 indexw-%offset%))
          (setf (f2cl-lib:fref iblock-%data% (im) ((1 *))
                               iblock-%offset%)
                  (f2cl-lib:fref iblock-%data% (je) ((1 *))
                                 iblock-%offset%))))
      label81))
    (setf jee 0)
    (f2cl-lib:fdo (je im (f2cl-lib:int-add je 1))
                  ((> je m) nil)
      (tagbody
        (setf jee (f2cl-lib:int-add jee 1))
        (setf (f2cl-lib:fref w-%data% (jee) ((1 *)) w-%offset%)
                (f2cl-lib:fref w-%data% (je) ((1 *)) w-%offset%))
        (setf (f2cl-lib:fref werr-%data% (jee) ((1 *)) werr-%offset%)
                (f2cl-lib:fref werr-%data% (je) ((1 *)) werr-%offset%))
        (setf (f2cl-lib:fref indexw-%data% (jee) ((1 *))
                             indexw-%offset%)
                (f2cl-lib:fref indexw-%data% (je) ((1 *))
                               indexw-%offset%))
        (setf (f2cl-lib:fref iblock-%data% (jee) ((1 *))
                             iblock-%offset%)
                (f2cl-lib:fref iblock-%data% (je) ((1 *))
                               iblock-%offset%))
      label82))
    (setf m (f2cl-lib:int-add (f2cl-lib:int-sub m im) 1))))
```

```
(cond
 ((or (> idiscl 0) (> idiscu 0))
  (cond
   ((> idiscl 0) (setf wkill wu)
    (f2cl-lib:fdo (jdisc 1 (f2cl-lib:int-add jdisc 1))
                  ((> jdisc idiscl) nil)
      (tagbody
        (setf iw 0)
        (f2cl-lib:fdo (je 1 (f2cl-lib:int-add je 1))
                      ((> je m) nil)
          (tagbody
            (cond
             ((and (/= (f2cl-lib:fref iblock (je) ((1 *))) 0)
                   (or (< (f2cl-lib:fref w (je) ((1 *))) wkill)
                       (= iw 0)))
              (setf iw je)
              (setf wkill
                      (f2cl-lib:fref w-%data% (je) ((1 *))
                                     w-%offset%))))
           label90))
        (setf (f2cl-lib:fref iblock-%data% (iw) ((1 *))
                             iblock-%offset%)
              0)
      label100))))
  (cond
   ((> idiscu 0) (setf wkill wl)
    (f2cl-lib:fdo (jdisc 1 (f2cl-lib:int-add jdisc 1))
                  ((> jdisc idiscu) nil)
      (tagbody
        (setf iw 0)
        (f2cl-lib:fdo (je 1 (f2cl-lib:int-add je 1))
                      ((> je m) nil)
          (tagbody
            (cond
             ((and (/= (f2cl-lib:fref iblock (je) ((1 *))) 0)
                   (or (>= (f2cl-lib:fref w (je) ((1 *))) wkill)
                       (= iw 0)))
              (setf iw je)
              (setf wkill
                      (f2cl-lib:fref w-%data% (je) ((1 *))
                                     w-%offset%))))
           label110))
        (setf (f2cl-lib:fref iblock-%data% (iw) ((1 *))
                             iblock-%offset%)
              0)
      label120))))
  (setf im 0)
  (f2cl-lib:fdo (je 1 (f2cl-lib:int-add je 1))
                ((> je m) nil)
    (tagbody
```

```
          (cond
           ((/= (f2cl-lib:fref iblock (je) ((1 *))) 0)
            (setf im (f2cl-lib:int-add im 1))
            (setf (f2cl-lib:fref w-%data% (im) ((1 *)) w-%offset%)
                    (f2cl-lib:fref w-%data% (je) ((1 *)) w-%offset%))
            (setf (f2cl-lib:fref werr-%data% (im) ((1 *)) werr-%offset%)
                    (f2cl-lib:fref werr-%data% (je) ((1 *))
                                   werr-%offset%))
            (setf (f2cl-lib:fref indexw-%data% (im) ((1 *))
                                 indexw-%offset%)
                    (f2cl-lib:fref indexw-%data% (je) ((1 *))
                                   indexw-%offset%))
            (setf (f2cl-lib:fref iblock-%data% (im) ((1 *))
                                 iblock-%offset%)
                    (f2cl-lib:fref iblock-%data% (je) ((1 *))
                                   iblock-%offset%))))
        label130))
     (setf m im)))
  (cond
   ((or (< idiscl 0) (< idiscu 0)) (setf toofew f2cl-lib:%true%)))))
(cond
 ((or (and (= irange allrng) (/= m n))
      (and (= irange indrng)
           (/= m (f2cl-lib:int-add iu (f2cl-lib:int-sub il) 1))))
  (setf toofew f2cl-lib:%true%)))
(cond
 ((and
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame order "E")
     (declare (ignore var-1))
     (when var-0 (setf order var-0))
     ret-val)
   (> nsplit 1))
  (f2cl-lib:fdo (je 1 (f2cl-lib:int-add je 1))
                ((> je (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (setf ie 0)
      (setf tmp1 (f2cl-lib:fref w-%data% (je) ((1 *)) w-%offset%))
      (f2cl-lib:fdo (j (f2cl-lib:int-add je 1) (f2cl-lib:int-add j 1))
                    ((> j m) nil)
        (tagbody
          (cond
           ((< (f2cl-lib:fref w (j) ((1 *))) tmp1) (setf ie j)
            (setf tmp1
                    (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%))))
         label140))
      (cond
       ((/= ie 0)
        (setf tmp2
                (f2cl-lib:fref werr-%data% (ie) ((1 *)) werr-%offset%))
```

```
                (setf itmp1
                        (f2cl-lib:fref iblock-%data% (ie) ((1 *))
                                       iblock-%offset%))
                (setf itmp2
                        (f2cl-lib:fref indexw-%data% (ie) ((1 *))
                                       indexw-%offset%))
                (setf (f2cl-lib:fref w-%data% (ie) ((1 *)) w-%offset%)
                        (f2cl-lib:fref w-%data% (je) ((1 *)) w-%offset%))
                (setf (f2cl-lib:fref werr-%data% (ie) ((1 *)) werr-%offset%)
                        (f2cl-lib:fref werr-%data% (je) ((1 *)) werr-%offset%))
                (setf (f2cl-lib:fref iblock-%data% (ie) ((1 *))
                                     iblock-%offset%)
                        (f2cl-lib:fref iblock-%data% (je) ((1 *))
                                       iblock-%offset%))
                (setf (f2cl-lib:fref indexw-%data% (ie) ((1 *))
                                     indexw-%offset%)
                        (f2cl-lib:fref indexw-%data% (je) ((1 *))
                                       indexw-%offset%))
                (setf (f2cl-lib:fref w-%data% (je) ((1 *)) w-%offset%) tmp1)
                (setf (f2cl-lib:fref werr-%data% (je) ((1 *)) werr-%offset%)
                        tmp2)
                (setf (f2cl-lib:fref iblock-%data% (je) ((1 *))
                                     iblock-%offset%)
                        itmp1)
                (setf (f2cl-lib:fref indexw-%data% (je) ((1 *))
                                     indexw-%offset%)
                        itmp2)))
            label150))))
        (setf info 0)
        (if ncnvrg
            (setf info (f2cl-lib:int-add info 1)))
        (if toofew
            (setf info (f2cl-lib:int-add info 2)))
        (go end_label)
       end_label
        (return
         (values range order n nil nil nil nil nil nil nil nil nil nil nil nil
                 m nil nil wl wu nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlarrd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
```

```
  (array double-float (*))
  (double-float)
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (double-float)
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (double-float) (double-float)
  (array fortran-to-lisp::integer4
   (*))
  (array fortran-to-lisp::integer4
   (*))
  (array double-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::range
  fortran-to-lisp::order
  fortran-to-lisp::n nil nil nil
  nil nil nil nil nil nil nil nil
  nil fortran-to-lisp::m nil nil
  fortran-to-lisp::wl
  fortran-to-lisp::wu nil nil nil
  nil fortran-to-lisp::info)
:calls '(fortran-to-lisp::dlaebz))))
```

———-

90.23 dlarre

90.23.1 Axiom unit tests

— dlarre.input —

```
)set break resume
)sys rm -f dlarre.output
)spool dlarre.output
)set message test on
)set message auto off
```

```
)clear all

)spool
)lisp (bye)
```

———-

90.23.2 Axiom help page

— dlarre.help —

```
=====================================================================
dlarre examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlarre (RANGE, N, VL, VU, IL, IU, D, E, E2, RTOL1, RTOL2,
           SPLTOL, NSPLIT, ISPLIT, M, W, WERR, WGAP, IBLOCK, INDEXW, GERS,
           PIVMIN, WORK, IWORK, INFO)
           DLARRE given the tridiagonal matrix T, sets small off-diagonal
           elements to zero and for each unreduced block Ti, finds base
           representations and eigenvalues.

Function/Subroutine Documentation
   subroutine dlarre (character RANGE, integer N, double precision VL, double
       precision VU, integer IL, integer IU, double precision, dimension( * )
       D, double precision, dimension( * ) E, double precision, dimension( * )
       E2, double precision RTOL1, double precision RTOL2, double precision
       SPLTOL, integer NSPLIT, integer, dimension( * ) ISPLIT, integer M,
       double precision, dimension( * ) W, double precision, dimension( * )
       WERR, double precision, dimension( * ) WGAP, integer, dimension( * )
       IBLOCK, integer, dimension( * ) INDEXW, double precision, dimension( *
       ) GERS, double precision PIVMIN, double precision, dimension( * ) WORK,
       integer, dimension( * ) IWORK, integer INFO)
       DLARRE given the tridiagonal matrix T, sets small off-diagonal elements
       to zero and for each unreduced block Ti, finds base representations and
       eigenvalues.

Purpose:
   To find the desired eigenvalues of a given real symmetric
   tridiagonal matrix T, DLARRE sets any "small" off-diagonal
   elements to zero, and for each unreduced block T_i, it finds
   (a) a suitable shift at one end of the block's spectrum,
```

```
   (b) the base representation, T_i - sigma_i I = L_i D_i L_i^T, and
   (c) eigenvalues of each L_i D_i L_i^T.
   The representations and eigenvalues found are then used by
   DSTEMR to compute the eigenvectors of T.
   The accuracy varies depending on whether bisection is used to
   find a few eigenvalues or the dqds algorithm (subroutine DLASQ2) to
   conpute all and then discard any unwanted one.
   As an added benefit, DLARRE also outputs the n
   Gerschgorin intervals for the matrices L_i D_i L_i^T.

Parameters:

 RANGE
   RANGE is CHARACTER*1
   = 'A': ("All")   all eigenvalues will be found.
   = 'V': ("Value") all eigenvalues in the half-open interval
          (VL, VU] will be found.
   = 'I': ("Index") the IL-th through IU-th eigenvalues (of the
          entire matrix) will be found.

 N
   N is INTEGER
   The order of the matrix. N > 0.

 VL
   VL is DOUBLE PRECISION

 VU
   VU is DOUBLE PRECISION
   If RANGE='V', the lower and upper bounds for the eigenvalues.
   Eigenvalues less than or equal to VL, or greater than VU,
   will not be returned.  VL < VU.
   If RANGE='I' or ='A', DLARRE computes bounds on the desired
   part of the spectrum.

 IL
   IL is INTEGER

 IU
   IU is INTEGER
   If RANGE='I', the indices (in ascending order) of the
   smallest and largest eigenvalues to be returned.
   1 <= IL <= IU <= N.

 D
   D is DOUBLE PRECISION array, dimension (N)
   On entry, the N diagonal elements of the tridiagonal matrix T.
   On exit, the N diagonal elements of the diagonal matrices D_i.

 E
```

```
  E is DOUBLE PRECISION array, dimension (N)
  On entry, the first (N-1) entries contain the subdiagonal
  elements of the tridiagonal matrix T; E(N) need not be set.
  On exit, E contains the subdiagonal elements of the unit
  bidiagonal matrices L_i. The entries E( ISPLIT( I ) ),
  1 <= I <= NSPLIT, contain the base points sigma_i on output.

E2
  E2 is DOUBLE PRECISION array, dimension (N)
  On entry, the first (N-1) entries contain the SQUARES of the
  subdiagonal elements of the tridiagonal matrix T;
  E2(N) need not be set.
  On exit, the entries E2( ISPLIT( I ) ),
  1 <= I <= NSPLIT, have been set to zero

RTOL1
  RTOL1 is DOUBLE PRECISION

RTOL2
  RTOL2 is DOUBLE PRECISION
  Parameters for bisection.
  An interval [LEFT,RIGHT] has converged if
  RIGHT-LEFT.LT.MAX( RTOL1*GAP, RTOL2*MAX(|LEFT|,|RIGHT|) )

SPLTOL
  SPLTOL is DOUBLE PRECISION
  The threshold for splitting.

NSPLIT
  NSPLIT is INTEGER
  The number of blocks T splits into. 1 <= NSPLIT <= N.

ISPLIT
  ISPLIT is INTEGER array, dimension (N)
  The splitting points, at which T breaks up into blocks.
  The first block consists of rows/columns 1 to ISPLIT(1),
  the second of rows/columns ISPLIT(1)+1 through ISPLIT(2),
  etc., and the NSPLIT-th consists of rows/columns
  ISPLIT(NSPLIT-1)+1 through ISPLIT(NSPLIT)=N.

M
  M is INTEGER
  The total number of eigenvalues (of all L_i D_i L_i^T) found.

W
  W is DOUBLE PRECISION array, dimension (N)
  The first M elements contain the eigenvalues. The
  eigenvalues of each of the blocks, L_i D_i L_i^T, are
  sorted in ascending order ( DLARRE may use the
  remaining N-M elements as workspace).
```

```
WERR
  WERR is DOUBLE PRECISION array, dimension (N)
  The error bound on the corresponding eigenvalue in W.

WGAP
  WGAP is DOUBLE PRECISION array, dimension (N)
  The separation from the right neighbor eigenvalue in W.
  The gap is only with respect to the eigenvalues of the same block
  as each block has its own representation tree.
  Exception: at the right end of a block we store the left gap

IBLOCK
  IBLOCK is INTEGER array, dimension (N)
  The indices of the blocks (submatrices) associated with the
  corresponding eigenvalues in W; IBLOCK(i)=1 if eigenvalue
  W(i) belongs to the first block from the top, =2 if W(i)
  belongs to the second block, etc.

INDEXW
  INDEXW is INTEGER array, dimension (N)
  The indices of the eigenvalues within each block (submatrix);
  for example, INDEXW(i)= 10 and IBLOCK(i)=2 imply that the
  i-th eigenvalue W(i) is the 10-th eigenvalue in block 2

GERS
  GERS is DOUBLE PRECISION array, dimension (2*N)
  The N Gerschgorin intervals (the i-th Gerschgorin interval
  is (GERS(2*i-1), GERS(2*i)).

PIVMIN
  PIVMIN is DOUBLE PRECISION
  The minimum pivot in the Sturm sequence for T.

WORK
  WORK is DOUBLE PRECISION array, dimension (6*N)
  Workspace.

IWORK
  IWORK is INTEGER array, dimension (5*N)
  Workspace.

INFO
  INFO is INTEGER
  = 0:  successful exit
  > 0:  A problem occured in DLARRE.
  < 0:  One of the called subroutines signaled an internal problem.
        Needs inspection of the corresponding parameter IINFO
        for further information.
```

```
   =-1:  Problem in DLARRD.
   = 2:  No base representation could be found in MAXTRY iterations.
         Increasing MAXTRY and recompilation might be a remedy.
   =-3:  Problem in DLARRB when computing the refined root
         representation for DLASQ2.
   =-4:  Problem in DLARRB when preforming bisection on the
         desired part of the spectrum.
   =-5:  Problem in DLASQ2.
   =-6:  Problem in DLASQ2.

Further Details:

   The base representations are required to suffer very little
   element growth and consequently define all their eigenvalues to
   high relative accuracy.
```

———-

90.23.3 fortran code

— dlarre.f —

```
*  =====================================================================
      SUBROUTINE DLARRE( RANGE, N, VL, VU, IL, IU, D, E, E2,
     $                   RTOL1, RTOL2, SPLTOL, NSPLIT, ISPLIT, M,
     $                   W, WERR, WGAP, IBLOCK, INDEXW, GERS, PIVMIN,
     $                   WORK, IWORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          RANGE
      INTEGER            IL, INFO, IU, M, N, NSPLIT
      DOUBLE PRECISION  PIVMIN, RTOL1, RTOL2, SPLTOL, VL, VU
*     ..
*     .. Array Arguments ..
      INTEGER            IBLOCK( * ), ISPLIT( * ), IWORK( * ),
     $                   INDEXW( * )
      DOUBLE PRECISION   D( * ), E( * ), E2( * ), GERS( * ),
     $                   W( * ),WERR( * ), WGAP( * ), WORK( * )
*     ..
*
*  =====================================================================
*
```

```
*     .. Parameters ..
      DOUBLE PRECISION   FAC, FOUR, FOURTH, FUDGE, HALF, HNDRD,
     $                   MAXGROWTH, ONE, PERT, TWO, ZERO
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0,
     $                     TWO = 2.0D0, FOUR=4.0D0,
     $                     HNDRD = 100.0D0,
     $                     PERT = 8.0D0,
     $                     HALF = ONE/TWO, FOURTH = ONE/FOUR, FAC= HALF,
     $                     MAXGROWTH = 64.0D0, FUDGE = 2.0D0 )
      INTEGER            MAXTRY, ALLRNG, INDRNG, VALRNG
      PARAMETER          ( MAXTRY = 6, ALLRNG = 1, INDRNG = 2,
     $                     VALRNG = 3 )
*     ..
*     .. Local Scalars ..
      LOGICAL            FORCEB, NOREP, USEDQD
      INTEGER            CNT, CNT1, CNT2, I, IBEGIN, IDUM, IEND, IINFO,
     $                   IN, INDL, INDU, IRANGE, J, JBLK, MB, MM,
     $                   WBEGIN, WEND
      DOUBLE PRECISION   AVGAP, BSRTOL, CLWDTH, DMAX, DPIVOT, EABS,
     $                   EMAX, EOLD, EPS, GL, GU, ISLEFT, ISRGHT, RTL,
     $                   RTOL, S1, S2, SAFMIN, SGNDEF, SIGMA, SPDIAM,
     $                   TAU, TMP, TMP1


*     ..
*     .. Local Arrays ..
      INTEGER            ISEED( 4 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION            DLAMCH
      EXTERNAL           DLAMCH, LSAME

*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DLARNV, DLARRA, DLARRB, DLARRC, DLARRD,
     $                   DLASQ2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN

*     ..
*     .. Executable Statements ..
*

      INFO = 0

*
*     Decode RANGE
*
```

```
      IF( LSAME( RANGE, 'A' ) ) THEN
         IRANGE = ALLRNG
      ELSE IF( LSAME( RANGE, 'V' ) ) THEN
         IRANGE = VALRNG
      ELSE IF( LSAME( RANGE, 'I' ) ) THEN
         IRANGE = INDRNG
      END IF

      M = 0

*     Get machine constants
      SAFMIN = DLAMCH( 'S' )
      EPS = DLAMCH( 'P' )

*     Set parameters
      RTL = SQRT(EPS)
      BSRTOL = SQRT(EPS)

*     Treat case of 1x1 matrix for quick return
      IF( N.EQ.1 ) THEN
         IF( (IRANGE.EQ.ALLRNG).OR.
     $       ((IRANGE.EQ.VALRNG).AND.(D(1).GT.VL).AND.(D(1).LE.VU)).OR.
     $       ((IRANGE.EQ.INDRNG).AND.(IL.EQ.1).AND.(IU.EQ.1)) ) THEN
            M = 1
            W(1) = D(1)
*           The computation error of the eigenvalue is zero
            WERR(1) = ZERO
            WGAP(1) = ZERO
            IBLOCK( 1 ) = 1
            INDEXW( 1 ) = 1
            GERS(1) = D( 1 )
            GERS(2) = D( 1 )
         ENDIF
*        store the shift for the initial RRR, which is zero in this case
         E(1) = ZERO
         RETURN
      END IF

*     General case: tridiagonal matrix of order > 1
*
*     Init WERR, WGAP. Compute Gerschgorin intervals and spectral diameter.
*     Compute maximum off-diagonal entry and pivmin.
      GL = D(1)
      GU = D(1)
      EOLD = ZERO
      EMAX = ZERO
      E(N) = ZERO
      DO 5 I = 1,N
         WERR(I) = ZERO
         WGAP(I) = ZERO
```

```
         EABS = ABS( E(I) )
         IF( EABS .GE. EMAX ) THEN
            EMAX = EABS
         END IF
         TMP1 = EABS + EOLD
         GERS( 2*I-1) = D(I) - TMP1
         GL =  MIN( GL, GERS( 2*I - 1))
         GERS( 2*I ) = D(I) + TMP1
         GU = MAX( GU, GERS(2*I) )
         EOLD  = EABS
 5    CONTINUE
*     The minimum pivot allowed in the Sturm sequence for T
      PIVMIN = SAFMIN * MAX( ONE, EMAX**2 )
*     Compute spectral diameter. The Gerschgorin bounds give an
*     estimate that is wrong by at most a factor of SQRT(2)
      SPDIAM = GU - GL

*     Compute splitting points
      CALL DLARRA( N, D, E, E2, SPLTOL, SPDIAM,
     $                    NSPLIT, ISPLIT, IINFO )

*     Can force use of bisection instead of faster DQDS.
*     Option left in the code for future multisection work.
      FORCEB = .FALSE.

*     Initialize USEDQD, DQDS should be used for ALLRNG unless someone
*     explicitly wants bisection.
      USEDQD = (( IRANGE.EQ.ALLRNG ) .AND. (.NOT.FORCEB))

      IF( (IRANGE.EQ.ALLRNG) .AND. (.NOT. FORCEB) ) THEN
*        Set interval [VL,VU] that contains all eigenvalues
         VL = GL
         VU = GU
      ELSE
*        We call DLARRD to find crude approximations to the eigenvalues
*        in the desired range. In case IRANGE = INDRNG, we also obtain the
*        interval (VL,VU] that contains all the wanted eigenvalues.
*        An interval [LEFT,RIGHT] has converged if
*        RIGHT-LEFT.LT.RTOL*MAX(ABS(LEFT),ABS(RIGHT))
*        DLARRD needs a WORK of size 4*N, IWORK of size 3*N
         CALL DLARRD( RANGE, 'B', N, VL, VU, IL, IU, GERS,
     $                    BSRTOL, D, E, E2, PIVMIN, NSPLIT, ISPLIT,
     $                    MM, W, WERR, VL, VU, IBLOCK, INDEXW,
     $                    WORK, IWORK, IINFO )
         IF( IINFO.NE.0 ) THEN
            INFO = -1
            RETURN
         ENDIF
*        Make sure that the entries M+1 to N in W, WERR, IBLOCK, INDEXW are 0
         DO 14 I = MM+1,N
```

```
            W( I ) = ZERO
            WERR( I ) = ZERO
            IBLOCK( I ) = 0
            INDEXW( I ) = 0
 14      CONTINUE
      END IF


***
*     Loop over unreduced blocks
      IBEGIN = 1
      WBEGIN = 1
      DO 170 JBLK = 1, NSPLIT
         IEND = ISPLIT( JBLK )
         IN = IEND - IBEGIN + 1

*        1 X 1 block
         IF( IN.EQ.1 ) THEN
            IF( (IRANGE.EQ.ALLRNG).OR.( (IRANGE.EQ.VALRNG).AND.
     $         ( D( IBEGIN ).GT.VL ).AND.( D( IBEGIN ).LE.VU ) )
     $        .OR. ( (IRANGE.EQ.INDRNG).AND.(IBLOCK(WBEGIN).EQ.JBLK))
     $        ) THEN
               M = M + 1
               W( M ) = D( IBEGIN )
               WERR(M) = ZERO
*              The gap for a single block doesn't matter for the later
*              algorithm and is assigned an arbitrary large value
               WGAP(M) = ZERO
               IBLOCK( M ) = JBLK
               INDEXW( M ) = 1
               WBEGIN = WBEGIN + 1
            ENDIF
*           E( IEND ) holds the shift for the initial RRR
            E( IEND ) = ZERO
            IBEGIN = IEND + 1
            GO TO 170
         END IF
*
*        Blocks of size larger than 1x1
*
*        E( IEND ) will hold the shift for the initial RRR, for now set it =0
         E( IEND ) = ZERO
*
*        Find local outer bounds GL,GU for the block
         GL = D(IBEGIN)
         GU = D(IBEGIN)
         DO 15 I = IBEGIN , IEND
            GL = MIN( GERS( 2*I-1 ), GL )
            GU = MAX( GERS( 2*I ), GU )
 15      CONTINUE
```

```
      SPDIAM = GU - GL

      IF(.NOT. ((IRANGE.EQ.ALLRNG).AND.(.NOT.FORCEB)) ) THEN
*        Count the number of eigenvalues in the current block.
         MB = 0
         DO 20 I = WBEGIN,MM
            IF( IBLOCK(I).EQ.JBLK ) THEN
               MB = MB+1
            ELSE
               GOTO 21
            ENDIF
 20      CONTINUE
 21      CONTINUE

         IF( MB.EQ.0) THEN
*           No eigenvalue in the current block lies in the desired range
*           E( IEND ) holds the shift for the initial RRR
            E( IEND ) = ZERO
            IBEGIN = IEND + 1
            GO TO 170
         ELSE

*           Decide whether dqds or bisection is more efficient
            USEDQD = ( (MB .GT. FAC*IN) .AND. (.NOT.FORCEB) )
            WEND = WBEGIN + MB - 1
*           Calculate gaps for the current block
*           In later stages, when representations for individual
*           eigenvalues are different, we use SIGMA = E( IEND ).
            SIGMA = ZERO
            DO 30 I = WBEGIN, WEND - 1
               WGAP( I ) = MAX( ZERO,
     $                     W(I+1)-WERR(I+1) - (W(I)+WERR(I)) )
 30         CONTINUE
            WGAP( WEND ) = MAX( ZERO,
     $                  VU - SIGMA - (W( WEND )+WERR( WEND )))
*           Find local index of the first and last desired evalue.
            INDL = INDEXW(WBEGIN)
            INDU = INDEXW( WEND )
         ENDIF
      ENDIF
      IF(( (IRANGE.EQ.ALLRNG) .AND. (.NOT. FORCEB) ).OR.USEDQD) THEN
*        Case of DQDS
*        Find approximations to the extremal eigenvalues of the block
         CALL DLARRK( IN, 1, GL, GU, D(IBEGIN),
     $             E2(IBEGIN), PIVMIN, RTL, TMP, TMP1, IINFO )
         IF( IINFO.NE.0 ) THEN
            INFO = -1
            RETURN
         ENDIF
         ISLEFT = MAX(GL, TMP - TMP1
```

```
     $              - HNDRD * EPS* ABS(TMP - TMP1))

            CALL DLARRK( IN, IN, GL, GU, D(IBEGIN),
     $              E2(IBEGIN), PIVMIN, RTL, TMP, TMP1, IINFO )
            IF( IINFO.NE.0 ) THEN
               INFO = -1
               RETURN
            ENDIF
            ISRGHT = MIN(GU, TMP + TMP1
     $                + HNDRD * EPS * ABS(TMP + TMP1))
*           Improve the estimate of the spectral diameter
            SPDIAM = ISRGHT - ISLEFT
         ELSE
*           Case of bisection
*           Find approximations to the wanted extremal eigenvalues
            ISLEFT = MAX(GL, W(WBEGIN) - WERR(WBEGIN)
     $                - HNDRD * EPS*ABS(W(WBEGIN)- WERR(WBEGIN) ))
            ISRGHT = MIN(GU,W(WEND) + WERR(WEND)
     $                + HNDRD * EPS * ABS(W(WEND)+ WERR(WEND)))
         ENDIF


*        Decide whether the base representation for the current block
*        L_JBLK D_JBLK L_JBLK^T = T_JBLK - sigma_JBLK I
*        should be on the left or the right end of the current block.
*        The strategy is to shift to the end which is "more populated"
*        Furthermore, decide whether to use DQDS for the computation of
*        the eigenvalue approximations at the end of DLARRE or bisection.
*        dqds is chosen if all eigenvalues are desired or the number of
*        eigenvalues to be computed is large compared to the blocksize.
         IF( ( IRANGE.EQ.ALLRNG ) .AND. (.NOT.FORCEB) ) THEN
*           If all the eigenvalues have to be computed, we use dqd
            USEDQD = .TRUE.
*           INDL is the local index of the first eigenvalue to compute
            INDL = 1
            INDU = IN
*           MB =  number of eigenvalues to compute
            MB = IN
            WEND = WBEGIN + MB - 1
*           Define 1/4 and 3/4 points of the spectrum
            S1 = ISLEFT + FOURTH * SPDIAM
            S2 = ISRGHT - FOURTH * SPDIAM
         ELSE
*           DLARRD has computed IBLOCK and INDEXW for each eigenvalue
*           approximation.
*           choose sigma
            IF( USEDQD ) THEN
               S1 = ISLEFT + FOURTH * SPDIAM
               S2 = ISRGHT - FOURTH * SPDIAM
            ELSE
```

```
            TMP = MIN(ISRGHT,VU) -  MAX(ISLEFT,VL)
            S1 =  MAX(ISLEFT,VL) + FOURTH * TMP
            S2 =  MIN(ISRGHT,VU) - FOURTH * TMP
         ENDIF
      ENDIF

*     Compute the negcount at the 1/4 and 3/4 points
      IF(MB.GT.1) THEN
         CALL DLARRC( 'T', IN, S1, S2, D(IBEGIN),
     $                   E(IBEGIN), PIVMIN, CNT, CNT1, CNT2, IINFO)
      ENDIF

      IF(MB.EQ.1) THEN
         SIGMA = GL
         SGNDEF = ONE
      ELSEIF( CNT1 - INDL .GE. INDU - CNT2 ) THEN
         IF( ( IRANGE.EQ.ALLRNG ) .AND. (.NOT.FORCEB) ) THEN
            SIGMA = MAX(ISLEFT,GL)
         ELSEIF( USEDQD ) THEN
*           use Gerschgorin bound as shift to get pos def matrix
*           for dqds
            SIGMA = ISLEFT
         ELSE
*           use approximation of the first desired eigenvalue of the
*           block as shift
            SIGMA = MAX(ISLEFT,VL)
         ENDIF
         SGNDEF = ONE
      ELSE
         IF( ( IRANGE.EQ.ALLRNG ) .AND. (.NOT.FORCEB) ) THEN
            SIGMA = MIN(ISRGHT,GU)
         ELSEIF( USEDQD ) THEN
*           use Gerschgorin bound as shift to get neg def matrix
*           for dqds
            SIGMA = ISRGHT
         ELSE
*           use approximation of the first desired eigenvalue of the
*           block as shift
            SIGMA = MIN(ISRGHT,VU)
         ENDIF
         SGNDEF = -ONE
      ENDIF


*     An initial SIGMA has been chosen that will be used for computing
*     T - SIGMA I = L D L^T
*     Define the increment TAU of the shift in case the initial shift
*     needs to be refined to obtain a factorization with not too much
*     element growth.
      IF( USEDQD ) THEN
```

```
*           The initial SIGMA was to the outer end of the spectrum
*           the matrix is definite and we need not retreat.
            TAU = SPDIAM*EPS*N + TWO*PIVMIN
            TAU = MAX( TAU,TWO*EPS*ABS(SIGMA) )
         ELSE
            IF(MB.GT.1) THEN
               CLWDTH = W(WEND) + WERR(WEND) - W(WBEGIN) - WERR(WBEGIN)
               AVGAP = ABS(CLWDTH / DBLE(WEND-WBEGIN))
               IF( SGNDEF.EQ.ONE ) THEN
                  TAU = HALF*MAX(WGAP(WBEGIN),AVGAP)
                  TAU = MAX(TAU,WERR(WBEGIN))
               ELSE
                  TAU = HALF*MAX(WGAP(WEND-1),AVGAP)
                  TAU = MAX(TAU,WERR(WEND))
               ENDIF
            ELSE
               TAU = WERR(WBEGIN)
            ENDIF
         ENDIF
*
         DO 80 IDUM = 1, MAXTRY
*           Compute L D L^T factorization of tridiagonal matrix T - sigma I.
*           Store D in WORK(1:IN), L in WORK(IN+1:2*IN), and reciprocals of
*           pivots in WORK(2*IN+1:3*IN)
            DPIVOT = D( IBEGIN ) - SIGMA
            WORK( 1 ) = DPIVOT
            DMAX = ABS( WORK(1) )
            J = IBEGIN
            DO 70 I = 1, IN - 1
               WORK( 2*IN+I ) = ONE / WORK( I )
               TMP = E( J )*WORK( 2*IN+I )
               WORK( IN+I ) = TMP
               DPIVOT = ( D( J+1 )-SIGMA ) - TMP*E( J )
               WORK( I+1 ) = DPIVOT
               DMAX = MAX( DMAX, ABS(DPIVOT) )
               J = J + 1
 70         CONTINUE
*           check for element growth
            IF( DMAX .GT. MAXGROWTH*SPDIAM ) THEN
               NOREP = .TRUE.
            ELSE
               NOREP = .FALSE.
            ENDIF
            IF( USEDQD .AND. .NOT.NOREP ) THEN
*              Ensure the definiteness of the representation
*              All entries of D (of L D L^T) must have the same sign
               DO 71 I = 1, IN
                  TMP = SGNDEF*WORK( I )
                  IF( TMP.LT.ZERO ) NOREP = .TRUE.
 71            CONTINUE
```

```
            ENDIF
            IF(NOREP) THEN
*              Note that in the case of IRANGE=ALLRNG, we use the Gerschgorin
*              shift which makes the matrix definite. So we should end up
*              here really only in the case of IRANGE = VALRNG or INDRNG.
               IF( IDUM.EQ.MAXTRY-1 ) THEN
                  IF( SGNDEF.EQ.ONE ) THEN
*                    The fudged Gerschgorin shift should succeed
                     SIGMA =
     $                    GL - FUDGE*SPDIAM*EPS*N - FUDGE*TWO*PIVMIN
                  ELSE
                     SIGMA =
     $                    GU + FUDGE*SPDIAM*EPS*N + FUDGE*TWO*PIVMIN
                  END IF
               ELSE
                  SIGMA = SIGMA - SGNDEF * TAU
                  TAU = TWO * TAU
               END IF
            ELSE
*              an initial RRR is found
               GO TO 83
            END IF
 80      CONTINUE
*        if the program reaches this point, no base representation could be
*        found in MAXTRY iterations.
         INFO = 2
         RETURN

 83      CONTINUE
*        At this point, we have found an initial base representation
*        T - SIGMA I = L D L^T with not too much element growth.
*        Store the shift.
         E( IEND ) = SIGMA
*        Store D and L.
         CALL DCOPY( IN, WORK, 1, D( IBEGIN ), 1 )
         CALL DCOPY( IN-1, WORK( IN+1 ), 1, E( IBEGIN ), 1 )


         IF(MB.GT.1 ) THEN
*
*           Perturb each entry of the base representation by a small
*           (but random) relative amount to overcome difficulties with
*           glued matrices.
*
            DO 122 I = 1, 4
               ISEED( I ) = 1
 122        CONTINUE

            CALL DLARNV(2, ISEED, 2*IN-1, WORK(1))
            DO 125 I = 1,IN-1
```

```
               D(IBEGIN+I-1) = D(IBEGIN+I-1)*(ONE+EPS*PERT*WORK(I))
               E(IBEGIN+I-1) = E(IBEGIN+I-1)*(ONE+EPS*PERT*WORK(IN+I))
 125        CONTINUE
            D(IEND) = D(IEND)*(ONE+EPS*FOUR*WORK(IN))
*
         ENDIF
*
*        Don't update the Gerschgorin intervals because keeping track
*        of the updates would be too much work in DLARRV.
*        We update W instead and use it to locate the proper Gerschgorin
*        intervals.

*        Compute the required eigenvalues of L D L' by bisection or dqds
         IF ( .NOT.USEDQD ) THEN
*           If DLARRD has been used, shift the eigenvalue approximations
*           according to their representation. This is necessary for
*           a uniform DLARRV since dqds computes eigenvalues of the
*           shifted representation. In DLARRV, W will always hold the
*           UNshifted eigenvalue approximation.
            DO 134 J=WBEGIN,WEND
               W(J) = W(J) - SIGMA
               WERR(J) = WERR(J) + ABS(W(J)) * EPS
 134        CONTINUE
*           call DLARRB to reduce eigenvalue error of the approximations
*           from DLARRD
            DO 135 I = IBEGIN, IEND-1
               WORK( I ) = D( I ) * E( I )**2
 135        CONTINUE
*           use bisection to find EV from INDL to INDU
            CALL DLARRB(IN, D(IBEGIN), WORK(IBEGIN),
     $                  INDL, INDU, RTOL1, RTOL2, INDL-1,
     $                  W(WBEGIN), WGAP(WBEGIN), WERR(WBEGIN),
     $                  WORK( 2*N+1 ), IWORK, PIVMIN, SPDIAM,
     $                  IN, IINFO )
            IF( IINFO .NE. 0 ) THEN
               INFO = -4
               RETURN
            END IF
*           DLARRB computes all gaps correctly except for the last one
*           Record distance to VU/GU
            WGAP( WEND ) = MAX( ZERO,
     $           ( VU-SIGMA ) - ( W( WEND ) + WERR( WEND ) ) )
            DO 138 I = INDL, INDU
               M = M + 1
               IBLOCK(M) = JBLK
               INDEXW(M) = I
 138        CONTINUE
         ELSE
*           Call dqds to get all eigs (and then possibly delete unwanted
*           eigenvalues).
```

```
*          Note that dqds finds the eigenvalues of the L D L^T representation
*          of T to high relative accuracy. High relative accuracy
*          might be lost when the shift of the RRR is subtracted to obtain
*          the eigenvalues of T. However, T is not guaranteed to define its
*          eigenvalues to high relative accuracy anyway.
*          Set RTOL to the order of the tolerance used in DLASQ2
*          This is an ESTIMATED error, the worst case bound is 4*N*EPS
*          which is usually too large and requires unnecessary work to be
*          done by bisection when computing the eigenvectors
           RTOL = LOG(DBLE(IN)) * FOUR * EPS
           J = IBEGIN
           DO 140 I = 1, IN - 1
              WORK( 2*I-1 ) = ABS( D( J ) )
              WORK( 2*I ) = E( J )*E( J )*WORK( 2*I-1 )
              J = J + 1
 140       CONTINUE
           WORK( 2*IN-1 ) = ABS( D( IEND ) )
           WORK( 2*IN ) = ZERO
           CALL DLASQ2( IN, WORK, IINFO )
           IF( IINFO .NE. 0 ) THEN
*             If IINFO = -5 then an index is part of a tight cluster
*             and should be changed. The index is in IWORK(1) and the
*             gap is in WORK(N+1)
              INFO = -5
              RETURN
           ELSE
*             Test that all eigenvalues are positive as expected
              DO 149 I = 1, IN
                 IF( WORK( I ).LT.ZERO ) THEN
                    INFO = -6
                    RETURN
                 ENDIF
 149          CONTINUE
           END IF
           IF( SGNDEF.GT.ZERO ) THEN
              DO 150 I = INDL, INDU
                 M = M + 1
                 W( M ) = WORK( IN-I+1 )
                 IBLOCK( M ) = JBLK
                 INDEXW( M ) = I
 150          CONTINUE
           ELSE
              DO 160 I = INDL, INDU
                 M = M + 1
                 W( M ) = -WORK( I )
                 IBLOCK( M ) = JBLK
                 INDEXW( M ) = I
 160          CONTINUE
           END IF
```

```
            DO 165 I = M - MB + 1, M
*              the value of RTOL below should be the tolerance in DLASQ2
               WERR( I ) = RTOL * ABS( W(I) )
 165        CONTINUE
            DO 166 I = M - MB + 1, M - 1
*              compute the right gap between the intervals
               WGAP( I ) = MAX( ZERO,
     $                          W(I+1)-WERR(I+1) - (W(I)+WERR(I)) )
 166        CONTINUE
            WGAP( M ) = MAX( ZERO,
     $           ( VU-SIGMA ) - ( W( M ) + WERR( M ) ) )
         END IF
*        proceed with next block
         IBEGIN = IEND + 1
         WBEGIN = WEND + 1
 170  CONTINUE
*

      RETURN
*
*     end of DLARRE
*
      END
```

———-

90.23.4 lisp code

— dlarre.lisp —

```
(let* ((zero 0.0d0)
       (one 1.0d0)
       (two 2.0d0)
       (four 4.0d0)
       (hndrd 100.0d0)
       (pert 8.0d0)
       (half (f2cl-lib:f2cl/ one two))
       (fourth$ (f2cl-lib:f2cl/ one four))
       (fac half)
       (maxgrowth 64.0d0)
       (fudge 2.0d0)
       (maxtry 6)
       (allrng 1)
       (indrng 2)
       (valrng 3))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
```

```
          (type (double-float 4.0d0 4.0d0) four)
          (type (double-float 100.0d0 100.0d0) hndrd)
          (type (double-float 8.0d0 8.0d0) pert)
          (type (double-float) half)
          (type (double-float) fourth$)
          (type (double-float) fac)
          (type (double-float 64.0d0 64.0d0) maxgrowth)
          (type (double-float 2.0d0 2.0d0) fudge)
          (type (f2cl-lib:integer4 6 6) maxtry)
          (type (f2cl-lib:integer4 1 1) allrng)
          (type (f2cl-lib:integer4 2 2) indrng)
          (type (f2cl-lib:integer4 3 3) valrng)
          (ignorable zero one two four hndrd pert half fourth$ fac maxgrowth
           fudge maxtry allrng indrng valrng))
(defun dlarre
       (range n vl vu il iu d e e2 rtol1 rtol2 spltol nsplit isplit m w werr
        wgap iblock indexw gers pivmin work iwork info)
  (declare (type (array f2cl-lib:integer4 (*)) iwork indexw iblock isplit)
           (type (array double-float (*)) work gers wgap werr w e2 e d)
           (type (double-float) pivmin spltol rtol2 rtol1 vu vl)
           (type (f2cl-lib:integer4) info m nsplit iu il n)
           (type (string *) range))
  (f2cl-lib:with-multi-array-data
      ((range character range-%data% range-%offset%)
       (d double-float d-%data% d-%offset%)
       (e double-float e-%data% e-%offset%)
       (e2 double-float e2-%data% e2-%offset%)
       (w double-float w-%data% w-%offset%)
       (werr double-float werr-%data% werr-%offset%)
       (wgap double-float wgap-%data% wgap-%offset%)
       (gers double-float gers-%data% gers-%offset%)
       (work double-float work-%data% work-%offset%)
       (isplit f2cl-lib:integer4 isplit-%data% isplit-%offset%)
       (iblock f2cl-lib:integer4 iblock-%data% iblock-%offset%)
       (indexw f2cl-lib:integer4 indexw-%data% indexw-%offset%)
       (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
    (prog ((iseed (make-array 4 :element-type 'f2cl-lib:integer4))
           (avgap 0.0d0) (bsrtol 0.0d0) (clwdth 0.0d0) (dmax 0.0d0)
           (dpivot 0.0d0) (eabs 0.0d0) (emax 0.0d0) (eold 0.0d0) (eps 0.0d0)
           (gl 0.0d0) (gu 0.0d0) (isleft 0.0d0) (isrght 0.0d0) (rtl 0.0d0)
           (rtol 0.0d0) (s1 0.0d0) (s2 0.0d0) (safmin 0.0d0) (sgndef 0.0d0)
           (sigma 0.0d0) (spdiam 0.0d0) (tau 0.0d0) (tmp 0.0d0) (tmp1 0.0d0)
           (cnt 0) (cnt1 0) (cnt2 0) (i 0) (ibegin 0) (idum 0) (iend 0)
           (iinfo 0) (in 0) (indl 0) (indu 0) (irange 0) (j 0) (jblk 0)
           (mb 0) (mm 0) (wbegin 0) (wend 0) (forceb nil) (norep nil)
           (usedqd nil))
      (declare (type (array f2cl-lib:integer4 (4)) iseed)
               (type (double-float) avgap bsrtol clwdth dmax dpivot eabs emax
                eold eps gl gu isleft isrght rtl rtol s1 s2 safmin sgndef
                sigma spdiam tau tmp tmp1)
```

```
          (type (f2cl-lib:integer4) cnt cnt1 cnt2 i ibegin idum iend
           iinfo in indl indu irange j jblk mb mm wbegin wend)
          (type f2cl-lib:logical forceb norep usedqd))
(setf info 0)
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame range "A")
    (declare (ignore var-1))
    (when var-0 (setf range var-0))
    ret-val)
  (setf irange allrng))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame range "V")
    (declare (ignore var-1))
    (when var-0 (setf range var-0))
    ret-val)
  (setf irange valrng))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame range "I")
    (declare (ignore var-1))
    (when var-0 (setf range var-0))
    ret-val)
  (setf irange indrng)))
(setf m 0)
(setf safmin (dlamch "S"))
(setf eps (dlamch "P"))
(setf rtl (f2cl-lib:fsqrt eps))
(setf bsrtol (f2cl-lib:fsqrt eps))
(cond
 ((= n 1)
  (cond
   ((or (= irange allrng)
        (and (= irange valrng) (> (f2cl-lib:fref d (1) ((1 *))) vl)
             (<= (f2cl-lib:fref d (1) ((1 *))) vu))
        (and (= irange indrng) (= il 1) (= iu 1)))
    (setf m 1)
    (setf (f2cl-lib:fref w-%data% (1) ((1 *)) w-%offset%)
            (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
    (setf (f2cl-lib:fref werr-%data% (1) ((1 *)) werr-%offset%) zero)
    (setf (f2cl-lib:fref wgap-%data% (1) ((1 *)) wgap-%offset%) zero)
    (setf (f2cl-lib:fref iblock-%data% (1) ((1 *)) iblock-%offset%) 1)
    (setf (f2cl-lib:fref indexw-%data% (1) ((1 *)) indexw-%offset%) 1)
    (setf (f2cl-lib:fref gers-%data% (1) ((1 *)) gers-%offset%)
            (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
    (setf (f2cl-lib:fref gers-%data% (2) ((1 *)) gers-%offset%)
            (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))))
  (setf (f2cl-lib:fref e-%data% (1) ((1 *)) e-%offset%) zero)
  (go end_label)))
(setf gl (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
(setf gu (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
```

```
(setf eold zero)
(setf emax zero)
(setf (f2cl-lib:fref e-%data% (n) ((1 *)) e-%offset%) zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref werr-%data% (i) ((1 *)) werr-%offset%) zero)
    (setf (f2cl-lib:fref wgap-%data% (i) ((1 *)) wgap-%offset%) zero)
    (setf eabs (abs (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
    (cond ((>= eabs emax) (setf emax eabs)))
    (setf tmp1 (+ eabs eold))
    (setf (f2cl-lib:fref gers-%data%
                         ((f2cl-lib:int-sub (f2cl-lib:int-mul 2 i) 1))
                         ((1 *)) gers-%offset%)
            (- (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) tmp1))
    (setf gl
            (min gl
                 (f2cl-lib:fref gers-%data%
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-mul 2 i) 1))
                                ((1 *)) gers-%offset%)))
    (setf (f2cl-lib:fref gers-%data% ((f2cl-lib:int-mul 2 i)) ((1 *))
                         gers-%offset%)
            (+ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) tmp1))
    (setf gu
            (max gu
                 (f2cl-lib:fref gers-%data% ((f2cl-lib:int-mul 2 i))
                                ((1 *)) gers-%offset%)))
    (setf eold eabs)
   label5))
(setf pivmin (* safmin (max one (expt emax 2))))
(setf spdiam (- gu gl))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dlarra n d e e2 spltol spdiam nsplit isplit iinfo)
  (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7))
  (setf nsplit var-6)
  (setf iinfo var-8))
(setf forceb f2cl-lib:%false%)
(setf usedqd (and (= irange allrng) (not forceb)))
(cond ((and (= irange allrng) (not forceb)) (setf vl gl) (setf vu gu))
      (t
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17
            var-18 var-19 var-20 var-21 var-22 var-23 var-24)
           (dlarrd range "B" n vl vu il iu gers bsrtol d e e2 pivmin
            nsplit isplit mm w werr vl vu iblock indexw work iwork
            iinfo)
         (declare
```

```
            (ignore var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11 var-12 var-13 var-14 var-16 var-17 var-20
             var-21 var-22 var-23))
           (setf range var-0)
           (setf n var-2)
           (setf mm var-15)
           (setf vl var-18)
           (setf vu var-19)
           (setf iinfo var-24))
         (cond ((/= iinfo 0) (setf info -1) (go end_label)))
         (f2cl-lib:fdo (i (f2cl-lib:int-add mm 1) (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf (f2cl-lib:fref w-%data% (i) ((1 *)) w-%offset%) zero)
             (setf (f2cl-lib:fref werr-%data% (i) ((1 *)) werr-%offset%)
                     zero)
             (setf (f2cl-lib:fref iblock-%data% (i) ((1 *))
                                  iblock-%offset%)
                     0)
             (setf (f2cl-lib:fref indexw-%data% (i) ((1 *))
                                  indexw-%offset%)
                     0)
            label14))))
(setf ibegin 1)
(setf wbegin 1)
(f2cl-lib:fdo (jblk 1 (f2cl-lib:int-add jblk 1))
              ((> jblk nsplit) nil)
  (tagbody
    (setf iend
            (f2cl-lib:fref isplit-%data% (jblk) ((1 *))
                           isplit-%offset%))
    (setf in (f2cl-lib:int-add (f2cl-lib:int-sub iend ibegin) 1))
    (cond
     ((= in 1)
      (cond
       ((or (= irange allrng)
            (and (= irange valrng)
                 (> (f2cl-lib:fref d (ibegin) ((1 *))) vl)
                 (<= (f2cl-lib:fref d (ibegin) ((1 *))) vu))
            (and (= irange indrng)
                 (= (f2cl-lib:fref iblock (wbegin) ((1 *))) jblk)))
        (setf m (f2cl-lib:int-add m 1))
        (setf (f2cl-lib:fref w-%data% (m) ((1 *)) w-%offset%)
                (f2cl-lib:fref d-%data% (ibegin) ((1 *)) d-%offset%))
        (setf (f2cl-lib:fref werr-%data% (m) ((1 *)) werr-%offset%)
                zero)
        (setf (f2cl-lib:fref wgap-%data% (m) ((1 *)) wgap-%offset%)
                zero)
        (setf (f2cl-lib:fref iblock-%data% (m) ((1 *)) iblock-%offset%)
                jblk)
```

```
    (setf (f2cl-lib:fref indexw-%data% (m) ((1 *)) indexw-%offset%)
            1)
    (setf wbegin (f2cl-lib:int-add wbegin 1))))
  (setf (f2cl-lib:fref e-%data% (iend) ((1 *)) e-%offset%) zero)
  (setf ibegin (f2cl-lib:int-add iend 1)) (go label170)))
(setf (f2cl-lib:fref e-%data% (iend) ((1 *)) e-%offset%) zero)
(setf gl (f2cl-lib:fref d-%data% (ibegin) ((1 *)) d-%offset%))
(setf gu (f2cl-lib:fref d-%data% (ibegin) ((1 *)) d-%offset%))
(f2cl-lib:fdo (i ibegin (f2cl-lib:int-add i 1))
              ((> i iend) nil)
  (tagbody
    (setf gl
            (min
             (f2cl-lib:fref gers-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-mul 2 i) 1))
                            ((1 *)) gers-%offset%)
             gl))
    (setf gu
            (max
             (f2cl-lib:fref gers-%data% ((f2cl-lib:int-mul 2 i))
                            ((1 *)) gers-%offset%)
             gu))
   label15))
(setf spdiam (- gu gl))
(cond
 ((not (and (= irange allrng) (not forceb)))
  (tagbody
    (setf mb 0)
    (f2cl-lib:fdo (i wbegin (f2cl-lib:int-add i 1))
                  ((> i mm) nil)
      (tagbody
        (cond
         ((= (f2cl-lib:fref iblock (i) ((1 *))) jblk)
          (setf mb (f2cl-lib:int-add mb 1)))
         (t (go label21)))
       label20))
   label21
    (cond
     ((= mb 0)
      (setf (f2cl-lib:fref e-%data% (iend) ((1 *)) e-%offset%)
              zero)
      (setf ibegin (f2cl-lib:int-add iend 1)) (go label170))
     (t (setf usedqd (and (> mb (* fac in)) (not forceb)))
      (setf wend (f2cl-lib:int-sub (f2cl-lib:int-add wbegin mb) 1))
      (setf sigma zero)
      (f2cl-lib:fdo (i wbegin (f2cl-lib:int-add i 1))
                    ((> i
                        (f2cl-lib:int-add wend
                                          (f2cl-lib:int-sub 1)))
```

```
                    nil)
          (tagbody
            (setf (f2cl-lib:fref wgap-%data% (i) ((1 *))
                                 wgap-%offset%)
                  (max zero
                       (-
                        (f2cl-lib:fref w-%data%
                                       ((f2cl-lib:int-add i 1))
                                       ((1 *)) w-%offset%)
                        (f2cl-lib:fref werr-%data%
                                       ((f2cl-lib:int-add i 1))
                                       ((1 *)) werr-%offset%)
                        (+
                         (f2cl-lib:fref w-%data% (i) ((1 *))
                                        w-%offset%)
                         (f2cl-lib:fref werr-%data% (i) ((1 *))
                                        werr-%offset%)))))
           label30))
        (setf (f2cl-lib:fref wgap-%data% (wend) ((1 *))
                             wgap-%offset%)
                (max zero
                     (- vu sigma
                        (+
                         (f2cl-lib:fref w-%data% (wend) ((1 *))
                                        w-%offset%)
                         (f2cl-lib:fref werr-%data% (wend) ((1 *))
                                        werr-%offset%)))))
        (setf indl
                (f2cl-lib:fref indexw-%data% (wbegin) ((1 *))
                               indexw-%offset%))
        (setf indu
                (f2cl-lib:fref indexw-%data% (wend) ((1 *))
                               indexw-%offset%)))))))
(cond
 ((or (and (= irange allrng) (not forceb)) usedqd)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dlarrk in 1 gl gu
       (f2cl-lib:array-slice d-%data% double-float (ibegin) ((1 *))
                             d-%offset%)
       (f2cl-lib:array-slice e2-%data% double-float (ibegin)
                             ((1 *)) e2-%offset%)
       pivmin rtl tmp tmp1 iinfo)
    (declare (ignore var-1 var-4 var-5))
    (when var-0 (setf in var-0))
    (when var-2 (setf gl var-2))
    (when var-3 (setf gu var-3))
    (when var-6 (setf pivmin var-6))
    (when var-7 (setf rtl var-7))
```

```
  (when var-8 (setf tmp var-8))
  (when var-9 (setf tmp1 var-9))
  (when var-10 (setf iinfo var-10)))
 (cond ((/= iinfo 0) (setf info -1) (go end_label)))
 (setf isleft
         (max gl
              (+ (- tmp tmp1)
                 (* (- hndrd) eps (abs (- tmp tmp1))))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (dlarrk in in gl gu
      (f2cl-lib:array-slice d-%data% double-float (ibegin) ((1 *))
                            d-%offset%)
      (f2cl-lib:array-slice e2-%data% double-float (ibegin)
                            ((1 *)) e2-%offset%)
      pivmin rtl tmp tmp1 iinfo)
  (declare (ignore var-4 var-5))
  (when var-0 (setf in var-0))
  (when var-1 (setf in var-1))
  (when var-2 (setf gl var-2))
  (when var-3 (setf gu var-3))
  (when var-6 (setf pivmin var-6))
  (when var-7 (setf rtl var-7))
  (when var-8 (setf tmp var-8))
  (when var-9 (setf tmp1 var-9))
  (when var-10 (setf iinfo var-10)))
 (cond ((/= iinfo 0) (setf info -1) (go end_label)))
 (setf isrght
         (min gu (+ tmp tmp1 (* hndrd eps (abs (+ tmp tmp1))))))
 (setf spdiam (- isrght isleft)))
(t
 (setf isleft
         (max gl
              (+
               (-
                (f2cl-lib:fref w-%data% (wbegin) ((1 *))
                               w-%offset%)
                (f2cl-lib:fref werr-%data% (wbegin) ((1 *))
                               werr-%offset%))
               (* (- hndrd) eps
                  (abs
                   (-
                    (f2cl-lib:fref w-%data% (wbegin) ((1 *))
                                   w-%offset%)
                    (f2cl-lib:fref werr-%data% (wbegin) ((1 *))
                                   werr-%offset%)))))))
 (setf isrght
         (min gu
              (+
```

```
                    (f2cl-lib:fref w-%data% (wend) ((1 *)) w-%offset%)
                    (f2cl-lib:fref werr-%data% (wend) ((1 *))
                                   werr-%offset%)
                    (* hndrd eps
                       (abs
                        (+
                         (f2cl-lib:fref w-%data% (wend) ((1 *))
                                        w-%offset%)
                         (f2cl-lib:fref werr-%data% (wend) ((1 *))
                                        werr-%offset%)))))))))
(cond
 ((and (= irange allrng) (not forceb))
  (setf usedqd f2cl-lib:%true%) (setf indl 1) (setf indu in)
  (setf mb in)
  (setf wend (f2cl-lib:int-sub (f2cl-lib:int-add wbegin mb) 1))
  (setf s1 (+ isleft (* fourth$ spdiam)))
  (setf s2 (- isrght (* fourth$ spdiam))))
 (t
  (cond
   (usedqd (setf s1 (+ isleft (* fourth$ spdiam)))
    (setf s2 (- isrght (* fourth$ spdiam))))
   (t (setf tmp (- (min isrght vu) (max isleft vl)))
    (setf s1 (+ (max isleft vl) (* fourth$ tmp)))
    (setf s2 (- (min isrght vu) (* fourth$ tmp)))))))
(cond
 ((> mb 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dlarrc "T" in s1 s2
       (f2cl-lib:array-slice d-%data% double-float (ibegin) ((1 *))
                             d-%offset%)
       (f2cl-lib:array-slice e-%data% double-float (ibegin) ((1 *))
                             e-%offset%)
       pivmin cnt cnt1 cnt2 iinfo)
    (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6))
    (setf cnt var-7)
    (setf cnt1 var-8)
    (setf cnt2 var-9)
    (setf iinfo var-10))))
(cond ((= mb 1) (setf sigma gl) (setf sgndef one))
      ((>= (f2cl-lib:int-add cnt1 (f2cl-lib:int-sub indl))
           (f2cl-lib:int-add indu (f2cl-lib:int-sub cnt2)))
       (cond
        ((and (= irange allrng) (not forceb))
         (setf sigma (max isleft gl)))
        (usedqd (setf sigma isleft))
        (t (setf sigma (max isleft vl))))
       (setf sgndef one))
      (t
```

```
        (cond
         ((and (= irange allrng) (not forceb))
          (setf sigma (min isrght gu)))
         (usedqd (setf sigma isrght))
         (t (setf sigma (min isrght vu))))
        (setf sgndef (- one))))
(cond
 (usedqd (setf tau (+ (* spdiam eps n) (* two pivmin)))
  (setf tau (max tau (* two eps (abs sigma)))))
 (t
  (cond
   ((> mb 1)
    (setf clwdth
            (-
             (+ (f2cl-lib:fref w-%data% (wend) ((1 *)) w-%offset%)
                (f2cl-lib:fref werr-%data% (wend) ((1 *))
                               werr-%offset%))
             (f2cl-lib:fref w-%data% (wbegin) ((1 *)) w-%offset%)
             (f2cl-lib:fref werr-%data% (wbegin) ((1 *))
                            werr-%offset%)))
    (setf avgap
            (abs
             (/ clwdth
                (f2cl-lib:dble (f2cl-lib:int-sub wend wbegin)))))
    (cond
     ((= sgndef one)
      (setf tau
              (* half
                 (max
                  (f2cl-lib:fref wgap-%data% (wbegin) ((1 *))
                                 wgap-%offset%)
                  avgap)))
      (setf tau
              (max tau
                   (f2cl-lib:fref werr-%data% (wbegin) ((1 *))
                                  werr-%offset%))))
     (t
      (setf tau
              (* half
                 (max
                  (f2cl-lib:fref wgap-%data%
                                 ((f2cl-lib:int-sub wend 1))
                                 ((1 *)) wgap-%offset%)
                  avgap)))
      (setf tau
              (max tau
                   (f2cl-lib:fref werr-%data% (wend) ((1 *))
                                  werr-%offset%))))))
   (t
    (setf tau
```

```
            (f2cl-lib:fref werr-%data% (wbegin) ((1 *))
                           werr-%offset%))))))
(f2cl-lib:fdo (idum 1 (f2cl-lib:int-add idum 1))
              ((> idum maxtry) nil)
  (tagbody
    (setf dpivot
            (- (f2cl-lib:fref d-%data% (ibegin) ((1 *)) d-%offset%)
               sigma))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            dpivot)
    (setf dmax
            (abs
             (f2cl-lib:fref work-%data% (1) ((1 *))
                            work-%offset%)))
    (setf j ibegin)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add in (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-mul 2 in) i))
                             ((1 *)) work-%offset%)
                (/ one
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)))
        (setf tmp
                (* (f2cl-lib:fref e-%data% (j) ((1 *)) e-%offset%)
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add
                                    (f2cl-lib:int-mul 2 in) i))
                                  ((1 *)) work-%offset%)))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add in i))
                             ((1 *)) work-%offset%)
                tmp)
        (setf dpivot
                (-
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-add j 1))
                                ((1 *)) d-%offset%)
                 sigma
                 (* tmp
                    (f2cl-lib:fref e-%data% (j) ((1 *))
                                   e-%offset%))))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1))
                             ((1 *)) work-%offset%)
                dpivot)
        (setf dmax (max dmax (abs dpivot)))
        (setf j (f2cl-lib:int-add j 1))
       label70))
    (cond
```

```
        ((> dmax (* maxgrowth spdiam)) (setf norep f2cl-lib:%true%))
        (t (setf norep f2cl-lib:%false%)))
       (cond
        ((and usedqd (not norep))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i in) nil)
           (tagbody
             (setf tmp
                     (* sgndef
                        (f2cl-lib:fref work-%data% (i) ((1 *))
                                       work-%offset%)))
             (if (< tmp zero)
                 (setf norep f2cl-lib:%true%))
            label71))))
       (cond
        (norep
         (cond
          ((= idum (f2cl-lib:int-add maxtry (f2cl-lib:int-sub 1)))
           (cond
            ((= sgndef one)
             (setf sigma
                     (+ gl (* (- fudge) spdiam eps n)
                        (* (- fudge) two pivmin))))
            (t
             (setf sigma
                     (+ gu (* fudge spdiam eps n)
                        (* fudge two pivmin))))))
          (t (setf sigma (- sigma (* sgndef tau)))
           (setf tau (* two tau)))))
        (t (go label83)))
      label80))
    (setf info 2)
    (go end_label)
   label83
    (setf (f2cl-lib:fref e-%data% (iend) ((1 *)) e-%offset%) sigma)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy in work 1
         (f2cl-lib:array-slice d-%data% double-float (ibegin) ((1 *))
                               d-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf in var-0)))
    (dcopy (f2cl-lib:int-sub in 1)
     (f2cl-lib:array-slice work-%data% double-float ((+ in 1)) ((1 *))
                           work-%offset%)
     1
     (f2cl-lib:array-slice e-%data% double-float (ibegin) ((1 *))
                           e-%offset%)
     1)
    (cond
```

```
((> mb 1)
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i 4) nil)
   (tagbody (setf (f2cl-lib:fref iseed (i) ((1 4))) 1) label122))
 (dlarnv 2 iseed (f2cl-lib:int-sub (f2cl-lib:int-mul 2 in) 1)
  (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                        work-%offset%))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i (f2cl-lib:int-add in (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (setf (f2cl-lib:fref d-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add ibegin i) 1))
                          ((1 *)) d-%offset%)
           (*
            (f2cl-lib:fref d-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add ibegin i) 1))
                           ((1 *)) d-%offset%)
            (+ one
               (* eps pert
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)))))
     (setf (f2cl-lib:fref e-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add ibegin i) 1))
                          ((1 *)) e-%offset%)
           (*
            (f2cl-lib:fref e-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add ibegin i) 1))
                           ((1 *)) e-%offset%)
            (+ one
               (* eps pert
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add in i))
                                 ((1 *)) work-%offset%)))))
    label125))
 (setf (f2cl-lib:fref d-%data% (iend) ((1 *)) d-%offset%)
       (* (f2cl-lib:fref d-%data% (iend) ((1 *)) d-%offset%)
          (+ one
             (* eps four
                (f2cl-lib:fref work-%data% (in) ((1 *))
                               work-%offset%)))))))
(cond
 ((not usedqd)
  (f2cl-lib:fdo (j wbegin (f2cl-lib:int-add j 1))
                ((> j wend) nil)
    (tagbody
```

```
    (setf (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%)
            (- (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%)
               sigma))
    (setf (f2cl-lib:fref werr-%data% (j) ((1 *)) werr-%offset%)
            (+
             (f2cl-lib:fref werr-%data% (j) ((1 *))
                            werr-%offset%)
             (*
              (abs
               (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%))
              eps)))
   label134))
(f2cl-lib:fdo (i ibegin (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add iend (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (* (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
               (expt
                (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                2)))
   label135))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (dlarrb in
     (f2cl-lib:array-slice d-%data% double-float (ibegin) ((1 *))
                           d-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (ibegin)
                           ((1 *)) work-%offset%)
     indl indu rtol1 rtol2 (f2cl-lib:int-sub indl 1)
     (f2cl-lib:array-slice w-%data% double-float (wbegin) ((1 *))
                           w-%offset%)
     (f2cl-lib:array-slice wgap-%data% double-float (wbegin)
                           ((1 *)) wgap-%offset%)
     (f2cl-lib:array-slice werr-%data% double-float (wbegin)
                           ((1 *)) werr-%offset%)
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                           work-%offset%)
     iwork pivmin spdiam in iinfo)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
    var-9 var-10 var-11 var-12 var-13 var-14 var-15))
  (setf iinfo var-16))
(cond ((/= iinfo 0) (setf info -4) (go end_label)))
(setf (f2cl-lib:fref wgap-%data% (wend) ((1 *)) wgap-%offset%)
        (max zero
             (- vu sigma
                (+
```

```
                    (f2cl-lib:fref w-%data% (wend) ((1 *))
                                   w-%offset%)
                    (f2cl-lib:fref werr-%data% (wend) ((1 *))
                                   werr-%offset%)))))
  (f2cl-lib:fdo (i indl (f2cl-lib:int-add i 1))
                ((> i indu) nil)
    (tagbody
      (setf m (f2cl-lib:int-add m 1))
      (setf (f2cl-lib:fref iblock-%data% (m) ((1 *))
                           iblock-%offset%)
            jblk)
      (setf (f2cl-lib:fref indexw-%data% (m) ((1 *))
                           indexw-%offset%)
            i)
     label138)))
 (t (setf rtol (* (f2cl-lib:flog (f2cl-lib:dble in)) four eps))
  (setf j ibegin)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i (f2cl-lib:int-add in (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-mul 2 i) 1))
                           ((1 *)) work-%offset%)
            (abs
             (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-mul 2 i))
                           ((1 *)) work-%offset%)
            (* (f2cl-lib:fref e-%data% (j) ((1 *)) e-%offset%)
               (f2cl-lib:fref e-%data% (j) ((1 *)) e-%offset%)
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-mul 2 i) 1))
                              ((1 *)) work-%offset%)))
      (setf j (f2cl-lib:int-add j 1))
     label140))
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-sub (f2cl-lib:int-mul 2 in)
                                          1))
                       ((1 *)) work-%offset%)
        (abs (f2cl-lib:fref d-%data% (iend) ((1 *)) d-%offset%)))
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-mul 2 in))
                       ((1 *)) work-%offset%)
        zero)
  (multiple-value-bind (var-0 var-1 var-2)
      (dlasq2 in work iinfo)
    (declare (ignore var-1))
    (when var-0 (setf in var-0))
    (when var-2 (setf iinfo var-2)))
```

```
(cond ((/= iinfo 0) (setf info -5) (go end_label))
      (t
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i in) nil)
         (tagbody
           (cond
            ((< (f2cl-lib:fref work (i) ((1 *))) zero)
             (setf info -6) (go end_label)))
          label149))))
(cond
 ((> sgndef zero)
  (f2cl-lib:fdo (i indl (f2cl-lib:int-add i 1))
                ((> i indu) nil)
    (tagbody
      (setf m (f2cl-lib:int-add m 1))
      (setf (f2cl-lib:fref w-%data% (m) ((1 *)) w-%offset%)
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-sub in i) 1))
                             ((1 *)) work-%offset%))
      (setf (f2cl-lib:fref iblock-%data% (m) ((1 *))
                           iblock-%offset%)
              jblk)
      (setf (f2cl-lib:fref indexw-%data% (m) ((1 *))
                           indexw-%offset%)
              i)
     label150)))
 (t
  (f2cl-lib:fdo (i indl (f2cl-lib:int-add i 1))
                ((> i indu) nil)
    (tagbody
      (setf m (f2cl-lib:int-add m 1))
      (setf (f2cl-lib:fref w-%data% (m) ((1 *)) w-%offset%)
              (-
               (f2cl-lib:fref work-%data% (i) ((1 *))
                              work-%offset%)))
      (setf (f2cl-lib:fref iblock-%data% (m) ((1 *))
                           iblock-%offset%)
              jblk)
      (setf (f2cl-lib:fref indexw-%data% (m) ((1 *))
                           indexw-%offset%)
              i)
     label160))))
(f2cl-lib:fdo (i (f2cl-lib:int-add m (f2cl-lib:int-sub mb) 1)
               (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf (f2cl-lib:fref werr-%data% (i) ((1 *)) werr-%offset%)
            (* rtol
               (abs
```

```
                              (f2cl-lib:fref w-%data% (i) ((1 *))
                                             w-%offset%))))
                 label165))
               (f2cl-lib:fdo (i (f2cl-lib:int-add m (f2cl-lib:int-sub mb) 1)
                              (f2cl-lib:int-add i 1))
                             ((> i (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
                              nil)
                 (tagbody
                   (setf (f2cl-lib:fref wgap-%data% (i) ((1 *)) wgap-%offset%)
                           (max zero
                                (-
                                 (f2cl-lib:fref w-%data%
                                                ((f2cl-lib:int-add i 1)) ((1 *))
                                                w-%offset%)
                                 (f2cl-lib:fref werr-%data%
                                                ((f2cl-lib:int-add i 1)) ((1 *))
                                                werr-%offset%)
                                 (+
                                  (f2cl-lib:fref w-%data% (i) ((1 *))
                                                 w-%offset%)
                                  (f2cl-lib:fref werr-%data% (i) ((1 *))
                                                 werr-%offset%)))))
                 label166))
               (setf (f2cl-lib:fref wgap-%data% (m) ((1 *)) wgap-%offset%)
                       (max zero
                            (- vu sigma
                               (+
                                (f2cl-lib:fref w-%data% (m) ((1 *)) w-%offset%)
                                (f2cl-lib:fref werr-%data% (m) ((1 *))
                                               werr-%offset%)))))))
             (setf ibegin (f2cl-lib:int-add iend 1))
             (setf wbegin (f2cl-lib:int-add wend 1))
            label170))
         (go end_label)
        end_label
         (return
          (values range n vl vu nil nil nil nil nil nil nil nil nsplit nil m nil
                  nil nil nil nil nil pivmin nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlarre
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
```

```
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (double-float) (double-float)
  (double-float)
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (array fortran-to-lisp::integer4
   (*))
  (array double-float (*))
  (double-float)
  (array double-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::range
  fortran-to-lisp::n
  fortran-to-lisp::vl
  fortran-to-lisp::vu nil nil nil
  nil nil nil nil nil
  fortran-to-lisp::nsplit nil
  fortran-to-lisp::m nil nil nil
  nil nil nil
  fortran-to-lisp::pivmin nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::dlarrb
  fortran-to-lisp::dlarnv
  fortran-to-lisp::dlarrc
  fortran-to-lisp::dlarrd
  fortran-to-lisp::dlarra))))
```

90.24 dlarrf

90.24.1 Axiom unit tests

— dlarrf.input —

```
)set break resume
)sys rm -f dlarrf.output
)spool dlarrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.24.2 Axiom help page

— dlarrf.help —

```
======================================================================
dlarrf examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlarrf (N, D, L, LD, CLSTRT, CLEND, W, WGAP, WERR, SPDIAM,
           CLGAPL, CLGAPR, PIVMIN, SIGMA, DPLUS, LPLUS, WORK, INFO)
           DLARRF finds a new relatively robust representation such that at
           least one of the eigenvalues is relatively isolated.

Function/Subroutine Documentation
   subroutine dlarrf (integer N, double precision, dimension( * ) D, double
       precision, dimension( * ) L, double precision, dimension( * ) LD,
       integer CLSTRT, integer CLEND, double precision, dimension( * ) W,
       double precision, dimension( * ) WGAP, double precision, dimension( * )
       WERR, double precision SPDIAM, double precision CLGAPL, double
       precision CLGAPR, double precision PIVMIN, double precision SIGMA,
       double precision, dimension( * ) DPLUS, double precision, dimension( *
       ) LPLUS, double precision, dimension( * ) WORK, integer INFO)
```

```
      DLARRF finds a new relatively robust representation such that at least
      one of the eigenvalues is relatively isolated.

Purpose:
   Given the initial representation L D L^T and its cluster of close
   eigenvalues (in a relative measure), W( CLSTRT ), W( CLSTRT+1 ), ...
   W( CLEND ), DLARRF finds a new relatively robust representation
   L D L^T - SIGMA I = L(+) D(+) L(+)^T such that at least one of the
   eigenvalues of L(+) D(+) L(+)^T is relatively isolated.

Parameters:

 N
   N is INTEGER
   The order of the matrix (subblock, if the matrix splitted).

 D
   D is DOUBLE PRECISION array, dimension (N)
   The N diagonal elements of the diagonal matrix D.

 L
   L is DOUBLE PRECISION array, dimension (N-1)
   The (N-1) subdiagonal elements of the unit bidiagonal matrix L.

 LD
   LD is DOUBLE PRECISION array, dimension (N-1)
   The (N-1) elements L(i)*D(i).

 CLSTRT
   CLSTRT is INTEGER
   The index of the first eigenvalue in the cluster.

 CLEND
   CLEND is INTEGER
   The index of the last eigenvalue in the cluster.

 W
   W is DOUBLE PRECISION array, dimension
   dimension is >=  (CLEND-CLSTRT+1)
   The eigenvalue APPROXIMATIONS of L D L^T in ascending order.
   W( CLSTRT ) through W( CLEND ) form the cluster of relatively
   close eigenalues.

 WGAP
   WGAP is DOUBLE PRECISION array, dimension
   dimension is >=  (CLEND-CLSTRT+1)
   The separation from the right neighbor eigenvalue in W.

 WERR
   WERR is DOUBLE PRECISION array, dimension
```

```
  dimension is  >=  (CLEND-CLSTRT+1)
  WERR contain the semiwidth of the uncertainty
  interval of the corresponding eigenvalue APPROXIMATION in W

SPDIAM
  SPDIAM is DOUBLE PRECISION
  estimate of the spectral diameter obtained from the
  Gerschgorin intervals

CLGAPL
  CLGAPL is DOUBLE PRECISION

CLGAPR
  CLGAPR is DOUBLE PRECISION
  absolute gap on each end of the cluster.
  Set by the calling routine to protect against shifts too close
  to eigenvalues outside the cluster.

PIVMIN
  PIVMIN is DOUBLE PRECISION
  The minimum pivot allowed in the Sturm sequence.

SIGMA
  SIGMA is DOUBLE PRECISION
  The shift used to form L(+) D(+) L(+)^T.

DPLUS
  DPLUS is DOUBLE PRECISION array, dimension (N)
  The N diagonal elements of the diagonal matrix D(+).

LPLUS
  LPLUS is DOUBLE PRECISION array, dimension (N-1)
  The first (N-1) elements of LPLUS contain the subdiagonal
  elements of the unit bidiagonal matrix L(+).

WORK
  WORK is DOUBLE PRECISION array, dimension (2*N)
  Workspace.

INFO
  INFO is INTEGER
  Signals processing OK (=0) or failure (=1)
```

———-

90.24.3 fortran code

— dlarrf.f —

```
*  =====================================================================
      SUBROUTINE DLARRF( N, D, L, LD, CLSTRT, CLEND,
     $                   W, WGAP, WERR,
     $                   SPDIAM, CLGAPL, CLGAPR, PIVMIN, SIGMA,
     $                   DPLUS, LPLUS, WORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            CLSTRT, CLEND, INFO, N
      DOUBLE PRECISION   CLGAPL, CLGAPR, PIVMIN, SIGMA, SPDIAM
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), DPLUS( * ), L( * ), LD( * ),
     $          LPLUS( * ), W( * ), WGAP( * ), WERR( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   FOUR, MAXGROWTH1, MAXGROWTH2, ONE, QUART, TWO
      PARAMETER          ( ONE = 1.0D0, TWO = 2.0D0, FOUR = 4.0D0,
     $                     QUART = 0.25D0,
     $                     MAXGROWTH1 = 8.D0,
     $                     MAXGROWTH2 = 8.D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL   DORRR1, FORCER, NOFAIL, SAWNAN1, SAWNAN2, TRYRRR1
      INTEGER            I, INDX, KTRY, KTRYMAX, SLEFT, SRIGHT, SHIFT
      PARAMETER          ( KTRYMAX = 1, SLEFT = 1, SRIGHT = 2 )
      DOUBLE PRECISION   AVGAP, BESTSHIFT, CLWDTH, EPS, FACT, FAIL,
     $                   FAIL2, GROWTHBOUND, LDELTA, LDMAX, LSIGMA,
     $                   MAX1, MAX2, MINGAP, OLDP, PROD, RDELTA, RDMAX,
     $                   RRR1, RRR2, RSIGMA, S, SMLGROWTH, TMP, ZNM2
*     ..
*     .. External Functions ..
      LOGICAL DISNAN
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DISNAN, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY
```

```
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      FACT = DBLE(2**KTRYMAX)
      EPS = DLAMCH( 'Precision' )
      SHIFT = 0
      FORCER = .FALSE.


*     Note that we cannot guarantee that for any of the shifts tried,
*     the factorization has a small or even moderate element growth.
*     There could be Ritz values at both ends of the cluster and despite
*     backing off, there are examples where all factorizations tried
*     (in IEEE mode, allowing zero pivots & infinities) have INFINITE
*     element growth.
*     For this reason, we should use PIVMIN in this subroutine so that at
*     least the L D L^T factorization exists. It can be checked afterwards
*     whether the element growth caused bad residuals/orthogonality.

*     Decide whether the code should accept the best among all
*     representations despite large element growth or signal INFO=1
*     Setting NOFAIL to .FALSE. for quick fix for bug 113
      NOFAIL = .FALSE.
*

*     Compute the average gap length of the cluster
      CLWDTH = ABS(W(CLEND)-W(CLSTRT)) + WERR(CLEND) + WERR(CLSTRT)
      AVGAP = CLWDTH / DBLE(CLEND-CLSTRT)
      MINGAP = MIN(CLGAPL, CLGAPR)
*     Initial values for shifts to both ends of cluster
      LSIGMA = MIN(W( CLSTRT ),W( CLEND )) - WERR( CLSTRT )
      RSIGMA = MAX(W( CLSTRT ),W( CLEND )) + WERR( CLEND )

*     Use a small fudge to make sure that we really shift to the outside
      LSIGMA = LSIGMA - ABS(LSIGMA)* FOUR * EPS
      RSIGMA = RSIGMA + ABS(RSIGMA)* FOUR * EPS

*     Compute upper bounds for how much to back off the initial shifts
      LDMAX = QUART * MINGAP + TWO * PIVMIN
      RDMAX = QUART * MINGAP + TWO * PIVMIN

      LDELTA = MAX(AVGAP,WGAP( CLSTRT ))/FACT
      RDELTA = MAX(AVGAP,WGAP( CLEND-1 ))/FACT
*
*     Initialize the record of the best representation found
*
```

```
      S = DLAMCH( 'S' )
      SMLGROWTH = ONE / S
      FAIL = DBLE(N-1)*MINGAP/(SPDIAM*EPS)
      FAIL2 = DBLE(N-1)*MINGAP/(SPDIAM*SQRT(EPS))
      BESTSHIFT = LSIGMA
*
*     while (KTRY <= KTRYMAX)
      KTRY = 0
      GROWTHBOUND = MAXGROWTH1*SPDIAM

 5    CONTINUE
      SAWNAN1 = .FALSE.
      SAWNAN2 = .FALSE.
*     Ensure that we do not back off too much of the initial shifts
      LDELTA = MIN(LDMAX,LDELTA)
      RDELTA = MIN(RDMAX,RDELTA)

*     Compute the element growth when shifting to both ends of the cluster
*     accept the shift if there is no element growth at one of the two ends

*     Left end
      S = -LSIGMA
      DPLUS( 1 ) = D( 1 ) + S
      IF(ABS(DPLUS(1)).LT.PIVMIN) THEN
         DPLUS(1) = -PIVMIN
*        Need to set SAWNAN1 because refined RRR test should not be used
*        in this case
         SAWNAN1 = .TRUE.
      ENDIF
      MAX1 = ABS( DPLUS( 1 ) )
      DO 6 I = 1, N - 1
         LPLUS( I ) = LD( I ) / DPLUS( I )
         S = S*LPLUS( I )*L( I ) - LSIGMA
         DPLUS( I+1 ) = D( I+1 ) + S
         IF(ABS(DPLUS(I+1)).LT.PIVMIN) THEN
            DPLUS(I+1) = -PIVMIN
*           Need to set SAWNAN1 because refined RRR test should not be used
*           in this case
            SAWNAN1 = .TRUE.
         ENDIF
         MAX1 = MAX( MAX1,ABS(DPLUS(I+1)) )
 6    CONTINUE
      SAWNAN1 = SAWNAN1 .OR.  DISNAN( MAX1 )

      IF( FORCER .OR.
     $   (MAX1.LE.GROWTHBOUND .AND. .NOT.SAWNAN1 ) ) THEN
         SIGMA = LSIGMA
         SHIFT = SLEFT
         GOTO 100
      ENDIF
```

```
*     Right end
      S = -RSIGMA
      WORK( 1 ) = D( 1 ) + S
      IF(ABS(WORK(1)).LT.PIVMIN) THEN
         WORK(1) = -PIVMIN
*        Need to set SAWNAN2 because refined RRR test should not be used
*        in this case
         SAWNAN2 = .TRUE.
      ENDIF
      MAX2 = ABS( WORK( 1 ) )
      DO 7 I = 1, N - 1
         WORK( N+I ) = LD( I ) / WORK( I )
         S = S*WORK( N+I )*L( I ) - RSIGMA
         WORK( I+1 ) = D( I+1 ) + S
         IF(ABS(WORK(I+1)).LT.PIVMIN) THEN
            WORK(I+1) = -PIVMIN
*           Need to set SAWNAN2 because refined RRR test should not be used
*           in this case
            SAWNAN2 = .TRUE.
         ENDIF
         MAX2 = MAX( MAX2,ABS(WORK(I+1)) )
 7    CONTINUE
      SAWNAN2 = SAWNAN2 .OR.  DISNAN( MAX2 )

      IF( FORCER .OR.
     $   (MAX2.LE.GROWTHBOUND .AND. .NOT.SAWNAN2 ) ) THEN
         SIGMA = RSIGMA
         SHIFT = SRIGHT
         GOTO 100
      ENDIF
*     If we are at this point, both shifts led to too much element growth

*     Record the better of the two shifts (provided it didn't lead to NaN)
      IF(SAWNAN1.AND.SAWNAN2) THEN
*        both MAX1 and MAX2 are NaN
         GOTO 50
      ELSE
         IF( .NOT.SAWNAN1 ) THEN
            INDX = 1
            IF(MAX1.LE.SMLGROWTH) THEN
               SMLGROWTH = MAX1
               BESTSHIFT = LSIGMA
            ENDIF
         ENDIF
         IF( .NOT.SAWNAN2 ) THEN
            IF(SAWNAN1 .OR. MAX2.LE.MAX1) INDX = 2
            IF(MAX2.LE.SMLGROWTH) THEN
               SMLGROWTH = MAX2
               BESTSHIFT = RSIGMA
```

```
            ENDIF
         ENDIF
      ENDIF

*     If we are here, both the left and the right shift led to
*     element growth. If the element growth is moderate, then
*     we may still accept the representation, if it passes a
*     refined test for RRR. This test supposes that no NaN occurred.
*     Moreover, we use the refined RRR test only for isolated clusters.
      IF((CLWDTH.LT.MINGAP/DBLE(128)) .AND.
     $   (MIN(MAX1,MAX2).LT.FAIL2)
     $  .AND.(.NOT.SAWNAN1).AND.(.NOT.SAWNAN2)) THEN
         DORRR1 = .TRUE.
      ELSE
         DORRR1 = .FALSE.
      ENDIF
      TRYRRR1 = .TRUE.
      IF( TRYRRR1 .AND. DORRR1 ) THEN
      IF(INDX.EQ.1) THEN
         TMP = ABS( DPLUS( N ) )
         ZNM2 = ONE
         PROD = ONE
         OLDP = ONE
         DO 15 I = N-1, 1, -1
            IF( PROD .LE. EPS ) THEN
               PROD =
     $         ((DPLUS(I+1)*WORK(N+I+1))/(DPLUS(I)*WORK(N+I)))*OLDP
            ELSE
               PROD = PROD*ABS(WORK(N+I))
            END IF
            OLDP = PROD
            ZNM2 = ZNM2 + PROD**2
            TMP = MAX( TMP, ABS( DPLUS( I ) * PROD ))
 15      CONTINUE
         RRR1 = TMP/( SPDIAM * SQRT( ZNM2 ) )
         IF (RRR1.LE.MAXGROWTH2) THEN
            SIGMA = LSIGMA
            SHIFT = SLEFT
            GOTO 100
         ENDIF
      ELSE IF(INDX.EQ.2) THEN
         TMP = ABS( WORK( N ) )
         ZNM2 = ONE
         PROD = ONE
         OLDP = ONE
         DO 16 I = N-1, 1, -1
            IF( PROD .LE. EPS ) THEN
               PROD = ((WORK(I+1)*LPLUS(I+1))/(WORK(I)*LPLUS(I)))*OLDP
            ELSE
               PROD = PROD*ABS(LPLUS(I))
```

```
            END IF
            OLDP = PROD
            ZNM2 = ZNM2 + PROD**2
            TMP = MAX( TMP, ABS( WORK( I ) * PROD ))
 16      CONTINUE
         RRR2 = TMP/( SPDIAM * SQRT( ZNM2 ) )
         IF (RRR2.LE.MAXGROWTH2) THEN
            SIGMA = RSIGMA
            SHIFT = SRIGHT
            GOTO 100
         ENDIF
      END IF
      ENDIF

 50   CONTINUE

      IF (KTRY.LT.KTRYMAX) THEN
*        If we are here, both shifts failed also the RRR test.
*        Back off to the outside
         LSIGMA = MAX( LSIGMA - LDELTA,
     $     LSIGMA - LDMAX)
         RSIGMA = MIN( RSIGMA + RDELTA,
     $     RSIGMA + RDMAX )
         LDELTA = TWO * LDELTA
         RDELTA = TWO * RDELTA
         KTRY = KTRY + 1
         GOTO 5
      ELSE
*        None of the representations investigated satisfied our
*        criteria. Take the best one we found.
         IF((SMLGROWTH.LT.FAIL).OR.NOFAIL) THEN
            LSIGMA = BESTSHIFT
            RSIGMA = BESTSHIFT
            FORCER = .TRUE.
            GOTO 5
         ELSE
            INFO = 1
            RETURN
         ENDIF
      END IF

 100  CONTINUE
      IF (SHIFT.EQ.SLEFT) THEN
      ELSEIF (SHIFT.EQ.SRIGHT) THEN
*        store new L and D back into DPLUS, LPLUS
         CALL DCOPY( N, WORK, 1, DPLUS, 1 )
         CALL DCOPY( N-1, WORK(N+1), 1, LPLUS, 1 )
      ENDIF

      RETURN
```

```
*
*     End of DLARRF
*
      END
```

———-

90.24.4 lisp code

— **dlarrf.lisp** —

```
(let* ((one 1.0d0)
       (two 2.0d0)
       (four 4.0d0)
       (quart 0.25d0)
       (maxgrowth1 8.0d0)
       (maxgrowth2 8.0d0)
       (ktrymax 1)
       (sleft 1)
       (sright 2))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 4.0d0 4.0d0) four)
           (type (double-float 0.25d0 0.25d0) quart)
           (type (double-float 8.0d0 8.0d0) maxgrowth1)
           (type (double-float 8.0d0 8.0d0) maxgrowth2)
           (type (f2cl-lib:integer4 1 1) ktrymax)
           (type (f2cl-lib:integer4 1 1) sleft)
           (type (f2cl-lib:integer4 2 2) sright)
           (ignorable one two four quart maxgrowth1 maxgrowth2 ktrymax sleft
            sright))
  (defun dlarrf
         (n d l ld clstrt clend w wgap werr spdiam clgapl clgapr pivmin sigma
          dplus lplus work info)
    (declare (type (double-float) sigma pivmin clgapr clgapl spdiam)
             (type (array double-float (*)) work lplus dplus werr wgap w ld l
              d)
             (type (f2cl-lib:integer4) info clend clstrt n))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (l double-float l-%data% l-%offset%)
         (ld double-float ld-%data% ld-%offset%)
         (w double-float w-%data% w-%offset%)
         (wgap double-float wgap-%data% wgap-%offset%)
         (werr double-float werr-%data% werr-%offset%)
         (dplus double-float dplus-%data% dplus-%offset%)
         (lplus double-float lplus-%data% lplus-%offset%)
         (work double-float work-%data% work-%offset%))
```

```
(prog ((avgap 0.0d0) (bestshift 0.0d0) (clwdth 0.0d0) (eps 0.0d0)
       (fact 0.0d0) (fail 0.0d0) (fail2 0.0d0) (growthbound 0.0d0)
       (ldelta 0.0d0) (ldmax 0.0d0) (lsigma 0.0d0) (f2cl-lib:max1 0.0d0)
       (max2 0.0d0) (mingap 0.0d0) (oldp 0.0d0) (prod 0.0d0)
       (rdelta 0.0d0) (rdmax 0.0d0) (rrr1 0.0d0) (rrr2 0.0d0)
       (rsigma 0.0d0) (s 0.0d0) (smlgrowth 0.0d0) (tmp 0.0d0)
       (znm2 0.0d0) (i 0) (indx 0) (ktry 0) (shift 0) (dorrr1 nil)
       (forcer nil) (nofail nil) (sawnan1 nil) (sawnan2 nil)
       (tryrrr1 nil))
  (declare
   (type (double-float) avgap bestshift clwdth eps fact fail fail2
    growthbound ldelta ldmax lsigma f2cl-lib:max1 max2 mingap oldp prod
    rdelta rdmax rrr1 rrr2 rsigma s smlgrowth tmp znm2)
   (type (f2cl-lib:integer4) i indx ktry shift)
   (type f2cl-lib:logical dorrr1 forcer nofail sawnan1 sawnan2 tryrrr1))
  (setf info 0)
  (setf fact (f2cl-lib:dble (expt 2 ktrymax)))
  (setf eps (dlamch "Precision"))
  (setf shift 0)
  (setf forcer f2cl-lib:%false%)
  (setf nofail f2cl-lib:%false%)
  (setf clwdth
          (+
           (abs
            (- (f2cl-lib:fref w-%data% (clend) ((1 *)) w-%offset%)
               (f2cl-lib:fref w-%data% (clstrt) ((1 *)) w-%offset%)))
           (f2cl-lib:fref werr-%data% (clend) ((1 *)) werr-%offset%)
           (f2cl-lib:fref werr-%data% (clstrt) ((1 *)) werr-%offset%)))
  (setf avgap (/ clwdth (f2cl-lib:dble (f2cl-lib:int-sub clend clstrt))))
  (setf mingap (min clgapl clgapr))
  (setf lsigma
          (-
           (min (f2cl-lib:fref w-%data% (clstrt) ((1 *)) w-%offset%)
                (f2cl-lib:fref w-%data% (clend) ((1 *)) w-%offset%))
           (f2cl-lib:fref werr-%data% (clstrt) ((1 *)) werr-%offset%)))
  (setf rsigma
          (+
           (max (f2cl-lib:fref w-%data% (clstrt) ((1 *)) w-%offset%)
                (f2cl-lib:fref w-%data% (clend) ((1 *)) w-%offset%))
           (f2cl-lib:fref werr-%data% (clend) ((1 *)) werr-%offset%)))
  (setf lsigma (+ lsigma (* (- (abs lsigma)) four eps)))
  (setf rsigma (+ rsigma (* (abs rsigma) four eps)))
  (setf ldmax (+ (* quart mingap) (* two pivmin)))
  (setf rdmax (+ (* quart mingap) (* two pivmin)))
  (setf ldelta
          (/
           (max avgap
                (f2cl-lib:fref wgap-%data% (clstrt) ((1 *))
                               wgap-%offset%))
           fact))
```

```
 (setf rdelta
        (/
         (max avgap
              (f2cl-lib:fref wgap-%data% ((f2cl-lib:int-sub clend 1))
                             ((1 *)) wgap-%offset%))
         fact))
 (setf s (dlamch "S"))
 (setf smlgrowth (/ one s))
 (setf fail
        (/ (* (f2cl-lib:dble (f2cl-lib:int-sub n 1)) mingap)
           (* spdiam eps)))
 (setf fail2
        (/ (* (f2cl-lib:dble (f2cl-lib:int-sub n 1)) mingap)
           (* spdiam (f2cl-lib:fsqrt eps))))
 (setf bestshift lsigma)
 (setf ktry 0)
 (setf growthbound (* maxgrowth1 spdiam))
label5
 (setf sawnan1 f2cl-lib:%false%)
 (setf sawnan2 f2cl-lib:%false%)
 (setf ldelta (min ldmax ldelta))
 (setf rdelta (min rdmax rdelta))
 (setf s (- lsigma))
 (setf (f2cl-lib:fref dplus-%data% (1) ((1 *)) dplus-%offset%)
        (+ (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%) s))
 (cond
  ((< (abs (f2cl-lib:fref dplus (1) ((1 *)))) pivmin)
   (setf (f2cl-lib:fref dplus-%data% (1) ((1 *)) dplus-%offset%)
           (- pivmin))
   (setf sawnan1 f2cl-lib:%true%)))
 (setf f2cl-lib:max1
        (abs (f2cl-lib:fref dplus-%data% (1) ((1 *)) dplus-%offset%)))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
   (tagbody
     (setf (f2cl-lib:fref lplus-%data% (i) ((1 *)) lplus-%offset%)
             (/ (f2cl-lib:fref ld-%data% (i) ((1 *)) ld-%offset%)
                (f2cl-lib:fref dplus-%data% (i) ((1 *))
                               dplus-%offset%)))
     (setf s
             (-
              (* s
                 (f2cl-lib:fref lplus-%data% (i) ((1 *)) lplus-%offset%)
                 (f2cl-lib:fref l-%data% (i) ((1 *)) l-%offset%))
              lsigma))
     (setf (f2cl-lib:fref dplus-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                          dplus-%offset%)
             (+
              (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                             d-%offset%)
```

```
            s))
    (cond
     ((< (abs (f2cl-lib:fref dplus ((f2cl-lib:int-add i 1)) ((1 *))))
         pivmin)
      (setf (f2cl-lib:fref dplus-%data% ((f2cl-lib:int-add i 1))
                           ((1 *)) dplus-%offset%)
            (- pivmin))
      (setf sawnan1 f2cl-lib:%true%)))
    (setf f2cl-lib:max1
            (max f2cl-lib:max1
                 (abs
                  (f2cl-lib:fref dplus-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) dplus-%offset%))))
   label6))
(setf sawnan1 (or sawnan1 (disnan f2cl-lib:max1)))
(cond
 ((or forcer (and (<= f2cl-lib:max1 growthbound) (not sawnan1)))
  (setf sigma lsigma) (setf shift sleft) (go label100)))
(setf s (- rsigma))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (+ (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%) s))
(cond
 ((< (abs (f2cl-lib:fref work (1) ((1 *)))) pivmin)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (- pivmin))
  (setf sawnan2 f2cl-lib:%true%)))
(setf max2 (abs (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i)) ((1 *))
                         work-%offset%)
            (/ (f2cl-lib:fref ld-%data% (i) ((1 *)) ld-%offset%)
               (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)))
    (setf s
            (-
             (* s
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%)
                (f2cl-lib:fref l-%data% (i) ((1 *)) l-%offset%))
             rsigma))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         work-%offset%)
            (+
             (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                            d-%offset%)
             s))
    (cond
     ((< (abs (f2cl-lib:fref work ((f2cl-lib:int-add i 1)) ((1 *))))
         pivmin)
```

```
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                           work-%offset%)
              (- pivmin))
      (setf sawnan2 f2cl-lib:%true%)))
    (setf max2
            (max max2
                 (abs
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) work-%offset%))))
   label7))
(setf sawnan2
        (or sawnan2
            (multiple-value-bind (ret-val var-0)
                (disnan max2)
              (declare (ignore))
              (setf max2 var-0)
              ret-val)))
(cond
 ((or forcer (and (<= max2 growthbound) (not sawnan2)))
  (setf sigma rsigma) (setf shift sright) (go label100)))
(cond ((and sawnan1 sawnan2) (go label50))
      (t
       (cond
        ((not sawnan1) (setf indx 1)
         (cond
          ((<= f2cl-lib:max1 smlgrowth) (setf smlgrowth f2cl-lib:max1)
           (setf bestshift lsigma)))))
       (cond
        ((not sawnan2)
         (if (or sawnan1 (<= max2 f2cl-lib:max1))
             (setf indx 2))
         (cond
          ((<= max2 smlgrowth) (setf smlgrowth max2)
           (setf bestshift rsigma)))))))
(cond
 ((and (< clwdth (f2cl-lib:f2cl/ mingap (f2cl-lib:dble 128)))
       (< (min f2cl-lib:max1 max2) fail2) (not sawnan1) (not sawnan2))
  (setf dorrr1 f2cl-lib:%true%))
 (t (setf dorrr1 f2cl-lib:%false%)))
(setf tryrrr1 f2cl-lib:%true%)
(cond
 ((and tryrrr1 dorrr1)
  (cond
   ((= indx 1)
    (setf tmp
            (abs
             (f2cl-lib:fref dplus-%data% (n) ((1 *)) dplus-%offset%)))
    (setf znm2 one) (setf prod one) (setf oldp one)
    (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
```

```
                ((> i 1) nil)
      (tagbody
        (cond
         ((<= prod eps)
          (setf prod
                  (*
                   (/
                    (*
                     (f2cl-lib:fref dplus-%data%
                                    ((f2cl-lib:int-add i 1)) ((1 *))
                                    dplus-%offset%)
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add n i 1)) ((1 *))
                                    work-%offset%))
                    (*
                     (f2cl-lib:fref dplus-%data% (i) ((1 *))
                                    dplus-%offset%)
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add n i)) ((1 *))
                                    work-%offset%)))
                   oldp)))
         (t
          (setf prod
                  (* prod
                     (abs
                      (f2cl-lib:fref work-%data%
                                     ((f2cl-lib:int-add n i)) ((1 *))
                                     work-%offset%))))))
        (setf oldp prod)
        (setf znm2 (+ znm2 (expt prod 2)))
        (setf tmp
                (max tmp
                     (abs
                      (*
                       (f2cl-lib:fref dplus-%data% (i) ((1 *))
                                      dplus-%offset%)
                       prod))))
       label15))
    (setf rrr1 (/ tmp (* spdiam (f2cl-lib:fsqrt znm2))))
    (cond
     ((<= rrr1 maxgrowth2) (setf sigma lsigma) (setf shift sleft)
      (go label100))))
   ((= indx 2)
    (setf tmp
            (abs
             (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)))
    (setf znm2 one) (setf prod one) (setf oldp one)
    (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
```

```
          (tagbody
            (cond
             ((<= prod eps)
              (setf prod
                      (*
                       (/
                        (*
                         (f2cl-lib:fref work-%data%
                                        ((f2cl-lib:int-add i 1)) ((1 *))
                                        work-%offset%)
                         (f2cl-lib:fref lplus-%data%
                                        ((f2cl-lib:int-add i 1)) ((1 *))
                                        lplus-%offset%))
                        (*
                         (f2cl-lib:fref work-%data% (i) ((1 *))
                                        work-%offset%)
                         (f2cl-lib:fref lplus-%data% (i) ((1 *))
                                        lplus-%offset%)))
                       oldp)))
             (t
              (setf prod
                      (* prod
                         (abs
                          (f2cl-lib:fref lplus-%data% (i) ((1 *))
                                         lplus-%offset%))))))
            (setf oldp prod)
            (setf znm2 (+ znm2 (expt prod 2)))
            (setf tmp
                    (max tmp
                         (abs
                          (*
                           (f2cl-lib:fref work-%data% (i) ((1 *))
                                          work-%offset%)
                           prod))))
           label16))
        (setf rrr2 (/ tmp (* spdiam (f2cl-lib:fsqrt znm2))))
        (cond
         ((<= rrr2 maxgrowth2) (setf sigma rsigma) (setf shift sright)
          (go label100)))))))
label50
 (cond
  ((< ktry ktrymax)
   (setf lsigma (max (- lsigma ldelta) (- lsigma ldmax)))
   (setf rsigma (min (+ rsigma rdelta) (+ rsigma rdmax)))
   (setf ldelta (* two ldelta)) (setf rdelta (* two rdelta))
   (setf ktry (f2cl-lib:int-add ktry 1)) (go label5))
  (t
   (cond
    ((or (< smlgrowth fail) nofail) (setf lsigma bestshift)
     (setf rsigma bestshift) (setf forcer f2cl-lib:%true%) (go label5))
```

```
          (t (setf info 1) (go end_label)))))
      label100
       (cond ((= shift sleft))
             ((= shift sright)
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (dcopy n work 1 dplus 1)
                (declare (ignore var-1 var-2 var-3 var-4))
                (when var-0 (setf n var-0)))
              (dcopy (f2cl-lib:int-sub n 1)
               (f2cl-lib:array-slice work-%data% double-float ((+ n 1))
                                     ((1 *)) work-%offset%)
               1 lplus 1)))
       (go end_label)
      end_label
       (return
        (values n nil nil nil nil nil nil nil nil nil nil nil nil sigma nil
                nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlarrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (double-float) (double-float)
                                              (double-float) (double-float)
                                              (double-float)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::n nil nil nil
                                              nil nil nil nil nil nil nil nil
                                              nil fortran-to-lisp::sigma nil
                                              nil nil fortran-to-lisp::info)
                                            :calls '(fortran-to-lisp::disnan))))
```

90.25 dlarrj

90.25.1 Axiom unit tests

— dlarrj.input —

```
)set break resume
)sys rm -f dlarrj.output
)spool dlarrj.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.25.2 Axiom help page

— dlarrj.help —

```
======================================================================
dlarrj examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlarrj (N, D, E2, IFIRST, ILAST, RTOL, OFFSET, W, WERR,
           WORK, IWORK, PIVMIN, SPDIAM, INFO)
           DLARRJ performs refinement of the initial estimates of the
           eigenvalues of the matrix T.

Function/Subroutine Documentation
   subroutine dlarrj (integer N, double precision, dimension( * ) D, double
       precision, dimension( * ) E2, integer IFIRST, integer ILAST, double
       precision RTOL, integer OFFSET, double precision, dimension( * ) W,
       double precision, dimension( * ) WERR, double precision, dimension( * )
       WORK, integer, dimension( * ) IWORK, double precision PIVMIN, double
       precision SPDIAM, integer INFO)
       DLARRJ performs refinement of the initial estimates of the eigenvalues
       of the matrix T.
```

```
Purpose:
   Given the initial eigenvalue approximations of T, DLARRJ
   does  bisection to refine the eigenvalues of T,
   W( IFIRST-OFFSET ) through W( ILAST-OFFSET ), to more accuracy. Initial
   guesses for these eigenvalues are input in W, the corresponding estimate
   of the error in these guesses in WERR. During bisection, intervals
   [left, right] are maintained by storing their mid-points and
   semi-widths in the arrays W and WERR respectively.

Parameters:

 N
   N is INTEGER
   The order of the matrix.

 D
   D is DOUBLE PRECISION array, dimension (N)
   The N diagonal elements of T.

 E2
   E2 is DOUBLE PRECISION array, dimension (N-1)
   The Squares of the (N-1) subdiagonal elements of T.

 IFIRST
   IFIRST is INTEGER
   The index of the first eigenvalue to be computed.

 ILAST
   ILAST is INTEGER
   The index of the last eigenvalue to be computed.

 RTOL
   RTOL is DOUBLE PRECISION
   Tolerance for the convergence of the bisection intervals.
   An interval [LEFT,RIGHT] has converged if
   RIGHT-LEFT.LT.RTOL*MAX(|LEFT|,|RIGHT|).

 OFFSET
   OFFSET is INTEGER
   Offset for the arrays W and WERR, i.e., the IFIRST-OFFSET
   through ILAST-OFFSET elements of these arrays are to be used.

 W
   W is DOUBLE PRECISION array, dimension (N)
   On input, W( IFIRST-OFFSET ) through W( ILAST-OFFSET ) are
   estimates of the eigenvalues of L D L^T indexed IFIRST through ILAST.
   On output, these estimates are refined.

 WERR
```

```
  WERR is DOUBLE PRECISION array, dimension (N)
  On input, WERR( IFIRST-OFFSET ) through WERR( ILAST-OFFSET ) are
  the errors in the estimates of the corresponding elements in W.
  On output, these errors are refined.

WORK
  WORK is DOUBLE PRECISION array, dimension (2*N)
  Workspace.

IWORK
  IWORK is INTEGER array, dimension (2*N)
  Workspace.

PIVMIN
  PIVMIN is DOUBLE PRECISION
  The minimum pivot in the Sturm sequence for T.

SPDIAM
  SPDIAM is DOUBLE PRECISION
  The spectral diameter of T.

INFO
  INFO is INTEGER
  Error flag.
```

———-

90.25.3 fortran code

— dlarrj.f —

```
*  =====================================================================
      SUBROUTINE DLARRJ( N, D, E2, IFIRST, ILAST,
     $                   RTOL, OFFSET, W, WERR, WORK, IWORK,
     $                   PIVMIN, SPDIAM, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            IFIRST, ILAST, INFO, N, OFFSET
      DOUBLE PRECISION   PIVMIN, RTOL, SPDIAM
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
```

```
      DOUBLE PRECISION   D( * ), E2( * ), W( * ),
     $                   WERR( * ), WORK( * )
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE, TWO, HALF
      PARAMETER        ( ZERO = 0.0D0, ONE = 1.0D0, TWO = 2.0D0,
     $                   HALF = 0.5D0 )
      INTEGER   MAXITR
*     ..
*     .. Local Scalars ..
      INTEGER            CNT, I, I1, I2, II, ITER, J, K, NEXT, NINT,
     $                   OLNINT, P, PREV, SAVI1
      DOUBLE PRECISION   DPLUS, FAC, LEFT, MID, RIGHT, S, TMP, WIDTH
*
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
      MAXITR = INT( ( LOG( SPDIAM+PIVMIN )-LOG( PIVMIN ) ) /
     $           LOG( TWO ) ) + 2
*
*     Initialize unconverged intervals in [ WORK(2*I-1), WORK(2*I) ].
*     The Sturm Count, Count( WORK(2*I-1) ) is arranged to be I-1, while
*     Count( WORK(2*I) ) is stored in IWORK( 2*I ). The integer IWORK( 2*I-1 )
*     for an unconverged interval is set to the index of the next unconverged
*     interval, and is -1 or 0 for a converged interval. Thus a linked
*     list of unconverged intervals is set up.
*

      I1 = IFIRST
      I2 = ILAST
*     The number of unconverged intervals
      NINT = 0
*     The last unconverged interval found
      PREV = 0
      DO 75 I = I1, I2
         K = 2*I
         II = I - OFFSET
         LEFT = W( II ) - WERR( II )
         MID = W(II)
         RIGHT = W( II ) + WERR( II )
         WIDTH = RIGHT - MID
         TMP = MAX( ABS( LEFT ), ABS( RIGHT ) )
```

```
*        The following test prevents the test of converged intervals
         IF( WIDTH.LT.RTOL*TMP ) THEN
*           This interval has already converged and does not need refinement.
*           (Note that the gaps might change through refining the
*            eigenvalues, however, they can only get bigger.)
*           Remove it from the list.
            IWORK( K-1 ) = -1
*           Make sure that I1 always points to the first unconverged interval
            IF((I.EQ.I1).AND.(I.LT.I2)) I1 = I + 1
            IF((PREV.GE.I1).AND.(I.LE.I2)) IWORK( 2*PREV-1 ) = I + 1
         ELSE
*           unconverged interval found
            PREV = I
*           Make sure that [LEFT,RIGHT] contains the desired eigenvalue
*
*           Do while( CNT(LEFT).GT.I-1 )
*
            FAC = ONE
 20         CONTINUE
            CNT = 0
            S = LEFT
            DPLUS = D( 1 ) - S
            IF( DPLUS.LT.ZERO ) CNT = CNT + 1
            DO 30 J = 2, N
               DPLUS = D( J ) - S - E2( J-1 )/DPLUS
               IF( DPLUS.LT.ZERO ) CNT = CNT + 1
 30         CONTINUE
            IF( CNT.GT.I-1 ) THEN
               LEFT = LEFT - WERR( II )*FAC
               FAC = TWO*FAC
               GO TO 20
            END IF
*
*           Do while( CNT(RIGHT).LT.I )
*
            FAC = ONE
 50         CONTINUE
            CNT = 0
            S = RIGHT
            DPLUS = D( 1 ) - S
            IF( DPLUS.LT.ZERO ) CNT = CNT + 1
            DO 60 J = 2, N
               DPLUS = D( J ) - S - E2( J-1 )/DPLUS
               IF( DPLUS.LT.ZERO ) CNT = CNT + 1
 60         CONTINUE
            IF( CNT.LT.I ) THEN
               RIGHT = RIGHT + WERR( II )*FAC
               FAC = TWO*FAC
               GO TO 50
```

```
            END IF
            NINT = NINT + 1
            IWORK( K-1 ) = I + 1
            IWORK( K ) = CNT
         END IF
         WORK( K-1 ) = LEFT
         WORK( K ) = RIGHT
 75   CONTINUE


      SAVI1 = I1
*
*     Do while( NINT.GT.0 ), i.e. there are still unconverged intervals
*     and while (ITER.LT.MAXITR)
*
      ITER = 0
 80   CONTINUE
      PREV = I1 - 1
      I = I1
      OLNINT = NINT

      DO 100 P = 1, OLNINT
         K = 2*I
         II = I - OFFSET
         NEXT = IWORK( K-1 )
         LEFT = WORK( K-1 )
         RIGHT = WORK( K )
         MID = HALF*( LEFT + RIGHT )

*        semiwidth of interval
         WIDTH = RIGHT - MID
         TMP = MAX( ABS( LEFT ), ABS( RIGHT ) )

         IF( ( WIDTH.LT.RTOL*TMP ) .OR.
     $      (ITER.EQ.MAXITR) )THEN
*           reduce number of unconverged intervals
            NINT = NINT - 1
*           Mark interval as converged.
            IWORK( K-1 ) = 0
            IF( I1.EQ.I ) THEN
               I1 = NEXT
            ELSE
*              Prev holds the last unconverged interval previously examined
               IF(PREV.GE.I1) IWORK( 2*PREV-1 ) = NEXT
            END IF
            I = NEXT
            GO TO 100
         END IF
         PREV = I
*
```

```
*        Perform one bisection step
*
         CNT = 0
         S = MID
         DPLUS = D( 1 ) - S
         IF( DPLUS.LT.ZERO ) CNT = CNT + 1
         DO 90 J = 2, N
            DPLUS = D( J ) - S - E2( J-1 )/DPLUS
            IF( DPLUS.LT.ZERO ) CNT = CNT + 1
 90      CONTINUE
         IF( CNT.LE.I-1 ) THEN
            WORK( K-1 ) = MID
         ELSE
            WORK( K ) = MID
         END IF
         I = NEXT

 100  CONTINUE
      ITER = ITER + 1
*     do another loop if there are still unconverged intervals
*     However, in the last iteration, all intervals are accepted
*     since this is the best we can do.
      IF( ( NINT.GT.0 ).AND.(ITER.LE.MAXITR) ) GO TO 80
*
*
*     At this point, all the intervals have converged
      DO 110 I = SAVI1, ILAST
         K = 2*I
         II = I - OFFSET
*        All intervals marked by '0' have been refined.
         IF( IWORK( K-1 ).EQ.0 ) THEN
            W( II ) = HALF*( WORK( K-1 )+WORK( K ) )
            WERR( II ) = WORK( K ) - W( II )
         END IF
 110  CONTINUE
*

      RETURN
*
*     End of DLARRJ
*
      END
```

———-

90.25.4 lisp code

— dlarrj.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0) (two 2.0d0) (half 0.5d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 0.5d0 0.5d0) half)
           (ignorable zero one two half))
  (defun dlarrj
         (n d e2 ifirst ilast rtol offset w werr work iwork pivmin spdiam info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (double-float) spdiam pivmin rtol)
             (type (array double-float (*)) work werr w e2 d)
             (type (f2cl-lib:integer4) info offset ilast ifirst n))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (e2 double-float e2-%data% e2-%offset%)
         (w double-float w-%data% w-%offset%)
         (werr double-float werr-%data% werr-%offset%)
         (work double-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((dplus 0.0d0) (fac 0.0d0) (left 0.0d0) (mid 0.0d0) (right 0.0d0)
             (s 0.0d0) (tmp 0.0d0) (width 0.0d0) (cnt 0) (i 0) (i1 0) (i2 0)
             (ii 0) (iter 0) (j 0) (k 0) (next 0) (f2cl-lib:nint 0) (olnint 0)
             (p 0) (prev 0) (savi1 0) (maxitr 0))
        (declare (type (double-float) dplus fac left mid right s tmp width)
                 (type (f2cl-lib:integer4) cnt i i1 i2 ii iter j k next
                  f2cl-lib:nint olnint p prev savi1 maxitr))
        (setf info 0)
        (setf maxitr
                (f2cl-lib:int-add
                 (f2cl-lib:int
                  (/
                   (- (f2cl-lib:flog (+ spdiam pivmin)) (f2cl-lib:flog pivmin))
                   (f2cl-lib:flog two)))
                 2))
        (setf i1 ifirst)
        (setf i2 ilast)
        (setf f2cl-lib:nint 0)
        (setf prev 0)
        (f2cl-lib:fdo (i i1 (f2cl-lib:int-add i 1))
                      ((> i i2) nil)
          (tagbody
            (setf k (f2cl-lib:int-mul 2 i))
            (setf ii (f2cl-lib:int-sub i offset))
            (setf left
                    (- (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%)
                       (f2cl-lib:fref werr-%data% (ii) ((1 *)) werr-%offset%)))
            (setf mid (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%))
            (setf right
                    (+ (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%)
                       (f2cl-lib:fref werr-%data% (ii) ((1 *)) werr-%offset%)))
```

```
(setf width (- right mid))
(setf tmp (max (abs left) (abs right)))
(cond
 ((< width (* rtol tmp))
  (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-sub k 1))
                       ((1 *)) iwork-%offset%)
        -1)
  (if (and (= i i1) (< i i2))
      (setf i1 (f2cl-lib:int-add i 1)))
  (if (and (>= prev i1) (<= i i2))
      (setf (f2cl-lib:fref iwork-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-mul 2 prev) 1))
                           ((1 *)) iwork-%offset%)
            (f2cl-lib:int-add i 1))))
 (t
  (tagbody
    (setf prev i)
    (setf fac one)
   label20
    (setf cnt 0)
    (setf s left)
    (setf dplus
            (- (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%) s))
    (if (< dplus zero)
        (setf cnt (f2cl-lib:int-add cnt 1)))
    (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf dplus
                (+
                 (- (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                    s)
                 (/
                  (-
                   (f2cl-lib:fref e2-%data%
                                  ((f2cl-lib:int-sub j 1)) ((1 *))
                                  e2-%offset%))
                  dplus)))
        (if (< dplus zero)
            (setf cnt (f2cl-lib:int-add cnt 1)))
       label30))
    (cond
     ((> cnt (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
      (setf left
              (- left
                 (*
                  (f2cl-lib:fref werr-%data% (ii) ((1 *))
                                 werr-%offset%)
                  fac)))
```

```
            (setf fac (* two fac)) (go label20)))
          (setf fac one)
         label50
          (setf cnt 0)
          (setf s right)
          (setf dplus
                  (- (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%) s))
          (if (< dplus zero)
              (setf cnt (f2cl-lib:int-add cnt 1)))
          (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf dplus
                      (+
                       (- (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                          s)
                       (/
                        (-
                         (f2cl-lib:fref e2-%data%
                                        ((f2cl-lib:int-sub j 1)) ((1 *))
                                        e2-%offset%))
                        dplus)))
              (if (< dplus zero)
                  (setf cnt (f2cl-lib:int-add cnt 1)))
             label60))
          (cond
           ((< cnt i)
            (setf right
                    (+ right
                       (*
                        (f2cl-lib:fref werr-%data% (ii) ((1 *))
                                       werr-%offset%)
                        fac)))
            (setf fac (* two fac)) (go label50)))
          (setf f2cl-lib:nint (f2cl-lib:int-add f2cl-lib:nint 1))
          (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-sub k 1))
                               ((1 *)) iwork-%offset%)
                  (f2cl-lib:int-add i 1))
          (setf (f2cl-lib:fref iwork-%data% (k) ((1 *)) iwork-%offset%)
                  cnt))))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                           work-%offset%)
              left)
      (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%) right)
     label75))
  (setf savi1 i1)
  (setf iter 0)
 label80
  (setf prev (f2cl-lib:int-sub i1 1))
  (setf i i1)
```

```
(setf olnint f2cl-lib:nint)
(f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
              ((> p olnint) nil)
  (tagbody
    (setf k (f2cl-lib:int-mul 2 i))
    (setf ii (f2cl-lib:int-sub i offset))
    (setf next
            (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-sub k 1))
                           ((1 *)) iwork-%offset%))
    (setf left
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                           work-%offset%))
    (setf right (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%))
    (setf mid (* half (+ left right)))
    (setf width (- right mid))
    (setf tmp (max (abs left) (abs right)))
    (cond
     ((or (< width (* rtol tmp)) (= iter maxitr))
      (setf f2cl-lib:nint (f2cl-lib:int-sub f2cl-lib:nint 1))
      (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-sub k 1))
                           ((1 *)) iwork-%offset%)
              0)
      (cond ((= i1 i) (setf i1 next))
            (t
             (if (>= prev i1)
                 (setf (f2cl-lib:fref iwork-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-mul 2 prev) 1))
                                      ((1 *)) iwork-%offset%)
                       next))))
      (setf i next) (go label100)))
    (setf prev i)
    (setf cnt 0)
    (setf s mid)
    (setf dplus (- (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%) s))
    (if (< dplus zero)
        (setf cnt (f2cl-lib:int-add cnt 1)))
    (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf dplus
                (+
                 (- (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%) s)
                 (/
                  (-
                   (f2cl-lib:fref e2-%data% ((f2cl-lib:int-sub j 1))
                                  ((1 *)) e2-%offset%))
                  dplus)))
        (if (< dplus zero)
            (setf cnt (f2cl-lib:int-add cnt 1)))
```

```
              label90))
            (cond
             ((<= cnt (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                                   work-%offset%)
                    mid))
             (t
              (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                    mid)))
            (setf i next)
           label100))
        (setf iter (f2cl-lib:int-add iter 1))
        (if (and (> f2cl-lib:nint 0) (<= iter maxitr))
            (go label80))
        (f2cl-lib:fdo (i savi1 (f2cl-lib:int-add i 1))
                      ((> i ilast) nil)
          (tagbody
            (setf k (f2cl-lib:int-mul 2 i))
            (setf ii (f2cl-lib:int-sub i offset))
            (cond
             ((=
               (f2cl-lib:fref iwork ((f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                              ((1 *)))
               0)
              (setf (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%)
                    (* half
                       (+
                        (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1))
                                       ((1 *)) work-%offset%)
                        (f2cl-lib:fref work-%data% (k) ((1 *))
                                       work-%offset%))))
              (setf (f2cl-lib:fref werr-%data% (ii) ((1 *)) werr-%offset%)
                    (- (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                       (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%)))))
           label110))
        (go end_label)
       end_label
        (return
         (values nil nil nil nil nil nil nil nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlarrj
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4)
```

```
  (fortran-to-lisp::integer4)
  (double-float)
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (array double-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (double-float) (double-float)
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil nil nil nil nil nil
  nil nil nil nil nil
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

90.26 dlarrk

90.26.1 Axiom unit tests

— **dlarrk.input** —

```
)set break resume
)sys rm -f dlarrk.output
)spool dlarrk.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.26.2 Axiom help page

— **dlarrk.help** —

```
======================================================================
dlarrk examples
```

```
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlarrk (N, IW, GL, GU, D, E2, PIVMIN, RELTOL, W, WERR, INFO)
           DLARRK computes one eigenvalue of a symmetric tridiagonal matrix T
           to suitable accuracy.

Function/Subroutine Documentation
   subroutine dlarrk (integer N, integer IW, double precision GL, double
       precision GU, double precision, dimension( * ) D, double precision,
       dimension( * ) E2, double precision PIVMIN, double precision RELTOL,
       double precision W, double precision WERR, integer INFO)
       DLARRK computes one eigenvalue of a symmetric tridiagonal matrix T to
       suitable accuracy.

Purpose:
   DLARRK computes one eigenvalue of a symmetric tridiagonal
   matrix T to suitable accuracy. This is an auxiliary code to be
   called from DSTEMR.

   To avoid overflow, the matrix must be scaled so that its
   largest element is no greater than
      overflow**(1/2) * underflow**(1/4)
   in absolute value, and for greatest
   accuracy, it should not be much smaller than that.

Parameters:

 N
   N is INTEGER
   The order of the tridiagonal matrix T.  N >= 0.

 IW
   IW is INTEGER
   The index of the eigenvalues to be returned.

 GL
   GL is DOUBLE PRECISION

 GU
   GU is DOUBLE PRECISION
   An upper and a lower bound on the eigenvalue.

 D
   D is DOUBLE PRECISION array, dimension (N)
```

```
  The n diagonal elements of the tridiagonal matrix T.

 E2
  E2 is DOUBLE PRECISION array, dimension (N-1)
  The (n-1) squared off-diagonal elements of the tridiagonal matrix T.

 PIVMIN
  PIVMIN is DOUBLE PRECISION
  The minimum pivot allowed in the Sturm sequence for T.

 RELTOL
  RELTOL is DOUBLE PRECISION
  The minimum relative width of an interval.  When an interval
  is narrower than RELTOL times the larger (in
  magnitude) endpoint, then it is considered to be
  sufficiently small, i.e., converged.  Note: this should
  always be at least radix*machine epsilon.

 W
  W is DOUBLE PRECISION

 WERR
  WERR is DOUBLE PRECISION
  The error bound on the corresponding eigenvalue approximation in W.

 INFO
  INFO is INTEGER
  = 0:       Eigenvalue converged
  = -1:      Eigenvalue did NOT converge

Internal Parameters:

 FUDGE   DOUBLE PRECISION, default = 2
  A "fudge factor" to widen the Gershgorin intervals.
```

———-

90.26.3 fortran code

— **dlarrk.f** —

```
*  =====================================================================
      SUBROUTINE DLARRK( N, IW, GL, GU,
     $                   D, E2, PIVMIN, RELTOL, W, WERR, INFO)
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
```

```
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER   INFO, IW, N
      DOUBLE PRECISION    PIVMIN, RELTOL, GL, GU, W, WERR
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), E2( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   FUDGE, HALF, TWO, ZERO
      PARAMETER          ( HALF = 0.5D0, TWO = 2.0D0,
     $                     FUDGE = TWO, ZERO = 0.0D0 )
*     ..
*     .. Local Scalars ..
      INTEGER   I, IT, ITMAX, NEGCNT
      DOUBLE PRECISION   ATOLI, EPS, LEFT, MID, RIGHT, RTOLI, TMP1,
     $                   TMP2, TNORM
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL   DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, INT, LOG, MAX
*     ..
*     .. Executable Statements ..
*
*     Get machine constants
      EPS = DLAMCH( 'P' )

      TNORM = MAX( ABS( GL ), ABS( GU ) )
      RTOLI = RELTOL
      ATOLI = FUDGE*TWO*PIVMIN

      ITMAX = INT( ( LOG( TNORM+PIVMIN )-LOG( PIVMIN ) ) /
     $           LOG( TWO ) ) + 2

      INFO = -1

      LEFT = GL - FUDGE*TNORM*EPS*N - FUDGE*TWO*PIVMIN
      RIGHT = GU + FUDGE*TNORM*EPS*N + FUDGE*TWO*PIVMIN
      IT = 0

 10   CONTINUE
*
```

```
*     Check if interval converged or maximum number of iterations reached
*
      TMP1 = ABS( RIGHT - LEFT )
      TMP2 = MAX( ABS(RIGHT), ABS(LEFT) )
      IF( TMP1.LT.MAX( ATOLI, PIVMIN, RTOLI*TMP2 ) ) THEN
         INFO = 0
         GOTO 30
      ENDIF
      IF(IT.GT.ITMAX)
     $   GOTO 30

*
*     Count number of negative pivots for mid-point
*
      IT = IT + 1
      MID = HALF * (LEFT + RIGHT)
      NEGCNT = 0
      TMP1 = D( 1 ) - MID
      IF( ABS( TMP1 ).LT.PIVMIN )
     $   TMP1 = -PIVMIN
      IF( TMP1.LE.ZERO )
     $   NEGCNT = NEGCNT + 1
*
      DO 20 I = 2, N
         TMP1 = D( I ) - E2( I-1 ) / TMP1 - MID
         IF( ABS( TMP1 ).LT.PIVMIN )
     $      TMP1 = -PIVMIN
         IF( TMP1.LE.ZERO )
     $      NEGCNT = NEGCNT + 1
 20   CONTINUE

      IF(NEGCNT.GE.IW) THEN
         RIGHT = MID
      ELSE
         LEFT = MID
      ENDIF
      GOTO 10

 30   CONTINUE
*
*     Converged or maximum number of iterations reached
*
      W = HALF * (LEFT + RIGHT)
      WERR = HALF * ABS( RIGHT - LEFT )

      RETURN
*
*     End of DLARRK
*
      END
```

———-

90.26.4 lisp code

— dlarrk.lisp —

```
(let* ((half 0.5d0) (two 2.0d0) (fudge two) (zero 0.0d0))
  (declare (type (double-float 0.5d0 0.5d0) half)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float) fudge)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable half two fudge zero))
  (defun dlarrk (n iw gl gu d e2 pivmin reltol w werr info)
    (declare (type (array double-float (*)) e2 d)
             (type (double-float) werr w reltol pivmin gu gl)
             (type (f2cl-lib:integer4) info iw n))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (e2 double-float e2-%data% e2-%offset%))
      (prog ((atoli 0.0d0) (eps 0.0d0) (left 0.0d0) (mid 0.0d0) (right 0.0d0)
             (rtoli 0.0d0) (tmp1 0.0d0) (tmp2 0.0d0) (tnorm 0.0d0) (i 0) (it 0)
             (itmax 0) (negcnt 0))
        (declare
         (type (double-float) atoli eps left mid right rtoli tmp1 tmp2 tnorm)
         (type (f2cl-lib:integer4) i it itmax negcnt))
        (setf eps (dlamch "P"))
        (setf tnorm (max (abs gl) (abs gu)))
        (setf rtoli reltol)
        (setf atoli (* fudge two pivmin))
        (setf itmax
                (f2cl-lib:int-add
                 (f2cl-lib:int
                  (/
                   (- (f2cl-lib:flog (+ tnorm pivmin)) (f2cl-lib:flog pivmin))
                   (f2cl-lib:flog two)))
                 2))
        (setf info -1)
        (setf left (+ gl (* (- fudge) tnorm eps n) (* (- fudge) two pivmin)))
        (setf right (+ gu (* fudge tnorm eps n) (* fudge two pivmin)))
        (setf it 0)
       label10
        (setf tmp1 (abs (- right left)))
        (setf tmp2 (max (abs right) (abs left)))
        (cond
         ((< tmp1 (max atoli pivmin (* rtoli tmp2))) (setf info 0)
          (go label30)))
        (if (> it itmax)
            (go label30))
```

```
        (setf it (f2cl-lib:int-add it 1))
        (setf mid (* half (+ left right)))
        (setf negcnt 0)
        (setf tmp1 (- (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%) mid))
        (if (< (abs tmp1) pivmin)
            (setf tmp1 (- pivmin)))
        (if (<= tmp1 zero)
            (setf negcnt (f2cl-lib:int-add negcnt 1)))
        (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf tmp1
                    (-
                     (+ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                        (/
                         (-
                          (f2cl-lib:fref e2-%data% ((f2cl-lib:int-sub i 1))
                                         ((1 *)) e2-%offset%))
                         tmp1))
                     mid))
            (if (< (abs tmp1) pivmin)
                (setf tmp1 (- pivmin)))
            (if (<= tmp1 zero)
                (setf negcnt (f2cl-lib:int-add negcnt 1)))
           label20))
        (cond ((>= negcnt iw) (setf right mid)) (t (setf left mid)))
        (go label10)
       label30
        (setf w (* half (+ left right)))
        (setf werr (* half (abs (- right left))))
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil nil nil w werr info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlarrk
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (double-float) (double-float)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (double-float) (double-float)
                                              (double-float) (double-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
```

```
                                          fortran-to-lisp::w
                                          fortran-to-lisp::werr
                                          fortran-to-lisp::info)
                                        :calls 'nil)))
```

———-

90.27 dlarrr

90.27.1 Axiom unit tests

— **dlarrr.input** —

```
)set break resume
)sys rm -f dlarrr.output
)spool dlarrr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.27.2 Axiom help page

— **dlarrr.help** —

```
==================================================================
dlarrr examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlarrr (N, D, E, INFO)
           DLARRR performs tests to decide whether the symmetric tridiagonal
           matrix T warrants expensive computations which guarantee high
```

```
          relative accuracy in the eigenvalues.

Function/Subroutine Documentation
   subroutine dlarrr (integer N, double precision, dimension( * ) D, double
       precision, dimension( * ) E, integer INFO)
       DLARRR performs tests to decide whether the symmetric tridiagonal
       matrix T warrants expensive computations which guarantee high relative
       accuracy in the eigenvalues.

Purpose:
   Perform tests to decide whether the symmetric tridiagonal matrix T
   warrants expensive computations which guarantee high relative accuracy
   in the eigenvalues.

Parameters:

 N
   N is INTEGER
   The order of the matrix. N > 0.

 D
   D is DOUBLE PRECISION array, dimension (N)
   The N diagonal elements of the tridiagonal matrix T.

 E
   E is DOUBLE PRECISION array, dimension (N)
   On entry, the first (N-1) entries contain the subdiagonal
   elements of the tridiagonal matrix T; E(N) is set to ZERO.

 INFO
   INFO is INTEGER
   INFO = 0(default) : the matrix warrants computations preserving
       relative accuracy.
   INFO = 1          : the matrix warrants computations guaranteeing
       only absolute accuracy.
```

———-

90.27.3 fortran code

— dlarrr.f —

```
*  =====================================================================
      SUBROUTINE DLARRR( N, D, E, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
```

```
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            N, INFO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), E( * )
*     ..
*
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, RELCOND
      PARAMETER          ( ZERO = 0.0D0,
     $                     RELCOND = 0.999D0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      LOGICAL            YESREL
      DOUBLE PRECISION   EPS, SAFMIN, SMLNUM, RMIN, TMP, TMP2,
     $          OFFDIG, OFFDIG2

*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS
*     ..
*     .. Executable Statements ..
*
*     As a default, do NOT go for relative-accuracy preserving computations.
      INFO = 1

      SAFMIN = DLAMCH( 'Safe minimum' )
      EPS = DLAMCH( 'Precision' )
      SMLNUM = SAFMIN / EPS
      RMIN = SQRT( SMLNUM )

*     Tests for relative accuracy
*
*     Test for scaled diagonal dominance
*     Scale the diagonal entries to one and check whether the sum of the
*     off-diagonals is less than one
*
*     The sdd relative error bounds have a 1/(1- 2*x) factor in them,
*     x = max(OFFDIG + OFFDIG2), so when x is close to 1/2, no relative
```

```
*     accuracy is promised.  In the notation of the code fragment below,
*     1/(1 - (OFFDIG + OFFDIG2)) is the condition number.
*     We don't think it is worth going into "sdd mode" unless the relative
*     condition number is reasonable, not 1/macheps.
*     The threshold should be compatible with other thresholds used in the
*     code. We set  OFFDIG + OFFDIG2 <= .999 =: RELCOND, it corresponds
*     to losing at most 3 decimal digits: 1 / (1 - (OFFDIG + OFFDIG2)) <= 1000
*     instead of the current OFFDIG + OFFDIG2 < 1
*
      YESREL = .TRUE.
      OFFDIG = ZERO
      TMP = SQRT(ABS(D(1)))
      IF (TMP.LT.RMIN) YESREL = .FALSE.
      IF(.NOT.YESREL) GOTO 11
      DO 10 I = 2, N
         TMP2 = SQRT(ABS(D(I)))
         IF (TMP2.LT.RMIN) YESREL = .FALSE.
         IF(.NOT.YESREL) GOTO 11
         OFFDIG2 = ABS(E(I-1))/(TMP*TMP2)
         IF(OFFDIG+OFFDIG2.GE.RELCOND) YESREL = .FALSE.
         IF(.NOT.YESREL) GOTO 11
         TMP = TMP2
         OFFDIG = OFFDIG2
 10   CONTINUE
 11   CONTINUE

      IF( YESREL ) THEN
         INFO = 0
         RETURN
      ELSE
      ENDIF
*

*
*     *** MORE TO BE IMPLEMENTED ***
*

*
*     Test if the lower bidiagonal matrix L from T = L D L^T
*     (zero shift facto) is well conditioned
*

*
*     Test if the upper bidiagonal matrix U from T = U D U^T
*     (zero shift facto) is well conditioned.
*     In this case, the matrix needs to be flipped and, at the end
*     of the eigenvector computation, the flip needs to be applied
*     to the computed eigenvectors (and the support)
*
```

```
*
      RETURN
*
*     END OF DLARRR
*
      END
```

———-

90.27.4 lisp code

— **dlarrr.lisp** —

```
(let* ((zero 0.0d0) (relcond 0.999d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 0.999d0 0.999d0) relcond)
           (ignorable zero relcond))
  (defun dlarrr (n d e info)
    (declare (type (array double-float (*)) e d)
             (type (f2cl-lib:integer4) info n))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%))
      (prog ((eps 0.0d0) (safmin 0.0d0) (smlnum 0.0d0) (rmin 0.0d0) (tmp 0.0d0)
             (tmp2 0.0d0) (offdig 0.0d0) (offdig2 0.0d0) (yesrel nil) (i 0))
        (declare
         (type (double-float) eps safmin smlnum rmin tmp tmp2 offdig offdig2)
         (type f2cl-lib:logical yesrel)
         (type (f2cl-lib:integer4) i))
        (setf info 1)
        (setf safmin (dlamch "Safe minimum"))
        (setf eps (dlamch "Precision"))
        (setf smlnum (/ safmin eps))
        (setf rmin (f2cl-lib:fsqrt smlnum))
        (setf yesrel f2cl-lib:%true%)
        (setf offdig zero)
        (setf tmp
                (f2cl-lib:fsqrt
                 (abs (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))))
        (if (< tmp rmin)
            (setf yesrel f2cl-lib:%false%))
        (if (not yesrel)
            (go label11))
        (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf tmp2
                    (f2cl-lib:fsqrt
```

```
                 (abs (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))))
          (if (< tmp2 rmin)
              (setf yesrel f2cl-lib:%false%))
          (if (not yesrel)
              (go label11))
          (setf offdig2
                  (/
                   (abs
                    (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1)) ((1 *))
                                   e-%offset%))
                   (* tmp tmp2)))
          (if (>= (+ offdig offdig2) relcond)
              (setf yesrel f2cl-lib:%false%))
          (if (not yesrel)
              (go label11))
          (setf tmp tmp2)
          (setf offdig offdig2)
         label10))
      label11
       (cond (yesrel (setf info 0) (go end_label)) (t))
       (go end_label)
      end_label
       (return (values nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlarrr
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

90.28 dlartg

90.28.1 Axiom unit tests

— dlartg.input —

```
)set break resume
)sys rm -f dlartg.output
)spool dlartg.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.28.2 Axiom help page

— dlartg.help —

```
======================================================================
dlartg examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlartg (F, G, CS, SN, R)
           DLARTG generates a plane rotation with real cosine and real sine.

Function/Subroutine Documentation
   subroutine dlartg (double precision F, double precision G, double precision
       CS, double precision SN, double precision R)
       DLARTG generates a plane rotation with real cosine and real sine.

Purpose:
   DLARTG generate a plane rotation so that

      [  CS  SN  ]  .  [ F ]  =  [ R ]   where CS**2 + SN**2 = 1.
      [ -SN  CS  ]     [ G ]     [ 0 ]
```

```
   This is a slower, more accurate version of the BLAS1 routine DROTG,
   with the following other differences:
    F and G are unchanged on return.
     If G=0, then CS=1 and SN=0.
     If F=0 and (G .ne. 0), then CS=0 and SN=1 without doing any
   floating point operations (saves work in DBDSQR when
   there are zeros on the diagonal).

   If F exceeds G in magnitude, CS will be positive.

Parameters:

 F
   F is DOUBLE PRECISION
   The first component of vector to be rotated.

 G
   G is DOUBLE PRECISION
   The second component of vector to be rotated.

 CS
   CS is DOUBLE PRECISION
   The cosine of the rotation.

 SN
   SN is DOUBLE PRECISION
   The sine of the rotation.

 R
   R is DOUBLE PRECISION
   The nonzero component of the rotated vector.

   This version has a few statements commented out for thread safety
   (machine parameters are computed on each entry). 10 feb 03, SJH.
```

———-

90.28.3 fortran code

— dlartg.f —

```
*  =====================================================================
      SUBROUTINE DLARTG( F, G, CS, SN, R )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
```

```
*     September 2012
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   CS, F, G, R, SN
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D0 )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D0 )
*     ..
*     .. Local Scalars ..
*     LOGICAL            FIRST
      INTEGER            COUNT, I
      DOUBLE PRECISION   EPS, F1, G1, SAFMIN, SAFMN2, SAFMX2, SCALE
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, INT, LOG, MAX, SQRT
*     ..
*     .. Save statement ..
*     SAVE               FIRST, SAFMX2, SAFMIN, SAFMN2
*     ..
*     .. Data statements ..
*     DATA               FIRST / .TRUE. /
*     ..
*     .. Executable Statements ..
*
*     IF( FIRST ) THEN
         SAFMIN = DLAMCH( 'S' )
         EPS = DLAMCH( 'E' )
         SAFMN2 = DLAMCH( 'B' )**INT( LOG( SAFMIN / EPS ) /
     $           LOG( DLAMCH( 'B' ) ) / TWO )
         SAFMX2 = ONE / SAFMN2
*        FIRST = .FALSE.
*     END IF
      IF( G.EQ.ZERO ) THEN
         CS = ONE
         SN = ZERO
         R = F
      ELSE IF( F.EQ.ZERO ) THEN
         CS = ZERO
```

```
         SN = ONE
         R = G
      ELSE
         F1 = F
         G1 = G
         SCALE = MAX( ABS( F1 ), ABS( G1 ) )
         IF( SCALE.GE.SAFMX2 ) THEN
            COUNT = 0
   10       CONTINUE
            COUNT = COUNT + 1
            F1 = F1*SAFMN2
            G1 = G1*SAFMN2
            SCALE = MAX( ABS( F1 ), ABS( G1 ) )
            IF( SCALE.GE.SAFMX2 )
     $         GO TO 10
            R = SQRT( F1**2+G1**2 )
            CS = F1 / R
            SN = G1 / R
            DO 20 I = 1, COUNT
               R = R*SAFMX2
   20       CONTINUE
         ELSE IF( SCALE.LE.SAFMN2 ) THEN
            COUNT = 0
   30       CONTINUE
            COUNT = COUNT + 1
            F1 = F1*SAFMX2
            G1 = G1*SAFMX2
            SCALE = MAX( ABS( F1 ), ABS( G1 ) )
            IF( SCALE.LE.SAFMN2 )
     $         GO TO 30
            R = SQRT( F1**2+G1**2 )
            CS = F1 / R
            SN = G1 / R
            DO 40 I = 1, COUNT
               R = R*SAFMN2
   40       CONTINUE
         ELSE
            R = SQRT( F1**2+G1**2 )
            CS = F1 / R
            SN = G1 / R
         END IF
         IF( ABS( F ).GT.ABS( G ) .AND. CS.LT.ZERO ) THEN
            CS = -CS
            SN = -SN
            R = -R
         END IF
      END IF
      RETURN
*
*     End of DLARTG
```

```
*
      END
```

———-

90.28.4 lisp code

— **dlartg.lisp** —

———-

90.29 dlartgp

90.29.1 Axiom unit tests

— **dlartgp.input** —

```
)set break resume
)sys rm -f dlartgp.output
)spool dlartgp.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.29.2 Axiom help page

— **dlartgp.help** —

```
======================================================================
dlartgp examples
======================================================================

======================================================================
Man Page Details
======================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine dlartgp (F, G, CS, SN, R)
           DLARTGP generates a plane rotation so that the diagonal is
           nonnegative.

Function/Subroutine Documentation
   subroutine dlartgp (double precision F, double precision G, double
       precision CS, double precision SN, double precision R)
       DLARTGP generates a plane rotation so that the diagonal is nonnegative.

Purpose:
   DLARTGP generates a plane rotation so that

   [  CS  SN  ]  .  [ F ]  =  [ R ]   where CS**2 + SN**2 = 1.
   [ -SN  CS  ]     [ G ]     [ 0 ]

   This is a slower, more accurate version of the Level 1 BLAS routine DROTG,
   with the following other differences:
    F and G are unchanged on return.
     If G=0, then CS=(+/-)1 and SN=0.
     If F=0 and (G .ne. 0), then CS=0 and SN=(+/-)1.

   The sign is chosen so that R >= 0.

Parameters:

 F
   F is DOUBLE PRECISION
   The first component of vector to be rotated.

 G
   G is DOUBLE PRECISION
   The second component of vector to be rotated.

 CS
   CS is DOUBLE PRECISION
   The cosine of the rotation.

 SN
   SN is DOUBLE PRECISION
   The sine of the rotation.

 R
   R is DOUBLE PRECISION
   The nonzero component of the rotated vector.

   This version has a few statements commented out for thread safety
   (machine parameters are computed on each entry). 10 feb 03, SJH.
```

———-

90.29.3 fortran code

— dlartgp.f —

```
*  =====================================================================
      SUBROUTINE DLARTGP( F, G, CS, SN, R )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   CS, F, G, R, SN
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D0 )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D0 )
*     ..
*     .. Local Scalars ..
*     LOGICAL            FIRST
      INTEGER            COUNT, I
      DOUBLE PRECISION   EPS, F1, G1, SAFMIN, SAFMN2, SAFMX2, SCALE
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, INT, LOG, MAX, SIGN, SQRT
*     ..
*     .. Save statement ..
*     SAVE               FIRST, SAFMX2, SAFMIN, SAFMN2
*     ..
*     .. Data statements ..
*     DATA               FIRST / .TRUE. /
*     ..
```

```
*     .. Executable Statements ..
*
*     IF( FIRST ) THEN
         SAFMIN = DLAMCH( 'S' )
         EPS = DLAMCH( 'E' )
         SAFMN2 = DLAMCH( 'B' )**INT( LOG( SAFMIN / EPS ) /
     $           LOG( DLAMCH( 'B' ) ) / TWO )
         SAFMX2 = ONE / SAFMN2
*        FIRST = .FALSE.
*     END IF
      IF( G.EQ.ZERO ) THEN
         CS = SIGN( ONE, F )
         SN = ZERO
         R = ABS( F )
      ELSE IF( F.EQ.ZERO ) THEN
         CS = ZERO
         SN = SIGN( ONE, G )
         R = ABS( G )
      ELSE
         F1 = F
         G1 = G
         SCALE = MAX( ABS( F1 ), ABS( G1 ) )
         IF( SCALE.GE.SAFMX2 ) THEN
            COUNT = 0
   10       CONTINUE
            COUNT = COUNT + 1
            F1 = F1*SAFMN2
            G1 = G1*SAFMN2
            SCALE = MAX( ABS( F1 ), ABS( G1 ) )
            IF( SCALE.GE.SAFMX2 )
     $         GO TO 10
            R = SQRT( F1**2+G1**2 )
            CS = F1 / R
            SN = G1 / R
            DO 20 I = 1, COUNT
               R = R*SAFMX2
   20       CONTINUE
         ELSE IF( SCALE.LE.SAFMN2 ) THEN
            COUNT = 0
   30       CONTINUE
            COUNT = COUNT + 1
            F1 = F1*SAFMX2
            G1 = G1*SAFMX2
            SCALE = MAX( ABS( F1 ), ABS( G1 ) )
            IF( SCALE.LE.SAFMN2 )
     $         GO TO 30
            R = SQRT( F1**2+G1**2 )
            CS = F1 / R
            SN = G1 / R
            DO 40 I = 1, COUNT
```

```
               R = R*SAFMN2
   40       CONTINUE
         ELSE
            R = SQRT( F1**2+G1**2 )
            CS = F1 / R
            SN = G1 / R
         END IF
         IF( R.LT.ZERO ) THEN
            CS = -CS
            SN = -SN
            R = -R
         END IF
      END IF
      RETURN
*
*     End of DLARTGP
*
      END
```

———-

90.29.4 lisp code

— **dlartgp.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0) (two 2.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (ignorable zero one two))
  (defun dlartgp (f g cs sn r)
    (declare (type (double-float) r sn cs g f))
    (prog ((eps 0.0d0) (f1 0.0d0) (g1 0.0d0) (safmin 0.0d0) (safmn2 0.0d0)
           (safmx2 0.0d0) (scale 0.0d0) (i 0) (count$ 0))
      (declare (type (double-float) eps f1 g1 safmin safmn2 safmx2 scale)
               (type (f2cl-lib:integer4) count$ i))
      (setf safmin (dlamch "S"))
      (setf eps (dlamch "E"))
      (setf safmn2
              (expt (dlamch "B")
                    (f2cl-lib:int
                     (/
                      (/ (f2cl-lib:flog (/ safmin eps))
                         (f2cl-lib:flog (dlamch "B")))
                      two))))
      (setf safmx2 (/ one safmn2))
      (cond
       ((= g zero) (setf cs (f2cl-lib:sign one f)) (setf sn zero)
```

```
        (setf r (abs f)))
       ((= f zero) (setf cs zero) (setf sn (f2cl-lib:sign one g))
        (setf r (abs g)))
       (t (setf f1 f) (setf g1 g) (setf scale (max (abs f1) (abs g1)))
        (cond
         ((>= scale safmx2)
          (tagbody
            (setf count$ 0)
           label10
            (setf count$ (f2cl-lib:int-add count$ 1))
            (setf f1 (* f1 safmn2))
            (setf g1 (* g1 safmn2))
            (setf scale (max (abs f1) (abs g1)))
            (if (>= scale safmx2)
                (go label10))
            (setf r (f2cl-lib:fsqrt (+ (expt f1 2) (expt g1 2))))
            (setf cs (/ f1 r))
            (setf sn (/ g1 r))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i count$) nil)
              (tagbody (setf r (* r safmx2)) label20))))
         ((<= scale safmn2)
          (tagbody
            (setf count$ 0)
           label30
            (setf count$ (f2cl-lib:int-add count$ 1))
            (setf f1 (* f1 safmx2))
            (setf g1 (* g1 safmx2))
            (setf scale (max (abs f1) (abs g1)))
            (if (<= scale safmn2)
                (go label30))
            (setf r (f2cl-lib:fsqrt (+ (expt f1 2) (expt g1 2))))
            (setf cs (/ f1 r))
            (setf sn (/ g1 r))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i count$) nil)
              (tagbody (setf r (* r safmn2)) label40))))
         (t (setf r (f2cl-lib:fsqrt (+ (expt f1 2) (expt g1 2))))
          (setf cs (/ f1 r)) (setf sn (/ g1 r))))
        (cond ((< r zero) (setf cs (- cs)) (setf sn (- sn)) (setf r (- r))))))
      (go end_label)
     end_label
      (return (values nil nil cs sn r)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlartgp
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
```

```
'((double-float) (double-float)
  (double-float) (double-float)
  (double-float))
:return-values
'(nil nil fortran-to-lisp::cs
  fortran-to-lisp::sn
  fortran-to-lisp::r)
:calls 'nil)))
```

———-

90.30 dlaruv

90.30.1 Axiom unit tests

— **dlaruv.input** —

```
)set break resume
)sys rm -f dlaruv.output
)spool dlaruv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.30.2 Axiom help page

— **dlaruv.help** —

```
====================================================================
dlaruv examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
```

```
   Functions/Subroutines
       subroutine dlaruv (ISEED, N, X)
           DLARUV returns a vector of n random real numbers from a uniform
           distribution.

Function/Subroutine Documentation
   subroutine dlaruv (integer, dimension( 4 ) ISEED, integer N, double
       precision, dimension( n ) X)
       DLARUV returns a vector of n random real numbers from a uniform
       distribution.

Purpose:
   DLARUV returns a vector of n random real numbers from a uniform (0,1)
   distribution (n <= 128).

   This is an auxiliary routine called by DLARNV and ZLARNV.

Parameters:

 ISEED
   ISEED is INTEGER array, dimension (4)
   On entry, the seed of the random number generator; the array
   elements must be between 0 and 4095, and ISEED(4) must be odd.
   On exit, the seed is updated.

 N
   N is INTEGER
   The number of random numbers to be generated. N <= 128.

 X
   X is DOUBLE PRECISION array, dimension (N)
   The generated random numbers.

Further Details:

   This routine uses a multiplicative congruential method with modulus
   2**48 and multiplier 33952834046453 (see G.S.Fishman,
   'Multiplicative congruential random number generators with modulus
   2**b: an exhaustive analysis for b = 32 and a partial analysis for
   b = 48', Math. Comp. 189, pp 331-344, 1990).

   48-bit integers are stored in 4 integer array elements with 12 bits
   per element. Hence the routine is portable across machines with
   integers of 32 bits or more.
```

90.30.3 fortran code

— dlaruv.f —

```
*  =====================================================================
      SUBROUTINE DLARUV( ISEED, N, X )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            N
*     ..
*     .. Array Arguments ..
      INTEGER            ISEED( 4 )
      DOUBLE PRECISION   X( N )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D0 )
      INTEGER            LV, IPW2
      DOUBLE PRECISION   R
      PARAMETER          ( LV = 128, IPW2 = 4096, R = ONE / IPW2 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, I1, I2, I3, I4, IT1, IT2, IT3, IT4, J
*     ..
*     .. Local Arrays ..
      INTEGER            MM( LV, 4 )
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, MIN, MOD
*     ..
*     .. Data statements ..
      DATA               ( MM( 1, J ), J = 1, 4 ) / 494, 322, 2508,
     $                   2549 /
      DATA               ( MM( 2, J ), J = 1, 4 ) / 2637, 789, 3754,
     $                   1145 /
      DATA               ( MM( 3, J ), J = 1, 4 ) / 255, 1440, 1766,
     $                   2253 /
      DATA               ( MM( 4, J ), J = 1, 4 ) / 2008, 752, 3572,
     $                   305 /
      DATA               ( MM( 5, J ), J = 1, 4 ) / 1253, 2859, 2893,
     $                   3301 /
```

```
      DATA              ( MM( 6, J ), J = 1, 4 ) / 3344, 123, 307,
     $                  1065 /
      DATA              ( MM( 7, J ), J = 1, 4 ) / 4084, 1848, 1297,
     $                  3133 /
      DATA              ( MM( 8, J ), J = 1, 4 ) / 1739, 643, 3966,
     $                  2913 /
      DATA              ( MM( 9, J ), J = 1, 4 ) / 3143, 2405, 758,
     $                  3285 /
      DATA              ( MM( 10, J ), J = 1, 4 ) / 3468, 2638, 2598,
     $                  1241 /
      DATA              ( MM( 11, J ), J = 1, 4 ) / 688, 2344, 3406,
     $                  1197 /
      DATA              ( MM( 12, J ), J = 1, 4 ) / 1657, 46, 2922,
     $                  3729 /
      DATA              ( MM( 13, J ), J = 1, 4 ) / 1238, 3814, 1038,
     $                  2501 /
      DATA              ( MM( 14, J ), J = 1, 4 ) / 3166, 913, 2934,
     $                  1673 /
      DATA              ( MM( 15, J ), J = 1, 4 ) / 1292, 3649, 2091,
     $                  541 /
      DATA              ( MM( 16, J ), J = 1, 4 ) / 3422, 339, 2451,
     $                  2753 /
      DATA              ( MM( 17, J ), J = 1, 4 ) / 1270, 3808, 1580,
     $                  949 /
      DATA              ( MM( 18, J ), J = 1, 4 ) / 2016, 822, 1958,
     $                  2361 /
      DATA              ( MM( 19, J ), J = 1, 4 ) / 154, 2832, 2055,
     $                  1165 /
      DATA              ( MM( 20, J ), J = 1, 4 ) / 2862, 3078, 1507,
     $                  4081 /
      DATA              ( MM( 21, J ), J = 1, 4 ) / 697, 3633, 1078,
     $                  2725 /
      DATA              ( MM( 22, J ), J = 1, 4 ) / 1706, 2970, 3273,
     $                  3305 /
      DATA              ( MM( 23, J ), J = 1, 4 ) / 491, 637, 17,
     $                  3069 /
      DATA              ( MM( 24, J ), J = 1, 4 ) / 931, 2249, 854,
     $                  3617 /
      DATA              ( MM( 25, J ), J = 1, 4 ) / 1444, 2081, 2916,
     $                  3733 /
      DATA              ( MM( 26, J ), J = 1, 4 ) / 444, 4019, 3971,
     $                  409 /
      DATA              ( MM( 27, J ), J = 1, 4 ) / 3577, 1478, 2889,
     $                  2157 /
      DATA              ( MM( 28, J ), J = 1, 4 ) / 3944, 242, 3831,
     $                  1361 /
      DATA              ( MM( 29, J ), J = 1, 4 ) / 2184, 481, 2621,
     $                  3973 /
      DATA              ( MM( 30, J ), J = 1, 4 ) / 1661, 2075, 1541,
     $                  1865 /
```

```
      DATA              ( MM( 31, J ), J = 1, 4 ) / 3482, 4058, 893,
     $                  2525 /
      DATA              ( MM( 32, J ), J = 1, 4 ) / 657, 622, 736,
     $                  1409 /
      DATA              ( MM( 33, J ), J = 1, 4 ) / 3023, 3376, 3992,
     $                  3445 /
      DATA              ( MM( 34, J ), J = 1, 4 ) / 3618, 812, 787,
     $                  3577 /
      DATA              ( MM( 35, J ), J = 1, 4 ) / 1267, 234, 2125,
     $                  77 /
      DATA              ( MM( 36, J ), J = 1, 4 ) / 1828, 641, 2364,
     $                  3761 /
      DATA              ( MM( 37, J ), J = 1, 4 ) / 164, 4005, 2460,
     $                  2149 /
      DATA              ( MM( 38, J ), J = 1, 4 ) / 3798, 1122, 257,
     $                  1449 /
      DATA              ( MM( 39, J ), J = 1, 4 ) / 3087, 3135, 1574,
     $                  3005 /
      DATA              ( MM( 40, J ), J = 1, 4 ) / 2400, 2640, 3912,
     $                  225 /
      DATA              ( MM( 41, J ), J = 1, 4 ) / 2870, 2302, 1216,
     $                  85 /
      DATA              ( MM( 42, J ), J = 1, 4 ) / 3876, 40, 3248,
     $                  3673 /
      DATA              ( MM( 43, J ), J = 1, 4 ) / 1905, 1832, 3401,
     $                  3117 /
      DATA              ( MM( 44, J ), J = 1, 4 ) / 1593, 2247, 2124,
     $                  3089 /
      DATA              ( MM( 45, J ), J = 1, 4 ) / 1797, 2034, 2762,
     $                  1349 /
      DATA              ( MM( 46, J ), J = 1, 4 ) / 1234, 2637, 149,
     $                  2057 /
      DATA              ( MM( 47, J ), J = 1, 4 ) / 3460, 1287, 2245,
     $                  413 /
      DATA              ( MM( 48, J ), J = 1, 4 ) / 328, 1691, 166,
     $                  65 /
      DATA              ( MM( 49, J ), J = 1, 4 ) / 2861, 496, 466,
     $                  1845 /
      DATA              ( MM( 50, J ), J = 1, 4 ) / 1950, 1597, 4018,
     $                  697 /
      DATA              ( MM( 51, J ), J = 1, 4 ) / 617, 2394, 1399,
     $                  3085 /
      DATA              ( MM( 52, J ), J = 1, 4 ) / 2070, 2584, 190,
     $                  3441 /
      DATA              ( MM( 53, J ), J = 1, 4 ) / 3331, 1843, 2879,
     $                  1573 /
      DATA              ( MM( 54, J ), J = 1, 4 ) / 769, 336, 153,
     $                  3689 /
      DATA              ( MM( 55, J ), J = 1, 4 ) / 1558, 1472, 2320,
     $                  2941 /
```

```
      DATA              ( MM( 56, J ), J = 1, 4 ) / 2412, 2407, 18,
     $                  929 /
      DATA              ( MM( 57, J ), J = 1, 4 ) / 2800, 433, 712,
     $                  533 /
      DATA              ( MM( 58, J ), J = 1, 4 ) / 189, 2096, 2159,
     $                  2841 /
      DATA              ( MM( 59, J ), J = 1, 4 ) / 287, 1761, 2318,
     $                  4077 /
      DATA              ( MM( 60, J ), J = 1, 4 ) / 2045, 2810, 2091,
     $                  721 /
      DATA              ( MM( 61, J ), J = 1, 4 ) / 1227, 566, 3443,
     $                  2821 /
      DATA              ( MM( 62, J ), J = 1, 4 ) / 2838, 442, 1510,
     $                  2249 /
      DATA              ( MM( 63, J ), J = 1, 4 ) / 209, 41, 449,
     $                  2397 /
      DATA              ( MM( 64, J ), J = 1, 4 ) / 2770, 1238, 1956,
     $                  2817 /
      DATA              ( MM( 65, J ), J = 1, 4 ) / 3654, 1086, 2201,
     $                  245 /
      DATA              ( MM( 66, J ), J = 1, 4 ) / 3993, 603, 3137,
     $                  1913 /
      DATA              ( MM( 67, J ), J = 1, 4 ) / 192, 840, 3399,
     $                  1997 /
      DATA              ( MM( 68, J ), J = 1, 4 ) / 2253, 3168, 1321,
     $                  3121 /
      DATA              ( MM( 69, J ), J = 1, 4 ) / 3491, 1499, 2271,
     $                  997 /
      DATA              ( MM( 70, J ), J = 1, 4 ) / 2889, 1084, 3667,
     $                  1833 /
      DATA              ( MM( 71, J ), J = 1, 4 ) / 2857, 3438, 2703,
     $                  2877 /
      DATA              ( MM( 72, J ), J = 1, 4 ) / 2094, 2408, 629,
     $                  1633 /
      DATA              ( MM( 73, J ), J = 1, 4 ) / 1818, 1589, 2365,
     $                  981 /
      DATA              ( MM( 74, J ), J = 1, 4 ) / 688, 2391, 2431,
     $                  2009 /
      DATA              ( MM( 75, J ), J = 1, 4 ) / 1407, 288, 1113,
     $                  941 /
      DATA              ( MM( 76, J ), J = 1, 4 ) / 634, 26, 3922,
     $                  2449 /
      DATA              ( MM( 77, J ), J = 1, 4 ) / 3231, 512, 2554,
     $                  197 /
      DATA              ( MM( 78, J ), J = 1, 4 ) / 815, 1456, 184,
     $                  2441 /
      DATA              ( MM( 79, J ), J = 1, 4 ) / 3524, 171, 2099,
     $                  285 /
      DATA              ( MM( 80, J ), J = 1, 4 ) / 1914, 1677, 3228,
     $                  1473 /
```

```
      DATA              ( MM( 81, J ), J = 1, 4 ) / 516, 2657, 4012,
     $                  2741 /
      DATA              ( MM( 82, J ), J = 1, 4 ) / 164, 2270, 1921,
     $                  3129 /
      DATA              ( MM( 83, J ), J = 1, 4 ) / 303, 2587, 3452,
     $                  909 /
      DATA              ( MM( 84, J ), J = 1, 4 ) / 2144, 2961, 3901,
     $                  2801 /
      DATA              ( MM( 85, J ), J = 1, 4 ) / 3480, 1970, 572,
     $                  421 /
      DATA              ( MM( 86, J ), J = 1, 4 ) / 119, 1817, 3309,
     $                  4073 /
      DATA              ( MM( 87, J ), J = 1, 4 ) / 3357, 676, 3171,
     $                  2813 /
      DATA              ( MM( 88, J ), J = 1, 4 ) / 837, 1410, 817,
     $                  2337 /
      DATA              ( MM( 89, J ), J = 1, 4 ) / 2826, 3723, 3039,
     $                  1429 /
      DATA              ( MM( 90, J ), J = 1, 4 ) / 2332, 2803, 1696,
     $                  1177 /
      DATA              ( MM( 91, J ), J = 1, 4 ) / 2089, 3185, 1256,
     $                  1901 /
      DATA              ( MM( 92, J ), J = 1, 4 ) / 3780, 184, 3715,
     $                  81 /
      DATA              ( MM( 93, J ), J = 1, 4 ) / 1700, 663, 2077,
     $                  1669 /
      DATA              ( MM( 94, J ), J = 1, 4 ) / 3712, 499, 3019,
     $                  2633 /
      DATA              ( MM( 95, J ), J = 1, 4 ) / 150, 3784, 1497,
     $                  2269 /
      DATA              ( MM( 96, J ), J = 1, 4 ) / 2000, 1631, 1101,
     $                  129 /
      DATA              ( MM( 97, J ), J = 1, 4 ) / 3375, 1925, 717,
     $                  1141 /
      DATA              ( MM( 98, J ), J = 1, 4 ) / 1621, 3912, 51,
     $                  249 /
      DATA              ( MM( 99, J ), J = 1, 4 ) / 3090, 1398, 981,
     $                  3917 /
      DATA              ( MM( 100, J ), J = 1, 4 ) / 3765, 1349, 1978,
     $                  2481 /
      DATA              ( MM( 101, J ), J = 1, 4 ) / 1149, 1441, 1813,
     $                  3941 /
      DATA              ( MM( 102, J ), J = 1, 4 ) / 3146, 2224, 3881,
     $                  2217 /
      DATA              ( MM( 103, J ), J = 1, 4 ) / 33, 2411, 76,
     $                  2749 /
      DATA              ( MM( 104, J ), J = 1, 4 ) / 3082, 1907, 3846,
     $                  3041 /
      DATA              ( MM( 105, J ), J = 1, 4 ) / 2741, 3192, 3694,
     $                  1877 /
```

```
      DATA              ( MM( 106, J ), J = 1, 4 ) / 359, 2786, 1682,
     $                  345 /
      DATA              ( MM( 107, J ), J = 1, 4 ) / 3316, 382, 124,
     $                  2861 /
      DATA              ( MM( 108, J ), J = 1, 4 ) / 1749, 37, 1660,
     $                  1809 /
      DATA              ( MM( 109, J ), J = 1, 4 ) / 185, 759, 3997,
     $                  3141 /
      DATA              ( MM( 110, J ), J = 1, 4 ) / 2784, 2948, 479,
     $                  2825 /
      DATA              ( MM( 111, J ), J = 1, 4 ) / 2202, 1862, 1141,
     $                  157 /
      DATA              ( MM( 112, J ), J = 1, 4 ) / 2199, 3802, 886,
     $                  2881 /
      DATA              ( MM( 113, J ), J = 1, 4 ) / 1364, 2423, 3514,
     $                  3637 /
      DATA              ( MM( 114, J ), J = 1, 4 ) / 1244, 2051, 1301,
     $                  1465 /
      DATA              ( MM( 115, J ), J = 1, 4 ) / 2020, 2295, 3604,
     $                  2829 /
      DATA              ( MM( 116, J ), J = 1, 4 ) / 3160, 1332, 1888,
     $                  2161 /
      DATA              ( MM( 117, J ), J = 1, 4 ) / 2785, 1832, 1836,
     $                  3365 /
      DATA              ( MM( 118, J ), J = 1, 4 ) / 2772, 2405, 1990,
     $                  361 /
      DATA              ( MM( 119, J ), J = 1, 4 ) / 1217, 3638, 2058,
     $                  2685 /
      DATA              ( MM( 120, J ), J = 1, 4 ) / 1822, 3661, 692,
     $                  3745 /
      DATA              ( MM( 121, J ), J = 1, 4 ) / 1245, 327, 1194,
     $                  2325 /
      DATA              ( MM( 122, J ), J = 1, 4 ) / 2252, 3660, 20,
     $                  3609 /
      DATA              ( MM( 123, J ), J = 1, 4 ) / 3904, 716, 3285,
     $                  3821 /
      DATA              ( MM( 124, J ), J = 1, 4 ) / 2774, 1842, 2046,
     $                  3537 /
      DATA              ( MM( 125, J ), J = 1, 4 ) / 997, 3987, 2107,
     $                  517 /
      DATA              ( MM( 126, J ), J = 1, 4 ) / 2573, 1368, 3508,
     $                  3017 /
      DATA              ( MM( 127, J ), J = 1, 4 ) / 1148, 1848, 3525,
     $                  2141 /
      DATA              ( MM( 128, J ), J = 1, 4 ) / 545, 2366, 3801,
     $                  1537 /
*     ..
*     .. Executable Statements ..
*
      I1 = ISEED( 1 )
```

```
      I2 = ISEED( 2 )
      I3 = ISEED( 3 )
      I4 = ISEED( 4 )
*
      DO 10 I = 1, MIN( N, LV )
*
  20     CONTINUE
*
*        Multiply the seed by i-th power of the multiplier modulo 2**48
*
         IT4 = I4*MM( I, 4 )
         IT3 = IT4 / IPW2
         IT4 = IT4 - IPW2*IT3
         IT3 = IT3 + I3*MM( I, 4 ) + I4*MM( I, 3 )
         IT2 = IT3 / IPW2
         IT3 = IT3 - IPW2*IT2
         IT2 = IT2 + I2*MM( I, 4 ) + I3*MM( I, 3 ) + I4*MM( I, 2 )
         IT1 = IT2 / IPW2
         IT2 = IT2 - IPW2*IT1
         IT1 = IT1 + I1*MM( I, 4 ) + I2*MM( I, 3 ) + I3*MM( I, 2 ) +
     $         I4*MM( I, 1 )
         IT1 = MOD( IT1, IPW2 )
*
*        Convert 48-bit integer to a real number in the interval (0,1)
*
         X( I ) = R*( DBLE( IT1 )+R*( DBLE( IT2 )+R*( DBLE( IT3 )+R*
     $           DBLE( IT4 ) ) ) )
*
         IF (X( I ).EQ.1.0D0) THEN
*           If a real number has n bits of precision, and the first
*           n bits of the 48-bit integer above happen to be all 1 (which
*           will occur about once every 2**n calls), then X( I ) will
*           be rounded to exactly 1.0.
*           Since X( I ) is not supposed to return exactly 0.0 or 1.0,
*           the statistically correct thing to do in this situation is
*           simply to iterate again.
*           N.B. the case X( I ) = 0.0 should not be possible.
            I1 = I1 + 2
            I2 = I2 + 2
            I3 = I3 + 2
            I4 = I4 + 2
            GOTO 20
         END IF
*
   10 CONTINUE
*
*     Return final value of seed
*
      ISEED( 1 ) = IT1
      ISEED( 2 ) = IT2
```

```
      ISEED( 3 ) = IT3
      ISEED( 4 ) = IT4
      RETURN
*
*     End of DLARUV
*
      END
```

———-

90.30.4 lisp code

— **dlaruv.lisp** —

```
(let* ((one 1.0d0) (lv 128) (ipw2 4096) (r (f2cl-lib:f2cl/ one ipw2)))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (f2cl-lib:integer4 128 128) lv)
           (type (f2cl-lib:integer4 4096 4096) ipw2)
           (type (double-float) r)
           (ignorable one lv ipw2 r))
  (let ((mm
         (make-array (the fixnum (reduce #'* (list lv 4))) :element-type
                     'f2cl-lib:integer4)))
    (declare (type (array f2cl-lib:integer4 (*)) mm))
    (f2cl-lib:data-implied-do (((mm (1 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (494 322 2508 2549))
    (f2cl-lib:data-implied-do (((mm (2 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (2637 789 3754 1145))
    (f2cl-lib:data-implied-do (((mm (3 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (255 1440 1766 2253))
    (f2cl-lib:data-implied-do (((mm (4 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (2008 752 3572 305))
    (f2cl-lib:data-implied-do (((mm (5 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (1253 2859 2893 3301))
    (f2cl-lib:data-implied-do (((mm (6 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (3344 123 307 1065))
    (f2cl-lib:data-implied-do (((mm (7 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (4084 1848 1297 3133))
    (f2cl-lib:data-implied-do (((mm (8 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (1739 643 3966 2913))
    (f2cl-lib:data-implied-do (((mm (9 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (3143 2405 758 3285))
    (f2cl-lib:data-implied-do (((mm (10 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (3468 2638 2598 1241))
    (f2cl-lib:data-implied-do (((mm (11 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (688 2344 3406 1197))
    (f2cl-lib:data-implied-do (((mm (12 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (1657 46 2922 3729))
```

```
(f2cl-lib:data-implied-do (((mm (13 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1238 3814 1038 2501))
(f2cl-lib:data-implied-do (((mm (14 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3166 913 2934 1673))
(f2cl-lib:data-implied-do (((mm (15 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1292 3649 2091 541))
(f2cl-lib:data-implied-do (((mm (16 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3422 339 2451 2753))
(f2cl-lib:data-implied-do (((mm (17 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1270 3808 1580 949))
(f2cl-lib:data-implied-do (((mm (18 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2016 822 1958 2361))
(f2cl-lib:data-implied-do (((mm (19 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (154 2832 2055 1165))
(f2cl-lib:data-implied-do (((mm (20 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2862 3078 1507 4081))
(f2cl-lib:data-implied-do (((mm (21 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (697 3633 1078 2725))
(f2cl-lib:data-implied-do (((mm (22 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1706 2970 3273 3305))
(f2cl-lib:data-implied-do (((mm (23 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (491 637 17 3069))
(f2cl-lib:data-implied-do (((mm (24 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (931 2249 854 3617))
(f2cl-lib:data-implied-do (((mm (25 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1444 2081 2916 3733))
(f2cl-lib:data-implied-do (((mm (26 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (444 4019 3971 409))
(f2cl-lib:data-implied-do (((mm (27 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3577 1478 2889 2157))
(f2cl-lib:data-implied-do (((mm (28 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3944 242 3831 1361))
(f2cl-lib:data-implied-do (((mm (29 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2184 481 2621 3973))
(f2cl-lib:data-implied-do (((mm (30 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1661 2075 1541 1865))
(f2cl-lib:data-implied-do (((mm (31 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3482 4058 893 2525))
(f2cl-lib:data-implied-do (((mm (32 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (657 622 736 1409))
(f2cl-lib:data-implied-do (((mm (33 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3023 3376 3992 3445))
(f2cl-lib:data-implied-do (((mm (34 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3618 812 787 3577))
(f2cl-lib:data-implied-do (((mm (35 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1267 234 2125 77))
(f2cl-lib:data-implied-do (((mm (36 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1828 641 2364 3761))
(f2cl-lib:data-implied-do (((mm (37 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (164 4005 2460 2149))
```

```
(f2cl-lib:data-implied-do (((mm (38 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3798 1122 257 1449))
(f2cl-lib:data-implied-do (((mm (39 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3087 3135 1574 3005))
(f2cl-lib:data-implied-do (((mm (40 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2400 2640 3912 225))
(f2cl-lib:data-implied-do (((mm (41 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2870 2302 1216 85))
(f2cl-lib:data-implied-do (((mm (42 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3876 40 3248 3673))
(f2cl-lib:data-implied-do (((mm (43 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1905 1832 3401 3117))
(f2cl-lib:data-implied-do (((mm (44 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1593 2247 2124 3089))
(f2cl-lib:data-implied-do (((mm (45 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1797 2034 2762 1349))
(f2cl-lib:data-implied-do (((mm (46 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1234 2637 149 2057))
(f2cl-lib:data-implied-do (((mm (47 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3460 1287 2245 413))
(f2cl-lib:data-implied-do (((mm (48 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (328 1691 166 65))
(f2cl-lib:data-implied-do (((mm (49 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2861 496 466 1845))
(f2cl-lib:data-implied-do (((mm (50 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1950 1597 4018 697))
(f2cl-lib:data-implied-do (((mm (51 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (617 2394 1399 3085))
(f2cl-lib:data-implied-do (((mm (52 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2070 2584 190 3441))
(f2cl-lib:data-implied-do (((mm (53 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3331 1843 2879 1573))
(f2cl-lib:data-implied-do (((mm (54 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (769 336 153 3689))
(f2cl-lib:data-implied-do (((mm (55 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1558 1472 2320 2941))
(f2cl-lib:data-implied-do (((mm (56 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2412 2407 18 929))
(f2cl-lib:data-implied-do (((mm (57 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2800 433 712 533))
(f2cl-lib:data-implied-do (((mm (58 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (189 2096 2159 2841))
(f2cl-lib:data-implied-do (((mm (59 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (287 1761 2318 4077))
(f2cl-lib:data-implied-do (((mm (60 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2045 2810 2091 721))
(f2cl-lib:data-implied-do (((mm (61 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1227 566 3443 2821))
(f2cl-lib:data-implied-do (((mm (62 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2838 442 1510 2249))
```

```
(f2cl-lib:data-implied-do (((mm (63 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (209 41 449 2397))
(f2cl-lib:data-implied-do (((mm (64 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2770 1238 1956 2817))
(f2cl-lib:data-implied-do (((mm (65 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3654 1086 2201 245))
(f2cl-lib:data-implied-do (((mm (66 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3993 603 3137 1913))
(f2cl-lib:data-implied-do (((mm (67 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (192 840 3399 1997))
(f2cl-lib:data-implied-do (((mm (68 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2253 3168 1321 3121))
(f2cl-lib:data-implied-do (((mm (69 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3491 1499 2271 997))
(f2cl-lib:data-implied-do (((mm (70 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2889 1084 3667 1833))
(f2cl-lib:data-implied-do (((mm (71 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2857 3438 2703 2877))
(f2cl-lib:data-implied-do (((mm (72 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2094 2408 629 1633))
(f2cl-lib:data-implied-do (((mm (73 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1818 1589 2365 981))
(f2cl-lib:data-implied-do (((mm (74 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (688 2391 2431 2009))
(f2cl-lib:data-implied-do (((mm (75 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1407 288 1113 941))
(f2cl-lib:data-implied-do (((mm (76 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (634 26 3922 2449))
(f2cl-lib:data-implied-do (((mm (77 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3231 512 2554 197))
(f2cl-lib:data-implied-do (((mm (78 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (815 1456 184 2441))
(f2cl-lib:data-implied-do (((mm (79 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3524 171 2099 285))
(f2cl-lib:data-implied-do (((mm (80 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1914 1677 3228 1473))
(f2cl-lib:data-implied-do (((mm (81 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (516 2657 4012 2741))
(f2cl-lib:data-implied-do (((mm (82 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (164 2270 1921 3129))
(f2cl-lib:data-implied-do (((mm (83 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (303 2587 3452 909))
(f2cl-lib:data-implied-do (((mm (84 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2144 2961 3901 2801))
(f2cl-lib:data-implied-do (((mm (85 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3480 1970 572 421))
(f2cl-lib:data-implied-do (((mm (86 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (119 1817 3309 4073))
(f2cl-lib:data-implied-do (((mm (87 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3357 676 3171 2813))
```

```
(f2cl-lib:data-implied-do (((mm (88 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (837 1410 817 2337))
(f2cl-lib:data-implied-do (((mm (89 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2826 3723 3039 1429))
(f2cl-lib:data-implied-do (((mm (90 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2332 2803 1696 1177))
(f2cl-lib:data-implied-do (((mm (91 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2089 3185 1256 1901))
(f2cl-lib:data-implied-do (((mm (92 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3780 184 3715 81))
(f2cl-lib:data-implied-do (((mm (93 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1700 663 2077 1669))
(f2cl-lib:data-implied-do (((mm (94 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3712 499 3019 2633))
(f2cl-lib:data-implied-do (((mm (95 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (150 3784 1497 2269))
(f2cl-lib:data-implied-do (((mm (96 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2000 1631 1101 129))
(f2cl-lib:data-implied-do (((mm (97 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3375 1925 717 1141))
(f2cl-lib:data-implied-do (((mm (98 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1621 3912 51 249))
(f2cl-lib:data-implied-do (((mm (99 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3090 1398 981 3917))
(f2cl-lib:data-implied-do (((mm (100 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3765 1349 1978 2481))
(f2cl-lib:data-implied-do (((mm (101 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1149 1441 1813 3941))
(f2cl-lib:data-implied-do (((mm (102 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3146 2224 3881 2217))
(f2cl-lib:data-implied-do (((mm (103 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (33 2411 76 2749))
(f2cl-lib:data-implied-do (((mm (104 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3082 1907 3846 3041))
(f2cl-lib:data-implied-do (((mm (105 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2741 3192 3694 1877))
(f2cl-lib:data-implied-do (((mm (106 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (359 2786 1682 345))
(f2cl-lib:data-implied-do (((mm (107 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3316 382 124 2861))
(f2cl-lib:data-implied-do (((mm (108 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1749 37 1660 1809))
(f2cl-lib:data-implied-do (((mm (109 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (185 759 3997 3141))
(f2cl-lib:data-implied-do (((mm (110 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2784 2948 479 2825))
(f2cl-lib:data-implied-do (((mm (111 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2202 1862 1141 157))
(f2cl-lib:data-implied-do (((mm (112 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2199 3802 886 2881))
```

```
(f2cl-lib:data-implied-do (((mm (113 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1364 2423 3514 3637))
(f2cl-lib:data-implied-do (((mm (114 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1244 2051 1301 1465))
(f2cl-lib:data-implied-do (((mm (115 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2020 2295 3604 2829))
(f2cl-lib:data-implied-do (((mm (116 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3160 1332 1888 2161))
(f2cl-lib:data-implied-do (((mm (117 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2785 1832 1836 3365))
(f2cl-lib:data-implied-do (((mm (118 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2772 2405 1990 361))
(f2cl-lib:data-implied-do (((mm (119 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1217 3638 2058 2685))
(f2cl-lib:data-implied-do (((mm (120 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1822 3661 692 3745))
(f2cl-lib:data-implied-do (((mm (121 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1245 327 1194 2325))
(f2cl-lib:data-implied-do (((mm (122 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2252 3660 20 3609))
(f2cl-lib:data-implied-do (((mm (123 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3904 716 3285 3821))
(f2cl-lib:data-implied-do (((mm (124 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2774 1842 2046 3537))
(f2cl-lib:data-implied-do (((mm (125 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (997 3987 2107 517))
(f2cl-lib:data-implied-do (((mm (126 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2573 1368 3508 3017))
(f2cl-lib:data-implied-do (((mm (127 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1148 1848 3525 2141))
(f2cl-lib:data-implied-do (((mm (128 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (545 2366 3801 1537))
(defun dlaruv (iseed n x)
  (declare (type (array double-float (*)) x)
           (type (f2cl-lib:integer4) n)
           (type (array f2cl-lib:integer4 (*)) iseed))
  (f2cl-lib:with-multi-array-data
      ((iseed f2cl-lib:integer4 iseed-%data% iseed-%offset%)
       (x double-float x-%data% x-%offset%))
    (prog ((i 0) (i1 0) (i2 0) (i3 0) (i4 0) (it1 0) (it2 0) (it3 0)
           (it4 0) (j 0))
      (declare (type (f2cl-lib:integer4) i i1 i2 i3 i4 it1 it2 it3 it4 j))
      (setf i1 (f2cl-lib:fref iseed-%data% (1) ((1 4)) iseed-%offset%))
      (setf i2 (f2cl-lib:fref iseed-%data% (2) ((1 4)) iseed-%offset%))
      (setf i3 (f2cl-lib:fref iseed-%data% (3) ((1 4)) iseed-%offset%))
      (setf i4 (f2cl-lib:fref iseed-%data% (4) ((1 4)) iseed-%offset%))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (min (the f2cl-lib:integer4 n)
                             (the f2cl-lib:integer4 lv)))
```

```
             nil)
(tagbody
 label20
  (setf it4
          (f2cl-lib:int-mul i4
                            (f2cl-lib:fref mm (i 4)
                                           ((1 lv) (1 4)))))
  (setf it3 (the f2cl-lib:integer4 (truncate it4 ipw2)))
  (setf it4 (f2cl-lib:int-sub it4 (f2cl-lib:int-mul ipw2 it3)))
  (setf it3
          (f2cl-lib:int-add it3
                            (f2cl-lib:int-mul i3
                                              (f2cl-lib:fref mm
                                                             (i 4)
                                                             ((1
                                                               lv)
                                                              (1
                                                               4))))
                            (f2cl-lib:int-mul i4
                                              (f2cl-lib:fref mm
                                                             (i 3)
                                                             ((1
                                                               lv)
                                                              (1
                                                               4))))))
  (setf it2 (the f2cl-lib:integer4 (truncate it3 ipw2)))
  (setf it3 (f2cl-lib:int-sub it3 (f2cl-lib:int-mul ipw2 it2)))
  (setf it2
          (f2cl-lib:int-add it2
                            (f2cl-lib:int-mul i2
                                              (f2cl-lib:fref mm
                                                             (i 4)
                                                             ((1
                                                               lv)
                                                              (1
                                                               4))))
                            (f2cl-lib:int-mul i3
                                              (f2cl-lib:fref mm
                                                             (i 3)
                                                             ((1
                                                               lv)
                                                              (1
                                                               4))))
                            (f2cl-lib:int-mul i4
                                              (f2cl-lib:fref mm
                                                             (i 2)
                                                             ((1
                                                               lv)
                                                              (1
                                                               4))))))
```

```
        (setf it1 (the f2cl-lib:integer4 (truncate it2 ipw2)))
        (setf it2 (f2cl-lib:int-sub it2 (f2cl-lib:int-mul ipw2 it1)))
        (setf it1
                (f2cl-lib:int-add it1
                                  (f2cl-lib:int-mul i1
                                                    (f2cl-lib:fref mm
                                                                   (i 4)
                                                                   ((1
                                                                     lv)
                                                                    (1
                                                                     4))))
                                  (f2cl-lib:int-mul i2
                                                    (f2cl-lib:fref mm
                                                                   (i 3)
                                                                   ((1
                                                                     lv)
                                                                    (1
                                                                     4))))
                                  (f2cl-lib:int-mul i3
                                                    (f2cl-lib:fref mm
                                                                   (i 2)
                                                                   ((1
                                                                     lv)
                                                                    (1
                                                                     4))))
                                  (f2cl-lib:int-mul i4
                                                    (f2cl-lib:fref mm
                                                                   (i 1)
                                                                   ((1
                                                                     lv)
                                                                    (1
                                                                     4))))))
        (setf it1 (mod it1 ipw2))
        (setf (f2cl-lib:fref x-%data% (i) ((1 n)) x-%offset%)
                (* r
                   (+ (f2cl-lib:dble it1)
                      (* r
                         (+ (f2cl-lib:dble it2)
                            (* r
                               (+ (f2cl-lib:dble it3)
                                  (* r (f2cl-lib:dble it4)))))))))
        (cond
         ((= (f2cl-lib:fref x (i) ((1 n))) 1.0d0)
          (setf i1 (f2cl-lib:int-add i1 2))
          (setf i2 (f2cl-lib:int-add i2 2))
          (setf i3 (f2cl-lib:int-add i3 2))
          (setf i4 (f2cl-lib:int-add i4 2)) (go label20)))
      label10))
    (setf (f2cl-lib:fref iseed-%data% (1) ((1 4)) iseed-%offset%) it1)
    (setf (f2cl-lib:fref iseed-%data% (2) ((1 4)) iseed-%offset%) it2)
```

```
          (setf (f2cl-lib:fref iseed-%data% (3) ((1 4)) iseed-%offset%) it3)
          (setf (f2cl-lib:fref iseed-%data% (4) ((1 4)) iseed-%offset%) it4)
          (go end_label)
         end_label
          (return (values nil nil nil)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlaruv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*)))
                                            :return-values '(nil nil nil)
                                            :calls 'nil)))
```

———-

90.31 dlas2

90.31.1 Axiom unit tests

— **dlas2.input** —

```
)set break resume
)sys rm -f dlas2.output
)spool dlas2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.31.2 Axiom help page

— **dlas2.help** —

```
====================================================================
dlas2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlas2 (F, G, H, SSMIN, SSMAX)
           DLAS2 computes singular values of a 2-by-2 triangular matrix.

Function/Subroutine Documentation
   subroutine dlas2 (double precision F, double precision G, double precision
       H, double precision SSMIN, double precision SSMAX)
       DLAS2 computes singular values of a 2-by-2 triangular matrix.

Purpose:
   DLAS2  computes the singular values of the 2-by-2 matrix
     [  F   G  ]
     [  0   H  ].
   On return, SSMIN is the smaller singular value and SSMAX is the
   larger singular value.

Parameters:

 F
   F is DOUBLE PRECISION
   The (1,1) element of the 2-by-2 matrix.

 G
   G is DOUBLE PRECISION
   The (1,2) element of the 2-by-2 matrix.

 H
   H is DOUBLE PRECISION
   The (2,2) element of the 2-by-2 matrix.

 SSMIN
   SSMIN is DOUBLE PRECISION
   The smaller singular value.

 SSMAX
   SSMAX is DOUBLE PRECISION
   The larger singular value.

Further Details:

   Barring over/underflow, all output quantities are correct to within
```

```
a few units in the last place (ulps), even in the absence of a guard
digit in addition/subtraction.

In IEEE arithmetic, the code works correctly if one matrix element is
infinite.

Overflow will not occur unless the largest singular value itself
overflows, or is within a few ulps of overflow. (On machines with
partial overflow, like the Cray, overflow may occur if the largest
singular value is within a factor of 2 of overflow.)

Underflow is harmless if underflow is gradual. Otherwise, results
may correspond to a matrix modified by perturbations of size near
the underflow threshold.
```

———-

90.31.3 fortran code

— **dlas2.f** —

```
*  =====================================================================
      SUBROUTINE DLAS2( F, G, H, SSMIN, SSMAX )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   F, G, H, SSMAX, SSMIN
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D0 )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D0 )
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION   AS, AT, AU, C, FA, FHMN, FHMX, GA, HA
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          ABS, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      FA = ABS( F )
      GA = ABS( G )
      HA = ABS( H )
      FHMN = MIN( FA, HA )
      FHMX = MAX( FA, HA )
      IF( FHMN.EQ.ZERO ) THEN
         SSMIN = ZERO
         IF( FHMX.EQ.ZERO ) THEN
            SSMAX = GA
         ELSE
            SSMAX = MAX( FHMX, GA )*SQRT( ONE+
     $              ( MIN( FHMX, GA ) / MAX( FHMX, GA ) )**2 )
         END IF
      ELSE
         IF( GA.LT.FHMX ) THEN
            AS = ONE + FHMN / FHMX
            AT = ( FHMX-FHMN ) / FHMX
            AU = ( GA / FHMX )**2
            C = TWO / ( SQRT( AS*AS+AU )+SQRT( AT*AT+AU ) )
            SSMIN = FHMN*C
            SSMAX = FHMX / C
         ELSE
            AU = FHMX / GA
            IF( AU.EQ.ZERO ) THEN
*
*              Avoid possible harmful underflow if exponent range
*              asymmetric (true SSMIN may not underflow even if
*              AU underflows)
*
               SSMIN = ( FHMN*FHMX ) / GA
               SSMAX = GA
            ELSE
               AS = ONE + FHMN / FHMX
               AT = ( FHMX-FHMN ) / FHMX
               C = ONE / ( SQRT( ONE+( AS*AU )**2 )+
     $             SQRT( ONE+( AT*AU )**2 ) )
               SSMIN = ( FHMN*C )*AU
               SSMIN = SSMIN + SSMIN
               SSMAX = GA / ( C+C )
            END IF
         END IF
      END IF
      RETURN
*
*     End of DLAS2
*
```

```
      END
```

———-

90.31.4 lisp code

— **dlas2.lisp** —

———-

90.32 dlascl

90.32.1 Axiom unit tests

— **dlascl.input** —

```
)set break resume
)sys rm -f dlascl.output
)spool dlascl.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.32.2 Axiom help page

— **dlascl.help** —

```
==========================================================================
dlascl examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine dlascl (TYPE, KL, KU, CFROM, CTO, M, N, A, LDA, INFO)
           DLASCL multiplies a general rectangular matrix by a real scalar
           defined as cto/cfrom.

Function/Subroutine Documentation
   subroutine dlascl (character TYPE, integer KL, integer KU, double precision
       CFROM, double precision CTO, integer M, integer N, double precision,
       dimension( lda, * ) A, integer LDA, integer INFO)
       DLASCL multiplies a general rectangular matrix by a real scalar defined
       as cto/cfrom.

Purpose:
   DLASCL multiplies the M by N real matrix A by the real scalar
   CTO/CFROM.  This is done without over/underflow as long as the final
   result CTO*A(I,J)/CFROM does not over/underflow. TYPE specifies that
   A may be full, upper triangular, lower triangular, upper Hessenberg,
   or banded.

Parameters:

 TYPE
   TYPE is CHARACTER*1
   TYPE indices the storage type of the input matrix.
   = 'G':  A is a full matrix.
   = 'L':  A is a lower triangular matrix.
   = 'U':  A is an upper triangular matrix.
   = 'H':  A is an upper Hessenberg matrix.
   = 'B':  A is a symmetric band matrix with lower bandwidth KL
      and upper bandwidth KU and with the only the lower half stored.
   = 'Q':  A is a symmetric band matrix with lower bandwidth KL
      and upper bandwidth KU and with the only the upper half stored.
   = 'Z':  A is a band matrix with lower bandwidth KL and upper
      bandwidth KU. See DGBTRF for storage details.

 KL
   KL is INTEGER
   The lower bandwidth of A.  Referenced only if TYPE = 'B', 'Q' or 'Z'.

 KU
   KU is INTEGER
   The upper bandwidth of A.  Referenced only if TYPE = 'B', 'Q' or 'Z'.

 CFROM
   CFROM is DOUBLE PRECISION

 CTO
   CTO is DOUBLE PRECISION
```

```
  The matrix A is multiplied by CTO/CFROM. A(I,J) is computed
  without over/underflow if the final result CTO*A(I,J)/CFROM
  can be represented without over/underflow.  CFROM must be nonzero.

M
  M is INTEGER
  The number of rows of the matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  The matrix to be multiplied by CTO/CFROM.  See TYPE for the storage type.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

INFO
  INFO is INTEGER
  0  - successful exit
  < 0 - if INFO = -i, the i-th argument had an illegal value.
```

———-

90.32.3 fortran code

— **dlascl.f** —

```
*  =====================================================================
      SUBROUTINE DLASCL( TYPE, KL, KU, CFROM, CTO, M, N, A, LDA, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          TYPE
      INTEGER            INFO, KL, KU, LDA, M, N
      DOUBLE PRECISION   CFROM, CTO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * )
*     ..
```

```
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            DONE
      INTEGER            I, ITYPE, J, K1, K2, K3, K4
      DOUBLE PRECISION   BIGNUM, CFROM1, CFROMC, CTO1, CTOC, MUL, SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, DISNAN
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH, DISNAN
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
*
      IF( LSAME( TYPE, 'G' ) ) THEN
         ITYPE = 0
      ELSE IF( LSAME( TYPE, 'L' ) ) THEN
         ITYPE = 1
      ELSE IF( LSAME( TYPE, 'U' ) ) THEN
         ITYPE = 2
      ELSE IF( LSAME( TYPE, 'H' ) ) THEN
         ITYPE = 3
      ELSE IF( LSAME( TYPE, 'B' ) ) THEN
         ITYPE = 4
      ELSE IF( LSAME( TYPE, 'Q' ) ) THEN
         ITYPE = 5
      ELSE IF( LSAME( TYPE, 'Z' ) ) THEN
         ITYPE = 6
      ELSE
         ITYPE = -1
      END IF
*
      IF( ITYPE.EQ.-1 ) THEN
         INFO = -1
      ELSE IF( CFROM.EQ.ZERO .OR. DISNAN(CFROM) ) THEN
```

```
         INFO = -4
      ELSE IF( DISNAN(CTO) ) THEN
         INFO = -5
      ELSE IF( M.LT.0 ) THEN
         INFO = -6
      ELSE IF( N.LT.0 .OR. ( ITYPE.EQ.4 .AND. N.NE.M ) .OR.
     $         ( ITYPE.EQ.5 .AND. N.NE.M ) ) THEN
         INFO = -7
      ELSE IF( ITYPE.LE.3 .AND. LDA.LT.MAX( 1, M ) ) THEN
         INFO = -9
      ELSE IF( ITYPE.GE.4 ) THEN
         IF( KL.LT.0 .OR. KL.GT.MAX( M-1, 0 ) ) THEN
            INFO = -2
         ELSE IF( KU.LT.0 .OR. KU.GT.MAX( N-1, 0 ) .OR.
     $            ( ( ITYPE.EQ.4 .OR. ITYPE.EQ.5 ) .AND. KL.NE.KU ) )
     $             THEN
            INFO = -3
         ELSE IF( ( ITYPE.EQ.4 .AND. LDA.LT.KL+1 ) .OR.
     $            ( ITYPE.EQ.5 .AND. LDA.LT.KU+1 ) .OR.
     $            ( ITYPE.EQ.6 .AND. LDA.LT.2*KL+KU+1 ) ) THEN
            INFO = -9
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASCL', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. M.EQ.0 )
     $   RETURN
*
*     Get machine parameters
*
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
*
      CFROMC = CFROM
      CTOC = CTO
*
   10 CONTINUE
      CFROM1 = CFROMC*SMLNUM
      IF( CFROM1.EQ.CFROMC ) THEN
!        CFROMC is an inf.  Multiply by a correctly signed zero for
!        finite CTOC, or a NaN if CTOC is infinite.
         MUL = CTOC / CFROMC
         DONE = .TRUE.
         CTO1 = CTOC
```

```
      ELSE
         CTO1 = CTOC / BIGNUM
         IF( CTO1.EQ.CTOC ) THEN
!           CTOC is either 0 or an inf.  In both cases, CTOC itself
!           serves as the correct multiplication factor.
            MUL = CTOC
            DONE = .TRUE.
            CFROMC = ONE
         ELSE IF( ABS( CFROM1 ).GT.ABS( CTOC ) .AND. CTOC.NE.ZERO ) THEN
            MUL = SMLNUM
            DONE = .FALSE.
            CFROMC = CFROM1
         ELSE IF( ABS( CTO1 ).GT.ABS( CFROMC ) ) THEN
            MUL = BIGNUM
            DONE = .FALSE.
            CTOC = CTO1
         ELSE
            MUL = CTOC / CFROMC
            DONE = .TRUE.
         END IF
      END IF
*
      IF( ITYPE.EQ.0 ) THEN
*
*        Full matrix
*
         DO 30 J = 1, N
            DO 20 I = 1, M
               A( I, J ) = A( I, J )*MUL
   20       CONTINUE
   30    CONTINUE
*
      ELSE IF( ITYPE.EQ.1 ) THEN
*
*        Lower triangular matrix
*
         DO 50 J = 1, N
            DO 40 I = J, M
               A( I, J ) = A( I, J )*MUL
   40       CONTINUE
   50    CONTINUE
*
      ELSE IF( ITYPE.EQ.2 ) THEN
*
*        Upper triangular matrix
*
         DO 70 J = 1, N
            DO 60 I = 1, MIN( J, M )
               A( I, J ) = A( I, J )*MUL
   60       CONTINUE
```

```
   70    CONTINUE
*
      ELSE IF( ITYPE.EQ.3 ) THEN
*
*        Upper Hessenberg matrix
*
         DO 90 J = 1, N
            DO 80 I = 1, MIN( J+1, M )
               A( I, J ) = A( I, J )*MUL
   80       CONTINUE
   90    CONTINUE
*
      ELSE IF( ITYPE.EQ.4 ) THEN
*
*        Lower half of a symmetric band matrix
*
         K3 = KL + 1
         K4 = N + 1
         DO 110 J = 1, N
            DO 100 I = 1, MIN( K3, K4-J )
               A( I, J ) = A( I, J )*MUL
  100       CONTINUE
  110    CONTINUE
*
      ELSE IF( ITYPE.EQ.5 ) THEN
*
*        Upper half of a symmetric band matrix
*
         K1 = KU + 2
         K3 = KU + 1
         DO 130 J = 1, N
            DO 120 I = MAX( K1-J, 1 ), K3
               A( I, J ) = A( I, J )*MUL
  120       CONTINUE
  130    CONTINUE
*
      ELSE IF( ITYPE.EQ.6 ) THEN
*
*        Band matrix
*
         K1 = KL + KU + 2
         K2 = KL + 1
         K3 = 2*KL + KU + 1
         K4 = KL + KU + 1 + M
         DO 150 J = 1, N
            DO 140 I = MAX( K1-J, K2 ), MIN( K3, K4-J )
               A( I, J ) = A( I, J )*MUL
  140       CONTINUE
  150    CONTINUE
*
```

```
      END IF
*
      IF( .NOT.DONE )
     $   GO TO 10
*
      RETURN
*
*     End of DLASCL
*
      END
```

———-

90.32.4 lisp code

— **dlascl.lisp** —

```
(defun dlascl2 (m n d x ldx)
  (declare (type (array double-float (*)) x d)
           (type (f2cl-lib:integer4) ldx n m))
  (f2cl-lib:with-multi-array-data
      ((d double-float d-%data% d-%offset%)
       (x double-float x-%data% x-%offset%))
    (prog ((i 0) (j 0))
      (declare (type (f2cl-lib:integer4) j i))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *)) x-%offset%)
                      (*
                       (f2cl-lib:fref x-%data% (i j) ((1 ldx) (1 *))
                                      x-%offset%)
                       (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
             label100001))
         label100000))
      (go end_label)
     end_label
      (return (values nil nil nil nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::dlascl2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
```

```
'((fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array double-float (*))
  (array double-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil nil nil) :calls
'nil)))
```

———-

90.33 dlasd0

90.33.1 Axiom unit tests

— dlasd0.input —

```
)set break resume
)sys rm -f dlasd0.output
)spool dlasd0.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.33.2 Axiom help page

— dlasd0.help —

```
====================================================================
dlasd0 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
```

```
   Functions/Subroutines
       subroutine dlasd0 (N, SQRE, D, E, U, LDU, VT, LDVT, SMLSIZ, IWORK,
           WORK, INFO)
           DLASD0 computes the singular values of a real upper bidiagonal n-
           by-m matrix B with diagonal d and off-diagonal e. Used by sbdsdc.

Function/Subroutine Documentation
   subroutine dlasd0 (integer N, integer SQRE, double precision, dimension( *
       ) D, double precision, dimension( * ) E, double precision, dimension(
       ldu, * ) U, integer LDU, double precision, dimension( ldvt, * ) VT,
       integer LDVT, integer SMLSIZ, integer, dimension( * ) IWORK, double
       precision, dimension( * ) WORK, integer INFO)
       DLASD0 computes the singular values of a real upper bidiagonal n-by-m
       matrix B with diagonal d and off-diagonal e. Used by sbdsdc.

Purpose:
   Using a divide and conquer approach, DLASD0 computes the singular
   value decomposition (SVD) of a real upper bidiagonal N-by-M
   matrix B with diagonal D and offdiagonal E, where M = N + SQRE.
   The algorithm computes orthogonal matrices U and VT such that
   B = U * S * VT. The singular values S are overwritten on D.

   A related subroutine, DLASDA, computes only the singular values,
   and optionally, the singular vectors in compact form.

Parameters:

 N
   N is INTEGER
   On entry, the row dimension of the upper bidiagonal matrix.
   This is also the dimension of the main diagonal array D.

 SQRE
   SQRE is INTEGER
   Specifies the column dimension of the bidiagonal matrix.
   = 0: The bidiagonal matrix has column dimension M = N;
   = 1: The bidiagonal matrix has column dimension M = N+1;

 D
   D is DOUBLE PRECISION array, dimension (N)
   On entry D contains the main diagonal of the bidiagonal matrix.
   On exit D, if INFO = 0, contains its singular values.

 E
   E is DOUBLE PRECISION array, dimension (M-1)
   Contains the subdiagonal entries of the bidiagonal matrix.
   On exit, E has been destroyed.

 U
   U is DOUBLE PRECISION array, dimension at least (LDQ, N)
```

```
  On exit, U contains the left singular vectors.

LDU
  LDU is INTEGER
  On entry, leading dimension of U.

VT
  VT is DOUBLE PRECISION array, dimension at least (LDVT, M)
  On exit, VT**T contains the right singular vectors.

LDVT
  LDVT is INTEGER
  On entry, leading dimension of VT.

SMLSIZ
  SMLSIZ is INTEGER
  On entry, maximum size of the subproblems at the
  bottom of the computation tree.

IWORK
  IWORK is INTEGER work array.
  Dimension must be at least (8 * N)

WORK
  WORK is DOUBLE PRECISION work array.
  Dimension must be at least (3 * M**2 + 2 * M)

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if INFO = 1, a singular value did not converge
```

———-

90.33.3 fortran code

— dlasd0.f —

```
*  =====================================================================
      SUBROUTINE DLASD0( N, SQRE, D, E, U, LDU, VT, LDVT, SMLSIZ, IWORK,
     $                   WORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
```

```
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDU, LDVT, N, SMLSIZ, SQRE
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   D( * ), E( * ), U( LDU, * ), VT( LDVT, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, I1, IC, IDXQ, IDXQC, IM1, INODE, ITEMP, IWK,
     $                   J, LF, LL, LVL, M, NCC, ND, NDB1, NDIML, NDIMR,
     $                   NL, NLF, NLP1, NLVL, NR, NRF, NRP1, SQREI
      DOUBLE PRECISION   ALPHA, BETA
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASD1, DLASDQ, DLASDT, XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ( SQRE.LT.0 ) .OR. ( SQRE.GT.1 ) ) THEN
         INFO = -2
      END IF
*
      M = N + SQRE
*
      IF( LDU.LT.N ) THEN
         INFO = -6
      ELSE IF( LDVT.LT.M ) THEN
         INFO = -8
      ELSE IF( SMLSIZ.LT.3 ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASD0', -INFO )
         RETURN
      END IF
*
*     If the input matrix is too small, call DLASDQ to find the SVD.
*
      IF( N.LE.SMLSIZ ) THEN
```

```
         CALL DLASDQ( 'U', SQRE, N, M, N, 0, D, E, VT, LDVT, U, LDU, U,
     $                LDU, WORK, INFO )
         RETURN
      END IF
*
*     Set up the computation tree.
*
      INODE = 1
      NDIML = INODE + N
      NDIMR = NDIML + N
      IDXQ = NDIMR + N
      IWK = IDXQ + N
      CALL DLASDT( N, NLVL, ND, IWORK( INODE ), IWORK( NDIML ),
     $             IWORK( NDIMR ), SMLSIZ )
*
*     For the nodes on bottom level of the tree, solve
*     their subproblems by DLASDQ.
*
      NDB1 = ( ND+1 ) / 2
      NCC = 0
      DO 30 I = NDB1, ND
*
*     IC : center row of each node
*     NL : number of rows of left  subproblem
*     NR : number of rows of right subproblem
*     NLF: starting row of the left   subproblem
*     NRF: starting row of the right  subproblem
*
         I1 = I - 1
         IC = IWORK( INODE+I1 )
         NL = IWORK( NDIML+I1 )
         NLP1 = NL + 1
         NR = IWORK( NDIMR+I1 )
         NRP1 = NR + 1
         NLF = IC - NL
         NRF = IC + 1
         SQREI = 1
         CALL DLASDQ( 'U', SQREI, NL, NLP1, NL, NCC, D( NLF ), E( NLF ),
     $                VT( NLF, NLF ), LDVT, U( NLF, NLF ), LDU,
     $                U( NLF, NLF ), LDU, WORK, INFO )
         IF( INFO.NE.0 ) THEN
            RETURN
         END IF
         ITEMP = IDXQ + NLF - 2
         DO 10 J = 1, NL
            IWORK( ITEMP+J ) = J
   10    CONTINUE
         IF( I.EQ.ND ) THEN
            SQREI = SQRE
         ELSE
```

```
         SQREI = 1
      END IF
      NRP1 = NR + SQREI
      CALL DLASDQ( 'U', SQREI, NR, NRP1, NR, NCC, D( NRF ), E( NRF ),
     $               VT( NRF, NRF ), LDVT, U( NRF, NRF ), LDU,
     $               U( NRF, NRF ), LDU, WORK, INFO )
      IF( INFO.NE.0 ) THEN
         RETURN
      END IF
      ITEMP = IDXQ + IC
      DO 20 J = 1, NR
         IWORK( ITEMP+J-1 ) = J
   20 CONTINUE
   30 CONTINUE
*
*     Now conquer each subproblem bottom-up.
*
      DO 50 LVL = NLVL, 1, -1
*
*        Find the first node LF and last node LL on the
*        current level LVL.
*
      IF( LVL.EQ.1 ) THEN
         LF = 1
         LL = 1
      ELSE
         LF = 2**( LVL-1 )
         LL = 2*LF - 1
      END IF
      DO 40 I = LF, LL
         IM1 = I - 1
         IC = IWORK( INODE+IM1 )
         NL = IWORK( NDIML+IM1 )
         NR = IWORK( NDIMR+IM1 )
         NLF = IC - NL
         IF( ( SQRE.EQ.0 ) .AND. ( I.EQ.LL ) ) THEN
            SQREI = SQRE
         ELSE
            SQREI = 1
         END IF
         IDXQC = IDXQ + NLF - 1
         ALPHA = D( IC )
         BETA = E( IC )
         CALL DLASD1( NL, NR, SQREI, D( NLF ), ALPHA, BETA,
     $                  U( NLF, NLF ), LDU, VT( NLF, NLF ), LDVT,
     $                  IWORK( IDXQC ), IWORK( IWK ), WORK, INFO )
*
*        Report the possible convergence failure.
*
         IF( INFO.NE.0 ) THEN
```

```
              RETURN
            END IF
   40    CONTINUE
   50 CONTINUE
*
      RETURN
*
*     End of DLASD0
*
      END
```

———-

90.33.4 lisp code

— **dlasd0.lisp** —

———-

90.34 dlasd1

90.34.1 Axiom unit tests

— **dlasd1.input** —

```
)set break resume
)sys rm -f dlasd1.output
)spool dlasd1.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.34.2 Axiom help page

— **dlasd1.help** —

```
====================================================================
dlasd1 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlasd1 (NL, NR, SQRE, D, ALPHA, BETA, U, LDU, VT, LDVT,
           IDXQ, IWORK, WORK, INFO)
           DLASD1 computes the SVD of an upper bidiagonal matrix B of the
           specified size. Used by sbdsdc.

Function/Subroutine Documentation
   subroutine dlasd1 (integer NL, integer NR, integer SQRE, double precision,
       dimension( * ) D, double precision ALPHA, double precision BETA, double
       precision, dimension( ldu, * ) U, integer LDU, double precision,
       dimension( ldvt, * ) VT, integer LDVT, integer, dimension( * ) IDXQ,
       integer, dimension( * ) IWORK, double precision, dimension( * ) WORK,
       integer INFO)
       DLASD1 computes the SVD of an upper bidiagonal matrix B of the
       specified size. Used by sbdsdc.

Purpose:
   DLASD1 computes the SVD of an upper bidiagonal N-by-M matrix B,
   where N = NL + NR + 1 and M = N + SQRE. DLASD1 is called from DLASD0.

   A related subroutine DLASD7 handles the case in which the singular
   values (and the singular vectors in factored form) are desired.

   DLASD1 computes the SVD as follows:

                     ( D1(in)    0    0       0 )
         B = U(in) * (   Z1**T   a   Z2**T    b ) * VT(in)
                     (   0       0   D2(in)   0 )

           = U(out) * ( D(out) 0) * VT(out)

   where Z**T = (Z1**T a Z2**T b) = u**T VT**T, and u is a vector of
   dimension M with ALPHA and BETA in the NL+1 and NL+2 th entries and zeros
   elsewhere; and the entry b is empty if SQRE = 0.

   The left singular vectors of the original matrix are stored in U, and
   the transpose of the right singular vectors are stored in VT, and the
   singular values are in D.  The algorithm consists of three stages:

   The first stage consists of deflating the size of the problem
   when there are multiple singular values or when there are zeros in
```

```
  the Z vector.  For each such occurence the dimension of the
  secular equation problem is reduced by one.  This stage is
  performed by the routine DLASD2.

  The second stage consists of calculating the updated
  singular values. This is done by finding the square roots of the
  roots of the secular equation via the routine DLASD4 (as called
  by DLASD3). This routine also calculates the singular vectors of
  the current problem.

  The final stage consists of computing the updated singular vectors
  directly using the updated singular values.  The singular vectors
  for the current problem are multiplied with the singular vectors
  from the overall problem.

Parameters:

 NL
   NL is INTEGER
   The row dimension of the upper block.  NL >= 1.

 NR
   NR is INTEGER
   The row dimension of the lower block.  NR >= 1.

 SQRE
   SQRE is INTEGER
   = 0: the lower block is an NR-by-NR square matrix.
   = 1: the lower block is an NR-by-(NR+1) rectangular matrix.

   The bidiagonal matrix has row dimension N = NL + NR + 1,
   and column dimension M = N + SQRE.

 D
   D is DOUBLE PRECISION array, dimension (N = NL+NR+1).
   On entry D(1:NL,1:NL) contains the singular values of the
   upper block; and D(NL+2:N) contains the singular values of
   the lower block. On exit D(1:N) contains the singular values
   of the modified matrix.

 ALPHA
   ALPHA is DOUBLE PRECISION
   Contains the diagonal element associated with the added row.

 BETA
   BETA is DOUBLE PRECISION
   Contains the off-diagonal element associated with the added row.

 U
   U is DOUBLE PRECISION array, dimension(LDU,N)
```

```
  On entry U(1:NL, 1:NL) contains the left singular vectors of
  the upper block; U(NL+2:N, NL+2:N) contains the left singular
  vectors of the lower block. On exit U contains the left
  singular vectors of the bidiagonal matrix.

LDU
  LDU is INTEGER
  The leading dimension of the array U.  LDU >= max( 1, N ).

VT
  VT is DOUBLE PRECISION array, dimension(LDVT,M)
  where M = N + SQRE.
  On entry VT(1:NL+1, 1:NL+1)**T contains the right singular
  vectors of the upper block; VT(NL+2:M, NL+2:M)**T contains
  the right singular vectors of the lower block. On exit
  VT**T contains the right singular vectors of the bidiagonal matrix.

LDVT
  LDVT is INTEGER
  The leading dimension of the array VT.  LDVT >= max( 1, M ).

IDXQ
  IDXQ is INTEGER array, dimension(N)
  This contains the permutation which will reintegrate the
  subproblem just solved back into sorted order, i.e.
  D( IDXQ( I = 1, N ) ) will be in ascending order.

IWORK
  IWORK is INTEGER array, dimension( 4 * N )

WORK
  WORK is DOUBLE PRECISION array, dimension( 3*M**2 + 2*M )

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if INFO = 1, a singular value did not converge
```

———-

90.34.3 fortran code

— dlasd1.f —

```
*  =====================================================================
      SUBROUTINE DLASD1( NL, NR, SQRE, D, ALPHA, BETA, U, LDU, VT, LDVT,
```

```
     $                   IDXQ, IWORK, WORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDU, LDVT, NL, NR, SQRE
      DOUBLE PRECISION   ALPHA, BETA
*     ..
*     .. Array Arguments ..
      INTEGER            IDXQ( * ), IWORK( * )
      DOUBLE PRECISION   D( * ), U( LDU, * ), VT( LDVT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
*
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            COLTYP, I, IDX, IDXC, IDXP, IQ, ISIGMA, IU2,
     $                   IVT2, IZ, K, LDQ, LDU2, LDVT2, M, N, N1, N2
      DOUBLE PRECISION   ORGNRM
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLAMRG, DLASCL, DLASD2, DLASD3, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( NL.LT.1 ) THEN
         INFO = -1
      ELSE IF( NR.LT.1 ) THEN
         INFO = -2
      ELSE IF( ( SQRE.LT.0 ) .OR. ( SQRE.GT.1 ) ) THEN
         INFO = -3
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASD1', -INFO )
         RETURN
```

```
      END IF
*
      N = NL + NR + 1
      M = N + SQRE
*
*     The following values are for bookkeeping purposes only.  They are
*     integer pointers which indicate the portion of the workspace
*     used by a particular array in DLASD2 and DLASD3.
*
      LDU2 = N
      LDVT2 = M
*
      IZ = 1
      ISIGMA = IZ + M
      IU2 = ISIGMA + N
      IVT2 = IU2 + LDU2*N
      IQ = IVT2 + LDVT2*M
*
      IDX = 1
      IDXC = IDX + N
      COLTYP = IDXC + N
      IDXP = COLTYP + N
*
*     Scale.
*
      ORGNRM = MAX( ABS( ALPHA ), ABS( BETA ) )
      D( NL+1 ) = ZERO
      DO 10 I = 1, N
         IF( ABS( D( I ) ).GT.ORGNRM ) THEN
            ORGNRM = ABS( D( I ) )
         END IF
   10 CONTINUE
      CALL DLASCL( 'G', 0, 0, ORGNRM, ONE, N, 1, D, N, INFO )
      ALPHA = ALPHA / ORGNRM
      BETA = BETA / ORGNRM
*
*     Deflate singular values.
*
      CALL DLASD2( NL, NR, SQRE, K, D, WORK( IZ ), ALPHA, BETA, U, LDU,
     $             VT, LDVT, WORK( ISIGMA ), WORK( IU2 ), LDU2,
     $             WORK( IVT2 ), LDVT2, IWORK( IDXP ), IWORK( IDX ),
     $             IWORK( IDXC ), IDXQ, IWORK( COLTYP ), INFO )
*
*     Solve Secular Equation and update singular vectors.
*
      LDQ = K
      CALL DLASD3( NL, NR, SQRE, K, D, WORK( IQ ), LDQ, WORK( ISIGMA ),
     $             U, LDU, WORK( IU2 ), LDU2, VT, LDVT, WORK( IVT2 ),
     $             LDVT2, IWORK( IDXC ), IWORK( COLTYP ), WORK( IZ ),
     $             INFO )
```

```
*
*     Report the convergence failure.
*
      IF( INFO.NE.0 ) THEN
         RETURN
      END IF
*
*     Unscale.
*
      CALL DLASCL( 'G', 0, 0, ONE, ORGNRM, N, 1, D, N, INFO )
*
*     Prepare the IDXQ sorting permutation.
*
      N1 = K
      N2 = N - K
      CALL DLAMRG( N1, N2, D, 1, -1, IDXQ )
*
      RETURN
*
*     End of DLASD1
*
      END
```

———-

90.34.4 lisp code

— dlasd1.lisp —

———-

90.35 dlasd2

90.35.1 Axiom unit tests

— dlasd2.input —

```
)set break resume
)sys rm -f dlasd2.output
)spool dlasd2.output
)set message test on
)set message auto off
```

```
)clear all

)spool
)lisp (bye)
```

———-

90.35.2 Axiom help page

— **dlasd2.help** —

```
==================================================================
dlasd2 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlasd2 (NL, NR, SQRE, K, D, Z, ALPHA, BETA, U, LDU, VT,
           LDVT, DSIGMA, U2, LDU2, VT2, LDVT2, IDXP, IDX, IDXC, IDXQ, COLTYP,
           INFO)
           DLASD2 merges the two sets of singular values together into a
           single sorted set. Used by sbdsdc.

Function/Subroutine Documentation
   subroutine dlasd2 (integer NL, integer NR, integer SQRE, integer K, double
       precision, dimension( * ) D, double precision, dimension( * ) Z, double
       precision ALPHA, double precision BETA, double precision, dimension(
       ldu, * ) U, integer LDU, double precision, dimension( ldvt, * ) VT,
       integer LDVT, double precision, dimension( * ) DSIGMA, double
       precision, dimension( ldu2, * ) U2, integer LDU2, double precision,
       dimension( ldvt2, * ) VT2, integer LDVT2, integer, dimension( * ) IDXP,
       integer, dimension( * ) IDX, integer, dimension( * ) IDXC, integer,
       dimension( * ) IDXQ, integer, dimension( * ) COLTYP, integer INFO)
       DLASD2 merges the two sets of singular values together into a single
       sorted set. Used by sbdsdc.

Purpose:
   DLASD2 merges the two sets of singular values together into a single
   sorted set.  Then it tries to deflate the size of the problem.
   There are two ways in which deflation can occur:  when two or more
   singular values are close together or if there is a tiny entry in the
   Z vector.  For each such occurrence the order of the related secular
   equation problem is reduced by one.
```

```
  DLASD2 is called from DLASD1.

Parameters:

 NL
   NL is INTEGER
   The row dimension of the upper block.  NL >= 1.

 NR
   NR is INTEGER
   The row dimension of the lower block.  NR >= 1.

 SQRE
   SQRE is INTEGER
   = 0: the lower block is an NR-by-NR square matrix.
   = 1: the lower block is an NR-by-(NR+1) rectangular matrix.

   The bidiagonal matrix has N = NL + NR + 1 rows and
   M = N + SQRE >= N columns.

 K
   K is INTEGER
   Contains the dimension of the non-deflated matrix,
   This is the order of the related secular equation. 1 <= K <=N.

 D
   D is DOUBLE PRECISION array, dimension(N)
   On entry D contains the singular values of the two submatrices
   to be combined.  On exit D contains the trailing (N-K) updated
   singular values (those which were deflated) sorted into increasing order.

 Z
   Z is DOUBLE PRECISION array, dimension(N)
   On exit Z contains the updating row vector in the secular equation.

 ALPHA
   ALPHA is DOUBLE PRECISION
   Contains the diagonal element associated with the added row.

 BETA
   BETA is DOUBLE PRECISION
   Contains the off-diagonal element associated with the added row.

 U
   U is DOUBLE PRECISION array, dimension(LDU,N)
   On entry U contains the left singular vectors of two
   submatrices in the two square blocks with corners at (1,1),
   (NL, NL), and (NL+2, NL+2), (N,N).
   On exit U contains the trailing (N-K) updated left singular
   vectors (those which were deflated) in its last N-K columns.
```

```
LDU
  LDU is INTEGER
  The leading dimension of the array U.  LDU >= N.

VT
  VT is DOUBLE PRECISION array, dimension(LDVT,M)
  On entry VT**T contains the right singular vectors of two
  submatrices in the two square blocks with corners at (1,1),
  (NL+1, NL+1), and (NL+2, NL+2), (M,M).
  On exit VT**T contains the trailing (N-K) updated right singular
  vectors (those which were deflated) in its last N-K columns.
  In case SQRE =1, the last row of VT spans the right null space.

LDVT
  LDVT is INTEGER
  The leading dimension of the array VT.  LDVT >= M.

DSIGMA
  DSIGMA is DOUBLE PRECISION array, dimension (N)
  Contains a copy of the diagonal elements (K-1 singular values
  and one zero) in the secular equation.

U2
  U2 is DOUBLE PRECISION array, dimension(LDU2,N)
  Contains a copy of the first K-1 left singular vectors which
  will be used by DLASD3 in a matrix multiply (DGEMM) to solve
  for the new left singular vectors. U2 is arranged into four
  blocks. The first block contains a column with 1 at NL+1 and
  zero everywhere else; the second block contains non-zero
  entries only at and above NL; the third contains non-zero
  entries only below NL+1; and the fourth is dense.

LDU2
  LDU2 is INTEGER
  The leading dimension of the array U2.  LDU2 >= N.

VT2
  VT2 is DOUBLE PRECISION array, dimension(LDVT2,N)
  VT2**T contains a copy of the first K right singular vectors
  which will be used by DLASD3 in a matrix multiply (DGEMM) to
  solve for the new right singular vectors. VT2 is arranged into
  three blocks. The first block contains a row that corresponds
  to the special 0 diagonal element in SIGMA; the second block
  contains non-zeros only at and before NL +1; the third block
  contains non-zeros only at and after  NL +2.

LDVT2
  LDVT2 is INTEGER
  The leading dimension of the array VT2.  LDVT2 >= M.
```

```
IDXP
  IDXP is INTEGER array dimension(N)
  This will contain the permutation used to place deflated
  values of D at the end of the array. On output IDXP(2:K)
  points to the nondeflated D-values and IDXP(K+1:N)
  points to the deflated singular values.

IDX
  IDX is INTEGER array dimension(N)
  This will contain the permutation used to sort the contents of
  D into ascending order.

IDXC
  IDXC is INTEGER array dimension(N)
  This will contain the permutation used to arrange the columns
  of the deflated U matrix into three groups:  the first group
  contains non-zero entries only at and above NL, the second
  contains non-zero entries only below NL+2, and the third is dense.

IDXQ
  IDXQ is INTEGER array dimension(N)
  This contains the permutation which separately sorts the two
  sub-problems in D into ascending order.  Note that entries in
  the first hlaf of this permutation must first be moved one
  position backward; and entries in the second half
  must first have NL+1 added to their values.

COLTYP
  COLTYP is INTEGER array dimension(N)
  As workspace, this will contain a label which will indicate
  which of the following types a column in the U2 matrix or a
  row in the VT2 matrix is:
  1 : non-zero in the upper half only
  2 : non-zero in the lower half only
  3 : dense
  4 : deflated

  On exit, it is an array of dimension 4, with COLTYP(I) being
  the dimension of the I-th type columns.

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

90.35.3 fortran code

— dlasd2.f —

```
*  =====================================================================
      SUBROUTINE DLASD2( NL, NR, SQRE, K, D, Z, ALPHA, BETA, U, LDU, VT,
     $                   LDVT, DSIGMA, U2, LDU2, VT2, LDVT2, IDXP, IDX,
     $                   IDXC, IDXQ, COLTYP, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, K, LDU, LDU2, LDVT, LDVT2, NL, NR, SQRE
      DOUBLE PRECISION   ALPHA, BETA
*     ..
*     .. Array Arguments ..
      INTEGER            COLTYP( * ), IDX( * ), IDXC( * ), IDXP( * ),
     $                   IDXQ( * )
      DOUBLE PRECISION   D( * ), DSIGMA( * ), U( LDU, * ),
     $                   U2( LDU2, * ), VT( LDVT, * ), VT2( LDVT2, * ),
     $                   Z( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE, TWO, EIGHT
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0, TWO = 2.0D+0,
     $                   EIGHT = 8.0D+0 )
*     ..
*     .. Local Arrays ..
      INTEGER            CTOT( 4 ), PSM( 4 )
*     ..
*     .. Local Scalars ..
      INTEGER            CT, I, IDXI, IDXJ, IDXJP, J, JP, JPREV, K2, M,
     $                   N, NLP1, NLP2
      DOUBLE PRECISION   C, EPS, HLFTOL, S, TAU, TOL, Z1
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH, DLAPY2
      EXTERNAL           DLAMCH, DLAPY2
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DLACPY, DLAMRG, DLASET, DROT, XERBLA
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( NL.LT.1 ) THEN
         INFO = -1
      ELSE IF( NR.LT.1 ) THEN
         INFO = -2
      ELSE IF( ( SQRE.NE.1 ) .AND. ( SQRE.NE.0 ) ) THEN
         INFO = -3
      END IF
*
      N = NL + NR + 1
      M = N + SQRE
*
      IF( LDU.LT.N ) THEN
         INFO = -10
      ELSE IF( LDVT.LT.M ) THEN
         INFO = -12
      ELSE IF( LDU2.LT.N ) THEN
         INFO = -15
      ELSE IF( LDVT2.LT.M ) THEN
         INFO = -17
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASD2', -INFO )
         RETURN
      END IF
*
      NLP1 = NL + 1
      NLP2 = NL + 2
*
*     Generate the first part of the vector Z; and move the singular
*     values in the first part of D one position backward.
*
      Z1 = ALPHA*VT( NLP1, NLP1 )
      Z( 1 ) = Z1
      DO 10 I = NL, 1, -1
         Z( I+1 ) = ALPHA*VT( I, NLP1 )
         D( I+1 ) = D( I )
         IDXQ( I+1 ) = IDXQ( I ) + 1
   10 CONTINUE
*
*     Generate the second part of the vector Z.
*
      DO 20 I = NLP2, M
```

```
         Z( I ) = BETA*VT( I, NLP2 )
   20 CONTINUE
*
*     Initialize some reference arrays.
*
      DO 30 I = 2, NLP1
         COLTYP( I ) = 1
   30 CONTINUE
      DO 40 I = NLP2, N
         COLTYP( I ) = 2
   40 CONTINUE
*
*     Sort the singular values into increasing order
*
      DO 50 I = NLP2, N
         IDXQ( I ) = IDXQ( I ) + NLP1
   50 CONTINUE
*
*     DSIGMA, IDXC, IDXC, and the first column of U2
*     are used as storage space.
*
      DO 60 I = 2, N
         DSIGMA( I ) = D( IDXQ( I ) )
         U2( I, 1 ) = Z( IDXQ( I ) )
         IDXC( I ) = COLTYP( IDXQ( I ) )
   60 CONTINUE
*
      CALL DLAMRG( NL, NR, DSIGMA( 2 ), 1, 1, IDX( 2 ) )
*
      DO 70 I = 2, N
         IDXI = 1 + IDX( I )
         D( I ) = DSIGMA( IDXI )
         Z( I ) = U2( IDXI, 1 )
         COLTYP( I ) = IDXC( IDXI )
   70 CONTINUE
*
*     Calculate the allowable deflation tolerance
*
      EPS = DLAMCH( 'Epsilon' )
      TOL = MAX( ABS( ALPHA ), ABS( BETA ) )
      TOL = EIGHT*EPS*MAX( ABS( D( N ) ), TOL )
*
*     There are 2 kinds of deflation -- first a value in the z-vector
*     is small, second two (or more) singular values are very close
*     together (their difference is small).
*
*     If the value in the z-vector is small, we simply permute the
*     array so that the corresponding singular value is moved to the
*     end.
*
```

```
*     If two values in the D-vector are close, we perform a two-sided
*     rotation designed to make one of the corresponding z-vector
*     entries zero, and then permute the array so that the deflated
*     singular value is moved to the end.
*
*     If there are multiple singular values then the problem deflates.
*     Here the number of equal singular values are found.  As each equal
*     singular value is found, an elementary reflector is computed to
*     rotate the corresponding singular subspace so that the
*     corresponding components of Z are zero in this new basis.
*
      K = 1
      K2 = N + 1
      DO 80 J = 2, N
         IF( ABS( Z( J ) ).LE.TOL ) THEN
*
*           Deflate due to small z component.
*
            K2 = K2 - 1
            IDXP( K2 ) = J
            COLTYP( J ) = 4
            IF( J.EQ.N )
     $         GO TO 120
         ELSE
            JPREV = J
            GO TO 90
         END IF
   80 CONTINUE
   90 CONTINUE
      J = JPREV
  100 CONTINUE
      J = J + 1
      IF( J.GT.N )
     $   GO TO 110
      IF( ABS( Z( J ) ).LE.TOL ) THEN
*
*        Deflate due to small z component.
*
         K2 = K2 - 1
         IDXP( K2 ) = J
         COLTYP( J ) = 4
      ELSE
*
*        Check if singular values are close enough to allow deflation.
*
         IF( ABS( D( J )-D( JPREV ) ).LE.TOL ) THEN
*
*           Deflation is possible.
*
            S = Z( JPREV )
```

```
            C = Z( J )
*
*           Find sqrt(a**2+b**2) without overflow or
*           destructive underflow.
*
            TAU = DLAPY2( C, S )
            C = C / TAU
            S = -S / TAU
            Z( J ) = TAU
            Z( JPREV ) = ZERO
*
*           Apply back the Givens rotation to the left and right
*           singular vector matrices.
*
            IDXJP = IDXQ( IDX( JPREV )+1 )
            IDXJ = IDXQ( IDX( J )+1 )
            IF( IDXJP.LE.NLP1 ) THEN
               IDXJP = IDXJP - 1
            END IF
            IF( IDXJ.LE.NLP1 ) THEN
               IDXJ = IDXJ - 1
            END IF
            CALL DROT( N, U( 1, IDXJP ), 1, U( 1, IDXJ ), 1, C, S )
            CALL DROT( M, VT( IDXJP, 1 ), LDVT, VT( IDXJ, 1 ), LDVT, C,
     $                 S )
            IF( COLTYP( J ).NE.COLTYP( JPREV ) ) THEN
               COLTYP( J ) = 3
            END IF
            COLTYP( JPREV ) = 4
            K2 = K2 - 1
            IDXP( K2 ) = JPREV
            JPREV = J
         ELSE
            K = K + 1
            U2( K, 1 ) = Z( JPREV )
            DSIGMA( K ) = D( JPREV )
            IDXP( K ) = JPREV
            JPREV = J
         END IF
      END IF
      GO TO 100
  110 CONTINUE
*
*     Record the last singular value.
*
      K = K + 1
      U2( K, 1 ) = Z( JPREV )
      DSIGMA( K ) = D( JPREV )
      IDXP( K ) = JPREV
*
```

```
  120 CONTINUE
*
*     Count up the total number of the various types of columns, then
*     form a permutation which positions the four column types into
*     four groups of uniform structure (although one or more of these
*     groups may be empty).
*
      DO 130 J = 1, 4
         CTOT( J ) = 0
  130 CONTINUE
      DO 140 J = 2, N
         CT = COLTYP( J )
         CTOT( CT ) = CTOT( CT ) + 1
  140 CONTINUE
*
*     PSM(*) = Position in SubMatrix (of types 1 through 4)
*
      PSM( 1 ) = 2
      PSM( 2 ) = 2 + CTOT( 1 )
      PSM( 3 ) = PSM( 2 ) + CTOT( 2 )
      PSM( 4 ) = PSM( 3 ) + CTOT( 3 )
*
*     Fill out the IDXC array so that the permutation which it induces
*     will place all type-1 columns first, all type-2 columns next,
*     then all type-3's, and finally all type-4's, starting from the
*     second column. This applies similarly to the rows of VT.
*
      DO 150 J = 2, N
         JP = IDXP( J )
         CT = COLTYP( JP )
         IDXC( PSM( CT ) ) = J
         PSM( CT ) = PSM( CT ) + 1
  150 CONTINUE
*
*     Sort the singular values and corresponding singular vectors into
*     DSIGMA, U2, and VT2 respectively.  The singular values/vectors
*     which were not deflated go into the first K slots of DSIGMA, U2,
*     and VT2 respectively, while those which were deflated go into the
*     last N - K slots, except that the first column/row will be treated
*     separately.
*
      DO 160 J = 2, N
         JP = IDXP( J )
         DSIGMA( J ) = D( JP )
         IDXJ = IDXQ( IDX( IDXP( IDXC( J ) ) )+1 )
         IF( IDXJ.LE.NLP1 ) THEN
            IDXJ = IDXJ - 1
         END IF
         CALL DCOPY( N, U( 1, IDXJ ), 1, U2( 1, J ), 1 )
         CALL DCOPY( M, VT( IDXJ, 1 ), LDVT, VT2( J, 1 ), LDVT2 )
```

```
  160 CONTINUE
*
*     Determine DSIGMA(1), DSIGMA(2) and Z(1)
*
      DSIGMA( 1 ) = ZERO
      HLFTOL = TOL / TWO
      IF( ABS( DSIGMA( 2 ) ).LE.HLFTOL )
     $   DSIGMA( 2 ) = HLFTOL
      IF( M.GT.N ) THEN
         Z( 1 ) = DLAPY2( Z1, Z( M ) )
         IF( Z( 1 ).LE.TOL ) THEN
            C = ONE
            S = ZERO
            Z( 1 ) = TOL
         ELSE
            C = Z1 / Z( 1 )
            S = Z( M ) / Z( 1 )
         END IF
      ELSE
         IF( ABS( Z1 ).LE.TOL ) THEN
            Z( 1 ) = TOL
         ELSE
            Z( 1 ) = Z1
         END IF
      END IF
*
*     Move the rest of the updating row to Z.
*
      CALL DCOPY( K-1, U2( 2, 1 ), 1, Z( 2 ), 1 )
*
*     Determine the first column of U2, the first row of VT2 and the
*     last row of VT.
*
      CALL DLASET( 'A', N, 1, ZERO, ZERO, U2, LDU2 )
      U2( NLP1, 1 ) = ONE
      IF( M.GT.N ) THEN
         DO 170 I = 1, NLP1
            VT( M, I ) = -S*VT( NLP1, I )
            VT2( 1, I ) = C*VT( NLP1, I )
  170    CONTINUE
         DO 180 I = NLP2, M
            VT2( 1, I ) = S*VT( M, I )
            VT( M, I ) = C*VT( M, I )
  180    CONTINUE
      ELSE
         CALL DCOPY( M, VT( NLP1, 1 ), LDVT, VT2( 1, 1 ), LDVT2 )
      END IF
      IF( M.GT.N ) THEN
         CALL DCOPY( M, VT( M, 1 ), LDVT, VT2( M, 1 ), LDVT2 )
      END IF
```

```
*
*     The deflated singular values and their corresponding vectors go
*     into the back of D, U, and V respectively.
*
      IF( N.GT.K ) THEN
         CALL DCOPY( N-K, DSIGMA( K+1 ), 1, D( K+1 ), 1 )
         CALL DLACPY( 'A', N, N-K, U2( 1, K+1 ), LDU2, U( 1, K+1 ),
     $                LDU )
         CALL DLACPY( 'A', N-K, M, VT2( K+1, 1 ), LDVT2, VT( K+1, 1 ),
     $                LDVT )
      END IF
*
*     Copy CTOT into COLTYP for referencing in DLASD3.
*
      DO 190 J = 1, 4
         COLTYP( J ) = CTOT( J )
  190 CONTINUE
*
      RETURN
*
*     End of DLASD2
*
      END
```

———-

90.35.4 lisp code

— dlasd2.lisp —

———-

90.36 dlasd3

90.36.1 Axiom unit tests

— dlasd3.input —

```
)set break resume
)sys rm -f dlasd3.output
)spool dlasd3.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.36.2 Axiom help page

— **dlasd3.help** —

```
======================================================================
dlasd3 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlasd3 (NL, NR, SQRE, K, D, Q, LDQ, DSIGMA, U, LDU, U2,
           LDU2, VT, LDVT, VT2, LDVT2, IDXC, CTOT, Z, INFO)
           DLASD3 finds all square roots of the roots of the secular equation,
           as defined by the values in D and Z, and then updates the singular
           vectors by matrix multiplication. Used by sbdsdc.

Function/Subroutine Documentation
   subroutine dlasd3 (integer NL, integer NR, integer SQRE, integer K, double
       precision, dimension( * ) D, double precision, dimension( ldq, * ) Q,
       integer LDQ, double precision, dimension( * ) DSIGMA, double precision,
       dimension( ldu, * ) U, integer LDU, double precision, dimension( ldu2,
       * ) U2, integer LDU2, double precision, dimension( ldvt, * ) VT,
       integer LDVT, double precision, dimension( ldvt2, * ) VT2, integer
       LDVT2, integer, dimension( * ) IDXC, integer, dimension( * ) CTOT,
       double precision, dimension( * ) Z, integer INFO)
       DLASD3 finds all square roots of the roots of the secular equation, as
       defined by the values in D and Z, and then updates the singular vectors
       by matrix multiplication. Used by sbdsdc.

Purpose:
   DLASD3 finds all the square roots of the roots of the secular
   equation, as defined by the values in D and Z.  It makes the
   appropriate calls to DLASD4 and then updates the singular
   vectors by matrix multiplication.

   This code makes very mild assumptions about floating point
```

```
   arithmetic. It will work on machines with a guard digit in
   add/subtract, or on those binary machines without guard digits
   which subtract like the Cray XMP, Cray YMP, Cray C 90, or Cray 2.
   It could conceivably fail on hexadecimal or decimal machines
   without guard digits, but we know of none.

   DLASD3 is called from DLASD1.

Parameters:

 NL
   NL is INTEGER
   The row dimension of the upper block.  NL >= 1.

 NR
   NR is INTEGER
   The row dimension of the lower block.  NR >= 1.

 SQRE
   SQRE is INTEGER
   = 0: the lower block is an NR-by-NR square matrix.
   = 1: the lower block is an NR-by-(NR+1) rectangular matrix.

   The bidiagonal matrix has N = NL + NR + 1 rows and
   M = N + SQRE >= N columns.

 K
   K is INTEGER
   The size of the secular equation, 1 =< K = < N.

 D
   D is DOUBLE PRECISION array, dimension(K)
   On exit the square roots of the roots of the secular equation,
   in ascending order.

 Q
   Q is DOUBLE PRECISION array, dimension at least (LDQ,K).

 LDQ
   LDQ is INTEGER
   The leading dimension of the array Q.  LDQ >= K.

 DSIGMA
   DSIGMA is DOUBLE PRECISION array, dimension(K)
   The first K elements of this array contain the old roots
   of the deflated updating problem.  These are the poles
   of the secular equation.

 U
   U is DOUBLE PRECISION array, dimension (LDU, N)
```

```
  The last N - K columns of this matrix contain the deflated
  left singular vectors.

LDU
  LDU is INTEGER
  The leading dimension of the array U.  LDU >= N.

U2
  U2 is DOUBLE PRECISION array, dimension (LDU2, N)
  The first K columns of this matrix contain the non-deflated
  left singular vectors for the split problem.

LDU2
  LDU2 is INTEGER
  The leading dimension of the array U2.  LDU2 >= N.

VT
  VT is DOUBLE PRECISION array, dimension (LDVT, M)
  The last M - K columns of VT**T contain the deflated
  right singular vectors.

LDVT
  LDVT is INTEGER
  The leading dimension of the array VT.  LDVT >= N.

VT2
  VT2 is DOUBLE PRECISION array, dimension (LDVT2, N)
  The first K columns of VT2**T contain the non-deflated
  right singular vectors for the split problem.

LDVT2
  LDVT2 is INTEGER
  The leading dimension of the array VT2.  LDVT2 >= N.

IDXC
  IDXC is INTEGER array, dimension ( N )
  The permutation used to arrange the columns of U (and rows of
  VT) into three groups:  the first group contains non-zero
  entries only at and above (or before) NL +1; the second
  contains non-zero entries only at and below (or after) NL+2;
  and the third is dense. The first column of U and the row of
  VT are treated separately, however.

  The rows of the singular vectors found by DLASD4
  must be likewise permuted before the matrix multiplies can take place.

CTOT
  CTOT is INTEGER array, dimension ( 4 )
  A count of the total number of the various types of columns
  in U (or rows in VT), as described in IDXC. The fourth column
```

```
  type is any column which has been deflated.

Z
  Z is DOUBLE PRECISION array, dimension (K)
  The first K elements of this array contain the components
  of the deflation-adjusted updating row vector.

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if INFO = 1, a singular value did not converge
```

———-

90.36.3 fortran code

— **dlasd3.f** —

```
*  =====================================================================
      SUBROUTINE DLASD3( NL, NR, SQRE, K, D, Q, LDQ, DSIGMA, U, LDU, U2,
     $                   LDU2, VT, LDVT, VT2, LDVT2, IDXC, CTOT, Z,
     $                   INFO )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, K, LDQ, LDU, LDU2, LDVT, LDVT2, NL, NR,
     $                   SQRE
*     ..
*     .. Array Arguments ..
      INTEGER            CTOT( * ), IDXC( * )
      DOUBLE PRECISION   D( * ), DSIGMA( * ), Q( LDQ, * ), U( LDU, * ),
     $                   U2( LDU2, * ), VT( LDVT, * ), VT2( LDVT2, * ),
     $                   Z( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO, NEGONE
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0,
     $                   NEGONE = -1.0D+0 )
*     ..
```

```
*     .. Local Scalars ..
      INTEGER            CTEMP, I, J, JC, KTEMP, M, N, NLP1, NLP2, NRP1
      DOUBLE PRECISION   RHO, TEMP
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMC3, DNRM2
      EXTERNAL           DLAMC3, DNRM2
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DGEMM, DLACPY, DLASCL, DLASD4, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SIGN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( NL.LT.1 ) THEN
         INFO = -1
      ELSE IF( NR.LT.1 ) THEN
         INFO = -2
      ELSE IF( ( SQRE.NE.1 ) .AND. ( SQRE.NE.0 ) ) THEN
         INFO = -3
      END IF
*
      N = NL + NR + 1
      M = N + SQRE
      NLP1 = NL + 1
      NLP2 = NL + 2
*
      IF( ( K.LT.1 ) .OR. ( K.GT.N ) ) THEN
         INFO = -4
      ELSE IF( LDQ.LT.K ) THEN
         INFO = -7
      ELSE IF( LDU.LT.N ) THEN
         INFO = -10
      ELSE IF( LDU2.LT.N ) THEN
         INFO = -12
      ELSE IF( LDVT.LT.M ) THEN
         INFO = -14
      ELSE IF( LDVT2.LT.M ) THEN
         INFO = -16
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASD3', -INFO )
         RETURN
      END IF
```

```
*
*     Quick return if possible
*
      IF( K.EQ.1 ) THEN
         D( 1 ) = ABS( Z( 1 ) )
         CALL DCOPY( M, VT2( 1, 1 ), LDVT2, VT( 1, 1 ), LDVT )
         IF( Z( 1 ).GT.ZERO ) THEN
            CALL DCOPY( N, U2( 1, 1 ), 1, U( 1, 1 ), 1 )
         ELSE
            DO 10 I = 1, N
               U( I, 1 ) = -U2( I, 1 )
   10       CONTINUE
         END IF
         RETURN
      END IF
*
*     Modify values DSIGMA(i) to make sure all DSIGMA(i)-DSIGMA(j) can
*     be computed with high relative accuracy (barring over/underflow).
*     This is a problem on machines without a guard digit in
*     add/subtract (Cray XMP, Cray YMP, Cray C 90 and Cray 2).
*     The following code replaces DSIGMA(I) by 2*DSIGMA(I)-DSIGMA(I),
*     which on any of these machines zeros out the bottommost
*     bit of DSIGMA(I) if it is 1; this makes the subsequent
*     subtractions DSIGMA(I)-DSIGMA(J) unproblematic when cancellation
*     occurs. On binary machines with a guard digit (almost all
*     machines) it does not change DSIGMA(I) at all. On hexadecimal
*     and decimal machines with a guard digit, it slightly
*     changes the bottommost bits of DSIGMA(I). It does not account
*     for hexadecimal or decimal machines without guard digits
*     (we know of none). We use a subroutine call to compute
*     2*DSIGMA(I) to prevent optimizing compilers from eliminating
*     this code.
*
      DO 20 I = 1, K
         DSIGMA( I ) = DLAMC3( DSIGMA( I ), DSIGMA( I ) ) - DSIGMA( I )
   20 CONTINUE
*
*     Keep a copy of Z.
*
      CALL DCOPY( K, Z, 1, Q, 1 )
*
*     Normalize Z.
*
      RHO = DNRM2( K, Z, 1 )
      CALL DLASCL( 'G', 0, 0, RHO, ONE, K, 1, Z, K, INFO )
      RHO = RHO*RHO
*
*     Find the new singular values.
*
      DO 30 J = 1, K
```

```
         CALL DLASD4( K, J, DSIGMA, Z, U( 1, J ), RHO, D( J ),
     $                VT( 1, J ), INFO )
*
*        If the zero finder fails, report the convergence failure.
*
         IF( INFO.NE.0 ) THEN
            RETURN
         END IF
   30 CONTINUE
*
*     Compute updated Z.
*
      DO 60 I = 1, K
         Z( I ) = U( I, K )*VT( I, K )
         DO 40 J = 1, I - 1
            Z( I ) = Z( I )*( U( I, J )*VT( I, J ) /
     $               ( DSIGMA( I )-DSIGMA( J ) ) /
     $               ( DSIGMA( I )+DSIGMA( J ) ) )
   40    CONTINUE
         DO 50 J = I, K - 1
            Z( I ) = Z( I )*( U( I, J )*VT( I, J ) /
     $               ( DSIGMA( I )-DSIGMA( J+1 ) ) /
     $               ( DSIGMA( I )+DSIGMA( J+1 ) ) )
   50    CONTINUE
         Z( I ) = SIGN( SQRT( ABS( Z( I ) ) ), Q( I, 1 ) )
   60 CONTINUE
*
*     Compute left singular vectors of the modified diagonal matrix,
*     and store related information for the right singular vectors.
*
      DO 90 I = 1, K
         VT( 1, I ) = Z( 1 ) / U( 1, I ) / VT( 1, I )
         U( 1, I ) = NEGONE
         DO 70 J = 2, K
            VT( J, I ) = Z( J ) / U( J, I ) / VT( J, I )
            U( J, I ) = DSIGMA( J )*VT( J, I )
   70    CONTINUE
         TEMP = DNRM2( K, U( 1, I ), 1 )
         Q( 1, I ) = U( 1, I ) / TEMP
         DO 80 J = 2, K
            JC = IDXC( J )
            Q( J, I ) = U( JC, I ) / TEMP
   80    CONTINUE
   90 CONTINUE
*
*     Update the left singular vector matrix.
*
      IF( K.EQ.2 ) THEN
         CALL DGEMM( 'N', 'N', N, K, K, ONE, U2, LDU2, Q, LDQ, ZERO, U,
     $               LDU )
```

```
         GO TO 100
      END IF
      IF( CTOT( 1 ).GT.0 ) THEN
         CALL DGEMM( 'N', 'N', NL, K, CTOT( 1 ), ONE, U2( 1, 2 ), LDU2,
     $               Q( 2, 1 ), LDQ, ZERO, U( 1, 1 ), LDU )
         IF( CTOT( 3 ).GT.0 ) THEN
            KTEMP = 2 + CTOT( 1 ) + CTOT( 2 )
            CALL DGEMM( 'N', 'N', NL, K, CTOT( 3 ), ONE, U2( 1, KTEMP ),
     $                  LDU2, Q( KTEMP, 1 ), LDQ, ONE, U( 1, 1 ), LDU )
         END IF
      ELSE IF( CTOT( 3 ).GT.0 ) THEN
         KTEMP = 2 + CTOT( 1 ) + CTOT( 2 )
         CALL DGEMM( 'N', 'N', NL, K, CTOT( 3 ), ONE, U2( 1, KTEMP ),
     $               LDU2, Q( KTEMP, 1 ), LDQ, ZERO, U( 1, 1 ), LDU )
      ELSE
         CALL DLACPY( 'F', NL, K, U2, LDU2, U, LDU )
      END IF
      CALL DCOPY( K, Q( 1, 1 ), LDQ, U( NLP1, 1 ), LDU )
      KTEMP = 2 + CTOT( 1 )
      CTEMP = CTOT( 2 ) + CTOT( 3 )
      CALL DGEMM( 'N', 'N', NR, K, CTEMP, ONE, U2( NLP2, KTEMP ), LDU2,
     $            Q( KTEMP, 1 ), LDQ, ZERO, U( NLP2, 1 ), LDU )
*
*     Generate the right singular vectors.
*
  100 CONTINUE
      DO 120 I = 1, K
         TEMP = DNRM2( K, VT( 1, I ), 1 )
         Q( I, 1 ) = VT( 1, I ) / TEMP
         DO 110 J = 2, K
            JC = IDXC( J )
            Q( I, J ) = VT( JC, I ) / TEMP
  110    CONTINUE
  120 CONTINUE
*
*     Update the right singular vector matrix.
*
      IF( K.EQ.2 ) THEN
         CALL DGEMM( 'N', 'N', K, M, K, ONE, Q, LDQ, VT2, LDVT2, ZERO,
     $               VT, LDVT )
         RETURN
      END IF
      KTEMP = 1 + CTOT( 1 )
      CALL DGEMM( 'N', 'N', K, NLP1, KTEMP, ONE, Q( 1, 1 ), LDQ,
     $            VT2( 1, 1 ), LDVT2, ZERO, VT( 1, 1 ), LDVT )
      KTEMP = 2 + CTOT( 1 ) + CTOT( 2 )
      IF( KTEMP.LE.LDVT2 )
     $   CALL DGEMM( 'N', 'N', K, NLP1, CTOT( 3 ), ONE, Q( 1, KTEMP ),
     $               LDQ, VT2( KTEMP, 1 ), LDVT2, ONE, VT( 1, 1 ),
     $               LDVT )
```

```
*
      KTEMP = CTOT( 1 ) + 1
      NRP1 = NR + SQRE
      IF( KTEMP.GT.1 ) THEN
         DO 130 I = 1, K
            Q( I, KTEMP ) = Q( I, 1 )
  130    CONTINUE
         DO 140 I = NLP2, M
            VT2( KTEMP, I ) = VT2( 1, I )
  140    CONTINUE
      END IF
      CTEMP = 1 + CTOT( 2 ) + CTOT( 3 )
      CALL DGEMM( 'N', 'N', K, NRP1, CTEMP, ONE, Q( 1, KTEMP ), LDQ,
     $            VT2( KTEMP, NLP2 ), LDVT2, ZERO, VT( 1, NLP2 ), LDVT )
*
      RETURN
*
*     End of DLASD3
*
      END
```

———-

90.36.4 lisp code

— dlasd3.lisp —

———-

90.37 dlasd4

90.37.1 Axiom unit tests

— dlasd4.input —

```
)set break resume
)sys rm -f dlasd4.output
)spool dlasd4.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

90.37.2 Axiom help page

— **dlasd4.help** —

```
=====================================================================
dlasd4 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlasd4 (N, I, D, Z, DELTA, RHO, SIGMA, WORK, INFO)
           DLASD4 computes the square root of the i-th updated eigenvalue of a
           positive symmetric rank-one modification to a positive diagonal
           matrix. Used by dbdsdc.

Function/Subroutine Documentation
   subroutine dlasd4 (integer N, integer I, double precision, dimension( * )
       D, double precision, dimension( * ) Z, double precision, dimension( * )
       DELTA, double precision RHO, double precision SIGMA, double precision,
       dimension( * ) WORK, integer INFO)
       DLASD4 computes the square root of the i-th updated eigenvalue of a
       positive symmetric rank-one modification to a positive diagonal matrix.
       Used by dbdsdc.

Purpose:
   This subroutine computes the square root of the I-th updated
   eigenvalue of a positive symmetric rank-one modification to
   a positive diagonal matrix whose entries are given as the squares
   of the corresponding entries in the array d, and that

   0 <= D(i) < D(j)  for  i < j

   and that RHO > 0. This is arranged by the calling routine, and is
   no loss in generality.  The rank-one modified system is thus

   diag( D ) * diag( D ) +  RHO * Z * Z_transpose.

   where we assume the Euclidean norm of Z is 1.
```

```
   The method consists of approximating the rational functions in the
   secular equation by simpler interpolating rational functions.

Parameters:

 N
   N is INTEGER
   The length of all arrays.

 I
   I is INTEGER
   The index of the eigenvalue to be computed.  1 <= I <= N.

 D
   D is DOUBLE PRECISION array, dimension ( N )
   The original eigenvalues.  It is assumed that they are in
   order, 0 <= D(I) < D(J)  for I < J.

 Z
   Z is DOUBLE PRECISION array, dimension ( N )
   The components of the updating vector.

 DELTA
   DELTA is DOUBLE PRECISION array, dimension ( N )
   If N .ne. 1, DELTA contains (D(j) - sigma_I) in its  j-th
   component.  If N = 1, then DELTA(1) = 1.  The vector DELTA
   contains the information necessary to construct the
   (singular) eigenvectors.

 RHO
   RHO is DOUBLE PRECISION
   The scalar in the symmetric updating formula.

 SIGMA
   SIGMA is DOUBLE PRECISION
   The computed sigma_I, the I-th updated eigenvalue.

 WORK
   WORK is DOUBLE PRECISION array, dimension ( N )
   If N .ne. 1, WORK contains (D(j) + sigma_I) in its  j-th
   component.  If N = 1, then WORK( 1 ) = 1.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   > 0:  if INFO = 1, the updating process failed.

Internal Parameters:

   Logical variable ORGATI (origin-at-i?) is used for distinguishing
```

```
whether D(i) or D(i+1) is treated as the origin.

ORGATI = .true.    origin at i
ORGATI = .false.   origin at i+1

Logical variable SWTCH3 (switch-for-3-poles?) is for noting
if we are working with THREE poles!

MAXIT is the maximum number of iterations allowed for each
eigenvalue.
```

———-

90.37.3 fortran code

— dlasd4.f —

```
*  =====================================================================
      SUBROUTINE DLASD4( N, I, D, Z, DELTA, RHO, SIGMA, WORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      INTEGER            I, INFO, N
      DOUBLE PRECISION   RHO, SIGMA
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), DELTA( * ), WORK( * ), Z( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            MAXIT
      PARAMETER          ( MAXIT = 400 )
      DOUBLE PRECISION   ZERO, ONE, TWO, THREE, FOUR, EIGHT, TEN
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0, TWO = 2.0D+0,
     $                   THREE = 3.0D+0, FOUR = 4.0D+0, EIGHT = 8.0D+0,
     $                   TEN = 10.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ORGATI, SWTCH, SWTCH3, GEOMAVG
      INTEGER            II, IIM1, IIP1, IP1, ITER, J, NITER
      DOUBLE PRECISION   A, B, C, DELSQ, DELSQ2, SQ2, DPHI, DPSI, DTIIM,
```

```
     $                   DTIIP, DTIPSQ, DTISQ, DTNSQ, DTNSQ1, DW, EPS,
     $                   ERRETM, ETA, PHI, PREW, PSI, RHOINV, SGLB,
     $                   SGUB, TAU, TAU2, TEMP, TEMP1, TEMP2, W
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   DD( 3 ), ZZ( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLAED6, DLASD5
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Since this routine is called in an inner loop, we do no argument
*     checking.
*
*     Quick return for N=1 and 2.
*
      INFO = 0
      IF( N.EQ.1 ) THEN
*
*        Presumably, I=1 upon entry
*
         SIGMA = SQRT( D( 1 )*D( 1 )+RHO*Z( 1 )*Z( 1 ) )
         DELTA( 1 ) = ONE
         WORK( 1 ) = ONE
         RETURN
      END IF
      IF( N.EQ.2 ) THEN
         CALL DLASD5( I, D, Z, DELTA, RHO, SIGMA, WORK )
         RETURN
      END IF
*
*     Compute machine epsilon
*
      EPS = DLAMCH( 'Epsilon' )
      RHOINV = ONE / RHO
      TAU2= ZERO
*
*     The case I = N
*
      IF( I.EQ.N ) THEN
*
*        Initialize some basic variables
```

```
*
        II = N - 1
        NITER = 1
*
*       Calculate initial guess
*
        TEMP = RHO / TWO
*
*       If ||Z||_2 is not one, then TEMP should be set to
*       RHO * ||Z||_2^2 / TWO
*
        TEMP1 = TEMP / ( D( N )+SQRT( D( N )*D( N )+TEMP ) )
        DO 10 J = 1, N
           WORK( J ) = D( J ) + D( N ) + TEMP1
           DELTA( J ) = ( D( J )-D( N ) ) - TEMP1
   10   CONTINUE
*
        PSI = ZERO
        DO 20 J = 1, N - 2
           PSI = PSI + Z( J )*Z( J ) / ( DELTA( J )*WORK( J ) )
   20   CONTINUE
*
        C = RHOINV + PSI
        W = C + Z( II )*Z( II ) / ( DELTA( II )*WORK( II ) ) +
     $       Z( N )*Z( N ) / ( DELTA( N )*WORK( N ) )
*
        IF( W.LE.ZERO ) THEN
           TEMP1 = SQRT( D( N )*D( N )+RHO )
           TEMP = Z( N-1 )*Z( N-1 ) / ( ( D( N-1 )+TEMP1 )*
     $             ( D( N )-D( N-1 )+RHO / ( D( N )+TEMP1 ) ) ) +
     $             Z( N )*Z( N ) / RHO
*
*          The following TAU2 is to approximate
*          SIGMA_n^2 - D( N )*D( N )
*
           IF( C.LE.TEMP ) THEN
              TAU = RHO
           ELSE
              DELSQ = ( D( N )-D( N-1 ) )*( D( N )+D( N-1 ) )
              A = -C*DELSQ + Z( N-1 )*Z( N-1 ) + Z( N )*Z( N )
              B = Z( N )*Z( N )*DELSQ
              IF( A.LT.ZERO ) THEN
                 TAU2 = TWO*B / ( SQRT( A*A+FOUR*B*C )-A )
              ELSE
                 TAU2 = ( A+SQRT( A*A+FOUR*B*C ) ) / ( TWO*C )
              END IF
              TAU = TAU2 / ( D( N )+SQRT( D( N )*D( N )+TAU2 ) )
           END IF
*
*          It can be proved that
```

```
*              D(N)^2+RHO/2 <= SIGMA_n^2 < D(N)^2+TAU2 <= D(N)^2+RHO
*
         ELSE
            DELSQ = ( D( N )-D( N-1 ) )*( D( N )+D( N-1 ) )
            A = -C*DELSQ + Z( N-1 )*Z( N-1 ) + Z( N )*Z( N )
            B = Z( N )*Z( N )*DELSQ
*
*           The following TAU2 is to approximate
*           SIGMA_n^2 - D( N )*D( N )
*
            IF( A.LT.ZERO ) THEN
               TAU2 = TWO*B / ( SQRT( A*A+FOUR*B*C )-A )
            ELSE
               TAU2 = ( A+SQRT( A*A+FOUR*B*C ) ) / ( TWO*C )
            END IF
            TAU = TAU2 / ( D( N )+SQRT( D( N )*D( N )+TAU2 ) )

*
*           It can be proved that
*           D(N)^2 < D(N)^2+TAU2 < SIGMA(N)^2 < D(N)^2+RHO/2
*
         END IF
*
*        The following TAU is to approximate SIGMA_n - D( N )
*
*         TAU = TAU2 / ( D( N )+SQRT( D( N )*D( N )+TAU2 ) )
*
         SIGMA = D( N ) + TAU
         DO 30 J = 1, N
            DELTA( J ) = ( D( J )-D( N ) ) - TAU
            WORK( J ) = D( J ) + D( N ) + TAU
   30    CONTINUE
*
*        Evaluate PSI and the derivative DPSI
*
         DPSI = ZERO
         PSI = ZERO
         ERRETM = ZERO
         DO 40 J = 1, II
            TEMP = Z( J ) / ( DELTA( J )*WORK( J ) )
            PSI = PSI + Z( J )*TEMP
            DPSI = DPSI + TEMP*TEMP
            ERRETM = ERRETM + PSI
   40    CONTINUE
         ERRETM = ABS( ERRETM )
*
*        Evaluate PHI and the derivative DPHI
*
         TEMP = Z( N ) / ( DELTA( N )*WORK( N ) )
         PHI = Z( N )*TEMP
```

```
         DPHI = TEMP*TEMP
         ERRETM = EIGHT*( -PHI-PSI ) + ERRETM - PHI + RHOINV
*    $          + ABS( TAU2 )*( DPSI+DPHI )
*
         W = RHOINV + PHI + PSI
*
*        Test for convergence
*
         IF( ABS( W ).LE.EPS*ERRETM ) THEN
            GO TO 240
         END IF
*
*        Calculate the new step
*
         NITER = NITER + 1
         DTNSQ1 = WORK( N-1 )*DELTA( N-1 )
         DTNSQ = WORK( N )*DELTA( N )
         C = W - DTNSQ1*DPSI - DTNSQ*DPHI
         A = ( DTNSQ+DTNSQ1 )*W - DTNSQ*DTNSQ1*( DPSI+DPHI )
         B = DTNSQ*DTNSQ1*W
         IF( C.LT.ZERO )
     $      C = ABS( C )
         IF( C.EQ.ZERO ) THEN
            ETA = RHO - SIGMA*SIGMA
         ELSE IF( A.GE.ZERO ) THEN
            ETA = ( A+SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
         ELSE
            ETA = TWO*B / ( A-SQRT( ABS( A*A-FOUR*B*C ) ) )
         END IF
*
*        Note, eta should be positive if w is negative, and
*        eta should be negative otherwise. However,
*        if for some reason caused by roundoff, eta*w > 0,
*        we simply use one Newton step instead. This way
*        will guarantee eta*w < 0.
*
         IF( W*ETA.GT.ZERO )
     $      ETA = -W / ( DPSI+DPHI )
         TEMP = ETA - DTNSQ
         IF( TEMP.GT.RHO )
     $      ETA = RHO + DTNSQ
*
         ETA = ETA / ( SIGMA+SQRT( ETA+SIGMA*SIGMA ) )
         TAU = TAU + ETA
         SIGMA = SIGMA + ETA
*
         DO 50 J = 1, N
            DELTA( J ) = DELTA( J ) - ETA
            WORK( J ) = WORK( J ) + ETA
   50    CONTINUE
```

```
*
*        Evaluate PSI and the derivative DPSI
*
         DPSI = ZERO
         PSI = ZERO
         ERRETM = ZERO
         DO 60 J = 1, II
            TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
            PSI = PSI + Z( J )*TEMP
            DPSI = DPSI + TEMP*TEMP
            ERRETM = ERRETM + PSI
   60    CONTINUE
         ERRETM = ABS( ERRETM )
*
*        Evaluate PHI and the derivative DPHI
*
         TAU2 = WORK( N )*DELTA( N )
         TEMP = Z( N ) / TAU2
         PHI = Z( N )*TEMP
         DPHI = TEMP*TEMP
         ERRETM = EIGHT*( -PHI-PSI ) + ERRETM - PHI + RHOINV
*    $          + ABS( TAU2 )*( DPSI+DPHI )
*
         W = RHOINV + PHI + PSI
*
*        Main loop to update the values of the array   DELTA
*
         ITER = NITER + 1
*
         DO 90 NITER = ITER, MAXIT
*
*           Test for convergence
*
            IF( ABS( W ).LE.EPS*ERRETM ) THEN
               GO TO 240
            END IF
*
*           Calculate the new step
*
            DTNSQ1 = WORK( N-1 )*DELTA( N-1 )
            DTNSQ = WORK( N )*DELTA( N )
            C = W - DTNSQ1*DPSI - DTNSQ*DPHI
            A = ( DTNSQ+DTNSQ1 )*W - DTNSQ1*DTNSQ*( DPSI+DPHI )
            B = DTNSQ1*DTNSQ*W
            IF( A.GE.ZERO ) THEN
               ETA = ( A+SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
            ELSE
               ETA = TWO*B / ( A-SQRT( ABS( A*A-FOUR*B*C ) ) )
            END IF
*
```

```
*          Note, eta should be positive if w is negative, and
*          eta should be negative otherwise. However,
*          if for some reason caused by roundoff, eta*w > 0,
*          we simply use one Newton step instead. This way
*          will guarantee eta*w < 0.
*
           IF( W*ETA.GT.ZERO )
     $        ETA = -W / ( DPSI+DPHI )
           TEMP = ETA - DTNSQ
           IF( TEMP.LE.ZERO )
     $        ETA = ETA / TWO
*
           ETA = ETA / ( SIGMA+SQRT( ETA+SIGMA*SIGMA ) )
           TAU = TAU + ETA
           SIGMA = SIGMA + ETA
*
           DO 70 J = 1, N
              DELTA( J ) = DELTA( J ) - ETA
              WORK( J ) = WORK( J ) + ETA
   70      CONTINUE
*
*          Evaluate PSI and the derivative DPSI
*
           DPSI = ZERO
           PSI = ZERO
           ERRETM = ZERO
           DO 80 J = 1, II
              TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
              PSI = PSI + Z( J )*TEMP
              DPSI = DPSI + TEMP*TEMP
              ERRETM = ERRETM + PSI
   80      CONTINUE
           ERRETM = ABS( ERRETM )
*
*          Evaluate PHI and the derivative DPHI
*
           TAU2 = WORK( N )*DELTA( N )
           TEMP = Z( N ) / TAU2
           PHI = Z( N )*TEMP
           DPHI = TEMP*TEMP
           ERRETM = EIGHT*( -PHI-PSI ) + ERRETM - PHI + RHOINV
*    $            + ABS( TAU2 )*( DPSI+DPHI )
*
           W = RHOINV + PHI + PSI
   90    CONTINUE
*
*        Return with INFO = 1, NITER = MAXIT and not converged
*
         INFO = 1
         GO TO 240
```

```
*
*        End for the case I = N
*
      ELSE
*
*        The case for I < N
*
         NITER = 1
         IP1 = I + 1
*
*        Calculate initial guess
*
         DELSQ = ( D( IP1 )-D( I ) )*( D( IP1 )+D( I ) )
         DELSQ2 = DELSQ / TWO
         SQ2=SQRT( ( D( I )*D( I )+D( IP1 )*D( IP1 ) ) / TWO )
         TEMP = DELSQ2 / ( D( I )+SQ2 )
         DO 100 J = 1, N
            WORK( J ) = D( J ) + D( I ) + TEMP
            DELTA( J ) = ( D( J )-D( I ) ) - TEMP
  100    CONTINUE
*
         PSI = ZERO
         DO 110 J = 1, I - 1
            PSI = PSI + Z( J )*Z( J ) / ( WORK( J )*DELTA( J ) )
  110    CONTINUE
*
         PHI = ZERO
         DO 120 J = N, I + 2, -1
            PHI = PHI + Z( J )*Z( J ) / ( WORK( J )*DELTA( J ) )
  120    CONTINUE
         C = RHOINV + PSI + PHI
         W = C + Z( I )*Z( I ) / ( WORK( I )*DELTA( I ) ) +
     $       Z( IP1 )*Z( IP1 ) / ( WORK( IP1 )*DELTA( IP1 ) )
*
         GEOMAVG = .FALSE.
         IF( W.GT.ZERO ) THEN
*
*           d(i)^2 < the ith sigma^2 < (d(i)^2+d(i+1)^2)/2
*
*           We choose d(i) as origin.
*
            ORGATI = .TRUE.
            II = I
            SGLB = ZERO
            SGUB = DELSQ2  / ( D( I )+SQ2 )
            A = C*DELSQ + Z( I )*Z( I ) + Z( IP1 )*Z( IP1 )
            B = Z( I )*Z( I )*DELSQ
            IF( A.GT.ZERO ) THEN
               TAU2 = TWO*B / ( A+SQRT( ABS( A*A-FOUR*B*C ) ) )
            ELSE
```

```
            TAU2 = ( A-SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
         END IF
*
*        TAU2 now is an estimation of SIGMA^2 - D( I )^2. The
*        following, however, is the corresponding estimation of
*        SIGMA - D( I ).
*
         TAU = TAU2 / ( D( I )+SQRT( D( I )*D( I )+TAU2 ) )
         TEMP = SQRT(EPS)
         IF( (D(I).LE.TEMP*D(IP1)).AND.(ABS(Z(I)).LE.TEMP)
     $                              .AND.(D(I).GT.ZERO) ) THEN
            TAU = MIN( TEN*D(I), SGUB )
            GEOMAVG = .TRUE.
         END IF
      ELSE
*
*        (d(i)^2+d(i+1)^2)/2 <= the ith sigma^2 < d(i+1)^2/2
*
*        We choose d(i+1) as origin.
*
         ORGATI = .FALSE.
         II = IP1
         SGLB = -DELSQ2  / ( D( II )+SQ2 )
         SGUB = ZERO
         A = C*DELSQ - Z( I )*Z( I ) - Z( IP1 )*Z( IP1 )
         B = Z( IP1 )*Z( IP1 )*DELSQ
         IF( A.LT.ZERO ) THEN
            TAU2 = TWO*B / ( A-SQRT( ABS( A*A+FOUR*B*C ) ) )
         ELSE
            TAU2 = -( A+SQRT( ABS( A*A+FOUR*B*C ) ) ) / ( TWO*C )
         END IF
*
*        TAU2 now is an estimation of SIGMA^2 - D( IP1 )^2. The
*        following, however, is the corresponding estimation of
*        SIGMA - D( IP1 ).
*
         TAU = TAU2 / ( D( IP1 )+SQRT( ABS( D( IP1 )*D( IP1 )+
     $           TAU2 ) ) )
      END IF
*
      SIGMA = D( II ) + TAU
      DO 130 J = 1, N
         WORK( J ) = D( J ) + D( II ) + TAU
         DELTA( J ) = ( D( J )-D( II ) ) - TAU
  130 CONTINUE
      IIM1 = II - 1
      IIP1 = II + 1
*
*     Evaluate PSI and the derivative DPSI
*
```

```
         DPSI = ZERO
         PSI = ZERO
         ERRETM = ZERO
         DO 150 J = 1, IIM1
            TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
            PSI = PSI + Z( J )*TEMP
            DPSI = DPSI + TEMP*TEMP
            ERRETM = ERRETM + PSI
  150    CONTINUE
         ERRETM = ABS( ERRETM )
*
*        Evaluate PHI and the derivative DPHI
*
         DPHI = ZERO
         PHI = ZERO
         DO 160 J = N, IIP1, -1
            TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
            PHI = PHI + Z( J )*TEMP
            DPHI = DPHI + TEMP*TEMP
            ERRETM = ERRETM + PHI
  160    CONTINUE
*
         W = RHOINV + PHI + PSI
*
*        W is the value of the secular function with
*        its ii-th element removed.
*
         SWTCH3 = .FALSE.
         IF( ORGATI ) THEN
            IF( W.LT.ZERO )
     $         SWTCH3 = .TRUE.
         ELSE
            IF( W.GT.ZERO )
     $         SWTCH3 = .TRUE.
         END IF
         IF( II.EQ.1 .OR. II.EQ.N )
     $      SWTCH3 = .FALSE.
*
         TEMP = Z( II ) / ( WORK( II )*DELTA( II ) )
         DW = DPSI + DPHI + TEMP*TEMP
         TEMP = Z( II )*TEMP
         W = W + TEMP
         ERRETM = EIGHT*( PHI-PSI ) + ERRETM + TWO*RHOINV
     $          + THREE*ABS( TEMP )
*    $          + ABS( TAU2 )*DW
*
*        Test for convergence
*
         IF( ABS( W ).LE.EPS*ERRETM ) THEN
            GO TO 240
```

```
         END IF
*
         IF( W.LE.ZERO ) THEN
            SGLB = MAX( SGLB, TAU )
         ELSE
            SGUB = MIN( SGUB, TAU )
         END IF
*
*        Calculate the new step
*
         NITER = NITER + 1
         IF( .NOT.SWTCH3 ) THEN
            DTIPSQ = WORK( IP1 )*DELTA( IP1 )
            DTISQ = WORK( I )*DELTA( I )
            IF( ORGATI ) THEN
               C = W - DTIPSQ*DW + DELSQ*( Z( I ) / DTISQ )**2
            ELSE
               C = W - DTISQ*DW - DELSQ*( Z( IP1 ) / DTIPSQ )**2
            END IF
            A = ( DTIPSQ+DTISQ )*W - DTIPSQ*DTISQ*DW
            B = DTIPSQ*DTISQ*W
            IF( C.EQ.ZERO ) THEN
               IF( A.EQ.ZERO ) THEN
                  IF( ORGATI ) THEN
                     A = Z( I )*Z( I ) + DTIPSQ*DTIPSQ*( DPSI+DPHI )
                  ELSE
                     A = Z( IP1 )*Z( IP1 ) + DTISQ*DTISQ*( DPSI+DPHI )
                  END IF
               END IF
               ETA = B / A
            ELSE IF( A.LE.ZERO ) THEN
               ETA = ( A-SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
            ELSE
               ETA = TWO*B / ( A+SQRT( ABS( A*A-FOUR*B*C ) ) )
            END IF
         ELSE
*
*           Interpolation using THREE most relevant poles
*
            DTIIM = WORK( IIM1 )*DELTA( IIM1 )
            DTIIP = WORK( IIP1 )*DELTA( IIP1 )
            TEMP = RHOINV + PSI + PHI
            IF( ORGATI ) THEN
               TEMP1 = Z( IIM1 ) / DTIIM
               TEMP1 = TEMP1*TEMP1
               C = ( TEMP - DTIIP*( DPSI+DPHI ) ) -
     $             ( D( IIM1 )-D( IIP1 ) )*( D( IIM1 )+D( IIP1 ) )*TEMP1
               ZZ( 1 ) = Z( IIM1 )*Z( IIM1 )
               IF( DPSI.LT.TEMP1 ) THEN
                  ZZ( 3 ) = DTIIP*DTIIP*DPHI
```

```
            ELSE
               ZZ( 3 ) = DTIIP*DTIIP*( ( DPSI-TEMP1 )+DPHI )
            END IF
         ELSE
            TEMP1 = Z( IIP1 ) / DTIIP
            TEMP1 = TEMP1*TEMP1
            C = ( TEMP - DTIIM*( DPSI+DPHI ) ) -
     $          ( D( IIP1 )-D( IIM1 ) )*( D( IIM1 )+D( IIP1 ) )*TEMP1
            IF( DPHI.LT.TEMP1 ) THEN
               ZZ( 1 ) = DTIIM*DTIIM*DPSI
            ELSE
               ZZ( 1 ) = DTIIM*DTIIM*( DPSI+( DPHI-TEMP1 ) )
            END IF
            ZZ( 3 ) = Z( IIP1 )*Z( IIP1 )
         END IF
         ZZ( 2 ) = Z( II )*Z( II )
         DD( 1 ) = DTIIM
         DD( 2 ) = DELTA( II )*WORK( II )
         DD( 3 ) = DTIIP
         CALL DLAED6( NITER, ORGATI, C, DD, ZZ, W, ETA, INFO )
*
         IF( INFO.NE.0 ) THEN
*
*           If INFO is not 0, i.e., DLAED6 failed, switch back
*           to 2 pole interpolation.
*
            SWTCH3 = .FALSE.
            INFO = 0
            DTIPSQ = WORK( IP1 )*DELTA( IP1 )
            DTISQ = WORK( I )*DELTA( I )
            IF( ORGATI ) THEN
               C = W - DTIPSQ*DW + DELSQ*( Z( I ) / DTISQ )**2
            ELSE
               C = W - DTISQ*DW - DELSQ*( Z( IP1 ) / DTIPSQ )**2
            END IF
            A = ( DTIPSQ+DTISQ )*W - DTIPSQ*DTISQ*DW
            B = DTIPSQ*DTISQ*W
            IF( C.EQ.ZERO ) THEN
               IF( A.EQ.ZERO ) THEN
                  IF( ORGATI ) THEN
                     A = Z( I )*Z( I ) + DTIPSQ*DTIPSQ*( DPSI+DPHI )
                  ELSE
                     A = Z( IP1 )*Z( IP1 ) + DTISQ*DTISQ*( DPSI+DPHI)
                  END IF
               END IF
               ETA = B / A
            ELSE IF( A.LE.ZERO ) THEN
               ETA = ( A-SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
            ELSE
               ETA = TWO*B / ( A+SQRT( ABS( A*A-FOUR*B*C ) ) )
```

```
            END IF
         END IF
      END IF
*
*        Note, eta should be positive if w is negative, and
*        eta should be negative otherwise. However,
*        if for some reason caused by roundoff, eta*w > 0,
*        we simply use one Newton step instead. This way
*        will guarantee eta*w < 0.
*
         IF( W*ETA.GE.ZERO )
     $      ETA = -W / DW
*
         ETA = ETA / ( SIGMA+SQRT( SIGMA*SIGMA+ETA ) )
         TEMP = TAU + ETA
         IF( TEMP.GT.SGUB .OR. TEMP.LT.SGLB ) THEN
            IF( W.LT.ZERO ) THEN
               ETA = ( SGUB-TAU ) / TWO
            ELSE
               ETA = ( SGLB-TAU ) / TWO
            END IF
            IF( GEOMAVG ) THEN
               IF( W .LT. ZERO ) THEN
                  IF( TAU .GT. ZERO ) THEN
                     ETA = SQRT(SGUB*TAU)-TAU
                  END IF
               ELSE
                  IF( SGLB .GT. ZERO ) THEN
                     ETA = SQRT(SGLB*TAU)-TAU
                  END IF
               END IF
            END IF
         END IF
*
         PREW = W
*
         TAU = TAU + ETA
         SIGMA = SIGMA + ETA
*
         DO 170 J = 1, N
            WORK( J ) = WORK( J ) + ETA
            DELTA( J ) = DELTA( J ) - ETA
  170    CONTINUE
*
*        Evaluate PSI and the derivative DPSI
*
         DPSI = ZERO
         PSI = ZERO
         ERRETM = ZERO
         DO 180 J = 1, IIM1
```

```
            TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
            PSI = PSI + Z( J )*TEMP
            DPSI = DPSI + TEMP*TEMP
            ERRETM = ERRETM + PSI
  180    CONTINUE
         ERRETM = ABS( ERRETM )
*
*        Evaluate PHI and the derivative DPHI
*
         DPHI = ZERO
         PHI = ZERO
         DO 190 J = N, IIP1, -1
            TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
            PHI = PHI + Z( J )*TEMP
            DPHI = DPHI + TEMP*TEMP
            ERRETM = ERRETM + PHI
  190    CONTINUE
*
         TAU2 = WORK( II )*DELTA( II )
         TEMP = Z( II ) / TAU2
         DW = DPSI + DPHI + TEMP*TEMP
         TEMP = Z( II )*TEMP
         W = RHOINV + PHI + PSI + TEMP
         ERRETM = EIGHT*( PHI-PSI ) + ERRETM + TWO*RHOINV
     $          + THREE*ABS( TEMP )
*    $          + ABS( TAU2 )*DW
*
         SWTCH = .FALSE.
         IF( ORGATI ) THEN
            IF( -W.GT.ABS( PREW ) / TEN )
     $         SWTCH = .TRUE.
         ELSE
            IF( W.GT.ABS( PREW ) / TEN )
     $         SWTCH = .TRUE.
         END IF
*
*        Main loop to update the values of the array   DELTA and WORK
*
         ITER = NITER + 1
*
         DO 230 NITER = ITER, MAXIT
*
*           Test for convergence
*
            IF( ABS( W ).LE.EPS*ERRETM ) THEN
*     $          .OR. (SGUB-SGLB).LE.EIGHT*ABS(SGUB+SGLB) ) THEN
               GO TO 240
            END IF
*
            IF( W.LE.ZERO ) THEN
```

```
            SGLB = MAX( SGLB, TAU )
         ELSE
            SGUB = MIN( SGUB, TAU )
         END IF
*
*        Calculate the new step
*
         IF( .NOT.SWTCH3 ) THEN
            DTIPSQ = WORK( IP1 )*DELTA( IP1 )
            DTISQ = WORK( I )*DELTA( I )
            IF( .NOT.SWTCH ) THEN
               IF( ORGATI ) THEN
                  C = W - DTIPSQ*DW + DELSQ*( Z( I ) / DTISQ )**2
               ELSE
                  C = W - DTISQ*DW - DELSQ*( Z( IP1 ) / DTIPSQ )**2
               END IF
            ELSE
               TEMP = Z( II ) / ( WORK( II )*DELTA( II ) )
               IF( ORGATI ) THEN
                  DPSI = DPSI + TEMP*TEMP
               ELSE
                  DPHI = DPHI + TEMP*TEMP
               END IF
               C = W - DTISQ*DPSI - DTIPSQ*DPHI
            END IF
            A = ( DTIPSQ+DTISQ )*W - DTIPSQ*DTISQ*DW
            B = DTIPSQ*DTISQ*W
            IF( C.EQ.ZERO ) THEN
               IF( A.EQ.ZERO ) THEN
                  IF( .NOT.SWTCH ) THEN
                     IF( ORGATI ) THEN
                        A = Z( I )*Z( I ) + DTIPSQ*DTIPSQ*
     $                      ( DPSI+DPHI )
                     ELSE
                        A = Z( IP1 )*Z( IP1 ) +
     $                      DTISQ*DTISQ*( DPSI+DPHI )
                     END IF
                  ELSE
                     A = DTISQ*DTISQ*DPSI + DTIPSQ*DTIPSQ*DPHI
                  END IF
               END IF
               ETA = B / A
            ELSE IF( A.LE.ZERO ) THEN
               ETA = ( A-SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
            ELSE
               ETA = TWO*B / ( A+SQRT( ABS( A*A-FOUR*B*C ) ) )
            END IF
         ELSE
*
*           Interpolation using THREE most relevant poles
```

```
*
               DTIIM = WORK( IIM1 )*DELTA( IIM1 )
               DTIIP = WORK( IIP1 )*DELTA( IIP1 )
               TEMP = RHOINV + PSI + PHI
               IF( SWTCH ) THEN
                  C = TEMP - DTIIM*DPSI - DTIIP*DPHI
                  ZZ( 1 ) = DTIIM*DTIIM*DPSI
                  ZZ( 3 ) = DTIIP*DTIIP*DPHI
               ELSE
                  IF( ORGATI ) THEN
                     TEMP1 = Z( IIM1 ) / DTIIM
                     TEMP1 = TEMP1*TEMP1
                     TEMP2 = ( D( IIM1 )-D( IIP1 ) )*
     $                       ( D( IIM1 )+D( IIP1 ) )*TEMP1
                     C = TEMP - DTIIP*( DPSI+DPHI ) - TEMP2
                     ZZ( 1 ) = Z( IIM1 )*Z( IIM1 )
                     IF( DPSI.LT.TEMP1 ) THEN
                        ZZ( 3 ) = DTIIP*DTIIP*DPHI
                     ELSE
                        ZZ( 3 ) = DTIIP*DTIIP*( ( DPSI-TEMP1 )+DPHI )
                     END IF
                  ELSE
                     TEMP1 = Z( IIP1 ) / DTIIP
                     TEMP1 = TEMP1*TEMP1
                     TEMP2 = ( D( IIP1 )-D( IIM1 ) )*
     $                       ( D( IIM1 )+D( IIP1 ) )*TEMP1
                     C = TEMP - DTIIM*( DPSI+DPHI ) - TEMP2
                     IF( DPHI.LT.TEMP1 ) THEN
                        ZZ( 1 ) = DTIIM*DTIIM*DPSI
                     ELSE
                        ZZ( 1 ) = DTIIM*DTIIM*( DPSI+( DPHI-TEMP1 ) )
                     END IF
                     ZZ( 3 ) = Z( IIP1 )*Z( IIP1 )
                  END IF
               END IF
               DD( 1 ) = DTIIM
               DD( 2 ) = DELTA( II )*WORK( II )
               DD( 3 ) = DTIIP
               CALL DLAED6( NITER, ORGATI, C, DD, ZZ, W, ETA, INFO )
*
               IF( INFO.NE.0 ) THEN
*
*                 If INFO is not 0, i.e., DLAED6 failed, switch
*                 back to two pole interpolation
*
                  SWTCH3 = .FALSE.
                  INFO = 0
                  DTIPSQ = WORK( IP1 )*DELTA( IP1 )
                  DTISQ = WORK( I )*DELTA( I )
                  IF( .NOT.SWTCH ) THEN
```

```
                  IF( ORGATI ) THEN
                     C = W - DTIPSQ*DW + DELSQ*( Z( I )/DTISQ )**2
                  ELSE
                     C = W - DTISQ*DW - DELSQ*( Z( IP1 )/DTIPSQ )**2
                  END IF
               ELSE
                  TEMP = Z( II ) / ( WORK( II )*DELTA( II ) )
                  IF( ORGATI ) THEN
                     DPSI = DPSI + TEMP*TEMP
                  ELSE
                     DPHI = DPHI + TEMP*TEMP
                  END IF
                  C = W - DTISQ*DPSI - DTIPSQ*DPHI
               END IF
               A = ( DTIPSQ+DTISQ )*W - DTIPSQ*DTISQ*DW
               B = DTIPSQ*DTISQ*W
               IF( C.EQ.ZERO ) THEN
                  IF( A.EQ.ZERO ) THEN
                     IF( .NOT.SWTCH ) THEN
                        IF( ORGATI ) THEN
                           A = Z( I )*Z( I ) + DTIPSQ*DTIPSQ*
     $                         ( DPSI+DPHI )
                        ELSE
                           A = Z( IP1 )*Z( IP1 ) +
     $                         DTISQ*DTISQ*( DPSI+DPHI )
                        END IF
                     ELSE
                        A = DTISQ*DTISQ*DPSI + DTIPSQ*DTIPSQ*DPHI
                     END IF
                  END IF
                  ETA = B / A
               ELSE IF( A.LE.ZERO ) THEN
                  ETA = ( A-SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
               ELSE
                  ETA = TWO*B / ( A+SQRT( ABS( A*A-FOUR*B*C ) ) )
               END IF
            END IF
         END IF
*
*        Note, eta should be positive if w is negative, and
*        eta should be negative otherwise. However,
*        if for some reason caused by roundoff, eta*w > 0,
*        we simply use one Newton step instead. This way
*        will guarantee eta*w < 0.
*
         IF( W*ETA.GE.ZERO )
     $      ETA = -W / DW
*
         ETA = ETA / ( SIGMA+SQRT( SIGMA*SIGMA+ETA ) )
         TEMP=TAU+ETA
```

```
            IF( TEMP.GT.SGUB .OR. TEMP.LT.SGLB ) THEN
               IF( W.LT.ZERO ) THEN
                  ETA = ( SGUB-TAU ) / TWO
               ELSE
                  ETA = ( SGLB-TAU ) / TWO
               END IF
               IF( GEOMAVG ) THEN
                  IF( W .LT. ZERO ) THEN
                     IF( TAU .GT. ZERO ) THEN
                        ETA = SQRT(SGUB*TAU)-TAU
                     END IF
                  ELSE
                     IF( SGLB .GT. ZERO ) THEN
                        ETA = SQRT(SGLB*TAU)-TAU
                     END IF
                  END IF
               END IF
            END IF
*
            PREW = W
*
            TAU = TAU + ETA
            SIGMA = SIGMA + ETA
*
            DO 200 J = 1, N
               WORK( J ) = WORK( J ) + ETA
               DELTA( J ) = DELTA( J ) - ETA
  200       CONTINUE
*
*           Evaluate PSI and the derivative DPSI
*
            DPSI = ZERO
            PSI = ZERO
            ERRETM = ZERO
            DO 210 J = 1, IIM1
               TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
               PSI = PSI + Z( J )*TEMP
               DPSI = DPSI + TEMP*TEMP
               ERRETM = ERRETM + PSI
  210       CONTINUE
            ERRETM = ABS( ERRETM )
*
*           Evaluate PHI and the derivative DPHI
*
            DPHI = ZERO
            PHI = ZERO
            DO 220 J = N, IIP1, -1
               TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
               PHI = PHI + Z( J )*TEMP
               DPHI = DPHI + TEMP*TEMP
```

```
               ERRETM = ERRETM + PHI
  220       CONTINUE
*
            TAU2 = WORK( II )*DELTA( II )
            TEMP = Z( II ) / TAU2
            DW = DPSI + DPHI + TEMP*TEMP
            TEMP = Z( II )*TEMP
            W = RHOINV + PHI + PSI + TEMP
            ERRETM = EIGHT*( PHI-PSI ) + ERRETM + TWO*RHOINV
     $             + THREE*ABS( TEMP )
*    $             + ABS( TAU2 )*DW
*
            IF( W*PREW.GT.ZERO .AND. ABS( W ).GT.ABS( PREW ) / TEN )
     $         SWTCH = .NOT.SWTCH
*
  230    CONTINUE
*
*        Return with INFO = 1, NITER = MAXIT and not converged
*
         INFO = 1
*
      END IF
*
  240 CONTINUE
      RETURN
*
*     End of DLASD4
*
      END
```

———-

90.37.4 lisp code

— dlasd4.lisp —

———-

90.38 dlasd5

90.38.1 Axiom unit tests

— dlasd5.input —

```
)set break resume
)sys rm -f dlasd5.output
)spool dlasd5.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.38.2 Axiom help page

— **dlasd5.help** —

```
======================================================================
dlasd5 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlasd5 (I, D, Z, DELTA, RHO, DSIGMA, WORK)
           DLASD5 computes the square root of the i-th eigenvalue of a
           positive symmetric rank-one modification of a 2-by-2 diagonal
           matrix. Used by sbdsdc.

Function/Subroutine Documentation
   subroutine dlasd5 (integer I, double precision, dimension( 2 ) D, double
       precision, dimension( 2 ) Z, double precision, dimension( 2 ) DELTA,
       double precision RHO, double precision DSIGMA, double precision,
       dimension( 2 ) WORK)
       DLASD5 computes the square root of the i-th eigenvalue of a positive
       symmetric rank-one modification of a 2-by-2 diagonal matrix. Used by
       sbdsdc.

Purpose:
   This subroutine computes the square root of the I-th eigenvalue
   of a positive symmetric rank-one modification of a 2-by-2 diagonal
   matrix

   diag( D ) * diag( D ) +  RHO * Z * transpose(Z) .

   The diagonal entries in the array D are assumed to satisfy
```

```
   0 <= D(i) < D(j)  for  i < j .

   We also assume RHO > 0 and that the Euclidean norm of the vector Z is one.

Parameters:

 I
   I is INTEGER
   The index of the eigenvalue to be computed.  I = 1 or I = 2.

 D
   D is DOUBLE PRECISION array, dimension ( 2 )
   The original eigenvalues.  We assume 0 <= D(1) < D(2).

 Z
   Z is DOUBLE PRECISION array, dimension ( 2 )
   The components of the updating vector.

 DELTA
   DELTA is DOUBLE PRECISION array, dimension ( 2 )
   Contains (D(j) - sigma_I) in its  j-th component.
   The vector DELTA contains the information necessary
   to construct the eigenvectors.

 RHO
   RHO is DOUBLE PRECISION
   The scalar in the symmetric updating formula.

 DSIGMA
   DSIGMA is DOUBLE PRECISION
   The computed sigma_I, the I-th updated eigenvalue.

 WORK
   WORK is DOUBLE PRECISION array, dimension ( 2 )
   WORK contains (D(j) + sigma_I) in its  j-th component.
```

———-

90.38.3 fortran code

— dlasd5.f —

```
*  =====================================================================
      SUBROUTINE DLASD5( I, D, Z, DELTA, RHO, DSIGMA, WORK )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
```

```
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            I
      DOUBLE PRECISION   DSIGMA, RHO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( 2 ), DELTA( 2 ), WORK( 2 ), Z( 2 )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE, TWO, THREE, FOUR
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0, TWO = 2.0D+0,
     $                   THREE = 3.0D+0, FOUR = 4.0D+0 )
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION   B, C, DEL, DELSQ, TAU, W
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SQRT
*     ..
*     .. Executable Statements ..
*
      DEL = D( 2 ) - D( 1 )
      DELSQ = DEL*( D( 2 )+D( 1 ) )
      IF( I.EQ.1 ) THEN
         W = ONE + FOUR*RHO*( Z( 2 )*Z( 2 ) / ( D( 1 )+THREE*D( 2 ) )-
     $       Z( 1 )*Z( 1 ) / ( THREE*D( 1 )+D( 2 ) ) ) / DEL
         IF( W.GT.ZERO ) THEN
            B = DELSQ + RHO*( Z( 1 )*Z( 1 )+Z( 2 )*Z( 2 ) )
            C = RHO*Z( 1 )*Z( 1 )*DELSQ
*
*           B > ZERO, always
*
*           The following TAU is DSIGMA * DSIGMA - D( 1 ) * D( 1 )
*
            TAU = TWO*C / ( B+SQRT( ABS( B*B-FOUR*C ) ) )
*
*           The following TAU is DSIGMA - D( 1 )
*
            TAU = TAU / ( D( 1 )+SQRT( D( 1 )*D( 1 )+TAU ) )
            DSIGMA = D( 1 ) + TAU
            DELTA( 1 ) = -TAU
            DELTA( 2 ) = DEL - TAU
            WORK( 1 ) = TWO*D( 1 ) + TAU
            WORK( 2 ) = ( D( 1 )+TAU ) + D( 2 )
```

```
*          DELTA( 1 ) = -Z( 1 ) / TAU
*          DELTA( 2 ) = Z( 2 ) / ( DEL-TAU )
        ELSE
           B = -DELSQ + RHO*( Z( 1 )*Z( 1 )+Z( 2 )*Z( 2 ) )
           C = RHO*Z( 2 )*Z( 2 )*DELSQ
*
*          The following TAU is DSIGMA * DSIGMA - D( 2 ) * D( 2 )
*
           IF( B.GT.ZERO ) THEN
              TAU = -TWO*C / ( B+SQRT( B*B+FOUR*C ) )
           ELSE
              TAU = ( B-SQRT( B*B+FOUR*C ) ) / TWO
           END IF
*
*          The following TAU is DSIGMA - D( 2 )
*
           TAU = TAU / ( D( 2 )+SQRT( ABS( D( 2 )*D( 2 )+TAU ) ) )
           DSIGMA = D( 2 ) + TAU
           DELTA( 1 ) = -( DEL+TAU )
           DELTA( 2 ) = -TAU
           WORK( 1 ) = D( 1 ) + TAU + D( 2 )
           WORK( 2 ) = TWO*D( 2 ) + TAU
*          DELTA( 1 ) = -Z( 1 ) / ( DEL+TAU )
*          DELTA( 2 ) = -Z( 2 ) / TAU
        END IF
*       TEMP = SQRT( DELTA( 1 )*DELTA( 1 )+DELTA( 2 )*DELTA( 2 ) )
*       DELTA( 1 ) = DELTA( 1 ) / TEMP
*       DELTA( 2 ) = DELTA( 2 ) / TEMP
     ELSE
*
*       Now I=2
*
        B = -DELSQ + RHO*( Z( 1 )*Z( 1 )+Z( 2 )*Z( 2 ) )
        C = RHO*Z( 2 )*Z( 2 )*DELSQ
*
*       The following TAU is DSIGMA * DSIGMA - D( 2 ) * D( 2 )
*
        IF( B.GT.ZERO ) THEN
           TAU = ( B+SQRT( B*B+FOUR*C ) ) / TWO
        ELSE
           TAU = TWO*C / ( -B+SQRT( B*B+FOUR*C ) )
        END IF
*
*       The following TAU is DSIGMA - D( 2 )
*
        TAU = TAU / ( D( 2 )+SQRT( D( 2 )*D( 2 )+TAU ) )
        DSIGMA = D( 2 ) + TAU
        DELTA( 1 ) = -( DEL+TAU )
        DELTA( 2 ) = -TAU
        WORK( 1 ) = D( 1 ) + TAU + D( 2 )
```

```
         WORK( 2 ) = TWO*D( 2 ) + TAU
*        DELTA( 1 ) = -Z( 1 ) / ( DEL+TAU )
*        DELTA( 2 ) = -Z( 2 ) / TAU
*        TEMP = SQRT( DELTA( 1 )*DELTA( 1 )+DELTA( 2 )*DELTA( 2 ) )
*        DELTA( 1 ) = DELTA( 1 ) / TEMP
*        DELTA( 2 ) = DELTA( 2 ) / TEMP
      END IF
      RETURN
*
*     End of DLASD5
*
      END
```

———-

90.38.4 lisp code

— dlasd5.lisp —

———-

90.39 dlasd6

90.39.1 Axiom unit tests

— dlasd6.input —

```
)set break resume
)sys rm -f dlasd6.output
)spool dlasd6.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.39.2 Axiom help page

— dlasd6.help —

```
====================================================================
dlasd6 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlasd6 (ICOMPQ, NL, NR, SQRE, D, VF, VL, ALPHA, BETA, IDXQ,
           PERM, GIVPTR, GIVCOL, LDGCOL, GIVNUM, LDGNUM, POLES, DIFL, DIFR, Z,
           K, C, S, WORK, IWORK, INFO)
           DLASD6 computes the SVD of an updated upper bidiagonal matrix
           obtained by merging two smaller ones by appending a row. Used by
           sbdsdc.

Function/Subroutine Documentation
   subroutine dlasd6 (integer ICOMPQ, integer NL, integer NR, integer SQRE,
       double precision, dimension( * ) D, double precision, dimension( * )
       VF, double precision, dimension( * ) VL, double precision ALPHA, double
       precision BETA, integer, dimension( * ) IDXQ, integer, dimension( * )
       PERM, integer GIVPTR, integer, dimension( ldgcol, * ) GIVCOL, integer
       LDGCOL, double precision, dimension( ldgnum, * ) GIVNUM, integer
       LDGNUM, double precision, dimension( ldgnum, * ) POLES, double
       precision, dimension( * ) DIFL, double precision, dimension( * ) DIFR,
       double precision, dimension( * ) Z, integer K, double precision C,
       double precision S, double precision, dimension( * ) WORK, integer,
       dimension( * ) IWORK, integer INFO)
       DLASD6 computes the SVD of an updated upper bidiagonal matrix obtained
       by merging two smaller ones by appending a row. Used by sbdsdc.

Purpose:
   DLASD6 computes the SVD of an updated upper bidiagonal matrix B
   obtained by merging two smaller ones by appending a row. This
   routine is used only for the problem which requires all singular
   values and optionally singular vector matrices in factored form.
   B is an N-by-M matrix with N = NL + NR + 1 and M = N + SQRE.
   A related subroutine, DLASD1, handles the case in which all singular
   values and singular vectors of the bidiagonal matrix are desired.

   DLASD6 computes the SVD as follows:

                         ( D1(in)    0    0       0 )
             B = U(in) * (   Z1**T   a   Z2**T    b ) * VT(in)
```

```
                        (   0       0   D2(in)   0 )

              = U(out) * ( D(out) 0) * VT(out)

  where Z**T = (Z1**T a Z2**T b) = u**T VT**T, and u is a vector of
  dimension M with ALPHA and BETA in the NL+1 and NL+2 th
  entries and zeros elsewhere; and the entry b is empty if SQRE = 0.

  The singular values of B can be computed using D1, D2, the first
  components of all the right singular vectors of the lower block, and
  the last components of all the right singular vectors of the upper
  block. These components are stored and updated in VF and VL,
  respectively, in DLASD6. Hence U and VT are not explicitly
  referenced.

  The singular values are stored in D. The algorithm consists of two
  stages:

  The first stage consists of deflating the size of the problem
  when there are multiple singular values or if there is a zero
  in the Z vector. For each such occurence the dimension of the
  secular equation problem is reduced by one. This stage is
  performed by the routine DLASD7.

  The second stage consists of calculating the updated
  singular values. This is done by finding the roots of the
  secular equation via the routine DLASD4 (as called by DLASD8).
  This routine also updates VF and VL and computes the distances
  between the updated singular values and the old singular
  values.

  DLASD6 is called from DLASDA.

Parameters:

 ICOMPQ
   ICOMPQ is INTEGER
   Specifies whether singular vectors are to be computed in
   factored form:
   = 0: Compute singular values only.
   = 1: Compute singular vectors in factored form as well.

 NL
   NL is INTEGER
   The row dimension of the upper block.  NL >= 1.

 NR
   NR is INTEGER
   The row dimension of the lower block.  NR >= 1.
```

```
SQRE
  SQRE is INTEGER
  = 0: the lower block is an NR-by-NR square matrix.
  = 1: the lower block is an NR-by-(NR+1) rectangular matrix.

  The bidiagonal matrix has row dimension N = NL + NR + 1,
  and column dimension M = N + SQRE.

D
  D is DOUBLE PRECISION array, dimension ( NL+NR+1 ).
  On entry D(1:NL,1:NL) contains the singular values of the
  upper block, and D(NL+2:N) contains the singular values
  of the lower block. On exit D(1:N) contains the singular
  values of the modified matrix.

VF
  VF is DOUBLE PRECISION array, dimension ( M )
  On entry, VF(1:NL+1) contains the first components of all
  right singular vectors of the upper block; and VF(NL+2:M)
  contains the first components of all right singular vectors
  of the lower block. On exit, VF contains the first components
  of all right singular vectors of the bidiagonal matrix.

VL
  VL is DOUBLE PRECISION array, dimension ( M )
  On entry, VL(1:NL+1) contains the  last components of all
  right singular vectors of the upper block; and VL(NL+2:M)
  contains the last components of all right singular vectors of
  the lower block. On exit, VL contains the last components of
  all right singular vectors of the bidiagonal matrix.

ALPHA
  ALPHA is DOUBLE PRECISION
  Contains the diagonal element associated with the added row.

BETA
  BETA is DOUBLE PRECISION
  Contains the off-diagonal element associated with the added row.

IDXQ
  IDXQ is INTEGER array, dimension ( N )
  This contains the permutation which will reintegrate the
  subproblem just solved back into sorted order, i.e.
  D( IDXQ( I = 1, N ) ) will be in ascending order.

PERM
  PERM is INTEGER array, dimension ( N )
  The permutations (from deflation and sorting) to be applied
  to each block. Not referenced if ICOMPQ = 0.
```

```
GIVPTR
  GIVPTR is INTEGER
  The number of Givens rotations which took place in this
  subproblem. Not referenced if ICOMPQ = 0.

GIVCOL
  GIVCOL is INTEGER array, dimension ( LDGCOL, 2 )
  Each pair of numbers indicates a pair of columns to take place
  in a Givens rotation. Not referenced if ICOMPQ = 0.

LDGCOL
  LDGCOL is INTEGER
  leading dimension of GIVCOL, must be at least N.

GIVNUM
  GIVNUM is DOUBLE PRECISION array, dimension ( LDGNUM, 2 )
  Each number indicates the C or S value to be used in the
  corresponding Givens rotation. Not referenced if ICOMPQ = 0.

LDGNUM
  LDGNUM is INTEGER
  The leading dimension of GIVNUM and POLES, must be at least N.

POLES
  POLES is DOUBLE PRECISION array, dimension ( LDGNUM, 2 )
  On exit, POLES(1,*) is an array containing the new singular
  values obtained from solving the secular equation, and
  POLES(2,*) is an array containing the poles in the secular
  equation. Not referenced if ICOMPQ = 0.

DIFL
  DIFL is DOUBLE PRECISION array, dimension ( N )
  On exit, DIFL(I) is the distance between I-th updated
  (undeflated) singular value and the I-th (undeflated) old singular value.

DIFR
  DIFR is DOUBLE PRECISION array,
     dimension ( LDGNUM, 2 ) if ICOMPQ = 1 and
     dimension ( N ) if ICOMPQ = 0.
  On exit, DIFR(I, 1) is the distance between I-th updated
  (undeflated) singular value and the I+1-th (undeflated) old singular value.

  If ICOMPQ = 1, DIFR(1:K,2) is an array containing the
  normalizing factors for the right singular vector matrix.

  See DLASD8 for details on DIFL and DIFR.

Z
  Z is DOUBLE PRECISION array, dimension ( M )
  The first elements of this array contain the components
```

```
  of the deflation-adjusted updating row vector.

K
  K is INTEGER
  Contains the dimension of the non-deflated matrix,
  This is the order of the related secular equation. 1 <= K <=N.

C
  C is DOUBLE PRECISION
  C contains garbage if SQRE =0 and the C-value of a Givens
  rotation related to the right null space if SQRE = 1.

S
  S is DOUBLE PRECISION
  S contains garbage if SQRE =0 and the S-value of a Givens
  rotation related to the right null space if SQRE = 1.

WORK
  WORK is DOUBLE PRECISION array, dimension ( 4 * M )

IWORK
  IWORK is INTEGER array, dimension ( 3 * N )

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if INFO = 1, a singular value did not converge
```

———-

90.39.3 fortran code

— dlasd6.f —

```
*  =====================================================================
      SUBROUTINE DLASD6( ICOMPQ, NL, NR, SQRE, D, VF, VL, ALPHA, BETA,
     $                   IDXQ, PERM, GIVPTR, GIVCOL, LDGCOL, GIVNUM,
     $                   LDGNUM, POLES, DIFL, DIFR, Z, K, C, S, WORK,
     $                   IWORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
```

```
      INTEGER            GIVPTR, ICOMPQ, INFO, K, LDGCOL, LDGNUM, NL,
     $                   NR, SQRE
      DOUBLE PRECISION   ALPHA, BETA, C, S
*     ..
*     .. Array Arguments ..
      INTEGER            GIVCOL( LDGCOL, * ), IDXQ( * ), IWORK( * ),
     $                   PERM( * )
      DOUBLE PRECISION   D( * ), DIFL( * ), DIFR( * ),
     $                   GIVNUM( LDGNUM, * ), POLES( LDGNUM, * ),
     $                   VF( * ), VL( * ), WORK( * ), Z( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IDX, IDXC, IDXP, ISIGMA, IVFW, IVLW, IW, M,
     $                   N, N1, N2
      DOUBLE PRECISION   ORGNRM
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DLAMRG, DLASCL, DLASD7, DLASD8, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      N = NL + NR + 1
      M = N + SQRE
*
      IF( ( ICOMPQ.LT.0 ) .OR. ( ICOMPQ.GT.1 ) ) THEN
         INFO = -1
      ELSE IF( NL.LT.1 ) THEN
         INFO = -2
      ELSE IF( NR.LT.1 ) THEN
         INFO = -3
      ELSE IF( ( SQRE.LT.0 ) .OR. ( SQRE.GT.1 ) ) THEN
         INFO = -4
      ELSE IF( LDGCOL.LT.N ) THEN
         INFO = -14
      ELSE IF( LDGNUM.LT.N ) THEN
         INFO = -16
      END IF
```

```
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASD6', -INFO )
         RETURN
      END IF
*
*     The following values are for bookkeeping purposes only.  They are
*     integer pointers which indicate the portion of the workspace
*     used by a particular array in DLASD7 and DLASD8.
*
      ISIGMA = 1
      IW = ISIGMA + N
      IVFW = IW + M
      IVLW = IVFW + M
*
      IDX = 1
      IDXC = IDX + N
      IDXP = IDXC + N
*
*     Scale.
*
      ORGNRM = MAX( ABS( ALPHA ), ABS( BETA ) )
      D( NL+1 ) = ZERO
      DO 10 I = 1, N
         IF( ABS( D( I ) ).GT.ORGNRM ) THEN
            ORGNRM = ABS( D( I ) )
         END IF
   10 CONTINUE
      CALL DLASCL( 'G', 0, 0, ORGNRM, ONE, N, 1, D, N, INFO )
      ALPHA = ALPHA / ORGNRM
      BETA = BETA / ORGNRM
*
*     Sort and Deflate singular values.
*
      CALL DLASD7( ICOMPQ, NL, NR, SQRE, K, D, Z, WORK( IW ), VF,
     $             WORK( IVFW ), VL, WORK( IVLW ), ALPHA, BETA,
     $             WORK( ISIGMA ), IWORK( IDX ), IWORK( IDXP ), IDXQ,
     $             PERM, GIVPTR, GIVCOL, LDGCOL, GIVNUM, LDGNUM, C, S,
     $             INFO )
*
*     Solve Secular Equation, compute DIFL, DIFR, and update VF, VL.
*
      CALL DLASD8( ICOMPQ, K, D, Z, VF, VL, DIFL, DIFR, LDGNUM,
     $             WORK( ISIGMA ), WORK( IW ), INFO )
*
*     Report the possible convergence failure.
*
      IF( INFO.NE.0 ) THEN
         RETURN
      END IF
*
```

```
*     Save the poles if ICOMPQ = 1.
*
      IF( ICOMPQ.EQ.1 ) THEN
         CALL DCOPY( K, D, 1, POLES( 1, 1 ), 1 )
         CALL DCOPY( K, WORK( ISIGMA ), 1, POLES( 1, 2 ), 1 )
      END IF
*
*     Unscale.
*
      CALL DLASCL( 'G', 0, 0, ONE, ORGNRM, N, 1, D, N, INFO )
*
*     Prepare the IDXQ sorting permutation.
*
      N1 = K
      N2 = N - K
      CALL DLAMRG( N1, N2, D, 1, -1, IDXQ )
*
      RETURN
*
*     End of DLASD6
*
      END
```

———-

90.39.4 lisp code

— dlasd6.lisp —

———-

90.40 dlasd7

90.40.1 Axiom unit tests

— dlasd7.input —

```
)set break resume
)sys rm -f dlasd7.output
)spool dlasd7.output
)set message test on
)set message auto off
```

```
)clear all

)spool
)lisp (bye)
```

———-

90.40.2 Axiom help page

— dlasd7.help —

```
==================================================================
dlasd7 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlasd7 (ICOMPQ, NL, NR, SQRE, K, D, Z, ZW, VF, VFW, VL, VLW,
           ALPHA, BETA, DSIGMA, IDX, IDXP, IDXQ, PERM, GIVPTR, GIVCOL, LDGCOL,
           GIVNUM, LDGNUM, C, S, INFO)
           DLASD7 merges the two sets of singular values together into a
           single sorted set. Then it tries to deflate the size of the
           problem. Used by sbdsdc.

Function/Subroutine Documentation
   subroutine dlasd7 (integer ICOMPQ, integer NL, integer NR, integer SQRE,
       integer K, double precision, dimension( * ) D, double precision,
       dimension( * ) Z, double precision, dimension( * ) ZW, double
       precision, dimension( * ) VF, double precision, dimension( * ) VFW,
       double precision, dimension( * ) VL, double precision, dimension( * )
       VLW, double precision ALPHA, double precision BETA, double precision,
       dimension( * ) DSIGMA, integer, dimension( * ) IDX, integer, dimension(
       * ) IDXP, integer, dimension( * ) IDXQ, integer, dimension( * ) PERM,
       integer GIVPTR, integer, dimension( ldgcol, * ) GIVCOL, integer LDGCOL,
       double precision, dimension( ldgnum, * ) GIVNUM, integer LDGNUM, double
       precision C, double precision S, integer INFO)
       DLASD7 merges the two sets of singular values together into a single
       sorted set. Then it tries to deflate the size of the problem. Used by
       sbdsdc.

Purpose:
   DLASD7 merges the two sets of singular values together into a single
   sorted set. Then it tries to deflate the size of the problem. There
   are two ways in which deflation can occur:  when two or more singular
```

```
  values are close together or if there is a tiny entry in the Z
  vector. For each such occurrence the order of the related
  secular equation problem is reduced by one.

  DLASD7 is called from DLASD6.

Parameters:

 ICOMPQ
   ICOMPQ is INTEGER
   Specifies whether singular vectors are to be computed
   in compact form, as follows:
   = 0: Compute singular values only.
   = 1: Compute singular vectors of upper bidiagonal matrix in compact form.

 NL
   NL is INTEGER
   The row dimension of the upper block. NL >= 1.

 NR
   NR is INTEGER
   The row dimension of the lower block. NR >= 1.

 SQRE
   SQRE is INTEGER
   = 0: the lower block is an NR-by-NR square matrix.
   = 1: the lower block is an NR-by-(NR+1) rectangular matrix.

   The bidiagonal matrix has N = NL + NR + 1 rows and
   M = N + SQRE >= N columns.

 K
   K is INTEGER
   Contains the dimension of the non-deflated matrix, this is
   the order of the related secular equation. 1 <= K <=N.

 D
   D is DOUBLE PRECISION array, dimension ( N )
   On entry D contains the singular values of the two submatrices
   to be combined. On exit D contains the trailing (N-K) updated
   singular values (those which were deflated) sorted into increasing order.

 Z
   Z is DOUBLE PRECISION array, dimension ( M )
   On exit Z contains the updating row vector in the secular equation.

 ZW
   ZW is DOUBLE PRECISION array, dimension ( M )
   Workspace for Z.
```

```
VF
  VF is DOUBLE PRECISION array, dimension ( M )
  On entry, VF(1:NL+1) contains the first components of all
  right singular vectors of the upper block; and VF(NL+2:M)
  contains the first components of all right singular vectors
  of the lower block. On exit, VF contains the first components
  of all right singular vectors of the bidiagonal matrix.

VFW
  VFW is DOUBLE PRECISION array, dimension ( M )
  Workspace for VF.

VL
  VL is DOUBLE PRECISION array, dimension ( M )
  On entry, VL(1:NL+1) contains the  last components of all
  right singular vectors of the upper block; and VL(NL+2:M)
  contains the last components of all right singular vectors
  of the lower block. On exit, VL contains the last components
  of all right singular vectors of the bidiagonal matrix.

VLW
  VLW is DOUBLE PRECISION array, dimension ( M )
  Workspace for VL.

ALPHA
  ALPHA is DOUBLE PRECISION
  Contains the diagonal element associated with the added row.

BETA
  BETA is DOUBLE PRECISION
  Contains the off-diagonal element associated with the added row.

DSIGMA
  DSIGMA is DOUBLE PRECISION array, dimension ( N )
  Contains a copy of the diagonal elements (K-1 singular values
  and one zero) in the secular equation.

IDX
  IDX is INTEGER array, dimension ( N )
  This will contain the permutation used to sort the contents of
  D into ascending order.

IDXP
  IDXP is INTEGER array, dimension ( N )
  This will contain the permutation used to place deflated
  values of D at the end of the array. On output IDXP(2:K)
  points to the nondeflated D-values and IDXP(K+1:N)
  points to the deflated singular values.

IDXQ
```

```
  IDXQ is INTEGER array, dimension ( N )
  This contains the permutation which separately sorts the two
  sub-problems in D into ascending order.  Note that entries in
  the first half of this permutation must first be moved one
  position backward; and entries in the second half
  must first have NL+1 added to their values.

PERM
  PERM is INTEGER array, dimension ( N )
  The permutations (from deflation and sorting) to be applied
  to each singular block. Not referenced if ICOMPQ = 0.

GIVPTR
  GIVPTR is INTEGER
  The number of Givens rotations which took place in this
  subproblem. Not referenced if ICOMPQ = 0.

GIVCOL
  GIVCOL is INTEGER array, dimension ( LDGCOL, 2 )
  Each pair of numbers indicates a pair of columns to take place
  in a Givens rotation. Not referenced if ICOMPQ = 0.

LDGCOL
  LDGCOL is INTEGER
  The leading dimension of GIVCOL, must be at least N.

GIVNUM
  GIVNUM is DOUBLE PRECISION array, dimension ( LDGNUM, 2 )
  Each number indicates the C or S value to be used in the
  corresponding Givens rotation. Not referenced if ICOMPQ = 0.

LDGNUM
  LDGNUM is INTEGER
  The leading dimension of GIVNUM, must be at least N.

C
  C is DOUBLE PRECISION
  C contains garbage if SQRE =0 and the C-value of a Givens
  rotation related to the right null space if SQRE = 1.

S
  S is DOUBLE PRECISION
  S contains garbage if SQRE =0 and the S-value of a Givens
  rotation related to the right null space if SQRE = 1.

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

———-

90.40.3 fortran code

— dlasd7.f —

```
*  =====================================================================
      SUBROUTINE DLASD7( ICOMPQ, NL, NR, SQRE, K, D, Z, ZW, VF, VFW, VL,
     $                   VLW, ALPHA, BETA, DSIGMA, IDX, IDXP, IDXQ,
     $                   PERM, GIVPTR, GIVCOL, LDGCOL, GIVNUM, LDGNUM,
     $                   C, S, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            GIVPTR, ICOMPQ, INFO, K, LDGCOL, LDGNUM, NL,
     $                   NR, SQRE
      DOUBLE PRECISION   ALPHA, BETA, C, S
*     ..
*     .. Array Arguments ..
      INTEGER            GIVCOL( LDGCOL, * ), IDX( * ), IDXP( * ),
     $                   IDXQ( * ), PERM( * )
      DOUBLE PRECISION   D( * ), DSIGMA( * ), GIVNUM( LDGNUM, * ),
     $                   VF( * ), VFW( * ), VL( * ), VLW( * ), Z( * ),
     $                   ZW( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE, TWO, EIGHT
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0, TWO = 2.0D+0,
     $                   EIGHT = 8.0D+0 )
*     ..
*     .. Local Scalars ..
*
      INTEGER            I, IDXI, IDXJ, IDXJP, J, JP, JPREV, K2, M, N,
     $                   NLP1, NLP2
      DOUBLE PRECISION   EPS, HLFTOL, TAU, TOL, Z1
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DLAMRG, DROT, XERBLA
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH, DLAPY2
```

```
      EXTERNAL           DLAMCH, DLAPY2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      N = NL + NR + 1
      M = N + SQRE
*
      IF( ( ICOMPQ.LT.0 ) .OR. ( ICOMPQ.GT.1 ) ) THEN
         INFO = -1
      ELSE IF( NL.LT.1 ) THEN
         INFO = -2
      ELSE IF( NR.LT.1 ) THEN
         INFO = -3
      ELSE IF( ( SQRE.LT.0 ) .OR. ( SQRE.GT.1 ) ) THEN
         INFO = -4
      ELSE IF( LDGCOL.LT.N ) THEN
         INFO = -22
      ELSE IF( LDGNUM.LT.N ) THEN
         INFO = -24
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASD7', -INFO )
         RETURN
      END IF
*
      NLP1 = NL + 1
      NLP2 = NL + 2
      IF( ICOMPQ.EQ.1 ) THEN
         GIVPTR = 0
      END IF
*
*     Generate the first part of the vector Z and move the singular
*     values in the first part of D one position backward.
*
      Z1 = ALPHA*VL( NLP1 )
      VL( NLP1 ) = ZERO
      TAU = VF( NLP1 )
      DO 10 I = NL, 1, -1
         Z( I+1 ) = ALPHA*VL( I )
         VL( I ) = ZERO
         VF( I+1 ) = VF( I )
         D( I+1 ) = D( I )
         IDXQ( I+1 ) = IDXQ( I ) + 1
   10 CONTINUE
```

```
      VF( 1 ) = TAU
*
*     Generate the second part of the vector Z.
*
      DO 20 I = NLP2, M
         Z( I ) = BETA*VF( I )
         VF( I ) = ZERO
   20 CONTINUE
*
*     Sort the singular values into increasing order
*
      DO 30 I = NLP2, N
         IDXQ( I ) = IDXQ( I ) + NLP1
   30 CONTINUE
*
*     DSIGMA, IDXC, IDXC, and ZW are used as storage space.
*
      DO 40 I = 2, N
         DSIGMA( I ) = D( IDXQ( I ) )
         ZW( I ) = Z( IDXQ( I ) )
         VFW( I ) = VF( IDXQ( I ) )
         VLW( I ) = VL( IDXQ( I ) )
   40 CONTINUE
*
      CALL DLAMRG( NL, NR, DSIGMA( 2 ), 1, 1, IDX( 2 ) )
*
      DO 50 I = 2, N
         IDXI = 1 + IDX( I )
         D( I ) = DSIGMA( IDXI )
         Z( I ) = ZW( IDXI )
         VF( I ) = VFW( IDXI )
         VL( I ) = VLW( IDXI )
   50 CONTINUE
*
*     Calculate the allowable deflation tolerence
*
      EPS = DLAMCH( 'Epsilon' )
      TOL = MAX( ABS( ALPHA ), ABS( BETA ) )
      TOL = EIGHT*EIGHT*EPS*MAX( ABS( D( N ) ), TOL )
*
*     There are 2 kinds of deflation -- first a value in the z-vector
*     is small, second two (or more) singular values are very close
*     together (their difference is small).
*
*     If the value in the z-vector is small, we simply permute the
*     array so that the corresponding singular value is moved to the
*     end.
*
*     If two values in the D-vector are close, we perform a two-sided
*     rotation designed to make one of the corresponding z-vector
```

```
*     entries zero, and then permute the array so that the deflated
*     singular value is moved to the end.
*
*     If there are multiple singular values then the problem deflates.
*     Here the number of equal singular values are found.  As each equal
*     singular value is found, an elementary reflector is computed to
*     rotate the corresponding singular subspace so that the
*     corresponding components of Z are zero in this new basis.
*
      K = 1
      K2 = N + 1
      DO 60 J = 2, N
         IF( ABS( Z( J ) ).LE.TOL ) THEN
*
*           Deflate due to small z component.
*
            K2 = K2 - 1
            IDXP( K2 ) = J
            IF( J.EQ.N )
     $         GO TO 100
         ELSE
            JPREV = J
            GO TO 70
         END IF
   60 CONTINUE
   70 CONTINUE
      J = JPREV
   80 CONTINUE
      J = J + 1
      IF( J.GT.N )
     $   GO TO 90
      IF( ABS( Z( J ) ).LE.TOL ) THEN
*
*        Deflate due to small z component.
*
         K2 = K2 - 1
         IDXP( K2 ) = J
      ELSE
*
*        Check if singular values are close enough to allow deflation.
*
         IF( ABS( D( J )-D( JPREV ) ).LE.TOL ) THEN
*
*           Deflation is possible.
*
            S = Z( JPREV )
            C = Z( J )
*
*           Find sqrt(a**2+b**2) without overflow or
*           destructive underflow.
```

```
*
           TAU = DLAPY2( C, S )
           Z( J ) = TAU
           Z( JPREV ) = ZERO
           C = C / TAU
           S = -S / TAU
*
*          Record the appropriate Givens rotation
*
           IF( ICOMPQ.EQ.1 ) THEN
              GIVPTR = GIVPTR + 1
              IDXJP = IDXQ( IDX( JPREV )+1 )
              IDXJ = IDXQ( IDX( J )+1 )
              IF( IDXJP.LE.NLP1 ) THEN
                 IDXJP = IDXJP - 1
              END IF
              IF( IDXJ.LE.NLP1 ) THEN
                 IDXJ = IDXJ - 1
              END IF
              GIVCOL( GIVPTR, 2 ) = IDXJP
              GIVCOL( GIVPTR, 1 ) = IDXJ
              GIVNUM( GIVPTR, 2 ) = C
              GIVNUM( GIVPTR, 1 ) = S
           END IF
           CALL DROT( 1, VF( JPREV ), 1, VF( J ), 1, C, S )
           CALL DROT( 1, VL( JPREV ), 1, VL( J ), 1, C, S )
           K2 = K2 - 1
           IDXP( K2 ) = JPREV
           JPREV = J
        ELSE
           K = K + 1
           ZW( K ) = Z( JPREV )
           DSIGMA( K ) = D( JPREV )
           IDXP( K ) = JPREV
           JPREV = J
        END IF
     END IF
     GO TO 80
  90 CONTINUE
*
*    Record the last singular value.
*
     K = K + 1
     ZW( K ) = Z( JPREV )
     DSIGMA( K ) = D( JPREV )
     IDXP( K ) = JPREV
*
 100 CONTINUE
*
*    Sort the singular values into DSIGMA. The singular values which
```

```
*     were not deflated go into the first K slots of DSIGMA, except
*     that DSIGMA(1) is treated separately.
*
      DO 110 J = 2, N
         JP = IDXP( J )
         DSIGMA( J ) = D( JP )
         VFW( J ) = VF( JP )
         VLW( J ) = VL( JP )
  110 CONTINUE
      IF( ICOMPQ.EQ.1 ) THEN
         DO 120 J = 2, N
            JP = IDXP( J )
            PERM( J ) = IDXQ( IDX( JP )+1 )
            IF( PERM( J ).LE.NLP1 ) THEN
               PERM( J ) = PERM( J ) - 1
            END IF
  120    CONTINUE
      END IF
*
*     The deflated singular values go back into the last N - K slots of
*     D.
*
      CALL DCOPY( N-K, DSIGMA( K+1 ), 1, D( K+1 ), 1 )
*
*     Determine DSIGMA(1), DSIGMA(2), Z(1), VF(1), VL(1), VF(M), and
*     VL(M).
*
      DSIGMA( 1 ) = ZERO
      HLFTOL = TOL / TWO
      IF( ABS( DSIGMA( 2 ) ).LE.HLFTOL )
     $   DSIGMA( 2 ) = HLFTOL
      IF( M.GT.N ) THEN
         Z( 1 ) = DLAPY2( Z1, Z( M ) )
         IF( Z( 1 ).LE.TOL ) THEN
            C = ONE
            S = ZERO
            Z( 1 ) = TOL
         ELSE
            C = Z1 / Z( 1 )
            S = -Z( M ) / Z( 1 )
         END IF
         CALL DROT( 1, VF( M ), 1, VF( 1 ), 1, C, S )
         CALL DROT( 1, VL( M ), 1, VL( 1 ), 1, C, S )
      ELSE
         IF( ABS( Z1 ).LE.TOL ) THEN
            Z( 1 ) = TOL
         ELSE
            Z( 1 ) = Z1
         END IF
      END IF
```

```
*
*     Restore Z, VF, and VL.
*
      CALL DCOPY( K-1, ZW( 2 ), 1, Z( 2 ), 1 )
      CALL DCOPY( N-1, VFW( 2 ), 1, VF( 2 ), 1 )
      CALL DCOPY( N-1, VLW( 2 ), 1, VL( 2 ), 1 )
*
      RETURN
*
*     End of DLASD7
*
      END
```

———-

90.40.4 lisp code

— dlasd7.lisp —

———-

90.41 dlasd8

90.41.1 Axiom unit tests

— dlasd8.input —

```
)set break resume
)sys rm -f dlasd8.output
)spool dlasd8.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.41.2 Axiom help page

— dlasd8.help —

```
=====================================================================
dlasd8 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlasd8 (ICOMPQ, K, D, Z, VF, VL, DIFL, DIFR, LDDIFR, DSIGMA,
           WORK, INFO)
           DLASD8 finds the square roots of the roots of the secular equation,
           and stores, for each element in D, the distance to its two nearest
           poles. Used by sbdsdc.

Function/Subroutine Documentation
   subroutine dlasd8 (integer ICOMPQ, integer K, double precision, dimension(
       * ) D, double precision, dimension( * ) Z, double precision, dimension(
       * ) VF, double precision, dimension( * ) VL, double precision,
       dimension( * ) DIFL, double precision, dimension( lddifr, * ) DIFR,
       integer LDDIFR, double precision, dimension( * ) DSIGMA, double
       precision, dimension( * ) WORK, integer INFO)
       DLASD8 finds the square roots of the roots of the secular equation, and
       stores, for each element in D, the distance to its two nearest poles.
       Used by sbdsdc.

Purpose:
   DLASD8 finds the square roots of the roots of the secular equation,
   as defined by the values in DSIGMA and Z. It makes the appropriate
   calls to DLASD4, and stores, for each  element in D, the distance
   to its two nearest poles (elements in DSIGMA). It also updates
   the arrays VF and VL, the first and last components of all the
   right singular vectors of the original bidiagonal matrix.

   DLASD8 is called from DLASD6.

Parameters:

 ICOMPQ
   ICOMPQ is INTEGER
   Specifies whether singular vectors are to be computed in
   factored form in the calling routine:
   = 0: Compute singular values only.
   = 1: Compute singular vectors in factored form as well.
```

```
K
  K is INTEGER
  The number of terms in the rational function to be solved
  by DLASD4.  K >= 1.

D
  D is DOUBLE PRECISION array, dimension ( K )
  On output, D contains the updated singular values.

Z
  Z is DOUBLE PRECISION array, dimension ( K )
  On entry, the first K elements of this array contain the
  components of the deflation-adjusted updating row vector.
  On exit, Z is updated.

VF
  VF is DOUBLE PRECISION array, dimension ( K )
  On entry, VF contains  information passed through DBEDE8.
  On exit, VF contains the first K components of the first
  components of all right singular vectors of the bidiagonal matrix.

VL
  VL is DOUBLE PRECISION array, dimension ( K )
  On entry, VL contains  information passed through DBEDE8.
  On exit, VL contains the first K components of the last
  components of all right singular vectors of the bidiagonal matrix.

DIFL
  DIFL is DOUBLE PRECISION array, dimension ( K )
  On exit, DIFL(I) = D(I) - DSIGMA(I).

DIFR
  DIFR is DOUBLE PRECISION array,
    dimension ( LDDIFR, 2 ) if ICOMPQ = 1 and
    dimension ( K ) if ICOMPQ = 0.
  On exit, DIFR(I,1) = D(I) - DSIGMA(I+1), DIFR(K,1) is not
  defined and will not be referenced.

  If ICOMPQ = 1, DIFR(1:K,2) is an array containing the
  normalizing factors for the right singular vector matrix.

LDDIFR
  LDDIFR is INTEGER
  The leading dimension of DIFR, must be at least K.

DSIGMA
  DSIGMA is DOUBLE PRECISION array, dimension ( K )
  On entry, the first K elements of this array contain the old
  roots of the deflated updating problem.  These are the poles
```

```
  of the secular equation.
  On exit, the elements of DSIGMA may be very slightly altered in value.

 WORK
  WORK is DOUBLE PRECISION array, dimension at least 3 * K

 INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if INFO = 1, a singular value did not converge
```

———-

90.41.3 fortran code

— **dlasd8.f** —

```
*  =====================================================================
      SUBROUTINE DLASD8( ICOMPQ, K, D, Z, VF, VL, DIFL, DIFR, LDDIFR,
     $                   DSIGMA, WORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            ICOMPQ, INFO, K, LDDIFR
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), DIFL( * ), DIFR( LDDIFR, * ),
     $                   DSIGMA( * ), VF( * ), VL( * ), WORK( * ),
     $                   Z( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IWK1, IWK2, IWK2I, IWK3, IWK3I, J
      DOUBLE PRECISION   DIFLJ, DIFRJ, DJ, DSIGJ, DSIGJP, RHO, TEMP
*     ..
*     .. External Subroutines ..
```

```
      EXTERNAL           DCOPY, DLASCL, DLASD4, DLASET, XERBLA
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DDOT, DLAMC3, DNRM2
      EXTERNAL           DDOT, DLAMC3, DNRM2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SIGN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( ( ICOMPQ.LT.0 ) .OR. ( ICOMPQ.GT.1 ) ) THEN
         INFO = -1
      ELSE IF( K.LT.1 ) THEN
         INFO = -2
      ELSE IF( LDDIFR.LT.K ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASD8', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( K.EQ.1 ) THEN
         D( 1 ) = ABS( Z( 1 ) )
         DIFL( 1 ) = D( 1 )
         IF( ICOMPQ.EQ.1 ) THEN
            DIFL( 2 ) = ONE
            DIFR( 1, 2 ) = ONE
         END IF
         RETURN
      END IF
*
*     Modify values DSIGMA(i) to make sure all DSIGMA(i)-DSIGMA(j) can
*     be computed with high relative accuracy (barring over/underflow).
*     This is a problem on machines without a guard digit in
*     add/subtract (Cray XMP, Cray YMP, Cray C 90 and Cray 2).
*     The following code replaces DSIGMA(I) by 2*DSIGMA(I)-DSIGMA(I),
*     which on any of these machines zeros out the bottommost
*     bit of DSIGMA(I) if it is 1; this makes the subsequent
*     subtractions DSIGMA(I)-DSIGMA(J) unproblematic when cancellation
*     occurs. On binary machines with a guard digit (almost all
*     machines) it does not change DSIGMA(I) at all. On hexadecimal
*     and decimal machines with a guard digit, it slightly
```

```
*     changes the bottommost bits of DSIGMA(I). It does not account
*     for hexadecimal or decimal machines without guard digits
*     (we know of none). We use a subroutine call to compute
*     2*DLAMBDA(I) to prevent optimizing compilers from eliminating
*     this code.
*
      DO 10 I = 1, K
         DSIGMA( I ) = DLAMC3( DSIGMA( I ), DSIGMA( I ) ) - DSIGMA( I )
   10 CONTINUE
*
*     Book keeping.
*
      IWK1 = 1
      IWK2 = IWK1 + K
      IWK3 = IWK2 + K
      IWK2I = IWK2 - 1
      IWK3I = IWK3 - 1
*
*     Normalize Z.
*
      RHO = DNRM2( K, Z, 1 )
      CALL DLASCL( 'G', 0, 0, RHO, ONE, K, 1, Z, K, INFO )
      RHO = RHO*RHO
*
*     Initialize WORK(IWK3).
*
      CALL DLASET( 'A', K, 1, ONE, ONE, WORK( IWK3 ), K )
*
*     Compute the updated singular values, the arrays DIFL, DIFR,
*     and the updated Z.
*
      DO 40 J = 1, K
         CALL DLASD4( K, J, DSIGMA, Z, WORK( IWK1 ), RHO, D( J ),
     $                WORK( IWK2 ), INFO )
*
*        If the root finder fails, report the convergence failure.
*
         IF( INFO.NE.0 ) THEN
            RETURN
         END IF
         WORK( IWK3I+J ) = WORK( IWK3I+J )*WORK( J )*WORK( IWK2I+J )
         DIFL( J ) = -WORK( J )
         DIFR( J, 1 ) = -WORK( J+1 )
         DO 20 I = 1, J - 1
            WORK( IWK3I+I ) = WORK( IWK3I+I )*WORK( I )*
     $                        WORK( IWK2I+I ) / ( DSIGMA( I )-
     $                        DSIGMA( J ) ) / ( DSIGMA( I )+
     $                        DSIGMA( J ) )
   20    CONTINUE
         DO 30 I = J + 1, K
```

```
            WORK( IWK3I+I ) = WORK( IWK3I+I )*WORK( I )*
     $                        WORK( IWK2I+I ) / ( DSIGMA( I )-
     $                        DSIGMA( J ) ) / ( DSIGMA( I )+
     $                        DSIGMA( J ) )
   30    CONTINUE
   40 CONTINUE
*
*     Compute updated Z.
*
      DO 50 I = 1, K
         Z( I ) = SIGN( SQRT( ABS( WORK( IWK3I+I ) ) ), Z( I ) )
   50 CONTINUE
*
*     Update VF and VL.
*
      DO 80 J = 1, K
         DIFLJ = DIFL( J )
         DJ = D( J )
         DSIGJ = -DSIGMA( J )
         IF( J.LT.K ) THEN
            DIFRJ = -DIFR( J, 1 )
            DSIGJP = -DSIGMA( J+1 )
         END IF
         WORK( J ) = -Z( J ) / DIFLJ / ( DSIGMA( J )+DJ )
         DO 60 I = 1, J - 1
            WORK( I ) = Z( I ) / ( DLAMC3( DSIGMA( I ), DSIGJ )-DIFLJ )
     $                  / ( DSIGMA( I )+DJ )
   60    CONTINUE
         DO 70 I = J + 1, K
            WORK( I ) = Z( I ) / ( DLAMC3( DSIGMA( I ), DSIGJP )+DIFRJ )
     $                  / ( DSIGMA( I )+DJ )
   70    CONTINUE
         TEMP = DNRM2( K, WORK, 1 )
         WORK( IWK2I+J ) = DDOT( K, WORK, 1, VF, 1 ) / TEMP
         WORK( IWK3I+J ) = DDOT( K, WORK, 1, VL, 1 ) / TEMP
         IF( ICOMPQ.EQ.1 ) THEN
            DIFR( J, 2 ) = TEMP
         END IF
   80 CONTINUE
*
      CALL DCOPY( K, WORK( IWK2 ), 1, VF, 1 )
      CALL DCOPY( K, WORK( IWK3 ), 1, VL, 1 )
*
      RETURN
*
*     End of DLASD8
*
      END
```

———-

90.41.4 lisp code

— dlasd8.lisp —

———-

90.42 dlasda

90.42.1 Axiom unit tests

— dlasda.input —

```
)set break resume
)sys rm -f dlasda.output
)spool dlasda.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.42.2 Axiom help page

— dlasda.help —

```
======================================================================
dlasda examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
```

```
      subroutine dlasda (ICOMPQ, SMLSIZ, N, SQRE, D, E, U, LDU, VT, K, DIFL,
          DIFR, Z, POLES, GIVPTR, GIVCOL, LDGCOL, PERM, GIVNUM, C, S, WORK,
          IWORK, INFO)
          DLASDA computes the singular value decomposition (SVD) of a real
          upper bidiagonal matrix with diagonal d and off-diagonal e. Used by
          sbdsdc.

Function/Subroutine Documentation
   subroutine dlasda (integer ICOMPQ, integer SMLSIZ, integer N, integer SQRE,
       double precision, dimension( * ) D, double precision, dimension( * ) E,
       double precision, dimension( ldu, * ) U, integer LDU, double precision,
       dimension( ldu, * ) VT, integer, dimension( * ) K, double precision,
       dimension( ldu, * ) DIFL, double precision, dimension( ldu, * ) DIFR,
       double precision, dimension( ldu, * ) Z, double precision, dimension(
       ldu, * ) POLES, integer, dimension( * ) GIVPTR, integer, dimension(
       ldgcol, * ) GIVCOL, integer LDGCOL, integer, dimension( ldgcol, * )
       PERM, double precision, dimension( ldu, * ) GIVNUM, double precision,
       dimension( * ) C, double precision, dimension( * ) S, double precision,
       dimension( * ) WORK, integer, dimension( * ) IWORK, integer INFO)
       DLASDA computes the singular value decomposition (SVD) of a real upper
       bidiagonal matrix with diagonal d and off-diagonal e. Used by sbdsdc.

Purpose:
   Using a divide and conquer approach, DLASDA computes the singular
   value decomposition (SVD) of a real upper bidiagonal N-by-M matrix
   B with diagonal D and offdiagonal E, where M = N + SQRE. The
   algorithm computes the singular values in the SVD B = U * S * VT.
   The orthogonal matrices U and VT are optionally computed in
   compact form.

   A related subroutine, DLASD0, computes the singular values and
   the singular vectors in explicit form.

Parameters:

 ICOMPQ
   ICOMPQ is INTEGER
   Specifies whether singular vectors are to be computed
   in compact form, as follows
   = 0: Compute singular values only.
   = 1: Compute singular vectors of upper bidiagonal matrix in compact form.

 SMLSIZ
   SMLSIZ is INTEGER
   The maximum size of the subproblems at the bottom of the computation tree.

 N
   N is INTEGER
   The row dimension of the upper bidiagonal matrix. This is
   also the dimension of the main diagonal array D.
```

```
SQRE
  SQRE is INTEGER
  Specifies the column dimension of the bidiagonal matrix.
  = 0: The bidiagonal matrix has column dimension M = N;
  = 1: The bidiagonal matrix has column dimension M = N + 1.

D
  D is DOUBLE PRECISION array, dimension ( N )
  On entry D contains the main diagonal of the bidiagonal
  matrix. On exit D, if INFO = 0, contains its singular values.

E
  E is DOUBLE PRECISION array, dimension ( M-1 )
  Contains the subdiagonal entries of the bidiagonal matrix.
  On exit, E has been destroyed.

U
  U is DOUBLE PRECISION array,
  dimension ( LDU, SMLSIZ ) if ICOMPQ = 1, and not referenced
  if ICOMPQ = 0. If ICOMPQ = 1, on exit, U contains the left
  singular vector matrices of all subproblems at the bottom level.

LDU
  LDU is INTEGER, LDU = > N.
  The leading dimension of arrays U, VT, DIFL, DIFR, POLES, GIVNUM, and Z.

VT
  VT is DOUBLE PRECISION array,
  dimension ( LDU, SMLSIZ+1 ) if ICOMPQ = 1, and not referenced
  if ICOMPQ = 0. If ICOMPQ = 1, on exit, VT**T contains the right
  singular vector matrices of all subproblems at the bottom level.

K
  K is INTEGER array,
  dimension ( N ) if ICOMPQ = 1 and dimension 1 if ICOMPQ = 0.
  If ICOMPQ = 1, on exit, K(I) is the dimension of the I-th
  secular equation on the computation tree.

DIFL
  DIFL is DOUBLE PRECISION array, dimension ( LDU, NLVL ),
  where NLVL = floor(log_2 (N/SMLSIZ))).

DIFR
  DIFR is DOUBLE PRECISION array,
     dimension ( LDU, 2 * NLVL ) if ICOMPQ = 1 and
     dimension ( N ) if ICOMPQ = 0.
  If ICOMPQ = 1, on exit, DIFL(1:N, I) and DIFR(1:N, 2 * I - 1)
  record distances between singular values on the I-th
  level and singular values on the (I -1)-th level, and
```

```
  DIFR(1:N, 2 * I ) contains the normalizing factors for
  the right singular vector matrix. See DLASD8 for details.

Z
  Z is DOUBLE PRECISION array,
     dimension ( LDU, NLVL ) if ICOMPQ = 1 and
     dimension ( N ) if ICOMPQ = 0.
  The first K elements of Z(1, I) contain the components of
  the deflation-adjusted updating row vector for subproblems
  on the I-th level.

POLES
  POLES is DOUBLE PRECISION array,
  dimension ( LDU, 2 * NLVL ) if ICOMPQ = 1, and not referenced
  if ICOMPQ = 0. If ICOMPQ = 1, on exit, POLES(1, 2*I - 1) and
  POLES(1, 2*I) contain  the new and old singular values
  involved in the secular equations on the I-th level.

GIVPTR
  GIVPTR is INTEGER array,
  dimension ( N ) if ICOMPQ = 1, and not referenced if
  ICOMPQ = 0. If ICOMPQ = 1, on exit, GIVPTR( I ) records
  the number of Givens rotations performed on the I-th
  problem on the computation tree.

GIVCOL
  GIVCOL is INTEGER array,
  dimension ( LDGCOL, 2 * NLVL ) if ICOMPQ = 1, and not
  referenced if ICOMPQ = 0. If ICOMPQ = 1, on exit, for each I,
  GIVCOL(1, 2 *I - 1) and GIVCOL(1, 2 *I) record the locations
  of Givens rotations performed on the I-th level on the computation tree.

LDGCOL
  LDGCOL is INTEGER, LDGCOL = > N.
  The leading dimension of arrays GIVCOL and PERM.

PERM
  PERM is INTEGER array,
  dimension ( LDGCOL, NLVL ) if ICOMPQ = 1, and not referenced
  if ICOMPQ = 0. If ICOMPQ = 1, on exit, PERM(1, I) records
  permutations done on the I-th level of the computation tree.

GIVNUM
  GIVNUM is DOUBLE PRECISION array,
  dimension ( LDU,  2 * NLVL ) if ICOMPQ = 1, and not
  referenced if ICOMPQ = 0. If ICOMPQ = 1, on exit, for each I,
  GIVNUM(1, 2 *I - 1) and GIVNUM(1, 2 *I) record the C- and S-
  values of Givens rotations performed on the I-th level on
  the computation tree.
```

```
C
  C is DOUBLE PRECISION array,
  dimension ( N ) if ICOMPQ = 1, and dimension 1 if ICOMPQ = 0.
  If ICOMPQ = 1 and the I-th subproblem is not square, on exit,
  C( I ) contains the C-value of a Givens rotation related to
  the right null space of the I-th subproblem.

S
  S is DOUBLE PRECISION array, dimension ( N ) if
  ICOMPQ = 1, and dimension 1 if ICOMPQ = 0. If ICOMPQ = 1
  and the I-th subproblem is not square, on exit, S( I )
  contains the S-value of a Givens rotation related to
  the right null space of the I-th subproblem.

WORK
  WORK is DOUBLE PRECISION array, dimension
  (6 * N + (SMLSIZ + 1)*(SMLSIZ + 1)).

IWORK
  IWORK is INTEGER array.
  Dimension must be at least (7 * N).

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if INFO = 1, a singular value did not converge
```

90.42.3 fortran code

— dlasda.f —

```
*  =====================================================================
      SUBROUTINE DLASDA( ICOMPQ, SMLSIZ, N, SQRE, D, E, U, LDU, VT, K,
     $                   DIFL, DIFR, Z, POLES, GIVPTR, GIVCOL, LDGCOL,
     $                   PERM, GIVNUM, C, S, WORK, IWORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            ICOMPQ, INFO, LDGCOL, LDU, N, SMLSIZ, SQRE
*     ..
```

```
*     .. Array Arguments ..
      INTEGER            GIVCOL( LDGCOL, * ), GIVPTR( * ), IWORK( * ),
     $                   K( * ), PERM( LDGCOL, * )
      DOUBLE PRECISION   C( * ), D( * ), DIFL( LDU, * ), DIFR( LDU, * ),
     $                   E( * ), GIVNUM( LDU, * ), POLES( LDU, * ),
     $                   S( * ), U( LDU, * ), VT( LDU, * ), WORK( * ),
     $                   Z( LDU, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, I1, IC, IDXQ, IDXQI, IM1, INODE, ITEMP, IWK,
     $                   J, LF, LL, LVL, LVL2, M, NCC, ND, NDB1, NDIML,
     $                   NDIMR, NL, NLF, NLP1, NLVL, NR, NRF, NRP1, NRU,
     $                   NWORK1, NWORK2, SMLSZP, SQREI, VF, VFI, VL, VLI
      DOUBLE PRECISION   ALPHA, BETA
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DLASD6, DLASDQ, DLASDT, DLASET, XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( ( ICOMPQ.LT.0 ) .OR. ( ICOMPQ.GT.1 ) ) THEN
         INFO = -1
      ELSE IF( SMLSIZ.LT.3 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( ( SQRE.LT.0 ) .OR. ( SQRE.GT.1 ) ) THEN
         INFO = -4
      ELSE IF( LDU.LT.( N+SQRE ) ) THEN
         INFO = -8
      ELSE IF( LDGCOL.LT.N ) THEN
         INFO = -17
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASDA', -INFO )
         RETURN
      END IF
*
      M = N + SQRE
```

```
*
*     If the input matrix is too small, call DLASDQ to find the SVD.
*
      IF( N.LE.SMLSIZ ) THEN
         IF( ICOMPQ.EQ.0 ) THEN
            CALL DLASDQ( 'U', SQRE, N, 0, 0, 0, D, E, VT, LDU, U, LDU,
     $                   U, LDU, WORK, INFO )
         ELSE
            CALL DLASDQ( 'U', SQRE, N, M, N, 0, D, E, VT, LDU, U, LDU,
     $                   U, LDU, WORK, INFO )
         END IF
         RETURN
      END IF
*
*     Book-keeping and  set up the computation tree.
*
      INODE = 1
      NDIML = INODE + N
      NDIMR = NDIML + N
      IDXQ = NDIMR + N
      IWK = IDXQ + N
*
      NCC = 0
      NRU = 0
*
      SMLSZP = SMLSIZ + 1
      VF = 1
      VL = VF + M
      NWORK1 = VL + M
      NWORK2 = NWORK1 + SMLSZP*SMLSZP
*
      CALL DLASDT( N, NLVL, ND, IWORK( INODE ), IWORK( NDIML ),
     $             IWORK( NDIMR ), SMLSIZ )
*
*     for the nodes on bottom level of the tree, solve
*     their subproblems by DLASDQ.
*
      NDB1 = ( ND+1 ) / 2
      DO 30 I = NDB1, ND
*
*        IC : center row of each node
*        NL : number of rows of left  subproblem
*        NR : number of rows of right subproblem
*        NLF: starting row of the left   subproblem
*        NRF: starting row of the right  subproblem
*
         I1 = I - 1
         IC = IWORK( INODE+I1 )
         NL = IWORK( NDIML+I1 )
         NLP1 = NL + 1
```

```
      NR = IWORK( NDIMR+I1 )
      NLF = IC - NL
      NRF = IC + 1
      IDXQI = IDXQ + NLF - 2
      VFI = VF + NLF - 1
      VLI = VL + NLF - 1
      SQREI = 1
      IF( ICOMPQ.EQ.0 ) THEN
         CALL DLASET( 'A', NLP1, NLP1, ZERO, ONE, WORK( NWORK1 ),
     $                SMLSZP )
         CALL DLASDQ( 'U', SQREI, NL, NLP1, NRU, NCC, D( NLF ),
     $                E( NLF ), WORK( NWORK1 ), SMLSZP,
     $                WORK( NWORK2 ), NL, WORK( NWORK2 ), NL,
     $                WORK( NWORK2 ), INFO )
         ITEMP = NWORK1 + NL*SMLSZP
         CALL DCOPY( NLP1, WORK( NWORK1 ), 1, WORK( VFI ), 1 )
         CALL DCOPY( NLP1, WORK( ITEMP ), 1, WORK( VLI ), 1 )
      ELSE
         CALL DLASET( 'A', NL, NL, ZERO, ONE, U( NLF, 1 ), LDU )
         CALL DLASET( 'A', NLP1, NLP1, ZERO, ONE, VT( NLF, 1 ), LDU )
         CALL DLASDQ( 'U', SQREI, NL, NLP1, NL, NCC, D( NLF ),
     $                E( NLF ), VT( NLF, 1 ), LDU, U( NLF, 1 ), LDU,
     $                U( NLF, 1 ), LDU, WORK( NWORK1 ), INFO )
         CALL DCOPY( NLP1, VT( NLF, 1 ), 1, WORK( VFI ), 1 )
         CALL DCOPY( NLP1, VT( NLF, NLP1 ), 1, WORK( VLI ), 1 )
      END IF
      IF( INFO.NE.0 ) THEN
         RETURN
      END IF
      DO 10 J = 1, NL
         IWORK( IDXQI+J ) = J
10    CONTINUE
      IF( ( I.EQ.ND ) .AND. ( SQRE.EQ.0 ) ) THEN
         SQREI = 0
      ELSE
         SQREI = 1
      END IF
      IDXQI = IDXQI + NLP1
      VFI = VFI + NLP1
      VLI = VLI + NLP1
      NRP1 = NR + SQREI
      IF( ICOMPQ.EQ.0 ) THEN
         CALL DLASET( 'A', NRP1, NRP1, ZERO, ONE, WORK( NWORK1 ),
     $                SMLSZP )
         CALL DLASDQ( 'U', SQREI, NR, NRP1, NRU, NCC, D( NRF ),
     $                E( NRF ), WORK( NWORK1 ), SMLSZP,
     $                WORK( NWORK2 ), NR, WORK( NWORK2 ), NR,
     $                WORK( NWORK2 ), INFO )
         ITEMP = NWORK1 + ( NRP1-1 )*SMLSZP
         CALL DCOPY( NRP1, WORK( NWORK1 ), 1, WORK( VFI ), 1 )
```

```
            CALL DCOPY( NRP1, WORK( ITEMP ), 1, WORK( VLI ), 1 )
         ELSE
            CALL DLASET( 'A', NR, NR, ZERO, ONE, U( NRF, 1 ), LDU )
            CALL DLASET( 'A', NRP1, NRP1, ZERO, ONE, VT( NRF, 1 ), LDU )
            CALL DLASDQ( 'U', SQREI, NR, NRP1, NR, NCC, D( NRF ),
     $                   E( NRF ), VT( NRF, 1 ), LDU, U( NRF, 1 ), LDU,
     $                   U( NRF, 1 ), LDU, WORK( NWORK1 ), INFO )
            CALL DCOPY( NRP1, VT( NRF, 1 ), 1, WORK( VFI ), 1 )
            CALL DCOPY( NRP1, VT( NRF, NRP1 ), 1, WORK( VLI ), 1 )
         END IF
         IF( INFO.NE.0 ) THEN
            RETURN
         END IF
         DO 20 J = 1, NR
            IWORK( IDXQI+J ) = J
   20    CONTINUE
   30 CONTINUE
*
*     Now conquer each subproblem bottom-up.
*
      J = 2**NLVL
      DO 50 LVL = NLVL, 1, -1
         LVL2 = LVL*2 - 1
*
*        Find the first node LF and last node LL on
*        the current level LVL.
*
         IF( LVL.EQ.1 ) THEN
            LF = 1
            LL = 1
         ELSE
            LF = 2**( LVL-1 )
            LL = 2*LF - 1
         END IF
         DO 40 I = LF, LL
            IM1 = I - 1
            IC = IWORK( INODE+IM1 )
            NL = IWORK( NDIML+IM1 )
            NR = IWORK( NDIMR+IM1 )
            NLF = IC - NL
            NRF = IC + 1
            IF( I.EQ.LL ) THEN
               SQREI = SQRE
            ELSE
               SQREI = 1
            END IF
            VFI = VF + NLF - 1
            VLI = VL + NLF - 1
            IDXQI = IDXQ + NLF - 1
            ALPHA = D( IC )
```

```
         BETA = E( IC )
         IF( ICOMPQ.EQ.0 ) THEN
            CALL DLASD6( ICOMPQ, NL, NR, SQREI, D( NLF ),
     $                   WORK( VFI ), WORK( VLI ), ALPHA, BETA,
     $                   IWORK( IDXQI ), PERM, GIVPTR( 1 ), GIVCOL,
     $                   LDGCOL, GIVNUM, LDU, POLES, DIFL, DIFR, Z,
     $                   K( 1 ), C( 1 ), S( 1 ), WORK( NWORK1 ),
     $                   IWORK( IWK ), INFO )
         ELSE
            J = J - 1
            CALL DLASD6( ICOMPQ, NL, NR, SQREI, D( NLF ),
     $                   WORK( VFI ), WORK( VLI ), ALPHA, BETA,
     $                   IWORK( IDXQI ), PERM( NLF, LVL ),
     $                   GIVPTR( J ), GIVCOL( NLF, LVL2 ), LDGCOL,
     $                   GIVNUM( NLF, LVL2 ), LDU,
     $                   POLES( NLF, LVL2 ), DIFL( NLF, LVL ),
     $                   DIFR( NLF, LVL2 ), Z( NLF, LVL ), K( J ),
     $                   C( J ), S( J ), WORK( NWORK1 ),
     $                   IWORK( IWK ), INFO )
         END IF
         IF( INFO.NE.0 ) THEN
            RETURN
         END IF
   40    CONTINUE
   50 CONTINUE
*
      RETURN
*
*     End of DLASDA
*
      END
```

———-

90.42.4 lisp code

— dlasda.lisp —

———-

90.43 dlasdq

90.43.1 Axiom unit tests

— dlasdq.input —

```
)set break resume
)sys rm -f dlasdq.output
)spool dlasdq.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.43.2 Axiom help page

— dlasdq.help —

```
======================================================================
dlasdq examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlasdq (UPLO, SQRE, N, NCVT, NRU, NCC, D, E, VT, LDVT, U,
           LDU, C, LDC, WORK, INFO)
           DLASDQ computes the SVD of a real bidiagonal matrix with diagonal d
           and off-diagonal e. Used by sbdsdc.

Function/Subroutine Documentation
   subroutine dlasdq (character UPLO, integer SQRE, integer N, integer NCVT,
       integer NRU, integer NCC, double precision, dimension( * ) D, double
       precision, dimension( * ) E, double precision, dimension( ldvt, * ) VT,
       integer LDVT, double precision, dimension( ldu, * ) U, integer LDU,
       double precision, dimension( ldc, * ) C, integer LDC, double precision,
       dimension( * ) WORK, integer INFO)
       DLASDQ computes the SVD of a real bidiagonal matrix with diagonal d and
       off-diagonal e. Used by sbdsdc.
```

```
Purpose:
   DLASDQ computes the singular value decomposition (SVD) of a real
   (upper or lower) bidiagonal matrix with diagonal D and offdiagonal
   E, accumulating the transformations if desired. Letting B denote
   the input bidiagonal matrix, the algorithm computes orthogonal
   matrices Q and P such that B = Q * S * P**T (P**T denotes the transpose
   of P). The singular values S are overwritten on D.

   The input matrix U  is changed to U  * Q  if desired.
   The input matrix VT is changed to P**T * VT if desired.
   The input matrix C  is changed to Q**T * C  if desired.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the input bidiagonal matrix
   is upper or lower bidiagonal, and wether it is square are not.
   UPLO = 'U' or 'u'   B is upper bidiagonal.
   UPLO = 'L' or 'l'   B is lower bidiagonal.

 SQRE
   SQRE is INTEGER
   = 0: then the input matrix is N-by-N.
   = 1: then the input matrix is N-by-(N+1) if UPLU = 'U' and
        (N+1)-by-N if UPLU = 'L'.

   The bidiagonal matrix has
   N = NL + NR + 1 rows and
   M = N + SQRE >= N columns.

 N
   N is INTEGER
   On entry, N specifies the number of rows and columns
   in the matrix. N must be at least 0.

 NCVT
   NCVT is INTEGER
   On entry, NCVT specifies the number of columns of
   the matrix VT. NCVT must be at least 0.

 NRU
   NRU is INTEGER
   On entry, NRU specifies the number of rows of
   the matrix U. NRU must be at least 0.

 NCC
   NCC is INTEGER
   On entry, NCC specifies the number of columns of
```

```
  the matrix C. NCC must be at least 0.

D
  D is DOUBLE PRECISION array, dimension (N)
  On entry, D contains the diagonal entries of the
  bidiagonal matrix whose SVD is desired. On normal exit,
  D contains the singular values in ascending order.

E
  E is DOUBLE PRECISION array.
  dimension is (N-1) if SQRE = 0 and N if SQRE = 1.
  On entry, the entries of E contain the offdiagonal entries
  of the bidiagonal matrix whose SVD is desired. On normal
  exit, E will contain 0. If the algorithm does not converge,
  D and E will contain the diagonal and superdiagonal entries
  of a bidiagonal matrix orthogonally equivalent to the one given as input.

VT
  VT is DOUBLE PRECISION array, dimension (LDVT, NCVT)
  On entry, contains a matrix which on exit has been
  premultiplied by P**T, dimension N-by-NCVT if SQRE = 0
  and (N+1)-by-NCVT if SQRE = 1 (not referenced if NCVT=0).

LDVT
  LDVT is INTEGER
  On entry, LDVT specifies the leading dimension of VT as
  declared in the calling (sub) program. LDVT must be at
  least 1. If NCVT is nonzero LDVT must also be at least N.

U
  U is DOUBLE PRECISION array, dimension (LDU, N)
  On entry, contains a  matrix which on exit has been
  postmultiplied by Q, dimension NRU-by-N if SQRE = 0
  and NRU-by-(N+1) if SQRE = 1 (not referenced if NRU=0).

LDU
  LDU is INTEGER
  On entry, LDU  specifies the leading dimension of U as
  declared in the calling (sub) program. LDU must be at least max( 1, NRU )

C
  C is DOUBLE PRECISION array, dimension (LDC, NCC)
  On entry, contains an N-by-NCC matrix which on exit
  has been premultiplied by Q**T  dimension N-by-NCC if SQRE = 0
  and (N+1)-by-NCC if SQRE = 1 (not referenced if NCC=0).

LDC
  LDC is INTEGER
  On entry, LDC  specifies the leading dimension of C as
  declared in the calling (sub) program. LDC must be at
```

```
  least 1. If NCC is nonzero, LDC must also be at least N.

 WORK
  WORK is DOUBLE PRECISION array, dimension (4*N)
  Workspace. Only referenced if one of NCVT, NRU, or NCC is
  nonzero, and if N is at least 2.

 INFO
  INFO is INTEGER
  On exit, a value of 0 indicates a successful exit.
  If INFO < 0, argument number -INFO is illegal.
  If INFO > 0, the algorithm did not converge, and INFO
  specifies how many superdiagonals did not converge.
```

———-

90.43.3 fortran code

— dlasdq.f —

```
*  =====================================================================
      SUBROUTINE DLASDQ( UPLO, SQRE, N, NCVT, NRU, NCC, D, E, VT, LDVT,
     $                   U, LDU, C, LDC, WORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDC, LDU, LDVT, N, NCC, NCVT, NRU, SQRE
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   C( LDC, * ), D( * ), E( * ), U( LDU, * ),
     $                   VT( LDVT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ROTATE
      INTEGER            I, ISUB, IUPLO, J, NP1, SQRE1
```

```
      DOUBLE PRECISION   CS, R, SMIN, SN
*     ..
*     .. External Subroutines ..
      EXTERNAL           DBDSQR, DLARTG, DLASR, DSWAP, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IUPLO = 0
      IF( LSAME( UPLO, 'U' ) )
     $   IUPLO = 1
      IF( LSAME( UPLO, 'L' ) )
     $   IUPLO = 2
      IF( IUPLO.EQ.0 ) THEN
         INFO = -1
      ELSE IF( ( SQRE.LT.0 ) .OR. ( SQRE.GT.1 ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NCVT.LT.0 ) THEN
         INFO = -4
      ELSE IF( NRU.LT.0 ) THEN
         INFO = -5
      ELSE IF( NCC.LT.0 ) THEN
         INFO = -6
      ELSE IF( ( NCVT.EQ.0 .AND. LDVT.LT.1 ) .OR.
     $         ( NCVT.GT.0 .AND. LDVT.LT.MAX( 1, N ) ) ) THEN
         INFO = -10
      ELSE IF( LDU.LT.MAX( 1, NRU ) ) THEN
         INFO = -12
      ELSE IF( ( NCC.EQ.0 .AND. LDC.LT.1 ) .OR.
     $         ( NCC.GT.0 .AND. LDC.LT.MAX( 1, N ) ) ) THEN
         INFO = -14
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASDQ', -INFO )
         RETURN
      END IF
      IF( N.EQ.0 )
     $   RETURN
*
```

```
*     ROTATE is true if any singular vectors desired, false otherwise
*
      ROTATE = ( NCVT.GT.0 ) .OR. ( NRU.GT.0 ) .OR. ( NCC.GT.0 )
      NP1 = N + 1
      SQRE1 = SQRE
*
*     If matrix non-square upper bidiagonal, rotate to be lower
*     bidiagonal.  The rotations are on the right.
*
      IF( ( IUPLO.EQ.1 ) .AND. ( SQRE1.EQ.1 ) ) THEN
         DO 10 I = 1, N - 1
            CALL DLARTG( D( I ), E( I ), CS, SN, R )
            D( I ) = R
            E( I ) = SN*D( I+1 )
            D( I+1 ) = CS*D( I+1 )
            IF( ROTATE ) THEN
               WORK( I ) = CS
               WORK( N+I ) = SN
            END IF
   10    CONTINUE
         CALL DLARTG( D( N ), E( N ), CS, SN, R )
         D( N ) = R
         E( N ) = ZERO
         IF( ROTATE ) THEN
            WORK( N ) = CS
            WORK( N+N ) = SN
         END IF
         IUPLO = 2
         SQRE1 = 0
*
*        Update singular vectors if desired.
*
         IF( NCVT.GT.0 )
     $      CALL DLASR( 'L', 'V', 'F', NP1, NCVT, WORK( 1 ),
     $                  WORK( NP1 ), VT, LDVT )
      END IF
*
*     If matrix lower bidiagonal, rotate to be upper bidiagonal
*     by applying Givens rotations on the left.
*
      IF( IUPLO.EQ.2 ) THEN
         DO 20 I = 1, N - 1
            CALL DLARTG( D( I ), E( I ), CS, SN, R )
            D( I ) = R
            E( I ) = SN*D( I+1 )
            D( I+1 ) = CS*D( I+1 )
            IF( ROTATE ) THEN
               WORK( I ) = CS
               WORK( N+I ) = SN
            END IF
```

```
   20    CONTINUE
*
*        If matrix (N+1)-by-N lower bidiagonal, one additional
*        rotation is needed.
*
         IF( SQRE1.EQ.1 ) THEN
            CALL DLARTG( D( N ), E( N ), CS, SN, R )
            D( N ) = R
            IF( ROTATE ) THEN
               WORK( N ) = CS
               WORK( N+N ) = SN
            END IF
         END IF
*
*        Update singular vectors if desired.
*
         IF( NRU.GT.0 ) THEN
            IF( SQRE1.EQ.0 ) THEN
               CALL DLASR( 'R', 'V', 'F', NRU, N, WORK( 1 ),
     $                     WORK( NP1 ), U, LDU )
            ELSE
               CALL DLASR( 'R', 'V', 'F', NRU, NP1, WORK( 1 ),
     $                     WORK( NP1 ), U, LDU )
            END IF
         END IF
         IF( NCC.GT.0 ) THEN
            IF( SQRE1.EQ.0 ) THEN
               CALL DLASR( 'L', 'V', 'F', N, NCC, WORK( 1 ),
     $                     WORK( NP1 ), C, LDC )
            ELSE
               CALL DLASR( 'L', 'V', 'F', NP1, NCC, WORK( 1 ),
     $                     WORK( NP1 ), C, LDC )
            END IF
         END IF
      END IF
*
*     Call DBDSQR to compute the SVD of the reduced real
*     N-by-N upper bidiagonal matrix.
*
      CALL DBDSQR( 'U', N, NCVT, NRU, NCC, D, E, VT, LDVT, U, LDU, C,
     $             LDC, WORK, INFO )
*
*     Sort the singular values into ascending order (insertion sort on
*     singular values, but only one transposition per singular vector)
*
      DO 40 I = 1, N
*
*        Scan for smallest D(I).
*
         ISUB = I
```

```
         SMIN = D( I )
         DO 30 J = I + 1, N
            IF( D( J ).LT.SMIN ) THEN
               ISUB = J
               SMIN = D( J )
            END IF
   30    CONTINUE
         IF( ISUB.NE.I ) THEN
*
*           Swap singular values and vectors.
*
            D( ISUB ) = D( I )
            D( I ) = SMIN
            IF( NCVT.GT.0 )
     $         CALL DSWAP( NCVT, VT( ISUB, 1 ), LDVT, VT( I, 1 ), LDVT )
            IF( NRU.GT.0 )
     $         CALL DSWAP( NRU, U( 1, ISUB ), 1, U( 1, I ), 1 )
            IF( NCC.GT.0 )
     $         CALL DSWAP( NCC, C( ISUB, 1 ), LDC, C( I, 1 ), LDC )
         END IF
   40 CONTINUE
*
      RETURN
*
*     End of DLASDQ
*
      END
```

———-

90.43.4 lisp code

— **dlasdq.lisp** —

———-

90.44 dlasdt

90.44.1 Axiom unit tests

— **dlasdt.input** —

```
)set break resume
```

```
)sys rm -f dlasdt.output
)spool dlasdt.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

90.44.2 Axiom help page

— dlasdt.help —

```
==================================================================
dlasdt examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlasdt (N, LVL, ND, INODE, NDIML, NDIMR, MSUB)
           DLASDT creates a tree of subproblems for bidiagonal divide and
           conquer. Used by sbdsdc.

Function/Subroutine Documentation
   subroutine dlasdt (integer N, integer LVL, integer ND, integer, dimension(
       * ) INODE, integer, dimension( * ) NDIML, integer, dimension( * )
       NDIMR, integer MSUB)
       DLASDT creates a tree of subproblems for bidiagonal divide and conquer.
       Used by sbdsdc.

Purpose:
   DLASDT creates a tree of subproblems for bidiagonal divide and conquer.

Parameters:

 N
   N is INTEGER
   On entry, the number of diagonal elements of the bidiagonal matrix.

 LVL
   LVL is INTEGER
   On exit, the number of levels on the computation tree.
```

```
ND
  ND is INTEGER
  On exit, the number of nodes on the tree.

INODE
  INODE is INTEGER array, dimension ( N )
  On exit, centers of subproblems.

NDIML
  NDIML is INTEGER array, dimension ( N )
  On exit, row dimensions of left children.

NDIMR
  NDIMR is INTEGER array, dimension ( N )
  On exit, row dimensions of right children.

MSUB
  MSUB is INTEGER
  On entry, the maximum row dimension each subproblem at the
  bottom of the tree can be of.
```

———-

90.44.3 fortran code

— **dlasdt.f** —

```
*  =====================================================================
      SUBROUTINE DLASDT( N, LVL, ND, INODE, NDIML, NDIMR, MSUB )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            LVL, MSUB, N, ND
*     ..
*     .. Array Arguments ..
      INTEGER            INODE( * ), NDIML( * ), NDIMR( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   TWO
```

```
      PARAMETER          ( TWO = 2.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IL, IR, LLST, MAXN, NCRNT, NLVL
      DOUBLE PRECISION   TEMP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, INT, LOG, MAX
*     ..
*     .. Executable Statements ..
*
*     Find the number of levels on the tree.
*
      MAXN = MAX( 1, N )
      TEMP = LOG( DBLE( MAXN ) / DBLE( MSUB+1 ) ) / LOG( TWO )
      LVL = INT( TEMP ) + 1
*
      I = N / 2
      INODE( 1 ) = I + 1
      NDIML( 1 ) = I
      NDIMR( 1 ) = N - I - 1
      IL = 0
      IR = 1
      LLST = 1
      DO 20 NLVL = 1, LVL - 1
*
*        Constructing the tree at (NLVL+1)-st level. The number of
*        nodes created on this level is LLST * 2.
*
         DO 10 I = 0, LLST - 1
            IL = IL + 2
            IR = IR + 2
            NCRNT = LLST + I
            NDIML( IL ) = NDIML( NCRNT ) / 2
            NDIMR( IL ) = NDIML( NCRNT ) - NDIML( IL ) - 1
            INODE( IL ) = INODE( NCRNT ) - NDIMR( IL ) - 1
            NDIML( IR ) = NDIMR( NCRNT ) / 2
            NDIMR( IR ) = NDIMR( NCRNT ) - NDIML( IR ) - 1
            INODE( IR ) = INODE( NCRNT ) + NDIML( IR ) + 1
   10    CONTINUE
         LLST = LLST*2
   20 CONTINUE
      ND = LLST*2 - 1
*
      RETURN
*
*     End of DLASDT
*
      END
```

———-

90.44.4 lisp code

— dlasdt.lisp —

———-

90.45 dlaset

90.45.1 Axiom unit tests

— dlaset.input —

```
)set break resume
)sys rm -f dlaset.output
)spool dlaset.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.45.2 Axiom help page

— dlaset.help —

```
==========================================================================
dlaset examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
```

```
      subroutine dlaset (UPLO, M, N, ALPHA, BETA, A, LDA)
          DLASET initializes the off-diagonal elements and the diagonal
          elements of a matrix to given values.

Function/Subroutine Documentation
   subroutine dlaset (character UPLO, integer M, integer N, double precision
       ALPHA, double precision BETA, double precision, dimension( lda, * ) A,
       integer LDA)
       DLASET initializes the off-diagonal elements and the diagonal elements
       of a matrix to given values.

Purpose:
   DLASET initializes an m-by-n matrix A to BETA on the diagonal and
   ALPHA on the offdiagonals.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies the part of the matrix A to be set.
   = 'U': Upper triangular part is set; the strictly lower
          triangular part of A is not changed.
   = 'L': Lower triangular part is set; the strictly upper
          triangular part of A is not changed.
   Otherwise:  All of the matrix A is set.

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 ALPHA
   ALPHA is DOUBLE PRECISION
   The constant to which the offdiagonal elements are to be set.

 BETA
   BETA is DOUBLE PRECISION
   The constant to which the diagonal elements are to be set.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   On exit, the leading m-by-n submatrix of A is set as follows:

   if UPLO = 'U', A(i,j) = ALPHA, 1<=i<=j-1, 1<=j<=n,
   if UPLO = 'L', A(i,j) = ALPHA, j+1<=i<=m, 1<=j<=n,
   otherwise,     A(i,j) = ALPHA, 1<=i<=m, 1<=j<=n, i.ne.j,
```

```
  and, for all UPLO, A(i,i) = BETA, 1<=i<=min(m,n).

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).
```

———-

90.45.3 fortran code

— **dlaset.f** —

```
*  =====================================================================
      SUBROUTINE DLASET( UPLO, M, N, ALPHA, BETA, A, LDA )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            LDA, M, N
      DOUBLE PRECISION   ALPHA, BETA
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * )
*     ..
*
* =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MIN
*     ..
*     .. Executable Statements ..
*
      IF( LSAME( UPLO, 'U' ) ) THEN
*
*        Set the strictly upper triangular or trapezoidal part of the
*        array to ALPHA.
```

```
*
         DO 20 J = 2, N
            DO 10 I = 1, MIN( J-1, M )
               A( I, J ) = ALPHA
   10       CONTINUE
   20    CONTINUE
*
      ELSE IF( LSAME( UPLO, 'L' ) ) THEN
*
*        Set the strictly lower triangular or trapezoidal part of the
*        array to ALPHA.
*
         DO 40 J = 1, MIN( M, N )
            DO 30 I = J + 1, M
               A( I, J ) = ALPHA
   30       CONTINUE
   40    CONTINUE
*
      ELSE
*
*        Set the leading m-by-n submatrix to ALPHA.
*
         DO 60 J = 1, N
            DO 50 I = 1, M
               A( I, J ) = ALPHA
   50       CONTINUE
   60    CONTINUE
      END IF
*
*     Set the first min(M,N) diagonal elements to BETA.
*
      DO 70 I = 1, MIN( M, N )
         A( I, I ) = BETA
   70 CONTINUE
*
      RETURN
*
*     End of DLASET
*
      END
```

———-

90.45.4 lisp code

— dlaset.lisp —

———-

90.46 dlasr

90.46.1 Axiom unit tests

— **dlasr.input** —

```
)set break resume
)sys rm -f dlasr.output
)spool dlasr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.46.2 Axiom help page

— **dlasr.help** —

```
======================================================================
dlasr examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlasr (SIDE, PIVOT, DIRECT, M, N, C, S, A, LDA)
           DLASR applies a sequence of plane rotations to a general
           rectangular matrix.

Function/Subroutine Documentation
   subroutine dlasr (character SIDE, character PIVOT, character DIRECT,
       integer M, integer N, double precision, dimension( * ) C, double
       precision, dimension( * ) S, double precision, dimension( lda, * ) A,
       integer LDA)
       DLASR applies a sequence of plane rotations to a general rectangular
```

```
      matrix.

Purpose:
   DLASR applies a sequence of plane rotations to a real matrix A,
   from either the left or the right.

   When SIDE = 'L', the transformation takes the form

   A := P*A

   and when SIDE = 'R', the transformation takes the form

   A := A*P**T

   where P is an orthogonal matrix consisting of a sequence of z plane
   rotations, with z = M when SIDE = 'L' and z = N when SIDE = 'R',
   and P**T is the transpose of P.

   When DIRECT = 'F' (Forward sequence), then

   P = P(z-1) * ... * P(2) * P(1)

   and when DIRECT = 'B' (Backward sequence), then

   P = P(1) * P(2) * ... * P(z-1)

   where P(k) is a plane rotation matrix defined by the 2-by-2 rotation

   R(k) = (  c(k)  s(k) )
        = ( -s(k)  c(k) ).

   When PIVOT = 'V' (Variable pivot), the rotation is performed
   for the plane (k,k+1), i.e., P(k) has the form

          P(k) = (  1                                          )
                 (       ...                                   )
                 (              1                              )
                 (                   c(k)  s(k)                )
                 (                  -s(k)  c(k)                )
                 (                                1             )
                 (                                     ...      )
                 (                                          1  )

   where R(k) appears as a rank-2 modification to the identity matrix in
   rows and columns k and k+1.

   When PIVOT = 'T' (Top pivot), the rotation is performed for the
   plane (1,k+1), so P(k) has the form

          P(k) = (  c(k)                    s(k)                )
```

```
                ( 1                                    )
                (      ...                             )
                (             1                        )
                ( -s(k)            c(k)                )
                (                       1              )
                (                            ...       )
                (                                  1 )
```

where R(k) appears in rows and columns 1 and k+1.

Similarly, when PIVOT = 'B' (Bottom pivot), the rotation is performed for the plane (k,z), giving P(k) the form

```
       P(k) = ( 1                                     )
              (      ...                              )
              (             1                         )
              (                  c(k)            s(k) )
              (                        1              )
              (                             ...       )
              (                                  1    )
              (                 -s(k)            c(k) )
```

where R(k) appears in rows and columns k and z. The rotations are performed without ever forming P(k) explicitly.

Parameters:

SIDE
SIDE is CHARACTER*1
Specifies whether the plane rotation matrix P is applied to A on the left or the right.
= 'L': Left, compute A := P*A
= 'R': Right, compute A:= A*P**T

PIVOT
PIVOT is CHARACTER*1
Specifies the plane for which P(k) is a plane rotation matrix.
= 'V': Variable pivot, the plane (k,k+1)
= 'T': Top pivot, the plane (1,k+1)
= 'B': Bottom pivot, the plane (k,z)

DIRECT
DIRECT is CHARACTER*1
Specifies whether P is a forward or backward sequence of plane rotations.
= 'F': Forward, P = P(z-1)*...*P(2)*P(1)
= 'B': Backward, P = P(1)*P(2)*...*P(z-1)

M
M is INTEGER
The number of rows of the matrix A. If m <= 1, an immediate

```
  return is effected.

N
  N is INTEGER
  The number of columns of the matrix A.  If n <= 1, an
  immediate return is effected.

C
  C is DOUBLE PRECISION array, dimension
     (M-1) if SIDE = 'L'
     (N-1) if SIDE = 'R'
  The cosines c(k) of the plane rotations.

S
  S is DOUBLE PRECISION array, dimension
     (M-1) if SIDE = 'L'
     (N-1) if SIDE = 'R'
  The sines s(k) of the plane rotations.  The 2-by-2 plane
  rotation part of the matrix P(k), R(k), has the form
     R(k) = (  c(k)  s(k) )
            ( -s(k)  c(k) ).

A
  A is DOUBLE PRECISION array, dimension (LDA,N)
  The M-by-N matrix A.  On exit, A is overwritten by P*A if
  SIDE = 'R' or by A*P**T if SIDE = 'L'.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).
```

90.46.3 fortran code

— dlasr.f —

```
*  =====================================================================
      SUBROUTINE DLASR( SIDE, PIVOT, DIRECT, M, N, C, S, A, LDA )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          DIRECT, PIVOT, SIDE
```

```
      INTEGER            LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), C( * ), S( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, INFO, J
      DOUBLE PRECISION   CTEMP, STEMP, TEMP
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( .NOT.( LSAME( SIDE, 'L' ) .OR. LSAME( SIDE, 'R' ) ) ) THEN
         INFO = 1
      ELSE IF( .NOT.( LSAME( PIVOT, 'V' ) .OR. LSAME( PIVOT,
     $         'T' ) .OR. LSAME( PIVOT, 'B' ) ) ) THEN
         INFO = 2
      ELSE IF( .NOT.( LSAME( DIRECT, 'F' ) .OR. LSAME( DIRECT, 'B' ) ) )
     $          THEN
         INFO = 3
      ELSE IF( M.LT.0 ) THEN
         INFO = 4
      ELSE IF( N.LT.0 ) THEN
         INFO = 5
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = 9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASR ', INFO )
         RETURN
      END IF
```

```
*
*     Quick return if possible
*
      IF( ( M.EQ.0 ) .OR. ( N.EQ.0 ) )
     $   RETURN
      IF( LSAME( SIDE, 'L' ) ) THEN
*
*        Form  P * A
*
         IF( LSAME( PIVOT, 'V' ) ) THEN
            IF( LSAME( DIRECT, 'F' ) ) THEN
               DO 20 J = 1, M - 1
                  CTEMP = C( J )
                  STEMP = S( J )
                  IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                     DO 10 I = 1, N
                        TEMP = A( J+1, I )
                        A( J+1, I ) = CTEMP*TEMP - STEMP*A( J, I )
                        A( J, I ) = STEMP*TEMP + CTEMP*A( J, I )
   10                CONTINUE
                  END IF
   20          CONTINUE
            ELSE IF( LSAME( DIRECT, 'B' ) ) THEN
               DO 40 J = M - 1, 1, -1
                  CTEMP = C( J )
                  STEMP = S( J )
                  IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                     DO 30 I = 1, N
                        TEMP = A( J+1, I )
                        A( J+1, I ) = CTEMP*TEMP - STEMP*A( J, I )
                        A( J, I ) = STEMP*TEMP + CTEMP*A( J, I )
   30                CONTINUE
                  END IF
   40          CONTINUE
            END IF
         ELSE IF( LSAME( PIVOT, 'T' ) ) THEN
            IF( LSAME( DIRECT, 'F' ) ) THEN
               DO 60 J = 2, M
                  CTEMP = C( J-1 )
                  STEMP = S( J-1 )
                  IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                     DO 50 I = 1, N
                        TEMP = A( J, I )
                        A( J, I ) = CTEMP*TEMP - STEMP*A( 1, I )
                        A( 1, I ) = STEMP*TEMP + CTEMP*A( 1, I )
   50                CONTINUE
                  END IF
   60          CONTINUE
            ELSE IF( LSAME( DIRECT, 'B' ) ) THEN
               DO 80 J = M, 2, -1
```

```
                CTEMP = C( J-1 )
                STEMP = S( J-1 )
                IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                   DO 70 I = 1, N
                      TEMP = A( J, I )
                      A( J, I ) = CTEMP*TEMP - STEMP*A( 1, I )
                      A( 1, I ) = STEMP*TEMP + CTEMP*A( 1, I )
 70                CONTINUE
                END IF
 80          CONTINUE
          END IF
       ELSE IF( LSAME( PIVOT, 'B' ) ) THEN
          IF( LSAME( DIRECT, 'F' ) ) THEN
             DO 100 J = 1, M - 1
                CTEMP = C( J )
                STEMP = S( J )
                IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                   DO 90 I = 1, N
                      TEMP = A( J, I )
                      A( J, I ) = STEMP*A( M, I ) + CTEMP*TEMP
                      A( M, I ) = CTEMP*A( M, I ) - STEMP*TEMP
 90                CONTINUE
                END IF
 100         CONTINUE
          ELSE IF( LSAME( DIRECT, 'B' ) ) THEN
             DO 120 J = M - 1, 1, -1
                CTEMP = C( J )
                STEMP = S( J )
                IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                   DO 110 I = 1, N
                      TEMP = A( J, I )
                      A( J, I ) = STEMP*A( M, I ) + CTEMP*TEMP
                      A( M, I ) = CTEMP*A( M, I ) - STEMP*TEMP
 110               CONTINUE
                END IF
 120         CONTINUE
          END IF
       END IF
    ELSE IF( LSAME( SIDE, 'R' ) ) THEN
*
*       Form A * P**T
*
       IF( LSAME( PIVOT, 'V' ) ) THEN
          IF( LSAME( DIRECT, 'F' ) ) THEN
             DO 140 J = 1, N - 1
                CTEMP = C( J )
                STEMP = S( J )
                IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                   DO 130 I = 1, M
                      TEMP = A( I, J+1 )
```

```
                  A( I, J+1 ) = CTEMP*TEMP - STEMP*A( I, J )
                  A( I, J ) = STEMP*TEMP + CTEMP*A( I, J )
130            CONTINUE
            END IF
140      CONTINUE
         ELSE IF( LSAME( DIRECT, 'B' ) ) THEN
            DO 160 J = N - 1, 1, -1
               CTEMP = C( J )
               STEMP = S( J )
               IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                  DO 150 I = 1, M
                     TEMP = A( I, J+1 )
                     A( I, J+1 ) = CTEMP*TEMP - STEMP*A( I, J )
                     A( I, J ) = STEMP*TEMP + CTEMP*A( I, J )
150               CONTINUE
               END IF
160         CONTINUE
         END IF
      ELSE IF( LSAME( PIVOT, 'T' ) ) THEN
         IF( LSAME( DIRECT, 'F' ) ) THEN
            DO 180 J = 2, N
               CTEMP = C( J-1 )
               STEMP = S( J-1 )
               IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                  DO 170 I = 1, M
                     TEMP = A( I, J )
                     A( I, J ) = CTEMP*TEMP - STEMP*A( I, 1 )
                     A( I, 1 ) = STEMP*TEMP + CTEMP*A( I, 1 )
170               CONTINUE
               END IF
180         CONTINUE
         ELSE IF( LSAME( DIRECT, 'B' ) ) THEN
            DO 200 J = N, 2, -1
               CTEMP = C( J-1 )
               STEMP = S( J-1 )
               IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                  DO 190 I = 1, M
                     TEMP = A( I, J )
                     A( I, J ) = CTEMP*TEMP - STEMP*A( I, 1 )
                     A( I, 1 ) = STEMP*TEMP + CTEMP*A( I, 1 )
190               CONTINUE
               END IF
200         CONTINUE
         END IF
      ELSE IF( LSAME( PIVOT, 'B' ) ) THEN
         IF( LSAME( DIRECT, 'F' ) ) THEN
            DO 220 J = 1, N - 1
               CTEMP = C( J )
               STEMP = S( J )
               IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
```

```
                  DO 210 I = 1, M
                     TEMP = A( I, J )
                     A( I, J ) = STEMP*A( I, N ) + CTEMP*TEMP
                     A( I, N ) = CTEMP*A( I, N ) - STEMP*TEMP
 210              CONTINUE
               END IF
 220         CONTINUE
          ELSE IF( LSAME( DIRECT, 'B' ) ) THEN
             DO 240 J = N - 1, 1, -1
                CTEMP = C( J )
                STEMP = S( J )
                IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                   DO 230 I = 1, M
                      TEMP = A( I, J )
                      A( I, J ) = STEMP*A( I, N ) + CTEMP*TEMP
                      A( I, N ) = CTEMP*A( I, N ) - STEMP*TEMP
 230              CONTINUE
                END IF
 240         CONTINUE
          END IF
       END IF
    END IF
*
    RETURN
*
*   End of DLASR
*
    END
```

———-

90.46.4 lisp code

— dlasr.lisp —

———-

90.47 dlassq

90.47.1 Axiom unit tests

— dlassq.input —

```
)set break resume
)sys rm -f dlassq.output
)spool dlassq.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.47.2 Axiom help page

— **dlassq.help** —

```
==================================================================
dlassq examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlassq (N, X, INCX, SCALE, SUMSQ)
           DLASSQ updates a sum of squares represented in scaled form.

Function/Subroutine Documentation
   subroutine dlassq (integer N, double precision, dimension( * ) X, integer
       INCX, double precision SCALE, double precision SUMSQ)
       DLASSQ updates a sum of squares represented in scaled form.

Purpose:
   DLASSQ  returns the values  scl  and  smsq  such that

   ( scl**2 )*smsq = x( 1 )**2 +...+ x( n )**2 + ( scale**2 )*sumsq,

   where  x( i ) = X( 1 + ( i - 1 )*INCX ). The value of  sumsq  is
   assumed to be non-negative and  scl  returns the value

   scl = max( scale, abs( x( i ) ) ).

   scale and sumsq must be supplied in SCALE and SUMSQ and
   scl and smsq are overwritten on SCALE and SUMSQ respectively.

   The routine makes only one pass through the vector x.
```

```
Parameters:

 N
   N is INTEGER
   The number of elements to be used from the vector X.

 X
   X is DOUBLE PRECISION array, dimension (N)
   The vector for which a scaled sum of squares is computed.
   x( i )  = X( 1 + ( i - 1 )*INCX ), 1 <= i <= n.

 INCX
   INCX is INTEGER
   The increment between successive values of the vector X.
   INCX > 0.

 SCALE
   SCALE is DOUBLE PRECISION
   On entry, the value  scale  in the equation above.
   On exit, SCALE is overwritten with  scl , the scaling factor
   for the sum of squares.

 SUMSQ
   SUMSQ is DOUBLE PRECISION
   On entry, the value  sumsq  in the equation above.
   On exit, SUMSQ is overwritten with  smsq , the basic sum of
   squares from which  scl  has been factored out.
```

90.47.3 fortran code

— dlassq.f —

```
*  =====================================================================
      SUBROUTINE DLASSQ( N, X, INCX, SCALE, SUMSQ )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INCX, N
      DOUBLE PRECISION   SCALE, SUMSQ
*     ..
```

```
*     .. Array Arguments ..
      DOUBLE PRECISION   X( * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            IX
      DOUBLE PRECISION   ABSXI
*     ..
*     .. External Functions ..
      LOGICAL            DISNAN
      EXTERNAL           DISNAN
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS
*     ..
*     .. Executable Statements ..
*
      IF( N.GT.0 ) THEN
         DO 10 IX = 1, 1 + ( N-1 )*INCX, INCX
            ABSXI = ABS( X( IX ) )
            IF( ABSXI.GT.ZERO.OR.DISNAN( ABSXI ) ) THEN
               IF( SCALE.LT.ABSXI ) THEN
                  SUMSQ = 1 + SUMSQ*( SCALE / ABSXI )**2
                  SCALE = ABSXI
               ELSE
                  SUMSQ = SUMSQ + ( ABSXI / SCALE )**2
               END IF
            END IF
   10    CONTINUE
      END IF
      RETURN
*
*     End of DLASSQ
*
      END
```

———-

90.47.4 lisp code

— dlassq.lisp —

———-

90.48 dlasv2

90.48.1 Axiom unit tests

— dlasv2.input —

```
)set break resume
)sys rm -f dlasv2.output
)spool dlasv2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.48.2 Axiom help page

— dlasv2.help —

```
======================================================================
dlasv2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine dlasv2 (F, G, H, SSMIN, SSMAX, SNR, CSR, SNL, CSL)
           DLASV2 computes the singular value decomposition of a 2-by-2
           triangular matrix.

Function/Subroutine Documentation
   subroutine dlasv2 (double precision F, double precision G, double precision
       H, double precision SSMIN, double precision SSMAX, double precision
       SNR, double precision CSR, double precision SNL, double precision CSL)
```

```
      DLASV2 computes the singular value decomposition of a 2-by-2 triangular
      matrix.

Purpose:
   DLASV2 computes the singular value decomposition of a 2-by-2
   triangular matrix
     [  F   G  ]
     [  0   H  ].
   On return, abs(SSMAX) is the larger singular value, abs(SSMIN) is the
   smaller singular value, and (CSL,SNL) and (CSR,SNR) are the left and
   right singular vectors for abs(SSMAX), giving the decomposition

     [ CSL  SNL ] [  F   G  ] [ CSR -SNR ]  =  [ SSMAX   0   ]
     [-SNL  CSL ] [  0   H  ] [ SNR  CSR ]     [  0    SSMIN ].

Parameters:

 F
   F is DOUBLE PRECISION
   The (1,1) element of the 2-by-2 matrix.

 G
   G is DOUBLE PRECISION
   The (1,2) element of the 2-by-2 matrix.

 H
   H is DOUBLE PRECISION
   The (2,2) element of the 2-by-2 matrix.

 SSMIN
   SSMIN is DOUBLE PRECISION
   abs(SSMIN) is the smaller singular value.

 SSMAX
   SSMAX is DOUBLE PRECISION
   abs(SSMAX) is the larger singular value.

 SNL
   SNL is DOUBLE PRECISION

 CSL
   CSL is DOUBLE PRECISION
   The vector (CSL, SNL) is a unit left singular vector for the
   singular value abs(SSMAX).

 SNR
   SNR is DOUBLE PRECISION

 CSR
   CSR is DOUBLE PRECISION
```

```
   The vector (CSR, SNR) is a unit right singular vector for the
   singular value abs(SSMAX).

Further Details:

   Any input parameter may be aliased with any output parameter.

   Barring over/underflow and assuming a guard digit in subtraction, all
   output quantities are correct to within a few units in the last
   place (ulps).

   In IEEE arithmetic, the code works correctly if one matrix element is
   infinite.

   Overflow will not occur unless the largest singular value itself
   overflows or is within a few ulps of overflow. (On machines with
   partial overflow, like the Cray, overflow may occur if the largest
   singular value is within a factor of 2 of overflow.)

   Underflow is harmless if underflow is gradual. Otherwise, results
   may correspond to a matrix modified by perturbations of size near
   the underflow threshold.
```

———-

90.48.3 fortran code

— dlasv2.f —

```
*  ===================================================================
      SUBROUTINE DLASV2( F, G, H, SSMIN, SSMAX, SNR, CSR, SNL, CSL )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   CSL, CSR, F, G, H, SNL, SNR, SSMAX, SSMIN
*     ..
*
* ====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
      DOUBLE PRECISION   HALF
```

```
      PARAMETER          ( HALF = 0.5D0 )
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D0 )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D0 )
      DOUBLE PRECISION   FOUR
      PARAMETER          ( FOUR = 4.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            GASMAL, SWAP
      INTEGER            PMAX
      DOUBLE PRECISION   A, CLT, CRT, D, FA, FT, GA, GT, HA, HT, L, M,
     $                   MM, R, S, SLT, SRT, T, TEMP, TSIGN, TT
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SIGN, SQRT
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. Executable Statements ..
*
      FT = F
      FA = ABS( FT )
      HT = H
      HA = ABS( H )
*
*     PMAX points to the maximum absolute element of matrix
*       PMAX = 1 if F largest in absolute values
*       PMAX = 2 if G largest in absolute values
*       PMAX = 3 if H largest in absolute values
*
      PMAX = 1
      SWAP = ( HA.GT.FA )
      IF( SWAP ) THEN
         PMAX = 3
         TEMP = FT
         FT = HT
         HT = TEMP
         TEMP = FA
         FA = HA
         HA = TEMP
*
*        Now FA .ge. HA
*
      END IF
      GT = G
      GA = ABS( GT )
      IF( GA.EQ.ZERO ) THEN
```

```
*
*        Diagonal matrix
*
         SSMIN = HA
         SSMAX = FA
         CLT = ONE
         CRT = ONE
         SLT = ZERO
         SRT = ZERO
      ELSE
         GASMAL = .TRUE.
         IF( GA.GT.FA ) THEN
            PMAX = 2
            IF( ( FA / GA ).LT.DLAMCH( 'EPS' ) ) THEN
*
*              Case of very large GA
*
               GASMAL = .FALSE.
               SSMAX = GA
               IF( HA.GT.ONE ) THEN
                  SSMIN = FA / ( GA / HA )
               ELSE
                  SSMIN = ( FA / GA )*HA
               END IF
               CLT = ONE
               SLT = HT / GT
               SRT = ONE
               CRT = FT / GT
            END IF
         END IF
         IF( GASMAL ) THEN
*
*           Normal case
*
            D = FA - HA
            IF( D.EQ.FA ) THEN
*
*              Copes with infinite F or H
*
               L = ONE
            ELSE
               L = D / FA
            END IF
*
*           Note that 0 .le. L .le. 1
*
            M = GT / FT
*
*           Note that abs(M) .le. 1/macheps
*
```

```
            T = TWO - L
*
*           Note that T .ge. 1
*
            MM = M*M
            TT = T*T
            S = SQRT( TT+MM )
*
*           Note that 1 .le. S .le. 1 + 1/macheps
*
            IF( L.EQ.ZERO ) THEN
               R = ABS( M )
            ELSE
               R = SQRT( L*L+MM )
            END IF
*
*           Note that 0 .le. R .le. 1 + 1/macheps
*
            A = HALF*( S+R )
*
*           Note that 1 .le. A .le. 1 + abs(M)
*
            SSMIN = HA / A
            SSMAX = FA*A
            IF( MM.EQ.ZERO ) THEN
*
*              Note that M is very tiny
*
               IF( L.EQ.ZERO ) THEN
                  T = SIGN( TWO, FT )*SIGN( ONE, GT )
               ELSE
                  T = GT / SIGN( D, FT ) + M / T
               END IF
            ELSE
               T = ( M / ( S+T )+M / ( R+L ) )*( ONE+A )
            END IF
            L = SQRT( T*T+FOUR )
            CRT = TWO / L
            SRT = T / L
            CLT = ( CRT+SRT*M ) / A
            SLT = ( HT / FT )*SRT / A
         END IF
      END IF
      IF( SWAP ) THEN
         CSL = SRT
         SNL = CRT
         CSR = SLT
         SNR = CLT
      ELSE
         CSL = CLT
```

```
         SNL = SLT
         CSR = CRT
         SNR = SRT
      END IF
*
*     Correct signs of SSMAX and SSMIN
*
      IF( PMAX.EQ.1 )
     $   TSIGN = SIGN( ONE, CSR )*SIGN( ONE, CSL )*SIGN( ONE, F )
      IF( PMAX.EQ.2 )
     $   TSIGN = SIGN( ONE, SNR )*SIGN( ONE, CSL )*SIGN( ONE, G )
      IF( PMAX.EQ.3 )
     $   TSIGN = SIGN( ONE, SNR )*SIGN( ONE, SNL )*SIGN( ONE, H )
      SSMAX = SIGN( SSMAX, TSIGN )
      SSMIN = SIGN( SSMIN, TSIGN*SIGN( ONE, F )*SIGN( ONE, H ) )
      RETURN
*
*     End of DLASV2
*
      END
```

———-

90.48.4 lisp code

— dlasv2.lisp —

———-

90.49 dsecnd

90.49.1 Axiom unit tests

— dsecnd.input —

```
)set break resume
)sys rm -f dsecnd.output
)spool dsecnd.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

90.49.2 Axiom help page

— **dsecnd.help** —

```
======================================================================
dsecnd examples
======================================================================

======================================================================
Man Page Details
======================================================================

double precision function dsecnd()

Purpose:
   DSECND returns the user time for a process in seconds.
   This version gets the time from the EXTERNAL system function.
```

———-

90.49.3 fortran code

— **dsecnd.f** —

———-

90.49.4 lisp code

— **dsecnd.lisp** —

———-

90.50 ieeeck

90.50.1 Axiom unit tests

— ieeeck.input —

```
)set break resume
)sys rm -f ieeeck.output
)spool ieeeck.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.50.2 Axiom help page

— ieeeck.help —

```
======================================================================
ieeeck examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       integer function ieeeck (ISPEC, ZERO, ONE)
           IEEECK

Function/Subroutine Documentation
   integer function ieeeck (integer ISPEC, real ZERO, real ONE)
       IEEECK

Purpose:
   IEEECK is called from the ILAENV to verify that Infinity and
   possibly NaN arithmetic is safe (i.e. will not trap).

Parameters:

 ISPEC
```

```
   ISPEC is INTEGER
   Specifies whether to test just for inifinity arithmetic
   or whether to test for infinity and NaN arithmetic.
   = 0: Verify infinity arithmetic only.
   = 1: Verify infinity and NaN arithmetic.

 ZERO
   ZERO is REAL
   Must contain the value 0.0
   This is passed to prevent the compiler from optimizing away this code.

 ONE
   ONE is REAL
   Must contain the value 1.0
   This is passed to prevent the compiler from optimizing away this code.

 RETURN VALUE:  INTEGER
   = 0:  Arithmetic failed to produce the correct answers
   = 1:  Arithmetic produced the correct answers
```

90.50.3 fortran code

— ieeeck.f —

```
*  =====================================================================
      INTEGER          FUNCTION IEEECK( ISPEC, ZERO, ONE )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            ISPEC
      REAL               ONE, ZERO
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      REAL               NAN1, NAN2, NAN3, NAN4, NAN5, NAN6, NEGINF,
     $                   NEGZRO, NEWZRO, POSINF
*     ..
*     .. Executable Statements ..
      IEEECK = 1
```

```
*
      POSINF = ONE / ZERO
      IF( POSINF.LE.ONE ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      NEGINF = -ONE / ZERO
      IF( NEGINF.GE.ZERO ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      NEGZRO = ONE / ( NEGINF+ONE )
      IF( NEGZRO.NE.ZERO ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      NEGINF = ONE / NEGZRO
      IF( NEGINF.GE.ZERO ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      NEWZRO = NEGZRO + ZERO
      IF( NEWZRO.NE.ZERO ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      POSINF = ONE / NEWZRO
      IF( POSINF.LE.ONE ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      NEGINF = NEGINF*POSINF
      IF( NEGINF.GE.ZERO ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      POSINF = POSINF*POSINF
      IF( POSINF.LE.ONE ) THEN
         IEEECK = 0
         RETURN
      END IF
*
*
```

```
*
*
*     Return if we were only asked to check infinity arithmetic
*
      IF( ISPEC.EQ.0 )
     $   RETURN
*
      NAN1 = POSINF + NEGINF
*
      NAN2 = POSINF / NEGINF
*
      NAN3 = POSINF / POSINF
*
      NAN4 = POSINF*ZERO
*
      NAN5 = NEGINF*NEGZRO
*
      NAN6 = NAN5*ZERO
*
      IF( NAN1.EQ.NAN1 ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      IF( NAN2.EQ.NAN2 ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      IF( NAN3.EQ.NAN3 ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      IF( NAN4.EQ.NAN4 ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      IF( NAN5.EQ.NAN5 ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      IF( NAN6.EQ.NAN6 ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      RETURN
```

```
      END
```

———-

90.50.4 lisp code

— **ieeeck.lisp** —

———-

90.51 iladlc

90.51.1 Axiom unit tests

— **iladlc.input** —

```
)set break resume
)sys rm -f iladlc.output
)spool iladlc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.51.2 Axiom help page

— **iladlc.help** —

```
==========================================================================
iladlc examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       integer function iladlc (M, N, A, LDA)
           ILADLC scans a matrix for its last non-zero column.

Function/Subroutine Documentation
   integer function iladlc (integer M, integer N, double precision, dimension(
       lda, * ) A, integer LDA)
       ILADLC scans a matrix for its last non-zero column.

Purpose:
   ILADLC scans A for its last non-zero column.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.

 N
   N is INTEGER
   The number of columns of the matrix A.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
   The m by n matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A. LDA >= max(1,M).
```

90.51.3 fortran code

— iladlc.f —

```
*  =====================================================================
      INTEGER FUNCTION ILADLC( M, N, A, LDA )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            M, N, LDA
```

```
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION ZERO
      PARAMETER ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER I
*     ..
*     .. Executable Statements ..
*
*     Quick test for the common case where one corner is non-zero.
      IF( N.EQ.0 ) THEN
         ILADLC = N
      ELSE IF( A(1, N).NE.ZERO .OR. A(M, N).NE.ZERO ) THEN
         ILADLC = N
      ELSE
*     Now scan each column from the end, returning with the first non-zero.
         DO ILADLC = N, 1, -1
            DO I = 1, M
               IF( A(I, ILADLC).NE.ZERO ) RETURN
            END DO
         END DO
      END IF
      RETURN
      END
```

———-

90.51.4 lisp code

— iladlc.lisp —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun iladlc (m n a lda)
    (declare (type (array double-float (*)) a)
             (type (f2cl-lib:integer4) lda n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%))
      (prog ((i 0) (iladlc 0))
        (declare (type (f2cl-lib:integer4) i iladlc))
```

```
        (cond ((= n 0) (setf iladlc n))
              ((or (/= (f2cl-lib:fref a (1 n) ((1 lda) (1 *))) zero)
                   (/= (f2cl-lib:fref a (m n) ((1 lda) (1 *))) zero))
               (setf iladlc n))
              (t
               (f2cl-lib:fdo (iladlc n
                              (f2cl-lib:int-add iladlc (f2cl-lib:int-sub 1)))
                             ((> iladlc 1) nil)
                 (tagbody
                   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                 ((> i m) nil)
                     (tagbody
                       (if (/=
                            (f2cl-lib:fref a-%data% (i iladlc) ((1 lda) (1 *))
                                           a-%offset%)
                            zero)
                           (go end_label))
                      label100001))
                  label100000))))
        (go end_label)
       end_label
        (return (values iladlc nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::iladlc
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values '(nil nil nil nil)
                                            :calls 'nil)))
```

———-

90.52 iladlr

90.52.1 Axiom unit tests

— iladlr.input —

```
)set break resume
```

```
)sys rm -f iladlr.output
)spool iladlr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.52.2 Axiom help page

— iladlr.help —

```
=====================================================================
iladlr examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       integer function iladlr (M, N, A, LDA)
           ILADLR scans a matrix for its last non-zero row.

Function/Subroutine Documentation
   integer function iladlr (integer M, integer N, double precision, dimension(
       lda, * ) A, integer LDA)
       ILADLR scans a matrix for its last non-zero row.

Purpose:
   ILADLR scans A for its last non-zero row.

Parameters:

 M
   M is INTEGER
   The number of rows of the matrix A.

 N
   N is INTEGER
   The number of columns of the matrix A.

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
```

```
   The m by n matrix A.

 LDA
   LDA is INTEGER
   The leading dimension of the array A. LDA >= max(1,M).
```

90.52.3 fortran code

— iladlr.f —

```
*  =====================================================================
      INTEGER FUNCTION ILADLR( M, N, A, LDA )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            M, N, LDA
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION ZERO
      PARAMETER ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER I, J
*     ..
*     .. Executable Statements ..
*
*     Quick test for the common case where one corner is non-zero.
      IF( M.EQ.0 ) THEN
         ILADLR = M
      ELSE IF( A(M, 1).NE.ZERO .OR. A(M, N).NE.ZERO ) THEN
         ILADLR = M
      ELSE
*     Scan up each column tracking the last zero row seen.
         ILADLR = 0
         DO J = 1, N
```

```
          I=M
          DO WHILE((A(MAX(I,1),J).EQ.ZERO).AND.(I.GE.1))
             I=I-1
          ENDDO
          ILADLR = MAX( ILADLR, I )
       END DO
    END IF
    RETURN
    END
```

———-

90.52.4 lisp code

— iladlr.lisp —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun iladlr (m n a lda)
    (declare (type (array double-float (*)) a)
             (type (f2cl-lib:integer4) lda n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%))
      (prog ((i 0) (j 0) (iladlr 0))
        (declare (type (f2cl-lib:integer4) i j iladlr))
        (cond ((= m 0) (setf iladlr m))
              ((or (/= (f2cl-lib:fref a (m 1) ((1 lda) (1 *))) zero)
                   (/= (f2cl-lib:fref a (m n) ((1 lda) (1 *))) zero))
               (setf iladlr m))
              (t (setf iladlr 0)
               (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                             ((> j n) nil)
                 (tagbody
                   (setf i m)
                  label100001
                   (if (not
                        (and
                         (=
                          (f2cl-lib:fref a-%data%
                                         ((max (the f2cl-lib:integer4 i)
                                               (the f2cl-lib:integer4 1))
                                          j)
                                         ((1 lda) (1 *)) a-%offset%)
                          zero)
                         (>= i 1)))
                       (go label100002))
                   (setf i (f2cl-lib:int-sub i 1))
```

```
                  (go label100001)
                 label100002
                  (setf iladlr
                          (max (the f2cl-lib:integer4 iladlr)
                               (the f2cl-lib:integer4 i)))
                 label100000))))
        (go end_label)
       end_label
        (return (values iladlr nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::iladlr
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values '(nil nil nil nil)
                                            :calls 'nil)))
```

———-

90.53 ilaenv

90.53.1 Axiom unit tests

— ilaenv.input —

```
)set break resume
)sys rm -f ilaenv.output
)spool ilaenv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.53.2 Axiom help page

— ilaenv.help —

```
=====================================================================
ilaenv examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

integer function ilaenv(Integer ISPEC, character*(*) NAME,
                        character*(*) OPTS, Integer N1, Integer N2,
                        Integer N3, Integer N4)

Purpose:
   ILAENV is called from the LAPACK routines to choose problem-dependent
   parameters for the local environment. See ISPEC for a description of
   the parameters.

   ILAENV returns an INTEGER
   if ILAENV >= 0: ILAENV returns the value of the parameter specified by ISPEC
   if ILAENV < 0: if ELAENV = 0k, the kth argument had an illegal value

   This version provides a set of parameters which should give good,
   but not optimal, performance on many of the currently available
   computers. Users are encouraged to modify this subroutine to set
   the tuning parameters for their particular machine using the option
   and problem size information in the arguments.

   This routine will not function correctly if it is converted to all
   lower case. Converting it to all upper case is allowed.

Parameters:

 ISPEC
   ISPEC is INTEGER
   Specifies the parameter to be returned as the value of ILAENV
   =1: the optimal blocksize; if this value is 1, an unblocked
       algorithm will give the best performance
   =2: the minimum block size for which the block routine
       should be used; if the usable block size is less than
       this value, an unblocked routine should be used.
   =3: the crossover point (in a block routine, for N less
       than this value, an unblocked routine should be used)
   =4: the number of shifts, used in the nonsymmetric
       eigenvalue routines (DEPRECATED)
   =5: the minimum column dimension for blocking to be used;
```

```
      rectangular blocks must have dimension at least k by n,
      where k is given by ILAENV(2,...) and m by ILAENV(5,...)
  =6: the crossover point for the SVD (when reducing an m by n
      matrix to bidiagonal form, if max(m,n)/min(m,n) excees
      this value, a QR factorization is used first to reduce
      the matrix to a triangular form)
  =7: the number of processors
  =8: the crossover point for the multishift QR method
      for nonsymmetric eigenvalue problems (DEPRECATED)
  =9: maximum size of the subproblems at the bottom of the
      computation tree in the divide-and-conquer algorithm
      (used by xGELSD and xGESDD)
  =10: ieee NaN arithmetic can be trusted not to trap
  =11: infinity arithmetic can be trusted not to trap
  12 <= ISPEC <= 16
       xHSEQR or related subroutines
       see IPARMQ for detailed explanation

NAME
  NAME is CHARACTER*(*)
  The name of the calling subroutine, in either upper case or lower case.

OPTS
  OPTS is CHARACTER*(*)
  The character options to the subroutine NAME, concatenated
  into a single character string. For example, UPLO='U',
  TRANS='T', and DIAG='N' for a triangular routine would
  be specified as OPTS='UTN'

N1
  N1 is Integer

N2
  N2 is Integer

N3
  N3 is Integer

N4
  N4 is Integer
  Problem dimensions for the subroutine NAME; these may not all be required
```

———-

90.53.3 fortran code

— ilaenv.f —

```
*  =====================================================================
      INTEGER FUNCTION ILAENV( ISPEC, NAME, OPTS, N1, N2, N3, N4 )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER*( * )    NAME, OPTS
      INTEGER            ISPEC, N1, N2, N3, N4
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, IC, IZ, NB, NBMIN, NX
      LOGICAL            CNAME, SNAME
      CHARACTER          C1*1, C2*2, C4*2, C3*3, SUBNAM*6
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          CHAR, ICHAR, INT, MIN, REAL
*     ..
*     .. External Functions ..
      INTEGER            IEEECK, IPARMQ
      EXTERNAL           IEEECK, IPARMQ
*     ..
*     .. Executable Statements ..
*
      GO TO ( 10, 10, 10, 80, 90, 100, 110, 120,
     $        130, 140, 150, 160, 160, 160, 160, 160 )ISPEC
*
*     Invalid value for ISPEC
*
      ILAENV = -1
      RETURN
*
   10 CONTINUE
*
*     Convert NAME to upper case if the first character is lower case.
*
      ILAENV = 1
      SUBNAM = NAME
      IC = ICHAR( SUBNAM( 1: 1 ) )
      IZ = ICHAR( 'Z' )
      IF( IZ.EQ.90 .OR. IZ.EQ.122 ) THEN
*
*        ASCII character set
*
         IF( IC.GE.97 .AND. IC.LE.122 ) THEN
```

```
            SUBNAM( 1: 1 ) = CHAR( IC-32 )
            DO 20 I = 2, 6
               IC = ICHAR( SUBNAM( I: I ) )
               IF( IC.GE.97 .AND. IC.LE.122 )
     $            SUBNAM( I: I ) = CHAR( IC-32 )
   20       CONTINUE
         END IF
*
      ELSE IF( IZ.EQ.233 .OR. IZ.EQ.169 ) THEN
*
*        EBCDIC character set
*
         IF( ( IC.GE.129 .AND. IC.LE.137 ) .OR.
     $       ( IC.GE.145 .AND. IC.LE.153 ) .OR.
     $       ( IC.GE.162 .AND. IC.LE.169 ) ) THEN
            SUBNAM( 1: 1 ) = CHAR( IC+64 )
            DO 30 I = 2, 6
               IC = ICHAR( SUBNAM( I: I ) )
               IF( ( IC.GE.129 .AND. IC.LE.137 ) .OR.
     $             ( IC.GE.145 .AND. IC.LE.153 ) .OR.
     $             ( IC.GE.162 .AND. IC.LE.169 ) )SUBNAM( I:
     $             I ) = CHAR( IC+64 )
   30       CONTINUE
         END IF
*
      ELSE IF( IZ.EQ.218 .OR. IZ.EQ.250 ) THEN
*
*        Prime machines:  ASCII+128
*
         IF( IC.GE.225 .AND. IC.LE.250 ) THEN
            SUBNAM( 1: 1 ) = CHAR( IC-32 )
            DO 40 I = 2, 6
               IC = ICHAR( SUBNAM( I: I ) )
               IF( IC.GE.225 .AND. IC.LE.250 )
     $            SUBNAM( I: I ) = CHAR( IC-32 )
   40       CONTINUE
         END IF
      END IF
*
      C1 = SUBNAM( 1: 1 )
      SNAME = C1.EQ.'S' .OR. C1.EQ.'D'
      CNAME = C1.EQ.'C' .OR. C1.EQ.'Z'
      IF( .NOT.( CNAME .OR. SNAME ) )
     $   RETURN
      C2 = SUBNAM( 2: 3 )
      C3 = SUBNAM( 4: 6 )
      C4 = C3( 2: 3 )
*
      GO TO ( 50, 60, 70 )ISPEC
*
```

```
   50 CONTINUE
*
*     ISPEC = 1:  block size
*
*     In these examples, separate code is provided for setting NB for
*     real and complex.  We assume that NB will take the same value in
*     single or double precision.
*
      NB = 1
*
      IF( C2.EQ.'GE' ) THEN
         IF( C3.EQ.'TRF' ) THEN
            IF( SNAME ) THEN
               NB = 64
            ELSE
               NB = 64
            END IF
         ELSE IF( C3.EQ.'QRF' .OR. C3.EQ.'RQF' .OR. C3.EQ.'LQF' .OR.
     $           C3.EQ.'QLF' ) THEN
            IF( SNAME ) THEN
               NB = 32
            ELSE
               NB = 32
            END IF
         ELSE IF( C3.EQ.'HRD' ) THEN
            IF( SNAME ) THEN
               NB = 32
            ELSE
               NB = 32
            END IF
         ELSE IF( C3.EQ.'BRD' ) THEN
            IF( SNAME ) THEN
               NB = 32
            ELSE
               NB = 32
            END IF
         ELSE IF( C3.EQ.'TRI' ) THEN
            IF( SNAME ) THEN
               NB = 64
            ELSE
               NB = 64
            END IF
         END IF
      ELSE IF( C2.EQ.'PO' ) THEN
         IF( C3.EQ.'TRF' ) THEN
            IF( SNAME ) THEN
               NB = 64
            ELSE
               NB = 64
            END IF
```

```
      END IF
   ELSE IF( C2.EQ.'SY' ) THEN
      IF( C3.EQ.'TRF' ) THEN
         IF( SNAME ) THEN
            NB = 64
         ELSE
            NB = 64
         END IF
      ELSE IF( SNAME .AND. C3.EQ.'TRD' ) THEN
         NB = 32
      ELSE IF( SNAME .AND. C3.EQ.'GST' ) THEN
         NB = 64
      END IF
   ELSE IF( CNAME .AND. C2.EQ.'HE' ) THEN
      IF( C3.EQ.'TRF' ) THEN
         NB = 64
      ELSE IF( C3.EQ.'TRD' ) THEN
         NB = 32
      ELSE IF( C3.EQ.'GST' ) THEN
         NB = 64
      END IF
   ELSE IF( SNAME .AND. C2.EQ.'OR' ) THEN
      IF( C3( 1: 1 ).EQ.'G' ) THEN
         IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR. C4.EQ.
     $       'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR. C4.EQ.'BR' )
     $        THEN
            NB = 32
         END IF
      ELSE IF( C3( 1: 1 ).EQ.'M' ) THEN
         IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR. C4.EQ.
     $       'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR. C4.EQ.'BR' )
     $        THEN
            NB = 32
         END IF
      END IF
   ELSE IF( CNAME .AND. C2.EQ.'UN' ) THEN
      IF( C3( 1: 1 ).EQ.'G' ) THEN
         IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR. C4.EQ.
     $       'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR. C4.EQ.'BR' )
     $        THEN
            NB = 32
         END IF
      ELSE IF( C3( 1: 1 ).EQ.'M' ) THEN
         IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR. C4.EQ.
     $       'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR. C4.EQ.'BR' )
     $        THEN
            NB = 32
         END IF
      END IF
   ELSE IF( C2.EQ.'GB' ) THEN
```

```
      IF( C3.EQ.'TRF' ) THEN
         IF( SNAME ) THEN
            IF( N4.LE.64 ) THEN
               NB = 1
            ELSE
               NB = 32
            END IF
         ELSE
            IF( N4.LE.64 ) THEN
               NB = 1
            ELSE
               NB = 32
            END IF
         END IF
      END IF
   ELSE IF( C2.EQ.'PB' ) THEN
      IF( C3.EQ.'TRF' ) THEN
         IF( SNAME ) THEN
            IF( N2.LE.64 ) THEN
               NB = 1
            ELSE
               NB = 32
            END IF
         ELSE
            IF( N2.LE.64 ) THEN
               NB = 1
            ELSE
               NB = 32
            END IF
         END IF
      END IF
   ELSE IF( C2.EQ.'TR' ) THEN
      IF( C3.EQ.'TRI' ) THEN
         IF( SNAME ) THEN
            NB = 64
         ELSE
            NB = 64
         END IF
      END IF
   ELSE IF( C2.EQ.'LA' ) THEN
      IF( C3.EQ.'UUM' ) THEN
         IF( SNAME ) THEN
            NB = 64
         ELSE
            NB = 64
         END IF
      END IF
   ELSE IF( SNAME .AND. C2.EQ.'ST' ) THEN
      IF( C3.EQ.'EBZ' ) THEN
         NB = 1
```

```
         END IF
      ELSE IF( C2.EQ.'GG' ) THEN
         NB = 32
         IF( C3.EQ.'HD3' ) THEN
            IF( SNAME ) THEN
               NB = 32
            ELSE
               NB = 32
            END IF
         END IF
      END IF
      ILAENV = NB
      RETURN
*
   60 CONTINUE
*
*     ISPEC = 2:  minimum block size
*
      NBMIN = 2
      IF( C2.EQ.'GE' ) THEN
         IF( C3.EQ.'QRF' .OR. C3.EQ.'RQF' .OR. C3.EQ.'LQF' .OR. C3.EQ.
     $       'QLF' ) THEN
            IF( SNAME ) THEN
               NBMIN = 2
            ELSE
               NBMIN = 2
            END IF
         ELSE IF( C3.EQ.'HRD' ) THEN
            IF( SNAME ) THEN
               NBMIN = 2
            ELSE
               NBMIN = 2
            END IF
         ELSE IF( C3.EQ.'BRD' ) THEN
            IF( SNAME ) THEN
               NBMIN = 2
            ELSE
               NBMIN = 2
            END IF
         ELSE IF( C3.EQ.'TRI' ) THEN
            IF( SNAME ) THEN
               NBMIN = 2
            ELSE
               NBMIN = 2
            END IF
         END IF
      ELSE IF( C2.EQ.'SY' ) THEN
         IF( C3.EQ.'TRF' ) THEN
            IF( SNAME ) THEN
               NBMIN = 8
```

```
            ELSE
               NBMIN = 8
            END IF
         ELSE IF( SNAME .AND. C3.EQ.'TRD' ) THEN
            NBMIN = 2
         END IF
      ELSE IF( CNAME .AND. C2.EQ.'HE' ) THEN
         IF( C3.EQ.'TRD' ) THEN
            NBMIN = 2
         END IF
      ELSE IF( SNAME .AND. C2.EQ.'OR' ) THEN
         IF( C3( 1: 1 ).EQ.'G' ) THEN
            IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR. C4.EQ.
     $          'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR. C4.EQ.'BR' )
     $           THEN
               NBMIN = 2
            END IF
         ELSE IF( C3( 1: 1 ).EQ.'M' ) THEN
            IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR. C4.EQ.
     $          'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR. C4.EQ.'BR' )
     $           THEN
               NBMIN = 2
            END IF
         END IF
      ELSE IF( CNAME .AND. C2.EQ.'UN' ) THEN
         IF( C3( 1: 1 ).EQ.'G' ) THEN
            IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR. C4.EQ.
     $          'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR. C4.EQ.'BR' )
     $           THEN
               NBMIN = 2
            END IF
         ELSE IF( C3( 1: 1 ).EQ.'M' ) THEN
            IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR. C4.EQ.
     $          'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR. C4.EQ.'BR' )
     $           THEN
               NBMIN = 2
            END IF
         END IF
      ELSE IF( C2.EQ.'GG' ) THEN
         NBMIN = 2
         IF( C3.EQ.'HD3' ) THEN
            NBMIN = 2
         END IF
      END IF
      ILAENV = NBMIN
      RETURN
*
   70 CONTINUE
*
*     ISPEC = 3:  crossover point
```

```
*
      NX = 0
      IF( C2.EQ.'GE' ) THEN
         IF( C3.EQ.'QRF' .OR. C3.EQ.'RQF' .OR. C3.EQ.'LQF' .OR. C3.EQ.
     $       'QLF' ) THEN
            IF( SNAME ) THEN
               NX = 128
            ELSE
               NX = 128
            END IF
         ELSE IF( C3.EQ.'HRD' ) THEN
            IF( SNAME ) THEN
               NX = 128
            ELSE
               NX = 128
            END IF
         ELSE IF( C3.EQ.'BRD' ) THEN
            IF( SNAME ) THEN
               NX = 128
            ELSE
               NX = 128
            END IF
         END IF
      ELSE IF( C2.EQ.'SY' ) THEN
         IF( SNAME .AND. C3.EQ.'TRD' ) THEN
            NX = 32
         END IF
      ELSE IF( CNAME .AND. C2.EQ.'HE' ) THEN
         IF( C3.EQ.'TRD' ) THEN
            NX = 32
         END IF
      ELSE IF( SNAME .AND. C2.EQ.'OR' ) THEN
         IF( C3( 1: 1 ).EQ.'G' ) THEN
            IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR. C4.EQ.
     $          'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR. C4.EQ.'BR' )
     $           THEN
               NX = 128
            END IF
         END IF
      ELSE IF( CNAME .AND. C2.EQ.'UN' ) THEN
         IF( C3( 1: 1 ).EQ.'G' ) THEN
            IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR. C4.EQ.
     $          'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR. C4.EQ.'BR' )
     $           THEN
               NX = 128
            END IF
         END IF
      ELSE IF( C2.EQ.'GG' ) THEN
         NX = 128
         IF( C3.EQ.'HD3' ) THEN
```

```
            NX = 128
         END IF
      END IF
      ILAENV = NX
      RETURN
*
   80 CONTINUE
*
*     ISPEC = 4:  number of shifts (used by xHSEQR)
*
      ILAENV = 6
      RETURN
*
   90 CONTINUE
*
*     ISPEC = 5:  minimum column dimension (not used)
*
      ILAENV = 2
      RETURN
*
  100 CONTINUE
*
*     ISPEC = 6:  crossover point for SVD (used by xGELSS and xGESVD)
*
      ILAENV = INT( REAL( MIN( N1, N2 ) )*1.6E0 )
      RETURN
*
  110 CONTINUE
*
*     ISPEC = 7:  number of processors (not used)
*
      ILAENV = 1
      RETURN
*
  120 CONTINUE
*
*     ISPEC = 8:  crossover point for multishift (used by xHSEQR)
*
      ILAENV = 50
      RETURN
*
  130 CONTINUE
*
*     ISPEC = 9:  maximum size of the subproblems at the bottom of the
*                 computation tree in the divide-and-conquer algorithm
*                 (used by xGELSD and xGESDD)
*
      ILAENV = 25
      RETURN
*
```

```
  140 CONTINUE
*
*     ISPEC = 10: ieee NaN arithmetic can be trusted not to trap
*
*     ILAENV = 0
      ILAENV = 1
      IF( ILAENV.EQ.1 ) THEN
         ILAENV = IEEECK( 1, 0.0, 1.0 )
      END IF
      RETURN
*
  150 CONTINUE
*
*     ISPEC = 11: infinity arithmetic can be trusted not to trap
*
*     ILAENV = 0
      ILAENV = 1
      IF( ILAENV.EQ.1 ) THEN
         ILAENV = IEEECK( 0, 0.0, 1.0 )
      END IF
      RETURN
*
  160 CONTINUE
*
*     12 <= ISPEC <= 16: xHSEQR or related subroutines.
*
      ILAENV = IPARMQ( ISPEC, NAME, OPTS, N1, N2, N3, N4 )
      RETURN
*
*     End of ILAENV
*
      END
```

———-

90.53.4 lisp code

— ilaenv.lisp —

———-

90.54 ilaver

90.54.1 Axiom unit tests

— ilaver.input —

```
)set break resume
)sys rm -f ilaver.output
)spool ilaver.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.54.2 Axiom help page

— ilaver.help —

```
====================================================================
ilaver examples
====================================================================

====================================================================
Man Page Details
====================================================================

subroutine ilaver(Integer VERS_MAJOR, Integer VERS_MINOR, Integer VERS_PATCH)

Purpose:
   This subroutine returns the LAPACK version.

Parameter:

 VERS_MAJOR
   return the lapack major version

  VERS_MINOR
   return the lapack minor version from the major version

  VERS_PATCH
   return the lapack patch version from the minor version
```

———-

90.54.3 fortran code

— **ilaver.f** —

```
*  =====================================================================
      SUBROUTINE ILAVER( VERS_MAJOR, VERS_MINOR, VERS_PATCH )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*  =====================================================================
*
      INTEGER VERS_MAJOR, VERS_MINOR, VERS_PATCH
*  =====================================================================
      VERS_MAJOR = 3
      VERS_MINOR = 6
      VERS_PATCH = 0
*  =====================================================================
*
      RETURN
      END
```

———-

90.54.4 lisp code

— **ilaver.lisp** —

```
(defun ilaver (vers_major vers_minor vers_patch)
  (declare (type (f2cl-lib:integer4) vers_patch vers_minor vers_major))
  (prog ()
    (declare)
    (setf vers_major 3)
    (setf vers_minor 6)
    (setf vers_patch 0)
    (go end_label)
   end_label
    (return (values vers_major vers_minor vers_patch))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
```

```
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::ilaver
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::vers_major
                                              fortran-to-lisp::vers_minor
                                              fortran-to-lisp::vers_patch)
                                            :calls 'nil)))
```

———-

90.55 iparmq

90.55.1 Axiom unit tests

— **iparmq.input** —

```
)set break resume
)sys rm -f iparmq.output
)spool iparmq.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.55.2 Axiom help page

— **iparmq.help** —

```
==================================================================
iparmq examples
==================================================================

==================================================================
```

```
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       integer function iparmq (ISPEC, NAME, OPTS, N, ILO, IHI, LWORK)
           IPARMQ

Function/Subroutine Documentation
   integer function iparmq (integer ISPEC, character, dimension( * ) NAME,
       character, dimension( * ) OPTS, integer N, integer ILO, integer IHI,
       integer LWORK)
       IPARMQ

Purpose:
   This program sets problem and machine dependent parameters
   useful for xHSEQR and related subroutines for eigenvalue
   problems. It is called whenever
   IPARMQ is called with 12 <= ISPEC <= 16

Parameters:

 ISPEC
   ISPEC is integer scalar
   ISPEC specifies which tunable parameter IPARMQ should
   return.

   ISPEC=12: (INMIN)  Matrices of order nmin or less
      are sent directly to xLAHQR, the implicit
      double shift QR algorithm.  NMIN must be at least 11.

   ISPEC=13: (INWIN)  Size of the deflation window.
     This is best set greater than or equal to
      the number of simultaneous shifts NS.
      Larger matrices benefit from larger deflation windows.

   ISPEC=14: (INIBL) Determines when to stop nibbling and
      invest in an (expensive) multi-shift QR sweep.
      If the aggressive early deflation subroutine
      finds LD converged eigenvalues from an order
      NW deflation window and LD.GT.(NW*NIBBLE)/100,
      then the next QR sweep is skipped and early
      deflation is applied immediately to the
      remaining active diagonal block.  Setting
      IPARMQ(ISPEC=14) = 0 causes TTQRE to skip a
      multi-shift QR sweep whenever early deflation
      finds a converged eigenvalue.  Setting
      IPARMQ(ISPEC=14) greater than or equal to 100
      prevents TTQRE from skipping a multi-shift QR sweep.
```

```
  ISPEC=15: (NSHFTS) The number of simultaneous shifts in
     a multi-shift QR iteration.

  ISPEC=16: (IACC22) IPARMQ is set to 0, 1 or 2 with the
     following meanings.
     0:  During the multi-shift QR/QZ sweep,
         blocked eigenvalue reordering, blocked
         Hessenberg-triangular reduction,
         reflections and/or rotations are not
         accumulated when updating the
         far-from-diagonal matrix entries.
     1:  During the multi-shift QR/QZ sweep,
         blocked eigenvalue reordering, blocked
         Hessenberg-triangular reduction,
         reflections and/or rotations are
         accumulated, and matrix-matrix
         multiplication is used to update the
         far-from-diagonal matrix entries.
     2:  During the multi-shift QR/QZ sweep,
         blocked eigenvalue reordering, blocked
         Hessenberg-triangular reduction,
         reflections and/or rotations are
         accumulated, and 2-by-2 block structure
         is exploited during matrix-matrix multiplies.
  (If xTRMM is slower than xGEMM, then
  IPARMQ(ISPEC=16)=1 may be more efficient than
  IPARMQ(ISPEC=16)=2 despite the greater level of
  arithmetic work implied by the latter choice.)

NAME
  NAME is character string
  Name of the calling subroutine

OPTS
  OPTS is character string
  This is a concatenation of the string arguments to TTQRE.

N
  N is integer scalar
  N is the order of the Hessenberg matrix H.

ILO
  ILO is INTEGER

IHI
  IHI is INTEGER
  It is assumed that H is already upper triangular
  in rows and columns 1:ILO-1 and IHI+1:N.

LWORK
```

```
   LWORK is integer scalar
   The amount of workspace available.

Further Details:
   Little is known about how best to choose these parameters.
   It is possible to use different values of the parameters
   for each of CHSEQR, DHSEQR, SHSEQR and ZHSEQR.

   It is probably best to choose different parameters for
   different matrices and different parameters at different
   times during the iteration, but this has not been
   implemented --- yet.


   The best choices of most of the parameters depend
   in an ill-understood way on the relative execution
   rate of xLAQR3 and xLAQR5 and on the nature of each
   particular eigenvalue problem.  Experiment may be the
   only practical way to determine which choices are most effective.

   Following is a list of default values supplied by IPARMQ.
   These defaults may be adjusted in order to attain better
   performance in any particular computational environment.

   IPARMQ(ISPEC=12) The xLAHQR vs xLAQR0 crossover point.
      Default: 75. (Must be at least 11.)

   IPARMQ(ISPEC=13) Recommended deflation window size.
      This depends on ILO, IHI and NS, the
      number of simultaneous shifts returned
      by IPARMQ(ISPEC=15).  The default for
      (IHI-ILO+1).LE.500 is NS.  The default
      for (IHI-ILO+1).GT.500 is 3*NS/2.

   IPARMQ(ISPEC=14) Nibble crossover point.  Default: 14.

   IPARMQ(ISPEC=15) Number of simultaneous shifts, NS.
      a multi-shift QR iteration.

   If IHI-ILO+1 is ...

                    greater than      ...but less    ... the
                    or equal to ...      than        default is

                            0               30       NS =   2+
                           30               60       NS =   4+
                           60              150       NS =  10
                          150              590       NS =  **
                          590             3000       NS =  64
                         3000             6000       NS = 128
```

```
                        6000             infinity   NS = 256

                  (+)  By default matrices of this order are
                       passed to the implicit double shift routine
                       xLAHQR.  See IPARMQ(ISPEC=12) above.   These
                       values of NS are used only in case of a rare
                       xLAHQR failure.

                  (**) The asterisks (**) indicate an ad-hoc
                       function increasing from 10 to 64.

   IPARMQ(ISPEC=16) Select structured matrix multiply.
   (See ISPEC=16 above for details.)
   Default: 3.
```

———-

90.55.3 fortran code

— iparmq.f —

```
*  ====================================================================
      INTEGER FUNCTION IPARMQ( ISPEC, NAME, OPTS, N, ILO, IHI, LWORK )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            IHI, ILO, ISPEC, LWORK, N
      CHARACTER          NAME*( * ), OPTS*( * )
*
*  =====================================================================
*     .. Parameters ..
      INTEGER            INMIN, INWIN, INIBL, ISHFTS, IACC22
      PARAMETER          ( INMIN = 12, INWIN = 13, INIBL = 14,
     $                   ISHFTS = 15, IACC22 = 16 )
      INTEGER            NMIN, K22MIN, KACMIN, NIBBLE, KNWSWP
      PARAMETER          ( NMIN = 75, K22MIN = 14, KACMIN = 14,
     $                   NIBBLE = 14, KNWSWP = 500 )
      REAL               TWO
      PARAMETER          ( TWO = 2.0 )
*     ..
*     .. Local Scalars ..
      INTEGER            NH, NS
      INTEGER            I, IC, IZ
```

```
      CHARACTER          SUBNAM*6
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          LOG, MAX, MOD, NINT, REAL
*     ..
*     .. Executable Statements ..
      IF( ( ISPEC.EQ.ISHFTS ) .OR. ( ISPEC.EQ.INWIN ) .OR.
     $    ( ISPEC.EQ.IACC22 ) ) THEN
*
*        ==== Set the number simultaneous shifts ====
*
         NH = IHI - ILO + 1
         NS = 2
         IF( NH.GE.30 )
     $      NS = 4
         IF( NH.GE.60 )
     $      NS = 10
         IF( NH.GE.150 )
     $      NS = MAX( 10, NH / NINT( LOG( REAL( NH ) ) / LOG( TWO ) ) )
         IF( NH.GE.590 )
     $      NS = 64
         IF( NH.GE.3000 )
     $      NS = 128
         IF( NH.GE.6000 )
     $      NS = 256
         NS = MAX( 2, NS-MOD( NS, 2 ) )
      END IF
*
      IF( ISPEC.EQ.INMIN ) THEN
*
*
*        ===== Matrices of order smaller than NMIN get sent
*        .     to xLAHQR, the classic double shift algorithm.
*        .     This must be at least 11. ====
*
         IPARMQ = NMIN
*
      ELSE IF( ISPEC.EQ.INIBL ) THEN
*
*        ==== INIBL: skip a multi-shift qr iteration and
*        .    whenever aggressive early deflation finds
*        .    at least (NIBBLE*(window size)/100) deflations. ====
*
         IPARMQ = NIBBLE
*
      ELSE IF( ISPEC.EQ.ISHFTS ) THEN
*
*        ==== NSHFTS: The number of simultaneous shifts =====
*
         IPARMQ = NS
```

```
*
      ELSE IF( ISPEC.EQ.INWIN ) THEN
*
*        ==== NW: deflation window size.  ====
*
         IF( NH.LE.KNWSWP ) THEN
            IPARMQ = NS
         ELSE
            IPARMQ = 3*NS / 2
         END IF
*
      ELSE IF( ISPEC.EQ.IACC22 ) THEN
*
*        ==== IACC22: Whether to accumulate reflections
*        .     before updating the far-from-diagonal elements
*        .     and whether to use 2-by-2 block structure while
*        .     doing it.  A small amount of work could be saved
*        .     by making this choice dependent also upon the
*        .     NH=IHI-ILO+1.
*
*
*        Convert NAME to upper case if the first character is lower case.
*
         IPARMQ = 0
         SUBNAM = NAME
         IC = ICHAR( SUBNAM( 1: 1 ) )
         IZ = ICHAR( 'Z' )
         IF( IZ.EQ.90 .OR. IZ.EQ.122 ) THEN
*
*           ASCII character set
*
            IF( IC.GE.97 .AND. IC.LE.122 ) THEN
               SUBNAM( 1: 1 ) = CHAR( IC-32 )
               DO I = 2, 6
                  IC = ICHAR( SUBNAM( I: I ) )
                  IF( IC.GE.97 .AND. IC.LE.122 )
     $               SUBNAM( I: I ) = CHAR( IC-32 )
               END DO
            END IF
*
         ELSE IF( IZ.EQ.233 .OR. IZ.EQ.169 ) THEN
*
*           EBCDIC character set
*
            IF( ( IC.GE.129 .AND. IC.LE.137 ) .OR.
     $          ( IC.GE.145 .AND. IC.LE.153 ) .OR.
     $          ( IC.GE.162 .AND. IC.LE.169 ) ) THEN
               SUBNAM( 1: 1 ) = CHAR( IC+64 )
               DO I = 2, 6
                  IC = ICHAR( SUBNAM( I: I ) )
```

```
                IF( ( IC.GE.129 .AND. IC.LE.137 ) .OR.
     $              ( IC.GE.145 .AND. IC.LE.153 ) .OR.
     $              ( IC.GE.162 .AND. IC.LE.169 ) )SUBNAM( I:
     $              I ) = CHAR( IC+64 )
             END DO
          END IF
*
        ELSE IF( IZ.EQ.218 .OR. IZ.EQ.250 ) THEN
*
*          Prime machines:  ASCII+128
*
          IF( IC.GE.225 .AND. IC.LE.250 ) THEN
             SUBNAM( 1: 1 ) = CHAR( IC-32 )
             DO I = 2, 6
                IC = ICHAR( SUBNAM( I: I ) )
                IF( IC.GE.225 .AND. IC.LE.250 )
     $             SUBNAM( I: I ) = CHAR( IC-32 )
             END DO
          END IF
        END IF
*
        IF( SUBNAM( 2:6 ).EQ.'GGHRD' .OR.
     $       SUBNAM( 2:6 ).EQ.'GGHD3' ) THEN
          IPARMQ = 1
          IF( NH.GE.K22MIN )
     $       IPARMQ = 2
        ELSE IF ( SUBNAM( 4:6 ).EQ.'EXC' ) THEN
          IF( NH.GE.KACMIN )
     $       IPARMQ = 1
          IF( NH.GE.K22MIN )
     $       IPARMQ = 2
        ELSE IF ( SUBNAM( 2:6 ).EQ.'HSEQR' .OR.
     $            SUBNAM( 2:5 ).EQ.'LAQR' ) THEN
          IF( NS.GE.KACMIN )
     $       IPARMQ = 1
          IF( NS.GE.K22MIN )
     $       IPARMQ = 2
        END IF
*
      ELSE
*        ===== invalid value of ispec =====
        IPARMQ = -1
*
      END IF
*
*     ==== End of IPARMQ ====
*
      END
```

———-

90.55.4 lisp code

— iparmq.lisp —

```
(let* ((inmin 12)
       (inwin 13)
       (inibl 14)
       (ishfts 15)
       (iacc22 16)
       (nmin 75)
       (k22min 14)
       (kacmin 14)
       (nibble 14)
       (knwswp 500)
       (two 2.0))
  (declare (type (f2cl-lib:integer4 12 12) inmin)
           (type (f2cl-lib:integer4 13 13) inwin)
           (type (f2cl-lib:integer4 14 14) inibl)
           (type (f2cl-lib:integer4 15 15) ishfts)
           (type (f2cl-lib:integer4 16 16) iacc22)
           (type (f2cl-lib:integer4 75 75) nmin)
           (type (f2cl-lib:integer4 14 14) k22min)
           (type (f2cl-lib:integer4 14 14) kacmin)
           (type (f2cl-lib:integer4 14 14) nibble)
           (type (f2cl-lib:integer4 500 500) knwswp)
           (type (single-float 2.0 2.0) two)
           (ignorable inmin inwin inibl ishfts iacc22 nmin k22min kacmin nibble
            knwswp two))
  (defun iparmq (ispec name opts n ilo ihi lwork)
    (declare (type (string *) opts name)
             (type (f2cl-lib:integer4) lwork ihi ilo n ispec))
    (f2cl-lib:with-multi-array-data
        ((name character name-%data% name-%offset%)
         (opts character opts-%data% opts-%offset%))
      (prog ((subnam
              (make-array '(6) :element-type 'character :initial-element #\ ))
             (i 0) (ic 0) (iz 0) (nh 0) (ns 0) (iparmq 0))
        (declare (type (string 6) subnam)
                 (type (f2cl-lib:integer4) i ic iz nh ns iparmq))
        (cond
         ((or (= ispec ishfts) (= ispec inwin) (= ispec iacc22))
          (setf nh (f2cl-lib:int-add (f2cl-lib:int-sub ihi ilo) 1)) (setf ns 2)
          (if (>= nh 30)
              (setf ns 4))
          (if (>= nh 60)
              (setf ns 10))
          (if (>= nh 150)
              (setf ns
                      (max 10
```

```
                    (the f2cl-lib:integer4
                         (truncate nh
                                   (f2cl-lib:nint
                                    (/
                                     (f2cl-lib:flog (f2cl-lib:freal nh))
                                     (f2cl-lib:flog two))))))))
  (if (>= nh 590)
      (setf ns 64))
  (if (>= nh 3000)
      (setf ns 128))
  (if (>= nh 6000)
      (setf ns 256))
  (setf ns
          (max (the f2cl-lib:integer4 2)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-sub ns (mod ns 2)))))))
(cond ((= ispec inmin) (setf iparmq nmin))
      ((= ispec inibl) (setf iparmq nibble))
      ((= ispec ishfts) (setf iparmq ns))
      ((= ispec inwin)
       (cond ((<= nh knwswp) (setf iparmq ns))
             (t
              (setf iparmq
                      (the f2cl-lib:integer4 (truncate (* 3 ns) 2))))))
      ((= ispec iacc22) (setf iparmq 0)
       (f2cl-lib:f2cl-set-string subnam name (string 6))
       (setf ic (f2cl-lib:ichar (f2cl-lib:fref-string subnam (1 1))))
       (setf iz (f2cl-lib:ichar "Z"))
       (cond
        ((or (= iz 90) (= iz 122))
         (cond
          ((and (>= ic 97) (<= ic 122))
           (f2cl-lib:fset-string (f2cl-lib:fref-string subnam (1 1))
                                 (f2cl-lib:fchar
                                  (f2cl-lib:int-sub ic 32)))
           (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                         ((> i 6) nil)
             (tagbody
               (setf ic
                       (f2cl-lib:ichar
                        (f2cl-lib:fref-string subnam (i i))))
               (if (and (>= ic 97) (<= ic 122))
                   (f2cl-lib:fset-string
                    (f2cl-lib:fref-string subnam (i i))
                    (f2cl-lib:fchar (f2cl-lib:int-sub ic 32))))
              label100000)))))
        ((or (= iz 233) (= iz 169))
         (cond
          ((or (and (>= ic 129) (<= ic 137))
               (and (>= ic 145) (<= ic 153))
```

```
        (and (>= ic 162) (<= ic 169)))
    (f2cl-lib:fset-string (f2cl-lib:fref-string subnam (1 1))
                          (f2cl-lib:fchar
                           (f2cl-lib:int-add ic 64)))
    (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                  ((> i 6) nil)
      (tagbody
        (setf ic
                (f2cl-lib:ichar
                 (f2cl-lib:fref-string subnam (i i))))
        (if (or (and (>= ic 129) (<= ic 137))
                (and (>= ic 145) (<= ic 153))
                (and (>= ic 162) (<= ic 169)))
            (f2cl-lib:fset-string
             (f2cl-lib:fref-string subnam (i i))
             (f2cl-lib:fchar (f2cl-lib:int-add ic 64))))
       label100001)))))
 ((or (= iz 218) (= iz 250))
  (cond
   ((and (>= ic 225) (<= ic 250))
    (f2cl-lib:fset-string (f2cl-lib:fref-string subnam (1 1))
                          (f2cl-lib:fchar
                           (f2cl-lib:int-sub ic 32)))
    (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                  ((> i 6) nil)
      (tagbody
        (setf ic
                (f2cl-lib:ichar
                 (f2cl-lib:fref-string subnam (i i))))
        (if (and (>= ic 225) (<= ic 250))
            (f2cl-lib:fset-string
             (f2cl-lib:fref-string subnam (i i))
             (f2cl-lib:fchar (f2cl-lib:int-sub ic 32))))
       label100002))))))
(cond
 ((or
   (f2cl-lib:fstring-= (f2cl-lib:fref-string subnam (2 6))
                       "GGHRD")
   (f2cl-lib:fstring-= (f2cl-lib:fref-string subnam (2 6))
                       "GGHD3"))
  (setf iparmq 1)
  (if (>= nh k22min)
      (setf iparmq 2)))
 ((f2cl-lib:fstring-= (f2cl-lib:fref-string subnam (4 6)) "EXC")
  (if (>= nh kacmin)
      (setf iparmq 1))
  (if (>= nh k22min)
      (setf iparmq 2)))
 ((or
   (f2cl-lib:fstring-= (f2cl-lib:fref-string subnam (2 6))
```

```
                                    "HSEQR")
                 (f2cl-lib:fstring-= (f2cl-lib:fref-string subnam (2 5))
                                    "LAQR"))
                (if (>= ns kacmin)
                    (setf iparmq 1))
                (if (>= ns k22min)
                    (setf iparmq 2)))))
              (t (setf iparmq -1)))
       end_label
        (return (values iparmq nil nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::iparmq
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil)
                                            :calls 'nil)))
```

———-

90.56 lsame

90.56.1 Axiom unit tests

— **lsame.input** —

```
)set break resume
)sys rm -f lsame.output
)spool lsame.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.56.2 Axiom help page

— **lsame.help** —

```
==================================================================
lsame examples
==================================================================

==================================================================
Man Page Details
==================================================================

logical function lsame(character CA, character CB)

Purpose:
   LSAME returns .TRUE. if CA is the same letter as CB regardless of case

Parameters:

 CA
 CB
   CA and CB specify the single characters to be compared
```

———-

90.56.3 fortran code

— **lsame.f** —

```
*  ==================================================================
      LOGICAL          FUNCTION LSAMEN( N, CA, CB )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER*( * )    CA, CB
      INTEGER            N
*     ..
*
```

```
* ======================================================================
*
*     .. Local Scalars ..
      INTEGER            I
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          LEN
*     ..
*     .. Executable Statements ..
*
      LSAMEN = .FALSE.
      IF( LEN( CA ).LT.N .OR. LEN( CB ).LT.N )
     $   GO TO 20
*
*     Do for each character in the two strings.
*
      DO 10 I = 1, N
*
*        Test if the characters are equal using LSAME.
*
         IF( .NOT.LSAME( CA( I: I ), CB( I: I ) ) )
     $      GO TO 20
*
   10 CONTINUE
      LSAMEN = .TRUE.
*
   20 CONTINUE
      RETURN
*
*     End of LSAMEN
*
      END
```

———-

90.56.4 lisp code

— lsame.lisp —

```
(defun lsamen (n ca cb)
  (declare (type (string *) cb ca)
           (type (f2cl-lib:integer4) n))
  (f2cl-lib:with-multi-array-data
      ((ca character ca-%data% ca-%offset%)
```

```
       (cb character cb-%data% cb-%offset%))
    (prog ((i 0) (lsamen nil))
      (declare (type f2cl-lib:logical lsamen)
               (type (f2cl-lib:integer4) i))
      (setf lsamen f2cl-lib:%false%)
      (if (or (< (f2cl-lib:len ca) n) (< (f2cl-lib:len cb) n))
          (go label20))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (if (not
               (lsame (f2cl-lib:fref-string ca (i i))
                (f2cl-lib:fref-string cb (i i))))
              (go label20))
         label10))
      (setf lsamen f2cl-lib:%true%)
     label20
      (go end_label)
     end_label
      (return (values lsamen nil nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::lsamen
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (string) (string))
                                            :return-values '(nil nil nil)
                                            :calls 'nil)))
```

———-

90.57 lsamen

90.57.1 Axiom unit tests

— lsamen.input —

```
)set break resume
)sys rm -f lsamen.output
)spool lsamen.output
)set message test on
)set message auto off
```

```
)clear all

)spool
)lisp (bye)
```

———-

90.57.2 Axiom help page

— **lsamen.help** —

```
=====================================================================
lsamen examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       logical function lsamen (N, CA, CB)
           LSAMEN

Function/Subroutine Documentation
   logical function lsamen (integer N, character*( * ) CA, character*( * ) CB)
       LSAMEN

Purpose:
   LSAMEN  tests if the first N letters of CA are the same as the
   first N letters of CB, regardless of case.
   LSAMEN returns .TRUE. if CA and CB are equivalent except for case
   and .FALSE. otherwise.  LSAMEN also returns .FALSE. if LEN( CA )
   or LEN( CB ) is less than N.

Parameters:

 N
   N is INTEGER
   The number of characters in CA and CB to be compared.

 CA
   CA is CHARACTER*(*)

 CB
   CB is CHARACTER*(*)
   CA and CB specify two character strings of length at least N.
   Only the first N characters of each string will be accessed.
```

———-

90.57.3 fortran code

— **lsamen.f** —

```
*  =====================================================================
      LOGICAL          FUNCTION LSAMEN( N, CA, CB )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER*( * )    CA, CB
      INTEGER            N
*     ..
*
* =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          LEN
*     ..
*     .. Executable Statements ..
*
      LSAMEN = .FALSE.
      IF( LEN( CA ).LT.N .OR. LEN( CB ).LT.N )
     $   GO TO 20
*
*     Do for each character in the two strings.
*
      DO 10 I = 1, N
*
*        Test if the characters are equal using LSAME.
*
         IF( .NOT.LSAME( CA( I: I ), CB( I: I ) ) )
     $      GO TO 20
*
```

```
   10 CONTINUE
      LSAMEN = .TRUE.
*
   20 CONTINUE
      RETURN
*
*     End of LSAMEN
*
      END
```

———-

90.57.4 lisp code

— **lsamen.lisp** —

```
(defun lsamen (n ca cb)
  (declare (type (string *) cb ca)
           (type (f2cl-lib:integer4) n))
  (f2cl-lib:with-multi-array-data
      ((ca character ca-%data% ca-%offset%)
       (cb character cb-%data% cb-%offset%))
    (prog ((i 0) (lsamen nil))
      (declare (type f2cl-lib:logical lsamen)
               (type (f2cl-lib:integer4) i))
      (setf lsamen f2cl-lib:%false%)
      (if (or (< (f2cl-lib:len ca) n) (< (f2cl-lib:len cb) n))
          (go label20))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (if (not
               (lsame (f2cl-lib:fref-string ca (i i))
                (f2cl-lib:fref-string cb (i i))))
              (go label20))
         label10))
      (setf lsamen f2cl-lib:%true%)
     label20
      (go end_label)
     end_label
      (return (values lsamen nil nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::lsamen
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
```

```
                                        '((fortran-to-lisp::integer4)
                                          (string) (string))
                                        :return-values '(nil nil nil)
                                        :calls 'nil)))
```

———-

90.58 second

90.58.1 Axiom unit tests

— **second.input** —

```
)set break resume
)sys rm -f second.output
)spool second.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.58.2 Axiom help page

— **second.help** —

```
=====================================================================
second examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

real function second()

Purpose:
   SECOND returns the user time for a process in seconds.
   This version gets the time from the EXTERNAL system function ETIME
```

———-

90.58.3 fortran code

— second.f —

———-

90.58.4 lisp code

— second.lisp —

———-

90.59 sisnan

90.59.1 Axiom unit tests

— sisnan.input —

```
)set break resume
)sys rm -f sisnan.output
)spool sisnan.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.59.2 Axiom help page

— sisnan.help —

```
====================================================================
sisnan examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       logical function sisnan (SIN)
           SISNAN tests input for NaN.

Function/Subroutine Documentation
   logical function sisnan (real SIN)
       SISNAN tests input for NaN.

Purpose:
   SISNAN returns .TRUE. if its argument is NaN, and .FALSE.
   otherwise.  To be replaced by the Fortran 2003 intrinsic in the future.

Parameters:

 SIN
   SIN is REAL
   Input to test for NaN.
```

90.59.3 fortran code

— sisnan.f —

```
*  ====================================================================
      LOGICAL FUNCTION SISNAN( SIN )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      REAL               SIN
*     ..
*
*  ====================================================================
*
```

```
*  .. External Functions ..
      LOGICAL SLAISNAN
      EXTERNAL SLAISNAN
*  ..
*  .. Executable Statements ..
      SISNAN = SLAISNAN(SIN,SIN)
      RETURN
      END
```

———-

90.59.4 lisp code

— **sisnan.lisp** —

```
(defun sisnan (sin)
  (declare (type (single-float) sin))
  (prog ((sisnan nil))
    (declare (type f2cl-lib:logical sisnan))
    (setf sisnan (slaisnan sin sin))
    (go end_label)
   end_label
    (return (values sisnan nil))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sisnan
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types '((single-float))
                                            :return-values '(nil) :calls 'nil)))
```

———-

90.60 slabad

90.60.1 Axiom unit tests

— **slabad.input** —

```
)set break resume
)sys rm -f slabad.output
```

```
)spool slabad.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

90.60.2 Axiom help page

— slabad.help —

```
=====================================================================
slabad examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slabad (SMALL, LARGE)
           SLABAD

Function/Subroutine Documentation
   subroutine slabad (real SMALL, real LARGE)
       SLABAD

Purpose:
   SLABAD takes as input the values computed by SLAMCH for underflow and
   overflow, and returns the square root of each of these values if the
   log of LARGE is sufficiently large.  This subroutine is intended to
   identify machines with a large exponent range, such as the Crays, and
   redefine the underflow and overflow limits to be the square roots of
   the values computed by SLAMCH.  This subroutine is needed because
   SLAMCH does not compensate for poor arithmetic in the upper half of
   the exponent range, as is found on a Cray.

Parameters:

 SMALL
   SMALL is REAL
   On entry, the underflow threshold as computed by SLAMCH.
   On exit, if LOG10(LARGE) is sufficiently large, the square
   root of SMALL, otherwise unchanged.
```

```
 LARGE
   LARGE is REAL
   On entry, the overflow threshold as computed by SLAMCH.
   On exit, if LOG10(LARGE) is sufficiently large, the square
   root of LARGE, otherwise unchanged.
```

———-

90.60.3 fortran code

— **slabad.f** —

```
*  =====================================================================
      SUBROUTINE SLABAD( SMALL, LARGE )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      REAL               LARGE, SMALL
*     ..
*
*  =====================================================================
*
*     .. Intrinsic Functions ..
      INTRINSIC          LOG10, SQRT
*     ..
*     .. Executable Statements ..
*
*     If it looks like we're on a Cray, take the square root of
*     SMALL and LARGE to avoid overflow and underflow problems.
*
      IF( LOG10( LARGE ).GT.2000. ) THEN
         SMALL = SQRT( SMALL )
         LARGE = SQRT( LARGE )
      END IF
*
      RETURN
*
*     End of SLABAD
*
      END
```

———-

90.60.4 lisp code

— slabad.lisp —

```
(defun slabad (small large)
  (declare (type (single-float) large small))
  (prog ()
    (declare)
    (cond
     ((> (f2cl-lib:log10 large) 2000.0) (setf small (f2cl-lib:fsqrt small))
      (setf large (f2cl-lib:fsqrt large))))
    (go end_label)
   end_label
    (return (values small large))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slabad
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((single-float) (single-float))
                                            :return-values
                                            '(fortran-to-lisp::small
                                              fortran-to-lisp::large)
                                            :calls 'nil)))
```

———-

90.61 slacpy

90.61.1 Axiom unit tests

— slacpy.input —

```
)set break resume
)sys rm -f slacpy.output
)spool slacpy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.61.2 Axiom help page

— slacpy.help —

```
=====================================================================
slacpy examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slacpy (UPLO, M, N, A, LDA, B, LDB)
           SLACPY copies all or part of one two-dimensional array to another.

Function/Subroutine Documentation
   subroutine slacpy (character UPLO, integer M, integer N, real, dimension(
       lda, * ) A, integer LDA, real, dimension( ldb, * ) B, integer LDB)
       SLACPY copies all or part of one two-dimensional array to another.

Purpose:
   SLACPY copies all or part of a two-dimensional matrix A to another
   matrix B.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies the part of the matrix A to be copied to B.
   = 'U':      Upper triangular part
   = 'L':      Lower triangular part
   Otherwise:  All of the matrix A

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 A
   A is REAL array, dimension (LDA,N)
   The m by n matrix A.  If UPLO = 'U', only the upper triangle
```

```
  or trapezoid is accessed; if UPLO = 'L', only the lower
  triangle or trapezoid is accessed.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

B
  B is REAL array, dimension (LDB,N)
  On exit, B = A in the locations specified by UPLO.

LDB
  LDB is INTEGER
  The leading dimension of the array B.  LDB >= max(1,M).
```

———-

90.61.3 fortran code

— **slacpy.f** —

```
*  =====================================================================
      SUBROUTINE SLACPY( UPLO, M, N, A, LDA, B, LDB )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            LDA, LDB, M, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          MIN
*     ..
*     .. Executable Statements ..
*
      IF( LSAME( UPLO, 'U' ) ) THEN
         DO 20 J = 1, N
            DO 10 I = 1, MIN( J, M )
               B( I, J ) = A( I, J )
   10       CONTINUE
   20    CONTINUE
      ELSE IF( LSAME( UPLO, 'L' ) ) THEN
         DO 40 J = 1, N
            DO 30 I = J, M
               B( I, J ) = A( I, J )
   30       CONTINUE
   40    CONTINUE
      ELSE
         DO 60 J = 1, N
            DO 50 I = 1, M
               B( I, J ) = A( I, J )
   50       CONTINUE
   60    CONTINUE
      END IF
      RETURN
*
*     End of SLACPY
*
      END
```

———-

90.61.4 lisp code

— **slacpy.lisp** —

```
(defun slacpy (uplo m n a lda b ldb$)
  (declare (type (array single-float (*)) b a)
           (type (f2cl-lib:integer4) ldb$ lda n m)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a single-float a-%data% a-%offset%)
       (b single-float b-%data% b-%offset%))
    (prog ((i 0) (j 0))
      (declare (type (f2cl-lib:integer4) j i))
      (cond
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
```

```
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the f2cl-lib:integer4 j)
                               (the f2cl-lib:integer4 m)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%))
           label10))
       label20)))
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame uplo "L")
      (declare (ignore var-1))
      (when var-0 (setf uplo var-0))
      ret-val)
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%))
           label30))
       label40)))
  (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                 b-%offset%)
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%))
           label50))
       label60))))
 (go end_label)
end_label
```

```
      (return (values uplo nil nil nil nil nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slacpy
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil nil nil
                                              nil nil nil)
                                            :calls 'nil)))
```

———-

90.62 sladiv

90.62.1 Axiom unit tests

— sladiv.input —

```
)set break resume
)sys rm -f sladiv.output
)spool sladiv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.62.2 Axiom help page

— sladiv.help —

```
====================================================================
sladiv examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine sladiv (A, B, C, D, P, Q)
           SLADIV performs complex division in real arithmetic, avoiding
           unnecessary overflow.
       subroutine sladiv1 (A, B, C, D, P, Q)
       real function sladiv2 (A, B, C, D, R, T)

Function/Subroutine Documentation
   subroutine sladiv (real A, real B, real C, real D, real P, real Q)
       SLADIV performs complex division in real arithmetic, avoiding
       unnecessary overflow.

Purpose:
   SLADIV performs complex division in  real arithmetic

                         a + i*b
             p + i*q = ---------
                         c + i*d

   The algorithm is due to Michael Baudin and Robert L. Smith
   and can be found in the paper
   "A Robust Complex Division in Scilab"

Parameters:

 A
   A is REAL

 B
   B is REAL

 C
   C is REAL

 D
   D is REAL
   The scalars a, b, c, and d in the above expression.

 P
   P is REAL
```

```
Q
  Q is REAL
  The scalars p and q in the above expression.
```

90.62.3 fortran code

— sladiv.f —

```
*  =====================================================================
      SUBROUTINE SLADIV( A, B, C, D, P, Q )
*
*  -- LAPACK auxiliary routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     January 2013
*
*     .. Scalar Arguments ..
      REAL               A, B, C, D, P, Q
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               BS
      PARAMETER          ( BS = 2.0E0 )
      REAL               HALF
      PARAMETER          ( HALF = 0.5E0 )
      REAL               TWO
      PARAMETER          ( TWO = 2.0E0 )
*
*     .. Local Scalars ..
      REAL               AA, BB, CC, DD, AB, CD, S, OV, UN, BE, EPS
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLADIV1
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
```

```
      AA = A
      BB = B
      CC = C
      DD = D
      AB = MAX( ABS(A), ABS(B) )
      CD = MAX( ABS(C), ABS(D) )
      S = 1.0E0

      OV = SLAMCH( 'Overflow threshold' )
      UN = SLAMCH( 'Safe minimum' )
      EPS = SLAMCH( 'Epsilon' )
      BE = BS / (EPS*EPS)

      IF( AB >= HALF*OV ) THEN
         AA = HALF * AA
         BB = HALF * BB
         S  = TWO * S
      END IF
      IF( CD >= HALF*OV ) THEN
         CC = HALF * CC
         DD = HALF * DD
         S  = HALF * S
      END IF
      IF( AB <= UN*BS/EPS ) THEN
         AA = AA * BE
         BB = BB * BE
         S  = S / BE
      END IF
      IF( CD <= UN*BS/EPS ) THEN
         CC = CC * BE
         DD = DD * BE
         S  = S * BE
      END IF
      IF( ABS( D ).LE.ABS( C ) ) THEN
         CALL SLADIV1(AA, BB, CC, DD, P, Q)
      ELSE
         CALL SLADIV1(BB, AA, DD, CC, P, Q)
         Q = -Q
      END IF
      P = P * S
      Q = Q * S
*
      RETURN
*
*     End of SLADIV
*
      END
```

```
      SUBROUTINE SLADIV1( A, B, C, D, P, Q )
*
*  -- LAPACK auxiliary routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     January 2013
*
*     .. Scalar Arguments ..
      REAL               A, B, C, D, P, Q
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E0 )
*
*     .. Local Scalars ..
      REAL               R, T
*     ..
*     .. External Functions ..
      REAL               SLADIV2
      EXTERNAL           SLADIV2
*     ..
*     .. Executable Statements ..
*
      R = D / C
      T = ONE / (C + D * R)
      P = SLADIV2(A, B, C, D, R, T)
      A = -A
      Q = SLADIV2(B, A, C, D, R, T)
*
      RETURN
*
*     End of SLADIV1
*
      END

      REAL FUNCTION SLADIV2( A, B, C, D, R, T )
*
*  -- LAPACK auxiliary routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     January 2013
*
*     .. Scalar Arguments ..
      REAL               A, B, C, D, R, T
*     ..
*
*  =====================================================================
```

```
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E0 )
*
*     .. Local Scalars ..
      REAL               BR
*     ..
*     .. Executable Statements ..
*
      IF( R.NE.ZERO ) THEN
         BR = B * R
         if( BR.NE.ZERO ) THEN
            SLADIV2 = (A + BR) * T
         ELSE
            SLADIV2 = A * T + (B * T) * R
         END IF
      ELSE
         SLADIV2 = (A + D * (B / C)) * T
      END IF
*
      RETURN
*
*     End of SLADIV
*
      END
```

———-

90.62.4 lisp code

— sladiv.lisp —

```
(let* ((bs 2.0) (half 0.5) (two 2.0))
  (declare (type (single-float 2.0 2.0) bs)
           (type (single-float 0.5 0.5) half)
           (type (single-float 2.0 2.0) two)
           (ignorable bs half two))
  (defun sladiv (a b c d p q)
    (declare (type (single-float) q p d c b a))
    (prog ((aa 0.0) (bb 0.0) (cc 0.0) (dd 0.0) (ab 0.0) (cd 0.0) (s 0.0)
           (ov 0.0) (un 0.0) (be 0.0) (eps 0.0))
      (declare (type (single-float) aa bb cc dd ab cd s ov un be eps))
      (setf aa a)
      (setf bb b)
      (setf cc c)
      (setf dd d)
      (setf ab (max (abs a) (abs b)))
```

```
      (setf cd (max (abs c) (abs d)))
      (setf s 1.0)
      (setf ov (slamch "Overflow threshold"))
      (setf un (slamch "Safe minimum"))
      (setf eps (slamch "Epsilon"))
      (setf be (/ bs (* eps eps)))
      (cond
       ((>= ab (* half ov)) (setf aa (* half aa)) (setf bb (* half bb))
        (setf s (* two s))))
      (cond
       ((>= cd (* half ov)) (setf cc (* half cc)) (setf dd (* half dd))
        (setf s (* half s))))
      (cond
       ((<= ab (f2cl-lib:f2cl/ (* un bs) eps)) (setf aa (* aa be))
        (setf bb (* bb be)) (setf s (/ s be))))
      (cond
       ((<= cd (f2cl-lib:f2cl/ (* un bs) eps)) (setf cc (* cc be))
        (setf dd (* dd be)) (setf s (* s be))))
      (cond
       ((<= (abs d) (abs c))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (sladiv1 aa bb cc dd p q)
          (declare (ignore))
          (when var-0 (setf aa var-0))
          (when var-1 (setf bb var-1))
          (when var-2 (setf cc var-2))
          (when var-3 (setf dd var-3))
          (when var-4 (setf p var-4))
          (when var-5 (setf q var-5))))
       (t
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (sladiv1 bb aa dd cc p q)
          (declare (ignore))
          (when var-0 (setf bb var-0))
          (when var-1 (setf aa var-1))
          (when var-2 (setf dd var-2))
          (when var-3 (setf cc var-3))
          (when var-4 (setf p var-4))
          (when var-5 (setf q var-5)))
        (setf q (- q))))
      (setf p (* p s))
      (setf q (* q s))
      (go end_label)
     end_label
      (return (values nil nil nil nil p q)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sladiv
```

```
                fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((single-float) (single-float)
                                             (single-float) (single-float)
                                             (single-float) (single-float))
                                           :return-values
                                           '(nil nil nil nil
                                             fortran-to-lisp::p
                                             fortran-to-lisp::q)
                                           :calls 'nil)))


(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun sladiv1 (a b c d p q)
    (declare (type (single-float) q p d c b a))
    (prog ((r 0.0) (t$ 0.0))
      (declare (type (single-float) r t$))
      (setf r (/ d c))
      (setf t$ (/ one (+ c (* d r))))
      (setf p
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                  (sladiv2 a b c d r t$)
                (declare (ignore))
                (when var-0 (setf a var-0))
                (when var-1 (setf b var-1))
                (when var-2 (setf c var-2))
                (when var-3 (setf d var-3))
                (when var-4 (setf r var-4))
                (when var-5 (setf t$ var-5))
                ret-val))
      (setf a (- a))
      (setf q
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                  (sladiv2 b a c d r t$)
                (declare (ignore))
                (when var-0 (setf b var-0))
                (when var-1 (setf a var-1))
                (when var-2 (setf c var-2))
                (when var-3 (setf d var-3))
                (when var-4 (setf r var-4))
                (when var-5 (setf t$ var-5))
                ret-val))
      (go end_label)
     end_label
      (return (values a b c d p q)))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sladiv1
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((single-float) (single-float)
                                              (single-float) (single-float)
                                              (single-float) (single-float))
                                            :return-values
                                            '(fortran-to-lisp::a
                                              fortran-to-lisp::b
                                              fortran-to-lisp::c
                                              fortran-to-lisp::d
                                              fortran-to-lisp::p
                                              fortran-to-lisp::q)
                                            :calls 'nil)))


(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun sladiv2 (a b c d r t$)
    (declare (type (single-float) t$ r d c b a))
    (prog ((br 0.0) (sladiv2 0.0))
      (declare (type (single-float) br sladiv2))
      (cond
       ((/= r zero) (setf br (* b r))
        (cond ((/= br zero) (setf sladiv2 (* (+ a br) t$)))
              (t (setf sladiv2 (+ (* a t$) (* b t$ r))))))
       (t (setf sladiv2 (* (+ a (* d (/ b c))) t$))))
      (go end_label)
     end_label
      (return (values sladiv2 a b c d r t$)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::sladiv2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((single-float) (single-float)
                                              (single-float) (single-float)
                                              (single-float) (single-float))
                                            :return-values
                                            '(fortran-to-lisp::a
                                              fortran-to-lisp::b
                                              fortran-to-lisp::c
                                              fortran-to-lisp::d
                                              fortran-to-lisp::r
```

```
                                        fortran-to-lisp::t$)
                                      :calls 'nil)))
```

———-

90.63 slae2

90.63.1 Axiom unit tests

— **slae2.input** —

```
)set break resume
)sys rm -f slae2.output
)spool slae2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.63.2 Axiom help page

— **slae2.help** —

```
======================================================================
slae2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slae2 (A, B, C, RT1, RT2)
           SLAE2 computes the eigenvalues of a 2-by-2 symmetric matrix.

Function/Subroutine Documentation
   subroutine slae2 (real A, real B, real C, real RT1, real RT2)
```

```
      SLAE2 computes the eigenvalues of a 2-by-2 symmetric matrix.

Purpose:
   SLAE2  computes the eigenvalues of a 2-by-2 symmetric matrix
               [  A   B  ]
               [  B   C  ].
   On return, RT1 is the eigenvalue of larger absolute value, and RT2
   is the eigenvalue of smaller absolute value.

Parameters:

 A
   A is REAL
   The (1,1) element of the 2-by-2 matrix.

 B
   B is REAL
   The (1,2) and (2,1) elements of the 2-by-2 matrix.

 C
   C is REAL
   The (2,2) element of the 2-by-2 matrix.

 RT1
   RT1 is REAL
   The eigenvalue of larger absolute value.

 RT2
   RT2 is REAL
   The eigenvalue of smaller absolute value.

Further Details:

   RT1 is accurate to a few ulps barring over/underflow.

   RT2 may be inaccurate if there is massive cancellation in the
   determinant A*C-B*B; higher precision or correctly rounded or
   correctly truncated arithmetic would be needed to compute RT2
   accurately in all cases.

   Overflow is possible only if RT1 is within a factor of 5 of overflow.
   Underflow is harmless if the input data is 0 or exceeds
   underflow_threshold / macheps.
```

90.63.3 fortran code

— slae2.f —

```
*  =====================================================================
      SUBROUTINE SLAE2( A, B, C, RT1, RT2 )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      REAL               A, B, C, RT1, RT2
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E0 )
      REAL               TWO
      PARAMETER          ( TWO = 2.0E0 )
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E0 )
      REAL               HALF
      PARAMETER          ( HALF = 0.5E0 )
*     ..
*     .. Local Scalars ..
      REAL               AB, ACMN, ACMX, ADF, DF, RT, SM, TB
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SQRT
*     ..
*     .. Executable Statements ..
*
*     Compute the eigenvalues
*
      SM = A + C
      DF = A - C
      ADF = ABS( DF )
      TB = B + B
      AB = ABS( TB )
      IF( ABS( A ).GT.ABS( C ) ) THEN
         ACMX = A
         ACMN = C
      ELSE
         ACMX = C
         ACMN = A
```

```
      END IF
      IF( ADF.GT.AB ) THEN
         RT = ADF*SQRT( ONE+( AB / ADF )**2 )
      ELSE IF( ADF.LT.AB ) THEN
         RT = AB*SQRT( ONE+( ADF / AB )**2 )
      ELSE
*
*        Includes case AB=ADF=0
*
         RT = AB*SQRT( TWO )
      END IF
      IF( SM.LT.ZERO ) THEN
         RT1 = HALF*( SM-RT )
*
*        Order of execution important.
*        To get fully accurate smaller eigenvalue,
*        next line needs to be executed in higher precision.
*
         RT2 = ( ACMX / RT1 )*ACMN - ( B / RT1 )*B
      ELSE IF( SM.GT.ZERO ) THEN
         RT1 = HALF*( SM+RT )
*
*        Order of execution important.
*        To get fully accurate smaller eigenvalue,
*        next line needs to be executed in higher precision.
*
         RT2 = ( ACMX / RT1 )*ACMN - ( B / RT1 )*B
      ELSE
*
*        Includes case RT1 = RT2 = 0
*
         RT1 = HALF*RT
         RT2 = -HALF*RT
      END IF
      RETURN
*
*     End of SLAE2
*
      END
```

———-

90.63.4 lisp code

— slae2.lisp —

```
(let* ((one 1.0) (two 2.0) (zero 0.0) (half 0.5))
  (declare (type (single-float 1.0 1.0) one)
```

```
          (type (single-float 2.0 2.0) two)
          (type (single-float 0.0 0.0) zero)
          (type (single-float 0.5 0.5) half)
          (ignorable one two zero half))
  (defun slae2 (a b c rt1 rt2)
    (declare (type (single-float) rt2 rt1 c b a))
    (prog ((ab 0.0) (acmn 0.0) (acmx 0.0) (adf 0.0) (df 0.0) (rt 0.0) (sm 0.0)
           (tb 0.0))
      (declare (type (single-float) ab acmn acmx adf df rt sm tb))
      (setf sm (+ a c))
      (setf df (- a c))
      (setf adf (abs df))
      (setf tb (+ b b))
      (setf ab (abs tb))
      (cond ((> (abs a) (abs c)) (setf acmx a) (setf acmn c))
            (t (setf acmx c) (setf acmn a)))
      (cond
       ((> adf ab)
        (setf rt (* adf (f2cl-lib:fsqrt (+ one (expt (/ ab adf) 2))))))
       ((< adf ab)
        (setf rt (* ab (f2cl-lib:fsqrt (+ one (expt (/ adf ab) 2))))))
       (t (setf rt (* ab (f2cl-lib:fsqrt two)))))
      (cond
       ((< sm zero) (setf rt1 (* half (- sm rt)))
        (setf rt2 (- (* (/ acmx rt1) acmn) (* (/ b rt1) b))))
       ((> sm zero) (setf rt1 (* half (+ sm rt)))
        (setf rt2 (- (* (/ acmx rt1) acmn) (* (/ b rt1) b))))
       (t (setf rt1 (* half rt)) (setf rt2 (* (- half) rt))))
      (go end_label)
     end_label
      (return (values nil nil nil rt1 rt2)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slae2 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((single-float) (single-float)
                                              (single-float) (single-float)
                                              (single-float))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::rt1
                                              fortran-to-lisp::rt2)
                                            :calls 'nil)))
```

90.64 slaebz

90.64.1 Axiom unit tests

— slaebz.input —

```
)set break resume
)sys rm -f slaebz.output
)spool slaebz.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.64.2 Axiom help page

— slaebz.help —

```
======================================================================
slaebz examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slaebz (IJOB, NITMAX, N, MMAX, MINP, NBMIN, ABSTOL, RELTOL,
           PIVMIN, D, E, E2, NVAL, AB, C, MOUT, NAB, WORK, IWORK, INFO)
           SLAEBZ computes the number of eigenvalues of a real symmetric
           tridiagonal matrix which are less than or equal to a given value,
           and performs other tasks required by the routine sstebz.

Function/Subroutine Documentation
   subroutine slaebz (integer IJOB, integer NITMAX, integer N, integer MMAX,
       integer MINP, integer NBMIN, real ABSTOL, real RELTOL, real PIVMIN,
       real, dimension( * ) D, real, dimension( * ) E, real, dimension( * )
       E2, integer, dimension( * ) NVAL, real, dimension( mmax, * ) AB, real,
       dimension( * ) C, integer MOUT, integer, dimension( mmax, * ) NAB,
       real, dimension( * ) WORK, integer, dimension( * ) IWORK, integer INFO)
       SLAEBZ computes the number of eigenvalues of a real symmetric
```

```
      tridiagonal matrix which are less than or equal to a given value, and
      performs other tasks required by the routine sstebz.

Purpose:
   SLAEBZ contains the iteration loops which compute and use the
   function N(w), which is the count of eigenvalues of a symmetric
   tridiagonal matrix T less than or equal to its argument  w.  It
   performs a choice of two types of loops:

   IJOB=1, followed by
   IJOB=2: It takes as input a list of intervals and returns a list of
      sufficiently small intervals whose union contains the same
      eigenvalues as the union of the original intervals.
      The input intervals are (AB(j,1),AB(j,2)], j=1,...,MINP.
      The output interval (AB(j,1),AB(j,2)] will contain
      eigenvalues NAB(j,1)+1,...,NAB(j,2), where 1 <= j <= MOUT.

   IJOB=3: It performs a binary search in each input interval
      (AB(j,1),AB(j,2)] for a point  w(j)  such that
      N(w(j))=NVAL(j), and uses  C(j)  as the starting point of
      the search.  If such a w(j) is found, then on output
      AB(j,1)=AB(j,2)=w.  If no such w(j) is found, then on output
      (AB(j,1),AB(j,2)] will be a small interval containing the
      point where N(w) jumps through NVAL(j), unless that point
      lies outside the initial interval.

   Note that the intervals are in all cases half-open intervals,
   i.e., of the form  (a,b] , which includes  b  but not  a .

   To avoid underflow, the matrix should be scaled so that its largest
   element is no greater than  overflow**(1/2) * underflow**(1/4)
   in absolute value.  To assure the most accurate computation
   of small eigenvalues, the matrix should be scaled to be
   not much smaller than that, either.

   See W. Kahan "Accurate Eigenvalues of a Symmetric Tridiagonal
   Matrix", Report CS41, Computer Science Dept., Stanford
   University, July 21, 1966

   Note: the arguments are, in general, *not* checked for unreasonable values.

Parameters:

 IJOB
   IJOB is INTEGER
   Specifies what is to be done:
   = 1:  Compute NAB for the initial intervals.
   = 2:  Perform bisection iteration to find eigenvalues of T.
   = 3:  Perform bisection iteration to invert N(w), i.e.,
         to find a point which has a specified number of
```

```
        eigenvalues of T to its left.
        Other values will cause SLAEBZ to return with INFO=-1.

NITMAX
  NITMAX is INTEGER
  The maximum number of "levels" of bisection to be
  performed, i.e., an interval of width W will not be made
  smaller than 2^(-NITMAX) * W.  If not all intervals
  have converged after NITMAX iterations, then INFO is set
  to the number of non-converged intervals.

N
  N is INTEGER
  The dimension n of the tridiagonal matrix T.  It must be at least 1.

MMAX
  MMAX is INTEGER
  The maximum number of intervals.  If more than MMAX intervals
  are generated, then SLAEBZ will quit with INFO=MMAX+1.

MINP
  MINP is INTEGER
  The initial number of intervals.  It may not be greater than MMAX.

NBMIN
  NBMIN is INTEGER
  The smallest number of intervals that should be processed
  using a vector loop.  If zero, then only the scalar loop will be used.

ABSTOL
  ABSTOL is REAL
  The minimum (absolute) width of an interval.  When an
  interval is narrower than ABSTOL, or than RELTOL times the
  larger (in magnitude) endpoint, then it is considered to be
  sufficiently small, i.e., converged.  This must be at least zero.

RELTOL
  RELTOL is REAL
  The minimum relative width of an interval.  When an interval
  is narrower than ABSTOL, or than RELTOL times the larger (in
  magnitude) endpoint, then it is considered to be
  sufficiently small, i.e., converged.  Note: this should
  always be at least radix*machine epsilon.

PIVMIN
  PIVMIN is REAL
  The minimum absolute value of a "pivot" in the Sturm sequence loop.
  This must be at least  max |e(j)**2|*safe_min  and at
  least safe_min, where safe_min is at least
  the smallest number that can divide one without overflow.
```

```
D
  D is REAL array, dimension (N)
  The diagonal elements of the tridiagonal matrix T.

E
  E is REAL array, dimension (N)
  The offdiagonal elements of the tridiagonal matrix T in
  positions 1 through N-1.  E(N) is arbitrary.

E2
  E2 is REAL array, dimension (N)
  The squares of the offdiagonal elements of the tridiagonal
  matrix T.  E2(N) is ignored.

NVAL
  NVAL is INTEGER array, dimension (MINP)
  If IJOB=1 or 2, not referenced.
  If IJOB=3, the desired values of N(w).  The elements of NVAL
  will be reordered to correspond with the intervals in AB.
  Thus, NVAL(j) on output will not, in general be the same as
  NVAL(j) on input, but it will correspond with the interval
  (AB(j,1),AB(j,2)] on output.

AB
  AB is REAL array, dimension (MMAX,2)
  The endpoints of the intervals.  AB(j,1) is  a(j), the left
  endpoint of the j-th interval, and AB(j,2) is b(j), the
  right endpoint of the j-th interval.  The input intervals
  will, in general, be modified, split, and reordered by the calculation.

C
  C is REAL array, dimension (MMAX)
  If IJOB=1, ignored.
  If IJOB=2, workspace.
  If IJOB=3, then on input C(j) should be initialized to the
             first search point in the binary search.

MOUT
  MOUT is INTEGER
  If IJOB=1, the number of eigenvalues in the intervals.
  If IJOB=2 or 3, the number of intervals output.
  If IJOB=3, MOUT will equal MINP.

NAB
  NAB is INTEGER array, dimension (MMAX,2)
  If IJOB=1, then on output NAB(i,j) will be set to N(AB(i,j)).
  If IJOB=2, then on input, NAB(i,j) should be set.  It must
     satisfy the condition:
     N(AB(i,1)) <= NAB(i,1) <= NAB(i,2) <= N(AB(i,2)),
```

```
      which means that in interval i only eigenvalues
      NAB(i,1)+1,...,NAB(i,2) will be considered.  Usually,
      NAB(i,j)=N(AB(i,j)), from a previous call to SLAEBZ with IJOB=1.
      On output, NAB(i,j) will contain
      max(na(k),min(nb(k),N(AB(i,j)))), where k is the index of
      the input interval that the output interval
      (AB(j,1),AB(j,2)] came from, and na(k) and nb(k) are the
      the input values of NAB(k,1) and NAB(k,2).
   If IJOB=3, then on output, NAB(i,j) contains N(AB(i,j)),
      unless N(w) > NVAL(i) for all search points  w , in which
      case NAB(i,1) will not be modified, i.e., the output
      value will be the same as the input value (modulo
      reorderings -- see NVAL and AB), or unless N(w) < NVAL(i)
      for all search points  w , in which case NAB(i,2) will
      not be modified.  Normally, NAB should be set to some
      distinctive value(s) before SLAEBZ is called.

 WORK
   WORK is REAL array, dimension (MMAX)
   Workspace.

 IWORK
   IWORK is INTEGER array, dimension (MMAX)
   Workspace.

 INFO
   INFO is INTEGER
   = 0:       All intervals converged.
   = 1--MMAX: The last INFO intervals did not converge.
   = MMAX+1:  More than MMAX intervals were generated.

Further Details:

   This routine is intended to be called only by other LAPACK
   routines, thus the interface is less user-friendly.  It is intended
   for two purposes:

   (a) finding eigenvalues.  In this case, SLAEBZ should have one or
       more initial intervals set up in AB, and SLAEBZ should be called
       with IJOB=1.  This sets up NAB, and also counts the eigenvalues.
       Intervals with no eigenvalues would usually be thrown out at
       this point.  Also, if not all the eigenvalues in an interval i
       are desired, NAB(i,1) can be increased or NAB(i,2) decreased.
       For example, set NAB(i,1)=NAB(i,2)-1 to get the largest
       eigenvalue.  SLAEBZ is then called with IJOB=2 and MMAX
       no smaller than the value of MOUT returned by the call with
       IJOB=1.  After this (IJOB=2) call, eigenvalues NAB(i,1)+1
       through NAB(i,2) are approximately AB(i,1) (or AB(i,2)) to the
       tolerance specified by ABSTOL and RELTOL.
```

```
   (b) finding an interval (a',b'] containing eigenvalues w(f),...,w(l).
       In this case, start with a Gershgorin interval  (a,b).  Set up
       AB to contain 2 search intervals, both initially (a,b).  One
       NVAL element should contain  f-1  and the other should contain  l
       , while C should contain a and b, resp.  NAB(i,1) should be -1
       and NAB(i,2) should be N+1, to flag an error if the desired
       interval does not lie in (a,b).  SLAEBZ is then called with
       IJOB=3.  On exit, if w(f-1) < w(f), then one of the intervals --
       j -- will have AB(j,1)=AB(j,2) and NAB(j,1)=NAB(j,2)=f-1, while
       if, to the specified tolerance, w(f-k)=...=w(f+r), k > 0 and r
       >= 0, then the interval will have  N(AB(j,1))=NAB(j,1)=f-k and
       N(AB(j,2))=NAB(j,2)=f+r.  The cases w(l) < w(l+1) and
       w(l-r)=...=w(l+k) are handled similarly.
```

———-

90.64.3 fortran code

— **slaebz.f** —

```
*  =====================================================================
      SUBROUTINE SLAEBZ( IJOB, NITMAX, N, MMAX, MINP, NBMIN, ABSTOL,
     $                   RELTOL, PIVMIN, D, E, E2, NVAL, AB, C, MOUT,
     $                   NAB, WORK, IWORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            IJOB, INFO, MINP, MMAX, MOUT, N, NBMIN, NITMAX
      REAL               ABSTOL, PIVMIN, RELTOL
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * ), NAB( MMAX, * ), NVAL( * )
      REAL               AB( MMAX, * ), C( * ), D( * ), E( * ), E2( * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, TWO, HALF
      PARAMETER          ( ZERO = 0.0E0, TWO = 2.0E0,
     $                   HALF = 1.0E0 / TWO )
*     ..
```

```
*     .. Local Scalars ..
      INTEGER            ITMP1, ITMP2, J, JI, JIT, JP, KF, KFNEW, KL,
     $                   KLNEW
      REAL               TMP1, TMP2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Check for Errors
*
      INFO = 0
      IF( IJOB.LT.1 .OR. IJOB.GT.3 ) THEN
         INFO = -1
         RETURN
      END IF
*
*     Initialize NAB
*
      IF( IJOB.EQ.1 ) THEN
*
*        Compute the number of eigenvalues in the initial intervals.
*
         MOUT = 0
         DO 30 JI = 1, MINP
            DO 20 JP = 1, 2
               TMP1 = D( 1 ) - AB( JI, JP )
               IF( ABS( TMP1 ).LT.PIVMIN )
     $            TMP1 = -PIVMIN
               NAB( JI, JP ) = 0
               IF( TMP1.LE.ZERO )
     $            NAB( JI, JP ) = 1
*
               DO 10 J = 2, N
                  TMP1 = D( J ) - E2( J-1 ) / TMP1 - AB( JI, JP )
                  IF( ABS( TMP1 ).LT.PIVMIN )
     $               TMP1 = -PIVMIN
                  IF( TMP1.LE.ZERO )
     $               NAB( JI, JP ) = NAB( JI, JP ) + 1
   10          CONTINUE
   20       CONTINUE
            MOUT = MOUT + NAB( JI, 2 ) - NAB( JI, 1 )
   30    CONTINUE
         RETURN
      END IF
*
*     Initialize for loop
*
*     KF and KL have the following meaning:
```

```
*        Intervals 1,...,KF-1 have converged.
*        Intervals KF,...,KL  still need to be refined.
*
      KF = 1
      KL = MINP
*
*     If IJOB=2, initialize C.
*     If IJOB=3, use the user-supplied starting point.
*
      IF( IJOB.EQ.2 ) THEN
         DO 40 JI = 1, MINP
            C( JI ) = HALF*( AB( JI, 1 )+AB( JI, 2 ) )
   40    CONTINUE
      END IF
*
*     Iteration loop
*
      DO 130 JIT = 1, NITMAX
*
*        Loop over intervals
*
         IF( KL-KF+1.GE.NBMIN .AND. NBMIN.GT.0 ) THEN
*
*           Begin of Parallel Version of the loop
*
            DO 60 JI = KF, KL
*
*              Compute N(c), the number of eigenvalues less than c
*
               WORK( JI ) = D( 1 ) - C( JI )
               IWORK( JI ) = 0
               IF( WORK( JI ).LE.PIVMIN ) THEN
                  IWORK( JI ) = 1
                  WORK( JI ) = MIN( WORK( JI ), -PIVMIN )
               END IF
*
               DO 50 J = 2, N
                  WORK( JI ) = D( J ) - E2( J-1 ) / WORK( JI ) - C( JI )
                  IF( WORK( JI ).LE.PIVMIN ) THEN
                     IWORK( JI ) = IWORK( JI ) + 1
                     WORK( JI ) = MIN( WORK( JI ), -PIVMIN )
                  END IF
   50          CONTINUE
   60       CONTINUE
*
            IF( IJOB.LE.2 ) THEN
*
*              IJOB=2: Choose all intervals containing eigenvalues.
*
               KLNEW = KL
```

```
               DO 70 JI = KF, KL
*
*                 Insure that N(w) is monotone
*
                  IWORK( JI ) = MIN( NAB( JI, 2 ),
     $                           MAX( NAB( JI, 1 ), IWORK( JI ) ) )
*
*                 Update the Queue -- add intervals if both halves
*                 contain eigenvalues.
*
                  IF( IWORK( JI ).EQ.NAB( JI, 2 ) ) THEN
*
*                    No eigenvalue in the upper interval:
*                    just use the lower interval.
*
                     AB( JI, 2 ) = C( JI )
*
                  ELSE IF( IWORK( JI ).EQ.NAB( JI, 1 ) ) THEN
*
*                    No eigenvalue in the lower interval:
*                    just use the upper interval.
*
                     AB( JI, 1 ) = C( JI )
                  ELSE
                     KLNEW = KLNEW + 1
                     IF( KLNEW.LE.MMAX ) THEN
*
*                       Eigenvalue in both intervals -- add upper to
*                       queue.
*
                        AB( KLNEW, 2 ) = AB( JI, 2 )
                        NAB( KLNEW, 2 ) = NAB( JI, 2 )
                        AB( KLNEW, 1 ) = C( JI )
                        NAB( KLNEW, 1 ) = IWORK( JI )
                        AB( JI, 2 ) = C( JI )
                        NAB( JI, 2 ) = IWORK( JI )
                     ELSE
                        INFO = MMAX + 1
                     END IF
                  END IF
   70         CONTINUE
              IF( INFO.NE.0 )
     $           RETURN
              KL = KLNEW
           ELSE
*
*             IJOB=3: Binary search.  Keep only the interval containing
*                     w   s.t. N(w) = NVAL
*
              DO 80 JI = KF, KL
```

```
                 IF( IWORK( JI ).LE.NVAL( JI ) ) THEN
                    AB( JI, 1 ) = C( JI )
                    NAB( JI, 1 ) = IWORK( JI )
                 END IF
                 IF( IWORK( JI ).GE.NVAL( JI ) ) THEN
                    AB( JI, 2 ) = C( JI )
                    NAB( JI, 2 ) = IWORK( JI )
                 END IF
   80         CONTINUE
           END IF
*
        ELSE
*
*          End of Parallel Version of the loop
*
*          Begin of Serial Version of the loop
*
           KLNEW = KL
           DO 100 JI = KF, KL
*
*             Compute N(w), the number of eigenvalues less than w
*
              TMP1 = C( JI )
              TMP2 = D( 1 ) - TMP1
              ITMP1 = 0
              IF( TMP2.LE.PIVMIN ) THEN
                 ITMP1 = 1
                 TMP2 = MIN( TMP2, -PIVMIN )
              END IF
*
              DO 90 J = 2, N
                 TMP2 = D( J ) - E2( J-1 ) / TMP2 - TMP1
                 IF( TMP2.LE.PIVMIN ) THEN
                    ITMP1 = ITMP1 + 1
                    TMP2 = MIN( TMP2, -PIVMIN )
                 END IF
   90         CONTINUE
*
              IF( IJOB.LE.2 ) THEN
*
*                IJOB=2: Choose all intervals containing eigenvalues.
*
*                Insure that N(w) is monotone
*
                 ITMP1 = MIN( NAB( JI, 2 ),
     $                   MAX( NAB( JI, 1 ), ITMP1 ) )
*
*                Update the Queue -- add intervals if both halves
*                contain eigenvalues.
*
```

```
                 IF( ITMP1.EQ.NAB( JI, 2 ) ) THEN
*
*                   No eigenvalue in the upper interval:
*                   just use the lower interval.
*
                    AB( JI, 2 ) = TMP1
*
                 ELSE IF( ITMP1.EQ.NAB( JI, 1 ) ) THEN
*
*                   No eigenvalue in the lower interval:
*                   just use the upper interval.
*
                    AB( JI, 1 ) = TMP1
                 ELSE IF( KLNEW.LT.MMAX ) THEN
*
*                   Eigenvalue in both intervals -- add upper to queue.
*
                    KLNEW = KLNEW + 1
                    AB( KLNEW, 2 ) = AB( JI, 2 )
                    NAB( KLNEW, 2 ) = NAB( JI, 2 )
                    AB( KLNEW, 1 ) = TMP1
                    NAB( KLNEW, 1 ) = ITMP1
                    AB( JI, 2 ) = TMP1
                    NAB( JI, 2 ) = ITMP1
                 ELSE
                    INFO = MMAX + 1
                    RETURN
                 END IF
              ELSE
*
*                 IJOB=3: Binary search.  Keep only the interval
*                         containing  w  s.t. N(w) = NVAL
*
                 IF( ITMP1.LE.NVAL( JI ) ) THEN
                    AB( JI, 1 ) = TMP1
                    NAB( JI, 1 ) = ITMP1
                 END IF
                 IF( ITMP1.GE.NVAL( JI ) ) THEN
                    AB( JI, 2 ) = TMP1
                    NAB( JI, 2 ) = ITMP1
                 END IF
              END IF
  100       CONTINUE
            KL = KLNEW
*
         END IF
*
*        Check for convergence
*
         KFNEW = KF
```

```
         DO 110 JI = KF, KL
            TMP1 = ABS( AB( JI, 2 )-AB( JI, 1 ) )
            TMP2 = MAX( ABS( AB( JI, 2 ) ), ABS( AB( JI, 1 ) ) )
            IF( TMP1.LT.MAX( ABSTOL, PIVMIN, RELTOL*TMP2 ) .OR.
     $          NAB( JI, 1 ).GE.NAB( JI, 2 ) ) THEN
*
*              Converged -- Swap with position KFNEW,
*                           then increment KFNEW
*
               IF( JI.GT.KFNEW ) THEN
                  TMP1 = AB( JI, 1 )
                  TMP2 = AB( JI, 2 )
                  ITMP1 = NAB( JI, 1 )
                  ITMP2 = NAB( JI, 2 )
                  AB( JI, 1 ) = AB( KFNEW, 1 )
                  AB( JI, 2 ) = AB( KFNEW, 2 )
                  NAB( JI, 1 ) = NAB( KFNEW, 1 )
                  NAB( JI, 2 ) = NAB( KFNEW, 2 )
                  AB( KFNEW, 1 ) = TMP1
                  AB( KFNEW, 2 ) = TMP2
                  NAB( KFNEW, 1 ) = ITMP1
                  NAB( KFNEW, 2 ) = ITMP2
                  IF( IJOB.EQ.3 ) THEN
                     ITMP1 = NVAL( JI )
                     NVAL( JI ) = NVAL( KFNEW )
                     NVAL( KFNEW ) = ITMP1
                  END IF
               END IF
               KFNEW = KFNEW + 1
            END IF
  110    CONTINUE
         KF = KFNEW
*
*        Choose Midpoints
*
         DO 120 JI = KF, KL
            C( JI ) = HALF*( AB( JI, 1 )+AB( JI, 2 ) )
  120    CONTINUE
*
*        If no more intervals to refine, quit.
*
         IF( KF.GT.KL )
     $      GO TO 140
  130 CONTINUE
*
*     Converged
*
  140 CONTINUE
      INFO = MAX( KL+1-KF, 0 )
      MOUT = KL
```

```
*
      RETURN
*
*     End of SLAEBZ
*
      END
```

———-

90.64.4 lisp code

— **slaebz.lisp** —

```
(let* ((zero 0.0) (two 2.0) (half (f2cl-lib:f2cl/ 1.0 two)))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 2.0 2.0) two)
           (type (single-float) half)
           (ignorable zero two half))
  (defun slaebz
         (ijob nitmax n mmax minp nbmin abstol reltol pivmin d e e2 nval ab c
          mout nab work iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork nab nval)
             (type (array single-float (*)) work c ab e2 e d)
             (type (single-float) pivmin reltol abstol)
             (type (f2cl-lib:integer4) info mout nbmin minp mmax n nitmax
              ijob))
    (f2cl-lib:with-multi-array-data
        ((d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%)
         (e2 single-float e2-%data% e2-%offset%)
         (ab single-float ab-%data% ab-%offset%)
         (c single-float c-%data% c-%offset%)
         (work single-float work-%data% work-%offset%)
         (nval f2cl-lib:integer4 nval-%data% nval-%offset%)
         (nab f2cl-lib:integer4 nab-%data% nab-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((tmp1 0.0) (tmp2 0.0) (itmp1 0) (itmp2 0) (j 0) (ji 0) (jit 0)
             (jp 0) (kf 0) (kfnew 0) (kl 0) (klnew 0))
        (declare (type (single-float) tmp1 tmp2)
                 (type (f2cl-lib:integer4) itmp1 itmp2 j ji jit jp kf kfnew kl
                  klnew))
        (setf info 0)
        (cond ((or (< ijob 1) (> ijob 3)) (setf info -1) (go end_label)))
        (cond
         ((= ijob 1) (setf mout 0)
          (f2cl-lib:fdo (ji 1 (f2cl-lib:int-add ji 1))
                        ((> ji minp) nil)
            (tagbody
```

```
(f2cl-lib:fdo (jp 1 (f2cl-lib:int-add jp 1))
              ((> jp 2) nil)
  (tagbody
    (setf tmp1
            (- (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
               (f2cl-lib:fref ab-%data% (ji jp) ((1 mmax) (1 *))
                              ab-%offset%)))
    (if (< (abs tmp1) pivmin)
        (setf tmp1 (- pivmin)))
    (setf (f2cl-lib:fref nab-%data% (ji jp) ((1 mmax) (1 *))
                         nab-%offset%)
            0)
    (if (<= tmp1 zero)
        (setf (f2cl-lib:fref nab-%data% (ji jp) ((1 mmax) (1 *))
                             nab-%offset%)
              1))
    (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf tmp1
                (-
                 (+
                  (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                  (/
                   (-
                    (f2cl-lib:fref e2-%data%
                                   ((f2cl-lib:int-sub j 1))
                                   ((1 *)) e2-%offset%))
                   tmp1))
                 (f2cl-lib:fref ab-%data% (ji jp)
                                ((1 mmax) (1 *)) ab-%offset%)))
        (if (< (abs tmp1) pivmin)
            (setf tmp1 (- pivmin)))
        (if (<= tmp1 zero)
            (setf (f2cl-lib:fref nab-%data% (ji jp)
                                 ((1 mmax) (1 *)) nab-%offset%)
                    (f2cl-lib:int-add
                     (f2cl-lib:fref nab-%data% (ji jp)
                                    ((1 mmax) (1 *))
                                    nab-%offset%)
                     1)))
       label10))
   label20))
(setf mout
        (f2cl-lib:int-sub
         (f2cl-lib:int-add mout
                           (f2cl-lib:fref nab-%data% (ji 2)
                                          ((1 mmax) (1 *))
                                          nab-%offset%))
         (f2cl-lib:fref nab-%data% (ji 1) ((1 mmax) (1 *))
```

```
                                nab-%offset%)))
     label30))
  (go end_label)))
(setf kf 1)
(setf kl minp)
(cond
 ((= ijob 2)
  (f2cl-lib:fdo (ji 1 (f2cl-lib:int-add ji 1))
                ((> ji minp) nil)
    (tagbody
      (setf (f2cl-lib:fref c-%data% (ji) ((1 *)) c-%offset%)
              (* half
                 (+
                  (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
                                 ab-%offset%)
                  (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                                 ab-%offset%))))
     label40))))
(f2cl-lib:fdo (jit 1 (f2cl-lib:int-add jit 1))
              ((> jit nitmax) nil)
  (tagbody
    (cond
     ((and (>= (f2cl-lib:int-add kl (f2cl-lib:int-sub kf) 1) nbmin)
           (> nbmin 0))
      (f2cl-lib:fdo (ji kf (f2cl-lib:int-add ji 1))
                    ((> ji kl) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (ji) ((1 *)) work-%offset%)
                  (- (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
                     (f2cl-lib:fref c-%data% (ji) ((1 *)) c-%offset%)))
          (setf (f2cl-lib:fref iwork-%data% (ji) ((1 *))
                               iwork-%offset%)
                  0)
          (cond
           ((<= (f2cl-lib:fref work (ji) ((1 *))) pivmin)
            (setf (f2cl-lib:fref iwork-%data% (ji) ((1 *))
                                 iwork-%offset%)
                    1)
            (setf (f2cl-lib:fref work-%data% (ji) ((1 *))
                                 work-%offset%)
                    (min
                     (f2cl-lib:fref work-%data% (ji) ((1 *))
                                    work-%offset%)
                     (- pivmin)))))
          (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (setf (f2cl-lib:fref work-%data% (ji) ((1 *))
                                   work-%offset%)
                      (-
```

```
                    (+
                     (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                     (/
                      (-
                       (f2cl-lib:fref e2-%data%
                                      ((f2cl-lib:int-sub j 1))
                                      ((1 *)) e2-%offset%))
                      (f2cl-lib:fref work-%data% (ji) ((1 *))
                                     work-%offset%)))
                    (f2cl-lib:fref c-%data% (ji) ((1 *))
                                   c-%offset%)))
            (cond
             ((<= (f2cl-lib:fref work (ji) ((1 *))) pivmin)
              (setf (f2cl-lib:fref iwork-%data% (ji) ((1 *))
                                   iwork-%offset%)
                    (f2cl-lib:int-add
                     (f2cl-lib:fref iwork-%data% (ji) ((1 *))
                                    iwork-%offset%)
                     1))
              (setf (f2cl-lib:fref work-%data% (ji) ((1 *))
                                   work-%offset%)
                    (min
                     (f2cl-lib:fref work-%data% (ji) ((1 *))
                                    work-%offset%)
                     (- pivmin)))))
          label50))
      label60))
  (cond
   ((<= ijob 2) (setf klnew kl)
    (f2cl-lib:fdo (ji kf (f2cl-lib:int-add ji 1))
                  ((> ji kl) nil)
      (tagbody
        (setf (f2cl-lib:fref iwork-%data% (ji) ((1 *))
                             iwork-%offset%)
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:fref nab-%data% (ji 2)
                                   ((1 mmax) (1 *))
                                   nab-%offset%))
               (the f2cl-lib:integer4
                    (max
                     (the f2cl-lib:integer4
                          (f2cl-lib:fref nab-%data% (ji 1)
                                         ((1 mmax) (1 *))
                                         nab-%offset%))
                     (the f2cl-lib:integer4
                          (f2cl-lib:fref iwork-%data% (ji)
                                         ((1 *))
                                         iwork-%offset%))))))
        (cond
```

```
      ((= (f2cl-lib:fref iwork (ji) ((1 *)))
          (f2cl-lib:fref nab (ji 2) ((1 mmax) (1 *))))
       (setf (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                            ab-%offset%)
             (f2cl-lib:fref c-%data% (ji) ((1 *))
                            c-%offset%)))
      ((= (f2cl-lib:fref iwork (ji) ((1 *)))
          (f2cl-lib:fref nab (ji 1) ((1 mmax) (1 *))))
       (setf (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
                            ab-%offset%)
             (f2cl-lib:fref c-%data% (ji) ((1 *))
                            c-%offset%)))
      (t (setf klnew (f2cl-lib:int-add klnew 1))
       (cond
        ((<= klnew mmax)
         (setf (f2cl-lib:fref ab-%data% (klnew 2)
                              ((1 mmax) (1 *)) ab-%offset%)
               (f2cl-lib:fref ab-%data% (ji 2)
                              ((1 mmax) (1 *)) ab-%offset%))
         (setf (f2cl-lib:fref nab-%data% (klnew 2)
                              ((1 mmax) (1 *)) nab-%offset%)
               (f2cl-lib:fref nab-%data% (ji 2)
                              ((1 mmax) (1 *)) nab-%offset%))
         (setf (f2cl-lib:fref ab-%data% (klnew 1)
                              ((1 mmax) (1 *)) ab-%offset%)
               (f2cl-lib:fref c-%data% (ji) ((1 *))
                              c-%offset%))
         (setf (f2cl-lib:fref nab-%data% (klnew 1)
                              ((1 mmax) (1 *)) nab-%offset%)
               (f2cl-lib:fref iwork-%data% (ji) ((1 *))
                              iwork-%offset%))
         (setf (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                              ab-%offset%)
               (f2cl-lib:fref c-%data% (ji) ((1 *))
                              c-%offset%))
         (setf (f2cl-lib:fref nab-%data% (ji 2) ((1 mmax) (1 *))
                              nab-%offset%)
               (f2cl-lib:fref iwork-%data% (ji) ((1 *))
                              iwork-%offset%)))
        (t (setf info (f2cl-lib:int-add mmax 1))))))
    label70))
 (if (/= info 0)
     (go end_label))
 (setf kl klnew))
(t
 (f2cl-lib:fdo (ji kf (f2cl-lib:int-add ji 1))
               ((> ji kl) nil)
   (tagbody
     (cond
      ((<= (f2cl-lib:fref iwork (ji) ((1 *)))
```

```
                    (f2cl-lib:fref nval (ji) ((1 *))))
                (setf (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
                                     ab-%offset%)
                        (f2cl-lib:fref c-%data% (ji) ((1 *)) c-%offset%))
                (setf (f2cl-lib:fref nab-%data% (ji 1) ((1 mmax) (1 *))
                                     nab-%offset%)
                        (f2cl-lib:fref iwork-%data% (ji) ((1 *))
                                       iwork-%offset%))))
            (cond
             ((>= (f2cl-lib:fref iwork (ji) ((1 *)))
                  (f2cl-lib:fref nval (ji) ((1 *))))
              (setf (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                                     ab-%offset%)
                        (f2cl-lib:fref c-%data% (ji) ((1 *)) c-%offset%))
              (setf (f2cl-lib:fref nab-%data% (ji 2) ((1 mmax) (1 *))
                                     nab-%offset%)
                        (f2cl-lib:fref iwork-%data% (ji) ((1 *))
                                       iwork-%offset%))))
          label80)))))
      (t (setf klnew kl)
       (f2cl-lib:fdo (ji kf (f2cl-lib:int-add ji 1))
                     ((> ji kl) nil)
         (tagbody
           (setf tmp1 (f2cl-lib:fref c-%data% (ji) ((1 *)) c-%offset%))
           (setf tmp2
                   (- (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
                      tmp1))
           (setf itmp1 0)
           (cond
            ((<= tmp2 pivmin) (setf itmp1 1)
             (setf tmp2 (min tmp2 (- pivmin)))))
           (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                         ((> j n) nil)
             (tagbody
               (setf tmp2
                       (-
                        (+
                         (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                         (/
                          (-
                           (f2cl-lib:fref e2-%data%
                                          ((f2cl-lib:int-sub j 1))
                                          ((1 *)) e2-%offset%))
                          tmp2))
                        tmp1))
               (cond
                ((<= tmp2 pivmin)
                 (setf itmp1 (f2cl-lib:int-add itmp1 1))
                 (setf tmp2 (min tmp2 (- pivmin)))))
               label90))
```

```
(cond
 ((<= ijob 2)
  (setf itmp1
          (min
           (the f2cl-lib:integer4
                (f2cl-lib:fref nab-%data% (ji 2)
                               ((1 mmax) (1 *))
                               nab-%offset%))
           (the f2cl-lib:integer4
                (max
                 (the f2cl-lib:integer4
                      (f2cl-lib:fref nab-%data% (ji 1)
                                     ((1 mmax) (1 *))
                                     nab-%offset%))
                 (the f2cl-lib:integer4 itmp1)))))
  (cond
   ((= itmp1 (f2cl-lib:fref nab (ji 2) ((1 mmax) (1 *))))
    (setf (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                         ab-%offset%)
          tmp1))
   ((= itmp1 (f2cl-lib:fref nab (ji 1) ((1 mmax) (1 *))))
    (setf (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
                         ab-%offset%)
          tmp1))
   ((< klnew mmax) (setf klnew (f2cl-lib:int-add klnew 1))
    (setf (f2cl-lib:fref ab-%data% (klnew 2) ((1 mmax) (1 *))
                         ab-%offset%)
          (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                           ab-%offset%))
    (setf (f2cl-lib:fref nab-%data% (klnew 2)
                         ((1 mmax) (1 *)) nab-%offset%)
          (f2cl-lib:fref nab-%data% (ji 2) ((1 mmax) (1 *))
                           nab-%offset%))
    (setf (f2cl-lib:fref ab-%data% (klnew 1) ((1 mmax) (1 *))
                         ab-%offset%)
          tmp1)
    (setf (f2cl-lib:fref nab-%data% (klnew 1)
                         ((1 mmax) (1 *)) nab-%offset%)
          itmp1)
    (setf (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                         ab-%offset%)
          tmp1)
    (setf (f2cl-lib:fref nab-%data% (ji 2) ((1 mmax) (1 *))
                         nab-%offset%)
          itmp1))
   (t (setf info (f2cl-lib:int-add mmax 1)) (go end_label))))
 (t
  (cond
   ((<= itmp1 (f2cl-lib:fref nval (ji) ((1 *))))
    (setf (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
```

```
                                ab-%offset%)
                        tmp1)
                  (setf (f2cl-lib:fref nab-%data% (ji 1) ((1 mmax) (1 *))
                                ab-%offset%)
                        itmp1)))
              (cond
               ((>= itmp1 (f2cl-lib:fref nval (ji) ((1 *))))
                (setf (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                                ab-%offset%)
                        tmp1)
                (setf (f2cl-lib:fref nab-%data% (ji 2) ((1 mmax) (1 *))
                                nab-%offset%)
                        itmp1)))))
           label100))
        (setf kl klnew)))
      (setf kfnew kf)
      (f2cl-lib:fdo (ji kf (f2cl-lib:int-add ji 1))
                    ((> ji kl) nil)
        (tagbody
          (setf tmp1
                  (abs
                   (-
                    (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                                   ab-%offset%)
                    (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
                                   ab-%offset%))))
          (setf tmp2
                  (max
                   (abs
                    (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                                   ab-%offset%))
                   (abs
                    (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
                                   ab-%offset%))))
          (cond
           ((or (< tmp1 (max abstol pivmin (* reltol tmp2)))
                (>= (f2cl-lib:fref nab (ji 1) ((1 mmax) (1 *)))
                    (f2cl-lib:fref nab (ji 2) ((1 mmax) (1 *)))))
            (cond
             ((> ji kfnew)
              (setf tmp1
                      (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
                                     ab-%offset%))
              (setf tmp2
                      (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                                     ab-%offset%))
              (setf itmp1
                      (f2cl-lib:fref nab-%data% (ji 1) ((1 mmax) (1 *))
                                     nab-%offset%))
              (setf itmp2
```

```
                (f2cl-lib:fref nab-%data% (ji 2) ((1 mmax) (1 *))
                               nab-%offset%))
        (setf (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
                             ab-%offset%)
                (f2cl-lib:fref ab-%data% (kfnew 1) ((1 mmax) (1 *))
                               ab-%offset%))
        (setf (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                             ab-%offset%)
                (f2cl-lib:fref ab-%data% (kfnew 2) ((1 mmax) (1 *))
                               ab-%offset%))
        (setf (f2cl-lib:fref nab-%data% (ji 1) ((1 mmax) (1 *))
                             nab-%offset%)
                (f2cl-lib:fref nab-%data% (kfnew 1)
                               ((1 mmax) (1 *)) nab-%offset%))
        (setf (f2cl-lib:fref nab-%data% (ji 2) ((1 mmax) (1 *))
                             nab-%offset%)
                (f2cl-lib:fref nab-%data% (kfnew 2)
                               ((1 mmax) (1 *)) nab-%offset%))
        (setf (f2cl-lib:fref ab-%data% (kfnew 1) ((1 mmax) (1 *))
                             ab-%offset%)
                tmp1)
        (setf (f2cl-lib:fref ab-%data% (kfnew 2) ((1 mmax) (1 *))
                             ab-%offset%)
                tmp2)
        (setf (f2cl-lib:fref nab-%data% (kfnew 1) ((1 mmax) (1 *))
                             nab-%offset%)
                itmp1)
        (setf (f2cl-lib:fref nab-%data% (kfnew 2) ((1 mmax) (1 *))
                             nab-%offset%)
                itmp2)
        (cond
         ((= ijob 3)
          (setf itmp1
                  (f2cl-lib:fref nval-%data% (ji) ((1 *))
                                 nval-%offset%))
          (setf (f2cl-lib:fref nval-%data% (ji) ((1 *))
                               nval-%offset%)
                  (f2cl-lib:fref nval-%data% (kfnew) ((1 *))
                                 nval-%offset%))
          (setf (f2cl-lib:fref nval-%data% (kfnew) ((1 *))
                               nval-%offset%)
                  itmp1)))))
      (setf kfnew (f2cl-lib:int-add kfnew 1))))
   label110))
(setf kf kfnew)
(f2cl-lib:fdo (ji kf (f2cl-lib:int-add ji 1))
              ((> ji kl) nil)
  (tagbody
    (setf (f2cl-lib:fref c-%data% (ji) ((1 *)) c-%offset%)
            (* half
```

```
                          (+
                           (f2cl-lib:fref ab-%data% (ji 1) ((1 mmax) (1 *))
                                          ab-%offset%)
                           (f2cl-lib:fref ab-%data% (ji 2) ((1 mmax) (1 *))
                                          ab-%offset%))))
               label120))
           (if (> kf kl)
               (go label140))
          label130))
      label140
       (setf info
               (max
                (the f2cl-lib:integer4
                     (f2cl-lib:int-sub (f2cl-lib:int-add kl 1) kf))
                (the f2cl-lib:integer4 0)))
       (setf mout kl)
       (go end_label)
      end_label
       (return
        (values nil nil nil nil nil nil nil nil nil nil nil nil nil nil nil
                mout nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slaebz
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (fortran-to-lisp::integer4)
                                             (single-float) (single-float)
                                             (single-float)
                                             (array single-float (*))
                                             (array single-float (*))
                                             (array single-float (*))
                                             (array fortran-to-lisp::integer4
                                              (*))
                                             (array single-float (*))
                                             (array single-float (*))
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::integer4
                                              (*))
                                             (array single-float (*))
                                             (array fortran-to-lisp::integer4
                                              (*))
                                             (fortran-to-lisp::integer4))
```

```
:return-values
'(nil nil nil nil nil nil nil nil
  nil nil nil nil nil nil nil
  fortran-to-lisp::mout nil nil nil
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

90.65 slaev2

90.65.1 Axiom unit tests

— slaev2.input —

```
)set break resume
)sys rm -f slaev2.output
)spool slaev2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.65.2 Axiom help page

— slaev2.help —

```
=====================================================================
slaev2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slaev2 (A, B, C, RT1, RT2, CS1, SN1)
```

```
          SLAEV2 computes the eigenvalues and eigenvectors of a 2-by-2
          symmetric/Hermitian matrix.

Function/Subroutine Documentation
   subroutine slaev2 (real A, real B, real C, real RT1, real RT2, real CS1,
       real SN1)
       SLAEV2 computes the eigenvalues and eigenvectors of a 2-by-2
       symmetric/Hermitian matrix.

Purpose:
   SLAEV2 computes the eigendecomposition of a 2-by-2 symmetric matrix
               [  A   B  ]
               [  B   C  ].
   On return, RT1 is the eigenvalue of larger absolute value, RT2 is the
   eigenvalue of smaller absolute value, and (CS1,SN1) is the unit right
   eigenvector for RT1, giving the decomposition

               [ CS1  SN1 ] [  A   B  ] [ CS1 -SN1 ]  =  [ RT1  0  ]
               [-SN1  CS1 ] [  B   C  ] [ SN1  CS1 ]     [  0  RT2 ].

Parameters:

 A
   A is REAL
   The (1,1) element of the 2-by-2 matrix.

 B
   B is REAL
   The (1,2) element and the conjugate of the (2,1) element of
   the 2-by-2 matrix.

 C
   C is REAL
   The (2,2) element of the 2-by-2 matrix.

 RT1
   RT1 is REAL
   The eigenvalue of larger absolute value.

 RT2
   RT2 is REAL
   The eigenvalue of smaller absolute value.

 CS1
   CS1 is REAL

 SN1
   SN1 is REAL
   The vector (CS1, SN1) is a unit right eigenvector for RT1.
```

```
Further Details:

   RT1 is accurate to a few ulps barring over/underflow.

   RT2 may be inaccurate if there is massive cancellation in the
   determinant A*C-B*B; higher precision or correctly rounded or
   correctly truncated arithmetic would be needed to compute RT2
   accurately in all cases.

   CS1 and SN1 are accurate to a few ulps barring over/underflow.

   Overflow is possible only if RT1 is within a factor of 5 of overflow.
   Underflow is harmless if the input data is 0 or exceeds
   underflow_threshold / macheps.
```

———-

90.65.3 fortran code

— slaev2.f —

```
*  =====================================================================
      SUBROUTINE SLAEV2( A, B, C, RT1, RT2, CS1, SN1 )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      REAL               A, B, C, CS1, RT1, RT2, SN1
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E0 )
      REAL               TWO
      PARAMETER          ( TWO = 2.0E0 )
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E0 )
      REAL               HALF
      PARAMETER          ( HALF = 0.5E0 )
*     ..
*     .. Local Scalars ..
      INTEGER            SGN1, SGN2
```

```
      REAL               AB, ACMN, ACMX, ACS, ADF, CS, CT, DF, RT, SM,
     $                   TB, TN
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SQRT
*     ..
*     .. Executable Statements ..
*
*     Compute the eigenvalues
*
      SM = A + C
      DF = A - C
      ADF = ABS( DF )
      TB = B + B
      AB = ABS( TB )
      IF( ABS( A ).GT.ABS( C ) ) THEN
         ACMX = A
         ACMN = C
      ELSE
         ACMX = C
         ACMN = A
      END IF
      IF( ADF.GT.AB ) THEN
         RT = ADF*SQRT( ONE+( AB / ADF )**2 )
      ELSE IF( ADF.LT.AB ) THEN
         RT = AB*SQRT( ONE+( ADF / AB )**2 )
      ELSE
*
*        Includes case AB=ADF=0
*
         RT = AB*SQRT( TWO )
      END IF
      IF( SM.LT.ZERO ) THEN
         RT1 = HALF*( SM-RT )
         SGN1 = -1
*
*        Order of execution important.
*        To get fully accurate smaller eigenvalue,
*        next line needs to be executed in higher precision.
*
         RT2 = ( ACMX / RT1 )*ACMN - ( B / RT1 )*B
      ELSE IF( SM.GT.ZERO ) THEN
         RT1 = HALF*( SM+RT )
         SGN1 = 1
*
*        Order of execution important.
*        To get fully accurate smaller eigenvalue,
*        next line needs to be executed in higher precision.
*
         RT2 = ( ACMX / RT1 )*ACMN - ( B / RT1 )*B
```

```
      ELSE
*
*        Includes case RT1 = RT2 = 0
*
         RT1 = HALF*RT
         RT2 = -HALF*RT
         SGN1 = 1
      END IF
*
*     Compute the eigenvector
*
      IF( DF.GE.ZERO ) THEN
         CS = DF + RT
         SGN2 = 1
      ELSE
         CS = DF - RT
         SGN2 = -1
      END IF
      ACS = ABS( CS )
      IF( ACS.GT.AB ) THEN
         CT = -TB / CS
         SN1 = ONE / SQRT( ONE+CT*CT )
         CS1 = CT*SN1
      ELSE
         IF( AB.EQ.ZERO ) THEN
            CS1 = ONE
            SN1 = ZERO
         ELSE
            TN = -CS / TB
            CS1 = ONE / SQRT( ONE+TN*TN )
            SN1 = TN*CS1
         END IF
      END IF
      IF( SGN1.EQ.SGN2 ) THEN
         TN = CS1
         CS1 = -SN1
         SN1 = TN
      END IF
      RETURN
*
*     End of SLAEV2
*
      END
```

———-

90.65.4 lisp code

— slaev2.lisp —

```
(let* ((one 1.0) (two 2.0) (zero 0.0) (half 0.5))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 0.0 0.0) zero)
           (type (single-float 0.5 0.5) half)
           (ignorable one two zero half))
  (defun slaev2 (a b c rt1 rt2 cs1 sn1)
    (declare (type (single-float) sn1 cs1 rt2 rt1 c b a))
    (prog ((ab 0.0) (acmn 0.0) (acmx 0.0) (acs 0.0) (adf 0.0) (cs 0.0) (ct 0.0)
           (df 0.0) (rt 0.0) (sm 0.0) (tb 0.0) (tn 0.0) (sgn1 0) (sgn2 0))
      (declare (type (single-float) ab acmn acmx acs adf cs ct df rt sm tb tn)
               (type (f2cl-lib:integer4) sgn1 sgn2))
      (setf sm (+ a c))
      (setf df (- a c))
      (setf adf (abs df))
      (setf tb (+ b b))
      (setf ab (abs tb))
      (cond ((> (abs a) (abs c)) (setf acmx a) (setf acmn c))
            (t (setf acmx c) (setf acmn a)))
      (cond
       ((> adf ab)
        (setf rt (* adf (f2cl-lib:fsqrt (+ one (expt (/ ab adf) 2))))))
       ((< adf ab)
        (setf rt (* ab (f2cl-lib:fsqrt (+ one (expt (/ adf ab) 2))))))
       (t (setf rt (* ab (f2cl-lib:fsqrt two)))))
      (cond
       ((< sm zero) (setf rt1 (* half (- sm rt))) (setf sgn1 -1)
        (setf rt2 (- (* (/ acmx rt1) acmn) (* (/ b rt1) b))))
       ((> sm zero) (setf rt1 (* half (+ sm rt))) (setf sgn1 1)
        (setf rt2 (- (* (/ acmx rt1) acmn) (* (/ b rt1) b))))
       (t (setf rt1 (* half rt)) (setf rt2 (* (- half) rt)) (setf sgn1 1)))
      (cond ((>= df zero) (setf cs (+ df rt)) (setf sgn2 1))
            (t (setf cs (- df rt)) (setf sgn2 -1)))
      (setf acs (abs cs))
      (cond
       ((> acs ab) (setf ct (/ (- tb) cs))
        (setf sn1 (/ one (f2cl-lib:fsqrt (+ one (* ct ct)))))
        (setf cs1 (* ct sn1)))
       (t
        (cond ((= ab zero) (setf cs1 one) (setf sn1 zero))
              (t (setf tn (/ (- cs) tb))
               (setf cs1 (/ one (f2cl-lib:fsqrt (+ one (* tn tn)))))
               (setf sn1 (* tn cs1))))))
      (cond ((= sgn1 sgn2) (setf tn cs1) (setf cs1 (- sn1)) (setf sn1 tn)))
      (go end_label)
```

```
     end_label
      (return (values nil nil nil rt1 rt2 cs1 sn1)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slaev2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((single-float) (single-float)
                                              (single-float) (single-float)
                                              (single-float) (single-float)
                                              (single-float))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::rt1
                                              fortran-to-lisp::rt2
                                              fortran-to-lisp::cs1
                                              fortran-to-lisp::sn1)
                                            :calls 'nil)))
```

———-

90.66 slag2d

90.66.1 Axiom unit tests

— slag2d.input —

```
)set break resume
)sys rm -f slag2d.output
)spool slag2d.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.66.2 Axiom help page

— slag2d.help —

```
====================================================================
slag2d examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slag2d (M, N, SA, LDSA, A, LDA, INFO)
           SLAG2D converts a single precision matrix to a double precision
           matrix.

Function/Subroutine Documentation
   subroutine slag2d (integer M, integer N, real, dimension( ldsa, * ) SA,
       integer LDSA, double precision, dimension( lda, * ) A, integer LDA,
       integer INFO)
       SLAG2D converts a single precision matrix to a double precision matrix.

Purpose:
   SLAG2D converts a SINGLE PRECISION matrix, SA, to a DOUBLE
   PRECISION matrix, A.

   Note that while it is possible to overflow while converting
   from double to single, it is not possible to overflow when
   converting from single to double.

   This is an auxiliary routine so there is no argument checking.

Parameters:

 M
   M is INTEGER
   The number of lines of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.

 SA
   SA is REAL array, dimension (LDSA,N)
   On entry, the M-by-N coefficient matrix SA.

 LDSA
   LDSA is INTEGER
   The leading dimension of the array SA.  LDSA >= max(1,M).

 A
   A is DOUBLE PRECISION array, dimension (LDA,N)
```

```
  On exit, the M-by-N coefficient matrix A.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

INFO
  INFO is INTEGER
  = 0:  successful exit
```

90.66.3 fortran code

— slag2d.f —

```
*  =====================================================================
      SUBROUTINE SLAG2D( M, N, SA, LDSA, A, LDA, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDSA, M, N
*     ..
*     .. Array Arguments ..
      REAL               SA( LDSA, * )
      DOUBLE PRECISION   A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      DO 20 J = 1, N
         DO 10 I = 1, M
            A( I, J ) = SA( I, J )
   10    CONTINUE
   20 CONTINUE
      RETURN
*
```

```
*     End of SLAG2D
*
      END
```

———-

90.66.4 lisp code

— **slag2d.lisp** —

```
(defun slag2d (m n sa ldsa a lda info)
  (declare (type (array double-float (*)) a)
           (type (array single-float (*)) sa)
           (type (f2cl-lib:integer4) info lda ldsa n m))
  (f2cl-lib:with-multi-array-data
      ((sa single-float sa-%data% sa-%offset%)
       (a double-float a-%data% a-%offset%))
    (prog ((i 0) (j 0))
      (declare (type (f2cl-lib:integer4) j i))
      (setf info 0)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                      (coerce
                       (f2cl-lib:fref sa-%data% (i j) ((1 ldsa) (1 *))
                                      sa-%offset%)
                       'double-float))
             label10))
         label20))
      (go end_label)
     end_label
      (return (values nil nil nil nil nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slag2d
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array double-float (*))
```

```
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil nil nil nil
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

90.67 slagts

90.67.1 Axiom unit tests

— **slagts.input** —

```
)set break resume
)sys rm -f slagts.output
)spool slagts.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.67.2 Axiom help page

— **slagts.help** —

```
==================================================================
slagts examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slagts (JOB, N, A, B, C, D, IN, Y, TOL, INFO)
```

```
          SLAGTS solves the system of equations (T-I)x = y or (T-I)Tx =
          y,where T is a general tridiagonal matrix and  a scalar, using the
          LU factorization computed by slagtf.

Function/Subroutine Documentation
   subroutine slagts (integer JOB, integer N, real, dimension( * ) A, real,
       dimension( * ) B, real, dimension( * ) C, real, dimension( * ) D,
       integer, dimension( * ) IN, real, dimension( * ) Y, real TOL, integer
       INFO)
       SLAGTS solves the system of equations (T-I)x = y or (T-I)Tx = y,where
       T is a general tridiagonal matrix and  a scalar, using the LU
       factorization computed by slagtf.

Purpose:
   SLAGTS may be used to solve one of the systems of equations

   (T - lambda*I)*x = y   or   (T - lambda*I)**T*x = y,

   where T is an n by n tridiagonal matrix, for x, following the
   factorization of (T - lambda*I) as

   (T - lambda*I) = P*L*U ,

   by routine SLAGTF. The choice of equation to be solved is
   controlled by the argument JOB, and in each case there is an option
   to perturb zero or very small diagonal elements of U, this option
   being intended for use in applications such as inverse iteration.

Parameters:

 JOB
   JOB is INTEGER
   Specifies the job to be performed by SLAGTS as follows:
   =  1: The equations  (T - lambda*I)x = y  are to be solved,
      but diagonal elements of U are not to be perturbed.
   = -1: The equations  (T - lambda*I)x = y  are to be solved
      and, if overflow would otherwise occur, the diagonal
      elements of U are to be perturbed. See argument TOL below.
   =  2: The equations  (T - lambda*I)**Tx = y  are to be solved,
      but diagonal elements of U are not to be perturbed.
   = -2: The equations  (T - lambda*I)**Tx = y  are to be solved
      and, if overflow would otherwise occur, the diagonal
      elements of U are to be perturbed. See argument TOL below.

 N
   N is INTEGER
   The order of the matrix T.

 A
   A is REAL array, dimension (N)
```

```
  On entry, A must contain the diagonal elements of U as
  returned from SLAGTF.

B
  B is REAL array, dimension (N-1)
  On entry, B must contain the first super-diagonal elements of
  U as returned from SLAGTF.

C
  C is REAL array, dimension (N-1)
  On entry, C must contain the sub-diagonal elements of L as
  returned from SLAGTF.

D
  D is REAL array, dimension (N-2)
  On entry, D must contain the second super-diagonal elements
  of U as returned from SLAGTF.

IN
  IN is INTEGER array, dimension (N)
  On entry, IN must contain details of the matrix P as returned from SLAGTF.

Y
  Y is REAL array, dimension (N)
  On entry, the right hand side vector y.
  On exit, Y is overwritten by the solution vector x.

TOL
  TOL is REAL
  On entry, with  JOB .lt. 0, TOL should be the minimum
  perturbation to be made to very small diagonal elements of U.
  TOL should normally be chosen as about eps*norm(U), where eps
  is the relative machine precision, but if TOL is supplied as
  non-positive, then it is reset to eps*max( abs( u(i,j) ) ).
  If  JOB .gt. 0  then TOL is not referenced.

  On exit, TOL is changed as described above, only if TOL is
  non-positive on entry. Otherwise TOL is unchanged.

INFO
  INFO is INTEGER
  = 0   : successful exit
  .lt. 0: if INFO = -i, the i-th argument had an illegal value
  .gt. 0: overflow would occur when computing the INFO(th)
     element of the solution vector x. This can only occur
     when JOB is supplied as positive and either means
     that a diagonal element of U is very small, or that
     the elements of the right-hand side vector y are very large.
```

———-

90.67.3 fortran code

— **slagts.f** —

```
*  =====================================================================
      SUBROUTINE SLAGTS( JOB, N, A, B, C, D, IN, Y, TOL, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, JOB, N
      REAL               TOL
*     ..
*     .. Array Arguments ..
      INTEGER            IN( * )
      REAL               A( * ), B( * ), C( * ), D( * ), Y( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            K
      REAL               ABSAK, AK, BIGNUM, EPS, PERT, SFMIN, TEMP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SIGN
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      IF( ( ABS( JOB ).GT.2 ) .OR. ( JOB.EQ.0 ) ) THEN
         INFO = -1
```

```
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SLAGTS', -INFO )
         RETURN
      END IF
*
      IF( N.EQ.0 )
     $   RETURN
*
      EPS = SLAMCH( 'Epsilon' )
      SFMIN = SLAMCH( 'Safe minimum' )
      BIGNUM = ONE / SFMIN
*
      IF( JOB.LT.0 ) THEN
         IF( TOL.LE.ZERO ) THEN
            TOL = ABS( A( 1 ) )
            IF( N.GT.1 )
     $         TOL = MAX( TOL, ABS( A( 2 ) ), ABS( B( 1 ) ) )
            DO 10 K = 3, N
               TOL = MAX( TOL, ABS( A( K ) ), ABS( B( K-1 ) ),
     $               ABS( D( K-2 ) ) )
   10       CONTINUE
            TOL = TOL*EPS
            IF( TOL.EQ.ZERO )
     $         TOL = EPS
         END IF
      END IF
*
      IF( ABS( JOB ).EQ.1 ) THEN
         DO 20 K = 2, N
            IF( IN( K-1 ).EQ.0 ) THEN
               Y( K ) = Y( K ) - C( K-1 )*Y( K-1 )
            ELSE
               TEMP = Y( K-1 )
               Y( K-1 ) = Y( K )
               Y( K ) = TEMP - C( K-1 )*Y( K )
            END IF
   20    CONTINUE
         IF( JOB.EQ.1 ) THEN
            DO 30 K = N, 1, -1
               IF( K.LE.N-2 ) THEN
                  TEMP = Y( K ) - B( K )*Y( K+1 ) - D( K )*Y( K+2 )
               ELSE IF( K.EQ.N-1 ) THEN
                  TEMP = Y( K ) - B( K )*Y( K+1 )
               ELSE
                  TEMP = Y( K )
               END IF
               AK = A( K )
```

```
            ABSAK = ABS( AK )
            IF( ABSAK.LT.ONE ) THEN
               IF( ABSAK.LT.SFMIN ) THEN
                  IF( ABSAK.EQ.ZERO .OR. ABS( TEMP )*SFMIN.GT.ABSAK )
     $                 THEN
                     INFO = K
                     RETURN
                  ELSE
                     TEMP = TEMP*BIGNUM
                     AK = AK*BIGNUM
                  END IF
               ELSE IF( ABS( TEMP ).GT.ABSAK*BIGNUM ) THEN
                  INFO = K
                  RETURN
               END IF
            END IF
            Y( K ) = TEMP / AK
30       CONTINUE
      ELSE
         DO 50 K = N, 1, -1
            IF( K.LE.N-2 ) THEN
               TEMP = Y( K ) - B( K )*Y( K+1 ) - D( K )*Y( K+2 )
            ELSE IF( K.EQ.N-1 ) THEN
               TEMP = Y( K ) - B( K )*Y( K+1 )
            ELSE
               TEMP = Y( K )
            END IF
            AK = A( K )
            PERT = SIGN( TOL, AK )
40          CONTINUE
            ABSAK = ABS( AK )
            IF( ABSAK.LT.ONE ) THEN
               IF( ABSAK.LT.SFMIN ) THEN
                  IF( ABSAK.EQ.ZERO .OR. ABS( TEMP )*SFMIN.GT.ABSAK )
     $                 THEN
                     AK = AK + PERT
                     PERT = 2*PERT
                     GO TO 40
                  ELSE
                     TEMP = TEMP*BIGNUM
                     AK = AK*BIGNUM
                  END IF
               ELSE IF( ABS( TEMP ).GT.ABSAK*BIGNUM ) THEN
                  AK = AK + PERT
                  PERT = 2*PERT
                  GO TO 40
               END IF
            END IF
            Y( K ) = TEMP / AK
50       CONTINUE
```

```
         END IF
      ELSE
*
*        Come to here if  JOB = 2 or -2
*
         IF( JOB.EQ.2 ) THEN
            DO 60 K = 1, N
               IF( K.GE.3 ) THEN
                  TEMP = Y( K ) - B( K-1 )*Y( K-1 ) - D( K-2 )*Y( K-2 )
               ELSE IF( K.EQ.2 ) THEN
                  TEMP = Y( K ) - B( K-1 )*Y( K-1 )
               ELSE
                  TEMP = Y( K )
               END IF
               AK = A( K )
               ABSAK = ABS( AK )
               IF( ABSAK.LT.ONE ) THEN
                  IF( ABSAK.LT.SFMIN ) THEN
                     IF( ABSAK.EQ.ZERO .OR. ABS( TEMP )*SFMIN.GT.ABSAK )
     $                    THEN
                        INFO = K
                        RETURN
                     ELSE
                        TEMP = TEMP*BIGNUM
                        AK = AK*BIGNUM
                     END IF
                  ELSE IF( ABS( TEMP ).GT.ABSAK*BIGNUM ) THEN
                     INFO = K
                     RETURN
                  END IF
               END IF
               Y( K ) = TEMP / AK
   60       CONTINUE
         ELSE
            DO 80 K = 1, N
               IF( K.GE.3 ) THEN
                  TEMP = Y( K ) - B( K-1 )*Y( K-1 ) - D( K-2 )*Y( K-2 )
               ELSE IF( K.EQ.2 ) THEN
                  TEMP = Y( K ) - B( K-1 )*Y( K-1 )
               ELSE
                  TEMP = Y( K )
               END IF
               AK = A( K )
               PERT = SIGN( TOL, AK )
   70          CONTINUE
               ABSAK = ABS( AK )
               IF( ABSAK.LT.ONE ) THEN
                  IF( ABSAK.LT.SFMIN ) THEN
                     IF( ABSAK.EQ.ZERO .OR. ABS( TEMP )*SFMIN.GT.ABSAK )
     $                    THEN
```

```
                     AK = AK + PERT
                     PERT = 2*PERT
                     GO TO 70
                  ELSE
                     TEMP = TEMP*BIGNUM
                     AK = AK*BIGNUM
                  END IF
               ELSE IF( ABS( TEMP ).GT.ABSAK*BIGNUM ) THEN
                  AK = AK + PERT
                  PERT = 2*PERT
                  GO TO 70
               END IF
            END IF
            Y( K ) = TEMP / AK
   80       CONTINUE
         END IF
*
         DO 90 K = N, 2, -1
            IF( IN( K-1 ).EQ.0 ) THEN
               Y( K-1 ) = Y( K-1 ) - C( K-1 )*Y( K )
            ELSE
               TEMP = Y( K-1 )
               Y( K-1 ) = Y( K )
               Y( K ) = TEMP - C( K-1 )*Y( K )
            END IF
   90    CONTINUE
      END IF
*
*     End of SLAGTS
*
      END
```

———-

90.67.4 lisp code

— slagts.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun slagts (job n a b c d in y tol info)
    (declare (type (single-float) tol)
             (type (array f2cl-lib:integer4 (*)) in)
             (type (array single-float (*)) y d c b a)
             (type (f2cl-lib:integer4) info n job))
    (f2cl-lib:with-multi-array-data
```

```
  ((a single-float a-%data% a-%offset%)
   (b single-float b-%data% b-%offset%)
   (c single-float c-%data% c-%offset%)
   (d single-float d-%data% d-%offset%)
   (y single-float y-%data% y-%offset%)
   (in f2cl-lib:integer4 in-%data% in-%offset%))
(prog ((absak 0.0) (ak 0.0) (bignum 0.0) (eps 0.0) (pert 0.0) (sfmin 0.0)
       (temp 0.0) (k 0))
  (declare (type (single-float) absak ak bignum eps pert sfmin temp)
           (type (f2cl-lib:integer4) k))
  (setf info 0)
  (cond ((or (> (abs job) 2) (= job 0)) (setf info -1))
        ((< n 0) (setf info -2)))
  (cond
   ((/= info 0) (xerbla "SLAGTS" (f2cl-lib:int-sub info))
    (go end_label)))
  (if (= n 0)
      (go end_label))
  (setf eps (slamch "Epsilon"))
  (setf sfmin (slamch "Safe minimum"))
  (setf bignum (/ one sfmin))
  (cond
   ((< job 0)
    (cond
     ((<= tol zero)
      (setf tol (abs (f2cl-lib:fref a-%data% (1) ((1 *)) a-%offset%)))
      (if (> n 1)
          (setf tol
                  (max tol
                       (abs
                        (f2cl-lib:fref a-%data% (2) ((1 *)) a-%offset%))
                       (abs
                        (f2cl-lib:fref b-%data% (1) ((1 *))
                                       b-%offset%)))))
      (f2cl-lib:fdo (k 3 (f2cl-lib:int-add k 1))
                    ((> k n) nil)
        (tagbody
          (setf tol
                  (max tol
                       (abs
                        (f2cl-lib:fref a-%data% (k) ((1 *)) a-%offset%))
                       (abs
                        (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1))
                                       ((1 *)) b-%offset%))
                       (abs
                        (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub k 2))
                                       ((1 *)) d-%offset%))))
         label10))
      (setf tol (* tol eps))
      (if (= tol zero)
```

```
        (setf tol eps))))))
(cond
 ((= (abs job) 1)
  (f2cl-lib:fdo (k 2 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (cond
       ((=
         (f2cl-lib:fref in ((f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                        ((1 *)))
         0)
        (setf (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                (- (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                   (*
                    (f2cl-lib:fref c-%data% ((f2cl-lib:int-sub k 1))
                                   ((1 *)) c-%offset%)
                    (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1))
                                   ((1 *)) y-%offset%)))))
       (t
        (setf temp
                (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1))
                               ((1 *)) y-%offset%))
        (setf (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                             y-%offset%)
                (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%))
        (setf (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                (- temp
                   (*
                    (f2cl-lib:fref c-%data% ((f2cl-lib:int-sub k 1))
                                   ((1 *)) c-%offset%)
                    (f2cl-lib:fref y-%data% (k) ((1 *))
                                   y-%offset%))))))
    label20))
  (cond
   ((= job 1)
    (f2cl-lib:fdo (k n (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                  ((> k 1) nil)
      (tagbody
        (cond
         ((<= k (f2cl-lib:int-add n (f2cl-lib:int-sub 2)))
          (setf temp
                  (- (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                     (* (f2cl-lib:fref b-%data% (k) ((1 *)) b-%offset%)
                        (f2cl-lib:fref y-%data%
                                       ((f2cl-lib:int-add k 1)) ((1 *))
                                       y-%offset%))
                     (* (f2cl-lib:fref d-%data% (k) ((1 *)) d-%offset%)
                        (f2cl-lib:fref y-%data%
                                       ((f2cl-lib:int-add k 2)) ((1 *))
                                       y-%offset%)))))
```

```
        ((= k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
         (setf temp
                 (- (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                    (* (f2cl-lib:fref b-%data% (k) ((1 *)) b-%offset%)
                       (f2cl-lib:fref y-%data%
                                      ((f2cl-lib:int-add k 1)) ((1 *))
                                      y-%offset%)))))
        (t
         (setf temp (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%))))
       (setf ak (f2cl-lib:fref a-%data% (k) ((1 *)) a-%offset%))
       (setf absak (abs ak))
       (cond
        ((< absak one)
         (cond
          ((< absak sfmin)
           (cond
            ((or (= absak zero) (> (* (abs temp) sfmin) absak))
             (setf info k) (go end_label))
            (t (setf temp (* temp bignum)) (setf ak (* ak bignum)))))
          ((> (abs temp) (* absak bignum)) (setf info k)
           (go end_label)))))
       (setf (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
               (/ temp ak))
    label30)))
(t
 (f2cl-lib:fdo (k n (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
               ((> k 1) nil)
   (tagbody
     (cond
      ((<= k (f2cl-lib:int-add n (f2cl-lib:int-sub 2)))
       (setf temp
               (- (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                  (* (f2cl-lib:fref b-%data% (k) ((1 *)) b-%offset%)
                     (f2cl-lib:fref y-%data%
                                    ((f2cl-lib:int-add k 1)) ((1 *))
                                    y-%offset%))
                  (* (f2cl-lib:fref d-%data% (k) ((1 *)) d-%offset%)
                     (f2cl-lib:fref y-%data%
                                    ((f2cl-lib:int-add k 2)) ((1 *))
                                    y-%offset%)))))
      ((= k (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
       (setf temp
               (- (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                  (* (f2cl-lib:fref b-%data% (k) ((1 *)) b-%offset%)
                     (f2cl-lib:fref y-%data%
                                    ((f2cl-lib:int-add k 1)) ((1 *))
                                    y-%offset%)))))
      (t
       (setf temp (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%))))
     (setf ak (f2cl-lib:fref a-%data% (k) ((1 *)) a-%offset%))
```

```
        (setf pert (f2cl-lib:sign tol ak))
       label40
        (setf absak (abs ak))
        (cond
         ((< absak one)
          (cond
           ((< absak sfmin)
            (cond
             ((or (= absak zero) (> (* (abs temp) sfmin) absak))
              (setf ak (+ ak pert)) (setf pert (* 2 pert))
              (go label40))
             (t (setf temp (* temp bignum)) (setf ak (* ak bignum)))))
           ((> (abs temp) (* absak bignum)) (setf ak (+ ak pert))
            (setf pert (* 2 pert)) (go label40)))))
        (setf (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                (/ temp ak))
       label50)))))
(t
 (cond
  ((= job 2)
   (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                 ((> k n) nil)
     (tagbody
       (cond
        ((>= k 3)
         (setf temp
                 (- (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                    (*
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1))
                                    ((1 *)) b-%offset%)
                     (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1))
                                    ((1 *)) y-%offset%))
                    (*
                     (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub k 2))
                                    ((1 *)) d-%offset%)
                     (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 2))
                                    ((1 *)) y-%offset%)))))
        ((= k 2)
         (setf temp
                 (- (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                    (*
                     (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1))
                                    ((1 *)) b-%offset%)
                     (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1))
                                    ((1 *)) y-%offset%)))))
        (t
         (setf temp (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%))))
       (setf ak (f2cl-lib:fref a-%data% (k) ((1 *)) a-%offset%))
       (setf absak (abs ak))
       (cond
```

```
        ((< absak one)
         (cond
          ((< absak sfmin)
           (cond
            ((or (= absak zero) (> (* (abs temp) sfmin) absak))
             (setf info k) (go end_label))
            (t (setf temp (* temp bignum)) (setf ak (* ak bignum)))))
          ((> (abs temp) (* absak bignum)) (setf info k)
           (go end_label)))))
       (setf (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
               (/ temp ak))
      label60)))
    (t
     (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                   ((> k n) nil)
       (tagbody
         (cond
          ((>= k 3)
           (setf temp
                   (- (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                      (*
                       (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1))
                                      ((1 *)) b-%offset%)
                       (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1))
                                      ((1 *)) y-%offset%))
                      (*
                       (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub k 2))
                                      ((1 *)) d-%offset%)
                       (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 2))
                                      ((1 *)) y-%offset%)))))
          ((= k 2)
           (setf temp
                   (- (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                      (*
                       (f2cl-lib:fref b-%data% ((f2cl-lib:int-sub k 1))
                                      ((1 *)) b-%offset%)
                       (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1))
                                      ((1 *)) y-%offset%)))))
          (t
           (setf temp (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%))))
         (setf ak (f2cl-lib:fref a-%data% (k) ((1 *)) a-%offset%))
         (setf pert (f2cl-lib:sign tol ak))
        label70
         (setf absak (abs ak))
         (cond
          ((< absak one)
           (cond
            ((< absak sfmin)
             (cond
              ((or (= absak zero) (> (* (abs temp) sfmin) absak))
```

```
                    (setf ak (+ ak pert)) (setf pert (* 2 pert))
                    (go label70))
                   (t (setf temp (* temp bignum)) (setf ak (* ak bignum)))))
                ((> (abs temp) (* absak bignum)) (setf ak (+ ak pert))
                 (setf pert (* 2 pert)) (go label70)))))
             (setf (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                     (/ temp ak))
            label80))))
        (f2cl-lib:fdo (k n (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                      ((> k 2) nil)
          (tagbody
            (cond
             ((=
               (f2cl-lib:fref in ((f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                              ((1 *)))
               0)
             (setf (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                                  y-%offset%)
                     (-
                      (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1))
                                     ((1 *)) y-%offset%)
                      (*
                       (f2cl-lib:fref c-%data% ((f2cl-lib:int-sub k 1))
                                      ((1 *)) c-%offset%)
                       (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)))))
            (t
             (setf temp
                     (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1))
                                    ((1 *)) y-%offset%))
             (setf (f2cl-lib:fref y-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                                  y-%offset%)
                     (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%))
             (setf (f2cl-lib:fref y-%data% (k) ((1 *)) y-%offset%)
                     (- temp
                        (*
                         (f2cl-lib:fref c-%data% ((f2cl-lib:int-sub k 1))
                                        ((1 *)) c-%offset%)
                         (f2cl-lib:fref y-%data% (k) ((1 *))
                                        y-%offset%))))))
           label90))))
      end_label
       (return (values nil nil nil nil nil nil nil nil tol info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slagts
                 fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((fortran-to-lisp::integer4)
```

```
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (array single-float (*))
  (single-float)
  (fortran-to-lisp::integer4))
:return-values
'(nil nil nil nil nil nil nil nil
  fortran-to-lisp::tol
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

90.68 slaisnan

90.68.1 Axiom unit tests

— slaisnan.input —

```
)set break resume
)sys rm -f slaisnan.output
)spool slaisnan.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.68.2 Axiom help page

— slaisnan.help —

```
====================================================================
slaisnan examples
```

```
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       logical function slaisnan (SIN1, SIN2)
           SLAISNAN tests input for NaN by comparing two arguments for
           inequality.

Function/Subroutine Documentation
   logical function slaisnan (real SIN1, real SIN2)
       SLAISNAN tests input for NaN by comparing two arguments for inequality.

Purpose:
   This routine is not for general use.  It exists solely to avoid
   over-optimization in SISNAN.

   SLAISNAN checks for NaNs by comparing its two arguments for
   inequality.  NaN is the only floating-point value where NaN != NaN
   returns .TRUE.  To check for NaNs, pass the same variable as both
   arguments.

   A compiler must assume that the two arguments are
   not the same variable, and the test will not be optimized away.
   Interprocedural or whole-program optimization may delete this
   test.  The ISNAN functions will be replaced by the correct
   Fortran 03 intrinsic once the intrinsic is widely available.

Parameters:

 SIN1
   SIN1 is REAL

 SIN2
   SIN2 is REAL
   Two numbers to compare for inequality.
```

———-

90.68.3 fortran code

— slaisnan.f —

```
*  =====================================================================
```

```
      LOGICAL FUNCTION SLAISNAN( SIN1, SIN2 )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      REAL               SIN1, SIN2
*     ..
*
*  =====================================================================
*
*  .. Executable Statements ..
      SLAISNAN = (SIN1.NE.SIN2)
      RETURN
      END
```

———-

90.68.4 lisp code

— **slaisnan.lisp** —

```
(defun slaisnan (sin1 sin2)
  (declare (type (single-float) sin2 sin1))
  (prog ((slaisnan nil))
    (declare (type f2cl-lib:logical slaisnan))
    (setf slaisnan (coerce (/= sin1 sin2) 'f2cl-lib:logical))
    (go end_label)
   end_label
    (return (values slaisnan nil nil))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slaisnan
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((single-float) (single-float))
                                            :return-values '(nil nil) :calls
                                            'nil)))
```

———-

90.69 slamc1

90.69.1 Axiom unit tests

— slamc1.input —

```
)set break resume
)sys rm -f slamc1.output
)spool slamc1.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.69.2 Axiom help page

— slamc1.help —

```
=====================================================================
slamc1 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

subroutine slamc1(Integer BETA, Integer T, logical RND, logical IEEE1)

Purpose:
   determines the machine parameters given by BETA, T, RND, and IEEE1

Parameters:

 BETA
   The base of the machine
 T
   The number of (BETA) digits in the mantissa

 RND
   Specifies whether proper rounding (RND=.TRUE.) or chopping (RND=.FALSE.)
   occurs in addition. This may not be a reliable guide to the way in
   which the machine performs its arithmetic.
```

```
IEEE1
  Specifies whether rounding appears to be done in the IEEE
  'round to nearest' style.
```

———-

90.69.3 fortran code

— slamc1.f —

———-

90.69.4 lisp code

— slamc1.lisp —

———-

90.70 slamc2

90.70.1 Axiom unit tests

— slamc2.input —

```
)set break resume
)sys rm -f slamc2.output
)spool slamc2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.70.2 Axiom help page

— slamc2.help —

```
=====================================================================
slamc2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

subroutine slamc2(Integer BETA, Integer T, logical RND, real EPS,
                  Integer EMIN, real RMIN, Integer EMAX, real RMAX)

Purpose:
   determines the machine parameters specified in its argument list

 BETA
   The base of the machine

 T
   The number of (BETA) digits in the mantissa

 RND
   Specifies whether proper rounding (RND=.TRUE.) or chopping (RND=.FALSE.)
   occurs in addition. This may not be a reliable guide to the way in
   which the machine performs its arithmetic.

 EPS
   The smallest positive number such that
     fl(1.0-EPS) .LT. 1.0,
   where fl denotes the computed value.

 EMIN
   The minimum exponent before (gradual) underflow occurs.

 RMIN
   The smallest normalized number for the machine, given by
     BASE**(EMIN-1)
   where BASE is the floating point value of BETA

 EMAX
   The maximum exponent before overflow occurs

 RMAX
   The largest positive number for the machine, given by
     BASE**EMAX*(1-EPS)
   where BASE is the floating point value of BETA
```

———-

90.70.3 fortran code

— slamc2.f —

———-

90.70.4 lisp code

— slamc2.lisp —

———-

90.71 slamc3

90.71.1 Axiom unit tests

— slamc3.input —

```
)set break resume
)sys rm -f slamc3.output
)spool slamc3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.71.2 Axiom help page

— slamc3.help —

```
=====================================================================
slamc3 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

real function slamc3(real A, real B)

Purpose:
   intended to force A and B to be stored prior to doing the addition
   of A and B, for use in situations where optimizers might hold
   one of these in a register.

Parameters:

 A
 B
   The values A and B
```

———-

90.71.3 fortran code

— slamc3.f —

———-

90.71.4 lisp code

— slamc3.lisp —

———-

90.72 slamch

90.72.1 Axiom unit tests

— slamch.input —

```
)set break resume
)sys rm -f slamch.output
)spool slamch.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.72.2 Axiom help page

— slamch.help —

```
======================================================================
slamch examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       real function slamch (CMACH)
           SLAMCH

Purpose:
   SLAMCH determines single precision machine parameters.

Parameters:

 CMACH
   Specifies the value to be returned by SLAMCH:
   = 'E' or 'e',   SLAMCH := eps
   = 'S' or 's ,   SLAMCH := sfmin
   = 'B' or 'b',   SLAMCH := base
   = 'P' or 'p',   SLAMCH := eps*base
```

```
= 'N' or 'n',   SLAMCH := t
= 'R' or 'r',   SLAMCH := rnd
= 'M' or 'm',   SLAMCH := emin
= 'U' or 'u',   SLAMCH := rmin
= 'L' or 'l',   SLAMCH := emax
= 'O' or 'o',   SLAMCH := rmax
where
eps   = relative machine precision
sfmin = safe minimum, such that 1/sfmin does not overflow
base  = base of the machine
prec  = eps*base
t     = number of (base) digits in the mantissa
rnd   = 1.0 when rounding occurs in addition, 0.0 otherwise
emin  = minimum exponent before (gradual) underflow
rmin  = underflow threshold - base**(emin-1)
emax  = largest exponent before overflow
rmax  = overflow threshold  - (base**emax)*(1-eps)
```

———-

90.72.3 fortran code

— slamch.f —

———-

90.72.4 lisp code

— slamch.lisp —

———-

90.73 slaneg

90.73.1 Axiom unit tests

— slaneg.input —

```
)set break resume
)sys rm -f slaneg.output
)spool slaneg.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.73.2 Axiom help page

— **slaneg.help** —

```
======================================================================
slaneg examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       integer function slaneg (N, D, LLD, SIGMA, PIVMIN, R)
           SLANEG computes the Sturm count.

Function/Subroutine Documentation
   integer function slaneg (integer N, real, dimension( * ) D, real,
       dimension( * ) LLD, real SIGMA, real PIVMIN, integer R)
       SLANEG computes the Sturm count.

Purpose:
   SLANEG computes the Sturm count, the number of negative pivots
   encountered while factoring tridiagonal T - sigma I = L D L^T.
   This implementation works directly on the factors without forming
   the tridiagonal matrix T.  The Sturm count is also the number of
   eigenvalues of T less than sigma.

   This routine is called from SLARRB.

   The current routine does not use the PIVMIN parameter but rather
   requires IEEE-754 propagation of Infinities and NaNs.  This
   routine also has no input range restrictions but does require
   default exception handling such that x/0 produces Inf when x is
   non-zero, and Inf/Inf produces NaN.  For more information, see:
```

```
Parameters:

 N
   N is INTEGER
   The order of the matrix.

 D
   D is REAL array, dimension (N)
   The N diagonal elements of the diagonal matrix D.

 LLD
   LLD is REAL array, dimension (N-1)
   The (N-1) elements L(i)*L(i)*D(i).

 SIGMA
   SIGMA is REAL
   Shift amount in T - sigma I = L D L^T.

 PIVMIN
   PIVMIN is REAL
   The minimum pivot in the Sturm sequence.  May be used
   when zero pivots are encountered on non-IEEE-754 architectures.

 R
   R is INTEGER
   The twist index for the twisted factorization that is used for the negcount.
```

90.73.3 fortran code

— slaneg.f —

```
*  =====================================================================
      INTEGER FUNCTION SLANEG( N, D, LLD, SIGMA, PIVMIN, R )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            N, R
      REAL               PIVMIN, SIGMA
*     ..
*     .. Array Arguments ..
```

```
      REAL               D( * ), LLD( * )
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER        ( ZERO = 0.0E0, ONE = 1.0E0 )
*     Some architectures propagate Infinities and NaNs very slowly, so
*     the code computes counts in BLKLEN chunks.  Then a NaN can
*     propagate at most BLKLEN columns before being detected.  This is
*     not a general tuning parameter; it needs only to be just large
*     enough that the overhead is tiny in common cases.
      INTEGER BLKLEN
      PARAMETER ( BLKLEN = 128 )
*     ..
*     .. Local Scalars ..
      INTEGER            BJ, J, NEG1, NEG2, NEGCNT
      REAL               BSAV, DMINUS, DPLUS, GAMMA, P, T, TMP
      LOGICAL SAWNAN
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MIN, MAX
*     ..
*     .. External Functions ..
      LOGICAL SISNAN
      EXTERNAL SISNAN
*     ..
*     .. Executable Statements ..

      NEGCNT = 0

*     I) upper part: L D L^T - SIGMA I = L+ D+ L+^T
      T = -SIGMA
      DO 210 BJ = 1, R-1, BLKLEN
         NEG1 = 0
         BSAV = T
         DO 21 J = BJ, MIN(BJ+BLKLEN-1, R-1)
            DPLUS = D( J ) + T
            IF( DPLUS.LT.ZERO ) NEG1 = NEG1 + 1
            TMP = T / DPLUS
            T = TMP * LLD( J ) - SIGMA
 21      CONTINUE
         SAWNAN = SISNAN( T )
*     Run a slower version of the above loop if a NaN is detected.
*     A NaN should occur only with a zero pivot after an infinite
*     pivot.  In that case, substituting 1 for T/DPLUS is the
*     correct limit.
         IF( SAWNAN ) THEN
            NEG1 = 0
```

```
            T = BSAV
            DO 22 J = BJ, MIN(BJ+BLKLEN-1, R-1)
               DPLUS = D( J ) + T
               IF( DPLUS.LT.ZERO ) NEG1 = NEG1 + 1
               TMP = T / DPLUS
               IF (SISNAN(TMP)) TMP = ONE
               T = TMP * LLD(J) - SIGMA
 22         CONTINUE
         END IF
         NEGCNT = NEGCNT + NEG1
 210  CONTINUE
*
*     II) lower part: L D L^T - SIGMA I = U- D- U-^T
      P = D( N ) - SIGMA
      DO 230 BJ = N-1, R, -BLKLEN
         NEG2 = 0
         BSAV = P
         DO 23 J = BJ, MAX(BJ-BLKLEN+1, R), -1
            DMINUS = LLD( J ) + P
            IF( DMINUS.LT.ZERO ) NEG2 = NEG2 + 1
            TMP = P / DMINUS
            P = TMP * D( J ) - SIGMA
 23      CONTINUE
         SAWNAN = SISNAN( P )
*     As above, run a slower version that substitutes 1 for Inf/Inf.
*
         IF( SAWNAN ) THEN
            NEG2 = 0
            P = BSAV
            DO 24 J = BJ, MAX(BJ-BLKLEN+1, R), -1
               DMINUS = LLD( J ) + P
               IF( DMINUS.LT.ZERO ) NEG2 = NEG2 + 1
               TMP = P / DMINUS
               IF (SISNAN(TMP)) TMP = ONE
               P = TMP * D(J) - SIGMA
 24         CONTINUE
         END IF
         NEGCNT = NEGCNT + NEG2
 230  CONTINUE
*
*     III) Twist index
*       T was shifted by SIGMA initially.
      GAMMA = (T + SIGMA) + P
      IF( GAMMA.LT.ZERO ) NEGCNT = NEGCNT+1

      SLANEG = NEGCNT
      END

            _______
```

90.73.4 lisp code

— slaneg.lisp —

```
(let* ((zero 0.0) (one 1.0) (blklen 128))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (f2cl-lib:integer4 128 128) blklen)
           (ignorable zero one blklen))
  (defun slaneg (n d lld sigma pivmin r)
    (declare (type (single-float) pivmin sigma)
             (type (array single-float (*)) lld d)
             (type (f2cl-lib:integer4) r n))
    (f2cl-lib:with-multi-array-data
        ((d single-float d-%data% d-%offset%)
         (lld single-float lld-%data% lld-%offset%))
      (prog ((sawnan nil) (bsav 0.0) (dminus 0.0) (dplus 0.0) (gamma 0.0)
             (p 0.0) (t$ 0.0) (tmp 0.0) (bj 0) (j 0) (neg1 0) (neg2 0)
             (negcnt 0) (slaneg 0))
        (declare (type f2cl-lib:logical sawnan)
                 (type (single-float) bsav dminus dplus gamma p t$ tmp)
                 (type (f2cl-lib:integer4) bj j neg1 neg2 negcnt slaneg))
        (setf negcnt 0)
        (setf t$ (- sigma))
        (f2cl-lib:fdo (bj 1 (f2cl-lib:int-add bj blklen))
                      ((> bj (f2cl-lib:int-add r (f2cl-lib:int-sub 1))) nil)
          (tagbody
            (setf neg1 0)
            (setf bsav t$)
            (f2cl-lib:fdo (j bj (f2cl-lib:int-add j 1))
                          ((> j
                              (min
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add bj blklen
                                                      (f2cl-lib:int-sub 1)))
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add r
                                                      (f2cl-lib:int-sub 1)))))
                           nil)
              (tagbody
                (setf dplus
                        (+ (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%) t$))
                (if (< dplus zero)
                    (setf neg1 (f2cl-lib:int-add neg1 1)))
                (setf tmp (/ t$ dplus))
                (setf t$
                        (-
                         (* tmp
                            (f2cl-lib:fref lld-%data% (j) ((1 *))
```

```
                                        lld-%offset%))
                    sigma))
            label21))
        (setf sawnan (sisnan t$))
        (cond
         (sawnan (setf neg1 0) (setf t$ bsav)
          (f2cl-lib:fdo (j bj (f2cl-lib:int-add j 1))
                        ((> j
                            (min
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add bj blklen
                                                    (f2cl-lib:int-sub 1)))
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add r
                                                    (f2cl-lib:int-sub
                                                     1)))))
                         nil)
            (tagbody
              (setf dplus
                      (+ (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                         t$))
              (if (< dplus zero)
                  (setf neg1 (f2cl-lib:int-add neg1 1)))
              (setf tmp (/ t$ dplus))
              (if (sisnan tmp)
                  (setf tmp one))
              (setf t$
                      (-
                       (* tmp
                          (f2cl-lib:fref lld-%data% (j) ((1 *))
                                         lld-%offset%))
                       sigma))
             label22))))
        (setf negcnt (f2cl-lib:int-add negcnt neg1))
       label210))
    (setf p (- (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) sigma))
    (f2cl-lib:fdo (bj (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add bj (f2cl-lib:int-sub blklen)))
                  ((> bj r) nil)
      (tagbody
        (setf neg2 0)
        (setf bsav p)
        (f2cl-lib:fdo (j bj (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                      ((> j
                          (max
                           (the f2cl-lib:integer4
                                (f2cl-lib:int-add bj
                                                  (f2cl-lib:int-sub blklen)
                                                  1))
                           (the f2cl-lib:integer4 r)))
```

```
                    nil)
          (tagbody
            (setf dminus
                    (+ (f2cl-lib:fref lld-%data% (j) ((1 *)) lld-%offset%)
                       p))
            (if (< dminus zero)
                (setf neg2 (f2cl-lib:int-add neg2 1)))
            (setf tmp (/ p dminus))
            (setf p
                    (-
                     (* tmp
                        (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%))
                     sigma))
           label23))
        (setf sawnan (sisnan p))
        (cond
         (sawnan (setf neg2 0) (setf p bsav)
          (f2cl-lib:fdo (j bj (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                        ((> j
                            (max
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add bj
                                                    (f2cl-lib:int-sub
                                                     blklen)
                                                    1))
                             (the f2cl-lib:integer4 r)))
                         nil)
            (tagbody
              (setf dminus
                      (+
                       (f2cl-lib:fref lld-%data% (j) ((1 *)) lld-%offset%)
                       p))
              (if (< dminus zero)
                  (setf neg2 (f2cl-lib:int-add neg2 1)))
              (setf tmp (/ p dminus))
              (if (sisnan tmp)
                  (setf tmp one))
              (setf p
                      (-
                       (* tmp
                          (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%))
                       sigma))
             label24))))
        (setf negcnt (f2cl-lib:int-add negcnt neg2))
       label230))
    (setf gamma (+ t$ sigma p))
    (if (< gamma zero)
        (setf negcnt (f2cl-lib:int-add negcnt 1)))
    (setf slaneg negcnt)
   end_label
```

```
        (return (values slaneg nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slaneg
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil) :calls
                                            '(fortran-to-lisp::sisnan))))
```

———-

90.74 slanst

90.74.1 Axiom unit tests

— **slanst.input** —

```
)set break resume
)sys rm -f slanst.output
)spool slanst.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.74.2 Axiom help page

— **slanst.help** —

```
==========================================================================
```

```
slanst examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       real function slanst (NORM, N, D, E)
           SLANST returns the value of the 1-norm, or the Frobenius norm, or
           the infinity norm, or the element of largest absolute value of a
           real symmetric tridiagonal matrix.

Function/Subroutine Documentation
   real function slanst (character NORM, integer N, real, dimension( * ) D,
       real, dimension( * ) E)
       SLANST returns the value of the 1-norm, or the Frobenius norm, or the
       infinity norm, or the element of largest absolute value of a real
       symmetric tridiagonal matrix.

Purpose:
   SLANST  returns the value of the one norm,  or the Frobenius norm, or
   the  infinity norm,  or the  element of  largest absolute value  of a
   real symmetric tridiagonal matrix A.

 Returns:

    SLANST
       SLANST = ( max(abs(A(i,j))), NORM = 'M' or 'm'
                (
                ( norm1(A),         NORM = '1', 'O' or 'o'
                (
                ( normI(A),         NORM = 'I' or 'i'
                (
                ( normF(A),         NORM = 'F', 'f', 'E' or 'e'

   where  norm1  denotes the  one norm of a matrix (maximum column sum),
   normI  denotes the  infinity norm  of a matrix  (maximum row sum) and
   normF  denotes the  Frobenius norm of a matrix (square root of sum of
   squares).  Note that  max(abs(A(i,j)))  is not a consistent matrix norm.

Parameters:

 NORM
   NORM is CHARACTER*1
   Specifies the value to be returned in SLANST as described above.

 N
   N is INTEGER
```

```
  The order of the matrix A.  N >= 0.  When N = 0, SLANST is set to zero.

 D
  D is REAL array, dimension (N)
  The diagonal elements of A.

 E
  E is REAL array, dimension (N-1)
  The (n-1) sub-diagonal or super-diagonal elements of A.
```

90.74.3 fortran code

— slanst.f —

```
*  =====================================================================
      REAL             FUNCTION SLANST( NORM, N, D, E )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            N
*     ..
*     .. Array Arguments ..
      REAL               D( * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      REAL               ANORM, SCALE, SUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, SISNAN
      EXTERNAL           LSAME, SISNAN
*     ..
*     .. External Subroutines ..
```

```
      EXTERNAL           SLASSQ
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SQRT
*     ..
*     .. Executable Statements ..
*
      IF( N.LE.0 ) THEN
         ANORM = ZERO
      ELSE IF( LSAME( NORM, 'M' ) ) THEN
*
*        Find max(abs(A(i,j))).
*
         ANORM = ABS( D( N ) )
         DO 10 I = 1, N - 1
            SUM = ABS( D( I ) )
            IF( ANORM .LT. SUM .OR. SISNAN( SUM ) ) ANORM = SUM
            SUM = ABS( E( I ) )
            IF( ANORM .LT. SUM .OR. SISNAN( SUM ) ) ANORM = SUM
   10    CONTINUE
      ELSE IF( LSAME( NORM, 'O' ) .OR. NORM.EQ.'1' .OR.
     $         LSAME( NORM, 'I' ) ) THEN
*
*        Find norm1(A).
*
         IF( N.EQ.1 ) THEN
            ANORM = ABS( D( 1 ) )
         ELSE
            ANORM = ABS( D( 1 ) )+ABS( E( 1 ) )
            SUM = ABS( E( N-1 ) )+ABS( D( N ) )
            IF( ANORM .LT. SUM .OR. SISNAN( SUM ) ) ANORM = SUM
            DO 20 I = 2, N - 1
               SUM = ABS( D( I ) )+ABS( E( I ) )+ABS( E( I-1 ) )
               IF( ANORM .LT. SUM .OR. SISNAN( SUM ) ) ANORM = SUM
   20       CONTINUE
         END IF
      ELSE IF( ( LSAME( NORM, 'F' ) ) .OR. ( LSAME( NORM, 'E' ) ) ) THEN
*
*        Find normF(A).
*
         SCALE = ZERO
         SUM = ONE
         IF( N.GT.1 ) THEN
            CALL SLASSQ( N-1, E, 1, SCALE, SUM )
            SUM = 2*SUM
         END IF
         CALL SLASSQ( N, D, 1, SCALE, SUM )
         ANORM = SCALE*SQRT( SUM )
      END IF
*
```

```
      SLANST = ANORM
      RETURN
*
*     End of SLANST
*
      END
```

———-

90.74.4 lisp code

— **slanst.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun slanst (norm n d e)
    (declare (type (array single-float (*)) e d)
             (type (f2cl-lib:integer4) n)
             (type (string *) norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%))
      (prog ((anorm 0.0) (scale 0.0) (sum 0.0) (i 0) (slanst 0.0))
        (declare (type (f2cl-lib:integer4) i)
                 (type (single-float) anorm scale sum slanst))
        (cond ((<= n 0) (setf anorm zero))
              ((multiple-value-bind (ret-val var-0 var-1)
                   (lsame norm "M")
                 (declare (ignore var-1))
                 (when var-0 (setf norm var-0))
                 ret-val)
               (setf anorm
                       (abs (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                              nil)
                 (tagbody
                   (setf sum
                           (abs
                            (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
                   (if (or (< anorm sum) (sisnan sum))
                       (setf anorm sum))
                   (setf sum
                           (abs
                            (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
```

```
      (if (or (< anorm sum) (sisnan sum))
          (setf anorm sum))
    label10)))
((or
  (multiple-value-bind (ret-val var-0 var-1)
      (lsame norm "O")
    (declare (ignore var-1))
    (when var-0 (setf norm var-0))
    ret-val)
  (f2cl-lib:fstring-= norm "1")
  (multiple-value-bind (ret-val var-0 var-1)
      (lsame norm "I")
    (declare (ignore var-1))
    (when var-0 (setf norm var-0))
    ret-val))
 (cond
  ((= n 1)
   (setf anorm
           (abs
            (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))))
  (t
   (setf anorm
           (+
            (abs (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
            (abs
             (f2cl-lib:fref e-%data% (1) ((1 *)) e-%offset%))))
   (setf sum
           (+
            (abs
             (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub n 1))
                            ((1 *)) e-%offset%))
            (abs
             (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%))))
   (if (or (< anorm sum) (sisnan sum))
       (setf anorm sum))
   (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                 ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                  nil)
     (tagbody
       (setf sum
               (+
                (abs
                 (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
                (abs
                 (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))
                (abs
                 (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                ((1 *)) e-%offset%))))
       (if (or (< anorm sum) (sisnan sum))
           (setf anorm sum))
```

```
                 label20)))))
            ((or
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame norm "F")
                (declare (ignore var-1))
                (when var-0 (setf norm var-0))
                ret-val)
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame norm "E")
                (declare (ignore var-1))
                (when var-0 (setf norm var-0))
                ret-val))
             (setf scale zero) (setf sum one)
             (cond
              ((> n 1)
               (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                   (slassq (f2cl-lib:int-sub n 1) e 1 scale sum)
                 (declare (ignore var-0 var-1 var-2))
                 (when var-3 (setf scale var-3))
                 (when var-4 (setf sum var-4)))
               (setf sum (* 2 sum))))
             (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                 (slassq n d 1 scale sum)
               (declare (ignore var-1 var-2))
               (when var-0 (setf n var-0))
               (when var-3 (setf scale var-3))
               (when var-4 (setf sum var-4)))
             (setf anorm (* scale (f2cl-lib:fsqrt sum)))))
        (setf slanst anorm)
        (go end_label)
       end_label
        (return (values slanst norm n nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slanst
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*)))
                                            :return-values
                                            '(fortran-to-lisp::norm
                                              fortran-to-lisp::n nil nil)
                                            :calls '(fortran-to-lisp::sisnan))))
```

———-

90.75 slapy2

90.75.1 Axiom unit tests

— **slapy2.input** —

```
)set break resume
)sys rm -f slapy2.output
)spool slapy2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.75.2 Axiom help page

— **slapy2.help** —

```
==================================================================
slapy2 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       real function slapy2 (X, Y)
           SLAPY2 returns sqrt(x2+y2).

Function/Subroutine Documentation
   real function slapy2 (real X, real Y)
       SLAPY2 returns sqrt(x2+y2).

Purpose:
   SLAPY2 returns sqrt(x**2+y**2), taking care not to cause unnecessary
   overflow.
```

```
Parameters:

 X
   X is REAL

 Y
   Y is REAL
   X and Y specify the values x and y.
```

90.75.3 fortran code

— slapy2.f —

```
*  =====================================================================
      REAL             FUNCTION SLAPY2( X, Y )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      REAL               X, Y
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E0 )
      REAL               ONE
      PARAMETER          ( ONE = 1.0E0 )
*     ..
*     .. Local Scalars ..
      REAL               W, XABS, YABS, Z
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      XABS = ABS( X )
      YABS = ABS( Y )
      W = MAX( XABS, YABS )
```

```
      Z = MIN( XABS, YABS )
      IF( Z.EQ.ZERO ) THEN
         SLAPY2 = W
      ELSE
         SLAPY2 = W*SQRT( ONE+( Z / W )**2 )
      END IF
      RETURN
*
*     End of SLAPY2
*
      END
```

———-

90.75.4 lisp code

— slapy2.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun slapy2 (x y)
    (declare (type (single-float) y x))
    (prog ((w 0.0) (xabs 0.0) (yabs 0.0) (z 0.0) (slapy2 0.0))
      (declare (type (single-float) w xabs yabs z slapy2))
      (setf xabs (abs x))
      (setf yabs (abs y))
      (setf w (max xabs yabs))
      (setf z (min xabs yabs))
      (cond ((= z zero) (setf slapy2 w))
            (t (setf slapy2 (* w (f2cl-lib:fsqrt (+ one (expt (/ z w) 2)))))))
      (go end_label)
     end_label
      (return (values slapy2 nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slapy2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((single-float) (single-float))
                                            :return-values '(nil nil) :calls
                                            'nil)))
```

———-

90.76 slapy3

90.76.1 Axiom unit tests

— slapy3.input —

```
)set break resume
)sys rm -f slapy3.output
)spool slapy3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.76.2 Axiom help page

— slapy3.help —

```
======================================================================
slapy3 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       real function slapy3 (X, Y, Z)
           SLAPY3 returns sqrt(x2+y2+z2).

Function/Subroutine Documentation
   real function slapy3 (real X, real Y, real Z)
       SLAPY3 returns sqrt(x2+y2+z2).

Purpose:
   SLAPY3 returns sqrt(x**2+y**2+z**2), taking care not to cause
   unnecessary overflow.
```

```
Parameters:

 X
   X is REAL

 Y
   Y is REAL

 Z
   Z is REAL
   X, Y and Z specify the values x, y and z.
```

———-

90.76.3 fortran code

— **slapy3.f** —

```
*  =====================================================================
      REAL             FUNCTION SLAPY3( X, Y, Z )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      REAL               X, Y, Z
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E0 )
*     ..
*     .. Local Scalars ..
      REAL               W, XABS, YABS, ZABS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
      XABS = ABS( X )
      YABS = ABS( Y )
```

```
      ZABS = ABS( Z )
      W = MAX( XABS, YABS, ZABS )
      IF( W.EQ.ZERO ) THEN
*     W can be zero for max(0,nan,0)
*     adding all three entries together will make sure
*     NaN will not disappear.
         SLAPY3 =  XABS + YABS + ZABS
      ELSE
         SLAPY3 = W*SQRT( ( XABS / W )**2+( YABS / W )**2+
     $            ( ZABS / W )**2 )
      END IF
      RETURN
*
*     End of SLAPY3
*
      END
```

———-

90.76.4 lisp code

— slapy3.lisp —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun slapy3 (x y z)
    (declare (type (single-float) z y x))
    (prog ((w 0.0) (xabs 0.0) (yabs 0.0) (zabs 0.0) (slapy3 0.0))
      (declare (type (single-float) w xabs yabs zabs slapy3))
      (setf xabs (abs x))
      (setf yabs (abs y))
      (setf zabs (abs z))
      (setf w (max xabs yabs zabs))
      (cond ((= w zero) (setf slapy3 (+ xabs yabs zabs)))
            (t
             (setf slapy3
                     (* w
                        (f2cl-lib:fsqrt
                         (+ (expt (/ xabs w) 2) (expt (/ yabs w) 2)
                            (expt (/ zabs w) 2)))))))
      (go end_label)
     end_label
      (return (values slapy3 nil nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
```

```
  (setf (gethash 'fortran-to-lisp::slapy3
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((single-float) (single-float)
                                              (single-float))
                                            :return-values '(nil nil nil)
                                            :calls 'nil)))
```

———-

90.77 slarnv

90.77.1 Axiom unit tests

— **slarnv.input** —

```
)set break resume
)sys rm -f slarnv.output
)spool slarnv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.77.2 Axiom help page

— **slarnv.help** —

```
======================================================================
slarnv examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
```

```
       subroutine slarnv (IDIST, ISEED, N, X)
           SLARNV returns a vector of random numbers from a uniform or normal
           distribution.

Function/Subroutine Documentation
   subroutine slarnv (integer IDIST, integer, dimension( 4 ) ISEED, integer N,
       real, dimension( * ) X)
       SLARNV returns a vector of random numbers from a uniform or normal
       distribution.

Purpose:
   SLARNV returns a vector of n random real numbers from a uniform or
   normal distribution.

Parameters:

 IDIST
   IDIST is INTEGER
   Specifies the distribution of the random numbers:
   = 1:  uniform (0,1)
   = 2:  uniform (-1,1)
   = 3:  normal (0,1)

 ISEED
   ISEED is INTEGER array, dimension (4)
   On entry, the seed of the random number generator; the array
   elements must be between 0 and 4095, and ISEED(4) must be odd.
   On exit, the seed is updated.

 N
   N is INTEGER
   The number of random numbers to be generated.

 X
   X is REAL array, dimension (N)
   The generated random numbers.

Further Details:

   This routine calls the auxiliary routine SLARUV to generate random
   real numbers from a uniform (0,1) distribution, in batches of up to
   128 using vectorisable code. The Box-Muller method is used to
   transform numbers from a uniform to a normal distribution.
```

———-

90.77.3 fortran code

— slarnv.f —

```
*  =====================================================================
      SUBROUTINE SLARNV( IDIST, ISEED, N, X )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            IDIST, N
*     ..
*     .. Array Arguments ..
      INTEGER            ISEED( 4 )
      REAL               X( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, TWO
      PARAMETER          ( ONE = 1.0E+0, TWO = 2.0E+0 )
      INTEGER            LV
      PARAMETER          ( LV = 128 )
      REAL               TWOPI
      PARAMETER          ( TWOPI = 6.2831853071795864769252867663E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IL, IL2, IV
*     ..
*     .. Local Arrays ..
      REAL               U( LV )
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          COS, LOG, MIN, SQRT
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLARUV
*     ..
*     .. Executable Statements ..
*
      DO 40 IV = 1, N, LV / 2
         IL = MIN( LV / 2, N-IV+1 )
         IF( IDIST.EQ.3 ) THEN
            IL2 = 2*IL
         ELSE
```

```
            IL2 = IL
         END IF
*
*        Call SLARUV to generate IL2 numbers from a uniform (0,1)
*        distribution (IL2 <= LV)
*
         CALL SLARUV( ISEED, IL2, U )
*
         IF( IDIST.EQ.1 ) THEN
*
*           Copy generated numbers
*
            DO 10 I = 1, IL
               X( IV+I-1 ) = U( I )
   10       CONTINUE
         ELSE IF( IDIST.EQ.2 ) THEN
*
*           Convert generated numbers to uniform (-1,1) distribution
*
            DO 20 I = 1, IL
               X( IV+I-1 ) = TWO*U( I ) - ONE
   20       CONTINUE
         ELSE IF( IDIST.EQ.3 ) THEN
*
*           Convert generated numbers to normal (0,1) distribution
*
            DO 30 I = 1, IL
               X( IV+I-1 ) = SQRT( -TWO*LOG( U( 2*I-1 ) ) )*
     $                       COS( TWOPI*U( 2*I ) )
   30       CONTINUE
         END IF
   40 CONTINUE
      RETURN
*
*     End of SLARNV
*
      END
```

———-

90.77.4 lisp code

— slarnv.lisp —

```
(let* ((one 1.0) (two 2.0) (lv 128) (twopi 6.2831855))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (f2cl-lib:integer4 128 128) lv)
```

```
          (type (single-float 6.2831855 6.2831855) twopi)
          (ignorable one two lv twopi))
 (defun slarnv (idist iseed n x)
   (declare (type (array single-float (*)) x)
            (type (array f2cl-lib:integer4 (*)) iseed)
            (type (f2cl-lib:integer4) n idist))
   (f2cl-lib:with-multi-array-data
       ((iseed f2cl-lib:integer4 iseed-%data% iseed-%offset%)
        (x single-float x-%data% x-%offset%))
     (prog ((u (make-array lv :element-type 'single-float)) (i 0) (il 0)
            (il2 0) (iv 0))
       (declare (type (array single-float (*)) u)
                (type (f2cl-lib:integer4) i il il2 iv))
       (f2cl-lib:fdo (iv 1 (+ iv (the f2cl-lib:integer4 (truncate lv 2))))
                     ((> iv n) nil)
         (tagbody
           (setf il
                   (min (the f2cl-lib:integer4 (truncate lv 2))
                        (f2cl-lib:int-add (f2cl-lib:int-sub n iv) 1)))
           (cond ((= idist 3) (setf il2 (f2cl-lib:int-mul 2 il)))
                 (t (setf il2 il)))
           (multiple-value-bind (var-0 var-1 var-2)
               (slaruv iseed il2 u)
             (declare (ignore var-0 var-2))
             (when var-1 (setf il2 var-1)))
           (cond
            ((= idist 1)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i il) nil)
               (tagbody
                 (setf (f2cl-lib:fref x-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add iv i) 1))
                                      ((1 *)) x-%offset%)
                       (f2cl-lib:fref u (i) ((1 lv))))
               label10)))
            ((= idist 2)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i il) nil)
               (tagbody
                 (setf (f2cl-lib:fref x-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add iv i) 1))
                                      ((1 *)) x-%offset%)
                       (- (* two (f2cl-lib:fref u (i) ((1 lv)))) one))
               label20)))
            ((= idist 3)
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i il) nil)
               (tagbody
```

```
                 (setf (f2cl-lib:fref x-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-add iv i) 1))
                                      ((1 *)) x-%offset%)
                         (*
                          (f2cl-lib:fsqrt
                           (* (- two)
                              (f2cl-lib:flog
                               (f2cl-lib:fref u
                                              ((f2cl-lib:int-sub
                                                (f2cl-lib:int-mul 2 i) 1))
                                              ((1 lv))))))
                          (cos
                           (* twopi
                              (f2cl-lib:fref u ((f2cl-lib:int-mul 2 i))
                                             ((1 lv)))))))
                 label30))))
           label40))
        (go end_label)
       end_label
        (return (values nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slarnv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*)))
                                            :return-values '(nil nil nil nil)
                                            :calls 'nil)))
```

———-

90.78 slarra

90.78.1 Axiom unit tests

— slarra.input —

```
)set break resume
```

```
)sys rm -f slarra.output
)spool slarra.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.78.2 Axiom help page

— slarra.help —

```
======================================================================
slarra examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slarra (N, D, E, E2, SPLTOL, TNRM, NSPLIT, ISPLIT, INFO)
           SLARRA computes the splitting points with the specified threshold.

Function/Subroutine Documentation
   subroutine slarra (integer N, real, dimension( * ) D, real, dimension( * )
       E, real, dimension( * ) E2, real SPLTOL, real TNRM, integer NSPLIT,
       integer, dimension( * ) ISPLIT, integer INFO)
       SLARRA computes the splitting points with the specified threshold.

Purpose:
   Compute the splitting points with threshold SPLTOL.
   SLARRA sets any "small" off-diagonal elements to zero.

Parameters:

 N
   N is INTEGER
   The order of the matrix. N > 0.

 D
   D is REAL array, dimension (N)
   On entry, the N diagonal elements of the tridiagonal matrix T.
```

```
E
  E is REAL array, dimension (N)
  On entry, the first (N-1) entries contain the subdiagonal
  elements of the tridiagonal matrix T; E(N) need not be set.
  On exit, the entries E( ISPLIT( I ) ), 1 <= I <= NSPLIT,
  are set to zero, the other entries of E are untouched.

E2
  E2 is REAL array, dimension (N)
  On entry, the first (N-1) entries contain the SQUARES of the
  subdiagonal elements of the tridiagonal matrix T;
  E2(N) need not be set.
  On exit, the entries E2( ISPLIT( I ) ),
  1 <= I <= NSPLIT, have been set to zero

SPLTOL
  SPLTOL is REAL
  The threshold for splitting. Two criteria can be used:
  SPLTOL<0 : criterion based on absolute off-diagonal value
  SPLTOL>0 : criterion that preserves relative accuracy

TNRM
  TNRM is REAL
  The norm of the matrix.

NSPLIT
  NSPLIT is INTEGER
  The number of blocks T splits into. 1 <= NSPLIT <= N.

ISPLIT
  ISPLIT is INTEGER array, dimension (N)
  The splitting points, at which T breaks up into blocks.
  The first block consists of rows/columns 1 to ISPLIT(1),
  the second of rows/columns ISPLIT(1)+1 through ISPLIT(2),
  etc., and the NSPLIT-th consists of rows/columns
  ISPLIT(NSPLIT-1)+1 through ISPLIT(NSPLIT)=N.

INFO
  INFO is INTEGER
  = 0:  successful exit
```

———-

90.78.3 fortran code

— slarra.f —

```
*  =====================================================================
      SUBROUTINE SLARRA( N, D, E, E2, SPLTOL, TNRM,
     $                   NSPLIT, ISPLIT, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, N, NSPLIT
      REAL               SPLTOL, TNRM
*     ..
*     .. Array Arguments ..
      INTEGER            ISPLIT( * )
      REAL               D( * ), E( * ), E2( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      REAL               EABS, TMP1

*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS
*     ..
*     .. Executable Statements ..
*
      INFO = 0

*     Compute splitting points
      NSPLIT = 1
      IF(SPLTOL.LT.ZERO) THEN
*        Criterion based on absolute off-diagonal value
         TMP1 = ABS(SPLTOL)* TNRM
         DO 9 I = 1, N-1
            EABS = ABS( E(I) )
            IF( EABS .LE. TMP1) THEN
               E(I) = ZERO
               E2(I) = ZERO
               ISPLIT( NSPLIT ) = I
               NSPLIT = NSPLIT + 1
            END IF
 9       CONTINUE
```

```
      ELSE
*        Criterion that guarantees relative accuracy
         DO 10 I = 1, N-1
            EABS = ABS( E(I) )
            IF( EABS .LE. SPLTOL * SQRT(ABS(D(I)))*SQRT(ABS(D(I+1))) )
     $      THEN
               E(I) = ZERO
               E2(I) = ZERO
               ISPLIT( NSPLIT ) = I
               NSPLIT = NSPLIT + 1
            END IF
 10      CONTINUE
      ENDIF
      ISPLIT( NSPLIT ) = N

      RETURN
*
*     End of SLARRA
*
      END
```

———-

90.78.4 lisp code

— **slarra.lisp** —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun slarra (n d e e2 spltol tnrm nsplit isplit info)
    (declare (type (array f2cl-lib:integer4 (*)) isplit)
             (type (single-float) tnrm spltol)
             (type (array single-float (*)) e2 e d)
             (type (f2cl-lib:integer4) info nsplit n))
    (f2cl-lib:with-multi-array-data
        ((d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%)
         (e2 single-float e2-%data% e2-%offset%)
         (isplit f2cl-lib:integer4 isplit-%data% isplit-%offset%))
      (prog ((eabs 0.0) (tmp1 0.0) (i 0))
        (declare (type (single-float) eabs tmp1)
                 (type (f2cl-lib:integer4) i))
        (setf info 0)
        (setf nsplit 1)
        (cond
         ((< spltol zero) (setf tmp1 (* (abs spltol) tnrm))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                    ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
          (tagbody
            (setf eabs (abs (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
            (cond
             ((<= eabs tmp1)
              (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) zero)
              (setf (f2cl-lib:fref e2-%data% (i) ((1 *)) e2-%offset%) zero)
              (setf (f2cl-lib:fref isplit-%data% (nsplit) ((1 *))
                                   isplit-%offset%)
                    i)
              (setf nsplit (f2cl-lib:int-add nsplit 1))))
           label9)))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
          (tagbody
            (setf eabs (abs (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
            (cond
             ((<= eabs
                  (* spltol
                     (f2cl-lib:fsqrt (abs (f2cl-lib:fref d (i) ((1 *)))))
                     (f2cl-lib:fsqrt
                      (abs
                       (f2cl-lib:fref d ((f2cl-lib:int-add i 1)) ((1 *)))))))
              (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) zero)
              (setf (f2cl-lib:fref e2-%data% (i) ((1 *)) e2-%offset%) zero)
              (setf (f2cl-lib:fref isplit-%data% (nsplit) ((1 *))
                                   isplit-%offset%)
                    i)
              (setf nsplit (f2cl-lib:int-add nsplit 1))))
           label10))))
        (setf (f2cl-lib:fref isplit-%data% (nsplit) ((1 *)) isplit-%offset%) n)
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil nsplit nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slarra
                 fortran-to-lisp::*f2cl-function-info*)
         (fortran-to-lisp::make-f2cl-finfo :arg-types
                                           '((fortran-to-lisp::integer4)
                                             (array single-float (*))
                                             (array single-float (*))
                                             (array single-float (*))
                                             (single-float) (single-float)
                                             (fortran-to-lisp::integer4)
                                             (array fortran-to-lisp::integer4
                                              (*))
```

```
                                                  (fortran-to-lisp::integer4))
                                                :return-values
                                                '(nil nil nil nil nil nil
                                                  fortran-to-lisp::nsplit nil
                                                  fortran-to-lisp::info)
                                                :calls 'nil)))
```

———-

90.79 slarrb

90.79.1 Axiom unit tests

— **slarrb.input** —

```
)set break resume
)sys rm -f slarrb.output
)spool slarrb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.79.2 Axiom help page

— **slarrb.help** —

```
====================================================================
slarrb examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slarrb (N, D, LLD, IFIRST, ILAST, RTOL1, RTOL2, OFFSET, W,
```

```
          WGAP, WERR, WORK, IWORK, PIVMIN, SPDIAM, TWIST, INFO)
          SLARRB provides limited bisection to locate eigenvalues for more
          accuracy.

Function/Subroutine Documentation
   subroutine slarrb (integer N, real, dimension( * ) D, real, dimension( * )
       LLD, integer IFIRST, integer ILAST, real RTOL1, real RTOL2, integer
       OFFSET, real, dimension( * ) W, real, dimension( * ) WGAP, real,
       dimension( * ) WERR, real, dimension( * ) WORK, integer, dimension( * )
       IWORK, real PIVMIN, real SPDIAM, integer TWIST, integer INFO)
       SLARRB provides limited bisection to locate eigenvalues for more
       accuracy.

Purpose:
   Given the relatively robust representation(RRR) L D L^T, SLARRB
   does "limited" bisection to refine the eigenvalues of L D L^T,
   W( IFIRST-OFFSET ) through W( ILAST-OFFSET ), to more accuracy. Initial
   guesses for these eigenvalues are input in W, the corresponding estimate
   of the error in these guesses and their gaps are input in WERR
   and WGAP, respectively. During bisection, intervals
   [left, right] are maintained by storing their mid-points and
   semi-widths in the arrays W and WERR respectively.

Parameters:

 N
   N is INTEGER
   The order of the matrix.

 D
   D is REAL array, dimension (N)
   The N diagonal elements of the diagonal matrix D.

 LLD
   LLD is REAL array, dimension (N-1)
   The (N-1) elements L(i)*L(i)*D(i).

 IFIRST
   IFIRST is INTEGER
   The index of the first eigenvalue to be computed.

 ILAST
   ILAST is INTEGER
   The index of the last eigenvalue to be computed.

 RTOL1
   RTOL1 is REAL

 RTOL2
   RTOL2 is REAL
```

```
  Tolerance for the convergence of the bisection intervals.
  An interval [LEFT,RIGHT] has converged if
  RIGHT-LEFT.LT.MAX( RTOL1*GAP, RTOL2*MAX(|LEFT|,|RIGHT|) )
  where GAP is the (estimated) distance to the nearest eigenvalue.

OFFSET
  OFFSET is INTEGER
  Offset for the arrays W, WGAP and WERR, i.e., the IFIRST-OFFSET
  through ILAST-OFFSET elements of these arrays are to be used.

W
  W is REAL array, dimension (N)
  On input, W( IFIRST-OFFSET ) through W( ILAST-OFFSET ) are
  estimates of the eigenvalues of L D L^T indexed IFIRST through ILAST.
  On output, these estimates are refined.

WGAP
  WGAP is REAL array, dimension (N-1)
  On input, the (estimated) gaps between consecutive
  eigenvalues of L D L^T, i.e., WGAP(I-OFFSET) is the gap between
  eigenvalues I and I+1. Note that if IFIRST.EQ.ILAST
  then WGAP(IFIRST-OFFSET) must be set to ZERO.
  On output, these gaps are refined.

WERR
  WERR is REAL array, dimension (N)
  On input, WERR( IFIRST-OFFSET ) through WERR( ILAST-OFFSET ) are
  the errors in the estimates of the corresponding elements in W.
  On output, these errors are refined.

WORK
  WORK is REAL array, dimension (2*N)
  Workspace.

IWORK
  IWORK is INTEGER array, dimension (2*N)
  Workspace.

PIVMIN
  PIVMIN is REAL
  The minimum pivot in the Sturm sequence.

SPDIAM
  SPDIAM is REAL
  The spectral diameter of the matrix.

TWIST
  TWIST is INTEGER
  The twist index for the twisted factorization that is used
  for the negcount.
```

```
  TWIST = N: Compute negcount from L D L^T - LAMBDA I = L+ D+ L+^T
  TWIST = 1: Compute negcount from L D L^T - LAMBDA I = U- D- U-^T
  TWIST = R: Compute negcount from L D L^T - LAMBDA I = N(r) D(r) N(r)

 INFO
  INFO is INTEGER
  Error flag.
```

———-

90.79.3 fortran code

— **slarrb.f** —

```
*  =====================================================================
      SUBROUTINE SLARRB( N, D, LLD, IFIRST, ILAST, RTOL1,
     $                   RTOL2, OFFSET, W, WGAP, WERR, WORK, IWORK,
     $                   PIVMIN, SPDIAM, TWIST, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            IFIRST, ILAST, INFO, N, OFFSET, TWIST
      REAL               PIVMIN, RTOL1, RTOL2, SPDIAM
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      REAL               D( * ), LLD( * ), W( * ),
     $                   WERR( * ), WGAP( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, TWO, HALF
      PARAMETER        ( ZERO = 0.0E0, TWO = 2.0E0,
     $                   HALF = 0.5E0 )
      INTEGER   MAXITR
*     ..
*     .. Local Scalars ..
      INTEGER            I, I1, II, IP, ITER, K, NEGCNT, NEXT, NINT,
     $                   OLNINT, PREV, R
      REAL               BACK, CVRGD, GAP, LEFT, LGAP, MID, MNWDTH,
     $                   RGAP, RIGHT, TMP, WIDTH
```

```
*     ..
*     .. External Functions ..
      INTEGER            SLANEG
      EXTERNAL           SLANEG
*
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
      MAXITR = INT( ( LOG( SPDIAM+PIVMIN )-LOG( PIVMIN ) ) /
     $           LOG( TWO ) ) + 2
      MNWDTH = TWO * PIVMIN
*
      R = TWIST
      IF((R.LT.1).OR.(R.GT.N)) R = N
*
*     Initialize unconverged intervals in [ WORK(2*I-1), WORK(2*I) ].
*     The Sturm Count, Count( WORK(2*I-1) ) is arranged to be I-1, while
*     Count( WORK(2*I) ) is stored in IWORK( 2*I ). The integer IWORK( 2*I-1 )
*     for an unconverged interval is set to the index of the next unconverged
*     interval, and is -1 or 0 for a converged interval. Thus a linked
*     list of unconverged intervals is set up.
*
      I1 = IFIRST
*     The number of unconverged intervals
      NINT = 0
*     The last unconverged interval found
      PREV = 0

      RGAP = WGAP( I1-OFFSET )
      DO 75 I = I1, ILAST
         K = 2*I
         II = I - OFFSET
         LEFT = W( II ) - WERR( II )
         RIGHT = W( II ) + WERR( II )
         LGAP = RGAP
         RGAP = WGAP( II )
         GAP = MIN( LGAP, RGAP )

*        Make sure that [LEFT,RIGHT] contains the desired eigenvalue
*        Compute negcount from dstqds facto L+D+L+^T = L D L^T - LEFT
*
*        Do while( NEGCNT(LEFT).GT.I-1 )
*
         BACK = WERR( II )
 20      CONTINUE
```

```
         NEGCNT = SLANEG( N, D, LLD, LEFT, PIVMIN, R )
         IF( NEGCNT.GT.I-1 ) THEN
            LEFT = LEFT - BACK
            BACK = TWO*BACK
            GO TO 20
         END IF
*
*        Do while( NEGCNT(RIGHT).LT.I )
*        Compute negcount from dstqds facto L+D+L+^T = L D L^T - RIGHT
*
         BACK = WERR( II )
 50      CONTINUE

         NEGCNT = SLANEG( N, D, LLD, RIGHT, PIVMIN, R )
          IF( NEGCNT.LT.I ) THEN
             RIGHT = RIGHT + BACK
             BACK = TWO*BACK
             GO TO 50
          END IF
         WIDTH = HALF*ABS( LEFT - RIGHT )
         TMP = MAX( ABS( LEFT ), ABS( RIGHT ) )
         CVRGD = MAX(RTOL1*GAP,RTOL2*TMP)
         IF( WIDTH.LE.CVRGD .OR. WIDTH.LE.MNWDTH ) THEN
*           This interval has already converged and does not need refinement.
*           (Note that the gaps might change through refining the
*            eigenvalues, however, they can only get bigger.)
*           Remove it from the list.
            IWORK( K-1 ) = -1
*           Make sure that I1 always points to the first unconverged interval
            IF((I.EQ.I1).AND.(I.LT.ILAST)) I1 = I + 1
            IF((PREV.GE.I1).AND.(I.LE.ILAST)) IWORK( 2*PREV-1 ) = I + 1
         ELSE
*           unconverged interval found
            PREV = I
            NINT = NINT + 1
            IWORK( K-1 ) = I + 1
            IWORK( K ) = NEGCNT
         END IF
         WORK( K-1 ) = LEFT
         WORK( K ) = RIGHT
 75   CONTINUE

*
*     Do while( NINT.GT.0 ), i.e. there are still unconverged intervals
*     and while (ITER.LT.MAXITR)
*
      ITER = 0
 80   CONTINUE
      PREV = I1 - 1
      I = I1
```

```
      OLNINT = NINT

      DO 100 IP = 1, OLNINT
         K = 2*I
         II = I - OFFSET
         RGAP = WGAP( II )
         LGAP = RGAP
         IF(II.GT.1) LGAP = WGAP( II-1 )
         GAP = MIN( LGAP, RGAP )
         NEXT = IWORK( K-1 )
         LEFT = WORK( K-1 )
         RIGHT = WORK( K )
         MID = HALF*( LEFT + RIGHT )

*        semiwidth of interval
         WIDTH = RIGHT - MID
         TMP = MAX( ABS( LEFT ), ABS( RIGHT ) )
         CVRGD = MAX(RTOL1*GAP,RTOL2*TMP)
         IF( ( WIDTH.LE.CVRGD ) .OR. ( WIDTH.LE.MNWDTH ).OR.
     $       ( ITER.EQ.MAXITR ) )THEN
*           reduce number of unconverged intervals
            NINT = NINT - 1
*           Mark interval as converged.
            IWORK( K-1 ) = 0
            IF( I1.EQ.I ) THEN
               I1 = NEXT
            ELSE
*              Prev holds the last unconverged interval previously examined
               IF(PREV.GE.I1) IWORK( 2*PREV-1 ) = NEXT
            END IF
            I = NEXT
            GO TO 100
         END IF
         PREV = I
*
*        Perform one bisection step
*
         NEGCNT = SLANEG( N, D, LLD, MID, PIVMIN, R )
         IF( NEGCNT.LE.I-1 ) THEN
            WORK( K-1 ) = MID
         ELSE
            WORK( K ) = MID
         END IF
         I = NEXT
 100  CONTINUE
      ITER = ITER + 1
*     do another loop if there are still unconverged intervals
*     However, in the last iteration, all intervals are accepted
*     since this is the best we can do.
      IF( ( NINT.GT.0 ).AND.(ITER.LE.MAXITR) ) GO TO 80
```

```
*
*
*     At this point, all the intervals have converged
      DO 110 I = IFIRST, ILAST
         K = 2*I
         II = I - OFFSET
*        All intervals marked by '0' have been refined.
         IF( IWORK( K-1 ).EQ.0 ) THEN
            W( II ) = HALF*( WORK( K-1 )+WORK( K ) )
            WERR( II ) = WORK( K ) - W( II )
         END IF
 110  CONTINUE
*
      DO 111 I = IFIRST+1, ILAST
         K = 2*I
         II = I - OFFSET
         WGAP( II-1 ) = MAX( ZERO,
     $                     W(II) - WERR (II) - W( II-1 ) - WERR( II-1 ))
 111  CONTINUE

      RETURN
*
*     End of SLARRB
*
      END
```

———-

90.79.4 lisp code

— slarrb.lisp —

```
(let* ((zero 0.0) (two 2.0) (half 0.5))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 2.0 2.0) two)
           (type (single-float 0.5 0.5) half)
           (ignorable zero two half))
  (defun slarrb
         (n d lld ifirst ilast rtol1 rtol2 offset w wgap werr work iwork pivmin
          spdiam twist info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (single-float) spdiam pivmin rtol2 rtol1)
             (type (array single-float (*)) work werr wgap w lld d)
             (type (f2cl-lib:integer4) info twist offset ilast ifirst n))
    (f2cl-lib:with-multi-array-data
        ((d single-float d-%data% d-%offset%)
         (lld single-float lld-%data% lld-%offset%)
         (w single-float w-%data% w-%offset%)
```

```
    (wgap single-float wgap-%data% wgap-%offset%)
    (werr single-float werr-%data% werr-%offset%)
    (work single-float work-%data% work-%offset%)
    (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((back 0.0) (cvrgd 0.0) (gap 0.0) (left 0.0) (lgap 0.0) (mid 0.0)
         (mnwdth 0.0) (rgap 0.0) (right 0.0) (tmp 0.0) (width 0.0) (i 0)
         (i1 0) (ii 0) (ip 0) (iter 0) (k 0) (negcnt 0) (next 0)
         (f2cl-lib:nint 0) (olnint 0) (prev 0) (r 0) (maxitr 0))
    (declare
     (type (single-float) back cvrgd gap left lgap mid mnwdth rgap right
      tmp width)
     (type (f2cl-lib:integer4) i i1 ii ip iter k negcnt next f2cl-lib:nint
      olnint prev r maxitr))
    (setf info 0)
    (setf maxitr
            (f2cl-lib:int-add
             (f2cl-lib:int
              (/
               (- (f2cl-lib:flog (+ spdiam pivmin)) (f2cl-lib:flog pivmin))
               (f2cl-lib:flog two)))
             2))
    (setf mnwdth (* two pivmin))
    (setf r twist)
    (if (or (< r 1) (> r n))
        (setf r n))
    (setf i1 ifirst)
    (setf f2cl-lib:nint 0)
    (setf prev 0)
    (setf rgap
            (f2cl-lib:fref wgap-%data% ((f2cl-lib:int-sub i1 offset))
                           ((1 *)) wgap-%offset%))
    (f2cl-lib:fdo (i i1 (f2cl-lib:int-add i 1))
                  ((> i ilast) nil)
      (tagbody
        (setf k (f2cl-lib:int-mul 2 i))
        (setf ii (f2cl-lib:int-sub i offset))
        (setf left
                (- (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%)
                   (f2cl-lib:fref werr-%data% (ii) ((1 *)) werr-%offset%)))
        (setf right
                (+ (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%)
                   (f2cl-lib:fref werr-%data% (ii) ((1 *)) werr-%offset%)))
        (setf lgap rgap)
        (setf rgap (f2cl-lib:fref wgap-%data% (ii) ((1 *)) wgap-%offset%))
        (setf gap (min lgap rgap))
        (setf back (f2cl-lib:fref werr-%data% (ii) ((1 *)) werr-%offset%))
       label20
        (setf negcnt (slaneg n d lld left pivmin r))
        (cond
         ((> negcnt (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
```

```
        (setf left (- left back)) (setf back (* two back)) (go label20)))
      (setf back (f2cl-lib:fref werr-%data% (ii) ((1 *)) werr-%offset%))
     label50
      (setf negcnt (slaneg n d lld right pivmin r))
      (cond
       ((< negcnt i) (setf right (+ right back)) (setf back (* two back))
        (go label50)))
      (setf width (* half (abs (- left right))))
      (setf tmp (max (abs left) (abs right)))
      (setf cvrgd (max (* rtol1 gap) (* rtol2 tmp)))
      (cond
       ((or (<= width cvrgd) (<= width mnwdth))
        (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-sub k 1))
                             ((1 *)) iwork-%offset%)
                -1)
        (if (and (= i i1) (< i ilast))
            (setf i1 (f2cl-lib:int-add i 1)))
        (if (and (>= prev i1) (<= i ilast))
            (setf (f2cl-lib:fref iwork-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-mul 2 prev) 1))
                                 ((1 *)) iwork-%offset%)
                  (f2cl-lib:int-add i 1))))
       (t (setf prev i)
        (setf f2cl-lib:nint (f2cl-lib:int-add f2cl-lib:nint 1))
        (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-sub k 1))
                             ((1 *)) iwork-%offset%)
                (f2cl-lib:int-add i 1))
        (setf (f2cl-lib:fref iwork-%data% (k) ((1 *)) iwork-%offset%)
                negcnt)))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                           work-%offset%)
              left)
      (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%) right)
     label75))
  (setf iter 0)
 label80
  (setf prev (f2cl-lib:int-sub i1 1))
  (setf i i1)
  (setf olnint f2cl-lib:nint)
  (f2cl-lib:fdo (ip 1 (f2cl-lib:int-add ip 1))
                ((> ip olnint) nil)
    (tagbody
      (setf k (f2cl-lib:int-mul 2 i))
      (setf ii (f2cl-lib:int-sub i offset))
      (setf rgap (f2cl-lib:fref wgap-%data% (ii) ((1 *)) wgap-%offset%))
      (setf lgap rgap)
      (if (> ii 1)
          (setf lgap
                  (f2cl-lib:fref wgap-%data% ((f2cl-lib:int-sub ii 1))
```

```
                                ((1 *)) wgap-%offset%)))
    (setf gap (min lgap rgap))
    (setf next
            (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-sub k 1))
                           ((1 *)) iwork-%offset%))
    (setf left
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                           work-%offset%))
    (setf right (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%))
    (setf mid (* half (+ left right)))
    (setf width (- right mid))
    (setf tmp (max (abs left) (abs right)))
    (setf cvrgd (max (* rtol1 gap) (* rtol2 tmp)))
    (cond
     ((or (<= width cvrgd) (<= width mnwdth) (= iter maxitr))
      (setf f2cl-lib:nint (f2cl-lib:int-sub f2cl-lib:nint 1))
      (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-sub k 1))
                           ((1 *)) iwork-%offset%)
              0)
      (cond ((= i1 i) (setf i1 next))
            (t
             (if (>= prev i1)
                 (setf (f2cl-lib:fref iwork-%data%
                                      ((f2cl-lib:int-sub
                                        (f2cl-lib:int-mul 2 prev) 1))
                                      ((1 *)) iwork-%offset%)
                       next))))
      (setf i next) (go label100)))
    (setf prev i)
    (setf negcnt (slaneg n d lld mid pivmin r))
    (cond
     ((<= negcnt (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                           work-%offset%)
              mid))
     (t
      (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
              mid)))
    (setf i next)
   label100))
(setf iter (f2cl-lib:int-add iter 1))
(if (and (> f2cl-lib:nint 0) (<= iter maxitr))
    (go label80))
(f2cl-lib:fdo (i ifirst (f2cl-lib:int-add i 1))
              ((> i ilast) nil)
  (tagbody
    (setf k (f2cl-lib:int-mul 2 i))
    (setf ii (f2cl-lib:int-sub i offset))
    (cond
     ((=
```

```
              (f2cl-lib:fref iwork ((f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                             ((1 *)))
              0)
             (setf (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%)
                     (* half
                        (+
                         (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1))
                                        ((1 *)) work-%offset%)
                         (f2cl-lib:fref work-%data% (k) ((1 *))
                                        work-%offset%))))
             (setf (f2cl-lib:fref werr-%data% (ii) ((1 *)) werr-%offset%)
                     (- (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                        (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%)))))
          label110))
        (f2cl-lib:fdo (i (f2cl-lib:int-add ifirst 1) (f2cl-lib:int-add i 1))
                      ((> i ilast) nil)
          (tagbody
            (setf k (f2cl-lib:int-mul 2 i))
            (setf ii (f2cl-lib:int-sub i offset))
            (setf (f2cl-lib:fref wgap-%data% ((f2cl-lib:int-sub ii 1)) ((1 *))
                                 wgap-%offset%)
                  (max zero
                       (- (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%)
                          (f2cl-lib:fref werr-%data% (ii) ((1 *))
                                         werr-%offset%)
                          (f2cl-lib:fref w-%data% ((f2cl-lib:int-sub ii 1))
                                         ((1 *)) w-%offset%)
                          (f2cl-lib:fref werr-%data%
                                         ((f2cl-lib:int-sub ii 1)) ((1 *))
                                         werr-%offset%))))
          label111))
        (go end_label)
       end_label
        (return
         (values nil nil nil nil nil nil nil nil nil nil nil nil nil nil nil
                 nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slarrb
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (fortran-to-lisp::integer4)
```

```
                                                  (array single-float (*))
                                                  (array single-float (*))
                                                  (array single-float (*))
                                                  (array single-float (*))
                                                  (array fortran-to-lisp::integer4
                                                   (*))
                                                  (single-float) (single-float)
                                                  (fortran-to-lisp::integer4)
                                                  (fortran-to-lisp::integer4))
                                                :return-values
                                                '(nil nil nil nil nil nil nil nil
                                                  nil nil nil nil nil nil nil nil
                                                  fortran-to-lisp::info)
                                                :calls '(fortran-to-lisp::slaneg))))
```

———-

90.80 slarrc

90.80.1 Axiom unit tests

— **slarrc.input** —

```
)set break resume
)sys rm -f slarrc.output
)spool slarrc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.80.2 Axiom help page

— **slarrc.help** —

```
======================================================================
slarrc examples
======================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slarrc (JOBT, N, VL, VU, D, E, PIVMIN, EIGCNT, LCNT, RCNT,
           INFO)
           SLARRC computes the number of eigenvalues of the symmetric
           tridiagonal matrix.

Function/Subroutine Documentation
   subroutine slarrc (character JOBT, integer N, real VL, real VU, real,
       dimension( * ) D, real, dimension( * ) E, real PIVMIN, integer EIGCNT,
       integer LCNT, integer RCNT, integer INFO)
       SLARRC computes the number of eigenvalues of the symmetric tridiagonal
       matrix.

Purpose:
   Find the number of eigenvalues of the symmetric tridiagonal matrix T
   that are in the interval (VL,VU] if JOBT = 'T', and of L D L^T
   if JOBT = 'L'.

Parameters:

 JOBT
   JOBT is CHARACTER*1
   = 'T':  Compute Sturm count for matrix T.
   = 'L':  Compute Sturm count for matrix L D L^T.

 N
   N is INTEGER
   The order of the matrix. N > 0.

 VL
   VL is DOUBLE PRECISION

 VU
   VU is DOUBLE PRECISION
   The lower and upper bounds for the eigenvalues.

 D
   D is DOUBLE PRECISION array, dimension (N)
   JOBT = 'T': The N diagonal elements of the tridiagonal matrix T.
   JOBT = 'L': The N diagonal elements of the diagonal matrix D.

 E
   E is DOUBLE PRECISION array, dimension (N)
   JOBT = 'T': The N-1 offdiagonal elements of the matrix T.
```

```
  JOBT = 'L': The N-1 offdiagonal elements of the matrix L.

PIVMIN
  PIVMIN is REAL
  The minimum pivot in the Sturm sequence for T.

EIGCNT
  EIGCNT is INTEGER
  The number of eigenvalues of the symmetric tridiagonal matrix T
  that are in the interval (VL,VU]

LCNT
  LCNT is INTEGER

RCNT
  RCNT is INTEGER
  The left and right negcounts of the interval.

INFO
  INFO is INTEGER
```

90.80.3 fortran code

— slarrc.f —

```
*  =====================================================================
      SUBROUTINE SLARRC( JOBT, N, VL, VU, D, E, PIVMIN,
     $                   EIGCNT, LCNT, RCNT, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          JOBT
      INTEGER            EIGCNT, INFO, LCNT, N, RCNT
      REAL               PIVMIN, VL, VU
*     ..
*     .. Array Arguments ..
      REAL               D( * ), E( * )
*     ..
*
*  =====================================================================
*
```

```
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      LOGICAL            MATT
      REAL               LPIVOT, RPIVOT, SL, SU, TMP, TMP2

*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      LCNT = 0
      RCNT = 0
      EIGCNT = 0
      MATT = LSAME( JOBT, 'T' )


      IF (MATT) THEN
*        Sturm sequence count on T
         LPIVOT = D( 1 ) - VL
         RPIVOT = D( 1 ) - VU
         IF( LPIVOT.LE.ZERO ) THEN
            LCNT = LCNT + 1
         ENDIF
         IF( RPIVOT.LE.ZERO ) THEN
            RCNT = RCNT + 1
         ENDIF
         DO 10 I = 1, N-1
            TMP = E(I)**2
            LPIVOT = ( D( I+1 )-VL ) - TMP/LPIVOT
            RPIVOT = ( D( I+1 )-VU ) - TMP/RPIVOT
            IF( LPIVOT.LE.ZERO ) THEN
               LCNT = LCNT + 1
            ENDIF
            IF( RPIVOT.LE.ZERO ) THEN
               RCNT = RCNT + 1
            ENDIF
 10      CONTINUE
      ELSE
*        Sturm sequence count on L D L^T
         SL = -VL
         SU = -VU
         DO 20 I = 1, N - 1
            LPIVOT = D( I ) + SL
```

```
            RPIVOT = D( I ) + SU
            IF( LPIVOT.LE.ZERO ) THEN
               LCNT = LCNT + 1
            ENDIF
            IF( RPIVOT.LE.ZERO ) THEN
               RCNT = RCNT + 1
            ENDIF
            TMP = E(I) * D(I) * E(I)
*
            TMP2 = TMP / LPIVOT
            IF( TMP2.EQ.ZERO ) THEN
               SL =  TMP - VL
            ELSE
               SL = SL*TMP2 - VL
            END IF
*
            TMP2 = TMP / RPIVOT
            IF( TMP2.EQ.ZERO ) THEN
               SU =  TMP - VU
            ELSE
               SU = SU*TMP2 - VU
            END IF
 20      CONTINUE
         LPIVOT = D( N ) + SL
         RPIVOT = D( N ) + SU
         IF( LPIVOT.LE.ZERO ) THEN
            LCNT = LCNT + 1
         ENDIF
         IF( RPIVOT.LE.ZERO ) THEN
            RCNT = RCNT + 1
         ENDIF
      ENDIF
      EIGCNT = RCNT - LCNT

      RETURN
*
*     end of SLARRC
*
      END
```

———-

90.80.4 lisp code

— slarrc.lisp —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
```

```
          (ignorable zero))
  (defun slarrc (jobt n vl vu d e pivmin eigcnt lcnt rcnt info)
    (declare (type (array single-float (*)) e d)
             (type (single-float) pivmin vu vl)
             (type (f2cl-lib:integer4) info rcnt lcnt eigcnt n)
             (type (string *) jobt))
    (f2cl-lib:with-multi-array-data
        ((jobt character jobt-%data% jobt-%offset%)
         (d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%))
      (prog ((lpivot 0.0) (rpivot 0.0) (sl 0.0) (su 0.0) (tmp 0.0) (tmp2 0.0)
             (matt nil) (i 0))
        (declare (type (single-float) lpivot rpivot sl su tmp tmp2)
                 (type f2cl-lib:logical matt)
                 (type (f2cl-lib:integer4) i))
        (setf info 0)
        (setf lcnt 0)
        (setf rcnt 0)
        (setf eigcnt 0)
        (setf matt
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame jobt "T")
                  (declare (ignore var-1))
                  (when var-0 (setf jobt var-0))
                  ret-val))
        (cond
         (matt
          (setf lpivot (- (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%) vl))
          (setf rpivot (- (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%) vu))
          (cond ((<= lpivot zero) (setf lcnt (f2cl-lib:int-add lcnt 1))))
          (cond ((<= rpivot zero) (setf rcnt (f2cl-lib:int-add rcnt 1))))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
            (tagbody
              (setf tmp
                      (expt (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) 2))
              (setf lpivot
                      (+
                       (-
                        (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                       ((1 *)) d-%offset%)
                        vl)
                       (/ (- tmp) lpivot)))
              (setf rpivot
                      (+
                       (-
                        (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                       ((1 *)) d-%offset%)
                        vu)
                       (/ (- tmp) rpivot)))
```

```
              (cond ((<= lpivot zero) (setf lcnt (f2cl-lib:int-add lcnt 1))))
              (cond ((<= rpivot zero) (setf rcnt (f2cl-lib:int-add rcnt 1))))
             label10)))
          (t (setf sl (- vl)) (setf su (- vu))
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
             (tagbody
              (setf lpivot
                      (+ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) sl))
              (setf rpivot
                      (+ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) su))
              (cond ((<= lpivot zero) (setf lcnt (f2cl-lib:int-add lcnt 1))))
              (cond ((<= rpivot zero) (setf rcnt (f2cl-lib:int-add rcnt 1))))
              (setf tmp
                      (* (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                         (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                         (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
              (setf tmp2 (/ tmp lpivot))
              (cond ((= tmp2 zero) (setf sl (- tmp vl)))
                    (t (setf sl (- (* sl tmp2) vl))))
              (setf tmp2 (/ tmp rpivot))
              (cond ((= tmp2 zero) (setf su (- tmp vu)))
                    (t (setf su (- (* su tmp2) vu))))
             label20))
           (setf lpivot (+ (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) sl))
           (setf rpivot (+ (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) su))
           (cond ((<= lpivot zero) (setf lcnt (f2cl-lib:int-add lcnt 1))))
           (cond ((<= rpivot zero) (setf rcnt (f2cl-lib:int-add rcnt 1))))))
         (setf eigcnt (f2cl-lib:int-sub rcnt lcnt))
         (go end_label)
        end_label
         (return (values jobt nil nil nil nil nil nil eigcnt lcnt rcnt info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slarrc
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (single-float)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4))
                                            :return-values
```

```
                                        '(fortran-to-lisp::jobt nil nil nil
                                          nil nil nil
                                          fortran-to-lisp::eigcnt
                                          fortran-to-lisp::lcnt
                                          fortran-to-lisp::rcnt
                                          fortran-to-lisp::info)
                                        :calls 'nil)))
```

———-

90.81 slarrd

90.81.1 Axiom unit tests

— **slarrd.input** —

```
)set break resume
)sys rm -f slarrd.output
)spool slarrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.81.2 Axiom help page

— **slarrd.help** —

```
==================================================================
slarrd examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
```

```
      subroutine slarrd (RANGE, ORDER, N, VL, VU, IL, IU, GERS, RELTOL, D, E,
          E2, PIVMIN, NSPLIT, ISPLIT, M, W, WERR, WL, WU, IBLOCK, INDEXW,
          WORK, IWORK, INFO)
          SLARRD computes the eigenvalues of a symmetric tridiagonal matrix
          to suitable accuracy.

Function/Subroutine Documentation
   subroutine slarrd (character RANGE, character ORDER, integer N, real VL,
       real VU, integer IL, integer IU, real, dimension( * ) GERS, real
       RELTOL, real, dimension( * ) D, real, dimension( * ) E, real,
       dimension( * ) E2, real PIVMIN, integer NSPLIT, integer, dimension( * )
       ISPLIT, integer M, real, dimension( * ) W, real, dimension( * ) WERR,
       real WL, real WU, integer, dimension( * ) IBLOCK, integer, dimension( *
       ) INDEXW, real, dimension( * ) WORK, integer, dimension( * ) IWORK,
       integer INFO)
       SLARRD computes the eigenvalues of a symmetric tridiagonal matrix to
       suitable accuracy.

Purpose:
   SLARRD computes the eigenvalues of a symmetric tridiagonal
   matrix T to suitable accuracy. This is an auxiliary code to be
   called from SSTEMR.
   The user may ask for all eigenvalues, all eigenvalues
   in the half-open interval (VL, VU], or the IL-th through IU-th
   eigenvalues.

   To avoid overflow, the matrix must be scaled so that its
   largest element is no greater than
     overflow**(1/2) * underflow**(1/4)
   in absolute value, and for greatest
   accuracy, it should not be much smaller than that.

Parameters:

 RANGE
   RANGE is CHARACTER*1
   = 'A': ("All")   all eigenvalues will be found.
   = 'V': ("Value") all eigenvalues in the half-open interval
          (VL, VU] will be found.
   = 'I': ("Index") the IL-th through IU-th eigenvalues (of the
           entire matrix) will be found.

 ORDER
   ORDER is CHARACTER*1
   = 'B': ("By Block") the eigenvalues will be grouped by
          split-off block (see IBLOCK, ISPLIT) and
          ordered from smallest to largest within the block.
   = 'E': ("Entire matrix")
          the eigenvalues for the entire matrix
          will be ordered from smallest to largest.
```

```
N
  N is INTEGER
  The order of the tridiagonal matrix T.  N >= 0.

VL
  VL is REAL

VU
  VU is REAL
  If RANGE='V', the lower and upper bounds of the interval to
  be searched for eigenvalues.  Eigenvalues less than or equal
  to VL, or greater than VU, will not be returned.  VL < VU.
  Not referenced if RANGE = 'A' or 'I'.

IL
  IL is INTEGER

IU
  IU is INTEGER
  If RANGE='I', the indices (in ascending order) of the
  smallest and largest eigenvalues to be returned.
  1 <= IL <= IU <= N, if N > 0; IL = 1 and IU = 0 if N = 0.
  Not referenced if RANGE = 'A' or 'V'.

GERS
  GERS is REAL array, dimension (2*N)
  The N Gerschgorin intervals (the i-th Gerschgorin interval
  is (GERS(2*i-1), GERS(2*i)).

RELTOL
  RELTOL is REAL
  The minimum relative width of an interval.  When an interval
  is narrower than RELTOL times the larger (in
  magnitude) endpoint, then it is considered to be
  sufficiently small, i.e., converged.  Note: this should
  always be at least radix*machine epsilon.

D
  D is REAL array, dimension (N)
  The n diagonal elements of the tridiagonal matrix T.

E
  E is REAL array, dimension (N-1)
  The (n-1) off-diagonal elements of the tridiagonal matrix T.

E2
  E2 is REAL array, dimension (N-1)
  The (n-1) squared off-diagonal elements of the tridiagonal matrix T.
```

```
PIVMIN
  PIVMIN is REAL
  The minimum pivot allowed in the Sturm sequence for T.

NSPLIT
  NSPLIT is INTEGER
  The number of diagonal blocks in the matrix T. 1 <= NSPLIT <= N.

ISPLIT
  ISPLIT is INTEGER array, dimension (N)
  The splitting points, at which T breaks up into submatrices.
  The first submatrix consists of rows/columns 1 to ISPLIT(1),
  the second of rows/columns ISPLIT(1)+1 through ISPLIT(2),
  etc., and the NSPLIT-th consists of rows/columns
  ISPLIT(NSPLIT-1)+1 through ISPLIT(NSPLIT)=N.
  (Only the first NSPLIT elements will actually be used, but
  since the user cannot know a priori what value NSPLIT will
  have, N words must be reserved for ISPLIT.)

M
  M is INTEGER
  The actual number of eigenvalues found. 0 <= M <= N.
  (See also the description of INFO=2,3.)

W
  W is REAL array, dimension (N)
  On exit, the first M elements of W will contain the
  eigenvalue approximations. SLARRD computes an interval
  I_j = (a_j, b_j] that includes eigenvalue j. The eigenvalue
  approximation is given as the interval midpoint
  W(j)= ( a_j + b_j)/2. The corresponding error is bounded by
  WERR(j) = abs( a_j - b_j)/2

WERR
  WERR is REAL array, dimension (N)
  The error bound on the corresponding eigenvalue approximation in W.

WL
  WL is REAL

WU
  WU is REAL
  The interval (WL, WU] contains all the wanted eigenvalues.
  If RANGE='V', then WL=VL and WU=VU.
  If RANGE='A', then WL and WU are the global Gerschgorin bounds
      on the spectrum.
  If RANGE='I', then WL and WU are computed by SLAEBZ from the
      index range specified.

IBLOCK
```

```
  IBLOCK is INTEGER array, dimension (N)
  At each row/column j where E(j) is zero or small, the
  matrix T is considered to split into a block diagonal
  matrix.  On exit, if INFO = 0, IBLOCK(i) specifies to which
  block (from 1 to the number of blocks) the eigenvalue W(i)
  belongs.  (SLARRD may use the remaining N-M elements as workspace.)

 INDEXW
  INDEXW is INTEGER array, dimension (N)
  The indices of the eigenvalues within each block (submatrix);
  for example, INDEXW(i)= j and IBLOCK(i)=k imply that the
  i-th eigenvalue W(i) is the j-th eigenvalue in block k.

 WORK
  WORK is REAL array, dimension (4*N)

 IWORK
  IWORK is INTEGER array, dimension (3*N)

 INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  if INFO = -i, the i-th argument had an illegal value
  > 0:  some or all of the eigenvalues failed to converge or
        were not computed:
  =1 or 3: Bisection failed to converge for some
        eigenvalues; these eigenvalues are flagged by a
        negative block number.  The effect is that the
        eigenvalues may not be as accurate as the
        absolute and relative tolerances.  This is
        generally caused by unexpectedly inaccurate arithmetic.
  =2 or 3: RANGE='I' only: Not all of the eigenvalues IL:IU were found.
        Effect: M < IU+1-IL
        Cause:  non-monotonic arithmetic, causing the
                Sturm sequence to be non-monotonic.
        Cure:   recalculate, using RANGE='A', and pick
                out eigenvalues IL:IU.  In some cases,
                increasing the PARAMETER "FUDGE" may make things work.
  = 4:    RANGE='I', and the Gershgorin interval
        initially used was too small.  No eigenvalues were computed.
        Probable cause: your machine has sloppy floating-point arithmetic.
        Cure: Increase the PARAMETER "FUDGE", recompile, and try again.

Internal Parameters:

 FUDGE   REAL, default = 2
  A "fudge factor" to widen the Gershgorin intervals.  Ideally,
  a value of 1 should work, but on machines with sloppy
  arithmetic, this needs to be larger.  The default for
  publicly released versions should be large enough to handle
```

```
the worst machine around.  Note that this has no effect
on accuracy of the solution.
```

———-

90.81.3 fortran code

— **slarrd.f** —

```
*  =====================================================================
      SUBROUTINE SLARRD( RANGE, ORDER, N, VL, VU, IL, IU, GERS,
     $                   RELTOL, D, E, E2, PIVMIN, NSPLIT, ISPLIT,
     $                   M, W, WERR, WL, WU, IBLOCK, INDEXW,
     $                   WORK, IWORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          ORDER, RANGE
      INTEGER            IL, INFO, IU, M, N, NSPLIT
      REAL                PIVMIN, RELTOL, VL, VU, WL, WU
*     ..
*     .. Array Arguments ..
      INTEGER            IBLOCK( * ), INDEXW( * ),
     $                   ISPLIT( * ), IWORK( * )
      REAL               D( * ), E( * ), E2( * ),
     $                   GERS( * ), W( * ), WERR( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE, TWO, HALF, FUDGE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0,
     $                     TWO = 2.0E0, HALF = ONE/TWO,
     $                     FUDGE = TWO )
      INTEGER   ALLRNG, VALRNG, INDRNG
      PARAMETER ( ALLRNG = 1, VALRNG = 2, INDRNG = 3 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NCNVRG, TOOFEW
      INTEGER            I, IB, IBEGIN, IDISCL, IDISCU, IE, IEND, IINFO,
     $                   IM, IN, IOFF, IOUT, IRANGE, ITMAX, ITMP1,
     $                   ITMP2, IW, IWOFF, J, JBLK, JDISC, JE, JEE, NB,
```

```
     $                   NWL, NWU
      REAL               ATOLI, EPS, GL, GU, RTOLI, TMP1, TMP2,
     $                   TNORM, UFLOW, WKILL, WLU, WUL

*     ..
*     .. Local Arrays ..
      INTEGER            IDUMMA( 1 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      REAL               SLAMCH
      EXTERNAL           LSAME, ILAENV, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLAEBZ
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, INT, LOG, MAX, MIN
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
*     Decode RANGE
*
      IF( LSAME( RANGE, 'A' ) ) THEN
         IRANGE = ALLRNG
      ELSE IF( LSAME( RANGE, 'V' ) ) THEN
         IRANGE = VALRNG
      ELSE IF( LSAME( RANGE, 'I' ) ) THEN
         IRANGE = INDRNG
      ELSE
         IRANGE = 0
      END IF
*
*     Check for Errors
*
      IF( IRANGE.LE.0 ) THEN
         INFO = -1
      ELSE IF( .NOT.(LSAME(ORDER,'B').OR.LSAME(ORDER,'E')) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( IRANGE.EQ.VALRNG ) THEN
         IF( VL.GE.VU )
     $      INFO = -5
      ELSE IF( IRANGE.EQ.INDRNG .AND.
     $        ( IL.LT.1 .OR. IL.GT.MAX( 1, N ) ) ) THEN
         INFO = -6
```

```
      ELSE IF( IRANGE.EQ.INDRNG .AND.
     $        ( IU.LT.MIN( N, IL ) .OR. IU.GT.N ) ) THEN
         INFO = -7
      END IF
*
      IF( INFO.NE.0 ) THEN
         RETURN
      END IF

*     Initialize error flags
      INFO = 0
      NCNVRG = .FALSE.
      TOOFEW = .FALSE.

*     Quick return if possible
      M = 0
      IF( N.EQ.0 ) RETURN

*     Simplification:
      IF( IRANGE.EQ.INDRNG .AND. IL.EQ.1 .AND. IU.EQ.N ) IRANGE = 1

*     Get machine constants
      EPS = SLAMCH( 'P' )
      UFLOW = SLAMCH( 'U' )


*     Special Case when N=1
*     Treat case of 1x1 matrix for quick return
      IF( N.EQ.1 ) THEN
         IF( (IRANGE.EQ.ALLRNG).OR.
     $       ((IRANGE.EQ.VALRNG).AND.(D(1).GT.VL).AND.(D(1).LE.VU)).OR.
     $       ((IRANGE.EQ.INDRNG).AND.(IL.EQ.1).AND.(IU.EQ.1)) ) THEN
            M = 1
            W(1) = D(1)
*           The computation error of the eigenvalue is zero
            WERR(1) = ZERO
            IBLOCK( 1 ) = 1
            INDEXW( 1 ) = 1
         ENDIF
         RETURN
      END IF

*     NB is the minimum vector length for vector bisection, or 0
*     if only scalar is to be done.
      NB = ILAENV( 1, 'SSTEBZ', ' ', N, -1, -1, -1 )
      IF( NB.LE.1 ) NB = 0

*     Find global spectral radius
      GL = D(1)
      GU = D(1)
```

```
      DO 5 I = 1,N
         GL =  MIN( GL, GERS( 2*I - 1))
         GU = MAX( GU, GERS(2*I) )
 5    CONTINUE
*     Compute global Gerschgorin bounds and spectral diameter
      TNORM = MAX( ABS( GL ), ABS( GU ) )
      GL = GL - FUDGE*TNORM*EPS*N - FUDGE*TWO*PIVMIN
      GU = GU + FUDGE*TNORM*EPS*N + FUDGE*TWO*PIVMIN
*     [JAN/28/2009] remove the line below since SPDIAM variable not use
*     SPDIAM = GU - GL
*     Input arguments for SLAEBZ:
*     The relative tolerance.  An interval (a,b] lies within
*     "relative tolerance" if  b-a < RELTOL*max(|a|,|b|),
      RTOLI = RELTOL
*     Set the absolute tolerance for interval convergence to zero to force
*     interval convergence based on relative size of the interval.
*     This is dangerous because intervals might not converge when RELTOL is
*     small. But at least a very small number should be selected so that for
*     strongly graded matrices, the code can get relatively accurate
*     eigenvalues.
      ATOLI = FUDGE*TWO*UFLOW + FUDGE*TWO*PIVMIN

      IF( IRANGE.EQ.INDRNG ) THEN

*        RANGE='I': Compute an interval containing eigenvalues
*        IL through IU. The initial interval [GL,GU] from the global
*        Gerschgorin bounds GL and GU is refined by SLAEBZ.
         ITMAX = INT( ( LOG( TNORM+PIVMIN )-LOG( PIVMIN ) ) /
     $           LOG( TWO ) ) + 2
         WORK( N+1 ) = GL
         WORK( N+2 ) = GL
         WORK( N+3 ) = GU
         WORK( N+4 ) = GU
         WORK( N+5 ) = GL
         WORK( N+6 ) = GU
         IWORK( 1 ) = -1
         IWORK( 2 ) = -1
         IWORK( 3 ) = N + 1
         IWORK( 4 ) = N + 1
         IWORK( 5 ) = IL - 1
         IWORK( 6 ) = IU
*
         CALL SLAEBZ( 3, ITMAX, N, 2, 2, NB, ATOLI, RTOLI, PIVMIN,
     $         D, E, E2, IWORK( 5 ), WORK( N+1 ), WORK( N+5 ), IOUT,
     $                IWORK, W, IBLOCK, IINFO )
         IF( IINFO .NE. 0 ) THEN
            INFO = IINFO
            RETURN
         END IF
*        On exit, output intervals may not be ordered by ascending negcount
```

```
         IF( IWORK( 6 ).EQ.IU ) THEN
            WL = WORK( N+1 )
            WLU = WORK( N+3 )
            NWL = IWORK( 1 )
            WU = WORK( N+4 )
            WUL = WORK( N+2 )
            NWU = IWORK( 4 )
         ELSE
            WL = WORK( N+2 )
            WLU = WORK( N+4 )
            NWL = IWORK( 2 )
            WU = WORK( N+3 )
            WUL = WORK( N+1 )
            NWU = IWORK( 3 )
         END IF
*        On exit, the interval [WL, WLU] contains a value with negcount NWL,
*        and [WUL, WU] contains a value with negcount NWU.
         IF( NWL.LT.0 .OR. NWL.GE.N .OR. NWU.LT.1 .OR. NWU.GT.N ) THEN
            INFO = 4
            RETURN
         END IF

      ELSEIF( IRANGE.EQ.VALRNG ) THEN
         WL = VL
         WU = VU

      ELSEIF( IRANGE.EQ.ALLRNG ) THEN
         WL = GL
         WU = GU
      ENDIF



*     Find Eigenvalues -- Loop Over blocks and recompute NWL and NWU.
*     NWL accumulates the number of eigenvalues .le. WL,
*     NWU accumulates the number of eigenvalues .le. WU
      M = 0
      IEND = 0
      INFO = 0
      NWL = 0
      NWU = 0
*
      DO 70 JBLK = 1, NSPLIT
         IOFF = IEND
         IBEGIN = IOFF + 1
         IEND = ISPLIT( JBLK )
         IN = IEND - IOFF
*
         IF( IN.EQ.1 ) THEN
*           1x1 block
```

```
            IF( WL.GE.D( IBEGIN )-PIVMIN )
     $         NWL = NWL + 1
            IF( WU.GE.D( IBEGIN )-PIVMIN )
     $         NWU = NWU + 1
            IF( IRANGE.EQ.ALLRNG .OR.
     $           ( WL.LT.D( IBEGIN )-PIVMIN
     $             .AND. WU.GE. D( IBEGIN )-PIVMIN ) ) THEN
               M = M + 1
               W( M ) = D( IBEGIN )
               WERR(M) = ZERO
*              The gap for a single block doesn't matter for the later
*              algorithm and is assigned an arbitrary large value
               IBLOCK( M ) = JBLK
               INDEXW( M ) = 1
            END IF

*        Disabled 2x2 case because of a failure on the following matrix
*        RANGE = 'I', IL = IU = 4
*          Original Tridiagonal, d = [
*           -0.150102010615740E+00
*           -0.849897989384260E+00
*           -0.128208148052635E-15
*            0.128257718286320E-15
*          ];
*          e = [
*           -0.357171383266986E+00
*           -0.180411241501588E-15
*           -0.175152352710251E-15
*          ];
*
*         ELSE IF( IN.EQ.2 ) THEN
**           2x2 block
*            DISC = SQRT( (HALF*(D(IBEGIN)-D(IEND)))**2 + E(IBEGIN)**2 )
*            TMP1 = HALF*(D(IBEGIN)+D(IEND))
*            L1 = TMP1 - DISC
*            IF( WL.GE. L1-PIVMIN )
*     $         NWL = NWL + 1
*            IF( WU.GE. L1-PIVMIN )
*     $         NWU = NWU + 1
*            IF( IRANGE.EQ.ALLRNG .OR. ( WL.LT.L1-PIVMIN .AND. WU.GE.
*     $          L1-PIVMIN ) ) THEN
*               M = M + 1
*               W( M ) = L1
**              The uncertainty of eigenvalues of a 2x2 matrix is very small
*               WERR( M ) = EPS * ABS( W( M ) ) * TWO
*               IBLOCK( M ) = JBLK
*               INDEXW( M ) = 1
*            ENDIF
*            L2 = TMP1 + DISC
*            IF( WL.GE. L2-PIVMIN )
```

```
*     $         NWL = NWL + 1
*            IF( WU.GE. L2-PIVMIN )
*     $         NWU = NWU + 1
*            IF( IRANGE.EQ.ALLRNG .OR. ( WL.LT.L2-PIVMIN .AND. WU.GE.
*     $          L2-PIVMIN ) ) THEN
*              M = M + 1
*              W( M ) = L2
**             The uncertainty of eigenvalues of a 2x2 matrix is very small
*              WERR( M ) = EPS * ABS( W( M ) ) * TWO
*              IBLOCK( M ) = JBLK
*              INDEXW( M ) = 2
*            ENDIF
         ELSE
*           General Case - block of size IN >= 2
*           Compute local Gerschgorin interval and use it as the initial
*           interval for SLAEBZ
            GU = D( IBEGIN )
            GL = D( IBEGIN )
            TMP1 = ZERO

            DO 40 J = IBEGIN, IEND
               GL =  MIN( GL, GERS( 2*J - 1))
               GU = MAX( GU, GERS(2*J) )
   40       CONTINUE
*           [JAN/28/2009]
*           change SPDIAM by TNORM in lines 2 and 3 thereafter
*           line 1: remove computation of SPDIAM (not useful anymore)
*           SPDIAM = GU - GL
*           GL = GL - FUDGE*SPDIAM*EPS*IN - FUDGE*PIVMIN
*           GU = GU + FUDGE*SPDIAM*EPS*IN + FUDGE*PIVMIN
            GL = GL - FUDGE*TNORM*EPS*IN - FUDGE*PIVMIN
            GU = GU + FUDGE*TNORM*EPS*IN + FUDGE*PIVMIN
*
            IF( IRANGE.GT.1 ) THEN
               IF( GU.LT.WL ) THEN
*                 the local block contains none of the wanted eigenvalues
                  NWL = NWL + IN
                  NWU = NWU + IN
                  GO TO 70
               END IF
*              refine search interval if possible, only range (WL,WU] matters
               GL = MAX( GL, WL )
               GU = MIN( GU, WU )
               IF( GL.GE.GU )
     $            GO TO 70
            END IF

*           Find negcount of initial interval boundaries GL and GU
            WORK( N+1 ) = GL
            WORK( N+IN+1 ) = GU
```

```
            CALL SLAEBZ( 1, 0, IN, IN, 1, NB, ATOLI, RTOLI, PIVMIN,
     $                   D( IBEGIN ), E( IBEGIN ), E2( IBEGIN ),
     $                   IDUMMA, WORK( N+1 ), WORK( N+2*IN+1 ), IM,
     $                   IWORK, W( M+1 ), IBLOCK( M+1 ), IINFO )
            IF( IINFO .NE. 0 ) THEN
               INFO = IINFO
               RETURN
            END IF
*
            NWL = NWL + IWORK( 1 )
            NWU = NWU + IWORK( IN+1 )
            IWOFF = M - IWORK( 1 )

*           Compute Eigenvalues
            ITMAX = INT( ( LOG( GU-GL+PIVMIN )-LOG( PIVMIN ) ) /
     $              LOG( TWO ) ) + 2
            CALL SLAEBZ( 2, ITMAX, IN, IN, 1, NB, ATOLI, RTOLI, PIVMIN,
     $                   D( IBEGIN ), E( IBEGIN ), E2( IBEGIN ),
     $                   IDUMMA, WORK( N+1 ), WORK( N+2*IN+1 ), IOUT,
     $                   IWORK, W( M+1 ), IBLOCK( M+1 ), IINFO )
            IF( IINFO .NE. 0 ) THEN
               INFO = IINFO
               RETURN
            END IF
*
*           Copy eigenvalues into W and IBLOCK
*           Use -JBLK for block number for unconverged eigenvalues.
*           Loop over the number of output intervals from SLAEBZ
            DO 60 J = 1, IOUT
*              eigenvalue approximation is middle point of interval
               TMP1 = HALF*( WORK( J+N )+WORK( J+IN+N ) )
*              semi length of error interval
               TMP2 = HALF*ABS( WORK( J+N )-WORK( J+IN+N ) )
               IF( J.GT.IOUT-IINFO ) THEN
*                 Flag non-convergence.
                  NCNVRG = .TRUE.
                  IB = -JBLK
               ELSE
                  IB = JBLK
               END IF
               DO 50 JE = IWORK( J ) + 1 + IWOFF,
     $                 IWORK( J+IN ) + IWOFF
                  W( JE ) = TMP1
                  WERR( JE ) = TMP2
                  INDEXW( JE ) = JE - IWOFF
                  IBLOCK( JE ) = IB
   50          CONTINUE
   60       CONTINUE
*
            M = M + IM
```

```
         END IF
   70 CONTINUE

*     If RANGE='I', then (WL,WU) contains eigenvalues NWL+1,...,NWU
*     If NWL+1 < IL or NWU > IU, discard extra eigenvalues.
      IF( IRANGE.EQ.INDRNG ) THEN
         IDISCL = IL - 1 - NWL
         IDISCU = NWU - IU
*
         IF( IDISCL.GT.0 ) THEN
            IM = 0
            DO 80 JE = 1, M
*              Remove some of the smallest eigenvalues from the left so that
*              at the end IDISCL =0. Move all eigenvalues up to the left.
               IF( W( JE ).LE.WLU .AND. IDISCL.GT.0 ) THEN
                  IDISCL = IDISCL - 1
               ELSE
                  IM = IM + 1
                  W( IM ) = W( JE )
                  WERR( IM ) = WERR( JE )
                  INDEXW( IM ) = INDEXW( JE )
                  IBLOCK( IM ) = IBLOCK( JE )
               END IF
 80         CONTINUE
            M = IM
         END IF
         IF( IDISCU.GT.0 ) THEN
*           Remove some of the largest eigenvalues from the right so that
*           at the end IDISCU =0. Move all eigenvalues up to the left.
            IM=M+1
            DO 81 JE = M, 1, -1
               IF( W( JE ).GE.WUL .AND. IDISCU.GT.0 ) THEN
                  IDISCU = IDISCU - 1
               ELSE
                  IM = IM - 1
                  W( IM ) = W( JE )
                  WERR( IM ) = WERR( JE )
                  INDEXW( IM ) = INDEXW( JE )
                  IBLOCK( IM ) = IBLOCK( JE )
               END IF
 81         CONTINUE
            JEE = 0
            DO 82 JE = IM, M
               JEE = JEE + 1
               W( JEE ) = W( JE )
               WERR( JEE ) = WERR( JE )
               INDEXW( JEE ) = INDEXW( JE )
               IBLOCK( JEE ) = IBLOCK( JE )
 82         CONTINUE
            M = M-IM+1
```

```
         END IF

         IF( IDISCL.GT.0 .OR. IDISCU.GT.0 ) THEN
*           Code to deal with effects of bad arithmetic. (If N(w) is
*           monotone non-decreasing, this should never happen.)
*           Some low eigenvalues to be discarded are not in (WL,WLU],
*           or high eigenvalues to be discarded are not in (WUL,WU]
*           so just kill off the smallest IDISCL/largest IDISCU
*           eigenvalues, by marking the corresponding IBLOCK = 0
            IF( IDISCL.GT.0 ) THEN
               WKILL = WU
               DO 100 JDISC = 1, IDISCL
                  IW = 0
                  DO 90 JE = 1, M
                     IF( IBLOCK( JE ).NE.0 .AND.
     $                    ( W( JE ).LT.WKILL .OR. IW.EQ.0 ) ) THEN
                        IW = JE
                        WKILL = W( JE )
                     END IF
 90               CONTINUE
                  IBLOCK( IW ) = 0
 100           CONTINUE
            END IF
            IF( IDISCU.GT.0 ) THEN
               WKILL = WL
               DO 120 JDISC = 1, IDISCU
                  IW = 0
                  DO 110 JE = 1, M
                     IF( IBLOCK( JE ).NE.0 .AND.
     $                    ( W( JE ).GE.WKILL .OR. IW.EQ.0 ) ) THEN
                        IW = JE
                        WKILL = W( JE )
                     END IF
 110              CONTINUE
                  IBLOCK( IW ) = 0
 120           CONTINUE
            END IF
*           Now erase all eigenvalues with IBLOCK set to zero
            IM = 0
            DO 130 JE = 1, M
               IF( IBLOCK( JE ).NE.0 ) THEN
                  IM = IM + 1
                  W( IM ) = W( JE )
                  WERR( IM ) = WERR( JE )
                  INDEXW( IM ) = INDEXW( JE )
                  IBLOCK( IM ) = IBLOCK( JE )
               END IF
 130        CONTINUE
            M = IM
         END IF
```

```
         IF( IDISCL.LT.0 .OR. IDISCU.LT.0 ) THEN
            TOOFEW = .TRUE.
         END IF
      END IF
*
      IF(( IRANGE.EQ.ALLRNG .AND. M.NE.N ).OR.
     $   ( IRANGE.EQ.INDRNG .AND. M.NE.IU-IL+1 ) ) THEN
         TOOFEW = .TRUE.
      END IF

*     If ORDER='B', do nothing the eigenvalues are already sorted by
*        block.
*     If ORDER='E', sort the eigenvalues from smallest to largest

      IF( LSAME(ORDER,'E') .AND. NSPLIT.GT.1 ) THEN
         DO 150 JE = 1, M - 1
            IE = 0
            TMP1 = W( JE )
            DO 140 J = JE + 1, M
               IF( W( J ).LT.TMP1 ) THEN
                  IE = J
                  TMP1 = W( J )
               END IF
  140       CONTINUE
            IF( IE.NE.0 ) THEN
               TMP2 = WERR( IE )
               ITMP1 = IBLOCK( IE )
               ITMP2 = INDEXW( IE )
               W( IE ) = W( JE )
               WERR( IE ) = WERR( JE )
               IBLOCK( IE ) = IBLOCK( JE )
               INDEXW( IE ) = INDEXW( JE )
               W( JE ) = TMP1
               WERR( JE ) = TMP2
               IBLOCK( JE ) = ITMP1
               INDEXW( JE ) = ITMP2
            END IF
  150    CONTINUE
      END IF
*
      INFO = 0
      IF( NCNVRG )
     $   INFO = INFO + 1
      IF( TOOFEW )
     $   INFO = INFO + 2
      RETURN
*
*     End of SLARRD
*
      END
```

———-

90.81.4 lisp code

— slarrd.lisp —

```
(let* ((zero 0.0)
       (one 1.0)
       (two 2.0)
       (half (f2cl-lib:f2cl/ one two))
       (fudge two)
       (allrng 1)
       (valrng 2)
       (indrng 3))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float) half)
           (type (single-float) fudge)
           (type (f2cl-lib:integer4 1 1) allrng)
           (type (f2cl-lib:integer4 2 2) valrng)
           (type (f2cl-lib:integer4 3 3) indrng)
           (ignorable zero one two half fudge allrng valrng indrng))
  (defun slarrd
         (range order n vl vu il iu gers reltol d e e2 pivmin nsplit isplit m w
          werr wl wu iblock indexw work iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork indexw iblock isplit)
             (type (array single-float (*)) work werr w e2 e d gers)
             (type (single-float) wu wl pivmin reltol vu vl)
             (type (f2cl-lib:integer4) info m nsplit iu il n)
             (type (string *) order range))
    (f2cl-lib:with-multi-array-data
        ((range character range-%data% range-%offset%)
         (order character order-%data% order-%offset%)
         (gers single-float gers-%data% gers-%offset%)
         (d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%)
         (e2 single-float e2-%data% e2-%offset%)
         (w single-float w-%data% w-%offset%)
         (werr single-float werr-%data% werr-%offset%)
         (work single-float work-%data% work-%offset%)
         (isplit f2cl-lib:integer4 isplit-%data% isplit-%offset%)
         (iblock f2cl-lib:integer4 iblock-%data% iblock-%offset%)
         (indexw f2cl-lib:integer4 indexw-%data% indexw-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((idumma (make-array 1 :element-type 'f2cl-lib:integer4))
             (atoli 0.0) (eps 0.0) (gl 0.0) (gu 0.0) (rtoli 0.0) (tmp1 0.0)
             (tmp2 0.0) (tnorm 0.0) (uflow 0.0) (wkill 0.0) (wlu 0.0) (wul 0.0)
```

```
     (i 0) (ib 0) (ibegin 0) (idiscl 0) (idiscu 0) (ie 0) (iend 0)
     (iinfo 0) (im 0) (in 0) (ioff 0) (iout 0) (irange 0) (itmax 0)
     (itmp1 0) (itmp2 0) (iw 0) (iwoff 0) (j 0) (jblk 0) (jdisc 0)
     (je 0) (jee 0) (nb 0) (nwl 0) (nwu 0) (ncnvrg nil) (toofew nil))
(declare (type (array f2cl-lib:integer4 (1)) idumma)
         (type (single-float) atoli eps gl gu rtoli tmp1 tmp2 tnorm
          uflow wkill wlu wul)
         (type (f2cl-lib:integer4) i ib ibegin idiscl idiscu ie iend
          iinfo im in ioff iout irange itmax itmp1 itmp2 iw iwoff j
          jblk jdisc je jee nb nwl nwu)
         (type f2cl-lib:logical ncnvrg toofew))
(setf info 0)
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame range "A")
    (declare (ignore var-1))
    (when var-0 (setf range var-0))
    ret-val)
  (setf irange allrng))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame range "V")
    (declare (ignore var-1))
    (when var-0 (setf range var-0))
    ret-val)
  (setf irange valrng))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame range "I")
    (declare (ignore var-1))
    (when var-0 (setf range var-0))
    ret-val)
  (setf irange indrng))
 (t (setf irange 0)))
(cond ((<= irange 0) (setf info -1))
      ((not
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame order "B")
           (declare (ignore var-1))
           (when var-0 (setf order var-0))
           ret-val)
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame order "E")
           (declare (ignore var-1))
           (when var-0 (setf order var-0))
           ret-val)))
       (setf info -2))
      ((< n 0) (setf info -3))
      ((= irange valrng)
       (if (>= vl vu)
           (setf info -5)))
```

```
      ((and (= irange indrng)
            (or (< il 1)
                (> il
                   (max (the f2cl-lib:integer4 1)
                        (the f2cl-lib:integer4 n)))))
       (setf info -6))
      ((and (= irange indrng)
            (or
             (< iu
                (min (the f2cl-lib:integer4 n)
                     (the f2cl-lib:integer4 il)))
             (> iu n)))
       (setf info -7)))
(cond ((/= info 0) (go end_label)))
(setf info 0)
(setf ncnvrg f2cl-lib:%false%)
(setf toofew f2cl-lib:%false%)
(setf m 0)
(if (= n 0)
    (go end_label))
(if (and (= irange indrng) (= il 1) (= iu n))
    (setf irange 1))
(setf eps (slamch "P"))
(setf uflow (slamch "U"))
(cond
 ((= n 1)
  (cond
   ((or (= irange allrng)
        (and (= irange valrng) (> (f2cl-lib:fref d (1) ((1 *))) vl)
             (<= (f2cl-lib:fref d (1) ((1 *))) vu))
        (and (= irange indrng) (= il 1) (= iu 1)))
    (setf m 1)
    (setf (f2cl-lib:fref w-%data% (1) ((1 *)) w-%offset%)
            (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
    (setf (f2cl-lib:fref werr-%data% (1) ((1 *)) werr-%offset%) zero)
    (setf (f2cl-lib:fref iblock-%data% (1) ((1 *)) iblock-%offset%) 1)
    (setf (f2cl-lib:fref indexw-%data% (1) ((1 *)) indexw-%offset%)
            1)))
  (go end_label)))
(setf nb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 1 "SSTEBZ" " " n -1 -1 -1)
          (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6))
          (setf n var-3)
          ret-val))
(if (<= nb 1)
    (setf nb 0))
(setf gl (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
(setf gu (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
```

```
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf gl
            (min gl
                 (f2cl-lib:fref gers-%data%
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-mul 2 i) 1))
                                ((1 *)) gers-%offset%)))
    (setf gu
            (max gu
                 (f2cl-lib:fref gers-%data% ((f2cl-lib:int-mul 2 i))
                                ((1 *)) gers-%offset%)))
   label5))
(setf tnorm (max (abs gl) (abs gu)))
(setf gl (+ gl (* (- fudge) tnorm eps n) (* (- fudge) two pivmin)))
(setf gu (+ gu (* fudge tnorm eps n) (* fudge two pivmin)))
(setf rtoli reltol)
(setf atoli (+ (* fudge two uflow) (* fudge two pivmin)))
(cond
 ((= irange indrng)
  (setf itmax
          (f2cl-lib:int-add
           (f2cl-lib:int
            (/
             (- (f2cl-lib:flog (+ tnorm pivmin))
                (f2cl-lib:flog pivmin))
             (f2cl-lib:flog two)))
           2))
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 1)) ((1 *))
                       work-%offset%)
          gl)
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 2)) ((1 *))
                       work-%offset%)
          gl)
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 3)) ((1 *))
                       work-%offset%)
          gu)
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 4)) ((1 *))
                       work-%offset%)
          gu)
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 5)) ((1 *))
                       work-%offset%)
          gl)
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 6)) ((1 *))
                       work-%offset%)
          gu)
  (setf (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%) -1)
  (setf (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%) -1)
  (setf (f2cl-lib:fref iwork-%data% (3) ((1 *)) iwork-%offset%)
```

```
        (f2cl-lib:int-add n 1))
(setf (f2cl-lib:fref iwork-%data% (4) ((1 *)) iwork-%offset%)
        (f2cl-lib:int-add n 1))
(setf (f2cl-lib:fref iwork-%data% (5) ((1 *)) iwork-%offset%)
        (f2cl-lib:int-sub il 1))
(setf (f2cl-lib:fref iwork-%data% (6) ((1 *)) iwork-%offset%) iu)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18
     var-19)
    (slaebz 3 itmax n 2 2 nb atoli rtoli pivmin d e e2
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (5) ((1 *))
                           iwork-%offset%)
     (f2cl-lib:array-slice work-%data% single-float ((+ n 1)) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float ((+ n 5)) ((1 *))
                           work-%offset%)
     iout iwork w iblock iinfo)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
    var-9 var-10 var-11 var-12 var-13 var-14 var-16 var-17 var-18))
  (setf iout var-15)
  (setf iinfo var-19))
(cond ((/= iinfo 0) (setf info iinfo) (go end_label)))
(cond
 ((= (f2cl-lib:fref iwork (6) ((1 *))) iu)
  (setf wl
          (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 1)) ((1 *))
                         work-%offset%))
  (setf wlu
          (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 3)) ((1 *))
                         work-%offset%))
  (setf nwl (f2cl-lib:fref iwork-%data% (1) ((1 *)) iwork-%offset%))
  (setf wu
          (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 4)) ((1 *))
                         work-%offset%))
  (setf wul
          (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 2)) ((1 *))
                         work-%offset%))
  (setf nwu (f2cl-lib:fref iwork-%data% (4) ((1 *)) iwork-%offset%)))
 (t
  (setf wl
          (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 2)) ((1 *))
                         work-%offset%))
  (setf wlu
          (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 4)) ((1 *))
                         work-%offset%))
  (setf nwl (f2cl-lib:fref iwork-%data% (2) ((1 *)) iwork-%offset%))
  (setf wu
          (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 3)) ((1 *))
```

```
                              work-%offset%))
    (setf wul
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 1)) ((1 *))
                              work-%offset%))
    (setf nwu
            (f2cl-lib:fref iwork-%data% (3) ((1 *)) iwork-%offset%))))
  (cond
   ((or (< nwl 0) (>= nwl n) (< nwu 1) (> nwu n)) (setf info 4)
    (go end_label))))
 ((= irange valrng) (setf wl vl) (setf wu vu))
 ((= irange allrng) (setf wl gl) (setf wu gu)))
(setf m 0)
(setf iend 0)
(setf info 0)
(setf nwl 0)
(setf nwu 0)
(f2cl-lib:fdo (jblk 1 (f2cl-lib:int-add jblk 1))
              ((> jblk nsplit) nil)
  (tagbody
    (setf ioff iend)
    (setf ibegin (f2cl-lib:int-add ioff 1))
    (setf iend
            (f2cl-lib:fref isplit-%data% (jblk) ((1 *))
                              isplit-%offset%))
    (setf in (f2cl-lib:int-sub iend ioff))
    (cond
     ((= in 1)
      (if (>= wl
              (- (f2cl-lib:fref d-%data% (ibegin) ((1 *)) d-%offset%)
                 pivmin))
          (setf nwl (f2cl-lib:int-add nwl 1)))
      (if (>= wu
              (- (f2cl-lib:fref d-%data% (ibegin) ((1 *)) d-%offset%)
                 pivmin))
          (setf nwu (f2cl-lib:int-add nwu 1)))
      (cond
       ((or (= irange allrng)
            (and
             (< wl (+ (f2cl-lib:fref d (ibegin) ((1 *))) (- pivmin)))
             (>= wu
                 (+ (f2cl-lib:fref d (ibegin) ((1 *))) (- pivmin)))))
        (setf m (f2cl-lib:int-add m 1))
        (setf (f2cl-lib:fref w-%data% (m) ((1 *)) w-%offset%)
                (f2cl-lib:fref d-%data% (ibegin) ((1 *)) d-%offset%))
        (setf (f2cl-lib:fref werr-%data% (m) ((1 *)) werr-%offset%)
                zero)
        (setf (f2cl-lib:fref iblock-%data% (m) ((1 *)) iblock-%offset%)
                jblk)
        (setf (f2cl-lib:fref indexw-%data% (m) ((1 *)) indexw-%offset%)
                1))))
```

```
(t (setf gu (f2cl-lib:fref d-%data% (ibegin) ((1 *)) d-%offset%))
 (setf gl (f2cl-lib:fref d-%data% (ibegin) ((1 *)) d-%offset%))
 (setf tmp1 zero)
 (f2cl-lib:fdo (j ibegin (f2cl-lib:int-add j 1))
               ((> j iend) nil)
   (tagbody
     (setf gl
             (min gl
                  (f2cl-lib:fref gers-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-mul 2 j) 1))
                                 ((1 *)) gers-%offset%)))
     (setf gu
             (max gu
                  (f2cl-lib:fref gers-%data%
                                 ((f2cl-lib:int-mul 2 j)) ((1 *))
                                 gers-%offset%)))
    label40))
 (setf gl (- (+ gl (* (- fudge) tnorm eps in)) (* fudge pivmin)))
 (setf gu (+ gu (* fudge tnorm eps in) (* fudge pivmin)))
 (cond
  ((> irange 1)
   (cond
    ((< gu wl) (setf nwl (f2cl-lib:int-add nwl in))
     (setf nwu (f2cl-lib:int-add nwu in)) (go label70)))
   (setf gl (max gl wl)) (setf gu (min gu wu))
   (if (>= gl gu)
       (go label70))))
 (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n 1)) ((1 *))
                      work-%offset%)
         gl)
 (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n in 1))
                      ((1 *)) work-%offset%)
         gu)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17
      var-18 var-19)
     (slaebz 1 0 in in 1 nb atoli rtoli pivmin
      (f2cl-lib:array-slice d-%data% single-float (ibegin) ((1 *))
                            d-%offset%)
      (f2cl-lib:array-slice e-%data% single-float (ibegin) ((1 *))
                            e-%offset%)
      (f2cl-lib:array-slice e2-%data% single-float (ibegin)
                            ((1 *)) e2-%offset%)
      idumma
      (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float
                            ((+ n (f2cl-lib:int-mul 2 in) 1))
```

```
                          ((1 *)) work-%offset%)
     im iwork
     (f2cl-lib:array-slice w-%data% single-float ((+ m 1))
                          ((1 *)) w-%offset%)
     (f2cl-lib:array-slice iblock-%data% f2cl-lib:integer4
                          ((+ m 1)) ((1 *)) iblock-%offset%)
     iinfo)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
    var-9 var-10 var-11 var-12 var-13 var-14 var-16 var-17
    var-18))
  (setf im var-15)
  (setf iinfo var-19))
(cond ((/= iinfo 0) (setf info iinfo) (go end_label)))
(setf nwl
        (f2cl-lib:int-add nwl
                          (f2cl-lib:fref iwork-%data% (1) ((1 *))
                                         iwork-%offset%)))
(setf nwu
        (f2cl-lib:int-add nwu
                          (f2cl-lib:fref iwork-%data%
                                         ((f2cl-lib:int-add in
                                                            1))
                                         ((1 *))
                                         iwork-%offset%)))
(setf iwoff
        (f2cl-lib:int-sub m
                          (f2cl-lib:fref iwork-%data% (1) ((1 *))
                                         iwork-%offset%)))
(setf itmax
        (f2cl-lib:int-add
         (f2cl-lib:int
          (/
           (- (f2cl-lib:flog (+ (- gu gl) pivmin))
              (f2cl-lib:flog pivmin))
           (f2cl-lib:flog two)))
         2))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17
     var-18 var-19)
    (slaebz 2 itmax in in 1 nb atoli rtoli pivmin
     (f2cl-lib:array-slice d-%data% single-float (ibegin) ((1 *))
                          d-%offset%)
     (f2cl-lib:array-slice e-%data% single-float (ibegin) ((1 *))
                          e-%offset%)
     (f2cl-lib:array-slice e2-%data% single-float (ibegin)
                          ((1 *)) e2-%offset%)
     idumma
     (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
```

```
                              ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float
                              ((+ n (f2cl-lib:int-mul 2 in) 1))
                              ((1 *)) work-%offset%)
      iout iwork
      (f2cl-lib:array-slice w-%data% single-float ((+ m 1))
                              ((1 *)) w-%offset%)
      (f2cl-lib:array-slice iblock-%data% f2cl-lib:integer4
                              ((+ m 1)) ((1 *)) iblock-%offset%)
      iinfo)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12 var-13 var-14 var-16 var-17
      var-18))
    (setf iout var-15)
    (setf iinfo var-19))
  (cond ((/= iinfo 0) (setf info iinfo) (go end_label)))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j iout) nil)
    (tagbody
      (setf tmp1
              (* half
                 (+
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add j n)) ((1 *))
                                 work-%offset%)
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add j in n))
                                 ((1 *)) work-%offset%))))
      (setf tmp2
              (* half
                 (abs
                  (-
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add j n)) ((1 *))
                                  work-%offset%)
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add j in n))
                                  ((1 *)) work-%offset%)))))
      (cond
       ((> j (f2cl-lib:int-add iout (f2cl-lib:int-sub iinfo)))
        (setf ncnvrg f2cl-lib:%true%)
        (setf ib (f2cl-lib:int-sub jblk)))
       (t (setf ib jblk)))
      (f2cl-lib:fdo (je
                     (f2cl-lib:int-add
                      (f2cl-lib:fref iwork-%data% (j) ((1 *))
                                     iwork-%offset%)
                      1 iwoff)
                     (f2cl-lib:int-add je 1))
```

```
                          ((> je
                              (f2cl-lib:int-add
                               (f2cl-lib:fref iwork
                                              ((f2cl-lib:int-add j in))
                                              ((1 *)))
                               iwoff))
                           nil)
                 (tagbody
                   (setf (f2cl-lib:fref w-%data% (je) ((1 *)) w-%offset%)
                         tmp1)
                   (setf (f2cl-lib:fref werr-%data% (je) ((1 *))
                                        werr-%offset%)
                         tmp2)
                   (setf (f2cl-lib:fref indexw-%data% (je) ((1 *))
                                        indexw-%offset%)
                         (f2cl-lib:int-sub je iwoff))
                   (setf (f2cl-lib:fref iblock-%data% (je) ((1 *))
                                        iblock-%offset%)
                         ib)
                  label50))
              label60))
          (setf m (f2cl-lib:int-add m im))))
      label70))
    (cond
     ((= irange indrng) (setf idiscl (f2cl-lib:int-sub il 1 nwl))
      (setf idiscu (f2cl-lib:int-sub nwu iu))
      (cond
       ((> idiscl 0) (setf im 0)
        (f2cl-lib:fdo (je 1 (f2cl-lib:int-add je 1))
                      ((> je m) nil)
          (tagbody
            (cond
             ((and (<= (f2cl-lib:fref w (je) ((1 *))) wlu) (> idiscl 0))
              (setf idiscl (f2cl-lib:int-sub idiscl 1)))
             (t (setf im (f2cl-lib:int-add im 1))
              (setf (f2cl-lib:fref w-%data% (im) ((1 *)) w-%offset%)
                    (f2cl-lib:fref w-%data% (je) ((1 *)) w-%offset%))
              (setf (f2cl-lib:fref werr-%data% (im) ((1 *)) werr-%offset%)
                    (f2cl-lib:fref werr-%data% (je) ((1 *))
                                   werr-%offset%))
              (setf (f2cl-lib:fref indexw-%data% (im) ((1 *))
                                   indexw-%offset%)
                    (f2cl-lib:fref indexw-%data% (je) ((1 *))
                                   indexw-%offset%))
              (setf (f2cl-lib:fref iblock-%data% (im) ((1 *))
                                   iblock-%offset%)
                    (f2cl-lib:fref iblock-%data% (je) ((1 *))
                                   iblock-%offset%))))
           label80))
        (setf m im)))
```

```
(cond
 ((> idiscu 0) (setf im (f2cl-lib:int-add m 1))
  (f2cl-lib:fdo (je m (f2cl-lib:int-add je (f2cl-lib:int-sub 1)))
                ((> je 1) nil)
    (tagbody
      (cond
       ((and (>= (f2cl-lib:fref w (je) ((1 *))) wul) (> idiscu 0))
        (setf idiscu (f2cl-lib:int-sub idiscu 1)))
       (t (setf im (f2cl-lib:int-sub im 1))
        (setf (f2cl-lib:fref w-%data% (im) ((1 *)) w-%offset%)
                (f2cl-lib:fref w-%data% (je) ((1 *)) w-%offset%))
        (setf (f2cl-lib:fref werr-%data% (im) ((1 *)) werr-%offset%)
                (f2cl-lib:fref werr-%data% (je) ((1 *))
                                werr-%offset%))
        (setf (f2cl-lib:fref indexw-%data% (im) ((1 *))
                              indexw-%offset%)
                (f2cl-lib:fref indexw-%data% (je) ((1 *))
                                indexw-%offset%))
        (setf (f2cl-lib:fref iblock-%data% (im) ((1 *))
                              iblock-%offset%)
                (f2cl-lib:fref iblock-%data% (je) ((1 *))
                                iblock-%offset%))))
     label81))
  (setf jee 0)
  (f2cl-lib:fdo (je im (f2cl-lib:int-add je 1))
                ((> je m) nil)
    (tagbody
      (setf jee (f2cl-lib:int-add jee 1))
      (setf (f2cl-lib:fref w-%data% (jee) ((1 *)) w-%offset%)
              (f2cl-lib:fref w-%data% (je) ((1 *)) w-%offset%))
      (setf (f2cl-lib:fref werr-%data% (jee) ((1 *)) werr-%offset%)
              (f2cl-lib:fref werr-%data% (je) ((1 *)) werr-%offset%))
      (setf (f2cl-lib:fref indexw-%data% (jee) ((1 *))
                            indexw-%offset%)
              (f2cl-lib:fref indexw-%data% (je) ((1 *))
                              indexw-%offset%))
      (setf (f2cl-lib:fref iblock-%data% (jee) ((1 *))
                            iblock-%offset%)
              (f2cl-lib:fref iblock-%data% (je) ((1 *))
                              iblock-%offset%))
     label82))
  (setf m (f2cl-lib:int-add (f2cl-lib:int-sub m im) 1))))
(cond
 ((or (> idiscl 0) (> idiscu 0))
  (cond
   ((> idiscl 0) (setf wkill wu)
    (f2cl-lib:fdo (jdisc 1 (f2cl-lib:int-add jdisc 1))
                  ((> jdisc idiscl) nil)
      (tagbody
        (setf iw 0)
```

```
      (f2cl-lib:fdo (je 1 (f2cl-lib:int-add je 1))
                    ((> je m) nil)
        (tagbody
          (cond
           ((and (/= (f2cl-lib:fref iblock (je) ((1 *))) 0)
                 (or (< (f2cl-lib:fref w (je) ((1 *))) wkill)
                     (= iw 0)))
            (setf iw je)
            (setf wkill
                    (f2cl-lib:fref w-%data% (je) ((1 *))
                                   w-%offset%))))
           label90))
      (setf (f2cl-lib:fref iblock-%data% (iw) ((1 *))
                           iblock-%offset%)
              0)
     label100))))
(cond
 ((> idiscu 0) (setf wkill wl)
  (f2cl-lib:fdo (jdisc 1 (f2cl-lib:int-add jdisc 1))
                ((> jdisc idiscu) nil)
    (tagbody
      (setf iw 0)
      (f2cl-lib:fdo (je 1 (f2cl-lib:int-add je 1))
                    ((> je m) nil)
        (tagbody
          (cond
           ((and (/= (f2cl-lib:fref iblock (je) ((1 *))) 0)
                 (or (>= (f2cl-lib:fref w (je) ((1 *))) wkill)
                     (= iw 0)))
            (setf iw je)
            (setf wkill
                    (f2cl-lib:fref w-%data% (je) ((1 *))
                                   w-%offset%))))
           label110))
      (setf (f2cl-lib:fref iblock-%data% (iw) ((1 *))
                           iblock-%offset%)
              0)
     label120))))
(setf im 0)
(f2cl-lib:fdo (je 1 (f2cl-lib:int-add je 1))
              ((> je m) nil)
  (tagbody
    (cond
     ((/= (f2cl-lib:fref iblock (je) ((1 *))) 0)
      (setf im (f2cl-lib:int-add im 1))
      (setf (f2cl-lib:fref w-%data% (im) ((1 *)) w-%offset%)
              (f2cl-lib:fref w-%data% (je) ((1 *)) w-%offset%))
      (setf (f2cl-lib:fref werr-%data% (im) ((1 *)) werr-%offset%)
              (f2cl-lib:fref werr-%data% (je) ((1 *))
                             werr-%offset%))
```

```
          (setf (f2cl-lib:fref indexw-%data% (im) ((1 *))
                               indexw-%offset%)
                  (f2cl-lib:fref indexw-%data% (je) ((1 *))
                                 indexw-%offset%))
          (setf (f2cl-lib:fref iblock-%data% (im) ((1 *))
                               iblock-%offset%)
                  (f2cl-lib:fref iblock-%data% (je) ((1 *))
                                 iblock-%offset%))))
      label130))
    (setf m im)))
  (cond
   ((or (< idiscl 0) (< idiscu 0)) (setf toofew f2cl-lib:%true%)))))
(cond
 ((or (and (= irange allrng) (/= m n))
      (and (= irange indrng)
           (/= m (f2cl-lib:int-add iu (f2cl-lib:int-sub il) 1))))
  (setf toofew f2cl-lib:%true%)))
(cond
 ((and
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame order "E")
     (declare (ignore var-1))
     (when var-0 (setf order var-0))
     ret-val)
   (> nsplit 1))
  (f2cl-lib:fdo (je 1 (f2cl-lib:int-add je 1))
                ((> je (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (setf ie 0)
      (setf tmp1 (f2cl-lib:fref w-%data% (je) ((1 *)) w-%offset%))
      (f2cl-lib:fdo (j (f2cl-lib:int-add je 1) (f2cl-lib:int-add j 1))
                    ((> j m) nil)
        (tagbody
          (cond
           ((< (f2cl-lib:fref w (j) ((1 *))) tmp1) (setf ie j)
            (setf tmp1
                    (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%))))
         label140))
      (cond
       ((/= ie 0)
        (setf tmp2
                (f2cl-lib:fref werr-%data% (ie) ((1 *)) werr-%offset%))
        (setf itmp1
                (f2cl-lib:fref iblock-%data% (ie) ((1 *))
                               iblock-%offset%))
        (setf itmp2
                (f2cl-lib:fref indexw-%data% (ie) ((1 *))
                               indexw-%offset%))
        (setf (f2cl-lib:fref w-%data% (ie) ((1 *)) w-%offset%)
                (f2cl-lib:fref w-%data% (je) ((1 *)) w-%offset%))
```

```
                (setf (f2cl-lib:fref werr-%data% (ie) ((1 *)) werr-%offset%)
                        (f2cl-lib:fref werr-%data% (je) ((1 *)) werr-%offset%))
                (setf (f2cl-lib:fref iblock-%data% (ie) ((1 *))
                                     iblock-%offset%)
                        (f2cl-lib:fref iblock-%data% (je) ((1 *))
                                       iblock-%offset%))
                (setf (f2cl-lib:fref indexw-%data% (ie) ((1 *))
                                     indexw-%offset%)
                        (f2cl-lib:fref indexw-%data% (je) ((1 *))
                                       indexw-%offset%))
                (setf (f2cl-lib:fref w-%data% (je) ((1 *)) w-%offset%) tmp1)
                (setf (f2cl-lib:fref werr-%data% (je) ((1 *)) werr-%offset%)
                        tmp2)
                (setf (f2cl-lib:fref iblock-%data% (je) ((1 *))
                                     iblock-%offset%)
                        itmp1)
                (setf (f2cl-lib:fref indexw-%data% (je) ((1 *))
                                     indexw-%offset%)
                        itmp2)))
             label150))))
        (setf info 0)
        (if ncnvrg
            (setf info (f2cl-lib:int-add info 1)))
        (if toofew
            (setf info (f2cl-lib:int-add info 2)))
        (go end_label)
       end_label
        (return
         (values range order n nil nil nil nil nil nil nil nil nil nil nil nil
                 m nil nil wl wu nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slarrd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string)
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (single-float)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (single-float)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
```

```
   (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (single-float) (single-float)
  (array fortran-to-lisp::integer4
   (*))
  (array fortran-to-lisp::integer4
   (*))
  (array single-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::range
  fortran-to-lisp::order
  fortran-to-lisp::n nil nil nil
  nil nil nil nil nil nil nil nil
  nil fortran-to-lisp::m nil nil
  fortran-to-lisp::wl
  fortran-to-lisp::wu nil nil nil
  nil fortran-to-lisp::info)
:calls
'(fortran-to-lisp::slaebz
  fortran-to-lisp::ilaenv))))
```

———-

90.82 slarre

90.82.1 Axiom unit tests

— slarre.input —

```
)set break resume
)sys rm -f slarre.output
)spool slarre.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.82.2 Axiom help page

— slarre.help —

```
=====================================================================
slarre examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slarre (RANGE, N, VL, VU, IL, IU, D, E, E2, RTOL1, RTOL2,
           SPLTOL, NSPLIT, ISPLIT, M, W, WERR, WGAP, IBLOCK, INDEXW, GERS,
           PIVMIN, WORK, IWORK, INFO)
           SLARRE given the tridiagonal matrix T, sets small off-diagonal
           elements to zero and for each unreduced block Ti, finds base
           representations and eigenvalues.

Function/Subroutine Documentation
   subroutine slarre (character RANGE, integer N, real VL, real VU, integer
       IL, integer IU, real, dimension( * ) D, real, dimension( * ) E, real,
       dimension( * ) E2, real RTOL1, real RTOL2, real SPLTOL, integer NSPLIT,
       integer, dimension( * ) ISPLIT, integer M, real, dimension( * ) W,
       real, dimension( * ) WERR, real, dimension( * ) WGAP, integer,
       dimension( * ) IBLOCK, integer, dimension( * ) INDEXW, real, dimension(
       * ) GERS, real PIVMIN, real, dimension( * ) WORK, integer, dimension( *
       ) IWORK, integer INFO)
       SLARRE given the tridiagonal matrix T, sets small off-diagonal elements
       to zero and for each unreduced block Ti, finds base representations and
       eigenvalues.

Purpose:
   To find the desired eigenvalues of a given real symmetric
   tridiagonal matrix T, SLARRE sets any "small" off-diagonal
   elements to zero, and for each unreduced block T_i, it finds
   (a) a suitable shift at one end of the block's spectrum,
   (b) the base representation, T_i - sigma_i I = L_i D_i L_i^T, and
   (c) eigenvalues of each L_i D_i L_i^T.
   The representations and eigenvalues found are then used by
   SSTEMR to compute the eigenvectors of T.
   The accuracy varies depending on whether bisection is used to
   find a few eigenvalues or the dqds algorithm (subroutine SLASQ2) to
   conpute all and then discard any unwanted one.
   As an added benefit, SLARRE also outputs the n
   Gerschgorin intervals for the matrices L_i D_i L_i^T.
```

```
Parameters:

 RANGE
   RANGE is CHARACTER*1
   = 'A': ("All")   all eigenvalues will be found.
   = 'V': ("Value") all eigenvalues in the half-open interval
          (VL, VU] will be found.
   = 'I': ("Index") the IL-th through IU-th eigenvalues (of the
           entire matrix) will be found.

 N
   N is INTEGER
   The order of the matrix. N > 0.

 VL
   VL is REAL

 VU
   VU is REAL
   If RANGE='V', the lower and upper bounds for the eigenvalues.
   Eigenvalues less than or equal to VL, or greater than VU,
   will not be returned.  VL < VU.
   If RANGE='I' or ='A', SLARRE computes bounds on the desired
   part of the spectrum.

 IL
   IL is INTEGER

 IU
   IU is INTEGER
   If RANGE='I', the indices (in ascending order) of the
   smallest and largest eigenvalues to be returned.
   1 <= IL <= IU <= N.

 D
   D is REAL array, dimension (N)
   On entry, the N diagonal elements of the tridiagonal matrix T.
   On exit, the N diagonal elements of the diagonal matrices D_i.

 E
   E is REAL array, dimension (N)
   On entry, the first (N-1) entries contain the subdiagonal
   elements of the tridiagonal matrix T; E(N) need not be set.
   On exit, E contains the subdiagonal elements of the unit
   bidiagonal matrices L_i. The entries E( ISPLIT( I ) ),
   1 <= I <= NSPLIT, contain the base points sigma_i on output.

 E2
   E2 is REAL array, dimension (N)
   On entry, the first (N-1) entries contain the SQUARES of the
```

```
  subdiagonal elements of the tridiagonal matrix T;
  E2(N) need not be set.
  On exit, the entries E2( ISPLIT( I ) ),
  1 <= I <= NSPLIT, have been set to zero

RTOL1
  RTOL1 is REAL

RTOL2
  RTOL2 is REAL
  Parameters for bisection.
  An interval [LEFT,RIGHT] has converged if
  RIGHT-LEFT.LT.MAX( RTOL1*GAP, RTOL2*MAX(|LEFT|,|RIGHT|) )

SPLTOL
  SPLTOL is REAL
  The threshold for splitting.

NSPLIT
  NSPLIT is INTEGER
  The number of blocks T splits into. 1 <= NSPLIT <= N.

ISPLIT
  ISPLIT is INTEGER array, dimension (N)
  The splitting points, at which T breaks up into blocks.
  The first block consists of rows/columns 1 to ISPLIT(1),
  the second of rows/columns ISPLIT(1)+1 through ISPLIT(2),
  etc., and the NSPLIT-th consists of rows/columns
  ISPLIT(NSPLIT-1)+1 through ISPLIT(NSPLIT)=N.

M
  M is INTEGER
  The total number of eigenvalues (of all L_i D_i L_i^T) found.

W
  W is REAL array, dimension (N)
  The first M elements contain the eigenvalues. The
  eigenvalues of each of the blocks, L_i D_i L_i^T, are
  sorted in ascending order ( SLARRE may use the
  remaining N-M elements as workspace).

WERR
  WERR is REAL array, dimension (N)
  The error bound on the corresponding eigenvalue in W.

WGAP
  WGAP is REAL array, dimension (N)
  The separation from the right neighbor eigenvalue in W.
  The gap is only with respect to the eigenvalues of the same block
  as each block has its own representation tree.
```

```
  Exception: at the right end of a block we store the left gap

IBLOCK
  IBLOCK is INTEGER array, dimension (N)
  The indices of the blocks (submatrices) associated with the
  corresponding eigenvalues in W; IBLOCK(i)=1 if eigenvalue
  W(i) belongs to the first block from the top, =2 if W(i)
  belongs to the second block, etc.

INDEXW
  INDEXW is INTEGER array, dimension (N)
  The indices of the eigenvalues within each block (submatrix);
  for example, INDEXW(i)= 10 and IBLOCK(i)=2 imply that the
  i-th eigenvalue W(i) is the 10-th eigenvalue in block 2

GERS
  GERS is REAL array, dimension (2*N)
  The N Gerschgorin intervals (the i-th Gerschgorin interval
  is (GERS(2*i-1), GERS(2*i)).

PIVMIN
  PIVMIN is REAL
  The minimum pivot in the Sturm sequence for T.

WORK
  WORK is REAL array, dimension (6*N)
  Workspace.

IWORK
  IWORK is INTEGER array, dimension (5*N)
  Workspace.

INFO
  INFO is INTEGER
  = 0:  successful exit
  > 0:  A problem occured in SLARRE.
  < 0:  One of the called subroutines signaled an internal problem.
        Needs inspection of the corresponding parameter IINFO
        for further information.

  =-1:  Problem in SLARRD.
  = 2:  No base representation could be found in MAXTRY iterations.
        Increasing MAXTRY and recompilation might be a remedy.
  =-3:  Problem in SLARRB when computing the refined root
        representation for SLASQ2.
  =-4:  Problem in SLARRB when preforming bisection on the
        desired part of the spectrum.
  =-5:  Problem in SLASQ2.
  =-6:  Problem in SLASQ2.
```

```
Further Details:

   The base representations are required to suffer very little
   element growth and consequently define all their eigenvalues to
   high relative accuracy.
```

———-

90.82.3 fortran code

— **slarre.f** —

```
*  =====================================================================
      SUBROUTINE SLARRE( RANGE, N, VL, VU, IL, IU, D, E, E2,
     $                   RTOL1, RTOL2, SPLTOL, NSPLIT, ISPLIT, M,
     $                   W, WERR, WGAP, IBLOCK, INDEXW, GERS, PIVMIN,
     $                   WORK, IWORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          RANGE
      INTEGER            IL, INFO, IU, M, N, NSPLIT
      REAL               PIVMIN, RTOL1, RTOL2, SPLTOL, VL, VU
*     ..
*     .. Array Arguments ..
      INTEGER            IBLOCK( * ), ISPLIT( * ), IWORK( * ),
     $                   INDEXW( * )
      REAL               D( * ), E( * ), E2( * ), GERS( * ),
     $                   W( * ),WERR( * ), WGAP( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               FAC, FOUR, FOURTH, FUDGE, HALF, HNDRD,
     $                   MAXGROWTH, ONE, PERT, TWO, ZERO
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0,
     $                     TWO = 2.0E0, FOUR=4.0E0,
     $                     HNDRD = 100.0E0,
     $                     PERT = 4.0E0,
     $                     HALF = ONE/TWO, FOURTH = ONE/FOUR, FAC= HALF,
     $                     MAXGROWTH = 64.0E0, FUDGE = 2.0E0 )
      INTEGER            MAXTRY, ALLRNG, INDRNG, VALRNG
```

```
      PARAMETER          ( MAXTRY = 6, ALLRNG = 1, INDRNG = 2,
     $                     VALRNG = 3 )
*     ..
*     .. Local Scalars ..
      LOGICAL            FORCEB, NOREP, USEDQD
      INTEGER            CNT, CNT1, CNT2, I, IBEGIN, IDUM, IEND, IINFO,
     $                   IN, INDL, INDU, IRANGE, J, JBLK, MB, MM,
     $                   WBEGIN, WEND
      REAL               AVGAP, BSRTOL, CLWDTH, DMAX, DPIVOT, EABS,
     $                   EMAX, EOLD, EPS, GL, GU, ISLEFT, ISRGHT, RTL,
     $                   RTOL, S1, S2, SAFMIN, SGNDEF, SIGMA, SPDIAM,
     $                   TAU, TMP, TMP1


*     ..
*     .. Local Arrays ..
      INTEGER            ISEED( 4 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL                        SLAMCH
      EXTERNAL           SLAMCH, LSAME

*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SLARNV, SLARRA, SLARRB, SLARRC, SLARRD,
     $                   SLASQ2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN

*     ..
*     .. Executable Statements ..
*

      INFO = 0

*
*     Decode RANGE
*
      IF( LSAME( RANGE, 'A' ) ) THEN
         IRANGE = ALLRNG
      ELSE IF( LSAME( RANGE, 'V' ) ) THEN
         IRANGE = VALRNG
      ELSE IF( LSAME( RANGE, 'I' ) ) THEN
         IRANGE = INDRNG
      END IF

      M = 0
```

```
*     Get machine constants
      SAFMIN = SLAMCH( 'S' )
      EPS = SLAMCH( 'P' )

*     Set parameters
      RTL = HNDRD*EPS
*     If one were ever to ask for less initial precision in BSRTOL,
*     one should keep in mind that for the subset case, the extremal
*     eigenvalues must be at least as accurate as the current setting
*     (eigenvalues in the middle need not as much accuracy)
      BSRTOL = SQRT(EPS)*(0.5E-3)

*     Treat case of 1x1 matrix for quick return
      IF( N.EQ.1 ) THEN
         IF( (IRANGE.EQ.ALLRNG).OR.
     $       ((IRANGE.EQ.VALRNG).AND.(D(1).GT.VL).AND.(D(1).LE.VU)).OR.
     $       ((IRANGE.EQ.INDRNG).AND.(IL.EQ.1).AND.(IU.EQ.1)) ) THEN
            M = 1
            W(1) = D(1)
*           The computation error of the eigenvalue is zero
            WERR(1) = ZERO
            WGAP(1) = ZERO
            IBLOCK( 1 ) = 1
            INDEXW( 1 ) = 1
            GERS(1) = D( 1 )
            GERS(2) = D( 1 )
         ENDIF
*        store the shift for the initial RRR, which is zero in this case
         E(1) = ZERO
         RETURN
      END IF

*     General case: tridiagonal matrix of order > 1
*
*     Init WERR, WGAP. Compute Gerschgorin intervals and spectral diameter.
*     Compute maximum off-diagonal entry and pivmin.
      GL = D(1)
      GU = D(1)
      EOLD = ZERO
      EMAX = ZERO
      E(N) = ZERO
      DO 5 I = 1,N
         WERR(I) = ZERO
         WGAP(I) = ZERO
         EABS = ABS( E(I) )
         IF( EABS .GE. EMAX ) THEN
            EMAX = EABS
         END IF
         TMP1 = EABS + EOLD
         GERS( 2*I-1) = D(I) - TMP1
```

```
         GL =  MIN( GL, GERS( 2*I - 1))
         GERS( 2*I ) = D(I) + TMP1
         GU = MAX( GU, GERS(2*I) )
         EOLD  = EABS
 5    CONTINUE
*     The minimum pivot allowed in the Sturm sequence for T
      PIVMIN = SAFMIN * MAX( ONE, EMAX**2 )
*     Compute spectral diameter. The Gerschgorin bounds give an
*     estimate that is wrong by at most a factor of SQRT(2)
      SPDIAM = GU - GL

*     Compute splitting points
      CALL SLARRA( N, D, E, E2, SPLTOL, SPDIAM,
     $                    NSPLIT, ISPLIT, IINFO )

*     Can force use of bisection instead of faster DQDS.
*     Option left in the code for future multisection work.
      FORCEB = .FALSE.

*     Initialize USEDQD, DQDS should be used for ALLRNG unless someone
*     explicitly wants bisection.
      USEDQD = (( IRANGE.EQ.ALLRNG ) .AND. (.NOT.FORCEB))

      IF( (IRANGE.EQ.ALLRNG) .AND. (.NOT. FORCEB) ) THEN
*        Set interval [VL,VU] that contains all eigenvalues
         VL = GL
         VU = GU
      ELSE
*        We call SLARRD to find crude approximations to the eigenvalues
*        in the desired range. In case IRANGE = INDRNG, we also obtain the
*        interval (VL,VU] that contains all the wanted eigenvalues.
*        An interval [LEFT,RIGHT] has converged if
*        RIGHT-LEFT.LT.RTOL*MAX(ABS(LEFT),ABS(RIGHT))
*        SLARRD needs a WORK of size 4*N, IWORK of size 3*N
         CALL SLARRD( RANGE, 'B', N, VL, VU, IL, IU, GERS,
     $                    BSRTOL, D, E, E2, PIVMIN, NSPLIT, ISPLIT,
     $                    MM, W, WERR, VL, VU, IBLOCK, INDEXW,
     $                    WORK, IWORK, IINFO )
         IF( IINFO.NE.0 ) THEN
            INFO = -1
            RETURN
         ENDIF
*        Make sure that the entries M+1 to N in W, WERR, IBLOCK, INDEXW are 0
         DO 14 I = MM+1,N
            W( I ) = ZERO
            WERR( I ) = ZERO
            IBLOCK( I ) = 0
            INDEXW( I ) = 0
 14      CONTINUE
      END IF
```

```
***
*     Loop over unreduced blocks
      IBEGIN = 1
      WBEGIN = 1
      DO 170 JBLK = 1, NSPLIT
         IEND = ISPLIT( JBLK )
         IN = IEND - IBEGIN + 1

*        1 X 1 block
         IF( IN.EQ.1 ) THEN
            IF( (IRANGE.EQ.ALLRNG).OR.( (IRANGE.EQ.VALRNG).AND.
     $         ( D( IBEGIN ).GT.VL ).AND.( D( IBEGIN ).LE.VU ) )
     $        .OR. ( (IRANGE.EQ.INDRNG).AND.(IBLOCK(WBEGIN).EQ.JBLK))
     $        ) THEN
               M = M + 1
               W( M ) = D( IBEGIN )
               WERR(M) = ZERO
*              The gap for a single block doesn't matter for the later
*              algorithm and is assigned an arbitrary large value
               WGAP(M) = ZERO
               IBLOCK( M ) = JBLK
               INDEXW( M ) = 1
               WBEGIN = WBEGIN + 1
            ENDIF
*           E( IEND ) holds the shift for the initial RRR
            E( IEND ) = ZERO
            IBEGIN = IEND + 1
            GO TO 170
         END IF
*
*        Blocks of size larger than 1x1
*
*        E( IEND ) will hold the shift for the initial RRR, for now set it =0
         E( IEND ) = ZERO
*
*        Find local outer bounds GL,GU for the block
         GL = D(IBEGIN)
         GU = D(IBEGIN)
         DO 15 I = IBEGIN , IEND
            GL = MIN( GERS( 2*I-1 ), GL )
            GU = MAX( GERS( 2*I ), GU )
 15      CONTINUE
         SPDIAM = GU - GL

         IF(.NOT. ((IRANGE.EQ.ALLRNG).AND.(.NOT.FORCEB)) ) THEN
*           Count the number of eigenvalues in the current block.
            MB = 0
            DO 20 I = WBEGIN,MM
```

```
               IF( IBLOCK(I).EQ.JBLK ) THEN
                  MB = MB+1
               ELSE
                  GOTO 21
               ENDIF
 20         CONTINUE
 21         CONTINUE

            IF( MB.EQ.0) THEN
*              No eigenvalue in the current block lies in the desired range
*              E( IEND ) holds the shift for the initial RRR
               E( IEND ) = ZERO
               IBEGIN = IEND + 1
               GO TO 170
            ELSE

*              Decide whether dqds or bisection is more efficient
               USEDQD = ( (MB .GT. FAC*IN) .AND. (.NOT.FORCEB) )
               WEND = WBEGIN + MB - 1
*              Calculate gaps for the current block
*              In later stages, when representations for individual
*              eigenvalues are different, we use SIGMA = E( IEND ).
               SIGMA = ZERO
               DO 30 I = WBEGIN, WEND - 1
                  WGAP( I ) = MAX( ZERO,
     $                       W(I+1)-WERR(I+1) - (W(I)+WERR(I)) )
 30            CONTINUE
               WGAP( WEND ) = MAX( ZERO,
     $                    VU - SIGMA - (W( WEND )+WERR( WEND )))
*              Find local index of the first and last desired evalue.
               INDL = INDEXW(WBEGIN)
               INDU = INDEXW( WEND )
            ENDIF
         ENDIF
         IF(( (IRANGE.EQ.ALLRNG) .AND. (.NOT. FORCEB) ).OR.USEDQD) THEN
*           Case of DQDS
*           Find approximations to the extremal eigenvalues of the block
            CALL SLARRK( IN, 1, GL, GU, D(IBEGIN),
     $              E2(IBEGIN), PIVMIN, RTL, TMP, TMP1, IINFO )
            IF( IINFO.NE.0 ) THEN
               INFO = -1
               RETURN
            ENDIF
            ISLEFT = MAX(GL, TMP - TMP1
     $              - HNDRD * EPS* ABS(TMP - TMP1))

            CALL SLARRK( IN, IN, GL, GU, D(IBEGIN),
     $              E2(IBEGIN), PIVMIN, RTL, TMP, TMP1, IINFO )
            IF( IINFO.NE.0 ) THEN
               INFO = -1
```

```
            RETURN
         ENDIF
         ISRGHT = MIN(GU, TMP + TMP1
     $                + HNDRD * EPS * ABS(TMP + TMP1))
*        Improve the estimate of the spectral diameter
         SPDIAM = ISRGHT - ISLEFT
      ELSE
*        Case of bisection
*        Find approximations to the wanted extremal eigenvalues
         ISLEFT = MAX(GL, W(WBEGIN) - WERR(WBEGIN)
     $                 - HNDRD * EPS*ABS(W(WBEGIN)- WERR(WBEGIN) ))
         ISRGHT = MIN(GU,W(WEND) + WERR(WEND)
     $                 + HNDRD * EPS * ABS(W(WEND)+ WERR(WEND)))
      ENDIF


*     Decide whether the base representation for the current block
*     L_JBLK D_JBLK L_JBLK^T = T_JBLK - sigma_JBLK I
*     should be on the left or the right end of the current block.
*     The strategy is to shift to the end which is "more populated"
*     Furthermore, decide whether to use DQDS for the computation of
*     the eigenvalue approximations at the end of SLARRE or bisection.
*     dqds is chosen if all eigenvalues are desired or the number of
*     eigenvalues to be computed is large compared to the blocksize.
      IF( ( IRANGE.EQ.ALLRNG ) .AND. (.NOT.FORCEB) ) THEN
*        If all the eigenvalues have to be computed, we use dqd
         USEDQD = .TRUE.
*        INDL is the local index of the first eigenvalue to compute
         INDL = 1
         INDU = IN
*        MB =  number of eigenvalues to compute
         MB = IN
         WEND = WBEGIN + MB - 1
*        Define 1/4 and 3/4 points of the spectrum
         S1 = ISLEFT + FOURTH * SPDIAM
         S2 = ISRGHT - FOURTH * SPDIAM
      ELSE
*        SLARRD has computed IBLOCK and INDEXW for each eigenvalue
*        approximation.
*        choose sigma
         IF( USEDQD ) THEN
            S1 = ISLEFT + FOURTH * SPDIAM
            S2 = ISRGHT - FOURTH * SPDIAM
         ELSE
            TMP = MIN(ISRGHT,VU) -  MAX(ISLEFT,VL)
            S1 =  MAX(ISLEFT,VL) + FOURTH * TMP
            S2 =  MIN(ISRGHT,VU) - FOURTH * TMP
         ENDIF
      ENDIF
```

```
*        Compute the negcount at the 1/4 and 3/4 points
         IF(MB.GT.1) THEN
            CALL SLARRC( 'T', IN, S1, S2, D(IBEGIN),
     $                   E(IBEGIN), PIVMIN, CNT, CNT1, CNT2, IINFO)
         ENDIF

         IF(MB.EQ.1) THEN
            SIGMA = GL
            SGNDEF = ONE
         ELSEIF( CNT1 - INDL .GE. INDU - CNT2 ) THEN
            IF( ( IRANGE.EQ.ALLRNG ) .AND. (.NOT.FORCEB) ) THEN
               SIGMA = MAX(ISLEFT,GL)
            ELSEIF( USEDQD ) THEN
*              use Gerschgorin bound as shift to get pos def matrix
*              for dqds
               SIGMA = ISLEFT
            ELSE
*              use approximation of the first desired eigenvalue of the
*              block as shift
               SIGMA = MAX(ISLEFT,VL)
            ENDIF
            SGNDEF = ONE
         ELSE
            IF( ( IRANGE.EQ.ALLRNG ) .AND. (.NOT.FORCEB) ) THEN
               SIGMA = MIN(ISRGHT,GU)
            ELSEIF( USEDQD ) THEN
*              use Gerschgorin bound as shift to get neg def matrix
*              for dqds
               SIGMA = ISRGHT
            ELSE
*              use approximation of the first desired eigenvalue of the
*              block as shift
               SIGMA = MIN(ISRGHT,VU)
            ENDIF
            SGNDEF = -ONE
         ENDIF


*        An initial SIGMA has been chosen that will be used for computing
*        T - SIGMA I = L D L^T
*        Define the increment TAU of the shift in case the initial shift
*        needs to be refined to obtain a factorization with not too much
*        element growth.
         IF( USEDQD ) THEN
*           The initial SIGMA was to the outer end of the spectrum
*           the matrix is definite and we need not retreat.
            TAU = SPDIAM*EPS*N + TWO*PIVMIN
            TAU = MAX( TAU,TWO*EPS*ABS(SIGMA) )
         ELSE
            IF(MB.GT.1) THEN
```

```
               CLWDTH = W(WEND) + WERR(WEND) - W(WBEGIN) - WERR(WBEGIN)
               AVGAP = ABS(CLWDTH / REAL(WEND-WBEGIN))
               IF( SGNDEF.EQ.ONE ) THEN
                  TAU = HALF*MAX(WGAP(WBEGIN),AVGAP)
                  TAU = MAX(TAU,WERR(WBEGIN))
               ELSE
                  TAU = HALF*MAX(WGAP(WEND-1),AVGAP)
                  TAU = MAX(TAU,WERR(WEND))
               ENDIF
            ELSE
               TAU = WERR(WBEGIN)
            ENDIF
         ENDIF
*
         DO 80 IDUM = 1, MAXTRY
*           Compute L D L^T factorization of tridiagonal matrix T - sigma I.
*           Store D in WORK(1:IN), L in WORK(IN+1:2*IN), and reciprocals of
*           pivots in WORK(2*IN+1:3*IN)
            DPIVOT = D( IBEGIN ) - SIGMA
            WORK( 1 ) = DPIVOT
            DMAX = ABS( WORK(1) )
            J = IBEGIN
            DO 70 I = 1, IN - 1
               WORK( 2*IN+I ) = ONE / WORK( I )
               TMP = E( J )*WORK( 2*IN+I )
               WORK( IN+I ) = TMP
               DPIVOT = ( D( J+1 )-SIGMA ) - TMP*E( J )
               WORK( I+1 ) = DPIVOT
               DMAX = MAX( DMAX, ABS(DPIVOT) )
               J = J + 1
 70         CONTINUE
*           check for element growth
            IF( DMAX .GT. MAXGROWTH*SPDIAM ) THEN
               NOREP = .TRUE.
            ELSE
               NOREP = .FALSE.
            ENDIF
            IF( USEDQD .AND. .NOT.NOREP ) THEN
*              Ensure the definiteness of the representation
*              All entries of D (of L D L^T) must have the same sign
               DO 71 I = 1, IN
                  TMP = SGNDEF*WORK( I )
                  IF( TMP.LT.ZERO ) NOREP = .TRUE.
 71            CONTINUE
            ENDIF
            IF(NOREP) THEN
*              Note that in the case of IRANGE=ALLRNG, we use the Gerschgorin
*              shift which makes the matrix definite. So we should end up
*              here really only in the case of IRANGE = VALRNG or INDRNG.
               IF( IDUM.EQ.MAXTRY-1 ) THEN
```

```
                  IF( SGNDEF.EQ.ONE ) THEN
*                    The fudged Gerschgorin shift should succeed
                     SIGMA =
     $                    GL - FUDGE*SPDIAM*EPS*N - FUDGE*TWO*PIVMIN
                  ELSE
                     SIGMA =
     $                    GU + FUDGE*SPDIAM*EPS*N + FUDGE*TWO*PIVMIN
                  END IF
               ELSE
                  SIGMA = SIGMA - SGNDEF * TAU
                  TAU = TWO * TAU
               END IF
            ELSE
*              an initial RRR is found
               GO TO 83
            END IF
 80      CONTINUE
*        if the program reaches this point, no base representation could be
*        found in MAXTRY iterations.
         INFO = 2
         RETURN

 83      CONTINUE
*        At this point, we have found an initial base representation
*        T - SIGMA I = L D L^T with not too much element growth.
*        Store the shift.
         E( IEND ) = SIGMA
*        Store D and L.
         CALL SCOPY( IN, WORK, 1, D( IBEGIN ), 1 )
         CALL SCOPY( IN-1, WORK( IN+1 ), 1, E( IBEGIN ), 1 )


         IF(MB.GT.1 ) THEN
*
*           Perturb each entry of the base representation by a small
*           (but random) relative amount to overcome difficulties with
*           glued matrices.
*
            DO 122 I = 1, 4
               ISEED( I ) = 1
 122        CONTINUE

            CALL SLARNV(2, ISEED, 2*IN-1, WORK(1))
            DO 125 I = 1,IN-1
               D(IBEGIN+I-1) = D(IBEGIN+I-1)*(ONE+EPS*PERT*WORK(I))
               E(IBEGIN+I-1) = E(IBEGIN+I-1)*(ONE+EPS*PERT*WORK(IN+I))
 125        CONTINUE
            D(IEND) = D(IEND)*(ONE+EPS*FOUR*WORK(IN))
*
         ENDIF
```

```
*
*        Don't update the Gerschgorin intervals because keeping track
*        of the updates would be too much work in SLARRV.
*        We update W instead and use it to locate the proper Gerschgorin
*        intervals.

*        Compute the required eigenvalues of L D L' by bisection or dqds
         IF ( .NOT.USEDQD ) THEN
*           If SLARRD has been used, shift the eigenvalue approximations
*           according to their representation. This is necessary for
*           a uniform SLARRV since dqds computes eigenvalues of the
*           shifted representation. In SLARRV, W will always hold the
*           UNshifted eigenvalue approximation.
            DO 134 J=WBEGIN,WEND
               W(J) = W(J) - SIGMA
               WERR(J) = WERR(J) + ABS(W(J)) * EPS
 134        CONTINUE
*           call SLARRB to reduce eigenvalue error of the approximations
*           from SLARRD
            DO 135 I = IBEGIN, IEND-1
               WORK( I ) = D( I ) * E( I )**2
 135        CONTINUE
*           use bisection to find EV from INDL to INDU
            CALL SLARRB(IN, D(IBEGIN), WORK(IBEGIN),
     $                  INDL, INDU, RTOL1, RTOL2, INDL-1,
     $                  W(WBEGIN), WGAP(WBEGIN), WERR(WBEGIN),
     $                  WORK( 2*N+1 ), IWORK, PIVMIN, SPDIAM,
     $                  IN, IINFO )
            IF( IINFO .NE. 0 ) THEN
               INFO = -4
               RETURN
            END IF
*           SLARRB computes all gaps correctly except for the last one
*           Record distance to VU/GU
            WGAP( WEND ) = MAX( ZERO,
     $           ( VU-SIGMA ) - ( W( WEND ) + WERR( WEND ) ) )
            DO 138 I = INDL, INDU
               M = M + 1
               IBLOCK(M) = JBLK
               INDEXW(M) = I
 138        CONTINUE
         ELSE
*           Call dqds to get all eigs (and then possibly delete unwanted
*           eigenvalues).
*           Note that dqds finds the eigenvalues of the L D L^T representation
*           of T to high relative accuracy. High relative accuracy
*           might be lost when the shift of the RRR is subtracted to obtain
*           the eigenvalues of T. However, T is not guaranteed to define its
*           eigenvalues to high relative accuracy anyway.
*           Set RTOL to the order of the tolerance used in SLASQ2
```

```
*           This is an ESTIMATED error, the worst case bound is 4*N*EPS
*           which is usually too large and requires unnecessary work to be
*           done by bisection when computing the eigenvectors
            RTOL = LOG(REAL(IN)) * FOUR * EPS
            J = IBEGIN
            DO 140 I = 1, IN - 1
               WORK( 2*I-1 ) = ABS( D( J ) )
               WORK( 2*I ) = E( J )*E( J )*WORK( 2*I-1 )
               J = J + 1
  140       CONTINUE
            WORK( 2*IN-1 ) = ABS( D( IEND ) )
            WORK( 2*IN ) = ZERO
            CALL SLASQ2( IN, WORK, IINFO )
            IF( IINFO .NE. 0 ) THEN
*              If IINFO = -5 then an index is part of a tight cluster
*              and should be changed. The index is in IWORK(1) and the
*              gap is in WORK(N+1)
               INFO = -5
               RETURN
            ELSE
*              Test that all eigenvalues are positive as expected
               DO 149 I = 1, IN
                  IF( WORK( I ).LT.ZERO ) THEN
                     INFO = -6
                     RETURN
                  ENDIF
 149           CONTINUE
            END IF
            IF( SGNDEF.GT.ZERO ) THEN
               DO 150 I = INDL, INDU
                  M = M + 1
                  W( M ) = WORK( IN-I+1 )
                  IBLOCK( M ) = JBLK
                  INDEXW( M ) = I
 150           CONTINUE
            ELSE
               DO 160 I = INDL, INDU
                  M = M + 1
                  W( M ) = -WORK( I )
                  IBLOCK( M ) = JBLK
                  INDEXW( M ) = I
 160           CONTINUE
            END IF

            DO 165 I = M - MB + 1, M
*              the value of RTOL below should be the tolerance in SLASQ2
               WERR( I ) = RTOL * ABS( W(I) )
 165        CONTINUE
            DO 166 I = M - MB + 1, M - 1
*              compute the right gap between the intervals
```

```
              WGAP( I ) = MAX( ZERO,
     $                          W(I+1)-WERR(I+1) - (W(I)+WERR(I)) )
 166        CONTINUE
            WGAP( M ) = MAX( ZERO,
     $           ( VU-SIGMA ) - ( W( M ) + WERR( M ) ) )
         END IF
*        proceed with next block
         IBEGIN = IEND + 1
         WBEGIN = WEND + 1
 170  CONTINUE
*

      RETURN
*
*     end of SLARRE
*
      END
```

———-

90.82.4 lisp code

— slarre.lisp —

```
(let* ((zero 0.0)
       (one 1.0)
       (two 2.0)
       (four 4.0)
       (hndrd 100.0)
       (pert 4.0)
       (half (f2cl-lib:f2cl/ one two))
       (fourth$ (f2cl-lib:f2cl/ one four))
       (fac half)
       (maxgrowth 64.0)
       (fudge 2.0)
       (maxtry 6)
       (allrng 1)
       (indrng 2)
       (valrng 3))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 4.0 4.0) four)
           (type (single-float 100.0 100.0) hndrd)
           (type (single-float 4.0 4.0) pert)
           (type (single-float) half)
           (type (single-float) fourth$)
           (type (single-float) fac)
```

```
          (type (single-float 64.0 64.0) maxgrowth)
          (type (single-float 2.0 2.0) fudge)
          (type (f2cl-lib:integer4 6 6) maxtry)
          (type (f2cl-lib:integer4 1 1) allrng)
          (type (f2cl-lib:integer4 2 2) indrng)
          (type (f2cl-lib:integer4 3 3) valrng)
          (ignorable zero one two four hndrd pert half fourth$ fac maxgrowth
           fudge maxtry allrng indrng valrng))
(defun slarre
       (range n vl vu il iu d e e2 rtol1 rtol2 spltol nsplit isplit m w werr
        wgap iblock indexw gers pivmin work iwork info)
  (declare (type (array f2cl-lib:integer4 (*)) iwork indexw iblock isplit)
           (type (array single-float (*)) work gers wgap werr w e2 e d)
           (type (single-float) pivmin spltol rtol2 rtol1 vu vl)
           (type (f2cl-lib:integer4) info m nsplit iu il n)
           (type (string *) range))
  (f2cl-lib:with-multi-array-data
      ((range character range-%data% range-%offset%)
       (d single-float d-%data% d-%offset%)
       (e single-float e-%data% e-%offset%)
       (e2 single-float e2-%data% e2-%offset%)
       (w single-float w-%data% w-%offset%)
       (werr single-float werr-%data% werr-%offset%)
       (wgap single-float wgap-%data% wgap-%offset%)
       (gers single-float gers-%data% gers-%offset%)
       (work single-float work-%data% work-%offset%)
       (isplit f2cl-lib:integer4 isplit-%data% isplit-%offset%)
       (iblock f2cl-lib:integer4 iblock-%data% iblock-%offset%)
       (indexw f2cl-lib:integer4 indexw-%data% indexw-%offset%)
       (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
    (prog ((iseed (make-array 4 :element-type 'f2cl-lib:integer4))
           (avgap 0.0) (bsrtol 0.0) (clwdth 0.0) (dmax 0.0) (dpivot 0.0)
           (eabs 0.0) (emax 0.0) (eold 0.0) (eps 0.0) (gl 0.0) (gu 0.0)
           (isleft 0.0) (isrght 0.0) (rtl 0.0) (rtol 0.0) (s1 0.0) (s2 0.0)
           (safmin 0.0) (sgndef 0.0) (sigma 0.0) (spdiam 0.0) (tau 0.0)
           (tmp 0.0) (tmp1 0.0) (cnt 0) (cnt1 0) (cnt2 0) (i 0) (ibegin 0)
           (idum 0) (iend 0) (iinfo 0) (in 0) (indl 0) (indu 0) (irange 0)
           (j 0) (jblk 0) (mb 0) (mm 0) (wbegin 0) (wend 0) (forceb nil)
           (norep nil) (usedqd nil))
      (declare (type (array f2cl-lib:integer4 (4)) iseed)
               (type (single-float) avgap bsrtol clwdth dmax dpivot eabs emax
                eold eps gl gu isleft isrght rtl rtol s1 s2 safmin sgndef
                sigma spdiam tau tmp tmp1)
               (type (f2cl-lib:integer4) cnt cnt1 cnt2 i ibegin idum iend
                iinfo in indl indu irange j jblk mb mm wbegin wend)
               (type f2cl-lib:logical forceb norep usedqd))
      (setf info 0)
      (cond
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame range "A")
```

```
    (declare (ignore var-1))
    (when var-0 (setf range var-0))
    ret-val)
  (setf irange allrng))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame range "V")
    (declare (ignore var-1))
    (when var-0 (setf range var-0))
    ret-val)
  (setf irange valrng))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame range "I")
    (declare (ignore var-1))
    (when var-0 (setf range var-0))
    ret-val)
  (setf irange indrng)))
(setf m 0)
(setf safmin (slamch "S"))
(setf eps (slamch "P"))
(setf rtl (* hndrd eps))
(setf bsrtol (* (f2cl-lib:fsqrt eps) 5.e-4))
(cond
 ((= n 1)
  (cond
   ((or (= irange allrng)
        (and (= irange valrng) (> (f2cl-lib:fref d (1) ((1 *))) vl)
             (<= (f2cl-lib:fref d (1) ((1 *))) vu))
        (and (= irange indrng) (= il 1) (= iu 1)))
    (setf m 1)
    (setf (f2cl-lib:fref w-%data% (1) ((1 *)) w-%offset%)
            (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
    (setf (f2cl-lib:fref werr-%data% (1) ((1 *)) werr-%offset%) zero)
    (setf (f2cl-lib:fref wgap-%data% (1) ((1 *)) wgap-%offset%) zero)
    (setf (f2cl-lib:fref iblock-%data% (1) ((1 *)) iblock-%offset%) 1)
    (setf (f2cl-lib:fref indexw-%data% (1) ((1 *)) indexw-%offset%) 1)
    (setf (f2cl-lib:fref gers-%data% (1) ((1 *)) gers-%offset%)
            (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
    (setf (f2cl-lib:fref gers-%data% (2) ((1 *)) gers-%offset%)
            (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))))
  (setf (f2cl-lib:fref e-%data% (1) ((1 *)) e-%offset%) zero)
  (go end_label)))
(setf gl (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
(setf gu (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
(setf eold zero)
(setf emax zero)
(setf (f2cl-lib:fref e-%data% (n) ((1 *)) e-%offset%) zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref werr-%data% (i) ((1 *)) werr-%offset%) zero)
```

```
    (setf (f2cl-lib:fref wgap-%data% (i) ((1 *)) wgap-%offset%) zero)
    (setf eabs (abs (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
    (cond ((>= eabs emax) (setf emax eabs)))
    (setf tmp1 (+ eabs eold))
    (setf (f2cl-lib:fref gers-%data%
                         ((f2cl-lib:int-sub (f2cl-lib:int-mul 2 i) 1))
                         ((1 *)) gers-%offset%)
            (- (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) tmp1))
    (setf gl
            (min gl
                 (f2cl-lib:fref gers-%data%
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-mul 2 i) 1))
                                ((1 *)) gers-%offset%)))
    (setf (f2cl-lib:fref gers-%data% ((f2cl-lib:int-mul 2 i)) ((1 *))
                         gers-%offset%)
            (+ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) tmp1))
    (setf gu
            (max gu
                 (f2cl-lib:fref gers-%data% ((f2cl-lib:int-mul 2 i))
                                ((1 *)) gers-%offset%)))
    (setf eold eabs)
   label5))
(setf pivmin (* safmin (max one (expt emax 2))))
(setf spdiam (- gu gl))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (slarra n d e e2 spltol spdiam nsplit isplit iinfo)
  (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7))
  (setf nsplit var-6)
  (setf iinfo var-8))
(setf forceb f2cl-lib:%false%)
(setf usedqd (and (= irange allrng) (not forceb)))
(cond ((and (= irange allrng) (not forceb)) (setf vl gl) (setf vu gu))
      (t
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17
            var-18 var-19 var-20 var-21 var-22 var-23 var-24)
           (slarrd range "B" n vl vu il iu gers bsrtol d e e2 pivmin
            nsplit isplit mm w werr vl vu iblock indexw work iwork
            iinfo)
         (declare
          (ignore var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13 var-14 var-16 var-17 var-20
           var-21 var-22 var-23))
         (setf range var-0)
         (setf n var-2)
         (setf mm var-15)
         (setf vl var-18)
```

```
            (setf vu var-19)
            (setf iinfo var-24))
          (cond ((/= iinfo 0) (setf info -1) (go end_label)))
          (f2cl-lib:fdo (i (f2cl-lib:int-add mm 1) (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref w-%data% (i) ((1 *)) w-%offset%) zero)
              (setf (f2cl-lib:fref werr-%data% (i) ((1 *)) werr-%offset%)
                      zero)
              (setf (f2cl-lib:fref iblock-%data% (i) ((1 *))
                                   iblock-%offset%)
                      0)
              (setf (f2cl-lib:fref indexw-%data% (i) ((1 *))
                                   indexw-%offset%)
                      0)
           label14))))
(setf ibegin 1)
(setf wbegin 1)
(f2cl-lib:fdo (jblk 1 (f2cl-lib:int-add jblk 1))
              ((> jblk nsplit) nil)
  (tagbody
    (setf iend
            (f2cl-lib:fref isplit-%data% (jblk) ((1 *))
                           isplit-%offset%))
    (setf in (f2cl-lib:int-add (f2cl-lib:int-sub iend ibegin) 1))
    (cond
     ((= in 1)
      (cond
       ((or (= irange allrng)
            (and (= irange valrng)
                 (> (f2cl-lib:fref d (ibegin) ((1 *))) vl)
                 (<= (f2cl-lib:fref d (ibegin) ((1 *))) vu))
            (and (= irange indrng)
                 (= (f2cl-lib:fref iblock (wbegin) ((1 *))) jblk)))
        (setf m (f2cl-lib:int-add m 1))
        (setf (f2cl-lib:fref w-%data% (m) ((1 *)) w-%offset%)
                (f2cl-lib:fref d-%data% (ibegin) ((1 *)) d-%offset%))
        (setf (f2cl-lib:fref werr-%data% (m) ((1 *)) werr-%offset%)
                zero)
        (setf (f2cl-lib:fref wgap-%data% (m) ((1 *)) wgap-%offset%)
                zero)
        (setf (f2cl-lib:fref iblock-%data% (m) ((1 *)) iblock-%offset%)
                jblk)
        (setf (f2cl-lib:fref indexw-%data% (m) ((1 *)) indexw-%offset%)
                1)
        (setf wbegin (f2cl-lib:int-add wbegin 1))))
      (setf (f2cl-lib:fref e-%data% (iend) ((1 *)) e-%offset%) zero)
      (setf ibegin (f2cl-lib:int-add iend 1)) (go label170)))
    (setf (f2cl-lib:fref e-%data% (iend) ((1 *)) e-%offset%) zero)
    (setf gl (f2cl-lib:fref d-%data% (ibegin) ((1 *)) d-%offset%))
```

```
(setf gu (f2cl-lib:fref d-%data% (ibegin) ((1 *)) d-%offset%))
(f2cl-lib:fdo (i ibegin (f2cl-lib:int-add i 1))
              ((> i iend) nil)
  (tagbody
    (setf gl
            (min
             (f2cl-lib:fref gers-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-mul 2 i) 1))
                            ((1 *)) gers-%offset%)
             gl))
    (setf gu
            (max
             (f2cl-lib:fref gers-%data% ((f2cl-lib:int-mul 2 i))
                            ((1 *)) gers-%offset%)
             gu))
   label15))
(setf spdiam (- gu gl))
(cond
 ((not (and (= irange allrng) (not forceb)))
  (tagbody
    (setf mb 0)
    (f2cl-lib:fdo (i wbegin (f2cl-lib:int-add i 1))
                  ((> i mm) nil)
      (tagbody
        (cond
         ((= (f2cl-lib:fref iblock (i) ((1 *))) jblk)
          (setf mb (f2cl-lib:int-add mb 1)))
         (t (go label21)))
       label20))
   label21
    (cond
     ((= mb 0)
      (setf (f2cl-lib:fref e-%data% (iend) ((1 *)) e-%offset%)
              zero)
      (setf ibegin (f2cl-lib:int-add iend 1)) (go label170))
     (t (setf usedqd (and (> mb (* fac in)) (not forceb)))
      (setf wend (f2cl-lib:int-sub (f2cl-lib:int-add wbegin mb) 1))
      (setf sigma zero)
      (f2cl-lib:fdo (i wbegin (f2cl-lib:int-add i 1))
                    ((> i
                        (f2cl-lib:int-add wend
                                          (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref wgap-%data% (i) ((1 *))
                               wgap-%offset%)
                  (max zero
                       (-
                        (f2cl-lib:fref w-%data%
```

```
                                          ((f2cl-lib:int-add i 1))
                                          ((1 *)) w-%offset%)
                                 (f2cl-lib:fref werr-%data%
                                          ((f2cl-lib:int-add i 1))
                                          ((1 *)) werr-%offset%)
                                 (+
                                  (f2cl-lib:fref w-%data% (i) ((1 *))
                                           w-%offset%)
                                  (f2cl-lib:fref werr-%data% (i) ((1 *))
                                           werr-%offset%)))))
                 label30))
             (setf (f2cl-lib:fref wgap-%data% (wend) ((1 *))
                                  wgap-%offset%)
                   (max zero
                        (- vu sigma
                           (+
                            (f2cl-lib:fref w-%data% (wend) ((1 *))
                                           w-%offset%)
                            (f2cl-lib:fref werr-%data% (wend) ((1 *))
                                           werr-%offset%)))))
             (setf indl
                   (f2cl-lib:fref indexw-%data% (wbegin) ((1 *))
                                  indexw-%offset%))
             (setf indu
                   (f2cl-lib:fref indexw-%data% (wend) ((1 *))
                                  indexw-%offset%)))))))
        (cond
         ((or (and (= irange allrng) (not forceb)) usedqd)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10)
              (slarrk in 1 gl gu
               (f2cl-lib:array-slice d-%data% single-float (ibegin) ((1 *))
                                     d-%offset%)
               (f2cl-lib:array-slice e2-%data% single-float (ibegin)
                                     ((1 *)) e2-%offset%)
               pivmin rtl tmp tmp1 iinfo)
            (declare (ignore var-1 var-4 var-5))
            (when var-0 (setf in var-0))
            (when var-2 (setf gl var-2))
            (when var-3 (setf gu var-3))
            (when var-6 (setf pivmin var-6))
            (when var-7 (setf rtl var-7))
            (when var-8 (setf tmp var-8))
            (when var-9 (setf tmp1 var-9))
            (when var-10 (setf iinfo var-10)))
          (cond ((/= iinfo 0) (setf info -1) (go end_label)))
          (setf isleft
                (max gl
                     (+ (- tmp tmp1)
```

```
                (* (- hndrd) eps (abs (- tmp tmp1))))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (slarrk in in gl gu
      (f2cl-lib:array-slice d-%data% single-float (ibegin) ((1 *))
                            d-%offset%)
      (f2cl-lib:array-slice e2-%data% single-float (ibegin)
                            ((1 *)) e2-%offset%)
      pivmin rtl tmp tmp1 iinfo)
   (declare (ignore var-4 var-5))
   (when var-0 (setf in var-0))
   (when var-1 (setf in var-1))
   (when var-2 (setf gl var-2))
   (when var-3 (setf gu var-3))
   (when var-6 (setf pivmin var-6))
   (when var-7 (setf rtl var-7))
   (when var-8 (setf tmp var-8))
   (when var-9 (setf tmp1 var-9))
   (when var-10 (setf iinfo var-10)))
 (cond ((/= iinfo 0) (setf info -1) (go end_label)))
 (setf isrght
         (min gu (+ tmp tmp1 (* hndrd eps (abs (+ tmp tmp1))))))
 (setf spdiam (- isrght isleft)))
(t
 (setf isleft
         (max gl
              (+
               (-
                (f2cl-lib:fref w-%data% (wbegin) ((1 *))
                               w-%offset%)
                (f2cl-lib:fref werr-%data% (wbegin) ((1 *))
                               werr-%offset%))
               (* (- hndrd) eps
                  (abs
                   (-
                    (f2cl-lib:fref w-%data% (wbegin) ((1 *))
                                   w-%offset%)
                    (f2cl-lib:fref werr-%data% (wbegin) ((1 *))
                                   werr-%offset%)))))))
 (setf isrght
         (min gu
              (+
               (f2cl-lib:fref w-%data% (wend) ((1 *)) w-%offset%)
               (f2cl-lib:fref werr-%data% (wend) ((1 *))
                              werr-%offset%)
               (* hndrd eps
                  (abs
                   (+
                    (f2cl-lib:fref w-%data% (wend) ((1 *))
```

```
                                        w-%offset%)
                        (f2cl-lib:fref werr-%data% (wend) ((1 *))
                                        werr-%offset%)))))))))
(cond
 ((and (= irange allrng) (not forceb))
  (setf usedqd f2cl-lib:%true%) (setf indl 1) (setf indu in)
  (setf mb in)
  (setf wend (f2cl-lib:int-sub (f2cl-lib:int-add wbegin mb) 1))
  (setf s1 (+ isleft (* fourth$ spdiam)))
  (setf s2 (- isrght (* fourth$ spdiam))))
 (t
  (cond
   (usedqd (setf s1 (+ isleft (* fourth$ spdiam)))
    (setf s2 (- isrght (* fourth$ spdiam))))
   (t (setf tmp (- (min isrght vu) (max isleft vl)))
    (setf s1 (+ (max isleft vl) (* fourth$ tmp)))
    (setf s2 (- (min isrght vu) (* fourth$ tmp)))))))
(cond
 ((> mb 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (slarrc "T" in s1 s2
       (f2cl-lib:array-slice d-%data% single-float (ibegin) ((1 *))
                             d-%offset%)
       (f2cl-lib:array-slice e-%data% single-float (ibegin) ((1 *))
                             e-%offset%)
       pivmin cnt cnt1 cnt2 iinfo)
    (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6))
    (setf cnt var-7)
    (setf cnt1 var-8)
    (setf cnt2 var-9)
    (setf iinfo var-10))))
(cond ((= mb 1) (setf sigma gl) (setf sgndef one))
      ((>= (f2cl-lib:int-add cnt1 (f2cl-lib:int-sub indl))
           (f2cl-lib:int-add indu (f2cl-lib:int-sub cnt2)))
       (cond
        ((and (= irange allrng) (not forceb))
         (setf sigma (max isleft gl)))
        (usedqd (setf sigma isleft))
        (t (setf sigma (max isleft vl))))
       (setf sgndef one))
      (t
       (cond
        ((and (= irange allrng) (not forceb))
         (setf sigma (min isrght gu)))
        (usedqd (setf sigma isrght))
        (t (setf sigma (min isrght vu))))
       (setf sgndef (- one))))
(cond
```

```
(usedqd (setf tau (+ (* spdiam eps n) (* two pivmin)))
 (setf tau (max tau (* two eps (abs sigma)))))
(t
 (cond
  ((> mb 1)
   (setf clwdth
           (-
            (+ (f2cl-lib:fref w-%data% (wend) ((1 *)) w-%offset%)
               (f2cl-lib:fref werr-%data% (wend) ((1 *))
                              werr-%offset%))
            (f2cl-lib:fref w-%data% (wbegin) ((1 *)) w-%offset%)
            (f2cl-lib:fref werr-%data% (wbegin) ((1 *))
                           werr-%offset%)))
   (setf avgap
           (abs
            (/ clwdth
               (f2cl-lib:freal (f2cl-lib:int-sub wend wbegin)))))
   (cond
    ((= sgndef one)
     (setf tau
             (* half
                (max
                 (f2cl-lib:fref wgap-%data% (wbegin) ((1 *))
                                wgap-%offset%)
                 avgap)))
     (setf tau
             (max tau
                  (f2cl-lib:fref werr-%data% (wbegin) ((1 *))
                                 werr-%offset%))))
    (t
     (setf tau
             (* half
                (max
                 (f2cl-lib:fref wgap-%data%
                                ((f2cl-lib:int-sub wend 1))
                                ((1 *)) wgap-%offset%)
                 avgap)))
     (setf tau
             (max tau
                  (f2cl-lib:fref werr-%data% (wend) ((1 *))
                                 werr-%offset%))))))
  (t
   (setf tau
           (f2cl-lib:fref werr-%data% (wbegin) ((1 *))
                          werr-%offset%))))))
(f2cl-lib:fdo (idum 1 (f2cl-lib:int-add idum 1))
              ((> idum maxtry) nil)
 (tagbody
   (setf dpivot
           (- (f2cl-lib:fref d-%data% (ibegin) ((1 *)) d-%offset%)
```

```
           sigma))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        dpivot)
(setf dmax
        (abs
         (f2cl-lib:fref work-%data% (1) ((1 *))
                        work-%offset%)))
(setf j ibegin)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add in (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add
                           (f2cl-lib:int-mul 2 in) i))
                         ((1 *)) work-%offset%)
            (/ one
               (f2cl-lib:fref work-%data% (i) ((1 *))
                              work-%offset%)))
    (setf tmp
            (* (f2cl-lib:fref e-%data% (j) ((1 *)) e-%offset%)
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add
                                (f2cl-lib:int-mul 2 in) i))
                              ((1 *)) work-%offset%)))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add in i))
                         ((1 *)) work-%offset%)
            tmp)
    (setf dpivot
            (-
             (f2cl-lib:fref d-%data% ((f2cl-lib:int-add j 1))
                            ((1 *)) d-%offset%)
             sigma
             (* tmp
                (f2cl-lib:fref e-%data% (j) ((1 *))
                               e-%offset%))))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1))
                         ((1 *)) work-%offset%)
            dpivot)
    (setf dmax (max dmax (abs dpivot)))
    (setf j (f2cl-lib:int-add j 1))
   label70))
(cond
 ((> dmax (* maxgrowth spdiam)) (setf norep f2cl-lib:%true%))
 (t (setf norep f2cl-lib:%false%)))
(cond
 ((and usedqd (not norep))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i in) nil)
    (tagbody
```

```
            (setf tmp
                    (* sgndef
                       (f2cl-lib:fref work-%data% (i) ((1 *))
                                      work-%offset%)))
            (if (< tmp zero)
                (setf norep f2cl-lib:%true%))
          label71))))
      (cond
       (norep
        (cond
         ((= idum (f2cl-lib:int-add maxtry (f2cl-lib:int-sub 1)))
          (cond
           ((= sgndef one)
            (setf sigma
                    (+ gl (* (- fudge) spdiam eps n)
                       (* (- fudge) two pivmin))))
           (t
            (setf sigma
                    (+ gu (* fudge spdiam eps n)
                       (* fudge two pivmin))))))
         (t (setf sigma (- sigma (* sgndef tau)))
          (setf tau (* two tau)))))
       (t (go label83)))
     label80))
  (setf info 2)
  (go end_label)
 label83
  (setf (f2cl-lib:fref e-%data% (iend) ((1 *)) e-%offset%) sigma)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (scopy in work 1
       (f2cl-lib:array-slice d-%data% single-float (ibegin) ((1 *))
                             d-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf in var-0)))
  (scopy (f2cl-lib:int-sub in 1)
   (f2cl-lib:array-slice work-%data% single-float ((+ in 1)) ((1 *))
                         work-%offset%)
   1
   (f2cl-lib:array-slice e-%data% single-float (ibegin) ((1 *))
                         e-%offset%)
   1)
  (cond
   ((> mb 1)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i 4) nil)
      (tagbody (setf (f2cl-lib:fref iseed (i) ((1 4))) 1) label122))
    (slarnv 2 iseed (f2cl-lib:int-sub (f2cl-lib:int-mul 2 in) 1)
     (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                           work-%offset%))
```

```
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i (f2cl-lib:int-add in (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (setf (f2cl-lib:fref d-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add ibegin i) 1))
                           ((1 *)) d-%offset%)
              (*
               (f2cl-lib:fref d-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add ibegin i) 1))
                              ((1 *)) d-%offset%)
               (+ one
                  (* eps pert
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)))))
      (setf (f2cl-lib:fref e-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add ibegin i) 1))
                           ((1 *)) e-%offset%)
              (*
               (f2cl-lib:fref e-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add ibegin i) 1))
                              ((1 *)) e-%offset%)
               (+ one
                  (* eps pert
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add in i))
                                    ((1 *)) work-%offset%)))))
     label125))
  (setf (f2cl-lib:fref d-%data% (iend) ((1 *)) d-%offset%)
          (* (f2cl-lib:fref d-%data% (iend) ((1 *)) d-%offset%)
             (+ one
                (* eps four
                   (f2cl-lib:fref work-%data% (in) ((1 *))
                                  work-%offset%)))))))
(cond
 ((not usedqd)
  (f2cl-lib:fdo (j wbegin (f2cl-lib:int-add j 1))
                ((> j wend) nil)
    (tagbody
      (setf (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%)
              (- (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%)
                 sigma))
      (setf (f2cl-lib:fref werr-%data% (j) ((1 *)) werr-%offset%)
              (+
               (f2cl-lib:fref werr-%data% (j) ((1 *))
                              werr-%offset%)
```

```
              (*
               (abs
                (f2cl-lib:fref w-%data% (j) ((1 *)) w-%offset%))
               eps)))
   label134))
(f2cl-lib:fdo (i ibegin (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add iend (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (* (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
               (expt
                (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                2)))
   label135))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16)
    (slarrb in
     (f2cl-lib:array-slice d-%data% single-float (ibegin) ((1 *))
                           d-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (ibegin)
                           ((1 *)) work-%offset%)
     indl indu rtol1 rtol2 (f2cl-lib:int-sub indl 1)
     (f2cl-lib:array-slice w-%data% single-float (wbegin) ((1 *))
                           w-%offset%)
     (f2cl-lib:array-slice wgap-%data% single-float (wbegin)
                           ((1 *)) wgap-%offset%)
     (f2cl-lib:array-slice werr-%data% single-float (wbegin)
                           ((1 *)) werr-%offset%)
     (f2cl-lib:array-slice work-%data% single-float
                           ((+ (f2cl-lib:int-mul 2 n) 1)) ((1 *))
                           work-%offset%)
     iwork pivmin spdiam in iinfo)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
    var-9 var-10 var-11 var-12 var-13 var-14 var-15))
  (setf iinfo var-16))
(cond ((/= iinfo 0) (setf info -4) (go end_label)))
(setf (f2cl-lib:fref wgap-%data% (wend) ((1 *)) wgap-%offset%)
        (max zero
             (- vu sigma
                (+
                 (f2cl-lib:fref w-%data% (wend) ((1 *))
                                w-%offset%)
                 (f2cl-lib:fref werr-%data% (wend) ((1 *))
                                werr-%offset%)))))
(f2cl-lib:fdo (i indl (f2cl-lib:int-add i 1))
              ((> i indu) nil)
  (tagbody
```

```
      (setf m (f2cl-lib:int-add m 1))
      (setf (f2cl-lib:fref iblock-%data% (m) ((1 *))
                           iblock-%offset%)
              jblk)
      (setf (f2cl-lib:fref indexw-%data% (m) ((1 *))
                           indexw-%offset%)
              i)
    label138)))
 (t (setf rtol (* (f2cl-lib:flog (f2cl-lib:freal in)) four eps))
  (setf j ibegin)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i (f2cl-lib:int-add in (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-mul 2 i) 1))
                           ((1 *)) work-%offset%)
              (abs
               (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-mul 2 i))
                           ((1 *)) work-%offset%)
              (* (f2cl-lib:fref e-%data% (j) ((1 *)) e-%offset%)
                 (f2cl-lib:fref e-%data% (j) ((1 *)) e-%offset%)
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-mul 2 i) 1))
                                ((1 *)) work-%offset%)))
      (setf j (f2cl-lib:int-add j 1))
    label140))
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-sub (f2cl-lib:int-mul 2 in)
                                          1))
                       ((1 *)) work-%offset%)
         (abs (f2cl-lib:fref d-%data% (iend) ((1 *)) d-%offset%)))
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-mul 2 in))
                       ((1 *)) work-%offset%)
         zero)
  (multiple-value-bind (var-0 var-1 var-2)
      (slasq2 in work iinfo)
    (declare (ignore var-1))
    (when var-0 (setf in var-0))
    (when var-2 (setf iinfo var-2)))
  (cond ((/= iinfo 0) (setf info -5) (go end_label))
        (t
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i in) nil)
           (tagbody
             (cond
              ((< (f2cl-lib:fref work (i) ((1 *))) zero)
```

```
            (setf info -6) (go end_label)))
         label149))))
(cond
 ((> sgndef zero)
  (f2cl-lib:fdo (i indl (f2cl-lib:int-add i 1))
                ((> i indu) nil)
    (tagbody
      (setf m (f2cl-lib:int-add m 1))
      (setf (f2cl-lib:fref w-%data% (m) ((1 *)) w-%offset%)
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-sub in i) 1))
                             ((1 *)) work-%offset%))
      (setf (f2cl-lib:fref iblock-%data% (m) ((1 *))
                           iblock-%offset%)
              jblk)
      (setf (f2cl-lib:fref indexw-%data% (m) ((1 *))
                           indexw-%offset%)
              i)
    label150)))
 (t
  (f2cl-lib:fdo (i indl (f2cl-lib:int-add i 1))
                ((> i indu) nil)
    (tagbody
      (setf m (f2cl-lib:int-add m 1))
      (setf (f2cl-lib:fref w-%data% (m) ((1 *)) w-%offset%)
              (-
               (f2cl-lib:fref work-%data% (i) ((1 *))
                              work-%offset%)))
      (setf (f2cl-lib:fref iblock-%data% (m) ((1 *))
                           iblock-%offset%)
              jblk)
      (setf (f2cl-lib:fref indexw-%data% (m) ((1 *))
                           indexw-%offset%)
              i)
    label160))))
(f2cl-lib:fdo (i (f2cl-lib:int-add m (f2cl-lib:int-sub mb) 1)
               (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf (f2cl-lib:fref werr-%data% (i) ((1 *)) werr-%offset%)
            (* rtol
               (abs
                (f2cl-lib:fref w-%data% (i) ((1 *))
                               w-%offset%))))
   label165))
(f2cl-lib:fdo (i (f2cl-lib:int-add m (f2cl-lib:int-sub mb) 1)
               (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
               nil)
```

```
                (tagbody
                  (setf (f2cl-lib:fref wgap-%data% (i) ((1 *)) wgap-%offset%)
                          (max zero
                               (-
                                (f2cl-lib:fref w-%data%
                                               ((f2cl-lib:int-add i 1)) ((1 *))
                                               w-%offset%)
                                (f2cl-lib:fref werr-%data%
                                               ((f2cl-lib:int-add i 1)) ((1 *))
                                               werr-%offset%)
                                (+
                                 (f2cl-lib:fref w-%data% (i) ((1 *))
                                                w-%offset%)
                                 (f2cl-lib:fref werr-%data% (i) ((1 *))
                                                werr-%offset%)))))
                 label166))
               (setf (f2cl-lib:fref wgap-%data% (m) ((1 *)) wgap-%offset%)
                       (max zero
                            (- vu sigma
                               (+
                                (f2cl-lib:fref w-%data% (m) ((1 *)) w-%offset%)
                                (f2cl-lib:fref werr-%data% (m) ((1 *))
                                               werr-%offset%)))))))
            (setf ibegin (f2cl-lib:int-add iend 1))
            (setf wbegin (f2cl-lib:int-add wend 1))
           label170))
        (go end_label)
       end_label
        (return
         (values range n vl vu nil nil nil nil nil nil nil nil nsplit nil m nil
                 nil nil nil nil nil pivmin nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slarre
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (single-float)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
```

```
   (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (array fortran-to-lisp::integer4
   (*))
  (array single-float (*))
  (single-float)
  (array single-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::range
  fortran-to-lisp::n
  fortran-to-lisp::vl
  fortran-to-lisp::vu nil nil nil
  nil nil nil nil nil
  fortran-to-lisp::nsplit nil
  fortran-to-lisp::m nil nil nil
  nil nil nil
  fortran-to-lisp::pivmin nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::slarrb
  fortran-to-lisp::slarnv
  fortran-to-lisp::slarrc
  fortran-to-lisp::slarrd
  fortran-to-lisp::slarra))))
```

———-

90.83 slarrf

90.83.1 Axiom unit tests

— slarrf.input —

```
)set break resume
)sys rm -f slarrf.output
)spool slarrf.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.83.2 Axiom help page

— **slarrf.help** —

```
==================================================================
slarrf examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slarrf (N, D, L, LD, CLSTRT, CLEND, W, WGAP, WERR, SPDIAM,
           CLGAPL, CLGAPR, PIVMIN, SIGMA, DPLUS, LPLUS, WORK, INFO)
           SLARRF finds a new relatively robust representation such that at
           least one of the eigenvalues is relatively isolated.

Function/Subroutine Documentation
   subroutine slarrf (integer N, real, dimension( * ) D, real, dimension( * )
       L, real, dimension( * ) LD, integer CLSTRT, integer CLEND, real,
       dimension( * ) W, real, dimension( * ) WGAP, real, dimension( * ) WERR,
       real SPDIAM, real CLGAPL, real CLGAPR, real PIVMIN, real SIGMA, real,
       dimension( * ) DPLUS, real, dimension( * ) LPLUS, real, dimension( * )
       WORK, integer INFO)
       SLARRF finds a new relatively robust representation such that at least
       one of the eigenvalues is relatively isolated.

Purpose:
   Given the initial representation L D L^T and its cluster of close
   eigenvalues (in a relative measure), W( CLSTRT ), W( CLSTRT+1 ), ...
   W( CLEND ), SLARRF finds a new relatively robust representation
   L D L^T - SIGMA I = L(+) D(+) L(+)^T such that at least one of the
   eigenvalues of L(+) D(+) L(+)^T is relatively isolated.

Parameters:

 N
   N is INTEGER
```

```
  The order of the matrix (subblock, if the matrix splitted).

D
  D is REAL array, dimension (N)
  The N diagonal elements of the diagonal matrix D.

L
  L is REAL array, dimension (N-1)
  The (N-1) subdiagonal elements of the unit bidiagonal matrix L.

LD
  LD is REAL array, dimension (N-1)
  The (N-1) elements L(i)*D(i).

CLSTRT
  CLSTRT is INTEGER
  The index of the first eigenvalue in the cluster.

CLEND
  CLEND is INTEGER
  The index of the last eigenvalue in the cluster.

W
  W is REAL array, dimension
  dimension is >=  (CLEND-CLSTRT+1)
  The eigenvalue APPROXIMATIONS of L D L^T in ascending order.
  W( CLSTRT ) through W( CLEND ) form the cluster of relatively
  close eigenalues.

WGAP
  WGAP is REAL array, dimension
  dimension is >=  (CLEND-CLSTRT+1)
  The separation from the right neighbor eigenvalue in W.

WERR
  WERR is REAL array, dimension
  dimension is >=  (CLEND-CLSTRT+1)
  WERR contain the semiwidth of the uncertainty
  interval of the corresponding eigenvalue APPROXIMATION in W

SPDIAM
  SPDIAM is REAL
  estimate of the spectral diameter obtained from the Gerschgorin intervals

CLGAPL
  CLGAPL is REAL

CLGAPR
  CLGAPR is REAL
  absolute gap on each end of the cluster.
```

```
  Set by the calling routine to protect against shifts too close
  to eigenvalues outside the cluster.

PIVMIN
  PIVMIN is REAL
  The minimum pivot allowed in the Sturm sequence.

SIGMA
  SIGMA is REAL
  The shift used to form L(+) D(+) L(+)^T.

DPLUS
  DPLUS is REAL array, dimension (N)
  The N diagonal elements of the diagonal matrix D(+).

LPLUS
  LPLUS is REAL array, dimension (N-1)
  The first (N-1) elements of LPLUS contain the subdiagonal
  elements of the unit bidiagonal matrix L(+).

WORK
  WORK is REAL array, dimension (2*N)
  Workspace.

INFO
  INFO is INTEGER
  Signals processing OK (=0) or failure (=1)
```

———-

90.83.3 fortran code

— slarrf.f —

```
*  =====================================================================
      SUBROUTINE SLARRF( N, D, L, LD, CLSTRT, CLEND,
     $                   W, WGAP, WERR,
     $                   SPDIAM, CLGAPL, CLGAPR, PIVMIN, SIGMA,
     $                   DPLUS, LPLUS, WORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            CLSTRT, CLEND, INFO, N
```

```
      REAL               CLGAPL, CLGAPR, PIVMIN, SIGMA, SPDIAM
*     ..
*     .. Array Arguments ..
      REAL               D( * ), DPLUS( * ), L( * ), LD( * ),
     $          LPLUS( * ), W( * ), WGAP( * ), WERR( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               MAXGROWTH1, MAXGROWTH2, ONE, QUART, TWO
      PARAMETER          ( ONE = 1.0E0, TWO = 2.0E0,
     $                     QUART = 0.25E0,
     $                     MAXGROWTH1 = 8.E0,
     $                     MAXGROWTH2 = 8.E0 )
*     ..
*     .. Local Scalars ..
      LOGICAL   DORRR1, FORCER, NOFAIL, SAWNAN1, SAWNAN2, TRYRRR1
      INTEGER            I, INDX, KTRY, KTRYMAX, SLEFT, SRIGHT, SHIFT
      PARAMETER          ( KTRYMAX = 1, SLEFT = 1, SRIGHT = 2 )
      REAL               AVGAP, BESTSHIFT, CLWDTH, EPS, FACT, FAIL,
     $                   FAIL2, GROWTHBOUND, LDELTA, LDMAX, LSIGMA,
     $                   MAX1, MAX2, MINGAP, OLDP, PROD, RDELTA, RDMAX,
     $                   RRR1, RRR2, RSIGMA, S, SMLGROWTH, TMP, ZNM2
*     ..
*     .. External Functions ..
      LOGICAL SISNAN
      REAL               SLAMCH
      EXTERNAL           SISNAN, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      FACT = REAL(2**KTRYMAX)
      EPS = SLAMCH( 'Precision' )
      SHIFT = 0
      FORCER = .FALSE.


*     Note that we cannot guarantee that for any of the shifts tried,
*     the factorization has a small or even moderate element growth.
*     There could be Ritz values at both ends of the cluster and despite
*     backing off, there are examples where all factorizations tried
*     (in IEEE mode, allowing zero pivots & infinities) have INFINITE
```

```
*     element growth.
*     For this reason, we should use PIVMIN in this subroutine so that at
*     least the L D L^T factorization exists. It can be checked afterwards
*     whether the element growth caused bad residuals/orthogonality.

*     Decide whether the code should accept the best among all
*     representations despite large element growth or signal INFO=1
*     Setting NOFAIL to .FALSE. for quick fix for bug 113
      NOFAIL = .FALSE.
*

*     Compute the average gap length of the cluster
      CLWDTH = ABS(W(CLEND)-W(CLSTRT)) + WERR(CLEND) + WERR(CLSTRT)
      AVGAP = CLWDTH / REAL(CLEND-CLSTRT)
      MINGAP = MIN(CLGAPL, CLGAPR)
*     Initial values for shifts to both ends of cluster
      LSIGMA = MIN(W( CLSTRT ),W( CLEND )) - WERR( CLSTRT )
      RSIGMA = MAX(W( CLSTRT ),W( CLEND )) + WERR( CLEND )

*     Use a small fudge to make sure that we really shift to the outside
      LSIGMA = LSIGMA - ABS(LSIGMA)* TWO * EPS
      RSIGMA = RSIGMA + ABS(RSIGMA)* TWO * EPS

*     Compute upper bounds for how much to back off the initial shifts
      LDMAX = QUART * MINGAP + TWO * PIVMIN
      RDMAX = QUART * MINGAP + TWO * PIVMIN

      LDELTA = MAX(AVGAP,WGAP( CLSTRT ))/FACT
      RDELTA = MAX(AVGAP,WGAP( CLEND-1 ))/FACT
*
*     Initialize the record of the best representation found
*
      S = SLAMCH( 'S' )
      SMLGROWTH = ONE / S
      FAIL = REAL(N-1)*MINGAP/(SPDIAM*EPS)
      FAIL2 = REAL(N-1)*MINGAP/(SPDIAM*SQRT(EPS))
      BESTSHIFT = LSIGMA
*
*     while (KTRY <= KTRYMAX)
      KTRY = 0
      GROWTHBOUND = MAXGROWTH1*SPDIAM

 5    CONTINUE
      SAWNAN1 = .FALSE.
      SAWNAN2 = .FALSE.
*     Ensure that we do not back off too much of the initial shifts
      LDELTA = MIN(LDMAX,LDELTA)
      RDELTA = MIN(RDMAX,RDELTA)

*     Compute the element growth when shifting to both ends of the cluster
```

```
*     accept the shift if there is no element growth at one of the two ends

*     Left end
      S = -LSIGMA
      DPLUS( 1 ) = D( 1 ) + S
      IF(ABS(DPLUS(1)).LT.PIVMIN) THEN
         DPLUS(1) = -PIVMIN
*        Need to set SAWNAN1 because refined RRR test should not be used
*        in this case
         SAWNAN1 = .TRUE.
      ENDIF
      MAX1 = ABS( DPLUS( 1 ) )
      DO 6 I = 1, N - 1
         LPLUS( I ) = LD( I ) / DPLUS( I )
         S = S*LPLUS( I )*L( I ) - LSIGMA
         DPLUS( I+1 ) = D( I+1 ) + S
         IF(ABS(DPLUS(I+1)).LT.PIVMIN) THEN
            DPLUS(I+1) = -PIVMIN
*           Need to set SAWNAN1 because refined RRR test should not be used
*           in this case
            SAWNAN1 = .TRUE.
         ENDIF
         MAX1 = MAX( MAX1,ABS(DPLUS(I+1)) )
 6    CONTINUE
      SAWNAN1 = SAWNAN1 .OR.  SISNAN( MAX1 )

      IF( FORCER .OR.
     $   (MAX1.LE.GROWTHBOUND .AND. .NOT.SAWNAN1 ) ) THEN
         SIGMA = LSIGMA
         SHIFT = SLEFT
         GOTO 100
      ENDIF

*     Right end
      S = -RSIGMA
      WORK( 1 ) = D( 1 ) + S
      IF(ABS(WORK(1)).LT.PIVMIN) THEN
         WORK(1) = -PIVMIN
*        Need to set SAWNAN2 because refined RRR test should not be used
*        in this case
         SAWNAN2 = .TRUE.
      ENDIF
      MAX2 = ABS( WORK( 1 ) )
      DO 7 I = 1, N - 1
         WORK( N+I ) = LD( I ) / WORK( I )
         S = S*WORK( N+I )*L( I ) - RSIGMA
         WORK( I+1 ) = D( I+1 ) + S
         IF(ABS(WORK(I+1)).LT.PIVMIN) THEN
            WORK(I+1) = -PIVMIN
*           Need to set SAWNAN2 because refined RRR test should not be used
```

```
*           in this case
            SAWNAN2 = .TRUE.
         ENDIF
         MAX2 = MAX( MAX2,ABS(WORK(I+1)) )
 7    CONTINUE
      SAWNAN2 = SAWNAN2 .OR.  SISNAN( MAX2 )

      IF( FORCER .OR.
     $   (MAX2.LE.GROWTHBOUND .AND. .NOT.SAWNAN2 ) ) THEN
         SIGMA = RSIGMA
         SHIFT = SRIGHT
         GOTO 100
      ENDIF
*     If we are at this point, both shifts led to too much element growth

*     Record the better of the two shifts (provided it didn't lead to NaN)
      IF(SAWNAN1.AND.SAWNAN2) THEN
*        both MAX1 and MAX2 are NaN
         GOTO 50
      ELSE
         IF( .NOT.SAWNAN1 ) THEN
            INDX = 1
            IF(MAX1.LE.SMLGROWTH) THEN
               SMLGROWTH = MAX1
               BESTSHIFT = LSIGMA
            ENDIF
         ENDIF
         IF( .NOT.SAWNAN2 ) THEN
            IF(SAWNAN1 .OR. MAX2.LE.MAX1) INDX = 2
            IF(MAX2.LE.SMLGROWTH) THEN
               SMLGROWTH = MAX2
               BESTSHIFT = RSIGMA
            ENDIF
         ENDIF
      ENDIF

*     If we are here, both the left and the right shift led to
*     element growth. If the element growth is moderate, then
*     we may still accept the representation, if it passes a
*     refined test for RRR. This test supposes that no NaN occurred.
*     Moreover, we use the refined RRR test only for isolated clusters.
      IF((CLWDTH.LT.MINGAP/REAL(128)) .AND.
     $   (MIN(MAX1,MAX2).LT.FAIL2)
     $  .AND.(.NOT.SAWNAN1).AND.(.NOT.SAWNAN2)) THEN
         DORRR1 = .TRUE.
      ELSE
         DORRR1 = .FALSE.
      ENDIF
      TRYRRR1 = .TRUE.
      IF( TRYRRR1 .AND. DORRR1 ) THEN
```

```
      IF(INDX.EQ.1) THEN
         TMP = ABS( DPLUS( N ) )
         ZNM2 = ONE
         PROD = ONE
         OLDP = ONE
         DO 15 I = N-1, 1, -1
            IF( PROD .LE. EPS ) THEN
               PROD =
     $         ((DPLUS(I+1)*WORK(N+I+1))/(DPLUS(I)*WORK(N+I)))*OLDP
            ELSE
               PROD = PROD*ABS(WORK(N+I))
            END IF
            OLDP = PROD
            ZNM2 = ZNM2 + PROD**2
            TMP = MAX( TMP, ABS( DPLUS( I ) * PROD ))
 15      CONTINUE
         RRR1 = TMP/( SPDIAM * SQRT( ZNM2 ) )
         IF (RRR1.LE.MAXGROWTH2) THEN
            SIGMA = LSIGMA
            SHIFT = SLEFT
            GOTO 100
         ENDIF
      ELSE IF(INDX.EQ.2) THEN
         TMP = ABS( WORK( N ) )
         ZNM2 = ONE
         PROD = ONE
         OLDP = ONE
         DO 16 I = N-1, 1, -1
            IF( PROD .LE. EPS ) THEN
               PROD = ((WORK(I+1)*LPLUS(I+1))/(WORK(I)*LPLUS(I)))*OLDP
            ELSE
               PROD = PROD*ABS(LPLUS(I))
            END IF
            OLDP = PROD
            ZNM2 = ZNM2 + PROD**2
            TMP = MAX( TMP, ABS( WORK( I ) * PROD ))
 16      CONTINUE
         RRR2 = TMP/( SPDIAM * SQRT( ZNM2 ) )
         IF (RRR2.LE.MAXGROWTH2) THEN
            SIGMA = RSIGMA
            SHIFT = SRIGHT
            GOTO 100
         ENDIF
      END IF
      ENDIF

 50   CONTINUE

      IF (KTRY.LT.KTRYMAX) THEN
*        If we are here, both shifts failed also the RRR test.
```

```
*        Back off to the outside
         LSIGMA = MAX( LSIGMA - LDELTA,
     $     LSIGMA - LDMAX)
         RSIGMA = MIN( RSIGMA + RDELTA,
     $     RSIGMA + RDMAX )
         LDELTA = TWO * LDELTA
         RDELTA = TWO * RDELTA
         KTRY = KTRY + 1
         GOTO 5
      ELSE
*        None of the representations investigated satisfied our
*        criteria. Take the best one we found.
         IF((SMLGROWTH.LT.FAIL).OR.NOFAIL) THEN
            LSIGMA = BESTSHIFT
            RSIGMA = BESTSHIFT
            FORCER = .TRUE.
            GOTO 5
         ELSE
            INFO = 1
            RETURN
         ENDIF
      END IF

 100  CONTINUE
      IF (SHIFT.EQ.SLEFT) THEN
      ELSEIF (SHIFT.EQ.SRIGHT) THEN
*        store new L and D back into DPLUS, LPLUS
         CALL SCOPY( N, WORK, 1, DPLUS, 1 )
         CALL SCOPY( N-1, WORK(N+1), 1, LPLUS, 1 )
      ENDIF

      RETURN
*
*     End of SLARRF
*
      END
```

———-

90.83.4 lisp code

— slarrf.lisp —

```
(let* ((one 1.0)
       (two 2.0)
       (quart 0.25)
       (maxgrowth1 8.0)
       (maxgrowth2 8.0)
```

```
      (ktrymax 1)
      (sleft 1)
      (sright 2))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 0.25 0.25) quart)
           (type (single-float 8.0 8.0) maxgrowth1)
           (type (single-float 8.0 8.0) maxgrowth2)
           (type (f2cl-lib:integer4 1 1) ktrymax)
           (type (f2cl-lib:integer4 1 1) sleft)
           (type (f2cl-lib:integer4 2 2) sright)
           (ignorable one two quart maxgrowth1 maxgrowth2 ktrymax sleft
            sright))
  (defun slarrf
         (n d l ld clstrt clend w wgap werr spdiam clgapl clgapr pivmin sigma
          dplus lplus work info)
    (declare (type (single-float) sigma pivmin clgapr clgapl spdiam)
             (type (array single-float (*)) work lplus dplus werr wgap w ld l
              d)
             (type (f2cl-lib:integer4) info clend clstrt n))
    (f2cl-lib:with-multi-array-data
        ((d single-float d-%data% d-%offset%)
         (l single-float l-%data% l-%offset%)
         (ld single-float ld-%data% ld-%offset%)
         (w single-float w-%data% w-%offset%)
         (wgap single-float wgap-%data% wgap-%offset%)
         (werr single-float werr-%data% werr-%offset%)
         (dplus single-float dplus-%data% dplus-%offset%)
         (lplus single-float lplus-%data% lplus-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((avgap 0.0) (bestshift 0.0) (clwdth 0.0) (eps 0.0) (fact 0.0)
             (fail 0.0) (fail2 0.0) (growthbound 0.0) (ldelta 0.0) (ldmax 0.0)
             (lsigma 0.0) (f2cl-lib:max1 0.0) (max2 0.0) (mingap 0.0)
             (oldp 0.0) (prod 0.0) (rdelta 0.0) (rdmax 0.0) (rrr1 0.0)
             (rrr2 0.0) (rsigma 0.0) (s 0.0) (smlgrowth 0.0) (tmp 0.0)
             (znm2 0.0) (i 0) (indx 0) (ktry 0) (shift 0) (dorrr1 nil)
             (forcer nil) (nofail nil) (sawnan1 nil) (sawnan2 nil)
             (tryrrr1 nil))
        (declare
         (type (single-float) avgap bestshift clwdth eps fact fail fail2
          growthbound ldelta ldmax lsigma f2cl-lib:max1 max2 mingap oldp prod
          rdelta rdmax rrr1 rrr2 rsigma s smlgrowth tmp znm2)
         (type (f2cl-lib:integer4) i indx ktry shift)
         (type f2cl-lib:logical dorrr1 forcer nofail sawnan1 sawnan2 tryrrr1))
        (setf info 0)
        (setf fact (f2cl-lib:freal (expt 2 ktrymax)))
        (setf eps (slamch "Precision"))
        (setf shift 0)
        (setf forcer f2cl-lib:%false%)
        (setf nofail f2cl-lib:%false%)
```

```
 (setf clwdth
         (+
          (abs
           (- (f2cl-lib:fref w-%data% (clend) ((1 *)) w-%offset%)
              (f2cl-lib:fref w-%data% (clstrt) ((1 *)) w-%offset%)))
          (f2cl-lib:fref werr-%data% (clend) ((1 *)) werr-%offset%)
          (f2cl-lib:fref werr-%data% (clstrt) ((1 *)) werr-%offset%)))
 (setf avgap
         (/ clwdth (f2cl-lib:freal (f2cl-lib:int-sub clend clstrt))))
 (setf mingap (min clgapl clgapr))
 (setf lsigma
         (-
          (min (f2cl-lib:fref w-%data% (clstrt) ((1 *)) w-%offset%)
               (f2cl-lib:fref w-%data% (clend) ((1 *)) w-%offset%))
          (f2cl-lib:fref werr-%data% (clstrt) ((1 *)) werr-%offset%)))
 (setf rsigma
         (+
          (max (f2cl-lib:fref w-%data% (clstrt) ((1 *)) w-%offset%)
               (f2cl-lib:fref w-%data% (clend) ((1 *)) w-%offset%))
          (f2cl-lib:fref werr-%data% (clend) ((1 *)) werr-%offset%)))
 (setf lsigma (+ lsigma (* (- (abs lsigma)) two eps)))
 (setf rsigma (+ rsigma (* (abs rsigma) two eps)))
 (setf ldmax (+ (* quart mingap) (* two pivmin)))
 (setf rdmax (+ (* quart mingap) (* two pivmin)))
 (setf ldelta
         (/
          (max avgap
               (f2cl-lib:fref wgap-%data% (clstrt) ((1 *))
                              wgap-%offset%))
          fact))
 (setf rdelta
         (/
          (max avgap
               (f2cl-lib:fref wgap-%data% ((f2cl-lib:int-sub clend 1))
                              ((1 *)) wgap-%offset%))
          fact))
 (setf s (slamch "S"))
 (setf smlgrowth (/ one s))
 (setf fail
         (/ (* (f2cl-lib:freal (f2cl-lib:int-sub n 1)) mingap)
            (* spdiam eps)))
 (setf fail2
         (/ (* (f2cl-lib:freal (f2cl-lib:int-sub n 1)) mingap)
            (* spdiam (f2cl-lib:fsqrt eps))))
 (setf bestshift lsigma)
 (setf ktry 0)
 (setf growthbound (* maxgrowth1 spdiam))
label5
 (setf sawnan1 f2cl-lib:%false%)
 (setf sawnan2 f2cl-lib:%false%)
```

```
(setf ldelta (min ldmax ldelta))
(setf rdelta (min rdmax rdelta))
(setf s (- lsigma))
(setf (f2cl-lib:fref dplus-%data% (1) ((1 *)) dplus-%offset%)
        (+ (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%) s))
(cond
 ((< (abs (f2cl-lib:fref dplus (1) ((1 *)))) pivmin)
  (setf (f2cl-lib:fref dplus-%data% (1) ((1 *)) dplus-%offset%)
          (- pivmin))
  (setf sawnan1 f2cl-lib:%true%)))
(setf f2cl-lib:max1
        (abs (f2cl-lib:fref dplus-%data% (1) ((1 *)) dplus-%offset%)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (setf (f2cl-lib:fref lplus-%data% (i) ((1 *)) lplus-%offset%)
            (/ (f2cl-lib:fref ld-%data% (i) ((1 *)) ld-%offset%)
               (f2cl-lib:fref dplus-%data% (i) ((1 *))
                              dplus-%offset%)))
    (setf s
            (-
             (* s
                (f2cl-lib:fref lplus-%data% (i) ((1 *)) lplus-%offset%)
                (f2cl-lib:fref l-%data% (i) ((1 *)) l-%offset%))
             lsigma))
    (setf (f2cl-lib:fref dplus-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         dplus-%offset%)
            (+
             (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                            d-%offset%)
             s))
    (cond
     ((< (abs (f2cl-lib:fref dplus ((f2cl-lib:int-add i 1)) ((1 *))))
         pivmin)
      (setf (f2cl-lib:fref dplus-%data% ((f2cl-lib:int-add i 1))
                           ((1 *)) dplus-%offset%)
              (- pivmin))
      (setf sawnan1 f2cl-lib:%true%)))
    (setf f2cl-lib:max1
            (max f2cl-lib:max1
                 (abs
                  (f2cl-lib:fref dplus-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) dplus-%offset%))))
   label6))
(setf sawnan1 (or sawnan1 (sisnan f2cl-lib:max1)))
(cond
 ((or forcer (and (<= f2cl-lib:max1 growthbound) (not sawnan1)))
  (setf sigma lsigma) (setf shift sleft) (go label100)))
(setf s (- rsigma))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
```

```
        (+ (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%) s))
(cond
 ((< (abs (f2cl-lib:fref work (1) ((1 *)))) pivmin)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (- pivmin))
  (setf sawnan2 f2cl-lib:%true%)))
(setf max2 (abs (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i)) ((1 *))
                         work-%offset%)
            (/ (f2cl-lib:fref ld-%data% (i) ((1 *)) ld-%offset%)
               (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)))
    (setf s
            (-
             (* s
                (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%)
                (f2cl-lib:fref l-%data% (i) ((1 *)) l-%offset%))
             rsigma))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         work-%offset%)
            (+
             (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                            d-%offset%)
             s))
    (cond
     ((< (abs (f2cl-lib:fref work ((f2cl-lib:int-add i 1)) ((1 *))))
         pivmin)
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                           work-%offset%)
              (- pivmin))
      (setf sawnan2 f2cl-lib:%true%)))
    (setf max2
            (max max2
                 (abs
                  (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) work-%offset%))))
   label7))
(setf sawnan2 (or sawnan2 (sisnan max2)))
(cond
 ((or forcer (and (<= max2 growthbound) (not sawnan2)))
  (setf sigma rsigma) (setf shift sright) (go label100)))
(cond ((and sawnan1 sawnan2) (go label50))
      (t
       (cond
        ((not sawnan1) (setf indx 1)
         (cond
          ((<= f2cl-lib:max1 smlgrowth) (setf smlgrowth f2cl-lib:max1)
```

```
            (setf bestshift lsigma)))))
        (cond
         ((not sawnan2)
          (if (or sawnan1 (<= max2 f2cl-lib:max1))
              (setf indx 2))
          (cond
           ((<= max2 smlgrowth) (setf smlgrowth max2)
            (setf bestshift rsigma)))))))
(cond
 ((and (< clwdth (f2cl-lib:f2cl/ mingap (f2cl-lib:freal 128)))
       (< (min f2cl-lib:max1 max2) fail2) (not sawnan1) (not sawnan2))
  (setf dorrr1 f2cl-lib:%true%))
 (t (setf dorrr1 f2cl-lib:%false%)))
(setf tryrrr1 f2cl-lib:%true%)
(cond
 ((and tryrrr1 dorrr1)
  (cond
   ((= indx 1)
    (setf tmp
            (abs
             (f2cl-lib:fref dplus-%data% (n) ((1 *)) dplus-%offset%)))
    (setf znm2 one) (setf prod one) (setf oldp one)
    (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
      (tagbody
        (cond
         ((<= prod eps)
          (setf prod
                  (*
                   (/
                    (*
                     (f2cl-lib:fref dplus-%data%
                                    ((f2cl-lib:int-add i 1)) ((1 *))
                                    dplus-%offset%)
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add n i 1)) ((1 *))
                                    work-%offset%))
                    (*
                     (f2cl-lib:fref dplus-%data% (i) ((1 *))
                                    dplus-%offset%)
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add n i)) ((1 *))
                                    work-%offset%)))
                   oldp)))
         (t
          (setf prod
                  (* prod
                     (abs
                      (f2cl-lib:fref work-%data%
```

```
                                    ((f2cl-lib:int-add n i)) ((1 *))
                                    work-%offset%))))))
     (setf oldp prod)
     (setf znm2 (+ znm2 (expt prod 2)))
     (setf tmp
             (max tmp
                  (abs
                   (*
                    (f2cl-lib:fref dplus-%data% (i) ((1 *))
                                   dplus-%offset%)
                    prod))))
    label15))
 (setf rrr1 (/ tmp (* spdiam (f2cl-lib:fsqrt znm2))))
 (cond
  ((<= rrr1 maxgrowth2) (setf sigma lsigma) (setf shift sleft)
   (go label100))))
((= indx 2)
 (setf tmp
         (abs
          (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)))
 (setf znm2 one) (setf prod one) (setf oldp one)
 (f2cl-lib:fdo (i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
               ((> i 1) nil)
   (tagbody
     (cond
      ((<= prod eps)
       (setf prod
               (*
                (/
                 (*
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add i 1)) ((1 *))
                                 work-%offset%)
                  (f2cl-lib:fref lplus-%data%
                                 ((f2cl-lib:int-add i 1)) ((1 *))
                                 lplus-%offset%))
                 (*
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)
                  (f2cl-lib:fref lplus-%data% (i) ((1 *))
                                 lplus-%offset%)))
                oldp)))
      (t
       (setf prod
               (* prod
                  (abs
                   (f2cl-lib:fref lplus-%data% (i) ((1 *))
                                  lplus-%offset%))))))
     (setf oldp prod)
```

```
                (setf znm2 (+ znm2 (expt prod 2)))
                (setf tmp
                        (max tmp
                             (abs
                              (*
                               (f2cl-lib:fref work-%data% (i) ((1 *))
                                              work-%offset%)
                               prod))))
               label16))
            (setf rrr2 (/ tmp (* spdiam (f2cl-lib:fsqrt znm2))))
            (cond
             ((<= rrr2 maxgrowth2) (setf sigma rsigma) (setf shift sright)
              (go label100)))))))
       label50
        (cond
         ((< ktry ktrymax)
          (setf lsigma (max (- lsigma ldelta) (- lsigma ldmax)))
          (setf rsigma (min (+ rsigma rdelta) (+ rsigma rdmax)))
          (setf ldelta (* two ldelta)) (setf rdelta (* two rdelta))
          (setf ktry (f2cl-lib:int-add ktry 1)) (go label5))
         (t
          (cond
           ((or (< smlgrowth fail) nofail) (setf lsigma bestshift)
            (setf rsigma bestshift) (setf forcer f2cl-lib:%true%) (go label5))
           (t (setf info 1) (go end_label)))))
       label100
        (cond ((= shift sleft))
              ((= shift sright)
               (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                   (scopy n work 1 dplus 1)
                 (declare (ignore var-1 var-2 var-3 var-4))
                 (when var-0 (setf n var-0)))
               (scopy (f2cl-lib:int-sub n 1)
                (f2cl-lib:array-slice work-%data% single-float ((+ n 1))
                                      ((1 *)) work-%offset%)
                1 lplus 1)))
        (go end_label)
       end_label
        (return
         (values n nil nil nil nil nil nil nil nil nil nil nil nil sigma nil
                 nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slarrf
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array single-float (*))
```

```
  (array single-float (*))
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (single-float) (single-float)
  (single-float) (single-float)
  (single-float)
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::n nil nil nil
  nil nil nil nil nil nil nil nil
  nil fortran-to-lisp::sigma nil
  nil nil fortran-to-lisp::info)
:calls '(fortran-to-lisp::sisnan))))
```

———-

90.84 slarrj

90.84.1 Axiom unit tests

— **slarrj.input** —

```
)set break resume
)sys rm -f slarrj.output
)spool slarrj.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.84.2 Axiom help page

— slarrj.help —

```
==================================================================
slarrj examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slarrj (N, D, E2, IFIRST, ILAST, RTOL, OFFSET, W, WERR,
           WORK, IWORK, PIVMIN, SPDIAM, INFO)
           SLARRJ performs refinement of the initial estimates of the
           eigenvalues of the matrix T.

Function/Subroutine Documentation
   subroutine slarrj (integer N, real, dimension( * ) D, real, dimension( * )
       E2, integer IFIRST, integer ILAST, real RTOL, integer OFFSET, real,
       dimension( * ) W, real, dimension( * ) WERR, real, dimension( * ) WORK,
       integer, dimension( * ) IWORK, real PIVMIN, real SPDIAM, integer INFO)
       SLARRJ performs refinement of the initial estimates of the eigenvalues
       of the matrix T.

Purpose:
   Given the initial eigenvalue approximations of T, SLARRJ
   does  bisection to refine the eigenvalues of T,
   W( IFIRST-OFFSET ) through W( ILAST-OFFSET ), to more accuracy. Initial
   guesses for these eigenvalues are input in W, the corresponding estimate
   of the error in these guesses in WERR. During bisection, intervals
   [left, right] are maintained by storing their mid-points and
   semi-widths in the arrays W and WERR respectively.

Parameters:

 N
   N is INTEGER
   The order of the matrix.

 D
   D is REAL array, dimension (N)
   The N diagonal elements of T.

 E2
   E2 is REAL array, dimension (N-1)
   The Squares of the (N-1) subdiagonal elements of T.
```

```
IFIRST
  IFIRST is INTEGER
  The index of the first eigenvalue to be computed.

ILAST
  ILAST is INTEGER
  The index of the last eigenvalue to be computed.

RTOL
  RTOL is REAL
  Tolerance for the convergence of the bisection intervals.
  An interval [LEFT,RIGHT] has converged if
  RIGHT-LEFT.LT.RTOL*MAX(|LEFT|,|RIGHT|).

OFFSET
  OFFSET is INTEGER
  Offset for the arrays W and WERR, i.e., the IFIRST-OFFSET
  through ILAST-OFFSET elements of these arrays are to be used.

W
  W is REAL array, dimension (N)
  On input, W( IFIRST-OFFSET ) through W( ILAST-OFFSET ) are
  estimates of the eigenvalues of L D L^T indexed IFIRST through ILAST.
  On output, these estimates are refined.

WERR
  WERR is REAL array, dimension (N)
  On input, WERR( IFIRST-OFFSET ) through WERR( ILAST-OFFSET ) are
  the errors in the estimates of the corresponding elements in W.
  On output, these errors are refined.

WORK
  WORK is REAL array, dimension (2*N)
  Workspace.

IWORK
  IWORK is INTEGER array, dimension (2*N)
  Workspace.

PIVMIN
  PIVMIN is REAL
  The minimum pivot in the Sturm sequence for T.

SPDIAM
  SPDIAM is REAL
  The spectral diameter of T.

INFO
  INFO is INTEGER
```

Error flag.

———-

90.84.3 fortran code

— **slarrj.f** —

```
*  ===================================================================
      SUBROUTINE SLARRJ( N, D, E2, IFIRST, ILAST,
     $                   RTOL, OFFSET, W, WERR, WORK, IWORK,
     $                   PIVMIN, SPDIAM, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            IFIRST, ILAST, INFO, N, OFFSET
      REAL               PIVMIN, RTOL, SPDIAM
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      REAL               D( * ), E2( * ), W( * ),
     $                   WERR( * ), WORK( * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE, TWO, HALF
      PARAMETER        ( ZERO = 0.0E0, ONE = 1.0E0, TWO = 2.0E0,
     $                   HALF = 0.5E0 )
      INTEGER   MAXITR
*     ..
*     .. Local Scalars ..
      INTEGER            CNT, I, I1, I2, II, ITER, J, K, NEXT, NINT,
     $                   OLNINT, P, PREV, SAVI1
      REAL               DPLUS, FAC, LEFT, MID, RIGHT, S, TMP, WIDTH
*
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
```

```
      INFO = 0
*
      MAXITR = INT( ( LOG( SPDIAM+PIVMIN )-LOG( PIVMIN ) ) /
     $           LOG( TWO ) ) + 2
*
*     Initialize unconverged intervals in [ WORK(2*I-1), WORK(2*I) ].
*     The Sturm Count, Count( WORK(2*I-1) ) is arranged to be I-1, while
*     Count( WORK(2*I) ) is stored in IWORK( 2*I ). The integer IWORK( 2*I-1 )
*     for an unconverged interval is set to the index of the next unconverged
*     interval, and is -1 or 0 for a converged interval. Thus a linked
*     list of unconverged intervals is set up.
*

      I1 = IFIRST
      I2 = ILAST
*     The number of unconverged intervals
      NINT = 0
*     The last unconverged interval found
      PREV = 0
      DO 75 I = I1, I2
         K = 2*I
         II = I - OFFSET
         LEFT = W( II ) - WERR( II )
         MID = W(II)
         RIGHT = W( II ) + WERR( II )
         WIDTH = RIGHT - MID
         TMP = MAX( ABS( LEFT ), ABS( RIGHT ) )

*        The following test prevents the test of converged intervals
         IF( WIDTH.LT.RTOL*TMP ) THEN
*           This interval has already converged and does not need refinement.
*           (Note that the gaps might change through refining the
*            eigenvalues, however, they can only get bigger.)
*           Remove it from the list.
            IWORK( K-1 ) = -1
*           Make sure that I1 always points to the first unconverged interval
            IF((I.EQ.I1).AND.(I.LT.I2)) I1 = I + 1
            IF((PREV.GE.I1).AND.(I.LE.I2)) IWORK( 2*PREV-1 ) = I + 1
         ELSE
*           unconverged interval found
            PREV = I
*           Make sure that [LEFT,RIGHT] contains the desired eigenvalue
*
*           Do while( CNT(LEFT).GT.I-1 )
*
            FAC = ONE
 20         CONTINUE
            CNT = 0
            S = LEFT
            DPLUS = D( 1 ) - S
```

```
            IF( DPLUS.LT.ZERO ) CNT = CNT + 1
            DO 30 J = 2, N
               DPLUS = D( J ) - S - E2( J-1 )/DPLUS
               IF( DPLUS.LT.ZERO ) CNT = CNT + 1
 30         CONTINUE
            IF( CNT.GT.I-1 ) THEN
               LEFT = LEFT - WERR( II )*FAC
               FAC = TWO*FAC
               GO TO 20
            END IF
*
*           Do while( CNT(RIGHT).LT.I )
*
            FAC = ONE
 50         CONTINUE
            CNT = 0
            S = RIGHT
            DPLUS = D( 1 ) - S
            IF( DPLUS.LT.ZERO ) CNT = CNT + 1
            DO 60 J = 2, N
               DPLUS = D( J ) - S - E2( J-1 )/DPLUS
               IF( DPLUS.LT.ZERO ) CNT = CNT + 1
 60         CONTINUE
            IF( CNT.LT.I ) THEN
               RIGHT = RIGHT + WERR( II )*FAC
               FAC = TWO*FAC
               GO TO 50
            END IF
            NINT = NINT + 1
            IWORK( K-1 ) = I + 1
            IWORK( K ) = CNT
         END IF
         WORK( K-1 ) = LEFT
         WORK( K ) = RIGHT
 75   CONTINUE


      SAVI1 = I1
*
*     Do while( NINT.GT.0 ), i.e. there are still unconverged intervals
*     and while (ITER.LT.MAXITR)
*
      ITER = 0
 80   CONTINUE
      PREV = I1 - 1
      I = I1
      OLNINT = NINT

      DO 100 P = 1, OLNINT
         K = 2*I
```

```
         II = I - OFFSET
         NEXT = IWORK( K-1 )
         LEFT = WORK( K-1 )
         RIGHT = WORK( K )
         MID = HALF*( LEFT + RIGHT )

*        semiwidth of interval
         WIDTH = RIGHT - MID
         TMP = MAX( ABS( LEFT ), ABS( RIGHT ) )

         IF( ( WIDTH.LT.RTOL*TMP ) .OR.
     $      (ITER.EQ.MAXITR) )THEN
*           reduce number of unconverged intervals
            NINT = NINT - 1
*           Mark interval as converged.
            IWORK( K-1 ) = 0
            IF( I1.EQ.I ) THEN
               I1 = NEXT
            ELSE
*              Prev holds the last unconverged interval previously examined
               IF(PREV.GE.I1) IWORK( 2*PREV-1 ) = NEXT
            END IF
            I = NEXT
            GO TO 100
         END IF
         PREV = I
*
*        Perform one bisection step
*
         CNT = 0
         S = MID
         DPLUS = D( 1 ) - S
         IF( DPLUS.LT.ZERO ) CNT = CNT + 1
         DO 90 J = 2, N
            DPLUS = D( J ) - S - E2( J-1 )/DPLUS
            IF( DPLUS.LT.ZERO ) CNT = CNT + 1
 90      CONTINUE
         IF( CNT.LE.I-1 ) THEN
            WORK( K-1 ) = MID
         ELSE
            WORK( K ) = MID
         END IF
         I = NEXT

 100  CONTINUE
      ITER = ITER + 1
*     do another loop if there are still unconverged intervals
*     However, in the last iteration, all intervals are accepted
*     since this is the best we can do.
      IF( ( NINT.GT.0 ).AND.(ITER.LE.MAXITR) ) GO TO 80
```

```
*
*
*     At this point, all the intervals have converged
      DO 110 I = SAVI1, ILAST
         K = 2*I
         II = I - OFFSET
*        All intervals marked by '0' have been refined.
         IF( IWORK( K-1 ).EQ.0 ) THEN
            W( II ) = HALF*( WORK( K-1 )+WORK( K ) )
            WERR( II ) = WORK( K ) - W( II )
         END IF
 110  CONTINUE
*

      RETURN
*
*     End of SLARRJ
*
      END
```

———-

90.84.4 lisp code

— **slarrj.lisp** —

```
(let* ((zero 0.0) (one 1.0) (two 2.0) (half 0.5))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 0.5 0.5) half)
           (ignorable zero one two half))
  (defun slarrj
         (n d e2 ifirst ilast rtol offset w werr work iwork pivmin spdiam info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (single-float) spdiam pivmin rtol)
             (type (array single-float (*)) work werr w e2 d)
             (type (f2cl-lib:integer4) info offset ilast ifirst n))
    (f2cl-lib:with-multi-array-data
        ((d single-float d-%data% d-%offset%)
         (e2 single-float e2-%data% e2-%offset%)
         (w single-float w-%data% w-%offset%)
         (werr single-float werr-%data% werr-%offset%)
         (work single-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((dplus 0.0) (fac 0.0) (left 0.0) (mid 0.0) (right 0.0) (s 0.0)
             (tmp 0.0) (width 0.0) (cnt 0) (i 0) (i1 0) (i2 0) (ii 0) (iter 0)
             (j 0) (k 0) (next 0) (f2cl-lib:nint 0) (olnint 0) (p 0) (prev 0)
```

```
     (savi1 0) (maxitr 0))
(declare (type (single-float) dplus fac left mid right s tmp width)
         (type (f2cl-lib:integer4) cnt i i1 i2 ii iter j k next
          f2cl-lib:nint olnint p prev savi1 maxitr))
(setf info 0)
(setf maxitr
        (f2cl-lib:int-add
         (f2cl-lib:int
          (/
           (- (f2cl-lib:flog (+ spdiam pivmin)) (f2cl-lib:flog pivmin))
           (f2cl-lib:flog two)))
         2))
(setf i1 ifirst)
(setf i2 ilast)
(setf f2cl-lib:nint 0)
(setf prev 0)
(f2cl-lib:fdo (i i1 (f2cl-lib:int-add i 1))
              ((> i i2) nil)
  (tagbody
    (setf k (f2cl-lib:int-mul 2 i))
    (setf ii (f2cl-lib:int-sub i offset))
    (setf left
            (- (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%)
               (f2cl-lib:fref werr-%data% (ii) ((1 *)) werr-%offset%)))
    (setf mid (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%))
    (setf right
            (+ (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%)
               (f2cl-lib:fref werr-%data% (ii) ((1 *)) werr-%offset%)))
    (setf width (- right mid))
    (setf tmp (max (abs left) (abs right)))
    (cond
     ((< width (* rtol tmp))
      (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-sub k 1))
                           ((1 *)) iwork-%offset%)
            -1)
      (if (and (= i i1) (< i i2))
          (setf i1 (f2cl-lib:int-add i 1)))
      (if (and (>= prev i1) (<= i i2))
          (setf (f2cl-lib:fref iwork-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-mul 2 prev) 1))
                               ((1 *)) iwork-%offset%)
                (f2cl-lib:int-add i 1))))
     (t
      (tagbody
        (setf prev i)
        (setf fac one)
       label20
        (setf cnt 0)
        (setf s left)
```

```
 (setf dplus
         (- (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%) s))
 (if (< dplus zero)
     (setf cnt (f2cl-lib:int-add cnt 1)))
 (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf dplus
             (+
              (- (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                 s)
              (/
               (-
                (f2cl-lib:fref e2-%data%
                               ((f2cl-lib:int-sub j 1)) ((1 *))
                               e2-%offset%))
               dplus)))
     (if (< dplus zero)
         (setf cnt (f2cl-lib:int-add cnt 1)))
    label30))
 (cond
  ((> cnt (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
   (setf left
           (- left
              (*
               (f2cl-lib:fref werr-%data% (ii) ((1 *))
                              werr-%offset%)
               fac)))
   (setf fac (* two fac)) (go label20)))
 (setf fac one)
label50
 (setf cnt 0)
 (setf s right)
 (setf dplus
         (- (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%) s))
 (if (< dplus zero)
     (setf cnt (f2cl-lib:int-add cnt 1)))
 (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf dplus
             (+
              (- (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                 s)
              (/
               (-
                (f2cl-lib:fref e2-%data%
                               ((f2cl-lib:int-sub j 1)) ((1 *))
                               e2-%offset%))
               dplus)))
```

```
                (if (< dplus zero)
                    (setf cnt (f2cl-lib:int-add cnt 1)))
               label60))
            (cond
             ((< cnt i)
              (setf right
                      (+ right
                         (*
                          (f2cl-lib:fref werr-%data% (ii) ((1 *))
                                         werr-%offset%)
                          fac)))
              (setf fac (* two fac)) (go label50)))
            (setf f2cl-lib:nint (f2cl-lib:int-add f2cl-lib:nint 1))
            (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-sub k 1))
                                 ((1 *)) iwork-%offset%)
                    (f2cl-lib:int-add i 1))
            (setf (f2cl-lib:fref iwork-%data% (k) ((1 *)) iwork-%offset%)
                    cnt))))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                             work-%offset%)
                left)
        (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%) right)
       label75))
     (setf savi1 i1)
     (setf iter 0)
    label80
     (setf prev (f2cl-lib:int-sub i1 1))
     (setf i i1)
     (setf olnint f2cl-lib:nint)
     (f2cl-lib:fdo (p 1 (f2cl-lib:int-add p 1))
                   ((> p olnint) nil)
       (tagbody
         (setf k (f2cl-lib:int-mul 2 i))
         (setf ii (f2cl-lib:int-sub i offset))
         (setf next
                 (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-sub k 1))
                                ((1 *)) iwork-%offset%))
         (setf left
                 (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                                work-%offset%))
         (setf right (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%))
         (setf mid (* half (+ left right)))
         (setf width (- right mid))
         (setf tmp (max (abs left) (abs right)))
         (cond
          ((or (< width (* rtol tmp)) (= iter maxitr))
           (setf f2cl-lib:nint (f2cl-lib:int-sub f2cl-lib:nint 1))
           (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-sub k 1))
                                ((1 *)) iwork-%offset%)
                   0)
```

```
        (cond ((= i1 i) (setf i1 next))
              (t
               (if (>= prev i1)
                   (setf (f2cl-lib:fref iwork-%data%
                                        ((f2cl-lib:int-sub
                                          (f2cl-lib:int-mul 2 prev) 1))
                                        ((1 *)) iwork-%offset%)
                         next))))
        (setf i next) (go label100)))
      (setf prev i)
      (setf cnt 0)
      (setf s mid)
      (setf dplus (- (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%) s))
      (if (< dplus zero)
          (setf cnt (f2cl-lib:int-add cnt 1)))
      (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf dplus
                  (+
                   (- (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%) s)
                   (/
                    (-
                     (f2cl-lib:fref e2-%data% ((f2cl-lib:int-sub j 1))
                                    ((1 *)) e2-%offset%))
                    dplus)))
          (if (< dplus zero)
              (setf cnt (f2cl-lib:int-add cnt 1)))
         label90))
      (cond
       ((<= cnt (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                             work-%offset%)
              mid))
       (t
        (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
              mid)))
      (setf i next)
     label100))
  (setf iter (f2cl-lib:int-add iter 1))
  (if (and (> f2cl-lib:nint 0) (<= iter maxitr))
      (go label80))
  (f2cl-lib:fdo (i savi1 (f2cl-lib:int-add i 1))
                ((> i ilast) nil)
    (tagbody
      (setf k (f2cl-lib:int-mul 2 i))
      (setf ii (f2cl-lib:int-sub i offset))
      (cond
       ((=
         (f2cl-lib:fref iwork ((f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
```

```
                              ((1 *)))
               0)
              (setf (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%)
                      (* half
                         (+
                          (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub k 1))
                                         ((1 *)) work-%offset%)
                          (f2cl-lib:fref work-%data% (k) ((1 *))
                                         work-%offset%))))
              (setf (f2cl-lib:fref werr-%data% (ii) ((1 *)) werr-%offset%)
                      (- (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                         (f2cl-lib:fref w-%data% (ii) ((1 *)) w-%offset%)))))
           label110))
        (go end_label)
       end_label
        (return
         (values nil nil nil nil nil nil nil nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slarrj
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (single-float)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (single-float) (single-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
                                              nil nil nil nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

90.85 slarrk

90.85.1 Axiom unit tests

— slarrk.input —

```
)set break resume
)sys rm -f slarrk.output
)spool slarrk.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.85.2 Axiom help page

— slarrk.help —

```
======================================================================
slarrk examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slarrk (N, IW, GL, GU, D, E2, PIVMIN, RELTOL, W, WERR, INFO)
           SLARRK computes one eigenvalue of a symmetric tridiagonal matrix T
           to suitable accuracy.

Function/Subroutine Documentation
   subroutine slarrk (integer N, integer IW, real GL, real GU, real,
       dimension( * ) D, real, dimension( * ) E2, real PIVMIN, real RELTOL,
       real W, real WERR, integer INFO)
       SLARRK computes one eigenvalue of a symmetric tridiagonal matrix T to
       suitable accuracy.

Purpose:
   SLARRK computes one eigenvalue of a symmetric tridiagonal
   matrix T to suitable accuracy. This is an auxiliary code to be
```

```
  called from SSTEMR.

  To avoid overflow, the matrix must be scaled so that its
  largest element is no greater than
      overflow**(1/2) * underflow**(1/4)
  in absolute value, and for greatest
  accuracy, it should not be much smaller than that.

Parameters:

N
  N is INTEGER
  The order of the tridiagonal matrix T.  N >= 0.

IW
  IW is INTEGER
  The index of the eigenvalues to be returned.

GL
  GL is REAL

GU
  GU is REAL
  An upper and a lower bound on the eigenvalue.

D
  D is REAL array, dimension (N)
  The n diagonal elements of the tridiagonal matrix T.

E2
  E2 is REAL array, dimension (N-1)
  The (n-1) squared off-diagonal elements of the tridiagonal matrix T.

PIVMIN
  PIVMIN is REAL
  The minimum pivot allowed in the Sturm sequence for T.

RELTOL
  RELTOL is REAL
  The minimum relative width of an interval.  When an interval
  is narrower than RELTOL times the larger (in
  magnitude) endpoint, then it is considered to be
  sufficiently small, i.e., converged.  Note: this should
  always be at least radix*machine epsilon.

W
  W is REAL

WERR
  WERR is REAL
```

```
   The error bound on the corresponding eigenvalue approximation in W.

 INFO
   INFO is INTEGER
   = 0:       Eigenvalue converged
   = -1:      Eigenvalue did NOT converge

Internal Parameters:

 FUDGE   REAL            , default = 2
   A "fudge factor" to widen the Gershgorin intervals.
```

———-

90.85.3 fortran code

— **slarrk.f** —

```
*  =====================================================================
      SUBROUTINE SLARRK( N, IW, GL, GU,
     $                   D, E2, PIVMIN, RELTOL, W, WERR, INFO)
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER   INFO, IW, N
      REAL               PIVMIN, RELTOL, GL, GU, W, WERR
*     ..
*     .. Array Arguments ..
      REAL               D( * ), E2( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               FUDGE, HALF, TWO, ZERO
      PARAMETER          ( HALF = 0.5E0, TWO = 2.0E0,
     $                     FUDGE = TWO, ZERO = 0.0E0 )
*     ..
*     .. Local Scalars ..
      INTEGER   I, IT, ITMAX, NEGCNT
      REAL               ATOLI, EPS, LEFT, MID, RIGHT, RTOLI, TMP1,
     $                   TMP2, TNORM
*     ..
```

```
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL   SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, INT, LOG, MAX
*     ..
*     .. Executable Statements ..
*
*     Get machine constants
      EPS = SLAMCH( 'P' )

      TNORM = MAX( ABS( GL ), ABS( GU ) )
      RTOLI = RELTOL
      ATOLI = FUDGE*TWO*PIVMIN

      ITMAX = INT( ( LOG( TNORM+PIVMIN )-LOG( PIVMIN ) ) /
     $           LOG( TWO ) ) + 2

      INFO = -1

      LEFT = GL - FUDGE*TNORM*EPS*N - FUDGE*TWO*PIVMIN
      RIGHT = GU + FUDGE*TNORM*EPS*N + FUDGE*TWO*PIVMIN
      IT = 0

 10   CONTINUE
*
*     Check if interval converged or maximum number of iterations reached
*
      TMP1 = ABS( RIGHT - LEFT )
      TMP2 = MAX( ABS(RIGHT), ABS(LEFT) )
      IF( TMP1.LT.MAX( ATOLI, PIVMIN, RTOLI*TMP2 ) ) THEN
         INFO = 0
         GOTO 30
      ENDIF
      IF(IT.GT.ITMAX)
     $   GOTO 30

*
*     Count number of negative pivots for mid-point
*
      IT = IT + 1
      MID = HALF * (LEFT + RIGHT)
      NEGCNT = 0
      TMP1 = D( 1 ) - MID
      IF( ABS( TMP1 ).LT.PIVMIN )
     $   TMP1 = -PIVMIN
      IF( TMP1.LE.ZERO )
     $   NEGCNT = NEGCNT + 1
*
```

```
      DO 20 I = 2, N
         TMP1 = D( I ) - E2( I-1 ) / TMP1 - MID
         IF( ABS( TMP1 ).LT.PIVMIN )
     $      TMP1 = -PIVMIN
         IF( TMP1.LE.ZERO )
     $      NEGCNT = NEGCNT + 1
 20   CONTINUE

      IF(NEGCNT.GE.IW) THEN
         RIGHT = MID
      ELSE
         LEFT = MID
      ENDIF
      GOTO 10

 30   CONTINUE
*
*     Converged or maximum number of iterations reached
*
      W = HALF * (LEFT + RIGHT)
      WERR = HALF * ABS( RIGHT - LEFT )

      RETURN
*
*     End of SLARRK
*
      END
```

90.85.4 lisp code

— slarrk.lisp —

```
(let* ((half 0.5) (two 2.0) (fudge two) (zero 0.0))
  (declare (type (single-float 0.5 0.5) half)
           (type (single-float 2.0 2.0) two)
           (type (single-float) fudge)
           (type (single-float 0.0 0.0) zero)
           (ignorable half two fudge zero))
  (defun slarrk (n iw gl gu d e2 pivmin reltol w werr info)
    (declare (type (array single-float (*)) e2 d)
             (type (single-float) werr w reltol pivmin gu gl)
             (type (f2cl-lib:integer4) info iw n))
    (f2cl-lib:with-multi-array-data
        ((d single-float d-%data% d-%offset%)
         (e2 single-float e2-%data% e2-%offset%))
      (prog ((atoli 0.0) (eps 0.0) (left 0.0) (mid 0.0) (right 0.0) (rtoli 0.0)
```

```
     (tmp1 0.0) (tmp2 0.0) (tnorm 0.0) (i 0) (it 0) (itmax 0)
     (negcnt 0))
 (declare
  (type (single-float) atoli eps left mid right rtoli tmp1 tmp2 tnorm)
  (type (f2cl-lib:integer4) i it itmax negcnt))
 (setf eps (slamch "P"))
 (setf tnorm (max (abs gl) (abs gu)))
 (setf rtoli reltol)
 (setf atoli (* fudge two pivmin))
 (setf itmax
         (f2cl-lib:int-add
          (f2cl-lib:int
           (/
            (- (f2cl-lib:flog (+ tnorm pivmin)) (f2cl-lib:flog pivmin))
            (f2cl-lib:flog two)))
          2))
 (setf info -1)
 (setf left (+ gl (* (- fudge) tnorm eps n) (* (- fudge) two pivmin)))
 (setf right (+ gu (* fudge tnorm eps n) (* fudge two pivmin)))
 (setf it 0)
label10
 (setf tmp1 (abs (- right left)))
 (setf tmp2 (max (abs right) (abs left)))
 (cond
  ((< tmp1 (max atoli pivmin (* rtoli tmp2))) (setf info 0)
   (go label30)))
 (if (> it itmax)
     (go label30))
 (setf it (f2cl-lib:int-add it 1))
 (setf mid (* half (+ left right)))
 (setf negcnt 0)
 (setf tmp1 (- (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%) mid))
 (if (< (abs tmp1) pivmin)
     (setf tmp1 (- pivmin)))
 (if (<= tmp1 zero)
     (setf negcnt (f2cl-lib:int-add negcnt 1)))
 (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf tmp1
             (-
              (+ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                 (/
                  (-
                   (f2cl-lib:fref e2-%data% ((f2cl-lib:int-sub i 1))
                                  ((1 *)) e2-%offset%))
                  tmp1))
              mid))
     (if (< (abs tmp1) pivmin)
         (setf tmp1 (- pivmin)))
```

```
            (if (<= tmp1 zero)
                (setf negcnt (f2cl-lib:int-add negcnt 1)))
           label20))
        (cond ((>= negcnt iw) (setf right mid)) (t (setf left mid)))
        (go label10)
       label30
        (setf w (* half (+ left right)))
        (setf werr (* half (abs (- right left))))
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil nil nil w werr info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slarrk
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (single-float) (single-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil nil nil nil nil
                                              fortran-to-lisp::w
                                              fortran-to-lisp::werr
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

90.86 slarrr

90.86.1 Axiom unit tests

— slarrr.input —

```
)set break resume
)sys rm -f slarrr.output
)spool slarrr.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.86.2 Axiom help page

— slarrr.help —

```
=====================================================================
slarrr examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slarrr (N, D, E, INFO)
           SLARRR performs tests to decide whether the symmetric tridiagonal
           matrix T warrants expensive computations which guarantee high
           relative accuracy in the eigenvalues.

Function/Subroutine Documentation
   subroutine slarrr (integer N, real, dimension( * ) D, real, dimension( * )
       E, integer INFO)
       SLARRR performs tests to decide whether the symmetric tridiagonal
       matrix T warrants expensive computations which guarantee high relative
       accuracy in the eigenvalues.

Purpose:
   Perform tests to decide whether the symmetric tridiagonal matrix T
   warrants expensive computations which guarantee high relative accuracy
   in the eigenvalues.

Parameters:

 N
   N is INTEGER
   The order of the matrix. N > 0.

 D
   D is REAL array, dimension (N)
   The N diagonal elements of the tridiagonal matrix T.
```

```
 E
   E is REAL array, dimension (N)
   On entry, the first (N-1) entries contain the subdiagonal
   elements of the tridiagonal matrix T; E(N) is set to ZERO.

 INFO
   INFO is INTEGER
   INFO = 0(default) : the matrix warrants computations preserving
         relative accuracy.
   INFO = 1 : the matrix warrants computations guaranteeing
         only absolute accuracy.
```

90.86.3 fortran code

— slarrr.f —

```
*  =====================================================================
      SUBROUTINE SLARRR( N, D, E, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            N, INFO
*     ..
*     .. Array Arguments ..
      REAL               D( * ), E( * )
*     ..
*
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, RELCOND
      PARAMETER          ( ZERO = 0.0E0,
     $                     RELCOND = 0.999E0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      LOGICAL            YESREL
      REAL               EPS, SAFMIN, SMLNUM, RMIN, TMP, TMP2,
     $          OFFDIG, OFFDIG2
```

```
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS
*     ..
*     .. Executable Statements ..
*
*     As a default, do NOT go for relative-accuracy preserving computations.
      INFO = 1

      SAFMIN = SLAMCH( 'Safe minimum' )
      EPS = SLAMCH( 'Precision' )
      SMLNUM = SAFMIN / EPS
      RMIN = SQRT( SMLNUM )

*     Tests for relative accuracy
*
*     Test for scaled diagonal dominance
*     Scale the diagonal entries to one and check whether the sum of the
*     off-diagonals is less than one
*
*     The sdd relative error bounds have a 1/(1- 2*x) factor in them,
*     x = max(OFFDIG + OFFDIG2), so when x is close to 1/2, no relative
*     accuracy is promised.  In the notation of the code fragment below,
*     1/(1 - (OFFDIG + OFFDIG2)) is the condition number.
*     We don't think it is worth going into "sdd mode" unless the relative
*     condition number is reasonable, not 1/macheps.
*     The threshold should be compatible with other thresholds used in the
*     code. We set  OFFDIG + OFFDIG2 <= .999 =: RELCOND, it corresponds
*     to losing at most 3 decimal digits: 1 / (1 - (OFFDIG + OFFDIG2)) <= 1000
*     instead of the current OFFDIG + OFFDIG2 < 1
*
      YESREL = .TRUE.
      OFFDIG = ZERO
      TMP = SQRT(ABS(D(1)))
      IF (TMP.LT.RMIN) YESREL = .FALSE.
      IF(.NOT.YESREL) GOTO 11
      DO 10 I = 2, N
         TMP2 = SQRT(ABS(D(I)))
         IF (TMP2.LT.RMIN) YESREL = .FALSE.
         IF(.NOT.YESREL) GOTO 11
         OFFDIG2 = ABS(E(I-1))/(TMP*TMP2)
         IF(OFFDIG+OFFDIG2.GE.RELCOND) YESREL = .FALSE.
         IF(.NOT.YESREL) GOTO 11
         TMP = TMP2
         OFFDIG = OFFDIG2
```

```
 10   CONTINUE
 11   CONTINUE

      IF( YESREL ) THEN
         INFO = 0
         RETURN
      ELSE
      ENDIF
*

*
*     *** MORE TO BE IMPLEMENTED ***
*

*
*     Test if the lower bidiagonal matrix L from T = L D L^T
*     (zero shift facto) is well conditioned
*

*
*     Test if the upper bidiagonal matrix U from T = U D U^T
*     (zero shift facto) is well conditioned.
*     In this case, the matrix needs to be flipped and, at the end
*     of the eigenvector computation, the flip needs to be applied
*     to the computed eigenvectors (and the support)
*

*
      RETURN
*
*     END OF SLARRR
*
      END
```

———-

90.86.4 lisp code

— slarrr.lisp —

```
(let* ((zero 0.0) (relcond 0.999))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 0.999 0.999) relcond)
           (ignorable zero relcond))
  (defun slarrr (n d e info)
    (declare (type (array single-float (*)) e d)
             (type (f2cl-lib:integer4) info n))
    (f2cl-lib:with-multi-array-data
```

```
  ((d single-float d-%data% d-%offset%)
   (e single-float e-%data% e-%offset%))
(prog ((eps 0.0) (safmin 0.0) (smlnum 0.0) (rmin 0.0) (tmp 0.0)
       (tmp2 0.0) (offdig 0.0) (offdig2 0.0) (yesrel nil) (i 0))
  (declare
   (type (single-float) eps safmin smlnum rmin tmp tmp2 offdig offdig2)
   (type f2cl-lib:logical yesrel)
   (type (f2cl-lib:integer4) i))
  (setf info 1)
  (setf safmin (slamch "Safe minimum"))
  (setf eps (slamch "Precision"))
  (setf smlnum (/ safmin eps))
  (setf rmin (f2cl-lib:fsqrt smlnum))
  (setf yesrel f2cl-lib:%true%)
  (setf offdig zero)
  (setf tmp
          (f2cl-lib:fsqrt
           (abs (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))))
  (if (< tmp rmin)
      (setf yesrel f2cl-lib:%false%))
  (if (not yesrel)
      (go label11))
  (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf tmp2
              (f2cl-lib:fsqrt
               (abs (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))))
      (if (< tmp2 rmin)
          (setf yesrel f2cl-lib:%false%))
      (if (not yesrel)
          (go label11))
      (setf offdig2
              (/
               (abs
                (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1)) ((1 *))
                               e-%offset%))
               (* tmp tmp2)))
      (if (>= (+ offdig offdig2) relcond)
          (setf yesrel f2cl-lib:%false%))
      (if (not yesrel)
          (go label11))
      (setf tmp tmp2)
      (setf offdig offdig2)
     label10))
 label11
  (cond (yesrel (setf info 0) (go end_label)) (t))
  (go end_label)
 end_label
  (return (values nil nil nil info))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slarrr
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-figure (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

———-

90.87 slartg

90.87.1 Axiom unit tests

— **slartg.input** —

```
)set break resume
)sys rm -f slartg.output
)spool slartg.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.87.2 Axiom help page

— **slartg.help** —

```
==========================================================================
slartg examples
```

```
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slartg (F, G, CS, SN, R)
           SLARTG generates a plane rotation with real cosine and real sine.

Function/Subroutine Documentation
   subroutine slartg (real F, real G, real CS, real SN, real R)
       SLARTG generates a plane rotation with real cosine and real sine.

Purpose:
   SLARTG generate a plane rotation so that

   [  CS  SN  ]  .  [ F ]  =  [ R ]   where CS**2 + SN**2 = 1.
   [ -SN  CS  ]     [ G ]     [ 0 ]

   This is a slower, more accurate version of the BLAS1 routine SROTG,
   with the following other differences:
   F and G are unchanged on return.
   If G=0, then CS=1 and SN=0.
     If F=0 and (G .ne. 0), then CS=0 and SN=1 without doing any
     floating point operations (saves work in SBDSQR when
     there are zeros on the diagonal).

   If F exceeds G in magnitude, CS will be positive.

Parameters:

 F
   F is REAL
   The first component of vector to be rotated.

 G
   G is REAL
   The second component of vector to be rotated.

 CS
   CS is REAL
   The cosine of the rotation.

 SN
   SN is REAL
   The sine of the rotation.

 R
```

```
R is REAL
The nonzero component of the rotated vector.

This version has a few statements commented out for thread safety
(machine parameters are computed on each entry). 10 feb 03, SJH.
```

———-

90.87.3 fortran code

— **slartg.f** —

```
*  =====================================================================
      SUBROUTINE SLARTG( F, G, CS, SN, R )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      REAL               CS, F, G, R, SN
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E0 )
      REAL               ONE
      PARAMETER          ( ONE = 1.0E0 )
      REAL               TWO
      PARAMETER          ( TWO = 2.0E0 )
*     ..
*     .. Local Scalars ..
*     LOGICAL            FIRST
      INTEGER            COUNT, I
      REAL               EPS, F1, G1, SAFMIN, SAFMN2, SAFMX2, SCALE
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, INT, LOG, MAX, SQRT
*     ..
*     .. Save statement ..
```

```
*     SAVE               FIRST, SAFMX2, SAFMIN, SAFMN2
*     ..
*     .. Data statements ..
*     DATA               FIRST / .TRUE. /
*     ..
*     .. Executable Statements ..
*
*     IF( FIRST ) THEN
         SAFMIN = SLAMCH( 'S' )
         EPS = SLAMCH( 'E' )
         SAFMN2 = SLAMCH( 'B' )**INT( LOG( SAFMIN / EPS ) /
     $           LOG( SLAMCH( 'B' ) ) / TWO )
         SAFMX2 = ONE / SAFMN2
*        FIRST = .FALSE.
*     END IF
      IF( G.EQ.ZERO ) THEN
         CS = ONE
         SN = ZERO
         R = F
      ELSE IF( F.EQ.ZERO ) THEN
         CS = ZERO
         SN = ONE
         R = G
      ELSE
         F1 = F
         G1 = G
         SCALE = MAX( ABS( F1 ), ABS( G1 ) )
         IF( SCALE.GE.SAFMX2 ) THEN
            COUNT = 0
   10       CONTINUE
            COUNT = COUNT + 1
            F1 = F1*SAFMN2
            G1 = G1*SAFMN2
            SCALE = MAX( ABS( F1 ), ABS( G1 ) )
            IF( SCALE.GE.SAFMX2 )
     $         GO TO 10
            R = SQRT( F1**2+G1**2 )
            CS = F1 / R
            SN = G1 / R
            DO 20 I = 1, COUNT
               R = R*SAFMX2
   20       CONTINUE
         ELSE IF( SCALE.LE.SAFMN2 ) THEN
            COUNT = 0
   30       CONTINUE
            COUNT = COUNT + 1
            F1 = F1*SAFMX2
            G1 = G1*SAFMX2
            SCALE = MAX( ABS( F1 ), ABS( G1 ) )
            IF( SCALE.LE.SAFMN2 )
```

```
     $         GO TO 30
            R = SQRT( F1**2+G1**2 )
            CS = F1 / R
            SN = G1 / R
            DO 40 I = 1, COUNT
               R = R*SAFMN2
   40       CONTINUE
         ELSE
            R = SQRT( F1**2+G1**2 )
            CS = F1 / R
            SN = G1 / R
         END IF
         IF( ABS( F ).GT.ABS( G ) .AND. CS.LT.ZERO ) THEN
            CS = -CS
            SN = -SN
            R = -R
         END IF
      END IF
      RETURN
*
*     End of SLARTG
*
      END
```

———-

90.87.4 lisp code

— slartg.lisp —

```
(let* ((zero 0.0) (one 1.0) (two 2.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (ignorable zero one two))
  (defun slartg (f g cs sn r)
    (declare (type (single-float) r sn cs g f))
    (prog ((eps 0.0) (f1 0.0) (g1 0.0) (safmin 0.0) (safmn2 0.0) (safmx2 0.0)
           (scale 0.0) (i 0) (count$ 0))
      (declare (type (single-float) eps f1 g1 safmin safmn2 safmx2 scale)
               (type (f2cl-lib:integer4) count$ i))
      (setf safmin (slamch "S"))
      (setf eps (slamch "E"))
      (setf safmn2
              (expt (slamch "B")
                    (f2cl-lib:int
                     (/
                      (/ (f2cl-lib:flog (/ safmin eps))
```

```
                        (f2cl-lib:flog (slamch "B")))
                    two))))
      (setf safmx2 (/ one safmn2))
      (cond ((= g zero) (setf cs one) (setf sn zero) (setf r f))
            ((= f zero) (setf cs zero) (setf sn one) (setf r g))
            (t (setf f1 f) (setf g1 g) (setf scale (max (abs f1) (abs g1)))
             (cond
              ((>= scale safmx2)
               (tagbody
                 (setf count$ 0)
                label10
                 (setf count$ (f2cl-lib:int-add count$ 1))
                 (setf f1 (* f1 safmn2))
                 (setf g1 (* g1 safmn2))
                 (setf scale (max (abs f1) (abs g1)))
                 (if (>= scale safmx2)
                     (go label10))
                 (setf r (f2cl-lib:fsqrt (+ (expt f1 2) (expt g1 2))))
                 (setf cs (/ f1 r))
                 (setf sn (/ g1 r))
                 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                               ((> i count$) nil)
                   (tagbody (setf r (* r safmx2)) label20))))
              ((<= scale safmn2)
               (tagbody
                 (setf count$ 0)
                label30
                 (setf count$ (f2cl-lib:int-add count$ 1))
                 (setf f1 (* f1 safmx2))
                 (setf g1 (* g1 safmx2))
                 (setf scale (max (abs f1) (abs g1)))
                 (if (<= scale safmn2)
                     (go label30))
                 (setf r (f2cl-lib:fsqrt (+ (expt f1 2) (expt g1 2))))
                 (setf cs (/ f1 r))
                 (setf sn (/ g1 r))
                 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                               ((> i count$) nil)
                   (tagbody (setf r (* r safmn2)) label40))))
              (t (setf r (f2cl-lib:fsqrt (+ (expt f1 2) (expt g1 2))))
               (setf cs (/ f1 r)) (setf sn (/ g1 r))))
             (cond
              ((and (> (abs f) (abs g)) (< cs zero)) (setf cs (- cs))
               (setf sn (- sn)) (setf r (- r))))))
      (go end_label)
     end_label
      (return (values nil nil cs sn r)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
```

```
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slartg
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((single-float) (single-float)
                                              (single-float) (single-float)
                                              (single-float))
                                            :return-values
                                            '(nil nil fortran-to-lisp::cs
                                              fortran-to-lisp::sn
                                              fortran-to-lisp::r)
                                            :calls 'nil)))
```

———-

90.88 slartgp

90.88.1 Axiom unit tests

— **slartgp.input** —

```
)set break resume
)sys rm -f slartgp.output
)spool slartgp.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.88.2 Axiom help page

— **slartgp.help** —

```
=====================================================================
slartgp examples
=====================================================================

=====================================================================
```

```
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slartgp (F, G, CS, SN, R)
           SLARTGP generates a plane rotation so that the diagonal is
           nonnegative.

Function/Subroutine Documentation
   subroutine slartgp (real F, real G, real CS, real SN, real R)
       SLARTGP generates a plane rotation so that the diagonal is nonnegative.

Purpose:
   SLARTGP generates a plane rotation so that

   [  CS  SN  ]  .  [ F ]  =  [ R ]   where CS**2 + SN**2 = 1.
   [ -SN  CS  ]     [ G ]     [ 0 ]

   This is a slower, more accurate version of the Level 1 BLAS routine SROTG,
   with the following other differences:
   F and G are unchanged on return.
     If G=0, then CS=(+/-)1 and SN=0.
     If F=0 and (G .ne. 0), then CS=0 and SN=(+/-)1.

   The sign is chosen so that R >= 0.

Parameters:

 F
   F is REAL
   The first component of vector to be rotated.

 G
   G is REAL
   The second component of vector to be rotated.

 CS
   CS is REAL
   The cosine of the rotation.

 SN
   SN is REAL
   The sine of the rotation.

 R
   R is REAL
   The nonzero component of the rotated vector.

   This version has a few statements commented out for thread safety
```

```
(machine parameters are computed on each entry). 10 feb 03, SJH.
```

———-

90.88.3 fortran code

— **slartgp.f** —

```
*  =====================================================================
      SUBROUTINE SLARTGP( F, G, CS, SN, R )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      REAL               CS, F, G, R, SN
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E0 )
      REAL               ONE
      PARAMETER          ( ONE = 1.0E0 )
      REAL               TWO
      PARAMETER          ( TWO = 2.0E0 )
*     ..
*     .. Local Scalars ..
*     LOGICAL            FIRST
      INTEGER            COUNT, I
      REAL               EPS, F1, G1, SAFMIN, SAFMN2, SAFMX2, SCALE
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, INT, LOG, MAX, SIGN, SQRT
*     ..
*     .. Save statement ..
*     SAVE               FIRST, SAFMX2, SAFMIN, SAFMN2
*     ..
*     .. Data statements ..
*     DATA               FIRST / .TRUE. /
```

```
*     ..
*     .. Executable Statements ..
*
*     IF( FIRST ) THEN
         SAFMIN = SLAMCH( 'S' )
         EPS = SLAMCH( 'E' )
         SAFMN2 = SLAMCH( 'B' )**INT( LOG( SAFMIN / EPS ) /
     $           LOG( SLAMCH( 'B' ) ) / TWO )
         SAFMX2 = ONE / SAFMN2
*        FIRST = .FALSE.
*     END IF
      IF( G.EQ.ZERO ) THEN
         CS = SIGN( ONE, F )
         SN = ZERO
         R = ABS( F )
      ELSE IF( F.EQ.ZERO ) THEN
         CS = ZERO
         SN = SIGN( ONE, G )
         R = ABS( G )
      ELSE
         F1 = F
         G1 = G
         SCALE = MAX( ABS( F1 ), ABS( G1 ) )
         IF( SCALE.GE.SAFMX2 ) THEN
            COUNT = 0
   10       CONTINUE
            COUNT = COUNT + 1
            F1 = F1*SAFMN2
            G1 = G1*SAFMN2
            SCALE = MAX( ABS( F1 ), ABS( G1 ) )
            IF( SCALE.GE.SAFMX2 )
     $         GO TO 10
            R = SQRT( F1**2+G1**2 )
            CS = F1 / R
            SN = G1 / R
            DO 20 I = 1, COUNT
               R = R*SAFMX2
   20       CONTINUE
         ELSE IF( SCALE.LE.SAFMN2 ) THEN
            COUNT = 0
   30       CONTINUE
            COUNT = COUNT + 1
            F1 = F1*SAFMX2
            G1 = G1*SAFMX2
            SCALE = MAX( ABS( F1 ), ABS( G1 ) )
            IF( SCALE.LE.SAFMN2 )
     $         GO TO 30
            R = SQRT( F1**2+G1**2 )
            CS = F1 / R
            SN = G1 / R
```

```
            DO 40 I = 1, COUNT
               R = R*SAFMN2
   40       CONTINUE
         ELSE
            R = SQRT( F1**2+G1**2 )
            CS = F1 / R
            SN = G1 / R
         END IF
         IF( R.LT.ZERO ) THEN
            CS = -CS
            SN = -SN
            R = -R
         END IF
      END IF
      RETURN
*
*     End of SLARTG
*
      END
```

———-

90.88.4 lisp code

— slartgp.lisp —

```
(let* ((zero 0.0) (one 1.0) (two 2.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (ignorable zero one two))
  (defun slartgp (f g cs sn r)
    (declare (type (single-float) r sn cs g f))
    (prog ((eps 0.0) (f1 0.0) (g1 0.0) (safmin 0.0) (safmn2 0.0) (safmx2 0.0)
           (scale 0.0) (i 0) (count$ 0))
      (declare (type (single-float) eps f1 g1 safmin safmn2 safmx2 scale)
               (type (f2cl-lib:integer4) count$ i))
      (setf safmin (slamch "S"))
      (setf eps (slamch "E"))
      (setf safmn2
              (expt (slamch "B")
                    (f2cl-lib:int
                     (/
                      (/ (f2cl-lib:flog (/ safmin eps))
                         (f2cl-lib:flog (slamch "B")))
                      two))))
      (setf safmx2 (/ one safmn2))
      (cond
```

```
      ((= g zero) (setf cs (f2cl-lib:sign one f)) (setf sn zero)
       (setf r (abs f)))
      ((= f zero) (setf cs zero) (setf sn (f2cl-lib:sign one g))
       (setf r (abs g)))
      (t (setf f1 f) (setf g1 g) (setf scale (max (abs f1) (abs g1)))
       (cond
        ((>= scale safmx2)
         (tagbody
           (setf count$ 0)
          label10
           (setf count$ (f2cl-lib:int-add count$ 1))
           (setf f1 (* f1 safmn2))
           (setf g1 (* g1 safmn2))
           (setf scale (max (abs f1) (abs g1)))
           (if (>= scale safmx2)
               (go label10))
           (setf r (f2cl-lib:fsqrt (+ (expt f1 2) (expt g1 2))))
           (setf cs (/ f1 r))
           (setf sn (/ g1 r))
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i count$) nil)
             (tagbody (setf r (* r safmx2)) label20))))
        ((<= scale safmn2)
         (tagbody
           (setf count$ 0)
          label30
           (setf count$ (f2cl-lib:int-add count$ 1))
           (setf f1 (* f1 safmx2))
           (setf g1 (* g1 safmx2))
           (setf scale (max (abs f1) (abs g1)))
           (if (<= scale safmn2)
               (go label30))
           (setf r (f2cl-lib:fsqrt (+ (expt f1 2) (expt g1 2))))
           (setf cs (/ f1 r))
           (setf sn (/ g1 r))
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i count$) nil)
             (tagbody (setf r (* r safmn2)) label40))))
        (t (setf r (f2cl-lib:fsqrt (+ (expt f1 2) (expt g1 2))))
         (setf cs (/ f1 r)) (setf sn (/ g1 r))))
       (cond ((< r zero) (setf cs (- cs)) (setf sn (- sn)) (setf r (- r))))))
     (go end_label)
    end_label
     (return (values nil nil cs sn r)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slartgp
                 fortran-to-lisp::*f2cl-function-info*)
```

```
(fortran-to-lisp::make-f2cl-finfo :arg-types
                                  '((single-float) (single-float)
                                    (single-float) (single-float)
                                    (single-float))
                                  :return-values
                                  '(nil nil fortran-to-lisp::cs
                                    fortran-to-lisp::sn
                                    fortran-to-lisp::r)
                                  :calls 'nil)))
```

———-

90.89 slaruv

90.89.1 Axiom unit tests

— **slaruv.input** —

```
)set break resume
)sys rm -f slaruv.output
)spool slaruv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.89.2 Axiom help page

— **slaruv.help** —

```
======================================================================
slaruv examples
======================================================================

======================================================================
Man Page Details
======================================================================
```

```
SYNOPSIS
   Functions/Subroutines
       subroutine slaruv (ISEED, N, X)
           SLARUV returns a vector of n random real numbers from a uniform
           distribution.

Function/Subroutine Documentation
   subroutine slaruv (integer, dimension( 4 ) ISEED, integer N, real,
       dimension( n ) X)
       SLARUV returns a vector of n random real numbers from a uniform
       distribution.

Purpose:
   SLARUV returns a vector of n random real numbers from a uniform (0,1)
   distribution (n <= 128).

   This is an auxiliary routine called by SLARNV and CLARNV.

Parameters:

 ISEED
   ISEED is INTEGER array, dimension (4)
   On entry, the seed of the random number generator; the array
   elements must be between 0 and 4095, and ISEED(4) must be odd.
   On exit, the seed is updated.

 N
   N is INTEGER
   The number of random numbers to be generated. N <= 128.

 X
   X is REAL array, dimension (N)
   The generated random numbers.

Further Details:

   This routine uses a multiplicative congruential method with modulus
   2**48 and multiplier 33952834046453 (see G.S.Fishman,
   'Multiplicative congruential random number generators with modulus
   2**b: an exhaustive analysis for b = 32 and a partial analysis for
   b = 48', Math. Comp. 189, pp 331-344, 1990).

   48-bit integers are stored in 4 integer array elements with 12 bits
   per element. Hence the routine is portable across machines with
   integers of 32 bits or more.
```

90.89.3 fortran code

— slaruv.f —

```
*  =====================================================================
      SUBROUTINE SLARUV( ISEED, N, X )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            N
*     ..
*     .. Array Arguments ..
      INTEGER            ISEED( 4 )
      REAL               X( N )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E0 )
      INTEGER            LV, IPW2
      REAL               R
      PARAMETER          ( LV = 128, IPW2 = 4096, R = ONE / IPW2 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, I1, I2, I3, I4, IT1, IT2, IT3, IT4, J
*     ..
*     .. Local Arrays ..
      INTEGER            MM( LV, 4 )
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MIN, MOD, REAL
*     ..
*     .. Data statements ..
      DATA               ( MM( 1, J ), J = 1, 4 ) / 494, 322, 2508,
     $                   2549 /
      DATA               ( MM( 2, J ), J = 1, 4 ) / 2637, 789, 3754,
     $                   1145 /
      DATA               ( MM( 3, J ), J = 1, 4 ) / 255, 1440, 1766,
     $                   2253 /
      DATA               ( MM( 4, J ), J = 1, 4 ) / 2008, 752, 3572,
     $                   305 /
      DATA               ( MM( 5, J ), J = 1, 4 ) / 1253, 2859, 2893,
     $                   3301 /
```

```
      DATA              ( MM( 6, J ), J = 1, 4 ) / 3344, 123, 307,
     $                  1065 /
      DATA              ( MM( 7, J ), J = 1, 4 ) / 4084, 1848, 1297,
     $                  3133 /
      DATA              ( MM( 8, J ), J = 1, 4 ) / 1739, 643, 3966,
     $                  2913 /
      DATA              ( MM( 9, J ), J = 1, 4 ) / 3143, 2405, 758,
     $                  3285 /
      DATA              ( MM( 10, J ), J = 1, 4 ) / 3468, 2638, 2598,
     $                  1241 /
      DATA              ( MM( 11, J ), J = 1, 4 ) / 688, 2344, 3406,
     $                  1197 /
      DATA              ( MM( 12, J ), J = 1, 4 ) / 1657, 46, 2922,
     $                  3729 /
      DATA              ( MM( 13, J ), J = 1, 4 ) / 1238, 3814, 1038,
     $                  2501 /
      DATA              ( MM( 14, J ), J = 1, 4 ) / 3166, 913, 2934,
     $                  1673 /
      DATA              ( MM( 15, J ), J = 1, 4 ) / 1292, 3649, 2091,
     $                  541 /
      DATA              ( MM( 16, J ), J = 1, 4 ) / 3422, 339, 2451,
     $                  2753 /
      DATA              ( MM( 17, J ), J = 1, 4 ) / 1270, 3808, 1580,
     $                  949 /
      DATA              ( MM( 18, J ), J = 1, 4 ) / 2016, 822, 1958,
     $                  2361 /
      DATA              ( MM( 19, J ), J = 1, 4 ) / 154, 2832, 2055,
     $                  1165 /
      DATA              ( MM( 20, J ), J = 1, 4 ) / 2862, 3078, 1507,
     $                  4081 /
      DATA              ( MM( 21, J ), J = 1, 4 ) / 697, 3633, 1078,
     $                  2725 /
      DATA              ( MM( 22, J ), J = 1, 4 ) / 1706, 2970, 3273,
     $                  3305 /
      DATA              ( MM( 23, J ), J = 1, 4 ) / 491, 637, 17,
     $                  3069 /
      DATA              ( MM( 24, J ), J = 1, 4 ) / 931, 2249, 854,
     $                  3617 /
      DATA              ( MM( 25, J ), J = 1, 4 ) / 1444, 2081, 2916,
     $                  3733 /
      DATA              ( MM( 26, J ), J = 1, 4 ) / 444, 4019, 3971,
     $                  409 /
      DATA              ( MM( 27, J ), J = 1, 4 ) / 3577, 1478, 2889,
     $                  2157 /
      DATA              ( MM( 28, J ), J = 1, 4 ) / 3944, 242, 3831,
     $                  1361 /
      DATA              ( MM( 29, J ), J = 1, 4 ) / 2184, 481, 2621,
     $                  3973 /
      DATA              ( MM( 30, J ), J = 1, 4 ) / 1661, 2075, 1541,
     $                  1865 /
```

```
      DATA              ( MM( 31, J ), J = 1, 4 ) / 3482, 4058, 893,
     $                  2525 /
      DATA              ( MM( 32, J ), J = 1, 4 ) / 657, 622, 736,
     $                  1409 /
      DATA              ( MM( 33, J ), J = 1, 4 ) / 3023, 3376, 3992,
     $                  3445 /
      DATA              ( MM( 34, J ), J = 1, 4 ) / 3618, 812, 787,
     $                  3577 /
      DATA              ( MM( 35, J ), J = 1, 4 ) / 1267, 234, 2125,
     $                  77 /
      DATA              ( MM( 36, J ), J = 1, 4 ) / 1828, 641, 2364,
     $                  3761 /
      DATA              ( MM( 37, J ), J = 1, 4 ) / 164, 4005, 2460,
     $                  2149 /
      DATA              ( MM( 38, J ), J = 1, 4 ) / 3798, 1122, 257,
     $                  1449 /
      DATA              ( MM( 39, J ), J = 1, 4 ) / 3087, 3135, 1574,
     $                  3005 /
      DATA              ( MM( 40, J ), J = 1, 4 ) / 2400, 2640, 3912,
     $                  225 /
      DATA              ( MM( 41, J ), J = 1, 4 ) / 2870, 2302, 1216,
     $                  85 /
      DATA              ( MM( 42, J ), J = 1, 4 ) / 3876, 40, 3248,
     $                  3673 /
      DATA              ( MM( 43, J ), J = 1, 4 ) / 1905, 1832, 3401,
     $                  3117 /
      DATA              ( MM( 44, J ), J = 1, 4 ) / 1593, 2247, 2124,
     $                  3089 /
      DATA              ( MM( 45, J ), J = 1, 4 ) / 1797, 2034, 2762,
     $                  1349 /
      DATA              ( MM( 46, J ), J = 1, 4 ) / 1234, 2637, 149,
     $                  2057 /
      DATA              ( MM( 47, J ), J = 1, 4 ) / 3460, 1287, 2245,
     $                  413 /
      DATA              ( MM( 48, J ), J = 1, 4 ) / 328, 1691, 166,
     $                  65 /
      DATA              ( MM( 49, J ), J = 1, 4 ) / 2861, 496, 466,
     $                  1845 /
      DATA              ( MM( 50, J ), J = 1, 4 ) / 1950, 1597, 4018,
     $                  697 /
      DATA              ( MM( 51, J ), J = 1, 4 ) / 617, 2394, 1399,
     $                  3085 /
      DATA              ( MM( 52, J ), J = 1, 4 ) / 2070, 2584, 190,
     $                  3441 /
      DATA              ( MM( 53, J ), J = 1, 4 ) / 3331, 1843, 2879,
     $                  1573 /
      DATA              ( MM( 54, J ), J = 1, 4 ) / 769, 336, 153,
     $                  3689 /
      DATA              ( MM( 55, J ), J = 1, 4 ) / 1558, 1472, 2320,
     $                  2941 /
```

```
      DATA              ( MM( 56, J ), J = 1, 4 ) / 2412, 2407, 18,
     $                  929 /
      DATA              ( MM( 57, J ), J = 1, 4 ) / 2800, 433, 712,
     $                  533 /
      DATA              ( MM( 58, J ), J = 1, 4 ) / 189, 2096, 2159,
     $                  2841 /
      DATA              ( MM( 59, J ), J = 1, 4 ) / 287, 1761, 2318,
     $                  4077 /
      DATA              ( MM( 60, J ), J = 1, 4 ) / 2045, 2810, 2091,
     $                  721 /
      DATA              ( MM( 61, J ), J = 1, 4 ) / 1227, 566, 3443,
     $                  2821 /
      DATA              ( MM( 62, J ), J = 1, 4 ) / 2838, 442, 1510,
     $                  2249 /
      DATA              ( MM( 63, J ), J = 1, 4 ) / 209, 41, 449,
     $                  2397 /
      DATA              ( MM( 64, J ), J = 1, 4 ) / 2770, 1238, 1956,
     $                  2817 /
      DATA              ( MM( 65, J ), J = 1, 4 ) / 3654, 1086, 2201,
     $                  245 /
      DATA              ( MM( 66, J ), J = 1, 4 ) / 3993, 603, 3137,
     $                  1913 /
      DATA              ( MM( 67, J ), J = 1, 4 ) / 192, 840, 3399,
     $                  1997 /
      DATA              ( MM( 68, J ), J = 1, 4 ) / 2253, 3168, 1321,
     $                  3121 /
      DATA              ( MM( 69, J ), J = 1, 4 ) / 3491, 1499, 2271,
     $                  997 /
      DATA              ( MM( 70, J ), J = 1, 4 ) / 2889, 1084, 3667,
     $                  1833 /
      DATA              ( MM( 71, J ), J = 1, 4 ) / 2857, 3438, 2703,
     $                  2877 /
      DATA              ( MM( 72, J ), J = 1, 4 ) / 2094, 2408, 629,
     $                  1633 /
      DATA              ( MM( 73, J ), J = 1, 4 ) / 1818, 1589, 2365,
     $                  981 /
      DATA              ( MM( 74, J ), J = 1, 4 ) / 688, 2391, 2431,
     $                  2009 /
      DATA              ( MM( 75, J ), J = 1, 4 ) / 1407, 288, 1113,
     $                  941 /
      DATA              ( MM( 76, J ), J = 1, 4 ) / 634, 26, 3922,
     $                  2449 /
      DATA              ( MM( 77, J ), J = 1, 4 ) / 3231, 512, 2554,
     $                  197 /
      DATA              ( MM( 78, J ), J = 1, 4 ) / 815, 1456, 184,
     $                  2441 /
      DATA              ( MM( 79, J ), J = 1, 4 ) / 3524, 171, 2099,
     $                  285 /
      DATA              ( MM( 80, J ), J = 1, 4 ) / 1914, 1677, 3228,
     $                  1473 /
```

```
 DATA              ( MM( 81, J ), J = 1, 4 ) / 516, 2657, 4012,
$                  2741 /
 DATA              ( MM( 82, J ), J = 1, 4 ) / 164, 2270, 1921,
$                  3129 /
 DATA              ( MM( 83, J ), J = 1, 4 ) / 303, 2587, 3452,
$                  909 /
 DATA              ( MM( 84, J ), J = 1, 4 ) / 2144, 2961, 3901,
$                  2801 /
 DATA              ( MM( 85, J ), J = 1, 4 ) / 3480, 1970, 572,
$                  421 /
 DATA              ( MM( 86, J ), J = 1, 4 ) / 119, 1817, 3309,
$                  4073 /
 DATA              ( MM( 87, J ), J = 1, 4 ) / 3357, 676, 3171,
$                  2813 /
 DATA              ( MM( 88, J ), J = 1, 4 ) / 837, 1410, 817,
$                  2337 /
 DATA              ( MM( 89, J ), J = 1, 4 ) / 2826, 3723, 3039,
$                  1429 /
 DATA              ( MM( 90, J ), J = 1, 4 ) / 2332, 2803, 1696,
$                  1177 /
 DATA              ( MM( 91, J ), J = 1, 4 ) / 2089, 3185, 1256,
$                  1901 /
 DATA              ( MM( 92, J ), J = 1, 4 ) / 3780, 184, 3715,
$                  81 /
 DATA              ( MM( 93, J ), J = 1, 4 ) / 1700, 663, 2077,
$                  1669 /
 DATA              ( MM( 94, J ), J = 1, 4 ) / 3712, 499, 3019,
$                  2633 /
 DATA              ( MM( 95, J ), J = 1, 4 ) / 150, 3784, 1497,
$                  2269 /
 DATA              ( MM( 96, J ), J = 1, 4 ) / 2000, 1631, 1101,
$                  129 /
 DATA              ( MM( 97, J ), J = 1, 4 ) / 3375, 1925, 717,
$                  1141 /
 DATA              ( MM( 98, J ), J = 1, 4 ) / 1621, 3912, 51,
$                  249 /
 DATA              ( MM( 99, J ), J = 1, 4 ) / 3090, 1398, 981,
$                  3917 /
 DATA              ( MM( 100, J ), J = 1, 4 ) / 3765, 1349, 1978,
$                  2481 /
 DATA              ( MM( 101, J ), J = 1, 4 ) / 1149, 1441, 1813,
$                  3941 /
 DATA              ( MM( 102, J ), J = 1, 4 ) / 3146, 2224, 3881,
$                  2217 /
 DATA              ( MM( 103, J ), J = 1, 4 ) / 33, 2411, 76,
$                  2749 /
 DATA              ( MM( 104, J ), J = 1, 4 ) / 3082, 1907, 3846,
$                  3041 /
 DATA              ( MM( 105, J ), J = 1, 4 ) / 2741, 3192, 3694,
$                  1877 /
```

```
      DATA              ( MM( 106, J ), J = 1, 4 ) / 359, 2786, 1682,
     $                  345 /
      DATA              ( MM( 107, J ), J = 1, 4 ) / 3316, 382, 124,
     $                  2861 /
      DATA              ( MM( 108, J ), J = 1, 4 ) / 1749, 37, 1660,
     $                  1809 /
      DATA              ( MM( 109, J ), J = 1, 4 ) / 185, 759, 3997,
     $                  3141 /
      DATA              ( MM( 110, J ), J = 1, 4 ) / 2784, 2948, 479,
     $                  2825 /
      DATA              ( MM( 111, J ), J = 1, 4 ) / 2202, 1862, 1141,
     $                  157 /
      DATA              ( MM( 112, J ), J = 1, 4 ) / 2199, 3802, 886,
     $                  2881 /
      DATA              ( MM( 113, J ), J = 1, 4 ) / 1364, 2423, 3514,
     $                  3637 /
      DATA              ( MM( 114, J ), J = 1, 4 ) / 1244, 2051, 1301,
     $                  1465 /
      DATA              ( MM( 115, J ), J = 1, 4 ) / 2020, 2295, 3604,
     $                  2829 /
      DATA              ( MM( 116, J ), J = 1, 4 ) / 3160, 1332, 1888,
     $                  2161 /
      DATA              ( MM( 117, J ), J = 1, 4 ) / 2785, 1832, 1836,
     $                  3365 /
      DATA              ( MM( 118, J ), J = 1, 4 ) / 2772, 2405, 1990,
     $                  361 /
      DATA              ( MM( 119, J ), J = 1, 4 ) / 1217, 3638, 2058,
     $                  2685 /
      DATA              ( MM( 120, J ), J = 1, 4 ) / 1822, 3661, 692,
     $                  3745 /
      DATA              ( MM( 121, J ), J = 1, 4 ) / 1245, 327, 1194,
     $                  2325 /
      DATA              ( MM( 122, J ), J = 1, 4 ) / 2252, 3660, 20,
     $                  3609 /
      DATA              ( MM( 123, J ), J = 1, 4 ) / 3904, 716, 3285,
     $                  3821 /
      DATA              ( MM( 124, J ), J = 1, 4 ) / 2774, 1842, 2046,
     $                  3537 /
      DATA              ( MM( 125, J ), J = 1, 4 ) / 997, 3987, 2107,
     $                  517 /
      DATA              ( MM( 126, J ), J = 1, 4 ) / 2573, 1368, 3508,
     $                  3017 /
      DATA              ( MM( 127, J ), J = 1, 4 ) / 1148, 1848, 3525,
     $                  2141 /
      DATA              ( MM( 128, J ), J = 1, 4 ) / 545, 2366, 3801,
     $                  1537 /
*     ..
*     .. Executable Statements ..
*
      I1 = ISEED( 1 )
```

```
      I2 = ISEED( 2 )
      I3 = ISEED( 3 )
      I4 = ISEED( 4 )
*
      DO 10 I = 1, MIN( N, LV )
*
  20     CONTINUE
*
*        Multiply the seed by i-th power of the multiplier modulo 2**48
*
         IT4 = I4*MM( I, 4 )
         IT3 = IT4 / IPW2
         IT4 = IT4 - IPW2*IT3
         IT3 = IT3 + I3*MM( I, 4 ) + I4*MM( I, 3 )
         IT2 = IT3 / IPW2
         IT3 = IT3 - IPW2*IT2
         IT2 = IT2 + I2*MM( I, 4 ) + I3*MM( I, 3 ) + I4*MM( I, 2 )
         IT1 = IT2 / IPW2
         IT2 = IT2 - IPW2*IT1
         IT1 = IT1 + I1*MM( I, 4 ) + I2*MM( I, 3 ) + I3*MM( I, 2 ) +
     $         I4*MM( I, 1 )
         IT1 = MOD( IT1, IPW2 )
*
*        Convert 48-bit integer to a real number in the interval (0,1)
*
         X( I ) = R*( REAL( IT1 )+R*( REAL( IT2 )+R*( REAL( IT3 )+R*
     $            REAL( IT4 ) ) ) )
*
         IF (X( I ).EQ.1.0) THEN
*           If a real number has n bits of precision, and the first
*           n bits of the 48-bit integer above happen to be all 1 (which
*           will occur about once every 2**n calls), then X( I ) will
*           be rounded to exactly 1.0. In IEEE single precision arithmetic,
*           this will happen relatively often since n = 24.
*           Since X( I ) is not supposed to return exactly 0.0 or 1.0,
*           the statistically correct thing to do in this situation is
*           simply to iterate again.
*           N.B. the case X( I ) = 0.0 should not be possible.
            I1 = I1 + 2
            I2 = I2 + 2
            I3 = I3 + 2
            I4 = I4 + 2
            GOTO 20
         END IF
*
   10 CONTINUE
*
*     Return final value of seed
*
      ISEED( 1 ) = IT1
```

```
      ISEED( 2 ) = IT2
      ISEED( 3 ) = IT3
      ISEED( 4 ) = IT4
      RETURN
*
*     End of SLARUV
*
      END
```

90.89.4 lisp code

— slaruv.lisp —

```
(let* ((one 1.0) (lv 128) (ipw2 4096) (r (f2cl-lib:f2cl/ one ipw2)))
  (declare (type (single-float 1.0 1.0) one)
           (type (f2cl-lib:integer4 128 128) lv)
           (type (f2cl-lib:integer4 4096 4096) ipw2)
           (type (single-float) r)
           (ignorable one lv ipw2 r))
  (let ((mm
         (make-array (the fixnum (reduce #'* (list lv 4))) :element-type
                     'f2cl-lib:integer4)))
    (declare (type (array f2cl-lib:integer4 (*)) mm))
    (f2cl-lib:data-implied-do (((mm (1 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (494 322 2508 2549))
    (f2cl-lib:data-implied-do (((mm (2 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (2637 789 3754 1145))
    (f2cl-lib:data-implied-do (((mm (3 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (255 1440 1766 2253))
    (f2cl-lib:data-implied-do (((mm (4 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (2008 752 3572 305))
    (f2cl-lib:data-implied-do (((mm (5 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (1253 2859 2893 3301))
    (f2cl-lib:data-implied-do (((mm (6 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (3344 123 307 1065))
    (f2cl-lib:data-implied-do (((mm (7 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (4084 1848 1297 3133))
    (f2cl-lib:data-implied-do (((mm (8 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (1739 643 3966 2913))
    (f2cl-lib:data-implied-do (((mm (9 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (3143 2405 758 3285))
    (f2cl-lib:data-implied-do (((mm (10 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (3468 2638 2598 1241))
    (f2cl-lib:data-implied-do (((mm (11 j))) (j 1 4)) (((1 lv) (1 4)))
                              ((f2cl-lib:integer4)) (688 2344 3406 1197))
    (f2cl-lib:data-implied-do (((mm (12 j))) (j 1 4)) (((1 lv) (1 4)))
```

```
                          ((f2cl-lib:integer4)) (1657 46 2922 3729))
(f2cl-lib:data-implied-do (((mm (13 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1238 3814 1038 2501))
(f2cl-lib:data-implied-do (((mm (14 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3166 913 2934 1673))
(f2cl-lib:data-implied-do (((mm (15 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1292 3649 2091 541))
(f2cl-lib:data-implied-do (((mm (16 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3422 339 2451 2753))
(f2cl-lib:data-implied-do (((mm (17 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1270 3808 1580 949))
(f2cl-lib:data-implied-do (((mm (18 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2016 822 1958 2361))
(f2cl-lib:data-implied-do (((mm (19 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (154 2832 2055 1165))
(f2cl-lib:data-implied-do (((mm (20 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2862 3078 1507 4081))
(f2cl-lib:data-implied-do (((mm (21 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (697 3633 1078 2725))
(f2cl-lib:data-implied-do (((mm (22 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1706 2970 3273 3305))
(f2cl-lib:data-implied-do (((mm (23 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (491 637 17 3069))
(f2cl-lib:data-implied-do (((mm (24 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (931 2249 854 3617))
(f2cl-lib:data-implied-do (((mm (25 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1444 2081 2916 3733))
(f2cl-lib:data-implied-do (((mm (26 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (444 4019 3971 409))
(f2cl-lib:data-implied-do (((mm (27 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3577 1478 2889 2157))
(f2cl-lib:data-implied-do (((mm (28 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3944 242 3831 1361))
(f2cl-lib:data-implied-do (((mm (29 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2184 481 2621 3973))
(f2cl-lib:data-implied-do (((mm (30 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1661 2075 1541 1865))
(f2cl-lib:data-implied-do (((mm (31 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3482 4058 893 2525))
(f2cl-lib:data-implied-do (((mm (32 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (657 622 736 1409))
(f2cl-lib:data-implied-do (((mm (33 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3023 3376 3992 3445))
(f2cl-lib:data-implied-do (((mm (34 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3618 812 787 3577))
(f2cl-lib:data-implied-do (((mm (35 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1267 234 2125 77))
(f2cl-lib:data-implied-do (((mm (36 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1828 641 2364 3761))
(f2cl-lib:data-implied-do (((mm (37 j))) (j 1 4)) (((1 lv) (1 4)))
```

```
                          ((f2cl-lib:integer4)) (164 4005 2460 2149))
(f2cl-lib:data-implied-do (((mm (38 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3798 1122 257 1449))
(f2cl-lib:data-implied-do (((mm (39 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3087 3135 1574 3005))
(f2cl-lib:data-implied-do (((mm (40 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2400 2640 3912 225))
(f2cl-lib:data-implied-do (((mm (41 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2870 2302 1216 85))
(f2cl-lib:data-implied-do (((mm (42 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3876 40 3248 3673))
(f2cl-lib:data-implied-do (((mm (43 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1905 1832 3401 3117))
(f2cl-lib:data-implied-do (((mm (44 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1593 2247 2124 3089))
(f2cl-lib:data-implied-do (((mm (45 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1797 2034 2762 1349))
(f2cl-lib:data-implied-do (((mm (46 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1234 2637 149 2057))
(f2cl-lib:data-implied-do (((mm (47 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3460 1287 2245 413))
(f2cl-lib:data-implied-do (((mm (48 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (328 1691 166 65))
(f2cl-lib:data-implied-do (((mm (49 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2861 496 466 1845))
(f2cl-lib:data-implied-do (((mm (50 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1950 1597 4018 697))
(f2cl-lib:data-implied-do (((mm (51 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (617 2394 1399 3085))
(f2cl-lib:data-implied-do (((mm (52 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2070 2584 190 3441))
(f2cl-lib:data-implied-do (((mm (53 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3331 1843 2879 1573))
(f2cl-lib:data-implied-do (((mm (54 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (769 336 153 3689))
(f2cl-lib:data-implied-do (((mm (55 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1558 1472 2320 2941))
(f2cl-lib:data-implied-do (((mm (56 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2412 2407 18 929))
(f2cl-lib:data-implied-do (((mm (57 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2800 433 712 533))
(f2cl-lib:data-implied-do (((mm (58 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (189 2096 2159 2841))
(f2cl-lib:data-implied-do (((mm (59 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (287 1761 2318 4077))
(f2cl-lib:data-implied-do (((mm (60 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2045 2810 2091 721))
(f2cl-lib:data-implied-do (((mm (61 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1227 566 3443 2821))
(f2cl-lib:data-implied-do (((mm (62 j))) (j 1 4)) (((1 lv) (1 4)))
```

```
                          ((f2cl-lib:integer4)) (2838 442 1510 2249))
(f2cl-lib:data-implied-do (((mm (63 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (209 41 449 2397))
(f2cl-lib:data-implied-do (((mm (64 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2770 1238 1956 2817))
(f2cl-lib:data-implied-do (((mm (65 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3654 1086 2201 245))
(f2cl-lib:data-implied-do (((mm (66 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3993 603 3137 1913))
(f2cl-lib:data-implied-do (((mm (67 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (192 840 3399 1997))
(f2cl-lib:data-implied-do (((mm (68 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2253 3168 1321 3121))
(f2cl-lib:data-implied-do (((mm (69 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3491 1499 2271 997))
(f2cl-lib:data-implied-do (((mm (70 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2889 1084 3667 1833))
(f2cl-lib:data-implied-do (((mm (71 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2857 3438 2703 2877))
(f2cl-lib:data-implied-do (((mm (72 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2094 2408 629 1633))
(f2cl-lib:data-implied-do (((mm (73 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1818 1589 2365 981))
(f2cl-lib:data-implied-do (((mm (74 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (688 2391 2431 2009))
(f2cl-lib:data-implied-do (((mm (75 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1407 288 1113 941))
(f2cl-lib:data-implied-do (((mm (76 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (634 26 3922 2449))
(f2cl-lib:data-implied-do (((mm (77 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3231 512 2554 197))
(f2cl-lib:data-implied-do (((mm (78 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (815 1456 184 2441))
(f2cl-lib:data-implied-do (((mm (79 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3524 171 2099 285))
(f2cl-lib:data-implied-do (((mm (80 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1914 1677 3228 1473))
(f2cl-lib:data-implied-do (((mm (81 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (516 2657 4012 2741))
(f2cl-lib:data-implied-do (((mm (82 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (164 2270 1921 3129))
(f2cl-lib:data-implied-do (((mm (83 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (303 2587 3452 909))
(f2cl-lib:data-implied-do (((mm (84 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2144 2961 3901 2801))
(f2cl-lib:data-implied-do (((mm (85 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3480 1970 572 421))
(f2cl-lib:data-implied-do (((mm (86 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (119 1817 3309 4073))
(f2cl-lib:data-implied-do (((mm (87 j))) (j 1 4)) (((1 lv) (1 4)))
```

```
                          ((f2cl-lib:integer4)) (3357 676 3171 2813))
(f2cl-lib:data-implied-do (((mm (88 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (837 1410 817 2337))
(f2cl-lib:data-implied-do (((mm (89 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2826 3723 3039 1429))
(f2cl-lib:data-implied-do (((mm (90 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2332 2803 1696 1177))
(f2cl-lib:data-implied-do (((mm (91 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2089 3185 1256 1901))
(f2cl-lib:data-implied-do (((mm (92 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3780 184 3715 81))
(f2cl-lib:data-implied-do (((mm (93 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1700 663 2077 1669))
(f2cl-lib:data-implied-do (((mm (94 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3712 499 3019 2633))
(f2cl-lib:data-implied-do (((mm (95 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (150 3784 1497 2269))
(f2cl-lib:data-implied-do (((mm (96 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2000 1631 1101 129))
(f2cl-lib:data-implied-do (((mm (97 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3375 1925 717 1141))
(f2cl-lib:data-implied-do (((mm (98 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1621 3912 51 249))
(f2cl-lib:data-implied-do (((mm (99 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3090 1398 981 3917))
(f2cl-lib:data-implied-do (((mm (100 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3765 1349 1978 2481))
(f2cl-lib:data-implied-do (((mm (101 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1149 1441 1813 3941))
(f2cl-lib:data-implied-do (((mm (102 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3146 2224 3881 2217))
(f2cl-lib:data-implied-do (((mm (103 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (33 2411 76 2749))
(f2cl-lib:data-implied-do (((mm (104 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3082 1907 3846 3041))
(f2cl-lib:data-implied-do (((mm (105 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2741 3192 3694 1877))
(f2cl-lib:data-implied-do (((mm (106 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (359 2786 1682 345))
(f2cl-lib:data-implied-do (((mm (107 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3316 382 124 2861))
(f2cl-lib:data-implied-do (((mm (108 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1749 37 1660 1809))
(f2cl-lib:data-implied-do (((mm (109 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (185 759 3997 3141))
(f2cl-lib:data-implied-do (((mm (110 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2784 2948 479 2825))
(f2cl-lib:data-implied-do (((mm (111 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2202 1862 1141 157))
(f2cl-lib:data-implied-do (((mm (112 j))) (j 1 4)) (((1 lv) (1 4)))
```

```
                          ((f2cl-lib:integer4)) (2199 3802 886 2881))
(f2cl-lib:data-implied-do (((mm (113 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1364 2423 3514 3637))
(f2cl-lib:data-implied-do (((mm (114 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1244 2051 1301 1465))
(f2cl-lib:data-implied-do (((mm (115 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2020 2295 3604 2829))
(f2cl-lib:data-implied-do (((mm (116 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3160 1332 1888 2161))
(f2cl-lib:data-implied-do (((mm (117 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2785 1832 1836 3365))
(f2cl-lib:data-implied-do (((mm (118 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2772 2405 1990 361))
(f2cl-lib:data-implied-do (((mm (119 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1217 3638 2058 2685))
(f2cl-lib:data-implied-do (((mm (120 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1822 3661 692 3745))
(f2cl-lib:data-implied-do (((mm (121 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1245 327 1194 2325))
(f2cl-lib:data-implied-do (((mm (122 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2252 3660 20 3609))
(f2cl-lib:data-implied-do (((mm (123 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (3904 716 3285 3821))
(f2cl-lib:data-implied-do (((mm (124 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2774 1842 2046 3537))
(f2cl-lib:data-implied-do (((mm (125 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (997 3987 2107 517))
(f2cl-lib:data-implied-do (((mm (126 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (2573 1368 3508 3017))
(f2cl-lib:data-implied-do (((mm (127 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (1148 1848 3525 2141))
(f2cl-lib:data-implied-do (((mm (128 j))) (j 1 4)) (((1 lv) (1 4)))
                          ((f2cl-lib:integer4)) (545 2366 3801 1537))
(defun slaruv (iseed n x)
  (declare (type (array single-float (*)) x)
           (type (f2cl-lib:integer4) n)
           (type (array f2cl-lib:integer4 (*)) iseed))
  (f2cl-lib:with-multi-array-data
      ((iseed f2cl-lib:integer4 iseed-%data% iseed-%offset%)
       (x single-float x-%data% x-%offset%))
    (prog ((i 0) (i1 0) (i2 0) (i3 0) (i4 0) (it1 0) (it2 0) (it3 0)
           (it4 0) (j 0))
      (declare (type (f2cl-lib:integer4) i i1 i2 i3 i4 it1 it2 it3 it4 j))
      (setf i1 (f2cl-lib:fref iseed-%data% (1) ((1 4)) iseed-%offset%))
      (setf i2 (f2cl-lib:fref iseed-%data% (2) ((1 4)) iseed-%offset%))
      (setf i3 (f2cl-lib:fref iseed-%data% (3) ((1 4)) iseed-%offset%))
      (setf i4 (f2cl-lib:fref iseed-%data% (4) ((1 4)) iseed-%offset%))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (min (the f2cl-lib:integer4 n)
```

```
                        (the f2cl-lib:integer4 lv)))
             nil)
(tagbody
 label20
  (setf it4
          (f2cl-lib:int-mul i4
                            (f2cl-lib:fref mm (i 4)
                                           ((1 lv) (1 4)))))
  (setf it3 (the f2cl-lib:integer4 (truncate it4 ipw2)))
  (setf it4 (f2cl-lib:int-sub it4 (f2cl-lib:int-mul ipw2 it3)))
  (setf it3
          (f2cl-lib:int-add it3
                            (f2cl-lib:int-mul i3
                                              (f2cl-lib:fref mm
                                                             (i 4)
                                                             ((1
                                                               lv)
                                                              (1
                                                               4))))
                            (f2cl-lib:int-mul i4
                                              (f2cl-lib:fref mm
                                                             (i 3)
                                                             ((1
                                                               lv)
                                                              (1
                                                               4))))))
  (setf it2 (the f2cl-lib:integer4 (truncate it3 ipw2)))
  (setf it3 (f2cl-lib:int-sub it3 (f2cl-lib:int-mul ipw2 it2)))
  (setf it2
          (f2cl-lib:int-add it2
                            (f2cl-lib:int-mul i2
                                              (f2cl-lib:fref mm
                                                             (i 4)
                                                             ((1
                                                               lv)
                                                              (1
                                                               4))))
                            (f2cl-lib:int-mul i3
                                              (f2cl-lib:fref mm
                                                             (i 3)
                                                             ((1
                                                               lv)
                                                              (1
                                                               4))))
                            (f2cl-lib:int-mul i4
                                              (f2cl-lib:fref mm
                                                             (i 2)
                                                             ((1
                                                               lv)
                                                              (1
```

```
                                                                  4))))))
        (setf it1 (the f2cl-lib:integer4 (truncate it2 ipw2)))
        (setf it2 (f2cl-lib:int-sub it2 (f2cl-lib:int-mul ipw2 it1)))
        (setf it1
                (f2cl-lib:int-add it1
                                  (f2cl-lib:int-mul i1
                                                    (f2cl-lib:fref mm
                                                                   (i 4)
                                                                   ((1
                                                                     lv)
                                                                    (1
                                                                     4))))
                                  (f2cl-lib:int-mul i2
                                                    (f2cl-lib:fref mm
                                                                   (i 3)
                                                                   ((1
                                                                     lv)
                                                                    (1
                                                                     4))))
                                  (f2cl-lib:int-mul i3
                                                    (f2cl-lib:fref mm
                                                                   (i 2)
                                                                   ((1
                                                                     lv)
                                                                    (1
                                                                     4))))
                                  (f2cl-lib:int-mul i4
                                                    (f2cl-lib:fref mm
                                                                   (i 1)
                                                                   ((1
                                                                     lv)
                                                                    (1
                                                                     4))))))
        (setf it1 (mod it1 ipw2))
        (setf (f2cl-lib:fref x-%data% (i) ((1 n)) x-%offset%)
                (* r
                   (+ (f2cl-lib:freal it1)
                      (* r
                         (+ (f2cl-lib:freal it2)
                            (* r
                               (+ (f2cl-lib:freal it3)
                                  (* r (f2cl-lib:freal it4)))))))))
        (cond
         ((= (f2cl-lib:fref x (i) ((1 n))) 1.0)
          (setf i1 (f2cl-lib:int-add i1 2))
          (setf i2 (f2cl-lib:int-add i2 2))
          (setf i3 (f2cl-lib:int-add i3 2))
          (setf i4 (f2cl-lib:int-add i4 2)) (go label20)))
      label10))
    (setf (f2cl-lib:fref iseed-%data% (1) ((1 4)) iseed-%offset%) it1)
```

```
          (setf (f2cl-lib:fref iseed-%data% (2) ((1 4)) iseed-%offset%) it2)
          (setf (f2cl-lib:fref iseed-%data% (3) ((1 4)) iseed-%offset%) it3)
          (setf (f2cl-lib:fref iseed-%data% (4) ((1 4)) iseed-%offset%) it4)
          (go end_label)
         end_label
          (return (values nil nil nil)))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slaruv
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*)))
                                            :return-values '(nil nil nil)
                                            :calls 'nil)))
```

———-

90.90 slas2

90.90.1 Axiom unit tests

— **slas2.input** —

```
)set break resume
)sys rm -f slas2.output
)spool slas2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.90.2 Axiom help page

— **slas2.help** —

```
====================================================================
slas2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slas2 (F, G, H, SSMIN, SSMAX)
           SLAS2 computes singular values of a 2-by-2 triangular matrix.

Function/Subroutine Documentation
   subroutine slas2 (real F, real G, real H, real SSMIN, real SSMAX)
       SLAS2 computes singular values of a 2-by-2 triangular matrix.

Purpose:
   SLAS2  computes the singular values of the 2-by-2 matrix
      [  F   G  ]
      [  0   H  ].
   On return, SSMIN is the smaller singular value and SSMAX is the
   larger singular value.

Parameters:

 F
   F is REAL
   The (1,1) element of the 2-by-2 matrix.

 G
   G is REAL
   The (1,2) element of the 2-by-2 matrix.

 H
   H is REAL
   The (2,2) element of the 2-by-2 matrix.

 SSMIN
   SSMIN is REAL
   The smaller singular value.

 SSMAX
   SSMAX is REAL
   The larger singular value.

Further Details:

   Barring over/underflow, all output quantities are correct to within
   a few units in the last place (ulps), even in the absence of a guard
```

```
digit in addition/subtraction.

In IEEE arithmetic, the code works correctly if one matrix element is
infinite.

Overflow will not occur unless the largest singular value itself
overflows, or is within a few ulps of overflow. (On machines with
partial overflow, like the Cray, overflow may occur if the largest
singular value is within a factor of 2 of overflow.)

Underflow is harmless if underflow is gradual. Otherwise, results
may correspond to a matrix modified by perturbations of size near
the underflow threshold.
```

———-

90.90.3 fortran code

— **slas2.f** —

```
*  =====================================================================
      SUBROUTINE SLAS2( F, G, H, SSMIN, SSMAX )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      REAL               F, G, H, SSMAX, SSMIN
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E0 )
      REAL               ONE
      PARAMETER          ( ONE = 1.0E0 )
      REAL               TWO
      PARAMETER          ( TWO = 2.0E0 )
*     ..
*     .. Local Scalars ..
      REAL               AS, AT, AU, C, FA, FHMN, FHMX, GA, HA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT
```

```
*     ..
*     .. Executable Statements ..
*
      FA = ABS( F )
      GA = ABS( G )
      HA = ABS( H )
      FHMN = MIN( FA, HA )
      FHMX = MAX( FA, HA )
      IF( FHMN.EQ.ZERO ) THEN
         SSMIN = ZERO
         IF( FHMX.EQ.ZERO ) THEN
            SSMAX = GA
         ELSE
            SSMAX = MAX( FHMX, GA )*SQRT( ONE+
     $              ( MIN( FHMX, GA ) / MAX( FHMX, GA ) )**2 )
         END IF
      ELSE
         IF( GA.LT.FHMX ) THEN
            AS = ONE + FHMN / FHMX
            AT = ( FHMX-FHMN ) / FHMX
            AU = ( GA / FHMX )**2
            C = TWO / ( SQRT( AS*AS+AU )+SQRT( AT*AT+AU ) )
            SSMIN = FHMN*C
            SSMAX = FHMX / C
         ELSE
            AU = FHMX / GA
            IF( AU.EQ.ZERO ) THEN
*
*              Avoid possible harmful underflow if exponent range
*              asymmetric (true SSMIN may not underflow even if
*              AU underflows)
*
               SSMIN = ( FHMN*FHMX ) / GA
               SSMAX = GA
            ELSE
               AS = ONE + FHMN / FHMX
               AT = ( FHMX-FHMN ) / FHMX
               C = ONE / ( SQRT( ONE+( AS*AU )**2 )+
     $             SQRT( ONE+( AT*AU )**2 ) )
               SSMIN = ( FHMN*C )*AU
               SSMIN = SSMIN + SSMIN
               SSMAX = GA / ( C+C )
            END IF
         END IF
      END IF
      RETURN
*
*     End of SLAS2
*
      END
```

———-

90.90.4 lisp code

— slas2.lisp —

```
(let* ((zero 0.0) (one 1.0) (two 2.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (ignorable zero one two))
  (defun slas2 (f g h ssmin ssmax)
    (declare (type (single-float) ssmax ssmin h g f))
    (prog ((as 0.0) (at 0.0) (au 0.0) (c 0.0) (fa 0.0) (fhmn 0.0) (fhmx 0.0)
           (ga 0.0) (ha 0.0))
      (declare (type (single-float) as at au c fa fhmn fhmx ga ha))
      (setf fa (abs f))
      (setf ga (abs g))
      (setf ha (abs h))
      (setf fhmn (min fa ha))
      (setf fhmx (max fa ha))
      (cond
       ((= fhmn zero) (setf ssmin zero)
        (cond ((= fhmx zero) (setf ssmax ga))
              (t
               (setf ssmax
                       (* (max fhmx ga)
                          (f2cl-lib:fsqrt
                           (+ one
                              (expt (/ (min fhmx ga) (max fhmx ga)) 2))))))))
       (t
        (cond
         ((< ga fhmx) (setf as (+ one (/ fhmn fhmx)))
          (setf at (/ (- fhmx fhmn) fhmx)) (setf au (expt (/ ga fhmx) 2))
          (setf c
                  (/ two
                     (+ (f2cl-lib:fsqrt (+ (* as as) au))
                        (f2cl-lib:fsqrt (+ (* at at) au)))))
          (setf ssmin (* fhmn c)) (setf ssmax (/ fhmx c)))
         (t (setf au (/ fhmx ga))
          (cond ((= au zero) (setf ssmin (/ (* fhmn fhmx) ga)) (setf ssmax ga))
                (t (setf as (+ one (/ fhmn fhmx)))
                 (setf at (/ (- fhmx fhmn) fhmx))
                 (setf c
                         (/ one
                            (+ (f2cl-lib:fsqrt (+ one (expt (* as au) 2)))
                               (f2cl-lib:fsqrt (+ one (expt (* at au) 2))))))
                 (setf ssmin (* fhmn c au)) (setf ssmin (+ ssmin ssmin))
```

```
                (setf ssmax (/ ga (+ c c)))))))))
      (go end_label)
     end_label
      (return (values nil nil nil ssmin ssmax)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slas2 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((single-float) (single-float)
                                              (single-float) (single-float)
                                              (single-float))
                                            :return-values
                                            '(nil nil nil
                                              fortran-to-lisp::ssmin
                                              fortran-to-lisp::ssmax)
                                            :calls 'nil)))
```

———-

90.91 slascl

90.91.1 Axiom unit tests

— slascl.input —

```
)set break resume
)sys rm -f slascl.output
)spool slascl.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.91.2 Axiom help page

— slascl.help —

```
=====================================================================
slascl examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slascl (TYPE, KL, KU, CFROM, CTO, M, N, A, LDA, INFO)
           SLASCL multiplies a general rectangular matrix by a real scalar
           defined as cto/cfrom.

Function/Subroutine Documentation
   subroutine slascl (character TYPE, integer KL, integer KU, real CFROM, real
       CTO, integer M, integer N, real, dimension( lda, * ) A, integer LDA,
       integer INFO)
       SLASCL multiplies a general rectangular matrix by a real scalar defined
       as cto/cfrom.

Purpose:
   SLASCL multiplies the M by N real matrix A by the real scalar
   CTO/CFROM.  This is done without over/underflow as long as the final
   result CTO*A(I,J)/CFROM does not over/underflow. TYPE specifies that
   A may be full, upper triangular, lower triangular, upper Hessenberg,
   or banded.

Parameters:

 TYPE
   TYPE is CHARACTER*1
   TYPE indices the storage type of the input matrix.
   = 'G':  A is a full matrix.
   = 'L':  A is a lower triangular matrix.
   = 'U':  A is an upper triangular matrix.
   = 'H':  A is an upper Hessenberg matrix.
   = 'B':  A is a symmetric band matrix with lower bandwidth KL
      and upper bandwidth KU and with the only the lower half stored.
   = 'Q':  A is a symmetric band matrix with lower bandwidth KL
      and upper bandwidth KU and with the only the upper half stored.
   = 'Z':  A is a band matrix with lower bandwidth KL and upper
      bandwidth KU. See SGBTRF for storage details.

 KL
   KL is INTEGER
   The lower bandwidth of A.  Referenced only if TYPE = 'B', 'Q' or 'Z'.

 KU
   KU is INTEGER
```

```
  The upper bandwidth of A.  Referenced only if TYPE = 'B', 'Q' or 'Z'.

CFROM
  CFROM is REAL

CTO
  CTO is REAL

  The matrix A is multiplied by CTO/CFROM. A(I,J) is computed
  without over/underflow if the final result CTO*A(I,J)/CFROM
  can be represented without over/underflow.  CFROM must be nonzero.

M
  M is INTEGER
  The number of rows of the matrix A.  M >= 0.

N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

A
  A is REAL array, dimension (LDA,N)
  The matrix to be multiplied by CTO/CFROM.  See TYPE for the storage type.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).

INFO
  INFO is INTEGER
  0  - successful exit
  < 0 - if INFO = -i, the i-th argument had an illegal value.
```

———-

90.91.3 fortran code

— slascl.f —

```
*  =====================================================================
      SUBROUTINE SLASCL( TYPE, KL, KU, CFROM, CTO, M, N, A, LDA, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
```

```
*     .. Scalar Arguments ..
      CHARACTER          TYPE
      INTEGER            INFO, KL, KU, LDA, M, N
      REAL               CFROM, CTO
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E0, ONE = 1.0E0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            DONE
      INTEGER            I, ITYPE, J, K1, K2, K3, K4
      REAL               BIGNUM, CFROM1, CFROMC, CTO1, CTOC, MUL, SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, SISNAN
      REAL               SLAMCH
      EXTERNAL           LSAME, SLAMCH, SISNAN
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
*
      IF( LSAME( TYPE, 'G' ) ) THEN
         ITYPE = 0
      ELSE IF( LSAME( TYPE, 'L' ) ) THEN
         ITYPE = 1
      ELSE IF( LSAME( TYPE, 'U' ) ) THEN
         ITYPE = 2
      ELSE IF( LSAME( TYPE, 'H' ) ) THEN
         ITYPE = 3
      ELSE IF( LSAME( TYPE, 'B' ) ) THEN
         ITYPE = 4
      ELSE IF( LSAME( TYPE, 'Q' ) ) THEN
         ITYPE = 5
      ELSE IF( LSAME( TYPE, 'Z' ) ) THEN
```

```
         ITYPE = 6
      ELSE
         ITYPE = -1
      END IF
*
      IF( ITYPE.EQ.-1 ) THEN
         INFO = -1
      ELSE IF( CFROM.EQ.ZERO .OR. SISNAN(CFROM) ) THEN
         INFO = -4
      ELSE IF( SISNAN(CTO) ) THEN
         INFO = -5
      ELSE IF( M.LT.0 ) THEN
         INFO = -6
      ELSE IF( N.LT.0 .OR. ( ITYPE.EQ.4 .AND. N.NE.M ) .OR.
     $         ( ITYPE.EQ.5 .AND. N.NE.M ) ) THEN
         INFO = -7
      ELSE IF( ITYPE.LE.3 .AND. LDA.LT.MAX( 1, M ) ) THEN
         INFO = -9
      ELSE IF( ITYPE.GE.4 ) THEN
         IF( KL.LT.0 .OR. KL.GT.MAX( M-1, 0 ) ) THEN
            INFO = -2
         ELSE IF( KU.LT.0 .OR. KU.GT.MAX( N-1, 0 ) .OR.
     $            ( ( ITYPE.EQ.4 .OR. ITYPE.EQ.5 ) .AND. KL.NE.KU ) )
     $             THEN
            INFO = -3
         ELSE IF( ( ITYPE.EQ.4 .AND. LDA.LT.KL+1 ) .OR.
     $            ( ITYPE.EQ.5 .AND. LDA.LT.KU+1 ) .OR.
     $            ( ITYPE.EQ.6 .AND. LDA.LT.2*KL+KU+1 ) ) THEN
            INFO = -9
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SLASCL', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. M.EQ.0 )
     $   RETURN
*
*     Get machine parameters
*
      SMLNUM = SLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
*
      CFROMC = CFROM
      CTOC = CTO
*
```

```
   10 CONTINUE
      CFROM1 = CFROMC*SMLNUM
      IF( CFROM1.EQ.CFROMC ) THEN
!        CFROMC is an inf.  Multiply by a correctly signed zero for
!        finite CTOC, or a NaN if CTOC is infinite.
         MUL = CTOC / CFROMC
         DONE = .TRUE.
         CTO1 = CTOC
      ELSE
         CTO1 = CTOC / BIGNUM
         IF( CTO1.EQ.CTOC ) THEN
!           CTOC is either 0 or an inf.  In both cases, CTOC itself
!           serves as the correct multiplication factor.
            MUL = CTOC
            DONE = .TRUE.
            CFROMC = ONE
         ELSE IF( ABS( CFROM1 ).GT.ABS( CTOC ) .AND. CTOC.NE.ZERO ) THEN
            MUL = SMLNUM
            DONE = .FALSE.
            CFROMC = CFROM1
         ELSE IF( ABS( CTO1 ).GT.ABS( CFROMC ) ) THEN
            MUL = BIGNUM
            DONE = .FALSE.
            CTOC = CTO1
         ELSE
            MUL = CTOC / CFROMC
            DONE = .TRUE.
         END IF
      END IF
*
      IF( ITYPE.EQ.0 ) THEN
*
*        Full matrix
*
         DO 30 J = 1, N
            DO 20 I = 1, M
               A( I, J ) = A( I, J )*MUL
   20       CONTINUE
   30    CONTINUE
*
      ELSE IF( ITYPE.EQ.1 ) THEN
*
*        Lower triangular matrix
*
         DO 50 J = 1, N
            DO 40 I = J, M
               A( I, J ) = A( I, J )*MUL
   40       CONTINUE
   50    CONTINUE
*
```

```
      ELSE IF( ITYPE.EQ.2 ) THEN
*
*        Upper triangular matrix
*
         DO 70 J = 1, N
            DO 60 I = 1, MIN( J, M )
               A( I, J ) = A( I, J )*MUL
   60       CONTINUE
   70    CONTINUE
*
      ELSE IF( ITYPE.EQ.3 ) THEN
*
*        Upper Hessenberg matrix
*
         DO 90 J = 1, N
            DO 80 I = 1, MIN( J+1, M )
               A( I, J ) = A( I, J )*MUL
   80       CONTINUE
   90    CONTINUE
*
      ELSE IF( ITYPE.EQ.4 ) THEN
*
*        Lower half of a symmetric band matrix
*
         K3 = KL + 1
         K4 = N + 1
         DO 110 J = 1, N
            DO 100 I = 1, MIN( K3, K4-J )
               A( I, J ) = A( I, J )*MUL
  100       CONTINUE
  110    CONTINUE
*
      ELSE IF( ITYPE.EQ.5 ) THEN
*
*        Upper half of a symmetric band matrix
*
         K1 = KU + 2
         K3 = KU + 1
         DO 130 J = 1, N
            DO 120 I = MAX( K1-J, 1 ), K3
               A( I, J ) = A( I, J )*MUL
  120       CONTINUE
  130    CONTINUE
*
      ELSE IF( ITYPE.EQ.6 ) THEN
*
*        Band matrix
*
         K1 = KL + KU + 2
         K2 = KL + 1
```

```
         K3 = 2*KL + KU + 1
         K4 = KL + KU + 1 + M
         DO 150 J = 1, N
            DO 140 I = MAX( K1-J, K2 ), MIN( K3, K4-J )
               A( I, J ) = A( I, J )*MUL
  140       CONTINUE
  150    CONTINUE
*
      END IF
*
      IF( .NOT.DONE )
     $   GO TO 10
*
      RETURN
*
*     End of SLASCL
*
      END
```

———-

90.91.4 lisp code

— slascl.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun slascl (type kl ku cfrom cto m n a lda info)
    (declare (type (array single-float (*)) a)
             (type (single-float) cto cfrom)
             (type (f2cl-lib:integer4) info lda n m ku kl)
             (type (string *) type))
    (f2cl-lib:with-multi-array-data
        ((type character type-%data% type-%offset%)
         (a single-float a-%data% a-%offset%))
      (prog ((bignum 0.0) (cfrom1 0.0) (cfromc 0.0) (cto1 0.0) (ctoc 0.0)
             (mul 0.0) (smlnum 0.0) (i 0) (itype 0) (j 0) (k1 0) (k2 0) (k3 0)
             (k4 0) (done nil))
        (declare
         (type (single-float) bignum cfrom1 cfromc cto1 ctoc mul smlnum)
         (type (f2cl-lib:integer4) i itype j k1 k2 k3 k4)
         (type f2cl-lib:logical done))
        (setf info 0)
        (cond
         ((multiple-value-bind (ret-val var-0 var-1)
              (lsame type "G")
```

```
      (declare (ignore var-1))
      (when var-0 (setf type var-0))
      ret-val)
    (setf itype 0))
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame type "L")
      (declare (ignore var-1))
      (when var-0 (setf type var-0))
      ret-val)
    (setf itype 1))
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame type "U")
      (declare (ignore var-1))
      (when var-0 (setf type var-0))
      ret-val)
    (setf itype 2))
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame type "H")
      (declare (ignore var-1))
      (when var-0 (setf type var-0))
      ret-val)
    (setf itype 3))
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame type "B")
      (declare (ignore var-1))
      (when var-0 (setf type var-0))
      ret-val)
    (setf itype 4))
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame type "Q")
      (declare (ignore var-1))
      (when var-0 (setf type var-0))
      ret-val)
    (setf itype 5))
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame type "Z")
      (declare (ignore var-1))
      (when var-0 (setf type var-0))
      ret-val)
    (setf itype 6))
   (t (setf itype -1)))
  (cond ((= itype (f2cl-lib:int-sub 1)) (setf info -1))
        ((or (= cfrom zero) (sisnan cfrom)) (setf info -4))
        ((sisnan cto) (setf info -5)) ((< m 0) (setf info -6))
        ((or (< n 0) (and (= itype 4) (/= n m))
             (and (= itype 5) (/= n m)))
         (setf info -7))
        ((and (<= itype 3)
              (< lda
                 (max (the f2cl-lib:integer4 1)
```

```
                    (the f2cl-lib:integer4 m))))
        (setf info -9))
       ((>= itype 4)
        (cond
         ((or (< kl 0)
              (> kl
                 (max
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
                  (the f2cl-lib:integer4 0))))
          (setf info -2))
         ((or (< ku 0)
              (> ku
                 (max
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                  (the f2cl-lib:integer4 0)))
              (and (or (= itype 4) (= itype 5)) (/= kl ku)))
          (setf info -3))
         ((or (and (= itype 4) (< lda (f2cl-lib:int-add kl 1)))
              (and (= itype 5) (< lda (f2cl-lib:int-add ku 1)))
              (and (= itype 6)
                   (< lda
                      (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))))
          (setf info -9)))))
 (cond
  ((/= info 0) (xerbla "SLASCL" (f2cl-lib:int-sub info))
   (go end_label)))
 (if (or (= n 0) (= m 0))
     (go end_label))
 (setf smlnum (slamch "S"))
 (setf bignum (/ one smlnum))
 (setf cfromc cfrom)
 (setf ctoc cto)
label10
 (setf cfrom1 (* cfromc smlnum))
 (cond
  ((= cfrom1 cfromc) (setf mul (/ ctoc cfromc))
   (setf done f2cl-lib:%true%) (setf cto1 ctoc))
  (t (setf cto1 (/ ctoc bignum))
   (cond
    ((= cto1 ctoc) (setf mul ctoc) (setf done f2cl-lib:%true%)
     (setf cfromc one))
    ((and (> (abs cfrom1) (abs ctoc)) (/= ctoc zero)) (setf mul smlnum)
     (setf done f2cl-lib:%false%) (setf cfromc cfrom1))
    ((> (abs cto1) (abs cfromc)) (setf mul bignum)
     (setf done f2cl-lib:%false%) (setf ctoc cto1))
    (t (setf mul (/ ctoc cfromc)) (setf done f2cl-lib:%true%)))))
 (cond
  ((= itype 0)
```

```
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i m) nil)
       (tagbody
         (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                              a-%offset%)
                 (*
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%)
                  mul))
        label20))
    label30)))
((= itype 1)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                   ((> i m) nil)
       (tagbody
         (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                              a-%offset%)
                 (*
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%)
                  mul))
        label40))
    label50)))
((= itype 2)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i
                       (min (the f2cl-lib:integer4 j)
                            (the f2cl-lib:integer4 m)))
                    nil)
       (tagbody
         (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                              a-%offset%)
                 (*
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%)
                  mul))
        label60))
    label70)))
((= itype 3)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
```

```
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (min
                         (the f2cl-lib:integer4 (f2cl-lib:int-add j 1))
                         (the f2cl-lib:integer4 m)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                  (*
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                   mul))
         label80))
    label90)))
((= itype 4) (setf k3 (f2cl-lib:int-add kl 1))
 (setf k4 (f2cl-lib:int-add n 1))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (min (the f2cl-lib:integer4 k3)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add k4
                                                    (f2cl-lib:int-sub
                                                     j)))))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                  (*
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                   mul))
         label100))
    label110)))
((= itype 5) (setf k1 (f2cl-lib:int-add ku 2))
 (setf k3 (f2cl-lib:int-add ku 1))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i
                     (max
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add k1 (f2cl-lib:int-sub j)))
                      (the f2cl-lib:integer4 1))
                     (f2cl-lib:int-add i 1))
                    ((> i k3) nil)
```

```
                (tagbody
                  (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%)
                          (*
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%)
                           mul))
                 label120))
             label130)))
         ((= itype 6) (setf k1 (f2cl-lib:int-add kl ku 2))
          (setf k2 (f2cl-lib:int-add kl 1))
          (setf k3 (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
          (setf k4 (f2cl-lib:int-add kl ku 1 m))
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (f2cl-lib:fdo (i
                             (max
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add k1 (f2cl-lib:int-sub j)))
                              (the f2cl-lib:integer4 k2))
                             (f2cl-lib:int-add i 1))
                            ((> i
                                (min (the f2cl-lib:integer4 k3)
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-add k4
                                                            (f2cl-lib:int-sub
                                                             j)))))
                             nil)
                (tagbody
                  (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%)
                          (*
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%)
                           mul))
                 label140))
             label150))))
        (if (not done)
            (go label10))
        (go end_label)
       end_label
        (return (values type nil nil nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slascl
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
```

```
'((string)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (single-float) (single-float)
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4))
:return-values
'(type nil nil nil nil nil nil nil
  nil fortran-to-lisp::info)
:calls '(fortran-to-lisp::sisnan))))
```

———-

90.92 slasd0

90.92.1 Axiom unit tests

— slasd0.input —

```
)set break resume
)sys rm -f slasd0.output
)spool slasd0.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.92.2 Axiom help page

— slasd0.help —

```
==========================================================================
slasd0 examples
==========================================================================
```

```
====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slasd0 (N, SQRE, D, E, U, LDU, VT, LDVT, SMLSIZ, IWORK,
           WORK, INFO)
           SLASD0 computes the singular values of a real upper bidiagonal n-
           by-m matrix B with diagonal d and off-diagonal e. Used by sbdsdc.

Function/Subroutine Documentation
   subroutine slasd0 (integer N, integer SQRE, real, dimension( * ) D, real,
       dimension( * ) E, real, dimension( ldu, * ) U, integer LDU, real,
       dimension( ldvt, * ) VT, integer LDVT, integer SMLSIZ, integer,
       dimension( * ) IWORK, real, dimension( * ) WORK, integer INFO)
       SLASD0 computes the singular values of a real upper bidiagonal n-by-m
       matrix B with diagonal d and off-diagonal e. Used by sbdsdc.

Purpose:
   Using a divide and conquer approach, SLASD0 computes the singular
   value decomposition (SVD) of a real upper bidiagonal N-by-M
   matrix B with diagonal D and offdiagonal E, where M = N + SQRE.
   The algorithm computes orthogonal matrices U and VT such that
   B = U * S * VT. The singular values S are overwritten on D.

   A related subroutine, SLASDA, computes only the singular values,
   and optionally, the singular vectors in compact form.

Parameters:

 N
   N is INTEGER
   On entry, the row dimension of the upper bidiagonal matrix.
   This is also the dimension of the main diagonal array D.

 SQRE
   SQRE is INTEGER
   Specifies the column dimension of the bidiagonal matrix.
   = 0: The bidiagonal matrix has column dimension M = N;
   = 1: The bidiagonal matrix has column dimension M = N+1;

 D
   D is REAL array, dimension (N)
   On entry D contains the main diagonal of the bidiagonal matrix.
   On exit D, if INFO = 0, contains its singular values.

 E
   E is REAL array, dimension (M-1)
   Contains the subdiagonal entries of the bidiagonal matrix.
```

```
  On exit, E has been destroyed.

U
  U is REAL array, dimension at least (LDQ, N)
  On exit, U contains the left singular vectors.

LDU
  LDU is INTEGER
  On entry, leading dimension of U.

VT
  VT is REAL array, dimension at least (LDVT, M)
  On exit, VT**T contains the right singular vectors.

LDVT
  LDVT is INTEGER
  On entry, leading dimension of VT.

SMLSIZ
  SMLSIZ is INTEGER
  On entry, maximum size of the subproblems at the
  bottom of the computation tree.

IWORK
  IWORK is INTEGER array, dimension (8*N)

WORK
  WORK is REAL array, dimension (3*M**2+2*M)

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if INFO = 1, a singular value did not converge
```

———-

90.92.3 fortran code

— **slasd0.f** —

```
*  =====================================================================
      SUBROUTINE SLASD0( N, SQRE, D, E, U, LDU, VT, LDVT, SMLSIZ, IWORK,
     $                   WORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
```

```
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDU, LDVT, N, SMLSIZ, SQRE
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      REAL               D( * ), E( * ), U( LDU, * ), VT( LDVT, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, I1, IC, IDXQ, IDXQC, IM1, INODE, ITEMP, IWK,
     $                   J, LF, LL, LVL, M, NCC, ND, NDB1, NDIML, NDIMR,
     $                   NL, NLF, NLP1, NLVL, NR, NRF, NRP1, SQREI
      REAL               ALPHA, BETA
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLASD1, SLASDQ, SLASDT, XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ( SQRE.LT.0 ) .OR. ( SQRE.GT.1 ) ) THEN
         INFO = -2
      END IF
*
      M = N + SQRE
*
      IF( LDU.LT.N ) THEN
         INFO = -6
      ELSE IF( LDVT.LT.M ) THEN
         INFO = -8
      ELSE IF( SMLSIZ.LT.3 ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SLASD0', -INFO )
         RETURN
      END IF
*
*     If the input matrix is too small, call SLASDQ to find the SVD.
```

```
*
      IF( N.LE.SMLSIZ ) THEN
         CALL SLASDQ( 'U', SQRE, N, M, N, 0, D, E, VT, LDVT, U, LDU, U,
     $                LDU, WORK, INFO )
         RETURN
      END IF
*
*     Set up the computation tree.
*
      INODE = 1
      NDIML = INODE + N
      NDIMR = NDIML + N
      IDXQ = NDIMR + N
      IWK = IDXQ + N
      CALL SLASDT( N, NLVL, ND, IWORK( INODE ), IWORK( NDIML ),
     $             IWORK( NDIMR ), SMLSIZ )
*
*     For the nodes on bottom level of the tree, solve
*     their subproblems by SLASDQ.
*
      NDB1 = ( ND+1 ) / 2
      NCC = 0
      DO 30 I = NDB1, ND
*
*     IC : center row of each node
*     NL : number of rows of left  subproblem
*     NR : number of rows of right subproblem
*     NLF: starting row of the left   subproblem
*     NRF: starting row of the right  subproblem
*
         I1 = I - 1
         IC = IWORK( INODE+I1 )
         NL = IWORK( NDIML+I1 )
         NLP1 = NL + 1
         NR = IWORK( NDIMR+I1 )
         NRP1 = NR + 1
         NLF = IC - NL
         NRF = IC + 1
         SQREI = 1
         CALL SLASDQ( 'U', SQREI, NL, NLP1, NL, NCC, D( NLF ), E( NLF ),
     $                VT( NLF, NLF ), LDVT, U( NLF, NLF ), LDU,
     $                U( NLF, NLF ), LDU, WORK, INFO )
         IF( INFO.NE.0 ) THEN
            RETURN
         END IF
         ITEMP = IDXQ + NLF - 2
         DO 10 J = 1, NL
            IWORK( ITEMP+J ) = J
   10    CONTINUE
         IF( I.EQ.ND ) THEN
```

```
         SQREI = SQRE
      ELSE
         SQREI = 1
      END IF
      NRP1 = NR + SQREI
      CALL SLASDQ( 'U', SQREI, NR, NRP1, NR, NCC, D( NRF ), E( NRF ),
     $               VT( NRF, NRF ), LDVT, U( NRF, NRF ), LDU,
     $               U( NRF, NRF ), LDU, WORK, INFO )
      IF( INFO.NE.0 ) THEN
         RETURN
      END IF
      ITEMP = IDXQ + IC
      DO 20 J = 1, NR
         IWORK( ITEMP+J-1 ) = J
   20 CONTINUE
   30 CONTINUE
*
*     Now conquer each subproblem bottom-up.
*
      DO 50 LVL = NLVL, 1, -1
*
*        Find the first node LF and last node LL on the
*        current level LVL.
*
         IF( LVL.EQ.1 ) THEN
            LF = 1
            LL = 1
         ELSE
            LF = 2**( LVL-1 )
            LL = 2*LF - 1
         END IF
         DO 40 I = LF, LL
            IM1 = I - 1
            IC = IWORK( INODE+IM1 )
            NL = IWORK( NDIML+IM1 )
            NR = IWORK( NDIMR+IM1 )
            NLF = IC - NL
            IF( ( SQRE.EQ.0 ) .AND. ( I.EQ.LL ) ) THEN
               SQREI = SQRE
            ELSE
               SQREI = 1
            END IF
            IDXQC = IDXQ + NLF - 1
            ALPHA = D( IC )
            BETA = E( IC )
            CALL SLASD1( NL, NR, SQREI, D( NLF ), ALPHA, BETA,
     $                   U( NLF, NLF ), LDU, VT( NLF, NLF ), LDVT,
     $                   IWORK( IDXQC ), IWORK( IWK ), WORK, INFO )
*
*     Report the possible convergence failure.
```

```
*
           IF( INFO.NE.0 ) THEN
              RETURN
           END IF
   40    CONTINUE
   50 CONTINUE
*
      RETURN
*
*     End of SLASD0
*
      END
```

———-

90.92.4 lisp code

— **slasd0.lisp** —

```
(defun slasd0 (n sqre d e u ldu vt ldvt smlsiz iwork work info)
  (declare (type (array f2cl-lib:integer4 (*)) iwork)
           (type (array single-float (*)) work vt u e d)
           (type (f2cl-lib:integer4) info smlsiz ldvt ldu sqre n))
  (f2cl-lib:with-multi-array-data
      ((d single-float d-%data% d-%offset%)
       (e single-float e-%data% e-%offset%)
       (u single-float u-%data% u-%offset%)
       (vt single-float vt-%data% vt-%offset%)
       (work single-float work-%data% work-%offset%)
       (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
    (prog ((alpha 0.0) (beta 0.0) (i 0) (i1 0) (ic 0) (idxq 0) (idxqc 0)
           (im1 0) (inode 0) (itemp 0) (iwk 0) (j 0) (lf 0) (ll 0) (lvl 0)
           (m 0) (ncc 0) (nd 0) (ndb1 0) (ndiml 0) (ndimr 0) (nl 0) (nlf 0)
           (nlp1 0) (nlvl 0) (nr 0) (nrf 0) (nrp1 0) (sqrei 0))
      (declare
       (type (f2cl-lib:integer4) sqrei nrp1 nrf nr nlvl nlp1 nlf nl ndimr ndiml
        ndb1 nd ncc m lvl ll lf j iwk itemp inode im1 idxqc idxq ic i1 i)
       (type (single-float) beta alpha))
      (setf info 0)
      (cond ((< n 0) (setf info -1))
            ((or (< sqre 0) (> sqre 1)) (setf info -2)))
      (setf m (f2cl-lib:int-add n sqre))
      (cond ((< ldu n) (setf info -6)) ((< ldvt m) (setf info -8))
            ((< smlsiz 3) (setf info -9)))
      (cond
       ((/= info 0) (xerbla "SLASD0" (f2cl-lib:int-sub info)) (go end_label)))
      (cond
       ((<= n smlsiz)
```

```
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
       var-11 var-12 var-13 var-14 var-15)
      (slasdq "U" sqre n m n 0 d e vt ldvt u ldu u ldu work info)
    (declare (ignore var-0 var-5 var-6 var-7 var-8 var-10 var-12 var-14))
    (when var-1 (setf sqre var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-4 (setf n var-4))
    (when var-9 (setf ldvt var-9))
    (when var-11 (setf ldu var-11))
    (when var-13 (setf ldu var-13))
    (when var-15 (setf info var-15)))
  (go end_label)))
(setf inode 1)
(setf ndiml (f2cl-lib:int-add inode n))
(setf ndimr (f2cl-lib:int-add ndiml n))
(setf idxq (f2cl-lib:int-add ndimr n))
(setf iwk (f2cl-lib:int-add idxq n))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slasdt n nlvl nd
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (inode) ((1 *))
                           iwork-%offset%)
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (ndiml) ((1 *))
                           iwork-%offset%)
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (ndimr) ((1 *))
                           iwork-%offset%)
     smlsiz)
  (declare (ignore var-3 var-4 var-5))
  (when var-0 (setf n var-0))
  (when var-1 (setf nlvl var-1))
  (when var-2 (setf nd var-2))
  (when var-6 (setf smlsiz var-6)))
(setf ndb1 (the f2cl-lib:integer4 (truncate (+ nd 1) 2)))
(setf ncc 0)
(f2cl-lib:fdo (i ndb1 (f2cl-lib:int-add i 1))
              ((> i nd) nil)
  (tagbody
    (setf i1 (f2cl-lib:int-sub i 1))
    (setf ic
            (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add inode i1))
                           ((1 *)) iwork-%offset%))
    (setf nl
            (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add ndiml i1))
                           ((1 *)) iwork-%offset%))
    (setf nlp1 (f2cl-lib:int-add nl 1))
    (setf nr
            (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add ndimr i1))
                           ((1 *)) iwork-%offset%))
    (setf nrp1 (f2cl-lib:int-add nr 1))
```

```
(setf nlf (f2cl-lib:int-sub ic nl))
(setf nrf (f2cl-lib:int-add ic 1))
(setf sqrei 1)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15)
    (slasdq "U" sqrei nl nlp1 nl ncc
     (f2cl-lib:array-slice d-%data% single-float (nlf) ((1 *))
                           d-%offset%)
     (f2cl-lib:array-slice e-%data% single-float (nlf) ((1 *))
                           e-%offset%)
     (f2cl-lib:array-slice vt-%data% single-float (nlf nlf)
                           ((1 ldvt) (1 *)) vt-%offset%)
     ldvt
     (f2cl-lib:array-slice u-%data% single-float (nlf nlf)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu
     (f2cl-lib:array-slice u-%data% single-float (nlf nlf)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu work info)
  (declare (ignore var-0 var-6 var-7 var-8 var-10 var-12 var-14))
  (when var-1 (setf sqrei var-1))
  (when var-2 (setf nl var-2))
  (when var-3 (setf nlp1 var-3))
  (when var-4 (setf nl var-4))
  (when var-5 (setf ncc var-5))
  (when var-9 (setf ldvt var-9))
  (when var-11 (setf ldu var-11))
  (when var-13 (setf ldu var-13))
  (when var-15 (setf info var-15)))
(cond ((/= info 0) (go end_label)))
(setf itemp (f2cl-lib:int-sub (f2cl-lib:int-add idxq nlf) 2))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nl) nil)
  (tagbody
    (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add itemp j))
                         ((1 *)) iwork-%offset%)
            j)
   label10))
(cond ((= i nd) (setf sqrei sqre)) (t (setf sqrei 1)))
(setf nrp1 (f2cl-lib:int-add nr sqrei))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15)
    (slasdq "U" sqrei nr nrp1 nr ncc
     (f2cl-lib:array-slice d-%data% single-float (nrf) ((1 *))
                           d-%offset%)
     (f2cl-lib:array-slice e-%data% single-float (nrf) ((1 *))
                           e-%offset%)
     (f2cl-lib:array-slice vt-%data% single-float (nrf nrf)
```

```
                                ((1 ldvt) (1 *)) vt-%offset%)
            ldvt
            (f2cl-lib:array-slice u-%data% single-float (nrf nrf)
                                ((1 ldu) (1 *)) u-%offset%)
            ldu
            (f2cl-lib:array-slice u-%data% single-float (nrf nrf)
                                ((1 ldu) (1 *)) u-%offset%)
            ldu work info)
        (declare (ignore var-0 var-6 var-7 var-8 var-10 var-12 var-14))
        (when var-1 (setf sqrei var-1))
        (when var-2 (setf nr var-2))
        (when var-3 (setf nrp1 var-3))
        (when var-4 (setf nr var-4))
        (when var-5 (setf ncc var-5))
        (when var-9 (setf ldvt var-9))
        (when var-11 (setf ldu var-11))
        (when var-13 (setf ldu var-13))
        (when var-15 (setf info var-15)))
      (cond ((/= info 0) (go end_label)))
      (setf itemp (f2cl-lib:int-add idxq ic))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j nr) nil)
        (tagbody
          (setf (f2cl-lib:fref iwork-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add itemp j) 1))
                               ((1 *)) iwork-%offset%)
                j)
         label20))
    label30))
(f2cl-lib:fdo (lvl nlvl (f2cl-lib:int-add lvl (f2cl-lib:int-sub 1)))
              ((> lvl 1) nil)
  (tagbody
    (cond ((= lvl 1) (setf lf 1) (setf ll 1))
          (t (setf lf (expt 2 (f2cl-lib:int-sub lvl 1)))
           (setf ll (f2cl-lib:int-sub (f2cl-lib:int-mul 2 lf) 1))))
    (f2cl-lib:fdo (i lf (f2cl-lib:int-add i 1))
                  ((> i ll) nil)
      (tagbody
        (setf im1 (f2cl-lib:int-sub i 1))
        (setf ic
                (f2cl-lib:fref iwork-%data%
                               ((f2cl-lib:int-add inode im1)) ((1 *))
                               iwork-%offset%))
        (setf nl
                (f2cl-lib:fref iwork-%data%
                               ((f2cl-lib:int-add ndiml im1)) ((1 *))
                               iwork-%offset%))
        (setf nr
                (f2cl-lib:fref iwork-%data%
```

```
                                  ((f2cl-lib:int-add ndimr im1)) ((1 *))
                                  iwork-%offset%))
                (setf nlf (f2cl-lib:int-sub ic nl))
                (cond ((and (= sqre 0) (= i ll)) (setf sqrei sqre))
                      (t (setf sqrei 1)))
                (setf idxqc (f2cl-lib:int-sub (f2cl-lib:int-add idxq nlf) 1))
                (setf alpha (f2cl-lib:fref d-%data% (ic) ((1 *)) d-%offset%))
                (setf beta (f2cl-lib:fref e-%data% (ic) ((1 *)) e-%offset%))
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                     var-10 var-11 var-12 var-13)
                    (slasd1 nl nr sqrei
                     (f2cl-lib:array-slice d-%data% single-float (nlf) ((1 *))
                                           d-%offset%)
                     alpha beta
                     (f2cl-lib:array-slice u-%data% single-float (nlf nlf)
                                           ((1 ldu) (1 *)) u-%offset%)
                     ldu
                     (f2cl-lib:array-slice vt-%data% single-float (nlf nlf)
                                           ((1 ldvt) (1 *)) vt-%offset%)
                     ldvt
                     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (idxqc)
                                           ((1 *)) iwork-%offset%)
                     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (iwk)
                                           ((1 *)) iwork-%offset%)
                     work info)
                  (declare (ignore var-3 var-6 var-8 var-10 var-11 var-12))
                  (when var-0 (setf nl var-0))
                  (when var-1 (setf nr var-1))
                  (when var-2 (setf sqrei var-2))
                  (when var-4 (setf alpha var-4))
                  (when var-5 (setf beta var-5))
                  (when var-7 (setf ldu var-7))
                  (when var-9 (setf ldvt var-9))
                  (when var-13 (setf info var-13)))
                (cond ((/= info 0) (go end_label)))
               label40))
           label50))
       (go end_label)
      end_label
       (return (values n sqre nil nil nil ldu nil ldvt smlsiz nil nil info)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slasd0
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
```

```
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (array single-float (*))
  (fortran-to-lisp::integer4)
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::n
  fortran-to-lisp::sqre nil nil nil
  fortran-to-lisp::ldu nil
  fortran-to-lisp::ldvt
  fortran-to-lisp::smlsiz nil nil
  fortran-to-lisp::info)
:calls 'nil)))
```

———-

90.93 slasd1

90.93.1 Axiom unit tests

— slasd1.input —

```
)set break resume
)sys rm -f slasd1.output
)spool slasd1.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.93.2 Axiom help page

— slasd1.help —

```
====================================================================
slasd1 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slasd1 (NL, NR, SQRE, D, ALPHA, BETA, U, LDU, VT, LDVT,
           IDXQ, IWORK, WORK, INFO)
           SLASD1 computes the SVD of an upper bidiagonal matrix B of the
           specified size. Used by sbdsdc.

Function/Subroutine Documentation
   subroutine slasd1 (integer NL, integer NR, integer SQRE, real, dimension( *
       ) D, real ALPHA, real BETA, real, dimension( ldu, * ) U, integer LDU,
       real, dimension( ldvt, * ) VT, integer LDVT, integer, dimension( * )
       IDXQ, integer, dimension( * ) IWORK, real, dimension( * ) WORK, integer
       INFO)
       SLASD1 computes the SVD of an upper bidiagonal matrix B of the
       specified size. Used by sbdsdc.

Purpose:
   SLASD1 computes the SVD of an upper bidiagonal N-by-M matrix B,
   where N = NL + NR + 1 and M = N + SQRE. SLASD1 is called from SLASD0.

   A related subroutine SLASD7 handles the case in which the singular
   values (and the singular vectors in factored form) are desired.

   SLASD1 computes the SVD as follows:

                         ( D1(in)    0    0       0 )
             B = U(in) * (   Z1**T   a   Z2**T    b ) * VT(in)
                         (   0       0   D2(in)   0 )

               = U(out) * ( D(out) 0) * VT(out)

   where Z**T = (Z1**T a Z2**T b) = u**T VT**T, and u is a vector of
   dimension M with ALPHA and BETA in the NL+1 and NL+2 th
   entries and zeros elsewhere; and the entry b is empty if SQRE = 0.

   The left singular vectors of the original matrix are stored in U, and
   the transpose of the right singular vectors are stored in VT, and the
   singular values are in D.  The algorithm consists of three stages:

   The first stage consists of deflating the size of the problem
   when there are multiple singular values or when there are zeros in
   the Z vector.  For each such occurence the dimension of the
```

```
  secular equation problem is reduced by one.  This stage is
  performed by the routine SLASD2.

  The second stage consists of calculating the updated
  singular values. This is done by finding the square roots of the
  roots of the secular equation via the routine SLASD4 (as called
  by SLASD3). This routine also calculates the singular vectors of
  the current problem.

  The final stage consists of computing the updated singular vectors
  directly using the updated singular values.  The singular vectors
  for the current problem are multiplied with the singular vectors
  from the overall problem.

Parameters:

 NL
   NL is INTEGER
   The row dimension of the upper block.  NL >= 1.

 NR
   NR is INTEGER
   The row dimension of the lower block.  NR >= 1.

 SQRE
   SQRE is INTEGER
   = 0: the lower block is an NR-by-NR square matrix.
   = 1: the lower block is an NR-by-(NR+1) rectangular matrix.

   The bidiagonal matrix has row dimension N = NL + NR + 1,
   and column dimension M = N + SQRE.

 D
   D is REAL array, dimension (NL+NR+1).
   N = NL+NR+1
   On entry D(1:NL,1:NL) contains the singular values of the
   upper block; and D(NL+2:N) contains the singular values of
   the lower block. On exit D(1:N) contains the singular values
   of the modified matrix.

 ALPHA
   ALPHA is REAL
   Contains the diagonal element associated with the added row.

 BETA
   BETA is REAL
   Contains the off-diagonal element associated with the added row.

 U
   U is REAL array, dimension (LDU,N)
```

```
  On entry U(1:NL, 1:NL) contains the left singular vectors of
  the upper block; U(NL+2:N, NL+2:N) contains the left singular
  vectors of the lower block. On exit U contains the left
  singular vectors of the bidiagonal matrix.

LDU
  LDU is INTEGER
  The leading dimension of the array U.  LDU >= max( 1, N ).

VT
  VT is REAL array, dimension (LDVT,M)
  where M = N + SQRE.
  On entry VT(1:NL+1, 1:NL+1)**T contains the right singular
  vectors of the upper block; VT(NL+2:M, NL+2:M)**T contains
  the right singular vectors of the lower block. On exit
  VT**T contains the right singular vectors of the bidiagonal matrix.

LDVT
  LDVT is INTEGER
  The leading dimension of the array VT.  LDVT >= max( 1, M ).

IDXQ
  IDXQ is INTEGER array, dimension (N)
  This contains the permutation which will reintegrate the
  subproblem just solved back into sorted order, i.e.
  D( IDXQ( I = 1, N ) ) will be in ascending order.

IWORK
  IWORK is INTEGER array, dimension (4*N)

WORK
  WORK is REAL array, dimension (3*M**2+2*M)

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if INFO = 1, a singular value did not converge
```

———-

90.93.3 fortran code

— slasd1.f —

```
*  =====================================================================
      SUBROUTINE SLASD1( NL, NR, SQRE, D, ALPHA, BETA, U, LDU, VT, LDVT,
```

```
     $                   IDXQ, IWORK, WORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDU, LDVT, NL, NR, SQRE
      REAL               ALPHA, BETA
*     ..
*     .. Array Arguments ..
      INTEGER            IDXQ( * ), IWORK( * )
      REAL               D( * ), U( LDU, * ), VT( LDVT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
*
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            COLTYP, I, IDX, IDXC, IDXP, IQ, ISIGMA, IU2,
     $                   IVT2, IZ, K, LDQ, LDU2, LDVT2, M, N, N1, N2
      REAL               ORGNRM
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLAMRG, SLASCL, SLASD2, SLASD3, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( NL.LT.1 ) THEN
         INFO = -1
      ELSE IF( NR.LT.1 ) THEN
         INFO = -2
      ELSE IF( ( SQRE.LT.0 ) .OR. ( SQRE.GT.1 ) ) THEN
         INFO = -3
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SLASD1', -INFO )
         RETURN
```

```
      END IF
*
      N = NL + NR + 1
      M = N + SQRE
*
*     The following values are for bookkeeping purposes only.  They are
*     integer pointers which indicate the portion of the workspace
*     used by a particular array in SLASD2 and SLASD3.
*
      LDU2 = N
      LDVT2 = M
*
      IZ = 1
      ISIGMA = IZ + M
      IU2 = ISIGMA + N
      IVT2 = IU2 + LDU2*N
      IQ = IVT2 + LDVT2*M
*
      IDX = 1
      IDXC = IDX + N
      COLTYP = IDXC + N
      IDXP = COLTYP + N
*
*     Scale.
*
      ORGNRM = MAX( ABS( ALPHA ), ABS( BETA ) )
      D( NL+1 ) = ZERO
      DO 10 I = 1, N
         IF( ABS( D( I ) ).GT.ORGNRM ) THEN
            ORGNRM = ABS( D( I ) )
         END IF
   10 CONTINUE
      CALL SLASCL( 'G', 0, 0, ORGNRM, ONE, N, 1, D, N, INFO )
      ALPHA = ALPHA / ORGNRM
      BETA = BETA / ORGNRM
*
*     Deflate singular values.
*
      CALL SLASD2( NL, NR, SQRE, K, D, WORK( IZ ), ALPHA, BETA, U, LDU,
     $             VT, LDVT, WORK( ISIGMA ), WORK( IU2 ), LDU2,
     $             WORK( IVT2 ), LDVT2, IWORK( IDXP ), IWORK( IDX ),
     $             IWORK( IDXC ), IDXQ, IWORK( COLTYP ), INFO )
*
*     Solve Secular Equation and update singular vectors.
*
      LDQ = K
      CALL SLASD3( NL, NR, SQRE, K, D, WORK( IQ ), LDQ, WORK( ISIGMA ),
     $             U, LDU, WORK( IU2 ), LDU2, VT, LDVT, WORK( IVT2 ),
     $             LDVT2, IWORK( IDXC ), IWORK( COLTYP ), WORK( IZ ),
     $             INFO )
```

```
*
*     Report the possible convergence failure.
*
      IF( INFO.NE.0 ) THEN
         RETURN
      END IF
*
*     Unscale.
*
      CALL SLASCL( 'G', 0, 0, ONE, ORGNRM, N, 1, D, N, INFO )
*
*     Prepare the IDXQ sorting permutation.
*
      N1 = K
      N2 = N - K
      CALL SLAMRG( N1, N2, D, 1, -1, IDXQ )
*
      RETURN
*
*     End of SLASD1
*
      END
```

90.93.4 lisp code

— slasd1.lisp —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun slasd1 (nl nr sqre d alpha beta u ldu vt ldvt idxq iwork work info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork idxq)
             (type (single-float) beta alpha)
             (type (array single-float (*)) work vt u d)
             (type (f2cl-lib:integer4) info ldvt ldu sqre nr nl))
    (f2cl-lib:with-multi-array-data
        ((d single-float d-%data% d-%offset%)
         (u single-float u-%data% u-%offset%)
         (vt single-float vt-%data% vt-%offset%)
         (work single-float work-%data% work-%offset%)
         (idxq f2cl-lib:integer4 idxq-%data% idxq-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((orgnrm 0.0) (coltyp 0) (i 0) (idx 0) (idxc 0) (idxp 0) (iq 0)
             (isigma 0) (iu2 0) (ivt2 0) (iz 0) (k 0) (ldq 0) (ldu2 0)
             (ldvt2 0) (m 0) (n 0) (n1 0) (n2 0))
```

```
(declare (type (single-float) orgnrm)
         (type (f2cl-lib:integer4) coltyp i idx idxc idxp iq isigma iu2
          ivt2 iz k ldq ldu2 ldvt2 m n n1 n2))
(setf info 0)
(cond ((< nl 1) (setf info -1)) ((< nr 1) (setf info -2))
      ((or (< sqre 0) (> sqre 1)) (setf info -3)))
(cond
 ((/= info 0) (xerbla "SLASD1" (f2cl-lib:int-sub info))
  (go end_label)))
(setf n (f2cl-lib:int-add nl nr 1))
(setf m (f2cl-lib:int-add n sqre))
(setf ldu2 n)
(setf ldvt2 m)
(setf iz 1)
(setf isigma (f2cl-lib:int-add iz m))
(setf iu2 (f2cl-lib:int-add isigma n))
(setf ivt2 (f2cl-lib:int-add iu2 (f2cl-lib:int-mul ldu2 n)))
(setf iq (f2cl-lib:int-add ivt2 (f2cl-lib:int-mul ldvt2 m)))
(setf idx 1)
(setf idxc (f2cl-lib:int-add idx n))
(setf coltyp (f2cl-lib:int-add idxc n))
(setf idxp (f2cl-lib:int-add coltyp n))
(setf orgnrm (max (abs alpha) (abs beta)))
(setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add nl 1)) ((1 *))
                     d-%offset%)
        zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (abs (f2cl-lib:fref d (i) ((1 *)))) orgnrm)
      (setf orgnrm
              (abs (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))))
   label10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (slascl "G" 0 0 orgnrm one n 1 d n info)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8))
  (setf info var-9))
(setf alpha (/ alpha orgnrm))
(setf beta (/ beta orgnrm))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19
     var-20 var-21 var-22)
    (slasd2 nl nr sqre k d
     (f2cl-lib:array-slice work-%data% single-float (iz) ((1 *))
                           work-%offset%)
     alpha beta u ldu vt ldvt
```

```
      (f2cl-lib:array-slice work-%data% single-float (isigma) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% single-float (iu2) ((1 *))
                            work-%offset%)
      ldu2
      (f2cl-lib:array-slice work-%data% single-float (ivt2) ((1 *))
                            work-%offset%)
      ldvt2
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (idxp)
                            ((1 *)) iwork-%offset%)
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (idx) ((1 *))
                            iwork-%offset%)
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (idxc)
                            ((1 *)) iwork-%offset%)
      idxq
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (coltyp)
                            ((1 *)) iwork-%offset%)
      info)
    (declare
     (ignore var-4 var-5 var-8 var-10 var-12 var-13 var-15 var-17 var-18
      var-19 var-20 var-21))
    (when var-0 (setf nl var-0))
    (when var-1 (setf nr var-1))
    (when var-2 (setf sqre var-2))
    (when var-3 (setf k var-3))
    (when var-6 (setf alpha var-6))
    (when var-7 (setf beta var-7))
    (when var-9 (setf ldu var-9))
    (when var-11 (setf ldvt var-11))
    (when var-14 (setf ldu2 var-14))
    (when var-16 (setf ldvt2 var-16))
    (when var-22 (setf info var-22)))
  (setf ldq k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
       var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19)
      (slasd3 nl nr sqre k d
       (f2cl-lib:array-slice work-%data% single-float (iq) ((1 *))
                             work-%offset%)
       ldq
       (f2cl-lib:array-slice work-%data% single-float (isigma) ((1 *))
                             work-%offset%)
       u ldu
       (f2cl-lib:array-slice work-%data% single-float (iu2) ((1 *))
                             work-%offset%)
       ldu2 vt ldvt
       (f2cl-lib:array-slice work-%data% single-float (ivt2) ((1 *))
                             work-%offset%)
       ldvt2
       (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (idxc)
```

```
                                  ((1 *)) iwork-%offset%)
              (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (coltyp)
                                  ((1 *)) iwork-%offset%)
              (f2cl-lib:array-slice work-%data% single-float (iz) ((1 *))
                                  work-%offset%)
              info)
            (declare
             (ignore var-4 var-5 var-7 var-8 var-10 var-12 var-14 var-16 var-17
              var-18))
            (when var-0 (setf nl var-0))
            (when var-1 (setf nr var-1))
            (when var-2 (setf sqre var-2))
            (when var-3 (setf k var-3))
            (when var-6 (setf ldq var-6))
            (when var-9 (setf ldu var-9))
            (when var-11 (setf ldu2 var-11))
            (when var-13 (setf ldvt var-13))
            (when var-15 (setf ldvt2 var-15))
            (when var-19 (setf info var-19)))
          (cond ((/= info 0) (go end_label)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (slascl "G" 0 0 one orgnrm n 1 d n info)
            (declare
             (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8))
            (setf info var-9))
          (setf n1 k)
          (setf n2 (f2cl-lib:int-sub n k))
          (slamrg n1 n2 d 1 -1 idxq)
          (go end_label)
         end_label
          (return
           (values nl nr sqre nil alpha beta nil ldu nil ldvt nil nil nil
                   info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slasd1
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
```

```
  (array fortran-to-lisp::integer4
   (*))
  (array fortran-to-lisp::integer4
   (*))
  (array single-float (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::nl
  fortran-to-lisp::nr
  fortran-to-lisp::sqre nil
  fortran-to-lisp::alpha
  fortran-to-lisp::beta nil
  fortran-to-lisp::ldu nil
  fortran-to-lisp::ldvt nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::slamrg
  fortran-to-lisp::slascl))))
```

———-

90.94 slasd2

90.94.1 Axiom unit tests

— slasd2.input —

```
)set break resume
)sys rm -f slasd2.output
)spool slasd2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.94.2 Axiom help page

— slasd2.help —

```
====================================================================
slasd2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slasd2 (NL, NR, SQRE, K, D, Z, ALPHA, BETA, U, LDU, VT,
           LDVT, DSIGMA, U2, LDU2, VT2, LDVT2, IDXP, IDX, IDXC, IDXQ, COLTYP,
           INFO)
           SLASD2 merges the two sets of singular values together into a
           single sorted set. Used by sbdsdc.

Function/Subroutine Documentation
   subroutine slasd2 (integer NL, integer NR, integer SQRE, integer K, real,
       dimension( * ) D, real, dimension( * ) Z, real ALPHA, real BETA, real,
       dimension( ldu, * ) U, integer LDU, real, dimension( ldvt, * ) VT,
       integer LDVT, real, dimension( * ) DSIGMA, real, dimension( ldu2, * )
       U2, integer LDU2, real, dimension( ldvt2, * ) VT2, integer LDVT2,
       integer, dimension( * ) IDXP, integer, dimension( * ) IDX, integer,
       dimension( * ) IDXC, integer, dimension( * ) IDXQ, integer, dimension(
       * ) COLTYP, integer INFO)
       SLASD2 merges the two sets of singular values together into a single
       sorted set. Used by sbdsdc.

Purpose:
   SLASD2 merges the two sets of singular values together into a single
   sorted set.  Then it tries to deflate the size of the problem.
   There are two ways in which deflation can occur:  when two or more
   singular values are close together or if there is a tiny entry in the
   Z vector.  For each such occurrence the order of the related secular
   equation problem is reduced by one.

   SLASD2 is called from SLASD1.

Parameters:

 NL
   NL is INTEGER
   The row dimension of the upper block.  NL >= 1.

 NR
   NR is INTEGER
   The row dimension of the lower block.  NR >= 1.

 SQRE
   SQRE is INTEGER
```

```
  = 0: the lower block is an NR-by-NR square matrix.
  = 1: the lower block is an NR-by-(NR+1) rectangular matrix.

  The bidiagonal matrix has N = NL + NR + 1 rows and
  M = N + SQRE >= N columns.

K
  K is INTEGER
  Contains the dimension of the non-deflated matrix,
  This is the order of the related secular equation. 1 <= K <=N.

D
  D is REAL array, dimension (N)
  On entry D contains the singular values of the two submatrices
  to be combined.  On exit D contains the trailing (N-K) updated
  singular values (those which were deflated) sorted into increasing order.

Z
  Z is REAL array, dimension (N)
  On exit Z contains the updating row vector in the secular equation.

ALPHA
  ALPHA is REAL
  Contains the diagonal element associated with the added row.

BETA
  BETA is REAL
  Contains the off-diagonal element associated with the added row.

U
  U is REAL array, dimension (LDU,N)
  On entry U contains the left singular vectors of two
  submatrices in the two square blocks with corners at (1,1),
  (NL, NL), and (NL+2, NL+2), (N,N).
  On exit U contains the trailing (N-K) updated left singular
  vectors (those which were deflated) in its last N-K columns.

LDU
  LDU is INTEGER
  The leading dimension of the array U.  LDU >= N.

VT
  VT is REAL array, dimension (LDVT,M)
  On entry VT**T contains the right singular vectors of two
  submatrices in the two square blocks with corners at (1,1),
  (NL+1, NL+1), and (NL+2, NL+2), (M,M).
  On exit VT**T contains the trailing (N-K) updated right singular
  vectors (those which were deflated) in its last N-K columns.
  In case SQRE =1, the last row of VT spans the right null space.
```

```
LDVT
  LDVT is INTEGER
  The leading dimension of the array VT.  LDVT >= M.

DSIGMA
  DSIGMA is REAL array, dimension (N)
  Contains a copy of the diagonal elements (K-1 singular values
  and one zero) in the secular equation.

U2
  U2 is REAL array, dimension (LDU2,N)
  Contains a copy of the first K-1 left singular vectors which
  will be used by SLASD3 in a matrix multiply (SGEMM) to solve
  for the new left singular vectors. U2 is arranged into four
  blocks. The first block contains a column with 1 at NL+1 and
  zero everywhere else; the second block contains non-zero
  entries only at and above NL; the third contains non-zero
  entries only below NL+1; and the fourth is dense.

LDU2
  LDU2 is INTEGER
  The leading dimension of the array U2.  LDU2 >= N.

VT2
  VT2 is REAL array, dimension (LDVT2,N)
  VT2**T contains a copy of the first K right singular vectors
  which will be used by SLASD3 in a matrix multiply (SGEMM) to
  solve for the new right singular vectors. VT2 is arranged into
  three blocks. The first block contains a row that corresponds
  to the special 0 diagonal element in SIGMA; the second block
  contains non-zeros only at and before NL +1; the third block
  contains non-zeros only at and after  NL +2.

LDVT2
  LDVT2 is INTEGER
  The leading dimension of the array VT2.  LDVT2 >= M.

IDXP
  IDXP is INTEGER array, dimension (N)
  This will contain the permutation used to place deflated
  values of D at the end of the array. On output IDXP(2:K)
  points to the nondeflated D-values and IDXP(K+1:N)
  points to the deflated singular values.

IDX
  IDX is INTEGER array, dimension (N)
  This will contain the permutation used to sort the contents of
  D into ascending order.

IDXC
```

```
  IDXC is INTEGER array, dimension (N)
  This will contain the permutation used to arrange the columns
  of the deflated U matrix into three groups:  the first group
  contains non-zero entries only at and above NL, the second
  contains non-zero entries only below NL+2, and the third is dense.

IDXQ
  IDXQ is INTEGER array, dimension (N)
  This contains the permutation which separately sorts the two
  sub-problems in D into ascending order.  Note that entries in
  the first hlaf of this permutation must first be moved one
  position backward; and entries in the second half
  must first have NL+1 added to their values.

COLTYP
  COLTYP is INTEGER array, dimension (N)
  As workspace, this will contain a label which will indicate
  which of the following types a column in the U2 matrix or a
  row in the VT2 matrix is:
  1 : non-zero in the upper half only
  2 : non-zero in the lower half only
  3 : dense
  4 : deflated

  On exit, it is an array of dimension 4, with COLTYP(I) being
  the dimension of the I-th type columns.

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

———-

90.94.3 fortran code

— slasd2.f —

```
*  =====================================================================
      SUBROUTINE SLASD2( NL, NR, SQRE, K, D, Z, ALPHA, BETA, U, LDU, VT,
     $                   LDVT, DSIGMA, U2, LDU2, VT2, LDVT2, IDXP, IDX,
     $                   IDXC, IDXQ, COLTYP, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
```

```
*
*     .. Scalar Arguments ..
      INTEGER            INFO, K, LDU, LDU2, LDVT, LDVT2, NL, NR, SQRE
      REAL               ALPHA, BETA
*     ..
*     .. Array Arguments ..
      INTEGER            COLTYP( * ), IDX( * ), IDXC( * ), IDXP( * ),
     $                   IDXQ( * )
      REAL               D( * ), DSIGMA( * ), U( LDU, * ),
     $                   U2( LDU2, * ), VT( LDVT, * ), VT2( LDVT2, * ),
     $                   Z( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE, TWO, EIGHT
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0, TWO = 2.0E+0,
     $                   EIGHT = 8.0E+0 )
*     ..
*     .. Local Arrays ..
      INTEGER            CTOT( 4 ), PSM( 4 )
*     ..
*     .. Local Scalars ..
      INTEGER            CT, I, IDXI, IDXJ, IDXJP, J, JP, JPREV, K2, M,
     $                   N, NLP1, NLP2
      REAL               C, EPS, HLFTOL, S, TAU, TOL, Z1
*     ..
*     .. External Functions ..
      REAL               SLAMCH, SLAPY2
      EXTERNAL           SLAMCH, SLAPY2
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SLACPY, SLAMRG, SLASET, SROT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( NL.LT.1 ) THEN
         INFO = -1
      ELSE IF( NR.LT.1 ) THEN
         INFO = -2
      ELSE IF( ( SQRE.NE.1 ) .AND. ( SQRE.NE.0 ) ) THEN
         INFO = -3
```

```
      END IF
*
      N = NL + NR + 1
      M = N + SQRE
*
      IF( LDU.LT.N ) THEN
         INFO = -10
      ELSE IF( LDVT.LT.M ) THEN
         INFO = -12
      ELSE IF( LDU2.LT.N ) THEN
         INFO = -15
      ELSE IF( LDVT2.LT.M ) THEN
         INFO = -17
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SLASD2', -INFO )
         RETURN
      END IF
*
      NLP1 = NL + 1
      NLP2 = NL + 2
*
*     Generate the first part of the vector Z; and move the singular
*     values in the first part of D one position backward.
*
      Z1 = ALPHA*VT( NLP1, NLP1 )
      Z( 1 ) = Z1
      DO 10 I = NL, 1, -1
         Z( I+1 ) = ALPHA*VT( I, NLP1 )
         D( I+1 ) = D( I )
         IDXQ( I+1 ) = IDXQ( I ) + 1
   10 CONTINUE
*
*     Generate the second part of the vector Z.
*
      DO 20 I = NLP2, M
         Z( I ) = BETA*VT( I, NLP2 )
   20 CONTINUE
*
*     Initialize some reference arrays.
*
      DO 30 I = 2, NLP1
         COLTYP( I ) = 1
   30 CONTINUE
      DO 40 I = NLP2, N
         COLTYP( I ) = 2
   40 CONTINUE
*
*     Sort the singular values into increasing order
*
```

```
      DO 50 I = NLP2, N
         IDXQ( I ) = IDXQ( I ) + NLP1
   50 CONTINUE
*
*     DSIGMA, IDXC, IDXC, and the first column of U2
*     are used as storage space.
*
      DO 60 I = 2, N
         DSIGMA( I ) = D( IDXQ( I ) )
         U2( I, 1 ) = Z( IDXQ( I ) )
         IDXC( I ) = COLTYP( IDXQ( I ) )
   60 CONTINUE
*
      CALL SLAMRG( NL, NR, DSIGMA( 2 ), 1, 1, IDX( 2 ) )
*
      DO 70 I = 2, N
         IDXI = 1 + IDX( I )
         D( I ) = DSIGMA( IDXI )
         Z( I ) = U2( IDXI, 1 )
         COLTYP( I ) = IDXC( IDXI )
   70 CONTINUE
*
*     Calculate the allowable deflation tolerance
*
      EPS = SLAMCH( 'Epsilon' )
      TOL = MAX( ABS( ALPHA ), ABS( BETA ) )
      TOL = EIGHT*EPS*MAX( ABS( D( N ) ), TOL )
*
*     There are 2 kinds of deflation -- first a value in the z-vector
*     is small, second two (or more) singular values are very close
*     together (their difference is small).
*
*     If the value in the z-vector is small, we simply permute the
*     array so that the corresponding singular value is moved to the
*     end.
*
*     If two values in the D-vector are close, we perform a two-sided
*     rotation designed to make one of the corresponding z-vector
*     entries zero, and then permute the array so that the deflated
*     singular value is moved to the end.
*
*     If there are multiple singular values then the problem deflates.
*     Here the number of equal singular values are found.  As each equal
*     singular value is found, an elementary reflector is computed to
*     rotate the corresponding singular subspace so that the
*     corresponding components of Z are zero in this new basis.
*
      K = 1
      K2 = N + 1
      DO 80 J = 2, N
```

```
         IF( ABS( Z( J ) ).LE.TOL ) THEN
*
*           Deflate due to small z component.
*
            K2 = K2 - 1
            IDXP( K2 ) = J
            COLTYP( J ) = 4
            IF( J.EQ.N )
     $         GO TO 120
         ELSE
            JPREV = J
            GO TO 90
         END IF
   80 CONTINUE
   90 CONTINUE
      J = JPREV
  100 CONTINUE
      J = J + 1
      IF( J.GT.N )
     $   GO TO 110
      IF( ABS( Z( J ) ).LE.TOL ) THEN
*
*        Deflate due to small z component.
*
         K2 = K2 - 1
         IDXP( K2 ) = J
         COLTYP( J ) = 4
      ELSE
*
*        Check if singular values are close enough to allow deflation.
*
         IF( ABS( D( J )-D( JPREV ) ).LE.TOL ) THEN
*
*           Deflation is possible.
*
            S = Z( JPREV )
            C = Z( J )
*
*           Find sqrt(a**2+b**2) without overflow or
*           destructive underflow.
*
            TAU = SLAPY2( C, S )
            C = C / TAU
            S = -S / TAU
            Z( J ) = TAU
            Z( JPREV ) = ZERO
*
*           Apply back the Givens rotation to the left and right
*           singular vector matrices.
*
```

```
            IDXJP = IDXQ( IDX( JPREV )+1 )
            IDXJ = IDXQ( IDX( J )+1 )
            IF( IDXJP.LE.NLP1 ) THEN
               IDXJP = IDXJP - 1
            END IF
            IF( IDXJ.LE.NLP1 ) THEN
               IDXJ = IDXJ - 1
            END IF
            CALL SROT( N, U( 1, IDXJP ), 1, U( 1, IDXJ ), 1, C, S )
            CALL SROT( M, VT( IDXJP, 1 ), LDVT, VT( IDXJ, 1 ), LDVT, C,
     $                 S )
            IF( COLTYP( J ).NE.COLTYP( JPREV ) ) THEN
               COLTYP( J ) = 3
            END IF
            COLTYP( JPREV ) = 4
            K2 = K2 - 1
            IDXP( K2 ) = JPREV
            JPREV = J
         ELSE
            K = K + 1
            U2( K, 1 ) = Z( JPREV )
            DSIGMA( K ) = D( JPREV )
            IDXP( K ) = JPREV
            JPREV = J
         END IF
      END IF
      GO TO 100
  110 CONTINUE
*
*     Record the last singular value.
*
      K = K + 1
      U2( K, 1 ) = Z( JPREV )
      DSIGMA( K ) = D( JPREV )
      IDXP( K ) = JPREV
*
  120 CONTINUE
*
*     Count up the total number of the various types of columns, then
*     form a permutation which positions the four column types into
*     four groups of uniform structure (although one or more of these
*     groups may be empty).
*
      DO 130 J = 1, 4
         CTOT( J ) = 0
  130 CONTINUE
      DO 140 J = 2, N
         CT = COLTYP( J )
         CTOT( CT ) = CTOT( CT ) + 1
  140 CONTINUE
```

```
*
*     PSM(*) = Position in SubMatrix (of types 1 through 4)
*
      PSM( 1 ) = 2
      PSM( 2 ) = 2 + CTOT( 1 )
      PSM( 3 ) = PSM( 2 ) + CTOT( 2 )
      PSM( 4 ) = PSM( 3 ) + CTOT( 3 )
*
*     Fill out the IDXC array so that the permutation which it induces
*     will place all type-1 columns first, all type-2 columns next,
*     then all type-3's, and finally all type-4's, starting from the
*     second column. This applies similarly to the rows of VT.
*
      DO 150 J = 2, N
         JP = IDXP( J )
         CT = COLTYP( JP )
         IDXC( PSM( CT ) ) = J
         PSM( CT ) = PSM( CT ) + 1
  150 CONTINUE
*
*     Sort the singular values and corresponding singular vectors into
*     DSIGMA, U2, and VT2 respectively.  The singular values/vectors
*     which were not deflated go into the first K slots of DSIGMA, U2,
*     and VT2 respectively, while those which were deflated go into the
*     last N - K slots, except that the first column/row will be treated
*     separately.
*
      DO 160 J = 2, N
         JP = IDXP( J )
         DSIGMA( J ) = D( JP )
         IDXJ = IDXQ( IDX( IDXP( IDXC( J ) ) )+1 )
         IF( IDXJ.LE.NLP1 ) THEN
            IDXJ = IDXJ - 1
         END IF
         CALL SCOPY( N, U( 1, IDXJ ), 1, U2( 1, J ), 1 )
         CALL SCOPY( M, VT( IDXJ, 1 ), LDVT, VT2( J, 1 ), LDVT2 )
  160 CONTINUE
*
*     Determine DSIGMA(1), DSIGMA(2) and Z(1)
*
      DSIGMA( 1 ) = ZERO
      HLFTOL = TOL / TWO
      IF( ABS( DSIGMA( 2 ) ).LE.HLFTOL )
     $   DSIGMA( 2 ) = HLFTOL
      IF( M.GT.N ) THEN
         Z( 1 ) = SLAPY2( Z1, Z( M ) )
         IF( Z( 1 ).LE.TOL ) THEN
            C = ONE
            S = ZERO
            Z( 1 ) = TOL
```

```
         ELSE
            C = Z1 / Z( 1 )
            S = Z( M ) / Z( 1 )
         END IF
      ELSE
         IF( ABS( Z1 ).LE.TOL ) THEN
            Z( 1 ) = TOL
         ELSE
            Z( 1 ) = Z1
         END IF
      END IF
*
*     Move the rest of the updating row to Z.
*
      CALL SCOPY( K-1, U2( 2, 1 ), 1, Z( 2 ), 1 )
*
*     Determine the first column of U2, the first row of VT2 and the
*     last row of VT.
*
      CALL SLASET( 'A', N, 1, ZERO, ZERO, U2, LDU2 )
      U2( NLP1, 1 ) = ONE
      IF( M.GT.N ) THEN
         DO 170 I = 1, NLP1
            VT( M, I ) = -S*VT( NLP1, I )
            VT2( 1, I ) = C*VT( NLP1, I )
  170    CONTINUE
         DO 180 I = NLP2, M
            VT2( 1, I ) = S*VT( M, I )
            VT( M, I ) = C*VT( M, I )
  180    CONTINUE
      ELSE
         CALL SCOPY( M, VT( NLP1, 1 ), LDVT, VT2( 1, 1 ), LDVT2 )
      END IF
      IF( M.GT.N ) THEN
         CALL SCOPY( M, VT( M, 1 ), LDVT, VT2( M, 1 ), LDVT2 )
      END IF
*
*     The deflated singular values and their corresponding vectors go
*     into the back of D, U, and V respectively.
*
      IF( N.GT.K ) THEN
         CALL SCOPY( N-K, DSIGMA( K+1 ), 1, D( K+1 ), 1 )
         CALL SLACPY( 'A', N, N-K, U2( 1, K+1 ), LDU2, U( 1, K+1 ),
     $                LDU )
         CALL SLACPY( 'A', N-K, M, VT2( K+1, 1 ), LDVT2, VT( K+1, 1 ),
     $                LDVT )
      END IF
*
*     Copy CTOT into COLTYP for referencing in SLASD3.
*
```

```
      DO 190 J = 1, 4
         COLTYP( J ) = CTOT( J )
  190 CONTINUE
*
      RETURN
*
*     End of SLASD2
*
      END
```

———-

90.94.4 lisp code

— **slasd2.lisp** —

```
(let* ((zero 0.0) (one 1.0) (two 2.0) (eight 8.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 8.0 8.0) eight)
           (ignorable zero one two eight))
  (defun slasd2
         (nl nr sqre k d z alpha beta u ldu vt ldvt dsigma u2 ldu2 vt2 ldvt2
          idxp idx idxc idxq coltyp info)
    (declare (type (array f2cl-lib:integer4 (*)) coltyp idxq idxc idx idxp)
             (type (single-float) beta alpha)
             (type (array single-float (*)) vt2 u2 dsigma vt u z d)
             (type (f2cl-lib:integer4) info ldvt2 ldu2 ldvt ldu k sqre nr nl))
    (f2cl-lib:with-multi-array-data
        ((d single-float d-%data% d-%offset%)
         (z single-float z-%data% z-%offset%)
         (u single-float u-%data% u-%offset%)
         (vt single-float vt-%data% vt-%offset%)
         (dsigma single-float dsigma-%data% dsigma-%offset%)
         (u2 single-float u2-%data% u2-%offset%)
         (vt2 single-float vt2-%data% vt2-%offset%)
         (idxp f2cl-lib:integer4 idxp-%data% idxp-%offset%)
         (idx f2cl-lib:integer4 idx-%data% idx-%offset%)
         (idxc f2cl-lib:integer4 idxc-%data% idxc-%offset%)
         (idxq f2cl-lib:integer4 idxq-%data% idxq-%offset%)
         (coltyp f2cl-lib:integer4 coltyp-%data% coltyp-%offset%))
      (prog ((c 0.0) (eps 0.0) (hlftol 0.0) (s 0.0) (tau 0.0) (tol 0.0)
             (z1 0.0) (ct 0) (i 0) (idxi 0) (idxj 0) (idxjp 0) (j 0) (jp 0)
             (jprev 0) (k2 0) (m 0) (n 0) (nlp1 0) (nlp2 0)
             (ctot (make-array 4 :element-type 'f2cl-lib:integer4))
             (psm (make-array 4 :element-type 'f2cl-lib:integer4)))
        (declare (type (single-float) c eps hlftol s tau tol z1)
```

```
         (type (f2cl-lib:integer4) ct i idxi idxj idxjp j jp jprev k2 m
          n nlp1 nlp2)
         (type (array f2cl-lib:integer4 (4)) ctot psm))
(setf info 0)
(cond ((< nl 1) (setf info -1)) ((< nr 1) (setf info -2))
      ((and (/= sqre 1) (/= sqre 0)) (setf info -3)))
(setf n (f2cl-lib:int-add nl nr 1))
(setf m (f2cl-lib:int-add n sqre))
(cond ((< ldu n) (setf info -10)) ((< ldvt m) (setf info -12))
      ((< ldu2 n) (setf info -15)) ((< ldvt2 m) (setf info -17)))
(cond
 ((/= info 0) (xerbla "SLASD2" (f2cl-lib:int-sub info))
  (go end_label)))
(setf nlp1 (f2cl-lib:int-add nl 1))
(setf nlp2 (f2cl-lib:int-add nl 2))
(setf z1
        (* alpha
           (f2cl-lib:fref vt-%data% (nlp1 nlp1) ((1 ldvt) (1 *))
                          vt-%offset%)))
(setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%) z1)
(f2cl-lib:fdo (i nl (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         z-%offset%)
            (* alpha
               (f2cl-lib:fref vt-%data% (i nlp1) ((1 ldvt) (1 *))
                              vt-%offset%)))
    (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         d-%offset%)
            (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
    (setf (f2cl-lib:fref idxq-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         idxq-%offset%)
            (f2cl-lib:int-add
             (f2cl-lib:fref idxq-%data% (i) ((1 *)) idxq-%offset%) 1))
   label10))
(f2cl-lib:fdo (i nlp2 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
            (* beta
               (f2cl-lib:fref vt-%data% (i nlp2) ((1 ldvt) (1 *))
                              vt-%offset%)))
   label20))
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i nlp1) nil)
  (tagbody
    (setf (f2cl-lib:fref coltyp-%data% (i) ((1 *)) coltyp-%offset%) 1)
   label30))
(f2cl-lib:fdo (i nlp2 (f2cl-lib:int-add i 1))
```

```
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref coltyp-%data% (i) ((1 *)) coltyp-%offset%) 2)
   label40))
(f2cl-lib:fdo (i nlp2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref idxq-%data% (i) ((1 *)) idxq-%offset%)
            (f2cl-lib:int-add
             (f2cl-lib:fref idxq-%data% (i) ((1 *)) idxq-%offset%)
             nlp1))
   label50))
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref dsigma-%data% (i) ((1 *)) dsigma-%offset%)
            (f2cl-lib:fref d-%data% ((f2cl-lib:fref idxq (i) ((1 *))))
                           ((1 *)) d-%offset%))
    (setf (f2cl-lib:fref u2-%data% (i 1) ((1 ldu2) (1 *)) u2-%offset%)
            (f2cl-lib:fref z-%data% ((f2cl-lib:fref idxq (i) ((1 *))))
                           ((1 *)) z-%offset%))
    (setf (f2cl-lib:fref idxc-%data% (i) ((1 *)) idxc-%offset%)
            (f2cl-lib:fref coltyp-%data%
                           ((f2cl-lib:fref idxq (i) ((1 *)))) ((1 *))
                           coltyp-%offset%))
   label60))
(slamrg nl nr
 (f2cl-lib:array-slice dsigma-%data% single-float (2) ((1 *))
                       dsigma-%offset%)
 1 1
 (f2cl-lib:array-slice idx-%data% f2cl-lib:integer4 (2) ((1 *))
                       idx-%offset%))
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf idxi
            (f2cl-lib:int-add 1
                              (f2cl-lib:fref idx-%data% (i) ((1 *))
                                             idx-%offset%)))
    (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
            (f2cl-lib:fref dsigma-%data% (idxi) ((1 *))
                           dsigma-%offset%))
    (setf (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
            (f2cl-lib:fref u2-%data% (idxi 1) ((1 ldu2) (1 *))
                           u2-%offset%))
    (setf (f2cl-lib:fref coltyp-%data% (i) ((1 *)) coltyp-%offset%)
            (f2cl-lib:fref idxc-%data% (idxi) ((1 *)) idxc-%offset%))
   label70))
(setf eps (slamch "Epsilon"))
(setf tol (max (abs alpha) (abs beta)))
```

```
 (setf tol
         (* eight eps
            (max (abs (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%))
                 tol)))
 (setf k 1)
 (setf k2 (f2cl-lib:int-add n 1))
 (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
      ((<= (abs (f2cl-lib:fref z (j) ((1 *)))) tol)
       (setf k2 (f2cl-lib:int-sub k2 1))
       (setf (f2cl-lib:fref idxp-%data% (k2) ((1 *)) idxp-%offset%) j)
       (setf (f2cl-lib:fref coltyp-%data% (j) ((1 *)) coltyp-%offset%)
               4)
       (if (= j n)
           (go label120)))
      (t (setf jprev j) (go label90)))
    label80))
label90
 (setf j jprev)
label100
 (setf j (f2cl-lib:int-add j 1))
 (if (> j n)
     (go label110))
 (cond
  ((<= (abs (f2cl-lib:fref z (j) ((1 *)))) tol)
   (setf k2 (f2cl-lib:int-sub k2 1))
   (setf (f2cl-lib:fref idxp-%data% (k2) ((1 *)) idxp-%offset%) j)
   (setf (f2cl-lib:fref coltyp-%data% (j) ((1 *)) coltyp-%offset%) 4))
  (t
   (cond
    ((<=
      (abs
       (+ (f2cl-lib:fref d (j) ((1 *)))
          (- (f2cl-lib:fref d (jprev) ((1 *))))))
      tol)
     (setf s (f2cl-lib:fref z-%data% (jprev) ((1 *)) z-%offset%))
     (setf c (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%))
     (setf tau (slapy2 c s)) (setf c (/ c tau)) (setf s (/ (- s) tau))
     (setf (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%) tau)
     (setf (f2cl-lib:fref z-%data% (jprev) ((1 *)) z-%offset%) zero)
     (setf idxjp
             (f2cl-lib:fref idxq-%data%
                            ((f2cl-lib:int-add
                              (f2cl-lib:fref idx (jprev) ((1 *))) 1))
                            ((1 *)) idxq-%offset%))
     (setf idxj
             (f2cl-lib:fref idxq-%data%
                            ((f2cl-lib:int-add
```

```
                              (f2cl-lib:fref idx (j) ((1 *))) 1))
                            ((1 *)) idxq-%offset%))
      (cond ((<= idxjp nlp1) (setf idxjp (f2cl-lib:int-sub idxjp 1))))
      (cond ((<= idxj nlp1) (setf idxj (f2cl-lib:int-sub idxj 1))))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (srot n
           (f2cl-lib:array-slice u-%data% single-float (1 idxjp)
                                 ((1 ldu) (1 *)) u-%offset%)
           1
           (f2cl-lib:array-slice u-%data% single-float (1 idxj)
                                 ((1 ldu) (1 *)) u-%offset%)
           1 c s)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf n var-0))
        (when var-5 (setf c var-5))
        (when var-6 (setf s var-6)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (srot m
           (f2cl-lib:array-slice vt-%data% single-float (idxjp 1)
                                 ((1 ldvt) (1 *)) vt-%offset%)
           ldvt
           (f2cl-lib:array-slice vt-%data% single-float (idxj 1)
                                 ((1 ldvt) (1 *)) vt-%offset%)
           ldvt c s)
        (declare (ignore var-1 var-3))
        (when var-0 (setf m var-0))
        (when var-2 (setf ldvt var-2))
        (when var-4 (setf ldvt var-4))
        (when var-5 (setf c var-5))
        (when var-6 (setf s var-6)))
      (cond
       ((/= (f2cl-lib:fref coltyp (j) ((1 *)))
            (f2cl-lib:fref coltyp (jprev) ((1 *))))
        (setf (f2cl-lib:fref coltyp-%data% (j) ((1 *)) coltyp-%offset%)
                3)))
      (setf (f2cl-lib:fref coltyp-%data% (jprev) ((1 *)) coltyp-%offset%)
              4)
      (setf k2 (f2cl-lib:int-sub k2 1))
      (setf (f2cl-lib:fref idxp-%data% (k2) ((1 *)) idxp-%offset%) jprev)
      (setf jprev j))
     (t (setf k (f2cl-lib:int-add k 1))
      (setf (f2cl-lib:fref u2-%data% (k 1) ((1 ldu2) (1 *)) u2-%offset%)
              (f2cl-lib:fref z-%data% (jprev) ((1 *)) z-%offset%))
      (setf (f2cl-lib:fref dsigma-%data% (k) ((1 *)) dsigma-%offset%)
              (f2cl-lib:fref d-%data% (jprev) ((1 *)) d-%offset%))
      (setf (f2cl-lib:fref idxp-%data% (k) ((1 *)) idxp-%offset%) jprev)
      (setf jprev j)))))
  (go label100)
 label110
  (setf k (f2cl-lib:int-add k 1))
```

```
 (setf (f2cl-lib:fref u2-%data% (k 1) ((1 ldu2) (1 *)) u2-%offset%)
        (f2cl-lib:fref z-%data% (jprev) ((1 *)) z-%offset%))
 (setf (f2cl-lib:fref dsigma-%data% (k) ((1 *)) dsigma-%offset%)
        (f2cl-lib:fref d-%data% (jprev) ((1 *)) d-%offset%))
 (setf (f2cl-lib:fref idxp-%data% (k) ((1 *)) idxp-%offset%) jprev)
label120
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j 4) nil)
   (tagbody (setf (f2cl-lib:fref ctot (j) ((1 4))) 0) label130))
 (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf ct (f2cl-lib:fref coltyp-%data% (j) ((1 *)) coltyp-%offset%))
     (setf (f2cl-lib:fref ctot (ct) ((1 4)))
             (f2cl-lib:int-add (f2cl-lib:fref ctot (ct) ((1 4))) 1))
    label140))
 (setf (f2cl-lib:fref psm (1) ((1 4))) 2)
 (setf (f2cl-lib:fref psm (2) ((1 4)))
        (f2cl-lib:int-add 2 (f2cl-lib:fref ctot (1) ((1 4)))))
 (setf (f2cl-lib:fref psm (3) ((1 4)))
        (f2cl-lib:int-add (f2cl-lib:fref psm (2) ((1 4)))
                          (f2cl-lib:fref ctot (2) ((1 4)))))
 (setf (f2cl-lib:fref psm (4) ((1 4)))
        (f2cl-lib:int-add (f2cl-lib:fref psm (3) ((1 4)))
                          (f2cl-lib:fref ctot (3) ((1 4)))))
 (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf jp (f2cl-lib:fref idxp-%data% (j) ((1 *)) idxp-%offset%))
     (setf ct
             (f2cl-lib:fref coltyp-%data% (jp) ((1 *)) coltyp-%offset%))
     (setf (f2cl-lib:fref idxc-%data% ((f2cl-lib:fref psm (ct) ((1 4))))
                          ((1 *)) idxc-%offset%)
             j)
     (setf (f2cl-lib:fref psm (ct) ((1 4)))
             (f2cl-lib:int-add (f2cl-lib:fref psm (ct) ((1 4))) 1))
    label150))
 (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf jp (f2cl-lib:fref idxp-%data% (j) ((1 *)) idxp-%offset%))
     (setf (f2cl-lib:fref dsigma-%data% (j) ((1 *)) dsigma-%offset%)
             (f2cl-lib:fref d-%data% (jp) ((1 *)) d-%offset%))
     (setf idxj
             (f2cl-lib:fref idxq-%data%
                            ((f2cl-lib:int-add
                              (f2cl-lib:fref idx
                                             ((f2cl-lib:fref idxp
                                                             ((f2cl-lib:fref
                                                               idxc (j)
```

```
                                                                  ((1 *))))
                                                                ((1 *))))
                                                   ((1 *)))
                                 1))
                               ((1 *)) idxq-%offset%))
    (cond ((<= idxj nlp1) (setf idxj (f2cl-lib:int-sub idxj 1))))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy n
         (f2cl-lib:array-slice u-%data% single-float (1 idxj)
                               ((1 ldu) (1 *)) u-%offset%)
         1
         (f2cl-lib:array-slice u2-%data% single-float (1 j)
                               ((1 ldu2) (1 *)) u2-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy m
         (f2cl-lib:array-slice vt-%data% single-float (idxj 1)
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt
         (f2cl-lib:array-slice vt2-%data% single-float (j 1)
                               ((1 ldvt2) (1 *)) vt2-%offset%)
         ldvt2)
      (declare (ignore var-1 var-3))
      (when var-0 (setf m var-0))
      (when var-2 (setf ldvt var-2))
      (when var-4 (setf ldvt2 var-4)))
  label160))
(setf (f2cl-lib:fref dsigma-%data% (1) ((1 *)) dsigma-%offset%) zero)
(setf hlftol (/ tol two))
(if (<= (abs (f2cl-lib:fref dsigma-%data% (2) ((1 *)) dsigma-%offset%))
        hlftol)
    (setf (f2cl-lib:fref dsigma-%data% (2) ((1 *)) dsigma-%offset%)
            hlftol))
(cond
 ((> m n)
  (setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)
          (slapy2 z1 (f2cl-lib:fref z-%data% (m) ((1 *)) z-%offset%)))
  (cond
   ((<= (f2cl-lib:fref z (1) ((1 *))) tol) (setf c one) (setf s zero)
    (setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%) tol))
   (t (setf c (/ z1 (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)))
    (setf s
            (/ (f2cl-lib:fref z-%data% (m) ((1 *)) z-%offset%)
               (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%))))))
 (t
  (cond
   ((<= (abs z1) tol)
    (setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%) tol))
```

```
   (t (setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%) z1)))))
(scopy (f2cl-lib:int-sub k 1)
 (f2cl-lib:array-slice u2-%data% single-float (2 1) ((1 ldu2) (1 *))
                       u2-%offset%)
 1 (f2cl-lib:array-slice z-%data% single-float (2) ((1 *)) z-%offset%)
 1)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "A" n 1 zero zero u2 ldu2)
  (declare (ignore var-0 var-2 var-5))
  (when var-1 (setf n var-1))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldu2 var-6)))
(setf (f2cl-lib:fref u2-%data% (nlp1 1) ((1 ldu2) (1 *)) u2-%offset%)
        one)
(cond
 ((> m n)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i nlp1) nil)
    (tagbody
      (setf (f2cl-lib:fref vt-%data% (m i) ((1 ldvt) (1 *))
                           vt-%offset%)
              (* (- s)
                 (f2cl-lib:fref vt-%data% (nlp1 i) ((1 ldvt) (1 *))
                                vt-%offset%)))
      (setf (f2cl-lib:fref vt2-%data% (1 i) ((1 ldvt2) (1 *))
                           vt2-%offset%)
              (* c
                 (f2cl-lib:fref vt-%data% (nlp1 i) ((1 ldvt) (1 *))
                                vt-%offset%)))
     label170))
  (f2cl-lib:fdo (i nlp2 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf (f2cl-lib:fref vt2-%data% (1 i) ((1 ldvt2) (1 *))
                           vt2-%offset%)
              (* s
                 (f2cl-lib:fref vt-%data% (m i) ((1 ldvt) (1 *))
                                vt-%offset%)))
      (setf (f2cl-lib:fref vt-%data% (m i) ((1 ldvt) (1 *))
                           vt-%offset%)
              (* c
                 (f2cl-lib:fref vt-%data% (m i) ((1 ldvt) (1 *))
                                vt-%offset%)))
     label180)))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (scopy m
       (f2cl-lib:array-slice vt-%data% single-float (nlp1 1)
                             ((1 ldvt) (1 *)) vt-%offset%)
```

```
        ldvt
        (f2cl-lib:array-slice vt2-%data% single-float (1 1)
                              ((1 ldvt2) (1 *)) vt2-%offset%)
        ldvt2)
    (declare (ignore var-1 var-3))
    (when var-0 (setf m var-0))
    (when var-2 (setf ldvt var-2))
    (when var-4 (setf ldvt2 var-4)))))
(cond
 ((> m n)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (scopy m
        (f2cl-lib:array-slice vt-%data% single-float (m 1)
                              ((1 ldvt) (1 *)) vt-%offset%)
        ldvt
        (f2cl-lib:array-slice vt2-%data% single-float (m 1)
                              ((1 ldvt2) (1 *)) vt2-%offset%)
        ldvt2)
    (declare (ignore var-1 var-3))
    (when var-0 (setf m var-0))
    (when var-2 (setf ldvt var-2))
    (when var-4 (setf ldvt2 var-4)))))
(cond
 ((> n k)
  (scopy (f2cl-lib:int-sub n k)
   (f2cl-lib:array-slice dsigma-%data% single-float ((+ k 1)) ((1 *))
                         dsigma-%offset%)
   1
   (f2cl-lib:array-slice d-%data% single-float ((+ k 1)) ((1 *))
                         d-%offset%)
   1)
  (slacpy "A" n (f2cl-lib:int-sub n k)
   (f2cl-lib:array-slice u2-%data% single-float
                         (1 (f2cl-lib:int-add k 1)) ((1 ldu2) (1 *))
                         u2-%offset%)
   ldu2
   (f2cl-lib:array-slice u-%data% single-float
                         (1 (f2cl-lib:int-add k 1)) ((1 ldu) (1 *))
                         u-%offset%)
   ldu)
  (slacpy "A" (f2cl-lib:int-sub n k) m
   (f2cl-lib:array-slice vt2-%data% single-float ((+ k 1) 1)
                         ((1 ldvt2) (1 *)) vt2-%offset%)
   ldvt2
   (f2cl-lib:array-slice vt-%data% single-float ((+ k 1) 1)
                         ((1 ldvt) (1 *)) vt-%offset%)
   ldvt)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j 4) nil)
  (tagbody
```

```
            (setf (f2cl-lib:fref coltyp-%data% (j) ((1 *)) coltyp-%offset%)
                    (f2cl-lib:fref ctot (j) ((1 4))))
           label190))
         (go end_label)
        end_label
         (return
          (values nil nil nil k nil nil nil nil nil nil nil ldvt nil nil ldu2
                  nil ldvt2 nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slasd2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil fortran-to-lisp::k
                                              nil nil nil nil nil nil nil
                                              fortran-to-lisp::ldvt nil nil
                                              fortran-to-lisp::ldu2 nil
                                              fortran-to-lisp::ldvt2 nil nil
                                              nil nil nil
                                              fortran-to-lisp::info)
```

```
:calls
'(fortran-to_lisp::slacpy
  fortran-to-lisp::slapy2
  fortran-to-lisp::slamrg))))
```

———-

90.95 slasd3

90.95.1 Axiom unit tests

— **slasd3.input** —

```
)set break resume
)sys rm -f slasd3.output
)spool slasd3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.95.2 Axiom help page

— **slasd3.help** —

```
==================================================================
slasd3 examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slasd3 (NL, NR, SQRE, K, D, Q, LDQ, DSIGMA, U, LDU, U2,
           LDU2, VT, LDVT, VT2, LDVT2, IDXC, CTOT, Z, INFO)
           SLASD3 finds all square roots of the roots of the secular equation,
```

```
          as defined by the values in D and Z, and then updates the singular
          vectors by matrix multiplication. Used by sbdsdc.

Function/Subroutine Documentation
   subroutine slasd3 (integer NL, integer NR, integer SQRE, integer K, real,
       dimension( * ) D, real, dimension( ldq, * ) Q, integer LDQ, real,
       dimension( * ) DSIGMA, real, dimension( ldu, * ) U, integer LDU, real,
       dimension( ldu2, * ) U2, integer LDU2, real, dimension( ldvt, * ) VT,
       integer LDVT, real, dimension( ldvt2, * ) VT2, integer LDVT2, integer,
       dimension( * ) IDXC, integer, dimension( * ) CTOT, real, dimension( * )
       Z, integer INFO)
       SLASD3 finds all square roots of the roots of the secular equation, as
       defined by the values in D and Z, and then updates the singular vectors
       by matrix multiplication. Used by sbdsdc.

Purpose:
   SLASD3 finds all the square roots of the roots of the secular
   equation, as defined by the values in D and Z.  It makes the
   appropriate calls to SLASD4 and then updates the singular
   vectors by matrix multiplication.

   This code makes very mild assumptions about floating point
   arithmetic. It will work on machines with a guard digit in
   add/subtract, or on those binary machines without guard digits
   which subtract like the Cray XMP, Cray YMP, Cray C 90, or Cray 2.
   It could conceivably fail on hexadecimal or decimal machines
   without guard digits, but we know of none.

   SLASD3 is called from SLASD1.

Parameters:

 NL
   NL is INTEGER
   The row dimension of the upper block.  NL >= 1.

 NR
   NR is INTEGER
   The row dimension of the lower block.  NR >= 1.

 SQRE
   SQRE is INTEGER
   = 0: the lower block is an NR-by-NR square matrix.
   = 1: the lower block is an NR-by-(NR+1) rectangular matrix.

   The bidiagonal matrix has N = NL + NR + 1 rows and
   M = N + SQRE >= N columns.

 K
   K is INTEGER
```

```
  The size of the secular equation, 1 =< K = < N.

D
  D is REAL array, dimension(K)
  On exit the square roots of the roots of the secular equation,
  in ascending order.

Q
  Q is REAL array, dimension at least (LDQ,K).

LDQ
  LDQ is INTEGER
  The leading dimension of the array Q.  LDQ >= K.

DSIGMA
  DSIGMA is REAL array, dimension(K)
  The first K elements of this array contain the old roots
  of the deflated updating problem.  These are the poles
  of the secular equation.

U
  U is REAL array, dimension (LDU, N)
  The last N - K columns of this matrix contain the deflated
  left singular vectors.

LDU
  LDU is INTEGER
  The leading dimension of the array U.  LDU >= N.

U2
  U2 is REAL array, dimension (LDU2, N)
  The first K columns of this matrix contain the non-deflated
  left singular vectors for the split problem.

LDU2
  LDU2 is INTEGER
  The leading dimension of the array U2.  LDU2 >= N.

VT
  VT is REAL array, dimension (LDVT, M)
  The last M - K columns of VT**T contain the deflated
  right singular vectors.

LDVT
  LDVT is INTEGER
  The leading dimension of the array VT.  LDVT >= N.

VT2
  VT2 is REAL array, dimension (LDVT2, N)
  The first K columns of VT2**T contain the non-deflated
```

```
  right singular vectors for the split problem.

LDVT2
  LDVT2 is INTEGER
  The leading dimension of the array VT2.  LDVT2 >= N.

IDXC
  IDXC is INTEGER array, dimension (N)
  The permutation used to arrange the columns of U (and rows of
  VT) into three groups:  the first group contains non-zero
  entries only at and above (or before) NL +1; the second
  contains non-zero entries only at and below (or after) NL+2;
  and the third is dense. The first column of U and the row of
  VT are treated separately, however.

  The rows of the singular vectors found by SLASD4
  must be likewise permuted before the matrix multiplies can take place.

CTOT
  CTOT is INTEGER array, dimension (4)
  A count of the total number of the various types of columns
  in U (or rows in VT), as described in IDXC. The fourth column
  type is any column which has been deflated.

Z
  Z is REAL array, dimension (K)
  The first K elements of this array contain the components
  of the deflation-adjusted updating row vector.

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if INFO = 1, a singular value did not converge
```

———-

90.95.3 fortran code

— **slasd3.f** —

```
*  =====================================================================
      SUBROUTINE SLASD3( NL, NR, SQRE, K, D, Q, LDQ, DSIGMA, U, LDU, U2,
     $                   LDU2, VT, LDVT, VT2, LDVT2, IDXC, CTOT, Z,
     $                   INFO )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
```

```
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, K, LDQ, LDU, LDU2, LDVT, LDVT2, NL, NR,
     $                   SQRE
*     ..
*     .. Array Arguments ..
      INTEGER            CTOT( * ), IDXC( * )
      REAL               D( * ), DSIGMA( * ), Q( LDQ, * ), U( LDU, * ),
     $                   U2( LDU2, * ), VT( LDVT, * ), VT2( LDVT2, * ),
     $                   Z( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO, NEGONE
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0,
     $                     NEGONE = -1.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            CTEMP, I, J, JC, KTEMP, M, N, NLP1, NLP2, NRP1
      REAL               RHO, TEMP
*     ..
*     .. External Functions ..
      REAL               SLAMC3, SNRM2
      EXTERNAL           SLAMC3, SNRM2
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SGEMM, SLACPY, SLASCL, SLASD4, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SIGN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( NL.LT.1 ) THEN
         INFO = -1
      ELSE IF( NR.LT.1 ) THEN
         INFO = -2
      ELSE IF( ( SQRE.NE.1 ) .AND. ( SQRE.NE.0 ) ) THEN
         INFO = -3
      END IF
*
```

```
      N = NL + NR + 1
      M = N + SQRE
      NLP1 = NL + 1
      NLP2 = NL + 2
*
      IF( ( K.LT.1 ) .OR. ( K.GT.N ) ) THEN
         INFO = -4
      ELSE IF( LDQ.LT.K ) THEN
         INFO = -7
      ELSE IF( LDU.LT.N ) THEN
         INFO = -10
      ELSE IF( LDU2.LT.N ) THEN
         INFO = -12
      ELSE IF( LDVT.LT.M ) THEN
         INFO = -14
      ELSE IF( LDVT2.LT.M ) THEN
         INFO = -16
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SLASD3', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( K.EQ.1 ) THEN
         D( 1 ) = ABS( Z( 1 ) )
         CALL SCOPY( M, VT2( 1, 1 ), LDVT2, VT( 1, 1 ), LDVT )
         IF( Z( 1 ).GT.ZERO ) THEN
            CALL SCOPY( N, U2( 1, 1 ), 1, U( 1, 1 ), 1 )
         ELSE
            DO 10 I = 1, N
               U( I, 1 ) = -U2( I, 1 )
   10       CONTINUE
         END IF
         RETURN
      END IF
*
*     Modify values DSIGMA(i) to make sure all DSIGMA(i)-DSIGMA(j) can
*     be computed with high relative accuracy (barring over/underflow).
*     This is a problem on machines without a guard digit in
*     add/subtract (Cray XMP, Cray YMP, Cray C 90 and Cray 2).
*     The following code replaces DSIGMA(I) by 2*DSIGMA(I)-DSIGMA(I),
*     which on any of these machines zeros out the bottommost
*     bit of DSIGMA(I) if it is 1; this makes the subsequent
*     subtractions DSIGMA(I)-DSIGMA(J) unproblematic when cancellation
*     occurs. On binary machines with a guard digit (almost all
*     machines) it does not change DSIGMA(I) at all. On hexadecimal
*     and decimal machines with a guard digit, it slightly
*     changes the bottommost bits of DSIGMA(I). It does not account
```

```
*     for hexadecimal or decimal machines without guard digits
*     (we know of none). We use a subroutine call to compute
*     2*DSIGMA(I) to prevent optimizing compilers from eliminating
*     this code.
*
      DO 20 I = 1, K
         DSIGMA( I ) = SLAMC3( DSIGMA( I ), DSIGMA( I ) ) - DSIGMA( I )
   20 CONTINUE
*
*     Keep a copy of Z.
*
      CALL SCOPY( K, Z, 1, Q, 1 )
*
*     Normalize Z.
*
      RHO = SNRM2( K, Z, 1 )
      CALL SLASCL( 'G', 0, 0, RHO, ONE, K, 1, Z, K, INFO )
      RHO = RHO*RHO
*
*     Find the new singular values.
*
      DO 30 J = 1, K
         CALL SLASD4( K, J, DSIGMA, Z, U( 1, J ), RHO, D( J ),
     $                VT( 1, J ), INFO )
*
*        If the zero finder fails, report the convergence failure.
*
         IF( INFO.NE.0 ) THEN
            RETURN
         END IF
   30 CONTINUE
*
*     Compute updated Z.
*
      DO 60 I = 1, K
         Z( I ) = U( I, K )*VT( I, K )
         DO 40 J = 1, I - 1
            Z( I ) = Z( I )*( U( I, J )*VT( I, J ) /
     $               ( DSIGMA( I )-DSIGMA( J ) ) /
     $               ( DSIGMA( I )+DSIGMA( J ) ) )
   40    CONTINUE
         DO 50 J = I, K - 1
            Z( I ) = Z( I )*( U( I, J )*VT( I, J ) /
     $               ( DSIGMA( I )-DSIGMA( J+1 ) ) /
     $               ( DSIGMA( I )+DSIGMA( J+1 ) ) )
   50    CONTINUE
         Z( I ) = SIGN( SQRT( ABS( Z( I ) ) ), Q( I, 1 ) )
   60 CONTINUE
*
*     Compute left singular vectors of the modified diagonal matrix,
```

```
*     and store related information for the right singular vectors.
*
      DO 90 I = 1, K
         VT( 1, I ) = Z( 1 ) / U( 1, I ) / VT( 1, I )
         U( 1, I ) = NEGONE
         DO 70 J = 2, K
            VT( J, I ) = Z( J ) / U( J, I ) / VT( J, I )
            U( J, I ) = DSIGMA( J )*VT( J, I )
   70    CONTINUE
         TEMP = SNRM2( K, U( 1, I ), 1 )
         Q( 1, I ) = U( 1, I ) / TEMP
         DO 80 J = 2, K
            JC = IDXC( J )
            Q( J, I ) = U( JC, I ) / TEMP
   80    CONTINUE
   90 CONTINUE
*
*     Update the left singular vector matrix.
*
      IF( K.EQ.2 ) THEN
         CALL SGEMM( 'N', 'N', N, K, K, ONE, U2, LDU2, Q, LDQ, ZERO, U,
     $               LDU )
         GO TO 100
      END IF
      IF( CTOT( 1 ).GT.0 ) THEN
         CALL SGEMM( 'N', 'N', NL, K, CTOT( 1 ), ONE, U2( 1, 2 ), LDU2,
     $               Q( 2, 1 ), LDQ, ZERO, U( 1, 1 ), LDU )
         IF( CTOT( 3 ).GT.0 ) THEN
            KTEMP = 2 + CTOT( 1 ) + CTOT( 2 )
            CALL SGEMM( 'N', 'N', NL, K, CTOT( 3 ), ONE, U2( 1, KTEMP ),
     $                  LDU2, Q( KTEMP, 1 ), LDQ, ONE, U( 1, 1 ), LDU )
         END IF
      ELSE IF( CTOT( 3 ).GT.0 ) THEN
         KTEMP = 2 + CTOT( 1 ) + CTOT( 2 )
         CALL SGEMM( 'N', 'N', NL, K, CTOT( 3 ), ONE, U2( 1, KTEMP ),
     $               LDU2, Q( KTEMP, 1 ), LDQ, ZERO, U( 1, 1 ), LDU )
      ELSE
         CALL SLACPY( 'F', NL, K, U2, LDU2, U, LDU )
      END IF
      CALL SCOPY( K, Q( 1, 1 ), LDQ, U( NLP1, 1 ), LDU )
      KTEMP = 2 + CTOT( 1 )
      CTEMP = CTOT( 2 ) + CTOT( 3 )
      CALL SGEMM( 'N', 'N', NR, K, CTEMP, ONE, U2( NLP2, KTEMP ), LDU2,
     $            Q( KTEMP, 1 ), LDQ, ZERO, U( NLP2, 1 ), LDU )
*
*     Generate the right singular vectors.
*
  100 CONTINUE
      DO 120 I = 1, K
         TEMP = SNRM2( K, VT( 1, I ), 1 )
```

```
         Q( I, 1 ) = VT( 1, I ) / TEMP
         DO 110 J = 2, K
            JC = IDXC( J )
            Q( I, J ) = VT( JC, I ) / TEMP
  110    CONTINUE
  120 CONTINUE
*
*     Update the right singular vector matrix.
*
      IF( K.EQ.2 ) THEN
         CALL SGEMM( 'N', 'N', K, M, K, ONE, Q, LDQ, VT2, LDVT2, ZERO,
     $               VT, LDVT )
         RETURN
      END IF
      KTEMP = 1 + CTOT( 1 )
      CALL SGEMM( 'N', 'N', K, NLP1, KTEMP, ONE, Q( 1, 1 ), LDQ,
     $            VT2( 1, 1 ), LDVT2, ZERO, VT( 1, 1 ), LDVT )
      KTEMP = 2 + CTOT( 1 ) + CTOT( 2 )
      IF( KTEMP.LE.LDVT2 )
     $   CALL SGEMM( 'N', 'N', K, NLP1, CTOT( 3 ), ONE, Q( 1, KTEMP ),
     $               LDQ, VT2( KTEMP, 1 ), LDVT2, ONE, VT( 1, 1 ),
     $               LDVT )
*
      KTEMP = CTOT( 1 ) + 1
      NRP1 = NR + SQRE
      IF( KTEMP.GT.1 ) THEN
         DO 130 I = 1, K
            Q( I, KTEMP ) = Q( I, 1 )
  130    CONTINUE
         DO 140 I = NLP2, M
            VT2( KTEMP, I ) = VT2( 1, I )
  140    CONTINUE
      END IF
      CTEMP = 1 + CTOT( 2 ) + CTOT( 3 )
      CALL SGEMM( 'N', 'N', K, NRP1, CTEMP, ONE, Q( 1, KTEMP ), LDQ,
     $            VT2( KTEMP, NLP2 ), LDVT2, ZERO, VT( 1, NLP2 ), LDVT )
*
      RETURN
*
*     End of SLASD3
*
      END
```

———-

90.95.4 lisp code

— slasd3.lisp —

```
(let* ((one 1.0) (zero 0.0) (negone (- 1.0)))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (type (single-float) negone)
           (ignorable one zero negone))
  (defun slasd3
         (nl nr sqre k d q ldq dsigma u ldu u2 ldu2 vt ldvt vt2 ldvt2 idxc ctot
          z info)
    (declare (type (array f2cl-lib:integer4 (*)) ctot idxc)
             (type (array single-float (*)) z vt2 vt u2 u dsigma q d)
             (type (f2cl-lib:integer4) info ldvt2 ldvt ldu2 ldu ldq k sqre nr
              nl))
    (f2cl-lib:with-multi-array-data
        ((d single-float d-%data% d-%offset%)
         (q single-float q-%data% q-%offset%)
         (dsigma single-float dsigma-%data% dsigma-%offset%)
         (u single-float u-%data% u-%offset%)
         (u2 single-float u2-%data% u2-%offset%)
         (vt single-float vt-%data% vt-%offset%)
         (vt2 single-float vt2-%data% vt2-%offset%)
         (z single-float z-%data% z-%offset%)
         (idxc f2cl-lib:integer4 idxc-%data% idxc-%offset%)
         (ctot f2cl-lib:integer4 ctot-%data% ctot-%offset%))
      (prog ((rho 0.0) (temp 0.0) (ctemp 0) (i 0) (j 0) (jc 0) (ktemp 0) (m 0)
             (n 0) (nlp1 0) (nlp2 0) (nrp1 0))
        (declare (type (single-float) rho temp)
                 (type (f2cl-lib:integer4) ctemp i j jc ktemp m n nlp1 nlp2
                  nrp1))
        (setf info 0)
        (cond ((< nl 1) (setf info -1)) ((< nr 1) (setf info -2))
              ((and (/= sqre 1) (/= sqre 0)) (setf info -3)))
        (setf n (f2cl-lib:int-add nl nr 1))
        (setf m (f2cl-lib:int-add n sqre))
        (setf nlp1 (f2cl-lib:int-add nl 1))
        (setf nlp2 (f2cl-lib:int-add nl 2))
        (cond ((or (< k 1) (> k n)) (setf info -4)) ((< ldq k) (setf info -7))
              ((< ldu n) (setf info -10)) ((< ldu2 n) (setf info -12))
              ((< ldvt m) (setf info -14)) ((< ldvt2 m) (setf info -16)))
        (cond
         ((/= info 0) (xerbla "SLASD3" (f2cl-lib:int-sub info))
          (go end_label)))
        (cond
         ((= k 1)
          (setf (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
                  (abs (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (scopy m
               (f2cl-lib:array-slice vt2-%data% single-float (1 1)
                                     ((1 ldvt2) (1 *)) vt2-%offset%)
               ldvt2
```

```
        (f2cl-lib:array-slice vt-%data% single-float (1 1)
                              ((1 ldvt) (1 *)) vt-%offset%)
        ldvt)
      (declare (ignore var-1 var-3))
      (when var-0 (setf m var-0))
      (when var-2 (setf ldvt2 var-2))
      (when var-4 (setf ldvt var-4)))
    (cond
     ((> (f2cl-lib:fref z (1) ((1 *))) zero)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (scopy n
           (f2cl-lib:array-slice u2-%data% single-float (1 1)
                                 ((1 ldu2) (1 *)) u2-%offset%)
           1
           (f2cl-lib:array-slice u-%data% single-float (1 1)
                                 ((1 ldu) (1 *)) u-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf n var-0))))
     (t
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref u-%data% (i 1) ((1 ldu) (1 *)) u-%offset%)
                  (-
                   (f2cl-lib:fref u2-%data% (i 1) ((1 ldu2) (1 *))
                                  u2-%offset%)))
        label10))))
    (go end_label)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i k) nil)
    (tagbody
      (setf (f2cl-lib:fref dsigma-%data% (i) ((1 *)) dsigma-%offset%)
              (-
               (slamc3
                (f2cl-lib:array-slice dsigma-%data% single-float (i)
                                      ((1 *)) dsigma-%offset%)
                (f2cl-lib:array-slice dsigma-%data% single-float (i)
                                      ((1 *)) dsigma-%offset%))
               (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                              dsigma-%offset%)))
     label20))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (scopy k z 1 q 1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf k var-0)))
  (setf rho
          (multiple-value-bind (ret-val var-0 var-1 var-2)
              (snrm2 k z 1)
            (declare (ignore var-1 var-2))
```

```
          (when var-0 (setf k var-0))
          ret-val))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (slascl "G" 0 0 rho one k 1 z k info)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8))
  (setf info var-9))
(setf rho (* rho rho))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j k) nil)
  (tagbody
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (slasd4 k j dsigma z
         (f2cl-lib:array-slice u-%data% single-float (1 j)
                               ((1 ldu) (1 *)) u-%offset%)
         rho
         (f2cl-lib:array-slice d-%data% single-float (j) ((1 *))
                               d-%offset%)
         (f2cl-lib:array-slice vt-%data% single-float (1 j)
                               ((1 ldvt) (1 *)) vt-%offset%)
         info)
      (declare (ignore var-2 var-3 var-4 var-6 var-7))
      (when var-0 (setf k var-0))
      (when var-1 (setf j var-1))
      (when var-5 (setf rho var-5))
      (when var-8 (setf info var-8)))
    (cond ((/= info 0) (go end_label)))
   label30))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i k) nil)
  (tagbody
    (setf (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
            (*
             (f2cl-lib:fref u-%data% (i k) ((1 ldu) (1 *)) u-%offset%)
             (f2cl-lib:fref vt-%data% (i k) ((1 ldvt) (1 *))
                            vt-%offset%)))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j (f2cl-lib:int-add i (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (setf (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                (* (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                   (/
                    (/
                     (*
                      (f2cl-lib:fref u-%data% (i j) ((1 ldu) (1 *))
                                     u-%offset%)
                      (f2cl-lib:fref vt-%data% (i j) ((1 ldvt) (1 *))
                                     vt-%offset%))
```

```
                        (-
                         (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                                        dsigma-%offset%)
                         (f2cl-lib:fref dsigma-%data% (j) ((1 *))
                                        dsigma-%offset%)))
                       (+
                        (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                                       dsigma-%offset%)
                        (f2cl-lib:fref dsigma-%data% (j) ((1 *))
                                       dsigma-%offset%)))))
        label40))
     (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                   ((> j (f2cl-lib:int-add k (f2cl-lib:int-sub 1))) nil)
       (tagbody
         (setf (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                 (* (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                    (/
                     (/
                      (*
                       (f2cl-lib:fref u-%data% (i j) ((1 ldu) (1 *))
                                      u-%offset%)
                       (f2cl-lib:fref vt-%data% (i j) ((1 ldvt) (1 *))
                                      vt-%offset%))
                      (-
                       (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                                      dsigma-%offset%)
                       (f2cl-lib:fref dsigma-%data%
                                      ((f2cl-lib:int-add j 1)) ((1 *))
                                      dsigma-%offset%)))
                     (+
                      (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                                     dsigma-%offset%)
                      (f2cl-lib:fref dsigma-%data%
                                     ((f2cl-lib:int-add j 1)) ((1 *))
                                     dsigma-%offset%)))))
        label50))
     (setf (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
             (f2cl-lib:sign
              (f2cl-lib:fsqrt
               (abs (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)))
              (f2cl-lib:fref q-%data% (i 1) ((1 ldq) (1 *))
                             q-%offset%)))
   label60))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i k) nil)
  (tagbody
    (setf (f2cl-lib:fref vt-%data% (1 i) ((1 ldvt) (1 *)) vt-%offset%)
            (/
             (/ (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)
                (f2cl-lib:fref u-%data% (1 i) ((1 ldu) (1 *))
```

```
                                u-%offset%))
            (f2cl-lib:fref vt-%data% (1 i) ((1 ldvt) (1 *))
                           vt-%offset%)))
      (setf (f2cl-lib:fref u-%data% (1 i) ((1 ldu) (1 *)) u-%offset%)
            negone)
      (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                    ((> j k) nil)
        (tagbody
          (setf (f2cl-lib:fref vt-%data% (j i) ((1 ldvt) (1 *))
                               vt-%offset%)
                (/
                 (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                    (f2cl-lib:fref u-%data% (j i) ((1 ldu) (1 *))
                                   u-%offset%))
                 (f2cl-lib:fref vt-%data% (j i) ((1 ldvt) (1 *))
                                vt-%offset%)))
          (setf (f2cl-lib:fref u-%data% (j i) ((1 ldu) (1 *)) u-%offset%)
                (*
                 (f2cl-lib:fref dsigma-%data% (j) ((1 *))
                                dsigma-%offset%)
                 (f2cl-lib:fref vt-%data% (j i) ((1 ldvt) (1 *))
                                vt-%offset%)))
         label70))
      (setf temp
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (snrm2 k
                 (f2cl-lib:array-slice u-%data% single-float (1 i)
                                       ((1 ldu) (1 *)) u-%offset%)
                 1)
               (declare (ignore var-1 var-2))
               (when var-0 (setf k var-0))
               ret-val))
      (setf (f2cl-lib:fref q-%data% (1 i) ((1 ldq) (1 *)) q-%offset%)
            (/
             (f2cl-lib:fref u-%data% (1 i) ((1 ldu) (1 *)) u-%offset%)
             temp))
      (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                    ((> j k) nil)
        (tagbody
          (setf jc (f2cl-lib:fref idxc-%data% (j) ((1 *)) idxc-%offset%))
          (setf (f2cl-lib:fref q-%data% (j i) ((1 ldq) (1 *)) q-%offset%)
                (/
                 (f2cl-lib:fref u-%data% (jc i) ((1 ldu) (1 *))
                                u-%offset%)
                 temp))
         label80))
     label90))
  (cond
   ((= k 2)
    (multiple-value-bind
```

```
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (sgemm "N" "N" n k k one u2 ldu2 q ldq zero u ldu)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf n var-2))
      (when var-3 (setf k var-3))
      (when var-4 (setf k var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf ldu2 var-7))
      (when var-9 (setf ldq var-9))
      (when var-10 (setf zero var-10))
      (when var-12 (setf ldu var-12)))
    (go label100)))
  (cond
   ((> (f2cl-lib:fref ctot (1) ((1 *))) 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (sgemm "N" "N" nl k
         (f2cl-lib:array-slice ctot-%data% f2cl-lib:integer4 (1) ((1 *))
                               ctot-%offset%)
         one
         (f2cl-lib:array-slice u2-%data% single-float (1 2)
                               ((1 ldu2) (1 *)) u2-%offset%)
         ldu2
         (f2cl-lib:array-slice q-%data% single-float (2 1)
                               ((1 ldq) (1 *)) q-%offset%)
         ldq zero
         (f2cl-lib:array-slice u-%data% single-float (1 1)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
      (when var-2 (setf nl var-2))
      (when var-3 (setf k var-3))
      (when var-5 (setf one var-5))
      (when var-7 (setf ldu2 var-7))
      (when var-9 (setf ldq var-9))
      (when var-10 (setf zero var-10))
      (when var-12 (setf ldu var-12)))
    (cond
     ((> (f2cl-lib:fref ctot (3) ((1 *))) 0)
      (setf ktemp
              (f2cl-lib:int-add 2
                                (f2cl-lib:fref ctot-%data% (1) ((1 *))
                                               ctot-%offset%)
                                (f2cl-lib:fref ctot-%data% (2) ((1 *))
                                               ctot-%offset%)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
```

```
          (sgemm "N" "N" nl k
           (f2cl-lib:array-slice ctot-%data% f2cl-lib:integer4 (3)
                                 ((1 *)) ctot-%offset%)
           one
           (f2cl-lib:array-slice u2-%data% single-float (1 ktemp)
                                 ((1 ldu2) (1 *)) u2-%offset%)
           ldu2
           (f2cl-lib:array-slice q-%data% single-float (ktemp 1)
                                 ((1 ldq) (1 *)) q-%offset%)
           ldq one
           (f2cl-lib:array-slice u-%data% single-float (1 1)
                                 ((1 ldu) (1 *)) u-%offset%)
           ldu)
         (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
         (when var-2 (setf nl var-2))
         (when var-3 (setf k var-3))
         (when var-5 (setf one var-5))
         (when var-7 (setf ldu2 var-7))
         (when var-9 (setf ldq var-9))
         (when var-10 (setf one var-10))
         (when var-12 (setf ldu var-12))))))
    ((> (f2cl-lib:fref ctot (3) ((1 *))) 0)
     (setf ktemp
             (f2cl-lib:int-add 2
                               (f2cl-lib:fref ctot-%data% (1) ((1 *))
                                              ctot-%offset%)
                               (f2cl-lib:fref ctot-%data% (2) ((1 *))
                                              ctot-%offset%)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12)
         (sgemm "N" "N" nl k
          (f2cl-lib:array-slice ctot-%data% f2cl-lib:integer4 (3) ((1 *))
                                ctot-%offset%)
          one
          (f2cl-lib:array-slice u2-%data% single-float (1 ktemp)
                                ((1 ldu2) (1 *)) u2-%offset%)
          ldu2
          (f2cl-lib:array-slice q-%data% single-float (ktemp 1)
                                ((1 ldq) (1 *)) q-%offset%)
          ldq zero
          (f2cl-lib:array-slice u-%data% single-float (1 1)
                                ((1 ldu) (1 *)) u-%offset%)
          ldu)
       (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
       (when var-2 (setf nl var-2))
       (when var-3 (setf k var-3))
       (when var-5 (setf one var-5))
       (when var-7 (setf ldu2 var-7))
       (when var-9 (setf ldq var-9))
```

```
        (when var-10 (setf zero var-10))
        (when var-12 (setf ldu var-12))))
     (t (slacpy "F" nl k u2 ldu2 u ldu)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy k
         (f2cl-lib:array-slice q-%data% single-float (1 1) ((1 ldq) (1 *))
                               q-%offset%)
         ldq
         (f2cl-lib:array-slice u-%data% single-float (nlp1 1)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-1 var-3))
      (when var-0 (setf k var-0))
      (when var-2 (setf ldq var-2))
      (when var-4 (setf ldu var-4)))
    (setf ktemp
            (f2cl-lib:int-add 2
                              (f2cl-lib:fref ctot-%data% (1) ((1 *))
                                             ctot-%offset%)))
    (setf ctemp
            (f2cl-lib:int-add
             (f2cl-lib:fref ctot-%data% (2) ((1 *)) ctot-%offset%)
             (f2cl-lib:fref ctot-%data% (3) ((1 *)) ctot-%offset%)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
         var-11 var-12)
        (sgemm "N" "N" nr k ctemp one
         (f2cl-lib:array-slice u2-%data% single-float (nlp2 ktemp)
                               ((1 ldu2) (1 *)) u2-%offset%)
         ldu2
         (f2cl-lib:array-slice q-%data% single-float (ktemp 1)
                               ((1 ldq) (1 *)) q-%offset%)
         ldq zero
         (f2cl-lib:array-slice u-%data% single-float (nlp2 1)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf nr var-2))
      (when var-3 (setf k var-3))
      (when var-4 (setf ctemp var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf ldu2 var-7))
      (when var-9 (setf ldq var-9))
      (when var-10 (setf zero var-10))
      (when var-12 (setf ldu var-12)))
  label100
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i k) nil)
      (tagbody
        (setf temp
```

```
                (multiple-value-bind (ret-val var-0 var-1 var-2)
                    (snrm2 k
                     (f2cl-lib:array-slice vt-%data% single-float (1 i)
                                           ((1 ldvt) (1 *)) vt-%offset%)
                     1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf k var-0))
                  ret-val))
    (setf (f2cl-lib:fref q-%data% (i 1) ((1 ldq) (1 *)) q-%offset%)
            (/
             (f2cl-lib:fref vt-%data% (1 i) ((1 ldvt) (1 *))
                            vt-%offset%)
             temp))
    (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                  ((> j k) nil)
      (tagbody
        (setf jc (f2cl-lib:fref idxc-%data% (j) ((1 *)) idxc-%offset%))
        (setf (f2cl-lib:fref q-%data% (i j) ((1 ldq) (1 *)) q-%offset%)
                (/
                 (f2cl-lib:fref vt-%data% (jc i) ((1 ldvt) (1 *))
                                vt-%offset%)
                 temp))
       label110))
   label120))
(cond
 ((= k 2)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (sgemm "N" "N" k m k one q ldq vt2 ldvt2 zero vt ldvt)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf k var-2))
    (when var-3 (setf m var-3))
    (when var-4 (setf k var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldq var-7))
    (when var-9 (setf ldvt2 var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldvt var-12)))
  (go end_label)))
(setf ktemp
        (f2cl-lib:int-add 1
                          (f2cl-lib:fref ctot-%data% (1) ((1 *))
                                         ctot-%offset%)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (sgemm "N" "N" k nlp1 ktemp one
     (f2cl-lib:array-slice q-%data% single-float (1 1) ((1 ldq) (1 *))
                           q-%offset%)
```

```
      ldq
      (f2cl-lib:array-slice vt2-%data% single-float (1 1)
                            ((1 ldvt2) (1 *)) vt2-%offset%)
      ldvt2 zero
      (f2cl-lib:array-slice vt-%data% single-float (1 1)
                            ((1 ldvt) (1 *)) vt-%offset%)
      ldvt)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf k var-2))
  (when var-3 (setf nlp1 var-3))
  (when var-4 (setf ktemp var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldq var-7))
  (when var-9 (setf ldvt2 var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf ldvt var-12)))
(setf ktemp
        (f2cl-lib:int-add 2
                          (f2cl-lib:fref ctot-%data% (1) ((1 *))
                                         ctot-%offset%)
                          (f2cl-lib:fref ctot-%data% (2) ((1 *))
                                         ctot-%offset%)))
(if (<= ktemp ldvt2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (sgemm "N" "N" k nlp1
         (f2cl-lib:array-slice ctot-%data% f2cl-lib:integer4 (3)
                               ((1 *)) ctot-%offset%)
         one
         (f2cl-lib:array-slice q-%data% single-float (1 ktemp)
                               ((1 ldq) (1 *)) q-%offset%)
         ldq
         (f2cl-lib:array-slice vt2-%data% single-float (ktemp 1)
                               ((1 ldvt2) (1 *)) vt2-%offset%)
         ldvt2 one
         (f2cl-lib:array-slice vt-%data% single-float (1 1)
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt)
      (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
      (when var-2 (setf k var-2))
      (when var-3 (setf nlp1 var-3))
      (when var-5 (setf one var-5))
      (when var-7 (setf ldq var-7))
      (when var-9 (setf ldvt2 var-9))
      (when var-10 (setf one var-10))
      (when var-12 (setf ldvt var-12))))
(setf ktemp
        (f2cl-lib:int-add
         (f2cl-lib:fref ctot-%data% (1) ((1 *)) ctot-%offset%) 1))
```

```
 (setf nrp1 (f2cl-lib:int-add nr sqre))
 (cond
  ((> ktemp 1)
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i k) nil)
     (tagbody
       (setf (f2cl-lib:fref q-%data% (i ktemp) ((1 ldq) (1 *))
                            q-%offset%)
               (f2cl-lib:fref q-%data% (i 1) ((1 ldq) (1 *))
                              q-%offset%))
      label130))
   (f2cl-lib:fdo (i nlp2 (f2cl-lib:int-add i 1))
                 ((> i m) nil)
     (tagbody
       (setf (f2cl-lib:fref vt2-%data% (ktemp i) ((1 ldvt2) (1 *))
                            vt2-%offset%)
               (f2cl-lib:fref vt2-%data% (1 i) ((1 ldvt2) (1 *))
                              vt2-%offset%))
      label140))))
 (setf ctemp
         (f2cl-lib:int-add 1
                           (f2cl-lib:fref ctot-%data% (2) ((1 *))
                                          ctot-%offset%)
                           (f2cl-lib:fref ctot-%data% (3) ((1 *))
                                          ctot-%offset%)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
      var-11 var-12)
     (sgemm "N" "N" k nrp1 ctemp one
      (f2cl-lib:array-slice q-%data% single-float (1 ktemp)
                            ((1 ldq) (1 *)) q-%offset%)
      ldq
      (f2cl-lib:array-slice vt2-%data% single-float (ktemp nlp2)
                            ((1 ldvt2) (1 *)) vt2-%offset%)
      ldvt2 zero
      (f2cl-lib:array-slice vt-%data% single-float (1 nlp2)
                            ((1 ldvt) (1 *)) vt-%offset%)
      ldvt)
   (declare (ignore var-0 var-1 var-6 var-8 var-11))
   (when var-2 (setf k var-2))
   (when var-3 (setf nrp1 var-3))
   (when var-4 (setf ctemp var-4))
   (when var-5 (setf one var-5))
   (when var-7 (setf ldq var-7))
   (when var-9 (setf ldvt2 var-9))
   (when var-10 (setf zero var-10))
   (when var-12 (setf ldvt var-12)))
 (go end_label)
end_label
 (return
```

```
        (values nl nr nil k nil nil ldq nil nil ldu nil ldu2 nil ldvt nil
                ldvt2 nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slasd3
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::nl
                                              fortran-to-lisp::nr nil
                                              fortran-to-lisp::k nil nil
                                              fortran-to-lisp::ldq nil nil
                                              fortran-to-lisp::ldu nil
                                              fortran-to-lisp::ldu2 nil
                                              fortran-to-lisp::ldvt nil
                                              fortran-to-lisp::ldvt2 nil nil
                                              nil fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::slacpy
                                              fortran-to-lisp::slascl))))
```

90.96 slasd4

90.96.1 Axiom unit tests

— slasd4.input —

```
)set break resume
)sys rm -f slasd4.output
)spool slasd4.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.96.2 Axiom help page

— slasd4.help —

```
======================================================================
slasd4 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slasd4 (N, I, D, Z, DELTA, RHO, SIGMA, WORK, INFO)
           SLASD4 computes the square root of the i-th updated eigenvalue of a
           positive symmetric rank-one modification to a positive diagonal
           matrix. Used by sbdsdc.

Function/Subroutine Documentation
   subroutine slasd4 (integer N, integer I, real, dimension( * ) D, real,
       dimension( * ) Z, real, dimension( * ) DELTA, real RHO, real SIGMA,
       real, dimension( * ) WORK, integer INFO)
       SLASD4 computes the square root of the i-th updated eigenvalue of a
       positive symmetric rank-one modification to a positive diagonal matrix.
       Used by sbdsdc.

Purpose:
```

```
  This subroutine computes the square root of the I-th updated
  eigenvalue of a positive symmetric rank-one modification to
  a positive diagonal matrix whose entries are given as the squares
  of the corresponding entries in the array d, and that

                 0 <= D(i) < D(j)  for  i < j

  and that RHO > 0. This is arranged by the calling routine, and is
  no loss in generality.  The rank-one modified system is thus

                 diag( D ) * diag( D ) +  RHO * Z * Z_transpose.

  where we assume the Euclidean norm of Z is 1.

  The method consists of approximating the rational functions in the
  secular equation by simpler interpolating rational functions.

Parameters:

 N
   N is INTEGER
   The length of all arrays.

 I
   I is INTEGER
   The index of the eigenvalue to be computed.  1 <= I <= N.

 D
   D is REAL array, dimension ( N )
   The original eigenvalues.  It is assumed that they are in
   order, 0 <= D(I) < D(J)  for I < J.

 Z
   Z is REAL array, dimension ( N )
   The components of the updating vector.

 DELTA
   DELTA is REAL array, dimension ( N )
   If N .ne. 1, DELTA contains (D(j) - sigma_I) in its  j-th
   component.  If N = 1, then DELTA(1) = 1.  The vector DELTA
   contains the information necessary to construct the
   (singular) eigenvectors.

 RHO
   RHO is REAL
   The scalar in the symmetric updating formula.

 SIGMA
   SIGMA is REAL
   The computed sigma_I, the I-th updated eigenvalue.
```

```
 WORK
   WORK is REAL array, dimension ( N )
   If N .ne. 1, WORK contains (D(j) + sigma_I) in its  j-th
   component.  If N = 1, then WORK( 1 ) = 1.

 INFO
   INFO is INTEGER
   = 0:  successful exit
   > 0:  if INFO = 1, the updating process failed.

Internal Parameters:

   Logical variable ORGATI (origin-at-i?) is used for distinguishing
   whether D(i) or D(i+1) is treated as the origin.

                      ORGATI = .true.    origin at i
                      ORGATI = .false.   origin at i+1

   Logical variable SWTCH3 (switch-for-3-poles?) is for noting
   if we are working with THREE poles!

   MAXIT is the maximum number of iterations allowed for each
   eigenvalue.
```

———-

90.96.3 fortran code

— **slasd4.f** —

```
*  =====================================================================
      SUBROUTINE SLASD4( N, I, D, Z, DELTA, RHO, SIGMA, WORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      INTEGER            I, INFO, N
      REAL   RHO, SIGMA
*     ..
*     .. Array Arguments ..
      REAL   D( * ), DELTA( * ), WORK( * ), Z( * )
*     ..
*
```

```
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            MAXIT
      PARAMETER          ( MAXIT = 400 )
      REAL               ZERO, ONE, TWO, THREE, FOUR, EIGHT, TEN
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0, TWO = 2.0E+0,
     $                   THREE = 3.0E+0, FOUR = 4.0E+0, EIGHT = 8.0E+0,
     $                   TEN = 10.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ORGATI, SWTCH, SWTCH3, GEOMAVG
      INTEGER            II, IIM1, IIP1, IP1, ITER, J, NITER
      REAL               A, B, C, DELSQ, DELSQ2, SQ2, DPHI, DPSI, DTIIM,
     $                   DTIIP, DTIPSQ, DTISQ, DTNSQ, DTNSQ1, DW, EPS,
     $                   ERRETM, ETA, PHI, PREW, PSI, RHOINV, SGLB,
     $                   SGUB, TAU, TAU2, TEMP, TEMP1, TEMP2, W
*     ..
*     .. Local Arrays ..
      REAL               DD( 3 ), ZZ( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           SLAED6, SLASD5
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Since this routine is called in an inner loop, we do no argument
*     checking.
*
*     Quick return for N=1 and 2.
*
      INFO = 0
      IF( N.EQ.1 ) THEN
*
*        Presumably, I=1 upon entry
*
         SIGMA = SQRT( D( 1 )*D( 1 )+RHO*Z( 1 )*Z( 1 ) )
         DELTA( 1 ) = ONE
         WORK( 1 ) = ONE
         RETURN
      END IF
      IF( N.EQ.2 ) THEN
         CALL SLASD5( I, D, Z, DELTA, RHO, SIGMA, WORK )
```

```
         RETURN
      END IF
*
*     Compute machine epsilon
*
      EPS = SLAMCH( 'Epsilon' )
      RHOINV = ONE / RHO
      TAU2= ZERO
*
*     The case I = N
*
      IF( I.EQ.N ) THEN
*
*        Initialize some basic variables
*
         II = N - 1
         NITER = 1
*
*        Calculate initial guess
*
         TEMP = RHO / TWO
*
*        If ||Z||_2 is not one, then TEMP should be set to
*        RHO * ||Z||_2^2 / TWO
*
         TEMP1 = TEMP / ( D( N )+SQRT( D( N )*D( N )+TEMP ) )
         DO 10 J = 1, N
            WORK( J ) = D( J ) + D( N ) + TEMP1
            DELTA( J ) = ( D( J )-D( N ) ) - TEMP1
   10    CONTINUE
*
         PSI = ZERO
         DO 20 J = 1, N - 2
            PSI = PSI + Z( J )*Z( J ) / ( DELTA( J )*WORK( J ) )
   20    CONTINUE
*
         C = RHOINV + PSI
         W = C + Z( II )*Z( II ) / ( DELTA( II )*WORK( II ) ) +
     $       Z( N )*Z( N ) / ( DELTA( N )*WORK( N ) )
*
         IF( W.LE.ZERO ) THEN
            TEMP1 = SQRT( D( N )*D( N )+RHO )
            TEMP = Z( N-1 )*Z( N-1 ) / ( ( D( N-1 )+TEMP1 )*
     $              ( D( N )-D( N-1 )+RHO / ( D( N )+TEMP1 ) ) ) +
     $              Z( N )*Z( N ) / RHO
*
*           The following TAU2 is to approximate
*           SIGMA_n^2 - D( N )*D( N )
*
            IF( C.LE.TEMP ) THEN
```

```
               TAU = RHO
            ELSE
               DELSQ = ( D( N )-D( N-1 ) )*( D( N )+D( N-1 ) )
               A = -C*DELSQ + Z( N-1 )*Z( N-1 ) + Z( N )*Z( N )
               B = Z( N )*Z( N )*DELSQ
               IF( A.LT.ZERO ) THEN
                  TAU2 = TWO*B / ( SQRT( A*A+FOUR*B*C )-A )
               ELSE
                  TAU2 = ( A+SQRT( A*A+FOUR*B*C ) ) / ( TWO*C )
               END IF
               TAU = TAU2 / ( D( N )+SQRT( D( N )*D( N )+TAU2 ) )
            END IF
*
*           It can be proved that
*               D(N)^2+RHO/2 <= SIGMA_n^2 < D(N)^2+TAU2 <= D(N)^2+RHO
*
         ELSE
            DELSQ = ( D( N )-D( N-1 ) )*( D( N )+D( N-1 ) )
            A = -C*DELSQ + Z( N-1 )*Z( N-1 ) + Z( N )*Z( N )
            B = Z( N )*Z( N )*DELSQ
*
*           The following TAU2 is to approximate
*           SIGMA_n^2 - D( N )*D( N )
*
            IF( A.LT.ZERO ) THEN
               TAU2 = TWO*B / ( SQRT( A*A+FOUR*B*C )-A )
            ELSE
               TAU2 = ( A+SQRT( A*A+FOUR*B*C ) ) / ( TWO*C )
            END IF
            TAU = TAU2 / ( D( N )+SQRT( D( N )*D( N )+TAU2 ) )

*
*           It can be proved that
*           D(N)^2 < D(N)^2+TAU2 < SIGMA(N)^2 < D(N)^2+RHO/2
*
         END IF
*
*        The following TAU is to approximate SIGMA_n - D( N )
*
*         TAU = TAU2 / ( D( N )+SQRT( D( N )*D( N )+TAU2 ) )
*
         SIGMA = D( N ) + TAU
         DO 30 J = 1, N
            DELTA( J ) = ( D( J )-D( N ) ) - TAU
            WORK( J ) = D( J ) + D( N ) + TAU
   30    CONTINUE
*
*        Evaluate PSI and the derivative DPSI
*
         DPSI = ZERO
```

```
      PSI = ZERO
      ERRETM = ZERO
      DO 40 J = 1, II
         TEMP = Z( J ) / ( DELTA( J )*WORK( J ) )
         PSI = PSI + Z( J )*TEMP
         DPSI = DPSI + TEMP*TEMP
         ERRETM = ERRETM + PSI
 40   CONTINUE
      ERRETM = ABS( ERRETM )
*
*     Evaluate PHI and the derivative DPHI
*
      TEMP = Z( N ) / ( DELTA( N )*WORK( N ) )
      PHI = Z( N )*TEMP
      DPHI = TEMP*TEMP
      ERRETM = EIGHT*( -PHI-PSI ) + ERRETM - PHI + RHOINV
*    $         + ABS( TAU2 )*( DPSI+DPHI )
*
      W = RHOINV + PHI + PSI
*
*     Test for convergence
*
      IF( ABS( W ).LE.EPS*ERRETM ) THEN
         GO TO 240
      END IF
*
*     Calculate the new step
*
      NITER = NITER + 1
      DTNSQ1 = WORK( N-1 )*DELTA( N-1 )
      DTNSQ = WORK( N )*DELTA( N )
      C = W - DTNSQ1*DPSI - DTNSQ*DPHI
      A = ( DTNSQ+DTNSQ1 )*W - DTNSQ*DTNSQ1*( DPSI+DPHI )
      B = DTNSQ*DTNSQ1*W
      IF( C.LT.ZERO )
     $   C = ABS( C )
      IF( C.EQ.ZERO ) THEN
         ETA = RHO - SIGMA*SIGMA
      ELSE IF( A.GE.ZERO ) THEN
         ETA = ( A+SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
      ELSE
         ETA = TWO*B / ( A-SQRT( ABS( A*A-FOUR*B*C ) ) )
      END IF
*
*     Note, eta should be positive if w is negative, and
*     eta should be negative otherwise. However,
*     if for some reason caused by roundoff, eta*w > 0,
*     we simply use one Newton step instead. This way
*     will guarantee eta*w < 0.
*
```

```
         IF( W*ETA.GT.ZERO )
     $      ETA = -W / ( DPSI+DPHI )
         TEMP = ETA - DTNSQ
         IF( TEMP.GT.RHO )
     $      ETA = RHO + DTNSQ
*
         ETA = ETA / ( SIGMA+SQRT( ETA+SIGMA*SIGMA ) )
         TAU = TAU + ETA
         SIGMA = SIGMA + ETA
*
         DO 50 J = 1, N
            DELTA( J ) = DELTA( J ) - ETA
            WORK( J ) = WORK( J ) + ETA
   50    CONTINUE
*
*        Evaluate PSI and the derivative DPSI
*
         DPSI = ZERO
         PSI = ZERO
         ERRETM = ZERO
         DO 60 J = 1, II
            TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
            PSI = PSI + Z( J )*TEMP
            DPSI = DPSI + TEMP*TEMP
            ERRETM = ERRETM + PSI
   60    CONTINUE
         ERRETM = ABS( ERRETM )
*
*        Evaluate PHI and the derivative DPHI
*
         TAU2 = WORK( N )*DELTA( N )
         TEMP = Z( N ) / TAU2
         PHI = Z( N )*TEMP
         DPHI = TEMP*TEMP
         ERRETM = EIGHT*( -PHI-PSI ) + ERRETM - PHI + RHOINV
*    $          + ABS( TAU2 )*( DPSI+DPHI )
*
         W = RHOINV + PHI + PSI
*
*        Main loop to update the values of the array   DELTA
*
         ITER = NITER + 1
*
         DO 90 NITER = ITER, MAXIT
*
*           Test for convergence
*
            IF( ABS( W ).LE.EPS*ERRETM ) THEN
               GO TO 240
            END IF
```

```
*
*          Calculate the new step
*
           DTNSQ1 = WORK( N-1 )*DELTA( N-1 )
           DTNSQ = WORK( N )*DELTA( N )
           C = W - DTNSQ1*DPSI - DTNSQ*DPHI
           A = ( DTNSQ+DTNSQ1 )*W - DTNSQ1*DTNSQ*( DPSI+DPHI )
           B = DTNSQ1*DTNSQ*W
           IF( A.GE.ZERO ) THEN
              ETA = ( A+SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
           ELSE
              ETA = TWO*B / ( A-SQRT( ABS( A*A-FOUR*B*C ) ) )
           END IF
*
*          Note, eta should be positive if w is negative, and
*          eta should be negative otherwise. However,
*          if for some reason caused by roundoff, eta*w > 0,
*          we simply use one Newton step instead. This way
*          will guarantee eta*w < 0.
*
           IF( W*ETA.GT.ZERO )
     $        ETA = -W / ( DPSI+DPHI )
           TEMP = ETA - DTNSQ
           IF( TEMP.LE.ZERO )
     $        ETA = ETA / TWO
*
           ETA = ETA / ( SIGMA+SQRT( ETA+SIGMA*SIGMA ) )
           TAU = TAU + ETA
           SIGMA = SIGMA + ETA
*
           DO 70 J = 1, N
              DELTA( J ) = DELTA( J ) - ETA
              WORK( J ) = WORK( J ) + ETA
   70      CONTINUE
*
*          Evaluate PSI and the derivative DPSI
*
           DPSI = ZERO
           PSI = ZERO
           ERRETM = ZERO
           DO 80 J = 1, II
              TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
              PSI = PSI + Z( J )*TEMP
              DPSI = DPSI + TEMP*TEMP
              ERRETM = ERRETM + PSI
   80      CONTINUE
           ERRETM = ABS( ERRETM )
*
*          Evaluate PHI and the derivative DPHI
*
```

```
            TAU2 = WORK( N )*DELTA( N )
            TEMP = Z( N ) / TAU2
            PHI = Z( N )*TEMP
            DPHI = TEMP*TEMP
            ERRETM = EIGHT*( -PHI-PSI ) + ERRETM - PHI + RHOINV
*    $             + ABS( TAU2 )*( DPSI+DPHI )
*
            W = RHOINV + PHI + PSI
   90    CONTINUE
*
*        Return with INFO = 1, NITER = MAXIT and not converged
*
         INFO = 1
         GO TO 240
*
*        End for the case I = N
*
      ELSE
*
*        The case for I < N
*
         NITER = 1
         IP1 = I + 1
*
*        Calculate initial guess
*
         DELSQ = ( D( IP1 )-D( I ) )*( D( IP1 )+D( I ) )
         DELSQ2 = DELSQ / TWO
         SQ2=SQRT( ( D( I )*D( I )+D( IP1 )*D( IP1 ) ) / TWO )
         TEMP = DELSQ2 / ( D( I )+SQ2 )
         DO 100 J = 1, N
            WORK( J ) = D( J ) + D( I ) + TEMP
            DELTA( J ) = ( D( J )-D( I ) ) - TEMP
  100    CONTINUE
*
         PSI = ZERO
         DO 110 J = 1, I - 1
            PSI = PSI + Z( J )*Z( J ) / ( WORK( J )*DELTA( J ) )
  110    CONTINUE
*
         PHI = ZERO
         DO 120 J = N, I + 2, -1
            PHI = PHI + Z( J )*Z( J ) / ( WORK( J )*DELTA( J ) )
  120    CONTINUE
         C = RHOINV + PSI + PHI
         W = C + Z( I )*Z( I ) / ( WORK( I )*DELTA( I ) ) +
     $       Z( IP1 )*Z( IP1 ) / ( WORK( IP1 )*DELTA( IP1 ) )
*
         GEOMAVG = .FALSE.
         IF( W.GT.ZERO ) THEN
```

```
*
*          d(i)^2 < the ith sigma^2 < (d(i)^2+d(i+1)^2)/2
*
*          We choose d(i) as origin.
*
           ORGATI = .TRUE.
           II = I
           SGLB = ZERO
           SGUB = DELSQ2  / ( D( I )+SQ2 )
           A = C*DELSQ + Z( I )*Z( I ) + Z( IP1 )*Z( IP1 )
           B = Z( I )*Z( I )*DELSQ
           IF( A.GT.ZERO ) THEN
              TAU2 = TWO*B / ( A+SQRT( ABS( A*A-FOUR*B*C ) ) )
           ELSE
              TAU2 = ( A-SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
           END IF
*
*          TAU2 now is an estimation of SIGMA^2 - D( I )^2. The
*          following, however, is the corresponding estimation of
*          SIGMA - D( I ).
*
           TAU = TAU2 / ( D( I )+SQRT( D( I )*D( I )+TAU2 ) )
           TEMP = SQRT(EPS)
           IF( (D(I).LE.TEMP*D(IP1)).AND.(ABS(Z(I)).LE.TEMP)
     $                                 .AND.(D(I).GT.ZERO) ) THEN
              TAU = MIN( TEN*D(I), SGUB )
              GEOMAVG = .TRUE.
           END IF
        ELSE
*
*          (d(i)^2+d(i+1)^2)/2 <= the ith sigma^2 < d(i+1)^2/2
*
*          We choose d(i+1) as origin.
*
           ORGATI = .FALSE.
           II = IP1
           SGLB = -DELSQ2  / ( D( II )+SQ2 )
           SGUB = ZERO
           A = C*DELSQ - Z( I )*Z( I ) - Z( IP1 )*Z( IP1 )
           B = Z( IP1 )*Z( IP1 )*DELSQ
           IF( A.LT.ZERO ) THEN
              TAU2 = TWO*B / ( A-SQRT( ABS( A*A+FOUR*B*C ) ) )
           ELSE
              TAU2 = -( A+SQRT( ABS( A*A+FOUR*B*C ) ) ) / ( TWO*C )
           END IF
*
*          TAU2 now is an estimation of SIGMA^2 - D( IP1 )^2. The
*          following, however, is the corresponding estimation of
*          SIGMA - D( IP1 ).
*
```

```
            TAU = TAU2 / ( D( IP1 )+SQRT( ABS( D( IP1 )*D( IP1 )+
     $            TAU2 ) ) )
         END IF
*
         SIGMA = D( II ) + TAU
         DO 130 J = 1, N
            WORK( J ) = D( J ) + D( II ) + TAU
            DELTA( J ) = ( D( J )-D( II ) ) - TAU
  130    CONTINUE
         IIM1 = II - 1
         IIP1 = II + 1
*
*        Evaluate PSI and the derivative DPSI
*
         DPSI = ZERO
         PSI = ZERO
         ERRETM = ZERO
         DO 150 J = 1, IIM1
            TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
            PSI = PSI + Z( J )*TEMP
            DPSI = DPSI + TEMP*TEMP
            ERRETM = ERRETM + PSI
  150    CONTINUE
         ERRETM = ABS( ERRETM )
*
*        Evaluate PHI and the derivative DPHI
*
         DPHI = ZERO
         PHI = ZERO
         DO 160 J = N, IIP1, -1
            TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
            PHI = PHI + Z( J )*TEMP
            DPHI = DPHI + TEMP*TEMP
            ERRETM = ERRETM + PHI
  160    CONTINUE
*
         W = RHOINV + PHI + PSI
*
*        W is the value of the secular function with
*        its ii-th element removed.
*
         SWTCH3 = .FALSE.
         IF( ORGATI ) THEN
            IF( W.LT.ZERO )
     $         SWTCH3 = .TRUE.
         ELSE
            IF( W.GT.ZERO )
     $         SWTCH3 = .TRUE.
         END IF
         IF( II.EQ.1 .OR. II.EQ.N )
```

```
     $      SWTCH3 = .FALSE.
*
         TEMP = Z( II ) / ( WORK( II )*DELTA( II ) )
         DW = DPSI + DPHI + TEMP*TEMP
         TEMP = Z( II )*TEMP
         W = W + TEMP
         ERRETM = EIGHT*( PHI-PSI ) + ERRETM + TWO*RHOINV
     $          + THREE*ABS( TEMP )
*    $          + ABS( TAU2 )*DW
*
*        Test for convergence
*
         IF( ABS( W ).LE.EPS*ERRETM ) THEN
            GO TO 240
         END IF
*
         IF( W.LE.ZERO ) THEN
            SGLB = MAX( SGLB, TAU )
         ELSE
            SGUB = MIN( SGUB, TAU )
         END IF
*
*        Calculate the new step
*
         NITER = NITER + 1
         IF( .NOT.SWTCH3 ) THEN
            DTIPSQ = WORK( IP1 )*DELTA( IP1 )
            DTISQ = WORK( I )*DELTA( I )
            IF( ORGATI ) THEN
               C = W - DTIPSQ*DW + DELSQ*( Z( I ) / DTISQ )**2
            ELSE
               C = W - DTISQ*DW - DELSQ*( Z( IP1 ) / DTIPSQ )**2
            END IF
            A = ( DTIPSQ+DTISQ )*W - DTIPSQ*DTISQ*DW
            B = DTIPSQ*DTISQ*W
            IF( C.EQ.ZERO ) THEN
               IF( A.EQ.ZERO ) THEN
                  IF( ORGATI ) THEN
                     A = Z( I )*Z( I ) + DTIPSQ*DTIPSQ*( DPSI+DPHI )
                  ELSE
                     A = Z( IP1 )*Z( IP1 ) + DTISQ*DTISQ*( DPSI+DPHI )
                  END IF
               END IF
               ETA = B / A
            ELSE IF( A.LE.ZERO ) THEN
               ETA = ( A-SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
            ELSE
               ETA = TWO*B / ( A+SQRT( ABS( A*A-FOUR*B*C ) ) )
            END IF
         ELSE
```

```
*
*          Interpolation using THREE most relevant poles
*
           DTIIM = WORK( IIM1 )*DELTA( IIM1 )
           DTIIP = WORK( IIP1 )*DELTA( IIP1 )
           TEMP = RHOINV + PSI + PHI
           IF( ORGATI ) THEN
              TEMP1 = Z( IIM1 ) / DTIIM
              TEMP1 = TEMP1*TEMP1
              C = ( TEMP - DTIIP*( DPSI+DPHI ) ) -
     $            ( D( IIM1 )-D( IIP1 ) )*( D( IIM1 )+D( IIP1 ) )*TEMP1
              ZZ( 1 ) = Z( IIM1 )*Z( IIM1 )
              IF( DPSI.LT.TEMP1 ) THEN
                 ZZ( 3 ) = DTIIP*DTIIP*DPHI
              ELSE
                 ZZ( 3 ) = DTIIP*DTIIP*( ( DPSI-TEMP1 )+DPHI )
              END IF
           ELSE
              TEMP1 = Z( IIP1 ) / DTIIP
              TEMP1 = TEMP1*TEMP1
              C = ( TEMP - DTIIM*( DPSI+DPHI ) ) -
     $            ( D( IIP1 )-D( IIM1 ) )*( D( IIM1 )+D( IIP1 ) )*TEMP1
              IF( DPHI.LT.TEMP1 ) THEN
                 ZZ( 1 ) = DTIIM*DTIIM*DPSI
              ELSE
                 ZZ( 1 ) = DTIIM*DTIIM*( DPSI+( DPHI-TEMP1 ) )
              END IF
              ZZ( 3 ) = Z( IIP1 )*Z( IIP1 )
           END IF
           ZZ( 2 ) = Z( II )*Z( II )
           DD( 1 ) = DTIIM
           DD( 2 ) = DELTA( II )*WORK( II )
           DD( 3 ) = DTIIP
           CALL SLAED6( NITER, ORGATI, C, DD, ZZ, W, ETA, INFO )
*
           IF( INFO.NE.0 ) THEN
*
*             If INFO is not 0, i.e., SLAED6 failed, switch back
*             to 2 pole interpolation.
*
              SWTCH3 = .FALSE.
              INFO = 0
              DTIPSQ = WORK( IP1 )*DELTA( IP1 )
              DTISQ = WORK( I )*DELTA( I )
              IF( ORGATI ) THEN
                 C = W - DTIPSQ*DW + DELSQ*( Z( I ) / DTISQ )**2
              ELSE
                 C = W - DTISQ*DW - DELSQ*( Z( IP1 ) / DTIPSQ )**2
              END IF
              A = ( DTIPSQ+DTISQ )*W - DTIPSQ*DTISQ*DW
```

```
               B = DTIPSQ*DTISQ*W
               IF( C.EQ.ZERO ) THEN
                  IF( A.EQ.ZERO ) THEN
                     IF( ORGATI ) THEN
                        A = Z( I )*Z( I ) + DTIPSQ*DTIPSQ*( DPSI+DPHI )
                     ELSE
                        A = Z( IP1 )*Z( IP1 ) + DTISQ*DTISQ*( DPSI+DPHI)
                     END IF
                  END IF
                  ETA = B / A
               ELSE IF( A.LE.ZERO ) THEN
                  ETA = ( A-SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
               ELSE
                  ETA = TWO*B / ( A+SQRT( ABS( A*A-FOUR*B*C ) ) )
               END IF
            END IF
         END IF
*
*        Note, eta should be positive if w is negative, and
*        eta should be negative otherwise. However,
*        if for some reason caused by roundoff, eta*w > 0,
*        we simply use one Newton step instead. This way
*        will guarantee eta*w < 0.
*
         IF( W*ETA.GE.ZERO )
     $      ETA = -W / DW
*
         ETA = ETA / ( SIGMA+SQRT( SIGMA*SIGMA+ETA ) )
         TEMP = TAU + ETA
         IF( TEMP.GT.SGUB .OR. TEMP.LT.SGLB ) THEN
            IF( W.LT.ZERO ) THEN
               ETA = ( SGUB-TAU ) / TWO
            ELSE
               ETA = ( SGLB-TAU ) / TWO
            END IF
            IF( GEOMAVG ) THEN
               IF( W .LT. ZERO ) THEN
                  IF( TAU .GT. ZERO ) THEN
                     ETA = SQRT(SGUB*TAU)-TAU
                  END IF
               ELSE
                  IF( SGLB .GT. ZERO ) THEN
                     ETA = SQRT(SGLB*TAU)-TAU
                  END IF
               END IF
            END IF
         END IF
*
         PREW = W
*
```

```
         TAU = TAU + ETA
         SIGMA = SIGMA + ETA
*
         DO 170 J = 1, N
            WORK( J ) = WORK( J ) + ETA
            DELTA( J ) = DELTA( J ) - ETA
  170    CONTINUE
*
*        Evaluate PSI and the derivative DPSI
*
         DPSI = ZERO
         PSI = ZERO
         ERRETM = ZERO
         DO 180 J = 1, IIM1
            TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
            PSI = PSI + Z( J )*TEMP
            DPSI = DPSI + TEMP*TEMP
            ERRETM = ERRETM + PSI
  180    CONTINUE
         ERRETM = ABS( ERRETM )
*
*        Evaluate PHI and the derivative DPHI
*
         DPHI = ZERO
         PHI = ZERO
         DO 190 J = N, IIP1, -1
            TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
            PHI = PHI + Z( J )*TEMP
            DPHI = DPHI + TEMP*TEMP
            ERRETM = ERRETM + PHI
  190    CONTINUE
*
         TAU2 = WORK( II )*DELTA( II )
         TEMP = Z( II ) / TAU2
         DW = DPSI + DPHI + TEMP*TEMP
         TEMP = Z( II )*TEMP
         W = RHOINV + PHI + PSI + TEMP
         ERRETM = EIGHT*( PHI-PSI ) + ERRETM + TWO*RHOINV
     $          + THREE*ABS( TEMP )
*    $          + ABS( TAU2 )*DW
*
         SWTCH = .FALSE.
         IF( ORGATI ) THEN
            IF( -W.GT.ABS( PREW ) / TEN )
     $         SWTCH = .TRUE.
         ELSE
            IF( W.GT.ABS( PREW ) / TEN )
     $         SWTCH = .TRUE.
         END IF
*
```

```
*        Main loop to update the values of the array   DELTA and WORK
*
         ITER = NITER + 1
*
         DO 230 NITER = ITER, MAXIT
*
*           Test for convergence
*
            IF( ABS( W ).LE.EPS*ERRETM ) THEN
*     $          .OR. (SGUB-SGLB).LE.EIGHT*ABS(SGUB+SGLB) ) THEN
               GO TO 240
            END IF
*
            IF( W.LE.ZERO ) THEN
               SGLB = MAX( SGLB, TAU )
            ELSE
               SGUB = MIN( SGUB, TAU )
            END IF
*
*           Calculate the new step
*
            IF( .NOT.SWTCH3 ) THEN
               DTIPSQ = WORK( IP1 )*DELTA( IP1 )
               DTISQ = WORK( I )*DELTA( I )
               IF( .NOT.SWTCH ) THEN
                  IF( ORGATI ) THEN
                     C = W - DTIPSQ*DW + DELSQ*( Z( I ) / DTISQ )**2
                  ELSE
                     C = W - DTISQ*DW - DELSQ*( Z( IP1 ) / DTIPSQ )**2
                  END IF
               ELSE
                  TEMP = Z( II ) / ( WORK( II )*DELTA( II ) )
                  IF( ORGATI ) THEN
                     DPSI = DPSI + TEMP*TEMP
                  ELSE
                     DPHI = DPHI + TEMP*TEMP
                  END IF
                  C = W - DTISQ*DPSI - DTIPSQ*DPHI
               END IF
               A = ( DTIPSQ+DTISQ )*W - DTIPSQ*DTISQ*DW
               B = DTIPSQ*DTISQ*W
               IF( C.EQ.ZERO ) THEN
                  IF( A.EQ.ZERO ) THEN
                     IF( .NOT.SWTCH ) THEN
                        IF( ORGATI ) THEN
                           A = Z( I )*Z( I ) + DTIPSQ*DTIPSQ*
     $                         ( DPSI+DPHI )
                        ELSE
                           A = Z( IP1 )*Z( IP1 ) +
     $                         DTISQ*DTISQ*( DPSI+DPHI )
```

```
                  END IF
               ELSE
                  A = DTISQ*DTISQ*DPSI + DTIPSQ*DTIPSQ*DPHI
               END IF
            END IF
            ETA = B / A
         ELSE IF( A.LE.ZERO ) THEN
            ETA = ( A-SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
         ELSE
            ETA = TWO*B / ( A+SQRT( ABS( A*A-FOUR*B*C ) ) )
         END IF
      ELSE
*
*        Interpolation using THREE most relevant poles
*
         DTIIM = WORK( IIM1 )*DELTA( IIM1 )
         DTIIP = WORK( IIP1 )*DELTA( IIP1 )
         TEMP = RHOINV + PSI + PHI
         IF( SWTCH ) THEN
            C = TEMP - DTIIM*DPSI - DTIIP*DPHI
            ZZ( 1 ) = DTIIM*DTIIM*DPSI
            ZZ( 3 ) = DTIIP*DTIIP*DPHI
         ELSE
            IF( ORGATI ) THEN
               TEMP1 = Z( IIM1 ) / DTIIM
               TEMP1 = TEMP1*TEMP1
               TEMP2 = ( D( IIM1 )-D( IIP1 ) )*
     $                 ( D( IIM1 )+D( IIP1 ) )*TEMP1
               C = TEMP - DTIIP*( DPSI+DPHI ) - TEMP2
               ZZ( 1 ) = Z( IIM1 )*Z( IIM1 )
               IF( DPSI.LT.TEMP1 ) THEN
                  ZZ( 3 ) = DTIIP*DTIIP*DPHI
               ELSE
                  ZZ( 3 ) = DTIIP*DTIIP*( ( DPSI-TEMP1 )+DPHI )
               END IF
            ELSE
               TEMP1 = Z( IIP1 ) / DTIIP
               TEMP1 = TEMP1*TEMP1
               TEMP2 = ( D( IIP1 )-D( IIM1 ) )*
     $                 ( D( IIM1 )+D( IIP1 ) )*TEMP1
               C = TEMP - DTIIM*( DPSI+DPHI ) - TEMP2
               IF( DPHI.LT.TEMP1 ) THEN
                  ZZ( 1 ) = DTIIM*DTIIM*DPSI
               ELSE
                  ZZ( 1 ) = DTIIM*DTIIM*( DPSI+( DPHI-TEMP1 ) )
               END IF
               ZZ( 3 ) = Z( IIP1 )*Z( IIP1 )
            END IF
         END IF
         DD( 1 ) = DTIIM
```

```
            DD( 2 ) = DELTA( II )*WORK( II )
            DD( 3 ) = DTIIP
            CALL SLAED6( NITER, ORGATI, C, DD, ZZ, W, ETA, INFO )
*
            IF( INFO.NE.0 ) THEN
*
*              If INFO is not 0, i.e., SLAED6 failed, switch
*              back to two pole interpolation
*
               SWTCH3 = .FALSE.
               INFO = 0
               DTIPSQ = WORK( IP1 )*DELTA( IP1 )
               DTISQ = WORK( I )*DELTA( I )
               IF( .NOT.SWTCH ) THEN
                  IF( ORGATI ) THEN
                     C = W - DTIPSQ*DW + DELSQ*( Z( I )/DTISQ )**2
                  ELSE
                     C = W - DTISQ*DW - DELSQ*( Z( IP1 )/DTIPSQ )**2
                  END IF
               ELSE
                  TEMP = Z( II ) / ( WORK( II )*DELTA( II ) )
                  IF( ORGATI ) THEN
                     DPSI = DPSI + TEMP*TEMP
                  ELSE
                     DPHI = DPHI + TEMP*TEMP
                  END IF
                  C = W - DTISQ*DPSI - DTIPSQ*DPHI
               END IF
               A = ( DTIPSQ+DTISQ )*W - DTIPSQ*DTISQ*DW
               B = DTIPSQ*DTISQ*W
               IF( C.EQ.ZERO ) THEN
                  IF( A.EQ.ZERO ) THEN
                     IF( .NOT.SWTCH ) THEN
                        IF( ORGATI ) THEN
                           A = Z( I )*Z( I ) + DTIPSQ*DTIPSQ*
     $                         ( DPSI+DPHI )
                        ELSE
                           A = Z( IP1 )*Z( IP1 ) +
     $                         DTISQ*DTISQ*( DPSI+DPHI )
                        END IF
                     ELSE
                        A = DTISQ*DTISQ*DPSI + DTIPSQ*DTIPSQ*DPHI
                     END IF
                  END IF
                  ETA = B / A
               ELSE IF( A.LE.ZERO ) THEN
                  ETA = ( A-SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
               ELSE
                  ETA = TWO*B / ( A+SQRT( ABS( A*A-FOUR*B*C ) ) )
               END IF
```

```
               END IF
            END IF
*
*           Note, eta should be positive if w is negative, and
*           eta should be negative otherwise. However,
*           if for some reason caused by roundoff, eta*w > 0,
*           we simply use one Newton step instead. This way
*           will guarantee eta*w < 0.
*
            IF( W*ETA.GE.ZERO )
     $         ETA = -W / DW
*
            ETA = ETA / ( SIGMA+SQRT( SIGMA*SIGMA+ETA ) )
            TEMP=TAU+ETA
            IF( TEMP.GT.SGUB .OR. TEMP.LT.SGLB ) THEN
               IF( W.LT.ZERO ) THEN
                  ETA = ( SGUB-TAU ) / TWO
               ELSE
                  ETA = ( SGLB-TAU ) / TWO
               END IF
               IF( GEOMAVG ) THEN
                  IF( W .LT. ZERO ) THEN
                     IF( TAU .GT. ZERO ) THEN
                        ETA = SQRT(SGUB*TAU)-TAU
                     END IF
                  ELSE
                     IF( SGLB .GT. ZERO ) THEN
                        ETA = SQRT(SGLB*TAU)-TAU
                     END IF
                  END IF
               END IF
            END IF
*
            PREW = W
*
            TAU = TAU + ETA
            SIGMA = SIGMA + ETA
*
            DO 200 J = 1, N
               WORK( J ) = WORK( J ) + ETA
               DELTA( J ) = DELTA( J ) - ETA
  200       CONTINUE
*
*           Evaluate PSI and the derivative DPSI
*
            DPSI = ZERO
            PSI = ZERO
            ERRETM = ZERO
            DO 210 J = 1, IIM1
               TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
```

```
               PSI = PSI + Z( J )*TEMP
               DPSI = DPSI + TEMP*TEMP
               ERRETM = ERRETM + PSI
  210       CONTINUE
            ERRETM = ABS( ERRETM )
*
*           Evaluate PHI and the derivative DPHI
*
            DPHI = ZERO
            PHI = ZERO
            DO 220 J = N, IIP1, -1
               TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
               PHI = PHI + Z( J )*TEMP
               DPHI = DPHI + TEMP*TEMP
               ERRETM = ERRETM + PHI
  220       CONTINUE
*
            TAU2 = WORK( II )*DELTA( II )
            TEMP = Z( II ) / TAU2
            DW = DPSI + DPHI + TEMP*TEMP
            TEMP = Z( II )*TEMP
            W = RHOINV + PHI + PSI + TEMP
            ERRETM = EIGHT*( PHI-PSI ) + ERRETM + TWO*RHOINV
     $             + THREE*ABS( TEMP )
*    $             + ABS( TAU2 )*DW
*
            IF( W*PREW.GT.ZERO .AND. ABS( W ).GT.ABS( PREW ) / TEN )
     $         SWTCH = .NOT.SWTCH
*
  230    CONTINUE
*
*        Return with INFO = 1, NITER = MAXIT and not converged
*
         INFO = 1
*
      END IF
*
  240 CONTINUE
      RETURN
*
*     End of SLASD4
*
      END
```

———-

90.96.4 lisp code

— slasd4.lisp —

```
(let* ((maxit 400)
       (zero 0.0)
       (one 1.0)
       (two 2.0)
       (three 3.0)
       (four 4.0)
       (eight 8.0)
       (ten 10.0))
  (declare (type (f2cl-lib:integer4 400 400) maxit)
           (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 3.0 3.0) three)
           (type (single-float 4.0 4.0) four)
           (type (single-float 8.0 8.0) eight)
           (type (single-float 10.0 10.0) ten)
           (ignorable maxit zero one two three four eight ten))
  (defun slasd4 (n i d z delta rho sigma work info)
    (declare (type (single-float) sigma rho)
             (type (array single-float (*)) work delta z d)
             (type (f2cl-lib:integer4) info i n))
    (f2cl-lib:with-multi-array-data
        ((d single-float d-%data% d-%offset%)
         (z single-float z-%data% z-%offset%)
         (delta single-float delta-%data% delta-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((dd (make-array 3 :element-type 'single-float))
             (zz (make-array 3 :element-type 'single-float)) (a 0.0) (b 0.0)
             (c 0.0) (delsq 0.0) (delsq2 0.0) (sq2 0.0) (dphi 0.0) (dpsi 0.0)
             (dtiim 0.0) (dtiip 0.0) (dtipsq 0.0) (dtisq 0.0) (dtnsq 0.0)
             (dtnsq1 0.0) (dw 0.0) (eps 0.0) (erretm 0.0) (eta 0.0) (phi 0.0)
             (prew 0.0) (psi 0.0) (rhoinv 0.0) (sglb 0.0) (sgub 0.0) (tau 0.0)
             (tau2 0.0) (temp 0.0) (temp1 0.0) (temp2 0.0) (w 0.0) (ii 0)
             (iim1 0) (iip1 0) (ip1 0) (iter 0) (j 0) (niter 0) (orgati nil)
             (swtch nil) (swtch3 nil) (geomavg nil))
        (declare (type (array single-float (3)) dd zz)
                 (type (single-float) a b c delsq delsq2 sq2 dphi dpsi dtiim
                  dtiip dtipsq dtisq dtnsq dtnsq1 dw eps erretm eta phi prew
                  psi rhoinv sglb sgub tau tau2 temp temp1 temp2 w)
                 (type (f2cl-lib:integer4) ii iim1 iip1 ip1 iter j niter)
                 (type f2cl-lib:logical orgati swtch swtch3 geomavg))
        (setf info 0)
        (cond
         ((= n 1)
          (setf sigma
```

```
            (f2cl-lib:fsqrt
             (+
              (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
                 (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
              (* rho (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)
                 (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)))))
  (setf (f2cl-lib:fref delta-%data% (1) ((1 *)) delta-%offset%) one)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) one)
  (go end_label)))
(cond
 ((= n 2)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slasd5 i d z delta rho sigma work)
    (declare (ignore var-1 var-2 var-3 var-6))
    (when var-0 (setf i var-0))
    (when var-4 (setf rho var-4))
    (when var-5 (setf sigma var-5)))
  (go end_label)))
(setf eps (slamch "Epsilon"))
(setf rhoinv (/ one rho))
(setf tau2 zero)
(cond
 ((= i n) (setf ii (f2cl-lib:int-sub n 1)) (setf niter 1)
  (setf temp (/ rho two))
  (setf temp1
          (/ temp
             (+ (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                (f2cl-lib:fsqrt
                 (+
                  (* (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                     (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%))
                  temp)))))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
              (+ (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                 (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                 temp1))
      (setf (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
              (- (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                 (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                 temp1))
     label10))
  (setf psi zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 2))) nil)
    (tagbody
      (setf psi
              (+ psi
```

```
                (/
                 (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                    (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%))
                 (*
                  (f2cl-lib:fref delta-%data% (j) ((1 *))
                                 delta-%offset%)
                  (f2cl-lib:fref work-%data% (j) ((1 *))
                                 work-%offset%)))))
   label20))
(setf c (+ rhoinv psi))
(setf w
        (+ c
           (/
            (* (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%)
               (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%))
            (*
             (f2cl-lib:fref delta-%data% (ii) ((1 *)) delta-%offset%)
             (f2cl-lib:fref work-%data% (ii) ((1 *)) work-%offset%)))
           (/
            (* (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%)
               (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%))
            (*
             (f2cl-lib:fref delta-%data% (n) ((1 *)) delta-%offset%)
             (f2cl-lib:fref work-%data% (n) ((1 *))
                            work-%offset%)))))
(cond
 ((<= w zero)
  (setf temp1
          (f2cl-lib:fsqrt
           (+
            (* (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
               (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%))
            rho)))
  (setf temp
          (+
           (/
            (*
             (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub n 1)) ((1 *))
                            z-%offset%)
             (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub n 1)) ((1 *))
                            z-%offset%))
            (*
             (+
              (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                              ((1 *)) d-%offset%)
              temp1)
             (+
              (- (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                ((1 *)) d-%offset%))
```

```
                    (/ rho
                       (+ (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                          temp1)))))
                 (/
                  (* (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%)
                     (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%))
                  rho)))
          (cond ((<= c temp) (setf tau rho))
                (t
                 (setf delsq
                         (*
                          (- (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                             (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                            ((1 *)) d-%offset%))
                          (+ (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                             (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                            ((1 *)) d-%offset%))))
                 (setf a
                         (+ (* (- c) delsq)
                            (*
                             (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub n 1))
                                            ((1 *)) z-%offset%)
                             (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub n 1))
                                            ((1 *)) z-%offset%))
                            (*
                             (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%)
                             (f2cl-lib:fref z-%data% (n) ((1 *))
                                            z-%offset%))))
                 (setf b
                         (* (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%)
                            (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%)
                            delsq))
                 (cond
                  ((< a zero)
                   (setf tau2
                           (/ (* two b)
                              (- (f2cl-lib:fsqrt (+ (* a a) (* four b c)))
                                 a))))
                  (t
                   (setf tau2
                           (/ (+ a (f2cl-lib:fsqrt (+ (* a a) (* four b c))))
                              (* two c)))))
                 (setf tau
                         (/ tau2
                            (+
                             (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                             (f2cl-lib:fsqrt
                              (+
                               (*
                                (f2cl-lib:fref d-%data% (n) ((1 *))
```

```
                                        d-%offset%)
                          (f2cl-lib:fref d-%data% (n) ((1 *))
                                        d-%offset%))
                        tau2))))))))
      (t
       (setf delsq
               (*
                (- (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                   (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                  ((1 *)) d-%offset%))
                (+ (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                   (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                  ((1 *)) d-%offset%))))
       (setf a
               (+ (* (- c) delsq)
                  (*
                   (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub n 1))
                                  ((1 *)) z-%offset%)
                   (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub n 1))
                                  ((1 *)) z-%offset%))
                  (* (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%)
                     (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%))))
       (setf b
               (* (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%)
                  (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%) delsq))
       (cond
        ((< a zero)
         (setf tau2
                 (/ (* two b)
                    (- (f2cl-lib:fsqrt (+ (* a a) (* four b c))) a))))
        (t
         (setf tau2
                 (/ (+ a (f2cl-lib:fsqrt (+ (* a a) (* four b c))))
                    (* two c)))))
       (setf tau
               (/ tau2
                  (+ (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                     (f2cl-lib:fsqrt
                      (+
                       (* (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                          (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%))
                       tau2)))))))
    (setf sigma (+ (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) tau))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
                (- (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                   (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) tau))
        (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
```

```
            (+ (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
               (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) tau))
   label30))
(setf dpsi zero) (setf psi zero) (setf erretm zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j ii) nil)
  (tagbody
    (setf temp
            (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
               (*
                (f2cl-lib:fref delta-%data% (j) ((1 *))
                               delta-%offset%)
                (f2cl-lib:fref work-%data% (j) ((1 *))
                               work-%offset%))))
    (setf psi
            (+ psi
               (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                  temp)))
    (setf dpsi (+ dpsi (* temp temp)))
    (setf erretm (+ erretm psi))
   label40))
(setf erretm (abs erretm))
(setf temp
        (/ (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%)
           (* (f2cl-lib:fref delta-%data% (n) ((1 *)) delta-%offset%)
              (f2cl-lib:fref work-%data% (n) ((1 *))
                             work-%offset%))))
(setf phi (* (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%) temp))
(setf dphi (* temp temp))
(setf erretm (+ (- (+ (* eight (- (- psi) phi)) erretm) phi) rhoinv))
(setf w (+ rhoinv phi psi))
(cond ((<= (abs w) (* eps erretm)) (go label240)))
(setf niter (f2cl-lib:int-add niter 1))
(setf dtnsq1
        (*
         (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub n 1)) ((1 *))
                        work-%offset%)
         (f2cl-lib:fref delta-%data% ((f2cl-lib:int-sub n 1)) ((1 *))
                        delta-%offset%)))
(setf dtnsq
        (* (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
           (f2cl-lib:fref delta-%data% (n) ((1 *)) delta-%offset%)))
(setf c (- w (* dtnsq1 dpsi) (* dtnsq dphi)))
(setf a
        (+ (* (+ dtnsq dtnsq1) w)
           (* (- dtnsq) dtnsq1 (+ dpsi dphi))))
(setf b (* dtnsq dtnsq1 w))
(if (< c zero)
    (setf c (abs c)))
(cond ((= c zero) (setf eta (- rho (* sigma sigma))))
```

```
      ((>= a zero)
       (setf eta
               (/
                (+ a
                   (f2cl-lib:fsqrt
                    (abs (+ (* a a) (* (- four) b c)))))
                (* two c))))
      (t
       (setf eta
               (/ (* two b)
                  (- a
                     (f2cl-lib:fsqrt
                      (abs (+ (* a a) (* (- four) b c)))))))))
(if (> (* w eta) zero)
    (setf eta (/ (- w) (+ dpsi dphi))))
(setf temp (- eta dtnsq))
(if (> temp rho)
    (setf eta (+ rho dtnsq)))
(setf eta (/ eta (+ sigma (f2cl-lib:fsqrt (+ eta (* sigma sigma))))))
(setf tau (+ tau eta)) (setf sigma (+ sigma eta))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
            (-
             (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
             eta))
    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
            (+ (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
               eta))
   label50))
(setf dpsi zero) (setf psi zero) (setf erretm zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j ii) nil)
  (tagbody
    (setf temp
            (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
               (*
                (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                (f2cl-lib:fref delta-%data% (j) ((1 *))
                               delta-%offset%))))
    (setf psi
            (+ psi
               (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                  temp)))
    (setf dpsi (+ dpsi (* temp temp)))
    (setf erretm (+ erretm psi))
   label60))
(setf erretm (abs erretm))
(setf tau2
```

```
        (* (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
           (f2cl-lib:fref delta-%data% (n) ((1 *)) delta-%offset%)))
(setf temp (/ (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%) tau2))
(setf phi (* (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%) temp))
(setf dphi (* temp temp))
(setf erretm (+ (- (+ (* eight (- (- psi) phi)) erretm) phi) rhoinv))
(setf w (+ rhoinv phi psi)) (setf iter (f2cl-lib:int-add niter 1))
(f2cl-lib:fdo (niter iter (f2cl-lib:int-add niter 1))
              ((> niter maxit) nil)
  (tagbody
    (cond ((<= (abs w) (* eps erretm)) (go label240)))
    (setf dtnsq1
            (*
             (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub n 1))
                            ((1 *)) work-%offset%)
             (f2cl-lib:fref delta-%data% ((f2cl-lib:int-sub n 1))
                            ((1 *)) delta-%offset%)))
    (setf dtnsq
            (* (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
               (f2cl-lib:fref delta-%data% (n) ((1 *))
                              delta-%offset%)))
    (setf c (- w (* dtnsq1 dpsi) (* dtnsq dphi)))
    (setf a
            (+ (* (+ dtnsq dtnsq1) w)
               (* (- dtnsq1) dtnsq (+ dpsi dphi))))
    (setf b (* dtnsq1 dtnsq w))
    (cond
     ((>= a zero)
      (setf eta
              (/
               (+ a
                  (f2cl-lib:fsqrt
                   (abs (+ (* a a) (* (- four) b c)))))
               (* two c))))
     (t
      (setf eta
              (/ (* two b)
                 (- a
                    (f2cl-lib:fsqrt
                     (abs (+ (* a a) (* (- four) b c))))))))
    (if (> (* w eta) zero)
        (setf eta (/ (- w) (+ dpsi dphi))))
    (setf temp (- eta dtnsq))
    (if (<= temp zero)
        (setf eta (/ eta two)))
    (setf eta
            (/ eta
               (+ sigma (f2cl-lib:fsqrt (+ eta (* sigma sigma))))))
    (setf tau (+ tau eta))
    (setf sigma (+ sigma eta))
```

```
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (setf (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
                  (-
                   (f2cl-lib:fref delta-%data% (j) ((1 *))
                                  delta-%offset%)
                   eta))
          (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                  (+
                   (f2cl-lib:fref work-%data% (j) ((1 *))
                                  work-%offset%)
                   eta))
         label70))
      (setf dpsi zero)
      (setf psi zero)
      (setf erretm zero)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j ii) nil)
        (tagbody
          (setf temp
                  (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                     (*
                      (f2cl-lib:fref work-%data% (j) ((1 *))
                                     work-%offset%)
                      (f2cl-lib:fref delta-%data% (j) ((1 *))
                                     delta-%offset%))))
          (setf psi
                  (+ psi
                     (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                        temp)))
          (setf dpsi (+ dpsi (* temp temp)))
          (setf erretm (+ erretm psi))
         label80))
      (setf erretm (abs erretm))
      (setf tau2
              (* (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
                 (f2cl-lib:fref delta-%data% (n) ((1 *))
                                delta-%offset%)))
      (setf temp
              (/ (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%) tau2))
      (setf phi
              (* (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%) temp))
      (setf dphi (* temp temp))
      (setf erretm
              (+ (- (+ (* eight (- (- psi) phi)) erretm) phi) rhoinv))
      (setf w (+ rhoinv phi psi))
     label90))
  (setf info 1) (go label240))
 (t (setf niter 1) (setf ip1 (f2cl-lib:int-add i 1))
```

```
(setf delsq
        (*
         (- (f2cl-lib:fref d-%data% (ip1) ((1 *)) d-%offset%)
            (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
         (+ (f2cl-lib:fref d-%data% (ip1) ((1 *)) d-%offset%)
            (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))))
(setf delsq2 (/ delsq two))
(setf sq2
        (f2cl-lib:fsqrt
         (/
          (+
           (* (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
              (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
           (* (f2cl-lib:fref d-%data% (ip1) ((1 *)) d-%offset%)
              (f2cl-lib:fref d-%data% (ip1) ((1 *)) d-%offset%)))
          two)))
(setf temp
        (/ delsq2
           (+ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) sq2)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
            (+ (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
               (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) temp))
    (setf (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
            (- (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
               (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) temp))
   label100))
(setf psi zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j (f2cl-lib:int-add i (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (setf psi
            (+ psi
               (/
                (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                   (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%))
                (*
                 (f2cl-lib:fref work-%data% (j) ((1 *))
                                work-%offset%)
                 (f2cl-lib:fref delta-%data% (j) ((1 *))
                                delta-%offset%)))))
   label110))
(setf phi zero)
(f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j (f2cl-lib:int-add i 2)) nil)
  (tagbody
    (setf phi
            (+ phi
```

```
                (/
                 (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                    (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%))
                 (*
                  (f2cl-lib:fref work-%data% (j) ((1 *))
                                 work-%offset%)
                  (f2cl-lib:fref delta-%data% (j) ((1 *))
                                 delta-%offset%)))))
   label120))
(setf c (+ rhoinv psi phi))
(setf w
        (+ c
           (/
            (* (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
               (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%))
            (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               (f2cl-lib:fref delta-%data% (i) ((1 *))
                              delta-%offset%)))
           (/
            (* (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%)
               (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%))
            (*
             (f2cl-lib:fref work-%data% (ip1) ((1 *)) work-%offset%)
             (f2cl-lib:fref delta-%data% (ip1) ((1 *))
                            delta-%offset%)))))
(setf geomavg f2cl-lib:%false%)
(cond
 ((> w zero) (setf orgati f2cl-lib:%true%) (setf ii i)
  (setf sglb zero)
  (setf sgub
          (/ delsq2
             (+ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                sq2)))
  (setf a
          (+ (* c delsq)
             (* (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%))
             (* (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%)
                (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%))))
  (setf b
          (* (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
             (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%) delsq))
  (cond
   ((> a zero)
    (setf tau2
            (/ (* two b)
               (+ a
                  (f2cl-lib:fsqrt
                   (abs (+ (* a a) (* (- four) b c))))))))
   (t
```

```
    (setf tau2
            (/
             (- a
                (f2cl-lib:fsqrt (abs (+ (* a a) (* (- four) b c)))))
             (* two c)))))
  (setf tau
          (/ tau2
             (+ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                (f2cl-lib:fsqrt
                 (+
                  (* (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                     (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
                  tau2)))))
  (setf temp (f2cl-lib:fsqrt eps))
  (cond
   ((and
     (<= (f2cl-lib:fref d (i) ((1 *)))
         (* temp (f2cl-lib:fref d (ip1) ((1 *)))))
     (<= (abs (f2cl-lib:fref z (i) ((1 *)))) temp)
     (> (f2cl-lib:fref d (i) ((1 *))) zero))
    (setf tau
            (min
             (* ten (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
             sgub))
    (setf geomavg f2cl-lib:%true%))))
 (t (setf orgati f2cl-lib:%false%) (setf ii ip1)
  (setf sglb
          (/ (- delsq2)
             (+ (f2cl-lib:fref d-%data% (ii) ((1 *)) d-%offset%)
                sq2)))
  (setf sgub zero)
  (setf a
          (- (* c delsq)
             (* (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%))
             (* (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%)
                (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%))))
  (setf b
          (* (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%)
             (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%)
             delsq))
  (cond
   ((< a zero)
    (setf tau2
            (/ (* two b)
               (- a
                  (f2cl-lib:fsqrt (abs (+ (* a a) (* four b c)))))))
   (t
    (setf tau2
            (/
```

```
                (-
                 (+ a (f2cl-lib:fsqrt (abs (+ (* a a) (* four b c))))))
                (* two c)))))
  (setf tau
          (/ tau2
             (+ (f2cl-lib:fref d-%data% (ip1) ((1 *)) d-%offset%)
                (f2cl-lib:fsqrt
                 (abs
                  (+
                   (*
                    (f2cl-lib:fref d-%data% (ip1) ((1 *)) d-%offset%)
                    (f2cl-lib:fref d-%data% (ip1) ((1 *))
                                   d-%offset%))
                   tau2))))))))
(setf sigma (+ (f2cl-lib:fref d-%data% (ii) ((1 *)) d-%offset%) tau))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
            (+ (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
               (f2cl-lib:fref d-%data% (ii) ((1 *)) d-%offset%) tau))
    (setf (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
            (- (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
               (f2cl-lib:fref d-%data% (ii) ((1 *)) d-%offset%) tau))
   label130))
(setf iim1 (f2cl-lib:int-sub ii 1))
(setf iip1 (f2cl-lib:int-add ii 1)) (setf dpsi zero) (setf psi zero)
(setf erretm zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j iim1) nil)
  (tagbody
    (setf temp
            (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
               (*
                (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                (f2cl-lib:fref delta-%data% (j) ((1 *))
                               delta-%offset%))))
    (setf psi
            (+ psi
               (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                  temp)))
    (setf dpsi (+ dpsi (* temp temp)))
    (setf erretm (+ erretm psi))
   label150))
(setf erretm (abs erretm)) (setf dphi zero) (setf phi zero)
(f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j iip1) nil)
  (tagbody
    (setf temp
            (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
```

```
                (*
                 (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                 (f2cl-lib:fref delta-%data% (j) ((1 *))
                                delta-%offset%))))
    (setf phi
            (+ phi
               (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                  temp)))
    (setf dphi (+ dphi (* temp temp)))
    (setf erretm (+ erretm phi))
   label160))
(setf w (+ rhoinv phi psi)) (setf swtch3 f2cl-lib:%false%)
(cond
 (orgati
  (if (< w zero)
      (setf swtch3 f2cl-lib:%true%)))
 (t
  (if (> w zero)
      (setf swtch3 f2cl-lib:%true%))))
(if (or (= ii 1) (= ii n))
    (setf swtch3 f2cl-lib:%false%))
(setf temp
        (/ (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%)
           (* (f2cl-lib:fref work-%data% (ii) ((1 *)) work-%offset%)
              (f2cl-lib:fref delta-%data% (ii) ((1 *))
                             delta-%offset%))))
(setf dw (+ dpsi dphi (* temp temp)))
(setf temp (* (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%) temp))
(setf w (+ w temp))
(setf erretm
        (+ (* eight (- phi psi)) erretm (* two rhoinv)
           (* three (abs temp))))
(cond ((<= (abs w) (* eps erretm)) (go label240)))
(cond ((<= w zero) (setf sglb (max sglb tau)))
      (t (setf sgub (min sgub tau))))
(setf niter (f2cl-lib:int-add niter 1))
(cond
 ((not swtch3)
  (setf dtipsq
          (* (f2cl-lib:fref work-%data% (ip1) ((1 *)) work-%offset%)
             (f2cl-lib:fref delta-%data% (ip1) ((1 *))
                            delta-%offset%)))
  (setf dtisq
          (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
             (f2cl-lib:fref delta-%data% (i) ((1 *))
                            delta-%offset%)))
  (cond
   (orgati
    (setf c
            (+ (- w (* dtipsq dw))
```

```
                (* delsq
                   (expt
                    (/ (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                       dtisq)
                    2)))))
         (t
          (setf c
                  (- w (* dtisq dw)
                     (* delsq
                        (expt
                         (/
                          (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%)
                          dtipsq)
                         2))))))
        (setf a (+ (* (+ dtipsq dtisq) w) (* (- dtipsq) dtisq dw)))
        (setf b (* dtipsq dtisq w))
        (cond
         ((= c zero)
          (cond
           ((= a zero)
            (cond
             (orgati
              (setf a
                      (+
                       (* (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                          (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%))
                       (* dtipsq dtipsq (+ dpsi dphi)))))
             (t
              (setf a
                      (+
                       (* (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%)
                          (f2cl-lib:fref z-%data% (ip1) ((1 *))
                                         z-%offset%))
                       (* dtisq dtisq (+ dpsi dphi))))))))
          (setf eta (/ b a)))
         ((<= a zero)
          (setf eta
                  (/
                   (- a
                      (f2cl-lib:fsqrt (abs (+ (* a a) (* (- four) b c)))))
                   (* two c))))
         (t
          (setf eta
                  (/ (* two b)
                     (+ a
                        (f2cl-lib:fsqrt
                         (abs (+ (* a a) (* (- four) b c)))))))))
       (t
        (setf dtiim
                (* (f2cl-lib:fref work-%data% (iim1) ((1 *)) work-%offset%)
```

```
          (f2cl-lib:fref delta-%data% (iim1) ((1 *))
                         delta-%offset%)))
(setf dtiip
        (* (f2cl-lib:fref work-%data% (iip1) ((1 *)) work-%offset%)
           (f2cl-lib:fref delta-%data% (iip1) ((1 *))
                         delta-%offset%)))
(setf temp (+ rhoinv psi phi))
(cond
 (orgati
  (setf temp1
          (/ (f2cl-lib:fref z-%data% (iim1) ((1 *)) z-%offset%)
             dtiim))
  (setf temp1 (* temp1 temp1))
  (setf c
          (+ (- temp (* dtiip (+ dpsi dphi)))
             (*
              (-
               (-
                (f2cl-lib:fref d-%data% (iim1) ((1 *)) d-%offset%)
                (f2cl-lib:fref d-%data% (iip1) ((1 *))
                               d-%offset%)))
              (+ (f2cl-lib:fref d-%data% (iim1) ((1 *)) d-%offset%)
                 (f2cl-lib:fref d-%data% (iip1) ((1 *))
                                d-%offset%))
              temp1)))
  (setf (f2cl-lib:fref zz (1) ((1 3)))
          (* (f2cl-lib:fref z-%data% (iim1) ((1 *)) z-%offset%)
             (f2cl-lib:fref z-%data% (iim1) ((1 *)) z-%offset%)))
  (cond
   ((< dpsi temp1)
    (setf (f2cl-lib:fref zz (3) ((1 3))) (* dtiip dtiip dphi)))
   (t
    (setf (f2cl-lib:fref zz (3) ((1 3)))
            (* dtiip dtiip (+ (- dpsi temp1) dphi))))))
 (t
  (setf temp1
          (/ (f2cl-lib:fref z-%data% (iip1) ((1 *)) z-%offset%)
             dtiip))
  (setf temp1 (* temp1 temp1))
  (setf c
          (+ (- temp (* dtiim (+ dpsi dphi)))
             (*
              (-
               (-
                (f2cl-lib:fref d-%data% (iip1) ((1 *)) d-%offset%)
                (f2cl-lib:fref d-%data% (iim1) ((1 *))
                               d-%offset%)))
              (+ (f2cl-lib:fref d-%data% (iim1) ((1 *)) d-%offset%)
                 (f2cl-lib:fref d-%data% (iip1) ((1 *))
                                d-%offset%))
```

```
              temp1)))
  (cond
   ((< dphi temp1)
    (setf (f2cl-lib:fref zz (1) ((1 3))) (* dtiim dtiim dpsi)))
   (t
    (setf (f2cl-lib:fref zz (1) ((1 3)))
            (* dtiim dtiim (+ dpsi (- dphi temp1))))))
  (setf (f2cl-lib:fref zz (3) ((1 3)))
          (* (f2cl-lib:fref z-%data% (iip1) ((1 *)) z-%offset%)
             (f2cl-lib:fref z-%data% (iip1) ((1 *)) z-%offset%)))))
(setf (f2cl-lib:fref zz (2) ((1 3)))
        (* (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%)
           (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%)))
(setf (f2cl-lib:fref dd (1) ((1 3))) dtiim)
(setf (f2cl-lib:fref dd (2) ((1 3)))
        (* (f2cl-lib:fref delta-%data% (ii) ((1 *)) delta-%offset%)
           (f2cl-lib:fref work-%data% (ii) ((1 *)) work-%offset%)))
(setf (f2cl-lib:fref dd (3) ((1 3))) dtiip)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (slaed6 niter orgati c dd zz w eta info)
  (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5))
  (setf eta var-6)
  (setf info var-7))
(cond
 ((/= info 0) (setf swtch3 f2cl-lib:%false%) (setf info 0)
  (setf dtipsq
          (*
           (f2cl-lib:fref work-%data% (ip1) ((1 *)) work-%offset%)
           (f2cl-lib:fref delta-%data% (ip1) ((1 *))
                          delta-%offset%)))
  (setf dtisq
          (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
             (f2cl-lib:fref delta-%data% (i) ((1 *))
                            delta-%offset%)))
  (cond
   (orgati
    (setf c
            (+ (- w (* dtipsq dw))
               (* delsq
                  (expt
                   (/
                    (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                    dtisq)
                   2)))))
   (t
    (setf c
            (- w (* dtisq dw)
               (* delsq
                  (expt
```

```
                        (/
                         (f2cl-lib:fref z-%data% (ip1) ((1 *))
                                        z-%offset%)
                         dtipsq)
                        2))))))
    (setf a (+ (* (+ dtipsq dtisq) w) (* (- dtipsq) dtisq dw)))
    (setf b (* dtipsq dtisq w))
    (cond
     ((= c zero)
      (cond
       ((= a zero)
        (cond
         (orgati
          (setf a
                  (+
                   (* (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                      (f2cl-lib:fref z-%data% (i) ((1 *))
                                     z-%offset%))
                   (* dtipsq dtipsq (+ dpsi dphi)))))
         (t
          (setf a
                  (+
                   (*
                    (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%)
                    (f2cl-lib:fref z-%data% (ip1) ((1 *))
                                   z-%offset%))
                   (* dtisq dtisq (+ dpsi dphi))))))))
      (setf eta (/ b a)))
     ((<= a zero)
      (setf eta
              (/
               (- a
                  (f2cl-lib:fsqrt
                   (abs (+ (* a a) (* (- four) b c)))))
               (* two c))))
     (t
      (setf eta
              (/ (* two b)
                 (+ a
                    (f2cl-lib:fsqrt
                     (abs (+ (* a a) (* (- four) b c)))))))))))
(if (>= (* w eta) zero)
    (setf eta (/ (- w) dw)))
(setf eta (/ eta (+ sigma (f2cl-lib:fsqrt (+ (* sigma sigma) eta)))))
(setf temp (+ tau eta))
(cond
 ((or (> temp sgub) (< temp sglb))
  (cond ((< w zero) (setf eta (/ (- sgub tau) two)))
        (t (setf eta (/ (- sglb tau) two))))
  (cond
```

```
   (geomavg
    (cond
     ((< w zero)
      (cond
       ((> tau zero)
        (setf eta (- (f2cl-lib:fsqrt (* sgub tau)) tau)))))
     (t
      (cond
       ((> sglb zero)
        (setf eta (- (f2cl-lib:fsqrt (* sglb tau)) tau))))))))))
(setf prew w) (setf tau (+ tau eta)) (setf sigma (+ sigma eta))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
            (+ (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
               eta))
    (setf (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
            (-
             (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
             eta))
   label170))
(setf dpsi zero) (setf psi zero) (setf erretm zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j iim1) nil)
  (tagbody
    (setf temp
            (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
               (*
                (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                (f2cl-lib:fref delta-%data% (j) ((1 *))
                               delta-%offset%))))
    (setf psi
            (+ psi
               (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                  temp)))
    (setf dpsi (+ dpsi (* temp temp)))
    (setf erretm (+ erretm psi))
   label180))
(setf erretm (abs erretm)) (setf dphi zero) (setf phi zero)
(f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j iip1) nil)
  (tagbody
    (setf temp
            (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
               (*
                (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                (f2cl-lib:fref delta-%data% (j) ((1 *))
                               delta-%offset%))))
    (setf phi
```

```
            (+ phi
               (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                  temp)))
    (setf dphi (+ dphi (* temp temp)))
    (setf erretm (+ erretm phi))
   label190))
(setf tau2
        (* (f2cl-lib:fref work-%data% (ii) ((1 *)) work-%offset%)
           (f2cl-lib:fref delta-%data% (ii) ((1 *)) delta-%offset%)))
(setf temp (/ (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%) tau2))
(setf dw (+ dpsi dphi (* temp temp)))
(setf temp (* (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%) temp))
(setf w (+ rhoinv phi psi temp))
(setf erretm
        (+ (* eight (- phi psi)) erretm (* two rhoinv)
           (* three (abs temp))))
(setf swtch f2cl-lib:%false%)
(cond
 (orgati
  (if (> (- w) (/ (abs prew) ten))
      (setf swtch f2cl-lib:%true%)))
 (t
  (if (> w (/ (abs prew) ten))
      (setf swtch f2cl-lib:%true%))))
(setf iter (f2cl-lib:int-add niter 1))
(f2cl-lib:fdo (niter iter (f2cl-lib:int-add niter 1))
              ((> niter maxit) nil)
  (tagbody
    (cond ((<= (abs w) (* eps erretm)) (go label240)))
    (cond ((<= w zero) (setf sglb (max sglb tau)))
          (t (setf sgub (min sgub tau))))
    (cond
     ((not swtch3)
      (setf dtipsq
              (*
               (f2cl-lib:fref work-%data% (ip1) ((1 *))
                              work-%offset%)
               (f2cl-lib:fref delta-%data% (ip1) ((1 *))
                              delta-%offset%)))
      (setf dtisq
              (*
               (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               (f2cl-lib:fref delta-%data% (i) ((1 *))
                              delta-%offset%)))
      (cond
       ((not swtch)
        (cond
         (orgati
          (setf c
                  (+ (- w (* dtipsq dw))
```

```
                 (* delsq
                    (expt
                     (/
                      (f2cl-lib:fref z-%data% (i) ((1 *))
                                     z-%offset%)
                      dtisq)
                     2)))))
    (t
     (setf c
             (- w (* dtisq dw)
                (* delsq
                   (expt
                    (/
                     (f2cl-lib:fref z-%data% (ip1) ((1 *))
                                    z-%offset%)
                     dtipsq)
                    2)))))))
  (t
   (setf temp
           (/ (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%)
              (*
               (f2cl-lib:fref work-%data% (ii) ((1 *))
                              work-%offset%)
               (f2cl-lib:fref delta-%data% (ii) ((1 *))
                              delta-%offset%))))
   (cond (orgati (setf dpsi (+ dpsi (* temp temp))))
         (t (setf dphi (+ dphi (* temp temp)))))
   (setf c (- w (* dtisq dpsi) (* dtipsq dphi)))))
 (setf a (+ (* (+ dtipsq dtisq) w) (* (- dtipsq) dtisq dw)))
 (setf b (* dtipsq dtisq w))
 (cond
  ((= c zero)
   (cond
    ((= a zero)
     (cond
      ((not swtch)
       (cond
        (orgati
         (setf a
                 (+
                  (*
                   (f2cl-lib:fref z-%data% (i) ((1 *))
                                  z-%offset%)
                   (f2cl-lib:fref z-%data% (i) ((1 *))
                                  z-%offset%))
                  (* dtipsq dtipsq (+ dpsi dphi)))))
        (t
         (setf a
                 (+
                  (*
```

```
                    (f2cl-lib:fref z-%data% (ip1) ((1 *))
                                   z-%offset%)
                    (f2cl-lib:fref z-%data% (ip1) ((1 *))
                                   z-%offset%))
                   (* dtisq dtisq (+ dpsi dphi)))))))
      (t
       (setf a
               (+ (* dtisq dtisq dpsi)
                  (* dtipsq dtipsq dphi)))))))
   (setf eta (/ b a)))
  ((<= a zero)
   (setf eta
           (/
            (- a
               (f2cl-lib:fsqrt
                (abs (+ (* a a) (* (- four) b c)))))
            (* two c))))
  (t
   (setf eta
           (/ (* two b)
              (+ a
                 (f2cl-lib:fsqrt
                  (abs (+ (* a a) (* (- four) b c)))))))))
(t
 (setf dtiim
         (*
          (f2cl-lib:fref work-%data% (iim1) ((1 *))
                         work-%offset%)
          (f2cl-lib:fref delta-%data% (iim1) ((1 *))
                         delta-%offset%)))
 (setf dtiip
         (*
          (f2cl-lib:fref work-%data% (iip1) ((1 *))
                         work-%offset%)
          (f2cl-lib:fref delta-%data% (iip1) ((1 *))
                         delta-%offset%)))
 (setf temp (+ rhoinv psi phi))
 (cond
  (swtch (setf c (- temp (* dtiim dpsi) (* dtiip dphi)))
   (setf (f2cl-lib:fref zz (1) ((1 3))) (* dtiim dtiim dpsi))
   (setf (f2cl-lib:fref zz (3) ((1 3))) (* dtiip dtiip dphi)))
  (t
   (cond
    (orgati
     (setf temp1
             (/
              (f2cl-lib:fref z-%data% (iim1) ((1 *)) z-%offset%)
              dtiim))
     (setf temp1 (* temp1 temp1))
     (setf temp2
```

```
        (*
         (-
          (f2cl-lib:fref d-%data% (iim1) ((1 *))
                         d-%offset%)
          (f2cl-lib:fref d-%data% (iip1) ((1 *))
                         d-%offset%))
         (+
          (f2cl-lib:fref d-%data% (iim1) ((1 *))
                         d-%offset%)
          (f2cl-lib:fref d-%data% (iip1) ((1 *))
                         d-%offset%))
         temp1))
 (setf c (- temp (* dtiip (+ dpsi dphi)) temp2))
 (setf (f2cl-lib:fref zz (1) ((1 3)))
        (*
         (f2cl-lib:fref z-%data% (iim1) ((1 *)) z-%offset%)
         (f2cl-lib:fref z-%data% (iim1) ((1 *))
                        z-%offset%)))
 (cond
  ((< dpsi temp1)
   (setf (f2cl-lib:fref zz (3) ((1 3)))
         (* dtiip dtiip dphi)))
  (t
   (setf (f2cl-lib:fref zz (3) ((1 3)))
         (* dtiip dtiip (+ (- dpsi temp1) dphi))))))
(t
 (setf temp1
        (/
         (f2cl-lib:fref z-%data% (iip1) ((1 *)) z-%offset%)
         dtiip))
 (setf temp1 (* temp1 temp1))
 (setf temp2
        (*
         (-
          (f2cl-lib:fref d-%data% (iip1) ((1 *))
                         d-%offset%)
          (f2cl-lib:fref d-%data% (iim1) ((1 *))
                         d-%offset%))
         (+
          (f2cl-lib:fref d-%data% (iim1) ((1 *))
                         d-%offset%)
          (f2cl-lib:fref d-%data% (iip1) ((1 *))
                         d-%offset%))
         temp1))
 (setf c (- temp (* dtiim (+ dpsi dphi)) temp2))
 (cond
  ((< dphi temp1)
   (setf (f2cl-lib:fref zz (1) ((1 3)))
         (* dtiim dtiim dpsi)))
  (t
```

```
      (setf (f2cl-lib:fref zz (1) ((1 3)))
              (* dtiim dtiim (+ dpsi (- dphi temp1))))))
    (setf (f2cl-lib:fref zz (3) ((1 3)))
            (*
             (f2cl-lib:fref z-%data% (iip1) ((1 *)) z-%offset%)
             (f2cl-lib:fref z-%data% (iip1) ((1 *))
                            z-%offset%)))))))
(setf (f2cl-lib:fref dd (1) ((1 3))) dtiim)
(setf (f2cl-lib:fref dd (2) ((1 3)))
        (*
         (f2cl-lib:fref delta-%data% (ii) ((1 *))
                        delta-%offset%)
         (f2cl-lib:fref work-%data% (ii) ((1 *))
                        work-%offset%)))
(setf (f2cl-lib:fref dd (3) ((1 3))) dtiip)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (slaed6 niter orgati c dd zz w eta info)
  (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5))
  (setf eta var-6)
  (setf info var-7))
(cond
 ((/= info 0) (setf swtch3 f2cl-lib:%false%) (setf info 0)
  (setf dtipsq
          (*
           (f2cl-lib:fref work-%data% (ip1) ((1 *))
                          work-%offset%)
           (f2cl-lib:fref delta-%data% (ip1) ((1 *))
                          delta-%offset%)))
  (setf dtisq
          (*
           (f2cl-lib:fref work-%data% (i) ((1 *))
                          work-%offset%)
           (f2cl-lib:fref delta-%data% (i) ((1 *))
                          delta-%offset%)))
  (cond
   ((not swtch)
    (cond
     (orgati
      (setf c
              (+ (- w (* dtipsq dw))
                 (* delsq
                    (expt
                     (/
                      (f2cl-lib:fref z-%data% (i) ((1 *))
                                     z-%offset%)
                      dtisq)
                     2)))))
     (t
      (setf c
```

```
                (- w (* dtisq dw)
                   (* delsq
                      (expt
                       (/
                        (f2cl-lib:fref z-%data% (ip1) ((1 *))
                                       z-%offset%)
                        dtipsq)
                       2)))))))
 (t
  (setf temp
          (/ (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%)
             (*
              (f2cl-lib:fref work-%data% (ii) ((1 *))
                             work-%offset%)
              (f2cl-lib:fref delta-%data% (ii) ((1 *))
                             delta-%offset%))))
  (cond (orgati (setf dpsi (+ dpsi (* temp temp))))
        (t (setf dphi (+ dphi (* temp temp)))))
  (setf c (- w (* dtisq dpsi) (* dtipsq dphi)))))
(setf a (+ (* (+ dtipsq dtisq) w) (* (- dtipsq) dtisq dw)))
(setf b (* dtipsq dtisq w))
(cond
 ((= c zero)
  (cond
   ((= a zero)
    (cond
     ((not swtch)
      (cond
       (orgati
        (setf a
                (+
                 (*
                  (f2cl-lib:fref z-%data% (i) ((1 *))
                                 z-%offset%)
                  (f2cl-lib:fref z-%data% (i) ((1 *))
                                 z-%offset%))
                 (* dtipsq dtipsq (+ dpsi dphi)))))
       (t
        (setf a
                (+
                 (*
                  (f2cl-lib:fref z-%data% (ip1) ((1 *))
                                 z-%offset%)
                  (f2cl-lib:fref z-%data% (ip1) ((1 *))
                                 z-%offset%))
                 (* dtisq dtisq (+ dpsi dphi)))))))
     (t
      (setf a
              (+ (* dtisq dtisq dpsi)
                 (* dtipsq dtipsq dphi)))))))
```

```
      (setf eta (/ b a)))
     ((<= a zero)
      (setf eta
              (/
               (- a
                  (f2cl-lib:fsqrt
                   (abs (+ (* a a) (* (- four) b c)))))
               (* two c))))
     (t
      (setf eta
              (/ (* two b)
                 (+ a
                    (f2cl-lib:fsqrt
                     (abs (+ (* a a) (* (- four) b c))))))))))))
(if (>= (* w eta) zero)
    (setf eta (/ (- w) dw)))
(setf eta
        (/ eta
           (+ sigma (f2cl-lib:fsqrt (+ (* sigma sigma) eta)))))
(setf temp (+ tau eta))
(cond
 ((or (> temp sgub) (< temp sglb))
  (cond ((< w zero) (setf eta (/ (- sgub tau) two)))
        (t (setf eta (/ (- sglb tau) two))))
  (cond
   (geomavg
    (cond
     ((< w zero)
      (cond
       ((> tau zero)
        (setf eta (- (f2cl-lib:fsqrt (* sgub tau)) tau)))))
     (t
      (cond
       ((> sglb zero)
        (setf eta (- (f2cl-lib:fsqrt (* sglb tau)) tau))))))))))
(setf prew w)
(setf tau (+ tau eta))
(setf sigma (+ sigma eta))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
            (+
             (f2cl-lib:fref work-%data% (j) ((1 *))
                            work-%offset%)
             eta))
    (setf (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
            (-
             (f2cl-lib:fref delta-%data% (j) ((1 *))
                            delta-%offset%)
```

```
             eta))
   label200))
(setf dpsi zero)
(setf psi zero)
(setf erretm zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j iim1) nil)
  (tagbody
    (setf temp
            (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
               (*
                (f2cl-lib:fref work-%data% (j) ((1 *))
                               work-%offset%)
                (f2cl-lib:fref delta-%data% (j) ((1 *))
                               delta-%offset%))))
    (setf psi
            (+ psi
               (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                  temp)))
    (setf dpsi (+ dpsi (* temp temp)))
    (setf erretm (+ erretm psi))
   label210))
(setf erretm (abs erretm))
(setf dphi zero)
(setf phi zero)
(f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j iip1) nil)
  (tagbody
    (setf temp
            (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
               (*
                (f2cl-lib:fref work-%data% (j) ((1 *))
                               work-%offset%)
                (f2cl-lib:fref delta-%data% (j) ((1 *))
                               delta-%offset%))))
    (setf phi
            (+ phi
               (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                  temp)))
    (setf dphi (+ dphi (* temp temp)))
    (setf erretm (+ erretm phi))
   label220))
(setf tau2
        (* (f2cl-lib:fref work-%data% (ii) ((1 *)) work-%offset%)
           (f2cl-lib:fref delta-%data% (ii) ((1 *))
                          delta-%offset%)))
(setf temp
        (/ (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%)
           tau2))
(setf dw (+ dpsi dphi (* temp temp)))
```

```
            (setf temp
                    (* (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%)
                       temp))
            (setf w (+ rhoinv phi psi temp))
            (setf erretm
                    (+ (* eight (- phi psi)) erretm (* two rhoinv)
                       (* three (abs temp))))
            (if (and (> (* w prew) zero) (> (abs w) (/ (abs prew) ten)))
                (setf swtch (not swtch)))
          label230))
        (setf info 1)))
      label240
       (go end_label)
      end_label
       (return (values nil i nil nil nil rho sigma nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slasd4
                 fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((fortran-to-lisp::integer4)
                                            (fortran-to-lisp::integer4)
                                            (array single-float (*))
                                            (array single-float (*))
                                            (array single-float (*))
                                            (single-float) (single-float)
                                            (array single-float (*))
                                            (fortran-to-lisp::integer4))
                                          :return-values
                                          '(nil fortran-to-lisp::i nil nil
                                            nil fortran-to-lisp::rho
                                            fortran-to-lisp::sigma nil
                                            fortran-to-lisp::info)
                                          :calls '(fortran-to-lisp::slaed6))))
```

———-

90.97 slasd5

90.97.1 Axiom unit tests

— slasd5.input —

```
)set break resume
)sys rm -f slasd5.output
)spool slasd5.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.97.2 Axiom help page

— slasd5.help —

```
======================================================================
slasd5 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slasd5 (I, D, Z, DELTA, RHO, DSIGMA, WORK)
           SLASD5 computes the square root of the i-th eigenvalue of a
           positive symmetric rank-one modification of a 2-by-2 diagonal
           matrix. Used by sbdsdc.

Function/Subroutine Documentation
   subroutine slasd5 (integer I, real, dimension( 2 ) D, real, dimension( 2 )
       Z, real, dimension( 2 ) DELTA, real RHO, real DSIGMA, real, dimension(
       2 ) WORK)
       SLASD5 computes the square root of the i-th eigenvalue of a positive
       symmetric rank-one modification of a 2-by-2 diagonal matrix. Used by
       sbdsdc.

Purpose:
   This subroutine computes the square root of the I-th eigenvalue
   of a positive symmetric rank-one modification of a 2-by-2 diagonal
   matrix

                       diag( D ) * diag( D ) +  RHO * Z * transpose(Z) .

   The diagonal entries in the array D are assumed to satisfy
```

```
                     0 <= D(i) < D(j)  for  i < j .

   We also assume RHO > 0 and that the Euclidean norm of the vector Z is one.

Parameters:

 I
   I is INTEGER
   The index of the eigenvalue to be computed.  I = 1 or I = 2.

 D
   D is REAL array, dimension (2)
   The original eigenvalues.  We assume 0 <= D(1) < D(2).

 Z
   Z is REAL array, dimension (2)
   The components of the updating vector.

 DELTA
   DELTA is REAL array, dimension (2)
   Contains (D(j) - sigma_I) in its  j-th component.
   The vector DELTA contains the information necessary
   to construct the eigenvectors.

 RHO
   RHO is REAL
   The scalar in the symmetric updating formula.

 DSIGMA
   DSIGMA is REAL
   The computed sigma_I, the I-th updated eigenvalue.

 WORK
   WORK is REAL array, dimension (2)
   WORK contains (D(j) + sigma_I) in its  j-th component.
```

———-

90.97.3 fortran code

— **slasd5.f** —

```
*  =====================================================================
      SUBROUTINE SLASD5( I, D, Z, DELTA, RHO, DSIGMA, WORK )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
```

```
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            I
      REAL               DSIGMA, RHO
*     ..
*     .. Array Arguments ..
      REAL               D( 2 ), DELTA( 2 ), WORK( 2 ), Z( 2 )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE, TWO, THREE, FOUR
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0, TWO = 2.0E+0,
     $                   THREE = 3.0E+0, FOUR = 4.0E+0 )
*     ..
*     .. Local Scalars ..
      REAL               B, C, DEL, DELSQ, TAU, W
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SQRT
*     ..
*     .. Executable Statements ..
*
      DEL = D( 2 ) - D( 1 )
      DELSQ = DEL*( D( 2 )+D( 1 ) )
      IF( I.EQ.1 ) THEN
         W = ONE + FOUR*RHO*( Z( 2 )*Z( 2 ) / ( D( 1 )+THREE*D( 2 ) )-
     $       Z( 1 )*Z( 1 ) / ( THREE*D( 1 )+D( 2 ) ) ) / DEL
         IF( W.GT.ZERO ) THEN
            B = DELSQ + RHO*( Z( 1 )*Z( 1 )+Z( 2 )*Z( 2 ) )
            C = RHO*Z( 1 )*Z( 1 )*DELSQ
*
*           B > ZERO, always
*
*           The following TAU is DSIGMA * DSIGMA - D( 1 ) * D( 1 )
*
            TAU = TWO*C / ( B+SQRT( ABS( B*B-FOUR*C ) ) )
*
*           The following TAU is DSIGMA - D( 1 )
*
            TAU = TAU / ( D( 1 )+SQRT( D( 1 )*D( 1 )+TAU ) )
            DSIGMA = D( 1 ) + TAU
            DELTA( 1 ) = -TAU
            DELTA( 2 ) = DEL - TAU
            WORK( 1 ) = TWO*D( 1 ) + TAU
            WORK( 2 ) = ( D( 1 )+TAU ) + D( 2 )
*           DELTA( 1 ) = -Z( 1 ) / TAU
```

```
*          DELTA( 2 ) = Z( 2 ) / ( DEL-TAU )
        ELSE
           B = -DELSQ + RHO*( Z( 1 )*Z( 1 )+Z( 2 )*Z( 2 ) )
           C = RHO*Z( 2 )*Z( 2 )*DELSQ
*
*          The following TAU is DSIGMA * DSIGMA - D( 2 ) * D( 2 )
*
           IF( B.GT.ZERO ) THEN
              TAU = -TWO*C / ( B+SQRT( B*B+FOUR*C ) )
           ELSE
              TAU = ( B-SQRT( B*B+FOUR*C ) ) / TWO
           END IF
*
*          The following TAU is DSIGMA - D( 2 )
*
           TAU = TAU / ( D( 2 )+SQRT( ABS( D( 2 )*D( 2 )+TAU ) ) )
           DSIGMA = D( 2 ) + TAU
           DELTA( 1 ) = -( DEL+TAU )
           DELTA( 2 ) = -TAU
           WORK( 1 ) = D( 1 ) + TAU + D( 2 )
           WORK( 2 ) = TWO*D( 2 ) + TAU
*          DELTA( 1 ) = -Z( 1 ) / ( DEL+TAU )
*          DELTA( 2 ) = -Z( 2 ) / TAU
        END IF
*       TEMP = SQRT( DELTA( 1 )*DELTA( 1 )+DELTA( 2 )*DELTA( 2 ) )
*       DELTA( 1 ) = DELTA( 1 ) / TEMP
*       DELTA( 2 ) = DELTA( 2 ) / TEMP
     ELSE
*
*       Now I=2
*
        B = -DELSQ + RHO*( Z( 1 )*Z( 1 )+Z( 2 )*Z( 2 ) )
        C = RHO*Z( 2 )*Z( 2 )*DELSQ
*
*       The following TAU is DSIGMA * DSIGMA - D( 2 ) * D( 2 )
*
        IF( B.GT.ZERO ) THEN
           TAU = ( B+SQRT( B*B+FOUR*C ) ) / TWO
        ELSE
           TAU = TWO*C / ( -B+SQRT( B*B+FOUR*C ) )
        END IF
*
*       The following TAU is DSIGMA - D( 2 )
*
        TAU = TAU / ( D( 2 )+SQRT( D( 2 )*D( 2 )+TAU ) )
        DSIGMA = D( 2 ) + TAU
        DELTA( 1 ) = -( DEL+TAU )
        DELTA( 2 ) = -TAU
        WORK( 1 ) = D( 1 ) + TAU + D( 2 )
        WORK( 2 ) = TWO*D( 2 ) + TAU
```

```
*        DELTA( 1 ) = -Z( 1 ) / ( DEL+TAU )
*        DELTA( 2 ) = -Z( 2 ) / TAU
*        TEMP = SQRT( DELTA( 1 )*DELTA( 1 )+DELTA( 2 )*DELTA( 2 ) )
*        DELTA( 1 ) = DELTA( 1 ) / TEMP
*        DELTA( 2 ) = DELTA( 2 ) / TEMP
      END IF
      RETURN
*
*     End of SLASD5
*
      END
```

———-

90.97.4 lisp code

— slasd5.lisp —

```
(let* ((zero 0.0) (one 1.0) (two 2.0) (three 3.0) (four 4.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 3.0 3.0) three)
           (type (single-float 4.0 4.0) four)
           (ignorable zero one two three four))
  (defun slasd5 (i d z delta rho dsigma work)
    (declare (type (single-float) dsigma rho)
             (type (array single-float (*)) work delta z d)
             (type (f2cl-lib:integer4) i))
    (f2cl-lib:with-multi-array-data
        ((d single-float d-%data% d-%offset%)
         (z single-float z-%data% z-%offset%)
         (delta single-float delta-%data% delta-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((b 0.0) (c 0.0) (del 0.0) (delsq 0.0) (tau 0.0) (w 0.0))
        (declare (type (single-float) b c del delsq tau w))
        (setf del
                (- (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%)
                   (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%)))
        (setf delsq
                (* del
                   (+ (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%)
                      (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%))))
        (cond
         ((= i 1)
          (setf w
                  (+ one
                     (/
```

```
                (* four rho
                   (+
                    (/
                     (* (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%)
                        (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%))
                     (+ (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%)
                        (* three
                           (f2cl-lib:fref d-%data% (2) ((1 2))
                                          d-%offset%))))
                    (/
                     (*
                      (- (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%))
                      (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%))
                     (+
                      (* three
                         (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%))
                      (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%)))))
                del)))
        (cond
         ((> w zero)
          (setf b
                  (+ delsq
                     (* rho
                        (+
                         (* (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%)
                            (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%))
                         (* (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%)
                            (f2cl-lib:fref z-%data% (2) ((1 2))
                                           z-%offset%))))))
          (setf c
                  (* rho (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%)
                     (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%) delsq))
          (setf tau
                  (/ (* two c)
                     (+ b (f2cl-lib:fsqrt (abs (- (* b b) (* four c)))))))
          (setf tau
                  (/ tau
                     (+ (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%)
                        (f2cl-lib:fsqrt
                         (+
                          (* (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%)
                             (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%))
                          tau)))))
          (setf dsigma
                  (+ (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%) tau))
          (setf (f2cl-lib:fref delta-%data% (1) ((1 2)) delta-%offset%)
                  (- tau))
          (setf (f2cl-lib:fref delta-%data% (2) ((1 2)) delta-%offset%)
                  (- del tau))
          (setf (f2cl-lib:fref work-%data% (1) ((1 2)) work-%offset%)
```

```
            (+ (* two (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%))
               tau))
    (setf (f2cl-lib:fref work-%data% (2) ((1 2)) work-%offset%)
            (+ (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%) tau
               (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%))))
   (t
    (setf b
            (-
             (* rho
                (+
                 (* (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%)
                    (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%))
                 (* (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%)
                    (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%))))
             delsq))
    (setf c
            (* rho (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%)
               (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%) delsq))
    (cond
     ((> b zero)
      (setf tau
              (/ (* (- two) c)
                 (+ b (f2cl-lib:fsqrt (+ (* b b) (* four c)))))))
     (t
      (setf tau
              (/ (- b (f2cl-lib:fsqrt (+ (* b b) (* four c)))) two))))
    (setf tau
            (/ tau
               (+ (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%)
                  (f2cl-lib:fsqrt
                   (abs
                    (+
                     (* (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%)
                        (f2cl-lib:fref d-%data% (2) ((1 2))
                                       d-%offset%))
                     tau))))))
    (setf dsigma
            (+ (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%) tau))
    (setf (f2cl-lib:fref delta-%data% (1) ((1 2)) delta-%offset%)
            (- (+ del tau)))
    (setf (f2cl-lib:fref delta-%data% (2) ((1 2)) delta-%offset%)
            (- tau))
    (setf (f2cl-lib:fref work-%data% (1) ((1 2)) work-%offset%)
            (+ (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%) tau
               (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%)))
    (setf (f2cl-lib:fref work-%data% (2) ((1 2)) work-%offset%)
            (+ (* two (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%))
               tau)))))
  (t
   (setf b
```

```
                (-
                 (* rho
                    (+
                     (* (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%)
                        (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%))
                     (* (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%)
                        (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%))))
                 delsq))
        (setf c
                (* rho (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%)
                   (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%) delsq))
        (cond
         ((> b zero)
          (setf tau (/ (+ b (f2cl-lib:fsqrt (+ (* b b) (* four c)))) two)))
         (t
          (setf tau
                  (/ (* two c)
                     (- (f2cl-lib:fsqrt (+ (* b b) (* four c))) b)))))
        (setf tau
                (/ tau
                   (+ (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%)
                      (f2cl-lib:fsqrt
                       (+
                        (* (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%)
                           (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%))
                        tau)))))
        (setf dsigma (+ (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%) tau))
        (setf (f2cl-lib:fref delta-%data% (1) ((1 2)) delta-%offset%)
                (- (+ del tau)))
        (setf (f2cl-lib:fref delta-%data% (2) ((1 2)) delta-%offset%)
                (- tau))
        (setf (f2cl-lib:fref work-%data% (1) ((1 2)) work-%offset%)
                (+ (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%) tau
                   (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%)))
        (setf (f2cl-lib:fref work-%data% (2) ((1 2)) work-%offset%)
                (+ (* two (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%))
                   tau))))
      (go end_label)
     end_label
      (return (values nil nil nil nil nil dsigma nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slasd5
                 fortran-to-lisp::*f2cl-function-info*)
        (fortran-to-lisp::make-f2cl-finfo :arg-types
                                          '((fortran-to-lisp::integer4)
                                            (array single-float (*))
                                            (array single-float (*))
```

```
  (array single-float (*))
  (single-float) (single-float)
  (array single-float (*)))
:return-values
'(nil nil nil nil nil
  fortran-to-lisp::dsigma nil)
:calls 'nil)))
```

———-

90.98 slasd6

90.98.1 Axiom unit tests

— **slasd6.input** —

```
)set break resume
)sys rm -f slasd6.output
)spool slasd6.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.98.2 Axiom help page

— **slasd6.help** —

```
==========================================================================
slasd6 examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

SYNOPSIS
   Functions/Subroutines
```

```
subroutine slasd6 (ICOMPQ, NL, NR, SQRE, D, VF, VL, ALPHA, BETA, IDXQ,
    PERM, GIVPTR, GIVCOL, LDGCOL, GIVNUM, LDGNUM, POLES, DIFL, DIFR, Z,
    K, C, S, WORK, IWORK, INFO)
    SLASD6 computes the SVD of an updated upper bidiagonal matrix
    obtained by merging two smaller ones by appending a row. Used by
    sbdsdc.
```

Function/Subroutine Documentation

```
subroutine slasd6 (integer ICOMPQ, integer NL, integer NR, integer SQRE,
    real, dimension( * ) D, real, dimension( * ) VF, real, dimension( * )
    VL, real ALPHA, real BETA, integer, dimension( * ) IDXQ, integer,
    dimension( * ) PERM, integer GIVPTR, integer, dimension( ldgcol, * )
    GIVCOL, integer LDGCOL, real, dimension( ldgnum, * ) GIVNUM, integer
    LDGNUM, real, dimension( ldgnum, * ) POLES, real, dimension( * ) DIFL,
    real, dimension( * ) DIFR, real, dimension( * ) Z, integer K, real C,
    real S, real, dimension( * ) WORK, integer, dimension( * ) IWORK,
    integer INFO)
    SLASD6 computes the SVD of an updated upper bidiagonal matrix obtained
    by merging two smaller ones by appending a row. Used by sbdsdc.
```

Purpose:

```
SLASD6 computes the SVD of an updated upper bidiagonal matrix B
obtained by merging two smaller ones by appending a row. This
routine is used only for the problem which requires all singular
values and optionally singular vector matrices in factored form.
B is an N-by-M matrix with N = NL + NR + 1 and M = N + SQRE.
A related subroutine, SLASD1, handles the case in which all singular
values and singular vectors of the bidiagonal matrix are desired.

SLASD6 computes the SVD as follows:

                        ( D1(in)    0    0       0 )
            B = U(in) * (   Z1**T   a   Z2**T    b ) * VT(in)
                        (   0       0   D2(in)   0 )

              = U(out) * ( D(out) 0) * VT(out)

where Z**T = (Z1**T a Z2**T b) = u**T VT**T, and u is a vector of
dimension M with ALPHA and BETA in the NL+1 and NL+2 th
entries and zeros elsewhere; and the entry b is empty if SQRE = 0.

The singular values of B can be computed using D1, D2, the first
components of all the right singular vectors of the lower block, and
the last components of all the right singular vectors of the upper
block. These components are stored and updated in VF and VL,
respectively, in SLASD6. Hence U and VT are not explicitly
referenced.

The singular values are stored in D. The algorithm consists of two
stages:
```

```
  The first stage consists of deflating the size of the problem
  when there are multiple singular values or if there is a zero
  in the Z vector. For each such occurence the dimension of the
  secular equation problem is reduced by one. This stage is
  performed by the routine SLASD7.

  The second stage consists of calculating the updated
  singular values. This is done by finding the roots of the
  secular equation via the routine SLASD4 (as called by SLASD8).
  This routine also updates VF and VL and computes the distances
  between the updated singular values and the old singular
  values.

  SLASD6 is called from SLASDA.

Parameters:

 ICOMPQ
   ICOMPQ is INTEGER
   Specifies whether singular vectors are to be computed in
   factored form:
   = 0: Compute singular values only.
   = 1: Compute singular vectors in factored form as well.

 NL
   NL is INTEGER
   The row dimension of the upper block.  NL >= 1.

 NR
   NR is INTEGER
   The row dimension of the lower block.  NR >= 1.

 SQRE
   SQRE is INTEGER
   = 0: the lower block is an NR-by-NR square matrix.
   = 1: the lower block is an NR-by-(NR+1) rectangular matrix.

   The bidiagonal matrix has row dimension N = NL + NR + 1,
   and column dimension M = N + SQRE.

 D
   D is REAL array, dimension (NL+NR+1).
   On entry D(1:NL,1:NL) contains the singular values of the
   upper block, and D(NL+2:N) contains the singular values
   of the lower block. On exit D(1:N) contains the singular
   values of the modified matrix.

 VF
   VF is REAL array, dimension (M)
```

```
  On entry, VF(1:NL+1) contains the first components of all
  right singular vectors of the upper block; and VF(NL+2:M)
  contains the first components of all right singular vectors
  of the lower block. On exit, VF contains the first components
  of all right singular vectors of the bidiagonal matrix.

VL
  VL is REAL array, dimension (M)
  On entry, VL(1:NL+1) contains the  last components of all
  right singular vectors of the upper block; and VL(NL+2:M)
  contains the last components of all right singular vectors of
  the lower block. On exit, VL contains the last components of
  all right singular vectors of the bidiagonal matrix.

ALPHA
  ALPHA is REAL
  Contains the diagonal element associated with the added row.

BETA
  BETA is REAL
  Contains the off-diagonal element associated with the added row.

IDXQ
  IDXQ is INTEGER array, dimension (N)
  This contains the permutation which will reintegrate the
  subproblem just solved back into sorted order, i.e.
  D( IDXQ( I = 1, N ) ) will be in ascending order.

PERM
  PERM is INTEGER array, dimension ( N )
  The permutations (from deflation and sorting) to be applied
  to each block. Not referenced if ICOMPQ = 0.

GIVPTR
  GIVPTR is INTEGER
  The number of Givens rotations which took place in this
  subproblem. Not referenced if ICOMPQ = 0.

GIVCOL
  GIVCOL is INTEGER array, dimension ( LDGCOL, 2 )
  Each pair of numbers indicates a pair of columns to take place
  in a Givens rotation. Not referenced if ICOMPQ = 0.

LDGCOL
  LDGCOL is INTEGER
  leading dimension of GIVCOL, must be at least N.

GIVNUM
  GIVNUM is REAL array, dimension ( LDGNUM, 2 )
  Each number indicates the C or S value to be used in the
```

```
  corresponding Givens rotation. Not referenced if ICOMPQ = 0.

LDGNUM
  LDGNUM is INTEGER
  The leading dimension of GIVNUM and POLES, must be at least N.

POLES
  POLES is REAL array, dimension ( LDGNUM, 2 )
  On exit, POLES(1,*) is an array containing the new singular
  values obtained from solving the secular equation, and
  POLES(2,*) is an array containing the poles in the secular
  equation. Not referenced if ICOMPQ = 0.

DIFL
  DIFL is REAL array, dimension ( N )
  On exit, DIFL(I) is the distance between I-th updated
  (undeflated) singular value and the I-th (undeflated) old singular value.

DIFR
  DIFR is REAL array,
     dimension ( LDGNUM, 2 ) if ICOMPQ = 1 and
     dimension ( N ) if ICOMPQ = 0.
  On exit, DIFR(I, 1) is the distance between I-th updated
  (undeflated) singular value and the I+1-th (undeflated) old singular value.

  If ICOMPQ = 1, DIFR(1:K,2) is an array containing the
  normalizing factors for the right singular vector matrix.

  See SLASD8 for details on DIFL and DIFR.

Z
  Z is REAL array, dimension ( M )
  The first elements of this array contain the components
  of the deflation-adjusted updating row vector.

K
  K is INTEGER
  Contains the dimension of the non-deflated matrix,
  This is the order of the related secular equation. 1 <= K <=N.

C
  C is REAL
  C contains garbage if SQRE =0 and the C-value of a Givens
  rotation related to the right null space if SQRE = 1.

S
  S is REAL
  S contains garbage if SQRE =0 and the S-value of a Givens
  rotation related to the right null space if SQRE = 1.
```

```
 WORK
   WORK is REAL array, dimension ( 4 * M )

 IWORK
   IWORK is INTEGER array, dimension ( 3 * N )

 INFO
   INFO is INTEGER
   = 0:  successful exit.
   < 0:  if INFO = -i, the i-th argument had an illegal value.
   > 0:  if INFO = 1, a singular value did not converge
```

———-

90.98.3 fortran code

— **slasd6.f** —

```
*  =====================================================================
      SUBROUTINE SLASD6( ICOMPQ, NL, NR, SQRE, D, VF, VL, ALPHA, BETA,
     $                   IDXQ, PERM, GIVPTR, GIVCOL, LDGCOL, GIVNUM,
     $                   LDGNUM, POLES, DIFL, DIFR, Z, K, C, S, WORK,
     $                   IWORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            GIVPTR, ICOMPQ, INFO, K, LDGCOL, LDGNUM, NL,
     $                   NR, SQRE
      REAL               ALPHA, BETA, C, S
*     ..
*     .. Array Arguments ..
      INTEGER            GIVCOL( LDGCOL, * ), IDXQ( * ), IWORK( * ),
     $                   PERM( * )
      REAL               D( * ), DIFL( * ), DIFR( * ),
     $                   GIVNUM( LDGNUM, * ), POLES( LDGNUM, * ),
     $                   VF( * ), VL( * ), WORK( * ), Z( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
```

```
*     ..
*     .. Local Scalars ..
      INTEGER            I, IDX, IDXC, IDXP, ISIGMA, IVFW, IVLW, IW, M,
     $                   N, N1, N2
      REAL               ORGNRM
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SLAMRG, SLASCL, SLASD7, SLASD8, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      N = NL + NR + 1
      M = N + SQRE
*
      IF( ( ICOMPQ.LT.0 ) .OR. ( ICOMPQ.GT.1 ) ) THEN
         INFO = -1
      ELSE IF( NL.LT.1 ) THEN
         INFO = -2
      ELSE IF( NR.LT.1 ) THEN
         INFO = -3
      ELSE IF( ( SQRE.LT.0 ) .OR. ( SQRE.GT.1 ) ) THEN
         INFO = -4
      ELSE IF( LDGCOL.LT.N ) THEN
         INFO = -14
      ELSE IF( LDGNUM.LT.N ) THEN
         INFO = -16
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SLASD6', -INFO )
         RETURN
      END IF
*
*     The following values are for bookkeeping purposes only.  They are
*     integer pointers which indicate the portion of the workspace
*     used by a particular array in SLASD7 and SLASD8.
*
      ISIGMA = 1
      IW = ISIGMA + N
      IVFW = IW + M
      IVLW = IVFW + M
*
      IDX = 1
      IDXC = IDX + N
      IDXP = IDXC + N
```

```
*
*     Scale.
*
      ORGNRM = MAX( ABS( ALPHA ), ABS( BETA ) )
      D( NL+1 ) = ZERO
      DO 10 I = 1, N
         IF( ABS( D( I ) ).GT.ORGNRM ) THEN
            ORGNRM = ABS( D( I ) )
         END IF
   10 CONTINUE
      CALL SLASCL( 'G', 0, 0, ORGNRM, ONE, N, 1, D, N, INFO )
      ALPHA = ALPHA / ORGNRM
      BETA = BETA / ORGNRM
*
*     Sort and Deflate singular values.
*
      CALL SLASD7( ICOMPQ, NL, NR, SQRE, K, D, Z, WORK( IW ), VF,
     $             WORK( IVFW ), VL, WORK( IVLW ), ALPHA, BETA,
     $             WORK( ISIGMA ), IWORK( IDX ), IWORK( IDXP ), IDXQ,
     $             PERM, GIVPTR, GIVCOL, LDGCOL, GIVNUM, LDGNUM, C, S,
     $             INFO )
*
*     Solve Secular Equation, compute DIFL, DIFR, and update VF, VL.
*
      CALL SLASD8( ICOMPQ, K, D, Z, VF, VL, DIFL, DIFR, LDGNUM,
     $             WORK( ISIGMA ), WORK( IW ), INFO )
*
*     Report the possible convergence failure.
*
      IF( INFO.NE.0 ) THEN
         RETURN
      END IF
*
*     Save the poles if ICOMPQ = 1.
*
      IF( ICOMPQ.EQ.1 ) THEN
         CALL SCOPY( K, D, 1, POLES( 1, 1 ), 1 )
         CALL SCOPY( K, WORK( ISIGMA ), 1, POLES( 1, 2 ), 1 )
      END IF
*
*     Unscale.
*
      CALL SLASCL( 'G', 0, 0, ONE, ORGNRM, N, 1, D, N, INFO )
*
*     Prepare the IDXQ sorting permutation.
*
      N1 = K
      N2 = N - K
      CALL SLAMRG( N1, N2, D, 1, -1, IDXQ )
*
```

```
      RETURN
*
*     End of SLASD6
*
      END
```

———-

90.98.4 lisp code

— **slasd6.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun slasd6
         (icompq nl nr sqre d vf vl alpha beta idxq perm givptr givcol ldgcol
          givnum ldgnum poles difl difr z k c s work iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork givcol perm idxq)
             (type (single-float) s c beta alpha)
             (type (array single-float (*)) work z difr difl poles givnum vl vf
              d)
             (type (f2cl-lib:integer4) info k ldgnum ldgcol givptr sqre nr nl
              icompq))
    (f2cl-lib:with-multi-array-data
        ((d single-float d-%data% d-%offset%)
         (vf single-float vf-%data% vf-%offset%)
         (vl single-float vl-%data% vl-%offset%)
         (givnum single-float givnum-%data% givnum-%offset%)
         (poles single-float poles-%data% poles-%offset%)
         (difl single-float difl-%data% difl-%offset%)
         (difr single-float difr-%data% difr-%offset%)
         (z single-float z-%data% z-%offset%)
         (work single-float work-%data% work-%offset%)
         (idxq f2cl-lib:integer4 idxq-%data% idxq-%offset%)
         (perm f2cl-lib:integer4 perm-%data% perm-%offset%)
         (givcol f2cl-lib:integer4 givcol-%data% givcol-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((orgnrm 0.0) (i 0) (idx 0) (idxc 0) (idxp 0) (isigma 0) (ivfw 0)
             (ivlw 0) (iw 0) (m 0) (n 0) (n1 0) (n2 0))
        (declare (type (single-float) orgnrm)
                 (type (f2cl-lib:integer4) i idx idxc idxp isigma ivfw ivlw iw
                  m n n1 n2))
        (setf info 0)
        (setf n (f2cl-lib:int-add nl nr 1))
        (setf m (f2cl-lib:int-add n sqre))
        (cond ((or (< icompq 0) (> icompq 1)) (setf info -1))
```

```
      ((< nl 1) (setf info -2)) ((< nr 1) (setf info -3))
      ((or (< sqre 0) (> sqre 1)) (setf info -4))
      ((< ldgcol n) (setf info -14)) ((< ldgnum n) (setf info -16)))
(cond
 ((/= info 0) (xerbla "SLASD6" (f2cl-lib:int-sub info))
  (go end_label)))
(setf isigma 1)
(setf iw (f2cl-lib:int-add isigma n))
(setf ivfw (f2cl-lib:int-add iw m))
(setf ivlw (f2cl-lib:int-add ivfw m))
(setf idx 1)
(setf idxc (f2cl-lib:int-add idx n))
(setf idxp (f2cl-lib:int-add idxc n))
(setf orgnrm (max (abs alpha) (abs beta)))
(setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add nl 1)) ((1 *))
                     d-%offset%)
        zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (abs (f2cl-lib:fref d (i) ((1 *)))) orgnrm)
      (setf orgnrm
              (abs (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))))
   label10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (slascl "G" 0 0 orgnrm one n 1 d n info)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8))
  (setf info var-9))
(setf alpha (/ alpha orgnrm))
(setf beta (/ beta orgnrm))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19
     var-20 var-21 var-22 var-23 var-24 var-25 var-26)
    (slasd7 icompq nl nr sqre k d z
     (f2cl-lib:array-slice work-%data% single-float (iw) ((1 *))
                           work-%offset%)
     vf
     (f2cl-lib:array-slice work-%data% single-float (ivfw) ((1 *))
                           work-%offset%)
     vl
     (f2cl-lib:array-slice work-%data% single-float (ivlw) ((1 *))
                           work-%offset%)
     alpha beta
     (f2cl-lib:array-slice work-%data% single-float (isigma) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (idx) ((1 *))
```

```
                          iwork-%offset%)
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (idxp)
                          ((1 *)) iwork-%offset%)
      idxq perm givptr givcol ldgcol givnum ldgnum c s info)
  (declare
   (ignore var-5 var-6 var-7 var-8 var-9 var-10 var-11 var-14 var-15
    var-16 var-17 var-18 var-20 var-22))
  (when var-0 (setf icompq var-0))
  (when var-1 (setf nl var-1))
  (when var-2 (setf nr var-2))
  (when var-3 (setf sqre var-3))
  (when var-4 (setf k var-4))
  (when var-12 (setf alpha var-12))
  (when var-13 (setf beta var-13))
  (when var-19 (setf givptr var-19))
  (when var-21 (setf ldgcol var-21))
  (when var-23 (setf ldgnum var-23))
  (when var-24 (setf c var-24))
  (when var-25 (setf s var-25))
  (when var-26 (setf info var-26)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11)
    (slasd8 icompq k d z vf vl difl difr ldgnum
     (f2cl-lib:array-slice work-%data% single-float (isigma) ((1 *))
                          work-%offset%)
     (f2cl-lib:array-slice work-%data% single-float (iw) ((1 *))
                          work-%offset%)
     info)
  (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-9 var-10))
  (when var-0 (setf icompq var-0))
  (when var-1 (setf k var-1))
  (when var-8 (setf ldgnum var-8))
  (when var-11 (setf info var-11)))
(cond ((/= info 0) (go end_label)))
(cond
 ((= icompq 1)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (scopy k d 1
       (f2cl-lib:array-slice poles-%data% single-float (1 1)
                            ((1 ldgnum) (1 *)) poles-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf k var-0)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (scopy k
       (f2cl-lib:array-slice work-%data% single-float (isigma) ((1 *))
                            work-%offset%)
       1
       (f2cl-lib:array-slice poles-%data% single-float (1 2)
```

```
                                    ((1 ldgnum) (1 *)) poles-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf k var-0)))))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
            (slascl "G" 0 0 one orgnrm n 1 d n info)
          (declare
           (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8))
          (setf info var-9))
        (setf n1 k)
        (setf n2 (f2cl-lib:int-sub n k))
        (slamrg n1 n2 d 1 -1 idxq)
        (go end_label)
       end_label
        (return
         (values icompq nl nr sqre nil nil nil alpha beta nil nil givptr nil
                 ldgcol nil ldgnum nil nil nil nil k c s nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slasd6
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (array single-float (*))
```

```
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::icompq
  fortran-to-lisp::nl
  fortran-to-lisp::nr
  fortran-to-lisp::sqre nil nil nil
  fortran-to-lisp::alpha
  fortran-to-lisp::beta nil nil
  fortran-to-lisp::givptr nil
  fortran-to-lisp::ldgcol nil
  fortran-to-lisp::ldgnum nil nil
  nil nil fortran-to-lisp::k
  fortran-to-lisp::c
  fortran-to-lisp::s nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::slamrg
  fortran-to-lisp::slascl))))
```

———-

90.99 slasd7

90.99.1 Axiom unit tests

— **slasd7.input** —

```
)set break resume
)sys rm -f slasd7.output
)spool slasd7.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.99.2 Axiom help page

— slasd7.help —

```
=====================================================================
slasd7 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slasd7 (ICOMPQ, NL, NR, SQRE, K, D, Z, ZW, VF, VFW, VL, VLW,
           ALPHA, BETA, DSIGMA, IDX, IDXP, IDXQ, PERM, GIVPTR, GIVCOL, LDGCOL,
           GIVNUM, LDGNUM, C, S, INFO)
           SLASD7 merges the two sets of singular values together into a
           single sorted set. Then it tries to deflate the size of the
           problem. Used by sbdsdc.

Function/Subroutine Documentation
   subroutine slasd7 (integer ICOMPQ, integer NL, integer NR, integer SQRE,
       integer K, real, dimension( * ) D, real, dimension( * ) Z, real,
       dimension( * ) ZW, real, dimension( * ) VF, real, dimension( * ) VFW,
       real, dimension( * ) VL, real, dimension( * ) VLW, real ALPHA, real
       BETA, real, dimension( * ) DSIGMA, integer, dimension( * ) IDX,
       integer, dimension( * ) IDXP, integer, dimension( * ) IDXQ, integer,
       dimension( * ) PERM, integer GIVPTR, integer, dimension( ldgcol, * )
       GIVCOL, integer LDGCOL, real, dimension( ldgnum, * ) GIVNUM, integer
       LDGNUM, real C, real S, integer INFO)
       SLASD7 merges the two sets of singular values together into a single
       sorted set. Then it tries to deflate the size of the problem. Used by
       sbdsdc.

Purpose:
   SLASD7 merges the two sets of singular values together into a single
   sorted set. Then it tries to deflate the size of the problem. There
   are two ways in which deflation can occur:  when two or more singular
   values are close together or if there is a tiny entry in the Z
   vector. For each such occurrence the order of the related
   secular equation problem is reduced by one.

   SLASD7 is called from SLASD6.

Parameters:

 ICOMPQ
   ICOMPQ is INTEGER
```

```
  Specifies whether singular vectors are to be computed
  in compact form, as follows:
  = 0: Compute singular values only.
  = 1: Compute singular vectors of upper bidiagonal matrix in compact form.

NL
  NL is INTEGER
  The row dimension of the upper block. NL >= 1.

NR
  NR is INTEGER
  The row dimension of the lower block. NR >= 1.

SQRE
  SQRE is INTEGER
  = 0: the lower block is an NR-by-NR square matrix.
  = 1: the lower block is an NR-by-(NR+1) rectangular matrix.

  The bidiagonal matrix has
  N = NL + NR + 1 rows and
  M = N + SQRE >= N columns.

K
  K is INTEGER
  Contains the dimension of the non-deflated matrix, this is
  the order of the related secular equation. 1 <= K <=N.

D
  D is REAL array, dimension ( N )
  On entry D contains the singular values of the two submatrices
  to be combined. On exit D contains the trailing (N-K) updated
  singular values (those which were deflated) sorted into increasing order.

Z
  Z is REAL array, dimension ( M )
  On exit Z contains the updating row vector in the secular equation.

ZW
  ZW is REAL array, dimension ( M )
  Workspace for Z.

VF
  VF is REAL array, dimension ( M )
  On entry, VF(1:NL+1) contains the first components of all
  right singular vectors of the upper block; and VF(NL+2:M)
  contains the first components of all right singular vectors
  of the lower block. On exit, VF contains the first components
  of all right singular vectors of the bidiagonal matrix.

VFW
```

```
  VFW is REAL array, dimension ( M )
  Workspace for VF.

VL
  VL is REAL array, dimension ( M )
  On entry, VL(1:NL+1) contains the  last components of all
  right singular vectors of the upper block; and VL(NL+2:M)
  contains the last components of all right singular vectors
  of the lower block. On exit, VL contains the last components
  of all right singular vectors of the bidiagonal matrix.

VLW
  VLW is REAL array, dimension ( M )
  Workspace for VL.

ALPHA
  ALPHA is REAL
  Contains the diagonal element associated with the added row.

BETA
  BETA is REAL
  Contains the off-diagonal element associated with the added row.

DSIGMA
  DSIGMA is REAL array, dimension ( N )
  Contains a copy of the diagonal elements (K-1 singular values
  and one zero) in the secular equation.

IDX
  IDX is INTEGER array, dimension ( N )
  This will contain the permutation used to sort the contents of
  D into ascending order.

IDXP
  IDXP is INTEGER array, dimension ( N )
  This will contain the permutation used to place deflated
  values of D at the end of the array. On output IDXP(2:K)
  points to the nondeflated D-values and IDXP(K+1:N)
  points to the deflated singular values.

IDXQ
  IDXQ is INTEGER array, dimension ( N )
  This contains the permutation which separately sorts the two
  sub-problems in D into ascending order.  Note that entries in
  the first half of this permutation must first be moved one
  position backward; and entries in the second half
  must first have NL+1 added to their values.

PERM
  PERM is INTEGER array, dimension ( N )
```

```
  The permutations (from deflation and sorting) to be applied
  to each singular block. Not referenced if ICOMPQ = 0.

GIVPTR
  GIVPTR is INTEGER
  The number of Givens rotations which took place in this
  subproblem. Not referenced if ICOMPQ = 0.

GIVCOL
  GIVCOL is INTEGER array, dimension ( LDGCOL, 2 )
  Each pair of numbers indicates a pair of columns to take place
  in a Givens rotation. Not referenced if ICOMPQ = 0.

LDGCOL
  LDGCOL is INTEGER
  The leading dimension of GIVCOL, must be at least N.

GIVNUM
  GIVNUM is REAL array, dimension ( LDGNUM, 2 )
  Each number indicates the C or S value to be used in the
  corresponding Givens rotation. Not referenced if ICOMPQ = 0.

LDGNUM
  LDGNUM is INTEGER
  The leading dimension of GIVNUM, must be at least N.

C
  C is REAL
  C contains garbage if SQRE =0 and the C-value of a Givens
  rotation related to the right null space if SQRE = 1.

S
  S is REAL
  S contains garbage if SQRE =0 and the S-value of a Givens
  rotation related to the right null space if SQRE = 1.

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

———-

90.99.3 fortran code

— slasd7.f —

```
*  =====================================================================
      SUBROUTINE SLASD7( ICOMPQ, NL, NR, SQRE, K, D, Z, ZW, VF, VFW, VL,
     $                   VLW, ALPHA, BETA, DSIGMA, IDX, IDXP, IDXQ,
     $                   PERM, GIVPTR, GIVCOL, LDGCOL, LDGNUM, LDGNUM,
     $                   C, S, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            GIVPTR, ICOMPQ, INFO, K, LDGCOL, LDGNUM, NL,
     $                   NR, SQRE
      REAL               ALPHA, BETA, C, S
*     ..
*     .. Array Arguments ..
      INTEGER            GIVCOL( LDGCOL, * ), IDX( * ), IDXP( * ),
     $                   IDXQ( * ), PERM( * )
      REAL               D( * ), DSIGMA( * ), GIVNUM( LDGNUM, * ),
     $                   VF( * ), VFW( * ), VL( * ), VLW( * ), Z( * ),
     $                   ZW( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE, TWO, EIGHT
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0, TWO = 2.0E+0,
     $                   EIGHT = 8.0E+0 )
*     ..
*     .. Local Scalars ..
*
      INTEGER            I, IDXI, IDXJ, IDXJP, J, JP, JPREV, K2, M, N,
     $                   NLP1, NLP2
      REAL               EPS, HLFTOL, TAU, TOL, Z1
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SLAMRG, SROT, XERBLA
*     ..
*     .. External Functions ..
      REAL               SLAMCH, SLAPY2
      EXTERNAL           SLAMCH, SLAPY2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
```

```
*
      INFO = 0
      N = NL + NR + 1
      M = N + SQRE
*
      IF( ( ICOMPQ.LT.0 ) .OR. ( ICOMPQ.GT.1 ) ) THEN
         INFO = -1
      ELSE IF( NL.LT.1 ) THEN
         INFO = -2
      ELSE IF( NR.LT.1 ) THEN
         INFO = -3
      ELSE IF( ( SQRE.LT.0 ) .OR. ( SQRE.GT.1 ) ) THEN
         INFO = -4
      ELSE IF( LDGCOL.LT.N ) THEN
         INFO = -22
      ELSE IF( LDGNUM.LT.N ) THEN
         INFO = -24
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SLASD7', -INFO )
         RETURN
      END IF
*
      NLP1 = NL + 1
      NLP2 = NL + 2
      IF( ICOMPQ.EQ.1 ) THEN
         GIVPTR = 0
      END IF
*
*     Generate the first part of the vector Z and move the singular
*     values in the first part of D one position backward.
*
      Z1 = ALPHA*VL( NLP1 )
      VL( NLP1 ) = ZERO
      TAU = VF( NLP1 )
      DO 10 I = NL, 1, -1
         Z( I+1 ) = ALPHA*VL( I )
         VL( I ) = ZERO
         VF( I+1 ) = VF( I )
         D( I+1 ) = D( I )
         IDXQ( I+1 ) = IDXQ( I ) + 1
   10 CONTINUE
      VF( 1 ) = TAU
*
*     Generate the second part of the vector Z.
*
      DO 20 I = NLP2, M
         Z( I ) = BETA*VF( I )
         VF( I ) = ZERO
   20 CONTINUE
```

```
*
*     Sort the singular values into increasing order
*
      DO 30 I = NLP2, N
         IDXQ( I ) = IDXQ( I ) + NLP1
   30 CONTINUE
*
*     DSIGMA, IDXC, IDXC, and ZW are used as storage space.
*
      DO 40 I = 2, N
         DSIGMA( I ) = D( IDXQ( I ) )
         ZW( I ) = Z( IDXQ( I ) )
         VFW( I ) = VF( IDXQ( I ) )
         VLW( I ) = VL( IDXQ( I ) )
   40 CONTINUE
*
      CALL SLAMRG( NL, NR, DSIGMA( 2 ), 1, 1, IDX( 2 ) )
*
      DO 50 I = 2, N
         IDXI = 1 + IDX( I )
         D( I ) = DSIGMA( IDXI )
         Z( I ) = ZW( IDXI )
         VF( I ) = VFW( IDXI )
         VL( I ) = VLW( IDXI )
   50 CONTINUE
*
*     Calculate the allowable deflation tolerence
*
      EPS = SLAMCH( 'Epsilon' )
      TOL = MAX( ABS( ALPHA ), ABS( BETA ) )
      TOL = EIGHT*EIGHT*EPS*MAX( ABS( D( N ) ), TOL )
*
*     There are 2 kinds of deflation -- first a value in the z-vector
*     is small, second two (or more) singular values are very close
*     together (their difference is small).
*
*     If the value in the z-vector is small, we simply permute the
*     array so that the corresponding singular value is moved to the
*     end.
*
*     If two values in the D-vector are close, we perform a two-sided
*     rotation designed to make one of the corresponding z-vector
*     entries zero, and then permute the array so that the deflated
*     singular value is moved to the end.
*
*     If there are multiple singular values then the problem deflates.
*     Here the number of equal singular values are found.  As each equal
*     singular value is found, an elementary reflector is computed to
*     rotate the corresponding singular subspace so that the
*     corresponding components of Z are zero in this new basis.
```

```
*
      K = 1
      K2 = N + 1
      DO 60 J = 2, N
         IF( ABS( Z( J ) ).LE.TOL ) THEN
*
*           Deflate due to small z component.
*
            K2 = K2 - 1
            IDXP( K2 ) = J
            IF( J.EQ.N )
     $         GO TO 100
         ELSE
            JPREV = J
            GO TO 70
         END IF
   60 CONTINUE
   70 CONTINUE
      J = JPREV
   80 CONTINUE
      J = J + 1
      IF( J.GT.N )
     $   GO TO 90
      IF( ABS( Z( J ) ).LE.TOL ) THEN
*
*        Deflate due to small z component.
*
         K2 = K2 - 1
         IDXP( K2 ) = J
      ELSE
*
*        Check if singular values are close enough to allow deflation.
*
         IF( ABS( D( J )-D( JPREV ) ).LE.TOL ) THEN
*
*           Deflation is possible.
*
            S = Z( JPREV )
            C = Z( J )
*
*           Find sqrt(a**2+b**2) without overflow or
*           destructive underflow.
*
            TAU = SLAPY2( C, S )
            Z( J ) = TAU
            Z( JPREV ) = ZERO
            C = C / TAU
            S = -S / TAU
*
*           Record the appropriate Givens rotation
```

```
*
            IF( ICOMPQ.EQ.1 ) THEN
               GIVPTR = GIVPTR + 1
               IDXJP = IDXQ( IDX( JPREV )+1 )
               IDXJ = IDXQ( IDX( J )+1 )
               IF( IDXJP.LE.NLP1 ) THEN
                  IDXJP = IDXJP - 1
               END IF
               IF( IDXJ.LE.NLP1 ) THEN
                  IDXJ = IDXJ - 1
               END IF
               GIVCOL( GIVPTR, 2 ) = IDXJP
               GIVCOL( GIVPTR, 1 ) = IDXJ
               GIVNUM( GIVPTR, 2 ) = C
               GIVNUM( GIVPTR, 1 ) = S
            END IF
            CALL SROT( 1, VF( JPREV ), 1, VF( J ), 1, C, S )
            CALL SROT( 1, VL( JPREV ), 1, VL( J ), 1, C, S )
            K2 = K2 - 1
            IDXP( K2 ) = JPREV
            JPREV = J
         ELSE
            K = K + 1
            ZW( K ) = Z( JPREV )
            DSIGMA( K ) = D( JPREV )
            IDXP( K ) = JPREV
            JPREV = J
         END IF
      END IF
      GO TO 80
   90 CONTINUE
*
*     Record the last singular value.
*
      K = K + 1
      ZW( K ) = Z( JPREV )
      DSIGMA( K ) = D( JPREV )
      IDXP( K ) = JPREV
*
  100 CONTINUE
*
*     Sort the singular values into DSIGMA. The singular values which
*     were not deflated go into the first K slots of DSIGMA, except
*     that DSIGMA(1) is treated separately.
*
      DO 110 J = 2, N
         JP = IDXP( J )
         DSIGMA( J ) = D( JP )
         VFW( J ) = VF( JP )
         VLW( J ) = VL( JP )
```

```
  110 CONTINUE
      IF( ICOMPQ.EQ.1 ) THEN
         DO 120 J = 2, N
            JP = IDXP( J )
            PERM( J ) = IDXQ( IDX( JP )+1 )
            IF( PERM( J ).LE.NLP1 ) THEN
               PERM( J ) = PERM( J ) - 1
            END IF
  120    CONTINUE
      END IF
*
*     The deflated singular values go back into the last N - K slots of
*     D.
*
      CALL SCOPY( N-K, DSIGMA( K+1 ), 1, D( K+1 ), 1 )
*
*     Determine DSIGMA(1), DSIGMA(2), Z(1), VF(1), VL(1), VF(M), and
*     VL(M).
*
      DSIGMA( 1 ) = ZERO
      HLFTOL = TOL / TWO
      IF( ABS( DSIGMA( 2 ) ).LE.HLFTOL )
     $   DSIGMA( 2 ) = HLFTOL
      IF( M.GT.N ) THEN
         Z( 1 ) = SLAPY2( Z1, Z( M ) )
         IF( Z( 1 ).LE.TOL ) THEN
            C = ONE
            S = ZERO
            Z( 1 ) = TOL
         ELSE
            C = Z1 / Z( 1 )
            S = -Z( M ) / Z( 1 )
         END IF
         CALL SROT( 1, VF( M ), 1, VF( 1 ), 1, C, S )
         CALL SROT( 1, VL( M ), 1, VL( 1 ), 1, C, S )
      ELSE
         IF( ABS( Z1 ).LE.TOL ) THEN
            Z( 1 ) = TOL
         ELSE
            Z( 1 ) = Z1
         END IF
      END IF
*
*     Restore Z, VF, and VL.
*
      CALL SCOPY( K-1, ZW( 2 ), 1, Z( 2 ), 1 )
      CALL SCOPY( N-1, VFW( 2 ), 1, VF( 2 ), 1 )
      CALL SCOPY( N-1, VLW( 2 ), 1, VL( 2 ), 1 )
*
      RETURN
```

```
*
*     End of SLASD7
*
      END
```

———-

90.99.4 lisp code

— **slasd7.lisp** —

```
(let* ((zero 0.0) (one 1.0) (two 2.0) (eight 8.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 8.0 8.0) eight)
           (ignorable zero one two eight))
  (defun slasd7
         (icompq nl nr sqre k d z zw vf vfw vl vlw alpha beta dsigma idx idxp
          idxq perm givptr givcol ldgcol givnum ldgnum c s info)
    (declare (type (array f2cl-lib:integer4 (*)) givcol perm idxq idxp idx)
             (type (single-float) s c beta alpha)
             (type (array single-float (*)) givnum dsigma vlw vl vfw vf zw z d)
             (type (f2cl-lib:integer4) info ldgnum ldgcol givptr k sqre nr nl
              icompq))
    (f2cl-lib:with-multi-array-data
        ((d single-float d-%data% d-%offset%)
         (z single-float z-%data% z-%offset%)
         (zw single-float zw-%data% zw-%offset%)
         (vf single-float vf-%data% vf-%offset%)
         (vfw single-float vfw-%data% vfw-%offset%)
         (vl single-float vl-%data% vl-%offset%)
         (vlw single-float vlw-%data% vlw-%offset%)
         (dsigma single-float dsigma-%data% dsigma-%offset%)
         (givnum single-float givnum-%data% givnum-%offset%)
         (idx f2cl-lib:integer4 idx-%data% idx-%offset%)
         (idxp f2cl-lib:integer4 idxp-%data% idxp-%offset%)
         (idxq f2cl-lib:integer4 idxq-%data% idxq-%offset%)
         (perm f2cl-lib:integer4 perm-%data% perm-%offset%)
         (givcol f2cl-lib:integer4 givcol-%data% givcol-%offset%))
      (prog ((eps 0.0) (hlftol 0.0) (tau 0.0) (tol 0.0) (z1 0.0) (i 0) (idxi 0)
             (idxj 0) (idxjp 0) (j 0) (jp 0) (jprev 0) (k2 0) (m 0) (n 0)
             (nlp1 0) (nlp2 0))
        (declare (type (single-float) eps hlftol tau tol z1)
                 (type (f2cl-lib:integer4) i idxi idxj idxjp j jp jprev k2 m n
                  nlp1 nlp2))
        (setf info 0)
        (setf n (f2cl-lib:int-add nl nr 1))
```

```
(setf m (f2cl-lib:int-add n sqre))
(cond ((or (< icompq 0) (> icompq 1)) (setf info -1))
      ((< nl 1) (setf info -2)) ((< nr 1) (setf info -3))
      ((or (< sqre 0) (> sqre 1)) (setf info -4))
      ((< ldgcol n) (setf info -22)) ((< ldgnum n) (setf info -24)))
(cond
 ((/= info 0) (xerbla "SLASD7" (f2cl-lib:int-sub info))
  (go end_label)))
(setf nlp1 (f2cl-lib:int-add nl 1))
(setf nlp2 (f2cl-lib:int-add nl 2))
(cond ((= icompq 1) (setf givptr 0)))
(setf z1
        (* alpha (f2cl-lib:fref vl-%data% (nlp1) ((1 *)) vl-%offset%)))
(setf (f2cl-lib:fref vl-%data% (nlp1) ((1 *)) vl-%offset%) zero)
(setf tau (f2cl-lib:fref vf-%data% (nlp1) ((1 *)) vf-%offset%))
(f2cl-lib:fdo (i nl (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         z-%offset%)
            (* alpha
               (f2cl-lib:fref vl-%data% (i) ((1 *)) vl-%offset%)))
    (setf (f2cl-lib:fref vl-%data% (i) ((1 *)) vl-%offset%) zero)
    (setf (f2cl-lib:fref vf-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         vf-%offset%)
            (f2cl-lib:fref vf-%data% (i) ((1 *)) vf-%offset%))
    (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         d-%offset%)
            (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
    (setf (f2cl-lib:fref idxq-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         idxq-%offset%)
            (f2cl-lib:int-add
             (f2cl-lib:fref idxq-%data% (i) ((1 *)) idxq-%offset%) 1))
   label10))
(setf (f2cl-lib:fref vf-%data% (1) ((1 *)) vf-%offset%) tau)
(f2cl-lib:fdo (i nlp2 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
            (* beta (f2cl-lib:fref vf-%data% (i) ((1 *)) vf-%offset%)))
    (setf (f2cl-lib:fref vf-%data% (i) ((1 *)) vf-%offset%) zero)
   label20))
(f2cl-lib:fdo (i nlp2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref idxq-%data% (i) ((1 *)) idxq-%offset%)
            (f2cl-lib:int-add
             (f2cl-lib:fref idxq-%data% (i) ((1 *)) idxq-%offset%)
             nlp1))
   label30))
```

```
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref dsigma-%data% (i) ((1 *)) dsigma-%offset%)
            (f2cl-lib:fref d-%data% ((f2cl-lib:fref idxq (i) ((1 *))))
                           ((1 *)) d-%offset%))
    (setf (f2cl-lib:fref zw-%data% (i) ((1 *)) zw-%offset%)
            (f2cl-lib:fref z-%data% ((f2cl-lib:fref idxq (i) ((1 *))))
                           ((1 *)) z-%offset%))
    (setf (f2cl-lib:fref vfw-%data% (i) ((1 *)) vfw-%offset%)
            (f2cl-lib:fref vf-%data% ((f2cl-lib:fref idxq (i) ((1 *))))
                           ((1 *)) vf-%offset%))
    (setf (f2cl-lib:fref vlw-%data% (i) ((1 *)) vlw-%offset%)
            (f2cl-lib:fref vl-%data% ((f2cl-lib:fref idxq (i) ((1 *))))
                           ((1 *)) vl-%offset%))
   label40))
(slamrg nl nr
 (f2cl-lib:array-slice dsigma-%data% single-float (2) ((1 *))
                       dsigma-%offset%)
 1 1
 (f2cl-lib:array-slice idx-%data% f2cl-lib:integer4 (2) ((1 *))
                       idx-%offset%))
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf idxi
            (f2cl-lib:int-add 1
                              (f2cl-lib:fref idx-%data% (i) ((1 *))
                                             idx-%offset%)))
    (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
            (f2cl-lib:fref dsigma-%data% (idxi) ((1 *))
                           dsigma-%offset%))
    (setf (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
            (f2cl-lib:fref zw-%data% (idxi) ((1 *)) zw-%offset%))
    (setf (f2cl-lib:fref vf-%data% (i) ((1 *)) vf-%offset%)
            (f2cl-lib:fref vfw-%data% (idxi) ((1 *)) vfw-%offset%))
    (setf (f2cl-lib:fref vl-%data% (i) ((1 *)) vl-%offset%)
            (f2cl-lib:fref vlw-%data% (idxi) ((1 *)) vlw-%offset%))
   label50))
(setf eps (slamch "Epsilon"))
(setf tol (max (abs alpha) (abs beta)))
(setf tol
        (* eight eight eps
           (max (abs (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%))
                tol)))
(setf k 1)
(setf k2 (f2cl-lib:int-add n 1))
(f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
```

```
      (cond
       ((<= (abs (f2cl-lib:fref z (j) ((1 *)))) tol)
        (setf k2 (f2cl-lib:int-sub k2 1))
        (setf (f2cl-lib:fref idxp-%data% (k2) ((1 *)) idxp-%offset%) j)
        (if (= j n)
            (go label100)))
       (t (setf jprev j) (go label70)))
     label60))
 label70
  (setf j jprev)
 label80
  (setf j (f2cl-lib:int-add j 1))
  (if (> j n)
      (go label90))
  (cond
   ((<= (abs (f2cl-lib:fref z (j) ((1 *)))) tol)
    (setf k2 (f2cl-lib:int-sub k2 1))
    (setf (f2cl-lib:fref idxp-%data% (k2) ((1 *)) idxp-%offset%) j))
   (t
    (cond
     ((<=
       (abs
        (+ (f2cl-lib:fref d (j) ((1 *)))
           (- (f2cl-lib:fref d (jprev) ((1 *))))))
       tol)
      (setf s (f2cl-lib:fref z-%data% (jprev) ((1 *)) z-%offset%))
      (setf c (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%))
      (setf tau (slapy2 c s))
      (setf (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%) tau)
      (setf (f2cl-lib:fref z-%data% (jprev) ((1 *)) z-%offset%) zero)
      (setf c (/ c tau)) (setf s (/ (- s) tau))
      (cond
       ((= icompq 1) (setf givptr (f2cl-lib:int-add givptr 1))
        (setf idxjp
                (f2cl-lib:fref idxq-%data%
                               ((f2cl-lib:int-add
                                 (f2cl-lib:fref idx (jprev) ((1 *))) 1))
                               ((1 *)) idxq-%offset%))
        (setf idxj
                (f2cl-lib:fref idxq-%data%
                               ((f2cl-lib:int-add
                                 (f2cl-lib:fref idx (j) ((1 *))) 1))
                               ((1 *)) idxq-%offset%))
        (cond ((<= idxjp nlp1) (setf idxjp (f2cl-lib:int-sub idxjp 1))))
        (cond ((<= idxj nlp1) (setf idxj (f2cl-lib:int-sub idxj 1))))
        (setf (f2cl-lib:fref givcol-%data% (givptr 2) ((1 ldgcol) (1 *))
                             givcol-%offset%)
                idxjp)
        (setf (f2cl-lib:fref givcol-%data% (givptr 1) ((1 ldgcol) (1 *))
                             givcol-%offset%)
```

```
                idxj)
        (setf (f2cl-lib:fref givnum-%data% (givptr 2) ((1 ldgnum) (1 *))
                             givnum-%offset%)
                c)
        (setf (f2cl-lib:fref givnum-%data% (givptr 1) ((1 ldgnum) (1 *))
                             givnum-%offset%)
                s)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (srot 1
           (f2cl-lib:array-slice vf-%data% single-float (jprev) ((1 *))
                                 vf-%offset%)
           1
           (f2cl-lib:array-slice vf-%data% single-float (j) ((1 *))
                                 vf-%offset%)
           1 c s)
        (declare (ignore var-0 var-1 var-2 var-3 var-4))
        (when var-5 (setf c var-5))
        (when var-6 (setf s var-6)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (srot 1
           (f2cl-lib:array-slice vl-%data% single-float (jprev) ((1 *))
                                 vl-%offset%)
           1
           (f2cl-lib:array-slice vl-%data% single-float (j) ((1 *))
                                 vl-%offset%)
           1 c s)
        (declare (ignore var-0 var-1 var-2 var-3 var-4))
        (when var-5 (setf c var-5))
        (when var-6 (setf s var-6)))
      (setf k2 (f2cl-lib:int-sub k2 1))
      (setf (f2cl-lib:fref idxp-%data% (k2) ((1 *)) idxp-%offset%) jprev)
      (setf jprev j))
     (t (setf k (f2cl-lib:int-add k 1))
      (setf (f2cl-lib:fref zw-%data% (k) ((1 *)) zw-%offset%)
              (f2cl-lib:fref z-%data% (jprev) ((1 *)) z-%offset%))
      (setf (f2cl-lib:fref dsigma-%data% (k) ((1 *)) dsigma-%offset%)
              (f2cl-lib:fref d-%data% (jprev) ((1 *)) d-%offset%))
      (setf (f2cl-lib:fref idxp-%data% (k) ((1 *)) idxp-%offset%) jprev)
      (setf jprev j)))))
  (go label80)
 label90
  (setf k (f2cl-lib:int-add k 1))
  (setf (f2cl-lib:fref zw-%data% (k) ((1 *)) zw-%offset%)
          (f2cl-lib:fref z-%data% (jprev) ((1 *)) z-%offset%))
  (setf (f2cl-lib:fref dsigma-%data% (k) ((1 *)) dsigma-%offset%)
          (f2cl-lib:fref d-%data% (jprev) ((1 *)) d-%offset%))
  (setf (f2cl-lib:fref idxp-%data% (k) ((1 *)) idxp-%offset%) jprev)
 label100
  (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                ((> j n) nil)
```

```
  (tagbody
    (setf jp (f2cl-lib:fref idxp-%data% (j) ((1 *)) idxp-%offset%))
    (setf (f2cl-lib:fref dsigma-%data% (j) ((1 *)) dsigma-%offset%)
            (f2cl-lib:fref d-%data% (jp) ((1 *)) d-%offset%))
    (setf (f2cl-lib:fref vfw-%data% (j) ((1 *)) vfw-%offset%)
            (f2cl-lib:fref vf-%data% (jp) ((1 *)) vf-%offset%))
    (setf (f2cl-lib:fref vlw-%data% (j) ((1 *)) vlw-%offset%)
            (f2cl-lib:fref vl-%data% (jp) ((1 *)) vl-%offset%))
   label110))
(cond
 ((= icompq 1)
  (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf jp (f2cl-lib:fref idxp-%data% (j) ((1 *)) idxp-%offset%))
      (setf (f2cl-lib:fref perm-%data% (j) ((1 *)) perm-%offset%)
              (f2cl-lib:fref idxq-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:fref idx (jp) ((1 *))) 1))
                             ((1 *)) idxq-%offset%))
      (cond
       ((<= (f2cl-lib:fref perm (j) ((1 *))) nlp1)
        (setf (f2cl-lib:fref perm-%data% (j) ((1 *)) perm-%offset%)
                (f2cl-lib:int-sub
                 (f2cl-lib:fref perm-%data% (j) ((1 *)) perm-%offset%)
                 1))))
     label120))))
(scopy (f2cl-lib:int-sub n k)
 (f2cl-lib:array-slice dsigma-%data% single-float ((+ k 1)) ((1 *))
                       dsigma-%offset%)
 1
 (f2cl-lib:array-slice d-%data% single-float ((+ k 1)) ((1 *))
                       d-%offset%)
 1)
(setf (f2cl-lib:fref dsigma-%data% (1) ((1 *)) dsigma-%offset%) zero)
(setf hlftol (/ tol two))
(if (<= (abs (f2cl-lib:fref dsigma-%data% (2) ((1 *)) dsigma-%offset%))
        hlftol)
    (setf (f2cl-lib:fref dsigma-%data% (2) ((1 *)) dsigma-%offset%)
            hlftol))
(cond
 ((> m n)
  (setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)
          (slapy2 z1 (f2cl-lib:fref z-%data% (m) ((1 *)) z-%offset%)))
  (cond
   ((<= (f2cl-lib:fref z (1) ((1 *))) tol) (setf c one) (setf s zero)
    (setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%) tol))
   (t (setf c (/ z1 (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)))
    (setf s
            (/ (- (f2cl-lib:fref z-%data% (m) ((1 *)) z-%offset%))
```

```
                (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)))))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (srot 1
         (f2cl-lib:array-slice vf-%data% single-float (m) ((1 *))
                               vf-%offset%)
         1
         (f2cl-lib:array-slice vf-%data% single-float (1) ((1 *))
                               vf-%offset%)
         1 c s)
      (declare (ignore var-0 var-1 var-2 var-3 var-4))
      (when var-5 (setf c var-5))
      (when var-6 (setf s var-6)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (srot 1
         (f2cl-lib:array-slice vl-%data% single-float (m) ((1 *))
                               vl-%offset%)
         1
         (f2cl-lib:array-slice vl-%data% single-float (1) ((1 *))
                               vl-%offset%)
         1 c s)
      (declare (ignore var-0 var-1 var-2 var-3 var-4))
      (when var-5 (setf c var-5))
      (when var-6 (setf s var-6))))
   (t
    (cond
     ((<= (abs z1) tol)
      (setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%) tol))
     (t (setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%) z1)))))
  (scopy (f2cl-lib:int-sub k 1)
   (f2cl-lib:array-slice zw-%data% single-float (2) ((1 *)) zw-%offset%)
   1 (f2cl-lib:array-slice z-%data% single-float (2) ((1 *)) z-%offset%)
   1)
  (scopy (f2cl-lib:int-sub n 1)
   (f2cl-lib:array-slice vfw-%data% single-float (2) ((1 *))
                         vfw-%offset%)
   1
   (f2cl-lib:array-slice vf-%data% single-float (2) ((1 *)) vf-%offset%)
   1)
  (scopy (f2cl-lib:int-sub n 1)
   (f2cl-lib:array-slice vlw-%data% single-float (2) ((1 *))
                         vlw-%offset%)
   1
   (f2cl-lib:array-slice vl-%data% single-float (2) ((1 *)) vl-%offset%)
   1)
  (go end_label)
 end_label
  (return
   (values nil nil nil nil k nil nil nil nil nil nil nil nil nil nil nil
           nil nil nil givptr nil nil nil nil c s info))))))
```

```
(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slasd7
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (single-float) (single-float)
                                              (array single-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil nil nil nil
                                              fortran-to-lisp::k nil nil nil
                                              nil nil nil nil nil nil nil nil
                                              nil nil nil
                                              fortran-to-lisp::givptr nil nil
                                              nil nil fortran-to-lisp::c
                                              fortran-to-lisp::s
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::slapy2
                                              fortran-to-lisp::slamrg))))
```

———-

90.100 slasd8

90.100.1 Axiom unit tests

— **slasd8.input** —

```
)set break resume
)sys rm -f slasd8.output
)spool slasd8.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.100.2 Axiom help page

— **slasd8.help** —

```
======================================================================
slasd8 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slasd8 (ICOMPQ, K, D, Z, VF, VL, DIFL, DIFR, LDDIFR, DSIGMA,
           WORK, INFO)
           SLASD8 finds the square roots of the roots of the secular equation,
           and stores, for each element in D, the distance to its two nearest
           poles. Used by sbdsdc.

Function/Subroutine Documentation
   subroutine slasd8 (integer ICOMPQ, integer K, real, dimension( * ) D, real,
       dimension( * ) Z, real, dimension( * ) VF, real, dimension( * ) VL,
       real, dimension( * ) DIFL, real, dimension( lddifr, * ) DIFR, integer
```

```
      LDDIFR, real, dimension( * ) DSIGMA, real, dimension( * ) WORK, integer
      INFO)
      SLASD8 finds the square roots of the roots of the secular equation, and
      stores, for each element in D, the distance to its two nearest poles.
      Used by sbdsdc.

Purpose:
   SLASD8 finds the square roots of the roots of the secular equation,
   as defined by the values in DSIGMA and Z. It makes the appropriate
   calls to SLASD4, and stores, for each  element in D, the distance
   to its two nearest poles (elements in DSIGMA). It also updates
   the arrays VF and VL, the first and last components of all the
   right singular vectors of the original bidiagonal matrix.

   SLASD8 is called from SLASD6.

Parameters:

 ICOMPQ
   ICOMPQ is INTEGER
   Specifies whether singular vectors are to be computed in
   factored form in the calling routine:
   = 0: Compute singular values only.
   = 1: Compute singular vectors in factored form as well.

 K
   K is INTEGER
   The number of terms in the rational function to be solved
   by SLASD4.  K >= 1.

 D
   D is REAL array, dimension ( K )
   On output, D contains the updated singular values.

 Z
   Z is REAL array, dimension ( K )
   On entry, the first K elements of this array contain the
   components of the deflation-adjusted updating row vector.
   On exit, Z is updated.

 VF
   VF is REAL array, dimension ( K )
   On entry, VF contains  information passed through DBEDE8.
   On exit, VF contains the first K components of the first
   components of all right singular vectors of the bidiagonal matrix.

 VL
   VL is REAL array, dimension ( K )
   On entry, VL contains  information passed through DBEDE8.
   On exit, VL contains the first K components of the last
```

```
  components of all right singular vectors of the bidiagonal matrix.

 DIFL
   DIFL is REAL array, dimension ( K )
   On exit, DIFL(I) = D(I) - DSIGMA(I).

 DIFR
   DIFR is REAL array,
      dimension ( LDDIFR, 2 ) if ICOMPQ = 1 and
      dimension ( K ) if ICOMPQ = 0.
   On exit, DIFR(I,1) = D(I) - DSIGMA(I+1), DIFR(K,1) is not
   defined and will not be referenced.

   If ICOMPQ = 1, DIFR(1:K,2) is an array containing the
   normalizing factors for the right singular vector matrix.

 LDDIFR
   LDDIFR is INTEGER
   The leading dimension of DIFR, must be at least K.

 DSIGMA
   DSIGMA is REAL array, dimension ( K )
   On entry, the first K elements of this array contain the old
   roots of the deflated updating problem.  These are the poles
   of the secular equation.
   On exit, the elements of DSIGMA may be very slightly altered in value.

 WORK
   WORK is REAL array, dimension at least 3 * K

 INFO
   INFO is INTEGER
   = 0:  successful exit.
   < 0:  if INFO = -i, the i-th argument had an illegal value.
   > 0:  if INFO = 1, a singular value did not converge
```

———-

90.100.3 fortran code

— **slasd8.f** —

```
*  =====================================================================
      SUBROUTINE SLASD8( ICOMPQ, K, D, Z, VF, VL, DIFL, DIFR, LDDIFR,
     $                   DSIGMA, WORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
```

```
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            ICOMPQ, INFO, K, LDDIFR
*     ..
*     .. Array Arguments ..
      REAL               D( * ), DIFL( * ), DIFR( LDDIFR, * ),
     $                   DSIGMA( * ), VF( * ), VL( * ), WORK( * ),
     $                   Z( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE
      PARAMETER          ( ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IWK1, IWK2, IWK2I, IWK3, IWK3I, J
      REAL               DIFLJ, DIFRJ, DJ, DSIGJ, DSIGJP, RHO, TEMP
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SLASCL, SLASD4, SLASET, XERBLA
*     ..
*     .. External Functions ..
      REAL               SDOT, SLAMC3, SNRM2
      EXTERNAL           SDOT, SLAMC3, SNRM2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SIGN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( ( ICOMPQ.LT.0 ) .OR. ( ICOMPQ.GT.1 ) ) THEN
         INFO = -1
      ELSE IF( K.LT.1 ) THEN
         INFO = -2
      ELSE IF( LDDIFR.LT.K ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SLASD8', -INFO )
         RETURN
      END IF
```

```
*
*     Quick return if possible
*
      IF( K.EQ.1 ) THEN
         D( 1 ) = ABS( Z( 1 ) )
         DIFL( 1 ) = D( 1 )
         IF( ICOMPQ.EQ.1 ) THEN
            DIFL( 2 ) = ONE
            DIFR( 1, 2 ) = ONE
         END IF
         RETURN
      END IF
*
*     Modify values DSIGMA(i) to make sure all DSIGMA(i)-DSIGMA(j) can
*     be computed with high relative accuracy (barring over/underflow).
*     This is a problem on machines without a guard digit in
*     add/subtract (Cray XMP, Cray YMP, Cray C 90 and Cray 2).
*     The following code replaces DSIGMA(I) by 2*DSIGMA(I)-DSIGMA(I),
*     which on any of these machines zeros out the bottommost
*     bit of DSIGMA(I) if it is 1; this makes the subsequent
*     subtractions DSIGMA(I)-DSIGMA(J) unproblematic when cancellation
*     occurs. On binary machines with a guard digit (almost all
*     machines) it does not change DSIGMA(I) at all. On hexadecimal
*     and decimal machines with a guard digit, it slightly
*     changes the bottommost bits of DSIGMA(I). It does not account
*     for hexadecimal or decimal machines without guard digits
*     (we know of none). We use a subroutine call to compute
*     2*DLAMBDA(I) to prevent optimizing compilers from eliminating
*     this code.
*
      DO 10 I = 1, K
         DSIGMA( I ) = SLAMC3( DSIGMA( I ), DSIGMA( I ) ) - DSIGMA( I )
   10 CONTINUE
*
*     Book keeping.
*
      IWK1 = 1
      IWK2 = IWK1 + K
      IWK3 = IWK2 + K
      IWK2I = IWK2 - 1
      IWK3I = IWK3 - 1
*
*     Normalize Z.
*
      RHO = SNRM2( K, Z, 1 )
      CALL SLASCL( 'G', 0, 0, RHO, ONE, K, 1, Z, K, INFO )
      RHO = RHO*RHO
*
*     Initialize WORK(IWK3).
*
```

```
      CALL SLASET( 'A', K, 1, ONE, ONE, WORK( IWK3 ), K )
*
*     Compute the updated singular values, the arrays DIFL, DIFR,
*     and the updated Z.
*
      DO 40 J = 1, K
         CALL SLASD4( K, J, DSIGMA, Z, WORK( IWK1 ), RHO, D( J ),
     $                WORK( IWK2 ), INFO )
*
*        If the root finder fails, report the convergence failure.
*
         IF( INFO.NE.0 ) THEN
            RETURN
         END IF
         WORK( IWK3I+J ) = WORK( IWK3I+J )*WORK( J )*WORK( IWK2I+J )
         DIFL( J ) = -WORK( J )
         DIFR( J, 1 ) = -WORK( J+1 )
         DO 20 I = 1, J - 1
            WORK( IWK3I+I ) = WORK( IWK3I+I )*WORK( I )*
     $                        WORK( IWK2I+I ) / ( DSIGMA( I )-
     $                        DSIGMA( J ) ) / ( DSIGMA( I )+
     $                        DSIGMA( J ) )
   20    CONTINUE
         DO 30 I = J + 1, K
            WORK( IWK3I+I ) = WORK( IWK3I+I )*WORK( I )*
     $                        WORK( IWK2I+I ) / ( DSIGMA( I )-
     $                        DSIGMA( J ) ) / ( DSIGMA( I )+
     $                        DSIGMA( J ) )
   30    CONTINUE
   40 CONTINUE
*
*     Compute updated Z.
*
      DO 50 I = 1, K
         Z( I ) = SIGN( SQRT( ABS( WORK( IWK3I+I ) ) ), Z( I ) )
   50 CONTINUE
*
*     Update VF and VL.
*
      DO 80 J = 1, K
         DIFLJ = DIFL( J )
         DJ = D( J )
         DSIGJ = -DSIGMA( J )
         IF( J.LT.K ) THEN
            DIFRJ = -DIFR( J, 1 )
            DSIGJP = -DSIGMA( J+1 )
         END IF
         WORK( J ) = -Z( J ) / DIFLJ / ( DSIGMA( J )+DJ )
         DO 60 I = 1, J - 1
            WORK( I ) = Z( I ) / ( SLAMC3( DSIGMA( I ), DSIGJ )-DIFLJ )
```

```
     $                  / ( DSIGMA( I )+DJ )
   60    CONTINUE
         DO 70 I = J + 1, K
            WORK( I ) = Z( I ) / ( SLAMC3( DSIGMA( I ), DSIGJP )+DIFRJ )
     $                  / ( DSIGMA( I )+DJ )
   70    CONTINUE
         TEMP = SNRM2( K, WORK, 1 )
         WORK( IWK2I+J ) = SDOT( K, WORK, 1, VF, 1 ) / TEMP
         WORK( IWK3I+J ) = SDOT( K, WORK, 1, VL, 1 ) / TEMP
         IF( ICOMPQ.EQ.1 ) THEN
            DIFR( J, 2 ) = TEMP
         END IF
   80 CONTINUE
*
      CALL SCOPY( K, WORK( IWK2 ), 1, VF, 1 )
      CALL SCOPY( K, WORK( IWK3 ), 1, VL, 1 )
*
      RETURN
*
*     End of SLASD8
*
      END
```

———-

90.100.4 lisp code

— slasd8.lisp —

```
(let* ((one 1.0))
  (declare (type (single-float 1.0 1.0) one)
           (ignorable one))
  (defun slasd8 (icompq k d z vf vl difl difr lddifr dsigma work info)
    (declare (type (array single-float (*)) work dsigma difr difl vl vf z d)
             (type (f2cl-lib:integer4) info lddifr k icompq))
    (f2cl-lib:with-multi-array-data
        ((d single-float d-%data% d-%offset%)
         (z single-float z-%data% z-%offset%)
         (vf single-float vf-%data% vf-%offset%)
         (vl single-float vl-%data% vl-%offset%)
         (difl single-float difl-%data% difl-%offset%)
         (difr single-float difr-%data% difr-%offset%)
         (dsigma single-float dsigma-%data% dsigma-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((diflj 0.0) (difrj 0.0) (dj 0.0) (dsigj 0.0) (dsigjp 0.0)
             (rho 0.0) (temp 0.0) (i 0) (iwk1 0) (iwk2 0) (iwk2i 0) (iwk3 0)
             (iwk3i 0) (j 0))
```

```
(declare (type (single-float) diflj difrj dj dsigj dsigjp rho temp)
         (type (f2cl-lib:integer4) i iwk1 iwk2 iwk2i iwk3 iwk3i j))
(setf info 0)
(cond ((or (< icompq 0) (> icompq 1)) (setf info -1))
      ((< k 1) (setf info -2)) ((< lddifr k) (setf info -9)))
(cond
 ((/= info 0) (xerbla "SLASD8" (f2cl-lib:int-sub info))
  (go end_label)))
(cond
 ((= k 1)
  (setf (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
          (abs (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)))
  (setf (f2cl-lib:fref difl-%data% (1) ((1 *)) difl-%offset%)
          (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
  (cond
   ((= icompq 1)
    (setf (f2cl-lib:fref difl-%data% (2) ((1 *)) difl-%offset%) one)
    (setf (f2cl-lib:fref difr-%data% (1 2) ((1 lddifr) (1 *))
                         difr-%offset%)
            one)))
  (go end_label)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i k) nil)
  (tagbody
    (setf (f2cl-lib:fref dsigma-%data% (i) ((1 *)) dsigma-%offset%)
            (-
             (slamc3
              (f2cl-lib:array-slice dsigma-%data% single-float (i)
                                    ((1 *)) dsigma-%offset%)
              (f2cl-lib:array-slice dsigma-%data% single-float (i)
                                    ((1 *)) dsigma-%offset%))
             (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                            dsigma-%offset%)))
   label10))
(setf iwk1 1)
(setf iwk2 (f2cl-lib:int-add iwk1 k))
(setf iwk3 (f2cl-lib:int-add iwk2 k))
(setf iwk2i (f2cl-lib:int-sub iwk2 1))
(setf iwk3i (f2cl-lib:int-sub iwk3 1))
(setf rho
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (snrm2 k z 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf k var-0))
          ret-val))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (slascl "G" 0 0 rho one k 1 z k info)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8))
```

```
  (setf info var-9))
(setf rho (* rho rho))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "A" k 1 one one
     (f2cl-lib:array-slice work-%data% single-float (iwk3) ((1 *))
                           work-%offset%)
     k)
  (declare (ignore var-0 var-2 var-5))
  (when var-1 (setf k var-1))
  (when var-3 (setf one var-3))
  (when var-4 (setf one var-4))
  (when var-6 (setf k var-6)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j k) nil)
  (tagbody
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (slasd4 k j dsigma z
         (f2cl-lib:array-slice work-%data% single-float (iwk1) ((1 *))
                               work-%offset%)
         rho (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
         (f2cl-lib:array-slice work-%data% single-float (iwk2) ((1 *))
                               work-%offset%)
         info)
      (declare (ignore var-0 var-2 var-3 var-4 var-7))
      (setf j var-1)
      (setf rho var-5)
      (setf (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%) var-6)
      (setf info var-8))
    (cond ((/= info 0) (go end_label)))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add iwk3i j))
                         ((1 *)) work-%offset%)
            (*
             (f2cl-lib:fref work-%data% ((f2cl-lib:int-add iwk3i j))
                            ((1 *)) work-%offset%)
             (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
             (f2cl-lib:fref work-%data% ((f2cl-lib:int-add iwk2i j))
                            ((1 *)) work-%offset%)))
    (setf (f2cl-lib:fref difl-%data% (j) ((1 *)) difl-%offset%)
            (- (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)))
    (setf (f2cl-lib:fref difr-%data% (j 1) ((1 lddifr) (1 *))
                         difr-%offset%)
            (-
             (f2cl-lib:fref work-%data% ((f2cl-lib:int-add j 1))
                            ((1 *)) work-%offset%)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add iwk3i i))
                             ((1 *)) work-%offset%)
```

```
                (/
                 (/
                  (*
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add iwk3i i)) ((1 *))
                                  work-%offset%)
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add iwk2i i)) ((1 *))
                                  work-%offset%))
                  (-
                   (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                                  dsigma-%offset%)
                   (f2cl-lib:fref dsigma-%data% (j) ((1 *))
                                  dsigma-%offset%)))
                 (+
                  (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                                 dsigma-%offset%)
                  (f2cl-lib:fref dsigma-%data% (j) ((1 *))
                                 dsigma-%offset%))))
    label20))
 (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
               ((> i k) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add iwk3i i))
                         ((1 *)) work-%offset%)
                (/
                 (/
                  (*
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add iwk3i i)) ((1 *))
                                  work-%offset%)
                   (f2cl-lib:fref work-%data% (i) ((1 *))
                                  work-%offset%)
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-add iwk2i i)) ((1 *))
                                  work-%offset%))
                  (-
                   (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                                  dsigma-%offset%)
                   (f2cl-lib:fref dsigma-%data% (j) ((1 *))
                                  dsigma-%offset%)))
                 (+
                  (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                                 dsigma-%offset%)
                  (f2cl-lib:fref dsigma-%data% (j) ((1 *))
                                 dsigma-%offset%))))
    label30))
label40))
```

```
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i k) nil)
  (tagbody
    (setf (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
            (f2cl-lib:sign
             (f2cl-lib:fsqrt
              (abs
               (f2cl-lib:fref work-%data% ((f2cl-lib:int-add iwk3i i))
                              ((1 *)) work-%offset%)))
             (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)))
   label50))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j k) nil)
  (tagbody
    (setf diflj (f2cl-lib:fref difl-%data% (j) ((1 *)) difl-%offset%))
    (setf dj (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%))
    (setf dsigj
            (-
             (f2cl-lib:fref dsigma-%data% (j) ((1 *))
                            dsigma-%offset%)))
    (cond
     ((< j k)
      (setf difrj
              (-
               (f2cl-lib:fref difr-%data% (j 1) ((1 lddifr) (1 *))
                              difr-%offset%)))
      (setf dsigjp
              (-
               (f2cl-lib:fref dsigma-%data% ((f2cl-lib:int-add j 1))
                              ((1 *)) dsigma-%offset%)))))
    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
            (/
             (/ (- (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%))
                diflj)
             (+
              (f2cl-lib:fref dsigma-%data% (j) ((1 *)) dsigma-%offset%)
              dj)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (/
                 (/ (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                    (-
                     (multiple-value-bind (ret-val var-0 var-1)
                         (slamc3
                          (f2cl-lib:array-slice dsigma-%data%
                                                single-float (i)
                                                ((1 *))
                                                dsigma-%offset%)
```

```
                        dsigj)
                      (declare (ignore var-0))
                      (when var-1 (setf dsigj var-1))
                      ret-val)
                    diflj))
                (+
                 (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                                dsigma-%offset%)
                 dj)))
   label60))
(f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
              ((> i k) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (/
             (/ (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                (+
                 (multiple-value-bind (ret-val var-0 var-1)
                     (slamc3
                      (f2cl-lib:array-slice dsigma-%data%
                                            single-float (i)
                                            ((1 *))
                                            dsigma-%offset%)
                      dsigjp)
                    (declare (ignore var-0))
                    (when var-1 (setf dsigjp var-1))
                    ret-val)
                 difrj))
             (+
              (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                             dsigma-%offset%)
              dj)))
   label70))
(setf temp
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (snrm2 k work 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf k var-0))
          ret-val))
(setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add iwk2i j))
                     ((1 *)) work-%offset%)
        (/
         (multiple-value-bind
             (ret-val var-0 var-1 var-2 var-3 var-4)
             (sdot k work 1 vf 1)
           (declare (ignore var-1 var-2 var-3 var-4))
           (when var-0 (setf k var-0))
           ret-val)
         temp))
(setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add iwk3i j))
```

```
                                ((1 *)) work-%offset%)
                     (/
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4)
                          (sdot k work 1 vl 1)
                        (declare (ignore var-1 var-2 var-3 var-4))
                        (when var-0 (setf k var-0))
                        ret-val)
                      temp))
              (cond
               ((= icompq 1)
                (setf (f2cl-lib:fref difr-%data% (j 2) ((1 lddifr) (1 *))
                                     difr-%offset%)
                      temp)))
           label80))
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (scopy k
              (f2cl-lib:array-slice work-%data% single-float (iwk2) ((1 *))
                                    work-%offset%)
              1 vf 1)
           (declare (ignore var-1 var-2 var-3 var-4))
           (when var-0 (setf k var-0)))
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (scopy k
              (f2cl-lib:array-slice work-%data% single-float (iwk3) ((1 *))
                                    work-%offset%)
              1 vl 1)
           (declare (ignore var-1 var-2 var-3 var-4))
           (when var-0 (setf k var-0)))
         (go end_label)
        end_label
         (return (values nil k nil nil nil nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slasd8
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
```

```
  (fortran-to-lisp::integer4))
:return-values
'(nil fortran-to-lisp::k nil nil
  nil nil nil nil nil nil nil
  fortran-to-lisp::info)
:calls
'(fortran-to-lisp::slasd4
  fortran-to-lisp::slascl))))
```

———-

90.101 slasda

90.101.1 Axiom unit tests

— **slasda.input** —

```
)set break resume
)sys rm -f slasda.output
)spool slasda.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.101.2 Axiom help page

— **slasda.help** —

```
=====================================================================
slasda examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
```

```
   Functions/Subroutines
       subroutine slasda (ICOMPQ, SMLSIZ, N, SQRE, D, E, U, LDU, VT, K, DIFL,
           DIFR, Z, POLES, GIVPTR, GIVCOL, LDGCOL, PERM, GIVNUM, C, S, WORK,
           IWORK, INFO)
           SLASDA computes the singular value decomposition (SVD) of a real
           upper bidiagonal matrix with diagonal d and off-diagonal e. Used by
           sbdsdc.

Function/Subroutine Documentation
   subroutine slasda (integer ICOMPQ, integer SMLSIZ, integer N, integer SQRE,
       real, dimension( * ) D, real, dimension( * ) E, real, dimension( ldu, *
       ) U, integer LDU, real, dimension( ldu, * ) VT, integer, dimension( * )
       K, real, dimension( ldu, * ) DIFL, real, dimension( ldu, * ) DIFR,
       real, dimension( ldu, * ) Z, real, dimension( ldu, * ) POLES, integer,
       dimension( * ) GIVPTR, integer, dimension( ldgcol, * ) GIVCOL, integer
       LDGCOL, integer, dimension( ldgcol, * ) PERM, real, dimension( ldu, * )
       GIVNUM, real, dimension( * ) C, real, dimension( * ) S, real,
       dimension( * ) WORK, integer, dimension( * ) IWORK, integer INFO)
       SLASDA computes the singular value decomposition (SVD) of a real upper
       bidiagonal matrix with diagonal d and off-diagonal e. Used by sbdsdc.

Purpose:
   Using a divide and conquer approach, SLASDA computes the singular
   value decomposition (SVD) of a real upper bidiagonal N-by-M matrix
   B with diagonal D and offdiagonal E, where M = N + SQRE. The
   algorithm computes the singular values in the SVD B = U * S * VT.
   The orthogonal matrices U and VT are optionally computed in
   compact form.

   A related subroutine, SLASD0, computes the singular values and
   the singular vectors in explicit form.

Parameters:

 ICOMPQ
   ICOMPQ is INTEGER
   Specifies whether singular vectors are to be computed
   in compact form, as follows
   = 0: Compute singular values only.
   = 1: Compute singular vectors of upper bidiagonal matrix in compact form.

 SMLSIZ
   SMLSIZ is INTEGER
   The maximum size of the subproblems at the bottom of the computation tree.

 N
   N is INTEGER
   The row dimension of the upper bidiagonal matrix. This is
   also the dimension of the main diagonal array D.
```

```
SQRE
  SQRE is INTEGER
  Specifies the column dimension of the bidiagonal matrix.
  = 0: The bidiagonal matrix has column dimension M = N;
  = 1: The bidiagonal matrix has column dimension M = N + 1.

D
  D is REAL array, dimension ( N )
  On entry D contains the main diagonal of the bidiagonal
  matrix. On exit D, if INFO = 0, contains its singular values.

E
  E is REAL array, dimension ( M-1 )
  Contains the subdiagonal entries of the bidiagonal matrix.
  On exit, E has been destroyed.

U
  U is REAL array,
  dimension ( LDU, SMLSIZ ) if ICOMPQ = 1, and not referenced
  if ICOMPQ = 0. If ICOMPQ = 1, on exit, U contains the left
  singular vector matrices of all subproblems at the bottom level.

LDU
  LDU is INTEGER, LDU = > N.
  The leading dimension of arrays U, VT, DIFL, DIFR, POLES, GIVNUM, and Z.

VT
  VT is REAL array,
  dimension ( LDU, SMLSIZ+1 ) if ICOMPQ = 1, and not referenced
  if ICOMPQ = 0. If ICOMPQ = 1, on exit, VT**T contains the right
  singular vector matrices of all subproblems at the bottom level.

K
  K is INTEGER array, dimension ( N )
  if ICOMPQ = 1 and dimension 1 if ICOMPQ = 0.
  If ICOMPQ = 1, on exit, K(I) is the dimension of the I-th
  secular equation on the computation tree.

DIFL
  DIFL is REAL array, dimension ( LDU, NLVL ),
  where NLVL = floor(log_2 (N/SMLSIZ))).

DIFR
  DIFR is REAL array,
     dimension ( LDU, 2 * NLVL ) if ICOMPQ = 1 and
     dimension ( N ) if ICOMPQ = 0.
  If ICOMPQ = 1, on exit, DIFL(1:N, I) and DIFR(1:N, 2 * I - 1)
  record distances between singular values on the I-th
  level and singular values on the (I -1)-th level, and
  DIFR(1:N, 2 * I ) contains the normalizing factors for
```

```
  the right singular vector matrix. See SLASD8 for details.

Z
  Z is REAL array,
     dimension ( LDU, NLVL ) if ICOMPQ = 1 and
     dimension ( N ) if ICOMPQ = 0.
  The first K elements of Z(1, I) contain the components of
  the deflation-adjusted updating row vector for subproblems
  on the I-th level.

POLES
  POLES is REAL array,
  dimension ( LDU, 2 * NLVL ) if ICOMPQ = 1, and not referenced
  if ICOMPQ = 0. If ICOMPQ = 1, on exit, POLES(1, 2*I - 1) and
  POLES(1, 2*I) contain  the new and old singular values
  involved in the secular equations on the I-th level.

GIVPTR
  GIVPTR is INTEGER array,
  dimension ( N ) if ICOMPQ = 1, and not referenced if
  ICOMPQ = 0. If ICOMPQ = 1, on exit, GIVPTR( I ) records
  the number of Givens rotations performed on the I-th
  problem on the computation tree.

GIVCOL
  GIVCOL is INTEGER array,
  dimension ( LDGCOL, 2 * NLVL ) if ICOMPQ = 1, and not
  referenced if ICOMPQ = 0. If ICOMPQ = 1, on exit, for each I,
  GIVCOL(1, 2 *I - 1) and GIVCOL(1, 2 *I) record the locations
  of Givens rotations performed on the I-th level on the computation tree.

LDGCOL
  LDGCOL is INTEGER, LDGCOL = > N.
  The leading dimension of arrays GIVCOL and PERM.

PERM
  PERM is INTEGER array, dimension ( LDGCOL, NLVL )
  if ICOMPQ = 1, and not referenced
  if ICOMPQ = 0. If ICOMPQ = 1, on exit, PERM(1, I) records
  permutations done on the I-th level of the computation tree.

GIVNUM
  GIVNUM is REAL array,
  dimension ( LDU,  2 * NLVL ) if ICOMPQ = 1, and not
  referenced if ICOMPQ = 0. If ICOMPQ = 1, on exit, for each I,
  GIVNUM(1, 2 *I - 1) and GIVNUM(1, 2 *I) record the C- and S-
  values of Givens rotations performed on the I-th level on
  the computation tree.

C
```

```
  C is REAL array,
  dimension ( N ) if ICOMPQ = 1, and dimension 1 if ICOMPQ = 0.
  If ICOMPQ = 1 and the I-th subproblem is not square, on exit,
  C( I ) contains the C-value of a Givens rotation related to
  the right null space of the I-th subproblem.

S
  S is REAL array, dimension ( N ) if
  ICOMPQ = 1, and dimension 1 if ICOMPQ = 0. If ICOMPQ = 1
  and the I-th subproblem is not square, on exit, S( I )
  contains the S-value of a Givens rotation related to
  the right null space of the I-th subproblem.

WORK
  WORK is REAL array, dimension
  (6 * N + (SMLSIZ + 1)*(SMLSIZ + 1)).

IWORK
  IWORK is INTEGER array, dimension (7*N).

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if INFO = 1, a singular value did not converge
```

———-

90.101.3 fortran code

— **slasda.f** —

```
*  =====================================================================
      SUBROUTINE SLASDA( ICOMPQ, SMLSIZ, N, SQRE, D, E, U, LDU, VT, K,
     $                   DIFL, DIFR, Z, POLES, GIVPTR, GIVCOL, LDGCOL,
     $                   PERM, GIVNUM, C, S, WORK, IWORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            ICOMPQ, INFO, LDGCOL, LDU, N, SMLSIZ, SQRE
*     ..
*     .. Array Arguments ..
      INTEGER            GIVCOL( LDGCOL, * ), GIVPTR( * ), IWORK( * ),
```

```
     $                   K( * ), PERM( LDGCOL, * )
      REAL               C( * ), D( * ), DIFL( LDU, * ), DIFR( LDU, * ),
     $                   E( * ), GIVNUM( LDU, * ), POLES( LDU, * ),
     $                   S( * ), U( LDU, * ), VT( LDU, * ), WORK( * ),
     $                   Z( LDU, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO, ONE
      PARAMETER          ( ZERO = 0.0E+0, ONE = 1.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, I1, IC, IDXQ, IDXQI, IM1, INODE, ITEMP, IWK,
     $                   J, LF, LL, LVL, LVL2, M, NCC, ND, NDB1, NDIML,
     $                   NDIMR, NL, NLF, NLP1, NLVL, NR, NRF, NRP1, NRU,
     $                   NWORK1, NWORK2, SMLSZP, SQREI, VF, VFI, VL, VLI
      REAL               ALPHA, BETA
*     ..
*     .. External Subroutines ..
      EXTERNAL           SCOPY, SLASD6, SLASDQ, SLASDT, SLASET, XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( ( ICOMPQ.LT.0 ) .OR. ( ICOMPQ.GT.1 ) ) THEN
         INFO = -1
      ELSE IF( SMLSIZ.LT.3 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( ( SQRE.LT.0 ) .OR. ( SQRE.GT.1 ) ) THEN
         INFO = -4
      ELSE IF( LDU.LT.( N+SQRE ) ) THEN
         INFO = -8
      ELSE IF( LDGCOL.LT.N ) THEN
         INFO = -17
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SLASDA', -INFO )
         RETURN
      END IF
*
      M = N + SQRE
*
*     If the input matrix is too small, call SLASDQ to find the SVD.
```

```
*
      IF( N.LE.SMLSIZ ) THEN
         IF( ICOMPQ.EQ.0 ) THEN
            CALL SLASDQ( 'U', SQRE, N, 0, 0, 0, D, E, VT, LDU, U, LDU,
     $                   U, LDU, WORK, INFO )
         ELSE
            CALL SLASDQ( 'U', SQRE, N, M, N, 0, D, E, VT, LDU, U, LDU,
     $                   U, LDU, WORK, INFO )
         END IF
         RETURN
      END IF
*
*     Book-keeping and  set up the computation tree.
*
      INODE = 1
      NDIML = INODE + N
      NDIMR = NDIML + N
      IDXQ = NDIMR + N
      IWK = IDXQ + N
*
      NCC = 0
      NRU = 0
*
      SMLSZP = SMLSIZ + 1
      VF = 1
      VL = VF + M
      NWORK1 = VL + M
      NWORK2 = NWORK1 + SMLSZP*SMLSZP
*
      CALL SLASDT( N, NLVL, ND, IWORK( INODE ), IWORK( NDIML ),
     $             IWORK( NDIMR ), SMLSIZ )
*
*     for the nodes on bottom level of the tree, solve
*     their subproblems by SLASDQ.
*
      NDB1 = ( ND+1 ) / 2
      DO 30 I = NDB1, ND
*
*        IC : center row of each node
*        NL : number of rows of left  subproblem
*        NR : number of rows of right subproblem
*        NLF: starting row of the left   subproblem
*        NRF: starting row of the right  subproblem
*
         I1 = I - 1
         IC = IWORK( INODE+I1 )
         NL = IWORK( NDIML+I1 )
         NLP1 = NL + 1
         NR = IWORK( NDIMR+I1 )
         NLF = IC - NL
```

```
      NRF = IC + 1
      IDXQI = IDXQ + NLF - 2
      VFI = VF + NLF - 1
      VLI = VL + NLF - 1
      SQREI = 1
      IF( ICOMPQ.EQ.0 ) THEN
         CALL SLASET( 'A', NLP1, NLP1, ZERO, ONE, WORK( NWORK1 ),
     $                SMLSZP )
         CALL SLASDQ( 'U', SQREI, NL, NLP1, NRU, NCC, D( NLF ),
     $                E( NLF ), WORK( NWORK1 ), SMLSZP,
     $                WORK( NWORK2 ), NL, WORK( NWORK2 ), NL,
     $                WORK( NWORK2 ), INFO )
         ITEMP = NWORK1 + NL*SMLSZP
         CALL SCOPY( NLP1, WORK( NWORK1 ), 1, WORK( VFI ), 1 )
         CALL SCOPY( NLP1, WORK( ITEMP ), 1, WORK( VLI ), 1 )
      ELSE
         CALL SLASET( 'A', NL, NL, ZERO, ONE, U( NLF, 1 ), LDU )
         CALL SLASET( 'A', NLP1, NLP1, ZERO, ONE, VT( NLF, 1 ), LDU )
         CALL SLASDQ( 'U', SQREI, NL, NLP1, NL, NCC, D( NLF ),
     $                E( NLF ), VT( NLF, 1 ), LDU, U( NLF, 1 ), LDU,
     $                U( NLF, 1 ), LDU, WORK( NWORK1 ), INFO )
         CALL SCOPY( NLP1, VT( NLF, 1 ), 1, WORK( VFI ), 1 )
         CALL SCOPY( NLP1, VT( NLF, NLP1 ), 1, WORK( VLI ), 1 )
      END IF
      IF( INFO.NE.0 ) THEN
         RETURN
      END IF
      DO 10 J = 1, NL
         IWORK( IDXQI+J ) = J
10    CONTINUE
      IF( ( I.EQ.ND ) .AND. ( SQRE.EQ.0 ) ) THEN
         SQREI = 0
      ELSE
         SQREI = 1
      END IF
      IDXQI = IDXQI + NLP1
      VFI = VFI + NLP1
      VLI = VLI + NLP1
      NRP1 = NR + SQREI
      IF( ICOMPQ.EQ.0 ) THEN
         CALL SLASET( 'A', NRP1, NRP1, ZERO, ONE, WORK( NWORK1 ),
     $                SMLSZP )
         CALL SLASDQ( 'U', SQREI, NR, NRP1, NRU, NCC, D( NRF ),
     $                E( NRF ), WORK( NWORK1 ), SMLSZP,
     $                WORK( NWORK2 ), NR, WORK( NWORK2 ), NR,
     $                WORK( NWORK2 ), INFO )
         ITEMP = NWORK1 + ( NRP1-1 )*SMLSZP
         CALL SCOPY( NRP1, WORK( NWORK1 ), 1, WORK( VFI ), 1 )
         CALL SCOPY( NRP1, WORK( ITEMP ), 1, WORK( VLI ), 1 )
      ELSE
```

```
            CALL SLASET( 'A', NR, NR, ZERO, ONE, U( NRF, 1 ), LDU )
            CALL SLASET( 'A', NRP1, NRP1, ZERO, ONE, VT( NRF, 1 ), LDU )
            CALL SLASDQ( 'U', SQREI, NR, NRP1, NR, NCC, D( NRF ),
     $                   E( NRF ), VT( NRF, 1 ), LDU, U( NRF, 1 ), LDU,
     $                   U( NRF, 1 ), LDU, WORK( NWORK1 ), INFO )
            CALL SCOPY( NRP1, VT( NRF, 1 ), 1, WORK( VFI ), 1 )
            CALL SCOPY( NRP1, VT( NRF, NRP1 ), 1, WORK( VLI ), 1 )
         END IF
         IF( INFO.NE.0 ) THEN
            RETURN
         END IF
         DO 20 J = 1, NR
            IWORK( IDXQI+J ) = J
   20    CONTINUE
   30 CONTINUE
*
*     Now conquer each subproblem bottom-up.
*
      J = 2**NLVL
      DO 50 LVL = NLVL, 1, -1
         LVL2 = LVL*2 - 1
*
*        Find the first node LF and last node LL on
*        the current level LVL.
*
         IF( LVL.EQ.1 ) THEN
            LF = 1
            LL = 1
         ELSE
            LF = 2**( LVL-1 )
            LL = 2*LF - 1
         END IF
         DO 40 I = LF, LL
            IM1 = I - 1
            IC = IWORK( INODE+IM1 )
            NL = IWORK( NDIML+IM1 )
            NR = IWORK( NDIMR+IM1 )
            NLF = IC - NL
            NRF = IC + 1
            IF( I.EQ.LL ) THEN
               SQREI = SQRE
            ELSE
               SQREI = 1
            END IF
            VFI = VF + NLF - 1
            VLI = VL + NLF - 1
            IDXQI = IDXQ + NLF - 1
            ALPHA = D( IC )
            BETA = E( IC )
            IF( ICOMPQ.EQ.0 ) THEN
```

```
            CALL SLASD6( ICOMPQ, NL, NR, SQREI, D( NLF ),
     $                   WORK( VFI ), WORK( VLI ), ALPHA, BETA,
     $                   IWORK( IDXQI ), PERM, GIVPTR( 1 ), GIVCOL,
     $                   LDGCOL, GIVNUM, LDU, POLES, DIFL, DIFR, Z,
     $                   K( 1 ), C( 1 ), S( 1 ), WORK( NWORK1 ),
     $                   IWORK( IWK ), INFO )
         ELSE
            J = J - 1
            CALL SLASD6( ICOMPQ, NL, NR, SQREI, D( NLF ),
     $                   WORK( VFI ), WORK( VLI ), ALPHA, BETA,
     $                   IWORK( IDXQI ), PERM( NLF, LVL ),
     $                   GIVPTR( J ), GIVCOL( NLF, LVL2 ), LDGCOL,
     $                   GIVNUM( NLF, LVL2 ), LDU,
     $                   POLES( NLF, LVL2 ), DIFL( NLF, LVL ),
     $                   DIFR( NLF, LVL2 ), Z( NLF, LVL ), K( J ),
     $                   C( J ), S( J ), WORK( NWORK1 ),
     $                   IWORK( IWK ), INFO )
         END IF
         IF( INFO.NE.0 ) THEN
            RETURN
         END IF
   40    CONTINUE
   50 CONTINUE
*
      RETURN
*
*     End of SLASDA
*
      END
```

———-

90.101.4 lisp code

— **slasda.lisp** —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (ignorable zero one))
  (defun slasda
         (icompq smlsiz n sqre d e u ldu vt k difl difr z poles givptr givcol
          ldgcol perm givnum c s work iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork perm givcol givptr k)
             (type (array single-float (*)) work s c givnum poles z difr difl
              vt u e d)
             (type (f2cl-lib:integer4) info ldgcol ldu sqre n smlsiz icompq))
    (f2cl-lib:with-multi-array-data
```

```
  ((d single-float d-%data% d-%offset%)
   (e single-float e-%data% e-%offset%)
   (u single-float u-%data% u-%offset%)
   (vt single-float vt-%data% vt-%offset%)
   (difl single-float difl-%data% difl-%offset%)
   (difr single-float difr-%data% difr-%offset%)
   (z single-float z-%data% z-%offset%)
   (poles single-float poles-%data% poles-%offset%)
   (givnum single-float givnum-%data% givnum-%offset%)
   (c single-float c-%data% c-%offset%)
   (s single-float s-%data% s-%offset%)
   (work single-float work-%data% work-%offset%)
   (k f2cl-lib:integer4 k-%data% k-%offset%)
   (givptr f2cl-lib:integer4 givptr-%data% givptr-%offset%)
   (givcol f2cl-lib:integer4 givcol-%data% givcol-%offset%)
   (perm f2cl-lib:integer4 perm-%data% perm-%offset%)
   (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
(prog ((alpha 0.0) (beta 0.0) (i 0) (i1 0) (ic 0) (idxq 0) (idxqi 0)
       (im1 0) (inode 0) (itemp 0) (iwk 0) (j 0) (lf 0) (ll 0) (lvl 0)
       (lvl2 0) (m 0) (ncc 0) (nd 0) (ndb1 0) (ndiml 0) (ndimr 0) (nl 0)
       (nlf 0) (nlp1 0) (nlvl 0) (nr 0) (nrf 0) (nrp1 0) (nru 0)
       (nwork1 0) (nwork2 0) (smlszp 0) (sqrei 0) (vf 0) (vfi 0) (vl 0)
       (vli 0))
  (declare (type (single-float) alpha beta)
           (type (f2cl-lib:integer4) i i1 ic idxq idxqi im1 inode itemp
            iwk j lf ll lvl lvl2 m ncc nd ndb1 ndiml ndimr nl nlf nlp1
            nlvl nr nrf nrp1 nru nwork1 nwork2 smlszp sqrei vf vfi vl
            vli))
  (setf info 0)
  (cond ((or (< icompq 0) (> icompq 1)) (setf info -1))
        ((< smlsiz 3) (setf info -2)) ((< n 0) (setf info -3))
        ((or (< sqre 0) (> sqre 1)) (setf info -4))
        ((< ldu (f2cl-lib:int-add n sqre)) (setf info -8))
        ((< ldgcol n) (setf info -17)))
  (cond
   ((/= info 0) (xerbla "SLASDA" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf m (f2cl-lib:int-add n sqre))
  (cond
   ((<= n smlsiz)
    (cond
     ((= icompq 0)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13 var-14 var-15)
          (slasdq "U" sqre n 0 0 0 d e vt ldu u ldu u ldu work info)
        (declare
         (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-8 var-10 var-12
          var-14))
        (when var-1 (setf sqre var-1))
```

```
      (when var-2 (setf n var-2))
      (when var-9 (setf ldu var-9))
      (when var-11 (setf ldu var-11))
      (when var-13 (setf ldu var-13))
      (when var-15 (setf info var-15))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14 var-15)
        (slasdq "U" sqre n m n 0 d e vt ldu u ldu u ldu work info)
      (declare
       (ignore var-0 var-5 var-6 var-7 var-8 var-10 var-12 var-14))
      (when var-1 (setf sqre var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf m var-3))
      (when var-4 (setf n var-4))
      (when var-9 (setf ldu var-9))
      (when var-11 (setf ldu var-11))
      (when var-13 (setf ldu var-13))
      (when var-15 (setf info var-15)))))
  (go end_label)))
(setf inode 1)
(setf ndiml (f2cl-lib:int-add inode n))
(setf ndimr (f2cl-lib:int-add ndiml n))
(setf idxq (f2cl-lib:int-add ndimr n))
(setf iwk (f2cl-lib:int-add idxq n))
(setf ncc 0)
(setf nru 0)
(setf smlszp (f2cl-lib:int-add smlsiz 1))
(setf vf 1)
(setf vl (f2cl-lib:int-add vf m))
(setf nwork1 (f2cl-lib:int-add vl m))
(setf nwork2
        (f2cl-lib:int-add nwork1 (f2cl-lib:int-mul smlszp smlszp)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slasdt n nlvl nd
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (inode)
                           ((1 *)) iwork-%offset%)
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (ndiml)
                           ((1 *)) iwork-%offset%)
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (ndimr)
                           ((1 *)) iwork-%offset%)
     smlsiz)
  (declare (ignore var-3 var-4 var-5))
  (when var-0 (setf n var-0))
  (when var-1 (setf nlvl var-1))
  (when var-2 (setf nd var-2))
  (when var-6 (setf smlsiz var-6)))
(setf ndb1 (the f2cl-lib:integer4 (truncate (+ nd 1) 2)))
(f2cl-lib:fdo (i ndb1 (f2cl-lib:int-add i 1))
```

```
            ((> i nd) nil)
(tagbody
  (setf i1 (f2cl-lib:int-sub i 1))
  (setf ic
          (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add inode i1))
                         ((1 *)) iwork-%offset%))
  (setf nl
          (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add ndiml i1))
                         ((1 *)) iwork-%offset%))
  (setf nlp1 (f2cl-lib:int-add nl 1))
  (setf nr
          (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add ndimr i1))
                         ((1 *)) iwork-%offset%))
  (setf nlf (f2cl-lib:int-sub ic nl))
  (setf nrf (f2cl-lib:int-add ic 1))
  (setf idxqi (f2cl-lib:int-sub (f2cl-lib:int-add idxq nlf) 2))
  (setf vfi (f2cl-lib:int-sub (f2cl-lib:int-add vf nlf) 1))
  (setf vli (f2cl-lib:int-sub (f2cl-lib:int-add vl nlf) 1))
  (setf sqrei 1)
  (cond
   ((= icompq 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "A" nlp1 nlp1 zero one
         (f2cl-lib:array-slice work-%data% single-float (nwork1)
                               ((1 *)) work-%offset%)
         smlszp)
      (declare (ignore var-0 var-5))
      (when var-1 (setf nlp1 var-1))
      (when var-2 (setf nlp1 var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf smlszp var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14 var-15)
        (slasdq "U" sqrei nl nlp1 nru ncc
         (f2cl-lib:array-slice d-%data% single-float (nlf) ((1 *))
                               d-%offset%)
         (f2cl-lib:array-slice e-%data% single-float (nlf) ((1 *))
                               e-%offset%)
         (f2cl-lib:array-slice work-%data% single-float (nwork1)
                               ((1 *)) work-%offset%)
         smlszp
         (f2cl-lib:array-slice work-%data% single-float (nwork2)
                               ((1 *)) work-%offset%)
         nl
         (f2cl-lib:array-slice work-%data% single-float (nwork2)
                               ((1 *)) work-%offset%)
         nl
         (f2cl-lib:array-slice work-%data% single-float (nwork2)
```

```
                              ((1 *)) work-%offset%)
        info)
      (declare (ignore var-0 var-6 var-7 var-8 var-10 var-12 var-14))
      (when var-1 (setf sqrei var-1))
      (when var-2 (setf nl var-2))
      (when var-3 (setf nlp1 var-3))
      (when var-4 (setf nru var-4))
      (when var-5 (setf ncc var-5))
      (when var-9 (setf smlszp var-9))
      (when var-11 (setf nl var-11))
      (when var-13 (setf nl var-13))
      (when var-15 (setf info var-15)))
    (setf itemp
            (f2cl-lib:int-add nwork1 (f2cl-lib:int-mul nl smlszp)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy nlp1
         (f2cl-lib:array-slice work-%data% single-float (nwork1)
                              ((1 *)) work-%offset%)
         1
         (f2cl-lib:array-slice work-%data% single-float (vfi) ((1 *))
                              work-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf nlp1 var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (scopy nlp1
         (f2cl-lib:array-slice work-%data% single-float (itemp)
                              ((1 *)) work-%offset%)
         1
         (f2cl-lib:array-slice work-%data% single-float (vli) ((1 *))
                              work-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf nlp1 var-0))))
  (t
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "A" nl nl zero one
         (f2cl-lib:array-slice u-%data% single-float (nlf 1)
                              ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-5))
      (when var-1 (setf nl var-1))
      (when var-2 (setf nl var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldu var-6)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (slaset "A" nlp1 nlp1 zero one
         (f2cl-lib:array-slice vt-%data% single-float (nlf 1)
                              ((1 ldu) (1 *)) vt-%offset%)
```

```
      ldu)
  (declare (ignore var-0 var-5))
  (when var-1 (setf nlp1 var-1))
  (when var-2 (setf nlp1 var-2))
  (when var-3 (setf zero var-3))
  (when var-4 (setf one var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15)
    (slasdq "U" sqrei nl nlp1 nl ncc
     (f2cl-lib:array-slice d-%data% single-float (nlf) ((1 *))
                           d-%offset%)
     (f2cl-lib:array-slice e-%data% single-float (nlf) ((1 *))
                           e-%offset%)
     (f2cl-lib:array-slice vt-%data% single-float (nlf 1)
                           ((1 ldu) (1 *)) vt-%offset%)
     ldu
     (f2cl-lib:array-slice u-%data% single-float (nlf 1)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu
     (f2cl-lib:array-slice u-%data% single-float (nlf 1)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu
     (f2cl-lib:array-slice work-%data% single-float (nwork1)
                           ((1 *)) work-%offset%)
     info)
  (declare (ignore var-0 var-6 var-7 var-8 var-10 var-12 var-14))
  (when var-1 (setf sqrei var-1))
  (when var-2 (setf nl var-2))
  (when var-3 (setf nlp1 var-3))
  (when var-4 (setf nl var-4))
  (when var-5 (setf ncc var-5))
  (when var-9 (setf ldu var-9))
  (when var-11 (setf ldu var-11))
  (when var-13 (setf ldu var-13))
  (when var-15 (setf info var-15)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (scopy nlp1
     (f2cl-lib:array-slice vt-%data% single-float (nlf 1)
                           ((1 ldu) (1 *)) vt-%offset%)
     1
     (f2cl-lib:array-slice work-%data% single-float (vfi) ((1 *))
                           work-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf nlp1 var-0)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (scopy nlp1
     (f2cl-lib:array-slice vt-%data% single-float (nlf nlp1)
```

```
                                ((1 ldu) (1 *)) vt-%offset%)
        1
        (f2cl-lib:array-slice work-%data% single-float (vli) ((1 *))
                                work-%offset%)
        1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf nlp1 var-0)))))
(cond ((/= info 0) (go end_label)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nl) nil)
  (tagbody
    (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add idxqi j))
                         ((1 *)) iwork-%offset%)
            j)
   label10))
(cond ((and (= i nd) (= sqre 0)) (setf sqrei 0))
      (t (setf sqrei 1)))
(setf idxqi (f2cl-lib:int-add idxqi nlp1))
(setf vfi (f2cl-lib:int-add vfi nlp1))
(setf vli (f2cl-lib:int-add vli nlp1))
(setf nrp1 (f2cl-lib:int-add nr sqrei))
(cond
 ((= icompq 0)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "A" nrp1 nrp1 zero one
       (f2cl-lib:array-slice work-%data% single-float (nwork1)
                                ((1 *)) work-%offset%)
       smlszp)
    (declare (ignore var-0 var-5))
    (when var-1 (setf nrp1 var-1))
    (when var-2 (setf nrp1 var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf smlszp var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (slasdq "U" sqrei nr nrp1 nru ncc
       (f2cl-lib:array-slice d-%data% single-float (nrf) ((1 *))
                                d-%offset%)
       (f2cl-lib:array-slice e-%data% single-float (nrf) ((1 *))
                                e-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (nwork1)
                                ((1 *)) work-%offset%)
       smlszp
       (f2cl-lib:array-slice work-%data% single-float (nwork2)
                                ((1 *)) work-%offset%)
       nr
       (f2cl-lib:array-slice work-%data% single-float (nwork2)
                                ((1 *)) work-%offset%)
```

```
      nr
      (f2cl-lib:array-slice work-%data% single-float (nwork2)
                            ((1 *)) work-%offset%)
      info)
    (declare (ignore var-0 var-6 var-7 var-8 var-10 var-12 var-14))
    (when var-1 (setf sqrei var-1))
    (when var-2 (setf nr var-2))
    (when var-3 (setf nrp1 var-3))
    (when var-4 (setf nru var-4))
    (when var-5 (setf ncc var-5))
    (when var-9 (setf smlszp var-9))
    (when var-11 (setf nr var-11))
    (when var-13 (setf nr var-13))
    (when var-15 (setf info var-15)))
  (setf itemp
          (f2cl-lib:int-add nwork1
                            (f2cl-lib:int-mul
                             (f2cl-lib:int-sub nrp1 1) smlszp)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (scopy nrp1
      (f2cl-lib:array-slice work-%data% single-float (nwork1)
                            ((1 *)) work-%offset%)
      1
      (f2cl-lib:array-slice work-%data% single-float (vfi) ((1 *))
                            work-%offset%)
      1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf nrp1 var-0)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (scopy nrp1
      (f2cl-lib:array-slice work-%data% single-float (itemp)
                            ((1 *)) work-%offset%)
      1
      (f2cl-lib:array-slice work-%data% single-float (vli) ((1 *))
                            work-%offset%)
      1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf nrp1 var-0))))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (slaset "A" nr nr zero one
      (f2cl-lib:array-slice u-%data% single-float (nrf 1)
                            ((1 ldu) (1 *)) u-%offset%)
      ldu)
    (declare (ignore var-0 var-5))
    (when var-1 (setf nr var-1))
    (when var-2 (setf nr var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldu var-6)))
```

```
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (slaset "A" nrp1 nrp1 zero one
     (f2cl-lib:array-slice vt-%data% single-float (nrf 1)
                           ((1 ldu) (1 *)) vt-%offset%)
     ldu)
  (declare (ignore var-0 var-5))
  (when var-1 (setf nrp1 var-1))
  (when var-2 (setf nrp1 var-2))
  (when var-3 (setf zero var-3))
  (when var-4 (setf one var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15)
    (slasdq "U" sqrei nr nrp1 nr ncc
     (f2cl-lib:array-slice d-%data% single-float (nrf) ((1 *))
                           d-%offset%)
     (f2cl-lib:array-slice e-%data% single-float (nrf) ((1 *))
                           e-%offset%)
     (f2cl-lib:array-slice vt-%data% single-float (nrf 1)
                           ((1 ldu) (1 *)) vt-%offset%)
     ldu
     (f2cl-lib:array-slice u-%data% single-float (nrf 1)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu
     (f2cl-lib:array-slice u-%data% single-float (nrf 1)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu
     (f2cl-lib:array-slice work-%data% single-float (nwork1)
                           ((1 *)) work-%offset%)
     info)
  (declare (ignore var-0 var-6 var-7 var-8 var-10 var-12 var-14))
  (when var-1 (setf sqrei var-1))
  (when var-2 (setf nr var-2))
  (when var-3 (setf nrp1 var-3))
  (when var-4 (setf nr var-4))
  (when var-5 (setf ncc var-5))
  (when var-9 (setf ldu var-9))
  (when var-11 (setf ldu var-11))
  (when var-13 (setf ldu var-13))
  (when var-15 (setf info var-15)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (scopy nrp1
     (f2cl-lib:array-slice vt-%data% single-float (nrf 1)
                           ((1 ldu) (1 *)) vt-%offset%)
     1
     (f2cl-lib:array-slice work-%data% single-float (vfi) ((1 *))
                           work-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-4))
```

```
        (when var-0 (setf nrp1 var-0)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (scopy nrp1
           (f2cl-lib:array-slice vt-%data% single-float (nrf nrp1)
                                 ((1 ldu) (1 *)) vt-%offset%)
           1
           (f2cl-lib:array-slice work-%data% single-float (vli) ((1 *))
                                 work-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf nrp1 var-0)))))
    (cond ((/= info 0) (go end_label)))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j nr) nil)
      (tagbody
        (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add idxqi j))
                             ((1 *)) iwork-%offset%)
              j)
       label20))
   label30))
(setf j (expt 2 nlvl))
(f2cl-lib:fdo (lvl nlvl (f2cl-lib:int-add lvl (f2cl-lib:int-sub 1)))
              ((> lvl 1) nil)
  (tagbody
    (setf lvl2 (f2cl-lib:int-sub (f2cl-lib:int-mul lvl 2) 1))
    (cond ((= lvl 1) (setf lf 1) (setf ll 1))
          (t (setf lf (expt 2 (f2cl-lib:int-sub lvl 1)))
           (setf ll (f2cl-lib:int-sub (f2cl-lib:int-mul 2 lf) 1))))
    (f2cl-lib:fdo (i lf (f2cl-lib:int-add i 1))
                  ((> i ll) nil)
      (tagbody
        (setf im1 (f2cl-lib:int-sub i 1))
        (setf ic
                (f2cl-lib:fref iwork-%data%
                               ((f2cl-lib:int-add inode im1)) ((1 *))
                               iwork-%offset%))
        (setf nl
                (f2cl-lib:fref iwork-%data%
                               ((f2cl-lib:int-add ndiml im1)) ((1 *))
                               iwork-%offset%))
        (setf nr
                (f2cl-lib:fref iwork-%data%
                               ((f2cl-lib:int-add ndimr im1)) ((1 *))
                               iwork-%offset%))
        (setf nlf (f2cl-lib:int-sub ic nl))
        (setf nrf (f2cl-lib:int-add ic 1))
        (cond ((= i ll) (setf sqrei sqre)) (t (setf sqrei 1)))
        (setf vfi (f2cl-lib:int-sub (f2cl-lib:int-add vf nlf) 1))
        (setf vli (f2cl-lib:int-sub (f2cl-lib:int-add vl nlf) 1))
        (setf idxqi (f2cl-lib:int-sub (f2cl-lib:int-add idxq nlf) 1))
```

```
(setf alpha (f2cl-lib:fref d-%data% (ic) ((1 *)) d-%offset%))
(setf beta (f2cl-lib:fref e-%data% (ic) ((1 *)) e-%offset%))
(cond
 ((= icompq 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
       var-17 var-18 var-19 var-20 var-21 var-22 var-23 var-24
       var-25)
      (slasd6 icompq nl nr sqrei
       (f2cl-lib:array-slice d-%data% single-float (nlf)
                             ((1 *)) d-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (vfi)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (vli)
                             ((1 *)) work-%offset%)
       alpha beta
       (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                             (idxqi) ((1 *)) iwork-%offset%)
       perm
       (f2cl-lib:fref givptr-%data% (1) ((1 *))
                      givptr-%offset%)
       givcol ldgcol givnum ldu poles difl difr z
       (f2cl-lib:fref k-%data% (1) ((1 *)) k-%offset%)
       (f2cl-lib:fref c-%data% (1) ((1 *)) c-%offset%)
       (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%)
       (f2cl-lib:array-slice work-%data% single-float (nwork1)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                             (iwk) ((1 *)) iwork-%offset%)
       info)
    (declare
     (ignore var-4 var-5 var-6 var-9 var-10 var-12 var-14
      var-16 var-17 var-18 var-19 var-23 var-24))
    (setf icompq var-0)
    (setf nl var-1)
    (setf nr var-2)
    (setf sqrei var-3)
    (setf alpha var-7)
    (setf beta var-8)
    (setf (f2cl-lib:fref givptr-%data% (1) ((1 *))
                         givptr-%offset%)
          var-11)
    (setf ldgcol var-13)
    (setf ldu var-15)
    (setf (f2cl-lib:fref k-%data% (1) ((1 *)) k-%offset%)
          var-20)
    (setf (f2cl-lib:fref c-%data% (1) ((1 *)) c-%offset%)
          var-21)
    (setf (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%)
```

```
            var-22)
    (setf info var-25)))
  (t (setf j (f2cl-lib:int-sub j 1))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
        var-17 var-18 var-19 var-20 var-21 var-22 var-23 var-24
        var-25)
       (slasd6 icompq nl nr sqrei
        (f2cl-lib:array-slice d-%data% single-float (nlf)
                              ((1 *)) d-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (vfi)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (vli)
                              ((1 *)) work-%offset%)
        alpha beta
        (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                              (idxqi) ((1 *)) iwork-%offset%)
        (f2cl-lib:array-slice perm-%data% f2cl-lib:integer4
                              (nlf lvl) ((1 ldgcol) (1 *))
                              perm-%offset%)
        (f2cl-lib:fref givptr-%data% (j) ((1 *))
                       givptr-%offset%)
        (f2cl-lib:array-slice givcol-%data% f2cl-lib:integer4
                              (nlf lvl2) ((1 ldgcol) (1 *))
                              givcol-%offset%)
        ldgcol
        (f2cl-lib:array-slice givnum-%data% single-float
                              (nlf lvl2) ((1 ldu) (1 *))
                              givnum-%offset%)
        ldu
        (f2cl-lib:array-slice poles-%data% single-float
                              (nlf lvl2) ((1 ldu) (1 *))
                              poles-%offset%)
        (f2cl-lib:array-slice difl-%data% single-float (nlf lvl)
                              ((1 ldu) (1 *)) difl-%offset%)
        (f2cl-lib:array-slice difr-%data% single-float
                              (nlf lvl2) ((1 ldu) (1 *))
                              difr-%offset%)
        (f2cl-lib:array-slice z-%data% single-float (nlf lvl)
                              ((1 ldu) (1 *)) z-%offset%)
        (f2cl-lib:fref k-%data% (j) ((1 *)) k-%offset%)
        (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
        (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
        (f2cl-lib:array-slice work-%data% single-float (nwork1)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                              (iwk) ((1 *)) iwork-%offset%)
        info)
    (declare
```

```
                    (ignore var-4 var-5 var-6 var-9 var-10 var-12 var-14
                     var-16 var-17 var-18 var-19 var-23 var-24))
                   (setf icompq var-0)
                   (setf nl var-1)
                   (setf nr var-2)
                   (setf sqrei var-3)
                   (setf alpha var-7)
                   (setf beta var-8)
                   (setf (f2cl-lib:fref givptr-%data% (j) ((1 *))
                                        givptr-%offset%)
                           var-11)
                   (setf ldgcol var-13)
                   (setf ldu var-15)
                   (setf (f2cl-lib:fref k-%data% (j) ((1 *)) k-%offset%)
                           var-20)
                   (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                           var-21)
                   (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                           var-22)
                   (setf info var-25))))
                 (cond ((/= info 0) (go end_label)))
                label40))
            label50))
        (go end_label)
       end_label
        (return
         (values icompq smlsiz n sqre nil nil nil ldu nil nil nil nil nil nil
                 nil nil ldgcol nil nil nil nil nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slasda
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
```

```
  (array fortran-to-lisp::integer4
   (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4)
  (array fortran-to-lisp::integer4
   (*))
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array single-float (*))
  (array fortran-to-lisp::integer4
   (*))
  (fortran-to-lisp::integer4))
:return-values
'(fortran-to-lisp::icompq
  fortran-to-lisp::smlsiz
  fortran-to-lisp::n
  fortran-to-lisp::sqre nil nil nil
  fortran-to-lisp::ldu nil nil nil
  nil nil nil nil nil
  fortran-to-lisp::ldgcol nil nil
  nil nil nil nil
  fortran-to-lisp::info)
:calls '(fortran-to-lisp::slasd6))))
```

———-

90.102 slasdq

90.102.1 Axiom unit tests

— **slasdq.input** —

```
)set break resume
)sys rm -f slasdq.output
)spool slasdq.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.102.2 Axiom help page

— slasdq.help —

```
==================================================================
slasdq examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slasdq (UPLO, SQRE, N, NCVT, NRU, NCC, D, E, VT, LDVT, U,
           LDU, C, LDC, WORK, INFO)
           SLASDQ computes the SVD of a real bidiagonal matrix with diagonal d
           and off-diagonal e. Used by sbdsdc.

Function/Subroutine Documentation
   subroutine slasdq (character UPLO, integer SQRE, integer N, integer NCVT,
       integer NRU, integer NCC, real, dimension( * ) D, real, dimension( * )
       E, real, dimension( ldvt, * ) VT, integer LDVT, real, dimension( ldu, *
       ) U, integer LDU, real, dimension( ldc, * ) C, integer LDC, real,
       dimension( * ) WORK, integer INFO)
       SLASDQ computes the SVD of a real bidiagonal matrix with diagonal d and
       off-diagonal e. Used by sbdsdc.

Purpose:
   SLASDQ computes the singular value decomposition (SVD) of a real
   (upper or lower) bidiagonal matrix with diagonal D and offdiagonal
   E, accumulating the transformations if desired. Letting B denote
   the input bidiagonal matrix, the algorithm computes orthogonal
   matrices Q and P such that B = Q * S * P**T (P**T denotes the transpose
   of P). The singular values S are overwritten on D.

   The input matrix U  is changed to U  * Q  if desired.
   The input matrix VT is changed to P**T * VT if desired.
   The input matrix C  is changed to Q**T * C  if desired.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   On entry, UPLO specifies whether the input bidiagonal matrix
   is upper or lower bidiagonal, and wether it is square are not.
   UPLO = 'U' or 'u'   B is upper bidiagonal.
   UPLO = 'L' or 'l'   B is lower bidiagonal.
```

```
SQRE
  SQRE is INTEGER
  = 0: then the input matrix is N-by-N.
  = 1: then the input matrix is N-by-(N+1) if UPLU = 'U' and
  (N+1)-by-N if UPLU = 'L'.

  The bidiagonal matrix has
  N = NL + NR + 1 rows and
  M = N + SQRE >= N columns.

N
  N is INTEGER
  On entry, N specifies the number of rows and columns
  in the matrix. N must be at least 0.

NCVT
  NCVT is INTEGER
  On entry, NCVT specifies the number of columns of
  the matrix VT. NCVT must be at least 0.

NRU
  NRU is INTEGER
  On entry, NRU specifies the number of rows of
  the matrix U. NRU must be at least 0.

NCC
  NCC is INTEGER
  On entry, NCC specifies the number of columns of
  the matrix C. NCC must be at least 0.

D
  D is REAL array, dimension (N)
  On entry, D contains the diagonal entries of the
  bidiagonal matrix whose SVD is desired. On normal exit,
  D contains the singular values in ascending order.

E
  E is REAL array.
  dimension is (N-1) if SQRE = 0 and N if SQRE = 1.
  On entry, the entries of E contain the offdiagonal entries
  of the bidiagonal matrix whose SVD is desired. On normal
  exit, E will contain 0. If the algorithm does not converge,
  D and E will contain the diagonal and superdiagonal entries
  of a bidiagonal matrix orthogonally equivalent to the one given as input.

VT
  VT is REAL array, dimension (LDVT, NCVT)
  On entry, contains a matrix which on exit has been
  premultiplied by P**T, dimension N-by-NCVT if SQRE = 0
  and (N+1)-by-NCVT if SQRE = 1 (not referenced if NCVT=0).
```

```
LDVT
  LDVT is INTEGER
  On entry, LDVT specifies the leading dimension of VT as
  declared in the calling (sub) program. LDVT must be at
  least 1. If NCVT is nonzero LDVT must also be at least N.

U
  U is REAL array, dimension (LDU, N)
  On entry, contains a  matrix which on exit has been
  postmultiplied by Q, dimension NRU-by-N if SQRE = 0
  and NRU-by-(N+1) if SQRE = 1 (not referenced if NRU=0).

LDU
  LDU is INTEGER
  On entry, LDU  specifies the leading dimension of U as
  declared in the calling (sub) program. LDU must be at least max( 1, NRU ) .

C
  C is REAL array, dimension (LDC, NCC)
  On entry, contains an N-by-NCC matrix which on exit
  has been premultiplied by Q**T  dimension N-by-NCC if SQRE = 0
  and (N+1)-by-NCC if SQRE = 1 (not referenced if NCC=0).

LDC
  LDC is INTEGER
  On entry, LDC  specifies the leading dimension of C as
  declared in the calling (sub) program. LDC must be at
  least 1. If NCC is nonzero, LDC must also be at least N.

WORK
  WORK is REAL array, dimension (4*N)
  Workspace. Only referenced if one of NCVT, NRU, or NCC is
  nonzero, and if N is at least 2.

INFO
  INFO is INTEGER
  On exit, a value of 0 indicates a successful exit.
  If INFO < 0, argument number -INFO is illegal.
  If INFO > 0, the algorithm did not converge, and INFO
  specifies how many superdiagonals did not converge.
```

———-

90.102.3 fortran code

— slasdq.f —

```
*  =====================================================================
      SUBROUTINE SLASDQ( UPLO, SQRE, N, NCVT, NRU, NCC, D, E, VT, LDVT,
     $                   U, LDU, C, LDC, WORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDC, LDU, LDVT, N, NCC, NCVT, NRU, SQRE
*     ..
*     .. Array Arguments ..
      REAL               C( LDC, * ), D( * ), E( * ), U( LDU, * ),
     $                   VT( LDVT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ROTATE
      INTEGER            I, ISUB, IUPLO, J, NP1, SQRE1
      REAL               CS, R, SMIN, SN
*     ..
*     .. External Subroutines ..
      EXTERNAL           SBDSQR, SLARTG, SLASR, SSWAP, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IUPLO = 0
      IF( LSAME( UPLO, 'U' ) )
     $   IUPLO = 1
      IF( LSAME( UPLO, 'L' ) )
     $   IUPLO = 2
      IF( IUPLO.EQ.0 ) THEN
```

```
         INFO = -1
      ELSE IF( ( SQRE.LT.0 ) .OR. ( SQRE.GT.1 ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NCVT.LT.0 ) THEN
         INFO = -4
      ELSE IF( NRU.LT.0 ) THEN
         INFO = -5
      ELSE IF( NCC.LT.0 ) THEN
         INFO = -6
      ELSE IF( ( NCVT.EQ.0 .AND. LDVT.LT.1 ) .OR.
     $         ( NCVT.GT.0 .AND. LDVT.LT.MAX( 1, N ) ) ) THEN
         INFO = -10
      ELSE IF( LDU.LT.MAX( 1, NRU ) ) THEN
         INFO = -12
      ELSE IF( ( NCC.EQ.0 .AND. LDC.LT.1 ) .OR.
     $         ( NCC.GT.0 .AND. LDC.LT.MAX( 1, N ) ) ) THEN
         INFO = -14
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'SLASDQ', -INFO )
         RETURN
      END IF
      IF( N.EQ.0 )
     $   RETURN
*
*     ROTATE is true if any singular vectors desired, false otherwise
*
      ROTATE = ( NCVT.GT.0 ) .OR. ( NRU.GT.0 ) .OR. ( NCC.GT.0 )
      NP1 = N + 1
      SQRE1 = SQRE
*
*     If matrix non-square upper bidiagonal, rotate to be lower
*     bidiagonal.  The rotations are on the right.
*
      IF( ( IUPLO.EQ.1 ) .AND. ( SQRE1.EQ.1 ) ) THEN
         DO 10 I = 1, N - 1
            CALL SLARTG( D( I ), E( I ), CS, SN, R )
            D( I ) = R
            E( I ) = SN*D( I+1 )
            D( I+1 ) = CS*D( I+1 )
            IF( ROTATE ) THEN
               WORK( I ) = CS
               WORK( N+I ) = SN
            END IF
   10    CONTINUE
         CALL SLARTG( D( N ), E( N ), CS, SN, R )
         D( N ) = R
         E( N ) = ZERO
```

```
         IF( ROTATE ) THEN
            WORK( N ) = CS
            WORK( N+N ) = SN
         END IF
         IUPLO = 2
         SQRE1 = 0
*
*        Update singular vectors if desired.
*
         IF( NCVT.GT.0 )
     $      CALL SLASR( 'L', 'V', 'F', NP1, NCVT, WORK( 1 ),
     $                  WORK( NP1 ), VT, LDVT )
      END IF
*
*     If matrix lower bidiagonal, rotate to be upper bidiagonal
*     by applying Givens rotations on the left.
*
      IF( IUPLO.EQ.2 ) THEN
         DO 20 I = 1, N - 1
            CALL SLARTG( D( I ), E( I ), CS, SN, R )
            D( I ) = R
            E( I ) = SN*D( I+1 )
            D( I+1 ) = CS*D( I+1 )
            IF( ROTATE ) THEN
               WORK( I ) = CS
               WORK( N+I ) = SN
            END IF
   20    CONTINUE
*
*        If matrix (N+1)-by-N lower bidiagonal, one additional
*        rotation is needed.
*
         IF( SQRE1.EQ.1 ) THEN
            CALL SLARTG( D( N ), E( N ), CS, SN, R )
            D( N ) = R
            IF( ROTATE ) THEN
               WORK( N ) = CS
               WORK( N+N ) = SN
            END IF
         END IF
*
*        Update singular vectors if desired.
*
         IF( NRU.GT.0 ) THEN
            IF( SQRE1.EQ.0 ) THEN
               CALL SLASR( 'R', 'V', 'F', NRU, N, WORK( 1 ),
     $                     WORK( NP1 ), U, LDU )
            ELSE
               CALL SLASR( 'R', 'V', 'F', NRU, NP1, WORK( 1 ),
     $                     WORK( NP1 ), U, LDU )
```

```
         END IF
      END IF
      IF( NCC.GT.0 ) THEN
         IF( SQRE1.EQ.0 ) THEN
            CALL SLASR( 'L', 'V', 'F', N, NCC, WORK( 1 ),
     $                  WORK( NP1 ), C, LDC )
         ELSE
            CALL SLASR( 'L', 'V', 'F', NP1, NCC, WORK( 1 ),
     $                  WORK( NP1 ), C, LDC )
         END IF
      END IF
   END IF
*
*     Call SBDSQR to compute the SVD of the reduced real
*     N-by-N upper bidiagonal matrix.
*
   CALL SBDSQR( 'U', N, NCVT, NRU, NCC, D, E, VT, LDVT, U, LDU, C,
     $          LDC, WORK, INFO )
*
*     Sort the singular values into ascending order (insertion sort on
*     singular values, but only one transposition per singular vector)
*
   DO 40 I = 1, N
*
*       Scan for smallest D(I).
*
      ISUB = I
      SMIN = D( I )
      DO 30 J = I + 1, N
         IF( D( J ).LT.SMIN ) THEN
            ISUB = J
            SMIN = D( J )
         END IF
30    CONTINUE
      IF( ISUB.NE.I ) THEN
*
*          Swap singular values and vectors.
*
         D( ISUB ) = D( I )
         D( I ) = SMIN
         IF( NCVT.GT.0 )
     $      CALL SSWAP( NCVT, VT( ISUB, 1 ), LDVT, VT( I, 1 ), LDVT )
         IF( NRU.GT.0 )
     $      CALL SSWAP( NRU, U( 1, ISUB ), 1, U( 1, I ), 1 )
         IF( NCC.GT.0 )
     $      CALL SSWAP( NCC, C( ISUB, 1 ), LDC, C( I, 1 ), LDC )
      END IF
40 CONTINUE
*
   RETURN
```

```
*
*     End of SLASDQ
*
      END
```

———-

90.102.4 lisp code

— **slasdq.lisp** —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun slasdq (uplo sqre n ncvt nru ncc d e vt ldvt u ldu c ldc work info)
    (declare (type (array single-float (*)) work c u vt e d)
             (type (f2cl-lib:integer4) info ldc ldu ldvt ncc nru ncvt n sqre)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%)
         (vt single-float vt-%data% vt-%offset%)
         (u single-float u-%data% u-%offset%)
         (c single-float c-%data% c-%offset%)
         (work single-float work-%data% work-%offset%))
      (prog ((cs 0.0) (r 0.0) (smin 0.0) (sn 0.0) (i 0) (isub 0) (iuplo 0)
             (j 0) (np1 0) (sqre1 0) (rotate nil))
        (declare (type (single-float) cs r smin sn)
                 (type (f2cl-lib:integer4) i isub iuplo j np1 sqre1)
                 (type f2cl-lib:logical rotate))
        (setf info 0)
        (setf iuplo 0)
        (if (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)
            (setf iuplo 1))
        (if (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)
            (setf iuplo 2))
        (cond ((= iuplo 0) (setf info -1))
              ((or (< sqre 0) (> sqre 1)) (setf info -2))
              ((< n 0) (setf info -3)) ((< ncvt 0) (setf info -4))
```

```
      ((< nru 0) (setf info -5)) ((< ncc 0) (setf info -6))
      ((or (and (= ncvt 0) (< ldvt 1))
           (and (> ncvt 0)
                (< ldvt
                   (max (the f2cl-lib:integer4 1)
                        (the f2cl-lib:integer4 n)))))
       (setf info -10))
      ((< ldu
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nru)))
       (setf info -12))
      ((or (and (= ncc 0) (< ldc 1))
           (and (> ncc 0)
                (< ldc
                   (max (the f2cl-lib:integer4 1)
                        (the f2cl-lib:integer4 n)))))
       (setf info -14)))
(cond
 ((/= info 0) (xerbla "SLASDQ" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(setf rotate (or (> ncvt 0) (> nru 0) (> ncc 0)))
(setf np1 (f2cl-lib:int-add n 1))
(setf sqre1 sqre)
(cond
 ((and (= iuplo 1) (= sqre1 1))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (slartg (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
           (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) cs sn r)
        (declare (ignore var-0 var-1))
        (setf cs var-2)
        (setf sn var-3)
        (setf r var-4))
      (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) r)
      (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
              (* sn
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                ((1 *)) d-%offset%)))
      (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                           d-%offset%)
              (* cs
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                ((1 *)) d-%offset%)))
      (cond
       (rotate
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) cs)
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
```

```
                                ((1 *)) work-%offset%)
                  sn)))
       label10))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (slartg (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
         (f2cl-lib:fref e-%data% (n) ((1 *)) e-%offset%) cs sn r)
      (declare (ignore var-0 var-1))
      (setf cs var-2)
      (setf sn var-3)
      (setf r var-4))
    (setf (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) r)
    (setf (f2cl-lib:fref e-%data% (n) ((1 *)) e-%offset%) zero)
    (cond
     (rotate
      (setf (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%) cs)
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n n)) ((1 *))
                           work-%offset%)
               sn)))
    (setf iuplo 2) (setf sqre1 0)
    (if (> ncvt 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (slasr "L" "V" "F" np1 ncvt
             (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                                   work-%offset%)
             (f2cl-lib:array-slice work-%data% single-float (np1) ((1 *))
                                   work-%offset%)
             vt ldvt)
          (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
          (when var-3 (setf np1 var-3))
          (when var-4 (setf ncvt var-4))
          (when var-8 (setf ldvt var-8))))))
(cond
 ((= iuplo 2)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (slartg (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
           (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) cs sn r)
        (declare (ignore var-0 var-1))
        (setf cs var-2)
        (setf sn var-3)
        (setf r var-4))
      (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) r)
      (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
              (* sn
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                ((1 *)) d-%offset%)))
      (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
```

```
                          d-%offset%)
                 (* cs
                    (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                   ((1 *)) d-%offset%)))
        (cond
         (rotate
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) cs)
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                               ((1 *)) work-%offset%)
                sn)))
       label20))
    (cond
     ((= sqre1 1)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (slartg (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
           (f2cl-lib:fref e-%data% (n) ((1 *)) e-%offset%) cs sn r)
        (declare (ignore var-0 var-1))
        (setf cs var-2)
        (setf sn var-3)
        (setf r var-4))
      (setf (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) r)
      (cond
       (rotate
        (setf (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%) cs)
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n n)) ((1 *))
                             work-%offset%)
                sn)))))
    (cond
     ((> nru 0)
      (cond
       ((= sqre1 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (slasr "R" "V" "F" nru n
             (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                                   work-%offset%)
             (f2cl-lib:array-slice work-%data% single-float (np1) ((1 *))
                                   work-%offset%)
             u ldu)
          (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
          (when var-3 (setf nru var-3))
          (when var-4 (setf n var-4))
          (when var-8 (setf ldu var-8))))
       (t
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (slasr "R" "V" "F" nru np1
             (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                                   work-%offset%)
             (f2cl-lib:array-slice work-%data% single-float (np1) ((1 *))
```

```
                                 work-%offset%)
            u ldu)
          (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
          (when var-3 (setf nru var-3))
          (when var-4 (setf np1 var-4))
          (when var-8 (setf ldu var-8)))))))
    (cond
     ((> ncc 0)
      (cond
       ((= sqre1 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (slasr "L" "V" "F" n ncc
             (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                                   work-%offset%)
             (f2cl-lib:array-slice work-%data% single-float (np1) ((1 *))
                                   work-%offset%)
             c ldc)
          (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
          (when var-3 (setf n var-3))
          (when var-4 (setf ncc var-4))
          (when var-8 (setf ldc var-8))))
       (t
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (slasr "L" "V" "F" np1 ncc
             (f2cl-lib:array-slice work-%data% single-float (1) ((1 *))
                                   work-%offset%)
             (f2cl-lib:array-slice work-%data% single-float (np1) ((1 *))
                                   work-%offset%)
             c ldc)
          (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
          (when var-3 (setf np1 var-3))
          (when var-4 (setf ncc var-4))
          (when var-8 (setf ldc var-8)))))))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
       var-11 var-12 var-13 var-14)
      (sbdsqr "U" n ncvt nru ncc d e vt ldvt u ldu c ldc work info)
    (declare (ignore var-0 var-5 var-6 var-7 var-9 var-11 var-13))
    (setf n var-1)
    (setf ncvt var-2)
    (setf nru var-3)
    (setf ncc var-4)
    (setf ldvt var-8)
    (setf ldu var-10)
    (setf ldc var-12)
    (setf info var-14))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
```

```
(tagbody
  (setf isub i)
  (setf smin (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
  (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (cond
       ((< (f2cl-lib:fref d (j) ((1 *))) smin) (setf isub j)
        (setf smin (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%))))
     label30))
  (cond
   ((/= isub i)
    (setf (f2cl-lib:fref d-%data% (isub) ((1 *)) d-%offset%)
            (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
    (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) smin)
    (if (> ncvt 0)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (sswap ncvt
             (f2cl-lib:array-slice vt-%data% single-float (isub 1)
                                   ((1 ldvt) (1 *)) vt-%offset%)
             ldvt
             (f2cl-lib:array-slice vt-%data% single-float (i 1)
                                   ((1 ldvt) (1 *)) vt-%offset%)
             ldvt)
          (declare (ignore var-1 var-3))
          (when var-0 (setf ncvt var-0))
          (when var-2 (setf ldvt var-2))
          (when var-4 (setf ldvt var-4))))
    (if (> nru 0)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (sswap nru
             (f2cl-lib:array-slice u-%data% single-float (1 isub)
                                   ((1 ldu) (1 *)) u-%offset%)
             1
             (f2cl-lib:array-slice u-%data% single-float (1 i)
                                   ((1 ldu) (1 *)) u-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf nru var-0))))
    (if (> ncc 0)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (sswap ncc
             (f2cl-lib:array-slice c-%data% single-float (isub 1)
                                   ((1 ldc) (1 *)) c-%offset%)
             ldc
             (f2cl-lib:array-slice c-%data% single-float (i 1)
                                   ((1 ldc) (1 *)) c-%offset%)
             ldc)
          (declare (ignore var-1 var-3))
          (when var-0 (setf ncc var-0))
```

```
                    (when var-2 (setf ldc var-2))
                    (when var-4 (setf ldc var-4))))))
           label40))
         (go end_label)
        end_label
         (return
          (values uplo nil n ncvt nru ncc nil nil nil ldvt nil ldu nil ldc nil
                  info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slasdq
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::uplo nil
                                              fortran-to-lisp::n
                                              fortran-to-lisp::ncvt
                                              fortran-to-lisp::nru
                                              fortran-to-lisp::ncc nil nil nil
                                              fortran-to-lisp::ldvt nil
                                              fortran-to-lisp::ldu nil
                                              fortran-to-lisp::ldc nil
                                              fortran-to-lisp::info)
                                            :calls
                                            '(fortran-to-lisp::sbdsqr
                                              fortran-to-lisp::slartg))))
```

90.103 slasdt

90.103.1 Axiom unit tests

— slasdt.input —

```
)set break resume
)sys rm -f slasdt.output
)spool slasdt.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.103.2 Axiom help page

— slasdt.help —

```
======================================================================
slasdt examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slasdt (N, LVL, ND, INODE, NDIML, NDIMR, MSUB)
           SLASDT creates a tree of subproblems for bidiagonal divide and
           conquer. Used by sbdsdc.

Function/Subroutine Documentation
   subroutine slasdt (integer N, integer LVL, integer ND, integer, dimension(
       * ) INODE, integer, dimension( * ) NDIML, integer, dimension( * )
       NDIMR, integer MSUB)
       SLASDT creates a tree of subproblems for bidiagonal divide and conquer.
       Used by sbdsdc.

Purpose:
   SLASDT creates a tree of subproblems for bidiagonal divide and conquer.
```

```
Parameters:

 N
   N is INTEGER
   On entry, the number of diagonal elements of the bidiagonal matrix.

 LVL
   LVL is INTEGER
   On exit, the number of levels on the computation tree.

 ND
   ND is INTEGER
   On exit, the number of nodes on the tree.

 INODE
   INODE is INTEGER array, dimension ( N )
   On exit, centers of subproblems.

 NDIML
   NDIML is INTEGER array, dimension ( N )
   On exit, row dimensions of left children.

 NDIMR
   NDIMR is INTEGER array, dimension ( N )
   On exit, row dimensions of right children.

 MSUB
   MSUB is INTEGER
   On entry, the maximum row dimension each subproblem at the
   bottom of the tree can be of.
```

90.103.3 fortran code

— slasdt.f —

```
*  =====================================================================
      SUBROUTINE SLASDT( N, LVL, ND, INODE, NDIML, NDIMR, MSUB )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            LVL, MSUB, N, ND
```

```
*     ..
*     .. Array Arguments ..
      INTEGER            INODE( * ), NDIML( * ), NDIMR( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               TWO
      PARAMETER          ( TWO = 2.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IL, IR, LLST, MAXN, NCRNT, NLVL
      REAL               TEMP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          INT, LOG, MAX, REAL
*     ..
*     .. Executable Statements ..
*
*     Find the number of levels on the tree.
*
      MAXN = MAX( 1, N )
      TEMP = LOG( REAL( MAXN ) / REAL( MSUB+1 ) ) / LOG( TWO )
      LVL = INT( TEMP ) + 1
*
      I = N / 2
      INODE( 1 ) = I + 1
      NDIML( 1 ) = I
      NDIMR( 1 ) = N - I - 1
      IL = 0
      IR = 1
      LLST = 1
      DO 20 NLVL = 1, LVL - 1
*
*        Constructing the tree at (NLVL+1)-st level. The number of
*        nodes created on this level is LLST * 2.
*
         DO 10 I = 0, LLST - 1
            IL = IL + 2
            IR = IR + 2
            NCRNT = LLST + I
            NDIML( IL ) = NDIML( NCRNT ) / 2
            NDIMR( IL ) = NDIML( NCRNT ) - NDIML( IL ) - 1
            INODE( IL ) = INODE( NCRNT ) - NDIMR( IL ) - 1
            NDIML( IR ) = NDIMR( NCRNT ) / 2
            NDIMR( IR ) = NDIMR( NCRNT ) - NDIML( IR ) - 1
            INODE( IR ) = INODE( NCRNT ) + NDIML( IR ) + 1
   10    CONTINUE
         LLST = LLST*2
```

```
   20 CONTINUE
      ND = LLST*2 - 1
*
      RETURN
*
*     End of SLASDT
*
      END
```

———-

90.103.4 lisp code

— **slasdt.lisp** —

```
(let* ((two 2.0))
  (declare (type (single-float 2.0 2.0) two)
           (ignorable two))
  (defun slasdt (n lvl nd inode ndiml ndimr msub)
    (declare (type (array f2cl-lib:integer4 (*)) ndimr ndiml inode)
             (type (f2cl-lib:integer4) msub nd lvl n))
    (f2cl-lib:with-multi-array-data
        ((inode f2cl-lib:integer4 inode-%data% inode-%offset%)
         (ndiml f2cl-lib:integer4 ndiml-%data% ndiml-%offset%)
         (ndimr f2cl-lib:integer4 ndimr-%data% ndimr-%offset%))
      (prog ((temp 0.0) (i 0) (il 0) (ir 0) (llst 0) (maxn 0) (ncrnt 0)
             (nlvl 0))
        (declare (type (single-float) temp)
                 (type (f2cl-lib:integer4) i il ir llst maxn ncrnt nlvl))
        (setf maxn (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf temp
                (/
                 (f2cl-lib:flog
                  (/ (f2cl-lib:freal maxn)
                     (f2cl-lib:freal (f2cl-lib:int-add msub 1))))
                 (f2cl-lib:flog two)))
        (setf lvl (f2cl-lib:int-add (f2cl-lib:int temp) 1))
        (setf i (the f2cl-lib:integer4 (truncate n 2)))
        (setf (f2cl-lib:fref inode-%data% (1) ((1 *)) inode-%offset%)
                (f2cl-lib:int-add i 1))
        (setf (f2cl-lib:fref ndiml-%data% (1) ((1 *)) ndiml-%offset%) i)
        (setf (f2cl-lib:fref ndimr-%data% (1) ((1 *)) ndimr-%offset%)
                (f2cl-lib:int-sub n i 1))
        (setf il 0)
        (setf ir 1)
        (setf llst 1)
        (f2cl-lib:fdo (nlvl 1 (f2cl-lib:int-add nlvl 1))
                      ((> nlvl (f2cl-lib:int-add lvl (f2cl-lib:int-sub 1))) nil)
```

```
(tagbody
  (f2cl-lib:fdo (i 0 (f2cl-lib:int-add i 1))
                ((> i (f2cl-lib:int-add llst (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (setf il (f2cl-lib:int-add il 2))
      (setf ir (f2cl-lib:int-add ir 2))
      (setf ncrnt (f2cl-lib:int-add llst i))
      (setf (f2cl-lib:fref ndiml-%data% (il) ((1 *)) ndiml-%offset%)
              (the f2cl-lib:integer4
                   (truncate
                    (f2cl-lib:fref ndiml-%data% (ncrnt) ((1 *))
                                   ndiml-%offset%)
                    2)))
      (setf (f2cl-lib:fref ndimr-%data% (il) ((1 *)) ndimr-%offset%)
              (f2cl-lib:int-sub
               (f2cl-lib:fref ndiml-%data% (ncrnt) ((1 *))
                              ndiml-%offset%)
               (f2cl-lib:fref ndiml-%data% (il) ((1 *))
                              ndiml-%offset%)
               1))
      (setf (f2cl-lib:fref inode-%data% (il) ((1 *)) inode-%offset%)
              (f2cl-lib:int-sub
               (f2cl-lib:fref inode-%data% (ncrnt) ((1 *))
                              inode-%offset%)
               (f2cl-lib:fref ndimr-%data% (il) ((1 *))
                              ndimr-%offset%)
               1))
      (setf (f2cl-lib:fref ndiml-%data% (ir) ((1 *)) ndiml-%offset%)
              (the f2cl-lib:integer4
                   (truncate
                    (f2cl-lib:fref ndimr-%data% (ncrnt) ((1 *))
                                   ndimr-%offset%)
                    2)))
      (setf (f2cl-lib:fref ndimr-%data% (ir) ((1 *)) ndimr-%offset%)
              (f2cl-lib:int-sub
               (f2cl-lib:fref ndimr-%data% (ncrnt) ((1 *))
                              ndimr-%offset%)
               (f2cl-lib:fref ndiml-%data% (ir) ((1 *))
                              ndiml-%offset%)
               1))
      (setf (f2cl-lib:fref inode-%data% (ir) ((1 *)) inode-%offset%)
              (f2cl-lib:int-add
               (f2cl-lib:fref inode-%data% (ncrnt) ((1 *))
                              inode-%offset%)
               (f2cl-lib:fref ndiml-%data% (ir) ((1 *))
                              ndiml-%offset%)
               1))
    label10))
  (setf llst (f2cl-lib:int-mul llst 2))
```

```
          label20))
        (setf nd (f2cl-lib:int-sub (f2cl-lib:int-mul llst 2) 1))
        (go end_label)
       end_label
        (return (values nil lvl nd nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slasdt
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (array fortran-to-lisp::integer4
                                               (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(nil fortran-to-lisp::lvl
                                              fortran-to-lisp::nd nil nil nil
                                              nil)
                                            :calls 'nil)))
```

———-

90.104 slaset

90.104.1 Axiom unit tests

— **slaset.input** —

```
)set break resume
)sys rm -f slaset.output
)spool slaset.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.104.2 Axiom help page

— slaset.help —

```
=====================================================================
slaset examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slaset (UPLO, M, N, ALPHA, BETA, A, LDA)
           SLASET initializes the off-diagonal elements and the diagonal
           elements of a matrix to given values.

Function/Subroutine Documentation
   subroutine slaset (character UPLO, integer M, integer N, real ALPHA, real
       BETA, real, dimension( lda, * ) A, integer LDA)
       SLASET initializes the off-diagonal elements and the diagonal elements
       of a matrix to given values.

Purpose:
   SLASET initializes an m-by-n matrix A to BETA on the diagonal and
   ALPHA on the offdiagonals.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   Specifies the part of the matrix A to be set.
   = 'U':      Upper triangular part is set; the strictly lower
        triangular part of A is not changed.
   = 'L':      Lower triangular part is set; the strictly upper
        triangular part of A is not changed.
   Otherwise:  All of the matrix A is set.

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.

 N
   N is INTEGER
   The number of columns of the matrix A.  N >= 0.
```

```
 ALPHA
   ALPHA is REAL
   The constant to which the offdiagonal elements are to be set.

 BETA
   BETA is REAL
   The constant to which the diagonal elements are to be set.

 A
   A is REAL array, dimension (LDA,N)
   On exit, the leading m-by-n submatrix of A is set as follows:

   if UPLO = 'U', A(i,j) = ALPHA, 1<=i<=j-1, 1<=j<=n,
   if UPLO = 'L', A(i,j) = ALPHA, j+1<=i<=m, 1<=j<=n,
   otherwise,     A(i,j) = ALPHA, 1<=i<=m, 1<=j<=n, i.ne.j,

   and, for all UPLO, A(i,i) = BETA, 1<=i<=min(m,n).

 LDA
   LDA is INTEGER
   The leading dimension of the array A.  LDA >= max(1,M).
```

———-

90.104.3 fortran code

— **slaset.f** —

```
*  =====================================================================
      SUBROUTINE SLASET( UPLO, M, N, ALPHA, BETA, A, LDA )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            LDA, M, N
      REAL               ALPHA, BETA
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * )
*     ..
*
* =====================================================================
```

```
*
*     .. Local Scalars ..
      INTEGER            I, J
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MIN
*     ..
*     .. Executable Statements ..
*
      IF( LSAME( UPLO, 'U' ) ) THEN
*
*        Set the strictly upper triangular or trapezoidal part of the
*        array to ALPHA.
*
         DO 20 J = 2, N
            DO 10 I = 1, MIN( J-1, M )
               A( I, J ) = ALPHA
   10       CONTINUE
   20    CONTINUE
*
      ELSE IF( LSAME( UPLO, 'L' ) ) THEN
*
*        Set the strictly lower triangular or trapezoidal part of the
*        array to ALPHA.
*
         DO 40 J = 1, MIN( M, N )
            DO 30 I = J + 1, M
               A( I, J ) = ALPHA
   30       CONTINUE
   40    CONTINUE
*
      ELSE
*
*        Set the leading m-by-n submatrix to ALPHA.
*
         DO 60 J = 1, N
            DO 50 I = 1, M
               A( I, J ) = ALPHA
   50       CONTINUE
   60    CONTINUE
      END IF
*
*     Set the first min(M,N) diagonal elements to BETA.
*
      DO 70 I = 1, MIN( M, N )
         A( I, I ) = BETA
```

```
   70 CONTINUE
*
      RETURN
*
*     End of SLASET
*
      END
```

———-

90.104.4 lisp code

— **slaset.lisp** —

```
(defun slaset (uplo m n alpha beta a lda)
  (declare (type (array single-float (*)) a)
           (type (single-float) beta alpha)
           (type (f2cl-lib:integer4) lda n m)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a single-float a-%data% a-%offset%))
    (prog ((i 0) (j 0))
      (declare (type (f2cl-lib:integer4) j i))
      (cond
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)
        (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i
                              (min
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                               (the f2cl-lib:integer4 m)))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                        alpha)
               label10))
           label20)))
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
```

```
          (when var-0 (setf uplo var-0))
          ret-val)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j
                          (min (the f2cl-lib:integer4 m)
                               (the f2cl-lib:integer4 n)))
                       nil)
          (tagbody
            (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                        alpha)
               label30))
           label40)))
       (t
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                        alpha)
               label50))
           label60))))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (min (the f2cl-lib:integer4 m)
                             (the f2cl-lib:integer4 n)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%) beta)
         label70))
      (go end_label)
     end_label
      (return (values uplo nil nil nil nil nil nil)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slaset
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
```

```
                                        :return-values
                                        '(fortran-to-lisp::uplo nil nil nil
                                          nil nil nil)
                                        :calls 'nil)))
```

———-

90.105 slasr

90.105.1 Axiom unit tests

— **slasr.input** —

```
)set break resume
)sys rm -f slasr.output
)spool slasr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.105.2 Axiom help page

— **slasr.help** —

```
======================================================================
slasr examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slasr (SIDE, PIVOT, DIRECT, M, N, C, S, A, LDA)
           SLASR applies a sequence of plane rotations to a general
           rectangular matrix.
```

```
Function/Subroutine Documentation
   subroutine slasr (character SIDE, character PIVOT, character DIRECT,
       integer M, integer N, real, dimension( * ) C, real, dimension( * ) S,
       real, dimension( lda, * ) A, integer LDA)
       SLASR applies a sequence of plane rotations to a general rectangular
       matrix.

Purpose:
   SLASR applies a sequence of plane rotations to a real matrix A,
   from either the left or the right.

   When SIDE = 'L', the transformation takes the form

   A := P*A

   and when SIDE = 'R', the transformation takes the form

   A := A*P**T

   where P is an orthogonal matrix consisting of a sequence of z plane
   rotations, with z = M when SIDE = 'L' and z = N when SIDE = 'R',
   and P**T is the transpose of P.

   When DIRECT = 'F' (Forward sequence), then

   P = P(z-1) * ... * P(2) * P(1)

   and when DIRECT = 'B' (Backward sequence), then

   P = P(1) * P(2) * ... * P(z-1)

   where P(k) is a plane rotation matrix defined by the 2-by-2 rotation

   R(k) = (  c(k)  s(k) )
        = ( -s(k)  c(k) ).

   When PIVOT = 'V' (Variable pivot), the rotation is performed
   for the plane (k,k+1), i.e., P(k) has the form

       P(k) = (  1                                          )
              (       ...                                   )
              (             1                               )
              (                  c(k)  s(k)                 )
              (                 -s(k)  c(k)                 )
              (                                1             )
              (                                     ...      )
              (                                            1 )

   where R(k) appears as a rank-2 modification to the identity matrix in
```

```
rows and columns k and k+1.

When PIVOT = 'T' (Top pivot), the rotation is performed for the
plane (1,k+1), so P(k) has the form

    P(k) = (  c(k)                    s(k)                 )
           (         1                                     )
           (              ...                              )
           (                    1                          )
           ( -s(k)                   c(k)                  )
           (                                1              )
           (                                     ...       )
           (                                            1 )

where R(k) appears in rows and columns 1 and k+1.

Similarly, when PIVOT = 'B' (Bottom pivot), the rotation is
performed for the plane (k,z), giving P(k) the form

    P(k) = ( 1                                             )
           (      ...                                      )
           (             1                                 )
           (                  c(k)                   s(k) )
           (                        1                     )
           (                             ...              )
           (                                   1          )
           (                 -s(k)                   c(k) )

where R(k) appears in rows and columns k and z.  The rotations are
performed without ever forming P(k) explicitly.

Parameters:

 SIDE
   SIDE is CHARACTER*1
   Specifies whether the plane rotation matrix P is applied to
   A on the left or the right.
   = 'L':  Left, compute A := P*A
   = 'R':  Right, compute A:= A*P**T

 PIVOT
   PIVOT is CHARACTER*1
   Specifies the plane for which P(k) is a plane rotation matrix.
   = 'V':  Variable pivot, the plane (k,k+1)
   = 'T':  Top pivot, the plane (1,k+1)
   = 'B':  Bottom pivot, the plane (k,z)

 DIRECT
   DIRECT is CHARACTER*1
   Specifies whether P is a forward or backward sequence of
```

```
  plane rotations.
  = 'F':  Forward, P = P(z-1)*...*P(2)*P(1)
  = 'B':  Backward, P = P(1)*P(2)*...*P(z-1)

M
  M is INTEGER
  The number of rows of the matrix A.  If m <= 1, an immediate
  return is effected.

N
  N is INTEGER
  The number of columns of the matrix A.  If n <= 1, an
  immediate return is effected.

C
  C is REAL array, dimension (M-1) if SIDE = 'L', (N-1) if SIDE = 'R'
  The cosines c(k) of the plane rotations.

S
  S is REAL array, dimension (M-1) if SIDE = 'L', (N-1) if SIDE = 'R'
  The sines s(k) of the plane rotations.  The 2-by-2 plane
  rotation part of the matrix P(k), R(k), has the form
     R(k) = (  c(k)  s(k) )
            ( -s(k)  c(k) ).

A
  A is REAL array, dimension (LDA,N)
  The M-by-N matrix A.  On exit, A is overwritten by P*A if
  SIDE = 'R' or by A*P**T if SIDE = 'L'.

LDA
  LDA is INTEGER
  The leading dimension of the array A.  LDA >= max(1,M).
```

———-

90.105.3 fortran code

— **slasr.f** —

```
*  =====================================================================
      SUBROUTINE SLASR( SIDE, PIVOT, DIRECT, M, N, C, S, A, LDA )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
```

```
*
*     .. Scalar Arguments ..
      CHARACTER          DIRECT, PIVOT, SIDE
      INTEGER            LDA, M, N
*     ..
*     .. Array Arguments ..
      REAL               A( LDA, * ), C( * ), S( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ONE, ZERO
      PARAMETER          ( ONE = 1.0E+0, ZERO = 0.0E+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, INFO, J
      REAL               CTEMP, STEMP, TEMP
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( .NOT.( LSAME( SIDE, 'L' ) .OR. LSAME( SIDE, 'R' ) ) ) THEN
         INFO = 1
      ELSE IF( .NOT.( LSAME( PIVOT, 'V' ) .OR. LSAME( PIVOT,
     $         'T' ) .OR. LSAME( PIVOT, 'B' ) ) ) THEN
         INFO = 2
      ELSE IF( .NOT.( LSAME( DIRECT, 'F' ) .OR. LSAME( DIRECT, 'B' ) ) )
     $          THEN
         INFO = 3
      ELSE IF( M.LT.0 ) THEN
         INFO = 4
      ELSE IF( N.LT.0 ) THEN
         INFO = 5
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = 9
      END IF
      IF( INFO.NE.0 ) THEN
```

```
         CALL XERBLA( 'SLASR ', INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( ( M.EQ.0 ) .OR. ( N.EQ.0 ) )
     $   RETURN
      IF( LSAME( SIDE, 'L' ) ) THEN
*
*        Form  P * A
*
         IF( LSAME( PIVOT, 'V' ) ) THEN
            IF( LSAME( DIRECT, 'F' ) ) THEN
               DO 20 J = 1, M - 1
                  CTEMP = C( J )
                  STEMP = S( J )
                  IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                     DO 10 I = 1, N
                        TEMP = A( J+1, I )
                        A( J+1, I ) = CTEMP*TEMP - STEMP*A( J, I )
                        A( J, I ) = STEMP*TEMP + CTEMP*A( J, I )
   10                CONTINUE
                  END IF
   20          CONTINUE
            ELSE IF( LSAME( DIRECT, 'B' ) ) THEN
               DO 40 J = M - 1, 1, -1
                  CTEMP = C( J )
                  STEMP = S( J )
                  IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                     DO 30 I = 1, N
                        TEMP = A( J+1, I )
                        A( J+1, I ) = CTEMP*TEMP - STEMP*A( J, I )
                        A( J, I ) = STEMP*TEMP + CTEMP*A( J, I )
   30                CONTINUE
                  END IF
   40          CONTINUE
            END IF
         ELSE IF( LSAME( PIVOT, 'T' ) ) THEN
            IF( LSAME( DIRECT, 'F' ) ) THEN
               DO 60 J = 2, M
                  CTEMP = C( J-1 )
                  STEMP = S( J-1 )
                  IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                     DO 50 I = 1, N
                        TEMP = A( J, I )
                        A( J, I ) = CTEMP*TEMP - STEMP*A( 1, I )
                        A( 1, I ) = STEMP*TEMP + CTEMP*A( 1, I )
   50                CONTINUE
                  END IF
```

```
   60         CONTINUE
            ELSE IF( LSAME( DIRECT, 'B' ) ) THEN
               DO 80 J = M, 2, -1
                  CTEMP = C( J-1 )
                  STEMP = S( J-1 )
                  IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                     DO 70 I = 1, N
                        TEMP = A( J, I )
                        A( J, I ) = CTEMP*TEMP - STEMP*A( 1, I )
                        A( 1, I ) = STEMP*TEMP + CTEMP*A( 1, I )
   70                CONTINUE
                  END IF
   80         CONTINUE
            END IF
         ELSE IF( LSAME( PIVOT, 'B' ) ) THEN
            IF( LSAME( DIRECT, 'F' ) ) THEN
               DO 100 J = 1, M - 1
                  CTEMP = C( J )
                  STEMP = S( J )
                  IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                     DO 90 I = 1, N
                        TEMP = A( J, I )
                        A( J, I ) = STEMP*A( M, I ) + CTEMP*TEMP
                        A( M, I ) = CTEMP*A( M, I ) - STEMP*TEMP
   90                CONTINUE
                  END IF
  100         CONTINUE
            ELSE IF( LSAME( DIRECT, 'B' ) ) THEN
               DO 120 J = M - 1, 1, -1
                  CTEMP = C( J )
                  STEMP = S( J )
                  IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                     DO 110 I = 1, N
                        TEMP = A( J, I )
                        A( J, I ) = STEMP*A( M, I ) + CTEMP*TEMP
                        A( M, I ) = CTEMP*A( M, I ) - STEMP*TEMP
  110                CONTINUE
                  END IF
  120         CONTINUE
            END IF
         END IF
      ELSE IF( LSAME( SIDE, 'R' ) ) THEN
*
*        Form A * P**T
*
         IF( LSAME( PIVOT, 'V' ) ) THEN
            IF( LSAME( DIRECT, 'F' ) ) THEN
               DO 140 J = 1, N - 1
                  CTEMP = C( J )
                  STEMP = S( J )
```

```
               IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                  DO 130 I = 1, M
                     TEMP = A( I, J+1 )
                     A( I, J+1 ) = CTEMP*TEMP - STEMP*A( I, J )
                     A( I, J ) = STEMP*TEMP + CTEMP*A( I, J )
 130              CONTINUE
               END IF
 140         CONTINUE
         ELSE IF( LSAME( DIRECT, 'B' ) ) THEN
            DO 160 J = N - 1, 1, -1
               CTEMP = C( J )
               STEMP = S( J )
               IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                  DO 150 I = 1, M
                     TEMP = A( I, J+1 )
                     A( I, J+1 ) = CTEMP*TEMP - STEMP*A( I, J )
                     A( I, J ) = STEMP*TEMP + CTEMP*A( I, J )
 150              CONTINUE
               END IF
 160         CONTINUE
         END IF
      ELSE IF( LSAME( PIVOT, 'T' ) ) THEN
         IF( LSAME( DIRECT, 'F' ) ) THEN
            DO 180 J = 2, N
               CTEMP = C( J-1 )
               STEMP = S( J-1 )
               IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                  DO 170 I = 1, M
                     TEMP = A( I, J )
                     A( I, J ) = CTEMP*TEMP - STEMP*A( I, 1 )
                     A( I, 1 ) = STEMP*TEMP + CTEMP*A( I, 1 )
 170              CONTINUE
               END IF
 180         CONTINUE
         ELSE IF( LSAME( DIRECT, 'B' ) ) THEN
            DO 200 J = N, 2, -1
               CTEMP = C( J-1 )
               STEMP = S( J-1 )
               IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                  DO 190 I = 1, M
                     TEMP = A( I, J )
                     A( I, J ) = CTEMP*TEMP - STEMP*A( I, 1 )
                     A( I, 1 ) = STEMP*TEMP + CTEMP*A( I, 1 )
 190              CONTINUE
               END IF
 200         CONTINUE
         END IF
      ELSE IF( LSAME( PIVOT, 'B' ) ) THEN
         IF( LSAME( DIRECT, 'F' ) ) THEN
            DO 220 J = 1, N - 1
```

```
                 CTEMP = C( J )
                 STEMP = S( J )
                 IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                    DO 210 I = 1, M
                       TEMP = A( I, J )
                       A( I, J ) = STEMP*A( I, N ) + CTEMP*TEMP
                       A( I, N ) = CTEMP*A( I, N ) - STEMP*TEMP
  210               CONTINUE
                 END IF
  220         CONTINUE
           ELSE IF( LSAME( DIRECT, 'B' ) ) THEN
              DO 240 J = N - 1, 1, -1
                 CTEMP = C( J )
                 STEMP = S( J )
                 IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                    DO 230 I = 1, M
                       TEMP = A( I, J )
                       A( I, J ) = STEMP*A( I, N ) + CTEMP*TEMP
                       A( I, N ) = CTEMP*A( I, N ) - STEMP*TEMP
  230               CONTINUE
                 END IF
  240         CONTINUE
           END IF
        END IF
     END IF
*
     RETURN
*
*    End of SLASR
*
     END
```

———-

90.105.4 lisp code

— **slasr.lisp** —

```
(let* ((one 1.0) (zero 0.0))
  (declare (type (single-float 1.0 1.0) one)
           (type (single-float 0.0 0.0) zero)
           (ignorable one zero))
  (defun slasr (side pivot direct m n c s a lda)
    (declare (type (array single-float (*)) a s c)
             (type (f2cl-lib:integer4) lda n m)
             (type (string *) direct pivot side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
```

```
    (pivot character pivot-%data% pivot-%offset%)
    (direct character direct-%data% direct-%offset%)
    (c single-float c-%data% c-%offset%)
    (s single-float s-%data% s-%offset%)
    (a single-float a-%data% a-%offset%))
  (prog ((ctemp 0.0) (stemp 0.0) (temp 0.0) (i 0) (info 0) (j 0))
    (declare (type (single-float) ctemp stemp temp)
             (type (f2cl-lib:integer4) i info j))
    (setf info 0)
    (cond
     ((not
       (or
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "L")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val)
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "R")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val)))
      (setf info 1))
     ((not
       (or
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame pivot "V")
          (declare (ignore var-1))
          (when var-0 (setf pivot var-0))
          ret-val)
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame pivot "T")
          (declare (ignore var-1))
          (when var-0 (setf pivot var-0))
          ret-val)
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame pivot "B")
          (declare (ignore var-1))
          (when var-0 (setf pivot var-0))
          ret-val)))
      (setf info 2))
     ((not
       (or
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame direct "F")
          (declare (ignore var-1))
          (when var-0 (setf direct var-0))
          ret-val)
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame direct "B")
```

```
      (declare (ignore var-1))
      (when var-0 (setf direct var-0))
      ret-val)))
  (setf info 3))
 ((< m 0) (setf info 4)) ((< n 0) (setf info 5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info 9)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "SLASR " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= m 0) (= n 0))
    (go end_label))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame side "L")
    (declare (ignore var-1))
    (when var-0 (setf side var-0))
    ret-val)
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame pivot "V")
      (declare (ignore var-1))
      (when var-0 (setf pivot var-0))
      ret-val)
    (cond
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame direct "F")
        (declare (ignore var-1))
        (when var-0 (setf direct var-0))
        ret-val)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
                     nil)
        (tagbody
          (setf ctemp (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
          (setf stemp (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
          (cond
           ((or (/= ctemp one) (/= stemp zero))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf temp
                        (f2cl-lib:fref a-%data%
                                       ((f2cl-lib:int-add j 1) i)
                                       ((1 lda) (1 *)) a-%offset%))
                (setf (f2cl-lib:fref a-%data%
```

```
                                    ((f2cl-lib:int-add j 1) i)
                                    ((1 lda) (1 *)) a-%offset%)
                            (- (* ctemp temp)
                               (* stemp
                                  (f2cl-lib:fref a-%data% (j i)
                                                 ((1 lda) (1 *))
                                                 a-%offset%))))
                     (setf (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                    a-%offset%)
                            (+ (* stemp temp)
                               (* ctemp
                                  (f2cl-lib:fref a-%data% (j i)
                                                 ((1 lda) (1 *))
                                                 a-%offset%))))
                  label10))))
          label20)))
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame direct "B")
          (declare (ignore var-1))
          (when var-0 (setf direct var-0))
          ret-val)
        (f2cl-lib:fdo (j (f2cl-lib:int-add m (f2cl-lib:int-sub 1))
                       (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                      ((> j 1) nil)
          (tagbody
            (setf ctemp (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
            (setf stemp (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
            (cond
             ((or (/= ctemp one) (/= stemp zero))
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i n) nil)
                (tagbody
                  (setf temp
                          (f2cl-lib:fref a-%data%
                                         ((f2cl-lib:int-add j 1) i)
                                         ((1 lda) (1 *)) a-%offset%))
                  (setf (f2cl-lib:fref a-%data%
                                    ((f2cl-lib:int-add j 1) i)
                                    ((1 lda) (1 *)) a-%offset%)
                          (- (* ctemp temp)
                             (* stemp
                                (f2cl-lib:fref a-%data% (j i)
                                               ((1 lda) (1 *))
                                               a-%offset%))))
                  (setf (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                    a-%offset%)
                          (+ (* stemp temp)
                             (* ctemp
                                (f2cl-lib:fref a-%data% (j i)
                                               ((1 lda) (1 *))
```

```
                                          a-%offset%))))
            label30))))
      label40)))))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame pivot "T")
   (declare (ignore var-1))
   (when var-0 (setf pivot var-0))
   ret-val)
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame direct "F")
     (declare (ignore var-1))
     (when var-0 (setf direct var-0))
     ret-val)
   (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                 ((> j m) nil)
     (tagbody
       (setf ctemp
               (f2cl-lib:fref c-%data% ((f2cl-lib:int-sub j 1))
                              ((1 *)) c-%offset%))
       (setf stemp
               (f2cl-lib:fref s-%data% ((f2cl-lib:int-sub j 1))
                              ((1 *)) s-%offset%))
       (cond
        ((or (/= ctemp one) (/= stemp zero))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf temp
                     (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                    a-%offset%))
             (setf (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                  a-%offset%)
                     (- (* ctemp temp)
                        (* stemp
                           (f2cl-lib:fref a-%data% (1 i)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
             (setf (f2cl-lib:fref a-%data% (1 i) ((1 lda) (1 *))
                                  a-%offset%)
                     (+ (* stemp temp)
                        (* ctemp
                           (f2cl-lib:fref a-%data% (1 i)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
            label50))))
      label60)))
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame direct "B")
     (declare (ignore var-1))
```

```
      (when var-0 (setf direct var-0))
      ret-val)
    (f2cl-lib:fdo (j m (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                  ((> j 2) nil)
      (tagbody
        (setf ctemp
                (f2cl-lib:fref c-%data% ((f2cl-lib:int-sub j 1))
                               ((1 *)) c-%offset%))
        (setf stemp
                (f2cl-lib:fref s-%data% ((f2cl-lib:int-sub j 1))
                               ((1 *)) s-%offset%))
        (cond
         ((or (/= ctemp one) (/= stemp zero))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf temp
                      (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                   a-%offset%)
                      (- (* ctemp temp)
                         (* stemp
                            (f2cl-lib:fref a-%data% (1 i)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
              (setf (f2cl-lib:fref a-%data% (1 i) ((1 lda) (1 *))
                                   a-%offset%)
                      (+ (* stemp temp)
                         (* ctemp
                            (f2cl-lib:fref a-%data% (1 i)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
            label70))))
     label80)))))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame pivot "B")
   (declare (ignore var-1))
   (when var-0 (setf pivot var-0))
   ret-val)
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame direct "F")
     (declare (ignore var-1))
     (when var-0 (setf direct var-0))
     ret-val)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
                  nil)
     (tagbody
```

```
      (setf ctemp (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
      (setf stemp (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
      (cond
       ((or (/= ctemp one) (/= stemp zero))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf temp
                    (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                   a-%offset%))
            (setf (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                 a-%offset%)
                    (+
                     (* stemp
                        (f2cl-lib:fref a-%data% (m i)
                                       ((1 lda) (1 *)) a-%offset%))
                     (* ctemp temp)))
            (setf (f2cl-lib:fref a-%data% (m i) ((1 lda) (1 *))
                                 a-%offset%)
                    (-
                     (* ctemp
                        (f2cl-lib:fref a-%data% (m i)
                                       ((1 lda) (1 *)) a-%offset%))
                     (* stemp temp)))
           label90))))
    label100)))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame direct "B")
   (declare (ignore var-1))
   (when var-0 (setf direct var-0))
   ret-val)
 (f2cl-lib:fdo (j (f2cl-lib:int-add m (f2cl-lib:int-sub 1))
                (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (setf ctemp (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
     (setf stemp (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
     (cond
      ((or (/= ctemp one) (/= stemp zero))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i n) nil)
         (tagbody
           (setf temp
                   (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                  a-%offset%))
           (setf (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                a-%offset%)
                   (+
                    (* stemp
                       (f2cl-lib:fref a-%data% (m i)
```

```
                                        ((1 lda) (1 *)) a-%offset%))
                         (* ctemp temp)))
                  (setf (f2cl-lib:fref a-%data% (m i) ((1 lda) (1 *))
                                       a-%offset%)
                        (-
                         (* ctemp
                            (f2cl-lib:fref a-%data% (m i)
                                           ((1 lda) (1 *)) a-%offset%))
                         (* stemp temp)))
                label110))))
         label120)))))))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame side "R")
   (declare (ignore var-1))
   (when var-0 (setf side var-0))
   ret-val)
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame pivot "V")
     (declare (ignore var-1))
     (when var-0 (setf pivot var-0))
     ret-val)
   (cond
    ((multiple-value-bind (ret-val var-0 var-1)
         (lsame direct "F")
       (declare (ignore var-1))
       (when var-0 (setf direct var-0))
       ret-val)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                    nil)
       (tagbody
         (setf ctemp (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
         (setf stemp (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
         (cond
          ((or (/= ctemp one) (/= stemp zero))
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf temp
                       (f2cl-lib:fref a-%data%
                                      (i (f2cl-lib:int-add j 1))
                                      ((1 lda) (1 *)) a-%offset%))
               (setf (f2cl-lib:fref a-%data%
                                    (i (f2cl-lib:int-add j 1))
                                    ((1 lda) (1 *)) a-%offset%)
                       (- (* ctemp temp)
                          (* stemp
                             (f2cl-lib:fref a-%data% (i j)
                                            ((1 lda) (1 *))
```

```
                                        a-%offset%))))
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                        (+ (* stemp temp)
                           (* ctemp
                              (f2cl-lib:fref a-%data% (i j)
                                             ((1 lda) (1 *))
                                             a-%offset%))))
              label130))))
         label140)))
    ((multiple-value-bind (ret-val var-0 var-1)
         (lsame direct "B")
       (declare (ignore var-1))
       (when var-0 (setf direct var-0))
       ret-val)
     (f2cl-lib:fdo (j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                    (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                   ((> j 1) nil)
       (tagbody
         (setf ctemp (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
         (setf stemp (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
         (cond
          ((or (/= ctemp one) (/= stemp zero))
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (setf temp
                       (f2cl-lib:fref a-%data%
                                      (i (f2cl-lib:int-add j 1))
                                      ((1 lda) (1 *)) a-%offset%))
               (setf (f2cl-lib:fref a-%data%
                                    (i (f2cl-lib:int-add j 1))
                                    ((1 lda) (1 *)) a-%offset%)
                       (- (* ctemp temp)
                          (* stemp
                             (f2cl-lib:fref a-%data% (i j)
                                            ((1 lda) (1 *))
                                            a-%offset%))))
               (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%)
                       (+ (* stemp temp)
                          (* ctemp
                             (f2cl-lib:fref a-%data% (i j)
                                            ((1 lda) (1 *))
                                            a-%offset%))))
              label150))))
         label160)))))
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame pivot "T")
     (declare (ignore var-1))
```

```
  (when var-0 (setf pivot var-0))
  ret-val)
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame direct "F")
    (declare (ignore var-1))
    (when var-0 (setf direct var-0))
    ret-val)
  (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf ctemp
              (f2cl-lib:fref c-%data% ((f2cl-lib:int-sub j 1))
                             ((1 *)) c-%offset%))
      (setf stemp
              (f2cl-lib:fref s-%data% ((f2cl-lib:int-sub j 1))
                             ((1 *)) s-%offset%))
      (cond
       ((or (/= ctemp one) (/= stemp zero))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf temp
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%))
            (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%)
                    (- (* ctemp temp)
                       (* stemp
                          (f2cl-lib:fref a-%data% (i 1)
                                         ((1 lda) (1 *))
                                         a-%offset%))))
            (setf (f2cl-lib:fref a-%data% (i 1) ((1 lda) (1 *))
                                 a-%offset%)
                    (+ (* stemp temp)
                       (* ctemp
                          (f2cl-lib:fref a-%data% (i 1)
                                         ((1 lda) (1 *))
                                         a-%offset%))))
           label170))))
     label180)))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame direct "B")
    (declare (ignore var-1))
    (when var-0 (setf direct var-0))
    ret-val)
  (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                ((> j 2) nil)
    (tagbody
      (setf ctemp
```

```
                (f2cl-lib:fref c-%data% ((f2cl-lib:int-sub j 1))
                               ((1 *)) c-%offset%))
        (setf stemp
                (f2cl-lib:fref s-%data% ((f2cl-lib:int-sub j 1))
                               ((1 *)) s-%offset%))
        (cond
         ((or (/= ctemp one) (/= stemp zero))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf temp
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%)
                      (- (* ctemp temp)
                         (* stemp
                            (f2cl-lib:fref a-%data% (i 1)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
              (setf (f2cl-lib:fref a-%data% (i 1) ((1 lda) (1 *))
                                   a-%offset%)
                      (+ (* stemp temp)
                         (* ctemp
                            (f2cl-lib:fref a-%data% (i 1)
                                           ((1 lda) (1 *))
                                           a-%offset%))))
            label190))))
      label200)))))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame pivot "B")
   (declare (ignore var-1))
   (when var-0 (setf pivot var-0))
   ret-val)
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame direct "F")
     (declare (ignore var-1))
     (when var-0 (setf direct var-0))
     ret-val)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                  nil)
     (tagbody
       (setf ctemp (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
       (setf stemp (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
       (cond
        ((or (/= ctemp one) (/= stemp zero))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
```

```
            (tagbody
              (setf temp
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))
              (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                   a-%offset%)
                      (+
                       (* stemp
                          (f2cl-lib:fref a-%data% (i n)
                                         ((1 lda) (1 *)) a-%offset%))
                       (* ctemp temp)))
              (setf (f2cl-lib:fref a-%data% (i n) ((1 lda) (1 *))
                                   a-%offset%)
                      (-
                       (* ctemp
                          (f2cl-lib:fref a-%data% (i n)
                                         ((1 lda) (1 *)) a-%offset%))
                       (* stemp temp)))
             label210))))
    label220)))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame direct "B")
   (declare (ignore var-1))
   (when var-0 (setf direct var-0))
   ret-val)
 (f2cl-lib:fdo (j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (setf ctemp (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
     (setf stemp (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
     (cond
      ((or (/= ctemp one) (/= stemp zero))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf temp
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%))
           (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                   (+
                    (* stemp
                       (f2cl-lib:fref a-%data% (i n)
                                      ((1 lda) (1 *)) a-%offset%))
                    (* ctemp temp)))
           (setf (f2cl-lib:fref a-%data% (i n) ((1 lda) (1 *))
                                a-%offset%)
                   (-
                    (* ctemp
```

```
                                    (f2cl-lib:fref a-%data% (i n)
                                                   ((1 lda) (1 *)) a-%offset%))
                                (* stemp temp)))
                        label230))))
                  label240))))))))
        (go end_label)
       end_label
        (return (values side pivot direct nil nil nil nil nil nil))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slasr fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string) (string) (string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::side
                                              fortran-to-lisp::pivot
                                              fortran-to-lisp::direct nil nil
                                              nil nil nil nil)
                                            :calls 'nil)))
```

———-

90.106 slassq

90.106.1 Axiom unit tests

— **slassq.input** —

```
)set break resume
)sys rm -f slassq.output
)spool slassq.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.106.2 Axiom help page

— slassq.help —

```
==================================================================
slassq examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slassq (N, X, INCX, SCALE, SUMSQ)
           SLASSQ updates a sum of squares represented in scaled form.

Function/Subroutine Documentation
   subroutine slassq (integer N, real, dimension( * ) X, integer INCX, real
       SCALE, real SUMSQ)
       SLASSQ updates a sum of squares represented in scaled form.

Purpose:
   SLASSQ  returns the values  scl  and  smsq  such that

     ( scl**2 )*smsq = x( 1 )**2 +...+ x( n )**2 + ( scale**2 )*sumsq,

   where  x( i ) = X( 1 + ( i - 1 )*INCX ). The value of  sumsq  is
   assumed to be non-negative and  scl  returns the value

     scl = max( scale, abs( x( i ) ) ).

   scale and sumsq must be supplied in SCALE and SUMSQ and
   scl and smsq are overwritten on SCALE and SUMSQ respectively.

   The routine makes only one pass through the vector x.

Parameters:

 N
   N is INTEGER
   The number of elements to be used from the vector X.

 X
   X is REAL array, dimension (N)
   The vector for which a scaled sum of squares is computed.
```

```
  x( i )  = X( 1 + ( i - 1 )*INCX ), 1 <= i <= n.

 INCX
   INCX is INTEGER
   The increment between successive values of the vector X.
   INCX > 0.

 SCALE
   SCALE is REAL
   On entry, the value  scale  in the equation above.
   On exit, SCALE is overwritten with  scl , the scaling factor
   for the sum of squares.

 SUMSQ
   SUMSQ is REAL
   On entry, the value  sumsq  in the equation above.
   On exit, SUMSQ is overwritten with  smsq , the basic sum of
   squares from which  scl  has been factored out.
```

90.106.3 fortran code

— slassq.f —

```
*  =====================================================================
      SUBROUTINE SLASSQ( N, X, INCX, SCALE, SUMSQ )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INCX, N
      REAL               SCALE, SUMSQ
*     ..
*     .. Array Arguments ..
      REAL               X( * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
*     ..
```

```
*     .. Local Scalars ..
      INTEGER            IX
      REAL               ABSXI
*     ..
*     .. External Functions ..
      LOGICAL            SISNAN
      EXTERNAL           SISNAN
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS
*     ..
*     .. Executable Statements ..
*
      IF( N.GT.0 ) THEN
         DO 10 IX = 1, 1 + ( N-1 )*INCX, INCX
            ABSXI = ABS( X( IX ) )
            IF( ABSXI.GT.ZERO.OR.SISNAN( ABSXI ) ) THEN
               IF( SCALE.LT.ABSXI ) THEN
                  SUMSQ = 1 + SUMSQ*( SCALE / ABSXI )**2
                  SCALE = ABSXI
               ELSE
                  SUMSQ = SUMSQ + ( ABSXI / SCALE )**2
               END IF
            END IF
   10    CONTINUE
      END IF
      RETURN
*
*     End of SLASSQ
*
      END
```

———-

90.106.4 lisp code

— **slassq.lisp** —

```
(let* ((zero 0.0))
  (declare (type (single-float 0.0 0.0) zero)
           (ignorable zero))
  (defun slassq (n x incx scale sumsq)
    (declare (type (single-float) sumsq scale)
             (type (array single-float (*)) x)
             (type (f2cl-lib:integer4) incx n))
    (f2cl-lib:with-multi-array-data
        ((x single-float x-%data% x-%offset%))
      (prog ((absxi 0.0) (ix 0))
```

```
        (declare (type (single-float) absxi)
                 (type (f2cl-lib:integer4) ix))
        (cond
         ((> n 0)
          (f2cl-lib:fdo (ix 1 (f2cl-lib:int-add ix incx))
                        ((> ix
                            (f2cl-lib:int-add 1
                                              (f2cl-lib:int-mul
                                               (f2cl-lib:int-add n
                                                                 (f2cl-lib:int-sub
                                                                  1))
                                               incx)))
                         nil)
            (tagbody
              (setf absxi
                      (abs (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)))
              (cond
               ((or (> absxi zero) (sisnan absxi))
                (cond
                 ((< scale absxi)
                  (setf sumsq (+ 1 (* sumsq (expt (/ scale absxi) 2))))
                  (setf scale absxi))
                 (t (setf sumsq (+ sumsq (expt (/ absxi scale) 2)))))))
             label10))))
        (go end_label)
       end_label
        (return (values nil nil nil scale sumsq))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slassq
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4)
                                              (single-float) (single-float))
                                            :return-values
                                            '(nil nil nil
                                              fortran-to-lisp::scale
                                              fortran-to-lisp::sumsq)
                                            :calls '(fortran-to-lisp::sisnan))))
```

———-

90.107 slasv2

90.107.1 Axiom unit tests

— slasv2.input —

```
)set break resume
)sys rm -f slasv2.output
)spool slasv2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.107.2 Axiom help page

— slasv2.help —

```
======================================================================
slasv2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine slasv2 (F, G, H, SSMIN, SSMAX, SNR, CSR, SNL, CSL)
           SLASV2 computes the singular value decomposition of a 2-by-2
           triangular matrix.

Function/Subroutine Documentation
   subroutine slasv2 (real F, real G, real H, real SSMIN, real SSMAX, real
       SNR, real CSR, real SNL, real CSL)
       SLASV2 computes the singular value decomposition of a 2-by-2 triangular
       matrix.

Purpose:
   SLASV2 computes the singular value decomposition of a 2-by-2
   triangular matrix
               [  F   G  ]
```

```
              [  0   H  ].
   On return, abs(SSMAX) is the larger singular value, abs(SSMIN) is the
   smaller singular value, and (CSL,SNL) and (CSR,SNR) are the left and
   right singular vectors for abs(SSMAX), giving the decomposition

              [ CSL  SNL ] [  F   G  ] [ CSR -SNR ]  =  [ SSMAX   0   ]
              [-SNL  CSL ] [  0   H  ] [ SNR  CSR ]     [  0    SSMIN ].

Parameters:

 F
   F is REAL
   The (1,1) element of the 2-by-2 matrix.

 G
   G is REAL
   The (1,2) element of the 2-by-2 matrix.

 H
   H is REAL
   The (2,2) element of the 2-by-2 matrix.

 SSMIN
   SSMIN is REAL
   abs(SSMIN) is the smaller singular value.

 SSMAX
   SSMAX is REAL
   abs(SSMAX) is the larger singular value.

 SNL
   SNL is REAL

 CSL
   CSL is REAL
   The vector (CSL, SNL) is a unit left singular vector for the
   singular value abs(SSMAX).

 SNR
   SNR is REAL

 CSR
   CSR is REAL
   The vector (CSR, SNR) is a unit right singular vector for the
   singular value abs(SSMAX).

Further Details:

   Any input parameter may be aliased with any output parameter.
```

```
Barring over/underflow and assuming a guard digit in subtraction, all
output quantities are correct to within a few units in the last
place (ulps).

In IEEE arithmetic, the code works correctly if one matrix element is
infinite.

Overflow will not occur unless the largest singular value itself
overflows or is within a few ulps of overflow. (On machines with
partial overflow, like the Cray, overflow may occur if the largest
singular value is within a factor of 2 of overflow.)

Underflow is harmless if underflow is gradual. Otherwise, results
may correspond to a matrix modified by perturbations of size near
the underflow threshold.
```

———-

90.107.3 fortran code

— **slasv2.f** —

```
*  =====================================================================
      SUBROUTINE SLASV2( F, G, H, SSMIN, SSMAX, SNR, CSR, SNL, CSL )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      REAL               CSL, CSR, F, G, H, SNL, SNR, SSMAX, SSMIN
*     ..
*
* ====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E0 )
      REAL               HALF
      PARAMETER          ( HALF = 0.5E0 )
      REAL               ONE
      PARAMETER          ( ONE = 1.0E0 )
      REAL               TWO
      PARAMETER          ( TWO = 2.0E0 )
      REAL               FOUR
      PARAMETER          ( FOUR = 4.0E0 )
```

```
*     ..
*     .. Local Scalars ..
      LOGICAL            GASMAL, SWAP
      INTEGER            PMAX
      REAL               A, CLT, CRT, D, FA, FT, GA, GT, HA, HT, L, M,
     $                   MM, R, S, SLT, SRT, T, TEMP, TSIGN, TT
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SIGN, SQRT
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      EXTERNAL           SLAMCH
*     ..
*     .. Executable Statements ..
*
      FT = F
      FA = ABS( FT )
      HT = H
      HA = ABS( H )
*
*     PMAX points to the maximum absolute element of matrix
*       PMAX = 1 if F largest in absolute values
*       PMAX = 2 if G largest in absolute values
*       PMAX = 3 if H largest in absolute values
*
      PMAX = 1
      SWAP = ( HA.GT.FA )
      IF( SWAP ) THEN
         PMAX = 3
         TEMP = FT
         FT = HT
         HT = TEMP
         TEMP = FA
         FA = HA
         HA = TEMP
*
*        Now FA .ge. HA
*
      END IF
      GT = G
      GA = ABS( GT )
      IF( GA.EQ.ZERO ) THEN
*
*        Diagonal matrix
*
         SSMIN = HA
         SSMAX = FA
         CLT = ONE
         CRT = ONE
```

```
         SLT = ZERO
         SRT = ZERO
      ELSE
         GASMAL = .TRUE.
         IF( GA.GT.FA ) THEN
            PMAX = 2
            IF( ( FA / GA ).LT.SLAMCH( 'EPS' ) ) THEN
*
*              Case of very large GA
*
               GASMAL = .FALSE.
               SSMAX = GA
               IF( HA.GT.ONE ) THEN
                  SSMIN = FA / ( GA / HA )
               ELSE
                  SSMIN = ( FA / GA )*HA
               END IF
               CLT = ONE
               SLT = HT / GT
               SRT = ONE
               CRT = FT / GT
            END IF
         END IF
         IF( GASMAL ) THEN
*
*           Normal case
*
            D = FA - HA
            IF( D.EQ.FA ) THEN
*
*              Copes with infinite F or H
*
               L = ONE
            ELSE
               L = D / FA
            END IF
*
*           Note that 0 .le. L .le. 1
*
            M = GT / FT
*
*           Note that abs(M) .le. 1/macheps
*
            T = TWO - L
*
*           Note that T .ge. 1
*
            MM = M*M
            TT = T*T
            S = SQRT( TT+MM )
```

```
*
*          Note that 1 .le. S .le. 1 + 1/macheps
*
           IF( L.EQ.ZERO ) THEN
              R = ABS( M )
           ELSE
              R = SQRT( L*L+MM )
           END IF
*
*          Note that 0 .le. R .le. 1 + 1/macheps
*
           A = HALF*( S+R )
*
*          Note that 1 .le. A .le. 1 + abs(M)
*
           SSMIN = HA / A
           SSMAX = FA*A
           IF( MM.EQ.ZERO ) THEN
*
*             Note that M is very tiny
*
              IF( L.EQ.ZERO ) THEN
                 T = SIGN( TWO, FT )*SIGN( ONE, GT )
              ELSE
                 T = GT / SIGN( D, FT ) + M / T
              END IF
           ELSE
              T = ( M / ( S+T )+M / ( R+L ) )*( ONE+A )
           END IF
           L = SQRT( T*T+FOUR )
           CRT = TWO / L
           SRT = T / L
           CLT = ( CRT+SRT*M ) / A
           SLT = ( HT / FT )*SRT / A
        END IF
     END IF
     IF( SWAP ) THEN
        CSL = SRT
        SNL = CRT
        CSR = SLT
        SNR = CLT
     ELSE
        CSL = CLT
        SNL = SLT
        CSR = CRT
        SNR = SRT
     END IF
*
*    Correct signs of SSMAX and SSMIN
*
```

```
      IF( PMAX.EQ.1 )
     $   TSIGN = SIGN( ONE, CSR )*SIGN( ONE, CSL )*SIGN( ONE, F )
      IF( PMAX.EQ.2 )
     $   TSIGN = SIGN( ONE, SNR )*SIGN( ONE, CSL )*SIGN( ONE, G )
      IF( PMAX.EQ.3 )
     $   TSIGN = SIGN( ONE, SNR )*SIGN( ONE, SNL )*SIGN( ONE, H )
      SSMAX = SIGN( SSMAX, TSIGN )
      SSMIN = SIGN( SSMIN, TSIGN*SIGN( ONE, F )*SIGN( ONE, H ) )
      RETURN
*
*     End of SLASV2
*
      END
```

———-

90.107.4 lisp code

— slasv2.lisp —

```
(let* ((zero 0.0) (half 0.5) (one 1.0) (two 2.0) (four 4.0))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 0.5 0.5) half)
           (type (single-float 1.0 1.0) one)
           (type (single-float 2.0 2.0) two)
           (type (single-float 4.0 4.0) four)
           (ignorable zero half one two four))
  (defun slasv2 (f g h ssmin ssmax snr csr snl csl)
    (declare (type (single-float) csl snl csr snr ssmax ssmin h g f))
    (prog ((a 0.0) (clt 0.0) (crt 0.0) (d 0.0) (fa 0.0) (ft 0.0) (ga 0.0)
           (gt 0.0) (ha 0.0) (ht 0.0) (l 0.0) (m 0.0) (mm 0.0) (r 0.0) (s 0.0)
           (slt 0.0) (srt 0.0) (t$ 0.0) (temp 0.0) (tsign 0.0) (tt 0.0)
           (pmax 0) (gasmal nil) (swap nil))
      (declare
       (type (single-float) a clt crt d fa ft ga gt ha ht l m mm r s slt srt t$
        temp tsign tt)
       (type (f2cl-lib:integer4) pmax)
       (type f2cl-lib:logical gasmal swap))
      (setf ft f)
      (setf fa (abs ft))
      (setf ht h)
      (setf ha (abs h))
      (setf pmax 1)
      (setf swap (> ha fa))
      (cond
       (swap (setf pmax 3) (setf temp ft) (setf ft ht) (setf ht temp)
        (setf temp fa) (setf fa ha) (setf ha temp)))
      (setf gt g)
```

```
(setf ga (abs gt))
(cond
 ((= ga zero) (setf ssmin ha) (setf ssmax fa) (setf clt one)
  (setf crt one) (setf slt zero) (setf srt zero))
 (t (setf gasmal f2cl-lib:%true%)
  (cond
   ((> ga fa) (setf pmax 2)
    (cond
     ((< (f2cl-lib:f2cl/ fa ga) (slamch "EPS"))
      (setf gasmal f2cl-lib:%false%) (setf ssmax ga)
      (cond ((> ha one) (setf ssmin (/ fa (/ ga ha))))
            (t (setf ssmin (* (/ fa ga) ha))))
      (setf clt one) (setf slt (/ ht gt)) (setf srt one)
      (setf crt (/ ft gt))))))
  (cond
   (gasmal (setf d (- fa ha))
    (cond ((= d fa) (setf l one)) (t (setf l (/ d fa))))
    (setf m (/ gt ft)) (setf t$ (- two l)) (setf mm (* m m))
    (setf tt (* t$ t$)) (setf s (f2cl-lib:fsqrt (+ tt mm)))
    (cond ((= l zero) (setf r (abs m)))
          (t (setf r (f2cl-lib:fsqrt (+ (* l l) mm)))))
    (setf a (* half (+ s r))) (setf ssmin (/ ha a)) (setf ssmax (* fa a))
    (cond
     ((= mm zero)
      (cond
       ((= l zero)
        (setf t$ (* (f2cl-lib:sign two ft) (f2cl-lib:sign one gt))))
       (t (setf t$ (+ (/ gt (f2cl-lib:sign d ft)) (/ m t$))))))
     (t (setf t$ (* (+ (/ m (+ s t$)) (/ m (+ r l))) (+ one a)))))
    (setf l (f2cl-lib:fsqrt (+ (* t$ t$) four))) (setf crt (/ two l))
    (setf srt (/ t$ l)) (setf clt (/ (+ crt (* srt m)) a))
    (setf slt (/ (* (/ ht ft) srt) a))))))
(cond (swap (setf csl srt) (setf snl crt) (setf csr slt) (setf snr clt))
      (t (setf csl clt) (setf snl slt) (setf csr crt) (setf snr srt)))
(if (= pmax 1)
    (setf tsign
            (* (f2cl-lib:sign one csr) (f2cl-lib:sign one csl)
               (f2cl-lib:sign one f))))
(if (= pmax 2)
    (setf tsign
            (* (f2cl-lib:sign one snr) (f2cl-lib:sign one csl)
               (f2cl-lib:sign one g))))
(if (= pmax 3)
    (setf tsign
            (* (f2cl-lib:sign one snr) (f2cl-lib:sign one snl)
               (f2cl-lib:sign one h))))
(setf ssmax (f2cl-lib:sign ssmax tsign))
(setf ssmin
        (f2cl-lib:sign ssmin
                       (* tsign (f2cl-lib:sign one f)
```

```
                                (f2cl-lib:sign one h))))
      (go end_label)
     end_label
      (return (values nil nil nil ssmin ssmax snr csr snl csl)))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::slasv2
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((single-float) (single-float)
                                              (single-float) (single-float)
                                              (single-float) (single-float)
                                              (single-float) (single-float)
                                              (single-float))
                                            :return-values
                                            '(nil nil nil
                                              fortran-to-lisp::ssmin
                                              fortran-to-lisp::ssmax
                                              fortran-to-lisp::snr
                                              fortran-to-lisp::csr
                                              fortran-to-lisp::snl
                                              fortran-to-lisp::csl)
                                            :calls 'nil)))
```

———-

90.108 tstiee

90.108.1 Axiom unit tests

— **tstiee.input** —

```
)set break resume
)sys rm -f tstiee.output
)spool tstiee.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.108.2 Axiom help page

— tstiee.help —

```
==================================================================
tstiee examples
==================================================================

==================================================================
Man Page Details
==================================================================

SYNOPSIS
   Functions/Subroutines
       program tstiee
           TSTIEE
       integer function ilaenv (ISPEC, NAME, OPTS, N1, N2, N3, N4)
       integer function ieeeck (ISPEC, ZERO, ONE)

Function/Subroutine Documentation
   integer function ieeeck (integer ISPEC, real ZERO, real ONE)
   integer function ilaenv (integer ISPEC, character*( * ) NAME, character*( *
       ) OPTS, integer N1, integer N2, integer N3, integer N4)
   program tstiee ()
       TSTIEE
```

———-

90.108.3 fortran code

— tstiee.f —

———-

90.108.4 lisp code

— tstiee.lisp —

———-

90.109 xerbla_array

90.109.1 Axiom unit tests

— xerblaarray.input —

```
)set break resume
)sys rm -f xerblaarray.output
)spool xerblaarray.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.109.2 Axiom help page

— xerbla_array.help —

```
====================================================================
xerbla_array examples
====================================================================

====================================================================
Man Page Details
====================================================================

subroutine xerbla_array(character(1), dimension(srname_len) SRNAME_ARRAY,
                        integer SRNAME_LEN,
                        integer INFO)
Purpose:
   assists other languages in calling XERBLA, the LAPACK
   and BLAS error handler. Rather than taking a Fortran string argument
   as the function's name, XERBLA_ARRAY takes an array of single
   characters along with the array's length. XERBLA_ARRAY then copies
   up to 32 charaacters of that array into a Fortran string and passes
   that to XERBLA. If called with a non-positive SRNAME_LEN,
   XERBLA_ARRAY will call XERBLA with a string of all blank characters.

   Say some macro or other device makes XERBLA_ARRAY available to C99
   by a name lapack_xerbla and with a common Fortran calling convention.
   Then a C99 program could invoke XERBLA via:
      {
```

```
         int flen = strlen(__func__);
         lapack_xerbla(__func__,&flen,&info);
      }

   Providing XERBLA_ARRAY is not necessary for intercepting LAPACK
   errors. XERBLA_ARRAY calls XERBLA.

Paramenters:

 SRNAME_ARRAY
   CHARACTER(1) array, dimension (SRNAME_LEN)
   The name of the routine which called XERBLA_ARRAY

 SRNAME_LEN
   INTEGER
   The length of the name in SRNAME_ARRY

 INFO
   INTEGER
   The position of the invalid parameter in the parameter list of the
   calling routine.
```

———-

90.109.3 fortran code

— **xerbla_array.f** —

```
*  =====================================================================
      SUBROUTINE XERBLA_ARRAY( SRNAME_ARRAY, SRNAME_LEN, INFO)
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER SRNAME_LEN, INFO
*     ..
*     .. Array Arguments ..
      CHARACTER(1) SRNAME_ARRAY(SRNAME_LEN)
*     ..
*
* =====================================================================
*
*     ..
*     .. Local Scalars ..
```

```
      INTEGER I
*     ..
*     .. Local Arrays ..
      CHARACTER*32 SRNAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC MIN, LEN
*     ..
*     .. External Functions ..
      EXTERNAL XERBLA
*     ..
*     .. Executable Statements ..
      SRNAME = ''
      DO I = 1, MIN( SRNAME_LEN, LEN( SRNAME ) )
         SRNAME( I:I ) = SRNAME_ARRAY( I )
      END DO

      CALL XERBLA( SRNAME, INFO )

      RETURN
      END
```

———-

90.109.4 lisp code

— xerbla_array.lisp —

———-

90.110 cbbcsd - CS Decomposition of Unitary Matrix

See Sutton [Sutt13] "Computing the Complete CS Decomposition"

90.110.1 Axiom unit tests

— cbbcsd.input —

```
)set break resume
)sys rm -f cbbcsd.output
)spool cbbcsd.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.110.2 Axiom help page

— cbbcsd.help —

```
=====================================================================
cbbcsd examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

NAME
       cbbcsd.f -

SYNOPSIS
   Functions/Subroutines
       subroutine cbbcsd (JOBU1, JOBU2, JOBV1T, JOBV2T, TRANS, M, P, Q, THETA,
           PHI, U1, LDU1, U2, LDU2, V1T, LDV1T, V2T, LDV2T, B11D, B11E, B12D,
           B12E, B21D, B21E, B22D, B22E, RWORK, LRWORK, INFO)
           CBBCSD

Function/Subroutine Documentation
   subroutine cbbcsd (character JOBU1, character JOBU2, character JOBV1T,
       character JOBV2T, character TRANS, integer M, integer P, integer Q,
       real, dimension( * ) THETA, real, dimension( * ) PHI, complex,
       dimension( ldu1, * ) U1, integer LDU1, complex, dimension( ldu2, * )
       U2, integer LDU2, complex, dimension( ldv1t, * ) V1T, integer LDV1T,
       complex, dimension( ldv2t, * ) V2T, integer LDV2T, real, dimension( * )
       B11D, real, dimension( * ) B11E, real, dimension( * ) B12D, real,
       dimension( * ) B12E, real, dimension( * ) B21D, real, dimension( * )
       B21E, real, dimension( * ) B22D, real, dimension( * ) B22E, real,
       dimension( * ) RWORK, integer LRWORK, integer INFO)
       CBBCSD

Purpose:
   CBBCSD computes the CS decomposition of a unitary matrix in
   bidiagonal-block form,
```

```
          [ B11 | B12 0  0 ]
          [  0  |  0 -I  0 ]
      X = [----------------]
          [ B21 | B22 0  0 ]
          [  0  |  0  0  I ]

                                [  C | -S  0  0 ]
                      [ U1 |    ] [  0 |  0 -I  0 ] [ V1 |    ]**H
                    = [---------] [---------------] [---------]   .
                      [    | U2 ] [  S |  C  0  0 ] [    | V2 ]
                                [  0 |  0  0  I ]
```

X is M-by-M, its top-left block is P-by-Q, and Q must be no larger than P, M-P, or M-Q. (If Q is not the smallest index, then X must be transposed and/or permuted. This can be done in constant time using the TRANS and SIGNS options. See CUNCSD for details.)

The bidiagonal matrices B11, B12, B21, and B22 are represented implicitly by angles THETA(1:Q) and PHI(1:Q-1).

The unitary matrices U1, U2, V1T, and V2T are input/output. The input matrices are pre- or post-multiplied by the appropriate singular vector matrices.

Parameters:

JOBU1
  JOBU1 is CHARACTER
  = 'Y': U1 is updated;
  otherwise: U1 is not updated.

JOBU2
  JOBU2 is CHARACTER
  = 'Y': U2 is updated;
  otherwise: U2 is not updated.

JOBV1T
  JOBV1T is CHARACTER
  = 'Y': V1T is updated;
  otherwise: V1T is not updated.

JOBV2T
  JOBV2T is CHARACTER
  = 'Y': V2T is updated;
  otherwise: V2T is not updated.

TRANS
  TRANS is CHARACTER
  = 'T': X, U1, U2, V1T, and V2T are stored in row-major order;

```
  otherwise:  X, U1, U2, V1T, and V2T are stored in column-major order.

M
  M is INTEGER
  The number of rows and columns in X, the unitary matrix in
  bidiagonal-block form.

P
  P is INTEGER
  The number of rows in the top-left block of X. 0 <= P <= M.

Q
  Q is INTEGER
  The number of columns in the top-left block of X.
  0 <= Q <= MIN(P,M-P,M-Q).

THETA
  THETA is REAL array, dimension (Q)
  On entry, the angles THETA(1),...,THETA(Q) that, along with
  PHI(1), ...,PHI(Q-1), define the matrix in bidiagonal-block
  form. On exit, the angles whose cosines and sines define the
  diagonal blocks in the CS decomposition.

PHI
  PHI is REAL array, dimension (Q-1)
  The angles PHI(1),...,PHI(Q-1) that, along with THETA(1),...,
  THETA(Q), define the matrix in bidiagonal-block form.

U1
  U1 is COMPLEX array, dimension (LDU1,P)
  On entry, an LDU1-by-P matrix. On exit, U1 is postmultiplied
  by the left singular vector matrix common to [ B11 ; 0 ] and
  [ B12 0 0 ; 0 -I 0 0 ].

LDU1
  LDU1 is INTEGER
  The leading dimension of the array U1.

U2
  U2 is COMPLEX array, dimension (LDU2,M-P)
  On entry, an LDU2-by-(M-P) matrix. On exit, U2 is
  postmultiplied by the left singular vector matrix common to
  [ B21 ; 0 ] and [ B22 0 0 ; 0 0 I ].

LDU2
  LDU2 is INTEGER
  The leading dimension of the array U2.

V1T
  V1T is COMPLEX array, dimension (LDV1T,Q)
```

```
  On entry, a LDV1T-by-Q matrix. On exit, V1T is premultiplied
  by the conjugate transpose of the right singular vector
  matrix common to [ B11 ; 0 ] and [ B21 ; 0 ].

LDV1T
  LDV1T is INTEGER
  The leading dimension of the array V1T.

V2T
  V2T is COMPLEX array, dimenison (LDV2T,M-Q)
  On entry, a LDV2T-by-(M-Q) matrix. On exit, V2T is
  premultiplied by the conjugate transpose of the right
  singular vector matrix common to [ B12 0 0 ; 0 -I 0 ] and
  [ B22 0 0 ; 0 0 I ].

LDV2T
  LDV2T is INTEGER
  The leading dimension of the array V2T.

B11D
  B11D is REAL array, dimension (Q)
  When CBBCSD converges, B11D contains the cosines of THETA(1),
  ..., THETA(Q). If CBBCSD fails to converge, then B11D
  contains the diagonal of the partially reduced top-left block.

B11E
  B11E is REAL array, dimension (Q-1)
  When CBBCSD converges, B11E contains zeros. If CBBCSD fails
  to converge, then B11E contains the superdiagonal of the
  partially reduced top-left block.

B12D
  B12D is REAL array, dimension (Q)
  When CBBCSD converges, B12D contains the negative sines of
  THETA(1), ..., THETA(Q). If CBBCSD fails to converge, then
  B12D contains the diagonal of the partially reduced top-right block.

B12E
  B12E is REAL array, dimension (Q-1)
  When CBBCSD converges, B12E contains zeros. If CBBCSD fails
  to converge, then B12E contains the subdiagonal of the
  partially reduced top-right block.

B21D
  B21D is REAL array, dimension (Q)
  When CBBCSD converges, B21D contains the negative sines of
  THETA(1), ..., THETA(Q). If CBBCSD fails to converge, then
  B21D contains the diagonal of the partially reduced bottom-left block.

B21E
```

```
  B21E is REAL array, dimension (Q-1)
  When CBBCSD converges, B21E contains zeros. If CBBCSD fails
  to converge, then B21E contains the subdiagonal of the
  partially reduced bottom-left block.

B22D
  B22D is REAL array, dimension (Q)
  When CBBCSD converges, B22D contains the negative sines of
  THETA(1), ..., THETA(Q). If CBBCSD fails to converge, then
  B22D contains the diagonal of the partially reduced bottom-right block.

B22E
  B22E is REAL array, dimension (Q-1)
  When CBBCSD converges, B22E contains zeros. If CBBCSD fails
  to converge, then B22E contains the subdiagonal of the
  partially reduced bottom-right block.

RWORK
  RWORK is REAL array, dimension (MAX(1,LWORK))
  On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LRWORK
  LRWORK is INTEGER
  The dimension of the array RWORK. LRWORK >= MAX(1,8*Q).

  If LRWORK = -1, then a workspace query is assumed; the
  routine only calculates the optimal size of the RWORK array,
  returns this value as the first entry of the work array, and
  no error message related to LRWORK is issued by XERBLA.

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
  > 0:  if CBBCSD did not converge, INFO specifies the number
  of nonzero entries in PHI, and B11D, B11E, etc.,
  contain the partially reduced matrix.

Internal Parameters:

TOLMUL  REAL, default = MAX(10,MIN(100,EPS**(-1/8)))
  TOLMUL controls the convergence criterion of the QR loop.
  Angles THETA(i), PHI(i) are rounded to 0 or PI/2 when they
  are within TOLMUL*EPS of either bound.

      References:
          [1] Brian D. Sutton. Computing the complete CS decomposition.
          Numer. Algorithms, 50(1):33-65, 2009.
```

———-

90.110.3 fortran code

— cbbcsd.f —

```
*  =====================================================================
      SUBROUTINE CBBCSD( JOBU1, JOBU2, JOBV1T, JOBV2T, TRANS, M, P, Q,
     $                   THETA, PHI, U1, LDU1, U2, LDU2, V1T, LDV1T,
     $                   V2T, LDV2T, B11D, B11E, B12D, B12E, B21D, B21E,
     $                   B22D, B22E, RWORK, LRWORK, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER          JOBU1, JOBU2, JOBV1T, JOBV2T, TRANS
      INTEGER            INFO, LDU1, LDU2, LDV1T, LDV2T, LRWORK, M, P, Q
*     ..
*     .. Array Arguments ..
      REAL               B11D( * ), B11E( * ), B12D( * ), B12E( * ),
     $                   B21D( * ), B21E( * ), B22D( * ), B22E( * ),
     $                   PHI( * ), THETA( * ), RWORK( * )
      COMPLEX            U1( LDU1, * ), U2( LDU2, * ), V1T( LDV1T, * ),
     $                   V2T( LDV2T, * )
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
      INTEGER            MAXITR
      PARAMETER          ( MAXITR = 6 )
      REAL               HUNDRED, MEIGHTH, ONE, PIOVER2, TEN, ZERO
      PARAMETER          ( HUNDRED = 100.0E0, MEIGHTH = -0.125E0,
     $                     ONE = 1.0E0, PIOVER2 = 1.57079632679489662E0,
     $                     TEN = 10.0E0, ZERO = 0.0E0 )
      COMPLEX            NEGONECOMPLEX
      PARAMETER          ( NEGONECOMPLEX = (-1.0E0,0.0E0) )
*     ..
*     .. Local Scalars ..
      LOGICAL            COLMAJOR, LQUERY, RESTART11, RESTART12,
     $                   RESTART21, RESTART22, WANTU1, WANTU2, WANTV1T,
     $                   WANTV2T
      INTEGER            I, IMIN, IMAX, ITER, IU1CS, IU1SN, IU2CS,
     $                   IU2SN, IV1TCS, IV1TSN, IV2TCS, IV2TSN, J,
     $                   LRWORKMIN, LRWORKOPT, MAXIT, MINI
```

```
      REAL               B11BULGE, B12BULGE, B21BULGE, B22BULGE, DUMMY,
     $                   EPS, MU, NU, R, SIGMA11, SIGMA21,
     $                   TEMP, THETAMAX, THETAMIN, THRESH, TOL, TOLMUL,
     $                   UNFL, X1, X2, Y1, Y2
*
*     .. External Subroutines ..
      EXTERNAL           CLASR, CSCAL, CSWAP, SLARTGP, SLARTGS, SLAS2,
     $                   XERBLA
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      LOGICAL            LSAME
      EXTERNAL           LSAME, SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, ATAN2, COS, MAX, MIN, SIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test input arguments
*
      INFO = 0
      LQUERY = LRWORK .EQ. -1
      WANTU1 = LSAME( JOBU1, 'Y' )
      WANTU2 = LSAME( JOBU2, 'Y' )
      WANTV1T = LSAME( JOBV1T, 'Y' )
      WANTV2T = LSAME( JOBV2T, 'Y' )
      COLMAJOR = .NOT. LSAME( TRANS, 'T' )
*
      IF( M .LT. 0 ) THEN
         INFO = -6
      ELSE IF( P .LT. 0 .OR. P .GT. M ) THEN
         INFO = -7
      ELSE IF( Q .LT. 0 .OR. Q .GT. M ) THEN
         INFO = -8
      ELSE IF( Q .GT. P .OR. Q .GT. M-P .OR. Q .GT. M-Q ) THEN
         INFO = -8
      ELSE IF( WANTU1 .AND. LDU1 .LT. P ) THEN
         INFO = -12
      ELSE IF( WANTU2 .AND. LDU2 .LT. M-P ) THEN
         INFO = -14
      ELSE IF( WANTV1T .AND. LDV1T .LT. Q ) THEN
         INFO = -16
      ELSE IF( WANTV2T .AND. LDV2T .LT. M-Q ) THEN
         INFO = -18
      END IF
*
*     Quick return if Q = 0
*
      IF( INFO .EQ. 0 .AND. Q .EQ. 0 ) THEN
```

```
         LRWORKMIN = 1
         RWORK(1) = LRWORKMIN
         RETURN
      END IF
*
*     Compute workspace
*
      IF( INFO .EQ. 0 ) THEN
         IU1CS = 1
         IU1SN = IU1CS + Q
         IU2CS = IU1SN + Q
         IU2SN = IU2CS + Q
         IV1TCS = IU2SN + Q
         IV1TSN = IV1TCS + Q
         IV2TCS = IV1TSN + Q
         IV2TSN = IV2TCS + Q
         LRWORKOPT = IV2TSN + Q - 1
         LRWORKMIN = LRWORKOPT
         RWORK(1) = LRWORKOPT
         IF( LRWORK .LT. LRWORKMIN .AND. .NOT. LQUERY ) THEN
            INFO = -28
         END IF
      END IF
*
      IF( INFO .NE. 0 ) THEN
         CALL XERBLA( 'CBBCSD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = SLAMCH( 'Epsilon' )
      UNFL = SLAMCH( 'Safe minimum' )
      TOLMUL = MAX( TEN, MIN( HUNDRED, EPS**MEIGHTH ) )
      TOL = TOLMUL*EPS
      THRESH = MAX( TOL, MAXITR*Q*Q*UNFL )
*
*     Test for negligible sines or cosines
*
      DO I = 1, Q
         IF( THETA(I) .LT. THRESH ) THEN
            THETA(I) = ZERO
         ELSE IF( THETA(I) .GT. PIOVER2-THRESH ) THEN
            THETA(I) = PIOVER2
         END IF
      END DO
      DO I = 1, Q-1
         IF( PHI(I) .LT. THRESH ) THEN
```

```
            PHI(I) = ZERO
         ELSE IF( PHI(I) .GT. PIOVER2-THRESH ) THEN
            PHI(I) = PIOVER2
         END IF
      END DO
*
*     Initial deflation
*
      IMAX = Q
      DO WHILE( IMAX .GT. 1 )
         IF( PHI(IMAX-1) .NE. ZERO ) THEN
            EXIT
         END IF
         IMAX = IMAX - 1
      END DO
      IMIN = IMAX - 1
      IF  ( IMIN .GT. 1 ) THEN
         DO WHILE( PHI(IMIN-1) .NE. ZERO )
            IMIN = IMIN - 1
            IF  ( IMIN .LE. 1 ) EXIT
         END DO
      END IF
*
*     Initialize iteration counter
*
      MAXIT = MAXITR*Q*Q
      ITER = 0
*
*     Begin main iteration loop
*
      DO WHILE( IMAX .GT. 1 )
*
*        Compute the matrix entries
*
         B11D(IMIN) = COS( THETA(IMIN) )
         B21D(IMIN) = -SIN( THETA(IMIN) )
         DO I = IMIN, IMAX - 1
            B11E(I) = -SIN( THETA(I) ) * SIN( PHI(I) )
            B11D(I+1) = COS( THETA(I+1) ) * COS( PHI(I) )
            B12D(I) = SIN( THETA(I) ) * COS( PHI(I) )
            B12E(I) = COS( THETA(I+1) ) * SIN( PHI(I) )
            B21E(I) = -COS( THETA(I) ) * SIN( PHI(I) )
            B21D(I+1) = -SIN( THETA(I+1) ) * COS( PHI(I) )
            B22D(I) = COS( THETA(I) ) * COS( PHI(I) )
            B22E(I) = -SIN( THETA(I+1) ) * SIN( PHI(I) )
         END DO
         B12D(IMAX) = SIN( THETA(IMAX) )
         B22D(IMAX) = COS( THETA(IMAX) )
*
*        Abort if not converging; otherwise, increment ITER
```

```
*
         IF( ITER .GT. MAXIT ) THEN
            INFO = 0
            DO I = 1, Q
               IF( PHI(I) .NE. ZERO )
     $            INFO = INFO + 1
            END DO
            RETURN
         END IF
*
         ITER = ITER + IMAX - IMIN
*
*        Compute shifts
*
         THETAMAX = THETA(IMIN)
         THETAMIN = THETA(IMIN)
         DO I = IMIN+1, IMAX
            IF( THETA(I) > THETAMAX )
     $         THETAMAX = THETA(I)
            IF( THETA(I) < THETAMIN )
     $         THETAMIN = THETA(I)
         END DO
*
         IF( THETAMAX .GT. PIOVER2 - THRESH ) THEN
*
*           Zero on diagonals of B11 and B22; induce deflation with a
*           zero shift
*
            MU = ZERO
            NU = ONE
*
         ELSE IF( THETAMIN .LT. THRESH ) THEN
*
*           Zero on diagonals of B12 and B22; induce deflation with a
*           zero shift
*
            MU = ONE
            NU = ZERO
*
         ELSE
*
*           Compute shifts for B11 and B21 and use the lesser
*
            CALL SLAS2( B11D(IMAX-1), B11E(IMAX-1), B11D(IMAX), SIGMA11,
     $                  DUMMY )
            CALL SLAS2( B21D(IMAX-1), B21E(IMAX-1), B21D(IMAX), SIGMA21,
     $                  DUMMY )
*
            IF( SIGMA11 .LE. SIGMA21 ) THEN
               MU = SIGMA11
```

```
               NU = SQRT( ONE - MU**2 )
               IF( MU .LT. THRESH ) THEN
                  MU = ZERO
                  NU = ONE
               END IF
            ELSE
               NU = SIGMA21
               MU = SQRT( 1.0 - NU**2 )
               IF( NU .LT. THRESH ) THEN
                  MU = ONE
                  NU = ZERO
               END IF
            END IF
         END IF
*
*        Rotate to produce bulges in B11 and B21
*
         IF( MU .LE. NU ) THEN
            CALL SLARTGS( B11D(IMIN), B11E(IMIN), MU,
     $                    RWORK(IV1TCS+IMIN-1), RWORK(IV1TSN+IMIN-1) )
         ELSE
            CALL SLARTGS( B21D(IMIN), B21E(IMIN), NU,
     $                    RWORK(IV1TCS+IMIN-1), RWORK(IV1TSN+IMIN-1) )
         END IF
*
         TEMP = RWORK(IV1TCS+IMIN-1)*B11D(IMIN) +
     $          RWORK(IV1TSN+IMIN-1)*B11E(IMIN)
         B11E(IMIN) = RWORK(IV1TCS+IMIN-1)*B11E(IMIN) -
     $                RWORK(IV1TSN+IMIN-1)*B11D(IMIN)
         B11D(IMIN) = TEMP
         B11BULGE = RWORK(IV1TSN+IMIN-1)*B11D(IMIN+1)
         B11D(IMIN+1) = RWORK(IV1TCS+IMIN-1)*B11D(IMIN+1)
         TEMP = RWORK(IV1TCS+IMIN-1)*B21D(IMIN) +
     $          RWORK(IV1TSN+IMIN-1)*B21E(IMIN)
         B21E(IMIN) = RWORK(IV1TCS+IMIN-1)*B21E(IMIN) -
     $                RWORK(IV1TSN+IMIN-1)*B21D(IMIN)
         B21D(IMIN) = TEMP
         B21BULGE = RWORK(IV1TSN+IMIN-1)*B21D(IMIN+1)
         B21D(IMIN+1) = RWORK(IV1TCS+IMIN-1)*B21D(IMIN+1)
*
*        Compute THETA(IMIN)
*
         THETA( IMIN ) = ATAN2( SQRT( B21D(IMIN)**2+B21BULGE**2 ),
     $                   SQRT( B11D(IMIN)**2+B11BULGE**2 ) )
*
*        Chase the bulges in B11(IMIN+1,IMIN) and B21(IMIN+1,IMIN)
*
         IF( B11D(IMIN)**2+B11BULGE**2 .GT. THRESH**2 ) THEN
            CALL SLARTGP( B11BULGE, B11D(IMIN), RWORK(IU1SN+IMIN-1),
     $                    RWORK(IU1CS+IMIN-1), R )
```

```
         ELSE IF( MU .LE. NU ) THEN
            CALL SLARTGS( B11E( IMIN ), B11D( IMIN + 1 ), MU,
     $                    RWORK(IU1CS+IMIN-1), RWORK(IU1SN+IMIN-1) )
         ELSE
            CALL SLARTGS( B12D( IMIN ), B12E( IMIN ), NU,
     $                    RWORK(IU1CS+IMIN-1), RWORK(IU1SN+IMIN-1) )
         END IF
         IF( B21D(IMIN)**2+B21BULGE**2 .GT. THRESH**2 ) THEN
            CALL SLARTGP( B21BULGE, B21D(IMIN), RWORK(IU2SN+IMIN-1),
     $                    RWORK(IU2CS+IMIN-1), R )
         ELSE IF( NU .LT. MU ) THEN
            CALL SLARTGS( B21E( IMIN ), B21D( IMIN + 1 ), NU,
     $                    RWORK(IU2CS+IMIN-1), RWORK(IU2SN+IMIN-1) )
         ELSE
            CALL SLARTGS( B22D(IMIN), B22E(IMIN), MU,
     $                    RWORK(IU2CS+IMIN-1), RWORK(IU2SN+IMIN-1) )
         END IF
         RWORK(IU2CS+IMIN-1) = -RWORK(IU2CS+IMIN-1)
         RWORK(IU2SN+IMIN-1) = -RWORK(IU2SN+IMIN-1)
*
         TEMP = RWORK(IU1CS+IMIN-1)*B11E(IMIN) +
     $          RWORK(IU1SN+IMIN-1)*B11D(IMIN+1)
         B11D(IMIN+1) = RWORK(IU1CS+IMIN-1)*B11D(IMIN+1) -
     $                  RWORK(IU1SN+IMIN-1)*B11E(IMIN)
         B11E(IMIN) = TEMP
         IF( IMAX .GT. IMIN+1 ) THEN
            B11BULGE = RWORK(IU1SN+IMIN-1)*B11E(IMIN+1)
            B11E(IMIN+1) = RWORK(IU1CS+IMIN-1)*B11E(IMIN+1)
         END IF
         TEMP = RWORK(IU1CS+IMIN-1)*B12D(IMIN) +
     $          RWORK(IU1SN+IMIN-1)*B12E(IMIN)
         B12E(IMIN) = RWORK(IU1CS+IMIN-1)*B12E(IMIN) -
     $                RWORK(IU1SN+IMIN-1)*B12D(IMIN)
         B12D(IMIN) = TEMP
         B12BULGE = RWORK(IU1SN+IMIN-1)*B12D(IMIN+1)
         B12D(IMIN+1) = RWORK(IU1CS+IMIN-1)*B12D(IMIN+1)
         TEMP = RWORK(IU2CS+IMIN-1)*B21E(IMIN) +
     $          RWORK(IU2SN+IMIN-1)*B21D(IMIN+1)
         B21D(IMIN+1) = RWORK(IU2CS+IMIN-1)*B21D(IMIN+1) -
     $                  RWORK(IU2SN+IMIN-1)*B21E(IMIN)
         B21E(IMIN) = TEMP
         IF( IMAX .GT. IMIN+1 ) THEN
            B21BULGE = RWORK(IU2SN+IMIN-1)*B21E(IMIN+1)
            B21E(IMIN+1) = RWORK(IU2CS+IMIN-1)*B21E(IMIN+1)
         END IF
         TEMP = RWORK(IU2CS+IMIN-1)*B22D(IMIN) +
     $          RWORK(IU2SN+IMIN-1)*B22E(IMIN)
         B22E(IMIN) = RWORK(IU2CS+IMIN-1)*B22E(IMIN) -
     $                RWORK(IU2SN+IMIN-1)*B22D(IMIN)
         B22D(IMIN) = TEMP
```

```
         B22BULGE = RWORK(IU2SN+IMIN-1)*B22D(IMIN+1)
         B22D(IMIN+1) = RWORK(IU2CS+IMIN-1)*B22D(IMIN+1)
*
*        Inner loop: chase bulges from B11(IMIN,IMIN+2),
*        B12(IMIN,IMIN+1), B21(IMIN,IMIN+2), and B22(IMIN,IMIN+1) to
*        bottom-right
*
         DO I = IMIN+1, IMAX-1
*
*           Compute PHI(I-1)
*
            X1 = SIN(THETA(I-1))*B11E(I-1) + COS(THETA(I-1))*B21E(I-1)
            X2 = SIN(THETA(I-1))*B11BULGE + COS(THETA(I-1))*B21BULGE
            Y1 = SIN(THETA(I-1))*B12D(I-1) + COS(THETA(I-1))*B22D(I-1)
            Y2 = SIN(THETA(I-1))*B12BULGE + COS(THETA(I-1))*B22BULGE
*
            PHI(I-1) = ATAN2( SQRT(X1**2+X2**2), SQRT(Y1**2+Y2**2) )
*
*           Determine if there are bulges to chase or if a new direct
*           summand has been reached
*
            RESTART11 = B11E(I-1)**2 + B11BULGE**2 .LE. THRESH**2
            RESTART21 = B21E(I-1)**2 + B21BULGE**2 .LE. THRESH**2
            RESTART12 = B12D(I-1)**2 + B12BULGE**2 .LE. THRESH**2
            RESTART22 = B22D(I-1)**2 + B22BULGE**2 .LE. THRESH**2
*
*           If possible, chase bulges from B11(I-1,I+1), B12(I-1,I),
*           B21(I-1,I+1), and B22(I-1,I). If necessary, restart bulge-
*           chasing by applying the original shift again.
*
            IF( .NOT. RESTART11 .AND. .NOT. RESTART21 ) THEN
               CALL SLARTGP( X2, X1, RWORK(IV1TSN+I-1),
     $                       RWORK(IV1TCS+I-1), R )
            ELSE IF( .NOT. RESTART11 .AND. RESTART21 ) THEN
               CALL SLARTGP( B11BULGE, B11E(I-1), RWORK(IV1TSN+I-1),
     $                       RWORK(IV1TCS+I-1), R )
            ELSE IF( RESTART11 .AND. .NOT. RESTART21 ) THEN
               CALL SLARTGP( B21BULGE, B21E(I-1), RWORK(IV1TSN+I-1),
     $                       RWORK(IV1TCS+I-1), R )
            ELSE IF( MU .LE. NU ) THEN
               CALL SLARTGS( B11D(I), B11E(I), MU, RWORK(IV1TCS+I-1),
     $                       RWORK(IV1TSN+I-1) )
            ELSE
               CALL SLARTGS( B21D(I), B21E(I), NU, RWORK(IV1TCS+I-1),
     $                       RWORK(IV1TSN+I-1) )
            END IF
            RWORK(IV1TCS+I-1) = -RWORK(IV1TCS+I-1)
            RWORK(IV1TSN+I-1) = -RWORK(IV1TSN+I-1)
            IF( .NOT. RESTART12 .AND. .NOT. RESTART22 ) THEN
               CALL SLARTGP( Y2, Y1, RWORK(IV2TSN+I-1-1),
```

```
     $                     RWORK(IV2TCS+I-1-1), R )
            ELSE IF( .NOT. RESTART12 .AND. RESTART22 ) THEN
               CALL SLARTGP( B12BULGE, B12D(I-1), RWORK(IV2TSN+I-1-1),
     $                     RWORK(IV2TCS+I-1-1), R )
            ELSE IF( RESTART12 .AND. .NOT. RESTART22 ) THEN
               CALL SLARTGP( B22BULGE, B22D(I-1), RWORK(IV2TSN+I-1-1),
     $                     RWORK(IV2TCS+I-1-1), R )
            ELSE IF( NU .LT. MU ) THEN
               CALL SLARTGS( B12E(I-1), B12D(I), NU,
     $                     RWORK(IV2TCS+I-1-1), RWORK(IV2TSN+I-1-1) )
            ELSE
               CALL SLARTGS( B22E(I-1), B22D(I), MU,
     $                     RWORK(IV2TCS+I-1-1), RWORK(IV2TSN+I-1-1) )
            END IF
*
            TEMP = RWORK(IV1TCS+I-1)*B11D(I) + RWORK(IV1TSN+I-1)*B11E(I)
            B11E(I) = RWORK(IV1TCS+I-1)*B11E(I) -
     $                RWORK(IV1TSN+I-1)*B11D(I)
            B11D(I) = TEMP
            B11BULGE = RWORK(IV1TSN+I-1)*B11D(I+1)
            B11D(I+1) = RWORK(IV1TCS+I-1)*B11D(I+1)
            TEMP = RWORK(IV1TCS+I-1)*B21D(I) + RWORK(IV1TSN+I-1)*B21E(I)
            B21E(I) = RWORK(IV1TCS+I-1)*B21E(I) -
     $                RWORK(IV1TSN+I-1)*B21D(I)
            B21D(I) = TEMP
            B21BULGE = RWORK(IV1TSN+I-1)*B21D(I+1)
            B21D(I+1) = RWORK(IV1TCS+I-1)*B21D(I+1)
            TEMP = RWORK(IV2TCS+I-1-1)*B12E(I-1) +
     $             RWORK(IV2TSN+I-1-1)*B12D(I)
            B12D(I) = RWORK(IV2TCS+I-1-1)*B12D(I) -
     $                RWORK(IV2TSN+I-1-1)*B12E(I-1)
            B12E(I-1) = TEMP
            B12BULGE = RWORK(IV2TSN+I-1-1)*B12E(I)
            B12E(I) = RWORK(IV2TCS+I-1-1)*B12E(I)
            TEMP = RWORK(IV2TCS+I-1-1)*B22E(I-1) +
     $             RWORK(IV2TSN+I-1-1)*B22D(I)
            B22D(I) = RWORK(IV2TCS+I-1-1)*B22D(I) -
     $                RWORK(IV2TSN+I-1-1)*B22E(I-1)
            B22E(I-1) = TEMP
            B22BULGE = RWORK(IV2TSN+I-1-1)*B22E(I)
            B22E(I) = RWORK(IV2TCS+I-1-1)*B22E(I)
*
*           Compute THETA(I)
*
            X1 = COS(PHI(I-1))*B11D(I) + SIN(PHI(I-1))*B12E(I-1)
            X2 = COS(PHI(I-1))*B11BULGE + SIN(PHI(I-1))*B12BULGE
            Y1 = COS(PHI(I-1))*B21D(I) + SIN(PHI(I-1))*B22E(I-1)
            Y2 = COS(PHI(I-1))*B21BULGE + SIN(PHI(I-1))*B22BULGE
*
            THETA(I) = ATAN2( SQRT(Y1**2+Y2**2), SQRT(X1**2+X2**2) )
```

```
*
*          Determine if there are bulges to chase or if a new direct
*          summand has been reached
*
           RESTART11 =   B11D(I)**2 + B11BULGE**2 .LE. THRESH**2
           RESTART12 = B12E(I-1)**2 + B12BULGE**2 .LE. THRESH**2
           RESTART21 =   B21D(I)**2 + B21BULGE**2 .LE. THRESH**2
           RESTART22 = B22E(I-1)**2 + B22BULGE**2 .LE. THRESH**2
*
*          If possible, chase bulges from B11(I+1,I), B12(I+1,I-1),
*          B21(I+1,I), and B22(I+1,I-1). If necessary, restart bulge-
*          chasing by applying the original shift again.
*
           IF( .NOT. RESTART11 .AND. .NOT. RESTART12 ) THEN
              CALL SLARTGP( X2, X1, RWORK(IU1SN+I-1), RWORK(IU1CS+I-1),
     $                      R )
           ELSE IF( .NOT. RESTART11 .AND. RESTART12 ) THEN
              CALL SLARTGP( B11BULGE, B11D(I), RWORK(IU1SN+I-1),
     $                      RWORK(IU1CS+I-1), R )
           ELSE IF( RESTART11 .AND. .NOT. RESTART12 ) THEN
              CALL SLARTGP( B12BULGE, B12E(I-1), RWORK(IU1SN+I-1),
     $                      RWORK(IU1CS+I-1), R )
           ELSE IF( MU .LE. NU ) THEN
              CALL SLARTGS( B11E(I), B11D(I+1), MU, RWORK(IU1CS+I-1),
     $                      RWORK(IU1SN+I-1) )
           ELSE
              CALL SLARTGS( B12D(I), B12E(I), NU, RWORK(IU1CS+I-1),
     $                      RWORK(IU1SN+I-1) )
           END IF
           IF( .NOT. RESTART21 .AND. .NOT. RESTART22 ) THEN
              CALL SLARTGP( Y2, Y1, RWORK(IU2SN+I-1), RWORK(IU2CS+I-1),
     $                      R )
           ELSE IF( .NOT. RESTART21 .AND. RESTART22 ) THEN
              CALL SLARTGP( B21BULGE, B21D(I), RWORK(IU2SN+I-1),
     $                      RWORK(IU2CS+I-1), R )
           ELSE IF( RESTART21 .AND. .NOT. RESTART22 ) THEN
              CALL SLARTGP( B22BULGE, B22E(I-1), RWORK(IU2SN+I-1),
     $                      RWORK(IU2CS+I-1), R )
           ELSE IF( NU .LT. MU ) THEN
              CALL SLARTGS( B21E(I), B21E(I+1), NU, RWORK(IU2CS+I-1),
     $                      RWORK(IU2SN+I-1) )
           ELSE
              CALL SLARTGS( B22D(I), B22E(I), MU, RWORK(IU2CS+I-1),
     $                      RWORK(IU2SN+I-1) )
           END IF
           RWORK(IU2CS+I-1) = -RWORK(IU2CS+I-1)
           RWORK(IU2SN+I-1) = -RWORK(IU2SN+I-1)
*
           TEMP = RWORK(IU1CS+I-1)*B11E(I) + RWORK(IU1SN+I-1)*B11D(I+1)
           B11D(I+1) = RWORK(IU1CS+I-1)*B11D(I+1) -
```

```
     $                 RWORK(IU1SN+I-1)*B11E(I)
            B11E(I) = TEMP
            IF( I .LT. IMAX - 1 ) THEN
               B11BULGE = RWORK(IU1SN+I-1)*B11E(I+1)
               B11E(I+1) = RWORK(IU1CS+I-1)*B11E(I+1)
            END IF
            TEMP = RWORK(IU2CS+I-1)*B21E(I) + RWORK(IU2SN+I-1)*B21D(I+1)
            B21D(I+1) = RWORK(IU2CS+I-1)*B21D(I+1) -
     $                 RWORK(IU2SN+I-1)*B21E(I)
            B21E(I) = TEMP
            IF( I .LT. IMAX - 1 ) THEN
               B21BULGE = RWORK(IU2SN+I-1)*B21E(I+1)
               B21E(I+1) = RWORK(IU2CS+I-1)*B21E(I+1)
            END IF
            TEMP = RWORK(IU1CS+I-1)*B12D(I) + RWORK(IU1SN+I-1)*B12E(I)
            B12E(I) = RWORK(IU1CS+I-1)*B12E(I) -
     $                RWORK(IU1SN+I-1)*B12D(I)
            B12D(I) = TEMP
            B12BULGE = RWORK(IU1SN+I-1)*B12D(I+1)
            B12D(I+1) = RWORK(IU1CS+I-1)*B12D(I+1)
            TEMP = RWORK(IU2CS+I-1)*B22D(I) + RWORK(IU2SN+I-1)*B22E(I)
            B22E(I) = RWORK(IU2CS+I-1)*B22E(I) -
     $                RWORK(IU2SN+I-1)*B22D(I)
            B22D(I) = TEMP
            B22BULGE = RWORK(IU2SN+I-1)*B22D(I+1)
            B22D(I+1) = RWORK(IU2CS+I-1)*B22D(I+1)
*
         END DO
*
*        Compute PHI(IMAX-1)
*
         X1 = SIN(THETA(IMAX-1))*B11E(IMAX-1) +
     $       COS(THETA(IMAX-1))*B21E(IMAX-1)
         Y1 = SIN(THETA(IMAX-1))*B12D(IMAX-1) +
     $       COS(THETA(IMAX-1))*B22D(IMAX-1)
         Y2 = SIN(THETA(IMAX-1))*B12BULGE + COS(THETA(IMAX-1))*B22BULGE
*
         PHI(IMAX-1) = ATAN2( ABS(X1), SQRT(Y1**2+Y2**2) )
*
*        Chase bulges from B12(IMAX-1,IMAX) and B22(IMAX-1,IMAX)
*
         RESTART12 = B12D(IMAX-1)**2 + B12BULGE**2 .LE. THRESH**2
         RESTART22 = B22D(IMAX-1)**2 + B22BULGE**2 .LE. THRESH**2
*
         IF( .NOT. RESTART12 .AND. .NOT. RESTART22 ) THEN
            CALL SLARTGP( Y2, Y1, RWORK(IV2TSN+IMAX-1-1),
     $                   RWORK(IV2TCS+IMAX-1-1), R )
         ELSE IF( .NOT. RESTART12 .AND. RESTART22 ) THEN
            CALL SLARTGP( B12BULGE, B12D(IMAX-1),
     $                   RWORK(IV2TSN+IMAX-1-1),
```

```
     $                   RWORK(IV2TCS+IMAX-1-1), R )
         ELSE IF( RESTART12 .AND. .NOT. RESTART22 ) THEN
            CALL SLARTGP( B22BULGE, B22D(IMAX-1),
     $                   RWORK(IV2TSN+IMAX-1-1),
     $                   RWORK(IV2TCS+IMAX-1-1), R )
         ELSE IF( NU .LT. MU ) THEN
            CALL SLARTGS( B12E(IMAX-1), B12D(IMAX), NU,
     $                   RWORK(IV2TCS+IMAX-1-1),
     $                   RWORK(IV2TSN+IMAX-1-1) )
         ELSE
            CALL SLARTGS( B22E(IMAX-1), B22D(IMAX), MU,
     $                   RWORK(IV2TCS+IMAX-1-1),
     $                   RWORK(IV2TSN+IMAX-1-1) )
         END IF
*
         TEMP = RWORK(IV2TCS+IMAX-1-1)*B12E(IMAX-1) +
     $          RWORK(IV2TSN+IMAX-1-1)*B12D(IMAX)
         B12D(IMAX) = RWORK(IV2TCS+IMAX-1-1)*B12D(IMAX) -
     $               RWORK(IV2TSN+IMAX-1-1)*B12E(IMAX-1)
         B12E(IMAX-1) = TEMP
         TEMP = RWORK(IV2TCS+IMAX-1-1)*B22E(IMAX-1) +
     $          RWORK(IV2TSN+IMAX-1-1)*B22D(IMAX)
         B22D(IMAX) = RWORK(IV2TCS+IMAX-1-1)*B22D(IMAX) -
     $               RWORK(IV2TSN+IMAX-1-1)*B22E(IMAX-1)
         B22E(IMAX-1) = TEMP
*
*        Update singular vectors
*
         IF( WANTU1 ) THEN
            IF( COLMAJOR ) THEN
               CALL CLASR( 'R', 'V', 'F', P, IMAX-IMIN+1,
     $                     RWORK(IU1CS+IMIN-1), RWORK(IU1SN+IMIN-1),
     $                     U1(1,IMIN), LDU1 )
            ELSE
               CALL CLASR( 'L', 'V', 'F', IMAX-IMIN+1, P,
     $                     RWORK(IU1CS+IMIN-1), RWORK(IU1SN+IMIN-1),
     $                     U1(IMIN,1), LDU1 )
            END IF
         END IF
         IF( WANTU2 ) THEN
            IF( COLMAJOR ) THEN
               CALL CLASR( 'R', 'V', 'F', M-P, IMAX-IMIN+1,
     $                     RWORK(IU2CS+IMIN-1), RWORK(IU2SN+IMIN-1),
     $                     U2(1,IMIN), LDU2 )
            ELSE
               CALL CLASR( 'L', 'V', 'F', IMAX-IMIN+1, M-P,
     $                     RWORK(IU2CS+IMIN-1), RWORK(IU2SN+IMIN-1),
     $                     U2(IMIN,1), LDU2 )
            END IF
         END IF
```

```
         IF( WANTV1T ) THEN
            IF( COLMAJOR ) THEN
               CALL CLASR( 'L', 'V', 'F', IMAX-IMIN+1, Q,
     $                     RWORK(IV1TCS+IMIN-1), RWORK(IV1TSN+IMIN-1),
     $                     V1T(IMIN,1), LDV1T )
            ELSE
               CALL CLASR( 'R', 'V', 'F', Q, IMAX-IMIN+1,
     $                     RWORK(IV1TCS+IMIN-1), RWORK(IV1TSN+IMIN-1),
     $                     V1T(1,IMIN), LDV1T )
            END IF
         END IF
         IF( WANTV2T ) THEN
            IF( COLMAJOR ) THEN
               CALL CLASR( 'L', 'V', 'F', IMAX-IMIN+1, M-Q,
     $                     RWORK(IV2TCS+IMIN-1), RWORK(IV2TSN+IMIN-1),
     $                     V2T(IMIN,1), LDV2T )
            ELSE
               CALL CLASR( 'R', 'V', 'F', M-Q, IMAX-IMIN+1,
     $                     RWORK(IV2TCS+IMIN-1), RWORK(IV2TSN+IMIN-1),
     $                     V2T(1,IMIN), LDV2T )
            END IF
         END IF
*
*        Fix signs on B11(IMAX-1,IMAX) and B21(IMAX-1,IMAX)
*
         IF( B11E(IMAX-1)+B21E(IMAX-1) .GT. 0 ) THEN
            B11D(IMAX) = -B11D(IMAX)
            B21D(IMAX) = -B21D(IMAX)
            IF( WANTV1T ) THEN
               IF( COLMAJOR ) THEN
                  CALL CSCAL( Q, NEGONECOMPLEX, V1T(IMAX,1), LDV1T )
               ELSE
                  CALL CSCAL( Q, NEGONECOMPLEX, V1T(1,IMAX), 1 )
               END IF
            END IF
         END IF
*
*        Compute THETA(IMAX)
*
         X1 = COS(PHI(IMAX-1))*B11D(IMAX) +
     $        SIN(PHI(IMAX-1))*B12E(IMAX-1)
         Y1 = COS(PHI(IMAX-1))*B21D(IMAX) +
     $        SIN(PHI(IMAX-1))*B22E(IMAX-1)
*
         THETA(IMAX) = ATAN2( ABS(Y1), ABS(X1) )
*
*        Fix signs on B11(IMAX,IMAX), B12(IMAX,IMAX-1), B21(IMAX,IMAX),
*        and B22(IMAX,IMAX-1)
*
         IF( B11D(IMAX)+B12E(IMAX-1) .LT. 0 ) THEN
```

```
            B12D(IMAX) = -B12D(IMAX)
            IF( WANTU1 ) THEN
               IF( COLMAJOR ) THEN
                  CALL CSCAL( P, NEGONECOMPLEX, U1(1,IMAX), 1 )
               ELSE
                  CALL CSCAL( P, NEGONECOMPLEX, U1(IMAX,1), LDU1 )
               END IF
            END IF
         END IF
         IF( B21D(IMAX)+B22E(IMAX-1) .GT. 0 ) THEN
            B22D(IMAX) = -B22D(IMAX)
            IF( WANTU2 ) THEN
               IF( COLMAJOR ) THEN
                  CALL CSCAL( M-P, NEGONECOMPLEX, U2(1,IMAX), 1 )
               ELSE
                  CALL CSCAL( M-P, NEGONECOMPLEX, U2(IMAX,1), LDU2 )
               END IF
            END IF
         END IF
*
*        Fix signs on B12(IMAX,IMAX) and B22(IMAX,IMAX)
*
         IF( B12D(IMAX)+B22D(IMAX) .LT. 0 ) THEN
            IF( WANTV2T ) THEN
               IF( COLMAJOR ) THEN
                  CALL CSCAL( M-Q, NEGONECOMPLEX, V2T(IMAX,1), LDV2T )
               ELSE
                  CALL CSCAL( M-Q, NEGONECOMPLEX, V2T(1,IMAX), 1 )
               END IF
            END IF
         END IF
*
*        Test for negligible sines or cosines
*
         DO I = IMIN, IMAX
            IF( THETA(I) .LT. THRESH ) THEN
               THETA(I) = ZERO
            ELSE IF( THETA(I) .GT. PIOVER2-THRESH ) THEN
               THETA(I) = PIOVER2
            END IF
         END DO
         DO I = IMIN, IMAX-1
            IF( PHI(I) .LT. THRESH ) THEN
               PHI(I) = ZERO
            ELSE IF( PHI(I) .GT. PIOVER2-THRESH ) THEN
               PHI(I) = PIOVER2
            END IF
         END DO
*
*        Deflate
```

```
*
         IF (IMAX .GT. 1) THEN
            DO WHILE( PHI(IMAX-1) .EQ. ZERO )
               IMAX = IMAX - 1
               IF (IMAX .LE. 1) EXIT
            END DO
         END IF
         IF( IMIN .GT. IMAX - 1 )
     $      IMIN = IMAX - 1
         IF (IMIN .GT. 1) THEN
            DO WHILE (PHI(IMIN-1) .NE. ZERO)
                IMIN = IMIN - 1
                IF (IMIN .LE. 1) EXIT
            END DO
         END IF
*
*        Repeat main iteration loop
*
      END DO
*
*     Postprocessing: order THETA from least to greatest
*
      DO I = 1, Q
*
         MINI = I
         THETAMIN = THETA(I)
         DO J = I+1, Q
            IF( THETA(J) .LT. THETAMIN ) THEN
               MINI = J
               THETAMIN = THETA(J)
            END IF
         END DO
*
         IF( MINI .NE. I ) THEN
            THETA(MINI) = THETA(I)
            THETA(I) = THETAMIN
            IF( COLMAJOR ) THEN
               IF( WANTU1 )
     $            CALL CSWAP( P, U1(1,I), 1, U1(1,MINI), 1 )
               IF( WANTU2 )
     $            CALL CSWAP( M-P, U2(1,I), 1, U2(1,MINI), 1 )
               IF( WANTV1T )
     $            CALL CSWAP( Q, V1T(I,1), LDV1T, V1T(MINI,1), LDV1T )
               IF( WANTV2T )
     $            CALL CSWAP( M-Q, V2T(I,1), LDV2T, V2T(MINI,1),
     $               LDV2T )
            ELSE
               IF( WANTU1 )
     $            CALL CSWAP( P, U1(I,1), LDU1, U1(MINI,1), LDU1 )
               IF( WANTU2 )
```

```
     $            CALL CSWAP( M-P, U2(I,1), LDU2, U2(MINI,1), LDU2 )
               IF( WANTV1T )
     $            CALL CSWAP( Q, V1T(1,I), 1, V1T(1,MINI), 1 )
               IF( WANTV2T )
     $            CALL CSWAP( M-Q, V2T(1,I), 1, V2T(1,MINI), 1 )
            END IF
         END IF
*
      END DO
*
      RETURN
*
*     End of CBBCSD
*
      END

*  =====================================================================
      SUBROUTINE CBBCSD( JOBU1, JOBU2, JOBV1T, JOBV2T, TRANS, M, P, Q,
     $                   THETA, PHI, U1, LDU1, U2, LDU2, V1T, LDV1T,
     $                   V2T, LDV2T, B11D, B11E, B12D, B12E, B21D, B21E,
     $                   B22D, B22E, RWORK, LRWORK, INFO )
*
*  -- LAPACK computational routine (version 3.5.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2013
*
*     .. Scalar Arguments ..
      CHARACTER          JOBU1, JOBU2, JOBV1T, JOBV2T, TRANS
      INTEGER            INFO, LDU1, LDU2, LDV1T, LDV2T, LRWORK, M, P, Q
*     ..
*     .. Array Arguments ..
      REAL               B11D( * ), B11E( * ), B12D( * ), B12E( * ),
     $                   B21D( * ), B21E( * ), B22D( * ), B22E( * ),
     $                   PHI( * ), THETA( * ), RWORK( * )
      COMPLEX            U1( LDU1, * ), U2( LDU2, * ), V1T( LDV1T, * ),
     $                   V2T( LDV2T, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            MAXITR
      PARAMETER          ( MAXITR = 6 )
      REAL               HUNDRED, MEIGHTH, ONE, PIOVER2, TEN, ZERO
      PARAMETER          ( HUNDRED = 100.0E0, MEIGHTH = -0.125E0,
     $                     ONE = 1.0E0, PIOVER2 = 1.57079632679489662E0,
     $                     TEN = 10.0E0, ZERO = 0.0E0 )
      COMPLEX            NEGONECOMPLEX
      PARAMETER          ( NEGONECOMPLEX = (-1.0E0,0.0E0) )
```

```
*     ..
*     .. Local Scalars ..
      LOGICAL            COLMAJOR, LQUERY, RESTART11, RESTART12,
     $                   RESTART21, RESTART22, WANTU1, WANTU2, WANTV1T,
     $                   WANTV2T
      INTEGER            I, IMIN, IMAX, ITER, IU1CS, IU1SN, IU2CS,
     $                   IU2SN, IV1TCS, IV1TSN, IV2TCS, IV2TSN, J,
     $                   LRWORKMIN, LRWORKOPT, MAXIT, MINI
      REAL               B11BULGE, B12BULGE, B21BULGE, B22BULGE, DUMMY,
     $                   EPS, MU, NU, R, SIGMA11, SIGMA21,
     $                   TEMP, THETAMAX, THETAMIN, THRESH, TOL, TOLMUL,
     $                   UNFL, X1, X2, Y1, Y2
*
*     .. External Subroutines ..
      EXTERNAL           CLASR, CSCAL, CSWAP, SLARTGP, SLARTGS, SLAS2,
     $                   XERBLA
*     ..
*     .. External Functions ..
      REAL               SLAMCH
      LOGICAL            LSAME
      EXTERNAL           LSAME, SLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, ATAN2, COS, MAX, MIN, SIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test input arguments
*
      INFO = 0
      LQUERY = LRWORK .EQ. -1
      WANTU1 = LSAME( JOBU1, 'Y' )
      WANTU2 = LSAME( JOBU2, 'Y' )
      WANTV1T = LSAME( JOBV1T, 'Y' )
      WANTV2T = LSAME( JOBV2T, 'Y' )
      COLMAJOR = .NOT. LSAME( TRANS, 'T' )
*
      IF( M .LT. 0 ) THEN
         INFO = -6
      ELSE IF( P .LT. 0 .OR. P .GT. M ) THEN
         INFO = -7
      ELSE IF( Q .LT. 0 .OR. Q .GT. M ) THEN
         INFO = -8
      ELSE IF( Q .GT. P .OR. Q .GT. M-P .OR. Q .GT. M-Q ) THEN
         INFO = -8
      ELSE IF( WANTU1 .AND. LDU1 .LT. P ) THEN
         INFO = -12
      ELSE IF( WANTU2 .AND. LDU2 .LT. M-P ) THEN
         INFO = -14
      ELSE IF( WANTV1T .AND. LDV1T .LT. Q ) THEN
```

```
         INFO = -16
      ELSE IF( WANTV2T .AND. LDV2T .LT. M-Q ) THEN
         INFO = -18
      END IF
*
*     Quick return if Q = 0
*
      IF( INFO .EQ. 0 .AND. Q .EQ. 0 ) THEN
         LRWORKMIN = 1
         RWORK(1) = LRWORKMIN
         RETURN
      END IF
*
*     Compute workspace
*
      IF( INFO .EQ. 0 ) THEN
         IU1CS = 1
         IU1SN = IU1CS + Q
         IU2CS = IU1SN + Q
         IU2SN = IU2CS + Q
         IV1TCS = IU2SN + Q
         IV1TSN = IV1TCS + Q
         IV2TCS = IV1TSN + Q
         IV2TSN = IV2TCS + Q
         LRWORKOPT = IV2TSN + Q - 1
         LRWORKMIN = LRWORKOPT
         RWORK(1) = LRWORKOPT
         IF( LRWORK .LT. LRWORKMIN .AND. .NOT. LQUERY ) THEN
            INFO = -28
         END IF
      END IF
*
      IF( INFO .NE. 0 ) THEN
         CALL XERBLA( 'CBBCSD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = SLAMCH( 'Epsilon' )
      UNFL = SLAMCH( 'Safe minimum' )
      TOLMUL = MAX( TEN, MIN( HUNDRED, EPS**MEIGHTH ) )
      TOL = TOLMUL*EPS
      THRESH = MAX( TOL, MAXITR*Q*Q*UNFL )
*
*     Test for negligible sines or cosines
*
      DO I = 1, Q
```

```
         IF( THETA(I) .LT. THRESH ) THEN
            THETA(I) = ZERO
         ELSE IF( THETA(I) .GT. PIOVER2-THRESH ) THEN
            THETA(I) = PIOVER2
         END IF
      END DO
      DO I = 1, Q-1
         IF( PHI(I) .LT. THRESH ) THEN
            PHI(I) = ZERO
         ELSE IF( PHI(I) .GT. PIOVER2-THRESH ) THEN
            PHI(I) = PIOVER2
         END IF
      END DO
*
*     Initial deflation
*
      IMAX = Q
      DO WHILE( IMAX .GT. 1 )
         IF( PHI(IMAX-1) .NE. ZERO ) THEN
            EXIT
         END IF
         IMAX = IMAX - 1
      END DO
      IMIN = IMAX - 1
      IF  ( IMIN .GT. 1 ) THEN
         DO WHILE( PHI(IMIN-1) .NE. ZERO )
            IMIN = IMIN - 1
            IF  ( IMIN .LE. 1 ) EXIT
         END DO
      END IF
*
*     Initialize iteration counter
*
      MAXIT = MAXITR*Q*Q
      ITER = 0
*
*     Begin main iteration loop
*
      DO WHILE( IMAX .GT. 1 )
*
*        Compute the matrix entries
*
         B11D(IMIN) = COS( THETA(IMIN) )
         B21D(IMIN) = -SIN( THETA(IMIN) )
         DO I = IMIN, IMAX - 1
            B11E(I) = -SIN( THETA(I) ) * SIN( PHI(I) )
            B11D(I+1) = COS( THETA(I+1) ) * COS( PHI(I) )
            B12D(I) = SIN( THETA(I) ) * COS( PHI(I) )
            B12E(I) = COS( THETA(I+1) ) * SIN( PHI(I) )
            B21E(I) = -COS( THETA(I) ) * SIN( PHI(I) )
```

```
            B21D(I+1) = -SIN( THETA(I+1) ) * COS( PHI(I) )
            B22D(I) = COS( THETA(I) ) * COS( PHI(I) )
            B22E(I) = -SIN( THETA(I+1) ) * SIN( PHI(I) )
         END DO
         B12D(IMAX) = SIN( THETA(IMAX) )
         B22D(IMAX) = COS( THETA(IMAX) )
*
*        Abort if not converging; otherwise, increment ITER
*
         IF( ITER .GT. MAXIT ) THEN
            INFO = 0
            DO I = 1, Q
               IF( PHI(I) .NE. ZERO )
     $            INFO = INFO + 1
            END DO
            RETURN
         END IF
*
         ITER = ITER + IMAX - IMIN
*
*        Compute shifts
*
         THETAMAX = THETA(IMIN)
         THETAMIN = THETA(IMIN)
         DO I = IMIN+1, IMAX
            IF( THETA(I) > THETAMAX )
     $         THETAMAX = THETA(I)
            IF( THETA(I) < THETAMIN )
     $         THETAMIN = THETA(I)
         END DO
*
         IF( THETAMAX .GT. PIOVER2 - THRESH ) THEN
*
*           Zero on diagonals of B11 and B22; induce deflation with a
*           zero shift
*
            MU = ZERO
            NU = ONE
*
         ELSE IF( THETAMIN .LT. THRESH ) THEN
*
*           Zero on diagonals of B12 and B22; induce deflation with a
*           zero shift
*
            MU = ONE
            NU = ZERO
*
         ELSE
*
*           Compute shifts for B11 and B21 and use the lesser
```

```
*
            CALL SLAS2( B11D(IMAX-1), B11E(IMAX-1), B11D(IMAX), SIGMA11,
     $                  DUMMY )
            CALL SLAS2( B21D(IMAX-1), B21E(IMAX-1), B21D(IMAX), SIGMA21,
     $                  DUMMY )
*
            IF( SIGMA11 .LE. SIGMA21 ) THEN
               MU = SIGMA11
               NU = SQRT( ONE - MU**2 )
               IF( MU .LT. THRESH ) THEN
                  MU = ZERO
                  NU = ONE
               END IF
            ELSE
               NU = SIGMA21
               MU = SQRT( 1.0 - NU**2 )
               IF( NU .LT. THRESH ) THEN
                  MU = ONE
                  NU = ZERO
               END IF
            END IF
         END IF
*
*        Rotate to produce bulges in B11 and B21
*
         IF( MU .LE. NU ) THEN
            CALL SLARTGS( B11D(IMIN), B11E(IMIN), MU,
     $                    RWORK(IV1TCS+IMIN-1), RWORK(IV1TSN+IMIN-1) )
         ELSE
            CALL SLARTGS( B21D(IMIN), B21E(IMIN), NU,
     $                    RWORK(IV1TCS+IMIN-1), RWORK(IV1TSN+IMIN-1) )
         END IF
*
         TEMP = RWORK(IV1TCS+IMIN-1)*B11D(IMIN) +
     $          RWORK(IV1TSN+IMIN-1)*B11E(IMIN)
         B11E(IMIN) = RWORK(IV1TCS+IMIN-1)*B11E(IMIN) -
     $                RWORK(IV1TSN+IMIN-1)*B11D(IMIN)
         B11D(IMIN) = TEMP
         B11BULGE = RWORK(IV1TSN+IMIN-1)*B11D(IMIN+1)
         B11D(IMIN+1) = RWORK(IV1TCS+IMIN-1)*B11D(IMIN+1)
         TEMP = RWORK(IV1TCS+IMIN-1)*B21D(IMIN) +
     $          RWORK(IV1TSN+IMIN-1)*B21E(IMIN)
         B21E(IMIN) = RWORK(IV1TCS+IMIN-1)*B21E(IMIN) -
     $                RWORK(IV1TSN+IMIN-1)*B21D(IMIN)
         B21D(IMIN) = TEMP
         B21BULGE = RWORK(IV1TSN+IMIN-1)*B21D(IMIN+1)
         B21D(IMIN+1) = RWORK(IV1TCS+IMIN-1)*B21D(IMIN+1)
*
*        Compute THETA(IMIN)
*
```

```
         THETA( IMIN ) = ATAN2( SQRT( B21D(IMIN)**2+B21BULGE**2 ),
     $                   SQRT( B11D(IMIN)**2+B11BULGE**2 ) )
*
*        Chase the bulges in B11(IMIN+1,IMIN) and B21(IMIN+1,IMIN)
*
         IF( B11D(IMIN)**2+B11BULGE**2 .GT. THRESH**2 ) THEN
            CALL SLARTGP( B11BULGE, B11D(IMIN), RWORK(IU1SN+IMIN-1),
     $                    RWORK(IU1CS+IMIN-1), R )
         ELSE IF( MU .LE. NU ) THEN
            CALL SLARTGS( B11E( IMIN ), B11D( IMIN + 1 ), MU,
     $                    RWORK(IU1CS+IMIN-1), RWORK(IU1SN+IMIN-1) )
         ELSE
            CALL SLARTGS( B12D( IMIN ), B12E( IMIN ), NU,
     $                    RWORK(IU1CS+IMIN-1), RWORK(IU1SN+IMIN-1) )
         END IF
         IF( B21D(IMIN)**2+B21BULGE**2 .GT. THRESH**2 ) THEN
            CALL SLARTGP( B21BULGE, B21D(IMIN), RWORK(IU2SN+IMIN-1),
     $                    RWORK(IU2CS+IMIN-1), R )
         ELSE IF( NU .LT. MU ) THEN
            CALL SLARTGS( B21E( IMIN ), B21D( IMIN + 1 ), NU,
     $                    RWORK(IU2CS+IMIN-1), RWORK(IU2SN+IMIN-1) )
         ELSE
            CALL SLARTGS( B22D(IMIN), B22E(IMIN), MU,
     $                    RWORK(IU2CS+IMIN-1), RWORK(IU2SN+IMIN-1) )
         END IF
         RWORK(IU2CS+IMIN-1) = -RWORK(IU2CS+IMIN-1)
         RWORK(IU2SN+IMIN-1) = -RWORK(IU2SN+IMIN-1)
*
         TEMP = RWORK(IU1CS+IMIN-1)*B11E(IMIN) +
     $          RWORK(IU1SN+IMIN-1)*B11D(IMIN+1)
         B11D(IMIN+1) = RWORK(IU1CS+IMIN-1)*B11D(IMIN+1) -
     $                  RWORK(IU1SN+IMIN-1)*B11E(IMIN)
         B11E(IMIN) = TEMP
         IF( IMAX .GT. IMIN+1 ) THEN
            B11BULGE = RWORK(IU1SN+IMIN-1)*B11E(IMIN+1)
            B11E(IMIN+1) = RWORK(IU1CS+IMIN-1)*B11E(IMIN+1)
         END IF
         TEMP = RWORK(IU1CS+IMIN-1)*B12D(IMIN) +
     $          RWORK(IU1SN+IMIN-1)*B12E(IMIN)
         B12E(IMIN) = RWORK(IU1CS+IMIN-1)*B12E(IMIN) -
     $                RWORK(IU1SN+IMIN-1)*B12D(IMIN)
         B12D(IMIN) = TEMP
         B12BULGE = RWORK(IU1SN+IMIN-1)*B12D(IMIN+1)
         B12D(IMIN+1) = RWORK(IU1CS+IMIN-1)*B12D(IMIN+1)
         TEMP = RWORK(IU2CS+IMIN-1)*B21E(IMIN) +
     $          RWORK(IU2SN+IMIN-1)*B21D(IMIN+1)
         B21D(IMIN+1) = RWORK(IU2CS+IMIN-1)*B21D(IMIN+1) -
     $                  RWORK(IU2SN+IMIN-1)*B21E(IMIN)
         B21E(IMIN) = TEMP
         IF( IMAX .GT. IMIN+1 ) THEN
```

```
            B21BULGE = RWORK(IU2SN+IMIN-1)*B21E(IMIN+1)
            B21E(IMIN+1) = RWORK(IU2CS+IMIN-1)*B21E(IMIN+1)
         END IF
         TEMP = RWORK(IU2CS+IMIN-1)*B22D(IMIN) +
     $          RWORK(IU2SN+IMIN-1)*B22E(IMIN)
         B22E(IMIN) = RWORK(IU2CS+IMIN-1)*B22E(IMIN) -
     $               RWORK(IU2SN+IMIN-1)*B22D(IMIN)
         B22D(IMIN) = TEMP
         B22BULGE = RWORK(IU2SN+IMIN-1)*B22D(IMIN+1)
         B22D(IMIN+1) = RWORK(IU2CS+IMIN-1)*B22D(IMIN+1)
*
*        Inner loop: chase bulges from B11(IMIN,IMIN+2),
*        B12(IMIN,IMIN+1), B21(IMIN,IMIN+2), and B22(IMIN,IMIN+1) to
*        bottom-right
*
         DO I = IMIN+1, IMAX-1
*
*           Compute PHI(I-1)
*
            X1 = SIN(THETA(I-1))*B11E(I-1) + COS(THETA(I-1))*B21E(I-1)
            X2 = SIN(THETA(I-1))*B11BULGE + COS(THETA(I-1))*B21BULGE
            Y1 = SIN(THETA(I-1))*B12D(I-1) + COS(THETA(I-1))*B22D(I-1)
            Y2 = SIN(THETA(I-1))*B12BULGE + COS(THETA(I-1))*B22BULGE
*
            PHI(I-1) = ATAN2( SQRT(X1**2+X2**2), SQRT(Y1**2+Y2**2) )
*
*           Determine if there are bulges to chase or if a new direct
*           summand has been reached
*
            RESTART11 = B11E(I-1)**2 + B11BULGE**2 .LE. THRESH**2
            RESTART21 = B21E(I-1)**2 + B21BULGE**2 .LE. THRESH**2
            RESTART12 = B12D(I-1)**2 + B12BULGE**2 .LE. THRESH**2
            RESTART22 = B22D(I-1)**2 + B22BULGE**2 .LE. THRESH**2
*
*           If possible, chase bulges from B11(I-1,I+1), B12(I-1,I),
*           B21(I-1,I+1), and B22(I-1,I). If necessary, restart bulge-
*           chasing by applying the original shift again.
*
            IF( .NOT. RESTART11 .AND. .NOT. RESTART21 ) THEN
               CALL SLARTGP( X2, X1, RWORK(IV1TSN+I-1),
     $                       RWORK(IV1TCS+I-1), R )
            ELSE IF( .NOT. RESTART11 .AND. RESTART21 ) THEN
               CALL SLARTGP( B11BULGE, B11E(I-1), RWORK(IV1TSN+I-1),
     $                       RWORK(IV1TCS+I-1), R )
            ELSE IF( RESTART11 .AND. .NOT. RESTART21 ) THEN
               CALL SLARTGP( B21BULGE, B21E(I-1), RWORK(IV1TSN+I-1),
     $                       RWORK(IV1TCS+I-1), R )
            ELSE IF( MU .LE. NU ) THEN
               CALL SLARTGS( B11D(I), B11E(I), MU, RWORK(IV1TCS+I-1),
     $                       RWORK(IV1TSN+I-1) )
```

```
            ELSE
               CALL SLARTGS( B21D(I), B21E(I), NU, RWORK(IV1TCS+I-1),
     $                       RWORK(IV1TSN+I-1) )
            END IF
            RWORK(IV1TCS+I-1) = -RWORK(IV1TCS+I-1)
            RWORK(IV1TSN+I-1) = -RWORK(IV1TSN+I-1)
            IF( .NOT. RESTART12 .AND. .NOT. RESTART22 ) THEN
               CALL SLARTGP( Y2, Y1, RWORK(IV2TSN+I-1-1),
     $                       RWORK(IV2TCS+I-1-1), R )
            ELSE IF( .NOT. RESTART12 .AND. RESTART22 ) THEN
               CALL SLARTGP( B12BULGE, B12D(I-1), RWORK(IV2TSN+I-1-1),
     $                       RWORK(IV2TCS+I-1-1), R )
            ELSE IF( RESTART12 .AND. .NOT. RESTART22 ) THEN
               CALL SLARTGP( B22BULGE, B22D(I-1), RWORK(IV2TSN+I-1-1),
     $                       RWORK(IV2TCS+I-1-1), R )
            ELSE IF( NU .LT. MU ) THEN
               CALL SLARTGS( B12E(I-1), B12D(I), NU,
     $                       RWORK(IV2TCS+I-1-1), RWORK(IV2TSN+I-1-1) )
            ELSE
               CALL SLARTGS( B22E(I-1), B22D(I), MU,
     $                       RWORK(IV2TCS+I-1-1), RWORK(IV2TSN+I-1-1) )
            END IF
*
            TEMP = RWORK(IV1TCS+I-1)*B11D(I) + RWORK(IV1TSN+I-1)*B11E(I)
            B11E(I) = RWORK(IV1TCS+I-1)*B11E(I) -
     $                RWORK(IV1TSN+I-1)*B11D(I)
            B11D(I) = TEMP
            B11BULGE = RWORK(IV1TSN+I-1)*B11D(I+1)
            B11D(I+1) = RWORK(IV1TCS+I-1)*B11D(I+1)
            TEMP = RWORK(IV1TCS+I-1)*B21D(I) + RWORK(IV1TSN+I-1)*B21E(I)
            B21E(I) = RWORK(IV1TCS+I-1)*B21E(I) -
     $                RWORK(IV1TSN+I-1)*B21D(I)
            B21D(I) = TEMP
            B21BULGE = RWORK(IV1TSN+I-1)*B21D(I+1)
            B21D(I+1) = RWORK(IV1TCS+I-1)*B21D(I+1)
            TEMP = RWORK(IV2TCS+I-1-1)*B12E(I-1) +
     $             RWORK(IV2TSN+I-1-1)*B12D(I)
            B12D(I) = RWORK(IV2TCS+I-1-1)*B12D(I) -
     $                RWORK(IV2TSN+I-1-1)*B12E(I-1)
            B12E(I-1) = TEMP
            B12BULGE = RWORK(IV2TSN+I-1-1)*B12E(I)
            B12E(I) = RWORK(IV2TCS+I-1-1)*B12E(I)
            TEMP = RWORK(IV2TCS+I-1-1)*B22E(I-1) +
     $             RWORK(IV2TSN+I-1-1)*B22D(I)
            B22D(I) = RWORK(IV2TCS+I-1-1)*B22D(I) -
     $                RWORK(IV2TSN+I-1-1)*B22E(I-1)
            B22E(I-1) = TEMP
            B22BULGE = RWORK(IV2TSN+I-1-1)*B22E(I)
            B22E(I) = RWORK(IV2TCS+I-1-1)*B22E(I)
*
```

```
*           Compute THETA(I)
*
            X1 = COS(PHI(I-1))*B11D(I) + SIN(PHI(I-1))*B12E(I-1)
            X2 = COS(PHI(I-1))*B11BULGE + SIN(PHI(I-1))*B12BULGE
            Y1 = COS(PHI(I-1))*B21D(I) + SIN(PHI(I-1))*B22E(I-1)
            Y2 = COS(PHI(I-1))*B21BULGE + SIN(PHI(I-1))*B22BULGE
*
            THETA(I) = ATAN2( SQRT(Y1**2+Y2**2), SQRT(X1**2+X2**2) )
*
*           Determine if there are bulges to chase or if a new direct
*           summand has been reached
*
            RESTART11 =   B11D(I)**2 + B11BULGE**2 .LE. THRESH**2
            RESTART12 = B12E(I-1)**2 + B12BULGE**2 .LE. THRESH**2
            RESTART21 =   B21D(I)**2 + B21BULGE**2 .LE. THRESH**2
            RESTART22 = B22E(I-1)**2 + B22BULGE**2 .LE. THRESH**2
*
*           If possible, chase bulges from B11(I+1,I), B12(I+1,I-1),
*           B21(I+1,I), and B22(I+1,I-1). If necessary, restart bulge-
*           chasing by applying the original shift again.
*
            IF( .NOT. RESTART11 .AND. .NOT. RESTART12 ) THEN
               CALL SLARTGP( X2, X1, RWORK(IU1SN+I-1), RWORK(IU1CS+I-1),
     $                       R )
            ELSE IF( .NOT. RESTART11 .AND. RESTART12 ) THEN
               CALL SLARTGP( B11BULGE, B11D(I), RWORK(IU1SN+I-1),
     $                       RWORK(IU1CS+I-1), R )
            ELSE IF( RESTART11 .AND. .NOT. RESTART12 ) THEN
               CALL SLARTGP( B12BULGE, B12E(I-1), RWORK(IU1SN+I-1),
     $                       RWORK(IU1CS+I-1), R )
            ELSE IF( MU .LE. NU ) THEN
               CALL SLARTGS( B11E(I), B11D(I+1), MU, RWORK(IU1CS+I-1),
     $                       RWORK(IU1SN+I-1) )
            ELSE
               CALL SLARTGS( B12D(I), B12E(I), NU, RWORK(IU1CS+I-1),
     $                       RWORK(IU1SN+I-1) )
            END IF
            IF( .NOT. RESTART21 .AND. .NOT. RESTART22 ) THEN
               CALL SLARTGP( Y2, Y1, RWORK(IU2SN+I-1), RWORK(IU2CS+I-1),
     $                       R )
            ELSE IF( .NOT. RESTART21 .AND. RESTART22 ) THEN
               CALL SLARTGP( B21BULGE, B21D(I), RWORK(IU2SN+I-1),
     $                       RWORK(IU2CS+I-1), R )
            ELSE IF( RESTART21 .AND. .NOT. RESTART22 ) THEN
               CALL SLARTGP( B22BULGE, B22E(I-1), RWORK(IU2SN+I-1),
     $                       RWORK(IU2CS+I-1), R )
            ELSE IF( NU .LT. MU ) THEN
               CALL SLARTGS( B21E(I), B21E(I+1), NU, RWORK(IU2CS+I-1),
     $                       RWORK(IU2SN+I-1) )
            ELSE
```

```
               CALL SLARTGS( B22D(I), B22E(I), MU, RWORK(IU2CS+I-1),
     $                       RWORK(IU2SN+I-1) )
            END IF
            RWORK(IU2CS+I-1) = -RWORK(IU2CS+I-1)
            RWORK(IU2SN+I-1) = -RWORK(IU2SN+I-1)
*
            TEMP = RWORK(IU1CS+I-1)*B11E(I) + RWORK(IU1SN+I-1)*B11D(I+1)
            B11D(I+1) = RWORK(IU1CS+I-1)*B11D(I+1) -
     $                  RWORK(IU1SN+I-1)*B11E(I)
            B11E(I) = TEMP
            IF( I .LT. IMAX - 1 ) THEN
               B11BULGE = RWORK(IU1SN+I-1)*B11E(I+1)
               B11E(I+1) = RWORK(IU1CS+I-1)*B11E(I+1)
            END IF
            TEMP = RWORK(IU2CS+I-1)*B21E(I) + RWORK(IU2SN+I-1)*B21D(I+1)
            B21D(I+1) = RWORK(IU2CS+I-1)*B21D(I+1) -
     $                  RWORK(IU2SN+I-1)*B21E(I)
            B21E(I) = TEMP
            IF( I .LT. IMAX - 1 ) THEN
               B21BULGE = RWORK(IU2SN+I-1)*B21E(I+1)
               B21E(I+1) = RWORK(IU2CS+I-1)*B21E(I+1)
            END IF
            TEMP = RWORK(IU1CS+I-1)*B12D(I) + RWORK(IU1SN+I-1)*B12E(I)
            B12E(I) = RWORK(IU1CS+I-1)*B12E(I) -
     $                RWORK(IU1SN+I-1)*B12D(I)
            B12D(I) = TEMP
            B12BULGE = RWORK(IU1SN+I-1)*B12D(I+1)
            B12D(I+1) = RWORK(IU1CS+I-1)*B12D(I+1)
            TEMP = RWORK(IU2CS+I-1)*B22D(I) + RWORK(IU2SN+I-1)*B22E(I)
            B22E(I) = RWORK(IU2CS+I-1)*B22E(I) -
     $                RWORK(IU2SN+I-1)*B22D(I)
            B22D(I) = TEMP
            B22BULGE = RWORK(IU2SN+I-1)*B22D(I+1)
            B22D(I+1) = RWORK(IU2CS+I-1)*B22D(I+1)
*
         END DO
*
*        Compute PHI(IMAX-1)
*
         X1 = SIN(THETA(IMAX-1))*B11E(IMAX-1) +
     $        COS(THETA(IMAX-1))*B21E(IMAX-1)
         Y1 = SIN(THETA(IMAX-1))*B12D(IMAX-1) +
     $        COS(THETA(IMAX-1))*B22D(IMAX-1)
         Y2 = SIN(THETA(IMAX-1))*B12BULGE + COS(THETA(IMAX-1))*B22BULGE
*
         PHI(IMAX-1) = ATAN2( ABS(X1), SQRT(Y1**2+Y2**2) )
*
*        Chase bulges from B12(IMAX-1,IMAX) and B22(IMAX-1,IMAX)
*
         RESTART12 = B12D(IMAX-1)**2 + B12BULGE**2 .LE. THRESH**2
```

```
      RESTART22 = B22D(IMAX-1)**2 + B22BULGE**2 .LE. THRESH**2
*
      IF( .NOT. RESTART12 .AND. .NOT. RESTART22 ) THEN
         CALL SLARTGP( Y2, Y1, RWORK(IV2TSN+IMAX-1-1),
     $                 RWORK(IV2TCS+IMAX-1-1), R )
      ELSE IF( .NOT. RESTART12 .AND. RESTART22 ) THEN
         CALL SLARTGP( B12BULGE, B12D(IMAX-1),
     $                 RWORK(IV2TSN+IMAX-1-1),
     $                 RWORK(IV2TCS+IMAX-1-1), R )
      ELSE IF( RESTART12 .AND. .NOT. RESTART22 ) THEN
         CALL SLARTGP( B22BULGE, B22D(IMAX-1),
     $                 RWORK(IV2TSN+IMAX-1-1),
     $                 RWORK(IV2TCS+IMAX-1-1), R )
      ELSE IF( NU .LT. MU ) THEN
         CALL SLARTGS( B12E(IMAX-1), B12D(IMAX), NU,
     $                 RWORK(IV2TCS+IMAX-1-1),
     $                 RWORK(IV2TSN+IMAX-1-1) )
      ELSE
         CALL SLARTGS( B22E(IMAX-1), B22D(IMAX), MU,
     $                 RWORK(IV2TCS+IMAX-1-1),
     $                 RWORK(IV2TSN+IMAX-1-1) )
      END IF
*
      TEMP = RWORK(IV2TCS+IMAX-1-1)*B12E(IMAX-1) +
     $       RWORK(IV2TSN+IMAX-1-1)*B12D(IMAX)
      B12D(IMAX) = RWORK(IV2TCS+IMAX-1-1)*B12D(IMAX) -
     $             RWORK(IV2TSN+IMAX-1-1)*B12E(IMAX-1)
      B12E(IMAX-1) = TEMP
      TEMP = RWORK(IV2TCS+IMAX-1-1)*B22E(IMAX-1) +
     $       RWORK(IV2TSN+IMAX-1-1)*B22D(IMAX)
      B22D(IMAX) = RWORK(IV2TCS+IMAX-1-1)*B22D(IMAX) -
     $             RWORK(IV2TSN+IMAX-1-1)*B22E(IMAX-1)
      B22E(IMAX-1) = TEMP
*
*     Update singular vectors
*
      IF( WANTU1 ) THEN
         IF( COLMAJOR ) THEN
            CALL CLASR( 'R', 'V', 'F', P, IMAX-IMIN+1,
     $                  RWORK(IU1CS+IMIN-1), RWORK(IU1SN+IMIN-1),
     $                  U1(1,IMIN), LDU1 )
         ELSE
            CALL CLASR( 'L', 'V', 'F', IMAX-IMIN+1, P,
     $                  RWORK(IU1CS+IMIN-1), RWORK(IU1SN+IMIN-1),
     $                  U1(IMIN,1), LDU1 )
         END IF
      END IF
      IF( WANTU2 ) THEN
         IF( COLMAJOR ) THEN
            CALL CLASR( 'R', 'V', 'F', M-P, IMAX-IMIN+1,
```

```
     $                    RWORK(IU2CS+IMIN-1), RWORK(IU2SN+IMIN-1),
     $                    U2(1,IMIN), LDU2 )
            ELSE
               CALL CLASR( 'L', 'V', 'F', IMAX-IMIN+1, M-P,
     $                    RWORK(IU2CS+IMIN-1), RWORK(IU2SN+IMIN-1),
     $                    U2(IMIN,1), LDU2 )
            END IF
         END IF
         IF( WANTV1T ) THEN
            IF( COLMAJOR ) THEN
               CALL CLASR( 'L', 'V', 'F', IMAX-IMIN+1, Q,
     $                    RWORK(IV1TCS+IMIN-1), RWORK(IV1TSN+IMIN-1),
     $                    V1T(IMIN,1), LDV1T )
            ELSE
               CALL CLASR( 'R', 'V', 'F', Q, IMAX-IMIN+1,
     $                    RWORK(IV1TCS+IMIN-1), RWORK(IV1TSN+IMIN-1),
     $                    V1T(1,IMIN), LDV1T )
            END IF
         END IF
         IF( WANTV2T ) THEN
            IF( COLMAJOR ) THEN
               CALL CLASR( 'L', 'V', 'F', IMAX-IMIN+1, M-Q,
     $                    RWORK(IV2TCS+IMIN-1), RWORK(IV2TSN+IMIN-1),
     $                    V2T(IMIN,1), LDV2T )
            ELSE
               CALL CLASR( 'R', 'V', 'F', M-Q, IMAX-IMIN+1,
     $                    RWORK(IV2TCS+IMIN-1), RWORK(IV2TSN+IMIN-1),
     $                    V2T(1,IMIN), LDV2T )
            END IF
         END IF
*
*        Fix signs on B11(IMAX-1,IMAX) and B21(IMAX-1,IMAX)
*
         IF( B11E(IMAX-1)+B21E(IMAX-1) .GT. 0 ) THEN
            B11D(IMAX) = -B11D(IMAX)
            B21D(IMAX) = -B21D(IMAX)
            IF( WANTV1T ) THEN
               IF( COLMAJOR ) THEN
                  CALL CSCAL( Q, NEGONECOMPLEX, V1T(IMAX,1), LDV1T )
               ELSE
                  CALL CSCAL( Q, NEGONECOMPLEX, V1T(1,IMAX), 1 )
               END IF
            END IF
         END IF
*
*        Compute THETA(IMAX)
*
         X1 = COS(PHI(IMAX-1))*B11D(IMAX) +
     $        SIN(PHI(IMAX-1))*B12E(IMAX-1)
         Y1 = COS(PHI(IMAX-1))*B21D(IMAX) +
```

```
     $        SIN(PHI(IMAX-1))*B22E(IMAX-1)
*
         THETA(IMAX) = ATAN2( ABS(Y1), ABS(X1) )
*
*        Fix signs on B11(IMAX,IMAX), B12(IMAX,IMAX-1), B21(IMAX,IMAX),
*        and B22(IMAX,IMAX-1)
*
         IF( B11D(IMAX)+B12E(IMAX-1) .LT. 0 ) THEN
            B12D(IMAX) = -B12D(IMAX)
            IF( WANTU1 ) THEN
               IF( COLMAJOR ) THEN
                  CALL CSCAL( P, NEGONECOMPLEX, U1(1,IMAX), 1 )
               ELSE
                  CALL CSCAL( P, NEGONECOMPLEX, U1(IMAX,1), LDU1 )
               END IF
            END IF
         END IF
         IF( B21D(IMAX)+B22E(IMAX-1) .GT. 0 ) THEN
            B22D(IMAX) = -B22D(IMAX)
            IF( WANTU2 ) THEN
               IF( COLMAJOR ) THEN
                  CALL CSCAL( M-P, NEGONECOMPLEX, U2(1,IMAX), 1 )
               ELSE
                  CALL CSCAL( M-P, NEGONECOMPLEX, U2(IMAX,1), LDU2 )
               END IF
            END IF
         END IF
*
*        Fix signs on B12(IMAX,IMAX) and B22(IMAX,IMAX)
*
         IF( B12D(IMAX)+B22D(IMAX) .LT. 0 ) THEN
            IF( WANTV2T ) THEN
               IF( COLMAJOR ) THEN
                  CALL CSCAL( M-Q, NEGONECOMPLEX, V2T(IMAX,1), LDV2T )
               ELSE
                  CALL CSCAL( M-Q, NEGONECOMPLEX, V2T(1,IMAX), 1 )
               END IF
            END IF
         END IF
*
*        Test for negligible sines or cosines
*
         DO I = IMIN, IMAX
            IF( THETA(I) .LT. THRESH ) THEN
               THETA(I) = ZERO
            ELSE IF( THETA(I) .GT. PIOVER2-THRESH ) THEN
               THETA(I) = PIOVER2
            END IF
         END DO
         DO I = IMIN, IMAX-1
```

```
            IF( PHI(I) .LT. THRESH ) THEN
               PHI(I) = ZERO
            ELSE IF( PHI(I) .GT. PIOVER2-THRESH ) THEN
               PHI(I) = PIOVER2
            END IF
         END DO
*
*        Deflate
*
         IF (IMAX .GT. 1) THEN
            DO WHILE( PHI(IMAX-1) .EQ. ZERO )
               IMAX = IMAX - 1
               IF (IMAX .LE. 1) EXIT
            END DO
         END IF
         IF( IMIN .GT. IMAX - 1 )
     $      IMIN = IMAX - 1
         IF (IMIN .GT. 1) THEN
            DO WHILE (PHI(IMIN-1) .NE. ZERO)
                IMIN = IMIN - 1
                IF (IMIN .LE. 1) EXIT
            END DO
         END IF
*
*        Repeat main iteration loop
*
      END DO
*
*     Postprocessing: order THETA from least to greatest
*
      DO I = 1, Q
*
         MINI = I
         THETAMIN = THETA(I)
         DO J = I+1, Q
            IF( THETA(J) .LT. THETAMIN ) THEN
               MINI = J
               THETAMIN = THETA(J)
            END IF
         END DO
*
         IF( MINI .NE. I ) THEN
            THETA(MINI) = THETA(I)
            THETA(I) = THETAMIN
            IF( COLMAJOR ) THEN
               IF( WANTU1 )
     $            CALL CSWAP( P, U1(1,I), 1, U1(1,MINI), 1 )
               IF( WANTU2 )
     $            CALL CSWAP( M-P, U2(1,I), 1, U2(1,MINI), 1 )
               IF( WANTV1T )
```

```
     $            CALL CSWAP( Q, V1T(I,1), LDV1T, V1T(MINI,1), LDV1T )
               IF( WANTV2T )
     $            CALL CSWAP( M-Q, V2T(I,1), LDV2T, V2T(MINI,1),
     $               LDV2T )
            ELSE
               IF( WANTU1 )
     $            CALL CSWAP( P, U1(I,1), LDU1, U1(MINI,1), LDU1 )
               IF( WANTU2 )
     $            CALL CSWAP( M-P, U2(I,1), LDU2, U2(MINI,1), LDU2 )
               IF( WANTV1T )
     $            CALL CSWAP( Q, V1T(1,I), 1, V1T(1,MINI), 1 )
               IF( WANTV2T )
     $            CALL CSWAP( M-Q, V2T(1,I), 1, V2T(1,MINI), 1 )
            END IF
         END IF
*
      END DO
*
      RETURN
*
*     End of CBBCSD
*
      END
```

———-

90.110.4 lisp code

— cbbcsd.lisp —

```
(let* ((maxitr 6)
       (hundred 100.0)
       (meighth (- 0.125))
       (one 1.0)
       (piover2 1.5707964)
       (ten 10.0)
       (zero 0.0)
       (negonecomplex (f2cl-lib:cmplx (- 1.0) 0.0)))
  (declare (type (f2cl-lib:integer4 6 6) maxitr)
           (type (single-float 100.0 100.0) hundred)
           (type (single-float) meighth)
           (type (single-float 1.0 1.0) one)
           (type (single-float 1.5707964 1.5707964) piover2)
           (type (single-float 10.0 10.0) ten)
           (type (single-float 0.0 0.0) zero)
           (type (f2cl-lib:complex8) negonecomplex)
           (ignorable maxitr hundred meighth one piover2 ten zero
```

```
          negonecomplex))
(defun cbbcsd
       (jobu1 jobu2 jobv1t jobv2t trans m p q theta phi u1 ldu1 u2 ldu2 v1t
        ldv1t v2t ldv2t b11d b11e b12d b12e b21d b21e b22d b22e rwork lrwork
        info)
  (declare (type (array f2cl-lib:complex8 (*)) v2t v1t u2 u1)
           (type (array single-float (*)) rwork b22e b22d b21e b21d b12e b12d
            b11e b11d phi theta)
           (type (f2cl-lib:integer4) info lrwork ldv2t ldv1t ldu2 ldu1 q p m)
           (type (string *) trans jobv2t jobv1t jobu2 jobu1))
  (f2cl-lib:with-multi-array-data
      ((jobu1 character jobu1-%data% jobu1-%offset%)
       (jobu2 character jobu2-%data% jobu2-%offset%)
       (jobv1t character jobv1t-%data% jobv1t-%offset%)
       (jobv2t character jobv2t-%data% jobv2t-%offset%)
       (trans character trans-%data% trans-%offset%)
       (theta single-float theta-%data% theta-%offset%)
       (phi single-float phi-%data% phi-%offset%)
       (b11d single-float b11d-%data% b11d-%offset%)
       (b11e single-float b11e-%data% b11e-%offset%)
       (b12d single-float b12d-%data% b12d-%offset%)
       (b12e single-float b12e-%data% b12e-%offset%)
       (b21d single-float b21d-%data% b21d-%offset%)
       (b21e single-float b21e-%data% b21e-%offset%)
       (b22d single-float b22d-%data% b22d-%offset%)
       (b22e single-float b22e-%data% b22e-%offset%)
       (rwork single-float rwork-%data% rwork-%offset%)
       (u1 f2cl-lib:complex8 u1-%data% u1-%offset%)
       (u2 f2cl-lib:complex8 u2-%data% u2-%offset%)
       (v1t f2cl-lib:complex8 v1t-%data% v1t-%offset%)
       (v2t f2cl-lib:complex8 v2t-%data% v2t-%offset%))
    (prog ((b11bulge 0.0) (b12bulge 0.0) (b21bulge 0.0) (b22bulge 0.0)
           (dummy 0.0) (eps 0.0) (mu 0.0) (nu 0.0) (r 0.0) (sigma11 0.0)
           (sigma21 0.0) (temp 0.0) (thetamax 0.0) (thetamin 0.0)
           (thresh 0.0) (tol 0.0) (tolmul 0.0) (unfl 0.0) (x1 0.0) (x2 0.0)
           (y1 0.0) (y2 0.0) (i 0) (imin 0) (imax 0) (iter 0) (iu1cs 0)
           (iu1sn 0) (iu2cs 0) (iu2sn 0) (iv1tcs 0) (iv1tsn 0) (iv2tcs 0)
           (iv2tsn 0) (j 0) (lrworkmin 0) (lrworkopt 0) (maxit 0) (mini 0)
           (colmajor nil) (lquery nil) (restart11 nil) (restart12 nil)
           (restart21 nil) (restart22 nil) (wantu1 nil) (wantu2 nil)
           (wantv1t nil) (wantv2t nil))
      (declare
       (type (single-float) b11bulge b12bulge b21bulge b22bulge dummy eps mu
        nu r sigma11 sigma21 temp thetamax thetamin thresh tol tolmul unfl x1
        x2 y1 y2)
       (type (f2cl-lib:integer4) i imin imax iter iu1cs iu1sn iu2cs iu2sn
        iv1tcs iv1tsn iv2tcs iv2tsn j lrworkmin lrworkopt maxit mini)
       (type f2cl-lib:logical colmajor lquery restart11 restart12 restart21
        restart22 wantu1 wantu2 wantv1t wantv2t))
      (setf info 0)
```

```
(setf lquery (coerce (= lrwork -1) 'f2cl-lib:logical))
(setf wantu1
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobu1 "Y")
          (declare (ignore var-1))
          (when var-0 (setf jobu1 var-0))
          ret-val))
(setf wantu2
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobu2 "Y")
          (declare (ignore var-1))
          (when var-0 (setf jobu2 var-0))
          ret-val))
(setf wantv1t
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobv1t "Y")
          (declare (ignore var-1))
          (when var-0 (setf jobv1t var-0))
          ret-val))
(setf wantv2t
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobv2t "Y")
          (declare (ignore var-1))
          (when var-0 (setf jobv2t var-0))
          ret-val))
(setf colmajor
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame trans "T")
           (declare (ignore var-1))
           (when var-0 (setf trans var-0))
           ret-val)))
(cond ((< m 0) (setf info -6)) ((or (< p 0) (> p m)) (setf info -7))
      ((or (< q 0) (> q m)) (setf info -8))
      ((or (> q p) (> q (f2cl-lib:int-add m (f2cl-lib:int-sub p)))
           (> q (f2cl-lib:int-add m (f2cl-lib:int-sub q))))
       (setf info -8))
      ((and wantu1 (< ldu1 p)) (setf info -12))
      ((and wantu2 (< ldu2 (f2cl-lib:int-add m (f2cl-lib:int-sub p))))
       (setf info -14))
      ((and wantv1t (< ldv1t q)) (setf info -16))
      ((and wantv2t
            (< ldv2t (f2cl-lib:int-add m (f2cl-lib:int-sub q))))
       (setf info -18)))
(cond
 ((and (= info 0) (= q 0)) (setf lrworkmin 1)
  (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
          (coerce (the f2cl-lib:integer4 lrworkmin) 'single-float))
  (go end_label)))
(cond
```

```
   ((= info 0) (setf iu1cs 1) (setf iu1sn (f2cl-lib:int-add iu1cs q))
    (setf iu2cs (f2cl-lib:int-add iu1sn q))
    (setf iu2sn (f2cl-lib:int-add iu2cs q))
    (setf iv1tcs (f2cl-lib:int-add iu2sn q))
    (setf iv1tsn (f2cl-lib:int-add iv1tcs q))
    (setf iv2tcs (f2cl-lib:int-add iv1tsn q))
    (setf iv2tsn (f2cl-lib:int-add iv2tcs q))
    (setf lrworkopt (f2cl-lib:int-sub (f2cl-lib:int-add iv2tsn q) 1))
    (setf lrworkmin lrworkopt)
    (setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%)
            (coerce (the f2cl-lib:integer4 lrworkopt) 'single-float))
    (cond ((and (< lrwork lrworkmin) (not lquery)) (setf info -28)))))
  (cond
   ((/= info 0) (xerbla "CBBCSD" (f2cl-lib:int-sub info)) (go end_label))
   (lquery (go end_label)))
  (setf eps (slamch "Epsilon"))
  (setf unfl (slamch "Safe minimum"))
  (setf tolmul (max ten (min hundred (expt eps meighth))))
  (setf tol (* tolmul eps))
  (setf thresh (max tol (* maxitr q q unfl)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i q) nil)
    (tagbody
      (cond
       ((< (f2cl-lib:fref theta (i) ((1 *))) thresh)
        (setf (f2cl-lib:fref theta-%data% (i) ((1 *)) theta-%offset%)
                zero))
       ((> (f2cl-lib:fref theta (i) ((1 *))) (+ piover2 (- thresh)))
        (setf (f2cl-lib:fref theta-%data% (i) ((1 *)) theta-%offset%)
                piover2)))
     label100000))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i (f2cl-lib:int-add q (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (cond
       ((< (f2cl-lib:fref phi (i) ((1 *))) thresh)
        (setf (f2cl-lib:fref phi-%data% (i) ((1 *)) phi-%offset%) zero))
       ((> (f2cl-lib:fref phi (i) ((1 *))) (+ piover2 (- thresh)))
        (setf (f2cl-lib:fref phi-%data% (i) ((1 *)) phi-%offset%)
                piover2)))
     label100001))
  (setf imax q)
 label100002
  (if (not (> imax 1))
      (go label100003))
  (cond
   ((/=
     (f2cl-lib:fref phi ((f2cl-lib:int-add imax (f2cl-lib:int-sub 1)))
                    ((1 *)))
     zero)
```

```
    (go f2cl-lib::exit)))
  (setf imax (f2cl-lib:int-sub imax 1))
  (go label100002)
 label100003
  (setf imin (f2cl-lib:int-sub imax 1))
  (cond
   ((> imin 1)
    (tagbody
     label100004
      (if (not
           (/=
            (f2cl-lib:fref phi-%data% ((f2cl-lib:int-sub imin 1)) ((1 *))
                           phi-%offset%)
            zero))
          (go label100005))
      (setf imin (f2cl-lib:int-sub imin 1))
      (if (<= imin 1)
          (go f2cl-lib::exit))
      (go label100004)
     label100005)))
  (setf maxit (f2cl-lib:int-mul maxitr q q))
  (setf iter 0)
 label100006
  (if (not (> imax 1))
      (go label100007))
  (setf (f2cl-lib:fref b11d-%data% (imin) ((1 *)) b11d-%offset%)
          (cos
           (f2cl-lib:fref theta-%data% (imin) ((1 *)) theta-%offset%)))
  (setf (f2cl-lib:fref b21d-%data% (imin) ((1 *)) b21d-%offset%)
          (-
           (sin
            (f2cl-lib:fref theta-%data% (imin) ((1 *)) theta-%offset%))))
  (f2cl-lib:fdo (i imin (f2cl-lib:int-add i 1))
                ((> i (f2cl-lib:int-add imax (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (setf (f2cl-lib:fref b11e-%data% (i) ((1 *)) b11e-%offset%)
              (*
               (-
                (sin
                 (f2cl-lib:fref theta-%data% (i) ((1 *))
                                theta-%offset%)))
               (sin
                (f2cl-lib:fref phi-%data% (i) ((1 *)) phi-%offset%))))
      (setf (f2cl-lib:fref b11d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                           b11d-%offset%)
              (*
               (cos
                (f2cl-lib:fref theta-%data% ((f2cl-lib:int-add i 1))
                               ((1 *)) theta-%offset%))
               (cos
```

```
                (f2cl-lib:fref phi-%data% (i) ((1 *)) phi-%offset%))))
    (setf (f2cl-lib:fref b12d-%data% (i) ((1 *)) b12d-%offset%)
            (*
             (sin
              (f2cl-lib:fref theta-%data% (i) ((1 *)) theta-%offset%))
             (cos
              (f2cl-lib:fref phi-%data% (i) ((1 *)) phi-%offset%))))
    (setf (f2cl-lib:fref b12e-%data% (i) ((1 *)) b12e-%offset%)
            (*
             (cos
              (f2cl-lib:fref theta-%data% ((f2cl-lib:int-add i 1))
                             ((1 *)) theta-%offset%))
             (sin
              (f2cl-lib:fref phi-%data% (i) ((1 *)) phi-%offset%))))
    (setf (f2cl-lib:fref b21e-%data% (i) ((1 *)) b21e-%offset%)
            (*
             (-
              (cos
               (f2cl-lib:fref theta-%data% (i) ((1 *))
                              theta-%offset%)))
             (sin
              (f2cl-lib:fref phi-%data% (i) ((1 *)) phi-%offset%))))
    (setf (f2cl-lib:fref b21d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                          b21d-%offset%)
            (*
             (-
              (sin
               (f2cl-lib:fref theta-%data% ((f2cl-lib:int-add i 1))
                              ((1 *)) theta-%offset%)))
             (cos
              (f2cl-lib:fref phi-%data% (i) ((1 *)) phi-%offset%))))
    (setf (f2cl-lib:fref b22d-%data% (i) ((1 *)) b22d-%offset%)
            (*
             (cos
              (f2cl-lib:fref theta-%data% (i) ((1 *)) theta-%offset%))
             (cos
              (f2cl-lib:fref phi-%data% (i) ((1 *)) phi-%offset%))))
    (setf (f2cl-lib:fref b22e-%data% (i) ((1 *)) b22e-%offset%)
            (*
             (-
              (sin
               (f2cl-lib:fref theta-%data% ((f2cl-lib:int-add i 1))
                              ((1 *)) theta-%offset%)))
             (sin
              (f2cl-lib:fref phi-%data% (i) ((1 *)) phi-%offset%))))
   label100008))
(setf (f2cl-lib:fref b12d-%data% (imax) ((1 *)) b12d-%offset%)
        (sin
         (f2cl-lib:fref theta-%data% (imax) ((1 *)) theta-%offset%)))
(setf (f2cl-lib:fref b22d-%data% (imax) ((1 *)) b22d-%offset%)
```

```
        (cos
         (f2cl-lib:fref theta-%data% (imax) ((1 *)) theta-%offset%)))
(cond
 ((> iter maxit) (setf info 0)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i q) nil)
    (tagbody
      (if (/= (f2cl-lib:fref phi-%data% (i) ((1 *)) phi-%offset%) zero)
          (setf info (f2cl-lib:int-add info 1)))
     label100009))
  (go end_label)))
(setf iter (f2cl-lib:int-sub (f2cl-lib:int-add iter imax) imin))
(setf thetamax
        (f2cl-lib:fref theta-%data% (imin) ((1 *)) theta-%offset%))
(setf thetamin
        (f2cl-lib:fref theta-%data% (imin) ((1 *)) theta-%offset%))
(f2cl-lib:fdo (i (f2cl-lib:int-add imin 1) (f2cl-lib:int-add i 1))
              ((> i imax) nil)
  (tagbody
    (if (> (f2cl-lib:fref theta-%data% (i) ((1 *)) theta-%offset%)
           thetamax)
        (setf thetamax
                (f2cl-lib:fref theta-%data% (i) ((1 *))
                               theta-%offset%)))
    (if (< (f2cl-lib:fref theta-%data% (i) ((1 *)) theta-%offset%)
           thetamin)
        (setf thetamin
                (f2cl-lib:fref theta-%data% (i) ((1 *))
                               theta-%offset%)))
   label100010))
(cond
 ((> thetamax (+ piover2 (- thresh))) (setf mu zero) (setf nu one))
 ((< thetamin thresh) (setf mu one) (setf nu zero))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slas2
       (f2cl-lib:array-slice b11d-%data% single-float
                             ((+ imax (f2cl-lib:int-sub 1))) ((1 *))
                             b11d-%offset%)
       (f2cl-lib:array-slice b11e-%data% single-float
                             ((+ imax (f2cl-lib:int-sub 1))) ((1 *))
                             b11e-%offset%)
       (f2cl-lib:array-slice b11d-%data% single-float (imax) ((1 *))
                             b11d-%offset%)
       sigma11 dummy)
    (declare (ignore var-0 var-1 var-2))
    (when var-3 (setf sigma11 var-3))
    (when var-4 (setf dummy var-4)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slas2
```

```
        (f2cl-lib:array-slice b21d-%data% single-float
                              ((+ imax (f2cl-lib:int-sub 1))) ((1 *))
                              b21d-%offset%)
        (f2cl-lib:array-slice b21e-%data% single-float
                              ((+ imax (f2cl-lib:int-sub 1))) ((1 *))
                              b21e-%offset%)
        (f2cl-lib:array-slice b21d-%data% single-float (imax) ((1 *))
                              b21d-%offset%)
        sigma21 dummy)
     (declare (ignore var-0 var-1 var-2))
     (when var-3 (setf sigma21 var-3))
     (when var-4 (setf dummy var-4)))
   (cond
    ((<= sigma11 sigma21) (setf mu sigma11)
     (setf nu (f2cl-lib:fsqrt (- one (expt mu 2))))
     (cond ((< mu thresh) (setf mu zero) (setf nu one))))
    (t (setf nu sigma21) (setf mu (f2cl-lib:fsqrt (- 1.0 (expt nu 2))))
     (cond ((< nu thresh) (setf mu one) (setf nu zero)))))))
(cond
 ((<= mu nu)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgs
       (f2cl-lib:array-slice b11d-%data% single-float (imin) ((1 *))
                             b11d-%offset%)
       (f2cl-lib:array-slice b11e-%data% single-float (imin) ((1 *))
                             b11e-%offset%)
       mu
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iv1tcs imin (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iv1tsn imin (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%))
     (declare (ignore var-0 var-1 var-3 var-4))
     (when var-2 (setf mu var-2))))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgs
       (f2cl-lib:array-slice b21d-%data% single-float (imin) ((1 *))
                             b21d-%offset%)
       (f2cl-lib:array-slice b21e-%data% single-float (imin) ((1 *))
                             b21e-%offset%)
       nu
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iv1tcs imin (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iv1tsn imin (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%))
     (declare (ignore var-0 var-1 var-3 var-4))
```

```
    (when var-2 (setf nu var-2)))))
(setf temp
        (+
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv1tcs imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b11d-%data% (imin) ((1 *)) b11d-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv1tsn imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b11e-%data% (imin) ((1 *)) b11e-%offset%))))
(setf (f2cl-lib:fref b11e-%data% (imin) ((1 *)) b11e-%offset%)
        (-
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv1tcs imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b11e-%data% (imin) ((1 *)) b11e-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv1tsn imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b11d-%data% (imin) ((1 *)) b11d-%offset%))))
(setf (f2cl-lib:fref b11d-%data% (imin) ((1 *)) b11d-%offset%) temp)
(setf b11bulge
        (*
         (f2cl-lib:fref rwork-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add iv1tsn imin) 1))
                        ((1 *)) rwork-%offset%)
         (f2cl-lib:fref b11d-%data% ((f2cl-lib:int-add imin 1)) ((1 *))
                        b11d-%offset%)))
(setf (f2cl-lib:fref b11d-%data% ((f2cl-lib:int-add imin 1)) ((1 *))
                     b11d-%offset%)
        (*
         (f2cl-lib:fref rwork-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add iv1tcs imin) 1))
                        ((1 *)) rwork-%offset%)
         (f2cl-lib:fref b11d-%data% ((f2cl-lib:int-add imin 1)) ((1 *))
                        b11d-%offset%)))
(setf temp
        (+
         (*
```

```
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv1tcs imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b21d-%data% (imin) ((1 *)) b21d-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv1tsn imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b21e-%data% (imin) ((1 *)) b21e-%offset%))))
(setf (f2cl-lib:fref b21e-%data% (imin) ((1 *)) b21e-%offset%)
        (-
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv1tcs imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b21e-%data% (imin) ((1 *)) b21e-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv1tsn imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b21d-%data% (imin) ((1 *)) b21d-%offset%))))
(setf (f2cl-lib:fref b21d-%data% (imin) ((1 *)) b21d-%offset%) temp)
(setf b21bulge
        (*
         (f2cl-lib:fref rwork-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add iv1tsn imin) 1))
                        ((1 *)) rwork-%offset%)
         (f2cl-lib:fref b21d-%data% ((f2cl-lib:int-add imin 1)) ((1 *))
                        b21d-%offset%)))
(setf (f2cl-lib:fref b21d-%data% ((f2cl-lib:int-add imin 1)) ((1 *))
                     b21d-%offset%)
        (*
         (f2cl-lib:fref rwork-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add iv1tcs imin) 1))
                        ((1 *)) rwork-%offset%)
         (f2cl-lib:fref b21d-%data% ((f2cl-lib:int-add imin 1)) ((1 *))
                        b21d-%offset%)))
(setf (f2cl-lib:fref theta-%data% (imin) ((1 *)) theta-%offset%)
        (f2cl-lib:atan2
         (f2cl-lib:fsqrt
          (+
           (expt
            (f2cl-lib:fref b21d-%data% (imin) ((1 *)) b21d-%offset%) 2)
           (expt b21bulge 2)))
```

```
          (f2cl-lib:fsqrt
           (+
            (expt
             (f2cl-lib:fref b11d-%data% (imin) ((1 *)) b11d-%offset%) 2)
            (expt b11bulge 2)))))
(cond
 ((> (+ (expt (f2cl-lib:fref b11d (imin) ((1 *))) 2) (expt b11bulge 2))
     (expt thresh 2))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgp b11bulge
       (f2cl-lib:array-slice b11d-%data% single-float (imin) ((1 *))
                             b11d-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iu1sn imin (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iu1cs imin (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       r)
    (declare (ignore var-1 var-2 var-3))
    (when var-0 (setf b11bulge var-0))
    (when var-4 (setf r var-4))))
 ((<= mu nu)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgs
       (f2cl-lib:array-slice b11e-%data% single-float (imin) ((1 *))
                             b11e-%offset%)
       (f2cl-lib:array-slice b11d-%data% single-float ((+ imin 1))
                             ((1 *)) b11d-%offset%)
       mu
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iu1cs imin (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iu1sn imin (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%))
    (declare (ignore var-0 var-1 var-3 var-4))
    (when var-2 (setf mu var-2))))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgs
       (f2cl-lib:array-slice b12d-%data% single-float (imin) ((1 *))
                             b12d-%offset%)
       (f2cl-lib:array-slice b12e-%data% single-float (imin) ((1 *))
                             b12e-%offset%)
       nu
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iu1cs imin (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
```

```
                              ((+ iu1sn imin (f2cl-lib:int-sub 1)))
                              ((1 *)) rwork-%offset%))
    (declare (ignore var-0 var-1 var-3 var-4))
    (when var-2 (setf nu var-2)))))
(cond
 ((> (+ (expt (f2cl-lib:fref b21d (imin) ((1 *))) 2) (expt b21bulge 2))
     (expt thresh 2))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgp b21bulge
       (f2cl-lib:array-slice b21d-%data% single-float (imin) ((1 *))
                              b21d-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                              ((+ iu2sn imin (f2cl-lib:int-sub 1)))
                              ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                              ((+ iu2cs imin (f2cl-lib:int-sub 1)))
                              ((1 *)) rwork-%offset%)
       r)
    (declare (ignore var-1 var-2 var-3))
    (when var-0 (setf b21bulge var-0))
    (when var-4 (setf r var-4))))
 ((< nu mu)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgs
       (f2cl-lib:array-slice b21e-%data% single-float (imin) ((1 *))
                              b21e-%offset%)
       (f2cl-lib:array-slice b21d-%data% single-float ((+ imin 1))
                              ((1 *)) b21d-%offset%)
       nu
       (f2cl-lib:array-slice rwork-%data% single-float
                              ((+ iu2cs imin (f2cl-lib:int-sub 1)))
                              ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                              ((+ iu2sn imin (f2cl-lib:int-sub 1)))
                              ((1 *)) rwork-%offset%))
    (declare (ignore var-0 var-1 var-3 var-4))
    (when var-2 (setf nu var-2))))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgs
       (f2cl-lib:array-slice b22d-%data% single-float (imin) ((1 *))
                              b22d-%offset%)
       (f2cl-lib:array-slice b22e-%data% single-float (imin) ((1 *))
                              b22e-%offset%)
       mu
       (f2cl-lib:array-slice rwork-%data% single-float
                              ((+ iu2cs imin (f2cl-lib:int-sub 1)))
                              ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                              ((+ iu2sn imin (f2cl-lib:int-sub 1)))
```

```
                              ((1 *)) rwork-%offset%))
    (declare (ignore var-0 var-1 var-3 var-4))
    (when var-2 (setf mu var-2)))))
(setf (f2cl-lib:fref rwork-%data%
                     ((f2cl-lib:int-sub (f2cl-lib:int-add iu2cs imin)
                                        1))
                     ((1 *)) rwork-%offset%)
        (-
         (f2cl-lib:fref rwork-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add iu2cs imin) 1))
                        ((1 *)) rwork-%offset%)))
(setf (f2cl-lib:fref rwork-%data%
                     ((f2cl-lib:int-sub (f2cl-lib:int-add iu2sn imin)
                                        1))
                     ((1 *)) rwork-%offset%)
        (-
         (f2cl-lib:fref rwork-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add iu2sn imin) 1))
                        ((1 *)) rwork-%offset%)))
(setf temp
        (+
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu1cs imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b11e-%data% (imin) ((1 *)) b11e-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu1sn imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b11d-%data% ((f2cl-lib:int-add imin 1))
                         ((1 *)) b11d-%offset%))))
(setf (f2cl-lib:fref b11d-%data% ((f2cl-lib:int-add imin 1)) ((1 *))
                     b11d-%offset%)
        (-
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu1cs imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b11d-%data% ((f2cl-lib:int-add imin 1))
                         ((1 *)) b11d-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu1sn imin) 1))
```

```
                          ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b11e-%data% (imin) ((1 *)) b11e-%offset%))))
(setf (f2cl-lib:fref b11e-%data% (imin) ((1 *)) b11e-%offset%) temp)
(cond
 ((> imax (f2cl-lib:int-add imin 1))
  (setf b11bulge
          (*
           (f2cl-lib:fref rwork-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add iu1sn imin) 1))
                          ((1 *)) rwork-%offset%)
           (f2cl-lib:fref b11e-%data% ((f2cl-lib:int-add imin 1))
                          ((1 *)) b11e-%offset%)))
  (setf (f2cl-lib:fref b11e-%data% ((f2cl-lib:int-add imin 1)) ((1 *))
                       b11e-%offset%)
          (*
           (f2cl-lib:fref rwork-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add iu1cs imin) 1))
                          ((1 *)) rwork-%offset%)
           (f2cl-lib:fref b11e-%data% ((f2cl-lib:int-add imin 1))
                          ((1 *)) b11e-%offset%)))))
(setf temp
        (+
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu1cs imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b12d-%data% (imin) ((1 *)) b12d-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu1sn imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b12e-%data% (imin) ((1 *)) b12e-%offset%))))
(setf (f2cl-lib:fref b12e-%data% (imin) ((1 *)) b12e-%offset%)
        (-
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu1cs imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b12e-%data% (imin) ((1 *)) b12e-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu1sn imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b12d-%data% (imin) ((1 *)) b12d-%offset%))))
```

```
(setf (f2cl-lib:fref b12d-%data% (imin) ((1 *)) b12d-%offset%) temp)
(setf b12bulge
        (*
         (f2cl-lib:fref rwork-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add iu1sn imin) 1))
                        ((1 *)) rwork-%offset%)
         (f2cl-lib:fref b12d-%data% ((f2cl-lib:int-add imin 1)) ((1 *))
                        b12d-%offset%)))
(setf (f2cl-lib:fref b12d-%data% ((f2cl-lib:int-add imin 1)) ((1 *))
                     b12d-%offset%)
        (*
         (f2cl-lib:fref rwork-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add iu1cs imin) 1))
                        ((1 *)) rwork-%offset%)
         (f2cl-lib:fref b12d-%data% ((f2cl-lib:int-add imin 1)) ((1 *))
                        b12d-%offset%)))
(setf temp
        (+
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu2cs imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b21e-%data% (imin) ((1 *)) b21e-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu2sn imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b21d-%data% ((f2cl-lib:int-add imin 1))
                         ((1 *)) b21d-%offset%))))
(setf (f2cl-lib:fref b21d-%data% ((f2cl-lib:int-add imin 1)) ((1 *))
                     b21d-%offset%)
        (-
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu2cs imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b21d-%data% ((f2cl-lib:int-add imin 1))
                         ((1 *)) b21d-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu2sn imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b21e-%data% (imin) ((1 *)) b21e-%offset%))))
(setf (f2cl-lib:fref b21e-%data% (imin) ((1 *)) b21e-%offset%) temp)
```

```
(cond
 ((> imax (f2cl-lib:int-add imin 1))
  (setf b21bulge
          (*
           (f2cl-lib:fref rwork-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add iu2sn imin) 1))
                          ((1 *)) rwork-%offset%)
           (f2cl-lib:fref b21e-%data% ((f2cl-lib:int-add imin 1))
                          ((1 *)) b21e-%offset%)))
  (setf (f2cl-lib:fref b21e-%data% ((f2cl-lib:int-add imin 1)) ((1 *))
                       b21e-%offset%)
          (*
           (f2cl-lib:fref rwork-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add iu2cs imin) 1))
                          ((1 *)) rwork-%offset%)
           (f2cl-lib:fref b21e-%data% ((f2cl-lib:int-add imin 1))
                          ((1 *)) b21e-%offset%)))))
(setf temp
        (+
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu2cs imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b22d-%data% (imin) ((1 *)) b22d-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu2sn imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b22e-%data% (imin) ((1 *)) b22e-%offset%))))
(setf (f2cl-lib:fref b22e-%data% (imin) ((1 *)) b22e-%offset%)
        (-
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu2cs imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b22e-%data% (imin) ((1 *)) b22e-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu2sn imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b22d-%data% (imin) ((1 *)) b22d-%offset%))))
(setf (f2cl-lib:fref b22d-%data% (imin) ((1 *)) b22d-%offset%) temp)
(setf b22bulge
        (*
```

```
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu2sn imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b22d-%data% ((f2cl-lib:int-add imin 1)) ((1 *))
                         b22d-%offset%)))
(setf (f2cl-lib:fref b22d-%data% ((f2cl-lib:int-add imin 1)) ((1 *))
                     b22d-%offset%)
        (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu2cs imin) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b22d-%data% ((f2cl-lib:int-add imin 1)) ((1 *))
                         b22d-%offset%)))
(f2cl-lib:fdo (i (f2cl-lib:int-add imin 1) (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add imax (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (setf x1
            (+
             (*
              (sin
               (f2cl-lib:fref theta-%data% ((f2cl-lib:int-sub i 1))
                              ((1 *)) theta-%offset%))
              (f2cl-lib:fref b11e-%data% ((f2cl-lib:int-sub i 1))
                             ((1 *)) b11e-%offset%))
             (*
              (cos
               (f2cl-lib:fref theta-%data% ((f2cl-lib:int-sub i 1))
                              ((1 *)) theta-%offset%))
              (f2cl-lib:fref b21e-%data% ((f2cl-lib:int-sub i 1))
                             ((1 *)) b21e-%offset%))))
    (setf x2
            (+
             (*
              (sin
               (f2cl-lib:fref theta-%data% ((f2cl-lib:int-sub i 1))
                              ((1 *)) theta-%offset%))
              b11bulge)
             (*
              (cos
               (f2cl-lib:fref theta-%data% ((f2cl-lib:int-sub i 1))
                              ((1 *)) theta-%offset%))
              b21bulge)))
    (setf y1
            (+
             (*
              (sin
               (f2cl-lib:fref theta-%data% ((f2cl-lib:int-sub i 1))
                              ((1 *)) theta-%offset%))
```

```
          (f2cl-lib:fref b12d-%data% ((f2cl-lib:int-sub i 1))
                         ((1 *)) b12d-%offset%))
         (*
          (cos
           (f2cl-lib:fref theta-%data% ((f2cl-lib:int-sub i 1))
                          ((1 *)) theta-%offset%))
          (f2cl-lib:fref b22d-%data% ((f2cl-lib:int-sub i 1))
                         ((1 *)) b22d-%offset%))))
(setf y2
        (+
         (*
          (sin
           (f2cl-lib:fref theta-%data% ((f2cl-lib:int-sub i 1))
                          ((1 *)) theta-%offset%))
          b12bulge)
         (*
          (cos
           (f2cl-lib:fref theta-%data% ((f2cl-lib:int-sub i 1))
                          ((1 *)) theta-%offset%))
          b22bulge)))
(setf (f2cl-lib:fref phi-%data% ((f2cl-lib:int-sub i 1)) ((1 *))
                     phi-%offset%)
        (f2cl-lib:atan2
         (f2cl-lib:fsqrt (+ (expt x1 2) (expt x2 2)))
         (f2cl-lib:fsqrt (+ (expt y1 2) (expt y2 2)))))
(setf restart11
        (<=
         (+
          (expt
           (f2cl-lib:fref b11e-%data% ((f2cl-lib:int-sub i 1))
                          ((1 *)) b11e-%offset%)
           2)
          (expt b11bulge 2))
         (expt thresh 2)))
(setf restart21
        (<=
         (+
          (expt
           (f2cl-lib:fref b21e-%data% ((f2cl-lib:int-sub i 1))
                          ((1 *)) b21e-%offset%)
           2)
          (expt b21bulge 2))
         (expt thresh 2)))
(setf restart12
        (<=
         (+
          (expt
           (f2cl-lib:fref b12d-%data% ((f2cl-lib:int-sub i 1))
                          ((1 *)) b12d-%offset%)
           2)
```

```
          (expt b12bulge 2))
         (expt thresh 2)))
(setf restart22
        (<=
         (+
          (expt
           (f2cl-lib:fref b22d-%data% ((f2cl-lib:int-sub i 1))
                          ((1 *)) b22d-%offset%)
           2)
          (expt b22bulge 2))
         (expt thresh 2)))
(cond
 ((and (not restart11) (not restart21))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgp x2 x1
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iv1tsn i (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iv1tcs i (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       r)
    (declare (ignore var-2 var-3))
    (when var-0 (setf x2 var-0))
    (when var-1 (setf x1 var-1))
    (when var-4 (setf r var-4))))
 ((and (not restart11) restart21)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgp b11bulge
       (f2cl-lib:array-slice b11e-%data% single-float
                             ((+ i (f2cl-lib:int-sub 1))) ((1 *))
                             b11e-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iv1tsn i (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iv1tcs i (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       r)
    (declare (ignore var-1 var-2 var-3))
    (when var-0 (setf b11bulge var-0))
    (when var-4 (setf r var-4))))
 ((and restart11 (not restart21))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgp b21bulge
       (f2cl-lib:array-slice b21e-%data% single-float
                             ((+ i (f2cl-lib:int-sub 1))) ((1 *))
                             b21e-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iv1tsn i (f2cl-lib:int-sub 1)))
```

```
                                ((1 *)) rwork-%offset%)
          (f2cl-lib:array-slice rwork-%data% single-float
                                ((+ iv1tcs i (f2cl-lib:int-sub 1)))
                                ((1 *)) rwork-%offset%)
          r)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf b21bulge var-0))
      (when var-4 (setf r var-4))))
   ((<= mu nu)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (slartgs
          (f2cl-lib:array-slice b11d-%data% single-float (i) ((1 *))
                                b11d-%offset%)
          (f2cl-lib:array-slice b11e-%data% single-float (i) ((1 *))
                                b11e-%offset%)
          mu
          (f2cl-lib:array-slice rwork-%data% single-float
                                ((+ iv1tcs i (f2cl-lib:int-sub 1)))
                                ((1 *)) rwork-%offset%)
          (f2cl-lib:array-slice rwork-%data% single-float
                                ((+ iv1tsn i (f2cl-lib:int-sub 1)))
                                ((1 *)) rwork-%offset%))
      (declare (ignore var-0 var-1 var-3 var-4))
      (when var-2 (setf mu var-2))))
   (t
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (slartgs
          (f2cl-lib:array-slice b21d-%data% single-float (i) ((1 *))
                                b21d-%offset%)
          (f2cl-lib:array-slice b21e-%data% single-float (i) ((1 *))
                                b21e-%offset%)
          nu
          (f2cl-lib:array-slice rwork-%data% single-float
                                ((+ iv1tcs i (f2cl-lib:int-sub 1)))
                                ((1 *)) rwork-%offset%)
          (f2cl-lib:array-slice rwork-%data% single-float
                                ((+ iv1tsn i (f2cl-lib:int-sub 1)))
                                ((1 *)) rwork-%offset%))
      (declare (ignore var-0 var-1 var-3 var-4))
      (when var-2 (setf nu var-2)))))
  (setf (f2cl-lib:fref rwork-%data%
                       ((f2cl-lib:int-sub (f2cl-lib:int-add iv1tcs i)
                                          1))
                       ((1 *)) rwork-%offset%)
         (-
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv1tcs i) 1))
                         ((1 *)) rwork-%offset%)))
  (setf (f2cl-lib:fref rwork-%data%
```

```
                        ((f2cl-lib:int-sub (f2cl-lib:int-add iv1tsn i)
                                           1))
                        ((1 *)) rwork-%offset%)
        (-
         (f2cl-lib:fref rwork-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add iv1tsn i) 1))
                        ((1 *)) rwork-%offset%)))
(cond
 ((and (not restart12) (not restart22))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgp y2 y1
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iv2tsn i (f2cl-lib:int-sub 1)
                                 (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iv2tcs i (f2cl-lib:int-sub 1)
                                 (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       r)
    (declare (ignore var-2 var-3))
    (when var-0 (setf y2 var-0))
    (when var-1 (setf y1 var-1))
    (when var-4 (setf r var-4))))
 ((and (not restart12) restart22)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgp b12bulge
       (f2cl-lib:array-slice b12d-%data% single-float
                             ((+ i (f2cl-lib:int-sub 1))) ((1 *))
                             b12d-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iv2tsn i (f2cl-lib:int-sub 1)
                                 (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iv2tcs i (f2cl-lib:int-sub 1)
                                 (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       r)
    (declare (ignore var-1 var-2 var-3))
    (when var-0 (setf b12bulge var-0))
    (when var-4 (setf r var-4))))
 ((and restart12 (not restart22))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgp b22bulge
       (f2cl-lib:array-slice b22d-%data% single-float
                             ((+ i (f2cl-lib:int-sub 1))) ((1 *))
                             b22d-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
```

```
                                ((+ iv2tsn i (f2cl-lib:int-sub 1)
                                    (f2cl-lib:int-sub 1)))
                                ((1 *)) rwork-%offset%)
        (f2cl-lib:array-slice rwork-%data% single-float
                                ((+ iv2tcs i (f2cl-lib:int-sub 1)
                                    (f2cl-lib:int-sub 1)))
                                ((1 *)) rwork-%offset%)
        r)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf b22bulge var-0))
      (when var-4 (setf r var-4))))
   ((< nu mu)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (slartgs
        (f2cl-lib:array-slice b12e-%data% single-float
                                ((+ i (f2cl-lib:int-sub 1))) ((1 *))
                                b12e-%offset%)
        (f2cl-lib:array-slice b12d-%data% single-float (i) ((1 *))
                                b12d-%offset%)
        nu
        (f2cl-lib:array-slice rwork-%data% single-float
                                ((+ iv2tcs i (f2cl-lib:int-sub 1)
                                    (f2cl-lib:int-sub 1)))
                                ((1 *)) rwork-%offset%)
        (f2cl-lib:array-slice rwork-%data% single-float
                                ((+ iv2tsn i (f2cl-lib:int-sub 1)
                                    (f2cl-lib:int-sub 1)))
                                ((1 *)) rwork-%offset%))
      (declare (ignore var-0 var-1 var-3 var-4))
      (when var-2 (setf nu var-2))))
   (t
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (slartgs
        (f2cl-lib:array-slice b22e-%data% single-float
                                ((+ i (f2cl-lib:int-sub 1))) ((1 *))
                                b22e-%offset%)
        (f2cl-lib:array-slice b22d-%data% single-float (i) ((1 *))
                                b22d-%offset%)
        mu
        (f2cl-lib:array-slice rwork-%data% single-float
                                ((+ iv2tcs i (f2cl-lib:int-sub 1)
                                    (f2cl-lib:int-sub 1)))
                                ((1 *)) rwork-%offset%)
        (f2cl-lib:array-slice rwork-%data% single-float
                                ((+ iv2tsn i (f2cl-lib:int-sub 1)
                                    (f2cl-lib:int-sub 1)))
                                ((1 *)) rwork-%offset%))
      (declare (ignore var-0 var-1 var-3 var-4))
      (when var-2 (setf mu var-2)))))
  (setf temp
```

```
        (+
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv1tcs i) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b11d-%data% (i) ((1 *)) b11d-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv1tsn i) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b11e-%data% (i) ((1 *)) b11e-%offset%))))
(setf (f2cl-lib:fref b11e-%data% (i) ((1 *)) b11e-%offset%)
        (-
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv1tcs i) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b11e-%data% (i) ((1 *)) b11e-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv1tsn i) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b11d-%data% (i) ((1 *)) b11d-%offset%))))
(setf (f2cl-lib:fref b11d-%data% (i) ((1 *)) b11d-%offset%) temp)
(setf b11bulge
        (*
         (f2cl-lib:fref rwork-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add iv1tsn i) 1))
                        ((1 *)) rwork-%offset%)
         (f2cl-lib:fref b11d-%data% ((f2cl-lib:int-add i 1))
                        ((1 *)) b11d-%offset%)))
(setf (f2cl-lib:fref b11d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                     b11d-%offset%)
        (*
         (f2cl-lib:fref rwork-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add iv1tcs i) 1))
                        ((1 *)) rwork-%offset%)
         (f2cl-lib:fref b11d-%data% ((f2cl-lib:int-add i 1))
                        ((1 *)) b11d-%offset%)))
(setf temp
        (+
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
```

```
                              (f2cl-lib:int-add iv1tcs i) 1))
                            ((1 *)) rwork-%offset%)
           (f2cl-lib:fref b21d-%data% (i) ((1 *)) b21d-%offset%))
          (*
           (f2cl-lib:fref rwork-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add iv1tsn i) 1))
                            ((1 *)) rwork-%offset%)
           (f2cl-lib:fref b21e-%data% (i) ((1 *)) b21e-%offset%))))
(setf (f2cl-lib:fref b21e-%data% (i) ((1 *)) b21e-%offset%)
        (-
         (*
           (f2cl-lib:fref rwork-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add iv1tcs i) 1))
                            ((1 *)) rwork-%offset%)
           (f2cl-lib:fref b21e-%data% (i) ((1 *)) b21e-%offset%))
          (*
           (f2cl-lib:fref rwork-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add iv1tsn i) 1))
                            ((1 *)) rwork-%offset%)
           (f2cl-lib:fref b21d-%data% (i) ((1 *)) b21d-%offset%))))
(setf (f2cl-lib:fref b21d-%data% (i) ((1 *)) b21d-%offset%) temp)
(setf b21bulge
        (*
         (f2cl-lib:fref rwork-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add iv1tsn i) 1))
                          ((1 *)) rwork-%offset%)
         (f2cl-lib:fref b21d-%data% ((f2cl-lib:int-add i 1))
                          ((1 *)) b21d-%offset%)))
(setf (f2cl-lib:fref b21d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                      b21d-%offset%)
        (*
         (f2cl-lib:fref rwork-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add iv1tcs i) 1))
                          ((1 *)) rwork-%offset%)
         (f2cl-lib:fref b21d-%data% ((f2cl-lib:int-add i 1))
                          ((1 *)) b21d-%offset%)))
(setf temp
        (+
         (*
           (f2cl-lib:fref rwork-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add iv2tcs i) 1 1))
                            ((1 *)) rwork-%offset%)
           (f2cl-lib:fref b12e-%data% ((f2cl-lib:int-sub i 1))
                            ((1 *)) b12e-%offset%))
```

```
          (*
           (f2cl-lib:fref rwork-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add iv2tsn i) 1 1))
                          ((1 *)) rwork-%offset%)
           (f2cl-lib:fref b12d-%data% (i) ((1 *)) b12d-%offset%))))
(setf (f2cl-lib:fref b12d-%data% (i) ((1 *)) b12d-%offset%)
        (-
          (*
           (f2cl-lib:fref rwork-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add iv2tcs i) 1 1))
                          ((1 *)) rwork-%offset%)
           (f2cl-lib:fref b12d-%data% (i) ((1 *)) b12d-%offset%))
          (*
           (f2cl-lib:fref rwork-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add iv2tsn i) 1 1))
                          ((1 *)) rwork-%offset%)
           (f2cl-lib:fref b12e-%data% ((f2cl-lib:int-sub i 1))
                          ((1 *)) b12e-%offset%))))
(setf (f2cl-lib:fref b12e-%data% ((f2cl-lib:int-sub i 1)) ((1 *))
                     b12e-%offset%)
        temp)
(setf b12bulge
        (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv2tsn i) 1 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b12e-%data% (i) ((1 *)) b12e-%offset%)))
(setf (f2cl-lib:fref b12e-%data% (i) ((1 *)) b12e-%offset%)
        (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv2tcs i) 1 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b12e-%data% (i) ((1 *)) b12e-%offset%)))
(setf temp
        (+
          (*
           (f2cl-lib:fref rwork-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add iv2tcs i) 1 1))
                          ((1 *)) rwork-%offset%)
           (f2cl-lib:fref b22e-%data% ((f2cl-lib:int-sub i 1))
                          ((1 *)) b22e-%offset%))
          (*
           (f2cl-lib:fref rwork-%data%
                          ((f2cl-lib:int-sub
```

```
                              (f2cl-lib:int-add iv2tsn i) 1 1))
                            ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b22d-%data% (i) ((1 *)) b22d-%offset%))))
(setf (f2cl-lib:fref b22d-%data% (i) ((1 *)) b22d-%offset%)
        (-
         (*
          (f2cl-lib:fref rwork-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add iv2tcs i) 1 1))
                            ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b22d-%data% (i) ((1 *)) b22d-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add iv2tsn i) 1 1))
                            ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b22e-%data% ((f2cl-lib:int-sub i 1))
                            ((1 *)) b22e-%offset%))))
(setf (f2cl-lib:fref b22e-%data% ((f2cl-lib:int-sub i 1)) ((1 *))
                      b22e-%offset%)
        temp)
(setf b22bulge
        (*
         (f2cl-lib:fref rwork-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add iv2tsn i) 1 1))
                           ((1 *)) rwork-%offset%)
         (f2cl-lib:fref b22e-%data% (i) ((1 *)) b22e-%offset%)))
(setf (f2cl-lib:fref b22e-%data% (i) ((1 *)) b22e-%offset%)
        (*
         (f2cl-lib:fref rwork-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add iv2tcs i) 1 1))
                           ((1 *)) rwork-%offset%)
         (f2cl-lib:fref b22e-%data% (i) ((1 *)) b22e-%offset%)))
(setf x1
        (+
         (*
          (cos
           (f2cl-lib:fref phi-%data% ((f2cl-lib:int-sub i 1))
                            ((1 *)) phi-%offset%))
          (f2cl-lib:fref b11d-%data% (i) ((1 *)) b11d-%offset%))
         (*
          (sin
           (f2cl-lib:fref phi-%data% ((f2cl-lib:int-sub i 1))
                            ((1 *)) phi-%offset%))
          (f2cl-lib:fref b12e-%data% ((f2cl-lib:int-sub i 1))
                            ((1 *)) b12e-%offset%))))
(setf x2
        (+
```

```
          (*
           (cos
            (f2cl-lib:fref phi-%data% ((f2cl-lib:int-sub i 1))
                           ((1 *)) phi-%offset%))
           b11bulge)
          (*
           (sin
            (f2cl-lib:fref phi-%data% ((f2cl-lib:int-sub i 1))
                           ((1 *)) phi-%offset%))
           b12bulge)))
(setf y1
        (+
         (*
          (cos
           (f2cl-lib:fref phi-%data% ((f2cl-lib:int-sub i 1))
                           ((1 *)) phi-%offset%))
          (f2cl-lib:fref b21d-%data% (i) ((1 *)) b21d-%offset%))
         (*
          (sin
           (f2cl-lib:fref phi-%data% ((f2cl-lib:int-sub i 1))
                           ((1 *)) phi-%offset%))
          (f2cl-lib:fref b22e-%data% ((f2cl-lib:int-sub i 1))
                          ((1 *)) b22e-%offset%))))
(setf y2
        (+
         (*
          (cos
           (f2cl-lib:fref phi-%data% ((f2cl-lib:int-sub i 1))
                           ((1 *)) phi-%offset%))
          b21bulge)
         (*
          (sin
           (f2cl-lib:fref phi-%data% ((f2cl-lib:int-sub i 1))
                           ((1 *)) phi-%offset%))
          b22bulge)))
(setf (f2cl-lib:fref theta-%data% (i) ((1 *)) theta-%offset%)
        (f2cl-lib:atan2
         (f2cl-lib:fsqrt (+ (expt y1 2) (expt y2 2)))
         (f2cl-lib:fsqrt (+ (expt x1 2) (expt x2 2)))))
(setf restart11
        (<=
         (+
          (expt
           (f2cl-lib:fref b11d-%data% (i) ((1 *)) b11d-%offset%) 2)
          (expt b11bulge 2))
         (expt thresh 2)))
(setf restart12
        (<=
         (+
          (expt
```

```
            (f2cl-lib:fref b12e-%data% ((f2cl-lib:int-sub i 1))
                           ((1 *)) b12e-%offset%)
            2)
           (expt b12bulge 2))
          (expt thresh 2)))
(setf restart21
        (<=
         (+
          (expt
           (f2cl-lib:fref b21d-%data% (i) ((1 *)) b21d-%offset%) 2)
          (expt b21bulge 2))
         (expt thresh 2)))
(setf restart22
        (<=
         (+
          (expt
           (f2cl-lib:fref b22e-%data% ((f2cl-lib:int-sub i 1))
                           ((1 *)) b22e-%offset%)
           2)
          (expt b22bulge 2))
         (expt thresh 2)))
(cond
 ((and (not restart11) (not restart12))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgp x2 x1
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iu1sn i (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iu1cs i (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       r)
    (declare (ignore var-2 var-3))
    (when var-0 (setf x2 var-0))
    (when var-1 (setf x1 var-1))
    (when var-4 (setf r var-4))))
 ((and (not restart11) restart12)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgp b11bulge
       (f2cl-lib:array-slice b11d-%data% single-float (i) ((1 *))
                             b11d-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iu1sn i (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iu1cs i (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       r)
    (declare (ignore var-1 var-2 var-3))
    (when var-0 (setf b11bulge var-0))
```

```
    (when var-4 (setf r var-4))))
  ((and restart11 (not restart12))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
       (slartgp b12bulge
        (f2cl-lib:array-slice b12e-%data% single-float
                              ((+ i (f2cl-lib:int-sub 1))) ((1 *))
                              b12e-%offset%)
        (f2cl-lib:array-slice rwork-%data% single-float
                              ((+ iu1sn i (f2cl-lib:int-sub 1)))
                              ((1 *)) rwork-%offset%)
        (f2cl-lib:array-slice rwork-%data% single-float
                              ((+ iu1cs i (f2cl-lib:int-sub 1)))
                              ((1 *)) rwork-%offset%)
        r)
    (declare (ignore var-1 var-2 var-3))
    (when var-0 (setf b12bulge var-0))
    (when var-4 (setf r var-4))))
  ((<= mu nu)
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
       (slartgs
        (f2cl-lib:array-slice b11e-%data% single-float (i) ((1 *))
                              b11e-%offset%)
        (f2cl-lib:array-slice b11d-%data% single-float ((+ i 1))
                              ((1 *)) b11d-%offset%)
        mu
        (f2cl-lib:array-slice rwork-%data% single-float
                              ((+ iu1cs i (f2cl-lib:int-sub 1)))
                              ((1 *)) rwork-%offset%)
        (f2cl-lib:array-slice rwork-%data% single-float
                              ((+ iu1sn i (f2cl-lib:int-sub 1)))
                              ((1 *)) rwork-%offset%))
    (declare (ignore var-0 var-1 var-3 var-4))
    (when var-2 (setf mu var-2))))
  (t
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
       (slartgs
        (f2cl-lib:array-slice b12d-%data% single-float (i) ((1 *))
                              b12d-%offset%)
        (f2cl-lib:array-slice b12e-%data% single-float (i) ((1 *))
                              b12e-%offset%)
        nu
        (f2cl-lib:array-slice rwork-%data% single-float
                              ((+ iu1cs i (f2cl-lib:int-sub 1)))
                              ((1 *)) rwork-%offset%)
        (f2cl-lib:array-slice rwork-%data% single-float
                              ((+ iu1sn i (f2cl-lib:int-sub 1)))
                              ((1 *)) rwork-%offset%))
    (declare (ignore var-0 var-1 var-3 var-4))
    (when var-2 (setf nu var-2)))))
(cond
```

```
((and (not restart21) (not restart22))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (slartgp y2 y1
      (f2cl-lib:array-slice rwork-%data% single-float
                            ((+ iu2sn i (f2cl-lib:int-sub 1)))
                            ((1 *)) rwork-%offset%)
      (f2cl-lib:array-slice rwork-%data% single-float
                            ((+ iu2cs i (f2cl-lib:int-sub 1)))
                            ((1 *)) rwork-%offset%)
      r)
   (declare (ignore var-2 var-3))
   (when var-0 (setf y2 var-0))
   (when var-1 (setf y1 var-1))
   (when var-4 (setf r var-4))))
((and (not restart21) restart22)
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (slartgp b21bulge
      (f2cl-lib:array-slice b21d-%data% single-float (i) ((1 *))
                            b21d-%offset%)
      (f2cl-lib:array-slice rwork-%data% single-float
                            ((+ iu2sn i (f2cl-lib:int-sub 1)))
                            ((1 *)) rwork-%offset%)
      (f2cl-lib:array-slice rwork-%data% single-float
                            ((+ iu2cs i (f2cl-lib:int-sub 1)))
                            ((1 *)) rwork-%offset%)
      r)
   (declare (ignore var-1 var-2 var-3))
   (when var-0 (setf b21bulge var-0))
   (when var-4 (setf r var-4))))
((and restart21 (not restart22))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (slartgp b22bulge
      (f2cl-lib:array-slice b22e-%data% single-float
                            ((+ i (f2cl-lib:int-sub 1))) ((1 *))
                            b22e-%offset%)
      (f2cl-lib:array-slice rwork-%data% single-float
                            ((+ iu2sn i (f2cl-lib:int-sub 1)))
                            ((1 *)) rwork-%offset%)
      (f2cl-lib:array-slice rwork-%data% single-float
                            ((+ iu2cs i (f2cl-lib:int-sub 1)))
                            ((1 *)) rwork-%offset%)
      r)
   (declare (ignore var-1 var-2 var-3))
   (when var-0 (setf b22bulge var-0))
   (when var-4 (setf r var-4))))
((< nu mu)
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (slartgs
      (f2cl-lib:array-slice b21e-%data% single-float (i) ((1 *))
                            b21e-%offset%)
```

```
        (f2cl-lib:array-slice b21e-%data% single-float ((+ i 1))
                              ((1 *)) b21e-%offset%)
        nu
        (f2cl-lib:array-slice rwork-%data% single-float
                              ((+ iu2cs i (f2cl-lib:int-sub 1)))
                              ((1 *)) rwork-%offset%)
        (f2cl-lib:array-slice rwork-%data% single-float
                              ((+ iu2sn i (f2cl-lib:int-sub 1)))
                              ((1 *)) rwork-%offset%))
    (declare (ignore var-0 var-1 var-3 var-4))
    (when var-2 (setf nu var-2))))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgs
        (f2cl-lib:array-slice b22d-%data% single-float (i) ((1 *))
                              b22d-%offset%)
        (f2cl-lib:array-slice b22e-%data% single-float (i) ((1 *))
                              b22e-%offset%)
        mu
        (f2cl-lib:array-slice rwork-%data% single-float
                              ((+ iu2cs i (f2cl-lib:int-sub 1)))
                              ((1 *)) rwork-%offset%)
        (f2cl-lib:array-slice rwork-%data% single-float
                              ((+ iu2sn i (f2cl-lib:int-sub 1)))
                              ((1 *)) rwork-%offset%))
    (declare (ignore var-0 var-1 var-3 var-4))
    (when var-2 (setf mu var-2)))))
(setf (f2cl-lib:fref rwork-%data%
                     ((f2cl-lib:int-sub (f2cl-lib:int-add iu2cs i)
                                        1))
                     ((1 *)) rwork-%offset%)
        (-
         (f2cl-lib:fref rwork-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add iu2cs i) 1))
                        ((1 *)) rwork-%offset%)))
(setf (f2cl-lib:fref rwork-%data%
                     ((f2cl-lib:int-sub (f2cl-lib:int-add iu2sn i)
                                        1))
                     ((1 *)) rwork-%offset%)
        (-
         (f2cl-lib:fref rwork-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add iu2sn i) 1))
                        ((1 *)) rwork-%offset%)))
(setf temp
        (+
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
```

```
                              (f2cl-lib:int-add iu1cs i) 1))
                            ((1 *)) rwork-%offset%)
           (f2cl-lib:fref b11e-%data% (i) ((1 *)) b11e-%offset%))
          (*
           (f2cl-lib:fref rwork-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add iu1sn i) 1))
                            ((1 *)) rwork-%offset%)
           (f2cl-lib:fref b11d-%data% ((f2cl-lib:int-add i 1))
                            ((1 *)) b11d-%offset%))))
(setf (f2cl-lib:fref b11d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                     b11d-%offset%)
        (-
         (*
          (f2cl-lib:fref rwork-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add iu1cs i) 1))
                            ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b11d-%data% ((f2cl-lib:int-add i 1))
                            ((1 *)) b11d-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add iu1sn i) 1))
                            ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b11e-%data% (i) ((1 *)) b11e-%offset%))))
(setf (f2cl-lib:fref b11e-%data% (i) ((1 *)) b11e-%offset%) temp)
(cond
 ((< i (f2cl-lib:int-add imax (f2cl-lib:int-sub 1)))
  (setf b11bulge
          (*
           (f2cl-lib:fref rwork-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add iu1sn i) 1))
                            ((1 *)) rwork-%offset%)
           (f2cl-lib:fref b11e-%data% ((f2cl-lib:int-add i 1))
                            ((1 *)) b11e-%offset%)))
  (setf (f2cl-lib:fref b11e-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                       b11e-%offset%)
          (*
           (f2cl-lib:fref rwork-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add iu1cs i) 1))
                            ((1 *)) rwork-%offset%)
           (f2cl-lib:fref b11e-%data% ((f2cl-lib:int-add i 1))
                            ((1 *)) b11e-%offset%)))))
(setf temp
        (+
         (*
          (f2cl-lib:fref rwork-%data%
```

```
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add iu2cs i) 1))
                              ((1 *)) rwork-%offset%)
                (f2cl-lib:fref b21e-%data% (i) ((1 *)) b21e-%offset%))
               (*
                (f2cl-lib:fref rwork-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add iu2sn i) 1))
                              ((1 *)) rwork-%offset%)
                (f2cl-lib:fref b21d-%data% ((f2cl-lib:int-add i 1))
                              ((1 *)) b21d-%offset%))))
(setf (f2cl-lib:fref b21d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                      b21d-%offset%)
        (-
         (*
          (f2cl-lib:fref rwork-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add iu2cs i) 1))
                              ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b21d-%data% ((f2cl-lib:int-add i 1))
                              ((1 *)) b21d-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add iu2sn i) 1))
                              ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b21e-%data% (i) ((1 *)) b21e-%offset%))))
(setf (f2cl-lib:fref b21e-%data% (i) ((1 *)) b21e-%offset%) temp)
(cond
 ((< i (f2cl-lib:int-add imax (f2cl-lib:int-sub 1)))
  (setf b21bulge
          (*
           (f2cl-lib:fref rwork-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add iu2sn i) 1))
                              ((1 *)) rwork-%offset%)
           (f2cl-lib:fref b21e-%data% ((f2cl-lib:int-add i 1))
                              ((1 *)) b21e-%offset%)))
  (setf (f2cl-lib:fref b21e-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                        b21e-%offset%)
          (*
           (f2cl-lib:fref rwork-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-add iu2cs i) 1))
                              ((1 *)) rwork-%offset%)
           (f2cl-lib:fref b21e-%data% ((f2cl-lib:int-add i 1))
                              ((1 *)) b21e-%offset%)))))
(setf temp
        (+
         (*
```

```
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu1cs i) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b12d-%data% (i) ((1 *)) b12d-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu1sn i) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b12e-%data% (i) ((1 *)) b12e-%offset%))))
(setf (f2cl-lib:fref b12e-%data% (i) ((1 *)) b12e-%offset%)
        (-
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu1cs i) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b12e-%data% (i) ((1 *)) b12e-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu1sn i) 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b12d-%data% (i) ((1 *)) b12d-%offset%))))
(setf (f2cl-lib:fref b12d-%data% (i) ((1 *)) b12d-%offset%) temp)
(setf b12bulge
        (*
         (f2cl-lib:fref rwork-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add iu1sn i) 1))
                        ((1 *)) rwork-%offset%)
         (f2cl-lib:fref b12d-%data% ((f2cl-lib:int-add i 1))
                        ((1 *)) b12d-%offset%)))
(setf (f2cl-lib:fref b12d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                     b12d-%offset%)
        (*
         (f2cl-lib:fref rwork-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add iu1cs i) 1))
                        ((1 *)) rwork-%offset%)
         (f2cl-lib:fref b12d-%data% ((f2cl-lib:int-add i 1))
                        ((1 *)) b12d-%offset%)))
(setf temp
        (+
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iu2cs i) 1))
                         ((1 *)) rwork-%offset%)
```

```
                 (f2cl-lib:fref b22d-%data% (i) ((1 *)) b22d-%offset%))
                (*
                 (f2cl-lib:fref rwork-%data%
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-add iu2sn i) 1))
                                ((1 *)) rwork-%offset%)
                 (f2cl-lib:fref b22e-%data% (i) ((1 *)) b22e-%offset%))))
    (setf (f2cl-lib:fref b22e-%data% (i) ((1 *)) b22e-%offset%)
            (-
             (*
              (f2cl-lib:fref rwork-%data%
                             ((f2cl-lib:int-sub
                               (f2cl-lib:int-add iu2cs i) 1))
                             ((1 *)) rwork-%offset%)
              (f2cl-lib:fref b22e-%data% (i) ((1 *)) b22e-%offset%))
             (*
              (f2cl-lib:fref rwork-%data%
                             ((f2cl-lib:int-sub
                               (f2cl-lib:int-add iu2sn i) 1))
                             ((1 *)) rwork-%offset%)
              (f2cl-lib:fref b22d-%data% (i) ((1 *)) b22d-%offset%))))
    (setf (f2cl-lib:fref b22d-%data% (i) ((1 *)) b22d-%offset%) temp)
    (setf b22bulge
            (*
             (f2cl-lib:fref rwork-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add iu2sn i) 1))
                            ((1 *)) rwork-%offset%)
             (f2cl-lib:fref b22d-%data% ((f2cl-lib:int-add i 1))
                            ((1 *)) b22d-%offset%)))
    (setf (f2cl-lib:fref b22d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         b22d-%offset%)
            (*
             (f2cl-lib:fref rwork-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add iu2cs i) 1))
                            ((1 *)) rwork-%offset%)
             (f2cl-lib:fref b22d-%data% ((f2cl-lib:int-add i 1))
                            ((1 *)) b22d-%offset%)))
   label100011))
(setf x1
        (+
         (*
          (sin
           (f2cl-lib:fref theta-%data% ((f2cl-lib:int-sub imax 1))
                          ((1 *)) theta-%offset%))
          (f2cl-lib:fref b11e-%data% ((f2cl-lib:int-sub imax 1))
                         ((1 *)) b11e-%offset%))
         (*
          (cos
```

```
            (f2cl-lib:fref theta-%data% ((f2cl-lib:int-sub imax 1))
                           ((1 *)) theta-%offset%))
           (f2cl-lib:fref b21e-%data% ((f2cl-lib:int-sub imax 1))
                          ((1 *)) b21e-%offset%))))
(setf y1
        (+
         (*
          (sin
           (f2cl-lib:fref theta-%data% ((f2cl-lib:int-sub imax 1))
                           ((1 *)) theta-%offset%))
           (f2cl-lib:fref b12d-%data% ((f2cl-lib:int-sub imax 1))
                          ((1 *)) b12d-%offset%))
         (*
          (cos
           (f2cl-lib:fref theta-%data% ((f2cl-lib:int-sub imax 1))
                           ((1 *)) theta-%offset%))
           (f2cl-lib:fref b22d-%data% ((f2cl-lib:int-sub imax 1))
                          ((1 *)) b22d-%offset%))))
(setf y2
        (+
         (*
          (sin
           (f2cl-lib:fref theta-%data% ((f2cl-lib:int-sub imax 1))
                           ((1 *)) theta-%offset%))
          b12bulge)
         (*
          (cos
           (f2cl-lib:fref theta-%data% ((f2cl-lib:int-sub imax 1))
                           ((1 *)) theta-%offset%))
          b22bulge)))
(setf (f2cl-lib:fref phi-%data% ((f2cl-lib:int-sub imax 1)) ((1 *))
                     phi-%offset%)
        (f2cl-lib:atan2 (abs x1)
                        (f2cl-lib:fsqrt (+ (expt y1 2) (expt y2 2)))))
(setf restart12
        (<=
         (+
          (expt
           (f2cl-lib:fref b12d-%data% ((f2cl-lib:int-sub imax 1))
                           ((1 *)) b12d-%offset%)
           2)
          (expt b12bulge 2))
         (expt thresh 2)))
(setf restart22
        (<=
         (+
          (expt
           (f2cl-lib:fref b22d-%data% ((f2cl-lib:int-sub imax 1))
                           ((1 *)) b22d-%offset%)
           2)
```

```
          (expt b22bulge 2))
         (expt thresh 2)))
(cond
 ((and (not restart12) (not restart22))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgp y2 y1
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iv2tsn imax (f2cl-lib:int-sub 1)
                                 (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iv2tcs imax (f2cl-lib:int-sub 1)
                                 (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       r)
    (declare (ignore var-2 var-3))
    (when var-0 (setf y2 var-0))
    (when var-1 (setf y1 var-1))
    (when var-4 (setf r var-4))))
 ((and (not restart12) restart22)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgp b12bulge
       (f2cl-lib:array-slice b12d-%data% single-float
                             ((+ imax (f2cl-lib:int-sub 1))) ((1 *))
                             b12d-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iv2tsn imax (f2cl-lib:int-sub 1)
                                 (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iv2tcs imax (f2cl-lib:int-sub 1)
                                 (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       r)
    (declare (ignore var-1 var-2 var-3))
    (when var-0 (setf b12bulge var-0))
    (when var-4 (setf r var-4))))
 ((and restart12 (not restart22))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slartgp b22bulge
       (f2cl-lib:array-slice b22d-%data% single-float
                             ((+ imax (f2cl-lib:int-sub 1))) ((1 *))
                             b22d-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iv2tsn imax (f2cl-lib:int-sub 1)
                                 (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ iv2tcs imax (f2cl-lib:int-sub 1)
                                 (f2cl-lib:int-sub 1)))
```

```
                              ((1 *)) rwork-%offset%)
        r)
      (declare (ignore var-1 var-2 var-3))
      (when var-0 (setf b22bulge var-0))
      (when var-4 (setf r var-4))))
   ((< nu mu)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (slartgs
         (f2cl-lib:array-slice b12e-%data% single-float
                               ((+ imax (f2cl-lib:int-sub 1))) ((1 *))
                               b12e-%offset%)
         (f2cl-lib:array-slice b12d-%data% single-float (imax) ((1 *))
                               b12d-%offset%)
         nu
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ iv2tcs imax (f2cl-lib:int-sub 1)
                                   (f2cl-lib:int-sub 1)))
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ iv2tsn imax (f2cl-lib:int-sub 1)
                                   (f2cl-lib:int-sub 1)))
                               ((1 *)) rwork-%offset%))
      (declare (ignore var-0 var-1 var-3 var-4))
      (when var-2 (setf nu var-2))))
   (t
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (slartgs
         (f2cl-lib:array-slice b22e-%data% single-float
                               ((+ imax (f2cl-lib:int-sub 1))) ((1 *))
                               b22e-%offset%)
         (f2cl-lib:array-slice b22d-%data% single-float (imax) ((1 *))
                               b22d-%offset%)
         mu
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ iv2tcs imax (f2cl-lib:int-sub 1)
                                   (f2cl-lib:int-sub 1)))
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ iv2tsn imax (f2cl-lib:int-sub 1)
                                   (f2cl-lib:int-sub 1)))
                               ((1 *)) rwork-%offset%))
      (declare (ignore var-0 var-1 var-3 var-4))
      (when var-2 (setf mu var-2)))))
  (setf temp
          (+
           (*
            (f2cl-lib:fref rwork-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add iv2tcs imax) 1 1))
                           ((1 *)) rwork-%offset%)
```

```
          (f2cl-lib:fref b12e-%data% ((f2cl-lib:int-sub imax 1))
                         ((1 *)) b12e-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv2tsn imax) 1 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b12d-%data% (imax) ((1 *)) b12d-%offset%))))
(setf (f2cl-lib:fref b12d-%data% (imax) ((1 *)) b12d-%offset%)
        (-
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv2tcs imax) 1 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b12d-%data% (imax) ((1 *)) b12d-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv2tsn imax) 1 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b12e-%data% ((f2cl-lib:int-sub imax 1))
                         ((1 *)) b12e-%offset%))))
(setf (f2cl-lib:fref b12e-%data% ((f2cl-lib:int-sub imax 1)) ((1 *))
                     b12e-%offset%)
        temp)
(setf temp
        (+
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv2tcs imax) 1 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b22e-%data% ((f2cl-lib:int-sub imax 1))
                         ((1 *)) b22e-%offset%))
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv2tsn imax) 1 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b22d-%data% (imax) ((1 *)) b22d-%offset%))))
(setf (f2cl-lib:fref b22d-%data% (imax) ((1 *)) b22d-%offset%)
        (-
         (*
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv2tcs imax) 1 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b22d-%data% (imax) ((1 *)) b22d-%offset%))
         (*
```

```
          (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iv2tsn imax) 1 1))
                         ((1 *)) rwork-%offset%)
          (f2cl-lib:fref b22e-%data% ((f2cl-lib:int-sub imax 1))
                         ((1 *)) b22e-%offset%))))
(setf (f2cl-lib:fref b22e-%data% ((f2cl-lib:int-sub imax 1)) ((1 *))
                     b22e-%offset%)
        temp)
(cond
 (wantu1
  (cond
   (colmajor
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clasr "R" "V" "F" p
         (f2cl-lib:int-add (f2cl-lib:int-sub imax imin) 1)
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ iu1cs imin (f2cl-lib:int-sub 1)))
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ iu1sn imin (f2cl-lib:int-sub 1)))
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice u1-%data% f2cl-lib:complex8 (1 imin)
                               ((1 ldu1) (1 *)) u1-%offset%)
         ldu1)
      (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-7))
      (when var-3 (setf p var-3))
      (when var-8 (setf ldu1 var-8))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clasr "L" "V" "F"
         (f2cl-lib:int-add (f2cl-lib:int-sub imax imin) 1) p
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ iu1cs imin (f2cl-lib:int-sub 1)))
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ iu1sn imin (f2cl-lib:int-sub 1)))
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice u1-%data% f2cl-lib:complex8 (imin 1)
                               ((1 ldu1) (1 *)) u1-%offset%)
         ldu1)
      (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
      (when var-4 (setf p var-4))
      (when var-8 (setf ldu1 var-8)))))))
(cond
 (wantu2
  (cond
   (colmajor
```

```
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (clasr "R" "V" "F" (f2cl-lib:int-sub m p)
          (f2cl-lib:int-add (f2cl-lib:int-sub imax imin) 1)
          (f2cl-lib:array-slice rwork-%data% single-float
                                ((+ iu2cs imin (f2cl-lib:int-sub 1)))
                                ((1 *)) rwork-%offset%)
          (f2cl-lib:array-slice rwork-%data% single-float
                                ((+ iu2sn imin (f2cl-lib:int-sub 1)))
                                ((1 *)) rwork-%offset%)
          (f2cl-lib:array-slice u2-%data% f2cl-lib:complex8 (1 imin)
                                ((1 ldu2) (1 *)) u2-%offset%)
          ldu2)
       (declare
        (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7))
       (when var-8 (setf ldu2 var-8))))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (clasr "L" "V" "F"
          (f2cl-lib:int-add (f2cl-lib:int-sub imax imin) 1)
          (f2cl-lib:int-sub m p)
          (f2cl-lib:array-slice rwork-%data% single-float
                                ((+ iu2cs imin (f2cl-lib:int-sub 1)))
                                ((1 *)) rwork-%offset%)
          (f2cl-lib:array-slice rwork-%data% single-float
                                ((+ iu2sn imin (f2cl-lib:int-sub 1)))
                                ((1 *)) rwork-%offset%)
          (f2cl-lib:array-slice u2-%data% f2cl-lib:complex8 (imin 1)
                                ((1 ldu2) (1 *)) u2-%offset%)
          ldu2)
       (declare
        (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7))
       (when var-8 (setf ldu2 var-8)))))))
(cond
 (wantv1t
  (cond
   (colmajor
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clasr "L" "V" "F"
         (f2cl-lib:int-add (f2cl-lib:int-sub imax imin) 1) q
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ iv1tcs imin (f2cl-lib:int-sub 1)))
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ iv1tsn imin (f2cl-lib:int-sub 1)))
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice v1t-%data% f2cl-lib:complex8 (imin 1)
                               ((1 ldv1t) (1 *)) v1t-%offset%)
```

```
          ldv1t)
        (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
        (when var-4 (setf q var-4))
        (when var-8 (setf ldv1t var-8))))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (clasr "R" "V" "F" q
          (f2cl-lib:int-add (f2cl-lib:int-sub imax imin) 1)
          (f2cl-lib:array-slice rwork-%data% single-float
                                ((+ iv1tcs imin (f2cl-lib:int-sub 1)))
                                ((1 *)) rwork-%offset%)
          (f2cl-lib:array-slice rwork-%data% single-float
                                ((+ iv1tsn imin (f2cl-lib:int-sub 1)))
                                ((1 *)) rwork-%offset%)
          (f2cl-lib:array-slice v1t-%data% f2cl-lib:complex8 (1 imin)
                                ((1 ldv1t) (1 *)) v1t-%offset%)
          ldv1t)
        (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-7))
        (when var-3 (setf q var-3))
        (when var-8 (setf ldv1t var-8)))))))
(cond
 (wantv2t
  (cond
   (colmajor
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clasr "L" "V" "F"
         (f2cl-lib:int-add (f2cl-lib:int-sub imax imin) 1)
         (f2cl-lib:int-sub m q)
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ iv2tcs imin (f2cl-lib:int-sub 1)))
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ iv2tsn imin (f2cl-lib:int-sub 1)))
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice v2t-%data% f2cl-lib:complex8 (imin 1)
                               ((1 ldv2t) (1 *)) v2t-%offset%)
         ldv2t)
       (declare
        (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7))
       (when var-8 (setf ldv2t var-8))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clasr "R" "V" "F" (f2cl-lib:int-sub m q)
         (f2cl-lib:int-add (f2cl-lib:int-sub imax imin) 1)
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ iv2tcs imin (f2cl-lib:int-sub 1)))
                               ((1 *)) rwork-%offset%)
```

```
          (f2cl-lib:array-slice rwork-%data% single-float
                                ((+ iv2tsn imin (f2cl-lib:int-sub 1)))
                                ((1 *)) rwork-%offset%)
          (f2cl-lib:array-slice v2t-%data% f2cl-lib:complex8 (1 imin)
                                ((1 ldv2t) (1 *)) v2t-%offset%)
          ldv2t)
       (declare
        (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7))
       (when var-8 (setf ldv2t var-8)))))))
(cond
 ((>
   (+
    (f2cl-lib:fref b11e ((f2cl-lib:int-add imax (f2cl-lib:int-sub 1)))
                    ((1 *)))
    (f2cl-lib:fref b21e ((f2cl-lib:int-add imax (f2cl-lib:int-sub 1)))
                    ((1 *))))
   0)
  (setf (f2cl-lib:fref b11d-%data% (imax) ((1 *)) b11d-%offset%)
          (- (f2cl-lib:fref b11d-%data% (imax) ((1 *)) b11d-%offset%)))
  (setf (f2cl-lib:fref b21d-%data% (imax) ((1 *)) b21d-%offset%)
          (- (f2cl-lib:fref b21d-%data% (imax) ((1 *)) b21d-%offset%)))
  (cond
   (wantv1t
    (cond
     (colmajor
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (cscal q negonecomplex
           (f2cl-lib:array-slice v1t-%data% f2cl-lib:complex8 (imax 1)
                                 ((1 ldv1t) (1 *)) v1t-%offset%)
           ldv1t)
        (declare (ignore var-2))
        (when var-0 (setf q var-0))
        (when var-1 (setf negonecomplex var-1))
        (when var-3 (setf ldv1t var-3))))
     (t
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (cscal q negonecomplex
           (f2cl-lib:array-slice v1t-%data% f2cl-lib:complex8 (1 imax)
                                 ((1 ldv1t) (1 *)) v1t-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf q var-0))
        (when var-1 (setf negonecomplex var-1)))))))))
(setf x1
        (+
         (*
          (cos
           (f2cl-lib:fref phi-%data% ((f2cl-lib:int-sub imax 1))
                          ((1 *)) phi-%offset%))
          (f2cl-lib:fref b11d-%data% (imax) ((1 *)) b11d-%offset%))
```

```
          (*
           (sin
            (f2cl-lib:fref phi-%data% ((f2cl-lib:int-sub imax 1))
                           ((1 *)) phi-%offset%))
           (f2cl-lib:fref b12e-%data% ((f2cl-lib:int-sub imax 1))
                          ((1 *)) b12e-%offset%))))
(setf y1
        (+
         (*
          (cos
           (f2cl-lib:fref phi-%data% ((f2cl-lib:int-sub imax 1))
                           ((1 *)) phi-%offset%))
           (f2cl-lib:fref b21d-%data% (imax) ((1 *)) b21d-%offset%))
         (*
          (sin
           (f2cl-lib:fref phi-%data% ((f2cl-lib:int-sub imax 1))
                           ((1 *)) phi-%offset%))
           (f2cl-lib:fref b22e-%data% ((f2cl-lib:int-sub imax 1))
                          ((1 *)) b22e-%offset%))))
(setf (f2cl-lib:fref theta-%data% (imax) ((1 *)) theta-%offset%)
        (f2cl-lib:atan2 (abs y1) (abs x1)))
(cond
 ((<
   (+ (f2cl-lib:fref b11d (imax) ((1 *)))
      (f2cl-lib:fref b12e
                     ((f2cl-lib:int-add imax (f2cl-lib:int-sub 1)))
                     ((1 *))))
   0)
  (setf (f2cl-lib:fref b12d-%data% (imax) ((1 *)) b12d-%offset%)
          (- (f2cl-lib:fref b12d-%data% (imax) ((1 *)) b12d-%offset%)))
  (cond
   (wantu1
    (cond
     (colmajor
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (cscal p negonecomplex
           (f2cl-lib:array-slice u1-%data% f2cl-lib:complex8 (1 imax)
                                 ((1 ldu1) (1 *)) u1-%offset%)
           1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf p var-0))
        (when var-1 (setf negonecomplex var-1))))
     (t
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (cscal p negonecomplex
           (f2cl-lib:array-slice u1-%data% f2cl-lib:complex8 (imax 1)
                                 ((1 ldu1) (1 *)) u1-%offset%)
           ldu1)
        (declare (ignore var-2))
        (when var-0 (setf p var-0))
```

```
        (when var-1 (setf negonecomplex var-1))
        (when var-3 (setf ldu1 var-3)))))))))
(cond
 ((>
   (+ (f2cl-lib:fref b21d (imax) ((1 *)))
      (f2cl-lib:fref b22e
                     ((f2cl-lib:int-add imax (f2cl-lib:int-sub 1)))
                     ((1 *))))
   0)
  (setf (f2cl-lib:fref b22d-%data% (imax) ((1 *)) b22d-%offset%)
          (- (f2cl-lib:fref b22d-%data% (imax) ((1 *)) b22d-%offset%)))
  (cond
   (wantu2
    (cond
     (colmajor
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (cscal (f2cl-lib:int-sub m p) negonecomplex
           (f2cl-lib:array-slice u2-%data% f2cl-lib:complex8 (1 imax)
                                 ((1 ldu2) (1 *)) u2-%offset%)
           1)
        (declare (ignore var-0 var-2 var-3))
        (when var-1 (setf negonecomplex var-1))))
     (t
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (cscal (f2cl-lib:int-sub m p) negonecomplex
           (f2cl-lib:array-slice u2-%data% f2cl-lib:complex8 (imax 1)
                                 ((1 ldu2) (1 *)) u2-%offset%)
           ldu2)
        (declare (ignore var-0 var-2))
        (when var-1 (setf negonecomplex var-1))
        (when var-3 (setf ldu2 var-3)))))))))
(cond
 ((<
   (+ (f2cl-lib:fref b12d (imax) ((1 *)))
      (f2cl-lib:fref b22d (imax) ((1 *))))
   0)
  (cond
   (wantv2t
    (cond
     (colmajor
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (cscal (f2cl-lib:int-sub m q) negonecomplex
           (f2cl-lib:array-slice v2t-%data% f2cl-lib:complex8 (imax 1)
                                 ((1 ldv2t) (1 *)) v2t-%offset%)
           ldv2t)
        (declare (ignore var-0 var-2))
        (when var-1 (setf negonecomplex var-1))
        (when var-3 (setf ldv2t var-3))))
     (t
      (multiple-value-bind (var-0 var-1 var-2 var-3)
```

```
          (cscal (f2cl-lib:int-sub m q) negonecomplex
           (f2cl-lib:array-slice v2t-%data% f2cl-lib:complex8 (1 imax)
                                 ((1 ldv2t) (1 *)) v2t-%offset%)
           1)
        (declare (ignore var-0 var-2 var-3))
        (when var-1 (setf negonecomplex var-1)))))))))
(f2cl-lib:fdo (i imin (f2cl-lib:int-add i 1))
              ((> i imax) nil)
  (tagbody
    (cond
     ((< (f2cl-lib:fref theta (i) ((1 *))) thresh)
      (setf (f2cl-lib:fref theta-%data% (i) ((1 *)) theta-%offset%)
            zero))
     ((> (f2cl-lib:fref theta (i) ((1 *))) (+ piover2 (- thresh)))
      (setf (f2cl-lib:fref theta-%data% (i) ((1 *)) theta-%offset%)
            piover2)))
   label100012))
(f2cl-lib:fdo (i imin (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add imax (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (cond
     ((< (f2cl-lib:fref phi (i) ((1 *))) thresh)
      (setf (f2cl-lib:fref phi-%data% (i) ((1 *)) phi-%offset%) zero))
     ((> (f2cl-lib:fref phi (i) ((1 *))) (+ piover2 (- thresh)))
      (setf (f2cl-lib:fref phi-%data% (i) ((1 *)) phi-%offset%)
            piover2)))
   label100013))
(cond
 ((> imax 1)
  (tagbody
   label100014
    (if (not
         (=
          (f2cl-lib:fref phi-%data% ((f2cl-lib:int-sub imax 1)) ((1 *))
                         phi-%offset%)
          zero))
        (go label100015))
    (setf imax (f2cl-lib:int-sub imax 1))
    (if (<= imax 1)
        (go f2cl-lib::exit))
    (go label100014)
   label100015)))
(if (> imin (f2cl-lib:int-sub imax 1))
    (setf imin (f2cl-lib:int-sub imax 1)))
(cond
 ((> imin 1)
  (tagbody
   label100016
    (if (not
         (/=
```

```
              (f2cl-lib:fref phi-%data% ((f2cl-lib:int-sub imin 1)) ((1 *))
                             phi-%offset%)
              zero))
            (go label100017))
        (setf imin (f2cl-lib:int-sub imin 1))
        (if (<= imin 1)
            (go f2cl-lib::exit))
        (go label100016)
       label100017)))
    (go label100006)
   label100007
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i q) nil)
      (tagbody
        (setf mini i)
        (setf thetamin
                (f2cl-lib:fref theta-%data% (i) ((1 *)) theta-%offset%))
        (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                      ((> j q) nil)
          (tagbody
            (cond
             ((< (f2cl-lib:fref theta (j) ((1 *))) thetamin) (setf mini j)
              (setf thetamin
                      (f2cl-lib:fref theta-%data% (j) ((1 *))
                                     theta-%offset%))))
           label100019))
        (cond
         ((/= mini i)
          (setf (f2cl-lib:fref theta-%data% (mini) ((1 *)) theta-%offset%)
                  (f2cl-lib:fref theta-%data% (i) ((1 *)) theta-%offset%))
          (setf (f2cl-lib:fref theta-%data% (i) ((1 *)) theta-%offset%)
                  thetamin)
          (cond
           (colmajor
            (if wantu1
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (cswap p
                     (f2cl-lib:array-slice u1-%data% f2cl-lib:complex8
                                           (1 i) ((1 ldu1) (1 *))
                                           u1-%offset%)
                     1
                     (f2cl-lib:array-slice u1-%data% f2cl-lib:complex8
                                           (1 mini) ((1 ldu1) (1 *))
                                           u1-%offset%)
                     1)
                  (declare (ignore var-1 var-2 var-3 var-4))
                  (when var-0 (setf p var-0))))
            (if wantu2
                (cswap (f2cl-lib:int-sub m p)
                 (f2cl-lib:array-slice u2-%data% f2cl-lib:complex8 (1 i)
```

```
                              ((1 ldu2) (1 *)) u2-%offset%)
      1
      (f2cl-lib:array-slice u2-%data% f2cl-lib:complex8 (1 mini)
                              ((1 ldu2) (1 *)) u2-%offset%)
      1))
 (if wantv1t
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (cswap q
          (f2cl-lib:array-slice v1t-%data% f2cl-lib:complex8
                                (i 1) ((1 ldv1t) (1 *))
                                v1t-%offset%)
          ldv1t
          (f2cl-lib:array-slice v1t-%data% f2cl-lib:complex8
                                (mini 1) ((1 ldv1t) (1 *))
                                v1t-%offset%)
          ldv1t)
       (declare (ignore var-1 var-3))
       (when var-0 (setf q var-0))
       (when var-2 (setf ldv1t var-2))
       (when var-4 (setf ldv1t var-4))))
 (if wantv2t
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (cswap (f2cl-lib:int-sub m q)
          (f2cl-lib:array-slice v2t-%data% f2cl-lib:complex8
                                (i 1) ((1 ldv2t) (1 *))
                                v2t-%offset%)
          ldv2t
          (f2cl-lib:array-slice v2t-%data% f2cl-lib:complex8
                                (mini 1) ((1 ldv2t) (1 *))
                                v2t-%offset%)
          ldv2t)
       (declare (ignore var-0 var-1 var-3))
       (when var-2 (setf ldv2t var-2))
       (when var-4 (setf ldv2t var-4)))))
(t
 (if wantu1
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (cswap p
          (f2cl-lib:array-slice u1-%data% f2cl-lib:complex8
                                (i 1) ((1 ldu1) (1 *))
                                u1-%offset%)
          ldu1
          (f2cl-lib:array-slice u1-%data% f2cl-lib:complex8
                                (mini 1) ((1 ldu1) (1 *))
                                u1-%offset%)
          ldu1)
       (declare (ignore var-1 var-3))
       (when var-0 (setf p var-0))
       (when var-2 (setf ldu1 var-2))
       (when var-4 (setf ldu1 var-4))))
```

```
            (if wantu2
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (cswap (f2cl-lib:int-sub m p)
                     (f2cl-lib:array-slice u2-%data% f2cl-lib:complex8
                                           (i 1) ((1 ldu2) (1 *))
                                           u2-%offset%)
                     ldu2
                     (f2cl-lib:array-slice u2-%data% f2cl-lib:complex8
                                           (mini 1) ((1 ldu2) (1 *))
                                           u2-%offset%)
                     ldu2)
                  (declare (ignore var-0 var-1 var-3))
                  (when var-2 (setf ldu2 var-2))
                  (when var-4 (setf ldu2 var-4))))
            (if wantv1t
                (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                    (cswap q
                     (f2cl-lib:array-slice v1t-%data% f2cl-lib:complex8
                                           (1 i) ((1 ldv1t) (1 *))
                                           v1t-%offset%)
                     1
                     (f2cl-lib:array-slice v1t-%data% f2cl-lib:complex8
                                           (1 mini) ((1 ldv1t) (1 *))
                                           v1t-%offset%)
                     1)
                  (declare (ignore var-1 var-2 var-3 var-4))
                  (when var-0 (setf q var-0))))
            (if wantv2t
                (cswap (f2cl-lib:int-sub m q)
                 (f2cl-lib:array-slice v2t-%data% f2cl-lib:complex8 (1 i)
                                       ((1 ldv2t) (1 *)) v2t-%offset%)
                 1
                 (f2cl-lib:array-slice v2t-%data% f2cl-lib:complex8
                                       (1 mini) ((1 ldv2t) (1 *))
                                       v2t-%offset%)
                 1))))))
    label100018))
  (go end_label)
end_label
  (return
   (values jobu1 jobu2 jobv1t jobv2t trans nil p q nil nil nil ldu1 nil
           ldu2 nil ldv1t nil ldv2t nil nil nil nil nil nil nil nil nil
           nil info))))))
```

———-

90.111 cbdsqr - compute singular values from an SVD

See Demmel [Demm88] and Fernando [Fern92]

90.111.1 Axiom unit tests

— cbdsqr.input —

```
)set break resume
)sys rm -f cbdsqr.output
)spool cbdsqr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.111.2 Axiom help page

— cbdsqr.help —

```
======================================================================
cbdsqr examples
======================================================================

======================================================================
Man Page Details
======================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cbdsqr (UPLO, N, NCVT, NRU, NCC, D, E, VT, LDVT, U, LDU, C,
           LDC, RWORK, INFO)
           CBDSQR

Function/Subroutine Documentation
   subroutine cbdsqr (character UPLO, integer N, integer NCVT, integer NRU,
       integer NCC, real, dimension( * ) D, real, dimension( * ) E, complex,
       dimension( ldvt, * ) VT, integer LDVT, complex, dimension( ldu, * ) U,
       integer LDU, complex, dimension( ldc, * ) C, integer LDC, real,
       dimension( * ) RWORK, integer INFO)
       CBDSQR
```

```
Purpose:
   CBDSQR computes the singular values and, optionally, the right and/or
   left singular vectors from the singular value decomposition (SVD) of
   a real N-by-N (upper or lower) bidiagonal matrix B using the implicit
   zero-shift QR algorithm.  The SVD of B has the form

   B = Q * S * P**H

   where S is the diagonal matrix of singular values, Q is an orthogonal
   matrix of left singular vectors, and P is an orthogonal matrix of
   right singular vectors.  If left singular vectors are requested, this
   subroutine actually returns U*Q instead of Q, and, if right singular
   vectors are requested, this subroutine returns P**H*VT instead of
   P**H, for given complex input matrices U and VT.  When U and VT are
   the unitary matrices that reduce a general matrix A to bidiagonal
   form: A = U*B*VT, as computed by CGEBRD, then

   A = (U*Q) * S * (P**H*VT)

   is the SVD of A.  Optionally, the subroutine may also compute Q**H*C
   for a given complex input matrix C.

Parameters:

 UPLO
   UPLO is CHARACTER*1
   = 'U':  B is upper bidiagonal;
   = 'L':  B is lower bidiagonal.

 N
   N is INTEGER
   The order of the matrix B.  N >= 0.

 NCVT
   NCVT is INTEGER
   The number of columns of the matrix VT. NCVT >= 0.

 NRU
   NRU is INTEGER
   The number of rows of the matrix U. NRU >= 0.

 NCC
   NCC is INTEGER
   The number of columns of the matrix C. NCC >= 0.

 D
   D is REAL array, dimension (N)
   On entry, the n diagonal elements of the bidiagonal matrix B.
   On exit, if INFO=0, the singular values of B in decreasing order.
```

```
E
  E is REAL array, dimension (N-1)
  On entry, the N-1 offdiagonal elements of the bidiagonal
  matrix B.
  On exit, if INFO = 0, E is destroyed; if INFO > 0, D and E
  will contain the diagonal and superdiagonal elements of a
  bidiagonal matrix orthogonally equivalent to the one given as input.

  VT
  VT is COMPLEX array, dimension (LDVT, NCVT)
  On entry, an N-by-NCVT matrix VT.
  On exit, VT is overwritten by P**H * VT.
  Not referenced if NCVT = 0.

LDVT
  LDVT is INTEGER
  The leading dimension of the array VT.
  LDVT >= max(1,N) if NCVT > 0; LDVT >= 1 if NCVT = 0.

U
  U is COMPLEX array, dimension (LDU, N)
  On entry, an NRU-by-N matrix U.
  On exit, U is overwritten by U * Q.
  Not referenced if NRU = 0.

LDU
  LDU is INTEGER
  The leading dimension of the array U.  LDU >= max(1,NRU).

C
  C is COMPLEX array, dimension (LDC, NCC)
  On entry, an N-by-NCC matrix C.
  On exit, C is overwritten by Q**H * C.
  Not referenced if NCC = 0.

LDC
  LDC is INTEGER
  The leading dimension of the array C.
  LDC >= max(1,N) if NCC > 0; LDC >=1 if NCC = 0.

RWORK
  RWORK is REAL array, dimension (4*N)

INFO
  INFO is INTEGER
  = 0:  successful exit
  < 0:  If INFO = -i, the i-th argument had an illegal value
  > 0:  the algorithm did not converge; D and E contain the
  elements of a bidiagonal matrix which is orthogonally
```

```
   similar to the input matrix B;  if INFO = i, i
   elements of E have not converged to zero.

Internal Parameters:

 TOLMUL  REAL, default = max(10,min(100,EPS**(-1/8)))
   TOLMUL controls the convergence criterion of the QR loop.

   If it is positive, TOLMUL*EPS is the desired relative
   precision in the computed singular values.

   If it is negative, abs(TOLMUL*EPS*sigma_max) is the
   desired absolute accuracy in the computed singular
   values (corresponds to relative accuracy
   abs(TOLMUL*EPS) in the largest singular value.

   abs(TOLMUL) should be between 1 and 1/EPS, and preferably
   between 10 (for fast convergence) and .1/EPS
   (for there to be some accuracy in the results).

   Default is to lose at either one eighth or 2 of the
   available decimal digits in each computed singular value
   (whichever is smaller).

 MAXITR  INTEGER, default = 6
   MAXITR controls the maximum number of passes of the
   algorithm through its inner loop. The algorithms stops
   (and so fails to converge) if the number of passes
   through the inner loop exceeds MAXITR*N**2.
```

90.111.3 fortran code

— cbdsqr.f —

```
*  ===================================================================
      SUBROUTINE CBDSQR( UPLO, N, NCVT, NRU, NCC, D, E, VT, LDVT, U,
     $                   LDU, C, LDC, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
```

```
      INTEGER            INFO, LDC, LDU, LDVT, N, NCC, NCVT, NRU
*     ..
*     .. Array Arguments ..
      REAL               D( * ), E( * ), RWORK( * )
      COMPLEX            C( LDC, * ), U( LDU, * ), VT( LDVT, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E0 )
      REAL               ONE
      PARAMETER          ( ONE = 1.0E0 )
      REAL               NEGONE
      PARAMETER          ( NEGONE = -1.0E0 )
      REAL               HNDRTH
      PARAMETER          ( HNDRTH = 0.01E0 )
      REAL               TEN
      PARAMETER          ( TEN = 10.0E0 )
      REAL               HNDRD
      PARAMETER          ( HNDRD = 100.0E0 )
      REAL               MEIGTH
      PARAMETER          ( MEIGTH = -0.125E0 )
      INTEGER            MAXITR
      PARAMETER          ( MAXITR = 6 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LOWER, ROTATE
      INTEGER            I, IDIR, ISUB, ITER, J, LL, LLL, M, MAXIT, NM1,
     $                   NM12, NM13, OLDLL, OLDM
      REAL               ABSE, ABSS, COSL, COSR, CS, EPS, F, G, H, MU,
     $                   OLDCS, OLDSN, R, SHIFT, SIGMN, SIGMX, SINL,
     $                   SINR, SLL, SMAX, SMIN, SMINL, SMINOA,
     $                   SN, THRESH, TOL, TOLMUL, UNFL
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH
      EXTERNAL           LSAME, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLASR, CSROT, CSSCAL, CSWAP, SLARTG, SLAS2,
     $                   SLASQ1, SLASV2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, REAL, SIGN, SQRT
*     ..
*     .. Executable Statements ..
*
```

```
*     Test the input parameters.
*
      INFO = 0
      LOWER = LSAME( UPLO, 'L' )
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LOWER ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NCVT.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRU.LT.0 ) THEN
         INFO = -4
      ELSE IF( NCC.LT.0 ) THEN
         INFO = -5
      ELSE IF( ( NCVT.EQ.0 .AND. LDVT.LT.1 ) .OR.
     $         ( NCVT.GT.0 .AND. LDVT.LT.MAX( 1, N ) ) ) THEN
         INFO = -9
      ELSE IF( LDU.LT.MAX( 1, NRU ) ) THEN
         INFO = -11
      ELSE IF( ( NCC.EQ.0 .AND. LDC.LT.1 ) .OR.
     $         ( NCC.GT.0 .AND. LDC.LT.MAX( 1, N ) ) ) THEN
         INFO = -13
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CBDSQR', -INFO )
         RETURN
      END IF
      IF( N.EQ.0 )
     $   RETURN
      IF( N.EQ.1 )
     $   GO TO 160
*
*     ROTATE is true if any singular vectors desired, false otherwise
*
      ROTATE = ( NCVT.GT.0 ) .OR. ( NRU.GT.0 ) .OR. ( NCC.GT.0 )
*
*     If no singular vectors desired, use qd algorithm
*
      IF( .NOT.ROTATE ) THEN
         CALL SLASQ1( N, D, E, RWORK, INFO )
*
*     If INFO equals 2, dqds didn't finish, try to finish
*
         IF( INFO .NE. 2 ) RETURN
         INFO = 0
      END IF
*
      NM1 = N - 1
      NM12 = NM1 + NM1
      NM13 = NM12 + NM1
```

```
      IDIR = 0
*
*     Get machine constants
*
      EPS = SLAMCH( 'Epsilon' )
      UNFL = SLAMCH( 'Safe minimum' )
*
*     If matrix lower bidiagonal, rotate to be upper bidiagonal
*     by applying Givens rotations on the left
*
      IF( LOWER ) THEN
         DO 10 I = 1, N - 1
            CALL SLARTG( D( I ), E( I ), CS, SN, R )
            D( I ) = R
            E( I ) = SN*D( I+1 )
            D( I+1 ) = CS*D( I+1 )
            RWORK( I ) = CS
            RWORK( NM1+I ) = SN
   10    CONTINUE
*
*        Update singular vectors if desired
*
         IF( NRU.GT.0 )
     $      CALL CLASR( 'R', 'V', 'F', NRU, N, RWORK( 1 ), RWORK( N ),
     $                  U, LDU )
         IF( NCC.GT.0 )
     $      CALL CLASR( 'L', 'V', 'F', N, NCC, RWORK( 1 ), RWORK( N ),
     $                  C, LDC )
      END IF
*
*     Compute singular values to relative accuracy TOL
*     (By setting TOL to be negative, algorithm will compute
*     singular values to absolute accuracy ABS(TOL)*norm(input matrix))
*
      TOLMUL = MAX( TEN, MIN( HNDRD, EPS**MEIGTH ) )
      TOL = TOLMUL*EPS
*
*     Compute approximate maximum, minimum singular values
*
      SMAX = ZERO
      DO 20 I = 1, N
         SMAX = MAX( SMAX, ABS( D( I ) ) )
   20 CONTINUE
      DO 30 I = 1, N - 1
         SMAX = MAX( SMAX, ABS( E( I ) ) )
   30 CONTINUE
      SMINL = ZERO
      IF( TOL.GE.ZERO ) THEN
*
*        Relative accuracy desired
```

```
*
         SMINOA = ABS( D( 1 ) )
         IF( SMINOA.EQ.ZERO )
     $      GO TO 50
         MU = SMINOA
         DO 40 I = 2, N
            MU = ABS( D( I ) )*( MU / ( MU+ABS( E( I-1 ) ) ) )
            SMINOA = MIN( SMINOA, MU )
            IF( SMINOA.EQ.ZERO )
     $         GO TO 50
   40    CONTINUE
   50    CONTINUE
         SMINOA = SMINOA / SQRT( REAL( N ) )
         THRESH = MAX( TOL*SMINOA, MAXITR*N*N*UNFL )
      ELSE
*
*        Absolute accuracy desired
*
         THRESH = MAX( ABS( TOL )*SMAX, MAXITR*N*N*UNFL )
      END IF
*
*     Prepare for main iteration loop for the singular values
*     (MAXIT is the maximum number of passes through the inner
*     loop permitted before nonconvergence signalled.)
*
      MAXIT = MAXITR*N*N
      ITER = 0
      OLDLL = -1
      OLDM = -1
*
*     M points to last element of unconverged part of matrix
*
      M = N
*
*     Begin main iteration loop
*
   60 CONTINUE
*
*     Check for convergence or exceeding iteration count
*
      IF( M.LE.1 )
     $   GO TO 160
      IF( ITER.GT.MAXIT )
     $   GO TO 200
*
*     Find diagonal block of matrix to work on
*
      IF( TOL.LT.ZERO .AND. ABS( D( M ) ).LE.THRESH )
     $   D( M ) = ZERO
      SMAX = ABS( D( M ) )
```

```
      SMIN = SMAX
      DO 70 LLL = 1, M - 1
         LL = M - LLL
         ABSS = ABS( D( LL ) )
         ABSE = ABS( E( LL ) )
         IF( TOL.LT.ZERO .AND. ABSS.LE.THRESH )
     $      D( LL ) = ZERO
         IF( ABSE.LE.THRESH )
     $      GO TO 80
         SMIN = MIN( SMIN, ABSS )
         SMAX = MAX( SMAX, ABSS, ABSE )
   70 CONTINUE
      LL = 0
      GO TO 90
   80 CONTINUE
      E( LL ) = ZERO
*
*     Matrix splits since E(LL) = 0
*
      IF( LL.EQ.M-1 ) THEN
*
*        Convergence of bottom singular value, return to top of loop
*
         M = M - 1
         GO TO 60
      END IF
   90 CONTINUE
      LL = LL + 1
*
*     E(LL) through E(M-1) are nonzero, E(LL-1) is zero
*
      IF( LL.EQ.M-1 ) THEN
*
*        2 by 2 block, handle separately
*
         CALL SLASV2( D( M-1 ), E( M-1 ), D( M ), SIGMN, SIGMX, SINR,
     $                COSR, SINL, COSL )
         D( M-1 ) = SIGMX
         E( M-1 ) = ZERO
         D( M ) = SIGMN
*
*        Compute singular vectors, if desired
*
         IF( NCVT.GT.0 )
     $      CALL CSROT( NCVT, VT( M-1, 1 ), LDVT, VT( M, 1 ), LDVT,
     $                  COSR, SINR )
         IF( NRU.GT.0 )
     $      CALL CSROT( NRU, U( 1, M-1 ), 1, U( 1, M ), 1, COSL, SINL )
         IF( NCC.GT.0 )
     $      CALL CSROT( NCC, C( M-1, 1 ), LDC, C( M, 1 ), LDC, COSL,
```

```
     $                  SINL )
         M = M - 2
         GO TO 60
      END IF
*
*     If working on new submatrix, choose shift direction
*     (from larger end diagonal element towards smaller)
*
      IF( LL.GT.OLDM .OR. M.LT.OLDLL ) THEN
         IF( ABS( D( LL ) ).GE.ABS( D( M ) ) ) THEN
*
*           Chase bulge from top (big end) to bottom (small end)
*
            IDIR = 1
         ELSE
*
*           Chase bulge from bottom (big end) to top (small end)
*
            IDIR = 2
         END IF
      END IF
*
*     Apply convergence tests
*
      IF( IDIR.EQ.1 ) THEN
*
*        Run convergence test in forward direction
*        First apply standard test to bottom of matrix
*
         IF( ABS( E( M-1 ) ).LE.ABS( TOL )*ABS( D( M ) ) .OR.
     $       ( TOL.LT.ZERO .AND. ABS( E( M-1 ) ).LE.THRESH ) ) THEN
            E( M-1 ) = ZERO
            GO TO 60
         END IF
*
         IF( TOL.GE.ZERO ) THEN
*
*           If relative accuracy desired,
*           apply convergence criterion forward
*
            MU = ABS( D( LL ) )
            SMINL = MU
            DO 100 LLL = LL, M - 1
               IF( ABS( E( LLL ) ).LE.TOL*MU ) THEN
                  E( LLL ) = ZERO
                  GO TO 60
               END IF
               MU = ABS( D( LLL+1 ) )*( MU / ( MU+ABS( E( LLL ) ) ) )
               SMINL = MIN( SMINL, MU )
  100       CONTINUE
```

```
         END IF
*
      ELSE
*
*        Run convergence test in backward direction
*        First apply standard test to top of matrix
*
         IF( ABS( E( LL ) ).LE.ABS( TOL )*ABS( D( LL ) ) .OR.
     $       ( TOL.LT.ZERO .AND. ABS( E( LL ) ).LE.THRESH ) ) THEN
            E( LL ) = ZERO
            GO TO 60
         END IF
*
         IF( TOL.GE.ZERO ) THEN
*
*           If relative accuracy desired,
*           apply convergence criterion backward
*
            MU = ABS( D( M ) )
            SMINL = MU
            DO 110 LLL = M - 1, LL, -1
               IF( ABS( E( LLL ) ).LE.TOL*MU ) THEN
                  E( LLL ) = ZERO
                  GO TO 60
               END IF
               MU = ABS( D( LLL ) )*( MU / ( MU+ABS( E( LLL ) ) ) )
               SMINL = MIN( SMINL, MU )
  110       CONTINUE
         END IF
      END IF
      OLDLL = LL
      OLDM = M
*
*     Compute shift.  First, test if shifting would ruin relative
*     accuracy, and if so set the shift to zero.
*
      IF( TOL.GE.ZERO .AND. N*TOL*( SMINL / SMAX ).LE.
     $    MAX( EPS, HNDRTH*TOL ) ) THEN
*
*        Use a zero shift to avoid loss of relative accuracy
*
         SHIFT = ZERO
      ELSE
*
*        Compute the shift from 2-by-2 block at end of matrix
*
         IF( IDIR.EQ.1 ) THEN
            SLL = ABS( D( LL ) )
            CALL SLAS2( D( M-1 ), E( M-1 ), D( M ), SHIFT, R )
         ELSE
```

```
            SLL = ABS( D( M ) )
            CALL SLAS2( D( LL ), E( LL ), D( LL+1 ), SHIFT, R )
         END IF
*
*        Test if shift negligible, and if so set to zero
*
         IF( SLL.GT.ZERO ) THEN
            IF( ( SHIFT / SLL )**2.LT.EPS )
     $         SHIFT = ZERO
         END IF
      END IF
*
*     Increment iteration count
*
      ITER = ITER + M - LL
*
*     If SHIFT = 0, do simplified QR iteration
*
      IF( SHIFT.EQ.ZERO ) THEN
         IF( IDIR.EQ.1 ) THEN
*
*           Chase bulge from top to bottom
*           Save cosines and sines for later singular vector updates
*
            CS = ONE
            OLDCS = ONE
            DO 120 I = LL, M - 1
               CALL SLARTG( D( I )*CS, E( I ), CS, SN, R )
               IF( I.GT.LL )
     $            E( I-1 ) = OLDSN*R
               CALL SLARTG( OLDCS*R, D( I+1 )*SN, OLDCS, OLDSN, D( I ) )
               RWORK( I-LL+1 ) = CS
               RWORK( I-LL+1+NM1 ) = SN
               RWORK( I-LL+1+NM12 ) = OLDCS
               RWORK( I-LL+1+NM13 ) = OLDSN
  120       CONTINUE
            H = D( M )*CS
            D( M ) = H*OLDCS
            E( M-1 ) = H*OLDSN
*
*           Update singular vectors
*
            IF( NCVT.GT.0 )
     $         CALL CLASR( 'L', 'V', 'F', M-LL+1, NCVT, RWORK( 1 ),
     $                     RWORK( N ), VT( LL, 1 ), LDVT )
            IF( NRU.GT.0 )
     $         CALL CLASR( 'R', 'V', 'F', NRU, M-LL+1, RWORK( NM12+1 ),
     $                     RWORK( NM13+1 ), U( 1, LL ), LDU )
            IF( NCC.GT.0 )
     $         CALL CLASR( 'L', 'V', 'F', M-LL+1, NCC, RWORK( NM12+1 ),
```

```
     $                    RWORK( NM13+1 ), C( LL, 1 ), LDC )
*
*          Test convergence
*
           IF( ABS( E( M-1 ) ).LE.THRESH )
     $        E( M-1 ) = ZERO
*
        ELSE
*
*          Chase bulge from bottom to top
*          Save cosines and sines for later singular vector updates
*
           CS = ONE
           OLDCS = ONE
           DO 130 I = M, LL + 1, -1
              CALL SLARTG( D( I )*CS, E( I-1 ), CS, SN, R )
              IF( I.LT.M )
     $           E( I ) = OLDSN*R
              CALL SLARTG( OLDCS*R, D( I-1 )*SN, OLDCS, OLDSN, D( I ) )
              RWORK( I-LL ) = CS
              RWORK( I-LL+NM1 ) = -SN
              RWORK( I-LL+NM12 ) = OLDCS
              RWORK( I-LL+NM13 ) = -OLDSN
  130      CONTINUE
           H = D( LL )*CS
           D( LL ) = H*OLDCS
           E( LL ) = H*OLDSN
*
*          Update singular vectors
*
           IF( NCVT.GT.0 )
     $        CALL CLASR( 'L', 'V', 'B', M-LL+1, NCVT, RWORK( NM12+1 ),
     $                    RWORK( NM13+1 ), VT( LL, 1 ), LDVT )
           IF( NRU.GT.0 )
     $        CALL CLASR( 'R', 'V', 'B', NRU, M-LL+1, RWORK( 1 ),
     $                    RWORK( N ), U( 1, LL ), LDU )
           IF( NCC.GT.0 )
     $        CALL CLASR( 'L', 'V', 'B', M-LL+1, NCC, RWORK( 1 ),
     $                    RWORK( N ), C( LL, 1 ), LDC )
*
*          Test convergence
*
           IF( ABS( E( LL ) ).LE.THRESH )
     $        E( LL ) = ZERO
        END IF
     ELSE
*
*       Use nonzero shift
*
        IF( IDIR.EQ.1 ) THEN
```

```
*
*          Chase bulge from top to bottom
*          Save cosines and sines for later singular vector updates
*
           F = ( ABS( D( LL ) )-SHIFT )*
     $         ( SIGN( ONE, D( LL ) )+SHIFT / D( LL ) )
           G = E( LL )
           DO 140 I = LL, M - 1
              CALL SLARTG( F, G, COSR, SINR, R )
              IF( I.GT.LL )
     $           E( I-1 ) = R
              F = COSR*D( I ) + SINR*E( I )
              E( I ) = COSR*E( I ) - SINR*D( I )
              G = SINR*D( I+1 )
              D( I+1 ) = COSR*D( I+1 )
              CALL SLARTG( F, G, COSL, SINL, R )
              D( I ) = R
              F = COSL*E( I ) + SINL*D( I+1 )
              D( I+1 ) = COSL*D( I+1 ) - SINL*E( I )
              IF( I.LT.M-1 ) THEN
                 G = SINL*E( I+1 )
                 E( I+1 ) = COSL*E( I+1 )
              END IF
              RWORK( I-LL+1 ) = COSR
              RWORK( I-LL+1+NM1 ) = SINR
              RWORK( I-LL+1+NM12 ) = COSL
              RWORK( I-LL+1+NM13 ) = SINL
  140      CONTINUE
           E( M-1 ) = F
*
*          Update singular vectors
*
           IF( NCVT.GT.0 )
     $        CALL CLASR( 'L', 'V', 'F', M-LL+1, NCVT, RWORK( 1 ),
     $                    RWORK( N ), VT( LL, 1 ), LDVT )
           IF( NRU.GT.0 )
     $        CALL CLASR( 'R', 'V', 'F', NRU, M-LL+1, RWORK( NM12+1 ),
     $                    RWORK( NM13+1 ), U( 1, LL ), LDU )
           IF( NCC.GT.0 )
     $        CALL CLASR( 'L', 'V', 'F', M-LL+1, NCC, RWORK( NM12+1 ),
     $                    RWORK( NM13+1 ), C( LL, 1 ), LDC )
*
*          Test convergence
*
           IF( ABS( E( M-1 ) ).LE.THRESH )
     $        E( M-1 ) = ZERO
*
        ELSE
*
*          Chase bulge from bottom to top
```

```
*           Save cosines and sines for later singular vector updates
*
            F = ( ABS( D( M ) )-SHIFT )*( SIGN( ONE, D( M ) )+SHIFT /
     $          D( M ) )
            G = E( M-1 )
            DO 150 I = M, LL + 1, -1
               CALL SLARTG( F, G, COSR, SINR, R )
               IF( I.LT.M )
     $            E( I ) = R
               F = COSR*D( I ) + SINR*E( I-1 )
               E( I-1 ) = COSR*E( I-1 ) - SINR*D( I )
               G = SINR*D( I-1 )
               D( I-1 ) = COSR*D( I-1 )
               CALL SLARTG( F, G, COSL, SINL, R )
               D( I ) = R
               F = COSL*E( I-1 ) + SINL*D( I-1 )
               D( I-1 ) = COSL*D( I-1 ) - SINL*E( I-1 )
               IF( I.GT.LL+1 ) THEN
                  G = SINL*E( I-2 )
                  E( I-2 ) = COSL*E( I-2 )
               END IF
               RWORK( I-LL ) = COSR
               RWORK( I-LL+NM1 ) = -SINR
               RWORK( I-LL+NM12 ) = COSL
               RWORK( I-LL+NM13 ) = -SINL
  150       CONTINUE
            E( LL ) = F
*
*           Test convergence
*
            IF( ABS( E( LL ) ).LE.THRESH )
     $         E( LL ) = ZERO
*
*           Update singular vectors if desired
*
            IF( NCVT.GT.0 )
     $         CALL CLASR( 'L', 'V', 'B', M-LL+1, NCVT, RWORK( NM12+1 ),
     $                     RWORK( NM13+1 ), VT( LL, 1 ), LDVT )
            IF( NRU.GT.0 )
     $         CALL CLASR( 'R', 'V', 'B', NRU, M-LL+1, RWORK( 1 ),
     $                     RWORK( N ), U( 1, LL ), LDU )
            IF( NCC.GT.0 )
     $         CALL CLASR( 'L', 'V', 'B', M-LL+1, NCC, RWORK( 1 ),
     $                     RWORK( N ), C( LL, 1 ), LDC )
         END IF
      END IF
*
*     QR iteration finished, go back and check convergence
*
      GO TO 60
```

```
*
*     All singular values converged, so make them positive
*
  160 CONTINUE
      DO 170 I = 1, N
         IF( D( I ).LT.ZERO ) THEN
            D( I ) = -D( I )
*
*           Change sign of singular vectors, if desired
*
            IF( NCVT.GT.0 )
     $         CALL CSSCAL( NCVT, NEGONE, VT( I, 1 ), LDVT )
         END IF
  170 CONTINUE
*
*     Sort the singular values into decreasing order (insertion sort on
*     singular values, but only one transposition per singular vector)
*
      DO 190 I = 1, N - 1
*
*        Scan for smallest D(I)
*
         ISUB = 1
         SMIN = D( 1 )
         DO 180 J = 2, N + 1 - I
            IF( D( J ).LE.SMIN ) THEN
               ISUB = J
               SMIN = D( J )
            END IF
  180    CONTINUE
         IF( ISUB.NE.N+1-I ) THEN
*
*           Swap singular values and vectors
*
            D( ISUB ) = D( N+1-I )
            D( N+1-I ) = SMIN
            IF( NCVT.GT.0 )
     $         CALL CSWAP( NCVT, VT( ISUB, 1 ), LDVT, VT( N+1-I, 1 ),
     $                     LDVT )
            IF( NRU.GT.0 )
     $         CALL CSWAP( NRU, U( 1, ISUB ), 1, U( 1, N+1-I ), 1 )
            IF( NCC.GT.0 )
     $         CALL CSWAP( NCC, C( ISUB, 1 ), LDC, C( N+1-I, 1 ), LDC )
         END IF
  190 CONTINUE
      GO TO 220
*
*     Maximum number of iterations exceeded, failure to converge
*
  200 CONTINUE
```

```
      INFO = 0
      DO 210 I = 1, N - 1
         IF( E( I ).NE.ZERO )
     $      INFO = INFO + 1
  210 CONTINUE
  220 CONTINUE
      RETURN
*
*     End of CBDSQR
*
      END
*  =====================================================================
      SUBROUTINE CBDSQR( UPLO, N, NCVT, NRU, NCC, D, E, VT, LDVT, U,
     $                   LDU, C, LDC, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDC, LDU, LDVT, N, NCC, NCVT, NRU
*     ..
*     .. Array Arguments ..
      REAL               D( * ), E( * ), RWORK( * )
      COMPLEX            C( LDC, * ), U( LDU, * ), VT( LDVT, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E0 )
      REAL               ONE
      PARAMETER          ( ONE = 1.0E0 )
      REAL               NEGONE
      PARAMETER          ( NEGONE = -1.0E0 )
      REAL               HNDRTH
      PARAMETER          ( HNDRTH = 0.01E0 )
      REAL               TEN
      PARAMETER          ( TEN = 10.0E0 )
      REAL               HNDRD
      PARAMETER          ( HNDRD = 100.0E0 )
      REAL               MEIGTH
      PARAMETER          ( MEIGTH = -0.125E0 )
      INTEGER            MAXITR
      PARAMETER          ( MAXITR = 6 )
*     ..
*     .. Local Scalars ..
```

```
      LOGICAL            LOWER, ROTATE
      INTEGER            I, IDIR, ISUB, ITER, J, LL, LLL, M, MAXIT, NM1,
     $                   NM12, NM13, OLDLL, OLDM
      REAL               ABSE, ABSS, COSL, COSR, CS, EPS, F, G, H, MU,
     $                   OLDCS, OLDSN, R, SHIFT, SIGMN, SIGMX, SINL,
     $                   SINR, SLL, SMAX, SMIN, SMINL, SMINOA,
     $                   SN, THRESH, TOL, TOLMUL, UNFL
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      REAL               SLAMCH
      EXTERNAL           LSAME, SLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLASR, CSROT, CSSCAL, CSWAP, SLARTG, SLAS2,
     $                   SLASQ1, SLASV2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, REAL, SIGN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      LOWER = LSAME( UPLO, 'L' )
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LOWER ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NCVT.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRU.LT.0 ) THEN
         INFO = -4
      ELSE IF( NCC.LT.0 ) THEN
         INFO = -5
      ELSE IF( ( NCVT.EQ.0 .AND. LDVT.LT.1 ) .OR.
     $         ( NCVT.GT.0 .AND. LDVT.LT.MAX( 1, N ) ) ) THEN
         INFO = -9
      ELSE IF( LDU.LT.MAX( 1, NRU ) ) THEN
         INFO = -11
      ELSE IF( ( NCC.EQ.0 .AND. LDC.LT.1 ) .OR.
     $         ( NCC.GT.0 .AND. LDC.LT.MAX( 1, N ) ) ) THEN
         INFO = -13
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CBDSQR', -INFO )
         RETURN
      END IF
      IF( N.EQ.0 )
```

```
     $   RETURN
      IF( N.EQ.1 )
     $   GO TO 160
*
*     ROTATE is true if any singular vectors desired, false otherwise
*
      ROTATE = ( NCVT.GT.0 ) .OR. ( NRU.GT.0 ) .OR. ( NCC.GT.0 )
*
*     If no singular vectors desired, use qd algorithm
*
      IF( .NOT.ROTATE ) THEN
         CALL SLASQ1( N, D, E, RWORK, INFO )
*
*     If INFO equals 2, dqds didn't finish, try to finish
*
         IF( INFO .NE. 2 ) RETURN
         INFO = 0
      END IF
*
      NM1 = N - 1
      NM12 = NM1 + NM1
      NM13 = NM12 + NM1
      IDIR = 0
*
*     Get machine constants
*
      EPS = SLAMCH( 'Epsilon' )
      UNFL = SLAMCH( 'Safe minimum' )
*
*     If matrix lower bidiagonal, rotate to be upper bidiagonal
*     by applying Givens rotations on the left
*
      IF( LOWER ) THEN
         DO 10 I = 1, N - 1
            CALL SLARTG( D( I ), E( I ), CS, SN, R )
            D( I ) = R
            E( I ) = SN*D( I+1 )
            D( I+1 ) = CS*D( I+1 )
            RWORK( I ) = CS
            RWORK( NM1+I ) = SN
   10    CONTINUE
*
*        Update singular vectors if desired
*
         IF( NRU.GT.0 )
     $      CALL CLASR( 'R', 'V', 'F', NRU, N, RWORK( 1 ), RWORK( N ),
     $                  U, LDU )
         IF( NCC.GT.0 )
     $      CALL CLASR( 'L', 'V', 'F', N, NCC, RWORK( 1 ), RWORK( N ),
     $                  C, LDC )
```

```
      END IF
*
*     Compute singular values to relative accuracy TOL
*     (By setting TOL to be negative, algorithm will compute
*     singular values to absolute accuracy ABS(TOL)*norm(input matrix))
*
      TOLMUL = MAX( TEN, MIN( HNDRD, EPS**MEIGTH ) )
      TOL = TOLMUL*EPS
*
*     Compute approximate maximum, minimum singular values
*
      SMAX = ZERO
      DO 20 I = 1, N
         SMAX = MAX( SMAX, ABS( D( I ) ) )
   20 CONTINUE
      DO 30 I = 1, N - 1
         SMAX = MAX( SMAX, ABS( E( I ) ) )
   30 CONTINUE
      SMINL = ZERO
      IF( TOL.GE.ZERO ) THEN
*
*        Relative accuracy desired
*
         SMINOA = ABS( D( 1 ) )
         IF( SMINOA.EQ.ZERO )
     $      GO TO 50
         MU = SMINOA
         DO 40 I = 2, N
            MU = ABS( D( I ) )*( MU / ( MU+ABS( E( I-1 ) ) ) )
            SMINOA = MIN( SMINOA, MU )
            IF( SMINOA.EQ.ZERO )
     $         GO TO 50
   40    CONTINUE
   50    CONTINUE
         SMINOA = SMINOA / SQRT( REAL( N ) )
         THRESH = MAX( TOL*SMINOA, MAXITR*N*N*UNFL )
      ELSE
*
*        Absolute accuracy desired
*
         THRESH = MAX( ABS( TOL )*SMAX, MAXITR*N*N*UNFL )
      END IF
*
*     Prepare for main iteration loop for the singular values
*     (MAXIT is the maximum number of passes through the inner
*     loop permitted before nonconvergence signalled.)
*
      MAXIT = MAXITR*N*N
      ITER = 0
      OLDLL = -1
```

```
      OLDM = -1
*
*     M points to last element of unconverged part of matrix
*
      M = N
*
*     Begin main iteration loop
*
   60 CONTINUE
*
*     Check for convergence or exceeding iteration count
*
      IF( M.LE.1 )
     $   GO TO 160
      IF( ITER.GT.MAXIT )
     $   GO TO 200
*
*     Find diagonal block of matrix to work on
*
      IF( TOL.LT.ZERO .AND. ABS( D( M ) ).LE.THRESH )
     $   D( M ) = ZERO
      SMAX = ABS( D( M ) )
      SMIN = SMAX
      DO 70 LLL = 1, M - 1
         LL = M - LLL
         ABSS = ABS( D( LL ) )
         ABSE = ABS( E( LL ) )
         IF( TOL.LT.ZERO .AND. ABSS.LE.THRESH )
     $      D( LL ) = ZERO
         IF( ABSE.LE.THRESH )
     $      GO TO 80
         SMIN = MIN( SMIN, ABSS )
         SMAX = MAX( SMAX, ABSS, ABSE )
   70 CONTINUE
      LL = 0
      GO TO 90
   80 CONTINUE
      E( LL ) = ZERO
*
*     Matrix splits since E(LL) = 0
*
      IF( LL.EQ.M-1 ) THEN
*
*        Convergence of bottom singular value, return to top of loop
*
         M = M - 1
         GO TO 60
      END IF
   90 CONTINUE
      LL = LL + 1
```

```
*
*     E(LL) through E(M-1) are nonzero, E(LL-1) is zero
*
      IF( LL.EQ.M-1 ) THEN
*
*        2 by 2 block, handle separately
*
         CALL SLASV2( D( M-1 ), E( M-1 ), D( M ), SIGMN, SIGMX, SINR,
     $                COSR, SINL, COSL )
         D( M-1 ) = SIGMX
         E( M-1 ) = ZERO
         D( M ) = SIGMN
*
*        Compute singular vectors, if desired
*
         IF( NCVT.GT.0 )
     $      CALL CSROT( NCVT, VT( M-1, 1 ), LDVT, VT( M, 1 ), LDVT,
     $                  COSR, SINR )
         IF( NRU.GT.0 )
     $      CALL CSROT( NRU, U( 1, M-1 ), 1, U( 1, M ), 1, COSL, SINL )
         IF( NCC.GT.0 )
     $      CALL CSROT( NCC, C( M-1, 1 ), LDC, C( M, 1 ), LDC, COSL,
     $                  SINL )
         M = M - 2
         GO TO 60
      END IF
*
*     If working on new submatrix, choose shift direction
*     (from larger end diagonal element towards smaller)
*
      IF( LL.GT.OLDM .OR. M.LT.OLDLL ) THEN
         IF( ABS( D( LL ) ).GE.ABS( D( M ) ) ) THEN
*
*           Chase bulge from top (big end) to bottom (small end)
*
            IDIR = 1
         ELSE
*
*           Chase bulge from bottom (big end) to top (small end)
*
            IDIR = 2
         END IF
      END IF
*
*     Apply convergence tests
*
      IF( IDIR.EQ.1 ) THEN
*
*        Run convergence test in forward direction
*        First apply standard test to bottom of matrix
```

```
*
         IF( ABS( E( M-1 ) ).LE.ABS( TOL )*ABS( D( M ) ) .OR.
     $       ( TOL.LT.ZERO .AND. ABS( E( M-1 ) ).LE.THRESH ) ) THEN
            E( M-1 ) = ZERO
            GO TO 60
         END IF
*
         IF( TOL.GE.ZERO ) THEN
*
*           If relative accuracy desired,
*           apply convergence criterion forward
*
            MU = ABS( D( LL ) )
            SMINL = MU
            DO 100 LLL = LL, M - 1
               IF( ABS( E( LLL ) ).LE.TOL*MU ) THEN
                  E( LLL ) = ZERO
                  GO TO 60
               END IF
               MU = ABS( D( LLL+1 ) )*( MU / ( MU+ABS( E( LLL ) ) ) )
               SMINL = MIN( SMINL, MU )
  100       CONTINUE
         END IF
*
      ELSE
*
*        Run convergence test in backward direction
*        First apply standard test to top of matrix
*
         IF( ABS( E( LL ) ).LE.ABS( TOL )*ABS( D( LL ) ) .OR.
     $       ( TOL.LT.ZERO .AND. ABS( E( LL ) ).LE.THRESH ) ) THEN
            E( LL ) = ZERO
            GO TO 60
         END IF
*
         IF( TOL.GE.ZERO ) THEN
*
*           If relative accuracy desired,
*           apply convergence criterion backward
*
            MU = ABS( D( M ) )
            SMINL = MU
            DO 110 LLL = M - 1, LL, -1
               IF( ABS( E( LLL ) ).LE.TOL*MU ) THEN
                  E( LLL ) = ZERO
                  GO TO 60
               END IF
               MU = ABS( D( LLL ) )*( MU / ( MU+ABS( E( LLL ) ) ) )
               SMINL = MIN( SMINL, MU )
  110       CONTINUE
```

```
         END IF
      END IF
      OLDLL = LL
      OLDM = M
*
*     Compute shift.  First, test if shifting would ruin relative
*     accuracy, and if so set the shift to zero.
*
      IF( TOL.GE.ZERO .AND. N*TOL*( SMINL / SMAX ).LE.
     $    MAX( EPS, HNDRTH*TOL ) ) THEN
*
*        Use a zero shift to avoid loss of relative accuracy
*
         SHIFT = ZERO
      ELSE
*
*        Compute the shift from 2-by-2 block at end of matrix
*
         IF( IDIR.EQ.1 ) THEN
            SLL = ABS( D( LL ) )
            CALL SLAS2( D( M-1 ), E( M-1 ), D( M ), SHIFT, R )
         ELSE
            SLL = ABS( D( M ) )
            CALL SLAS2( D( LL ), E( LL ), D( LL+1 ), SHIFT, R )
         END IF
*
*        Test if shift negligible, and if so set to zero
*
         IF( SLL.GT.ZERO ) THEN
            IF( ( SHIFT / SLL )**2.LT.EPS )
     $         SHIFT = ZERO
         END IF
      END IF
*
*     Increment iteration count
*
      ITER = ITER + M - LL
*
*     If SHIFT = 0, do simplified QR iteration
*
      IF( SHIFT.EQ.ZERO ) THEN
         IF( IDIR.EQ.1 ) THEN
*
*           Chase bulge from top to bottom
*           Save cosines and sines for later singular vector updates
*
            CS = ONE
            OLDCS = ONE
            DO 120 I = LL, M - 1
               CALL SLARTG( D( I )*CS, E( I ), CS, SN, R )
```

```
               IF( I.GT.LL )
     $            E( I-1 ) = OLDSN*R
               CALL SLARTG( OLDCS*R, D( I+1 )*SN, OLDCS, OLDSN, D( I ) )
               RWORK( I-LL+1 ) = CS
               RWORK( I-LL+1+NM1 ) = SN
               RWORK( I-LL+1+NM12 ) = OLDCS
               RWORK( I-LL+1+NM13 ) = OLDSN
  120       CONTINUE
            H = D( M )*CS
            D( M ) = H*OLDCS
            E( M-1 ) = H*OLDSN
*
*           Update singular vectors
*
            IF( NCVT.GT.0 )
     $         CALL CLASR( 'L', 'V', 'F', M-LL+1, NCVT, RWORK( 1 ),
     $                     RWORK( N ), VT( LL, 1 ), LDVT )
            IF( NRU.GT.0 )
     $         CALL CLASR( 'R', 'V', 'F', NRU, M-LL+1, RWORK( NM12+1 ),
     $                     RWORK( NM13+1 ), U( 1, LL ), LDU )
            IF( NCC.GT.0 )
     $         CALL CLASR( 'L', 'V', 'F', M-LL+1, NCC, RWORK( NM12+1 ),
     $                     RWORK( NM13+1 ), C( LL, 1 ), LDC )
*
*           Test convergence
*
            IF( ABS( E( M-1 ) ).LE.THRESH )
     $         E( M-1 ) = ZERO
*
         ELSE
*
*           Chase bulge from bottom to top
*           Save cosines and sines for later singular vector updates
*
            CS = ONE
            OLDCS = ONE
            DO 130 I = M, LL + 1, -1
               CALL SLARTG( D( I )*CS, E( I-1 ), CS, SN, R )
               IF( I.LT.M )
     $            E( I ) = OLDSN*R
               CALL SLARTG( OLDCS*R, D( I-1 )*SN, OLDCS, OLDSN, D( I ) )
               RWORK( I-LL ) = CS
               RWORK( I-LL+NM1 ) = -SN
               RWORK( I-LL+NM12 ) = OLDCS
               RWORK( I-LL+NM13 ) = -OLDSN
  130       CONTINUE
            H = D( LL )*CS
            D( LL ) = H*OLDCS
            E( LL ) = H*OLDSN
*
```

```
*          Update singular vectors
*
           IF( NCVT.GT.0 )
     $        CALL CLASR( 'L', 'V', 'B', M-LL+1, NCVT, RWORK( NM12+1 ),
     $                    RWORK( NM13+1 ), VT( LL, 1 ), LDVT )
           IF( NRU.GT.0 )
     $        CALL CLASR( 'R', 'V', 'B', NRU, M-LL+1, RWORK( 1 ),
     $                    RWORK( N ), U( 1, LL ), LDU )
           IF( NCC.GT.0 )
     $        CALL CLASR( 'L', 'V', 'B', M-LL+1, NCC, RWORK( 1 ),
     $                    RWORK( N ), C( LL, 1 ), LDC )
*
*          Test convergence
*
           IF( ABS( E( LL ) ).LE.THRESH )
     $        E( LL ) = ZERO
        END IF
     ELSE
*
*       Use nonzero shift
*
        IF( IDIR.EQ.1 ) THEN
*
*          Chase bulge from top to bottom
*          Save cosines and sines for later singular vector updates
*
           F = ( ABS( D( LL ) )-SHIFT )*
     $         ( SIGN( ONE, D( LL ) )+SHIFT / D( LL ) )
           G = E( LL )
           DO 140 I = LL, M - 1
              CALL SLARTG( F, G, COSR, SINR, R )
              IF( I.GT.LL )
     $           E( I-1 ) = R
              F = COSR*D( I ) + SINR*E( I )
              E( I ) = COSR*E( I ) - SINR*D( I )
              G = SINR*D( I+1 )
              D( I+1 ) = COSR*D( I+1 )
              CALL SLARTG( F, G, COSL, SINL, R )
              D( I ) = R
              F = COSL*E( I ) + SINL*D( I+1 )
              D( I+1 ) = COSL*D( I+1 ) - SINL*E( I )
              IF( I.LT.M-1 ) THEN
                 G = SINL*E( I+1 )
                 E( I+1 ) = COSL*E( I+1 )
              END IF
              RWORK( I-LL+1 ) = COSR
              RWORK( I-LL+1+NM1 ) = SINR
              RWORK( I-LL+1+NM12 ) = COSL
              RWORK( I-LL+1+NM13 ) = SINL
  140      CONTINUE
```

```
            E( M-1 ) = F
*
*           Update singular vectors
*
            IF( NCVT.GT.0 )
     $         CALL CLASR( 'L', 'V', 'F', M-LL+1, NCVT, RWORK( 1 ),
     $                     RWORK( N ), VT( LL, 1 ), LDVT )
            IF( NRU.GT.0 )
     $         CALL CLASR( 'R', 'V', 'F', NRU, M-LL+1, RWORK( NM12+1 ),
     $                     RWORK( NM13+1 ), U( 1, LL ), LDU )
            IF( NCC.GT.0 )
     $         CALL CLASR( 'L', 'V', 'F', M-LL+1, NCC, RWORK( NM12+1 ),
     $                     RWORK( NM13+1 ), C( LL, 1 ), LDC )
*
*           Test convergence
*
            IF( ABS( E( M-1 ) ).LE.THRESH )
     $         E( M-1 ) = ZERO
*
         ELSE
*
*           Chase bulge from bottom to top
*           Save cosines and sines for later singular vector updates
*
            F = ( ABS( D( M ) )-SHIFT )*( SIGN( ONE, D( M ) )+SHIFT /
     $          D( M ) )
            G = E( M-1 )
            DO 150 I = M, LL + 1, -1
               CALL SLARTG( F, G, COSR, SINR, R )
               IF( I.LT.M )
     $            E( I ) = R
               F = COSR*D( I ) + SINR*E( I-1 )
               E( I-1 ) = COSR*E( I-1 ) - SINR*D( I )
               G = SINR*D( I-1 )
               D( I-1 ) = COSR*D( I-1 )
               CALL SLARTG( F, G, COSL, SINL, R )
               D( I ) = R
               F = COSL*E( I-1 ) + SINL*D( I-1 )
               D( I-1 ) = COSL*D( I-1 ) - SINL*E( I-1 )
               IF( I.GT.LL+1 ) THEN
                  G = SINL*E( I-2 )
                  E( I-2 ) = COSL*E( I-2 )
               END IF
               RWORK( I-LL ) = COSR
               RWORK( I-LL+NM1 ) = -SINR
               RWORK( I-LL+NM12 ) = COSL
               RWORK( I-LL+NM13 ) = -SINL
  150       CONTINUE
            E( LL ) = F
*
```

```
*           Test convergence
*
            IF( ABS( E( LL ) ).LE.THRESH )
     $         E( LL ) = ZERO
*
*           Update singular vectors if desired
*
            IF( NCVT.GT.0 )
     $         CALL CLASR( 'L', 'V', 'B', M-LL+1, NCVT, RWORK( NM12+1 ),
     $                     RWORK( NM13+1 ), VT( LL, 1 ), LDVT )
            IF( NRU.GT.0 )
     $         CALL CLASR( 'R', 'V', 'B', NRU, M-LL+1, RWORK( 1 ),
     $                     RWORK( N ), U( 1, LL ), LDU )
            IF( NCC.GT.0 )
     $         CALL CLASR( 'L', 'V', 'B', M-LL+1, NCC, RWORK( 1 ),
     $                     RWORK( N ), C( LL, 1 ), LDC )
         END IF
      END IF
*
*     QR iteration finished, go back and check convergence
*
      GO TO 60
*
*     All singular values converged, so make them positive
*
  160 CONTINUE
      DO 170 I = 1, N
         IF( D( I ).LT.ZERO ) THEN
            D( I ) = -D( I )
*
*           Change sign of singular vectors, if desired
*
            IF( NCVT.GT.0 )
     $         CALL CSSCAL( NCVT, NEGONE, VT( I, 1 ), LDVT )
         END IF
  170 CONTINUE
*
*     Sort the singular values into decreasing order (insertion sort on
*     singular values, but only one transposition per singular vector)
*
      DO 190 I = 1, N - 1
*
*        Scan for smallest D(I)
*
         ISUB = 1
         SMIN = D( 1 )
         DO 180 J = 2, N + 1 - I
            IF( D( J ).LE.SMIN ) THEN
               ISUB = J
               SMIN = D( J )
```

```
            END IF
  180    CONTINUE
         IF( ISUB.NE.N+1-I ) THEN
*
*           Swap singular values and vectors
*
            D( ISUB ) = D( N+1-I )
            D( N+1-I ) = SMIN
            IF( NCVT.GT.0 )
     $         CALL CSWAP( NCVT, VT( ISUB, 1 ), LDVT, VT( N+1-I, 1 ),
     $                     LDVT )
            IF( NRU.GT.0 )
     $         CALL CSWAP( NRU, U( 1, ISUB ), 1, U( 1, N+1-I ), 1 )
            IF( NCC.GT.0 )
     $         CALL CSWAP( NCC, C( ISUB, 1 ), LDC, C( N+1-I, 1 ), LDC )
         END IF
  190 CONTINUE
      GO TO 220
*
*     Maximum number of iterations exceeded, failure to converge
*
  200 CONTINUE
      INFO = 0
      DO 210 I = 1, N - 1
         IF( E( I ).NE.ZERO )
     $      INFO = INFO + 1
  210 CONTINUE
  220 CONTINUE
      RETURN
*
*     End of CBDSQR
*
      END
```

———-

90.111.4 lisp code

— cbdsqr.lisp —

```
(let* ((zero 0.0)
       (one 1.0)
       (negone (- 1.0))
       (hndrth 0.01)
       (ten 10.0)
       (hndrd 100.0)
       (meigth (- 0.125))
```

```
      (maxitr 6))
  (declare (type (single-float 0.0 0.0) zero)
           (type (single-float 1.0 1.0) one)
           (type (single-float) negone)
           (type (single-float 0.01 0.01) hndrth)
           (type (single-float 10.0 10.0) ten)
           (type (single-float 100.0 100.0) hndrd)
           (type (single-float) meigth)
           (type (f2cl-lib:integer4 6 6) maxitr)
           (ignorable zero one negone hndrth ten hndrd meigth maxitr))
  (defun cbdsqr (uplo n ncvt nru ncc d e vt ldvt u ldu c ldc rwork info)
    (declare (type (array f2cl-lib:complex8 (*)) c u vt)
             (type (array single-float (*)) rwork e d)
             (type (f2cl-lib:integer4) info ldc ldu ldvt ncc nru ncvt n)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%)
         (vt f2cl-lib:complex8 vt-%data% vt-%offset%)
         (u f2cl-lib:complex8 u-%data% u-%offset%)
         (c f2cl-lib:complex8 c-%data% c-%offset%))
      (prog ((abse 0.0) (abss 0.0) (cosl 0.0) (cosr 0.0) (cs 0.0) (eps 0.0)
             (f 0.0) (g 0.0) (h 0.0) (mu 0.0) (oldcs 0.0) (oldsn 0.0) (r 0.0)
             (shift 0.0) (sigmn 0.0) (sigmx 0.0) (sinl 0.0) (sinr 0.0)
             (sll 0.0) (smax 0.0) (smin 0.0) (sminl 0.0) (sminoa 0.0) (sn 0.0)
             (thresh 0.0) (tol 0.0) (tolmul 0.0) (unfl 0.0) (i 0) (idir 0)
             (isub 0) (iter 0) (j 0) (ll 0) (lll 0) (m 0) (maxit 0) (nm1 0)
             (nm12 0) (nm13 0) (oldll 0) (oldm 0) (lower nil) (rotate nil))
        (declare
         (type (single-float) abse abss cosl cosr cs eps f g h mu oldcs oldsn r
          shift sigmn sigmx sinl sinr sll smax smin sminl sminoa sn thresh tol
          tolmul unfl)
         (type (f2cl-lib:integer4) i idir isub iter j ll lll m maxit nm1 nm12
          nm13 oldll oldm)
         (type f2cl-lib:logical lower rotate))
        (setf info 0)
        (setf lower
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame uplo "L")
                  (declare (ignore var-1))
                  (when var-0 (setf uplo var-0))
                  ret-val))
        (cond
         ((and
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
```

```
      (when var-0 (setf uplo var-0))
      ret-val))
   (not lower))
  (setf info -1))
 ((< n 0) (setf info -2)) ((< ncvt 0) (setf info -3))
 ((< nru 0) (setf info -4)) ((< ncc 0) (setf info -5))
 ((or (and (= ncvt 0) (< ldvt 1))
      (and (> ncvt 0)
           (< ldvt
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 n)))))
  (setf info -9))
 ((< ldu (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nru)))
  (setf info -11))
 ((or (and (= ncc 0) (< ldc 1))
      (and (> ncc 0)
           (< ldc
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 n)))))
  (setf info -13)))
(cond
 ((/= info 0) (xerbla "CBDSQR" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(if (= n 1)
    (go label160))
(setf rotate (or (> ncvt 0) (> nru 0) (> ncc 0)))
(cond
 ((not rotate)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (slasq1 n d e rwork info)
    (declare (ignore var-1 var-2 var-3))
    (when var-0 (setf n var-0))
    (when var-4 (setf info var-4)))
  (if (/= info 2)
      (go end_label))
  (setf info 0)))
(setf nm1 (f2cl-lib:int-sub n 1))
(setf nm12 (f2cl-lib:int-add nm1 nm1))
(setf nm13 (f2cl-lib:int-add nm12 nm1))
(setf idir 0)
(setf eps (slamch "Epsilon"))
(setf unfl (slamch "Safe minimum"))
(cond
 (lower
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
            (slartg
             (f2cl-lib:array-slice d-%data% single-float (i) ((1 *))
                                   d-%offset%)
             (f2cl-lib:array-slice e-%data% single-float (i) ((1 *))
                                   e-%offset%)
             cs sn r)
          (declare (ignore var-0 var-1))
          (when var-2 (setf cs var-2))
          (when var-3 (setf sn var-3))
          (when var-4 (setf r var-4)))
        (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) r)
        (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                (* sn
                   (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                  ((1 *)) d-%offset%)))
        (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                             d-%offset%)
                (* cs
                   (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                  ((1 *)) d-%offset%)))
        (setf (f2cl-lib:fref rwork-%data% (i) ((1 *)) rwork-%offset%) cs)
        (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-add nm1 i))
                             ((1 *)) rwork-%offset%)
                sn)
      label10))
    (if (> nru 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (clasr "R" "V" "F" nru n
             (f2cl-lib:array-slice rwork-%data% single-float (1) ((1 *))
                                   rwork-%offset%)
             (f2cl-lib:array-slice rwork-%data% single-float (n) ((1 *))
                                   rwork-%offset%)
             u ldu)
          (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
          (when var-3 (setf nru var-3))
          (when var-4 (setf n var-4))
          (when var-8 (setf ldu var-8))))
    (if (> ncc 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (clasr "L" "V" "F" n ncc
             (f2cl-lib:array-slice rwork-%data% single-float (1) ((1 *))
                                   rwork-%offset%)
             (f2cl-lib:array-slice rwork-%data% single-float (n) ((1 *))
                                   rwork-%offset%)
             c ldc)
          (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
          (when var-3 (setf n var-3))
          (when var-4 (setf ncc var-4))
```

```
        (when var-8 (setf ldc var-8))))))
(setf tolmul (max ten (min hndrd (expt eps meigth))))
(setf tol (* tolmul eps))
(setf smax zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf smax
            (max smax
                 (abs
                  (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))))
   label20))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (setf smax
            (max smax
                 (abs
                  (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))))
   label30))
(setf sminl zero)
(cond
 ((>= tol zero)
  (tagbody
    (setf sminoa (abs (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)))
    (if (= sminoa zero)
        (go label50))
    (setf mu sminoa)
    (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf mu
                (*
                 (abs (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
                 (/ mu
                    (+ mu
                       (abs
                        (f2cl-lib:fref e-%data%
                                       ((f2cl-lib:int-sub i 1)) ((1 *))
                                       e-%offset%))))))
        (setf sminoa (min sminoa mu))
        (if (= sminoa zero)
            (go label50))
       label40))
   label50
    (setf sminoa (/ sminoa (f2cl-lib:fsqrt (f2cl-lib:freal n))))
    (setf thresh (max (* tol sminoa) (* maxitr n n unfl)))))
 (t (setf thresh (max (* (abs tol) smax) (* maxitr n n unfl)))))
(setf maxit (f2cl-lib:int-mul maxitr n n))
(setf iter 0)
```

```
 (setf oldll -1)
 (setf oldm -1)
 (setf m n)
label60
 (if (<= m 1)
     (go label160))
 (if (> iter maxit)
     (go label200))
 (if (and (< tol zero)
          (<= (abs (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%))
              thresh))
     (setf (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%) zero))
 (setf smax (abs (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%)))
 (setf smin smax)
 (f2cl-lib:fdo (lll 1 (f2cl-lib:int-add lll 1))
               ((> lll (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
   (tagbody
     (setf ll (f2cl-lib:int-sub m lll))
     (setf abss (abs (f2cl-lib:fref d-%data% (ll) ((1 *)) d-%offset%)))
     (setf abse (abs (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%)))
     (if (and (< tol zero) (<= abss thresh))
         (setf (f2cl-lib:fref d-%data% (ll) ((1 *)) d-%offset%) zero))
     (if (<= abse thresh)
         (go label80))
     (setf smin (min smin abss))
     (setf smax (max smax abss abse))
    label70))
 (setf ll 0)
 (go label90)
label80
 (setf (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%) zero)
 (cond
  ((= ll (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
   (setf m (f2cl-lib:int-sub m 1)) (go label60)))
label90
 (setf ll (f2cl-lib:int-add ll 1))
 (cond
  ((= ll (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (slasv2
        (f2cl-lib:array-slice d-%data% single-float
                              ((+ m (f2cl-lib:int-sub 1))) ((1 *))
                              d-%offset%)
        (f2cl-lib:array-slice e-%data% single-float
                              ((+ m (f2cl-lib:int-sub 1))) ((1 *))
                              e-%offset%)
        (f2cl-lib:array-slice d-%data% single-float (m) ((1 *))
                              d-%offset%)
        sigmn sigmx sinr cosr sinl cosl)
```

```
  (declare (ignore var-0 var-1 var-2))
  (when var-3 (setf sigmn var-3))
  (when var-4 (setf sigmx var-4))
  (when var-5 (setf sinr var-5))
  (when var-6 (setf cosr var-6))
  (when var-7 (setf sinl var-7))
  (when var-8 (setf cosl var-8)))
(setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                     d-%offset%)
        sigmx)
(setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                     e-%offset%)
        zero)
(setf (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%) sigmn)
(if (> ncvt 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (csrot ncvt
         (f2cl-lib:array-slice vt-%data% f2cl-lib:complex8
                               ((+ m (f2cl-lib:int-sub 1)) 1)
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt
         (f2cl-lib:array-slice vt-%data% f2cl-lib:complex8 (m 1)
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt cosr sinr)
      (declare (ignore var-1 var-3))
      (when var-0 (setf ncvt var-0))
      (when var-2 (setf ldvt var-2))
      (when var-4 (setf ldvt var-4))
      (when var-5 (setf cosr var-5))
      (when var-6 (setf sinr var-6))))
(if (> nru 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (csrot nru
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8
                               (1 (f2cl-lib:int-sub m 1))
                               ((1 ldu) (1 *)) u-%offset%)
         1
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 m)
                               ((1 ldu) (1 *)) u-%offset%)
         1 cosl sinl)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf nru var-0))
      (when var-5 (setf cosl var-5))
      (when var-6 (setf sinl var-6))))
(if (> ncc 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (csrot ncc
         (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                               ((+ m (f2cl-lib:int-sub 1)) 1)
                               ((1 ldc) (1 *)) c-%offset%)
```

```
            ldc
            (f2cl-lib:array-slice c-%data% f2cl-lib:complex8 (m 1)
                                  ((1 ldc) (1 *)) c-%offset%)
            ldc cosl sinl)
          (declare (ignore var-1 var-3))
          (when var-0 (setf ncc var-0))
          (when var-2 (setf ldc var-2))
          (when var-4 (setf ldc var-4))
          (when var-5 (setf cosl var-5))
          (when var-6 (setf sinl var-6))))
  (setf m (f2cl-lib:int-sub m 2)) (go label60)))
(cond
 ((or (> ll oldm) (< m oldll))
  (cond
   ((>= (abs (f2cl-lib:fref d (ll) ((1 *))))
        (abs (f2cl-lib:fref d (m) ((1 *)))))
    (setf idir 1))
   (t (setf idir 2)))))
(cond
 ((= idir 1)
  (cond
   ((or
     (<=
      (abs
       (f2cl-lib:fref e ((f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
                      ((1 *))))
      (* (abs tol) (abs (f2cl-lib:fref d (m) ((1 *))))))
     (and (< tol zero)
          (<=
           (abs
            (f2cl-lib:fref e
                           ((f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
                           ((1 *))))
           thresh)))
    (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                         e-%offset%)
            zero)
    (go label60)))
  (cond
   ((>= tol zero)
    (setf mu (abs (f2cl-lib:fref d-%data% (ll) ((1 *)) d-%offset%)))
    (setf sminl mu)
    (f2cl-lib:fdo (lll ll (f2cl-lib:int-add lll 1))
                  ((> lll (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (cond
         ((<= (abs (f2cl-lib:fref e (lll) ((1 *)))) (* tol mu))
          (setf (f2cl-lib:fref e-%data% (lll) ((1 *)) e-%offset%) zero)
          (go label60)))
```

```
        (setf mu
                (*
                 (abs
                  (f2cl-lib:fref d-%data% ((f2cl-lib:int-add lll 1))
                                 ((1 *)) d-%offset%))
                 (/ mu
                    (+ mu
                       (abs
                        (f2cl-lib:fref e-%data% (lll) ((1 *))
                                       e-%offset%))))))
        (setf sminl (min sminl mu))
       label100)))))
 (t
  (cond
   ((or
     (<= (abs (f2cl-lib:fref e (ll) ((1 *))))
         (* (abs tol) (abs (f2cl-lib:fref d (ll) ((1 *))))))
     (and (< tol zero)
          (<= (abs (f2cl-lib:fref e (ll) ((1 *)))) thresh)))
    (setf (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%) zero)
    (go label60)))
  (cond
   ((>= tol zero)
    (setf mu (abs (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%)))
    (setf sminl mu)
    (f2cl-lib:fdo (lll (f2cl-lib:int-add m (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add lll (f2cl-lib:int-sub 1)))
                  ((> lll ll) nil)
      (tagbody
        (cond
         ((<= (abs (f2cl-lib:fref e (lll) ((1 *)))) (* tol mu))
          (setf (f2cl-lib:fref e-%data% (lll) ((1 *)) e-%offset%) zero)
          (go label60)))
        (setf mu
                (*
                 (abs
                  (f2cl-lib:fref d-%data% (lll) ((1 *)) d-%offset%))
                 (/ mu
                    (+ mu
                       (abs
                        (f2cl-lib:fref e-%data% (lll) ((1 *))
                                       e-%offset%))))))
        (setf sminl (min sminl mu))
       label110))))))
(setf oldll ll)
(setf oldm m)
(cond
 ((and (>= tol zero)
       (<= (* n tol (f2cl-lib:f2cl/ sminl smax))
           (max eps (* hndrth tol))))
```

```
  (setf shift zero))
 (t
  (cond
   ((= idir 1)
    (setf sll (abs (f2cl-lib:fref d-%data% (ll) ((1 *)) d-%offset%)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (slas2
         (f2cl-lib:array-slice d-%data% single-float
                               ((+ m (f2cl-lib:int-sub 1))) ((1 *))
                               d-%offset%)
         (f2cl-lib:array-slice e-%data% single-float
                               ((+ m (f2cl-lib:int-sub 1))) ((1 *))
                               e-%offset%)
         (f2cl-lib:array-slice d-%data% single-float (m) ((1 *))
                               d-%offset%)
         shift r)
      (declare (ignore var-0 var-1 var-2))
      (when var-3 (setf shift var-3))
      (when var-4 (setf r var-4))))
   (t (setf sll (abs (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (slas2
         (f2cl-lib:array-slice d-%data% single-float (ll) ((1 *))
                               d-%offset%)
         (f2cl-lib:array-slice e-%data% single-float (ll) ((1 *))
                               e-%offset%)
         (f2cl-lib:array-slice d-%data% single-float ((+ ll 1)) ((1 *))
                               d-%offset%)
         shift r)
      (declare (ignore var-0 var-1 var-2))
      (when var-3 (setf shift var-3))
      (when var-4 (setf r var-4)))))
  (cond
   ((> sll zero)
    (if (< (expt (/ shift sll) 2) eps)
        (setf shift zero))))))
(setf iter (f2cl-lib:int-sub (f2cl-lib:int-add iter m) ll))
(cond
 ((= shift zero)
  (cond
   ((= idir 1) (setf cs one) (setf oldcs one)
    (f2cl-lib:fdo (i ll (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (slartg
             (* (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) cs)
             (f2cl-lib:array-slice e-%data% single-float (i) ((1 *))
                                   e-%offset%)
             cs sn r)
```

```
        (declare (ignore var-0 var-1))
        (when var-2 (setf cs var-2))
        (when var-3 (setf sn var-3))
        (when var-4 (setf r var-4)))
      (if (> i ll)
          (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                ((1 *)) e-%offset%)
                (* oldsn r)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (slartg (* oldcs r)
           (*
            (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                           d-%offset%)
            sn)
           oldcs oldsn
           (f2cl-lib:array-slice d-%data% single-float (i) ((1 *))
                                 d-%offset%))
        (declare (ignore var-0 var-1 var-4))
        (when var-2 (setf oldcs var-2))
        (when var-3 (setf oldsn var-3)))
      (setf (f2cl-lib:fref rwork-%data%
                           ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                              1))
                           ((1 *)) rwork-%offset%)
              cs)
      (setf (f2cl-lib:fref rwork-%data%
                           ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                              1 nm1))
                           ((1 *)) rwork-%offset%)
              sn)
      (setf (f2cl-lib:fref rwork-%data%
                           ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                              1 nm12))
                           ((1 *)) rwork-%offset%)
              oldcs)
      (setf (f2cl-lib:fref rwork-%data%
                           ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                              1 nm13))
                           ((1 *)) rwork-%offset%)
              oldsn)
    label120))
  (setf h (* (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%) cs))
  (setf (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%) (* h oldcs))
  (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                       e-%offset%)
        (* h oldsn))
  (if (> ncvt 0)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (clasr "L" "V" "F"
```

```
            (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1) ncvt
            (f2cl-lib:array-slice rwork-%data% single-float (1)
                                  ((1 *)) rwork-%offset%)
            (f2cl-lib:array-slice rwork-%data% single-float (n)
                                  ((1 *)) rwork-%offset%)
            (f2cl-lib:array-slice vt-%data% f2cl-lib:complex8 (ll 1)
                                  ((1 ldvt) (1 *)) vt-%offset%)
            ldvt)
          (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
          (when var-4 (setf ncvt var-4))
          (when var-8 (setf ldvt var-8))))
    (if (> nru 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (clasr "R" "V" "F" nru
             (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1)
             (f2cl-lib:array-slice rwork-%data% single-float
                                   ((+ nm12 1)) ((1 *)) rwork-%offset%)
             (f2cl-lib:array-slice rwork-%data% single-float
                                   ((+ nm13 1)) ((1 *)) rwork-%offset%)
             (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 ll)
                                   ((1 ldu) (1 *)) u-%offset%)
             ldu)
          (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-7))
          (when var-3 (setf nru var-3))
          (when var-8 (setf ldu var-8))))
    (if (> ncc 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (clasr "L" "V" "F"
             (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1) ncc
             (f2cl-lib:array-slice rwork-%data% single-float
                                   ((+ nm12 1)) ((1 *)) rwork-%offset%)
             (f2cl-lib:array-slice rwork-%data% single-float
                                   ((+ nm13 1)) ((1 *)) rwork-%offset%)
             (f2cl-lib:array-slice c-%data% f2cl-lib:complex8 (ll 1)
                                   ((1 ldc) (1 *)) c-%offset%)
             ldc)
          (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
          (when var-4 (setf ncc var-4))
          (when var-8 (setf ldc var-8))))
    (if (<=
         (abs
          (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                         e-%offset%))
         thresh)
        (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                             e-%offset%)
              zero)))
    (t (setf cs one) (setf oldcs one)
```

```
(f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i (f2cl-lib:int-add ll 1)) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (slartg
         (* (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) cs)
         (f2cl-lib:array-slice e-%data% single-float
                               ((+ i (f2cl-lib:int-sub 1))) ((1 *))
                               e-%offset%)
         cs sn r)
      (declare (ignore var-0 var-1))
      (when var-2 (setf cs var-2))
      (when var-3 (setf sn var-3))
      (when var-4 (setf r var-4)))
    (if (< i m)
        (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                (* oldsn r)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (slartg (* oldcs r)
         (*
          (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub i 1)) ((1 *))
                         d-%offset%)
          sn)
         oldcs oldsn
         (f2cl-lib:array-slice d-%data% single-float (i) ((1 *))
                               d-%offset%))
      (declare (ignore var-0 var-1 var-4))
      (when var-2 (setf oldcs var-2))
      (when var-3 (setf oldsn var-3)))
    (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-sub i ll))
                         ((1 *)) rwork-%offset%)
            cs)
    (setf (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                            nm1))
                         ((1 *)) rwork-%offset%)
            (- sn))
    (setf (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                            nm12))
                         ((1 *)) rwork-%offset%)
            oldcs)
    (setf (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                            nm13))
                         ((1 *)) rwork-%offset%)
            (- oldsn))
   label130))
(setf h (* (f2cl-lib:fref d-%data% (ll) ((1 *)) d-%offset%) cs))
(setf (f2cl-lib:fref d-%data% (ll) ((1 *)) d-%offset%) (* h oldcs))
```

```
(setf (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%) (* h oldsn))
(if (> ncvt 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clasr "L" "V" "B"
         (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1) ncvt
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ nm12 1)) ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ nm13 1)) ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice vt-%data% f2cl-lib:complex8 (ll 1)
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt)
      (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
      (when var-4 (setf ncvt var-4))
      (when var-8 (setf ldvt var-8))))
(if (> nru 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clasr "R" "V" "B" nru
         (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1)
         (f2cl-lib:array-slice rwork-%data% single-float (1)
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice rwork-%data% single-float (n)
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 ll)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-7))
      (when var-3 (setf nru var-3))
      (when var-8 (setf ldu var-8))))
(if (> ncc 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clasr "L" "V" "B"
         (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1) ncc
         (f2cl-lib:array-slice rwork-%data% single-float (1)
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice rwork-%data% single-float (n)
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice c-%data% f2cl-lib:complex8 (ll 1)
                               ((1 ldc) (1 *)) c-%offset%)
         ldc)
      (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
      (when var-4 (setf ncc var-4))
      (when var-8 (setf ldc var-8))))
(if (<= (abs (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%))
        thresh)
    (setf (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%)
            zero)))))
```

```
(t
 (cond
  ((= idir 1)
   (setf f
           (*
            (- (abs (f2cl-lib:fref d-%data% (ll) ((1 *)) d-%offset%))
               shift)
            (+
             (f2cl-lib:sign one
                            (f2cl-lib:fref d-%data% (ll) ((1 *))
                                           d-%offset%))
             (/ shift
                (f2cl-lib:fref d-%data% (ll) ((1 *)) d-%offset%)))))
   (setf g (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%))
   (f2cl-lib:fdo (i ll (f2cl-lib:int-add i 1))
                 ((> i (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (slartg f g cosr sinr r)
         (declare (ignore))
         (when var-0 (setf f var-0))
         (when var-1 (setf g var-1))
         (when var-2 (setf cosr var-2))
         (when var-3 (setf sinr var-3))
         (when var-4 (setf r var-4)))
       (if (> i ll)
           (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                 ((1 *)) e-%offset%)
                 r))
       (setf f
               (+
                (* cosr
                   (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
                (* sinr
                   (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))))
       (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
               (-
                (* cosr
                   (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))
                (* sinr
                   (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))))
       (setf g
               (* sinr
                  (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) d-%offset%)))
       (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                            d-%offset%)
               (* cosr
                  (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                 ((1 *)) d-%offset%)))
```

```
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (slartg f g cosl sinl r)
  (declare (ignore))
  (when var-0 (setf f var-0))
  (when var-1 (setf g var-1))
  (when var-2 (setf cosl var-2))
  (when var-3 (setf sinl var-3))
  (when var-4 (setf r var-4)))
(setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) r)
(setf f
        (+
         (* cosl
            (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))
         (* sinl
            (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                           ((1 *)) d-%offset%))))
(setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                     d-%offset%)
        (-
         (* cosl
            (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                           ((1 *)) d-%offset%))
         (* sinl
            (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))))
(cond
 ((< i (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
  (setf g
          (* sinl
             (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1))
                            ((1 *)) e-%offset%)))
  (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1))
                        ((1 *)) e-%offset%)
          (* cosl
             (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1))
                            ((1 *)) e-%offset%)))))
(setf (f2cl-lib:fref rwork-%data%
                     ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                        1))
                     ((1 *)) rwork-%offset%)
        cosr)
(setf (f2cl-lib:fref rwork-%data%
                     ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                        1 nm1))
                     ((1 *)) rwork-%offset%)
        sinr)
(setf (f2cl-lib:fref rwork-%data%
                     ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                        1 nm12))
                     ((1 *)) rwork-%offset%)
        cosl)
```

```
    (setf (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                            1 nm13))
                         ((1 *)) rwork-%offset%)
          sinl)
   label140))
(setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                     e-%offset%)
      f)
(if (> ncvt 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clasr "L" "V" "F"
         (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1) ncvt
         (f2cl-lib:array-slice rwork-%data% single-float (1)
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice rwork-%data% single-float (n)
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice vt-%data% f2cl-lib:complex8 (ll 1)
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt)
      (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
      (when var-4 (setf ncvt var-4))
      (when var-8 (setf ldvt var-8))))
(if (> nru 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clasr "R" "V" "F" nru
         (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1)
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ nm12 1)) ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ nm13 1)) ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 ll)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-7))
      (when var-3 (setf nru var-3))
      (when var-8 (setf ldu var-8))))
(if (> ncc 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clasr "L" "V" "F"
         (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1) ncc
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ nm12 1)) ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ nm13 1)) ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice c-%data% f2cl-lib:complex8 (ll 1)
                               ((1 ldc) (1 *)) c-%offset%)
```

```
          ldc)
        (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
        (when var-4 (setf ncc var-4))
        (when var-8 (setf ldc var-8))))
  (if (<=
       (abs
        (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                       e-%offset%))
       thresh)
      (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                           e-%offset%)
            zero)))
 (t
  (setf f
          (*
           (- (abs (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%))
              shift)
           (+
            (f2cl-lib:sign one
                           (f2cl-lib:fref d-%data% (m) ((1 *))
                                          d-%offset%))
            (/ shift
               (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%)))))
  (setf g
          (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                         e-%offset%))
  (f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                ((> i (f2cl-lib:int-add ll 1)) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (slartg f g cosr sinr r)
        (declare (ignore))
        (when var-0 (setf f var-0))
        (when var-1 (setf g var-1))
        (when var-2 (setf cosr var-2))
        (when var-3 (setf sinr var-3))
        (when var-4 (setf r var-4)))
      (if (< i m)
          (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) r))
      (setf f
              (+
               (* cosr
                  (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
               (* sinr
                  (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                 ((1 *)) e-%offset%))))
      (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1)) ((1 *))
                           e-%offset%)
              (-
               (* cosr
```

```
            (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                           ((1 *)) e-%offset%))
         (* sinr
            (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))))
(setf g
        (* sinr
           (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub i 1))
                          ((1 *)) d-%offset%)))
(setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub i 1)) ((1 *))
                     d-%offset%)
        (* cosr
           (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub i 1))
                          ((1 *)) d-%offset%)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (slartg f g cosl sinl r)
  (declare (ignore))
  (when var-0 (setf f var-0))
  (when var-1 (setf g var-1))
  (when var-2 (setf cosl var-2))
  (when var-3 (setf sinl var-3))
  (when var-4 (setf r var-4)))
(setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) r)
(setf f
        (+
         (* cosl
            (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                           ((1 *)) e-%offset%))
         (* sinl
            (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub i 1))
                           ((1 *)) d-%offset%))))
(setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub i 1)) ((1 *))
                     d-%offset%)
        (-
         (* cosl
            (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub i 1))
                           ((1 *)) d-%offset%))
         (* sinl
            (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                           ((1 *)) e-%offset%))))
(cond
 ((> i (f2cl-lib:int-add ll 1))
  (setf g
          (* sinl
             (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 2))
                            ((1 *)) e-%offset%)))
  (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 2))
                        ((1 *)) e-%offset%)
          (* cosl
             (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 2))
                            ((1 *)) e-%offset%)))))
```

```
    (setf (f2cl-lib:fref rwork-%data% ((f2cl-lib:int-sub i ll))
                         ((1 *)) rwork-%offset%)
            cosr)
    (setf (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                            nm1))
                         ((1 *)) rwork-%offset%)
            (- sinr))
    (setf (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                            nm12))
                         ((1 *)) rwork-%offset%)
            cosl)
    (setf (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                            nm13))
                         ((1 *)) rwork-%offset%)
            (- sinl))
   label150))
(setf (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%) f)
(if (<= (abs (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%))
        thresh)
    (setf (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%) zero))
(if (> ncvt 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clasr "L" "V" "B"
         (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1) ncvt
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ nm12 1)) ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ nm13 1)) ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice vt-%data% f2cl-lib:complex8 (ll 1)
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt)
      (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
      (when var-4 (setf ncvt var-4))
      (when var-8 (setf ldvt var-8))))
(if (> nru 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (clasr "R" "V" "B" nru
         (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1)
         (f2cl-lib:array-slice rwork-%data% single-float (1)
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice rwork-%data% single-float (n)
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice u-%data% f2cl-lib:complex8 (1 ll)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
```

```
            (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-7))
            (when var-3 (setf nru var-3))
            (when var-8 (setf ldu var-8))))
        (if (> ncc 0)
            (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (clasr "L" "V" "B"
               (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1) ncc
               (f2cl-lib:array-slice rwork-%data% single-float (1)
                                     ((1 *)) rwork-%offset%)
               (f2cl-lib:array-slice rwork-%data% single-float (n)
                                     ((1 *)) rwork-%offset%)
               (f2cl-lib:array-slice c-%data% f2cl-lib:complex8 (ll 1)
                                     ((1 ldc) (1 *)) c-%offset%)
               ldc)
            (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
            (when var-4 (setf ncc var-4))
            (when var-8 (setf ldc var-8))))))))
 (go label60)
label160
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (cond
      ((< (f2cl-lib:fref d (i) ((1 *))) zero)
       (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
               (- (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
       (if (> ncvt 0)
           (multiple-value-bind (var-0 var-1 var-2 var-3)
               (csscal ncvt negone
                (f2cl-lib:array-slice vt-%data% f2cl-lib:complex8 (i 1)
                                      ((1 ldvt) (1 *)) vt-%offset%)
                ldvt)
              (declare (ignore var-2))
              (when var-0 (setf ncvt var-0))
              (when var-1 (setf negone var-1))
              (when var-3 (setf ldvt var-3))))))
    label170))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
   (tagbody
     (setf isub 1)
     (setf smin (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
     (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                   ((> j (f2cl-lib:int-add n 1 (f2cl-lib:int-sub i)))
                    nil)
       (tagbody
         (cond
          ((<= (f2cl-lib:fref d (j) ((1 *))) smin) (setf isub j)
           (setf smin (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%))))
```

```
   label180))
(cond
 ((/= isub (f2cl-lib:int-add n 1 (f2cl-lib:int-sub i)))
  (setf (f2cl-lib:fref d-%data% (isub) ((1 *)) d-%offset%)
          (f2cl-lib:fref d-%data%
                         ((f2cl-lib:int-sub (f2cl-lib:int-add n 1)
                                            i))
                         ((1 *)) d-%offset%))
  (setf (f2cl-lib:fref d-%data%
                       ((f2cl-lib:int-sub (f2cl-lib:int-add n 1)
                                          i))
                       ((1 *)) d-%offset%)
          smin)
  (if (> ncvt 0)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (cswap ncvt
           (f2cl-lib:array-slice vt-%data% f2cl-lib:complex8
                                 (isub 1) ((1 ldvt) (1 *))
                                 vt-%offset%)
           ldvt
           (f2cl-lib:array-slice vt-%data% f2cl-lib:complex8
                                 ((+ n 1 (f2cl-lib:int-sub i)) 1)
                                 ((1 ldvt) (1 *)) vt-%offset%)
           ldvt)
        (declare (ignore var-1 var-3))
        (when var-0 (setf ncvt var-0))
        (when var-2 (setf ldvt var-2))
        (when var-4 (setf ldvt var-4))))
  (if (> nru 0)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (cswap nru
           (f2cl-lib:array-slice u-%data% f2cl-lib:complex8
                                 (1 isub) ((1 ldu) (1 *))
                                 u-%offset%)
           1
           (f2cl-lib:array-slice u-%data% f2cl-lib:complex8
                                 (1
                                  (f2cl-lib:int-sub
                                    (f2cl-lib:int-add n 1) i))
                                 ((1 ldu) (1 *)) u-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf nru var-0))))
  (if (> ncc 0)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (cswap ncc
           (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                 (isub 1) ((1 ldc) (1 *))
                                 c-%offset%)
           ldc
```

```
                    (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                          ((+ n 1 (f2cl-lib:int-sub i)) 1)
                                          ((1 ldc) (1 *)) c-%offset%)
                    ldc)
                  (declare (ignore var-1 var-3))
                  (when var-0 (setf ncc var-0))
                  (when var-2 (setf ldc var-2))
                  (when var-4 (setf ldc var-4))))))
          label190))
        (go label220)
      label200
        (setf info 0)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
          (tagbody
            (if (/= (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) zero)
                (setf info (f2cl-lib:int-add info 1)))
          label210))
      label220
        (go end_label)
      end_label
        (return
         (values uplo n ncvt nru ncc nil nil nil ldvt nil ldu nil ldc nil
                 info))))))
```

———-

90.112 cgbbrd - reduce complex band matrix to real upper bidiagonal

90.112.1 Axiom unit tests

— **cgbbrd.input** —

```
)set break resume
)sys rm -f cgbbrd.output
)spool cgbbrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.112.2 Axiom help page

— cgbbrd.help —

```
====================================================================
cgbbrd examples
====================================================================

====================================================================
Man Page Details
====================================================================

SYNOPSIS
   Functions/Subroutines
       subroutine cgbbrd (VECT, M, N, NCC, KL, KU, AB, LDAB, D, E, Q, LDQ, PT,
           LDPT, C, LDC, WORK, RWORK, INFO)
           CGBBRD

Function/Subroutine Documentation
   subroutine cgbbrd (character VECT, integer M, integer N, integer NCC,
       integer KL, integer KU, complex, dimension( ldab, * ) AB, integer LDAB,
       real, dimension( * ) D, real, dimension( * ) E, complex, dimension(
       ldq, * ) Q, integer LDQ, complex, dimension( ldpt, * ) PT, integer
       LDPT, complex, dimension( ldc, * ) C, integer LDC, complex, dimension(
       * ) WORK, real, dimension( * ) RWORK, integer INFO)
       CGBBRD

Purpose:
   CGBBRD reduces a complex general m-by-n band matrix A to real upper
   bidiagonal form B by a unitary transformation: Q**H * A * P = B.

   The routine computes B, and optionally forms Q or P**H, or computes
   Q**H*C for a given matrix C.

Parameters:

 VECT
   VECT is CHARACTER*1
   Specifies whether or not the matrices Q and P**H are to be
   formed.
   = 'N': do not form Q or P**H;
   = 'Q': form Q only;
   = 'P': form P**H only;
   = 'B': form both.

 M
   M is INTEGER
   The number of rows of the matrix A.  M >= 0.
```

```
N
  N is INTEGER
  The number of columns of the matrix A.  N >= 0.

NCC
  NCC is INTEGER
  The number of columns of the matrix C.  NCC >= 0.

KL
  KL is INTEGER
  The number of subdiagonals of the matrix A. KL >= 0.

KU
  KU is INTEGER
  The number of superdiagonals of the matrix A. KU >= 0.

AB
  AB is COMPLEX array, dimension (LDAB,N)
  On entry, the m-by-n band matrix A, stored in rows 1 to
  KL+KU+1. The j-th column of A is stored in the j-th column of
  the array AB as follows:
  AB(ku+1+i-j,j) = A(i,j) for max(1,j-ku)<=i<=min(m,j+kl).
  On exit, A is overwritten by values generated during the
  reduction.

LDAB
  LDAB is INTEGER
  The leading dimension of the array A. LDAB >= KL+KU+1.

D
  D is REAL array, dimension (min(M,N))
  The diagonal elements of the bidiagonal matrix B.

E
  E is REAL array, dimension (min(M,N)-1)
  The superdiagonal elements of the bidiagonal matrix B.

Q
  Q is COMPLEX array, dimension (LDQ,M)
  If VECT = 'Q' or 'B', the m-by-m unitary matrix Q.
  If VECT = 'N' or 'P', the array Q is not referenced.

LDQ
  LDQ is INTEGER
  The leading dimension of the array Q.
  LDQ >= max(1,M) if VECT = 'Q' or 'B'; LDQ >= 1 otherwise.

PT
  PT is COMPLEX array, dimension (LDPT,N)
  If VECT = 'P' or 'B', the n-by-n unitary matrix P'.
```

```
  If VECT = 'N' or 'Q', the array PT is not referenced.

LDPT
  LDPT is INTEGER
  The leading dimension of the array PT.
  LDPT >= max(1,N) if VECT = 'P' or 'B'; LDPT >= 1 otherwise.

C
  C is COMPLEX array, dimension (LDC,NCC)
  On entry, an m-by-ncc matrix C.
  On exit, C is overwritten by Q**H*C.
  C is not referenced if NCC = 0.

LDC
  LDC is INTEGER
  The leading dimension of the array C.
  LDC >= max(1,M) if NCC > 0; LDC >= 1 if NCC = 0.

WORK
  WORK is COMPLEX array, dimension (max(M,N))

RWORK
  RWORK is REAL array, dimension (max(M,N))

INFO
  INFO is INTEGER
  = 0:  successful exit.
  < 0:  if INFO = -i, the i-th argument had an illegal value.
```

———-

90.112.3 fortran code

— cgbbrd.f —

```
*  ===================================================================
      SUBROUTINE CGBBRD( VECT, M, N, NCC, KL, KU, AB, LDAB, D, E, Q,
     $                   LDQ, PT, LDPT, C, LDC, WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          VECT
      INTEGER            INFO, KL, KU, LDAB, LDC, LDPT, LDQ, M, N, NCC
```

```
*     ..
*     .. Array Arguments ..
      REAL               D( * ), E( * ), RWORK( * )
      COMPLEX            AB( LDAB, * ), C( LDC, * ), PT( LDPT, * ),
     $                   Q( LDQ, * ), WORK( * )
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
      COMPLEX            CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0E+0, 0.0E+0 ),
     $                   CONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            WANTB, WANTC, WANTPT, WANTQ
      INTEGER            I, INCA, J, J1, J2, KB, KB1, KK, KLM, KLU1,
     $                   KUN, L, MINMN, ML, ML0, MU, MU0, NR, NRT
      REAL               ABST, RC
      COMPLEX            RA, RB, RS, T
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLARGV, CLARTG, CLARTV, CLASET, CROT, CSCAL,
     $                   XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, CONJG, MAX, MIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      WANTB = LSAME( VECT, 'B' )
      WANTQ = LSAME( VECT, 'Q' ) .OR. WANTB
      WANTPT = LSAME( VECT, 'P' ) .OR. WANTB
      WANTC = NCC.GT.0
      KLU1 = KL + KU + 1
      INFO = 0
      IF( .NOT.WANTQ .AND. .NOT.WANTPT .AND. .NOT.LSAME( VECT, 'N' ) )
     $     THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
```

```
         INFO = -3
      ELSE IF( NCC.LT.0 ) THEN
         INFO = -4
      ELSE IF( KL.LT.0 ) THEN
         INFO = -5
      ELSE IF( KU.LT.0 ) THEN
         INFO = -6
      ELSE IF( LDAB.LT.KLU1 ) THEN
         INFO = -8
      ELSE IF( LDQ.LT.1 .OR. WANTQ .AND. LDQ.LT.MAX( 1, M ) ) THEN
         INFO = -12
      ELSE IF( LDPT.LT.1 .OR. WANTPT .AND. LDPT.LT.MAX( 1, N ) ) THEN
         INFO = -14
      ELSE IF( LDC.LT.1 .OR. WANTC .AND. LDC.LT.MAX( 1, M ) ) THEN
         INFO = -16
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGBBRD', -INFO )
         RETURN
      END IF
*
*     Initialize Q and P**H to the unit matrix, if needed
*
      IF( WANTQ )
     $   CALL CLASET( 'Full', M, M, CZERO, CONE, Q, LDQ )
      IF( WANTPT )
     $   CALL CLASET( 'Full', N, N, CZERO, CONE, PT, LDPT )
*
*     Quick return if possible.
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
      MINMN = MIN( M, N )
*
      IF( KL+KU.GT.1 ) THEN
*
*        Reduce to upper bidiagonal form if KU > 0; if KU = 0, reduce
*        first to lower bidiagonal form and then transform to upper
*        bidiagonal
*
         IF( KU.GT.0 ) THEN
            ML0 = 1
            MU0 = 2
         ELSE
            ML0 = 2
            MU0 = 1
         END IF
*
*        Wherever possible, plane rotations are generated and applied in
```

```
*        vector operations of length NR over the index set J1:J2:KLU1.
*
*        The complex sines of the plane rotations are stored in WORK,
*        and the real cosines in RWORK.
*
         KLM = MIN( M-1, KL )
         KUN = MIN( N-1, KU )
         KB = KLM + KUN
         KB1 = KB + 1
         INCA = KB1*LDAB
         NR = 0
         J1 = KLM + 2
         J2 = 1 - KUN
*
         DO 90 I = 1, MINMN
*
*           Reduce i-th column and i-th row of matrix to bidiagonal form
*
            ML = KLM + 1
            MU = KUN + 1
            DO 80 KK = 1, KB
               J1 = J1 + KB
               J2 = J2 + KB
*
*              generate plane rotations to annihilate nonzero elements
*              which have been created below the band
*
               IF( NR.GT.0 )
     $            CALL CLARGV( NR, AB( KLU1, J1-KLM-1 ), INCA,
     $                         WORK( J1 ), KB1, RWORK( J1 ), KB1 )
*
*              apply plane rotations from the left
*
               DO 10 L = 1, KB
                  IF( J2-KLM+L-1.GT.N ) THEN
                     NRT = NR - 1
                  ELSE
                     NRT = NR
                  END IF
                  IF( NRT.GT.0 )
     $               CALL CLARTV( NRT, AB( KLU1-L, J1-KLM+L-1 ), INCA,
     $                            AB( KLU1-L+1, J1-KLM+L-1 ), INCA,
     $                            RWORK( J1 ), WORK( J1 ), KB1 )
   10          CONTINUE
*
               IF( ML.GT.ML0 ) THEN
                  IF( ML.LE.M-I+1 ) THEN
*
                     generate plane rotation to annihilate a(i+ml-1,i)
*                    within the band, and apply rotation from the left
```

```
*
                 CALL CLARTG( AB( KU+ML-1, I ), AB( KU+ML, I ),
     $                        RWORK( I+ML-1 ), WORK( I+ML-1 ), RA )
                 AB( KU+ML-1, I ) = RA
                 IF( I.LT.N )
     $              CALL CROT( MIN( KU+ML-2, N-I ),
     $                         AB( KU+ML-2, I+1 ), LDAB-1,
     $                         AB( KU+ML-1, I+1 ), LDAB-1,
     $                         RWORK( I+ML-1 ), WORK( I+ML-1 ) )
              END IF
              NR = NR + 1
              J1 = J1 - KB1
           END IF
*
           IF( WANTQ ) THEN
*
*             accumulate product of plane rotations in Q
*
              DO 20 J = J1, J2, KB1
                 CALL CROT( M, Q( 1, J-1 ), 1, Q( 1, J ), 1,
     $                      RWORK( J ), CONJG( WORK( J ) ) )
   20         CONTINUE
           END IF
*
           IF( WANTC ) THEN
*
*             apply plane rotations to C
*
              DO 30 J = J1, J2, KB1
                 CALL CROT( NCC, C( J-1, 1 ), LDC, C( J, 1 ), LDC,
     $                      RWORK( J ), WORK( J ) )
   30         CONTINUE
           END IF
*
           IF( J2+KUN.GT.N ) THEN
*
*             adjust J2 to keep within the bounds of the matrix
*
              NR = NR - 1
              J2 = J2 - KB1
           END IF
*
           DO 40 J = J1, J2, KB1
*
*             create nonzero element a(j-1,j+ku) above the band
*             and store it in WORK(n+1:2*n)
*
              WORK( J+KUN ) = WORK( J )*AB( 1, J+KUN )
              AB( 1, J+KUN ) = RWORK( J )*AB( 1, J+KUN )
   40      CONTINUE
```

```
*
*              generate plane rotations to annihilate nonzero elements
*              which have been generated above the band
*
               IF( NR.GT.0 )
     $            CALL CLARGV( NR, AB( 1, J1+KUN-1 ), INCA,
     $                         WORK( J1+KUN ), KB1, RWORK( J1+KUN ),
     $                         KB1 )
*
*              apply plane rotations from the right
*
               DO 50 L = 1, KB
                  IF( J2+L-1.GT.M ) THEN
                     NRT = NR - 1
                  ELSE
                     NRT = NR
                  END IF
                  IF( NRT.GT.0 )
     $               CALL CLARTV( NRT, AB( L+1, J1+KUN-1 ), INCA,
     $                            AB( L, J1+KUN ), INCA,
     $                            RWORK( J1+KUN ), WORK( J1+KUN ), KB1 )
   50          CONTINUE
*
               IF( ML.EQ.ML0 .AND. MU.GT.MU0 ) THEN
                  IF( MU.LE.N-I+1 ) THEN
*
*                    generate plane rotation to annihilate a(i,i+mu-1)
*                    within the band, and apply rotation from the right
*
                     CALL CLARTG( AB( KU-MU+3, I+MU-2 ),
     $                            AB( KU-MU+2, I+MU-1 ),
     $                            RWORK( I+MU-1 ), WORK( I+MU-1 ), RA )
                     AB( KU-MU+3, I+MU-2 ) = RA
                     CALL CROT( MIN( KL+MU-2, M-I ),
     $                          AB( KU-MU+4, I+MU-2 ), 1,
     $                          AB( KU-MU+3, I+MU-1 ), 1,
     $                          RWORK( I+MU-1 ), WORK( I+MU-1 ) )
                  END IF
                  NR = NR + 1
                  J1 = J1 - KB1
               END IF
*
               IF( WANTPT ) THEN
*
*                accumulate product of plane rotations in P**H
*
                  DO 60 J = J1, J2, KB1
                     CALL CROT( N, PT( J+KUN-1, 1 ), LDPT,
     $                          PT( J+KUN, 1 ), LDPT, RWORK( J+KUN ),
     $                          CONJG( WORK( J+KUN ) ) )
```

```
   60            CONTINUE
              END IF
*
              IF( J2+KB.GT.M ) THEN
*
*                adjust J2 to keep within the bounds of the matrix
*
                NR = NR - 1
                J2 = J2 - KB1
              END IF
*
              DO 70 J = J1, J2, KB1
*
*                create nonzero element a(j+kl+ku,j+ku-1) below the
*                band and store it in WORK(1:n)
*
                WORK( J+KB ) = WORK( J+KUN )*AB( KLU1, J+KUN )
                AB( KLU1, J+KUN ) = RWORK( J+KUN )*AB( KLU1, J+KUN )
   70         CONTINUE
*
              IF( ML.GT.ML0 ) THEN
                ML = ML - 1
              ELSE
                MU = MU - 1
              END IF
   80       CONTINUE
   90    CONTINUE
      END IF
*
      IF( KU.EQ.0 .AND. KL.GT.0 ) THEN
*
*        A has been reduced to complex lower bidiagonal form
*
*        Transform lower bidiagonal form to upper bidiagonal by applying
*        plane rotations from the left, overwriting superdiagonal
*        elements on subdiagonal elements
*
         DO 100 I = 1, MIN( M-1, N )
            CALL CLARTG( AB( 1, I ), AB( 2, I ), RC, RS, RA )
            AB( 1, I ) = RA
            IF( I.LT.N ) THEN
               AB( 2, I ) = RS*AB( 1, I+1 )
               AB( 1, I+1 ) = RC*AB( 1, I+1 )
            END IF
            IF( WANTQ )
     $         CALL CROT( M, Q( 1, I ), 1, Q( 1, I+1 ), 1, RC,
     $                    CONJG( RS ) )
            IF( WANTC )
     $         CALL CROT( NCC, C( I, 1 ), LDC, C( I+1, 1 ), LDC, RC,
     $                    RS )
```

```
  100    CONTINUE
      ELSE
*
*        A has been reduced to complex upper bidiagonal form or is
*        diagonal
*
         IF( KU.GT.0 .AND. M.LT.N ) THEN
*
*           Annihilate a(m,m+1) by applying plane rotations from the
*           right
*
            RB = AB( KU, M+1 )
            DO 110 I = M, 1, -1
               CALL CLARTG( AB( KU+1, I ), RB, RC, RS, RA )
               AB( KU+1, I ) = RA
               IF( I.GT.1 ) THEN
                  RB = -CONJG( RS )*AB( KU, I )
                  AB( KU, I ) = RC*AB( KU, I )
               END IF
               IF( WANTPT )
     $            CALL CROT( N, PT( I, 1 ), LDPT, PT( M+1, 1 ), LDPT,
     $                       RC, CONJG( RS ) )
  110       CONTINUE
         END IF
      END IF
*
*     Make diagonal and superdiagonal elements real, storing them in D
*     and E
*
      T = AB( KU+1, 1 )
      DO 120 I = 1, MINMN
         ABST = ABS( T )
         D( I ) = ABST
         IF( ABST.NE.ZERO ) THEN
            T = T / ABST
         ELSE
            T = CONE
         END IF
         IF( WANTQ )
     $      CALL CSCAL( M, T, Q( 1, I ), 1 )
         IF( WANTC )
     $      CALL CSCAL( NCC, CONJG( T ), C( I, 1 ), LDC )
         IF( I.LT.MINMN ) THEN
            IF( KU.EQ.0 .AND. KL.EQ.0 ) THEN
               E( I ) = ZERO
               T = AB( 1, I+1 )
            ELSE
               IF( KU.EQ.0 ) THEN
                  T = AB( 2, I )*CONJG( T )
               ELSE
```

```
                T = AB( KU, I+1 )*CONJG( T )
             END IF
             ABST = ABS( T )
             E( I ) = ABST
             IF( ABST.NE.ZERO ) THEN
                T = T / ABST
             ELSE
                T = CONE
             END IF
             IF( WANTPT )
     $          CALL CSCAL( N, T, PT( I+1, 1 ), LDPT )
             T = AB( KU+1, I+1 )*CONJG( T )
          END IF
       END IF
  120 CONTINUE
      RETURN
*
*     End of CGBBRD
*
      END
*  =====================================================================
      SUBROUTINE CGBBRD( VECT, M, N, NCC, KL, KU, AB, LDAB, D, E, Q,
     $                   LDQ, PT, LDPT, C, LDC, WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          VECT
      INTEGER            INFO, KL, KU, LDAB, LDC, LDPT, LDQ, M, N, NCC
*     ..
*     .. Array Arguments ..
      REAL               D( * ), E( * ), RWORK( * )
      COMPLEX            AB( LDAB, * ), C( LDC, * ), PT( LDPT, * ),
     $                   Q( LDQ, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      REAL               ZERO
      PARAMETER          ( ZERO = 0.0E+0 )
      COMPLEX            CZERO, CONE
      PARAMETER          ( CZERO = ( 0.0E+0, 0.0E+0 ),
     $                   CONE = ( 1.0E+0, 0.0E+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            WANTB, WANTC, WANTPT, WANTQ
```

```
      INTEGER            I, INCA, J, J1, J2, KB, KB1, KK, KLM, KLU1,
     $                   KUN, L, MINMN, ML, ML0, MU, MU0, NR, NRT
      REAL               ABST, RC
      COMPLEX            RA, RB, RS, T
*     ..
*     .. External Subroutines ..
      EXTERNAL           CLARGV, CLARTG, CLARTV, CLASET, CROT, CSCAL,
     $                   XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, CONJG, MAX, MIN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      WANTB = LSAME( VECT, 'B' )
      WANTQ = LSAME( VECT, 'Q' ) .OR. WANTB
      WANTPT = LSAME( VECT, 'P' ) .OR. WANTB
      WANTC = NCC.GT.0
      KLU1 = KL + KU + 1
      INFO = 0
      IF( .NOT.WANTQ .AND. .NOT.WANTPT .AND. .NOT.LSAME( VECT, 'N' ) )
     $     THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( NCC.LT.0 ) THEN
         INFO = -4
      ELSE IF( KL.LT.0 ) THEN
         INFO = -5
      ELSE IF( KU.LT.0 ) THEN
         INFO = -6
      ELSE IF( LDAB.LT.KLU1 ) THEN
         INFO = -8
      ELSE IF( LDQ.LT.1 .OR. WANTQ .AND. LDQ.LT.MAX( 1, M ) ) THEN
         INFO = -12
      ELSE IF( LDPT.LT.1 .OR. WANTPT .AND. LDPT.LT.MAX( 1, N ) ) THEN
         INFO = -14
      ELSE IF( LDC.LT.1 .OR. WANTC .AND. LDC.LT.MAX( 1, M ) ) THEN
         INFO = -16
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'CGBBRD', -INFO )
```

```
         RETURN
      END IF
*
*     Initialize Q and P**H to the unit matrix, if needed
*
      IF( WANTQ )
     $   CALL CLASET( 'Full', M, M, CZERO, CONE, Q, LDQ )
      IF( WANTPT )
     $   CALL CLASET( 'Full', N, N, CZERO, CONE, PT, LDPT )
*
*     Quick return if possible.
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
      MINMN = MIN( M, N )
*
      IF( KL+KU.GT.1 ) THEN
*
*        Reduce to upper bidiagonal form if KU > 0; if KU = 0, reduce
*        first to lower bidiagonal form and then transform to upper
*        bidiagonal
*
         IF( KU.GT.0 ) THEN
            ML0 = 1
            MU0 = 2
         ELSE
            ML0 = 2
            MU0 = 1
         END IF
*
*        Wherever possible, plane rotations are generated and applied in
*        vector operations of length NR over the index set J1:J2:KLU1.
*
*        The complex sines of the plane rotations are stored in WORK,
*        and the real cosines in RWORK.
*
         KLM = MIN( M-1, KL )
         KUN = MIN( N-1, KU )
         KB = KLM + KUN
         KB1 = KB + 1
         INCA = KB1*LDAB
         NR = 0
         J1 = KLM + 2
         J2 = 1 - KUN
*
         DO 90 I = 1, MINMN
*
*           Reduce i-th column and i-th row of matrix to bidiagonal form
*
```

```
            ML = KLM + 1
            MU = KUN + 1
            DO 80 KK = 1, KB
               J1 = J1 + KB
               J2 = J2 + KB
*
*              generate plane rotations to annihilate nonzero elements
*              which have been created below the band
*
               IF( NR.GT.0 )
     $            CALL CLARGV( NR, AB( KLU1, J1-KLM-1 ), INCA,
     $                         WORK( J1 ), KB1, RWORK( J1 ), KB1 )
*
*              apply plane rotations from the left
*
               DO 10 L = 1, KB
                  IF( J2-KLM+L-1.GT.N ) THEN
                     NRT = NR - 1
                  ELSE
                     NRT = NR
                  END IF
                  IF( NRT.GT.0 )
     $               CALL CLARTV( NRT, AB( KLU1-L, J1-KLM+L-1 ), INCA,
     $                            AB( KLU1-L+1, J1-KLM+L-1 ), INCA,
     $                            RWORK( J1 ), WORK( J1 ), KB1 )
   10          CONTINUE
*
               IF( ML.GT.ML0 ) THEN
                  IF( ML.LE.M-I+1 ) THEN
*
*                    generate plane rotation to annihilate a(i+ml-1,i)
*                    within the band, and apply rotation from the left
*
                     CALL CLARTG( AB( KU+ML-1, I ), AB( KU+ML, I ),
     $                            RWORK( I+ML-1 ), WORK( I+ML-1 ), RA )
                     AB( KU+ML-1, I ) = RA
                     IF( I.LT.N )
     $                  CALL CROT( MIN( KU+ML-2, N-I ),
     $                             AB( KU+ML-2, I+1 ), LDAB-1,
     $                             AB( KU+ML-1, I+1 ), LDAB-1,
     $                             RWORK( I+ML-1 ), WORK( I+ML-1 ) )
                  END IF
                  NR = NR + 1
                  J1 = J1 - KB1
               END IF
*
               IF( WANTQ ) THEN
*
*                accumulate product of plane rotations in Q
*
```

```
                 DO 20 J = J1, J2, KB1
                    CALL CROT( M, Q( 1, J-1 ), 1, Q( 1, J ), 1,
     $                         RWORK( J ), CONJG( WORK( J ) ) )
   20            CONTINUE
              END IF
*
              IF( WANTC ) THEN
*
*                apply plane rotations to C
*
                 DO 30 J = J1, J2, KB1
                    CALL CROT( NCC, C( J-1, 1 ), LDC, C( J, 1 ), LDC,
     $                         RWORK( J ), WORK( J ) )
   30            CONTINUE
              END IF
*
              IF( J2+KUN.GT.N ) THEN
*
*                adjust J2 to keep within the bounds of the matrix
*
                 NR = NR - 1
                 J2 = J2 - KB1
              END IF
*
              DO 40 J = J1, J2, KB1
*
*                create nonzero element a(j-1,j+ku) above the band
*                and store it in WORK(n+1:2*n)
*
                 WORK( J+KUN ) = WORK( J )*AB( 1, J+KUN )
                 AB( 1, J+KUN ) = RWORK( J )*AB( 1, J+KUN )
   40         CONTINUE
*
*             generate plane rotations to annihilate nonzero elements
*             which have been generated above the band
*
              IF( NR.GT.0 )
     $           CALL CLARGV( NR, AB( 1, J1+KUN-1 ), INCA,
     $                        WORK( J1+KUN ), KB1, RWORK( J1+KUN ),
     $                        KB1 )
*
*             apply plane rotations from the right
*
              DO 50 L = 1, KB
                 IF( J2+L-1.GT.M ) THEN
                    NRT = NR - 1
                 ELSE
                    NRT = NR
                 END IF
                 IF( NRT.GT.0 )
```

```
     $              CALL CLARTV( NRT, AB( L+1, J1+KUN-1 ), INCA,
     $                           AB( L, J1+KUN ), INCA,
     $                           RWORK( J1+KUN ), WORK( J1+KUN ), KB1 )
   50          CONTINUE
*
               IF( ML.EQ.ML0 .AND. MU.GT.MU0 ) THEN
                  IF( MU.LE.N-I+1 ) THEN
*
*                    generate plane rotation to annihilate a(i,i+mu-1)
*                    within the band, and apply rotation from the right
*
                     CALL CLARTG( AB( KU-MU+3, I+MU-2 ),
     $                            AB( KU-MU+2, I+MU-1 ),
     $                            RWORK( I+MU-1 ), WORK( I+MU-1 ), RA )
                     AB( KU-MU+3, I+MU-2 ) = RA
                     CALL CROT( MIN( KL+MU-2, M-I ),
     $                          AB( KU-MU+4, I+MU-2 ), 1,
     $                          AB( KU-MU+3, I+MU-1 ), 1,
     $                          RWORK( I+MU-1 ), WORK( I+MU-1 ) )
                  END IF
                  NR = NR + 1
                  J1 = J1 - KB1
               END IF
*
               IF( WANTPT ) THEN
*
*                 accumulate product of plane rotations in P**H
*
                  DO 60 J = J1, J2, KB1
                     CALL CROT( N, PT( J+KUN-1, 1 ), LDPT,
     $                          PT( J+KUN, 1 ), LDPT, RWORK( J+KUN ),
     $                          CONJG( WORK( J+KUN ) ) )
   60             CONTINUE
               END IF
*
               IF( J2+KB.GT.M ) THEN
*
*                 adjust J2 to keep within the bounds of the matrix
*
                  NR = NR - 1
                  J2 = J2 - KB1
               END IF
*
               DO 70 J = J1, J2, KB1
*
*                 create nonzero element a(j+kl+ku,j+ku-1) below the
*                 band and store it in WORK(1:n)
*
                  WORK( J+KB ) = WORK( J+KUN )*AB( KLU1, J+KUN )
                  AB( KLU1, J+KUN ) = RWORK( J+KUN )*AB( KLU1, J+KUN )
```

```
   70          CONTINUE
*
               IF( ML.GT.ML0 ) THEN
                  ML = ML - 1
               ELSE
                  MU = MU - 1
               END IF
   80       CONTINUE
   90    CONTINUE
      END IF
*
      IF( KU.EQ.0 .AND. KL.GT.0 ) THEN
*
*        A has been reduced to complex lower bidiagonal form
*
*        Transform lower bidiagonal form to upper bidiagonal by applying
*        plane rotations from the left, overwriting superdiagonal
*        elements on subdiagonal elements
*
         DO 100 I = 1, MIN( M-1, N )
            CALL CLARTG( AB( 1, I ), AB( 2, I ), RC, RS, RA )
            AB( 1, I ) = RA
            IF( I.LT.N ) THEN
               AB( 2, I ) = RS*AB( 1, I+1 )
               AB( 1, I+1 ) = RC*AB( 1, I+1 )
            END IF
            IF( WANTQ )
     $         CALL CROT( M, Q( 1, I ), 1, Q( 1, I+1 ), 1, RC,
     $                    CONJG( RS ) )
            IF( WANTC )
     $         CALL CROT( NCC, C( I, 1 ), LDC, C( I+1, 1 ), LDC, RC,
     $                    RS )
  100    CONTINUE
      ELSE
*
*        A has been reduced to complex upper bidiagonal form or is
*        diagonal
*
         IF( KU.GT.0 .AND. M.LT.N ) THEN
*
*           Annihilate a(m,m+1) by applying plane rotations from the
*           right
*
            RB = AB( KU, M+1 )
            DO 110 I = M, 1, -1
               CALL CLARTG( AB( KU+1, I ), RB, RC, RS, RA )
               AB( KU+1, I ) = RA
               IF( I.GT.1 ) THEN
                  RB = -CONJG( RS )*AB( KU, I )
                  AB( KU, I ) = RC*AB( KU, I )
```

```
               END IF
               IF( WANTPT )
     $            CALL CROT( N, PT( I, 1 ), LDPT, PT( M+1, 1 ), LDPT,
     $                       RC, CONJG( RS ) )
  110       CONTINUE
         END IF
      END IF
*
*     Make diagonal and superdiagonal elements real, storing them in D
*     and E
*
      T = AB( KU+1, 1 )
      DO 120 I = 1, MINMN
         ABST = ABS( T )
         D( I ) = ABST
         IF( ABST.NE.ZERO ) THEN
            T = T / ABST
         ELSE
            T = CONE
         END IF
         IF( WANTQ )
     $      CALL CSCAL( M, T, Q( 1, I ), 1 )
         IF( WANTC )
     $      CALL CSCAL( NCC, CONJG( T ), C( I, 1 ), LDC )
         IF( I.LT.MINMN ) THEN
            IF( KU.EQ.0 .AND. KL.EQ.0 ) THEN
               E( I ) = ZERO
               T = AB( 1, I+1 )
            ELSE
               IF( KU.EQ.0 ) THEN
                  T = AB( 2, I )*CONJG( T )
               ELSE
                  T = AB( KU, I+1 )*CONJG( T )
               END IF
               ABST = ABS( T )
               E( I ) = ABST
               IF( ABST.NE.ZERO ) THEN
                  T = T / ABST
               ELSE
                  T = CONE
               END IF
               IF( WANTPT )
     $            CALL CSCAL( N, T, PT( I+1, 1 ), LDPT )
               T = AB( KU+1, I+1 )*CONJG( T )
            END IF
         END IF
  120 CONTINUE
      RETURN
*
*     End of CGBBRD
```

```
*
      END
```

———-

90.112.4 lisp code

— **cgbbrd.lisp** —

```
(let* ((zero 0.0)
       (czero (f2cl-lib:cmplx 0.0 0.0))
       (cone (f2cl-lib:cmplx 1.0 0.0)))
  (declare (type (single-float 0.0 0.0) zero)
           (type (f2cl-lib:complex8) czero)
           (type (f2cl-lib:complex8) cone)
           (ignorable zero czero cone))
  (defun cgbbrd
         (vect m n ncc kl ku ab ldab d e q ldq pt ldpt c ldc work rwork info)
    (declare (type (array single-float (*)) rwork e d)
             (type (array f2cl-lib:complex8 (*)) work c pt q ab)
             (type (f2cl-lib:integer4) info ldc ldpt ldq ldab ku kl ncc n m)
             (type (string *) vect))
    (f2cl-lib:with-multi-array-data
        ((vect character vect-%data% vect-%offset%)
         (ab f2cl-lib:complex8 ab-%data% ab-%offset%)
         (q f2cl-lib:complex8 q-%data% q-%offset%)
         (pt f2cl-lib:complex8 pt-%data% pt-%offset%)
         (c f2cl-lib:complex8 c-%data% c-%offset%)
         (work f2cl-lib:complex8 work-%data% work-%offset%)
         (d single-float d-%data% d-%offset%)
         (e single-float e-%data% e-%offset%)
         (rwork single-float rwork-%data% rwork-%offset%))
      (prog ((ra #C(0.0 0.0)) (rb #C(0.0 0.0)) (rs #C(0.0 0.0)) (abst 0.0)
             (rc 0.0) (i 0) (inca 0) (j 0) (j1 0) (j2 0) (kb 0) (kb1 0) (kk 0)
             (klm 0) (klu1 0) (kun 0) (l 0) (minmn 0) (ml 0) (ml0 0) (mu 0)
             (mu0 0) (nr 0) (nrt 0) (wantb nil) (wantc nil) (wantpt nil)
             (wantq nil) (t$ #C(0.0 0.0)))
        (declare (type (f2cl-lib:complex8) t$ ra rb rs)
                 (type (single-float) abst rc)
                 (type (f2cl-lib:integer4) i inca j j1 j2 kb kb1 kk klm klu1
                  kun l minmn ml ml0 mu mu0 nr nrt)
                 (type f2cl-lib:logical wantb wantc wantpt wantq))
        (setf wantb
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame vect "B")
                  (declare (ignore var-1))
                  (when var-0 (setf vect var-0))
```

```
          ret-val))
(setf wantq
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame vect "Q")
           (declare (ignore var-1))
           (when var-0 (setf vect var-0))
           ret-val)
         wantb))
(setf wantpt
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame vect "P")
           (declare (ignore var-1))
           (when var-0 (setf vect var-0))
           ret-val)
         wantb))
(setf wantc (> ncc 0))
(setf klu1 (f2cl-lib:int-add kl ku 1))
(setf info 0)
(cond
 ((and (not wantq) (not wantpt)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame vect "N")
          (declare (ignore var-1))
          (when var-0 (setf vect var-0))
          ret-val)))
  (setf info -1))
 ((< m 0) (setf info -2)) ((< n 0) (setf info -3))
 ((< ncc 0) (setf info -4)) ((< kl 0) (setf info -5))
 ((< ku 0) (setf info -6)) ((< ldab klu1) (setf info -8))
 ((or (< ldq 1)
      (and wantq
           (< ldq
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 m)))))
  (setf info -12))
 ((or (< ldpt 1)
      (and wantpt
           (< ldpt
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 n)))))
  (setf info -14))
 ((or (< ldc 1)
      (and wantc
           (< ldc
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 m)))))
  (setf info -16)))
```

```
(cond
 ((/= info 0) (xerbla "CGBBRD" (f2cl-lib:int-sub info))
  (go end_label)))
(if wantq
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "Full" m m czero cone q ldq)
      (declare (ignore var-0 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf m var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldq var-6))))
(if wantpt
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (claset "Full" n n czero cone pt ldpt)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf czero var-3))
      (when var-4 (setf cone var-4))
      (when var-6 (setf ldpt var-6))))
(if (or (= m 0) (= n 0))
    (go end_label))
(setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(cond
 ((> (f2cl-lib:int-add kl ku) 1)
  (cond ((> ku 0) (setf ml0 1) (setf mu0 2))
        (t (setf ml0 2) (setf mu0 1)))
  (setf klm
          (min (the f2cl-lib:integer4 (f2cl-lib:int-sub m 1))
               (the f2cl-lib:integer4 kl)))
  (setf kun
          (min (the f2cl-lib:integer4 (f2cl-lib:int-sub n 1))
               (the f2cl-lib:integer4 ku)))
  (setf kb (f2cl-lib:int-add klm kun))
  (setf kb1 (f2cl-lib:int-add kb 1))
  (setf inca (f2cl-lib:int-mul kb1 ldab)) (setf nr 0)
  (setf j1 (f2cl-lib:int-add klm 2)) (setf j2 (f2cl-lib:int-sub 1 kun))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i minmn) nil)
    (tagbody
      (setf ml (f2cl-lib:int-add klm 1))
      (setf mu (f2cl-lib:int-add kun 1))
      (f2cl-lib:fdo (kk 1 (f2cl-lib:int-add kk 1))
                    ((> kk kb) nil)
        (tagbody
          (setf j1 (f2cl-lib:int-add j1 kb))
          (setf j2 (f2cl-lib:int-add j2 kb))
          (if (> nr 0)
              (multiple-value-bind
```

```
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (clargv nr
             (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                   (klu1
                                    (f2cl-lib:int-sub j1 klm 1))
                                   ((1 ldab) (1 *)) ab-%offset%)
             inca
             (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                   (j1) ((1 *)) work-%offset%)
             kb1
             (f2cl-lib:array-slice rwork-%data% single-float (j1)
                                   ((1 *)) rwork-%offset%)
             kb1)
          (declare (ignore var-1 var-3 var-5))
          (when var-0 (setf nr var-0))
          (when var-2 (setf inca var-2))
          (when var-4 (setf kb1 var-4))
          (when var-6 (setf kb1 var-6))))
    (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                  ((> l kb) nil)
      (tagbody
        (cond
         ((>
           (f2cl-lib:int-add j2 (f2cl-lib:int-sub klm) l
                             (f2cl-lib:int-sub 1))
           n)
          (setf nrt (f2cl-lib:int-sub nr 1)))
         (t (setf nrt nr)))
        (if (> nrt 0)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
                (clartv nrt
                 (f2cl-lib:array-slice ab-%data%
                                       f2cl-lib:complex8
                                       ((+ klu1
                                           (f2cl-lib:int-sub l))
                                        (f2cl-lib:int-sub
                                         (f2cl-lib:int-add
                                          (f2cl-lib:int-sub j1
                                                            klm)
                                          l)
                                         1))
                                       ((1 ldab) (1 *))
                                       ab-%offset%)
                 inca
                 (f2cl-lib:array-slice ab-%data%
                                       f2cl-lib:complex8
                                       ((+ klu1
                                           (f2cl-lib:int-sub l)
                                           1)
```

```
                                        (f2cl-lib:int-sub
                                         (f2cl-lib:int-add
                                          (f2cl-lib:int-sub j1
                                                            klm)
                                          l)
                                         1))
                                       ((1 ldab) (1 *))
                                       ab-%offset%)
                   inca
                   (f2cl-lib:array-slice rwork-%data% single-float
                                         (j1) ((1 *))
                                         rwork-%offset%)
                   (f2cl-lib:array-slice work-%data%
                                         f2cl-lib:complex8 (j1)
                                         ((1 *)) work-%offset%)
                   kb1)
                 (declare (ignore var-1 var-3 var-5 var-6))
                 (when var-0 (setf nrt var-0))
                 (when var-2 (setf inca var-2))
                 (when var-4 (setf inca var-4))
                 (when var-7 (setf kb1 var-7))))
   label10))
(cond
 ((> ml ml0)
  (cond
   ((<= ml (f2cl-lib:int-add m (f2cl-lib:int-sub i) 1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (clartg
         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                               ((+ ku ml
                                   (f2cl-lib:int-sub 1))
                                i)
                               ((1 ldab) (1 *)) ab-%offset%)
         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                               ((+ ku ml) i) ((1 ldab) (1 *))
                               ab-%offset%)
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ i ml
                                   (f2cl-lib:int-sub 1)))
                               ((1 *)) rwork-%offset%)
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                               ((+ i ml
                                   (f2cl-lib:int-sub 1)))
                               ((1 *)) work-%offset%)
         ra)
      (declare (ignore var-0 var-1 var-2 var-3))
      (when var-4 (setf ra var-4)))
    (setf (f2cl-lib:fref ab-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add ku ml) 1)
```

```
                                 i)
                                ((1 ldab) (1 *)) ab-%offset%)
                     ra)
            (if (< i n)
                (crot
                 (min
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-sub (f2cl-lib:int-add ku ml) 2))
                  (the f2cl-lib:integer4 (f2cl-lib:int-sub n i)))
                 (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                       ((+ ku ml
                                           (f2cl-lib:int-sub 2))
                                        (f2cl-lib:int-add i 1))
                                       ((1 ldab) (1 *)) ab-%offset%)
                 (f2cl-lib:int-sub ldab 1)
                 (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                       ((+ ku ml
                                           (f2cl-lib:int-sub 1))
                                        (f2cl-lib:int-add i 1))
                                       ((1 ldab) (1 *)) ab-%offset%)
                 (f2cl-lib:int-sub ldab 1)
                 (f2cl-lib:array-slice rwork-%data% single-float
                                       ((+ i ml
                                           (f2cl-lib:int-sub 1)))
                                       ((1 *)) rwork-%offset%)
                 (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                       ((+ i ml
                                           (f2cl-lib:int-sub 1)))
                                       ((1 *)) work-%offset%)))))
          (setf nr (f2cl-lib:int-add nr 1))
          (setf j1 (f2cl-lib:int-sub j1 kb1))))
        (cond
         (wantq
          (f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
                        ((> j j2) nil)
            (tagbody
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (crot m
                   (f2cl-lib:array-slice q-%data% f2cl-lib:complex8
                                         (1 (f2cl-lib:int-sub j 1))
                                         ((1 ldq) (1 *)) q-%offset%)
                   1
                   (f2cl-lib:array-slice q-%data% f2cl-lib:complex8
                                         (1 j) ((1 ldq) (1 *))
                                         q-%offset%)
                   1
                   (f2cl-lib:array-slice rwork-%data% single-float
                                         (j) ((1 *)) rwork-%offset%)
                   (f2cl-lib:conjg
```

```
            (f2cl-lib:fref work-%data% (j) ((1 *))
                           work-%offset%)))
        (declare
         (ignore var-1 var-2 var-3 var-4 var-5 var-6))
        (when var-0 (setf m var-0)))
     label20))))
(cond
 (wantc
  (f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
                ((> j j2) nil)
    (tagbody
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (crot ncc
           (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                 ((+ j (f2cl-lib:int-sub 1))
                                  1)
                                 ((1 ldc) (1 *)) c-%offset%)
           ldc
           (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                 (j 1) ((1 ldc) (1 *))
                                 c-%offset%)
           ldc
           (f2cl-lib:array-slice rwork-%data% single-float
                                 (j) ((1 *)) rwork-%offset%)
           (f2cl-lib:array-slice work-%data%
                                 f2cl-lib:complex8 (j)
                                 ((1 *)) work-%offset%))
        (declare (ignore var-1 var-3 var-5 var-6))
        (when var-0 (setf ncc var-0))
        (when var-2 (setf ldc var-2))
        (when var-4 (setf ldc var-4)))
     label30))))
(cond
 ((> (f2cl-lib:int-add j2 kun) n)
  (setf nr (f2cl-lib:int-sub nr 1))
  (setf j2 (f2cl-lib:int-sub j2 kb1))))
(f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
              ((> j j2) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add j kun)) ((1 *))
                         work-%offset%)
            (*
             (f2cl-lib:fref work-%data% (j) ((1 *))
                            work-%offset%)
             (f2cl-lib:fref ab-%data%
                            (1 (f2cl-lib:int-add j kun))
                            ((1 ldab) (1 *)) ab-%offset%)))
    (setf (f2cl-lib:fref ab-%data%
```

```
                            (1 (f2cl-lib:int-add j kun))
                            ((1 ldab) (1 *)) ab-%offset%)
                (*
                 (f2cl-lib:fref rwork-%data% (j) ((1 *))
                                rwork-%offset%)
                 (f2cl-lib:fref ab-%data%
                                (1 (f2cl-lib:int-add j kun))
                                ((1 ldab) (1 *)) ab-%offset%)))
   label40))
(if (> nr 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (clargv nr
         (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                               (1
                                (f2cl-lib:int-sub
                                 (f2cl-lib:int-add j1 kun)
                                 1))
                               ((1 ldab) (1 *)) ab-%offset%)
         inca
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                               ((+ j1 kun)) ((1 *))
                               work-%offset%)
         kb1
         (f2cl-lib:array-slice rwork-%data% single-float
                               ((+ j1 kun)) ((1 *))
                               rwork-%offset%)
         kb1)
      (declare (ignore var-1 var-3 var-5))
      (when var-0 (setf nr var-0))
      (when var-2 (setf inca var-2))
      (when var-4 (setf kb1 var-4))
      (when var-6 (setf kb1 var-6))))
(f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
              ((> l kb) nil)
  (tagbody
    (cond
     ((> (f2cl-lib:int-add j2 l (f2cl-lib:int-sub 1)) m)
      (setf nrt (f2cl-lib:int-sub nr 1)))
     (t (setf nrt nr)))
    (if (> nrt 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (clartv nrt
             (f2cl-lib:array-slice ab-%data%
                                   f2cl-lib:complex8
                                   ((+ l 1)
                                    (f2cl-lib:int-sub
                                     (f2cl-lib:int-add j1
                                                       kun)
```

```
                                        1))
                                      ((1 ldab) (1 *))
                                      ab-%offset%)
                        inca
                        (f2cl-lib:array-slice ab-%data%
                                      f2cl-lib:complex8
                                      (l
                                       (f2cl-lib:int-add j1
                                                         kun))
                                      ((1 ldab) (1 *))
                                      ab-%offset%)
                        inca
                        (f2cl-lib:array-slice rwork-%data% single-float
                                      ((+ j1 kun)) ((1 *))
                                      rwork-%offset%)
                        (f2cl-lib:array-slice work-%data%
                                      f2cl-lib:complex8
                                      ((+ j1 kun)) ((1 *))
                                      work-%offset%)
                        kb1)
                     (declare (ignore var-1 var-3 var-5 var-6))
                     (when var-0 (setf nrt var-0))
                     (when var-2 (setf inca var-2))
                     (when var-4 (setf inca var-4))
                     (when var-7 (setf kb1 var-7))))
           label50))
        (cond
         ((and (= ml ml0) (> mu mu0))
          (cond
           ((<= mu (f2cl-lib:int-add n (f2cl-lib:int-sub i) 1))
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (clartg
                 (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                       ((+ ku (f2cl-lib:int-sub mu)
                                           3)
                                        (f2cl-lib:int-sub
                                         (f2cl-lib:int-add i mu) 2))
                                       ((1 ldab) (1 *)) ab-%offset%)
                 (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                       ((+ ku (f2cl-lib:int-sub mu)
                                           2)
                                        (f2cl-lib:int-sub
                                         (f2cl-lib:int-add i mu) 1))
                                       ((1 ldab) (1 *)) ab-%offset%)
                 (f2cl-lib:array-slice rwork-%data% single-float
                                       ((+ i mu
                                           (f2cl-lib:int-sub 1)))
                                       ((1 *)) rwork-%offset%)
                 (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                                       ((+ i mu
```

```
                                    (f2cl-lib:int-sub 1)))
                                ((1 *)) work-%offset%)
            ra)
        (declare (ignore var-0 var-1 var-2 var-3))
        (when var-4 (setf ra var-4)))
      (setf (f2cl-lib:fref ab-%data%
                           ((f2cl-lib:int-add
                             (f2cl-lib:int-sub ku mu) 3)
                            (f2cl-lib:int-sub
                             (f2cl-lib:int-add i mu) 2))
                           ((1 ldab) (1 *)) ab-%offset%)
              ra)
      (crot
       (min
        (the f2cl-lib:integer4
             (f2cl-lib:int-sub (f2cl-lib:int-add kl mu) 2))
        (the f2cl-lib:integer4 (f2cl-lib:int-sub m i)))
       (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                             ((+ ku (f2cl-lib:int-sub mu) 4)
                              (f2cl-lib:int-sub
                               (f2cl-lib:int-add i mu) 2))
                             ((1 ldab) (1 *)) ab-%offset%)
       1
       (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                             ((+ ku (f2cl-lib:int-sub mu) 3)
                              (f2cl-lib:int-sub
                               (f2cl-lib:int-add i mu) 1))
                             ((1 ldab) (1 *)) ab-%offset%)
       1
       (f2cl-lib:array-slice rwork-%data% single-float
                             ((+ i mu (f2cl-lib:int-sub 1)))
                             ((1 *)) rwork-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex8
                             ((+ i mu (f2cl-lib:int-sub 1)))
                             ((1 *)) work-%offset%))))
    (setf nr (f2cl-lib:int-add nr 1))
    (setf j1 (f2cl-lib:int-sub j1 kb1))))
  (cond
   (wantpt
    (f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
                  ((> j j2) nil)
      (tagbody
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (crot n
             (f2cl-lib:array-slice pt-%data% f2cl-lib:complex8
                                   ((+ j kun
                                       (f2cl-lib:int-sub 1))
                                    1)
                                   ((1 ldpt) (1 *))
```

```
                                  pt-%offset%)
                ldpt
                (f2cl-lib:array-slice pt-%data% f2cl-lib:complex8
                                  ((+ j kun) 1)
                                  ((1 ldpt) (1 *))
                                  pt-%offset%)
                ldpt
                (f2cl-lib:array-slice rwork-%data% single-float
                                  ((+ j kun)) ((1 *))
                                  rwork-%offset%)
                (f2cl-lib:conjg
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add j kun)) ((1 *))
                                work-%offset%)))
            (declare (ignore var-1 var-3 var-5 var-6))
            (when var-0 (setf n var-0))
            (when var-2 (setf ldpt var-2))
            (when var-4 (setf ldpt var-4)))
         label60))))
  (cond
   ((> (f2cl-lib:int-add j2 kb) m)
    (setf nr (f2cl-lib:int-sub nr 1))
    (setf j2 (f2cl-lib:int-sub j2 kb1))))
  (f2cl-lib:fdo (j j1 (f2cl-lib:int-add j kb1))
                ((> j j2) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add j kb)) ((1 *))
                           work-%offset%)
             (*
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add j kun))
                             ((1 *)) work-%offset%)
              (f2cl-lib:fref ab-%data%
                             (klu1 (f2cl-lib:int-add j kun))
                             ((1 ldab) (1 *)) ab-%offset%)))
      (setf (f2cl-lib:fref ab-%data%
                           (klu1 (f2cl-lib:int-add j kun))
                           ((1 ldab) (1 *)) ab-%offset%)
             (*
              (f2cl-lib:fref rwork-%data%
                             ((f2cl-lib:int-add j kun))
                             ((1 *)) rwork-%offset%)
              (f2cl-lib:fref ab-%data%
                             (klu1 (f2cl-lib:int-add j kun))
                             ((1 ldab) (1 *)) ab-%offset%)))
     label70))
  (cond ((> ml ml0) (setf ml (f2cl-lib:int-sub ml 1)))
        (t (setf mu (f2cl-lib:int-sub mu 1))))
 label80))
```

```
    label90))))
(cond
 ((and (= ku 0) (> kl 0))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
                     (the f2cl-lib:integer4 n)))
                 nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (clartg
           (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8 (1 i)
                                 ((1 ldab) (1 *)) ab-%offset%)
           (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8 (2 i)
                                 ((1 ldab) (1 *)) ab-%offset%)
           rc rs ra)
        (declare (ignore var-0 var-1))
        (when var-2 (setf rc var-2))
        (when var-3 (setf rs var-3))
        (when var-4 (setf ra var-4)))
      (setf (f2cl-lib:fref ab-%data% (1 i) ((1 ldab) (1 *))
                           ab-%offset%)
            ra)
      (cond
       ((< i n)
        (setf (f2cl-lib:fref ab-%data% (2 i) ((1 ldab) (1 *))
                             ab-%offset%)
              (* rs
                 (f2cl-lib:fref ab-%data% (1 (f2cl-lib:int-add i 1))
                                ((1 ldab) (1 *)) ab-%offset%)))
        (setf (f2cl-lib:fref ab-%data% (1 (f2cl-lib:int-add i 1))
                             ((1 ldab) (1 *)) ab-%offset%)
              (* rc
                 (f2cl-lib:fref ab-%data% (1 (f2cl-lib:int-add i 1))
                                ((1 ldab) (1 *)) ab-%offset%)))))
      (if wantq
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (crot m
               (f2cl-lib:array-slice q-%data% f2cl-lib:complex8 (1 i)
                                     ((1 ldq) (1 *)) q-%offset%)
               1
               (f2cl-lib:array-slice q-%data% f2cl-lib:complex8
                                     (1 (f2cl-lib:int-add i 1))
                                     ((1 ldq) (1 *)) q-%offset%)
               1 rc (f2cl-lib:conjg rs))
            (declare (ignore var-1 var-2 var-3 var-4 var-6))
            (when var-0 (setf m var-0))
```

```
              (when var-5 (setf rc var-5))))
        (if wantc
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (crot ncc
                 (f2cl-lib:array-slice c-%data% f2cl-lib:complex8 (i 1)
                                       ((1 ldc) (1 *)) c-%offset%)
                 ldc
                 (f2cl-lib:array-slice c-%data% f2cl-lib:complex8
                                       ((+ i 1) 1) ((1 ldc) (1 *))
                                       c-%offset%)
                 ldc rc rs)
              (declare (ignore var-1 var-3))
              (when var-0 (setf ncc var-0))
              (when var-2 (setf ldc var-2))
              (when var-4 (setf ldc var-4))
              (when var-5 (setf rc var-5))
              (when var-6 (setf rs var-6))))
       label100)))
    (t
     (cond
      ((and (> ku 0) (< m n))
       (setf rb
              (f2cl-lib:fref ab-%data% (ku (f2cl-lib:int-add m 1))
                             ((1 ldab) (1 *)) ab-%offset%))
       (f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     ((> i 1) nil)
         (tagbody
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
               (clartg
                (f2cl-lib:array-slice ab-%data% f2cl-lib:complex8
                                      ((+ ku 1) i) ((1 ldab) (1 *))
                                      ab-%offset%)
                rb rc rs ra)
             (declare (ignore var-0))
             (when var-1 (setf rb var-1))
             (when var-2 (setf rc var-2))
             (when var-3 (setf rs var-3))
             (when var-4 (setf ra var-4)))
           (setf (f2cl-lib:fref ab-%data% ((f2cl-lib:int-add ku 1) i)
                                ((1 ldab) (1 *)) ab-%offset%)
                 ra)
           (cond
            ((> i 1)
             (setf rb
                     (* (- (f2cl-lib:conjg rs))
                        (f2cl-lib:fref ab-%data% (ku i) ((1 ldab) (1 *))
                                       ab-%offset%)))
             (setf (f2cl-lib:fref ab-%data% (ku i) ((1 ldab) (1 *))
                                  ab-%offset%)
```

```
                    (* rc
                       (f2cl-lib:fref ab-%data% (ku i) ((1 ldab) (1 *))
                                      ab-%offset%)))))
            (if wantpt
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (crot n
                     (f2cl-lib:array-slice pt-%data% f2cl-lib:complex8
                                           (i 1) ((1 ldpt) (1 *))
                                           pt-%offset%)
                     ldpt
                     (f2cl-lib:array-slice pt-%data% f2cl-lib:complex8
                                           ((+ m 1) 1) ((1 ldpt) (1 *))
                                           pt-%offset%)
                     ldpt rc (f2cl-lib:conjg rs))
                  (declare (ignore var-1 var-3 var-6))
                  (when var-0 (setf n var-0))
                  (when var-2 (setf ldpt var-2))
                  (when var-4 (setf ldpt var-4))
                  (when var-5 (setf rc var-5))))
           label110))))))
(setf t$
        (f2cl-lib:fref ab-%data% ((f2cl-lib:int-add ku 1) 1)
                       ((1 ldab) (1 *)) ab-%offset%))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i minmn) nil)
  (tagbody
    (setf abst (abs t$))
    (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) abst)
    (cond ((/= abst zero) (setf t$ (/ t$ abst))) (t (setf t$ cone)))
    (if wantq
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (cscal m t$
             (f2cl-lib:array-slice q-%data% f2cl-lib:complex8 (1 i)
                                   ((1 ldq) (1 *)) q-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf m var-0))
          (when var-1 (setf t$ var-1))))
    (if wantc
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (cscal ncc (f2cl-lib:conjg t$)
             (f2cl-lib:array-slice c-%data% f2cl-lib:complex8 (i 1)
                                   ((1 ldc) (1 *)) c-%offset%)
             ldc)
          (declare (ignore var-1 var-2))
          (when var-0 (setf ncc var-0))
          (when var-3 (setf ldc var-3))))
    (cond
     ((< i minmn)
```

```
              (cond
               ((and (= ku 0) (= kl 0))
                (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) zero)
                (setf t$
                        (f2cl-lib:fref ab-%data% (1 (f2cl-lib:int-add i 1))
                                       ((1 ldab) (1 *)) ab-%offset%)))
               (t
                (cond
                 ((= ku 0)
                  (setf t$
                          (*
                           (f2cl-lib:fref ab-%data% (2 i) ((1 ldab) (1 *))
                                          ab-%offset%)
                           (f2cl-lib:conjg t$))))
                 (t
                  (setf t$
                          (*
                           (f2cl-lib:fref ab-%data% (ku (f2cl-lib:int-add i 1))
                                          ((1 ldab) (1 *)) ab-%offset%)
                           (f2cl-lib:conjg t$)))))
                (setf abst (abs t$))
                (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) abst)
                (cond ((/= abst zero) (setf t$ (/ t$ abst)))
                      (t (setf t$ cone)))
                (if wantpt
                    (multiple-value-bind (var-0 var-1 var-2 var-3)
                        (cscal n t$
                         (f2cl-lib:array-slice pt-%data% f2cl-lib:complex8
                                               ((+ i 1) 1) ((1 ldpt) (1 *))
                                               pt-%offset%)
                         ldpt)
                       (declare (ignore var-2))
                       (when var-0 (setf n var-0))
                       (when var-1 (setf t$ var-1))
                       (when var-3 (setf ldpt var-3))))
                (setf t$
                        (*
                         (f2cl-lib:fref ab-%data%
                                        ((f2cl-lib:int-add ku 1)
                                         (f2cl-lib:int-add i 1))
                                        ((1 ldab) (1 *)) ab-%offset%)
                         (f2cl-lib:conjg t$)))))))
          label120))
       (go end_label)
      end_label
       (return
        (values vect m n ncc nil nil nil nil nil nil nil ldq nil ldpt nil ldc
                nil nil info))))))

(in-package #-gcl #:cl-user #+gcl "CL-USER")
```

```
#+#.(cl:if (cl:find-package '#:f2cl) '(and) '(or))
(eval-when (:load-toplevel :compile-toplevel :execute)
  (setf (gethash 'fortran-to-lisp::cgbbrd
                 fortran-to-lisp::*f2cl-function-info*)
          (fortran-to-lisp::make-f2cl-finfo :arg-types
                                            '((string)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array single-float (*))
                                              (array single-float (*))
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (fortran-to-lisp::integer4)
                                              (array fortran-to-lisp::complex8
                                               (*))
                                              (array single-float (*))
                                              (fortran-to-lisp::integer4))
                                            :return-values
                                            '(fortran-to-lisp::vect
                                              fortran-to-lisp::m
                                              fortran-to-lisp::n
                                              fortran-to-lisp::ncc nil nil nil
                                              nil nil nil nil
                                              fortran-to-lisp::ldq nil
                                              fortran-to-lisp::ldpt nil
                                              fortran-to-lisp::ldc nil nil
                                              fortran-to-lisp::info)
                                            :calls 'nil)))
```

90.113 dbdsdc

90.113.1 Axiom unit tests

— dbdsdc.input —

```
)set break resume
)sys rm -f dbdsdc.output
)spool dbdsdc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.113.2 Axiom help page

— dbdsdc.help —

```
====================================================================
dbdsdc examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DBDSDC - the singular value decomposition (SVD) of a real N-by-N (upper
       or lower) bidiagonal matrix B

SYNOPSIS
       SUBROUTINE DBDSDC( UPLO, COMPQ, N, D, E, U, LDU, VT, LDVT, Q, IQ, WORK,
                          IWORK, INFO )

           CHARACTER      COMPQ, UPLO

           INTEGER        INFO, LDU, LDVT, N

           INTEGER        IQ( * ), IWORK( * )

           DOUBLE         PRECISION  D(  * ), E( * ), Q( * ), U( LDU, * ), VT(
                          LDVT, * ), WORK( * )
```

PURPOSE

DBDSDC computes the singular value decomposition (SVD) of a real N-by-N (upper or lower) bidiagonal matrix B: B = U * S * VT, using a divide and conquer method, where S is a diagonal matrix with non-negative diagonal elements (the singular values of B), and U and VT are orthogonal matrices of left and right singular vectors, respectively. DBDSDC can be used to compute all singular values, and optionally, singular vectors or singular vectors in compact form.

This code makes very mild assumptions about floating point arithmetic. It will work on machines with a guard digit in add/subtract, or on those binary machines without guard digits which subtract like the Cray X-MP, Cray Y-MP, Cray C-90, or Cray-2. It could conceivably fail on hexadecimal or decimal machines without guard digits, but we know of none. See DLASD3 for details.

The code currently calls DLASDQ if singular values only are desired. However, it can be slightly modified to compute singular values using the divide and conquer method.

ARGUMENTS

UPLO (input) CHARACTER*1
= 'U': B is upper bidiagonal.
= 'L': B is lower bidiagonal.

COMPQ (input) CHARACTER*1
Specifies whether singular vectors are to be computed as follows:
= 'N': Compute singular values only;
= 'P': Compute singular values and compute singular vectors in compact form; = 'I': Compute singular values and singular vectors.

N (input) INTEGER
The order of the matrix B. N >= 0.

D (input/output) DOUBLE PRECISION array, dimension (N)
On entry, the n diagonal elements of the bidiagonal matrix B. On exit, if INFO=0, the singular values of B.

E (input/output) DOUBLE PRECISION array, dimension (N-1)
On entry, the elements of E contain the offdiagonal elements of the bidiagonal matrix whose SVD is desired. On exit, E has been destroyed.

U (output) DOUBLE PRECISION array, dimension (LDU,N)
If COMPQ = 'I', then: On exit, if INFO = 0, U contains the left singular vectors of the bidiagonal matrix. For other val-

```
          ues of COMPQ, U is not referenced.

     LDU  (input) INTEGER
          The  leading  dimension of the array U.  LDU >= 1.  If singular
          vectors are desired, then LDU >= max( 1, N ).

     VT   (output) DOUBLE PRECISION array, dimension (LDVT,N)
          If  COMPQ = 'I', then: On exit, if INFO = 0, VT'  contains  the
          right  singular  vectors  of  the bidiagonal matrix.  For other
          values of COMPQ, VT is not referenced.

     LDVT (input) INTEGER
          The leading dimension of the array VT.  LDVT >= 1.  If singular
          vectors are desired, then LDVT >= max( 1, N ).

     Q    (output) DOUBLE PRECISION array, dimension (LDQ)
          If   COMPQ  = 'P', then: On exit, if INFO = 0, Q and IQ contain
          the left and right singular vectors in a compact form,  requir-
          ing  O(N log N) space instead of 2*N**2.  In particular, Q con-
          tains all the DOUBLE PRECISION data in LDQ >= N*(11 +  2*SMLSIZ
          +  8*INT(LOG_2(N/(SMLSIZ+1)))) words of memory, where SMLSIZ is
          returned by ILAENV and is equal to the maximum size of the sub-
          problems  at  the bottom of the computation tree (usually about
          25).  For other values of COMPQ, Q is not referenced.

     IQ   (output) INTEGER array, dimension (LDIQ)
          If  COMPQ = 'P', then: On exit, if INFO = 0, Q and  IQ  contain
          the  left and right singular vectors in a compact form, requir-
          ing O(N log N) space instead of 2*N**2.  In particular, IQ con-
          tains  all  INTEGER  data in LDIQ >= N*(3 + 3*INT(LOG_2(N/(SML-
          SIZ+1)))) words of memory, where SMLSIZ is returned  by  ILAENV
          and is equal to the maximum size of the subproblems at the bot-
          tom of the computation tree (usually about 25).  For other val-
          ues of COMPQ, IQ is not referenced.

     WORK (workspace) DOUBLE PRECISION array, dimension (MAX(1,LWORK))
          If  COMPQ  =  'N'  then  LWORK >= (4 * N).  If COMPQ = 'P' then
          LWORK >= (6 * N).  If COMPQ = 'I' then LWORK >= (3 * N**2 + 4 *
          N).

     IWORK (workspace) INTEGER array, dimension (8*N)

     INFO (output) INTEGER
          = 0:  successful exit.
          < 0:  if INFO = -i, the i-th argument had an illegal value.
          >  0:   The algorithm failed to compute an singular value.  The
          update process of divide and conquer failed.

Further Details
===============
```

```
Based on contributions by
   Ming Gu and Huan Ren, Computer Science Division, University of
   California at Berkeley, USA
```

The input arguments are:

- uplo - simple-array character (1)
- compq - (simple-array character (1)
- n - fixnum
- d - array doublefloat
- e - array doublefloat
- u - array doublefloat
- ldu - fixnum
- vt - doublefloat
- ldvt - fixnum
- q - array doublefloat
- iq - array fixnum
- work - array doublefloat
- iwork - array fixnum
- info - fixnum

The return values are:

- uplo - nil
- compq - nil
- n - nil
- d - nil
- e - nil
- u - nil
- ldu - nil

- vt - nil
- ldvt - nil
- q - nil
- iq - nil
- work - nil
- iwork - nil
- info - info

[dlasr p15210]
[dswap p??]
[dlasda p15068]
[dlasd0 p14913]
[dlamch p14686]
[dlascl p14900]
[dlanst p14740]
[dlaset p15108]
[dlasdq p15089]
[dlartg p14890]
[dcopy p??]
[ilaenv p15476]
[xerbla p??]
[char-equal p??]

90.113.3 fortran code

— dbdsdc.f —

```
*  =====================================================================
      SUBROUTINE DBDSDC( UPLO, COMPQ, N, D, E, U, LDU, VT, LDVT, Q, IQ,
     $                   WORK, IWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          COMPQ, UPLO
      INTEGER            INFO, LDU, LDVT, N
*     ..
```

```
*     .. Array Arguments ..
      INTEGER            IQ( * ), IWORK( * )
      DOUBLE PRECISION   D( * ), E( * ), Q( * ), U( LDU, * ),
     $                   VT( LDVT, * ), WORK( * )
*     ..
*
*  ====================================================================
*  Changed dimension statement in comment describing E from (N) to
*  (N-1).  Sven, 17 Feb 05.
*  ====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE, TWO
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0, TWO = 2.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            DIFL, DIFR, GIVCOL, GIVNUM, GIVPTR, I, IC,
     $                   ICOMPQ, IERR, II, IS, IU, IUPLO, IVT, J, K, KK,
     $                   MLVL, NM1, NSIZE, PERM, POLES, QSTART, SMLSIZ,
     $                   SMLSZP, SQRE, START, WSTART, Z
      DOUBLE PRECISION   CS, EPS, ORGNRM, P, R, SN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANST
      EXTERNAL           LSAME, ILAENV, DLAMCH, DLANST
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DLARTG, DLASCL, DLASD0, DLASDA, DLASDQ,
     $                   DLASET, DLASR, DSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, INT, LOG, SIGN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IUPLO = 0
      IF( LSAME( UPLO, 'U' ) )
     $   IUPLO = 1
      IF( LSAME( UPLO, 'L' ) )
     $   IUPLO = 2
      IF( LSAME( COMPQ, 'N' ) ) THEN
         ICOMPQ = 0
      ELSE IF( LSAME( COMPQ, 'P' ) ) THEN
         ICOMPQ = 1
```

```
      ELSE IF( LSAME( COMPQ, 'I' ) ) THEN
         ICOMPQ = 2
      ELSE
         ICOMPQ = -1
      END IF
      IF( IUPLO.EQ.0 ) THEN
         INFO = -1
      ELSE IF( ICOMPQ.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( ( LDU.LT.1 ) .OR. ( ( ICOMPQ.EQ.2 ) .AND. ( LDU.LT.
     $         N ) ) ) THEN
         INFO = -7
      ELSE IF( ( LDVT.LT.1 ) .OR. ( ( ICOMPQ.EQ.2 ) .AND. ( LDVT.LT.
     $         N ) ) ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DBDSDC', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
      SMLSIZ = ILAENV( 9, 'DBDSDC', ' ', 0, 0, 0, 0 )
      IF( N.EQ.1 ) THEN
         IF( ICOMPQ.EQ.1 ) THEN
            Q( 1 ) = SIGN( ONE, D( 1 ) )
            Q( 1+SMLSIZ*N ) = ONE
         ELSE IF( ICOMPQ.EQ.2 ) THEN
            U( 1, 1 ) = SIGN( ONE, D( 1 ) )
            VT( 1, 1 ) = ONE
         END IF
         D( 1 ) = ABS( D( 1 ) )
         RETURN
      END IF
      NM1 = N - 1
*
*     If matrix lower bidiagonal, rotate to be upper bidiagonal
*     by applying Givens rotations on the left
*
      WSTART = 1
      QSTART = 3
      IF( ICOMPQ.EQ.1 ) THEN
         CALL DCOPY( N, D, 1, Q( 1 ), 1 )
         CALL DCOPY( N-1, E, 1, Q( N+1 ), 1 )
      END IF
```

```
      IF( IUPLO.EQ.2 ) THEN
         QSTART = 5
         WSTART = 2*N - 1
         DO 10 I = 1, N - 1
            CALL DLARTG( D( I ), E( I ), CS, SN, R )
            D( I ) = R
            E( I ) = SN*D( I+1 )
            D( I+1 ) = CS*D( I+1 )
            IF( ICOMPQ.EQ.1 ) THEN
               Q( I+2*N ) = CS
               Q( I+3*N ) = SN
            ELSE IF( ICOMPQ.EQ.2 ) THEN
               WORK( I ) = CS
               WORK( NM1+I ) = -SN
            END IF
   10    CONTINUE
      END IF
*
*     If ICOMPQ = 0, use DLASDQ to compute the singular values.
*
      IF( ICOMPQ.EQ.0 ) THEN
         CALL DLASDQ( 'U', 0, N, 0, 0, 0, D, E, VT, LDVT, U, LDU, U,
     $                LDU, WORK( WSTART ), INFO )
         GO TO 40
      END IF
*
*     If N is smaller than the minimum divide size SMLSIZ, then solve
*     the problem with another solver.
*
      IF( N.LE.SMLSIZ ) THEN
         IF( ICOMPQ.EQ.2 ) THEN
            CALL DLASET( 'A', N, N, ZERO, ONE, U, LDU )
            CALL DLASET( 'A', N, N, ZERO, ONE, VT, LDVT )
            CALL DLASDQ( 'U', 0, N, N, N, 0, D, E, VT, LDVT, U, LDU, U,
     $                   LDU, WORK( WSTART ), INFO )
         ELSE IF( ICOMPQ.EQ.1 ) THEN
            IU = 1
            IVT = IU + N
            CALL DLASET( 'A', N, N, ZERO, ONE, Q( IU+( QSTART-1 )*N ),
     $                   N )
            CALL DLASET( 'A', N, N, ZERO, ONE, Q( IVT+( QSTART-1 )*N ),
     $                   N )
            CALL DLASDQ( 'U', 0, N, N, N, 0, D, E,
     $                   Q( IVT+( QSTART-1 )*N ), N,
     $                   Q( IU+( QSTART-1 )*N ), N,
     $                   Q( IU+( QSTART-1 )*N ), N, WORK( WSTART ),
     $                   INFO )
         END IF
         GO TO 40
      END IF
```

```
*
      IF( ICOMPQ.EQ.2 ) THEN
         CALL DLASET( 'A', N, N, ZERO, ONE, U, LDU )
         CALL DLASET( 'A', N, N, ZERO, ONE, VT, LDVT )
      END IF
*
*     Scale.
*
      ORGNRM = DLANST( 'M', N, D, E )
      IF( ORGNRM.EQ.ZERO )
     $   RETURN
      CALL DLASCL( 'G', 0, 0, ORGNRM, ONE, N, 1, D, N, IERR )
      CALL DLASCL( 'G', 0, 0, ORGNRM, ONE, NM1, 1, E, NM1, IERR )
*
      EPS = (0.9D+0)*DLAMCH( 'Epsilon' )
*
      MLVL = INT( LOG( DBLE( N ) / DBLE( SMLSIZ+1 ) ) / LOG( TWO ) ) + 1
      SMLSZP = SMLSIZ + 1
*
      IF( ICOMPQ.EQ.1 ) THEN
         IU = 1
         IVT = 1 + SMLSIZ
         DIFL = IVT + SMLSZP
         DIFR = DIFL + MLVL
         Z = DIFR + MLVL*2
         IC = Z + MLVL
         IS = IC + 1
         POLES = IS + 1
         GIVNUM = POLES + 2*MLVL
*
         K = 1
         GIVPTR = 2
         PERM = 3
         GIVCOL = PERM + MLVL
      END IF
*
      DO 20 I = 1, N
         IF( ABS( D( I ) ).LT.EPS ) THEN
            D( I ) = SIGN( EPS, D( I ) )
         END IF
   20 CONTINUE
*
      START = 1
      SQRE = 0
*
      DO 30 I = 1, NM1
         IF( ( ABS( E( I ) ).LT.EPS ) .OR. ( I.EQ.NM1 ) ) THEN
*
*        Subproblem found. First determine its size and then
*        apply divide and conquer on it.
```

```
*
            IF( I.LT.NM1 ) THEN
*
*        A subproblem with E(I) small for I < NM1.
*
               NSIZE = I - START + 1
            ELSE IF( ABS( E( I ) ).GE.EPS ) THEN
*
*        A subproblem with E(NM1) not too small but I = NM1.
*
               NSIZE = N - START + 1
            ELSE
*
*        A subproblem with E(NM1) small. This implies an
*        1-by-1 subproblem at D(N). Solve this 1-by-1 problem
*        first.
*
               NSIZE = I - START + 1
               IF( ICOMPQ.EQ.2 ) THEN
                  U( N, N ) = SIGN( ONE, D( N ) )
                  VT( N, N ) = ONE
               ELSE IF( ICOMPQ.EQ.1 ) THEN
                  Q( N+( QSTART-1 )*N ) = SIGN( ONE, D( N ) )
                  Q( N+( SMLSIZ+QSTART-1 )*N ) = ONE
               END IF
               D( N ) = ABS( D( N ) )
            END IF
            IF( ICOMPQ.EQ.2 ) THEN
               CALL DLASD0( NSIZE, SQRE, D( START ), E( START ),
     $                      U( START, START ), LDU, VT( START, START ),
     $                      LDVT, SMLSIZ, IWORK, WORK( WSTART ), INFO )
            ELSE
               CALL DLASDA( ICOMPQ, SMLSIZ, NSIZE, SQRE, D( START ),
     $                      E( START ), Q( START+( IU+QSTART-2 )*N ), N,
     $                      Q( START+( IVT+QSTART-2 )*N ),
     $                      IQ( START+K*N ), Q( START+( DIFL+QSTART-2 )*
     $                      N ), Q( START+( DIFR+QSTART-2 )*N ),
     $                      Q( START+( Z+QSTART-2 )*N ),
     $                      Q( START+( POLES+QSTART-2 )*N ),
     $                      IQ( START+GIVPTR*N ), IQ( START+GIVCOL*N ),
     $                      N, IQ( START+PERM*N ),
     $                      Q( START+( GIVNUM+QSTART-2 )*N ),
     $                      Q( START+( IC+QSTART-2 )*N ),
     $                      Q( START+( IS+QSTART-2 )*N ),
     $                      WORK( WSTART ), IWORK, INFO )
            END IF
            IF( INFO.NE.0 ) THEN
               RETURN
            END IF
            START = I + 1
```

```
         END IF
   30 CONTINUE
*
*     Unscale
*
      CALL DLASCL( 'G', 0, 0, ONE, ORGNRM, N, 1, D, N, IERR )
   40 CONTINUE
*
*     Use Selection Sort to minimize swaps of singular vectors
*
      DO 60 II = 2, N
         I = II - 1
         KK = I
         P = D( I )
         DO 50 J = II, N
            IF( D( J ).GT.P ) THEN
               KK = J
               P = D( J )
            END IF
   50    CONTINUE
         IF( KK.NE.I ) THEN
            D( KK ) = D( I )
            D( I ) = P
            IF( ICOMPQ.EQ.1 ) THEN
               IQ( I ) = KK
            ELSE IF( ICOMPQ.EQ.2 ) THEN
               CALL DSWAP( N, U( 1, I ), 1, U( 1, KK ), 1 )
               CALL DSWAP( N, VT( I, 1 ), LDVT, VT( KK, 1 ), LDVT )
            END IF
         ELSE IF( ICOMPQ.EQ.1 ) THEN
            IQ( I ) = I
         END IF
   60 CONTINUE
*
*     If ICOMPQ = 1, use IQ(N,1) as the indicator for UPLO
*
      IF( ICOMPQ.EQ.1 ) THEN
         IF( IUPLO.EQ.1 ) THEN
            IQ( N ) = 1
         ELSE
            IQ( N ) = 0
         END IF
      END IF
*
*     If B is lower bidiagonal, update U by those Givens rotations
*     which rotated B to be upper bidiagonal
*
      IF( ( IUPLO.EQ.2 ) .AND. ( ICOMPQ.EQ.2 ) )
     $   CALL DLASR( 'L', 'V', 'B', N, N, WORK( 1 ), WORK( N ), U, LDU )
*
```

```
      RETURN
*
*     End of DBDSDC
*
      END
      SUBROUTINE DBDSDC( UPLO, COMPQ, N, D, E, U, LDU, VT, LDVT, Q, IQ,
     $                   WORK, IWORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     December 1, 1999
*
*     .. Scalar Arguments ..
      CHARACTER          COMPQ, UPLO
      INTEGER            INFO, LDU, LDVT, N
*     ..
*     .. Array Arguments ..
      INTEGER            IQ( * ), IWORK( * )
      DOUBLE PRECISION   D( * ), E( * ), Q( * ), U( LDU, * ),
     $                   VT( LDVT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE, TWO
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0, TWO = 2.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            DIFL, DIFR, GIVCOL, GIVNUM, GIVPTR, I, IC,
     $                   ICOMPQ, IERR, II, IS, IU, IUPLO, IVT, J, K, KK,
     $                   MLVL, NM1, NSIZE, PERM, POLES, QSTART, SMLSIZ,
     $                   SMLSZP, SQRE, START, WSTART, Z
      DOUBLE PRECISION   CS, EPS, ORGNRM, P, R, SN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANST
      EXTERNAL           LSAME, ILAENV, DLAMCH, DLANST
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DLARTG, DLASCL, DLASD0, DLASDA, DLASDQ,
     $                   DLASET, DLASR, DSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, INT, LOG, SIGN
*     ..
*     .. Executable Statements ..
```

```
*
*     Test the input parameters.
*
      INFO = 0
*
      IUPLO = 0
      IF( LSAME( UPLO, 'U' ) )
     $   IUPLO = 1
      IF( LSAME( UPLO, 'L' ) )
     $   IUPLO = 2
      IF( LSAME( COMPQ, 'N' ) ) THEN
         ICOMPQ = 0
      ELSE IF( LSAME( COMPQ, 'P' ) ) THEN
         ICOMPQ = 1
      ELSE IF( LSAME( COMPQ, 'I' ) ) THEN
         ICOMPQ = 2
      ELSE
         ICOMPQ = -1
      END IF
      IF( IUPLO.EQ.0 ) THEN
         INFO = -1
      ELSE IF( ICOMPQ.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( ( LDU.LT.1 ) .OR. ( ( ICOMPQ.EQ.2 ) .AND. ( LDU.LT.
     $         N ) ) ) THEN
         INFO = -7
      ELSE IF( ( LDVT.LT.1 ) .OR. ( ( ICOMPQ.EQ.2 ) .AND. ( LDVT.LT.
     $         N ) ) ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DBDSDC', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
      SMLSIZ = ILAENV( 9, 'DBDSDC', ' ', 0, 0, 0, 0 )
      IF( N.EQ.1 ) THEN
         IF( ICOMPQ.EQ.1 ) THEN
            Q( 1 ) = SIGN( ONE, D( 1 ) )
            Q( 1+SMLSIZ*N ) = ONE
         ELSE IF( ICOMPQ.EQ.2 ) THEN
            U( 1, 1 ) = SIGN( ONE, D( 1 ) )
            VT( 1, 1 ) = ONE
         END IF
```

```
         D( 1 ) = ABS( D( 1 ) )
         RETURN
      END IF
      NM1 = N - 1
*
*     If matrix lower bidiagonal, rotate to be upper bidiagonal
*     by applying Givens rotations on the left
*
      WSTART = 1
      QSTART = 3
      IF( ICOMPQ.EQ.1 ) THEN
         CALL DCOPY( N, D, 1, Q( 1 ), 1 )
         CALL DCOPY( N-1, E, 1, Q( N+1 ), 1 )
      END IF
      IF( IUPLO.EQ.2 ) THEN
         QSTART = 5
         WSTART = 2*N - 1
         DO 10 I = 1, N - 1
            CALL DLARTG( D( I ), E( I ), CS, SN, R )
            D( I ) = R
            E( I ) = SN*D( I+1 )
            D( I+1 ) = CS*D( I+1 )
            IF( ICOMPQ.EQ.1 ) THEN
               Q( I+2*N ) = CS
               Q( I+3*N ) = SN
            ELSE IF( ICOMPQ.EQ.2 ) THEN
               WORK( I ) = CS
               WORK( NM1+I ) = -SN
            END IF
   10    CONTINUE
      END IF
*
*     If ICOMPQ = 0, use DLASDQ to compute the singular values.
*
      IF( ICOMPQ.EQ.0 ) THEN
         CALL DLASDQ( 'U', 0, N, 0, 0, 0, D, E, VT, LDVT, U, LDU, U,
     $                LDU, WORK( WSTART ), INFO )
         GO TO 40
      END IF
*
*     If N is smaller than the minimum divide size SMLSIZ, then solve
*     the problem with another solver.
*
      IF( N.LE.SMLSIZ ) THEN
         IF( ICOMPQ.EQ.2 ) THEN
            CALL DLASET( 'A', N, N, ZERO, ONE, U, LDU )
            CALL DLASET( 'A', N, N, ZERO, ONE, VT, LDVT )
            CALL DLASDQ( 'U', 0, N, N, N, 0, D, E, VT, LDVT, U, LDU, U,
     $                   LDU, WORK( WSTART ), INFO )
         ELSE IF( ICOMPQ.EQ.1 ) THEN
```

```
            IU = 1
            IVT = IU + N
            CALL DLASET( 'A', N, N, ZERO, ONE, Q( IU+( QSTART-1 )*N ),
     $                   N )
            CALL DLASET( 'A', N, N, ZERO, ONE, Q( IVT+( QSTART-1 )*N ),
     $                   N )
            CALL DLASDQ( 'U', 0, N, N, N, 0, D, E,
     $                   Q( IVT+( QSTART-1 )*N ), N,
     $                   Q( IU+( QSTART-1 )*N ), N,
     $                   Q( IU+( QSTART-1 )*N ), N, WORK( WSTART ),
     $                   INFO )
         END IF
         GO TO 40
      END IF
*
      IF( ICOMPQ.EQ.2 ) THEN
         CALL DLASET( 'A', N, N, ZERO, ONE, U, LDU )
         CALL DLASET( 'A', N, N, ZERO, ONE, VT, LDVT )
      END IF
*
*     Scale.
*
      ORGNRM = DLANST( 'M', N, D, E )
      IF( ORGNRM.EQ.ZERO )
     $   RETURN
      CALL DLASCL( 'G', 0, 0, ORGNRM, ONE, N, 1, D, N, IERR )
      CALL DLASCL( 'G', 0, 0, ORGNRM, ONE, NM1, 1, E, NM1, IERR )
*
      EPS = DLAMCH( 'Epsilon' )
*
      MLVL = INT( LOG( DBLE( N ) / DBLE( SMLSIZ+1 ) ) / LOG( TWO ) ) + 1
      SMLSZP = SMLSIZ + 1
*
      IF( ICOMPQ.EQ.1 ) THEN
         IU = 1
         IVT = 1 + SMLSIZ
         DIFL = IVT + SMLSZP
         DIFR = DIFL + MLVL
         Z = DIFR + MLVL*2
         IC = Z + MLVL
         IS = IC + 1
         POLES = IS + 1
         GIVNUM = POLES + 2*MLVL
*
         K = 1
         GIVPTR = 2
         PERM = 3
         GIVCOL = PERM + MLVL
      END IF
*
```

```
      DO 20 I = 1, N
         IF( ABS( D( I ) ).LT.EPS ) THEN
            D( I ) = SIGN( EPS, D( I ) )
         END IF
   20 CONTINUE
*
      START = 1
      SQRE = 0
*
      DO 30 I = 1, NM1
         IF( ( ABS( E( I ) ).LT.EPS ) .OR. ( I.EQ.NM1 ) ) THEN
*
*        Subproblem found. First determine its size and then
*        apply divide and conquer on it.
*
            IF( I.LT.NM1 ) THEN
*
*        A subproblem with E(I) small for I < NM1.
*
               NSIZE = I - START + 1
            ELSE IF( ABS( E( I ) ).GE.EPS ) THEN
*
*        A subproblem with E(NM1) not too small but I = NM1.
*
               NSIZE = N - START + 1
            ELSE
*
*        A subproblem with E(NM1) small. This implies an
*        1-by-1 subproblem at D(N). Solve this 1-by-1 problem
*        first.
*
               NSIZE = I - START + 1
               IF( ICOMPQ.EQ.2 ) THEN
                  U( N, N ) = SIGN( ONE, D( N ) )
                  VT( N, N ) = ONE
               ELSE IF( ICOMPQ.EQ.1 ) THEN
                  Q( N+( QSTART-1 )*N ) = SIGN( ONE, D( N ) )
                  Q( N+( SMLSIZ+QSTART-1 )*N ) = ONE
               END IF
               D( N ) = ABS( D( N ) )
            END IF
            IF( ICOMPQ.EQ.2 ) THEN
               CALL DLASD0( NSIZE, SQRE, D( START ), E( START ),
     $                      U( START, START ), LDU, VT( START, START ),
     $                      LDVT, SMLSIZ, IWORK, WORK( WSTART ), INFO )
            ELSE
               CALL DLASDA( ICOMPQ, SMLSIZ, NSIZE, SQRE, D( START ),
     $                      E( START ), Q( START+( IU+QSTART-2 )*N ), N,
     $                      Q( START+( IVT+QSTART-2 )*N ),
     $                      IQ( START+K*N ), Q( START+( DIFL+QSTART-2 )*
```

```
     $                     N ), Q( START+( DIFR+QSTART-2 )*N ),
     $                     Q( START+( Z+QSTART-2 )*N ),
     $                     Q( START+( POLES+QSTART-2 )*N ),
     $                     IQ( START+GIVPTR*N ), IQ( START+GIVCOL*N ),
     $                     N, IQ( START+PERM*N ),
     $                     Q( START+( GIVNUM+QSTART-2 )*N ),
     $                     Q( START+( IC+QSTART-2 )*N ),
     $                     Q( START+( IS+QSTART-2 )*N ),
     $                     WORK( WSTART ), IWORK, INFO )
              IF( INFO.NE.0 ) THEN
                 RETURN
              END IF
           END IF
           START = I + 1
        END IF
   30 CONTINUE
*
*     Unscale
*
      CALL DLASCL( 'G', 0, 0, ONE, ORGNRM, N, 1, D, N, IERR )
   40 CONTINUE
*
*     Use Selection Sort to minimize swaps of singular vectors
*
      DO 60 II = 2, N
         I = II - 1
         KK = I
         P = D( I )
         DO 50 J = II, N
            IF( D( J ).GT.P ) THEN
               KK = J
               P = D( J )
            END IF
   50    CONTINUE
         IF( KK.NE.I ) THEN
            D( KK ) = D( I )
            D( I ) = P
            IF( ICOMPQ.EQ.1 ) THEN
               IQ( I ) = KK
            ELSE IF( ICOMPQ.EQ.2 ) THEN
               CALL DSWAP( N, U( 1, I ), 1, U( 1, KK ), 1 )
               CALL DSWAP( N, VT( I, 1 ), LDVT, VT( KK, 1 ), LDVT )
            END IF
         ELSE IF( ICOMPQ.EQ.1 ) THEN
            IQ( I ) = I
         END IF
   60 CONTINUE
*
*     If ICOMPQ = 1, use IQ(N,1) as the indicator for UPLO
*
```

```
      IF( ICOMPQ.EQ.1 ) THEN
         IF( IUPLO.EQ.1 ) THEN
            IQ( N ) = 1
         ELSE
            IQ( N ) = 0
         END IF
      END IF
*
*     If B is lower bidiagonal, update U by those Givens rotations
*     which rotated B to be upper bidiagonal
*
      IF( ( IUPLO.EQ.2 ) .AND. ( ICOMPQ.EQ.2 ) )
     $   CALL DLASR( 'L', 'V', 'B', N, N, WORK( 1 ), WORK( N ), U, LDU )
*
      RETURN
*
*     End of DBDSDC
*
      END
```

———-

90.113.4 lisp code

— dbdsdc.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0) (two 2.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (ignorable zero one two))
  (defun dbdsdc (uplo compq n d e u ldu vt ldvt q iq work iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork iq)
             (type (array double-float (*)) work q vt u e d)
             (type (f2cl-lib:integer4) info ldvt ldu n)
             (type (string *) compq uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (compq character compq-%data% compq-%offset%)
         (d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%)
         (u double-float u-%data% u-%offset%)
         (vt double-float vt-%data% vt-%offset%)
         (q double-float q-%data% q-%offset%)
         (work double-float work-%data% work-%offset%)
         (iq f2cl-lib:integer4 iq-%data% iq-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
```

```
(prog ((cs 0.0d0) (eps 0.0d0) (orgnrm 0.0d0) (p 0.0d0) (r 0.0d0)
       (sn 0.0d0) (difl 0) (difr 0) (givcol 0) (givnum 0) (givptr 0)
       (i 0) (ic 0) (icompq 0) (ierr 0) (ii 0) (is 0) (iu 0) (iuplo 0)
       (ivt 0) (j 0) (k 0) (kk 0) (mlvl 0) (nm1 0) (nsize 0) (perm 0)
       (poles 0) (qstart 0) (smlsiz 0) (smlszp 0) (sqre 0) (start 0)
       (wstart 0) (z 0))
  (declare (type (double-float) cs eps orgnrm p r sn)
           (type (f2cl-lib:integer4) difl difr givcol givnum givptr i ic
            icompq ierr ii is iu iuplo ivt j k kk mlvl nm1 nsize perm
            poles qstart smlsiz smlszp sqre start wstart z))
  (setf info 0)
  (setf iuplo 0)
  (if (multiple-value-bind (ret-val var-0 var-1)
          (lsame uplo "U")
        (declare (ignore var-1))
        (when var-0 (setf uplo var-0))
        ret-val)
      (setf iuplo 1))
  (if (multiple-value-bind (ret-val var-0 var-1)
          (lsame uplo "L")
        (declare (ignore var-1))
        (when var-0 (setf uplo var-0))
        ret-val)
      (setf iuplo 2))
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame compq "N")
      (declare (ignore var-1))
      (when var-0 (setf compq var-0))
      ret-val)
    (setf icompq 0))
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame compq "P")
      (declare (ignore var-1))
      (when var-0 (setf compq var-0))
      ret-val)
    (setf icompq 1))
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame compq "I")
      (declare (ignore var-1))
      (when var-0 (setf compq var-0))
      ret-val)
    (setf icompq 2))
   (t (setf icompq -1)))
  (cond ((= iuplo 0) (setf info -1)) ((< icompq 0) (setf info -2))
        ((< n 0) (setf info -3))
        ((or (< ldu 1) (and (= icompq 2) (< ldu n))) (setf info -7))
        ((or (< ldvt 1) (and (= icompq 2) (< ldvt n))) (setf info -9)))
  (cond
   ((/= info 0) (xerbla "DBDSDC" (f2cl-lib:int-sub info))
```

```
  (go end_label)))
(if (= n 0)
    (go end_label))
(setf smlsiz (ilaenv 9 "DBDSDC" " " 0 0 0 0))
(cond
 ((= n 1)
  (cond
   ((= icompq 1)
    (setf (f2cl-lib:fref q-%data% (1) ((1 *)) q-%offset%)
            (f2cl-lib:sign one
                           (f2cl-lib:fref d-%data% (1) ((1 *))
                                          d-%offset%)))
    (setf (f2cl-lib:fref q-%data%
                         ((f2cl-lib:int-add 1
                                            (f2cl-lib:int-mul smlsiz
                                                              n)))
                         ((1 *)) q-%offset%)
            one))
   ((= icompq 2)
    (setf (f2cl-lib:fref u-%data% (1 1) ((1 ldu) (1 *)) u-%offset%)
            (f2cl-lib:sign one
                           (f2cl-lib:fref d-%data% (1) ((1 *))
                                          d-%offset%)))
    (setf (f2cl-lib:fref vt-%data% (1 1) ((1 ldvt) (1 *)) vt-%offset%)
            one)))
  (setf (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
          (abs (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)))
  (go end_label)))
(setf nm1 (f2cl-lib:int-sub n 1))
(setf wstart 1)
(setf qstart 3)
(cond
 ((= icompq 1)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy n d 1
       (f2cl-lib:array-slice q-%data% double-float (1) ((1 *))
                             q-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf n var-0)))
  (dcopy (f2cl-lib:int-sub n 1) e 1
   (f2cl-lib:array-slice q-%data% double-float ((+ n 1)) ((1 *))
                         q-%offset%)
   1)))
(cond
 ((= iuplo 2) (setf qstart 5)
  (setf wstart (f2cl-lib:int-sub (f2cl-lib:int-mul 2 n) 1))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
    (tagbody
```

```
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dlartg
             (f2cl-lib:array-slice d-%data% double-float (i) ((1 *))
                                   d-%offset%)
             (f2cl-lib:array-slice e-%data% double-float (i) ((1 *))
                                   e-%offset%)
             cs sn r)
          (declare (ignore var-0 var-1))
          (when var-2 (setf cs var-2))
          (when var-3 (setf sn var-3))
          (when var-4 (setf r var-4)))
        (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) r)
        (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                (* sn
                   (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                  ((1 *)) d-%offset%)))
        (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                             d-%offset%)
                (* cs
                   (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                  ((1 *)) d-%offset%)))
        (cond
         ((= icompq 1)
          (setf (f2cl-lib:fref q-%data%
                               ((f2cl-lib:int-add i
                                                  (f2cl-lib:int-mul 2
                                                                    n)))
                               ((1 *)) q-%offset%)
                  cs)
          (setf (f2cl-lib:fref q-%data%
                               ((f2cl-lib:int-add i
                                                  (f2cl-lib:int-mul 3
                                                                    n)))
                               ((1 *)) q-%offset%)
                  sn))
         ((= icompq 2)
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) cs)
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add nm1 i))
                               ((1 *)) work-%offset%)
                  (- sn))))
      label10))))
(cond
 ((= icompq 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (dlasdq "U" 0 n 0 0 0 d e vt ldvt u ldu u ldu
       (f2cl-lib:array-slice work-%data% double-float (wstart) ((1 *))
                             work-%offset%)
       info)
```

```
    (declare
     (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-10
      var-12 var-14))
    (when var-2 (setf n var-2))
    (when var-9 (setf ldvt var-9))
    (when var-11 (setf ldu var-11))
    (when var-13 (setf ldu var-13))
    (when var-15 (setf info var-15)))
  (go label40)))
(cond
 ((<= n smlsiz)
  (cond
   ((= icompq 2)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "A" n n zero one u ldu)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldu var-6)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "A" n n zero one vt ldvt)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldvt var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14 var-15)
        (dlasdq "U" 0 n n n 0 d e vt ldvt u ldu u ldu
         (f2cl-lib:array-slice work-%data% double-float (wstart)
                               ((1 *)) work-%offset%)
         info)
      (declare
       (ignore var-0 var-1 var-5 var-6 var-7 var-8 var-10 var-12
        var-14))
      (when var-2 (setf n var-2))
      (when var-3 (setf n var-3))
      (when var-4 (setf n var-4))
      (when var-9 (setf ldvt var-9))
      (when var-11 (setf ldu var-11))
      (when var-13 (setf ldu var-13))
      (when var-15 (setf info var-15))))
   ((= icompq 1) (setf iu 1) (setf ivt (f2cl-lib:int-add iu n))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "A" n n zero one
         (f2cl-lib:array-slice q-%data% double-float
```

```
                              ((+ iu
                                  (f2cl-lib:int-mul
                                   (f2cl-lib:int-add qstart
                                                     (f2cl-lib:int-sub
                                                      1))
                                   n)))
                              ((1 *)) q-%offset%)
     n)
  (declare (ignore var-0 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf zero var-3))
  (when var-4 (setf one var-4))
  (when var-6 (setf n var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "A" n n zero one
     (f2cl-lib:array-slice q-%data% double-float
                              ((+ ivt
                                  (f2cl-lib:int-mul
                                   (f2cl-lib:int-add qstart
                                                     (f2cl-lib:int-sub
                                                      1))
                                   n)))
                              ((1 *)) q-%offset%)
     n)
  (declare (ignore var-0 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf zero var-3))
  (when var-4 (setf one var-4))
  (when var-6 (setf n var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15)
    (dlasdq "U" 0 n n n 0 d e
     (f2cl-lib:array-slice q-%data% double-float
                              ((+ ivt
                                  (f2cl-lib:int-mul
                                   (f2cl-lib:int-add qstart
                                                     (f2cl-lib:int-sub
                                                      1))
                                   n)))
                              ((1 *)) q-%offset%)
     n
     (f2cl-lib:array-slice q-%data% double-float
                              ((+ iu
                                  (f2cl-lib:int-mul
                                   (f2cl-lib:int-add qstart
                                                     (f2cl-lib:int-sub
                                                      1))
```

```
                                    n)))
                                ((1 *)) q-%offset%)
          n
          (f2cl-lib:array-slice q-%data% double-float
                                ((+ iu
                                    (f2cl-lib:int-mul
                                     (f2cl-lib:int-add qstart
                                                       (f2cl-lib:int-sub
                                                        1))
                                     n)))
                                ((1 *)) q-%offset%)
          n
          (f2cl-lib:array-slice work-%data% double-float (wstart)
                                ((1 *)) work-%offset%)
          info)
        (declare
         (ignore var-0 var-1 var-5 var-6 var-7 var-8 var-10 var-12
          var-14))
        (when var-2 (setf n var-2))
        (when var-3 (setf n var-3))
        (when var-4 (setf n var-4))
        (when var-9 (setf n var-9))
        (when var-11 (setf n var-11))
        (when var-13 (setf n var-13))
        (when var-15 (setf info var-15)))))
  (go label40)))
(cond
 ((= icompq 2)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" n n zero one u ldu)
    (declare (ignore var-0 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" n n zero one vt ldvt)
    (declare (ignore var-0 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldvt var-6)))))
(setf orgnrm
        (multiple-value-bind (ret-val var-0 var-1 var-2 var-3)
            (dlanst "M" n d e)
          (declare (ignore var-0 var-2 var-3))
          (when var-1 (setf n var-1))
          ret-val))
```

```
(if (= orgnrm zero)
    (go end_label))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dlascl "G" 0 0 orgnrm one n 1 d n ierr)
  (declare (ignore var-0 var-1 var-2 var-6 var-7))
  (when var-3 (setf orgnrm var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf n var-5))
  (when var-8 (setf n var-8))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dlascl "G" 0 0 orgnrm one nm1 1 e nm1 ierr)
  (declare (ignore var-0 var-1 var-2 var-6 var-7))
  (when var-3 (setf orgnrm var-3))
  (when var-4 (setf one var-4))
  (when var-5 (setf nm1 var-5))
  (when var-8 (setf nm1 var-8))
  (when var-9 (setf ierr var-9)))
(setf eps (* 0.9d0 (dlamch "Epsilon")))
(setf mlvl
        (f2cl-lib:int-add
         (f2cl-lib:int
          (/
           (f2cl-lib:flog
            (/ (f2cl-lib:dble n)
               (f2cl-lib:dble (f2cl-lib:int-add smlsiz 1))))
           (f2cl-lib:flog two)))
         1))
(setf smlszp (f2cl-lib:int-add smlsiz 1))
(cond
 ((= icompq 1) (setf iu 1) (setf ivt (f2cl-lib:int-add 1 smlsiz))
  (setf difl (f2cl-lib:int-add ivt smlszp))
  (setf difr (f2cl-lib:int-add difl mlvl))
  (setf z (f2cl-lib:int-add difr (f2cl-lib:int-mul mlvl 2)))
  (setf ic (f2cl-lib:int-add z mlvl)) (setf is (f2cl-lib:int-add ic 1))
  (setf poles (f2cl-lib:int-add is 1))
  (setf givnum (f2cl-lib:int-add poles (f2cl-lib:int-mul 2 mlvl)))
  (setf k 1) (setf givptr 2) (setf perm 3)
  (setf givcol (f2cl-lib:int-add perm mlvl))))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((< (abs (f2cl-lib:fref d (i) ((1 *)))) eps)
      (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
              (f2cl-lib:sign eps
                             (f2cl-lib:fref d-%data% (i) ((1 *))
                                            d-%offset%)))))
```

```
   label20))
(setf start 1)
(setf sqre 0)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i nm1) nil)
  (tagbody
    (cond
     ((or (< (abs (f2cl-lib:fref e (i) ((1 *)))) eps) (= i nm1))
      (cond
       ((< i nm1)
        (setf nsize (f2cl-lib:int-add (f2cl-lib:int-sub i start) 1)))
       ((>= (abs (f2cl-lib:fref e (i) ((1 *)))) eps)
        (setf nsize (f2cl-lib:int-add (f2cl-lib:int-sub n start) 1)))
       (t (setf nsize (f2cl-lib:int-add (f2cl-lib:int-sub i start) 1))
        (cond
         ((= icompq 2)
          (setf (f2cl-lib:fref u-%data% (n n) ((1 ldu) (1 *))
                               u-%offset%)
                  (f2cl-lib:sign one
                                 (f2cl-lib:fref d-%data% (n) ((1 *))
                                                d-%offset%)))
          (setf (f2cl-lib:fref vt-%data% (n n) ((1 ldvt) (1 *))
                               vt-%offset%)
                  one))
         ((= icompq 1)
          (setf (f2cl-lib:fref q-%data%
                               ((f2cl-lib:int-add n
                                                  (f2cl-lib:int-mul
                                                   (f2cl-lib:int-sub
                                                    qstart 1)
                                                   n)))
                               ((1 *)) q-%offset%)
                  (f2cl-lib:sign one
                                 (f2cl-lib:fref d-%data% (n) ((1 *))
                                                d-%offset%)))
          (setf (f2cl-lib:fref q-%data%
                               ((f2cl-lib:int-add n
                                                  (f2cl-lib:int-mul
                                                   (f2cl-lib:int-sub
                                                    (f2cl-lib:int-add
                                                     smlsiz qstart)
                                                    1)
                                                   n)))
                               ((1 *)) q-%offset%)
                  one)))
        (setf (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                (abs
                 (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))))
      (cond
       ((= icompq 2)
```

```
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11)
     (dlasd0 nsize sqre
      (f2cl-lib:array-slice d-%data% double-float (start)
                            ((1 *)) d-%offset%)
      (f2cl-lib:array-slice e-%data% double-float (start)
                            ((1 *)) e-%offset%)
      (f2cl-lib:array-slice u-%data% double-float (start start)
                            ((1 ldu) (1 *)) u-%offset%)
      ldu
      (f2cl-lib:array-slice vt-%data% double-float (start start)
                            ((1 ldvt) (1 *)) vt-%offset%)
      ldvt smlsiz iwork
      (f2cl-lib:array-slice work-%data% double-float (wstart)
                            ((1 *)) work-%offset%)
      info)
   (declare (ignore var-2 var-3 var-4 var-6 var-9 var-10))
   (when var-0 (setf nsize var-0))
   (when var-1 (setf sqre var-1))
   (when var-5 (setf ldu var-5))
   (when var-7 (setf ldvt var-7))
   (when var-8 (setf smlsiz var-8))
   (when var-11 (setf info var-11))))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
      var-17 var-18 var-19 var-20 var-21 var-22 var-23)
     (dlasda icompq smlsiz nsize sqre
      (f2cl-lib:array-slice d-%data% double-float (start)
                            ((1 *)) d-%offset%)
      (f2cl-lib:array-slice e-%data% double-float (start)
                            ((1 *)) e-%offset%)
      (f2cl-lib:array-slice q-%data% double-float
                            ((+ start
                                (f2cl-lib:int-mul
                                 (f2cl-lib:int-add iu qstart
                                                   (f2cl-lib:int-sub
                                                    2))
                                 n)))
                            ((1 *)) q-%offset%)
      n
      (f2cl-lib:array-slice q-%data% double-float
                            ((+ start
                                (f2cl-lib:int-mul
                                 (f2cl-lib:int-add ivt qstart
                                                   (f2cl-lib:int-sub
                                                    2))
                                 n)))
```

```
                        ((1 *)) q-%offset%)
(f2cl-lib:array-slice iq-%data% f2cl-lib:integer4
                        ((+ start (f2cl-lib:int-mul k n)))
                        ((1 *)) iq-%offset%)
(f2cl-lib:array-slice q-%data% double-float
                        ((+ start
                            (f2cl-lib:int-mul
                             (f2cl-lib:int-add difl qstart
                                               (f2cl-lib:int-sub
                                                2))
                             n)))
                        ((1 *)) q-%offset%)
(f2cl-lib:array-slice q-%data% double-float
                        ((+ start
                            (f2cl-lib:int-mul
                             (f2cl-lib:int-add difr qstart
                                               (f2cl-lib:int-sub
                                                2))
                             n)))
                        ((1 *)) q-%offset%)
(f2cl-lib:array-slice q-%data% double-float
                        ((+ start
                            (f2cl-lib:int-mul
                             (f2cl-lib:int-add z qstart
                                               (f2cl-lib:int-sub
                                                2))
                             n)))
                        ((1 *)) q-%offset%)
(f2cl-lib:array-slice q-%data% double-float
                        ((+ start
                            (f2cl-lib:int-mul
                             (f2cl-lib:int-add poles qstart
                                               (f2cl-lib:int-sub
                                                2))
                             n)))
                        ((1 *)) q-%offset%)
(f2cl-lib:array-slice iq-%data% f2cl-lib:integer4
                        ((+ start
                            (f2cl-lib:int-mul givptr n)))
                        ((1 *)) iq-%offset%)
(f2cl-lib:array-slice iq-%data% f2cl-lib:integer4
                        ((+ start
                            (f2cl-lib:int-mul givcol n)))
                        ((1 *)) iq-%offset%)
n
(f2cl-lib:array-slice iq-%data% f2cl-lib:integer4
                        ((+ start
                            (f2cl-lib:int-mul perm n)))
                        ((1 *)) iq-%offset%)
(f2cl-lib:array-slice q-%data% double-float
```

```
                                        ((+ start
                                            (f2cl-lib:int-mul
                                             (f2cl-lib:int-add givnum qstart
                                                               (f2cl-lib:int-sub
                                                                2))
                                             n)))
                                        ((1 *)) q-%offset%)
                    (f2cl-lib:array-slice q-%data% double-float
                                        ((+ start
                                            (f2cl-lib:int-mul
                                             (f2cl-lib:int-add ic qstart
                                                               (f2cl-lib:int-sub
                                                                2))
                                             n)))
                                        ((1 *)) q-%offset%)
                    (f2cl-lib:array-slice q-%data% double-float
                                        ((+ start
                                            (f2cl-lib:int-mul
                                             (f2cl-lib:int-add is qstart
                                                               (f2cl-lib:int-sub
                                                                2))
                                             n)))
                                        ((1 *)) q-%offset%)
                    (f2cl-lib:array-slice work-%data% double-float (wstart)
                                        ((1 *)) work-%offset%)
                    iwork info)
                  (declare
                   (ignore var-4 var-5 var-6 var-8 var-9 var-10 var-11 var-12
                    var-13 var-14 var-15 var-17 var-18 var-19 var-20 var-21
                    var-22))
                  (when var-0 (setf icompq var-0))
                  (when var-1 (setf smlsiz var-1))
                  (when var-2 (setf nsize var-2))
                  (when var-3 (setf sqre var-3))
                  (when var-7 (setf n var-7))
                  (when var-16 (setf n var-16))
                  (when var-23 (setf info var-23)))))
            (cond ((/= info 0) (go end_label)))
            (setf start (f2cl-lib:int-add i 1))))
        label30))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (dlascl "G" 0 0 one orgnrm n 1 d n ierr)
       (declare (ignore var-0 var-1 var-2 var-6 var-7))
       (when var-3 (setf one var-3))
       (when var-4 (setf orgnrm var-4))
       (when var-5 (setf n var-5))
       (when var-8 (setf n var-8))
       (when var-9 (setf ierr var-9)))
    label40
```

```
(f2cl-lib:fdo (ii 2 (f2cl-lib:int-add ii 1))
              ((> ii n) nil)
  (tagbody
    (setf i (f2cl-lib:int-sub ii 1))
    (setf kk i)
    (setf p (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
    (f2cl-lib:fdo (j ii (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (cond
         ((> (f2cl-lib:fref d (j) ((1 *))) p) (setf kk j)
          (setf p (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%))))
       label50))
    (cond
     ((/= kk i)
      (setf (f2cl-lib:fref d-%data% (kk) ((1 *)) d-%offset%)
              (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
      (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) p)
      (cond
       ((= icompq 1)
        (setf (f2cl-lib:fref iq-%data% (i) ((1 *)) iq-%offset%) kk))
       ((= icompq 2)
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dswap n
             (f2cl-lib:array-slice u-%data% double-float (1 i)
                                   ((1 ldu) (1 *)) u-%offset%)
             1
             (f2cl-lib:array-slice u-%data% double-float (1 kk)
                                   ((1 ldu) (1 *)) u-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf n var-0)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dswap n
             (f2cl-lib:array-slice vt-%data% double-float (i 1)
                                   ((1 ldvt) (1 *)) vt-%offset%)
             ldvt
             (f2cl-lib:array-slice vt-%data% double-float (kk 1)
                                   ((1 ldvt) (1 *)) vt-%offset%)
             ldvt)
          (declare (ignore var-1 var-3))
          (when var-0 (setf n var-0))
          (when var-2 (setf ldvt var-2))
          (when var-4 (setf ldvt var-4))))))
     ((= icompq 1)
      (setf (f2cl-lib:fref iq-%data% (i) ((1 *)) iq-%offset%) i)))
   label60))
(cond
 ((= icompq 1)
  (cond
```

```
          ((= iuplo 1)
           (setf (f2cl-lib:fref iq-%data% (n) ((1 *)) iq-%offset%) 1))
          (t (setf (f2cl-lib:fref iq-%data% (n) ((1 *)) iq-%offset%) 0)))))
       (if (and (= iuplo 2) (= icompq 2))
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
               (dlasr "L" "V" "B" n n
                (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                                      work-%offset%)
                (f2cl-lib:array-slice work-%data% double-float (n) ((1 *))
                                      work-%offset%)
                u ldu)
             (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
             (when var-3 (setf n var-3))
             (when var-4 (setf n var-4))
             (when var-8 (setf ldu var-8))))
       (go end_label)
      end_label
       (return
        (values uplo compq n nil nil nil ldu nil ldvt nil nil nil nil
                info))))))
```

———-

90.114 dbdsqr

90.114.1 Axiom unit tests

— **dbdsqr.input** —

```
)set break resume
)sys rm -f dbdsqr.output
)spool dbdsqr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.114.2 Axiom help page

— dbdsqr.help —

```
=====================================================================
dbdsqr examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

NAME
       DBDSQR  -  the  singular  values and, optionally, the right and/or left
       singular vectors from the singular value decomposition (SVD) of a  real
       N-by-N  (upper  or  lower) bidiagonal matrix B using the implicit zero-
       shift QR algorithm

SYNOPSIS
       SUBROUTINE DBDSQR( UPLO, N, NCVT, NRU, NCC, D, E, VT, LDVT, U, LDU,  C,
                          LDC, WORK, INFO )

           CHARACTER      UPLO

           INTEGER        INFO, LDC, LDU, LDVT, N, NCC, NCVT, NRU

           DOUBLE         PRECISION  C( LDC, * ), D( * ), E( * ), U( LDU, * ),
                          VT( LDVT, * ), WORK( * )

PURPOSE
       DBDSQR computes the singular values and, optionally, the  right  and/or
       left  singular vectors from the singular value decomposition (SVD) of a
       real N-by-N (upper or lower) bidiagonal matrix  B  using  the  implicit
       zero-shift QR algorithm.  The SVD of B has the form

          B = Q * S * P**T

       where  S  is the diagonal matrix of singular values, Q is an orthogonal
       matrix of left singular vectors, and P is an orthogonal matrix of right
       singular vectors.  If left singular vectors are requested, this subrou-
       tine actually returns U*Q instead of Q, and, if right singular  vectors
       are  requested,  this  subroutine  returns P**T*VT instead of P**T, for
       given real input matrices U and VT.  When U and VT are  the  orthogonal
       matrices  that  reduce  a  general  matrix  A  to bidiagonal form:  A =
       U*B*VT, as computed by DGEBRD, then

          A = (U*Q) * S * (P**T*VT)

       is the SVD of A.  Optionally, the subroutine may  also  compute  Q**T*C
       for a given real input matrix C.

       See "Computing  Small Singular Values of Bidiagonal Matrices With Guar-
```

```
anteed High Relative Accuracy," by J. Demmel and W. Kahan, LAPACK Work-
ing Note #3 (or SIAM J. Sci. Statist. Comput. vol. 11, no. 5, pp.
873-912, Sept 1990) and
"Accurate singular values and differential qd algorithms," by B. Par-
lett and V. Fernando, Technical Report CPAM-554, Mathematics Depart-
ment, University of California at Berkeley, July 1992 for a detailed
description of the algorithm.


ARGUMENTS
      UPLO    (input) CHARACTER*1
              = 'U':  B is upper bidiagonal;
              = 'L':  B is lower bidiagonal.

      N       (input) INTEGER
              The order of the matrix B.  N >= 0.

      NCVT    (input) INTEGER
              The number of columns of the matrix VT. NCVT >= 0.

      NRU     (input) INTEGER
              The number of rows of the matrix U. NRU >= 0.

      NCC     (input) INTEGER
              The number of columns of the matrix C. NCC >= 0.

      D       (input/output) DOUBLE PRECISION array, dimension (N)
              On entry, the n diagonal elements of the bidiagonal matrix B.
              On exit, if INFO=0, the singular values of B in decreasing
              order.

      E       (input/output) DOUBLE PRECISION array, dimension (N-1)
              On entry, the N-1 offdiagonal elements of the bidiagonal matrix
              B.  On exit, if INFO = 0, E is destroyed; if INFO > 0, D and E
              will contain the diagonal and superdiagonal elements of a bidi-
              agonal matrix orthogonally equivalent to the one given as
              input.

      VT      (input/output) DOUBLE PRECISION array, dimension (LDVT, NCVT)
              On entry, an N-by-NCVT matrix VT.  On exit, VT is overwritten
              by P**T * VT.  Not referenced if NCVT = 0.

      LDVT    (input) INTEGER
              The leading dimension of the array VT.  LDVT >= max(1,N) if
              NCVT > 0; LDVT >= 1 if NCVT = 0.

      U       (input/output) DOUBLE PRECISION array, dimension (LDU, N)
              On entry, an NRU-by-N matrix U.  On exit, U is overwritten by U
              * Q.  Not referenced if NRU = 0.
```

```
      LDU     (input) INTEGER
              The leading dimension of the array U.  LDU >= max(1,NRU).

      C       (input/output) DOUBLE PRECISION array, dimension (LDC, NCC)
              On entry, an N-by-NCC matrix C.  On exit, C is  overwritten  by
              Q**T * C.  Not referenced if NCC = 0.

      LDC     (input) INTEGER
              The leading dimension of the array C.  LDC >= max(1,N) if NCC >
              0; LDC >=1 if NCC = 0.

      WORK    (workspace) DOUBLE PRECISION array, dimension (2*N)
              if NCVT = NRU = NCC = 0, (max(1, 4*N)) otherwise

      INFO    (output) INTEGER
              = 0:  successful exit
              < 0:  If INFO = -i, the i-th argument had an illegal value
              > 0:  the algorithm did not converge; D and E contain the  ele-
              ments  of  a bidiagonal matrix which is orthogonally similar to
              the input matrix B;  if INFO = i, i elements of E have not con-
              verged to zero.

PARAMETERS
      TOLMUL  DOUBLE PRECISION, default = max(10,min(100,EPS**(-1/8)))
              TOLMUL  controls  the convergence criterion of the QR loop.  If
              it is positive, TOLMUL*EPS is the desired relative precision in
              the  computed  singular  values.   If  it is negative, abs(TOL-
              MUL*EPS*sigma_max) is the desired absolute accuracy in the com-
              puted  singular  values  (corresponds  to  relative  accuracy
              abs(TOLMUL*EPS) in the  largest  singular  value.   abs(TOLMUL)
              should  be  between 1 and 1/EPS, and preferably between 10 (for
              fast convergence) and .1/EPS (for there to be some accuracy  in
              the  results).  Default is to lose at either one eighth or 2 of
              the available decimal digits in each  computed  singular  value
              (whichever is smaller).

      MAXITR  INTEGER, default = 6
              MAXITR  controls  the maximum number of passes of the algorithm
              through its inner loop. The algorithms stops (and so  fails  to
              converge)  if  the  number  of  passes  through  the inner loop
              exceeds MAXITR*N**2.
```

———-

90.114.3 fortran code

— dbdsqr.f —

```
*  =========================
*>
*> \verbatim
*>  TOLMUL  DOUBLE PRECISION, default = max(10,min(100,EPS**(-1/8)))
*>          TOLMUL controls the convergence criterion of the QR loop.
*>          If it is positive, TOLMUL*EPS is the desired relative
*>             precision in the computed singular values.
*>          If it is negative, abs(TOLMUL*EPS*sigma_max) is the
*>             desired absolute accuracy in the computed singular
*>             values (corresponds to relative accuracy
*>             abs(TOLMUL*EPS) in the largest singular value.
*>          abs(TOLMUL) should be between 1 and 1/EPS, and preferably
*>             between 10 (for fast convergence) and .1/EPS
*>             (for there to be some accuracy in the results).
*>          Default is to lose at either one eighth or 2 of the
*>             available decimal digits in each computed singular value
*>             (whichever is smaller).
*>
*>  MAXITR  INTEGER, default = 6
*>          MAXITR controls the maximum number of passes of the
*>          algorithm through its inner loop. The algorithms stops
*>          (and so fails to converge) if the number of passes
*>          through the inner loop exceeds MAXITR*N**2.
*> \endverbatim
*
*  Authors:
*  ========
*
*> \author Univ. of Tennessee
*> \author Univ. of California Berkeley
*> \author Univ. of Colorado Denver
*> \author NAG Ltd.
*
*> \date November 2011
*
*> \ingroup auxOTHERcomputational
*
*  =====================================================================
      SUBROUTINE DBDSQR( UPLO, N, NCVT, NRU, NCC, D, E, VT, LDVT, U,
     $                   LDU, C, LDC, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDC, LDU, LDVT, N, NCC, NCVT, NRU
*     ..
```

```
*     .. Array Arguments ..
      DOUBLE PRECISION   C( LDC, * ), D( * ), E( * ), U( LDU, * ),
     $                   VT( LDVT, * ), WORK( * )
*     ..
*
*  ====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D0 )
      DOUBLE PRECISION   NEGONE
      PARAMETER          ( NEGONE = -1.0D0 )
      DOUBLE PRECISION   HNDRTH
      PARAMETER          ( HNDRTH = 0.01D0 )
      DOUBLE PRECISION   TEN
      PARAMETER          ( TEN = 10.0D0 )
      DOUBLE PRECISION   HNDRD
      PARAMETER          ( HNDRD = 100.0D0 )
      DOUBLE PRECISION   MEIGTH
      PARAMETER          ( MEIGTH = -0.125D0 )
      INTEGER            MAXITR
      PARAMETER          ( MAXITR = 6 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LOWER, ROTATE
      INTEGER            I, IDIR, ISUB, ITER, J, LL, LLL, M, MAXIT, NM1,
     $                   NM12, NM13, OLDLL, OLDM
      DOUBLE PRECISION   ABSE, ABSS, COSL, COSR, CS, EPS, F, G, H, MU,
     $                   OLDCS, OLDSN, R, SHIFT, SIGMN, SIGMX, SINL,
     $                   SINR, SLL, SMAX, SMIN, SMINL, SMINOA,
     $                   SN, THRESH, TOL, TOLMUL, UNFL
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARTG, DLAS2, DLASQ1, DLASR, DLASV2, DROT,
     $                   DSCAL, DSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, MAX, MIN, SIGN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
```

```
      INFO = 0
      LOWER = LSAME( UPLO, 'L' )
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LOWER ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NCVT.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRU.LT.0 ) THEN
         INFO = -4
      ELSE IF( NCC.LT.0 ) THEN
         INFO = -5
      ELSE IF( ( NCVT.EQ.0 .AND. LDVT.LT.1 ) .OR.
     $         ( NCVT.GT.0 .AND. LDVT.LT.MAX( 1, N ) ) ) THEN
         INFO = -9
      ELSE IF( LDU.LT.MAX( 1, NRU ) ) THEN
         INFO = -11
      ELSE IF( ( NCC.EQ.0 .AND. LDC.LT.1 ) .OR.
     $         ( NCC.GT.0 .AND. LDC.LT.MAX( 1, N ) ) ) THEN
         INFO = -13
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DBDSQR', -INFO )
         RETURN
      END IF
      IF( N.EQ.0 )
     $   RETURN
      IF( N.EQ.1 )
     $   GO TO 160
*
*     ROTATE is true if any singular vectors desired, false otherwise
*
      ROTATE = ( NCVT.GT.0 ) .OR. ( NRU.GT.0 ) .OR. ( NCC.GT.0 )
*
*     If no singular vectors desired, use qd algorithm
*
      IF( .NOT.ROTATE ) THEN
         CALL DLASQ1( N, D, E, WORK, INFO )
*
*     If INFO equals 2, dqds didn't finish, try to finish
*
         IF( INFO .NE. 2 ) RETURN
         INFO = 0
      END IF
*
      NM1 = N - 1
      NM12 = NM1 + NM1
      NM13 = NM12 + NM1
      IDIR = 0
*
```

```
*     Get machine constants
*
      EPS = DLAMCH( 'Epsilon' )
      UNFL = DLAMCH( 'Safe minimum' )
*
*     If matrix lower bidiagonal, rotate to be upper bidiagonal
*     by applying Givens rotations on the left
*
      IF( LOWER ) THEN
         DO 10 I = 1, N - 1
            CALL DLARTG( D( I ), E( I ), CS, SN, R )
            D( I ) = R
            E( I ) = SN*D( I+1 )
            D( I+1 ) = CS*D( I+1 )
            WORK( I ) = CS
            WORK( NM1+I ) = SN
   10    CONTINUE
*
*        Update singular vectors if desired
*
         IF( NRU.GT.0 )
     $      CALL DLASR( 'R', 'V', 'F', NRU, N, WORK( 1 ), WORK( N ), U,
     $                  LDU )
         IF( NCC.GT.0 )
     $      CALL DLASR( 'L', 'V', 'F', N, NCC, WORK( 1 ), WORK( N ), C,
     $                  LDC )
      END IF
*
*     Compute singular values to relative accuracy TOL
*     (By setting TOL to be negative, algorithm will compute
*     singular values to absolute accuracy ABS(TOL)*norm(input matrix))
*
      TOLMUL = MAX( TEN, MIN( HNDRD, EPS**MEIGTH ) )
      TOL = TOLMUL*EPS
*
*     Compute approximate maximum, minimum singular values
*
      SMAX = ZERO
      DO 20 I = 1, N
         SMAX = MAX( SMAX, ABS( D( I ) ) )
   20 CONTINUE
      DO 30 I = 1, N - 1
         SMAX = MAX( SMAX, ABS( E( I ) ) )
   30 CONTINUE
      SMINL = ZERO
      IF( TOL.GE.ZERO ) THEN
*
*        Relative accuracy desired
*
         SMINOA = ABS( D( 1 ) )
```

```
         IF( SMINOA.EQ.ZERO )
     $      GO TO 50
         MU = SMINOA
         DO 40 I = 2, N
            MU = ABS( D( I ) )*( MU / ( MU+ABS( E( I-1 ) ) ) )
            SMINOA = MIN( SMINOA, MU )
            IF( SMINOA.EQ.ZERO )
     $         GO TO 50
   40    CONTINUE
   50    CONTINUE
         SMINOA = SMINOA / SQRT( DBLE( N ) )
         THRESH = MAX( TOL*SMINOA, MAXITR*N*N*UNFL )
      ELSE
*
*        Absolute accuracy desired
*
         THRESH = MAX( ABS( TOL )*SMAX, MAXITR*N*N*UNFL )
      END IF
*
*     Prepare for main iteration loop for the singular values
*     (MAXIT is the maximum number of passes through the inner
*     loop permitted before nonconvergence signalled.)
*
      MAXIT = MAXITR*N*N
      ITER = 0
      OLDLL = -1
      OLDM = -1
*
*     M points to last element of unconverged part of matrix
*
      M = N
*
*     Begin main iteration loop
*
   60 CONTINUE
*
*     Check for convergence or exceeding iteration count
*
      IF( M.LE.1 )
     $   GO TO 160
      IF( ITER.GT.MAXIT )
     $   GO TO 200
*
*     Find diagonal block of matrix to work on
*
      IF( TOL.LT.ZERO .AND. ABS( D( M ) ).LE.THRESH )
     $   D( M ) = ZERO
      SMAX = ABS( D( M ) )
      SMIN = SMAX
      DO 70 LLL = 1, M - 1
```

```
         LL = M - LLL
         ABSS = ABS( D( LL ) )
         ABSE = ABS( E( LL ) )
         IF( TOL.LT.ZERO .AND. ABSS.LE.THRESH )
     $      D( LL ) = ZERO
         IF( ABSE.LE.THRESH )
     $      GO TO 80
         SMIN = MIN( SMIN, ABSS )
         SMAX = MAX( SMAX, ABSS, ABSE )
   70 CONTINUE
      LL = 0
      GO TO 90
   80 CONTINUE
      E( LL ) = ZERO
*
*     Matrix splits since E(LL) = 0
*
      IF( LL.EQ.M-1 ) THEN
*
*        Convergence of bottom singular value, return to top of loop
*
         M = M - 1
         GO TO 60
      END IF
   90 CONTINUE
      LL = LL + 1
*
*     E(LL) through E(M-1) are nonzero, E(LL-1) is zero
*
      IF( LL.EQ.M-1 ) THEN
*
*        2 by 2 block, handle separately
*
         CALL DLASV2( D( M-1 ), E( M-1 ), D( M ), SIGMN, SIGMX, SINR,
     $                COSR, SINL, COSL )
         D( M-1 ) = SIGMX
         E( M-1 ) = ZERO
         D( M ) = SIGMN
*
*        Compute singular vectors, if desired
*
         IF( NCVT.GT.0 )
     $      CALL DROT( NCVT, VT( M-1, 1 ), LDVT, VT( M, 1 ), LDVT, COSR,
     $                 SINR )
         IF( NRU.GT.0 )
     $      CALL DROT( NRU, U( 1, M-1 ), 1, U( 1, M ), 1, COSL, SINL )
         IF( NCC.GT.0 )
     $      CALL DROT( NCC, C( M-1, 1 ), LDC, C( M, 1 ), LDC, COSL,
     $                 SINL )
         M = M - 2
```

```
         GO TO 60
      END IF
*
*     If working on new submatrix, choose shift direction
*     (from larger end diagonal element towards smaller)
*
      IF( LL.GT.OLDM .OR. M.LT.OLDLL ) THEN
         IF( ABS( D( LL ) ).GE.ABS( D( M ) ) ) THEN
*
*           Chase bulge from top (big end) to bottom (small end)
*
            IDIR = 1
         ELSE
*
*           Chase bulge from bottom (big end) to top (small end)
*
            IDIR = 2
         END IF
      END IF
*
*     Apply convergence tests
*
      IF( IDIR.EQ.1 ) THEN
*
*        Run convergence test in forward direction
*        First apply standard test to bottom of matrix
*
         IF( ABS( E( M-1 ) ).LE.ABS( TOL )*ABS( D( M ) ) .OR.
     $       ( TOL.LT.ZERO .AND. ABS( E( M-1 ) ).LE.THRESH ) ) THEN
            E( M-1 ) = ZERO
            GO TO 60
         END IF
*
         IF( TOL.GE.ZERO ) THEN
*
*           If relative accuracy desired,
*           apply convergence criterion forward
*
            MU = ABS( D( LL ) )
            SMINL = MU
            DO 100 LLL = LL, M - 1
               IF( ABS( E( LLL ) ).LE.TOL*MU ) THEN
                  E( LLL ) = ZERO
                  GO TO 60
               END IF
               MU = ABS( D( LLL+1 ) )*( MU / ( MU+ABS( E( LLL ) ) ) )
               SMINL = MIN( SMINL, MU )
  100       CONTINUE
         END IF
*
```

```
      ELSE
*
*        Run convergence test in backward direction
*        First apply standard test to top of matrix
*
         IF( ABS( E( LL ) ).LE.ABS( TOL )*ABS( D( LL ) ) .OR.
     $       ( TOL.LT.ZERO .AND. ABS( E( LL ) ).LE.THRESH ) ) THEN
            E( LL ) = ZERO
            GO TO 60
         END IF
*
         IF( TOL.GE.ZERO ) THEN
*
*           If relative accuracy desired,
*           apply convergence criterion backward
*
            MU = ABS( D( M ) )
            SMINL = MU
            DO 110 LLL = M - 1, LL, -1
               IF( ABS( E( LLL ) ).LE.TOL*MU ) THEN
                  E( LLL ) = ZERO
                  GO TO 60
               END IF
               MU = ABS( D( LLL ) )*( MU / ( MU+ABS( E( LLL ) ) ) )
               SMINL = MIN( SMINL, MU )
  110       CONTINUE
         END IF
      END IF
      OLDLL = LL
      OLDM = M
*
*     Compute shift.  First, test if shifting would ruin relative
*     accuracy, and if so set the shift to zero.
*
      IF( TOL.GE.ZERO .AND. N*TOL*( SMINL / SMAX ).LE.
     $    MAX( EPS, HNDRTH*TOL ) ) THEN
*
*        Use a zero shift to avoid loss of relative accuracy
*
         SHIFT = ZERO
      ELSE
*
*        Compute the shift from 2-by-2 block at end of matrix
*
         IF( IDIR.EQ.1 ) THEN
            SLL = ABS( D( LL ) )
            CALL DLAS2( D( M-1 ), E( M-1 ), D( M ), SHIFT, R )
         ELSE
            SLL = ABS( D( M ) )
            CALL DLAS2( D( LL ), E( LL ), D( LL+1 ), SHIFT, R )
```

```
         END IF
*
*        Test if shift negligible, and if so set to zero
*
         IF( SLL.GT.ZERO ) THEN
            IF( ( SHIFT / SLL )**2.LT.EPS )
     $         SHIFT = ZERO
         END IF
      END IF
*
*     Increment iteration count
*
      ITER = ITER + M - LL
*
*     If SHIFT = 0, do simplified QR iteration
*
      IF( SHIFT.EQ.ZERO ) THEN
         IF( IDIR.EQ.1 ) THEN
*
*           Chase bulge from top to bottom
*           Save cosines and sines for later singular vector updates
*
            CS = ONE
            OLDCS = ONE
            DO 120 I = LL, M - 1
               CALL DLARTG( D( I )*CS, E( I ), CS, SN, R )
               IF( I.GT.LL )
     $            E( I-1 ) = OLDSN*R
               CALL DLARTG( OLDCS*R, D( I+1 )*SN, OLDCS, OLDSN, D( I ) )
               WORK( I-LL+1 ) = CS
               WORK( I-LL+1+NM1 ) = SN
               WORK( I-LL+1+NM12 ) = OLDCS
               WORK( I-LL+1+NM13 ) = OLDSN
  120       CONTINUE
            H = D( M )*CS
            D( M ) = H*OLDCS
            E( M-1 ) = H*OLDSN
*
*           Update singular vectors
*
            IF( NCVT.GT.0 )
     $         CALL DLASR( 'L', 'V', 'F', M-LL+1, NCVT, WORK( 1 ),
     $                     WORK( N ), VT( LL, 1 ), LDVT )
            IF( NRU.GT.0 )
     $         CALL DLASR( 'R', 'V', 'F', NRU, M-LL+1, WORK( NM12+1 ),
     $                     WORK( NM13+1 ), U( 1, LL ), LDU )
            IF( NCC.GT.0 )
     $         CALL DLASR( 'L', 'V', 'F', M-LL+1, NCC, WORK( NM12+1 ),
     $                     WORK( NM13+1 ), C( LL, 1 ), LDC )
*
```

```
*           Test convergence
*
            IF( ABS( E( M-1 ) ).LE.THRESH )
     $         E( M-1 ) = ZERO
*
         ELSE
*
*           Chase bulge from bottom to top
*           Save cosines and sines for later singular vector updates
*
            CS = ONE
            OLDCS = ONE
            DO 130 I = M, LL + 1, -1
               CALL DLARTG( D( I )*CS, E( I-1 ), CS, SN, R )
               IF( I.LT.M )
     $            E( I ) = OLDSN*R
               CALL DLARTG( OLDCS*R, D( I-1 )*SN, OLDCS, OLDSN, D( I ) )
               WORK( I-LL ) = CS
               WORK( I-LL+NM1 ) = -SN
               WORK( I-LL+NM12 ) = OLDCS
               WORK( I-LL+NM13 ) = -OLDSN
  130       CONTINUE
            H = D( LL )*CS
            D( LL ) = H*OLDCS
            E( LL ) = H*OLDSN
*
*           Update singular vectors
*
            IF( NCVT.GT.0 )
     $         CALL DLASR( 'L', 'V', 'B', M-LL+1, NCVT, WORK( NM12+1 ),
     $                     WORK( NM13+1 ), VT( LL, 1 ), LDVT )
            IF( NRU.GT.0 )
     $         CALL DLASR( 'R', 'V', 'B', NRU, M-LL+1, WORK( 1 ),
     $                     WORK( N ), U( 1, LL ), LDU )
            IF( NCC.GT.0 )
     $         CALL DLASR( 'L', 'V', 'B', M-LL+1, NCC, WORK( 1 ),
     $                     WORK( N ), C( LL, 1 ), LDC )
*
*           Test convergence
*
            IF( ABS( E( LL ) ).LE.THRESH )
     $         E( LL ) = ZERO
         END IF
      ELSE
*
*        Use nonzero shift
*
         IF( IDIR.EQ.1 ) THEN
*
*           Chase bulge from top to bottom
```

```
*          Save cosines and sines for later singular vector updates
*
           F = ( ABS( D( LL ) )-SHIFT )*
     $         ( SIGN( ONE, D( LL ) )+SHIFT / D( LL ) )
           G = E( LL )
           DO 140 I = LL, M - 1
              CALL DLARTG( F, G, COSR, SINR, R )
              IF( I.GT.LL )
     $           E( I-1 ) = R
              F = COSR*D( I ) + SINR*E( I )
              E( I ) = COSR*E( I ) - SINR*D( I )
              G = SINR*D( I+1 )
              D( I+1 ) = COSR*D( I+1 )
              CALL DLARTG( F, G, COSL, SINL, R )
              D( I ) = R
              F = COSL*E( I ) + SINL*D( I+1 )
              D( I+1 ) = COSL*D( I+1 ) - SINL*E( I )
              IF( I.LT.M-1 ) THEN
                 G = SINL*E( I+1 )
                 E( I+1 ) = COSL*E( I+1 )
              END IF
              WORK( I-LL+1 ) = COSR
              WORK( I-LL+1+NM1 ) = SINR
              WORK( I-LL+1+NM12 ) = COSL
              WORK( I-LL+1+NM13 ) = SINL
  140      CONTINUE
           E( M-1 ) = F
*
*          Update singular vectors
*
           IF( NCVT.GT.0 )
     $        CALL DLASR( 'L', 'V', 'F', M-LL+1, NCVT, WORK( 1 ),
     $                    WORK( N ), VT( LL, 1 ), LDVT )
           IF( NRU.GT.0 )
     $        CALL DLASR( 'R', 'V', 'F', NRU, M-LL+1, WORK( NM12+1 ),
     $                    WORK( NM13+1 ), U( 1, LL ), LDU )
           IF( NCC.GT.0 )
     $        CALL DLASR( 'L', 'V', 'F', M-LL+1, NCC, WORK( NM12+1 ),
     $                    WORK( NM13+1 ), C( LL, 1 ), LDC )
*
*          Test convergence
*
           IF( ABS( E( M-1 ) ).LE.THRESH )
     $        E( M-1 ) = ZERO
*
        ELSE
*
*          Chase bulge from bottom to top
*          Save cosines and sines for later singular vector updates
*
```

```
            F = ( ABS( D( M ) )-SHIFT )*( SIGN( ONE, D( M ) )+SHIFT /
     $          D( M ) )
            G = E( M-1 )
            DO 150 I = M, LL + 1, -1
               CALL DLARTG( F, G, COSR, SINR, R )
               IF( I.LT.M )
     $            E( I ) = R
               F = COSR*D( I ) + SINR*E( I-1 )
               E( I-1 ) = COSR*E( I-1 ) - SINR*D( I )
               G = SINR*D( I-1 )
               D( I-1 ) = COSR*D( I-1 )
               CALL DLARTG( F, G, COSL, SINL, R )
               D( I ) = R
               F = COSL*E( I-1 ) + SINL*D( I-1 )
               D( I-1 ) = COSL*D( I-1 ) - SINL*E( I-1 )
               IF( I.GT.LL+1 ) THEN
                  G = SINL*E( I-2 )
                  E( I-2 ) = COSL*E( I-2 )
               END IF
               WORK( I-LL ) = COSR
               WORK( I-LL+NM1 ) = -SINR
               WORK( I-LL+NM12 ) = COSL
               WORK( I-LL+NM13 ) = -SINL
  150       CONTINUE
            E( LL ) = F
*
*           Test convergence
*
            IF( ABS( E( LL ) ).LE.THRESH )
     $         E( LL ) = ZERO
*
*           Update singular vectors if desired
*
            IF( NCVT.GT.0 )
     $         CALL DLASR( 'L', 'V', 'B', M-LL+1, NCVT, WORK( NM12+1 ),
     $                     WORK( NM13+1 ), VT( LL, 1 ), LDVT )
            IF( NRU.GT.0 )
     $         CALL DLASR( 'R', 'V', 'B', NRU, M-LL+1, WORK( 1 ),
     $                     WORK( N ), U( 1, LL ), LDU )
            IF( NCC.GT.0 )
     $         CALL DLASR( 'L', 'V', 'B', M-LL+1, NCC, WORK( 1 ),
     $                     WORK( N ), C( LL, 1 ), LDC )
         END IF
      END IF
*
*     QR iteration finished, go back and check convergence
*
      GO TO 60
*
*     All singular values converged, so make them positive
```

```
*
  160 CONTINUE
      DO 170 I = 1, N
         IF( D( I ).LT.ZERO ) THEN
            D( I ) = -D( I )
*
*           Change sign of singular vectors, if desired
*
            IF( NCVT.GT.0 )
     $         CALL DSCAL( NCVT, NEGONE, VT( I, 1 ), LDVT )
         END IF
  170 CONTINUE
*
*     Sort the singular values into decreasing order (insertion sort on
*     singular values, but only one transposition per singular vector)
*
      DO 190 I = 1, N - 1
*
*        Scan for smallest D(I)
*
         ISUB = 1
         SMIN = D( 1 )
         DO 180 J = 2, N + 1 - I
            IF( D( J ).LE.SMIN ) THEN
               ISUB = J
               SMIN = D( J )
            END IF
  180    CONTINUE
         IF( ISUB.NE.N+1-I ) THEN
*
*           Swap singular values and vectors
*
            D( ISUB ) = D( N+1-I )
            D( N+1-I ) = SMIN
            IF( NCVT.GT.0 )
     $         CALL DSWAP( NCVT, VT( ISUB, 1 ), LDVT, VT( N+1-I, 1 ),
     $                     LDVT )
            IF( NRU.GT.0 )
     $         CALL DSWAP( NRU, U( 1, ISUB ), 1, U( 1, N+1-I ), 1 )
            IF( NCC.GT.0 )
     $         CALL DSWAP( NCC, C( ISUB, 1 ), LDC, C( N+1-I, 1 ), LDC )
         END IF
  190 CONTINUE
      GO TO 220
*
*     Maximum number of iterations exceeded, failure to converge
*
  200 CONTINUE
      INFO = 0
      DO 210 I = 1, N - 1
```

```
         IF( E( I ).NE.ZERO )
     $      INFO = INFO + 1
  210 CONTINUE
  220 CONTINUE
      RETURN
*
*     End of DBDSQR
*
      END
      SUBROUTINE DBDSQR( UPLO, N, NCVT, NRU, NCC, D, E, VT, LDVT, U,
     $                   LDU, C, LDC, WORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1999
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDC, LDU, LDVT, N, NCC, NCVT, NRU
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   C( LDC, * ), D( * ), E( * ), U( LDU, * ),
     $                   VT( LDVT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D0 )
      DOUBLE PRECISION   NEGONE
      PARAMETER          ( NEGONE = -1.0D0 )
      DOUBLE PRECISION   HNDRTH
      PARAMETER          ( HNDRTH = 0.01D0 )
      DOUBLE PRECISION   TEN
      PARAMETER          ( TEN = 10.0D0 )
      DOUBLE PRECISION   HNDRD
      PARAMETER          ( HNDRD = 100.0D0 )
      DOUBLE PRECISION   MEIGTH
      PARAMETER          ( MEIGTH = -0.125D0 )
      INTEGER            MAXITR
      PARAMETER          ( MAXITR = 6 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LOWER, ROTATE
      INTEGER            I, IDIR, ISUB, ITER, J, LL, LLL, M, MAXIT, NM1,
     $                   NM12, NM13, OLDLL, OLDM
```

```
      DOUBLE PRECISION   ABSE, ABSS, COSL, COSR, CS, EPS, F, G, H, MU,
     $                   OLDCS, OLDSN, R, SHIFT, SIGMN, SIGMX, SINL,
     $                   SINR, SLL, SMAX, SMIN, SMINL, SMINLO, SMINOA,
     $                   SN, THRESH, TOL, TOLMUL, UNFL
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARTG, DLAS2, DLASQ1, DLASR, DLASV2, DROT,
     $                   DSCAL, DSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, MAX, MIN, SIGN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      LOWER = LSAME( UPLO, 'L' )
      IF( .NOT.LSAME( UPLO, 'U' ) .AND. .NOT.LOWER ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NCVT.LT.0 ) THEN
         INFO = -3
      ELSE IF( NRU.LT.0 ) THEN
         INFO = -4
      ELSE IF( NCC.LT.0 ) THEN
         INFO = -5
      ELSE IF( ( NCVT.EQ.0 .AND. LDVT.LT.1 ) .OR.
     $         ( NCVT.GT.0 .AND. LDVT.LT.MAX( 1, N ) ) ) THEN
         INFO = -9
      ELSE IF( LDU.LT.MAX( 1, NRU ) ) THEN
         INFO = -11
      ELSE IF( ( NCC.EQ.0 .AND. LDC.LT.1 ) .OR.
     $         ( NCC.GT.0 .AND. LDC.LT.MAX( 1, N ) ) ) THEN
         INFO = -13
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DBDSQR', -INFO )
         RETURN
      END IF
      IF( N.EQ.0 )
     $   RETURN
      IF( N.EQ.1 )
     $   GO TO 160
```

```
*
*     ROTATE is true if any singular vectors desired, false otherwise
*
      ROTATE = ( NCVT.GT.0 ) .OR. ( NRU.GT.0 ) .OR. ( NCC.GT.0 )
*
*     If no singular vectors desired, use qd algorithm
*
      IF( .NOT.ROTATE ) THEN
         CALL DLASQ1( N, D, E, WORK, INFO )
         RETURN
      END IF
*
      NM1 = N - 1
      NM12 = NM1 + NM1
      NM13 = NM12 + NM1
      IDIR = 0
*
*     Get machine constants
*
      EPS = DLAMCH( 'Epsilon' )
      UNFL = DLAMCH( 'Safe minimum' )
*
*     If matrix lower bidiagonal, rotate to be upper bidiagonal
*     by applying Givens rotations on the left
*
      IF( LOWER ) THEN
         DO 10 I = 1, N - 1
            CALL DLARTG( D( I ), E( I ), CS, SN, R )
            D( I ) = R
            E( I ) = SN*D( I+1 )
            D( I+1 ) = CS*D( I+1 )
            WORK( I ) = CS
            WORK( NM1+I ) = SN
   10    CONTINUE
*
*        Update singular vectors if desired
*
         IF( NRU.GT.0 )
     $      CALL DLASR( 'R', 'V', 'F', NRU, N, WORK( 1 ), WORK( N ), U,
     $                  LDU )
         IF( NCC.GT.0 )
     $      CALL DLASR( 'L', 'V', 'F', N, NCC, WORK( 1 ), WORK( N ), C,
     $                  LDC )
      END IF
*
*     Compute singular values to relative accuracy TOL
*     (By setting TOL to be negative, algorithm will compute
*     singular values to absolute accuracy ABS(TOL)*norm(input matrix))
*
      TOLMUL = MAX( TEN, MIN( HNDRD, EPS**MEIGTH ) )
```

```
      TOL = TOLMUL*EPS
*
*     Compute approximate maximum, minimum singular values
*
      SMAX = ZERO
      DO 20 I = 1, N
         SMAX = MAX( SMAX, ABS( D( I ) ) )
   20 CONTINUE
      DO 30 I = 1, N - 1
         SMAX = MAX( SMAX, ABS( E( I ) ) )
   30 CONTINUE
      SMINL = ZERO
      IF( TOL.GE.ZERO ) THEN
*
*        Relative accuracy desired
*
         SMINOA = ABS( D( 1 ) )
         IF( SMINOA.EQ.ZERO )
     $      GO TO 50
         MU = SMINOA
         DO 40 I = 2, N
            MU = ABS( D( I ) )*( MU / ( MU+ABS( E( I-1 ) ) ) )
            SMINOA = MIN( SMINOA, MU )
            IF( SMINOA.EQ.ZERO )
     $         GO TO 50
   40    CONTINUE
   50    CONTINUE
         SMINOA = SMINOA / SQRT( DBLE( N ) )
         THRESH = MAX( TOL*SMINOA, MAXITR*N*N*UNFL )
      ELSE
*
*        Absolute accuracy desired
*
         THRESH = MAX( ABS( TOL )*SMAX, MAXITR*N*N*UNFL )
      END IF
*
*     Prepare for main iteration loop for the singular values
*     (MAXIT is the maximum number of passes through the inner
*     loop permitted before nonconvergence signalled.)
*
      MAXIT = MAXITR*N*N
      ITER = 0
      OLDLL = -1
      OLDM = -1
*
*     M points to last element of unconverged part of matrix
*
      M = N
*
*     Begin main iteration loop
```

```
*
   60 CONTINUE
*
*     Check for convergence or exceeding iteration count
*
      IF( M.LE.1 )
     $   GO TO 160
      IF( ITER.GT.MAXIT )
     $   GO TO 200
*
*     Find diagonal block of matrix to work on
*
      IF( TOL.LT.ZERO .AND. ABS( D( M ) ).LE.THRESH )
     $   D( M ) = ZERO
      SMAX = ABS( D( M ) )
      SMIN = SMAX
      DO 70 LLL = 1, M - 1
         LL = M - LLL
         ABSS = ABS( D( LL ) )
         ABSE = ABS( E( LL ) )
         IF( TOL.LT.ZERO .AND. ABSS.LE.THRESH )
     $      D( LL ) = ZERO
         IF( ABSE.LE.THRESH )
     $      GO TO 80
         SMIN = MIN( SMIN, ABSS )
         SMAX = MAX( SMAX, ABSS, ABSE )
   70 CONTINUE
      LL = 0
      GO TO 90
   80 CONTINUE
      E( LL ) = ZERO
*
*     Matrix splits since E(LL) = 0
*
      IF( LL.EQ.M-1 ) THEN
*
*        Convergence of bottom singular value, return to top of loop
*
         M = M - 1
         GO TO 60
      END IF
   90 CONTINUE
      LL = LL + 1
*
*     E(LL) through E(M-1) are nonzero, E(LL-1) is zero
*
      IF( LL.EQ.M-1 ) THEN
*
*        2 by 2 block, handle separately
*
```

```
         CALL DLASV2( D( M-1 ), E( M-1 ), D( M ), SIGMN, SIGMX, SINR,
     $                COSR, SINL, COSL )
         D( M-1 ) = SIGMX
         E( M-1 ) = ZERO
         D( M ) = SIGMN
*
*        Compute singular vectors, if desired
*
         IF( NCVT.GT.0 )
     $      CALL DROT( NCVT, VT( M-1, 1 ), LDVT, VT( M, 1 ), LDVT, COSR,
     $                 SINR )
         IF( NRU.GT.0 )
     $      CALL DROT( NRU, U( 1, M-1 ), 1, U( 1, M ), 1, COSL, SINL )
         IF( NCC.GT.0 )
     $      CALL DROT( NCC, C( M-1, 1 ), LDC, C( M, 1 ), LDC, COSL,
     $                 SINL )
         M = M - 2
         GO TO 60
      END IF
*
*     If working on new submatrix, choose shift direction
*     (from larger end diagonal element towards smaller)
*
      IF( LL.GT.OLDM .OR. M.LT.OLDLL ) THEN
         IF( ABS( D( LL ) ).GE.ABS( D( M ) ) ) THEN
*
*           Chase bulge from top (big end) to bottom (small end)
*
            IDIR = 1
         ELSE
*
*           Chase bulge from bottom (big end) to top (small end)
*
            IDIR = 2
         END IF
      END IF
*
*     Apply convergence tests
*
      IF( IDIR.EQ.1 ) THEN
*
*        Run convergence test in forward direction
*        First apply standard test to bottom of matrix
*
         IF( ABS( E( M-1 ) ).LE.ABS( TOL )*ABS( D( M ) ) .OR.
     $       ( TOL.LT.ZERO .AND. ABS( E( M-1 ) ).LE.THRESH ) ) THEN
            E( M-1 ) = ZERO
            GO TO 60
         END IF
*
```

```
         IF( TOL.GE.ZERO ) THEN
*
*           If relative accuracy desired,
*           apply convergence criterion forward
*
            MU = ABS( D( LL ) )
            SMINL = MU
            DO 100 LLL = LL, M - 1
               IF( ABS( E( LLL ) ).LE.TOL*MU ) THEN
                  E( LLL ) = ZERO
                  GO TO 60
               END IF
               SMINLO = SMINL
               MU = ABS( D( LLL+1 ) )*( MU / ( MU+ABS( E( LLL ) ) ) )
               SMINL = MIN( SMINL, MU )
  100       CONTINUE
         END IF
*
      ELSE
*
*        Run convergence test in backward direction
*        First apply standard test to top of matrix
*
         IF( ABS( E( LL ) ).LE.ABS( TOL )*ABS( D( LL ) ) .OR.
     $       ( TOL.LT.ZERO .AND. ABS( E( LL ) ).LE.THRESH ) ) THEN
            E( LL ) = ZERO
            GO TO 60
         END IF
*
         IF( TOL.GE.ZERO ) THEN
*
*           If relative accuracy desired,
*           apply convergence criterion backward
*
            MU = ABS( D( M ) )
            SMINL = MU
            DO 110 LLL = M - 1, LL, -1
               IF( ABS( E( LLL ) ).LE.TOL*MU ) THEN
                  E( LLL ) = ZERO
                  GO TO 60
               END IF
               SMINLO = SMINL
               MU = ABS( D( LLL ) )*( MU / ( MU+ABS( E( LLL ) ) ) )
               SMINL = MIN( SMINL, MU )
  110       CONTINUE
         END IF
      END IF
      OLDLL = LL
      OLDM = M
*
```

```
*     Compute shift.  First, test if shifting would ruin relative
*     accuracy, and if so set the shift to zero.
*
      IF( TOL.GE.ZERO .AND. N*TOL*( SMINL / SMAX ).LE.
     $    MAX( EPS, HNDRTH*TOL ) ) THEN
*
*        Use a zero shift to avoid loss of relative accuracy
*
         SHIFT = ZERO
      ELSE
*
*        Compute the shift from 2-by-2 block at end of matrix
*
         IF( IDIR.EQ.1 ) THEN
            SLL = ABS( D( LL ) )
            CALL DLAS2( D( M-1 ), E( M-1 ), D( M ), SHIFT, R )
         ELSE
            SLL = ABS( D( M ) )
            CALL DLAS2( D( LL ), E( LL ), D( LL+1 ), SHIFT, R )
         END IF
*
*        Test if shift negligible, and if so set to zero
*
         IF( SLL.GT.ZERO ) THEN
            IF( ( SHIFT / SLL )**2.LT.EPS )
     $         SHIFT = ZERO
         END IF
      END IF
*
*     Increment iteration count
*
      ITER = ITER + M - LL
*
*     If SHIFT = 0, do simplified QR iteration
*
      IF( SHIFT.EQ.ZERO ) THEN
         IF( IDIR.EQ.1 ) THEN
*
*           Chase bulge from top to bottom
*           Save cosines and sines for later singular vector updates
*
            CS = ONE
            OLDCS = ONE
            DO 120 I = LL, M - 1
               CALL DLARTG( D( I )*CS, E( I ), CS, SN, R )
               IF( I.GT.LL )
     $            E( I-1 ) = OLDSN*R
               CALL DLARTG( OLDCS*R, D( I+1 )*SN, OLDCS, OLDSN, D( I ) )
               WORK( I-LL+1 ) = CS
               WORK( I-LL+1+NM1 ) = SN
```

```
               WORK( I-LL+1+NM12 ) = OLDCS
               WORK( I-LL+1+NM13 ) = OLDSN
  120       CONTINUE
            H = D( M )*CS
            D( M ) = H*OLDCS
            E( M-1 ) = H*OLDSN
*
*           Update singular vectors
*
            IF( NCVT.GT.0 )
     $         CALL DLASR( 'L', 'V', 'F', M-LL+1, NCVT, WORK( 1 ),
     $                     WORK( N ), VT( LL, 1 ), LDVT )
            IF( NRU.GT.0 )
     $         CALL DLASR( 'R', 'V', 'F', NRU, M-LL+1, WORK( NM12+1 ),
     $                     WORK( NM13+1 ), U( 1, LL ), LDU )
            IF( NCC.GT.0 )
     $         CALL DLASR( 'L', 'V', 'F', M-LL+1, NCC, WORK( NM12+1 ),
     $                     WORK( NM13+1 ), C( LL, 1 ), LDC )
*
*           Test convergence
*
            IF( ABS( E( M-1 ) ).LE.THRESH )
     $         E( M-1 ) = ZERO
*
         ELSE
*
*           Chase bulge from bottom to top
*           Save cosines and sines for later singular vector updates
*
            CS = ONE
            OLDCS = ONE
            DO 130 I = M, LL + 1, -1
               CALL DLARTG( D( I )*CS, E( I-1 ), CS, SN, R )
               IF( I.LT.M )
     $            E( I ) = OLDSN*R
               CALL DLARTG( OLDCS*R, D( I-1 )*SN, OLDCS, OLDSN, D( I ) )
               WORK( I-LL ) = CS
               WORK( I-LL+NM1 ) = -SN
               WORK( I-LL+NM12 ) = OLDCS
               WORK( I-LL+NM13 ) = -OLDSN
  130       CONTINUE
            H = D( LL )*CS
            D( LL ) = H*OLDCS
            E( LL ) = H*OLDSN
*
*           Update singular vectors
*
            IF( NCVT.GT.0 )
     $         CALL DLASR( 'L', 'V', 'B', M-LL+1, NCVT, WORK( NM12+1 ),
     $                     WORK( NM13+1 ), VT( LL, 1 ), LDVT )
```

```
            IF( NRU.GT.0 )
     $         CALL DLASR( 'R', 'V', 'B', NRU, M-LL+1, WORK( 1 ),
     $                     WORK( N ), U( 1, LL ), LDU )
            IF( NCC.GT.0 )
     $         CALL DLASR( 'L', 'V', 'B', M-LL+1, NCC, WORK( 1 ),
     $                     WORK( N ), C( LL, 1 ), LDC )
*
*           Test convergence
*
            IF( ABS( E( LL ) ).LE.THRESH )
     $         E( LL ) = ZERO
         END IF
      ELSE
*
*        Use nonzero shift
*
         IF( IDIR.EQ.1 ) THEN
*
*           Chase bulge from top to bottom
*           Save cosines and sines for later singular vector updates
*
            F = ( ABS( D( LL ) )-SHIFT )*
     $          ( SIGN( ONE, D( LL ) )+SHIFT / D( LL ) )
            G = E( LL )
            DO 140 I = LL, M - 1
               CALL DLARTG( F, G, COSR, SINR, R )
               IF( I.GT.LL )
     $            E( I-1 ) = R
               F = COSR*D( I ) + SINR*E( I )
               E( I ) = COSR*E( I ) - SINR*D( I )
               G = SINR*D( I+1 )
               D( I+1 ) = COSR*D( I+1 )
               CALL DLARTG( F, G, COSL, SINL, R )
               D( I ) = R
               F = COSL*E( I ) + SINL*D( I+1 )
               D( I+1 ) = COSL*D( I+1 ) - SINL*E( I )
               IF( I.LT.M-1 ) THEN
                  G = SINL*E( I+1 )
                  E( I+1 ) = COSL*E( I+1 )
               END IF
               WORK( I-LL+1 ) = COSR
               WORK( I-LL+1+NM1 ) = SINR
               WORK( I-LL+1+NM12 ) = COSL
               WORK( I-LL+1+NM13 ) = SINL
  140       CONTINUE
            E( M-1 ) = F
*
*           Update singular vectors
*
            IF( NCVT.GT.0 )
```

```
     $         CALL DLASR( 'L', 'V', 'F', M-LL+1, NCVT, WORK( 1 ),
     $                     WORK( N ), VT( LL, 1 ), LDVT )
           IF( NRU.GT.0 )
     $         CALL DLASR( 'R', 'V', 'F', NRU, M-LL+1, WORK( NM12+1 ),
     $                     WORK( NM13+1 ), U( 1, LL ), LDU )
           IF( NCC.GT.0 )
     $         CALL DLASR( 'L', 'V', 'F', M-LL+1, NCC, WORK( NM12+1 ),
     $                     WORK( NM13+1 ), C( LL, 1 ), LDC )
*
*          Test convergence
*
           IF( ABS( E( M-1 ) ).LE.THRESH )
     $         E( M-1 ) = ZERO
*
        ELSE
*
*          Chase bulge from bottom to top
*          Save cosines and sines for later singular vector updates
*
           F = ( ABS( D( M ) )-SHIFT )*( SIGN( ONE, D( M ) )+SHIFT /
     $         D( M ) )
           G = E( M-1 )
           DO 150 I = M, LL + 1, -1
              CALL DLARTG( F, G, COSR, SINR, R )
              IF( I.LT.M )
     $           E( I ) = R
              F = COSR*D( I ) + SINR*E( I-1 )
              E( I-1 ) = COSR*E( I-1 ) - SINR*D( I )
              G = SINR*D( I-1 )
              D( I-1 ) = COSR*D( I-1 )
              CALL DLARTG( F, G, COSL, SINL, R )
              D( I ) = R
              F = COSL*E( I-1 ) + SINL*D( I-1 )
              D( I-1 ) = COSL*D( I-1 ) - SINL*E( I-1 )
              IF( I.GT.LL+1 ) THEN
                 G = SINL*E( I-2 )
                 E( I-2 ) = COSL*E( I-2 )
              END IF
              WORK( I-LL ) = COSR
              WORK( I-LL+NM1 ) = -SINR
              WORK( I-LL+NM12 ) = COSL
              WORK( I-LL+NM13 ) = -SINL
  150      CONTINUE
           E( LL ) = F
*
*          Test convergence
*
           IF( ABS( E( LL ) ).LE.THRESH )
     $         E( LL ) = ZERO
*
```

```
*          Update singular vectors if desired
*
           IF( NCVT.GT.0 )
     $        CALL DLASR( 'L', 'V', 'B', M-LL+1, NCVT, WORK( NM12+1 ),
     $                    WORK( NM13+1 ), VT( LL, 1 ), LDVT )
           IF( NRU.GT.0 )
     $        CALL DLASR( 'R', 'V', 'B', NRU, M-LL+1, WORK( 1 ),
     $                    WORK( N ), U( 1, LL ), LDU )
           IF( NCC.GT.0 )
     $        CALL DLASR( 'L', 'V', 'B', M-LL+1, NCC, WORK( 1 ),
     $                    WORK( N ), C( LL, 1 ), LDC )
        END IF
     END IF
*
*    QR iteration finished, go back and check convergence
*
     GO TO 60
*
*    All singular values converged, so make them positive
*
  160 CONTINUE
     DO 170 I = 1, N
        IF( D( I ).LT.ZERO ) THEN
           D( I ) = -D( I )
*
*          Change sign of singular vectors, if desired
*
           IF( NCVT.GT.0 )
     $        CALL DSCAL( NCVT, NEGONE, VT( I, 1 ), LDVT )
        END IF
  170 CONTINUE
*
*    Sort the singular values into decreasing order (insertion sort on
*    singular values, but only one transposition per singular vector)
*
     DO 190 I = 1, N - 1
*
*       Scan for smallest D(I)
*
        ISUB = 1
        SMIN = D( 1 )
        DO 180 J = 2, N + 1 - I
           IF( D( J ).LE.SMIN ) THEN
              ISUB = J
              SMIN = D( J )
           END IF
  180    CONTINUE
        IF( ISUB.NE.N+1-I ) THEN
*
*          Swap singular values and vectors
```

```
*
            D( ISUB ) = D( N+1-I )
            D( N+1-I ) = SMIN
            IF( NCVT.GT.0 )
     $         CALL DSWAP( NCVT, VT( ISUB, 1 ), LDVT, VT( N+1-I, 1 ),
     $                     LDVT )
            IF( NRU.GT.0 )
     $         CALL DSWAP( NRU, U( 1, ISUB ), 1, U( 1, N+1-I ), 1 )
            IF( NCC.GT.0 )
     $         CALL DSWAP( NCC, C( ISUB, 1 ), LDC, C( N+1-I, 1 ), LDC )
         END IF
  190 CONTINUE
      GO TO 220
*
*     Maximum number of iterations exceeded, failure to converge
*
  200 CONTINUE
      INFO = 0
      DO 210 I = 1, N - 1
         IF( E( I ).NE.ZERO )
     $      INFO = INFO + 1
  210 CONTINUE
  220 CONTINUE
      RETURN
*
*     End of DBDSQR
*
      END
```

———-

90.114.4 lisp code

— **dbdsqr.lisp** —

```
(let* ((zero 0.0d0)
       (one 1.0d0)
       (negone (- 1.0d0))
       (hndrth 0.01d0)
       (ten 10.0d0)
       (hndrd 100.0d0)
       (meigth (- 0.125d0))
       (maxitr 6))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float) negone)
           (type (double-float 0.01d0 0.01d0) hndrth)
```

```
          (type (double-float 10.0d0 10.0d0) ten)
          (type (double-float 100.0d0 100.0d0) hndrd)
          (type (double-float) meigth)
          (type (f2cl-lib:integer4 6 6) maxitr)
          (ignorable zero one negone hndrth ten hndrd meigth maxitr))
 (defun dbdsqr (uplo n ncvt nru ncc d e vt ldvt u ldu c ldc work info)
   (declare (type (array double-float (*)) work c u vt e d)
            (type (f2cl-lib:integer4) info ldc ldu ldvt ncc nru ncvt n)
            (type (string *) uplo))
   (f2cl-lib:with-multi-array-data
       ((uplo character uplo-%data% uplo-%offset%)
        (d double-float d-%data% d-%offset%)
        (e double-float e-%data% e-%offset%)
        (vt double-float vt-%data% vt-%offset%)
        (u double-float u-%data% u-%offset%)
        (c double-float c-%data% c-%offset%)
        (work double-float work-%data% work-%offset%))
     (prog ((abse 0.0d0) (abss 0.0d0) (cosl 0.0d0) (cosr 0.0d0) (cs 0.0d0)
            (eps 0.0d0) (f 0.0d0) (g 0.0d0) (h 0.0d0) (mu 0.0d0) (oldcs 0.0d0)
            (oldsn 0.0d0) (r 0.0d0) (shift 0.0d0) (sigmn 0.0d0) (sigmx 0.0d0)
            (sinl 0.0d0) (sinr 0.0d0) (sll 0.0d0) (smax 0.0d0) (smin 0.0d0)
            (sminl 0.0d0) (sminoa 0.0d0) (sn 0.0d0) (thresh 0.0d0) (tol 0.0d0)
            (tolmul 0.0d0) (unfl 0.0d0) (i 0) (idir 0) (isub 0) (iter 0) (j 0)
            (ll 0) (lll 0) (m 0) (maxit 0) (nm1 0) (nm12 0) (nm13 0) (oldll 0)
            (oldm 0) (lower nil) (rotate nil))
       (declare
        (type (double-float) abse abss cosl cosr cs eps f g h mu oldcs oldsn r
         shift sigmn sigmx sinl sinr sll smax smin sminl sminoa sn thresh tol
         tolmul unfl)
        (type (f2cl-lib:integer4) i idir isub iter j ll lll m maxit nm1 nm12
         nm13 oldll oldm)
        (type f2cl-lib:logical lower rotate))
       (setf info 0)
       (setf lower
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame uplo "L")
                 (declare (ignore var-1))
                 (when var-0 (setf uplo var-0))
                 ret-val))
       (cond
        ((and
          (not
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame uplo "U")
             (declare (ignore var-1))
             (when var-0 (setf uplo var-0))
             ret-val))
          (not lower))
         (setf info -1))
        ((< n 0) (setf info -2)) ((< ncvt 0) (setf info -3))
```

```
 ((< nru 0) (setf info -4)) ((< ncc 0) (setf info -5))
 ((or (and (= ncvt 0) (< ldvt 1))
      (and (> ncvt 0)
           (< ldvt
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 n)))))
  (setf info -9))
 ((< ldu (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nru)))
  (setf info -11))
 ((or (and (= ncc 0) (< ldc 1))
      (and (> ncc 0)
           (< ldc
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 n)))))
  (setf info -13)))
(cond
 ((/= info 0) (xerbla "DBDSQR" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(if (= n 1)
    (go label160))
(setf rotate (or (> ncvt 0) (> nru 0) (> ncc 0)))
(cond
 ((not rotate)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dlasq1 n d e work info)
    (declare (ignore var-1 var-2 var-3))
    (when var-0 (setf n var-0))
    (when var-4 (setf info var-4)))
  (if (/= info 2)
      (go end_label))
  (setf info 0)))
(setf nm1 (f2cl-lib:int-sub n 1))
(setf nm12 (f2cl-lib:int-add nm1 nm1))
(setf nm13 (f2cl-lib:int-add nm12 nm1))
(setf idir 0)
(setf eps (dlamch "Epsilon"))
(setf unfl (dlamch "Safe minimum"))
(cond
 (lower
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dlartg
           (f2cl-lib:array-slice d-%data% double-float (i) ((1 *))
                                 d-%offset%)
           (f2cl-lib:array-slice e-%data% double-float (i) ((1 *))
                                 e-%offset%)
```

```
            cs sn r)
          (declare (ignore var-0 var-1))
          (when var-2 (setf cs var-2))
          (when var-3 (setf sn var-3))
          (when var-4 (setf r var-4)))
        (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) r)
        (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                (* sn
                   (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                  ((1 *)) d-%offset%)))
        (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                             d-%offset%)
                (* cs
                   (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                  ((1 *)) d-%offset%)))
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) cs)
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add nm1 i))
                             ((1 *)) work-%offset%)
                sn)
      label10))
    (if (> nru 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (dlasr "R" "V" "F" nru n
             (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                                   work-%offset%)
             (f2cl-lib:array-slice work-%data% double-float (n) ((1 *))
                                   work-%offset%)
             u ldu)
          (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
          (when var-3 (setf nru var-3))
          (when var-4 (setf n var-4))
          (when var-8 (setf ldu var-8))))
    (if (> ncc 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (dlasr "L" "V" "F" n ncc
             (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                                   work-%offset%)
             (f2cl-lib:array-slice work-%data% double-float (n) ((1 *))
                                   work-%offset%)
             c ldc)
          (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
          (when var-3 (setf n var-3))
          (when var-4 (setf ncc var-4))
          (when var-8 (setf ldc var-8))))))
  (setf tolmul (max ten (min hndrd (expt eps meigth))))
  (setf tol (* tolmul eps))
  (setf smax zero)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                ((> i n) nil)
    (tagbody
      (setf smax
              (max smax
                   (abs
                    (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))))
     label20))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (setf smax
              (max smax
                   (abs
                    (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))))
     label30))
   (setf sminl zero)
   (cond
    ((>= tol zero)
     (tagbody
      (setf sminoa (abs (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)))
      (if (= sminoa zero)
          (go label50))
      (setf mu sminoa)
      (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
       (tagbody
        (setf mu
                (*
                 (abs (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
                 (/ mu
                    (+ mu
                       (abs
                        (f2cl-lib:fref e-%data%
                                       ((f2cl-lib:int-sub i 1)) ((1 *))
                                       e-%offset%))))))
        (setf sminoa (min sminoa mu))
        (if (= sminoa zero)
            (go label50))
       label40))
     label50
      (setf sminoa (/ sminoa (f2cl-lib:fsqrt (f2cl-lib:dble n))))
      (setf thresh (max (* tol sminoa) (* maxitr n n unfl)))))
    (t (setf thresh (max (* (abs tol) smax) (* maxitr n n unfl)))))
   (setf maxit (f2cl-lib:int-mul maxitr n n))
   (setf iter 0)
   (setf oldll -1)
   (setf oldm -1)
   (setf m n)
  label60
   (if (<= m 1)
```

```
    (go label160))
 (if (> iter maxit)
    (go label200))
 (if (and (< tol zero)
          (<= (abs (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%))
              thresh))
    (setf (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%) zero))
 (setf smax (abs (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%)))
 (setf smin smax)
 (f2cl-lib:fdo (lll 1 (f2cl-lib:int-add lll 1))
               ((> lll (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
   (tagbody
     (setf ll (f2cl-lib:int-sub m lll))
     (setf abss (abs (f2cl-lib:fref d-%data% (ll) ((1 *)) d-%offset%)))
     (setf abse (abs (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%)))
     (if (and (< tol zero) (<= abss thresh))
         (setf (f2cl-lib:fref d-%data% (ll) ((1 *)) d-%offset%) zero))
     (if (<= abse thresh)
         (go label80))
     (setf smin (min smin abss))
     (setf smax (max smax abss abse))
    label70))
 (setf ll 0)
 (go label90)
label80
 (setf (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%) zero)
 (cond
  ((= ll (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
   (setf m (f2cl-lib:int-sub m 1)) (go label60)))
label90
 (setf ll (f2cl-lib:int-add ll 1))
 (cond
  ((= ll (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (dlasv2
        (f2cl-lib:array-slice d-%data% double-float
                              ((+ m (f2cl-lib:int-sub 1))) ((1 *))
                              d-%offset%)
        (f2cl-lib:array-slice e-%data% double-float
                              ((+ m (f2cl-lib:int-sub 1))) ((1 *))
                              e-%offset%)
        (f2cl-lib:array-slice d-%data% double-float (m) ((1 *))
                              d-%offset%)
        sigmn sigmx sinr cosr sinl cosl)
     (declare (ignore var-0 var-1 var-2))
     (when var-3 (setf sigmn var-3))
     (when var-4 (setf sigmx var-4))
     (when var-5 (setf sinr var-5))
     (when var-6 (setf cosr var-6))
```

```
  (when var-7 (setf sinl var-7))
  (when var-8 (setf cosl var-8)))
(setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                     d-%offset%)
        sigmx)
(setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                     e-%offset%)
        zero)
(setf (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%) sigmn)
(if (> ncvt 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (drot ncvt
         (f2cl-lib:array-slice vt-%data% double-float
                               ((+ m (f2cl-lib:int-sub 1)) 1)
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt
         (f2cl-lib:array-slice vt-%data% double-float (m 1)
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt cosr sinr)
      (declare (ignore var-1 var-3))
      (when var-0 (setf ncvt var-0))
      (when var-2 (setf ldvt var-2))
      (when var-4 (setf ldvt var-4))
      (when var-5 (setf cosr var-5))
      (when var-6 (setf sinr var-6))))
(if (> nru 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (drot nru
         (f2cl-lib:array-slice u-%data% double-float
                               (1 (f2cl-lib:int-sub m 1))
                               ((1 ldu) (1 *)) u-%offset%)
         1
         (f2cl-lib:array-slice u-%data% double-float (1 m)
                               ((1 ldu) (1 *)) u-%offset%)
         1 cosl sinl)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf nru var-0))
      (when var-5 (setf cosl var-5))
      (when var-6 (setf sinl var-6))))
(if (> ncc 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (drot ncc
         (f2cl-lib:array-slice c-%data% double-float
                               ((+ m (f2cl-lib:int-sub 1)) 1)
                               ((1 ldc) (1 *)) c-%offset%)
         ldc
         (f2cl-lib:array-slice c-%data% double-float (m 1)
                               ((1 ldc) (1 *)) c-%offset%)
         ldc cosl sinl)
      (declare (ignore var-1 var-3))
```

```
        (when var-0 (setf ncc var-0))
        (when var-2 (setf ldc var-2))
        (when var-4 (setf ldc var-4))
        (when var-5 (setf cosl var-5))
        (when var-6 (setf sinl var-6))))
  (setf m (f2cl-lib:int-sub m 2)) (go label60)))
(cond
 ((or (> ll oldm) (< m oldll))
  (cond
   ((>= (abs (f2cl-lib:fref d (ll) ((1 *))))
        (abs (f2cl-lib:fref d (m) ((1 *)))))
    (setf idir 1))
   (t (setf idir 2)))))
(cond
 ((= idir 1)
  (cond
   ((or
     (<=
      (abs
       (f2cl-lib:fref e ((f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
                      ((1 *))))
      (* (abs tol) (abs (f2cl-lib:fref d (m) ((1 *))))))
     (and (< tol zero)
          (<=
           (abs
            (f2cl-lib:fref e
                           ((f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
                           ((1 *))))
           thresh)))
    (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                         e-%offset%)
            zero)
    (go label60)))
  (cond
   ((>= tol zero)
    (setf mu (abs (f2cl-lib:fref d-%data% (ll) ((1 *)) d-%offset%)))
    (setf sminl mu)
    (f2cl-lib:fdo (lll ll (f2cl-lib:int-add lll 1))
                  ((> lll (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (cond
         ((<= (abs (f2cl-lib:fref e (lll) ((1 *)))) (* tol mu))
          (setf (f2cl-lib:fref e-%data% (lll) ((1 *)) e-%offset%) zero)
          (go label60)))
        (setf mu
                (*
                 (abs
                  (f2cl-lib:fref d-%data% ((f2cl-lib:int-add lll 1))
                                 ((1 *)) d-%offset%))
```

```
                  (/ mu
                     (+ mu
                        (abs
                         (f2cl-lib:fref e-%data% (lll) ((1 *))
                                        e-%offset%))))))
        (setf sminl (min sminl mu))
       label100)))))
 (t
  (cond
   ((or
     (<= (abs (f2cl-lib:fref e (ll) ((1 *))))
         (* (abs tol) (abs (f2cl-lib:fref d (ll) ((1 *))))))
     (and (< tol zero)
          (<= (abs (f2cl-lib:fref e (ll) ((1 *)))) thresh)))
    (setf (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%) zero)
    (go label60)))
  (cond
   ((>= tol zero)
    (setf mu (abs (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%)))
    (setf sminl mu)
    (f2cl-lib:fdo (lll (f2cl-lib:int-add m (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add lll (f2cl-lib:int-sub 1)))
                  ((> lll ll) nil)
      (tagbody
        (cond
         ((<= (abs (f2cl-lib:fref e (lll) ((1 *)))) (* tol mu))
          (setf (f2cl-lib:fref e-%data% (lll) ((1 *)) e-%offset%) zero)
          (go label60)))
        (setf mu
                (*
                 (abs
                  (f2cl-lib:fref d-%data% (lll) ((1 *)) d-%offset%))
                 (/ mu
                    (+ mu
                       (abs
                        (f2cl-lib:fref e-%data% (lll) ((1 *))
                                       e-%offset%))))))
        (setf sminl (min sminl mu))
       label110))))))
(setf oldll ll)
(setf oldm m)
(cond
 ((and (>= tol zero)
       (<= (* n tol (f2cl-lib:f2cl/ sminl smax))
           (max eps (* hndrth tol))))
  (setf shift zero))
 (t
  (cond
   ((= idir 1)
    (setf sll (abs (f2cl-lib:fref d-%data% (ll) ((1 *)) d-%offset%)))
```

```
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dlas2
         (f2cl-lib:array-slice d-%data% double-float
                               ((+ m (f2cl-lib:int-sub 1))) ((1 *))
                               d-%offset%)
         (f2cl-lib:array-slice e-%data% double-float
                               ((+ m (f2cl-lib:int-sub 1))) ((1 *))
                               e-%offset%)
         (f2cl-lib:array-slice d-%data% double-float (m) ((1 *))
                               d-%offset%)
         shift r)
      (declare (ignore var-0 var-1 var-2))
      (when var-3 (setf shift var-3))
      (when var-4 (setf r var-4))))
   (t (setf sll (abs (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dlas2
         (f2cl-lib:array-slice d-%data% double-float (ll) ((1 *))
                               d-%offset%)
         (f2cl-lib:array-slice e-%data% double-float (ll) ((1 *))
                               e-%offset%)
         (f2cl-lib:array-slice d-%data% double-float ((+ ll 1)) ((1 *))
                               d-%offset%)
         shift r)
      (declare (ignore var-0 var-1 var-2))
      (when var-3 (setf shift var-3))
      (when var-4 (setf r var-4)))))
  (cond
   ((> sll zero)
    (if (< (expt (/ shift sll) 2) eps)
        (setf shift zero))))))
(setf iter (f2cl-lib:int-sub (f2cl-lib:int-add iter m) ll))
(cond
 ((= shift zero)
  (cond
   ((= idir 1) (setf cs one) (setf oldcs one)
    (f2cl-lib:fdo (i ll (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dlartg
             (* (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) cs)
             (f2cl-lib:array-slice e-%data% double-float (i) ((1 *))
                                   e-%offset%)
             cs sn r)
          (declare (ignore var-0 var-1))
          (when var-2 (setf cs var-2))
          (when var-3 (setf sn var-3))
          (when var-4 (setf r var-4)))
        (if (> i ll)
```

```
            (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                 ((1 *)) e-%offset%)
                  (* oldsn r)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dlartg (* oldcs r)
             (*
              (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                             d-%offset%)
              sn)
             oldcs oldsn
             (f2cl-lib:array-slice d-%data% double-float (i) ((1 *))
                                   d-%offset%))
          (declare (ignore var-0 var-1 var-4))
          (when var-2 (setf oldcs var-2))
          (when var-3 (setf oldsn var-3)))
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                                1))
                             ((1 *)) work-%offset%)
                cs)
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                                1 nm1))
                             ((1 *)) work-%offset%)
                sn)
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                                1 nm12))
                             ((1 *)) work-%offset%)
                oldcs)
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                                1 nm13))
                             ((1 *)) work-%offset%)
                oldsn)
      label120))
    (setf h (* (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%) cs))
    (setf (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%) (* h oldcs))
    (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                         e-%offset%)
            (* h oldsn))
    (if (> ncvt 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (dlasr "L" "V" "F"
             (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1) ncvt
             (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                                   work-%offset%)
             (f2cl-lib:array-slice work-%data% double-float (n) ((1 *))
                                   work-%offset%)
```

```
          (f2cl-lib:array-slice vt-%data% double-float (ll 1)
                                ((1 ldvt) (1 *)) vt-%offset%)
          ldvt)
        (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
        (when var-4 (setf ncvt var-4))
        (when var-8 (setf ldvt var-8))))
  (if (> nru 0)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dlasr "R" "V" "F" nru
           (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1)
           (f2cl-lib:array-slice work-%data% double-float
                                 ((+ nm12 1)) ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% double-float
                                 ((+ nm13 1)) ((1 *)) work-%offset%)
           (f2cl-lib:array-slice u-%data% double-float (1 ll)
                                 ((1 ldu) (1 *)) u-%offset%)
           ldu)
        (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-7))
        (when var-3 (setf nru var-3))
        (when var-8 (setf ldu var-8))))
  (if (> ncc 0)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dlasr "L" "V" "F"
           (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1) ncc
           (f2cl-lib:array-slice work-%data% double-float
                                 ((+ nm12 1)) ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% double-float
                                 ((+ nm13 1)) ((1 *)) work-%offset%)
           (f2cl-lib:array-slice c-%data% double-float (ll 1)
                                 ((1 ldc) (1 *)) c-%offset%)
           ldc)
        (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
        (when var-4 (setf ncc var-4))
        (when var-8 (setf ldc var-8))))
  (if (<=
       (abs
        (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                       e-%offset%))
       thresh)
      (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                           e-%offset%)
            zero)))
 (t (setf cs one) (setf oldcs one)
  (f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                ((> i (f2cl-lib:int-add ll 1)) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dlartg
```

```
            (* (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) cs)
            (f2cl-lib:array-slice e-%data% double-float
                                  ((+ i (f2cl-lib:int-sub 1))) ((1 *))
                                  e-%offset%)
            cs sn r)
        (declare (ignore var-0 var-1))
        (when var-2 (setf cs var-2))
        (when var-3 (setf sn var-3))
        (when var-4 (setf r var-4)))
      (if (< i m)
          (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                  (* oldsn r)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dlartg (* oldcs r)
           (*
            (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub i 1)) ((1 *))
                           d-%offset%)
            sn)
           oldcs oldsn
           (f2cl-lib:array-slice d-%data% double-float (i) ((1 *))
                                 d-%offset%))
        (declare (ignore var-0 var-1 var-4))
        (when var-2 (setf oldcs var-2))
        (when var-3 (setf oldsn var-3)))
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub i ll))
                           ((1 *)) work-%offset%)
              cs)
      (setf (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                              nm1))
                           ((1 *)) work-%offset%)
              (- sn))
      (setf (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                              nm12))
                           ((1 *)) work-%offset%)
              oldcs)
      (setf (f2cl-lib:fref work-%data%
                           ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                              nm13))
                           ((1 *)) work-%offset%)
              (- oldsn))
    label130))
  (setf h (* (f2cl-lib:fref d-%data% (ll) ((1 *)) d-%offset%) cs))
  (setf (f2cl-lib:fref d-%data% (ll) ((1 *)) d-%offset%) (* h oldcs))
  (setf (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%) (* h oldsn))
  (if (> ncvt 0)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dlasr "L" "V" "B"
```

```
                (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1) ncvt
                (f2cl-lib:array-slice work-%data% double-float
                                      ((+ nm12 1)) ((1 *)) work-%offset%)
                (f2cl-lib:array-slice work-%data% double-float
                                      ((+ nm13 1)) ((1 *)) work-%offset%)
                (f2cl-lib:array-slice vt-%data% double-float (ll 1)
                                      ((1 ldvt) (1 *)) vt-%offset%)
                ldvt)
              (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
              (when var-4 (setf ncvt var-4))
              (when var-8 (setf ldvt var-8))))
        (if (> nru 0)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (dlasr "R" "V" "B" nru
                 (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1)
                 (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                                       work-%offset%)
                 (f2cl-lib:array-slice work-%data% double-float (n) ((1 *))
                                       work-%offset%)
                 (f2cl-lib:array-slice u-%data% double-float (1 ll)
                                       ((1 ldu) (1 *)) u-%offset%)
                 ldu)
              (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-7))
              (when var-3 (setf nru var-3))
              (when var-8 (setf ldu var-8))))
        (if (> ncc 0)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (dlasr "L" "V" "B"
                 (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1) ncc
                 (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                                       work-%offset%)
                 (f2cl-lib:array-slice work-%data% double-float (n) ((1 *))
                                       work-%offset%)
                 (f2cl-lib:array-slice c-%data% double-float (ll 1)
                                       ((1 ldc) (1 *)) c-%offset%)
                 ldc)
              (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
              (when var-4 (setf ncc var-4))
              (when var-8 (setf ldc var-8))))
        (if (<= (abs (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%))
                thresh)
            (setf (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%)
                  zero)))))
    (t
     (cond
      ((= idir 1)
       (setf f
               (*
```

```
          (- (abs (f2cl-lib:fref d-%data% (ll) ((1 *)) d-%offset%))
             shift)
          (+
           (f2cl-lib:sign one
                          (f2cl-lib:fref d-%data% (ll) ((1 *))
                                         d-%offset%))
           (/ shift
              (f2cl-lib:fref d-%data% (ll) ((1 *)) d-%offset%)))))
(setf g (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%))
(f2cl-lib:fdo (i ll (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add m (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dlartg f g cosr sinr r)
      (declare (ignore))
      (when var-0 (setf f var-0))
      (when var-1 (setf g var-1))
      (when var-2 (setf cosr var-2))
      (when var-3 (setf sinr var-3))
      (when var-4 (setf r var-4)))
    (if (> i ll)
        (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                             ((1 *)) e-%offset%)
              r))
    (setf f
            (+
             (* cosr
                (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
             (* sinr
                (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))))
    (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
            (-
             (* cosr
                (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))
             (* sinr
                (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))))
    (setf g
            (* sinr
               (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                              ((1 *)) d-%offset%)))
    (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         d-%offset%)
            (* cosr
               (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                              ((1 *)) d-%offset%)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dlartg f g cosl sinl r)
      (declare (ignore))
      (when var-0 (setf f var-0))
      (when var-1 (setf g var-1))
```

```
  (when var-2 (setf cosl var-2))
  (when var-3 (setf sinl var-3))
  (when var-4 (setf r var-4)))
(setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) r)
(setf f
        (+
         (* cosl
            (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))
         (* sinl
            (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                           ((1 *)) d-%offset%))))
(setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                     d-%offset%)
        (-
         (* cosl
            (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                           ((1 *)) d-%offset%))
         (* sinl
            (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))))
(cond
 ((< i (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
  (setf g
          (* sinl
             (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1))
                            ((1 *)) e-%offset%)))
  (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1))
                        ((1 *)) e-%offset%)
          (* cosl
             (f2cl-lib:fref e-%data% ((f2cl-lib:int-add i 1))
                            ((1 *)) e-%offset%)))))
(setf (f2cl-lib:fref work-%data%
                     ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                        1))
                     ((1 *)) work-%offset%)
        cosr)
(setf (f2cl-lib:fref work-%data%
                     ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                        1 nm1))
                     ((1 *)) work-%offset%)
        sinr)
(setf (f2cl-lib:fref work-%data%
                     ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                        1 nm12))
                     ((1 *)) work-%offset%)
        cosl)
(setf (f2cl-lib:fref work-%data%
                     ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                        1 nm13))
                     ((1 *)) work-%offset%)
        sinl)
```

```
   label140))
(setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                     e-%offset%)
        f)
(if (> ncvt 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dlasr "L" "V" "F"
         (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1) ncvt
         (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                               work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (n) ((1 *))
                               work-%offset%)
         (f2cl-lib:array-slice vt-%data% double-float (ll 1)
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt)
      (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
      (when var-4 (setf ncvt var-4))
      (when var-8 (setf ldvt var-8))))
(if (> nru 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dlasr "R" "V" "F" nru
         (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1)
         (f2cl-lib:array-slice work-%data% double-float
                               ((+ nm12 1)) ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float
                               ((+ nm13 1)) ((1 *)) work-%offset%)
         (f2cl-lib:array-slice u-%data% double-float (1 ll)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-7))
      (when var-3 (setf nru var-3))
      (when var-8 (setf ldu var-8))))
(if (> ncc 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dlasr "L" "V" "F"
         (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1) ncc
         (f2cl-lib:array-slice work-%data% double-float
                               ((+ nm12 1)) ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float
                               ((+ nm13 1)) ((1 *)) work-%offset%)
         (f2cl-lib:array-slice c-%data% double-float (ll 1)
                               ((1 ldc) (1 *)) c-%offset%)
         ldc)
      (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
      (when var-4 (setf ncc var-4))
      (when var-8 (setf ldc var-8))))
(if (<=
```

```
      (abs
       (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                      e-%offset%))
      thresh)
     (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                          e-%offset%)
           zero)))
(t
 (setf f
        (*
         (- (abs (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%))
            shift)
         (+
          (f2cl-lib:sign one
                          (f2cl-lib:fref d-%data% (m) ((1 *))
                                         d-%offset%))
          (/ shift
             (f2cl-lib:fref d-%data% (m) ((1 *)) d-%offset%)))))
 (setf g
        (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub m 1)) ((1 *))
                        e-%offset%))
 (f2cl-lib:fdo (i m (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
               ((> i (f2cl-lib:int-add ll 1)) nil)
   (tagbody
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (dlartg f g cosr sinr r)
       (declare (ignore))
       (when var-0 (setf f var-0))
       (when var-1 (setf g var-1))
       (when var-2 (setf cosr var-2))
       (when var-3 (setf sinr var-3))
       (when var-4 (setf r var-4)))
     (if (< i m)
         (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) r))
     (setf f
             (+
              (* cosr
                 (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
              (* sinr
                 (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                ((1 *)) e-%offset%))))
     (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1)) ((1 *))
                          e-%offset%)
             (-
              (* cosr
                 (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                ((1 *)) e-%offset%))
              (* sinr
                 (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))))
     (setf g
```

```
        (* sinr
           (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub i 1))
                          ((1 *)) d-%offset%)))
(setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub i 1)) ((1 *))
                     d-%offset%)
        (* cosr
           (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub i 1))
                          ((1 *)) d-%offset%)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dlartg f g cosl sinl r)
  (declare (ignore))
  (when var-0 (setf f var-0))
  (when var-1 (setf g var-1))
  (when var-2 (setf cosl var-2))
  (when var-3 (setf sinl var-3))
  (when var-4 (setf r var-4)))
(setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) r)
(setf f
        (+
         (* cosl
            (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                           ((1 *)) e-%offset%))
         (* sinl
            (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub i 1))
                           ((1 *)) d-%offset%))))
(setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub i 1)) ((1 *))
                     d-%offset%)
        (-
         (* cosl
            (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub i 1))
                           ((1 *)) d-%offset%))
         (* sinl
            (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                           ((1 *)) e-%offset%))))
(cond
 ((> i (f2cl-lib:int-add ll 1))
  (setf g
          (* sinl
             (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 2))
                            ((1 *)) e-%offset%)))
  (setf (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 2))
                        ((1 *)) e-%offset%)
          (* cosl
             (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 2))
                            ((1 *)) e-%offset%)))))
(setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub i ll))
                     ((1 *)) work-%offset%)
        cosr)
(setf (f2cl-lib:fref work-%data%
                     ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
```

```
                                                nm1))
                                ((1 *)) work-%offset%)
                        (- sinr))
                (setf (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                                nm12))
                                ((1 *)) work-%offset%)
                        cosl)
                (setf (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add (f2cl-lib:int-sub i ll)
                                                nm13))
                                ((1 *)) work-%offset%)
                        (- sinl))
            label150))
        (setf (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%) f)
        (if (<= (abs (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%))
                thresh)
            (setf (f2cl-lib:fref e-%data% (ll) ((1 *)) e-%offset%) zero))
        (if (> ncvt 0)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (dlasr "L" "V" "B"
                 (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1) ncvt
                 (f2cl-lib:array-slice work-%data% double-float
                                       ((+ nm12 1)) ((1 *)) work-%offset%)
                 (f2cl-lib:array-slice work-%data% double-float
                                       ((+ nm13 1)) ((1 *)) work-%offset%)
                 (f2cl-lib:array-slice vt-%data% double-float (ll 1)
                                       ((1 ldvt) (1 *)) vt-%offset%)
                 ldvt)
              (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
              (when var-4 (setf ncvt var-4))
              (when var-8 (setf ldvt var-8))))
        (if (> nru 0)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (dlasr "R" "V" "B" nru
                 (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1)
                 (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                                       work-%offset%)
                 (f2cl-lib:array-slice work-%data% double-float (n) ((1 *))
                                       work-%offset%)
                 (f2cl-lib:array-slice u-%data% double-float (1 ll)
                                       ((1 ldu) (1 *)) u-%offset%)
                 ldu)
              (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-7))
              (when var-3 (setf nru var-3))
              (when var-8 (setf ldu var-8))))
        (if (> ncc 0)
            (multiple-value-bind
```

```
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (dlasr "L" "V" "B"
               (f2cl-lib:int-add (f2cl-lib:int-sub m ll) 1) ncc
               (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                                     work-%offset%)
               (f2cl-lib:array-slice work-%data% double-float (n) ((1 *))
                                     work-%offset%)
               (f2cl-lib:array-slice c-%data% double-float (ll 1)
                                     ((1 ldc) (1 *)) c-%offset%)
               ldc)
            (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
            (when var-4 (setf ncc var-4))
            (when var-8 (setf ldc var-8))))))))
 (go label60)
label160
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (cond
      ((< (f2cl-lib:fref d (i) ((1 *))) zero)
       (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
               (- (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
       (if (> ncvt 0)
           (multiple-value-bind (var-0 var-1 var-2 var-3)
               (dscal ncvt negone
                (f2cl-lib:array-slice vt-%data% double-float (i 1)
                                      ((1 ldvt) (1 *)) vt-%offset%)
                ldvt)
             (declare (ignore var-2))
             (when var-0 (setf ncvt var-0))
             (when var-1 (setf negone var-1))
             (when var-3 (setf ldvt var-3))))))
    label170))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
   (tagbody
     (setf isub 1)
     (setf smin (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
     (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                   ((> j (f2cl-lib:int-add n 1 (f2cl-lib:int-sub i)))
                    nil)
       (tagbody
         (cond
          ((<= (f2cl-lib:fref d (j) ((1 *))) smin) (setf isub j)
           (setf smin (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%))))
        label180))
     (cond
      ((/= isub (f2cl-lib:int-add n 1 (f2cl-lib:int-sub i)))
       (setf (f2cl-lib:fref d-%data% (isub) ((1 *)) d-%offset%)
               (f2cl-lib:fref d-%data%
```

```
                        ((f2cl-lib:int-sub (f2cl-lib:int-add n 1)
                                           i))
                        ((1 *)) d-%offset%))
(setf (f2cl-lib:fref d-%data%
                     ((f2cl-lib:int-sub (f2cl-lib:int-add n 1)
                                        i))
                     ((1 *)) d-%offset%)
        smin)
(if (> ncvt 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dswap ncvt
         (f2cl-lib:array-slice vt-%data% double-float (isub 1)
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt
         (f2cl-lib:array-slice vt-%data% double-float
                               ((+ n 1 (f2cl-lib:int-sub i)) 1)
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt)
      (declare (ignore var-1 var-3))
      (when var-0 (setf ncvt var-0))
      (when var-2 (setf ldvt var-2))
      (when var-4 (setf ldvt var-4))))
(if (> nru 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dswap nru
         (f2cl-lib:array-slice u-%data% double-float (1 isub)
                               ((1 ldu) (1 *)) u-%offset%)
         1
         (f2cl-lib:array-slice u-%data% double-float
                               (1
                                (f2cl-lib:int-sub
                                  (f2cl-lib:int-add n 1) i))
                               ((1 ldu) (1 *)) u-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf nru var-0))))
(if (> ncc 0)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dswap ncc
         (f2cl-lib:array-slice c-%data% double-float (isub 1)
                               ((1 ldc) (1 *)) c-%offset%)
         ldc
         (f2cl-lib:array-slice c-%data% double-float
                               ((+ n 1 (f2cl-lib:int-sub i)) 1)
                               ((1 ldc) (1 *)) c-%offset%)
         ldc)
      (declare (ignore var-1 var-3))
      (when var-0 (setf ncc var-0))
      (when var-2 (setf ldc var-2))
      (when var-4 (setf ldc var-4))))))
```

```
          label190))
        (go label220)
       label200
        (setf info 0)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
          (tagbody
            (if (/= (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) zero)
                (setf info (f2cl-lib:int-add info 1)))
          label210))
       label220
        (go end_label)
       end_label
        (return
         (values uplo n ncvt nru ncc nil nil nil ldvt nil ldu nil ldc nil
                 info))))))
```

———-

90.115 ddisna

90.115.1 Axiom unit tests

— **ddisna.input** —

```
)set break resume
)sys rm -f ddisna.output
)spool ddisna.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.115.2 Axiom help page

— **ddisna.help** —

```
======================================================================
ddisna examples
```

```
=====================================================================

=====================================================================
Man Page Details
=====================================================================

NAME
       DDISNA  -  the  reciprocal  condition numbers for the eigenvectors of a
       real symmetric or complex Hermitian matrix or for  the  left  or  right
       singular vectors of a general m-by-n matrix

SYNOPSIS
       SUBROUTINE DDISNA( JOB, M, N, D, SEP, INFO )

           CHARACTER      JOB

           INTEGER        INFO, M, N

           DOUBLE         PRECISION D( * ), SEP( * )

PURPOSE
       DDISNA  computes  the reciprocal condition numbers for the eigenvectors
       of a real symmetric or complex Hermitian matrix  or  for  the  left  or
       right  singular vectors of a general m-by-n matrix. The reciprocal con-
       dition number is the 'gap' between the corresponding eigenvalue or sin-
       gular value and the nearest other one.

       The  bound on the error, measured by angle in radians, in the I-th com-
       puted vector is given by

              DLAMCH( 'E' ) * ( ANORM / SEP( I ) )

       where ANORM = 2-norm(A) = max( abs( D(j) ) ).  SEP(I) is not allowed to
       be  smaller  than DLAMCH( 'E' )*ANORM in order to limit the size of the
       error bound.

       DDISNA may also be used to compute error bounds for eigenvectors of the
       generalized symmetric definite eigenproblem.


ARGUMENTS
       JOB     (input) CHARACTER*1
               Specifies  for  which  problem the reciprocal condition numbers
               should be computed:
               = 'E':  the eigenvectors of a symmetric/Hermitian matrix;
               = 'L':  the left singular vectors of a general matrix;
               = 'R':  the right singular vectors of a general matrix.

       M       (input) INTEGER
               The number of rows of the matrix. M >= 0.
```

```
N       (input) INTEGER
        If JOB = 'L' or 'R', the number of columns of  the  matrix,  in
        which case N >= 0. Ignored if JOB = 'E'.

D       (input) DOUBLE PRECISION array, dimension (M) if JOB = 'E'
        dimension  (min(M,N))  if  JOB = 'L' or 'R' The eigenvalues (if
        JOB = 'E') or singular values (if JOB =  'L'  or  'R')  of  the
        matrix,  in  either increasing or decreasing order. If singular
        values, they must be non-negative.

SEP     (output) DOUBLE PRECISION array, dimension (M) if JOB = 'E'
        dimension (min(M,N)) if JOB = 'L' or 'R' The reciprocal  condi-
        tion numbers of the vectors.

INFO    (output) INTEGER
        = 0:  successful exit.
        < 0:  if INFO = -i, the i-th argument had an illegal value.
```

90.115.3 fortran code

— ddisna.f —

```
*  =====================================================================
      SUBROUTINE DDISNA( JOB, M, N, D, SEP, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOB
      INTEGER            INFO, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), SEP( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
```

```
*     .. Local Scalars ..
      LOGICAL            DECR, EIGEN, INCR, LEFT, RIGHT, SING
      INTEGER            I, K
      DOUBLE PRECISION   ANORM, EPS, NEWGAP, OLDGAP, SAFMIN, THRESH
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      EIGEN = LSAME( JOB, 'E' )
      LEFT = LSAME( JOB, 'L' )
      RIGHT = LSAME( JOB, 'R' )
      SING = LEFT .OR. RIGHT
      IF( EIGEN ) THEN
         K = M
      ELSE IF( SING ) THEN
         K = MIN( M, N )
      END IF
      IF( .NOT.EIGEN .AND. .NOT.SING ) THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( K.LT.0 ) THEN
         INFO = -3
      ELSE
         INCR = .TRUE.
         DECR = .TRUE.
         DO 10 I = 1, K - 1
            IF( INCR )
     $         INCR = INCR .AND. D( I ).LE.D( I+1 )
            IF( DECR )
     $         DECR = DECR .AND. D( I ).GE.D( I+1 )
   10    CONTINUE
         IF( SING .AND. K.GT.0 ) THEN
            IF( INCR )
     $         INCR = INCR .AND. ZERO.LE.D( 1 )
            IF( DECR )
     $         DECR = DECR .AND. D( K ).GE.ZERO
```

```
         END IF
         IF( .NOT.( INCR .OR. DECR ) )
     $      INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DDISNA', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( K.EQ.0 )
     $   RETURN
*
*     Compute reciprocal condition numbers
*
      IF( K.EQ.1 ) THEN
         SEP( 1 ) = DLAMCH( 'O' )
      ELSE
         OLDGAP = ABS( D( 2 )-D( 1 ) )
         SEP( 1 ) = OLDGAP
         DO 20 I = 2, K - 1
            NEWGAP = ABS( D( I+1 )-D( I ) )
            SEP( I ) = MIN( OLDGAP, NEWGAP )
            OLDGAP = NEWGAP
   20    CONTINUE
         SEP( K ) = OLDGAP
      END IF
      IF( SING ) THEN
         IF( ( LEFT .AND. M.GT.N ) .OR. ( RIGHT .AND. M.LT.N ) ) THEN
            IF( INCR )
     $         SEP( 1 ) = MIN( SEP( 1 ), D( 1 ) )
            IF( DECR )
     $         SEP( K ) = MIN( SEP( K ), D( K ) )
         END IF
      END IF
*
*     Ensure that reciprocal condition numbers are not less than
*     threshold, in order to limit the size of the error bound
*
      EPS = DLAMCH( 'E' )
      SAFMIN = DLAMCH( 'S' )
      ANORM = MAX( ABS( D( 1 ) ), ABS( D( K ) ) )
      IF( ANORM.EQ.ZERO ) THEN
         THRESH = EPS
      ELSE
         THRESH = MAX( EPS*ANORM, SAFMIN )
      END IF
      DO 30 I = 1, K
         SEP( I ) = MAX( SEP( I ), THRESH )
```

```
   30 CONTINUE
*
      RETURN
*
*     End of DDISNA
*
      END
      SUBROUTINE DDISNA( JOB, M, N, D, SEP, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     September 30, 1994
*
*     .. Scalar Arguments ..
      CHARACTER          JOB
      INTEGER            INFO, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), SEP( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            DECR, EIGEN, INCR, LEFT, RIGHT, SING
      INTEGER            I, K
      DOUBLE PRECISION   ANORM, EPS, NEWGAP, OLDGAP, SAFMIN, THRESH
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      EIGEN = LSAME( JOB, 'E' )
```

```
      LEFT = LSAME( JOB, 'L' )
      RIGHT = LSAME( JOB, 'R' )
      SING = LEFT .OR. RIGHT
      IF( EIGEN ) THEN
         K = M
      ELSE IF( SING ) THEN
         K = MIN( M, N )
      END IF
      IF( .NOT.EIGEN .AND. .NOT.SING ) THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( K.LT.0 ) THEN
         INFO = -3
      ELSE
         INCR = .TRUE.
         DECR = .TRUE.
         DO 10 I = 1, K - 1
            IF( INCR )
     $         INCR = INCR .AND. D( I ).LE.D( I+1 )
            IF( DECR )
     $         DECR = DECR .AND. D( I ).GE.D( I+1 )
   10    CONTINUE
         IF( SING .AND. K.GT.0 ) THEN
            IF( INCR )
     $         INCR = INCR .AND. ZERO.LE.D( 1 )
            IF( DECR )
     $         DECR = DECR .AND. D( K ).GE.ZERO
         END IF
         IF( .NOT.( INCR .OR. DECR ) )
     $      INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DDISNA', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( K.EQ.0 )
     $   RETURN
*
*     Compute reciprocal condition numbers
*
      IF( K.EQ.1 ) THEN
         SEP( 1 ) = DLAMCH( 'O' )
      ELSE
         OLDGAP = ABS( D( 2 )-D( 1 ) )
         SEP( 1 ) = OLDGAP
         DO 20 I = 2, K - 1
```

```
            NEWGAP = ABS( D( I+1 )-D( I ) )
            SEP( I ) = MIN( OLDGAP, NEWGAP )
            OLDGAP = NEWGAP
   20    CONTINUE
         SEP( K ) = OLDGAP
      END IF
      IF( SING ) THEN
         IF( ( LEFT .AND. M.GT.N ) .OR. ( RIGHT .AND. M.LT.N ) ) THEN
            IF( INCR )
     $         SEP( 1 ) = MIN( SEP( 1 ), D( 1 ) )
            IF( DECR )
     $         SEP( K ) = MIN( SEP( K ), D( K ) )
         END IF
      END IF
*
*     Ensure that reciprocal condition numbers are not less than
*     threshold, in order to limit the size of the error bound
*
      EPS = DLAMCH( 'E' )
      SAFMIN = DLAMCH( 'S' )
      ANORM = MAX( ABS( D( 1 ) ), ABS( D( K ) ) )
      IF( ANORM.EQ.ZERO ) THEN
         THRESH = EPS
      ELSE
         THRESH = MAX( EPS*ANORM, SAFMIN )
      END IF
      DO 30 I = 1, K
         SEP( I ) = MAX( SEP( I ), THRESH )
   30 CONTINUE
*
      RETURN
*
*     End of DDISNA
*
      END
```

———-

90.115.4 lisp code

— ddisna.lisp —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun ddisna (job m n d sep info)
    (declare (type (array double-float (*)) sep d)
```

```
          (type (f2cl-lib:integer4) info n m)
          (type (string *) job))
    (f2cl-lib:with-multi-array-data
        ((job character job-%data% job-%offset%)
         (d double-float d-%data% d-%offset%)
         (sep double-float sep-%data% sep-%offset%))
      (prog ((anorm 0.0d0) (eps 0.0d0) (newgap 0.0d0) (oldgap 0.0d0)
          (safmin 0.0d0) (thresh 0.0d0) (i 0) (k 0) (decr nil) (eigen nil)
          (incr nil) (left nil) (right nil) (sing nil))
        (declare (type (double-float) anorm eps newgap oldgap safmin thresh)
                 (type (f2cl-lib:integer4) i k)
                 (type f2cl-lib:logical decr eigen incr left right sing))
        (setf info 0)
        (setf eigen
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame job "E")
                  (declare (ignore var-1))
                  (when var-0 (setf job var-0))
                  ret-val))
        (setf left
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame job "L")
                  (declare (ignore var-1))
                  (when var-0 (setf job var-0))
                  ret-val))
        (setf right
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame job "R")
                  (declare (ignore var-1))
                  (when var-0 (setf job var-0))
                  ret-val))
        (setf sing (or left right))
        (cond (eigen (setf k m))
              (sing
               (setf k
                       (min (the f2cl-lib:integer4 m)
                            (the f2cl-lib:integer4 n)))))
        (cond ((and (not eigen) (not sing)) (setf info -1))
              ((< m 0) (setf info -2)) ((< k 0) (setf info -3))
              (t (setf incr f2cl-lib:%true%) (setf decr f2cl-lib:%true%)
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                              nil)
                 (tagbody
                   (if incr
                       (setf incr
                               (and incr
                                    (<=
                                     (f2cl-lib:fref d-%data% (i) ((1 *))
                                                    d-%offset%)
```

```
                                (f2cl-lib:fref d-%data%
                                               ((f2cl-lib:int-add i 1))
                                               ((1 *)) d-%offset%)))))
                (if decr
                    (setf decr
                            (and decr
                                 (>=
                                 (f2cl-lib:fref d-%data% (i) ((1 *))
                                               d-%offset%)
                                 (f2cl-lib:fref d-%data%
                                               ((f2cl-lib:int-add i 1))
                                               ((1 *)) d-%offset%)))))
                  label10))
               (cond
                ((and sing (> k 0))
                 (if incr
                     (setf incr
                             (and incr
                                  (<= zero
                                      (f2cl-lib:fref d-%data% (1) ((1 *))
                                                     d-%offset%)))))
                 (if decr
                     (setf decr
                             (and decr
                                  (>=
                                   (f2cl-lib:fref d-%data% (k) ((1 *))
                                                  d-%offset%)
                                   zero))))))
               (if (not (or incr decr))
                   (setf info -4))))
        (cond
         ((/= info 0) (xerbla "DDISNA" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (= k 0)
            (go end_label))
        (cond
         ((= k 1)
          (setf (f2cl-lib:fref sep-%data% (1) ((1 *)) sep-%offset%)
                  (dlamch "O")))
         (t
          (setf oldgap
                  (abs
                   (- (f2cl-lib:fref d-%data% (2) ((1 *)) d-%offset%)
                      (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))))
          (setf (f2cl-lib:fref sep-%data% (1) ((1 *)) sep-%offset%) oldgap)
          (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                        ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub 1))) nil)
            (tagbody
              (setf newgap
                      (abs
```

```
                (-
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                ((1 *)) d-%offset%)
                 (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))))
       (setf (f2cl-lib:fref sep-%data% (i) ((1 *)) sep-%offset%)
               (min oldgap newgap))
       (setf oldgap newgap)
      label20))
   (setf (f2cl-lib:fref sep-%data% (k) ((1 *)) sep-%offset%) oldgap)))
  (cond
   (sing
    (cond
     ((or (and left (> m n)) (and right (< m n)))
      (if incr
          (setf (f2cl-lib:fref sep-%data% (1) ((1 *)) sep-%offset%)
                  (min
                   (f2cl-lib:fref sep-%data% (1) ((1 *)) sep-%offset%)
                   (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))))
      (if decr
          (setf (f2cl-lib:fref sep-%data% (k) ((1 *)) sep-%offset%)
                  (min
                   (f2cl-lib:fref sep-%data% (k) ((1 *)) sep-%offset%)
                   (f2cl-lib:fref d-%data% (k) ((1 *)) d-%offset%))))))))
  (setf eps (dlamch "E"))
  (setf safmin (dlamch "S"))
  (setf anorm
          (max (abs (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
               (abs (f2cl-lib:fref d-%data% (k) ((1 *)) d-%offset%))))
  (cond ((= anorm zero) (setf thresh eps))
        (t (setf thresh (max (* eps anorm) safmin))))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i k) nil)
    (tagbody
      (setf (f2cl-lib:fref sep-%data% (i) ((1 *)) sep-%offset%)
              (max (f2cl-lib:fref sep-%data% (i) ((1 *)) sep-%offset%)
                   thresh))
     label30))
  (go end_label)
 end_label
  (return (values job nil nil nil nil info))))))
```

———-

90.116 dgebak

90.116.1 Axiom unit tests

— dgebak.input —

```
)set break resume
)sys rm -f dgebak.output
)spool dgebak.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.116.2 Axiom help page

— dgebak.help —

```
======================================================================
dgebak examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DGEBAK  -  the  right  or left eigenvectors of a real general matrix by
       backward transformation on the computed eigenvectors  of  the  balanced
       matrix output by DGEBAL

SYNOPSIS
       SUBROUTINE DGEBAK( JOB, SIDE, N, ILO, IHI, SCALE, M, V, LDV, INFO )

           CHARACTER      JOB, SIDE

           INTEGER        IHI, ILO, INFO, LDV, M, N

           DOUBLE         PRECISION SCALE( * ), V( LDV, * )

PURPOSE
       DGEBAK forms the right or left eigenvectors of a real general matrix by
```

```
      backward transformation on the computed eigenvectors  of  the  balanced
      matrix output by DGEBAL.


ARGUMENTS
      JOB     (input) CHARACTER*1
              Specifies  the type of backward transformation required: = 'N',
              do nothing, return immediately; = 'P', do backward  transforma-
              tion  for  permutation  only; = 'S', do backward transformation
              for scaling only; = 'B', do backward transformations  for  both
              permutation  and scaling.  JOB must be the same as the argument
              JOB supplied to DGEBAL.

      SIDE    (input) CHARACTER*1
              = 'R':  V contains right eigenvectors;
              = 'L':  V contains left eigenvectors.

      N       (input) INTEGER
              The number of rows of the matrix V.  N >= 0.

      ILO     (input) INTEGER
              IHI     (input) INTEGER The integers ILO and IHI determined  by
              DGEBAL.   1  <=  ILO <= IHI <= N, if N > 0; ILO=1 and IHI=0, if
              N=0.

      SCALE   (input) DOUBLE PRECISION array, dimension (N)
              Details of the permutation and scaling factors, as returned  by
              DGEBAL.

      M       (input) INTEGER
              The number of columns of the matrix V.  M >= 0.

      V       (input/output) DOUBLE PRECISION array, dimension (LDV,M)
              On entry, the matrix of right or left eigenvectors to be trans-
              formed, as returned by DHSEIN or DTREVC.  On exit, V  is  over-
              written by the transformed eigenvectors.

      LDV     (input) INTEGER
              The leading dimension of the array V. LDV >= max(1,N).

      INFO    (output) INTEGER
              = 0:  successful exit
              < 0:  if INFO = -i, the i-th argument had an illegal value.
```

90.116.3 fortran code

— dgebak.f —

```
*  =====================================================================
      SUBROUTINE DGEBAK( JOB, SIDE, N, ILO, IHI, SCALE, M, V, LDV,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOB, SIDE
      INTEGER            IHI, ILO, INFO, LDV, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   SCALE( * ), V( LDV, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LEFTV, RIGHTV
      INTEGER            I, II, K
      DOUBLE PRECISION   S
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DSCAL, DSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Decode and Test the input parameters
*
      RIGHTV = LSAME( SIDE, 'R' )
      LEFTV = LSAME( SIDE, 'L' )
*
```

```
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.RIGHTV .AND. .NOT.LEFTV ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -5
      ELSE IF( M.LT.0 ) THEN
         INFO = -7
      ELSE IF( LDV.LT.MAX( 1, N ) ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEBAK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
      IF( M.EQ.0 )
     $   RETURN
      IF( LSAME( JOB, 'N' ) )
     $   RETURN
*
      IF( ILO.EQ.IHI )
     $   GO TO 30
*
*     Backward balance
*
      IF( LSAME( JOB, 'S' ) .OR. LSAME( JOB, 'B' ) ) THEN
*
         IF( RIGHTV ) THEN
            DO 10 I = ILO, IHI
               S = SCALE( I )
               CALL DSCAL( M, S, V( I, 1 ), LDV )
   10       CONTINUE
         END IF
*
         IF( LEFTV ) THEN
            DO 20 I = ILO, IHI
               S = ONE / SCALE( I )
               CALL DSCAL( M, S, V( I, 1 ), LDV )
   20       CONTINUE
```

```
         END IF
*
      END IF
*
*     Backward permutation
*
*     For  I = ILO-1 step -1 until 1,
*              IHI+1 step 1 until N do --
*
   30 CONTINUE
      IF( LSAME( JOB, 'P' ) .OR. LSAME( JOB, 'B' ) ) THEN
         IF( RIGHTV ) THEN
            DO 40 II = 1, N
               I = II
               IF( I.GE.ILO .AND. I.LE.IHI )
     $            GO TO 40
               IF( I.LT.ILO )
     $            I = ILO - II
               K = SCALE( I )
               IF( K.EQ.I )
     $            GO TO 40
               CALL DSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   40       CONTINUE
         END IF
*
         IF( LEFTV ) THEN
            DO 50 II = 1, N
               I = II
               IF( I.GE.ILO .AND. I.LE.IHI )
     $            GO TO 50
               IF( I.LT.ILO )
     $            I = ILO - II
               K = SCALE( I )
               IF( K.EQ.I )
     $            GO TO 50
               CALL DSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   50       CONTINUE
         END IF
      END IF
*
      RETURN
*
*     End of DGEBAK
*
      END
      SUBROUTINE DGEBAK( JOB, SIDE, N, ILO, IHI, SCALE, M, V, LDV,
     $                   INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
```

```
*     Courant Institute, Argonne National Lab, and Rice University
*     September 30, 1994
*
*     .. Scalar Arguments ..
      CHARACTER          JOB, SIDE
      INTEGER            IHI, ILO, INFO, LDV, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   SCALE( * ), V( LDV, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LEFTV, RIGHTV
      INTEGER            I, II, K
      DOUBLE PRECISION   S
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DSCAL, DSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Decode and Test the input parameters
*
      RIGHTV = LSAME( SIDE, 'R' )
      LEFTV = LSAME( SIDE, 'L' )
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.RIGHTV .AND. .NOT.LEFTV ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
```

```
         INFO = -5
      ELSE IF( M.LT.0 ) THEN
         INFO = -7
      ELSE IF( LDV.LT.MAX( 1, N ) ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEBAK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
      IF( M.EQ.0 )
     $   RETURN
      IF( LSAME( JOB, 'N' ) )
     $   RETURN
*
      IF( ILO.EQ.IHI )
     $   GO TO 30
*
*     Backward balance
*
      IF( LSAME( JOB, 'S' ) .OR. LSAME( JOB, 'B' ) ) THEN
*
         IF( RIGHTV ) THEN
            DO 10 I = ILO, IHI
               S = SCALE( I )
               CALL DSCAL( M, S, V( I, 1 ), LDV )
   10       CONTINUE
         END IF
*
         IF( LEFTV ) THEN
            DO 20 I = ILO, IHI
               S = ONE / SCALE( I )
               CALL DSCAL( M, S, V( I, 1 ), LDV )
   20       CONTINUE
         END IF
*
      END IF
*
*     Backward permutation
*
*     For  I = ILO-1 step -1 until 1,
*              IHI+1 step 1 until N do --
*
   30 CONTINUE
      IF( LSAME( JOB, 'P' ) .OR. LSAME( JOB, 'B' ) ) THEN
```

```
         IF( RIGHTV ) THEN
            DO 40 II = 1, N
               I = II
               IF( I.GE.ILO .AND. I.LE.IHI )
     $            GO TO 40
               IF( I.LT.ILO )
     $            I = ILO - II
               K = SCALE( I )
               IF( K.EQ.I )
     $            GO TO 40
               CALL DSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   40       CONTINUE
         END IF
*
         IF( LEFTV ) THEN
            DO 50 II = 1, N
               I = II
               IF( I.GE.ILO .AND. I.LE.IHI )
     $            GO TO 50
               IF( I.LT.ILO )
     $            I = ILO - II
               K = SCALE( I )
               IF( K.EQ.I )
     $            GO TO 50
               CALL DSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   50       CONTINUE
         END IF
      END IF
*
      RETURN
*
*     End of DGEBAK
*
      END
```

———-

90.116.4 lisp code

— dgebak.lisp —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dgebak (job side n ilo ihi scale m v ldv info)
    (declare (type (array double-float (*)) v scale)
             (type (f2cl-lib:integer4) info ldv m ihi ilo n)
```

```
          (type (string *) side job))
    (f2cl-lib:with-multi-array-data
        ((job character job-%data% job-%offset%)
         (side character side-%data% side-%offset%)
         (scale double-float scale-%data% scale-%offset%)
         (v double-float v-%data% v-%offset%))
      (prog ((s 0.0d0) (i 0) (ii 0) (k 0) (leftv nil) (rightv nil))
        (declare (type (double-float) s)
                 (type (f2cl-lib:integer4) i ii k)
                 (type f2cl-lib:logical leftv rightv))
        (setf rightv
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame side "R")
                  (declare (ignore var-1))
                  (when var-0 (setf side var-0))
                  ret-val))
        (setf leftv
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame side "L")
                  (declare (ignore var-1))
                  (when var-0 (setf side var-0))
                  ret-val))
        (setf info 0)
        (cond
         ((and
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame job "N")
              (declare (ignore var-1))
              (when var-0 (setf job var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame job "P")
              (declare (ignore var-1))
              (when var-0 (setf job var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame job "S")
              (declare (ignore var-1))
              (when var-0 (setf job var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame job "B")
              (declare (ignore var-1))
              (when var-0 (setf job var-0))
              ret-val)))
          (setf info -1))
```

```
 ((and (not rightv) (not leftv)) (setf info -2))
 ((< n 0) (setf info -3))
 ((or (< ilo 1)
      (> ilo
         (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n))))
  (setf info -4))
 ((or
   (< ihi (min (the f2cl-lib:integer4 ilo) (the f2cl-lib:integer4 n)))
   (> ihi n))
  (setf info -5))
 ((< m 0) (setf info -7))
 ((< ldv (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9)))
(cond
 ((/= info 0) (xerbla "DGEBAK" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(if (= m 0)
    (go end_label))
(if (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "N")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)
    (go end_label))
(if (= ilo ihi)
    (go label30))
(cond
 ((or
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame job "S")
     (declare (ignore var-1))
     (when var-0 (setf job var-0))
     ret-val)
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame job "B")
     (declare (ignore var-1))
     (when var-0 (setf job var-0))
     ret-val))
  (cond
   (rightv
    (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                  ((> i ihi) nil)
      (tagbody
        (setf s
                (f2cl-lib:fref scale-%data% (i) ((1 *))
                               scale-%offset%))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal m s
```

```
                (f2cl-lib:array-slice v-%data% double-float (i 1)
                                      ((1 ldv) (1 *)) v-%offset%)
                ldv)
            (declare (ignore var-2))
            (when var-0 (setf m var-0))
            (when var-1 (setf s var-1))
            (when var-3 (setf ldv var-3)))
        label10))))
    (cond
     (leftv
      (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                    ((> i ihi) nil)
        (tagbody
          (setf s
                  (/ one
                     (f2cl-lib:fref scale-%data% (i) ((1 *))
                                    scale-%offset%)))
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (dscal m s
                (f2cl-lib:array-slice v-%data% double-float (i 1)
                                      ((1 ldv) (1 *)) v-%offset%)
                ldv)
            (declare (ignore var-2))
            (when var-0 (setf m var-0))
            (when var-1 (setf s var-1))
            (when var-3 (setf ldv var-3)))
        label20))))))
label30
 (cond
  ((or
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "P")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "B")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (cond
    (rightv
     (f2cl-lib:fdo (ii 1 (f2cl-lib:int-add ii 1))
                   ((> ii n) nil)
       (tagbody
         (setf i ii)
         (if (and (>= i ilo) (<= i ihi))
             (go label40))
         (if (< i ilo)
             (setf i (f2cl-lib:int-sub ilo ii)))
```

```
          (setf k
                  (f2cl-lib:int
                   (f2cl-lib:fref scale-%data% (i) ((1 *))
                                  scale-%offset%)))
          (if (= k i)
              (go label40))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dswap m
               (f2cl-lib:array-slice v-%data% double-float (i 1)
                                     ((1 ldv) (1 *)) v-%offset%)
               ldv
               (f2cl-lib:array-slice v-%data% double-float (k 1)
                                     ((1 ldv) (1 *)) v-%offset%)
               ldv)
            (declare (ignore var-1 var-3))
            (when var-0 (setf m var-0))
            (when var-2 (setf ldv var-2))
            (when var-4 (setf ldv var-4)))
        label40))))
   (cond
    (leftv
     (f2cl-lib:fdo (ii 1 (f2cl-lib:int-add ii 1))
                   ((> ii n) nil)
       (tagbody
         (setf i ii)
         (if (and (>= i ilo) (<= i ihi))
             (go label50))
         (if (< i ilo)
             (setf i (f2cl-lib:int-sub ilo ii)))
         (setf k
                 (f2cl-lib:int
                  (f2cl-lib:fref scale-%data% (i) ((1 *))
                                 scale-%offset%)))
         (if (= k i)
             (go label50))
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (dswap m
              (f2cl-lib:array-slice v-%data% double-float (i 1)
                                    ((1 ldv) (1 *)) v-%offset%)
              ldv
              (f2cl-lib:array-slice v-%data% double-float (k 1)
                                    ((1 ldv) (1 *)) v-%offset%)
              ldv)
           (declare (ignore var-1 var-3))
           (when var-0 (setf m var-0))
           (when var-2 (setf ldv var-2))
           (when var-4 (setf ldv var-4)))
        label50))))))
 (go end_label)
end_label
```

```
        (return (values job side nil nil nil nil m nil ldv info))))))
```

———-

90.117 dgebal

90.117.1 Axiom unit tests

— **dgebal.input** —

```
)set break resume
)sys rm -f dgebal.output
)spool dgebal.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.117.2 Axiom help page

— **dgebal.help** —

```
======================================================================
dgebal examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DGEBAL - a general real matrix A

SYNOPSIS
       SUBROUTINE DGEBAL( JOB, N, A, LDA, ILO, IHI, SCALE, INFO )

           CHARACTER      JOB

           INTEGER        IHI, ILO, INFO, LDA, N
```

```
      DOUBLE         PRECISION A( LDA, * ), SCALE( * )
```

PURPOSE

DGEBAL balances a general real matrix A. This involves, first, permuting A by a similarity transformation to isolate eigenvalues in the first 1 to ILO-1 and last IHI+1 to N elements on the diagonal; and second, applying a diagonal similarity transformation to rows and columns ILO to IHI to make the rows and columns as close in norm as possible. Both steps are optional.

Balancing may reduce the 1-norm of the matrix, and improve the accuracy of the computed eigenvalues and/or eigenvectors.

ARGUMENTS

JOB (input) CHARACTER*1
Specifies the operations to be performed on A:
= 'N': none: simply set ILO = 1, IHI = N, SCALE(I) = 1.0 for i = 1,...,N; = 'P': permute only;
= 'S': scale only;
= 'B': both permute and scale.

N (input) INTEGER
The order of the matrix A. N >= 0.

A (input/output) DOUBLE PRECISION array, dimension (LDA,N)
On entry, the input matrix A. On exit, A is overwritten by the balanced matrix. If JOB = 'N', A is not referenced. See Further Details. LDA (input) INTEGER The leading dimension of the array A. LDA >= max(1,N).

ILO (output) INTEGER
IHI (output) INTEGER ILO and IHI are set to integers such that on exit A(i,j) = 0 if i > j and j = 1,...,ILO-1 or I = IHI+1,...,N. If JOB = 'N' or 'S', ILO = 1 and IHI = N.

SCALE (output) DOUBLE PRECISION array, dimension (N)
Details of the permutations and scaling factors applied to A. If P(j) is the index of the row and column interchanged with row and column j and D(j) is the scaling factor applied to row and column j, then SCALE(j) = P(j) for j = 1,...,ILO-1 = D(j) for j = ILO,...,IHI = P(j) for j = IHI+1,...,N. The order in which the interchanges are made is N to IHI+1, then 1 to ILO-1.

INFO (output) INTEGER
= 0: successful exit.
< 0: if INFO = -i, the i-th argument had an illegal value.

```
FURTHER DETAILS
       The  permutations  consist of row and column interchanges which put the
       matrix in the form

                 ( T1   X   Y  )
          P A P = (  0   B   Z  )
                 (  0   0   T2 )

       where T1 and T2 are upper triangular  matrices  whose  eigenvalues  lie
       along  the  diagonal.  The column indices ILO and IHI mark the starting
       and ending columns of the submatrix B. Balancing consists of applying a
       diagonal  similarity  transformation inv(D) * B * D to make the 1-norms
       of each row of B and its corresponding column nearly equal.  The output
       matrix is

          ( T1     X*D          Y    )
          (  0  inv(D)*B*D  inv(D)*Z ).
          (  0      0           T2   )

       Information  about  the  permutations  P  and  the diagonal matrix D is
       returned in the vector SCALE.

       This subroutine is based on the EISPACK routine BALANC.

  Modified by Tzu-Yi Chen, Computer Science Division, University of
    California at Berkeley, USA
```

———-

90.117.3 fortran code

— dgebal.f —

```
*  =====================================================================
      SUBROUTINE DGEBAL( JOB, N, A, LDA, ILO, IHI, SCALE, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOB
      INTEGER            IHI, ILO, INFO, LDA, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), SCALE( * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   SCLFAC
      PARAMETER          ( SCLFAC = 2.0D+0 )
      DOUBLE PRECISION   FACTOR
      PARAMETER          ( FACTOR = 0.95D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOCONV
      INTEGER            I, ICA, IEXC, IRA, J, K, L, M
      DOUBLE PRECISION   C, CA, F, G, R, RA, S, SFMAX1, SFMAX2, SFMIN1,
     $                   SFMIN2
*     ..
*     .. External Functions ..
      LOGICAL            DISNAN, LSAME
      INTEGER            IDAMAX
      DOUBLE PRECISION   DLAMCH, DNRM2
      EXTERNAL           DISNAN, LSAME, IDAMAX, DLAMCH, DNRM2
*     ..
*     .. External Subroutines ..
      EXTERNAL           DSCAL, DSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     Test the input parameters
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEBAL', -INFO )
         RETURN
      END IF
*
      K = 1
      L = N
*
      IF( N.EQ.0 )
```

```
     $   GO TO 210
*
      IF( LSAME( JOB, 'N' ) ) THEN
         DO 10 I = 1, N
            SCALE( I ) = ONE
   10    CONTINUE
         GO TO 210
      END IF
*
      IF( LSAME( JOB, 'S' ) )
     $   GO TO 120
*
*     Permutation to isolate eigenvalues if possible
*
      GO TO 50
*
*     Row and column exchange.
*
   20 CONTINUE
      SCALE( M ) = J
      IF( J.EQ.M )
     $   GO TO 30
*
      CALL DSWAP( L, A( 1, J ), 1, A( 1, M ), 1 )
      CALL DSWAP( N-K+1, A( J, K ), LDA, A( M, K ), LDA )
*
   30 CONTINUE
      GO TO ( 40, 80 )IEXC
*
*     Search for rows isolating an eigenvalue and push them down.
*
   40 CONTINUE
      IF( L.EQ.1 )
     $   GO TO 210
      L = L - 1
*
   50 CONTINUE
      DO 70 J = L, 1, -1
*
         DO 60 I = 1, L
            IF( I.EQ.J )
     $         GO TO 60
            IF( A( J, I ).NE.ZERO )
     $         GO TO 70
   60    CONTINUE
*
         M = L
         IEXC = 1
         GO TO 20
   70 CONTINUE
```

```
*
      GO TO 90
*
*     Search for columns isolating an eigenvalue and push them left.
*
   80 CONTINUE
      K = K + 1
*
   90 CONTINUE
      DO 110 J = K, L
*
         DO 100 I = K, L
            IF( I.EQ.J )
     $         GO TO 100
            IF( A( I, J ).NE.ZERO )
     $         GO TO 110
  100    CONTINUE
*
         M = K
         IEXC = 2
         GO TO 20
  110 CONTINUE
*
  120 CONTINUE
      DO 130 I = K, L
         SCALE( I ) = ONE
  130 CONTINUE
*
      IF( LSAME( JOB, 'P' ) )
     $   GO TO 210
*
*     Balance the submatrix in rows K to L.
*
*     Iterative loop for norm reduction
*
      SFMIN1 = DLAMCH( 'S' ) / DLAMCH( 'P' )
      SFMAX1 = ONE / SFMIN1
      SFMIN2 = SFMIN1*SCLFAC
      SFMAX2 = ONE / SFMIN2
*
  140 CONTINUE
      NOCONV = .FALSE.
*
      DO 200 I = K, L
*
         C = DNRM2( L-K+1, A( K, I ), 1 )
         R = DNRM2( L-K+1, A( I, K ), LDA )
         ICA = IDAMAX( L, A( 1, I ), 1 )
         CA = ABS( A( ICA, I ) )
         IRA = IDAMAX( N-K+1, A( I, K ), LDA )
```

```
         RA = ABS( A( I, IRA+K-1 ) )
*
*        Guard against zero C or R due to underflow.
*
         IF( C.EQ.ZERO .OR. R.EQ.ZERO )
     $      GO TO 200
         G = R / SCLFAC
         F = ONE
         S = C + R
  160    CONTINUE
         IF( C.GE.G .OR. MAX( F, C, CA ).GE.SFMAX2 .OR.
     $       MIN( R, G, RA ).LE.SFMIN2 )GO TO 170
            IF( DISNAN( C+F+CA+R+G+RA ) ) THEN
*
*           Exit if NaN to avoid infinite loop
*
            INFO = -3
            CALL XERBLA( 'DGEBAL', -INFO )
            RETURN
         END IF
         F = F*SCLFAC
         C = C*SCLFAC
         CA = CA*SCLFAC
         R = R / SCLFAC
         G = G / SCLFAC
         RA = RA / SCLFAC
         GO TO 160
*
  170    CONTINUE
         G = C / SCLFAC
  180    CONTINUE
         IF( G.LT.R .OR. MAX( R, RA ).GE.SFMAX2 .OR.
     $       MIN( F, C, G, CA ).LE.SFMIN2 )GO TO 190
         F = F / SCLFAC
         C = C / SCLFAC
         G = G / SCLFAC
         CA = CA / SCLFAC
         R = R*SCLFAC
         RA = RA*SCLFAC
         GO TO 180
*
*        Now balance.
*
  190    CONTINUE
         IF( ( C+R ).GE.FACTOR*S )
     $      GO TO 200
         IF( F.LT.ONE .AND. SCALE( I ).LT.ONE ) THEN
            IF( F*SCALE( I ).LE.SFMIN1 )
     $         GO TO 200
         END IF
```

```
         IF( F.GT.ONE .AND. SCALE( I ).GT.ONE ) THEN
            IF( SCALE( I ).GE.SFMAX1 / F )
     $         GO TO 200
         END IF
         G = ONE / F
         SCALE( I ) = SCALE( I )*F
         NOCONV = .TRUE.
*
         CALL DSCAL( N-K+1, G, A( I, K ), LDA )
         CALL DSCAL( L, F, A( 1, I ), 1 )
*
  200 CONTINUE
*
      IF( NOCONV )
     $   GO TO 140
*
  210 CONTINUE
      ILO = K
      IHI = L
*
      RETURN
*
*     End of DGEBAL
*
      END
      SUBROUTINE DGEBAL( JOB, N, A, LDA, ILO, IHI, SCALE, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      CHARACTER          JOB
      INTEGER            IHI, ILO, INFO, LDA, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), SCALE( * )
*     ..
*
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   SCLFAC
      PARAMETER          ( SCLFAC = 0.8D+1 )
      DOUBLE PRECISION   FACTOR
      PARAMETER          ( FACTOR = 0.95D+0 )
```

```
*     ..
*     .. Local Scalars ..
      LOGICAL            NOCONV
      INTEGER            I, ICA, IEXC, IRA, J, K, L, M
      DOUBLE PRECISION   C, CA, F, G, R, RA, S, SFMAX1, SFMAX2, SFMIN1,
     $                   SFMIN2
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, IDAMAX, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DSCAL, DSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEBAL', -INFO )
         RETURN
      END IF
*
      K = 1
      L = N
*
      IF( N.EQ.0 )
     $   GO TO 210
*
      IF( LSAME( JOB, 'N' ) ) THEN
         DO 10 I = 1, N
            SCALE( I ) = ONE
   10    CONTINUE
         GO TO 210
      END IF
*
```

```
      IF( LSAME( JOB, 'S' ) )
     $   GO TO 120
*
*     Permutation to isolate eigenvalues if possible
*
      GO TO 50
*
*     Row and column exchange.
*
   20 CONTINUE
      SCALE( M ) = J
      IF( J.EQ.M )
     $   GO TO 30
*
      CALL DSWAP( L, A( 1, J ), 1, A( 1, M ), 1 )
      CALL DSWAP( N-K+1, A( J, K ), LDA, A( M, K ), LDA )
*
   30 CONTINUE
      GO TO ( 40, 80 )IEXC
*
*     Search for rows isolating an eigenvalue and push them down.
*
   40 CONTINUE
      IF( L.EQ.1 )
     $   GO TO 210
      L = L - 1
*
   50 CONTINUE
      DO 70 J = L, 1, -1
*
         DO 60 I = 1, L
            IF( I.EQ.J )
     $         GO TO 60
            IF( A( J, I ).NE.ZERO )
     $         GO TO 70
   60    CONTINUE
*
         M = L
         IEXC = 1
         GO TO 20
   70 CONTINUE
*
      GO TO 90
*
*     Search for columns isolating an eigenvalue and push them left.
*
   80 CONTINUE
      K = K + 1
*
   90 CONTINUE
```

```
      DO 110 J = K, L
*
         DO 100 I = K, L
            IF( I.EQ.J )
     $         GO TO 100
            IF( A( I, J ).NE.ZERO )
     $         GO TO 110
  100    CONTINUE
*
         M = K
         IEXC = 2
         GO TO 20
  110 CONTINUE
*
  120 CONTINUE
      DO 130 I = K, L
         SCALE( I ) = ONE
  130 CONTINUE
*
      IF( LSAME( JOB, 'P' ) )
     $   GO TO 210
*
*     Balance the submatrix in rows K to L.
*
*     Iterative loop for norm reduction
*
      SFMIN1 = DLAMCH( 'S' ) / DLAMCH( 'P' )
      SFMAX1 = ONE / SFMIN1
      SFMIN2 = SFMIN1*SCLFAC
      SFMAX2 = ONE / SFMIN2
  140 CONTINUE
      NOCONV = .FALSE.
*
      DO 200 I = K, L
         C = ZERO
         R = ZERO
*
         DO 150 J = K, L
            IF( J.EQ.I )
     $         GO TO 150
            C = C + ABS( A( J, I ) )
            R = R + ABS( A( I, J ) )
  150    CONTINUE
         ICA = IDAMAX( L, A( 1, I ), 1 )
         CA = ABS( A( ICA, I ) )
         IRA = IDAMAX( N-K+1, A( I, K ), LDA )
         RA = ABS( A( I, IRA+K-1 ) )
*
*        Guard against zero C or R due to underflow.
*
```

```
         IF( C.EQ.ZERO .OR. R.EQ.ZERO )
     $      GO TO 200
         G = R / SCLFAC
         F = ONE
         S = C + R
  160    CONTINUE
         IF( C.GE.G .OR. MAX( F, C, CA ).GE.SFMAX2 .OR.
     $       MIN( R, G, RA ).LE.SFMIN2 )GO TO 170
         F = F*SCLFAC
         C = C*SCLFAC
         CA = CA*SCLFAC
         R = R / SCLFAC
         G = G / SCLFAC
         RA = RA / SCLFAC
         GO TO 160
*
  170    CONTINUE
         G = C / SCLFAC
  180    CONTINUE
         IF( G.LT.R .OR. MAX( R, RA ).GE.SFMAX2 .OR.
     $       MIN( F, C, G, CA ).LE.SFMIN2 )GO TO 190
         F = F / SCLFAC
         C = C / SCLFAC
         G = G / SCLFAC
         CA = CA / SCLFAC
         R = R*SCLFAC
         RA = RA*SCLFAC
         GO TO 180
*
*        Now balance.
*
  190    CONTINUE
         IF( ( C+R ).GE.FACTOR*S )
     $      GO TO 200
         IF( F.LT.ONE .AND. SCALE( I ).LT.ONE ) THEN
            IF( F*SCALE( I ).LE.SFMIN1 )
     $         GO TO 200
         END IF
         IF( F.GT.ONE .AND. SCALE( I ).GT.ONE ) THEN
            IF( SCALE( I ).GE.SFMAX1 / F )
     $         GO TO 200
         END IF
         G = ONE / F
         SCALE( I ) = SCALE( I )*F
         NOCONV = .TRUE.
*
         CALL DSCAL( N-K+1, G, A( I, K ), LDA )
         CALL DSCAL( L, F, A( 1, I ), 1 )
*
  200 CONTINUE
```

```
*
      IF( NOCONV )
     $   GO TO 140
*
  210 CONTINUE
      ILO = K
      IHI = L
*
      RETURN
*
*     End of DGEBAL
*
      END
```

———-

90.117.4 lisp code

— **dgebal.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0) (sclfac 2.0d0) (factor 0.95d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) sclfac)
           (type (double-float 0.95d0 0.95d0) factor)
           (ignorable zero one sclfac factor))
  (defun dgebal (job n a lda ilo ihi scale info)
    (declare (type (array double-float (*)) scale a)
             (type (f2cl-lib:integer4) info ihi ilo lda n)
             (type (string *) job))
    (f2cl-lib:with-multi-array-data
        ((job character job-%data% job-%offset%)
         (a double-float a-%data% a-%offset%)
         (scale double-float scale-%data% scale-%offset%))
      (prog ((c 0.0d0) (ca 0.0d0) (f 0.0d0) (g 0.0d0) (r 0.0d0) (ra 0.0d0)
             (s 0.0d0) (sfmax1 0.0d0) (sfmax2 0.0d0) (sfmin1 0.0d0)
             (sfmin2 0.0d0) (i 0) (ica 0) (iexc 0) (ira 0) (j 0) (k 0) (l 0)
             (m 0) (noconv nil))
        (declare
         (type (double-float) c ca f g r ra s sfmax1 sfmax2 sfmin1 sfmin2)
         (type (f2cl-lib:integer4) i ica iexc ira j k l m)
         (type f2cl-lib:logical noconv))
        (setf info 0)
        (cond
         ((and
           (not
            (multiple-value-bind (ret-val var-0 var-1)
```

```
          (lsame job "N")
        (declare (ignore var-1))
        (when var-0 (setf job var-0))
        ret-val))
     (not
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame job "P")
        (declare (ignore var-1))
        (when var-0 (setf job var-0))
        ret-val))
     (not
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame job "S")
        (declare (ignore var-1))
        (when var-0 (setf job var-0))
        ret-val))
     (not
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame job "B")
        (declare (ignore var-1))
        (when var-0 (setf job var-0))
        ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4)))
  (cond
   ((/= info 0) (xerbla "DGEBAL" (f2cl-lib:int-sub info))
    (go end_label)))
  (setf k 1)
  (setf l n)
  (if (= n 0)
      (go label210))
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame job "N")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%)
                one)
       label10))
    (go label210)))
  (if (multiple-value-bind (ret-val var-0 var-1)
          (lsame job "S")
        (declare (ignore var-1))
        (when var-0 (setf job var-0))
```

```
      ret-val)
    (go label120))
 (go label50)
label20
 (setf (f2cl-lib:fref scale-%data% (m) ((1 *)) scale-%offset%)
        (coerce (the f2cl-lib:integer4 j) 'double-float))
 (if (= j m)
    (go label30))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dswap l
     (f2cl-lib:array-slice a-%data% double-float (1 j) ((1 lda) (1 *))
                           a-%offset%)
     1
     (f2cl-lib:array-slice a-%data% double-float (1 m) ((1 lda) (1 *))
                           a-%offset%)
     1)
   (declare (ignore var-1 var-2 var-3 var-4))
   (when var-0 (setf l var-0)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dswap (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
     (f2cl-lib:array-slice a-%data% double-float (j k) ((1 lda) (1 *))
                           a-%offset%)
     lda
     (f2cl-lib:array-slice a-%data% double-float (m k) ((1 lda) (1 *))
                           a-%offset%)
     lda)
   (declare (ignore var-0 var-1 var-3))
   (when var-2 (setf lda var-2))
   (when var-4 (setf lda var-4)))
label30
 (f2cl-lib:computed-goto (label40 label80) iexc)
label40
 (if (= l 1)
    (go label210))
 (setf l (f2cl-lib:int-sub l 1))
label50
 (f2cl-lib:fdo (j l (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i l) nil)
       (tagbody
         (if (= i j)
             (go label60))
         (if (/=
              (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *)) a-%offset%)
              zero)
             (go label70))
        label60))
     (setf m l)
```

```
      (setf iexc 1)
      (go label20)
     label70))
  (go label90)
 label80
  (setf k (f2cl-lib:int-add k 1))
 label90
  (f2cl-lib:fdo (j k (f2cl-lib:int-add j 1))
                ((> j l) nil)
    (tagbody
      (f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
                    ((> i l) nil)
        (tagbody
          (if (= i j)
              (go label100))
          (if (/=
               (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
               zero)
              (go label110))
         label100))
      (setf m k)
      (setf iexc 2)
      (go label20)
     label110))
 label120
  (f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
                ((> i l) nil)
    (tagbody
      (setf (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%) one)
     label130))
  (if (multiple-value-bind (ret-val var-0 var-1)
          (lsame job "P")
        (declare (ignore var-1))
        (when var-0 (setf job var-0))
        ret-val)
      (go label210))
  (setf sfmin1 (/ (dlamch "S") (dlamch "P")))
  (setf sfmax1 (/ one sfmin1))
  (setf sfmin2 (* sfmin1 sclfac))
  (setf sfmax2 (/ one sfmin2))
 label140
  (setf noconv f2cl-lib:%false%)
  (f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
                ((> i l) nil)
    (tagbody
      (setf c
              (dnrm2 (f2cl-lib:int-add (f2cl-lib:int-sub l k) 1)
               (f2cl-lib:array-slice a-%data% double-float (k i)
                                     ((1 lda) (1 *)) a-%offset%)
               1))
```

```
 (setf r
         (multiple-value-bind (ret-val var-0 var-1 var-2)
             (dnrm2 (f2cl-lib:int-add (f2cl-lib:int-sub l k) 1)
              (f2cl-lib:array-slice a-%data% double-float (i k)
                                    ((1 lda) (1 *)) a-%offset%)
              lda)
           (declare (ignore var-0 var-1))
           (when var-2 (setf lda var-2))
           ret-val))
 (setf ica
         (multiple-value-bind (ret-val var-0 var-1 var-2)
             (idamax l
              (f2cl-lib:array-slice a-%data% double-float (1 i)
                                    ((1 lda) (1 *)) a-%offset%)
              1)
           (declare (ignore var-1 var-2))
           (when var-0 (setf l var-0))
           ret-val))
 (setf ca
         (abs
          (f2cl-lib:fref a-%data% (ica i) ((1 lda) (1 *))
                         a-%offset%)))
 (setf ira
         (multiple-value-bind (ret-val var-0 var-1 var-2)
             (idamax (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
              (f2cl-lib:array-slice a-%data% double-float (i k)
                                    ((1 lda) (1 *)) a-%offset%)
              lda)
           (declare (ignore var-0 var-1))
           (when var-2 (setf lda var-2))
           ret-val))
 (setf ra
         (abs
          (f2cl-lib:fref a-%data%
                         (i
                          (f2cl-lib:int-sub (f2cl-lib:int-add ira k)
                                            1))
                         ((1 lda) (1 *)) a-%offset%)))
 (if (or (= c zero) (= r zero))
     (go label200))
 (setf g (/ r sclfac))
 (setf f one)
 (setf s (+ c r))
label160
 (if (or (>= c g) (>= (max f c ca) sfmax2) (<= (min r g ra) sfmin2))
     (go label170))
 (cond
  ((disnan (+ c f ca r g ra)) (setf info -3)
   (xerbla "DGEBAL" (f2cl-lib:int-sub info)) (go end_label)))
 (setf f (* f sclfac))
```

```
 (setf c (* c sclfac))
 (setf ca (* ca sclfac))
 (setf r (/ r sclfac))
 (setf g (/ g sclfac))
 (setf ra (/ ra sclfac))
 (go label160)
label170
 (setf g (/ c sclfac))
label180
 (if (or (< g r) (>= (max r ra) sfmax2) (<= (min f c g ca) sfmin2))
     (go label190))
 (setf f (/ f sclfac))
 (setf c (/ c sclfac))
 (setf g (/ g sclfac))
 (setf ca (/ ca sclfac))
 (setf r (* r sclfac))
 (setf ra (* ra sclfac))
 (go label180)
label190
 (if (>= (+ c r) (* factor s))
     (go label200))
 (cond
  ((and (< f one) (< (f2cl-lib:fref scale (i) ((1 *))) one))
   (if (<=
        (* f
           (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%))
        sfmin1)
       (go label200))))
 (cond
  ((and (> f one) (> (f2cl-lib:fref scale (i) ((1 *))) one))
   (if (>= (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%)
           (/ sfmax1 f))
       (go label200))))
 (setf g (/ one f))
 (setf (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%)
         (* (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%)
            f))
 (setf noconv f2cl-lib:%true%)
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (dscal (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1) g
      (f2cl-lib:array-slice a-%data% double-float (i k)
                            ((1 lda) (1 *)) a-%offset%)
      lda)
   (declare (ignore var-0 var-2))
   (when var-1 (setf g var-1))
   (when var-3 (setf lda var-3)))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (dscal l f
      (f2cl-lib:array-slice a-%data% double-float (1 i)
                            ((1 lda) (1 *)) a-%offset%)
```

```
                1)
              (declare (ignore var-2 var-3))
              (when var-0 (setf l var-0))
              (when var-1 (setf f var-1)))
           label200))
        (if noconv
            (go label140))
       label210
        (setf ilo k)
        (setf ihi l)
        (go end_label)
       end_label
        (return (values job nil nil lda ilo ihi nil info))))))
```

———-

90.118 dgebd2

90.118.1 Axiom unit tests

— **dgebd2.input** —

```
)set break resume
)sys rm -f dgebd2.output
)spool dgebd2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.118.2 Axiom help page

— **dgebd2.help** —

```
=====================================================================
dgebd2 examples
=====================================================================

=====================================================================
```

```
Man Page Details
=====================================================================

NAME
       DGEBD2  -  a  real general m by n matrix A to upper or lower bidiagonal
       form B by an orthogonal transformation

SYNOPSIS
       SUBROUTINE DGEBD2( M, N, A, LDA, D, E, TAUQ, TAUP, WORK, INFO )

           INTEGER        INFO, LDA, M, N

           DOUBLE         PRECISION A( LDA, * ), D( * ), E( * ),  TAUP(  *  ),
                          TAUQ( * ), WORK( * )

PURPOSE
       DGEBD2 reduces a real general m by n matrix A to upper or lower bidiag-
       onal form B by an orthogonal transformation: Q' * A * P = B.

       If m >= n, B is upper bidiagonal; if m < n, B is lower bidiagonal.


ARGUMENTS
       M       (input) INTEGER
               The number of rows in the matrix A.  M >= 0.

       N       (input) INTEGER
               The number of columns in the matrix A.  N >= 0.

       A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
               On entry, the m by n general matrix to be reduced.  On exit, if
               m  >= n, the diagonal and the first superdiagonal are overwrit-
               ten with the upper bidiagonal matrix B; the elements below  the
               diagonal,  with the array TAUQ, represent the orthogonal matrix
               Q as a product of elementary reflectors, and the elements above
               the  first  superdiagonal,  with  the array TAUP, represent the
               orthogonal matrix P as a product of elementary reflectors; if m
               <  n,  the  diagonal  and the first subdiagonal are overwritten
               with the lower bidiagonal matrix  B;  the  elements  below  the
               first  subdiagonal, with the array TAUQ, represent the orthogo-
               nal matrix Q as a product of  elementary  reflectors,  and  the
               elements above the diagonal, with the array TAUP, represent the
               orthogonal matrix P as a product of elementary reflectors.  See
               Further Details.  LDA     (input) INTEGER The leading dimension
               of the array A.  LDA >= max(1,M).

       D       (output) DOUBLE PRECISION array, dimension (min(M,N))
               The diagonal elements  of  the  bidiagonal  matrix  B:  D(i)  =
               A(i,i).
```

```
E       (output) DOUBLE PRECISION array, dimension (min(M,N)-1)
        The  off-diagonal  elements of the bidiagonal matrix B: if m >=
        n, E(i) = A(i,i+1) for i =  1,2,...,n-1;  if  m  <  n,  E(i)  =
        A(i+1,i) for i = 1,2,...,m-1.

TAUQ    (output) DOUBLE PRECISION array dimension (min(M,N))
        The scalar factors of the elementary reflectors which represent
        the orthogonal matrix Q. See Further Details.  TAUP    (output)
        DOUBLE PRECISION array, dimension (min(M,N)) The scalar factors
        of the elementary reflectors  which  represent  the  orthogonal
        matrix P. See Further Details.  WORK    (workspace) DOUBLE PRE-
        CISION array, dimension (max(M,N))

INFO    (output) INTEGER
        = 0: successful exit.
        < 0: if INFO = -i, the i-th argument had an illegal value.

FURTHER DETAILS
       The matrices Q and P are represented as products of elementary  reflec-
       tors:

       If m >= n,

          Q = H(1) H(2) . . . H(n)  and  P = G(1) G(2) . . . G(n-1)

       Each H(i) and G(i) has the form:

          H(i) = I - tauq * v * v'  and G(i) = I - taup * u * u'

       where  tauq  and  taup  are real scalars, and v and u are real vectors;
       v(1:i-1) = 0, v(i) = 1, and v(i+1:m) is stored on exit  in  A(i+1:m,i);
       u(1:i)  =  0, u(i+1) = 1, and u(i+2:n) is stored on exit in A(i,i+2:n);
       tauq is stored in TAUQ(i) and taup in TAUP(i).

       If m < n,

          Q = H(1) H(2) . . . H(m-1)  and  P = G(1) G(2) . . . G(m)

       Each H(i) and G(i) has the form:

          H(i) = I - tauq * v * v'  and G(i) = I - taup * u * u'

       where tauq and taup are real scalars, and v and  u  are  real  vectors;
       v(1:i)  =  0, v(i+1) = 1, and v(i+2:m) is stored on exit in A(i+2:m,i);
       u(1:i-1) = 0, u(i) = 1, and u(i+1:n) is stored on exit  in  A(i,i+1:n);
       tauq is stored in TAUQ(i) and taup in TAUP(i).

       The contents of A on exit are illustrated by the following examples:

       m = 6 and n = 5 (m > n):          m = 5 and n = 6 (m < n):
```

```
( d   e   u1  u1  u1 )          ( d   u1  u1  u1  u1  u1 )
( v1  d   e   u2  u2 )          ( e   d   u2  u2  u2  u2 )
( v1  v2  d   e   u3 )          ( v1  e   d   u3  u3  u3 )
( v1  v2  v3  d   e  )          ( v1  v2  e   d   u4  u4 )
( v1  v2  v3  v4  d  )          ( v1  v2  v3  e   d   u5 )
( v1  v2  v3  v4  v5 )
```

where d and e denote diagonal and off-diagonal elements of B, vi denotes an element of the vector defining H(i), and ui an element of the vector defining G(i).

———-

90.118.3 fortran code

— dgebd2.f —

```
*  =====================================================================
      SUBROUTINE DGEBD2( M, N, A, LDA, D, E, TAUQ, TAUP, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), D( * ), E( * ), TAUP( * ),
     $                   TAUQ( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARF, DLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.LT.0 ) THEN
         CALL XERBLA( 'DGEBD2', -INFO )
         RETURN
      END IF
*
      IF( M.GE.N ) THEN
*
*        Reduce to upper bidiagonal form
*
         DO 10 I = 1, N
*
*           Generate elementary reflector H(i) to annihilate A(i+1:m,i)
*
            CALL DLARFG( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
     $                   TAUQ( I ) )
            D( I ) = A( I, I )
            A( I, I ) = ONE
*
*           Apply H(i) to A(i:m,i+1:n) from the left
*
            IF( I.LT.N )
     $         CALL DLARF( 'Left', M-I+1, N-I, A( I, I ), 1, TAUQ( I ),
     $                     A( I, I+1 ), LDA, WORK )
            A( I, I ) = D( I )
*
            IF( I.LT.N ) THEN
*
*              Generate elementary reflector G(i) to annihilate
*              A(i,i+2:n)
*
               CALL DLARFG( N-I, A( I, I+1 ), A( I, MIN( I+2, N ) ),
     $                      LDA, TAUP( I ) )
               E( I ) = A( I, I+1 )
               A( I, I+1 ) = ONE
*
*              Apply G(i) to A(i+1:m,i+1:n) from the right
```

```
*
               CALL DLARF( 'Right', M-I, N-I, A( I, I+1 ), LDA,
     $                     TAUP( I ), A( I+1, I+1 ), LDA, WORK )
               A( I, I+1 ) = E( I )
            ELSE
               TAUP( I ) = ZERO
            END IF
   10    CONTINUE
      ELSE
*
*        Reduce to lower bidiagonal form
*
         DO 20 I = 1, M
*
*           Generate elementary reflector G(i) to annihilate A(i,i+1:n)
*
            CALL DLARFG( N-I+1, A( I, I ), A( I, MIN( I+1, N ) ), LDA,
     $                   TAUP( I ) )
            D( I ) = A( I, I )
            A( I, I ) = ONE
*
*           Apply G(i) to A(i+1:m,i:n) from the right
*
            IF( I.LT.M )
     $         CALL DLARF( 'Right', M-I, N-I+1, A( I, I ), LDA,
     $                     TAUP( I ), A( I+1, I ), LDA, WORK )
            A( I, I ) = D( I )
*
            IF( I.LT.M ) THEN
*
               Generate elementary reflector H(i) to annihilate
*              A(i+2:m,i)
*
               CALL DLARFG( M-I, A( I+1, I ), A( MIN( I+2, M ), I ), 1,
     $                      TAUQ( I ) )
               E( I ) = A( I+1, I )
               A( I+1, I ) = ONE
*
*              Apply H(i) to A(i+1:m,i+1:n) from the left
*
               CALL DLARF( 'Left', M-I, N-I, A( I+1, I ), 1, TAUQ( I ),
     $                     A( I+1, I+1 ), LDA, WORK )
               A( I+1, I ) = E( I )
            ELSE
               TAUQ( I ) = ZERO
            END IF
   20    CONTINUE
      END IF
      RETURN
*
```

```
*     End of DGEBD2
*
      END
      SUBROUTINE DGEBD2( M, N, A, LDA, D, E, TAUQ, TAUP, WORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     February 29, 1992
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), D( * ), E( * ), TAUP( * ),
     $                   TAUQ( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARF, DLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.LT.0 ) THEN
         CALL XERBLA( 'DGEBD2', -INFO )
         RETURN
      END IF
*
```

```
      IF( M.GE.N ) THEN
*
*        Reduce to upper bidiagonal form
*
         DO 10 I = 1, N
*
*           Generate elementary reflector H(i) to annihilate A(i+1:m,i)
*
            CALL DLARFG( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
     $                   TAUQ( I ) )
            D( I ) = A( I, I )
            A( I, I ) = ONE
*
*           Apply H(i) to A(i:m,i+1:n) from the left
*
            CALL DLARF( 'Left', M-I+1, N-I, A( I, I ), 1, TAUQ( I ),
     $                  A( I, I+1 ), LDA, WORK )
            A( I, I ) = D( I )
*
            IF( I.LT.N ) THEN
*
*              Generate elementary reflector G(i) to annihilate
*              A(i,i+2:n)
*
               CALL DLARFG( N-I, A( I, I+1 ), A( I, MIN( I+2, N ) ),
     $                       LDA, TAUP( I ) )
               E( I ) = A( I, I+1 )
               A( I, I+1 ) = ONE
*
*              Apply G(i) to A(i+1:m,i+1:n) from the right
*
               CALL DLARF( 'Right', M-I, N-I, A( I, I+1 ), LDA,
     $                     TAUP( I ), A( I+1, I+1 ), LDA, WORK )
               A( I, I+1 ) = E( I )
            ELSE
               TAUP( I ) = ZERO
            END IF
   10    CONTINUE
      ELSE
*
*        Reduce to lower bidiagonal form
*
         DO 20 I = 1, M
*
*           Generate elementary reflector G(i) to annihilate A(i,i+1:n)
*
            CALL DLARFG( N-I+1, A( I, I ), A( I, MIN( I+1, N ) ), LDA,
     $                   TAUP( I ) )
            D( I ) = A( I, I )
            A( I, I ) = ONE
```

```
*
*          Apply G(i) to A(i+1:m,i:n) from the right
*
           CALL DLARF( 'Right', M-I, N-I+1, A( I, I ), LDA, TAUP( I ),
     $                 A( MIN( I+1, M ), I ), LDA, WORK )
           A( I, I ) = D( I )
*
           IF( I.LT.M ) THEN
*
*             Generate elementary reflector H(i) to annihilate
*             A(i+2:m,i)
*
              CALL DLARFG( M-I, A( I+1, I ), A( MIN( I+2, M ), I ), 1,
     $                     TAUQ( I ) )
              E( I ) = A( I+1, I )
              A( I+1, I ) = ONE
*
*             Apply H(i) to A(i+1:m,i+1:n) from the left
*
              CALL DLARF( 'Left', M-I, N-I, A( I+1, I ), 1, TAUQ( I ),
     $                    A( I+1, I+1 ), LDA, WORK )
              A( I+1, I ) = E( I )
           ELSE
              TAUQ( I ) = ZERO
           END IF
   20    CONTINUE
      END IF
      RETURN
*
*     End of DGEBD2
*
      END
```

———-

90.118.4 lisp code

— dgebd2.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dgebd2 (m n a lda d e tauq taup work info)
    (declare (type (array double-float (*)) work taup tauq e d a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
```

```
    ((a double-float a-%data% a-%offset%)
     (d double-float d-%data% d-%offset%)
     (e double-float e-%data% e-%offset%)
     (tauq double-float tauq-%data% tauq-%offset%)
     (taup double-float taup-%data% taup-%offset%)
     (work double-float work-%data% work-%offset%))
  (prog ((i 0))
    (declare (type (f2cl-lib:integer4) i))
    (setf info 0)
    (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
          ((< lda
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
           (setf info -4)))
    (cond
     ((< info 0) (xerbla "DGEBD2" (f2cl-lib:int-sub info)) (go end_label)))
    (cond
     ((>= m n)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (dlarfg (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
           (f2cl-lib:array-slice a-%data% double-float (i i)
                                 ((1 lda) (1 *)) a-%offset%)
           (f2cl-lib:array-slice a-%data% double-float
                                 ((min (f2cl-lib:int-add i 1) m) i)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice tauq-%data% double-float (i) ((1 *))
                                 tauq-%offset%))
          (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                  (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                                 a-%offset%))
          (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                  one)
          (if (< i n)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                  (dlarf "Left" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
                   (f2cl-lib:int-sub n i)
                   (f2cl-lib:array-slice a-%data% double-float (i i)
                                         ((1 lda) (1 *)) a-%offset%)
                   1
                   (f2cl-lib:array-slice tauq-%data% double-float (i)
                                         ((1 *)) tauq-%offset%)
                   (f2cl-lib:array-slice a-%data% double-float
                                         (i (f2cl-lib:int-add i 1))
                                         ((1 lda) (1 *)) a-%offset%)
                   lda work)
                (declare
                 (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
```

```
      (when var-7 (setf lda var-7))))
(setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
        (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
(cond
 ((< i n)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dlarfg (f2cl-lib:int-sub n i)
       (f2cl-lib:array-slice a-%data% double-float
                             (i (f2cl-lib:int-add i 1))
                             ((1 lda) (1 *)) a-%offset%)
       (f2cl-lib:array-slice a-%data% double-float
                             (i
                              (min
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add i 2))
                               (the f2cl-lib:integer4 n)))
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice taup-%data% double-float (i) ((1 *))
                             taup-%offset%))
    (declare (ignore var-0 var-1 var-2 var-4))
    (when var-3 (setf lda var-3)))
  (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
          (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
                         ((1 lda) (1 *)) a-%offset%))
  (setf (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
                        ((1 lda) (1 *)) a-%offset%)
          one)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dlarf "Right" (f2cl-lib:int-sub m i)
       (f2cl-lib:int-sub n i)
       (f2cl-lib:array-slice a-%data% double-float
                             (i (f2cl-lib:int-add i 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice taup-%data% double-float (i) ((1 *))
                             taup-%offset%)
       (f2cl-lib:array-slice a-%data% double-float
                             ((+ i 1) (f2cl-lib:int-add i 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda work)
    (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8))
    (when var-4 (setf lda var-4))
    (when var-7 (setf lda var-7)))
  (setf (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
                        ((1 lda) (1 *)) a-%offset%)
          (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
 (t
  (setf (f2cl-lib:fref taup-%data% (i) ((1 *)) taup-%offset%)
```

```
                zero)))
    label10)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dlarfg (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
           (f2cl-lib:array-slice a-%data% double-float (i i)
                                 ((1 lda) (1 *)) a-%offset%)
           (f2cl-lib:array-slice a-%data% double-float
                                 (i
                                  (min
                                   (the f2cl-lib:integer4
                                        (f2cl-lib:int-add i 1))
                                   (the f2cl-lib:integer4 n)))
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice taup-%data% double-float (i) ((1 *))
                                 taup-%offset%))
        (declare (ignore var-0 var-1 var-2 var-4))
        (when var-3 (setf lda var-3)))
      (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
              (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                             a-%offset%))
      (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
              one)
      (if (< i m)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (dlarf "Right" (f2cl-lib:int-sub m i)
               (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
               (f2cl-lib:array-slice a-%data% double-float (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice taup-%data% double-float (i)
                                     ((1 *)) taup-%offset%)
               (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                                     ((1 lda) (1 *)) a-%offset%)
               lda work)
            (declare
             (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8))
            (when var-4 (setf lda var-4))
            (when var-7 (setf lda var-7))))
      (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
              (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
      (cond
       ((< i m)
        (dlarfg (f2cl-lib:int-sub m i)
         (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
```

```
                                ((1 lda) (1 *)) a-%offset%)
             (f2cl-lib:array-slice a-%data% double-float
                                ((min (f2cl-lib:int-add i 2) m) i)
                                ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice tauq-%data% double-float (i) ((1 *))
                                tauq-%offset%))
            (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                    (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                                ((1 lda) (1 *)) a-%offset%))
            (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                              ((1 lda) (1 *)) a-%offset%)
                    one)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (dlarf "Left" (f2cl-lib:int-sub m i) (f2cl-lib:int-sub n i)
                 (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                                      ((1 lda) (1 *)) a-%offset%)
                 1
                 (f2cl-lib:array-slice tauq-%data% double-float (i) ((1 *))
                                      tauq-%offset%)
                 (f2cl-lib:array-slice a-%data% double-float
                                      ((+ i 1) (f2cl-lib:int-add i 1))
                                      ((1 lda) (1 *)) a-%offset%)
                 lda work)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
              (when var-7 (setf lda var-7)))
            (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                              ((1 lda) (1 *)) a-%offset%)
                    (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
           (t
            (setf (f2cl-lib:fref tauq-%data% (i) ((1 *)) tauq-%offset%)
                    zero)))
         label20))))
  (go end_label)
 end_label
  (return (values nil nil nil lda nil nil nil nil nil info))))))
```

———-

90.119 dgebrd

90.119.1 Axiom unit tests

— **dgebrd.input** —

```
)set break resume
)sys rm -f dgebrd.output
)spool dgebrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.119.2 Axiom help page

— **dgebrd.help** —

```
==================================================================
dgebrd examples
==================================================================

==================================================================
Man Page Details
==================================================================

NAME
       DGEBRD  -  a  general real M-by-N matrix A to upper or lower bidiagonal
       form B by an orthogonal transformation

SYNOPSIS
       SUBROUTINE DGEBRD( M, N, A, LDA, D, E, TAUQ, TAUP, WORK, LWORK, INFO )

           INTEGER        INFO, LDA, LWORK, M, N

           DOUBLE         PRECISION A( LDA, * ), D( * ), E( * ),  TAUP(  *  ),
                          TAUQ( * ), WORK( * )

PURPOSE
       DGEBRD reduces a general real M-by-N matrix A to upper or lower bidiag-
       onal form B by an orthogonal transformation: Q**T * A * P = B.

       If m >= n, B is upper bidiagonal; if m < n, B is lower bidiagonal.


ARGUMENTS
       M       (input) INTEGER
               The number of rows in the matrix A.  M >= 0.

       N       (input) INTEGER
```

```
        The number of columns in the matrix A.  N >= 0.

A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
        On entry, the M-by-N general matrix to be reduced.  On exit, if
        m  >= n, the diagonal and the first superdiagonal are overwrit-
        ten with the upper bidiagonal matrix B; the elements below  the
        diagonal,  with the array TAUQ, represent the orthogonal matrix
        Q as a product of elementary reflectors, and the elements above
        the  first  superdiagonal,  with  the array TAUP, represent the
        orthogonal matrix P as a product of elementary reflectors; if m
        <  n,  the  diagonal  and the first subdiagonal are overwritten
        with the lower bidiagonal matrix  B;  the  elements  below  the
        first  subdiagonal, with the array TAUQ, represent the orthogo-
        nal matrix Q as a product of  elementary  reflectors,  and  the
        elements above the diagonal, with the array TAUP, represent the
        orthogonal matrix P as a product of elementary reflectors.  See
        Further Details.  LDA     (input) INTEGER The leading dimension
        of the array A.  LDA >= max(1,M).

D       (output) DOUBLE PRECISION array, dimension (min(M,N))
        The diagonal elements  of  the  bidiagonal  matrix  B:  D(i)  =
        A(i,i).

E       (output) DOUBLE PRECISION array, dimension (min(M,N)-1)
        The  off-diagonal  elements of the bidiagonal matrix B: if m >=
        n, E(i) = A(i,i+1) for i =  1,2,...,n-1;  if  m  <  n,  E(i)  =
        A(i+1,i) for i = 1,2,...,m-1.

TAUQ    (output) DOUBLE PRECISION array dimension (min(M,N))
        The scalar factors of the elementary reflectors which represent
        the orthogonal matrix Q. See Further Details.  TAUP    (output)
        DOUBLE PRECISION array, dimension (min(M,N)) The scalar factors
        of the elementary reflectors  which  represent  the  orthogonal
        matrix P. See Further Details.  WORK    (workspace/output) DOU-
        BLE PRECISION array, dimension (MAX(1,LWORK)) On exit, if  INFO
        = 0, WORK(1) returns the optimal LWORK.

LWORK   (input) INTEGER
        The  length of the array WORK.  LWORK >= max(1,M,N).  For opti-
        mum performance LWORK >= (M+N)*NB,  where  NB  is  the  optimal
        blocksize.

        If  LWORK  = -1, then a workspace query is assumed; the routine
        only calculates the optimal size of  the  WORK  array,  returns
        this  value  as the first entry of the WORK array, and no error
        message related to LWORK is issued by XERBLA.

INFO    (output) INTEGER
        = 0:  successful exit
        < 0:  if INFO = -i, the i-th argument had an illegal value.
```

FURTHER DETAILS

The matrices Q and P are represented as products of elementary reflectors:

If m >= n,

```
Q = H(1) H(2) . . . H(n)  and  P = G(1) G(2) . . . G(n-1)
```

Each H(i) and G(i) has the form:

```
H(i) = I - tauq * v * v'  and G(i) = I - taup * u * u'
```

where tauq and taup are real scalars, and v and u are real vectors; v(1:i-1) = 0, v(i) = 1, and v(i+1:m) is stored on exit in A(i+1:m,i); u(1:i) = 0, u(i+1) = 1, and u(i+2:n) is stored on exit in A(i,i+2:n); tauq is stored in TAUQ(i) and taup in TAUP(i).

If m < n,

```
Q = H(1) H(2) . . . H(m-1)  and  P = G(1) G(2) . . . G(m)
```

Each H(i) and G(i) has the form:

```
H(i) = I - tauq * v * v'  and G(i) = I - taup * u * u'
```

where tauq and taup are real scalars, and v and u are real vectors; v(1:i) = 0, v(i+1) = 1, and v(i+2:m) is stored on exit in A(i+2:m,i); u(1:i-1) = 0, u(i) = 1, and u(i+1:n) is stored on exit in A(i,i+1:n); tauq is stored in TAUQ(i) and taup in TAUP(i).

The contents of A on exit are illustrated by the following examples:

m = 6 and n = 5 (m > n):

```
( d   e   u1  u1  u1 )
( v1  d   e   u2  u2 )
( v1  v2  d   e   u3 )
( v1  v2  v3  d   e  )
( v1  v2  v3  v4  d  )
( v1  v2  v3  v4  v5 )
```

m = 5 and n = 6 (m < n):

```
( d   u1  u1  u1  u1  u1 )
( e   d   u2  u2  u2  u2 )
( v1  e   d   u3  u3  u3 )
( v1  v2  e   d   u4  u4 )
( v1  v2  v3  e   d   u5 )
```

where d and e denote diagonal and off-diagonal elements of B, vi denotes an element of the vector defining H(i), and ui an element of the vector defining G(i).

90.119.3 fortran code

— dgebrd.f —

```
*  =====================================================================
      SUBROUTINE DGEBRD( M, N, A, LDA, D, E, TAUQ, TAUP, WORK, LWORK,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), D( * ), E( * ), TAUP( * ),
     $                   TAUQ( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IINFO, J, LDWRKX, LDWRKY, LWKOPT, MINMN, NB,
     $                   NBMIN, NX
      DOUBLE PRECISION   WS
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEBD2, DGEMM, DLABRD, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      NB = MAX( 1, ILAENV( 1, 'DGEBRD', ' ', M, N, -1, -1 ) )
```

```
      LWKOPT = ( M+N )*NB
      WORK( 1 ) = DBLE( LWKOPT )
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, M, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -10
      END IF
      IF( INFO.LT.0 ) THEN
         CALL XERBLA( 'DGEBRD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      MINMN = MIN( M, N )
      IF( MINMN.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      WS = MAX( M, N )
      LDWRKX = M
      LDWRKY = N
*
      IF( NB.GT.1 .AND. NB.LT.MINMN ) THEN
*
*        Set the crossover point NX.
*
         NX = MAX( NB, ILAENV( 3, 'DGEBRD', ' ', M, N, -1, -1 ) )
*
*        Determine when to switch from blocked to unblocked code.
*
         IF( NX.LT.MINMN ) THEN
            WS = ( M+N )*NB
            IF( LWORK.LT.WS ) THEN
*
*              Not enough work space for the optimal NB, consider using
*              a smaller block size.
*
               NBMIN = ILAENV( 2, 'DGEBRD', ' ', M, N, -1, -1 )
               IF( LWORK.GE.( M+N )*NBMIN ) THEN
                  NB = LWORK / ( M+N )
               ELSE
```

```
                NB = 1
                NX = MINMN
             END IF
          END IF
       END IF
    ELSE
       NX = MINMN
    END IF
*
    DO 30 I = 1, MINMN - NX, NB
*
*      Reduce rows and columns i:i+nb-1 to bidiagonal form and return
*      the matrices X and Y which are needed to update the unreduced
*      part of the matrix
*
       CALL DLABRD( M-I+1, N-I+1, NB, A( I, I ), LDA, D( I ), E( I ),
     $               TAUQ( I ), TAUP( I ), WORK, LDWRKX,
     $               WORK( LDWRKX*NB+1 ), LDWRKY )
*
*      Update the trailing submatrix A(i+nb:m,i+nb:n), using an update
*      of the form  A := A - V*Y**T - X*U**T
*
       CALL DGEMM( 'No transpose', 'Transpose', M-I-NB+1, N-I-NB+1,
     $              NB, -ONE, A( I+NB, I ), LDA,
     $              WORK( LDWRKX*NB+NB+1 ), LDWRKY, ONE,
     $              A( I+NB, I+NB ), LDA )
       CALL DGEMM( 'No transpose', 'No transpose', M-I-NB+1, N-I-NB+1,
     $              NB, -ONE, WORK( NB+1 ), LDWRKX, A( I, I+NB ), LDA,
     $              ONE, A( I+NB, I+NB ), LDA )
*
*      Copy diagonal and off-diagonal elements of B back into A
*
       IF( M.GE.N ) THEN
          DO 10 J = I, I + NB - 1
             A( J, J ) = D( J )
             A( J, J+1 ) = E( J )
 10       CONTINUE
       ELSE
          DO 20 J = I, I + NB - 1
             A( J, J ) = D( J )
             A( J+1, J ) = E( J )
 20       CONTINUE
       END IF
 30 CONTINUE
*
*    Use unblocked code to reduce the remainder of the matrix
*
    CALL DGEBD2( M-I+1, N-I+1, A( I, I ), LDA, D( I ), E( I ),
     $            TAUQ( I ), TAUP( I ), WORK, IINFO )
    WORK( 1 ) = WS
```

```
      RETURN
*
*     End of DGEBRD
*
      END
      SUBROUTINE DGEBRD( M, N, A, LDA, D, E, TAUQ, TAUP, WORK, LWORK,
     $                   INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), D( * ), E( * ), TAUP( * ),
     $                   TAUQ( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IINFO, J, LDWRKX, LDWRKY, LWKOPT, MINMN, NB,
     $                   NBMIN, NX
      DOUBLE PRECISION   WS
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEBD2, DGEMM, DLABRD, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      NB = MAX( 1, ILAENV( 1, 'DGEBRD', ' ', M, N, -1, -1 ) )
      LWKOPT = ( M+N )*NB
```

```
      WORK( 1 ) = DBLE( LWKOPT )
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, M, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -10
      END IF
      IF( INFO.LT.0 ) THEN
         CALL XERBLA( 'DGEBRD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      MINMN = MIN( M, N )
      IF( MINMN.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      WS = MAX( M, N )
      LDWRKX = M
      LDWRKY = N
*
      IF( NB.GT.1 .AND. NB.LT.MINMN ) THEN
*
*        Set the crossover point NX.
*
         NX = MAX( NB, ILAENV( 3, 'DGEBRD', ' ', M, N, -1, -1 ) )
*
*        Determine when to switch from blocked to unblocked code.
*
         IF( NX.LT.MINMN ) THEN
            WS = ( M+N )*NB
            IF( LWORK.LT.WS ) THEN
*
*              Not enough work space for the optimal NB, consider using
*              a smaller block size.
*
               NBMIN = ILAENV( 2, 'DGEBRD', ' ', M, N, -1, -1 )
               IF( LWORK.GE.( M+N )*NBMIN ) THEN
                  NB = LWORK / ( M+N )
               ELSE
                  NB = 1
```

```
                 NX = MINMN
              END IF
           END IF
        END IF
      ELSE
        NX = MINMN
      END IF
*
      DO 30 I = 1, MINMN - NX, NB
*
*        Reduce rows and columns i:i+nb-1 to bidiagonal form and return
*        the matrices X and Y which are needed to update the unreduced
*        part of the matrix
*
         CALL DLABRD( M-I+1, N-I+1, NB, A( I, I ), LDA, D( I ), E( I ),
     $                TAUQ( I ), TAUP( I ), WORK, LDWRKX,
     $                WORK( LDWRKX*NB+1 ), LDWRKY )
*
*        Update the trailing submatrix A(i+nb:m,i+nb:n), using an update
*        of the form  A := A - V*Y' - X*U'
*
         CALL DGEMM( 'No transpose', 'Transpose', M-I-NB+1, N-I-NB+1,
     $               NB, -ONE, A( I+NB, I ), LDA,
     $               WORK( LDWRKX*NB+NB+1 ), LDWRKY, ONE,
     $               A( I+NB, I+NB ), LDA )
         CALL DGEMM( 'No transpose', 'No transpose', M-I-NB+1, N-I-NB+1,
     $               NB, -ONE, WORK( NB+1 ), LDWRKX, A( I, I+NB ), LDA,
     $               ONE, A( I+NB, I+NB ), LDA )
*
*        Copy diagonal and off-diagonal elements of B back into A
*
         IF( M.GE.N ) THEN
            DO 10 J = I, I + NB - 1
               A( J, J ) = D( J )
               A( J, J+1 ) = E( J )
   10       CONTINUE
         ELSE
            DO 20 J = I, I + NB - 1
               A( J, J ) = D( J )
               A( J+1, J ) = E( J )
   20       CONTINUE
         END IF
   30 CONTINUE
*
*     Use unblocked code to reduce the remainder of the matrix
*
      CALL DGEBD2( M-I+1, N-I+1, A( I, I ), LDA, D( I ), E( I ),
     $             TAUQ( I ), TAUP( I ), WORK, IINFO )
      WORK( 1 ) = WS
      RETURN
```

```
*
*     End of DGEBRD
*
      END
```

———-

90.119.4 lisp code

— **dgebrd.lisp** —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dgebrd (m n a lda d e tauq taup work lwork info)
    (declare (type (array double-float (*)) work taup tauq e d a)
             (type (f2cl-lib:integer4) info lwork lda n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%)
         (tauq double-float tauq-%data% tauq-%offset%)
         (taup double-float taup-%data% taup-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((ws 0.0d0) (i 0) (iinfo 0) (j 0) (ldwrkx 0) (ldwrky 0) (lwkopt 0)
             (minmn 0) (nb 0) (nbmin 0) (nx 0) (lquery nil))
        (declare (type (double-float) ws)
                 (type (f2cl-lib:integer4) i iinfo j ldwrkx ldwrky lwkopt minmn
                  nb nbmin nx)
                 (type f2cl-lib:logical lquery))
        (setf info 0)
        (setf nb
                (max (the f2cl-lib:integer4 1)
                     (the f2cl-lib:integer4
                          (multiple-value-bind
                              (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                               var-6)
                              (ilaenv 1 "DGEBRD" " " m n -1 -1)
                            (declare (ignore var-0 var-1 var-2 var-5 var-6))
                            (when var-3 (setf m var-3))
                            (when var-4 (setf n var-4))
                            ret-val))))
        (setf lwkopt (f2cl-lib:int-mul (f2cl-lib:int-add m n) nb))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (f2cl-lib:dble lwkopt))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
```

```
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -4))
      ((and
        (< lwork
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)
                (the f2cl-lib:integer4 n)))
        (not lquery))
       (setf info -10)))
(cond
 ((< info 0) (xerbla "DGEBRD" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(cond
 ((= minmn 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'double-float))
  (go end_label)))
(setf ws
        (coerce
         (the f2cl-lib:integer4
              (max (the f2cl-lib:integer4 m)
                   (the f2cl-lib:integer4 n)))
         'double-float))
(setf ldwrkx m)
(setf ldwrky n)
(cond
 ((and (> nb 1) (< nb minmn))
  (setf nx
          (max (the f2cl-lib:integer4 nb)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "DGEBRD" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (when var-3 (setf m var-3))
                      (when var-4 (setf n var-4))
                      ret-val))))
  (cond
   ((< nx minmn)
    (setf ws
            (coerce
             (the f2cl-lib:integer4
                  (f2cl-lib:int-mul (f2cl-lib:int-add m n) nb))
             'double-float))
    (cond
     ((< lwork ws)
      (setf nbmin
              (multiple-value-bind
```

```
                      (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                      (ilaenv 2 "DGEBRD" " " m n -1 -1)
                    (declare (ignore var-0 var-1 var-2 var-5 var-6))
                    (when var-3 (setf m var-3))
                    (when var-4 (setf n var-4))
                    ret-val))
          (cond
           ((>= lwork (f2cl-lib:int-mul (f2cl-lib:int-add m n) nbmin))
            (setf nb (the f2cl-lib:integer4 (truncate lwork (+ m n)))))
           (t (setf nb 1) (setf nx minmn))))))))
     (t (setf nx minmn)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                  ((> i (f2cl-lib:int-add minmn (f2cl-lib:int-sub nx))) nil)
      (tagbody
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11 var-12)
            (dlabrd (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) nb
             (f2cl-lib:array-slice a-%data% double-float (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice d-%data% double-float (i) ((1 *))
                                   d-%offset%)
             (f2cl-lib:array-slice e-%data% double-float (i) ((1 *))
                                   e-%offset%)
             (f2cl-lib:array-slice tauq-%data% double-float (i) ((1 *))
                                   tauq-%offset%)
             (f2cl-lib:array-slice taup-%data% double-float (i) ((1 *))
                                   taup-%offset%)
             work ldwrkx
             (f2cl-lib:array-slice work-%data% double-float
                                   ((+ (f2cl-lib:int-mul ldwrkx nb) 1))
                                   ((1 *)) work-%offset%)
             ldwrky)
          (declare
           (ignore var-0 var-1 var-3 var-5 var-6 var-7 var-8 var-9 var-11))
          (when var-2 (setf nb var-2))
          (when var-4 (setf lda var-4))
          (when var-10 (setf ldwrkx var-10))
          (when var-12 (setf ldwrky var-12)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11 var-12)
            (dgemm "No transpose" "Transpose"
             (f2cl-lib:int-add (f2cl-lib:int-sub m i nb) 1)
             (f2cl-lib:int-add (f2cl-lib:int-sub n i nb) 1) nb (- one)
             (f2cl-lib:array-slice a-%data% double-float ((+ i nb) i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
```

```
      (f2cl-lib:array-slice work-%data% double-float
                            ((+ (f2cl-lib:int-mul ldwrkx nb) nb 1))
                            ((1 *)) work-%offset%)
      ldwrky one
      (f2cl-lib:array-slice a-%data% double-float
                            ((+ i nb) (f2cl-lib:int-add i nb))
                            ((1 lda) (1 *)) a-%offset%)
      lda)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8 var-11))
    (when var-4 (setf nb var-4))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldwrky var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf lda var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dgemm "No transpose" "No transpose"
       (f2cl-lib:int-add (f2cl-lib:int-sub m i nb) 1)
       (f2cl-lib:int-add (f2cl-lib:int-sub n i nb) 1) nb (- one)
       (f2cl-lib:array-slice work-%data% double-float ((+ nb 1))
                            ((1 *)) work-%offset%)
       ldwrkx
       (f2cl-lib:array-slice a-%data% double-float
                            (i (f2cl-lib:int-add i nb))
                            ((1 lda) (1 *)) a-%offset%)
       lda one
       (f2cl-lib:array-slice a-%data% double-float
                            ((+ i nb) (f2cl-lib:int-add i nb))
                            ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8 var-11))
    (when var-4 (setf nb var-4))
    (when var-7 (setf ldwrkx var-7))
    (when var-9 (setf lda var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf lda var-12)))
  (cond
   ((>= m n)
    (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                  ((> j (f2cl-lib:int-add i nb (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                            a-%offset%)
              (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%))
        (setf (f2cl-lib:fref a-%data% (j (f2cl-lib:int-add j 1))
                            ((1 lda) (1 *)) a-%offset%)
```

```
                    (f2cl-lib:fref e-%data% (j) ((1 *)) e-%offset%))
            label10)))
        (t
         (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                       ((> j (f2cl-lib:int-add i nb (f2cl-lib:int-sub 1)))
                        nil)
           (tagbody
             (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                   a-%offset%)
                    (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%))
             (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add j 1) j)
                                   ((1 lda) (1 *)) a-%offset%)
                    (f2cl-lib:fref e-%data% (j) ((1 *)) e-%offset%))
            label20))))
    label30))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dgebd2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
       (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
       (f2cl-lib:array-slice a-%data% double-float (i i) ((1 lda) (1 *))
                             a-%offset%)
       lda
       (f2cl-lib:array-slice d-%data% double-float (i) ((1 *))
                             d-%offset%)
       (f2cl-lib:array-slice e-%data% double-float (i) ((1 *))
                             e-%offset%)
       (f2cl-lib:array-slice tauq-%data% double-float (i) ((1 *))
                             tauq-%offset%)
       (f2cl-lib:array-slice taup-%data% double-float (i) ((1 *))
                             taup-%offset%)
       work iinfo)
    (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6 var-7 var-8))
    (setf lda var-3)
    (setf iinfo var-9))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) ws)
  (go end_label)
 end_label
  (return (values m n nil lda nil nil nil nil nil nil info))))))
```

———-

90.120 dgeev

90.120.1 Axiom unit tests

— **dgeev.input** —

```
)set break resume
)sys rm -f dgeev.output
)spool dgeev.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.120.2 Axiom help page

— **dgeev.help** —

```
======================================================================
dgeev examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DGEEV  - for an N-by-N real nonsymmetric matrix A, the eigenvalues and,
       optionally, the left and/or right eigenvectors

SYNOPSIS
       SUBROUTINE DGEEV( JOBVL, JOBVR, N, A, LDA, WR, WI, VL, LDVL, VR,  LDVR,
                         WORK, LWORK, INFO )

           CHARACTER     JOBVL, JOBVR

           INTEGER       INFO, LDA, LDVL, LDVR, LWORK, N

           DOUBLE        PRECISION  A( LDA, * ), VL( LDVL, * ), VR( LDVR, * ),
                         WI( * ), WORK( * ), WR( * )

PURPOSE
       DGEEV computes for an N-by-N real nonsymmetric matrix A, the  eigenval-
       ues and, optionally, the left and/or right eigenvectors.

       The right eigenvector v(j) of A satisfies
                        A * v(j) = lambda(j) * v(j)
       where lambda(j) is its eigenvalue.
       The left eigenvector u(j) of A satisfies
                     u(j)**H * A = lambda(j) * u(j)**H
```

```
      where u(j)**H denotes the conjugate transpose of u(j).

      The  computed  eigenvectors are normalized to have Euclidean norm equal
      to 1 and largest component real.


ARGUMENTS
      JOBVL   (input) CHARACTER*1
              = 'N': left eigenvectors of A are not computed;
              = 'V': left eigenvectors of A are computed.

      JOBVR   (input) CHARACTER*1
              = 'N': right eigenvectors of A are not computed;
              = 'V': right eigenvectors of A are computed.

      N       (input) INTEGER
              The order of the matrix A. N >= 0.

      A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
              On entry, the N-by-N matrix A.  On exit, A has  been  overwrit-
              ten.

      LDA     (input) INTEGER
              The leading dimension of the array A.  LDA >= max(1,N).

      WR      (output) DOUBLE PRECISION array, dimension (N)
              WI       (output)  DOUBLE PRECISION array, dimension (N) WR and
              WI contain the real and imaginary parts, respectively,  of  the
              computed  eigenvalues.   Complex conjugate pairs of eigenvalues
              appear consecutively with the eigenvalue  having  the  positive
              imaginary part first.

      VL      (output) DOUBLE PRECISION array, dimension (LDVL,N)
              If JOBVL = 'V', the left eigenvectors u(j) are stored one after
              another in the columns of VL, in the same order as their eigen-
              values.   If  JOBVL  =  'N', VL is not referenced.  If the j-th
              eigenvalue is real, then u(j) = VL(:,j), the j-th column of VL.
              If  the  j-th and (j+1)-st eigenvalues form a complex conjugate
              pair, then u(j) = VL(:,j) + i*VL(:,j+1) and
              u(j+1) = VL(:,j) - i*VL(:,j+1).

      LDVL    (input) INTEGER
              The leading dimension of the array VL.  LDVL >= 1; if  JOBVL  =
              'V', LDVL >= N.

      VR      (output) DOUBLE PRECISION array, dimension (LDVR,N)
              If  JOBVR  =  'V',  the  right eigenvectors v(j) are stored one
              after another in the columns of VR, in the same order as  their
              eigenvalues.   If JOBVR = 'N', VR is not referenced.  If the j-
              th eigenvalue is real, then v(j) = VR(:,j), the j-th column  of
```

```
            VR.  If the j-th and (j+1)-st eigenvalues form a complex conju-
            gate pair, then v(j) = VR(:,j) + i*VR(:,j+1) and
            v(j+1) = VR(:,j) - i*VR(:,j+1).

      LDVR    (input) INTEGER
            The leading dimension of the array VR.  LDVR >= 1; if  JOBVR  =
            'V', LDVR >= N.

      WORK       (workspace/output)   DOUBLE   PRECISION   array,   dimension
      (MAX(1,LWORK))
            On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

      LWORK   (input) INTEGER
            The  dimension  of the array WORK.  LWORK >= max(1,3*N), and if
            JOBVL = 'V' or JOBVR = 'V', LWORK >=  4*N.   For  good  perfor-
            mance, LWORK must generally be larger.

            If  LWORK  = -1, then a workspace query is assumed; the routine
            only calculates the optimal size of  the  WORK  array,  returns
            this  value  as the first entry of the WORK array, and no error
            message related to LWORK is issued by XERBLA.

      INFO    (output) INTEGER
            = 0:  successful exit
            < 0:  if INFO = -i, the i-th argument had an illegal value.
            > 0:  if INFO = i, the QR algorithm failed to compute  all  the
            eigenvalues,  and  no eigenvectors have been computed; elements
            i+1:N of WR and WI contain eigenvalues which have converged.
```

———-

90.120.3 fortran code

— dgeev.f —

```
*  =====================================================================
      SUBROUTINE DGEEV( JOBVL, JOBVR, N, A, LDA, WR, WI, VL, LDVL, VR,
     $                  LDVR, WORK, LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVL, JOBVR
      INTEGER            INFO, LDA, LDVL, LDVR, LWORK, N
```

```
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   WI( * ), WORK( * ), WR( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, SCALEA, WANTVL, WANTVR
      CHARACTER          SIDE
      INTEGER            HSWORK, I, IBAL, IERR, IHI, ILO, ITAU, IWRK, K,
     $                   MAXWRK, MINWRK, NOUT
      DOUBLE PRECISION   ANRM, BIGNUM, CS, CSCALE, EPS, R, SCL, SMLNUM,
     $                   SN
*     ..
*     .. Local Arrays ..
      LOGICAL            SELECT( 1 )
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEBAK, DGEBAL, DGEHRD, DHSEQR, DLABAD, DLACPY,
     $                   DLARTG, DLASCL, DORGHR, DROT, DSCAL, DTREVC,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX, ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE, DLAPY2, DNRM2
      EXTERNAL           LSAME, IDAMAX, ILAENV, DLAMCH, DLANGE, DLAPY2,
     $                   DNRM2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      WANTVL = LSAME( JOBVL, 'V' )
      WANTVR = LSAME( JOBVR, 'V' )
      IF( ( .NOT.WANTVL ) .AND. ( .NOT.LSAME( JOBVL, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTVR ) .AND. ( .NOT.LSAME( JOBVR, 'N' ) ) ) THEN
```

```
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDVL.LT.1 .OR. ( WANTVL .AND. LDVL.LT.N ) ) THEN
         INFO = -9
      ELSE IF( LDVR.LT.1 .OR. ( WANTVR .AND. LDVR.LT.N ) ) THEN
         INFO = -11
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by DHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            MAXWRK = 1
         ELSE
            MAXWRK = 2*N + N*ILAENV( 1, 'DGEHRD', ' ', N, 1, N, 0 )
            IF( WANTVL ) THEN
               MINWRK = 4*N
               MAXWRK = MAX( MAXWRK, 2*N + ( N - 1 )*ILAENV( 1,
     $                       'DORGHR', ' ', N, 1, N, -1 ) )
               CALL DHSEQR( 'S', 'V', N, 1, N, A, LDA, WR, WI, VL, LDVL,
     $                WORK, -1, INFO )
               HSWORK = WORK( 1 )
               MAXWRK = MAX( MAXWRK, N + 1, N + HSWORK )
               MAXWRK = MAX( MAXWRK, 4*N )
            ELSE IF( WANTVR ) THEN
               MINWRK = 4*N
               MAXWRK = MAX( MAXWRK, 2*N + ( N - 1 )*ILAENV( 1,
     $                       'DORGHR', ' ', N, 1, N, -1 ) )
               CALL DHSEQR( 'S', 'V', N, 1, N, A, LDA, WR, WI, VR, LDVR,
     $                WORK, -1, INFO )
               HSWORK = WORK( 1 )
               MAXWRK = MAX( MAXWRK, N + 1, N + HSWORK )
               MAXWRK = MAX( MAXWRK, 4*N )
            ELSE
               MINWRK = 3*N
               CALL DHSEQR( 'E', 'N', N, 1, N, A, LDA, WR, WI, VR, LDVR,
     $                WORK, -1, INFO )
               HSWORK = WORK( 1 )
```

```
               MAXWRK = MAX( MAXWRK, N + 1, N + HSWORK )
            END IF
            MAXWRK = MAX( MAXWRK, MINWRK )
         END IF
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -13
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEEV ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = DLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
     $   CALL DLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*     Balance the matrix
*     (Workspace: need N)
*
      IBAL = 1
```

```
      CALL DGEBAL( 'B', N, A, LDA, ILO, IHI, WORK( IBAL ), IERR )
*
*     Reduce to upper Hessenberg form
*     (Workspace: need 3*N, prefer 2*N+N*NB)
*
      ITAU = IBAL + N
      IWRK = ITAU + N
      CALL DGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
      IF( WANTVL ) THEN
*
*        Want left eigenvectors
*        Copy Householder vectors to VL
*
         SIDE = 'L'
         CALL DLACPY( 'L', N, N, A, LDA, VL, LDVL )
*
*        Generate orthogonal matrix in VL
*        (Workspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*
         CALL DORGHR( N, ILO, IHI, VL, LDVL, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VL
*        (Workspace: need N+1, prefer N+HSWORK (see comments) )
*
         IWRK = ITAU
         CALL DHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, WR, WI, VL, LDVL,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
         IF( WANTVR ) THEN
*
*           Want left and right eigenvectors
*           Copy Schur vectors to VR
*
            SIDE = 'B'
            CALL DLACPY( 'F', N, N, VL, LDVL, VR, LDVR )
         END IF
*
      ELSE IF( WANTVR ) THEN
*
*        Want right eigenvectors
*        Copy Householder vectors to VR
*
         SIDE = 'R'
         CALL DLACPY( 'L', N, N, A, LDA, VR, LDVR )
*
*        Generate orthogonal matrix in VR
*        (Workspace: need 3*N-1, prefer 2*N+(N-1)*NB)
```

```
*
         CALL DORGHR( N, ILO, IHI, VR, LDVR, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VR
*        (Workspace: need N+1, prefer N+HSWORK (see comments) )
*
         IWRK = ITAU
         CALL DHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, WR, WI, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
      ELSE
*
*        Compute eigenvalues only
*        (Workspace: need N+1, prefer N+HSWORK (see comments) )
*
         IWRK = ITAU
         CALL DHSEQR( 'E', 'N', N, ILO, IHI, A, LDA, WR, WI, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
      END IF
*
*     If INFO > 0 from DHSEQR, then quit
*
      IF( INFO.GT.0 )
     $   GO TO 50
*
      IF( WANTVL .OR. WANTVR ) THEN
*
*        Compute left and/or right eigenvectors
*        (Workspace: need 4*N)
*
         CALL DTREVC( SIDE, 'B', SELECT, N, A, LDA, VL, LDVL, VR, LDVR,
     $                N, NOUT, WORK( IWRK ), IERR )
      END IF
*
      IF( WANTVL ) THEN
*
*        Undo balancing of left eigenvectors
*        (Workspace: need N)
*
         CALL DGEBAK( 'B', 'L', N, ILO, IHI, WORK( IBAL ), N, VL, LDVL,
     $                IERR )
*
*        Normalize left eigenvectors and make largest component real
*
         DO 20 I = 1, N
            IF( WI( I ).EQ.ZERO ) THEN
               SCL = ONE / DNRM2( N, VL( 1, I ), 1 )
               CALL DSCAL( N, SCL, VL( 1, I ), 1 )
            ELSE IF( WI( I ).GT.ZERO ) THEN
```

```
               SCL = ONE / DLAPY2( DNRM2( N, VL( 1, I ), 1 ),
     $               DNRM2( N, VL( 1, I+1 ), 1 ) )
               CALL DSCAL( N, SCL, VL( 1, I ), 1 )
               CALL DSCAL( N, SCL, VL( 1, I+1 ), 1 )
               DO 10 K = 1, N
                  WORK( IWRK+K-1 ) = VL( K, I )**2 + VL( K, I+1 )**2
   10          CONTINUE
               K = IDAMAX( N, WORK( IWRK ), 1 )
               CALL DLARTG( VL( K, I ), VL( K, I+1 ), CS, SN, R )
               CALL DROT( N, VL( 1, I ), 1, VL( 1, I+1 ), 1, CS, SN )
               VL( K, I+1 ) = ZERO
            END IF
   20    CONTINUE
      END IF
*
      IF( WANTVR ) THEN
*
*        Undo balancing of right eigenvectors
*        (Workspace: need N)
*
         CALL DGEBAK( 'B', 'R', N, ILO, IHI, WORK( IBAL ), N, VR, LDVR,
     $                IERR )
*
*        Normalize right eigenvectors and make largest component real
*
         DO 40 I = 1, N
            IF( WI( I ).EQ.ZERO ) THEN
               SCL = ONE / DNRM2( N, VR( 1, I ), 1 )
               CALL DSCAL( N, SCL, VR( 1, I ), 1 )
            ELSE IF( WI( I ).GT.ZERO ) THEN
               SCL = ONE / DLAPY2( DNRM2( N, VR( 1, I ), 1 ),
     $               DNRM2( N, VR( 1, I+1 ), 1 ) )
               CALL DSCAL( N, SCL, VR( 1, I ), 1 )
               CALL DSCAL( N, SCL, VR( 1, I+1 ), 1 )
               DO 30 K = 1, N
                  WORK( IWRK+K-1 ) = VR( K, I )**2 + VR( K, I+1 )**2
   30          CONTINUE
               K = IDAMAX( N, WORK( IWRK ), 1 )
               CALL DLARTG( VR( K, I ), VR( K, I+1 ), CS, SN, R )
               CALL DROT( N, VR( 1, I ), 1, VR( 1, I+1 ), 1, CS, SN )
               VR( K, I+1 ) = ZERO
            END IF
   40    CONTINUE
      END IF
*
*     Undo scaling if necessary
*
   50 CONTINUE
      IF( SCALEA ) THEN
         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, WR( INFO+1 ),
```

```
     $                MAX( N-INFO, 1 ), IERR )
         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, WI( INFO+1 ),
     $                MAX( N-INFO, 1 ), IERR )
         IF( INFO.GT.0 ) THEN
            CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WR, N,
     $                   IERR )
            CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WI, N,
     $                   IERR )
         END IF
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of DGEEV
*
      END
      SUBROUTINE DGEEV( JOBVL, JOBVR, N, A, LDA, WR, WI, VL, LDVL, VR,
     $                  LDVR, WORK, LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     December 8, 1999
*
*     .. Scalar Arguments ..
      CHARACTER          JOBVL, JOBVR
      INTEGER            INFO, LDA, LDVL, LDVR, LWORK, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   WI( * ), WORK( * ), WR( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, SCALEA, WANTVL, WANTVR
      CHARACTER          SIDE
      INTEGER            HSWORK, I, IBAL, IERR, IHI, ILO, ITAU, IWRK, K,
     $                   MAXB, MAXWRK, MINWRK, NOUT
      DOUBLE PRECISION   ANRM, BIGNUM, CS, CSCALE, EPS, R, SCL, SMLNUM,
     $                   SN
*     ..
*     .. Local Arrays ..
      LOGICAL            SELECT( 1 )
```

```
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEBAK, DGEBAL, DGEHRD, DHSEQR, DLACPY, DLARTG,
     $                   DLASCL, DORGHR, DROT, DSCAL, DTREVC, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX, ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE, DLAPY2, DNRM2
      EXTERNAL           LSAME, IDAMAX, ILAENV, DLAMCH, DLANGE, DLAPY2,
     $                   DNRM2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      WANTVL = LSAME( JOBVL, 'V' )
      WANTVR = LSAME( JOBVR, 'V' )
      IF( ( .NOT.WANTVL ) .AND. ( .NOT.LSAME( JOBVL, 'N' ) ) ) THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTVR ) .AND. ( .NOT.LSAME( JOBVR, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDVL.LT.1 .OR. ( WANTVL .AND. LDVL.LT.N ) ) THEN
         INFO = -9
      ELSE IF( LDVR.LT.1 .OR. ( WANTVR .AND. LDVR.LT.N ) ) THEN
         INFO = -11
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by DHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.)
*
      MINWRK = 1
      IF( INFO.EQ.0 .AND. ( LWORK.GE.1 .OR. LQUERY ) ) THEN
```

```
         MAXWRK = 2*N + N*ILAENV( 1, 'DGEHRD', ' ', N, 1, N, 0 )
         IF( ( .NOT.WANTVL ) .AND. ( .NOT.WANTVR ) ) THEN
            MINWRK = MAX( 1, 3*N )
            MAXB = MAX( ILAENV( 8, 'DHSEQR', 'EN', N, 1, N, -1 ), 2 )
            K = MIN( MAXB, N, MAX( 2, ILAENV( 4, 'DHSEQR', 'EN', N, 1,
     $          N, -1 ) ) )
            HSWORK = MAX( K*( K+2 ), 2*N )
            MAXWRK = MAX( MAXWRK, N+1, N+HSWORK )
         ELSE
            MINWRK = MAX( 1, 4*N )
            MAXWRK = MAX( MAXWRK, 2*N+( N-1 )*
     $               ILAENV( 1, 'DORGHR', ' ', N, 1, N, -1 ) )
            MAXB = MAX( ILAENV( 8, 'DHSEQR', 'SV', N, 1, N, -1 ), 2 )
            K = MIN( MAXB, N, MAX( 2, ILAENV( 4, 'DHSEQR', 'SV', N, 1,
     $          N, -1 ) ) )
            HSWORK = MAX( K*( K+2 ), 2*N )
            MAXWRK = MAX( MAXWRK, N+1, N+HSWORK )
            MAXWRK = MAX( MAXWRK, 4*N )
         END IF
         WORK( 1 ) = MAXWRK
      END IF
      IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
         INFO = -13
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEEV ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = DLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
```

```
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
     $   CALL DLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*     Balance the matrix
*     (Workspace: need N)
*
      IBAL = 1
      CALL DGEBAL( 'B', N, A, LDA, ILO, IHI, WORK( IBAL ), IERR )
*
*     Reduce to upper Hessenberg form
*     (Workspace: need 3*N, prefer 2*N+N*NB)
*
      ITAU = IBAL + N
      IWRK = ITAU + N
      CALL DGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
      IF( WANTVL ) THEN
*
*        Want left eigenvectors
*        Copy Householder vectors to VL
*
         SIDE = 'L'
         CALL DLACPY( 'L', N, N, A, LDA, VL, LDVL )
*
*        Generate orthogonal matrix in VL
*        (Workspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*
         CALL DORGHR( N, ILO, IHI, VL, LDVL, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VL
*        (Workspace: need N+1, prefer N+HSWORK (see comments) )
*
         IWRK = ITAU
         CALL DHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, WR, WI, VL, LDVL,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
         IF( WANTVR ) THEN
*
*           Want left and right eigenvectors
*           Copy Schur vectors to VR
*
            SIDE = 'B'
```

```
            CALL DLACPY( 'F', N, N, VL, LDVL, VR, LDVR )
         END IF
*
      ELSE IF( WANTVR ) THEN
*
*        Want right eigenvectors
*        Copy Householder vectors to VR
*
         SIDE = 'R'
         CALL DLACPY( 'L', N, N, A, LDA, VR, LDVR )
*
*        Generate orthogonal matrix in VR
*        (Workspace: need 3*N-1, prefer 2*N+(N-1)*NB)
*
         CALL DORGHR( N, ILO, IHI, VR, LDVR, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VR
*        (Workspace: need N+1, prefer N+HSWORK (see comments) )
*
         IWRK = ITAU
         CALL DHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, WR, WI, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
      ELSE
*
*        Compute eigenvalues only
*        (Workspace: need N+1, prefer N+HSWORK (see comments) )
*
         IWRK = ITAU
         CALL DHSEQR( 'E', 'N', N, ILO, IHI, A, LDA, WR, WI, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
      END IF
*
*     If INFO > 0 from DHSEQR, then quit
*
      IF( INFO.GT.0 )
     $   GO TO 50
*
      IF( WANTVL .OR. WANTVR ) THEN
*
*        Compute left and/or right eigenvectors
*        (Workspace: need 4*N)
*
         CALL DTREVC( SIDE, 'B', SELECT, N, A, LDA, VL, LDVL, VR, LDVR,
     $                N, NOUT, WORK( IWRK ), IERR )
      END IF
*
      IF( WANTVL ) THEN
*
```

```
*        Undo balancing of left eigenvectors
*        (Workspace: need N)
*
         CALL DGEBAK( 'B', 'L', N, ILO, IHI, WORK( IBAL ), N, VL, LDVL,
     $                IERR )
*
*        Normalize left eigenvectors and make largest component real
*
         DO 20 I = 1, N
            IF( WI( I ).EQ.ZERO ) THEN
               SCL = ONE / DNRM2( N, VL( 1, I ), 1 )
               CALL DSCAL( N, SCL, VL( 1, I ), 1 )
            ELSE IF( WI( I ).GT.ZERO ) THEN
               SCL = ONE / DLAPY2( DNRM2( N, VL( 1, I ), 1 ),
     $               DNRM2( N, VL( 1, I+1 ), 1 ) )
               CALL DSCAL( N, SCL, VL( 1, I ), 1 )
               CALL DSCAL( N, SCL, VL( 1, I+1 ), 1 )
               DO 10 K = 1, N
                  WORK( IWRK+K-1 ) = VL( K, I )**2 + VL( K, I+1 )**2
   10          CONTINUE
               K = IDAMAX( N, WORK( IWRK ), 1 )
               CALL DLARTG( VL( K, I ), VL( K, I+1 ), CS, SN, R )
               CALL DROT( N, VL( 1, I ), 1, VL( 1, I+1 ), 1, CS, SN )
               VL( K, I+1 ) = ZERO
            END IF
   20    CONTINUE
      END IF
*
      IF( WANTVR ) THEN
*
*        Undo balancing of right eigenvectors
*        (Workspace: need N)
*
         CALL DGEBAK( 'B', 'R', N, ILO, IHI, WORK( IBAL ), N, VR, LDVR,
     $                IERR )
*
*        Normalize right eigenvectors and make largest component real
*
         DO 40 I = 1, N
            IF( WI( I ).EQ.ZERO ) THEN
               SCL = ONE / DNRM2( N, VR( 1, I ), 1 )
               CALL DSCAL( N, SCL, VR( 1, I ), 1 )
            ELSE IF( WI( I ).GT.ZERO ) THEN
               SCL = ONE / DLAPY2( DNRM2( N, VR( 1, I ), 1 ),
     $               DNRM2( N, VR( 1, I+1 ), 1 ) )
               CALL DSCAL( N, SCL, VR( 1, I ), 1 )
               CALL DSCAL( N, SCL, VR( 1, I+1 ), 1 )
               DO 30 K = 1, N
                  WORK( IWRK+K-1 ) = VR( K, I )**2 + VR( K, I+1 )**2
   30          CONTINUE
```

```
              K = IDAMAX( N, WORK( IWRK ), 1 )
              CALL DLARTG( VR( K, I ), VR( K, I+1 ), CS, SN, R )
              CALL DROT( N, VR( 1, I ), 1, VR( 1, I+1 ), 1, CS, SN )
              VR( K, I+1 ) = ZERO
           END IF
   40    CONTINUE
      END IF
*
*     Undo scaling if necessary
*
   50 CONTINUE
      IF( SCALEA ) THEN
         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, WR( INFO+1 ),
     $                MAX( N-INFO, 1 ), IERR )
         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, WI( INFO+1 ),
     $                MAX( N-INFO, 1 ), IERR )
         IF( INFO.GT.0 ) THEN
            CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WR, N,
     $                   IERR )
            CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WI, N,
     $                   IERR )
         END IF
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of DGEEV
*
      END
```

———-

90.120.4 lisp code

— dgeev.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dgeev (jobvl jobvr n a lda wr wi vl ldvl vr ldvr work lwork info)
    (declare (type (array double-float (*)) work vr vl wi wr a)
             (type (f2cl-lib:integer4) info lwork ldvr ldvl lda n)
             (type (string *) jobvr jobvl))
    (f2cl-lib:with-multi-array-data
        ((jobvl character jobvl-%data% jobvl-%offset%)
```

```
    (jobvr character jobvr-%data% jobvr-%offset%)
    (a double-float a-%data% a-%offset%)
    (wr double-float wr-%data% wr-%offset%)
    (wi double-float wi-%data% wi-%offset%)
    (vl double-float vl-%data% vl-%offset%)
    (vr double-float vr-%data% vr-%offset%)
    (work double-float work-%data% work-%offset%))
  (prog ((dum (make-array 1 :element-type 'double-float))
         (select (make-array 1 :element-type 't)) (anrm 0.0d0)
         (bignum 0.0d0) (cs 0.0d0) (cscale 0.0d0) (eps 0.0d0) (r 0.0d0)
         (scl 0.0d0) (smlnum 0.0d0) (sn 0.0d0) (hswork 0) (i 0) (ibal 0)
         (ierr 0) (ihi 0) (ilo 0) (itau 0) (iwrk 0) (k 0) (maxwrk 0)
         (minwrk 0) (nout 0)
         (side
          (make-array '(1) :element-type 'character :initial-element #\ ))
         (lquery nil) (scalea nil) (wantvl nil) (wantvr nil))
    (declare (type (array double-float (1)) dum)
             (type (array f2cl-lib:logical (1)) select)
             (type (double-float) anrm bignum cs cscale eps r scl smlnum
              sn)
             (type (f2cl-lib:integer4) hswork i ibal ierr ihi ilo itau iwrk
              k maxwrk minwrk nout)
             (type (string 1) side)
             (type f2cl-lib:logical lquery scalea wantvl wantvr))
    (setf info 0)
    (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
    (setf wantvl
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobvl "V")
              (declare (ignore var-1))
              (when var-0 (setf jobvl var-0))
              ret-val))
    (setf wantvr
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobvr "V")
              (declare (ignore var-1))
              (when var-0 (setf jobvr var-0))
              ret-val))
    (cond
     ((and (not wantvl)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame jobvl "N")
              (declare (ignore var-1))
              (when var-0 (setf jobvl var-0))
              ret-val)))
      (setf info -1))
     ((and (not wantvr)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
```

```
              (lsame jobvr "N")
            (declare (ignore var-1))
            (when var-0 (setf jobvr var-0))
            ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((or (< ldvl 1) (and wantvl (< ldvl n))) (setf info -9))
 ((or (< ldvr 1) (and wantvr (< ldvr n))) (setf info -11)))
(cond
 ((= info 0)
  (cond ((= n 0) (setf minwrk 1) (setf maxwrk 1))
        (t
         (setf maxwrk
                 (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                   (f2cl-lib:int-mul n
                                                     (multiple-value-bind
                                                         (ret-val var-0
                                                          var-1 var-2
                                                          var-3 var-4
                                                          var-5 var-6)
                                                         (ilaenv 1
                                                          "DGEHRD" " "
                                                          n 1 n 0)
                                                       (declare
                                                        (ignore var-0
                                                         var-1 var-2
                                                         var-4 var-6))
                                                       (when var-3
                                                         (setf n
                                                                 var-3))
                                                       (when var-5
                                                         (setf n
                                                                 var-5))
                                                       ret-val))))
         (cond
          (wantvl (setf minwrk (f2cl-lib:int-mul 4 n))
           (setf maxwrk
                   (max (the f2cl-lib:integer4 maxwrk)
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                               (f2cl-lib:int-mul
                                                (f2cl-lib:int-sub n 1)
                                                (multiple-value-bind
                                                    (ret-val var-0
                                                     var-1 var-2 var-3
                                                     var-4 var-5 var-6)
                                                    (ilaenv 1 "DORGHR"
                                                     " " n 1 n -1)
```

```
                                        (declare
                                         (ignore var-0 var-1
                                          var-2 var-4 var-6))
                                        (when var-3
                                          (setf n var-3))
                                        (when var-5
                                          (setf n var-5))
                                        ret-val))))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12 var-13)
     (dhseqr "S" "V" n 1 n a lda wr wi vl ldvl work -1 info)
   (declare
    (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-9 var-11
     var-12))
   (when var-2 (setf n var-2))
   (when var-4 (setf n var-4))
   (when var-6 (setf lda var-6))
   (when var-10 (setf ldvl var-10))
   (when var-13 (setf info var-13)))
 (setf hswork
         (f2cl-lib:int
          (f2cl-lib:fref work-%data% (1) ((1 *))
                         work-%offset%)))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4 (f2cl-lib:int-add n 1))
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n hswork))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-mul 4 n)))))
(wantvr (setf minwrk (f2cl-lib:int-mul 4 n))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     (f2cl-lib:int-mul
                                      (f2cl-lib:int-sub n 1)
                                      (multiple-value-bind
                                          (ret-val var-0
                                           var-1 var-2 var-3
                                           var-4 var-5 var-6)
                                          (ilaenv 1 "DORGHR"
                                           " " n 1 n -1)
                                        (declare
                                         (ignore var-0 var-1
                                          var-2 var-4 var-6))
                                        (when var-3
```

```
                                          (setf n var-3))
                                        (when var-5
                                          (setf n var-5))
                                        ret-val))))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12 var-13)
     (dhseqr "S" "V" n 1 n a lda wr wi vr ldvr work -1 info)
   (declare
    (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-9 var-11
     var-12))
   (when var-2 (setf n var-2))
   (when var-4 (setf n var-4))
   (when var-6 (setf lda var-6))
   (when var-10 (setf ldvr var-10))
   (when var-13 (setf info var-13)))
 (setf hswork
         (f2cl-lib:int
          (f2cl-lib:fref work-%data% (1) ((1 *))
                         work-%offset%)))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4 (f2cl-lib:int-add n 1))
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n hswork))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-mul 4 n)))))
(t (setf minwrk (f2cl-lib:int-mul 3 n))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12 var-13)
     (dhseqr "E" "N" n 1 n a lda wr wi vr ldvr work -1 info)
   (declare
    (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-9 var-11
     var-12))
   (when var-2 (setf n var-2))
   (when var-4 (setf n var-4))
   (when var-6 (setf lda var-6))
   (when var-10 (setf ldvr var-10))
   (when var-13 (setf info var-13)))
 (setf hswork
         (f2cl-lib:int
          (f2cl-lib:fref work-%data% (1) ((1 *))
                         work-%offset%)))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4 (f2cl-lib:int-add n 1))
              (the f2cl-lib:integer4
```

```
                              (f2cl-lib:int-add n hswork))))))
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4 minwrk)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 maxwrk) 'double-float))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -13)))))
(cond
 ((/= info 0) (xerbla "DGEEV " (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(setf eps (dlamch "P"))
(setf smlnum (dlamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "M" n n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf scalea f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf scalea f2cl-lib:%true%)
  (setf cscale smlnum))
 ((> anrm bignum) (setf scalea f2cl-lib:%true%) (setf cscale bignum)))
(if scalea
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 anrm cscale n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf cscale var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(setf ibal 1)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dgebal "B" n a lda ilo ihi
```

```
      (f2cl-lib:array-slice work-%data% double-float (ibal) ((1 *))
                            work-%offset%)
      ierr)
  (declare (ignore var-0 var-1 var-2 var-6))
  (setf lda var-3)
  (setf ilo var-4)
  (setf ihi var-5)
  (setf ierr var-7))
(setf itau (f2cl-lib:int-add ibal n))
(setf iwrk (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dgehrd n ilo ihi a lda
     (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf ilo var-1))
  (when var-2 (setf ihi var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(cond
 (wantvl (f2cl-lib:f2cl-set-string side "L" (string 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "L" n n a lda vl ldvl)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvl var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dorghr n ilo ihi vl ldvl
       (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf n var-0))
    (when var-1 (setf ilo var-1))
    (when var-2 (setf ihi var-2))
    (when var-4 (setf ldvl var-4))
    (when var-8 (setf ierr var-8)))
  (setf iwrk itau)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
      var-10 var-11 var-12 var-13)
     (dhseqr "S" "V" n ilo ihi a lda wr wi vl ldvl
      (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                            work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
    (declare
     (ignore var-0 var-1 var-5 var-7 var-8 var-9 var-11 var-12))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-10 (setf ldvl var-10))
    (when var-13 (setf info var-13)))
  (cond
   (wantvr (f2cl-lib:f2cl-set-string side "B" (string 1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "F" n n vl ldvl vr ldvr)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf ldvl var-4))
      (when var-6 (setf ldvr var-6))))))
 (wantvr (f2cl-lib:f2cl-set-string side "R" (string 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "L" n n a lda vr ldvr)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvr var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dorghr n ilo ihi vr ldvr
       (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf n var-0))
    (when var-1 (setf ilo var-1))
    (when var-2 (setf ihi var-2))
    (when var-4 (setf ldvr var-4))
    (when var-8 (setf ierr var-8)))
  (setf iwrk itau)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dhseqr "S" "V" n ilo ihi a lda wr wi vr ldvr
       (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
```

```
                              work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
    (declare
     (ignore var-0 var-1 var-5 var-7 var-8 var-9 var-11 var-12))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-10 (setf ldvr var-10))
    (when var-13 (setf info var-13))))
 (t (setf iwrk itau)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dhseqr "E" "N" n ilo ihi a lda wr wi vr ldvr
       (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                              work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
    (declare
     (ignore var-0 var-1 var-5 var-7 var-8 var-9 var-11 var-12))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-10 (setf ldvr var-10))
    (when var-13 (setf info var-13)))))
(if (> info 0)
    (go label50))
(cond
 ((or wantvl wantvr)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dtrevc side "B" select n a lda vl ldvl vr ldvr n nout
       (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                              work-%offset%)
       ierr)
    (declare (ignore var-1 var-2 var-4 var-6 var-8 var-12))
    (when var-0 (setf side var-0))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldvl var-7))
    (when var-9 (setf ldvr var-9))
    (when var-10 (setf n var-10))
    (when var-11 (setf nout var-11))
    (when var-13 (setf ierr var-13)))))
(cond
 (wantvl
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
```

```
      (dgebak "B" "L" n ilo ihi
       (f2cl-lib:array-slice work-%data% double-float (ibal) ((1 *))
                             work-%offset%)
       n vl ldvl ierr)
    (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7))
    (setf n var-6)
    (setf ldvl var-8)
    (setf ierr var-9))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref wi (i) ((1 *))) zero)
        (setf scl
                (/ one
                   (multiple-value-bind (ret-val var-0 var-1 var-2)
                       (dnrm2 n
                        (f2cl-lib:array-slice vl-%data% double-float
                                              (1 i) ((1 ldvl) (1 *))
                                              vl-%offset%)
                        1)
                     (declare (ignore var-1 var-2))
                     (when var-0 (setf n var-0))
                     ret-val)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal n scl
             (f2cl-lib:array-slice vl-%data% double-float (1 i)
                                   ((1 ldvl) (1 *)) vl-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-1 (setf scl var-1))))
       ((> (f2cl-lib:fref wi (i) ((1 *))) zero)
        (setf scl
                (/ one
                   (dlapy2
                    (multiple-value-bind (ret-val var-0 var-1 var-2)
                        (dnrm2 n
                         (f2cl-lib:array-slice vl-%data% double-float
                                               (1 i) ((1 ldvl) (1 *))
                                               vl-%offset%)
                         1)
                      (declare (ignore var-1 var-2))
                      (when var-0 (setf n var-0))
                      ret-val)
                    (multiple-value-bind (ret-val var-0 var-1 var-2)
                        (dnrm2 n
                         (f2cl-lib:array-slice vl-%data% double-float
                                               (1
                                                (f2cl-lib:int-add i 1))
```

```
                                        ((1 ldvl) (1 *))
                                        vl-%offset%)
                 1)
              (declare (ignore var-1 var-2))
              (when var-0 (setf n var-0))
              ret-val))))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (dscal n scl
     (f2cl-lib:array-slice vl-%data% double-float (1 i)
                           ((1 ldvl) (1 *)) vl-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf scl var-1)))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (dscal n scl
     (f2cl-lib:array-slice vl-%data% double-float
                           (1 (f2cl-lib:int-add i 1))
                           ((1 ldvl) (1 *)) vl-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf scl var-1)))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iwrk k) 1))
                         ((1 *)) work-%offset%)
            (+
             (expt
              (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                             vl-%offset%)
              2)
             (expt
              (f2cl-lib:fref vl-%data%
                             (k (f2cl-lib:int-add i 1))
                             ((1 ldvl) (1 *)) vl-%offset%)
              2)))
   label10))
(setf k
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (idamax n
             (f2cl-lib:array-slice work-%data% double-float
                                   (iwrk) ((1 *))
                                   work-%offset%)
             1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
```

```
                ret-val))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dlartg
             (f2cl-lib:array-slice vl-%data% double-float (k i)
                                   ((1 ldvl) (1 *)) vl-%offset%)
             (f2cl-lib:array-slice vl-%data% double-float
                                   (k (f2cl-lib:int-add i 1))
                                   ((1 ldvl) (1 *)) vl-%offset%)
             cs sn r)
          (declare (ignore var-0 var-1))
          (when var-2 (setf cs var-2))
          (when var-3 (setf sn var-3))
          (when var-4 (setf r var-4)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (drot n
             (f2cl-lib:array-slice vl-%data% double-float (1 i)
                                   ((1 ldvl) (1 *)) vl-%offset%)
             1
             (f2cl-lib:array-slice vl-%data% double-float
                                   (1 (f2cl-lib:int-add i 1))
                                   ((1 ldvl) (1 *)) vl-%offset%)
             1 cs sn)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf n var-0))
          (when var-5 (setf cs var-5))
          (when var-6 (setf sn var-6)))
        (setf (f2cl-lib:fref vl-%data% (k (f2cl-lib:int-add i 1))
                             ((1 ldvl) (1 *)) vl-%offset%)
              zero)))
    label20))))
(cond
 (wantvr
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dgebak "B" "R" n ilo ihi
       (f2cl-lib:array-slice work-%data% double-float (ibal) ((1 *))
                             work-%offset%)
       n vr ldvr ierr)
    (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7))
    (setf n var-6)
    (setf ldvr var-8)
    (setf ierr var-9))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref wi (i) ((1 *))) zero)
        (setf scl
                (/ one
```

```
                (multiple-value-bind (ret-val var-0 var-1 var-2)
                    (dnrm2 n
                     (f2cl-lib:array-slice vr-%data% double-float
                                           (1 i) ((1 ldvr) (1 *))
                                           vr-%offset%)
                     1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf n var-0))
                  ret-val)))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (dscal n scl
      (f2cl-lib:array-slice vr-%data% double-float (1 i)
                            ((1 ldvr) (1 *)) vr-%offset%)
      1)
   (declare (ignore var-2 var-3))
   (when var-0 (setf n var-0))
   (when var-1 (setf scl var-1))))
((> (f2cl-lib:fref wi (i) ((1 *))) zero)
 (setf scl
         (/ one
            (dlapy2
             (multiple-value-bind (ret-val var-0 var-1 var-2)
                 (dnrm2 n
                  (f2cl-lib:array-slice vr-%data% double-float
                                        (1 i) ((1 ldvr) (1 *))
                                        vr-%offset%)
                  1)
               (declare (ignore var-1 var-2))
               (when var-0 (setf n var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1 var-2)
                 (dnrm2 n
                  (f2cl-lib:array-slice vr-%data% double-float
                                        (1
                                         (f2cl-lib:int-add i 1))
                                        ((1 ldvr) (1 *))
                                        vr-%offset%)
                  1)
               (declare (ignore var-1 var-2))
               (when var-0 (setf n var-0))
               ret-val))))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (dscal n scl
      (f2cl-lib:array-slice vr-%data% double-float (1 i)
                            ((1 ldvr) (1 *)) vr-%offset%)
      1)
   (declare (ignore var-2 var-3))
   (when var-0 (setf n var-0))
   (when var-1 (setf scl var-1)))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
```

```
    (dscal n scl
     (f2cl-lib:array-slice vr-%data% double-float
                           (1 (f2cl-lib:int-add i 1))
                           ((1 ldvr) (1 *)) vr-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf scl var-1)))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add iwrk k) 1))
                         ((1 *)) work-%offset%)
          (+
           (expt
            (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                           vr-%offset%)
            2)
           (expt
            (f2cl-lib:fref vr-%data%
                           (k (f2cl-lib:int-add i 1))
                           ((1 ldvr) (1 *)) vr-%offset%)
            2)))
   label30))
(setf k
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (idamax n
             (f2cl-lib:array-slice work-%data% double-float
                                   (iwrk) ((1 *))
                                   work-%offset%)
             1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dlartg
     (f2cl-lib:array-slice vr-%data% double-float (k i)
                           ((1 ldvr) (1 *)) vr-%offset%)
     (f2cl-lib:array-slice vr-%data% double-float
                           (k (f2cl-lib:int-add i 1))
                           ((1 ldvr) (1 *)) vr-%offset%)
     cs sn r)
  (declare (ignore var-0 var-1))
  (when var-2 (setf cs var-2))
  (when var-3 (setf sn var-3))
  (when var-4 (setf r var-4)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
                (drot n
                 (f2cl-lib:array-slice vr-%data% double-float (1 i)
                                       ((1 ldvr) (1 *)) vr-%offset%)
                 1
                 (f2cl-lib:array-slice vr-%data% double-float
                                       (1 (f2cl-lib:int-add i 1))
                                       ((1 ldvr) (1 *)) vr-%offset%)
                 1 cs sn)
               (declare (ignore var-1 var-2 var-3 var-4))
               (when var-0 (setf n var-0))
               (when var-5 (setf cs var-5))
               (when var-6 (setf sn var-6)))
             (setf (f2cl-lib:fref vr-%data% (k (f2cl-lib:int-add i 1))
                                  ((1 ldvr) (1 *)) vr-%offset%)
                   zero)))
         label40))))
    label50
     (cond
      (scalea
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
           (dlascl "G" 0 0 cscale anrm (f2cl-lib:int-sub n info) 1
            (f2cl-lib:array-slice wr-%data% double-float ((+ info 1))
                                  ((1 *)) wr-%offset%)
            (max (the f2cl-lib:integer4 (f2cl-lib:int-sub n info))
                 (the f2cl-lib:integer4 1))
            ierr)
         (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
         (when var-3 (setf cscale var-3))
         (when var-4 (setf anrm var-4))
         (when var-9 (setf ierr var-9)))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
           (dlascl "G" 0 0 cscale anrm (f2cl-lib:int-sub n info) 1
            (f2cl-lib:array-slice wi-%data% double-float ((+ info 1))
                                  ((1 *)) wi-%offset%)
            (max (the f2cl-lib:integer4 (f2cl-lib:int-sub n info))
                 (the f2cl-lib:integer4 1))
            ierr)
         (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
         (when var-3 (setf cscale var-3))
         (when var-4 (setf anrm var-4))
         (when var-9 (setf ierr var-9)))
       (cond
        ((> info 0)
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
             (dlascl "G" 0 0 cscale anrm (f2cl-lib:int-sub ilo 1) 1 wr n
              ierr)
           (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
```

```
            (when var-3 (setf cscale var-3))
            (when var-4 (setf anrm var-4))
            (when var-8 (setf n var-8))
            (when var-9 (setf ierr var-9)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (dlascl "G" 0 0 cscale anrm (f2cl-lib:int-sub ilo 1) 1 wi n
               ierr)
            (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
            (when var-3 (setf cscale var-3))
            (when var-4 (setf anrm var-4))
            (when var-8 (setf n var-8))
            (when var-9 (setf ierr var-9)))))))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 maxwrk) 'double-float))
      (go end_label)
     end_label
      (return
       (values jobvl jobvr n nil lda nil nil nil ldvl nil ldvr nil nil
               info))))))
```

———-

90.121 dgeevx

90.121.1 Axiom unit tests

— **dgeevx.input** —

```
)set break resume
)sys rm -f dgeevx.output
)spool dgeevx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.121.2 Axiom help page

— **dgeevx.help** —

```
====================================================================
dgeevx examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DGEEVX - for an N-by-N real nonsymmetric matrix A, the eigenvalues and,
       optionally, the left and/or right eigenvectors

SYNOPSIS
       SUBROUTINE DGEEVX( BALANC, JOBVL, JOBVR, SENSE, N, A, LDA, WR, WI,  VL,
                          LDVL,  VR,  LDVR,  ILO,  IHI,  SCALE, ABNRM, RCONDE,
                          RCONDV, WORK, LWORK, IWORK, INFO )

           CHARACTER      BALANC, JOBVL, JOBVR, SENSE

           INTEGER        IHI, ILO, INFO, LDA, LDVL, LDVR, LWORK, N

           DOUBLE         PRECISION ABNRM

           INTEGER        IWORK( * )

           DOUBLE         PRECISION A( LDA, * ), RCONDE( *  ),  RCONDV(  *  ),
                          SCALE(  *  ), VL( LDVL, * ), VR( LDVR, * ), WI( * ),
                          WORK( * ), WR( * )

PURPOSE
       DGEEVX computes for an N-by-N real nonsymmetric matrix A, the eigenval-
       ues and, optionally, the left and/or right eigenvectors.

       Optionally  also, it computes a balancing transformation to improve the
       conditioning of the eigenvalues and eigenvectors (ILO, IHI, SCALE,  and
       ABNRM),  reciprocal condition numbers for the eigenvalues (RCONDE), and
       reciprocal condition numbers for the right
       eigenvectors (RCONDV).

       The right eigenvector v(j) of A satisfies
                        A * v(j) = lambda(j) * v(j)
       where lambda(j) is its eigenvalue.
       The left eigenvector u(j) of A satisfies
                     u(j)**H * A = lambda(j) * u(j)**H
       where u(j)**H denotes the conjugate transpose of u(j).

       The computed eigenvectors are normalized to have Euclidean  norm  equal
       to 1 and largest component real.

       Balancing a matrix means permuting the rows and columns to make it more
```

nearly upper triangular, and applying a diagonal similarity transformation D * A * D**(-1), where D is a diagonal matrix, to make its rows and columns closer in norm and the condition numbers of its eigenvalues and eigenvectors smaller. The computed reciprocal condition numbers correspond to the balanced matrix. Permuting rows and columns will not change the condition numbers (in exact arithmetic) but diagonal scaling will. For further explanation of balancing, see section 4.10.2 of the LAPACK Users' Guide.

ARGUMENTS

BALANC (input) CHARACTER*1
Indicates how the input matrix should be diagonally scaled and/or permuted to improve the conditioning of its eigenvalues.
= 'N': Do not diagonally scale or permute;
= 'P': Perform permutations to make the matrix more nearly upper triangular. Do not diagonally scale; = 'S': Diagonally scale the matrix, i.e. replace A by D*A*D**(-1), where D is a diagonal matrix chosen to make the rows and columns of A more equal in norm. Do not permute; = 'B': Both diagonally scale and permute A.

Computed reciprocal condition numbers will be for the matrix after balancing and/or permuting. Permuting does not change condition numbers (in exact arithmetic), but balancing does.

JOBVL (input) CHARACTER*1
= 'N': left eigenvectors of A are not computed;
= 'V': left eigenvectors of A are computed. If SENSE = 'E' or 'B', JOBVL must = 'V'.

JOBVR (input) CHARACTER*1
= 'N': right eigenvectors of A are not computed;
= 'V': right eigenvectors of A are computed. If SENSE = 'E' or 'B', JOBVR must = 'V'.

SENSE (input) CHARACTER*1
Determines which reciprocal condition numbers are computed. = 'N': None are computed;
= 'E': Computed for eigenvalues only;
= 'V': Computed for right eigenvectors only;
= 'B': Computed for eigenvalues and right eigenvectors.

If SENSE = 'E' or 'B', both left and right eigenvectors must also be computed (JOBVL = 'V' and JOBVR = 'V').

N (input) INTEGER
The order of the matrix A. N >= 0.

A (input/output) DOUBLE PRECISION array, dimension (LDA,N)

```
        On entry, the N-by-N matrix A.  On exit, A has  been  overwrit-
        ten.   If JOBVL = 'V' or JOBVR = 'V', A contains the real Schur
        form of the balanced version of the input matrix A.

LDA     (input) INTEGER
        The leading dimension of the array A.  LDA >= max(1,N).

WR      (output) DOUBLE PRECISION array, dimension (N)
        WI      (output) DOUBLE PRECISION array, dimension (N)  WR  and
        WI  contain  the real and imaginary parts, respectively, of the
        computed eigenvalues.  Complex conjugate pairs  of  eigenvalues
        will  appear consecutively with the eigenvalue having the posi-
        tive imaginary part first.

VL      (output) DOUBLE PRECISION array, dimension (LDVL,N)
        If JOBVL = 'V', the left eigenvectors u(j) are stored one after
        another in the columns of VL, in the same order as their eigen-
        values.  If JOBVL = 'N', VL is not  referenced.   If  the  j-th
        eigenvalue is real, then u(j) = VL(:,j), the j-th column of VL.
        If the j-th and (j+1)-st eigenvalues form a  complex  conjugate
        pair, then u(j) = VL(:,j) + i*VL(:,j+1) and
        u(j+1) = VL(:,j) - i*VL(:,j+1).

LDVL    (input) INTEGER
        The  leading  dimension of the array VL.  LDVL >= 1; if JOBVL =
        'V', LDVL >= N.

VR      (output) DOUBLE PRECISION array, dimension (LDVR,N)
        If JOBVR = 'V', the right  eigenvectors  v(j)  are  stored  one
        after  another in the columns of VR, in the same order as their
        eigenvalues.  If JOBVR = 'N', VR is not referenced.  If the  j-
        th  eigenvalue is real, then v(j) = VR(:,j), the j-th column of
        VR.  If the j-th and (j+1)-st eigenvalues form a complex conju-
        gate pair, then v(j) = VR(:,j) + i*VR(:,j+1) and
        v(j+1) = VR(:,j) - i*VR(:,j+1).

LDVR    (input) INTEGER
        The leading dimension of the array VR.  LDVR >= 1, and if JOBVR
        = 'V', LDVR >= N.

ILO     (output) INTEGER
        IHI     (output) INTEGER ILO and IHI are integer values  deter-
        mined  when  A  was balanced.  The balanced A(i,j) = 0 if I > J
        and J = 1,...,ILO-1 or I = IHI+1,...,N.

SCALE   (output) DOUBLE PRECISION array, dimension (N)
        Details of the permutations and scaling  factors  applied  when
        balancing A.  If P(j) is the index of the row and column inter-
        changed with row and column j, and D(j) is the  scaling  factor
        applied  to  row and column j, then SCALE(J) = P(J),    for J =
```

```
        1,...,ILO-1 = D(J),    for J = ILO,...,IHI = P(J)     for  J  =
        IHI+1,...,N.  The order in which the interchanges are made is N
        to IHI+1, then 1 to ILO-1.

ABNRM   (output) DOUBLE PRECISION
        The one-norm of the balanced matrix (the maximum of the sum  of
        absolute values of elements of any column).

RCONDE  (output) DOUBLE PRECISION array, dimension (N)
        RCONDE(j) is the reciprocal condition number of the j-th eigen-
        value.

RCONDV  (output) DOUBLE PRECISION array, dimension (N)
        RCONDV(j) is the reciprocal condition number of the j-th  right
        eigenvector.

WORK       (workspace/output)   DOUBLE   PRECISION   array,   dimension
(MAX(1,LWORK))
        On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK   (input) INTEGER
        The dimension of the array WORK.   If SENSE = 'N' or 'E', LWORK
        >= max(1,2*N), and if JOBVL = 'V' or JOBVR = 'V', LWORK >= 3*N.
        If SENSE = 'V' or 'B', LWORK >= N*(N+6).  For good performance,
        LWORK must generally be larger.

        If LWORK = -1, then a workspace query is assumed;  the  routine
        only  calculates  the  optimal  size of the WORK array, returns
        this value as the first entry of the WORK array, and  no  error
        message related to LWORK is issued by XERBLA.

IWORK   (workspace) INTEGER array, dimension (2*N-2)
        If SENSE = 'N' or 'E', not referenced.

INFO    (output) INTEGER
        = 0:  successful exit
        < 0:  if INFO = -i, the i-th argument had an illegal value.
        >  0:   if INFO = i, the QR algorithm failed to compute all the
        eigenvalues, and no eigenvectors or condition numbers have been
        computed;  elements  1:ILO-1  and  i+1:N  of  WR and WI contain
        eigenvalues which have converged.
```

———-

90.121.3 fortran code

— dgeevx.f —

```
*  =====================================================================
      SUBROUTINE DGEEVX( BALANC, JOBVL, JOBVR, SENSE, N, A, LDA, WR, WI,
     $                   VL, LDVL, VR, LDVR, ILO, IHI, SCALE, ABNRM,
     $                   RCONDE, RCONDV, WORK, LWORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          BALANC, JOBVL, JOBVR, SENSE
      INTEGER            IHI, ILO, INFO, LDA, LDVL, LDVR, LWORK, N
      DOUBLE PRECISION   ABNRM
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), RCONDE( * ), RCONDV( * ),
     $                   SCALE( * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   WI( * ), WORK( * ), WR( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, SCALEA, WANTVL, WANTVR, WNTSNB, WNTSNE,
     $                   WNTSNN, WNTSNV
      CHARACTER          JOB, SIDE
      INTEGER            HSWORK, I, ICOND, IERR, ITAU, IWRK, K, MAXWRK,
     $                   MINWRK, NOUT
      DOUBLE PRECISION   ANRM, BIGNUM, CS, CSCALE, EPS, R, SCL, SMLNUM,
     $                   SN
*     ..
*     .. Local Arrays ..
      LOGICAL            SELECT( 1 )
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEBAK, DGEBAL, DGEHRD, DHSEQR, DLABAD, DLACPY,
     $                   DLARTG, DLASCL, DORGHR, DROT, DSCAL, DTREVC,
     $                   DTRSNA, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX, ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE, DLAPY2, DNRM2
```

```
      EXTERNAL           LSAME, IDAMAX, ILAENV, DLAMCH, DLANGE, DLAPY2,
     $                   DNRM2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      WANTVL = LSAME( JOBVL, 'V' )
      WANTVR = LSAME( JOBVR, 'V' )
      WNTSNN = LSAME( SENSE, 'N' )
      WNTSNE = LSAME( SENSE, 'E' )
      WNTSNV = LSAME( SENSE, 'V' )
      WNTSNB = LSAME( SENSE, 'B' )
      IF( .NOT.( LSAME( BALANC, 'N' ) .OR. LSAME( BALANC,
     $    'S' ) .OR. LSAME( BALANC, 'P' ) .OR. LSAME( BALANC, 'B' ) ) )
     $     THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTVL ) .AND. ( .NOT.LSAME( JOBVL, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( ( .NOT.WANTVR ) .AND. ( .NOT.LSAME( JOBVR, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( .NOT.( WNTSNN .OR. WNTSNE .OR. WNTSNB .OR. WNTSNV ) .OR.
     $         ( ( WNTSNE .OR. WNTSNB ) .AND. .NOT.( WANTVL .AND.
     $         WANTVR ) ) ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDVL.LT.1 .OR. ( WANTVL .AND. LDVL.LT.N ) ) THEN
         INFO = -11
      ELSE IF( LDVR.LT.1 .OR. ( WANTVR .AND. LDVR.LT.N ) ) THEN
         INFO = -13
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by DHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.)
*
```

```
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            MAXWRK = 1
         ELSE
            MAXWRK = N + N*ILAENV( 1, 'DGEHRD', ' ', N, 1, N, 0 )
*
            IF( WANTVL ) THEN
               CALL DHSEQR( 'S', 'V', N, 1, N, A, LDA, WR, WI, VL, LDVL,
     $                 WORK, -1, INFO )
            ELSE IF( WANTVR ) THEN
               CALL DHSEQR( 'S', 'V', N, 1, N, A, LDA, WR, WI, VR, LDVR,
     $                 WORK, -1, INFO )
            ELSE
               IF( WNTSNN ) THEN
                  CALL DHSEQR( 'E', 'N', N, 1, N, A, LDA, WR, WI, VR,
     $                 LDVR, WORK, -1, INFO )
               ELSE
                  CALL DHSEQR( 'S', 'N', N, 1, N, A, LDA, WR, WI, VR,
     $                 LDVR, WORK, -1, INFO )
               END IF
            END IF
            HSWORK = WORK( 1 )
*
            IF( ( .NOT.WANTVL ) .AND. ( .NOT.WANTVR ) ) THEN
               MINWRK = 2*N
               IF( .NOT.WNTSNN )
     $            MINWRK = MAX( MINWRK, N*N+6*N )
               MAXWRK = MAX( MAXWRK, HSWORK )
               IF( .NOT.WNTSNN )
     $            MAXWRK = MAX( MAXWRK, N*N + 6*N )
            ELSE
               MINWRK = 3*N
               IF( ( .NOT.WNTSNN ) .AND. ( .NOT.WNTSNE ) )
     $            MINWRK = MAX( MINWRK, N*N + 6*N )
               MAXWRK = MAX( MAXWRK, HSWORK )
               MAXWRK = MAX( MAXWRK, N + ( N - 1 )*ILAENV( 1, 'DORGHR',
     $                        ' ', N, 1, N, -1 ) )
               IF( ( .NOT.WNTSNN ) .AND. ( .NOT.WNTSNE ) )
     $            MAXWRK = MAX( MAXWRK, N*N + 6*N )
               MAXWRK = MAX( MAXWRK, 3*N )
            END IF
            MAXWRK = MAX( MAXWRK, MINWRK )
         END IF
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
            INFO = -21
         END IF
      END IF
```

```
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEEVX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ICOND = 0
      ANRM = DLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
     $   CALL DLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*     Balance the matrix and compute ABNRM
*
      CALL DGEBAL( BALANC, N, A, LDA, ILO, IHI, SCALE, IERR )
      ABNRM = DLANGE( '1', N, N, A, LDA, DUM )
      IF( SCALEA ) THEN
         DUM( 1 ) = ABNRM
         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, 1, 1, DUM, 1, IERR )
         ABNRM = DUM( 1 )
      END IF
*
*     Reduce to upper Hessenberg form
*     (Workspace: need 2*N, prefer N+N*NB)
*
```

```
      ITAU = 1
      IWRK = ITAU + N
      CALL DGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
      IF( WANTVL ) THEN
*
*        Want left eigenvectors
*        Copy Householder vectors to VL
*
         SIDE = 'L'
         CALL DLACPY( 'L', N, N, A, LDA, VL, LDVL )
*
*        Generate orthogonal matrix in VL
*        (Workspace: need 2*N-1, prefer N+(N-1)*NB)
*
         CALL DORGHR( N, ILO, IHI, VL, LDVL, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VL
*        (Workspace: need 1, prefer HSWORK (see comments) )
*
         IWRK = ITAU
         CALL DHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, WR, WI, VL, LDVL,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
         IF( WANTVR ) THEN
*
*           Want left and right eigenvectors
*           Copy Schur vectors to VR
*
            SIDE = 'B'
            CALL DLACPY( 'F', N, N, VL, LDVL, VR, LDVR )
         END IF
*
      ELSE IF( WANTVR ) THEN
*
*        Want right eigenvectors
*        Copy Householder vectors to VR
*
         SIDE = 'R'
         CALL DLACPY( 'L', N, N, A, LDA, VR, LDVR )
*
*        Generate orthogonal matrix in VR
*        (Workspace: need 2*N-1, prefer N+(N-1)*NB)
*
         CALL DORGHR( N, ILO, IHI, VR, LDVR, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VR
```

```
*        (Workspace: need 1, prefer HSWORK (see comments) )
*
         IWRK = ITAU
         CALL DHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, WR, WI, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
      ELSE
*
*        Compute eigenvalues only
*        If condition numbers desired, compute Schur form
*
         IF( WNTSNN ) THEN
            JOB = 'E'
         ELSE
            JOB = 'S'
         END IF
*
*        (Workspace: need 1, prefer HSWORK (see comments) )
*
         IWRK = ITAU
         CALL DHSEQR( JOB, 'N', N, ILO, IHI, A, LDA, WR, WI, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
      END IF
*
*     If INFO > 0 from DHSEQR, then quit
*
      IF( INFO.GT.0 )
     $   GO TO 50
*
      IF( WANTVL .OR. WANTVR ) THEN
*
*        Compute left and/or right eigenvectors
*        (Workspace: need 3*N)
*
         CALL DTREVC( SIDE, 'B', SELECT, N, A, LDA, VL, LDVL, VR, LDVR,
     $                N, NOUT, WORK( IWRK ), IERR )
      END IF
*
*     Compute condition numbers if desired
*     (Workspace: need N*N+6*N unless SENSE = 'E')
*
      IF( .NOT.WNTSNN ) THEN
         CALL DTRSNA( SENSE, 'A', SELECT, N, A, LDA, VL, LDVL, VR, LDVR,
     $                RCONDE, RCONDV, N, NOUT, WORK( IWRK ), N, IWORK,
     $                ICOND )
      END IF
*
      IF( WANTVL ) THEN
*
*        Undo balancing of left eigenvectors
```

```
*
         CALL DGEBAK( BALANC, 'L', N, ILO, IHI, SCALE, N, VL, LDVL,
     $                IERR )
*
*        Normalize left eigenvectors and make largest component real
*
         DO 20 I = 1, N
            IF( WI( I ).EQ.ZERO ) THEN
               SCL = ONE / DNRM2( N, VL( 1, I ), 1 )
               CALL DSCAL( N, SCL, VL( 1, I ), 1 )
            ELSE IF( WI( I ).GT.ZERO ) THEN
               SCL = ONE / DLAPY2( DNRM2( N, VL( 1, I ), 1 ),
     $               DNRM2( N, VL( 1, I+1 ), 1 ) )
               CALL DSCAL( N, SCL, VL( 1, I ), 1 )
               CALL DSCAL( N, SCL, VL( 1, I+1 ), 1 )
               DO 10 K = 1, N
                  WORK( K ) = VL( K, I )**2 + VL( K, I+1 )**2
   10          CONTINUE
               K = IDAMAX( N, WORK, 1 )
               CALL DLARTG( VL( K, I ), VL( K, I+1 ), CS, SN, R )
               CALL DROT( N, VL( 1, I ), 1, VL( 1, I+1 ), 1, CS, SN )
               VL( K, I+1 ) = ZERO
            END IF
   20    CONTINUE
      END IF
*
      IF( WANTVR ) THEN
*
*        Undo balancing of right eigenvectors
*
         CALL DGEBAK( BALANC, 'R', N, ILO, IHI, SCALE, N, VR, LDVR,
     $                IERR )
*
*        Normalize right eigenvectors and make largest component real
*
         DO 40 I = 1, N
            IF( WI( I ).EQ.ZERO ) THEN
               SCL = ONE / DNRM2( N, VR( 1, I ), 1 )
               CALL DSCAL( N, SCL, VR( 1, I ), 1 )
            ELSE IF( WI( I ).GT.ZERO ) THEN
               SCL = ONE / DLAPY2( DNRM2( N, VR( 1, I ), 1 ),
     $               DNRM2( N, VR( 1, I+1 ), 1 ) )
               CALL DSCAL( N, SCL, VR( 1, I ), 1 )
               CALL DSCAL( N, SCL, VR( 1, I+1 ), 1 )
               DO 30 K = 1, N
                  WORK( K ) = VR( K, I )**2 + VR( K, I+1 )**2
   30          CONTINUE
               K = IDAMAX( N, WORK, 1 )
               CALL DLARTG( VR( K, I ), VR( K, I+1 ), CS, SN, R )
               CALL DROT( N, VR( 1, I ), 1, VR( 1, I+1 ), 1, CS, SN )
```

```
               VR( K, I+1 ) = ZERO
            END IF
   40    CONTINUE
      END IF
*
*     Undo scaling if necessary
*
   50 CONTINUE
      IF( SCALEA ) THEN
         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, WR( INFO+1 ),
     $                MAX( N-INFO, 1 ), IERR )
         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, WI( INFO+1 ),
     $                MAX( N-INFO, 1 ), IERR )
         IF( INFO.EQ.0 ) THEN
            IF( ( WNTSNV .OR. WNTSNB ) .AND. ICOND.EQ.0 )
     $         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N, 1, RCONDV, N,
     $                      IERR )
         ELSE
            CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WR, N,
     $                   IERR )
            CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WI, N,
     $                   IERR )
         END IF
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of DGEEVX
*
      END
      SUBROUTINE DGEEVX( BALANC, JOBVL, JOBVR, SENSE, N, A, LDA, WR, WI,
     $                   VL, LDVL, VR, LDVR, ILO, IHI, SCALE, ABNRM,
     $                   RCONDE, RCONDV, WORK, LWORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      CHARACTER          BALANC, JOBVL, JOBVR, SENSE
      INTEGER            IHI, ILO, INFO, LDA, LDVL, LDVR, LWORK, N
      DOUBLE PRECISION   ABNRM
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), RCONDE( * ), RCONDV( * ),
     $                   SCALE( * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   WI( * ), WORK( * ), WR( * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, SCALEA, WANTVL, WANTVR, WNTSNB, WNTSNE,
     $                   WNTSNN, WNTSNV
      CHARACTER          JOB, SIDE
      INTEGER            HSWORK, I, ICOND, IERR, ITAU, IWRK, K, MAXB,
     $                   MAXWRK, MINWRK, NOUT
      DOUBLE PRECISION   ANRM, BIGNUM, CS, CSCALE, EPS, R, SCL, SMLNUM,
     $                   SN
*     ..
*     .. Local Arrays ..
      LOGICAL            SELECT( 1 )
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEBAK, DGEBAL, DGEHRD, DHSEQR, DLACPY, DLARTG,
     $                   DLASCL, DORGHR, DROT, DSCAL, DTREVC, DTRSNA,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX, ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE, DLAPY2, DNRM2
      EXTERNAL           LSAME, IDAMAX, ILAENV, DLAMCH, DLANGE, DLAPY2,
     $                   DNRM2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      WANTVL = LSAME( JOBVL, 'V' )
      WANTVR = LSAME( JOBVR, 'V' )
      WNTSNN = LSAME( SENSE, 'N' )
      WNTSNE = LSAME( SENSE, 'E' )
      WNTSNV = LSAME( SENSE, 'V' )
      WNTSNB = LSAME( SENSE, 'B' )
      IF( .NOT.( LSAME( BALANC, 'N' ) .OR. LSAME( BALANC,
     $    'S' ) .OR. LSAME( BALANC, 'P' ) .OR. LSAME( BALANC, 'B' ) ) )
```

```
     $     THEN
         INFO = -1
      ELSE IF( ( .NOT.WANTVL ) .AND. ( .NOT.LSAME( JOBVL, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( ( .NOT.WANTVR ) .AND. ( .NOT.LSAME( JOBVR, 'N' ) ) ) THEN
         INFO = -3
      ELSE IF( .NOT.( WNTSNN .OR. WNTSNE .OR. WNTSNB .OR. WNTSNV ) .OR.
     $         ( ( WNTSNE .OR. WNTSNB ) .AND. .NOT.( WANTVL .AND.
     $         WANTVR ) ) ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDVL.LT.1 .OR. ( WANTVL .AND. LDVL.LT.N ) ) THEN
         INFO = -11
      ELSE IF( LDVR.LT.1 .OR. ( WANTVR .AND. LDVR.LT.N ) ) THEN
         INFO = -13
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by DHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.)
*
      MINWRK = 1
      IF( INFO.EQ.0 .AND. ( LWORK.GE.1 .OR. LQUERY ) ) THEN
         MAXWRK = N + N*ILAENV( 1, 'DGEHRD', ' ', N, 1, N, 0 )
         IF( ( .NOT.WANTVL ) .AND. ( .NOT.WANTVR ) ) THEN
            MINWRK = MAX( 1, 2*N )
            IF( .NOT.WNTSNN )
     $         MINWRK = MAX( MINWRK, N*N+6*N )
            MAXB = MAX( ILAENV( 8, 'DHSEQR', 'SN', N, 1, N, -1 ), 2 )
            IF( WNTSNN ) THEN
               K = MIN( MAXB, N, MAX( 2, ILAENV( 4, 'DHSEQR', 'EN', N,
     $             1, N, -1 ) ) )
            ELSE
               K = MIN( MAXB, N, MAX( 2, ILAENV( 4, 'DHSEQR', 'SN', N,
     $             1, N, -1 ) ) )
            END IF
            HSWORK = MAX( K*( K+2 ), 2*N )
            MAXWRK = MAX( MAXWRK, 1, HSWORK )
            IF( .NOT.WNTSNN )
     $         MAXWRK = MAX( MAXWRK, N*N+6*N )
         ELSE
```

```
            MINWRK = MAX( 1, 3*N )
            IF( ( .NOT.WNTSNN ) .AND. ( .NOT.WNTSNE ) )
     $         MINWRK = MAX( MINWRK, N*N+6*N )
            MAXB = MAX( ILAENV( 8, 'DHSEQR', 'SN', N, 1, N, -1 ), 2 )
            K = MIN( MAXB, N, MAX( 2, ILAENV( 4, 'DHSEQR', 'EN', N, 1,
     $          N, -1 ) ) )
            HSWORK = MAX( K*( K+2 ), 2*N )
            MAXWRK = MAX( MAXWRK, 1, HSWORK )
            MAXWRK = MAX( MAXWRK, N+( N-1 )*
     $               ILAENV( 1, 'DORGHR', ' ', N, 1, N, -1 ) )
            IF( ( .NOT.WNTSNN ) .AND. ( .NOT.WNTSNE ) )
     $         MAXWRK = MAX( MAXWRK, N*N+6*N )
            MAXWRK = MAX( MAXWRK, 3*N, 1 )
         END IF
         WORK( 1 ) = MAXWRK
      END IF
      IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
         INFO = -21
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEEVX', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ICOND = 0
      ANRM = DLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
```

```
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
     $   CALL DLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*     Balance the matrix and compute ABNRM
*
      CALL DGEBAL( BALANC, N, A, LDA, ILO, IHI, SCALE, IERR )
      ABNRM = DLANGE( '1', N, N, A, LDA, DUM )
      IF( SCALEA ) THEN
         DUM( 1 ) = ABNRM
         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, 1, 1, DUM, 1, IERR )
         ABNRM = DUM( 1 )
      END IF
*
*     Reduce to upper Hessenberg form
*     (Workspace: need 2*N, prefer N+N*NB)
*
      ITAU = 1
      IWRK = ITAU + N
      CALL DGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
      IF( WANTVL ) THEN
*
*        Want left eigenvectors
*        Copy Householder vectors to VL
*
         SIDE = 'L'
         CALL DLACPY( 'L', N, N, A, LDA, VL, LDVL )
*
*        Generate orthogonal matrix in VL
*        (Workspace: need 2*N-1, prefer N+(N-1)*NB)
*
         CALL DORGHR( N, ILO, IHI, VL, LDVL, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VL
*        (Workspace: need 1, prefer HSWORK (see comments) )
*
         IWRK = ITAU
         CALL DHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, WR, WI, VL, LDVL,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
         IF( WANTVR ) THEN
*
*           Want left and right eigenvectors
*           Copy Schur vectors to VR
*
            SIDE = 'B'
```

```
            CALL DLACPY( 'F', N, N, VL, LDVL, VR, LDVR )
         END IF
*
      ELSE IF( WANTVR ) THEN
*
*        Want right eigenvectors
*        Copy Householder vectors to VR
*
         SIDE = 'R'
         CALL DLACPY( 'L', N, N, A, LDA, VR, LDVR )
*
*        Generate orthogonal matrix in VR
*        (Workspace: need 2*N-1, prefer N+(N-1)*NB)
*
         CALL DORGHR( N, ILO, IHI, VR, LDVR, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VR
*        (Workspace: need 1, prefer HSWORK (see comments) )
*
         IWRK = ITAU
         CALL DHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, WR, WI, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
      ELSE
*
*        Compute eigenvalues only
*        If condition numbers desired, compute Schur form
*
         IF( WNTSNN ) THEN
            JOB = 'E'
         ELSE
            JOB = 'S'
         END IF
*
*        (Workspace: need 1, prefer HSWORK (see comments) )
*
         IWRK = ITAU
         CALL DHSEQR( JOB, 'N', N, ILO, IHI, A, LDA, WR, WI, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
      END IF
*
*     If INFO > 0 from DHSEQR, then quit
*
      IF( INFO.GT.0 )
     $   GO TO 50
*
      IF( WANTVL .OR. WANTVR ) THEN
*
*        Compute left and/or right eigenvectors
```

```
*        (Workspace: need 3*N)
*
         CALL DTREVC( SIDE, 'B', SELECT, N, A, LDA, VL, LDVL, VR, LDVR,
     $                N, NOUT, WORK( IWRK ), IERR )
      END IF
*
*     Compute condition numbers if desired
*     (Workspace: need N*N+6*N unless SENSE = 'E')
*
      IF( .NOT.WNTSNN ) THEN
         CALL DTRSNA( SENSE, 'A', SELECT, N, A, LDA, VL, LDVL, VR, LDVR,
     $                RCONDE, RCONDV, N, NOUT, WORK( IWRK ), N, IWORK,
     $                ICOND )
      END IF
*
      IF( WANTVL ) THEN
*
*        Undo balancing of left eigenvectors
*
         CALL DGEBAK( BALANC, 'L', N, ILO, IHI, SCALE, N, VL, LDVL,
     $                IERR )
*
*        Normalize left eigenvectors and make largest component real
*
         DO 20 I = 1, N
            IF( WI( I ).EQ.ZERO ) THEN
               SCL = ONE / DNRM2( N, VL( 1, I ), 1 )
               CALL DSCAL( N, SCL, VL( 1, I ), 1 )
            ELSE IF( WI( I ).GT.ZERO ) THEN
               SCL = ONE / DLAPY2( DNRM2( N, VL( 1, I ), 1 ),
     $               DNRM2( N, VL( 1, I+1 ), 1 ) )
               CALL DSCAL( N, SCL, VL( 1, I ), 1 )
               CALL DSCAL( N, SCL, VL( 1, I+1 ), 1 )
               DO 10 K = 1, N
                  WORK( K ) = VL( K, I )**2 + VL( K, I+1 )**2
   10          CONTINUE
               K = IDAMAX( N, WORK, 1 )
               CALL DLARTG( VL( K, I ), VL( K, I+1 ), CS, SN, R )
               CALL DROT( N, VL( 1, I ), 1, VL( 1, I+1 ), 1, CS, SN )
               VL( K, I+1 ) = ZERO
            END IF
   20    CONTINUE
      END IF
*
      IF( WANTVR ) THEN
*
*        Undo balancing of right eigenvectors
*
         CALL DGEBAK( BALANC, 'R', N, ILO, IHI, SCALE, N, VR, LDVR,
     $                IERR )
```

```
*
*       Normalize right eigenvectors and make largest component real
*
        DO 40 I = 1, N
           IF( WI( I ).EQ.ZERO ) THEN
              SCL = ONE / DNRM2( N, VR( 1, I ), 1 )
              CALL DSCAL( N, SCL, VR( 1, I ), 1 )
           ELSE IF( WI( I ).GT.ZERO ) THEN
              SCL = ONE / DLAPY2( DNRM2( N, VR( 1, I ), 1 ),
     $              DNRM2( N, VR( 1, I+1 ), 1 ) )
              CALL DSCAL( N, SCL, VR( 1, I ), 1 )
              CALL DSCAL( N, SCL, VR( 1, I+1 ), 1 )
              DO 30 K = 1, N
                 WORK( K ) = VR( K, I )**2 + VR( K, I+1 )**2
   30         CONTINUE
              K = IDAMAX( N, WORK, 1 )
              CALL DLARTG( VR( K, I ), VR( K, I+1 ), CS, SN, R )
              CALL DROT( N, VR( 1, I ), 1, VR( 1, I+1 ), 1, CS, SN )
              VR( K, I+1 ) = ZERO
           END IF
   40   CONTINUE
      END IF
*
*     Undo scaling if necessary
*
   50 CONTINUE
      IF( SCALEA ) THEN
         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, WR( INFO+1 ),
     $               MAX( N-INFO, 1 ), IERR )
         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, WI( INFO+1 ),
     $               MAX( N-INFO, 1 ), IERR )
         IF( INFO.EQ.0 ) THEN
            IF( ( WNTSNV .OR. WNTSNB ) .AND. ICOND.EQ.0 )
     $         CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, N, 1, RCONDV, N,
     $                      IERR )
         ELSE
            CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WR, N,
     $                   IERR )
            CALL DLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, WI, N,
     $                   IERR )
         END IF
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
*
*     End of DGEEVX
*
      END
```

———-

90.121.4 lisp code

— **dgeevx.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dgeevx
         (balanc jobvl jobvr sense n a lda wr wi vl ldvl vr ldvr ilo ihi scale
          abnrm rconde rcondv work lwork iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (double-float) abnrm)
             (type (array double-float (*)) work rcondv rconde scale vr vl wi
              wr a)
             (type (f2cl-lib:integer4) info lwork ihi ilo ldvr ldvl lda n)
             (type (string *) sense jobvr jobvl balanc))
    (f2cl-lib:with-multi-array-data
        ((balanc character balanc-%data% balanc-%offset%)
         (jobvl character jobvl-%data% jobvl-%offset%)
         (jobvr character jobvr-%data% jobvr-%offset%)
         (sense character sense-%data% sense-%offset%)
         (a double-float a-%data% a-%offset%)
         (wr double-float wr-%data% wr-%offset%)
         (wi double-float wi-%data% wi-%offset%)
         (vl double-float vl-%data% vl-%offset%)
         (vr double-float vr-%data% vr-%offset%)
         (scale double-float scale-%data% scale-%offset%)
         (rconde double-float rconde-%data% rconde-%offset%)
         (rcondv double-float rcondv-%data% rcondv-%offset%)
         (work double-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((dum (make-array 1 :element-type 'double-float))
             (select (make-array 1 :element-type 't)) (anrm 0.0d0)
             (bignum 0.0d0) (cs 0.0d0) (cscale 0.0d0) (eps 0.0d0) (r 0.0d0)
             (scl 0.0d0) (smlnum 0.0d0) (sn 0.0d0) (hswork 0) (i 0) (icond 0)
             (ierr 0) (itau 0) (iwrk 0) (k 0) (maxwrk 0) (minwrk 0) (nout 0)
             (job
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (side
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (lquery nil) (scalea nil) (wantvl nil) (wantvr nil) (wntsnb nil)
             (wntsne nil) (wntsnn nil) (wntsnv nil))
        (declare (type (array double-float (1)) dum)
                 (type (array f2cl-lib:logical (1)) select)
                 (type (double-float) anrm bignum cs cscale eps r scl smlnum
```

```
          sn)
         (type (f2cl-lib:integer4) hswork i icond ierr itau iwrk k
          maxwrk minwrk nout)
         (type (string 1) job side)
         (type f2cl-lib:logical lquery scalea wantvl wantvr wntsnb
          wntsne wntsnn wntsnv))
(setf info 0)
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(setf wantvl
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvl "V")
          (declare (ignore var-1))
          (when var-0 (setf jobvl var-0))
          ret-val))
(setf wantvr
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvr "V")
          (declare (ignore var-1))
          (when var-0 (setf jobvr var-0))
          ret-val))
(setf wntsnn
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "N")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wntsne
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "E")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wntsnv
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "V")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(setf wntsnb
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame sense "B")
          (declare (ignore var-1))
          (when var-0 (setf sense var-0))
          ret-val))
(cond
 ((not
   (or
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "N")
      (declare (ignore var-1))
```

```
      (when var-0 (setf balanc var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "S")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "P")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame balanc "B")
      (declare (ignore var-1))
      (when var-0 (setf balanc var-0))
      ret-val)))
  (setf info -1))
 ((and (not wantvl)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvl "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvl var-0))
          ret-val)))
  (setf info -2))
 ((and (not wantvr)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvr "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvr var-0))
          ret-val)))
  (setf info -3))
 ((or (not (or wntsnn wntsne wntsnb wntsnv))
      (and (or wntsne wntsnb) (not (and wantvl wantvr))))
  (setf info -4))
 ((< n 0) (setf info -5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7))
 ((or (< ldvl 1) (and wantvl (< ldvl n))) (setf info -11))
 ((or (< ldvr 1) (and wantvr (< ldvr n))) (setf info -13)))
(cond
 ((= info 0)
  (cond ((= n 0) (setf minwrk 1) (setf maxwrk 1))
        (t
         (setf maxwrk
                 (f2cl-lib:int-add n
                                   (f2cl-lib:int-mul n
                                                     (multiple-value-bind
```

```
                                                        (ret-val var-0
                                                         var-1 var-2
                                                         var-3 var-4
                                                         var-5 var-6)
                                                        (ilaenv 1
                                                         "DGEHRD" " "
                                                         n 1 n 0)
                                                      (declare
                                                       (ignore var-0
                                                        var-1 var-2
                                                        var-4 var-6))
                                                      (when var-3
                                                        (setf n
                                                                var-3))
                                                      (when var-5
                                                        (setf n
                                                                var-5))
                                                      ret-val))))
                                  (cond
                                   (wantvl
                                    (multiple-value-bind
                                        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                                         var-9 var-10 var-11 var-12 var-13)
                                        (dhseqr "S" "V" n 1 n a lda wr wi vl ldvl work -1 info)
                                      (declare
                                       (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-9 var-11
                                        var-12))
                                      (when var-2 (setf n var-2))
                                      (when var-4 (setf n var-4))
                                      (when var-6 (setf lda var-6))
                                      (when var-10 (setf ldvl var-10))
                                      (when var-13 (setf info var-13))))
                                   (wantvr
                                    (multiple-value-bind
                                        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                                         var-9 var-10 var-11 var-12 var-13)
                                        (dhseqr "S" "V" n 1 n a lda wr wi vr ldvr work -1 info)
                                      (declare
                                       (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-9 var-11
                                        var-12))
                                      (when var-2 (setf n var-2))
                                      (when var-4 (setf n var-4))
                                      (when var-6 (setf lda var-6))
                                      (when var-10 (setf ldvr var-10))
                                      (when var-13 (setf info var-13))))
                                   (t
                                    (cond
                                     (wntsnn
                                      (multiple-value-bind
                                          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
          var-9 var-10 var-11 var-12 var-13)
         (dhseqr "E" "N" n 1 n a lda wr wi vr ldvr work -1
          info)
       (declare
        (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-9
         var-11 var-12))
       (when var-2 (setf n var-2))
       (when var-4 (setf n var-4))
       (when var-6 (setf lda var-6))
       (when var-10 (setf ldvr var-10))
       (when var-13 (setf info var-13))))
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
          var-9 var-10 var-11 var-12 var-13)
         (dhseqr "S" "N" n 1 n a lda wr wi vr ldvr work -1
          info)
       (declare
        (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-9
         var-11 var-12))
       (when var-2 (setf n var-2))
       (when var-4 (setf n var-4))
       (when var-6 (setf lda var-6))
       (when var-10 (setf ldvr var-10))
       (when var-13 (setf info var-13)))))))
(setf hswork
        (f2cl-lib:int
         (f2cl-lib:fref work-%data% (1) ((1 *))
                        work-%offset%)))
(cond
 ((and (not wantvl) (not wantvr))
  (setf minwrk (f2cl-lib:int-mul 2 n))
  (if (not wntsnn)
      (setf minwrk
              (max (the f2cl-lib:integer4 minwrk)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add
                         (f2cl-lib:int-mul n n)
                         (f2cl-lib:int-mul 6 n))))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4 hswork)))
  (if (not wntsnn)
      (setf maxwrk
              (max (the f2cl-lib:integer4 maxwrk)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add
                         (f2cl-lib:int-mul n n)
                         (f2cl-lib:int-mul 6 n)))))))
 (t (setf minwrk (f2cl-lib:int-mul 3 n))
```

```
                (if (and (not wntsnn) (not wntsne))
                    (setf minwrk
                            (max (the f2cl-lib:integer4 minwrk)
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add
                                       (f2cl-lib:int-mul n n)
                                       (f2cl-lib:int-mul 6 n))))))
                (setf maxwrk
                        (max (the f2cl-lib:integer4 maxwrk)
                             (the f2cl-lib:integer4 hswork)))
                (setf maxwrk
                        (max (the f2cl-lib:integer4 maxwrk)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add n
                                                    (f2cl-lib:int-mul
                                                     (f2cl-lib:int-sub n 1)
                                                     (multiple-value-bind
                                                         (ret-val var-0
                                                          var-1 var-2 var-3
                                                          var-4 var-5 var-6)
                                                         (ilaenv 1 "DORGHR"
                                                          " " n 1 n -1)
                                                       (declare
                                                        (ignore var-0 var-1
                                                         var-2 var-4 var-6))
                                                       (when var-3
                                                         (setf n var-3))
                                                       (when var-5
                                                         (setf n var-5))
                                                       ret-val))))))
                (if (and (not wntsnn) (not wntsne))
                    (setf maxwrk
                            (max (the f2cl-lib:integer4 maxwrk)
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add
                                       (f2cl-lib:int-mul n n)
                                       (f2cl-lib:int-mul 6 n))))))
                (setf maxwrk
                        (max (the f2cl-lib:integer4 maxwrk)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-mul 3 n))))))
            (setf maxwrk
                    (max (the f2cl-lib:integer4 maxwrk)
                         (the f2cl-lib:integer4 minwrk)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 maxwrk) 'double-float))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -21)))))
(cond
 ((/= info 0) (xerbla "DGEEVX" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
```

```
(if (= n 0)
    (go end_label))
(setf eps (dlamch "P"))
(setf smlnum (dlamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf icond 0)
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "M" n n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf scalea f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf scalea f2cl-lib:%true%)
  (setf cscale smlnum))
 ((> anrm bignum) (setf scalea f2cl-lib:%true%) (setf cscale bignum)))
(if scalea
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 anrm cscale n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf cscale var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dgebal balanc n a lda ilo ihi scale ierr)
  (declare (ignore var-1 var-2 var-6))
  (setf balanc var-0)
  (setf lda var-3)
  (setf ilo var-4)
  (setf ihi var-5)
  (setf ierr var-7))
(setf abnrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "1" n n a lda dum)
```

```
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(cond
 (scalea (setf (f2cl-lib:fref dum (1) ((1 1))) abnrm)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 cscale anrm 1 1 dum 1 ierr)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
    (when var-3 (setf cscale var-3))
    (when var-4 (setf anrm var-4))
    (when var-9 (setf ierr var-9)))
  (setf abnrm (f2cl-lib:fref dum (1) ((1 1))))))
(setf itau 1)
(setf iwrk (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dgehrd n ilo ihi a lda
     (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                           work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf ilo var-1))
  (when var-2 (setf ihi var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(cond
 (wantvl (f2cl-lib:f2cl-set-string side "L" (string 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "L" n n a lda vl ldvl)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvl var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dorghr n ilo ihi vl ldvl
       (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf n var-0))
```

```
    (when var-1 (setf ilo var-1))
    (when var-2 (setf ihi var-2))
    (when var-4 (setf ldvl var-4))
    (when var-8 (setf ierr var-8)))
  (setf iwrk itau)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dhseqr "S" "V" n ilo ihi a lda wr wi vl ldvl
       (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
    (declare
     (ignore var-0 var-1 var-5 var-7 var-8 var-9 var-11 var-12))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-10 (setf ldvl var-10))
    (when var-13 (setf info var-13)))
  (cond
   (wantvr (f2cl-lib:f2cl-set-string side "B" (string 1))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "F" n n vl ldvl vr ldvr)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf ldvl var-4))
      (when var-6 (setf ldvr var-6))))))
 (wantvr (f2cl-lib:f2cl-set-string side "R" (string 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "L" n n a lda vr ldvr)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvr var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dorghr n ilo ihi vr ldvr
       (f2cl-lib:array-slice work-%data% double-float (itau) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf n var-0))
    (when var-1 (setf ilo var-1))
    (when var-2 (setf ihi var-2))
    (when var-4 (setf ldvr var-4))
```

```
    (when var-8 (setf ierr var-8)))
  (setf iwrk itau)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dhseqr "S" "V" n ilo ihi a lda wr wi vr ldvr
       (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
    (declare
     (ignore var-0 var-1 var-5 var-7 var-8 var-9 var-11 var-12))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-10 (setf ldvr var-10))
    (when var-13 (setf info var-13))))
 (t
  (cond (wntsnn (f2cl-lib:f2cl-set-string job "E" (string 1)))
        (t (f2cl-lib:f2cl-set-string job "S" (string 1))))
  (setf iwrk itau)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dhseqr job "N" n ilo ihi a lda wr wi vr ldvr
       (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                             work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
    (declare (ignore var-1 var-5 var-7 var-8 var-9 var-11 var-12))
    (when var-0 (setf job var-0))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-10 (setf ldvr var-10))
    (when var-13 (setf info var-13)))))
(if (> info 0)
    (go label50))
(cond
 ((or wantvl wantvr)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dtrevc side "B" select n a lda vl ldvl vr ldvr n nout
       (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                             work-%offset%)
       ierr)
    (declare (ignore var-1 var-2 var-4 var-6 var-8 var-12))
    (when var-0 (setf side var-0))
    (when var-3 (setf n var-3))
```

```
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldvl var-7))
    (when var-9 (setf ldvr var-9))
    (when var-10 (setf n var-10))
    (when var-11 (setf nout var-11))
    (when var-13 (setf ierr var-13)))))
(cond
 ((not wntsnn)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17)
      (dtrsna sense "A" select n a lda vl ldvl vr ldvr rconde rcondv n
       nout
       (f2cl-lib:array-slice work-%data% double-float (iwrk) ((1 *))
                             work-%offset%)
       n iwork icond)
    (declare
     (ignore var-1 var-2 var-4 var-6 var-8 var-10 var-11 var-14
      var-16))
    (when var-0 (setf sense var-0))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldvl var-7))
    (when var-9 (setf ldvr var-9))
    (when var-12 (setf n var-12))
    (when var-13 (setf nout var-13))
    (when var-15 (setf n var-15))
    (when var-17 (setf icond var-17)))))
(cond
 (wantvl
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dgebak balanc "L" n ilo ihi scale n vl ldvl ierr)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-7))
    (setf balanc var-0)
    (setf n var-6)
    (setf ldvl var-8)
    (setf ierr var-9))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref wi (i) ((1 *))) zero)
        (setf scl
                (/ one
                   (multiple-value-bind (ret-val var-0 var-1 var-2)
                       (dnrm2 n
                        (f2cl-lib:array-slice vl-%data% double-float
                                              (1 i) ((1 ldvl) (1 *))
                                              vl-%offset%)
```

```
                1)
              (declare (ignore var-1 var-2))
              (when var-0 (setf n var-0))
              ret-val)))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (dscal n scl
      (f2cl-lib:array-slice vl-%data% double-float (1 i)
                            ((1 ldvl) (1 *)) vl-%offset%)
      1)
   (declare (ignore var-2 var-3))
   (when var-0 (setf n var-0))
   (when var-1 (setf scl var-1))))
((> (f2cl-lib:fref wi (i) ((1 *))) zero)
 (setf scl
         (/ one
            (dlapy2
             (multiple-value-bind (ret-val var-0 var-1 var-2)
                 (dnrm2 n
                  (f2cl-lib:array-slice vl-%data% double-float
                                        (1 i) ((1 ldvl) (1 *))
                                        vl-%offset%)
                  1)
               (declare (ignore var-1 var-2))
               (when var-0 (setf n var-0))
               ret-val)
             (multiple-value-bind (ret-val var-0 var-1 var-2)
                 (dnrm2 n
                  (f2cl-lib:array-slice vl-%data% double-float
                                        (1
                                         (f2cl-lib:int-add i 1))
                                        ((1 ldvl) (1 *))
                                        vl-%offset%)
                  1)
               (declare (ignore var-1 var-2))
               (when var-0 (setf n var-0))
               ret-val))))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (dscal n scl
      (f2cl-lib:array-slice vl-%data% double-float (1 i)
                            ((1 ldvl) (1 *)) vl-%offset%)
      1)
   (declare (ignore var-2 var-3))
   (when var-0 (setf n var-0))
   (when var-1 (setf scl var-1)))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (dscal n scl
      (f2cl-lib:array-slice vl-%data% double-float
                            (1 (f2cl-lib:int-add i 1))
                            ((1 ldvl) (1 *)) vl-%offset%)
      1)
```

```
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf scl var-1)))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
            (+
             (expt
              (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                             vl-%offset%)
              2)
             (expt
              (f2cl-lib:fref vl-%data%
                             (k (f2cl-lib:int-add i 1))
                             ((1 ldvl) (1 *)) vl-%offset%)
              2)))
   label10))
(setf k
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (idamax n work 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dlartg
     (f2cl-lib:array-slice vl-%data% double-float (k i)
                           ((1 ldvl) (1 *)) vl-%offset%)
     (f2cl-lib:array-slice vl-%data% double-float
                           (k (f2cl-lib:int-add i 1))
                           ((1 ldvl) (1 *)) vl-%offset%)
     cs sn r)
  (declare (ignore var-0 var-1))
  (when var-2 (setf cs var-2))
  (when var-3 (setf sn var-3))
  (when var-4 (setf r var-4)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (drot n
     (f2cl-lib:array-slice vl-%data% double-float (1 i)
                           ((1 ldvl) (1 *)) vl-%offset%)
     1
     (f2cl-lib:array-slice vl-%data% double-float
                           (1 (f2cl-lib:int-add i 1))
                           ((1 ldvl) (1 *)) vl-%offset%)
     1 cs sn)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0))
  (when var-5 (setf cs var-5))
  (when var-6 (setf sn var-6)))
```

```
        (setf (f2cl-lib:fref vl-%data% (k (f2cl-lib:int-add i 1))
                             ((1 ldvl) (1 *)) vl-%offset%)
              zero)))
    label20))))
(cond
 (wantvr
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dgebak balanc "R" n ilo ihi scale n vr ldvr ierr)
    (declare (ignore var-1 var-2 var-3 var-4 var-5 var-7))
    (setf balanc var-0)
    (setf n var-6)
    (setf ldvr var-8)
    (setf ierr var-9))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (cond
       ((= (f2cl-lib:fref wi (i) ((1 *))) zero)
        (setf scl
                (/ one
                   (multiple-value-bind (ret-val var-0 var-1 var-2)
                       (dnrm2 n
                        (f2cl-lib:array-slice vr-%data% double-float
                                              (1 i) ((1 ldvr) (1 *))
                                              vr-%offset%)
                        1)
                      (declare (ignore var-1 var-2))
                      (when var-0 (setf n var-0))
                      ret-val)))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal n scl
             (f2cl-lib:array-slice vr-%data% double-float (1 i)
                                   ((1 ldvr) (1 *)) vr-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-1 (setf scl var-1))))
       ((> (f2cl-lib:fref wi (i) ((1 *))) zero)
        (setf scl
                (/ one
                   (dlapy2
                    (multiple-value-bind (ret-val var-0 var-1 var-2)
                        (dnrm2 n
                         (f2cl-lib:array-slice vr-%data% double-float
                                               (1 i) ((1 ldvr) (1 *))
                                               vr-%offset%)
                         1)
                       (declare (ignore var-1 var-2))
                       (when var-0 (setf n var-0))
```

```
                ret-val)
              (multiple-value-bind (ret-val var-0 var-1 var-2)
                  (dnrm2 n
                   (f2cl-lib:array-slice vr-%data% double-float
                                         (1
                                          (f2cl-lib:int-add i 1))
                                         ((1 ldvr) (1 *))
                                         vr-%offset%)
                   1)
                (declare (ignore var-1 var-2))
                (when var-0 (setf n var-0))
                ret-val))))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (dscal n scl
     (f2cl-lib:array-slice vr-%data% double-float (1 i)
                           ((1 ldvr) (1 *)) vr-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf scl var-1)))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (dscal n scl
     (f2cl-lib:array-slice vr-%data% double-float
                           (1 (f2cl-lib:int-add i 1))
                           ((1 ldvr) (1 *)) vr-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf scl var-1)))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
            (+
             (expt
              (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                             vr-%offset%)
              2)
             (expt
              (f2cl-lib:fref vr-%data%
                             (k (f2cl-lib:int-add i 1))
                             ((1 ldvr) (1 *)) vr-%offset%)
              2)))
   label30))
(setf k
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (idamax n work 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
```

```
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (dlartg
                 (f2cl-lib:array-slice vr-%data% double-float (k i)
                                       ((1 ldvr) (1 *)) vr-%offset%)
                 (f2cl-lib:array-slice vr-%data% double-float
                                       (k (f2cl-lib:int-add i 1))
                                       ((1 ldvr) (1 *)) vr-%offset%)
                 cs sn r)
              (declare (ignore var-0 var-1))
              (when var-2 (setf cs var-2))
              (when var-3 (setf sn var-3))
              (when var-4 (setf r var-4)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (drot n
                 (f2cl-lib:array-slice vr-%data% double-float (1 i)
                                       ((1 ldvr) (1 *)) vr-%offset%)
                 1
                 (f2cl-lib:array-slice vr-%data% double-float
                                       (1 (f2cl-lib:int-add i 1))
                                       ((1 ldvr) (1 *)) vr-%offset%)
                 1 cs sn)
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf n var-0))
              (when var-5 (setf cs var-5))
              (when var-6 (setf sn var-6)))
            (setf (f2cl-lib:fref vr-%data% (k (f2cl-lib:int-add i 1))
                                 ((1 ldvr) (1 *)) vr-%offset%)
                  zero)))
        label40))))
  label50
   (cond
    (scalea
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (dlascl "G" 0 0 cscale anrm (f2cl-lib:int-sub n info) 1
          (f2cl-lib:array-slice wr-%data% double-float ((+ info 1))
                                ((1 *)) wr-%offset%)
          (max (the f2cl-lib:integer4 (f2cl-lib:int-sub n info))
               (the f2cl-lib:integer4 1))
          ierr)
       (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
       (when var-3 (setf cscale var-3))
       (when var-4 (setf anrm var-4))
       (when var-9 (setf ierr var-9)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (dlascl "G" 0 0 cscale anrm (f2cl-lib:int-sub n info) 1
          (f2cl-lib:array-slice wi-%data% double-float ((+ info 1))
                                ((1 *)) wi-%offset%)
```

```
          (max (the f2cl-lib:integer4 (f2cl-lib:int-sub n info))
               (the f2cl-lib:integer4 1))
          ierr)
      (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
      (when var-3 (setf cscale var-3))
      (when var-4 (setf anrm var-4))
      (when var-9 (setf ierr var-9)))
    (cond
     ((= info 0)
      (if (and (or wntsnv wntsnb) (= icond 0))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9)
              (dlascl "G" 0 0 cscale anrm n 1 rcondv n ierr)
            (declare (ignore var-0 var-1 var-2 var-6 var-7))
            (when var-3 (setf cscale var-3))
            (when var-4 (setf anrm var-4))
            (when var-5 (setf n var-5))
            (when var-8 (setf n var-8))
            (when var-9 (setf ierr var-9)))))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (dlascl "G" 0 0 cscale anrm (f2cl-lib:int-sub ilo 1) 1 wr n
           ierr)
        (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
        (when var-3 (setf cscale var-3))
        (when var-4 (setf anrm var-4))
        (when var-8 (setf n var-8))
        (when var-9 (setf ierr var-9)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (dlascl "G" 0 0 cscale anrm (f2cl-lib:int-sub ilo 1) 1 wi n
           ierr)
        (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
        (when var-3 (setf cscale var-3))
        (when var-4 (setf anrm var-4))
        (when var-8 (setf n var-8))
        (when var-9 (setf ierr var-9)))))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 maxwrk) 'double-float))
  (go end_label)
 end_label
  (return
   (values balanc jobvl jobvr sense n nil lda nil nil nil ldvl nil ldvr
           ilo ihi nil abnrm nil nil nil nil nil info))))))
```

———-

90.122 dgehd2

90.122.1 Axiom unit tests

— dgehd2.input —

```
)set break resume
)sys rm -f dgehd2.output
)spool dgehd2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.122.2 Axiom help page

— dgehd2.help —

```
====================================================================
dgehd2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DGEHD2  -  a  real  general  matrix  A to upper Hessenberg form H by an
       orthogonal similarity transformation

SYNOPSIS
       SUBROUTINE DGEHD2( N, ILO, IHI, A, LDA, TAU, WORK, INFO )

           INTEGER        IHI, ILO, INFO, LDA, N

           DOUBLE         PRECISION A( LDA, * ), TAU( * ), WORK( * )

PURPOSE
       DGEHD2 reduces a real general matrix A to upper Hessenberg form H by an
       orthogonal similarity transformation:  Q' * A * Q = H .
```

ARGUMENTS

N (input) INTEGER
The order of the matrix A. N >= 0.

ILO (input) INTEGER
IHI (input) INTEGER It is assumed that A is already upper triangular in rows and columns 1:ILO-1 and IHI+1:N. ILO and IHI are normally set by a previous call to DGEBAL; otherwise they should be set to 1 and N respectively. See Further Details.

A (input/output) DOUBLE PRECISION array, dimension (LDA,N)
On entry, the n by n general matrix to be reduced. On exit, the upper triangle and the first subdiagonal of A are overwritten with the upper Hessenberg matrix H, and the elements below the first subdiagonal, with the array TAU, represent the orthogonal matrix Q as a product of elementary reflectors. See Further Details. LDA (input) INTEGER The leading dimension of the array A. LDA >= max(1,N).

TAU (output) DOUBLE PRECISION array, dimension (N-1)
The scalar factors of the elementary reflectors (see Further Details).

WORK (workspace) DOUBLE PRECISION array, dimension (N)

INFO (output) INTEGER
= 0: successful exit.
< 0: if INFO = -i, the i-th argument had an illegal value.

FURTHER DETAILS

The matrix Q is represented as a product of (ihi-ilo) elementary reflectors

```
Q = H(ilo) H(ilo+1) . . . H(ihi-1).
```

Each H(i) has the form

```
H(i) = I - tau * v * v'
```

where tau is a real scalar, and v is a real vector with v(1:i) = 0, v(i+1) = 1 and v(ihi+1:n) = 0; v(i+2:ihi) is stored on exit in A(i+2:ihi,i), and tau in TAU(i).

The contents of A are illustrated by the following example, with n = 7, ilo = 2 and ihi = 6:

```
on entry,                        on exit,

( a   a   a   a   a   a   a )    (  a   a   h   h   h   h   a )
(     a   a   a   a   a   a )    (      a   h   h   h   h   a )
```

```
(     a   a   a   a   a   a )    (      h   h   h   h   h   h )
(     a   a   a   a   a   a )    (      v2  h   h   h   h   h )
(     a   a   a   a   a   a )    (      v2  v3  h   h   h   h )
(     a   a   a   a   a   a )    (      v2  v3  v4  h   h   h )
(                         a )    (                          a )
```

where a denotes an element of the original matrix A, h denotes a modified element of the upper Hessenberg matrix H, and vi denotes an element of the vector defining H(i).

———-

90.122.3 fortran code

— dgehd2.f —

```
*  =====================================================================
      SUBROUTINE DGEHD2( N, ILO, IHI, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            IHI, ILO, INFO, LDA, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      DOUBLE PRECISION   AII
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARF, DLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
```

```
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -2
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEHD2', -INFO )
         RETURN
      END IF
*
      DO 10 I = ILO, IHI - 1
*
*        Compute elementary reflector H(i) to annihilate A(i+2:ihi,i)
*
         CALL DLARFG( IHI-I, A( I+1, I ), A( MIN( I+2, N ), I ), 1,
     $                TAU( I ) )
         AII = A( I+1, I )
         A( I+1, I ) = ONE
*
*        Apply H(i) to A(1:ihi,i+1:ihi) from the right
*
         CALL DLARF( 'Right', IHI, IHI-I, A( I+1, I ), 1, TAU( I ),
     $               A( 1, I+1 ), LDA, WORK )
*
*        Apply H(i) to A(i+1:ihi,i+1:n) from the left
*
         CALL DLARF( 'Left', IHI-I, N-I, A( I+1, I ), 1, TAU( I ),
     $               A( I+1, I+1 ), LDA, WORK )
*
         A( I+1, I ) = AII
   10 CONTINUE
*
      RETURN
*
*     End of DGEHD2
*
      END
      SUBROUTINE DGEHD2( N, ILO, IHI, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
```

```
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1992
*
*     .. Scalar Arguments ..
      INTEGER            IHI, ILO, INFO, LDA, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      DOUBLE PRECISION   AII
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARF, DLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -2
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEHD2', -INFO )
         RETURN
      END IF
*
      DO 10 I = ILO, IHI - 1
*
*        Compute elementary reflector H(i) to annihilate A(i+2:ihi,i)
*
         CALL DLARFG( IHI-I, A( I+1, I ), A( MIN( I+2, N ), I ), 1,
```

```
     $                TAU( I ) )
         AII = A( I+1, I )
         A( I+1, I ) = ONE
*
*        Apply H(i) to A(1:ihi,i+1:ihi) from the right
*
         CALL DLARF( 'Right', IHI, IHI-I, A( I+1, I ), 1, TAU( I ),
     $               A( 1, I+1 ), LDA, WORK )
*
*        Apply H(i) to A(i+1:ihi,i+1:n) from the left
*
         CALL DLARF( 'Left', IHI-I, N-I, A( I+1, I ), 1, TAU( I ),
     $               A( I+1, I+1 ), LDA, WORK )
*
         A( I+1, I ) = AII
   10 CONTINUE
*
      RETURN
*
*     End of DGEHD2
*
      END
```

———-

90.122.4 lisp code

— dgehd2.lisp —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dgehd2 (n ilo ihi a lda tau work info)
    (declare (type (array double-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lda ihi ilo n))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((aii 0.0d0) (i 0))
        (declare (type (double-float) aii)
                 (type (f2cl-lib:integer4) i))
        (setf info 0)
        (cond ((< n 0) (setf info -1))
              ((or (< ilo 1)
                   (> ilo
                      (max (the f2cl-lib:integer4 1)
```

```
                    (the f2cl-lib:integer4 n))))
       (setf info -2))
      ((or
        (< ihi
           (min (the f2cl-lib:integer4 ilo) (the f2cl-lib:integer4 n)))
        (> ihi n))
       (setf info -3))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -5)))
(cond
 ((/= info 0) (xerbla "DGEHD2" (f2cl-lib:int-sub info))
  (go end_label)))
(f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add ihi (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (dlarfg (f2cl-lib:int-sub ihi i)
     (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                           ((1 lda) (1 *)) a-%offset%)
     (f2cl-lib:array-slice a-%data% double-float
                           ((min (f2cl-lib:int-add i 2) n) i)
                           ((1 lda) (1 *)) a-%offset%)
     1
     (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                           tau-%offset%))
    (setf aii
            (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                           ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                         ((1 lda) (1 *)) a-%offset%)
            one)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dlarf "Right" ihi (f2cl-lib:int-sub ihi i)
         (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                               ((1 lda) (1 *)) a-%offset%)
         1
         (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                               tau-%offset%)
         (f2cl-lib:array-slice a-%data% double-float
                               (1 (f2cl-lib:int-add i 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda work)
      (declare (ignore var-0 var-2 var-3 var-4 var-5 var-6 var-8))
      (when var-1 (setf ihi var-1))
      (when var-7 (setf lda var-7)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dlarf "Left" (f2cl-lib:int-sub ihi i) (f2cl-lib:int-sub n i)
         (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
```

```
                                  ((1 lda) (1 *)) a-%offset%)
                1
                (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                  tau-%offset%)
                (f2cl-lib:array-slice a-%data% double-float
                                  ((+ i 1) (f2cl-lib:int-add i 1))
                                  ((1 lda) (1 *)) a-%offset%)
                lda work)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
              (when var-7 (setf lda var-7)))
            (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                              ((1 lda) (1 *)) a-%offset%)
                  aii)
          label10))
      (go end_label)
     end_label
      (return (values nil nil ihi nil lda nil nil info))))))
```

———-

90.123 dgehrd

90.123.1 Axiom unit tests

— **dgehrd.input** —

```
)set break resume
)sys rm -f dgehrd.output
)spool dgehrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.123.2 Axiom help page

— **dgehrd.help** —

```
==================================================================
```

```
dgehrd examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DGEHRD  -  Reduce a real general matrix A to upper Hessenberg form H by
       an orthogonal similarity transformation

SYNOPSIS
       SUBROUTINE DGEHRD( N, ILO, IHI, A, LDA, TAU, WORK, LWORK, INFO )

           INTEGER        IHI, ILO, INFO, LDA, LWORK, N

           DOUBLE         PRECISION A( LDA, * ), TAU( * ), WORK( * )

PURPOSE
       DGEHRD reduces a real general matrix A to upper Hessenberg form H by an
       orthogonal similarity transformation:  Q' * A * Q = H .


ARGUMENTS
       N       (input) INTEGER
               The order of the matrix A.  N >= 0.

       ILO     (input) INTEGER
               IHI      (input)  INTEGER It is assumed that A is already upper
               triangular in rows and columns 1:ILO-1 and IHI+1:N. ILO and IHI
               are  normally  set by a previous call to DGEBAL; otherwise they
               should be set to 1 and N respectively. See Further Details.

       A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
               On entry, the N-by-N general matrix to be  reduced.   On  exit,
               the upper triangle and the first subdiagonal of A are overwrit-
               ten with the upper Hessenberg matrix H, and the elements  below
               the  first  subdiagonal,  with  the  array  TAU,  represent the
               orthogonal matrix Q as a product of elementary reflectors.  See
               Further Details.  LDA     (input) INTEGER The leading dimension
               of the array A.  LDA >= max(1,N).

       TAU     (output) DOUBLE PRECISION array, dimension (N-1)
               The scalar factors of the elementary  reflectors  (see  Further
               Details).  Elements 1:ILO-1 and IHI:N-1 of TAU are set to zero.

       WORK    (workspace/output) DOUBLE PRECISION array, dimension (LWORK)
               On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

       LWORK   (input) INTEGER
```

```
        The length of the array WORK.  LWORK >= max(1,N).  For  optimum
        performance LWORK >= N*NB, where NB is the optimal blocksize.

        If  LWORK  = -1, then a workspace query is assumed; the routine
        only calculates the optimal size of  the  WORK  array,  returns
        this  value  as the first entry of the WORK array, and no error
        message related to LWORK is issued by XERBLA.

INFO    (output) INTEGER
        = 0:  successful exit
        < 0:  if INFO = -i, the i-th argument had an illegal value.

FURTHER DETAILS
      The matrix Q is  represented  as  a  product  of  (ihi-ilo)  elementary
      reflectors

         Q = H(ilo) H(ilo+1) . . . H(ihi-1).

      Each H(i) has the form

         H(i) = I - tau * v * v'

      where tau is a real scalar, and v is a real vector with
      v(1:i) = 0, v(i+1) = 1 and v(ihi+1:n) = 0; v(i+2:ihi) is stored on exit
      in A(i+2:ihi,i), and tau in TAU(i).

      The contents of A are illustrated by the following example, with n = 7,
      ilo = 2 and ihi = 6:

      on entry,                        on exit,

      ( a   a   a   a   a   a   a )    (  a   a   h   h   h   h   a )
      (     a   a   a   a   a   a )    (      a   h   h   h   h   a )
      (     a   a   a   a   a   a )    (      h   h   h   h   h   h )
      (     a   a   a   a   a   a )    (      v2  h   h   h   h   h )
      (     a   a   a   a   a   a )    (      v2  v3  h   h   h   h )
      (     a   a   a   a   a   a )    (      v2  v3  v4  h   h   h )
      (                         a )    (                          a )

      where  a denotes an element of the original matrix A, h denotes a modi-
      fied element of the upper Hessenberg matrix H, and vi denotes  an  ele-
      ment of the vector defining H(i).

      See Quintana-Orti and Van  de  Geijn (2005).
```

90.123.3 fortran code

— dgehrd.f —

```
*  =====================================================================
      SUBROUTINE DGEHRD( N, ILO, IHI, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            IHI, ILO, INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION  A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            NBMAX, LDT, TSIZE
      PARAMETER          ( NBMAX = 64, LDT = NBMAX+1,
     $                     TSIZE = LDT*NBMAX )
      DOUBLE PRECISION  ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0,
     $                     ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWT, J, LDWORK, LWKOPT, NB,
     $                   NBMIN, NH, NX
      DOUBLE PRECISION  EI
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DGEHD2, DGEMM, DLAHR2, DLARFB, DTRMM,
     $                   XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
```

```
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -2
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -8
      END IF
*
      IF( INFO.EQ.0 ) THEN
*
*        Compute the workspace requirements
*
         NB = MIN( NBMAX, ILAENV( 1, 'DGEHRD', ' ', N, ILO, IHI, -1 ) )
         LWKOPT = N*NB + TSIZE
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEHRD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Set elements 1:ILO-1 and IHI:N-1 of TAU to zero
*
      DO 10 I = 1, ILO - 1
         TAU( I ) = ZERO
   10 CONTINUE
      DO 20 I = MAX( 1, IHI ), N - 1
         TAU( I ) = ZERO
   20 CONTINUE
*
*     Quick return if possible
*
      NH = IHI - ILO + 1
      IF( NH.LE.1 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
*     Determine the block size
*
```

```
      NB = MIN( NBMAX, ILAENV( 1, 'DGEHRD', ' ', N, ILO, IHI, -1 ) )
      NBMIN = 2
      IF( NB.GT.1 .AND. NB.LT.NH ) THEN
*
*        Determine when to cross over from blocked to unblocked code
*        (last block is always handled by unblocked code)
*
         NX = MAX( NB, ILAENV( 3, 'DGEHRD', ' ', N, ILO, IHI, -1 ) )
         IF( NX.LT.NH ) THEN
*
*           Determine if workspace is large enough for blocked code
*
            IF( LWORK.LT.N*NB+TSIZE ) THEN
*
*              Not enough workspace to use optimal NB:  determine the
*              minimum value of NB, and reduce NB or force use of
*              unblocked code
*
               NBMIN = MAX( 2, ILAENV( 2, 'DGEHRD', ' ', N, ILO, IHI,
     $                 -1 ) )
               IF( LWORK.GE.(N*NBMIN + TSIZE) ) THEN
                  NB = (LWORK-TSIZE) / N
               ELSE
                  NB = 1
               END IF
            END IF
         END IF
      END IF
      LDWORK = N
*
      IF( NB.LT.NBMIN .OR. NB.GE.NH ) THEN
*
*        Use unblocked code below
*
         I = ILO
*
      ELSE
*
*        Use blocked code
*
         IWT = 1 + N*NB
         DO 40 I = ILO, IHI - 1 - NX, NB
            IB = MIN( NB, IHI-I )
*
*           Reduce columns i:i+ib-1 to Hessenberg form, returning the
*           matrices V and T of the block reflector H = I - V*T*V**T
*           which performs the reduction, and also the matrix Y = A*V*T
*
            CALL DLAHR2( IHI, I, IB, A( 1, I ), LDA, TAU( I ),
     $                   WORK( IWT ), LDT, WORK, LDWORK )
```

```
*
*          Apply the block reflector H to A(1:ihi,i+ib:ihi) from the
*          right, computing  A := A - Y * V**T. V(i+ib,ib-1) must be set
*          to 1
*
           EI = A( I+IB, I+IB-1 )
           A( I+IB, I+IB-1 ) = ONE
           CALL DGEMM( 'No transpose', 'Transpose',
     $                 IHI, IHI-I-IB+1,
     $                 IB, -ONE, WORK, LDWORK, A( I+IB, I ), LDA, ONE,
     $                 A( 1, I+IB ), LDA )
           A( I+IB, I+IB-1 ) = EI
*
*          Apply the block reflector H to A(1:i,i+1:i+ib-1) from the
*          right
*
           CALL DTRMM( 'Right', 'Lower', 'Transpose',
     $                 'Unit', I, IB-1,
     $                 ONE, A( I+1, I ), LDA, WORK, LDWORK )
           DO 30 J = 0, IB-2
              CALL DAXPY( I, -ONE, WORK( LDWORK*J+1 ), 1,
     $                    A( 1, I+J+1 ), 1 )
   30      CONTINUE
*
*          Apply the block reflector H to A(i+1:ihi,i+ib:n) from the
*          left
*
           CALL DLARFB( 'Left', 'Transpose', 'Forward',
     $                  'Columnwise',
     $                  IHI-I, N-I-IB+1, IB, A( I+1, I ), LDA,
     $                  WORK( IWT ), LDT, A( I+1, I+IB ), LDA,
     $                  WORK, LDWORK )
   40    CONTINUE
      END IF
*
*     Use unblocked code to reduce the rest of the matrix
*
      CALL DGEHD2( N, I, IHI, A, LDA, TAU, WORK, IINFO )
      WORK( 1 ) = LWKOPT
*
      RETURN
*
*     End of DGEHRD
*
      END
      SUBROUTINE DGEHRD( N, ILO, IHI, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
```

```
*     June 30, 1999
*
*     .. Scalar Arguments ..
      INTEGER            IHI, ILO, INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            NBMAX, LDT
      PARAMETER          ( NBMAX = 64, LDT = NBMAX+1 )
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, LDWORK, LWKOPT, NB, NBMIN,
     $                   NH, NX
      DOUBLE PRECISION   EI
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   T( LDT, NBMAX )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEHD2, DGEMM, DLAHRD, DLARFB, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      NB = MIN( NBMAX, ILAENV( 1, 'DGEHRD', ' ', N, ILO, IHI, -1 ) )
      LWKOPT = N*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -2
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
```

```
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEHRD', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Set elements 1:ILO-1 and IHI:N-1 of TAU to zero
*
      DO 10 I = 1, ILO - 1
         TAU( I ) = ZERO
   10 CONTINUE
      DO 20 I = MAX( 1, IHI ), N - 1
         TAU( I ) = ZERO
   20 CONTINUE
*
*     Quick return if possible
*
      NH = IHI - ILO + 1
      IF( NH.LE.1 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
*     Determine the block size.
*
      NB = MIN( NBMAX, ILAENV( 1, 'DGEHRD', ' ', N, ILO, IHI, -1 ) )
      NBMIN = 2
      IWS = 1
      IF( NB.GT.1 .AND. NB.LT.NH ) THEN
*
*        Determine when to cross over from blocked to unblocked code
*        (last block is always handled by unblocked code).
*
         NX = MAX( NB, ILAENV( 3, 'DGEHRD', ' ', N, ILO, IHI, -1 ) )
         IF( NX.LT.NH ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            IWS = N*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  determine the
*              minimum value of NB, and reduce NB or force use of
```

```
*             unblocked code.
*
              NBMIN = MAX( 2, ILAENV( 2, 'DGEHRD', ' ', N, ILO, IHI,
     $                -1 ) )
              IF( LWORK.GE.N*NBMIN ) THEN
                 NB = LWORK / N
              ELSE
                 NB = 1
              END IF
           END IF
        END IF
     END IF
     LDWORK = N
*
     IF( NB.LT.NBMIN .OR. NB.GE.NH ) THEN
*
*       Use unblocked code below
*
        I = ILO
*
     ELSE
*
*       Use blocked code
*
        DO 30 I = ILO, IHI - 1 - NX, NB
           IB = MIN( NB, IHI-I )
*
*          Reduce columns i:i+ib-1 to Hessenberg form, returning the
*          matrices V and T of the block reflector H = I - V*T*V'
*          which performs the reduction, and also the matrix Y = A*V*T
*
           CALL DLAHRD( IHI, I, IB, A( 1, I ), LDA, TAU( I ), T, LDT,
     $                  WORK, LDWORK )
*
*          Apply the block reflector H to A(1:ihi,i+ib:ihi) from the
*          right, computing  A := A - Y * V'. V(i+ib,ib-1) must be set
*          to 1.
*
           EI = A( I+IB, I+IB-1 )
           A( I+IB, I+IB-1 ) = ONE
           CALL DGEMM( 'No transpose', 'Transpose', IHI, IHI-I-IB+1,
     $                 IB, -ONE, WORK, LDWORK, A( I+IB, I ), LDA, ONE,
     $                 A( 1, I+IB ), LDA )
           A( I+IB, I+IB-1 ) = EI
*
*          Apply the block reflector H to A(i+1:ihi,i+ib:n) from the
*          left
*
           CALL DLARFB( 'Left', 'Transpose', 'Forward', 'Columnwise',
     $                  IHI-I, N-I-IB+1, IB, A( I+1, I ), LDA, T, LDT,
```

```
     $                   A( I+1, I+IB ), LDA, WORK, LDWORK )
   30    CONTINUE
      END IF
*
*     Use unblocked code to reduce the rest of the matrix
*
      CALL DGEHD2( N, I, IHI, A, LDA, TAU, WORK, IINFO )
      WORK( 1 ) = IWS
*
      RETURN
*
*     End of DGEHRD
*
      END
```

———-

90.123.4 lisp code

— dgehrd.lisp —

```
(let* ((nbmax 64)
       (ldt (+ nbmax 1))
       (tsize (* ldt nbmax))
       (zero 0.0d0)
       (one 1.0d0))
  (declare (type (f2cl-lib:integer4 64 64) nbmax)
           (type (f2cl-lib:integer4) ldt)
           (type (f2cl-lib:integer4) tsize)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable nbmax ldt tsize zero one))
  (defun dgehrd (n ilo ihi a lda tau work lwork info)
    (declare (type (array double-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lwork lda ihi ilo n))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((ei 0.0d0) (i 0) (ib 0) (iinfo 0) (iwt 0) (j 0) (ldwork 0)
             (lwkopt 0) (nb 0) (nbmin 0) (nh 0) (nx 0) (lquery nil))
        (declare (type (double-float) ei)
                 (type (f2cl-lib:integer4) i ib iinfo iwt j ldwork lwkopt nb
                  nbmin nh nx)
                 (type f2cl-lib:logical lquery))
        (setf info 0)
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
```

```
(cond ((< n 0) (setf info -1))
      ((or (< ilo 1)
           (> ilo
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 n))))
       (setf info -2))
      ((or
        (< ihi
           (min (the f2cl-lib:integer4 ilo) (the f2cl-lib:integer4 n)))
        (> ihi n))
       (setf info -3))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -5))
      ((and
        (< lwork
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (not lquery))
       (setf info -8)))
(cond
 ((= info 0)
  (setf nb
          (min (the f2cl-lib:integer4 nbmax)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 1 "DGEHRD" " " n ilo ihi -1)
                      (declare (ignore var-0 var-1 var-2 var-6))
                      (when var-3 (setf n var-3))
                      (when var-4 (setf ilo var-4))
                      (when var-5 (setf ihi var-5))
                      ret-val))))
  (setf lwkopt (f2cl-lib:int-add (f2cl-lib:int-mul n nb) tsize))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))))
(cond
 ((/= info 0) (xerbla "DGEHRD" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (setf (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%) zero)
   label10))
(f2cl-lib:fdo (i
               (max (the f2cl-lib:integer4 1)
                    (the f2cl-lib:integer4 ihi))
               (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
  (tagbody
```

```
    (setf (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%) zero)
   label20))
(setf nh (f2cl-lib:int-add (f2cl-lib:int-sub ihi ilo) 1))
(cond
 ((<= nh 1)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'double-float))
  (go end_label)))
(setf nb
        (min (the f2cl-lib:integer4 nbmax)
             (the f2cl-lib:integer4
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                       var-6)
                      (ilaenv 1 "DGEHRD" " " n ilo ihi -1)
                    (declare (ignore var-0 var-1 var-2 var-6))
                    (when var-3 (setf n var-3))
                    (when var-4 (setf ilo var-4))
                    (when var-5 (setf ihi var-5))
                    ret-val))))
(setf nbmin 2)
(cond
 ((and (> nb 1) (< nb nh))
  (setf nx
          (max (the f2cl-lib:integer4 nb)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "DGEHRD" " " n ilo ihi -1)
                      (declare (ignore var-0 var-1 var-2 var-6))
                      (when var-3 (setf n var-3))
                      (when var-4 (setf ilo var-4))
                      (when var-5 (setf ihi var-5))
                      ret-val))))
  (cond
   ((< nx nh)
    (cond
     ((< lwork (f2cl-lib:int-add (f2cl-lib:int-mul n nb) tsize))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4
                             var-5 var-6)
                            (ilaenv 2 "DGEHRD" " " n ilo ihi -1)
                          (declare (ignore var-0 var-1 var-2 var-6))
                          (when var-3 (setf n var-3))
                          (when var-4 (setf ilo var-4))
                          (when var-5 (setf ihi var-5))
```

```
                          ret-val))))
        (cond
         ((>= lwork (f2cl-lib:int-add (f2cl-lib:int-mul n nbmin) tsize))
          (setf nb (the f2cl-lib:integer4 (truncate (- lwork tsize) n))))
         (t (setf nb 1)))))))))
  (setf ldwork n)
  (cond ((or (< nb nbmin) (>= nb nh)) (setf i ilo))
        (t (setf iwt (f2cl-lib:int-add 1 (f2cl-lib:int-mul n nb)))
         (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i nb))
                       ((> i
                           (f2cl-lib:int-add ihi (f2cl-lib:int-sub 1)
                                             (f2cl-lib:int-sub nx)))
                        nil)
           (tagbody
             (setf ib
                     (min (the f2cl-lib:integer4 nb)
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-sub ihi i))))
             (multiple-value-bind
                 (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                  var-9)
                 (dlahr2 ihi i ib
                  (f2cl-lib:array-slice a-%data% double-float (1 i)
                                        ((1 lda) (1 *)) a-%offset%)
                  lda
                  (f2cl-lib:array-slice tau-%data% double-float (i)
                                        ((1 *)) tau-%offset%)
                  (f2cl-lib:array-slice work-%data% double-float (iwt)
                                        ((1 *)) work-%offset%)
                  ldt work ldwork)
               (declare (ignore var-3 var-5 var-6 var-8))
               (when var-0 (setf ihi var-0))
               (when var-1 (setf i var-1))
               (when var-2 (setf ib var-2))
               (when var-4 (setf lda var-4))
               (when var-7 (setf ldt var-7))
               (when var-9 (setf ldwork var-9)))
             (setf ei
                     (f2cl-lib:fref a-%data%
                                    ((f2cl-lib:int-add i ib)
                                     (f2cl-lib:int-sub
                                      (f2cl-lib:int-add i ib) 1))
                                    ((1 lda) (1 *)) a-%offset%))
             (setf (f2cl-lib:fref a-%data%
                                  ((f2cl-lib:int-add i ib)
                                   (f2cl-lib:int-sub
                                    (f2cl-lib:int-add i ib) 1))
                                  ((1 lda) (1 *)) a-%offset%)
                     one)
             (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (dgemm "No transpose" "Transpose" ihi
     (f2cl-lib:int-add (f2cl-lib:int-sub ihi i ib) 1) ib
     (- one) work ldwork
     (f2cl-lib:array-slice a-%data% double-float
                           ((+ i ib) i) ((1 lda) (1 *))
                           a-%offset%)
     lda one
     (f2cl-lib:array-slice a-%data% double-float
                           (1 (f2cl-lib:int-add i ib))
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare
   (ignore var-0 var-1 var-3 var-5 var-6 var-8 var-11))
  (when var-2 (setf ihi var-2))
  (when var-4 (setf ib var-4))
  (when var-7 (setf ldwork var-7))
  (when var-9 (setf lda var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf lda var-12)))
(setf (f2cl-lib:fref a-%data%
                     ((f2cl-lib:int-add i ib)
                      (f2cl-lib:int-sub
                       (f2cl-lib:int-add i ib) 1))
                     ((1 lda) (1 *)) a-%offset%)
        ei)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dtrmm "Right" "Lower" "Transpose" "Unit" i
     (f2cl-lib:int-sub ib 1) one
     (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                           ((1 lda) (1 *)) a-%offset%)
     lda work ldwork)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9))
  (when var-4 (setf i var-4))
  (when var-6 (setf one var-6))
  (when var-8 (setf lda var-8))
  (when var-10 (setf ldwork var-10)))
(f2cl-lib:fdo (j 0 (f2cl-lib:int-add j 1))
              ((> j
                  (f2cl-lib:int-add ib
                                    (f2cl-lib:int-sub 2)))
               nil)
  (tagbody
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5)
        (daxpy i (- one)
```

```
                        (f2cl-lib:array-slice work-%data% double-float
                                              ((+
                                                (f2cl-lib:int-mul ldwork j)
                                                1))
                                              ((1 *)) work-%offset%)
                        1
                        (f2cl-lib:array-slice a-%data% double-float
                                              (1 (f2cl-lib:int-add i j 1))
                                              ((1 lda) (1 *)) a-%offset%)
                        1)
                      (declare (ignore var-1 var-2 var-3 var-4 var-5))
                      (when var-0 (setf i var-0)))
                   label30))
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                     var-9 var-10 var-11 var-12 var-13 var-14)
                    (dlarfb "Left" "Transpose" "Forward" "Columnwise"
                     (f2cl-lib:int-sub ihi i)
                     (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1) ib
                     (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                                           ((1 lda) (1 *)) a-%offset%)
                     lda
                     (f2cl-lib:array-slice work-%data% double-float (iwt)
                                           ((1 *)) work-%offset%)
                     ldt
                     (f2cl-lib:array-slice a-%data% double-float
                                           ((+ i 1) (f2cl-lib:int-add i ib))
                                           ((1 lda) (1 *)) a-%offset%)
                     lda work ldwork)
                  (declare
                   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9
                    var-11 var-13))
                  (when var-6 (setf ib var-6))
                  (when var-8 (setf lda var-8))
                  (when var-10 (setf ldt var-10))
                  (when var-12 (setf lda var-12))
                  (when var-14 (setf ldwork var-14)))
              label40))))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dgehd2 n i ihi a lda tau work iinfo)
    (declare (ignore var-0 var-1 var-3 var-5 var-6))
    (setf ihi var-2)
    (setf lda var-4)
    (setf iinfo var-7))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
  (go end_label)
end_label
  (return (values n ilo ihi nil lda nil nil nil info))))))
```

———-

90.124 dgelq2

90.124.1 Axiom unit tests

— **dgelq2.input** —

```
)set break resume
)sys rm -f dgelq2.output
)spool dgelq2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.124.2 Axiom help page

— **dgelq2.help** —

```
======================================================================
dgelq2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DGELQ2 - an LQ factorization of a real m by n matrix A

SYNOPSIS
       SUBROUTINE DGELQ2( M, N, A, LDA, TAU, WORK, INFO )

           INTEGER        INFO, LDA, M, N

           DOUBLE         PRECISION A( LDA, * ), TAU( * ), WORK( * )

PURPOSE
       DGELQ2  computes an LQ factorization of a real m by n matrix A: A = L *
```

```
       Q.


ARGUMENTS
       M       (input) INTEGER
               The number of rows of the matrix A.  M >= 0.

       N       (input) INTEGER
               The number of columns of the matrix A.  N >= 0.

       A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
               On entry, the m by n matrix A.  On exit, the  elements  on  and
               below the diagonal of the array contain the m by min(m,n) lower
               trapezoidal matrix L (L is lower triangular if  m  <=  n);  the
               elements  above the diagonal, with the array TAU, represent the
               orthogonal matrix Q as a product of elementary reflectors  (see
               Further  Details).   LDA     (input) INTEGER The leading dimen-
               sion of the array A.  LDA >= max(1,M).

       TAU     (output) DOUBLE PRECISION array, dimension (min(M,N))
               The scalar factors of the elementary  reflectors  (see  Further
               Details).

       WORK    (workspace) DOUBLE PRECISION array, dimension (M)

       INFO    (output) INTEGER
               = 0: successful exit
               < 0: if INFO = -i, the i-th argument had an illegal value

FURTHER DETAILS
       The matrix Q is represented as a product of elementary reflectors

          Q = H(k) . . . H(2) H(1), where k = min(m,n).

       Each H(i) has the form

          H(i) = I - tau * v * v'

       where tau is a real scalar, and v is a real vector with
       v(1:i-1)  =  0  and v(i) = 1; v(i+1:n) is stored on exit in A(i,i+1:n),
       and tau in TAU(i).
```

———-

90.124.3 fortran code

— dgelq2.f —

```
*  =====================================================================
      SUBROUTINE DGELQ2( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      DOUBLE PRECISION   AII
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARF, DLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGELQ2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
```

```
      DO 10 I = 1, K
*
*        Generate elementary reflector H(i) to annihilate A(i,i+1:n)
*
         CALL DLARFG( N-I+1, A( I, I ), A( I, MIN( I+1, N ) ), LDA,
     $                TAU( I ) )
         IF( I.LT.M ) THEN
*
*           Apply H(i) to A(i+1:m,i:n) from the right
*
            AII = A( I, I )
            A( I, I ) = ONE
            CALL DLARF( 'Right', M-I, N-I+1, A( I, I ), LDA, TAU( I ),
     $                  A( I+1, I ), LDA, WORK )
            A( I, I ) = AII
         END IF
   10 CONTINUE
      RETURN
*
*     End of DGELQ2
*
      END
      SUBROUTINE DGELQ2( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     February 29, 1992
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      DOUBLE PRECISION   AII
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARF, DLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
```

```
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGELQ2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = 1, K
*
*        Generate elementary reflector H(i) to annihilate A(i,i+1:n)
*
         CALL DLARFG( N-I+1, A( I, I ), A( I, MIN( I+1, N ) ), LDA,
     $                TAU( I ) )
         IF( I.LT.M ) THEN
*
*           Apply H(i) to A(i+1:m,i:n) from the right
*
            AII = A( I, I )
            A( I, I ) = ONE
            CALL DLARF( 'Right', M-I, N-I+1, A( I, I ), LDA, TAU( I ),
     $                  A( I+1, I ), LDA, WORK )
            A( I, I ) = AII
         END IF
   10 CONTINUE
      RETURN
*
*     End of DGELQ2
*
      END
```

———-

90.124.4 lisp code

— dgelq2.lisp —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dgelq2 (m n a lda tau work info)
    (declare (type (array double-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((aii 0.0d0) (i 0) (k 0))
        (declare (type (double-float) aii)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "DGELQ2" (f2cl-lib:int-sub info))
          (go end_label)))
        (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i k) nil)
          (tagbody
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (dlarfg (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
                 (f2cl-lib:array-slice a-%data% double-float (i i)
                                       ((1 lda) (1 *)) a-%offset%)
                 (f2cl-lib:array-slice a-%data% double-float
                                       (i
                                        (min
                                         (the f2cl-lib:integer4
                                              (f2cl-lib:int-add i 1))
                                         (the f2cl-lib:integer4 n)))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                       tau-%offset%))
              (declare (ignore var-0 var-1 var-2 var-4))
              (when var-3 (setf lda var-3)))
            (cond
             ((< i m)
              (setf aii
                      (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
```

```
                                a-%offset%))
              (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                      one)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                  (dlarf "Right" (f2cl-lib:int-sub m i)
                   (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
                   (f2cl-lib:array-slice a-%data% double-float (i i)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda
                   (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                         tau-%offset%)
                   (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda work)
                (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8))
                (when var-4 (setf lda var-4))
                (when var-7 (setf lda var-7)))
              (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                      aii)))
          label10))
       (go end_label)
      end_label
       (return (values nil nil nil lda nil nil info))))))
```

———-

90.125 dgelqf

90.125.1 Axiom unit tests

— **dgelqf.input** —

```
)set break resume
)sys rm -f dgelqf.output
)spool dgelqf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.125.2 Axiom help page

— dgelqf.help —

```
====================================================================
dgelqf examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DGELQF - an LQ factorization of a real M-by-N matrix A

SYNOPSIS
       SUBROUTINE DGELQF( M, N, A, LDA, TAU, WORK, LWORK, INFO )

           INTEGER        INFO, LDA, LWORK, M, N

           DOUBLE         PRECISION A( LDA, * ), TAU( * ), WORK( * )

PURPOSE
       DGELQF  computes an LQ factorization of a real M-by-N matrix A: A = L *
       Q.


ARGUMENTS
       M       (input) INTEGER
               The number of rows of the matrix A.  M >= 0.

       N       (input) INTEGER
               The number of columns of the matrix A.  N >= 0.

       A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
               On entry, the M-by-N matrix A.  On exit, the  elements  on  and
               below the diagonal of the array contain the m-by-min(m,n) lower
               trapezoidal matrix L (L is lower triangular if  m  <=  n);  the
               elements  above the diagonal, with the array TAU, represent the
               orthogonal matrix Q as a product of elementary reflectors  (see
               Further  Details).   LDA     (input) INTEGER The leading dimen-
               sion of the array A.  LDA >= max(1,M).

       TAU     (output) DOUBLE PRECISION array, dimension (min(M,N))
               The scalar factors of the elementary  reflectors  (see  Further
               Details).

       WORK       (workspace/output)   DOUBLE   PRECISION   array,   dimension
       (MAX(1,LWORK))
```

```
               On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

       LWORK   (input) INTEGER
               The dimension of the array WORK.  LWORK >= max(1,M).  For opti-
               mum performance LWORK >= M*NB, where NB is the  optimal  block-
               size.

               If  LWORK  = -1, then a workspace query is assumed; the routine
               only calculates the optimal size of  the  WORK  array,  returns
               this  value  as the first entry of the WORK array, and no error
               message related to LWORK is issued by XERBLA.

       INFO    (output) INTEGER
               = 0:  successful exit
               < 0:  if INFO = -i, the i-th argument had an illegal value

FURTHER DETAILS
       The matrix Q is represented as a product of elementary reflectors

          Q = H(k) . . . H(2) H(1), where k = min(m,n).

       Each H(i) has the form

          H(i) = I - tau * v * v'

       where tau is a real scalar, and v is a real vector with
       v(1:i-1) = 0 and v(i) = 1; v(i+1:n) is stored on  exit  in  A(i,i+1:n),
       and tau in TAU(i).
```

———-

90.125.3 fortran code

— dgelqf.f —

```
*  =====================================================================
      SUBROUTINE DGELQF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
```

```
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, LDWORK, LWKOPT, NB,
     $                   NBMIN, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGELQ2, DLARFB, DLARFT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NB = ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
      LWKOPT = M*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, M ) .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGELQF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      K = MIN( M, N )
      IF( K.EQ.0 ) THEN
         WORK( 1 ) = 1
```

```
         RETURN
      END IF
*
      NBMIN = 2
      NX = 0
      IWS = M
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'DGELQF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = M
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'DGELQF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially
*
         DO 10 I = 1, K - NX, NB
            IB = MIN( K-I+1, NB )
*
*           Compute the LQ factorization of the current block
*           A(i:i+ib-1,i:n)
*
            CALL DGELQ2( IB, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                   IINFO )
            IF( I+IB.LE.M ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i) H(i+1) . . . H(i+ib-1)
*
               CALL DLARFT( 'Forward', 'Rowwise', N-I+1, IB, A( I, I ),
     $                      LDA, TAU( I ), WORK, LDWORK )
*
*              Apply H to A(i+ib:m,i:n) from the right
```

```
*
               CALL DLARFB( 'Right', 'No transpose', 'Forward',
     $                      'Rowwise', M-I-IB+1, N-I+1, IB, A( I, I ),
     $                      LDA, WORK, LDWORK, A( I+IB, I ), LDA,
     $                      WORK( IB+1 ), LDWORK )
            END IF
   10    CONTINUE
      ELSE
         I = 1
      END IF
*
*     Use unblocked code to factor the last or only block.
*
      IF( I.LE.K )
     $   CALL DGELQ2( M-I+1, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                IINFO )
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of DGELQF
*
      END
      SUBROUTINE DGELQF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, LDWORK, LWKOPT, NB,
     $                   NBMIN, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGELQ2, DLARFB, DLARFT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
```

```
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NB = ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
      LWKOPT = M*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, M ) .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGELQF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      K = MIN( M, N )
      IF( K.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      NX = 0
      IWS = M
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'DGELQF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = M
```

```
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'DGELQF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially
*
         DO 10 I = 1, K - NX, NB
            IB = MIN( K-I+1, NB )
*
*           Compute the LQ factorization of the current block
*           A(i:i+ib-1,i:n)
*
            CALL DGELQ2( IB, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                   IINFO )
            IF( I+IB.LE.M ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i) H(i+1) . . . H(i+ib-1)
*
               CALL DLARFT( 'Forward', 'Rowwise', N-I+1, IB, A( I, I ),
     $                      LDA, TAU( I ), WORK, LDWORK )
*
*              Apply H to A(i+ib:m,i:n) from the right
*
               CALL DLARFB( 'Right', 'No transpose', 'Forward',
     $                      'Rowwise', M-I-IB+1, N-I+1, IB, A( I, I ),
     $                      LDA, WORK, LDWORK, A( I+IB, I ), LDA,
     $                      WORK( IB+1 ), LDWORK )
            END IF
   10    CONTINUE
      ELSE
         I = 1
      END IF
*
*     Use unblocked code to factor the last or only block.
*
      IF( I.LE.K )
     $   CALL DGELQ2( M-I+1, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                IINFO )
```

```
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of DGELQF
*
      END
```

———-

90.125.4 lisp code

— **dgelqf.lisp** —

```
(defun dgelqf (m n a lda tau work lwork info)
  (declare (type (array double-float (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a double-float a-%data% a-%offset%)
       (tau double-float tau-%data% tau-%offset%)
       (work double-float work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ldwork 0) (lwkopt 0) (nb 0)
           (nbmin 0) (nx 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
               (type (f2cl-lib:integer4) nx nbmin nb lwkopt ldwork k iws iinfo
                ib i))
      (setf info 0)
      (setf nb
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 1 "DGELQF" " " m n -1 -1)
                (declare (ignore var-0 var-1 var-2 var-5 var-6))
                (when var-3 (setf m var-3))
                (when var-4 (setf n var-4))
                ret-val))
      (setf lwkopt (f2cl-lib:int-mul m nb))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -4))
            ((and
              (< lwork
                 (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
              (not lquery))
             (setf info -7)))
```

```
(cond
 ((/= info 0) (xerbla "DGELQF" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(cond
 ((= k 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'double-float))
  (go end_label)))
(setf nbmin 2)
(setf nx 0)
(setf iws m)
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "DGELQF" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (when var-3 (setf m var-3))
                      (when var-4 (setf n var-4))
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork m) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                             var-6)
                            (ilaenv 2 "DGELQF" " " m n -1 -1)
                          (declare
                           (ignore var-0 var-1 var-2 var-5 var-6))
                          (when var-3 (setf m var-3))
                          (when var-4 (setf n var-4))
                          ret-val))))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub nx))) nil)
    (tagbody
      (setf ib
              (min
               (the f2cl-lib:integer4
```

```
                (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
          (the f2cl-lib:integer4 nb)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dgelq2 ib (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
     (f2cl-lib:array-slice a-%data% double-float (i i)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                           tau-%offset%)
     work iinfo)
  (declare (ignore var-0 var-1 var-2 var-4 var-5))
  (setf lda var-3)
  (setf iinfo var-6))
(cond
 ((<= (f2cl-lib:int-add i ib) m)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dlarft "Forward" "Rowwise"
       (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) ib
       (f2cl-lib:array-slice a-%data% double-float (i i)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                             tau-%offset%)
       work ldwork)
    (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
    (when var-3 (setf ib var-3))
    (when var-5 (setf lda var-5))
    (when var-8 (setf ldwork var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (dlarfb "Right" "No transpose" "Forward" "Rowwise"
       (f2cl-lib:int-add (f2cl-lib:int-sub m i ib) 1)
       (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) ib
       (f2cl-lib:array-slice a-%data% double-float (i i)
                             ((1 lda) (1 *)) a-%offset%)
       lda work ldwork
       (f2cl-lib:array-slice a-%data% double-float ((+ i ib) i)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice work-%data% double-float ((+ ib 1))
                             ((1 *)) work-%offset%)
       ldwork)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
      var-13))
    (when var-6 (setf ib var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldwork var-10))
```

```
              (when var-12 (setf lda var-12))
              (when var-14 (setf ldwork var-14)))))
          label10)))
      (t (setf i 1)))
     (if (<= i k)
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
             (dgelq2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
              (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
              (f2cl-lib:array-slice a-%data% double-float (i i)
                                    ((1 lda) (1 *)) a-%offset%)
              lda
              (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                    tau-%offset%)
              work iinfo)
            (declare (ignore var-0 var-1 var-2 var-4 var-5))
            (setf lda var-3)
            (setf iinfo var-6)))
     (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
             (coerce (the f2cl-lib:integer4 iws) 'double-float))
     (go end_label)
    end_label
     (return (values m n nil lda nil nil nil info)))))
```

———-

90.126 dgeqr2

90.126.1 Axiom unit tests

— **dgeqr2.input** —

```
)set break resume
)sys rm -f dgeqr2.output
)spool dgeqr2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.126.2 Axiom help page

— dgeqr2.help —

```
====================================================================
dgeqr2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DGEQR2 - a QR factorization of a real m by n matrix A

SYNOPSIS
       SUBROUTINE DGEQR2( M, N, A, LDA, TAU, WORK, INFO )

           INTEGER        INFO, LDA, M, N

           DOUBLE         PRECISION A( LDA, * ), TAU( * ), WORK( * )

PURPOSE
       DGEQR2  computes  a QR factorization of a real m by n matrix A: A = Q *
       R.


ARGUMENTS
       M       (input) INTEGER
               The number of rows of the matrix A.  M >= 0.

       N       (input) INTEGER
               The number of columns of the matrix A.  N >= 0.

       A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
               On entry, the m by n matrix A.  On exit, the  elements  on  and
               above the diagonal of the array contain the min(m,n) by n upper
               trapezoidal matrix R (R is upper triangular if  m  >=  n);  the
               elements  below the diagonal, with the array TAU, represent the
               orthogonal matrix Q as a product of elementary reflectors  (see
               Further  Details).   LDA     (input) INTEGER The leading dimen-
               sion of the array A.  LDA >= max(1,M).

       TAU     (output) DOUBLE PRECISION array, dimension (min(M,N))
               The scalar factors of the elementary  reflectors  (see  Further
               Details).

       WORK    (workspace) DOUBLE PRECISION array, dimension (N)
```

```
       INFO    (output) INTEGER
               = 0: successful exit
               < 0: if INFO = -i, the i-th argument had an illegal value

FURTHER DETAILS
       The matrix Q is represented as a product of elementary reflectors

          Q = H(1) H(2) . . . H(k), where k = min(m,n).

       Each H(i) has the form

          H(i) = I - tau * v * v'

       where tau is a real scalar, and v is a real vector with
       v(1:i-1)  =  0  and v(i) = 1; v(i+1:m) is stored on exit in A(i+1:m,i),
       and tau in TAU(i).
```

———-

90.126.3 fortran code

— dgeqr2.f —

```
*  =====================================================================
      SUBROUTINE DGEQR2( M, N, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      DOUBLE PRECISION   AII
```

```
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARF, DLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEQR2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = 1, K
*
*        Generate elementary reflector H(i) to annihilate A(i+1:m,i)
*
         CALL DLARFG( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
     $                TAU( I ) )
         IF( I.LT.N ) THEN
*
*           Apply H(i) to A(i:m,i+1:n) from the left
*
            AII = A( I, I )
            A( I, I ) = ONE
            CALL DLARF( 'Left', M-I+1, N-I, A( I, I ), 1, TAU( I ),
     $                  A( I, I+1 ), LDA, WORK )
            A( I, I ) = AII
         END IF
   10 CONTINUE
      RETURN
*
*     End of DGEQR2
*
      END
      SUBROUTINE DGEQR2( M, N, A, LDA, TAU, WORK, INFO )
*
```

```
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     February 29, 1992
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, K
      DOUBLE PRECISION   AII
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARF, DLARFG, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEQR2', -INFO )
         RETURN
      END IF
*
      K = MIN( M, N )
*
      DO 10 I = 1, K
*
*        Generate elementary reflector H(i) to annihilate A(i+1:m,i)
```

```
*
         CALL DLARFG( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
     $               TAU( I ) )
         IF( I.LT.N ) THEN
*
*           Apply H(i) to A(i:m,i+1:n) from the left
*
            AII = A( I, I )
            A( I, I ) = ONE
            CALL DLARF( 'Left', M-I+1, N-I, A( I, I ), 1, TAU( I ),
     $                  A( I, I+1 ), LDA, WORK )
            A( I, I ) = AII
         END IF
   10 CONTINUE
      RETURN
*
*     End of DGEQR2
*
      END
```

———-

90.126.4 lisp code

— **dgeqr2.lisp** —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dgeqr2 (m n a lda tau work info)
    (declare (type (array double-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((aii 0.0d0) (i 0) (k 0))
        (declare (type (double-float) aii)
                 (type (f2cl-lib:integer4) i k))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "DGEQR2" (f2cl-lib:int-sub info))
          (go end_label)))
```

```
 (setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i k) nil)
   (tagbody
     (dlarfg (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
      (f2cl-lib:array-slice a-%data% double-float (i i) ((1 lda) (1 *))
                            a-%offset%)
      (f2cl-lib:array-slice a-%data% double-float
                            ((min (f2cl-lib:int-add i 1) m) i)
                            ((1 lda) (1 *)) a-%offset%)
      1
      (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                            tau-%offset%))
     (cond
      ((< i n)
       (setf aii
               (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                              a-%offset%))
       (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
               one)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
           (dlarf "Left" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
            (f2cl-lib:int-sub n i)
            (f2cl-lib:array-slice a-%data% double-float (i i)
                                  ((1 lda) (1 *)) a-%offset%)
            1
            (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                  tau-%offset%)
            (f2cl-lib:array-slice a-%data% double-float
                                  (i (f2cl-lib:int-add i 1))
                                  ((1 lda) (1 *)) a-%offset%)
            lda work)
         (declare
          (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
         (when var-7 (setf lda var-7)))
       (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
               aii)))
   label10))
 (go end_label)
end_label
 (return (values nil nil nil lda nil nil info))))))
```

———-

90.127 dgeqrf

90.127.1 Axiom unit tests

— **dgeqrf.input** —

```
)set break resume
)sys rm -f dgeqrf.output
)spool dgeqrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.127.2 Axiom help page

— **dgeqrf.help** —

```
======================================================================
dgeqrf examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DGEQRF - a QR factorization of a real M-by-N matrix A

SYNOPSIS
       SUBROUTINE DGEQRF( M, N, A, LDA, TAU, WORK, LWORK, INFO )

           INTEGER        INFO, LDA, LWORK, M, N

           DOUBLE         PRECISION A( LDA, * ), TAU( * ), WORK( * )

PURPOSE
       DGEQRF  computes  a QR factorization of a real M-by-N matrix A: A = Q *
       R.


ARGUMENTS
```

```
M       (input) INTEGER
        The number of rows of the matrix A.  M >= 0.

N       (input) INTEGER
        The number of columns of the matrix A.  N >= 0.

A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
        On entry, the M-by-N matrix A.  On exit, the  elements  on  and
        above the diagonal of the array contain the min(M,N)-by-N upper
        trapezoidal matrix R (R is upper triangular if  m  >=  n);  the
        elements  below the diagonal, with the array TAU, represent the
        orthogonal matrix Q as a product of min(m,n) elementary reflec-
        tors (see Further Details).

LDA     (input) INTEGER
        The leading dimension of the array A.  LDA >= max(1,M).

TAU     (output) DOUBLE PRECISION array, dimension (min(M,N))
        The  scalar  factors  of the elementary reflectors (see Further
        Details).

WORK      (workspace/output)   DOUBLE   PRECISION   array,    dimension
(MAX(1,LWORK))
        On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK   (input) INTEGER
        The dimension of the array WORK.  LWORK >= max(1,N).  For opti-
        mum  performance  LWORK >= N*NB, where NB is the optimal block-
        size.

        If LWORK = -1, then a workspace query is assumed;  the  routine
        only  calculates  the  optimal  size of the WORK array, returns
        this value as the first entry of the WORK array, and  no  error
        message related to LWORK is issued by XERBLA.

INFO    (output) INTEGER
        = 0:  successful exit
        < 0:  if INFO = -i, the i-th argument had an illegal value

FURTHER DETAILS
       The matrix Q is represented as a product of elementary reflectors

          Q = H(1) H(2) . . . H(k), where k = min(m,n).

       Each H(i) has the form

          H(i) = I - tau * v * v'

       where tau is a real scalar, and v is a real vector with
       v(1:i-1)  =  0  and v(i) = 1; v(i+1:m) is stored on exit in A(i+1:m,i),
```

and tau in TAU(i).

———-

90.127.3 fortran code

— **dgeqrf.f** —

```
*  =====================================================================
      SUBROUTINE DGEQRF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, LDWORK, LWKOPT, NB,
     $                   NBMIN, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEQR2, DLARFB, DLARFT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NB = ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
      LWKOPT = N*NB
```

```
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEQRF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      K = MIN( M, N )
      IF( K.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      NX = 0
      IWS = N
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'DGEQRF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = N
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'DGEQRF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
```

```
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially
*
         DO 10 I = 1, K - NX, NB
            IB = MIN( K-I+1, NB )
*
*           Compute the QR factorization of the current block
*           A(i:m,i:i+ib-1)
*
            CALL DGEQR2( M-I+1, IB, A( I, I ), LDA, TAU( I ), WORK,
     $                   IINFO )
            IF( I+IB.LE.N ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i) H(i+1) . . . H(i+ib-1)
*
               CALL DLARFT( 'Forward', 'Columnwise', M-I+1, IB,
     $                      A( I, I ), LDA, TAU( I ), WORK, LDWORK )
*
*              Apply H**T to A(i:m,i+ib:n) from the left
*
               CALL DLARFB( 'Left', 'Transpose', 'Forward',
     $                      'Columnwise', M-I+1, N-I-IB+1, IB,
     $                      A( I, I ), LDA, WORK, LDWORK, A( I, I+IB ),
     $                      LDA, WORK( IB+1 ), LDWORK )
            END IF
   10    CONTINUE
      ELSE
         I = 1
      END IF
*
*     Use unblocked code to factor the last or only block.
*
      IF( I.LE.K )
     $   CALL DGEQR2( M-I+1, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                IINFO )
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of DGEQRF
*
      END
      SUBROUTINE DGEQRF( M, N, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
```

```
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, K, LDWORK, LWKOPT, NB,
     $                   NBMIN, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEQR2, DLARFB, DLARFT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NB = ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
      LWKOPT = N*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGEQRF', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
```

```
      END IF
*
*     Quick return if possible
*
      K = MIN( M, N )
      IF( K.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      NX = 0
      IWS = N
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'DGEQRF', ' ', M, N, -1, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = N
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'DGEQRF', ' ', M, N, -1,
     $                 -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code initially
*
         DO 10 I = 1, K - NX, NB
            IB = MIN( K-I+1, NB )
*
*           Compute the QR factorization of the current block
*           A(i:m,i:i+ib-1)
*
            CALL DGEQR2( M-I+1, IB, A( I, I ), LDA, TAU( I ), WORK,
     $                   IINFO )
            IF( I+IB.LE.N ) THEN
*
```

```
*              Form the triangular factor of the block reflector
*              H = H(i) H(i+1) . . . H(i+ib-1)
*
               CALL DLARFT( 'Forward', 'Columnwise', M-I+1, IB,
     $                      A( I, I ), LDA, TAU( I ), WORK, LDWORK )
*
*              Apply H' to A(i:m,i+ib:n) from the left
*
               CALL DLARFB( 'Left', 'Transpose', 'Forward',
     $                      'Columnwise', M-I+1, N-I-IB+1, IB,
     $                      A( I, I ), LDA, WORK, LDWORK, A( I, I+IB ),
     $                      LDA, WORK( IB+1 ), LDWORK )
            END IF
   10    CONTINUE
      ELSE
         I = 1
      END IF
*
*     Use unblocked code to factor the last or only block.
*
      IF( I.LE.K )
     $   CALL DGEQR2( M-I+1, N-I+1, A( I, I ), LDA, TAU( I ), WORK,
     $                IINFO )
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of DGEQRF
*
      END
```

90.127.4 lisp code

— dgeqrf.lisp —

```
(defun dgeqrf (m n a lda tau work lwork info)
  (declare (type (array double-float (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda n m))
  (f2cl-lib:with-multi-array-data
      ((a double-float a-%data% a-%offset%)
       (tau double-float tau-%data% tau-%offset%)
       (work double-float work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (k 0) (ldwork 0) (lwkopt 0) (nb 0)
           (nbmin 0) (nx 0) (lquery nil))
      (declare (type f2cl-lib:logical lquery)
```

```
         (type (f2cl-lib:integer4) nx nbmin nb lwkopt ldwork k iws iinfo
          ib i))
(setf info 0)
(setf nb
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 1 "DGEQRF" " " m n -1 -1)
          (declare (ignore var-0 var-1 var-2 var-5 var-6))
          (when var-3 (setf m var-3))
          (when var-4 (setf n var-4))
          ret-val))
(setf lwkopt (f2cl-lib:int-mul n nb))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
      ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -4))
      ((and
        (< lwork
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (not lquery))
       (setf info -7)))
(cond
 ((/= info 0) (xerbla "DGEQRF" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(setf k (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
(cond
 ((= k 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'double-float))
  (go end_label)))
(setf nbmin 2)
(setf nx 0)
(setf iws n)
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "DGEQRF" " " m n -1 -1)
                      (declare (ignore var-0 var-1 var-2 var-5 var-6))
                      (when var-3 (setf m var-3))
                      (when var-4 (setf n var-4))
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork n) (setf iws (f2cl-lib:int-mul ldwork nb))
```

```
      (cond
       ((< lwork iws)
        (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
        (setf nbmin
                (max (the f2cl-lib:integer4 2)
                     (the f2cl-lib:integer4
                          (multiple-value-bind
                              (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                               var-6)
                              (ilaenv 2 "DGEQRF" " " m n -1 -1)
                            (declare
                             (ignore var-0 var-1 var-2 var-5 var-6))
                            (when var-3 (setf m var-3))
                            (when var-4 (setf n var-4))
                            ret-val))))))))))
  (cond
   ((and (>= nb nbmin) (< nb k) (< nx k))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i nb))
                  ((> i (f2cl-lib:int-add k (f2cl-lib:int-sub nx))) nil)
      (tagbody
        (setf ib
                (min
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))
                 (the f2cl-lib:integer4 nb)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dgeqr2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
             (f2cl-lib:array-slice a-%data% double-float (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                   tau-%offset%)
             work iinfo)
          (declare (ignore var-0 var-1 var-2 var-4 var-5))
          (setf lda var-3)
          (setf iinfo var-6))
        (cond
         ((<= (f2cl-lib:int-add i ib) n)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (dlarft "Forward" "Columnwise"
               (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
               (f2cl-lib:array-slice a-%data% double-float (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                     tau-%offset%)
               work ldwork)
            (declare (ignore var-0 var-1 var-2 var-4 var-6 var-7))
            (when var-3 (setf ib var-3))
```

```
            (when var-5 (setf lda var-5))
            (when var-8 (setf ldwork var-8)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10 var-11 var-12 var-13 var-14)
              (dlarfb "Left" "Transpose" "Forward" "Columnwise"
               (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
               (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1) ib
               (f2cl-lib:array-slice a-%data% double-float (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               lda work ldwork
               (f2cl-lib:array-slice a-%data% double-float
                                     (i (f2cl-lib:int-add i ib))
                                     ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice work-%data% double-float ((+ ib 1))
                                     ((1 *)) work-%offset%)
               ldwork)
            (declare
             (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9 var-11
              var-13))
            (when var-6 (setf ib var-6))
            (when var-8 (setf lda var-8))
            (when var-10 (setf ldwork var-10))
            (when var-12 (setf lda var-12))
            (when var-14 (setf ldwork var-14)))))
      label10)))
  (t (setf i 1)))
 (if (<= i k)
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (dgeqr2 (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
          (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
          (f2cl-lib:array-slice a-%data% double-float (i i)
                                ((1 lda) (1 *)) a-%offset%)
          lda
          (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                tau-%offset%)
          work iinfo)
       (declare (ignore var-0 var-1 var-2 var-4 var-5))
       (setf lda var-3)
       (setf iinfo var-6)))
 (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
         (coerce (the f2cl-lib:integer4 iws) 'double-float))
 (go end_label)
end_label
 (return (values m n nil lda nil nil nil info)))))
```

———-

90.128 dgesdd

90.128.1 Axiom unit tests

— dgesdd.input —

```
)set break resume
)sys rm -f dgesdd.output
)spool dgesdd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.128.2 Axiom help page

— dgesdd.help —

```
====================================================================
dgesdd examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DGESDD - the singular value decomposition (SVD) of a real M-by-N matrix
       A, optionally computing the left and right singular vectors

SYNOPSIS
       SUBROUTINE DGESDD( JOBZ, M, N, A, LDA,  S,  U,  LDU,  VT,  LDVT,  WORK,
                          LWORK, IWORK, INFO )

           CHARACTER      JOBZ

           INTEGER        INFO, LDA, LDU, LDVT, LWORK, M, N

           INTEGER        IWORK( * )

           DOUBLE         PRECISION  A(  LDA,  *  ),  S( * ), U( LDU, * ), VT(
                          LDVT, * ), WORK( * )
```

```
PURPOSE
       DGESDD computes the singular value decomposition (SVD) of a real M-by-N
       matrix A, optionally computing the left and right singular vectors.  If
       singular vectors are desired, it uses a divide-and-conquer algorithm.

       The SVD is written

            A = U * SIGMA * transpose(V)

       where SIGMA is an M-by-N matrix which is zero except for  its  min(m,n)
       diagonal elements, U is an M-by-M orthogonal matrix, and V is an N-by-N
       orthogonal matrix.  The diagonal elements of  SIGMA  are  the  singular
       values  of  A;  they  are  real  and  non-negative, and are returned in
       descending order.  The first min(m,n) columns of U and V are  the  left
       and right singular vectors of A.

       Note that the routine returns VT = V**T, not V.

       The  divide  and  conquer  algorithm  makes very mild assumptions about
       floating point arithmetic. It will work on machines with a guard  digit
       in add/subtract, or on those binary machines without guard digits which
       subtract like the Cray X-MP, Cray Y-MP, Cray C-90, or Cray-2. It  could
       conceivably  fail on hexadecimal or decimal machines without guard dig-
       its, but we know of none.


ARGUMENTS
       JOBZ    (input) CHARACTER*1
               Specifies options for computing all or part of the matrix U:
               = 'A':  all M columns of U and all N rows of V**T are  returned
               in the arrays U and VT; = 'S':  the first min(M,N) columns of U
               and the first min(M,N) rows of V**T are returned in the  arrays
               U and VT; = 'O':  If M >= N, the first N columns of U are over-
               written on the array A and all rows of V**T are returned in the
               array VT; otherwise, all columns of U are returned in the array
               U and the first M rows of V**T are overwritten in the array  A;
               = 'N':  no columns of U or rows of V**T are computed.

       M       (input) INTEGER
               The number of rows of the input matrix A.  M >= 0.

       N       (input) INTEGER
               The number of columns of the input matrix A.  N >= 0.

       A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
               On  entry,  the M-by-N matrix A.  On exit, if JOBZ = 'O',  A is
               overwritten with the first N columns of U  (the  left  singular
               vectors,  stored  columnwise)  if M >= N; A is overwritten with
               the first M rows of V**T (the right  singular  vectors,  stored
```

```
        rowwise)  otherwise.   if  JOBZ .ne. 'O', the contents of A are
        destroyed.

LDA     (input) INTEGER
        The leading dimension of the array A.  LDA >= max(1,M).

S       (output) DOUBLE PRECISION array, dimension (min(M,N))
        The singular values of A, sorted so that S(i) >= S(i+1).

U       (output) DOUBLE PRECISION array, dimension (LDU,UCOL)
        UCOL = M if JOBZ = 'A' or JOBZ = 'O' and M < N; UCOL = min(M,N)
        if  JOBZ  = 'S'.  If JOBZ = 'A' or JOBZ = 'O' and M < N, U con-
        tains the M-by-M orthogonal matrix U; if JOBZ = 'S', U contains
        the  first  min(M,N)  columns  of U (the left singular vectors,
        stored columnwise); if JOBZ = 'O' and M >= N, or JOBZ = 'N',  U
        is not referenced.

LDU     (input) INTEGER
        The  leading dimension of the array U.  LDU >= 1; if JOBZ = 'S'
        or 'A' or JOBZ = 'O' and M < N, LDU >= M.

VT      (output) DOUBLE PRECISION array, dimension (LDVT,N)
        If JOBZ = 'A' or JOBZ = 'O' and M >= N, VT contains the  N-by-N
        orthogonal  matrix  V**T;  if JOBZ = 'S', VT contains the first
        min(M,N) rows of V**T (the right singular vectors, stored  row-
        wise); if JOBZ = 'O' and M < N, or JOBZ = 'N', VT is not refer-
        enced.

LDVT    (input) INTEGER
        The leading dimension of the array VT.  LDVT >= 1;  if  JOBZ  =
        'A' or JOBZ = 'O' and M >= N, LDVT >= N; if JOBZ = 'S', LDVT >=
        min(M,N).

WORK      (workspace/output)   DOUBLE   PRECISION   array,    dimension
(MAX(1,LWORK))
        On exit, if INFO = 0, WORK(1) returns the optimal LWORK;

LWORK   (input) INTEGER
        The dimension of the array WORK. LWORK >= 1.  If  JOBZ  =  'N',
        LWORK >= 3*min(M,N) + max(max(M,N),7*min(M,N)).  If JOBZ = 'O',
        LWORK            >=            3*min(M,N)*min(M,N)            +
        max(max(M,N),5*min(M,N)*min(M,N)+4*min(M,N)).  If JOBZ = 'S' or
        'A'        LWORK        >=        3*min(M,N)*min(M,N)         +
        max(max(M,N),4*min(M,N)*min(M,N)+4*min(M,N)).  For good perfor-
        mance, LWORK should generally be larger.  If  LWORK  =  -1  but
        other  input  arguments  are legal, WORK(1) returns the optimal
        LWORK.

IWORK   (workspace) INTEGER array, dimension (8*min(M,N))
```

```
      INFO    (output) INTEGER
              = 0:  successful exit.
              < 0:  if INFO = -i, the i-th argument had an illegal value.
              > 0:  DBDSDC did not converge, updating process failed.

  Further Details
  ===============

  Based on contributions by
     Ming Gu and Huan Ren, Computer Science Division, University of
     California at Berkeley, USA
```

———-

90.128.3 fortran code

— **dgesdd.f** —

```
*  =====================================================================
      SUBROUTINE DGESDD( JOBZ, M, N, A, LDA, S, U, LDU, VT, LDVT, WORK,
     $                   LWORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      CHARACTER          JOBZ
      INTEGER            INFO, LDA, LDU, LDVT, LWORK, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), S( * ), U( LDU, * ),
     $                   VT( LDVT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, WNTQA, WNTQAS, WNTQN, WNTQO, WNTQS
      INTEGER            BDSPAC, BLK, CHUNK, I, IE, IERR, IL,
     $                   IR, ISCL, ITAU, ITAUP, ITAUQ, IU, IVT, LDWKVT,
```

```
     $                   LDWRKL, LDWRKR, LDWRKU, MAXWRK, MINMN, MINWRK,
     $                   MNTHR, NWORK, WRKBL
      DOUBLE PRECISION   ANRM, BIGNUM, EPS, SMLNUM
*     ..
*     .. Local Arrays ..
      INTEGER            IDUM( 1 )
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DBDSDC, DGEBRD, DGELQF, DGEMM, DGEQRF, DLACPY,
     $                   DLASCL, DLASET, DORGBR, DORGLQ, DORGQR, DORMBR,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           DLAMCH, DLANGE, ILAENV, LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          INT, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      MINMN = MIN( M, N )
      WNTQA = LSAME( JOBZ, 'A' )
      WNTQS = LSAME( JOBZ, 'S' )
      WNTQAS = WNTQA .OR. WNTQS
      WNTQO = LSAME( JOBZ, 'O' )
      WNTQN = LSAME( JOBZ, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
*
      IF( .NOT.( WNTQA .OR. WNTQS .OR. WNTQO .OR. WNTQN ) ) THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDU.LT.1 .OR. ( WNTQAS .AND. LDU.LT.M ) .OR.
     $         ( WNTQO .AND. M.LT.N .AND. LDU.LT.M ) ) THEN
         INFO = -8
      ELSE IF( LDVT.LT.1 .OR. ( WNTQA .AND. LDVT.LT.N ) .OR.
     $         ( WNTQS .AND. LDVT.LT.MINMN ) .OR.
     $         ( WNTQO .AND. M.GE.N .AND. LDVT.LT.N ) ) THEN
         INFO = -10
```

```
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = 1
         MAXWRK = 1
         IF( M.GE.N .AND. MINMN.GT.0 ) THEN
*
*           Compute space needed for DBDSDC
*
            MNTHR = INT( MINMN*11.0D0 / 6.0D0 )
            IF( WNTQN ) THEN
               BDSPAC = 7*N
            ELSE
               BDSPAC = 3*N*N + 4*N
            END IF
            IF( M.GE.MNTHR ) THEN
               IF( WNTQN ) THEN
*
*                 Path 1 (M much larger than N, JOBZ='N')
*
                  WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1,
     $                    -1 )
                  WRKBL = MAX( WRKBL, 3*N+2*N*
     $                    ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                  MAXWRK = MAX( WRKBL, BDSPAC+N )
                  MINWRK = BDSPAC + N
               ELSE IF( WNTQO ) THEN
*
*                 Path 2 (M much larger than N, JOBZ='O')
*
                  WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
                  WRKBL = MAX( WRKBL, N+N*ILAENV( 1, 'DORGQR', ' ', M,
     $                    N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 3*N+2*N*
     $                    ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                  WRKBL = MAX( WRKBL, 3*N+N*
     $                    ILAENV( 1, 'DORMBR', 'QLN', N, N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 3*N+N*
     $                    ILAENV( 1, 'DORMBR', 'PRT', N, N, N, -1 ) )
                  WRKBL = MAX( WRKBL, BDSPAC+3*N )
                  MAXWRK = WRKBL + 2*N*N
                  MINWRK = BDSPAC + 2*N*N + 3*N
               ELSE IF( WNTQS ) THEN
```

```
*
*                Path 3 (M much larger than N, JOBZ='S')
*
                 WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, N+N*ILAENV( 1, 'DORGQR', ' ', M,
     $                   N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+2*N*
     $                   ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'QLN', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'PRT', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*N )
                 MAXWRK = WRKBL + N*N
                 MINWRK = BDSPAC + N*N + 3*N
              ELSE IF( WNTQA ) THEN
*
*                Path 4 (M much larger than N, JOBZ='A')
*
                 WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, N+M*ILAENV( 1, 'DORGQR', ' ', M,
     $                   M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+2*N*
     $                   ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'QLN', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'PRT', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*N )
                 MAXWRK = WRKBL + N*N
                 MINWRK = BDSPAC + N*N + 2*N + M
              END IF
           ELSE
*
*             Path 5 (M at least N, but not much larger)
*
              WRKBL = 3*N + ( M+N )*ILAENV( 1, 'DGEBRD', ' ', M, N, -1,
     $                -1 )
              IF( WNTQN ) THEN
                 MAXWRK = MAX( WRKBL, BDSPAC+3*N )
                 MINWRK = 3*N + MAX( M, BDSPAC )
              ELSE IF( WNTQO ) THEN
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'PRT', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*N )
                 MAXWRK = WRKBL + M*N
                 MINWRK = 3*N + MAX( M, N*N+BDSPAC )
              ELSE IF( WNTQS ) THEN
```

```
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'PRT', N, N, N, -1 ) )
                 MAXWRK = MAX( WRKBL, BDSPAC+3*N )
                 MINWRK = 3*N + MAX( M, BDSPAC )
              ELSE IF( WNTQA ) THEN
                 WRKBL = MAX( WRKBL, 3*N+M*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'PRT', N, N, N, -1 ) )
                 MAXWRK = MAX( MAXWRK, BDSPAC+3*N )
                 MINWRK = 3*N + MAX( M, BDSPAC )
              END IF
           END IF
        ELSE IF( MINMN.GT.0 ) THEN
*
*          Compute space needed for DBDSDC
*
           MNTHR = INT( MINMN*11.0D0 / 6.0D0 )
           IF( WNTQN ) THEN
              BDSPAC = 7*M
           ELSE
              BDSPAC = 3*M*M + 4*M
           END IF
           IF( N.GE.MNTHR ) THEN
              IF( WNTQN ) THEN
*
*                Path 1t (N much larger than M, JOBZ='N')
*
                 WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1,
     $                   -1 )
                 WRKBL = MAX( WRKBL, 3*M+2*M*
     $                   ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
                 MAXWRK = MAX( WRKBL, BDSPAC+M )
                 MINWRK = BDSPAC + M
              ELSE IF( WNTQO ) THEN
*
*                Path 2t (N much larger than M, JOBZ='O')
*
                 WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, M+M*ILAENV( 1, 'DORGLQ', ' ', M,
     $                   N, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+2*M*
     $                   ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'PRT', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*M )
```

```
                 MAXWRK = WRKBL + 2*M*M
                 MINWRK = BDSPAC + 2*M*M + 3*M
              ELSE IF( WNTQS ) THEN
*
*                Path 3t (N much larger than M, JOBZ='S')
*
                 WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, M+M*ILAENV( 1, 'DORGLQ', ' ', M,
     $                   N, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+2*M*
     $                   ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'PRT', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*M )
                 MAXWRK = WRKBL + M*M
                 MINWRK = BDSPAC + M*M + 3*M
              ELSE IF( WNTQA ) THEN
*
*                Path 4t (N much larger than M, JOBZ='A')
*
                 WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, M+N*ILAENV( 1, 'DORGLQ', ' ', N,
     $                   N, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+2*M*
     $                   ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'PRT', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*M )
                 MAXWRK = WRKBL + M*M
                 MINWRK = BDSPAC + M*M + 3*M
              END IF
           ELSE
*
*             Path 5t (N greater than M, but not much larger)
*
              WRKBL = 3*M + ( M+N )*ILAENV( 1, 'DGEBRD', ' ', M, N, -1,
     $                -1 )
              IF( WNTQN ) THEN
                 MAXWRK = MAX( WRKBL, BDSPAC+3*M )
                 MINWRK = 3*M + MAX( N, BDSPAC )
              ELSE IF( WNTQO ) THEN
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'PRT', M, N, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*M )
```

```
                 MAXWRK = WRKBL + M*N
                 MINWRK = 3*M + MAX( N, M*M+BDSPAC )
              ELSE IF( WNTQS ) THEN
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'PRT', M, N, M, -1 ) )
                 MAXWRK = MAX( WRKBL, BDSPAC+3*M )
                 MINWRK = 3*M + MAX( N, BDSPAC )
              ELSE IF( WNTQA ) THEN
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'PRT', N, N, M, -1 ) )
                 MAXWRK = MAX( WRKBL, BDSPAC+3*M )
                 MINWRK = 3*M + MAX( N, BDSPAC )
              END IF
           END IF
        END IF
        MAXWRK = MAX( MAXWRK, MINWRK )
        WORK( 1 ) = MAXWRK
*
        IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
           INFO = -12
        END IF
     END IF
*
     IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'DGESDD', -INFO )
        RETURN
     ELSE IF( LQUERY ) THEN
        RETURN
     END IF
*
*    Quick return if possible
*
     IF( M.EQ.0 .OR. N.EQ.0 ) THEN
        RETURN
     END IF
*
*    Get machine constants
*
     EPS = DLAMCH( 'P' )
     SMLNUM = SQRT( DLAMCH( 'S' ) ) / EPS
     BIGNUM = ONE / SMLNUM
*
*    Scale A if max element outside range [SMLNUM,BIGNUM]
*
     ANRM = DLANGE( 'M', M, N, A, LDA, DUM )
     ISCL = 0
```

```
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ISCL = 1
         CALL DLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, IERR )
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ISCL = 1
         CALL DLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, IERR )
      END IF
*
      IF( M.GE.N ) THEN
*
*        A has at least as many rows as columns. If A has sufficiently
*        more rows than columns, first reduce using the QR
*        decomposition (if sufficient workspace available)
*
         IF( M.GE.MNTHR ) THEN
*
            IF( WNTQN ) THEN
*
*              Path 1 (M much larger than N, JOBZ='N')
*              No singular vectors to be computed
*
               ITAU = 1
               NWORK = ITAU + N
*
*              Compute A=Q*R
*              (Workspace: need 2*N, prefer N+N*NB)
*
               CALL DGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Zero out below R
*
               CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ), LDA )
               IE = 1
               ITAUQ = IE + N
               ITAUP = ITAUQ + N
               NWORK = ITAUP + N
*
*              Bidiagonalize R in A
*              (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
               CALL DGEBRD( N, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                      IERR )
               NWORK = IE + N
*
*              Perform bidiagonal SVD, computing singular values only
*              (Workspace: need N+BDSPAC)
*
               CALL DBDSDC( 'U', 'N', N, S, WORK( IE ), DUM, 1, DUM, 1,
```

```
     $                     DUM, IDUM, WORK( NWORK ), IWORK, INFO )
*
            ELSE IF( WNTQO ) THEN
*
*              Path 2 (M much larger than N, JOBZ = 'O')
*              N left singular vectors to be overwritten on A and
*              N right singular vectors to be computed in VT
*
               IR = 1
*
*              WORK(IR) is LDWRKR by N
*
               IF( LWORK.GE.LDA*N+N*N+3*N+BDSPAC ) THEN
                  LDWRKR = LDA
               ELSE
                  LDWRKR = ( LWORK-N*N-3*N-BDSPAC ) / N
               END IF
               ITAU = IR + LDWRKR*N
               NWORK = ITAU + N
*
*              Compute A=Q*R
*              (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
               CALL DGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*              Copy R to WORK(IR), zeroing out below it
*
               CALL DLACPY( 'U', N, N, A, LDA, WORK( IR ), LDWRKR )
               CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, WORK( IR+1 ),
     $                     LDWRKR )
*
*              Generate Q in A
*              (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
               CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
               IE = ITAU
               ITAUQ = IE + N
               ITAUP = ITAUQ + N
               NWORK = ITAUP + N
*
*              Bidiagonalize R in VT, copying result to WORK(IR)
*              (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
               CALL DGEBRD( N, N, WORK( IR ), LDWRKR, S, WORK( IE ),
     $                     WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*              WORK(IU) is N by N
```

```
*
               IU = NWORK
               NWORK = IU + N*N
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in WORK(IU) and computing right
*              singular vectors of bidiagonal matrix in VT
*              (Workspace: need N+N*N+BDSPAC)
*
               CALL DBDSDC( 'U', 'I', N, S, WORK( IE ), WORK( IU ), N,
     $                      VT, LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                      INFO )
*
*              Overwrite WORK(IU) by left singular vectors of R
*              and VT by right singular vectors of R
*              (Workspace: need 2*N*N+3*N, prefer 2*N*N+2*N+N*NB)
*
               CALL DORMBR( 'Q', 'L', 'N', N, N, N, WORK( IR ), LDWRKR,
     $                      WORK( ITAUQ ), WORK( IU ), N, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
               CALL DORMBR( 'P', 'R', 'T', N, N, N, WORK( IR ), LDWRKR,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Multiply Q in A by left singular vectors of R in
*              WORK(IU), storing result in WORK(IR) and copying to A
*              (Workspace: need 2*N*N, prefer N*N+M*N)
*
               DO 10 I = 1, M, LDWRKR
                  CHUNK = MIN( M-I+1, LDWRKR )
                  CALL DGEMM( 'N', 'N', CHUNK, N, N, ONE, A( I, 1 ),
     $                        LDA, WORK( IU ), N, ZERO, WORK( IR ),
     $                        LDWRKR )
                  CALL DLACPY( 'F', CHUNK, N, WORK( IR ), LDWRKR,
     $                         A( I, 1 ), LDA )
   10          CONTINUE
*
            ELSE IF( WNTQS ) THEN
*
*              Path 3 (M much larger than N, JOBZ='S')
*              N left singular vectors to be computed in U and
*              N right singular vectors to be computed in VT
*
               IR = 1
*
*              WORK(IR) is N by N
*
               LDWRKR = N
               ITAU = IR + LDWRKR*N
               NWORK = ITAU + N
```

```
*
*               Compute A=Q*R
*               (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                CALL DGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                       LWORK-NWORK+1, IERR )
*
*               Copy R to WORK(IR), zeroing out below it
*
                CALL DLACPY( 'U', N, N, A, LDA, WORK( IR ), LDWRKR )
                CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, WORK( IR+1 ),
     $                       LDWRKR )
*
*               Generate Q in A
*               (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                       WORK( NWORK ), LWORK-NWORK+1, IERR )
                IE = ITAU
                ITAUQ = IE + N
                ITAUP = ITAUQ + N
                NWORK = ITAUP + N
*
*               Bidiagonalize R in WORK(IR)
*               (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                CALL DGEBRD( N, N, WORK( IR ), LDWRKR, S, WORK( IE ),
     $                       WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                       LWORK-NWORK+1, IERR )
*
*               Perform bidiagonal SVD, computing left singular vectors
*               of bidiagoal matrix in U and computing right singular
*               vectors of bidiagonal matrix in VT
*               (Workspace: need N+BDSPAC)
*
                CALL DBDSDC( 'U', 'I', N, S, WORK( IE ), U, LDU, VT,
     $                       LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                       INFO )
*
*               Overwrite U by left singular vectors of R and VT
*               by right singular vectors of R
*               (Workspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*
                CALL DORMBR( 'Q', 'L', 'N', N, N, N, WORK( IR ), LDWRKR,
     $                       WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                       LWORK-NWORK+1, IERR )
*
                CALL DORMBR( 'P', 'R', 'T', N, N, N, WORK( IR ), LDWRKR,
     $                       WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                       LWORK-NWORK+1, IERR )
```

```
*
*             Multiply Q in A by left singular vectors of R in
*             WORK(IR), storing result in U
*             (Workspace: need N*N)
*
              CALL DLACPY( 'F', N, N, U, LDU, WORK( IR ), LDWRKR )
              CALL DGEMM( 'N', 'N', M, N, N, ONE, A, LDA, WORK( IR ),
     $                    LDWRKR, ZERO, U, LDU )
*
            ELSE IF( WNTQA ) THEN
*
*             Path 4 (M much larger than N, JOBZ='A')
*             M left singular vectors to be computed in U and
*             N right singular vectors to be computed in VT
*
              IU = 1
*
*             WORK(IU) is N by N
*
              LDWRKU = N
              ITAU = IU + LDWRKU*N
              NWORK = ITAU + N
*
*             Compute A=Q*R, copying result to U
*             (Workspace: need N*N+N+M, prefer N*N+N+M*NB)
*
              CALL DGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
              CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*             Generate Q in U
*             (Workspace: need N*N+N+M, prefer N*N+N+M*NB)
              CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*             Produce R in A, zeroing out other entries
*
              CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ), LDA )
              IE = ITAU
              ITAUQ = IE + N
              ITAUP = ITAUQ + N
              NWORK = ITAUP + N
*
*             Bidiagonalize R in A
*             (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
              CALL DGEBRD( N, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                     WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                     IERR )
*
```

```
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in WORK(IU) and computing right
*              singular vectors of bidiagonal matrix in VT
*              (Workspace: need N+N*N+BDSPAC)
*
               CALL DBDSDC( 'U', 'I', N, S, WORK( IE ), WORK( IU ), N,
     $                      VT, LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                      INFO )
*
*              Overwrite WORK(IU) by left singular vectors of R and VT
*              by right singular vectors of R
*              (Workspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*
               CALL DORMBR( 'Q', 'L', 'N', N, N, N, A, LDA,
     $                      WORK( ITAUQ ), WORK( IU ), LDWRKU,
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
               CALL DORMBR( 'P', 'R', 'T', N, N, N, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Multiply Q in U by left singular vectors of R in
*              WORK(IU), storing result in A
*              (Workspace: need N*N)
*
               CALL DGEMM( 'N', 'N', M, N, N, ONE, U, LDU, WORK( IU ),
     $                     LDWRKU, ZERO, A, LDA )
*
*              Copy left singular vectors of A from A to U
*
               CALL DLACPY( 'F', M, N, A, LDA, U, LDU )
*
            END IF
*
         ELSE
*
*           M .LT. MNTHR
*
*           Path 5 (M at least N, but not much larger)
*           Reduce to bidiagonal form without QR decomposition
*
            IE = 1
            ITAUQ = IE + N
            ITAUP = ITAUQ + N
            NWORK = ITAUP + N
*
*           Bidiagonalize A
*           (Workspace: need 3*N+M, prefer 3*N+(M+N)*NB)
*
            CALL DGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                   WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
```

```
     $                  IERR )
            IF( WNTQN ) THEN
*
*              Perform bidiagonal SVD, only computing singular values
*              (Workspace: need N+BDSPAC)
*
               CALL DBDSDC( 'U', 'N', N, S, WORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, WORK( NWORK ), IWORK, INFO )
            ELSE IF( WNTQO ) THEN
               IU = NWORK
               IF( LWORK.GE.M*N+3*N+BDSPAC ) THEN
*
*                 WORK( IU ) is M by N
*
                  LDWRKU = M
                  NWORK = IU + LDWRKU*N
                  CALL DLASET( 'F', M, N, ZERO, ZERO, WORK( IU ),
     $                         LDWRKU )
               ELSE
*
*                 WORK( IU ) is N by N
*
                  LDWRKU = N
                  NWORK = IU + LDWRKU*N
*
*                 WORK(IR) is LDWRKR by N
*
                  IR = NWORK
                  LDWRKR = ( LWORK-N*N-3*N ) / N
               END IF
               NWORK = IU + LDWRKU*N
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in WORK(IU) and computing right
*              singular vectors of bidiagonal matrix in VT
*              (Workspace: need N+N*N+BDSPAC)
*
               CALL DBDSDC( 'U', 'I', N, S, WORK( IE ), WORK( IU ),
     $                      LDWRKU, VT, LDVT, DUM, IDUM, WORK( NWORK ),
     $                      IWORK, INFO )
*
*              Overwrite VT by right singular vectors of A
*              (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
               CALL DORMBR( 'P', 'R', 'T', N, N, N, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
               IF( LWORK.GE.M*N+3*N+BDSPAC ) THEN
*
```

```
*                 Overwrite WORK(IU) by left singular vectors of A
*                 (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                  CALL DORMBR( 'Q', 'L', 'N', M, N, N, A, LDA,
     $                         WORK( ITAUQ ), WORK( IU ), LDWRKU,
     $                         WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                 Copy left singular vectors of A from WORK(IU) to A
*
                  CALL DLACPY( 'F', M, N, WORK( IU ), LDWRKU, A, LDA )
               ELSE
*
*                 Generate Q in A
*                 (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                  CALL DORGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
     $                         WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                 Multiply Q in A by left singular vectors of
*                 bidiagonal matrix in WORK(IU), storing result in
*                 WORK(IR) and copying to A
*                 (Workspace: need 2*N*N, prefer N*N+M*N)
*
                  DO 20 I = 1, M, LDWRKR
                     CHUNK = MIN( M-I+1, LDWRKR )
                     CALL DGEMM( 'N', 'N', CHUNK, N, N, ONE, A( I, 1 ),
     $                           LDA, WORK( IU ), LDWRKU, ZERO,
     $                           WORK( IR ), LDWRKR )
                     CALL DLACPY( 'F', CHUNK, N, WORK( IR ), LDWRKR,
     $                            A( I, 1 ), LDA )
   20             CONTINUE
               END IF
*
            ELSE IF( WNTQS ) THEN
*
               Perform bidiagonal SVD, computing left singular vectors
               of bidiagonal matrix in U and computing right singular
               vectors of bidiagonal matrix in VT
               (Workspace: need N+BDSPAC)
*
               CALL DLASET( 'F', M, N, ZERO, ZERO, U, LDU )
               CALL DBDSDC( 'U', 'I', N, S, WORK( IE ), U, LDU, VT,
     $                      LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                      INFO )
*
               Overwrite U by left singular vectors of A and VT
               by right singular vectors of A
               (Workspace: need 3*N, prefer 2*N+N*NB)
*
               CALL DORMBR( 'Q', 'L', 'N', M, N, N, A, LDA,
```

```
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
               CALL DORMBR( 'P', 'R', 'T', N, N, N, A, LDA,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
            ELSE IF( WNTQA ) THEN
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in U and computing right singular
*              vectors of bidiagonal matrix in VT
*              (Workspace: need N+BDSPAC)
*
               CALL DLASET( 'F', M, M, ZERO, ZERO, U, LDU )
               CALL DBDSDC( 'U', 'I', N, S, WORK( IE ), U, LDU, VT,
     $                     LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                     INFO )
*
*              Set the right corner of U to identity matrix
*
               IF( M.GT.N ) THEN
                  CALL DLASET( 'F', M-N, M-N, ZERO, ONE, U( N+1, N+1 ),
     $                        LDU )
               END IF
*
*              Overwrite U by left singular vectors of A and VT
*              by right singular vectors of A
*              (Workspace: need N*N+2*N+M, prefer N*N+2*N+M*NB)
*
               CALL DORMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
               CALL DORMBR( 'P', 'R', 'T', N, N, M, A, LDA,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
            END IF
*
         END IF
*
      ELSE
*
         A has more columns than rows. If A has sufficiently more
         columns than rows, first reduce using the LQ decomposition (if
         sufficient workspace available)
*
         IF( N.GE.MNTHR ) THEN
*
            IF( WNTQN ) THEN
*
               Path 1t (N much larger than M, JOBZ='N')
               No singular vectors to be computed
```

```
*
               ITAU = 1
               NWORK = ITAU + M
*
*              Compute A=L*Q
*              (Workspace: need 2*M, prefer M+M*NB)
*
               CALL DGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Zero out above L
*
               CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ), LDA )
               IE = 1
               ITAUQ = IE + M
               ITAUP = ITAUQ + M
               NWORK = ITAUP + M
*
*              Bidiagonalize L in A
*              (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
               CALL DGEBRD( M, M, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                      IERR )
               NWORK = IE + M
*
*              Perform bidiagonal SVD, computing singular values only
*              (Workspace: need M+BDSPAC)
*
               CALL DBDSDC( 'U', 'N', M, S, WORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, WORK( NWORK ), IWORK, INFO )
*
            ELSE IF( WNTQO ) THEN
*
*              Path 2t (N much larger than M, JOBZ='O')
*              M right singular vectors to be overwritten on A and
*              M left singular vectors to be computed in U
*
               IVT = 1
*
*              IVT is M by M
*
               IL = IVT + M*M
               IF( LWORK.GE.M*N+M*M+3*M+BDSPAC ) THEN
*
*                 WORK(IL) is M by N
*
                  LDWRKL = M
                  CHUNK = N
               ELSE
```

```
               LDWRKL = M
               CHUNK = ( LWORK-M*M ) / M
            END IF
            ITAU = IL + LDWRKL*M
            NWORK = ITAU + M
*
*           Compute A=L*Q
*           (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
            CALL DGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                   LWORK-NWORK+1, IERR )
*
*           Copy L to WORK(IL), zeroing about above it
*
            CALL DLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWRKL )
            CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                   WORK( IL+LDWRKL ), LDWRKL )
*
*           Generate Q in A
*           (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
            CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                   WORK( NWORK ), LWORK-NWORK+1, IERR )
            IE = ITAU
            ITAUQ = IE + M
            ITAUP = ITAUQ + M
            NWORK = ITAUP + M
*
*           Bidiagonalize L in WORK(IL)
*           (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
            CALL DGEBRD( M, M, WORK( IL ), LDWRKL, S, WORK( IE ),
     $                   WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                   LWORK-NWORK+1, IERR )
*
*           Perform bidiagonal SVD, computing left singular vectors
*           of bidiagonal matrix in U, and computing right singular
*           vectors of bidiagonal matrix in WORK(IVT)
*           (Workspace: need M+M*M+BDSPAC)
*
            CALL DBDSDC( 'U', 'I', M, S, WORK( IE ), U, LDU,
     $                   WORK( IVT ), M, DUM, IDUM, WORK( NWORK ),
     $                   IWORK, INFO )
*
*           Overwrite U by left singular vectors of L and WORK(IVT)
*           by right singular vectors of L
*           (Workspace: need 2*M*M+3*M, prefer 2*M*M+2*M+M*NB)
*
            CALL DORMBR( 'Q', 'L', 'N', M, M, M, WORK( IL ), LDWRKL,
     $                   WORK( ITAUQ ), U, LDU, WORK( NWORK ),
```

```
     $                     LWORK-NWORK+1, IERR )
               CALL DORMBR( 'P', 'R', 'T', M, M, M, WORK( IL ), LDWRKL,
     $                     WORK( ITAUP ), WORK( IVT ), M,
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Multiply right singular vectors of L in WORK(IVT) by Q
*              in A, storing result in WORK(IL) and copying to A
*              (Workspace: need 2*M*M, prefer M*M+M*N)
*
               DO 30 I = 1, N, CHUNK
                  BLK = MIN( N-I+1, CHUNK )
                  CALL DGEMM( 'N', 'N', M, BLK, M, ONE, WORK( IVT ), M,
     $                        A( 1, I ), LDA, ZERO, WORK( IL ), LDWRKL )
                  CALL DLACPY( 'F', M, BLK, WORK( IL ), LDWRKL,
     $                         A( 1, I ), LDA )
   30          CONTINUE
*
            ELSE IF( WNTQS ) THEN
*
               Path 3t (N much larger than M, JOBZ='S')
*              M right singular vectors to be computed in VT and
*              M left singular vectors to be computed in U
*
               IL = 1
*
*              WORK(IL) is M by M
*
               LDWRKL = M
               ITAU = IL + LDWRKL*M
               NWORK = ITAU + M
*
*              Compute A=L*Q
*              (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
               CALL DGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*              Copy L to WORK(IL), zeroing out above it
*
               CALL DLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWRKL )
               CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                     WORK( IL+LDWRKL ), LDWRKL )
*
*              Generate Q in A
*              (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
               CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
               IE = ITAU
               ITAUQ = IE + M
```

```
               ITAUP = ITAUQ + M
               NWORK = ITAUP + M
*
*              Bidiagonalize L in WORK(IU), copying result to U
*              (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
               CALL DGEBRD( M, M, WORK( IL ), LDWRKL, S, WORK( IE ),
     $                      WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in U and computing right singular
*              vectors of bidiagonal matrix in VT
*              (Workspace: need M+BDSPAC)
*
               CALL DBDSDC( 'U', 'I', M, S, WORK( IE ), U, LDU, VT,
     $                      LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                      INFO )
*
*              Overwrite U by left singular vectors of L and VT
*              by right singular vectors of L
*              (Workspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*
               CALL DORMBR( 'Q', 'L', 'N', M, M, M, WORK( IL ), LDWRKL,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
               CALL DORMBR( 'P', 'R', 'T', M, M, M, WORK( IL ), LDWRKL,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Multiply right singular vectors of L in WORK(IL) by
*              Q in A, storing result in VT
*              (Workspace: need M*M)
*
               CALL DLACPY( 'F', M, M, VT, LDVT, WORK( IL ), LDWRKL )
               CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IL ), LDWRKL,
     $                     A, LDA, ZERO, VT, LDVT )
*
            ELSE IF( WNTQA ) THEN
*
*              Path 4t (N much larger than M, JOBZ='A')
*              N right singular vectors to be computed in VT and
*              M left singular vectors to be computed in U
*
               IVT = 1
*
*              WORK(IVT) is M by M
*
               LDWKVT = M
               ITAU = IVT + LDWKVT*M
```

```
               NWORK = ITAU + M
*
*              Compute A=L*Q, copying result to VT
*              (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
               CALL DGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
               CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*              Generate Q in VT
*              (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
               CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Produce L in A, zeroing out other entries
*
               CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ), LDA )
               IE = ITAU
               ITAUQ = IE + M
               ITAUP = ITAUQ + M
               NWORK = ITAUP + M
*
*              Bidiagonalize L in A
*              (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
               CALL DGEBRD( M, M, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                     WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                     IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in U and computing right singular
*              vectors of bidiagonal matrix in WORK(IVT)
*              (Workspace: need M+M*M+BDSPAC)
*
               CALL DBDSDC( 'U', 'I', M, S, WORK( IE ), U, LDU,
     $                     WORK( IVT ), LDWKVT, DUM, IDUM,
     $                     WORK( NWORK ), IWORK, INFO )
*
*              Overwrite U by left singular vectors of L and WORK(IVT)
*              by right singular vectors of L
*              (Workspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*
               CALL DORMBR( 'Q', 'L', 'N', M, M, M, A, LDA,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
               CALL DORMBR( 'P', 'R', 'T', M, M, M, A, LDA,
     $                     WORK( ITAUP ), WORK( IVT ), LDWKVT,
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
*
```

```
*             Multiply right singular vectors of L in WORK(IVT) by
*             Q in VT, storing result in A
*             (Workspace: need M*M)
*
              CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IVT ), LDWKVT,
     $                    VT, LDVT, ZERO, A, LDA )
*
*             Copy right singular vectors of A from A to VT
*
              CALL DLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
           END IF
*
        ELSE
*
*          N .LT. MNTHR
*
*          Path 5t (N greater than M, but not much larger)
*          Reduce to bidiagonal form without LQ decomposition
*
           IE = 1
           ITAUQ = IE + M
           ITAUP = ITAUQ + M
           NWORK = ITAUP + M
*
*          Bidiagonalize A
*          (Workspace: need 3*M+N, prefer 3*M+(M+N)*NB)
*
           CALL DGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                  WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                  IERR )
           IF( WNTQN ) THEN
*
*             Perform bidiagonal SVD, only computing singular values
*             (Workspace: need M+BDSPAC)
*
              CALL DBDSDC( 'L', 'N', M, S, WORK( IE ), DUM, 1, DUM, 1,
     $                     DUM, IDUM, WORK( NWORK ), IWORK, INFO )
           ELSE IF( WNTQO ) THEN
              LDWKVT = M
              IVT = NWORK
              IF( LWORK.GE.M*N+3*M+BDSPAC ) THEN
*
*                WORK( IVT ) is M by N
*
                 CALL DLASET( 'F', M, N, ZERO, ZERO, WORK( IVT ),
     $                        LDWKVT )
                 NWORK = IVT + LDWKVT*N
              ELSE
*
```

```
*                 WORK( IVT ) is M by M
*
                  NWORK = IVT + LDWKVT*M
                  IL = NWORK
*
*                 WORK(IL) is M by CHUNK
*
                  CHUNK = ( LWORK-M*M-3*M ) / M
               END IF
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in U and computing right singular
*              vectors of bidiagonal matrix in WORK(IVT)
*              (Workspace: need M*M+BDSPAC)
*
               CALL DBDSDC( 'L', 'I', M, S, WORK( IE ), U, LDU,
     $                      WORK( IVT ), LDWKVT, DUM, IDUM,
     $                      WORK( NWORK ), IWORK, INFO )
*
*              Overwrite U by left singular vectors of A
*              (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
               CALL DORMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
               IF( LWORK.GE.M*N+3*M+BDSPAC ) THEN
*
*                 Overwrite WORK(IVT) by left singular vectors of A
*                 (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                  CALL DORMBR( 'P', 'R', 'T', M, N, M, A, LDA,
     $                         WORK( ITAUP ), WORK( IVT ), LDWKVT,
     $                         WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                 Copy right singular vectors of A from WORK(IVT) to A
*
                  CALL DLACPY( 'F', M, N, WORK( IVT ), LDWKVT, A, LDA )
               ELSE
*
*                 Generate P**T in A
*                 (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                  CALL DORGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                         WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                 Multiply Q in A by right singular vectors of
*                 bidiagonal matrix in WORK(IVT), storing result in
*                 WORK(IL) and copying to A
*                 (Workspace: need 2*M*M, prefer M*M+M*N)
```

```
*
                 DO 40 I = 1, N, CHUNK
                    BLK = MIN( N-I+1, CHUNK )
                    CALL DGEMM( 'N', 'N', M, BLK, M, ONE, WORK( IVT ),
     $                          LDWKVT, A( 1, I ), LDA, ZERO,
     $                          WORK( IL ), M )
                    CALL DLACPY( 'F', M, BLK, WORK( IL ), M, A( 1, I ),
     $                           LDA )
   40            CONTINUE
              END IF
           ELSE IF( WNTQS ) THEN
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in U and computing right singular
*             vectors of bidiagonal matrix in VT
*             (Workspace: need M+BDSPAC)
*
              CALL DLASET( 'F', M, N, ZERO, ZERO, VT, LDVT )
              CALL DBDSDC( 'L', 'I', M, S, WORK( IE ), U, LDU, VT,
     $                     LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                     INFO )
*
*             Overwrite U by left singular vectors of A and VT
*             by right singular vectors of A
*             (Workspace: need 3*M, prefer 2*M+M*NB)
*
              CALL DORMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
              CALL DORMBR( 'P', 'R', 'T', M, N, M, A, LDA,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
           ELSE IF( WNTQA ) THEN
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in U and computing right singular
*             vectors of bidiagonal matrix in VT
*             (Workspace: need M+BDSPAC)
*
              CALL DLASET( 'F', N, N, ZERO, ZERO, VT, LDVT )
              CALL DBDSDC( 'L', 'I', M, S, WORK( IE ), U, LDU, VT,
     $                     LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                     INFO )
*
*             Set the right corner of VT to identity matrix
*
              IF( N.GT.M ) THEN
                 CALL DLASET( 'F', N-M, N-M, ZERO, ONE, VT( M+1, M+1 ),
     $                        LDVT )
              END IF
```

```
*
*             Overwrite U by left singular vectors of A and VT
*             by right singular vectors of A
*             (Workspace: need 2*M+N, prefer 2*M+N*NB)
*
              CALL DORMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
              CALL DORMBR( 'P', 'R', 'T', N, N, M, A, LDA,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
           END IF
*
        END IF
*
      END IF
*
*     Undo scaling if necessary
*
      IF( ISCL.EQ.1 ) THEN
         IF( ANRM.GT.BIGNUM )
     $      CALL DLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( ANRM.LT.SMLNUM )
     $      CALL DLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
      END IF
*
*     Return optimal workspace in WORK(1)
*
      WORK( 1 ) = MAXWRK
*
      RETURN
*
*     End of DGESDD
*
      END
      SUBROUTINE DGESDD( JOBZ, M, N, A, LDA, S, U, LDU, VT, LDVT, WORK,
     $                   LWORK, IWORK, INFO )
*
*  -- LAPACK driver routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1999
*
*     .. Scalar Arguments ..
      CHARACTER          JOBZ
      INTEGER            INFO, LDA, LDU, LDVT, LWORK, M, N
*     ..
*     .. Array Arguments ..
```

```
      INTEGER            IWORK( * )
      DOUBLE PRECISION   A( LDA, * ), S( * ), U( LDU, * ),
     $                   VT( LDVT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, WNTQA, WNTQAS, WNTQN, WNTQO, WNTQS
      INTEGER            BDSPAC, BLK, CHUNK, I, IE, IERR, IL,
     $                   IR, ISCL, ITAU, ITAUP, ITAUQ, IU, IVT, LDWKVT,
     $                   LDWRKL, LDWRKR, LDWRKU, MAXWRK, MINMN, MINWRK,
     $                   MNTHR, NWORK, WRKBL
      DOUBLE PRECISION   ANRM, BIGNUM, EPS, SMLNUM
*     ..
*     .. Local Arrays ..
      INTEGER            IDUM( 1 )
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DBDSDC, DGEBRD, DGELQF, DGEMM, DGEQRF, DLACPY,
     $                   DLASCL, DLASET, DORGBR, DORGLQ, DORGQR, DORMBR,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           DLAMCH, DLANGE, ILAENV, LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, INT, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      MINMN = MIN( M, N )
      MNTHR = INT( MINMN*11.0D0 / 6.0D0 )
      WNTQA = LSAME( JOBZ, 'A' )
      WNTQS = LSAME( JOBZ, 'S' )
      WNTQAS = WNTQA .OR. WNTQS
      WNTQO = LSAME( JOBZ, 'O' )
      WNTQN = LSAME( JOBZ, 'N' )
      MINWRK = 1
```

```
      MAXWRK = 1
      LQUERY = ( LWORK.EQ.-1 )
*
      IF( .NOT.( WNTQA .OR. WNTQS .OR. WNTQO .OR. WNTQN ) ) THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LDU.LT.1 .OR. ( WNTQAS .AND. LDU.LT.M ) .OR.
     $         ( WNTQO .AND. M.LT.N .AND. LDU.LT.M ) ) THEN
         INFO = -8
      ELSE IF( LDVT.LT.1 .OR. ( WNTQA .AND. LDVT.LT.N ) .OR.
     $         ( WNTQS .AND. LDVT.LT.MINMN ) .OR.
     $         ( WNTQO .AND. M.GE.N .AND. LDVT.LT.N ) ) THEN
         INFO = -10
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 .AND. M.GT.0 .AND. N.GT.0 ) THEN
         IF( M.GE.N ) THEN
*
*           Compute space needed for DBDSDC
*
            IF( WNTQN ) THEN
               BDSPAC = 7*N
            ELSE
               BDSPAC = 3*N*N + 4*N
            END IF
            IF( M.GE.MNTHR ) THEN
               IF( WNTQN ) THEN
*
*                 Path 1 (M much larger than N, JOBZ='N')
*
                  WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1,
     $                    -1 )
                  WRKBL = MAX( WRKBL, 3*N+2*N*
     $                    ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                  MAXWRK = MAX( WRKBL, BDSPAC+N )
                  MINWRK = BDSPAC + N
               ELSE IF( WNTQO ) THEN
*
```

```
*                Path 2 (M much larger than N, JOBZ='O')
*
                 WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, N+N*ILAENV( 1, 'DORGQR', ' ', M,
     $                   N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+2*N*
     $                   ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'QLN', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'PRT', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*N )
                 MAXWRK = WRKBL + 2*N*N
                 MINWRK = BDSPAC + 2*N*N + 3*N
              ELSE IF( WNTQS ) THEN
*
*                Path 3 (M much larger than N, JOBZ='S')
*
                 WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, N+N*ILAENV( 1, 'DORGQR', ' ', M,
     $                   N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+2*N*
     $                   ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'QLN', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'PRT', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*N )
                 MAXWRK = WRKBL + N*N
                 MINWRK = BDSPAC + N*N + 3*N
              ELSE IF( WNTQA ) THEN
*
*                Path 4 (M much larger than N, JOBZ='A')
*
                 WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, N+M*ILAENV( 1, 'DORGQR', ' ', M,
     $                   M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+2*N*
     $                   ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'QLN', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORMBR', 'PRT', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*N )
                 MAXWRK = WRKBL + N*N
                 MINWRK = BDSPAC + N*N + 3*N
              END IF
           ELSE
*
*                Path 5 (M at least N, but not much larger)
```

```
*
               WRKBL = 3*N + ( M+N )*ILAENV( 1, 'DGEBRD', ' ', M, N, -1,
     $                 -1 )
               IF( WNTQN ) THEN
                  MAXWRK = MAX( WRKBL, BDSPAC+3*N )
                  MINWRK = 3*N + MAX( M, BDSPAC )
               ELSE IF( WNTQO ) THEN
                  WRKBL = MAX( WRKBL, 3*N+N*
     $                    ILAENV( 1, 'DORMBR', 'QLN', M, N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 3*N+N*
     $                    ILAENV( 1, 'DORMBR', 'PRT', N, N, N, -1 ) )
                  WRKBL = MAX( WRKBL, BDSPAC+3*N )
                  MAXWRK = WRKBL + M*N
                  MINWRK = 3*N + MAX( M, N*N+BDSPAC )
               ELSE IF( WNTQS ) THEN
                  WRKBL = MAX( WRKBL, 3*N+N*
     $                    ILAENV( 1, 'DORMBR', 'QLN', M, N, N, -1 ) )
                  WRKBL = MAX( WRKBL, 3*N+N*
     $                    ILAENV( 1, 'DORMBR', 'PRT', N, N, N, -1 ) )
                  MAXWRK = MAX( WRKBL, BDSPAC+3*N )
                  MINWRK = 3*N + MAX( M, BDSPAC )
               ELSE IF( WNTQA ) THEN
                  WRKBL = MAX( WRKBL, 3*N+M*
     $                    ILAENV( 1, 'DORMBR', 'QLN', M, M, N, -1 ) )
                  WRKBL = MAX( WRKBL, 3*N+N*
     $                    ILAENV( 1, 'DORMBR', 'PRT', N, N, N, -1 ) )
                  MAXWRK = MAX( MAXWRK, BDSPAC+3*N )
                  MINWRK = 3*N + MAX( M, BDSPAC )
               END IF
            END IF
         ELSE
*
*           Compute space needed for DBDSDC
*
            IF( WNTQN ) THEN
               BDSPAC = 7*M
            ELSE
               BDSPAC = 3*M*M + 4*M
            END IF
            IF( N.GE.MNTHR ) THEN
               IF( WNTQN ) THEN
*
*                 Path 1t (N much larger than M, JOBZ='N')
*
                  WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1,
     $                    -1 )
                  WRKBL = MAX( WRKBL, 3*M+2*M*
     $                    ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
                  MAXWRK = MAX( WRKBL, BDSPAC+M )
                  MINWRK = BDSPAC + M
```

```
            ELSE IF( WNTQO ) THEN
*
*              Path 2t (N much larger than M, JOBZ='O')
*
               WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
               WRKBL = MAX( WRKBL, M+M*ILAENV( 1, 'DORGLQ', ' ', M,
     $                 N, M, -1 ) )
               WRKBL = MAX( WRKBL, 3*M+2*M*
     $                 ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
               WRKBL = MAX( WRKBL, 3*M+M*
     $                 ILAENV( 1, 'DORMBR', 'QLN', M, M, M, -1 ) )
               WRKBL = MAX( WRKBL, 3*M+M*
     $                 ILAENV( 1, 'DORMBR', 'PRT', M, M, M, -1 ) )
               WRKBL = MAX( WRKBL, BDSPAC+3*M )
               MAXWRK = WRKBL + 2*M*M
               MINWRK = BDSPAC + 2*M*M + 3*M
            ELSE IF( WNTQS ) THEN
*
*              Path 3t (N much larger than M, JOBZ='S')
*
               WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
               WRKBL = MAX( WRKBL, M+M*ILAENV( 1, 'DORGLQ', ' ', M,
     $                 N, M, -1 ) )
               WRKBL = MAX( WRKBL, 3*M+2*M*
     $                 ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
               WRKBL = MAX( WRKBL, 3*M+M*
     $                 ILAENV( 1, 'DORMBR', 'QLN', M, M, M, -1 ) )
               WRKBL = MAX( WRKBL, 3*M+M*
     $                 ILAENV( 1, 'DORMBR', 'PRT', M, M, M, -1 ) )
               WRKBL = MAX( WRKBL, BDSPAC+3*M )
               MAXWRK = WRKBL + M*M
               MINWRK = BDSPAC + M*M + 3*M
            ELSE IF( WNTQA ) THEN
*
*              Path 4t (N much larger than M, JOBZ='A')
*
               WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
               WRKBL = MAX( WRKBL, M+N*ILAENV( 1, 'DORGLQ', ' ', N,
     $                 N, M, -1 ) )
               WRKBL = MAX( WRKBL, 3*M+2*M*
     $                 ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
               WRKBL = MAX( WRKBL, 3*M+M*
     $                 ILAENV( 1, 'DORMBR', 'QLN', M, M, M, -1 ) )
               WRKBL = MAX( WRKBL, 3*M+M*
     $                 ILAENV( 1, 'DORMBR', 'PRT', M, M, M, -1 ) )
               WRKBL = MAX( WRKBL, BDSPAC+3*M )
               MAXWRK = WRKBL + M*M
               MINWRK = BDSPAC + M*M + 3*M
            END IF
         ELSE
```

```
*
*             Path 5t (N greater than M, but not much larger)
*
              WRKBL = 3*M + ( M+N )*ILAENV( 1, 'DGEBRD', ' ', M, N, -1,
     $                -1 )
              IF( WNTQN ) THEN
                 MAXWRK = MAX( WRKBL, BDSPAC+3*M )
                 MINWRK = 3*M + MAX( N, BDSPAC )
              ELSE IF( WNTQO ) THEN
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'PRT', M, N, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC+3*M )
                 MAXWRK = WRKBL + M*N
                 MINWRK = 3*M + MAX( N, M*M+BDSPAC )
              ELSE IF( WNTQS ) THEN
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'PRT', M, N, M, -1 ) )
                 MAXWRK = MAX( WRKBL, BDSPAC+3*M )
                 MINWRK = 3*M + MAX( N, BDSPAC )
              ELSE IF( WNTQA ) THEN
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'QLN', M, M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORMBR', 'PRT', N, N, M, -1 ) )
                 MAXWRK = MAX( WRKBL, BDSPAC+3*M )
                 MINWRK = 3*M + MAX( N, BDSPAC )
              END IF
           END IF
        END IF
        WORK( 1 ) = MAXWRK
     END IF
*
     IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
        INFO = -12
     END IF
     IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'DGESDD', -INFO )
        RETURN
     ELSE IF( LQUERY ) THEN
        RETURN
     END IF
*
*    Quick return if possible
*
     IF( M.EQ.0 .OR. N.EQ.0 ) THEN
        IF( LWORK.GE.1 )
```

```
     $      WORK( 1 ) = ONE
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = SQRT( DLAMCH( 'S' ) ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = DLANGE( 'M', M, N, A, LDA, DUM )
      ISCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ISCL = 1
         CALL DLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, IERR )
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ISCL = 1
         CALL DLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, IERR )
      END IF
*
      IF( M.GE.N ) THEN
*
*        A has at least as many rows as columns. If A has sufficiently
*        more rows than columns, first reduce using the QR
*        decomposition (if sufficient workspace available)
*
         IF( M.GE.MNTHR ) THEN
*
            IF( WNTQN ) THEN
*
*              Path 1 (M much larger than N, JOBZ='N')
*              No singular vectors to be computed
*
               ITAU = 1
               NWORK = ITAU + N
*
*              Compute A=Q*R
*              (Workspace: need 2*N, prefer N+N*NB)
*
               CALL DGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Zero out below R
*
               CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ), LDA )
               IE = 1
               ITAUQ = IE + N
               ITAUP = ITAUQ + N
```

```
               NWORK = ITAUP + N
*
*              Bidiagonalize R in A
*              (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
               CALL DGEBRD( N, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                      IERR )
               NWORK = IE + N
*
*              Perform bidiagonal SVD, computing singular values only
*              (Workspace: need N+BDSPAC)
*
               CALL DBDSDC( 'U', 'N', N, S, WORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, WORK( NWORK ), IWORK, INFO )
*
            ELSE IF( WNTQO ) THEN
*
*              Path 2 (M much larger than N, JOBZ = 'O')
*              N left singular vectors to be overwritten on A and
*              N right singular vectors to be computed in VT
*
               IR = 1
*
*              WORK(IR) is LDWRKR by N
*
               IF( LWORK.GE.LDA*N+N*N+3*N+BDSPAC ) THEN
                  LDWRKR = LDA
               ELSE
                  LDWRKR = ( LWORK-N*N-3*N-BDSPAC ) / N
               END IF
               ITAU = IR + LDWRKR*N
               NWORK = ITAU + N
*
*              Compute A=Q*R
*              (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
               CALL DGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy R to WORK(IR), zeroing out below it
*
               CALL DLACPY( 'U', N, N, A, LDA, WORK( IR ), LDWRKR )
               CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, WORK( IR+1 ),
     $                      LDWRKR )
*
*              Generate Q in A
*              (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
               CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
```

```
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
              IE = ITAU
              ITAUQ = IE + N
              ITAUP = ITAUQ + N
              NWORK = ITAUP + N
*
*             Bidiagonalize R in VT, copying result to WORK(IR)
*             (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
              CALL DGEBRD( N, N, WORK( IR ), LDWRKR, S, WORK( IE ),
     $                     WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*             WORK(IU) is N by N
*
              IU = NWORK
              NWORK = IU + N*N
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in WORK(IU) and computing right
*             singular vectors of bidiagonal matrix in VT
*             (Workspace: need N+N*N+BDSPAC)
*
              CALL DBDSDC( 'U', 'I', N, S, WORK( IE ), WORK( IU ), N,
     $                     VT, LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                     INFO )
*
*             Overwrite WORK(IU) by left singular vectors of R
*             and VT by right singular vectors of R
*             (Workspace: need 2*N*N+3*N, prefer 2*N*N+2*N+N*NB)
*
              CALL DORMBR( 'Q', 'L', 'N', N, N, N, WORK( IR ), LDWRKR,
     $                     WORK( ITAUQ ), WORK( IU ), N, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
              CALL DORMBR( 'P', 'R', 'T', N, N, N, WORK( IR ), LDWRKR,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*             Multiply Q in A by left singular vectors of R in
*             WORK(IU), storing result in WORK(IR) and copying to A
*             (Workspace: need 2*N*N, prefer N*N+M*N)
*
              DO 10 I = 1, M, LDWRKR
                 CHUNK = MIN( M-I+1, LDWRKR )
                 CALL DGEMM( 'N', 'N', CHUNK, N, N, ONE, A( I, 1 ),
     $                       LDA, WORK( IU ), N, ZERO, WORK( IR ),
     $                       LDWRKR )
                 CALL DLACPY( 'F', CHUNK, N, WORK( IR ), LDWRKR,
     $                        A( I, 1 ), LDA )
   10         CONTINUE
```

```
*
            ELSE IF( WNTQS ) THEN
*
*              Path 3 (M much larger than N, JOBZ='S')
*              N left singular vectors to be computed in U and
*              N right singular vectors to be computed in VT
*
               IR = 1
*
*              WORK(IR) is N by N
*
               LDWRKR = N
               ITAU = IR + LDWRKR*N
               NWORK = ITAU + N
*
*              Compute A=Q*R
*              (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
               CALL DGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy R to WORK(IR), zeroing out below it
*
               CALL DLACPY( 'U', N, N, A, LDA, WORK( IR ), LDWRKR )
               CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, WORK( IR+1 ),
     $                      LDWRKR )
*
*              Generate Q in A
*              (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
               CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
               IE = ITAU
               ITAUQ = IE + N
               ITAUP = ITAUQ + N
               NWORK = ITAUP + N
*
*              Bidiagonalize R in WORK(IR)
*              (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
               CALL DGEBRD( N, N, WORK( IR ), LDWRKR, S, WORK( IE ),
     $                      WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagoal matrix in U and computing right singular
*              vectors of bidiagonal matrix in VT
*              (Workspace: need N+BDSPAC)
*
               CALL DBDSDC( 'U', 'I', N, S, WORK( IE ), U, LDU, VT,
```

```
     $                     LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                     INFO )
*
*             Overwrite U by left singular vectors of R and VT
*             by right singular vectors of R
*             (Workspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*
              CALL DORMBR( 'Q', 'L', 'N', N, N, N, WORK( IR ), LDWRKR,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
              CALL DORMBR( 'P', 'R', 'T', N, N, N, WORK( IR ), LDWRKR,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*             Multiply Q in A by left singular vectors of R in
*             WORK(IR), storing result in U
*             (Workspace: need N*N)
*
              CALL DLACPY( 'F', N, N, U, LDU, WORK( IR ), LDWRKR )
              CALL DGEMM( 'N', 'N', M, N, N, ONE, A, LDA, WORK( IR ),
     $                    LDWRKR, ZERO, U, LDU )
*
           ELSE IF( WNTQA ) THEN
*
*             Path 4 (M much larger than N, JOBZ='A')
*             M left singular vectors to be computed in U and
*             N right singular vectors to be computed in VT
*
              IU = 1
*
*             WORK(IU) is N by N
*
              LDWRKU = N
              ITAU = IU + LDWRKU*N
              NWORK = ITAU + N
*
*             Compute A=Q*R, copying result to U
*             (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
              CALL DGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
              CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*             Generate Q in U
*             (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
              CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*             Produce R in A, zeroing out other entries
```

```
*
               CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ), LDA )
               IE = ITAU
               ITAUQ = IE + N
               ITAUP = ITAUQ + N
               NWORK = ITAUP + N
*
*              Bidiagonalize R in A
*              (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
               CALL DGEBRD( N, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                      IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in WORK(IU) and computing right
*              singular vectors of bidiagonal matrix in VT
*              (Workspace: need N+N*N+BDSPAC)
*
               CALL DBDSDC( 'U', 'I', N, S, WORK( IE ), WORK( IU ), N,
     $                      VT, LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                      INFO )
*
*              Overwrite WORK(IU) by left singular vectors of R and VT
*              by right singular vectors of R
*              (Workspace: need N*N+3*N, prefer N*N+2*N+N*NB)
*
               CALL DORMBR( 'Q', 'L', 'N', N, N, N, A, LDA,
     $                      WORK( ITAUQ ), WORK( IU ), LDWRKU,
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
               CALL DORMBR( 'P', 'R', 'T', N, N, N, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Multiply Q in U by left singular vectors of R in
*              WORK(IU), storing result in A
*              (Workspace: need N*N)
*
               CALL DGEMM( 'N', 'N', M, N, N, ONE, U, LDU, WORK( IU ),
     $                     LDWRKU, ZERO, A, LDA )
*
*              Copy left singular vectors of A from A to U
*
               CALL DLACPY( 'F', M, N, A, LDA, U, LDU )
*
            END IF
*
         ELSE
*
*           M .LT. MNTHR
```

```
*
*          Path 5 (M at least N, but not much larger)
*          Reduce to bidiagonal form without QR decomposition
*
           IE = 1
           ITAUQ = IE + N
           ITAUP = ITAUQ + N
           NWORK = ITAUP + N
*
*          Bidiagonalize A
*          (Workspace: need 3*N+M, prefer 3*N+(M+N)*NB)
*
           CALL DGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                  WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                  IERR )
           IF( WNTQN ) THEN
*
*             Perform bidiagonal SVD, only computing singular values
*             (Workspace: need N+BDSPAC)
*
              CALL DBDSDC( 'U', 'N', N, S, WORK( IE ), DUM, 1, DUM, 1,
     $                     DUM, IDUM, WORK( NWORK ), IWORK, INFO )
           ELSE IF( WNTQO ) THEN
              IU = NWORK
              IF( LWORK.GE.M*N+3*N+BDSPAC ) THEN
*
*                WORK( IU ) is M by N
*
                 LDWRKU = M
                 NWORK = IU + LDWRKU*N
                 CALL DLASET( 'F', M, N, ZERO, ZERO, WORK( IU ),
     $                        LDWRKU )
              ELSE
*
*                WORK( IU ) is N by N
*
                 LDWRKU = N
                 NWORK = IU + LDWRKU*N
*
*                WORK(IR) is LDWRKR by N
*
                 IR = NWORK
                 LDWRKR = ( LWORK-N*N-3*N ) / N
              END IF
              NWORK = IU + LDWRKU*N
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in WORK(IU) and computing right
*             singular vectors of bidiagonal matrix in VT
*             (Workspace: need N+N*N+BDSPAC)
```

```
*
               CALL DBDSDC( 'U', 'I', N, S, WORK( IE ), WORK( IU ),
     $                      LDWRKU, VT, LDVT, DUM, IDUM, WORK( NWORK ),
     $                      IWORK, INFO )
*
*              Overwrite VT by right singular vectors of A
*              (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
               CALL DORMBR( 'P', 'R', 'T', N, N, N, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
               IF( LWORK.GE.M*N+3*N+BDSPAC ) THEN
*
*                 Overwrite WORK(IU) by left singular vectors of A
*                 (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                  CALL DORMBR( 'Q', 'L', 'N', M, N, N, A, LDA,
     $                         WORK( ITAUQ ), WORK( IU ), LDWRKU,
     $                         WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                 Copy left singular vectors of A from WORK(IU) to A
*
                  CALL DLACPY( 'F', M, N, WORK( IU ), LDWRKU, A, LDA )
               ELSE
*
*                 Generate Q in A
*                 (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                  CALL DORGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
     $                         WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                 Multiply Q in A by left singular vectors of
*                 bidiagonal matrix in WORK(IU), storing result in
*                 WORK(IR) and copying to A
*                 (Workspace: need 2*N*N, prefer N*N+M*N)
*
                  DO 20 I = 1, M, LDWRKR
                     CHUNK = MIN( M-I+1, LDWRKR )
                     CALL DGEMM( 'N', 'N', CHUNK, N, N, ONE, A( I, 1 ),
     $                           LDA, WORK( IU ), LDWRKU, ZERO,
     $                           WORK( IR ), LDWRKR )
                     CALL DLACPY( 'F', CHUNK, N, WORK( IR ), LDWRKR,
     $                            A( I, 1 ), LDA )
   20             CONTINUE
               END IF
*
            ELSE IF( WNTQS ) THEN
*
*              Perform bidiagonal SVD, computing left singular vectors
```

```
*             of bidiagonal matrix in U and computing right singular
*             vectors of bidiagonal matrix in VT
*             (Workspace: need N+BDSPAC)
*
              CALL DLASET( 'F', M, N, ZERO, ZERO, U, LDU )
              CALL DBDSDC( 'U', 'I', N, S, WORK( IE ), U, LDU, VT,
     $                     LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                     INFO )
*
*             Overwrite U by left singular vectors of A and VT
*             by right singular vectors of A
*             (Workspace: need 3*N, prefer 2*N+N*NB)
*
              CALL DORMBR( 'Q', 'L', 'N', M, N, N, A, LDA,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
              CALL DORMBR( 'P', 'R', 'T', N, N, N, A, LDA,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
           ELSE IF( WNTQA ) THEN
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in U and computing right singular
*             vectors of bidiagonal matrix in VT
*             (Workspace: need N+BDSPAC)
*
              CALL DLASET( 'F', M, M, ZERO, ZERO, U, LDU )
              CALL DBDSDC( 'U', 'I', N, S, WORK( IE ), U, LDU, VT,
     $                     LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                     INFO )
*
*             Set the right corner of U to identity matrix
*
              CALL DLASET( 'F', M-N, M-N, ZERO, ONE, U( N+1, N+1 ),
     $                     LDU )
*
*             Overwrite U by left singular vectors of A and VT
*             by right singular vectors of A
*             (Workspace: need N*N+2*N+M, prefer N*N+2*N+M*NB)
*
              CALL DORMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
              CALL DORMBR( 'P', 'R', 'T', N, N, M, A, LDA,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
           END IF
*
        END IF
*
```

```
      ELSE
*
*        A has more columns than rows. If A has sufficiently more
*        columns than rows, first reduce using the LQ decomposition (if
*        sufficient workspace available)
*
         IF( N.GE.MNTHR ) THEN
*
            IF( WNTQN ) THEN
*
*              Path 1t (N much larger than M, JOBZ='N')
*              No singular vectors to be computed
*
               ITAU = 1
               NWORK = ITAU + M
*
*              Compute A=L*Q
*              (Workspace: need 2*M, prefer M+M*NB)
*
               CALL DGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Zero out above L
*
               CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ), LDA )
               IE = 1
               ITAUQ = IE + M
               ITAUP = ITAUQ + M
               NWORK = ITAUP + M
*
*              Bidiagonalize L in A
*              (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
               CALL DGEBRD( M, M, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                      IERR )
               NWORK = IE + M
*
*              Perform bidiagonal SVD, computing singular values only
*              (Workspace: need M+BDSPAC)
*
               CALL DBDSDC( 'U', 'N', M, S, WORK( IE ), DUM, 1, DUM, 1,
     $                      DUM, IDUM, WORK( NWORK ), IWORK, INFO )
*
            ELSE IF( WNTQO ) THEN
*
*              Path 2t (N much larger than M, JOBZ='O')
*              M right singular vectors to be overwritten on A and
*              M left singular vectors to be computed in U
*
```

```
               IVT = 1
*
*              IVT is M by M
*
               IL = IVT + M*M
               IF( LWORK.GE.M*N+M*M+3*M+BDSPAC ) THEN
*
*                 WORK(IL) is M by N
*
                  LDWRKL = M
                  CHUNK = N
               ELSE
                  LDWRKL = M
                  CHUNK = ( LWORK-M*M ) / M
               END IF
               ITAU = IL + LDWRKL*M
               NWORK = ITAU + M
*
*              Compute A=L*Q
*              (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
               CALL DGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy L to WORK(IL), zeroing about above it
*
               CALL DLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWRKL )
               CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                      WORK( IL+LDWRKL ), LDWRKL )
*
*              Generate Q in A
*              (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
               CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
               IE = ITAU
               ITAUQ = IE + M
               ITAUP = ITAUQ + M
               NWORK = ITAUP + M
*
*              Bidiagonalize L in WORK(IL)
*              (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
               CALL DGEBRD( M, M, WORK( IL ), LDWRKL, S, WORK( IE ),
     $                      WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in U, and computing right singular
*              vectors of bidiagonal matrix in WORK(IVT)
```

```
*              (Workspace: need M+M*M+BDSPAC)
*
               CALL DBDSDC( 'U', 'I', M, S, WORK( IE ), U, LDU,
     $                      WORK( IVT ), M, DUM, IDUM, WORK( NWORK ),
     $                      IWORK, INFO )
*
*              Overwrite U by left singular vectors of L and WORK(IVT)
*              by right singular vectors of L
*              (Workspace: need 2*M*M+3*M, prefer 2*M*M+2*M+M*NB)
*
               CALL DORMBR( 'Q', 'L', 'N', M, M, M, WORK( IL ), LDWRKL,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
               CALL DORMBR( 'P', 'R', 'T', M, M, M, WORK( IL ), LDWRKL,
     $                      WORK( ITAUP ), WORK( IVT ), M,
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Multiply right singular vectors of L in WORK(IVT) by Q
*              in A, storing result in WORK(IL) and copying to A
*              (Workspace: need 2*M*M, prefer M*M+M*N)
*
               DO 30 I = 1, N, CHUNK
                  BLK = MIN( N-I+1, CHUNK )
                  CALL DGEMM( 'N', 'N', M, BLK, M, ONE, WORK( IVT ), M,
     $                        A( 1, I ), LDA, ZERO, WORK( IL ), LDWRKL )
                  CALL DLACPY( 'F', M, BLK, WORK( IL ), LDWRKL,
     $                         A( 1, I ), LDA )
   30          CONTINUE
*
            ELSE IF( WNTQS ) THEN
*
               Path 3t (N much larger than M, JOBZ='S')
*              M right singular vectors to be computed in VT and
*              M left singular vectors to be computed in U
*
               IL = 1
*
*              WORK(IL) is M by M
*
               LDWRKL = M
               ITAU = IL + LDWRKL*M
               NWORK = ITAU + M
*
*              Compute A=L*Q
*              (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
               CALL DGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
*              Copy L to WORK(IL), zeroing out above it
```

```
*
              CALL DLACPY( 'L', M, M, A, LDA, WORK( IL ), LDWRKL )
              CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                     WORK( IL+LDWRKL ), LDWRKL )
*
*             Generate Q in A
*             (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
              CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
              IE = ITAU
              ITAUQ = IE + M
              ITAUP = ITAUQ + M
              NWORK = ITAUP + M
*
*             Bidiagonalize L in WORK(IU), copying result to U
*             (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
              CALL DGEBRD( M, M, WORK( IL ), LDWRKL, S, WORK( IE ),
     $                     WORK( ITAUQ ), WORK( ITAUP ), WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*             Perform bidiagonal SVD, computing left singular vectors
*             of bidiagonal matrix in U and computing right singular
*             vectors of bidiagonal matrix in VT
*             (Workspace: need M+BDSPAC)
*
              CALL DBDSDC( 'U', 'I', M, S, WORK( IE ), U, LDU, VT,
     $                     LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                     INFO )
*
*             Overwrite U by left singular vectors of L and VT
*             by right singular vectors of L
*             (Workspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*
              CALL DORMBR( 'Q', 'L', 'N', M, M, M, WORK( IL ), LDWRKL,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
              CALL DORMBR( 'P', 'R', 'T', M, M, M, WORK( IL ), LDWRKL,
     $                     WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
*
*             Multiply right singular vectors of L in WORK(IL) by
*             Q in A, storing result in VT
*             (Workspace: need M*M)
*
              CALL DLACPY( 'F', M, M, VT, LDVT, WORK( IL ), LDWRKL )
              CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IL ), LDWRKL,
     $                    A, LDA, ZERO, VT, LDVT )
*
```

```
            ELSE IF( WNTQA ) THEN
*
*              Path 4t (N much larger than M, JOBZ='A')
*              N right singular vectors to be computed in VT and
*              M left singular vectors to be computed in U
*
               IVT = 1
*
*              WORK(IVT) is M by M
*
               LDWKVT = M
               ITAU = IVT + LDWKVT*M
               NWORK = ITAU + M
*
*              Compute A=L*Q, copying result to VT
*              (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
               CALL DGELQF( M, N, A, LDA, WORK( ITAU ), WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
               CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*              Generate Q in VT
*              (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
               CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                      WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*              Produce L in A, zeroing out other entries
*
               CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ), LDA )
               IE = ITAU
               ITAUQ = IE + M
               ITAUP = ITAUQ + M
               NWORK = ITAUP + M
*
*              Bidiagonalize L in A
*              (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
               CALL DGEBRD( M, M, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                      IERR )
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in U and computing right singular
*              vectors of bidiagonal matrix in WORK(IVT)
*              (Workspace: need M+M*M+BDSPAC)
*
               CALL DBDSDC( 'U', 'I', M, S, WORK( IE ), U, LDU,
     $                      WORK( IVT ), LDWKVT, DUM, IDUM,
     $                      WORK( NWORK ), IWORK, INFO )
```

```
*
*             Overwrite U by left singular vectors of L and WORK(IVT)
*             by right singular vectors of L
*             (Workspace: need M*M+3*M, prefer M*M+2*M+M*NB)
*
              CALL DORMBR( 'Q', 'L', 'N', M, M, M, A, LDA,
     $                     WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                     LWORK-NWORK+1, IERR )
              CALL DORMBR( 'P', 'R', 'T', M, M, M, A, LDA,
     $                     WORK( ITAUP ), WORK( IVT ), LDWKVT,
     $                     WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*             Multiply right singular vectors of L in WORK(IVT) by
*             Q in VT, storing result in A
*             (Workspace: need M*M)
*
              CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IVT ), LDWKVT,
     $                    VT, LDVT, ZERO, A, LDA )
*
*             Copy right singular vectors of A from A to VT
*
              CALL DLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
           END IF
*
        ELSE
*
*          N .LT. MNTHR
*
*          Path 5t (N greater than M, but not much larger)
*          Reduce to bidiagonal form without LQ decomposition
*
           IE = 1
           ITAUQ = IE + M
           ITAUP = ITAUQ + M
           NWORK = ITAUP + M
*
*          Bidiagonalize A
*          (Workspace: need 3*M+N, prefer 3*M+(M+N)*NB)
*
           CALL DGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                  WORK( ITAUP ), WORK( NWORK ), LWORK-NWORK+1,
     $                  IERR )
           IF( WNTQN ) THEN
*
*             Perform bidiagonal SVD, only computing singular values
*             (Workspace: need M+BDSPAC)
*
              CALL DBDSDC( 'L', 'N', M, S, WORK( IE ), DUM, 1, DUM, 1,
     $                     DUM, IDUM, WORK( NWORK ), IWORK, INFO )
```

```
            ELSE IF( WNTQO ) THEN
               LDWKVT = M
               IVT = NWORK
               IF( LWORK.GE.M*N+3*M+BDSPAC ) THEN
*
*                 WORK( IVT ) is M by N
*
                  CALL DLASET( 'F', M, N, ZERO, ZERO, WORK( IVT ),
     $                         LDWKVT )
                  NWORK = IVT + LDWKVT*N
               ELSE
*
*                 WORK( IVT ) is M by M
*
                  NWORK = IVT + LDWKVT*M
                  IL = NWORK
*
*                 WORK(IL) is M by CHUNK
*
                  CHUNK = ( LWORK-M*M-3*M ) / M
               END IF
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in U and computing right singular
*              vectors of bidiagonal matrix in WORK(IVT)
*              (Workspace: need M*M+BDSPAC)
*
               CALL DBDSDC( 'L', 'I', M, S, WORK( IE ), U, LDU,
     $                      WORK( IVT ), LDWKVT, DUM, IDUM,
     $                      WORK( NWORK ), IWORK, INFO )
*
*              Overwrite U by left singular vectors of A
*              (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
               CALL DORMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
*
               IF( LWORK.GE.M*N+3*M+BDSPAC ) THEN
*
*                 Overwrite WORK(IVT) by left singular vectors of A
*                 (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                  CALL DORMBR( 'P', 'R', 'T', M, N, M, A, LDA,
     $                         WORK( ITAUP ), WORK( IVT ), LDWKVT,
     $                         WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                 Copy right singular vectors of A from WORK(IVT) to A
*
                  CALL DLACPY( 'F', M, N, WORK( IVT ), LDWKVT, A, LDA )
```

```
               ELSE
*
*                 Generate P**T in A
*                 (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                  CALL DORGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                         WORK( NWORK ), LWORK-NWORK+1, IERR )
*
*                 Multiply Q in A by right singular vectors of
*                 bidiagonal matrix in WORK(IVT), storing result in
*                 WORK(IL) and copying to A
*                 (Workspace: need 2*M*M, prefer M*M+M*N)
*
                  DO 40 I = 1, N, CHUNK
                     BLK = MIN( N-I+1, CHUNK )
                     CALL DGEMM( 'N', 'N', M, BLK, M, ONE, WORK( IVT ),
     $                           LDWKVT, A( 1, I ), LDA, ZERO,
     $                           WORK( IL ), M )
                     CALL DLACPY( 'F', M, BLK, WORK( IL ), M, A( 1, I ),
     $                            LDA )
   40             CONTINUE
               END IF
            ELSE IF( WNTQS ) THEN
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in U and computing right singular
*              vectors of bidiagonal matrix in VT
*              (Workspace: need M+BDSPAC)
*
               CALL DLASET( 'F', M, N, ZERO, ZERO, VT, LDVT )
               CALL DBDSDC( 'L', 'I', M, S, WORK( IE ), U, LDU, VT,
     $                      LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                      INFO )
*
*              Overwrite U by left singular vectors of A and VT
*              by right singular vectors of A
*              (Workspace: need 3*M, prefer 2*M+M*NB)
*
               CALL DORMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
               CALL DORMBR( 'P', 'R', 'T', M, N, M, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
            ELSE IF( WNTQA ) THEN
*
*              Perform bidiagonal SVD, computing left singular vectors
*              of bidiagonal matrix in U and computing right singular
*              vectors of bidiagonal matrix in VT
*              (Workspace: need M+BDSPAC)
```

```
*
               CALL DLASET( 'F', N, N, ZERO, ZERO, VT, LDVT )
               CALL DBDSDC( 'L', 'I', M, S, WORK( IE ), U, LDU, VT,
     $                      LDVT, DUM, IDUM, WORK( NWORK ), IWORK,
     $                      INFO )
*
*              Set the right corner of VT to identity matrix
*
               CALL DLASET( 'F', N-M, N-M, ZERO, ONE, VT( M+1, M+1 ),
     $                      LDVT )
*
*              Overwrite U by left singular vectors of A and VT
*              by right singular vectors of A
*              (Workspace: need 2*M+N, prefer 2*M+N*NB)
*
               CALL DORMBR( 'Q', 'L', 'N', M, M, N, A, LDA,
     $                      WORK( ITAUQ ), U, LDU, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
               CALL DORMBR( 'P', 'R', 'T', N, N, M, A, LDA,
     $                      WORK( ITAUP ), VT, LDVT, WORK( NWORK ),
     $                      LWORK-NWORK+1, IERR )
            END IF
*
         END IF
*
      END IF
*
*     Undo scaling if necessary
*
      IF( ISCL.EQ.1 ) THEN
         IF( ANRM.GT.BIGNUM )
     $      CALL DLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( ANRM.LT.SMLNUM )
     $      CALL DLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
      END IF
*
*     Return optimal workspace in WORK(1)
*
      WORK( 1 ) = DBLE( MAXWRK )
*
      RETURN
*
*     End of DGESDD
*
      END
```

———-

90.128.4 lisp code

— dgesdd.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dgesdd (jobz m n a lda s u ldu vt ldvt work lwork iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork)
             (type (array double-float (*)) work vt u s a)
             (type (f2cl-lib:integer4) info lwork ldvt ldu lda n m)
             (type (string *) jobz))
    (f2cl-lib:with-multi-array-data
        ((jobz character jobz-%data% jobz-%offset%)
         (a double-float a-%data% a-%offset%)
         (s double-float s-%data% s-%offset%)
         (u double-float u-%data% u-%offset%)
         (vt double-float vt-%data% vt-%offset%)
         (work double-float work-%data% work-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((dum (make-array 1 :element-type 'double-float))
             (idum (make-array 1 :element-type 'f2cl-lib:integer4))
             (anrm 0.0d0) (bignum 0.0d0) (eps 0.0d0) (smlnum 0.0d0) (bdspac 0)
             (blk 0) (chunk 0) (i 0) (ie 0) (ierr 0) (il 0) (ir 0) (iscl 0)
             (itau 0) (itaup 0) (itauq 0) (iu 0) (ivt 0) (ldwkvt 0) (ldwrkl 0)
             (ldwrkr 0) (ldwrku 0) (maxwrk 0) (minmn 0) (minwrk 0) (mnthr 0)
             (nwork 0) (wrkbl 0) (lquery nil) (wntqa nil) (wntqas nil)
             (wntqn nil) (wntqo nil) (wntqs nil))
        (declare (type (array double-float (1)) dum)
                 (type (array f2cl-lib:integer4 (1)) idum)
                 (type (double-float) anrm bignum eps smlnum)
                 (type (f2cl-lib:integer4) bdspac blk chunk i ie ierr il ir
                  iscl itau itaup itauq iu ivt ldwkvt ldwrkl ldwrkr ldwrku
                  maxwrk minmn minwrk mnthr nwork wrkbl)
                 (type f2cl-lib:logical lquery wntqa wntqas wntqn wntqo wntqs))
        (setf info 0)
        (setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (setf wntqa
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame jobz "A")
                  (declare (ignore var-1))
                  (when var-0 (setf jobz var-0))
                  ret-val))
        (setf wntqs
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame jobz "S")
                  (declare (ignore var-1))
                  (when var-0 (setf jobz var-0))
```

```
          ret-val))
(setf wntqas (or wntqa wntqs))
(setf wntqo
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobz "O")
          (declare (ignore var-1))
          (when var-0 (setf jobz var-0))
          ret-val))
(setf wntqn
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobz "N")
          (declare (ignore var-1))
          (when var-0 (setf jobz var-0))
          ret-val))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((not (or wntqa wntqs wntqo wntqn)) (setf info -1))
      ((< m 0) (setf info -2)) ((< n 0) (setf info -3))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -5))
      ((or (< ldu 1) (and wntqas (< ldu m))
           (and wntqo (< m n) (< ldu m)))
       (setf info -8))
      ((or (< ldvt 1) (and wntqa (< ldvt n)) (and wntqs (< ldvt minmn))
           (and wntqo (>= m n) (< ldvt n)))
       (setf info -10)))
(cond
 ((= info 0) (setf minwrk 1) (setf maxwrk 1)
  (cond
   ((and (>= m n) (> minmn 0))
    (setf mnthr (f2cl-lib:int (/ (* minmn 11.0d0) 6.0d0)))
    (cond (wntqn (setf bdspac (f2cl-lib:int-mul 7 n)))
          (t
           (setf bdspac
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n n)
                                     (f2cl-lib:int-mul 4 n)))))
    (cond
     ((>= m mnthr)
      (cond
       (wntqn
        (setf wrkbl
                (f2cl-lib:int-add n
                                  (f2cl-lib:int-mul n
                                                    (multiple-value-bind
                                                        (ret-val var-0
                                                         var-1 var-2
                                                         var-3 var-4
                                                         var-5 var-6)
                                                        (ilaenv 1
                                                         "DGEQRF" " " m
```

```
                                                    n -1 -1)
                                                 (declare
                                                  (ignore var-0
                                                   var-1 var-2
                                                   var-5 var-6))
                                                 (when var-3
                                                   (setf m var-3))
                                                 (when var-4
                                                   (setf n var-4))
                                                 ret-val))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul 2 n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "DGEBRD"
                                                           " "
                                                           n n
                                                           -1
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-5
                                                          var-6))
                                                        (when
                                                            var-3
                                                          (setf n
                                                                  var-3))
                                                        (when
                                                            var-4
                                                          (setf n
                                                                  var-4))
                                                        ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
```

```
                    (f2cl-lib:int-add bdspac n))))
 (setf minwrk (f2cl-lib:int-add bdspac n)))
(wntqo
 (setf wrkbl
         (f2cl-lib:int-add n
                           (f2cl-lib:int-mul n
                                             (multiple-value-bind
                                                 (ret-val var-0
                                                  var-1 var-2
                                                  var-3 var-4
                                                  var-5 var-6)
                                                 (ilaenv 1
                                                  "DGEQRF" " " m
                                                  n -1 -1)
                                               (declare
                                                (ignore var-0
                                                 var-1 var-2
                                                 var-5 var-6))
                                               (when var-3
                                                 (setf m var-3))
                                               (when var-4
                                                 (setf n var-4))
                                               ret-val))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n
                                     (f2cl-lib:int-mul n
                                                       (multiple-value-bind
                                                           (ret-val
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-3
                                                            var-4
                                                            var-5
                                                            var-6)
                                                           (ilaenv
                                                            1
                                                            "DORGQR"
                                                            " "
                                                            m n
                                                            n
                                                            -1)
                                                         (declare
                                                          (ignore
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-6))
```

```
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf n
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf n
                                                                            var-5))
                                                                  ret-val))))))
                                (setf wrkbl
                                        (max (the f2cl-lib:integer4 wrkbl)
                                             (the f2cl-lib:integer4
                                                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                                                    (f2cl-lib:int-mul 2 n
                                                                                      (multiple-value-bind
                                                                                          (ret-val
                                                                                           var-0
                                                                                           var-1
                                                                                           var-2
                                                                                           var-3
                                                                                           var-4
                                                                                           var-5
                                                                                           var-6)
                                                                                          (ilaenv
                                                                                           1
                                                                                           "DGEBRD"
                                                                                           " "
                                                                                           n n
                                                                                           -1
                                                                                           -1)
                                                                                      (declare
                                                                                       (ignore
                                                                                        var-0
                                                                                        var-1
                                                                                        var-2
                                                                                        var-5
                                                                                        var-6))
                                                                                      (when
                                                                                          var-3
                                                                                        (setf n
                                                                                                var-3))
                                                                                      (when
                                                                                          var-4
                                                                                        (setf n
                                                                                                var-4))
                                                                                      ret-val))))))
```

```
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "DORMBR"
                                                           "QLN"
                                                           n n
                                                           n
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (when
                                                            var-3
                                                          (setf n
                                                                  var-3))
                                                        (when
                                                            var-4
                                                          (setf n
                                                                  var-4))
                                                        (when
                                                            var-5
                                                          (setf n
                                                                  var-5))
                                                        ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
```

```
                                                                var-3
                                                                var-4
                                                                var-5
                                                                var-6)
                                                               (ilaenv
                                                                1
                                                                "DORMBR"
                                                                "PRT"
                                                                n n
                                                                n
                                                                -1)
                                                             (declare
                                                              (ignore
                                                               var-0
                                                               var-1
                                                               var-2
                                                               var-6))
                                                             (when
                                                                 var-3
                                                               (setf n
                                                                       var-3))
                                                             (when
                                                                 var-4
                                                               (setf n
                                                                       var-4))
                                                             (when
                                                                 var-5
                                                               (setf n
                                                                       var-5))
                                                             ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add bdspac
                                                    (f2cl-lib:int-mul 3 n)))))
                (setf maxwrk (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 n n)))
                (setf minwrk
                        (f2cl-lib:int-add bdspac (f2cl-lib:int-mul 2 n n)
                                          (f2cl-lib:int-mul 3 n))))
               (wntqs
                (setf wrkbl
                        (f2cl-lib:int-add n
                                          (f2cl-lib:int-mul n
                                                            (multiple-value-bind
                                                                (ret-val var-0
                                                                 var-1 var-2
                                                                 var-3 var-4
                                                                 var-5 var-6)
                                                                (ilaenv 1
                                                                 "DGEQRF" " " m
```

```
                                                        n -1 -1)
                                                     (declare
                                                      (ignore var-0
                                                       var-1 var-2
                                                       var-5 var-6))
                                                     (when var-3
                                                       (setf m var-3))
                                                     (when var-4
                                                       (setf n var-4))
                                                     ret-val))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add n
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "DORGQR"
                                                           " "
                                                           m n
                                                           n
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (when
                                                            var-3
                                                          (setf m
                                                                  var-3))
                                                        (when
                                                            var-4
                                                          (setf n
                                                                  var-4))
                                                        (when
                                                            var-5
                                                          (setf n
                                                                  var-5))
                                                        ret-val))))))
```

```
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul 2 n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "DGEBRD"
                                                           " "
                                                           n n
                                                           -1
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-5
                                                          var-6))
                                                        (when
                                                            var-3
                                                          (setf n
                                                                  var-3))
                                                        (when
                                                            var-4
                                                          (setf n
                                                                  var-4))
                                                        ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
```

```
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "DORMBR"
                                                                    "QLN"
                                                                    n n
                                                                    n
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf n
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf n
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf n
                                                                            var-5))
                                                                  ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "DORMBR"
                                                           "PRT"
                                                           n n
                                                           n
                                                           -1)
                                                          (declare
                                                           (ignore
```

```
                                                                var-0
                                                                var-1
                                                                var-2
                                                                var-6))
                                                              (when
                                                                  var-3
                                                                (setf n
                                                                        var-3))
                                                              (when
                                                                  var-4
                                                                (setf n
                                                                        var-4))
                                                              (when
                                                                  var-5
                                                                (setf n
                                                                        var-5))
                                                              ret-val))))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add bdspac
                                     (f2cl-lib:int-mul 3 n)))))
 (setf maxwrk (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul n n)))
 (setf minwrk
         (f2cl-lib:int-add bdspac (f2cl-lib:int-mul n n)
                           (f2cl-lib:int-mul 3 n))))
(wntqa
 (setf wrkbl
         (f2cl-lib:int-add n
                           (f2cl-lib:int-mul n
                                             (multiple-value-bind
                                                 (ret-val var-0
                                                  var-1 var-2
                                                  var-3 var-4
                                                  var-5 var-6)
                                                 (ilaenv 1
                                                  "DGEQRF" " " m
                                                  n -1 -1)
                                               (declare
                                                (ignore var-0
                                                 var-1 var-2
                                                 var-5 var-6))
                                               (when var-3
                                                 (setf m var-3))
                                               (when var-4
                                                 (setf n var-4))
                                               ret-val))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
```

```
                                (f2cl-lib:int-add n
                                                  (f2cl-lib:int-mul m
                                                                    (multiple-value-bind
                                                                        (ret-val
                                                                         var-0
                                                                         var-1
                                                                         var-2
                                                                         var-3
                                                                         var-4
                                                                         var-5
                                                                         var-6)
                                                                        (ilaenv
                                                                         1
                                                                         "DORGQR"
                                                                         " "
                                                                         m m
                                                                         n
                                                                         -1)
                                                                      (declare
                                                                       (ignore
                                                                        var-0
                                                                        var-1
                                                                        var-2
                                                                        var-6))
                                                                      (when
                                                                          var-3
                                                                        (setf m
                                                                                var-3))
                                                                      (when
                                                                          var-4
                                                                        (setf m
                                                                                var-4))
                                                                      (when
                                                                          var-5
                                                                        (setf n
                                                                                var-5))
                                                                      ret-val))))))
                        (setf wrkbl
                              (max (the f2cl-lib:integer4 wrkbl)
                                   (the f2cl-lib:integer4
                                        (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                                          (f2cl-lib:int-mul 2 n
                                                                            (multiple-value-bind
                                                                                (ret-val
                                                                                 var-0
                                                                                 var-1
                                                                                 var-2
                                                                                 var-3
                                                                                 var-4
                                                                                 var-5
```

```
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "DGEBRD"
                                                                    " "
                                                                    n n
                                                                    -1
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-5
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                   (setf n
                                                                           var-3))
                                                                  (when
                                                                      var-4
                                                                   (setf n
                                                                           var-4))
                                                                  ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "DORMBR"
                                                           "QLN"
                                                           n n
                                                           n
                                                           -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
```

```
                                                                  var-6))
                                                                (when
                                                                    var-3
                                                                  (setf n
                                                                          var-3))
                                                                (when
                                                                    var-4
                                                                  (setf n
                                                                          var-4))
                                                                (when
                                                                    var-5
                                                                  (setf n
                                                                          var-5))
                                                                ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "DORMBR"
                                                           "PRT"
                                                           n n
                                                           n
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (when
                                                            var-3
                                                          (setf n
                                                                  var-3))
                                                        (when
                                                            var-4
                                                          (setf n
                                                                  var-4))
                                                        (when
```

```
                                                                    var-5
                                                                  (setf n
                                                                          var-5))
                                                                ret-val))))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add bdspac
                                        (f2cl-lib:int-mul 3 n)))))
    (setf maxwrk (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul n n)))
    (setf minwrk
            (f2cl-lib:int-add bdspac (f2cl-lib:int-mul n n)
                              (f2cl-lib:int-mul 2 n) m)))))
(t
 (setf wrkbl
         (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                           (f2cl-lib:int-mul
                            (f2cl-lib:int-add m n)
                            (multiple-value-bind
                                (ret-val var-0 var-1 var-2 var-3
                                 var-4 var-5 var-6)
                                (ilaenv 1 "DGEBRD" " " m n -1 -1)
                              (declare
                               (ignore var-0 var-1 var-2 var-5
                                var-6))
                              (when var-3 (setf m var-3))
                              (when var-4 (setf n var-4))
                              ret-val))))
 (cond
  (wntqn
   (setf maxwrk
           (max (the f2cl-lib:integer4 wrkbl)
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add bdspac
                                       (f2cl-lib:int-mul 3 n)))))
   (setf minwrk
           (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                             (max (the f2cl-lib:integer4 m)
                                  (the f2cl-lib:integer4
                                       bdspac)))))
  (wntqo
   (setf wrkbl
           (max (the f2cl-lib:integer4 wrkbl)
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                       (f2cl-lib:int-mul n
                                                         (multiple-value-bind
                                                             (ret-val
                                                              var-0
                                                              var-1
```

```
                                                                    var-2
                                                                    var-3
                                                                    var-4
                                                                    var-5
                                                                    var-6)
                                                                   (ilaenv
                                                                    1
                                                                    "DORMBR"
                                                                    "QLN"
                                                                    m n
                                                                    n
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf n
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf n
                                                                            var-5))
                                                                  ret-val))))))
                    (setf wrkbl
                            (max (the f2cl-lib:integer4 wrkbl)
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                                        (f2cl-lib:int-mul n
                                                                          (multiple-value-bind
                                                                              (ret-val
                                                                               var-0
                                                                               var-1
                                                                               var-2
                                                                               var-3
                                                                               var-4
                                                                               var-5
                                                                               var-6)
                                                                              (ilaenv
                                                                               1
                                                                               "DORMBR"
                                                                               "PRT"
                                                                               n n
```

```
                                                                    n
                                                                    -1)
                                                                 (declare
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-6))
                                                                 (when
                                                                     var-3
                                                                   (setf n
                                                                           var-3))
                                                                 (when
                                                                     var-4
                                                                   (setf n
                                                                           var-4))
                                                                 (when
                                                                     var-5
                                                                   (setf n
                                                                           var-5))
                                                                 ret-val))))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add bdspac
                                     (f2cl-lib:int-mul 3 n)))))
 (setf maxwrk (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul m n)))
 (setf minwrk
         (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                           (max (the f2cl-lib:integer4 m)
                                (the f2cl-lib:integer4
                                     (f2cl-lib:int-add
                                      (f2cl-lib:int-mul n n)
                                      bdspac))))))
(wntqs
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     (f2cl-lib:int-mul n
                                                       (multiple-value-bind
                                                           (ret-val
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-3
                                                            var-4
                                                            var-5
                                                            var-6)
                                                           (ilaenv
```

```
                                                                    1
                                                                    "DORMBR"
                                                                    "QLN"
                                                                    m n
                                                                    n
                                                                    -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf n
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf n
                                                                            var-5))
                                                                  ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                    (f2cl-lib:int-mul n
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "DORMBR"
                                                           "PRT"
                                                           n n
                                                           n
                                                           -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
```

```
                                                                var-2
                                                                var-6))
                                                              (when
                                                                  var-3
                                                                (setf n
                                                                        var-3))
                                                              (when
                                                                  var-4
                                                                (setf n
                                                                        var-4))
                                                              (when
                                                                  var-5
                                                                (setf n
                                                                        var-5))
                                                              ret-val))))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add bdspac
                                     (f2cl-lib:int-mul 3 n)))))
 (setf minwrk
         (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                           (max (the f2cl-lib:integer4 m)
                                (the f2cl-lib:integer4
                                     bdspac)))))
(wntqa
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     (f2cl-lib:int-mul m
                                                       (multiple-value-bind
                                                           (ret-val
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-3
                                                            var-4
                                                            var-5
                                                            var-6)
                                                           (ilaenv
                                                            1
                                                            "DORMBR"
                                                            "QLN"
                                                            m m
                                                            n
                                                            -1)
                                                          (declare
                                                           (ignore
                                                            var-0
```

```
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf m
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf n
                                                                            var-5))
                                                                  ret-val))))))
                  (setf wrkbl
                          (max (the f2cl-lib:integer4 wrkbl)
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                                      (f2cl-lib:int-mul n
                                                                        (multiple-value-bind
                                                                            (ret-val
                                                                             var-0
                                                                             var-1
                                                                             var-2
                                                                             var-3
                                                                             var-4
                                                                             var-5
                                                                             var-6)
                                                                            (ilaenv
                                                                             1
                                                                             "DORMBR"
                                                                             "PRT"
                                                                             n n
                                                                             n
                                                                             -1)
                                                                          (declare
                                                                           (ignore
                                                                            var-0
                                                                            var-1
                                                                            var-2
                                                                            var-6))
                                                                          (when
                                                                              var-3
                                                                            (setf n
                                                                                    var-3))
                                                                          (when
                                                                              var-4
                                                                            (setf n
```

```
                                                                                var-4))
                                                                          (when
                                                                              var-5
                                                                            (setf n
                                                                                var-5))
                                                                          ret-val))))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add bdspac
                                            (f2cl-lib:int-mul 3 n)))))
        (setf minwrk
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                  (max (the f2cl-lib:integer4 m)
                                       (the f2cl-lib:integer4
                                            bdspac)))))))))
    ((> minmn 0) (setf mnthr (f2cl-lib:int (/ (* minmn 11.0d0) 6.0d0)))
     (cond (wntqn (setf bdspac (f2cl-lib:int-mul 7 m)))
           (t
            (setf bdspac
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 m m)
                                      (f2cl-lib:int-mul 4 m)))))
     (cond
      ((>= n mnthr)
       (cond
        (wntqn
         (setf wrkbl
                 (f2cl-lib:int-add m
                                   (f2cl-lib:int-mul m
                                                     (multiple-value-bind
                                                         (ret-val var-0
                                                          var-1 var-2
                                                          var-3 var-4
                                                          var-5 var-6)
                                                         (ilaenv 1
                                                          "DGELQF" " " m
                                                          n -1 -1)
                                                       (declare
                                                        (ignore var-0
                                                         var-1 var-2
                                                         var-5 var-6))
                                                       (when var-3
                                                         (setf m var-3))
                                                       (when var-4
                                                         (setf n var-4))
                                                       ret-val))))
        (setf wrkbl
                (max (the f2cl-lib:integer4 wrkbl)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
```

```
                                                    (f2cl-lib:int-mul 2 m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "DGEBRD"
                                                                           " "
                                                                           m m
                                                                           -1
                                                                           -1)
                                                                        (declare
                                                                         (ignore
                                                                          var-0
                                                                          var-1
                                                                          var-2
                                                                          var-5
                                                                          var-6))
                                                                        (when
                                                                            var-3
                                                                          (setf m
                                                                                  var-3))
                                                                        (when
                                                                            var-4
                                                                          (setf m
                                                                                  var-4))
                                                                        ret-val))))))
                 (setf maxwrk
                         (max (the f2cl-lib:integer4 wrkbl)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add bdspac m))))
                 (setf minwrk (f2cl-lib:int-add bdspac m)))
                (wntqo
                 (setf wrkbl
                         (f2cl-lib:int-add m
                                           (f2cl-lib:int-mul m
                                                             (multiple-value-bind
                                                                 (ret-val var-0
                                                                  var-1 var-2
                                                                  var-3 var-4
                                                                  var-5 var-6)
                                                                 (ilaenv 1
                                                                  "DGELQF" " " m
                                                                  n -1 -1)
```

```
                                                    (declare
                                                     (ignore var-0
                                                      var-1 var-2
                                                      var-5 var-6))
                                                    (when var-3
                                                      (setf m var-3))
                                                    (when var-4
                                                      (setf n var-4))
                                                    ret-val))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add m
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "DORGLQ"
                                                           " "
                                                           m n
                                                           m
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (when
                                                            var-3
                                                          (setf m
                                                                var-3))
                                                        (when
                                                            var-4
                                                          (setf n
                                                                var-4))
                                                        (when
                                                            var-5
                                                          (setf m
                                                                var-5))
                                                        ret-val))))))
(setf wrkbl
```

```
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    (f2cl-lib:int-mul 2 m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "DGEBRD"
                                                           " "
                                                           m m
                                                           -1
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-5
                                                          var-6))
                                                        (when
                                                            var-3
                                                          (setf m
                                                                  var-3))
                                                        (when
                                                            var-4
                                                          (setf m
                                                                  var-4))
                                                        ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
```

```
                                                                  (ilaenv
                                                                   1
                                                                   "DORMBR"
                                                                   "QLN"
                                                                   m m
                                                                   m
                                                                   -1)
                                                                 (declare
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-6))
                                                                 (when
                                                                     var-3
                                                                   (setf m
                                                                           var-3))
                                                                 (when
                                                                     var-4
                                                                   (setf m
                                                                           var-4))
                                                                 (when
                                                                     var-5
                                                                   (setf m
                                                                           var-5))
                                                                 ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "DORMBR"
                                                           "PRT"
                                                           m m
                                                           m
                                                           -1)
                                                         (declare
                                                          (ignore
                                                           var-0
```

```
                                                                var-1
                                                                var-2
                                                                var-6))
                                                              (when
                                                                  var-3
                                                                (setf m
                                                                        var-3))
                                                              (when
                                                                  var-4
                                                                (setf m
                                                                        var-4))
                                                              (when
                                                                  var-5
                                                                (setf m
                                                                        var-5))
                                                              ret-val))))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add bdspac
                                     (f2cl-lib:int-mul 3 m)))))
 (setf maxwrk (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 m m)))
 (setf minwrk
         (f2cl-lib:int-add bdspac (f2cl-lib:int-mul 2 m m)
                           (f2cl-lib:int-mul 3 m))))
(wntqs
 (setf wrkbl
         (f2cl-lib:int-add m
                           (f2cl-lib:int-mul m
                                             (multiple-value-bind
                                                 (ret-val var-0
                                                  var-1 var-2
                                                  var-3 var-4
                                                  var-5 var-6)
                                                 (ilaenv 1
                                                  "DGELQF" " " m
                                                  n -1 -1)
                                               (declare
                                                (ignore var-0
                                                 var-1 var-2
                                                 var-5 var-6))
                                               (when var-3
                                                 (setf m var-3))
                                               (when var-4
                                                 (setf n var-4))
                                               ret-val))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add m
```

```
                                                    (f2cl-lib:int-mul m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "DORGLQ"
                                                                           " "
                                                                           m n
                                                                           m
                                                                           -1)
                                                                        (declare
                                                                         (ignore
                                                                          var-0
                                                                          var-1
                                                                          var-2
                                                                          var-6))
                                                                        (when
                                                                            var-3
                                                                          (setf m
                                                                                  var-3))
                                                                        (when
                                                                            var-4
                                                                          (setf n
                                                                                  var-4))
                                                                        (when
                                                                            var-5
                                                                          (setf m
                                                                                  var-5))
                                                                        ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                                    (f2cl-lib:int-mul 2 m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
```

```
                                                                    (ilaenv
                                                                     1
                                                                     "DGEBRD"
                                                                     " "
                                                                     m m
                                                                     -1
                                                                     -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-5
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf m
                                                                            var-4))
                                                                  ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                                    (f2cl-lib:int-mul m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "DORMBR"
                                                                           "QLN"
                                                                           m m
                                                                           m
                                                                           -1)
                                                                          (declare
                                                                           (ignore
                                                                            var-0
                                                                            var-1
                                                                            var-2
                                                                            var-6))
```

```
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf m
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf m
                                                                            var-5))
                                                                  ret-val))))))
                    (setf wrkbl
                            (max (the f2cl-lib:integer4 wrkbl)
                                 (the f2cl-lib:integer4
                                      (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                                        (f2cl-lib:int-mul m
                                                                          (multiple-value-bind
                                                                              (ret-val
                                                                               var-0
                                                                               var-1
                                                                               var-2
                                                                               var-3
                                                                               var-4
                                                                               var-5
                                                                               var-6)
                                                                              (ilaenv
                                                                               1
                                                                               "DORMBR"
                                                                               "PRT"
                                                                               m m
                                                                               m
                                                                               -1)
                                                                  (declare
                                                                   (ignore
                                                                    var-0
                                                                    var-1
                                                                    var-2
                                                                    var-6))
                                                                  (when
                                                                      var-3
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf m
                                                                            var-4))
                                                                  (when
                                                                      var-5
```

```
                                                                  (setf m
                                                                          var-5))
                                                                ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add bdspac
                                                    (f2cl-lib:int-mul 3 m)))))
                (setf maxwrk (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul m m)))
                (setf minwrk
                        (f2cl-lib:int-add bdspac (f2cl-lib:int-mul m m)
                                          (f2cl-lib:int-mul 3 m))))
               (wntqa
                (setf wrkbl
                        (f2cl-lib:int-add m
                                          (f2cl-lib:int-mul m
                                                            (multiple-value-bind
                                                                (ret-val var-0
                                                                 var-1 var-2
                                                                 var-3 var-4
                                                                 var-5 var-6)
                                                                (ilaenv 1
                                                                 "DGELQF" " " m
                                                                 n -1 -1)
                                                              (declare
                                                               (ignore var-0
                                                                var-1 var-2
                                                                var-5 var-6))
                                                              (when var-3
                                                                (setf m var-3))
                                                              (when var-4
                                                                (setf n var-4))
                                                              ret-val))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add m
                                                    (f2cl-lib:int-mul n
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "DORGLQ"
```

```
                                                                    " "
                                                                    n n
                                                                    m
                                                                    -1)
                                                                 (declare
                                                                  (ignore
                                                                   var-0
                                                                   var-1
                                                                   var-2
                                                                   var-6))
                                                                 (when
                                                                     var-3
                                                                   (setf n
                                                                           var-3))
                                                                 (when
                                                                     var-4
                                                                   (setf n
                                                                           var-4))
                                                                 (when
                                                                     var-5
                                                                   (setf m
                                                                           var-5))
                                                                 ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    (f2cl-lib:int-mul 2 m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "DGEBRD"
                                                           " "
                                                           m m
                                                           -1
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-5
```

```
                                                                  var-6))
                                                                (when
                                                                    var-3
                                                                  (setf m
                                                                          var-3))
                                                                (when
                                                                    var-4
                                                                  (setf m
                                                                          var-4))
                                                                ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "DORMBR"
                                                           "QLN"
                                                           m m
                                                           m
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (when
                                                            var-3
                                                          (setf m
                                                                  var-3))
                                                        (when
                                                            var-4
                                                          (setf m
                                                                  var-4))
                                                        (when
                                                            var-5
                                                          (setf m
                                                                  var-5))
                                                        ret-val))))))
```

```
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                        (f2cl-lib:int-mul m
                                                          (multiple-value-bind
                                                              (ret-val
                                                               var-0
                                                               var-1
                                                               var-2
                                                               var-3
                                                               var-4
                                                               var-5
                                                               var-6)
                                                              (ilaenv
                                                               1
                                                               "DORMBR"
                                                               "PRT"
                                                               m m
                                                               m
                                                               -1)
                                                            (declare
                                                             (ignore
                                                              var-0
                                                              var-1
                                                              var-2
                                                              var-6))
                                                            (when
                                                                var-3
                                                              (setf m
                                                                      var-3))
                                                            (when
                                                                var-4
                                                              (setf m
                                                                      var-4))
                                                            (when
                                                                var-5
                                                              (setf m
                                                                      var-5))
                                                            ret-val))))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add bdspac
                                        (f2cl-lib:int-mul 3 m)))))
    (setf maxwrk (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul m m)))
    (setf minwrk
            (f2cl-lib:int-add bdspac (f2cl-lib:int-mul m m)
                              (f2cl-lib:int-mul 3 m))))))
(t
```

```
(setf wrkbl
        (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                          (f2cl-lib:int-mul
                           (f2cl-lib:int-add m n)
                           (multiple-value-bind
                               (ret-val var-0 var-1 var-2 var-3
                                var-4 var-5 var-6)
                               (ilaenv 1 "DGEBRD" " " m n -1 -1)
                             (declare
                              (ignore var-0 var-1 var-2 var-5
                               var-6))
                             (when var-3 (setf m var-3))
                             (when var-4 (setf n var-4))
                             ret-val))))
(cond
 (wntqn
  (setf maxwrk
          (max (the f2cl-lib:integer4 wrkbl)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add bdspac
                                      (f2cl-lib:int-mul 3 m)))))
  (setf minwrk
          (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                            (max (the f2cl-lib:integer4 n)
                                 (the f2cl-lib:integer4
                                      bdspac)))))
 (wntqo
  (setf wrkbl
          (max (the f2cl-lib:integer4 wrkbl)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                      (f2cl-lib:int-mul m
                                                        (multiple-value-bind
                                                            (ret-val
                                                             var-0
                                                             var-1
                                                             var-2
                                                             var-3
                                                             var-4
                                                             var-5
                                                             var-6)
                                                            (ilaenv
                                                             1
                                                             "DORMBR"
                                                             "QLN"
                                                             m m
                                                             n
                                                             -1)
                                                          (declare
                                                           (ignore
```

```
                                                                var-0
                                                                var-1
                                                                var-2
                                                                var-6))
                                                              (when
                                                                  var-3
                                                                (setf m
                                                                        var-3))
                                                              (when
                                                                  var-4
                                                                (setf m
                                                                        var-4))
                                                              (when
                                                                  var-5
                                                                (setf n
                                                                        var-5))
                                                              ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    (f2cl-lib:int-mul m
                                                          (multiple-value-bind
                                                              (ret-val
                                                               var-0
                                                               var-1
                                                               var-2
                                                               var-3
                                                               var-4
                                                               var-5
                                                               var-6)
                                                              (ilaenv
                                                               1
                                                               "DORMBR"
                                                               "PRT"
                                                               m n
                                                               m
                                                               -1)
                                                              (declare
                                                               (ignore
                                                                var-0
                                                                var-1
                                                                var-2
                                                                var-6))
                                                              (when
                                                                  var-3
                                                                (setf m
                                                                        var-3))
                                                              (when
                                                                  var-4
```

```
                                                                  (setf n
                                                                          var-4))
                                                                (when
                                                                    var-5
                                                                  (setf m
                                                                          var-5))
                                                                ret-val))))))
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add bdspac
                                                    (f2cl-lib:int-mul 3 m)))))
                (setf maxwrk (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul m n)))
                (setf minwrk
                        (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                          (max (the f2cl-lib:integer4 n)
                                               (the f2cl-lib:integer4
                                                    (f2cl-lib:int-add
                                                     (f2cl-lib:int-mul m m)
                                                     bdspac))))))
               (wntqs
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                                    (f2cl-lib:int-mul m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "DORMBR"
                                                                           "QLN"
                                                                           m m
                                                                           n
                                                                           -1)
                                                                        (declare
                                                                         (ignore
                                                                          var-0
                                                                          var-1
                                                                          var-2
                                                                          var-6))
                                                                        (when
                                                                            var-3
```

```
                                                                    (setf m
                                                                            var-3))
                                                                  (when
                                                                      var-4
                                                                    (setf m
                                                                            var-4))
                                                                  (when
                                                                      var-5
                                                                    (setf n
                                                                            var-5))
                                                                  ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "DORMBR"
                                                           "PRT"
                                                           m n
                                                           m
                                                           -1)
                                                          (declare
                                                           (ignore
                                                            var-0
                                                            var-1
                                                            var-2
                                                            var-6))
                                                          (when
                                                              var-3
                                                            (setf m
                                                                    var-3))
                                                          (when
                                                              var-4
                                                            (setf n
                                                                    var-4))
                                                          (when
                                                              var-5
                                                            (setf m
                                                                    var-5))
```

```
                                                                ret-val))))))
                (setf maxwrk
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add bdspac
                                                    (f2cl-lib:int-mul 3 m)))))
                (setf minwrk
                        (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                          (max (the f2cl-lib:integer4 n)
                                               (the f2cl-lib:integer4
                                                    bdspac)))))
               (wntqa
                (setf wrkbl
                        (max (the f2cl-lib:integer4 wrkbl)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                                    (f2cl-lib:int-mul m
                                                                      (multiple-value-bind
                                                                          (ret-val
                                                                           var-0
                                                                           var-1
                                                                           var-2
                                                                           var-3
                                                                           var-4
                                                                           var-5
                                                                           var-6)
                                                                          (ilaenv
                                                                           1
                                                                           "DORMBR"
                                                                           "QLN"
                                                                           m m
                                                                           n
                                                                           -1)
                                                                        (declare
                                                                         (ignore
                                                                          var-0
                                                                          var-1
                                                                          var-2
                                                                          var-6))
                                                                        (when
                                                                            var-3
                                                                          (setf m
                                                                                  var-3))
                                                                        (when
                                                                            var-4
                                                                          (setf m
                                                                                  var-4))
                                                                        (when
                                                                            var-5
                                                                          (setf n
```

```
                                                                    var-5))
                                                              ret-val))))))
(setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    (f2cl-lib:int-mul m
                                                      (multiple-value-bind
                                                          (ret-val
                                                           var-0
                                                           var-1
                                                           var-2
                                                           var-3
                                                           var-4
                                                           var-5
                                                           var-6)
                                                          (ilaenv
                                                           1
                                                           "DORMBR"
                                                           "PRT"
                                                           n n
                                                           m
                                                           -1)
                                                        (declare
                                                         (ignore
                                                          var-0
                                                          var-1
                                                          var-2
                                                          var-6))
                                                        (when
                                                            var-3
                                                          (setf n
                                                                  var-3))
                                                        (when
                                                            var-4
                                                          (setf n
                                                                  var-4))
                                                        (when
                                                            var-5
                                                          (setf m
                                                                  var-5))
                                                        ret-val))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add bdspac
                                    (f2cl-lib:int-mul 3 m)))))
(setf minwrk
        (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                          (max (the f2cl-lib:integer4 n)
```

```
                                        (the f2cl-lib:integer4
                                             bdspac)))))))))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4 minwrk)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 maxwrk) 'double-float))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -12)))))
(cond
 ((/= info 0) (xerbla "DGESDD" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond ((or (= m 0) (= n 0)) (go end_label)))
(setf eps (dlamch "P"))
(setf smlnum (/ (f2cl-lib:fsqrt (dlamch "S")) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "M" m n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iscl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf iscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 anrm smlnum m n a lda ierr)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf ierr var-9))))
 ((> anrm bignum) (setf iscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 anrm bignum m n a lda ierr)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf ierr var-9)))))
(cond
 ((>= m n)
```

```
(cond
 ((>= m mnthr)
  (cond
   (wntqn (setf itau 1) (setf nwork (f2cl-lib:int-add itau n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (dgeqrf m n a lda
         (f2cl-lib:array-slice work-%data% double-float (itau)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (nwork)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
      (declare (ignore var-2 var-4 var-5 var-6))
      (setf m var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf ierr var-7))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
         zero zero
         (f2cl-lib:array-slice a-%data% double-float (2 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf lda var-6)))
    (setf ie 1) (setf itauq (f2cl-lib:int-add ie n))
    (setf itaup (f2cl-lib:int-add itauq n))
    (setf nwork (f2cl-lib:int-add itaup n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (dgebrd n n a lda s
         (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                               work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (itauq)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (itaup)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (nwork)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
      (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
      (setf n var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf ierr var-10))
    (setf nwork (f2cl-lib:int-add ie n))
    (multiple-value-bind
```

```
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dbdsdc "U" "N" n s
       (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                             work-%offset%)
       dum 1 dum 1 dum idum
       (f2cl-lib:array-slice work-%data% double-float (nwork)
                             ((1 *)) work-%offset%)
       iwork info)
    (declare
     (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12))
    (setf n var-2)
    (setf info var-13)))
  (wntqo (setf ir 1)
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul lda n)
                           (f2cl-lib:int-mul n n)
                           (f2cl-lib:int-mul 3 n) bdspac))
    (setf ldwrkr lda))
    (t
    (setf ldwrkr
            (the f2cl-lib:integer4
                 (truncate (- lwork (* n n) (* 3 n) bdspac) n)))))
   (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
   (setf nwork (f2cl-lib:int-add itau n))
   (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% double-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "U" n n a lda
       (f2cl-lib:array-slice work-%data% double-float (ir) ((1 *))
                             work-%offset%)
       ldwrkr)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldwrkr var-6)))
```

```
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice work-%data% double-float ((+ ir 1))
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorgqr m n n a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf nwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgebrd n n
     (f2cl-lib:array-slice work-%data% double-float (ir) ((1 *))
                           work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(setf iu nwork)
(setf nwork (f2cl-lib:int-add iu (f2cl-lib:int-mul n n)))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dbdsdc "U" "I" n s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iu) ((1 *))
                           work-%offset%)
     n vt ldvt dum idum
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (setf n var-2)
  (setf n var-6)
  (setf ldvt var-8)
  (setf info var-13))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "Q" "L" "N" n n n
     (f2cl-lib:array-slice work-%data% double-float (ir) ((1 *))
                           work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iu) ((1 *))
                           work-%offset%)
     n
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldwrkr var-7))
  (when var-10 (setf n var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "P" "R" "T" n n n
     (f2cl-lib:array-slice work-%data% double-float (ir) ((1 *))
                           work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% double-float (itaup)
```

```
                              ((1 *)) work-%offset%)
      vt ldvt
      (f2cl-lib:array-slice work-%data% double-float (nwork)
                              ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldwrkr var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i ldwrkr))
              ((> i m) nil)
  (tagbody
    (setf chunk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
             (the f2cl-lib:integer4 ldwrkr)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (dgemm "N" "N" chunk n n one
         (f2cl-lib:array-slice a-%data% double-float (i 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice work-%data% double-float (iu)
                               ((1 *)) work-%offset%)
         n zero
         (f2cl-lib:array-slice work-%data% double-float (ir)
                               ((1 *)) work-%offset%)
         ldwrkr)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf chunk var-2))
      (when var-3 (setf n var-3))
      (when var-4 (setf n var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf lda var-7))
      (when var-9 (setf n var-9))
      (when var-10 (setf zero var-10))
      (when var-12 (setf ldwrkr var-12)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "F" chunk n
         (f2cl-lib:array-slice work-%data% double-float (ir)
                               ((1 *)) work-%offset%)
         ldwrkr
         (f2cl-lib:array-slice a-%data% double-float (i 1)
```

```
                                  ((1 lda) (1 *)) a-%offset%)
            lda)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf chunk var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf ldwrkr var-4))
        (when var-6 (setf lda var-6)))
    label10)))
(wntqs (setf ir 1) (setf ldwrkr n)
 (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
 (setf nwork (f2cl-lib:int-add itau n))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (dgeqrf m n a lda
      (f2cl-lib:array-slice work-%data% double-float (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-7))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlacpy "U" n n a lda
      (f2cl-lib:array-slice work-%data% double-float (ir) ((1 *))
                            work-%offset%)
      ldwrkr)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldwrkr var-6)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
      zero zero
      (f2cl-lib:array-slice work-%data% double-float ((+ ir 1))
                            ((1 *)) work-%offset%)
      ldwrkr)
   (declare (ignore var-0 var-1 var-2 var-5))
   (when var-3 (setf zero var-3))
   (when var-4 (setf zero var-4))
   (when var-6 (setf ldwrkr var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (dorgqr m n n a lda
      (f2cl-lib:array-slice work-%data% double-float (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (nwork)
```

```
                              ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf nwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgebrd n n
     (f2cl-lib:array-slice work-%data% double-float (ir) ((1 *))
                              work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                              work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                              ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                              ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                              ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dbdsdc "U" "I" n s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                              work-%offset%)
     u ldu vt ldvt dum idum
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                              ((1 *)) work-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (setf n var-2)
  (setf ldu var-6)
  (setf ldvt var-8)
  (setf info var-13))
(multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "Q" "L" "N" n n n
     (f2cl-lib:array-slice work-%data% double-float (ir) ((1 *))
                           work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldwrkr var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "P" "R" "T" n n n
     (f2cl-lib:array-slice work-%data% double-float (ir) ((1 *))
                           work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldwrkr var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "F" n n u ldu
     (f2cl-lib:array-slice work-%data% double-float (ir) ((1 *))
                           work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
```

```
    (when var-6 (setf ldwrkr var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dgemm "N" "N" m n n one a lda
       (f2cl-lib:array-slice work-%data% double-float (ir) ((1 *))
                             work-%offset%)
       ldwrkr zero u ldu)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldwrkr var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldu var-12))))
 (wntqa (setf iu 1) (setf ldwrku n)
  (setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
  (setf nwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% double-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dorgqr m m n u ldu
       (f2cl-lib:array-slice work-%data% double-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf m var-0))
```

```
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice a-%data% double-float (2 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf nwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgebrd n n a lda s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dbdsdc "U" "I" n s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iu) ((1 *))
                           work-%offset%)
     n vt ldvt dum idum
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
```

```
  (setf n var-2)
  (setf n var-6)
  (setf ldvt var-8)
  (setf info var-13))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "Q" "L" "N" n n n a lda
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iu) ((1 *))
                           work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldwrku var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "P" "R" "T" n n n a lda
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dgemm "N" "N" m n n one u ldu
     (f2cl-lib:array-slice work-%data% double-float (iu) ((1 *))
                           work-%offset%)
     ldwrku zero a lda)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
```

```
      (when var-2 (setf m var-2))
      (when var-3 (setf n var-3))
      (when var-4 (setf n var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf ldu var-7))
      (when var-9 (setf ldwrku var-9))
      (when var-10 (setf zero var-10))
      (when var-12 (setf lda var-12)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "F" m n a lda u ldu)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-6 (setf ldu var-6))))))
  (t (setf ie 1) (setf itauq (f2cl-lib:int-add ie n))
   (setf itaup (f2cl-lib:int-add itauq n))
   (setf nwork (f2cl-lib:int-add itaup n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (dgebrd m n a lda s
        (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (nwork) ((1 *))
                              work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-10))
   (cond
    (wntqn
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12 var-13)
         (dbdsdc "U" "N" n s
          (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                                work-%offset%)
          dum 1 dum 1 dum idum
          (f2cl-lib:array-slice work-%data% double-float (nwork)
                                ((1 *)) work-%offset%)
          iwork info)
       (declare
        (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
      var-10 var-11 var-12))
    (setf n var-2)
    (setf info var-13)))
 (wntqo (setf iu nwork)
  (cond
   ((>= lwork
        (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                          (f2cl-lib:int-mul 3 n) bdspac))
    (setf ldwrku m)
    (setf nwork (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "F" m n zero zero
         (f2cl-lib:array-slice work-%data% double-float (iu)
                               ((1 *)) work-%offset%)
         ldwrku)
      (declare (ignore var-0 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldwrku var-6))))
   (t (setf ldwrku n)
    (setf nwork (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
    (setf ir nwork)
    (setf ldwrkr
            (the f2cl-lib:integer4
                 (truncate (- lwork (* n n) (* 3 n)) n)))))
  (setf nwork (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dbdsdc "U" "I" n s
       (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iu) ((1 *))
                             work-%offset%)
       ldwrku vt ldvt dum idum
       (f2cl-lib:array-slice work-%data% double-float (nwork)
                             ((1 *)) work-%offset%)
       iwork info)
    (declare
     (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
      var-11 var-12))
    (setf n var-2)
    (setf ldwrku var-6)
    (setf ldvt var-8)
    (setf info var-13))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
      var-10 var-11 var-12 var-13)
     (dormbr "P" "R" "T" n n n a lda
      (f2cl-lib:array-slice work-%data% double-float (itaup)
                            ((1 *)) work-%offset%)
      vt ldvt
      (f2cl-lib:array-slice work-%data% double-float (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
   (declare
    (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
   (when var-3 (setf n var-3))
   (when var-4 (setf n var-4))
   (when var-5 (setf n var-5))
   (when var-7 (setf lda var-7))
   (when var-10 (setf ldvt var-10))
   (when var-13 (setf ierr var-13)))
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                         (f2cl-lib:int-mul 3 n) bdspac))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10 var-11 var-12 var-13)
       (dormbr "Q" "L" "N" m n n a lda
        (f2cl-lib:array-slice work-%data% double-float (itauq)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (iu)
                              ((1 *)) work-%offset%)
        ldwrku
        (f2cl-lib:array-slice work-%data% double-float (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
     (declare
      (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
     (when var-3 (setf m var-3))
     (when var-4 (setf n var-4))
     (when var-5 (setf n var-5))
     (when var-7 (setf lda var-7))
     (when var-10 (setf ldwrku var-10))
     (when var-13 (setf ierr var-13)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlacpy "F" m n
        (f2cl-lib:array-slice work-%data% double-float (iu)
                              ((1 *)) work-%offset%)
        ldwrku a lda)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf m var-1))
     (when var-2 (setf n var-2))
     (when var-4 (setf ldwrku var-4))
```

```
    (when var-6 (setf lda var-6))))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9)
       (dorgbr "Q" m n n a lda
        (f2cl-lib:array-slice work-%data% double-float (itauq)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i ldwrkr))
                 ((> i m) nil)
    (tagbody
      (setf chunk
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
               (the f2cl-lib:integer4 ldwrkr)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (dgemm "N" "N" chunk n n one
           (f2cl-lib:array-slice a-%data% double-float (i 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice work-%data% double-float (iu)
                                 ((1 *)) work-%offset%)
           ldwrku zero
           (f2cl-lib:array-slice work-%data% double-float (ir)
                                 ((1 *)) work-%offset%)
           ldwrkr)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf chunk var-2))
        (when var-3 (setf n var-3))
        (when var-4 (setf n var-4))
        (when var-5 (setf one var-5))
        (when var-7 (setf lda var-7))
        (when var-9 (setf ldwrku var-9))
        (when var-10 (setf zero var-10))
        (when var-12 (setf ldwrkr var-12)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlacpy "F" chunk n
```

```
            (f2cl-lib:array-slice work-%data% double-float (ir)
                                  ((1 *)) work-%offset%)
            ldwrkr
            (f2cl-lib:array-slice a-%data% double-float (i 1)
                                  ((1 lda) (1 *)) a-%offset%)
            lda)
         (declare (ignore var-0 var-3 var-5))
         (when var-1 (setf chunk var-1))
         (when var-2 (setf n var-2))
         (when var-4 (setf ldwrkr var-4))
         (when var-6 (setf lda var-6)))
      label20)))))
(wntqs
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlaset "F" m n zero zero u ldu)
   (declare (ignore var-0 var-5))
   (when var-1 (setf m var-1))
   (when var-2 (setf n var-2))
   (when var-3 (setf zero var-3))
   (when var-4 (setf zero var-4))
   (when var-6 (setf ldu var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13)
     (dbdsdc "U" "I" n s
      (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                            work-%offset%)
      u ldu vt ldvt dum idum
      (f2cl-lib:array-slice work-%data% double-float (nwork)
                            ((1 *)) work-%offset%)
      iwork info)
   (declare
    (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
     var-11 var-12))
   (setf n var-2)
   (setf ldu var-6)
   (setf ldvt var-8)
   (setf info var-13))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13)
     (dormbr "Q" "L" "N" m n n a lda
      (f2cl-lib:array-slice work-%data% double-float (itauq)
                            ((1 *)) work-%offset%)
      u ldu
      (f2cl-lib:array-slice work-%data% double-float (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
   (declare
    (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
```

```
    (when var-3 (setf m var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf n var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldu var-10))
    (when var-13 (setf ierr var-13)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dormbr "P" "R" "T" n n n a lda
       (f2cl-lib:array-slice work-%data% double-float (itaup)
                             ((1 *)) work-%offset%)
       vt ldvt
       (f2cl-lib:array-slice work-%data% double-float (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf n var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldvt var-10))
    (when var-13 (setf ierr var-13))))
 (wntqa
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "F" m m zero zero u ldu)
    (declare (ignore var-0 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dbdsdc "U" "I" n s
       (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                             work-%offset%)
       u ldu vt ldvt dum idum
       (f2cl-lib:array-slice work-%data% double-float (nwork)
                             ((1 *)) work-%offset%)
       iwork info)
    (declare
     (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
      var-11 var-12))
    (setf n var-2)
    (setf ldu var-6)
    (setf ldvt var-8)
    (setf info var-13))
```

```
(cond
 ((> m n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "F" (f2cl-lib:int-sub m n) (f2cl-lib:int-sub m n)
       zero one
       (f2cl-lib:array-slice u-%data% double-float
                             ((+ n 1) (f2cl-lib:int-add n 1))
                             ((1 ldu) (1 *)) u-%offset%)
       ldu)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldu var-6)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "Q" "L" "N" m m n a lda
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "P" "R" "T" n n m a lda
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13))))))))
```

```
(t
 (cond
  ((>= n mnthr)
   (cond
    (wntqn (setf itau 1) (setf nwork (f2cl-lib:int-add itau m))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (dgelqf m n a lda
          (f2cl-lib:array-slice work-%data% double-float (itau)
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% double-float (nwork)
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
       (declare (ignore var-2 var-4 var-5 var-6))
       (setf m var-0)
       (setf n var-1)
       (setf lda var-3)
       (setf ierr var-7))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
          zero zero
          (f2cl-lib:array-slice a-%data% double-float (1 2)
                                ((1 lda) (1 *)) a-%offset%)
          lda)
       (declare (ignore var-0 var-1 var-2 var-5))
       (when var-3 (setf zero var-3))
       (when var-4 (setf zero var-4))
       (when var-6 (setf lda var-6)))
     (setf ie 1) (setf itauq (f2cl-lib:int-add ie m))
     (setf itaup (f2cl-lib:int-add itauq m))
     (setf nwork (f2cl-lib:int-add itaup m))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (dgebrd m m a lda s
          (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                                work-%offset%)
          (f2cl-lib:array-slice work-%data% double-float (itauq)
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% double-float (itaup)
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% double-float (nwork)
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
       (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
       (setf m var-0)
       (setf m var-1)
       (setf lda var-3)
       (setf ierr var-10))
     (setf nwork (f2cl-lib:int-add ie m))
```

```
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13)
     (dbdsdc "U" "N" m s
      (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                            work-%offset%)
      dum 1 dum 1 dum idum
      (f2cl-lib:array-slice work-%data% double-float (nwork)
                            ((1 *)) work-%offset%)
      iwork info)
   (declare
    (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12))
   (setf m var-2)
   (setf info var-13)))
(wntqo (setf ivt 1)
 (setf il (f2cl-lib:int-add ivt (f2cl-lib:int-mul m m)))
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                         (f2cl-lib:int-mul m m)
                         (f2cl-lib:int-mul 3 m) bdspac))
   (setf ldwrkl m) (setf chunk n))
  (t (setf ldwrkl m)
   (setf chunk
           (the f2cl-lib:integer4
                (truncate (- lwork (* m m)) m)))))
 (setf itau (f2cl-lib:int-add il (f2cl-lib:int-mul ldwrkl m)))
 (setf nwork (f2cl-lib:int-add itau m))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (dgelqf m n a lda
      (f2cl-lib:array-slice work-%data% double-float (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-7))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlacpy "L" m m a lda
      (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                            work-%offset%)
      ldwrkl)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf m var-1))
   (when var-2 (setf m var-2))
```

```
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkl var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ il ldwrkl)) ((1 *)) work-%offset%)
     ldwrkl)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrkl var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorglq m n m a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf nwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgebrd m m
     (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                           work-%offset%)
     ldwrkl s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkl var-3)
  (setf ierr var-10))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dbdsdc "U" "I" m s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% double-float (ivt) ((1 *))
                           work-%offset%)
     m dum idum
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (setf m var-2)
  (setf ldu var-6)
  (setf m var-8)
  (setf info var-13))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "Q" "L" "N" m m m
     (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                           work-%offset%)
     ldwrkl
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwrkl var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "P" "R" "T" m m m
     (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                           work-%offset%)
     ldwrkl
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
```

```
     (f2cl-lib:array-slice work-%data% double-float (ivt) ((1 *))
                           work-%offset%)
     m
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwrkl var-7))
  (when var-10 (setf m var-10))
  (when var-13 (setf ierr var-13)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
              ((> i n) nil)
  (tagbody
    (setf blk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
             (the f2cl-lib:integer4 chunk)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (dgemm "N" "N" m blk m one
         (f2cl-lib:array-slice work-%data% double-float (ivt)
                               ((1 *)) work-%offset%)
         m
         (f2cl-lib:array-slice a-%data% double-float (1 i)
                               ((1 lda) (1 *)) a-%offset%)
         lda zero
         (f2cl-lib:array-slice work-%data% double-float (il)
                               ((1 *)) work-%offset%)
         ldwrkl)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf blk var-3))
      (when var-4 (setf m var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf m var-7))
      (when var-9 (setf lda var-9))
      (when var-10 (setf zero var-10))
      (when var-12 (setf ldwrkl var-12)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "F" m blk
         (f2cl-lib:array-slice work-%data% double-float (il)
                               ((1 *)) work-%offset%)
         ldwrkl
```

```
          (f2cl-lib:array-slice a-%data% double-float (1 i)
                                ((1 lda) (1 *)) a-%offset%)
          lda)
       (declare (ignore var-0 var-3 var-5))
       (when var-1 (setf m var-1))
       (when var-2 (setf blk var-2))
       (when var-4 (setf ldwrkl var-4))
       (when var-6 (setf lda var-6)))
    label30)))
(wntqs (setf il 1) (setf ldwrkl m)
 (setf itau (f2cl-lib:int-add il (f2cl-lib:int-mul ldwrkl m)))
 (setf nwork (f2cl-lib:int-add itau m))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (dgelqf m n a lda
      (f2cl-lib:array-slice work-%data% double-float (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-7))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlacpy "L" m m a lda
      (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                            work-%offset%)
      ldwrkl)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf m var-1))
   (when var-2 (setf m var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldwrkl var-6)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
      zero zero
      (f2cl-lib:array-slice work-%data% double-float
                            ((+ il ldwrkl)) ((1 *)) work-%offset%)
      ldwrkl)
   (declare (ignore var-0 var-1 var-2 var-5))
   (when var-3 (setf zero var-3))
   (when var-4 (setf zero var-4))
   (when var-6 (setf ldwrkl var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (dorglq m n m a lda
      (f2cl-lib:array-slice work-%data% double-float (itau)
                            ((1 *)) work-%offset%)
```

```
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf nwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgebrd m m
     (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                           work-%offset%)
     ldwrkl s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkl var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dbdsdc "U" "I" m s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     u ldu vt ldvt dum idum
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     iwork info)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (setf m var-2)
  (setf ldu var-6)
  (setf ldvt var-8)
  (setf info var-13))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "Q" "L" "N" m m m
     (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                           work-%offset%)
     ldwrkl
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwrkl var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "P" "R" "T" m m m
     (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                           work-%offset%)
     ldwrkl
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldwrkl var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "F" m m vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                           work-%offset%)
     ldwrkl)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
```

```
    (when var-4 (setf ldvt var-4))
    (when var-6 (setf ldwrkl var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dgemm "N" "N" m n m one
       (f2cl-lib:array-slice work-%data% double-float (il) ((1 *))
                             work-%offset%)
       ldwrkl a lda zero vt ldvt)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldwrkl var-7))
    (when var-9 (setf lda var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldvt var-12))))
 (wntqa (setf ivt 1) (setf ldwkvt m)
  (setf itau (f2cl-lib:int-add ivt (f2cl-lib:int-mul ldwkvt m)))
  (setf nwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dgelqf m n a lda
       (f2cl-lib:array-slice work-%data% double-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dorglq n n m vt ldvt
       (f2cl-lib:array-slice work-%data% double-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
```

```
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice a-%data% double-float (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf nwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgebrd m m a lda s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dbdsdc "U" "I" m s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% double-float (ivt) ((1 *))
                           work-%offset%)
     ldwkvt dum idum
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     iwork info)
  (declare
```

```
   (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
    var-11 var-12))
  (setf m var-2)
  (setf ldu var-6)
  (setf ldwkvt var-8)
  (setf info var-13))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "Q" "L" "N" m m m a lda
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "P" "R" "T" m m m a lda
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (ivt) ((1 *))
                           work-%offset%)
     ldwkvt
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldwkvt var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12)
    (dgemm "N" "N" m n m one
     (f2cl-lib:array-slice work-%data% double-float (ivt) ((1 *))
                           work-%offset%)
```

```
          ldwkvt vt ldvt zero a lda)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf n var-3))
        (when var-4 (setf m var-4))
        (when var-5 (setf one var-5))
        (when var-7 (setf ldwkvt var-7))
        (when var-9 (setf ldvt var-9))
        (when var-10 (setf zero var-10))
        (when var-12 (setf lda var-12)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlacpy "F" m n a lda vt ldvt)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldvt var-6))))))
  (t (setf ie 1) (setf itauq (f2cl-lib:int-add ie m))
   (setf itaup (f2cl-lib:int-add itauq m))
   (setf nwork (f2cl-lib:int-add itaup m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (dgebrd m n a lda s
        (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (nwork) ((1 *))
                              work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-10))
   (cond
    (wntqn
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12 var-13)
         (dbdsdc "L" "N" m s
          (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                                work-%offset%)
          dum 1 dum 1 dum idum
          (f2cl-lib:array-slice work-%data% double-float (nwork)
                                ((1 *)) work-%offset%)
          iwork info)
```

```
    (declare
     (ignore var-0 var-1 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12))
    (setf m var-2)
    (setf info var-13)))
  (wntqo (setf ldwkvt m) (setf ivt nwork)
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                           (f2cl-lib:int-mul 3 m) bdspac))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (dlaset "F" m n zero zero
          (f2cl-lib:array-slice work-%data% double-float (ivt)
                                ((1 *)) work-%offset%)
          ldwkvt)
       (declare (ignore var-0 var-5))
       (when var-1 (setf m var-1))
       (when var-2 (setf n var-2))
       (when var-3 (setf zero var-3))
       (when var-4 (setf zero var-4))
       (when var-6 (setf ldwkvt var-6)))
     (setf nwork
             (f2cl-lib:int-add ivt (f2cl-lib:int-mul ldwkvt n))))
    (t
     (setf nwork (f2cl-lib:int-add ivt (f2cl-lib:int-mul ldwkvt m)))
     (setf il nwork)
     (setf chunk
             (the f2cl-lib:integer4
                  (truncate (- lwork (* m m) (* 3 m)) m)))))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13)
       (dbdsdc "L" "I" m s
        (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                              work-%offset%)
        u ldu
        (f2cl-lib:array-slice work-%data% double-float (ivt) ((1 *))
                              work-%offset%)
        ldwkvt dum idum
        (f2cl-lib:array-slice work-%data% double-float (nwork)
                              ((1 *)) work-%offset%)
        iwork info)
     (declare
      (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
       var-11 var-12))
     (setf m var-2)
     (setf ldu var-6)
     (setf ldwkvt var-8)
     (setf info var-13))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "Q" "L" "N" m m n a lda
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(cond
 ((>= lwork
      (f2cl-lib:int-add (f2cl-lib:int-mul m n)
                        (f2cl-lib:int-mul 3 m) bdspac))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13)
      (dormbr "P" "R" "T" m n m a lda
       (f2cl-lib:array-slice work-%data% double-float (itaup)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (ivt)
                             ((1 *)) work-%offset%)
       ldwkvt
       (f2cl-lib:array-slice work-%data% double-float (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf m var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldwkvt var-10))
    (when var-13 (setf ierr var-13)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "F" m n
       (f2cl-lib:array-slice work-%data% double-float (ivt)
                             ((1 *)) work-%offset%)
       ldwkvt a lda)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
```

```
    (when var-2 (setf n var-2))
    (when var-4 (setf ldwkvt var-4))
    (when var-6 (setf lda var-6))))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9)
       (dorgbr "P" m n m a lda
        (f2cl-lib:array-slice work-%data% double-float (itaup)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (nwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
                 ((> i n) nil)
    (tagbody
      (setf blk
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
               (the f2cl-lib:integer4 chunk)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (dgemm "N" "N" m blk m one
           (f2cl-lib:array-slice work-%data% double-float (ivt)
                                 ((1 *)) work-%offset%)
           ldwkvt
           (f2cl-lib:array-slice a-%data% double-float (1 i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda zero
           (f2cl-lib:array-slice work-%data% double-float (il)
                                 ((1 *)) work-%offset%)
           m)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf m var-2))
        (when var-3 (setf blk var-3))
        (when var-4 (setf m var-4))
        (when var-5 (setf one var-5))
        (when var-7 (setf ldwkvt var-7))
        (when var-9 (setf lda var-9))
        (when var-10 (setf zero var-10))
        (when var-12 (setf m var-12)))
      (multiple-value-bind
```

```
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dlacpy "F" m blk
             (f2cl-lib:array-slice work-%data% double-float (il)
                                   ((1 *)) work-%offset%)
             m
             (f2cl-lib:array-slice a-%data% double-float (1 i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf blk var-2))
          (when var-4 (setf m var-4))
          (when var-6 (setf lda var-6)))
      label40)))))
(wntqs
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlaset "F" m n zero zero vt ldvt)
   (declare (ignore var-0 var-5))
   (when var-1 (setf m var-1))
   (when var-2 (setf n var-2))
   (when var-3 (setf zero var-3))
   (when var-4 (setf zero var-4))
   (when var-6 (setf ldvt var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13)
     (dbdsdc "L" "I" m s
      (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                            work-%offset%)
      u ldu vt ldvt dum idum
      (f2cl-lib:array-slice work-%data% double-float (nwork)
                            ((1 *)) work-%offset%)
      iwork info)
   (declare
    (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
     var-11 var-12))
   (setf m var-2)
   (setf ldu var-6)
   (setf ldvt var-8)
   (setf info var-13))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13)
     (dormbr "Q" "L" "N" m m n a lda
      (f2cl-lib:array-slice work-%data% double-float (itauq)
                            ((1 *)) work-%offset%)
      u ldu
      (f2cl-lib:array-slice work-%data% double-float (nwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
```

```
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf n var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldu var-10))
    (when var-13 (setf ierr var-13)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dormbr "P" "R" "T" m n m a lda
       (f2cl-lib:array-slice work-%data% double-float (itaup)
                             ((1 *)) work-%offset%)
       vt ldvt
       (f2cl-lib:array-slice work-%data% double-float (nwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
    (declare
     (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
    (when var-3 (setf m var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf m var-5))
    (when var-7 (setf lda var-7))
    (when var-10 (setf ldvt var-10))
    (when var-13 (setf ierr var-13))))
 (wntqa
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "F" n n zero zero vt ldvt)
    (declare (ignore var-0 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf zero var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dbdsdc "L" "I" m s
       (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                             work-%offset%)
       u ldu vt ldvt dum idum
       (f2cl-lib:array-slice work-%data% double-float (nwork)
                             ((1 *)) work-%offset%)
       iwork info)
    (declare
     (ignore var-0 var-1 var-3 var-4 var-5 var-7 var-9 var-10
      var-11 var-12))
    (setf m var-2)
    (setf ldu var-6)
```

```
  (setf ldvt var-8)
  (setf info var-13))
(cond
 ((> n m)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "F" (f2cl-lib:int-sub n m) (f2cl-lib:int-sub n m)
       zero one
       (f2cl-lib:array-slice vt-%data% double-float
                             ((+ m 1) (f2cl-lib:int-add m 1))
                             ((1 ldvt) (1 *)) vt-%offset%)
       ldvt)
    (declare (ignore var-0 var-1 var-2 var-5))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldvt var-6)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "Q" "L" "N" m m n a lda
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13)
    (dormbr "P" "R" "T" n n m a lda
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (nwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork nwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
```

```
            (when var-10 (setf ldvt var-10))
            (when var-13 (setf ierr var-13)))))))))
    (cond
     ((= iscl 1)
      (if (> anrm bignum)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (dlascl "G" 0 0 bignum anrm minmn 1 s minmn ierr)
            (declare (ignore var-0 var-1 var-2 var-6 var-7))
            (when var-3 (setf bignum var-3))
            (when var-4 (setf anrm var-4))
            (when var-5 (setf minmn var-5))
            (when var-8 (setf minmn var-8))
            (when var-9 (setf ierr var-9))))
      (if (< anrm smlnum)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
              (dlascl "G" 0 0 smlnum anrm minmn 1 s minmn ierr)
            (declare (ignore var-0 var-1 var-2 var-6 var-7))
            (when var-3 (setf smlnum var-3))
            (when var-4 (setf anrm var-4))
            (when var-5 (setf minmn var-5))
            (when var-8 (setf minmn var-8))
            (when var-9 (setf ierr var-9))))))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce (the f2cl-lib:integer4 maxwrk) 'double-float))
    (go end_label)
   end_label
    (return
     (values jobz m n nil lda nil nil ldu nil ldvt nil nil nil info))))))
```

———-

90.129 dgesvd

90.129.1 Axiom unit tests

— **dgesvd.input** —

```
)set break resume
)sys rm -f dgesvd.output
)spool dgesvd.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

90.129.2 Axiom help page

— **dgesvd.help** —

```
=====================================================================
dgesvd examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

NAME
       DGESVD - the singular value decomposition (SVD) of a real M-by-N matrix
       A, optionally computing the left and/or right singular vectors

SYNOPSIS
       SUBROUTINE DGESVD( JOBU, JOBVT, M, N, A, LDA,  S,  U,  LDU,  VT,  LDVT,
                          WORK, LWORK, INFO )

           CHARACTER      JOBU, JOBVT

           INTEGER        INFO, LDA, LDU, LDVT, LWORK, M, N

           DOUBLE         PRECISION  A(  LDA,  *  ),  S( * ), U( LDU, * ), VT(
                          LDVT, * ), WORK( * )

  Purpose
  =======

  DGESVD computes the singular value decomposition (SVD) of a real
  M-by-N matrix A, optionally computing the left and/or right singular
  vectors. The SVD is written

       A = U * SIGMA * transpose(V)

  where SIGMA is an M-by-N matrix which is zero except for its
  min(m,n) diagonal elements, U is an M-by-M orthogonal matrix, and
  V is an N-by-N orthogonal matrix.  The diagonal elements of SIGMA
  are the singular values of A; they are real and non-negative, and
  are returned in descending order.  The first min(m,n) columns of
  U and V are the left and right singular vectors of A.
```

```
Note that the routine returns V**T, not V.

Arguments
=========

JOBU    (input) CHARACTER*1
        Specifies options for computing all or part of the matrix U:
        = 'A':  all M columns of U are returned in array U:
        = 'S':  the first min(m,n) columns of U (the left singular
                vectors) are returned in the array U;
        = 'O':  the first min(m,n) columns of U (the left singular
                vectors) are overwritten on the array A;
        = 'N':  no columns of U (no left singular vectors) are
                computed.

JOBVT   (input) CHARACTER*1
        Specifies options for computing all or part of the matrix
        V**T:
        = 'A':  all N rows of V**T are returned in the array VT;
        = 'S':  the first min(m,n) rows of V**T (the right singular
                vectors) are returned in the array VT;
        = 'O':  the first min(m,n) rows of V**T (the right singular
                vectors) are overwritten on the array A;
        = 'N':  no rows of V**T (no right singular vectors) are
                computed.

        JOBVT and JOBU cannot both be 'O'.

M       (input) INTEGER
        The number of rows of the input matrix A.  M >= 0.

N       (input) INTEGER
        The number of columns of the input matrix A.  N >= 0.

A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
        On entry, the M-by-N matrix A.
        On exit,
        if JOBU = 'O',  A is overwritten with the first min(m,n)
                        columns of U (the left singular vectors,
                        stored columnwise);
        if JOBVT = 'O', A is overwritten with the first min(m,n)
                        rows of V**T (the right singular vectors,
                        stored rowwise);
        if JOBU .ne. 'O' and JOBVT .ne. 'O', the contents of A
                        are destroyed.

LDA     (input) INTEGER
        The leading dimension of the array A.  LDA >= max(1,M).

S       (output) DOUBLE PRECISION array, dimension (min(M,N))
```

```
        The singular values of A, sorted so that S(i) >= S(i+1).

U       (output) DOUBLE PRECISION array, dimension (LDU,UCOL)
        (LDU,M) if JOBU = 'A' or (LDU,min(M,N)) if JOBU = 'S'.
        If JOBU = 'A', U contains the M-by-M orthogonal matrix U;
        if JOBU = 'S', U contains the first min(m,n) columns of U
        (the left singular vectors, stored columnwise);
        if JOBU = 'N' or 'O', U is not referenced.

LDU     (input) INTEGER
        The leading dimension of the array U.  LDU >= 1; if
        JOBU = 'S' or 'A', LDU >= M.

VT      (output) DOUBLE PRECISION array, dimension (LDVT,N)
        If JOBVT = 'A', VT contains the N-by-N orthogonal matrix
        V**T;
        if JOBVT = 'S', VT contains the first min(m,n) rows of
        V**T (the right singular vectors, stored rowwise);
        if JOBVT = 'N' or 'O', VT is not referenced.

LDVT    (input) INTEGER
        The leading dimension of the array VT.  LDVT >= 1; if
        JOBVT = 'A', LDVT >= N; if JOBVT = 'S', LDVT >= min(M,N).

WORK    (workspace/output) DOUBLE PRECISION array, dimension (LWORK)
        On exit, if INFO = 0, WORK(1) returns the optimal LWORK;
        if INFO > 0, WORK(2:MIN(M,N)) contains the unconverged
        superdiagonal elements of an upper bidiagonal matrix B
        whose diagonal is in S (not necessarily sorted). B
        satisfies A = U * B * VT, so it has the same singular values
        as A, and singular vectors related by U and VT.

LWORK   (input) INTEGER
        The dimension of the array WORK. LWORK >= 1.
        LWORK >= MAX(3*MIN(M,N)+MAX(M,N),5*MIN(M,N)).
        For good performance, LWORK should generally be larger.

        If LWORK = -1, then a workspace query is assumed; the routine
        only calculates the optimal size of the WORK array, returns
        this value as the first entry of the WORK array, and no error
        message related to LWORK is issued by XERBLA.

INFO    (output) INTEGER
        = 0:  successful exit.
        < 0:  if INFO = -i, the i-th argument had an illegal value.
        > 0:  if DBDSQR did not converge, INFO specifies how many
              superdiagonals of an intermediate bidiagonal form B
              did not converge to zero. See the description of WORK
              above for details.
```

———-

90.129.3 fortran code

— **dgesvd.f** —

```
*  =====================================================================
      SUBROUTINE DGESVD( JOBU, JOBVT, M, N, A, LDA, S, U, LDU,
     $                   VT, LDVT, WORK, LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.1) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     April 2012
*
*     .. Scalar Arguments ..
      CHARACTER          JOBU, JOBVT
      INTEGER            INFO, LDA, LDU, LDVT, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), S( * ), U( LDU, * ),
     $                   VT( LDVT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, WNTUA, WNTUAS, WNTUN, WNTUO, WNTUS,
     $                   WNTVA, WNTVAS, WNTVN, WNTVO, WNTVS
      INTEGER            BDSPAC, BLK, CHUNK, I, IE, IERR, IR, ISCL,
     $                   ITAU, ITAUP, ITAUQ, IU, IWORK, LDWRKR, LDWRKU,
     $                   MAXWRK, MINMN, MINWRK, MNTHR, NCU, NCVT, NRU,
     $                   NRVT, WRKBL
      INTEGER            LWORK_DGEQRF, LWORK_DORGQR_N, LWORK_DORGQR_M,
     $                   LWORK_DGEBRD, LWORK_DORGBR_P, LWORK_DORGBR_Q,
     $                   LWORK_DGELQF, LWORK_DORGLQ_N, LWORK_DORGLQ_M
      DOUBLE PRECISION   ANRM, BIGNUM, EPS, SMLNUM
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DBDSQR, DGEBRD, DGELQF, DGEMM, DGEQRF, DLACPY,
     $                   DLASCL, DLASET, DORGBR, DORGLQ, DORGQR, DORMBR,
```

```
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           LSAME, ILAENV, DLAMCH, DLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      MINMN = MIN( M, N )
      WNTUA = LSAME( JOBU, 'A' )
      WNTUS = LSAME( JOBU, 'S' )
      WNTUAS = WNTUA .OR. WNTUS
      WNTUO = LSAME( JOBU, 'O' )
      WNTUN = LSAME( JOBU, 'N' )
      WNTVA = LSAME( JOBVT, 'A' )
      WNTVS = LSAME( JOBVT, 'S' )
      WNTVAS = WNTVA .OR. WNTVS
      WNTVO = LSAME( JOBVT, 'O' )
      WNTVN = LSAME( JOBVT, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
*
      IF( .NOT.( WNTUA .OR. WNTUS .OR. WNTUO .OR. WNTUN ) ) THEN
         INFO = -1
      ELSE IF( .NOT.( WNTVA .OR. WNTVS .OR. WNTVO .OR. WNTVN ) .OR.
     $         ( WNTVO .AND. WNTUO ) ) THEN
         INFO = -2
      ELSE IF( M.LT.0 ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -6
      ELSE IF( LDU.LT.1 .OR. ( WNTUAS .AND. LDU.LT.M ) ) THEN
         INFO = -9
      ELSE IF( LDVT.LT.1 .OR. ( WNTVA .AND. LDVT.LT.N ) .OR.
     $         ( WNTVS .AND. LDVT.LT.MINMN ) ) THEN
         INFO = -11
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
```

```
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 ) THEN
         MINWRK = 1
         MAXWRK = 1
         IF( M.GE.N .AND. MINMN.GT.0 ) THEN
*
*           Compute space needed for DBDSQR
*
            MNTHR = ILAENV( 6, 'DGESVD', JOBU // JOBVT, M, N, 0, 0 )
            BDSPAC = 5*N
*           Compute space needed for DGEQRF
            CALL DGEQRF( M, N, A, LDA, DUM(1), DUM(1), -1, IERR )
            LWORK_DGEQRF=DUM(1)
*           Compute space needed for DORGQR
            CALL DORGQR( M, N, N, A, LDA, DUM(1), DUM(1), -1, IERR )
            LWORK_DORGQR_N=DUM(1)
            CALL DORGQR( M, M, N, A, LDA, DUM(1), DUM(1), -1, IERR )
            LWORK_DORGQR_M=DUM(1)
*           Compute space needed for DGEBRD
            CALL DGEBRD( N, N, A, LDA, S, DUM(1), DUM(1),
     $                   DUM(1), DUM(1), -1, IERR )
            LWORK_DGEBRD=DUM(1)
*           Compute space needed for DORGBR P
            CALL DORGBR( 'P', N, N, N, A, LDA, DUM(1),
     $                   DUM(1), -1, IERR )
            LWORK_DORGBR_P=DUM(1)
*           Compute space needed for DORGBR Q
            CALL DORGBR( 'Q', N, N, N, A, LDA, DUM(1),
     $                   DUM(1), -1, IERR )
            LWORK_DORGBR_Q=DUM(1)
*
            IF( M.GE.MNTHR ) THEN
               IF( WNTUN ) THEN
*
*                 Path 1 (M much larger than N, JOBU='N')
*
                  MAXWRK = N + LWORK_DGEQRF
                  MAXWRK = MAX( MAXWRK, 3*N+LWORK_DGEBRD )
                  IF( WNTVO .OR. WNTVAS )
     $               MAXWRK = MAX( MAXWRK, 3*N+LWORK_DORGBR_P )
                  MAXWRK = MAX( MAXWRK, BDSPAC )
                  MINWRK = MAX( 4*N, BDSPAC )
               ELSE IF( WNTUO .AND. WNTVN ) THEN
*
*                 Path 2 (M much larger than N, JOBU='O', JOBVT='N')
*
                  WRKBL = N + LWORK_DGEQRF
```

```
                 WRKBL = MAX( WRKBL, N+LWORK_DORGQR_N )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = MAX( N*N+WRKBL, N*N+M*N+N )
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUO .AND. WNTVAS ) THEN
*
*                Path 3 (M much larger than N, JOBU='O', JOBVT='S' or
*                'A')
*
                 WRKBL = N + LWORK_DGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_DORGQR_N )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = MAX( N*N+WRKBL, N*N+M*N+N )
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUS .AND. WNTVN ) THEN
*
*                Path 4 (M much larger than N, JOBU='S', JOBVT='N')
*
                 WRKBL = N + LWORK_DGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_DORGQR_N )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUS .AND. WNTVO ) THEN
*
*                Path 5 (M much larger than N, JOBU='S', JOBVT='O')
*
                 WRKBL = N + LWORK_DGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_DORGQR_N )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = 2*N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUS .AND. WNTVAS ) THEN
*
*                Path 6 (M much larger than N, JOBU='S', JOBVT='S' or
*                'A')
*
                 WRKBL = N + LWORK_DGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_DORGQR_N )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DGEBRD )
```

```
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUA .AND. WNTVN ) THEN
*
*                Path 7 (M much larger than N, JOBU='A', JOBVT='N')
*
                 WRKBL = N + LWORK_DGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_DORGQR_M )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUA .AND. WNTVO ) THEN
*
*                Path 8 (M much larger than N, JOBU='A', JOBVT='O')
*
                 WRKBL = N + LWORK_DGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_DORGQR_M )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = 2*N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
              ELSE IF( WNTUA .AND. WNTVAS ) THEN
*
*                Path 9 (M much larger than N, JOBU='A', JOBVT='S' or
*                'A')
*
                 WRKBL = N + LWORK_DGEQRF
                 WRKBL = MAX( WRKBL, N+LWORK_DORGQR_M )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, 3*N+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
              END IF
           ELSE
*
*             Path 10 (M at least N, but not much larger)
*
              CALL DGEBRD( M, N, A, LDA, S, DUM(1), DUM(1),
     $                     DUM(1), DUM(1), -1, IERR )
              LWORK_DGEBRD=DUM(1)
              MAXWRK = 3*N + LWORK_DGEBRD
```

```
               IF( WNTUS .OR. WNTUO ) THEN
                  CALL DORGBR( 'Q', M, N, N, A, LDA, DUM(1),
     $                   DUM(1), -1, IERR )
                  LWORK_DORGBR_Q=DUM(1)
                  MAXWRK = MAX( MAXWRK, 3*N+LWORK_DORGBR_Q )
               END IF
               IF( WNTUA ) THEN
                  CALL DORGBR( 'Q', M, M, N, A, LDA, DUM(1),
     $                   DUM(1), -1, IERR )
                  LWORK_DORGBR_Q=DUM(1)
                  MAXWRK = MAX( MAXWRK, 3*N+LWORK_DORGBR_Q )
               END IF
               IF( .NOT.WNTVN ) THEN
                 MAXWRK = MAX( MAXWRK, 3*N+LWORK_DORGBR_P )
               END IF
               MAXWRK = MAX( MAXWRK, BDSPAC )
               MINWRK = MAX( 3*N+M, BDSPAC )
            END IF
         ELSE IF( MINMN.GT.0 ) THEN
*
*           Compute space needed for DBDSQR
*
            MNTHR = ILAENV( 6, 'DGESVD', JOBU // JOBVT, M, N, 0, 0 )
            BDSPAC = 5*M
*           Compute space needed for DGELQF
            CALL DGELQF( M, N, A, LDA, DUM(1), DUM(1), -1, IERR )
            LWORK_DGELQF=DUM(1)
*           Compute space needed for DORGLQ
            CALL DORGLQ( N, N, M, DUM(1), N, DUM(1), DUM(1), -1, IERR )
            LWORK_DORGLQ_N=DUM(1)
            CALL DORGLQ( M, N, M, A, LDA, DUM(1), DUM(1), -1, IERR )
            LWORK_DORGLQ_M=DUM(1)
*           Compute space needed for DGEBRD
            CALL DGEBRD( M, M, A, LDA, S, DUM(1), DUM(1),
     $                   DUM(1), DUM(1), -1, IERR )
            LWORK_DGEBRD=DUM(1)
*            Compute space needed for DORGBR P
            CALL DORGBR( 'P', M, M, M, A, N, DUM(1),
     $                   DUM(1), -1, IERR )
            LWORK_DORGBR_P=DUM(1)
*           Compute space needed for DORGBR Q
            CALL DORGBR( 'Q', M, M, M, A, N, DUM(1),
     $                   DUM(1), -1, IERR )
            LWORK_DORGBR_Q=DUM(1)
            IF( N.GE.MNTHR ) THEN
               IF( WNTVN ) THEN
*
*                 Path 1t(N much larger than M, JOBVT='N')
*
                  MAXWRK = M + LWORK_DGELQF
```

```
                 MAXWRK = MAX( MAXWRK, 3*M+LWORK_DGEBRD )
                 IF( WNTUO .OR. WNTUAS )
     $              MAXWRK = MAX( MAXWRK, 3*M+LWORK_DORGBR_Q )
                 MAXWRK = MAX( MAXWRK, BDSPAC )
                 MINWRK = MAX( 4*M, BDSPAC )
              ELSE IF( WNTVO .AND. WNTUN ) THEN
*
*                Path 2t(N much larger than M, JOBU='N', JOBVT='O')
*
                 WRKBL = M + LWORK_DGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_DORGLQ_M )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = MAX( M*M+WRKBL, M*M+M*N+M )
                 MINWRK = MAX( 3*M+N, BDSPAC )
              ELSE IF( WNTVO .AND. WNTUAS ) THEN
*
*                Path 3t(N much larger than M, JOBU='S' or 'A',
*                JOBVT='O')
*
                 WRKBL = M + LWORK_DGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_DORGLQ_M )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = MAX( M*M+WRKBL, M*M+M*N+M )
                 MINWRK = MAX( 3*M+N, BDSPAC )
              ELSE IF( WNTVS .AND. WNTUN ) THEN
*
*                Path 4t(N much larger than M, JOBU='N', JOBVT='S')
*
                 WRKBL = M + LWORK_DGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_DORGLQ_M )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
              ELSE IF( WNTVS .AND. WNTUO ) THEN
*
*                Path 5t(N much larger than M, JOBU='O', JOBVT='S')
*
                 WRKBL = M + LWORK_DGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_DORGLQ_M )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
```

```
                 MAXWRK = 2*M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
              ELSE IF( WNTVS .AND. WNTUAS ) THEN
*
*                Path 6t(N much larger than M, JOBU='S' or 'A',
*                JOBVT='S')
*
                 WRKBL = M + LWORK_DGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_DORGLQ_M )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
              ELSE IF( WNTVA .AND. WNTUN ) THEN
*
*                Path 7t(N much larger than M, JOBU='N', JOBVT='A')
*
                 WRKBL = M + LWORK_DGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_DORGLQ_N )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
              ELSE IF( WNTVA .AND. WNTUO ) THEN
*
*                Path 8t(N much larger than M, JOBU='O', JOBVT='A')
*
                 WRKBL = M + LWORK_DGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_DORGLQ_N )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = 2*M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
              ELSE IF( WNTVA .AND. WNTUAS ) THEN
*
*                Path 9t(N much larger than M, JOBU='S' or 'A',
*                JOBVT='A')
*
                 WRKBL = M + LWORK_DGELQF
                 WRKBL = MAX( WRKBL, M+LWORK_DORGLQ_N )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DGEBRD )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_P )
                 WRKBL = MAX( WRKBL, 3*M+LWORK_DORGBR_Q )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = M*M + WRKBL
```

```
                 MINWRK = MAX( 3*M+N, BDSPAC )
              END IF
           ELSE
*
*             Path 10t(N greater than M, but not much larger)
*
              CALL DGEBRD( M, N, A, LDA, S, DUM(1), DUM(1),
     $                     DUM(1), DUM(1), -1, IERR )
              LWORK_DGEBRD=DUM(1)
              MAXWRK = 3*M + LWORK_DGEBRD
              IF( WNTVS .OR. WNTVO ) THEN
*               Compute space needed for DORGBR P
                CALL DORGBR( 'P', M, N, M, A, N, DUM(1),
     $                     DUM(1), -1, IERR )
                LWORK_DORGBR_P=DUM(1)
                MAXWRK = MAX( MAXWRK, 3*M+LWORK_DORGBR_P )
              END IF
              IF( WNTVA ) THEN
                CALL DORGBR( 'P', N, N, M, A, N, DUM(1),
     $                     DUM(1), -1, IERR )
                LWORK_DORGBR_P=DUM(1)
                MAXWRK = MAX( MAXWRK, 3*M+LWORK_DORGBR_P )
              END IF
              IF( .NOT.WNTUN ) THEN
                 MAXWRK = MAX( MAXWRK, 3*M+LWORK_DORGBR_Q )
              END IF
              MAXWRK = MAX( MAXWRK, BDSPAC )
              MINWRK = MAX( 3*M+N, BDSPAC )
           END IF
        END IF
        MAXWRK = MAX( MAXWRK, MINWRK )
        WORK( 1 ) = MAXWRK
*
        IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
           INFO = -13
        END IF
     END IF
*
     IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'DGESVD', -INFO )
        RETURN
     ELSE IF( LQUERY ) THEN
        RETURN
     END IF
*
*    Quick return if possible
*
     IF( M.EQ.0 .OR. N.EQ.0 ) THEN
        RETURN
     END IF
```

```
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = SQRT( DLAMCH( 'S' ) ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = DLANGE( 'M', M, N, A, LDA, DUM )
      ISCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ISCL = 1
         CALL DLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, IERR )
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ISCL = 1
         CALL DLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, IERR )
      END IF
*
      IF( M.GE.N ) THEN
*
*        A has at least as many rows as columns. If A has sufficiently
*        more rows than columns, first reduce using the QR
*        decomposition (if sufficient workspace available)
*
         IF( M.GE.MNTHR ) THEN
*
            IF( WNTUN ) THEN
*
*              Path 1 (M much larger than N, JOBU='N')
*              No left singular vectors to be computed
*
               ITAU = 1
               IWORK = ITAU + N
*
*              Compute A=Q*R
*              (Workspace: need 2*N, prefer N+N*NB)
*
               CALL DGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
     $                      LWORK-IWORK+1, IERR )
*
*              Zero out below R
*
               CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ), LDA )
               IE = 1
               ITAUQ = IE + N
               ITAUP = ITAUQ + N
               IWORK = ITAUP + N
*
*              Bidiagonalize R in A
```

```
*              (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
               CALL DGEBRD( N, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                      IERR )
               NCVT = 0
               IF( WNTVO .OR. WNTVAS ) THEN
*
*                 If right singular vectors desired, generate P'.
*                 (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                 CALL DORGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 NCVT = N
               END IF
               IWORK = IE + N
*
*              Perform bidiagonal QR iteration, computing right
*              singular vectors of A in A if desired
*              (Workspace: need BDSPAC)
*
               CALL DBDSQR( 'U', N, NCVT, 0, 0, S, WORK( IE ), A, LDA,
     $                      DUM, 1, DUM, 1, WORK( IWORK ), INFO )
*
*              If right singular vectors desired in VT, copy them there
*
               IF( WNTVAS )
     $            CALL DLACPY( 'F', N, N, A, LDA, VT, LDVT )
*
            ELSE IF( WNTUO .AND. WNTVN ) THEN
*
*              Path 2 (M much larger than N, JOBU='O', JOBVT='N')
*              N left singular vectors to be overwritten on A and
*              no right singular vectors to be computed
*
               IF( LWORK.GE.N*N+MAX( 4*N, BDSPAC ) ) THEN
*
*                 Sufficient workspace for a fast algorithm
*
                 IR = 1
                 IF( LWORK.GE.MAX( WRKBL, LDA*N+N )+LDA*N ) THEN
*
*                   WORK(IU) is LDA by N, WORK(IR) is LDA by N
*
                   LDWRKU = LDA
                   LDWRKR = LDA
                 ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N+N )+N*N ) THEN
*
*                   WORK(IU) is LDA by N, WORK(IR) is N by N
*
```

```
                 LDWRKU = LDA
                 LDWRKR = N
              ELSE
*
*                WORK(IU) is LDWRKU by N, WORK(IR) is N by N
*
                 LDWRKU = ( LWORK-N*N-N ) / N
                 LDWRKR = N
              END IF
              ITAU = IR + LDWRKR*N
              IWORK = ITAU + N
*
*             Compute A=Q*R
*             (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
              CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*             Copy R to WORK(IR) and zero out below it
*
              CALL DLACPY( 'U', N, N, A, LDA, WORK( IR ), LDWRKR )
              CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, WORK( IR+1 ),
     $                     LDWRKR )
*
*             Generate Q in A
*             (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
              CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
              IE = ITAU
              ITAUQ = IE + N
              ITAUP = ITAUQ + N
              IWORK = ITAUP + N
*
*             Bidiagonalize R in WORK(IR)
*             (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
              CALL DGEBRD( N, N, WORK( IR ), LDWRKR, S, WORK( IE ),
     $                     WORK( ITAUQ ), WORK( ITAUP ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*             Generate left vectors bidiagonalizing R
*             (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
              CALL DORGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                     WORK( ITAUQ ), WORK( IWORK ),
     $                     LWORK-IWORK+1, IERR )
              IWORK = IE + N
*
*             Perform bidiagonal QR iteration, computing left
```

```
*                 singular vectors of R in WORK(IR)
*                 (Workspace: need N*N+BDSPAC)
*
                  CALL DBDSQR( 'U', N, 0, N, 0, S, WORK( IE ), DUM, 1,
     $                         WORK( IR ), LDWRKR, DUM, 1,
     $                         WORK( IWORK ), INFO )
                  IU = IE + N
*
*                 Multiply Q in A by left singular vectors of R in
*                 WORK(IR), storing result in WORK(IU) and copying to A
*                 (Workspace: need N*N+2*N, prefer N*N+M*N+N)
*
                  DO 10 I = 1, M, LDWRKU
                     CHUNK = MIN( M-I+1, LDWRKU )
                     CALL DGEMM( 'N', 'N', CHUNK, N, N, ONE, A( I, 1 ),
     $                           LDA, WORK( IR ), LDWRKR, ZERO,
     $                           WORK( IU ), LDWRKU )
                     CALL DLACPY( 'F', CHUNK, N, WORK( IU ), LDWRKU,
     $                            A( I, 1 ), LDA )
   10             CONTINUE
*
               ELSE
*
*                 Insufficient workspace for a fast algorithm
*
                  IE = 1
                  ITAUQ = IE + N
                  ITAUP = ITAUQ + N
                  IWORK = ITAUP + N
*
*                 Bidiagonalize A
*                 (Workspace: need 3*N+M, prefer 3*N+(M+N)*NB)
*
                  CALL DGEBRD( M, N, A, LDA, S, WORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Generate left vectors bidiagonalizing A
*                 (Workspace: need 4*N, prefer 3*N+N*NB)
*
                  CALL DORGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IWORK = IE + N
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of A in A
*                 (Workspace: need BDSPAC)
*
                  CALL DBDSQR( 'U', N, 0, M, 0, S, WORK( IE ), DUM, 1,
     $                         A, LDA, DUM, 1, WORK( IWORK ), INFO )
```

```
*
               END IF
*
            ELSE IF( WNTUO .AND. WNTVAS ) THEN
*
*              Path 3 (M much larger than N, JOBU='O', JOBVT='S' or 'A')
*              N left singular vectors to be overwritten on A and
*              N right singular vectors to be computed in VT
*
               IF( LWORK.GE.N*N+MAX( 4*N, BDSPAC ) ) THEN
*
*                 Sufficient workspace for a fast algorithm
*
                  IR = 1
                  IF( LWORK.GE.MAX( WRKBL, LDA*N+N )+LDA*N ) THEN
*
*                    WORK(IU) is LDA by N and WORK(IR) is LDA by N
*
                     LDWRKU = LDA
                     LDWRKR = LDA
                  ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N+N )+N*N ) THEN
*
*                    WORK(IU) is LDA by N and WORK(IR) is N by N
*
                     LDWRKU = LDA
                     LDWRKR = N
                  ELSE
*
*                    WORK(IU) is LDWRKU by N and WORK(IR) is N by N
*
                     LDWRKU = ( LWORK-N*N-N ) / N
                     LDWRKR = N
                  END IF
                  ITAU = IR + LDWRKR*N
                  IWORK = ITAU + N
*
*                 Compute A=Q*R
*                 (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                  CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Copy R to VT, zeroing out below it
*
                  CALL DLACPY( 'U', N, N, A, LDA, VT, LDVT )
                  IF( N.GT.1 )
     $               CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                            VT( 2, 1 ), LDVT )
*
*                 Generate Q in A
```

```
*                 (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                  CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IE = ITAU
                  ITAUQ = IE + N
                  ITAUP = ITAUQ + N
                  IWORK = ITAUP + N
*
*                 Bidiagonalize R in VT, copying result to WORK(IR)
*                 (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                  CALL DGEBRD( N, N, VT, LDVT, S, WORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  CALL DLACPY( 'L', N, N, VT, LDVT, WORK( IR ), LDWRKR )
*
*                 Generate left vectors bidiagonalizing R in WORK(IR)
*                 (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                  CALL DORGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                         WORK( ITAUQ ), WORK( IWORK ),
     $                         LWORK-IWORK+1, IERR )
*
*                 Generate right vectors bidiagonalizing R in VT
*                 (Workspace: need N*N+4*N-1, prefer N*N+3*N+(N-1)*NB)
*
                  CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IWORK = IE + N
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of R in WORK(IR) and computing right
*                 singular vectors of R in VT
*                 (Workspace: need N*N+BDSPAC)
*
                  CALL DBDSQR( 'U', N, N, N, 0, S, WORK( IE ), VT, LDVT,
     $                         WORK( IR ), LDWRKR, DUM, 1,
     $                         WORK( IWORK ), INFO )
                  IU = IE + N
*
*                 Multiply Q in A by left singular vectors of R in
*                 WORK(IR), storing result in WORK(IU) and copying to A
*                 (Workspace: need N*N+2*N, prefer N*N+M*N+N)
*
                  DO 20 I = 1, M, LDWRKU
                     CHUNK = MIN( M-I+1, LDWRKU )
                     CALL DGEMM( 'N', 'N', CHUNK, N, N, ONE, A( I, 1 ),
     $                           LDA, WORK( IR ), LDWRKR, ZERO,
     $                           WORK( IU ), LDWRKU )
```

```
                 CALL DLACPY( 'F', CHUNK, N, WORK( IU ), LDWRKU,
     $                        A( I, 1 ), LDA )
   20         CONTINUE
*
            ELSE
*
*              Insufficient workspace for a fast algorithm
*
               ITAU = 1
               IWORK = ITAU + N
*
*              Compute A=Q*R
*              (Workspace: need 2*N, prefer N+N*NB)
*
               CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*              Copy R to VT, zeroing out below it
*
               CALL DLACPY( 'U', N, N, A, LDA, VT, LDVT )
               IF( N.GT.1 )
     $            CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                        VT( 2, 1 ), LDVT )
*
*              Generate Q in A
*              (Workspace: need 2*N, prefer N+N*NB)
*
               CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
               IE = ITAU
               ITAUQ = IE + N
               ITAUP = ITAUQ + N
               IWORK = ITAUP + N
*
*              Bidiagonalize R in VT
*              (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
               CALL DGEBRD( N, N, VT, LDVT, S, WORK( IE ),
     $                      WORK( ITAUQ ), WORK( ITAUP ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*              Multiply Q in A by left vectors bidiagonalizing R
*              (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
               CALL DORMBR( 'Q', 'R', 'N', M, N, N, VT, LDVT,
     $                      WORK( ITAUQ ), A, LDA, WORK( IWORK ),
     $                      LWORK-IWORK+1, IERR )
*
*              Generate right vectors bidiagonalizing R in VT
*              (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
```

```
*
                 CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IWORK = IE + N
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of A in A and computing right
*                singular vectors of A in VT
*                (Workspace: need BDSPAC)
*
                 CALL DBDSQR( 'U', N, N, M, 0, S, WORK( IE ), VT, LDVT,
     $                        A, LDA, DUM, 1, WORK( IWORK ), INFO )
*
              END IF
*
           ELSE IF( WNTUS ) THEN
*
              IF( WNTVN ) THEN
*
                 Path 4 (M much larger than N, JOBU='S', JOBVT='N')
*                N left singular vectors to be computed in U and
*                no right singular vectors to be computed
*
                 IF( LWORK.GE.N*N+MAX( 4*N, BDSPAC ) ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IR = 1
                    IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                      WORK(IR) is LDA by N
*
                       LDWRKR = LDA
                    ELSE
*
*                      WORK(IR) is N by N
*
                       LDWRKR = N
                    END IF
                    ITAU = IR + LDWRKR*N
                    IWORK = ITAU + N
*
*                   Compute A=Q*R
*                   (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                    CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy R to WORK(IR), zeroing out below it
*
```

```
                 CALL DLACPY( 'U', N, N, A, LDA, WORK( IR ),
     $                        LDWRKR )
                 CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                        WORK( IR+1 ), LDWRKR )
*
*                Generate Q in A
*                (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                 CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = ITAU
                 ITAUQ = IE + N
                 ITAUP = ITAUQ + N
                 IWORK = ITAUP + N
*
*                Bidiagonalize R in WORK(IR)
*                (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                 CALL DGEBRD( N, N, WORK( IR ), LDWRKR, S,
     $                        WORK( IE ), WORK( ITAUQ ),
     $                        WORK( ITAUP ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate left vectors bidiagonalizing R in WORK(IR)
*                (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                 CALL DORGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                        WORK( ITAUQ ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
                 IWORK = IE + N
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of R in WORK(IR)
*                (Workspace: need N*N+BDSPAC)
*
                 CALL DBDSQR( 'U', N, 0, N, 0, S, WORK( IE ), DUM,
     $                        1, WORK( IR ), LDWRKR, DUM, 1,
     $                        WORK( IWORK ), INFO )
*
*                Multiply Q in A by left singular vectors of R in
*                WORK(IR), storing result in U
*                (Workspace: need N*N)
*
                 CALL DGEMM( 'N', 'N', M, N, N, ONE, A, LDA,
     $                       WORK( IR ), LDWRKR, ZERO, U, LDU )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
```

```
                 ITAU = 1
                 IWORK = ITAU + N
*
*                Compute A=Q*R, copying result to U
*                (Workspace: need 2*N, prefer N+N*NB)
*
                 CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                Generate Q in U
*                (Workspace: need 2*N, prefer N+N*NB)
*
                 CALL DORGQR( M, N, N, U, LDU, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = ITAU
                 ITAUQ = IE + N
                 ITAUP = ITAUQ + N
                 IWORK = ITAUP + N
*
*                Zero out below R in A
*
                 CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ),
     $                        LDA )
*
*                Bidiagonalize R in A
*                (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                 CALL DGEBRD( N, N, A, LDA, S, WORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Multiply Q in U by left vectors bidiagonalizing R
*                (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                 CALL DORMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                        WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
                 IWORK = IE + N
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of A in U
*                (Workspace: need BDSPAC)
*
                 CALL DBDSQR( 'U', N, 0, M, 0, S, WORK( IE ), DUM,
     $                        1, U, LDU, DUM, 1, WORK( IWORK ),
     $                        INFO )
*
               END IF
*
```

```
               ELSE IF( WNTVO ) THEN
*
*                 Path 5 (M much larger than N, JOBU='S', JOBVT='O')
*                 N left singular vectors to be computed in U and
*                 N right singular vectors to be overwritten on A
*
                  IF( LWORK.GE.2*N*N+MAX( 4*N, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+2*LDA*N ) THEN
*
*                       WORK(IU) is LDA by N and WORK(IR) is LDA by N
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*N
                        LDWRKR = LDA
                     ELSE IF( LWORK.GE.WRKBL+( LDA+N )*N ) THEN
*
*                       WORK(IU) is LDA by N and WORK(IR) is N by N
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*N
                        LDWRKR = N
                     ELSE
*
*                       WORK(IU) is N by N and WORK(IR) is N by N
*
                        LDWRKU = N
                        IR = IU + LDWRKU*N
                        LDWRKR = N
                     END IF
                     ITAU = IR + LDWRKR*N
                     IWORK = ITAU + N
*
*                    Compute A=Q*R
*                    (Workspace: need 2*N*N+2*N, prefer 2*N*N+N+N*NB)
*
                     CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy R to WORK(IU), zeroing out below it
*
                     CALL DLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                            LDWRKU )
                     CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                            WORK( IU+1 ), LDWRKU )
*
*                    Generate Q in A
```

```
*                    (Workspace: need 2*N*N+2*N, prefer 2*N*N+N+N*NB)
*
                     CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Bidiagonalize R in WORK(IU), copying result to
*                    WORK(IR)
*                    (Workspace: need 2*N*N+4*N,
*                                prefer 2*N*N+3*N+2*N*NB)
*
                     CALL DGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                            WORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'U', N, N, WORK( IU ), LDWRKU,
     $                            WORK( IR ), LDWRKR )
*
*                    Generate left bidiagonalizing vectors in WORK(IU)
*                    (Workspace: need 2*N*N+4*N, prefer 2*N*N+3*N+N*NB)
*
                     CALL DORGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                            WORK( ITAUQ ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in WORK(IR)
*                    (Workspace: need 2*N*N+4*N-1,
*                                prefer 2*N*N+3*N+(N-1)*NB)
*
                     CALL DORGBR( 'P', N, N, N, WORK( IR ), LDWRKR,
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IU) and computing
*                    right singular vectors of R in WORK(IR)
*                    (Workspace: need 2*N*N+BDSPAC)
*
                     CALL DBDSQR( 'U', N, N, N, 0, S, WORK( IE ),
     $                            WORK( IR ), LDWRKR, WORK( IU ),
     $                            LDWRKU, DUM, 1, WORK( IWORK ), INFO )
*
*                    Multiply Q in A by left singular vectors of R in
*                    WORK(IU), storing result in U
*                    (Workspace: need N*N)
*
```

```
                 CALL DGEMM( 'N', 'N', M, N, N, ONE, A, LDA,
     $                       WORK( IU ), LDWRKU, ZERO, U, LDU )
*
*                Copy right singular vectors of R to A
*                (Workspace: need N*N)
*
                 CALL DLACPY( 'F', N, N, WORK( IR ), LDWRKR, A,
     $                        LDA )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + N
*
*                Compute A=Q*R, copying result to U
*                (Workspace: need 2*N, prefer N+N*NB)
*
                 CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                Generate Q in U
*                (Workspace: need 2*N, prefer N+N*NB)
*
                 CALL DORGQR( M, N, N, U, LDU, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = ITAU
                 ITAUQ = IE + N
                 ITAUP = ITAUQ + N
                 IWORK = ITAUP + N
*
*                Zero out below R in A
*
                 CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ),
     $                        LDA )
*
*                Bidiagonalize R in A
*                (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                 CALL DGEBRD( N, N, A, LDA, S, WORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Multiply Q in U by left vectors bidiagonalizing R
*                (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                 CALL DORMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                        WORK( ITAUQ ), U, LDU, WORK( IWORK ),
```

```
     $                          LWORK-IWORK+1, IERR )
*
*                    Generate right vectors bidiagonalizing R in A
*                    (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                     CALL DORGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in A
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', N, N, M, 0, S, WORK( IE ), A,
     $                           LDA, U, LDU, DUM, 1, WORK( IWORK ),
     $                           INFO )
*
                  END IF
*
               ELSE IF( WNTVAS ) THEN
*
*                  Path 6 (M much larger than N, JOBU='S', JOBVT='S'
*                          or 'A')
*                  N left singular vectors to be computed in U and
*                  N right singular vectors to be computed in VT
*
                  IF( LWORK.GE.N*N+MAX( 4*N, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                       WORK(IU) is LDA by N
*
                        LDWRKU = LDA
                     ELSE
*
*                       WORK(IU) is N by N
*
                        LDWRKU = N
                     END IF
                     ITAU = IU + LDWRKU*N
                     IWORK = ITAU + N
*
*                    Compute A=Q*R
*                    (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                     CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
```

```
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy R to WORK(IU), zeroing out below it
*
                     CALL DLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                           LDWRKU )
                     CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                           WORK( IU+1 ), LDWRKU )
*
*                    Generate Q in A
*                    (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                     CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Bidiagonalize R in WORK(IU), copying result to VT
*                    (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                     CALL DGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'U', N, N, WORK( IU ), LDWRKU, VT,
     $                           LDVT )
*
*                    Generate left bidiagonalizing vectors in WORK(IU)
*                    (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                     CALL DORGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in VT
*                    (Workspace: need N*N+4*N-1,
*                                prefer N*N+3*N+(N-1)*NB)
*
                     CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IU) and computing
*                    right singular vectors of R in VT
*                    (Workspace: need N*N+BDSPAC)
*
                     CALL DBDSQR( 'U', N, N, N, 0, S, WORK( IE ), VT,
```

```
     $                          LDVT, WORK( IU ), LDWRKU, DUM, 1,
     $                          WORK( IWORK ), INFO )
*
*                   Multiply Q in A by left singular vectors of R in
*                   WORK(IU), storing result in U
*                   (Workspace: need N*N)
*
                    CALL DGEMM( 'N', 'N', M, N, N, ONE, A, LDA,
     $                         WORK( IU ), LDWRKU, ZERO, U, LDU )
*
                 ELSE
*
*                   Insufficient workspace for a fast algorithm
*
                    ITAU = 1
                    IWORK = ITAU + N
*
*                   Compute A=Q*R, copying result to U
*                   (Workspace: need 2*N, prefer N+N*NB)
*
                    CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                    CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                   Generate Q in U
*                   (Workspace: need 2*N, prefer N+N*NB)
*
                    CALL DORGQR( M, N, N, U, LDU, WORK( ITAU ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy R to VT, zeroing out below it
*
                    CALL DLACPY( 'U', N, N, A, LDA, VT, LDVT )
                    IF( N.GT.1 )
     $                 CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                              VT( 2, 1 ), LDVT )
                    IE = ITAU
                    ITAUQ = IE + N
                    ITAUP = ITAUQ + N
                    IWORK = ITAUP + N
*
*                   Bidiagonalize R in VT
*                   (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                    CALL DGEBRD( N, N, VT, LDVT, S, WORK( IE ),
     $                          WORK( ITAUQ ), WORK( ITAUP ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Multiply Q in U by left bidiagonalizing vectors
*                   in VT
```

```
*                    (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                     CALL DORMBR( 'Q', 'R', 'N', M, N, N, VT, LDVT,
     $                            WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in VT
*                    (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                     CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', N, N, M, 0, S, WORK( IE ), VT,
     $                            LDVT, U, LDU, DUM, 1, WORK( IWORK ),
     $                            INFO )
*
                  END IF
*
               END IF
*
            ELSE IF( WNTUA ) THEN
*
               IF( WNTVN ) THEN
*
                  Path 7 (M much larger than N, JOBU='A', JOBVT='N')
*                 M left singular vectors to be computed in U and
*                 no right singular vectors to be computed
*
                  IF( LWORK.GE.N*N+MAX( N+M, 4*N, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IR = 1
                     IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                       WORK(IR) is LDA by N
*
                        LDWRKR = LDA
                     ELSE
*
*                       WORK(IR) is N by N
*
                        LDWRKR = N
                     END IF
```

```
                 ITAU = IR + LDWRKR*N
                 IWORK = ITAU + N
*
*                Compute A=Q*R, copying result to U
*                (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                 CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                Copy R to WORK(IR), zeroing out below it
*
                 CALL DLACPY( 'U', N, N, A, LDA, WORK( IR ),
     $                        LDWRKR )
                 CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                        WORK( IR+1 ), LDWRKR )
*
*                Generate Q in U
*                (Workspace: need N*N+N+M, prefer N*N+N+M*NB)
*
                 CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = ITAU
                 ITAUQ = IE + N
                 ITAUP = ITAUQ + N
                 IWORK = ITAUP + N
*
*                Bidiagonalize R in WORK(IR)
*                (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                 CALL DGEBRD( N, N, WORK( IR ), LDWRKR, S,
     $                        WORK( IE ), WORK( ITAUQ ),
     $                        WORK( ITAUP ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate left bidiagonalizing vectors in WORK(IR)
*                (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                 CALL DORGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                        WORK( ITAUQ ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
                 IWORK = IE + N
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of R in WORK(IR)
*                (Workspace: need N*N+BDSPAC)
*
                 CALL DBDSQR( 'U', N, 0, N, 0, S, WORK( IE ), DUM,
     $                        1, WORK( IR ), LDWRKR, DUM, 1,
     $                        WORK( IWORK ), INFO )
```

```
*
*                    Multiply Q in U by left singular vectors of R in
*                    WORK(IR), storing result in A
*                    (Workspace: need N*N)
*
                     CALL DGEMM( 'N', 'N', M, N, N, ONE, U, LDU,
     $                           WORK( IR ), LDWRKR, ZERO, A, LDA )
*
*                    Copy left singular vectors of A from A to U
*
                     CALL DLACPY( 'F', M, N, A, LDA, U, LDU )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (Workspace: need 2*N, prefer N+N*NB)
*
                     CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                    Generate Q in U
*                    (Workspace: need N+M, prefer N+M*NB)
*
                     CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Zero out below R in A
*
                     CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ),
     $                            LDA )
*
*                    Bidiagonalize R in A
*                    (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                     CALL DGEBRD( N, N, A, LDA, S, WORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply Q in U by left bidiagonalizing vectors
*                    in A
```

```
*                    (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                     CALL DORMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                            WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', N, 0, M, 0, S, WORK( IE ), DUM,
     $                            1, U, LDU, DUM, 1, WORK( IWORK ),
     $                            INFO )
*
                  END IF
*
               ELSE IF( WNTVO ) THEN
*
                  Path 8 (M much larger than N, JOBU='A', JOBVT='O')
*                 M left singular vectors to be computed in U and
*                 N right singular vectors to be overwritten on A
*
                  IF( LWORK.GE.2*N*N+MAX( N+M, 4*N, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+2*LDA*N ) THEN
*
*                       WORK(IU) is LDA by N and WORK(IR) is LDA by N
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*N
                        LDWRKR = LDA
                     ELSE IF( LWORK.GE.WRKBL+( LDA+N )*N ) THEN
*
*                       WORK(IU) is LDA by N and WORK(IR) is N by N
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*N
                        LDWRKR = N
                     ELSE
*
*                       WORK(IU) is N by N and WORK(IR) is N by N
*
                        LDWRKU = N
                        IR = IU + LDWRKU*N
                        LDWRKR = N
                     END IF
```

```
                    ITAU = IR + LDWRKR*N
                    IWORK = ITAU + N
*
*                   Compute A=Q*R, copying result to U
*                   (Workspace: need 2*N*N+2*N, prefer 2*N*N+N+N*NB)
*
                    CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                   Generate Q in U
*                   (Workspace: need 2*N*N+N+M, prefer 2*N*N+N+M*NB)
*
                    CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy R to WORK(IU), zeroing out below it
*
                    CALL DLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                           LDWRKU )
                    CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                           WORK( IU+1 ), LDWRKU )
                    IE = ITAU
                    ITAUQ = IE + N
                    ITAUP = ITAUQ + N
                    IWORK = ITAUP + N
*
*                   Bidiagonalize R in WORK(IU), copying result to
*                   WORK(IR)
*                   (Workspace: need 2*N*N+4*N,
*                               prefer 2*N*N+3*N+2*N*NB)
*
                    CALL DGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    CALL DLACPY( 'U', N, N, WORK( IU ), LDWRKU,
     $                           WORK( IR ), LDWRKR )
*
*                   Generate left bidiagonalizing vectors in WORK(IU)
*                   (Workspace: need 2*N*N+4*N, prefer 2*N*N+3*N+N*NB)
*
                    CALL DORGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate right bidiagonalizing vectors in WORK(IR)
*                   (Workspace: need 2*N*N+4*N-1,
*                               prefer 2*N*N+3*N+(N-1)*NB)
*
```

```
                 CALL DORGBR( 'P', N, N, N, WORK( IR ), LDWRKR,
     $                        WORK( ITAUP ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
                 IWORK = IE + N
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of R in WORK(IU) and computing
*                right singular vectors of R in WORK(IR)
*                (Workspace: need 2*N*N+BDSPAC)
*
                 CALL DBDSQR( 'U', N, N, N, 0, S, WORK( IE ),
     $                        WORK( IR ), LDWRKR, WORK( IU ),
     $                        LDWRKU, DUM, 1, WORK( IWORK ), INFO )
*
*                Multiply Q in U by left singular vectors of R in
*                WORK(IU), storing result in A
*                (Workspace: need N*N)
*
                 CALL DGEMM( 'N', 'N', M, N, N, ONE, U, LDU,
     $                       WORK( IU ), LDWRKU, ZERO, A, LDA )
*
*                Copy left singular vectors of A from A to U
*
                 CALL DLACPY( 'F', M, N, A, LDA, U, LDU )
*
*                Copy right singular vectors of R from WORK(IR) to A
*
                 CALL DLACPY( 'F', N, N, WORK( IR ), LDWRKR, A,
     $                        LDA )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + N
*
*                Compute A=Q*R, copying result to U
*                (Workspace: need 2*N, prefer N+N*NB)
*
                 CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                Generate Q in U
*                (Workspace: need N+M, prefer N+M*NB)
*
                 CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = ITAU
```

```
                    ITAUQ = IE + N
                    ITAUP = ITAUQ + N
                    IWORK = ITAUP + N
*
*                   Zero out below R in A
*
                    CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ),
     $                           LDA )
*
*                   Bidiagonalize R in A
*                   (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                    CALL DGEBRD( N, N, A, LDA, S, WORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Multiply Q in U by left bidiagonalizing vectors
*                   in A
*                   (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                    CALL DORMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                           WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate right bidiagonalizing vectors in A
*                   (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                    CALL DORGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IWORK = IE + N
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of A in U and computing right
*                   singular vectors of A in A
*                   (Workspace: need BDSPAC)
*
                    CALL DBDSQR( 'U', N, N, M, 0, S, WORK( IE ), A,
     $                           LDA, U, LDU, DUM, 1, WORK( IWORK ),
     $                           INFO )
*
                 END IF
*
              ELSE IF( WNTVAS ) THEN
*
*                Path 9 (M much larger than N, JOBU='A', JOBVT='S'
*                        or 'A')
*                M left singular vectors to be computed in U and
*                N right singular vectors to be computed in VT
*
                 IF( LWORK.GE.N*N+MAX( N+M, 4*N, BDSPAC ) ) THEN
```

```
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                       WORK(IU) is LDA by N
*
                        LDWRKU = LDA
                     ELSE
*
*                       WORK(IU) is N by N
*
                        LDWRKU = N
                     END IF
                     ITAU = IU + LDWRKU*N
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                     CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                    Generate Q in U
*                    (Workspace: need N*N+N+M, prefer N*N+N+M*NB)
*
                     CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy R to WORK(IU), zeroing out below it
*
                     CALL DLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                            LDWRKU )
                     CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                            WORK( IU+1 ), LDWRKU )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Bidiagonalize R in WORK(IU), copying result to VT
*                    (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                     CALL DGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                            WORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'U', N, N, WORK( IU ), LDWRKU, VT,
```

```
     $                          LDVT )
*
*                    Generate left bidiagonalizing vectors in WORK(IU)
*                    (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                     CALL DORGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in VT
*                    (Workspace: need N*N+4*N-1,
*                                prefer N*N+3*N+(N-1)*NB)
*
                     CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IU) and computing
*                    right singular vectors of R in VT
*                    (Workspace: need N*N+BDSPAC)
*
                     CALL DBDSQR( 'U', N, N, N, 0, S, WORK( IE ), VT,
     $                           LDVT, WORK( IU ), LDWRKU, DUM, 1,
     $                           WORK( IWORK ), INFO )
*
*                    Multiply Q in U by left singular vectors of R in
*                    WORK(IU), storing result in A
*                    (Workspace: need N*N)
*
                     CALL DGEMM( 'N', 'N', M, N, N, ONE, U, LDU,
     $                          WORK( IU ), LDWRKU, ZERO, A, LDA )
*
*                    Copy left singular vectors of A from A to U
*
                     CALL DLACPY( 'F', M, N, A, LDA, U, LDU )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (Workspace: need 2*N, prefer N+N*NB)
*
                     CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
```

```
*
*                    Generate Q in U
*                    (Workspace: need N+M, prefer N+M*NB)
*
                     CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy R from A to VT, zeroing out below it
*
                     CALL DLACPY( 'U', N, N, A, LDA, VT, LDVT )
                     IF( N.GT.1 )
     $                  CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                               VT( 2, 1 ), LDVT )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Bidiagonalize R in VT
*                    (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                     CALL DGEBRD( N, N, VT, LDVT, S, WORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply Q in U by left bidiagonalizing vectors
*                    in VT
*                    (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                     CALL DORMBR( 'Q', 'R', 'N', M, N, N, VT, LDVT,
     $                            WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in VT
*                    (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                     CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', N, N, M, 0, S, WORK( IE ), VT,
     $                            LDVT, U, LDU, DUM, 1, WORK( IWORK ),
     $                            INFO )
*
                  END IF
```

```
*
               END IF
*
            END IF
*
         ELSE
*
*           M .LT. MNTHR
*
*           Path 10 (M at least N, but not much larger)
*           Reduce to bidiagonal form without QR decomposition
*
            IE = 1
            ITAUQ = IE + N
            ITAUP = ITAUQ + N
            IWORK = ITAUP + N
*
*           Bidiagonalize A
*           (Workspace: need 3*N+M, prefer 3*N+(M+N)*NB)
*
            CALL DGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                   WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                   IERR )
            IF( WNTUAS ) THEN
*
*              If left singular vectors desired in U, copy result to U
*              and generate left bidiagonalizing vectors in U
*              (Workspace: need 3*N+NCU, prefer 3*N+NCU*NB)
*
               CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
               IF( WNTUS )
     $            NCU = N
               IF( WNTUA )
     $            NCU = M
               CALL DORGBR( 'Q', M, NCU, N, U, LDU, WORK( ITAUQ ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IF( WNTVAS ) THEN
*
*              If right singular vectors desired in VT, copy result to
*              VT and generate right bidiagonalizing vectors in VT
*              (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
               CALL DLACPY( 'U', N, N, A, LDA, VT, LDVT )
               CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IF( WNTUO ) THEN
*
*              If left singular vectors desired in A, generate left
```

```
*             bidiagonalizing vectors in A
*             (Workspace: need 4*N, prefer 3*N+N*NB)
*
              CALL DORGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
           IF( WNTVO ) THEN
*
*             If right singular vectors desired in A, generate right
*             bidiagonalizing vectors in A
*             (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
              CALL DORGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
           IWORK = IE + N
           IF( WNTUAS .OR. WNTUO )
     $        NRU = M
           IF( WNTUN )
     $        NRU = 0
           IF( WNTVAS .OR. WNTVO )
     $        NCVT = N
           IF( WNTVN )
     $        NCVT = 0
           IF( ( .NOT.WNTUO ) .AND. ( .NOT.WNTVO ) ) THEN
*
*             Perform bidiagonal QR iteration, if desired, computing
*             left singular vectors in U and computing right singular
*             vectors in VT
*             (Workspace: need BDSPAC)
*
              CALL DBDSQR( 'U', N, NCVT, NRU, 0, S, WORK( IE ), VT,
     $                     LDVT, U, LDU, DUM, 1, WORK( IWORK ), INFO )
           ELSE IF( ( .NOT.WNTUO ) .AND. WNTVO ) THEN
*
*             Perform bidiagonal QR iteration, if desired, computing
*             left singular vectors in U and computing right singular
*             vectors in A
*             (Workspace: need BDSPAC)
*
              CALL DBDSQR( 'U', N, NCVT, NRU, 0, S, WORK( IE ), A, LDA,
     $                     U, LDU, DUM, 1, WORK( IWORK ), INFO )
           ELSE
*
*             Perform bidiagonal QR iteration, if desired, computing
*             left singular vectors in A and computing right singular
*             vectors in VT
*             (Workspace: need BDSPAC)
*
              CALL DBDSQR( 'U', N, NCVT, NRU, 0, S, WORK( IE ), VT,
```

```
     $                     LDVT, A, LDA, DUM, 1, WORK( IWORK ), INFO )
            END IF
*
         END IF
*
      ELSE
*
         A has more columns than rows. If A has sufficiently more
         columns than rows, first reduce using the LQ decomposition (if
         sufficient workspace available)
*
         IF( N.GE.MNTHR ) THEN
*
            IF( WNTVN ) THEN
*
               Path 1t(N much larger than M, JOBVT='N')
               No right singular vectors to be computed
*
               ITAU = 1
               IWORK = ITAU + M
*
               Compute A=L*Q
               (Workspace: need 2*M, prefer M+M*NB)
*
               CALL DGELQF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
     $                      LWORK-IWORK+1, IERR )
*
               Zero out above L
*
               CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ), LDA )
               IE = 1
               ITAUQ = IE + M
               ITAUP = ITAUQ + M
               IWORK = ITAUP + M
*
               Bidiagonalize L in A
               (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
               CALL DGEBRD( M, M, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                      IERR )
               IF( WNTUO .OR. WNTUAS ) THEN
*
                  If left singular vectors desired, generate Q
                  (Workspace: need 4*M, prefer 3*M+M*NB)
*
                  CALL DORGBR( 'Q', M, M, M, A, LDA, WORK( ITAUQ ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
               END IF
               IWORK = IE + M
```

```
               NRU = 0
               IF( WNTUO .OR. WNTUAS )
     $            NRU = M
*
*              Perform bidiagonal QR iteration, computing left singular
*              vectors of A in A if desired
*              (Workspace: need BDSPAC)
*
               CALL DBDSQR( 'U', M, 0, NRU, 0, S, WORK( IE ), DUM, 1, A,
     $                      LDA, DUM, 1, WORK( IWORK ), INFO )
*
*              If left singular vectors desired in U, copy them there
*
               IF( WNTUAS )
     $            CALL DLACPY( 'F', M, M, A, LDA, U, LDU )
*
            ELSE IF( WNTVO .AND. WNTUN ) THEN
*
*              Path 2t(N much larger than M, JOBU='N', JOBVT='O')
*              M right singular vectors to be overwritten on A and
*              no left singular vectors to be computed
*
               IF( LWORK.GE.M*M+MAX( 4*M, BDSPAC ) ) THEN
*
*                Sufficient workspace for a fast algorithm
*
                 IR = 1
                 IF( LWORK.GE.MAX( WRKBL, LDA*N+M )+LDA*M ) THEN
*
*                  WORK(IU) is LDA by N and WORK(IR) is LDA by M
*
                    LDWRKU = LDA
                    CHUNK = N
                    LDWRKR = LDA
                 ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N+M )+M*M ) THEN
*
*                  WORK(IU) is LDA by N and WORK(IR) is M by M
*
                    LDWRKU = LDA
                    CHUNK = N
                    LDWRKR = M
                 ELSE
*
*                  WORK(IU) is M by CHUNK and WORK(IR) is M by M
*
                    LDWRKU = M
                    CHUNK = ( LWORK-M*M-M ) / M
                    LDWRKR = M
                 END IF
                 ITAU = IR + LDWRKR*M
```

```
                  IWORK = ITAU + M
*
*                 Compute A=L*Q
*                 (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                  CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Copy L to WORK(IR) and zero out above it
*
                  CALL DLACPY( 'L', M, M, A, LDA, WORK( IR ), LDWRKR )
                  CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                         WORK( IR+LDWRKR ), LDWRKR )
*
*                 Generate Q in A
*                 (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                  CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IE = ITAU
                  ITAUQ = IE + M
                  ITAUP = ITAUQ + M
                  IWORK = ITAUP + M
*
*                 Bidiagonalize L in WORK(IR)
*                 (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                  CALL DGEBRD( M, M, WORK( IR ), LDWRKR, S, WORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Generate right vectors bidiagonalizing L
*                 (Workspace: need M*M+4*M-1, prefer M*M+3*M+(M-1)*NB)
*
                  CALL DORGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                         WORK( ITAUP ), WORK( IWORK ),
     $                         LWORK-IWORK+1, IERR )
                  IWORK = IE + M
*
*                 Perform bidiagonal QR iteration, computing right
*                 singular vectors of L in WORK(IR)
*                 (Workspace: need M*M+BDSPAC)
*
                  CALL DBDSQR( 'U', M, M, 0, 0, S, WORK( IE ),
     $                         WORK( IR ), LDWRKR, DUM, 1, DUM, 1,
     $                         WORK( IWORK ), INFO )
                  IU = IE + M
*
*                 Multiply right singular vectors of L in WORK(IR) by Q
*                 in A, storing result in WORK(IU) and copying to A
```

```
*                 (Workspace: need M*M+2*M, prefer M*M+M*N+M)
*
                  DO 30 I = 1, N, CHUNK
                     BLK = MIN( N-I+1, CHUNK )
                     CALL DGEMM( 'N', 'N', M, BLK, M, ONE, WORK( IR ),
     $                           LDWRKR, A( 1, I ), LDA, ZERO,
     $                           WORK( IU ), LDWRKU )
                     CALL DLACPY( 'F', M, BLK, WORK( IU ), LDWRKU,
     $                            A( 1, I ), LDA )
   30             CONTINUE
*
               ELSE
*
*                 Insufficient workspace for a fast algorithm
*
                  IE = 1
                  ITAUQ = IE + M
                  ITAUP = ITAUQ + M
                  IWORK = ITAUP + M
*
*                 Bidiagonalize A
*                 (Workspace: need 3*M+N, prefer 3*M+(M+N)*NB)
*
                  CALL DGEBRD( M, N, A, LDA, S, WORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Generate right vectors bidiagonalizing A
*                 (Workspace: need 4*M, prefer 3*M+M*NB)
*
                  CALL DORGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IWORK = IE + M
*
*                 Perform bidiagonal QR iteration, computing right
*                 singular vectors of A in A
*                 (Workspace: need BDSPAC)
*
                  CALL DBDSQR( 'L', M, N, 0, 0, S, WORK( IE ), A, LDA,
     $                         DUM, 1, DUM, 1, WORK( IWORK ), INFO )
*
               END IF
*
            ELSE IF( WNTVO .AND. WNTUAS ) THEN
*
*              Path 3t(N much larger than M, JOBU='S' or 'A', JOBVT='O')
*              M right singular vectors to be overwritten on A and
*              M left singular vectors to be computed in U
*
               IF( LWORK.GE.M*M+MAX( 4*M, BDSPAC ) ) THEN
```

```
*
*                 Sufficient workspace for a fast algorithm
*
                  IR = 1
                  IF( LWORK.GE.MAX( WRKBL, LDA*N+M )+LDA*M ) THEN
*
*                    WORK(IU) is LDA by N and WORK(IR) is LDA by M
*
                     LDWRKU = LDA
                     CHUNK = N
                     LDWRKR = LDA
                  ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N+M )+M*M ) THEN
*
*                    WORK(IU) is LDA by N and WORK(IR) is M by M
*
                     LDWRKU = LDA
                     CHUNK = N
                     LDWRKR = M
                  ELSE
*
*                    WORK(IU) is M by CHUNK and WORK(IR) is M by M
*
                     LDWRKU = M
                     CHUNK = ( LWORK-M*M-M ) / M
                     LDWRKR = M
                  END IF
                  ITAU = IR + LDWRKR*M
                  IWORK = ITAU + M
*
*                 Compute A=L*Q
*                 (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                  CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Copy L to U, zeroing about above it
*
                  CALL DLACPY( 'L', M, M, A, LDA, U, LDU )
                  CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, U( 1, 2 ),
     $                         LDU )
*
*                 Generate Q in A
*                 (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                  CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IE = ITAU
                  ITAUQ = IE + M
                  ITAUP = ITAUQ + M
                  IWORK = ITAUP + M
```

```
*
*                 Bidiagonalize L in U, copying result to WORK(IR)
*                 (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                  CALL DGEBRD( M, M, U, LDU, S, WORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  CALL DLACPY( 'U', M, M, U, LDU, WORK( IR ), LDWRKR )
*
*                 Generate right vectors bidiagonalizing L in WORK(IR)
*                 (Workspace: need M*M+4*M-1, prefer M*M+3*M+(M-1)*NB)
*
                  CALL DORGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                         WORK( ITAUP ), WORK( IWORK ),
     $                         LWORK-IWORK+1, IERR )
*
*                 Generate left vectors bidiagonalizing L in U
*                 (Workspace: need M*M+4*M, prefer M*M+3*M+M*NB)
*
                  CALL DORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IWORK = IE + M
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of L in U, and computing right
*                 singular vectors of L in WORK(IR)
*                 (Workspace: need M*M+BDSPAC)
*
                  CALL DBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                         WORK( IR ), LDWRKR, U, LDU, DUM, 1,
     $                         WORK( IWORK ), INFO )
                  IU = IE + M
*
*                 Multiply right singular vectors of L in WORK(IR) by Q
*                 in A, storing result in WORK(IU) and copying to A
*                 (Workspace: need M*M+2*M, prefer M*M+M*N+M))
*
                  DO 40 I = 1, N, CHUNK
                     BLK = MIN( N-I+1, CHUNK )
                     CALL DGEMM( 'N', 'N', M, BLK, M, ONE, WORK( IR ),
     $                           LDWRKR, A( 1, I ), LDA, ZERO,
     $                           WORK( IU ), LDWRKU )
                     CALL DLACPY( 'F', M, BLK, WORK( IU ), LDWRKU,
     $                            A( 1, I ), LDA )
   40             CONTINUE
*
               ELSE
*
*                 Insufficient workspace for a fast algorithm
*
```

```
                 ITAU = 1
                 IWORK = ITAU + M
*
*                Compute A=L*Q
*                (Workspace: need 2*M, prefer M+M*NB)
*
                 CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy L to U, zeroing out above it
*
                 CALL DLACPY( 'L', M, M, A, LDA, U, LDU )
                 CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, U( 1, 2 ),
     $                        LDU )
*
*                Generate Q in A
*                (Workspace: need 2*M, prefer M+M*NB)
*
                 CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = ITAU
                 ITAUQ = IE + M
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Bidiagonalize L in U
*                (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                 CALL DGEBRD( M, M, U, LDU, S, WORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Multiply right vectors bidiagonalizing L by Q in A
*                (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                 CALL DORMBR( 'P', 'L', 'T', M, N, M, U, LDU,
     $                        WORK( ITAUP ), A, LDA, WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate left vectors bidiagonalizing L in U
*                (Workspace: need 4*M, prefer 3*M+M*NB)
*
                 CALL DORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of A in U and computing right
*                singular vectors of A in A
*                (Workspace: need BDSPAC)
```

```
*
                 CALL DBDSQR( 'U', M, N, M, 0, S, WORK( IE ), A, LDA,
     $                        U, LDU, DUM, 1, WORK( IWORK ), INFO )
*
              END IF
*
           ELSE IF( WNTVS ) THEN
*
              IF( WNTUN ) THEN
*
*                Path 4t(N much larger than M, JOBU='N', JOBVT='S')
*                M right singular vectors to be computed in VT and
*                no left singular vectors to be computed
*
                 IF( LWORK.GE.M*M+MAX( 4*M, BDSPAC ) ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IR = 1
                    IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                      WORK(IR) is LDA by M
*
                       LDWRKR = LDA
                    ELSE
*
*                      WORK(IR) is M by M
*
                       LDWRKR = M
                    END IF
                    ITAU = IR + LDWRKR*M
                    IWORK = ITAU + M
*
*                   Compute A=L*Q
*                   (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                    CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy L to WORK(IR), zeroing out above it
*
                    CALL DLACPY( 'L', M, M, A, LDA, WORK( IR ),
     $                           LDWRKR )
                    CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                           WORK( IR+LDWRKR ), LDWRKR )
*
*                   Generate Q in A
*                   (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                    CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
```

```
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                      IE = ITAU
                      ITAUQ = IE + M
                      ITAUP = ITAUQ + M
                      IWORK = ITAUP + M
*
*                     Bidiagonalize L in WORK(IR)
*                     (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                      CALL DGEBRD( M, M, WORK( IR ), LDWRKR, S,
     $                          WORK( IE ), WORK( ITAUQ ),
     $                          WORK( ITAUP ), WORK( IWORK ),
     $                          LWORK-IWORK+1, IERR )
*
*                     Generate right vectors bidiagonalizing L in
*                     WORK(IR)
*                     (Workspace: need M*M+4*M, prefer M*M+3*M+(M-1)*NB)
*
                      CALL DORGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                          WORK( ITAUP ), WORK( IWORK ),
     $                          LWORK-IWORK+1, IERR )
                      IWORK = IE + M
*
*                     Perform bidiagonal QR iteration, computing right
*                     singular vectors of L in WORK(IR)
*                     (Workspace: need M*M+BDSPAC)
*
                      CALL DBDSQR( 'U', M, M, 0, 0, S, WORK( IE ),
     $                          WORK( IR ), LDWRKR, DUM, 1, DUM, 1,
     $                          WORK( IWORK ), INFO )
*
*                     Multiply right singular vectors of L in WORK(IR) by
*                     Q in A, storing result in VT
*                     (Workspace: need M*M)
*
                      CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IR ),
     $                         LDWRKR, A, LDA, ZERO, VT, LDVT )
*
                   ELSE
*
*                     Insufficient workspace for a fast algorithm
*
                      ITAU = 1
                      IWORK = ITAU + M
*
*                     Compute A=L*Q
*                     (Workspace: need 2*M, prefer M+M*NB)
*
                      CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
```

```
*
*                    Copy result to VT
*
                     CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL DORGLQ( M, N, M, VT, LDVT, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Zero out above L in A
*
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ),
     $                            LDA )
*
*                    Bidiagonalize L in A
*                    (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                     CALL DGEBRD( M, M, A, LDA, S, WORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply right vectors bidiagonalizing L by Q in VT
*                    (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                     CALL DORMBR( 'P', 'L', 'T', M, N, M, A, LDA,
     $                            WORK( ITAUP ), VT, LDVT,
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing right
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', M, N, 0, 0, S, WORK( IE ), VT,
     $                            LDVT, DUM, 1, DUM, 1, WORK( IWORK ),
     $                            INFO )
*
                  END IF
*
               ELSE IF( WNTUO ) THEN
*
*                  Path 5t(N much larger than M, JOBU='O', JOBVT='S')
*                  M right singular vectors to be computed in VT and
*                  M left singular vectors to be overwritten on A
```

```
*
                  IF( LWORK.GE.2*M*M+MAX( 4*M, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+2*LDA*M ) THEN
*
*                       WORK(IU) is LDA by M and WORK(IR) is LDA by M
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*M
                        LDWRKR = LDA
                     ELSE IF( LWORK.GE.WRKBL+( LDA+M )*M ) THEN
*
*                       WORK(IU) is LDA by M and WORK(IR) is M by M
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*M
                        LDWRKR = M
                     ELSE
*
*                       WORK(IU) is M by M and WORK(IR) is M by M
*
                        LDWRKU = M
                        IR = IU + LDWRKU*M
                        LDWRKR = M
                     END IF
                     ITAU = IR + LDWRKR*M
                     IWORK = ITAU + M
*
*                    Compute A=L*Q
*                    (Workspace: need 2*M*M+2*M, prefer 2*M*M+M+M*NB)
*
                     CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to WORK(IU), zeroing out below it
*
                     CALL DLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                            LDWRKU )
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                            WORK( IU+LDWRKU ), LDWRKU )
*
*                    Generate Q in A
*                    (Workspace: need 2*M*M+2*M, prefer 2*M*M+M+M*NB)
*
                     CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
```

```
                 ITAUQ = IE + M
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Bidiagonalize L in WORK(IU), copying result to
*                WORK(IR)
*                (Workspace: need 2*M*M+4*M,
*                            prefer 2*M*M+3*M+2*M*NB)
*
                 CALL DGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                        WORK( IE ), WORK( ITAUQ ),
     $                        WORK( ITAUP ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
                 CALL DLACPY( 'L', M, M, WORK( IU ), LDWRKU,
     $                        WORK( IR ), LDWRKR )
*
*                Generate right bidiagonalizing vectors in WORK(IU)
*                (Workspace: need 2*M*M+4*M-1,
*                            prefer 2*M*M+3*M+(M-1)*NB)
*
                 CALL DORGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                        WORK( ITAUP ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate left bidiagonalizing vectors in WORK(IR)
*                (Workspace: need 2*M*M+4*M, prefer 2*M*M+3*M+M*NB)
*
                 CALL DORGBR( 'Q', M, M, M, WORK( IR ), LDWRKR,
     $                        WORK( ITAUQ ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
                 IWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of L in WORK(IR) and computing
*                right singular vectors of L in WORK(IU)
*                (Workspace: need 2*M*M+BDSPAC)
*
                 CALL DBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                        WORK( IU ), LDWRKU, WORK( IR ),
     $                        LDWRKR, DUM, 1, WORK( IWORK ), INFO )
*
*                Multiply right singular vectors of L in WORK(IU) by
*                Q in A, storing result in VT
*                (Workspace: need M*M)
*
                 CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IU ),
     $                       LDWRKU, A, LDA, ZERO, VT, LDVT )
*
*                Copy left singular vectors of L to A
*                (Workspace: need M*M)
```

```
*
                 CALL DLACPY( 'F', M, M, WORK( IR ), LDWRKR, A,
     $                        LDA )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + M
*
*                Compute A=L*Q, copying result to VT
*                (Workspace: need 2*M, prefer M+M*NB)
*
                 CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                Generate Q in VT
*                (Workspace: need 2*M, prefer M+M*NB)
*
                 CALL DORGLQ( M, N, M, VT, LDVT, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = ITAU
                 ITAUQ = IE + M
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Zero out above L in A
*
                 CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ),
     $                        LDA )
*
*                Bidiagonalize L in A
*                (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                 CALL DGEBRD( M, M, A, LDA, S, WORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Multiply right vectors bidiagonalizing L by Q in VT
*                (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                 CALL DORMBR( 'P', 'L', 'T', M, N, M, A, LDA,
     $                        WORK( ITAUP ), VT, LDVT,
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Generate left bidiagonalizing vectors of L in A
*                (Workspace: need 4*M, prefer 3*M+M*NB)
*
```

```
                 CALL DORGBR( 'Q', M, M, M, A, LDA, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IWORK = IE + M
*
*                Perform bidiagonal QR iteration, compute left
*                singular vectors of A in A and compute right
*                singular vectors of A in VT
*                (Workspace: need BDSPAC)
*
                 CALL DBDSQR( 'U', M, N, M, 0, S, WORK( IE ), VT,
     $                        LDVT, A, LDA, DUM, 1, WORK( IWORK ),
     $                        INFO )
*
              END IF
*
           ELSE IF( WNTUAS ) THEN
*
              Path 6t(N much larger than M, JOBU='S' or 'A',
*                     JOBVT='S')
*             M right singular vectors to be computed in VT and
*             M left singular vectors to be computed in U
*
              IF( LWORK.GE.M*M+MAX( 4*M, BDSPAC ) ) THEN
*
*                Sufficient workspace for a fast algorithm
*
                 IU = 1
                 IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                   WORK(IU) is LDA by N
*
                    LDWRKU = LDA
                 ELSE
*
*                   WORK(IU) is LDA by M
*
                    LDWRKU = M
                 END IF
                 ITAU = IU + LDWRKU*M
                 IWORK = ITAU + M
*
*                Compute A=L*Q
*                (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                 CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy L to WORK(IU), zeroing out above it
*
                 CALL DLACPY( 'L', M, M, A, LDA, WORK( IU ),
```

```
     $                          LDWRKU )
                      CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                          WORK( IU+LDWRKU ), LDWRKU )
*
*                     Generate Q in A
*                     (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                      CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                      IE = ITAU
                      ITAUQ = IE + M
                      ITAUP = ITAUQ + M
                      IWORK = ITAUP + M
*
*                     Bidiagonalize L in WORK(IU), copying result to U
*                     (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                      CALL DGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                          WORK( IE ), WORK( ITAUQ ),
     $                          WORK( ITAUP ), WORK( IWORK ),
     $                          LWORK-IWORK+1, IERR )
                      CALL DLACPY( 'L', M, M, WORK( IU ), LDWRKU, U,
     $                          LDU )
*
*                     Generate right bidiagonalizing vectors in WORK(IU)
*                     (Workspace: need M*M+4*M-1,
*                                 prefer M*M+3*M+(M-1)*NB)
*
                      CALL DORGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                          WORK( ITAUP ), WORK( IWORK ),
     $                          LWORK-IWORK+1, IERR )
*
*                     Generate left bidiagonalizing vectors in U
*                     (Workspace: need M*M+4*M, prefer M*M+3*M+M*NB)
*
                      CALL DORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                      IWORK = IE + M
*
*                     Perform bidiagonal QR iteration, computing left
*                     singular vectors of L in U and computing right
*                     singular vectors of L in WORK(IU)
*                     (Workspace: need M*M+BDSPAC)
*
                      CALL DBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                          WORK( IU ), LDWRKU, U, LDU, DUM, 1,
     $                          WORK( IWORK ), INFO )
*
*                     Multiply right singular vectors of L in WORK(IU) by
*                     Q in A, storing result in VT
```

```
*                    (Workspace: need M*M)
*
                     CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IU ),
     $                           LDWRKU, A, LDA, ZERO, VT, LDVT )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL DORGLQ( M, N, M, VT, LDVT, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to U, zeroing out above it
*
                     CALL DLACPY( 'L', M, M, A, LDA, U, LDU )
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, U( 1, 2 ),
     $                            LDU )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Bidiagonalize L in U
*                    (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                     CALL DGEBRD( M, M, U, LDU, S, WORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply right bidiagonalizing vectors in U by Q
*                    in VT
*                    (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                     CALL DORMBR( 'P', 'L', 'T', M, N, M, U, LDU,
     $                            WORK( ITAUP ), VT, LDVT,
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
```

```
*                    Generate left bidiagonalizing vectors in U
*                    (Workspace: need 4*M, prefer 3*M+M*NB)
*
                     CALL DORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', M, N, M, 0, S, WORK( IE ), VT,
     $                            LDVT, U, LDU, DUM, 1, WORK( IWORK ),
     $                            INFO )
*
                  END IF
*
               END IF
*
            ELSE IF( WNTVA ) THEN
*
               IF( WNTUN ) THEN
*
*                 Path 7t(N much larger than M, JOBU='N', JOBVT='A')
*                 N right singular vectors to be computed in VT and
*                 no left singular vectors to be computed
*
                  IF( LWORK.GE.M*M+MAX( N+M, 4*M, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IR = 1
                     IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                       WORK(IR) is LDA by M
*
                        LDWRKR = LDA
                     ELSE
*
*                       WORK(IR) is M by M
*
                        LDWRKR = M
                     END IF
                     ITAU = IR + LDWRKR*M
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
```

```
                    CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                   Copy L to WORK(IR), zeroing out above it
*
                    CALL DLACPY( 'L', M, M, A, LDA, WORK( IR ),
     $                           LDWRKR )
                    CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                           WORK( IR+LDWRKR ), LDWRKR )
*
*                   Generate Q in VT
*                   (Workspace: need M*M+M+N, prefer M*M+M+N*NB)
*
                    CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = ITAU
                    ITAUQ = IE + M
                    ITAUP = ITAUQ + M
                    IWORK = ITAUP + M
*
*                   Bidiagonalize L in WORK(IR)
*                   (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                    CALL DGEBRD( M, M, WORK( IR ), LDWRKR, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate right bidiagonalizing vectors in WORK(IR)
*                   (Workspace: need M*M+4*M-1,
*                               prefer M*M+3*M+(M-1)*NB)
*
                    CALL DORGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    IWORK = IE + M
*
*                   Perform bidiagonal QR iteration, computing right
*                   singular vectors of L in WORK(IR)
*                   (Workspace: need M*M+BDSPAC)
*
                    CALL DBDSQR( 'U', M, M, 0, 0, S, WORK( IE ),
     $                           WORK( IR ), LDWRKR, DUM, 1, DUM, 1,
     $                           WORK( IWORK ), INFO )
*
*                   Multiply right singular vectors of L in WORK(IR) by
*                   Q in VT, storing result in A
*                   (Workspace: need M*M)
*
```

```
                 CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IR ),
     $                       LDWRKR, VT, LDVT, ZERO, A, LDA )
*
*                Copy right singular vectors of A from A to VT
*
                 CALL DLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + M
*
*                Compute A=L*Q, copying result to VT
*                (Workspace: need 2*M, prefer M+M*NB)
*
                 CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                Generate Q in VT
*                (Workspace: need M+N, prefer M+N*NB)
*
                 CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = ITAU
                 ITAUQ = IE + M
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Zero out above L in A
*
                 CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ),
     $                        LDA )
*
*                Bidiagonalize L in A
*                (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                 CALL DGEBRD( M, M, A, LDA, S, WORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Multiply right bidiagonalizing vectors in A by Q
*                in VT
*                (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                 CALL DORMBR( 'P', 'L', 'T', M, N, M, A, LDA,
     $                        WORK( ITAUP ), VT, LDVT,
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
```

```
                 IWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing right
*                singular vectors of A in VT
*                (Workspace: need BDSPAC)
*
                 CALL DBDSQR( 'U', M, N, 0, 0, S, WORK( IE ), VT,
     $                        LDVT, DUM, 1, DUM, 1, WORK( IWORK ),
     $                        INFO )
*
              END IF
*
           ELSE IF( WNTUO ) THEN
*
              Path 8t(N much larger than M, JOBU='O', JOBVT='A')
              N right singular vectors to be computed in VT and
              M left singular vectors to be overwritten on A
*
              IF( LWORK.GE.2*M*M+MAX( N+M, 4*M, BDSPAC ) ) THEN
*
                 Sufficient workspace for a fast algorithm
*
                 IU = 1
                 IF( LWORK.GE.WRKBL+2*LDA*M ) THEN
*
                    WORK(IU) is LDA by M and WORK(IR) is LDA by M
*
                    LDWRKU = LDA
                    IR = IU + LDWRKU*M
                    LDWRKR = LDA
                 ELSE IF( LWORK.GE.WRKBL+( LDA+M )*M ) THEN
*
                    WORK(IU) is LDA by M and WORK(IR) is M by M
*
                    LDWRKU = LDA
                    IR = IU + LDWRKU*M
                    LDWRKR = M
                 ELSE
*
                    WORK(IU) is M by M and WORK(IR) is M by M
*
                    LDWRKU = M
                    IR = IU + LDWRKU*M
                    LDWRKR = M
                 END IF
                 ITAU = IR + LDWRKR*M
                 IWORK = ITAU + M
*
                 Compute A=L*Q, copying result to VT
*                (Workspace: need 2*M*M+2*M, prefer 2*M*M+M+M*NB)
```

```
*
                    CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                   Generate Q in VT
*                   (Workspace: need 2*M*M+M+N, prefer 2*M*M+M+N*NB)
*
                    CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy L to WORK(IU), zeroing out above it
*
                    CALL DLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                           LDWRKU )
                    CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                           WORK( IU+LDWRKU ), LDWRKU )
                    IE = ITAU
                    ITAUQ = IE + M
                    ITAUP = ITAUQ + M
                    IWORK = ITAUP + M
*
*                   Bidiagonalize L in WORK(IU), copying result to
*                   WORK(IR)
*                   (Workspace: need 2*M*M+4*M,
*                               prefer 2*M*M+3*M+2*M*NB)
*
                    CALL DGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    CALL DLACPY( 'L', M, M, WORK( IU ), LDWRKU,
     $                           WORK( IR ), LDWRKR )
*
*                   Generate right bidiagonalizing vectors in WORK(IU)
*                   (Workspace: need 2*M*M+4*M-1,
*                               prefer 2*M*M+3*M+(M-1)*NB)
*
                    CALL DORGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate left bidiagonalizing vectors in WORK(IR)
*                   (Workspace: need 2*M*M+4*M, prefer 2*M*M+3*M+M*NB)
*
                    CALL DORGBR( 'Q', M, M, M, WORK( IR ), LDWRKR,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    IWORK = IE + M
*
```

```
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of L in WORK(IR) and computing
*                 right singular vectors of L in WORK(IU)
*                 (Workspace: need 2*M*M+BDSPAC)
*
                  CALL DBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                         WORK( IU ), LDWRKU, WORK( IR ),
     $                         LDWRKR, DUM, 1, WORK( IWORK ), INFO )
*
*                 Multiply right singular vectors of L in WORK(IU) by
*                 Q in VT, storing result in A
*                 (Workspace: need M*M)
*
                  CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IU ),
     $                        LDWRKU, VT, LDVT, ZERO, A, LDA )
*
*                 Copy right singular vectors of A from A to VT
*
                  CALL DLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
*                 Copy left singular vectors of A from WORK(IR) to A
*
                  CALL DLACPY( 'F', M, M, WORK( IR ), LDWRKR, A,
     $                         LDA )
*
               ELSE
*
*                 Insufficient workspace for a fast algorithm
*
                  ITAU = 1
                  IWORK = ITAU + M
*
*                 Compute A=L*Q, copying result to VT
*                 (Workspace: need 2*M, prefer M+M*NB)
*
                  CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                 Generate Q in VT
*                 (Workspace: need M+N, prefer M+N*NB)
*
                  CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IE = ITAU
                  ITAUQ = IE + M
                  ITAUP = ITAUQ + M
                  IWORK = ITAUP + M
*
*                 Zero out above L in A
```

```
*
                    CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ),
     $                           LDA )
*
*                   Bidiagonalize L in A
*                   (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                    CALL DGEBRD( M, M, A, LDA, S, WORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Multiply right bidiagonalizing vectors in A by Q
*                   in VT
*                   (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                    CALL DORMBR( 'P', 'L', 'T', M, N, M, A, LDA,
     $                           WORK( ITAUP ), VT, LDVT,
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Generate left bidiagonalizing vectors in A
*                   (Workspace: need 4*M, prefer 3*M+M*NB)
*
                    CALL DORGBR( 'Q', M, M, M, A, LDA, WORK( ITAUQ ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IWORK = IE + M
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of A in A and computing right
*                   singular vectors of A in VT
*                   (Workspace: need BDSPAC)
*
                    CALL DBDSQR( 'U', M, N, M, 0, S, WORK( IE ), VT,
     $                           LDVT, A, LDA, DUM, 1, WORK( IWORK ),
     $                           INFO )
*
                 END IF
*
              ELSE IF( WNTUAS ) THEN
*
*                Path 9t(N much larger than M, JOBU='S' or 'A',
*                        JOBVT='A')
*                N right singular vectors to be computed in VT and
*                M left singular vectors to be computed in U
*
                 IF( LWORK.GE.M*M+MAX( N+M, 4*M, BDSPAC ) ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IU = 1
                    IF( LWORK.GE.WRKBL+LDA*M ) THEN
```

```
*
*                       WORK(IU) is LDA by M
*
                        LDWRKU = LDA
                     ELSE
*
*                       WORK(IU) is M by M
*
                        LDWRKU = M
                     END IF
                     ITAU = IU + LDWRKU*M
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                     CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (Workspace: need M*M+M+N, prefer M*M+M+N*NB)
*
                     CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to WORK(IU), zeroing out above it
*
                     CALL DLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                            LDWRKU )
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                            WORK( IU+LDWRKU ), LDWRKU )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Bidiagonalize L in WORK(IU), copying result to U
*                    (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                     CALL DGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                            WORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'L', M, M, WORK( IU ), LDWRKU, U,
     $                            LDU )
*
*                    Generate right bidiagonalizing vectors in WORK(IU)
*                    (Workspace: need M*M+4*M, prefer M*M+3*M+(M-1)*NB)
*
```

```
                 CALL DORGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                        WORK( ITAUP ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate left bidiagonalizing vectors in U
*                (Workspace: need M*M+4*M, prefer M*M+3*M+M*NB)
*
                 CALL DORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of L in U and computing right
*                singular vectors of L in WORK(IU)
*                (Workspace: need M*M+BDSPAC)
*
                 CALL DBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                        WORK( IU ), LDWRKU, U, LDU, DUM, 1,
     $                        WORK( IWORK ), INFO )
*
*                Multiply right singular vectors of L in WORK(IU) by
*                Q in VT, storing result in A
*                (Workspace: need M*M)
*
                 CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IU ),
     $                       LDWRKU, VT, LDVT, ZERO, A, LDA )
*
*                Copy right singular vectors of A from A to VT
*
                 CALL DLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + M
*
*                Compute A=L*Q, copying result to VT
*                (Workspace: need 2*M, prefer M+M*NB)
*
                 CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                Generate Q in VT
*                (Workspace: need M+N, prefer M+N*NB)
*
                 CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
```

```
*
*                   Copy L to U, zeroing out above it
*
                    CALL DLACPY( 'L', M, M, A, LDA, U, LDU )
                    CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, U( 1, 2 ),
     $                           LDU )
                    IE = ITAU
                    ITAUQ = IE + M
                    ITAUP = ITAUQ + M
                    IWORK = ITAUP + M
*
*                   Bidiagonalize L in U
*                   (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                    CALL DGEBRD( M, M, U, LDU, S, WORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Multiply right bidiagonalizing vectors in U by Q
*                   in VT
*                   (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                    CALL DORMBR( 'P', 'L', 'T', M, N, M, U, LDU,
     $                           WORK( ITAUP ), VT, LDVT,
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Generate left bidiagonalizing vectors in U
*                   (Workspace: need 4*M, prefer 3*M+M*NB)
*
                    CALL DORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IWORK = IE + M
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of A in U and computing right
*                   singular vectors of A in VT
*                   (Workspace: need BDSPAC)
*
                    CALL DBDSQR( 'U', M, N, M, 0, S, WORK( IE ), VT,
     $                           LDVT, U, LDU, DUM, 1, WORK( IWORK ),
     $                           INFO )
*
                 END IF
*
              END IF
*
           END IF
*
        ELSE
*
```

```
*          N .LT. MNTHR
*
*          Path 10t(N greater than M, but not much larger)
*          Reduce to bidiagonal form without LQ decomposition
*
           IE = 1
           ITAUQ = IE + M
           ITAUP = ITAUQ + M
           IWORK = ITAUP + M
*
*          Bidiagonalize A
*          (Workspace: need 3*M+N, prefer 3*M+(M+N)*NB)
*
           CALL DGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                  WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                  IERR )
           IF( WNTUAS ) THEN
*
*             If left singular vectors desired in U, copy result to U
*             and generate left bidiagonalizing vectors in U
*             (Workspace: need 4*M-1, prefer 3*M+(M-1)*NB)
*
              CALL DLACPY( 'L', M, M, A, LDA, U, LDU )
              CALL DORGBR( 'Q', M, M, N, U, LDU, WORK( ITAUQ ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
           IF( WNTVAS ) THEN
*
*             If right singular vectors desired in VT, copy result to
*             VT and generate right bidiagonalizing vectors in VT
*             (Workspace: need 3*M+NRVT, prefer 3*M+NRVT*NB)
*
              CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
              IF( WNTVA )
     $           NRVT = N
              IF( WNTVS )
     $           NRVT = M
              CALL DORGBR( 'P', NRVT, N, M, VT, LDVT, WORK( ITAUP ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
           IF( WNTUO ) THEN
*
*             If left singular vectors desired in A, generate left
*             bidiagonalizing vectors in A
*             (Workspace: need 4*M-1, prefer 3*M+(M-1)*NB)
*
              CALL DORGBR( 'Q', M, M, N, A, LDA, WORK( ITAUQ ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
           IF( WNTVO ) THEN
```

```
*
*             If right singular vectors desired in A, generate right
*             bidiagonalizing vectors in A
*             (Workspace: need 4*M, prefer 3*M+M*NB)
*
              CALL DORGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
           IWORK = IE + M
           IF( WNTUAS .OR. WNTUO )
     $        NRU = M
           IF( WNTUN )
     $        NRU = 0
           IF( WNTVAS .OR. WNTVO )
     $        NCVT = N
           IF( WNTVN )
     $        NCVT = 0
           IF( ( .NOT.WNTUO ) .AND. ( .NOT.WNTVO ) ) THEN
*
*             Perform bidiagonal QR iteration, if desired, computing
*             left singular vectors in U and computing right singular
*             vectors in VT
*             (Workspace: need BDSPAC)
*
              CALL DBDSQR( 'L', M, NCVT, NRU, 0, S, WORK( IE ), VT,
     $                     LDVT, U, LDU, DUM, 1, WORK( IWORK ), INFO )
           ELSE IF( ( .NOT.WNTUO ) .AND. WNTVO ) THEN
*
*             Perform bidiagonal QR iteration, if desired, computing
*             left singular vectors in U and computing right singular
*             vectors in A
*             (Workspace: need BDSPAC)
*
              CALL DBDSQR( 'L', M, NCVT, NRU, 0, S, WORK( IE ), A, LDA,
     $                     U, LDU, DUM, 1, WORK( IWORK ), INFO )
           ELSE
*
*             Perform bidiagonal QR iteration, if desired, computing
*             left singular vectors in A and computing right singular
*             vectors in VT
*             (Workspace: need BDSPAC)
*
              CALL DBDSQR( 'L', M, NCVT, NRU, 0, S, WORK( IE ), VT,
     $                     LDVT, A, LDA, DUM, 1, WORK( IWORK ), INFO )
           END IF
*
        END IF
*
     END IF
*
```

```
*     If DBDSQR failed to converge, copy unconverged superdiagonals
*     to WORK( 2:MINMN )
*
      IF( INFO.NE.0 ) THEN
         IF( IE.GT.2 ) THEN
            DO 50 I = 1, MINMN - 1
               WORK( I+1 ) = WORK( I+IE-1 )
   50       CONTINUE
         END IF
         IF( IE.LT.2 ) THEN
            DO 60 I = MINMN - 1, 1, -1
               WORK( I+1 ) = WORK( I+IE-1 )
   60       CONTINUE
         END IF
      END IF
*
*     Undo scaling if necessary
*
      IF( ISCL.EQ.1 ) THEN
         IF( ANRM.GT.BIGNUM )
     $      CALL DLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( INFO.NE.0 .AND. ANRM.GT.BIGNUM )
     $      CALL DLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN-1, 1, WORK( 2 ),
     $                   MINMN, IERR )
         IF( ANRM.LT.SMLNUM )
     $      CALL DLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( INFO.NE.0 .AND. ANRM.LT.SMLNUM )
     $      CALL DLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN-1, 1, WORK( 2 ),
     $                   MINMN, IERR )
      END IF
*
*     Return optimal workspace in WORK(1)
*
      WORK( 1 ) = MAXWRK
*
      RETURN
*
*     End of DGESVD
*
      END
      SUBROUTINE DGESVD( JOBU, JOBVT, M, N, A, LDA, S, U, LDU, VT, LDVT,
     $                   WORK, LWORK, INFO )
*
*  -- LAPACK driver routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1999
*
```

```
*     .. Scalar Arguments ..
      CHARACTER          JOBU, JOBVT
      INTEGER            INFO, LDA, LDU, LDVT, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), S( * ), U( LDU, * ),
     $                   VT( LDVT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, WNTUA, WNTUAS, WNTUN, WNTUO, WNTUS,
     $                   WNTVA, WNTVAS, WNTVN, WNTVO, WNTVS
      INTEGER            BDSPAC, BLK, CHUNK, I, IE, IERR, IR, ISCL,
     $                   ITAU, ITAUP, ITAUQ, IU, IWORK, LDWRKR, LDWRKU,
     $                   MAXWRK, MINMN, MINWRK, MNTHR, NCU, NCVT, NRU,
     $                   NRVT, WRKBL
      DOUBLE PRECISION   ANRM, BIGNUM, EPS, SMLNUM
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DBDSQR, DGEBRD, DGELQF, DGEMM, DGEQRF, DLACPY,
     $                   DLASCL, DLASET, DORGBR, DORGLQ, DORGQR, DORMBR,
     $                   XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           LSAME, ILAENV, DLAMCH, DLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      MINMN = MIN( M, N )
      MNTHR = ILAENV( 6, 'DGESVD', JOBU // JOBVT, M, N, 0, 0 )
      WNTUA = LSAME( JOBU, 'A' )
      WNTUS = LSAME( JOBU, 'S' )
```

```
      WNTUAS = WNTUA .OR. WNTUS
      WNTUO = LSAME( JOBU, 'O' )
      WNTUN = LSAME( JOBU, 'N' )
      WNTVA = LSAME( JOBVT, 'A' )
      WNTVS = LSAME( JOBVT, 'S' )
      WNTVAS = WNTVA .OR. WNTVS
      WNTVO = LSAME( JOBVT, 'O' )
      WNTVN = LSAME( JOBVT, 'N' )
      MINWRK = 1
      LQUERY = ( LWORK.EQ.-1 )
*
      IF( .NOT.( WNTUA .OR. WNTUS .OR. WNTUO .OR. WNTUN ) ) THEN
         INFO = -1
      ELSE IF( .NOT.( WNTVA .OR. WNTVS .OR. WNTVO .OR. WNTVN ) .OR.
     $         ( WNTVO .AND. WNTUO ) ) THEN
         INFO = -2
      ELSE IF( M.LT.0 ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -6
      ELSE IF( LDU.LT.1 .OR. ( WNTUAS .AND. LDU.LT.M ) ) THEN
         INFO = -9
      ELSE IF( LDVT.LT.1 .OR. ( WNTVA .AND. LDVT.LT.N ) .OR.
     $         ( WNTVS .AND. LDVT.LT.MINMN ) ) THEN
         INFO = -11
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       NB refers to the optimal block size for the immediately
*       following subroutine, as returned by ILAENV.)
*
      IF( INFO.EQ.0 .AND. ( LWORK.GE.1 .OR. LQUERY ) .AND. M.GT.0 .AND.
     $    N.GT.0 ) THEN
         IF( M.GE.N ) THEN
*
*           Compute space needed for DBDSQR
*
            BDSPAC = 5*N
            IF( M.GE.MNTHR ) THEN
               IF( WNTUN ) THEN
*
*                 Path 1 (M much larger than N, JOBU='N')
*
                  MAXWRK = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1,
     $                     -1 )
```

```
                 MAXWRK = MAX( MAXWRK, 3*N+2*N*
     $                   ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                 IF( WNTVO .OR. WNTVAS )
     $              MAXWRK = MAX( MAXWRK, 3*N+( N-1 )*
     $                         ILAENV( 1, 'DORGBR', 'P', N, N, N, -1 ) )
                 MAXWRK = MAX( MAXWRK, BDSPAC )
                 MINWRK = MAX( 4*N, BDSPAC )
                 MAXWRK = MAX( MAXWRK, MINWRK )
              ELSE IF( WNTUO .AND. WNTVN ) THEN
*
*                Path 2 (M much larger than N, JOBU='O', JOBVT='N')
*
                 WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, N+N*ILAENV( 1, 'DORGQR', ' ', M,
     $                   N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+2*N*
     $                   ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORGBR', 'Q', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = MAX( N*N+WRKBL, N*N+M*N+N )
                 MINWRK = MAX( 3*N+M, BDSPAC )
                 MAXWRK = MAX( MAXWRK, MINWRK )
              ELSE IF( WNTUO .AND. WNTVAS ) THEN
*
*                Path 3 (M much larger than N, JOBU='O', JOBVT='S' or
*                'A')
*
                 WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, N+N*ILAENV( 1, 'DORGQR', ' ', M,
     $                   N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+2*N*
     $                   ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORGBR', 'Q', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+( N-1 )*
     $                   ILAENV( 1, 'DORGBR', 'P', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = MAX( N*N+WRKBL, N*N+M*N+N )
                 MINWRK = MAX( 3*N+M, BDSPAC )
                 MAXWRK = MAX( MAXWRK, MINWRK )
              ELSE IF( WNTUS .AND. WNTVN ) THEN
*
*                Path 4 (M much larger than N, JOBU='S', JOBVT='N')
*
                 WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, N+N*ILAENV( 1, 'DORGQR', ' ', M,
     $                   N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+2*N*
     $                   ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
```

```
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORGBR', 'Q', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
                 MAXWRK = MAX( MAXWRK, MINWRK )
              ELSE IF( WNTUS .AND. WNTVO ) THEN
*
*                Path 5 (M much larger than N, JOBU='S', JOBVT='O')
*
                 WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, N+N*ILAENV( 1, 'DORGQR', ' ', M,
     $                   N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+2*N*
     $                   ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORGBR', 'Q', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+( N-1 )*
     $                   ILAENV( 1, 'DORGBR', 'P', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = 2*N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
                 MAXWRK = MAX( MAXWRK, MINWRK )
              ELSE IF( WNTUS .AND. WNTVAS ) THEN
*
*                Path 6 (M much larger than N, JOBU='S', JOBVT='S' or
*                'A')
*
                 WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, N+N*ILAENV( 1, 'DORGQR', ' ', M,
     $                   N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+2*N*
     $                   ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORGBR', 'Q', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+( N-1 )*
     $                   ILAENV( 1, 'DORGBR', 'P', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
                 MAXWRK = MAX( MAXWRK, MINWRK )
              ELSE IF( WNTUA .AND. WNTVN ) THEN
*
*                Path 7 (M much larger than N, JOBU='A', JOBVT='N')
*
                 WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, N+M*ILAENV( 1, 'DORGQR', ' ', M,
     $                   M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+2*N*
     $                   ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
```

```
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORGBR', 'Q', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
                 MAXWRK = MAX( MAXWRK, MINWRK )
              ELSE IF( WNTUA .AND. WNTVO ) THEN
*
*                Path 8 (M much larger than N, JOBU='A', JOBVT='O')
*
                 WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, N+M*ILAENV( 1, 'DORGQR', ' ', M,
     $                   M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+2*N*
     $                   ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORGBR', 'Q', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+( N-1 )*
     $                   ILAENV( 1, 'DORGBR', 'P', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = 2*N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
                 MAXWRK = MAX( MAXWRK, MINWRK )
              ELSE IF( WNTUA .AND. WNTVAS ) THEN
*
*                Path 9 (M much larger than N, JOBU='A', JOBVT='S' or
*                'A')
*
                 WRKBL = N + N*ILAENV( 1, 'DGEQRF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, N+M*ILAENV( 1, 'DORGQR', ' ', M,
     $                   M, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+2*N*
     $                   ILAENV( 1, 'DGEBRD', ' ', N, N, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+N*
     $                   ILAENV( 1, 'DORGBR', 'Q', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, 3*N+( N-1 )*
     $                   ILAENV( 1, 'DORGBR', 'P', N, N, N, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = N*N + WRKBL
                 MINWRK = MAX( 3*N+M, BDSPAC )
                 MAXWRK = MAX( MAXWRK, MINWRK )
              END IF
           ELSE
*
*             Path 10 (M at least N, but not much larger)
*
              MAXWRK = 3*N + ( M+N )*ILAENV( 1, 'DGEBRD', ' ', M, N,
     $                 -1, -1 )
              IF( WNTUS .OR. WNTUO )
     $           MAXWRK = MAX( MAXWRK, 3*N+N*
```

```
     $                    ILAENV( 1, 'DORGBR', 'Q', M, N, N, -1 ) )
               IF( WNTUA )
     $            MAXWRK = MAX( MAXWRK, 3*N+M*
     $                    ILAENV( 1, 'DORGBR', 'Q', M, M, N, -1 ) )
               IF( .NOT.WNTVN )
     $            MAXWRK = MAX( MAXWRK, 3*N+( N-1 )*
     $                    ILAENV( 1, 'DORGBR', 'P', N, N, N, -1 ) )
               MAXWRK = MAX( MAXWRK, BDSPAC )
               MINWRK = MAX( 3*N+M, BDSPAC )
               MAXWRK = MAX( MAXWRK, MINWRK )
            END IF
         ELSE
*
*           Compute space needed for DBDSQR
*
            BDSPAC = 5*M
            IF( N.GE.MNTHR ) THEN
               IF( WNTVN ) THEN
*
*                Path 1t(N much larger than M, JOBVT='N')
*
                 MAXWRK = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1,
     $                    -1 )
                 MAXWRK = MAX( MAXWRK, 3*M+2*M*
     $                    ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
                 IF( WNTUO .OR. WNTUAS )
     $              MAXWRK = MAX( MAXWRK, 3*M+M*
     $                         ILAENV( 1, 'DORGBR', 'Q', M, M, M, -1 ) )
                 MAXWRK = MAX( MAXWRK, BDSPAC )
                 MINWRK = MAX( 4*M, BDSPAC )
                 MAXWRK = MAX( MAXWRK, MINWRK )
               ELSE IF( WNTVO .AND. WNTUN ) THEN
*
*                Path 2t(N much larger than M, JOBU='N', JOBVT='O')
*
                 WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, M+M*ILAENV( 1, 'DORGLQ', ' ', M,
     $                   N, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+2*M*
     $                   ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+( M-1 )*
     $                   ILAENV( 1, 'DORGBR', 'P', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = MAX( M*M+WRKBL, M*M+M*N+M )
                 MINWRK = MAX( 3*M+N, BDSPAC )
                 MAXWRK = MAX( MAXWRK, MINWRK )
               ELSE IF( WNTVO .AND. WNTUAS ) THEN
*
*                Path 3t(N much larger than M, JOBU='S' or 'A',
*                JOBVT='O')
```

```
*
                 WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, M+M*ILAENV( 1, 'DORGLQ', ' ', M,
     $                   N, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+2*M*
     $                   ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+( M-1 )*
     $                   ILAENV( 1, 'DORGBR', 'P', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORGBR', 'Q', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = MAX( M*M+WRKBL, M*M+M*N+M )
                 MINWRK = MAX( 3*M+N, BDSPAC )
                 MAXWRK = MAX( MAXWRK, MINWRK )
              ELSE IF( WNTVS .AND. WNTUN ) THEN
*
*                Path 4t(N much larger than M, JOBU='N', JOBVT='S')
*
                 WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, M+M*ILAENV( 1, 'DORGLQ', ' ', M,
     $                   N, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+2*M*
     $                   ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+( M-1 )*
     $                   ILAENV( 1, 'DORGBR', 'P', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
                 MAXWRK = MAX( MAXWRK, MINWRK )
              ELSE IF( WNTVS .AND. WNTUO ) THEN
*
*                Path 5t(N much larger than M, JOBU='O', JOBVT='S')
*
                 WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, M+M*ILAENV( 1, 'DORGLQ', ' ', M,
     $                   N, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+2*M*
     $                   ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+( M-1 )*
     $                   ILAENV( 1, 'DORGBR', 'P', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORGBR', 'Q', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = 2*M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
                 MAXWRK = MAX( MAXWRK, MINWRK )
              ELSE IF( WNTVS .AND. WNTUAS ) THEN
*
*                Path 6t(N much larger than M, JOBU='S' or 'A',
*                JOBVT='S')
```

```
*
                 WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, M+M*ILAENV( 1, 'DORGLQ', ' ', M,
     $                   N, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+2*M*
     $                   ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+( M-1 )*
     $                   ILAENV( 1, 'DORGBR', 'P', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORGBR', 'Q', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
                 MAXWRK = MAX( MAXWRK, MINWRK )
              ELSE IF( WNTVA .AND. WNTUN ) THEN
*
*                Path 7t(N much larger than M, JOBU='N', JOBVT='A')
*
                 WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, M+N*ILAENV( 1, 'DORGLQ', ' ', N,
     $                   N, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+2*M*
     $                   ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+( M-1 )*
     $                   ILAENV( 1, 'DORGBR', 'P', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
                 MAXWRK = MAX( MAXWRK, MINWRK )
              ELSE IF( WNTVA .AND. WNTUO ) THEN
*
*                Path 8t(N much larger than M, JOBU='O', JOBVT='A')
*
                 WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, M+N*ILAENV( 1, 'DORGLQ', ' ', N,
     $                   N, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+2*M*
     $                   ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+( M-1 )*
     $                   ILAENV( 1, 'DORGBR', 'P', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORGBR', 'Q', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = 2*M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
                 MAXWRK = MAX( MAXWRK, MINWRK )
              ELSE IF( WNTVA .AND. WNTUAS ) THEN
*
*                Path 9t(N much larger than M, JOBU='S' or 'A',
*                JOBVT='A')
```

```
*
                 WRKBL = M + M*ILAENV( 1, 'DGELQF', ' ', M, N, -1, -1 )
                 WRKBL = MAX( WRKBL, M+N*ILAENV( 1, 'DORGLQ', ' ', N,
     $                   N, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+2*M*
     $                   ILAENV( 1, 'DGEBRD', ' ', M, M, -1, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+( M-1 )*
     $                   ILAENV( 1, 'DORGBR', 'P', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, 3*M+M*
     $                   ILAENV( 1, 'DORGBR', 'Q', M, M, M, -1 ) )
                 WRKBL = MAX( WRKBL, BDSPAC )
                 MAXWRK = M*M + WRKBL
                 MINWRK = MAX( 3*M+N, BDSPAC )
                 MAXWRK = MAX( MAXWRK, MINWRK )
              END IF
           ELSE
*
*             Path 10t(N greater than M, but not much larger)
*
              MAXWRK = 3*M + ( M+N )*ILAENV( 1, 'DGEBRD', ' ', M, N,
     $                 -1, -1 )
              IF( WNTVS .OR. WNTVO )
     $           MAXWRK = MAX( MAXWRK, 3*M+M*
     $                    ILAENV( 1, 'DORGBR', 'P', M, N, M, -1 ) )
              IF( WNTVA )
     $           MAXWRK = MAX( MAXWRK, 3*M+N*
     $                    ILAENV( 1, 'DORGBR', 'P', N, N, M, -1 ) )
              IF( .NOT.WNTUN )
     $           MAXWRK = MAX( MAXWRK, 3*M+( M-1 )*
     $                    ILAENV( 1, 'DORGBR', 'Q', M, M, M, -1 ) )
              MAXWRK = MAX( MAXWRK, BDSPAC )
              MINWRK = MAX( 3*M+N, BDSPAC )
              MAXWRK = MAX( MAXWRK, MINWRK )
           END IF
        END IF
        WORK( 1 ) = MAXWRK
     END IF
*
     IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
        INFO = -13
     END IF
     IF( INFO.NE.0 ) THEN
        CALL XERBLA( 'DGESVD', -INFO )
        RETURN
     ELSE IF( LQUERY ) THEN
        RETURN
     END IF
*
*    Quick return if possible
*
```

```
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         IF( LWORK.GE.1 )
     $      WORK( 1 ) = ONE
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = SQRT( DLAMCH( 'S' ) ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = DLANGE( 'M', M, N, A, LDA, DUM )
      ISCL = 0
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         ISCL = 1
         CALL DLASCL( 'G', 0, 0, ANRM, SMLNUM, M, N, A, LDA, IERR )
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         ISCL = 1
         CALL DLASCL( 'G', 0, 0, ANRM, BIGNUM, M, N, A, LDA, IERR )
      END IF
*
      IF( M.GE.N ) THEN
*
*        A has at least as many rows as columns. If A has sufficiently
*        more rows than columns, first reduce using the QR
*        decomposition (if sufficient workspace available)
*
         IF( M.GE.MNTHR ) THEN
*
            IF( WNTUN ) THEN
*
*              Path 1 (M much larger than N, JOBU='N')
*              No left singular vectors to be computed
*
               ITAU = 1
               IWORK = ITAU + N
*
*              Compute A=Q*R
*              (Workspace: need 2*N, prefer N+N*NB)
*
               CALL DGEQRF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
     $                      LWORK-IWORK+1, IERR )
*
*              Zero out below R
*
               CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ), LDA )
               IE = 1
```

```
            ITAUQ = IE + N
            ITAUP = ITAUQ + N
            IWORK = ITAUP + N
*
*           Bidiagonalize R in A
*           (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
            CALL DGEBRD( N, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                   WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                   IERR )
            NCVT = 0
            IF( WNTVO .OR. WNTVAS ) THEN
*
*              If right singular vectors desired, generate P'.
*              (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
               CALL DORGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
               NCVT = N
            END IF
            IWORK = IE + N
*
*           Perform bidiagonal QR iteration, computing right
*           singular vectors of A in A if desired
*           (Workspace: need BDSPAC)
*
            CALL DBDSQR( 'U', N, NCVT, 0, 0, S, WORK( IE ), A, LDA,
     $                   DUM, 1, DUM, 1, WORK( IWORK ), INFO )
*
*           If right singular vectors desired in VT, copy them there
*
            IF( WNTVAS )
     $         CALL DLACPY( 'F', N, N, A, LDA, VT, LDVT )
*
         ELSE IF( WNTUO .AND. WNTVN ) THEN
*
*           Path 2 (M much larger than N, JOBU='O', JOBVT='N')
*           N left singular vectors to be overwritten on A and
*           no right singular vectors to be computed
*
            IF( LWORK.GE.N*N+MAX( 4*N, BDSPAC ) ) THEN
*
*              Sufficient workspace for a fast algorithm
*
               IR = 1
               IF( LWORK.GE.MAX( WRKBL, LDA*N+N )+LDA*N ) THEN
*
*                 WORK(IU) is LDA by N, WORK(IR) is LDA by N
*
                 LDWRKU = LDA
```

```
                 LDWRKR = LDA
              ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N+N )+N*N ) THEN
*
*                WORK(IU) is LDA by N, WORK(IR) is N by N
*
                 LDWRKU = LDA
                 LDWRKR = N
              ELSE
*
*                WORK(IU) is LDWRKU by N, WORK(IR) is N by N
*
                 LDWRKU = ( LWORK-N*N-N ) / N
                 LDWRKR = N
              END IF
              ITAU = IR + LDWRKR*N
              IWORK = ITAU + N
*
*             Compute A=Q*R
*             (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
              CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*             Copy R to WORK(IR) and zero out below it
*
              CALL DLACPY( 'U', N, N, A, LDA, WORK( IR ), LDWRKR )
              CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, WORK( IR+1 ),
     $                     LDWRKR )
*
*             Generate Q in A
*             (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
              CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
              IE = ITAU
              ITAUQ = IE + N
              ITAUP = ITAUQ + N
              IWORK = ITAUP + N
*
*             Bidiagonalize R in WORK(IR)
*             (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
              CALL DGEBRD( N, N, WORK( IR ), LDWRKR, S, WORK( IE ),
     $                     WORK( ITAUQ ), WORK( ITAUP ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*             Generate left vectors bidiagonalizing R
*             (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
              CALL DORGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
```

```
     $                        WORK( ITAUQ ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
                  IWORK = IE + N
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of R in WORK(IR)
*                 (Workspace: need N*N+BDSPAC)
*
                  CALL DBDSQR( 'U', N, 0, N, 0, S, WORK( IE ), DUM, 1,
     $                        WORK( IR ), LDWRKR, DUM, 1,
     $                        WORK( IWORK ), INFO )
                  IU = IE + N
*
*                 Multiply Q in A by left singular vectors of R in
*                 WORK(IR), storing result in WORK(IU) and copying to A
*                 (Workspace: need N*N+2*N, prefer N*N+M*N+N)
*
                  DO 10 I = 1, M, LDWRKU
                     CHUNK = MIN( M-I+1, LDWRKU )
                     CALL DGEMM( 'N', 'N', CHUNK, N, N, ONE, A( I, 1 ),
     $                           LDA, WORK( IR ), LDWRKR, ZERO,
     $                           WORK( IU ), LDWRKU )
                     CALL DLACPY( 'F', CHUNK, N, WORK( IU ), LDWRKU,
     $                            A( I, 1 ), LDA )
   10             CONTINUE
*
               ELSE
*
*                 Insufficient workspace for a fast algorithm
*
                  IE = 1
                  ITAUQ = IE + N
                  ITAUP = ITAUQ + N
                  IWORK = ITAUP + N
*
*                 Bidiagonalize A
*                 (Workspace: need 3*N+M, prefer 3*N+(M+N)*NB)
*
                  CALL DGEBRD( M, N, A, LDA, S, WORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Generate left vectors bidiagonalizing A
*                 (Workspace: need 4*N, prefer 3*N+N*NB)
*
                  CALL DORGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IWORK = IE + N
*
*                 Perform bidiagonal QR iteration, computing left
```

```
*                 singular vectors of A in A
*                 (Workspace: need BDSPAC)
*
                  CALL DBDSQR( 'U', N, 0, M, 0, S, WORK( IE ), DUM, 1,
     $                         A, LDA, DUM, 1, WORK( IWORK ), INFO )
*
               END IF
*
            ELSE IF( WNTUO .AND. WNTVAS ) THEN
*
               Path 3 (M much larger than N, JOBU='O', JOBVT='S' or 'A')
               N left singular vectors to be overwritten on A and
               N right singular vectors to be computed in VT
*
               IF( LWORK.GE.N*N+MAX( 4*N, BDSPAC ) ) THEN
*
                  Sufficient workspace for a fast algorithm
*
                  IR = 1
                  IF( LWORK.GE.MAX( WRKBL, LDA*N+N )+LDA*N ) THEN
*
                     WORK(IU) is LDA by N and WORK(IR) is LDA by N
*
                     LDWRKU = LDA
                     LDWRKR = LDA
                  ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N+N )+N*N ) THEN
*
                     WORK(IU) is LDA by N and WORK(IR) is N by N
*
                     LDWRKU = LDA
                     LDWRKR = N
                  ELSE
*
                     WORK(IU) is LDWRKU by N and WORK(IR) is N by N
*
                     LDWRKU = ( LWORK-N*N-N ) / N
                     LDWRKR = N
                  END IF
                  ITAU = IR + LDWRKR*N
                  IWORK = ITAU + N
*
                  Compute A=Q*R
*                 (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                  CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Copy R to VT, zeroing out below it
*
                  CALL DLACPY( 'U', N, N, A, LDA, VT, LDVT )
```

```
                CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, VT( 2, 1 ),
     $                       LDVT )
*
*               Generate Q in A
*               (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                       WORK( IWORK ), LWORK-IWORK+1, IERR )
                IE = ITAU
                ITAUQ = IE + N
                ITAUP = ITAUQ + N
                IWORK = ITAUP + N
*
*               Bidiagonalize R in VT, copying result to WORK(IR)
*               (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                CALL DGEBRD( N, N, VT, LDVT, S, WORK( IE ),
     $                       WORK( ITAUQ ), WORK( ITAUP ),
     $                       WORK( IWORK ), LWORK-IWORK+1, IERR )
                CALL DLACPY( 'L', N, N, VT, LDVT, WORK( IR ), LDWRKR )
*
*               Generate left vectors bidiagonalizing R in WORK(IR)
*               (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                CALL DORGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                       WORK( ITAUQ ), WORK( IWORK ),
     $                       LWORK-IWORK+1, IERR )
*
*               Generate right vectors bidiagonalizing R in VT
*               (Workspace: need N*N+4*N-1, prefer N*N+3*N+(N-1)*NB)
*
                CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                       WORK( IWORK ), LWORK-IWORK+1, IERR )
                IWORK = IE + N
*
*               Perform bidiagonal QR iteration, computing left
*               singular vectors of R in WORK(IR) and computing right
*               singular vectors of R in VT
*               (Workspace: need N*N+BDSPAC)
*
                CALL DBDSQR( 'U', N, N, N, 0, S, WORK( IE ), VT, LDVT,
     $                       WORK( IR ), LDWRKR, DUM, 1,
     $                       WORK( IWORK ), INFO )
                IU = IE + N
*
*               Multiply Q in A by left singular vectors of R in
*               WORK(IR), storing result in WORK(IU) and copying to A
*               (Workspace: need N*N+2*N, prefer N*N+M*N+N)
*
                DO 20 I = 1, M, LDWRKU
```

```
                 CHUNK = MIN( M-I+1, LDWRKU )
                 CALL DGEMM( 'N', 'N', CHUNK, N, N, ONE, A( I, 1 ),
     $                       LDA, WORK( IR ), LDWRKR, ZERO,
     $                       WORK( IU ), LDWRKU )
                 CALL DLACPY( 'F', CHUNK, N, WORK( IU ), LDWRKU,
     $                        A( I, 1 ), LDA )
   20         CONTINUE
*
            ELSE
*
*              Insufficient workspace for a fast algorithm
*
               ITAU = 1
               IWORK = ITAU + N
*
*              Compute A=Q*R
*              (Workspace: need 2*N, prefer N+N*NB)
*
               CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*              Copy R to VT, zeroing out below it
*
               CALL DLACPY( 'U', N, N, A, LDA, VT, LDVT )
               CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, VT( 2, 1 ),
     $                      LDVT )
*
*              Generate Q in A
*              (Workspace: need 2*N, prefer N+N*NB)
*
               CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
               IE = ITAU
               ITAUQ = IE + N
               ITAUP = ITAUQ + N
               IWORK = ITAUP + N
*
*              Bidiagonalize R in VT
*              (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
               CALL DGEBRD( N, N, VT, LDVT, S, WORK( IE ),
     $                      WORK( ITAUQ ), WORK( ITAUP ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*              Multiply Q in A by left vectors bidiagonalizing R
*              (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
               CALL DORMBR( 'Q', 'R', 'N', M, N, N, VT, LDVT,
     $                      WORK( ITAUQ ), A, LDA, WORK( IWORK ),
     $                      LWORK-IWORK+1, IERR )
```

```
*
*                Generate right vectors bidiagonalizing R in VT
*                (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                 CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IWORK = IE + N
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of A in A and computing right
*                singular vectors of A in VT
*                (Workspace: need BDSPAC)
*
                 CALL DBDSQR( 'U', N, N, M, 0, S, WORK( IE ), VT, LDVT,
     $                        A, LDA, DUM, 1, WORK( IWORK ), INFO )
*
              END IF
*
           ELSE IF( WNTUS ) THEN
*
              IF( WNTVN ) THEN
*
*                Path 4 (M much larger than N, JOBU='S', JOBVT='N')
*                N left singular vectors to be computed in U and
*                no right singular vectors to be computed
*
                 IF( LWORK.GE.N*N+MAX( 4*N, BDSPAC ) ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IR = 1
                    IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                      WORK(IR) is LDA by N
*
                       LDWRKR = LDA
                    ELSE
*
*                      WORK(IR) is N by N
*
                       LDWRKR = N
                    END IF
                    ITAU = IR + LDWRKR*N
                    IWORK = ITAU + N
*
*                   Compute A=Q*R
*                   (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                    CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
```

```
*
*                   Copy R to WORK(IR), zeroing out below it
*
                    CALL DLACPY( 'U', N, N, A, LDA, WORK( IR ),
     $                           LDWRKR )
                    CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                           WORK( IR+1 ), LDWRKR )
*
*                   Generate Q in A
*                   (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                    CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = ITAU
                    ITAUQ = IE + N
                    ITAUP = ITAUQ + N
                    IWORK = ITAUP + N
*
*                   Bidiagonalize R in WORK(IR)
*                   (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                    CALL DGEBRD( N, N, WORK( IR ), LDWRKR, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate left vectors bidiagonalizing R in WORK(IR)
*                   (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                    CALL DORGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    IWORK = IE + N
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of R in WORK(IR)
*                   (Workspace: need N*N+BDSPAC)
*
                    CALL DBDSQR( 'U', N, 0, N, 0, S, WORK( IE ), DUM,
     $                           1, WORK( IR ), LDWRKR, DUM, 1,
     $                           WORK( IWORK ), INFO )
*
*                   Multiply Q in A by left singular vectors of R in
*                   WORK(IR), storing result in U
*                   (Workspace: need N*N)
*
                    CALL DGEMM( 'N', 'N', M, N, N, ONE, A, LDA,
     $                          WORK( IR ), LDWRKR, ZERO, U, LDU )
*
                 ELSE
```

```
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (Workspace: need 2*N, prefer N+N*NB)
*
                     CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                    Generate Q in U
*                    (Workspace: need 2*N, prefer N+N*NB)
*
                     CALL DORGQR( M, N, N, U, LDU, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Zero out below R in A
*
                     CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ),
     $                            LDA )
*
*                    Bidiagonalize R in A
*                    (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                     CALL DGEBRD( N, N, A, LDA, S, WORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply Q in U by left vectors bidiagonalizing R
*                    (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                     CALL DORMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                            WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', N, 0, M, 0, S, WORK( IE ), DUM,
     $                            1, U, LDU, DUM, 1, WORK( IWORK ),
     $                            INFO )
```

```
*
                 END IF
*
              ELSE IF( WNTVO ) THEN
*
*                Path 5 (M much larger than N, JOBU='S', JOBVT='O')
*                N left singular vectors to be computed in U and
*                N right singular vectors to be overwritten on A
*
                 IF( LWORK.GE.2*N*N+MAX( 4*N, BDSPAC ) ) THEN
*
*                   Sufficient workspace for a fast algorithm
*
                    IU = 1
                    IF( LWORK.GE.WRKBL+2*LDA*N ) THEN
*
*                      WORK(IU) is LDA by N and WORK(IR) is LDA by N
*
                       LDWRKU = LDA
                       IR = IU + LDWRKU*N
                       LDWRKR = LDA
                    ELSE IF( LWORK.GE.WRKBL+( LDA+N )*N ) THEN
*
*                      WORK(IU) is LDA by N and WORK(IR) is N by N
*
                       LDWRKU = LDA
                       IR = IU + LDWRKU*N
                       LDWRKR = N
                    ELSE
*
*                      WORK(IU) is N by N and WORK(IR) is N by N
*
                       LDWRKU = N
                       IR = IU + LDWRKU*N
                       LDWRKR = N
                    END IF
                    ITAU = IR + LDWRKR*N
                    IWORK = ITAU + N
*
*                   Compute A=Q*R
*                   (Workspace: need 2*N*N+2*N, prefer 2*N*N+N+N*NB)
*
                    CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy R to WORK(IU), zeroing out below it
*
                    CALL DLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                           LDWRKU )
                    CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
```

```
     $                          WORK( IU+1 ), LDWRKU )
*
*                    Generate Q in A
*                    (Workspace: need 2*N*N+2*N, prefer 2*N*N+N+N*NB)
*
                     CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Bidiagonalize R in WORK(IU), copying result to
*                    WORK(IR)
*                    (Workspace: need 2*N*N+4*N,
*                                prefer 2*N*N+3*N+2*N*NB)
*
                     CALL DGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'U', N, N, WORK( IU ), LDWRKU,
     $                           WORK( IR ), LDWRKR )
*
*                    Generate left bidiagonalizing vectors in WORK(IU)
*                    (Workspace: need 2*N*N+4*N, prefer 2*N*N+3*N+N*NB)
*
                     CALL DORGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in WORK(IR)
*                    (Workspace: need 2*N*N+4*N-1,
*                                prefer 2*N*N+3*N+(N-1)*NB)
*
                     CALL DORGBR( 'P', N, N, N, WORK( IR ), LDWRKR,
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IU) and computing
*                    right singular vectors of R in WORK(IR)
*                    (Workspace: need 2*N*N+BDSPAC)
*
                     CALL DBDSQR( 'U', N, N, N, 0, S, WORK( IE ),
     $                           WORK( IR ), LDWRKR, WORK( IU ),
     $                           LDWRKU, DUM, 1, WORK( IWORK ), INFO )
*
*                    Multiply Q in A by left singular vectors of R in
```

```
*                    WORK(IU), storing result in U
*                    (Workspace: need N*N)
*
                     CALL DGEMM( 'N', 'N', M, N, N, ONE, A, LDA,
     $                           WORK( IU ), LDWRKU, ZERO, U, LDU )
*
*                    Copy right singular vectors of R to A
*                    (Workspace: need N*N)
*
                     CALL DLACPY( 'F', N, N, WORK( IR ), LDWRKR, A,
     $                            LDA )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (Workspace: need 2*N, prefer N+N*NB)
*
                     CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                    Generate Q in U
*                    (Workspace: need 2*N, prefer N+N*NB)
*
                     CALL DORGQR( M, N, N, U, LDU, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Zero out below R in A
*
                     CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ),
     $                            LDA )
*
*                    Bidiagonalize R in A
*                    (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                     CALL DGEBRD( N, N, A, LDA, S, WORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply Q in U by left vectors bidiagonalizing R
*                    (Workspace: need 3*N+M, prefer 3*N+M*NB)
```

```
*
                 CALL DORMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                        WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate right vectors bidiagonalizing R in A
*                (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                 CALL DORGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IWORK = IE + N
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of A in U and computing right
*                singular vectors of A in A
*                (Workspace: need BDSPAC)
*
                 CALL DBDSQR( 'U', N, N, M, 0, S, WORK( IE ), A,
     $                        LDA, U, LDU, DUM, 1, WORK( IWORK ),
     $                        INFO )
*
              END IF
*
           ELSE IF( WNTVAS ) THEN
*
              Path 6 (M much larger than N, JOBU='S', JOBVT='S'
*                     or 'A')
              N left singular vectors to be computed in U and
              N right singular vectors to be computed in VT
*
              IF( LWORK.GE.N*N+MAX( 4*N, BDSPAC ) ) THEN
*
                 Sufficient workspace for a fast algorithm
*
                 IU = 1
                 IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
                    WORK(IU) is LDA by N
*
                    LDWRKU = LDA
                 ELSE
*
                    WORK(IU) is N by N
*
                    LDWRKU = N
                 END IF
                 ITAU = IU + LDWRKU*N
                 IWORK = ITAU + N
*
*                Compute A=Q*R
```

```
*                    (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                     CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy R to WORK(IU), zeroing out below it
*
                     CALL DLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                            LDWRKU )
                     CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                            WORK( IU+1 ), LDWRKU )
*
*                    Generate Q in A
*                    (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                     CALL DORGQR( M, N, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Bidiagonalize R in WORK(IU), copying result to VT
*                    (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                     CALL DGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                            WORK( IE ), WORK( ITAUQ ),
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'U', N, N, WORK( IU ), LDWRKU, VT,
     $                            LDVT )
*
*                    Generate left bidiagonalizing vectors in WORK(IU)
*                    (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                     CALL DORGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                            WORK( ITAUQ ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in VT
*                    (Workspace: need N*N+4*N-1,
*                                prefer N*N+3*N+(N-1)*NB)
*
                     CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IU) and computing
*                    right singular vectors of R in VT
```

```
*                    (Workspace: need N*N+BDSPAC)
*
                     CALL DBDSQR( 'U', N, N, N, 0, S, WORK( IE ), VT,
     $                            LDVT, WORK( IU ), LDWRKU, DUM, 1,
     $                            WORK( IWORK ), INFO )
*
*                    Multiply Q in A by left singular vectors of R in
*                    WORK(IU), storing result in U
*                    (Workspace: need N*N)
*
                     CALL DGEMM( 'N', 'N', M, N, N, ONE, A, LDA,
     $                           WORK( IU ), LDWRKU, ZERO, U, LDU )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (Workspace: need 2*N, prefer N+N*NB)
*
                     CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                    Generate Q in U
*                    (Workspace: need 2*N, prefer N+N*NB)
*
                     CALL DORGQR( M, N, N, U, LDU, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy R to VT, zeroing out below it
*
                     CALL DLACPY( 'U', N, N, A, LDA, VT, LDVT )
                     CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, VT( 2, 1 ),
     $                            LDVT )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Bidiagonalize R in VT
*                    (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                     CALL DGEBRD( N, N, VT, LDVT, S, WORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
```

```
*                    Multiply Q in U by left bidiagonalizing vectors
*                    in VT
*                    (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                     CALL DORMBR( 'Q', 'R', 'N', M, N, N, VT, LDVT,
     $                            WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in VT
*                    (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                     CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', N, N, M, 0, S, WORK( IE ), VT,
     $                            LDVT, U, LDU, DUM, 1, WORK( IWORK ),
     $                            INFO )
*
                  END IF
*
               END IF
*
            ELSE IF( WNTUA ) THEN
*
               IF( WNTVN ) THEN
*
                  Path 7 (M much larger than N, JOBU='A', JOBVT='N')
*                 M left singular vectors to be computed in U and
*                 no right singular vectors to be computed
*
                  IF( LWORK.GE.N*N+MAX( N+M, 4*N, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IR = 1
                     IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                       WORK(IR) is LDA by N
*
                        LDWRKR = LDA
                     ELSE
*
*                       WORK(IR) is N by N
*
```

```
                 LDWRKR = N
              END IF
              ITAU = IR + LDWRKR*N
              IWORK = ITAU + N
*
*             Compute A=Q*R, copying result to U
*             (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
              CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
              CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*             Copy R to WORK(IR), zeroing out below it
*
              CALL DLACPY( 'U', N, N, A, LDA, WORK( IR ),
     $                     LDWRKR )
              CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                     WORK( IR+1 ), LDWRKR )
*
*             Generate Q in U
*             (Workspace: need N*N+N+M, prefer N*N+N+M*NB)
*
              CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
              IE = ITAU
              ITAUQ = IE + N
              ITAUP = ITAUQ + N
              IWORK = ITAUP + N
*
*             Bidiagonalize R in WORK(IR)
*             (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
              CALL DGEBRD( N, N, WORK( IR ), LDWRKR, S,
     $                     WORK( IE ), WORK( ITAUQ ),
     $                     WORK( ITAUP ), WORK( IWORK ),
     $                     LWORK-IWORK+1, IERR )
*
*             Generate left bidiagonalizing vectors in WORK(IR)
*             (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
              CALL DORGBR( 'Q', N, N, N, WORK( IR ), LDWRKR,
     $                     WORK( ITAUQ ), WORK( IWORK ),
     $                     LWORK-IWORK+1, IERR )
              IWORK = IE + N
*
*             Perform bidiagonal QR iteration, computing left
*             singular vectors of R in WORK(IR)
*             (Workspace: need N*N+BDSPAC)
*
              CALL DBDSQR( 'U', N, 0, N, 0, S, WORK( IE ), DUM,
```

```
     $                          1, WORK( IR ), LDWRKR, DUM, 1,
     $                          WORK( IWORK ), INFO )
*
*                    Multiply Q in U by left singular vectors of R in
*                    WORK(IR), storing result in A
*                    (Workspace: need N*N)
*
                     CALL DGEMM( 'N', 'N', M, N, N, ONE, U, LDU,
     $                         WORK( IR ), LDWRKR, ZERO, A, LDA )
*
*                    Copy left singular vectors of A from A to U
*
                     CALL DLACPY( 'F', M, N, A, LDA, U, LDU )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (Workspace: need 2*N, prefer N+N*NB)
*
                     CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                    Generate Q in U
*                    (Workspace: need N+M, prefer N+M*NB)
*
                     CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + N
                     ITAUP = ITAUQ + N
                     IWORK = ITAUP + N
*
*                    Zero out below R in A
*
                     CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ),
     $                          LDA )
*
*                    Bidiagonalize R in A
*                    (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                     CALL DGEBRD( N, N, A, LDA, S, WORK( IE ),
     $                          WORK( ITAUQ ), WORK( ITAUP ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
```

```
*                    Multiply Q in U by left bidiagonalizing vectors
*                    in A
*                    (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                     CALL DORMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                            WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', N, 0, M, 0, S, WORK( IE ), DUM,
     $                            1, U, LDU, DUM, 1, WORK( IWORK ),
     $                            INFO )
*
                  END IF
*
               ELSE IF( WNTVO ) THEN
*
                  Path 8 (M much larger than N, JOBU='A', JOBVT='O')
*                 M left singular vectors to be computed in U and
*                 N right singular vectors to be overwritten on A
*
                  IF( LWORK.GE.2*N*N+MAX( N+M, 4*N, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+2*LDA*N ) THEN
*
*                       WORK(IU) is LDA by N and WORK(IR) is LDA by N
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*N
                        LDWRKR = LDA
                     ELSE IF( LWORK.GE.WRKBL+( LDA+N )*N ) THEN
*
*                       WORK(IU) is LDA by N and WORK(IR) is N by N
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*N
                        LDWRKR = N
                     ELSE
*
*                       WORK(IU) is N by N and WORK(IR) is N by N
*
                        LDWRKU = N
                        IR = IU + LDWRKU*N
```

```
                      LDWRKR = N
                   END IF
                   ITAU = IR + LDWRKR*N
                   IWORK = ITAU + N
*
*                  Compute A=Q*R, copying result to U
*                  (Workspace: need 2*N*N+2*N, prefer 2*N*N+N+N*NB)
*
                   CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                   CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                  Generate Q in U
*                  (Workspace: need 2*N*N+N+M, prefer 2*N*N+N+M*NB)
*
                   CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                  Copy R to WORK(IU), zeroing out below it
*
                   CALL DLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                           LDWRKU )
                   CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                           WORK( IU+1 ), LDWRKU )
                   IE = ITAU
                   ITAUQ = IE + N
                   ITAUP = ITAUQ + N
                   IWORK = ITAUP + N
*
*                  Bidiagonalize R in WORK(IU), copying result to
*                  WORK(IR)
*                  (Workspace: need 2*N*N+4*N,
*                              prefer 2*N*N+3*N+2*N*NB)
*
                   CALL DGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                   CALL DLACPY( 'U', N, N, WORK( IU ), LDWRKU,
     $                           WORK( IR ), LDWRKR )
*
*                  Generate left bidiagonalizing vectors in WORK(IU)
*                  (Workspace: need 2*N*N+4*N, prefer 2*N*N+3*N+N*NB)
*
                   CALL DORGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                  Generate right bidiagonalizing vectors in WORK(IR)
*                  (Workspace: need 2*N*N+4*N-1,
```

```
*                               prefer 2*N*N+3*N+(N-1)*NB)
*
                     CALL DORGBR( 'P', N, N, N, WORK( IR ), LDWRKR,
     $                            WORK( ITAUP ), WORK( IWORK ),
     $                            LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IU) and computing
*                    right singular vectors of R in WORK(IR)
*                    (Workspace: need 2*N*N+BDSPAC)
*
                     CALL DBDSQR( 'U', N, N, N, 0, S, WORK( IE ),
     $                            WORK( IR ), LDWRKR, WORK( IU ),
     $                            LDWRKU, DUM, 1, WORK( IWORK ), INFO )
*
*                    Multiply Q in U by left singular vectors of R in
*                    WORK(IU), storing result in A
*                    (Workspace: need N*N)
*
                     CALL DGEMM( 'N', 'N', M, N, N, ONE, U, LDU,
     $                           WORK( IU ), LDWRKU, ZERO, A, LDA )
*
*                    Copy left singular vectors of A from A to U
*
                     CALL DLACPY( 'F', M, N, A, LDA, U, LDU )
*
*                    Copy right singular vectors of R from WORK(IR) to A
*
                     CALL DLACPY( 'F', N, N, WORK( IR ), LDWRKR, A,
     $                            LDA )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (Workspace: need 2*N, prefer N+N*NB)
*
                     CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                    Generate Q in U
*                    (Workspace: need N+M, prefer N+M*NB)
*
                     CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
```

```
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                      IE = ITAU
                      ITAUQ = IE + N
                      ITAUP = ITAUQ + N
                      IWORK = ITAUP + N
*
*                     Zero out below R in A
*
                      CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, A( 2, 1 ),
     $                          LDA )
*
*                     Bidiagonalize R in A
*                     (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                      CALL DGEBRD( N, N, A, LDA, S, WORK( IE ),
     $                          WORK( ITAUQ ), WORK( ITAUP ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                     Multiply Q in U by left bidiagonalizing vectors
*                     in A
*                     (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                      CALL DORMBR( 'Q', 'R', 'N', M, N, N, A, LDA,
     $                          WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                          LWORK-IWORK+1, IERR )
*
*                     Generate right bidiagonalizing vectors in A
*                     (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                      CALL DORGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                      IWORK = IE + N
*
*                     Perform bidiagonal QR iteration, computing left
*                     singular vectors of A in U and computing right
*                     singular vectors of A in A
*                     (Workspace: need BDSPAC)
*
                      CALL DBDSQR( 'U', N, N, M, 0, S, WORK( IE ), A,
     $                          LDA, U, LDU, DUM, 1, WORK( IWORK ),
     $                          INFO )
*
                   END IF
*
                ELSE IF( WNTVAS ) THEN
*
*                  Path 9 (M much larger than N, JOBU='A', JOBVT='S'
*                          or 'A')
*                  M left singular vectors to be computed in U and
*                  N right singular vectors to be computed in VT
```

```
*
                IF( LWORK.GE.N*N+MAX( N+M, 4*N, BDSPAC ) ) THEN
*
*                  Sufficient workspace for a fast algorithm
*
                   IU = 1
                   IF( LWORK.GE.WRKBL+LDA*N ) THEN
*
*                     WORK(IU) is LDA by N
*
                      LDWRKU = LDA
                   ELSE
*
*                     WORK(IU) is N by N
*
                      LDWRKU = N
                   END IF
                   ITAU = IU + LDWRKU*N
                   IWORK = ITAU + N
*
*                  Compute A=Q*R, copying result to U
*                  (Workspace: need N*N+2*N, prefer N*N+N+N*NB)
*
                   CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                   CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                  Generate Q in U
*                  (Workspace: need N*N+N+M, prefer N*N+N+M*NB)
*
                   CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                  Copy R to WORK(IU), zeroing out below it
*
                   CALL DLACPY( 'U', N, N, A, LDA, WORK( IU ),
     $                           LDWRKU )
                   CALL DLASET( 'L', N-1, N-1, ZERO, ZERO,
     $                           WORK( IU+1 ), LDWRKU )
                   IE = ITAU
                   ITAUQ = IE + N
                   ITAUP = ITAUQ + N
                   IWORK = ITAUP + N
*
*                  Bidiagonalize R in WORK(IU), copying result to VT
*                  (Workspace: need N*N+4*N, prefer N*N+3*N+2*N*NB)
*
                   CALL DGEBRD( N, N, WORK( IU ), LDWRKU, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
```

```
     $                          LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'U', N, N, WORK( IU ), LDWRKU, VT,
     $                          LDVT )
*
*                    Generate left bidiagonalizing vectors in WORK(IU)
*                    (Workspace: need N*N+4*N, prefer N*N+3*N+N*NB)
*
                     CALL DORGBR( 'Q', N, N, N, WORK( IU ), LDWRKU,
     $                          WORK( ITAUQ ), WORK( IWORK ),
     $                          LWORK-IWORK+1, IERR )
*
*                    Generate right bidiagonalizing vectors in VT
*                    (Workspace: need N*N+4*N-1,
*                                prefer N*N+3*N+(N-1)*NB)
*
                     CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + N
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of R in WORK(IU) and computing
*                    right singular vectors of R in VT
*                    (Workspace: need N*N+BDSPAC)
*
                     CALL DBDSQR( 'U', N, N, N, 0, S, WORK( IE ), VT,
     $                          LDVT, WORK( IU ), LDWRKU, DUM, 1,
     $                          WORK( IWORK ), INFO )
*
*                    Multiply Q in U by left singular vectors of R in
*                    WORK(IU), storing result in A
*                    (Workspace: need N*N)
*
                     CALL DGEMM( 'N', 'N', M, N, N, ONE, U, LDU,
     $                         WORK( IU ), LDWRKU, ZERO, A, LDA )
*
*                    Copy left singular vectors of A from A to U
*
                     CALL DLACPY( 'F', M, N, A, LDA, U, LDU )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + N
*
*                    Compute A=Q*R, copying result to U
*                    (Workspace: need 2*N, prefer N+N*NB)
*
                     CALL DGEQRF( M, N, A, LDA, WORK( ITAU ),
```

```
     $                    WORK( IWORK ), LWORK-IWORK+1, IERR )
                  CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
*
*                 Generate Q in U
*                 (Workspace: need N+M, prefer N+M*NB)
*
                  CALL DORGQR( M, M, N, U, LDU, WORK( ITAU ),
     $                    WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Copy R from A to VT, zeroing out below it
*
                  CALL DLACPY( 'U', N, N, A, LDA, VT, LDVT )
                  CALL DLASET( 'L', N-1, N-1, ZERO, ZERO, VT( 2, 1 ),
     $                    LDVT )
                  IE = ITAU
                  ITAUQ = IE + N
                  ITAUP = ITAUQ + N
                  IWORK = ITAUP + N
*
*                 Bidiagonalize R in VT
*                 (Workspace: need 4*N, prefer 3*N+2*N*NB)
*
                  CALL DGEBRD( N, N, VT, LDVT, S, WORK( IE ),
     $                    WORK( ITAUQ ), WORK( ITAUP ),
     $                    WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Multiply Q in U by left bidiagonalizing vectors
*                 in VT
*                 (Workspace: need 3*N+M, prefer 3*N+M*NB)
*
                  CALL DORMBR( 'Q', 'R', 'N', M, N, N, VT, LDVT,
     $                    WORK( ITAUQ ), U, LDU, WORK( IWORK ),
     $                    LWORK-IWORK+1, IERR )
*
*                 Generate right bidiagonalizing vectors in VT
*                 (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
                  CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                    WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IWORK = IE + N
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of A in U and computing right
*                 singular vectors of A in VT
*                 (Workspace: need BDSPAC)
*
                  CALL DBDSQR( 'U', N, N, M, 0, S, WORK( IE ), VT,
     $                    LDVT, U, LDU, DUM, 1, WORK( IWORK ),
     $                    INFO )
*
```

```
                 END IF
*
              END IF
*
           END IF
*
        ELSE
*
*          M .LT. MNTHR
*
*          Path 10 (M at least N, but not much larger)
*          Reduce to bidiagonal form without QR decomposition
*
           IE = 1
           ITAUQ = IE + N
           ITAUP = ITAUQ + N
           IWORK = ITAUP + N
*
*          Bidiagonalize A
*          (Workspace: need 3*N+M, prefer 3*N+(M+N)*NB)
*
           CALL DGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                  WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                  IERR )
           IF( WNTUAS ) THEN
*
*             If left singular vectors desired in U, copy result to U
*             and generate left bidiagonalizing vectors in U
*             (Workspace: need 3*N+NCU, prefer 3*N+NCU*NB)
*
              CALL DLACPY( 'L', M, N, A, LDA, U, LDU )
              IF( WNTUS )
     $           NCU = N
              IF( WNTUA )
     $           NCU = M
              CALL DORGBR( 'Q', M, NCU, N, U, LDU, WORK( ITAUQ ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
           IF( WNTVAS ) THEN
*
*             If right singular vectors desired in VT, copy result to
*             VT and generate right bidiagonalizing vectors in VT
*             (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
              CALL DLACPY( 'U', N, N, A, LDA, VT, LDVT )
              CALL DORGBR( 'P', N, N, N, VT, LDVT, WORK( ITAUP ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
           IF( WNTUO ) THEN
*
```

```
*             If left singular vectors desired in A, generate left
*             bidiagonalizing vectors in A
*             (Workspace: need 4*N, prefer 3*N+N*NB)
*
              CALL DORGBR( 'Q', M, N, N, A, LDA, WORK( ITAUQ ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
           IF( WNTVO ) THEN
*
*             If right singular vectors desired in A, generate right
*             bidiagonalizing vectors in A
*             (Workspace: need 4*N-1, prefer 3*N+(N-1)*NB)
*
              CALL DORGBR( 'P', N, N, N, A, LDA, WORK( ITAUP ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
           IWORK = IE + N
           IF( WNTUAS .OR. WNTUO )
     $        NRU = M
           IF( WNTUN )
     $        NRU = 0
           IF( WNTVAS .OR. WNTVO )
     $        NCVT = N
           IF( WNTVN )
     $        NCVT = 0
           IF( ( .NOT.WNTUO ) .AND. ( .NOT.WNTVO ) ) THEN
*
*             Perform bidiagonal QR iteration, if desired, computing
*             left singular vectors in U and computing right singular
*             vectors in VT
*             (Workspace: need BDSPAC)
*
              CALL DBDSQR( 'U', N, NCVT, NRU, 0, S, WORK( IE ), VT,
     $                     LDVT, U, LDU, DUM, 1, WORK( IWORK ), INFO )
           ELSE IF( ( .NOT.WNTUO ) .AND. WNTVO ) THEN
*
*             Perform bidiagonal QR iteration, if desired, computing
*             left singular vectors in U and computing right singular
*             vectors in A
*             (Workspace: need BDSPAC)
*
              CALL DBDSQR( 'U', N, NCVT, NRU, 0, S, WORK( IE ), A, LDA,
     $                     U, LDU, DUM, 1, WORK( IWORK ), INFO )
           ELSE
*
*             Perform bidiagonal QR iteration, if desired, computing
*             left singular vectors in A and computing right singular
*             vectors in VT
*             (Workspace: need BDSPAC)
*
```

```
               CALL DBDSQR( 'U', N, NCVT, NRU, 0, S, WORK( IE ), VT,
     $                      LDVT, A, LDA, DUM, 1, WORK( IWORK ), INFO )
            END IF
*
         END IF
*
      ELSE
*
         A has more columns than rows. If A has sufficiently more
         columns than rows, first reduce using the LQ decomposition (if
         sufficient workspace available)
*
         IF( N.GE.MNTHR ) THEN
*
            IF( WNTVN ) THEN
*
               Path 1t(N much larger than M, JOBVT='N')
               No right singular vectors to be computed
*
               ITAU = 1
               IWORK = ITAU + M
*
               Compute A=L*Q
               (Workspace: need 2*M, prefer M+M*NB)
*
               CALL DGELQF( M, N, A, LDA, WORK( ITAU ), WORK( IWORK ),
     $                      LWORK-IWORK+1, IERR )
*
               Zero out above L
*
               CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ), LDA )
               IE = 1
               ITAUQ = IE + M
               ITAUP = ITAUQ + M
               IWORK = ITAUP + M
*
               Bidiagonalize L in A
               (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
               CALL DGEBRD( M, M, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                      WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                      IERR )
               IF( WNTUO .OR. WNTUAS ) THEN
*
                  If left singular vectors desired, generate Q
                  (Workspace: need 4*M, prefer 3*M+M*NB)
*
                  CALL DORGBR( 'Q', M, M, M, A, LDA, WORK( ITAUQ ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
               END IF
```

```
               IWORK = IE + M
               NRU = 0
               IF( WNTUO .OR. WNTUAS )
     $            NRU = M
*
*              Perform bidiagonal QR iteration, computing left singular
*              vectors of A in A if desired
*              (Workspace: need BDSPAC)
*
               CALL DBDSQR( 'U', M, 0, NRU, 0, S, WORK( IE ), DUM, 1, A,
     $                      LDA, DUM, 1, WORK( IWORK ), INFO )
*
*              If left singular vectors desired in U, copy them there
*
               IF( WNTUAS )
     $            CALL DLACPY( 'F', M, M, A, LDA, U, LDU )
*
            ELSE IF( WNTVO .AND. WNTUN ) THEN
*
*              Path 2t(N much larger than M, JOBU='N', JOBVT='O')
*              M right singular vectors to be overwritten on A and
*              no left singular vectors to be computed
*
               IF( LWORK.GE.M*M+MAX( 4*M, BDSPAC ) ) THEN
*
*                Sufficient workspace for a fast algorithm
*
                 IR = 1
                 IF( LWORK.GE.MAX( WRKBL, LDA*N+M )+LDA*M ) THEN
*
*                  WORK(IU) is LDA by N and WORK(IR) is LDA by M
*
                    LDWRKU = LDA
                    CHUNK = N
                    LDWRKR = LDA
                 ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N+M )+M*M ) THEN
*
*                  WORK(IU) is LDA by N and WORK(IR) is M by M
*
                    LDWRKU = LDA
                    CHUNK = N
                    LDWRKR = M
                 ELSE
*
*                  WORK(IU) is M by CHUNK and WORK(IR) is M by M
*
                    LDWRKU = M
                    CHUNK = ( LWORK-M*M-M ) / M
                    LDWRKR = M
                 END IF
```

```
                 ITAU = IR + LDWRKR*M
                 IWORK = ITAU + M
*
*                Compute A=L*Q
*                (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                 CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy L to WORK(IR) and zero out above it
*
                 CALL DLACPY( 'L', M, M, A, LDA, WORK( IR ), LDWRKR )
                 CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                        WORK( IR+LDWRKR ), LDWRKR )
*
*                Generate Q in A
*                (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                 CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = ITAU
                 ITAUQ = IE + M
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Bidiagonalize L in WORK(IR)
*                (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                 CALL DGEBRD( M, M, WORK( IR ), LDWRKR, S, WORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Generate right vectors bidiagonalizing L
*                (Workspace: need M*M+4*M-1, prefer M*M+3*M+(M-1)*NB)
*
                 CALL DORGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                        WORK( ITAUP ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
                 IWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing right
*                singular vectors of L in WORK(IR)
*                (Workspace: need M*M+BDSPAC)
*
                 CALL DBDSQR( 'U', M, M, 0, 0, S, WORK( IE ),
     $                        WORK( IR ), LDWRKR, DUM, 1, DUM, 1,
     $                        WORK( IWORK ), INFO )
                 IU = IE + M
*
*                Multiply right singular vectors of L in WORK(IR) by Q
```

```
*                 in A, storing result in WORK(IU) and copying to A
*                 (Workspace: need M*M+2*M, prefer M*M+M*N+M)
*
                  DO 30 I = 1, N, CHUNK
                     BLK = MIN( N-I+1, CHUNK )
                     CALL DGEMM( 'N', 'N', M, BLK, M, ONE, WORK( IR ),
     $                           LDWRKR, A( 1, I ), LDA, ZERO,
     $                           WORK( IU ), LDWRKU )
                     CALL DLACPY( 'F', M, BLK, WORK( IU ), LDWRKU,
     $                            A( 1, I ), LDA )
   30             CONTINUE
*
               ELSE
*
*                 Insufficient workspace for a fast algorithm
*
                  IE = 1
                  ITAUQ = IE + M
                  ITAUP = ITAUQ + M
                  IWORK = ITAUP + M
*
*                 Bidiagonalize A
*                 (Workspace: need 3*M+N, prefer 3*M+(M+N)*NB)
*
                  CALL DGEBRD( M, N, A, LDA, S, WORK( IE ),
     $                         WORK( ITAUQ ), WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Generate right vectors bidiagonalizing A
*                 (Workspace: need 4*M, prefer 3*M+M*NB)
*
                  CALL DORGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IWORK = IE + M
*
*                 Perform bidiagonal QR iteration, computing right
*                 singular vectors of A in A
*                 (Workspace: need BDSPAC)
*
                  CALL DBDSQR( 'L', M, N, 0, 0, S, WORK( IE ), A, LDA,
     $                         DUM, 1, DUM, 1, WORK( IWORK ), INFO )
*
               END IF
*
            ELSE IF( WNTVO .AND. WNTUAS ) THEN
*
               Path 3t(N much larger than M, JOBU='S' or 'A', JOBVT='O')
*              M right singular vectors to be overwritten on A and
*              M left singular vectors to be computed in U
*
```

```
               IF( LWORK.GE.M*M+MAX( 4*M, BDSPAC ) ) THEN
*
*                 Sufficient workspace for a fast algorithm
*
                  IR = 1
                  IF( LWORK.GE.MAX( WRKBL, LDA*N+M )+LDA*M ) THEN
*
*                    WORK(IU) is LDA by N and WORK(IR) is LDA by M
*
                     LDWRKU = LDA
                     CHUNK = N
                     LDWRKR = LDA
                  ELSE IF( LWORK.GE.MAX( WRKBL, LDA*N+M )+M*M ) THEN
*
*                    WORK(IU) is LDA by N and WORK(IR) is M by M
*
                     LDWRKU = LDA
                     CHUNK = N
                     LDWRKR = M
                  ELSE
*
*                    WORK(IU) is M by CHUNK and WORK(IR) is M by M
*
                     LDWRKU = M
                     CHUNK = ( LWORK-M*M-M ) / M
                     LDWRKR = M
                  END IF
                  ITAU = IR + LDWRKR*M
                  IWORK = ITAU + M
*
*                 Compute A=L*Q
*                 (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                  CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                 Copy L to U, zeroing about above it
*
                  CALL DLACPY( 'L', M, M, A, LDA, U, LDU )
                  CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, U( 1, 2 ),
     $                         LDU )
*
*                 Generate Q in A
*                 (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                  CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                         WORK( IWORK ), LWORK-IWORK+1, IERR )
                  IE = ITAU
                  ITAUQ = IE + M
                  ITAUP = ITAUQ + M
```

```
                 IWORK = ITAUP + M
*
*                Bidiagonalize L in U, copying result to WORK(IR)
*                (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                 CALL DGEBRD( M, M, U, LDU, S, WORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL DLACPY( 'U', M, M, U, LDU, WORK( IR ), LDWRKR )
*
*                Generate right vectors bidiagonalizing L in WORK(IR)
*                (Workspace: need M*M+4*M-1, prefer M*M+3*M+(M-1)*NB)
*
                 CALL DORGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                        WORK( ITAUP ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate left vectors bidiagonalizing L in U
*                (Workspace: need M*M+4*M, prefer M*M+3*M+M*NB)
*
                 CALL DORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of L in U, and computing right
*                singular vectors of L in WORK(IR)
*                (Workspace: need M*M+BDSPAC)
*
                 CALL DBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                        WORK( IR ), LDWRKR, U, LDU, DUM, 1,
     $                        WORK( IWORK ), INFO )
                 IU = IE + M
*
*                Multiply right singular vectors of L in WORK(IR) by Q
*                in A, storing result in WORK(IU) and copying to A
*                (Workspace: need M*M+2*M, prefer M*M+M*N+M))
*
                 DO 40 I = 1, N, CHUNK
                    BLK = MIN( N-I+1, CHUNK )
                    CALL DGEMM( 'N', 'N', M, BLK, M, ONE, WORK( IR ),
     $                          LDWRKR, A( 1, I ), LDA, ZERO,
     $                          WORK( IU ), LDWRKU )
                    CALL DLACPY( 'F', M, BLK, WORK( IU ), LDWRKU,
     $                           A( 1, I ), LDA )
   40            CONTINUE
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
```

```
*
                 ITAU = 1
                 IWORK = ITAU + M
*
*                Compute A=L*Q
*                (Workspace: need 2*M, prefer M+M*NB)
*
                 CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy L to U, zeroing out above it
*
                 CALL DLACPY( 'L', M, M, A, LDA, U, LDU )
                 CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, U( 1, 2 ),
     $                        LDU )
*
*                Generate Q in A
*                (Workspace: need 2*M, prefer M+M*NB)
*
                 CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = ITAU
                 ITAUQ = IE + M
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Bidiagonalize L in U
*                (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                 CALL DGEBRD( M, M, U, LDU, S, WORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Multiply right vectors bidiagonalizing L by Q in A
*                (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                 CALL DORMBR( 'P', 'L', 'T', M, N, M, U, LDU,
     $                        WORK( ITAUP ), A, LDA, WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate left vectors bidiagonalizing L in U
*                (Workspace: need 4*M, prefer 3*M+M*NB)
*
                 CALL DORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of A in U and computing right
*                singular vectors of A in A
```

```
*                 (Workspace: need BDSPAC)
*
                  CALL DBDSQR( 'U', M, N, M, 0, S, WORK( IE ), A, LDA,
     $                         U, LDU, DUM, 1, WORK( IWORK ), INFO )
*
               END IF
*
            ELSE IF( WNTVS ) THEN
*
               IF( WNTUN ) THEN
*
                  Path 4t(N much larger than M, JOBU='N', JOBVT='S')
*                 M right singular vectors to be computed in VT and
*                 no left singular vectors to be computed
*
                  IF( LWORK.GE.M*M+MAX( 4*M, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IR = 1
                     IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                       WORK(IR) is LDA by M
*
                        LDWRKR = LDA
                     ELSE
*
*                       WORK(IR) is M by M
*
                        LDWRKR = M
                     END IF
                     ITAU = IR + LDWRKR*M
                     IWORK = ITAU + M
*
*                    Compute A=L*Q
*                    (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                     CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to WORK(IR), zeroing out above it
*
                     CALL DLACPY( 'L', M, M, A, LDA, WORK( IR ),
     $                            LDWRKR )
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                            WORK( IR+LDWRKR ), LDWRKR )
*
*                    Generate Q in A
*                    (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
```

```
                 CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = ITAU
                 ITAUQ = IE + M
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Bidiagonalize L in WORK(IR)
*                (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                 CALL DGEBRD( M, M, WORK( IR ), LDWRKR, S,
     $                        WORK( IE ), WORK( ITAUQ ),
     $                        WORK( ITAUP ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate right vectors bidiagonalizing L in
*                WORK(IR)
*                (Workspace: need M*M+4*M, prefer M*M+3*M+(M-1)*NB)
*
                 CALL DORGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                        WORK( ITAUP ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
                 IWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing right
*                singular vectors of L in WORK(IR)
*                (Workspace: need M*M+BDSPAC)
*
                 CALL DBDSQR( 'U', M, M, 0, 0, S, WORK( IE ),
     $                        WORK( IR ), LDWRKR, DUM, 1, DUM, 1,
     $                        WORK( IWORK ), INFO )
*
*                Multiply right singular vectors of L in WORK(IR) by
*                Q in A, storing result in VT
*                (Workspace: need M*M)
*
                 CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IR ),
     $                       LDWRKR, A, LDA, ZERO, VT, LDVT )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + M
*
*                Compute A=L*Q
*                (Workspace: need 2*M, prefer M+M*NB)
*
                 CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
```

```
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy result to VT
*
                     CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL DORGLQ( M, N, M, VT, LDVT, WORK( ITAU ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Zero out above L in A
*
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ),
     $                          LDA )
*
*                    Bidiagonalize L in A
*                    (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                     CALL DGEBRD( M, M, A, LDA, S, WORK( IE ),
     $                          WORK( ITAUQ ), WORK( ITAUP ),
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply right vectors bidiagonalizing L by Q in VT
*                    (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                     CALL DORMBR( 'P', 'L', 'T', M, N, M, A, LDA,
     $                          WORK( ITAUP ), VT, LDVT,
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing right
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', M, N, 0, 0, S, WORK( IE ), VT,
     $                          LDVT, DUM, 1, DUM, 1, WORK( IWORK ),
     $                          INFO )
*
                  END IF
*
               ELSE IF( WNTUO ) THEN
*
*                  Path 5t(N much larger than M, JOBU='O', JOBVT='S')
*                  M right singular vectors to be computed in VT and
```

```
*                 M left singular vectors to be overwritten on A
*
                  IF( LWORK.GE.2*M*M+MAX( 4*M, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+2*LDA*M ) THEN
*
*                       WORK(IU) is LDA by M and WORK(IR) is LDA by M
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*M
                        LDWRKR = LDA
                     ELSE IF( LWORK.GE.WRKBL+( LDA+M )*M ) THEN
*
*                       WORK(IU) is LDA by M and WORK(IR) is M by M
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*M
                        LDWRKR = M
                     ELSE
*
*                       WORK(IU) is M by M and WORK(IR) is M by M
*
                        LDWRKU = M
                        IR = IU + LDWRKU*M
                        LDWRKR = M
                     END IF
                     ITAU = IR + LDWRKR*M
                     IWORK = ITAU + M
*
*                    Compute A=L*Q
*                    (Workspace: need 2*M*M+2*M, prefer 2*M*M+M+M*NB)
*
                     CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to WORK(IU), zeroing out below it
*
                     CALL DLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                            LDWRKU )
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                            WORK( IU+LDWRKU ), LDWRKU )
*
*                    Generate Q in A
*                    (Workspace: need 2*M*M+2*M, prefer 2*M*M+M+M*NB)
*
                     CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
```

```
                  IE = ITAU
                  ITAUQ = IE + M
                  ITAUP = ITAUQ + M
                  IWORK = ITAUP + M
*
*                 Bidiagonalize L in WORK(IU), copying result to
*                 WORK(IR)
*                 (Workspace: need 2*M*M+4*M,
*                             prefer 2*M*M+3*M+2*M*NB)
*
                  CALL DGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                         WORK( IE ), WORK( ITAUQ ),
     $                         WORK( ITAUP ), WORK( IWORK ),
     $                         LWORK-IWORK+1, IERR )
                  CALL DLACPY( 'L', M, M, WORK( IU ), LDWRKU,
     $                         WORK( IR ), LDWRKR )
*
*                 Generate right bidiagonalizing vectors in WORK(IU)
*                 (Workspace: need 2*M*M+4*M-1,
*                             prefer 2*M*M+3*M+(M-1)*NB)
*
                  CALL DORGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                         WORK( ITAUP ), WORK( IWORK ),
     $                         LWORK-IWORK+1, IERR )
*
*                 Generate left bidiagonalizing vectors in WORK(IR)
*                 (Workspace: need 2*M*M+4*M, prefer 2*M*M+3*M+M*NB)
*
                  CALL DORGBR( 'Q', M, M, M, WORK( IR ), LDWRKR,
     $                         WORK( ITAUQ ), WORK( IWORK ),
     $                         LWORK-IWORK+1, IERR )
                  IWORK = IE + M
*
*                 Perform bidiagonal QR iteration, computing left
*                 singular vectors of L in WORK(IR) and computing
*                 right singular vectors of L in WORK(IU)
*                 (Workspace: need 2*M*M+BDSPAC)
*
                  CALL DBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                         WORK( IU ), LDWRKU, WORK( IR ),
     $                         LDWRKR, DUM, 1, WORK( IWORK ), INFO )
*
*                 Multiply right singular vectors of L in WORK(IU) by
*                 Q in A, storing result in VT
*                 (Workspace: need M*M)
*
                  CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IU ),
     $                        LDWRKU, A, LDA, ZERO, VT, LDVT )
*
*                 Copy left singular vectors of L to A
```

```
*                    (Workspace: need M*M)
*
                     CALL DLACPY( 'F', M, M, WORK( IR ), LDWRKR, A,
     $                           LDA )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL DORGLQ( M, N, M, VT, LDVT, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Zero out above L in A
*
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ),
     $                           LDA )
*
*                    Bidiagonalize L in A
*                    (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                     CALL DGEBRD( M, M, A, LDA, S, WORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply right vectors bidiagonalizing L by Q in VT
*                    (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                     CALL DORMBR( 'P', 'L', 'T', M, N, M, A, LDA,
     $                           WORK( ITAUP ), VT, LDVT,
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors of L in A
*                    (Workspace: need 4*M, prefer 3*M+M*NB)
```

```
*
                 CALL DORGBR( 'Q', M, M, M, A, LDA, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IWORK = IE + M
*
*                Perform bidiagonal QR iteration, compute left
*                singular vectors of A in A and compute right
*                singular vectors of A in VT
*                (Workspace: need BDSPAC)
*
                 CALL DBDSQR( 'U', M, N, M, 0, S, WORK( IE ), VT,
     $                        LDVT, A, LDA, DUM, 1, WORK( IWORK ),
     $                        INFO )
*
              END IF
*
           ELSE IF( WNTUAS ) THEN
*
              Path 6t(N much larger than M, JOBU='S' or 'A',
*                     JOBVT='S')
*             M right singular vectors to be computed in VT and
*             M left singular vectors to be computed in U
*
              IF( LWORK.GE.M*M+MAX( 4*M, BDSPAC ) ) THEN
*
*                Sufficient workspace for a fast algorithm
*
                 IU = 1
                 IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                   WORK(IU) is LDA by N
*
                    LDWRKU = LDA
                 ELSE
*
*                   WORK(IU) is LDA by M
*
                    LDWRKU = M
                 END IF
                 ITAU = IU + LDWRKU*M
                 IWORK = ITAU + M
*
*                Compute A=L*Q
*                (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                 CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Copy L to WORK(IU), zeroing out above it
*
```

```
                    CALL DLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                           LDWRKU )
                    CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                           WORK( IU+LDWRKU ), LDWRKU )
*
*                   Generate Q in A
*                   (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                    CALL DORGLQ( M, N, M, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = ITAU
                    ITAUQ = IE + M
                    ITAUP = ITAUQ + M
                    IWORK = ITAUP + M
*
*                   Bidiagonalize L in WORK(IU), copying result to U
*                   (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                    CALL DGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    CALL DLACPY( 'L', M, M, WORK( IU ), LDWRKU, U,
     $                           LDU )
*
*                   Generate right bidiagonalizing vectors in WORK(IU)
*                   (Workspace: need M*M+4*M-1,
*                               prefer M*M+3*M+(M-1)*NB)
*
                    CALL DORGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate left bidiagonalizing vectors in U
*                   (Workspace: need M*M+4*M, prefer M*M+3*M+M*NB)
*
                    CALL DORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IWORK = IE + M
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of L in U and computing right
*                   singular vectors of L in WORK(IU)
*                   (Workspace: need M*M+BDSPAC)
*
                    CALL DBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                           WORK( IU ), LDWRKU, U, LDU, DUM, 1,
     $                           WORK( IWORK ), INFO )
*
*                   Multiply right singular vectors of L in WORK(IU) by
```

```
*                    Q in A, storing result in VT
*                    (Workspace: need M*M)
*
                     CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IU ),
     $                           LDWRKU, A, LDA, ZERO, VT, LDVT )
*
                  ELSE
*
*                    Insufficient workspace for a fast algorithm
*
                     ITAU = 1
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                    Generate Q in VT
*                    (Workspace: need 2*M, prefer M+M*NB)
*
                     CALL DORGLQ( M, N, M, VT, LDVT, WORK( ITAU ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to U, zeroing out above it
*
                     CALL DLACPY( 'L', M, M, A, LDA, U, LDU )
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, U( 1, 2 ),
     $                            LDU )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Bidiagonalize L in U
*                    (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                     CALL DGEBRD( M, M, U, LDU, S, WORK( IE ),
     $                            WORK( ITAUQ ), WORK( ITAUP ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply right bidiagonalizing vectors in U by Q
*                    in VT
*                    (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                     CALL DORMBR( 'P', 'L', 'T', M, N, M, U, LDU,
     $                            WORK( ITAUP ), VT, LDVT,
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
```

```
*
*                    Generate left bidiagonalizing vectors in U
*                    (Workspace: need 4*M, prefer 3*M+M*NB)
*
                     CALL DORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                            WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', M, N, M, 0, S, WORK( IE ), VT,
     $                            LDVT, U, LDU, DUM, 1, WORK( IWORK ),
     $                            INFO )
*
                  END IF
*
               END IF
*
            ELSE IF( WNTVA ) THEN
*
               IF( WNTUN ) THEN
*
*                 Path 7t(N much larger than M, JOBU='N', JOBVT='A')
*                 N right singular vectors to be computed in VT and
*                 no left singular vectors to be computed
*
                  IF( LWORK.GE.M*M+MAX( N+M, 4*M, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IR = 1
                     IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                       WORK(IR) is LDA by M
*
                        LDWRKR = LDA
                     ELSE
*
*                       WORK(IR) is M by M
*
                        LDWRKR = M
                     END IF
                     ITAU = IR + LDWRKR*M
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
*                    (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
```

```
*
                    CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                   Copy L to WORK(IR), zeroing out above it
*
                    CALL DLACPY( 'L', M, M, A, LDA, WORK( IR ),
     $                           LDWRKR )
                    CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                           WORK( IR+LDWRKR ), LDWRKR )
*
*                   Generate Q in VT
*                   (Workspace: need M*M+M+N, prefer M*M+M+N*NB)
*
                    CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = ITAU
                    ITAUQ = IE + M
                    ITAUP = ITAUQ + M
                    IWORK = ITAUP + M
*
*                   Bidiagonalize L in WORK(IR)
*                   (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                    CALL DGEBRD( M, M, WORK( IR ), LDWRKR, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate right bidiagonalizing vectors in WORK(IR)
*                   (Workspace: need M*M+4*M-1,
*                               prefer M*M+3*M+(M-1)*NB)
*
                    CALL DORGBR( 'P', M, M, M, WORK( IR ), LDWRKR,
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    IWORK = IE + M
*
*                   Perform bidiagonal QR iteration, computing right
*                   singular vectors of L in WORK(IR)
*                   (Workspace: need M*M+BDSPAC)
*
                    CALL DBDSQR( 'U', M, M, 0, 0, S, WORK( IE ),
     $                           WORK( IR ), LDWRKR, DUM, 1, DUM, 1,
     $                           WORK( IWORK ), INFO )
*
*                   Multiply right singular vectors of L in WORK(IR) by
*                   Q in VT, storing result in A
*                   (Workspace: need M*M)
```

```
*
                 CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IR ),
     $                      LDWRKR, VT, LDVT, ZERO, A, LDA )
*
*                Copy right singular vectors of A from A to VT
*
                 CALL DLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + M
*
*                Compute A=L*Q, copying result to VT
*                (Workspace: need 2*M, prefer M+M*NB)
*
                 CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                Generate Q in VT
*                (Workspace: need M+N, prefer M+N*NB)
*
                 CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IE = ITAU
                 ITAUQ = IE + M
                 ITAUP = ITAUQ + M
                 IWORK = ITAUP + M
*
*                Zero out above L in A
*
                 CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ),
     $                        LDA )
*
*                Bidiagonalize L in A
*                (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                 CALL DGEBRD( M, M, A, LDA, S, WORK( IE ),
     $                        WORK( ITAUQ ), WORK( ITAUP ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                Multiply right bidiagonalizing vectors in A by Q
*                in VT
*                (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                 CALL DORMBR( 'P', 'L', 'T', M, N, M, A, LDA,
     $                        WORK( ITAUP ), VT, LDVT,
```

```
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing right
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', M, N, 0, 0, S, WORK( IE ), VT,
     $                          LDVT, DUM, 1, DUM, 1, WORK( IWORK ),
     $                          INFO )
*
                  END IF
*
               ELSE IF( WNTUO ) THEN
*
                  Path 8t(N much larger than M, JOBU='O', JOBVT='A')
*                 N right singular vectors to be computed in VT and
*                 M left singular vectors to be overwritten on A
*
                  IF( LWORK.GE.2*M*M+MAX( N+M, 4*M, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
                     IF( LWORK.GE.WRKBL+2*LDA*M ) THEN
*
*                       WORK(IU) is LDA by M and WORK(IR) is LDA by M
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*M
                        LDWRKR = LDA
                     ELSE IF( LWORK.GE.WRKBL+( LDA+M )*M ) THEN
*
*                       WORK(IU) is LDA by M and WORK(IR) is M by M
*
                        LDWRKU = LDA
                        IR = IU + LDWRKU*M
                        LDWRKR = M
                     ELSE
*
*                       WORK(IU) is M by M and WORK(IR) is M by M
*
                        LDWRKU = M
                        IR = IU + LDWRKU*M
                        LDWRKR = M
                     END IF
                     ITAU = IR + LDWRKR*M
                     IWORK = ITAU + M
*
*                    Compute A=L*Q, copying result to VT
```

```
*                   (Workspace: need 2*M*M+2*M, prefer 2*M*M+M+M*NB)
*
                    CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                   Generate Q in VT
*                   (Workspace: need 2*M*M+M+N, prefer 2*M*M+M+N*NB)
*
                    CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                   Copy L to WORK(IU), zeroing out above it
*
                    CALL DLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                           LDWRKU )
                    CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                           WORK( IU+LDWRKU ), LDWRKU )
                    IE = ITAU
                    ITAUQ = IE + M
                    ITAUP = ITAUQ + M
                    IWORK = ITAUP + M
*
*                   Bidiagonalize L in WORK(IU), copying result to
*                   WORK(IR)
*                   (Workspace: need 2*M*M+4*M,
*                               prefer 2*M*M+3*M+2*M*NB)
*
                    CALL DGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                           WORK( IE ), WORK( ITAUQ ),
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    CALL DLACPY( 'L', M, M, WORK( IU ), LDWRKU,
     $                           WORK( IR ), LDWRKR )
*
*                   Generate right bidiagonalizing vectors in WORK(IU)
*                   (Workspace: need 2*M*M+4*M-1,
*                               prefer 2*M*M+3*M+(M-1)*NB)
*
                    CALL DORGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                           WORK( ITAUP ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
*
*                   Generate left bidiagonalizing vectors in WORK(IR)
*                   (Workspace: need 2*M*M+4*M, prefer 2*M*M+3*M+M*NB)
*
                    CALL DORGBR( 'Q', M, M, M, WORK( IR ), LDWRKR,
     $                           WORK( ITAUQ ), WORK( IWORK ),
     $                           LWORK-IWORK+1, IERR )
                    IWORK = IE + M
```

```
*
*                   Perform bidiagonal QR iteration, computing left
*                   singular vectors of L in WORK(IR) and computing
*                   right singular vectors of L in WORK(IU)
*                   (Workspace: need 2*M*M+BDSPAC)
*
                    CALL DBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                           WORK( IU ), LDWRKU, WORK( IR ),
     $                           LDWRKR, DUM, 1, WORK( IWORK ), INFO )
*
*                   Multiply right singular vectors of L in WORK(IU) by
*                   Q in VT, storing result in A
*                   (Workspace: need M*M)
*
                    CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IU ),
     $                          LDWRKU, VT, LDVT, ZERO, A, LDA )
*
*                   Copy right singular vectors of A from A to VT
*
                    CALL DLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
*                   Copy left singular vectors of A from WORK(IR) to A
*
                    CALL DLACPY( 'F', M, M, WORK( IR ), LDWRKR, A,
     $                           LDA )
*
                 ELSE
*
*                   Insufficient workspace for a fast algorithm
*
                    ITAU = 1
                    IWORK = ITAU + M
*
*                   Compute A=L*Q, copying result to VT
*                   (Workspace: need 2*M, prefer M+M*NB)
*
                    CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                   Generate Q in VT
*                   (Workspace: need M+N, prefer M+N*NB)
*
                    CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                    IE = ITAU
                    ITAUQ = IE + M
                    ITAUP = ITAUQ + M
                    IWORK = ITAUP + M
*
```

```
*                    Zero out above L in A
*
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, A( 1, 2 ),
     $                           LDA )
*
*                    Bidiagonalize L in A
*                    (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                     CALL DGEBRD( M, M, A, LDA, S, WORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply right bidiagonalizing vectors in A by Q
*                    in VT
*                    (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                     CALL DORMBR( 'P', 'L', 'T', M, N, M, A, LDA,
     $                           WORK( ITAUP ), VT, LDVT,
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors in A
*                    (Workspace: need 4*M, prefer 3*M+M*NB)
*
                     CALL DORGBR( 'Q', M, M, M, A, LDA, WORK( ITAUQ ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in A and computing right
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', M, N, M, 0, S, WORK( IE ), VT,
     $                           LDVT, A, LDA, DUM, 1, WORK( IWORK ),
     $                           INFO )
*
                  END IF
*
               ELSE IF( WNTUAS ) THEN
*
                  Path 9t(N much larger than M, JOBU='S' or 'A',
*                         JOBVT='A')
*                 N right singular vectors to be computed in VT and
*                 M left singular vectors to be computed in U
*
                  IF( LWORK.GE.M*M+MAX( N+M, 4*M, BDSPAC ) ) THEN
*
*                    Sufficient workspace for a fast algorithm
*
                     IU = 1
```

```
                IF( LWORK.GE.WRKBL+LDA*M ) THEN
*
*                  WORK(IU) is LDA by M
*
                   LDWRKU = LDA
                ELSE
*
*                  WORK(IU) is M by M
*
                   LDWRKU = M
                END IF
                ITAU = IU + LDWRKU*M
                IWORK = ITAU + M
*
*               Compute A=L*Q, copying result to VT
*               (Workspace: need M*M+2*M, prefer M*M+M+M*NB)
*
                CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                       WORK( IWORK ), LWORK-IWORK+1, IERR )
                CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*               Generate Q in VT
*               (Workspace: need M*M+M+N, prefer M*M+M+N*NB)
*
                CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
     $                       WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*               Copy L to WORK(IU), zeroing out above it
*
                CALL DLACPY( 'L', M, M, A, LDA, WORK( IU ),
     $                       LDWRKU )
                CALL DLASET( 'U', M-1, M-1, ZERO, ZERO,
     $                       WORK( IU+LDWRKU ), LDWRKU )
                IE = ITAU
                ITAUQ = IE + M
                ITAUP = ITAUQ + M
                IWORK = ITAUP + M
*
*               Bidiagonalize L in WORK(IU), copying result to U
*               (Workspace: need M*M+4*M, prefer M*M+3*M+2*M*NB)
*
                CALL DGEBRD( M, M, WORK( IU ), LDWRKU, S,
     $                       WORK( IE ), WORK( ITAUQ ),
     $                       WORK( ITAUP ), WORK( IWORK ),
     $                       LWORK-IWORK+1, IERR )
                CALL DLACPY( 'L', M, M, WORK( IU ), LDWRKU, U,
     $                       LDU )
*
*               Generate right bidiagonalizing vectors in WORK(IU)
*               (Workspace: need M*M+4*M, prefer M*M+3*M+(M-1)*NB)
```

```
*
                 CALL DORGBR( 'P', M, M, M, WORK( IU ), LDWRKU,
     $                        WORK( ITAUP ), WORK( IWORK ),
     $                        LWORK-IWORK+1, IERR )
*
*                Generate left bidiagonalizing vectors in U
*                (Workspace: need M*M+4*M, prefer M*M+3*M+M*NB)
*
                 CALL DORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 IWORK = IE + M
*
*                Perform bidiagonal QR iteration, computing left
*                singular vectors of L in U and computing right
*                singular vectors of L in WORK(IU)
*                (Workspace: need M*M+BDSPAC)
*
                 CALL DBDSQR( 'U', M, M, M, 0, S, WORK( IE ),
     $                        WORK( IU ), LDWRKU, U, LDU, DUM, 1,
     $                        WORK( IWORK ), INFO )
*
*                Multiply right singular vectors of L in WORK(IU) by
*                Q in VT, storing result in A
*                (Workspace: need M*M)
*
                 CALL DGEMM( 'N', 'N', M, N, M, ONE, WORK( IU ),
     $                       LDWRKU, VT, LDVT, ZERO, A, LDA )
*
*                Copy right singular vectors of A from A to VT
*
                 CALL DLACPY( 'F', M, N, A, LDA, VT, LDVT )
*
              ELSE
*
*                Insufficient workspace for a fast algorithm
*
                 ITAU = 1
                 IWORK = ITAU + M
*
*                Compute A=L*Q, copying result to VT
*                (Workspace: need 2*M, prefer M+M*NB)
*
                 CALL DGELQF( M, N, A, LDA, WORK( ITAU ),
     $                        WORK( IWORK ), LWORK-IWORK+1, IERR )
                 CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
*
*                Generate Q in VT
*                (Workspace: need M+N, prefer M+N*NB)
*
                 CALL DORGLQ( N, N, M, VT, LDVT, WORK( ITAU ),
```

```
     $                          WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Copy L to U, zeroing out above it
*
                     CALL DLACPY( 'L', M, M, A, LDA, U, LDU )
                     CALL DLASET( 'U', M-1, M-1, ZERO, ZERO, U( 1, 2 ),
     $                           LDU )
                     IE = ITAU
                     ITAUQ = IE + M
                     ITAUP = ITAUQ + M
                     IWORK = ITAUP + M
*
*                    Bidiagonalize L in U
*                    (Workspace: need 4*M, prefer 3*M+2*M*NB)
*
                     CALL DGEBRD( M, M, U, LDU, S, WORK( IE ),
     $                           WORK( ITAUQ ), WORK( ITAUP ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Multiply right bidiagonalizing vectors in U by Q
*                    in VT
*                    (Workspace: need 3*M+N, prefer 3*M+N*NB)
*
                     CALL DORMBR( 'P', 'L', 'T', M, N, M, U, LDU,
     $                           WORK( ITAUP ), VT, LDVT,
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
*
*                    Generate left bidiagonalizing vectors in U
*                    (Workspace: need 4*M, prefer 3*M+M*NB)
*
                     CALL DORGBR( 'Q', M, M, M, U, LDU, WORK( ITAUQ ),
     $                           WORK( IWORK ), LWORK-IWORK+1, IERR )
                     IWORK = IE + M
*
*                    Perform bidiagonal QR iteration, computing left
*                    singular vectors of A in U and computing right
*                    singular vectors of A in VT
*                    (Workspace: need BDSPAC)
*
                     CALL DBDSQR( 'U', M, N, M, 0, S, WORK( IE ), VT,
     $                           LDVT, U, LDU, DUM, 1, WORK( IWORK ),
     $                           INFO )
*
                  END IF
*
               END IF
*
            END IF
*
         ELSE
```

```
*
*          N .LT. MNTHR
*
*          Path 10t(N greater than M, but not much larger)
*          Reduce to bidiagonal form without LQ decomposition
*
           IE = 1
           ITAUQ = IE + M
           ITAUP = ITAUQ + M
           IWORK = ITAUP + M
*
*          Bidiagonalize A
*          (Workspace: need 3*M+N, prefer 3*M+(M+N)*NB)
*
           CALL DGEBRD( M, N, A, LDA, S, WORK( IE ), WORK( ITAUQ ),
     $                  WORK( ITAUP ), WORK( IWORK ), LWORK-IWORK+1,
     $                  IERR )
           IF( WNTUAS ) THEN
*
*             If left singular vectors desired in U, copy result to U
*             and generate left bidiagonalizing vectors in U
*             (Workspace: need 4*M-1, prefer 3*M+(M-1)*NB)
*
              CALL DLACPY( 'L', M, M, A, LDA, U, LDU )
              CALL DORGBR( 'Q', M, M, N, U, LDU, WORK( ITAUQ ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
           IF( WNTVAS ) THEN
*
*             If right singular vectors desired in VT, copy result to
*             VT and generate right bidiagonalizing vectors in VT
*             (Workspace: need 3*M+NRVT, prefer 3*M+NRVT*NB)
*
              CALL DLACPY( 'U', M, N, A, LDA, VT, LDVT )
              IF( WNTVA )
     $           NRVT = N
              IF( WNTVS )
     $           NRVT = M
              CALL DORGBR( 'P', NRVT, N, M, VT, LDVT, WORK( ITAUP ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
           IF( WNTUO ) THEN
*
*             If left singular vectors desired in A, generate left
*             bidiagonalizing vectors in A
*             (Workspace: need 4*M-1, prefer 3*M+(M-1)*NB)
*
              CALL DORGBR( 'Q', M, M, N, A, LDA, WORK( ITAUQ ),
     $                     WORK( IWORK ), LWORK-IWORK+1, IERR )
           END IF
```

```
            IF( WNTVO ) THEN
*
*              If right singular vectors desired in A, generate right
*              bidiagonalizing vectors in A
*              (Workspace: need 4*M, prefer 3*M+M*NB)
*
               CALL DORGBR( 'P', M, N, M, A, LDA, WORK( ITAUP ),
     $                      WORK( IWORK ), LWORK-IWORK+1, IERR )
            END IF
            IWORK = IE + M
            IF( WNTUAS .OR. WNTUO )
     $         NRU = M
            IF( WNTUN )
     $         NRU = 0
            IF( WNTVAS .OR. WNTVO )
     $         NCVT = N
            IF( WNTVN )
     $         NCVT = 0
            IF( ( .NOT.WNTUO ) .AND. ( .NOT.WNTVO ) ) THEN
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in U and computing right singular
*              vectors in VT
*              (Workspace: need BDSPAC)
*
               CALL DBDSQR( 'L', M, NCVT, NRU, 0, S, WORK( IE ), VT,
     $                      LDVT, U, LDU, DUM, 1, WORK( IWORK ), INFO )
            ELSE IF( ( .NOT.WNTUO ) .AND. WNTVO ) THEN
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in U and computing right singular
*              vectors in A
*              (Workspace: need BDSPAC)
*
               CALL DBDSQR( 'L', M, NCVT, NRU, 0, S, WORK( IE ), A, LDA,
     $                      U, LDU, DUM, 1, WORK( IWORK ), INFO )
            ELSE
*
*              Perform bidiagonal QR iteration, if desired, computing
*              left singular vectors in A and computing right singular
*              vectors in VT
*              (Workspace: need BDSPAC)
*
               CALL DBDSQR( 'L', M, NCVT, NRU, 0, S, WORK( IE ), VT,
     $                      LDVT, A, LDA, DUM, 1, WORK( IWORK ), INFO )
            END IF
*
         END IF
*
      END IF
```

```
*
*     If DBDSQR failed to converge, copy unconverged superdiagonals
*     to WORK( 2:MINMN )
*
      IF( INFO.NE.0 ) THEN
         IF( IE.GT.2 ) THEN
            DO 50 I = 1, MINMN - 1
               WORK( I+1 ) = WORK( I+IE-1 )
   50       CONTINUE
         END IF
         IF( IE.LT.2 ) THEN
            DO 60 I = MINMN - 1, 1, -1
               WORK( I+1 ) = WORK( I+IE-1 )
   60       CONTINUE
         END IF
      END IF
*
*     Undo scaling if necessary
*
      IF( ISCL.EQ.1 ) THEN
         IF( ANRM.GT.BIGNUM )
     $      CALL DLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( INFO.NE.0 .AND. ANRM.GT.BIGNUM )
     $      CALL DLASCL( 'G', 0, 0, BIGNUM, ANRM, MINMN-1, 1, WORK( 2 ),
     $                   MINMN, IERR )
         IF( ANRM.LT.SMLNUM )
     $      CALL DLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN, 1, S, MINMN,
     $                   IERR )
         IF( INFO.NE.0 .AND. ANRM.LT.SMLNUM )
     $      CALL DLASCL( 'G', 0, 0, SMLNUM, ANRM, MINMN-1, 1, WORK( 2 ),
     $                   MINMN, IERR )
      END IF
*
*     Return optimal workspace in WORK(1)
*
      WORK( 1 ) = MAXWRK
*
      RETURN
*
*     End of DGESVD
*
      END
```

———-

90.129.4 lisp code

— dgesvd.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dgesvd (jobu jobvt m n a lda s u ldu vt ldvt work lwork info)
    (declare (type (array double-float (*)) work vt u s a)
             (type (f2cl-lib:integer4) info lwork ldvt ldu lda n m)
             (type (string *) jobvt jobu))
    (f2cl-lib:with-multi-array-data
        ((jobu character jobu-%data% jobu-%offset%)
         (jobvt character jobvt-%data% jobvt-%offset%)
         (a double-float a-%data% a-%offset%)
         (s double-float s-%data% s-%offset%)
         (u double-float u-%data% u-%offset%)
         (vt double-float vt-%data% vt-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((dum (make-array 1 :element-type 'double-float)) (anrm 0.0d0)
             (bignum 0.0d0) (eps 0.0d0) (smlnum 0.0d0) (lwork_dgeqrf 0)
             (lwork_dorgqr_n 0) (lwork_dorgqr_m 0) (lwork_dgebrd 0)
             (lwork_dorgbr_p 0) (lwork_dorgbr_q 0) (lwork_dgelqf 0)
             (lwork_dorglq_n 0) (lwork_dorglq_m 0) (bdspac 0) (blk 0) (chunk 0)
             (i 0) (ie 0) (ierr 0) (ir 0) (iscl 0) (itau 0) (itaup 0) (itauq 0)
             (iu 0) (iwork 0) (ldwrkr 0) (ldwrku 0) (maxwrk 0) (minmn 0)
             (minwrk 0) (mnthr 0) (ncu 0) (ncvt 0) (nru 0) (nrvt 0) (wrkbl 0)
             (lquery nil) (wntua nil) (wntuas nil) (wntun nil) (wntuo nil)
             (wntus nil) (wntva nil) (wntvas nil) (wntvn nil) (wntvo nil)
             (wntvs nil))
        (declare (type (array double-float (1)) dum)
                 (type (double-float) anrm bignum eps smlnum)
                 (type (f2cl-lib:integer4) lwork_dgeqrf lwork_dorgqr_n
                  lwork_dorgqr_m lwork_dgebrd lwork_dorgbr_p lwork_dorgbr_q
                  lwork_dgelqf lwork_dorglq_n lwork_dorglq_m bdspac blk chunk i
                  ie ierr ir iscl itau itaup itauq iu iwork ldwrkr ldwrku
                  maxwrk minmn minwrk mnthr ncu ncvt nru nrvt wrkbl)
                 (type f2cl-lib:logical lquery wntua wntuas wntun wntuo wntus
                  wntva wntvas wntvn wntvo wntvs))
        (setf info 0)
        (setf minmn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (setf wntua
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame jobu "A")
                  (declare (ignore var-1))
                  (when var-0 (setf jobu var-0))
                  ret-val))
        (setf wntus
```

```
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobu "S")
          (declare (ignore var-1))
          (when var-0 (setf jobu var-0))
          ret-val))
(setf wntuas (or wntua wntus))
(setf wntuo
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobu "O")
          (declare (ignore var-1))
          (when var-0 (setf jobu var-0))
          ret-val))
(setf wntun
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobu "N")
          (declare (ignore var-1))
          (when var-0 (setf jobu var-0))
          ret-val))
(setf wntva
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvt "A")
          (declare (ignore var-1))
          (when var-0 (setf jobvt var-0))
          ret-val))
(setf wntvs
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvt "S")
          (declare (ignore var-1))
          (when var-0 (setf jobvt var-0))
          ret-val))
(setf wntvas (or wntva wntvs))
(setf wntvo
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvt "O")
          (declare (ignore var-1))
          (when var-0 (setf jobvt var-0))
          ret-val))
(setf wntvn
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvt "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvt var-0))
          ret-val))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond ((not (or wntua wntus wntuo wntun)) (setf info -1))
      ((or (not (or wntva wntvs wntvo wntvn)) (and wntvo wntuo))
       (setf info -2))
      ((< m 0) (setf info -3)) ((< n 0) (setf info -4))
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
```

```
        (setf info -6))
       ((or (< ldu 1) (and wntuas (< ldu m))) (setf info -9))
       ((or (< ldvt 1) (and wntva (< ldvt n))
            (and wntvs (< ldvt minmn)))
        (setf info -11)))
(cond
 ((= info 0) (setf minwrk 1) (setf maxwrk 1)
  (cond
   ((and (>= m n) (> minmn 0))
    (setf mnthr
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (ilaenv 6 "DGESVD" (f2cl-lib:f2cl-// jobu jobvt) m n 0
                 0)
              (declare (ignore var-0 var-1 var-2 var-5 var-6))
              (when var-3 (setf m var-3))
              (when var-4 (setf n var-4))
              ret-val))
    (setf bdspac (f2cl-lib:int-mul 5 n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (dgeqrf m n a lda
         (f2cl-lib:array-slice dum double-float (1) ((1 1)))
         (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1 ierr)
      (declare (ignore var-2 var-4 var-5 var-6))
      (setf m var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf ierr var-7))
    (setf lwork_dgeqrf (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dorgqr m n n a lda
         (f2cl-lib:array-slice dum double-float (1) ((1 1)))
         (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1 ierr)
      (declare (ignore var-3 var-5 var-6 var-7))
      (when var-0 (setf m var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-8 (setf ierr var-8)))
    (setf lwork_dorgqr_n
            (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dorgqr m m n a lda
         (f2cl-lib:array-slice dum double-float (1) ((1 1)))
         (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1 ierr)
      (declare (ignore var-3 var-5 var-6 var-7))
      (when var-0 (setf m var-0))
```

```
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf lwork_dorgqr_m
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgebrd n n a lda s
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1 ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(setf lwork_dgebrd (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dorgbr "P" n n n a lda
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1 ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf ierr var-9)))
(setf lwork_dorgbr_p
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dorgbr "Q" n n n a lda
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1 ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf ierr var-9)))
(setf lwork_dorgbr_q
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(cond
 ((>= m mnthr)
  (cond
   (wntun (setf maxwrk (f2cl-lib:int-add n lwork_dgeqrf))
```

```
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dgebrd))))
 (if (or wntvo wntvas)
     (setf maxwrk
             (max (the f2cl-lib:integer4 maxwrk)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                         lwork_dorgbr_p)))))
 (setf maxwrk
         (max (the f2cl-lib:integer4 maxwrk)
              (the f2cl-lib:integer4 bdspac)))
 (setf minwrk
         (max (the f2cl-lib:integer4 (f2cl-lib:int-mul 4 n))
              (the f2cl-lib:integer4 bdspac))))
((and wntuo wntvn)
 (setf wrkbl (f2cl-lib:int-add n lwork_dgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_dorgqr_n))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dorgbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                                 (f2cl-lib:int-mul m n) n))))
 (setf minwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) m))
          (the f2cl-lib:integer4 bdspac))))
((and wntuo wntvas)
 (setf wrkbl (f2cl-lib:int-add n lwork_dgeqrf))
```

```
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_dorgqr_n))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dorgbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dorgbr_p))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                                 (f2cl-lib:int-mul m n) n))))
 (setf minwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) m))
          (the f2cl-lib:integer4 bdspac))))
((and wntus wntvn)
 (setf wrkbl (f2cl-lib:int-add n lwork_dgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_dorgqr_n))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dorgbr_q))))
```

```
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
 (setf minwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) m))
          (the f2cl-lib:integer4 bdspac))))
((and wntus wntvo)
 (setf wrkbl (f2cl-lib:int-add n lwork_dgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_dorgqr_n))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dorgbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dorgbr_p))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n n) wrkbl))
 (setf minwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) m))
          (the f2cl-lib:integer4 bdspac))))
((and wntus wntvas)
 (setf wrkbl (f2cl-lib:int-add n lwork_dgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_dorgqr_n))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dgebrd))))
```

```
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dorgbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dorgbr_p))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
 (setf minwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) m))
          (the f2cl-lib:integer4 bdspac))))
((and wntua wntvn)
 (setf wrkbl (f2cl-lib:int-add n lwork_dgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_dorgqr_m))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dorgbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
 (setf minwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) m))
          (the f2cl-lib:integer4 bdspac))))
((and wntua wntvo)
 (setf wrkbl (f2cl-lib:int-add n lwork_dgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_dorgqr_m))))
```

```
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dorgbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dorgbr_p))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 n n) wrkbl))
 (setf minwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) m))
          (the f2cl-lib:integer4 bdspac))))
((and wntua wntvas)
 (setf wrkbl (f2cl-lib:int-add n lwork_dgeqrf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add n lwork_dorgqr_m))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dorgbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                     lwork_dorgbr_p))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul n n) wrkbl))
 (setf minwrk
```

```
          (max
           (the f2cl-lib:integer4
                (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) m))
           (the f2cl-lib:integer4 bdspac))))))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (dgebrd m n a lda s
      (f2cl-lib:array-slice dum double-float (1) ((1 1)))
      (f2cl-lib:array-slice dum double-float (1) ((1 1)))
      (f2cl-lib:array-slice dum double-float (1) ((1 1)))
      (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1 ierr)
   (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-10))
 (setf lwork_dgebrd
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
 (setf maxwrk
        (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) lwork_dgebrd))
 (cond
  ((or wntus wntuo)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9)
       (dorgbr "Q" m n n a lda
        (f2cl-lib:array-slice dum double-float (1) ((1 1)))
        (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1
        ierr)
     (declare (ignore var-0 var-4 var-6 var-7 var-8))
     (when var-1 (setf m var-1))
     (when var-2 (setf n var-2))
     (when var-3 (setf n var-3))
     (when var-5 (setf lda var-5))
     (when var-9 (setf ierr var-9)))
   (setf lwork_dorgbr_q
          (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
   (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                      lwork_dorgbr_q))))))
 (cond
  (wntua
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9)
       (dorgbr "Q" m m n a lda
```

```
                (f2cl-lib:array-slice dum double-float (1) ((1 1)))
                (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1
                ierr)
            (declare (ignore var-0 var-4 var-6 var-7 var-8))
            (when var-1 (setf m var-1))
            (when var-2 (setf m var-2))
            (when var-3 (setf n var-3))
            (when var-5 (setf lda var-5))
            (when var-9 (setf ierr var-9)))
          (setf lwork_dorgbr_q
                  (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                              lwork_dorgbr_q))))))
        (cond
         ((not wntvn)
          (setf maxwrk
                  (max (the f2cl-lib:integer4 maxwrk)
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add (f2cl-lib:int-mul 3 n)
                                              lwork_dorgbr_p))))))
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4 bdspac)))
        (setf minwrk
                (max
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 3 n) m))
                 (the f2cl-lib:integer4 bdspac))))))
    ((> minmn 0)
     (setf mnthr
             (multiple-value-bind
                 (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                 (ilaenv 6 "DGESVD" (f2cl-lib:f2cl-// jobu jobvt) m n 0
                  0)
               (declare (ignore var-0 var-1 var-2 var-5 var-6))
               (when var-3 (setf m var-3))
               (when var-4 (setf n var-4))
               ret-val))
     (setf bdspac (f2cl-lib:int-mul 5 m))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (dgelqf m n a lda
          (f2cl-lib:array-slice dum double-float (1) ((1 1)))
          (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1 ierr)
       (declare (ignore var-2 var-4 var-5 var-6))
       (setf m var-0)
       (setf n var-1)
```

```
  (setf lda var-3)
  (setf ierr var-7))
(setf lwork_dgelqf (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorglq n n m
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) n
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1 ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf n var-4))
  (when var-8 (setf ierr var-8)))
(setf lwork_dorglq_n
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorglq m n m a lda
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1 ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf lwork_dorglq_m
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgebrd m m a lda s
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1 ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(setf lwork_dgebrd (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dorgbr "P" m m m a n
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1 ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
```

```
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf n var-5))
  (when var-9 (setf ierr var-9)))
(setf lwork_dorgbr_p
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dorgbr "Q" m m m a n
     (f2cl-lib:array-slice dum double-float (1) ((1 1)))
     (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1 ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf n var-5))
  (when var-9 (setf ierr var-9)))
(setf lwork_dorgbr_q
        (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
(cond
 ((>= n mnthr)
  (cond
   (wntvn (setf maxwrk (f2cl-lib:int-add m lwork_dgelqf))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                        lwork_dgebrd))))
    (if (or wntuo wntuas)
        (setf maxwrk
                (max (the f2cl-lib:integer4 maxwrk)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                            lwork_dorgbr_q)))))
    (setf maxwrk
            (max (the f2cl-lib:integer4 maxwrk)
                 (the f2cl-lib:integer4 bdspac)))
    (setf minwrk
            (max (the f2cl-lib:integer4 (f2cl-lib:int-mul 4 m))
                 (the f2cl-lib:integer4 bdspac))))
   ((and wntvo wntun)
    (setf wrkbl (f2cl-lib:int-add m lwork_dgelqf))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add m lwork_dorglq_m))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
```

```
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                      lwork_dgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     lwork_dorgbr_p))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                 (f2cl-lib:int-mul m n) m))))
 (setf minwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
          (the f2cl-lib:integer4 bdspac))))
((and wntvo wntuas)
 (setf wrkbl (f2cl-lib:int-add m lwork_dgelqf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add m lwork_dorglq_m))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     lwork_dgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     lwork_dorgbr_p))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     lwork_dorgbr_q))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
```

```
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                 (f2cl-lib:int-mul m n) m))))
 (setf minwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
          (the f2cl-lib:integer4 bdspac))))
((and wntvs wntun)
 (setf wrkbl (f2cl-lib:int-add m lwork_dgelqf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add m lwork_dorglq_m))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     lwork_dgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     lwork_dorgbr_p))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
 (setf minwrk
         (max
          (the f2cl-lib:integer4
               (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
          (the f2cl-lib:integer4 bdspac))))
((and wntvs wntuo)
 (setf wrkbl (f2cl-lib:int-add m lwork_dgelqf))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add m lwork_dorglq_m))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     lwork_dgebrd))))
 (setf wrkbl
         (max (the f2cl-lib:integer4 wrkbl)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                     lwork_dorgbr_p))))
 (setf wrkbl
```

```
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    lwork_dorgbr_q))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m m) wrkbl))
 (setf minwrk
        (max
         (the f2cl-lib:integer4
              (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
         (the f2cl-lib:integer4 bdspac))))
((and wntvs wntuas)
 (setf wrkbl (f2cl-lib:int-add m lwork_dgelqf))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add m lwork_dorglq_m))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    lwork_dgebrd))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    lwork_dorgbr_p))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    lwork_dorgbr_q))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
 (setf minwrk
        (max
         (the f2cl-lib:integer4
              (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
         (the f2cl-lib:integer4 bdspac))))
((and wntva wntun)
 (setf wrkbl (f2cl-lib:int-add m lwork_dgelqf))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add m lwork_dorglq_n))))
 (setf wrkbl
```

```
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    lwork_dgebrd))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    lwork_dorgbr_p))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
 (setf minwrk
        (max
         (the f2cl-lib:integer4
              (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
         (the f2cl-lib:integer4 bdspac))))
((and wntva wntuo)
 (setf wrkbl (f2cl-lib:int-add m lwork_dgelqf))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add m lwork_dorglq_n))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    lwork_dgebrd))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    lwork_dorgbr_p))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                    lwork_dorgbr_q))))
 (setf wrkbl
        (max (the f2cl-lib:integer4 wrkbl)
             (the f2cl-lib:integer4 bdspac)))
 (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul 2 m m) wrkbl))
 (setf minwrk
        (max
         (the f2cl-lib:integer4
              (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
         (the f2cl-lib:integer4 bdspac))))
((and wntva wntuas)
 (setf wrkbl (f2cl-lib:int-add m lwork_dgelqf))
```

```
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add m lwork_dorglq_n))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                        lwork_dgebrd))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                        lwork_dorgbr_p))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                        lwork_dorgbr_q))))
    (setf wrkbl
            (max (the f2cl-lib:integer4 wrkbl)
                 (the f2cl-lib:integer4 bdspac)))
    (setf maxwrk (f2cl-lib:int-add (f2cl-lib:int-mul m m) wrkbl))
    (setf minwrk
            (max
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
             (the f2cl-lib:integer4 bdspac))))))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (dgebrd m n a lda s
      (f2cl-lib:array-slice dum double-float (1) ((1 1)))
      (f2cl-lib:array-slice dum double-float (1) ((1 1)))
      (f2cl-lib:array-slice dum double-float (1) ((1 1)))
      (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1 ierr)
   (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-10))
 (setf lwork_dgebrd
         (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
 (setf maxwrk
         (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) lwork_dgebrd))
 (cond
  ((or wntvs wntvo)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
        var-9)
       (dorgbr "P" m n m a n
        (f2cl-lib:array-slice dum double-float (1) ((1 1)))
        (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1
        ierr)
     (declare (ignore var-0 var-4 var-6 var-7 var-8))
     (when var-1 (setf m var-1))
     (when var-2 (setf n var-2))
     (when var-3 (setf m var-3))
     (when var-5 (setf n var-5))
     (when var-9 (setf ierr var-9)))
   (setf lwork_dorgbr_p
           (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
   (setf maxwrk
           (max (the f2cl-lib:integer4 maxwrk)
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                       lwork_dorgbr_p))))))
(cond
 (wntva
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (dorgbr "P" n n m a n
       (f2cl-lib:array-slice dum double-float (1) ((1 1)))
       (f2cl-lib:array-slice dum double-float (1) ((1 1))) -1
       ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf n var-5))
    (when var-9 (setf ierr var-9)))
  (setf lwork_dorgbr_p
          (f2cl-lib:int (f2cl-lib:fref dum (1) ((1 1)))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                      lwork_dorgbr_p))))))
(cond
 ((not wntun)
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 m)
                                      lwork_dorgbr_q))))))
(setf maxwrk
        (max (the f2cl-lib:integer4 maxwrk)
             (the f2cl-lib:integer4 bdspac)))
```

```
      (setf minwrk
              (max
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul 3 m) n))
               (the f2cl-lib:integer4 bdspac)))))))
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4 minwrk)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 maxwrk) 'double-float))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -13)))))
(cond
 ((/= info 0) (xerbla "DGESVD" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond ((or (= m 0) (= n 0)) (go end_label)))
(setf eps (dlamch "P"))
(setf smlnum (/ (f2cl-lib:fsqrt (dlamch "S")) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (dlange "M" m n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf iscl 0)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf iscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 anrm smlnum m n a lda ierr)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf smlnum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
    (when var-9 (setf ierr var-9))))
 ((> anrm bignum) (setf iscl 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dlascl "G" 0 0 anrm bignum m n a lda ierr)
    (declare (ignore var-0 var-1 var-2 var-7))
    (when var-3 (setf anrm var-3))
    (when var-4 (setf bignum var-4))
    (when var-5 (setf m var-5))
    (when var-6 (setf n var-6))
    (when var-8 (setf lda var-8))
```

```
    (when var-9 (setf ierr var-9)))))
(cond
 ((>= m n)
  (cond
   ((>= m mnthr)
    (cond
     (wntun (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (dgeqrf m n a lda
           (f2cl-lib:array-slice work-%data% double-float (itau)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% double-float (iwork)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
        (declare (ignore var-2 var-4 var-5 var-6))
        (setf m var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf ierr var-7))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
           zero zero
           (f2cl-lib:array-slice a-%data% double-float (2 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-1 var-2 var-5))
        (when var-3 (setf zero var-3))
        (when var-4 (setf zero var-4))
        (when var-6 (setf lda var-6)))
      (setf ie 1) (setf itauq (f2cl-lib:int-add ie n))
      (setf itaup (f2cl-lib:int-add itauq n))
      (setf iwork (f2cl-lib:int-add itaup n))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (dgebrd n n a lda s
           (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                                 work-%offset%)
           (f2cl-lib:array-slice work-%data% double-float (itauq)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% double-float (itaup)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% double-float (iwork)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
        (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
        (setf n var-0)
        (setf n var-1)
        (setf lda var-3)
```

```
    (setf ierr var-10))
  (setf ncvt 0)
  (cond
   ((or wntvo wntvas)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9)
        (dorgbr "P" n n n a lda
         (f2cl-lib:array-slice work-%data% double-float (itaup)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (iwork)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
      (declare (ignore var-0 var-4 var-6 var-7 var-8))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf n var-3))
      (when var-5 (setf lda var-5))
      (when var-9 (setf ierr var-9)))
    (setf ncvt n)))
  (setf iwork (f2cl-lib:int-add ie n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (dbdsqr "U" n ncvt 0 0 s
       (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                             work-%offset%)
       a lda dum 1 dum 1
       (f2cl-lib:array-slice work-%data% double-float (iwork)
                             ((1 *)) work-%offset%)
       info)
    (declare
     (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
      var-11 var-12 var-13))
    (setf n var-1)
    (setf ncvt var-2)
    (setf lda var-8)
    (setf info var-14))
  (if wntvas
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlacpy "F" n n a lda vt ldvt)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldvt var-6)))))
 ((and wntuo wntvn)
  (cond
   ((>= lwork
```

```
    (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                      (max
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-mul 4 n))
                       (the f2cl-lib:integer4 bdspac))))
(setf ir 1)
(cond
 ((>= lwork
      (f2cl-lib:int-add
       (max (the f2cl-lib:integer4 wrkbl)
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-mul lda n)
                                   n)))
       (f2cl-lib:int-mul lda n)))
  (setf ldwrku lda) (setf ldwrkr lda))
 ((>= lwork
      (f2cl-lib:int-add
       (max (the f2cl-lib:integer4 wrkbl)
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-mul lda n)
                                   n)))
       (f2cl-lib:int-mul n n)))
  (setf ldwrku lda) (setf ldwrkr n))
 (t
  (setf ldwrku
          (the f2cl-lib:integer4
               (truncate (- lwork (* n n) n) n)))
  (setf ldwrkr n)))
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
(setf iwork (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dgeqrf m n a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
```

```
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice work-%data% double-float ((+ ir 1))
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorgqr m n n a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd n n
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
```

```
  (setf n var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "Q" n n n
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" n 0 n 0 s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     dum 1
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr dum 1
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
    var-11 var-12 var-13))
  (setf n var-1)
  (setf n var-3)
  (setf ldwrkr var-10)
  (setf info var-14))
(setf iu (f2cl-lib:int-add ie n))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i ldwrku))
              ((> i m) nil)
  (tagbody
    (setf chunk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
```

```
                (the f2cl-lib:integer4 ldwrku)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12)
            (dgemm "N" "N" chunk n n one
             (f2cl-lib:array-slice a-%data% double-float (i 1)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice work-%data% double-float (ir)
                                   ((1 *)) work-%offset%)
             ldwrkr zero
             (f2cl-lib:array-slice work-%data% double-float (iu)
                                   ((1 *)) work-%offset%)
             ldwrku)
          (declare (ignore var-0 var-1 var-6 var-8 var-11))
          (when var-2 (setf chunk var-2))
          (when var-3 (setf n var-3))
          (when var-4 (setf n var-4))
          (when var-5 (setf one var-5))
          (when var-7 (setf lda var-7))
          (when var-9 (setf ldwrkr var-9))
          (when var-10 (setf zero var-10))
          (when var-12 (setf ldwrku var-12)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dlacpy "F" chunk n
             (f2cl-lib:array-slice work-%data% double-float (iu)
                                   ((1 *)) work-%offset%)
             ldwrku
             (f2cl-lib:array-slice a-%data% double-float (i 1)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf chunk var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf ldwrku var-4))
          (when var-6 (setf lda var-6)))
      label10)))
  (t (setf ie 1) (setf itauq (f2cl-lib:int-add ie n))
   (setf itaup (f2cl-lib:int-add itauq n))
   (setf iwork (f2cl-lib:int-add itaup n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (dgebrd m n a lda s
        (f2cl-lib:array-slice work-%data% double-float (ie)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (itauq)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (itaup)
```

```
                                ((1 *)) work-%offset%)
            (f2cl-lib:array-slice work-%data% double-float (iwork)
                                ((1 *)) work-%offset%)
            (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
        (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
        (setf m var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf ierr var-10))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9)
          (dorgbr "Q" m n n a lda
           (f2cl-lib:array-slice work-%data% double-float (itauq)
                                ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% double-float (iwork)
                                ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
        (declare (ignore var-0 var-4 var-6 var-7 var-8))
        (when var-1 (setf m var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf n var-3))
        (when var-5 (setf lda var-5))
        (when var-9 (setf ierr var-9)))
      (setf iwork (f2cl-lib:int-add ie n))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12 var-13 var-14)
          (dbdsqr "U" n 0 m 0 s
           (f2cl-lib:array-slice work-%data% double-float (ie)
                                ((1 *)) work-%offset%)
           dum 1 a lda dum 1
           (f2cl-lib:array-slice work-%data% double-float (iwork)
                                ((1 *)) work-%offset%)
           info)
        (declare
         (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
          var-11 var-12 var-13))
        (setf n var-1)
        (setf m var-3)
        (setf lda var-10)
        (setf info var-14)))))
  ((and wntuo wntvas)
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-mul 4 n))
                            (the f2cl-lib:integer4 bdspac))))
```

```
(setf ir 1)
(cond
 ((>= lwork
      (f2cl-lib:int-add
       (max (the f2cl-lib:integer4 wrkbl)
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-mul lda n)
                                   n)))
       (f2cl-lib:int-mul lda n)))
  (setf ldwrku lda) (setf ldwrkr lda))
 ((>= lwork
      (f2cl-lib:int-add
       (max (the f2cl-lib:integer4 wrkbl)
            (the f2cl-lib:integer4
                 (f2cl-lib:int-add (f2cl-lib:int-mul lda n)
                                   n)))
       (f2cl-lib:int-mul n n)))
  (setf ldwrku lda) (setf ldwrkr n))
 (t
  (setf ldwrku
          (the f2cl-lib:integer4
               (truncate (- lwork (* n n) n) n)))
  (setf ldwrkr n)))
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
(setf iwork (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dgeqrf m n a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "U" n n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(if (> n 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "L" (f2cl-lib:int-sub n 1)
```

```
          (f2cl-lib:int-sub n 1) zero zero
          (f2cl-lib:array-slice vt-%data% double-float (2 1)
                                ((1 ldvt) (1 *)) vt-%offset%)
          ldvt)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldvt var-6))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorgqr m n n a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd n n vt ldvt s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldvt var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" n n vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
```

```
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldvt var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "Q" n n n
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "P" n n n vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" n n n 0 s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr dum 1
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
```

```
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf n var-1)
  (setf n var-2)
  (setf n var-3)
  (setf ldvt var-8)
  (setf ldwrkr var-10)
  (setf info var-14))
(setf iu (f2cl-lib:int-add ie n))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i ldwrku))
              ((> i m) nil)
  (tagbody
    (setf chunk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
             (the f2cl-lib:integer4 ldwrku)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (dgemm "N" "N" chunk n n one
         (f2cl-lib:array-slice a-%data% double-float (i 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice work-%data% double-float (ir)
                               ((1 *)) work-%offset%)
         ldwrkr zero
         (f2cl-lib:array-slice work-%data% double-float (iu)
                               ((1 *)) work-%offset%)
         ldwrku)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf chunk var-2))
      (when var-3 (setf n var-3))
      (when var-4 (setf n var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf lda var-7))
      (when var-9 (setf ldwrkr var-9))
      (when var-10 (setf zero var-10))
      (when var-12 (setf ldwrku var-12)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "F" chunk n
         (f2cl-lib:array-slice work-%data% double-float (iu)
                               ((1 *)) work-%offset%)
         ldwrku
         (f2cl-lib:array-slice a-%data% double-float (i 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda)
```

```
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf chunk var-1))
        (when var-2 (setf n var-2))
        (when var-4 (setf ldwrku var-4))
        (when var-6 (setf lda var-6)))
    label20)))
(t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (dgeqrf m n a lda
      (f2cl-lib:array-slice work-%data% double-float (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-7))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlacpy "U" n n a lda vt ldvt)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldvt var-6)))
 (if (> n 1)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (dlaset "L" (f2cl-lib:int-sub n 1)
          (f2cl-lib:int-sub n 1) zero zero
          (f2cl-lib:array-slice vt-%data% double-float (2 1)
                                ((1 ldvt) (1 *)) vt-%offset%)
          ldvt)
       (declare (ignore var-0 var-1 var-2 var-5))
       (when var-3 (setf zero var-3))
       (when var-4 (setf zero var-4))
       (when var-6 (setf ldvt var-6))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (dorgqr m n n a lda
      (f2cl-lib:array-slice work-%data% double-float (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
   (declare (ignore var-3 var-5 var-6 var-7))
   (when var-0 (setf m var-0))
```

```
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd n n vt ldvt s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldvt var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (dormbr "Q" "R" "N" m n n vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     a lda
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldvt var-7))
  (when var-10 (setf lda var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "P" n n n vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
```

```
                                ((1 *)) work-%offset%)
            (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
        (declare (ignore var-0 var-4 var-6 var-7 var-8))
        (when var-1 (setf n var-1))
        (when var-2 (setf n var-2))
        (when var-3 (setf n var-3))
        (when var-5 (setf ldvt var-5))
        (when var-9 (setf ierr var-9)))
      (setf iwork (f2cl-lib:int-add ie n))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12 var-13 var-14)
          (dbdsqr "U" n n m 0 s
           (f2cl-lib:array-slice work-%data% double-float (ie)
                                ((1 *)) work-%offset%)
           vt ldvt a lda dum 1
           (f2cl-lib:array-slice work-%data% double-float (iwork)
                                ((1 *)) work-%offset%)
           info)
        (declare
         (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
          var-13))
        (setf n var-1)
        (setf n var-2)
        (setf m var-3)
        (setf ldvt var-8)
        (setf lda var-10)
        (setf info var-14)))))
  (wntus
   (cond
    (wntvn
     (cond
      ((>= lwork
           (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                             (max
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-mul 4 n))
                              (the f2cl-lib:integer4 bdspac))))
        (setf ir 1)
        (cond
         ((>= lwork
              (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda n)))
          (setf ldwrkr lda))
         (t (setf ldwrkr n)))
        (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
        (setf iwork (f2cl-lib:int-add itau n))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (dgeqrf m n a lda
             (f2cl-lib:array-slice work-%data% double-float (itau)
```

```
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                          ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% double-float (ir)
                          ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice work-%data% double-float
                          ((+ ir 1)) ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorgqr m n n a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                          ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd n n
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "Q" n n n
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" n 0 n 0 s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     dum 1
```

```
      (f2cl-lib:array-slice work-%data% double-float (ir)
                            ((1 *)) work-%offset%)
      ldwrkr dum 1
      (f2cl-lib:array-slice work-%data% double-float (iwork)
                            ((1 *)) work-%offset%)
      info)
    (declare
     (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
      var-11 var-12 var-13))
    (setf n var-1)
    (setf n var-3)
    (setf ldwrkr var-10)
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "N" "N" m n n one a lda
       (f2cl-lib:array-slice work-%data% double-float (ir)
                            ((1 *)) work-%offset%)
      ldwrkr zero u ldu)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldwrkr var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldu var-12))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% double-float (itau)
                            ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwork)
                            ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
```

```
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorgqr m n n u ldu
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice a-%data% double-float (2 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd n n a lda s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
```

```
      (setf ierr var-10))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13)
        (dormbr "Q" "R" "N" m n n a lda
         (f2cl-lib:array-slice work-%data% double-float (itauq)
                               ((1 *)) work-%offset%)
         u ldu
         (f2cl-lib:array-slice work-%data% double-float (iwork)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
         ierr)
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
        var-12))
      (when var-3 (setf m var-3))
      (when var-4 (setf n var-4))
      (when var-5 (setf n var-5))
      (when var-7 (setf lda var-7))
      (when var-10 (setf ldu var-10))
      (when var-13 (setf ierr var-13)))
    (setf iwork (f2cl-lib:int-add ie n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14)
        (dbdsqr "U" n 0 m 0 s
         (f2cl-lib:array-slice work-%data% double-float (ie)
                               ((1 *)) work-%offset%)
         dum 1 u ldu dum 1
         (f2cl-lib:array-slice work-%data% double-float (iwork)
                               ((1 *)) work-%offset%)
         info)
      (declare
       (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
        var-11 var-12 var-13))
      (setf n var-1)
      (setf m var-3)
      (setf ldu var-10)
      (setf info var-14)))))
(wntvo
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul 2 n n)
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-mul 4 n))
                          (the f2cl-lib:integer4 bdspac))))
    (setf iu 1)
    (cond
     ((>= lwork
```

```
      (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 lda n)))
  (setf ldwrku lda)
  (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
  (setf ldwrkr lda))
 ((>= lwork
      (f2cl-lib:int-add wrkbl
                        (f2cl-lib:int-mul
                         (f2cl-lib:int-add lda n) n)))
  (setf ldwrku lda)
  (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
  (setf ldwrkr n))
 (t (setf ldwrku n)
  (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
  (setf ldwrkr n)))
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
(setf iwork (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dgeqrf m n a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ iu 1)) ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
```

```
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorgqr m n n a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd n n
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "U" n n
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% double-float (ir)
```

```
                          ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "Q" n n n
     (f2cl-lib:array-slice work-%data% double-float (iu)
                          ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                          ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "P" n n n
     (f2cl-lib:array-slice work-%data% double-float (ir)
                          ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                          ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" n n n 0 s
```

```
      (f2cl-lib:array-slice work-%data% double-float (ie)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (ir)
                            ((1 *)) work-%offset%)
      ldwrkr
      (f2cl-lib:array-slice work-%data% double-float (iu)
                            ((1 *)) work-%offset%)
      ldwrku dum 1
      (f2cl-lib:array-slice work-%data% double-float (iwork)
                            ((1 *)) work-%offset%)
      info)
    (declare
     (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
      var-13))
    (setf n var-1)
    (setf n var-2)
    (setf n var-3)
    (setf ldwrkr var-8)
    (setf ldwrku var-10)
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "N" "N" m n n one a lda
       (f2cl-lib:array-slice work-%data% double-float (iu)
                            ((1 *)) work-%offset%)
      ldwrku zero u ldu)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf lda var-7))
    (when var-9 (setf ldwrku var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldu var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "F" n n
       (f2cl-lib:array-slice work-%data% double-float (ir)
                            ((1 *)) work-%offset%)
      ldwrkr a lda)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldwrkr var-4))
    (when var-6 (setf lda var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
```

```
    (dgeqrf m n a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorgqr m n n u ldu
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice a-%data% double-float (2 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd n n a lda s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (dormbr "Q" "R" "N" m n n a lda
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "P" n n n a lda
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
```

```
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf n var-3))
      (when var-5 (setf lda var-5))
      (when var-9 (setf ierr var-9)))
    (setf iwork (f2cl-lib:int-add ie n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14)
        (dbdsqr "U" n n m 0 s
         (f2cl-lib:array-slice work-%data% double-float (ie)
                               ((1 *)) work-%offset%)
         a lda u ldu dum 1
         (f2cl-lib:array-slice work-%data% double-float (iwork)
                               ((1 *)) work-%offset%)
         info)
      (declare
       (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
        var-13))
      (setf n var-1)
      (setf n var-2)
      (setf m var-3)
      (setf lda var-8)
      (setf ldu var-10)
      (setf info var-14)))))
  (wntvas
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-mul 4 n))
                            (the f2cl-lib:integer4 bdspac))))
    (setf iu 1)
    (cond
     ((>= lwork
          (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda n)))
      (setf ldwrku lda))
     (t (setf ldwrku n)))
    (setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
    (setf iwork (f2cl-lib:int-add itau n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (dgeqrf m n a lda
         (f2cl-lib:array-slice work-%data% double-float (itau)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (iwork)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
         ierr)
```

```
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ iu 1)) ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorgqr m n n a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd n n
     (f2cl-lib:array-slice work-%data% double-float (iu)
```

```
                          ((1 *)) work-%offset%)
      ldwrku s
      (f2cl-lib:array-slice work-%data% double-float (ie)
                          ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (itauq)
                          ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (itaup)
                          ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (iwork)
                          ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "U" n n
     (f2cl-lib:array-slice work-%data% double-float (iu)
                          ((1 *)) work-%offset%)
     ldwrku vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "Q" n n n
     (f2cl-lib:array-slice work-%data% double-float (iu)
                          ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                          ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
     var-9)
    (dorgbr "P" n n n vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" n n n 0 s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku dum 1
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf n var-1)
  (setf n var-2)
  (setf n var-3)
  (setf ldvt var-8)
  (setf ldwrku var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (dgemm "N" "N" m n n one a lda
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku zero u ldu)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf lda var-7))
```

```
    (when var-9 (setf ldwrku var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldu var-12))))
  (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (dgeqrf m n a lda
        (f2cl-lib:array-slice work-%data% double-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (dorgqr m n n u ldu
        (f2cl-lib:array-slice work-%data% double-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf m var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldu var-4))
    (when var-8 (setf ierr var-8)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlacpy "U" n n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
   (if (> n 1)
```

```
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "L" (f2cl-lib:int-sub n 1)
         (f2cl-lib:int-sub n 1) zero zero
         (f2cl-lib:array-slice vt-%data% double-float (2 1)
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldvt var-6))))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd n n vt ldvt s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldvt var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (dormbr "Q" "R" "N" m n n vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
```

```
          (when var-5 (setf n var-5))
          (when var-7 (setf ldvt var-7))
          (when var-10 (setf ldu var-10))
          (when var-13 (setf ierr var-13)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9)
            (dorgbr "P" n n n vt ldvt
             (f2cl-lib:array-slice work-%data% double-float (itaup)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:array-slice work-%data% double-float (iwork)
                                   ((1 *)) work-%offset%)
             (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
             ierr)
          (declare (ignore var-0 var-4 var-6 var-7 var-8))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-3 (setf n var-3))
          (when var-5 (setf ldvt var-5))
          (when var-9 (setf ierr var-9)))
        (setf iwork (f2cl-lib:int-add ie n))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12 var-13 var-14)
            (dbdsqr "U" n n m 0 s
             (f2cl-lib:array-slice work-%data% double-float (ie)
                                   ((1 *)) work-%offset%)
             vt ldvt u ldu dum 1
             (f2cl-lib:array-slice work-%data% double-float (iwork)
                                   ((1 *)) work-%offset%)
             info)
          (declare
           (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
            var-13))
          (setf n var-1)
          (setf n var-2)
          (setf m var-3)
          (setf ldvt var-8)
          (setf ldu var-10)
          (setf info var-14)))))))
  (wntua
   (cond
    (wntvn
     (cond
      ((>= lwork
           (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                             (max
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add n m))
                              (the f2cl-lib:integer4
```

```
                            (f2cl-lib:int-mul 4 n))
                        (the f2cl-lib:integer4 bdspac))))
(setf ir 1)
(cond
 ((>= lwork
      (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda n)))
  (setf ldwrkr lda))
 (t (setf ldwrkr n)))
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
(setf iwork (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dgeqrf m n a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ ir 1)) ((1 *)) work-%offset%)
     ldwrkr)
```

```
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorgqr m m n u ldu
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd n n
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "Q" n n n
     (f2cl-lib:array-slice work-%data% double-float (ir)
```

```
                          ((1 *)) work-%offset%)
      ldwrkr
      (f2cl-lib:array-slice work-%data% double-float (itauq)
                          ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (iwork)
                          ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" n 0 n 0 s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                          ((1 *)) work-%offset%)
     dum 1
     (f2cl-lib:array-slice work-%data% double-float (ir)
                          ((1 *)) work-%offset%)
     ldwrkr dum 1
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                          ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
    var-11 var-12 var-13))
  (setf n var-1)
  (setf n var-3)
  (setf ldwrkr var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (dgemm "N" "N" m n n one u ldu
     (f2cl-lib:array-slice work-%data% double-float (ir)
                          ((1 *)) work-%offset%)
     ldwrkr zero a lda)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldu var-7))
  (when var-9 (setf ldwrkr var-9))
  (when var-10 (setf zero var-10))
```

```
    (when var-12 (setf lda var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "F" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% double-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "L" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dorgqr m m n u ldu
       (f2cl-lib:array-slice work-%data% double-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf m var-0))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldu var-4))
    (when var-8 (setf ierr var-8)))
  (setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
  (setf itaup (f2cl-lib:int-add itauq n))
  (setf iwork (f2cl-lib:int-add itaup n))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice a-%data% double-float (2 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd n n a lda s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (dormbr "Q" "R" "N" m n n a lda
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
```

```
      (when var-13 (setf ierr var-13)))
    (setf iwork (f2cl-lib:int-add ie n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14)
        (dbdsqr "U" n 0 m 0 s
         (f2cl-lib:array-slice work-%data% double-float (ie)
                               ((1 *)) work-%offset%)
         dum 1 u ldu dum 1
         (f2cl-lib:array-slice work-%data% double-float (iwork)
                               ((1 *)) work-%offset%)
         info)
      (declare
       (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9
        var-11 var-12 var-13))
      (setf n var-1)
      (setf m var-3)
      (setf ldu var-10)
      (setf info var-14)))))
(wntvo
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul 2 n n)
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add n m))
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-mul 4 n))
                          (the f2cl-lib:integer4 bdspac))))
    (setf iu 1)
    (cond
     ((>= lwork
          (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 lda n)))
      (setf ldwrku lda)
      (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
      (setf ldwrkr lda))
     ((>= lwork
          (f2cl-lib:int-add wrkbl
                            (f2cl-lib:int-mul
                             (f2cl-lib:int-add lda n) n)))
      (setf ldwrku lda)
      (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
      (setf ldwrkr n))
     (t (setf ldwrku n)
      (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
      (setf ldwrkr n)))
    (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr n)))
    (setf iwork (f2cl-lib:int-add itau n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
```

```
    (dgeqrf m n a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorgqr m m n u ldu
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
```

```
      (f2cl-lib:array-slice work-%data% double-float
                            ((+ iu 1)) ((1 *)) work-%offset%)
      ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrku var-6)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd n n
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "U" n n
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
```

```
    (dorgbr "Q" n n n
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "P" n n n
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" n n n 0 s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku dum 1
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
```

```
      info)
    (declare
     (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
      var-13))
    (setf n var-1)
    (setf n var-2)
    (setf n var-3)
    (setf ldwrkr var-8)
    (setf ldwrku var-10)
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "N" "N" m n n one u ldu
       (f2cl-lib:array-slice work-%data% double-float (iu)
                             ((1 *)) work-%offset%)
       ldwrku zero a lda)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf n var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldu var-7))
    (when var-9 (setf ldwrku var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf lda var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "F" m n a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "F" n n
       (f2cl-lib:array-slice work-%data% double-float (ir)
                             ((1 *)) work-%offset%)
       ldwrkr a lda)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf ldwrkr var-4))
    (when var-6 (setf lda var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dgeqrf m n a lda
       (f2cl-lib:array-slice work-%data% double-float (itau)
```

```
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                          ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorgqr m m n u ldu
     (f2cl-lib:array-slice work-%data% double-float (itau)
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                          ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice a-%data% double-float (2 1)
                          ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
```

```
    (dgebrd n n a lda s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (dormbr "Q" "R" "N" m n n a lda
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "P" n n n a lda
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
```

```
      (when var-3 (setf n var-3))
      (when var-5 (setf lda var-5))
      (when var-9 (setf ierr var-9)))
    (setf iwork (f2cl-lib:int-add ie n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14)
        (dbdsqr "U" n n m 0 s
         (f2cl-lib:array-slice work-%data% double-float (ie)
                               ((1 *)) work-%offset%)
         a lda u ldu dum 1
         (f2cl-lib:array-slice work-%data% double-float (iwork)
                               ((1 *)) work-%offset%)
         info)
      (declare
       (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
        var-13))
      (setf n var-1)
      (setf n var-2)
      (setf m var-3)
      (setf lda var-8)
      (setf ldu var-10)
      (setf info var-14)))))
  (wntvas
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul n n)
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add n m))
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-mul 4 n))
                            (the f2cl-lib:integer4 bdspac))))
    (setf iu 1)
    (cond
     ((>= lwork
          (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda n)))
      (setf ldwrku lda))
     (t (setf ldwrku n)))
    (setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku n)))
    (setf iwork (f2cl-lib:int-add itau n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (dgeqrf m n a lda
         (f2cl-lib:array-slice work-%data% double-float (itau)
                               ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (iwork)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
         ierr)
```

```
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m n a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorgqr m m n u ldu
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "U" n n a lda
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "L" (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
     zero zero
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ iu 1)) ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrku var-6)))
```

```
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd n n
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "U" n n
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "Q" n n n
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
```

```
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "P" n n n vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" n n n 0 s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku dum 1
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf n var-1)
  (setf n var-2)
  (setf n var-3)
  (setf ldvt var-8)
  (setf ldwrku var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (dgemm "N" "N" m n n one u ldu
     (f2cl-lib:array-slice work-%data% double-float (iu)
```

```
                                ((1 *)) work-%offset%)
        ldwrku zero a lda)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf n var-3))
      (when var-4 (setf n var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf ldu var-7))
      (when var-9 (setf ldwrku var-9))
      (when var-10 (setf zero var-10))
      (when var-12 (setf lda var-12)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "F" m n a lda u ldu)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-6 (setf ldu var-6))))
  (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau n))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (dgeqrf m n a lda
         (f2cl-lib:array-slice work-%data% double-float (itau)
                                ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (iwork)
                                ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
         ierr)
      (declare (ignore var-2 var-4 var-5 var-6))
      (setf m var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf ierr var-7))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "L" m n a lda u ldu)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-6 (setf ldu var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dorgqr m m n u ldu
         (f2cl-lib:array-slice work-%data% double-float (itau)
                                ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (iwork)
                                ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
```

```
      ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf ldu var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "U" n n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(if (> n 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "L" (f2cl-lib:int-sub n 1)
         (f2cl-lib:int-sub n 1) zero zero
         (f2cl-lib:array-slice vt-%data% double-float (2 1)
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt)
      (declare (ignore var-0 var-1 var-2 var-5))
      (when var-3 (setf zero var-3))
      (when var-4 (setf zero var-4))
      (when var-6 (setf ldvt var-6))))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie n))
(setf itaup (f2cl-lib:int-add itauq n))
(setf iwork (f2cl-lib:int-add itaup n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd n n vt ldvt s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf n var-0)
  (setf n var-1)
  (setf ldvt var-3)
  (setf ierr var-10))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (dormbr "Q" "R" "N" m n n vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     u ldu
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf n var-5))
  (when var-7 (setf ldvt var-7))
  (when var-10 (setf ldu var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "P" n n n vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf n var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf n var-3))
  (when var-5 (setf ldvt var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" n n m 0 s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     vt ldvt u ldu dum 1
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
```

```
          (setf n var-1)
          (setf n var-2)
          (setf m var-3)
          (setf ldvt var-8)
          (setf ldu var-10)
          (setf info var-14)))))))))
(t (setf ie 1) (setf itauq (f2cl-lib:int-add ie n))
 (setf itaup (f2cl-lib:int-add itauq n))
 (setf iwork (f2cl-lib:int-add itaup n))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (dgebrd m n a lda s
      (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                            work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
   (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-10))
 (cond
  (wntuas
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlacpy "L" m n a lda u ldu)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf m var-1))
     (when var-2 (setf n var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf ldu var-6)))
   (if wntus
       (setf ncu n))
   (if wntua
       (setf ncu m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (dorgbr "Q" m ncu n u ldu
        (f2cl-lib:array-slice work-%data% double-float (itauq)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
     (declare (ignore var-0 var-4 var-6 var-7 var-8))
     (when var-1 (setf m var-1))
```

```
    (when var-2 (setf ncu var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf ldu var-5))
    (when var-9 (setf ierr var-9)))))
(cond
 (wntvas
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "U" n n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dorgbr "P" n n n vt ldvt
       (f2cl-lib:array-slice work-%data% double-float (itaup)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf ldvt var-5))
    (when var-9 (setf ierr var-9)))))
(cond
 (wntuo
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dorgbr "Q" m n n a lda
       (f2cl-lib:array-slice work-%data% double-float (itauq)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))))
(cond
 (wntvo
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dorgbr "P" n n n a lda
       (f2cl-lib:array-slice work-%data% double-float (itaup)
                             ((1 *)) work-%offset%)
```

```
        (f2cl-lib:array-slice work-%data% double-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))))
(setf iwork (f2cl-lib:int-add ie n))
(if (or wntuas wntuo)
    (setf nru m))
(if wntun
    (setf nru 0))
(if (or wntvas wntvo)
    (setf ncvt n))
(if wntvn
    (setf ncvt 0))
(cond
 ((and (not wntuo) (not wntvo))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (dbdsqr "U" n ncvt nru 0 s
       (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                             work-%offset%)
       vt ldvt u ldu dum 1
       (f2cl-lib:array-slice work-%data% double-float (iwork)
                             ((1 *)) work-%offset%)
       info)
    (declare
     (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
      var-13))
    (setf n var-1)
    (setf ncvt var-2)
    (setf nru var-3)
    (setf ldvt var-8)
    (setf ldu var-10)
    (setf info var-14)))
 ((and (not wntuo) wntvo)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (dbdsqr "U" n ncvt nru 0 s
       (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                             work-%offset%)
       a lda u ldu dum 1
       (f2cl-lib:array-slice work-%data% double-float (iwork)
                             ((1 *)) work-%offset%)
       info)
```

```
          (declare
           (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
            var-13))
          (setf n var-1)
          (setf ncvt var-2)
          (setf nru var-3)
          (setf lda var-8)
          (setf ldu var-10)
          (setf info var-14)))
      (t
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10 var-11 var-12 var-13 var-14)
           (dbdsqr "U" n ncvt nru 0 s
            (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                                  work-%offset%)
            vt ldvt a lda dum 1
            (f2cl-lib:array-slice work-%data% double-float (iwork)
                                  ((1 *)) work-%offset%)
            info)
          (declare
           (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
            var-13))
          (setf n var-1)
          (setf ncvt var-2)
          (setf nru var-3)
          (setf ldvt var-8)
          (setf lda var-10)
          (setf info var-14)))))))
(t
 (cond
  ((>= n mnthr)
   (cond
    (wntvn (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (dgelqf m n a lda
          (f2cl-lib:array-slice work-%data% double-float (itau)
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% double-float (iwork)
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
         (declare (ignore var-2 var-4 var-5 var-6))
         (setf m var-0)
         (setf n var-1)
         (setf lda var-3)
         (setf ierr var-7))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
          zero zero
```

```
     (f2cl-lib:array-slice a-%data% double-float (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(setf ie 1) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgebrd m m a lda s
     (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(cond
 ((or wntuo wntuas)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9)
      (dorgbr "Q" m m m a lda
       (f2cl-lib:array-slice work-%data% double-float (itauq)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))))
(setf iwork (f2cl-lib:int-add ie m)) (setf nru 0)
(if (or wntuo wntuas)
    (setf nru m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
      var-10 var-11 var-12 var-13 var-14)
     (dbdsqr "U" m 0 nru 0 s
      (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                            work-%offset%)
      dum 1 a lda dum 1
      (f2cl-lib:array-slice work-%data% double-float (iwork)
                            ((1 *)) work-%offset%)
      info)
    (declare
     (ignore var-0 var-2 var-4 var-5 var-6 var-7 var-8 var-9 var-11
      var-12 var-13))
    (setf m var-1)
    (setf nru var-3)
    (setf lda var-10)
    (setf info var-14))
  (if wntuas
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlacpy "F" m m a lda u ldu)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf m var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldu var-6)))))
((and wntvo wntun)
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-mul 4 m))
                          (the f2cl-lib:integer4 bdspac))))
    (setf ir 1)
    (cond
     ((>= lwork
          (f2cl-lib:int-add
           (max (the f2cl-lib:integer4 wrkbl)
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul lda n)
                                       m)))
           (f2cl-lib:int-mul lda m)))
      (setf ldwrku lda) (setf chunk n) (setf ldwrkr lda))
     ((>= lwork
          (f2cl-lib:int-add
           (max (the f2cl-lib:integer4 wrkbl)
                (the f2cl-lib:integer4
                     (f2cl-lib:int-add (f2cl-lib:int-mul lda n)
                                       m)))
           (f2cl-lib:int-mul m m)))
      (setf ldwrku lda) (setf chunk n) (setf ldwrkr m))
```

```
 (t (setf ldwrku m)
  (setf chunk
          (the f2cl-lib:integer4
               (truncate (- lwork (* m m) m) m)))
  (setf ldwrkr m)))
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
(setf iwork (f2cl-lib:int-add itau m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dgelqf m n a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ ir ldwrkr)) ((1 *))
                           work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorglq m n m a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
```

```
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd m m
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "P" m m m
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" m m 0 0 s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr dum 1 dum 1
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
    var-11 var-12 var-13))
  (setf m var-1)
  (setf m var-2)
  (setf ldwrkr var-8)
  (setf info var-14))
(setf iu (f2cl-lib:int-add ie m))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
              ((> i n) nil)
  (tagbody
    (setf blk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
             (the f2cl-lib:integer4 chunk)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (dgemm "N" "N" m blk m one
         (f2cl-lib:array-slice work-%data% double-float (ir)
                               ((1 *)) work-%offset%)
         ldwrkr
         (f2cl-lib:array-slice a-%data% double-float (1 i)
                               ((1 lda) (1 *)) a-%offset%)
         lda zero
         (f2cl-lib:array-slice work-%data% double-float (iu)
                               ((1 *)) work-%offset%)
         ldwrku)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf blk var-3))
      (when var-4 (setf m var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf ldwrkr var-7))
      (when var-9 (setf lda var-9))
      (when var-10 (setf zero var-10))
      (when var-12 (setf ldwrku var-12)))
```

```
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlacpy "F" m blk
           (f2cl-lib:array-slice work-%data% double-float (iu)
                                 ((1 *)) work-%offset%)
           ldwrku
           (f2cl-lib:array-slice a-%data% double-float (1 i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf blk var-2))
        (when var-4 (setf ldwrku var-4))
        (when var-6 (setf lda var-6)))
     label30)))
  (t (setf ie 1) (setf itauq (f2cl-lib:int-add ie m))
   (setf itaup (f2cl-lib:int-add itauq m))
   (setf iwork (f2cl-lib:int-add itaup m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (dgebrd m n a lda s
        (f2cl-lib:array-slice work-%data% double-float (ie)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (itauq)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (itaup)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-10))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9)
       (dorgbr "P" m n m a lda
        (f2cl-lib:array-slice work-%data% double-float (itaup)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
     (declare (ignore var-0 var-4 var-6 var-7 var-8))
     (when var-1 (setf m var-1))
     (when var-2 (setf n var-2))
     (when var-3 (setf m var-3))
     (when var-5 (setf lda var-5))
```

```
      (when var-9 (setf ierr var-9)))
    (setf iwork (f2cl-lib:int-add ie m))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14)
        (dbdsqr "L" m n 0 0 s
         (f2cl-lib:array-slice work-%data% double-float (ie)
                               ((1 *)) work-%offset%)
         a lda dum 1 dum 1
         (f2cl-lib:array-slice work-%data% double-float (iwork)
                               ((1 *)) work-%offset%)
         info)
      (declare
       (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
        var-11 var-12 var-13))
      (setf m var-1)
      (setf n var-2)
      (setf lda var-8)
      (setf info var-14)))))
((and wntvo wntuas)
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-mul 4 m))
                          (the f2cl-lib:integer4 bdspac))))
   (setf ir 1)
   (cond
    ((>= lwork
         (f2cl-lib:int-add
          (max (the f2cl-lib:integer4 wrkbl)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul lda n)
                                      m)))
          (f2cl-lib:int-mul lda m)))
     (setf ldwrku lda) (setf chunk n) (setf ldwrkr lda))
    ((>= lwork
         (f2cl-lib:int-add
          (max (the f2cl-lib:integer4 wrkbl)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-mul lda n)
                                      m)))
          (f2cl-lib:int-mul m m)))
     (setf ldwrku lda) (setf chunk n) (setf ldwrkr m))
    (t (setf ldwrku m)
     (setf chunk
             (the f2cl-lib:integer4
                  (truncate (- lwork (* m m) m) m)))
     (setf ldwrkr m)))
```

```
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
(setf iwork (f2cl-lib:int-add itau m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dgelqf m n a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m m a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice u-%data% double-float (1 2)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorglq m n m a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
```

```
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd m m u ldu s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldu var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "U" m m u ldu
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldu var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "P" m m m
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
     var-9)
    (dorgbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" m m m 0 s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr u ldu dum 1
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf m var-1)
  (setf m var-2)
  (setf m var-3)
  (setf ldwrkr var-8)
  (setf ldu var-10)
  (setf info var-14))
(setf iu (f2cl-lib:int-add ie m))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i chunk))
              ((> i n) nil)
  (tagbody
    (setf blk
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
             (the f2cl-lib:integer4 chunk)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (dgemm "N" "N" m blk m one
         (f2cl-lib:array-slice work-%data% double-float (ir)
                               ((1 *)) work-%offset%)
```

```
            ldwrkr
            (f2cl-lib:array-slice a-%data% double-float (1 i)
                                  ((1 lda) (1 *)) a-%offset%)
            lda zero
            (f2cl-lib:array-slice work-%data% double-float (iu)
                                  ((1 *)) work-%offset%)
            ldwrku)
          (declare (ignore var-0 var-1 var-6 var-8 var-11))
          (when var-2 (setf m var-2))
          (when var-3 (setf blk var-3))
          (when var-4 (setf m var-4))
          (when var-5 (setf one var-5))
          (when var-7 (setf ldwrkr var-7))
          (when var-9 (setf lda var-9))
          (when var-10 (setf zero var-10))
          (when var-12 (setf ldwrku var-12)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dlacpy "F" m blk
             (f2cl-lib:array-slice work-%data% double-float (iu)
                                   ((1 *)) work-%offset%)
             ldwrku
             (f2cl-lib:array-slice a-%data% double-float (1 i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf m var-1))
          (when var-2 (setf blk var-2))
          (when var-4 (setf ldwrku var-4))
          (when var-6 (setf lda var-6)))
      label40)))
  (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (dgelqf m n a lda
        (f2cl-lib:array-slice work-%data% double-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-7))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlacpy "L" m m a lda u ldu)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf m var-1))
```

```
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice u-%data% double-float (1 2)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorglq m n m a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd m m u ldu s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldu var-3)
  (setf ierr var-10))
(multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (dormbr "P" "L" "T" m n m u ldu
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     a lda
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11 var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldu var-7))
  (when var-10 (setf lda var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" m n m 0 s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     a lda u ldu dum 1
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf m var-1)
  (setf n var-2)
  (setf m var-3)
  (setf lda var-8)
```

```
      (setf ldu var-10)
      (setf info var-14)))))
 (wntvs
  (cond
   (wntun
    (cond
     ((>= lwork
          (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                            (max
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-mul 4 m))
                             (the f2cl-lib:integer4 bdspac))))
      (setf ir 1)
      (cond
       ((>= lwork
            (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda m)))
        (setf ldwrkr lda))
       (t (setf ldwrkr m)))
      (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
      (setf iwork (f2cl-lib:int-add itau m))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (dgelqf m n a lda
           (f2cl-lib:array-slice work-%data% double-float (itau)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:array-slice work-%data% double-float (iwork)
                                 ((1 *)) work-%offset%)
           (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
           ierr)
        (declare (ignore var-2 var-4 var-5 var-6))
        (setf m var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf ierr var-7))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlacpy "L" m m a lda
           (f2cl-lib:array-slice work-%data% double-float (ir)
                                 ((1 *)) work-%offset%)
           ldwrkr)
        (declare (ignore var-0 var-3 var-5))
        (when var-1 (setf m var-1))
        (when var-2 (setf m var-2))
        (when var-4 (setf lda var-4))
        (when var-6 (setf ldwrkr var-6)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
           zero zero
           (f2cl-lib:array-slice work-%data% double-float
```

```
                              ((+ ir ldwrkr)) ((1 *))
                              work-%offset%)
      ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorglq m n m a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd m m
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
     var-9)
    (dorgbr "P" m m m
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" m m 0 0 s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr dum 1 dum 1
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
    var-11 var-12 var-13))
  (setf m var-1)
  (setf m var-2)
  (setf ldwrkr var-8)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (dgemm "N" "N" m n m one
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr a lda zero vt ldvt)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldwrkr var-7))
```

```
    (when var-9 (setf lda var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldvt var-12))))
  (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (dgelqf m n a lda
        (f2cl-lib:array-slice work-%data% double-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (dorglq m n m vt ldvt
        (f2cl-lib:array-slice work-%data% double-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf m var-0))
    (when var-1 (setf n var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf ldvt var-4))
    (when var-8 (setf ierr var-8)))
   (setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
   (setf itaup (f2cl-lib:int-add itauq m))
   (setf iwork (f2cl-lib:int-add itaup m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
        zero zero
        (f2cl-lib:array-slice a-%data% double-float (1 2)
                              ((1 lda) (1 *)) a-%offset%)
```

```
      lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd m m a lda s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (dormbr "P" "L" "T" m n m a lda
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" m n 0 0 s
```

```
          (f2cl-lib:array-slice work-%data% double-float (ie)
                                ((1 *)) work-%offset%)
          vt ldvt dum 1 dum 1
          (f2cl-lib:array-slice work-%data% double-float (iwork)
                                ((1 *)) work-%offset%)
          info)
        (declare
         (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
          var-11 var-12 var-13))
        (setf m var-1)
        (setf n var-2)
        (setf ldvt var-8)
        (setf info var-14)))))
  (wntuo
   (cond
    ((>= lwork
         (f2cl-lib:int-add (f2cl-lib:int-mul 2 m m)
                           (max
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-mul 4 m))
                            (the f2cl-lib:integer4 bdspac))))
     (setf iu 1)
     (cond
      ((>= lwork
           (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 lda m)))
       (setf ldwrku lda)
       (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
       (setf ldwrkr lda))
      ((>= lwork
           (f2cl-lib:int-add wrkbl
                             (f2cl-lib:int-mul
                              (f2cl-lib:int-add lda m) m)))
       (setf ldwrku lda)
       (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
       (setf ldwrkr m))
      (t (setf ldwrku m)
       (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
       (setf ldwrkr m)))
     (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
     (setf iwork (f2cl-lib:int-add itau m))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (dgelqf m n a lda
          (f2cl-lib:array-slice work-%data% double-float (itau)
                                ((1 *)) work-%offset%)
          (f2cl-lib:array-slice work-%data% double-float (iwork)
                                ((1 *)) work-%offset%)
          (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
          ierr)
       (declare (ignore var-2 var-4 var-5 var-6))
```

```
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ iu ldwrku)) ((1 *))
                           work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorglq m n m a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd m m
     (f2cl-lib:array-slice work-%data% double-float (iu)
```

```
                              ((1 *)) work-%offset%)
      ldwrku s
      (f2cl-lib:array-slice work-%data% double-float (ie)
                              ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (itauq)
                              ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (itaup)
                              ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (iwork)
                              ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m m
     (f2cl-lib:array-slice work-%data% double-float (iu)
                              ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% double-float (ir)
                              ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "P" m m m
     (f2cl-lib:array-slice work-%data% double-float (iu)
                              ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                              ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                              ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrku var-5))
```

```
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "Q" m m m
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" m m m 0 s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr dum 1
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf m var-1)
  (setf m var-2)
  (setf m var-3)
  (setf ldwrku var-8)
  (setf ldwrkr var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (dgemm "N" "N" m n m one
     (f2cl-lib:array-slice work-%data% double-float (iu)
```

```
                              ((1 *)) work-%offset%)
        ldwrku a lda zero vt ldvt)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf m var-2))
      (when var-3 (setf n var-3))
      (when var-4 (setf m var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf ldwrku var-7))
      (when var-9 (setf lda var-9))
      (when var-10 (setf zero var-10))
      (when var-12 (setf ldvt var-12)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "F" m m
         (f2cl-lib:array-slice work-%data% double-float (ir)
                              ((1 *)) work-%offset%)
        ldwrkr a lda)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf m var-2))
      (when var-4 (setf ldwrkr var-4))
      (when var-6 (setf lda var-6))))
  (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (dgelqf m n a lda
         (f2cl-lib:array-slice work-%data% double-float (itau)
                              ((1 *)) work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (iwork)
                              ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
         ierr)
      (declare (ignore var-2 var-4 var-5 var-6))
      (setf m var-0)
      (setf n var-1)
      (setf lda var-3)
      (setf ierr var-7))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "U" m n a lda vt ldvt)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf m var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf lda var-4))
      (when var-6 (setf ldvt var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dorglq m n m vt ldvt
         (f2cl-lib:array-slice work-%data% double-float (itau)
                              ((1 *)) work-%offset%)
```

```
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice a-%data% double-float (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd m m a lda s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (dormbr "P" "L" "T" m n m a lda
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
```

```
      vt ldvt
      (f2cl-lib:array-slice work-%data% double-float (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf lda var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "Q" m m m a lda
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                            ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                            ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf lda var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" m n m 0 s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                            ((1 *)) work-%offset%)
     vt ldvt a lda dum 1
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                            ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf m var-1)
  (setf n var-2)
  (setf m var-3)
  (setf ldvt var-8)
  (setf lda var-10)
  (setf info var-14)))))
```

```
(wntuas
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-mul 4 m))
                          (the f2cl-lib:integer4 bdspac))))
   (setf iu 1)
   (cond
    ((>= lwork
         (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda m)))
     (setf ldwrku lda))
    (t (setf ldwrku m)))
   (setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
   (setf iwork (f2cl-lib:int-add itau m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (dgelqf m n a lda
        (f2cl-lib:array-slice work-%data% double-float (itau)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (iwork)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
        ierr)
     (declare (ignore var-2 var-4 var-5 var-6))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
     (setf ierr var-7))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlacpy "L" m m a lda
        (f2cl-lib:array-slice work-%data% double-float (iu)
                              ((1 *)) work-%offset%)
        ldwrku)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf m var-1))
     (when var-2 (setf m var-2))
     (when var-4 (setf lda var-4))
     (when var-6 (setf ldwrku var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
        zero zero
        (f2cl-lib:array-slice work-%data% double-float
                              ((+ iu ldwrku)) ((1 *))
                              work-%offset%)
        ldwrku)
     (declare (ignore var-0 var-1 var-2 var-5))
```

```
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorglq m n m a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd m m
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m m
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku u ldu)
```

```
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "P" m m m
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" m m m 0 s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku u ldu dum 1
```

```
      (f2cl-lib:array-slice work-%data% double-float (iwork)
                            ((1 *)) work-%offset%)
      info)
    (declare
     (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
      var-13))
    (setf m var-1)
    (setf m var-2)
    (setf m var-3)
    (setf ldwrku var-8)
    (setf ldu var-10)
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "N" "N" m n m one
       (f2cl-lib:array-slice work-%data% double-float (iu)
                            ((1 *)) work-%offset%)
      ldwrku a lda zero vt ldvt)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldwrku var-7))
    (when var-9 (setf lda var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldvt var-12))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dgelqf m n a lda
       (f2cl-lib:array-slice work-%data% double-float (itau)
                            ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwork)
                            ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
```

```
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorglq m n m vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf m var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m m a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice u-%data% double-float (1 2)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldu var-6)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd m m u ldu s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
```

```
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldu var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (dormbr "P" "L" "T" m n m u ldu
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldu var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" m n m 0 s
     (f2cl-lib:array-slice work-%data% double-float (ie)
```

```
                                  ((1 *)) work-%offset%)
                vt ldvt u ldu dum 1
                (f2cl-lib:array-slice work-%data% double-float (iwork)
                                  ((1 *)) work-%offset%)
                info)
              (declare
               (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
                var-13))
              (setf m var-1)
              (setf n var-2)
              (setf m var-3)
              (setf ldvt var-8)
              (setf ldu var-10)
              (setf info var-14)))))))
      (wntva
       (cond
        (wntun
         (cond
          ((>= lwork
               (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                                 (max
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-add n m))
                                  (the f2cl-lib:integer4
                                       (f2cl-lib:int-mul 4 m))
                                  (the f2cl-lib:integer4 bdspac))))
           (setf ir 1)
           (cond
            ((>= lwork
                 (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda m)))
             (setf ldwrkr lda))
            (t (setf ldwrkr m)))
           (setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
           (setf iwork (f2cl-lib:int-add itau m))
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
               (dgelqf m n a lda
                (f2cl-lib:array-slice work-%data% double-float (itau)
                                  ((1 *)) work-%offset%)
                (f2cl-lib:array-slice work-%data% double-float (iwork)
                                  ((1 *)) work-%offset%)
                (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
                ierr)
             (declare (ignore var-2 var-4 var-5 var-6))
             (setf m var-0)
             (setf n var-1)
             (setf lda var-3)
             (setf ierr var-7))
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
    (dlacpy "U" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ ir ldwrkr)) ((1 *))
                           work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorglq n n m vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
```

```
    (dgebrd m m
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrkr var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "P" m m m
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" m m 0 0 s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr dum 1 dum 1
     (f2cl-lib:array-slice work-%data% double-float (iwork)
```

```
                              ((1 *)) work-%offset%)
      info)
    (declare
     (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
      var-11 var-12 var-13))
    (setf m var-1)
    (setf m var-2)
    (setf ldwrkr var-8)
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "N" "N" m n m one
       (f2cl-lib:array-slice work-%data% double-float (ir)
                              ((1 *)) work-%offset%)
       ldwrkr vt ldvt zero a lda)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldwrkr var-7))
    (when var-9 (setf ldvt var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf lda var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "F" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dgelqf m n a lda
       (f2cl-lib:array-slice work-%data% double-float (itau)
                              ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwork)
                              ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
    (dlacpy "U" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorglq n n m vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice a-%data% double-float (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd m m a lda s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
```

```
        (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
      (setf m var-0)
      (setf m var-1)
      (setf lda var-3)
      (setf ierr var-10))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13)
        (dormbr "P" "L" "T" m n m a lda
         (f2cl-lib:array-slice work-%data% double-float (itaup)
                               ((1 *)) work-%offset%)
         vt ldvt
         (f2cl-lib:array-slice work-%data% double-float (iwork)
                               ((1 *)) work-%offset%)
         (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
         ierr)
      (declare
       (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
        var-12))
      (when var-3 (setf m var-3))
      (when var-4 (setf n var-4))
      (when var-5 (setf m var-5))
      (when var-7 (setf lda var-7))
      (when var-10 (setf ldvt var-10))
      (when var-13 (setf ierr var-13)))
    (setf iwork (f2cl-lib:int-add ie m))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14)
        (dbdsqr "U" m n 0 0 s
         (f2cl-lib:array-slice work-%data% double-float (ie)
                               ((1 *)) work-%offset%)
         vt ldvt dum 1 dum 1
         (f2cl-lib:array-slice work-%data% double-float (iwork)
                               ((1 *)) work-%offset%)
         info)
      (declare
       (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-9 var-10
        var-11 var-12 var-13))
      (setf m var-1)
      (setf n var-2)
      (setf ldvt var-8)
      (setf info var-14)))))
(wntuo
 (cond
  ((>= lwork
       (f2cl-lib:int-add (f2cl-lib:int-mul 2 m m)
                         (max
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-add n m))
```

```
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-mul 4 m))
                        (the f2cl-lib:integer4 bdspac))))
(setf iu 1)
(cond
 ((>= lwork
      (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul 2 lda m)))
  (setf ldwrku lda)
  (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
  (setf ldwrkr lda))
 ((>= lwork
      (f2cl-lib:int-add wrkbl
                        (f2cl-lib:int-mul
                         (f2cl-lib:int-add lda m) m)))
  (setf ldwrku lda)
  (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
  (setf ldwrkr m))
 (t (setf ldwrku m)
  (setf ir (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
  (setf ldwrkr m)))
(setf itau (f2cl-lib:int-add ir (f2cl-lib:int-mul ldwrkr m)))
(setf iwork (f2cl-lib:int-add itau m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dgelqf m n a lda
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "U" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorglq n n m vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (itau)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
```

```
                          ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% double-float (iu)
                          ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice work-%data% double-float
                          ((+ iu ldwrku)) ((1 *))
                          work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrku var-6)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd m m
     (f2cl-lib:array-slice work-%data% double-float (iu)
                          ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                          ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                          ((1 *)) work-%offset%)
```

```
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m m
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldwrkr var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "P" m m m
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "Q" m m m
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
```

```
      (f2cl-lib:array-slice work-%data% double-float (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrkr var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14)
    (dbdsqr "U" m m m 0 s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% double-float (ir)
                           ((1 *)) work-%offset%)
     ldwrkr dum 1
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     info)
  (declare
   (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
    var-13))
  (setf m var-1)
  (setf m var-2)
  (setf m var-3)
  (setf ldwrku var-8)
  (setf ldwrkr var-10)
  (setf info var-14))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12)
    (dgemm "N" "N" m n m one
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku vt ldvt zero a lda)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf m var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf m var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldwrku var-7))
  (when var-9 (setf ldvt var-9))
  (when var-10 (setf zero var-10))
```

```
    (when var-12 (setf lda var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "F" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "F" m m
       (f2cl-lib:array-slice work-%data% double-float (ir)
                             ((1 *)) work-%offset%)
       ldwrkr a lda)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf ldwrkr var-4))
    (when var-6 (setf lda var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dgelqf m n a lda
       (f2cl-lib:array-slice work-%data% double-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
       ierr)
    (declare (ignore var-2 var-4 var-5 var-6))
    (setf m var-0)
    (setf n var-1)
    (setf lda var-3)
    (setf ierr var-7))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (dorglq n n m vt ldvt
       (f2cl-lib:array-slice work-%data% double-float (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwork)
                             ((1 *)) work-%offset%)
```

```
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice a-%data% double-float (1 2)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf lda var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd m m a lda s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf lda var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (dormbr "P" "L" "T" m n m a lda
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (iwork)
```

```
                                ((1 *)) work-%offset%)
                 (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
                 ierr)
               (declare
                (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
                 var-12))
               (when var-3 (setf m var-3))
               (when var-4 (setf n var-4))
               (when var-5 (setf m var-5))
               (when var-7 (setf lda var-7))
               (when var-10 (setf ldvt var-10))
               (when var-13 (setf ierr var-13)))
             (multiple-value-bind
                 (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                  var-9)
                 (dorgbr "Q" m m m a lda
                  (f2cl-lib:array-slice work-%data% double-float (itauq)
                                ((1 *)) work-%offset%)
                  (f2cl-lib:array-slice work-%data% double-float (iwork)
                                ((1 *)) work-%offset%)
                  (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
                  ierr)
               (declare (ignore var-0 var-4 var-6 var-7 var-8))
               (when var-1 (setf m var-1))
               (when var-2 (setf m var-2))
               (when var-3 (setf m var-3))
               (when var-5 (setf lda var-5))
               (when var-9 (setf ierr var-9)))
             (setf iwork (f2cl-lib:int-add ie m))
             (multiple-value-bind
                 (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                  var-9 var-10 var-11 var-12 var-13 var-14)
                 (dbdsqr "U" m n m 0 s
                  (f2cl-lib:array-slice work-%data% double-float (ie)
                                ((1 *)) work-%offset%)
                  vt ldvt a lda dum 1
                  (f2cl-lib:array-slice work-%data% double-float (iwork)
                                ((1 *)) work-%offset%)
                  info)
               (declare
                (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
                 var-13))
               (setf m var-1)
               (setf n var-2)
               (setf m var-3)
               (setf ldvt var-8)
               (setf lda var-10)
               (setf info var-14)))))
          (wntuas
           (cond
```

```
((>= lwork
     (f2cl-lib:int-add (f2cl-lib:int-mul m m)
                       (max
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add n m))
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-mul 4 m))
                        (the f2cl-lib:integer4 bdspac))))
 (setf iu 1)
 (cond
  ((>= lwork
       (f2cl-lib:int-add wrkbl (f2cl-lib:int-mul lda m)))
   (setf ldwrku lda))
  (t (setf ldwrku m)))
 (setf itau (f2cl-lib:int-add iu (f2cl-lib:int-mul ldwrku m)))
 (setf iwork (f2cl-lib:int-add itau m))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (dgelqf m n a lda
      (f2cl-lib:array-slice work-%data% double-float (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
   (declare (ignore var-2 var-4 var-5 var-6))
   (setf m var-0)
   (setf n var-1)
   (setf lda var-3)
   (setf ierr var-7))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (dlacpy "U" m n a lda vt ldvt)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf m var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldvt var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (dorglq n n m vt ldvt
      (f2cl-lib:array-slice work-%data% double-float (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% double-float (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
   (declare (ignore var-3 var-5 var-6 var-7))
   (when var-0 (setf n var-0))
   (when var-1 (setf n var-1))
```

```
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m m a lda
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldwrku var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ iu ldwrku)) ((1 *))
                           work-%offset%)
     ldwrku)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwrku var-6)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd m m
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
```

```
  (setf ldwrku var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m m
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldwrku var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "P" m m m
     (f2cl-lib:array-slice work-%data% double-float (iu)
                           ((1 *)) work-%offset%)
     ldwrku
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldwrku var-5))
  (when var-9 (setf ierr var-9)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-0 var-4 var-6 var-7 var-8))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-3 (setf m var-3))
  (when var-5 (setf ldu var-5))
  (when var-9 (setf ierr var-9)))
(setf iwork (f2cl-lib:int-add ie m))
(multiple-value-bind
```

```
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12 var-13 var-14)
      (dbdsqr "U" m m m 0 s
       (f2cl-lib:array-slice work-%data% double-float (ie)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iu)
                             ((1 *)) work-%offset%)
       ldwrku u ldu dum 1
       (f2cl-lib:array-slice work-%data% double-float (iwork)
                             ((1 *)) work-%offset%)
       info)
    (declare
     (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
      var-13))
    (setf m var-1)
    (setf m var-2)
    (setf m var-3)
    (setf ldwrku var-8)
    (setf ldu var-10)
    (setf info var-14))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "N" "N" m n m one
       (f2cl-lib:array-slice work-%data% double-float (iu)
                             ((1 *)) work-%offset%)
       ldwrku vt ldvt zero a lda)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-4 (setf m var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldwrku var-7))
    (when var-9 (setf ldvt var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf lda var-12)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "F" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6))))
 (t (setf itau 1) (setf iwork (f2cl-lib:int-add itau m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dgelqf m n a lda
       (f2cl-lib:array-slice work-%data% double-float (itau)
                             ((1 *)) work-%offset%)
```

```
      (f2cl-lib:array-slice work-%data% double-float (iwork)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
      ierr)
  (declare (ignore var-2 var-4 var-5 var-6))
  (setf m var-0)
  (setf n var-1)
  (setf lda var-3)
  (setf ierr var-7))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "U" m n a lda vt ldvt)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf n var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldvt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (dorglq n n m vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (itau)
                            ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                            ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf n var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf ldvt var-4))
  (when var-8 (setf ierr var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlacpy "L" m m a lda u ldu)
  (declare (ignore var-0 var-3 var-5))
  (when var-1 (setf m var-1))
  (when var-2 (setf m var-2))
  (when var-4 (setf lda var-4))
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "U" (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
     zero zero
     (f2cl-lib:array-slice u-%data% double-float (1 2)
                            ((1 ldu) (1 *)) u-%offset%)
     ldu)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
```

```
  (when var-6 (setf ldu var-6)))
(setf ie itau) (setf itauq (f2cl-lib:int-add ie m))
(setf itaup (f2cl-lib:int-add itauq m))
(setf iwork (f2cl-lib:int-add itaup m))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgebrd m m u ldu s
     (f2cl-lib:array-slice work-%data% double-float (ie)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
  (setf m var-0)
  (setf m var-1)
  (setf ldu var-3)
  (setf ierr var-10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13)
    (dormbr "P" "L" "T" m n m u ldu
     (f2cl-lib:array-slice work-%data% double-float (itaup)
                           ((1 *)) work-%offset%)
     vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (iwork)
                           ((1 *)) work-%offset%)
     (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
     ierr)
  (declare
   (ignore var-0 var-1 var-2 var-6 var-8 var-9 var-11
    var-12))
  (when var-3 (setf m var-3))
  (when var-4 (setf n var-4))
  (when var-5 (setf m var-5))
  (when var-7 (setf ldu var-7))
  (when var-10 (setf ldvt var-10))
  (when var-13 (setf ierr var-13)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9)
    (dorgbr "Q" m m m u ldu
     (f2cl-lib:array-slice work-%data% double-float (itauq)
                           ((1 *)) work-%offset%)
```

```
              (f2cl-lib:array-slice work-%data% double-float (iwork)
                                    ((1 *)) work-%offset%)
              (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1)
              ierr)
            (declare (ignore var-0 var-4 var-6 var-7 var-8))
            (when var-1 (setf m var-1))
            (when var-2 (setf m var-2))
            (when var-3 (setf m var-3))
            (when var-5 (setf ldu var-5))
            (when var-9 (setf ierr var-9)))
          (setf iwork (f2cl-lib:int-add ie m))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10 var-11 var-12 var-13 var-14)
              (dbdsqr "U" m n m 0 s
               (f2cl-lib:array-slice work-%data% double-float (ie)
                                     ((1 *)) work-%offset%)
               vt ldvt u ldu dum 1
               (f2cl-lib:array-slice work-%data% double-float (iwork)
                                     ((1 *)) work-%offset%)
               info)
            (declare
             (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
              var-13))
            (setf m var-1)
            (setf n var-2)
            (setf m var-3)
            (setf ldvt var-8)
            (setf ldu var-10)
            (setf info var-14)))))))))
  (t (setf ie 1) (setf itauq (f2cl-lib:int-add ie m))
   (setf itaup (f2cl-lib:int-add itauq m))
   (setf iwork (f2cl-lib:int-add itaup m))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (dgebrd m n a lda s
        (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (itauq) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (itaup) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (iwork) ((1 *))
                              work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
     (declare (ignore var-2 var-4 var-5 var-6 var-7 var-8 var-9))
     (setf m var-0)
     (setf n var-1)
     (setf lda var-3)
```

```
  (setf ierr var-10))
(cond
 (wntuas
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "L" m m a lda u ldu)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dorgbr "Q" m m n u ldu
       (f2cl-lib:array-slice work-%data% double-float (itauq)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf ldu var-5))
    (when var-9 (setf ierr var-9)))))
(cond
 (wntvas
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacpy "U" m n a lda vt ldvt)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvt var-6)))
  (if wntva
      (setf nrvt n))
  (if wntvs
      (setf nrvt m))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dorgbr "P" nrvt n m vt ldvt
       (f2cl-lib:array-slice work-%data% double-float (itaup)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf nrvt var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf ldvt var-5))
```

```
    (when var-9 (setf ierr var-9)))))
(cond
 (wntuo
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dorgbr "Q" m m n a lda
       (f2cl-lib:array-slice work-%data% double-float (itauq)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf m var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))))
(cond
 (wntvo
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (dorgbr "P" m n m a lda
       (f2cl-lib:array-slice work-%data% double-float (itaup)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iwork)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwork) 1) ierr)
    (declare (ignore var-0 var-4 var-6 var-7 var-8))
    (when var-1 (setf m var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf m var-3))
    (when var-5 (setf lda var-5))
    (when var-9 (setf ierr var-9)))))
(setf iwork (f2cl-lib:int-add ie m))
(if (or wntuas wntuo)
    (setf nru m))
(if wntun
    (setf nru 0))
(if (or wntvas wntvo)
    (setf ncvt n))
(if wntvn
    (setf ncvt 0))
(cond
 ((and (not wntuo) (not wntvo))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (dbdsqr "L" m ncvt nru 0 s
       (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                             work-%offset%)
```

```
      vt ldvt u ldu dum 1
      (f2cl-lib:array-slice work-%data% double-float (iwork)
                            ((1 *)) work-%offset%)
      info)
   (declare
    (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
     var-13))
   (setf m var-1)
   (setf ncvt var-2)
   (setf nru var-3)
   (setf ldvt var-8)
   (setf ldu var-10)
   (setf info var-14)))
((and (not wntuo) wntvo)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13 var-14)
     (dbdsqr "L" m ncvt nru 0 s
      (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                            work-%offset%)
      a lda u ldu dum 1
      (f2cl-lib:array-slice work-%data% double-float (iwork)
                            ((1 *)) work-%offset%)
      info)
   (declare
    (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
     var-13))
   (setf m var-1)
   (setf ncvt var-2)
   (setf nru var-3)
   (setf lda var-8)
   (setf ldu var-10)
   (setf info var-14)))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12 var-13 var-14)
     (dbdsqr "L" m ncvt nru 0 s
      (f2cl-lib:array-slice work-%data% double-float (ie) ((1 *))
                            work-%offset%)
      vt ldvt a lda dum 1
      (f2cl-lib:array-slice work-%data% double-float (iwork)
                            ((1 *)) work-%offset%)
      info)
   (declare
    (ignore var-0 var-4 var-5 var-6 var-7 var-9 var-11 var-12
     var-13))
   (setf m var-1)
   (setf ncvt var-2)
   (setf nru var-3)
```

```
        (setf ldvt var-8)
        (setf lda var-10)
        (setf info var-14)))))))
(cond
 ((/= info 0)
  (cond
   ((> ie 2)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add minmn (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1))
                             ((1 *)) work-%offset%)
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add i ie) 1))
                               ((1 *)) work-%offset%))
       label50))))
  (cond
   ((< ie 2)
    (f2cl-lib:fdo (i (f2cl-lib:int-add minmn (f2cl-lib:int-sub 1))
                   (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                  ((> i 1) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i 1))
                             ((1 *)) work-%offset%)
                (f2cl-lib:fref work-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-add i ie) 1))
                               ((1 *)) work-%offset%))
       label60))))))
(cond
 ((= iscl 1)
  (if (> anrm bignum)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (dlascl "G" 0 0 bignum anrm minmn 1 s minmn ierr)
        (declare (ignore var-0 var-1 var-2 var-6 var-7))
        (when var-3 (setf bignum var-3))
        (when var-4 (setf anrm var-4))
        (when var-5 (setf minmn var-5))
        (when var-8 (setf minmn var-8))
        (when var-9 (setf ierr var-9))))
  (if (and (/= info 0) (> anrm bignum))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (dlascl "G" 0 0 bignum anrm (f2cl-lib:int-sub minmn 1) 1
           (f2cl-lib:array-slice work-%data% double-float (2) ((1 *))
                                 work-%offset%)
           minmn ierr)
```

```
              (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
              (when var-3 (setf bignum var-3))
              (when var-4 (setf anrm var-4))
              (when var-8 (setf minmn var-8))
              (when var-9 (setf ierr var-9))))
          (if (< anrm smlnum)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (dlascl "G" 0 0 smlnum anrm minmn 1 s minmn ierr)
              (declare (ignore var-0 var-1 var-2 var-6 var-7))
              (when var-3 (setf smlnum var-3))
              (when var-4 (setf anrm var-4))
              (when var-5 (setf minmn var-5))
              (when var-8 (setf minmn var-8))
              (when var-9 (setf ierr var-9))))
          (if (and (/= info 0) (< anrm smlnum))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                  (dlascl "G" 0 0 smlnum anrm (f2cl-lib:int-sub minmn 1) 1
                   (f2cl-lib:array-slice work-%data% double-float (2) ((1 *))
                                         work-%offset%)
                   minmn ierr)
              (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
              (when var-3 (setf smlnum var-3))
              (when var-4 (setf anrm var-4))
              (when var-8 (setf minmn var-8))
              (when var-9 (setf ierr var-9))))))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 maxwrk) 'double-float))
      (go end_label)
     end_label
      (return
       (values jobu jobvt m n nil lda nil nil ldu nil ldvt nil nil info))))))
```

———-

90.130 dgesv

90.130.1 Axiom unit tests

— **dgesv.input** —

```
)set break resume
)sys rm -f dgesv.output
)spool dgesv.output
)set message test on
```

```
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.130.2 Axiom help page

— **dgesv.help** —

```
=====================================================================
dgesv examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

NAME
       DGESV - the solution to a real system of linear equations  A * X = B,

SYNOPSIS
       SUBROUTINE DGESV( N, NRHS, A, LDA, IPIV, B, LDB, INFO )

           INTEGER       INFO, LDA, LDB, N, NRHS

           INTEGER       IPIV( * )

           DOUBLE        PRECISION A( LDA, * ), B( LDB, * )

PURPOSE
       DGESV computes the solution to a real system of linear equations
          A  *  X  =  B, where A is an N-by-N matrix and X and B are N-by-NRHS
       matrices.

       The LU decomposition with partial pivoting and row interchanges is used
       to factor A as
          A = P * L * U,
       where  P  is a permutation matrix, L is unit lower triangular, and U is
       upper triangular.  The factored form of A is then  used  to  solve  the
       system of equations A * X = B.


ARGUMENTS
       N       (input) INTEGER
               The  number  of linear equations, i.e., the order of the matrix
```

```
          A.  N >= 0.

NRHS      (input) INTEGER
          The number of right hand sides, i.e., the number of columns  of
          the matrix B.  NRHS >= 0.

A         (input/output) DOUBLE PRECISION array, dimension (LDA,N)
          On  entry,  the N-by-N coefficient matrix A.  On exit, the fac-
          tors L and U from the factorization A = P*L*U; the unit  diago-
          nal elements of L are not stored.

LDA       (input) INTEGER
          The leading dimension of the array A.  LDA >= max(1,N).

IPIV      (output) INTEGER array, dimension (N)
          The  pivot  indices that define the permutation matrix P; row i
          of the matrix was interchanged with row IPIV(i).

B         (input/output) DOUBLE PRECISION array, dimension (LDB,NRHS)
          On entry, the N-by-NRHS matrix of right hand side matrix B.  On
          exit, if INFO = 0, the N-by-NRHS solution matrix X.

LDB       (input) INTEGER
          The leading dimension of the array B.  LDB >= max(1,N).

INFO      (output) INTEGER
          = 0:  successful exit
          < 0:  if INFO = -i, the i-th argument had an illegal value
          >  0:   if INFO = i, U(i,i) is exactly zero.  The factorization
          has been completed, but the factor U is  exactly  singular,  so
          the solution could not be computed.
```

———-

90.130.3 fortran code

— **dgesv.f** —

```
*  =====================================================================
      SUBROUTINE DGESV( N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
```

```
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. External Subroutines ..
      EXTERNAL           DGETRF, DGETRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGESV ', -INFO )
         RETURN
      END IF
*
*     Compute the LU factorization of A.
*
      CALL DGETRF( N, N, A, LDA, IPIV, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
         CALL DGETRS( 'No transpose', N, NRHS, A, LDA, IPIV, B, LDB,
     $                INFO )
      END IF
      RETURN
*
*     End of DGESV
*
      END
      SUBROUTINE DGESV( N, NRHS, A, LDA, IPIV, B, LDB, INFO )
```

```
*
*  -- LAPACK driver routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     March 31, 1993
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. External Subroutines ..
      EXTERNAL           DGETRF, DGETRS, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGESV ', -INFO )
         RETURN
      END IF
*
*     Compute the LU factorization of A.
*
      CALL DGETRF( N, N, A, LDA, IPIV, INFO )
      IF( INFO.EQ.0 ) THEN
*
*        Solve the system A*X = B, overwriting B with X.
*
         CALL DGETRS( 'No transpose', N, NRHS, A, LDA, IPIV, B, LDB,
     $                INFO )
```

```
      END IF
      RETURN
*
*     End of DGESV
*
      END
```

———-

90.130.4 lisp code

— **dgesv.lisp** —

```
(defun dgesv (n nrhs a lda ipiv b ldb$ info)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array double-float (*)) b a)
           (type (f2cl-lib:integer4) info ldb$ lda nrhs n))
  (f2cl-lib:with-multi-array-data
      ((a double-float a-%data% a-%offset%)
       (b double-float b-%data% b-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ()
      (declare)
      (setf info 0)
      (cond ((< n 0) (setf info -1)) ((< nrhs 0) (setf info -2))
            ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -4))
            ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (setf info -7)))
      (cond
       ((/= info 0) (xerbla "DGESV " (f2cl-lib:int-sub info)) (go end_label)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (dgetrf n n a lda ipiv info)
        (declare (ignore var-2 var-4))
        (setf n var-0)
        (setf n var-1)
        (setf lda var-3)
        (setf info var-5))
      (cond
       ((= info 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (dgetrs "No transpose" n nrhs a lda ipiv b ldb$ info)
          (declare (ignore var-0 var-3 var-5 var-6))
          (setf n var-1)
          (setf nrhs var-2)
          (setf lda var-4)
```

```
          (setf ldb$ var-7)
          (setf info var-8))))
      (go end_label)
     end_label
      (return (values n nrhs nil lda nil nil ldb$ info)))))
```

———-

90.131 dgetf2

90.131.1 Axiom unit tests

— **dgetf2.input** —

```
)set break resume
)sys rm -f dgetf2.output
)spool dgetf2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.131.2 Axiom help page

— **dgetf2.help** —

```
====================================================================
dgetf2 examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DGETF2 - an LU factorization of a general m-by-n matrix A using partial
       pivoting with row interchanges

SYNOPSIS
```

```
SUBROUTINE DGETF2( M, N, A, LDA, IPIV, INFO )

    INTEGER        INFO, LDA, M, N

    INTEGER        IPIV( * )

    DOUBLE         PRECISION A( LDA, * )
```

PURPOSE

DGETF2 computes an LU factorization of a general m-by-n matrix A using partial pivoting with row interchanges.

The factorization has the form
   A = P * L * U
where P is a permutation matrix, L is lower triangular with unit diagonal elements (lower trapezoidal if m > n), and U is upper triangular (upper trapezoidal if m < n).

This is the right-looking Level 2 BLAS version of the algorithm.

ARGUMENTS

M (input) INTEGER
The number of rows of the matrix A. M >= 0.

N (input) INTEGER
The number of columns of the matrix A. N >= 0.

A (input/output) DOUBLE PRECISION array, dimension (LDA,N)
On entry, the m by n matrix to be factored. On exit, the factors L and U from the factorization A = P*L*U; the unit diagonal elements of L are not stored.

LDA (input) INTEGER
The leading dimension of the array A. LDA >= max(1,M).

IPIV (output) INTEGER array, dimension (min(M,N))
The pivot indices; for 1 <= i <= min(M,N), row i of the matrix was interchanged with row IPIV(i).

INFO (output) INTEGER
= 0: successful exit
< 0: if INFO = -k, the k-th argument had an illegal value
> 0: if INFO = k, U(k,k) is exactly zero. The factorization has been completed, but the factor U is exactly singular, and division by zero will occur if it is used to solve a system of equations.

———-

90.131.3 fortran code

— dgetf2.f —

```
*  =====================================================================
      SUBROUTINE DGETF2( M, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION   SFMIN
      INTEGER            I, J, JP
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      INTEGER            IDAMAX
      EXTERNAL           DLAMCH, IDAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGER, DSCAL, DSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
```

```
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGETF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Compute machine safe minimum
*
      SFMIN = DLAMCH('S')
*
      DO 10 J = 1, MIN( M, N )
*
*        Find pivot and test for singularity.
*
         JP = J - 1 + IDAMAX( M-J+1, A( J, J ), 1 )
         IPIV( J ) = JP
         IF( A( JP, J ).NE.ZERO ) THEN
*
*           Apply the interchange to columns 1:N.
*
            IF( JP.NE.J )
     $         CALL DSWAP( N, A( J, 1 ), LDA, A( JP, 1 ), LDA )
*
*           Compute elements J+1:M of J-th column.
*
            IF( J.LT.M ) THEN
               IF( ABS(A( J, J )) .GE. SFMIN ) THEN
                  CALL DSCAL( M-J, ONE / A( J, J ), A( J+1, J ), 1 )
               ELSE
                 DO 20 I = 1, M-J
                    A( J+I, J ) = A( J+I, J ) / A( J, J )
   20            CONTINUE
               END IF
            END IF
*
         ELSE IF( INFO.EQ.0 ) THEN
*
            INFO = J
         END IF
*
         IF( J.LT.MIN( M, N ) ) THEN
*
```

```
*          Update trailing submatrix.
*
           CALL DGER( M-J, N-J, -ONE, A( J+1, J ), 1, A( J, J+1 ), LDA,
     $                 A( J+1, J+1 ), LDA )
        END IF
   10 CONTINUE
      RETURN
*
*     End of DGETF2
*
      END
      SUBROUTINE DGETF2( M, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1992
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            J, JP
*     ..
*     .. External Functions ..
      INTEGER            IDAMAX
      EXTERNAL           IDAMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGER, DSCAL, DSWAP, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
```

```
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGETF2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
      DO 10 J = 1, MIN( M, N )
*
*        Find pivot and test for singularity.
*
         JP = J - 1 + IDAMAX( M-J+1, A( J, J ), 1 )
         IPIV( J ) = JP
         IF( A( JP, J ).NE.ZERO ) THEN
*
*           Apply the interchange to columns 1:N.
*
            IF( JP.NE.J )
     $         CALL DSWAP( N, A( J, 1 ), LDA, A( JP, 1 ), LDA )
*
*           Compute elements J+1:M of J-th column.
*
            IF( J.LT.M )
     $         CALL DSCAL( M-J, ONE / A( J, J ), A( J+1, J ), 1 )
*
         ELSE IF( INFO.EQ.0 ) THEN
*
            INFO = J
         END IF
*
         IF( J.LT.MIN( M, N ) ) THEN
*
*           Update trailing submatrix.
*
            CALL DGER( M-J, N-J, -ONE, A( J+1, J ), 1, A( J, J+1 ), LDA,
     $                 A( J+1, J+1 ), LDA )
         END IF
   10 CONTINUE
      RETURN
*
```

```
*     End of DGETF2
*
      END
```

———-

90.131.4 lisp code

— **dgetf2.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dgetf2 (m n a lda ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array double-float (*)) a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((i 0) (j 0) (jp 0) (sfmin 0.0d0))
        (declare (type (f2cl-lib:integer4) i j jp)
                 (type (double-float) sfmin))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "DGETF2" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (or (= m 0) (= n 0))
            (go end_label))
        (setf sfmin (dlamch "S"))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j
                          (min (the f2cl-lib:integer4 m)
                               (the f2cl-lib:integer4 n)))
                       nil)
          (tagbody
            (setf jp
                    (f2cl-lib:int-add (f2cl-lib:int-sub j 1)
                                      (idamax
                                       (f2cl-lib:int-add (f2cl-lib:int-sub m j)
                                                         1)
                                       (f2cl-lib:array-slice a-%data%
```

```
                                                    double-float (j j)
                                                    ((1 lda) (1 *))
                                                    a-%offset%)
                              1)))
        (setf (f2cl-lib:fref ipiv-%data% (j) ((1 *)) ipiv-%offset%) jp)
        (cond
         ((/= (f2cl-lib:fref a (jp j) ((1 lda) (1 *))) zero)
          (if (/= jp j)
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (dswap n
                   (f2cl-lib:array-slice a-%data% double-float (j 1)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda
                   (f2cl-lib:array-slice a-%data% double-float (jp 1)
                                         ((1 lda) (1 *)) a-%offset%)
                   lda)
                (declare (ignore var-1 var-3))
                (when var-0 (setf n var-0))
                (when var-2 (setf lda var-2))
                (when var-4 (setf lda var-4))))
          (cond
           ((< j m)
            (cond
             ((>= (abs (f2cl-lib:fref a (j j) ((1 lda) (1 *)))) sfmin)
              (dscal (f2cl-lib:int-sub m j)
               (/ one
                  (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                 a-%offset%))
               (f2cl-lib:array-slice a-%data% double-float ((+ j 1) j)
                                     ((1 lda) (1 *)) a-%offset%)
               1))
             (t
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i
                                (f2cl-lib:int-add m (f2cl-lib:int-sub j)))
                             nil)
                (tagbody
                  (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add j i) j)
                                       ((1 lda) (1 *)) a-%offset%)
                        (/
                         (f2cl-lib:fref a-%data%
                                        ((f2cl-lib:int-add j i) j)
                                        ((1 lda) (1 *)) a-%offset%)
                         (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                        a-%offset%)))
                  label20)))))))
         ((= info 0) (setf info j)))
        (cond
         ((< j (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
          (multiple-value-bind
```

```
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (dger (f2cl-lib:int-sub m j) (f2cl-lib:int-sub n j) (- one)
                 (f2cl-lib:array-slice a-%data% double-float ((+ j 1) j)
                                       ((1 lda) (1 *)) a-%offset%)
                 1
                 (f2cl-lib:array-slice a-%data% double-float
                                       (j (f2cl-lib:int-add j 1))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda
                 (f2cl-lib:array-slice a-%data% double-float
                                       ((+ j 1) (f2cl-lib:int-add j 1))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
               (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7))
               (when var-6 (setf lda var-6))
               (when var-8 (setf lda var-8)))))
           label10))
         (go end_label)
        end_label
         (return (values nil n nil lda nil info))))))
```

———-

90.132 dgetrf

90.132.1 Axiom unit tests

— **dgetrf.input** —

```
)set break resume
)sys rm -f dgetrf.output
)spool dgetrf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.132.2 Axiom help page

— **dgetrf.help** —

```
=====================================================================
dgetrf examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

NAME
       DGETRF - an LU factorization of a general M-by-N matrix A using partial
       pivoting with row interchanges

SYNOPSIS
       SUBROUTINE DGETRF( M, N, A, LDA, IPIV, INFO )

           INTEGER        INFO, LDA, M, N

           INTEGER        IPIV( * )

           DOUBLE         PRECISION A( LDA, * )

PURPOSE
       DGETRF computes an LU factorization of a general M-by-N matrix A  using
       partial pivoting with row interchanges.

       The factorization has the form
          A = P * L * U
       where P is a permutation matrix, L is lower triangular with unit diago-
       nal elements (lower trapezoidal if m > n), and U  is  upper  triangular
       (upper trapezoidal if m < n).

       This is the right-looking Level 3 BLAS version of the algorithm.


ARGUMENTS
       M       (input) INTEGER
               The number of rows of the matrix A.  M >= 0.

       N       (input) INTEGER
               The number of columns of the matrix A.  N >= 0.

       A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
               On  entry, the M-by-N matrix to be factored.  On exit, the fac-
               tors L and U from the factorization A = P*L*U; the unit  diago-
               nal elements of L are not stored.

       LDA     (input) INTEGER
               The leading dimension of the array A.  LDA >= max(1,M).

       IPIV    (output) INTEGER array, dimension (min(M,N))
```

```
              The  pivot indices; for 1 <= i <= min(M,N), row i of the matrix
              was interchanged with row IPIV(i).

       INFO    (output) INTEGER
               = 0:  successful exit
               < 0:  if INFO = -i, the i-th argument had an illegal value
               > 0:  if INFO = i, U(i,i) is exactly  zero.  The  factorization
               has  been  completed, but the factor U is exactly singular, and
               division by zero will occur if it is used to solve a system  of
               equations.
```

———-

90.132.3 fortran code

— **dgetrf.f** —

```
*  =====================================================================
      SUBROUTINE DGETRF( M, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IINFO, J, JB, NB
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEMM, DGETRF2, DLASWP, DTRSM, XERBLA
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
```

```
      EXTERNAL           ILAENV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGETRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Determine the block size for this environment.
*
      NB = ILAENV( 1, 'DGETRF', ' ', M, N, -1, -1 )
      IF( NB.LE.1 .OR. NB.GE.MIN( M, N ) ) THEN
*
*        Use unblocked code.
*
         CALL DGETRF2( M, N, A, LDA, IPIV, INFO )
      ELSE
*
*        Use blocked code.
*
         DO 20 J = 1, MIN( M, N ), NB
            JB = MIN( MIN( M, N )-J+1, NB )
*
*           Factor diagonal and subdiagonal blocks and test for exact
*           singularity.
*
            CALL DGETRF2( M-J+1, JB, A( J, J ), LDA, IPIV( J ), IINFO )
*
*           Adjust INFO and the pivot indices.
*
            IF( INFO.EQ.0 .AND. IINFO.GT.0 )
```

```
     $         INFO = IINFO + J - 1
            DO 10 I = J, MIN( M, J+JB-1 )
               IPIV( I ) = J - 1 + IPIV( I )
   10       CONTINUE
*
*           Apply interchanges to columns 1:J-1.
*
            CALL DLASWP( J-1, A, LDA, J, J+JB-1, IPIV, 1 )
*
            IF( J+JB.LE.N ) THEN
*
*              Apply interchanges to columns J+JB:N.
*
               CALL DLASWP( N-J-JB+1, A( 1, J+JB ), LDA, J, J+JB-1,
     $                      IPIV, 1 )
*
*              Compute block row of U.
*
               CALL DTRSM( 'Left', 'Lower', 'No transpose', 'Unit', JB,
     $                     N-J-JB+1, ONE, A( J, J ), LDA, A( J, J+JB ),
     $                     LDA )
               IF( J+JB.LE.M ) THEN
*
*                 Update trailing submatrix.
*
                  CALL DGEMM( 'No transpose', 'No transpose', M-J-JB+1,
     $                        N-J-JB+1, JB, -ONE, A( J+JB, J ), LDA,
     $                        A( J, J+JB ), LDA, ONE, A( J+JB, J+JB ),
     $                        LDA )
               END IF
            END IF
   20    CONTINUE
      END IF
      RETURN
*
*     End of DGETRF
*
      END
      SUBROUTINE DGETRF( M, N, A, LDA, IPIV, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     March 31, 1993
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDA, M, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
```

```
      DOUBLE PRECISION   A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IINFO, J, JB, NB
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEMM, DGETF2, DLASWP, DTRSM, XERBLA
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGETRF', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
*     Determine the block size for this environment.
*
      NB = ILAENV( 1, 'DGETRF', ' ', M, N, -1, -1 )
      IF( NB.LE.1 .OR. NB.GE.MIN( M, N ) ) THEN
*
*        Use unblocked code.
```

```
*
         CALL DGETF2( M, N, A, LDA, IPIV, INFO )
      ELSE
*
*        Use blocked code.
*
         DO 20 J = 1, MIN( M, N ), NB
            JB = MIN( MIN( M, N )-J+1, NB )
*
*           Factor diagonal and subdiagonal blocks and test for exact
*           singularity.
*
            CALL DGETF2( M-J+1, JB, A( J, J ), LDA, IPIV( J ), IINFO )
*
*           Adjust INFO and the pivot indices.
*
            IF( INFO.EQ.0 .AND. IINFO.GT.0 )
     $         INFO = IINFO + J - 1
            DO 10 I = J, MIN( M, J+JB-1 )
               IPIV( I ) = J - 1 + IPIV( I )
   10       CONTINUE
*
*           Apply interchanges to columns 1:J-1.
*
            CALL DLASWP( J-1, A, LDA, J, J+JB-1, IPIV, 1 )
*
            IF( J+JB.LE.N ) THEN
*
*              Apply interchanges to columns J+JB:N.
*
               CALL DLASWP( N-J-JB+1, A( 1, J+JB ), LDA, J, J+JB-1,
     $                      IPIV, 1 )
*
*              Compute block row of U.
*
               CALL DTRSM( 'Left', 'Lower', 'No transpose', 'Unit', JB,
     $                     N-J-JB+1, ONE, A( J, J ), LDA, A( J, J+JB ),
     $                     LDA )
               IF( J+JB.LE.M ) THEN
*
*                 Update trailing submatrix.
*
                  CALL DGEMM( 'No transpose', 'No transpose', M-J-JB+1,
     $                        N-J-JB+1, JB, -ONE, A( J+JB, J ), LDA,
     $                        A( J, J+JB ), LDA, ONE, A( J+JB, J+JB ),
     $                        LDA )
               END IF
            END IF
   20    CONTINUE
      END IF
```

```
      RETURN
*
*     End of DGETRF
*
      END
```

———-

90.132.4 lisp code

— **dgetrf.lisp** —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dgetrf (m n a lda ipiv info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array double-float (*)) a)
             (type (f2cl-lib:integer4) info lda n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
      (prog ((i 0) (iinfo 0) (j 0) (jb 0) (nb 0))
        (declare (type (f2cl-lib:integer4) i iinfo j jb nb))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n 0) (setf info -2))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -4)))
        (cond
         ((/= info 0) (xerbla "DGETRF" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (or (= m 0) (= n 0))
            (go end_label))
        (setf nb
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (ilaenv 1 "DGETRF" " " m n -1 -1)
                  (declare (ignore var-0 var-1 var-2 var-5 var-6))
                  (when var-3 (setf m var-3))
                  (when var-4 (setf n var-4))
                  ret-val))
        (cond
         ((or (<= nb 1)
              (>= nb
                  (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n))))
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
```

```
      (dgetrf2 m n a lda ipiv info)
    (declare (ignore var-2 var-4))
    (when var-0 (setf m var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf lda var-3))
    (when var-5 (setf info var-5))))
  (t
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j nb))
                 ((> j
                     (min (the f2cl-lib:integer4 m)
                          (the f2cl-lib:integer4 n)))
                  nil)
    (tagbody
      (setf jb
              (min
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add
                     (f2cl-lib:int-sub
                      (min (the f2cl-lib:integer4 m)
                           (the f2cl-lib:integer4 n))
                      j)
                     1))
               (the f2cl-lib:integer4 nb)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
          (dgetrf2 (f2cl-lib:int-add (f2cl-lib:int-sub m j) 1) jb
           (f2cl-lib:array-slice a-%data% double-float (j j)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice ipiv-%data% f2cl-lib:integer4 (j)
                                 ((1 *)) ipiv-%offset%)
           iinfo)
        (declare (ignore var-0 var-2 var-4))
        (when var-1 (setf jb var-1))
        (when var-3 (setf lda var-3))
        (when var-5 (setf iinfo var-5)))
      (if (and (= info 0) (> iinfo 0))
          (setf info (f2cl-lib:int-sub (f2cl-lib:int-add iinfo j) 1)))
      (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                    ((> i
                        (min (the f2cl-lib:integer4 m)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add j jb
                                                    (f2cl-lib:int-sub
                                                     1)))))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref ipiv-%data% (i) ((1 *)) ipiv-%offset%)
                (f2cl-lib:int-add (f2cl-lib:int-sub j 1)
                                  (f2cl-lib:fref ipiv-%data% (i)
                                                 ((1 *))
```

```
                                                ipiv-%offset%)))
   label10))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaswp (f2cl-lib:int-sub j 1) a lda j
     (f2cl-lib:int-sub (f2cl-lib:int-add j jb) 1) ipiv 1)
  (declare (ignore var-0 var-1 var-4 var-5 var-6))
  (when var-2 (setf lda var-2))
  (when var-3 (setf j var-3)))
(cond
 ((<= (f2cl-lib:int-add j jb) n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaswp (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1)
       (f2cl-lib:array-slice a-%data% double-float
                             (1 (f2cl-lib:int-add j jb))
                             ((1 lda) (1 *)) a-%offset%)
       lda j (f2cl-lib:int-sub (f2cl-lib:int-add j jb) 1) ipiv 1)
    (declare (ignore var-0 var-1 var-4 var-5 var-6))
    (when var-2 (setf lda var-2))
    (when var-3 (setf j var-3)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (dtrsm "Left" "Lower" "No transpose" "Unit" jb
       (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) one
       (f2cl-lib:array-slice a-%data% double-float (j j)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice a-%data% double-float
                             (j (f2cl-lib:int-add j jb))
                             ((1 lda) (1 *)) a-%offset%)
       lda)
    (declare (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9))
    (when var-4 (setf jb var-4))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf lda var-10)))
  (cond
   ((<= (f2cl-lib:int-add j jb) m)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (dgemm "No transpose" "No transpose"
         (f2cl-lib:int-add (f2cl-lib:int-sub m j jb) 1)
         (f2cl-lib:int-add (f2cl-lib:int-sub n j jb) 1) jb
         (- one)
         (f2cl-lib:array-slice a-%data% double-float ((+ j jb) j)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice a-%data% double-float
```

```
                                        (j (f2cl-lib:int-add j jb))
                                        ((1 lda) (1 *)) a-%offset%)
                      lda one
                      (f2cl-lib:array-slice a-%data% double-float
                                        ((+ j jb) (f2cl-lib:int-add j jb))
                                        ((1 lda) (1 *)) a-%offset%)
                      lda)
                    (declare
                     (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8 var-11))
                    (when var-4 (setf jb var-4))
                    (when var-7 (setf lda var-7))
                    (when var-9 (setf lda var-9))
                    (when var-10 (setf one var-10))
                    (when var-12 (setf lda var-12)))))))
              label20))))
         (go end_label)
        end_label
         (return (values m n nil lda nil info))))))
```

———-

90.133 dgetrs

90.133.1 Axiom unit tests

— **dgetrs.input** —

```
)set break resume
)sys rm -f dgetrs.output
)spool dgetrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.133.2 Axiom help page

— **dgetrs.help** —

```
==========================================================================
```

```
dgetrs examples
===================================================================

===================================================================
Man Page Details
===================================================================

NAME
       DGETRS  -  a system of linear equations  A * X = B or A' * X = B with a
       general N-by-N matrix A using the LU factorization computed by DGETRF

SYNOPSIS
       SUBROUTINE DGETRS( TRANS, N, NRHS, A, LDA, IPIV, B, LDB, INFO )

           CHARACTER      TRANS

           INTEGER        INFO, LDA, LDB, N, NRHS

           INTEGER        IPIV( * )

           DOUBLE         PRECISION A( LDA, * ), B( LDB, * )

PURPOSE
       DGETRS solves a system of linear equations
          A * X = B  or  A' * X = B with a general N-by-N matrix A  using  the
       LU factorization computed by DGETRF.


ARGUMENTS
       TRANS   (input) CHARACTER*1
               Specifies the form of the system of equations:
               = 'N':  A * X = B  (No transpose)
               = 'T':  A'* X = B  (Transpose)
               = 'C':  A'* X = B  (Conjugate transpose = Transpose)

       N       (input) INTEGER
               The order of the matrix A.  N >= 0.

       NRHS    (input) INTEGER
               The  number of right hand sides, i.e., the number of columns of
               the matrix B.  NRHS >= 0.

       A       (input) DOUBLE PRECISION array, dimension (LDA,N)
               The factors L and U from the factorization A =  P*L*U  as  com-
               puted by DGETRF.

       LDA     (input) INTEGER
               The leading dimension of the array A.  LDA >= max(1,N).

       IPIV    (input) INTEGER array, dimension (N)
```

```
              The pivot indices from DGETRF; for 1<=i<=N, row i of the matrix
              was interchanged with row IPIV(i).

      B       (input/output) DOUBLE PRECISION array, dimension (LDB,NRHS)
              On entry, the right hand side matrix B.  On exit, the  solution
              matrix X.

      LDB     (input) INTEGER
              The leading dimension of the array B.  LDB >= max(1,N).

      INFO    (output) INTEGER
              = 0:  successful exit
              < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

90.133.3 fortran code

— dgetrs.f —

```
*  =====================================================================
      SUBROUTINE DGETRS( TRANS, N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
*     ..
*     .. External Functions ..
```

```
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASWP, DTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGETRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( NOTRAN ) THEN
*
*        Solve A * X = B.
*
*        Apply row interchanges to the right hand sides.
*
         CALL DLASWP( NRHS, B, LDB, 1, N, IPIV, 1 )
*
*        Solve L*X = B, overwriting B with X.
*
         CALL DTRSM( 'Left', 'Lower', 'No transpose', 'Unit', N, NRHS,
     $               ONE, A, LDA, B, LDB )
*
```

```
*        Solve U*X = B, overwriting B with X.
*
         CALL DTRSM( 'Left', 'Upper', 'No transpose', 'Non-unit', N,
     $               NRHS, ONE, A, LDA, B, LDB )
      ELSE
*
*        Solve A**T * X = B.
*
*        Solve U**T *X = B, overwriting B with X.
*
         CALL DTRSM( 'Left', 'Upper', 'Transpose', 'Non-unit', N, NRHS,
     $               ONE, A, LDA, B, LDB )
*
*        Solve L**T *X = B, overwriting B with X.
*
         CALL DTRSM( 'Left', 'Lower', 'Transpose', 'Unit', N, NRHS, ONE,
     $               A, LDA, B, LDB )
*
*        Apply row interchanges to the solution vectors.
*
         CALL DLASWP( NRHS, B, LDB, 1, N, IPIV, -1 )
      END IF
*
      RETURN
*
*     End of DGETRS
*
      END
      SUBROUTINE DGETRS( TRANS, N, NRHS, A, LDA, IPIV, B, LDB, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     March 31, 1993
*
*     .. Scalar Arguments ..
      CHARACTER          TRANS
      INTEGER            INFO, LDA, LDB, N, NRHS
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
```

```
*     .. Local Scalars ..
      LOGICAL            NOTRAN
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASWP, DTRSM, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      NOTRAN = LSAME( TRANS, 'N' )
      IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $    LSAME( TRANS, 'C' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( NRHS.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDB.LT.MAX( 1, N ) ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DGETRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. NRHS.EQ.0 )
     $   RETURN
*
      IF( NOTRAN ) THEN
*
*        Solve A * X = B.
*
*        Apply row interchanges to the right hand sides.
*
         CALL DLASWP( NRHS, B, LDB, 1, N, IPIV, 1 )
*
*        Solve L*X = B, overwriting B with X.
```

```
*
         CALL DTRSM( 'Left', 'Lower', 'No transpose', 'Unit', N, NRHS,
     $               ONE, A, LDA, B, LDB )
*
*        Solve U*X = B, overwriting B with X.
*
         CALL DTRSM( 'Left', 'Upper', 'No transpose', 'Non-unit', N,
     $               NRHS, ONE, A, LDA, B, LDB )
      ELSE
*
*        Solve A' * X = B.
*
*        Solve U'*X = B, overwriting B with X.
*
         CALL DTRSM( 'Left', 'Upper', 'Transpose', 'Non-unit', N, NRHS,
     $               ONE, A, LDA, B, LDB )
*
*        Solve L'*X = B, overwriting B with X.
*
         CALL DTRSM( 'Left', 'Lower', 'Transpose', 'Unit', N, NRHS, ONE,
     $               A, LDA, B, LDB )
*
*        Apply row interchanges to the solution vectors.
*
         CALL DLASWP( NRHS, B, LDB, 1, N, IPIV, -1 )
      END IF
*
      RETURN
*
*     End of DGETRS
*
      END
```

———-

90.133.4 lisp code

— dgetrs.lisp —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dgetrs (trans n nrhs a lda ipiv b ldb$ info)
    (declare (type (array f2cl-lib:integer4 (*)) ipiv)
             (type (array double-float (*)) b a)
             (type (f2cl-lib:integer4) info ldb$ lda nrhs n)
             (type (string *) trans))
```

```
(f2cl-lib:with-multi-array-data
    ((trans character trans-%data% trans-%offset%)
     (a double-float a-%data% a-%offset%)
     (b double-float b-%data% b-%offset%)
     (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
  (prog ((notran nil))
    (declare (type f2cl-lib:logical notran))
    (setf info 0)
    (setf notran
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "N")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val))
    (cond
     ((and (not notran)
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "T")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame trans "C")
              (declare (ignore var-1))
              (when var-0 (setf trans var-0))
              ret-val)))
      (setf info -1))
     ((< n 0) (setf info -2)) ((< nrhs 0) (setf info -3))
     ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -5))
     ((< ldb$ (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
      (setf info -8)))
    (cond
     ((/= info 0) (xerbla "DGETRS" (f2cl-lib:int-sub info))
      (go end_label)))
    (if (or (= n 0) (= nrhs 0))
        (go end_label))
    (cond
     (notran
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlaswp nrhs b ldb$ 1 n ipiv 1)
        (declare (ignore var-1 var-3 var-5 var-6))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf n var-4)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
```

```
          (dtrsm "Left" "Lower" "No transpose" "Unit" n nrhs one a lda b
           ldb$)
        (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
        (when var-4 (setf n var-4))
        (when var-5 (setf nrhs var-5))
        (when var-6 (setf one var-6))
        (when var-8 (setf lda var-8))
        (when var-10 (setf ldb$ var-10)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (dtrsm "Left" "Upper" "No transpose" "Non-unit" n nrhs one a lda
           b ldb$)
        (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
        (when var-4 (setf n var-4))
        (when var-5 (setf nrhs var-5))
        (when var-6 (setf one var-6))
        (when var-8 (setf lda var-8))
        (when var-10 (setf ldb$ var-10))))
    (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (dtrsm "Left" "Upper" "Transpose" "Non-unit" n nrhs one a lda b
           ldb$)
        (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
        (when var-4 (setf n var-4))
        (when var-5 (setf nrhs var-5))
        (when var-6 (setf one var-6))
        (when var-8 (setf lda var-8))
        (when var-10 (setf ldb$ var-10)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (dtrsm "Left" "Lower" "Transpose" "Unit" n nrhs one a lda b ldb$)
        (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
        (when var-4 (setf n var-4))
        (when var-5 (setf nrhs var-5))
        (when var-6 (setf one var-6))
        (when var-8 (setf lda var-8))
        (when var-10 (setf ldb$ var-10)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlaswp nrhs b ldb$ 1 n ipiv -1)
        (declare (ignore var-1 var-3 var-5 var-6))
        (when var-0 (setf nrhs var-0))
        (when var-2 (setf ldb$ var-2))
        (when var-4 (setf n var-4)))))
  (go end_label)
end_label
  (return (values trans n nrhs nil lda nil nil ldb$ info))))))
```

———-

90.134 dhseqr

90.134.1 Axiom unit tests

— **dhseqr.input** —

```
)set break resume
)sys rm -f dhseqr.output
)spool dhseqr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.134.2 Axiom help page

— **dhseqr.help** —

```
======================================================================
dhseqr examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DHSEQR - compute the eigenvalues of a Hessenberg matrix H  and, option-
       ally, the matrices T and Z from the Schur decomposition  H = Z T  Z**T,
       where T is an upper quasi-triangular matrix (the  Schur form), and Z is
       the orthogonal matrix of Schur vectors

       Optionally Z may be postmultiplied into an input orthogonal matrix Q,
       so that this routine can give the Schur factorization of a matrix A
       which has been reduced to the Hessenberg form H by the orthogonal
       matrix Q:  A = Q*H*Q**T = (QZ)*T*(QZ)**T.
```

```
SYNOPSIS
       SUBROUTINE DHSEQR( JOB, COMPZ, N, ILO, IHI, H, LDH,  WR,  WI,  Z,  LDZ,
                          WORK, LWORK, INFO )

           INTEGER        IHI, ILO, INFO, LDH, LDZ, LWORK, N

           CHARACTER      COMPZ, JOB

           DOUBLE         PRECISION  H( LDH, * ), WI( * ), WORK( * ), WR( * ),
                          Z( LDZ, * )

PURPOSE
          DHSEQR computes the eigenvalues of a Hessenberg matrix H
          and, optionally, the matrices T and Z from the Schur decomposition
          H = Z T Z**T, where T is an upper quasi-triangular matrix (the
          Schur form), and Z is the orthogonal matrix of Schur vectors.

          Optionally Z may be postmultiplied into an input orthogonal
          matrix Q so that this routine can give the Schur factorization
          of a matrix A which has been reduced to the Hessenberg form H
          by the orthogonal matrix Q:  A = Q*H*Q**T = (QZ)*T*(QZ)**T.


ARGUMENTS
  JOB     (input) CHARACTER*1
          = 'E':  compute eigenvalues only;
          = 'S':  compute eigenvalues and the Schur form T.

  COMPZ   (input) CHARACTER*1
          = 'N':  no Schur vectors are computed;
          = 'I':  Z is initialized to the unit matrix and the matrix Z
                  of Schur vectors of H is returned;
          = 'V':  Z must contain an orthogonal matrix Q on entry, and
                  the product Q*Z is returned.

  N       (input) INTEGER
          The order of the matrix H.  N >= 0.

  ILO     (input) INTEGER
  IHI     (input) INTEGER
          It is assumed that H is already upper triangular in rows
          and columns 1:ILO-1 and IHI+1:N. ILO and IHI are normally
          set by a previous call to DGEBAL, and then passed to SGEHRD
          when the matrix output by DGEBAL is reduced to Hessenberg
          form. Otherwise ILO and IHI should be set to 1 and N
          respectively.
          1 <= ILO <= IHI <= N, if N > 0; ILO=1 and IHI=0, if N=0.

  H       (input/output) DOUBLE PRECISION array, dimension (LDH,N)
```

```
         On entry, the upper Hessenberg matrix H.
         On exit, if JOB = 'S', H contains the upper quasi-triangular
         matrix T from the Schur decomposition (the Schur form);
         2-by-2 diagonal blocks (corresponding to complex conjugate
         pairs of eigenvalues) are returned in standard form, with
         H(i,i) = H(i+1,i+1) and H(i+1,i)*H(i,i+1) < 0. If JOB = 'E',
         the contents of H are unspecified on exit.

 LDH     (input) INTEGER
         The leading dimension of the array H. LDH >= max(1,N).

 WR      (output) DOUBLE PRECISION array, dimension (N)
 WI      (output) DOUBLE PRECISION array, dimension (N)
         The real and imaginary parts, respectively, of the computed
         eigenvalues. If two eigenvalues are computed as a complex
         conjugate pair, they are stored in consecutive elements of
         WR and WI, say the i-th and (i+1)th, with WI(i) > 0 and
         WI(i+1) < 0. If JOB = 'S', the eigenvalues are stored in the
         same order as on the diagonal of the Schur form returned in
         H, with WR(i) = H(i,i) and, if H(i:i+1,i:i+1) is a 2-by-2
         diagonal block, WI(i) = sqrt(H(i+1,i)*H(i,i+1)) and
         WI(i+1) = -WI(i).

 Z       (input/output) DOUBLE PRECISION array, dimension (LDZ,N)
         If COMPZ = 'N': Z is not referenced.
         If COMPZ = 'I': on entry, Z need not be set, and on exit, Z
         contains the orthogonal matrix Z of the Schur vectors of H.
         If COMPZ = 'V': on entry Z must contain an N-by-N matrix Q,
         which is assumed to be equal to the unit matrix except for
         the submatrix Z(ILO:IHI,ILO:IHI); on exit Z contains Q*Z.
         Normally Q is the orthogonal matrix generated by DORGHR after
         the call to DGEHRD which formed the Hessenberg matrix H.

 LDZ     (input) INTEGER
         The leading dimension of the array Z.
         LDZ >= max(1,N) if COMPZ = 'I' or 'V'; LDZ >= 1 otherwise.

 WORK    (workspace/output) DOUBLE PRECISION array, dimension (LWORK)
         On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK   (input) INTEGER
         The dimension of the array WORK.  LWORK >= max(1,N).

         If LWORK = -1, then a workspace query is assumed; the routine
         only calculates the optimal size of the WORK array, returns
         this value as the first entry of the WORK array, and no error
         message related to LWORK is issued by XERBLA.

 INFO    (output) INTEGER
         = 0:  successful exit
```

```
         < 0:  if INFO = -i, the i-th argument had an illegal value
         > 0:  if INFO = i, DHSEQR failed to compute all of the
               eigenvalues in a total of 30*(IHI-ILO+1) iterations;
               elements 1:ilo-1 and i+1:n of WR and WI contain those
               eigenvalues which have been successfully computed.
```

———-

90.134.3 fortran code

— dhseqr.f —

```
*  =====================================================================
      SUBROUTINE DHSEQR( JOB, COMPZ, N, ILO, IHI, H, LDH, WR, WI, Z,
     $                   LDZ, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            IHI, ILO, INFO, LDH, LDZ, LWORK, N
      CHARACTER          COMPZ, JOB
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   H( LDH, * ), WI( * ), WORK( * ), WR( * ),
     $                   Z( LDZ, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
*
*     ==== Matrices of order NTINY or smaller must be processed by
*     .    DLAHQR because of insufficient subdiagonal scratch space.
*     .    (This is a hard limit.) ====
      INTEGER            NTINY
      PARAMETER          ( NTINY = 11 )
*
*     ==== NL allocates some local workspace to help small matrices
*     .    through a rare DLAHQR failure.  NL .GT. NTINY = 11 is
*     .    required and NL .LE. NMIN = ILAENV(ISPEC=12,...) is recom-
*     .    mended.  (The default value of NMIN is 75.)  Using NL = 49
*     .    allows up to six simultaneous shifts and a 16-by-16
*     .    deflation window.  ====
      INTEGER            NL
```

```
      PARAMETER          ( NL = 49 )
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0d0, ONE = 1.0d0 )
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   HL( NL, NL ), WORKL( NL )
*     ..
*     .. Local Scalars ..
      INTEGER            I, KBOT, NMIN
      LOGICAL            INITZ, LQUERY, WANTT, WANTZ
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      LOGICAL            LSAME
      EXTERNAL           ILAENV, LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLACPY, DLAHQR, DLAQR0, DLASET, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     ==== Decode and check the input parameters. ====
*
      WANTT = LSAME( JOB, 'S' )
      INITZ = LSAME( COMPZ, 'I' )
      WANTZ = INITZ .OR. LSAME( COMPZ, 'V' )
      WORK( 1 ) = DBLE( MAX( 1, N ) )
      LQUERY = LWORK.EQ.-1
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'E' ) .AND. .NOT.WANTT ) THEN
         INFO = -1
      ELSE IF( .NOT.LSAME( COMPZ, 'N' ) .AND. .NOT.WANTZ ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -5
      ELSE IF( LDH.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDZ.LT.1 .OR. ( WANTZ .AND. LDZ.LT.MAX( 1, N ) ) ) THEN
         INFO = -11
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -13
      END IF
```

```
*
      IF( INFO.NE.0 ) THEN
*
*        ==== Quick return in case of invalid argument. ====
*
         CALL XERBLA( 'DHSEQR', -INFO )
         RETURN
*
      ELSE IF( N.EQ.0 ) THEN
*
*        ==== Quick return in case N = 0; nothing to do. ====
*
         RETURN
*
      ELSE IF( LQUERY ) THEN
*
*        ==== Quick return in case of a workspace query ====
*
         CALL DLAQRO( WANTT, WANTZ, N, ILO, IHI, H, LDH, WR, WI, ILO,
     $                IHI, Z, LDZ, WORK, LWORK, INFO )
*        ==== Ensure reported workspace size is backward-compatible with
*        .    previous LAPACK versions. ====
         WORK( 1 ) = MAX( DBLE( MAX( 1, N ) ), WORK( 1 ) )
         RETURN
*
      ELSE
*
*        ==== copy eigenvalues isolated by DGEBAL ====
*
         DO 10 I = 1, ILO - 1
            WR( I ) = H( I, I )
            WI( I ) = ZERO
   10    CONTINUE
         DO 20 I = IHI + 1, N
            WR( I ) = H( I, I )
            WI( I ) = ZERO
   20    CONTINUE
*
*        ==== Initialize Z, if requested ====
*
         IF( INITZ )
     $      CALL DLASET( 'A', N, N, ZERO, ONE, Z, LDZ )
*
*        ==== Quick return if possible ====
*
         IF( ILO.EQ.IHI ) THEN
            WR( ILO ) = H( ILO, ILO )
            WI( ILO ) = ZERO
            RETURN
         END IF
```

```
*
*        ==== DLAHQR/DLAQR0 crossover point ====
*
         NMIN = ILAENV( 12, 'DHSEQR', JOB( : 1 ) // COMPZ( : 1 ), N,
     $          ILO, IHI, LWORK )
         NMIN = MAX( NTINY, NMIN )
*
*        ==== DLAQR0 for big matrices; DLAHQR for small ones ====
*
         IF( N.GT.NMIN ) THEN
            CALL DLAQR0( WANTT, WANTZ, N, ILO, IHI, H, LDH, WR, WI, ILO,
     $                   IHI, Z, LDZ, WORK, LWORK, INFO )
         ELSE
*
*           ==== Small matrix ====
*
            CALL DLAHQR( WANTT, WANTZ, N, ILO, IHI, H, LDH, WR, WI, ILO,
     $                   IHI, Z, LDZ, INFO )
*
            IF( INFO.GT.0 ) THEN
*
*              ==== A rare DLAHQR failure!  DLAQR0 sometimes succeeds
*              .    when DLAHQR fails. ====
*
               KBOT = INFO
*
               IF( N.GE.NL ) THEN
*
*                 ==== Larger matrices have enough subdiagonal scratch
*                 .    space to call DLAQR0 directly. ====
*
                  CALL DLAQR0( WANTT, WANTZ, N, ILO, KBOT, H, LDH, WR,
     $                         WI, ILO, IHI, Z, LDZ, WORK, LWORK, INFO )
*
               ELSE
*
                  ==== Tiny matrices don't have enough subdiagonal
                  .    scratch space to benefit from DLAQR0.  Hence,
                  .    tiny matrices must be copied into a larger
                  .    array before calling DLAQR0. ====
*
                  CALL DLACPY( 'A', N, N, H, LDH, HL, NL )
                  HL( N+1, N ) = ZERO
                  CALL DLASET( 'A', NL, NL-N, ZERO, ZERO, HL( 1, N+1 ),
     $                         NL )
                  CALL DLAQR0( WANTT, WANTZ, NL, ILO, KBOT, HL, NL, WR,
     $                         WI, ILO, IHI, Z, LDZ, WORKL, NL, INFO )
                  IF( WANTT .OR. INFO.NE.0 )
     $               CALL DLACPY( 'A', N, N, HL, NL, H, LDH )
               END IF
```

```
          END IF
        END IF
*
*       ==== Clear out the trash, if necessary. ====
*
        IF( ( WANTT .OR. INFO.NE.0 ) .AND. N.GT.2 )
     $     CALL DLASET( 'L', N-2, N-2, ZERO, ZERO, H( 3, 1 ), LDH )
*
*       ==== Ensure reported workspace size is backward-compatible with
*       .    previous LAPACK versions. ====
*
        WORK( 1 ) = MAX( DBLE( MAX( 1, N ) ), WORK( 1 ) )
      END IF
*
*     ==== End of DHSEQR ====
*
      END
      SUBROUTINE DHSEQR( JOB, COMPZ, N, ILO, IHI, H, LDH, WR, WI, Z,
     $                   LDZ, WORK, LWORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      CHARACTER          COMPZ, JOB
      INTEGER            IHI, ILO, INFO, LDH, LDZ, LWORK, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   H( LDH, * ), WI( * ), WORK( * ), WR( * ),
     $                   Z( LDZ, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE, TWO
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0, TWO = 2.0D+0 )
      DOUBLE PRECISION   CONST
      PARAMETER          ( CONST = 1.5D+0 )
      INTEGER            NSMAX, LDS
      PARAMETER          ( NSMAX = 15, LDS = NSMAX )
*     ..
*     .. Local Scalars ..
      LOGICAL            INITZ, LQUERY, WANTT, WANTZ
      INTEGER            I, I1, I2, IERR, II, ITEMP, ITN, ITS, J, K, L,
     $                   MAXB, NH, NR, NS, NV
      DOUBLE PRECISION   ABSW, OVFL, SMLNUM, TAU, TEMP, TST1, ULP, UNFL
*     ..
```

```
*     .. Local Arrays ..
      DOUBLE PRECISION   S( LDS, NSMAX ), V( NSMAX+1 ), VV( NSMAX+1 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX, ILAENV
      DOUBLE PRECISION   DLAMCH, DLANHS, DLAPY2
      EXTERNAL           LSAME, IDAMAX, ILAENV, DLAMCH, DLANHS, DLAPY2
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DGEMV, DLACPY, DLAHQR, DLARFG, DLARFX,
     $                   DLASET, DSCAL, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Decode and test the input parameters
*
      WANTT = LSAME( JOB, 'S' )
      INITZ = LSAME( COMPZ, 'I' )
      WANTZ = INITZ .OR. LSAME( COMPZ, 'V' )
*
      INFO = 0
      WORK( 1 ) = MAX( 1, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.LSAME( JOB, 'E' ) .AND. .NOT.WANTT ) THEN
         INFO = -1
      ELSE IF( .NOT.LSAME( COMPZ, 'N' ) .AND. .NOT.WANTZ ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -5
      ELSE IF( LDH.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDZ.LT.1 .OR. WANTZ .AND. LDZ.LT.MAX( 1, N ) ) THEN
         INFO = -11
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -13
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DHSEQR', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
```

```
*
*     Initialize Z, if necessary
*
      IF( INITZ )
     $   CALL DLASET( 'Full', N, N, ZERO, ONE, Z, LDZ )
*
*     Store the eigenvalues isolated by DGEBAL.
*
      DO 10 I = 1, ILO - 1
         WR( I ) = H( I, I )
         WI( I ) = ZERO
   10 CONTINUE
      DO 20 I = IHI + 1, N
         WR( I ) = H( I, I )
         WI( I ) = ZERO
   20 CONTINUE
*
*     Quick return if possible.
*
      IF( N.EQ.0 )
     $   RETURN
      IF( ILO.EQ.IHI ) THEN
         WR( ILO ) = H( ILO, ILO )
         WI( ILO ) = ZERO
         RETURN
      END IF
*
*     Set rows and columns ILO to IHI to zero below the first
*     subdiagonal.
*
      DO 40 J = ILO, IHI - 2
         DO 30 I = J + 2, N
            H( I, J ) = ZERO
   30    CONTINUE
   40 CONTINUE
      NH = IHI - ILO + 1
*
*     Determine the order of the multi-shift QR algorithm to be used.
*
      NS = ILAENV( 4, 'DHSEQR', JOB // COMPZ, N, ILO, IHI, -1 )
      MAXB = ILAENV( 8, 'DHSEQR', JOB // COMPZ, N, ILO, IHI, -1 )
      IF( NS.LE.2 .OR. NS.GT.NH .OR. MAXB.GE.NH ) THEN
*
*        Use the standard double-shift algorithm
*
         CALL DLAHQR( WANTT, WANTZ, N, ILO, IHI, H, LDH, WR, WI, ILO,
     $                IHI, Z, LDZ, INFO )
         RETURN
      END IF
      MAXB = MAX( 3, MAXB )
```

```
      NS = MIN( NS, MAXB, NSMAX )
*
*     Now 2 < NS <= MAXB < NH.
*
*     Set machine-dependent constants for the stopping criterion.
*     If norm(H) <= sqrt(OVFL), overflow should not occur.
*
      UNFL = DLAMCH( 'Safe minimum' )
      OVFL = ONE / UNFL
      CALL DLABAD( UNFL, OVFL )
      ULP = DLAMCH( 'Precision' )
      SMLNUM = UNFL*( NH / ULP )
*
*     I1 and I2 are the indices of the first row and last column of H
*     to which transformations must be applied. If eigenvalues only are
*     being computed, I1 and I2 are set inside the main loop.
*
      IF( WANTT ) THEN
         I1 = 1
         I2 = N
      END IF
*
*     ITN is the total number of multiple-shift QR iterations allowed.
*
      ITN = 30*NH
*
*     The main loop begins here. I is the loop index and decreases from
*     IHI to ILO in steps of at most MAXB. Each iteration of the loop
*     works with the active submatrix in rows and columns L to I.
*     Eigenvalues I+1 to IHI have already converged. Either L = ILO or
*     H(L,L-1) is negligible so that the matrix splits.
*
      I = IHI
   50 CONTINUE
      L = ILO
      IF( I.LT.ILO )
     $   GO TO 170
*
*     Perform multiple-shift QR iterations on rows and columns ILO to I
*     until a submatrix of order at most MAXB splits off at the bottom
*     because a subdiagonal element has become negligible.
*
      DO 150 ITS = 0, ITN
*
*        Look for a single small subdiagonal element.
*
         DO 60 K = I, L + 1, -1
            TST1 = ABS( H( K-1, K-1 ) ) + ABS( H( K, K ) )
            IF( TST1.EQ.ZERO )
     $         TST1 = DLANHS( '1', I-L+1, H( L, L ), LDH, WORK )
```

```
            IF( ABS( H( K, K-1 ) ).LE.MAX( ULP*TST1, SMLNUM ) )
     $         GO TO 70
   60    CONTINUE
   70    CONTINUE
         L = K
         IF( L.GT.ILO ) THEN
*
*           H(L,L-1) is negligible.
*
            H( L, L-1 ) = ZERO
         END IF
*
*        Exit from loop if a submatrix of order <= MAXB has split off.
*
         IF( L.GE.I-MAXB+1 )
     $      GO TO 160
*
*        Now the active submatrix is in rows and columns L to I. If
*        eigenvalues only are being computed, only the active submatrix
*        need be transformed.
*
         IF( .NOT.WANTT ) THEN
            I1 = L
            I2 = I
         END IF
*
         IF( ITS.EQ.20 .OR. ITS.EQ.30 ) THEN
*
*           Exceptional shifts.
*
            DO 80 II = I - NS + 1, I
               WR( II ) = CONST*( ABS( H( II, II-1 ) )+
     $                    ABS( H( II, II ) ) )
               WI( II ) = ZERO
   80       CONTINUE
         ELSE
*
*           Use eigenvalues of trailing submatrix of order NS as shifts.
*
            CALL DLACPY( 'Full', NS, NS, H( I-NS+1, I-NS+1 ), LDH, S,
     $                   LDS )
            CALL DLAHQR( .FALSE., .FALSE., NS, 1, NS, S, LDS,
     $                   WR( I-NS+1 ), WI( I-NS+1 ), 1, NS, Z, LDZ,
     $                   IERR )
            IF( IERR.GT.0 ) THEN
*
*              If DLAHQR failed to compute all NS eigenvalues, use the
*              unconverged diagonal elements as the remaining shifts.
*
               DO 90 II = 1, IERR
```

```
                 WR( I-NS+II ) = S( II, II )
                 WI( I-NS+II ) = ZERO
   90         CONTINUE
           END IF
        END IF
*
*       Form the first column of (G-w(1)) (G-w(2)) . . . (G-w(ns))
*       where G is the Hessenberg submatrix H(L:I,L:I) and w is
*       the vector of shifts (stored in WR and WI). The result is
*       stored in the local array V.
*
        V( 1 ) = ONE
        DO 100 II = 2, NS + 1
           V( II ) = ZERO
  100   CONTINUE
        NV = 1
        DO 120 J = I - NS + 1, I
           IF( WI( J ).GE.ZERO ) THEN
              IF( WI( J ).EQ.ZERO ) THEN
*
*                real shift
*
                 CALL DCOPY( NV+1, V, 1, VV, 1 )
                 CALL DGEMV( 'No transpose', NV+1, NV, ONE, H( L, L ),
     $                       LDH, VV, 1, -WR( J ), V, 1 )
                 NV = NV + 1
              ELSE IF( WI( J ).GT.ZERO ) THEN
*
*                complex conjugate pair of shifts
*
                 CALL DCOPY( NV+1, V, 1, VV, 1 )
                 CALL DGEMV( 'No transpose', NV+1, NV, ONE, H( L, L ),
     $                       LDH, V, 1, -TWO*WR( J ), VV, 1 )
                 ITEMP = IDAMAX( NV+1, VV, 1 )
                 TEMP = ONE / MAX( ABS( VV( ITEMP ) ), SMLNUM )
                 CALL DSCAL( NV+1, TEMP, VV, 1 )
                 ABSW = DLAPY2( WR( J ), WI( J ) )
                 TEMP = ( TEMP*ABSW )*ABSW
                 CALL DGEMV( 'No transpose', NV+2, NV+1, ONE,
     $                       H( L, L ), LDH, VV, 1, TEMP, V, 1 )
                 NV = NV + 2
              END IF
*
*             Scale V(1:NV) so that max(abs(V(i))) = 1. If V is zero,
*             reset it to the unit vector.
*
              ITEMP = IDAMAX( NV, V, 1 )
              TEMP = ABS( V( ITEMP ) )
              IF( TEMP.EQ.ZERO ) THEN
                 V( 1 ) = ONE
```

```
                DO 110 II = 2, NV
                   V( II ) = ZERO
  110           CONTINUE
             ELSE
                TEMP = MAX( TEMP, SMLNUM )
                CALL DSCAL( NV, ONE / TEMP, V, 1 )
             END IF
          END IF
  120    CONTINUE
*
*        Multiple-shift QR step
*
         DO 140 K = L, I - 1
*
*           The first iteration of this loop determines a reflection G
*           from the vector V and applies it from left and right to H,
*           thus creating a nonzero bulge below the subdiagonal.
*
*           Each subsequent iteration determines a reflection G to
*           restore the Hessenberg form in the (K-1)th column, and thus
*           chases the bulge one step toward the bottom of the active
*           submatrix. NR is the order of G.
*
            NR = MIN( NS+1, I-K+1 )
            IF( K.GT.L )
     $         CALL DCOPY( NR, H( K, K-1 ), 1, V, 1 )
            CALL DLARFG( NR, V( 1 ), V( 2 ), 1, TAU )
            IF( K.GT.L ) THEN
               H( K, K-1 ) = V( 1 )
               DO 130 II = K + 1, I
                  H( II, K-1 ) = ZERO
  130          CONTINUE
            END IF
            V( 1 ) = ONE
*
*           Apply G from the left to transform the rows of the matrix in
*           columns K to I2.
*
            CALL DLARFX( 'Left', NR, I2-K+1, V, TAU, H( K, K ), LDH,
     $                   WORK )
*
*           Apply G from the right to transform the columns of the
*           matrix in rows I1 to min(K+NR,I).
*
            CALL DLARFX( 'Right', MIN( K+NR, I )-I1+1, NR, V, TAU,
     $                   H( I1, K ), LDH, WORK )
*
            IF( WANTZ ) THEN
*
*              Accumulate transformations in the matrix Z
```

```
*
               CALL DLARFX( 'Right', NH, NR, V, TAU, Z( ILO, K ), LDZ,
     $                      WORK )
            END IF
  140    CONTINUE
*
  150 CONTINUE
*
*     Failure to converge in remaining number of iterations
*
      INFO = I
      RETURN
*
  160 CONTINUE
*
*     A submatrix of order <= MAXB in rows and columns L to I has split
*     off. Use the double-shift QR algorithm to handle it.
*
      CALL DLAHQR( WANTT, WANTZ, N, L, I, H, LDH, WR, WI, ILO, IHI, Z,
     $             LDZ, INFO )
      IF( INFO.GT.0 )
     $   RETURN
*
*     Decrement number of remaining iterations, and return to start of
*     the main loop with a new value of I.
*
      ITN = ITN - ITS
      I = L - 1
      GO TO 50
*
  170 CONTINUE
      WORK( 1 ) = MAX( 1, N )
      RETURN
*
*     End of DHSEQR
*
      END
```

———-

90.134.4 lisp code

— dhseqr.lisp —

```
(let* ((ntiny 11) (nl 49) (zero 0.0d0) (one 1.0d0))
  (declare (type (f2cl-lib:integer4 11 11) ntiny)
           (type (f2cl-lib:integer4 49 49) nl)
```

```
          (type (double-float 0.0d0 0.0d0) zero)
          (type (double-float 1.0d0 1.0d0) one)
          (ignorable ntiny nl zero one))
 (defun dhseqr (job compz n ilo ihi h ldh wr wi z ldz work lwork info)
   (declare (type (array double-float (*)) work z wi wr h)
            (type (f2cl-lib:integer4) info lwork ldz ldh ihi ilo n)
            (type (string *) compz job))
   (f2cl-lib:with-multi-array-data
       ((job character job-%data% job-%offset%)
        (compz character compz-%data% compz-%offset%)
        (h double-float h-%data% h-%offset%)
        (wr double-float wr-%data% wr-%offset%)
        (wi double-float wi-%data% wi-%offset%)
        (z double-float z-%data% z-%offset%)
        (work double-float work-%data% work-%offset%))
     (prog ((initz nil) (lquery nil) (wantt nil) (wantz nil) (i 0) (kbot 0)
            (nmin 0)
            (hl
             (make-array (the fixnum (reduce #'* (list nl nl))) :element-type
                         'double-float))
            (workl (make-array nl :element-type 'double-float)))
       (declare (type f2cl-lib:logical initz lquery wantt wantz)
                (type (f2cl-lib:integer4) i kbot nmin)
                (type (array double-float (*)) hl workl))
       (setf wantt
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame job "S")
                 (declare (ignore var-1))
                 (when var-0 (setf job var-0))
                 ret-val))
       (setf initz
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame compz "I")
                 (declare (ignore var-1))
                 (when var-0 (setf compz var-0))
                 ret-val))
       (setf wantz
               (or initz
                   (multiple-value-bind (ret-val var-0 var-1)
                       (lsame compz "V")
                     (declare (ignore var-1))
                     (when var-0 (setf compz var-0))
                     ret-val)))
       (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
               (f2cl-lib:dble
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n))))
       (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
       (setf info 0)
       (cond
        ((and
```

```
    (not
     (multiple-value-bind (ret-val var-0 var-1)
         (lsame job "E")
       (declare (ignore var-1))
       (when var-0 (setf job var-0))
       ret-val))
    (not wantt))
   (setf info -1))
  ((and
    (not
     (multiple-value-bind (ret-val var-0 var-1)
         (lsame compz "N")
       (declare (ignore var-1))
       (when var-0 (setf compz var-0))
       ret-val))
    (not wantz))
   (setf info -2))
  ((< n 0) (setf info -3))
  ((or (< ilo 1)
       (> ilo
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n))))
   (setf info -4))
  ((or
    (< ihi (min (the f2cl-lib:integer4 ilo) (the f2cl-lib:integer4 n)))
    (> ihi n))
   (setf info -5))
  ((< ldh (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
   (setf info -7))
  ((or (< ldz 1)
       (and wantz
            (< ldz
               (max (the f2cl-lib:integer4 1)
                    (the f2cl-lib:integer4 n)))))
   (setf info -11))
  ((and
    (< lwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (not lquery))
   (setf info -13)))
 (cond
  ((/= info 0) (xerbla "DHSEQR" (f2cl-lib:int-sub info)) (go end_label))
  ((= n 0) (go end_label))
  (lquery
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12 var-13 var-14 var-15)
       (dlaqr0 wantt wantz n ilo ihi h ldh wr wi ilo ihi z ldz work
        lwork info)
     (declare (ignore var-5 var-7 var-8 var-11 var-13))
     (when var-0 (setf wantt var-0))
     (when var-1 (setf wantz var-1))
```

```
  (when var-2 (setf n var-2))
  (when var-3 (setf ilo var-3))
  (when var-4 (setf ihi var-4))
  (when var-6 (setf ldh var-6))
  (when var-9 (setf ilo var-9))
  (when var-10 (setf ihi var-10))
  (when var-12 (setf ldz var-12))
  (when var-14 (setf lwork var-14))
  (when var-15 (setf info var-15)))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (max
         (f2cl-lib:dble
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)))
(go end_label))
(t
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add ilo (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (setf (f2cl-lib:fref wr-%data% (i) ((1 *)) wr-%offset%)
            (f2cl-lib:fref h-%data% (i i) ((1 ldh) (1 *))
                           h-%offset%))
    (setf (f2cl-lib:fref wi-%data% (i) ((1 *)) wi-%offset%) zero)
   label10))
(f2cl-lib:fdo (i (f2cl-lib:int-add ihi 1) (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref wr-%data% (i) ((1 *)) wr-%offset%)
            (f2cl-lib:fref h-%data% (i i) ((1 ldh) (1 *))
                           h-%offset%))
    (setf (f2cl-lib:fref wi-%data% (i) ((1 *)) wi-%offset%) zero)
   label20))
(if initz
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlaset "A" n n zero one z ldz)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldz var-6))))
(cond
 ((= ilo ihi)
  (setf (f2cl-lib:fref wr-%data% (ilo) ((1 *)) wr-%offset%)
          (f2cl-lib:fref h-%data% (ilo ilo) ((1 ldh) (1 *))
                         h-%offset%))
  (setf (f2cl-lib:fref wi-%data% (ilo) ((1 *)) wi-%offset%) zero)
  (go end_label)))
(setf nmin
        (multiple-value-bind
```

```
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 12 "DHSEQR"
             (f2cl-lib:f2cl-// (f2cl-lib:fref-string job (1 1))
                               (f2cl-lib:fref-string compz (1 1)))
             n ilo ihi lwork)
          (declare (ignore var-0 var-1 var-2))
          (when var-3 (setf n var-3))
          (when var-4 (setf ilo var-4))
          (when var-5 (setf ihi var-5))
          (when var-6 (setf lwork var-6))
          ret-val))
(setf nmin
        (max (the f2cl-lib:integer4 ntiny)
             (the f2cl-lib:integer4 nmin)))
(cond
 ((> n nmin)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (dlaqr0 wantt wantz n ilo ihi h ldh wr wi ilo ihi z ldz work
       lwork info)
    (declare (ignore var-5 var-7 var-8 var-11 var-13))
    (when var-0 (setf wantt var-0))
    (when var-1 (setf wantz var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf ldh var-6))
    (when var-9 (setf ilo var-9))
    (when var-10 (setf ihi var-10))
    (when var-12 (setf ldz var-12))
    (when var-14 (setf lwork var-14))
    (when var-15 (setf info var-15))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13)
      (dlahqr wantt wantz n ilo ihi h ldh wr wi ilo ihi z ldz info)
    (declare (ignore var-5 var-7 var-8 var-11))
    (when var-0 (setf wantt var-0))
    (when var-1 (setf wantz var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf ldh var-6))
    (when var-9 (setf ilo var-9))
    (when var-10 (setf ihi var-10))
    (when var-12 (setf ldz var-12))
    (when var-13 (setf info var-13)))
  (cond
```

```
((> info 0) (setf kbot info)
 (cond
  ((>= n nl)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10 var-11 var-12 var-13 var-14 var-15)
       (dlaqr0 wantt wantz n ilo kbot h ldh wr wi ilo ihi z ldz
        work lwork info)
     (declare (ignore var-5 var-7 var-8 var-11 var-13))
     (when var-0 (setf wantt var-0))
     (when var-1 (setf wantz var-1))
     (when var-2 (setf n var-2))
     (when var-3 (setf ilo var-3))
     (when var-4 (setf kbot var-4))
     (when var-6 (setf ldh var-6))
     (when var-9 (setf ilo var-9))
     (when var-10 (setf ihi var-10))
     (when var-12 (setf ldz var-12))
     (when var-14 (setf lwork var-14))
     (when var-15 (setf info var-15))))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlacpy "A" n n h ldh hl nl)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf n var-1))
     (when var-2 (setf n var-2))
     (when var-4 (setf ldh var-4))
     (when var-6 (setf nl var-6)))
   (setf (f2cl-lib:fref hl ((f2cl-lib:int-add n 1) n)
                         ((1 nl) (1 nl)))
         zero)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (dlaset "A" nl (f2cl-lib:int-sub nl n) zero zero
        (f2cl-lib:array-slice hl double-float
                              (1 (f2cl-lib:int-add n 1))
                              ((1 nl) (1 nl)))
        nl)
     (declare (ignore var-0 var-2 var-5))
     (when var-1 (setf nl var-1))
     (when var-3 (setf zero var-3))
     (when var-4 (setf zero var-4))
     (when var-6 (setf nl var-6)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10 var-11 var-12 var-13 var-14 var-15)
       (dlaqr0 wantt wantz nl ilo kbot hl nl wr wi ilo ihi z ldz
        workl nl info)
     (declare (ignore var-5 var-7 var-8 var-11 var-13))
```

```
            (when var-0 (setf wantt var-0))
            (when var-1 (setf wantz var-1))
            (when var-2 (setf nl var-2))
            (when var-3 (setf ilo var-3))
            (when var-4 (setf kbot var-4))
            (when var-6 (setf nl var-6))
            (when var-9 (setf ilo var-9))
            (when var-10 (setf ihi var-10))
            (when var-12 (setf ldz var-12))
            (when var-14 (setf nl var-14))
            (when var-15 (setf info var-15)))
          (if (or wantt (/= info 0))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (dlacpy "A" n n hl nl h ldh)
                (declare (ignore var-0 var-3 var-5))
                (when var-1 (setf n var-1))
                (when var-2 (setf n var-2))
                (when var-4 (setf nl var-4))
                (when var-6 (setf ldh var-6)))))))))
    (if (and (or wantt (/= info 0)) (> n 2))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (dlaset "L" (f2cl-lib:int-sub n 2) (f2cl-lib:int-sub n 2)
             zero zero
             (f2cl-lib:array-slice h-%data% double-float (3 1)
                                   ((1 ldh) (1 *)) h-%offset%)
             ldh)
          (declare (ignore var-0 var-1 var-2 var-5))
          (when var-3 (setf zero var-3))
          (when var-4 (setf zero var-4))
          (when var-6 (setf ldh var-6))))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (max
             (f2cl-lib:dble
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
             (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)))))
end_label
 (return
  (values job compz n ilo ihi nil ldh nil nil nil ldz nil lwork
          info))))))
```

———-

90.135 disnan

90.135.1 Axiom unit tests

— disnan.input —

```
)set break resume
)sys rm -f disnan.output
)spool disnan.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.135.2 Axiom help page

— disnan.help —

```
=====================================================================
dhseqr examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

Online html documentation available at
           http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

     LOGICAL FUNCTION DISNAN( DIN )

     .. Scalar Arguments ..
     DOUBLE PRECISION   DIN
     ..


 Purpose:
 =============
```

```
 DISNAN returns .TRUE. if its argument is NaN, and .FALSE.
 otherwise.  To be replaced by the Fortran 2003 intrinsic in the
 future.


 Arguments:
 ==========

 [in] DIN
       DIN is DOUBLE PRECISION
       Input to test for NaN.
```

———-

90.135.3 fortran code

— **disnan.f** —

```
*  =====================================================================
      LOGICAL FUNCTION DISNAN( DIN )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   DIN
*     ..
*
*  =====================================================================
*
*  .. External Functions ..
      LOGICAL DLAISNAN
      EXTERNAL DLAISNAN
*  ..
*  .. Executable Statements ..
      DISNAN = DLAISNAN(DIN,DIN)
      RETURN
      END

*  =====================================================================
      LOGICAL FUNCTION DISNAN( DIN )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
```

```
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   DIN
*     ..
*
*  =====================================================================
*
*  .. External Functions ..
      LOGICAL DLAISNAN
      EXTERNAL DLAISNAN
*  ..
*  .. Executable Statements ..
      DISNAN = DLAISNAN(DIN,DIN)
      RETURN
      END
```

90.135.4 lisp code

— disnan.lisp —

```
(defun disnan (din)
  (declare (type (double-float) din))
  (prog ((disnan nil))
    (declare (type f2cl-lib:logical disnan))
    (setf disnan
            (multiple-value-bind (ret-val var-0 var-1)
                (dlaisnan din din)
              (declare (ignore))
              (when var-0 (setf din var-0))
              (when var-1 (setf din var-1))
              ret-val))
    (go end_label)
   end_label
    (return (values disnan din))))
```

90.136 dlabad

90.136.1 Axiom unit tests

— dlabad.input —

```
)set break resume
)sys rm -f dlabad.output
)spool dlabad.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.136.2 Axiom help page

— dlabad.help —

```
======================================================================
dlabad examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DLABAD  - a input the values computed by DLAMCH for underflow and over-
       flow, and returns the square root of each of these values if the log of
       LARGE is sufficiently large

SYNOPSIS
       SUBROUTINE DLABAD( SMALL, LARGE )

           DOUBLE         PRECISION LARGE, SMALL

PURPOSE
       DLABAD  takes  as input the values computed by DLAMCH for underflow and
       overflow, and returns the square root of each of these  values  if  the
       log  of  LARGE  is  sufficiently large.  This subroutine is intended to
       identify machines with a large exponent range, such as the  Crays,  and
       redefine  the  underflow  and overflow limits to be the square roots of
```

```
       the values computed by  DLAMCH.   This  subroutine  is  needed  because
       DLAMCH does not compensate for poor arithmetic in the upper half of the
       exponent range, as is found on a Cray.


ARGUMENTS
       SMALL   (input/output) DOUBLE PRECISION
               On entry, the underflow threshold as computed  by  DLAMCH.   On
               exit, if LOG10(LARGE) is sufficiently large, the square root of
               SMALL, otherwise unchanged.

       LARGE   (input/output) DOUBLE PRECISION
               On entry, the overflow threshold as  computed  by  DLAMCH.   On
               exit, if LOG10(LARGE) is sufficiently large, the square root of
               LARGE, otherwise unchanged.
```

90.136.3 fortran code

— dlabad.f —

```
*  =====================================================================
      SUBROUTINE DLABAD( SMALL, LARGE )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   LARGE, SMALL
*     ..
*
*  =====================================================================
*
*     .. Intrinsic Functions ..
      INTRINSIC          LOG10, SQRT
*     ..
*     .. Executable Statements ..
*
*     If it looks like we're on a Cray, take the square root of
*     SMALL and LARGE to avoid overflow and underflow problems.
*
      IF( LOG10( LARGE ).GT.2000.D0 ) THEN
         SMALL = SQRT( SMALL )
         LARGE = SQRT( LARGE )
```

```
      END IF
*
      RETURN
*
*     End of DLABAD
*
      END
      SUBROUTINE DLABAD( SMALL, LARGE )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1992
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   LARGE, SMALL
*     ..
*
*  =====================================================================
*
*     .. Intrinsic Functions ..
      INTRINSIC          LOG10, SQRT
*     ..
*     .. Executable Statements ..
*
*     If it looks like we're on a Cray, take the square root of
*     SMALL and LARGE to avoid overflow and underflow problems.
*
      IF( LOG10( LARGE ).GT.2000.D0 ) THEN
         SMALL = SQRT( SMALL )
         LARGE = SQRT( LARGE )
      END IF
*
      RETURN
*
*     End of DLABAD
*
      END
```

90.136.4 lisp code

— dlabad.lisp —

```
(defun dlabad (small large)
  (declare (type (double-float) large small))
```

```
  (prog ()
    (declare)
    (cond
     ((> (f2cl-lib:log10 large) 2000.0d0) (setf small (f2cl-lib:fsqrt small))
      (setf large (f2cl-lib:fsqrt large))))
    (go end_label)
   end_label
    (return (values small large))))
```

———-

90.137 dlabrd

90.137.1 Axiom unit tests

— **dlabrd.input** —

```
)set break resume
)sys rm -f dlabrd.output
)spool dlabrd.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.137.2 Axiom help page

— **dlabrd.help** —

```
====================================================================
dlabrd examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DLABRD  - the first NB rows and columns of a real general m by n matrix
```

A to upper or lower bidiagonal form by an orthogonal transformation Q' * A * P, and returns the matrices X and Y which are needed to apply the transformation to the unreduced part of A

SYNOPSIS

```
SUBROUTINE DLABRD( M, N, NB, A, LDA, D, E, TAUQ, TAUP, X, LDX, Y, LDY )

    INTEGER        LDA, LDX, LDY, M, N, NB

    DOUBLE         PRECISION  A(  LDA,  * ), D( * ), E( * ), TAUP( * ),
                   TAUQ( * ), X( LDX, * ), Y( LDY, * )
```

PURPOSE

DLABRD reduces the first NB rows and columns of a real general m by n matrix A to upper or lower bidiagonal form by an orthogonal transformation Q' * A * P, and returns the matrices X and Y which are needed to apply the transformation to the unreduced part of A.

If m >= n, A is reduced to upper bidiagonal form; if m < n, to lower bidiagonal form.

This is an auxiliary routine called by DGEBRD

ARGUMENTS

M (input) INTEGER
The number of rows in the matrix A.

N (input) INTEGER
The number of columns in the matrix A.

NB (input) INTEGER
The number of leading rows and columns of A to be reduced.

A (input/output) DOUBLE PRECISION array, dimension (LDA,N)
On entry, the m by n general matrix to be reduced. On exit, the first NB rows and columns of the matrix are overwritten; the rest of the array is unchanged. If m >= n, elements on and below the diagonal in the first NB columns, with the array TAUQ, represent the orthogonal matrix Q as a product of elementary reflectors; and elements above the diagonal in the first NB rows, with the array TAUP, represent the orthogonal matrix P as a product of elementary reflectors. If m < n, elements below the diagonal in the first NB columns, with the array TAUQ, represent the orthogonal matrix Q as a product of elementary reflectors, and elements on and above the diagonal in the first NB rows, with the array TAUP, represent the orthogonal matrix P as a product of elementary reflectors. See Further Details. LDA (input) INTEGER The leading dimension of the array A. LDA >= max(1,M).

```
D       (output) DOUBLE PRECISION array, dimension (NB)
        The diagonal elements of the first NB rows and columns  of  the
        reduced matrix.  D(i) = A(i,i).

E       (output) DOUBLE PRECISION array, dimension (NB)
        The  off-diagonal  elements of the first NB rows and columns of
        the reduced matrix.

TAUQ    (output) DOUBLE PRECISION array dimension (NB)
        The scalar factors of the elementary reflectors which represent
        the orthogonal matrix Q. See Further Details.  TAUP    (output)
        DOUBLE PRECISION array, dimension (NB) The  scalar  factors  of
        the elementary reflectors which represent the orthogonal matrix
        P. See Further  Details.   X        (output)  DOUBLE  PRECISION
        array,  dimension  (LDX,NB)  The  m-by-nb  matrix X required to
        update the unreduced part of A.

LDX     (input) INTEGER
        The leading dimension of the array X. LDX >= M.

Y       (output) DOUBLE PRECISION array, dimension (LDY,NB)
        The n-by-nb matrix Y required to update the unreduced  part  of
        A.

LDY     (input) INTEGER
        The leading dimension of the array Y. LDY >= N.

FURTHER DETAILS
        The  matrices Q and P are represented as products of elementary reflec-
        tors:

           Q = H(1) H(2) . . . H(nb)  and  P = G(1) G(2) . . . G(nb)

        Each H(i) and G(i) has the form:

           H(i) = I - tauq * v * v'  and G(i) = I - taup * u * u'

        where tauq and taup are real scalars, and v and u are real vectors.

        If m >= n, v(1:i-1) = 0, v(i) = 1, and v(i:m)  is  stored  on  exit  in
        A(i:m,i);  u(1:i)  =  0,  u(i+1) = 1, and u(i+1:n) is stored on exit in
        A(i,i+1:n); tauq is stored in TAUQ(i) and taup in TAUP(i).

        If m < n, v(1:i) = 0, v(i+1) = 1, and v(i+1:m) is  stored  on  exit  in
        A(i+2:m,i);  u(1:i-1)  =  0,  u(i) = 1, and u(i:n) is stored on exit in
        A(i,i+1:n); tauq is stored in TAUQ(i) and taup in TAUP(i).

        The elements of the vectors v and u together form the m-by-nb matrix  V
        and  the nb-by-n matrix U' which are needed, with X and Y, to apply the
```

```
transformation to the unreduced part  of  the  matrix,  using  a  block
update of the form:  A := A - V*Y' - X*U'.

The  contents  of  A  on exit are illustrated by the following examples
with nb = 2:

m = 6 and n = 5 (m > n):          m = 5 and n = 6 (m < n):

  (  1   1   u1  u1  u1 )           (  1   u1  u1  u1  u1  u1 )
  (  v1  1   1   u2  u2 )           (  1   1   u2  u2  u2  u2 )
  (  v1  v2  a   a   a  )           (  v1  1   a   a   a   a  )
  (  v1  v2  a   a   a  )           (  v1  v2  a   a   a   a  )
  (  v1  v2  a   a   a  )           (  v1  v2  a   a   a   a  )
  (  v1  v2  a   a   a  )

where a denotes an element of the original matrix which  is  unchanged,
vi denotes an element of the vector defining H(i), and ui an element of
the vector defining G(i).
```

90.137.3 fortran code

— dlabrd.f —

```
*  =====================================================================
      SUBROUTINE DLABRD( M, N, NB, A, LDA, D, E, TAUQ, TAUP, X, LDX, Y,
     $                   LDY )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            LDA, LDX, LDY, M, N, NB
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), D( * ), E( * ), TAUP( * ),
     $                   TAUQ( * ), X( LDX, * ), Y( LDY, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
```

```
*     ..
*     .. Local Scalars ..
      INTEGER            I
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEMV, DLARFG, DSCAL
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MIN
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( M.LE.0 .OR. N.LE.0 )
     $   RETURN
*
      IF( M.GE.N ) THEN
*
*        Reduce to upper bidiagonal form
*
         DO 10 I = 1, NB
*
*           Update A(i:m,i)
*
            CALL DGEMV( 'No transpose', M-I+1, I-1, -ONE, A( I, 1 ),
     $                  LDA, Y( I, 1 ), LDY, ONE, A( I, I ), 1 )
            CALL DGEMV( 'No transpose', M-I+1, I-1, -ONE, X( I, 1 ),
     $                  LDX, A( 1, I ), 1, ONE, A( I, I ), 1 )
*
*           Generate reflection Q(i) to annihilate A(i+1:m,i)
*
            CALL DLARFG( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
     $                   TAUQ( I ) )
            D( I ) = A( I, I )
            IF( I.LT.N ) THEN
               A( I, I ) = ONE
*
*              Compute Y(i+1:n,i)
*
               CALL DGEMV( 'Transpose', M-I+1, N-I, ONE, A( I, I+1 ),
     $                     LDA, A( I, I ), 1, ZERO, Y( I+1, I ), 1 )
               CALL DGEMV( 'Transpose', M-I+1, I-1, ONE, A( I, 1 ), LDA,
     $                     A( I, I ), 1, ZERO, Y( 1, I ), 1 )
               CALL DGEMV( 'No transpose', N-I, I-1, -ONE, Y( I+1, 1 ),
     $                     LDY, Y( 1, I ), 1, ONE, Y( I+1, I ), 1 )
               CALL DGEMV( 'Transpose', M-I+1, I-1, ONE, X( I, 1 ), LDX,
     $                     A( I, I ), 1, ZERO, Y( 1, I ), 1 )
               CALL DGEMV( 'Transpose', I-1, N-I, -ONE, A( 1, I+1 ),
     $                     LDA, Y( 1, I ), 1, ONE, Y( I+1, I ), 1 )
```

```
               CALL DSCAL( N-I, TAUQ( I ), Y( I+1, I ), 1 )
*
*              Update A(i,i+1:n)
*
               CALL DGEMV( 'No transpose', N-I, I, -ONE, Y( I+1, 1 ),
     $                     LDY, A( I, 1 ), LDA, ONE, A( I, I+1 ), LDA )
               CALL DGEMV( 'Transpose', I-1, N-I, -ONE, A( 1, I+1 ),
     $                     LDA, X( I, 1 ), LDX, ONE, A( I, I+1 ), LDA )
*
*              Generate reflection P(i) to annihilate A(i,i+2:n)
*
               CALL DLARFG( N-I, A( I, I+1 ), A( I, MIN( I+2, N ) ),
     $                      LDA, TAUP( I ) )
               E( I ) = A( I, I+1 )
               A( I, I+1 ) = ONE
*
*              Compute X(i+1:m,i)
*
               CALL DGEMV( 'No transpose', M-I, N-I, ONE, A( I+1, I+1 ),
     $                     LDA, A( I, I+1 ), LDA, ZERO, X( I+1, I ), 1 )
               CALL DGEMV( 'Transpose', N-I, I, ONE, Y( I+1, 1 ), LDY,
     $                     A( I, I+1 ), LDA, ZERO, X( 1, I ), 1 )
               CALL DGEMV( 'No transpose', M-I, I, -ONE, A( I+1, 1 ),
     $                     LDA, X( 1, I ), 1, ONE, X( I+1, I ), 1 )
               CALL DGEMV( 'No transpose', I-1, N-I, ONE, A( 1, I+1 ),
     $                     LDA, A( I, I+1 ), LDA, ZERO, X( 1, I ), 1 )
               CALL DGEMV( 'No transpose', M-I, I-1, -ONE, X( I+1, 1 ),
     $                     LDX, X( 1, I ), 1, ONE, X( I+1, I ), 1 )
               CALL DSCAL( M-I, TAUP( I ), X( I+1, I ), 1 )
            END IF
   10    CONTINUE
      ELSE
*
*        Reduce to lower bidiagonal form
*
         DO 20 I = 1, NB
*
*           Update A(i,i:n)
*
            CALL DGEMV( 'No transpose', N-I+1, I-1, -ONE, Y( I, 1 ),
     $                  LDY, A( I, 1 ), LDA, ONE, A( I, I ), LDA )
            CALL DGEMV( 'Transpose', I-1, N-I+1, -ONE, A( 1, I ), LDA,
     $                  X( I, 1 ), LDX, ONE, A( I, I ), LDA )
*
*           Generate reflection P(i) to annihilate A(i,i+1:n)
*
            CALL DLARFG( N-I+1, A( I, I ), A( I, MIN( I+1, N ) ), LDA,
     $                   TAUP( I ) )
            D( I ) = A( I, I )
            IF( I.LT.M ) THEN
```

```
              A( I, I ) = ONE
*
*             Compute X(i+1:m,i)
*
              CALL DGEMV( 'No transpose', M-I, N-I+1, ONE, A( I+1, I ),
     $                    LDA, A( I, I ), LDA, ZERO, X( I+1, I ), 1 )
              CALL DGEMV( 'Transpose', N-I+1, I-1, ONE, Y( I, 1 ), LDY,
     $                    A( I, I ), LDA, ZERO, X( 1, I ), 1 )
              CALL DGEMV( 'No transpose', M-I, I-1, -ONE, A( I+1, 1 ),
     $                    LDA, X( 1, I ), 1, ONE, X( I+1, I ), 1 )
              CALL DGEMV( 'No transpose', I-1, N-I+1, ONE, A( 1, I ),
     $                    LDA, A( I, I ), LDA, ZERO, X( 1, I ), 1 )
              CALL DGEMV( 'No transpose', M-I, I-1, -ONE, X( I+1, 1 ),
     $                    LDX, X( 1, I ), 1, ONE, X( I+1, I ), 1 )
              CALL DSCAL( M-I, TAUP( I ), X( I+1, I ), 1 )
*
*             Update A(i+1:m,i)
*
              CALL DGEMV( 'No transpose', M-I, I-1, -ONE, A( I+1, 1 ),
     $                    LDA, Y( I, 1 ), LDY, ONE, A( I+1, I ), 1 )
              CALL DGEMV( 'No transpose', M-I, I, -ONE, X( I+1, 1 ),
     $                    LDX, A( 1, I ), 1, ONE, A( I+1, I ), 1 )
*
*             Generate reflection Q(i) to annihilate A(i+2:m,i)
*
              CALL DLARFG( M-I, A( I+1, I ), A( MIN( I+2, M ), I ), 1,
     $                     TAUQ( I ) )
              E( I ) = A( I+1, I )
              A( I+1, I ) = ONE
*
*             Compute Y(i+1:n,i)
*
              CALL DGEMV( 'Transpose', M-I, N-I, ONE, A( I+1, I+1 ),
     $                    LDA, A( I+1, I ), 1, ZERO, Y( I+1, I ), 1 )
              CALL DGEMV( 'Transpose', M-I, I-1, ONE, A( I+1, 1 ), LDA,
     $                    A( I+1, I ), 1, ZERO, Y( 1, I ), 1 )
              CALL DGEMV( 'No transpose', N-I, I-1, -ONE, Y( I+1, 1 ),
     $                    LDY, Y( 1, I ), 1, ONE, Y( I+1, I ), 1 )
              CALL DGEMV( 'Transpose', M-I, I, ONE, X( I+1, 1 ), LDX,
     $                    A( I+1, I ), 1, ZERO, Y( 1, I ), 1 )
              CALL DGEMV( 'Transpose', I, N-I, -ONE, A( 1, I+1 ), LDA,
     $                    Y( 1, I ), 1, ONE, Y( I+1, I ), 1 )
              CALL DSCAL( N-I, TAUQ( I ), Y( I+1, I ), 1 )
           END IF
   20    CONTINUE
      END IF
      RETURN
*
*     End of DLABRD
*
```

```
      END
      SUBROUTINE DLABRD( M, N, NB, A, LDA, D, E, TAUQ, TAUP, X, LDX, Y,
     $                   LDY )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     February 29, 1992
*
*     .. Scalar Arguments ..
      INTEGER            LDA, LDX, LDY, M, N, NB
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), D( * ), E( * ), TAUP( * ),
     $                   TAUQ( * ), X( LDX, * ), Y( LDY, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEMV, DLARFG, DSCAL
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MIN
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( M.LE.0 .OR. N.LE.0 )
     $   RETURN
*
      IF( M.GE.N ) THEN
*
*        Reduce to upper bidiagonal form
*
         DO 10 I = 1, NB
*
*           Update A(i:m,i)
*
            CALL DGEMV( 'No transpose', M-I+1, I-1, -ONE, A( I, 1 ),
     $                  LDA, Y( I, 1 ), LDY, ONE, A( I, I ), 1 )
            CALL DGEMV( 'No transpose', M-I+1, I-1, -ONE, X( I, 1 ),
```

```
     $                 LDX, A( 1, I ), 1, ONE, A( I, I ), 1 )
*
*          Generate reflection Q(i) to annihilate A(i+1:m,i)
*
           CALL DLARFG( M-I+1, A( I, I ), A( MIN( I+1, M ), I ), 1,
     $                  TAUQ( I ) )
           D( I ) = A( I, I )
           IF( I.LT.N ) THEN
              A( I, I ) = ONE
*
*             Compute Y(i+1:n,i)
*
              CALL DGEMV( 'Transpose', M-I+1, N-I, ONE, A( I, I+1 ),
     $                    LDA, A( I, I ), 1, ZERO, Y( I+1, I ), 1 )
              CALL DGEMV( 'Transpose', M-I+1, I-1, ONE, A( I, 1 ), LDA,
     $                    A( I, I ), 1, ZERO, Y( 1, I ), 1 )
              CALL DGEMV( 'No transpose', N-I, I-1, -ONE, Y( I+1, 1 ),
     $                    LDY, Y( 1, I ), 1, ONE, Y( I+1, I ), 1 )
              CALL DGEMV( 'Transpose', M-I+1, I-1, ONE, X( I, 1 ), LDX,
     $                    A( I, I ), 1, ZERO, Y( 1, I ), 1 )
              CALL DGEMV( 'Transpose', I-1, N-I, -ONE, A( 1, I+1 ),
     $                    LDA, Y( 1, I ), 1, ONE, Y( I+1, I ), 1 )
              CALL DSCAL( N-I, TAUQ( I ), Y( I+1, I ), 1 )
*
*             Update A(i,i+1:n)
*
              CALL DGEMV( 'No transpose', N-I, I, -ONE, Y( I+1, 1 ),
     $                    LDY, A( I, 1 ), LDA, ONE, A( I, I+1 ), LDA )
              CALL DGEMV( 'Transpose', I-1, N-I, -ONE, A( 1, I+1 ),
     $                    LDA, X( I, 1 ), LDX, ONE, A( I, I+1 ), LDA )
*
*             Generate reflection P(i) to annihilate A(i,i+2:n)
*
              CALL DLARFG( N-I, A( I, I+1 ), A( I, MIN( I+2, N ) ),
     $                     LDA, TAUP( I ) )
              E( I ) = A( I, I+1 )
              A( I, I+1 ) = ONE
*
*             Compute X(i+1:m,i)
*
              CALL DGEMV( 'No transpose', M-I, N-I, ONE, A( I+1, I+1 ),
     $                    LDA, A( I, I+1 ), LDA, ZERO, X( I+1, I ), 1 )
              CALL DGEMV( 'Transpose', N-I, I, ONE, Y( I+1, 1 ), LDY,
     $                    A( I, I+1 ), LDA, ZERO, X( 1, I ), 1 )
              CALL DGEMV( 'No transpose', M-I, I, -ONE, A( I+1, 1 ),
     $                    LDA, X( 1, I ), 1, ONE, X( I+1, I ), 1 )
              CALL DGEMV( 'No transpose', I-1, N-I, ONE, A( 1, I+1 ),
     $                    LDA, A( I, I+1 ), LDA, ZERO, X( 1, I ), 1 )
              CALL DGEMV( 'No transpose', M-I, I-1, -ONE, X( I+1, 1 ),
     $                    LDX, X( 1, I ), 1, ONE, X( I+1, I ), 1 )
```

```
            CALL DSCAL( M-I, TAUP( I ), X( I+1, I ), 1 )
         END IF
   10    CONTINUE
      ELSE
*
*        Reduce to lower bidiagonal form
*
         DO 20 I = 1, NB
*
*           Update A(i,i:n)
*
            CALL DGEMV( 'No transpose', N-I+1, I-1, -ONE, Y( I, 1 ),
     $                  LDY, A( I, 1 ), LDA, ONE, A( I, I ), LDA )
            CALL DGEMV( 'Transpose', I-1, N-I+1, -ONE, A( 1, I ), LDA,
     $                  X( I, 1 ), LDX, ONE, A( I, I ), LDA )
*
*           Generate reflection P(i) to annihilate A(i,i+1:n)
*
            CALL DLARFG( N-I+1, A( I, I ), A( I, MIN( I+1, N ) ), LDA,
     $                   TAUP( I ) )
            D( I ) = A( I, I )
            IF( I.LT.M ) THEN
               A( I, I ) = ONE
*
*              Compute X(i+1:m,i)
*
               CALL DGEMV( 'No transpose', M-I, N-I+1, ONE, A( I+1, I ),
     $                     LDA, A( I, I ), LDA, ZERO, X( I+1, I ), 1 )
               CALL DGEMV( 'Transpose', N-I+1, I-1, ONE, Y( I, 1 ), LDY,
     $                     A( I, I ), LDA, ZERO, X( 1, I ), 1 )
               CALL DGEMV( 'No transpose', M-I, I-1, -ONE, A( I+1, 1 ),
     $                     LDA, X( 1, I ), 1, ONE, X( I+1, I ), 1 )
               CALL DGEMV( 'No transpose', I-1, N-I+1, ONE, A( 1, I ),
     $                     LDA, A( I, I ), LDA, ZERO, X( 1, I ), 1 )
               CALL DGEMV( 'No transpose', M-I, I-1, -ONE, X( I+1, 1 ),
     $                     LDX, X( 1, I ), 1, ONE, X( I+1, I ), 1 )
               CALL DSCAL( M-I, TAUP( I ), X( I+1, I ), 1 )
*
*              Update A(i+1:m,i)
*
               CALL DGEMV( 'No transpose', M-I, I-1, -ONE, A( I+1, 1 ),
     $                     LDA, Y( I, 1 ), LDY, ONE, A( I+1, I ), 1 )
               CALL DGEMV( 'No transpose', M-I, I, -ONE, X( I+1, 1 ),
     $                     LDX, A( 1, I ), 1, ONE, A( I+1, I ), 1 )
*
*              Generate reflection Q(i) to annihilate A(i+2:m,i)
*
               CALL DLARFG( M-I, A( I+1, I ), A( MIN( I+2, M ), I ), 1,
     $                      TAUQ( I ) )
               E( I ) = A( I+1, I )
```

```
              A( I+1, I ) = ONE
*
*             Compute Y(i+1:n,i)
*
              CALL DGEMV( 'Transpose', M-I, N-I, ONE, A( I+1, I+1 ),
     $                    LDA, A( I+1, I ), 1, ZERO, Y( I+1, I ), 1 )
              CALL DGEMV( 'Transpose', M-I, I-1, ONE, A( I+1, 1 ), LDA,
     $                    A( I+1, I ), 1, ZERO, Y( 1, I ), 1 )
              CALL DGEMV( 'No transpose', N-I, I-1, -ONE, Y( I+1, 1 ),
     $                    LDY, Y( 1, I ), 1, ONE, Y( I+1, I ), 1 )
              CALL DGEMV( 'Transpose', M-I, I, ONE, X( I+1, 1 ), LDX,
     $                    A( I+1, I ), 1, ZERO, Y( 1, I ), 1 )
              CALL DGEMV( 'Transpose', I, N-I, -ONE, A( 1, I+1 ), LDA,
     $                    Y( 1, I ), 1, ONE, Y( I+1, I ), 1 )
              CALL DSCAL( N-I, TAUQ( I ), Y( I+1, I ), 1 )
           END IF
   20    CONTINUE
      END IF
      RETURN
*
*     End of DLABRD
*
      END
```

———-

90.137.4 lisp code

— dlabrd.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dlabrd (m n nb a lda d e tauq taup x ldx y ldy)
    (declare (type (array double-float (*)) y x taup tauq e d a)
             (type (f2cl-lib:integer4) ldy ldx lda nb n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%)
         (tauq double-float tauq-%data% tauq-%offset%)
         (taup double-float taup-%data% taup-%offset%)
         (x double-float x-%data% x-%offset%)
         (y double-float y-%data% y-%offset%))
      (prog ((i 0))
        (declare (type (f2cl-lib:integer4) i))
```

```
(if (or (<= m 0) (<= n 0))
    (go end_label))
(cond
 ((>= m n)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i nb) nil)
    (tagbody
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (dgemv "No transpose"
           (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
           (f2cl-lib:int-sub i 1) (- one)
           (f2cl-lib:array-slice a-%data% double-float (i 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice y-%data% double-float (i 1)
                                 ((1 ldy) (1 *)) y-%offset%)
           ldy one
           (f2cl-lib:array-slice a-%data% double-float (i i)
                                 ((1 lda) (1 *)) a-%offset%)
           1)
        (declare
         (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9 var-10))
        (when var-5 (setf lda var-5))
        (when var-7 (setf ldy var-7))
        (when var-8 (setf one var-8)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (dgemv "No transpose"
           (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
           (f2cl-lib:int-sub i 1) (- one)
           (f2cl-lib:array-slice x-%data% double-float (i 1)
                                 ((1 ldx) (1 *)) x-%offset%)
           ldx
           (f2cl-lib:array-slice a-%data% double-float (1 i)
                                 ((1 lda) (1 *)) a-%offset%)
           1 one
           (f2cl-lib:array-slice a-%data% double-float (i i)
                                 ((1 lda) (1 *)) a-%offset%)
           1)
        (declare
         (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7 var-9
          var-10))
        (when var-5 (setf ldx var-5))
        (when var-8 (setf one var-8)))
      (dlarfg (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
       (f2cl-lib:array-slice a-%data% double-float (i i)
                             ((1 lda) (1 *)) a-%offset%)
```

```
 (f2cl-lib:array-slice a-%data% double-float
                       ((min (f2cl-lib:int-add i 1) m) i)
                       ((1 lda) (1 *)) a-%offset%)
 1
 (f2cl-lib:array-slice tauq-%data% double-float (i) ((1 *))
                       tauq-%offset%))
(setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
        (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
                       a-%offset%))
(cond
 ((< i n)
  (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
          one)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (dgemv "Transpose"
       (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
       (f2cl-lib:int-sub n i) one
       (f2cl-lib:array-slice a-%data% double-float
                             (i (f2cl-lib:int-add i 1))
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice a-%data% double-float (i i)
                             ((1 lda) (1 *)) a-%offset%)
       1 zero
       (f2cl-lib:array-slice y-%data% double-float ((+ i 1) i)
                             ((1 ldy) (1 *)) y-%offset%)
       1)
    (declare
     (ignore var-0 var-1 var-2 var-4 var-6 var-7 var-9 var-10))
    (when var-3 (setf one var-3))
    (when var-5 (setf lda var-5))
    (when var-8 (setf zero var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (dgemv "Transpose"
       (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
       (f2cl-lib:int-sub i 1) one
       (f2cl-lib:array-slice a-%data% double-float (i 1)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice a-%data% double-float (i i)
                             ((1 lda) (1 *)) a-%offset%)
       1 zero
       (f2cl-lib:array-slice y-%data% double-float (1 i)
                             ((1 ldy) (1 *)) y-%offset%)
       1)
    (declare
```

```
   (ignore var-0 var-1 var-2 var-4 var-6 var-7 var-9 var-10))
  (when var-3 (setf one var-3))
  (when var-5 (setf lda var-5))
  (when var-8 (setf zero var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgemv "No transpose" (f2cl-lib:int-sub n i)
     (f2cl-lib:int-sub i 1) (- one)
     (f2cl-lib:array-slice y-%data% double-float ((+ i 1) 1)
                           ((1 ldy) (1 *)) y-%offset%)
     ldy
     (f2cl-lib:array-slice y-%data% double-float (1 i)
                           ((1 ldy) (1 *)) y-%offset%)
     1 one
     (f2cl-lib:array-slice y-%data% double-float ((+ i 1) i)
                           ((1 ldy) (1 *)) y-%offset%)
     1)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7 var-9
    var-10))
  (when var-5 (setf ldy var-5))
  (when var-8 (setf one var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgemv "Transpose"
     (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
     (f2cl-lib:int-sub i 1) one
     (f2cl-lib:array-slice x-%data% double-float (i 1)
                           ((1 ldx) (1 *)) x-%offset%)
     ldx
     (f2cl-lib:array-slice a-%data% double-float (i i)
                           ((1 lda) (1 *)) a-%offset%)
     1 zero
     (f2cl-lib:array-slice y-%data% double-float (1 i)
                           ((1 ldy) (1 *)) y-%offset%)
     1)
  (declare
   (ignore var-0 var-1 var-2 var-4 var-6 var-7 var-9 var-10))
  (when var-3 (setf one var-3))
  (when var-5 (setf ldx var-5))
  (when var-8 (setf zero var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgemv "Transpose" (f2cl-lib:int-sub i 1)
     (f2cl-lib:int-sub n i) (- one)
     (f2cl-lib:array-slice a-%data% double-float
                           (1 (f2cl-lib:int-add i 1))
```

```
                              ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice y-%data% double-float (1 i)
                              ((1 ldy) (1 *)) y-%offset%)
     1 one
     (f2cl-lib:array-slice y-%data% double-float ((+ i 1) i)
                              ((1 ldy) (1 *)) y-%offset%)
     1)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7 var-9
    var-10))
  (when var-5 (setf lda var-5))
  (when var-8 (setf one var-8)))
(dscal (f2cl-lib:int-sub n i)
 (f2cl-lib:array-slice tauq-%data% double-float (i) ((1 *))
                       tauq-%offset%)
 (f2cl-lib:array-slice y-%data% double-float ((+ i 1) i)
                       ((1 ldy) (1 *)) y-%offset%)
 1)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgemv "No transpose" (f2cl-lib:int-sub n i) i (- one)
     (f2cl-lib:array-slice y-%data% double-float ((+ i 1) 1)
                              ((1 ldy) (1 *)) y-%offset%)
     ldy
     (f2cl-lib:array-slice a-%data% double-float (i 1)
                              ((1 lda) (1 *)) a-%offset%)
     lda one
     (f2cl-lib:array-slice a-%data% double-float
                              (i (f2cl-lib:int-add i 1))
                              ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-3 var-4 var-6 var-9))
  (when var-2 (setf i var-2))
  (when var-5 (setf ldy var-5))
  (when var-7 (setf lda var-7))
  (when var-8 (setf one var-8))
  (when var-10 (setf lda var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgemv "Transpose" (f2cl-lib:int-sub i 1)
     (f2cl-lib:int-sub n i) (- one)
     (f2cl-lib:array-slice a-%data% double-float
                              (1 (f2cl-lib:int-add i 1))
                              ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice x-%data% double-float (i 1)
                              ((1 ldx) (1 *)) x-%offset%)
```

```
      ldx one
      (f2cl-lib:array-slice a-%data% double-float
                            (i (f2cl-lib:int-add i 1))
                            ((1 lda) (1 *)) a-%offset%)
      lda)
  (declare (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9))
  (when var-5 (setf lda var-5))
  (when var-7 (setf ldx var-7))
  (when var-8 (setf one var-8))
  (when var-10 (setf lda var-10)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dlarfg (f2cl-lib:int-sub n i)
     (f2cl-lib:array-slice a-%data% double-float
                           (i (f2cl-lib:int-add i 1))
                           ((1 lda) (1 *)) a-%offset%)
     (f2cl-lib:array-slice a-%data% double-float
                           (i
                            (min
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add i 2))
                             (the f2cl-lib:integer4 n)))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice taup-%data% double-float (i) ((1 *))
                           taup-%offset%))
  (declare (ignore var-0 var-1 var-2 var-4))
  (when var-3 (setf lda var-3)))
(setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
        (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
                       ((1 lda) (1 *)) a-%offset%))
(setf (f2cl-lib:fref a-%data% (i (f2cl-lib:int-add i 1))
                      ((1 lda) (1 *)) a-%offset%)
        one)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgemv "No transpose" (f2cl-lib:int-sub m i)
     (f2cl-lib:int-sub n i) one
     (f2cl-lib:array-slice a-%data% double-float
                           ((+ i 1) (f2cl-lib:int-add i 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice a-%data% double-float
                           (i (f2cl-lib:int-add i 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda zero
     (f2cl-lib:array-slice x-%data% double-float ((+ i 1) i)
                           ((1 ldx) (1 *)) x-%offset%)
     1)
  (declare (ignore var-0 var-1 var-2 var-4 var-6 var-9 var-10))
```

```
  (when var-3 (setf one var-3))
  (when var-5 (setf lda var-5))
  (when var-7 (setf lda var-7))
  (when var-8 (setf zero var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgemv "Transpose" (f2cl-lib:int-sub n i) i one
     (f2cl-lib:array-slice y-%data% double-float ((+ i 1) 1)
                           ((1 ldy) (1 *)) y-%offset%)
     ldy
     (f2cl-lib:array-slice a-%data% double-float
                           (i (f2cl-lib:int-add i 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda zero
     (f2cl-lib:array-slice x-%data% double-float (1 i)
                           ((1 ldx) (1 *)) x-%offset%)
     1)
  (declare (ignore var-0 var-1 var-4 var-6 var-9 var-10))
  (when var-2 (setf i var-2))
  (when var-3 (setf one var-3))
  (when var-5 (setf ldy var-5))
  (when var-7 (setf lda var-7))
  (when var-8 (setf zero var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgemv "No transpose" (f2cl-lib:int-sub m i) i (- one)
     (f2cl-lib:array-slice a-%data% double-float ((+ i 1) 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice x-%data% double-float (1 i)
                           ((1 ldx) (1 *)) x-%offset%)
     1 one
     (f2cl-lib:array-slice x-%data% double-float ((+ i 1) i)
                           ((1 ldx) (1 *)) x-%offset%)
     1)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9 var-10))
  (when var-2 (setf i var-2))
  (when var-5 (setf lda var-5))
  (when var-8 (setf one var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgemv "No transpose" (f2cl-lib:int-sub i 1)
     (f2cl-lib:int-sub n i) one
     (f2cl-lib:array-slice a-%data% double-float
                           (1 (f2cl-lib:int-add i 1))
                           ((1 lda) (1 *)) a-%offset%)
```

```
                lda
                (f2cl-lib:array-slice a-%data% double-float
                                      (i (f2cl-lib:int-add i 1))
                                      ((1 lda) (1 *)) a-%offset%)
                lda zero
                (f2cl-lib:array-slice x-%data% double-float (1 i)
                                      ((1 ldx) (1 *)) x-%offset%)
                1)
              (declare (ignore var-0 var-1 var-2 var-4 var-6 var-9 var-10))
              (when var-3 (setf one var-3))
              (when var-5 (setf lda var-5))
              (when var-7 (setf lda var-7))
              (when var-8 (setf zero var-8)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10)
                (dgemv "No transpose" (f2cl-lib:int-sub m i)
                 (f2cl-lib:int-sub i 1) (- one)
                 (f2cl-lib:array-slice x-%data% double-float ((+ i 1) 1)
                                       ((1 ldx) (1 *)) x-%offset%)
                 ldx
                 (f2cl-lib:array-slice x-%data% double-float (1 i)
                                       ((1 ldx) (1 *)) x-%offset%)
                 1 one
                 (f2cl-lib:array-slice x-%data% double-float ((+ i 1) i)
                                       ((1 ldx) (1 *)) x-%offset%)
                 1)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7 var-9
                var-10))
              (when var-5 (setf ldx var-5))
              (when var-8 (setf one var-8)))
            (dscal (f2cl-lib:int-sub m i)
             (f2cl-lib:array-slice taup-%data% double-float (i) ((1 *))
                                   taup-%offset%)
             (f2cl-lib:array-slice x-%data% double-float ((+ i 1) i)
                                   ((1 ldx) (1 *)) x-%offset%)
             1)))
        label10)))
    (t
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i nb) nil)
       (tagbody
         (multiple-value-bind
             (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
              var-10)
             (dgemv "No transpose"
              (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
              (f2cl-lib:int-sub i 1) (- one)
              (f2cl-lib:array-slice y-%data% double-float (i 1)
```

```
                          ((1 ldy) (1 *)) y-%offset%)
     ldy
     (f2cl-lib:array-slice a-%data% double-float (i 1)
                          ((1 lda) (1 *)) a-%offset%)
     lda one
     (f2cl-lib:array-slice a-%data% double-float (i i)
                          ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9))
  (when var-5 (setf ldy var-5))
  (when var-7 (setf lda var-7))
  (when var-8 (setf one var-8))
  (when var-10 (setf lda var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dgemv "Transpose" (f2cl-lib:int-sub i 1)
     (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) (- one)
     (f2cl-lib:array-slice a-%data% double-float (1 i)
                          ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice x-%data% double-float (i 1)
                          ((1 ldx) (1 *)) x-%offset%)
     ldx one
     (f2cl-lib:array-slice a-%data% double-float (i i)
                          ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9))
  (when var-5 (setf lda var-5))
  (when var-7 (setf ldx var-7))
  (when var-8 (setf one var-8))
  (when var-10 (setf lda var-10)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dlarfg (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
     (f2cl-lib:array-slice a-%data% double-float (i i)
                          ((1 lda) (1 *)) a-%offset%)
     (f2cl-lib:array-slice a-%data% double-float
                          (i
                           (min
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add i 1))
                            (the f2cl-lib:integer4 n)))
                          ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice taup-%data% double-float (i) ((1 *))
                          taup-%offset%))
  (declare (ignore var-0 var-1 var-2 var-4))
  (when var-3 (setf lda var-3)))
(setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
        (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *))
```

```
                        a-%offset%))
(cond
 ((< i m)
  (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
          one)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (dgemv "No transpose" (f2cl-lib:int-sub m i)
       (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) one
       (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice a-%data% double-float (i i)
                             ((1 lda) (1 *)) a-%offset%)
       lda zero
       (f2cl-lib:array-slice x-%data% double-float ((+ i 1) i)
                             ((1 ldx) (1 *)) x-%offset%)
       1)
    (declare (ignore var-0 var-1 var-2 var-4 var-6 var-9 var-10))
    (when var-3 (setf one var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf lda var-7))
    (when var-8 (setf zero var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (dgemv "Transpose"
       (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
       (f2cl-lib:int-sub i 1) one
       (f2cl-lib:array-slice y-%data% double-float (i 1)
                             ((1 ldy) (1 *)) y-%offset%)
       ldy
       (f2cl-lib:array-slice a-%data% double-float (i i)
                             ((1 lda) (1 *)) a-%offset%)
       lda zero
       (f2cl-lib:array-slice x-%data% double-float (1 i)
                             ((1 ldx) (1 *)) x-%offset%)
       1)
    (declare (ignore var-0 var-1 var-2 var-4 var-6 var-9 var-10))
    (when var-3 (setf one var-3))
    (when var-5 (setf ldy var-5))
    (when var-7 (setf lda var-7))
    (when var-8 (setf zero var-8)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (dgemv "No transpose" (f2cl-lib:int-sub m i)
       (f2cl-lib:int-sub i 1) (- one)
       (f2cl-lib:array-slice a-%data% double-float ((+ i 1) 1)
```

```
                                ((1 lda) (1 *)) a-%offset%)
                        lda
                        (f2cl-lib:array-slice x-%data% double-float (1 i)
                                ((1 ldx) (1 *)) x-%offset%)
                        1 one
                        (f2cl-lib:array-slice x-%data% double-float ((+ i 1) i)
                                ((1 ldx) (1 *)) x-%offset%)
                        1)
                    (declare
                     (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7 var-9
                      var-10))
                    (when var-5 (setf lda var-5))
                    (when var-8 (setf one var-8)))
                  (multiple-value-bind
                      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                       var-9 var-10)
                      (dgemv "No transpose" (f2cl-lib:int-sub i 1)
                       (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) one
                       (f2cl-lib:array-slice a-%data% double-float (1 i)
                                ((1 lda) (1 *)) a-%offset%)
                       lda
                       (f2cl-lib:array-slice a-%data% double-float (i i)
                                ((1 lda) (1 *)) a-%offset%)
                       lda zero
                       (f2cl-lib:array-slice x-%data% double-float (1 i)
                                ((1 ldx) (1 *)) x-%offset%)
                       1)
                    (declare (ignore var-0 var-1 var-2 var-4 var-6 var-9 var-10))
                    (when var-3 (setf one var-3))
                    (when var-5 (setf lda var-5))
                    (when var-7 (setf lda var-7))
                    (when var-8 (setf zero var-8)))
                  (multiple-value-bind
                      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                       var-9 var-10)
                      (dgemv "No transpose" (f2cl-lib:int-sub m i)
                       (f2cl-lib:int-sub i 1) (- one)
                       (f2cl-lib:array-slice x-%data% double-float ((+ i 1) 1)
                                ((1 ldx) (1 *)) x-%offset%)
                       ldx
                       (f2cl-lib:array-slice x-%data% double-float (1 i)
                                ((1 ldx) (1 *)) x-%offset%)
                       1 one
                       (f2cl-lib:array-slice x-%data% double-float ((+ i 1) i)
                                ((1 ldx) (1 *)) x-%offset%)
                       1)
                    (declare
                     (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7 var-9
                      var-10))
                    (when var-5 (setf ldx var-5))
```

```
  (when var-8 (setf one var-8)))
(dscal (f2cl-lib:int-sub m i)
 (f2cl-lib:array-slice taup-%data% double-float (i) ((1 *))
                       taup-%offset%)
 (f2cl-lib:array-slice x-%data% double-float ((+ i 1) i)
                       ((1 ldx) (1 *)) x-%offset%)
 1)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgemv "No transpose" (f2cl-lib:int-sub m i)
     (f2cl-lib:int-sub i 1) (- one)
     (f2cl-lib:array-slice a-%data% double-float ((+ i 1) 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice y-%data% double-float (i 1)
                           ((1 ldy) (1 *)) y-%offset%)
     ldy one
     (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                           ((1 lda) (1 *)) a-%offset%)
     1)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9 var-10))
  (when var-5 (setf lda var-5))
  (when var-7 (setf ldy var-7))
  (when var-8 (setf one var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgemv "No transpose" (f2cl-lib:int-sub m i) i (- one)
     (f2cl-lib:array-slice x-%data% double-float ((+ i 1) 1)
                           ((1 ldx) (1 *)) x-%offset%)
     ldx
     (f2cl-lib:array-slice a-%data% double-float (1 i)
                           ((1 lda) (1 *)) a-%offset%)
     1 one
     (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                           ((1 lda) (1 *)) a-%offset%)
     1)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-6 var-7 var-9 var-10))
  (when var-2 (setf i var-2))
  (when var-5 (setf ldx var-5))
  (when var-8 (setf one var-8)))
(dlarfg (f2cl-lib:int-sub m i)
 (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                       ((1 lda) (1 *)) a-%offset%)
 (f2cl-lib:array-slice a-%data% double-float
                       ((min (f2cl-lib:int-add i 2) m) i)
                       ((1 lda) (1 *)) a-%offset%)
```

```
 1
 (f2cl-lib:array-slice tauq-%data% double-float (i) ((1 *))
                       tauq-%offset%))
(setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
        (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                       ((1 lda) (1 *)) a-%offset%))
(setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                     ((1 lda) (1 *)) a-%offset%)
        one)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgemv "Transpose" (f2cl-lib:int-sub m i)
     (f2cl-lib:int-sub n i) one
     (f2cl-lib:array-slice a-%data% double-float
                           ((+ i 1) (f2cl-lib:int-add i 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                           ((1 lda) (1 *)) a-%offset%)
     1 zero
     (f2cl-lib:array-slice y-%data% double-float ((+ i 1) i)
                           ((1 ldy) (1 *)) y-%offset%)
     1)
  (declare
   (ignore var-0 var-1 var-2 var-4 var-6 var-7 var-9 var-10))
  (when var-3 (setf one var-3))
  (when var-5 (setf lda var-5))
  (when var-8 (setf zero var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgemv "Transpose" (f2cl-lib:int-sub m i)
     (f2cl-lib:int-sub i 1) one
     (f2cl-lib:array-slice a-%data% double-float ((+ i 1) 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                           ((1 lda) (1 *)) a-%offset%)
     1 zero
     (f2cl-lib:array-slice y-%data% double-float (1 i)
                           ((1 ldy) (1 *)) y-%offset%)
     1)
  (declare
   (ignore var-0 var-1 var-2 var-4 var-6 var-7 var-9 var-10))
  (when var-3 (setf one var-3))
  (when var-5 (setf lda var-5))
  (when var-8 (setf zero var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
```

```
      var-9 var-10)
     (dgemv "No transpose" (f2cl-lib:int-sub n i)
      (f2cl-lib:int-sub i 1) (- one)
      (f2cl-lib:array-slice y-%data% double-float ((+ i 1) 1)
                            ((1 ldy) (1 *)) y-%offset%)
      ldy
      (f2cl-lib:array-slice y-%data% double-float (1 i)
                            ((1 ldy) (1 *)) y-%offset%)
      1 one
      (f2cl-lib:array-slice y-%data% double-float ((+ i 1) i)
                            ((1 ldy) (1 *)) y-%offset%)
      1)
   (declare
    (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7 var-9
     var-10))
   (when var-5 (setf ldy var-5))
   (when var-8 (setf one var-8)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10)
     (dgemv "Transpose" (f2cl-lib:int-sub m i) i one
      (f2cl-lib:array-slice x-%data% double-float ((+ i 1) 1)
                            ((1 ldx) (1 *)) x-%offset%)
      ldx
      (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                            ((1 lda) (1 *)) a-%offset%)
      1 zero
      (f2cl-lib:array-slice y-%data% double-float (1 i)
                            ((1 ldy) (1 *)) y-%offset%)
      1)
   (declare (ignore var-0 var-1 var-4 var-6 var-7 var-9 var-10))
   (when var-2 (setf i var-2))
   (when var-3 (setf one var-3))
   (when var-5 (setf ldx var-5))
   (when var-8 (setf zero var-8)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10)
     (dgemv "Transpose" i (f2cl-lib:int-sub n i) (- one)
      (f2cl-lib:array-slice a-%data% double-float
                            (1 (f2cl-lib:int-add i 1))
                            ((1 lda) (1 *)) a-%offset%)
      lda
      (f2cl-lib:array-slice y-%data% double-float (1 i)
                            ((1 ldy) (1 *)) y-%offset%)
      1 one
      (f2cl-lib:array-slice y-%data% double-float ((+ i 1) i)
                            ((1 ldy) (1 *)) y-%offset%)
      1)
   (declare
```

```
                 (ignore var-0 var-2 var-3 var-4 var-6 var-7 var-9 var-10))
                (when var-1 (setf i var-1))
                (when var-5 (setf lda var-5))
                (when var-8 (setf one var-8)))
              (dscal (f2cl-lib:int-sub n i)
               (f2cl-lib:array-slice tauq-%data% double-float (i) ((1 *))
                                     tauq-%offset%)
               (f2cl-lib:array-slice y-%data% double-float ((+ i 1) i)
                                     ((1 ldy) (1 *)) y-%offset%)
               1)))
            label20))))
       (go end_label)
      end_label
       (return
        (values nil nil nil nil lda nil nil nil nil nil ldx nil ldy))))))
```

———-

90.138 dlacon

90.138.1 Axiom unit tests

— **dlacon.input** —

```
)set break resume
)sys rm -f dlacon.output
)spool dlacon.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.138.2 Axiom help page

— **dlacon.help** —

```
========================================================================
dlacon examples
========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

NAME
       DLACON - the 1-norm of a square, real matrix A

SYNOPSIS
       SUBROUTINE DLACON( N, V, X, ISGN, EST, KASE )

           INTEGER        KASE, N

           DOUBLE         PRECISION EST

           INTEGER        ISGN( * )

           DOUBLE         PRECISION V( * ), X( * )

PURPOSE
       DLACON estimates the 1-norm of a square, real matrix A.  Reverse commu-
       nication is used for evaluating matrix-vector products.


ARGUMENTS
       N      (input) INTEGER
              The order of the matrix.  N >= 1.

       V      (workspace) DOUBLE PRECISION array, dimension (N)
              On the final return, V = A*W,  where  EST =  norm(V)/norm(W)  (W
              is not returned).

       X      (input/output) DOUBLE PRECISION array, dimension (N)
              On  an  intermediate  return,  X should be overwritten by A * X,
              if KASE=1, A' * X,  if KASE=2, and DLACON must be re-called with
              all the other parameters unchanged.

       ISGN   (workspace) INTEGER array, dimension (N)

       EST    (input/output) DOUBLE PRECISION
              On  entry  with  KASE  =  1  or  2  and  JUMP = 3, EST should be
              unchanged from the previous call to DLACON.  On exit, EST is  an
              estimate (a lower bound) for norm(A).

       KASE   (input/output) INTEGER
              On the initial call to DLACON, KASE should be 0.  On an interme-
              diate return, KASE will be 1 or 2, indicating whether  X  should
              be  overwritten  by  A * X  or A' * X.  On the final return from
              DLACON, KASE will again be 0.
```

```
FURTHER DETAILS
       Reference: N.J. Higham, "FORTRAN codes for estimating the one-norm of a
       real or complex matrix, with applications to condition estimation", ACM
       Trans. Math. Soft., vol. 14, no. 4, pp. 381-396, December 1988.
```

———-

90.138.3 fortran code

— dlacon.f —

```
*  =====================================================================
      SUBROUTINE DLACON( N, V, X, ISGN, EST, KASE )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      INTEGER            KASE, N
      DOUBLE PRECISION   EST
*     ..
*     .. Array Arguments ..
      INTEGER            ISGN( * )
      DOUBLE PRECISION   V( * ), X( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      DOUBLE PRECISION   ZERO, ONE, TWO
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0, TWO = 2.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, ITER, J, JLAST, JUMP
      DOUBLE PRECISION   ALTSGN, ESTOLD, TEMP
*     ..
*     .. External Functions ..
      INTEGER            IDAMAX
      DOUBLE PRECISION   DASUM
      EXTERNAL           IDAMAX, DASUM
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY
```

```
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, NINT, SIGN
*     ..
*     .. Save statement ..
      SAVE
*     ..
*     .. Executable Statements ..
*
      IF( KASE.EQ.0 ) THEN
         DO 10 I = 1, N
            X( I ) = ONE / DBLE( N )
   10    CONTINUE
         KASE = 1
         JUMP = 1
         RETURN
      END IF
*
      GO TO ( 20, 40, 70, 110, 140 )JUMP
*
*     ................ ENTRY   (JUMP = 1)
*     FIRST ITERATION.  X HAS BEEN OVERWRITTEN BY A*X.
*
   20 CONTINUE
      IF( N.EQ.1 ) THEN
         V( 1 ) = X( 1 )
         EST = ABS( V( 1 ) )
*        ... QUIT
         GO TO 150
      END IF
      EST = DASUM( N, X, 1 )
*
      DO 30 I = 1, N
         X( I ) = SIGN( ONE, X( I ) )
         ISGN( I ) = NINT( X( I ) )
   30 CONTINUE
      KASE = 2
      JUMP = 2
      RETURN
*
*     ................ ENTRY   (JUMP = 2)
*     FIRST ITERATION.  X HAS BEEN OVERWRITTEN BY TRANSPOSE(A)*X.
*
   40 CONTINUE
      J = IDAMAX( N, X, 1 )
      ITER = 2
*
*     MAIN LOOP - ITERATIONS 2,3,...,ITMAX.
*
   50 CONTINUE
```

```
      DO 60 I = 1, N
         X( I ) = ZERO
   60 CONTINUE
      X( J ) = ONE
      KASE = 1
      JUMP = 3
      RETURN
*
*     ................ ENTRY   (JUMP = 3)
*     X HAS BEEN OVERWRITTEN BY A*X.
*
   70 CONTINUE
      CALL DCOPY( N, X, 1, V, 1 )
      ESTOLD = EST
      EST = DASUM( N, V, 1 )
      DO 80 I = 1, N
         IF( NINT( SIGN( ONE, X( I ) ) ).NE.ISGN( I ) )
     $      GO TO 90
   80 CONTINUE
*     REPEATED SIGN VECTOR DETECTED, HENCE ALGORITHM HAS CONVERGED.
      GO TO 120
*
   90 CONTINUE
*     TEST FOR CYCLING.
      IF( EST.LE.ESTOLD )
     $   GO TO 120
*
      DO 100 I = 1, N
         X( I ) = SIGN( ONE, X( I ) )
         ISGN( I ) = NINT( X( I ) )
  100 CONTINUE
      KASE = 2
      JUMP = 4
      RETURN
*
*     ................ ENTRY   (JUMP = 4)
*     X HAS BEEN OVERWRITTEN BY TRANSPOSE(A)*X.
*
  110 CONTINUE
      JLAST = J
      J = IDAMAX( N, X, 1 )
      IF( ( X( JLAST ).NE.ABS( X( J ) ) ) .AND. ( ITER.LT.ITMAX ) ) THEN
         ITER = ITER + 1
         GO TO 50
      END IF
*
*     ITERATION COMPLETE.  FINAL STAGE.
*
  120 CONTINUE
      ALTSGN = ONE
```

```
      DO 130 I = 1, N
         X( I ) = ALTSGN*( ONE+DBLE( I-1 ) / DBLE( N-1 ) )
         ALTSGN = -ALTSGN
  130 CONTINUE
      KASE = 1
      JUMP = 5
      RETURN
*
*     ................ ENTRY   (JUMP = 5)
*     X HAS BEEN OVERWRITTEN BY A*X.
*
  140 CONTINUE
      TEMP = TWO*( DASUM( N, X, 1 ) / DBLE( 3*N ) )
      IF( TEMP.GT.EST ) THEN
         CALL DCOPY( N, X, 1, V, 1 )
         EST = TEMP
      END IF
*
  150 CONTINUE
      KASE = 0
      RETURN
*
*     End of DLACON
*
      END
      SUBROUTINE DLACON( N, V, X, ISGN, EST, KASE )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     February 29, 1992
*
*     .. Scalar Arguments ..
      INTEGER            KASE, N
      DOUBLE PRECISION   EST
*     ..
*     .. Array Arguments ..
      INTEGER            ISGN( * )
      DOUBLE PRECISION   V( * ), X( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 5 )
      DOUBLE PRECISION   ZERO, ONE, TWO
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0, TWO = 2.0D+0 )
*     ..
*     .. Local Scalars ..
```

```
      INTEGER            I, ITER, J, JLAST, JUMP
      DOUBLE PRECISION   ALTSGN, ESTOLD, TEMP
*     ..
*     .. External Functions ..
      INTEGER            IDAMAX
      DOUBLE PRECISION   DASUM
      EXTERNAL           IDAMAX, DASUM
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, NINT, SIGN
*     ..
*     .. Save statement ..
      SAVE
*     ..
*     .. Executable Statements ..
*
      IF( KASE.EQ.0 ) THEN
         DO 10 I = 1, N
            X( I ) = ONE / DBLE( N )
   10    CONTINUE
         KASE = 1
         JUMP = 1
         RETURN
      END IF
*
      GO TO ( 20, 40, 70, 110, 140 )JUMP
*
*     ................ ENTRY   (JUMP = 1)
*     FIRST ITERATION.  X HAS BEEN OVERWRITTEN BY A*X.
*
   20 CONTINUE
      IF( N.EQ.1 ) THEN
         V( 1 ) = X( 1 )
         EST = ABS( V( 1 ) )
*        ... QUIT
         GO TO 150
      END IF
      EST = DASUM( N, X, 1 )
*
      DO 30 I = 1, N
         X( I ) = SIGN( ONE, X( I ) )
         ISGN( I ) = NINT( X( I ) )
   30 CONTINUE
      KASE = 2
      JUMP = 2
      RETURN
*
```

```
*     ................ ENTRY   (JUMP = 2)
*     FIRST ITERATION.  X HAS BEEN OVERWRITTEN BY TRANDPOSE(A)*X.
*
   40 CONTINUE
      J = IDAMAX( N, X, 1 )
      ITER = 2
*
*     MAIN LOOP - ITERATIONS 2,3,...,ITMAX.
*
   50 CONTINUE
      DO 60 I = 1, N
         X( I ) = ZERO
   60 CONTINUE
      X( J ) = ONE
      KASE = 1
      JUMP = 3
      RETURN
*
*     ................ ENTRY   (JUMP = 3)
*     X HAS BEEN OVERWRITTEN BY A*X.
*
   70 CONTINUE
      CALL DCOPY( N, X, 1, V, 1 )
      ESTOLD = EST
      EST = DASUM( N, V, 1 )
      DO 80 I = 1, N
         IF( NINT( SIGN( ONE, X( I ) ) ).NE.ISGN( I ) )
     $      GO TO 90
   80 CONTINUE
*     REPEATED SIGN VECTOR DETECTED, HENCE ALGORITHM HAS CONVERGED.
      GO TO 120
*
   90 CONTINUE
*     TEST FOR CYCLING.
      IF( EST.LE.ESTOLD )
     $   GO TO 120
*
      DO 100 I = 1, N
         X( I ) = SIGN( ONE, X( I ) )
         ISGN( I ) = NINT( X( I ) )
  100 CONTINUE
      KASE = 2
      JUMP = 4
      RETURN
*
*     ................ ENTRY   (JUMP = 4)
*     X HAS BEEN OVERWRITTEN BY TRANDPOSE(A)*X.
*
  110 CONTINUE
      JLAST = J
```

```
      J = IDAMAX( N, X, 1 )
      IF( ( X( JLAST ).NE.ABS( X( J ) ) ) .AND. ( ITER.LT.ITMAX ) ) THEN
         ITER = ITER + 1
         GO TO 50
      END IF
*
*     ITERATION COMPLETE.  FINAL STAGE.
*
  120 CONTINUE
      ALTSGN = ONE
      DO 130 I = 1, N
         X( I ) = ALTSGN*( ONE+DBLE( I-1 ) / DBLE( N-1 ) )
         ALTSGN = -ALTSGN
  130 CONTINUE
      KASE = 1
      JUMP = 5
      RETURN
*
*     ................ ENTRY   (JUMP = 5)
*     X HAS BEEN OVERWRITTEN BY A*X.
*
  140 CONTINUE
      TEMP = TWO*( DASUM( N, X, 1 ) / DBLE( 3*N ) )
      IF( TEMP.GT.EST ) THEN
         CALL DCOPY( N, X, 1, V, 1 )
         EST = TEMP
      END IF
*
  150 CONTINUE
      KASE = 0
      RETURN
*
*     End of DLACON
*
      END
```

———-

90.138.4 lisp code

— dlacon.lisp —

```
(let* ((itmax 5) (zero 0.0d0) (one 1.0d0) (two 2.0d0))
  (declare (type (f2cl-lib:integer4 5 5) itmax)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
```

```
          (ignorable itmax zero one two))
  (let ((jump 0)
        (jlast 0)
        (j 0)
        (iter 0)
        (i 0)
        (temp 0.0d0)
        (estold 0.0d0)
        (altsgn 0.0d0))
    (declare (type (f2cl-lib:integer4) jump jlast j iter i)
             (type (double-float) temp estold altsgn))
    (defun dlacon (n v x isgn est kase)
      (declare (type (double-float) est)
               (type (array f2cl-lib:integer4 (*)) isgn)
               (type (array double-float (*)) x v)
               (type (f2cl-lib:integer4) kase n))
      (f2cl-lib:with-multi-array-data
          ((v double-float v-%data% v-%offset%)
           (x double-float x-%data% x-%offset%)
           (isgn f2cl-lib:integer4 isgn-%data% isgn-%offset%))
        (prog ()
          (declare)
          (cond
           ((= kase 0)
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i n) nil)
              (tagbody
                (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                        (/ one (f2cl-lib:dble n)))
               label10))
            (setf kase 1) (setf jump 1) (go end_label)))
          (f2cl-lib:computed-goto (label20 label40 label70 label110 label140)
                                  jump)
         label20
          (cond
           ((= n 1)
            (setf (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%)
                    (f2cl-lib:fref x-%data% (1) ((1 *)) x-%offset%))
            (setf est (abs (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%)))
            (go label150)))
          (setf est
                  (multiple-value-bind (ret-val var-0 var-1 var-2)
                      (dasum n x 1)
                    (declare (ignore var-1 var-2))
                    (when var-0 (setf n var-0))
                    ret-val))
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i n) nil)
            (tagbody
              (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
```

```
                (f2cl-lib:sign one
                               (f2cl-lib:fref x-%data% (i) ((1 *))
                                              x-%offset%)))
        (setf (f2cl-lib:fref isgn-%data% (i) ((1 *)) isgn-%offset%)
                (f2cl-lib:nint
                 (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)))
      label30))
    (setf kase 2)
    (setf jump 2)
    (go end_label)
   label40
    (setf j
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (idamax n x 1)
              (declare (ignore var-1 var-2))
              (when var-0 (setf n var-0))
              ret-val))
    (setf iter 2)
   label50
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%) zero)
      label60))
    (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%) one)
    (setf kase 1)
    (setf jump 3)
    (go end_label)
   label70
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy n x 1 v 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0)))
    (setf estold est)
    (setf est
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (dasum n v 1)
              (declare (ignore var-1 var-2))
              (when var-0 (setf n var-0))
              ret-val))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (if (/=
             (f2cl-lib:nint
              (f2cl-lib:sign one
                             (f2cl-lib:fref x-%data% (i) ((1 *))
                                            x-%offset%)))
             (f2cl-lib:fref isgn-%data% (i) ((1 *)) isgn-%offset%))
            (go label90))
```

```
    label80))
  (go label120)
 label90
  (if (<= est estold)
      (go label120))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
              (f2cl-lib:sign one
                             (f2cl-lib:fref x-%data% (i) ((1 *))
                                            x-%offset%)))
      (setf (f2cl-lib:fref isgn-%data% (i) ((1 *)) isgn-%offset%)
              (f2cl-lib:nint
               (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)))
    label100))
  (setf kase 2)
  (setf jump 4)
  (go end_label)
 label110
  (setf jlast j)
  (setf j
          (multiple-value-bind (ret-val var-0 var-1 var-2)
              (idamax n x 1)
            (declare (ignore var-1 var-2))
            (when var-0 (setf n var-0))
            ret-val))
  (cond
   ((and
     (/= (f2cl-lib:fref x (jlast) ((1 *)))
         (abs (f2cl-lib:fref x (j) ((1 *)))))
     (< iter itmax))
    (setf iter (f2cl-lib:int-add iter 1)) (go label50)))
 label120
  (setf altsgn one)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
              (* altsgn
                 (+ one
                    (/ (f2cl-lib:dble (f2cl-lib:int-sub i 1))
                       (f2cl-lib:dble (f2cl-lib:int-sub n 1))))))
      (setf altsgn (- altsgn))
    label130))
  (setf kase 1)
  (setf jump 5)
  (go end_label)
 label140
  (setf temp
```

```
                (* two
                   (/
                    (multiple-value-bind (ret-val var-0 var-1 var-2)
                        (dasum n x 1)
                      (declare (ignore var-1 var-2))
                      (when var-0 (setf n var-0))
                      ret-val)
                    (f2cl-lib:dble (f2cl-lib:int-mul 3 n)))))
          (cond
           ((> temp est)
            (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                (dcopy n x 1 v 1)
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf n var-0)))
            (setf est temp)))
         label150
          (setf kase 0)
          (go end_label)
         end_label
          (return (values n nil nil nil est kase)))))))
```

———-

90.139 dlacpy

90.139.1 Axiom unit tests

— **dlacpy.input** —

```
)set break resume
)sys rm -f dlacpy.output
)spool dlacpy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.139.2 Axiom help page

— **dlacpy.help** —

```
====================================================================
dlacpy examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DLACPY - all or part of a two-dimensional matrix A to another matrix B

SYNOPSIS
       SUBROUTINE DLACPY( UPLO, M, N, A, LDA, B, LDB )

           CHARACTER      UPLO

           INTEGER        LDA, LDB, M, N

           DOUBLE         PRECISION A( LDA, * ), B( LDB, * )

PURPOSE
       DLACPY  copies  all  or  part  of a two-dimensional matrix A to another
       matrix B.


ARGUMENTS
       UPLO    (input) CHARACTER*1
               Specifies the part of the matrix A to be copied to B.   =  'U':
               Upper triangular part
               = 'L':      Lower triangular part
               Otherwise:  All of the matrix A

       M       (input) INTEGER
               The number of rows of the matrix A.  M >= 0.

       N       (input) INTEGER
               The number of columns of the matrix A.  N >= 0.

       A       (input) DOUBLE PRECISION array, dimension (LDA,N)
               The m by n matrix A.  If UPLO = 'U', only the upper triangle or
               trapezoid is accessed; if UPLO = 'L', only the  lower  triangle
               or trapezoid is accessed.

       LDA     (input) INTEGER
               The leading dimension of the array A.  LDA >= max(1,M).

       B       (output) DOUBLE PRECISION array, dimension (LDB,N)
               On exit, B = A in the locations specified by UPLO.

       LDB     (input) INTEGER
```

```
              The leading dimension of the array B.  LDB >= max(1,M).
```

———-

90.139.3 fortran code

— **dlacpy.f** —

```
*  =====================================================================
      SUBROUTINE DLACPY( UPLO, M, N, A, LDA, B, LDB )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            LDA, LDB, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MIN
*     ..
*     .. Executable Statements ..
*
      IF( LSAME( UPLO, 'U' ) ) THEN
         DO 20 J = 1, N
            DO 10 I = 1, MIN( J, M )
               B( I, J ) = A( I, J )
   10       CONTINUE
   20    CONTINUE
      ELSE IF( LSAME( UPLO, 'L' ) ) THEN
         DO 40 J = 1, N
            DO 30 I = J, M
```

```
            B( I, J ) = A( I, J )
   30       CONTINUE
   40    CONTINUE
      ELSE
         DO 60 J = 1, N
            DO 50 I = 1, M
               B( I, J ) = A( I, J )
   50       CONTINUE
   60    CONTINUE
      END IF
      RETURN
*
*     End of DLACPY
*
      END
      SUBROUTINE DLACPY( UPLO, M, N, A, LDA, B, LDB )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     February 29, 1992
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            LDA, LDB, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MIN
*     ..
*     .. Executable Statements ..
*
      IF( LSAME( UPLO, 'U' ) ) THEN
         DO 20 J = 1, N
            DO 10 I = 1, MIN( J, M )
               B( I, J ) = A( I, J )
   10       CONTINUE
   20    CONTINUE
```

```
      ELSE IF( LSAME( UPLO, 'L' ) ) THEN
         DO 40 J = 1, N
            DO 30 I = J, M
               B( I, J ) = A( I, J )
   30       CONTINUE
   40    CONTINUE
      ELSE
         DO 60 J = 1, N
            DO 50 I = 1, M
               B( I, J ) = A( I, J )
   50       CONTINUE
   60    CONTINUE
      END IF
      RETURN
*
*     End of DLACPY
*
      END
```

90.139.4 lisp code

— dlacpy.lisp —

```
(defun dlacpy (uplo m n a lda b ldb$)
  (declare (type (array double-float (*)) b a)
           (type (f2cl-lib:integer4) ldb$ lda n m)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a double-float a-%data% a-%offset%)
       (b double-float b-%data% b-%offset%))
    (prog ((i 0) (j 0))
      (declare (type (f2cl-lib:integer4) j i))
      (cond
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i
                              (min (the f2cl-lib:integer4 j)
```

```
                                (the f2cl-lib:integer4 m)))
                        nil)
            (tagbody
              (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%))
             label10))
         label20)))
    ((multiple-value-bind (ret-val var-0 var-1)
         (lsame uplo "L")
       (declare (ignore var-1))
       (when var-0 (setf uplo var-0))
       ret-val)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%))
             label30))
         label40)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%))
             label50))
         label60))))
  (go end_label)
 end_label
  (return (values uplo nil nil nil nil nil nil)))))
```

———-

90.140 dladiv

90.140.1 Axiom unit tests

— dladiv.input —

```
)set break resume
)sys rm -f dladiv.output
)spool dladiv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.140.2 Axiom help page

— dladiv.help —

```
====================================================================
dladiv examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
  DLADIV performs complex division in  real arithmetic

                         a + i*b
             p + i*q = ---------
                         c + i*d

  The algorithm is due to Robert L. Smith and can be found
  in D. Knuth, The art of Computer Programming, Vol.2, p.195

 Arguments
 =========

  A       (input) DOUBLE PRECISION
  B       (input) DOUBLE PRECISION
  C       (input) DOUBLE PRECISION
```

```
 D       (input) DOUBLE PRECISION
         The scalars a, b, c, and d in the above expression.

 P       (output) DOUBLE PRECISION
 Q       (output) DOUBLE PRECISION
         The scalars p and q in the above expression.
```

———-

90.140.3 fortran code

— **dladiv.f** —

```
*  =====================================================================
      SUBROUTINE DLADIV( A, B, C, D, P, Q )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     January 2013
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   A, B, C, D, P, Q
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   BS
      PARAMETER          ( BS = 2.0D0 )
      DOUBLE PRECISION   HALF
      PARAMETER          ( HALF = 0.5D0 )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D0 )
*
*     .. Local Scalars ..
      DOUBLE PRECISION   AA, BB, CC, DD, AB, CD, S, OV, UN, BE, EPS
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLADIV1
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
```

```
*     ..
*     .. Executable Statements ..
*
      AA = A
      BB = B
      CC = C
      DD = D
      AB = MAX( ABS(A), ABS(B) )
      CD = MAX( ABS(C), ABS(D) )
      S = 1.0D0

      OV = DLAMCH( 'Overflow threshold' )
      UN = DLAMCH( 'Safe minimum' )
      EPS = DLAMCH( 'Epsilon' )
      BE = BS / (EPS*EPS)

      IF( AB >= HALF*OV ) THEN
         AA = HALF * AA
         BB = HALF * BB
         S  = TWO * S
      END IF
      IF( CD >= HALF*OV ) THEN
         CC = HALF * CC
         DD = HALF * DD
         S  = HALF * S
      END IF
      IF( AB <= UN*BS/EPS ) THEN
         AA = AA * BE
         BB = BB * BE
         S  = S / BE
      END IF
      IF( CD <= UN*BS/EPS ) THEN
         CC = CC * BE
         DD = DD * BE
         S  = S * BE
      END IF
      IF( ABS( D ).LE.ABS( C ) ) THEN
         CALL DLADIV1(AA, BB, CC, DD, P, Q)
      ELSE
         CALL DLADIV1(BB, AA, DD, CC, P, Q)
         Q = -Q
      END IF
      P = P * S
      Q = Q * S
*
      RETURN
*
*     End of DLADIV
*
      END
```

```
      SUBROUTINE DLADIV1( A, B, C, D, P, Q )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     January 2013
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   A, B, C, D, P, Q
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D0 )
*
*     .. Local Scalars ..
      DOUBLE PRECISION   R, T
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLADIV2
      EXTERNAL           DLADIV2
*     ..
*     .. Executable Statements ..
*
      R = D / C
      T = ONE / (C + D * R)
      P = DLADIV2(A, B, C, D, R, T)
      A = -A
      Q = DLADIV2(B, A, C, D, R, T)
*
      RETURN
*
*     End of DLADIV1
*
      END

      DOUBLE PRECISION FUNCTION DLADIV2( A, B, C, D, R, T )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     January 2013
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   A, B, C, D, R, T
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
*
*     .. Local Scalars ..
      DOUBLE PRECISION   BR
*     ..
*     .. Executable Statements ..
*
      IF( R.NE.ZERO ) THEN
         BR = B * R
         IF( BR.NE.ZERO ) THEN
            DLADIV2 = (A + BR) * T
         ELSE
            DLADIV2 = A * T + (B * T) * R
         END IF
      ELSE
         DLADIV2 = (A + D * (B / C)) * T
      END IF
*
      RETURN
*
*     End of DLADIV12
*
      END
      SUBROUTINE DLADIV( A, B, C, D, P, Q )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1992
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   A, B, C, D, P, Q
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      DOUBLE PRECISION   E, F
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS
*     ..
*     .. Executable Statements ..
*
```

```
      IF( ABS( D ).LT.ABS( C ) ) THEN
         E = D / C
         F = C + D*E
         P = ( A+B*E ) / F
         Q = ( B-A*E ) / F
      ELSE
         E = C / D
         F = D + C*E
         P = ( B+A*E ) / F
         Q = ( -A+B*E ) / F
      END IF
*
      RETURN
*
*     End of DLADIV
*
      END
```

———-

90.140.4 lisp code

— dladiv.lisp —

```
(let* ((bs 2.0d0) (half 0.5d0) (two 2.0d0))
  (declare (type (double-float 2.0d0 2.0d0) bs)
           (type (double-float 0.5d0 0.5d0) half)
           (type (double-float 2.0d0 2.0d0) two)
           (ignorable bs half two))
  (defun dladiv (a b c d p q)
    (declare (type (double-float) q p d c b a))
    (prog ((aa 0.0d0) (bb 0.0d0) (cc 0.0d0) (dd 0.0d0) (ab 0.0d0) (cd 0.0d0)
           (s 0.0d0) (ov 0.0d0) (un 0.0d0) (be 0.0d0) (eps 0.0d0))
      (declare (type (double-float) aa bb cc dd ab cd s ov un be eps))
      (setf aa a)
      (setf bb b)
      (setf cc c)
      (setf dd d)
      (setf ab (max (abs a) (abs b)))
      (setf cd (max (abs c) (abs d)))
      (setf s 1.0d0)
      (setf ov (dlamch "Overflow threshold"))
      (setf un (dlamch "Safe minimum"))
      (setf eps (dlamch "Epsilon"))
      (setf be (/ bs (* eps eps)))
      (cond
       ((>= ab (* half ov)) (setf aa (* half aa)) (setf bb (* half bb))
```

```
    (setf s (* two s))))
  (cond
   ((>= cd (* half ov)) (setf cc (* half cc)) (setf dd (* half dd))
    (setf s (* half s))))
  (cond
   ((<= ab (f2cl-lib:f2cl/ (* un bs) eps)) (setf aa (* aa be))
    (setf bb (* bb be)) (setf s (/ s be))))
  (cond
   ((<= cd (f2cl-lib:f2cl/ (* un bs) eps)) (setf cc (* cc be))
    (setf dd (* dd be)) (setf s (* s be))))
  (cond
   ((<= (abs d) (abs c))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (dladiv1 aa bb cc dd p q)
      (declare (ignore))
      (when var-0 (setf aa var-0))
      (when var-1 (setf bb var-1))
      (when var-2 (setf cc var-2))
      (when var-3 (setf dd var-3))
      (when var-4 (setf p var-4))
      (when var-5 (setf q var-5))))
   (t
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (dladiv1 bb aa dd cc p q)
      (declare (ignore))
      (when var-0 (setf bb var-0))
      (when var-1 (setf aa var-1))
      (when var-2 (setf dd var-2))
      (when var-3 (setf cc var-3))
      (when var-4 (setf p var-4))
      (when var-5 (setf q var-5)))
    (setf q (- q))))
  (setf p (* p s))
  (setf q (* q s))
  (go end_label)
 end_label
  (return (values nil nil nil nil p q)))))
```

———-

90.141 dlaed6

90.141.1 Axiom unit tests

— dlaed6.input —

```
)set break resume
)sys rm -f dlaed6.output
)spool dlaed6.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.141.2 Axiom help page

— **dlaed6.help** —

```
==================================================================
dlaed6 examples
==================================================================

==================================================================
Man Page Details
==================================================================

 Purpose
 =======

  DLAED6 computes the positive or negative root (closest to the origin)
  of
                   z(1)        z(2)        z(3)
  f(x) =   rho + --------- + ---------- + ---------
                  d(1)-x      d(2)-x      d(3)-x

  It is assumed that

        if ORGATI = .true. the root is between d(2) and d(3);
        otherwise it is between d(1) and d(2)

  This routine will be called by DLAED4 when necessary. In most cases,
  the root sought is the smallest in magnitude, though it might not be
  in some extremely rare situations.

  Arguments
  =========

  KNITER       (input) INTEGER
               Refer to DLAED4 for its significance.
```

```
 ORGATI       (input) LOGICAL
              If ORGATI is true, the needed root is between d(2) and
              d(3); otherwise it is between d(1) and d(2).  See
              DLAED4 for further details.

 RHO          (input) DOUBLE PRECISION
              Refer to the equation f(x) above.

 D            (input) DOUBLE PRECISION array, dimension (3)
              D satisfies d(1) < d(2) < d(3).

 Z            (input) DOUBLE PRECISION array, dimension (3)
              Each of the elements in z must be positive.

 FINIT        (input) DOUBLE PRECISION
              The value of f at 0. It is more accurate than the one
              evaluated inside this routine (if someone wants to do
              so).

 TAU          (output) DOUBLE PRECISION
              The root of the equation f(x).

 INFO         (output) INTEGER
              = 0: successful exit
              > 0: if INFO = 1, failure to converge

Further Details
===============

 Based on contributions by
    Ren-Cang Li, Computer Science Division, University of California
    at Berkeley, USA
```

———-

90.141.3 fortran code

— dlaed6.f —

```
*  =====================================================================
      SUBROUTINE DLAED6( KNITER, ORGATI, RHO, D, Z, FINIT, TAU, INFO )
*
*  -- LAPACK computational routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
```

```
*     .. Scalar Arguments ..
      LOGICAL            ORGATI
      INTEGER            INFO, KNITER
      DOUBLE PRECISION   FINIT, RHO, TAU
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( 3 ), Z( 3 )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            MAXIT
      PARAMETER          ( MAXIT = 40 )
      DOUBLE PRECISION   ZERO, ONE, TWO, THREE, FOUR, EIGHT
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0, TWO = 2.0D0,
     $                   THREE = 3.0D0, FOUR = 4.0D0, EIGHT = 8.0D0 )
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   DSCALE( 3 ), ZSCALE( 3 )
*     ..
*     .. Local Scalars ..
      LOGICAL            SCALE
      INTEGER            I, ITER, NITER
      DOUBLE PRECISION   A, B, BASE, C, DDF, DF, EPS, ERRETM, ETA, F,
     $                   FC, SCLFAC, SCLINV, SMALL1, SMALL2, SMINV1,
     $                   SMINV2, TEMP, TEMP1, TEMP2, TEMP3, TEMP4,
     $                   LBD, UBD
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, INT, LOG, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
      IF( ORGATI ) THEN
         LBD = D(2)
         UBD = D(3)
      ELSE
         LBD = D(1)
         UBD = D(2)
      END IF
      IF( FINIT .LT. ZERO )THEN
         LBD = ZERO
      ELSE
```

```
         UBD = ZERO
      END IF
*
      NITER = 1
      TAU = ZERO
      IF( KNITER.EQ.2 ) THEN
         IF( ORGATI ) THEN
            TEMP = ( D( 3 )-D( 2 ) ) / TWO
            C = RHO + Z( 1 ) / ( ( D( 1 )-D( 2 ) )-TEMP )
            A = C*( D( 2 )+D( 3 ) ) + Z( 2 ) + Z( 3 )
            B = C*D( 2 )*D( 3 ) + Z( 2 )*D( 3 ) + Z( 3 )*D( 2 )
         ELSE
            TEMP = ( D( 1 )-D( 2 ) ) / TWO
            C = RHO + Z( 3 ) / ( ( D( 3 )-D( 2 ) )-TEMP )
            A = C*( D( 1 )+D( 2 ) ) + Z( 1 ) + Z( 2 )
            B = C*D( 1 )*D( 2 ) + Z( 1 )*D( 2 ) + Z( 2 )*D( 1 )
         END IF
         TEMP = MAX( ABS( A ), ABS( B ), ABS( C ) )
         A = A / TEMP
         B = B / TEMP
         C = C / TEMP
         IF( C.EQ.ZERO ) THEN
            TAU = B / A
         ELSE IF( A.LE.ZERO ) THEN
            TAU = ( A-SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
         ELSE
            TAU = TWO*B / ( A+SQRT( ABS( A*A-FOUR*B*C ) ) )
         END IF
         IF( TAU .LT. LBD .OR. TAU .GT. UBD )
     $      TAU = ( LBD+UBD )/TWO
         IF( D(1).EQ.TAU .OR. D(2).EQ.TAU .OR. D(3).EQ.TAU ) THEN
            TAU = ZERO
         ELSE
            TEMP = FINIT + TAU*Z(1)/( D(1)*( D( 1 )-TAU ) ) +
     $                     TAU*Z(2)/( D(2)*( D( 2 )-TAU ) ) +
     $                     TAU*Z(3)/( D(3)*( D( 3 )-TAU ) )
            IF( TEMP .LE. ZERO )THEN
               LBD = TAU
            ELSE
               UBD = TAU
            END IF
            IF( ABS( FINIT ).LE.ABS( TEMP ) )
     $         TAU = ZERO
         END IF
      END IF
*
*     get machine parameters for possible scaling to avoid overflow
*
*     modified by Sven: parameters SMALL1, SMINV1, SMALL2,
*     SMINV2, EPS are not SAVEd anymore between one call to the
```

```
*     others but recomputed at each call
*
      EPS = DLAMCH( 'Epsilon' )
      BASE = DLAMCH( 'Base' )
      SMALL1 = BASE**( INT( LOG( DLAMCH( 'SafMin' ) ) / LOG( BASE ) /
     $         THREE ) )
      SMINV1 = ONE / SMALL1
      SMALL2 = SMALL1*SMALL1
      SMINV2 = SMINV1*SMINV1
*
*     Determine if scaling of inputs necessary to avoid overflow
*     when computing 1/TEMP**3
*
      IF( ORGATI ) THEN
         TEMP = MIN( ABS( D( 2 )-TAU ), ABS( D( 3 )-TAU ) )
      ELSE
         TEMP = MIN( ABS( D( 1 )-TAU ), ABS( D( 2 )-TAU ) )
      END IF
      SCALE = .FALSE.
      IF( TEMP.LE.SMALL1 ) THEN
         SCALE = .TRUE.
         IF( TEMP.LE.SMALL2 ) THEN
*
*        Scale up by power of radix nearest 1/SAFMIN**(2/3)
*
            SCLFAC = SMINV2
            SCLINV = SMALL2
         ELSE
*
*        Scale up by power of radix nearest 1/SAFMIN**(1/3)
*
            SCLFAC = SMINV1
            SCLINV = SMALL1
         END IF
*
*        Scaling up safe because D, Z, TAU scaled elsewhere to be O(1)
*
         DO 10 I = 1, 3
            DSCALE( I ) = D( I )*SCLFAC
            ZSCALE( I ) = Z( I )*SCLFAC
   10    CONTINUE
         TAU = TAU*SCLFAC
         LBD = LBD*SCLFAC
         UBD = UBD*SCLFAC
      ELSE
*
*        Copy D and Z to DSCALE and ZSCALE
*
         DO 20 I = 1, 3
            DSCALE( I ) = D( I )
```

```
            ZSCALE( I ) = Z( I )
   20    CONTINUE
      END IF
*
      FC = ZERO
      DF = ZERO
      DDF = ZERO
      DO 30 I = 1, 3
         TEMP = ONE / ( DSCALE( I )-TAU )
         TEMP1 = ZSCALE( I )*TEMP
         TEMP2 = TEMP1*TEMP
         TEMP3 = TEMP2*TEMP
         FC = FC + TEMP1 / DSCALE( I )
         DF = DF + TEMP2
         DDF = DDF + TEMP3
   30 CONTINUE
      F = FINIT + TAU*FC
*
      IF( ABS( F ).LE.ZERO )
     $   GO TO 60
      IF( F .LE. ZERO )THEN
         LBD = TAU
      ELSE
         UBD = TAU
      END IF
*
*        Iteration begins -- Use Gragg-Thornton-Warner cubic convergent
*                            scheme
*
*     It is not hard to see that
*
*           1) Iterations will go up monotonically
*              if FINIT < 0;
*
*           2) Iterations will go down monotonically
*              if FINIT > 0.
*
      ITER = NITER + 1
*
      DO 50 NITER = ITER, MAXIT
*
         IF( ORGATI ) THEN
            TEMP1 = DSCALE( 2 ) - TAU
            TEMP2 = DSCALE( 3 ) - TAU
         ELSE
            TEMP1 = DSCALE( 1 ) - TAU
            TEMP2 = DSCALE( 2 ) - TAU
         END IF
         A = ( TEMP1+TEMP2 )*F - TEMP1*TEMP2*DF
         B = TEMP1*TEMP2*F
```

```
         C = F - ( TEMP1+TEMP2 )*DF + TEMP1*TEMP2*DDF
         TEMP = MAX( ABS( A ), ABS( B ), ABS( C ) )
         A = A / TEMP
         B = B / TEMP
         C = C / TEMP
         IF( C.EQ.ZERO ) THEN
            ETA = B / A
         ELSE IF( A.LE.ZERO ) THEN
            ETA = ( A-SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
         ELSE
            ETA = TWO*B / ( A+SQRT( ABS( A*A-FOUR*B*C ) ) )
         END IF
         IF( F*ETA.GE.ZERO ) THEN
            ETA = -F / DF
         END IF
*
         TAU = TAU + ETA
         IF( TAU .LT. LBD .OR. TAU .GT. UBD )
     $      TAU = ( LBD + UBD )/TWO
*
         FC = ZERO
         ERRETM = ZERO
         DF = ZERO
         DDF = ZERO
         DO 40 I = 1, 3
            IF ( ( DSCALE( I )-TAU ).NE.ZERO ) THEN
               TEMP = ONE / ( DSCALE( I )-TAU )
               TEMP1 = ZSCALE( I )*TEMP
               TEMP2 = TEMP1*TEMP
               TEMP3 = TEMP2*TEMP
               TEMP4 = TEMP1 / DSCALE( I )
               FC = FC + TEMP4
               ERRETM = ERRETM + ABS( TEMP4 )
               DF = DF + TEMP2
               DDF = DDF + TEMP3
            ELSE
               GO TO 60
            END IF
   40    CONTINUE
         F = FINIT + TAU*FC
         ERRETM = EIGHT*( ABS( FINIT )+ABS( TAU )*ERRETM ) +
     $            ABS( TAU )*DF
         IF( ( ABS( F ).LE.FOUR*EPS*ERRETM ) .OR.
     $      ( (UBD-LBD).LE.FOUR*EPS*ABS(TAU) )  )
     $      GO TO 60
         IF( F .LE. ZERO )THEN
            LBD = TAU
         ELSE
            UBD = TAU
         END IF
```

```
   50 CONTINUE
      INFO = 1
   60 CONTINUE
*
*     Undo scaling
*
      IF( SCALE )
     $   TAU = TAU*SCLINV
      RETURN
*
*     End of DLAED6
*
      END
      SUBROUTINE DLAED6( KNITER, ORGATI, RHO, D, Z, FINIT, TAU, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Oak Ridge National Lab, Argonne National Lab,
*     Courant Institute, NAG Ltd., and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      LOGICAL            ORGATI
      INTEGER            INFO, KNITER
      DOUBLE PRECISION   FINIT, RHO, TAU
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( 3 ), Z( 3 )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            MAXIT
      PARAMETER          ( MAXIT = 20 )
      DOUBLE PRECISION   ZERO, ONE, TWO, THREE, FOUR, EIGHT
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0, TWO = 2.0D0,
     $                   THREE = 3.0D0, FOUR = 4.0D0, EIGHT = 8.0D0 )
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   DSCALE( 3 ), ZSCALE( 3 )
*     ..
*     .. Local Scalars ..
      LOGICAL            FIRST, SCALE
      INTEGER            I, ITER, NITER
      DOUBLE PRECISION   A, B, BASE, C, DDF, DF, EPS, ERRETM, ETA, F,
     $                   FC, SCLFAC, SCLINV, SMALL1, SMALL2, SMINV1,
```

```
     $                   SMINV2, TEMP, TEMP1, TEMP2, TEMP3, TEMP4
*     ..
*     .. Save statement ..
      SAVE               FIRST, SMALL1, SMINV1, SMALL2, SMINV2, EPS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, INT, LOG, MAX, MIN, SQRT
*     ..
*     .. Data statements ..
      DATA               FIRST / .TRUE. /
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
      NITER = 1
      TAU = ZERO
      IF( KNITER.EQ.2 ) THEN
         IF( ORGATI ) THEN
            TEMP = ( D( 3 )-D( 2 ) ) / TWO
            C = RHO + Z( 1 ) / ( ( D( 1 )-D( 2 ) )-TEMP )
            A = C*( D( 2 )+D( 3 ) ) + Z( 2 ) + Z( 3 )
            B = C*D( 2 )*D( 3 ) + Z( 2 )*D( 3 ) + Z( 3 )*D( 2 )
         ELSE
            TEMP = ( D( 1 )-D( 2 ) ) / TWO
            C = RHO + Z( 3 ) / ( ( D( 3 )-D( 2 ) )-TEMP )
            A = C*( D( 1 )+D( 2 ) ) + Z( 1 ) + Z( 2 )
            B = C*D( 1 )*D( 2 ) + Z( 1 )*D( 2 ) + Z( 2 )*D( 1 )
         END IF
         TEMP = MAX( ABS( A ), ABS( B ), ABS( C ) )
         A = A / TEMP
         B = B / TEMP
         C = C / TEMP
         IF( C.EQ.ZERO ) THEN
            TAU = B / A
         ELSE IF( A.LE.ZERO ) THEN
            TAU = ( A-SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
         ELSE
            TAU = TWO*B / ( A+SQRT( ABS( A*A-FOUR*B*C ) ) )
         END IF
         TEMP = RHO + Z( 1 ) / ( D( 1 )-TAU ) +
     $          Z( 2 ) / ( D( 2 )-TAU ) + Z( 3 ) / ( D( 3 )-TAU )
         IF( ABS( FINIT ).LE.ABS( TEMP ) )
     $      TAU = ZERO
      END IF
*
*     On first call to routine, get machine parameters for
*     possible scaling to avoid overflow
*
      IF( FIRST ) THEN
```

```
         EPS = DLAMCH( 'Epsilon' )
         BASE = DLAMCH( 'Base' )
         SMALL1 = BASE**( INT( LOG( DLAMCH( 'SafMin' ) ) / LOG( BASE ) /
     $           THREE ) )
         SMINV1 = ONE / SMALL1
         SMALL2 = SMALL1*SMALL1
         SMINV2 = SMINV1*SMINV1
         FIRST = .FALSE.
      END IF
*
*     Determine if scaling of inputs necessary to avoid overflow
*     when computing 1/TEMP**3
*
      IF( ORGATI ) THEN
         TEMP = MIN( ABS( D( 2 )-TAU ), ABS( D( 3 )-TAU ) )
      ELSE
         TEMP = MIN( ABS( D( 1 )-TAU ), ABS( D( 2 )-TAU ) )
      END IF
      SCALE = .FALSE.
      IF( TEMP.LE.SMALL1 ) THEN
         SCALE = .TRUE.
         IF( TEMP.LE.SMALL2 ) THEN
*
*        Scale up by power of radix nearest 1/SAFMIN**(2/3)
*
            SCLFAC = SMINV2
            SCLINV = SMALL2
         ELSE
*
*        Scale up by power of radix nearest 1/SAFMIN**(1/3)
*
            SCLFAC = SMINV1
            SCLINV = SMALL1
         END IF
*
*        Scaling up safe because D, Z, TAU scaled elsewhere to be O(1)
*
         DO 10 I = 1, 3
            DSCALE( I ) = D( I )*SCLFAC
            ZSCALE( I ) = Z( I )*SCLFAC
   10    CONTINUE
         TAU = TAU*SCLFAC
      ELSE
*
*        Copy D and Z to DSCALE and ZSCALE
*
         DO 20 I = 1, 3
            DSCALE( I ) = D( I )
            ZSCALE( I ) = Z( I )
   20    CONTINUE
```

```
      END IF
*
      FC = ZERO
      DF = ZERO
      DDF = ZERO
      DO 30 I = 1, 3
         TEMP = ONE / ( DSCALE( I )-TAU )
         TEMP1 = ZSCALE( I )*TEMP
         TEMP2 = TEMP1*TEMP
         TEMP3 = TEMP2*TEMP
         FC = FC + TEMP1 / DSCALE( I )
         DF = DF + TEMP2
         DDF = DDF + TEMP3
   30 CONTINUE
      F = FINIT + TAU*FC
*
      IF( ABS( F ).LE.ZERO )
     $   GO TO 60
*
*        Iteration begins
*
*     It is not hard to see that
*
*           1) Iterations will go up monotonically
*              if FINIT < 0;
*
*           2) Iterations will go down monotonically
*              if FINIT > 0.
*
      ITER = NITER + 1
*
      DO 50 NITER = ITER, MAXIT
*
         IF( ORGATI ) THEN
            TEMP1 = DSCALE( 2 ) - TAU
            TEMP2 = DSCALE( 3 ) - TAU
         ELSE
            TEMP1 = DSCALE( 1 ) - TAU
            TEMP2 = DSCALE( 2 ) - TAU
         END IF
         A = ( TEMP1+TEMP2 )*F - TEMP1*TEMP2*DF
         B = TEMP1*TEMP2*F
         C = F - ( TEMP1+TEMP2 )*DF + TEMP1*TEMP2*DDF
         TEMP = MAX( ABS( A ), ABS( B ), ABS( C ) )
         A = A / TEMP
         B = B / TEMP
         C = C / TEMP
         IF( C.EQ.ZERO ) THEN
            ETA = B / A
         ELSE IF( A.LE.ZERO ) THEN
```

```
         ETA = ( A-SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
      ELSE
         ETA = TWO*B / ( A+SQRT( ABS( A*A-FOUR*B*C ) ) )
      END IF
      IF( F*ETA.GE.ZERO ) THEN
         ETA = -F / DF
      END IF
*
      TEMP = ETA + TAU
      IF( ORGATI ) THEN
         IF( ETA.GT.ZERO .AND. TEMP.GE.DSCALE( 3 ) )
     $      ETA = ( DSCALE( 3 )-TAU ) / TWO
         IF( ETA.LT.ZERO .AND. TEMP.LE.DSCALE( 2 ) )
     $      ETA = ( DSCALE( 2 )-TAU ) / TWO
      ELSE
         IF( ETA.GT.ZERO .AND. TEMP.GE.DSCALE( 2 ) )
     $      ETA = ( DSCALE( 2 )-TAU ) / TWO
         IF( ETA.LT.ZERO .AND. TEMP.LE.DSCALE( 1 ) )
     $      ETA = ( DSCALE( 1 )-TAU ) / TWO
      END IF
      TAU = TAU + ETA
*
      FC = ZERO
      ERRETM = ZERO
      DF = ZERO
      DDF = ZERO
      DO 40 I = 1, 3
         TEMP = ONE / ( DSCALE( I )-TAU )
         TEMP1 = ZSCALE( I )*TEMP
         TEMP2 = TEMP1*TEMP
         TEMP3 = TEMP2*TEMP
         TEMP4 = TEMP1 / DSCALE( I )
         FC = FC + TEMP4
         ERRETM = ERRETM + ABS( TEMP4 )
         DF = DF + TEMP2
         DDF = DDF + TEMP3
   40 CONTINUE
      F = FINIT + TAU*FC
      ERRETM = EIGHT*( ABS( FINIT )+ABS( TAU )*ERRETM ) +
     $         ABS( TAU )*DF
      IF( ABS( F ).LE.EPS*ERRETM )
     $   GO TO 60
   50 CONTINUE
      INFO = 1
   60 CONTINUE
*
*     Undo scaling
*
      IF( SCALE )
     $   TAU = TAU*SCLINV
```

```
      RETURN
*
*     End of DLAED6
*
      END
```

———-

90.141.4 lisp code

— dlaed6.lisp —

```
(let* ((maxit 40)
       (zero 0.0d0)
       (one 1.0d0)
       (two 2.0d0)
       (three 3.0d0)
       (four 4.0d0)
       (eight 8.0d0))
  (declare (type (f2cl-lib:integer4 40 40) maxit)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 3.0d0 3.0d0) three)
           (type (double-float 4.0d0 4.0d0) four)
           (type (double-float 8.0d0 8.0d0) eight)
           (ignorable maxit zero one two three four eight))
  (defun dlaed6 (kniter orgati rho d z finit tau info)
    (declare (type (array double-float (*)) z d)
             (type (double-float) tau finit rho)
             (type f2cl-lib:logical orgati)
             (type (f2cl-lib:integer4) info kniter))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (z double-float z-%data% z-%offset%))
      (prog ((a 0.0d0) (b 0.0d0) (base 0.0d0) (c 0.0d0) (ddf 0.0d0) (df 0.0d0)
             (eps 0.0d0) (erretm 0.0d0) (eta 0.0d0) (f 0.0d0) (fc 0.0d0)
             (sclfac 0.0d0) (sclinv 0.0d0) (small1 0.0d0) (small2 0.0d0)
             (sminv1 0.0d0) (sminv2 0.0d0) (temp 0.0d0) (temp1 0.0d0)
             (temp2 0.0d0) (temp3 0.0d0) (temp4 0.0d0) (lbd 0.0d0) (ubd 0.0d0)
             (i 0) (iter 0) (niter 0) (scale nil)
             (dscale (make-array 3 :element-type 'double-float))
             (zscale (make-array 3 :element-type 'double-float)))
        (declare
         (type (double-float) a b base c ddf df eps erretm eta f fc sclfac
          sclinv small1 small2 sminv1 sminv2 temp temp1 temp2 temp3 temp4 lbd
          ubd)
```

```
 (type (f2cl-lib:integer4) i iter niter)
 (type f2cl-lib:logical scale)
 (type (array double-float (3)) dscale zscale))
(setf info 0)
(cond
 (orgati (setf lbd (f2cl-lib:fref d-%data% (2) ((1 3)) d-%offset%))
  (setf ubd (f2cl-lib:fref d-%data% (3) ((1 3)) d-%offset%)))
 (t (setf lbd (f2cl-lib:fref d-%data% (1) ((1 3)) d-%offset%))
  (setf ubd (f2cl-lib:fref d-%data% (2) ((1 3)) d-%offset%))))
(cond ((< finit zero) (setf lbd zero)) (t (setf ubd zero)))
(setf niter 1)
(setf tau zero)
(cond
 ((= kniter 2)
  (cond
   (orgati
    (setf temp
            (/
             (- (f2cl-lib:fref d-%data% (3) ((1 3)) d-%offset%)
                (f2cl-lib:fref d-%data% (2) ((1 3)) d-%offset%))
             two))
    (setf c
            (+ rho
               (/ (f2cl-lib:fref z-%data% (1) ((1 3)) z-%offset%)
                  (- (f2cl-lib:fref d-%data% (1) ((1 3)) d-%offset%)
                     (f2cl-lib:fref d-%data% (2) ((1 3)) d-%offset%)
                     temp))))
    (setf a
            (+
             (* c
                (+ (f2cl-lib:fref d-%data% (2) ((1 3)) d-%offset%)
                   (f2cl-lib:fref d-%data% (3) ((1 3)) d-%offset%)))
             (f2cl-lib:fref z-%data% (2) ((1 3)) z-%offset%)
             (f2cl-lib:fref z-%data% (3) ((1 3)) z-%offset%)))
    (setf b
            (+
             (* c (f2cl-lib:fref d-%data% (2) ((1 3)) d-%offset%)
                (f2cl-lib:fref d-%data% (3) ((1 3)) d-%offset%))
             (* (f2cl-lib:fref z-%data% (2) ((1 3)) z-%offset%)
                (f2cl-lib:fref d-%data% (3) ((1 3)) d-%offset%))
             (* (f2cl-lib:fref z-%data% (3) ((1 3)) z-%offset%)
                (f2cl-lib:fref d-%data% (2) ((1 3)) d-%offset%)))))
   (t
    (setf temp
            (/
             (- (f2cl-lib:fref d-%data% (1) ((1 3)) d-%offset%)
                (f2cl-lib:fref d-%data% (2) ((1 3)) d-%offset%))
             two))
    (setf c
            (+ rho
```

```
                (/ (f2cl-lib:fref z-%data% (3) ((1 3)) z-%offset%)
                   (- (f2cl-lib:fref d-%data% (3) ((1 3)) d-%offset%)
                      (f2cl-lib:fref d-%data% (2) ((1 3)) d-%offset%)
                      temp))))
  (setf a
          (+
           (* c
              (+ (f2cl-lib:fref d-%data% (1) ((1 3)) d-%offset%)
                 (f2cl-lib:fref d-%data% (2) ((1 3)) d-%offset%)))
           (f2cl-lib:fref z-%data% (1) ((1 3)) z-%offset%)
           (f2cl-lib:fref z-%data% (2) ((1 3)) z-%offset%)))
  (setf b
          (+
           (* c (f2cl-lib:fref d-%data% (1) ((1 3)) d-%offset%)
              (f2cl-lib:fref d-%data% (2) ((1 3)) d-%offset%))
           (* (f2cl-lib:fref z-%data% (1) ((1 3)) z-%offset%)
              (f2cl-lib:fref d-%data% (2) ((1 3)) d-%offset%))
           (* (f2cl-lib:fref z-%data% (2) ((1 3)) z-%offset%)
              (f2cl-lib:fref d-%data% (1) ((1 3)) d-%offset%))))))
(setf temp (max (abs a) (abs b) (abs c))) (setf a (/ a temp))
(setf b (/ b temp)) (setf c (/ c temp))
(cond ((= c zero) (setf tau (/ b a)))
      ((<= a zero)
       (setf tau
               (/
                (- a
                   (f2cl-lib:fsqrt
                    (abs (+ (* a a) (* (- four) b c)))))
                (* two c))))
      (t
       (setf tau
               (/ (* two b)
                  (+ a
                     (f2cl-lib:fsqrt
                      (abs (+ (* a a) (* (- four) b c)))))))))
(if (or (< tau lbd) (> tau ubd))
    (setf tau (/ (+ lbd ubd) two)))
(cond
 ((or (= (f2cl-lib:fref d (1) ((1 3))) tau)
      (= (f2cl-lib:fref d (2) ((1 3))) tau)
      (= (f2cl-lib:fref d (3) ((1 3))) tau))
  (setf tau zero))
 (t
  (setf temp
          (+ finit
             (/
              (* tau (f2cl-lib:fref z-%data% (1) ((1 3)) z-%offset%))
              (* (f2cl-lib:fref d-%data% (1) ((1 3)) d-%offset%)
                 (- (f2cl-lib:fref d-%data% (1) ((1 3)) d-%offset%)
                    tau)))
```

```
                    (/
                     (* tau (f2cl-lib:fref z-%data% (2) ((1 3)) z-%offset%))
                     (* (f2cl-lib:fref d-%data% (2) ((1 3)) d-%offset%)
                        (- (f2cl-lib:fref d-%data% (2) ((1 3)) d-%offset%)
                           tau)))
                    (/
                     (* tau (f2cl-lib:fref z-%data% (3) ((1 3)) z-%offset%))
                     (* (f2cl-lib:fref d-%data% (3) ((1 3)) d-%offset%)
                        (- (f2cl-lib:fref d-%data% (3) ((1 3)) d-%offset%)
                           tau)))))
    (cond ((<= temp zero) (setf lbd tau)) (t (setf ubd tau)))
    (if (<= (abs finit) (abs temp))
        (setf tau zero))))))
(setf eps (dlamch "Epsilon"))
(setf base (dlamch "Base"))
(setf small1
        (expt base
              (f2cl-lib:int
               (/
                (/ (f2cl-lib:flog (dlamch "SafMin"))
                   (f2cl-lib:flog base))
                three))))
(setf sminv1 (/ one small1))
(setf small2 (* small1 small1))
(setf sminv2 (* sminv1 sminv1))
(cond
 (orgati
  (setf temp
          (min
           (abs
            (- (f2cl-lib:fref d-%data% (2) ((1 3)) d-%offset%) tau))
           (abs
            (- (f2cl-lib:fref d-%data% (3) ((1 3)) d-%offset%) tau)))))
 (t
  (setf temp
          (min
           (abs
            (- (f2cl-lib:fref d-%data% (1) ((1 3)) d-%offset%) tau))
           (abs
            (- (f2cl-lib:fref d-%data% (2) ((1 3)) d-%offset%)
               tau))))))
(setf scale f2cl-lib:%false%)
(cond
 ((<= temp small1) (setf scale f2cl-lib:%true%)
  (cond ((<= temp small2) (setf sclfac sminv2) (setf sclinv small2))
        (t (setf sclfac sminv1) (setf sclinv small1)))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i 3) nil)
    (tagbody
      (setf (f2cl-lib:fref dscale (i) ((1 3)))
```

```
                (* (f2cl-lib:fref d-%data% (i) ((1 3)) d-%offset%)
                   sclfac))
      (setf (f2cl-lib:fref zscale (i) ((1 3)))
                (* (f2cl-lib:fref z-%data% (i) ((1 3)) z-%offset%)
                   sclfac))
     label10))
  (setf tau (* tau sclfac)) (setf lbd (* lbd sclfac))
  (setf ubd (* ubd sclfac)))
 (t
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i 3) nil)
    (tagbody
      (setf (f2cl-lib:fref dscale (i) ((1 3)))
                (f2cl-lib:fref d-%data% (i) ((1 3)) d-%offset%))
      (setf (f2cl-lib:fref zscale (i) ((1 3)))
                (f2cl-lib:fref z-%data% (i) ((1 3)) z-%offset%))
     label20))))
(setf fc zero)
(setf df zero)
(setf ddf zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i 3) nil)
  (tagbody
    (setf temp (/ one (- (f2cl-lib:fref dscale (i) ((1 3))) tau)))
    (setf temp1 (* (f2cl-lib:fref zscale (i) ((1 3))) temp))
    (setf temp2 (* temp1 temp))
    (setf temp3 (* temp2 temp))
    (setf fc (+ fc (/ temp1 (f2cl-lib:fref dscale (i) ((1 3))))))
    (setf df (+ df temp2))
    (setf ddf (+ ddf temp3))
   label30))
(setf f (+ finit (* tau fc)))
(if (<= (abs f) zero)
    (go label60))
(cond ((<= f zero) (setf lbd tau)) (t (setf ubd tau)))
(setf iter (f2cl-lib:int-add niter 1))
(f2cl-lib:fdo (niter iter (f2cl-lib:int-add niter 1))
              ((> niter maxit) nil)
  (tagbody
    (cond
     (orgati (setf temp1 (- (f2cl-lib:fref dscale (2) ((1 3))) tau))
      (setf temp2 (- (f2cl-lib:fref dscale (3) ((1 3))) tau)))
     (t (setf temp1 (- (f2cl-lib:fref dscale (1) ((1 3))) tau))
      (setf temp2 (- (f2cl-lib:fref dscale (2) ((1 3))) tau))))
    (setf a (+ (* (+ temp1 temp2) f) (* (- temp1) temp2 df)))
    (setf b (* temp1 temp2 f))
    (setf c (+ (- f (* (+ temp1 temp2) df)) (* temp1 temp2 ddf)))
    (setf temp (max (abs a) (abs b) (abs c)))
    (setf a (/ a temp))
    (setf b (/ b temp))
```

```
      (setf c (/ c temp))
      (cond ((= c zero) (setf eta (/ b a)))
            ((<= a zero)
             (setf eta
                     (/
                      (- a
                         (f2cl-lib:fsqrt
                          (abs (+ (* a a) (* (- four) b c)))))
                      (* two c))))
            (t
             (setf eta
                     (/ (* two b)
                        (+ a
                           (f2cl-lib:fsqrt
                            (abs (+ (* a a) (* (- four) b c)))))))))
      (cond ((>= (* f eta) zero) (setf eta (/ (- f) df))))
      (setf tau (+ tau eta))
      (if (or (< tau lbd) (> tau ubd))
          (setf tau (/ (+ lbd ubd) two)))
      (setf fc zero)
      (setf erretm zero)
      (setf df zero)
      (setf ddf zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i 3) nil)
        (tagbody
          (cond
           ((/= (+ (f2cl-lib:fref dscale (i) ((1 3))) (- tau)) zero)
            (setf temp
                    (/ one (- (f2cl-lib:fref dscale (i) ((1 3))) tau)))
            (setf temp1 (* (f2cl-lib:fref zscale (i) ((1 3))) temp))
            (setf temp2 (* temp1 temp)) (setf temp3 (* temp2 temp))
            (setf temp4 (/ temp1 (f2cl-lib:fref dscale (i) ((1 3)))))
            (setf fc (+ fc temp4)) (setf erretm (+ erretm (abs temp4)))
            (setf df (+ df temp2)) (setf ddf (+ ddf temp3)))
           (t (go label60)))
         label40))
      (setf f (+ finit (* tau fc)))
      (setf erretm
              (+ (* eight (+ (abs finit) (* (abs tau) erretm)))
                 (* (abs tau) df)))
      (if (or (<= (abs f) (* four eps erretm))
              (<= (- ubd lbd) (* four eps (abs tau))))
          (go label60))
      (cond ((<= f zero) (setf lbd tau)) (t (setf ubd tau)))
     label50))
  (setf info 1)
 label60
  (if scale
      (setf tau (* tau sclinv)))
```

```
        (go end_label)
       end_label
        (return (values nil nil nil nil nil nil tau info))))))
```

———-

90.142 dlaexc

90.142.1 Axiom unit tests

— **dlaexc.input** —

```
)set break resume
)sys rm -f dlaexc.output
)spool dlaexc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.142.2 Axiom help page

— **dlaexc.help** —

```
======================================================================
dlaexc examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DLAEXC  -  adjacent  diagonal  blocks T11 and T22 of order 1 or 2 in an
       upper quasi-triangular matrix T by an orthogonal similarity transforma-
       tion

SYNOPSIS
       SUBROUTINE DLAEXC( WANTQ, N, T, LDT, Q, LDQ, J1, N1, N2, WORK, INFO )
```

```
      LOGICAL        WANTQ

      INTEGER        INFO, J1, LDQ, LDT, N, N1, N2

      DOUBLE         PRECISION Q( LDQ, * ), T( LDT, * ), WORK( * )
```

PURPOSE

DLAEXC swaps adjacent diagonal blocks T11 and T22 of order 1 or 2 in an upper quasi-triangular matrix T by an orthogonal similarity transformation.

T must be in Schur canonical form, that is, block upper triangular with 1-by-1 and 2-by-2 diagonal blocks; each 2-by-2 diagonal block has its diagonal elemnts equal and its off-diagonal elements of opposite sign.

ARGUMENTS

WANTQ (input) LOGICAL
= .TRUE. : accumulate the transformation in the matrix Q;
= .FALSE.: do not accumulate the transformation.

N (input) INTEGER
The order of the matrix T. N >= 0.

T (input/output) DOUBLE PRECISION array, dimension (LDT,N)
On entry, the upper quasi-triangular matrix T, in Schur canonical form. On exit, the updated matrix T, again in Schur canonical form.

LDT (input) INTEGER
The leading dimension of the array T. LDT >= max(1,N).

Q (input/output) DOUBLE PRECISION array, dimension (LDQ,N)
On entry, if WANTQ is .TRUE., the orthogonal matrix Q. On exit, if WANTQ is .TRUE., the updated matrix Q. If WANTQ is .FALSE., Q is not referenced.

LDQ (input) INTEGER
The leading dimension of the array Q. LDQ >= 1; and if WANTQ is .TRUE., LDQ >= N.

J1 (input) INTEGER
The index of the first row of the first block T11.

N1 (input) INTEGER
The order of the first block T11. N1 = 0, 1 or 2.

N2 (input) INTEGER
The order of the second block T22. N2 = 0, 1 or 2.

```
WORK    (workspace) DOUBLE PRECISION array, dimension (N)

INFO    (output) INTEGER
        = 0: successful exit
        = 1: the transformed matrix T would be too far from Schur form;
        the blocks are not swapped and T and Q are unchanged.
```

———-

90.142.3 fortran code

— **dlaexc.f** —

```
*  =====================================================================
      SUBROUTINE DLAEXC( WANTQ, N, T, LDT, Q, LDQ, J1, N1, N2, WORK,
     $                   INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.2) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     September 2012
*
*     .. Scalar Arguments ..
      LOGICAL            WANTQ
      INTEGER            INFO, J1, LDQ, LDT, N, N1, N2
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   Q( LDQ, * ), T( LDT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   TEN
      PARAMETER          ( TEN = 1.0D+1 )
      INTEGER            LDD, LDX
      PARAMETER          ( LDD = 4, LDX = 2 )
*     ..
*     .. Local Scalars ..
      INTEGER            IERR, J2, J3, J4, K, ND
      DOUBLE PRECISION   CS, DNORM, EPS, SCALE, SMLNUM, SN, T11, T22,
     $                   T33, TAU, TAU1, TAU2, TEMP, THRESH, WI1, WI2,
     $                   WR1, WR2, XNORM
*     ..
```

```
*     .. Local Arrays ..
      DOUBLE PRECISION   D( LDD, 4 ), U( 3 ), U1( 3 ), U2( 3 ),
     $                   X( LDX, 2 )
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           DLAMCH, DLANGE
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLACPY, DLANV2, DLARFG, DLARFX, DLARTG, DLASY2,
     $                   DROT
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. N1.EQ.0 .OR. N2.EQ.0 )
     $   RETURN
      IF( J1+N1.GT.N )
     $   RETURN
*
      J2 = J1 + 1
      J3 = J1 + 2
      J4 = J1 + 3
*
      IF( N1.EQ.1 .AND. N2.EQ.1 ) THEN
*
*        Swap two 1-by-1 blocks.
*
         T11 = T( J1, J1 )
         T22 = T( J2, J2 )
*
*        Determine the transformation to perform the interchange.
*
         CALL DLARTG( T( J1, J2 ), T22-T11, CS, SN, TEMP )
*
*        Apply transformation to the matrix T.
*
         IF( J3.LE.N )
     $      CALL DROT( N-J1-1, T( J1, J3 ), LDT, T( J2, J3 ), LDT, CS,
     $                 SN )
         CALL DROT( J1-1, T( 1, J1 ), 1, T( 1, J2 ), 1, CS, SN )
*
         T( J1, J1 ) = T22
         T( J2, J2 ) = T11
```

```
*
         IF( WANTQ ) THEN
*
*           Accumulate transformation in the matrix Q.
*
            CALL DROT( N, Q( 1, J1 ), 1, Q( 1, J2 ), 1, CS, SN )
         END IF
*
      ELSE
*
*        Swapping involves at least one 2-by-2 block.
*
*        Copy the diagonal block of order N1+N2 to the local array D
*        and compute its norm.
*
         ND = N1 + N2
         CALL DLACPY( 'Full', ND, ND, T( J1, J1 ), LDT, D, LDD )
         DNORM = DLANGE( 'Max', ND, ND, D, LDD, WORK )
*
*        Compute machine-dependent threshold for test for accepting
*        swap.
*
         EPS = DLAMCH( 'P' )
         SMLNUM = DLAMCH( 'S' ) / EPS
         THRESH = MAX( TEN*EPS*DNORM, SMLNUM )
*
*        Solve T11*X - X*T22 = scale*T12 for X.
*
         CALL DLASY2( .FALSE., .FALSE., -1, N1, N2, D, LDD,
     $                D( N1+1, N1+1 ), LDD, D( 1, N1+1 ), LDD, SCALE, X,
     $                LDX, XNORM, IERR )
*
*        Swap the adjacent diagonal blocks.
*
         K = N1 + N1 + N2 - 3
         GO TO ( 10, 20, 30 )K
*
   10    CONTINUE
*
*        N1 = 1, N2 = 2: generate elementary reflector H so that:
*
*        ( scale, X11, X12 ) H = ( 0, 0, * )
*
         U( 1 ) = SCALE
         U( 2 ) = X( 1, 1 )
         U( 3 ) = X( 1, 2 )
         CALL DLARFG( 3, U( 3 ), U, 1, TAU )
         U( 3 ) = ONE
         T11 = T( J1, J1 )
*
```

```
*        Perform swap provisionally on diagonal block in D.
*
         CALL DLARFX( 'L', 3, 3, U, TAU, D, LDD, WORK )
         CALL DLARFX( 'R', 3, 3, U, TAU, D, LDD, WORK )
*
*        Test whether to reject swap.
*
         IF( MAX( ABS( D( 3, 1 ) ), ABS( D( 3, 2 ) ), ABS( D( 3,
     $       3 )-T11 ) ).GT.THRESH )GO TO 50
*
*        Accept swap: apply transformation to the entire matrix T.
*
         CALL DLARFX( 'L', 3, N-J1+1, U, TAU, T( J1, J1 ), LDT, WORK )
         CALL DLARFX( 'R', J2, 3, U, TAU, T( 1, J1 ), LDT, WORK )
*
         T( J3, J1 ) = ZERO
         T( J3, J2 ) = ZERO
         T( J3, J3 ) = T11
*
         IF( WANTQ ) THEN
*
*           Accumulate transformation in the matrix Q.
*
            CALL DLARFX( 'R', N, 3, U, TAU, Q( 1, J1 ), LDQ, WORK )
         END IF
         GO TO 40
*
   20    CONTINUE
*
*        N1 = 2, N2 = 1: generate elementary reflector H so that:
*
*        H (  -X11 ) = ( * )
*          (  -X21 ) = ( 0 )
*          ( scale ) = ( 0 )
*
         U( 1 ) = -X( 1, 1 )
         U( 2 ) = -X( 2, 1 )
         U( 3 ) = SCALE
         CALL DLARFG( 3, U( 1 ), U( 2 ), 1, TAU )
         U( 1 ) = ONE
         T33 = T( J3, J3 )
*
*        Perform swap provisionally on diagonal block in D.
*
         CALL DLARFX( 'L', 3, 3, U, TAU, D, LDD, WORK )
         CALL DLARFX( 'R', 3, 3, U, TAU, D, LDD, WORK )
*
*        Test whether to reject swap.
*
         IF( MAX( ABS( D( 2, 1 ) ), ABS( D( 3, 1 ) ), ABS( D( 1,
```

```
     $       1 )-T33 ) ).GT.THRESH )GO TO 50
*
*        Accept swap: apply transformation to the entire matrix T.
*
         CALL DLARFX( 'R', J3, 3, U, TAU, T( 1, J1 ), LDT, WORK )
         CALL DLARFX( 'L', 3, N-J1, U, TAU, T( J1, J2 ), LDT, WORK )
*
         T( J1, J1 ) = T33
         T( J2, J1 ) = ZERO
         T( J3, J1 ) = ZERO
*
         IF( WANTQ ) THEN
*
*           Accumulate transformation in the matrix Q.
*
            CALL DLARFX( 'R', N, 3, U, TAU, Q( 1, J1 ), LDQ, WORK )
         END IF
         GO TO 40
*
   30    CONTINUE
*
*        N1 = 2, N2 = 2: generate elementary reflectors H(1) and H(2) so
*        that:
*
*        H(2) H(1) (  -X11  -X12 ) = (  *  * )
*                  (  -X21  -X22 )   (  0  * )
*                  ( scale    0  )   (  0  0 )
*                  (    0  scale )   (  0  0 )
*
         U1( 1 ) = -X( 1, 1 )
         U1( 2 ) = -X( 2, 1 )
         U1( 3 ) = SCALE
         CALL DLARFG( 3, U1( 1 ), U1( 2 ), 1, TAU1 )
         U1( 1 ) = ONE
*
         TEMP = -TAU1*( X( 1, 2 )+U1( 2 )*X( 2, 2 ) )
         U2( 1 ) = -TEMP*U1( 2 ) - X( 2, 2 )
         U2( 2 ) = -TEMP*U1( 3 )
         U2( 3 ) = SCALE
         CALL DLARFG( 3, U2( 1 ), U2( 2 ), 1, TAU2 )
         U2( 1 ) = ONE
*
*        Perform swap provisionally on diagonal block in D.
*
         CALL DLARFX( 'L', 3, 4, U1, TAU1, D, LDD, WORK )
         CALL DLARFX( 'R', 4, 3, U1, TAU1, D, LDD, WORK )
         CALL DLARFX( 'L', 3, 4, U2, TAU2, D( 2, 1 ), LDD, WORK )
         CALL DLARFX( 'R', 4, 3, U2, TAU2, D( 1, 2 ), LDD, WORK )
*
*        Test whether to reject swap.
```

```
*
         IF( MAX( ABS( D( 3, 1 ) ), ABS( D( 3, 2 ) ), ABS( D( 4, 1 ) ),
     $       ABS( D( 4, 2 ) ) ).GT.THRESH )GO TO 50
*
*        Accept swap: apply transformation to the entire matrix T.
*
         CALL DLARFX( 'L', 3, N-J1+1, U1, TAU1, T( J1, J1 ), LDT, WORK )
         CALL DLARFX( 'R', J4, 3, U1, TAU1, T( 1, J1 ), LDT, WORK )
         CALL DLARFX( 'L', 3, N-J1+1, U2, TAU2, T( J2, J1 ), LDT, WORK )
         CALL DLARFX( 'R', J4, 3, U2, TAU2, T( 1, J2 ), LDT, WORK )
*
         T( J3, J1 ) = ZERO
         T( J3, J2 ) = ZERO
         T( J4, J1 ) = ZERO
         T( J4, J2 ) = ZERO
*
         IF( WANTQ ) THEN
*
*           Accumulate transformation in the matrix Q.
*
            CALL DLARFX( 'R', N, 3, U1, TAU1, Q( 1, J1 ), LDQ, WORK )
            CALL DLARFX( 'R', N, 3, U2, TAU2, Q( 1, J2 ), LDQ, WORK )
         END IF
*
   40    CONTINUE
*
         IF( N2.EQ.2 ) THEN
*
*           Standardize new 2-by-2 block T11
*
            CALL DLANV2( T( J1, J1 ), T( J1, J2 ), T( J2, J1 ),
     $                   T( J2, J2 ), WR1, WI1, WR2, WI2, CS, SN )
            CALL DROT( N-J1-1, T( J1, J1+2 ), LDT, T( J2, J1+2 ), LDT,
     $                 CS, SN )
            CALL DROT( J1-1, T( 1, J1 ), 1, T( 1, J2 ), 1, CS, SN )
            IF( WANTQ )
     $         CALL DROT( N, Q( 1, J1 ), 1, Q( 1, J2 ), 1, CS, SN )
         END IF
*
         IF( N1.EQ.2 ) THEN
*
*           Standardize new 2-by-2 block T22
*
            J3 = J1 + N2
            J4 = J3 + 1
            CALL DLANV2( T( J3, J3 ), T( J3, J4 ), T( J4, J3 ),
     $                   T( J4, J4 ), WR1, WI1, WR2, WI2, CS, SN )
            IF( J3+2.LE.N )
     $         CALL DROT( N-J3-1, T( J3, J3+2 ), LDT, T( J4, J3+2 ),
     $                    LDT, CS, SN )
```

```
            CALL DROT( J3-1, T( 1, J3 ), 1, T( 1, J4 ), 1, CS, SN )
            IF( WANTQ )
     $         CALL DROT( N, Q( 1, J3 ), 1, Q( 1, J4 ), 1, CS, SN )
         END IF
*
      END IF
      RETURN
*
*     Exit with INFO = 1 if swap was rejected.
*
   50 CONTINUE
      INFO = 1
      RETURN
*
*     End of DLAEXC
*
      END
      SUBROUTINE DLAEXC( WANTQ, N, T, LDT, Q, LDQ, J1, N1, N2, WORK,
     $                   INFO )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     February 29, 1992
*
*     .. Scalar Arguments ..
      LOGICAL            WANTQ
      INTEGER            INFO, J1, LDQ, LDT, N, N1, N2
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   Q( LDQ, * ), T( LDT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   TEN
      PARAMETER          ( TEN = 1.0D+1 )
      INTEGER            LDD, LDX
      PARAMETER          ( LDD = 4, LDX = 2 )
*     ..
*     .. Local Scalars ..
      INTEGER            IERR, J2, J3, J4, K, ND
      DOUBLE PRECISION   CS, DNORM, EPS, SCALE, SMLNUM, SN, T11, T22,
     $                   T33, TAU, TAU1, TAU2, TEMP, THRESH, WI1, WI2,
     $                   WR1, WR2, XNORM
*     ..
*     .. Local Arrays ..
```

```
      DOUBLE PRECISION   D( LDD, 4 ), U( 3 ), U1( 3 ), U2( 3 ),
     $                   X( LDX, 2 )
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH, DLANGE
      EXTERNAL           DLAMCH, DLANGE
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLACPY, DLANV2, DLARFG, DLARFX, DLARTG, DLASY2,
     $                   DROT
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. N1.EQ.0 .OR. N2.EQ.0 )
     $   RETURN
      IF( J1+N1.GT.N )
     $   RETURN
*
      J2 = J1 + 1
      J3 = J1 + 2
      J4 = J1 + 3
*
      IF( N1.EQ.1 .AND. N2.EQ.1 ) THEN
*
*        Swap two 1-by-1 blocks.
*
         T11 = T( J1, J1 )
         T22 = T( J2, J2 )
*
*        Determine the transformation to perform the interchange.
*
         CALL DLARTG( T( J1, J2 ), T22-T11, CS, SN, TEMP )
*
*        Apply transformation to the matrix T.
*
         IF( J3.LE.N )
     $      CALL DROT( N-J1-1, T( J1, J3 ), LDT, T( J2, J3 ), LDT, CS,
     $                 SN )
         CALL DROT( J1-1, T( 1, J1 ), 1, T( 1, J2 ), 1, CS, SN )
*
         T( J1, J1 ) = T22
         T( J2, J2 ) = T11
*
```

```
         IF( WANTQ ) THEN
*
*           Accumulate transformation in the matrix Q.
*
            CALL DROT( N, Q( 1, J1 ), 1, Q( 1, J2 ), 1, CS, SN )
         END IF
*
      ELSE
*
*        Swapping involves at least one 2-by-2 block.
*
*        Copy the diagonal block of order N1+N2 to the local array D
*        and compute its norm.
*
         ND = N1 + N2
         CALL DLACPY( 'Full', ND, ND, T( J1, J1 ), LDT, D, LDD )
         DNORM = DLANGE( 'Max', ND, ND, D, LDD, WORK )
*
*        Compute machine-dependent threshold for test for accepting
*        swap.
*
         EPS = DLAMCH( 'P' )
         SMLNUM = DLAMCH( 'S' ) / EPS
         THRESH = MAX( TEN*EPS*DNORM, SMLNUM )
*
*        Solve T11*X - X*T22 = scale*T12 for X.
*
         CALL DLASY2( .FALSE., .FALSE., -1, N1, N2, D, LDD,
     $                D( N1+1, N1+1 ), LDD, D( 1, N1+1 ), LDD, SCALE, X,
     $                LDX, XNORM, IERR )
*
*        Swap the adjacent diagonal blocks.
*
         K = N1 + N1 + N2 - 3
         GO TO ( 10, 20, 30 )K
*
   10    CONTINUE
*
*        N1 = 1, N2 = 2: generate elementary reflector H so that:
*
*        ( scale, X11, X12 ) H = ( 0, 0, * )
*
         U( 1 ) = SCALE
         U( 2 ) = X( 1, 1 )
         U( 3 ) = X( 1, 2 )
         CALL DLARFG( 3, U( 3 ), U, 1, TAU )
         U( 3 ) = ONE
         T11 = T( J1, J1 )
*
*        Perform swap provisionally on diagonal block in D.
```

```
*
        CALL DLARFX( 'L', 3, 3, U, TAU, D, LDD, WORK )
        CALL DLARFX( 'R', 3, 3, U, TAU, D, LDD, WORK )
*
*       Test whether to reject swap.
*
        IF( MAX( ABS( D( 3, 1 ) ), ABS( D( 3, 2 ) ), ABS( D( 3,
     $      3 )-T11 ) ).GT.THRESH )GO TO 50
*
*       Accept swap: apply transformation to the entire matrix T.
*
        CALL DLARFX( 'L', 3, N-J1+1, U, TAU, T( J1, J1 ), LDT, WORK )
        CALL DLARFX( 'R', J2, 3, U, TAU, T( 1, J1 ), LDT, WORK )
*
        T( J3, J1 ) = ZERO
        T( J3, J2 ) = ZERO
        T( J3, J3 ) = T11
*
        IF( WANTQ ) THEN
*
*          Accumulate transformation in the matrix Q.
*
           CALL DLARFX( 'R', N, 3, U, TAU, Q( 1, J1 ), LDQ, WORK )
        END IF
        GO TO 40
*
   20   CONTINUE
*
*       N1 = 2, N2 = 1: generate elementary reflector H so that:
*
*       H (  -X11 ) = ( * )
*         (  -X21 ) = ( 0 )
*         ( scale ) = ( 0 )
*
        U( 1 ) = -X( 1, 1 )
        U( 2 ) = -X( 2, 1 )
        U( 3 ) = SCALE
        CALL DLARFG( 3, U( 1 ), U( 2 ), 1, TAU )
        U( 1 ) = ONE
        T33 = T( J3, J3 )
*
*       Perform swap provisionally on diagonal block in D.
*
        CALL DLARFX( 'L', 3, 3, U, TAU, D, LDD, WORK )
        CALL DLARFX( 'R', 3, 3, U, TAU, D, LDD, WORK )
*
*       Test whether to reject swap.
*
        IF( MAX( ABS( D( 2, 1 ) ), ABS( D( 3, 1 ) ), ABS( D( 1,
     $      1 )-T33 ) ).GT.THRESH )GO TO 50
```

```
*
*        Accept swap: apply transformation to the entire matrix T.
*
         CALL DLARFX( 'R', J3, 3, U, TAU, T( 1, J1 ), LDT, WORK )
         CALL DLARFX( 'L', 3, N-J1, U, TAU, T( J1, J2 ), LDT, WORK )
*
         T( J1, J1 ) = T33
         T( J2, J1 ) = ZERO
         T( J3, J1 ) = ZERO
*
         IF( WANTQ ) THEN
*
*           Accumulate transformation in the matrix Q.
*
            CALL DLARFX( 'R', N, 3, U, TAU, Q( 1, J1 ), LDQ, WORK )
         END IF
         GO TO 40
*
   30    CONTINUE
*
*        N1 = 2, N2 = 2: generate elementary reflectors H(1) and H(2) so
*        that:
*
*        H(2) H(1) (  -X11  -X12 ) = (  *  * )
*                  (  -X21  -X22 )   (  0  * )
*                  ( scale    0  )   (  0  0 )
*                  (    0  scale )   (  0  0 )
*
         U1( 1 ) = -X( 1, 1 )
         U1( 2 ) = -X( 2, 1 )
         U1( 3 ) = SCALE
         CALL DLARFG( 3, U1( 1 ), U1( 2 ), 1, TAU1 )
         U1( 1 ) = ONE
*
         TEMP = -TAU1*( X( 1, 2 )+U1( 2 )*X( 2, 2 ) )
         U2( 1 ) = -TEMP*U1( 2 ) - X( 2, 2 )
         U2( 2 ) = -TEMP*U1( 3 )
         U2( 3 ) = SCALE
         CALL DLARFG( 3, U2( 1 ), U2( 2 ), 1, TAU2 )
         U2( 1 ) = ONE
*
*        Perform swap provisionally on diagonal block in D.
*
         CALL DLARFX( 'L', 3, 4, U1, TAU1, D, LDD, WORK )
         CALL DLARFX( 'R', 4, 3, U1, TAU1, D, LDD, WORK )
         CALL DLARFX( 'L', 3, 4, U2, TAU2, D( 2, 1 ), LDD, WORK )
         CALL DLARFX( 'R', 4, 3, U2, TAU2, D( 1, 2 ), LDD, WORK )
*
*        Test whether to reject swap.
*
```

```
      IF( MAX( ABS( D( 3, 1 ) ), ABS( D( 3, 2 ) ), ABS( D( 4, 1 ) ),
     $     ABS( D( 4, 2 ) ) ).GT.THRESH )GO TO 50
*
*     Accept swap: apply transformation to the entire matrix T.
*
      CALL DLARFX( 'L', 3, N-J1+1, U1, TAU1, T( J1, J1 ), LDT, WORK )
      CALL DLARFX( 'R', J4, 3, U1, TAU1, T( 1, J1 ), LDT, WORK )
      CALL DLARFX( 'L', 3, N-J1+1, U2, TAU2, T( J2, J1 ), LDT, WORK )
      CALL DLARFX( 'R', J4, 3, U2, TAU2, T( 1, J2 ), LDT, WORK )
*
      T( J3, J1 ) = ZERO
      T( J3, J2 ) = ZERO
      T( J4, J1 ) = ZERO
      T( J4, J2 ) = ZERO
*
      IF( WANTQ ) THEN
*
*        Accumulate transformation in the matrix Q.
*
         CALL DLARFX( 'R', N, 3, U1, TAU1, Q( 1, J1 ), LDQ, WORK )
         CALL DLARFX( 'R', N, 3, U2, TAU2, Q( 1, J2 ), LDQ, WORK )
      END IF
*
   40 CONTINUE
*
      IF( N2.EQ.2 ) THEN
*
*        Standardize new 2-by-2 block T11
*
         CALL DLANV2( T( J1, J1 ), T( J1, J2 ), T( J2, J1 ),
     $                T( J2, J2 ), WR1, WI1, WR2, WI2, CS, SN )
         CALL DROT( N-J1-1, T( J1, J1+2 ), LDT, T( J2, J1+2 ), LDT,
     $              CS, SN )
         CALL DROT( J1-1, T( 1, J1 ), 1, T( 1, J2 ), 1, CS, SN )
         IF( WANTQ )
     $      CALL DROT( N, Q( 1, J1 ), 1, Q( 1, J2 ), 1, CS, SN )
      END IF
*
      IF( N1.EQ.2 ) THEN
*
*        Standardize new 2-by-2 block T22
*
         J3 = J1 + N2
         J4 = J3 + 1
         CALL DLANV2( T( J3, J3 ), T( J3, J4 ), T( J4, J3 ),
     $                T( J4, J4 ), WR1, WI1, WR2, WI2, CS, SN )
         IF( J3+2.LE.N )
     $      CALL DROT( N-J3-1, T( J3, J3+2 ), LDT, T( J4, J3+2 ),
     $                 LDT, CS, SN )
         CALL DROT( J3-1, T( 1, J3 ), 1, T( 1, J4 ), 1, CS, SN )
```

```
            IF( WANTQ )
     $         CALL DROT( N, Q( 1, J3 ), 1, Q( 1, J4 ), 1, CS, SN )
         END IF
*
      END IF
      RETURN
*
*     Exit with INFO = 1 if swap was rejected.
*
   50 CONTINUE
      INFO = 1
      RETURN
*
*     End of DLAEXC
*
      END
```

———-

90.142.4 lisp code

— dlaexc.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0) (ten 10.0d0) (ldd 4) (ldx 2))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 10.0d0 10.0d0) ten)
           (type (f2cl-lib:integer4 4 4) ldd)
           (type (f2cl-lib:integer4 2 2) ldx)
           (ignorable zero one ten ldd ldx))
  (defun dlaexc (wantq n t$ ldt q ldq j1 n1 n2 work info)
    (declare (type (array double-float (*)) work q t$)
             (type (f2cl-lib:integer4) info n2 n1 j1 ldq ldt n)
             (type f2cl-lib:logical wantq))
    (f2cl-lib:with-multi-array-data
        ((t$ double-float t$-%data% t$-%offset%)
         (q double-float q-%data% q-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((d
              (make-array (the fixnum (reduce #'* (list ldd 4))) :element-type
                          'double-float))
             (u (make-array 3 :element-type 'double-float))
             (u1 (make-array 3 :element-type 'double-float))
             (u2 (make-array 3 :element-type 'double-float))
             (x
              (make-array (the fixnum (reduce #'* (list ldx 2))) :element-type
                          'double-float))
```

```
     (cs 0.0d0) (dnorm 0.0d0) (eps 0.0d0) (scale 0.0d0) (smlnum 0.0d0)
     (sn 0.0d0) (t11 0.0d0) (t22 0.0d0) (t33 0.0d0) (tau 0.0d0)
     (tau1 0.0d0) (tau2 0.0d0) (temp 0.0d0) (thresh 0.0d0) (wi1 0.0d0)
     (wi2 0.0d0) (wr1 0.0d0) (wr2 0.0d0) (xnorm 0.0d0) (ierr 0) (j2 0)
     (j3 0) (j4 0) (k 0) (nd 0))
(declare (type (array double-float (3)) u u1 u2)
         (type (array double-float (*)) d x)
         (type (double-float) cs dnorm eps scale smlnum sn t11 t22 t33
          tau tau1 tau2 temp thresh wi1 wi2 wr1 wr2 xnorm)
         (type (f2cl-lib:integer4) ierr j2 j3 j4 k nd))
(setf info 0)
(if (or (= n 0) (= n1 0) (= n2 0))
    (go end_label))
(if (> (f2cl-lib:int-add j1 n1) n)
    (go end_label))
(setf j2 (f2cl-lib:int-add j1 1))
(setf j3 (f2cl-lib:int-add j1 2))
(setf j4 (f2cl-lib:int-add j1 3))
(cond
 ((and (= n1 1) (= n2 1))
  (setf t11
          (f2cl-lib:fref t$-%data% (j1 j1) ((1 ldt) (1 *))
                         t$-%offset%))
  (setf t22
          (f2cl-lib:fref t$-%data% (j2 j2) ((1 ldt) (1 *))
                         t$-%offset%))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dlartg
       (f2cl-lib:array-slice t$-%data% double-float (j1 j2)
                             ((1 ldt) (1 *)) t$-%offset%)
       (- t22 t11) cs sn temp)
    (declare (ignore var-0 var-1))
    (when var-2 (setf cs var-2))
    (when var-3 (setf sn var-3))
    (when var-4 (setf temp var-4)))
  (if (<= j3 n)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (drot (f2cl-lib:int-sub n j1 1)
           (f2cl-lib:array-slice t$-%data% double-float (j1 j3)
                                 ((1 ldt) (1 *)) t$-%offset%)
           ldt
           (f2cl-lib:array-slice t$-%data% double-float (j2 j3)
                                 ((1 ldt) (1 *)) t$-%offset%)
           ldt cs sn)
        (declare (ignore var-0 var-1 var-3))
        (when var-2 (setf ldt var-2))
        (when var-4 (setf ldt var-4))
        (when var-5 (setf cs var-5))
        (when var-6 (setf sn var-6))))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
      (drot (f2cl-lib:int-sub j1 1)
       (f2cl-lib:array-slice t$-%data% double-float (1 j1)
                             ((1 ldt) (1 *)) t$-%offset%)
       1
       (f2cl-lib:array-slice t$-%data% double-float (1 j2)
                             ((1 ldt) (1 *)) t$-%offset%)
       1 cs sn)
    (declare (ignore var-0 var-1 var-2 var-3 var-4))
    (when var-5 (setf cs var-5))
    (when var-6 (setf sn var-6)))
  (setf (f2cl-lib:fref t$-%data% (j1 j1) ((1 ldt) (1 *)) t$-%offset%)
          t22)
  (setf (f2cl-lib:fref t$-%data% (j2 j2) ((1 ldt) (1 *)) t$-%offset%)
          t11)
  (cond
   (wantq
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (drot n
         (f2cl-lib:array-slice q-%data% double-float (1 j1)
                               ((1 ldq) (1 *)) q-%offset%)
         1
         (f2cl-lib:array-slice q-%data% double-float (1 j2)
                               ((1 ldq) (1 *)) q-%offset%)
         1 cs sn)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0))
      (when var-5 (setf cs var-5))
      (when var-6 (setf sn var-6))))))
 (t
  (tagbody
    (setf nd (f2cl-lib:int-add n1 n2))
    (dlacpy "Full" nd nd
     (f2cl-lib:array-slice t$-%data% double-float (j1 j1)
                           ((1 ldt) (1 *)) t$-%offset%)
     ldt d ldd)
    (setf dnorm
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                (dlange "Max" nd nd d ldd work)
              (declare (ignore var-0 var-3 var-5))
              (when var-1 (setf nd var-1))
              (when var-2 (setf nd var-2))
              (when var-4 (setf ldd var-4))
              ret-val))
    (setf eps (dlamch "P"))
    (setf smlnum (/ (dlamch "S") eps))
    (setf thresh (max (* ten eps dnorm) smlnum))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14 var-15)
```

```
      (dlasy2 f2cl-lib:%false% f2cl-lib:%false% -1 n1 n2 d ldd
       (f2cl-lib:array-slice d double-float
                             ((+ n1 1) (f2cl-lib:int-add n1 1))
                             ((1 ldd) (1 4)))
       ldd
       (f2cl-lib:array-slice d double-float
                             (1 (f2cl-lib:int-add n1 1))
                             ((1 ldd) (1 4)))
       ldd scale x ldx xnorm ierr)
    (declare (ignore var-0 var-1 var-2 var-5 var-7 var-9 var-12))
    (when var-3 (setf n1 var-3))
    (when var-4 (setf n2 var-4))
    (when var-6 (setf ldd var-6))
    (when var-8 (setf ldd var-8))
    (when var-10 (setf ldd var-10))
    (when var-11 (setf scale var-11))
    (when var-13 (setf ldx var-13))
    (when var-14 (setf xnorm var-14))
    (when var-15 (setf ierr var-15)))
  (setf k (f2cl-lib:int-sub (f2cl-lib:int-add n1 n1 n2) 3))
  (f2cl-lib:computed-goto (label10 label20 label30) k)
label10
  (setf (f2cl-lib:fref u (1) ((1 3))) scale)
  (setf (f2cl-lib:fref u (2) ((1 3)))
        (f2cl-lib:fref x (1 1) ((1 ldx) (1 2))))
  (setf (f2cl-lib:fref u (3) ((1 3)))
        (f2cl-lib:fref x (1 2) ((1 ldx) (1 2))))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dlarfg 3 (f2cl-lib:array-slice u double-float (3) ((1 3))) u 1
       tau)
    (declare (ignore var-0 var-1 var-2 var-3))
    (when var-4 (setf tau var-4)))
  (setf (f2cl-lib:fref u (3) ((1 3))) one)
  (setf t11
        (f2cl-lib:fref t$-%data% (j1 j1) ((1 ldt) (1 *))
                       t$-%offset%))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dlarfx "L" 3 3 u tau d ldd work)
    (declare (ignore var-0 var-1 var-2 var-3 var-5 var-7))
    (when var-4 (setf tau var-4))
    (when var-6 (setf ldd var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dlarfx "R" 3 3 u tau d ldd work)
    (declare (ignore var-0 var-1 var-2 var-3 var-5 var-7))
    (when var-4 (setf tau var-4))
    (when var-6 (setf ldd var-6)))
  (if (>
       (max (abs (f2cl-lib:fref d (3 1) ((1 ldd) (1 4))))
```

```
            (abs (f2cl-lib:fref d (3 2) ((1 ldd) (1 4))))
            (abs (- (f2cl-lib:fref d (3 3) ((1 ldd) (1 4))) t11)))
      thresh)
     (go label50))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (dlarfx "L" 3 (f2cl-lib:int-add (f2cl-lib:int-sub n j1) 1) u
      tau
      (f2cl-lib:array-slice t$-%data% double-float (j1 j1)
                            ((1 ldt) (1 *)) t$-%offset%)
      ldt work)
   (declare (ignore var-0 var-1 var-2 var-3 var-5 var-7))
   (when var-4 (setf tau var-4))
   (when var-6 (setf ldt var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (dlarfx "R" j2 3 u tau
      (f2cl-lib:array-slice t$-%data% double-float (1 j1)
                            ((1 ldt) (1 *)) t$-%offset%)
      ldt work)
   (declare (ignore var-0 var-2 var-3 var-5 var-7))
   (when var-1 (setf j2 var-1))
   (when var-4 (setf tau var-4))
   (when var-6 (setf ldt var-6)))
 (setf (f2cl-lib:fref t$-%data% (j3 j1) ((1 ldt) (1 *)) t$-%offset%)
         zero)
 (setf (f2cl-lib:fref t$-%data% (j3 j2) ((1 ldt) (1 *)) t$-%offset%)
         zero)
 (setf (f2cl-lib:fref t$-%data% (j3 j3) ((1 ldt) (1 *)) t$-%offset%)
         t11)
 (cond
  (wantq
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (dlarfx "R" n 3 u tau
        (f2cl-lib:array-slice q-%data% double-float (1 j1)
                              ((1 ldq) (1 *)) q-%offset%)
        ldq work)
     (declare (ignore var-0 var-2 var-3 var-5 var-7))
     (when var-1 (setf n var-1))
     (when var-4 (setf tau var-4))
     (when var-6 (setf ldq var-6)))))
 (go label40)
label20
 (setf (f2cl-lib:fref u (1) ((1 3)))
         (- (f2cl-lib:fref x (1 1) ((1 ldx) (1 2)))))
 (setf (f2cl-lib:fref u (2) ((1 3)))
         (- (f2cl-lib:fref x (2 1) ((1 ldx) (1 2)))))
 (setf (f2cl-lib:fref u (3) ((1 3))) scale)
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
    (dlarfg 3 (f2cl-lib:array-slice u double-float (1) ((1 3)))
     (f2cl-lib:array-slice u double-float (2) ((1 3))) 1 tau)
  (declare (ignore var-0 var-1 var-2 var-3))
  (when var-4 (setf tau var-4)))
(setf (f2cl-lib:fref u (1) ((1 3))) one)
(setf t33
        (f2cl-lib:fref t$-%data% (j3 j3) ((1 ldt) (1 *))
                       t$-%offset%))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dlarfx "L" 3 3 u tau d ldd work)
  (declare (ignore var-0 var-1 var-2 var-3 var-5 var-7))
  (when var-4 (setf tau var-4))
  (when var-6 (setf ldd var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dlarfx "R" 3 3 u tau d ldd work)
  (declare (ignore var-0 var-1 var-2 var-3 var-5 var-7))
  (when var-4 (setf tau var-4))
  (when var-6 (setf ldd var-6)))
(if (>
     (max (abs (f2cl-lib:fref d (2 1) ((1 ldd) (1 4))))
          (abs (f2cl-lib:fref d (3 1) ((1 ldd) (1 4))))
          (abs (- (f2cl-lib:fref d (1 1) ((1 ldd) (1 4))) t33)))
     thresh)
    (go label50))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dlarfx "R" j3 3 u tau
     (f2cl-lib:array-slice t$-%data% double-float (1 j1)
                           ((1 ldt) (1 *)) t$-%offset%)
     ldt work)
  (declare (ignore var-0 var-2 var-3 var-5 var-7))
  (when var-1 (setf j3 var-1))
  (when var-4 (setf tau var-4))
  (when var-6 (setf ldt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dlarfx "L" 3 (f2cl-lib:int-sub n j1) u tau
     (f2cl-lib:array-slice t$-%data% double-float (j1 j2)
                           ((1 ldt) (1 *)) t$-%offset%)
     ldt work)
  (declare (ignore var-0 var-1 var-2 var-3 var-5 var-7))
  (when var-4 (setf tau var-4))
  (when var-6 (setf ldt var-6)))
(setf (f2cl-lib:fref t$-%data% (j1 j1) ((1 ldt) (1 *)) t$-%offset%)
        t33)
(setf (f2cl-lib:fref t$-%data% (j2 j1) ((1 ldt) (1 *)) t$-%offset%)
        zero)
(setf (f2cl-lib:fref t$-%data% (j3 j1) ((1 ldt) (1 *)) t$-%offset%)
```

```
        zero)
 (cond
  (wantq
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
       (dlarfx "R" n 3 u tau
        (f2cl-lib:array-slice q-%data% double-float (1 j1)
                              ((1 ldq) (1 *)) q-%offset%)
        ldq work)
     (declare (ignore var-0 var-2 var-3 var-5 var-7))
     (when var-1 (setf n var-1))
     (when var-4 (setf tau var-4))
     (when var-6 (setf ldq var-6)))))
 (go label40)
label30
 (setf (f2cl-lib:fref u1 (1) ((1 3)))
        (- (f2cl-lib:fref x (1 1) ((1 ldx) (1 2)))))
 (setf (f2cl-lib:fref u1 (2) ((1 3)))
        (- (f2cl-lib:fref x (2 1) ((1 ldx) (1 2)))))
 (setf (f2cl-lib:fref u1 (3) ((1 3))) scale)
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (dlarfg 3 (f2cl-lib:array-slice u1 double-float (1) ((1 3)))
      (f2cl-lib:array-slice u1 double-float (2) ((1 3))) 1 tau1)
   (declare (ignore var-0 var-1 var-2 var-3))
   (when var-4 (setf tau1 var-4)))
 (setf (f2cl-lib:fref u1 (1) ((1 3))) one)
 (setf temp
        (* (- tau1)
           (+ (f2cl-lib:fref x (1 2) ((1 ldx) (1 2)))
              (* (f2cl-lib:fref u1 (2) ((1 3)))
                 (f2cl-lib:fref x (2 2) ((1 ldx) (1 2)))))))
 (setf (f2cl-lib:fref u2 (1) ((1 3)))
        (- (* (- temp) (f2cl-lib:fref u1 (2) ((1 3))))
           (f2cl-lib:fref x (2 2) ((1 ldx) (1 2)))))
 (setf (f2cl-lib:fref u2 (2) ((1 3)))
        (* (- temp) (f2cl-lib:fref u1 (3) ((1 3)))))
 (setf (f2cl-lib:fref u2 (3) ((1 3))) scale)
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
     (dlarfg 3 (f2cl-lib:array-slice u2 double-float (1) ((1 3)))
      (f2cl-lib:array-slice u2 double-float (2) ((1 3))) 1 tau2)
   (declare (ignore var-0 var-1 var-2 var-3))
   (when var-4 (setf tau2 var-4)))
 (setf (f2cl-lib:fref u2 (1) ((1 3))) one)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
     (dlarfx "L" 3 4 u1 tau1 d ldd work)
   (declare (ignore var-0 var-1 var-2 var-3 var-5 var-7))
   (when var-4 (setf tau1 var-4))
   (when var-6 (setf ldd var-6)))
 (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dlarfx "R" 4 3 u1 tau1 d ldd work)
  (declare (ignore var-0 var-1 var-2 var-3 var-5 var-7))
  (when var-4 (setf tau1 var-4))
  (when var-6 (setf ldd var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dlarfx "L" 3 4 u2 tau2
     (f2cl-lib:array-slice d double-float (2 1) ((1 ldd) (1 4)))
     ldd work)
  (declare (ignore var-0 var-1 var-2 var-3 var-5 var-7))
  (when var-4 (setf tau2 var-4))
  (when var-6 (setf ldd var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dlarfx "R" 4 3 u2 tau2
     (f2cl-lib:array-slice d double-float (1 2) ((1 ldd) (1 4)))
     ldd work)
  (declare (ignore var-0 var-1 var-2 var-3 var-5 var-7))
  (when var-4 (setf tau2 var-4))
  (when var-6 (setf ldd var-6)))
(if (>
     (max (abs (f2cl-lib:fref d (3 1) ((1 ldd) (1 4))))
          (abs (f2cl-lib:fref d (3 2) ((1 ldd) (1 4))))
          (abs (f2cl-lib:fref d (4 1) ((1 ldd) (1 4))))
          (abs (f2cl-lib:fref d (4 2) ((1 ldd) (1 4)))))
     thresh)
    (go label50))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dlarfx "L" 3 (f2cl-lib:int-add (f2cl-lib:int-sub n j1) 1) u1
     tau1
     (f2cl-lib:array-slice t$-%data% double-float (j1 j1)
                           ((1 ldt) (1 *)) t$-%offset%)
     ldt work)
  (declare (ignore var-0 var-1 var-2 var-3 var-5 var-7))
  (when var-4 (setf tau1 var-4))
  (when var-6 (setf ldt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dlarfx "R" j4 3 u1 tau1
     (f2cl-lib:array-slice t$-%data% double-float (1 j1)
                           ((1 ldt) (1 *)) t$-%offset%)
     ldt work)
  (declare (ignore var-0 var-2 var-3 var-5 var-7))
  (when var-1 (setf j4 var-1))
  (when var-4 (setf tau1 var-4))
  (when var-6 (setf ldt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
```

```
        (dlarfx "L" 3 (f2cl-lib:int-add (f2cl-lib:int-sub n j1) 1) u2
         tau2
         (f2cl-lib:array-slice t$-%data% double-float (j2 j1)
                               ((1 ldt) (1 *)) t$-%offset%)
         ldt work)
      (declare (ignore var-0 var-1 var-2 var-3 var-5 var-7))
      (when var-4 (setf tau2 var-4))
      (when var-6 (setf ldt var-6)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (dlarfx "R" j4 3 u2 tau2
         (f2cl-lib:array-slice t$-%data% double-float (1 j2)
                               ((1 ldt) (1 *)) t$-%offset%)
         ldt work)
      (declare (ignore var-0 var-2 var-3 var-5 var-7))
      (when var-1 (setf j4 var-1))
      (when var-4 (setf tau2 var-4))
      (when var-6 (setf ldt var-6)))
    (setf (f2cl-lib:fref t$-%data% (j3 j1) ((1 ldt) (1 *)) t$-%offset%)
            zero)
    (setf (f2cl-lib:fref t$-%data% (j3 j2) ((1 ldt) (1 *)) t$-%offset%)
            zero)
    (setf (f2cl-lib:fref t$-%data% (j4 j1) ((1 ldt) (1 *)) t$-%offset%)
            zero)
    (setf (f2cl-lib:fref t$-%data% (j4 j2) ((1 ldt) (1 *)) t$-%offset%)
            zero)
    (cond
     (wantq
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (dlarfx "R" n 3 u1 tau1
           (f2cl-lib:array-slice q-%data% double-float (1 j1)
                                 ((1 ldq) (1 *)) q-%offset%)
           ldq work)
        (declare (ignore var-0 var-2 var-3 var-5 var-7))
        (when var-1 (setf n var-1))
        (when var-4 (setf tau1 var-4))
        (when var-6 (setf ldq var-6)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (dlarfx "R" n 3 u2 tau2
           (f2cl-lib:array-slice q-%data% double-float (1 j2)
                                 ((1 ldq) (1 *)) q-%offset%)
           ldq work)
        (declare (ignore var-0 var-2 var-3 var-5 var-7))
        (when var-1 (setf n var-1))
        (when var-4 (setf tau2 var-4))
        (when var-6 (setf ldq var-6)))))
  label40
    (cond
```

```
((= n2 2)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
     (dlanv2
      (f2cl-lib:array-slice t$-%data% double-float (j1 j1)
                            ((1 ldt) (1 *)) t$-%offset%)
      (f2cl-lib:array-slice t$-%data% double-float (j1 j2)
                            ((1 ldt) (1 *)) t$-%offset%)
      (f2cl-lib:array-slice t$-%data% double-float (j2 j1)
                            ((1 ldt) (1 *)) t$-%offset%)
      (f2cl-lib:array-slice t$-%data% double-float (j2 j2)
                            ((1 ldt) (1 *)) t$-%offset%)
      wr1 wi1 wr2 wi2 cs sn)
   (declare (ignore var-0 var-1 var-2 var-3))
   (when var-4 (setf wr1 var-4))
   (when var-5 (setf wi1 var-5))
   (when var-6 (setf wr2 var-6))
   (when var-7 (setf wi2 var-7))
   (when var-8 (setf cs var-8))
   (when var-9 (setf sn var-9)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (drot (f2cl-lib:int-sub n j1 1)
      (f2cl-lib:array-slice t$-%data% double-float
                            (j1 (f2cl-lib:int-add j1 2))
                            ((1 ldt) (1 *)) t$-%offset%)
      ldt
      (f2cl-lib:array-slice t$-%data% double-float
                            (j2 (f2cl-lib:int-add j1 2))
                            ((1 ldt) (1 *)) t$-%offset%)
      ldt cs sn)
   (declare (ignore var-0 var-1 var-3))
   (when var-2 (setf ldt var-2))
   (when var-4 (setf ldt var-4))
   (when var-5 (setf cs var-5))
   (when var-6 (setf sn var-6)))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (drot (f2cl-lib:int-sub j1 1)
      (f2cl-lib:array-slice t$-%data% double-float (1 j1)
                            ((1 ldt) (1 *)) t$-%offset%)
      1
      (f2cl-lib:array-slice t$-%data% double-float (1 j2)
                            ((1 ldt) (1 *)) t$-%offset%)
      1 cs sn)
   (declare (ignore var-0 var-1 var-2 var-3 var-4))
   (when var-5 (setf cs var-5))
   (when var-6 (setf sn var-6)))
 (if wantq
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (drot n
```

```
                (f2cl-lib:array-slice q-%data% double-float (1 j1)
                                      ((1 ldq) (1 *)) q-%offset%)
                1
                (f2cl-lib:array-slice q-%data% double-float (1 j2)
                                      ((1 ldq) (1 *)) q-%offset%)
                1 cs sn)
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf n var-0))
              (when var-5 (setf cs var-5))
              (when var-6 (setf sn var-6))))))
      (cond
       ((= n1 2) (setf j3 (f2cl-lib:int-add j1 n2))
        (setf j4 (f2cl-lib:int-add j3 1))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
            (dlanv2
             (f2cl-lib:array-slice t$-%data% double-float (j3 j3)
                                   ((1 ldt) (1 *)) t$-%offset%)
             (f2cl-lib:array-slice t$-%data% double-float (j3 j4)
                                   ((1 ldt) (1 *)) t$-%offset%)
             (f2cl-lib:array-slice t$-%data% double-float (j4 j3)
                                   ((1 ldt) (1 *)) t$-%offset%)
             (f2cl-lib:array-slice t$-%data% double-float (j4 j4)
                                   ((1 ldt) (1 *)) t$-%offset%)
             wr1 wi1 wr2 wi2 cs sn)
          (declare (ignore var-0 var-1 var-2 var-3))
          (when var-4 (setf wr1 var-4))
          (when var-5 (setf wi1 var-5))
          (when var-6 (setf wr2 var-6))
          (when var-7 (setf wi2 var-7))
          (when var-8 (setf cs var-8))
          (when var-9 (setf sn var-9)))
        (if (<= (f2cl-lib:int-add j3 2) n)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (drot (f2cl-lib:int-sub n j3 1)
                 (f2cl-lib:array-slice t$-%data% double-float
                                       (j3 (f2cl-lib:int-add j3 2))
                                       ((1 ldt) (1 *)) t$-%offset%)
                 ldt
                 (f2cl-lib:array-slice t$-%data% double-float
                                       (j4 (f2cl-lib:int-add j3 2))
                                       ((1 ldt) (1 *)) t$-%offset%)
                 ldt cs sn)
              (declare (ignore var-0 var-1 var-3))
              (when var-2 (setf ldt var-2))
              (when var-4 (setf ldt var-4))
              (when var-5 (setf cs var-5))
              (when var-6 (setf sn var-6))))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
                (drot (f2cl-lib:int-sub j3 1)
                 (f2cl-lib:array-slice t$-%data% double-float (1 j3)
                                       ((1 ldt) (1 *)) t$-%offset%)
                 1
                 (f2cl-lib:array-slice t$-%data% double-float (1 j4)
                                       ((1 ldt) (1 *)) t$-%offset%)
                 1 cs sn)
              (declare (ignore var-0 var-1 var-2 var-3 var-4))
              (when var-5 (setf cs var-5))
              (when var-6 (setf sn var-6)))
            (if wantq
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (drot n
                     (f2cl-lib:array-slice q-%data% double-float (1 j3)
                                           ((1 ldq) (1 *)) q-%offset%)
                     1
                     (f2cl-lib:array-slice q-%data% double-float (1 j4)
                                           ((1 ldq) (1 *)) q-%offset%)
                     1 cs sn)
                  (declare (ignore var-1 var-2 var-3 var-4))
                  (when var-0 (setf n var-0))
                  (when var-5 (setf cs var-5))
                  (when var-6 (setf sn var-6)))))))))
 (go end_label)
label50
 (setf info 1)
 (go end_label)
end_label
 (return (values nil n nil ldt nil ldq nil n1 n2 nil info))))))
```

———-

90.143 dlahqr

90.143.1 Axiom unit tests

— dlahqr.input —

```
)set break resume
)sys rm -f dlahqr.output
)spool dlahqr.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

90.143.2 Axiom help page

— **dlahqr.help** —

```
==================================================================
dlahqr examples
==================================================================

==================================================================
Man Page Details
==================================================================

NAME
       DLAHQR - An auxiliary routine called by DHSEQR to update the  eigenval-
       ues and Schur decomposition already computed  by  DHSEQR,  by   dealing
       with the Hessenberg submatrix in rows and columns ILO to IHI

SYNOPSIS
       SUBROUTINE DLAHQR( WANTT,  WANTZ,  N,  ILO,  IHI, H, LDH, WR, WI, ILOZ,
                          IHIZ, Z, LDZ, INFO )

           INTEGER        IHI, IHIZ, ILO, ILOZ, INFO, LDH, LDZ, N

           LOGICAL        WANTT, WANTZ

           DOUBLE         PRECISION H( LDH, * ), WI( * ), WR( * ), Z( LDZ, * )

 Purpose
 =======

  DLAHQR is an auxiliary routine called by DHSEQR to update the
  eigenvalues and Schur decomposition already computed by DHSEQR, by
  dealing with the Hessenberg submatrix in rows and columns ILO to IHI.

 Arguments
 =========

  WANTT   (input) LOGICAL
          = .TRUE. : the full Schur form T is required;
          = .FALSE.: only eigenvalues are required.

  WANTZ   (input) LOGICAL
          = .TRUE. : the matrix of Schur vectors Z is required;
```

```
        = .FALSE.: Schur vectors are not required.

N       (input) INTEGER
        The order of the matrix H.  N >= 0.

ILO     (input) INTEGER
IHI     (input) INTEGER
        It is assumed that H is already upper quasi-triangular in
        rows and columns IHI+1:N, and that H(ILO,ILO-1) = 0 (unless
        ILO = 1). DLAHQR works primarily with the Hessenberg
        submatrix in rows and columns ILO to IHI, but applies
        transformations to all of H if WANTT is .TRUE..
        1 <= ILO <= max(1,IHI); IHI <= N.

H       (input/output) DOUBLE PRECISION array, dimension (LDH,N)
        On entry, the upper Hessenberg matrix H.
        On exit, if WANTT is .TRUE., H is upper quasi-triangular in
        rows and columns ILO:IHI, with any 2-by-2 diagonal blocks in
        standard form. If WANTT is .FALSE., the contents of H are
        unspecified on exit.

LDH     (input) INTEGER
        The leading dimension of the array H. LDH >= max(1,N).

WR      (output) DOUBLE PRECISION array, dimension (N)
WI      (output) DOUBLE PRECISION array, dimension (N)
        The real and imaginary parts, respectively, of the computed
        eigenvalues ILO to IHI are stored in the corresponding
        elements of WR and WI. If two eigenvalues are computed as a
        complex conjugate pair, they are stored in consecutive
        elements of WR and WI, say the i-th and (i+1)th, with
        WI(i) > 0 and WI(i+1) < 0. If WANTT is .TRUE., the
        eigenvalues are stored in the same order as on the diagonal
        of the Schur form returned in H, with WR(i) = H(i,i), and, if
        H(i:i+1,i:i+1) is a 2-by-2 diagonal block,
        WI(i) = sqrt(H(i+1,i)*H(i,i+1)) and WI(i+1) = -WI(i).

ILOZ    (input) INTEGER
IHIZ    (input) INTEGER
        Specify the rows of Z to which transformations must be
        applied if WANTZ is .TRUE..
        1 <= ILOZ <= ILO; IHI <= IHIZ <= N.

Z       (input/output) DOUBLE PRECISION array, dimension (LDZ,N)
        If WANTZ is .TRUE., on entry Z must contain the current
        matrix Z of transformations accumulated by DHSEQR, and on
        exit Z has been updated; transformations are applied only to
        the submatrix Z(ILOZ:IHIZ,ILO:IHI).
        If WANTZ is .FALSE., Z is not referenced.
```

```
 LDZ     (input) INTEGER
         The leading dimension of the array Z. LDZ >= max(1,N).

 INFO    (output) INTEGER
         = 0: successful exit
         > 0: DLAHQR failed to compute all the eigenvalues ILO to IHI
              in a total of 30*(IHI-ILO+1) iterations; if INFO = i,
              elements i+1:ihi of WR and WI contain those eigenvalues
              which have been successfully computed.

 Further Details
 ===============

 2-96 Based on modifications by
    David Day, Sandia National Laboratory, USA
```

90.143.3 fortran code

— dlahqr.f —

```
*  =====================================================================
      SUBROUTINE DLAHQR( WANTT, WANTZ, N, ILO, IHI, H, LDH, WR, WI,
     $                   ILOZ, IHIZ, Z, LDZ, INFO )
*
*  -- LAPACK auxiliary routine (version 3.6.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2015
*
*     .. Scalar Arguments ..
      INTEGER            IHI, IHIZ, ILO, ILOZ, INFO, LDH, LDZ, N
      LOGICAL            WANTT, WANTZ
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   H( LDH, * ), WI( * ), WR( * ), Z( LDZ, * )
*     ..
*
*  =====================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE, TWO
      PARAMETER          ( ZERO = 0.0d0, ONE = 1.0d0, TWO = 2.0d0 )
      DOUBLE PRECISION   DAT1, DAT2
      PARAMETER          ( DAT1 = 3.0d0 / 4.0d0, DAT2 = -0.4375d0 )
*     ..
```

```
*     .. Local Scalars ..
      DOUBLE PRECISION   AA, AB, BA, BB, CS, DET, H11, H12, H21, H21S,
     $                   H22, RT1I, RT1R, RT2I, RT2R, RTDISC, S, SAFMAX,
     $                   SAFMIN, SMLNUM, SN, SUM, T1, T2, T3, TR, TST,
     $                   ULP, V2, V3
      INTEGER            I, I1, I2, ITS, ITMAX, J, K, L, M, NH, NR, NZ
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   V( 3 )
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DLABAD, DLANV2, DLARFG, DROT
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
      IF( ILO.EQ.IHI ) THEN
         WR( ILO ) = H( ILO, ILO )
         WI( ILO ) = ZERO
         RETURN
      END IF
*
*     ==== clear out the trash ====
      DO 10 J = ILO, IHI - 3
         H( J+2, J ) = ZERO
         H( J+3, J ) = ZERO
   10 CONTINUE
      IF( ILO.LE.IHI-2 )
     $   H( IHI, IHI-2 ) = ZERO
*
      NH = IHI - ILO + 1
      NZ = IHIZ - ILOZ + 1
*
*     Set machine-dependent constants for the stopping criterion.
*
      SAFMIN = DLAMCH( 'SAFE MINIMUM' )
      SAFMAX = ONE / SAFMIN
      CALL DLABAD( SAFMIN, SAFMAX )
```

```
      ULP = DLAMCH( 'PRECISION' )
      SMLNUM = SAFMIN*( DBLE( NH ) / ULP )
*
*     I1 and I2 are the indices of the first row and last column of H
*     to which transformations must be applied. If eigenvalues only are
*     being computed, I1 and I2 are set inside the main loop.
*
      IF( WANTT ) THEN
         I1 = 1
         I2 = N
      END IF
*
*     ITMAX is the total number of QR iterations allowed.
*
      ITMAX = 30 * MAX( 10, NH )
*
*     The main loop begins here. I is the loop index and decreases from
*     IHI to ILO in steps of 1 or 2. Each iteration of the loop works
*     with the active submatrix in rows and columns L to I.
*     Eigenvalues I+1 to IHI have already converged. Either L = ILO or
*     H(L,L-1) is negligible so that the matrix splits.
*
      I = IHI
   20 CONTINUE
      L = ILO
      IF( I.LT.ILO )
     $   GO TO 160
*
*     Perform QR iterations on rows and columns ILO to I until a
*     submatrix of order 1 or 2 splits off at the bottom because a
*     subdiagonal element has become negligible.
*
      DO 140 ITS = 0, ITMAX
*
*        Look for a single small subdiagonal element.
*
         DO 30 K = I, L + 1, -1
            IF( ABS( H( K, K-1 ) ).LE.SMLNUM )
     $         GO TO 40
            TST = ABS( H( K-1, K-1 ) ) + ABS( H( K, K ) )
            IF( TST.EQ.ZERO ) THEN
               IF( K-2.GE.ILO )
     $            TST = TST + ABS( H( K-1, K-2 ) )
               IF( K+1.LE.IHI )
     $            TST = TST + ABS( H( K+1, K ) )
            END IF
*           ==== The following is a conservative small subdiagonal
*           .    deflation  criterion due to Ahues & Tisseur (LAWN 122,
*           .    1997). It has better mathematical foundation and
*           .    improves accuracy in some cases.  ====
```

```
            IF( ABS( H( K, K-1 ) ).LE.ULP*TST ) THEN
               AB = MAX( ABS( H( K, K-1 ) ), ABS( H( K-1, K ) ) )
               BA = MIN( ABS( H( K, K-1 ) ), ABS( H( K-1, K ) ) )
               AA = MAX( ABS( H( K, K ) ),
     $              ABS( H( K-1, K-1 )-H( K, K ) ) )
               BB = MIN( ABS( H( K, K ) ),
     $              ABS( H( K-1, K-1 )-H( K, K ) ) )
               S = AA + AB
               IF( BA*( AB / S ).LE.MAX( SMLNUM,
     $             ULP*( BB*( AA / S ) ) ) )GO TO 40
            END IF
   30    CONTINUE
   40    CONTINUE
         L = K
         IF( L.GT.ILO ) THEN
*
*          H(L,L-1) is negligible
*
            H( L, L-1 ) = ZERO
         END IF
*
*        Exit from loop if a submatrix of order 1 or 2 has split off.
*
         IF( L.GE.I-1 )
     $      GO TO 150
*
*        Now the active submatrix is in rows and columns L to I. If
*        eigenvalues only are being computed, only the active submatrix
*        need be transformed.
*
         IF( .NOT.WANTT ) THEN
            I1 = L
            I2 = I
         END IF
*
         IF( ITS.EQ.10 ) THEN
*
*          Exceptional shift.
*
            S = ABS( H( L+1, L ) ) + ABS( H( L+2, L+1 ) )
            H11 = DAT1*S + H( L, L )
            H12 = DAT2*S
            H21 = S
            H22 = H11
         ELSE IF( ITS.EQ.20 ) THEN
*
*          Exceptional shift.
*
            S = ABS( H( I, I-1 ) ) + ABS( H( I-1, I-2 ) )
            H11 = DAT1*S + H( I, I )
```

```
            H12 = DAT2*S
            H21 = S
            H22 = H11
         ELSE
*
*           Prepare to use Francis' double shift
*           (i.e. 2nd degree generalized Rayleigh quotient)
*
            H11 = H( I-1, I-1 )
            H21 = H( I, I-1 )
            H12 = H( I-1, I )
            H22 = H( I, I )
         END IF
         S = ABS( H11 ) + ABS( H12 ) + ABS( H21 ) + ABS( H22 )
         IF( S.EQ.ZERO ) THEN
            RT1R = ZERO
            RT1I = ZERO
            RT2R = ZERO
            RT2I = ZERO
         ELSE
            H11 = H11 / S
            H21 = H21 / S
            H12 = H12 / S
            H22 = H22 / S
            TR = ( H11+H22 ) / TWO
            DET = ( H11-TR )*( H22-TR ) - H12*H21
            RTDISC = SQRT( ABS( DET ) )
            IF( DET.GE.ZERO ) THEN
*
*              ==== complex conjugate shifts ====
*
               RT1R = TR*S
               RT2R = RT1R
               RT1I = RTDISC*S
               RT2I = -RT1I
            ELSE
*
*              ==== real shifts (use only one of them)  ====
*
               RT1R = TR + RTDISC
               RT2R = TR - RTDISC
               IF( ABS( RT1R-H22 ).LE.ABS( RT2R-H22 ) ) THEN
                  RT1R = RT1R*S
                  RT2R = RT1R
               ELSE
                  RT2R = RT2R*S
                  RT1R = RT2R
               END IF
               RT1I = ZERO
               RT2I = ZERO
```

```
         END IF
      END IF
*
*     Look for two consecutive small subdiagonal elements.
*
      DO 50 M = I - 2, L, -1
*        Determine the effect of starting the double-shift QR
*        iteration at row M, and see if this would make H(M,M-1)
*        negligible.  (The following uses scaling to avoid
*        overflows and most underflows.)
*
         H21S = H( M+1, M )
         S = ABS( H( M, M )-RT2R ) + ABS( RT2I ) + ABS( H21S )
         H21S = H( M+1, M ) / S
         V( 1 ) = H21S*H( M, M+1 ) + ( H( M, M )-RT1R )*
     $              ( ( H( M, M )-RT2R ) / S ) - RT1I*( RT2I / S )
         V( 2 ) = H21S*( H( M, M )+H( M+1, M+1 )-RT1R-RT2R )
         V( 3 ) = H21S*H( M+2, M+1 )
         S = ABS( V( 1 ) ) + ABS( V( 2 ) ) + ABS( V( 3 ) )
         V( 1 ) = V( 1 ) / S
         V( 2 ) = V( 2 ) / S
         V( 3 ) = V( 3 ) / S
         IF( M.EQ.L )
     $      GO TO 60
         IF( ABS( H( M, M-1 ) )*( ABS( V( 2 ) )+ABS( V( 3 ) ) ).LE.
     $       ULP*ABS( V( 1 ) )*( ABS( H( M-1, M-1 ) )+ABS( H( M,
     $       M ) )+ABS( H( M+1, M+1 ) ) ) )GO TO 60
   50 CONTINUE
   60 CONTINUE
*
*     Double-shift QR step
*
      DO 130 K = M, I - 1
*
*        The first iteration of this loop determines a reflection G
*        from the vector V and applies it from left and right to H,
*        thus creating a nonzero bulge below the subdiagonal.
*
*        Each subsequent iteration determines a reflection G to
*        restore the Hessenberg form in the (K-1)th column, and thus
*        chases the bulge one step toward the bottom of the active
*        submatrix. NR is the order of G.
*
         NR = MIN( 3, I-K+1 )
         IF( K.GT.M )
     $      CALL DCOPY( NR, H( K, K-1 ), 1, V, 1 )
         CALL DLARFG( NR, V( 1 ), V( 2 ), 1, T1 )
         IF( K.GT.M ) THEN
            H( K, K-1 ) = V( 1 )
            H( K+1, K-1 ) = ZERO
```

```
               IF( K.LT.I-1 )
     $            H( K+2, K-1 ) = ZERO
            ELSE IF( M.GT.L ) THEN
*               ==== Use the following instead of
*               .    H( K, K-1 ) = -H( K, K-1 ) to
*               .    avoid a bug when v(2) and v(3)
*               .    underflow. ====
               H( K, K-1 ) = H( K, K-1 )*( ONE-T1 )
            END IF
            V2 = V( 2 )
            T2 = T1*V2
            IF( NR.EQ.3 ) THEN
               V3 = V( 3 )
               T3 = T1*V3
*
*              Apply G from the left to transform the rows of the matrix
*              in columns K to I2.
*
               DO 70 J = K, I2
                  SUM = H( K, J ) + V2*H( K+1, J ) + V3*H( K+2, J )
                  H( K, J ) = H( K, J ) - SUM*T1
                  H( K+1, J ) = H( K+1, J ) - SUM*T2
                  H( K+2, J ) = H( K+2, J ) - SUM*T3
   70          CONTINUE
*
*              Apply G from the right to transform the columns of the
*              matrix in rows I1 to min(K+3,I).
*
               DO 80 J = I1, MIN( K+3, I )
                  SUM = H( J, K ) + V2*H( J, K+1 ) + V3*H( J, K+2 )
                  H( J, K ) = H( J, K ) - SUM*T1
                  H( J, K+1 ) = H( J, K+1 ) - SUM*T2
                  H( J, K+2 ) = H( J, K+2 ) - SUM*T3
   80          CONTINUE
*
               IF( WANTZ ) THEN
*
*                 Accumulate transformations in the matrix Z
*
                  DO 90 J = ILOZ, IHIZ
                     SUM = Z( J, K ) + V2*Z( J, K+1 ) + V3*Z( J, K+2 )
                     Z( J, K ) = Z( J, K ) - SUM*T1
                     Z( J, K+1 ) = Z( J, K+1 ) - SUM*T2
                     Z( J, K+2 ) = Z( J, K+2 ) - SUM*T3
   90             CONTINUE
               END IF
            ELSE IF( NR.EQ.2 ) THEN
*
*              Apply G from the left to transform the rows of the matrix
*              in columns K to I2.
```

```
*
              DO 100 J = K, I2
                 SUM = H( K, J ) + V2*H( K+1, J )
                 H( K, J ) = H( K, J ) - SUM*T1
                 H( K+1, J ) = H( K+1, J ) - SUM*T2
  100         CONTINUE
*
*             Apply G from the right to transform the columns of the
*             matrix in rows I1 to min(K+3,I).
*
              DO 110 J = I1, I
                 SUM = H( J, K ) + V2*H( J, K+1 )
                 H( J, K ) = H( J, K ) - SUM*T1
                 H( J, K+1 ) = H( J, K+1 ) - SUM*T2
  110         CONTINUE
*
              IF( WANTZ ) THEN
*
*                Accumulate transformations in the matrix Z
*
                 DO 120 J = ILOZ, IHIZ
                    SUM = Z( J, K ) + V2*Z( J, K+1 )
                    Z( J, K ) = Z( J, K ) - SUM*T1
                    Z( J, K+1 ) = Z( J, K+1 ) - SUM*T2
  120            CONTINUE
              END IF
           END IF
  130    CONTINUE
*
  140 CONTINUE
*
*     Failure to converge in remaining number of iterations
*
      INFO = I
      RETURN
*
  150 CONTINUE
*
      IF( L.EQ.I ) THEN
*
*        H(I,I-1) is negligible: one eigenvalue has converged.
*
         WR( I ) = H( I, I )
         WI( I ) = ZERO
      ELSE IF( L.EQ.I-1 ) THEN
*
*        H(I-1,I-2) is negligible: a pair of eigenvalues have converged.
*
*        Transform the 2-by-2 submatrix to standard Schur form,
*        and compute and store the eigenvalues.
```

```
*
         CALL DLANV2( H( I-1, I-1 ), H( I-1, I ), H( I, I-1 ),
     $                H( I, I ), WR( I-1 ), WI( I-1 ), WR( I ), WI( I ),
     $                CS, SN )
*
         IF( WANTT ) THEN
*
*           Apply the transformation to the rest of H.
*
            IF( I2.GT.I )
     $         CALL DROT( I2-I, H( I-1, I+1 ), LDH, H( I, I+1 ), LDH,
     $                    CS, SN )
            CALL DROT( I-I1-1, H( I1, I-1 ), 1, H( I1, I ), 1, CS, SN )
         END IF
         IF( WANTZ ) THEN
*
*           Apply the transformation to Z.
*
            CALL DROT( NZ, Z( ILOZ, I-1 ), 1, Z( ILOZ, I ), 1, CS, SN )
         END IF
      END IF
*
*     return to start of the main loop with new value of I.
*
      I = L - 1
      GO TO 20
*
  160 CONTINUE
      RETURN
*
*     End of DLAHQR
*
      END
      SUBROUTINE DLAHQR( WANTT, WANTZ, N, ILO, IHI, H, LDH, WR, WI,
     $                   ILOZ, IHIZ, Z, LDZ, INFO )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      LOGICAL            WANTT, WANTZ
      INTEGER            IHI, IHIZ, ILO, ILOZ, INFO, LDH, LDZ, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   H( LDH, * ), WI( * ), WR( * ), Z( LDZ, * )
*     ..
*
*  =====================================================================
```

```
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE, HALF
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0, HALF = 0.5D0 )
      DOUBLE PRECISION   DAT1, DAT2
      PARAMETER          ( DAT1 = 0.75D+0, DAT2 = -0.4375D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, I1, I2, ITN, ITS, J, K, L, M, NH, NR, NZ
      DOUBLE PRECISION   AVE, CS, DISC, H00, H10, H11, H12, H21, H22,
     $                   H33, H33S, H43H34, H44, H44S, OVFL, S, SMLNUM,
     $                   SN, SUM, T1, T2, T3, TST1, ULP, UNFL, V1, V2,
     $                   V3
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   V( 3 ), WORK( 1 )
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH, DLANHS
      EXTERNAL           DLAMCH, DLANHS
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DLANV2, DLARFG, DROT
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SIGN, SQRT
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
      IF( ILO.EQ.IHI ) THEN
         WR( ILO ) = H( ILO, ILO )
         WI( ILO ) = ZERO
         RETURN
      END IF
*
      NH = IHI - ILO + 1
      NZ = IHIZ - ILOZ + 1
*
*     Set machine-dependent constants for the stopping criterion.
*     If norm(H) <= sqrt(OVFL), overflow should not occur.
*
      UNFL = DLAMCH( 'Safe minimum' )
      OVFL = ONE / UNFL
      CALL DLABAD( UNFL, OVFL )
```

```
      ULP = DLAMCH( 'Precision' )
      SMLNUM = UNFL*( NH / ULP )
*
*     I1 and I2 are the indices of the first row and last column of H
*     to which transformations must be applied. If eigenvalues only are
*     being computed, I1 and I2 are set inside the main loop.
*
      IF( WANTT ) THEN
         I1 = 1
         I2 = N
      END IF
*
*     ITN is the total number of QR iterations allowed.
*
      ITN = 30*NH
*
*     The main loop begins here. I is the loop index and decreases from
*     IHI to ILO in steps of 1 or 2. Each iteration of the loop works
*     with the active submatrix in rows and columns L to I.
*     Eigenvalues I+1 to IHI have already converged. Either L = ILO or
*     H(L,L-1) is negligible so that the matrix splits.
*
      I = IHI
   10 CONTINUE
      L = ILO
      IF( I.LT.ILO )
     $   GO TO 150
*
*     Perform QR iterations on rows and columns ILO to I until a
*     submatrix of order 1 or 2 splits off at the bottom because a
*     subdiagonal element has become negligible.
*
      DO 130 ITS = 0, ITN
*
*        Look for a single small subdiagonal element.
*
         DO 20 K = I, L + 1, -1
            TST1 = ABS( H( K-1, K-1 ) ) + ABS( H( K, K ) )
            IF( TST1.EQ.ZERO )
     $         TST1 = DLANHS( '1', I-L+1, H( L, L ), LDH, WORK )
            IF( ABS( H( K, K-1 ) ).LE.MAX( ULP*TST1, SMLNUM ) )
     $         GO TO 30
   20    CONTINUE
   30    CONTINUE
         L = K
         IF( L.GT.ILO ) THEN
*
*           H(L,L-1) is negligible
*
            H( L, L-1 ) = ZERO
```

```
         END IF
*
*        Exit from loop if a submatrix of order 1 or 2 has split off.
*
         IF( L.GE.I-1 )
     $      GO TO 140
*
*        Now the active submatrix is in rows and columns L to I. If
*        eigenvalues only are being computed, only the active submatrix
*        need be transformed.
*
         IF( .NOT.WANTT ) THEN
            I1 = L
            I2 = I
         END IF
*
         IF( ITS.EQ.10 .OR. ITS.EQ.20 ) THEN
*
*           Exceptional shift.
*
            S = ABS( H( I, I-1 ) ) + ABS( H( I-1, I-2 ) )
            H44 = DAT1*S + H( I, I )
            H33 = H44
            H43H34 = DAT2*S*S
         ELSE
*
*           Prepare to use Francis' double shift
*           (i.e. 2nd degree generalized Rayleigh quotient)
*
            H44 = H( I, I )
            H33 = H( I-1, I-1 )
            H43H34 = H( I, I-1 )*H( I-1, I )
            S = H( I-1, I-2 )*H( I-1, I-2 )
            DISC = ( H33-H44 )*HALF
            DISC = DISC*DISC + H43H34
            IF( DISC.GT.ZERO ) THEN
*
*              Real roots: use Wilkinson's shift twice
*
               DISC = SQRT( DISC )
               AVE = HALF*( H33+H44 )
               IF( ABS( H33 )-ABS( H44 ).GT.ZERO ) THEN
                  H33 = H33*H44 - H43H34
                  H44 = H33 / ( SIGN( DISC, AVE )+AVE )
               ELSE
                  H44 = SIGN( DISC, AVE ) + AVE
               END IF
               H33 = H44
               H43H34 = ZERO
            END IF
```

```
         END IF
*
*        Look for two consecutive small subdiagonal elements.
*
         DO 40 M = I - 2, L, -1
*           Determine the effect of starting the double-shift QR
*           iteration at row M, and see if this would make H(M,M-1)
*           negligible.
*
            H11 = H( M, M )
            H22 = H( M+1, M+1 )
            H21 = H( M+1, M )
            H12 = H( M, M+1 )
            H44S = H44 - H11
            H33S = H33 - H11
            V1 = ( H33S*H44S-H43H34 ) / H21 + H12
            V2 = H22 - H11 - H33S - H44S
            V3 = H( M+2, M+1 )
            S = ABS( V1 ) + ABS( V2 ) + ABS( V3 )
            V1 = V1 / S
            V2 = V2 / S
            V3 = V3 / S
            V( 1 ) = V1
            V( 2 ) = V2
            V( 3 ) = V3
            IF( M.EQ.L )
     $         GO TO 50
            H00 = H( M-1, M-1 )
            H10 = H( M, M-1 )
            TST1 = ABS( V1 )*( ABS( H00 )+ABS( H11 )+ABS( H22 ) )
            IF( ABS( H10 )*( ABS( V2 )+ABS( V3 ) ).LE.ULP*TST1 )
     $         GO TO 50
   40    CONTINUE
   50    CONTINUE
*
*        Double-shift QR step
*
         DO 120 K = M, I - 1
*
*           The first iteration of this loop determines a reflection G
*           from the vector V and applies it from left and right to H,
*           thus creating a nonzero bulge below the subdiagonal.
*
*           Each subsequent iteration determines a reflection G to
*           restore the Hessenberg form in the (K-1)th column, and thus
*           chases the bulge one step toward the bottom of the active
*           submatrix. NR is the order of G.
*
            NR = MIN( 3, I-K+1 )
            IF( K.GT.M )
```

```
     $         CALL DCOPY( NR, H( K, K-1 ), 1, V, 1 )
            CALL DLARFG( NR, V( 1 ), V( 2 ), 1, T1 )
            IF( K.GT.M ) THEN
               H( K, K-1 ) = V( 1 )
               H( K+1, K-1 ) = ZERO
               IF( K.LT.I-1 )
     $            H( K+2, K-1 ) = ZERO
            ELSE IF( M.GT.L ) THEN
               H( K, K-1 ) = -H( K, K-1 )
            END IF
            V2 = V( 2 )
            T2 = T1*V2
            IF( NR.EQ.3 ) THEN
               V3 = V( 3 )
               T3 = T1*V3
*
*              Apply G from the left to transform the rows of the matrix
*              in columns K to I2.
*
               DO 60 J = K, I2
                  SUM = H( K, J ) + V2*H( K+1, J ) + V3*H( K+2, J )
                  H( K, J ) = H( K, J ) - SUM*T1
                  H( K+1, J ) = H( K+1, J ) - SUM*T2
                  H( K+2, J ) = H( K+2, J ) - SUM*T3
   60          CONTINUE
*
*              Apply G from the right to transform the columns of the
*              matrix in rows I1 to min(K+3,I).
*
               DO 70 J = I1, MIN( K+3, I )
                  SUM = H( J, K ) + V2*H( J, K+1 ) + V3*H( J, K+2 )
                  H( J, K ) = H( J, K ) - SUM*T1
                  H( J, K+1 ) = H( J, K+1 ) - SUM*T2
                  H( J, K+2 ) = H( J, K+2 ) - SUM*T3
   70          CONTINUE
*
               IF( WANTZ ) THEN
*
*                 Accumulate transformations in the matrix Z
*
                  DO 80 J = ILOZ, IHIZ
                     SUM = Z( J, K ) + V2*Z( J, K+1 ) + V3*Z( J, K+2 )
                     Z( J, K ) = Z( J, K ) - SUM*T1
                     Z( J, K+1 ) = Z( J, K+1 ) - SUM*T2
                     Z( J, K+2 ) = Z( J, K+2 ) - SUM*T3
   80             CONTINUE
               END IF
            ELSE IF( NR.EQ.2 ) THEN
*
*              Apply G from the left to transform the rows of the matrix
```

```
*              in columns K to I2.
*
               DO 90 J = K, I2
                  SUM = H( K, J ) + V2*H( K+1, J )
                  H( K, J ) = H( K, J ) - SUM*T1
                  H( K+1, J ) = H( K+1, J ) - SUM*T2
   90          CONTINUE
*
*              Apply G from the right to transform the columns of the
*              matrix in rows I1 to min(K+3,I).
*
               DO 100 J = I1, I
                  SUM = H( J, K ) + V2*H( J, K+1 )
                  H( J, K ) = H( J, K ) - SUM*T1
                  H( J, K+1 ) = H( J, K+1 ) - SUM*T2
  100          CONTINUE
*
               IF( WANTZ ) THEN
*
*                 Accumulate transformations in the matrix Z
*
                  DO 110 J = ILOZ, IHIZ
                     SUM = Z( J, K ) + V2*Z( J, K+1 )
                     Z( J, K ) = Z( J, K ) - SUM*T1
                     Z( J, K+1 ) = Z( J, K+1 ) - SUM*T2
  110             CONTINUE
               END IF
            END IF
  120    CONTINUE
*
  130 CONTINUE
*
*     Failure to converge in remaining number of iterations
*
      INFO = I
      RETURN
*
  140 CONTINUE
*
      IF( L.EQ.I ) THEN
*
*        H(I,I-1) is negligible: one eigenvalue has converged.
*
         WR( I ) = H( I, I )
         WI( I ) = ZERO
      ELSE IF( L.EQ.I-1 ) THEN
*
*        H(I-1,I-2) is negligible: a pair of eigenvalues have converged.
*
*        Transform the 2-by-2 submatrix to standard Schur form,
```

```
*        and compute and store the eigenvalues.
*
         CALL DLANV2( H( I-1, I-1 ), H( I-1, I ), H( I, I-1 ),
     $                H( I, I ), WR( I-1 ), WI( I-1 ), WR( I ), WI( I ),
     $                CS, SN )
*
         IF( WANTT ) THEN
*
*           Apply the transformation to the rest of H.
*
            IF( I2.GT.I )
     $         CALL DROT( I2-I, H( I-1, I+1 ), LDH, H( I, I+1 ), LDH,
     $                    CS, SN )
            CALL DROT( I-I1-1, H( I1, I-1 ), 1, H( I1, I ), 1, CS, SN )
         END IF
         IF( WANTZ ) THEN
*
*           Apply the transformation to Z.
*
            CALL DROT( NZ, Z( ILOZ, I-1 ), 1, Z( ILOZ, I ), 1, CS, SN )
         END IF
      END IF
*
*     Decrement number of remaining iterations, and return to start of
*     the main loop with new value of I.
*
      ITN = ITN - ITS
      I = L - 1
      GO TO 10
*
  150 CONTINUE
      RETURN
*
*     End of DLAHQR
*
      END
```

———-

90.143.4 lisp code

— dlahqr.lisp —

```
(let* ((zero 0.0d0)
       (one 1.0d0)
       (two 2.0d0)
       (dat1 (f2cl-lib:f2cl/ 3.0d0 4.0d0))
```

```
      (dat2 (- 0.4375d0)))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float) dat1)
           (type (double-float) dat2)
           (ignorable zero one two dat1 dat2))
  (defun dlahqr (wantt wantz n ilo ihi h ldh wr wi iloz ihiz z ldz info)
    (declare (type (array double-float (*)) z wi wr h)
             (type (f2cl-lib:integer4) info ldz ihiz iloz ldh ihi ilo n)
             (type f2cl-lib:logical wantz wantt))
    (f2cl-lib:with-multi-array-data
        ((h double-float h-%data% h-%offset%)
         (wr double-float wr-%data% wr-%offset%)
         (wi double-float wi-%data% wi-%offset%)
         (z double-float z-%data% z-%offset%))
      (prog ((v (make-array 3 :element-type 'double-float)) (i 0) (i1 0) (i2 0)
             (its 0) (itmax 0) (j 0) (k 0) (l 0) (m 0) (nh 0) (nr 0) (nz 0)
             (aa 0.0d0) (ab 0.0d0) (ba 0.0d0) (bb 0.0d0) (cs 0.0d0) (det 0.0d0)
             (h11 0.0d0) (h12 0.0d0) (h21 0.0d0) (h21s 0.0d0) (h22 0.0d0)
             (rt1i 0.0d0) (rt1r 0.0d0) (rt2i 0.0d0) (rt2r 0.0d0) (rtdisc 0.0d0)
             (s 0.0d0) (safmax 0.0d0) (safmin 0.0d0) (smlnum 0.0d0) (sn 0.0d0)
             (sum 0.0d0) (t1 0.0d0) (t2 0.0d0) (t3 0.0d0) (tr 0.0d0)
             (tst 0.0d0) (ulp 0.0d0) (v2 0.0d0) (v3 0.0d0))
        (declare (type (array double-float (3)) v)
                 (type (f2cl-lib:integer4) i i1 i2 its itmax j k l m nh nr nz)
                 (type (double-float) aa ab ba bb cs det h11 h12 h21 h21s h22
                  rt1i rt1r rt2i rt2r rtdisc s safmax safmin smlnum sn sum t1
                  t2 t3 tr tst ulp v2 v3))
        (setf info 0)
        (if (= n 0)
            (go end_label))
        (cond
         ((= ilo ihi)
          (setf (f2cl-lib:fref wr-%data% (ilo) ((1 *)) wr-%offset%)
                  (f2cl-lib:fref h-%data% (ilo ilo) ((1 ldh) (1 *))
                                 h-%offset%))
          (setf (f2cl-lib:fref wi-%data% (ilo) ((1 *)) wi-%offset%) zero)
          (go end_label)))
        (f2cl-lib:fdo (j ilo (f2cl-lib:int-add j 1))
                      ((> j (f2cl-lib:int-add ihi (f2cl-lib:int-sub 3))) nil)
          (tagbody
            (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add j 2) j)
                                  ((1 ldh) (1 *)) h-%offset%)
                    zero)
            (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add j 3) j)
                                  ((1 ldh) (1 *)) h-%offset%)
                    zero)
           label10))
        (if (<= ilo (f2cl-lib:int-sub ihi 2))
```

```
    (setf (f2cl-lib:fref h-%data% (ihi (f2cl-lib:int-sub ihi 2))
                          ((1 ldh) (1 *)) h-%offset%)
            zero))
 (setf nh (f2cl-lib:int-add (f2cl-lib:int-sub ihi ilo) 1))
 (setf nz (f2cl-lib:int-add (f2cl-lib:int-sub ihiz iloz) 1))
 (setf safmin (dlamch "SAFE MINIMUM"))
 (setf safmax (/ one safmin))
 (multiple-value-bind (var-0 var-1)
     (dlabad safmin safmax)
   (declare (ignore))
   (setf safmin var-0)
   (setf safmax var-1))
 (setf ulp (dlamch "PRECISION"))
 (setf smlnum (* safmin (/ (f2cl-lib:dble nh) ulp)))
 (cond (wantt (setf i1 1) (setf i2 n)))
 (setf itmax
         (f2cl-lib:int-mul 30
                            (max (the f2cl-lib:integer4 10)
                                 (the f2cl-lib:integer4 nh))))
 (setf i ihi)
label20
 (setf l ilo)
 (if (< i ilo)
     (go label160))
 (f2cl-lib:fdo (its 0 (f2cl-lib:int-add its 1))
               ((> its itmax) nil)
   (tagbody
     (f2cl-lib:fdo (k i (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                   ((> k (f2cl-lib:int-add l 1)) nil)
       (tagbody
         (if (<=
              (abs
               (f2cl-lib:fref h-%data% (k (f2cl-lib:int-sub k 1))
                              ((1 ldh) (1 *)) h-%offset%))
              smlnum)
             (go label40))
         (setf tst
                 (+
                  (abs
                   (f2cl-lib:fref h-%data%
                                  ((f2cl-lib:int-sub k 1)
                                   (f2cl-lib:int-sub k 1))
                                  ((1 ldh) (1 *)) h-%offset%))
                  (abs
                   (f2cl-lib:fref h-%data% (k k) ((1 ldh) (1 *))
                                  h-%offset%))))
         (cond
          ((= tst zero)
           (if (>= (f2cl-lib:int-sub k 2) ilo)
               (setf tst
```

```
                (+ tst
                   (abs
                    (f2cl-lib:fref h-%data%
                                   ((f2cl-lib:int-sub k 1)
                                    (f2cl-lib:int-sub k 2))
                                   ((1 ldh) (1 *))
                                   h-%offset%)))))
  (if (<= (f2cl-lib:int-add k 1) ihi)
      (setf tst
                (+ tst
                   (abs
                    (f2cl-lib:fref h-%data%
                                   ((f2cl-lib:int-add k 1) k)
                                   ((1 ldh) (1 *))
                                   h-%offset%)))))))
(cond
 ((<=
   (abs
    (f2cl-lib:fref h
                   (k
                    (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                   ((1 ldh) (1 *))))
   (* ulp tst))
  (setf ab
          (max
           (abs
            (f2cl-lib:fref h-%data% (k (f2cl-lib:int-sub k 1))
                           ((1 ldh) (1 *)) h-%offset%))
           (abs
            (f2cl-lib:fref h-%data% ((f2cl-lib:int-sub k 1) k)
                           ((1 ldh) (1 *)) h-%offset%))))
  (setf ba
          (min
           (abs
            (f2cl-lib:fref h-%data% (k (f2cl-lib:int-sub k 1))
                           ((1 ldh) (1 *)) h-%offset%))
           (abs
            (f2cl-lib:fref h-%data% ((f2cl-lib:int-sub k 1) k)
                           ((1 ldh) (1 *)) h-%offset%))))
  (setf aa
          (max
           (abs
            (f2cl-lib:fref h-%data% (k k) ((1 ldh) (1 *))
                           h-%offset%))
           (abs
            (-
             (f2cl-lib:fref h-%data%
                            ((f2cl-lib:int-sub k 1)
                             (f2cl-lib:int-sub k 1))
                            ((1 ldh) (1 *)) h-%offset%)
```

```
                    (f2cl-lib:fref h-%data% (k k) ((1 ldh) (1 *))
                                   h-%offset%)))))
        (setf bb
                (min
                 (abs
                  (f2cl-lib:fref h-%data% (k k) ((1 ldh) (1 *))
                                 h-%offset%))
                 (abs
                  (-
                   (f2cl-lib:fref h-%data%
                                  ((f2cl-lib:int-sub k 1)
                                   (f2cl-lib:int-sub k 1))
                                  ((1 ldh) (1 *)) h-%offset%)
                   (f2cl-lib:fref h-%data% (k k) ((1 ldh) (1 *))
                                  h-%offset%)))))
        (setf s (+ aa ab))
        (if (<= (* ba (/ ab s)) (max smlnum (* ulp (* bb (/ aa s)))))
            (go label40))))
    label30))
label40
 (setf l k)
 (cond
  ((> l ilo)
   (setf (f2cl-lib:fref h-%data% (l (f2cl-lib:int-sub l 1))
                         ((1 ldh) (1 *)) h-%offset%)
           zero)))
 (if (>= l (f2cl-lib:int-sub i 1))
     (go label150))
 (cond ((not wantt) (setf i1 l) (setf i2 i)))
 (cond
  ((= its 10)
   (setf s
           (+
            (abs
             (f2cl-lib:fref h-%data% ((f2cl-lib:int-add l 1) l)
                            ((1 ldh) (1 *)) h-%offset%))
            (abs
             (f2cl-lib:fref h-%data%
                            ((f2cl-lib:int-add l 2)
                             (f2cl-lib:int-add l 1))
                            ((1 ldh) (1 *)) h-%offset%))))
   (setf h11
           (+ (* dat1 s)
              (f2cl-lib:fref h-%data% (l l) ((1 ldh) (1 *))
                             h-%offset%)))
   (setf h12 (* dat2 s)) (setf h21 s) (setf h22 h11))
  ((= its 20)
   (setf s
           (+
            (abs
```

```
                (f2cl-lib:fref h-%data% (i (f2cl-lib:int-sub i 1))
                               ((1 ldh) (1 *)) h-%offset%))
               (abs
                (f2cl-lib:fref h-%data%
                               ((f2cl-lib:int-sub i 1)
                                (f2cl-lib:int-sub i 2))
                               ((1 ldh) (1 *)) h-%offset%))))
  (setf h11
          (+ (* dat1 s)
             (f2cl-lib:fref h-%data% (i i) ((1 ldh) (1 *))
                            h-%offset%)))
  (setf h12 (* dat2 s)) (setf h21 s) (setf h22 h11))
 (t
  (setf h11
          (f2cl-lib:fref h-%data%
                         ((f2cl-lib:int-sub i 1)
                          (f2cl-lib:int-sub i 1))
                         ((1 ldh) (1 *)) h-%offset%))
  (setf h21
          (f2cl-lib:fref h-%data% (i (f2cl-lib:int-sub i 1))
                         ((1 ldh) (1 *)) h-%offset%))
  (setf h12
          (f2cl-lib:fref h-%data% ((f2cl-lib:int-sub i 1) i)
                         ((1 ldh) (1 *)) h-%offset%))
  (setf h22
          (f2cl-lib:fref h-%data% (i i) ((1 ldh) (1 *))
                         h-%offset%))))
(setf s (+ (abs h11) (abs h12) (abs h21) (abs h22)))
(cond
 ((= s zero) (setf rt1r zero) (setf rt1i zero) (setf rt2r zero)
  (setf rt2i zero))
 (t (setf h11 (/ h11 s)) (setf h21 (/ h21 s)) (setf h12 (/ h12 s))
  (setf h22 (/ h22 s)) (setf tr (/ (+ h11 h22) two))
  (setf det (- (* (- h11 tr) (- h22 tr)) (* h12 h21)))
  (setf rtdisc (f2cl-lib:fsqrt (abs det)))
  (cond
   ((>= det zero) (setf rt1r (* tr s)) (setf rt2r rt1r)
    (setf rt1i (* rtdisc s)) (setf rt2i (- rt1i)))
   (t (setf rt1r (+ tr rtdisc)) (setf rt2r (- tr rtdisc))
    (cond
     ((<= (abs (+ rt1r (- h22))) (abs (+ rt2r (- h22))))
      (setf rt1r (* rt1r s)) (setf rt2r rt1r))
     (t (setf rt2r (* rt2r s)) (setf rt1r rt2r)))
    (setf rt1i zero) (setf rt2i zero)))))
(f2cl-lib:fdo (m (f2cl-lib:int-add i (f2cl-lib:int-sub 2))
               (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
              ((> m l) nil)
  (tagbody
    (setf h21s
            (f2cl-lib:fref h-%data% ((f2cl-lib:int-add m 1) m)
```

```
                        ((1 ldh) (1 *)) h-%offset%))
(setf s
        (+
         (abs
          (-
           (f2cl-lib:fref h-%data% (m m) ((1 ldh) (1 *))
                          h-%offset%)
           rt2r))
         (abs rt2i) (abs h21s)))
(setf h21s
        (/
         (f2cl-lib:fref h-%data% ((f2cl-lib:int-add m 1) m)
                        ((1 ldh) (1 *)) h-%offset%)
         s))
(setf (f2cl-lib:fref v (1) ((1 3)))
        (-
         (+
          (* h21s
             (f2cl-lib:fref h-%data% (m (f2cl-lib:int-add m 1))
                            ((1 ldh) (1 *)) h-%offset%))
          (*
           (-
            (f2cl-lib:fref h-%data% (m m) ((1 ldh) (1 *))
                           h-%offset%)
            rt1r)
           (/
            (-
             (f2cl-lib:fref h-%data% (m m) ((1 ldh) (1 *))
                            h-%offset%)
             rt2r)
            s)))
         (* rt1i (/ rt2i s))))
(setf (f2cl-lib:fref v (2) ((1 3)))
        (* h21s
           (-
            (+
             (f2cl-lib:fref h-%data% (m m) ((1 ldh) (1 *))
                            h-%offset%)
             (f2cl-lib:fref h-%data%
                            ((f2cl-lib:int-add m 1)
                             (f2cl-lib:int-add m 1))
                            ((1 ldh) (1 *)) h-%offset%))
            rt1r rt2r)))
(setf (f2cl-lib:fref v (3) ((1 3)))
        (* h21s
           (f2cl-lib:fref h-%data%
                          ((f2cl-lib:int-add m 2)
                           (f2cl-lib:int-add m 1))
                          ((1 ldh) (1 *)) h-%offset%)))
(setf s
```

```
                (+ (abs (f2cl-lib:fref v (1) ((1 3))))
                   (abs (f2cl-lib:fref v (2) ((1 3))))
                   (abs (f2cl-lib:fref v (3) ((1 3))))))
        (setf (f2cl-lib:fref v (1) ((1 3)))
                (/ (f2cl-lib:fref v (1) ((1 3))) s))
        (setf (f2cl-lib:fref v (2) ((1 3)))
                (/ (f2cl-lib:fref v (2) ((1 3))) s))
        (setf (f2cl-lib:fref v (3) ((1 3)))
                (/ (f2cl-lib:fref v (3) ((1 3))) s))
        (if (= m l)
            (go label60))
        (if (<=
             (*
              (abs
               (f2cl-lib:fref h-%data% (m (f2cl-lib:int-sub m 1))
                              ((1 ldh) (1 *)) h-%offset%))
              (+ (abs (f2cl-lib:fref v (2) ((1 3))))
                 (abs (f2cl-lib:fref v (3) ((1 3))))))
             (* ulp (abs (f2cl-lib:fref v (1) ((1 3))))
                (+
                 (abs
                  (f2cl-lib:fref h-%data%
                                 ((f2cl-lib:int-sub m 1)
                                  (f2cl-lib:int-sub m 1))
                                 ((1 ldh) (1 *)) h-%offset%))
                 (abs
                  (f2cl-lib:fref h-%data% (m m) ((1 ldh) (1 *))
                                 h-%offset%))
                 (abs
                  (f2cl-lib:fref h-%data%
                                 ((f2cl-lib:int-add m 1)
                                  (f2cl-lib:int-add m 1))
                                 ((1 ldh) (1 *)) h-%offset%)))))
            (go label60))
    label50))
label60
 (f2cl-lib:fdo (k m (f2cl-lib:int-add k 1))
               ((> k (f2cl-lib:int-add i (f2cl-lib:int-sub 1))) nil)
   (tagbody
     (setf nr
             (min (the f2cl-lib:integer4 3)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add (f2cl-lib:int-sub i k)
                                         1))))
     (if (> k m)
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (dcopy nr
              (f2cl-lib:array-slice h-%data% double-float
                                    (k (f2cl-lib:int-sub k 1))
                                    ((1 ldh) (1 *)) h-%offset%)
```

```
        1 v 1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf nr var-0))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dlarfg nr
     (f2cl-lib:array-slice v double-float (1) ((1 3)))
     (f2cl-lib:array-slice v double-float (2) ((1 3))) 1 t1)
  (declare (ignore var-1 var-2 var-3))
  (when var-0 (setf nr var-0))
  (when var-4 (setf t1 var-4)))
(cond
 ((> k m)
  (setf (f2cl-lib:fref h-%data% (k (f2cl-lib:int-sub k 1))
                       ((1 ldh) (1 *)) h-%offset%)
          (f2cl-lib:fref v (1) ((1 3))))
  (setf (f2cl-lib:fref h-%data%
                       ((f2cl-lib:int-add k 1)
                        (f2cl-lib:int-sub k 1))
                       ((1 ldh) (1 *)) h-%offset%)
          zero)
  (if (< k (f2cl-lib:int-sub i 1))
      (setf (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-add k 2)
                            (f2cl-lib:int-sub k 1))
                           ((1 ldh) (1 *)) h-%offset%)
              zero)))
 ((> m l)
  (setf (f2cl-lib:fref h-%data% (k (f2cl-lib:int-sub k 1))
                       ((1 ldh) (1 *)) h-%offset%)
          (*
           (f2cl-lib:fref h-%data% (k (f2cl-lib:int-sub k 1))
                          ((1 ldh) (1 *)) h-%offset%)
           (- one t1)))))
(setf v2 (f2cl-lib:fref v (2) ((1 3))))
(setf t2 (* t1 v2))
(cond
 ((= nr 3) (setf v3 (f2cl-lib:fref v (3) ((1 3))))
  (setf t3 (* t1 v3))
  (f2cl-lib:fdo (j k (f2cl-lib:int-add j 1))
                ((> j i2) nil)
    (tagbody
      (setf sum
              (+
               (f2cl-lib:fref h-%data% (k j) ((1 ldh) (1 *))
                              h-%offset%)
               (* v2
                  (f2cl-lib:fref h-%data%
                                 ((f2cl-lib:int-add k 1) j)
                                 ((1 ldh) (1 *)) h-%offset%))
               (* v3
```

```
                    (f2cl-lib:fref h-%data%
                                   ((f2cl-lib:int-add k 2) j)
                                   ((1 ldh) (1 *)) h-%offset%))))
        (setf (f2cl-lib:fref h-%data% (k j) ((1 ldh) (1 *))
                             h-%offset%)
              (-
               (f2cl-lib:fref h-%data% (k j) ((1 ldh) (1 *))
                              h-%offset%)
               (* sum t1)))
        (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add k 1) j)
                             ((1 ldh) (1 *)) h-%offset%)
              (-
               (f2cl-lib:fref h-%data%
                              ((f2cl-lib:int-add k 1) j)
                              ((1 ldh) (1 *)) h-%offset%)
               (* sum t2)))
        (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add k 2) j)
                             ((1 ldh) (1 *)) h-%offset%)
              (-
               (f2cl-lib:fref h-%data%
                              ((f2cl-lib:int-add k 2) j)
                              ((1 ldh) (1 *)) h-%offset%)
               (* sum t3)))
      label70))
    (f2cl-lib:fdo (j i1 (f2cl-lib:int-add j 1))
                  ((> j
                      (min
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add k 3))
                       (the f2cl-lib:integer4 i)))
                   nil)
      (tagbody
        (setf sum
              (+
               (f2cl-lib:fref h-%data% (j k) ((1 ldh) (1 *))
                              h-%offset%)
               (* v2
                  (f2cl-lib:fref h-%data%
                                 (j (f2cl-lib:int-add k 1))
                                 ((1 ldh) (1 *)) h-%offset%))
               (* v3
                  (f2cl-lib:fref h-%data%
                                 (j (f2cl-lib:int-add k 2))
                                 ((1 ldh) (1 *)) h-%offset%))))
        (setf (f2cl-lib:fref h-%data% (j k) ((1 ldh) (1 *))
                             h-%offset%)
              (-
               (f2cl-lib:fref h-%data% (j k) ((1 ldh) (1 *))
                              h-%offset%)
               (* sum t1)))
```

```
    (setf (f2cl-lib:fref h-%data% (j (f2cl-lib:int-add k 1))
                         ((1 ldh) (1 *)) h-%offset%)
            (-
             (f2cl-lib:fref h-%data%
                            (j (f2cl-lib:int-add k 1))
                            ((1 ldh) (1 *)) h-%offset%)
             (* sum t2)))
    (setf (f2cl-lib:fref h-%data% (j (f2cl-lib:int-add k 2))
                         ((1 ldh) (1 *)) h-%offset%)
            (-
             (f2cl-lib:fref h-%data%
                            (j (f2cl-lib:int-add k 2))
                            ((1 ldh) (1 *)) h-%offset%)
             (* sum t3)))
  label80))
(cond
 (wantz
  (f2cl-lib:fdo (j iloz (f2cl-lib:int-add j 1))
                ((> j ihiz) nil)
    (tagbody
      (setf sum
              (+
               (f2cl-lib:fref z-%data% (j k) ((1 ldz) (1 *))
                              z-%offset%)
               (* v2
                  (f2cl-lib:fref z-%data%
                                 (j (f2cl-lib:int-add k 1))
                                 ((1 ldz) (1 *)) z-%offset%))
               (* v3
                  (f2cl-lib:fref z-%data%
                                 (j (f2cl-lib:int-add k 2))
                                 ((1 ldz) (1 *))
                                 z-%offset%))))
      (setf (f2cl-lib:fref z-%data% (j k) ((1 ldz) (1 *))
                           z-%offset%)
              (-
               (f2cl-lib:fref z-%data% (j k) ((1 ldz) (1 *))
                              z-%offset%)
               (* sum t1)))
      (setf (f2cl-lib:fref z-%data%
                           (j (f2cl-lib:int-add k 1))
                           ((1 ldz) (1 *)) z-%offset%)
              (-
               (f2cl-lib:fref z-%data%
                              (j (f2cl-lib:int-add k 1))
                              ((1 ldz) (1 *)) z-%offset%)
               (* sum t2)))
      (setf (f2cl-lib:fref z-%data%
                           (j (f2cl-lib:int-add k 2))
                           ((1 ldz) (1 *)) z-%offset%)
```

```
                 (-
                  (f2cl-lib:fref z-%data%
                                 (j (f2cl-lib:int-add k 2))
                                 ((1 ldz) (1 *)) z-%offset%)
                  (* sum t3)))
      label90)))))
((= nr 2)
 (f2cl-lib:fdo (j k (f2cl-lib:int-add j 1))
               ((> j i2) nil)
   (tagbody
     (setf sum
             (+
              (f2cl-lib:fref h-%data% (k j) ((1 ldh) (1 *))
                             h-%offset%)
              (* v2
                 (f2cl-lib:fref h-%data%
                                ((f2cl-lib:int-add k 1) j)
                                ((1 ldh) (1 *)) h-%offset%))))
     (setf (f2cl-lib:fref h-%data% (k j) ((1 ldh) (1 *))
                          h-%offset%)
             (-
              (f2cl-lib:fref h-%data% (k j) ((1 ldh) (1 *))
                             h-%offset%)
              (* sum t1)))
     (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add k 1) j)
                          ((1 ldh) (1 *)) h-%offset%)
             (-
              (f2cl-lib:fref h-%data%
                             ((f2cl-lib:int-add k 1) j)
                             ((1 ldh) (1 *)) h-%offset%)
              (* sum t2)))
    label100))
 (f2cl-lib:fdo (j i1 (f2cl-lib:int-add j 1))
               ((> j i) nil)
   (tagbody
     (setf sum
             (+
              (f2cl-lib:fref h-%data% (j k) ((1 ldh) (1 *))
                             h-%offset%)
              (* v2
                 (f2cl-lib:fref h-%data%
                                (j (f2cl-lib:int-add k 1))
                                ((1 ldh) (1 *)) h-%offset%))))
     (setf (f2cl-lib:fref h-%data% (j k) ((1 ldh) (1 *))
                          h-%offset%)
             (-
              (f2cl-lib:fref h-%data% (j k) ((1 ldh) (1 *))
                             h-%offset%)
              (* sum t1)))
     (setf (f2cl-lib:fref h-%data% (j (f2cl-lib:int-add k 1))
```

```
                                        ((1 ldh) (1 *)) h-%offset%)
                                (-
                                 (f2cl-lib:fref h-%data%
                                                (j (f2cl-lib:int-add k 1))
                                                ((1 ldh) (1 *)) h-%offset%)
                                 (* sum t2)))
                        label110))
                     (cond
                      (wantz
                       (f2cl-lib:fdo (j iloz (f2cl-lib:int-add j 1))
                                     ((> j ihiz) nil)
                         (tagbody
                           (setf sum
                                   (+
                                    (f2cl-lib:fref z-%data% (j k) ((1 ldz) (1 *))
                                                   z-%offset%)
                                    (* v2
                                       (f2cl-lib:fref z-%data%
                                                      (j (f2cl-lib:int-add k 1))
                                                      ((1 ldz) (1 *))
                                                      z-%offset%))))
                           (setf (f2cl-lib:fref z-%data% (j k) ((1 ldz) (1 *))
                                                z-%offset%)
                                   (-
                                    (f2cl-lib:fref z-%data% (j k) ((1 ldz) (1 *))
                                                   z-%offset%)
                                    (* sum t1)))
                           (setf (f2cl-lib:fref z-%data%
                                                (j (f2cl-lib:int-add k 1))
                                                ((1 ldz) (1 *)) z-%offset%)
                                   (-
                                    (f2cl-lib:fref z-%data%
                                                   (j (f2cl-lib:int-add k 1))
                                                   ((1 ldz) (1 *)) z-%offset%)
                                    (* sum t2)))
                         label120))))))
                 label130))
             label140))
          (setf info i)
          (go end_label)
        label150
          (cond
           ((= l i)
            (setf (f2cl-lib:fref wr-%data% (i) ((1 *)) wr-%offset%)
                  (f2cl-lib:fref h-%data% (i i) ((1 ldh) (1 *)) h-%offset%))
            (setf (f2cl-lib:fref wi-%data% (i) ((1 *)) wi-%offset%) zero))
           ((= l (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
                (dlanv2
```

```
      (f2cl-lib:array-slice h-%data% double-float
                            ((+ i (f2cl-lib:int-sub 1))
                             (f2cl-lib:int-sub i 1))
                            ((1 ldh) (1 *)) h-%offset%)
      (f2cl-lib:array-slice h-%data% double-float
                            ((+ i (f2cl-lib:int-sub 1)) i)
                            ((1 ldh) (1 *)) h-%offset%)
      (f2cl-lib:array-slice h-%data% double-float
                            (i (f2cl-lib:int-sub i 1)) ((1 ldh) (1 *))
                            h-%offset%)
      (f2cl-lib:array-slice h-%data% double-float (i i)
                            ((1 ldh) (1 *)) h-%offset%)
      (f2cl-lib:array-slice wr-%data% double-float
                            ((+ i (f2cl-lib:int-sub 1))) ((1 *))
                            wr-%offset%)
      (f2cl-lib:array-slice wi-%data% double-float
                            ((+ i (f2cl-lib:int-sub 1))) ((1 *))
                            wi-%offset%)
      (f2cl-lib:array-slice wr-%data% double-float (i) ((1 *))
                            wr-%offset%)
      (f2cl-lib:array-slice wi-%data% double-float (i) ((1 *))
                            wi-%offset%)
      cs sn)
  (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7))
  (when var-8 (setf cs var-8))
  (when var-9 (setf sn var-9)))
(cond
 (wantt
  (if (> i2 i)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (drot (f2cl-lib:int-sub i2 i)
           (f2cl-lib:array-slice h-%data% double-float
                                 ((+ i (f2cl-lib:int-sub 1))
                                  (f2cl-lib:int-add i 1))
                                 ((1 ldh) (1 *)) h-%offset%)
           ldh
           (f2cl-lib:array-slice h-%data% double-float
                                 (i (f2cl-lib:int-add i 1))
                                 ((1 ldh) (1 *)) h-%offset%)
           ldh cs sn)
        (declare (ignore var-0 var-1 var-3))
        (when var-2 (setf ldh var-2))
        (when var-4 (setf ldh var-4))
        (when var-5 (setf cs var-5))
        (when var-6 (setf sn var-6))))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (drot (f2cl-lib:int-sub i i1 1)
       (f2cl-lib:array-slice h-%data% double-float
                             (i1 (f2cl-lib:int-sub i 1))
```

```
                                  ((1 ldh) (1 *)) h-%offset%)
                1
                (f2cl-lib:array-slice h-%data% double-float (i1 i)
                                  ((1 ldh) (1 *)) h-%offset%)
                1 cs sn)
            (declare (ignore var-0 var-1 var-2 var-3 var-4))
            (when var-5 (setf cs var-5))
            (when var-6 (setf sn var-6)))))
        (cond
         (wantz
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (drot nz
                (f2cl-lib:array-slice z-%data% double-float
                                  (iloz (f2cl-lib:int-sub i 1))
                                  ((1 ldz) (1 *)) z-%offset%)
                1
                (f2cl-lib:array-slice z-%data% double-float (iloz i)
                                  ((1 ldz) (1 *)) z-%offset%)
                1 cs sn)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf nz var-0))
            (when var-5 (setf cs var-5))
            (when var-6 (setf sn var-6)))))))
      (setf i (f2cl-lib:int-sub l 1))
      (go label20)
     label160
      (go end_label)
     end_label
      (return
       (values nil nil nil nil nil nil ldh nil nil nil nil nil nil info))))))
```

———-

90.144 dlahrd DEPRECATED see dlahr2

90.144.1 Axiom unit tests

— dlahrd.input —

```
)set break resume
)sys rm -f dlahrd.output
)spool dlahrd.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

90.144.2 Axiom help page

— **dlahrd.help** —

```
==================================================================
dlahrd examples
==================================================================

==================================================================
Man Page Details
==================================================================

NAME
       DLAHRD  -  the first NB columns of a real general n-by-(n-k+1) matrix A
       so that elements below the k-th subdiagonal are zero

SYNOPSIS
       SUBROUTINE DLAHRD( N, K, NB, A, LDA, TAU, T, LDT, Y, LDY )

           INTEGER        K, LDA, LDT, LDY, N, NB

           DOUBLE         PRECISION A( LDA, * ), T( LDT, NB ), TAU( NB  ),  Y(
                          LDY, NB )

PURPOSE
       DLAHRD  reduces  the  first  NB  columns of a real general n-by-(n-k+1)
       matrix A so that elements below the  k-th  subdiagonal  are  zero.  The
       reduction  is performed by an orthogonal similarity transformation Q' *
       A * Q. The routine returns the matrices V and T which determine Q as  a
       block reflector I - V*T*V', and also the matrix Y = A * V * T.

       This is an OBSOLETE auxiliary routine.
       This routine will be 'deprecated' in a  future release.
       Please use the new routine DLAHR2 instead.


ARGUMENTS
       N       (input) INTEGER
               The order of the matrix A.

       K       (input) INTEGER
               The offset for the reduction. Elements below the k-th subdiago-
               nal in the first NB columns are reduced to zero.
```

```
NB      (input) INTEGER
        The number of columns to be reduced.

A       (input/output) DOUBLE PRECISION array, dimension (LDA,N-K+1)
        On entry, the n-by-(n-k+1) general matrix A.  On exit, the ele-
        ments on and above the k-th subdiagonal in the first NB columns
        are overwritten with the corresponding elements of the  reduced
        matrix; the elements below the k-th subdiagonal, with the array
        TAU, represent the matrix Q as a product of elementary  reflec-
        tors.  The  other  columns  of  A  are  unchanged.  See Further
        Details.  LDA     (input) INTEGER The leading dimension of  the
        array A.  LDA >= max(1,N).

TAU     (output) DOUBLE PRECISION array, dimension (NB)
        The  scalar  factors  of the elementary reflectors. See Further
        Details.

T       (output) DOUBLE PRECISION array, dimension (LDT,NB)
        The upper triangular matrix T.

LDT     (input) INTEGER
        The leading dimension of the array T.  LDT >= NB.

Y       (output) DOUBLE PRECISION array, dimension (LDY,NB)
        The n-by-nb matrix Y.

LDY     (input) INTEGER
        The leading dimension of the array Y. LDY >= N.

FURTHER DETAILS
       The matrix Q is represented as a product of nb elementary reflectors

          Q = H(1) H(2) . . . H(nb).

       Each H(i) has the form

          H(i) = I - tau * v * v'

       where tau is a real scalar, and v is a real vector with
       v(1:i+k-1)  =  0,  v(i+k)  =  1;  v(i+k+1:n)  is  stored  on  exit   in
       A(i+k+1:n,i), and tau in TAU(i).

       The  elements of the vectors v together form the (n-k+1)-by-nb matrix V
       which is needed, with T and Y, to apply the transformation to the unre-
       duced  part  of  the  matrix,  using  an  update of the form: A := (I -
       V*T*V') * (A - Y*V').

       The contents of A on exit are illustrated by the following example with
       n = 7, k = 3 and nb = 2:
```

```
( a   h   a   a   a )
( a   h   a   a   a )
( a   h   a   a   a )
( h   h   a   a   a )
( v1  h   a   a   a )
( v1  v2  a   a   a )
( v1  v2  a   a   a )
```

where a denotes an element of the original matrix A, h denotes a modified element of the upper Hessenberg matrix H, and vi denotes an element of the vector defining H(i).

———-

90.144.3 fortran code

— dlahrd.f —

```
      SUBROUTINE DLAHRD( N, K, NB, A, LDA, TAU, T, LDT, Y, LDY )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      INTEGER            K, LDA, LDT, LDY, N, NB
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), T( LDT, NB ), TAU( NB ),
     $                   Y( LDY, NB )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      DOUBLE PRECISION   EI
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DCOPY, DGEMV, DLARFG, DSCAL, DTRMV
*     ..
```

```
*     .. Intrinsic Functions ..
      INTRINSIC          MIN
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( N.LE.1 )
     $   RETURN
*
      DO 10 I = 1, NB
         IF( I.GT.1 ) THEN
*
*           Update A(1:n,i)
*
*           Compute i-th column of A - Y * V'
*
            CALL DGEMV( 'No transpose', N, I-1, -ONE, Y, LDY,
     $                  A( K+I-1, 1 ), LDA, ONE, A( 1, I ), 1 )
*
*           Apply I - V * T' * V' to this column (call it b) from the
*           left, using the last column of T as workspace
*
*           Let  V = ( V1 )   and   b = ( b1 )   (first I-1 rows)
*                    ( V2 )             ( b2 )
*
*           where V1 is unit lower triangular
*
*           w := V1' * b1
*
            CALL DCOPY( I-1, A( K+1, I ), 1, T( 1, NB ), 1 )
            CALL DTRMV( 'Lower', 'Transpose', 'Unit', I-1, A( K+1, 1 ),
     $                  LDA, T( 1, NB ), 1 )
*
*           w := w + V2'*b2
*
            CALL DGEMV( 'Transpose', N-K-I+1, I-1, ONE, A( K+I, 1 ),
     $                  LDA, A( K+I, I ), 1, ONE, T( 1, NB ), 1 )
*
*           w := T'*w
*
            CALL DTRMV( 'Upper', 'Transpose', 'Non-unit', I-1, T, LDT,
     $                  T( 1, NB ), 1 )
*
*           b2 := b2 - V2*w
*
            CALL DGEMV( 'No transpose', N-K-I+1, I-1, -ONE, A( K+I, 1 ),
     $                  LDA, T( 1, NB ), 1, ONE, A( K+I, I ), 1 )
*
*           b1 := b1 - V1*w
```

```
*
            CALL DTRMV( 'Lower', 'No transpose', 'Unit', I-1,
     $                  A( K+1, 1 ), LDA, T( 1, NB ), 1 )
            CALL DAXPY( I-1, -ONE, T( 1, NB ), 1, A( K+1, I ), 1 )
*
            A( K+I-1, I-1 ) = EI
         END IF
*
*        Generate the elementary reflector H(i) to annihilate
*        A(k+i+1:n,i)
*
         CALL DLARFG( N-K-I+1, A( K+I, I ), A( MIN( K+I+1, N ), I ), 1,
     $                TAU( I ) )
         EI = A( K+I, I )
         A( K+I, I ) = ONE
*
*        Compute  Y(1:n,i)
*
         CALL DGEMV( 'No transpose', N, N-K-I+1, ONE, A( 1, I+1 ), LDA,
     $               A( K+I, I ), 1, ZERO, Y( 1, I ), 1 )
         CALL DGEMV( 'Transpose', N-K-I+1, I-1, ONE, A( K+I, 1 ), LDA,
     $               A( K+I, I ), 1, ZERO, T( 1, I ), 1 )
         CALL DGEMV( 'No transpose', N, I-1, -ONE, Y, LDY, T( 1, I ), 1,
     $               ONE, Y( 1, I ), 1 )
         CALL DSCAL( N, TAU( I ), Y( 1, I ), 1 )
*
*        Compute T(1:i,i)
*
         CALL DSCAL( I-1, -TAU( I ), T( 1, I ), 1 )
         CALL DTRMV( 'Upper', 'No transpose', 'Non-unit', I-1, T, LDT,
     $               T( 1, I ), 1 )
         T( I, I ) = TAU( I )
*
   10 CONTINUE
      A( K+NB, NB ) = EI
*
      RETURN
*
*     End of DLAHRD
*
      END
```

———-

90.144.4 lisp code

— dlahrd.lisp —

```
(let* ((zero 0.0) (one 1.0))
  (declare (type (double-float 0.0 0.0) zero)
           (type (double-float 1.0 1.0) one))
  (defun dlahrd (n k nb a lda tau t$ ldt y ldy)
    (declare (type (simple-array double-float (*)) y t$ tau a)
             (type fixnum ldy ldt lda nb k n))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (t$ double-float t$-%data% t$-%offset%)
         (y double-float y-%data% y-%offset%))
      (prog ((ei 0.0) (i 0))
        (declare (type (double-float) ei) (type fixnum i))
        (if (<= n 1) (go end_label))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i nb) nil)
          (tagbody
            (cond
              ((> i 1)
               (dgemv "No transpose" n (f2cl-lib:int-sub i 1) (- one) y ldy
                (f2cl-lib:array-slice a
                                      double-float
                                      ((+ k i (f2cl-lib:int-sub 1)) 1)
                                      ((1 lda) (1 *)))
                lda one
                (f2cl-lib:array-slice a double-float (1 i) ((1 lda) (1 *))) 1)
               (dcopy (f2cl-lib:int-sub i 1)
                (f2cl-lib:array-slice a
                                      double-float
                                      ((+ k 1) i)
                                      ((1 lda) (1 *)))
                1
                (f2cl-lib:array-slice t$ double-float (1 nb) ((1 ldt) (1 nb)))
                1)
               (dtrmv "Lower" "Transpose" "Unit" (f2cl-lib:int-sub i 1)
                (f2cl-lib:array-slice a
                                      double-float
                                      ((+ k 1) 1)
                                      ((1 lda) (1 *)))
                lda
                (f2cl-lib:array-slice t$ double-float (1 nb) ((1 ldt) (1 nb)))
                1)
               (dgemv "Transpose" (f2cl-lib:int-add (f2cl-lib:int-sub n k i) 1)
                (f2cl-lib:int-sub i 1) one
                (f2cl-lib:array-slice a
                                      double-float
                                      ((+ k i) 1)
                                      ((1 lda) (1 *)))
                lda
                (f2cl-lib:array-slice a
```

```
                          double-float
                          ((+ k i) i)
                          ((1 lda) (1 *)))
     1 one
     (f2cl-lib:array-slice t$ double-float (1 nb) ((1 ldt) (1 nb)))
     1)
    (dtrmv "Upper" "Transpose" "Non-unit" (f2cl-lib:int-sub i 1) t$
     ldt
     (f2cl-lib:array-slice t$ double-float (1 nb) ((1 ldt) (1 nb)))
     1)
    (dgemv "No transpose"
     (f2cl-lib:int-add (f2cl-lib:int-sub n k i) 1)
     (f2cl-lib:int-sub i 1) (- one)
     (f2cl-lib:array-slice a
                          double-float
                          ((+ k i) 1)
                          ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice t$ double-float (1 nb) ((1 ldt) (1 nb)))
     1 one
     (f2cl-lib:array-slice a
                          double-float
                          ((+ k i) i)
                          ((1 lda) (1 *)))
     1)
    (dtrmv "Lower" "No transpose" "Unit" (f2cl-lib:int-sub i 1)
     (f2cl-lib:array-slice a
                          double-float
                          ((+ k 1) 1)
                          ((1 lda) (1 *)))
     lda
     (f2cl-lib:array-slice t$ double-float (1 nb) ((1 ldt) (1 nb)))
     1)
    (daxpy (f2cl-lib:int-sub i 1) (- one)
     (f2cl-lib:array-slice t$ double-float (1 nb) ((1 ldt) (1 nb)))
     1
     (f2cl-lib:array-slice a
                          double-float
                          ((+ k 1) i)
                          ((1 lda) (1 *)))
     1)
    (setf (f2cl-lib:fref a-%data%
                         ((f2cl-lib:int-sub (f2cl-lib:int-add k i)
                                            1)
                          (f2cl-lib:int-sub i 1))
                         ((1 lda) (1 *))
                         a-%offset%)
          ei)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dlarfg (f2cl-lib:int-add (f2cl-lib:int-sub n k i) 1)
```

```
     (f2cl-lib:fref a-%data%
                    ((f2cl-lib:int-add k i) i)
                    ((1 lda) (1 *))
                    a-%offset%)
     (f2cl-lib:array-slice a
                           double-float
                           ((min (f2cl-lib:int-add k i 1) n) i)
                           ((1 lda) (1 *)))
     1 (f2cl-lib:fref tau-%data% (i) ((1 nb)) tau-%offset%))
  (declare (ignore var-0 var-2 var-3))
  (setf (f2cl-lib:fref a-%data%
                       ((f2cl-lib:int-add k i) i)
                       ((1 lda) (1 *))
                       a-%offset%)
          var-1)
  (setf (f2cl-lib:fref tau-%data% (i) ((1 nb)) tau-%offset%)
          var-4))
(setf ei
        (f2cl-lib:fref a-%data%
                       ((f2cl-lib:int-add k i) i)
                       ((1 lda) (1 *))
                       a-%offset%))
(setf (f2cl-lib:fref a-%data%
                     ((f2cl-lib:int-add k i) i)
                     ((1 lda) (1 *))
                     a-%offset%)
        one)
(dgemv "No transpose" n
 (f2cl-lib:int-add (f2cl-lib:int-sub n k i) 1) one
 (f2cl-lib:array-slice a
                       double-float
                       (1 (f2cl-lib:int-add i 1))
                       ((1 lda) (1 *)))
 lda
 (f2cl-lib:array-slice a double-float ((+ k i) i) ((1 lda) (1 *)))
 1 zero
 (f2cl-lib:array-slice y double-float (1 i) ((1 ldy) (1 nb))) 1)
(dgemv "Transpose" (f2cl-lib:int-add (f2cl-lib:int-sub n k i) 1)
 (f2cl-lib:int-sub i 1) one
 (f2cl-lib:array-slice a double-float ((+ k i) 1) ((1 lda) (1 *)))
 lda
 (f2cl-lib:array-slice a double-float ((+ k i) i) ((1 lda) (1 *)))
 1 zero
 (f2cl-lib:array-slice t$ double-float (1 i) ((1 ldt) (1 nb))) 1)
(dgemv "No transpose" n (f2cl-lib:int-sub i 1) (- one) y ldy
 (f2cl-lib:array-slice t$ double-float (1 i) ((1 ldt) (1 nb))) 1
 one (f2cl-lib:array-slice y double-float (1 i) ((1 ldy) (1 nb)))
 1)
(dscal n (f2cl-lib:fref tau-%data% (i) ((1 nb)) tau-%offset%)
 (f2cl-lib:array-slice y double-float (1 i) ((1 ldy) (1 nb))) 1)
```

```
            (dscal (f2cl-lib:int-sub i 1)
             (- (f2cl-lib:fref tau-%data% (i) ((1 nb)) tau-%offset%))
             (f2cl-lib:array-slice t$ double-float (1 i) ((1 ldt) (1 nb))) 1)
            (dtrmv "Upper" "No transpose" "Non-unit" (f2cl-lib:int-sub i 1) t$
             ldt (f2cl-lib:array-slice t$ double-float (1 i) ((1 ldt) (1 nb)))
             1)
            (setf (f2cl-lib:fref t$-%data% (i i) ((1 ldt) (1 nb)) t$-%offset%)
                    (f2cl-lib:fref tau-%data% (i) ((1 nb)) tau-%offset%))))
        (setf (f2cl-lib:fref a-%data%
                             ((f2cl-lib:int-add k nb) nb)
                             ((1 lda) (1 *))
                             a-%offset%)
                ei)
 end_label
        (return (values nil nil nil nil nil nil nil nil nil nil))))))
```

———-

90.145 dlaisnan

90.145.1 Axiom unit tests

— **dlaisnan.input** —

```
)set break resume
)sys rm -f dlaisnan.output
)spool dlaisnan.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.145.2 Axiom help page

— **dlaisnan.help** —

```
========================================================================
dlaisnan examples
========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

      LOGICAL FUNCTION DLAISNAN( DIN1, DIN2 )

      .. Scalar Arguments ..
      DOUBLE PRECISION   DIN1, DIN2
      ..


 Purpose:
 =============

 This routine is not for general use.  It exists solely to avoid
 over-optimization in DISNAN.

 DLAISNAN checks for NaNs by comparing its two arguments for
 inequality.  NaN is the only floating-point value where NaN != NaN
 returns .TRUE.  To check for NaNs, pass the same variable as both
 arguments.

 A compiler must assume that the two arguments are
 not the same variable, and the test will not be optimized away.
 Interprocedural or whole-program optimization may delete this
 test.  The ISNAN functions will be replaced by the correct
 Fortran 03 intrinsic once the intrinsic is widely available.

  Arguments:
  ==========

  [in] DIN1
          DIN1 is DOUBLE PRECISION

  [in] DIN2
          DIN2 is DOUBLE PRECISION
          Two numbers to compare for inequality.


            ———————-
```

90.145.3 fortran code

— dlaisnan.f —

```
*  =====================================================================
      LOGICAL FUNCTION DLAISNAN( DIN1, DIN2 )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   DIN1, DIN2
*     ..
*
*  =====================================================================
*
*  .. Executable Statements ..
      DLAISNAN = (DIN1.NE.DIN2)
      RETURN
      END
```

———-

90.145.4 lisp code

— dlaisnan.lisp —

```
(defun dlaisnan (din1 din2)
  (declare (type (double-float) din2 din1))
  (prog ((dlaisnan nil))
    (declare (type f2cl-lib:logical dlaisnan))
    (setf dlaisnan (coerce (/= din1 din2) 'f2cl-lib:logical))
    (go end_label)
   end_label
    (return (values dlaisnan nil nil))))
```

———-

90.146 dlaln2

90.146.1 Axiom unit tests

— dlaln2.input —

```
)set break resume
)sys rm -f dlaln2.output
)spool dlaln2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.146.2 Axiom help page

— dlaln2.help —

```
==================================================================
dlaln2 examples
==================================================================

==================================================================
Man Page Details
==================================================================

NAME
       DLALN2  - a system of the form (ca A - w D ) X = s B or (ca A' - w D) X
       = s B with possible scaling ("s") and perturbation of A

SYNOPSIS
       SUBROUTINE DLALN2( LTRANS, NA, NW, SMIN, CA, A, LDA, D1,  D2,  B,  LDB,
                          WR, WI, X, LDX, SCALE, XNORM, INFO )

           LOGICAL        LTRANS

           INTEGER        INFO, LDA, LDB, LDX, NA, NW

           DOUBLE         PRECISION CA, D1, D2, SCALE, SMIN, WI, WR, XNORM

           DOUBLE         PRECISION A( LDA, * ), B( LDB, * ), X( LDX, * )
```

```
Purpose
=======

 DLALN2 solves a system of the form  (ca A - w D ) X = s B
 or (ca A' - w D) X = s B   with possible scaling ("s") and
 perturbation of A.  (A' means A-transpose.)

 A is an NA x NA real matrix, ca is a real scalar, D is an NA x NA
 real diagonal matrix, w is a real or complex value, and X and B are
 NA x 1 matrices -- real if w is real, complex if w is complex.  NA
 may be 1 or 2.

 If w is complex, X and B are represented as NA x 2 matrices,
 the first column of each being the real part and the second
 being the imaginary part.

 "s" is a scaling factor (.LE. 1), computed by DLALN2, which is
 so chosen that X can be computed without overflow.  X is further
 scaled if necessary to assure that norm(ca A - w D)*norm(X) is less
 than overflow.

 If both singular values of (ca A - w D) are less than SMIN,
 SMIN*identity will be used instead of (ca A - w D).  If only one
 singular value is less than SMIN, one element of (ca A - w D) will be
 perturbed enough to make the smallest singular value roughly SMIN.
 If both singular values are at least SMIN, (ca A - w D) will not be
 perturbed.  In any case, the perturbation will be at most some small
 multiple of max( SMIN, ulp*norm(ca A - w D) ).  The singular values
 are computed by infinity-norm approximations, and thus will only be
 correct to a factor of 2 or so.

 Note: all input quantities are assumed to be smaller than overflow
 by a reasonable factor.  (See BIGNUM.)

Arguments
==========

 LTRANS  (input) LOGICAL
         =.TRUE.:  A-transpose will be used.
         =.FALSE.: A will be used (not transposed.)

 NA      (input) INTEGER
         The size of the matrix A.  It may (only) be 1 or 2.

 NW      (input) INTEGER
         1 if "w" is real, 2 if "w" is complex.  It may only be 1
         or 2.

 SMIN    (input) DOUBLE PRECISION
         The desired lower bound on the singular values of A.  This
```

```
        should be a safe distance away from underflow or overflow,
        say, between (underflow/machine precision) and  (machine
        precision * overflow ).  (See BIGNUM and ULP.)

CA      (input) DOUBLE PRECISION
        The coefficient c, which A is multiplied by.

A       (input) DOUBLE PRECISION array, dimension (LDA,NA)
        The NA x NA matrix A.

LDA     (input) INTEGER
        The leading dimension of A.  It must be at least NA.

D1      (input) DOUBLE PRECISION
        The 1,1 element in the diagonal matrix D.

D2      (input) DOUBLE PRECISION
        The 2,2 element in the diagonal matrix D.  Not used if NW=1.

B       (input) DOUBLE PRECISION array, dimension (LDB,NW)
        The NA x NW matrix B (right-hand side).  If NW=2 ("w" is
        complex), column 1 contains the real part of B and column 2
        contains the imaginary part.

LDB     (input) INTEGER
        The leading dimension of B.  It must be at least NA.

WR      (input) DOUBLE PRECISION
        The real part of the scalar "w".

WI      (input) DOUBLE PRECISION
        The imaginary part of the scalar "w".  Not used if NW=1.

X       (output) DOUBLE PRECISION array, dimension (LDX,NW)
        The NA x NW matrix X (unknowns), as computed by DLALN2.
        If NW=2 ("w" is complex), on exit, column 1 will contain
        the real part of X and column 2 will contain the imaginary
        part.

LDX     (input) INTEGER
        The leading dimension of X.  It must be at least NA.

SCALE   (output) DOUBLE PRECISION
        The scale factor that B must be multiplied by to insure
        that overflow does not occur when computing X.  Thus,
        (ca A - w D) X  will be SCALE*B, not B (ignoring
        perturbations of A.)  It will be at most 1.

XNORM   (output) DOUBLE PRECISION
        The infinity-norm of X, when X is regarded as an NA x NW
```

```
         real matrix.

  INFO    (output) INTEGER
         An error flag.  It will be set to zero if no error occurs,
         a negative number if an argument is in error, or a positive
         number if  ca A - w D  had to be perturbed.
         The possible values are:
         = 0: No error occurred, and (ca A - w D) did not have to be
                perturbed.
         = 1: (ca A - w D) had to be perturbed to make its smallest
              (or only) singular value greater than SMIN.
         NOTE: In the interests of speed, this routine does not
               check the inputs for errors.
```

———-

90.146.3 fortran code

— dlaln2.f —

```
      SUBROUTINE DLALN2( LTRANS, NA, NW, SMIN, CA, A, LDA, D1, D2, B,
     $                   LDB, WR, WI, X, LDX, SCALE, XNORM, INFO )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1992
*
*     .. Scalar Arguments ..
      LOGICAL            LTRANS
      INTEGER            INFO, LDA, LDB, LDX, NA, NW
      DOUBLE PRECISION   CA, D1, D2, SCALE, SMIN, WI, WR, XNORM
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), B( LDB, * ), X( LDX, * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D0 )
*     ..
*     .. Local Scalars ..
      INTEGER            ICMAX, J
```

```
      DOUBLE PRECISION   BBND, BI1, BI2, BIGNUM, BNORM, BR1, BR2, CI21,
     $                   CI22, CMAX, CNORM, CR21, CR22, CSI, CSR, LI21,
     $                   LR21, SMINI, SMLNUM, TEMP, U22ABS, UI11, UI11R,
     $                   UI12, UI12S, UI22, UR11, UR11R, UR12, UR12S,
     $                   UR22, XI1, XI2, XR1, XR2
*     ..
*     .. Local Arrays ..
      LOGICAL            RSWAP( 4 ), ZSWAP( 4 )
      INTEGER            IPIVOT( 4, 4 )
      DOUBLE PRECISION   CI( 2, 2 ), CIV( 4 ), CR( 2, 2 ), CRV( 4 )
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLADIV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Equivalences ..
c
c     *** F2CL cannot currently handle equivalences of arrays
c     *** So we do this by hand.  Since Fortran arrays are column-major
c     *** order, we have the following:
c     ***
c     *** ci(1,1) civ(1)
c     *** ci(2,1) civ(2)
c     *** ci(1,2) civ(3)
c     *** ci(2,2) civ(4)
c     ***
c     *** Similarly for CR.
c      EQUIVALENCE        ( CI( 1, 1 ), CIV( 1 ) ),
c     $                   ( CR( 1, 1 ), CRV( 1 ) )
*     ..
*     .. Data statements ..
      DATA               ZSWAP / .FALSE., .FALSE., .TRUE., .TRUE. /
      DATA               RSWAP / .FALSE., .TRUE., .FALSE., .TRUE. /
      DATA               IPIVOT / 1, 2, 3, 4, 2, 1, 4, 3, 3, 4, 1, 2, 4,
     $                   3, 2, 1 /
*     ..
*     .. Executable Statements ..
*
*     Compute BIGNUM
*
      SMLNUM = TWO*DLAMCH( 'Safe minimum' )
      BIGNUM = ONE / SMLNUM
      SMINI = MAX( SMIN, SMLNUM )
*
```

```
*     Don't check for input errors
*
      INFO = 0
*
*     Standard Initializations
*
      SCALE = ONE
*
      IF( NA.EQ.1 ) THEN
*
*        1 x 1  (i.e., scalar) system   C X = B
*
         IF( NW.EQ.1 ) THEN
*
*           Real 1x1 system.
*
*           C = ca A - w D
*
            CSR = CA*A( 1, 1 ) - WR*D1
            CNORM = ABS( CSR )
*
*           If | C | < SMINI, use C = SMINI
*
            IF( CNORM.LT.SMINI ) THEN
               CSR = SMINI
               CNORM = SMINI
               INFO = 1
            END IF
*
*           Check scaling for  X = B / C
*
            BNORM = ABS( B( 1, 1 ) )
            IF( CNORM.LT.ONE .AND. BNORM.GT.ONE ) THEN
               IF( BNORM.GT.BIGNUM*CNORM )
     $            SCALE = ONE / BNORM
            END IF
*
*           Compute X
*
            X( 1, 1 ) = ( B( 1, 1 )*SCALE ) / CSR
            XNORM = ABS( X( 1, 1 ) )
         ELSE
*
*           Complex 1x1 system (w is complex)
*
*           C = ca A - w D
*
            CSR = CA*A( 1, 1 ) - WR*D1
            CSI = -WI*D1
            CNORM = ABS( CSR ) + ABS( CSI )
```

```
*
*          If | C | < SMINI, use C = SMINI
*
           IF( CNORM.LT.SMINI ) THEN
              CSR = SMINI
              CSI = ZERO
              CNORM = SMINI
              INFO = 1
           END IF
*
*          Check scaling for  X = B / C
*
           BNORM = ABS( B( 1, 1 ) ) + ABS( B( 1, 2 ) )
           IF( CNORM.LT.ONE .AND. BNORM.GT.ONE ) THEN
              IF( BNORM.GT.BIGNUM*CNORM )
     $           SCALE = ONE / BNORM
           END IF
*
*          Compute X
*
           CALL DLADIV( SCALE*B( 1, 1 ), SCALE*B( 1, 2 ), CSR, CSI,
     $                  X( 1, 1 ), X( 1, 2 ) )
           XNORM = ABS( X( 1, 1 ) ) + ABS( X( 1, 2 ) )
        END IF
*
      ELSE
*
*       2x2 System
*
*       Compute the real part of  C = ca A - w D  (or  ca A' - w D )
*
c       *** F2CL original
c        CR( 1, 1 ) = CA*A( 1, 1 ) - WR*D1
c        CR( 2, 2 ) = CA*A( 2, 2 ) - WR*D2
c       *** F2CL replacement
        crv(1)  = CA*A( 1, 1 ) - WR*D1
        crv(4)  = CA*A( 2, 2 ) - WR*D2
        IF( LTRANS ) THEN
c           CR( 1, 2 ) = CA*A( 2, 1 )
c           CR( 2, 1 ) = CA*A( 1, 2 )
           crv( 3 ) = CA*A( 2, 1 )
           crv( 2 ) = CA*A( 1, 2 )
        ELSE
c           CR( 2, 1 ) = CA*A( 2, 1 )
c           CR( 1, 2 ) = CA*A( 1, 2 )
           crv( 2 ) = CA*A( 2, 1 )
           crv( 3 ) = CA*A( 1, 2 )
        END IF
*
        IF( NW.EQ.1 ) THEN
```

```
*
*          Real 2x2 system  (w is real)
*
*          Find the largest element in C
*
           CMAX = ZERO
           ICMAX = 0
*
           DO 10 J = 1, 4
              IF( ABS( CRV( J ) ).GT.CMAX ) THEN
                 CMAX = ABS( CRV( J ) )
                 ICMAX = J
              END IF
   10      CONTINUE
*
*          If norm(C) < SMINI, use SMINI*identity.
*
           IF( CMAX.LT.SMINI ) THEN
              BNORM = MAX( ABS( B( 1, 1 ) ), ABS( B( 2, 1 ) ) )
              IF( SMINI.LT.ONE .AND. BNORM.GT.ONE ) THEN
                 IF( BNORM.GT.BIGNUM*SMINI )
     $              SCALE = ONE / BNORM
              END IF
              TEMP = SCALE / SMINI
              X( 1, 1 ) = TEMP*B( 1, 1 )
              X( 2, 1 ) = TEMP*B( 2, 1 )
              XNORM = TEMP*BNORM
              INFO = 1
              RETURN
           END IF
*
*          Gaussian elimination with complete pivoting.
*
           UR11 = CRV( ICMAX )
           CR21 = CRV( IPIVOT( 2, ICMAX ) )
           UR12 = CRV( IPIVOT( 3, ICMAX ) )
           CR22 = CRV( IPIVOT( 4, ICMAX ) )
           UR11R = ONE / UR11
           LR21 = UR11R*CR21
           UR22 = CR22 - UR12*LR21
*
*          If smaller pivot < SMINI, use SMINI
*
           IF( ABS( UR22 ).LT.SMINI ) THEN
              UR22 = SMINI
              INFO = 1
           END IF
           IF( RSWAP( ICMAX ) ) THEN
              BR1 = B( 2, 1 )
              BR2 = B( 1, 1 )
```

```
            ELSE
               BR1 = B( 1, 1 )
               BR2 = B( 2, 1 )
            END IF
            BR2 = BR2 - LR21*BR1
            BBND = MAX( ABS( BR1*( UR22*UR11R ) ), ABS( BR2 ) )
            IF( BBND.GT.ONE .AND. ABS( UR22 ).LT.ONE ) THEN
               IF( BBND.GE.BIGNUM*ABS( UR22 ) )
     $            SCALE = ONE / BBND
            END IF
*
            XR2 = ( BR2*SCALE ) / UR22
            XR1 = ( SCALE*BR1 )*UR11R - XR2*( UR11R*UR12 )
            IF( ZSWAP( ICMAX ) ) THEN
               X( 1, 1 ) = XR2
               X( 2, 1 ) = XR1
            ELSE
               X( 1, 1 ) = XR1
               X( 2, 1 ) = XR2
            END IF
            XNORM = MAX( ABS( XR1 ), ABS( XR2 ) )
*
*           Further scaling if  norm(A) norm(X) > overflow
*
            IF( XNORM.GT.ONE .AND. CMAX.GT.ONE ) THEN
               IF( XNORM.GT.BIGNUM / CMAX ) THEN
                  TEMP = CMAX / BIGNUM
                  X( 1, 1 ) = TEMP*X( 1, 1 )
                  X( 2, 1 ) = TEMP*X( 2, 1 )
                  XNORM = TEMP*XNORM
                  SCALE = TEMP*SCALE
               END IF
            END IF
         ELSE
*
*           Complex 2x2 system  (w is complex)
*
*           Find the largest element in C
*
c           *** F2CL original
c            CI( 1, 1 ) = -WI*D1
c            CI( 2, 1 ) = ZERO
c            CI( 1, 2 ) = ZERO
c            CI( 2, 2 ) = -WI*D2
            civ( 1 ) = -WI*D1
            civ( 2 ) = ZERO
            civ( 3 ) = ZERO
            civ( 4 ) = -WI*D2
            CMAX = ZERO
            ICMAX = 0
```

```
*
            DO 20 J = 1, 4
               IF( ABS( CRV( J ) )+ABS( CIV( J ) ).GT.CMAX ) THEN
                  CMAX = ABS( CRV( J ) ) + ABS( CIV( J ) )
                  ICMAX = J
               END IF
   20       CONTINUE
*
*           If norm(C) < SMINI, use SMINI*identity.
*
            IF( CMAX.LT.SMINI ) THEN
               BNORM = MAX( ABS( B( 1, 1 ) )+ABS( B( 1, 2 ) ),
     $                 ABS( B( 2, 1 ) )+ABS( B( 2, 2 ) ) )
               IF( SMINI.LT.ONE .AND. BNORM.GT.ONE ) THEN
                  IF( BNORM.GT.BIGNUM*SMINI )
     $               SCALE = ONE / BNORM
               END IF
               TEMP = SCALE / SMINI
               X( 1, 1 ) = TEMP*B( 1, 1 )
               X( 2, 1 ) = TEMP*B( 2, 1 )
               X( 1, 2 ) = TEMP*B( 1, 2 )
               X( 2, 2 ) = TEMP*B( 2, 2 )
               XNORM = TEMP*BNORM
               INFO = 1
               RETURN
            END IF
*
*           Gaussian elimination with complete pivoting.
*
            UR11 = CRV( ICMAX )
            UI11 = CIV( ICMAX )
            CR21 = CRV( IPIVOT( 2, ICMAX ) )
            CI21 = CIV( IPIVOT( 2, ICMAX ) )
            UR12 = CRV( IPIVOT( 3, ICMAX ) )
            UI12 = CIV( IPIVOT( 3, ICMAX ) )
            CR22 = CRV( IPIVOT( 4, ICMAX ) )
            CI22 = CIV( IPIVOT( 4, ICMAX ) )
            IF( ICMAX.EQ.1 .OR. ICMAX.EQ.4 ) THEN
*
*              Code when off-diagonals of pivoted C are real
*
               IF( ABS( UR11 ).GT.ABS( UI11 ) ) THEN
                  TEMP = UI11 / UR11
                  UR11R = ONE / ( UR11*( ONE+TEMP**2 ) )
                  UI11R = -TEMP*UR11R
               ELSE
                  TEMP = UR11 / UI11
                  UI11R = -ONE / ( UI11*( ONE+TEMP**2 ) )
                  UR11R = -TEMP*UI11R
               END IF
```

```
            LR21 = CR21*UR11R
            LI21 = CR21*UI11R
            UR12S = UR12*UR11R
            UI12S = UR12*UI11R
            UR22 = CR22 - UR12*LR21
            UI22 = CI22 - UR12*LI21
         ELSE
*
*           Code when diagonals of pivoted C are real
*
            UR11R = ONE / UR11
            UI11R = ZERO
            LR21 = CR21*UR11R
            LI21 = CI21*UR11R
            UR12S = UR12*UR11R
            UI12S = UI12*UR11R
            UR22 = CR22 - UR12*LR21 + UI12*LI21
            UI22 = -UR12*LI21 - UI12*LR21
         END IF
         U22ABS = ABS( UR22 ) + ABS( UI22 )
*
*        If smaller pivot < SMINI, use SMINI
*
         IF( U22ABS.LT.SMINI ) THEN
            UR22 = SMINI
            UI22 = ZERO
            INFO = 1
         END IF
         IF( RSWAP( ICMAX ) ) THEN
            BR2 = B( 1, 1 )
            BR1 = B( 2, 1 )
            BI2 = B( 1, 2 )
            BI1 = B( 2, 2 )
         ELSE
            BR1 = B( 1, 1 )
            BR2 = B( 2, 1 )
            BI1 = B( 1, 2 )
            BI2 = B( 2, 2 )
         END IF
         BR2 = BR2 - LR21*BR1 + LI21*BI1
         BI2 = BI2 - LI21*BR1 - LR21*BI1
         BBND = MAX( ( ABS( BR1 )+ABS( BI1 ) )*
     $          ( U22ABS*( ABS( UR11R )+ABS( UI11R ) ) ),
     $          ABS( BR2 )+ABS( BI2 ) )
         IF( BBND.GT.ONE .AND. U22ABS.LT.ONE ) THEN
            IF( BBND.GE.BIGNUM*U22ABS ) THEN
               SCALE = ONE / BBND
               BR1 = SCALE*BR1
               BI1 = SCALE*BI1
               BR2 = SCALE*BR2
```

```
                BI2 = SCALE*BI2
             END IF
          END IF
*
          CALL DLADIV( BR2, BI2, UR22, UI22, XR2, XI2 )
          XR1 = UR11R*BR1 - UI11R*BI1 - UR12S*XR2 + UI12S*XI2
          XI1 = UI11R*BR1 + UR11R*BI1 - UI12S*XR2 - UR12S*XI2
          IF( ZSWAP( ICMAX ) ) THEN
             X( 1, 1 ) = XR2
             X( 2, 1 ) = XR1
             X( 1, 2 ) = XI2
             X( 2, 2 ) = XI1
          ELSE
             X( 1, 1 ) = XR1
             X( 2, 1 ) = XR2
             X( 1, 2 ) = XI1
             X( 2, 2 ) = XI2
          END IF
          XNORM = MAX( ABS( XR1 )+ABS( XI1 ), ABS( XR2 )+ABS( XI2 ) )
*
*         Further scaling if  norm(A) norm(X) > overflow
*
          IF( XNORM.GT.ONE .AND. CMAX.GT.ONE ) THEN
             IF( XNORM.GT.BIGNUM / CMAX ) THEN
                TEMP = CMAX / BIGNUM
                X( 1, 1 ) = TEMP*X( 1, 1 )
                X( 2, 1 ) = TEMP*X( 2, 1 )
                X( 1, 2 ) = TEMP*X( 1, 2 )
                X( 2, 2 ) = TEMP*X( 2, 2 )
                XNORM = TEMP*XNORM
                SCALE = TEMP*SCALE
             END IF
          END IF
       END IF
    END IF
*
    RETURN
*
*   End of DLALN2
*
    END
```

———-

90.146.4 lisp code

— **dlaln2.lisp** —

```
(let* ((zero 0.0) (one 1.0) (two 2.0))
  (declare (type (double-float 0.0 0.0) zero)
           (type (double-float 1.0 1.0) one)
           (type (double-float 2.0 2.0) two))
  (let ((zswap
         (make-array 4 :element-type 't :initial-contents '(nil nil t t)))
        (rswap
         (make-array 4 :element-type 't :initial-contents '(nil t nil t)))
        (ipivot
         (make-array 16
                     :element-type 'fixnum
                     :initial-contents '(1 2 3 4 2 1 4 3 3 4 1 2 4 3 2 1))))
    (declare (type (simple-array fixnum (16)) ipivot)
             (type (simple-array (member t nil) (4)) rswap zswap))
    (defun dlaln2
           (ltrans na nw smin ca a lda d1 d2 b ldb$ wr wi x ldx scale xnorm
            info)
      (declare (type (simple-array double-float (*)) x b a)
               (type (double-float) xnorm scale wi wr d2 d1 ca smin)
               (type fixnum info ldx ldb$ lda nw na)
               (type (member t nil) ltrans))
      (f2cl-lib:with-multi-array-data
          ((a double-float a-%data% a-%offset%)
           (b double-float b-%data% b-%offset%)
           (x double-float x-%data% x-%offset%))
        (prog ((ci (make-array 4 :element-type 'double-float))
               (civ (make-array 4 :element-type 'double-float))
               (cr (make-array 4 :element-type 'double-float))
               (crv (make-array 4 :element-type 'double-float)) (bbnd 0.0)
               (bi1 0.0) (bi2 0.0) (bignum 0.0) (bnorm 0.0) (br1 0.0) (br2 0.0)
               (ci21 0.0) (ci22 0.0) (cmax 0.0) (cnorm 0.0) (cr21 0.0)
               (cr22 0.0) (csi 0.0) (csr 0.0) (li21 0.0) (lr21 0.0) (smini 0.0)
               (smlnum 0.0) (temp 0.0) (u22abs 0.0) (ui11 0.0) (ui11r 0.0)
               (ui12 0.0) (ui12s 0.0) (ui22 0.0) (ur11 0.0) (ur11r 0.0)
               (ur12 0.0) (ur12s 0.0) (ur22 0.0) (xi1 0.0) (xi2 0.0) (xr1 0.0)
               (xr2 0.0) (icmax 0) (j 0))
          (declare (type (simple-array double-float (4)) ci civ cr crv)
                   (type (double-float) bbnd bi1 bi2 bignum bnorm br1 br2 ci21
                                        ci22 cmax cnorm cr21 cr22 csi csr li21
                                        lr21 smini smlnum temp u22abs ui11
                                        ui11r ui12 ui12s ui22 ur11 ur11r ur12
                                        ur12s ur22 xi1 xi2 xr1 xr2)
                   (type fixnum icmax j))
          (setf smlnum (* two (dlamch "Safe minimum")))
          (setf bignum (/ one smlnum))
          (setf smini (max smin smlnum))
          (setf info 0)
          (setf scale one)
          (cond
            ((= na 1)
```

```
(cond
  ((= nw 1)
   (setf csr
           (-
            (* ca
               (f2cl-lib:fref a-%data%
                              (1 1)
                              ((1 lda) (1 *))
                              a-%offset%))
            (* wr d1)))
   (setf cnorm (abs csr))
   (cond
     ((< cnorm smini)
      (setf csr smini)
      (setf cnorm smini)
      (setf info 1)))
   (setf bnorm
           (abs
            (f2cl-lib:fref b-%data%
                           (1 1)
                           ((1 ldb$) (1 *))
                           b-%offset%)))
   (cond
     ((and (< cnorm one) (> bnorm one))
      (if (> bnorm (* bignum cnorm)) (setf scale (/ one bnorm)))))
   (setf (f2cl-lib:fref x-%data% (1 1) ((1 ldx) (1 *)) x-%offset%)
           (/
            (*
             (f2cl-lib:fref b-%data%
                            (1 1)
                            ((1 ldb$) (1 *))
                            b-%offset%)
             scale)
            csr))
   (setf xnorm
           (abs
            (f2cl-lib:fref x-%data%
                           (1 1)
                           ((1 ldx) (1 *))
                           x-%offset%))))
  (t
   (setf csr
           (-
            (* ca
               (f2cl-lib:fref a-%data%
                              (1 1)
                              ((1 lda) (1 *))
                              a-%offset%))
            (* wr d1)))
   (setf csi (* (- wi) d1))
```

```
(setf cnorm (+ (abs csr) (abs csi)))
(cond
  ((< cnorm smini)
   (setf csr smini)
   (setf csi zero)
   (setf cnorm smini)
   (setf info 1)))
(setf bnorm
        (+
         (abs
          (f2cl-lib:fref b-%data%
                         (1 1)
                         ((1 ldb$) (1 *))
                         b-%offset%))
         (abs
          (f2cl-lib:fref b-%data%
                         (1 2)
                         ((1 ldb$) (1 *))
                         b-%offset%))))
(cond
  ((and (< cnorm one) (> bnorm one))
   (if (> bnorm (* bignum cnorm)) (setf scale (/ one bnorm)))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (dladiv
     (* scale
        (f2cl-lib:fref b-%data%
                       (1 1)
                       ((1 ldb$) (1 *))
                       b-%offset%))
     (* scale
        (f2cl-lib:fref b-%data%
                       (1 2)
                       ((1 ldb$) (1 *))
                       b-%offset%))
     csr csi
     (f2cl-lib:fref x-%data% (1 1) ((1 ldx) (1 *)) x-%offset%)
     (f2cl-lib:fref x-%data% (1 2) ((1 ldx) (1 *)) x-%offset%))
  (declare (ignore var-0 var-1 var-2 var-3))
  (setf (f2cl-lib:fref x-%data%
                       (1 1)
                       ((1 ldx) (1 *))
                       x-%offset%)
         var-4)
  (setf (f2cl-lib:fref x-%data%
                       (1 2)
                       ((1 ldx) (1 *))
                       x-%offset%)
         var-5))
(setf xnorm
        (+
```

```
                (abs
                 (f2cl-lib:fref x-%data%
                                (1 1)
                                ((1 ldx) (1 *))
                                x-%offset%))
                (abs
                 (f2cl-lib:fref x-%data%
                                (1 2)
                                ((1 ldx) (1 *))
                                x-%offset%)))))))
(t
 (setf (f2cl-lib:fref crv (1) ((1 4)))
         (-
          (* ca
             (f2cl-lib:fref a-%data%
                            (1 1)
                            ((1 lda) (1 *))
                            a-%offset%))
          (* wr d1)))
 (setf (f2cl-lib:fref crv (4) ((1 4)))
         (-
          (* ca
             (f2cl-lib:fref a-%data%
                            (2 2)
                            ((1 lda) (1 *))
                            a-%offset%))
          (* wr d2)))
 (cond
   (ltrans
    (setf (f2cl-lib:fref crv (3) ((1 4)))
            (* ca
               (f2cl-lib:fref a-%data%
                              (2 1)
                              ((1 lda) (1 *))
                              a-%offset%)))
    (setf (f2cl-lib:fref crv (2) ((1 4)))
            (* ca
               (f2cl-lib:fref a-%data%
                              (1 2)
                              ((1 lda) (1 *))
                              a-%offset%))))
   (t
    (setf (f2cl-lib:fref crv (2) ((1 4)))
            (* ca
               (f2cl-lib:fref a-%data%
                              (2 1)
                              ((1 lda) (1 *))
                              a-%offset%)))
    (setf (f2cl-lib:fref crv (3) ((1 4)))
            (* ca
```

```
                (f2cl-lib:fref a-%data%
                               (1 2)
                               ((1 lda) (1 *))
                               a-%offset%)))))
(cond
  ((= nw 1)
   (setf cmax zero)
   (setf icmax 0)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j 4) nil)
     (tagbody
       (cond
         ((> (abs (f2cl-lib:fref crv (j) ((1 4)))) cmax)
          (setf cmax (abs (f2cl-lib:fref crv (j) ((1 4)))))
          (setf icmax j)))))
   (cond
     ((< cmax smini)
      (setf bnorm
              (max
               (abs
                (f2cl-lib:fref b-%data%
                               (1 1)
                               ((1 ldb$) (1 *))
                               b-%offset%))
               (abs
                (f2cl-lib:fref b-%data%
                               (2 1)
                               ((1 ldb$) (1 *))
                               b-%offset%))))
      (cond
        ((and (< smini one) (> bnorm one))
         (if (> bnorm (* bignum smini))
             (setf scale (/ one bnorm)))))
      (setf temp (/ scale smini))
      (setf (f2cl-lib:fref x-%data%
                           (1 1)
                           ((1 ldx) (1 *))
                           x-%offset%)
              (* temp
                 (f2cl-lib:fref b-%data%
                                (1 1)
                                ((1 ldb$) (1 *))
                                b-%offset%)))
      (setf (f2cl-lib:fref x-%data%
                           (2 1)
                           ((1 ldx) (1 *))
                           x-%offset%)
              (* temp
                 (f2cl-lib:fref b-%data%
                                (2 1)
```

```
                              ((1 ldb$) (1 *))
                              b-%offset%)))
   (setf xnorm (* temp bnorm))
   (setf info 1)
   (go end_label)))
(setf ur11 (f2cl-lib:fref crv (icmax) ((1 4))))
(setf cr21
        (f2cl-lib:fref crv
                       ((f2cl-lib:fref ipivot
                                       (2 icmax)
                                       ((1 4) (1 4))))
                       ((1 4))))
(setf ur12
        (f2cl-lib:fref crv
                       ((f2cl-lib:fref ipivot
                                       (3 icmax)
                                       ((1 4) (1 4))))
                       ((1 4))))
(setf cr22
        (f2cl-lib:fref crv
                       ((f2cl-lib:fref ipivot
                                       (4 icmax)
                                       ((1 4) (1 4))))
                       ((1 4))))
(setf ur11r (/ one ur11))
(setf lr21 (* ur11r cr21))
(setf ur22 (- cr22 (* ur12 lr21)))
(cond
  ((< (abs ur22) smini)
   (setf ur22 smini)
   (setf info 1)))
(cond
  ((f2cl-lib:fref rswap (icmax) ((1 4)))
   (setf br1
           (f2cl-lib:fref b-%data%
                          (2 1)
                          ((1 ldb$) (1 *))
                          b-%offset%))
   (setf br2
           (f2cl-lib:fref b-%data%
                          (1 1)
                          ((1 ldb$) (1 *))
                          b-%offset%)))
  (t
   (setf br1
           (f2cl-lib:fref b-%data%
                          (1 1)
                          ((1 ldb$) (1 *))
                          b-%offset%))
   (setf br2
```

```
          (f2cl-lib:fref b-%data%
                         (2 1)
                         ((1 ldb$) (1 *))
                         b-%offset%))))
(setf br2 (- br2 (* lr21 br1)))
(setf bbnd (max (abs (* br1 (* ur22 ur11r))) (abs br2)))
(cond
  ((and (> bbnd one) (< (abs ur22) one))
   (if (>= bbnd (* bignum (abs ur22)))
       (setf scale (/ one bbnd)))))
(setf xr2 (/ (* br2 scale) ur22))
(setf xr1 (- (* scale br1 ur11r) (* xr2 (* ur11r ur12))))
(cond
  ((f2cl-lib:fref zswap (icmax) ((1 4)))
   (setf (f2cl-lib:fref x-%data%
                        (1 1)
                        ((1 ldx) (1 *))
                        x-%offset%)
          xr2)
   (setf (f2cl-lib:fref x-%data%
                        (2 1)
                        ((1 ldx) (1 *))
                        x-%offset%)
          xr1))
  (t
   (setf (f2cl-lib:fref x-%data%
                        (1 1)
                        ((1 ldx) (1 *))
                        x-%offset%)
          xr1)
   (setf (f2cl-lib:fref x-%data%
                        (2 1)
                        ((1 ldx) (1 *))
                        x-%offset%)
          xr2)))
(setf xnorm (max (abs xr1) (abs xr2)))
(cond
  ((and (> xnorm one) (> cmax one))
   (cond
     ((> xnorm (f2cl-lib:f2cl/ bignum cmax))
      (setf temp (/ cmax bignum))
      (setf (f2cl-lib:fref x-%data%
                           (1 1)
                           ((1 ldx) (1 *))
                           x-%offset%)
            (* temp
               (f2cl-lib:fref x-%data%
                              (1 1)
                              ((1 ldx) (1 *))
                              x-%offset%)))
```

```
        (setf (f2cl-lib:fref x-%data%
                             (2 1)
                             ((1 ldx) (1 *))
                             x-%offset%)
                (* temp
                   (f2cl-lib:fref x-%data%
                                  (2 1)
                                  ((1 ldx) (1 *))
                                  x-%offset%)))
        (setf xnorm (* temp xnorm))
        (setf scale (* temp scale)))))))
 (t
  (setf (f2cl-lib:fref civ (1) ((1 4))) (* (- wi) d1))
  (setf (f2cl-lib:fref civ (2) ((1 4))) zero)
  (setf (f2cl-lib:fref civ (3) ((1 4))) zero)
  (setf (f2cl-lib:fref civ (4) ((1 4))) (* (- wi) d2))
  (setf cmax zero)
  (setf icmax 0)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j 4) nil)
    (tagbody
      (cond
        ((>
          (+ (abs (f2cl-lib:fref crv (j) ((1 4))))
             (abs (f2cl-lib:fref civ (j) ((1 4)))))
          cmax)
         (setf cmax
                 (+ (abs (f2cl-lib:fref crv (j) ((1 4))))
                    (abs (f2cl-lib:fref civ (j) ((1 4))))))
         (setf icmax j)))))
  (cond
    ((< cmax smini)
     (setf bnorm
             (max
              (+
               (abs
                (f2cl-lib:fref b-%data%
                               (1 1)
                               ((1 ldb$) (1 *))
                               b-%offset%))
               (abs
                (f2cl-lib:fref b-%data%
                               (1 2)
                               ((1 ldb$) (1 *))
                               b-%offset%)))
              (+
               (abs
                (f2cl-lib:fref b-%data%
                               (2 1)
                               ((1 ldb$) (1 *))
```

```
                          b-%offset%))
          (abs
           (f2cl-lib:fref b-%data%
                          (2 2)
                          ((1 ldb$) (1 *))
                          b-%offset%)))))
(cond
  ((and (< smini one) (> bnorm one))
   (if (> bnorm (* bignum smini))
       (setf scale (/ one bnorm)))))
(setf temp (/ scale smini))
(setf (f2cl-lib:fref x-%data%
                     (1 1)
                     ((1 ldx) (1 *))
                     x-%offset%)
        (* temp
           (f2cl-lib:fref b-%data%
                          (1 1)
                          ((1 ldb$) (1 *))
                          b-%offset%)))
(setf (f2cl-lib:fref x-%data%
                     (2 1)
                     ((1 ldx) (1 *))
                     x-%offset%)
        (* temp
           (f2cl-lib:fref b-%data%
                          (2 1)
                          ((1 ldb$) (1 *))
                          b-%offset%)))
(setf (f2cl-lib:fref x-%data%
                     (1 2)
                     ((1 ldx) (1 *))
                     x-%offset%)
        (* temp
           (f2cl-lib:fref b-%data%
                          (1 2)
                          ((1 ldb$) (1 *))
                          b-%offset%)))
(setf (f2cl-lib:fref x-%data%
                     (2 2)
                     ((1 ldx) (1 *))
                     x-%offset%)
        (* temp
           (f2cl-lib:fref b-%data%
                          (2 2)
                          ((1 ldb$) (1 *))
                          b-%offset%)))
(setf xnorm (* temp bnorm))
(setf info 1)
(go end_label)))
```

```
(setf ur11 (f2cl-lib:fref crv (icmax) ((1 4))))
(setf ui11 (f2cl-lib:fref civ (icmax) ((1 4))))
(setf cr21
        (f2cl-lib:fref crv
                       ((f2cl-lib:fref ipivot
                                       (2 icmax)
                                       ((1 4) (1 4))))
                       ((1 4))))
(setf ci21
        (f2cl-lib:fref civ
                       ((f2cl-lib:fref ipivot
                                       (2 icmax)
                                       ((1 4) (1 4))))
                       ((1 4))))
(setf ur12
        (f2cl-lib:fref crv
                       ((f2cl-lib:fref ipivot
                                       (3 icmax)
                                       ((1 4) (1 4))))
                       ((1 4))))
(setf ui12
        (f2cl-lib:fref civ
                       ((f2cl-lib:fref ipivot
                                       (3 icmax)
                                       ((1 4) (1 4))))
                       ((1 4))))
(setf cr22
        (f2cl-lib:fref crv
                       ((f2cl-lib:fref ipivot
                                       (4 icmax)
                                       ((1 4) (1 4))))
                       ((1 4))))
(setf ci22
        (f2cl-lib:fref civ
                       ((f2cl-lib:fref ipivot
                                       (4 icmax)
                                       ((1 4) (1 4))))
                       ((1 4))))
(cond
  ((or (= icmax 1) (= icmax 4))
   (cond
     ((> (abs ur11) (abs ui11))
      (setf temp (/ ui11 ur11))
      (setf ur11r (/ one (* ur11 (+ one (expt temp 2)))))
      (setf ui11r (* (- temp) ur11r)))
     (t
      (setf temp (/ ur11 ui11))
      (setf ui11r (/ (- one) (* ui11 (+ one (expt temp 2)))))
      (setf ur11r (* (- temp) ui11r))))
   (setf lr21 (* cr21 ur11r))
```

```
    (setf li21 (* cr21 ui11r))
    (setf ur12s (* ur12 ur11r))
    (setf ui12s (* ur12 ui11r))
    (setf ur22 (- cr22 (* ur12 lr21)))
    (setf ui22 (- ci22 (* ur12 li21))))
   (t
    (setf ur11r (/ one ur11))
    (setf ui11r zero)
    (setf lr21 (* cr21 ur11r))
    (setf li21 (* ci21 ur11r))
    (setf ur12s (* ur12 ur11r))
    (setf ui12s (* ui12 ur11r))
    (setf ur22 (+ (- cr22 (* ur12 lr21)) (* ui12 li21)))
    (setf ui22 (- (* (- ur12) li21) (* ui12 lr21)))))
(setf u22abs (+ (abs ur22) (abs ui22)))
(cond
  ((< u22abs smini)
    (setf ur22 smini)
    (setf ui22 zero)
    (setf info 1)))
(cond
  ((f2cl-lib:fref rswap (icmax) ((1 4)))
    (setf br2
            (f2cl-lib:fref b-%data%
                           (1 1)
                           ((1 ldb$) (1 *))
                           b-%offset%))
    (setf br1
            (f2cl-lib:fref b-%data%
                           (2 1)
                           ((1 ldb$) (1 *))
                           b-%offset%))
    (setf bi2
            (f2cl-lib:fref b-%data%
                           (1 2)
                           ((1 ldb$) (1 *))
                           b-%offset%))
    (setf bi1
            (f2cl-lib:fref b-%data%
                           (2 2)
                           ((1 ldb$) (1 *))
                           b-%offset%)))
  (t
    (setf br1
            (f2cl-lib:fref b-%data%
                           (1 1)
                           ((1 ldb$) (1 *))
                           b-%offset%))
    (setf br2
            (f2cl-lib:fref b-%data%
```

```
                            (2 1)
                            ((1 ldb$) (1 *))
                            b-%offset%))
   (setf bi1
           (f2cl-lib:fref b-%data%
                            (1 2)
                            ((1 ldb$) (1 *))
                            b-%offset%))
   (setf bi2
           (f2cl-lib:fref b-%data%
                            (2 2)
                            ((1 ldb$) (1 *))
                            b-%offset%))))
(setf br2 (+ (- br2 (* lr21 br1)) (* li21 bi1)))
(setf bi2 (- bi2 (* li21 br1) (* lr21 bi1)))
(setf bbnd
        (max
         (* (+ (abs br1) (abs bi1))
            (* u22abs (+ (abs ur11r) (abs ui11r))))
         (+ (abs br2) (abs bi2))))
(cond
  ((and (> bbnd one) (< u22abs one))
   (cond
     ((>= bbnd (* bignum u22abs))
      (setf scale (/ one bbnd))
      (setf br1 (* scale br1))
      (setf bi1 (* scale bi1))
      (setf br2 (* scale br2))
      (setf bi2 (* scale bi2))))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (dladiv br2 bi2 ur22 ui22 xr2 xi2)
  (declare (ignore var-0 var-1 var-2 var-3))
  (setf xr2 var-4)
  (setf xi2 var-5))
(setf xr1
        (+ (- (* ur11r br1) (* ui11r bi1) (* ur12s xr2))
           (* ui12s xi2)))
(setf xi1
        (- (+ (* ui11r br1) (* ur11r bi1))
           (* ui12s xr2)
           (* ur12s xi2)))
(cond
  ((f2cl-lib:fref zswap (icmax) ((1 4)))
   (setf (f2cl-lib:fref x-%data%
                        (1 1)
                        ((1 ldx) (1 *))
                        x-%offset%)
           xr2)
   (setf (f2cl-lib:fref x-%data%
                        (2 1)
```

```
                                ((1 ldx) (1 *))
                                x-%offset%)
                        xr1)
                 (setf (f2cl-lib:fref x-%data%
                                (1 2)
                                ((1 ldx) (1 *))
                                x-%offset%)
                        xi2)
                 (setf (f2cl-lib:fref x-%data%
                                (2 2)
                                ((1 ldx) (1 *))
                                x-%offset%)
                        xi1))
                (t
                 (setf (f2cl-lib:fref x-%data%
                                (1 1)
                                ((1 ldx) (1 *))
                                x-%offset%)
                        xr1)
                 (setf (f2cl-lib:fref x-%data%
                                (2 1)
                                ((1 ldx) (1 *))
                                x-%offset%)
                        xr2)
                 (setf (f2cl-lib:fref x-%data%
                                (1 2)
                                ((1 ldx) (1 *))
                                x-%offset%)
                        xi1)
                 (setf (f2cl-lib:fref x-%data%
                                (2 2)
                                ((1 ldx) (1 *))
                                x-%offset%)
                        xi2)))
            (setf xnorm
                    (max (+ (abs xr1) (abs xi1)) (+ (abs xr2) (abs xi2))))
            (cond
              ((and (> xnorm one) (> cmax one))
               (cond
                 ((> xnorm (f2cl-lib:f2cl/ bignum cmax))
                  (setf temp (/ cmax bignum))
                  (setf (f2cl-lib:fref x-%data%
                                  (1 1)
                                  ((1 ldx) (1 *))
                                  x-%offset%)
                        (* temp
                           (f2cl-lib:fref x-%data%
                                          (1 1)
                                          ((1 ldx) (1 *))
                                          x-%offset%)))
```

```
                    (setf (f2cl-lib:fref x-%data%
                                         (2 1)
                                         ((1 ldx) (1 *))
                                         x-%offset%)
                          (* temp
                             (f2cl-lib:fref x-%data%
                                            (2 1)
                                            ((1 ldx) (1 *))
                                            x-%offset%)))
                    (setf (f2cl-lib:fref x-%data%
                                         (1 2)
                                         ((1 ldx) (1 *))
                                         x-%offset%)
                          (* temp
                             (f2cl-lib:fref x-%data%
                                            (1 2)
                                            ((1 ldx) (1 *))
                                            x-%offset%)))
                    (setf (f2cl-lib:fref x-%data%
                                         (2 2)
                                         ((1 ldx) (1 *))
                                         x-%offset%)
                          (* temp
                             (f2cl-lib:fref x-%data%
                                            (2 2)
                                            ((1 ldx) (1 *))
                                            x-%offset%)))
                    (setf xnorm (* temp xnorm))
                    (setf scale (* temp scale)))))))))
end_label
         (return
          (values nil
                  nil
                  nil
                  nil
                  nil
                  nil
                  nil
                  nil
                  nil
                  nil
                  nil
                  nil
                  nil
                  nil
                  scale
                  xnorm
                  info)))))))
```

———-

90.147 dlamch

90.147.1 Axiom unit tests

— **dlamch.input** —

```
)set break resume
)sys rm -f dlamch.output
)spool dlamch.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.147.2 Axiom help page

— **dlamch.help** —

```
======================================================================
dlamch examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DLAMCH - determine double precision machine parameters

SYNOPSIS
       DOUBLE PRECISION FUNCTION DLAMCH( CMACH )

           CHARACTER    CMACH

 Purpose
 =======

  DLAMCH determines double precision machine parameters.
```

```
 Arguments
 =========

  CMACH   (input) CHARACTER*1
          Specifies the value to be returned by DLAMCH:
          = 'E' or 'e',   DLAMCH := eps
          = 'S' or 's ,   DLAMCH := sfmin
          = 'B' or 'b',   DLAMCH := base
          = 'P' or 'p',   DLAMCH := eps*base
          = 'N' or 'n',   DLAMCH := t
          = 'R' or 'r',   DLAMCH := rnd
          = 'M' or 'm',   DLAMCH := emin
          = 'U' or 'u',   DLAMCH := rmin
          = 'L' or 'l',   DLAMCH := emax
          = 'O' or 'o',   DLAMCH := rmax

          where

          eps   = relative machine precision
          sfmin = safe minimum, such that 1/sfmin does not overflow
          base  = base of the machine
          prec  = eps*base
          t     = number of (base) digits in the mantissa
          rnd   = 1.0 when rounding occurs in addition, 0.0 otherwise
          emin  = minimum exponent before (gradual) underflow
          rmin  = underflow threshold - base**(emin-1)
          emax  = largest exponent before overflow
          rmax  = overflow threshold  - (base**emax)*(1-eps)
```

90.147.3 fortran code

— dlamch.f —

```
      DOUBLE PRECISION FUNCTION DLAMCH( CMACH )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1992
*
*     .. Scalar Arguments ..
      CHARACTER          CMACH
*     ..
*
```

```
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            FIRST, LRND
      INTEGER            BETA, IMAX, IMIN, IT
      DOUBLE PRECISION   BASE, EMAX, EMIN, EPS, PREC, RMACH, RMAX, RMIN,
     $                   RND, SFMIN, SMALL, T
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLAMC2
*     ..
*     .. Save statement ..
      SAVE               FIRST, EPS, SFMIN, BASE, T, RND, EMIN, RMIN,
     $                   EMAX, RMAX, PREC
*     ..
*     .. Data statements ..
      DATA               FIRST / .TRUE. /
*     ..
*     .. Executable Statements ..
*
      IF( FIRST ) THEN
         FIRST = .FALSE.
         CALL DLAMC2( BETA, IT, LRND, EPS, IMIN, RMIN, IMAX, RMAX )
         BASE = BETA
         T = IT
         IF( LRND ) THEN
            RND = ONE
            EPS = ( BASE**( 1-IT ) ) / 2
         ELSE
            RND = ZERO
            EPS = BASE**( 1-IT )
         END IF
         PREC = EPS*BASE
         EMIN = IMIN
         EMAX = IMAX
         SFMIN = RMIN
         SMALL = ONE / RMAX
         IF( SMALL.GE.SFMIN ) THEN
*
*           Use SMALL plus a bit, to avoid the possibility of rounding
*           causing overflow when computing  1/sfmin.
*
```

```
            SFMIN = SMALL*( ONE+EPS )
         END IF
      END IF
*
      IF( LSAME( CMACH, 'E' ) ) THEN
         RMACH = EPS
      ELSE IF( LSAME( CMACH, 'S' ) ) THEN
         RMACH = SFMIN
      ELSE IF( LSAME( CMACH, 'B' ) ) THEN
         RMACH = BASE
      ELSE IF( LSAME( CMACH, 'P' ) ) THEN
         RMACH = PREC
      ELSE IF( LSAME( CMACH, 'N' ) ) THEN
         RMACH = T
      ELSE IF( LSAME( CMACH, 'R' ) ) THEN
         RMACH = RND
      ELSE IF( LSAME( CMACH, 'M' ) ) THEN
         RMACH = EMIN
      ELSE IF( LSAME( CMACH, 'U' ) ) THEN
         RMACH = RMIN
      ELSE IF( LSAME( CMACH, 'L' ) ) THEN
         RMACH = EMAX
      ELSE IF( LSAME( CMACH, 'O' ) ) THEN
         RMACH = RMAX
      END IF
*
      DLAMCH = RMACH
      RETURN
*
*     End of DLAMCH
*
      END
```

———-

90.147.4 lisp code

— dlamch.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dlamch (cmach)
    (declare (type (string *) cmach))
    (f2cl-lib:with-multi-array-data
        ((cmach character cmach-%data% cmach-%offset%))
```

```
(prog ((rnd 0.0d0) (eps 0.0d0) (sfmin 0.0d0) (small 0.0d0) (rmach 0.0d0)
       (dlamch 0.0d0))
  (declare (type (double-float) rnd eps sfmin small rmach dlamch))
  (setf rnd one)
  (cond
   ((= one rnd)
    (setf eps
            (coerce
             (*
              (multiple-value-bind (ret-val var-0)
                  (epsilon zero)
                (declare (ignore))
                (when var-0 (setf zero var-0))
                ret-val)
              0.5)
             'double-float)))
   (t
    (setf eps
            (coerce
             (multiple-value-bind (ret-val var-0)
                 (epsilon zero)
               (declare (ignore))
               (when var-0 (setf zero var-0))
               ret-val)
             'double-float))))
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame cmach "E")
      (declare (ignore var-1))
      (when var-0 (setf cmach var-0))
      ret-val)
    (setf rmach eps))
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame cmach "S")
      (declare (ignore var-1))
      (when var-0 (setf cmach var-0))
      ret-val)
    (setf sfmin
            (coerce
             (multiple-value-bind (ret-val var-0)
                 (tiny zero)
               (declare (ignore))
               (when var-0 (setf zero var-0))
               ret-val)
             'double-float))
    (setf small
            (/ one
               (multiple-value-bind (ret-val var-0)
                   (huge zero)
                 (declare (ignore))
```

```
              (when var-0 (setf zero var-0))
              ret-val)))
 (cond ((>= small sfmin) (setf sfmin (* small (+ one eps)))))
 (setf rmach sfmin))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame cmach "B")
   (declare (ignore var-1))
   (when var-0 (setf cmach var-0))
   ret-val)
 (setf rmach
         (coerce
          (multiple-value-bind (ret-val var-0)
              (radix zero)
            (declare (ignore))
            (when var-0 (setf zero var-0))
            ret-val)
          'double-float)))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame cmach "P")
   (declare (ignore var-1))
   (when var-0 (setf cmach var-0))
   ret-val)
 (setf rmach
         (* eps
            (multiple-value-bind (ret-val var-0)
                (radix zero)
              (declare (ignore))
              (when var-0 (setf zero var-0))
              ret-val))))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame cmach "N")
   (declare (ignore var-1))
   (when var-0 (setf cmach var-0))
   ret-val)
 (setf rmach
         (coerce
          (multiple-value-bind (ret-val var-0)
              (digits zero)
            (declare (ignore))
            (when var-0 (setf zero var-0))
            ret-val)
          'double-float)))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame cmach "R")
   (declare (ignore var-1))
   (when var-0 (setf cmach var-0))
   ret-val)
 (setf rmach rnd))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame cmach "M")
```

```
   (declare (ignore var-1))
   (when var-0 (setf cmach var-0))
   ret-val)
 (setf rmach
         (coerce
          (the f2cl-lib:integer4
               (multiple-value-bind (ret-val var-0)
                   (minexponent zero)
                 (declare (ignore))
                 (when var-0 (setf zero var-0))
                 ret-val))
          'double-float)))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame cmach "U")
   (declare (ignore var-1))
   (when var-0 (setf cmach var-0))
   ret-val)
 (setf rmach
         (coerce
          (multiple-value-bind (ret-val var-0)
              (tiny zero)
            (declare (ignore))
            (when var-0 (setf zero var-0))
            ret-val)
          'double-float)))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame cmach "L")
   (declare (ignore var-1))
   (when var-0 (setf cmach var-0))
   ret-val)
 (setf rmach
         (coerce
          (the f2cl-lib:integer4
               (multiple-value-bind (ret-val var-0)
                   (maxexponent zero)
                 (declare (ignore))
                 (when var-0 (setf zero var-0))
                 ret-val))
          'double-float)))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame cmach "O")
   (declare (ignore var-1))
   (when var-0 (setf cmach var-0))
   ret-val)
 (setf rmach
         (coerce
          (multiple-value-bind (ret-val var-0)
              (huge zero)
            (declare (ignore))
            (when var-0 (setf zero var-0))
```

```
                    ret-val)
                  'double-float)))
         (t (setf rmach zero)))
        (setf dlamch rmach)
        (go end_label)
       end_label
        (return (values dlamch cmach))))))
```

———-

90.148 dlamc1

90.148.1 Axiom unit tests

— **dlamc1.input** —

```
)set break resume
)sys rm -f dlamc1.output
)spool dlamc1.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.148.2 Axiom help page

— **dlamc1.help** —

```
==========================================================================
dlamc1 examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

  -- LAPACK auxiliary routine (version 1.1) --
     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
     Courant Institute, Argonne National Lab, and Rice University
```

```
     October 31, 1992

     .. Scalar Arguments ..
<       LOGICAL            IEEE1, RND >
<       INTEGER            BETA, T >
     ..

  Purpose
  =======

  DLAMC1 determines the machine parameters given by BETA, T, RND, and
  IEEE1.

  Arguments
  =========

  BETA    (output) INTEGER
          The base of the machine.

  T       (output) INTEGER
          The number of ( BETA ) digits in the mantissa.

  RND     (output) LOGICAL
          Specifies whether proper rounding  ( RND = .TRUE. )  or
          chopping  ( RND = .FALSE. )  occurs in addition. This may not
          be a reliable guide to the way in which the machine performs
          its arithmetic.

  IEEE1   (output) LOGICAL
          Specifies whether rounding appears to be done in the IEEE
          'round to nearest' style.

  Further Details
  ===============

  See Malcolm M. A. (1972) Algorithms to reveal properties of
      floating-point arithmetic. Comms. of the ACM, 15, 949-951.

  See Gentleman W. M. and Marovich S. B. (1974) More on algorithms
      that reveal properties of floating point arithmetic units.
      Comms. of the ACM, 17, 276-277.
```

90.148.3 fortran code

— dlamc1.f —

```
*********************************************************************
*
      SUBROUTINE DLAMC1( BETA, T, RND, IEEE1 )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1992
*
*     .. Scalar Arguments ..
      LOGICAL            IEEE1, RND
      INTEGER            BETA, T
*     ..
*
* ===================================================================
*
*     .. Local Scalars ..
      LOGICAL            FIRST, LIEEE1, LRND
      INTEGER            LBETA, LT
      DOUBLE PRECISION   A, B, C, F, ONE, QTR, SAVEC, T1, T2
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMC3
      EXTERNAL           DLAMC3
*     ..
*     .. Save statement ..
      SAVE               FIRST, LIEEE1, LBETA, LRND, LT
*     ..
*     .. Data statements ..
      DATA               FIRST / .TRUE. /
*     ..
*     .. Executable Statements ..
*
      IF( FIRST ) THEN
         FIRST = .FALSE.
         ONE = 1
*
*        LBETA,  LIEEE1,  LT and  LRND  are the  local values  of  BETA,
*        IEEE1, T and RND.
*
*        Throughout this routine  we use the function  DLAMC3  to ensure
*        that relevant values are  stored and not held in registers,  or
*        are not affected by optimizers.
*
*        Compute  a = 2.0**m  with the  smallest positive integer m such
*        that
*
*           fl( a + 1.0 ) = a.
*
         A = 1
```

```
      C = 1
*
*+    WHILE( C.EQ.ONE )LOOP
   10 CONTINUE
      IF( C.EQ.ONE ) THEN
         A = 2*A
         C = DLAMC3( A, ONE )
         C = DLAMC3( C, -A )
         GO TO 10
      END IF
*+    END WHILE
*
*     Now compute  b = 2.0**m  with the smallest positive integer m
*     such that
*
*        fl( a + b ) .gt. a.
*
      B = 1
      C = DLAMC3( A, B )
*
*+    WHILE( C.EQ.A )LOOP
   20 CONTINUE
      IF( C.EQ.A ) THEN
         B = 2*B
         C = DLAMC3( A, B )
         GO TO 20
      END IF
*+    END WHILE
*
*     Now compute the base.  a and c  are neighbouring floating point
*     numbers  in the  interval  ( beta**t, beta**( t + 1 ) )  and so
*     their difference is beta. Adding 0.25 to c is to ensure that it
*     is truncated to beta and not ( beta - 1 ).
*
      QTR = ONE / 4
      SAVEC = C
      C = DLAMC3( C, -A )
      LBETA = C + QTR
*
*     Now determine whether rounding or chopping occurs,  by adding a
*     bit  less  than  beta/2  and a  bit  more  than  beta/2  to  a.
*
      B = LBETA
      F = DLAMC3( B / 2, -B / 100 )
      C = DLAMC3( F, A )
      IF( C.EQ.A ) THEN
         LRND = .TRUE.
      ELSE
         LRND = .FALSE.
      END IF
```

```
         F = DLAMC3( B / 2, B / 100 )
         C = DLAMC3( F, A )
         IF( ( LRND ) .AND. ( C.EQ.A ) )
     $      LRND = .FALSE.
*
*        Try and decide whether rounding is done in the  IEEE  'round to
*        nearest' style. B/2 is half a unit in the last place of the two
*        numbers A and SAVEC. Furthermore, A is even, i.e. has last  bit
*        zero, and SAVEC is odd. Thus adding B/2 to A should not  change
*        A, but adding B/2 to SAVEC should change SAVEC.
*
         T1 = DLAMC3( B / 2, A )
         T2 = DLAMC3( B / 2, SAVEC )
         LIEEE1 = ( T1.EQ.A ) .AND. ( T2.GT.SAVEC ) .AND. LRND
*
*        Now find  the  mantissa, t.  It should  be the  integer part of
*        log to the base beta of a,  however it is safer to determine  t
*        by powering.  So we find t as the smallest positive integer for
*        which
*
*           fl( beta**t + 1.0 ) = 1.0.
*
         LT = 0
         A = 1
         C = 1
*
*+       WHILE( C.EQ.ONE )LOOP
   30    CONTINUE
         IF( C.EQ.ONE ) THEN
            LT = LT + 1
            A = A*LBETA
            C = DLAMC3( A, ONE )
            C = DLAMC3( C, -A )
            GO TO 30
         END IF
*+       END WHILE
*
      END IF
*
      BETA = LBETA
      T = LT
      RND = LRND
      IEEE1 = LIEEE1
      RETURN
*
*     End of DLAMC1
*
      END
*
```

———-

90.148.4 lisp code

— dlamc1.lisp —

```
(let ((lieee1 nil) (lbeta 0) (lrnd nil) (f2cl-lib:lt 0) (first$ nil))
  (declare (type f2cl-lib:logical lieee1 lrnd first$)
           (type (f2cl-lib:integer4) lbeta f2cl-lib:lt))
  (setq first$ f2cl-lib:%true%)
  (defun dlamc1 (beta t$ rnd ieee1)
    (declare (type f2cl-lib:logical ieee1 rnd)
             (type (f2cl-lib:integer4) t$ beta))
    (prog ((a 0.0d0) (b 0.0d0) (c 0.0d0) (f 0.0d0) (one 0.0d0) (qtr 0.0d0)
           (savec 0.0d0) (t1 0.0d0) (t2 0.0d0))
      (declare (type (double-float) t2 t1 savec qtr one f c b a))
      (cond
       (first$
        (tagbody
          (setf one (coerce (the f2cl-lib:integer4 1) 'double-float))
          (setf a (coerce (the f2cl-lib:integer4 1) 'double-float))
          (setf c (coerce (the f2cl-lib:integer4 1) 'double-float))
         label10
          (cond
           ((= c one) (setf a (* 2 a)) (setf c (dlamc3 a one))
            (setf c (dlamc3 c (- a))) (go label10)))
          (setf b (coerce (the f2cl-lib:integer4 1) 'double-float))
          (setf c (dlamc3 a b))
         label20
          (cond ((= c a) (setf b (* 2 b)) (setf c (dlamc3 a b)) (go label20)))
          (setf qtr (/ one 4))
          (setf savec c)
          (setf c (dlamc3 c (- a)))
          (setf lbeta (f2cl-lib:int (+ c qtr)))
          (setf b (coerce (the f2cl-lib:integer4 lbeta) 'double-float))
          (setf f (dlamc3 (/ b 2) (/ (- b) 100)))
          (setf c (dlamc3 f a))
          (cond ((= c a) (setf lrnd f2cl-lib:%true%))
                (t (setf lrnd f2cl-lib:%false%)))
          (setf f (dlamc3 (/ b 2) (/ b 100)))
          (setf c (dlamc3 f a))
          (if (and lrnd (= c a))
              (setf lrnd f2cl-lib:%false%))
          (setf t1 (dlamc3 (/ b 2) a))
          (setf t2 (dlamc3 (/ b 2) savec))
          (setf lieee1 (and (= t1 a) (> t2 savec) lrnd))
          (setf f2cl-lib:lt 0)
          (setf a (coerce (the f2cl-lib:integer4 1) 'double-float))
```

```
          (setf c (coerce (the f2cl-lib:integer4 1) 'double-float))
         label30
          (cond
           ((= c one) (setf f2cl-lib:lt (f2cl-lib:int-add f2cl-lib:lt 1))
            (setf a (* a lbeta)) (setf c (dlamc3 a one))
            (setf c (dlamc3 c (- a))) (go label30))))))
      (setf beta lbeta)
      (setf t$ f2cl-lib:lt)
      (setf rnd lrnd)
      (setf ieee1 lieee1)
      (setf first$ f2cl-lib:%false%)
      (go end_label)
     end_label
      (return (values beta t$ rnd ieee1)))))
```

———-

90.149 dlamc2

90.149.1 Axiom unit tests

— **dlamc2.input** —

```
)set break resume
)sys rm -f dlamc2.output
)spool dlamc2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.149.2 Axiom help page

— **dlamc2.help** —

```
==========================================================================
dlamc2 examples
==========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

  -- LAPACK auxiliary routine (version 1.1) --
     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
     Courant Institute, Argonne National Lab, and Rice University
     October 31, 1992

     .. Scalar Arguments ..
<       LOGICAL            RND >
<       INTEGER            BETA, EMAX, EMIN, T >
<       DOUBLE PRECISION   EPS, RMAX, RMIN >
     ..

  Purpose
  =======

  DLAMC2 determines the machine parameters specified in its argument
  list.

  Arguments
  =========

  BETA    (output) INTEGER
          The base of the machine.

  T       (output) INTEGER
          The number of ( BETA ) digits in the mantissa.

  RND     (output) LOGICAL
          Specifies whether proper rounding  ( RND = .TRUE. )  or
          chopping  ( RND = .FALSE. )  occurs in addition. This may not
          be a reliable guide to the way in which the machine performs
          its arithmetic.

  EPS     (output) DOUBLE PRECISION
          The smallest positive number such that

             fl( 1.0 - EPS ) .LT. 1.0,

          where fl denotes the computed value.

  EMIN    (output) INTEGER
          The minimum exponent before (gradual) underflow occurs.

  RMIN    (output) DOUBLE PRECISION
          The smallest normalized number for the machine, given by
          BASE**( EMIN - 1 ), where  BASE  is the floating point value
          of BETA.
```

```
EMAX    (output) INTEGER
        The maximum exponent before overflow occurs.

RMAX    (output) DOUBLE PRECISION
        The largest positive number for the machine, given by
        BASE**EMAX * ( 1 - EPS ), where  BASE  is the floating point
        value of BETA.

Further Details
===============

The computation of  EPS  is based on a routine PARANOIA by
W. Kahan of the University of California at Berkeley.
```

———-

90.149.3 fortran code

— dlamc2.f —

```
***********************************************************************
*
      SUBROUTINE DLAMC2( BETA, T, RND, EPS, EMIN, RMIN, EMAX, RMAX )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1992
*
*     .. Scalar Arguments ..
      LOGICAL            RND
      INTEGER            BETA, EMAX, EMIN, T
      DOUBLE PRECISION   EPS, RMAX, RMIN
*     ..
*
* ====================================================================
*
*     .. Local Scalars ..
      LOGICAL            FIRST, IEEE, IWARN, LIEEE1, LRND
      INTEGER            GNMIN, GPMIN, I, LBETA, LEMAX, LEMIN, LT,
     $                   NGNMIN, NGPMIN
      DOUBLE PRECISION   A, B, C, HALF, LEPS, LRMAX, LRMIN, ONE, RBASE,
     $                   SIXTH, SMALL, THIRD, TWO, ZERO
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMC3
```

```
      EXTERNAL           DLAMC3
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLAMC1, DLAMC4, DLAMC5
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. Save statement ..
      SAVE               FIRST, IWARN, LBETA, LEMAX, LEMIN, LEPS, LRMAX,
     $                   LRMIN, LT
*     ..
*     .. Data statements ..
      DATA               FIRST / .TRUE. / , IWARN / .FALSE. /
*     ..
*     .. Executable Statements ..
*
      IF( FIRST ) THEN
         FIRST = .FALSE.
         ZERO = 0
         ONE = 1
         TWO = 2
*
*        LBETA, LT, LRND, LEPS, LEMIN and LRMIN  are the local values of
*        BETA, T, RND, EPS, EMIN and RMIN.
*
*        Throughout this routine  we use the function  DLAMC3  to ensure
*        that relevant values are stored  and not held in registers,  or
*        are not affected by optimizers.
*
*        DLAMC1 returns the parameters  LBETA, LT, LRND and LIEEE1.
*
         CALL DLAMC1( LBETA, LT, LRND, LIEEE1 )
*
*        Start to find EPS.
*
         B = LBETA
         A = B**( -LT )
         LEPS = A
*
*        Try some tricks to see whether or not this is the correct  EPS.
*
         B = TWO / 3
         HALF = ONE / 2
         SIXTH = DLAMC3( B, -HALF )
         THIRD = DLAMC3( SIXTH, SIXTH )
         B = DLAMC3( THIRD, -HALF )
         B = DLAMC3( B, SIXTH )
         B = ABS( B )
         IF( B.LT.LEPS )
```

```
     $      B = LEPS
*
        LEPS = 1
*
*+      WHILE( ( LEPS.GT.B ).AND.( B.GT.ZERO ) )LOOP
   10   CONTINUE
        IF( ( LEPS.GT.B ) .AND. ( B.GT.ZERO ) ) THEN
           LEPS = B
           C = DLAMC3( HALF*LEPS, ( TWO**5 )*( LEPS**2 ) )
           C = DLAMC3( HALF, -C )
           B = DLAMC3( HALF, C )
           C = DLAMC3( HALF, -B )
           B = DLAMC3( HALF, C )
           GO TO 10
        END IF
*+      END WHILE
*
        IF( A.LT.LEPS )
     $      LEPS = A
*
*       Computation of EPS complete.
*
*       Now find  EMIN.  Let A = + or - 1, and + or - (1 + BASE**(-3)).
*       Keep dividing  A by BETA until (gradual) underflow occurs. This
*       is detected when we cannot recover the previous A.
*
        RBASE = ONE / LBETA
        SMALL = ONE
        DO 20 I = 1, 3
           SMALL = DLAMC3( SMALL*RBASE, ZERO )
   20   CONTINUE
        A = DLAMC3( ONE, SMALL )
        CALL DLAMC4( NGPMIN, ONE, LBETA )
        CALL DLAMC4( NGNMIN, -ONE, LBETA )
        CALL DLAMC4( GPMIN, A, LBETA )
        CALL DLAMC4( GNMIN, -A, LBETA )
        IEEE = .FALSE.
*
        IF( ( NGPMIN.EQ.NGNMIN ) .AND. ( GPMIN.EQ.GNMIN ) ) THEN
           IF( NGPMIN.EQ.GPMIN ) THEN
              LEMIN = NGPMIN
*           ( Non twos-complement machines, no gradual underflow;
*             e.g.,  VAX )
           ELSE IF( ( GPMIN-NGPMIN ).EQ.3 ) THEN
              LEMIN = NGPMIN - 1 + LT
              IEEE = .TRUE.
*           ( Non twos-complement machines, with gradual underflow;
*             e.g., IEEE standard followers )
           ELSE
              LEMIN = MIN( NGPMIN, GPMIN )
```

```
*           ( A guess; no known machine )
              IWARN = .TRUE.
           END IF
*
        ELSE IF( ( NGPMIN.EQ.GPMIN ) .AND. ( NGNMIN.EQ.GNMIN ) ) THEN
           IF( ABS( NGPMIN-NGNMIN ).EQ.1 ) THEN
              LEMIN = MAX( NGPMIN, NGNMIN )
*           ( Twos-complement machines, no gradual underflow;
*             e.g., CYBER 205 )
           ELSE
              LEMIN = MIN( NGPMIN, NGNMIN )
*           ( A guess; no known machine )
              IWARN = .TRUE.
           END IF
*
        ELSE IF( ( ABS( NGPMIN-NGNMIN ).EQ.1 ) .AND.
     $            ( GPMIN.EQ.GNMIN ) ) THEN
           IF( ( GPMIN-MIN( NGPMIN, NGNMIN ) ).EQ.3 ) THEN
              LEMIN = MAX( NGPMIN, NGNMIN ) - 1 + LT
*           ( Twos-complement machines with gradual underflow;
*             no known machine )
           ELSE
              LEMIN = MIN( NGPMIN, NGNMIN )
*           ( A guess; no known machine )
              IWARN = .TRUE.
           END IF
*
        ELSE
           LEMIN = MIN( NGPMIN, NGNMIN, GPMIN, GNMIN )
*         ( A guess; no known machine )
           IWARN = .TRUE.
        END IF
***
* Comment out this if block if EMIN is ok
        IF( IWARN ) THEN
           FIRST = .TRUE.
           WRITE( 6, FMT = 9999 )LEMIN
        END IF
***
*
*       Assume IEEE arithmetic if we found denormalised  numbers above,
*       or if arithmetic seems to round in the  IEEE style,  determined
*       in routine DLAMC1. A true IEEE machine should have both  things
*       true; however, faulty machines may have one or the other.
*
        IEEE = IEEE .OR. LIEEE1
*
*       Compute  RMIN by successive division by  BETA. We could compute
*       RMIN as BASE**( EMIN - 1 ),  but some machines underflow during
*       this computation.
```

```
*
         LRMIN = 1
         DO 30 I = 1, 1 - LEMIN
            LRMIN = DLAMC3( LRMIN*RBASE, ZERO )
   30    CONTINUE
*
*        Finally, call DLAMC5 to compute EMAX and RMAX.
*
         CALL DLAMC5( LBETA, LT, LEMIN, IEEE, LEMAX, LRMAX )
      END IF
*
      BETA = LBETA
      T = LT
      RND = LRND
      EPS = LEPS
      EMIN = LEMIN
      RMIN = LRMIN
      EMAX = LEMAX
      RMAX = LRMAX
*
      RETURN
*
 9999 FORMAT( / / ' WARNING. The value EMIN may be incorrect:-',
     $      '  EMIN = ', I8, /
     $      ' If, after inspection, the value EMIN looks',
     $      ' acceptable please comment out ',
     $      / ' the IF block as marked within the code of routine',
     $      ' DLAMC2,', / ' otherwise supply EMIN explicitly.', / )
*
*     End of DLAMC2
*
      END
```

———-

90.149.4 lisp code

— **dlamc2.lisp** —

```
(let ((lbeta 0)
      (lemax 0)
      (lemin 0)
      (leps 0.0d0)
      (lrmax 0.0d0)
      (lrmin 0.0d0)
      (f2cl-lib:lt 0)
      (first$ nil)
      (iwarn nil))
```

```
(declare (type (f2cl-lib:integer4) lbeta lemax lemin f2cl-lib:lt)
         (type (double-float) leps lrmax lrmin)
         (type f2cl-lib:logical first$ iwarn))
(setq first$ f2cl-lib:%true%)
(setq iwarn f2cl-lib:%false%)
(defun dlamc2 (beta t$ rnd eps emin rmin emax rmax)
  (declare (type (double-float) rmax rmin eps)
           (type f2cl-lib:logical rnd)
           (type (f2cl-lib:integer4) emax emin t$ beta))
  (prog ((a 0.0d0) (b 0.0d0) (c 0.0d0) (half 0.0d0) (one 0.0d0) (rbase 0.0d0)
         (sixth$ 0.0d0) (small 0.0d0) (third$ 0.0d0) (two 0.0d0) (zero 0.0d0)
         (gnmin 0) (gpmin 0) (i 0) (ngnmin 0) (ngpmin 0) (ieee nil)
         (lieee1 nil) (lrnd nil))
    (declare (type f2cl-lib:logical lrnd lieee1 ieee)
             (type (f2cl-lib:integer4) ngpmin ngnmin i gpmin gnmin)
             (type (double-float) zero two third$ small sixth$ rbase one half
              c b a))
    (cond
     (first$
      (tagbody
        (setf zero (coerce (the f2cl-lib:integer4 0) 'double-float))
        (setf one (coerce (the f2cl-lib:integer4 1) 'double-float))
        (setf two (coerce (the f2cl-lib:integer4 2) 'double-float))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dlamc1 lbeta f2cl-lib:lt lrnd lieee1)
          (declare (ignore))
          (setf lbeta var-0)
          (setf f2cl-lib:lt var-1)
          (setf lrnd var-2)
          (setf lieee1 var-3))
        (setf b (coerce (the f2cl-lib:integer4 lbeta) 'double-float))
        (setf a (expt b (f2cl-lib:int-sub f2cl-lib:lt)))
        (setf leps a)
        (setf b (/ two 3))
        (setf half (/ one 2))
        (setf sixth$ (dlamc3 b (- half)))
        (setf third$ (dlamc3 sixth$ sixth$))
        (setf b (dlamc3 third$ (- half)))
        (setf b (dlamc3 b sixth$))
        (setf b (abs b))
        (if (< b leps)
            (setf b leps))
        (setf leps (coerce (the f2cl-lib:integer4 1) 'double-float))
       label10
        (cond
         ((and (> leps b) (> b zero)) (setf leps b)
          (setf c (dlamc3 (* half leps) (* (expt two 5) (expt leps 2))))
          (setf c (dlamc3 half (- c))) (setf b (dlamc3 half c))
          (setf c (dlamc3 half (- b))) (setf b (dlamc3 half c))
          (go label10)))
```

```
(if (< a leps)
    (setf leps a))
(setf rbase (/ one lbeta))
(setf small one)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i 3) nil)
  (tagbody (setf small (dlamc3 (* small rbase) zero)) label20))
(setf a (dlamc3 one small))
(multiple-value-bind (var-0 var-1 var-2)
    (dlamc4 ngpmin one lbeta)
  (declare (ignore))
  (when var-0 (setf ngpmin var-0))
  (when var-1 (setf one var-1))
  (when var-2 (setf lbeta var-2)))
(multiple-value-bind (var-0 var-1 var-2)
    (dlamc4 ngnmin (- one) lbeta)
  (declare (ignore var-1))
  (when var-0 (setf ngnmin var-0))
  (when var-2 (setf lbeta var-2)))
(multiple-value-bind (var-0 var-1 var-2)
    (dlamc4 gpmin a lbeta)
  (declare (ignore))
  (when var-0 (setf gpmin var-0))
  (when var-1 (setf a var-1))
  (when var-2 (setf lbeta var-2)))
(multiple-value-bind (var-0 var-1 var-2)
    (dlamc4 gnmin (- a) lbeta)
  (declare (ignore var-1))
  (when var-0 (setf gnmin var-0))
  (when var-2 (setf lbeta var-2)))
(setf ieee f2cl-lib:%false%)
(cond
 ((and (= ngpmin ngnmin) (= gpmin gnmin))
  (cond ((= ngpmin gpmin) (setf lemin ngpmin))
        ((= (f2cl-lib:int-add gpmin (f2cl-lib:int-sub ngpmin)) 3)
         (setf lemin
                 (f2cl-lib:int-add (f2cl-lib:int-sub ngpmin 1)
                                   f2cl-lib:lt))
         (setf ieee f2cl-lib:%true%))
        (t
         (setf lemin
                 (min (the f2cl-lib:integer4 ngpmin)
                      (the f2cl-lib:integer4 gpmin)))
         (setf iwarn f2cl-lib:%true%))))
 ((and (= ngpmin gpmin) (= ngnmin gnmin))
  (cond
   ((= (abs (f2cl-lib:int-add ngpmin (f2cl-lib:int-sub ngnmin))) 1)
    (setf lemin
            (max (the f2cl-lib:integer4 ngpmin)
                 (the f2cl-lib:integer4 ngnmin))))
```

```
    (t
     (setf lemin
             (min (the f2cl-lib:integer4 ngpmin)
                  (the f2cl-lib:integer4 ngnmin)))
     (setf iwarn f2cl-lib:%true%))))
  ((and
    (= (abs (f2cl-lib:int-add ngpmin (f2cl-lib:int-sub ngnmin))) 1)
    (= gpmin gnmin))
   (cond
    ((=
      (f2cl-lib:int-add gpmin
                        (f2cl-lib:int-sub
                         (min (the f2cl-lib:integer4 ngpmin)
                              (the f2cl-lib:integer4 ngnmin))))
      3)
     (setf lemin
             (f2cl-lib:int-add
              (f2cl-lib:int-sub
               (max (the f2cl-lib:integer4 ngpmin)
                    (the f2cl-lib:integer4 ngnmin))
               1)
              f2cl-lib:lt)))
    (t
     (setf lemin
             (min (the f2cl-lib:integer4 ngpmin)
                  (the f2cl-lib:integer4 ngnmin)))
     (setf iwarn f2cl-lib:%true%))))
  (t
   (setf lemin
           (min (the f2cl-lib:integer4 ngpmin)
                (the f2cl-lib:integer4 ngnmin)
                (the f2cl-lib:integer4 gpmin)
                (the f2cl-lib:integer4 gnmin)))
   (setf iwarn f2cl-lib:%true%)))
 (setf first$ f2cl-lib:%false%)
 (cond
  (iwarn (setf first$ f2cl-lib:%true%)
   (f2cl-lib:fformat 6
                     ("~%" "~%"
                      " WARNING. The value EMIN may be incorrect:-"
                      "  EMIN = " 1 (("~8D")) "~%"
                      " If, after inspection, the value EMIN looks"
                      " acceptable please comment out " "~%"
                      " the IF block as marked within the code of routine"
                      " DLAMC2," "~%"
                      " otherwise supply EMIN explicitly." "~%" "~%")
                     lemin)))
 (setf ieee (or ieee lieee1))
 (setf lrmin (coerce (the f2cl-lib:integer4 1) 'double-float))
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                      ((> i (f2cl-lib:int-add 1 (f2cl-lib:int-sub lemin)))
                       nil)
          (tagbody (setf lrmin (dlamc3 (* lrmin rbase) zero)) label30))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
            (dlamc5 lbeta f2cl-lib:lt lemin ieee lemax lrmax)
          (declare (ignore))
          (when var-0 (setf lbeta var-0))
          (when var-1 (setf f2cl-lib:lt var-1))
          (when var-2 (setf lemin var-2))
          (when var-3 (setf ieee var-3))
          (when var-4 (setf lemax var-4))
          (when var-5 (setf lrmax var-5))))))
     (setf beta lbeta)
     (setf t$ f2cl-lib:lt)
     (setf rnd lrnd)
     (setf eps leps)
     (setf emin lemin)
     (setf rmin lrmin)
     (setf emax lemax)
     (setf rmax lrmax)
     (go end_label)
    end_label
     (return (values beta t$ rnd eps emin rmin emax rmax)))))
```

———-

90.150 dlamc3

90.150.1 Axiom unit tests

— dlamc3.input —

```
)set break resume
)sys rm -f dlamc3.output
)spool dlamc3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.150.2 Axiom help page

— dlamc3.help —

```
=====================================================================
dlamc3 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

  -- LAPACK auxiliary routine (version 1.1) --
     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
     Courant Institute, Argonne National Lab, and Rice University
     October 31, 1992

     .. Scalar Arguments ..
<       DOUBLE PRECISION   A, B >
     ..

  Purpose
  =======

  DLAMC3  is intended to force  A  and  B  to be stored prior to doing
  the addition of  A  and  B ,  for use in situations where optimizers
  might hold one of these in a register.

  Arguments
  =========

  A, B    (input) DOUBLE PRECISION
          The values A and B.
```

———-

90.150.3 fortran code

— dlamc3.f —

```
**********************************************************************
*
      DOUBLE PRECISION FUNCTION DLAMC3( A, B )
*
*  -- LAPACK auxiliary routine (version 3.0) --
```

```
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1992
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   A, B
*     ..
*

* =====================================================================
*
*     .. Executable Statements ..
*
      DLAMC3 = A + B
*
      RETURN
*
*     End of DLAMC3
*
      END
*
```

———-

90.150.4 lisp code

— **dlamc3.lisp** —

```
(defun dlamc3 (a b)
  (declare (type (double-float) b a))
  (prog ((dlamc3 0.0d0))
    (declare (type (double-float) dlamc3))
    (setf dlamc3 (+ a b))
    (go end_label)
   end_label
    (return (values dlamc3 a nil))))
```

———-

90.151 dlamc4

90.151.1 Axiom unit tests

— dlamc4.input —

```
)set break resume
)sys rm -f dlamc4.output
)spool dlamc4.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.151.2 Axiom help page

— dlamc4.help —

```
====================================================================
dlamc4 examples
====================================================================

====================================================================
Man Page Details
====================================================================

  -- LAPACK auxiliary routine (version 1.1) --
     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
     Courant Institute, Argonne National Lab, and Rice University
     October 31, 1992

     .. Scalar Arguments ..
<       INTEGER            BASE, EMIN >
<       DOUBLE PRECISION   START >
     ..

  Purpose
  =======

  DLAMC4 is a service routine for DLAMC2.

  Arguments
  =========

  EMIN    (output) EMIN
          The minimum exponent before (gradual) underflow, computed by
          setting A = START and dividing by BASE until the previous A
          can not be recovered.
```

```
  START   (input) DOUBLE PRECISION
          The starting point for determining EMIN.

  BASE    (input) INTEGER
          The base of the machine.
```

———-

90.151.3 fortran code

— **dlamc4.f** —

```
***********************************************************************
*
      SUBROUTINE DLAMC4( EMIN, START, BASE )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1992
*
*     .. Scalar Arguments ..
      INTEGER            BASE, EMIN
      DOUBLE PRECISION   START
*     ..
*
* =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I
      DOUBLE PRECISION   A, B1, B2, C1, C2, D1, D2, ONE, RBASE, ZERO
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMC3
      EXTERNAL           DLAMC3
*     ..
*     .. Executable Statements ..
*
      A = START
      ONE = 1
      RBASE = ONE / BASE
      ZERO = 0
      EMIN = 1
      B1 = DLAMC3( A*RBASE, ZERO )
      C1 = A
      C2 = A
```

```
      D1 = A
      D2 = A
*+    WHILE( ( C1.EQ.A ).AND.( C2.EQ.A ).AND.
*    $       ( D1.EQ.A ).AND.( D2.EQ.A )      )LOOP
   10 CONTINUE
      IF( ( C1.EQ.A ) .AND. ( C2.EQ.A ) .AND. ( D1.EQ.A ) .AND.
     $    ( D2.EQ.A ) ) THEN
         EMIN = EMIN - 1
         A = B1
         B1 = DLAMC3( A / BASE, ZERO )
         C1 = DLAMC3( B1*BASE, ZERO )
         D1 = ZERO
         DO 20 I = 1, BASE
            D1 = D1 + B1
   20    CONTINUE
         B2 = DLAMC3( A*RBASE, ZERO )
         C2 = DLAMC3( B2 / RBASE, ZERO )
         D2 = ZERO
         DO 30 I = 1, BASE
            D2 = D2 + B2
   30    CONTINUE
         GO TO 10
      END IF
*+    END WHILE
*
      RETURN
*
*     End of DLAMC4
*
      END
*
```

———-

90.151.4 lisp code

— dlamc4.lisp —

```
(defun dlamc4 (emin start base)
  (declare (type (double-float) start)
           (type (f2cl-lib:integer4) base emin))
  (prog ((a 0.0d0) (b1 0.0d0) (b2 0.0d0) (c1 0.0d0) (c2 0.0d0) (d1 0.0d0)
         (d2 0.0d0) (one 0.0d0) (rbase 0.0d0) (zero 0.0d0) (i 0))
    (declare (type (f2cl-lib:integer4) i)
             (type (double-float) zero rbase one d2 d1 c2 c1 b2 b1 a))
    (setf a start)
    (setf one (coerce (the f2cl-lib:integer4 1) 'double-float))
```

```
    (setf rbase (/ one base))
    (setf zero (coerce (the f2cl-lib:integer4 0) 'double-float))
    (setf emin 1)
    (setf b1 (dlamc3 (* a rbase) zero))
    (setf c1 a)
    (setf c2 a)
    (setf d1 a)
    (setf d2 a)
   label10
    (cond
     ((and (= c1 a) (= c2 a) (= d1 a) (= d2 a))
      (setf emin (f2cl-lib:int-sub emin 1)) (setf a b1)
      (setf b1 (dlamc3 (/ a base) zero)) (setf c1 (dlamc3 (* b1 base) zero))
      (setf d1 zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i base) nil)
        (tagbody (setf d1 (+ d1 b1)) label20))
      (setf b2 (dlamc3 (* a rbase) zero)) (setf c2 (dlamc3 (/ b2 rbase) zero))
      (setf d2 zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i base) nil)
        (tagbody (setf d2 (+ d2 b2)) label30))
      (go label10)))
    (go end_label)
   end_label
    (return (values emin nil nil))))
```

———-

90.152 dlamc5

90.152.1 Axiom unit tests

— **dlamc5.input** —

```
)set break resume
)sys rm -f dlamc5.output
)spool dlamc5.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.152.2 Axiom help page

— dlamc5.help —

```
=====================================================================
dlamc5 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

  -- LAPACK auxiliary routine (version 1.1) --
     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
     Courant Institute, Argonne National Lab, and Rice University
     October 31, 1992

     .. Scalar Arguments ..
<       LOGICAL            IEEE >
<       INTEGER            BETA, EMAX, EMIN, P >
<       DOUBLE PRECISION   RMAX >
     ..

  Purpose
  =======

  DLAMC5 attempts to compute RMAX, the largest machine floating-point
  number, without overflow.  It assumes that EMAX + abs(EMIN) sum
  approximately to a power of 2.  It will fail on machines where this
  assumption does not hold, for example, the Cyber 205 (EMIN = -28625,
  EMAX = 28718).  It will also fail if the value supplied for EMIN is
  too large (i.e. too close to zero), probably with overflow.

  Arguments
  =========

  BETA    (input) INTEGER
          The base of floating-point arithmetic.

  P       (input) INTEGER
          The number of base BETA digits in the mantissa of a
          floating-point value.

  EMIN    (input) INTEGER
          The minimum exponent before (gradual) underflow.

  IEEE    (input) LOGICAL
          A logical flag specifying whether or not the arithmetic
          system is thought to comply with the IEEE standard.
```

```
EMAX    (output) INTEGER
        The largest exponent before overflow

RMAX    (output) DOUBLE PRECISION
        The largest machine floating-point number.
```

———-

90.152.3 fortran code

— dlamc5.f —

```
*********************************************************************
*
      SUBROUTINE DLAMC5( BETA, P, EMIN, IEEE, EMAX, RMAX )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1992
*
*     .. Scalar Arguments ..
      LOGICAL            IEEE
      INTEGER            BETA, EMAX, EMIN, P
      DOUBLE PRECISION   RMAX
*     ..
*
* ===================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      INTEGER            EXBITS, EXPSUM, I, LEXP, NBITS, TRY, UEXP
      DOUBLE PRECISION   OLDY, RECBAS, Y, Z
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMC3
      EXTERNAL           DLAMC3
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MOD
*     ..
*     .. Executable Statements ..
*
```

```
*     First compute LEXP and UEXP, two powers of 2 that bound
*     abs(EMIN). We then assume that EMAX + abs(EMIN) will sum
*     approximately to the bound that is closest to abs(EMIN).
*     (EMAX is the exponent of the required number RMAX).
*
      LEXP = 1
      EXBITS = 1
   10 CONTINUE
      TRY = LEXP*2
      IF( TRY.LE.( -EMIN ) ) THEN
         LEXP = TRY
         EXBITS = EXBITS + 1
         GO TO 10
      END IF
      IF( LEXP.EQ.-EMIN ) THEN
         UEXP = LEXP
      ELSE
         UEXP = TRY
         EXBITS = EXBITS + 1
      END IF
*
*     Now -LEXP is less than or equal to EMIN, and -UEXP is greater
*     than or equal to EMIN. EXBITS is the number of bits needed to
*     store the exponent.
*
      IF( ( UEXP+EMIN ).GT.( -LEXP-EMIN ) ) THEN
         EXPSUM = 2*LEXP
      ELSE
         EXPSUM = 2*UEXP
      END IF
*
*     EXPSUM is the exponent range, approximately equal to
*     EMAX - EMIN + 1 .
*
      EMAX = EXPSUM + EMIN - 1
      NBITS = 1 + EXBITS + P
*
*     NBITS is the total number of bits needed to store a
*     floating-point number.
*
      IF( ( MOD( NBITS, 2 ).EQ.1 ) .AND. ( BETA.EQ.2 ) ) THEN
*
*        Either there are an odd number of bits used to store a
*        floating-point number, which is unlikely, or some bits are
*        not used in the representation of numbers, which is possible,
*        (e.g. Cray machines) or the mantissa has an implicit bit,
*        (e.g. IEEE machines, Dec Vax machines), which is perhaps the
*        most likely. We have to assume the last alternative.
*        If this is true, then we need to reduce EMAX by one because
*        there must be some way of representing zero in an implicit-bit
```

```
*        system. On machines like Cray, we are reducing EMAX by one
*        unnecessarily.
*
         EMAX = EMAX - 1
      END IF
*
      IF( IEEE ) THEN
*
*        Assume we are on an IEEE machine which reserves one exponent
*        for infinity and NaN.
*
         EMAX = EMAX - 1
      END IF
*
*     Now create RMAX, the largest machine number, which should
*     be equal to (1.0 - BETA**(-P)) * BETA**EMAX .
*
*     First compute 1.0 - BETA**(-P), being careful that the
*     result is less than 1.0 .
*
      RECBAS = ONE / BETA
      Z = BETA - ONE
      Y = ZERO
      DO 20 I = 1, P
         Z = Z*RECBAS
         IF( Y.LT.ONE )
     $      OLDY = Y
         Y = DLAMC3( Y, Z )
   20 CONTINUE
      IF( Y.GE.ONE )
     $   Y = OLDY
*
*     Now multiply by BETA**EMAX to get RMAX.
*
      DO 30 I = 1, EMAX
         Y = DLAMC3( Y*BETA, ZERO )
   30 CONTINUE
*
      RMAX = Y
      RETURN
*
*     End of DLAMC5
*
      END
*
*
```

90.152.4 lisp code

— dlamc5.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dlamc5 (beta p emin ieee emax rmax)
    (declare (type (double-float) rmax)
             (type f2cl-lib:logical ieee)
             (type (f2cl-lib:integer4) emax emin p beta))
    (prog ((oldy 0.0d0) (recbas 0.0d0) (y 0.0d0) (z 0.0d0) (exbits 0)
           (expsum 0) (i 0) (lexp 0) (nbits 0) (try 0) (uexp 0))
      (declare (type (double-float) oldy recbas y z)
               (type (f2cl-lib:integer4) exbits expsum i lexp nbits try uexp))
      (setf lexp 1)
      (setf exbits 1)
     label10
      (setf try (f2cl-lib:int-mul lexp 2))
      (cond
       ((<= try (f2cl-lib:int-sub emin)) (setf lexp try)
        (setf exbits (f2cl-lib:int-add exbits 1)) (go label10)))
      (cond ((= lexp (f2cl-lib:int-sub emin)) (setf uexp lexp))
            (t (setf uexp try) (setf exbits (f2cl-lib:int-add exbits 1))))
      (cond
       ((> (f2cl-lib:int-add uexp emin)
           (f2cl-lib:int-add (f2cl-lib:int-sub lexp) (f2cl-lib:int-sub emin)))
        (setf expsum (f2cl-lib:int-mul 2 lexp)))
       (t (setf expsum (f2cl-lib:int-mul 2 uexp))))
      (setf emax (f2cl-lib:int-sub (f2cl-lib:int-add expsum emin) 1))
      (setf nbits (f2cl-lib:int-add 1 exbits p))
      (cond
       ((and (= (mod nbits 2) 1) (= beta 2))
        (setf emax (f2cl-lib:int-sub emax 1))))
      (cond (ieee (setf emax (f2cl-lib:int-sub emax 1))))
      (setf recbas (/ one beta))
      (setf z (- beta one))
      (setf y zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i p) nil)
        (tagbody
          (setf z (* z recbas))
          (if (< y one)
              (setf oldy y))
          (setf y
                  (multiple-value-bind (ret-val var-0 var-1)
                      (dlamc3 y z)
                    (declare (ignore var-1))
```

```
                    (setf y var-0)
                    ret-val))
         label20))
      (if (>= y one)
          (setf y oldy))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i emax) nil)
        (tagbody (setf y (dlamc3 (* y beta) zero)) label30))
      (setf rmax y)
      (go end_label)
     end_label
      (return (values beta nil emin ieee emax rmax)))))
```

———-

90.153 dlamrg

90.153.1 Axiom unit tests

— **dlamrg.input** —

```
)set break resume
)sys rm -f dlamrg.output
)spool dlamrg.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.153.2 Axiom help page

— **dlamrg.help** —

```
==========================================================================
dlamrg examples
==========================================================================

==========================================================================
Man Page Details
```

```
=====================================================================

NAME
       DLAMRG  -  create a permutation list which will merge the elements of A
       (which is composed of two independently sorted sets) into a single  set
       which is sorted in ascending order

SYNOPSIS
       SUBROUTINE DLAMRG( N1, N2, A, DTRD1, DTRD2, INDEX )

           INTEGER        DTRD1, DTRD2, N1, N2

           INTEGER        INDEX( * )

           DOUBLE         PRECISION A( * )

 Purpose
 =======

  DLAMRG will create a permutation list which will merge the elements
  of A (which is composed of two independently sorted sets) into a
  single set which is sorted in ascending order.

 Arguments
 =========

  N1     (input) INTEGER
  N2     (input) INTEGER
         These arguements contain the respective lengths of the two
         sorted lists to be merged.

  A      (input) DOUBLE PRECISION array, dimension (N1+N2)
         The first N1 elements of A contain a list of numbers which
         are sorted in either ascending or descending order.  Likewise
         for the final N2 elements.

  DTRD1  (input) INTEGER
  DTRD2  (input) INTEGER
         These are the strides to be taken through the array A.
         Allowable strides are 1 and -1.  They indicate whether a
         subset of A is sorted in ascending (DTRDx = 1) or descending
         (DTRDx = -1) order.

  INDX  (output) INTEGER array, dimension (N1+N2)
         On exit this array will contain a permutation such that
         if B( I ) = A( INDX( I ) ) for I=1,N1+N2, then B will be
         sorted in ascending order.

              _______-
```

90.153.3 fortran code

— dlamrg.f —

```
      SUBROUTINE DLAMRG( N1, N2, A, DTRD1, DTRD2, INDX )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     September 30, 1994
*
*     .. Scalar Arguments ..
      INTEGER            DTRD1, DTRD2, N1, N2
*     ..
*     .. Array Arguments ..
      INTEGER            INDX( * )
      DOUBLE PRECISION   A( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, IND1, IND2, N1SV, N2SV
*     ..
*     .. Executable Statements ..
*
      N1SV = N1
      N2SV = N2
      IF( DTRD1.GT.0 ) THEN
         IND1 = 1
      ELSE
         IND1 = N1
      END IF
      IF( DTRD2.GT.0 ) THEN
         IND2 = 1 + N1
      ELSE
         IND2 = N1 + N2
      END IF
      I = 1
*     while ( (N1SV > 0) & (N2SV > 0) )
   10 CONTINUE
      IF( N1SV.GT.0 .AND. N2SV.GT.0 ) THEN
         IF( A( IND1 ).LE.A( IND2 ) ) THEN
            INDX( I ) = IND1
            I = I + 1
            IND1 = IND1 + DTRD1
            N1SV = N1SV - 1
         ELSE
            INDX( I ) = IND2
```

```
            I = I + 1
            IND2 = IND2 + DTRD2
            N2SV = N2SV - 1
         END IF
         GO TO 10
      END IF
*     end while
      IF( N1SV.EQ.0 ) THEN
         DO 20 N1SV = 1, N2SV
            INDX( I ) = IND2
            I = I + 1
            IND2 = IND2 + DTRD2
   20    CONTINUE
      ELSE
*     N2SV .EQ. 0
         DO 30 N2SV = 1, N1SV
            INDX( I ) = IND1
            I = I + 1
            IND1 = IND1 + DTRD1
   30    CONTINUE
      END IF
*
      RETURN
*
*     End of DLAMRG
*
      END
```

———-

90.153.4 lisp code

— dlamrg.lisp —

```
(defun dlamrg (n1 n2 a dtrd1 dtrd2 index)
  (declare (type (array f2cl-lib:integer4 (*)) index)
           (type (array double-float (*)) a)
           (type (f2cl-lib:integer4) dtrd2 dtrd1 n2 n1))
  (f2cl-lib:with-multi-array-data
      ((a double-float a-%data% a-%offset%)
       (index f2cl-lib:integer4 index-%data% index-%offset%))
    (prog ((i 0) (ind1 0) (ind2 0) (n1sv 0) (n2sv 0))
      (declare (type (f2cl-lib:integer4) n2sv n1sv ind2 ind1 i))
      (setf n1sv n1)
      (setf n2sv n2)
      (cond ((> dtrd1 0) (setf ind1 1)) (t (setf ind1 n1)))
      (cond ((> dtrd2 0) (setf ind2 (f2cl-lib:int-add 1 n1)))
```

```
        (t (setf ind2 (f2cl-lib:int-add n1 n2))))
  (setf i 1)
 label10
  (cond
   ((and (> n1sv 0) (> n2sv 0))
    (cond
     ((<= (f2cl-lib:fref a (ind1) ((1 *)))
          (f2cl-lib:fref a (ind2) ((1 *))))
      (setf (f2cl-lib:fref index-%data% (i) ((1 *)) index-%offset%) ind1)
      (setf i (f2cl-lib:int-add i 1))
      (setf ind1 (f2cl-lib:int-add ind1 dtrd1))
      (setf n1sv (f2cl-lib:int-sub n1sv 1)))
     (t (setf (f2cl-lib:fref index-%data% (i) ((1 *)) index-%offset%) ind2)
      (setf i (f2cl-lib:int-add i 1))
      (setf ind2 (f2cl-lib:int-add ind2 dtrd2))
      (setf n2sv (f2cl-lib:int-sub n2sv 1))))
    (go label10)))
  (cond
   ((= n1sv 0)
    (f2cl-lib:fdo (n1sv 1 (f2cl-lib:int-add n1sv 1))
                  ((> n1sv n2sv) nil)
      (tagbody
        (setf (f2cl-lib:fref index-%data% (i) ((1 *)) index-%offset%) ind2)
        (setf i (f2cl-lib:int-add i 1))
        (setf ind2 (f2cl-lib:int-add ind2 dtrd2))
       label20)))
   (t
    (f2cl-lib:fdo (n2sv 1 (f2cl-lib:int-add n2sv 1))
                  ((> n2sv n1sv) nil)
      (tagbody
        (setf (f2cl-lib:fref index-%data% (i) ((1 *)) index-%offset%) ind1)
        (setf i (f2cl-lib:int-add i 1))
        (setf ind1 (f2cl-lib:int-add ind1 dtrd1))
       label30))))
  (go end_label)
 end_label
  (return (values nil nil nil nil nil nil)))))
```

———-

90.154 dlange

90.154.1 Axiom unit tests

— dlange.input —

```
)set break resume
)sys rm -f dlange.output
)spool dlange.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

90.154.2 Axiom help page

— dlange.help —

```
=====================================================================
dlange examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

NAME
       DLANGE  -  the  value  of  the  one norm, or the Frobenius norm, or the
       infinity norm, or the element of  largest  absolute  value  of  a  real
       matrix A

SYNOPSIS
       DOUBLE PRECISION FUNCTION DLANGE( NORM, M, N, A, LDA, WORK )

           CHARACTER    NORM

           INTEGER      LDA, M, N

           DOUBLE       PRECISION A( LDA, * ), WORK( * )

PURPOSE
       DLANGE   returns  the value of the one norm,  or the Frobenius norm, or
       the  infinity norm,  or the  element of  largest absolute value   of  a
       real matrix A.


DESCRIPTION
       DLANGE returns the value

          DLANGE = ( max(abs(A(i,j))), NORM = 'M' or 'm'
```

```
                 (
                 ( norm1(A),         NORM = '1', 'O' or 'o'
                 (
                 ( normI(A),         NORM = 'I' or 'i'
                 (
                 ( normF(A),         NORM = 'F', 'f', 'E' or 'e'

       where  norm1  denotes the  one norm of a matrix (maximum column sum),
       normI  denotes the  infinity norm  of a matrix  (maximum row  sum)  and
       normF  denotes  the  Frobenius norm of a matrix (square root of sum of
       squares).  Note that  max(abs(A(i,j)))  is  not  a  consistent  matrix
       norm.


ARGUMENTS
       NORM    (input) CHARACTER*1
               Specifies  the  value  to  be  returned  in DLANGE as described
               above.

       M       (input) INTEGER
               The number of rows of the matrix A.  M  >=  0.  When  M  =  0,
               DLANGE is set to zero.

       N       (input) INTEGER
               The  number  of  columns of the matrix A.  N >= 0.  When N = 0,
               DLANGE is set to zero.

       A       (input) DOUBLE PRECISION array, dimension (LDA,N)
               The m by n matrix A.

       LDA     (input) INTEGER
               The leading dimension of the array A.  LDA >= max(M,1).

       WORK    (workspace) DOUBLE PRECISION array, dimension (MAX(1,LWORK)),
               where LWORK >= M when NORM = 'I'; otherwise, WORK is not refer-
               enced.
```

———-

90.154.3 fortran code

— dlange.f —

```
      DOUBLE PRECISION FUNCTION DLANGE( NORM, M, N, A, LDA, WORK )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
```

```
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1992
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), WORK( * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   SCALE, SUM, VALUE
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASSQ
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      IF( MIN( M, N ).EQ.0 ) THEN
         VALUE = ZERO
      ELSE IF( LSAME( NORM, 'M' ) ) THEN
*
*        Find max(abs(A(i,j))).
*
         VALUE = ZERO
         DO 20 J = 1, N
            DO 10 I = 1, M
               VALUE = MAX( VALUE, ABS( A( I, J ) ) )
   10       CONTINUE
   20    CONTINUE
      ELSE IF( ( LSAME( NORM, 'O' ) ) .OR. ( NORM.EQ.'1' ) ) THEN
*
*        Find norm1(A).
*
         VALUE = ZERO
```

```
         DO 40 J = 1, N
            SUM = ZERO
            DO 30 I = 1, M
               SUM = SUM + ABS( A( I, J ) )
   30       CONTINUE
            VALUE = MAX( VALUE, SUM )
   40    CONTINUE
      ELSE IF( LSAME( NORM, 'I' ) ) THEN
*
*        Find normI(A).
*
         DO 50 I = 1, M
            WORK( I ) = ZERO
   50    CONTINUE
         DO 70 J = 1, N
            DO 60 I = 1, M
               WORK( I ) = WORK( I ) + ABS( A( I, J ) )
   60       CONTINUE
   70    CONTINUE
         VALUE = ZERO
         DO 80 I = 1, M
            VALUE = MAX( VALUE, WORK( I ) )
   80    CONTINUE
      ELSE IF( ( LSAME( NORM, 'F' ) ) .OR. ( LSAME( NORM, 'E' ) ) ) THEN
*
*        Find normF(A).
*
         SCALE = ZERO
         SUM = ONE
         DO 90 J = 1, N
            CALL DLASSQ( M, A( 1, J ), 1, SCALE, SUM )
   90    CONTINUE
         VALUE = SCALE*SQRT( SUM )
      END IF
*
      DLANGE = VALUE
      RETURN
*
*     End of DLANGE
*
      END
```

———-

90.154.4 lisp code

— **dlange.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dlange (norm m n a lda work)
    (declare (type (array double-float (*)) work a)
             (type (f2cl-lib:integer4) lda n m)
             (type (string *) norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (a double-float a-%data% a-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((scale 0.0d0) (sum 0.0d0) (value 0.0d0) (temp 0.0d0) (i 0) (j 0)
             (dlange 0.0d0))
        (declare (type (f2cl-lib:integer4) i j)
                 (type (double-float) scale sum value temp dlange))
        (cond
         ((= (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) 0)
          (setf value zero))
         ((multiple-value-bind (ret-val var-0 var-1)
              (lsame norm "M")
            (declare (ignore var-1))
            (when var-0 (setf norm var-0))
            ret-val)
          (setf value zero)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i m) nil)
                (tagbody
                  (setf temp
                          (abs
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%)))
                  (if (or (< value temp)
                          (multiple-value-bind (ret-val var-0)
                              (disnan temp)
                            (declare (ignore))
                            (setf temp var-0)
                            ret-val))
                      (setf value temp))
                label10))
            label20)))
         ((or
           (multiple-value-bind (ret-val var-0 var-1)
               (lsame norm "O")
             (declare (ignore var-1))
             (when var-0 (setf norm var-0))
             ret-val)
```

```
  (f2cl-lib:fstring-= norm "1"))
 (setf value zero)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf sum zero)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i m) nil)
       (tagbody
         (setf sum
                 (+ sum
                    (abs
                     (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%))))
        label30))
     (if (or (< value sum)
             (multiple-value-bind (ret-val var-0)
                 (disnan sum)
               (declare (ignore))
               (setf sum var-0)
               ret-val))
         (setf value sum))
    label40)))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame norm "I")
   (declare (ignore var-1))
   (when var-0 (setf norm var-0))
   ret-val)
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i m) nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) zero)
    label50))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i m) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (+
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)
                  (abs
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%))))
        label60))
    label70))
 (setf value zero)
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                  ((> i m) nil)
      (tagbody
        (setf temp (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
        (if (or (< value temp)
                (multiple-value-bind (ret-val var-0)
                    (disnan temp)
                  (declare (ignore))
                  (setf temp var-0)
                  ret-val))
            (setf value temp))
      label80)))
  ((or
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame norm "F")
      (declare (ignore var-1))
      (when var-0 (setf norm var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame norm "E")
      (declare (ignore var-1))
      (when var-0 (setf norm var-0))
      ret-val))
   (setf scale zero) (setf sum one)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (dlassq m
            (f2cl-lib:array-slice a-%data% double-float (1 j)
                                  ((1 lda) (1 *)) a-%offset%)
            1 scale sum)
         (declare (ignore var-1 var-2))
         (when var-0 (setf m var-0))
         (when var-3 (setf scale var-3))
         (when var-4 (setf sum var-4)))
      label90))
   (setf value (* scale (f2cl-lib:fsqrt sum)))))
 (setf dlange value)
 (go end_label)
end_label
 (return (values dlange norm m nil nil nil nil))))))
```

———-

90.155 dlanhs

90.155.1 Axiom unit tests

— dlanhs.input —

```
)set break resume
)sys rm -f dlanhs.output
)spool dlanhs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.155.2 Axiom help page

— dlanhs.help —

```
======================================================================
dlanhs examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DLANHS  -  the  value  of  the  one norm, or the Frobenius norm, or the
       infinity norm, or the element of largest absolute value of a Hessenberg
       matrix A

SYNOPSIS
       DOUBLE PRECISION FUNCTION DLANHS( NORM, N, A, LDA, WORK )

           CHARACTER    NORM

           INTEGER      LDA, N

           DOUBLE       PRECISION A( LDA, * ), WORK( * )

PURPOSE
       DLANHS   returns  the value of the one norm,  or the Frobenius norm, or
```

```
       the  infinity norm,  or the  element of  largest absolute value   of  a
       Hessenberg matrix A.


DESCRIPTION
       DLANHS returns the value

          DLANHS = ( max(abs(A(i,j))), NORM = 'M' or 'm'
                   (
                   ( norm1(A),         NORM = '1', 'O' or 'o'
                   (
                   ( normI(A),         NORM = 'I' or 'i'
                   (
                   ( normF(A),         NORM = 'F', 'f', 'E' or 'e'

       where   norm1   denotes the  one norm of a matrix (maximum column sum),
       normI  denotes the  infinity norm  of a matrix  (maximum row  sum)  and
       normF   denotes  the  Frobenius norm of a matrix (square root of sum of
       squares).  Note that  max(abs(A(i,j)))   is  not  a  consistent  matrix
       norm.


ARGUMENTS
       NORM    (input) CHARACTER*1
               Specifies  the  value  to  be  returned  in DLANHS as described
               above.

       N       (input) INTEGER
               The order of the matrix A.  N >= 0.  When N = 0, DLANHS is  set
               to zero.

       A       (input) DOUBLE PRECISION array, dimension (LDA,N)
               The  n  by n upper Hessenberg matrix A; the part of A below the
               first sub-diagonal is not referenced.

       LDA     (input) INTEGER
               The leading dimension of the array A.  LDA >= max(N,1).

       WORK    (workspace) DOUBLE PRECISION array, dimension (MAX(1,LWORK)),
               where LWORK >= N when NORM = 'I'; otherwise, WORK is not refer-
               enced.
```

———-

90.155.3 fortran code

— dlanhs.f —

```
      DOUBLE PRECISION FUNCTION DLANHS( NORM, N, A, LDA, WORK )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1992
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            LDA, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), WORK( * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   SCALE, SUM, VALUE
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASSQ
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      IF( N.EQ.0 ) THEN
         VALUE = ZERO
      ELSE IF( LSAME( NORM, 'M' ) ) THEN
*
*        Find max(abs(A(i,j))).
*
         VALUE = ZERO
         DO 20 J = 1, N
            DO 10 I = 1, MIN( N, J+1 )
               VALUE = MAX( VALUE, ABS( A( I, J ) ) )
   10       CONTINUE
   20    CONTINUE
      ELSE IF( ( LSAME( NORM, 'O' ) ) .OR. ( NORM.EQ.'1' ) ) THEN
```

```
*
*       Find norm1(A).
*
        VALUE = ZERO
        DO 40 J = 1, N
           SUM = ZERO
           DO 30 I = 1, MIN( N, J+1 )
              SUM = SUM + ABS( A( I, J ) )
   30      CONTINUE
           VALUE = MAX( VALUE, SUM )
   40   CONTINUE
      ELSE IF( LSAME( NORM, 'I' ) ) THEN
*
*       Find normI(A).
*
        DO 50 I = 1, N
           WORK( I ) = ZERO
   50   CONTINUE
        DO 70 J = 1, N
           DO 60 I = 1, MIN( N, J+1 )
              WORK( I ) = WORK( I ) + ABS( A( I, J ) )
   60      CONTINUE
   70   CONTINUE
        VALUE = ZERO
        DO 80 I = 1, N
           VALUE = MAX( VALUE, WORK( I ) )
   80   CONTINUE
      ELSE IF( ( LSAME( NORM, 'F' ) ) .OR. ( LSAME( NORM, 'E' ) ) ) THEN
*
*       Find normF(A).
*
        SCALE = ZERO
        SUM = ONE
        DO 90 J = 1, N
           CALL DLASSQ( MIN( N, J+1 ), A( 1, J ), 1, SCALE, SUM )
   90   CONTINUE
        VALUE = SCALE*SQRT( SUM )
      END IF
*
      DLANHS = VALUE
      RETURN
*
*     End of DLANHS
*
      END
```

———-

90.155.4 lisp code

— dlanhs.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dlanhs (norm n a lda work)
    (declare (type (array double-float (*)) work a)
             (type (f2cl-lib:integer4) lda n)
             (type (string *) norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (a double-float a-%data% a-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((scale 0.0d0) (sum 0.0d0) (value 0.0d0) (i 0) (j 0)
             (dlanhs 0.0d0))
        (declare (type (f2cl-lib:integer4) i j)
                 (type (double-float) scale sum value dlanhs))
        (cond ((= n 0) (setf value zero))
              ((multiple-value-bind (ret-val var-0 var-1)
                   (lsame norm "M")
                 (declare (ignore var-1))
                 (when var-0 (setf norm var-0))
                 ret-val)
               (setf value zero)
               (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                             ((> j n) nil)
                 (tagbody
                   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                 ((> i
                                     (min (the f2cl-lib:integer4 n)
                                          (the f2cl-lib:integer4
                                               (f2cl-lib:int-add j 1))))
                                  nil)
                     (tagbody
                       (setf sum
                               (abs
                                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                               a-%offset%)))
                       (if (or (< value sum)
                               (multiple-value-bind (ret-val var-0)
                                   (disnan sum)
                                 (declare (ignore))
                                 (setf sum var-0)
                                 ret-val))
                           (setf value sum))
                      label10))
```

```
    label20)))
((or
  (multiple-value-bind (ret-val var-0 var-1)
      (lsame norm "O")
    (declare (ignore var-1))
    (when var-0 (setf norm var-0))
    ret-val)
  (f2cl-lib:fstring-= norm "1"))
 (setf value zero)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf sum zero)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i
                       (min (the f2cl-lib:integer4 n)
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add j 1))))
                    nil)
       (tagbody
         (setf sum
                 (+ sum
                    (abs
                     (f2cl-lib:fref a-%data% (i j)
                                    ((1 lda) (1 *))
                                    a-%offset%))))
        label30))
     (if (or (< value sum)
             (multiple-value-bind (ret-val var-0)
                 (disnan sum)
               (declare (ignore))
               (setf sum var-0)
               ret-val))
         (setf value sum))
    label40)))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame norm "I")
   (declare (ignore var-1))
   (when var-0 (setf norm var-0))
   ret-val)
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i n) nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
             zero)
    label50))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                    ((> i
                        (min (the f2cl-lib:integer4 n)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add j 1))))
                     nil)
            (tagbody
              (setf (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%)
                    (+
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)
                     (abs
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))))
             label60))
        label70))
      (setf value zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf sum
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%))
          (if (or (< value sum)
                  (multiple-value-bind (ret-val var-0)
                      (disnan sum)
                    (declare (ignore))
                    (setf sum var-0)
                    ret-val))
              (setf value sum))
        label80)))
    ((or
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame norm "F")
        (declare (ignore var-1))
        (when var-0 (setf norm var-0))
        ret-val)
      (multiple-value-bind (ret-val var-0 var-1)
          (lsame norm "E")
        (declare (ignore var-1))
        (when var-0 (setf norm var-0))
        ret-val))
      (setf scale zero) (setf sum one)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dlassq
               (min (the f2cl-lib:integer4 n)
                    (the f2cl-lib:integer4 (f2cl-lib:int-add j 1)))
```

```
                      (f2cl-lib:array-slice a-%data% double-float (1 j)
                                            ((1 lda) (1 *)) a-%offset%)
                      1 scale sum)
                    (declare (ignore var-0 var-1 var-2))
                    (when var-3 (setf scale var-3))
                    (when var-4 (setf sum var-4)))
                 label90))
              (setf value (* scale (f2cl-lib:fsqrt sum)))))
       (setf dlanhs value)
       (go end_label)
      end_label
       (return (values dlanhs norm nil nil nil nil))))))
```

———-

90.156 dlanst

90.156.1 Axiom unit tests

— **dlanst.input** —

```
)set break resume
)sys rm -f dlanst.output
)spool dlanst.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.156.2 Axiom help page

— **dlanst.help** —

```
=====================================================================
dlanst examples
=====================================================================

=====================================================================
Man Page Details
```

```
=====================================================================

NAME
       DLANST  -  the  value  of  the  one norm, or the Frobenius norm, or the
       infinity norm, or the element of largest absolute value of a real  sym-
       metric tridiagonal matrix A

SYNOPSIS
       DOUBLE PRECISION FUNCTION DLANST( NORM, N, D, E )

           CHARACTER    NORM

           INTEGER      N

           DOUBLE       PRECISION D( * ), E( * )

PURPOSE
       DLANST   returns  the value of the one norm,  or the Frobenius norm, or
       the  infinity norm,  or the  element of  largest absolute value   of  a
       real symmetric tridiagonal matrix A.


DESCRIPTION
       DLANST returns the value

          DLANST = ( max(abs(A(i,j))), NORM = 'M' or 'm'
                   (
                   ( norm1(A),         NORM = '1', 'O' or 'o'
                   (
                   ( normI(A),         NORM = 'I' or 'i'
                   (
                   ( normF(A),         NORM = 'F', 'f', 'E' or 'e'

       where   norm1   denotes the  one norm of a matrix (maximum column sum),
       normI  denotes the  infinity norm  of a matrix  (maximum row  sum)  and
       normF   denotes  the  Frobenius norm of a matrix (square root of sum of
       squares).  Note that  max(abs(A(i,j)))   is  not  a  consistent  matrix
       norm.


ARGUMENTS
       NORM    (input) CHARACTER*1
               Specifies  the  value  to  be  returned  in DLANST as described
               above.

       N       (input) INTEGER
               The order of the matrix A.  N >= 0.  When N = 0, DLANST is  set
               to zero.

       D       (input) DOUBLE PRECISION array, dimension (N)
```

```
              The diagonal elements of A.

       E      (input) DOUBLE PRECISION array, dimension (N-1)
              The (n-1) sub-diagonal or super-diagonal elements of A.
```

———-

90.156.3 fortran code

— **dlanst.f** —

```
      DOUBLE PRECISION FUNCTION DLANST( NORM, N, D, E )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     February 29, 1992
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), E( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      DOUBLE PRECISION   ANORM, SCALE, SUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASSQ
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
```

```
*
      IF( N.LE.0 ) THEN
         ANORM = ZERO
      ELSE IF( LSAME( NORM, 'M' ) ) THEN
*
*        Find max(abs(A(i,j))).
*
         ANORM = ABS( D( N ) )
         DO 10 I = 1, N - 1
            ANORM = MAX( ANORM, ABS( D( I ) ) )
            ANORM = MAX( ANORM, ABS( E( I ) ) )
   10    CONTINUE
      ELSE IF( LSAME( NORM, 'O' ) .OR. NORM.EQ.'1' .OR.
     $         LSAME( NORM, 'I' ) ) THEN
*
*        Find norm1(A).
*
         IF( N.EQ.1 ) THEN
            ANORM = ABS( D( 1 ) )
         ELSE
            ANORM = MAX( ABS( D( 1 ) )+ABS( E( 1 ) ),
     $              ABS( E( N-1 ) )+ABS( D( N ) ) )
            DO 20 I = 2, N - 1
               ANORM = MAX( ANORM, ABS( D( I ) )+ABS( E( I ) )+
     $                 ABS( E( I-1 ) ) )
   20       CONTINUE
         END IF
      ELSE IF( ( LSAME( NORM, 'F' ) ) .OR. ( LSAME( NORM, 'E' ) ) ) THEN
*
*        Find normF(A).
*
         SCALE = ZERO
         SUM = ONE
         IF( N.GT.1 ) THEN
            CALL DLASSQ( N-1, E, 1, SCALE, SUM )
            SUM = 2*SUM
         END IF
         CALL DLASSQ( N, D, 1, SCALE, SUM )
         ANORM = SCALE*SQRT( SUM )
      END IF
*
      DLANST = ANORM
      RETURN
*
*     End of DLANST
*
      END
```

———-

90.156.4 lisp code

— dlanst.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dlanst (norm n d e)
    (declare (type (array double-float (*)) e d)
             (type (f2cl-lib:integer4) n)
             (type (string *) norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%))
      (prog ((anorm 0.0d0) (scale 0.0d0) (sum 0.0d0) (i 0) (dlanst 0.0d0))
        (declare (type (f2cl-lib:integer4) i)
                 (type (double-float) anorm scale sum dlanst))
        (cond ((<= n 0) (setf anorm zero))
              ((multiple-value-bind (ret-val var-0 var-1)
                   (lsame norm "M")
                 (declare (ignore var-1))
                 (when var-0 (setf norm var-0))
                 ret-val)
               (setf anorm
                       (abs (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                              nil)
                 (tagbody
                   (setf sum
                           (abs
                            (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
                   (if (or (< anorm sum)
                           (multiple-value-bind (ret-val var-0)
                               (disnan sum)
                             (declare (ignore))
                             (setf sum var-0)
                             ret-val))
                       (setf anorm sum))
                   (setf sum
                           (abs
                            (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)))
                   (if (or (< anorm sum)
                           (multiple-value-bind (ret-val var-0)
                               (disnan sum)
                             (declare (ignore))
                             (setf sum var-0)
```

```
                ret-val))
            (setf anorm sum))
      label10)))
  ((or
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame norm "O")
      (declare (ignore var-1))
      (when var-0 (setf norm var-0))
      ret-val)
    (f2cl-lib:fstring-= norm "1")
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame norm "I")
      (declare (ignore var-1))
      (when var-0 (setf norm var-0))
      ret-val))
   (cond
    ((= n 1)
     (setf anorm
             (abs
              (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))))
    (t
     (setf anorm
             (+
              (abs (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
              (abs
               (f2cl-lib:fref e-%data% (1) ((1 *)) e-%offset%))))
     (setf sum
             (+
              (abs
               (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub n 1))
                              ((1 *)) e-%offset%))
              (abs
               (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%))))
     (if (or (< anorm sum)
             (multiple-value-bind (ret-val var-0)
                 (disnan sum)
               (declare (ignore))
               (setf sum var-0)
               ret-val))
         (setf anorm sum))
     (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                   ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                    nil)
       (tagbody
         (setf sum
                 (+
                  (abs
                   (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
                  (abs
                   (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))
```

```
                          (abs
                           (f2cl-lib:fref e-%data% ((f2cl-lib:int-sub i 1))
                                          ((1 *)) e-%offset%))))
                 (if (or (< anorm sum)
                         (multiple-value-bind (ret-val var-0)
                             (disnan sum)
                           (declare (ignore))
                           (setf sum var-0)
                           ret-val))
                     (setf anorm sum))
               label20)))))
          ((or
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame norm "F")
              (declare (ignore var-1))
              (when var-0 (setf norm var-0))
              ret-val)
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame norm "E")
              (declare (ignore var-1))
              (when var-0 (setf norm var-0))
              ret-val))
           (setf scale zero) (setf sum one)
           (cond
            ((> n 1)
             (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                 (dlassq (f2cl-lib:int-sub n 1) e 1 scale sum)
               (declare (ignore var-0 var-1 var-2))
               (when var-3 (setf scale var-3))
               (when var-4 (setf sum var-4)))
             (setf sum (* 2 sum))))
           (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
               (dlassq n d 1 scale sum)
             (declare (ignore var-1 var-2))
             (when var-0 (setf n var-0))
             (when var-3 (setf scale var-3))
             (when var-4 (setf sum var-4)))
           (setf anorm (* scale (f2cl-lib:fsqrt sum)))))
  (setf dlanst anorm)
  (go end_label)
end_label
  (return (values dlanst norm n nil nil))))))
```

———-

90.157 dlanv2

90.157.1 Axiom unit tests

— dlanv2.input —

```
)set break resume
)sys rm -f dlanv2.output
)spool dlanv2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.157.2 Axiom help page

— dlanv2.help —

```
======================================================================
dlanv2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DLANV2  -  the Schur factorization of a real 2-by-2 nonsymmetric matrix
       in standard form

SYNOPSIS
       SUBROUTINE DLANV2( A, B, C, D, RT1R, RT1I, RT2R, RT2I, CS, SN )

           DOUBLE         PRECISION A, B, C, CS, D, RT1I, RT1R, RT2I, RT2R, SN

 Purpose
 =======

  DLANV2 computes the Schur factorization of a real 2-by-2 nonsymmetric
  matrix in standard form:

       [ A  B ] = [ CS -SN ] [ AA  BB ] [ CS  SN ]
```

```
      [ C  D ]   [ SN  CS ] [ CC  DD ] [-SN  CS ]

 where either
 1) CC = 0 so that AA and DD are real eigenvalues of the matrix, or
 2) AA = DD and BB*CC < 0, so that AA + or - sqrt(BB*CC) are complex
 conjugate eigenvalues.

Arguments
=========

 A       (input/output) DOUBLE PRECISION
 B       (input/output) DOUBLE PRECISION
 C       (input/output) DOUBLE PRECISION
 D       (input/output) DOUBLE PRECISION
         On entry, the elements of the input matrix.
         On exit, they are overwritten by the elements of the
         standardised Schur form.

 RT1R    (output) DOUBLE PRECISION
 RT1I    (output) DOUBLE PRECISION
 RT2R    (output) DOUBLE PRECISION
 RT2I    (output) DOUBLE PRECISION
         The real and imaginary parts of the eigenvalues. If the
         eigenvalues are a complex conjugate pair, RT1I > 0.

 CS      (output) DOUBLE PRECISION
 SN      (output) DOUBLE PRECISION
         Parameters of the rotation matrix.

 Further Details
 ===============

 Modified by V. Sima, Research Institute for Informatics, Bucharest,
 Romania, to reduce the risk of cancellation errors,
 when computing real eigenvalues, and to ensure, if possible, that
 abs(RT1R) >= abs(RT2R).
```

———-

90.157.3 fortran code

— dlanv2.f —

```
      SUBROUTINE DLANV2( A, B, C, D, RT1R, RT1I, RT2R, RT2I, CS, SN )
*
*  -- LAPACK driver routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
```

```
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   A, B, C, CS, D, RT1I, RT1R, RT2I, RT2R, SN
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, HALF, ONE
      PARAMETER          ( ZERO = 0.0D+0, HALF = 0.5D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   MULTPL
      PARAMETER          ( MULTPL = 4.0D+0 )
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION   AA, BB, BCMAX, BCMIS, CC, CS1, DD, EPS, P, SAB,
     $                   SAC, SCALE, SIGMA, SN1, TAU, TEMP, Z
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH, DLAPY2
      EXTERNAL           DLAMCH, DLAPY2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SIGN, SQRT
*     ..
*     .. Executable Statements ..
*
      EPS = DLAMCH( 'P' )
      IF( C.EQ.ZERO ) THEN
         CS = ONE
         SN = ZERO
         GO TO 10
*
      ELSE IF( B.EQ.ZERO ) THEN
*
*        Swap rows and columns
*
         CS = ZERO
         SN = ONE
         TEMP = D
         D = A
         A = TEMP
         B = -C
         C = ZERO
         GO TO 10
      ELSE IF( ( A-D ).EQ.ZERO .AND. SIGN( ONE, B ).NE.SIGN( ONE, C ) )
     $          THEN
         CS = ONE
         SN = ZERO
```

```
         GO TO 10
      ELSE
*
         TEMP = A - D
         P = HALF*TEMP
         BCMAX = MAX( ABS( B ), ABS( C ) )
         BCMIS = MIN( ABS( B ), ABS( C ) )*SIGN( ONE, B )*SIGN( ONE, C )
         SCALE = MAX( ABS( P ), BCMAX )
         Z = ( P / SCALE )*P + ( BCMAX / SCALE )*BCMIS
*
*        If Z is of the order of the machine accuracy, postpone the
*        decision on the nature of eigenvalues
*
         IF( Z.GE.MULTPL*EPS ) THEN
*
*           Real eigenvalues. Compute A and D.
*
            Z = P + SIGN( SQRT( SCALE )*SQRT( Z ), P )
            A = D + Z
            D = D - ( BCMAX / Z )*BCMIS
*
*           Compute B and the rotation matrix
*
            TAU = DLAPY2( C, Z )
            CS = Z / TAU
            SN = C / TAU
            B = B - C
            C = ZERO
         ELSE
*
*           Complex eigenvalues, or real (almost) equal eigenvalues.
*           Make diagonal elements equal.
*
            SIGMA = B + C
            TAU = DLAPY2( SIGMA, TEMP )
            CS = SQRT( HALF*( ONE+ABS( SIGMA ) / TAU ) )
            SN = -( P / ( TAU*CS ) )*SIGN( ONE, SIGMA )
*
*           Compute [ AA  BB ] = [ A  B ] [ CS -SN ]
*                   [ CC  DD ]   [ C  D ] [ SN  CS ]
*
            AA = A*CS + B*SN
            BB = -A*SN + B*CS
            CC = C*CS + D*SN
            DD = -C*SN + D*CS
*
*           Compute [ A  B ] = [ CS  SN ] [ AA  BB ]
*                   [ C  D ]   [-SN  CS ] [ CC  DD ]
*
            A = AA*CS + CC*SN
```

```
            B = BB*CS + DD*SN
            C = -AA*SN + CC*CS
            D = -BB*SN + DD*CS
*
            TEMP = HALF*( A+D )
            A = TEMP
            D = TEMP
*
            IF( C.NE.ZERO ) THEN
               IF( B.NE.ZERO ) THEN
                  IF( SIGN( ONE, B ).EQ.SIGN( ONE, C ) ) THEN
*
*                    Real eigenvalues: reduce to upper triangular form
*
                     SAB = SQRT( ABS( B ) )
                     SAC = SQRT( ABS( C ) )
                     P = SIGN( SAB*SAC, C )
                     TAU = ONE / SQRT( ABS( B+C ) )
                     A = TEMP + P
                     D = TEMP - P
                     B = B - C
                     C = ZERO
                     CS1 = SAB*TAU
                     SN1 = SAC*TAU
                     TEMP = CS*CS1 - SN*SN1
                     SN = CS*SN1 + SN*CS1
                     CS = TEMP
                  END IF
               ELSE
                  B = -C
                  C = ZERO
                  TEMP = CS
                  CS = -SN
                  SN = TEMP
               END IF
            END IF
         END IF
*
      END IF
*
   10 CONTINUE
*
*     Store eigenvalues in (RT1R,RT1I) and (RT2R,RT2I).
*
      RT1R = A
      RT2R = D
      IF( C.EQ.ZERO ) THEN
         RT1I = ZERO
         RT2I = ZERO
      ELSE
```

```
         RT1I = SQRT( ABS( B ) )*SQRT( ABS( C ) )
         RT2I = -RT1I
      END IF
      RETURN
*
*     End of DLANV2
*
      END
```

———-

90.157.4 lisp code

— **dlanv2.lisp** —

```
(let* ((zero 0.0d0) (half 0.5d0) (one 1.0d0) (multpl 4.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 0.5d0 0.5d0) half)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 4.0d0 4.0d0) multpl)
           (ignorable zero half one multpl))
  (defun dlanv2 (a b c d rt1r rt1i rt2r rt2i cs sn)
    (declare (type (double-float) sn cs rt2i rt2r rt1i rt1r d c b a))
    (prog ((aa 0.0d0) (bb 0.0d0) (bcmax 0.0d0) (bcmis 0.0d0) (cc 0.0d0)
           (cs1 0.0d0) (dd 0.0d0) (eps 0.0d0) (p 0.0d0) (sab 0.0d0) (sac 0.0d0)
           (scale 0.0d0) (sigma 0.0d0) (sn1 0.0d0) (tau 0.0d0) (temp 0.0d0)
           (z 0.0d0))
      (declare
       (type (double-float) aa bb bcmax bcmis cc cs1 dd eps p sab sac scale
        sigma sn1 tau temp z))
      (setf eps (dlamch "P"))
      (cond ((= c zero) (setf cs one) (setf sn zero) (go label10))
            ((= b zero) (setf cs zero) (setf sn one) (setf temp d) (setf d a)
             (setf a temp) (setf b (- c)) (setf c zero) (go label10))
            ((and (= (+ a (- d)) zero)
                  (/= (f2cl-lib:sign one b) (f2cl-lib:sign one c)))
             (setf cs one) (setf sn zero) (go label10))
            (t (setf temp (- a d)) (setf p (* half temp))
             (setf bcmax (max (abs b) (abs c)))
             (setf bcmis
                     (* (min (abs b) (abs c)) (f2cl-lib:sign one b)
                        (f2cl-lib:sign one c)))
             (setf scale (max (abs p) bcmax))
             (setf z (+ (* (/ p scale) p) (* (/ bcmax scale) bcmis)))
             (cond
              ((>= z (* multpl eps))
               (setf z
```

```
                (+ p
                   (f2cl-lib:sign
                    (* (f2cl-lib:fsqrt scale) (f2cl-lib:fsqrt z)) p)))
         (setf a (+ d z)) (setf d (- d (* (/ bcmax z) bcmis)))
         (setf tau
                (multiple-value-bind (ret-val var-0 var-1)
                    (dlapy2 c z)
                  (declare (ignore))
                  (when var-0 (setf c var-0))
                  (when var-1 (setf z var-1))
                  ret-val))
         (setf cs (/ z tau)) (setf sn (/ c tau)) (setf b (- b c))
         (setf c zero))
        (t (setf sigma (+ b c))
         (setf tau
                (multiple-value-bind (ret-val var-0 var-1)
                    (dlapy2 sigma temp)
                  (declare (ignore))
                  (when var-0 (setf sigma var-0))
                  (when var-1 (setf temp var-1))
                  ret-val))
         (setf cs (f2cl-lib:fsqrt (* half (+ one (/ (abs sigma) tau)))))
         (setf sn (* (- (/ p (* tau cs))) (f2cl-lib:sign one sigma)))
         (setf aa (+ (* a cs) (* b sn)))
         (setf bb (+ (* (- a) sn) (* b cs)))
         (setf cc (+ (* c cs) (* d sn)))
         (setf dd (+ (* (- c) sn) (* d cs)))
         (setf a (+ (* aa cs) (* cc sn)))
         (setf b (+ (* bb cs) (* dd sn)))
         (setf c (+ (* (- aa) sn) (* cc cs)))
         (setf d (+ (* (- bb) sn) (* dd cs)))
         (setf temp (* half (+ a d))) (setf a temp) (setf d temp)
         (cond
          ((/= c zero)
           (cond
            ((/= b zero)
             (cond
              ((= (f2cl-lib:sign one b) (f2cl-lib:sign one c))
               (setf sab (f2cl-lib:fsqrt (abs b)))
               (setf sac (f2cl-lib:fsqrt (abs c)))
               (setf p (f2cl-lib:sign (* sab sac) c))
               (setf tau (/ one (f2cl-lib:fsqrt (abs (+ b c)))))
               (setf a (+ temp p)) (setf d (- temp p)) (setf b (- b c))
               (setf c zero) (setf cs1 (* sab tau))
               (setf sn1 (* sac tau))
               (setf temp (- (* cs cs1) (* sn sn1)))
               (setf sn (+ (* cs sn1) (* sn cs1))) (setf cs temp))))
            (t (setf b (- c)) (setf c zero) (setf temp cs)
             (setf cs (- sn)) (setf sn temp)))))))))
label10
```

```
      (setf rt1r a)
      (setf rt2r d)
      (cond ((= c zero) (setf rt1i zero) (setf rt2i zero))
            (t
             (setf rt1i (* (f2cl-lib:fsqrt (abs b)) (f2cl-lib:fsqrt (abs c))))
             (setf rt2i (- rt1i))))
      (go end_label)
     end_label
      (return (values a b c d rt1r rt1i rt2r rt2i cs sn)))))
```

———-

90.158 dlapy2

90.158.1 Axiom unit tests

— **dlapy2.input** —

```
)set break resume
)sys rm -f dlapy2.output
)spool dlapy2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.158.2 Axiom help page

— **dlapy2.help** —

```
==================================================================
dlapy2 examples
==================================================================

==================================================================
Man Page Details
==================================================================

NAME
```

```
      DLAPY2 - sqrt(x**2+y**2), taking care not to cause unnecessary overflow

SYNOPSIS
      DOUBLE PRECISION FUNCTION DLAPY2( X, Y )

          DOUBLE       PRECISION X, Y

 Purpose
 =======

  DLAPY2 returns sqrt(x**2+y**2), taking care not to cause unnecessary
  overflow.

 Arguments
 =========

  X       (input) DOUBLE PRECISION
  Y       (input) DOUBLE PRECISION
          X and Y specify the values x and y.
```

90.158.3 fortran code

— dlapy2.f —

```
      DOUBLE PRECISION FUNCTION DLAPY2( X, Y )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1992
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   X, Y
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION   W, XABS, YABS, Z
```

```
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      XABS = ABS( X )
      YABS = ABS( Y )
      W = MAX( XABS, YABS )
      Z = MIN( XABS, YABS )
      IF( Z.EQ.ZERO ) THEN
         DLAPY2 = W
      ELSE
         DLAPY2 = W*SQRT( ONE+( Z / W )**2 )
      END IF
      RETURN
*
*     End of DLAPY2
*
      END
```

90.158.4 lisp code

— dlapy2.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dlapy2 (x y)
    (declare (type (double-float) y x))
    (prog ((w 0.0d0) (xabs 0.0d0) (yabs 0.0d0) (z 0.0d0) (dlapy2 0.0d0))
      (declare (type (double-float) w xabs yabs z dlapy2))
      (setf xabs (abs x))
      (setf yabs (abs y))
      (setf w (max xabs yabs))
      (setf z (min xabs yabs))
      (cond ((= z zero) (setf dlapy2 w))
            (t (setf dlapy2 (* w (f2cl-lib:fsqrt (+ one (expt (/ z w) 2)))))))
      (go end_label)
     end_label
      (return (values dlapy2 nil nil)))))
```

90.159 dlapy3

90.159.1 Axiom unit tests

— dlapy3.input —

```
)set break resume
)sys rm -f dlapy3.output
)spool dlapy3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.159.2 Axiom help page

— dlapy3.help —

```
======================================================================
dlapy3 examples
======================================================================

======================================================================
Man Page Details
======================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

      DOUBLE PRECISION FUNCTION DLAPY3( X, Y, Z )

      .. Scalar Arguments ..
      DOUBLE PRECISION   X, Y, Z
      ..


 Purpose:
 =============
```

```
DLAPY3 returns sqrt(x**2+y**2+z**2), taking care not to cause
unnecessary overflow.

Arguments:
==========

[in] X
         X is DOUBLE PRECISION

[in] Y
         Y is DOUBLE PRECISION

[in] Z
         Z is DOUBLE PRECISION
         X, Y and Z specify the values x, y and z.
```

90.159.3 fortran code

— dlapy3.f —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION DLAPY3( X, Y, Z )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   X, Y, Z
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION   W, XABS, YABS, ZABS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
```

```
*
      XABS = ABS( X )
      YABS = ABS( Y )
      ZABS = ABS( Z )
      W = MAX( XABS, YABS, ZABS )
      IF( W.EQ.ZERO ) THEN
*     W can be zero for max(0,nan,0)
*     adding all three entries together will make sure
*     NaN will not disappear.
         DLAPY3 =  XABS + YABS + ZABS
      ELSE
         DLAPY3 = W*SQRT( ( XABS / W )**2+( YABS / W )**2+
     $            ( ZABS / W )**2 )
      END IF
      RETURN
*
*     End of DLAPY3
*
      END
```

———-

90.159.4 lisp code

— **dlapy3.lisp** —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun dlapy3 (x y z)
    (declare (type (double-float) z y x))
    (prog ((w 0.0d0) (xabs 0.0d0) (yabs 0.0d0) (zabs 0.0d0) (dlapy3 0.0d0))
      (declare (type (double-float) w xabs yabs zabs dlapy3))
      (setf xabs (abs x))
      (setf yabs (abs y))
      (setf zabs (abs z))
      (setf w (max xabs yabs zabs))
      (cond ((= w zero) (setf dlapy3 (+ xabs yabs zabs)))
            (t
             (setf dlapy3
                     (* w
                        (f2cl-lib:fsqrt
                         (+ (expt (/ xabs w) 2) (expt (/ yabs w) 2)
                            (expt (/ zabs w) 2)))))))
      (go end_label)
     end_label
      (return (values dlapy3 nil nil nil)))))
```

———-

90.160 dlaqtr

90.160.1 Axiom unit tests

— **dlaqtr.input** —

```
)set break resume
)sys rm -f dlaqtr.output
)spool dlaqtr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.160.2 Axiom help page

— **dlaqtr.help** —

```
====================================================================
dlaqtr examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DLAQTR - the real quasi-triangular system   op(T)*p = scale*c, if LREAL
       = .TRUE

SYNOPSIS
       SUBROUTINE DLAQTR( LTRAN, LREAL, N, T, LDT, B, W, SCALE, X, WORK,  INFO
                          )

           LOGICAL        LREAL, LTRAN
```

```
      INTEGER        INFO, LDT, N

      DOUBLE         PRECISION SCALE, W

      DOUBLE         PRECISION B( * ), T( LDT, * ), WORK( * ), X( * )
```

Purpose
=======

```
 DLAQTR solves the real quasi-triangular system

              op(T)*p = scale*c,               if LREAL = .TRUE.

 or the complex quasi-triangular systems

            op(T + iB)*(p+iq) = scale*(c+id),  if LREAL = .FALSE.

 in real arithmetic, where T is upper quasi-triangular.
 If LREAL = .FALSE., then the first diagonal block of T must be
 1 by 1, B is the specially structured matrix

                B = [ b(1) b(2) ... b(n) ]
                    [       w            ]
                    [           w        ]
                    [               .    ]
                    [                 w  ]

 op(A) = A or A', A' denotes the conjugate transpose of
 matrix A.

 On input, X = [ c ].  On output, X = [ p ].
               [ d ]                  [ q ]

 This subroutine is designed for the condition number estimation
 in routine DTRSNA.
```

Arguments
=========

```
 LTRAN   (input) LOGICAL
         On entry, LTRAN specifies the option of conjugate transpose:
            = .FALSE.,    op(T+i*B) = T+i*B,
            = .TRUE.,     op(T+i*B) = (T+i*B)'.

 LREAL   (input) LOGICAL
         On entry, LREAL specifies the input matrix structure:
            = .FALSE.,    the input is complex
            = .TRUE.,     the input is real

 N       (input) INTEGER
```

```
         On entry, N specifies the order of T+i*B. N >= 0.

 T       (input) DOUBLE PRECISION array, dimension (LDT,N)
         On entry, T contains a matrix in Schur canonical form.
         If LREAL = .FALSE., then the first diagonal block of T mu
         be 1 by 1.

 LDT     (input) INTEGER
         The leading dimension of the matrix T. LDT >= max(1,N).

 B       (input) DOUBLE PRECISION array, dimension (N)
         On entry, B contains the elements to form the matrix
         B as described above.
         If LREAL = .TRUE., B is not referenced.

 W       (input) DOUBLE PRECISION
         On entry, W is the diagonal element of the matrix B.
         If LREAL = .TRUE., W is not referenced.

 SCALE   (output) DOUBLE PRECISION
         On exit, SCALE is the scale factor.

 X       (input/output) DOUBLE PRECISION array, dimension (2*N)
         On entry, X contains the right hand side of the system.
         On exit, X is overwritten by the solution.

 WORK    (workspace) DOUBLE PRECISION array, dimension (N)

 INFO    (output) INTEGER
         On exit, INFO is set to
            0: successful exit.
              1: the some diagonal 1 by 1 block has been perturbed by
                 a small number SMIN to keep nonsingularity.
              2: the some diagonal 2 by 2 block has been perturbed by
                 a small number in DLALN2 to keep nonsingularity.
         NOTE: In the interests of speed, this routine does not
               check the inputs for errors.
```

———-

90.160.3 fortran code

— dlaqtr.f —

```
      SUBROUTINE DLAQTR( LTRAN, LREAL, N, T, LDT, B, W, SCALE, X, WORK,
     $                   INFO )
*
```

```
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      LOGICAL            LREAL, LTRAN
      INTEGER            INFO, LDT, N
      DOUBLE PRECISION   SCALE, W
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   B( * ), T( LDT, * ), WORK( * ), X( * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN
      INTEGER            I, IERR, J, J1, J2, JNEXT, K, N1, N2
      DOUBLE PRECISION   BIGNUM, EPS, REC, SCALOC, SI, SMIN, SMINW,
     $                   SMLNUM, SR, TJJ, TMP, XJ, XMAX, XNORM, Z
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   D( 2, 2 ), V( 2, 2 )
*     ..
*     .. External Functions ..
      INTEGER            IDAMAX
      DOUBLE PRECISION   DASUM, DDOT, DLAMCH, DLANGE
      EXTERNAL           IDAMAX, DASUM, DDOT, DLAMCH, DLANGE
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DLADIV, DLALN2, DSCAL
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Do not test the input parameters for errors
*
      NOTRAN = .NOT.LTRAN
      INFO = 0
*
*     Quick return if possible
*
      IF( N.EQ.0 )
```

```
     $   RETURN
*
*     Set constants to control overflow
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' ) / EPS
      BIGNUM = ONE / SMLNUM
*
      XNORM = DLANGE( 'M', N, N, T, LDT, D )
      IF( .NOT.LREAL )
     $   XNORM = MAX( XNORM, ABS( W ), DLANGE( 'M', N, 1, B, N, D ) )
      SMIN = MAX( SMLNUM, EPS*XNORM )
*
*     Compute 1-norm of each column of strictly upper triangular
*     part of T to control overflow in triangular solver.
*
      WORK( 1 ) = ZERO
      DO 10 J = 2, N
         WORK( J ) = DASUM( J-1, T( 1, J ), 1 )
   10 CONTINUE
*
      IF( .NOT.LREAL ) THEN
         DO 20 I = 2, N
            WORK( I ) = WORK( I ) + ABS( B( I ) )
   20    CONTINUE
      END IF
*
      N2 = 2*N
      N1 = N
      IF( .NOT.LREAL )
     $   N1 = N2
      K = IDAMAX( N1, X, 1 )
      XMAX = ABS( X( K ) )
      SCALE = ONE
*
      IF( XMAX.GT.BIGNUM ) THEN
         SCALE = BIGNUM / XMAX
         CALL DSCAL( N1, SCALE, X, 1 )
         XMAX = BIGNUM
      END IF
*
      IF( LREAL ) THEN
*
         IF( NOTRAN ) THEN
*
*           Solve T*p = scale*c
*
            JNEXT = N
            DO 30 J = N, 1, -1
               IF( J.GT.JNEXT )
```

```
     $            GO TO 30
               J1 = J
               J2 = J
               JNEXT = J - 1
               IF( J.GT.1 ) THEN
                  IF( T( J, J-1 ).NE.ZERO ) THEN
                     J1 = J - 1
                     JNEXT = J - 2
                  END IF
               END IF
*
               IF( J1.EQ.J2 ) THEN
*
*                 Meet 1 by 1 diagonal block
*
*                 Scale to avoid overflow when computing
*                     x(j) = b(j)/T(j,j)
*
                  XJ = ABS( X( J1 ) )
                  TJJ = ABS( T( J1, J1 ) )
                  TMP = T( J1, J1 )
                  IF( TJJ.LT.SMIN ) THEN
                     TMP = SMIN
                     TJJ = SMIN
                     INFO = 1
                  END IF
*
                  IF( XJ.EQ.ZERO )
     $               GO TO 30
*
                  IF( TJJ.LT.ONE ) THEN
                     IF( XJ.GT.BIGNUM*TJJ ) THEN
                        REC = ONE / XJ
                        CALL DSCAL( N, REC, X, 1 )
                        SCALE = SCALE*REC
                        XMAX = XMAX*REC
                     END IF
                  END IF
                  X( J1 ) = X( J1 ) / TMP
                  XJ = ABS( X( J1 ) )
*
*                 Scale x if necessary to avoid overflow when adding a
*                 multiple of column j1 of T.
*
                  IF( XJ.GT.ONE ) THEN
                     REC = ONE / XJ
                     IF( WORK( J1 ).GT.( BIGNUM-XMAX )*REC ) THEN
                        CALL DSCAL( N, REC, X, 1 )
                        SCALE = SCALE*REC
                     END IF
```

```
                 END IF
                 IF( J1.GT.1 ) THEN
                    CALL DAXPY( J1-1, -X( J1 ), T( 1, J1 ), 1, X, 1 )
                    K = IDAMAX( J1-1, X, 1 )
                    XMAX = ABS( X( K ) )
                 END IF
*
              ELSE
*
*                Meet 2 by 2 diagonal block
*
*                Call 2 by 2 linear system solve, to take
*                care of possible overflow by scaling factor.
*
                 D( 1, 1 ) = X( J1 )
                 D( 2, 1 ) = X( J2 )
                 CALL DLALN2( .FALSE., 2, 1, SMIN, ONE, T( J1, J1 ),
     $                        LDT, ONE, ONE, D, 2, ZERO, ZERO, V, 2,
     $                        SCALOC, XNORM, IERR )
                 IF( IERR.NE.0 )
     $              INFO = 2
*
                 IF( SCALOC.NE.ONE ) THEN
                    CALL DSCAL( N, SCALOC, X, 1 )
                    SCALE = SCALE*SCALOC
                 END IF
                 X( J1 ) = V( 1, 1 )
                 X( J2 ) = V( 2, 1 )
*
*                Scale V(1,1) (= X(J1)) and/or V(2,1) (=X(J2))
*                to avoid overflow in updating right-hand side.
*
                 XJ = MAX( ABS( V( 1, 1 ) ), ABS( V( 2, 1 ) ) )
                 IF( XJ.GT.ONE ) THEN
                    REC = ONE / XJ
                    IF( MAX( WORK( J1 ), WORK( J2 ) ).GT.
     $                  ( BIGNUM-XMAX )*REC ) THEN
                       CALL DSCAL( N, REC, X, 1 )
                       SCALE = SCALE*REC
                    END IF
                 END IF
*
*                Update right-hand side
*
                 IF( J1.GT.1 ) THEN
                    CALL DAXPY( J1-1, -X( J1 ), T( 1, J1 ), 1, X, 1 )
                    CALL DAXPY( J1-1, -X( J2 ), T( 1, J2 ), 1, X, 1 )
                    K = IDAMAX( J1-1, X, 1 )
                    XMAX = ABS( X( K ) )
                 END IF
```

```
*
               END IF
*
   30       CONTINUE
*
         ELSE
*
*           Solve T'*p = scale*c
*
            JNEXT = 1
            DO 40 J = 1, N
               IF( J.LT.JNEXT )
     $            GO TO 40
               J1 = J
               J2 = J
               JNEXT = J + 1
               IF( J.LT.N ) THEN
                  IF( T( J+1, J ).NE.ZERO ) THEN
                     J2 = J + 1
                     JNEXT = J + 2
                  END IF
               END IF
*
               IF( J1.EQ.J2 ) THEN
*
*                 1 by 1 diagonal block
*
*                 Scale if necessary to avoid overflow in forming the
*                 right-hand side element by inner product.
*
                  XJ = ABS( X( J1 ) )
                  IF( XMAX.GT.ONE ) THEN
                     REC = ONE / XMAX
                     IF( WORK( J1 ).GT.( BIGNUM-XJ )*REC ) THEN
                        CALL DSCAL( N, REC, X, 1 )
                        SCALE = SCALE*REC
                        XMAX = XMAX*REC
                     END IF
                  END IF
*
                  X( J1 ) = X( J1 ) - DDOT( J1-1, T( 1, J1 ), 1, X, 1 )
*
                  XJ = ABS( X( J1 ) )
                  TJJ = ABS( T( J1, J1 ) )
                  TMP = T( J1, J1 )
                  IF( TJJ.LT.SMIN ) THEN
                     TMP = SMIN
                     TJJ = SMIN
                     INFO = 1
                  END IF
```

```
*
                IF( TJJ.LT.ONE ) THEN
                   IF( XJ.GT.BIGNUM*TJJ ) THEN
                      REC = ONE / XJ
                      CALL DSCAL( N, REC, X, 1 )
                      SCALE = SCALE*REC
                      XMAX = XMAX*REC
                   END IF
                END IF
                X( J1 ) = X( J1 ) / TMP
                XMAX = MAX( XMAX, ABS( X( J1 ) ) )
*
             ELSE
*
*               2 by 2 diagonal block
*
*               Scale if necessary to avoid overflow in forming the
*               right-hand side elements by inner product.
*
                XJ = MAX( ABS( X( J1 ) ), ABS( X( J2 ) ) )
                IF( XMAX.GT.ONE ) THEN
                   REC = ONE / XMAX
                   IF( MAX( WORK( J2 ), WORK( J1 ) ).GT.( BIGNUM-XJ )*
     $                 REC ) THEN
                      CALL DSCAL( N, REC, X, 1 )
                      SCALE = SCALE*REC
                      XMAX = XMAX*REC
                   END IF
                END IF
*
                D( 1, 1 ) = X( J1 ) - DDOT( J1-1, T( 1, J1 ), 1, X,
     $                      1 )
                D( 2, 1 ) = X( J2 ) - DDOT( J1-1, T( 1, J2 ), 1, X,
     $                      1 )
*
                CALL DLALN2( .TRUE., 2, 1, SMIN, ONE, T( J1, J1 ),
     $                       LDT, ONE, ONE, D, 2, ZERO, ZERO, V, 2,
     $                       SCALOC, XNORM, IERR )
                IF( IERR.NE.0 )
     $             INFO = 2
*
                IF( SCALOC.NE.ONE ) THEN
                   CALL DSCAL( N, SCALOC, X, 1 )
                   SCALE = SCALE*SCALOC
                END IF
                X( J1 ) = V( 1, 1 )
                X( J2 ) = V( 2, 1 )
                XMAX = MAX( ABS( X( J1 ) ), ABS( X( J2 ) ), XMAX )
*
             END IF
```

```
   40       CONTINUE
         END IF
*
      ELSE
*
         SMINW = MAX( EPS*ABS( W ), SMIN )
         IF( NOTRAN ) THEN
*
*           Solve (T + iB)*(p+iq) = c+id
*
            JNEXT = N
            DO 70 J = N, 1, -1
               IF( J.GT.JNEXT )
     $            GO TO 70
               J1 = J
               J2 = J
               JNEXT = J - 1
               IF( J.GT.1 ) THEN
                  IF( T( J, J-1 ).NE.ZERO ) THEN
                     J1 = J - 1
                     JNEXT = J - 2
                  END IF
               END IF
*
               IF( J1.EQ.J2 ) THEN
*
*                 1 by 1 diagonal block
*
*                 Scale if necessary to avoid overflow in division
*
                  Z = W
                  IF( J1.EQ.1 )
     $               Z = B( 1 )
                  XJ = ABS( X( J1 ) ) + ABS( X( N+J1 ) )
                  TJJ = ABS( T( J1, J1 ) ) + ABS( Z )
                  TMP = T( J1, J1 )
                  IF( TJJ.LT.SMINW ) THEN
                     TMP = SMINW
                     TJJ = SMINW
                     INFO = 1
                  END IF
*
                  IF( XJ.EQ.ZERO )
     $               GO TO 70
*
                  IF( TJJ.LT.ONE ) THEN
                     IF( XJ.GT.BIGNUM*TJJ ) THEN
                        REC = ONE / XJ
                        CALL DSCAL( N2, REC, X, 1 )
                        SCALE = SCALE*REC
```

```
                  XMAX = XMAX*REC
               END IF
            END IF
            CALL DLADIV( X( J1 ), X( N+J1 ), TMP, Z, SR, SI )
            X( J1 ) = SR
            X( N+J1 ) = SI
            XJ = ABS( X( J1 ) ) + ABS( X( N+J1 ) )
*
*           Scale x if necessary to avoid overflow when adding a
*           multiple of column j1 of T.
*
            IF( XJ.GT.ONE ) THEN
               REC = ONE / XJ
               IF( WORK( J1 ).GT.( BIGNUM-XMAX )*REC ) THEN
                  CALL DSCAL( N2, REC, X, 1 )
                  SCALE = SCALE*REC
               END IF
            END IF
*
            IF( J1.GT.1 ) THEN
               CALL DAXPY( J1-1, -X( J1 ), T( 1, J1 ), 1, X, 1 )
               CALL DAXPY( J1-1, -X( N+J1 ), T( 1, J1 ), 1,
     $                     X( N+1 ), 1 )
*
               X( 1 ) = X( 1 ) + B( J1 )*X( N+J1 )
               X( N+1 ) = X( N+1 ) - B( J1 )*X( J1 )
*
               XMAX = ZERO
               DO 50 K = 1, J1 - 1
                  XMAX = MAX( XMAX, ABS( X( K ) )+
     $                   ABS( X( K+N ) ) )
   50          CONTINUE
            END IF
*
         ELSE
*
*           Meet 2 by 2 diagonal block
*
            D( 1, 1 ) = X( J1 )
            D( 2, 1 ) = X( J2 )
            D( 1, 2 ) = X( N+J1 )
            D( 2, 2 ) = X( N+J2 )
            CALL DLALN2( .FALSE., 2, 2, SMINW, ONE, T( J1, J1 ),
     $                   LDT, ONE, ONE, D, 2, ZERO, -W, V, 2,
     $                   SCALOC, XNORM, IERR )
            IF( IERR.NE.0 )
     $         INFO = 2
*
            IF( SCALOC.NE.ONE ) THEN
               CALL DSCAL( 2*N, SCALOC, X, 1 )
```

```
                  SCALE = SCALOC*SCALE
               END IF
               X( J1 ) = V( 1, 1 )
               X( J2 ) = V( 2, 1 )
               X( N+J1 ) = V( 1, 2 )
               X( N+J2 ) = V( 2, 2 )
*
*              Scale X(J1), .... to avoid overflow in
*              updating right hand side.
*
               XJ = MAX( ABS( V( 1, 1 ) )+ABS( V( 1, 2 ) ),
     $              ABS( V( 2, 1 ) )+ABS( V( 2, 2 ) ) )
               IF( XJ.GT.ONE ) THEN
                  REC = ONE / XJ
                  IF( MAX( WORK( J1 ), WORK( J2 ) ).GT.
     $                ( BIGNUM-XMAX )*REC ) THEN
                     CALL DSCAL( N2, REC, X, 1 )
                     SCALE = SCALE*REC
                  END IF
               END IF
*
*              Update the right-hand side.
*
               IF( J1.GT.1 ) THEN
                  CALL DAXPY( J1-1, -X( J1 ), T( 1, J1 ), 1, X, 1 )
                  CALL DAXPY( J1-1, -X( J2 ), T( 1, J2 ), 1, X, 1 )
*
                  CALL DAXPY( J1-1, -X( N+J1 ), T( 1, J1 ), 1,
     $                        X( N+1 ), 1 )
                  CALL DAXPY( J1-1, -X( N+J2 ), T( 1, J2 ), 1,
     $                        X( N+1 ), 1 )
*
                  X( 1 ) = X( 1 ) + B( J1 )*X( N+J1 ) +
     $                   B( J2 )*X( N+J2 )
                  X( N+1 ) = X( N+1 ) - B( J1 )*X( J1 ) -
     $                     B( J2 )*X( J2 )
*
                  XMAX = ZERO
                  DO 60 K = 1, J1 - 1
                     XMAX = MAX( ABS( X( K ) )+ABS( X( K+N ) ),
     $                      XMAX )
   60             CONTINUE
               END IF
*
            END IF
   70       CONTINUE
*
         ELSE
*
*           Solve (T + iB)'*(p+iq) = c+id
```

```
*
            JNEXT = 1
            DO 80 J = 1, N
               IF( J.LT.JNEXT )
     $            GO TO 80
               J1 = J
               J2 = J
               JNEXT = J + 1
               IF( J.LT.N ) THEN
                  IF( T( J+1, J ).NE.ZERO ) THEN
                     J2 = J + 1
                     JNEXT = J + 2
                  END IF
               END IF
*
               IF( J1.EQ.J2 ) THEN
*
*                 1 by 1 diagonal block
*
*                 Scale if necessary to avoid overflow in forming the
*                 right-hand side element by inner product.
*
                  XJ = ABS( X( J1 ) ) + ABS( X( J1+N ) )
                  IF( XMAX.GT.ONE ) THEN
                     REC = ONE / XMAX
                     IF( WORK( J1 ).GT.( BIGNUM-XJ )*REC ) THEN
                        CALL DSCAL( N2, REC, X, 1 )
                        SCALE = SCALE*REC
                        XMAX = XMAX*REC
                     END IF
                  END IF
*
                  X( J1 ) = X( J1 ) - DDOT( J1-1, T( 1, J1 ), 1, X, 1 )
                  X( N+J1 ) = X( N+J1 ) - DDOT( J1-1, T( 1, J1 ), 1,
     $                        X( N+1 ), 1 )
                  IF( J1.GT.1 ) THEN
                     X( J1 ) = X( J1 ) - B( J1 )*X( N+1 )
                     X( N+J1 ) = X( N+J1 ) + B( J1 )*X( 1 )
                  END IF
                  XJ = ABS( X( J1 ) ) + ABS( X( J1+N ) )
*
                  Z = W
                  IF( J1.EQ.1 )
     $               Z = B( 1 )
*
*                 Scale if necessary to avoid overflow in
*                 complex division
*
                  TJJ = ABS( T( J1, J1 ) ) + ABS( Z )
                  TMP = T( J1, J1 )
```

```
                 IF( TJJ.LT.SMINW ) THEN
                    TMP = SMINW
                    TJJ = SMINW
                    INFO = 1
                 END IF
*
                 IF( TJJ.LT.ONE ) THEN
                    IF( XJ.GT.BIGNUM*TJJ ) THEN
                       REC = ONE / XJ
                       CALL DSCAL( N2, REC, X, 1 )
                       SCALE = SCALE*REC
                       XMAX = XMAX*REC
                    END IF
                 END IF
                 CALL DLADIV( X( J1 ), X( N+J1 ), TMP, -Z, SR, SI )
                 X( J1 ) = SR
                 X( J1+N ) = SI
                 XMAX = MAX( ABS( X( J1 ) )+ABS( X( J1+N ) ), XMAX )
*
              ELSE
*
*                2 by 2 diagonal block
*
*                Scale if necessary to avoid overflow in forming the
*                right-hand side element by inner product.
*
                 XJ = MAX( ABS( X( J1 ) )+ABS( X( N+J1 ) ),
     $                ABS( X( J2 ) )+ABS( X( N+J2 ) ) )
                 IF( XMAX.GT.ONE ) THEN
                    REC = ONE / XMAX
                    IF( MAX( WORK( J1 ), WORK( J2 ) ).GT.
     $                  ( BIGNUM-XJ ) / XMAX ) THEN
                       CALL DSCAL( N2, REC, X, 1 )
                       SCALE = SCALE*REC
                       XMAX = XMAX*REC
                    END IF
                 END IF
*
                 D( 1, 1 ) = X( J1 ) - DDOT( J1-1, T( 1, J1 ), 1, X,
     $                       1 )
                 D( 2, 1 ) = X( J2 ) - DDOT( J1-1, T( 1, J2 ), 1, X,
     $                       1 )
                 D( 1, 2 ) = X( N+J1 ) - DDOT( J1-1, T( 1, J1 ), 1,
     $                       X( N+1 ), 1 )
                 D( 2, 2 ) = X( N+J2 ) - DDOT( J1-1, T( 1, J2 ), 1,
     $                       X( N+1 ), 1 )
                 D( 1, 1 ) = D( 1, 1 ) - B( J1 )*X( N+1 )
                 D( 2, 1 ) = D( 2, 1 ) - B( J2 )*X( N+1 )
                 D( 1, 2 ) = D( 1, 2 ) + B( J1 )*X( 1 )
                 D( 2, 2 ) = D( 2, 2 ) + B( J2 )*X( 1 )
```

```
*
                 CALL DLALN2( .TRUE., 2, 2, SMINW, ONE, T( J1, J1 ),
     $                        LDT, ONE, ONE, D, 2, ZERO, W, V, 2,
     $                        SCALOC, XNORM, IERR )
                 IF( IERR.NE.0 )
     $              INFO = 2
*
                 IF( SCALOC.NE.ONE ) THEN
                    CALL DSCAL( N2, SCALOC, X, 1 )
                    SCALE = SCALOC*SCALE
                 END IF
                 X( J1 ) = V( 1, 1 )
                 X( J2 ) = V( 2, 1 )
                 X( N+J1 ) = V( 1, 2 )
                 X( N+J2 ) = V( 2, 2 )
                 XMAX = MAX( ABS( X( J1 ) )+ABS( X( N+J1 ) ),
     $                  ABS( X( J2 ) )+ABS( X( N+J2 ) ), XMAX )
*
              END IF
*
   80       CONTINUE
*
         END IF
*
      END IF
*
      RETURN
*
*     End of DLAQTR
*
      END
```

———-

90.160.4 lisp code

— dlaqtr.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dlaqtr (ltran lreal n t$ ldt b w scale x work info)
    (declare (type (double-float) scale w)
             (type (array double-float (*)) work x b t$)
             (type (f2cl-lib:integer4) info ldt n)
             (type f2cl-lib:logical lreal ltran))
```

```
(f2cl-lib:with-multi-array-data
    ((t$ double-float t$-%data% t$-%offset%)
     (b double-float b-%data% b-%offset%)
     (x double-float x-%data% x-%offset%)
     (work double-float work-%data% work-%offset%))
  (prog ((d (make-array 4 :element-type 'double-float))
         (v (make-array 4 :element-type 'double-float)) (bignum 0.0d0)
         (eps 0.0d0) (rec 0.0d0) (scaloc 0.0d0) (si 0.0d0) (smin 0.0d0)
         (sminw 0.0d0) (smlnum 0.0d0) (sr 0.0d0) (tjj 0.0d0) (tmp 0.0d0)
         (xj 0.0d0) (xmax 0.0d0) (xnorm 0.0d0) (z 0.0d0) (i 0) (ierr 0)
         (j 0) (j1 0) (j2 0) (jnext 0) (k 0) (n1 0) (n2 0) (notran nil))
    (declare (type (array double-float (4)) d v)
             (type (double-float) bignum eps rec scaloc si smin sminw
              smlnum sr tjj tmp xj xmax xnorm z)
             (type (f2cl-lib:integer4) i ierr j j1 j2 jnext k n1 n2)
             (type f2cl-lib:logical notran))
    (setf notran (not ltran))
    (setf info 0)
    (if (= n 0)
        (go end_label))
    (setf eps (dlamch "P"))
    (setf smlnum (/ (dlamch "S") eps))
    (setf bignum (/ one smlnum))
    (setf xnorm
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                (dlange "M" n n t$ ldt d)
              (declare (ignore var-0 var-2 var-3 var-4 var-5))
              (setf n var-1)
              ret-val))
    (if (not lreal)
        (setf xnorm
                (max xnorm (abs w)
                     (multiple-value-bind
                         (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
                         (dlange "M" n 1 b n d)
                       (declare (ignore var-0 var-2 var-3 var-4 var-5))
                       (setf n var-1)
                       ret-val))))
    (setf smin (max smlnum (* eps xnorm)))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) zero)
    (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                (dasum (f2cl-lib:int-sub j 1)
                 (f2cl-lib:array-slice t$-%data% double-float (1 j)
                                       ((1 ldt) (1 *)) t$-%offset%)
                 1))
       label10))
```

```
(cond
 ((not lreal)
  (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
              (+ (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (abs
                  (f2cl-lib:fref b-%data% (i) ((1 *)) b-%offset%))))
     label20))))
(setf n2 (f2cl-lib:int-mul 2 n))
(setf n1 n)
(if (not lreal)
    (setf n1 n2))
(setf k
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (idamax n1 x 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n1 var-0))
          ret-val))
(setf xmax (abs (f2cl-lib:fref x-%data% (k) ((1 *)) x-%offset%)))
(setf scale one)
(cond
 ((> xmax bignum) (setf scale (/ bignum xmax))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dscal n1 scale x 1)
    (declare (ignore var-2 var-3))
    (when var-0 (setf n1 var-0))
    (when var-1 (setf scale var-1)))
  (setf xmax bignum)))
(cond
 (lreal
  (cond
   (notran (setf jnext n)
    (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                  ((> j 1) nil)
      (tagbody
        (if (> j jnext)
            (go label30))
        (setf j1 j)
        (setf j2 j)
        (setf jnext (f2cl-lib:int-sub j 1))
        (cond
         ((> j 1)
          (cond
           ((/=
             (f2cl-lib:fref t$
                            (j
                             (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                            ((1 ldt) (1 *)))
```

```
     zero)
    (setf j1 (f2cl-lib:int-sub j 1))
    (setf jnext (f2cl-lib:int-sub j 2))))))
(cond
 ((= j1 j2)
  (setf xj
          (abs
           (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)))
  (setf tjj
          (abs
           (f2cl-lib:fref t$-%data% (j1 j1) ((1 ldt) (1 *))
                          t$-%offset%)))
  (setf tmp
          (f2cl-lib:fref t$-%data% (j1 j1) ((1 ldt) (1 *))
                         t$-%offset%))
  (cond
   ((< tjj smin) (setf tmp smin) (setf tjj smin)
    (setf info 1)))
  (if (= xj zero)
      (go label30))
  (cond
   ((< tjj one)
    (cond
     ((> xj (* bignum tjj)) (setf rec (/ one xj))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal n rec x 1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf n var-0))
        (when var-1 (setf rec var-1)))
      (setf scale (* scale rec)) (setf xmax (* xmax rec))))))
  (setf (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)
          (/ (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)
             tmp))
  (setf xj
          (abs
           (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)))
  (cond
   ((> xj one) (setf rec (/ one xj))
    (cond
     ((> (f2cl-lib:fref work (j1) ((1 *)))
         (* (+ bignum (- xmax)) rec))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dscal n rec x 1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf n var-0))
        (when var-1 (setf rec var-1)))
      (setf scale (* scale rec))))))
  (cond
   ((> j1 1)
    (daxpy (f2cl-lib:int-sub j1 1)
```

```
    (- (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%))
    (f2cl-lib:array-slice t$-%data% double-float (1 j1)
                          ((1 ldt) (1 *)) t$-%offset%)
    1 x 1)
   (setf k (idamax (f2cl-lib:int-sub j1 1) x 1))
   (setf xmax
           (abs
            (f2cl-lib:fref x-%data% (k) ((1 *))
                           x-%offset%))))))
(t
 (setf (f2cl-lib:fref d (1 1) ((1 2) (1 2)))
         (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%))
 (setf (f2cl-lib:fref d (2 1) ((1 2) (1 2)))
         (f2cl-lib:fref x-%data% (j2) ((1 *)) x-%offset%))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
      var-17)
     (dlaln2 f2cl-lib:%false% 2 1 smin one
      (f2cl-lib:array-slice t$-%data% double-float (j1 j1)
                            ((1 ldt) (1 *)) t$-%offset%)
      ldt one one d 2 zero zero v 2 scaloc xnorm ierr)
   (declare
    (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
     var-8 var-9 var-10 var-11 var-12 var-13 var-14))
   (setf scaloc var-15)
   (setf xnorm var-16)
   (setf ierr var-17))
 (if (/= ierr 0)
     (setf info 2))
 (cond
  ((/= scaloc one)
   (multiple-value-bind (var-0 var-1 var-2 var-3)
       (dscal n scaloc x 1)
     (declare (ignore var-2 var-3))
     (when var-0 (setf n var-0))
     (when var-1 (setf scaloc var-1)))
   (setf scale (* scale scaloc))))
 (setf (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)
         (f2cl-lib:fref v (1 1) ((1 2) (1 2))))
 (setf (f2cl-lib:fref x-%data% (j2) ((1 *)) x-%offset%)
         (f2cl-lib:fref v (2 1) ((1 2) (1 2))))
 (setf xj
         (max (abs (f2cl-lib:fref v (1 1) ((1 2) (1 2))))
              (abs (f2cl-lib:fref v (2 1) ((1 2) (1 2))))))
 (cond
  ((> xj one) (setf rec (/ one xj))
   (cond
    ((>
      (max (f2cl-lib:fref work (j1) ((1 *)))
```

```
                    (f2cl-lib:fref work (j2) ((1 *))))
                 (* (+ bignum (- xmax)) rec))
                (multiple-value-bind (var-0 var-1 var-2 var-3)
                    (dscal n rec x 1)
                  (declare (ignore var-2 var-3))
                  (when var-0 (setf n var-0))
                  (when var-1 (setf rec var-1)))
                (setf scale (* scale rec))))))
            (cond
             ((> j1 1)
              (daxpy (f2cl-lib:int-sub j1 1)
               (- (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%))
               (f2cl-lib:array-slice t$-%data% double-float (1 j1)
                                     ((1 ldt) (1 *)) t$-%offset%)
               1 x 1)
              (daxpy (f2cl-lib:int-sub j1 1)
               (- (f2cl-lib:fref x-%data% (j2) ((1 *)) x-%offset%))
               (f2cl-lib:array-slice t$-%data% double-float (1 j2)
                                     ((1 ldt) (1 *)) t$-%offset%)
               1 x 1)
              (setf k (idamax (f2cl-lib:int-sub j1 1) x 1))
              (setf xmax
                      (abs
                       (f2cl-lib:fref x-%data% (k) ((1 *))
                                      x-%offset%)))))))
        label30)))
    (t (setf jnext 1)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (if (< j jnext)
             (go label40))
         (setf j1 j)
         (setf j2 j)
         (setf jnext (f2cl-lib:int-add j 1))
         (cond
          ((< j n)
           (cond
            ((/=
              (f2cl-lib:fref t$ ((f2cl-lib:int-add j 1) j)
                             ((1 ldt) (1 *)))
              zero)
             (setf j2 (f2cl-lib:int-add j 1))
             (setf jnext (f2cl-lib:int-add j 2))))))
         (cond
          ((= j1 j2)
           (setf xj
                   (abs
                    (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)))
           (cond
```

```
 ((> xmax one) (setf rec (/ one xmax))
  (cond
   ((> (f2cl-lib:fref work (j1) ((1 *)))
       (* (+ bignum (- xj)) rec))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal n rec x 1)
      (declare (ignore var-2 var-3))
      (when var-0 (setf n var-0))
      (when var-1 (setf rec var-1)))
    (setf scale (* scale rec)) (setf xmax (* xmax rec))))))
(setf (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)
        (- (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)
           (ddot (f2cl-lib:int-sub j1 1)
            (f2cl-lib:array-slice t$-%data% double-float
                                  (1 j1) ((1 ldt) (1 *))
                                  t$-%offset%)
            1 x 1)))
(setf xj
        (abs
         (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)))
(setf tjj
        (abs
         (f2cl-lib:fref t$-%data% (j1 j1) ((1 ldt) (1 *))
                        t$-%offset%)))
(setf tmp
        (f2cl-lib:fref t$-%data% (j1 j1) ((1 ldt) (1 *))
                       t$-%offset%))
(cond
 ((< tjj smin) (setf tmp smin) (setf tjj smin)
  (setf info 1)))
(cond
 ((< tjj one)
  (cond
   ((> xj (* bignum tjj)) (setf rec (/ one xj))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal n rec x 1)
      (declare (ignore var-2 var-3))
      (when var-0 (setf n var-0))
      (when var-1 (setf rec var-1)))
    (setf scale (* scale rec)) (setf xmax (* xmax rec))))))
(setf (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)
        (/ (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)
           tmp))
(setf xmax
        (max xmax
             (abs
              (f2cl-lib:fref x-%data% (j1) ((1 *))
                             x-%offset%)))))
(t
 (setf xj
```

```
        (max
         (abs
          (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%))
         (abs
          (f2cl-lib:fref x-%data% (j2) ((1 *)) x-%offset%))))
(cond
 ((> xmax one) (setf rec (/ one xmax))
  (cond
   ((>
     (max (f2cl-lib:fref work (j2) ((1 *)))
          (f2cl-lib:fref work (j1) ((1 *))))
     (* (+ bignum (- xj)) rec))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal n rec x 1)
      (declare (ignore var-2 var-3))
      (when var-0 (setf n var-0))
      (when var-1 (setf rec var-1)))
    (setf scale (* scale rec)) (setf xmax (* xmax rec))))))
(setf (f2cl-lib:fref d (1 1) ((1 2) (1 2)))
        (- (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)
           (ddot (f2cl-lib:int-sub j1 1)
            (f2cl-lib:array-slice t$-%data% double-float
                                  (1 j1) ((1 ldt) (1 *))
                                  t$-%offset%)
            1 x 1)))
(setf (f2cl-lib:fref d (2 1) ((1 2) (1 2)))
        (- (f2cl-lib:fref x-%data% (j2) ((1 *)) x-%offset%)
           (ddot (f2cl-lib:int-sub j1 1)
            (f2cl-lib:array-slice t$-%data% double-float
                                  (1 j2) ((1 ldt) (1 *))
                                  t$-%offset%)
            1 x 1)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
    (dlaln2 f2cl-lib:%true% 2 1 smin one
     (f2cl-lib:array-slice t$-%data% double-float (j1 j1)
                           ((1 ldt) (1 *)) t$-%offset%)
     ldt one one d 2 zero zero v 2 scaloc xnorm ierr)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
    var-8 var-9 var-10 var-11 var-12 var-13 var-14))
  (setf scaloc var-15)
  (setf xnorm var-16)
  (setf ierr var-17))
(if (/= ierr 0)
    (setf info 2))
(cond
 ((/= scaloc one)
```

```
                (multiple-value-bind (var-0 var-1 var-2 var-3)
                    (dscal n scaloc x 1)
                  (declare (ignore var-2 var-3))
                  (when var-0 (setf n var-0))
                  (when var-1 (setf scaloc var-1)))
                (setf scale (* scale scaloc))))
              (setf (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)
                      (f2cl-lib:fref v (1 1) ((1 2) (1 2))))
              (setf (f2cl-lib:fref x-%data% (j2) ((1 *)) x-%offset%)
                      (f2cl-lib:fref v (2 1) ((1 2) (1 2))))
              (setf xmax
                      (max
                       (abs
                        (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%))
                       (abs
                        (f2cl-lib:fref x-%data% (j2) ((1 *)) x-%offset%))
                       xmax))))
          label40)))))
      (t (setf sminw (max (* eps (abs w)) smin))
       (cond
        (notran (setf jnext n)
         (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((> j 1) nil)
           (tagbody
             (if (> j jnext)
                 (go label70))
             (setf j1 j)
             (setf j2 j)
             (setf jnext (f2cl-lib:int-sub j 1))
             (cond
              ((> j 1)
               (cond
                ((/=
                  (f2cl-lib:fref t$
                                 (j
                                  (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                                 ((1 ldt) (1 *)))
                  zero)
                 (setf j1 (f2cl-lib:int-sub j 1))
                 (setf jnext (f2cl-lib:int-sub j 2))))))
             (cond
              ((= j1 j2) (setf z w)
               (if (= j1 1)
                   (setf z (f2cl-lib:fref b-%data% (1) ((1 *)) b-%offset%)))
               (setf xj
                       (+
                        (abs
                         (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%))
                        (abs
                         (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j1))
```

```
                        ((1 *)) x-%offset%))))
(setf tjj
        (+
         (abs
          (f2cl-lib:fref t$-%data% (j1 j1) ((1 ldt) (1 *))
                         t$-%offset%))
         (abs z)))
(setf tmp
        (f2cl-lib:fref t$-%data% (j1 j1) ((1 ldt) (1 *))
                       t$-%offset%))
(cond
 ((< tjj sminw) (setf tmp sminw) (setf tjj sminw)
  (setf info 1)))
(if (= xj zero)
    (go label70))
(cond
 ((< tjj one)
  (cond
   ((> xj (* bignum tjj)) (setf rec (/ one xj))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal n2 rec x 1)
      (declare (ignore var-2 var-3))
      (when var-0 (setf n2 var-0))
      (when var-1 (setf rec var-1)))
    (setf scale (* scale rec)) (setf xmax (* xmax rec))))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (dladiv (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)
     (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j1))
                    ((1 *)) x-%offset%)
     tmp z sr si)
  (declare (ignore var-0 var-1 var-2 var-3))
  (setf sr var-4)
  (setf si var-5))
(setf (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%) sr)
(setf (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j1))
                      ((1 *)) x-%offset%)
        si)
(setf xj
        (+
         (abs
          (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%))
         (abs
          (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j1))
                         ((1 *)) x-%offset%))))
(cond
 ((> xj one) (setf rec (/ one xj))
  (cond
   ((> (f2cl-lib:fref work (j1) ((1 *)))
       (* (+ bignum (- xmax)) rec))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
```

```
        (dscal n2 rec x 1)
      (declare (ignore var-2 var-3))
      (when var-0 (setf n2 var-0))
      (when var-1 (setf rec var-1)))
    (setf scale (* scale rec))))))
(cond
 ((> j1 1)
  (daxpy (f2cl-lib:int-sub j1 1)
   (- (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%))
   (f2cl-lib:array-slice t$-%data% double-float (1 j1)
                         ((1 ldt) (1 *)) t$-%offset%)
   1 x 1)
  (daxpy (f2cl-lib:int-sub j1 1)
   (-
    (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j1)) ((1 *))
                   x-%offset%))
   (f2cl-lib:array-slice t$-%data% double-float (1 j1)
                         ((1 ldt) (1 *)) t$-%offset%)
   1
   (f2cl-lib:array-slice x-%data% double-float ((+ n 1))
                         ((1 *)) x-%offset%)
   1)
  (setf (f2cl-lib:fref x-%data% (1) ((1 *)) x-%offset%)
        (+ (f2cl-lib:fref x-%data% (1) ((1 *)) x-%offset%)
           (*
            (f2cl-lib:fref b-%data% (j1) ((1 *))
                           b-%offset%)
            (f2cl-lib:fref x-%data%
                           ((f2cl-lib:int-add n j1))
                           ((1 *)) x-%offset%))))
  (setf (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n 1))
                       ((1 *)) x-%offset%)
        (-
         (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n 1))
                        ((1 *)) x-%offset%)
         (*
          (f2cl-lib:fref b-%data% (j1) ((1 *)) b-%offset%)
          (f2cl-lib:fref x-%data% (j1) ((1 *))
                         x-%offset%))))
  (setf xmax zero)
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k
                    (f2cl-lib:int-add j1
                                      (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (setf xmax
            (max xmax
                 (+
                  (abs
```

```
                        (f2cl-lib:fref x-%data% (k) ((1 *))
                                       x-%offset%))
                       (abs
                        (f2cl-lib:fref x-%data%
                                       ((f2cl-lib:int-add k n))
                                       ((1 *)) x-%offset%)))))
      label50)))))
(t
 (setf (f2cl-lib:fref d (1 1) ((1 2) (1 2)))
         (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%))
 (setf (f2cl-lib:fref d (2 1) ((1 2) (1 2)))
         (f2cl-lib:fref x-%data% (j2) ((1 *)) x-%offset%))
 (setf (f2cl-lib:fref d (1 2) ((1 2) (1 2)))
         (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j1))
                        ((1 *)) x-%offset%))
 (setf (f2cl-lib:fref d (2 2) ((1 2) (1 2)))
         (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j2))
                        ((1 *)) x-%offset%))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
      var-17)
     (dlaln2 f2cl-lib:%false% 2 2 sminw one
      (f2cl-lib:array-slice t$-%data% double-float (j1 j1)
                            ((1 ldt) (1 *)) t$-%offset%)
      ldt one one d 2 zero (- w) v 2 scaloc xnorm ierr)
   (declare
    (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
     var-8 var-9 var-10 var-11 var-12 var-13 var-14))
   (setf scaloc var-15)
   (setf xnorm var-16)
   (setf ierr var-17))
 (if (/= ierr 0)
     (setf info 2))
 (cond
  ((/= scaloc one)
   (multiple-value-bind (var-0 var-1 var-2 var-3)
       (dscal (f2cl-lib:int-mul 2 n) scaloc x 1)
     (declare (ignore var-0 var-2 var-3))
     (when var-1 (setf scaloc var-1)))
   (setf scale (* scaloc scale))))
 (setf (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)
         (f2cl-lib:fref v (1 1) ((1 2) (1 2))))
 (setf (f2cl-lib:fref x-%data% (j2) ((1 *)) x-%offset%)
         (f2cl-lib:fref v (2 1) ((1 2) (1 2))))
 (setf (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j1))
                       ((1 *)) x-%offset%)
         (f2cl-lib:fref v (1 2) ((1 2) (1 2))))
 (setf (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j2))
                       ((1 *)) x-%offset%)
```

```
        (f2cl-lib:fref v (2 2) ((1 2) (1 2))))
(setf xj
        (max
         (+ (abs (f2cl-lib:fref v (1 1) ((1 2) (1 2))))
            (abs (f2cl-lib:fref v (1 2) ((1 2) (1 2)))))
         (+ (abs (f2cl-lib:fref v (2 1) ((1 2) (1 2))))
            (abs (f2cl-lib:fref v (2 2) ((1 2) (1 2)))))))
(cond
 ((> xj one) (setf rec (/ one xj))
  (cond
   ((>
     (max (f2cl-lib:fref work (j1) ((1 *)))
          (f2cl-lib:fref work (j2) ((1 *))))
     (* (+ bignum (- xmax)) rec))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal n2 rec x 1)
      (declare (ignore var-2 var-3))
      (when var-0 (setf n2 var-0))
      (when var-1 (setf rec var-1)))
    (setf scale (* scale rec))))))
(cond
 ((> j1 1)
  (daxpy (f2cl-lib:int-sub j1 1)
   (- (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%))
   (f2cl-lib:array-slice t$-%data% double-float (1 j1)
                         ((1 ldt) (1 *)) t$-%offset%)
   1 x 1)
  (daxpy (f2cl-lib:int-sub j1 1)
   (- (f2cl-lib:fref x-%data% (j2) ((1 *)) x-%offset%))
   (f2cl-lib:array-slice t$-%data% double-float (1 j2)
                         ((1 ldt) (1 *)) t$-%offset%)
   1 x 1)
  (daxpy (f2cl-lib:int-sub j1 1)
   (-
    (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j1)) ((1 *))
                   x-%offset%))
   (f2cl-lib:array-slice t$-%data% double-float (1 j1)
                         ((1 ldt) (1 *)) t$-%offset%)
   1
   (f2cl-lib:array-slice x-%data% double-float ((+ n 1))
                         ((1 *)) x-%offset%)
   1)
  (daxpy (f2cl-lib:int-sub j1 1)
   (-
    (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j2)) ((1 *))
                   x-%offset%))
   (f2cl-lib:array-slice t$-%data% double-float (1 j2)
                         ((1 ldt) (1 *)) t$-%offset%)
   1
   (f2cl-lib:array-slice x-%data% double-float ((+ n 1))
```

```
                                ((1 *)) x-%offset%)
                 1)
                (setf (f2cl-lib:fref x-%data% (1) ((1 *)) x-%offset%)
                        (+ (f2cl-lib:fref x-%data% (1) ((1 *)) x-%offset%)
                           (*
                            (f2cl-lib:fref b-%data% (j1) ((1 *))
                                           b-%offset%)
                            (f2cl-lib:fref x-%data%
                                           ((f2cl-lib:int-add n j1))
                                           ((1 *)) x-%offset%))
                           (*
                            (f2cl-lib:fref b-%data% (j2) ((1 *))
                                           b-%offset%)
                            (f2cl-lib:fref x-%data%
                                           ((f2cl-lib:int-add n j2))
                                           ((1 *)) x-%offset%))))
                (setf (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n 1))
                                     ((1 *)) x-%offset%)
                        (-
                         (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n 1))
                                        ((1 *)) x-%offset%)
                         (*
                          (f2cl-lib:fref b-%data% (j1) ((1 *)) b-%offset%)
                          (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%))
                         (*
                          (f2cl-lib:fref b-%data% (j2) ((1 *)) b-%offset%)
                          (f2cl-lib:fref x-%data% (j2) ((1 *))
                                         x-%offset%))))
                (setf xmax zero)
                (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                              ((> k
                                  (f2cl-lib:int-add j1
                                                    (f2cl-lib:int-sub 1)))
                               nil)
                  (tagbody
                    (setf xmax
                            (max
                             (+
                              (abs
                               (f2cl-lib:fref x-%data% (k) ((1 *))
                                              x-%offset%))
                              (abs
                               (f2cl-lib:fref x-%data%
                                              ((f2cl-lib:int-add k n))
                                              ((1 *)) x-%offset%)))
                             xmax))
                    label60))))))
            label70)))
        (t (setf jnext 1)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
            ((> j n) nil)
(tagbody
  (if (< j jnext)
      (go label80))
  (setf j1 j)
  (setf j2 j)
  (setf jnext (f2cl-lib:int-add j 1))
  (cond
   ((< j n)
    (cond
     ((/=
       (f2cl-lib:fref t$ ((f2cl-lib:int-add j 1) j)
                      ((1 ldt) (1 *)))
       zero)
      (setf j2 (f2cl-lib:int-add j 1))
      (setf jnext (f2cl-lib:int-add j 2))))))
  (cond
   ((= j1 j2)
    (setf xj
            (+
             (abs
              (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%))
             (abs
              (f2cl-lib:fref x-%data% ((f2cl-lib:int-add j1 n))
                             ((1 *)) x-%offset%))))
    (cond
     ((> xmax one) (setf rec (/ one xmax))
      (cond
       ((> (f2cl-lib:fref work (j1) ((1 *)))
           (* (+ bignum (- xj)) rec))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal n2 rec x 1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n2 var-0))
          (when var-1 (setf rec var-1)))
        (setf scale (* scale rec)) (setf xmax (* xmax rec))))))
    (setf (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)
            (- (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)
               (ddot (f2cl-lib:int-sub j1 1)
                (f2cl-lib:array-slice t$-%data% double-float
                                      (1 j1) ((1 ldt) (1 *))
                                      t$-%offset%)
                1 x 1)))
    (setf (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j1))
                         ((1 *)) x-%offset%)
            (-
             (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j1))
                            ((1 *)) x-%offset%)
             (ddot (f2cl-lib:int-sub j1 1)
              (f2cl-lib:array-slice t$-%data% double-float (1 j1)
```

```
                                ((1 ldt) (1 *)) t$-%offset%)
          1
          (f2cl-lib:array-slice x-%data% double-float
                                ((+ n 1)) ((1 *)) x-%offset%)
          1)))
(cond
 ((> j1 1)
  (setf (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)
          (- (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)
             (*
              (f2cl-lib:fref b-%data% (j1) ((1 *))
                             b-%offset%)
              (f2cl-lib:fref x-%data%
                             ((f2cl-lib:int-add n 1)) ((1 *))
                             x-%offset%))))
  (setf (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j1))
                        ((1 *)) x-%offset%)
          (+
           (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j1))
                          ((1 *)) x-%offset%)
           (*
            (f2cl-lib:fref b-%data% (j1) ((1 *)) b-%offset%)
            (f2cl-lib:fref x-%data% (1) ((1 *))
                           x-%offset%))))))
(setf xj
        (+
         (abs
          (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%))
         (abs
          (f2cl-lib:fref x-%data% ((f2cl-lib:int-add j1 n))
                         ((1 *)) x-%offset%))))
(setf z w)
(if (= j1 1)
    (setf z (f2cl-lib:fref b-%data% (1) ((1 *)) b-%offset%)))
(setf tjj
        (+
         (abs
          (f2cl-lib:fref t$-%data% (j1 j1) ((1 ldt) (1 *))
                         t$-%offset%))
         (abs z)))
(setf tmp
        (f2cl-lib:fref t$-%data% (j1 j1) ((1 ldt) (1 *))
                       t$-%offset%))
(cond
 ((< tjj sminw) (setf tmp sminw) (setf tjj sminw)
  (setf info 1)))
(cond
 ((< tjj one)
  (cond
   ((> xj (* bignum tjj)) (setf rec (/ one xj))
```

```
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal n2 rec x 1)
      (declare (ignore var-2 var-3))
      (when var-0 (setf n2 var-0))
      (when var-1 (setf rec var-1)))
    (setf scale (* scale rec)) (setf xmax (* xmax rec))))))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
    (dladiv (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)
     (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j1))
                    ((1 *)) x-%offset%)
     tmp (- z) sr si)
   (declare (ignore var-0 var-1 var-2 var-3))
   (setf sr var-4)
   (setf si var-5))
 (setf (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%) sr)
 (setf (f2cl-lib:fref x-%data% ((f2cl-lib:int-add j1 n))
                      ((1 *)) x-%offset%)
        si)
 (setf xmax
        (max
         (+
          (abs
           (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%))
          (abs
           (f2cl-lib:fref x-%data% ((f2cl-lib:int-add j1 n))
                          ((1 *)) x-%offset%)))
         xmax)))
(t
 (setf xj
        (max
         (+
          (abs
           (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%))
          (abs
           (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j1))
                          ((1 *)) x-%offset%)))
         (+
          (abs
           (f2cl-lib:fref x-%data% (j2) ((1 *)) x-%offset%))
          (abs
           (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j2))
                          ((1 *)) x-%offset%)))))
 (cond
  ((> xmax one) (setf rec (/ one xmax))
   (cond
    ((>
      (max (f2cl-lib:fref work (j1) ((1 *)))
           (f2cl-lib:fref work (j2) ((1 *))))
      (f2cl-lib:f2cl/ (+ bignum (- xj)) xmax))
     (multiple-value-bind (var-0 var-1 var-2 var-3)
```

```
        (dscal n2 rec x 1)
      (declare (ignore var-2 var-3))
      (when var-0 (setf n2 var-0))
      (when var-1 (setf rec var-1)))
    (setf scale (* scale rec)) (setf xmax (* xmax rec))))))
(setf (f2cl-lib:fref d (1 1) ((1 2) (1 2)))
        (- (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)
           (ddot (f2cl-lib:int-sub j1 1)
            (f2cl-lib:array-slice t$-%data% double-float
                                  (1 j1) ((1 ldt) (1 *))
                                  t$-%offset%)
            1 x 1)))
(setf (f2cl-lib:fref d (2 1) ((1 2) (1 2)))
        (- (f2cl-lib:fref x-%data% (j2) ((1 *)) x-%offset%)
           (ddot (f2cl-lib:int-sub j1 1)
            (f2cl-lib:array-slice t$-%data% double-float
                                  (1 j2) ((1 ldt) (1 *))
                                  t$-%offset%)
            1 x 1)))
(setf (f2cl-lib:fref d (1 2) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j1))
                        ((1 *)) x-%offset%)
         (ddot (f2cl-lib:int-sub j1 1)
          (f2cl-lib:array-slice t$-%data% double-float (1 j1)
                                ((1 ldt) (1 *)) t$-%offset%)
          1
          (f2cl-lib:array-slice x-%data% double-float
                                ((+ n 1)) ((1 *)) x-%offset%)
          1)))
(setf (f2cl-lib:fref d (2 2) ((1 2) (1 2)))
        (-
         (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j2))
                        ((1 *)) x-%offset%)
         (ddot (f2cl-lib:int-sub j1 1)
          (f2cl-lib:array-slice t$-%data% double-float (1 j2)
                                ((1 ldt) (1 *)) t$-%offset%)
          1
          (f2cl-lib:array-slice x-%data% double-float
                                ((+ n 1)) ((1 *)) x-%offset%)
          1)))
(setf (f2cl-lib:fref d (1 1) ((1 2) (1 2)))
        (- (f2cl-lib:fref d (1 1) ((1 2) (1 2)))
           (*
            (f2cl-lib:fref b-%data% (j1) ((1 *)) b-%offset%)
            (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n 1))
                           ((1 *)) x-%offset%))))
(setf (f2cl-lib:fref d (2 1) ((1 2) (1 2)))
        (- (f2cl-lib:fref d (2 1) ((1 2) (1 2)))
           (*
```

```
            (f2cl-lib:fref b-%data% (j2) ((1 *)) b-%offset%)
            (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n 1))
                           ((1 *)) x-%offset%))))
(setf (f2cl-lib:fref d (1 2) ((1 2) (1 2)))
        (+ (f2cl-lib:fref d (1 2) ((1 2) (1 2)))
           (*
            (f2cl-lib:fref b-%data% (j1) ((1 *)) b-%offset%)
            (f2cl-lib:fref x-%data% (1) ((1 *))
                           x-%offset%))))
(setf (f2cl-lib:fref d (2 2) ((1 2) (1 2)))
        (+ (f2cl-lib:fref d (2 2) ((1 2) (1 2)))
           (*
            (f2cl-lib:fref b-%data% (j2) ((1 *)) b-%offset%)
            (f2cl-lib:fref x-%data% (1) ((1 *))
                           x-%offset%))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17)
    (dlaln2 f2cl-lib:%true% 2 2 sminw one
     (f2cl-lib:array-slice t$-%data% double-float (j1 j1)
                           ((1 ldt) (1 *)) t$-%offset%)
     ldt one one d 2 zero w v 2 scaloc xnorm ierr)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
    var-8 var-9 var-10 var-11 var-12 var-13 var-14))
  (setf scaloc var-15)
  (setf xnorm var-16)
  (setf ierr var-17))
(if (/= ierr 0)
    (setf info 2))
(cond
 ((/= scaloc one)
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dscal n2 scaloc x 1)
    (declare (ignore var-2 var-3))
    (when var-0 (setf n2 var-0))
    (when var-1 (setf scaloc var-1)))
  (setf scale (* scaloc scale))))
(setf (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%)
        (f2cl-lib:fref v (1 1) ((1 2) (1 2))))
(setf (f2cl-lib:fref x-%data% (j2) ((1 *)) x-%offset%)
        (f2cl-lib:fref v (2 1) ((1 2) (1 2))))
(setf (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j1))
                      ((1 *)) x-%offset%)
        (f2cl-lib:fref v (1 2) ((1 2) (1 2))))
(setf (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j2))
                      ((1 *)) x-%offset%)
        (f2cl-lib:fref v (2 2) ((1 2) (1 2))))
(setf xmax
```

```
                        (max
                         (+
                          (abs
                           (f2cl-lib:fref x-%data% (j1) ((1 *)) x-%offset%))
                          (abs
                           (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j1))
                                          ((1 *)) x-%offset%)))
                         (+
                          (abs
                           (f2cl-lib:fref x-%data% (j2) ((1 *)) x-%offset%))
                          (abs
                           (f2cl-lib:fref x-%data% ((f2cl-lib:int-add n j2))
                                          ((1 *)) x-%offset%)))
                         xmax))))
               label80))))))
        (go end_label)
       end_label
        (return (values nil nil n nil nil nil nil scale nil nil info))))))
```

———-

90.161 dlarfb

90.161.1 Axiom unit tests

— **dlarfb.input** —

```
)set break resume
)sys rm -f dlarfb.output
)spool dlarfb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.161.2 Axiom help page

— **dlarfb.help** —

```
==================================================================
```

```
dlarfb examples
===================================================================

===================================================================
Man Page Details
===================================================================

NAME
       DLARFB  - a real block reflector H or its transpose H' to a real m by n
       matrix C, from either the left or the right

SYNOPSIS
       SUBROUTINE DLARFB( SIDE, TRANS, DIRECT, STOREV, M, N,  K,  V,  LDV,  T,
                          LDT, C, LDC, WORK, LDWORK )

           CHARACTER      DIRECT, SIDE, STOREV, TRANS

           INTEGER        K, LDC, LDT, LDV, LDWORK, M, N

           DOUBLE         PRECISION  C(  LDC,  *  ), T( LDT, * ), V( LDV, * ),
                          WORK( LDWORK, * )

 Purpose
 =======

  DLARFB applies a real block reflector H or its transpose H' to a
  real m by n matrix C, from either the left or the right.

 Arguments
 =========

  SIDE    (input) CHARACTER*1
          = 'L': apply H or H' from the Left
          = 'R': apply H or H' from the Right

  TRANS   (input) CHARACTER*1
          = 'N': apply H (No transpose)
          = 'T': apply H' (Transpose)

  DIRECT  (input) CHARACTER*1
          Indicates how H is formed from a product of elementary
          reflectors
          = 'F': H = H(1) H(2) . . . H(k) (Forward)
          = 'B': H = H(k) . . . H(2) H(1) (Backward)

  STOREV  (input) CHARACTER*1
          Indicates how the vectors which define the elementary
          reflectors are stored:
          = 'C': Columnwise
          = 'R': Rowwise
```

```
M       (input) INTEGER
        The number of rows of the matrix C.

N       (input) INTEGER
        The number of columns of the matrix C.

K       (input) INTEGER
        The order of the matrix T (= the number of elementary
        reflectors whose product defines the block reflector).

V       (input) DOUBLE PRECISION array, dimension
                              (LDV,K) if STOREV = 'C'
                              (LDV,M) if STOREV = 'R' and SIDE = 'L'
                              (LDV,N) if STOREV = 'R' and SIDE = 'R'
        The matrix V. See further details.

LDV     (input) INTEGER
        The leading dimension of the array V.
        If STOREV = 'C' and SIDE = 'L', LDV >= max(1,M);
        if STOREV = 'C' and SIDE = 'R', LDV >= max(1,N);
        if STOREV = 'R', LDV >= K.

T       (input) DOUBLE PRECISION array, dimension (LDT,K)
        The triangular k by k matrix T in the representation of the
        block reflector.

LDT     (input) INTEGER
        The leading dimension of the array T. LDT >= K.

C       (input/output) DOUBLE PRECISION array, dimension (LDC,N)
        On entry, the m by n matrix C.
        On exit, C is overwritten by H*C or H'*C or C*H or C*H'.

LDC     (input) INTEGER
        The leading dimension of the array C. LDA >= max(1,M).

WORK    (workspace) DOUBLE PRECISION array, dimension (LDWORK,K)

LDWORK  (input) INTEGER
        The leading dimension of the array WORK.
        If SIDE = 'L', LDWORK >= max(1,N);
        if SIDE = 'R', LDWORK >= max(1,M).
```

90.161.3 fortran code

— dlarfb.f —

```
      SUBROUTINE DLARFB( SIDE, TRANS, DIRECT, STOREV, M, N, K, V, LDV,
     $                   T, LDT, C, LDC, WORK, LDWORK )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     February 29, 1992
*
*     .. Scalar Arguments ..
      CHARACTER          DIRECT, SIDE, STOREV, TRANS
      INTEGER            K, LDC, LDT, LDV, LDWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   C( LDC, * ), T( LDT, * ), V( LDV, * ),
     $                   WORK( LDWORK, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      CHARACTER          TRANST
      INTEGER            I, J
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DGEMM, DTRMM
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( M.LE.0 .OR. N.LE.0 )
     $   RETURN
*
      IF( LSAME( TRANS, 'N' ) ) THEN
         TRANST = 'T'
      ELSE
         TRANST = 'N'
```

```
      END IF
*
      IF( LSAME( STOREV, 'C' ) ) THEN
*
         IF( LSAME( DIRECT, 'F' ) ) THEN
*
*           Let  V =  ( V1 )    (first K rows)
*                     ( V2 )
*           where  V1  is unit lower triangular.
*
            IF( LSAME( SIDE, 'L' ) ) THEN
*
*              Form  H * C  or  H' * C  where  C = ( C1 )
*                                                  ( C2 )
*
*              W := C' * V  =  (C1'*V1 + C2'*V2)  (stored in WORK)
*
*              W := C1'
*
               DO 10 J = 1, K
                  CALL DCOPY( N, C( J, 1 ), LDC, WORK( 1, J ), 1 )
   10          CONTINUE
*
*              W := W * V1
*
               CALL DTRMM( 'Right', 'Lower', 'No transpose', 'Unit', N,
     $                     K, ONE, V, LDV, WORK, LDWORK )
               IF( M.GT.K ) THEN
*
*                 W := W + C2'*V2
*
                  CALL DGEMM( 'Transpose', 'No transpose', N, K, M-K,
     $                        ONE, C( K+1, 1 ), LDC, V( K+1, 1 ), LDV,
     $                        ONE, WORK, LDWORK )
               END IF
*
*              W := W * T'  or  W * T
*
               CALL DTRMM( 'Right', 'Upper', TRANST, 'Non-unit', N, K,
     $                     ONE, T, LDT, WORK, LDWORK )
*
*              C := C - V * W'
*
               IF( M.GT.K ) THEN
*
*                 C2 := C2 - V2 * W'
*
                  CALL DGEMM( 'No transpose', 'Transpose', M-K, N, K,
     $                        -ONE, V( K+1, 1 ), LDV, WORK, LDWORK, ONE,
     $                        C( K+1, 1 ), LDC )
```

```
               END IF
*
*              W := W * V1'
*
               CALL DTRMM( 'Right', 'Lower', 'Transpose', 'Unit', N, K,
     $                     ONE, V, LDV, WORK, LDWORK )
*
*              C1 := C1 - W'
*
               DO 30 J = 1, K
                  DO 20 I = 1, N
                     C( J, I ) = C( J, I ) - WORK( I, J )
   20             CONTINUE
   30          CONTINUE
*
            ELSE IF( LSAME( SIDE, 'R' ) ) THEN
*
*              Form  C * H  or  C * H'  where  C = ( C1  C2 )
*
*              W := C * V  =  (C1*V1 + C2*V2)  (stored in WORK)
*
*              W := C1
*
               DO 40 J = 1, K
                  CALL DCOPY( M, C( 1, J ), 1, WORK( 1, J ), 1 )
   40          CONTINUE
*
*              W := W * V1
*
               CALL DTRMM( 'Right', 'Lower', 'No transpose', 'Unit', M,
     $                     K, ONE, V, LDV, WORK, LDWORK )
               IF( N.GT.K ) THEN
*
*                 W := W + C2 * V2
*
                  CALL DGEMM( 'No transpose', 'No transpose', M, K, N-K,
     $                        ONE, C( 1, K+1 ), LDC, V( K+1, 1 ), LDV,
     $                        ONE, WORK, LDWORK )
               END IF
*
*              W := W * T  or  W * T'
*
               CALL DTRMM( 'Right', 'Upper', TRANS, 'Non-unit', M, K,
     $                     ONE, T, LDT, WORK, LDWORK )
*
*              C := C - W * V'
*
               IF( N.GT.K ) THEN
*
*                 C2 := C2 - W * V2'
```

```
*
                 CALL DGEMM( 'No transpose', 'Transpose', M, N-K, K,
     $                       -ONE, WORK, LDWORK, V( K+1, 1 ), LDV, ONE,
     $                       C( 1, K+1 ), LDC )
              END IF
*
*             W := W * V1'
*
              CALL DTRMM( 'Right', 'Lower', 'Transpose', 'Unit', M, K,
     $                    ONE, V, LDV, WORK, LDWORK )
*
*             C1 := C1 - W
*
              DO 60 J = 1, K
                 DO 50 I = 1, M
                    C( I, J ) = C( I, J ) - WORK( I, J )
   50            CONTINUE
   60         CONTINUE
           END IF
*
        ELSE
*
*          Let  V =  ( V1 )
*                    ( V2 )    (last K rows)
*          where  V2  is unit upper triangular.
*
           IF( LSAME( SIDE, 'L' ) ) THEN
*
*             Form  H * C  or  H' * C  where  C = ( C1 )
*                                                 ( C2 )
*
*             W := C' * V  =  (C1'*V1 + C2'*V2)  (stored in WORK)
*
*             W := C2'
*
              DO 70 J = 1, K
                 CALL DCOPY( N, C( M-K+J, 1 ), LDC, WORK( 1, J ), 1 )
   70         CONTINUE
*
*             W := W * V2
*
              CALL DTRMM( 'Right', 'Upper', 'No transpose', 'Unit', N,
     $                    K, ONE, V( M-K+1, 1 ), LDV, WORK, LDWORK )
              IF( M.GT.K ) THEN
*
*                W := W + C1'*V1
*
                 CALL DGEMM( 'Transpose', 'No transpose', N, K, M-K,
     $                       ONE, C, LDC, V, LDV, ONE, WORK, LDWORK )
              END IF
```

```
*
*             W := W * T'  or  W * T
*
              CALL DTRMM( 'Right', 'Lower', TRANST, 'Non-unit', N, K,
     $                    ONE, T, LDT, WORK, LDWORK )
*
*             C := C - V * W'
*
              IF( M.GT.K ) THEN
*
*                C1 := C1 - V1 * W'
*
                 CALL DGEMM( 'No transpose', 'Transpose', M-K, N, K,
     $                       -ONE, V, LDV, WORK, LDWORK, ONE, C, LDC )
              END IF
*
*             W := W * V2'
*
              CALL DTRMM( 'Right', 'Upper', 'Transpose', 'Unit', N, K,
     $                    ONE, V( M-K+1, 1 ), LDV, WORK, LDWORK )
*
*             C2 := C2 - W'
*
              DO 90 J = 1, K
                 DO 80 I = 1, N
                    C( M-K+J, I ) = C( M-K+J, I ) - WORK( I, J )
   80            CONTINUE
   90         CONTINUE
*
           ELSE IF( LSAME( SIDE, 'R' ) ) THEN
*
*             Form  C * H  or  C * H'  where  C = ( C1  C2 )
*
*             W := C * V  =  (C1*V1 + C2*V2)  (stored in WORK)
*
*             W := C2
*
              DO 100 J = 1, K
                 CALL DCOPY( M, C( 1, N-K+J ), 1, WORK( 1, J ), 1 )
  100         CONTINUE
*
*             W := W * V2
*
              CALL DTRMM( 'Right', 'Upper', 'No transpose', 'Unit', M,
     $                    K, ONE, V( N-K+1, 1 ), LDV, WORK, LDWORK )
              IF( N.GT.K ) THEN
*
*                W := W + C1 * V1
*
                 CALL DGEMM( 'No transpose', 'No transpose', M, K, N-K,
```

```
     $                       ONE, C, LDC, V, LDV, ONE, WORK, LDWORK )
               END IF
*
*              W := W * T  or  W * T'
*
               CALL DTRMM( 'Right', 'Lower', TRANS, 'Non-unit', M, K,
     $                     ONE, T, LDT, WORK, LDWORK )
*
*              C := C - W * V'
*
               IF( N.GT.K ) THEN
*
*                C1 := C1 - W * V1'
*
                 CALL DGEMM( 'No transpose', 'Transpose', M, N-K, K,
     $                       -ONE, WORK, LDWORK, V, LDV, ONE, C, LDC )
               END IF
*
*              W := W * V2'
*
               CALL DTRMM( 'Right', 'Upper', 'Transpose', 'Unit', M, K,
     $                     ONE, V( N-K+1, 1 ), LDV, WORK, LDWORK )
*
*              C2 := C2 - W
*
               DO 120 J = 1, K
                 DO 110 I = 1, M
                   C( I, N-K+J ) = C( I, N-K+J ) - WORK( I, J )
  110            CONTINUE
  120          CONTINUE
            END IF
         END IF
*
      ELSE IF( LSAME( STOREV, 'R' ) ) THEN
*
         IF( LSAME( DIRECT, 'F' ) ) THEN
*
*           Let  V =  ( V1  V2 )    (V1: first K columns)
*           where  V1  is unit upper triangular.
*
            IF( LSAME( SIDE, 'L' ) ) THEN
*
*              Form  H * C  or  H' * C  where  C = ( C1 )
*                                                  ( C2 )
*
*              W := C' * V'  =  (C1'*V1' + C2'*V2') (stored in WORK)
*
*              W := C1'
*
               DO 130 J = 1, K
```

```
                 CALL DCOPY( N, C( J, 1 ), LDC, WORK( 1, J ), 1 )
  130          CONTINUE
*
*              W := W * V1'
*
               CALL DTRMM( 'Right', 'Upper', 'Transpose', 'Unit', N, K,
     $                     ONE, V, LDV, WORK, LDWORK )
               IF( M.GT.K ) THEN
*
*                W := W + C2'*V2'
*
                 CALL DGEMM( 'Transpose', 'Transpose', N, K, M-K, ONE,
     $                       C( K+1, 1 ), LDC, V( 1, K+1 ), LDV, ONE,
     $                       WORK, LDWORK )
               END IF
*
*              W := W * T'  or  W * T
*
               CALL DTRMM( 'Right', 'Upper', TRANST, 'Non-unit', N, K,
     $                     ONE, T, LDT, WORK, LDWORK )
*
*              C := C - V' * W'
*
               IF( M.GT.K ) THEN
*
*                C2 := C2 - V2' * W'
*
                 CALL DGEMM( 'Transpose', 'Transpose', M-K, N, K, -ONE,
     $                       V( 1, K+1 ), LDV, WORK, LDWORK, ONE,
     $                       C( K+1, 1 ), LDC )
               END IF
*
*              W := W * V1
*
               CALL DTRMM( 'Right', 'Upper', 'No transpose', 'Unit', N,
     $                     K, ONE, V, LDV, WORK, LDWORK )
*
*              C1 := C1 - W'
*
               DO 150 J = 1, K
                 DO 140 I = 1, N
                    C( J, I ) = C( J, I ) - WORK( I, J )
  140            CONTINUE
  150          CONTINUE
*
            ELSE IF( LSAME( SIDE, 'R' ) ) THEN
*
*              Form  C * H  or  C * H'  where  C = ( C1  C2 )
*
*              W := C * V'  =  (C1*V1' + C2*V2')  (stored in WORK)
```

```
*
*              W := C1
*
               DO 160 J = 1, K
                  CALL DCOPY( M, C( 1, J ), 1, WORK( 1, J ), 1 )
  160          CONTINUE
*
*              W := W * V1'
*
               CALL DTRMM( 'Right', 'Upper', 'Transpose', 'Unit', M, K,
     $                     ONE, V, LDV, WORK, LDWORK )
               IF( N.GT.K ) THEN
*
*                 W := W + C2 * V2'
*
                  CALL DGEMM( 'No transpose', 'Transpose', M, K, N-K,
     $                        ONE, C( 1, K+1 ), LDC, V( 1, K+1 ), LDV,
     $                        ONE, WORK, LDWORK )
               END IF
*
*              W := W * T  or  W * T'
*
               CALL DTRMM( 'Right', 'Upper', TRANS, 'Non-unit', M, K,
     $                     ONE, T, LDT, WORK, LDWORK )
*
*              C := C - W * V
*
               IF( N.GT.K ) THEN
*
*                 C2 := C2 - W * V2
*
                  CALL DGEMM( 'No transpose', 'No transpose', M, N-K, K,
     $                        -ONE, WORK, LDWORK, V( 1, K+1 ), LDV, ONE,
     $                        C( 1, K+1 ), LDC )
               END IF
*
*              W := W * V1
*
               CALL DTRMM( 'Right', 'Upper', 'No transpose', 'Unit', M,
     $                     K, ONE, V, LDV, WORK, LDWORK )
*
*              C1 := C1 - W
*
               DO 180 J = 1, K
                  DO 170 I = 1, M
                     C( I, J ) = C( I, J ) - WORK( I, J )
  170             CONTINUE
  180          CONTINUE
*
            END IF
```

```
*
         ELSE
*
*           Let  V =  ( V1  V2 )    (V2: last K columns)
*           where  V2  is unit lower triangular.
*
            IF( LSAME( SIDE, 'L' ) ) THEN
*
*              Form  H * C  or  H' * C  where  C = ( C1 )
*                                                  ( C2 )
*
*              W := C' * V'  =  (C1'*V1' + C2'*V2') (stored in WORK)
*
*              W := C2'
*
               DO 190 J = 1, K
                  CALL DCOPY( N, C( M-K+J, 1 ), LDC, WORK( 1, J ), 1 )
  190          CONTINUE
*
*              W := W * V2'
*
               CALL DTRMM( 'Right', 'Lower', 'Transpose', 'Unit', N, K,
     $                     ONE, V( 1, M-K+1 ), LDV, WORK, LDWORK )
               IF( M.GT.K ) THEN
*
*                 W := W + C1'*V1'
*
                  CALL DGEMM( 'Transpose', 'Transpose', N, K, M-K, ONE,
     $                        C, LDC, V, LDV, ONE, WORK, LDWORK )
               END IF
*
*              W := W * T'  or  W * T
*
               CALL DTRMM( 'Right', 'Lower', TRANST, 'Non-unit', N, K,
     $                     ONE, T, LDT, WORK, LDWORK )
*
*              C := C - V' * W'
*
               IF( M.GT.K ) THEN
*
*                 C1 := C1 - V1' * W'
*
                  CALL DGEMM( 'Transpose', 'Transpose', M-K, N, K, -ONE,
     $                        V, LDV, WORK, LDWORK, ONE, C, LDC )
               END IF
*
*              W := W * V2
*
               CALL DTRMM( 'Right', 'Lower', 'No transpose', 'Unit', N,
     $                     K, ONE, V( 1, M-K+1 ), LDV, WORK, LDWORK )
```

```
*
*             C2 := C2 - W'
*
              DO 210 J = 1, K
                 DO 200 I = 1, N
                    C( M-K+J, I ) = C( M-K+J, I ) - WORK( I, J )
  200            CONTINUE
  210         CONTINUE
*
           ELSE IF( LSAME( SIDE, 'R' ) ) THEN
*
*             Form  C * H  or  C * H'  where  C = ( C1  C2 )
*
*             W := C * V'  =  (C1*V1' + C2*V2')  (stored in WORK)
*
*             W := C2
*
              DO 220 J = 1, K
                 CALL DCOPY( M, C( 1, N-K+J ), 1, WORK( 1, J ), 1 )
  220         CONTINUE
*
*             W := W * V2'
*
              CALL DTRMM( 'Right', 'Lower', 'Transpose', 'Unit', M, K,
     $                    ONE, V( 1, N-K+1 ), LDV, WORK, LDWORK )
              IF( N.GT.K ) THEN
*
*                W := W + C1 * V1'
*
                 CALL DGEMM( 'No transpose', 'Transpose', M, K, N-K,
     $                       ONE, C, LDC, V, LDV, ONE, WORK, LDWORK )
              END IF
*
*             W := W * T  or  W * T'
*
              CALL DTRMM( 'Right', 'Lower', TRANS, 'Non-unit', M, K,
     $                    ONE, T, LDT, WORK, LDWORK )
*
*             C := C - W * V
*
              IF( N.GT.K ) THEN
*
*                C1 := C1 - W * V1
*
                 CALL DGEMM( 'No transpose', 'No transpose', M, N-K, K,
     $                       -ONE, WORK, LDWORK, V, LDV, ONE, C, LDC )
              END IF
*
*             W := W * V2
*
```

```
              CALL DTRMM( 'Right', 'Lower', 'No transpose', 'Unit', M,
     $                    K, ONE, V( 1, N-K+1 ), LDV, WORK, LDWORK )
*
*             C1 := C1 - W
*
              DO 240 J = 1, K
                 DO 230 I = 1, M
                    C( I, N-K+J ) = C( I, N-K+J ) - WORK( I, J )
  230            CONTINUE
  240         CONTINUE
*
           END IF
*
        END IF
      END IF
*
      RETURN
*
*     End of DLARFB
*
      END
```

———-

90.161.4 lisp code

— dlarfb.lisp —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dlarfb (side trans direct storev m n k v ldv t$ ldt c ldc work ldwork)
    (declare (type (array double-float (*)) work c t$ v)
             (type (f2cl-lib:integer4) ldwork ldc ldt ldv k n m)
             (type (string *) storev direct trans side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (trans character trans-%data% trans-%offset%)
         (direct character direct-%data% direct-%offset%)
         (storev character storev-%data% storev-%offset%)
         (v double-float v-%data% v-%offset%)
         (t$ double-float t$-%data% t$-%offset%)
         (c double-float c-%data% c-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((i 0) (j 0)
             (transt
              (make-array '(1) :element-type 'character :initial-element #\ )))
```

```
(declare (type (f2cl-lib:integer4) i j)
         (type (string 1) transt))
(if (or (<= m 0) (<= n 0))
    (go end_label))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame trans "N")
    (declare (ignore var-1))
    (when var-0 (setf trans var-0))
    ret-val)
  (f2cl-lib:f2cl-set-string transt "T" (string 1)))
 (t (f2cl-lib:f2cl-set-string transt "N" (string 1))))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame storev "C")
    (declare (ignore var-1))
    (when var-0 (setf storev var-0))
    ret-val)
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame direct "F")
      (declare (ignore var-1))
      (when var-0 (setf direct var-0))
      ret-val)
    (cond
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame side "L")
        (declare (ignore var-1))
        (when var-0 (setf side var-0))
        ret-val)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j k) nil)
        (tagbody
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dcopy n
               (f2cl-lib:array-slice c-%data% double-float (j 1)
                                     ((1 ldc) (1 *)) c-%offset%)
               ldc
               (f2cl-lib:array-slice work-%data% double-float (1 j)
                                     ((1 ldwork) (1 *)) work-%offset%)
               1)
            (declare (ignore var-1 var-3 var-4))
            (when var-0 (setf n var-0))
            (when var-2 (setf ldc var-2)))
          label10))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (dtrmm "Right" "Lower" "No transpose" "Unit" n k one v ldv
           work ldwork)
```

```
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> m k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "Transpose" "No transpose" n k
       (f2cl-lib:int-sub m k) one
       (f2cl-lib:array-slice c-%data% double-float ((+ k 1) 1)
                             ((1 ldc) (1 *)) c-%offset%)
       ldc
       (f2cl-lib:array-slice v-%data% double-float ((+ k 1) 1)
                             ((1 ldv) (1 *)) v-%offset%)
       ldv one work ldwork)
    (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf k var-3))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldc var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldwork var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "Right" "Upper" transt "Non-unit" n k one t$ ldt work
     ldwork)
  (declare (ignore var-0 var-1 var-3 var-7 var-9))
  (when var-2 (setf transt var-2))
  (when var-4 (setf n var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> m k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "No transpose" "Transpose" (f2cl-lib:int-sub m k) n
       k (- one)
       (f2cl-lib:array-slice v-%data% double-float ((+ k 1) 1)
                             ((1 ldv) (1 *)) v-%offset%)
       ldv work ldwork one
       (f2cl-lib:array-slice c-%data% double-float ((+ k 1) 1)
```

```
                                ((1 ldc) (1 *)) c-%offset%)
        ldc)
      (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
      (when var-3 (setf n var-3))
      (when var-4 (setf k var-4))
      (when var-7 (setf ldv var-7))
      (when var-9 (setf ldwork var-9))
      (when var-10 (setf one var-10))
      (when var-12 (setf ldc var-12)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (dtrmm "Right" "Lower" "Transpose" "Unit" n k one v ldv work
       ldwork)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf k var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf ldv var-8))
    (when var-10 (setf ldwork var-10)))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j k) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref c-%data% (j i) ((1 ldc) (1 *))
                                c-%offset%)
                  (-
                   (f2cl-lib:fref c-%data% (j i) ((1 ldc) (1 *))
                                  c-%offset%)
                   (f2cl-lib:fref work-%data% (i j)
                                  ((1 ldwork) (1 *))
                                  work-%offset%)))
        label20))
     label30)))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame side "R")
    (declare (ignore var-1))
    (when var-0 (setf side var-0))
    ret-val)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j k) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy m
           (f2cl-lib:array-slice c-%data% double-float (1 j)
                                 ((1 ldc) (1 *)) c-%offset%)
           1
           (f2cl-lib:array-slice work-%data% double-float (1 j)
```

```
                                ((1 ldwork) (1 *)) work-%offset%)
          1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf m var-0)))
    label40))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "Right" "Lower" "No transpose" "Unit" m k one v ldv
     work ldwork)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf m var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> n k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "No transpose" "No transpose" m k
       (f2cl-lib:int-sub n k) one
       (f2cl-lib:array-slice c-%data% double-float
                             (1 (f2cl-lib:int-add k 1))
                             ((1 ldc) (1 *)) c-%offset%)
       ldc
       (f2cl-lib:array-slice v-%data% double-float ((+ k 1) 1)
                             ((1 ldv) (1 *)) v-%offset%)
       ldv one work ldwork)
    (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf k var-3))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldc var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldwork var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "Right" "Upper" trans "Non-unit" m k one t$ ldt work
     ldwork)
  (declare (ignore var-0 var-1 var-3 var-7 var-9))
  (when var-2 (setf trans var-2))
  (when var-4 (setf m var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldwork var-10)))
```

```
      (cond
       ((> n k)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12)
            (dgemm "No transpose" "Transpose" m (f2cl-lib:int-sub n k)
             k (- one) work ldwork
             (f2cl-lib:array-slice v-%data% double-float ((+ k 1) 1)
                                   ((1 ldv) (1 *)) v-%offset%)
             ldv one
             (f2cl-lib:array-slice c-%data% double-float
                                   (1 (f2cl-lib:int-add k 1))
                                   ((1 ldc) (1 *)) c-%offset%)
             ldc)
          (declare (ignore var-0 var-1 var-3 var-5 var-6 var-8 var-11))
          (when var-2 (setf m var-2))
          (when var-4 (setf k var-4))
          (when var-7 (setf ldwork var-7))
          (when var-9 (setf ldv var-9))
          (when var-10 (setf one var-10))
          (when var-12 (setf ldc var-12)))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (dtrmm "Right" "Lower" "Transpose" "Unit" m k one v ldv work
           ldwork)
        (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
        (when var-4 (setf m var-4))
        (when var-5 (setf k var-5))
        (when var-6 (setf one var-6))
        (when var-8 (setf ldv var-8))
        (when var-10 (setf ldwork var-10)))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j k) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                   c-%offset%)
                    (-
                     (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                    c-%offset%)
                     (f2cl-lib:fref work-%data% (i j)
                                    ((1 ldwork) (1 *))
                                    work-%offset%)))
             label50))
         label60)))))
    (t
     (cond
```

```
((multiple-value-bind (ret-val var-0 var-1)
     (lsame side "L")
   (declare (ignore var-1))
   (when var-0 (setf side var-0))
   ret-val)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j k) nil)
   (tagbody
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (dcopy n
          (f2cl-lib:array-slice c-%data% double-float
                                ((+ m (f2cl-lib:int-sub k) j) 1)
                                ((1 ldc) (1 *)) c-%offset%)
          ldc
          (f2cl-lib:array-slice work-%data% double-float (1 j)
                                ((1 ldwork) (1 *)) work-%offset%)
          1)
       (declare (ignore var-1 var-3 var-4))
       (when var-0 (setf n var-0))
       (when var-2 (setf ldc var-2)))
    label70))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (dtrmm "Right" "Upper" "No transpose" "Unit" n k one
      (f2cl-lib:array-slice v-%data% double-float
                            ((+ m (f2cl-lib:int-sub k) 1) 1)
                            ((1 ldv) (1 *)) v-%offset%)
      ldv work ldwork)
   (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
   (when var-4 (setf n var-4))
   (when var-5 (setf k var-5))
   (when var-6 (setf one var-6))
   (when var-8 (setf ldv var-8))
   (when var-10 (setf ldwork var-10)))
 (cond
  ((> m k)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10 var-11 var-12)
       (dgemm "Transpose" "No transpose" n k
        (f2cl-lib:int-sub m k) one c ldc v ldv one work ldwork)
     (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
     (when var-2 (setf n var-2))
     (when var-3 (setf k var-3))
     (when var-5 (setf one var-5))
     (when var-7 (setf ldc var-7))
     (when var-9 (setf ldv var-9))
     (when var-10 (setf one var-10))
     (when var-12 (setf ldwork var-12)))))
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "Right" "Lower" transt "Non-unit" n k one t$ ldt work
     ldwork)
  (declare (ignore var-0 var-1 var-3 var-7 var-9))
  (when var-2 (setf transt var-2))
  (when var-4 (setf n var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> m k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "No transpose" "Transpose" (f2cl-lib:int-sub m k) n
       k (- one) v ldv work ldwork one c ldc)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
    (when var-3 (setf n var-3))
    (when var-4 (setf k var-4))
    (when var-7 (setf ldv var-7))
    (when var-9 (setf ldwork var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldc var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "Right" "Upper" "Transpose" "Unit" n k one
     (f2cl-lib:array-slice v-%data% double-float
                           ((+ m (f2cl-lib:int-sub k) 1) 1)
                           ((1 ldv) (1 *)) v-%offset%)
     ldv work ldwork)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j k) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-sub m k) j)
                              i)
```

```
                                ((1 ldc) (1 *)) c-%offset%)
                        (-
                         (f2cl-lib:fref c-%data%
                                        ((f2cl-lib:int-add
                                          (f2cl-lib:int-sub m k) j)
                                         i)
                                        ((1 ldc) (1 *)) c-%offset%)
                         (f2cl-lib:fref work-%data% (i j)
                                        ((1 ldwork) (1 *))
                                        work-%offset%)))
            label80))
        label90)))
    ((multiple-value-bind (ret-val var-0 var-1)
         (lsame side "R")
       (declare (ignore var-1))
       (when var-0 (setf side var-0))
       ret-val)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j k) nil)
       (tagbody
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (dcopy m
              (f2cl-lib:array-slice c-%data% double-float
                                    (1
                                     (f2cl-lib:int-add
                                      (f2cl-lib:int-sub n k) j))
                                    ((1 ldc) (1 *)) c-%offset%)
              1
              (f2cl-lib:array-slice work-%data% double-float (1 j)
                                    ((1 ldwork) (1 *)) work-%offset%)
              1)
           (declare (ignore var-1 var-2 var-3 var-4))
           (when var-0 (setf m var-0)))
        label100))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10)
         (dtrmm "Right" "Upper" "No transpose" "Unit" m k one
          (f2cl-lib:array-slice v-%data% double-float
                                ((+ n (f2cl-lib:int-sub k) 1) 1)
                                ((1 ldv) (1 *)) v-%offset%)
          ldv work ldwork)
       (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
       (when var-4 (setf m var-4))
       (when var-5 (setf k var-5))
       (when var-6 (setf one var-6))
       (when var-8 (setf ldv var-8))
       (when var-10 (setf ldwork var-10)))
     (cond
      ((> n k)
```

```
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "No transpose" "No transpose" m k
       (f2cl-lib:int-sub n k) one c ldc v ldv one work ldwork)
    (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf k var-3))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldc var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldwork var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "Right" "Lower" trans "Non-unit" m k one t$ ldt work
     ldwork)
  (declare (ignore var-0 var-1 var-3 var-7 var-9))
  (when var-2 (setf trans var-2))
  (when var-4 (setf m var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> n k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "No transpose" "Transpose" m (f2cl-lib:int-sub n k)
       k (- one) work ldwork v ldv one c ldc)
    (declare (ignore var-0 var-1 var-3 var-5 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-4 (setf k var-4))
    (when var-7 (setf ldwork var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldc var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "Right" "Upper" "Transpose" "Unit" m k one
     (f2cl-lib:array-slice v-%data% double-float
                           ((+ n (f2cl-lib:int-sub k) 1) 1)
                           ((1 ldv) (1 *)) v-%offset%)
     ldv work ldwork)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf m var-4))
  (when var-5 (setf k var-5))
```

```
        (when var-6 (setf one var-6))
        (when var-8 (setf ldv var-8))
        (when var-10 (setf ldwork var-10)))
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j k) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref c-%data%
                                   (i
                                    (f2cl-lib:int-add
                                     (f2cl-lib:int-sub n k) j))
                                   ((1 ldc) (1 *)) c-%offset%)
                    (-
                     (f2cl-lib:fref c-%data%
                                    (i
                                     (f2cl-lib:int-add
                                      (f2cl-lib:int-sub n k) j))
                                    ((1 ldc) (1 *)) c-%offset%)
                     (f2cl-lib:fref work-%data% (i j)
                                    ((1 ldwork) (1 *))
                                    work-%offset%)))
            label110))
        label120)))))))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame storev "R")
   (declare (ignore var-1))
   (when var-0 (setf storev var-0))
   ret-val)
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame direct "F")
     (declare (ignore var-1))
     (when var-0 (setf direct var-0))
     ret-val)
   (cond
    ((multiple-value-bind (ret-val var-0 var-1)
         (lsame side "L")
       (declare (ignore var-1))
       (when var-0 (setf side var-0))
       ret-val)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j k) nil)
       (tagbody
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (dcopy n
              (f2cl-lib:array-slice c-%data% double-float (j 1)
                                    ((1 ldc) (1 *)) c-%offset%)
              ldc
```

```
          (f2cl-lib:array-slice work-%data% double-float (1 j)
                                ((1 ldwork) (1 *)) work-%offset%)
          1)
        (declare (ignore var-1 var-3 var-4))
        (when var-0 (setf n var-0))
        (when var-2 (setf ldc var-2)))
    label130))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "Right" "Upper" "Transpose" "Unit" n k one v ldv work
     ldwork)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> m k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "Transpose" "Transpose" n k (f2cl-lib:int-sub m k)
       one
       (f2cl-lib:array-slice c-%data% double-float ((+ k 1) 1)
                             ((1 ldc) (1 *)) c-%offset%)
       ldc
       (f2cl-lib:array-slice v-%data% double-float
                             (1 (f2cl-lib:int-add k 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv one work ldwork)
    (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf k var-3))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldc var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldwork var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "Right" "Upper" transt "Non-unit" n k one t$ ldt work
     ldwork)
  (declare (ignore var-0 var-1 var-3 var-7 var-9))
  (when var-2 (setf transt var-2))
  (when var-4 (setf n var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
```

```
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> m k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "Transpose" "Transpose" (f2cl-lib:int-sub m k) n k
       (- one)
       (f2cl-lib:array-slice v-%data% double-float
                             (1 (f2cl-lib:int-add k 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv work ldwork one
       (f2cl-lib:array-slice c-%data% double-float ((+ k 1) 1)
                             ((1 ldc) (1 *)) c-%offset%)
       ldc)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
    (when var-3 (setf n var-3))
    (when var-4 (setf k var-4))
    (when var-7 (setf ldv var-7))
    (when var-9 (setf ldwork var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldc var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "Right" "Upper" "No transpose" "Unit" n k one v ldv
     work ldwork)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j k) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data% (j i) ((1 ldc) (1 *))
                             c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (j i) ((1 ldc) (1 *))
                                c-%offset%)
                 (f2cl-lib:fref work-%data% (i j)
                                ((1 ldwork) (1 *))
                                work-%offset%)))
       label140))
   label150)))
```

```
((multiple-value-bind (ret-val var-0 var-1)
     (lsame side "R")
   (declare (ignore var-1))
   (when var-0 (setf side var-0))
   ret-val)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j k) nil)
   (tagbody
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (dcopy m
          (f2cl-lib:array-slice c-%data% double-float (1 j)
                                ((1 ldc) (1 *)) c-%offset%)
          1
          (f2cl-lib:array-slice work-%data% double-float (1 j)
                                ((1 ldwork) (1 *)) work-%offset%)
          1)
       (declare (ignore var-1 var-2 var-3 var-4))
       (when var-0 (setf m var-0)))
    label160))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (dtrmm "Right" "Upper" "Transpose" "Unit" m k one v ldv work
      ldwork)
   (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
   (when var-4 (setf m var-4))
   (when var-5 (setf k var-5))
   (when var-6 (setf one var-6))
   (when var-8 (setf ldv var-8))
   (when var-10 (setf ldwork var-10)))
 (cond
  ((> n k)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10 var-11 var-12)
       (dgemm "No transpose" "Transpose" m k
        (f2cl-lib:int-sub n k) one
        (f2cl-lib:array-slice c-%data% double-float
                              (1 (f2cl-lib:int-add k 1))
                              ((1 ldc) (1 *)) c-%offset%)
        ldc
        (f2cl-lib:array-slice v-%data% double-float
                              (1 (f2cl-lib:int-add k 1))
                              ((1 ldv) (1 *)) v-%offset%)
        ldv one work ldwork)
     (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
     (when var-2 (setf m var-2))
     (when var-3 (setf k var-3))
     (when var-5 (setf one var-5))
     (when var-7 (setf ldc var-7))
```

```
    (when var-9 (setf ldv var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldwork var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "Right" "Upper" trans "Non-unit" m k one t$ ldt work
     ldwork)
  (declare (ignore var-0 var-1 var-3 var-7 var-9))
  (when var-2 (setf trans var-2))
  (when var-4 (setf m var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> n k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "No transpose" "No transpose" m
       (f2cl-lib:int-sub n k) k (- one) work ldwork
       (f2cl-lib:array-slice v-%data% double-float
                             (1 (f2cl-lib:int-add k 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv one
       (f2cl-lib:array-slice c-%data% double-float
                             (1 (f2cl-lib:int-add k 1))
                             ((1 ldc) (1 *)) c-%offset%)
       ldc)
    (declare (ignore var-0 var-1 var-3 var-5 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-4 (setf k var-4))
    (when var-7 (setf ldwork var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldc var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "Right" "Upper" "No transpose" "Unit" m k one v ldv
     work ldwork)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf m var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j k) nil)
```

```
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                 c-%offset%)
                  (-
                   (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                  c-%offset%)
                   (f2cl-lib:fref work-%data% (i j)
                                  ((1 ldwork) (1 *))
                                  work-%offset%)))
           label170))
      label180)))))
(t
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame side "L")
     (declare (ignore var-1))
     (when var-0 (setf side var-0))
     ret-val)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j k) nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (dcopy n
            (f2cl-lib:array-slice c-%data% double-float
                                  ((+ m (f2cl-lib:int-sub k) j) 1)
                                  ((1 ldc) (1 *)) c-%offset%)
            ldc
            (f2cl-lib:array-slice work-%data% double-float (1 j)
                                  ((1 ldwork) (1 *)) work-%offset%)
            1)
         (declare (ignore var-1 var-3 var-4))
         (when var-0 (setf n var-0))
         (when var-2 (setf ldc var-2)))
      label190))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (dtrmm "Right" "Lower" "Transpose" "Unit" n k one
        (f2cl-lib:array-slice v-%data% double-float
                              (1
                               (f2cl-lib:int-add
                                (f2cl-lib:int-sub m k) 1))
                              ((1 ldv) (1 *)) v-%offset%)
        ldv work ldwork)
     (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
     (when var-4 (setf n var-4))
     (when var-5 (setf k var-5))
```

```
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> m k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "Transpose" "Transpose" n k (f2cl-lib:int-sub m k)
       one c ldc v ldv one work ldwork)
    (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf k var-3))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldc var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldwork var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "Right" "Lower" transt "Non-unit" n k one t$ ldt work
     ldwork)
  (declare (ignore var-0 var-1 var-3 var-7 var-9))
  (when var-2 (setf transt var-2))
  (when var-4 (setf n var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> m k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "Transpose" "Transpose" (f2cl-lib:int-sub m k) n k
       (- one) v ldv work ldwork one c ldc)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
    (when var-3 (setf n var-3))
    (when var-4 (setf k var-4))
    (when var-7 (setf ldv var-7))
    (when var-9 (setf ldwork var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldc var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "Right" "Lower" "No transpose" "Unit" n k one
     (f2cl-lib:array-slice v-%data% double-float
                           (1
```

```
                                (f2cl-lib:int-add
                                 (f2cl-lib:int-sub m k) 1))
                               ((1 ldv) (1 *)) v-%offset%)
      ldv work ldwork)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf k var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf ldv var-8))
    (when var-10 (setf ldwork var-10)))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j k) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref c-%data%
                               ((f2cl-lib:int-add
                                 (f2cl-lib:int-sub m k) j)
                                i)
                               ((1 ldc) (1 *)) c-%offset%)
                  (-
                   (f2cl-lib:fref c-%data%
                                  ((f2cl-lib:int-add
                                    (f2cl-lib:int-sub m k) j)
                                   i)
                                  ((1 ldc) (1 *)) c-%offset%)
                   (f2cl-lib:fref work-%data% (i j)
                                  ((1 ldwork) (1 *))
                                  work-%offset%)))
        label200))
    label210)))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame side "R")
   (declare (ignore var-1))
   (when var-0 (setf side var-0))
   ret-val)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j k) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy m
           (f2cl-lib:array-slice c-%data% double-float
                                 (1
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-sub n k) j))
                                 ((1 ldc) (1 *)) c-%offset%)
           1
           (f2cl-lib:array-slice work-%data% double-float (1 j)
                                 ((1 ldwork) (1 *)) work-%offset%)
```

```
          1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf m var-0)))
    label220))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "Right" "Lower" "Transpose" "Unit" m k one
     (f2cl-lib:array-slice v-%data% double-float
                           (1
                            (f2cl-lib:int-add
                             (f2cl-lib:int-sub n k) 1))
                           ((1 ldv) (1 *)) v-%offset%)
     ldv work ldwork)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf m var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> n k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "No transpose" "Transpose" m k
       (f2cl-lib:int-sub n k) one c ldc v ldv one work ldwork)
    (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf k var-3))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldc var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldwork var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "Right" "Lower" trans "Non-unit" m k one t$ ldt work
     ldwork)
  (declare (ignore var-0 var-1 var-3 var-7 var-9))
  (when var-2 (setf trans var-2))
  (when var-4 (setf m var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> n k)
  (multiple-value-bind
```

```
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (dgemm "No transpose" "No transpose" m
       (f2cl-lib:int-sub n k) k (- one) work ldwork v ldv one c
       ldc)
    (declare (ignore var-0 var-1 var-3 var-5 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-4 (setf k var-4))
    (when var-7 (setf ldwork var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldc var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (dtrmm "Right" "Lower" "No transpose" "Unit" m k one
     (f2cl-lib:array-slice v-%data% double-float
                           (1
                            (f2cl-lib:int-add
                             (f2cl-lib:int-sub n k) 1))
                           ((1 ldv) (1 *)) v-%offset%)
     ldv work ldwork)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf m var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j k) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data%
                             (i
                              (f2cl-lib:int-add
                               (f2cl-lib:int-sub n k) j))
                             ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data%
                              (i
                               (f2cl-lib:int-add
                                (f2cl-lib:int-sub n k) j))
                              ((1 ldc) (1 *)) c-%offset%)
               (f2cl-lib:fref work-%data% (i j)
                              ((1 ldwork) (1 *))
                              work-%offset%)))
        label230))
   label240))))))))
```

```
        (go end_label)
       end_label
        (return
         (values side trans direct storev m n k nil ldv nil ldt nil ldc nil
                 ldwork))))))
```

———-

90.162 dlarfg

90.162.1 Axiom unit tests

— **dlarfg.input** —

```
)set break resume
)sys rm -f dlarfg.output
)spool dlarfg.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.162.2 Axiom help page

— **dlarfg.help** —

```
====================================================================
dlarfg examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DLARFG  -  a  real elementary reflector H of order n, such that   H * (
       alpha ) = ( beta ), H' * H = I

SYNOPSIS
```

```
      SUBROUTINE DLARFG( N, ALPHA, X, INCX, TAU )

          INTEGER        INCX, N

          DOUBLE         PRECISION ALPHA, TAU

          DOUBLE         PRECISION X( * )

 Purpose
 =======

  DLARFG generates a real elementary reflector H of order n, such
  that

        H * ( alpha ) = ( beta ),   H' * H = I.
            (   x   )   (   0  )

  where alpha and beta are scalars, and x is an (n-1)-element real
  vector. H is represented in the form

        H = I - tau * ( 1 ) * ( 1 v' ) ,
                      ( v )

  where tau is a real scalar and v is a real (n-1)-element
  vector.

  If the elements of x are all zero, then tau = 0 and H is taken to be
  the unit matrix.

  Otherwise  1 <= tau <= 2.

 Arguments
 =========

  N       (input) INTEGER
          The order of the elementary reflector.

  ALPHA   (input/output) DOUBLE PRECISION
          On entry, the value alpha.
          On exit, it is overwritten with the value beta.

  X       (input/output) DOUBLE PRECISION array, dimension
                         (1+(N-2)*abs(INCX))
          On entry, the vector x.
          On exit, it is overwritten with the vector v.

  INCX    (input) INTEGER
          The increment between elements of X. INCX > 0.

  TAU     (output) DOUBLE PRECISION
```

```
         The value tau.
```

———-

90.162.3 fortran code

— **dlarfg.f** —

```
      SUBROUTINE DLARFG( N, ALPHA, X, INCX, TAU )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     September 30, 1994
*
*     .. Scalar Arguments ..
      INTEGER            INCX, N
      DOUBLE PRECISION   ALPHA, TAU
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   X( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            J, KNT
      DOUBLE PRECISION   BETA, RSAFMN, SAFMIN, XNORM
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH, DLAPY2, DNRM2
      EXTERNAL           DLAMCH, DLAPY2, DNRM2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SIGN
*     ..
*     .. External Subroutines ..
      EXTERNAL           DSCAL
*     ..
*     .. Executable Statements ..
*
      IF( N.LE.1 ) THEN
         TAU = ZERO
```

```
         RETURN
      END IF
*
      XNORM = DNRM2( N-1, X, INCX )
*
      IF( XNORM.EQ.ZERO ) THEN
*
*        H  =  I
*
         TAU = ZERO
      ELSE
*
*        general case
*
         BETA = -SIGN( DLAPY2( ALPHA, XNORM ), ALPHA )
         SAFMIN = DLAMCH( 'S' ) / DLAMCH( 'E' )
         IF( ABS( BETA ).LT.SAFMIN ) THEN
*
*           XNORM, BETA may be inaccurate; scale X and recompute them
*
            RSAFMN = ONE / SAFMIN
            KNT = 0
   10       CONTINUE
            KNT = KNT + 1
            CALL DSCAL( N-1, RSAFMN, X, INCX )
            BETA = BETA*RSAFMN
            ALPHA = ALPHA*RSAFMN
            IF( ABS( BETA ).LT.SAFMIN )
     $         GO TO 10
*
*           New BETA is at most 1, at least SAFMIN
*
            XNORM = DNRM2( N-1, X, INCX )
            BETA = -SIGN( DLAPY2( ALPHA, XNORM ), ALPHA )
            TAU = ( BETA-ALPHA ) / BETA
            CALL DSCAL( N-1, ONE / ( ALPHA-BETA ), X, INCX )
*
*           If ALPHA is subnormal, it may lose relative accuracy
*
            ALPHA = BETA
            DO 20 J = 1, KNT
               ALPHA = ALPHA*SAFMIN
   20       CONTINUE
         ELSE
            TAU = ( BETA-ALPHA ) / BETA
            CALL DSCAL( N-1, ONE / ( ALPHA-BETA ), X, INCX )
            ALPHA = BETA
         END IF
      END IF
*
```

```
      RETURN
*
*     End of DLARFG
*
      END
```

———-

90.162.4 lisp code

— dlarfg.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dlarfg (n alpha x incx tau)
    (declare (type (array double-float (*)) x)
             (type (double-float) tau alpha)
             (type (f2cl-lib:integer4) incx n))
    (f2cl-lib:with-multi-array-data
        ((x double-float x-%data% x-%offset%))
      (prog ((beta 0.0d0) (rsafmn 0.0d0) (safmin 0.0d0) (xnorm 0.0d0) (j 0)
             (knt 0))
        (declare (type (double-float) beta rsafmn safmin xnorm)
                 (type (f2cl-lib:integer4) j knt))
        (cond ((<= n 1) (setf tau zero) (go end_label)))
        (setf xnorm
                (multiple-value-bind (ret-val var-0 var-1 var-2)
                    (dnrm2 (f2cl-lib:int-sub n 1) x incx)
                  (declare (ignore var-0 var-1))
                  (when var-2 (setf incx var-2))
                  ret-val))
        (cond ((= xnorm zero) (setf tau zero))
              (t (setf beta (- (f2cl-lib:sign (dlapy2 alpha xnorm) alpha)))
               (setf safmin (/ (dlamch "S") (dlamch "E"))) (setf knt 0)
               (cond
                ((< (abs beta) safmin)
                 (tagbody
                   (setf rsafmn (/ one safmin))
                  label10
                   (setf knt (f2cl-lib:int-add knt 1))
                   (multiple-value-bind (var-0 var-1 var-2 var-3)
                       (dscal (f2cl-lib:int-sub n 1) rsafmn x incx)
                     (declare (ignore var-0 var-2))
                     (when var-1 (setf rsafmn var-1))
                     (when var-3 (setf incx var-3)))
```

```
                (setf beta (* beta rsafmn))
                (setf alpha (* alpha rsafmn))
                (if (< (abs beta) safmin)
                    (go label10))
                (setf xnorm
                        (multiple-value-bind (ret-val var-0 var-1 var-2)
                            (dnrm2 (f2cl-lib:int-sub n 1) x incx)
                          (declare (ignore var-0 var-1))
                          (when var-2 (setf incx var-2))
                          ret-val))
                (setf beta
                        (- (f2cl-lib:sign (dlapy2 alpha xnorm) alpha))))))
            (setf tau (/ (- beta alpha) beta))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (dscal (f2cl-lib:int-sub n 1) (/ one (- alpha beta)) x incx)
              (declare (ignore var-0 var-1 var-2))
              (when var-3 (setf incx var-3)))
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j knt) nil)
              (tagbody (setf beta (* beta safmin)) label20))
            (setf alpha beta)))
        (go end_label)
       end_label
        (return (values nil alpha nil incx tau))))))
```

———-

90.163 dlarf

90.163.1 Axiom unit tests

— **dlarf.input** —

```
)set break resume
)sys rm -f dlarf.output
)spool dlarf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.163.2 Axiom help page

— dlarf.help —

```
=====================================================================
dlarf examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

NAME
       DLARF  -  a real elementary reflector H to a real m by n matrix C, from
       either the left or the right

SYNOPSIS
       SUBROUTINE DLARF( SIDE, M, N, V, INCV, TAU, C, LDC, WORK )

           CHARACTER     SIDE

           INTEGER       INCV, LDC, M, N

           DOUBLE        PRECISION TAU

           DOUBLE        PRECISION C( LDC, * ), V( * ), WORK( * )

 Purpose
 =======

  DLARF applies a real elementary reflector H to a real m by n matrix
  C, from either the left or the right. H is represented in the form

        H = I - tau * v * v'

  where tau is a real scalar and v is a real vector.

  If tau = 0, then H is taken to be the unit matrix.

 Arguments
 =========

  SIDE    (input) CHARACTER*1
          = 'L': form  H * C
          = 'R': form  C * H

  M       (input) INTEGER
          The number of rows of the matrix C.
```

```
N       (input) INTEGER
        The number of columns of the matrix C.

V       (input) DOUBLE PRECISION array, dimension
                   (1 + (M-1)*abs(INCV)) if SIDE = 'L'
                or (1 + (N-1)*abs(INCV)) if SIDE = 'R'
        The vector v in the representation of H. V is not used if
        TAU = 0.

INCV    (input) INTEGER
        The increment between elements of v. INCV <> 0.

TAU     (input) DOUBLE PRECISION
        The value tau in the representation of H.

C       (input/output) DOUBLE PRECISION array, dimension (LDC,N)
        On entry, the m by n matrix C.
        On exit, C is overwritten by the matrix H * C if SIDE = 'L',
        or C * H if SIDE = 'R'.

LDC     (input) INTEGER
        The leading dimension of the array C. LDC >= max(1,M).

WORK    (workspace) DOUBLE PRECISION array, dimension
                       (N) if SIDE = 'L'
                    or (M) if SIDE = 'R'
```

———-

90.163.3 fortran code

— **dlarf.f** —

```
      SUBROUTINE DLARF( SIDE, M, N, V, INCV, TAU, C, LDC, WORK )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     February 29, 1992
*
*     .. Scalar Arguments ..
      CHARACTER          SIDE
      INTEGER            INCV, LDC, M, N
      DOUBLE PRECISION   TAU
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   C( LDC, * ), V( * ), WORK( * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEMV, DGER
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Executable Statements ..
*
      IF( LSAME( SIDE, 'L' ) ) THEN
*
*        Form  H * C
*
         IF( TAU.NE.ZERO ) THEN
*
*           w := C' * v
*
            CALL DGEMV( 'Transpose', M, N, ONE, C, LDC, V, INCV, ZERO,
     $                  WORK, 1 )
*
*           C := C - v * w'
*
            CALL DGER( M, N, -TAU, V, INCV, WORK, 1, C, LDC )
         END IF
      ELSE
*
*        Form  C * H
*
         IF( TAU.NE.ZERO ) THEN
*
*           w := C * v
*
            CALL DGEMV( 'No transpose', M, N, ONE, C, LDC, V, INCV,
     $                  ZERO, WORK, 1 )
*
*           C := C - w * v'
*
            CALL DGER( M, N, -TAU, WORK, 1, V, INCV, C, LDC )
         END IF
      END IF
      RETURN
*
```

```
*     End of DLARF
*
      END
```

———-

90.163.4 lisp code

— **dlarf.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dlarf (side m n v incv tau c ldc work)
    (declare (type (double-float) tau)
             (type (array double-float (*)) work c v)
             (type (f2cl-lib:integer4) ldc incv n m)
             (type (string *) side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (v double-float v-%data% v-%offset%)
         (c double-float c-%data% c-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((i 0) (lastv 0) (lastc 0) (applyleft nil))
        (declare (type (f2cl-lib:integer4) i lastv lastc)
                 (type f2cl-lib:logical applyleft))
        (setf applyleft
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame side "L")
                  (declare (ignore var-1))
                  (when var-0 (setf side var-0))
                  ret-val))
        (setf lastv 0)
        (setf lastc 0)
        (cond
         ((/= tau zero)
          (tagbody
            (cond (applyleft (setf lastv m)) (t (setf lastv n)))
            (cond
             ((> incv 0)
              (setf i
                      (f2cl-lib:int-add 1
                                        (f2cl-lib:int-mul
                                         (f2cl-lib:int-sub lastv 1) incv))))
             (t (setf i 1)))
           label100000
```

```
      (if (not
           (and (> lastv 0)
                (= (f2cl-lib:fref v-%data% (i) ((1 *)) v-%offset%)
                   zero)))
          (go label100001))
      (setf lastv (f2cl-lib:int-sub lastv 1))
      (setf i (f2cl-lib:int-sub i incv))
      (go label100000)
     label100001
      (cond
       (applyleft
        (setf lastc
                (multiple-value-bind (ret-val var-0 var-1 var-2 var-3)
                    (iladlc lastv n c ldc)
                  (declare (ignore var-2))
                  (when var-0 (setf lastv var-0))
                  (when var-1 (setf n var-1))
                  (when var-3 (setf ldc var-3))
                  ret-val)))
       (t
        (setf lastc
                (multiple-value-bind (ret-val var-0 var-1 var-2 var-3)
                    (iladlr m lastv c ldc)
                  (declare (ignore var-2))
                  (when var-0 (setf m var-0))
                  (when var-1 (setf lastv var-1))
                  (when var-3 (setf ldc var-3))
                  ret-val)))))))
  (cond
   (applyleft
    (cond
     ((> lastv 0)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (dgemv "Transpose" lastv lastc one c ldc v incv zero work 1)
        (declare (ignore var-0 var-4 var-6 var-9 var-10))
        (when var-1 (setf lastv var-1))
        (when var-2 (setf lastc var-2))
        (when var-3 (setf one var-3))
        (when var-5 (setf ldc var-5))
        (when var-7 (setf incv var-7))
        (when var-8 (setf zero var-8)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dger lastv lastc (- tau) v incv work 1 c ldc)
        (declare (ignore var-2 var-3 var-5 var-6 var-7))
        (when var-0 (setf lastv var-0))
        (when var-1 (setf lastc var-1))
        (when var-4 (setf incv var-4))
```

```
            (when var-8 (setf ldc var-8))))))
        (t
         (cond
          ((> lastv 0)
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                var-10)
               (dgemv "No transpose" lastc lastv one c ldc v incv zero work 1)
             (declare (ignore var-0 var-4 var-6 var-9 var-10))
             (when var-1 (setf lastc var-1))
             (when var-2 (setf lastv var-2))
             (when var-3 (setf one var-3))
             (when var-5 (setf ldc var-5))
             (when var-7 (setf incv var-7))
             (when var-8 (setf zero var-8)))
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
               (dger lastc lastv (- tau) work 1 v incv c ldc)
             (declare (ignore var-2 var-3 var-4 var-5 var-7))
             (when var-0 (setf lastc var-0))
             (when var-1 (setf lastv var-1))
             (when var-6 (setf incv var-6))
             (when var-8 (setf ldc var-8)))))))
       (go end_label)
      end_label
       (return (values side m n nil incv nil nil ldc nil))))))
```

———-

90.164 dlarft

90.164.1 Axiom unit tests

— **dlarft.input** —

```
)set break resume
)sys rm -f dlarft.output
)spool dlarft.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.164.2 Axiom help page

— dlarft.help —

```
====================================================================
dlarft examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DLARFT  -  the triangular factor T of a real block reflector H of order
       n, which is defined as a product of k elementary reflectors

SYNOPSIS
       SUBROUTINE DLARFT( DIRECT, STOREV, N, K, V, LDV, TAU, T, LDT )

           CHARACTER      DIRECT, STOREV

           INTEGER        K, LDT, LDV, N

           DOUBLE         PRECISION T( LDT, * ), TAU( * ), V( LDV, * )

 Purpose
 =======

  DLARFT forms the triangular factor T of a real block reflector H
  of order n, which is defined as a product of k elementary reflectors.

  If DIRECT = 'F', H = H(1) H(2) . . . H(k) and T is upper triangular;

  If DIRECT = 'B', H = H(k) . . . H(2) H(1) and T is lower triangular.

  If STOREV = 'C', the vector which defines the elementary reflector
  H(i) is stored in the i-th column of the array V, and

     H  =  I - V * T * V'

  If STOREV = 'R', the vector which defines the elementary reflector
  H(i) is stored in the i-th row of the array V, and

     H  =  I - V' * T * V

 Arguments
 =========

  DIRECT  (input) CHARACTER*1
```

```
        Specifies the order in which the elementary reflectors are
        multiplied to form the block reflector:
        = 'F': H = H(1) H(2) . . . H(k) (Forward)
        = 'B': H = H(k) . . . H(2) H(1) (Backward)

STOREV  (input) CHARACTER*1
        Specifies how the vectors which define the elementary
        reflectors are stored (see also Further Details):
        = 'C': columnwise
        = 'R': rowwise

N       (input) INTEGER
        The order of the block reflector H. N >= 0.

K       (input) INTEGER
        The order of the triangular factor T (= the number of
        elementary reflectors). K >= 1.

V       (input/output) DOUBLE PRECISION array, dimension
                            (LDV,K) if STOREV = 'C'
                            (LDV,N) if STOREV = 'R'
        The matrix V. See further details.

LDV     (input) INTEGER
        The leading dimension of the array V.
        If STOREV = 'C', LDV >= max(1,N); if STOREV = 'R', LDV >= K.

TAU     (input) DOUBLE PRECISION array, dimension (K)
        TAU(i) must contain the scalar factor of the elementary
        reflector H(i).

T       (output) DOUBLE PRECISION array, dimension (LDT,K)
        The k by k triangular factor T of the block reflector.
        If DIRECT = 'F', T is upper triangular; if DIRECT = 'B', T is
        lower triangular. The rest of the array is not used.

LDT     (input) INTEGER
        The leading dimension of the array T. LDT >= K.

Further Details
===============

The shape of the matrix V and the storage of the vectors which define
the H(i) is best illustrated by the following example with n = 5 and
k = 3. The elements equal to 1 are not stored; the corresponding
array elements are modified but restored on exit. The rest of the
array is not used.

DIRECT = 'F' and STOREV = 'C':         DIRECT = 'F' and STOREV = 'R':
```

```
              V = (  1       )                 V = (  1 v1 v1 v1 v1 )
                  ( v1  1    )                     (     1 v2 v2 v2 )
                  ( v1 v2  1 )                     (        1 v3 v3 )
                  ( v1 v2 v3 )
                  ( v1 v2 v3 )

  DIRECT = 'B' and STOREV = 'C':         DIRECT = 'B' and STOREV = 'R':

              V = ( v1 v2 v3 )                 V = ( v1 v1  1       )
                  ( v1 v2 v3 )                     ( v2 v2 v2  1    )
                  (  1 v2 v3 )                     ( v3 v3 v3 v3  1 )
                  (     1 v3 )
                  (        1 )
```

———-

90.164.3 fortran code

— dlarft.f —

```
      SUBROUTINE DLARFT( DIRECT, STOREV, N, K, V, LDV, TAU, T, LDT )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     February 29, 1992
*
*     .. Scalar Arguments ..
      CHARACTER          DIRECT, STOREV
      INTEGER            K, LDT, LDV, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   T( LDT, * ), TAU( * ), V( LDV, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   VII
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEMV, DTRMV
```

```
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
      IF( LSAME( DIRECT, 'F' ) ) THEN
         DO 20 I = 1, K
            IF( TAU( I ).EQ.ZERO ) THEN
*
*              H(i)  =  I
*
               DO 10 J = 1, I
                  T( J, I ) = ZERO
   10          CONTINUE
            ELSE
*
*              general case
*
               VII = V( I, I )
               V( I, I ) = ONE
               IF( LSAME( STOREV, 'C' ) ) THEN
*
*                T(1:i-1,i) := - tau(i) * V(i:n,1:i-1)' * V(i:n,i)
*
                 CALL DGEMV( 'Transpose', N-I+1, I-1, -TAU( I ),
     $                       V( I, 1 ), LDV, V( I, I ), 1, ZERO,
     $                       T( 1, I ), 1 )
               ELSE
*
*                T(1:i-1,i) := - tau(i) * V(1:i-1,i:n) * V(i,i:n)'
*
                 CALL DGEMV( 'No transpose', I-1, N-I+1, -TAU( I ),
     $                       V( 1, I ), LDV, V( I, I ), LDV, ZERO,
     $                       T( 1, I ), 1 )
               END IF
               V( I, I ) = VII
*
*              T(1:i-1,i) := T(1:i-1,1:i-1) * T(1:i-1,i)
*
               CALL DTRMV( 'Upper', 'No transpose', 'Non-unit', I-1, T,
     $                     LDT, T( 1, I ), 1 )
               T( I, I ) = TAU( I )
            END IF
```

```
   20    CONTINUE
      ELSE
         DO 40 I = K, 1, -1
            IF( TAU( I ).EQ.ZERO ) THEN
*
*              H(i)  =  I
*
               DO 30 J = I, K
                  T( J, I ) = ZERO
   30          CONTINUE
            ELSE
*
*              general case
*
               IF( I.LT.K ) THEN
                  IF( LSAME( STOREV, 'C' ) ) THEN
                     VII = V( N-K+I, I )
                     V( N-K+I, I ) = ONE
*
*                    T(i+1:k,i) :=
*                            - tau(i) * V(1:n-k+i,i+1:k)' * V(1:n-k+i,i)
*
                     CALL DGEMV( 'Transpose', N-K+I, K-I, -TAU( I ),
     $                           V( 1, I+1 ), LDV, V( 1, I ), 1, ZERO,
     $                           T( I+1, I ), 1 )
                     V( N-K+I, I ) = VII
                  ELSE
                     VII = V( I, N-K+I )
                     V( I, N-K+I ) = ONE
*
*                    T(i+1:k,i) :=
*                            - tau(i) * V(i+1:k,1:n-k+i) * V(i,1:n-k+i)'
*
                     CALL DGEMV( 'No transpose', K-I, N-K+I, -TAU( I ),
     $                           V( I+1, 1 ), LDV, V( I, 1 ), LDV, ZERO,
     $                           T( I+1, I ), 1 )
                     V( I, N-K+I ) = VII
                  END IF
*
*                 T(i+1:k,i) := T(i+1:k,i+1:k) * T(i+1:k,i)
*
                  CALL DTRMV( 'Lower', 'No transpose', 'Non-unit', K-I,
     $                        T( I+1, I+1 ), LDT, T( I+1, I ), 1 )
               END IF
               T( I, I ) = TAU( I )
            END IF
   40    CONTINUE
      END IF
      RETURN
*
```

```
*     End of DLARFT
*
      END
```

———-

90.164.4 lisp code

— dlarft.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dlarft (direct storev n k v ldv tau t$ ldt)
    (declare (type (array double-float (*)) t$ tau v)
             (type (f2cl-lib:integer4) ldt ldv k n)
             (type (string *) storev direct))
    (f2cl-lib:with-multi-array-data
        ((direct character direct-%data% direct-%offset%)
         (storev character storev-%data% storev-%offset%)
         (v double-float v-%data% v-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (t$ double-float t$-%data% t$-%offset%))
      (prog ((i 0) (j 0) (prevlastv 0) (lastv 0))
        (declare (type (f2cl-lib:integer4) i j prevlastv lastv))
        (if (= n 0)
            (go end_label))
        (cond
         ((multiple-value-bind (ret-val var-0 var-1)
              (lsame direct "F")
            (declare (ignore var-1))
            (when var-0 (setf direct var-0))
            ret-val)
          (setf prevlastv n)
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i k) nil)
            (tagbody
              (setf prevlastv
                      (max (the f2cl-lib:integer4 i)
                           (the f2cl-lib:integer4 prevlastv)))
              (cond
               ((= (f2cl-lib:fref tau (i) ((1 *))) zero)
                (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                              ((> j i) nil)
                  (tagbody
                    (setf (f2cl-lib:fref t$-%data% (j i) ((1 ldt) (1 *))
```

```
                                t$-%offset%)
                     zero)
            label100001)))
        (t
         (cond
          ((multiple-value-bind (ret-val var-0 var-1)
               (lsame storev "C")
             (declare (ignore var-1))
             (when var-0 (setf storev var-0))
             ret-val)
           (f2cl-lib:fdo (lastv n
                          (f2cl-lib:int-add lastv (f2cl-lib:int-sub 1)))
                         ((> lastv (f2cl-lib:int-add i 1)) nil)
             (tagbody
               (if (/=
                    (f2cl-lib:fref v-%data% (lastv i) ((1 ldv) (1 *))
                                   v-%offset%)
                    zero)
                   (go f2cl-lib::exit))
              label100002))
           (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                         ((> j
                             (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                          nil)
             (tagbody
               (setf (f2cl-lib:fref t$-%data% (j i) ((1 ldt) (1 *))
                                    t$-%offset%)
                     (*
                      (-
                       (f2cl-lib:fref tau-%data% (i) ((1 *))
                                      tau-%offset%))
                      (f2cl-lib:fref v-%data% (i j) ((1 ldv) (1 *))
                                     v-%offset%)))
              label100003))
           (setf j
                 (min (the f2cl-lib:integer4 lastv)
                      (the f2cl-lib:integer4 prevlastv)))
           (multiple-value-bind
               (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                var-9 var-10)
               (dgemv "Transpose" (f2cl-lib:int-sub j i)
                (f2cl-lib:int-sub i 1)
                (- (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
                (f2cl-lib:array-slice v-%data% double-float ((+ i 1) 1)
                                      ((1 ldv) (1 *)) v-%offset%)
                ldv
                (f2cl-lib:array-slice v-%data% double-float ((+ i 1) i)
                                      ((1 ldv) (1 *)) v-%offset%)
                1 one
                (f2cl-lib:array-slice t$-%data% double-float (1 i)
```

```
                              ((1 ldt) (1 *)) t$-%offset%)
      1)
   (declare
    (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7 var-9
     var-10))
   (when var-5 (setf ldv var-5))
   (when var-8 (setf one var-8))))
(t
 (f2cl-lib:fdo (lastv n
                (f2cl-lib:int-add lastv (f2cl-lib:int-sub 1)))
               ((> lastv (f2cl-lib:int-add i 1)) nil)
   (tagbody
     (if (/=
          (f2cl-lib:fref v-%data% (i lastv) ((1 ldv) (1 *))
                         v-%offset%)
          zero)
         (go f2cl-lib::exit))
    label100004))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j
                   (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (setf (f2cl-lib:fref t$-%data% (j i) ((1 ldt) (1 *))
                          t$-%offset%)
             (*
              (-
               (f2cl-lib:fref tau-%data% (i) ((1 *))
                              tau-%offset%))
              (f2cl-lib:fref v-%data% (j i) ((1 ldv) (1 *))
                             v-%offset%)))
    label100005))
 (setf j
        (min (the f2cl-lib:integer4 lastv)
             (the f2cl-lib:integer4 prevlastv)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10)
     (dgemv "No transpose" (f2cl-lib:int-sub i 1)
      (f2cl-lib:int-sub j i)
      (- (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
      (f2cl-lib:array-slice v-%data% double-float
                            (1 (f2cl-lib:int-add i 1))
                            ((1 ldv) (1 *)) v-%offset%)
      ldv
      (f2cl-lib:array-slice v-%data% double-float
                            (i (f2cl-lib:int-add i 1))
                            ((1 ldv) (1 *)) v-%offset%)
      ldv one
      (f2cl-lib:array-slice t$-%data% double-float (1 i)
```

```
                                        ((1 ldt) (1 *)) t$-%offset%)
                      1)
                  (declare
                   (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9 var-10))
                  (when var-5 (setf ldv var-5))
                  (when var-7 (setf ldv var-7))
                  (when var-8 (setf one var-8)))))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
                  (dtrmv "Upper" "No transpose" "Non-unit"
                   (f2cl-lib:int-sub i 1) t$ ldt
                   (f2cl-lib:array-slice t$-%data% double-float (1 i)
                                         ((1 ldt) (1 *)) t$-%offset%)
                   1)
                (declare (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7))
                (when var-5 (setf ldt var-5)))
              (setf (f2cl-lib:fref t$-%data% (i i) ((1 ldt) (1 *))
                                   t$-%offset%)
                    (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
              (cond
               ((> i 1)
                (setf prevlastv
                        (max (the f2cl-lib:integer4 prevlastv)
                             (the f2cl-lib:integer4 lastv))))
               (t (setf prevlastv lastv)))))
          label100000)))
      (t (setf prevlastv 1)
       (f2cl-lib:fdo (i k (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                     ((> i 1) nil)
         (tagbody
           (cond
            ((= (f2cl-lib:fref tau (i) ((1 *))) zero)
             (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                           ((> j k) nil)
               (tagbody
                 (setf (f2cl-lib:fref t$-%data% (j i) ((1 ldt) (1 *))
                                      t$-%offset%)
                       zero)
               label100007)))
            (t
             (cond
              ((< i k)
               (cond
                ((multiple-value-bind (ret-val var-0 var-1)
                     (lsame storev "C")
                   (declare (ignore var-1))
                   (when var-0 (setf storev var-0))
                   ret-val)
                 (f2cl-lib:fdo (lastv 1 (f2cl-lib:int-add lastv 1))
                               ((> lastv
```

```
                    (f2cl-lib:int-add i
                                      (f2cl-lib:int-sub 1)))
                nil)
  (tagbody
    (if (/=
         (f2cl-lib:fref v-%data% (lastv i) ((1 ldv) (1 *))
                        v-%offset%)
         zero)
        (go f2cl-lib::exit))
   label100008))
(f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
               (f2cl-lib:int-add j 1))
              ((> j k) nil)
  (tagbody
    (setf (f2cl-lib:fref t$-%data% (j i) ((1 ldt) (1 *))
                         t$-%offset%)
          (*
           (-
            (f2cl-lib:fref tau-%data% (i) ((1 *))
                           tau-%offset%))
           (f2cl-lib:fref v-%data%
                          ((f2cl-lib:int-add
                            (f2cl-lib:int-sub n k) i)
                          j)
                          ((1 ldv) (1 *)) v-%offset%)))
   label100009))
(setf j
        (max (the f2cl-lib:integer4 lastv)
             (the f2cl-lib:integer4 prevlastv)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10)
    (dgemv "Transpose"
     (f2cl-lib:int-sub
      (f2cl-lib:int-add (f2cl-lib:int-sub n k) i) j)
     (f2cl-lib:int-sub k i)
     (-
      (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
     (f2cl-lib:array-slice v-%data% double-float
                           (j (f2cl-lib:int-add i 1))
                           ((1 ldv) (1 *)) v-%offset%)
     ldv
     (f2cl-lib:array-slice v-%data% double-float (j i)
                           ((1 ldv) (1 *)) v-%offset%)
     1 one
     (f2cl-lib:array-slice t$-%data% double-float
                           ((+ i 1) i) ((1 ldt) (1 *))
                           t$-%offset%)
     1)
  (declare
```

```
    (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7 var-9
     var-10))
   (when var-5 (setf ldv var-5))
   (when var-8 (setf one var-8))))
(t
 (f2cl-lib:fdo (lastv 1 (f2cl-lib:int-add lastv 1))
               ((> lastv
                   (f2cl-lib:int-add i
                                     (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (if (/=
          (f2cl-lib:fref v-%data% (i lastv) ((1 ldv) (1 *))
                         v-%offset%)
          zero)
         (go f2cl-lib::exit))
    label100010))
 (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                (f2cl-lib:int-add j 1))
               ((> j k) nil)
   (tagbody
     (setf (f2cl-lib:fref t$-%data% (j i) ((1 ldt) (1 *))
                          t$-%offset%)
             (*
              (-
               (f2cl-lib:fref tau-%data% (i) ((1 *))
                              tau-%offset%))
              (f2cl-lib:fref v-%data%
                             (j
                              (f2cl-lib:int-add
                               (f2cl-lib:int-sub n k) i))
                             ((1 ldv) (1 *)) v-%offset%)))
    label100011))
 (setf j
        (max (the f2cl-lib:integer4 lastv)
             (the f2cl-lib:integer4 prevlastv)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10)
     (dgemv "No transpose" (f2cl-lib:int-sub k i)
      (f2cl-lib:int-sub
       (f2cl-lib:int-add (f2cl-lib:int-sub n k) i) j)
      (-
       (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
      (f2cl-lib:array-slice v-%data% double-float
                            ((+ i 1) j) ((1 ldv) (1 *))
                            v-%offset%)
      ldv
      (f2cl-lib:array-slice v-%data% double-float (i j)
                            ((1 ldv) (1 *)) v-%offset%)
```

```
                    ldv one
                    (f2cl-lib:array-slice t$-%data% double-float
                                          ((+ i 1) i) ((1 ldt) (1 *))
                                          t$-%offset%)
                    1)
                  (declare
                   (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9
                    var-10))
                  (when var-5 (setf ldv var-5))
                  (when var-7 (setf ldv var-7))
                  (when var-8 (setf one var-8)))))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
                  (dtrmv "Lower" "No transpose" "Non-unit"
                   (f2cl-lib:int-sub k i)
                   (f2cl-lib:array-slice t$-%data% double-float
                                         ((+ i 1) (f2cl-lib:int-add i 1))
                                         ((1 ldt) (1 *)) t$-%offset%)
                   ldt
                   (f2cl-lib:array-slice t$-%data% double-float ((+ i 1) i)
                                         ((1 ldt) (1 *)) t$-%offset%)
                   1)
                (declare
                 (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7))
                (when var-5 (setf ldt var-5)))
              (cond
               ((> i 1)
                (setf prevlastv
                        (min (the f2cl-lib:integer4 prevlastv)
                             (the f2cl-lib:integer4 lastv))))
               (t (setf prevlastv lastv)))))
            (setf (f2cl-lib:fref t$-%data% (i i) ((1 ldt) (1 *))
                                 t$-%offset%)
                  (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))))
        label100006))))
  (go end_label)
 end_label
  (return (values direct storev nil nil nil ldv nil nil ldt))))))
```

———-

90.165 dlarfx

90.165.1 Axiom unit tests

— dlarfx.input —

```
)set break resume
)sys rm -f dlarfx.output
)spool dlarfx.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

90.165.2 Axiom help page

— dlarfx.help —

```
======================================================================
dlarfx examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DLARFX  - a real elementary reflector H to a real m by n matrix C, from
       either the left or the right

SYNOPSIS
       SUBROUTINE DLARFX( SIDE, M, N, V, TAU, C, LDC, WORK )

           CHARACTER      SIDE

           INTEGER        LDC, M, N

           DOUBLE         PRECISION TAU

           DOUBLE         PRECISION C( LDC, * ), V( * ), WORK( * )

 Purpose
 =======

  DLARFX applies a real elementary reflector H to a real m by n
  matrix C, from either the left or the right. H is represented in the
  form

        H = I - tau * v * v'
```

```
 where tau is a real scalar and v is a real vector.

 If tau = 0, then H is taken to be the unit matrix

 This version uses inline code if H has order < 11.

Arguments
=========

 SIDE    (input) CHARACTER*1
         = 'L': form  H * C
         = 'R': form  C * H

 M       (input) INTEGER
         The number of rows of the matrix C.

 N       (input) INTEGER
         The number of columns of the matrix C.

 V       (input) DOUBLE PRECISION array, dimension (M) if SIDE = 'L'
                                    or (N) if SIDE = 'R'
         The vector v in the representation of H.

 TAU     (input) DOUBLE PRECISION
         The value tau in the representation of H.

 C       (input/output) DOUBLE PRECISION array, dimension (LDC,N)
         On entry, the m by n matrix C.
         On exit, C is overwritten by the matrix H * C if SIDE = 'L',
         or C * H if SIDE = 'R'.

 LDC     (input) INTEGER
         The leading dimension of the array C. LDA >= (1,M).

 WORK    (workspace) DOUBLE PRECISION array, dimension
                     (N) if SIDE = 'L'
                     or (M) if SIDE = 'R'
         WORK is not referenced if H has order < 11.
```

———-

90.165.3 fortran code

— **dlarfx.f** —

```
      SUBROUTINE DLARFX( SIDE, M, N, V, TAU, C, LDC, WORK )
*
```

```
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     February 29, 1992
*
*     .. Scalar Arguments ..
      CHARACTER          SIDE
      INTEGER            LDC, M, N
      DOUBLE PRECISION   TAU
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   C( LDC, * ), V( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            J
      DOUBLE PRECISION   SUM, T1, T10, T2, T3, T4, T5, T6, T7, T8, T9,
     $                   V1, V10, V2, V3, V4, V5, V6, V7, V8, V9
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DGEMV, DGER
*     ..
*     .. Executable Statements ..
*
      IF( TAU.EQ.ZERO )
     $   RETURN
      IF( LSAME( SIDE, 'L' ) ) THEN
*
*        Form  H * C, where H has order m.
*
         GO TO ( 10, 30, 50, 70, 90, 110, 130, 150,
     $           170, 190 )M
*
*        Code for general M
*
*        w := C'*v
*
         CALL DGEMV( 'Transpose', M, N, ONE, C, LDC, V, 1, ZERO, WORK,
     $               1 )
*
```

```
*        C := C - tau * v * w'
*
         CALL DGER( M, N, -TAU, V, 1, WORK, 1, C, LDC )
         GO TO 410
   10    CONTINUE
*
*        Special code for 1 x 1 Householder
*
         T1 = ONE - TAU*V( 1 )*V( 1 )
         DO 20 J = 1, N
            C( 1, J ) = T1*C( 1, J )
   20    CONTINUE
         GO TO 410
   30    CONTINUE
*
*        Special code for 2 x 2 Householder
*
         V1 = V( 1 )
         T1 = TAU*V1
         V2 = V( 2 )
         T2 = TAU*V2
         DO 40 J = 1, N
            SUM = V1*C( 1, J ) + V2*C( 2, J )
            C( 1, J ) = C( 1, J ) - SUM*T1
            C( 2, J ) = C( 2, J ) - SUM*T2
   40    CONTINUE
         GO TO 410
   50    CONTINUE
*
*        Special code for 3 x 3 Householder
*
         V1 = V( 1 )
         T1 = TAU*V1
         V2 = V( 2 )
         T2 = TAU*V2
         V3 = V( 3 )
         T3 = TAU*V3
         DO 60 J = 1, N
            SUM = V1*C( 1, J ) + V2*C( 2, J ) + V3*C( 3, J )
            C( 1, J ) = C( 1, J ) - SUM*T1
            C( 2, J ) = C( 2, J ) - SUM*T2
            C( 3, J ) = C( 3, J ) - SUM*T3
   60    CONTINUE
         GO TO 410
   70    CONTINUE
*
*        Special code for 4 x 4 Householder
*
         V1 = V( 1 )
         T1 = TAU*V1
```

```
         V2 = V( 2 )
         T2 = TAU*V2
         V3 = V( 3 )
         T3 = TAU*V3
         V4 = V( 4 )
         T4 = TAU*V4
         DO 80 J = 1, N
            SUM = V1*C( 1, J ) + V2*C( 2, J ) + V3*C( 3, J ) +
     $            V4*C( 4, J )
            C( 1, J ) = C( 1, J ) - SUM*T1
            C( 2, J ) = C( 2, J ) - SUM*T2
            C( 3, J ) = C( 3, J ) - SUM*T3
            C( 4, J ) = C( 4, J ) - SUM*T4
   80    CONTINUE
         GO TO 410
   90    CONTINUE
*
*        Special code for 5 x 5 Householder
*
         V1 = V( 1 )
         T1 = TAU*V1
         V2 = V( 2 )
         T2 = TAU*V2
         V3 = V( 3 )
         T3 = TAU*V3
         V4 = V( 4 )
         T4 = TAU*V4
         V5 = V( 5 )
         T5 = TAU*V5
         DO 100 J = 1, N
            SUM = V1*C( 1, J ) + V2*C( 2, J ) + V3*C( 3, J ) +
     $            V4*C( 4, J ) + V5*C( 5, J )
            C( 1, J ) = C( 1, J ) - SUM*T1
            C( 2, J ) = C( 2, J ) - SUM*T2
            C( 3, J ) = C( 3, J ) - SUM*T3
            C( 4, J ) = C( 4, J ) - SUM*T4
            C( 5, J ) = C( 5, J ) - SUM*T5
  100    CONTINUE
         GO TO 410
  110    CONTINUE
*
*        Special code for 6 x 6 Householder
*
         V1 = V( 1 )
         T1 = TAU*V1
         V2 = V( 2 )
         T2 = TAU*V2
         V3 = V( 3 )
         T3 = TAU*V3
         V4 = V( 4 )
```

```
         T4 = TAU*V4
         V5 = V( 5 )
         T5 = TAU*V5
         V6 = V( 6 )
         T6 = TAU*V6
         DO 120 J = 1, N
            SUM = V1*C( 1, J ) + V2*C( 2, J ) + V3*C( 3, J ) +
     $            V4*C( 4, J ) + V5*C( 5, J ) + V6*C( 6, J )
            C( 1, J ) = C( 1, J ) - SUM*T1
            C( 2, J ) = C( 2, J ) - SUM*T2
            C( 3, J ) = C( 3, J ) - SUM*T3
            C( 4, J ) = C( 4, J ) - SUM*T4
            C( 5, J ) = C( 5, J ) - SUM*T5
            C( 6, J ) = C( 6, J ) - SUM*T6
  120    CONTINUE
         GO TO 410
  130    CONTINUE
*
*        Special code for 7 x 7 Householder
*
         V1 = V( 1 )
         T1 = TAU*V1
         V2 = V( 2 )
         T2 = TAU*V2
         V3 = V( 3 )
         T3 = TAU*V3
         V4 = V( 4 )
         T4 = TAU*V4
         V5 = V( 5 )
         T5 = TAU*V5
         V6 = V( 6 )
         T6 = TAU*V6
         V7 = V( 7 )
         T7 = TAU*V7
         DO 140 J = 1, N
            SUM = V1*C( 1, J ) + V2*C( 2, J ) + V3*C( 3, J ) +
     $            V4*C( 4, J ) + V5*C( 5, J ) + V6*C( 6, J ) +
     $            V7*C( 7, J )
            C( 1, J ) = C( 1, J ) - SUM*T1
            C( 2, J ) = C( 2, J ) - SUM*T2
            C( 3, J ) = C( 3, J ) - SUM*T3
            C( 4, J ) = C( 4, J ) - SUM*T4
            C( 5, J ) = C( 5, J ) - SUM*T5
            C( 6, J ) = C( 6, J ) - SUM*T6
            C( 7, J ) = C( 7, J ) - SUM*T7
  140    CONTINUE
         GO TO 410
  150    CONTINUE
*
*        Special code for 8 x 8 Householder
```

```
*
         V1 = V( 1 )
         T1 = TAU*V1
         V2 = V( 2 )
         T2 = TAU*V2
         V3 = V( 3 )
         T3 = TAU*V3
         V4 = V( 4 )
         T4 = TAU*V4
         V5 = V( 5 )
         T5 = TAU*V5
         V6 = V( 6 )
         T6 = TAU*V6
         V7 = V( 7 )
         T7 = TAU*V7
         V8 = V( 8 )
         T8 = TAU*V8
         DO 160 J = 1, N
            SUM = V1*C( 1, J ) + V2*C( 2, J ) + V3*C( 3, J ) +
     $            V4*C( 4, J ) + V5*C( 5, J ) + V6*C( 6, J ) +
     $            V7*C( 7, J ) + V8*C( 8, J )
            C( 1, J ) = C( 1, J ) - SUM*T1
            C( 2, J ) = C( 2, J ) - SUM*T2
            C( 3, J ) = C( 3, J ) - SUM*T3
            C( 4, J ) = C( 4, J ) - SUM*T4
            C( 5, J ) = C( 5, J ) - SUM*T5
            C( 6, J ) = C( 6, J ) - SUM*T6
            C( 7, J ) = C( 7, J ) - SUM*T7
            C( 8, J ) = C( 8, J ) - SUM*T8
  160    CONTINUE
         GO TO 410
  170    CONTINUE
*
*        Special code for 9 x 9 Householder
*
         V1 = V( 1 )
         T1 = TAU*V1
         V2 = V( 2 )
         T2 = TAU*V2
         V3 = V( 3 )
         T3 = TAU*V3
         V4 = V( 4 )
         T4 = TAU*V4
         V5 = V( 5 )
         T5 = TAU*V5
         V6 = V( 6 )
         T6 = TAU*V6
         V7 = V( 7 )
         T7 = TAU*V7
         V8 = V( 8 )
```

```
         T8 = TAU*V8
         V9 = V( 9 )
         T9 = TAU*V9
         DO 180 J = 1, N
            SUM = V1*C( 1, J ) + V2*C( 2, J ) + V3*C( 3, J ) +
     $            V4*C( 4, J ) + V5*C( 5, J ) + V6*C( 6, J ) +
     $            V7*C( 7, J ) + V8*C( 8, J ) + V9*C( 9, J )
            C( 1, J ) = C( 1, J ) - SUM*T1
            C( 2, J ) = C( 2, J ) - SUM*T2
            C( 3, J ) = C( 3, J ) - SUM*T3
            C( 4, J ) = C( 4, J ) - SUM*T4
            C( 5, J ) = C( 5, J ) - SUM*T5
            C( 6, J ) = C( 6, J ) - SUM*T6
            C( 7, J ) = C( 7, J ) - SUM*T7
            C( 8, J ) = C( 8, J ) - SUM*T8
            C( 9, J ) = C( 9, J ) - SUM*T9
  180    CONTINUE
         GO TO 410
  190    CONTINUE
*
*        Special code for 10 x 10 Householder
*
         V1 = V( 1 )
         T1 = TAU*V1
         V2 = V( 2 )
         T2 = TAU*V2
         V3 = V( 3 )
         T3 = TAU*V3
         V4 = V( 4 )
         T4 = TAU*V4
         V5 = V( 5 )
         T5 = TAU*V5
         V6 = V( 6 )
         T6 = TAU*V6
         V7 = V( 7 )
         T7 = TAU*V7
         V8 = V( 8 )
         T8 = TAU*V8
         V9 = V( 9 )
         T9 = TAU*V9
         V10 = V( 10 )
         T10 = TAU*V10
         DO 200 J = 1, N
            SUM = V1*C( 1, J ) + V2*C( 2, J ) + V3*C( 3, J ) +
     $            V4*C( 4, J ) + V5*C( 5, J ) + V6*C( 6, J ) +
     $            V7*C( 7, J ) + V8*C( 8, J ) + V9*C( 9, J ) +
     $            V10*C( 10, J )
            C( 1, J ) = C( 1, J ) - SUM*T1
            C( 2, J ) = C( 2, J ) - SUM*T2
            C( 3, J ) = C( 3, J ) - SUM*T3
```

```
            C( 4, J ) = C( 4, J ) - SUM*T4
            C( 5, J ) = C( 5, J ) - SUM*T5
            C( 6, J ) = C( 6, J ) - SUM*T6
            C( 7, J ) = C( 7, J ) - SUM*T7
            C( 8, J ) = C( 8, J ) - SUM*T8
            C( 9, J ) = C( 9, J ) - SUM*T9
            C( 10, J ) = C( 10, J ) - SUM*T10
  200    CONTINUE
         GO TO 410
      ELSE
*
*        Form  C * H, where H has order n.
*
         GO TO ( 210, 230, 250, 270, 290, 310, 330, 350,
     $           370, 390 )N
*
*        Code for general N
*
*        w := C * v
*
         CALL DGEMV( 'No transpose', M, N, ONE, C, LDC, V, 1, ZERO,
     $               WORK, 1 )
*
*        C := C - tau * w * v'
*
         CALL DGER( M, N, -TAU, WORK, 1, V, 1, C, LDC )
         GO TO 410
  210    CONTINUE
*
*        Special code for 1 x 1 Householder
*
         T1 = ONE - TAU*V( 1 )*V( 1 )
         DO 220 J = 1, M
            C( J, 1 ) = T1*C( J, 1 )
  220    CONTINUE
         GO TO 410
  230    CONTINUE
*
*        Special code for 2 x 2 Householder
*
         V1 = V( 1 )
         T1 = TAU*V1
         V2 = V( 2 )
         T2 = TAU*V2
         DO 240 J = 1, M
            SUM = V1*C( J, 1 ) + V2*C( J, 2 )
            C( J, 1 ) = C( J, 1 ) - SUM*T1
            C( J, 2 ) = C( J, 2 ) - SUM*T2
  240    CONTINUE
         GO TO 410
```

```
  250    CONTINUE
*
*        Special code for 3 x 3 Householder
*
         V1 = V( 1 )
         T1 = TAU*V1
         V2 = V( 2 )
         T2 = TAU*V2
         V3 = V( 3 )
         T3 = TAU*V3
         DO 260 J = 1, M
            SUM = V1*C( J, 1 ) + V2*C( J, 2 ) + V3*C( J, 3 )
            C( J, 1 ) = C( J, 1 ) - SUM*T1
            C( J, 2 ) = C( J, 2 ) - SUM*T2
            C( J, 3 ) = C( J, 3 ) - SUM*T3
  260    CONTINUE
         GO TO 410
  270    CONTINUE
*
*        Special code for 4 x 4 Householder
*
         V1 = V( 1 )
         T1 = TAU*V1
         V2 = V( 2 )
         T2 = TAU*V2
         V3 = V( 3 )
         T3 = TAU*V3
         V4 = V( 4 )
         T4 = TAU*V4
         DO 280 J = 1, M
            SUM = V1*C( J, 1 ) + V2*C( J, 2 ) + V3*C( J, 3 ) +
     $            V4*C( J, 4 )
            C( J, 1 ) = C( J, 1 ) - SUM*T1
            C( J, 2 ) = C( J, 2 ) - SUM*T2
            C( J, 3 ) = C( J, 3 ) - SUM*T3
            C( J, 4 ) = C( J, 4 ) - SUM*T4
  280    CONTINUE
         GO TO 410
  290    CONTINUE
*
*        Special code for 5 x 5 Householder
*
         V1 = V( 1 )
         T1 = TAU*V1
         V2 = V( 2 )
         T2 = TAU*V2
         V3 = V( 3 )
         T3 = TAU*V3
         V4 = V( 4 )
         T4 = TAU*V4
```

```
         V5 = V( 5 )
         T5 = TAU*V5
         DO 300 J = 1, M
            SUM = V1*C( J, 1 ) + V2*C( J, 2 ) + V3*C( J, 3 ) +
     $            V4*C( J, 4 ) + V5*C( J, 5 )
            C( J, 1 ) = C( J, 1 ) - SUM*T1
            C( J, 2 ) = C( J, 2 ) - SUM*T2
            C( J, 3 ) = C( J, 3 ) - SUM*T3
            C( J, 4 ) = C( J, 4 ) - SUM*T4
            C( J, 5 ) = C( J, 5 ) - SUM*T5
  300    CONTINUE
         GO TO 410
  310    CONTINUE
*
*        Special code for 6 x 6 Householder
*
         V1 = V( 1 )
         T1 = TAU*V1
         V2 = V( 2 )
         T2 = TAU*V2
         V3 = V( 3 )
         T3 = TAU*V3
         V4 = V( 4 )
         T4 = TAU*V4
         V5 = V( 5 )
         T5 = TAU*V5
         V6 = V( 6 )
         T6 = TAU*V6
         DO 320 J = 1, M
            SUM = V1*C( J, 1 ) + V2*C( J, 2 ) + V3*C( J, 3 ) +
     $            V4*C( J, 4 ) + V5*C( J, 5 ) + V6*C( J, 6 )
            C( J, 1 ) = C( J, 1 ) - SUM*T1
            C( J, 2 ) = C( J, 2 ) - SUM*T2
            C( J, 3 ) = C( J, 3 ) - SUM*T3
            C( J, 4 ) = C( J, 4 ) - SUM*T4
            C( J, 5 ) = C( J, 5 ) - SUM*T5
            C( J, 6 ) = C( J, 6 ) - SUM*T6
  320    CONTINUE
         GO TO 410
  330    CONTINUE
*
*        Special code for 7 x 7 Householder
*
         V1 = V( 1 )
         T1 = TAU*V1
         V2 = V( 2 )
         T2 = TAU*V2
         V3 = V( 3 )
         T3 = TAU*V3
         V4 = V( 4 )
```

```
         T4 = TAU*V4
         V5 = V( 5 )
         T5 = TAU*V5
         V6 = V( 6 )
         T6 = TAU*V6
         V7 = V( 7 )
         T7 = TAU*V7
         DO 340 J = 1, M
            SUM = V1*C( J, 1 ) + V2*C( J, 2 ) + V3*C( J, 3 ) +
     $            V4*C( J, 4 ) + V5*C( J, 5 ) + V6*C( J, 6 ) +
     $            V7*C( J, 7 )
            C( J, 1 ) = C( J, 1 ) - SUM*T1
            C( J, 2 ) = C( J, 2 ) - SUM*T2
            C( J, 3 ) = C( J, 3 ) - SUM*T3
            C( J, 4 ) = C( J, 4 ) - SUM*T4
            C( J, 5 ) = C( J, 5 ) - SUM*T5
            C( J, 6 ) = C( J, 6 ) - SUM*T6
            C( J, 7 ) = C( J, 7 ) - SUM*T7
  340    CONTINUE
         GO TO 410
  350    CONTINUE
*
*        Special code for 8 x 8 Householder
*
         V1 = V( 1 )
         T1 = TAU*V1
         V2 = V( 2 )
         T2 = TAU*V2
         V3 = V( 3 )
         T3 = TAU*V3
         V4 = V( 4 )
         T4 = TAU*V4
         V5 = V( 5 )
         T5 = TAU*V5
         V6 = V( 6 )
         T6 = TAU*V6
         V7 = V( 7 )
         T7 = TAU*V7
         V8 = V( 8 )
         T8 = TAU*V8
         DO 360 J = 1, M
            SUM = V1*C( J, 1 ) + V2*C( J, 2 ) + V3*C( J, 3 ) +
     $            V4*C( J, 4 ) + V5*C( J, 5 ) + V6*C( J, 6 ) +
     $            V7*C( J, 7 ) + V8*C( J, 8 )
            C( J, 1 ) = C( J, 1 ) - SUM*T1
            C( J, 2 ) = C( J, 2 ) - SUM*T2
            C( J, 3 ) = C( J, 3 ) - SUM*T3
            C( J, 4 ) = C( J, 4 ) - SUM*T4
            C( J, 5 ) = C( J, 5 ) - SUM*T5
            C( J, 6 ) = C( J, 6 ) - SUM*T6
```

```
            C( J, 7 ) = C( J, 7 ) - SUM*T7
            C( J, 8 ) = C( J, 8 ) - SUM*T8
  360    CONTINUE
         GO TO 410
  370    CONTINUE
*
*        Special code for 9 x 9 Householder
*
         V1 = V( 1 )
         T1 = TAU*V1
         V2 = V( 2 )
         T2 = TAU*V2
         V3 = V( 3 )
         T3 = TAU*V3
         V4 = V( 4 )
         T4 = TAU*V4
         V5 = V( 5 )
         T5 = TAU*V5
         V6 = V( 6 )
         T6 = TAU*V6
         V7 = V( 7 )
         T7 = TAU*V7
         V8 = V( 8 )
         T8 = TAU*V8
         V9 = V( 9 )
         T9 = TAU*V9
         DO 380 J = 1, M
            SUM = V1*C( J, 1 ) + V2*C( J, 2 ) + V3*C( J, 3 ) +
     $            V4*C( J, 4 ) + V5*C( J, 5 ) + V6*C( J, 6 ) +
     $            V7*C( J, 7 ) + V8*C( J, 8 ) + V9*C( J, 9 )
            C( J, 1 ) = C( J, 1 ) - SUM*T1
            C( J, 2 ) = C( J, 2 ) - SUM*T2
            C( J, 3 ) = C( J, 3 ) - SUM*T3
            C( J, 4 ) = C( J, 4 ) - SUM*T4
            C( J, 5 ) = C( J, 5 ) - SUM*T5
            C( J, 6 ) = C( J, 6 ) - SUM*T6
            C( J, 7 ) = C( J, 7 ) - SUM*T7
            C( J, 8 ) = C( J, 8 ) - SUM*T8
            C( J, 9 ) = C( J, 9 ) - SUM*T9
  380    CONTINUE
         GO TO 410
  390    CONTINUE
*
*        Special code for 10 x 10 Householder
*
         V1 = V( 1 )
         T1 = TAU*V1
         V2 = V( 2 )
         T2 = TAU*V2
         V3 = V( 3 )
```

```
         T3 = TAU*V3
         V4 = V( 4 )
         T4 = TAU*V4
         V5 = V( 5 )
         T5 = TAU*V5
         V6 = V( 6 )
         T6 = TAU*V6
         V7 = V( 7 )
         T7 = TAU*V7
         V8 = V( 8 )
         T8 = TAU*V8
         V9 = V( 9 )
         T9 = TAU*V9
         V10 = V( 10 )
         T10 = TAU*V10
         DO 400 J = 1, M
            SUM = V1*C( J, 1 ) + V2*C( J, 2 ) + V3*C( J, 3 ) +
     $            V4*C( J, 4 ) + V5*C( J, 5 ) + V6*C( J, 6 ) +
     $            V7*C( J, 7 ) + V8*C( J, 8 ) + V9*C( J, 9 ) +
     $            V10*C( J, 10 )
            C( J, 1 ) = C( J, 1 ) - SUM*T1
            C( J, 2 ) = C( J, 2 ) - SUM*T2
            C( J, 3 ) = C( J, 3 ) - SUM*T3
            C( J, 4 ) = C( J, 4 ) - SUM*T4
            C( J, 5 ) = C( J, 5 ) - SUM*T5
            C( J, 6 ) = C( J, 6 ) - SUM*T6
            C( J, 7 ) = C( J, 7 ) - SUM*T7
            C( J, 8 ) = C( J, 8 ) - SUM*T8
            C( J, 9 ) = C( J, 9 ) - SUM*T9
            C( J, 10 ) = C( J, 10 ) - SUM*T10
  400    CONTINUE
         GO TO 410
      END IF
  410 CONTINUE
      RETURN
*
*     End of DLARFX
*
      END
```

———-

90.165.4 lisp code

— dlarfx.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
```

```
(declare (type (double-float 0.0d0 0.0d0) zero)
         (type (double-float 1.0d0 1.0d0) one)
         (ignorable zero one))
(defun dlarfx (side m n v tau c ldc work)
  (declare (type (double-float) tau)
           (type (array double-float (*)) work c v)
           (type (f2cl-lib:integer4) ldc n m)
           (type (string *) side))
  (f2cl-lib:with-multi-array-data
      ((side character side-%data% side-%offset%)
       (v double-float v-%data% v-%offset%)
       (c double-float c-%data% c-%offset%)
       (work double-float work-%data% work-%offset%))
    (prog ((sum 0.0d0) (t1 0.0d0) (t10 0.0d0) (t2 0.0d0) (t3 0.0d0)
           (t4 0.0d0) (t5 0.0d0) (t6 0.0d0) (t7 0.0d0) (t8 0.0d0) (t9 0.0d0)
           (v1 0.0d0) (v10 0.0d0) (v2 0.0d0) (v3 0.0d0) (v4 0.0d0) (v5 0.0d0)
           (v6 0.0d0) (v7 0.0d0) (v8 0.0d0) (v9 0.0d0) (j 0))
      (declare
       (type (double-float) sum t1 t10 t2 t3 t4 t5 t6 t7 t8 t9 v1 v10 v2 v3
        v4 v5 v6 v7 v8 v9)
       (type (f2cl-lib:integer4) j))
      (if (= tau zero)
          (go end_label))
      (cond
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame side "L")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val)
        (tagbody
          (f2cl-lib:computed-goto
           (label10 label30 label50 label70 label90 label110 label130
            label150 label170 label190)
           m)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (dlarf side m n v 1 tau c ldc work)
            (declare (ignore var-3 var-4 var-5 var-6 var-8))
            (setf side var-0)
            (setf m var-1)
            (setf n var-2)
            (setf ldc var-7))
          (go label410)
         label10
          (setf t1
                  (+ one
                     (* (- tau)
                        (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%)
                        (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))))
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                ((> j n) nil)
    (tagbody
      (setf (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *)) c-%offset%)
              (* t1
                 (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *))
                                c-%offset%)))
     label20))
  (go label410)
 label30
  (setf v1 (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))
  (setf t1 (* tau v1))
  (setf v2 (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%))
  (setf t2 (* tau v2))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf sum
              (+
               (* v1
                  (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *))
                                 c-%offset%))
               (* v2
                  (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *))
                                 c-%offset%))))
      (setf (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t1)))
      (setf (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t2)))
     label40))
  (go label410)
 label50
  (setf v1 (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))
  (setf t1 (* tau v1))
  (setf v2 (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%))
  (setf t2 (* tau v2))
  (setf v3 (f2cl-lib:fref v-%data% (3) ((1 *)) v-%offset%))
  (setf t3 (* tau v3))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf sum
              (+
               (* v1
                  (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *))
```

```
                                    c-%offset%))
                  (* v2
                     (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *))
                                    c-%offset%))
                  (* v3
                     (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *))
                                    c-%offset%))))
      (setf (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t1)))
      (setf (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t2)))
      (setf (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t3)))
    label60))
 (go label410)
label70
 (setf v1 (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))
 (setf t1 (* tau v1))
 (setf v2 (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%))
 (setf t2 (* tau v2))
 (setf v3 (f2cl-lib:fref v-%data% (3) ((1 *)) v-%offset%))
 (setf t3 (* tau v3))
 (setf v4 (f2cl-lib:fref v-%data% (4) ((1 *)) v-%offset%))
 (setf t4 (* tau v4))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf sum
             (+
              (* v1
                 (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *))
                                c-%offset%))
              (* v2
                 (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *))
                                c-%offset%))
              (* v3
                 (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *))
                                c-%offset%))
              (* v4
                 (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *))
                                c-%offset%))))
```

```
      (setf (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t1)))
      (setf (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t2)))
      (setf (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t3)))
      (setf (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t4)))
    label80))
  (go label410)
 label90
  (setf v1 (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))
  (setf t1 (* tau v1))
  (setf v2 (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%))
  (setf t2 (* tau v2))
  (setf v3 (f2cl-lib:fref v-%data% (3) ((1 *)) v-%offset%))
  (setf t3 (* tau v3))
  (setf v4 (f2cl-lib:fref v-%data% (4) ((1 *)) v-%offset%))
  (setf t4 (* tau v4))
  (setf v5 (f2cl-lib:fref v-%data% (5) ((1 *)) v-%offset%))
  (setf t5 (* tau v5))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf sum
              (+
               (* v1
                  (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *))
                                 c-%offset%))
               (* v2
                  (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *))
                                 c-%offset%))
               (* v3
                  (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *))
                                 c-%offset%))
               (* v4
                  (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *))
                                 c-%offset%))
```

```
                 (* v5
                    (f2cl-lib:fref c-%data% (5 j) ((1 ldc) (1 *))
                                   c-%offset%))))
      (setf (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t1)))
      (setf (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t2)))
      (setf (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t3)))
      (setf (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t4)))
      (setf (f2cl-lib:fref c-%data% (5 j) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (5 j) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t5)))
    label100))
 (go label410)
label110
 (setf v1 (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))
 (setf t1 (* tau v1))
 (setf v2 (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%))
 (setf t2 (* tau v2))
 (setf v3 (f2cl-lib:fref v-%data% (3) ((1 *)) v-%offset%))
 (setf t3 (* tau v3))
 (setf v4 (f2cl-lib:fref v-%data% (4) ((1 *)) v-%offset%))
 (setf t4 (* tau v4))
 (setf v5 (f2cl-lib:fref v-%data% (5) ((1 *)) v-%offset%))
 (setf t5 (* tau v5))
 (setf v6 (f2cl-lib:fref v-%data% (6) ((1 *)) v-%offset%))
 (setf t6 (* tau v6))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf sum
             (+
              (* v1
                 (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *))
```

```
                                    c-%offset%))
                  (* v2
                     (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *))
                                    c-%offset%))
                  (* v3
                     (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *))
                                    c-%offset%))
                  (* v4
                     (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *))
                                    c-%offset%))
                  (* v5
                     (f2cl-lib:fref c-%data% (5 j) ((1 ldc) (1 *))
                                    c-%offset%))
                  (* v6
                     (f2cl-lib:fref c-%data% (6 j) ((1 ldc) (1 *))
                                    c-%offset%))))
        (setf (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t1)))
        (setf (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t2)))
        (setf (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t3)))
        (setf (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t4)))
        (setf (f2cl-lib:fref c-%data% (5 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (5 j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t5)))
        (setf (f2cl-lib:fref c-%data% (6 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (6 j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t6)))
      label120))
  (go label410)
 label130
  (setf v1 (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))
```

```
(setf t1 (* tau v1))
(setf v2 (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%))
(setf t2 (* tau v2))
(setf v3 (f2cl-lib:fref v-%data% (3) ((1 *)) v-%offset%))
(setf t3 (* tau v3))
(setf v4 (f2cl-lib:fref v-%data% (4) ((1 *)) v-%offset%))
(setf t4 (* tau v4))
(setf v5 (f2cl-lib:fref v-%data% (5) ((1 *)) v-%offset%))
(setf t5 (* tau v5))
(setf v6 (f2cl-lib:fref v-%data% (6) ((1 *)) v-%offset%))
(setf t6 (* tau v6))
(setf v7 (f2cl-lib:fref v-%data% (7) ((1 *)) v-%offset%))
(setf t7 (* tau v7))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf sum
            (+
             (* v1
                (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *))
                               c-%offset%))
             (* v2
                (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *))
                               c-%offset%))
             (* v3
                (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *))
                               c-%offset%))
             (* v4
                (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *))
                               c-%offset%))
             (* v5
                (f2cl-lib:fref c-%data% (5 j) ((1 ldc) (1 *))
                               c-%offset%))
             (* v6
                (f2cl-lib:fref c-%data% (6 j) ((1 ldc) (1 *))
                               c-%offset%))
             (* v7
                (f2cl-lib:fref c-%data% (7 j) ((1 ldc) (1 *))
                               c-%offset%))))
    (setf (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *)) c-%offset%)
            (-
             (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *))
                            c-%offset%)
             (* sum t1)))
    (setf (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *)) c-%offset%)
            (-
             (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *))
                            c-%offset%)
             (* sum t2)))
    (setf (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *)) c-%offset%)
```

```
                (-
                 (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t3)))
        (setf (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t4)))
        (setf (f2cl-lib:fref c-%data% (5 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (5 j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t5)))
        (setf (f2cl-lib:fref c-%data% (6 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (6 j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t6)))
        (setf (f2cl-lib:fref c-%data% (7 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (7 j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t7)))
      label140))
   (go label410)
 label150
   (setf v1 (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))
   (setf t1 (* tau v1))
   (setf v2 (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%))
   (setf t2 (* tau v2))
   (setf v3 (f2cl-lib:fref v-%data% (3) ((1 *)) v-%offset%))
   (setf t3 (* tau v3))
   (setf v4 (f2cl-lib:fref v-%data% (4) ((1 *)) v-%offset%))
   (setf t4 (* tau v4))
   (setf v5 (f2cl-lib:fref v-%data% (5) ((1 *)) v-%offset%))
   (setf t5 (* tau v5))
   (setf v6 (f2cl-lib:fref v-%data% (6) ((1 *)) v-%offset%))
   (setf t6 (* tau v6))
   (setf v7 (f2cl-lib:fref v-%data% (7) ((1 *)) v-%offset%))
   (setf t7 (* tau v7))
   (setf v8 (f2cl-lib:fref v-%data% (8) ((1 *)) v-%offset%))
   (setf t8 (* tau v8))
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf sum
               (+
                (* v1
                   (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *))
```

```
                                c-%offset%))
                 (* v2
                    (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *))
                                c-%offset%))
                 (* v3
                    (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *))
                                c-%offset%))
                 (* v4
                    (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *))
                                c-%offset%))
                 (* v5
                    (f2cl-lib:fref c-%data% (5 j) ((1 ldc) (1 *))
                                c-%offset%))
                 (* v6
                    (f2cl-lib:fref c-%data% (6 j) ((1 ldc) (1 *))
                                c-%offset%))
                 (* v7
                    (f2cl-lib:fref c-%data% (7 j) ((1 ldc) (1 *))
                                c-%offset%))
                 (* v8
                    (f2cl-lib:fref c-%data% (8 j) ((1 ldc) (1 *))
                                c-%offset%))))
        (setf (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *))
                               c-%offset%)
                 (* sum t1)))
        (setf (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *))
                               c-%offset%)
                 (* sum t2)))
        (setf (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *))
                               c-%offset%)
                 (* sum t3)))
        (setf (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *))
                               c-%offset%)
                 (* sum t4)))
        (setf (f2cl-lib:fref c-%data% (5 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (5 j) ((1 ldc) (1 *))
                               c-%offset%)
                 (* sum t5)))
        (setf (f2cl-lib:fref c-%data% (6 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (6 j) ((1 ldc) (1 *))
```

```
                                c-%offset%)
                 (* sum t6)))
        (setf (f2cl-lib:fref c-%data% (7 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (7 j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t7)))
        (setf (f2cl-lib:fref c-%data% (8 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (8 j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t8)))
      label160))
   (go label410)
  label170
   (setf v1 (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))
   (setf t1 (* tau v1))
   (setf v2 (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%))
   (setf t2 (* tau v2))
   (setf v3 (f2cl-lib:fref v-%data% (3) ((1 *)) v-%offset%))
   (setf t3 (* tau v3))
   (setf v4 (f2cl-lib:fref v-%data% (4) ((1 *)) v-%offset%))
   (setf t4 (* tau v4))
   (setf v5 (f2cl-lib:fref v-%data% (5) ((1 *)) v-%offset%))
   (setf t5 (* tau v5))
   (setf v6 (f2cl-lib:fref v-%data% (6) ((1 *)) v-%offset%))
   (setf t6 (* tau v6))
   (setf v7 (f2cl-lib:fref v-%data% (7) ((1 *)) v-%offset%))
   (setf t7 (* tau v7))
   (setf v8 (f2cl-lib:fref v-%data% (8) ((1 *)) v-%offset%))
   (setf t8 (* tau v8))
   (setf v9 (f2cl-lib:fref v-%data% (9) ((1 *)) v-%offset%))
   (setf t9 (* tau v9))
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf sum
               (+
                (* v1
                   (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *))
                                  c-%offset%))
                (* v2
                   (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *))
                                  c-%offset%))
                (* v3
                   (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *))
                                  c-%offset%))
                (* v4
                   (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *))
                                  c-%offset%))
```

```
          (* v5
             (f2cl-lib:fref c-%data% (5 j) ((1 ldc) (1 *))
                            c-%offset%))
          (* v6
             (f2cl-lib:fref c-%data% (6 j) ((1 ldc) (1 *))
                            c-%offset%))
          (* v7
             (f2cl-lib:fref c-%data% (7 j) ((1 ldc) (1 *))
                            c-%offset%))
          (* v8
             (f2cl-lib:fref c-%data% (8 j) ((1 ldc) (1 *))
                            c-%offset%))
          (* v9
             (f2cl-lib:fref c-%data% (9 j) ((1 ldc) (1 *))
                            c-%offset%))))
(setf (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t1)))
(setf (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t2)))
(setf (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t3)))
(setf (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t4)))
(setf (f2cl-lib:fref c-%data% (5 j) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (5 j) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t5)))
(setf (f2cl-lib:fref c-%data% (6 j) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (6 j) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t6)))
(setf (f2cl-lib:fref c-%data% (7 j) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (7 j) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t7)))
```

```
      (setf (f2cl-lib:fref c-%data% (8 j) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (8 j) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t8)))
      (setf (f2cl-lib:fref c-%data% (9 j) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (9 j) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t9)))
    label180))
  (go label410)
 label190
  (setf v1 (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))
  (setf t1 (* tau v1))
  (setf v2 (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%))
  (setf t2 (* tau v2))
  (setf v3 (f2cl-lib:fref v-%data% (3) ((1 *)) v-%offset%))
  (setf t3 (* tau v3))
  (setf v4 (f2cl-lib:fref v-%data% (4) ((1 *)) v-%offset%))
  (setf t4 (* tau v4))
  (setf v5 (f2cl-lib:fref v-%data% (5) ((1 *)) v-%offset%))
  (setf t5 (* tau v5))
  (setf v6 (f2cl-lib:fref v-%data% (6) ((1 *)) v-%offset%))
  (setf t6 (* tau v6))
  (setf v7 (f2cl-lib:fref v-%data% (7) ((1 *)) v-%offset%))
  (setf t7 (* tau v7))
  (setf v8 (f2cl-lib:fref v-%data% (8) ((1 *)) v-%offset%))
  (setf t8 (* tau v8))
  (setf v9 (f2cl-lib:fref v-%data% (9) ((1 *)) v-%offset%))
  (setf t9 (* tau v9))
  (setf v10 (f2cl-lib:fref v-%data% (10) ((1 *)) v-%offset%))
  (setf t10 (* tau v10))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf sum
              (+
               (* v1
                  (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *))
                                 c-%offset%))
               (* v2
                  (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *))
                                 c-%offset%))
               (* v3
                  (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *))
                                 c-%offset%))
               (* v4
                  (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *))
                                 c-%offset%))
```

```
          (* v5
             (f2cl-lib:fref c-%data% (5 j) ((1 ldc) (1 *))
                            c-%offset%))
          (* v6
             (f2cl-lib:fref c-%data% (6 j) ((1 ldc) (1 *))
                            c-%offset%))
          (* v7
             (f2cl-lib:fref c-%data% (7 j) ((1 ldc) (1 *))
                            c-%offset%))
          (* v8
             (f2cl-lib:fref c-%data% (8 j) ((1 ldc) (1 *))
                            c-%offset%))
          (* v9
             (f2cl-lib:fref c-%data% (9 j) ((1 ldc) (1 *))
                            c-%offset%))
          (* v10
             (f2cl-lib:fref c-%data% (10 j) ((1 ldc) (1 *))
                            c-%offset%))))
(setf (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (1 j) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t1)))
(setf (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (2 j) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t2)))
(setf (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (3 j) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t3)))
(setf (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (4 j) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t4)))
(setf (f2cl-lib:fref c-%data% (5 j) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (5 j) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t5)))
(setf (f2cl-lib:fref c-%data% (6 j) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (6 j) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t6)))
(setf (f2cl-lib:fref c-%data% (7 j) ((1 ldc) (1 *)) c-%offset%)
        (-
```

```
                 (f2cl-lib:fref c-%data% (7 j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t7)))
          (setf (f2cl-lib:fref c-%data% (8 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (8 j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t8)))
          (setf (f2cl-lib:fref c-%data% (9 j) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (9 j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t9)))
          (setf (f2cl-lib:fref c-%data% (10 j) ((1 ldc) (1 *))
                               c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (10 j) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t10)))
        label200))
      (go label410)))
  (t
   (tagbody
     (f2cl-lib:computed-goto
      (label210 label230 label250 label270 label290 label310 label330
       label350 label370 label390)
      n)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
         (dlarf side m n v 1 tau c ldc work)
       (declare (ignore var-3 var-4 var-5 var-6 var-8))
       (setf side var-0)
       (setf m var-1)
       (setf n var-2)
       (setf ldc var-7))
     (go label410)
    label210
     (setf t1
             (+ one
                (* (- tau)
                   (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%)
                   (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))))
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j m) nil)
       (tagbody
         (setf (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *)) c-%offset%)
                 (* t1
                    (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *))
                                   c-%offset%)))
        label220))
```

```
 (go label410)
label230
 (setf v1 (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))
 (setf t1 (* tau v1))
 (setf v2 (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%))
 (setf t2 (* tau v2))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j m) nil)
   (tagbody
     (setf sum
             (+
              (* v1
                 (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *))
                                c-%offset%))
              (* v2
                 (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *))
                                c-%offset%))))
     (setf (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *)) c-%offset%)
             (-
              (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *))
                             c-%offset%)
              (* sum t1)))
     (setf (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *)) c-%offset%)
             (-
              (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *))
                             c-%offset%)
              (* sum t2)))
    label240))
 (go label410)
label250
 (setf v1 (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))
 (setf t1 (* tau v1))
 (setf v2 (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%))
 (setf t2 (* tau v2))
 (setf v3 (f2cl-lib:fref v-%data% (3) ((1 *)) v-%offset%))
 (setf t3 (* tau v3))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j m) nil)
   (tagbody
     (setf sum
             (+
              (* v1
                 (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *))
                                c-%offset%))
              (* v2
                 (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *))
                                c-%offset%))
              (* v3
                 (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *))
                                c-%offset%))))
```

```
      (setf (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t1)))
      (setf (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t2)))
      (setf (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t3)))
    label260))
 (go label410)
label270
 (setf v1 (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))
 (setf t1 (* tau v1))
 (setf v2 (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%))
 (setf t2 (* tau v2))
 (setf v3 (f2cl-lib:fref v-%data% (3) ((1 *)) v-%offset%))
 (setf t3 (* tau v3))
 (setf v4 (f2cl-lib:fref v-%data% (4) ((1 *)) v-%offset%))
 (setf t4 (* tau v4))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j m) nil)
   (tagbody
     (setf sum
             (+
              (* v1
                 (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *))
                                c-%offset%))
              (* v2
                 (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *))
                                c-%offset%))
              (* v3
                 (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *))
                                c-%offset%))
              (* v4
                 (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *))
                                c-%offset%))))
     (setf (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *)) c-%offset%)
             (-
              (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *))
                              c-%offset%)
              (* sum t1)))
     (setf (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *)) c-%offset%)
             (-
```

```
                 (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t2)))
        (setf (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t3)))
        (setf (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t4)))
      label280))
    (go label410)
   label290
    (setf v1 (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))
    (setf t1 (* tau v1))
    (setf v2 (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%))
    (setf t2 (* tau v2))
    (setf v3 (f2cl-lib:fref v-%data% (3) ((1 *)) v-%offset%))
    (setf t3 (* tau v3))
    (setf v4 (f2cl-lib:fref v-%data% (4) ((1 *)) v-%offset%))
    (setf t4 (* tau v4))
    (setf v5 (f2cl-lib:fref v-%data% (5) ((1 *)) v-%offset%))
    (setf t5 (* tau v5))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j m) nil)
      (tagbody
        (setf sum
                (+
                 (* v1
                    (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *))
                                   c-%offset%))
                 (* v2
                    (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *))
                                   c-%offset%))
                 (* v3
                    (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *))
                                   c-%offset%))
                 (* v4
                    (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *))
                                   c-%offset%))
                 (* v5
                    (f2cl-lib:fref c-%data% (j 5) ((1 ldc) (1 *))
                                   c-%offset%))))
        (setf (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *))
                                c-%offset%)
```

```
                 (* sum t1)))
        (setf (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t2)))
        (setf (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t3)))
        (setf (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t4)))
        (setf (f2cl-lib:fref c-%data% (j 5) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (j 5) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t5)))
      label300))
    (go label410)
  label310
    (setf v1 (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))
    (setf t1 (* tau v1))
    (setf v2 (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%))
    (setf t2 (* tau v2))
    (setf v3 (f2cl-lib:fref v-%data% (3) ((1 *)) v-%offset%))
    (setf t3 (* tau v3))
    (setf v4 (f2cl-lib:fref v-%data% (4) ((1 *)) v-%offset%))
    (setf t4 (* tau v4))
    (setf v5 (f2cl-lib:fref v-%data% (5) ((1 *)) v-%offset%))
    (setf t5 (* tau v5))
    (setf v6 (f2cl-lib:fref v-%data% (6) ((1 *)) v-%offset%))
    (setf t6 (* tau v6))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j m) nil)
      (tagbody
        (setf sum
                (+
                 (* v1
                    (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *))
                                   c-%offset%))
                 (* v2
                    (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *))
                                   c-%offset%))
                 (* v3
                    (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *))
                                   c-%offset%))
```

```
                 (* v4
                    (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *))
                                   c-%offset%))
                 (* v5
                    (f2cl-lib:fref c-%data% (j 5) ((1 ldc) (1 *))
                                   c-%offset%))
                 (* v6
                    (f2cl-lib:fref c-%data% (j 6) ((1 ldc) (1 *))
                                   c-%offset%))))
     (setf (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *)) c-%offset%)
             (-
              (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *))
                             c-%offset%)
              (* sum t1)))
     (setf (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *)) c-%offset%)
             (-
              (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *))
                             c-%offset%)
              (* sum t2)))
     (setf (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *)) c-%offset%)
             (-
              (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *))
                             c-%offset%)
              (* sum t3)))
     (setf (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *)) c-%offset%)
             (-
              (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *))
                             c-%offset%)
              (* sum t4)))
     (setf (f2cl-lib:fref c-%data% (j 5) ((1 ldc) (1 *)) c-%offset%)
             (-
              (f2cl-lib:fref c-%data% (j 5) ((1 ldc) (1 *))
                             c-%offset%)
              (* sum t5)))
     (setf (f2cl-lib:fref c-%data% (j 6) ((1 ldc) (1 *)) c-%offset%)
             (-
              (f2cl-lib:fref c-%data% (j 6) ((1 ldc) (1 *))
                             c-%offset%)
              (* sum t6)))
    label320))
 (go label410)
label330
 (setf v1 (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))
 (setf t1 (* tau v1))
 (setf v2 (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%))
 (setf t2 (* tau v2))
 (setf v3 (f2cl-lib:fref v-%data% (3) ((1 *)) v-%offset%))
 (setf t3 (* tau v3))
 (setf v4 (f2cl-lib:fref v-%data% (4) ((1 *)) v-%offset%))
 (setf t4 (* tau v4))
```

```
(setf v5 (f2cl-lib:fref v-%data% (5) ((1 *)) v-%offset%))
(setf t5 (* tau v5))
(setf v6 (f2cl-lib:fref v-%data% (6) ((1 *)) v-%offset%))
(setf t6 (* tau v6))
(setf v7 (f2cl-lib:fref v-%data% (7) ((1 *)) v-%offset%))
(setf t7 (* tau v7))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j m) nil)
  (tagbody
    (setf sum
            (+
             (* v1
                (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *))
                               c-%offset%))
             (* v2
                (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *))
                               c-%offset%))
             (* v3
                (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *))
                               c-%offset%))
             (* v4
                (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *))
                               c-%offset%))
             (* v5
                (f2cl-lib:fref c-%data% (j 5) ((1 ldc) (1 *))
                               c-%offset%))
             (* v6
                (f2cl-lib:fref c-%data% (j 6) ((1 ldc) (1 *))
                               c-%offset%))
             (* v7
                (f2cl-lib:fref c-%data% (j 7) ((1 ldc) (1 *))
                               c-%offset%))))
    (setf (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *)) c-%offset%)
            (-
             (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *))
                            c-%offset%)
             (* sum t1)))
    (setf (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *)) c-%offset%)
            (-
             (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *))
                            c-%offset%)
             (* sum t2)))
    (setf (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *)) c-%offset%)
            (-
             (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *))
                            c-%offset%)
             (* sum t3)))
    (setf (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *)) c-%offset%)
            (-
             (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *))
```

```
                                c-%offset%)
                 (* sum t4)))
        (setf (f2cl-lib:fref c-%data% (j 5) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (j 5) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t5)))
        (setf (f2cl-lib:fref c-%data% (j 6) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (j 6) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t6)))
        (setf (f2cl-lib:fref c-%data% (j 7) ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (j 7) ((1 ldc) (1 *))
                                c-%offset%)
                 (* sum t7)))
      label340))
    (go label410)
   label350
    (setf v1 (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))
    (setf t1 (* tau v1))
    (setf v2 (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%))
    (setf t2 (* tau v2))
    (setf v3 (f2cl-lib:fref v-%data% (3) ((1 *)) v-%offset%))
    (setf t3 (* tau v3))
    (setf v4 (f2cl-lib:fref v-%data% (4) ((1 *)) v-%offset%))
    (setf t4 (* tau v4))
    (setf v5 (f2cl-lib:fref v-%data% (5) ((1 *)) v-%offset%))
    (setf t5 (* tau v5))
    (setf v6 (f2cl-lib:fref v-%data% (6) ((1 *)) v-%offset%))
    (setf t6 (* tau v6))
    (setf v7 (f2cl-lib:fref v-%data% (7) ((1 *)) v-%offset%))
    (setf t7 (* tau v7))
    (setf v8 (f2cl-lib:fref v-%data% (8) ((1 *)) v-%offset%))
    (setf t8 (* tau v8))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j m) nil)
      (tagbody
        (setf sum
                (+
                 (* v1
                    (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *))
                                   c-%offset%))
                 (* v2
                    (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *))
                                   c-%offset%))
                 (* v3
                    (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *))
                                   c-%offset%))
```

```
          (* v4
             (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *))
                            c-%offset%))
          (* v5
             (f2cl-lib:fref c-%data% (j 5) ((1 ldc) (1 *))
                            c-%offset%))
          (* v6
             (f2cl-lib:fref c-%data% (j 6) ((1 ldc) (1 *))
                            c-%offset%))
          (* v7
             (f2cl-lib:fref c-%data% (j 7) ((1 ldc) (1 *))
                            c-%offset%))
          (* v8
             (f2cl-lib:fref c-%data% (j 8) ((1 ldc) (1 *))
                            c-%offset%))))
(setf (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t1)))
(setf (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t2)))
(setf (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t3)))
(setf (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t4)))
(setf (f2cl-lib:fref c-%data% (j 5) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 5) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t5)))
(setf (f2cl-lib:fref c-%data% (j 6) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 6) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t6)))
(setf (f2cl-lib:fref c-%data% (j 7) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 7) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t7)))
```

```
      (setf (f2cl-lib:fref c-%data% (j 8) ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data% (j 8) ((1 ldc) (1 *))
                              c-%offset%)
               (* sum t8)))
    label360))
 (go label410)
label370
 (setf v1 (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))
 (setf t1 (* tau v1))
 (setf v2 (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%))
 (setf t2 (* tau v2))
 (setf v3 (f2cl-lib:fref v-%data% (3) ((1 *)) v-%offset%))
 (setf t3 (* tau v3))
 (setf v4 (f2cl-lib:fref v-%data% (4) ((1 *)) v-%offset%))
 (setf t4 (* tau v4))
 (setf v5 (f2cl-lib:fref v-%data% (5) ((1 *)) v-%offset%))
 (setf t5 (* tau v5))
 (setf v6 (f2cl-lib:fref v-%data% (6) ((1 *)) v-%offset%))
 (setf t6 (* tau v6))
 (setf v7 (f2cl-lib:fref v-%data% (7) ((1 *)) v-%offset%))
 (setf t7 (* tau v7))
 (setf v8 (f2cl-lib:fref v-%data% (8) ((1 *)) v-%offset%))
 (setf t8 (* tau v8))
 (setf v9 (f2cl-lib:fref v-%data% (9) ((1 *)) v-%offset%))
 (setf t9 (* tau v9))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j m) nil)
   (tagbody
     (setf sum
             (+
              (* v1
                 (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *))
                                c-%offset%))
              (* v2
                 (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *))
                                c-%offset%))
              (* v3
                 (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *))
                                c-%offset%))
              (* v4
                 (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *))
                                c-%offset%))
              (* v5
                 (f2cl-lib:fref c-%data% (j 5) ((1 ldc) (1 *))
                                c-%offset%))
              (* v6
                 (f2cl-lib:fref c-%data% (j 6) ((1 ldc) (1 *))
                                c-%offset%))
              (* v7
```

```
            (f2cl-lib:fref c-%data% (j 7) ((1 ldc) (1 *))
                           c-%offset%))
         (* v8
            (f2cl-lib:fref c-%data% (j 8) ((1 ldc) (1 *))
                           c-%offset%))
         (* v9
            (f2cl-lib:fref c-%data% (j 9) ((1 ldc) (1 *))
                           c-%offset%))))
(setf (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t1)))
(setf (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t2)))
(setf (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t3)))
(setf (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t4)))
(setf (f2cl-lib:fref c-%data% (j 5) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 5) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t5)))
(setf (f2cl-lib:fref c-%data% (j 6) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 6) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t6)))
(setf (f2cl-lib:fref c-%data% (j 7) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 7) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t7)))
(setf (f2cl-lib:fref c-%data% (j 8) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 8) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t8)))
(setf (f2cl-lib:fref c-%data% (j 9) ((1 ldc) (1 *)) c-%offset%)
        (-
```

```
                (f2cl-lib:fref c-%data% (j 9) ((1 ldc) (1 *))
                               c-%offset%)
                (* sum t9)))
    label380))
  (go label410)
 label390
  (setf v1 (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%))
  (setf t1 (* tau v1))
  (setf v2 (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%))
  (setf t2 (* tau v2))
  (setf v3 (f2cl-lib:fref v-%data% (3) ((1 *)) v-%offset%))
  (setf t3 (* tau v3))
  (setf v4 (f2cl-lib:fref v-%data% (4) ((1 *)) v-%offset%))
  (setf t4 (* tau v4))
  (setf v5 (f2cl-lib:fref v-%data% (5) ((1 *)) v-%offset%))
  (setf t5 (* tau v5))
  (setf v6 (f2cl-lib:fref v-%data% (6) ((1 *)) v-%offset%))
  (setf t6 (* tau v6))
  (setf v7 (f2cl-lib:fref v-%data% (7) ((1 *)) v-%offset%))
  (setf t7 (* tau v7))
  (setf v8 (f2cl-lib:fref v-%data% (8) ((1 *)) v-%offset%))
  (setf t8 (* tau v8))
  (setf v9 (f2cl-lib:fref v-%data% (9) ((1 *)) v-%offset%))
  (setf t9 (* tau v9))
  (setf v10 (f2cl-lib:fref v-%data% (10) ((1 *)) v-%offset%))
  (setf t10 (* tau v10))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j m) nil)
    (tagbody
      (setf sum
              (+
               (* v1
                  (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *))
                                 c-%offset%))
               (* v2
                  (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *))
                                 c-%offset%))
               (* v3
                  (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *))
                                 c-%offset%))
               (* v4
                  (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *))
                                 c-%offset%))
               (* v5
                  (f2cl-lib:fref c-%data% (j 5) ((1 ldc) (1 *))
                                 c-%offset%))
               (* v6
                  (f2cl-lib:fref c-%data% (j 6) ((1 ldc) (1 *))
                                 c-%offset%))
               (* v7
```

```
            (f2cl-lib:fref c-%data% (j 7) ((1 ldc) (1 *))
                           c-%offset%))
         (* v8
            (f2cl-lib:fref c-%data% (j 8) ((1 ldc) (1 *))
                           c-%offset%))
         (* v9
            (f2cl-lib:fref c-%data% (j 9) ((1 ldc) (1 *))
                           c-%offset%))
         (* v10
            (f2cl-lib:fref c-%data% (j 10) ((1 ldc) (1 *))
                           c-%offset%))))
(setf (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 1) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t1)))
(setf (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 2) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t2)))
(setf (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 3) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t3)))
(setf (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 4) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t4)))
(setf (f2cl-lib:fref c-%data% (j 5) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 5) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t5)))
(setf (f2cl-lib:fref c-%data% (j 6) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 6) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t6)))
(setf (f2cl-lib:fref c-%data% (j 7) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 7) ((1 ldc) (1 *))
                        c-%offset%)
         (* sum t7)))
(setf (f2cl-lib:fref c-%data% (j 8) ((1 ldc) (1 *)) c-%offset%)
        (-
         (f2cl-lib:fref c-%data% (j 8) ((1 ldc) (1 *))
                        c-%offset%)
```

```
                        (* sum t8)))
                (setf (f2cl-lib:fref c-%data% (j 9) ((1 ldc) (1 *)) c-%offset%)
                        (-
                         (f2cl-lib:fref c-%data% (j 9) ((1 ldc) (1 *))
                                        c-%offset%)
                         (* sum t9)))
                (setf (f2cl-lib:fref c-%data% (j 10) ((1 ldc) (1 *))
                                      c-%offset%)
                        (-
                         (f2cl-lib:fref c-%data% (j 10) ((1 ldc) (1 *))
                                        c-%offset%)
                         (* sum t10)))
              label400))
            (go label410))))
      label410
       (go end_label)
      end_label
       (return (values side m n nil nil nil ldc nil))))))
```

———-

90.166 dlartg

90.166.1 Axiom unit tests

— **dlartg.input** —

```
)set break resume
)sys rm -f dlartg.output
)spool dlartg.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.166.2 Axiom help page

— **dlartg.help** —

```
==================================================================
```

```
dlartg examples
===================================================================

===================================================================
Man Page Details
===================================================================

NAME
       DLARTG - a plane rotation so that   [ CS SN ]

SYNOPSIS
       SUBROUTINE DLARTG( F, G, CS, SN, R )

           DOUBLE         PRECISION CS, F, G, R, SN

 Purpose
 =======

  DLARTG generate a plane rotation so that

     [  CS  SN  ]  .  [ F ]  =  [ R ]   where CS**2 + SN**2 = 1.
     [ -SN  CS  ]     [ G ]     [ 0 ]

  This is a slower, more accurate version of the BLAS1 routine DROTG,
  with the following other differences:
     F and G are unchanged on return.
     If G=0, then CS=1 and SN=0.
     If F=0 and (G .ne. 0), then CS=0 and SN=1 without doing any
        floating point operations (saves work in DBDSQR when
        there are zeros on the diagonal).

  If F exceeds G in magnitude, CS will be positive.

 Arguments
 =========

  F       (input) DOUBLE PRECISION
          The first component of vector to be rotated.

  G       (input) DOUBLE PRECISION
          The second component of vector to be rotated.

  CS      (output) DOUBLE PRECISION
          The cosine of the rotation.

  SN      (output) DOUBLE PRECISION
          The sine of the rotation.

  R       (output) DOUBLE PRECISION
          The nonzero component of the rotated vector.
```

———-

90.166.3 fortran code

— **dlartg.f** —

```
      SUBROUTINE DLARTG( F, G, CS, SN, R )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     September 30, 1994
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   CS, F, G, R, SN
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D0 )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            FIRST
      INTEGER            COUNT, I
      DOUBLE PRECISION   EPS, F1, G1, SAFMIN, SAFMN2, SAFMX2, SCALE
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, INT, LOG, MAX, SQRT
*     ..
*     .. Save statement ..
      SAVE               FIRST, SAFMX2, SAFMIN, SAFMN2
*     ..
*     .. Data statements ..
      DATA               FIRST / .TRUE. /
*     ..
*     .. Executable Statements ..
```

```
*
      IF( FIRST ) THEN
         FIRST = .FALSE.
         SAFMIN = DLAMCH( 'S' )
         EPS = DLAMCH( 'E' )
         SAFMN2 = DLAMCH( 'B' )**INT( LOG( SAFMIN / EPS ) /
     $           LOG( DLAMCH( 'B' ) ) / TWO )
         SAFMX2 = ONE / SAFMN2
      END IF
      IF( G.EQ.ZERO ) THEN
         CS = ONE
         SN = ZERO
         R = F
      ELSE IF( F.EQ.ZERO ) THEN
         CS = ZERO
         SN = ONE
         R = G
      ELSE
         F1 = F
         G1 = G
         SCALE = MAX( ABS( F1 ), ABS( G1 ) )
         IF( SCALE.GE.SAFMX2 ) THEN
            COUNT = 0
   10       CONTINUE
            COUNT = COUNT + 1
            F1 = F1*SAFMN2
            G1 = G1*SAFMN2
            SCALE = MAX( ABS( F1 ), ABS( G1 ) )
            IF( SCALE.GE.SAFMX2 )
     $         GO TO 10
            R = SQRT( F1**2+G1**2 )
            CS = F1 / R
            SN = G1 / R
            DO 20 I = 1, COUNT
               R = R*SAFMX2
   20       CONTINUE
         ELSE IF( SCALE.LE.SAFMN2 ) THEN
            COUNT = 0
   30       CONTINUE
            COUNT = COUNT + 1
            F1 = F1*SAFMX2
            G1 = G1*SAFMX2
            SCALE = MAX( ABS( F1 ), ABS( G1 ) )
            IF( SCALE.LE.SAFMN2 )
     $         GO TO 30
            R = SQRT( F1**2+G1**2 )
            CS = F1 / R
            SN = G1 / R
            DO 40 I = 1, COUNT
               R = R*SAFMN2
```

```
   40       CONTINUE
         ELSE
            R = SQRT( F1**2+G1**2 )
            CS = F1 / R
            SN = G1 / R
         END IF
         IF( ABS( F ).GT.ABS( G ) .AND. CS.LT.ZERO ) THEN
            CS = -CS
            SN = -SN
            R = -R
         END IF
      END IF
      RETURN
*
*     End of DLARTG
*
      END
```

———-

90.166.4 lisp code

— dlartg.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0) (two 2.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (ignorable zero one two))
  (defun dlartg (f g cs sn r)
    (declare (type (double-float) r sn cs g f))
    (prog ((eps 0.0d0) (f1 0.0d0) (g1 0.0d0) (safmin 0.0d0) (safmn2 0.0d0)
           (safmx2 0.0d0) (scale 0.0d0) (i 0) (count$ 0))
      (declare (type (double-float) eps f1 g1 safmin safmn2 safmx2 scale)
               (type (f2cl-lib:integer4) count$ i))
      (setf safmin (dlamch "S"))
      (setf eps (dlamch "E"))
      (setf safmn2
              (expt (dlamch "B")
                    (f2cl-lib:int
                     (/
                      (/ (f2cl-lib:flog (/ safmin eps))
                         (f2cl-lib:flog (dlamch "B")))
                      two))))
      (setf safmx2 (/ one safmn2))
      (cond ((= g zero) (setf cs one) (setf sn zero) (setf r f))
            ((= f zero) (setf cs zero) (setf sn one) (setf r g))
```

```
        (t (setf f1 f) (setf g1 g) (setf scale (max (abs f1) (abs g1)))
         (cond
          ((>= scale safmx2)
           (tagbody
             (setf count$ 0)
            label10
             (setf count$ (f2cl-lib:int-add count$ 1))
             (setf f1 (* f1 safmn2))
             (setf g1 (* g1 safmn2))
             (setf scale (max (abs f1) (abs g1)))
             (if (>= scale safmx2)
                 (go label10))
             (setf r (f2cl-lib:fsqrt (+ (expt f1 2) (expt g1 2))))
             (setf cs (/ f1 r))
             (setf sn (/ g1 r))
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i count$) nil)
               (tagbody (setf r (* r safmx2)) label20))))
          ((<= scale safmn2)
           (tagbody
             (setf count$ 0)
            label30
             (setf count$ (f2cl-lib:int-add count$ 1))
             (setf f1 (* f1 safmx2))
             (setf g1 (* g1 safmx2))
             (setf scale (max (abs f1) (abs g1)))
             (if (<= scale safmn2)
                 (go label30))
             (setf r (f2cl-lib:fsqrt (+ (expt f1 2) (expt g1 2))))
             (setf cs (/ f1 r))
             (setf sn (/ g1 r))
             (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                           ((> i count$) nil)
               (tagbody (setf r (* r safmn2)) label40))))
          (t (setf r (f2cl-lib:fsqrt (+ (expt f1 2) (expt g1 2))))
           (setf cs (/ f1 r)) (setf sn (/ g1 r))))
         (cond
          ((and (> (abs f) (abs g)) (< cs zero)) (setf cs (- cs))
           (setf sn (- sn)) (setf r (- r))))))
 (go end_label)
end_label
 (return (values nil nil cs sn r)))))
```

———-

90.167 dlas2

90.167.1 Axiom unit tests

— dlas2.input —

```
)set break resume
)sys rm -f dlas2.output
)spool dlas2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.167.2 Axiom help page

— dlas2.help —

```
======================================================================
dlas2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DLAS2 - the singular values of the 2-by-2 matrix  [ F G ]  [ 0 H ]

SYNOPSIS
       SUBROUTINE DLAS2( F, G, H, SSMIN, SSMAX )

           DOUBLE        PRECISION F, G, H, SSMAX, SSMIN

PURPOSE
       DLAS2  computes the singular values of the 2-by-2 matrix
          [  F   G  ]
          [   0    H   ].   On return, SSMIN is the smaller singular value and
       SSMAX is the larger singular value.


ARGUMENTS
```

```
       F       (input) DOUBLE PRECISION
               The (1,1) element of the 2-by-2 matrix.

       G       (input) DOUBLE PRECISION
               The (1,2) element of the 2-by-2 matrix.

       H       (input) DOUBLE PRECISION
               The (2,2) element of the 2-by-2 matrix.

       SSMIN   (output) DOUBLE PRECISION
               The smaller singular value.

       SSMAX   (output) DOUBLE PRECISION
               The larger singular value.

FURTHER DETAILS
       Barring over/underflow, all output quantities are correct to  within  a
       few  units  in  the  last  place (ulps), even in the absence of a guard
       digit in addition/subtraction.

       In IEEE arithmetic, the code works correctly if one matrix  element  is
       infinite.

       Overflow  will not occur unless the largest singular value itself over-
       flows, or is within a few ulps of overflow. (On machines  with  partial
       overflow,  like  the  Cray,  overflow may occur if the largest singular
       value is within a factor of 2 of overflow.)

       Underflow is harmless if underflow is gradual. Otherwise,  results  may
       correspond  to  a  matrix  modified  by  perturbations of size near the
       underflow threshold.
```

———-

90.167.3 fortran code

— **dlas2.f** —

```
      SUBROUTINE DLAS2( F, G, H, SSMIN, SSMAX )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     September 30, 1994
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   F, G, H, SSMAX, SSMIN
```

```
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D0 )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D0 )
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION   AS, AT, AU, C, FA, FHMN, FHMX, GA, HA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      FA = ABS( F )
      GA = ABS( G )
      HA = ABS( H )
      FHMN = MIN( FA, HA )
      FHMX = MAX( FA, HA )
      IF( FHMN.EQ.ZERO ) THEN
         SSMIN = ZERO
         IF( FHMX.EQ.ZERO ) THEN
            SSMAX = GA
         ELSE
            SSMAX = MAX( FHMX, GA )*SQRT( ONE+
     $              ( MIN( FHMX, GA ) / MAX( FHMX, GA ) )**2 )
         END IF
      ELSE
         IF( GA.LT.FHMX ) THEN
            AS = ONE + FHMN / FHMX
            AT = ( FHMX-FHMN ) / FHMX
            AU = ( GA / FHMX )**2
            C = TWO / ( SQRT( AS*AS+AU )+SQRT( AT*AT+AU ) )
            SSMIN = FHMN*C
            SSMAX = FHMX / C
         ELSE
            AU = FHMX / GA
            IF( AU.EQ.ZERO ) THEN
*
*              Avoid possible harmful underflow if exponent range
*              asymmetric (true SSMIN may not underflow even if
*              AU underflows)
*
               SSMIN = ( FHMN*FHMX ) / GA
```

```
               SSMAX = GA
            ELSE
               AS = ONE + FHMN / FHMX
               AT = ( FHMX-FHMN ) / FHMX
               C = ONE / ( SQRT( ONE+( AS*AU )**2 )+
     $             SQRT( ONE+( AT*AU )**2 ) )
               SSMIN = ( FHMN*C )*AU
               SSMIN = SSMIN + SSMIN
               SSMAX = GA / ( C+C )
            END IF
         END IF
      END IF
      RETURN
*
*     End of DLAS2
*
      END
```

———-

90.167.4 lisp code

— dlas2.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0) (two 2.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (ignorable zero one two))
  (defun dlas2 (f g h ssmin ssmax)
    (declare (type (double-float) ssmax ssmin h g f))
    (prog ((as 0.0d0) (at 0.0d0) (au 0.0d0) (c 0.0d0) (fa 0.0d0) (fhmn 0.0d0)
           (fhmx 0.0d0) (ga 0.0d0) (ha 0.0d0))
      (declare (type (double-float) as at au c fa fhmn fhmx ga ha))
      (setf fa (abs f))
      (setf ga (abs g))
      (setf ha (abs h))
      (setf fhmn (min fa ha))
      (setf fhmx (max fa ha))
      (cond
       ((= fhmn zero) (setf ssmin zero)
        (cond ((= fhmx zero) (setf ssmax ga))
              (t
               (setf ssmax
                       (* (max fhmx ga)
                          (f2cl-lib:fsqrt
                           (+ one
```

```
                          (expt (/ (min fhmx ga) (max fhmx ga)) 2)))))))
      (t
       (cond
        ((< ga fhmx) (setf as (+ one (/ fhmn fhmx)))
         (setf at (/ (- fhmx fhmn) fhmx)) (setf au (expt (/ ga fhmx) 2))
         (setf c
                 (/ two
                    (+ (f2cl-lib:fsqrt (+ (* as as) au))
                       (f2cl-lib:fsqrt (+ (* at at) au)))))
         (setf ssmin (* fhmn c)) (setf ssmax (/ fhmx c)))
        (t (setf au (/ fhmx ga))
         (cond ((= au zero) (setf ssmin (/ (* fhmn fhmx) ga)) (setf ssmax ga))
               (t (setf as (+ one (/ fhmn fhmx)))
                (setf at (/ (- fhmx fhmn) fhmx))
                (setf c
                        (/ one
                           (+ (f2cl-lib:fsqrt (+ one (expt (* as au) 2)))
                              (f2cl-lib:fsqrt (+ one (expt (* at au) 2))))))
                (setf ssmin (* fhmn c au)) (setf ssmin (+ ssmin ssmin))
                (setf ssmax (/ ga (+ c c)))))))))
     (go end_label)
    end_label
     (return (values nil nil nil ssmin ssmax)))))
```

———-

90.168 dlascl

90.168.1 Axiom unit tests

— **dlascl.input** —

```
)set break resume
)sys rm -f dlascl.output
)spool dlascl.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.168.2 Axiom help page

— dlascl.help —

```
====================================================================
dlascl examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DLASCL - the M by N real matrix A by the real scalar CTO/CFROM

SYNOPSIS
       SUBROUTINE DLASCL( TYPE, KL, KU, CFROM, CTO, M, N, A, LDA, INFO )

           CHARACTER      TYPE

           INTEGER        INFO, KL, KU, LDA, M, N

           DOUBLE         PRECISION CFROM, CTO

           DOUBLE         PRECISION A( LDA, * )

 Purpose
 =======

  DLASCL multiplies the M by N real matrix A by the real scalar
  CTO/CFROM.  This is done without over/underflow as long as the final
  result CTO*A(I,J)/CFROM does not over/underflow. TYPE specifies that
  A may be full, upper triangular, lower triangular, upper Hessenberg,
  or banded.

 Arguments
 =========

  TYPE    (input) CHARACTER*1
          TYPE indices the storage type of the input matrix.
          = 'G':  A is a full matrix.
          = 'L':  A is a lower triangular matrix.
          = 'U':  A is an upper triangular matrix.
          = 'H':  A is an upper Hessenberg matrix.
          = 'B':  A is a symmetric band matrix with lower bandwidth KL
                  and upper bandwidth KU and with the only the lower
                  half stored.
          = 'Q':  A is a symmetric band matrix with lower bandwidth KL
                  and upper bandwidth KU and with the only the upper
```

```
                half stored.
        = 'Z':  A is a band matrix with lower bandwidth KL and upper
                bandwidth KU.

 KL     (input) INTEGER
        The lower bandwidth of A.  Referenced only if TYPE = 'B',
        'Q' or 'Z'.

 KU     (input) INTEGER
        The upper bandwidth of A.  Referenced only if TYPE = 'B',
        'Q' or 'Z'.

 CFROM  (input) DOUBLE PRECISION
 CTO    (input) DOUBLE PRECISION
        The matrix A is multiplied by CTO/CFROM. A(I,J) is computed
        without over/underflow if the final result CTO*A(I,J)/CFROM
        can be represented without over/underflow.  CFROM must be
        nonzero.

 M      (input) INTEGER
        The number of rows of the matrix A.  M >= 0.

 N      (input) INTEGER
        The number of columns of the matrix A.  N >= 0.

 A      (input/output) DOUBLE PRECISION array, dimension (LDA,M)
        The matrix to be multiplied by CTO/CFROM.  See TYPE for the
        storage type.

 LDA    (input) INTEGER
        The leading dimension of the array A.  LDA >= max(1,M).

 INFO   (output) INTEGER
        0  - successful exit
        <0 - if INFO = -i, the i-th argument had an illegal value.
```

———-

90.168.3 fortran code

— **dlascl.f** —

```
      SUBROUTINE DLASCL( TYPE, KL, KU, CFROM, CTO, M, N, A, LDA, INFO )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
```

```
*     February 29, 1992
*
*     .. Scalar Arguments ..
      CHARACTER          TYPE
      INTEGER            INFO, KL, KU, LDA, M, N
      DOUBLE PRECISION   CFROM, CTO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            DONE
      INTEGER            I, ITYPE, J, K1, K2, K3, K4
      DOUBLE PRECISION   BIGNUM, CFROM1, CFROMC, CTO1, CTOC, MUL, SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
*
      IF( LSAME( TYPE, 'G' ) ) THEN
         ITYPE = 0
      ELSE IF( LSAME( TYPE, 'L' ) ) THEN
         ITYPE = 1
      ELSE IF( LSAME( TYPE, 'U' ) ) THEN
         ITYPE = 2
      ELSE IF( LSAME( TYPE, 'H' ) ) THEN
         ITYPE = 3
      ELSE IF( LSAME( TYPE, 'B' ) ) THEN
         ITYPE = 4
      ELSE IF( LSAME( TYPE, 'Q' ) ) THEN
```

```
         ITYPE = 5
      ELSE IF( LSAME( TYPE, 'Z' ) ) THEN
         ITYPE = 6
      ELSE
         ITYPE = -1
      END IF
*
      IF( ITYPE.EQ.-1 ) THEN
         INFO = -1
      ELSE IF( CFROM.EQ.ZERO ) THEN
         INFO = -4
      ELSE IF( M.LT.0 ) THEN
         INFO = -6
      ELSE IF( N.LT.0 .OR. ( ITYPE.EQ.4 .AND. N.NE.M ) .OR.
     $         ( ITYPE.EQ.5 .AND. N.NE.M ) ) THEN
         INFO = -7
      ELSE IF( ITYPE.LE.3 .AND. LDA.LT.MAX( 1, M ) ) THEN
         INFO = -9
      ELSE IF( ITYPE.GE.4 ) THEN
         IF( KL.LT.0 .OR. KL.GT.MAX( M-1, 0 ) ) THEN
            INFO = -2
         ELSE IF( KU.LT.0 .OR. KU.GT.MAX( N-1, 0 ) .OR.
     $            ( ( ITYPE.EQ.4 .OR. ITYPE.EQ.5 ) .AND. KL.NE.KU ) )
     $             THEN
            INFO = -3
         ELSE IF( ( ITYPE.EQ.4 .AND. LDA.LT.KL+1 ) .OR.
     $            ( ITYPE.EQ.5 .AND. LDA.LT.KU+1 ) .OR.
     $            ( ITYPE.EQ.6 .AND. LDA.LT.2*KL+KU+1 ) ) THEN
            INFO = -9
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASCL', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. M.EQ.0 )
     $   RETURN
*
*     Get machine parameters
*
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
*
      CFROMC = CFROM
      CTOC = CTO
*
```

```
   10 CONTINUE
      CFROM1 = CFROMC*SMLNUM
      CTO1 = CTOC / BIGNUM
      IF( ABS( CFROM1 ).GT.ABS( CTOC ) .AND. CTOC.NE.ZERO ) THEN
         MUL = SMLNUM
         DONE = .FALSE.
         CFROMC = CFROM1
      ELSE IF( ABS( CTO1 ).GT.ABS( CFROMC ) ) THEN
         MUL = BIGNUM
         DONE = .FALSE.
         CTOC = CTO1
      ELSE
         MUL = CTOC / CFROMC
         DONE = .TRUE.
      END IF
*
      IF( ITYPE.EQ.0 ) THEN
*
*        Full matrix
*
         DO 30 J = 1, N
            DO 20 I = 1, M
               A( I, J ) = A( I, J )*MUL
   20       CONTINUE
   30    CONTINUE
*
      ELSE IF( ITYPE.EQ.1 ) THEN
*
*        Lower triangular matrix
*
         DO 50 J = 1, N
            DO 40 I = J, M
               A( I, J ) = A( I, J )*MUL
   40       CONTINUE
   50    CONTINUE
*
      ELSE IF( ITYPE.EQ.2 ) THEN
*
*        Upper triangular matrix
*
         DO 70 J = 1, N
            DO 60 I = 1, MIN( J, M )
               A( I, J ) = A( I, J )*MUL
   60       CONTINUE
   70    CONTINUE
*
      ELSE IF( ITYPE.EQ.3 ) THEN
*
*        Upper Hessenberg matrix
*
```

```
         DO 90 J = 1, N
            DO 80 I = 1, MIN( J+1, M )
               A( I, J ) = A( I, J )*MUL
   80       CONTINUE
   90    CONTINUE
*
      ELSE IF( ITYPE.EQ.4 ) THEN
*
*        Lower half of a symmetric band matrix
*
         K3 = KL + 1
         K4 = N + 1
         DO 110 J = 1, N
            DO 100 I = 1, MIN( K3, K4-J )
               A( I, J ) = A( I, J )*MUL
  100       CONTINUE
  110    CONTINUE
*
      ELSE IF( ITYPE.EQ.5 ) THEN
*
*        Upper half of a symmetric band matrix
*
         K1 = KU + 2
         K3 = KU + 1
         DO 130 J = 1, N
            DO 120 I = MAX( K1-J, 1 ), K3
               A( I, J ) = A( I, J )*MUL
  120       CONTINUE
  130    CONTINUE
*
      ELSE IF( ITYPE.EQ.6 ) THEN
*
*        Band matrix
*
         K1 = KL + KU + 2
         K2 = KL + 1
         K3 = 2*KL + KU + 1
         K4 = KL + KU + 1 + M
         DO 150 J = 1, N
            DO 140 I = MAX( K1-J, K2 ), MIN( K3, K4-J )
               A( I, J ) = A( I, J )*MUL
  140       CONTINUE
  150    CONTINUE
*
      END IF
*
      IF( .NOT.DONE )
     $   GO TO 10
*
      RETURN
```

```
*
*     End of DLASCL
*
      END
```

———-

90.168.4 lisp code

— **dlascl.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dlascl (type kl ku cfrom cto m n a lda info)
    (declare (type (array double-float (*)) a)
             (type (double-float) cto cfrom)
             (type (f2cl-lib:integer4) info lda n m ku kl)
             (type (string *) type))
    (f2cl-lib:with-multi-array-data
        ((type character type-%data% type-%offset%)
         (a double-float a-%data% a-%offset%))
      (prog ((bignum 0.0d0) (cfrom1 0.0d0) (cfromc 0.0d0) (cto1 0.0d0)
             (ctoc 0.0d0) (mul 0.0d0) (smlnum 0.0d0) (i 0) (itype 0) (j 0)
             (k1 0) (k2 0) (k3 0) (k4 0) (done nil))
        (declare
         (type (double-float) bignum cfrom1 cfromc cto1 ctoc mul smlnum)
         (type (f2cl-lib:integer4) i itype j k1 k2 k3 k4)
         (type f2cl-lib:logical done))
        (setf info 0)
        (cond
         ((multiple-value-bind (ret-val var-0 var-1)
              (lsame type "G")
            (declare (ignore var-1))
            (when var-0 (setf type var-0))
            ret-val)
          (setf itype 0))
         ((multiple-value-bind (ret-val var-0 var-1)
              (lsame type "L")
            (declare (ignore var-1))
            (when var-0 (setf type var-0))
            ret-val)
          (setf itype 1))
         ((multiple-value-bind (ret-val var-0 var-1)
              (lsame type "U")
            (declare (ignore var-1))
```

```
    (when var-0 (setf type var-0))
    ret-val)
  (setf itype 2))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame type "H")
    (declare (ignore var-1))
    (when var-0 (setf type var-0))
    ret-val)
  (setf itype 3))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame type "B")
    (declare (ignore var-1))
    (when var-0 (setf type var-0))
    ret-val)
  (setf itype 4))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame type "Q")
    (declare (ignore var-1))
    (when var-0 (setf type var-0))
    ret-val)
  (setf itype 5))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame type "Z")
    (declare (ignore var-1))
    (when var-0 (setf type var-0))
    ret-val)
  (setf itype 6))
 (t (setf itype -1)))
(cond ((= itype (f2cl-lib:int-sub 1)) (setf info -1))
      ((or (= cfrom zero)
           (multiple-value-bind (ret-val var-0)
               (disnan cfrom)
             (declare (ignore))
             (setf cfrom var-0)
             ret-val))
       (setf info -4))
      ((multiple-value-bind (ret-val var-0)
           (disnan cto)
         (declare (ignore))
         (setf cto var-0)
         ret-val)
       (setf info -5))
      ((< m 0) (setf info -6))
      ((or (< n 0) (and (= itype 4) (/= n m))
           (and (= itype 5) (/= n m)))
       (setf info -7))
      ((and (<= itype 3)
            (< lda
               (max (the f2cl-lib:integer4 1)
                    (the f2cl-lib:integer4 m))))
```

```
          (setf info -9))
         ((>= itype 4)
          (cond
           ((or (< kl 0)
                (> kl
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
                    (the f2cl-lib:integer4 0))))
            (setf info -2))
           ((or (< ku 0)
                (> ku
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                    (the f2cl-lib:integer4 0)))
                (and (or (= itype 4) (= itype 5)) (/= kl ku)))
            (setf info -3))
           ((or (and (= itype 4) (< lda (f2cl-lib:int-add kl 1)))
                (and (= itype 5) (< lda (f2cl-lib:int-add ku 1)))
                (and (= itype 6)
                     (< lda
                        (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))))
            (setf info -9)))))
 (cond
  ((/= info 0) (xerbla "DLASCL" (f2cl-lib:int-sub info))
   (go end_label)))
 (if (or (= n 0) (= m 0))
     (go end_label))
 (setf smlnum (dlamch "S"))
 (setf bignum (/ one smlnum))
 (setf cfromc cfrom)
 (setf ctoc cto)
label10
 (setf cfrom1 (* cfromc smlnum))
 (cond
  ((= cfrom1 cfromc) (setf mul (/ ctoc cfromc))
   (setf done f2cl-lib:%true%) (setf cto1 ctoc))
  (t (setf cto1 (/ ctoc bignum))
   (cond
    ((= cto1 ctoc) (setf mul ctoc) (setf done f2cl-lib:%true%)
     (setf cfromc one))
    ((and (> (abs cfrom1) (abs ctoc)) (/= ctoc zero)) (setf mul smlnum)
     (setf done f2cl-lib:%false%) (setf cfromc cfrom1))
    ((> (abs cto1) (abs cfromc)) (setf mul bignum)
     (setf done f2cl-lib:%false%) (setf ctoc cto1))
    (t (setf mul (/ ctoc cfromc)) (setf done f2cl-lib:%true%)))))
 (cond
  ((= itype 0)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                  (*
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                   mul))
         label20))
     label30)))
  ((= itype 1)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                  (*
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                   mul))
         label40))
     label50)))
  ((= itype 2)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (min (the f2cl-lib:integer4 j)
                             (the f2cl-lib:integer4 m)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                  (*
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                   mul))
         label60))
     label70)))
  ((= itype 3)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
    (tagbody
```

```
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (min
                         (the f2cl-lib:integer4 (f2cl-lib:int-add j 1))
                         (the f2cl-lib:integer4 m)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                (*
                 (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                 mul))
         label80))
    label90)))
((= itype 4) (setf k3 (f2cl-lib:int-add kl 1))
 (setf k4 (f2cl-lib:int-add n 1))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i
                       (min (the f2cl-lib:integer4 k3)
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add k4
                                                   (f2cl-lib:int-sub
                                                    j)))))
                    nil)
       (tagbody
         (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%)
               (*
                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%)
                mul))
        label100))
    label110)))
((= itype 5) (setf k1 (f2cl-lib:int-add ku 2))
 (setf k3 (f2cl-lib:int-add ku 1))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i
                    (max
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add k1 (f2cl-lib:int-sub j)))
                     (the f2cl-lib:integer4 1))
                    (f2cl-lib:int-add i 1))
                   ((> i k3) nil)
       (tagbody
```

```
              (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%)
                      (*
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                       mul))
            label120))
        label130)))
    ((= itype 6) (setf k1 (f2cl-lib:int-add kl ku 2))
     (setf k2 (f2cl-lib:int-add kl 1))
     (setf k3 (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
     (setf k4 (f2cl-lib:int-add kl ku 1 m))
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i
                        (max
                         (the f2cl-lib:integer4
                              (f2cl-lib:int-add k1 (f2cl-lib:int-sub j)))
                         (the f2cl-lib:integer4 k2))
                        (f2cl-lib:int-add i 1))
                       ((> i
                           (min (the f2cl-lib:integer4 k3)
                                (the f2cl-lib:integer4
                                     (f2cl-lib:int-add k4
                                                       (f2cl-lib:int-sub
                                                        j)))))
                        nil)
           (tagbody
             (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%)
                      (*
                       (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                      a-%offset%)
                       mul))
            label140))
        label150))))
  (if (not done)
      (go label10))
  (go end_label)
 end_label
  (return (values type nil nil cfrom cto nil nil nil nil info))))))
```

———-

90.169 dlasd0

90.169.1 Axiom unit tests

— dlasd0.input —

```
)set break resume
)sys rm -f dlasd0.output
)spool dlasd0.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.169.2 Axiom help page

— dlasd0.help —

```
====================================================================
dlasd0 examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DLASD0  -  divide  and  conquer  approach, DLASD0 computes the singular
       value decomposition (SVD) of a real upper bidiagonal  N-by-M  matrix  B
       with diagonal D and offdiagonal E, where M = N + SQRE

SYNOPSIS
       SUBROUTINE DLASD0( N,  SQRE,  D,  E,  U,  LDU, VT, LDVT, SMLSIZ, IWORK,
                          WORK, INFO )

           INTEGER        INFO, LDU, LDVT, N, SMLSIZ, SQRE

           INTEGER        IWORK( * )

           DOUBLE         PRECISION D( * ), E( * ), U( LDU, * ), VT(  LDVT,  *
                          ), WORK( * )
```

```
Purpose
=======

 Using a divide and conquer approach, DLASD0 computes the singular
 value decomposition (SVD) of a real upper bidiagonal N-by-M
 matrix B with diagonal D and offdiagonal E, where M = N + SQRE.
 The algorithm computes orthogonal matrices U and VT such that
 B = U * S * VT. The singular values S are overwritten on D.

 A related subroutine, DLASDA, computes only the singular values,
 and optionally, the singular vectors in compact form.

 Arguments
 =========

 N      (input) INTEGER
        On entry, the row dimension of the upper bidiagonal matrix.
        This is also the dimension of the main diagonal array D.

 SQRE   (input) INTEGER
        Specifies the column dimension of the bidiagonal matrix.
        = 0: The bidiagonal matrix has column dimension M = N;
        = 1: The bidiagonal matrix has column dimension M = N+1;

 D      (input/output) DOUBLE PRECISION array, dimension (N)
        On entry D contains the main diagonal of the bidiagonal
        matrix.
        On exit D, if INFO = 0, contains its singular values.

 E      (input) DOUBLE PRECISION array, dimension (M-1)
        Contains the subdiagonal entries of the bidiagonal matrix.
        On exit, E has been destroyed.

 U      (output) DOUBLE PRECISION array, dimension at least (LDQ, N)
        On exit, U contains the left singular vectors.

 LDU    (input) INTEGER
        On entry, leading dimension of U.

 VT     (output) DOUBLE PRECISION array, dimension at least (LDVT, M)
        On exit, VT' contains the right singular vectors.

 LDVT   (input) INTEGER
        On entry, leading dimension of VT.

 SMLSIZ (input) INTEGER
        On entry, maximum size of the subproblems at the
        bottom of the computation tree.

 IWORK  INTEGER work array.
```

```
        Dimension must be at least (8 * N)

  WORK   DOUBLE PRECISION work array.
         Dimension must be at least (3 * M**2 + 2 * M)

  INFO   (output) INTEGER
          = 0:  successful exit.
          < 0:  if INFO = -i, the i-th argument had an illegal value.
          > 0:  if INFO = 1, an singular value did not converge

  Further Details
  ===============

  Based on contributions by
     Ming Gu and Huan Ren, Computer Science Division, University of
     California at Berkeley, USA
```

90.169.3 fortran code

— dlasd0.f —

```
      SUBROUTINE DLASD0( N, SQRE, D, E, U, LDU, VT, LDVT, SMLSIZ, IWORK,
     $                   WORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDU, LDVT, N, SMLSIZ, SQRE
*     ..
*     .. Array Arguments ..
      INTEGER            IWORK( * )
      DOUBLE PRECISION   D( * ), E( * ), U( LDU, * ), VT( LDVT, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, I1, IC, IDXQ, IDXQC, IM1, INODE, ITEMP, IWK,
     $                   J, LF, LL, LVL, M, NCC, ND, NDB1, NDIML, NDIMR,
     $                   NL, NLF, NLP1, NLVL, NR, NRF, NRP1, SQREI
      DOUBLE PRECISION   ALPHA, BETA
```

```
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASD1, DLASDQ, DLASDT, XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ( SQRE.LT.0 ) .OR. ( SQRE.GT.1 ) ) THEN
         INFO = -2
      END IF
*
      M = N + SQRE
*
      IF( LDU.LT.N ) THEN
         INFO = -6
      ELSE IF( LDVT.LT.M ) THEN
         INFO = -8
      ELSE IF( SMLSIZ.LT.3 ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASD0', -INFO )
         RETURN
      END IF
*
*     If the input matrix is too small, call DLASDQ to find the SVD.
*
      IF( N.LE.SMLSIZ ) THEN
         CALL DLASDQ( 'U', SQRE, N, M, N, 0, D, E, VT, LDVT, U, LDU, U,
     $                LDU, WORK, INFO )
         RETURN
      END IF
*
*     Set up the computation tree.
*
      INODE = 1
      NDIML = INODE + N
      NDIMR = NDIML + N
      IDXQ = NDIMR + N
      IWK = IDXQ + N
      CALL DLASDT( N, NLVL, ND, IWORK( INODE ), IWORK( NDIML ),
     $             IWORK( NDIMR ), SMLSIZ )
*
*     For the nodes on bottom level of the tree, solve
*     their subproblems by DLASDQ.
```

```
*
      NDB1 = ( ND+1 ) / 2
      NCC = 0
      DO 30 I = NDB1, ND
*
*     IC : center row of each node
*     NL : number of rows of left  subproblem
*     NR : number of rows of right subproblem
*     NLF: starting row of the left   subproblem
*     NRF: starting row of the right  subproblem
*
         I1 = I - 1
         IC = IWORK( INODE+I1 )
         NL = IWORK( NDIML+I1 )
         NLP1 = NL + 1
         NR = IWORK( NDIMR+I1 )
         NRP1 = NR + 1
         NLF = IC - NL
         NRF = IC + 1
         SQREI = 1
         CALL DLASDQ( 'U', SQREI, NL, NLP1, NL, NCC, D( NLF ), E( NLF ),
     $                VT( NLF, NLF ), LDVT, U( NLF, NLF ), LDU,
     $                U( NLF, NLF ), LDU, WORK, INFO )
         IF( INFO.NE.0 ) THEN
            RETURN
         END IF
         ITEMP = IDXQ + NLF - 2
         DO 10 J = 1, NL
            IWORK( ITEMP+J ) = J
   10    CONTINUE
         IF( I.EQ.ND ) THEN
            SQREI = SQRE
         ELSE
            SQREI = 1
         END IF
         NRP1 = NR + SQREI
         CALL DLASDQ( 'U', SQREI, NR, NRP1, NR, NCC, D( NRF ), E( NRF ),
     $                VT( NRF, NRF ), LDVT, U( NRF, NRF ), LDU,
     $                U( NRF, NRF ), LDU, WORK, INFO )
         IF( INFO.NE.0 ) THEN
            RETURN
         END IF
         ITEMP = IDXQ + IC
         DO 20 J = 1, NR
            IWORK( ITEMP+J-1 ) = J
   20    CONTINUE
   30 CONTINUE
*
*     Now conquer each subproblem bottom-up.
*
```

```
      DO 50 LVL = NLVL, 1, -1
*
*        Find the first node LF and last node LL on the
*        current level LVL.
*
         IF( LVL.EQ.1 ) THEN
            LF = 1
            LL = 1
         ELSE
            LF = 2**( LVL-1 )
            LL = 2*LF - 1
         END IF
         DO 40 I = LF, LL
            IM1 = I - 1
            IC = IWORK( INODE+IM1 )
            NL = IWORK( NDIML+IM1 )
            NR = IWORK( NDIMR+IM1 )
            NLF = IC - NL
            IF( ( SQRE.EQ.0 ) .AND. ( I.EQ.LL ) ) THEN
               SQREI = SQRE
            ELSE
               SQREI = 1
            END IF
            IDXQC = IDXQ + NLF - 1
            ALPHA = D( IC )
            BETA = E( IC )
            CALL DLASD1( NL, NR, SQREI, D( NLF ), ALPHA, BETA,
     $                   U( NLF, NLF ), LDU, VT( NLF, NLF ), LDVT,
     $                   IWORK( IDXQC ), IWORK( IWK ), WORK, INFO )
            IF( INFO.NE.0 ) THEN
               RETURN
            END IF
   40    CONTINUE
   50 CONTINUE
*
      RETURN
*
*     End of DLASD0
*
      END
```

———-

90.169.4 lisp code

— dlasd0.lisp —

```
(defun dlasd0 (n sqre d e u ldu vt ldvt smlsiz iwork work info)
  (declare (type (array f2cl-lib:integer4 (*)) iwork)
           (type (array double-float (*)) work vt u e d)
           (type (f2cl-lib:integer4) info smlsiz ldvt ldu sqre n))
  (f2cl-lib:with-multi-array-data
      ((d double-float d-%data% d-%offset%)
       (e double-float e-%data% e-%offset%)
       (u double-float u-%data% u-%offset%)
       (vt double-float vt-%data% vt-%offset%)
       (work double-float work-%data% work-%offset%)
       (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
    (prog ((alpha 0.0d0) (beta 0.0d0) (i 0) (i1 0) (ic 0) (idxq 0) (idxqc 0)
           (im1 0) (inode 0) (itemp 0) (iwk 0) (j 0) (lf 0) (ll 0) (lvl 0)
           (m 0) (ncc 0) (nd 0) (ndb1 0) (ndiml 0) (ndimr 0) (nl 0) (nlf 0)
           (nlp1 0) (nlvl 0) (nr 0) (nrf 0) (nrp1 0) (sqrei 0))
      (declare
       (type (f2cl-lib:integer4) sqrei nrp1 nrf nr nlvl nlp1 nlf nl ndimr ndiml
        ndb1 nd ncc m lvl ll lf j iwk itemp inode im1 idxqc idxq ic i1 i)
       (type (double-float) beta alpha))
      (setf info 0)
      (cond ((< n 0) (setf info -1))
            ((or (< sqre 0) (> sqre 1)) (setf info -2)))
      (setf m (f2cl-lib:int-add n sqre))
      (cond ((< ldu n) (setf info -6)) ((< ldvt m) (setf info -8))
            ((< smlsiz 3) (setf info -9)))
      (cond
       ((/= info 0) (xerbla "DLASD0" (f2cl-lib:int-sub info)) (go end_label)))
      (cond
       ((<= n smlsiz)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
             var-11 var-12 var-13 var-14 var-15)
            (dlasdq "U" sqre n m n 0 d e vt ldvt u ldu u ldu work info)
          (declare (ignore var-0 var-5 var-6 var-7 var-8 var-10 var-12 var-14))
          (when var-1 (setf sqre var-1))
          (when var-2 (setf n var-2))
          (when var-3 (setf m var-3))
          (when var-4 (setf n var-4))
          (when var-9 (setf ldvt var-9))
          (when var-11 (setf ldu var-11))
          (when var-13 (setf ldu var-13))
          (when var-15 (setf info var-15)))
        (go end_label)))
      (setf inode 1)
      (setf ndiml (f2cl-lib:int-add inode n))
      (setf ndimr (f2cl-lib:int-add ndiml n))
      (setf idxq (f2cl-lib:int-add ndimr n))
      (setf iwk (f2cl-lib:int-add idxq n))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (dlasdt n nlvl nd
```

```
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (inode) ((1 *))
                            iwork-%offset%)
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (ndiml) ((1 *))
                            iwork-%offset%)
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (ndimr) ((1 *))
                            iwork-%offset%)
      smlsiz)
  (declare (ignore var-3 var-4 var-5))
  (when var-0 (setf n var-0))
  (when var-1 (setf nlvl var-1))
  (when var-2 (setf nd var-2))
  (when var-6 (setf smlsiz var-6)))
(setf ndb1 (the f2cl-lib:integer4 (truncate (+ nd 1) 2)))
(setf ncc 0)
(f2cl-lib:fdo (i ndb1 (f2cl-lib:int-add i 1))
              ((> i nd) nil)
  (tagbody
    (setf i1 (f2cl-lib:int-sub i 1))
    (setf ic
            (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add inode i1))
                           ((1 *)) iwork-%offset%))
    (setf nl
            (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add ndiml i1))
                           ((1 *)) iwork-%offset%))
    (setf nlp1 (f2cl-lib:int-add nl 1))
    (setf nr
            (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add ndimr i1))
                           ((1 *)) iwork-%offset%))
    (setf nrp1 (f2cl-lib:int-add nr 1))
    (setf nlf (f2cl-lib:int-sub ic nl))
    (setf nrf (f2cl-lib:int-add ic 1))
    (setf sqrei 1)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12 var-13 var-14 var-15)
        (dlasdq "U" sqrei nl nlp1 nl ncc
         (f2cl-lib:array-slice d-%data% double-float (nlf) ((1 *))
                               d-%offset%)
         (f2cl-lib:array-slice e-%data% double-float (nlf) ((1 *))
                               e-%offset%)
         (f2cl-lib:array-slice vt-%data% double-float (nlf nlf)
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt
         (f2cl-lib:array-slice u-%data% double-float (nlf nlf)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu
         (f2cl-lib:array-slice u-%data% double-float (nlf nlf)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu work info)
      (declare (ignore var-0 var-6 var-7 var-8 var-10 var-12 var-14))
```

```
  (when var-1 (setf sqrei var-1))
  (when var-2 (setf nl var-2))
  (when var-3 (setf nlp1 var-3))
  (when var-4 (setf nl var-4))
  (when var-5 (setf ncc var-5))
  (when var-9 (setf ldvt var-9))
  (when var-11 (setf ldu var-11))
  (when var-13 (setf ldu var-13))
  (when var-15 (setf info var-15)))
(cond ((/= info 0) (go end_label)))
(setf itemp (f2cl-lib:int-sub (f2cl-lib:int-add idxq nlf) 2))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j nl) nil)
  (tagbody
    (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add itemp j))
                         ((1 *)) iwork-%offset%)
            j)
   label10))
(cond ((= i nd) (setf sqrei sqre)) (t (setf sqrei 1)))
(setf nrp1 (f2cl-lib:int-add nr sqrei))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15)
    (dlasdq "U" sqrei nr nrp1 nr ncc
     (f2cl-lib:array-slice d-%data% double-float (nrf) ((1 *))
                           d-%offset%)
     (f2cl-lib:array-slice e-%data% double-float (nrf) ((1 *))
                           e-%offset%)
     (f2cl-lib:array-slice vt-%data% double-float (nrf nrf)
                           ((1 ldvt) (1 *)) vt-%offset%)
     ldvt
     (f2cl-lib:array-slice u-%data% double-float (nrf nrf)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu
     (f2cl-lib:array-slice u-%data% double-float (nrf nrf)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu work info)
  (declare (ignore var-0 var-6 var-7 var-8 var-10 var-12 var-14))
  (when var-1 (setf sqrei var-1))
  (when var-2 (setf nr var-2))
  (when var-3 (setf nrp1 var-3))
  (when var-4 (setf nr var-4))
  (when var-5 (setf ncc var-5))
  (when var-9 (setf ldvt var-9))
  (when var-11 (setf ldu var-11))
  (when var-13 (setf ldu var-13))
  (when var-15 (setf info var-15)))
(cond ((/= info 0) (go end_label)))
(setf itemp (f2cl-lib:int-add idxq ic))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                    ((> j nr) nil)
          (tagbody
            (setf (f2cl-lib:fref iwork-%data%
                                 ((f2cl-lib:int-sub
                                   (f2cl-lib:int-add itemp j) 1))
                                 ((1 *)) iwork-%offset%)
                  j)
           label20))
       label30))
    (f2cl-lib:fdo (lvl nlvl (f2cl-lib:int-add lvl (f2cl-lib:int-sub 1)))
                  ((> lvl 1) nil)
      (tagbody
        (cond ((= lvl 1) (setf lf 1) (setf ll 1))
              (t (setf lf (expt 2 (f2cl-lib:int-sub lvl 1)))
               (setf ll (f2cl-lib:int-sub (f2cl-lib:int-mul 2 lf) 1))))
        (f2cl-lib:fdo (i lf (f2cl-lib:int-add i 1))
                      ((> i ll) nil)
          (tagbody
            (setf im1 (f2cl-lib:int-sub i 1))
            (setf ic
                    (f2cl-lib:fref iwork-%data%
                                   ((f2cl-lib:int-add inode im1)) ((1 *))
                                   iwork-%offset%))
            (setf nl
                    (f2cl-lib:fref iwork-%data%
                                   ((f2cl-lib:int-add ndiml im1)) ((1 *))
                                   iwork-%offset%))
            (setf nr
                    (f2cl-lib:fref iwork-%data%
                                   ((f2cl-lib:int-add ndimr im1)) ((1 *))
                                   iwork-%offset%))
            (setf nlf (f2cl-lib:int-sub ic nl))
            (cond ((and (= sqre 0) (= i ll)) (setf sqrei sqre))
                  (t (setf sqrei 1)))
            (setf idxqc (f2cl-lib:int-sub (f2cl-lib:int-add idxq nlf) 1))
            (setf alpha (f2cl-lib:fref d-%data% (ic) ((1 *)) d-%offset%))
            (setf beta (f2cl-lib:fref e-%data% (ic) ((1 *)) e-%offset%))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
                 var-10 var-11 var-12 var-13)
                (dlasd1 nl nr sqrei
                 (f2cl-lib:array-slice d-%data% double-float (nlf) ((1 *))
                                       d-%offset%)
                 alpha beta
                 (f2cl-lib:array-slice u-%data% double-float (nlf nlf)
                                       ((1 ldu) (1 *)) u-%offset%)
                 ldu
                 (f2cl-lib:array-slice vt-%data% double-float (nlf nlf)
                                       ((1 ldvt) (1 *)) vt-%offset%)
                 ldvt
```

```
                  (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (idxqc)
                                        ((1 *)) iwork-%offset%)
                  (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (iwk)
                                        ((1 *)) iwork-%offset%)
                  work info)
                (declare (ignore var-3 var-6 var-8 var-10 var-11 var-12))
                (when var-0 (setf nl var-0))
                (when var-1 (setf nr var-1))
                (when var-2 (setf sqrei var-2))
                (when var-4 (setf alpha var-4))
                (when var-5 (setf beta var-5))
                (when var-7 (setf ldu var-7))
                (when var-9 (setf ldvt var-9))
                (when var-13 (setf info var-13)))
              (cond ((/= info 0) (go end_label)))
            label40))
        label50))
      (go end_label)
     end_label
      (return (values n sqre nil nil nil ldu nil ldvt smlsiz nil nil info)))))
```

———-

90.170 dlasd1

90.170.1 Axiom unit tests

— **dlasd1.input** —

```
)set break resume
)sys rm -f dlasd1.output
)spool dlasd1.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.170.2 Axiom help page

— **dlasd1.help** —

```
=====================================================================
dlasd1 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

NAME
       DLASD1 - the SVD of an upper bidiagonal N-by-M matrix B,

SYNOPSIS
       SUBROUTINE DLASD1( NL,  NR,  SQRE,  D,  ALPHA,  BETA, U, LDU, VT, LDVT,
                          IDXQ, IWORK, WORK, INFO )

           INTEGER        INFO, LDU, LDVT, NL, NR, SQRE

           DOUBLE         PRECISION ALPHA, BETA

           INTEGER        IDXQ( * ), IWORK( * )

           DOUBLE         PRECISION D( * ), U( LDU, * ), VT( LDVT, * ),  WORK(
                          * )

  Purpose
  =======

  DLASD1 computes the SVD of an upper bidiagonal N-by-M matrix B,
  where N = NL + NR + 1 and M = N + SQRE. DLASD1 is called from DLASD0.

  A related subroutine DLASD7 handles the case in which the singular
  values (and the singular vectors in factored form) are desired.

  DLASD1 computes the SVD as follows:

               ( D1(in)  0    0     0 )
    B = U(in) * (   Z1'   a   Z2'    b ) * VT(in)
               (   0     0   D2(in) 0 )

      = U(out) * ( D(out) 0) * VT(out)

  where Z' = (Z1' a Z2' b) = u' VT', and u is a vector of dimension M
  with ALPHA and BETA in the NL+1 and NL+2 th entries and zeros
  elsewhere; and the entry b is empty if SQRE = 0.

  The left singular vectors of the original matrix are stored in U, and
  the transpose of the right singular vectors are stored in VT, and the
  singular values are in D.  The algorithm consists of three stages:

     The first stage consists of deflating the size of the problem
```

```
   when there are multiple singular values or when there are zeros in
   the Z vector.  For each such occurence the dimension of the
   secular equation problem is reduced by one.  This stage is
   performed by the routine DLASD2.

   The second stage consists of calculating the updated
   singular values. This is done by finding the square roots of the
   roots of the secular equation via the routine DLASD4 (as called
   by DLASD3). This routine also calculates the singular vectors of
   the current problem.

   The final stage consists of computing the updated singular vectors
   directly using the updated singular values.  The singular vectors
   for the current problem are multiplied with the singular vectors
   from the overall problem.

Arguments
=========

NL     (input) INTEGER
       The row dimension of the upper block.  NL >= 1.

NR     (input) INTEGER
       The row dimension of the lower block.  NR >= 1.

SQRE   (input) INTEGER
       = 0: the lower block is an NR-by-NR square matrix.
       = 1: the lower block is an NR-by-(NR+1) rectangular matrix.

       The bidiagonal matrix has row dimension N = NL + NR + 1,
       and column dimension M = N + SQRE.

D      (input/output) DOUBLE PRECISION array,
                      dimension (N = NL+NR+1).
       On entry D(1:NL,1:NL) contains the singular values of the
       upper block; and D(NL+2:N) contains the singular values of
       the lower block. On exit D(1:N) contains the singular values
       of the modified matrix.

ALPHA  (input) DOUBLE PRECISION
       Contains the diagonal element associated with the added row.

BETA   (input) DOUBLE PRECISION
       Contains the off-diagonal element associated with the added
       row.

U      (input/output) DOUBLE PRECISION array, dimension(LDU,N)
       On entry U(1:NL, 1:NL) contains the left singular vectors of
       the upper block; U(NL+2:N, NL+2:N) contains the left singular
       vectors of the lower block. On exit U contains the left
```

```
        singular vectors of the bidiagonal matrix.

 LDU    (input) INTEGER
        The leading dimension of the array U.  LDU >= max( 1, N ).

 VT     (input/output) DOUBLE PRECISION array, dimension(LDVT,M)
        where M = N + SQRE.
        On entry VT(1:NL+1, 1:NL+1)' contains the right singular
        vectors of the upper block; VT(NL+2:M, NL+2:M)' contains
        the right singular vectors of the lower block. On exit
        VT' contains the right singular vectors of the
        bidiagonal matrix.

 LDVT   (input) INTEGER
        The leading dimension of the array VT.  LDVT >= max( 1, M ).

 IDXQ  (output) INTEGER array, dimension(N)
        This contains the permutation which will reintegrate the
        subproblem just solved back into sorted order, i.e.
        D( IDXQ( I = 1, N ) ) will be in ascending order.

 IWORK  (workspace) INTEGER array, dimension( 4 * N )

 WORK   (workspace) DOUBLE PRECISION array, dimension( 3*M**2 + 2*M )

 INFO   (output) INTEGER
         = 0:  successful exit.
         < 0:  if INFO = -i, the i-th argument had an illegal value.
         > 0:  if INFO = 1, an singular value did not converge

 Further Details
 ===============

 Based on contributions by
    Ming Gu and Huan Ren, Computer Science Division, University of
    California at Berkeley, USA
```

———-

90.170.3 fortran code

— dlasd1.f —

```
      SUBROUTINE DLASD1( NL, NR, SQRE, D, ALPHA, BETA, U, LDU, VT, LDVT,
     $                   IDXQ, IWORK, WORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.0) --
```

```
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      INTEGER            INFO, LDU, LDVT, NL, NR, SQRE
      DOUBLE PRECISION   ALPHA, BETA
*     ..
*     .. Array Arguments ..
      INTEGER            IDXQ( * ), IWORK( * )
      DOUBLE PRECISION   D( * ), U( LDU, * ), VT( LDVT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
*
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            COLTYP, I, IDX, IDXC, IDXP, IQ, ISIGMA, IU2,
     $                   IVT2, IZ, K, LDQ, LDU2, LDVT2, M, N, N1, N2
      DOUBLE PRECISION   ORGNRM
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLAMRG, DLASCL, DLASD2, DLASD3, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( NL.LT.1 ) THEN
         INFO = -1
      ELSE IF( NR.LT.1 ) THEN
         INFO = -2
      ELSE IF( ( SQRE.LT.0 ) .OR. ( SQRE.GT.1 ) ) THEN
         INFO = -3
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASD1', -INFO )
         RETURN
      END IF
*
      N = NL + NR + 1
```

```
      M = N + SQRE
*
*     The following values are for bookkeeping purposes only.  They are
*     integer pointers which indicate the portion of the workspace
*     used by a particular array in DLASD2 and DLASD3.
*
      LDU2 = N
      LDVT2 = M
*
      IZ = 1
      ISIGMA = IZ + M
      IU2 = ISIGMA + N
      IVT2 = IU2 + LDU2*N
      IQ = IVT2 + LDVT2*M
*
      IDX = 1
      IDXC = IDX + N
      COLTYP = IDXC + N
      IDXP = COLTYP + N
*
*     Scale.
*
      ORGNRM = MAX( ABS( ALPHA ), ABS( BETA ) )
      D( NL+1 ) = ZERO
      DO 10 I = 1, N
         IF( ABS( D( I ) ).GT.ORGNRM ) THEN
            ORGNRM = ABS( D( I ) )
         END IF
   10 CONTINUE
      CALL DLASCL( 'G', 0, 0, ORGNRM, ONE, N, 1, D, N, INFO )
      ALPHA = ALPHA / ORGNRM
      BETA = BETA / ORGNRM
*
*     Deflate singular values.
*
      CALL DLASD2( NL, NR, SQRE, K, D, WORK( IZ ), ALPHA, BETA, U, LDU,
     $             VT, LDVT, WORK( ISIGMA ), WORK( IU2 ), LDU2,
     $             WORK( IVT2 ), LDVT2, IWORK( IDXP ), IWORK( IDX ),
     $             IWORK( IDXC ), IDXQ, IWORK( COLTYP ), INFO )
*
*     Solve Secular Equation and update singular vectors.
*
      LDQ = K
      CALL DLASD3( NL, NR, SQRE, K, D, WORK( IQ ), LDQ, WORK( ISIGMA ),
     $             U, LDU, WORK( IU2 ), LDU2, VT, LDVT, WORK( IVT2 ),
     $             LDVT2, IWORK( IDXC ), IWORK( COLTYP ), WORK( IZ ),
     $             INFO )
      IF( INFO.NE.0 ) THEN
         RETURN
      END IF
```

```
*
*     Unscale.
*
      CALL DLASCL( 'G', 0, 0, ONE, ORGNRM, N, 1, D, N, INFO )
*
*     Prepare the IDXQ sorting permutation.
*
      N1 = K
      N2 = N - K
      CALL DLAMRG( N1, N2, D, 1, -1, IDXQ )
*
      RETURN
*
*     End of DLASD1
*
      END
```

90.170.4 lisp code

— dlasd1.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dlasd1 (nl nr sqre d alpha beta u ldu vt ldvt idxq iwork work info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork idxq)
             (type (double-float) beta alpha)
             (type (array double-float (*)) work vt u d)
             (type (f2cl-lib:integer4) info ldvt ldu sqre nr nl))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (u double-float u-%data% u-%offset%)
         (vt double-float vt-%data% vt-%offset%)
         (work double-float work-%data% work-%offset%)
         (idxq f2cl-lib:integer4 idxq-%data% idxq-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((orgnrm 0.0d0) (coltyp 0) (i 0) (idx 0) (idxc 0) (idxp 0) (iq 0)
             (isigma 0) (iu2 0) (ivt2 0) (iz 0) (k 0) (ldq 0) (ldu2 0)
             (ldvt2 0) (m 0) (n 0) (n1 0) (n2 0))
        (declare (type (double-float) orgnrm)
                 (type (f2cl-lib:integer4) coltyp i idx idxc idxp iq isigma iu2
                  ivt2 iz k ldq ldu2 ldvt2 m n n1 n2))
        (setf info 0)
        (cond ((< nl 1) (setf info -1)) ((< nr 1) (setf info -2))
```

```
      ((or (< sqre 0) (> sqre 1)) (setf info -3)))
(cond
 ((/= info 0) (xerbla "DLASD1" (f2cl-lib:int-sub info))
  (go end_label)))
(setf n (f2cl-lib:int-add nl nr 1))
(setf m (f2cl-lib:int-add n sqre))
(setf ldu2 n)
(setf ldvt2 m)
(setf iz 1)
(setf isigma (f2cl-lib:int-add iz m))
(setf iu2 (f2cl-lib:int-add isigma n))
(setf ivt2 (f2cl-lib:int-add iu2 (f2cl-lib:int-mul ldu2 n)))
(setf iq (f2cl-lib:int-add ivt2 (f2cl-lib:int-mul ldvt2 m)))
(setf idx 1)
(setf idxc (f2cl-lib:int-add idx n))
(setf coltyp (f2cl-lib:int-add idxc n))
(setf idxp (f2cl-lib:int-add coltyp n))
(setf orgnrm (max (abs alpha) (abs beta)))
(setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add nl 1)) ((1 *))
                    d-%offset%)
        zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (abs (f2cl-lib:fref d (i) ((1 *)))) orgnrm)
      (setf orgnrm
              (abs (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))))
   label10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dlascl "G" 0 0 orgnrm one n 1 d n info)
  (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
  (setf orgnrm var-3)
  (setf one var-4)
  (setf info var-9))
(setf alpha (/ alpha orgnrm))
(setf beta (/ beta orgnrm))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19
     var-20 var-21 var-22)
    (dlasd2 nl nr sqre k d
     (f2cl-lib:array-slice work-%data% double-float (iz) ((1 *))
                           work-%offset%)
     alpha beta u ldu vt ldvt
     (f2cl-lib:array-slice work-%data% double-float (isigma) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iu2) ((1 *))
                           work-%offset%)
```

```
      ldu2
      (f2cl-lib:array-slice work-%data% double-float (ivt2) ((1 *))
                            work-%offset%)
      ldvt2
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (idxp)
                            ((1 *)) iwork-%offset%)
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (idx) ((1 *))
                            iwork-%offset%)
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (idxc)
                            ((1 *)) iwork-%offset%)
      idxq
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (coltyp)
                            ((1 *)) iwork-%offset%)
      info)
    (declare
     (ignore var-4 var-5 var-8 var-10 var-12 var-13 var-15 var-17 var-18
      var-19 var-20 var-21))
    (when var-0 (setf nl var-0))
    (when var-1 (setf nr var-1))
    (when var-2 (setf sqre var-2))
    (when var-3 (setf k var-3))
    (when var-6 (setf alpha var-6))
    (when var-7 (setf beta var-7))
    (when var-9 (setf ldu var-9))
    (when var-11 (setf ldvt var-11))
    (when var-14 (setf ldu2 var-14))
    (when var-16 (setf ldvt2 var-16))
    (when var-22 (setf info var-22)))
  (setf ldq k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
       var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19)
      (dlasd3 nl nr sqre k d
       (f2cl-lib:array-slice work-%data% double-float (iq) ((1 *))
                             work-%offset%)
       ldq
       (f2cl-lib:array-slice work-%data% double-float (isigma) ((1 *))
                             work-%offset%)
       u ldu
       (f2cl-lib:array-slice work-%data% double-float (iu2) ((1 *))
                             work-%offset%)
       ldu2 vt ldvt
       (f2cl-lib:array-slice work-%data% double-float (ivt2) ((1 *))
                             work-%offset%)
       ldvt2
       (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (idxc)
                             ((1 *)) iwork-%offset%)
       (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (coltyp)
                             ((1 *)) iwork-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (iz) ((1 *))
```

```
                                work-%offset%)
            info)
          (declare
           (ignore var-4 var-5 var-7 var-8 var-10 var-12 var-14 var-16 var-17
            var-18))
          (when var-0 (setf nl var-0))
          (when var-1 (setf nr var-1))
          (when var-2 (setf sqre var-2))
          (when var-3 (setf k var-3))
          (when var-6 (setf ldq var-6))
          (when var-9 (setf ldu var-9))
          (when var-11 (setf ldu2 var-11))
          (when var-13 (setf ldvt var-13))
          (when var-15 (setf ldvt2 var-15))
          (when var-19 (setf info var-19)))
        (cond ((/= info 0) (go end_label)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
            (dlascl "G" 0 0 one orgnrm n 1 d n info)
          (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
          (setf one var-3)
          (setf orgnrm var-4)
          (setf info var-9))
        (setf n1 k)
        (setf n2 (f2cl-lib:int-sub n k))
        (dlamrg n1 n2 d 1 -1 idxq)
        (go end_label)
       end_label
        (return
         (values nl nr sqre nil alpha beta nil ldu nil ldvt nil nil nil
                 info))))))
```

———-

90.171 dlasd2

90.171.1 Axiom unit tests

— dlasd2.input —

```
)set break resume
)sys rm -f dlasd2.output
)spool dlasd2.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

90.171.2 Axiom help page

— **dlasd2.help** —

```
=====================================================================
dlasd2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

NAME
       DLASD2  - the two sets of singular values together into a single sorted
       set

SYNOPSIS
       SUBROUTINE DLASD2( NL, NR, SQRE, K, D, Z,  ALPHA,  BETA,  U,  LDU,  VT,
                          LDVT, DSIGMA, U2, LDU2, VT2, LDVT2, IDXP, IDX, IDXC,
                          IDXQ, COLTYP, INFO )

           INTEGER        INFO, K, LDU, LDU2, LDVT, LDVT2, NL, NR, SQRE

           DOUBLE         PRECISION ALPHA, BETA

           INTEGER        COLTYP( * ), IDX( * ), IDXC( * ), IDXP( * ), IDXQ( *
                          )

           DOUBLE         PRECISION  D(  *  ),  DSIGMA(  * ), U( LDU, * ), U2(
                          LDU2, * ), VT( LDVT, * ), VT2( LDVT2, * ), Z( * )

  Purpose
  =======

  DLASD2 merges the two sets of singular values together into a single
  sorted set.  Then it tries to deflate the size of the problem.
  There are two ways in which deflation can occur:  when two or more
  singular values are close together or if there is a tiny entry in the
  Z vector.  For each such occurrence the order of the related secular
  equation problem is reduced by one.

  DLASD2 is called from DLASD1.
```

```
Arguments
=========

NL     (input) INTEGER
       The row dimension of the upper block.  NL >= 1.

NR     (input) INTEGER
       The row dimension of the lower block.  NR >= 1.

SQRE   (input) INTEGER
       = 0: the lower block is an NR-by-NR square matrix.
       = 1: the lower block is an NR-by-(NR+1) rectangular matrix.

       The bidiagonal matrix has N = NL + NR + 1 rows and
       M = N + SQRE >= N columns.

K      (output) INTEGER
       Contains the dimension of the non-deflated matrix,
       This is the order of the related secular equation. 1 <= K <=N.

D      (input/output) DOUBLE PRECISION array, dimension(N)
       On entry D contains the singular values of the two submatrices
       to be combined.  On exit D contains the trailing (N-K) updated
       singular values (those which were deflated) sorted into
       increasing order.

ALPHA  (input) DOUBLE PRECISION
       Contains the diagonal element associated with the added row.

BETA   (input) DOUBLE PRECISION
       Contains the off-diagonal element associated with the added
       row.

U      (input/output) DOUBLE PRECISION array, dimension(LDU,N)
       On entry U contains the left singular vectors of two
       submatrices in the two square blocks with corners at (1,1),
       (NL, NL), and (NL+2, NL+2), (N,N).
       On exit U contains the trailing (N-K) updated left singular
       vectors (those which were deflated) in its last N-K columns.

LDU    (input) INTEGER
       The leading dimension of the array U.  LDU >= N.

Z      (output) DOUBLE PRECISION array, dimension(N)
       On exit Z contains the updating row vector in the secular
       equation.

DSIGMA (output) DOUBLE PRECISION array, dimension (N)
       Contains a copy of the diagonal elements (K-1 singular values
```

```
        and one zero) in the secular equation.

  U2    (output) DOUBLE PRECISION array, dimension(LDU2,N)
        Contains a copy of the first K-1 left singular vectors which
        will be used by DLASD3 in a matrix multiply (DGEMM) to solve
        for the new left singular vectors. U2 is arranged into four
        blocks. The first block contains a column with 1 at NL+1 and
        zero everywhere else; the second block contains non-zero
        entries only at and above NL; the third contains non-zero
        entries only below NL+1; and the fourth is dense.

  LDU2  (input) INTEGER
        The leading dimension of the array U2.  LDU2 >= N.

  VT    (input/output) DOUBLE PRECISION array, dimension(LDVT,M)
        On entry VT' contains the right singular vectors of two
        submatrices in the two square blocks with corners at (1,1),
        (NL+1, NL+1), and (NL+2, NL+2), (M,M).
        On exit VT' contains the trailing (N-K) updated right singular
        vectors (those which were deflated) in its last N-K columns.
        In case SQRE =1, the last row of VT spans the right null
        space.

  LDVT  (input) INTEGER
        The leading dimension of the array VT.  LDVT >= M.

  VT2   (output) DOUBLE PRECISION array, dimension(LDVT2,N)
        VT2' contains a copy of the first K right singular vectors
        which will be used by DLASD3 in a matrix multiply (DGEMM) to
        solve for the new right singular vectors. VT2 is arranged into
        three blocks. The first block contains a row that corresponds
        to the special 0 diagonal element in SIGMA; the second block
        contains non-zeros only at and before NL +1; the third block
        contains non-zeros only at and after  NL +2.

  LDVT2 (input) INTEGER
        The leading dimension of the array VT2.  LDVT2 >= M.

  IDXP  (workspace) INTEGER array, dimension(N)
        This will contain the permutation used to place deflated
        values of D at the end of the array. On output IDXP(2:K)
        points to the nondeflated D-values and IDXP(K+1:N)
        points to the deflated singular values.

  IDX   (workspace) INTEGER array, dimension(N)
        This will contain the permutation used to sort the contents of
        D into ascending order.

  IDXC  (output) INTEGER array, dimension(N)
        This will contain the permutation used to arrange the columns
```

```
        of the deflated U matrix into three groups:  the first group
        contains non-zero entries only at and above NL, the second
        contains non-zero entries only below NL+2, and the third is
        dense.

  COLTYP (workspace/output) INTEGER array, dimension(N)
        As workspace, this will contain a label which will indicate
        which of the following types a column in the U2 matrix or a
        row in the VT2 matrix is:
        1 : non-zero in the upper half only
        2 : non-zero in the lower half only
        3 : dense
        4 : deflated

        On exit, it is an array of dimension 4, with COLTYP(I) being
        the dimension of the I-th type columns.

  IDXQ   (input) INTEGER array, dimension(N)
        This contains the permutation which separately sorts the two
        sub-problems in D into ascending order.  Note that entries in
        the first hlaf of this permutation must first be moved one
        position backward; and entries in the second half
        must first have NL+1 added to their values.

  INFO   (output) INTEGER
         = 0:  successful exit.
         < 0:  if INFO = -i, the i-th argument had an illegal value.

  Further Details
  ===============

  Based on contributions by
     Ming Gu and Huan Ren, Computer Science Division, University of
     California at Berkeley, USA
```

———-

90.171.3 fortran code

— dlasd2.f —

```
      SUBROUTINE DLASD2( NL, NR, SQRE, K, D, Z, ALPHA, BETA, U, LDU, VT,
     $                   LDVT, DSIGMA, U2, LDU2, VT2, LDVT2, IDXP, IDX,
     $                   IDXC, IDXQ, COLTYP, INFO )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Oak Ridge National Lab, Argonne National Lab,
```

```
*     Courant Institute, NAG Ltd., and Rice University
*     October 31, 1999
*
*     .. Scalar Arguments ..
      INTEGER            INFO, K, LDU, LDU2, LDVT, LDVT2, NL, NR, SQRE
      DOUBLE PRECISION   ALPHA, BETA
*     ..
*     .. Array Arguments ..
      INTEGER            COLTYP( * ), IDX( * ), IDXC( * ), IDXP( * ),
     $                   IDXQ( * )
      DOUBLE PRECISION   D( * ), DSIGMA( * ), U( LDU, * ),
     $                   U2( LDU2, * ), VT( LDVT, * ), VT2( LDVT2, * ),
     $                   Z( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE, TWO, EIGHT
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0, TWO = 2.0D+0,
     $                   EIGHT = 8.0D+0 )
*     ..
*     .. Local Arrays ..
      INTEGER            CTOT( 4 ), PSM( 4 )
*     ..
*     .. Local Scalars ..
      INTEGER            CT, I, IDXI, IDXJ, IDXJP, J, JP, JPREV, K2, M,
     $                   N, NLP1, NLP2
      DOUBLE PRECISION   C, EPS, HLFTOL, S, TAU, TOL, Z1
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH, DLAPY2
      EXTERNAL           DLAMCH, DLAPY2
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DLACPY, DLAMRG, DLASET, DROT, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( NL.LT.1 ) THEN
         INFO = -1
      ELSE IF( NR.LT.1 ) THEN
         INFO = -2
```

```
      ELSE IF( ( SQRE.NE.1 ) .AND. ( SQRE.NE.0 ) ) THEN
         INFO = -3
      END IF
*
      N = NL + NR + 1
      M = N + SQRE
*
      IF( LDU.LT.N ) THEN
         INFO = -10
      ELSE IF( LDVT.LT.M ) THEN
         INFO = -12
      ELSE IF( LDU2.LT.N ) THEN
         INFO = -15
      ELSE IF( LDVT2.LT.M ) THEN
         INFO = -17
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASD2', -INFO )
         RETURN
      END IF
*
      NLP1 = NL + 1
      NLP2 = NL + 2
*
*     Generate the first part of the vector Z; and move the singular
*     values in the first part of D one position backward.
*
      Z1 = ALPHA*VT( NLP1, NLP1 )
      Z( 1 ) = Z1
      DO 10 I = NL, 1, -1
         Z( I+1 ) = ALPHA*VT( I, NLP1 )
         D( I+1 ) = D( I )
         IDXQ( I+1 ) = IDXQ( I ) + 1
   10 CONTINUE
*
*     Generate the second part of the vector Z.
*
      DO 20 I = NLP2, M
         Z( I ) = BETA*VT( I, NLP2 )
   20 CONTINUE
*
*     Initialize some reference arrays.
*
      DO 30 I = 2, NLP1
         COLTYP( I ) = 1
   30 CONTINUE
      DO 40 I = NLP2, N
         COLTYP( I ) = 2
   40 CONTINUE
*
```

```
*     Sort the singular values into increasing order
*
      DO 50 I = NLP2, N
         IDXQ( I ) = IDXQ( I ) + NLP1
   50 CONTINUE
*
*     DSIGMA, IDXC, IDXC, and the first column of U2
*     are used as storage space.
*
      DO 60 I = 2, N
         DSIGMA( I ) = D( IDXQ( I ) )
         U2( I, 1 ) = Z( IDXQ( I ) )
         IDXC( I ) = COLTYP( IDXQ( I ) )
   60 CONTINUE
*
      CALL DLAMRG( NL, NR, DSIGMA( 2 ), 1, 1, IDX( 2 ) )
*
      DO 70 I = 2, N
         IDXI = 1 + IDX( I )
         D( I ) = DSIGMA( IDXI )
         Z( I ) = U2( IDXI, 1 )
         COLTYP( I ) = IDXC( IDXI )
   70 CONTINUE
*
*     Calculate the allowable deflation tolerance
*
      EPS = DLAMCH( 'Epsilon' )
      TOL = MAX( ABS( ALPHA ), ABS( BETA ) )
      TOL = EIGHT*EPS*MAX( ABS( D( N ) ), TOL )
*
*     There are 2 kinds of deflation -- first a value in the z-vector
*     is small, second two (or more) singular values are very close
*     together (their difference is small).
*
*     If the value in the z-vector is small, we simply permute the
*     array so that the corresponding singular value is moved to the
*     end.
*
*     If two values in the D-vector are close, we perform a two-sided
*     rotation designed to make one of the corresponding z-vector
*     entries zero, and then permute the array so that the deflated
*     singular value is moved to the end.
*
*     If there are multiple singular values then the problem deflates.
*     Here the number of equal singular values are found.  As each equal
*     singular value is found, an elementary reflector is computed to
*     rotate the corresponding singular subspace so that the
*     corresponding components of Z are zero in this new basis.
*
      K = 1
```

```
      K2 = N + 1
      DO 80 J = 2, N
         IF( ABS( Z( J ) ).LE.TOL ) THEN
*
*           Deflate due to small z component.
*
            K2 = K2 - 1
            IDXP( K2 ) = J
            COLTYP( J ) = 4
            IF( J.EQ.N )
     $         GO TO 120
         ELSE
            JPREV = J
            GO TO 90
         END IF
   80 CONTINUE
   90 CONTINUE
      J = JPREV
  100 CONTINUE
      J = J + 1
      IF( J.GT.N )
     $   GO TO 110
      IF( ABS( Z( J ) ).LE.TOL ) THEN
*
*        Deflate due to small z component.
*
         K2 = K2 - 1
         IDXP( K2 ) = J
         COLTYP( J ) = 4
      ELSE
*
*        Check if singular values are close enough to allow deflation.
*
         IF( ABS( D( J )-D( JPREV ) ).LE.TOL ) THEN
*
*           Deflation is possible.
*
            S = Z( JPREV )
            C = Z( J )
*
*           Find sqrt(a**2+b**2) without overflow or
*           destructive underflow.
*
            TAU = DLAPY2( C, S )
            C = C / TAU
            S = -S / TAU
            Z( J ) = TAU
            Z( JPREV ) = ZERO
*
*           Apply back the Givens rotation to the left and right
```

```
*           singular vector matrices.
*
            IDXJP = IDXQ( IDX( JPREV )+1 )
            IDXJ = IDXQ( IDX( J )+1 )
            IF( IDXJP.LE.NLP1 ) THEN
               IDXJP = IDXJP - 1
            END IF
            IF( IDXJ.LE.NLP1 ) THEN
               IDXJ = IDXJ - 1
            END IF
            CALL DROT( N, U( 1, IDXJP ), 1, U( 1, IDXJ ), 1, C, S )
            CALL DROT( M, VT( IDXJP, 1 ), LDVT, VT( IDXJ, 1 ), LDVT, C,
     $                 S )
            IF( COLTYP( J ).NE.COLTYP( JPREV ) ) THEN
               COLTYP( J ) = 3
            END IF
            COLTYP( JPREV ) = 4
            K2 = K2 - 1
            IDXP( K2 ) = JPREV
            JPREV = J
         ELSE
            K = K + 1
            U2( K, 1 ) = Z( JPREV )
            DSIGMA( K ) = D( JPREV )
            IDXP( K ) = JPREV
            JPREV = J
         END IF
      END IF
      GO TO 100
  110 CONTINUE
*
*     Record the last singular value.
*
      K = K + 1
      U2( K, 1 ) = Z( JPREV )
      DSIGMA( K ) = D( JPREV )
      IDXP( K ) = JPREV
*
  120 CONTINUE
*
*     Count up the total number of the various types of columns, then
*     form a permutation which positions the four column types into
*     four groups of uniform structure (although one or more of these
*     groups may be empty).
*
      DO 130 J = 1, 4
         CTOT( J ) = 0
  130 CONTINUE
      DO 140 J = 2, N
         CT = COLTYP( J )
```

```
         CTOT( CT ) = CTOT( CT ) + 1
  140 CONTINUE
*
*     PSM(*) = Position in SubMatrix (of types 1 through 4)
*
      PSM( 1 ) = 2
      PSM( 2 ) = 2 + CTOT( 1 )
      PSM( 3 ) = PSM( 2 ) + CTOT( 2 )
      PSM( 4 ) = PSM( 3 ) + CTOT( 3 )
*
*     Fill out the IDXC array so that the permutation which it induces
*     will place all type-1 columns first, all type-2 columns next,
*     then all type-3's, and finally all type-4's, starting from the
*     second column. This applies similarly to the rows of VT.
*
      DO 150 J = 2, N
         JP = IDXP( J )
         CT = COLTYP( JP )
         IDXC( PSM( CT ) ) = J
         PSM( CT ) = PSM( CT ) + 1
  150 CONTINUE
*
*     Sort the singular values and corresponding singular vectors into
*     DSIGMA, U2, and VT2 respectively.  The singular values/vectors
*     which were not deflated go into the first K slots of DSIGMA, U2,
*     and VT2 respectively, while those which were deflated go into the
*     last N - K slots, except that the first column/row will be treated
*     separately.
*
      DO 160 J = 2, N
         JP = IDXP( J )
         DSIGMA( J ) = D( JP )
         IDXJ = IDXQ( IDX( IDXP( IDXC( J ) ) )+1 )
         IF( IDXJ.LE.NLP1 ) THEN
            IDXJ = IDXJ - 1
         END IF
         CALL DCOPY( N, U( 1, IDXJ ), 1, U2( 1, J ), 1 )
         CALL DCOPY( M, VT( IDXJ, 1 ), LDVT, VT2( J, 1 ), LDVT2 )
  160 CONTINUE
*
*     Determine DSIGMA(1), DSIGMA(2) and Z(1)
*
      DSIGMA( 1 ) = ZERO
      HLFTOL = TOL / TWO
      IF( ABS( DSIGMA( 2 ) ).LE.HLFTOL )
     $   DSIGMA( 2 ) = HLFTOL
      IF( M.GT.N ) THEN
         Z( 1 ) = DLAPY2( Z1, Z( M ) )
         IF( Z( 1 ).LE.TOL ) THEN
            C = ONE
```

```
            S = ZERO
            Z( 1 ) = TOL
         ELSE
            C = Z1 / Z( 1 )
            S = Z( M ) / Z( 1 )
         END IF
      ELSE
         IF( ABS( Z1 ).LE.TOL ) THEN
            Z( 1 ) = TOL
         ELSE
            Z( 1 ) = Z1
         END IF
      END IF
*
*     Move the rest of the updating row to Z.
*
      CALL DCOPY( K-1, U2( 2, 1 ), 1, Z( 2 ), 1 )
*
*     Determine the first column of U2, the first row of VT2 and the
*     last row of VT.
*
      CALL DLASET( 'A', N, 1, ZERO, ZERO, U2, LDU2 )
      U2( NLP1, 1 ) = ONE
      IF( M.GT.N ) THEN
         DO 170 I = 1, NLP1
            VT( M, I ) = -S*VT( NLP1, I )
            VT2( 1, I ) = C*VT( NLP1, I )
  170    CONTINUE
         DO 180 I = NLP2, M
            VT2( 1, I ) = S*VT( M, I )
            VT( M, I ) = C*VT( M, I )
  180    CONTINUE
      ELSE
         CALL DCOPY( M, VT( NLP1, 1 ), LDVT, VT2( 1, 1 ), LDVT2 )
      END IF
      IF( M.GT.N ) THEN
         CALL DCOPY( M, VT( M, 1 ), LDVT, VT2( M, 1 ), LDVT2 )
      END IF
*
*     The deflated singular values and their corresponding vectors go
*     into the back of D, U, and V respectively.
*
      IF( N.GT.K ) THEN
         CALL DCOPY( N-K, DSIGMA( K+1 ), 1, D( K+1 ), 1 )
         CALL DLACPY( 'A', N, N-K, U2( 1, K+1 ), LDU2, U( 1, K+1 ),
     $                LDU )
         CALL DLACPY( 'A', N-K, M, VT2( K+1, 1 ), LDVT2, VT( K+1, 1 ),
     $                LDVT )
      END IF
*
```

```
*     Copy CTOT into COLTYP for referencing in DLASD3.
*
      DO 190 J = 1, 4
         COLTYP( J ) = CTOT( J )
  190 CONTINUE
*
      RETURN
*
*     End of DLASD2
*
      END
```

———-

90.171.4 lisp code

— **dlasd2.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0) (two 2.0d0) (eight 8.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 8.0d0 8.0d0) eight)
           (ignorable zero one two eight))
  (defun dlasd2
         (nl nr sqre k d z alpha beta u ldu vt ldvt dsigma u2 ldu2 vt2 ldvt2
          idxp idx idxc idxq coltyp info)
    (declare (type (array f2cl-lib:integer4 (*)) coltyp idxq idxc idx idxp)
             (type (double-float) beta alpha)
             (type (array double-float (*)) vt2 u2 dsigma vt u z d)
             (type (f2cl-lib:integer4) info ldvt2 ldu2 ldvt ldu k sqre nr nl))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (z double-float z-%data% z-%offset%)
         (u double-float u-%data% u-%offset%)
         (vt double-float vt-%data% vt-%offset%)
         (dsigma double-float dsigma-%data% dsigma-%offset%)
         (u2 double-float u2-%data% u2-%offset%)
         (vt2 double-float vt2-%data% vt2-%offset%)
         (idxp f2cl-lib:integer4 idxp-%data% idxp-%offset%)
         (idx f2cl-lib:integer4 idx-%data% idx-%offset%)
         (idxc f2cl-lib:integer4 idxc-%data% idxc-%offset%)
         (idxq f2cl-lib:integer4 idxq-%data% idxq-%offset%)
         (coltyp f2cl-lib:integer4 coltyp-%data% coltyp-%offset%))
      (prog ((c 0.0d0) (eps 0.0d0) (hlftol 0.0d0) (s 0.0d0) (tau 0.0d0)
             (tol 0.0d0) (z1 0.0d0) (ct 0) (i 0) (idxi 0) (idxj 0) (idxjp 0)
             (j 0) (jp 0) (jprev 0) (k2 0) (m 0) (n 0) (nlp1 0) (nlp2 0)
```

```
     (ctot (make-array 4 :element-type 'f2cl-lib:integer4))
     (psm (make-array 4 :element-type 'f2cl-lib:integer4)))
(declare (type (double-float) c eps hlftol s tau tol z1)
         (type (f2cl-lib:integer4) ct i idxi idxj idxjp j jp jprev k2 m
          n nlp1 nlp2)
         (type (array f2cl-lib:integer4 (4)) ctot psm))
(setf info 0)
(cond ((< nl 1) (setf info -1)) ((< nr 1) (setf info -2))
      ((and (/= sqre 1) (/= sqre 0)) (setf info -3)))
(setf n (f2cl-lib:int-add nl nr 1))
(setf m (f2cl-lib:int-add n sqre))
(cond ((< ldu n) (setf info -10)) ((< ldvt m) (setf info -12))
      ((< ldu2 n) (setf info -15)) ((< ldvt2 m) (setf info -17)))
(cond
 ((/= info 0) (xerbla "DLASD2" (f2cl-lib:int-sub info))
  (go end_label)))
(setf nlp1 (f2cl-lib:int-add nl 1))
(setf nlp2 (f2cl-lib:int-add nl 2))
(setf z1
        (* alpha
           (f2cl-lib:fref vt-%data% (nlp1 nlp1) ((1 ldvt) (1 *))
                          vt-%offset%)))
(setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%) z1)
(f2cl-lib:fdo (i nl (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         z-%offset%)
            (* alpha
               (f2cl-lib:fref vt-%data% (i nlp1) ((1 ldvt) (1 *))
                              vt-%offset%)))
    (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         d-%offset%)
            (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
    (setf (f2cl-lib:fref idxq-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         idxq-%offset%)
            (f2cl-lib:int-add
             (f2cl-lib:fref idxq-%data% (i) ((1 *)) idxq-%offset%) 1))
   label10))
(f2cl-lib:fdo (i nlp2 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
            (* beta
               (f2cl-lib:fref vt-%data% (i nlp2) ((1 ldvt) (1 *))
                              vt-%offset%)))
   label20))
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i nlp1) nil)
  (tagbody
```

```
    (setf (f2cl-lib:fref coltyp-%data% (i) ((1 *)) coltyp-%offset%) 1)
   label30))
(f2cl-lib:fdo (i nlp2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref coltyp-%data% (i) ((1 *)) coltyp-%offset%) 2)
   label40))
(f2cl-lib:fdo (i nlp2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref idxq-%data% (i) ((1 *)) idxq-%offset%)
            (f2cl-lib:int-add
             (f2cl-lib:fref idxq-%data% (i) ((1 *)) idxq-%offset%)
             nlp1))
   label50))
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref dsigma-%data% (i) ((1 *)) dsigma-%offset%)
            (f2cl-lib:fref d-%data% ((f2cl-lib:fref idxq (i) ((1 *))))
                           ((1 *)) d-%offset%))
    (setf (f2cl-lib:fref u2-%data% (i 1) ((1 ldu2) (1 *)) u2-%offset%)
            (f2cl-lib:fref z-%data% ((f2cl-lib:fref idxq (i) ((1 *))))
                           ((1 *)) z-%offset%))
    (setf (f2cl-lib:fref idxc-%data% (i) ((1 *)) idxc-%offset%)
            (f2cl-lib:fref coltyp-%data%
                           ((f2cl-lib:fref idxq (i) ((1 *)))) ((1 *))
                           coltyp-%offset%))
   label60))
(dlamrg nl nr
 (f2cl-lib:array-slice dsigma-%data% double-float (2) ((1 *))
                       dsigma-%offset%)
 1 1
 (f2cl-lib:array-slice idx-%data% f2cl-lib:integer4 (2) ((1 *))
                       idx-%offset%))
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf idxi
            (f2cl-lib:int-add 1
                              (f2cl-lib:fref idx-%data% (i) ((1 *))
                                             idx-%offset%)))
    (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
            (f2cl-lib:fref dsigma-%data% (idxi) ((1 *))
                           dsigma-%offset%))
    (setf (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
            (f2cl-lib:fref u2-%data% (idxi 1) ((1 ldu2) (1 *))
                           u2-%offset%))
    (setf (f2cl-lib:fref coltyp-%data% (i) ((1 *)) coltyp-%offset%)
            (f2cl-lib:fref idxc-%data% (idxi) ((1 *)) idxc-%offset%))
```

```
   label70))
 (setf eps (dlamch "Epsilon"))
 (setf tol (max (abs alpha) (abs beta)))
 (setf tol
         (* eight eps
            (max (abs (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%))
                 tol)))
 (setf k 1)
 (setf k2 (f2cl-lib:int-add n 1))
 (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (cond
      ((<= (abs (f2cl-lib:fref z (j) ((1 *)))) tol)
       (setf k2 (f2cl-lib:int-sub k2 1))
       (setf (f2cl-lib:fref idxp-%data% (k2) ((1 *)) idxp-%offset%) j)
       (setf (f2cl-lib:fref coltyp-%data% (j) ((1 *)) coltyp-%offset%)
               4)
       (if (= j n)
           (go label120)))
      (t (setf jprev j) (go label90)))
    label80))
label90
 (setf j jprev)
label100
 (setf j (f2cl-lib:int-add j 1))
 (if (> j n)
     (go label110))
 (cond
  ((<= (abs (f2cl-lib:fref z (j) ((1 *)))) tol)
   (setf k2 (f2cl-lib:int-sub k2 1))
   (setf (f2cl-lib:fref idxp-%data% (k2) ((1 *)) idxp-%offset%) j)
   (setf (f2cl-lib:fref coltyp-%data% (j) ((1 *)) coltyp-%offset%) 4))
  (t
   (cond
    ((<=
      (abs
       (+ (f2cl-lib:fref d (j) ((1 *)))
          (- (f2cl-lib:fref d (jprev) ((1 *))))))
      tol)
     (setf s (f2cl-lib:fref z-%data% (jprev) ((1 *)) z-%offset%))
     (setf c (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%))
     (setf tau (dlapy2 c s)) (setf c (/ c tau)) (setf s (/ (- s) tau))
     (setf (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%) tau)
     (setf (f2cl-lib:fref z-%data% (jprev) ((1 *)) z-%offset%) zero)
     (setf idxjp
             (f2cl-lib:fref idxq-%data%
                            ((f2cl-lib:int-add
                              (f2cl-lib:fref idx (jprev) ((1 *))) 1))
                            ((1 *)) idxq-%offset%))
```

```
(setf idxj
        (f2cl-lib:fref idxq-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:fref idx (j) ((1 *))) 1))
                       ((1 *)) idxq-%offset%))
(cond ((<= idxjp nlp1) (setf idxjp (f2cl-lib:int-sub idxjp 1))))
(cond ((<= idxj nlp1) (setf idxj (f2cl-lib:int-sub idxj 1))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (drot n
     (f2cl-lib:array-slice u-%data% double-float (1 idxjp)
                           ((1 ldu) (1 *)) u-%offset%)
     1
     (f2cl-lib:array-slice u-%data% double-float (1 idxj)
                           ((1 ldu) (1 *)) u-%offset%)
     1 c s)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (drot m
     (f2cl-lib:array-slice vt-%data% double-float (idxjp 1)
                           ((1 ldvt) (1 *)) vt-%offset%)
     ldvt
     (f2cl-lib:array-slice vt-%data% double-float (idxj 1)
                           ((1 ldvt) (1 *)) vt-%offset%)
     ldvt c s)
  (declare (ignore var-1 var-3))
  (when var-0 (setf m var-0))
  (when var-2 (setf ldvt var-2))
  (when var-4 (setf ldvt var-4))
  (when var-5 (setf c var-5))
  (when var-6 (setf s var-6)))
(cond
 ((/= (f2cl-lib:fref coltyp (j) ((1 *)))
      (f2cl-lib:fref coltyp (jprev) ((1 *))))
  (setf (f2cl-lib:fref coltyp-%data% (j) ((1 *)) coltyp-%offset%)
        3)))
(setf (f2cl-lib:fref coltyp-%data% (jprev) ((1 *)) coltyp-%offset%)
      4)
(setf k2 (f2cl-lib:int-sub k2 1))
(setf (f2cl-lib:fref idxp-%data% (k2) ((1 *)) idxp-%offset%) jprev)
(setf jprev j))
(t (setf k (f2cl-lib:int-add k 1))
(setf (f2cl-lib:fref u2-%data% (k 1) ((1 ldu2) (1 *)) u2-%offset%)
      (f2cl-lib:fref z-%data% (jprev) ((1 *)) z-%offset%))
(setf (f2cl-lib:fref dsigma-%data% (k) ((1 *)) dsigma-%offset%)
      (f2cl-lib:fref d-%data% (jprev) ((1 *)) d-%offset%))
(setf (f2cl-lib:fref idxp-%data% (k) ((1 *)) idxp-%offset%) jprev)
(setf jprev j)))))
```

```
 (go label100)
label110
 (setf k (f2cl-lib:int-add k 1))
 (setf (f2cl-lib:fref u2-%data% (k 1) ((1 ldu2) (1 *)) u2-%offset%)
        (f2cl-lib:fref z-%data% (jprev) ((1 *)) z-%offset%))
 (setf (f2cl-lib:fref dsigma-%data% (k) ((1 *)) dsigma-%offset%)
        (f2cl-lib:fref d-%data% (jprev) ((1 *)) d-%offset%))
 (setf (f2cl-lib:fref idxp-%data% (k) ((1 *)) idxp-%offset%) jprev)
label120
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j 4) nil)
   (tagbody (setf (f2cl-lib:fref ctot (j) ((1 4))) 0) label130))
 (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf ct (f2cl-lib:fref coltyp-%data% (j) ((1 *)) coltyp-%offset%))
     (setf (f2cl-lib:fref ctot (ct) ((1 4)))
             (f2cl-lib:int-add (f2cl-lib:fref ctot (ct) ((1 4))) 1))
    label140))
 (setf (f2cl-lib:fref psm (1) ((1 4))) 2)
 (setf (f2cl-lib:fref psm (2) ((1 4)))
        (f2cl-lib:int-add 2 (f2cl-lib:fref ctot (1) ((1 4)))))
 (setf (f2cl-lib:fref psm (3) ((1 4)))
        (f2cl-lib:int-add (f2cl-lib:fref psm (2) ((1 4)))
                          (f2cl-lib:fref ctot (2) ((1 4)))))
 (setf (f2cl-lib:fref psm (4) ((1 4)))
        (f2cl-lib:int-add (f2cl-lib:fref psm (3) ((1 4)))
                          (f2cl-lib:fref ctot (3) ((1 4)))))
 (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf jp (f2cl-lib:fref idxp-%data% (j) ((1 *)) idxp-%offset%))
     (setf ct
             (f2cl-lib:fref coltyp-%data% (jp) ((1 *)) coltyp-%offset%))
     (setf (f2cl-lib:fref idxc-%data% ((f2cl-lib:fref psm (ct) ((1 4))))
                         ((1 *)) idxc-%offset%)
             j)
     (setf (f2cl-lib:fref psm (ct) ((1 4)))
             (f2cl-lib:int-add (f2cl-lib:fref psm (ct) ((1 4))) 1))
    label150))
 (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf jp (f2cl-lib:fref idxp-%data% (j) ((1 *)) idxp-%offset%))
     (setf (f2cl-lib:fref dsigma-%data% (j) ((1 *)) dsigma-%offset%)
             (f2cl-lib:fref d-%data% (jp) ((1 *)) d-%offset%))
     (setf idxj
             (f2cl-lib:fref idxq-%data%
                            ((f2cl-lib:int-add
                              (f2cl-lib:fref idx
```

```
                                                  ((f2cl-lib:fref idxp
                                                                  ((f2cl-lib:fref
                                                                    idxc (j)
                                                                    ((1 *))))
                                                                  ((1 *))))
                                                  ((1 *)))
                                 1))
                               ((1 *)) idxq-%offset%))
      (cond ((<= idxj nlp1) (setf idxj (f2cl-lib:int-sub idxj 1))))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy n
           (f2cl-lib:array-slice u-%data% double-float (1 idxj)
                                 ((1 ldu) (1 *)) u-%offset%)
           1
           (f2cl-lib:array-slice u2-%data% double-float (1 j)
                                 ((1 ldu2) (1 *)) u2-%offset%)
           1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf n var-0)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dcopy m
           (f2cl-lib:array-slice vt-%data% double-float (idxj 1)
                                 ((1 ldvt) (1 *)) vt-%offset%)
           ldvt
           (f2cl-lib:array-slice vt2-%data% double-float (j 1)
                                 ((1 ldvt2) (1 *)) vt2-%offset%)
           ldvt2)
        (declare (ignore var-1 var-3))
        (when var-0 (setf m var-0))
        (when var-2 (setf ldvt var-2))
        (when var-4 (setf ldvt2 var-4)))
    label160))
  (setf (f2cl-lib:fref dsigma-%data% (1) ((1 *)) dsigma-%offset%) zero)
  (setf hlftol (/ tol two))
  (if (<= (abs (f2cl-lib:fref dsigma-%data% (2) ((1 *)) dsigma-%offset%))
          hlftol)
      (setf (f2cl-lib:fref dsigma-%data% (2) ((1 *)) dsigma-%offset%)
            hlftol))
  (cond
   ((> m n)
    (setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)
          (dlapy2 z1 (f2cl-lib:fref z-%data% (m) ((1 *)) z-%offset%)))
    (cond
     ((<= (f2cl-lib:fref z (1) ((1 *))) tol) (setf c one) (setf s zero)
      (setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%) tol))
     (t (setf c (/ z1 (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)))
      (setf s
            (/ (f2cl-lib:fref z-%data% (m) ((1 *)) z-%offset%)
               (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%))))))
   (t
```

```
  (cond
   ((<= (abs z1) tol)
    (setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%) tol))
   (t (setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%) z1)))))
(dcopy (f2cl-lib:int-sub k 1)
 (f2cl-lib:array-slice u2-%data% double-float (2 1) ((1 ldu2) (1 *))
                       u2-%offset%)
 1 (f2cl-lib:array-slice z-%data% double-float (2) ((1 *)) z-%offset%)
 1)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "A" n 1 zero zero u2 ldu2)
  (declare (ignore var-0 var-2 var-5))
  (when var-1 (setf n var-1))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldu2 var-6)))
(setf (f2cl-lib:fref u2-%data% (nlp1 1) ((1 ldu2) (1 *)) u2-%offset%)
        one)
(cond
 ((> m n)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i nlp1) nil)
    (tagbody
      (setf (f2cl-lib:fref vt-%data% (m i) ((1 ldvt) (1 *))
                           vt-%offset%)
              (* (- s)
                 (f2cl-lib:fref vt-%data% (nlp1 i) ((1 ldvt) (1 *))
                                vt-%offset%)))
      (setf (f2cl-lib:fref vt2-%data% (1 i) ((1 ldvt2) (1 *))
                           vt2-%offset%)
              (* c
                 (f2cl-lib:fref vt-%data% (nlp1 i) ((1 ldvt) (1 *))
                                vt-%offset%)))
     label170))
  (f2cl-lib:fdo (i nlp2 (f2cl-lib:int-add i 1))
                ((> i m) nil)
    (tagbody
      (setf (f2cl-lib:fref vt2-%data% (1 i) ((1 ldvt2) (1 *))
                           vt2-%offset%)
              (* s
                 (f2cl-lib:fref vt-%data% (m i) ((1 ldvt) (1 *))
                                vt-%offset%)))
      (setf (f2cl-lib:fref vt-%data% (m i) ((1 ldvt) (1 *))
                           vt-%offset%)
              (* c
                 (f2cl-lib:fref vt-%data% (m i) ((1 ldvt) (1 *))
                                vt-%offset%)))
     label180)))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
      (dcopy m
       (f2cl-lib:array-slice vt-%data% double-float (nlp1 1)
                             ((1 ldvt) (1 *)) vt-%offset%)
       ldvt
       (f2cl-lib:array-slice vt2-%data% double-float (1 1)
                             ((1 ldvt2) (1 *)) vt2-%offset%)
       ldvt2)
    (declare (ignore var-1 var-3))
    (when var-0 (setf m var-0))
    (when var-2 (setf ldvt var-2))
    (when var-4 (setf ldvt2 var-4)))))
(cond
 ((> m n)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy m
       (f2cl-lib:array-slice vt-%data% double-float (m 1)
                             ((1 ldvt) (1 *)) vt-%offset%)
       ldvt
       (f2cl-lib:array-slice vt2-%data% double-float (m 1)
                             ((1 ldvt2) (1 *)) vt2-%offset%)
       ldvt2)
    (declare (ignore var-1 var-3))
    (when var-0 (setf m var-0))
    (when var-2 (setf ldvt var-2))
    (when var-4 (setf ldvt2 var-4)))))
(cond
 ((> n k)
  (dcopy (f2cl-lib:int-sub n k)
   (f2cl-lib:array-slice dsigma-%data% double-float ((+ k 1)) ((1 *))
                         dsigma-%offset%)
   1
   (f2cl-lib:array-slice d-%data% double-float ((+ k 1)) ((1 *))
                         d-%offset%)
   1)
  (dlacpy "A" n (f2cl-lib:int-sub n k)
   (f2cl-lib:array-slice u2-%data% double-float
                         (1 (f2cl-lib:int-add k 1)) ((1 ldu2) (1 *))
                         u2-%offset%)
   ldu2
   (f2cl-lib:array-slice u-%data% double-float
                         (1 (f2cl-lib:int-add k 1)) ((1 ldu) (1 *))
                         u-%offset%)
   ldu)
  (dlacpy "A" (f2cl-lib:int-sub n k) m
   (f2cl-lib:array-slice vt2-%data% double-float ((+ k 1) 1)
                         ((1 ldvt2) (1 *)) vt2-%offset%)
   ldvt2
   (f2cl-lib:array-slice vt-%data% double-float ((+ k 1) 1)
                         ((1 ldvt) (1 *)) vt-%offset%)
   ldvt)))
```

```
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j 4) nil)
          (tagbody
            (setf (f2cl-lib:fref coltyp-%data% (j) ((1 *)) coltyp-%offset%)
                    (f2cl-lib:fref ctot (j) ((1 4))))
           label190))
        (go end_label)
       end_label
        (return
         (values nil nil nil k nil nil nil nil nil nil nil ldvt nil nil ldu2
                 nil ldvt2 nil nil nil nil nil info))))))
```

———-

90.172 dlasd3

90.172.1 Axiom unit tests

— **dlasd3.input** —

```
)set break resume
)sys rm -f dlasd3.output
)spool dlasd3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.172.2 Axiom help page

— **dlasd3.help** —

```
==========================================================================
dlasd3 examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
NAME
       DLASD3  - all the square roots of the roots of the secular equation, as
       defined by the values in D and Z

SYNOPSIS
       SUBROUTINE DLASD3( NL, NR, SQRE, K, D, Q,  LDQ,  DSIGMA,  U,  LDU,  U2,
                          LDU2, VT, LDVT, VT2, LDVT2, IDXC, CTOT, Z, INFO )

           INTEGER        INFO, K, LDQ, LDU, LDU2, LDVT, LDVT2, NL, NR, SQRE

           INTEGER        CTOT( * ), IDXC( * )

           DOUBLE         PRECISION  D( * ), DSIGMA( * ), Q( LDQ, * ), U( LDU,
                          * ), U2( LDU2, * ), VT( LDVT, * ), VT2( LDVT2, *  ),
                          Z( * )

  Purpose
  =======

  DLASD3 finds all the square roots of the roots of the secular
  equation, as defined by the values in D and Z.  It makes the
  appropriate calls to DLASD4 and then updates the singular
  vectors by matrix multiplication.

  This code makes very mild assumptions about floating point
  arithmetic. It will work on machines with a guard digit in
  add/subtract, or on those binary machines without guard digits
  which subtract like the Cray XMP, Cray YMP, Cray C 90, or Cray 2.
  It could conceivably fail on hexadecimal or decimal machines
  without guard digits, but we know of none.

  DLASD3 is called from DLASD1.

  Arguments
  =========

  NL     (input) INTEGER
         The row dimension of the upper block.  NL >= 1.

  NR     (input) INTEGER
         The row dimension of the lower block.  NR >= 1.

  SQRE   (input) INTEGER
         = 0: the lower block is an NR-by-NR square matrix.
         = 1: the lower block is an NR-by-(NR+1) rectangular matrix.

         The bidiagonal matrix has N = NL + NR + 1 rows and
         M = N + SQRE >= N columns.
```

```
K      (input) INTEGER
       The size of the secular equation, 1 =< K = < N.

D      (output) DOUBLE PRECISION array, dimension(K)
       On exit the square roots of the roots of the secular equation,
       in ascending order.

Q      (workspace) DOUBLE PRECISION array,
                   dimension at least (LDQ,K).

LDQ    (input) INTEGER
       The leading dimension of the array Q.  LDQ >= K.

DSIGMA (input) DOUBLE PRECISION array, dimension(K)
       The first K elements of this array contain the old roots
       of the deflated updating problem.  These are the poles
       of the secular equation.

U      (input) DOUBLE PRECISION array, dimension (LDU, N)
       The last N - K columns of this matrix contain the deflated
       left singular vectors.

LDU    (input) INTEGER
       The leading dimension of the array U.  LDU >= N.

U2     (input) DOUBLE PRECISION array, dimension (LDU2, N)
       The first K columns of this matrix contain the non-deflated
       left singular vectors for the split problem.

LDU2   (input) INTEGER
       The leading dimension of the array U2.  LDU2 >= N.

VT     (input) DOUBLE PRECISION array, dimension (LDVT, M)
       The last M - K columns of VT' contain the deflated
       right singular vectors.

LDVT   (input) INTEGER
       The leading dimension of the array VT.  LDVT >= N.

VT2    (input) DOUBLE PRECISION array, dimension (LDVT2, N)
       The first K columns of VT2' contain the non-deflated
       right singular vectors for the split problem.

LDVT2  (input) INTEGER
       The leading dimension of the array VT2.  LDVT2 >= N.

IDXC   (input) INTEGER array, dimension ( N )
       The permutation used to arrange the columns of U (and rows of
       VT) into three groups:  the first group contains non-zero
       entries only at and above (or before) NL +1; the second
```

```
        contains non-zero entries only at and below (or after) NL+2;
        and the third is dense. The first column of U and the row of
        VT are treated separately, however.

        The rows of the singular vectors found by DLASD4
        must be likewise permuted before the matrix multiplies can
        take place.

  CTOT   (input) INTEGER array, dimension ( 4 )
        A count of the total number of the various types of columns
        in U (or rows in VT), as described in IDXC. The fourth column
        type is any column which has been deflated.

  Z      (input) DOUBLE PRECISION array, dimension (K)
        The first K elements of this array contain the components
        of the deflation-adjusted updating row vector.

  INFO   (output) INTEGER
        = 0:  successful exit.
        < 0:  if INFO = -i, the i-th argument had an illegal value.
        > 0:  if INFO = 1, an singular value did not converge

  Further Details
  ===============

  Based on contributions by
     Ming Gu and Huan Ren, Computer Science Division, University of
     California at Berkeley, USA
```

90.172.3 fortran code

— dlasd3.f —

```
      SUBROUTINE DLASD3( NL, NR, SQRE, K, D, Q, LDQ, DSIGMA, U, LDU, U2,
     $                   LDU2, VT, LDVT, VT2, LDVT2, IDXC, CTOT, Z,
     $                   INFO )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Oak Ridge National Lab, Argonne National Lab,
*     Courant Institute, NAG Ltd., and Rice University
*     October 31, 1999
*
*     .. Scalar Arguments ..
      INTEGER            INFO, K, LDQ, LDU, LDU2, LDVT, LDVT2, NL, NR,
     $                   SQRE
```

```
*     ..
*     .. Array Arguments ..
      INTEGER            CTOT( * ), IDXC( * )
      DOUBLE PRECISION   D( * ), DSIGMA( * ), Q( LDQ, * ), U( LDU, * ),
     $                   U2( LDU2, * ), VT( LDVT, * ), VT2( LDVT2, * ),
     $                   Z( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO, NEGONE
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0,
     $                   NEGONE = -1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            CTEMP, I, J, JC, KTEMP, M, N, NLP1, NLP2, NRP1
      DOUBLE PRECISION   RHO, TEMP
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMC3, DNRM2
      EXTERNAL           DLAMC3, DNRM2
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DGEMM, DLACPY, DLASCL, DLASD4, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SIGN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( NL.LT.1 ) THEN
         INFO = -1
      ELSE IF( NR.LT.1 ) THEN
         INFO = -2
      ELSE IF( ( SQRE.NE.1 ) .AND. ( SQRE.NE.0 ) ) THEN
         INFO = -3
      END IF
*
      N = NL + NR + 1
      M = N + SQRE
      NLP1 = NL + 1
      NLP2 = NL + 2
*
      IF( ( K.LT.1 ) .OR. ( K.GT.N ) ) THEN
         INFO = -4
```

```
      ELSE IF( LDQ.LT.K ) THEN
         INFO = -7
      ELSE IF( LDU.LT.N ) THEN
         INFO = -10
      ELSE IF( LDU2.LT.N ) THEN
         INFO = -12
      ELSE IF( LDVT.LT.M ) THEN
         INFO = -14
      ELSE IF( LDVT2.LT.M ) THEN
         INFO = -16
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASD3', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( K.EQ.1 ) THEN
         D( 1 ) = ABS( Z( 1 ) )
         CALL DCOPY( M, VT2( 1, 1 ), LDVT2, VT( 1, 1 ), LDVT )
         IF( Z( 1 ).GT.ZERO ) THEN
            CALL DCOPY( N, U2( 1, 1 ), 1, U( 1, 1 ), 1 )
         ELSE
            DO 10 I = 1, N
               U( I, 1 ) = -U2( I, 1 )
   10       CONTINUE
         END IF
         RETURN
      END IF
*
*     Modify values DSIGMA(i) to make sure all DSIGMA(i)-DSIGMA(j) can
*     be computed with high relative accuracy (barring over/underflow).
*     This is a problem on machines without a guard digit in
*     add/subtract (Cray XMP, Cray YMP, Cray C 90 and Cray 2).
*     The following code replaces DSIGMA(I) by 2*DSIGMA(I)-DSIGMA(I),
*     which on any of these machines zeros out the bottommost
*     bit of DSIGMA(I) if it is 1; this makes the subsequent
*     subtractions DSIGMA(I)-DSIGMA(J) unproblematic when cancellation
*     occurs. On binary machines with a guard digit (almost all
*     machines) it does not change DSIGMA(I) at all. On hexadecimal
*     and decimal machines with a guard digit, it slightly
*     changes the bottommost bits of DSIGMA(I). It does not account
*     for hexadecimal or decimal machines without guard digits
*     (we know of none). We use a subroutine call to compute
*     2*DLAMBDA(I) to prevent optimizing compilers from eliminating
*     this code.
*
      DO 20 I = 1, K
         DSIGMA( I ) = DLAMC3( DSIGMA( I ), DSIGMA( I ) ) - DSIGMA( I )
```

```
   20 CONTINUE
*
*     Keep a copy of Z.
*
      CALL DCOPY( K, Z, 1, Q, 1 )
*
*     Normalize Z.
*
      RHO = DNRM2( K, Z, 1 )
      CALL DLASCL( 'G', 0, 0, RHO, ONE, K, 1, Z, K, INFO )
      RHO = RHO*RHO
*
*     Find the new singular values.
*
      DO 30 J = 1, K
         CALL DLASD4( K, J, DSIGMA, Z, U( 1, J ), RHO, D( J ),
     $                VT( 1, J ), INFO )
*
*        If the zero finder fails, the computation is terminated.
*
         IF( INFO.NE.0 ) THEN
            RETURN
         END IF
   30 CONTINUE
*
*     Compute updated Z.
*
      DO 60 I = 1, K
         Z( I ) = U( I, K )*VT( I, K )
         DO 40 J = 1, I - 1
            Z( I ) = Z( I )*( U( I, J )*VT( I, J ) /
     $               ( DSIGMA( I )-DSIGMA( J ) ) /
     $               ( DSIGMA( I )+DSIGMA( J ) ) )
   40    CONTINUE
         DO 50 J = I, K - 1
            Z( I ) = Z( I )*( U( I, J )*VT( I, J ) /
     $               ( DSIGMA( I )-DSIGMA( J+1 ) ) /
     $               ( DSIGMA( I )+DSIGMA( J+1 ) ) )
   50    CONTINUE
         Z( I ) = SIGN( SQRT( ABS( Z( I ) ) ), Q( I, 1 ) )
   60 CONTINUE
*
*     Compute left singular vectors of the modified diagonal matrix,
*     and store related information for the right singular vectors.
*
      DO 90 I = 1, K
         VT( 1, I ) = Z( 1 ) / U( 1, I ) / VT( 1, I )
         U( 1, I ) = NEGONE
         DO 70 J = 2, K
            VT( J, I ) = Z( J ) / U( J, I ) / VT( J, I )
```

```
            U( J, I ) = DSIGMA( J )*VT( J, I )
   70    CONTINUE
         TEMP = DNRM2( K, U( 1, I ), 1 )
         Q( 1, I ) = U( 1, I ) / TEMP
         DO 80 J = 2, K
            JC = IDXC( J )
            Q( J, I ) = U( JC, I ) / TEMP
   80    CONTINUE
   90 CONTINUE
*
*     Update the left singular vector matrix.
*
      IF( K.EQ.2 ) THEN
         CALL DGEMM( 'N', 'N', N, K, K, ONE, U2, LDU2, Q, LDQ, ZERO, U,
     $               LDU )
         GO TO 100
      END IF
      IF( CTOT( 1 ).GT.0 ) THEN
         CALL DGEMM( 'N', 'N', NL, K, CTOT( 1 ), ONE, U2( 1, 2 ), LDU2,
     $               Q( 2, 1 ), LDQ, ZERO, U( 1, 1 ), LDU )
         IF( CTOT( 3 ).GT.0 ) THEN
            KTEMP = 2 + CTOT( 1 ) + CTOT( 2 )
            CALL DGEMM( 'N', 'N', NL, K, CTOT( 3 ), ONE, U2( 1, KTEMP ),
     $                  LDU2, Q( KTEMP, 1 ), LDQ, ONE, U( 1, 1 ), LDU )
         END IF
      ELSE IF( CTOT( 3 ).GT.0 ) THEN
         KTEMP = 2 + CTOT( 1 ) + CTOT( 2 )
         CALL DGEMM( 'N', 'N', NL, K, CTOT( 3 ), ONE, U2( 1, KTEMP ),
     $               LDU2, Q( KTEMP, 1 ), LDQ, ZERO, U( 1, 1 ), LDU )
      ELSE
         CALL DLACPY( 'F', NL, K, U2, LDU2, U, LDU )
      END IF
      CALL DCOPY( K, Q( 1, 1 ), LDQ, U( NLP1, 1 ), LDU )
      KTEMP = 2 + CTOT( 1 )
      CTEMP = CTOT( 2 ) + CTOT( 3 )
      CALL DGEMM( 'N', 'N', NR, K, CTEMP, ONE, U2( NLP2, KTEMP ), LDU2,
     $            Q( KTEMP, 1 ), LDQ, ZERO, U( NLP2, 1 ), LDU )
*
*     Generate the right singular vectors.
*
  100 CONTINUE
      DO 120 I = 1, K
         TEMP = DNRM2( K, VT( 1, I ), 1 )
         Q( I, 1 ) = VT( 1, I ) / TEMP
         DO 110 J = 2, K
            JC = IDXC( J )
            Q( I, J ) = VT( JC, I ) / TEMP
  110    CONTINUE
  120 CONTINUE
*
```

```
*     Update the right singular vector matrix.
*
      IF( K.EQ.2 ) THEN
         CALL DGEMM( 'N', 'N', K, M, K, ONE, Q, LDQ, VT2, LDVT2, ZERO,
     $               VT, LDVT )
         RETURN
      END IF
      KTEMP = 1 + CTOT( 1 )
      CALL DGEMM( 'N', 'N', K, NLP1, KTEMP, ONE, Q( 1, 1 ), LDQ,
     $            VT2( 1, 1 ), LDVT2, ZERO, VT( 1, 1 ), LDVT )
      KTEMP = 2 + CTOT( 1 ) + CTOT( 2 )
      IF( KTEMP.LE.LDVT2 )
     $   CALL DGEMM( 'N', 'N', K, NLP1, CTOT( 3 ), ONE, Q( 1, KTEMP ),
     $               LDQ, VT2( KTEMP, 1 ), LDVT2, ONE, VT( 1, 1 ),
     $               LDVT )
*
      KTEMP = CTOT( 1 ) + 1
      NRP1 = NR + SQRE
      IF( KTEMP.GT.1 ) THEN
         DO 130 I = 1, K
            Q( I, KTEMP ) = Q( I, 1 )
  130    CONTINUE
         DO 140 I = NLP2, M
            VT2( KTEMP, I ) = VT2( 1, I )
  140    CONTINUE
      END IF
      CTEMP = 1 + CTOT( 2 ) + CTOT( 3 )
      CALL DGEMM( 'N', 'N', K, NRP1, CTEMP, ONE, Q( 1, KTEMP ), LDQ,
     $            VT2( KTEMP, NLP2 ), LDVT2, ZERO, VT( 1, NLP2 ), LDVT )
*
      RETURN
*
*     End of DLASD3
*
      END
```

———-

90.172.4 lisp code

— dlasd3.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0) (negone (- 1.0d0)))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float) negone)
           (ignorable one zero negone))
```

```
(defun dlasd3
       (nl nr sqre k d q ldq dsigma u ldu u2 ldu2 vt ldvt vt2 ldvt2 idxc ctot
        z info)
  (declare (type (array f2cl-lib:integer4 (*)) ctot idxc)
           (type (array double-float (*)) z vt2 vt u2 u dsigma q d)
           (type (f2cl-lib:integer4) info ldvt2 ldvt ldu2 ldu ldq k sqre nr
            nl))
  (f2cl-lib:with-multi-array-data
      ((d double-float d-%data% d-%offset%)
       (q double-float q-%data% q-%offset%)
       (dsigma double-float dsigma-%data% dsigma-%offset%)
       (u double-float u-%data% u-%offset%)
       (u2 double-float u2-%data% u2-%offset%)
       (vt double-float vt-%data% vt-%offset%)
       (vt2 double-float vt2-%data% vt2-%offset%)
       (z double-float z-%data% z-%offset%)
       (idxc f2cl-lib:integer4 idxc-%data% idxc-%offset%)
       (ctot f2cl-lib:integer4 ctot-%data% ctot-%offset%))
    (prog ((rho 0.0d0) (temp 0.0d0) (ctemp 0) (i 0) (j 0) (jc 0) (ktemp 0)
           (m 0) (n 0) (nlp1 0) (nlp2 0) (nrp1 0))
      (declare (type (double-float) rho temp)
               (type (f2cl-lib:integer4) ctemp i j jc ktemp m n nlp1 nlp2
                nrp1))
      (setf info 0)
      (cond ((< nl 1) (setf info -1)) ((< nr 1) (setf info -2))
            ((and (/= sqre 1) (/= sqre 0)) (setf info -3)))
      (setf n (f2cl-lib:int-add nl nr 1))
      (setf m (f2cl-lib:int-add n sqre))
      (setf nlp1 (f2cl-lib:int-add nl 1))
      (setf nlp2 (f2cl-lib:int-add nl 2))
      (cond ((or (< k 1) (> k n)) (setf info -4)) ((< ldq k) (setf info -7))
            ((< ldu n) (setf info -10)) ((< ldu2 n) (setf info -12))
            ((< ldvt m) (setf info -14)) ((< ldvt2 m) (setf info -16)))
      (cond
       ((/= info 0) (xerbla "DLASD3" (f2cl-lib:int-sub info))
        (go end_label)))
      (cond
       ((= k 1)
        (setf (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
                (abs (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dcopy m
             (f2cl-lib:array-slice vt2-%data% double-float (1 1)
                                   ((1 ldvt2) (1 *)) vt2-%offset%)
             ldvt2
             (f2cl-lib:array-slice vt-%data% double-float (1 1)
                                   ((1 ldvt) (1 *)) vt-%offset%)
             ldvt)
          (declare (ignore var-1 var-3))
          (when var-0 (setf m var-0))
```

```
    (when var-2 (setf ldvt2 var-2))
    (when var-4 (setf ldvt var-4)))
  (cond
   ((> (f2cl-lib:fref z (1) ((1 *))) zero)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy n
         (f2cl-lib:array-slice u2-%data% double-float (1 1)
                               ((1 ldu2) (1 *)) u2-%offset%)
         1
         (f2cl-lib:array-slice u-%data% double-float (1 1)
                               ((1 ldu) (1 *)) u-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf n var-0))))
   (t
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref u-%data% (i 1) ((1 ldu) (1 *)) u-%offset%)
                (-
                 (f2cl-lib:fref u2-%data% (i 1) ((1 ldu2) (1 *))
                                u2-%offset%)))
       label10))))
  (go end_label)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i k) nil)
  (tagbody
    (setf (f2cl-lib:fref dsigma-%data% (i) ((1 *)) dsigma-%offset%)
            (-
             (dlamc3
              (f2cl-lib:array-slice dsigma-%data% double-float (i)
                                    ((1 *)) dsigma-%offset%)
              (f2cl-lib:array-slice dsigma-%data% double-float (i)
                                    ((1 *)) dsigma-%offset%))
             (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                            dsigma-%offset%)))
   label20))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dcopy k z 1 q 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf k var-0)))
(setf rho
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (dnrm2 k z 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf k var-0))
          ret-val))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dlascl "G" 0 0 rho one k 1 z k info)
```

```
  (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
  (setf rho var-3)
  (setf one var-4)
  (setf info var-9))
(setf rho (* rho rho))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j k) nil)
  (tagbody
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dlasd4 k j dsigma z
         (f2cl-lib:array-slice u-%data% double-float (1 j)
                               ((1 ldu) (1 *)) u-%offset%)
         rho
         (f2cl-lib:array-slice d-%data% double-float (j) ((1 *))
                               d-%offset%)
         (f2cl-lib:array-slice vt-%data% double-float (1 j)
                               ((1 ldvt) (1 *)) vt-%offset%)
         info)
      (declare (ignore var-2 var-3 var-4 var-6 var-7))
      (when var-0 (setf k var-0))
      (when var-1 (setf j var-1))
      (when var-5 (setf rho var-5))
      (when var-8 (setf info var-8)))
    (cond ((/= info 0) (go end_label)))
   label30))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i k) nil)
  (tagbody
    (setf (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
            (*
             (f2cl-lib:fref u-%data% (i k) ((1 ldu) (1 *)) u-%offset%)
             (f2cl-lib:fref vt-%data% (i k) ((1 ldvt) (1 *))
                            vt-%offset%)))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j (f2cl-lib:int-add i (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (setf (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                (* (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                   (/
                    (/
                     (*
                      (f2cl-lib:fref u-%data% (i j) ((1 ldu) (1 *))
                                     u-%offset%)
                      (f2cl-lib:fref vt-%data% (i j) ((1 ldvt) (1 *))
                                     vt-%offset%))
                     (-
                      (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                                     dsigma-%offset%)
                      (f2cl-lib:fref dsigma-%data% (j) ((1 *))
```

```
                                        dsigma-%offset%)))
                          (+
                           (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                                          dsigma-%offset%)
                           (f2cl-lib:fref dsigma-%data% (j) ((1 *))
                                          dsigma-%offset%)))))
            label40))
        (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                      ((> j (f2cl-lib:int-add k (f2cl-lib:int-sub 1))) nil)
          (tagbody
            (setf (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                  (* (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                     (/
                      (/
                       (*
                        (f2cl-lib:fref u-%data% (i j) ((1 ldu) (1 *))
                                       u-%offset%)
                        (f2cl-lib:fref vt-%data% (i j) ((1 ldvt) (1 *))
                                       vt-%offset%))
                       (-
                        (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                                       dsigma-%offset%)
                        (f2cl-lib:fref dsigma-%data%
                                       ((f2cl-lib:int-add j 1)) ((1 *))
                                       dsigma-%offset%)))
                      (+
                       (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                                      dsigma-%offset%)
                       (f2cl-lib:fref dsigma-%data%
                                      ((f2cl-lib:int-add j 1)) ((1 *))
                                      dsigma-%offset%)))))
            label50))
        (setf (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
              (f2cl-lib:sign
               (f2cl-lib:fsqrt
                (abs (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)))
               (f2cl-lib:fref q-%data% (i 1) ((1 ldq) (1 *))
                              q-%offset%)))
      label60))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i k) nil)
      (tagbody
        (setf (f2cl-lib:fref vt-%data% (1 i) ((1 ldvt) (1 *)) vt-%offset%)
              (/
               (/ (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)
                  (f2cl-lib:fref u-%data% (1 i) ((1 ldu) (1 *))
                                 u-%offset%))
               (f2cl-lib:fref vt-%data% (1 i) ((1 ldvt) (1 *))
                              vt-%offset%)))
        (setf (f2cl-lib:fref u-%data% (1 i) ((1 ldu) (1 *)) u-%offset%)
```

```
                negone)
        (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                      ((> j k) nil)
          (tagbody
            (setf (f2cl-lib:fref vt-%data% (j i) ((1 ldvt) (1 *))
                                 vt-%offset%)
                  (/
                   (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                      (f2cl-lib:fref u-%data% (j i) ((1 ldu) (1 *))
                                     u-%offset%))
                   (f2cl-lib:fref vt-%data% (j i) ((1 ldvt) (1 *))
                                  vt-%offset%)))
            (setf (f2cl-lib:fref u-%data% (j i) ((1 ldu) (1 *)) u-%offset%)
                  (*
                   (f2cl-lib:fref dsigma-%data% (j) ((1 *))
                                  dsigma-%offset%)
                   (f2cl-lib:fref vt-%data% (j i) ((1 ldvt) (1 *))
                                  vt-%offset%)))
           label70))
        (setf temp
                (multiple-value-bind (ret-val var-0 var-1 var-2)
                    (dnrm2 k
                     (f2cl-lib:array-slice u-%data% double-float (1 i)
                                           ((1 ldu) (1 *)) u-%offset%)
                     1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf k var-0))
                  ret-val))
        (setf (f2cl-lib:fref q-%data% (1 i) ((1 ldq) (1 *)) q-%offset%)
                (/
                 (f2cl-lib:fref u-%data% (1 i) ((1 ldu) (1 *)) u-%offset%)
                 temp))
        (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                      ((> j k) nil)
          (tagbody
            (setf jc (f2cl-lib:fref idxc-%data% (j) ((1 *)) idxc-%offset%))
            (setf (f2cl-lib:fref q-%data% (j i) ((1 ldq) (1 *)) q-%offset%)
                  (/
                   (f2cl-lib:fref u-%data% (jc i) ((1 ldu) (1 *))
                                  u-%offset%)
                   temp))
           label80))
       label90))
    (cond
     ((= k 2)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (dgemm "N" "N" n k k one u2 ldu2 q ldq zero u ldu)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
```

```
    (when var-2 (setf n var-2))
    (when var-3 (setf k var-3))
    (when var-4 (setf k var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldu2 var-7))
    (when var-9 (setf ldq var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldu var-12)))
  (go label100)))
(cond
 ((> (f2cl-lib:fref ctot (1) ((1 *))) 0)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dgemm "N" "N" nl k
       (f2cl-lib:array-slice ctot-%data% f2cl-lib:integer4 (1) ((1 *))
                             ctot-%offset%)
       one
       (f2cl-lib:array-slice u2-%data% double-float (1 2)
                             ((1 ldu2) (1 *)) u2-%offset%)
       ldu2
       (f2cl-lib:array-slice q-%data% double-float (2 1)
                             ((1 ldq) (1 *)) q-%offset%)
       ldq zero
       (f2cl-lib:array-slice u-%data% double-float (1 1)
                             ((1 ldu) (1 *)) u-%offset%)
       ldu)
    (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
    (when var-2 (setf nl var-2))
    (when var-3 (setf k var-3))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldu2 var-7))
    (when var-9 (setf ldq var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldu var-12)))
  (cond
   ((> (f2cl-lib:fref ctot (3) ((1 *))) 0)
    (setf ktemp
            (f2cl-lib:int-add 2
                              (f2cl-lib:fref ctot-%data% (1) ((1 *))
                                             ctot-%offset%)
                              (f2cl-lib:fref ctot-%data% (2) ((1 *))
                                             ctot-%offset%)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (dgemm "N" "N" nl k
         (f2cl-lib:array-slice ctot-%data% f2cl-lib:integer4 (3)
                               ((1 *)) ctot-%offset%)
         one
```

```
            (f2cl-lib:array-slice u2-%data% double-float (1 ktemp)
                                  ((1 ldu2) (1 *)) u2-%offset%)
            ldu2
            (f2cl-lib:array-slice q-%data% double-float (ktemp 1)
                                  ((1 ldq) (1 *)) q-%offset%)
            ldq one
            (f2cl-lib:array-slice u-%data% double-float (1 1)
                                  ((1 ldu) (1 *)) u-%offset%)
            ldu)
        (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
        (when var-2 (setf nl var-2))
        (when var-3 (setf k var-3))
        (when var-5 (setf one var-5))
        (when var-7 (setf ldu2 var-7))
        (when var-9 (setf ldq var-9))
        (when var-10 (setf one var-10))
        (when var-12 (setf ldu var-12))))))
    ((> (f2cl-lib:fref ctot (3) ((1 *))) 0)
     (setf ktemp
             (f2cl-lib:int-add 2
                               (f2cl-lib:fref ctot-%data% (1) ((1 *))
                                              ctot-%offset%)
                               (f2cl-lib:fref ctot-%data% (2) ((1 *))
                                              ctot-%offset%)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
          var-10 var-11 var-12)
         (dgemm "N" "N" nl k
          (f2cl-lib:array-slice ctot-%data% f2cl-lib:integer4 (3) ((1 *))
                                ctot-%offset%)
          one
          (f2cl-lib:array-slice u2-%data% double-float (1 ktemp)
                                ((1 ldu2) (1 *)) u2-%offset%)
          ldu2
          (f2cl-lib:array-slice q-%data% double-float (ktemp 1)
                                ((1 ldq) (1 *)) q-%offset%)
          ldq zero
          (f2cl-lib:array-slice u-%data% double-float (1 1)
                                ((1 ldu) (1 *)) u-%offset%)
          ldu)
       (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
       (when var-2 (setf nl var-2))
       (when var-3 (setf k var-3))
       (when var-5 (setf one var-5))
       (when var-7 (setf ldu2 var-7))
       (when var-9 (setf ldq var-9))
       (when var-10 (setf zero var-10))
       (when var-12 (setf ldu var-12))))
    (t (dlacpy "F" nl k u2 ldu2 u ldu)))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
        (dcopy k
         (f2cl-lib:array-slice q-%data% double-float (1 1) ((1 ldq) (1 *))
                               q-%offset%)
         ldq
         (f2cl-lib:array-slice u-%data% double-float (nlp1 1)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-1 var-3))
      (when var-0 (setf k var-0))
      (when var-2 (setf ldq var-2))
      (when var-4 (setf ldu var-4)))
    (setf ktemp
            (f2cl-lib:int-add 2
                              (f2cl-lib:fref ctot-%data% (1) ((1 *))
                                             ctot-%offset%)))
    (setf ctemp
            (f2cl-lib:int-add
             (f2cl-lib:fref ctot-%data% (2) ((1 *)) ctot-%offset%)
             (f2cl-lib:fref ctot-%data% (3) ((1 *)) ctot-%offset%)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
         var-11 var-12)
        (dgemm "N" "N" nr k ctemp one
         (f2cl-lib:array-slice u2-%data% double-float (nlp2 ktemp)
                               ((1 ldu2) (1 *)) u2-%offset%)
         ldu2
         (f2cl-lib:array-slice q-%data% double-float (ktemp 1)
                               ((1 ldq) (1 *)) q-%offset%)
         ldq zero
         (f2cl-lib:array-slice u-%data% double-float (nlp2 1)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf nr var-2))
      (when var-3 (setf k var-3))
      (when var-4 (setf ctemp var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf ldu2 var-7))
      (when var-9 (setf ldq var-9))
      (when var-10 (setf zero var-10))
      (when var-12 (setf ldu var-12)))
  label100
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i k) nil)
      (tagbody
        (setf temp
                (multiple-value-bind (ret-val var-0 var-1 var-2)
                    (dnrm2 k
                     (f2cl-lib:array-slice vt-%data% double-float (1 i)
                                           ((1 ldvt) (1 *)) vt-%offset%)
```

```
                    1)
                 (declare (ignore var-1 var-2))
                 (when var-0 (setf k var-0))
                 ret-val))
     (setf (f2cl-lib:fref q-%data% (i 1) ((1 ldq) (1 *)) q-%offset%)
             (/
              (f2cl-lib:fref vt-%data% (1 i) ((1 ldvt) (1 *))
                             vt-%offset%)
              temp))
     (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                   ((> j k) nil)
       (tagbody
         (setf jc (f2cl-lib:fref idxc-%data% (j) ((1 *)) idxc-%offset%))
         (setf (f2cl-lib:fref q-%data% (i j) ((1 ldq) (1 *)) q-%offset%)
                 (/
                  (f2cl-lib:fref vt-%data% (jc i) ((1 ldvt) (1 *))
                                 vt-%offset%)
                  temp))
        label110))
   label120))
(cond
 ((= k 2)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (dgemm "N" "N" k m k one q ldq vt2 ldvt2 zero vt ldvt)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf k var-2))
    (when var-3 (setf m var-3))
    (when var-4 (setf k var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldq var-7))
    (when var-9 (setf ldvt2 var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldvt var-12)))
  (go end_label)))
(setf ktemp
        (f2cl-lib:int-add 1
                          (f2cl-lib:fref ctot-%data% (1) ((1 *))
                                         ctot-%offset%)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12)
    (dgemm "N" "N" k nlp1 ktemp one
     (f2cl-lib:array-slice q-%data% double-float (1 1) ((1 ldq) (1 *))
                           q-%offset%)
     ldq
     (f2cl-lib:array-slice vt2-%data% double-float (1 1)
                           ((1 ldvt2) (1 *)) vt2-%offset%)
     ldvt2 zero
```

```
     (f2cl-lib:array-slice vt-%data% double-float (1 1)
                           ((1 ldvt) (1 *)) vt-%offset%)
     ldvt)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf k var-2))
  (when var-3 (setf nlp1 var-3))
  (when var-4 (setf ktemp var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldq var-7))
  (when var-9 (setf ldvt2 var-9))
  (when var-10 (setf zero var-10))
  (when var-12 (setf ldvt var-12)))
(setf ktemp
        (f2cl-lib:int-add 2
                          (f2cl-lib:fref ctot-%data% (1) ((1 *))
                                         ctot-%offset%)
                          (f2cl-lib:fref ctot-%data% (2) ((1 *))
                                         ctot-%offset%)))
(if (<= ktemp ldvt2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (dgemm "N" "N" k nlp1
         (f2cl-lib:array-slice ctot-%data% f2cl-lib:integer4 (3)
                               ((1 *)) ctot-%offset%)
         one
         (f2cl-lib:array-slice q-%data% double-float (1 ktemp)
                               ((1 ldq) (1 *)) q-%offset%)
         ldq
         (f2cl-lib:array-slice vt2-%data% double-float (ktemp 1)
                               ((1 ldvt2) (1 *)) vt2-%offset%)
         ldvt2 one
         (f2cl-lib:array-slice vt-%data% double-float (1 1)
                               ((1 ldvt) (1 *)) vt-%offset%)
         ldvt)
      (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
      (when var-2 (setf k var-2))
      (when var-3 (setf nlp1 var-3))
      (when var-5 (setf one var-5))
      (when var-7 (setf ldq var-7))
      (when var-9 (setf ldvt2 var-9))
      (when var-10 (setf one var-10))
      (when var-12 (setf ldvt var-12))))
(setf ktemp
        (f2cl-lib:int-add
         (f2cl-lib:fref ctot-%data% (1) ((1 *)) ctot-%offset%) 1))
(setf nrp1 (f2cl-lib:int-add nr sqre))
(cond
 ((> ktemp 1)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
```

```
                  ((> i k) nil)
      (tagbody
        (setf (f2cl-lib:fref q-%data% (i ktemp) ((1 ldq) (1 *))
                             q-%offset%)
              (f2cl-lib:fref q-%data% (i 1) ((1 ldq) (1 *))
                             q-%offset%))
       label130))
    (f2cl-lib:fdo (i nlp2 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref vt2-%data% (ktemp i) ((1 ldvt2) (1 *))
                             vt2-%offset%)
              (f2cl-lib:fref vt2-%data% (1 i) ((1 ldvt2) (1 *))
                             vt2-%offset%))
       label140))))
  (setf ctemp
          (f2cl-lib:int-add 1
                            (f2cl-lib:fref ctot-%data% (2) ((1 *))
                                           ctot-%offset%)
                            (f2cl-lib:fref ctot-%data% (3) ((1 *))
                                           ctot-%offset%)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
       var-11 var-12)
      (dgemm "N" "N" k nrp1 ctemp one
       (f2cl-lib:array-slice q-%data% double-float (1 ktemp)
                             ((1 ldq) (1 *)) q-%offset%)
       ldq
       (f2cl-lib:array-slice vt2-%data% double-float (ktemp nlp2)
                             ((1 ldvt2) (1 *)) vt2-%offset%)
       ldvt2 zero
       (f2cl-lib:array-slice vt-%data% double-float (1 nlp2)
                             ((1 ldvt) (1 *)) vt-%offset%)
       ldvt)
    (declare (ignore var-0 var-1 var-6 var-8 var-11))
    (when var-2 (setf k var-2))
    (when var-3 (setf nrp1 var-3))
    (when var-4 (setf ctemp var-4))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldq var-7))
    (when var-9 (setf ldvt2 var-9))
    (when var-10 (setf zero var-10))
    (when var-12 (setf ldvt var-12)))
  (go end_label)
 end_label
  (return
   (values nl nr nil k nil nil ldq nil nil ldu nil ldu2 nil ldvt nil
           ldvt2 nil nil nil info))))))
```

———-

90.173 dlasd4

90.173.1 Axiom unit tests

— **dlasd4.input** —

```
)set break resume
)sys rm -f dlasd4.output
)spool dlasd4.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.173.2 Axiom help page

— **dlasd4.help** —

```
======================================================================
dlasd4 examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DLASD4  -  compute  the square root of the I-th updated eigenvalue of a
       positive symmetric rank-one modification to a positive diagonal  matrix
       whose  entries are given as the squares of the corresponding entries in
       the array d, and that   0 <= D(i) < D(j) for i < j  and that RHO > 0

SYNOPSIS
       SUBROUTINE DLASD4( N, I, D, Z, DELTA, RHO, SIGMA, WORK, INFO )

           INTEGER        I, INFO, N

           DOUBLE         PRECISION RHO, SIGMA
```

```
         DOUBLE         PRECISION D( * ), DELTA( * ), WORK( * ), Z( * )

Purpose
=======

This subroutine computes the square root of the I-th updated
eigenvalue of a positive symmetric rank-one modification to
a positive diagonal matrix whose entries are given as the squares
of the corresponding entries in the array d, and that

       0 <= D(i) < D(j)  for  i < j

and that RHO > 0. This is arranged by the calling routine, and is
no loss in generality.  The rank-one modified system is thus

       diag( D ) * diag( D ) +  RHO *  Z * Z_transpose.

where we assume the Euclidean norm of Z is 1.

The method consists of approximating the rational functions in the
secular equation by simpler interpolating rational functions.

Arguments
=========

N      (input) INTEGER
       The length of all arrays.

I      (input) INTEGER
       The index of the eigenvalue to be computed.  1 <= I <= N.

D      (input) DOUBLE PRECISION array, dimension ( N )
       The original eigenvalues.  It is assumed that they are in
       order, 0 <= D(I) < D(J)  for I < J.

Z      (input) DOUBLE PRECISION array, dimension ( N )
       The components of the updating vector.

DELTA  (output) DOUBLE PRECISION array, dimension ( N )
       If N .ne. 1, DELTA contains (D(j) - sigma_I) in its  j-th
       component.  If N = 1, then DELTA(1) = 1.  The vector DELTA
       contains the information necessary to construct the
       (singular) eigenvectors.

RHO    (input) DOUBLE PRECISION
       The scalar in the symmetric updating formula.

SIGMA  (output) DOUBLE PRECISION
       The computed lambda_I, the I-th updated eigenvalue.
```

```
WORK   (workspace) DOUBLE PRECISION array, dimension ( N )
       If N .ne. 1, WORK contains (D(j) + sigma_I) in its  j-th
       component.  If N = 1, then WORK( 1 ) = 1.

INFO   (output) INTEGER
       = 0:  successful exit
       > 0:  if INFO = 1, the updating process failed.

Internal Parameters
===================

Logical variable ORGATI (origin-at-i?) is used for distinguishing
whether D(i) or D(i+1) is treated as the origin.

          ORGATI = .true.    origin at i
          ORGATI = .false.   origin at i+1

Logical variable SWTCH3 (switch-for-3-poles?) is for noting
if we are working with THREE poles!

MAXIT is the maximum number of iterations allowed for each
eigenvalue.

Further Details
===============

Based on contributions by
   Ren-Cang Li, Computer Science Division, University of California
   at Berkeley, USA
```

———-

90.173.3 fortran code

— dlasd4.f —

```
      SUBROUTINE DLASD4( N, I, D, Z, DELTA, RHO, SIGMA, WORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Oak Ridge National Lab, Argonne National Lab,
*     Courant Institute, NAG Ltd., and Rice University
*     October 31, 1999
*
*     .. Scalar Arguments ..
      INTEGER            I, INFO, N
      DOUBLE PRECISION   RHO, SIGMA
```

```
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), DELTA( * ), WORK( * ), Z( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            MAXIT
      PARAMETER          ( MAXIT = 20 )
      DOUBLE PRECISION   ZERO, ONE, TWO, THREE, FOUR, EIGHT, TEN
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0, TWO = 2.0D+0,
     $                   THREE = 3.0D+0, FOUR = 4.0D+0, EIGHT = 8.0D+0,
     $                   TEN = 10.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ORGATI, SWTCH, SWTCH3
      INTEGER            II, IIM1, IIP1, IP1, ITER, J, NITER
      DOUBLE PRECISION   A, B, C, DELSQ, DELSQ2, DPHI, DPSI, DTIIM,
     $                   DTIIP, DTIPSQ, DTISQ, DTNSQ, DTNSQ1, DW, EPS,
     $                   ERRETM, ETA, PHI, PREW, PSI, RHOINV, SG2LB,
     $                   SG2UB, TAU, TEMP, TEMP1, TEMP2, W
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   DD( 3 ), ZZ( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLAED6, DLASD5
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Since this routine is called in an inner loop, we do no argument
*     checking.
*
*     Quick return for N=1 and 2.
*
      INFO = 0
      IF( N.EQ.1 ) THEN
*
*        Presumably, I=1 upon entry
*
         SIGMA = SQRT( D( 1 )*D( 1 )+RHO*Z( 1 )*Z( 1 ) )
         DELTA( 1 ) = ONE
```

```
         WORK( 1 ) = ONE
         RETURN
      END IF
      IF( N.EQ.2 ) THEN
         CALL DLASD5( I, D, Z, DELTA, RHO, SIGMA, WORK )
         RETURN
      END IF
*
*     Compute machine epsilon
*
      EPS = DLAMCH( 'Epsilon' )
      RHOINV = ONE / RHO
*
*     The case I = N
*
      IF( I.EQ.N ) THEN
*
*        Initialize some basic variables
*
         II = N - 1
         NITER = 1
*
*        Calculate initial guess
*
         TEMP = RHO / TWO
*
*        If ||Z||_2 is not one, then TEMP should be set to
*        RHO * ||Z||_2^2 / TWO
*
         TEMP1 = TEMP / ( D( N )+SQRT( D( N )*D( N )+TEMP ) )
         DO 10 J = 1, N
            WORK( J ) = D( J ) + D( N ) + TEMP1
            DELTA( J ) = ( D( J )-D( N ) ) - TEMP1
   10    CONTINUE
*
         PSI = ZERO
         DO 20 J = 1, N - 2
            PSI = PSI + Z( J )*Z( J ) / ( DELTA( J )*WORK( J ) )
   20    CONTINUE
*
         C = RHOINV + PSI
         W = C + Z( II )*Z( II ) / ( DELTA( II )*WORK( II ) ) +
     $       Z( N )*Z( N ) / ( DELTA( N )*WORK( N ) )
*
         IF( W.LE.ZERO ) THEN
            TEMP1 = SQRT( D( N )*D( N )+RHO )
            TEMP = Z( N-1 )*Z( N-1 ) / ( ( D( N-1 )+TEMP1 )*
     $              ( D( N )-D( N-1 )+RHO / ( D( N )+TEMP1 ) ) ) +
     $              Z( N )*Z( N ) / RHO
*
```

```
*          The following TAU is to approximate
*          SIGMA_n^2 - D( N )*D( N )
*
           IF( C.LE.TEMP ) THEN
              TAU = RHO
           ELSE
              DELSQ = ( D( N )-D( N-1 ) )*( D( N )+D( N-1 ) )
              A = -C*DELSQ + Z( N-1 )*Z( N-1 ) + Z( N )*Z( N )
              B = Z( N )*Z( N )*DELSQ
              IF( A.LT.ZERO ) THEN
                 TAU = TWO*B / ( SQRT( A*A+FOUR*B*C )-A )
              ELSE
                 TAU = ( A+SQRT( A*A+FOUR*B*C ) ) / ( TWO*C )
              END IF
           END IF
*
*          It can be proved that
*              D(N)^2+RHO/2 <= SIGMA_n^2 < D(N)^2+TAU <= D(N)^2+RHO
*
        ELSE
           DELSQ = ( D( N )-D( N-1 ) )*( D( N )+D( N-1 ) )
           A = -C*DELSQ + Z( N-1 )*Z( N-1 ) + Z( N )*Z( N )
           B = Z( N )*Z( N )*DELSQ
*
*          The following TAU is to approximate
*          SIGMA_n^2 - D( N )*D( N )
*
           IF( A.LT.ZERO ) THEN
              TAU = TWO*B / ( SQRT( A*A+FOUR*B*C )-A )
           ELSE
              TAU = ( A+SQRT( A*A+FOUR*B*C ) ) / ( TWO*C )
           END IF
*
*          It can be proved that
*          D(N)^2 < D(N)^2+TAU < SIGMA(N)^2 < D(N)^2+RHO/2
*
        END IF
*
*       The following ETA is to approximate SIGMA_n - D( N )
*
        ETA = TAU / ( D( N )+SQRT( D( N )*D( N )+TAU ) )
*
        SIGMA = D( N ) + ETA
        DO 30 J = 1, N
           DELTA( J ) = ( D( J )-D( I ) ) - ETA
           WORK( J ) = D( J ) + D( I ) + ETA
   30   CONTINUE
*
*       Evaluate PSI and the derivative DPSI
*
```

```
         DPSI = ZERO
         PSI = ZERO
         ERRETM = ZERO
         DO 40 J = 1, II
            TEMP = Z( J ) / ( DELTA( J )*WORK( J ) )
            PSI = PSI + Z( J )*TEMP
            DPSI = DPSI + TEMP*TEMP
            ERRETM = ERRETM + PSI
   40    CONTINUE
         ERRETM = ABS( ERRETM )
*
*        Evaluate PHI and the derivative DPHI
*
         TEMP = Z( N ) / ( DELTA( N )*WORK( N ) )
         PHI = Z( N )*TEMP
         DPHI = TEMP*TEMP
         ERRETM = EIGHT*( -PHI-PSI ) + ERRETM - PHI + RHOINV +
     $           ABS( TAU )*( DPSI+DPHI )
*
         W = RHOINV + PHI + PSI
*
*        Test for convergence
*
         IF( ABS( W ).LE.EPS*ERRETM ) THEN
            GO TO 240
         END IF
*
*        Calculate the new step
*
         NITER = NITER + 1
         DTNSQ1 = WORK( N-1 )*DELTA( N-1 )
         DTNSQ = WORK( N )*DELTA( N )
         C = W - DTNSQ1*DPSI - DTNSQ*DPHI
         A = ( DTNSQ+DTNSQ1 )*W - DTNSQ*DTNSQ1*( DPSI+DPHI )
         B = DTNSQ*DTNSQ1*W
         IF( C.LT.ZERO )
     $      C = ABS( C )
         IF( C.EQ.ZERO ) THEN
            ETA = RHO - SIGMA*SIGMA
         ELSE IF( A.GE.ZERO ) THEN
            ETA = ( A+SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
         ELSE
            ETA = TWO*B / ( A-SQRT( ABS( A*A-FOUR*B*C ) ) )
         END IF
*
*        Note, eta should be positive if w is negative, and
*        eta should be negative otherwise. However,
*        if for some reason caused by roundoff, eta*w > 0,
*        we simply use one Newton step instead. This way
*        will guarantee eta*w < 0.
```

```
*
         IF( W*ETA.GT.ZERO )
     $      ETA = -W / ( DPSI+DPHI )
         TEMP = ETA - DTNSQ
         IF( TEMP.GT.RHO )
     $      ETA = RHO + DTNSQ
*
         TAU = TAU + ETA
         ETA = ETA / ( SIGMA+SQRT( ETA+SIGMA*SIGMA ) )
         DO 50 J = 1, N
            DELTA( J ) = DELTA( J ) - ETA
            WORK( J ) = WORK( J ) + ETA
   50    CONTINUE
*
         SIGMA = SIGMA + ETA
*
*        Evaluate PSI and the derivative DPSI
*
         DPSI = ZERO
         PSI = ZERO
         ERRETM = ZERO
         DO 60 J = 1, II
            TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
            PSI = PSI + Z( J )*TEMP
            DPSI = DPSI + TEMP*TEMP
            ERRETM = ERRETM + PSI
   60    CONTINUE
         ERRETM = ABS( ERRETM )
*
*        Evaluate PHI and the derivative DPHI
*
         TEMP = Z( N ) / ( WORK( N )*DELTA( N ) )
         PHI = Z( N )*TEMP
         DPHI = TEMP*TEMP
         ERRETM = EIGHT*( -PHI-PSI ) + ERRETM - PHI + RHOINV +
     $           ABS( TAU )*( DPSI+DPHI )
*
         W = RHOINV + PHI + PSI
*
*        Main loop to update the values of the array   DELTA
*
         ITER = NITER + 1
*
         DO 90 NITER = ITER, MAXIT
*
*           Test for convergence
*
            IF( ABS( W ).LE.EPS*ERRETM ) THEN
               GO TO 240
            END IF
```

```
*
*           Calculate the new step
*
            DTNSQ1 = WORK( N-1 )*DELTA( N-1 )
            DTNSQ = WORK( N )*DELTA( N )
            C = W - DTNSQ1*DPSI - DTNSQ*DPHI
            A = ( DTNSQ+DTNSQ1 )*W - DTNSQ1*DTNSQ*( DPSI+DPHI )
            B = DTNSQ1*DTNSQ*W
            IF( A.GE.ZERO ) THEN
               ETA = ( A+SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
            ELSE
               ETA = TWO*B / ( A-SQRT( ABS( A*A-FOUR*B*C ) ) )
            END IF
*
*           Note, eta should be positive if w is negative, and
*           eta should be negative otherwise. However,
*           if for some reason caused by roundoff, eta*w > 0,
*           we simply use one Newton step instead. This way
*           will guarantee eta*w < 0.
*
            IF( W*ETA.GT.ZERO )
     $         ETA = -W / ( DPSI+DPHI )
            TEMP = ETA - DTNSQ
            IF( TEMP.LE.ZERO )
     $         ETA = ETA / TWO
*
            TAU = TAU + ETA
            ETA = ETA / ( SIGMA+SQRT( ETA+SIGMA*SIGMA ) )
            DO 70 J = 1, N
               DELTA( J ) = DELTA( J ) - ETA
               WORK( J ) = WORK( J ) + ETA
   70       CONTINUE
*
            SIGMA = SIGMA + ETA
*
*           Evaluate PSI and the derivative DPSI
*
            DPSI = ZERO
            PSI = ZERO
            ERRETM = ZERO
            DO 80 J = 1, II
               TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
               PSI = PSI + Z( J )*TEMP
               DPSI = DPSI + TEMP*TEMP
               ERRETM = ERRETM + PSI
   80       CONTINUE
            ERRETM = ABS( ERRETM )
*
*           Evaluate PHI and the derivative DPHI
*
```

```
            TEMP = Z( N ) / ( WORK( N )*DELTA( N ) )
            PHI = Z( N )*TEMP
            DPHI = TEMP*TEMP
            ERRETM = EIGHT*( -PHI-PSI ) + ERRETM - PHI + RHOINV +
     $               ABS( TAU )*( DPSI+DPHI )
*
            W = RHOINV + PHI + PSI
   90    CONTINUE
*
*        Return with INFO = 1, NITER = MAXIT and not converged
*
         INFO = 1
         GO TO 240
*
*        End for the case I = N
*
      ELSE
*
*        The case for I < N
*
         NITER = 1
         IP1 = I + 1
*
*        Calculate initial guess
*
         DELSQ = ( D( IP1 )-D( I ) )*( D( IP1 )+D( I ) )
         DELSQ2 = DELSQ / TWO
         TEMP = DELSQ2 / ( D( I )+SQRT( D( I )*D( I )+DELSQ2 ) )
         DO 100 J = 1, N
            WORK( J ) = D( J ) + D( I ) + TEMP
            DELTA( J ) = ( D( J )-D( I ) ) - TEMP
  100    CONTINUE
*
         PSI = ZERO
         DO 110 J = 1, I - 1
            PSI = PSI + Z( J )*Z( J ) / ( WORK( J )*DELTA( J ) )
  110    CONTINUE
*
         PHI = ZERO
         DO 120 J = N, I + 2, -1
            PHI = PHI + Z( J )*Z( J ) / ( WORK( J )*DELTA( J ) )
  120    CONTINUE
         C = RHOINV + PSI + PHI
         W = C + Z( I )*Z( I ) / ( WORK( I )*DELTA( I ) ) +
     $       Z( IP1 )*Z( IP1 ) / ( WORK( IP1 )*DELTA( IP1 ) )
*
         IF( W.GT.ZERO ) THEN
*
*           d(i)^2 < the ith sigma^2 < (d(i)^2+d(i+1)^2)/2
*
```

```
*           We choose d(i) as origin.
*
            ORGATI = .TRUE.
            SG2LB = ZERO
            SG2UB = DELSQ2
            A = C*DELSQ + Z( I )*Z( I ) + Z( IP1 )*Z( IP1 )
            B = Z( I )*Z( I )*DELSQ
            IF( A.GT.ZERO ) THEN
               TAU = TWO*B / ( A+SQRT( ABS( A*A-FOUR*B*C ) ) )
            ELSE
               TAU = ( A-SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
            END IF
*
*           TAU now is an estimation of SIGMA^2 - D( I )^2. The
*           following, however, is the corresponding estimation of
*           SIGMA - D( I ).
*
            ETA = TAU / ( D( I )+SQRT( D( I )*D( I )+TAU ) )
         ELSE
*
*           (d(i)^2+d(i+1)^2)/2 <= the ith sigma^2 < d(i+1)^2/2
*
*           We choose d(i+1) as origin.
*
            ORGATI = .FALSE.
            SG2LB = -DELSQ2
            SG2UB = ZERO
            A = C*DELSQ - Z( I )*Z( I ) - Z( IP1 )*Z( IP1 )
            B = Z( IP1 )*Z( IP1 )*DELSQ
            IF( A.LT.ZERO ) THEN
               TAU = TWO*B / ( A-SQRT( ABS( A*A+FOUR*B*C ) ) )
            ELSE
               TAU = -( A+SQRT( ABS( A*A+FOUR*B*C ) ) ) / ( TWO*C )
            END IF
*
*           TAU now is an estimation of SIGMA^2 - D( IP1 )^2. The
*           following, however, is the corresponding estimation of
*           SIGMA - D( IP1 ).
*
            ETA = TAU / ( D( IP1 )+SQRT( ABS( D( IP1 )*D( IP1 )+
     $            TAU ) ) )
         END IF
*
         IF( ORGATI ) THEN
            II = I
            SIGMA = D( I ) + ETA
            DO 130 J = 1, N
               WORK( J ) = D( J ) + D( I ) + ETA
               DELTA( J ) = ( D( J )-D( I ) ) - ETA
  130       CONTINUE
```

```
         ELSE
            II = I + 1
            SIGMA = D( IP1 ) + ETA
            DO 140 J = 1, N
               WORK( J ) = D( J ) + D( IP1 ) + ETA
               DELTA( J ) = ( D( J )-D( IP1 ) ) - ETA
  140       CONTINUE
         END IF
         IIM1 = II - 1
         IIP1 = II + 1
*
*        Evaluate PSI and the derivative DPSI
*
         DPSI = ZERO
         PSI = ZERO
         ERRETM = ZERO
         DO 150 J = 1, IIM1
            TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
            PSI = PSI + Z( J )*TEMP
            DPSI = DPSI + TEMP*TEMP
            ERRETM = ERRETM + PSI
  150    CONTINUE
         ERRETM = ABS( ERRETM )
*
*        Evaluate PHI and the derivative DPHI
*
         DPHI = ZERO
         PHI = ZERO
         DO 160 J = N, IIP1, -1
            TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
            PHI = PHI + Z( J )*TEMP
            DPHI = DPHI + TEMP*TEMP
            ERRETM = ERRETM + PHI
  160    CONTINUE
*
         W = RHOINV + PHI + PSI
*
*        W is the value of the secular function with
*        its ii-th element removed.
*
         SWTCH3 = .FALSE.
         IF( ORGATI ) THEN
            IF( W.LT.ZERO )
     $         SWTCH3 = .TRUE.
         ELSE
            IF( W.GT.ZERO )
     $         SWTCH3 = .TRUE.
         END IF
         IF( II.EQ.1 .OR. II.EQ.N )
     $      SWTCH3 = .FALSE.
```

```
*
         TEMP = Z( II ) / ( WORK( II )*DELTA( II ) )
         DW = DPSI + DPHI + TEMP*TEMP
         TEMP = Z( II )*TEMP
         W = W + TEMP
         ERRETM = EIGHT*( PHI-PSI ) + ERRETM + TWO*RHOINV +
     $           THREE*ABS( TEMP ) + ABS( TAU )*DW
*
*        Test for convergence
*
         IF( ABS( W ).LE.EPS*ERRETM ) THEN
            GO TO 240
         END IF
*
         IF( W.LE.ZERO ) THEN
            SG2LB = MAX( SG2LB, TAU )
         ELSE
            SG2UB = MIN( SG2UB, TAU )
         END IF
*
*        Calculate the new step
*
         NITER = NITER + 1
         IF( .NOT.SWTCH3 ) THEN
            DTIPSQ = WORK( IP1 )*DELTA( IP1 )
            DTISQ = WORK( I )*DELTA( I )
            IF( ORGATI ) THEN
               C = W - DTIPSQ*DW + DELSQ*( Z( I ) / DTISQ )**2
            ELSE
               C = W - DTISQ*DW - DELSQ*( Z( IP1 ) / DTIPSQ )**2
            END IF
            A = ( DTIPSQ+DTISQ )*W - DTIPSQ*DTISQ*DW
            B = DTIPSQ*DTISQ*W
            IF( C.EQ.ZERO ) THEN
               IF( A.EQ.ZERO ) THEN
                  IF( ORGATI ) THEN
                     A = Z( I )*Z( I ) + DTIPSQ*DTIPSQ*( DPSI+DPHI )
                  ELSE
                     A = Z( IP1 )*Z( IP1 ) + DTISQ*DTISQ*( DPSI+DPHI )
                  END IF
               END IF
               ETA = B / A
            ELSE IF( A.LE.ZERO ) THEN
               ETA = ( A-SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
            ELSE
               ETA = TWO*B / ( A+SQRT( ABS( A*A-FOUR*B*C ) ) )
            END IF
         ELSE
*
*           Interpolation using THREE most relevant poles
```

```
*
          DTIIM = WORK( IIM1 )*DELTA( IIM1 )
          DTIIP = WORK( IIP1 )*DELTA( IIP1 )
          TEMP = RHOINV + PSI + PHI
          IF( ORGATI ) THEN
             TEMP1 = Z( IIM1 ) / DTIIM
             TEMP1 = TEMP1*TEMP1
             C = ( TEMP - DTIIP*( DPSI+DPHI ) ) -
     $           ( D( IIM1 )-D( IIP1 ) )*( D( IIM1 )+D( IIP1 ) )*TEMP1
             ZZ( 1 ) = Z( IIM1 )*Z( IIM1 )
             IF( DPSI.LT.TEMP1 ) THEN
                ZZ( 3 ) = DTIIP*DTIIP*DPHI
             ELSE
                ZZ( 3 ) = DTIIP*DTIIP*( ( DPSI-TEMP1 )+DPHI )
             END IF
          ELSE
             TEMP1 = Z( IIP1 ) / DTIIP
             TEMP1 = TEMP1*TEMP1
             C = ( TEMP - DTIIM*( DPSI+DPHI ) ) -
     $           ( D( IIP1 )-D( IIM1 ) )*( D( IIM1 )+D( IIP1 ) )*TEMP1
             IF( DPHI.LT.TEMP1 ) THEN
                ZZ( 1 ) = DTIIM*DTIIM*DPSI
             ELSE
                ZZ( 1 ) = DTIIM*DTIIM*( DPSI+( DPHI-TEMP1 ) )
             END IF
             ZZ( 3 ) = Z( IIP1 )*Z( IIP1 )
          END IF
          ZZ( 2 ) = Z( II )*Z( II )
          DD( 1 ) = DTIIM
          DD( 2 ) = DELTA( II )*WORK( II )
          DD( 3 ) = DTIIP
          CALL DLAED6( NITER, ORGATI, C, DD, ZZ, W, ETA, INFO )
          IF( INFO.NE.0 )
     $       GO TO 240
       END IF
*
*      Note, eta should be positive if w is negative, and
*      eta should be negative otherwise. However,
*      if for some reason caused by roundoff, eta*w > 0,
*      we simply use one Newton step instead. This way
*      will guarantee eta*w < 0.
*
       IF( W*ETA.GE.ZERO )
     $    ETA = -W / DW
       IF( ORGATI ) THEN
          TEMP1 = WORK( I )*DELTA( I )
          TEMP = ETA - TEMP1
       ELSE
          TEMP1 = WORK( IP1 )*DELTA( IP1 )
          TEMP = ETA - TEMP1
```

```
         END IF
         IF( TEMP.GT.SG2UB .OR. TEMP.LT.SG2LB ) THEN
            IF( W.LT.ZERO ) THEN
               ETA = ( SG2UB-TAU ) / TWO
            ELSE
               ETA = ( SG2LB-TAU ) / TWO
            END IF
         END IF
*
         TAU = TAU + ETA
         ETA = ETA / ( SIGMA+SQRT( SIGMA*SIGMA+ETA ) )
*
         PREW = W
*
         SIGMA = SIGMA + ETA
         DO 170 J = 1, N
            WORK( J ) = WORK( J ) + ETA
            DELTA( J ) = DELTA( J ) - ETA
  170    CONTINUE
*
*        Evaluate PSI and the derivative DPSI
*
         DPSI = ZERO
         PSI = ZERO
         ERRETM = ZERO
         DO 180 J = 1, IIM1
            TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
            PSI = PSI + Z( J )*TEMP
            DPSI = DPSI + TEMP*TEMP
            ERRETM = ERRETM + PSI
  180    CONTINUE
         ERRETM = ABS( ERRETM )
*
*        Evaluate PHI and the derivative DPHI
*
         DPHI = ZERO
         PHI = ZERO
         DO 190 J = N, IIP1, -1
            TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
            PHI = PHI + Z( J )*TEMP
            DPHI = DPHI + TEMP*TEMP
            ERRETM = ERRETM + PHI
  190    CONTINUE
*
         TEMP = Z( II ) / ( WORK( II )*DELTA( II ) )
         DW = DPSI + DPHI + TEMP*TEMP
         TEMP = Z( II )*TEMP
         W = RHOINV + PHI + PSI + TEMP
         ERRETM = EIGHT*( PHI-PSI ) + ERRETM + TWO*RHOINV +
     $           THREE*ABS( TEMP ) + ABS( TAU )*DW
```

```
*
         IF( W.LE.ZERO ) THEN
            SG2LB = MAX( SG2LB, TAU )
         ELSE
            SG2UB = MIN( SG2UB, TAU )
         END IF
*
         SWTCH = .FALSE.
         IF( ORGATI ) THEN
            IF( -W.GT.ABS( PREW ) / TEN )
     $         SWTCH = .TRUE.
         ELSE
            IF( W.GT.ABS( PREW ) / TEN )
     $         SWTCH = .TRUE.
         END IF
*
*        Main loop to update the values of the array   DELTA and WORK
*
         ITER = NITER + 1
*
         DO 230 NITER = ITER, MAXIT
*
*           Test for convergence
*
            IF( ABS( W ).LE.EPS*ERRETM ) THEN
               GO TO 240
            END IF
*
*           Calculate the new step
*
            IF( .NOT.SWTCH3 ) THEN
               DTIPSQ = WORK( IP1 )*DELTA( IP1 )
               DTISQ = WORK( I )*DELTA( I )
               IF( .NOT.SWTCH ) THEN
                  IF( ORGATI ) THEN
                     C = W - DTIPSQ*DW + DELSQ*( Z( I ) / DTISQ )**2
                  ELSE
                     C = W - DTISQ*DW - DELSQ*( Z( IP1 ) / DTIPSQ )**2
                  END IF
               ELSE
                  TEMP = Z( II ) / ( WORK( II )*DELTA( II ) )
                  IF( ORGATI ) THEN
                     DPSI = DPSI + TEMP*TEMP
                  ELSE
                     DPHI = DPHI + TEMP*TEMP
                  END IF
                  C = W - DTISQ*DPSI - DTIPSQ*DPHI
               END IF
               A = ( DTIPSQ+DTISQ )*W - DTIPSQ*DTISQ*DW
               B = DTIPSQ*DTISQ*W
```

```
            IF( C.EQ.ZERO ) THEN
               IF( A.EQ.ZERO ) THEN
                  IF( .NOT.SWTCH ) THEN
                     IF( ORGATI ) THEN
                        A = Z( I )*Z( I ) + DTIPSQ*DTIPSQ*
     $                      ( DPSI+DPHI )
                     ELSE
                        A = Z( IP1 )*Z( IP1 ) +
     $                      DTISQ*DTISQ*( DPSI+DPHI )
                     END IF
                  ELSE
                     A = DTISQ*DTISQ*DPSI + DTIPSQ*DTIPSQ*DPHI
                  END IF
               END IF
               ETA = B / A
            ELSE IF( A.LE.ZERO ) THEN
               ETA = ( A-SQRT( ABS( A*A-FOUR*B*C ) ) ) / ( TWO*C )
            ELSE
               ETA = TWO*B / ( A+SQRT( ABS( A*A-FOUR*B*C ) ) )
            END IF
         ELSE
*
*           Interpolation using THREE most relevant poles
*
            DTIIM = WORK( IIM1 )*DELTA( IIM1 )
            DTIIP = WORK( IIP1 )*DELTA( IIP1 )
            TEMP = RHOINV + PSI + PHI
            IF( SWTCH ) THEN
               C = TEMP - DTIIM*DPSI - DTIIP*DPHI
               ZZ( 1 ) = DTIIM*DTIIM*DPSI
               ZZ( 3 ) = DTIIP*DTIIP*DPHI
            ELSE
               IF( ORGATI ) THEN
                  TEMP1 = Z( IIM1 ) / DTIIM
                  TEMP1 = TEMP1*TEMP1
                  TEMP2 = ( D( IIM1 )-D( IIP1 ) )*
     $                    ( D( IIM1 )+D( IIP1 ) )*TEMP1
                  C = TEMP - DTIIP*( DPSI+DPHI ) - TEMP2
                  ZZ( 1 ) = Z( IIM1 )*Z( IIM1 )
                  IF( DPSI.LT.TEMP1 ) THEN
                     ZZ( 3 ) = DTIIP*DTIIP*DPHI
                  ELSE
                     ZZ( 3 ) = DTIIP*DTIIP*( ( DPSI-TEMP1 )+DPHI )
                  END IF
               ELSE
                  TEMP1 = Z( IIP1 ) / DTIIP
                  TEMP1 = TEMP1*TEMP1
                  TEMP2 = ( D( IIP1 )-D( IIM1 ) )*
     $                    ( D( IIM1 )+D( IIP1 ) )*TEMP1
                  C = TEMP - DTIIM*( DPSI+DPHI ) - TEMP2
```

```
                  IF( DPHI.LT.TEMP1 ) THEN
                     ZZ( 1 ) = DTIIM*DTIIM*DPSI
                  ELSE
                     ZZ( 1 ) = DTIIM*DTIIM*( DPSI+( DPHI-TEMP1 ) )
                  END IF
                  ZZ( 3 ) = Z( IIP1 )*Z( IIP1 )
               END IF
            END IF
            DD( 1 ) = DTIIM
            DD( 2 ) = DELTA( II )*WORK( II )
            DD( 3 ) = DTIIP
            CALL DLAED6( NITER, ORGATI, C, DD, ZZ, W, ETA, INFO )
            IF( INFO.NE.0 )
     $         GO TO 240
         END IF
*
*        Note, eta should be positive if w is negative, and
*        eta should be negative otherwise. However,
*        if for some reason caused by roundoff, eta*w > 0,
*        we simply use one Newton step instead. This way
*        will guarantee eta*w < 0.
*
         IF( W*ETA.GE.ZERO )
     $      ETA = -W / DW
         IF( ORGATI ) THEN
            TEMP1 = WORK( I )*DELTA( I )
            TEMP = ETA - TEMP1
         ELSE
            TEMP1 = WORK( IP1 )*DELTA( IP1 )
            TEMP = ETA - TEMP1
         END IF
         IF( TEMP.GT.SG2UB .OR. TEMP.LT.SG2LB ) THEN
            IF( W.LT.ZERO ) THEN
               ETA = ( SG2UB-TAU ) / TWO
            ELSE
               ETA = ( SG2LB-TAU ) / TWO
            END IF
         END IF
*
         TAU = TAU + ETA
         ETA = ETA / ( SIGMA+SQRT( SIGMA*SIGMA+ETA ) )
*
         SIGMA = SIGMA + ETA
         DO 200 J = 1, N
            WORK( J ) = WORK( J ) + ETA
            DELTA( J ) = DELTA( J ) - ETA
  200    CONTINUE
*
         PREW = W
*
```

```
*           Evaluate PSI and the derivative DPSI
*
            DPSI = ZERO
            PSI = ZERO
            ERRETM = ZERO
            DO 210 J = 1, IIM1
               TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
               PSI = PSI + Z( J )*TEMP
               DPSI = DPSI + TEMP*TEMP
               ERRETM = ERRETM + PSI
  210       CONTINUE
            ERRETM = ABS( ERRETM )
*
*           Evaluate PHI and the derivative DPHI
*
            DPHI = ZERO
            PHI = ZERO
            DO 220 J = N, IIP1, -1
               TEMP = Z( J ) / ( WORK( J )*DELTA( J ) )
               PHI = PHI + Z( J )*TEMP
               DPHI = DPHI + TEMP*TEMP
               ERRETM = ERRETM + PHI
  220       CONTINUE
*
            TEMP = Z( II ) / ( WORK( II )*DELTA( II ) )
            DW = DPSI + DPHI + TEMP*TEMP
            TEMP = Z( II )*TEMP
            W = RHOINV + PHI + PSI + TEMP
            ERRETM = EIGHT*( PHI-PSI ) + ERRETM + TWO*RHOINV +
     $               THREE*ABS( TEMP ) + ABS( TAU )*DW
            IF( W*PREW.GT.ZERO .AND. ABS( W ).GT.ABS( PREW ) / TEN )
     $         SWTCH = .NOT.SWTCH
*
            IF( W.LE.ZERO ) THEN
               SG2LB = MAX( SG2LB, TAU )
            ELSE
               SG2UB = MIN( SG2UB, TAU )
            END IF
*
  230    CONTINUE
*
*        Return with INFO = 1, NITER = MAXIT and not converged
*
         INFO = 1
*
      END IF
*
  240 CONTINUE
      RETURN
*
```

```
*     End of DLASD4
*
      END
```

———-

90.173.4 lisp code

— **dlasd4.lisp** —

```
(let* ((maxit 400)
       (zero 0.0d0)
       (one 1.0d0)
       (two 2.0d0)
       (three 3.0d0)
       (four 4.0d0)
       (eight 8.0d0)
       (ten 10.0d0))
  (declare (type (f2cl-lib:integer4 400 400) maxit)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 3.0d0 3.0d0) three)
           (type (double-float 4.0d0 4.0d0) four)
           (type (double-float 8.0d0 8.0d0) eight)
           (type (double-float 10.0d0 10.0d0) ten)
           (ignorable maxit zero one two three four eight ten))
  (defun dlasd4 (n i d z delta rho sigma work info)
    (declare (type (double-float) sigma rho)
             (type (array double-float (*)) work delta z d)
             (type (f2cl-lib:integer4) info i n))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (z double-float z-%data% z-%offset%)
         (delta double-float delta-%data% delta-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((dd (make-array 3 :element-type 'double-float))
             (zz (make-array 3 :element-type 'double-float)) (a 0.0d0)
             (b 0.0d0) (c 0.0d0) (delsq 0.0d0) (delsq2 0.0d0) (sq2 0.0d0)
             (dphi 0.0d0) (dpsi 0.0d0) (dtiim 0.0d0) (dtiip 0.0d0)
             (dtipsq 0.0d0) (dtisq 0.0d0) (dtnsq 0.0d0) (dtnsq1 0.0d0)
             (dw 0.0d0) (eps 0.0d0) (erretm 0.0d0) (eta 0.0d0) (phi 0.0d0)
             (prew 0.0d0) (psi 0.0d0) (rhoinv 0.0d0) (sglb 0.0d0) (sgub 0.0d0)
             (tau 0.0d0) (tau2 0.0d0) (temp 0.0d0) (temp1 0.0d0) (temp2 0.0d0)
             (w 0.0d0) (ii 0) (iim1 0) (iip1 0) (ip1 0) (iter 0) (j 0)
             (niter 0) (orgati nil) (swtch nil) (swtch3 nil) (geomavg nil))
        (declare (type (array double-float (3)) dd zz)
```

```
          (type (double-float) a b c delsq delsq2 sq2 dphi dpsi dtiim
           dtiip dtipsq dtisq dtnsq dtnsq1 dw eps erretm eta phi prew
           psi rhoinv sglb sgub tau tau2 temp temp1 temp2 w)
          (type (f2cl-lib:integer4) ii iim1 iip1 ip1 iter j niter)
          (type f2cl-lib:logical orgati swtch swtch3 geomavg))
(setf info 0)
(cond
 ((= n 1)
  (setf sigma
          (f2cl-lib:fsqrt
           (+
            (* (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
               (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
            (* rho (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)
               (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)))))
  (setf (f2cl-lib:fref delta-%data% (1) ((1 *)) delta-%offset%) one)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) one)
  (go end_label)))
(cond
 ((= n 2)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlasd5 i d z delta rho sigma work)
    (declare (ignore var-1 var-2 var-3 var-6))
    (when var-0 (setf i var-0))
    (when var-4 (setf rho var-4))
    (when var-5 (setf sigma var-5)))
  (go end_label)))
(setf eps (dlamch "Epsilon"))
(setf rhoinv (/ one rho))
(setf tau2 zero)
(cond
 ((= i n) (setf ii (f2cl-lib:int-sub n 1)) (setf niter 1)
  (setf temp (/ rho two))
  (setf temp1
          (/ temp
             (+ (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                (f2cl-lib:fsqrt
                 (+
                  (* (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                     (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%))
                  temp)))))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
              (+ (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                 (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                 temp1))
      (setf (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
              (- (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
```

```
                (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                temp1))
   label10))
(setf psi zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 2))) nil)
  (tagbody
    (setf psi
            (+ psi
               (/
                (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                   (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%))
                (*
                 (f2cl-lib:fref delta-%data% (j) ((1 *))
                                delta-%offset%)
                 (f2cl-lib:fref work-%data% (j) ((1 *))
                                work-%offset%)))))
   label20))
(setf c (+ rhoinv psi))
(setf w
        (+ c
           (/
            (* (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%)
               (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%))
            (*
             (f2cl-lib:fref delta-%data% (ii) ((1 *)) delta-%offset%)
             (f2cl-lib:fref work-%data% (ii) ((1 *)) work-%offset%)))
           (/
            (* (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%)
               (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%))
            (*
             (f2cl-lib:fref delta-%data% (n) ((1 *)) delta-%offset%)
             (f2cl-lib:fref work-%data% (n) ((1 *))
                            work-%offset%)))))
(cond
 ((<= w zero)
  (setf temp1
          (f2cl-lib:fsqrt
           (+
            (* (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
               (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%))
            rho)))
  (setf temp
          (+
           (/
            (*
             (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub n 1)) ((1 *))
                            z-%offset%)
             (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub n 1)) ((1 *))
                            z-%offset%))
```

```
                 (*
                  (+
                   (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                  ((1 *)) d-%offset%)
                   temp1)
                  (+
                   (- (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                      (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                     ((1 *)) d-%offset%))
                   (/ rho
                      (+ (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                         temp1)))))
                (/
                 (* (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%)
                    (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%))
                 rho)))
          (cond ((<= c temp) (setf tau rho))
                (t
                 (setf delsq
                         (*
                          (- (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                             (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                            ((1 *)) d-%offset%))
                          (+ (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                             (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                            ((1 *)) d-%offset%))))
                 (setf a
                         (+ (* (- c) delsq)
                            (*
                             (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub n 1))
                                            ((1 *)) z-%offset%)
                             (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub n 1))
                                            ((1 *)) z-%offset%))
                            (*
                             (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%)
                             (f2cl-lib:fref z-%data% (n) ((1 *))
                                            z-%offset%))))
                 (setf b
                         (* (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%)
                            (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%)
                            delsq))
                 (cond
                  ((< a zero)
                   (setf tau2
                           (/ (* two b)
                              (- (f2cl-lib:fsqrt (+ (* a a) (* four b c)))
                                 a))))
                  (t
                   (setf tau2
                           (/ (+ a (f2cl-lib:fsqrt (+ (* a a) (* four b c))))
```

```
                        (* two c)))))
                (setf tau
                        (/ tau2
                           (+
                            (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                            (f2cl-lib:fsqrt
                             (+
                              (*
                               (f2cl-lib:fref d-%data% (n) ((1 *))
                                              d-%offset%)
                               (f2cl-lib:fref d-%data% (n) ((1 *))
                                              d-%offset%))
                              tau2))))))))
        (t
         (setf delsq
                 (*
                  (- (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                     (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                    ((1 *)) d-%offset%))
                  (+ (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                     (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub n 1))
                                    ((1 *)) d-%offset%))))
         (setf a
                 (+ (* (- c) delsq)
                    (*
                     (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub n 1))
                                    ((1 *)) z-%offset%)
                     (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub n 1))
                                    ((1 *)) z-%offset%))
                    (* (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%)
                       (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%))))
         (setf b
                 (* (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%)
                    (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%) delsq))
         (cond
          ((< a zero)
           (setf tau2
                   (/ (* two b)
                      (- (f2cl-lib:fsqrt (+ (* a a) (* four b c))) a))))
          (t
           (setf tau2
                   (/ (+ a (f2cl-lib:fsqrt (+ (* a a) (* four b c))))
                      (* two c)))))
         (setf tau
                 (/ tau2
                    (+ (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                       (f2cl-lib:fsqrt
                        (+
                         (* (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
                            (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%))
```

```
                  tau2)))))))
(setf sigma (+ (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) tau))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
            (- (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
               (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) tau))
    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
            (+ (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
               (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) tau))
   label30))
(setf dpsi zero) (setf psi zero) (setf erretm zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j ii) nil)
  (tagbody
    (setf temp
            (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
               (*
                (f2cl-lib:fref delta-%data% (j) ((1 *))
                               delta-%offset%)
                (f2cl-lib:fref work-%data% (j) ((1 *))
                               work-%offset%))))
    (setf psi
            (+ psi
               (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                  temp)))
    (setf dpsi (+ dpsi (* temp temp)))
    (setf erretm (+ erretm psi))
   label40))
(setf erretm (abs erretm))
(setf temp
        (/ (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%)
           (* (f2cl-lib:fref delta-%data% (n) ((1 *)) delta-%offset%)
              (f2cl-lib:fref work-%data% (n) ((1 *))
                             work-%offset%))))
(setf phi (* (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%) temp))
(setf dphi (* temp temp))
(setf erretm (+ (- (+ (* eight (- (- psi) phi)) erretm) phi) rhoinv))
(setf w (+ rhoinv phi psi))
(cond ((<= (abs w) (* eps erretm)) (go label240)))
(setf niter (f2cl-lib:int-add niter 1))
(setf dtnsq1
        (*
         (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub n 1)) ((1 *))
                        work-%offset%)
         (f2cl-lib:fref delta-%data% ((f2cl-lib:int-sub n 1)) ((1 *))
                        delta-%offset%)))
(setf dtnsq
        (* (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
```

```
           (f2cl-lib:fref delta-%data% (n) ((1 *)) delta-%offset%)))
(setf c (- w (* dtnsq1 dpsi) (* dtnsq dphi)))
(setf a
        (+ (* (+ dtnsq dtnsq1) w)
           (* (- dtnsq) dtnsq1 (+ dpsi dphi))))
(setf b (* dtnsq dtnsq1 w))
(if (< c zero)
    (setf c (abs c)))
(cond ((= c zero) (setf eta (- rho (* sigma sigma))))
      ((>= a zero)
       (setf eta
               (/
                (+ a
                   (f2cl-lib:fsqrt
                    (abs (+ (* a a) (* (- four) b c)))))
                (* two c))))
      (t
       (setf eta
               (/ (* two b)
                  (- a
                     (f2cl-lib:fsqrt
                      (abs (+ (* a a) (* (- four) b c)))))))))
(if (> (* w eta) zero)
    (setf eta (/ (- w) (+ dpsi dphi))))
(setf temp (- eta dtnsq))
(if (> temp rho)
    (setf eta (+ rho dtnsq)))
(setf eta (/ eta (+ sigma (f2cl-lib:fsqrt (+ eta (* sigma sigma))))))
(setf tau (+ tau eta)) (setf sigma (+ sigma eta))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
            (-
             (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
             eta))
    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
            (+ (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
               eta))
   label50))
(setf dpsi zero) (setf psi zero) (setf erretm zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j ii) nil)
  (tagbody
    (setf temp
            (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
               (*
                (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                (f2cl-lib:fref delta-%data% (j) ((1 *))
                               delta-%offset%))))
```

```
    (setf psi
            (+ psi
               (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                  temp)))
    (setf dpsi (+ dpsi (* temp temp)))
    (setf erretm (+ erretm psi))
   label60))
(setf erretm (abs erretm))
(setf tau2
        (* (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
           (f2cl-lib:fref delta-%data% (n) ((1 *)) delta-%offset%)))
(setf temp (/ (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%) tau2))
(setf phi (* (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%) temp))
(setf dphi (* temp temp))
(setf erretm (+ (- (+ (* eight (- (- psi) phi)) erretm) phi) rhoinv))
(setf w (+ rhoinv phi psi)) (setf iter (f2cl-lib:int-add niter 1))
(f2cl-lib:fdo (niter iter (f2cl-lib:int-add niter 1))
              ((> niter maxit) nil)
  (tagbody
    (cond ((<= (abs w) (* eps erretm)) (go label240)))
    (setf dtnsq1
            (*
             (f2cl-lib:fref work-%data% ((f2cl-lib:int-sub n 1))
                            ((1 *)) work-%offset%)
             (f2cl-lib:fref delta-%data% ((f2cl-lib:int-sub n 1))
                            ((1 *)) delta-%offset%)))
    (setf dtnsq
            (* (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
               (f2cl-lib:fref delta-%data% (n) ((1 *))
                              delta-%offset%)))
    (setf c (- w (* dtnsq1 dpsi) (* dtnsq dphi)))
    (setf a
            (+ (* (+ dtnsq dtnsq1) w)
               (* (- dtnsq1) dtnsq (+ dpsi dphi))))
    (setf b (* dtnsq1 dtnsq w))
    (cond
     ((>= a zero)
      (setf eta
              (/
               (+ a
                  (f2cl-lib:fsqrt
                   (abs (+ (* a a) (* (- four) b c)))))
               (* two c))))
     (t
      (setf eta
              (/ (* two b)
                 (- a
                    (f2cl-lib:fsqrt
                     (abs (+ (* a a) (* (- four) b c)))))))))
    (if (> (* w eta) zero)
```

```
    (setf eta (/ (- w) (+ dpsi dphi))))
(setf temp (- eta dtnsq))
(if (<= temp zero)
    (setf eta (/ eta two)))
(setf eta
        (/ eta
           (+ sigma (f2cl-lib:fsqrt (+ eta (* sigma sigma))))))
(setf tau (+ tau eta))
(setf sigma (+ sigma eta))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
            (-
             (f2cl-lib:fref delta-%data% (j) ((1 *))
                            delta-%offset%)
             eta))
    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
            (+
             (f2cl-lib:fref work-%data% (j) ((1 *))
                            work-%offset%)
             eta))
   label70))
(setf dpsi zero)
(setf psi zero)
(setf erretm zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j ii) nil)
  (tagbody
    (setf temp
            (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
               (*
                (f2cl-lib:fref work-%data% (j) ((1 *))
                               work-%offset%)
                (f2cl-lib:fref delta-%data% (j) ((1 *))
                               delta-%offset%))))
    (setf psi
            (+ psi
               (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                  temp)))
    (setf dpsi (+ dpsi (* temp temp)))
    (setf erretm (+ erretm psi))
   label80))
(setf erretm (abs erretm))
(setf tau2
        (* (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
           (f2cl-lib:fref delta-%data% (n) ((1 *))
                          delta-%offset%)))
(setf temp
        (/ (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%) tau2))
```

```
      (setf phi
              (* (f2cl-lib:fref z-%data% (n) ((1 *)) z-%offset%) temp))
      (setf dphi (* temp temp))
      (setf erretm
              (+ (- (+ (* eight (- (- psi) phi)) erretm) phi) rhoinv))
      (setf w (+ rhoinv phi psi))
     label90))
  (setf info 1) (go label240))
 (t (setf niter 1) (setf ip1 (f2cl-lib:int-add i 1))
  (setf delsq
          (*
           (- (f2cl-lib:fref d-%data% (ip1) ((1 *)) d-%offset%)
              (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
           (+ (f2cl-lib:fref d-%data% (ip1) ((1 *)) d-%offset%)
              (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))))
  (setf delsq2 (/ delsq two))
  (setf sq2
          (f2cl-lib:fsqrt
           (/
            (+
             (* (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
             (* (f2cl-lib:fref d-%data% (ip1) ((1 *)) d-%offset%)
                (f2cl-lib:fref d-%data% (ip1) ((1 *)) d-%offset%)))
            two)))
  (setf temp
          (/ delsq2
             (+ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) sq2)))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
              (+ (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                 (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) temp))
      (setf (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
              (- (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                 (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) temp))
     label100))
  (setf psi zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j (f2cl-lib:int-add i (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (setf psi
              (+ psi
                 (/
                  (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                     (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%))
                  (*
                   (f2cl-lib:fref work-%data% (j) ((1 *))
                                  work-%offset%)
```

```
                 (f2cl-lib:fref delta-%data% (j) ((1 *))
                                delta-%offset%)))))
   label110))
(setf phi zero)
(f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j (f2cl-lib:int-add i 2)) nil)
  (tagbody
    (setf phi
            (+ phi
               (/
                (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                   (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%))
                (*
                 (f2cl-lib:fref work-%data% (j) ((1 *))
                                work-%offset%)
                 (f2cl-lib:fref delta-%data% (j) ((1 *))
                                delta-%offset%)))))
   label120))
(setf c (+ rhoinv psi phi))
(setf w
        (+ c
           (/
            (* (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
               (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%))
            (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
               (f2cl-lib:fref delta-%data% (i) ((1 *))
                              delta-%offset%)))
           (/
            (* (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%)
               (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%))
            (*
             (f2cl-lib:fref work-%data% (ip1) ((1 *)) work-%offset%)
             (f2cl-lib:fref delta-%data% (ip1) ((1 *))
                            delta-%offset%)))))
(setf geomavg f2cl-lib:%false%)
(cond
 ((> w zero) (setf orgati f2cl-lib:%true%) (setf ii i)
  (setf sglb zero)
  (setf sgub
          (/ delsq2
             (+ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                sq2)))
  (setf a
          (+ (* c delsq)
             (* (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%))
             (* (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%)
                (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%))))
  (setf b
          (* (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
```

```
            (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%) delsq))
  (cond
   ((> a zero)
    (setf tau2
            (/ (* two b)
               (+ a
                  (f2cl-lib:fsqrt
                   (abs (+ (* a a) (* (- four) b c))))))))
   (t
    (setf tau2
            (/
             (- a
                (f2cl-lib:fsqrt (abs (+ (* a a) (* (- four) b c)))))
             (* two c)))))
  (setf tau
          (/ tau2
             (+ (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                (f2cl-lib:fsqrt
                 (+
                  (* (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                     (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
                  tau2)))))
  (setf temp (f2cl-lib:fsqrt eps))
  (cond
   ((and
     (<= (f2cl-lib:fref d (i) ((1 *)))
         (* temp (f2cl-lib:fref d (ip1) ((1 *)))))
     (<= (abs (f2cl-lib:fref z (i) ((1 *)))) temp)
     (> (f2cl-lib:fref d (i) ((1 *))) zero))
    (setf tau
            (min
             (* ten (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
             sgub))
    (setf geomavg f2cl-lib:%true%))))
 (t (setf orgati f2cl-lib:%false%) (setf ii ip1)
  (setf sglb
          (/ (- delsq2)
             (+ (f2cl-lib:fref d-%data% (ii) ((1 *)) d-%offset%)
                sq2)))
  (setf sgub zero)
  (setf a
          (- (* c delsq)
             (* (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%))
             (* (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%)
                (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%))))
  (setf b
          (* (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%)
             (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%)
             delsq))
```

```
  (cond
   ((< a zero)
    (setf tau2
            (/ (* two b)
               (- a
                  (f2cl-lib:fsqrt (abs (+ (* a a) (* four b c))))))))
   (t
    (setf tau2
            (/
             (-
              (+ a (f2cl-lib:fsqrt (abs (+ (* a a) (* four b c))))))
             (* two c)))))
  (setf tau
          (/ tau2
             (+ (f2cl-lib:fref d-%data% (ip1) ((1 *)) d-%offset%)
                (f2cl-lib:fsqrt
                 (abs
                  (+
                   (*
                    (f2cl-lib:fref d-%data% (ip1) ((1 *)) d-%offset%)
                    (f2cl-lib:fref d-%data% (ip1) ((1 *))
                                   d-%offset%))
                   tau2))))))))
(setf sigma (+ (f2cl-lib:fref d-%data% (ii) ((1 *)) d-%offset%) tau))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
            (+ (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
               (f2cl-lib:fref d-%data% (ii) ((1 *)) d-%offset%) tau))
    (setf (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
            (- (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
               (f2cl-lib:fref d-%data% (ii) ((1 *)) d-%offset%) tau))
   label130))
(setf iim1 (f2cl-lib:int-sub ii 1))
(setf iip1 (f2cl-lib:int-add ii 1)) (setf dpsi zero) (setf psi zero)
(setf erretm zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j iim1) nil)
  (tagbody
    (setf temp
            (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
               (*
                (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                (f2cl-lib:fref delta-%data% (j) ((1 *))
                               delta-%offset%))))
    (setf psi
            (+ psi
               (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                  temp)))
```

```
    (setf dpsi (+ dpsi (* temp temp)))
    (setf erretm (+ erretm psi))
   label150))
(setf erretm (abs erretm)) (setf dphi zero) (setf phi zero)
(f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j iip1) nil)
  (tagbody
    (setf temp
            (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
               (*
                (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                (f2cl-lib:fref delta-%data% (j) ((1 *))
                               delta-%offset%))))
    (setf phi
            (+ phi
               (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                  temp)))
    (setf dphi (+ dphi (* temp temp)))
    (setf erretm (+ erretm phi))
   label160))
(setf w (+ rhoinv phi psi)) (setf swtch3 f2cl-lib:%false%)
(cond
 (orgati
  (if (< w zero)
      (setf swtch3 f2cl-lib:%true%)))
 (t
  (if (> w zero)
      (setf swtch3 f2cl-lib:%true%))))
(if (or (= ii 1) (= ii n))
    (setf swtch3 f2cl-lib:%false%))
(setf temp
        (/ (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%)
           (* (f2cl-lib:fref work-%data% (ii) ((1 *)) work-%offset%)
              (f2cl-lib:fref delta-%data% (ii) ((1 *))
                             delta-%offset%))))
(setf dw (+ dpsi dphi (* temp temp)))
(setf temp (* (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%) temp))
(setf w (+ w temp))
(setf erretm
        (+ (* eight (- phi psi)) erretm (* two rhoinv)
           (* three (abs temp))))
(cond ((<= (abs w) (* eps erretm)) (go label240)))
(cond ((<= w zero) (setf sglb (max sglb tau)))
      (t (setf sgub (min sgub tau))))
(setf niter (f2cl-lib:int-add niter 1))
(cond
 ((not swtch3)
  (setf dtipsq
          (* (f2cl-lib:fref work-%data% (ip1) ((1 *)) work-%offset%)
             (f2cl-lib:fref delta-%data% (ip1) ((1 *))
```

```
                          delta-%offset%)))
(setf dtisq
        (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
           (f2cl-lib:fref delta-%data% (i) ((1 *))
                          delta-%offset%)))
(cond
 (orgati
  (setf c
          (+ (- w (* dtipsq dw))
             (* delsq
                (expt
                 (/ (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                    dtisq)
                 2)))))
 (t
  (setf c
          (- w (* dtisq dw)
             (* delsq
                (expt
                 (/
                  (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%)
                  dtipsq)
                 2))))))
(setf a (+ (* (+ dtipsq dtisq) w) (* (- dtipsq) dtisq dw)))
(setf b (* dtipsq dtisq w))
(cond
 ((= c zero)
  (cond
   ((= a zero)
    (cond
     (orgati
      (setf a
              (+
               (* (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                  (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%))
               (* dtipsq dtipsq (+ dpsi dphi)))))
     (t
      (setf a
              (+
               (* (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%)
                  (f2cl-lib:fref z-%data% (ip1) ((1 *))
                                 z-%offset%))
               (* dtisq dtisq (+ dpsi dphi))))))))
  (setf eta (/ b a)))
 ((<= a zero)
  (setf eta
          (/
           (- a
              (f2cl-lib:fsqrt (abs (+ (* a a) (* (- four) b c)))))
           (* two c))))
```

```
  (t
   (setf eta
           (/ (* two b)
              (+ a
                 (f2cl-lib:fsqrt
                  (abs (+ (* a a) (* (- four) b c)))))))))
 (t
  (setf dtiim
          (* (f2cl-lib:fref work-%data% (iim1) ((1 *)) work-%offset%)
             (f2cl-lib:fref delta-%data% (iim1) ((1 *))
                            delta-%offset%)))
  (setf dtiip
          (* (f2cl-lib:fref work-%data% (iip1) ((1 *)) work-%offset%)
             (f2cl-lib:fref delta-%data% (iip1) ((1 *))
                            delta-%offset%)))
  (setf temp (+ rhoinv psi phi))
  (cond
   (orgati
    (setf temp1
            (/ (f2cl-lib:fref z-%data% (iim1) ((1 *)) z-%offset%)
               dtiim))
    (setf temp1 (* temp1 temp1))
    (setf c
            (+ (- temp (* dtiip (+ dpsi dphi)))
               (*
                (-
                 (-
                  (f2cl-lib:fref d-%data% (iim1) ((1 *)) d-%offset%)
                  (f2cl-lib:fref d-%data% (iip1) ((1 *))
                                 d-%offset%)))
                (+ (f2cl-lib:fref d-%data% (iim1) ((1 *)) d-%offset%)
                   (f2cl-lib:fref d-%data% (iip1) ((1 *))
                                  d-%offset%))
                temp1)))
    (setf (f2cl-lib:fref zz (1) ((1 3)))
            (* (f2cl-lib:fref z-%data% (iim1) ((1 *)) z-%offset%)
               (f2cl-lib:fref z-%data% (iim1) ((1 *)) z-%offset%)))
    (cond
     ((< dpsi temp1)
      (setf (f2cl-lib:fref zz (3) ((1 3))) (* dtiip dtiip dphi)))
     (t
      (setf (f2cl-lib:fref zz (3) ((1 3)))
              (* dtiip dtiip (+ (- dpsi temp1) dphi))))))
   (t
    (setf temp1
            (/ (f2cl-lib:fref z-%data% (iip1) ((1 *)) z-%offset%)
               dtiip))
    (setf temp1 (* temp1 temp1))
    (setf c
            (+ (- temp (* dtiim (+ dpsi dphi)))
```

```
              (*
               (-
                (-
                 (f2cl-lib:fref d-%data% (iip1) ((1 *)) d-%offset%)
                 (f2cl-lib:fref d-%data% (iim1) ((1 *))
                                d-%offset%)))
               (+ (f2cl-lib:fref d-%data% (iim1) ((1 *)) d-%offset%)
                  (f2cl-lib:fref d-%data% (iip1) ((1 *))
                                 d-%offset%))
               temp1)))
  (cond
   ((< dphi temp1)
    (setf (f2cl-lib:fref zz (1) ((1 3))) (* dtiim dtiim dpsi)))
   (t
    (setf (f2cl-lib:fref zz (1) ((1 3)))
            (* dtiim dtiim (+ dpsi (- dphi temp1))))))
  (setf (f2cl-lib:fref zz (3) ((1 3)))
          (* (f2cl-lib:fref z-%data% (iip1) ((1 *)) z-%offset%)
             (f2cl-lib:fref z-%data% (iip1) ((1 *)) z-%offset%)))))
(setf (f2cl-lib:fref zz (2) ((1 3)))
        (* (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%)
           (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%)))
(setf (f2cl-lib:fref dd (1) ((1 3))) dtiim)
(setf (f2cl-lib:fref dd (2) ((1 3)))
        (* (f2cl-lib:fref delta-%data% (ii) ((1 *)) delta-%offset%)
           (f2cl-lib:fref work-%data% (ii) ((1 *)) work-%offset%)))
(setf (f2cl-lib:fref dd (3) ((1 3))) dtiip)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dlaed6 niter orgati c dd zz w eta info)
  (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5))
  (setf eta var-6)
  (setf info var-7))
(cond
 ((/= info 0) (setf swtch3 f2cl-lib:%false%) (setf info 0)
  (setf dtipsq
          (*
           (f2cl-lib:fref work-%data% (ip1) ((1 *)) work-%offset%)
           (f2cl-lib:fref delta-%data% (ip1) ((1 *))
                          delta-%offset%)))
  (setf dtisq
          (* (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
             (f2cl-lib:fref delta-%data% (i) ((1 *))
                            delta-%offset%)))
  (cond
   (orgati
    (setf c
            (+ (- w (* dtipsq dw))
               (* delsq
                  (expt
```

```
                 (/
                  (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                  dtisq)
                 2)))))
 (t
  (setf c
          (- w (* dtisq dw)
             (* delsq
                (expt
                 (/
                  (f2cl-lib:fref z-%data% (ip1) ((1 *))
                                 z-%offset%)
                  dtipsq)
                 2))))))
(setf a (+ (* (+ dtipsq dtisq) w) (* (- dtipsq) dtisq dw)))
(setf b (* dtipsq dtisq w))
(cond
 ((= c zero)
  (cond
   ((= a zero)
    (cond
     (orgati
      (setf a
              (+
               (* (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                  (f2cl-lib:fref z-%data% (i) ((1 *))
                                 z-%offset%))
               (* dtipsq dtipsq (+ dpsi dphi)))))
     (t
      (setf a
              (+
               (*
                (f2cl-lib:fref z-%data% (ip1) ((1 *)) z-%offset%)
                (f2cl-lib:fref z-%data% (ip1) ((1 *))
                               z-%offset%))
               (* dtisq dtisq (+ dpsi dphi))))))))
  (setf eta (/ b a)))
 ((<= a zero)
  (setf eta
          (/
           (- a
              (f2cl-lib:fsqrt
               (abs (+ (* a a) (* (- four) b c)))))
           (* two c))))
 (t
  (setf eta
          (/ (* two b)
             (+ a
                (f2cl-lib:fsqrt
                 (abs (+ (* a a) (* (- four) b c)))))))))))))
```

```
(if (>= (* w eta) zero)
    (setf eta (/ (- w) dw)))
(setf eta (/ eta (+ sigma (f2cl-lib:fsqrt (+ (* sigma sigma) eta)))))
(setf temp (+ tau eta))
(cond
 ((or (> temp sgub) (< temp sglb))
  (cond ((< w zero) (setf eta (/ (- sgub tau) two)))
        (t (setf eta (/ (- sglb tau) two))))
  (cond
   (geomavg
    (cond
     ((< w zero)
      (cond
       ((> tau zero)
        (setf eta (- (f2cl-lib:fsqrt (* sgub tau)) tau)))))
     (t
      (cond
       ((> sglb zero)
        (setf eta (- (f2cl-lib:fsqrt (* sglb tau)) tau)))))))))
(setf prew w) (setf tau (+ tau eta)) (setf sigma (+ sigma eta))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
            (+ (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
               eta))
    (setf (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
            (-
             (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
             eta))
   label170))
(setf dpsi zero) (setf psi zero) (setf erretm zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j iim1) nil)
  (tagbody
    (setf temp
            (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
               (*
                (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                (f2cl-lib:fref delta-%data% (j) ((1 *))
                               delta-%offset%))))
    (setf psi
            (+ psi
               (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                  temp)))
    (setf dpsi (+ dpsi (* temp temp)))
    (setf erretm (+ erretm psi))
   label180))
(setf erretm (abs erretm)) (setf dphi zero) (setf phi zero)
(f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
```

```
              ((> j iip1) nil)
  (tagbody
    (setf temp
            (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
               (*
                (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                (f2cl-lib:fref delta-%data% (j) ((1 *))
                               delta-%offset%))))
    (setf phi
            (+ phi
               (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                  temp)))
    (setf dphi (+ dphi (* temp temp)))
    (setf erretm (+ erretm phi))
   label190))
(setf tau2
        (* (f2cl-lib:fref work-%data% (ii) ((1 *)) work-%offset%)
           (f2cl-lib:fref delta-%data% (ii) ((1 *)) delta-%offset%)))
(setf temp (/ (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%) tau2))
(setf dw (+ dpsi dphi (* temp temp)))
(setf temp (* (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%) temp))
(setf w (+ rhoinv phi psi temp))
(setf erretm
        (+ (* eight (- phi psi)) erretm (* two rhoinv)
           (* three (abs temp))))
(setf swtch f2cl-lib:%false%)
(cond
 (orgati
  (if (> (- w) (/ (abs prew) ten))
      (setf swtch f2cl-lib:%true%)))
 (t
  (if (> w (/ (abs prew) ten))
      (setf swtch f2cl-lib:%true%))))
(setf iter (f2cl-lib:int-add niter 1))
(f2cl-lib:fdo (niter iter (f2cl-lib:int-add niter 1))
              ((> niter maxit) nil)
  (tagbody
    (cond ((<= (abs w) (* eps erretm)) (go label240)))
    (cond ((<= w zero) (setf sglb (max sglb tau)))
          (t (setf sgub (min sgub tau))))
    (cond
     ((not swtch3)
      (setf dtipsq
              (*
               (f2cl-lib:fref work-%data% (ip1) ((1 *))
                              work-%offset%)
               (f2cl-lib:fref delta-%data% (ip1) ((1 *))
                              delta-%offset%)))
      (setf dtisq
              (*
```

```
          (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
          (f2cl-lib:fref delta-%data% (i) ((1 *))
                         delta-%offset%)))
(cond
 ((not swtch)
  (cond
   (orgati
    (setf c
            (+ (- w (* dtipsq dw))
               (* delsq
                  (expt
                   (/
                    (f2cl-lib:fref z-%data% (i) ((1 *))
                                   z-%offset%)
                    dtisq)
                   2)))))
   (t
    (setf c
            (- w (* dtisq dw)
               (* delsq
                  (expt
                   (/
                    (f2cl-lib:fref z-%data% (ip1) ((1 *))
                                   z-%offset%)
                    dtipsq)
                   2)))))))
 (t
  (setf temp
          (/ (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%)
             (*
              (f2cl-lib:fref work-%data% (ii) ((1 *))
                             work-%offset%)
              (f2cl-lib:fref delta-%data% (ii) ((1 *))
                             delta-%offset%))))
  (cond (orgati (setf dpsi (+ dpsi (* temp temp))))
        (t (setf dphi (+ dphi (* temp temp)))))
  (setf c (- w (* dtisq dpsi) (* dtipsq dphi)))))
(setf a (+ (* (+ dtipsq dtisq) w) (* (- dtipsq) dtisq dw)))
(setf b (* dtipsq dtisq w))
(cond
 ((= c zero)
  (cond
   ((= a zero)
    (cond
     ((not swtch)
      (cond
       (orgati
        (setf a
                (+
                 (*
```

```
                    (f2cl-lib:fref z-%data% (i) ((1 *))
                                   z-%offset%)
                    (f2cl-lib:fref z-%data% (i) ((1 *))
                                   z-%offset%))
                   (* dtipsq dtipsq (+ dpsi dphi)))))
          (t
           (setf a
                  (+
                   (*
                    (f2cl-lib:fref z-%data% (ip1) ((1 *))
                                   z-%offset%)
                    (f2cl-lib:fref z-%data% (ip1) ((1 *))
                                   z-%offset%))
                   (* dtisq dtisq (+ dpsi dphi)))))))
        (t
         (setf a
                 (+ (* dtisq dtisq dpsi)
                    (* dtipsq dtipsq dphi)))))))
    (setf eta (/ b a)))
   ((<= a zero)
    (setf eta
            (/
             (- a
                (f2cl-lib:fsqrt
                 (abs (+ (* a a) (* (- four) b c)))))
             (* two c))))
   (t
    (setf eta
            (/ (* two b)
               (+ a
                  (f2cl-lib:fsqrt
                   (abs (+ (* a a) (* (- four) b c)))))))))
  (t
   (setf dtiim
           (*
            (f2cl-lib:fref work-%data% (iim1) ((1 *))
                           work-%offset%)
            (f2cl-lib:fref delta-%data% (iim1) ((1 *))
                           delta-%offset%)))
   (setf dtiip
           (*
            (f2cl-lib:fref work-%data% (iip1) ((1 *))
                           work-%offset%)
            (f2cl-lib:fref delta-%data% (iip1) ((1 *))
                           delta-%offset%)))
   (setf temp (+ rhoinv psi phi))
   (cond
    (swtch (setf c (- temp (* dtiim dpsi) (* dtiip dphi)))
     (setf (f2cl-lib:fref zz (1) ((1 3))) (* dtiim dtiim dpsi))
     (setf (f2cl-lib:fref zz (3) ((1 3))) (* dtiip dtiip dphi)))
```

```
(t
 (cond
  (orgati
   (setf temp1
           (/
            (f2cl-lib:fref z-%data% (iim1) ((1 *)) z-%offset%)
            dtiim))
   (setf temp1 (* temp1 temp1))
   (setf temp2
           (*
            (-
             (f2cl-lib:fref d-%data% (iim1) ((1 *))
                            d-%offset%)
             (f2cl-lib:fref d-%data% (iip1) ((1 *))
                            d-%offset%))
            (+
             (f2cl-lib:fref d-%data% (iim1) ((1 *))
                            d-%offset%)
             (f2cl-lib:fref d-%data% (iip1) ((1 *))
                            d-%offset%))
            temp1))
   (setf c (- temp (* dtiip (+ dpsi dphi)) temp2))
   (setf (f2cl-lib:fref zz (1) ((1 3)))
           (*
            (f2cl-lib:fref z-%data% (iim1) ((1 *)) z-%offset%)
            (f2cl-lib:fref z-%data% (iim1) ((1 *))
                           z-%offset%)))
   (cond
    ((< dpsi temp1)
     (setf (f2cl-lib:fref zz (3) ((1 3)))
             (* dtiip dtiip dphi)))
    (t
     (setf (f2cl-lib:fref zz (3) ((1 3)))
             (* dtiip dtiip (+ (- dpsi temp1) dphi))))))
  (t
   (setf temp1
           (/
            (f2cl-lib:fref z-%data% (iip1) ((1 *)) z-%offset%)
            dtiip))
   (setf temp1 (* temp1 temp1))
   (setf temp2
           (*
            (-
             (f2cl-lib:fref d-%data% (iip1) ((1 *))
                            d-%offset%)
             (f2cl-lib:fref d-%data% (iim1) ((1 *))
                            d-%offset%))
            (+
             (f2cl-lib:fref d-%data% (iim1) ((1 *))
                            d-%offset%)
```

```
                (f2cl-lib:fref d-%data% (iip1) ((1 *))
                               d-%offset%))
              temp1))
    (setf c (- temp (* dtiim (+ dpsi dphi)) temp2))
    (cond
     ((< dphi temp1)
      (setf (f2cl-lib:fref zz (1) ((1 3)))
              (* dtiim dtiim dpsi)))
     (t
      (setf (f2cl-lib:fref zz (1) ((1 3)))
              (* dtiim dtiim (+ dpsi (- dphi temp1))))))
    (setf (f2cl-lib:fref zz (3) ((1 3)))
            (*
             (f2cl-lib:fref z-%data% (iip1) ((1 *)) z-%offset%)
             (f2cl-lib:fref z-%data% (iip1) ((1 *))
                            z-%offset%)))))))
(setf (f2cl-lib:fref dd (1) ((1 3))) dtiim)
(setf (f2cl-lib:fref dd (2) ((1 3)))
        (*
         (f2cl-lib:fref delta-%data% (ii) ((1 *))
                        delta-%offset%)
         (f2cl-lib:fref work-%data% (ii) ((1 *))
                        work-%offset%)))
(setf (f2cl-lib:fref dd (3) ((1 3))) dtiip)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (dlaed6 niter orgati c dd zz w eta info)
  (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5))
  (setf eta var-6)
  (setf info var-7))
(cond
 ((/= info 0) (setf swtch3 f2cl-lib:%false%) (setf info 0)
  (setf dtipsq
          (*
           (f2cl-lib:fref work-%data% (ip1) ((1 *))
                          work-%offset%)
           (f2cl-lib:fref delta-%data% (ip1) ((1 *))
                          delta-%offset%)))
  (setf dtisq
          (*
           (f2cl-lib:fref work-%data% (i) ((1 *))
                          work-%offset%)
           (f2cl-lib:fref delta-%data% (i) ((1 *))
                          delta-%offset%)))
  (cond
   ((not swtch)
    (cond
     (orgati
      (setf c
              (+ (- w (* dtipsq dw))
```

```
                (* delsq
                   (expt
                    (/
                     (f2cl-lib:fref z-%data% (i) ((1 *))
                                    z-%offset%)
                     dtisq)
                    2)))))
   (t
    (setf c
            (- w (* dtisq dw)
                (* delsq
                   (expt
                    (/
                     (f2cl-lib:fref z-%data% (ip1) ((1 *))
                                    z-%offset%)
                     dtipsq)
                    2)))))))
 (t
  (setf temp
          (/ (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%)
             (*
              (f2cl-lib:fref work-%data% (ii) ((1 *))
                             work-%offset%)
              (f2cl-lib:fref delta-%data% (ii) ((1 *))
                             delta-%offset%))))
  (cond (orgati (setf dpsi (+ dpsi (* temp temp))))
        (t (setf dphi (+ dphi (* temp temp)))))
  (setf c (- w (* dtisq dpsi) (* dtipsq dphi)))))
(setf a (+ (* (+ dtipsq dtisq) w) (* (- dtipsq) dtisq dw)))
(setf b (* dtipsq dtisq w))
(cond
 ((= c zero)
  (cond
   ((= a zero)
    (cond
     ((not swtch)
      (cond
       (orgati
        (setf a
                (+
                 (*
                  (f2cl-lib:fref z-%data% (i) ((1 *))
                                 z-%offset%)
                  (f2cl-lib:fref z-%data% (i) ((1 *))
                                 z-%offset%))
                 (* dtipsq dtipsq (+ dpsi dphi)))))
       (t
        (setf a
                (+
                 (*
```

```
                        (f2cl-lib:fref z-%data% (ip1) ((1 *))
                                       z-%offset%)
                        (f2cl-lib:fref z-%data% (ip1) ((1 *))
                                       z-%offset%))
                       (* dtisq dtisq (+ dpsi dphi)))))))
          (t
           (setf a
                   (+ (* dtisq dtisq dpsi)
                      (* dtipsq dtipsq dphi)))))))
      (setf eta (/ b a)))
     ((<= a zero)
      (setf eta
              (/
               (- a
                  (f2cl-lib:fsqrt
                   (abs (+ (* a a) (* (- four) b c)))))
               (* two c))))
     (t
      (setf eta
              (/ (* two b)
                 (+ a
                    (f2cl-lib:fsqrt
                     (abs (+ (* a a) (* (- four) b c))))))))))))
(if (>= (* w eta) zero)
    (setf eta (/ (- w) dw)))
(setf eta
        (/ eta
           (+ sigma (f2cl-lib:fsqrt (+ (* sigma sigma) eta)))))
(setf temp (+ tau eta))
(cond
 ((or (> temp sgub) (< temp sglb))
  (cond ((< w zero) (setf eta (/ (- sgub tau) two)))
        (t (setf eta (/ (- sglb tau) two))))
  (cond
   (geomavg
    (cond
     ((< w zero)
      (cond
       ((> tau zero)
        (setf eta (- (f2cl-lib:fsqrt (* sgub tau)) tau)))))
     (t
      (cond
       ((> sglb zero)
        (setf eta (- (f2cl-lib:fsqrt (* sglb tau)) tau)))))))))
(setf prew w)
(setf tau (+ tau eta))
(setf sigma (+ sigma eta))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
```

```
    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
            (+
             (f2cl-lib:fref work-%data% (j) ((1 *))
                            work-%offset%)
             eta))
    (setf (f2cl-lib:fref delta-%data% (j) ((1 *)) delta-%offset%)
            (-
             (f2cl-lib:fref delta-%data% (j) ((1 *))
                            delta-%offset%)
             eta))
   label200))
(setf dpsi zero)
(setf psi zero)
(setf erretm zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j iim1) nil)
  (tagbody
    (setf temp
            (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
               (*
                (f2cl-lib:fref work-%data% (j) ((1 *))
                               work-%offset%)
                (f2cl-lib:fref delta-%data% (j) ((1 *))
                               delta-%offset%))))
    (setf psi
            (+ psi
               (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                  temp)))
    (setf dpsi (+ dpsi (* temp temp)))
    (setf erretm (+ erretm psi))
   label210))
(setf erretm (abs erretm))
(setf dphi zero)
(setf phi zero)
(f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j iip1) nil)
  (tagbody
    (setf temp
            (/ (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
               (*
                (f2cl-lib:fref work-%data% (j) ((1 *))
                               work-%offset%)
                (f2cl-lib:fref delta-%data% (j) ((1 *))
                               delta-%offset%))))
    (setf phi
            (+ phi
               (* (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%)
                  temp)))
    (setf dphi (+ dphi (* temp temp)))
    (setf erretm (+ erretm phi))
```

```
               label220))
            (setf tau2
                    (* (f2cl-lib:fref work-%data% (ii) ((1 *)) work-%offset%)
                       (f2cl-lib:fref delta-%data% (ii) ((1 *))
                                      delta-%offset%)))
            (setf temp
                    (/ (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%)
                       tau2))
            (setf dw (+ dpsi dphi (* temp temp)))
            (setf temp
                    (* (f2cl-lib:fref z-%data% (ii) ((1 *)) z-%offset%)
                       temp))
            (setf w (+ rhoinv phi psi temp))
            (setf erretm
                    (+ (* eight (- phi psi)) erretm (* two rhoinv)
                       (* three (abs temp))))
            (if (and (> (* w prew) zero) (> (abs w) (/ (abs prew) ten)))
                (setf swtch (not swtch)))
          label230))
      (setf info 1)))
  label240
   (go end_label)
  end_label
   (return (values nil i nil nil nil rho sigma nil info))))))
```

———-

90.174 dlasd5

90.174.1 Axiom unit tests

— **dlasd5.input** —

```
)set break resume
)sys rm -f dlasd5.output
)spool dlasd5.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.174.2 Axiom help page

— dlasd5.help —

```
====================================================================
dlasd5 examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DLASD5  -  compute the square root of the I-th eigenvalue of a positive
       symmetric rank-one modification of a 2-by-2 diagonal matrix   diag( D )
       *  diag(  D ) + RHO  The diagonal entries in the array D are assumed to
       satisfy   0 <= D(i) < D(j) for i < j

SYNOPSIS
       SUBROUTINE DLASD5( I, D, Z, DELTA, RHO, DSIGMA, WORK )

           INTEGER        I

           DOUBLE         PRECISION DSIGMA, RHO

           DOUBLE         PRECISION D( 2 ), DELTA( 2 ), WORK( 2 ), Z( 2 )

  Purpose
  =======

  This subroutine computes the square root of the I-th eigenvalue
  of a positive symmetric rank-one modification of a 2-by-2 diagonal
  matrix

             diag( D ) * diag( D ) +  RHO *  Z * transpose(Z) .

  The diagonal entries in the array D are assumed to satisfy

             0 <= D(i) < D(j)  for  i < j .

  We also assume RHO > 0 and that the Euclidean norm of the vector
  Z is one.

  Arguments
  =========

  I      (input) INTEGER
         The index of the eigenvalue to be computed.  I = 1 or I = 2.
```

```
D      (input) DOUBLE PRECISION array, dimension ( 2 )
       The original eigenvalues.  We assume 0 <= D(1) < D(2).

Z      (input) DOUBLE PRECISION array, dimension ( 2 )
       The components of the updating vector.

DELTA  (output) DOUBLE PRECISION array, dimension ( 2 )
       Contains (D(j) - lambda_I) in its  j-th component.
       The vector DELTA contains the information necessary
       to construct the eigenvectors.

RHO    (input) DOUBLE PRECISION
       The scalar in the symmetric updating formula.

DSIGMA (output) DOUBLE PRECISION
       The computed lambda_I, the I-th updated eigenvalue.

WORK   (workspace) DOUBLE PRECISION array, dimension ( 2 )
       WORK contains (D(j) + sigma_I) in its  j-th component.

Further Details
===============

Based on contributions by
   Ren-Cang Li, Computer Science Division, University of California
   at Berkeley, USA
```

———-

90.174.3 fortran code

— **dlasd5.f** —

```
      SUBROUTINE DLASD5( I, D, Z, DELTA, RHO, DSIGMA, WORK )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Oak Ridge National Lab, Argonne National Lab,
*     Courant Institute, NAG Ltd., and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      INTEGER            I
      DOUBLE PRECISION   DSIGMA, RHO
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( 2 ), DELTA( 2 ), WORK( 2 ), Z( 2 )
*     ..
```

```
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE, TWO, THREE, FOUR
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0, TWO = 2.0D+0,
     $                   THREE = 3.0D+0, FOUR = 4.0D+0 )
*     ..
*     .. Local Scalars ..
      DOUBLE PRECISION   B, C, DEL, DELSQ, TAU, W
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SQRT
*     ..
*     .. Executable Statements ..
*
      DEL = D( 2 ) - D( 1 )
      DELSQ = DEL*( D( 2 )+D( 1 ) )
      IF( I.EQ.1 ) THEN
         W = ONE + FOUR*RHO*( Z( 2 )*Z( 2 ) / ( D( 1 )+THREE*D( 2 ) )-
     $       Z( 1 )*Z( 1 ) / ( THREE*D( 1 )+D( 2 ) ) ) / DEL
         IF( W.GT.ZERO ) THEN
            B = DELSQ + RHO*( Z( 1 )*Z( 1 )+Z( 2 )*Z( 2 ) )
            C = RHO*Z( 1 )*Z( 1 )*DELSQ
*
*           B > ZERO, always
*
*           The following TAU is DSIGMA * DSIGMA - D( 1 ) * D( 1 )
*
            TAU = TWO*C / ( B+SQRT( ABS( B*B-FOUR*C ) ) )
*
*           The following TAU is DSIGMA - D( 1 )
*
            TAU = TAU / ( D( 1 )+SQRT( D( 1 )*D( 1 )+TAU ) )
            DSIGMA = D( 1 ) + TAU
            DELTA( 1 ) = -TAU
            DELTA( 2 ) = DEL - TAU
            WORK( 1 ) = TWO*D( 1 ) + TAU
            WORK( 2 ) = ( D( 1 )+TAU ) + D( 2 )
*           DELTA( 1 ) = -Z( 1 ) / TAU
*           DELTA( 2 ) = Z( 2 ) / ( DEL-TAU )
         ELSE
            B = -DELSQ + RHO*( Z( 1 )*Z( 1 )+Z( 2 )*Z( 2 ) )
            C = RHO*Z( 2 )*Z( 2 )*DELSQ
*
*           The following TAU is DSIGMA * DSIGMA - D( 2 ) * D( 2 )
*
            IF( B.GT.ZERO ) THEN
               TAU = -TWO*C / ( B+SQRT( B*B+FOUR*C ) )
            ELSE
```

```
               TAU = ( B-SQRT( B*B+FOUR*C ) ) / TWO
            END IF
*
*           The following TAU is DSIGMA - D( 2 )
*
            TAU = TAU / ( D( 2 )+SQRT( ABS( D( 2 )*D( 2 )+TAU ) ) )
            DSIGMA = D( 2 ) + TAU
            DELTA( 1 ) = -( DEL+TAU )
            DELTA( 2 ) = -TAU
            WORK( 1 ) = D( 1 ) + TAU + D( 2 )
            WORK( 2 ) = TWO*D( 2 ) + TAU
*           DELTA( 1 ) = -Z( 1 ) / ( DEL+TAU )
*           DELTA( 2 ) = -Z( 2 ) / TAU
         END IF
*        TEMP = SQRT( DELTA( 1 )*DELTA( 1 )+DELTA( 2 )*DELTA( 2 ) )
*        DELTA( 1 ) = DELTA( 1 ) / TEMP
*        DELTA( 2 ) = DELTA( 2 ) / TEMP
      ELSE
*
*        Now I=2
*
         B = -DELSQ + RHO*( Z( 1 )*Z( 1 )+Z( 2 )*Z( 2 ) )
         C = RHO*Z( 2 )*Z( 2 )*DELSQ
*
*        The following TAU is DSIGMA * DSIGMA - D( 2 ) * D( 2 )
*
         IF( B.GT.ZERO ) THEN
            TAU = ( B+SQRT( B*B+FOUR*C ) ) / TWO
         ELSE
            TAU = TWO*C / ( -B+SQRT( B*B+FOUR*C ) )
         END IF
*
*        The following TAU is DSIGMA - D( 2 )
*
         TAU = TAU / ( D( 2 )+SQRT( D( 2 )*D( 2 )+TAU ) )
         DSIGMA = D( 2 ) + TAU
         DELTA( 1 ) = -( DEL+TAU )
         DELTA( 2 ) = -TAU
         WORK( 1 ) = D( 1 ) + TAU + D( 2 )
         WORK( 2 ) = TWO*D( 2 ) + TAU
*        DELTA( 1 ) = -Z( 1 ) / ( DEL+TAU )
*        DELTA( 2 ) = -Z( 2 ) / TAU
*        TEMP = SQRT( DELTA( 1 )*DELTA( 1 )+DELTA( 2 )*DELTA( 2 ) )
*        DELTA( 1 ) = DELTA( 1 ) / TEMP
*        DELTA( 2 ) = DELTA( 2 ) / TEMP
      END IF
      RETURN
*
*     End of DLASD5
*
```

```
      END
```

———-

90.174.4 lisp code

— dlasd5.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0) (two 2.0d0) (three 3.0d0) (four 4.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 3.0d0 3.0d0) three)
           (type (double-float 4.0d0 4.0d0) four)
           (ignorable zero one two three four))
  (defun dlasd5 (i d z delta rho dsigma work)
    (declare (type (double-float) dsigma rho)
             (type (array double-float (*)) work delta z d)
             (type (f2cl-lib:integer4) i))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (z double-float z-%data% z-%offset%)
         (delta double-float delta-%data% delta-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((b 0.0d0) (c 0.0d0) (del 0.0d0) (delsq 0.0d0) (tau 0.0d0)
             (w 0.0d0))
        (declare (type (double-float) b c del delsq tau w))
        (setf del
                (- (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%)
                   (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%)))
        (setf delsq
                (* del
                   (+ (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%)
                      (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%))))
        (cond
         ((= i 1)
          (setf w
                  (+ one
                     (/
                      (* four rho
                         (+
                          (/
                           (* (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%)
                              (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%))
                           (+ (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%)
                              (* three
                                 (f2cl-lib:fref d-%data% (2) ((1 2))
```

```
                                        d-%offset%))))
                    (/
                     (*
                      (- (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%))
                      (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%))
                     (+
                      (* three
                         (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%))
                      (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%)))))
               del)))
  (cond
   ((> w zero)
    (setf b
            (+ delsq
               (* rho
                  (+
                   (* (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%)
                      (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%))
                   (* (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%)
                      (f2cl-lib:fref z-%data% (2) ((1 2))
                                     z-%offset%))))))
    (setf c
            (* rho (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%)
               (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%) delsq))
    (setf tau
            (/ (* two c)
               (+ b (f2cl-lib:fsqrt (abs (- (* b b) (* four c)))))))
    (setf tau
            (/ tau
               (+ (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%)
                  (f2cl-lib:fsqrt
                   (+
                    (* (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%)
                       (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%))
                    tau)))))
    (setf dsigma
            (+ (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%) tau))
    (setf (f2cl-lib:fref delta-%data% (1) ((1 2)) delta-%offset%)
            (- tau))
    (setf (f2cl-lib:fref delta-%data% (2) ((1 2)) delta-%offset%)
            (- del tau))
    (setf (f2cl-lib:fref work-%data% (1) ((1 2)) work-%offset%)
            (+ (* two (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%))
               tau))
    (setf (f2cl-lib:fref work-%data% (2) ((1 2)) work-%offset%)
            (+ (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%) tau
               (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%))))
   (t
    (setf b
            (-
```

```
                (* rho
                   (+
                    (* (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%)
                       (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%))
                    (* (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%)
                       (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%))))
                delsq))
        (setf c
                (* rho (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%)
                   (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%) delsq))
        (cond
         ((> b zero)
          (setf tau
                  (/ (* (- two) c)
                     (+ b (f2cl-lib:fsqrt (+ (* b b) (* four c)))))))
         (t
          (setf tau
                  (/ (- b (f2cl-lib:fsqrt (+ (* b b) (* four c)))) two))))
        (setf tau
                (/ tau
                   (+ (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%)
                      (f2cl-lib:fsqrt
                       (abs
                        (+
                         (* (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%)
                            (f2cl-lib:fref d-%data% (2) ((1 2))
                                           d-%offset%))
                         tau))))))
        (setf dsigma
                (+ (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%) tau))
        (setf (f2cl-lib:fref delta-%data% (1) ((1 2)) delta-%offset%)
                (- (+ del tau)))
        (setf (f2cl-lib:fref delta-%data% (2) ((1 2)) delta-%offset%)
                (- tau))
        (setf (f2cl-lib:fref work-%data% (1) ((1 2)) work-%offset%)
                (+ (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%) tau
                   (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%)))
        (setf (f2cl-lib:fref work-%data% (2) ((1 2)) work-%offset%)
                (+ (* two (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%))
                   tau)))))
(t
 (setf b
         (-
          (* rho
             (+
              (* (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%)
                 (f2cl-lib:fref z-%data% (1) ((1 2)) z-%offset%))
              (* (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%)
                 (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%))))
          delsq))
```

```
          (setf c
                  (* rho (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%)
                     (f2cl-lib:fref z-%data% (2) ((1 2)) z-%offset%) delsq))
          (cond
           ((> b zero)
            (setf tau (/ (+ b (f2cl-lib:fsqrt (+ (* b b) (* four c)))) two)))
           (t
            (setf tau
                    (/ (* two c)
                       (- (f2cl-lib:fsqrt (+ (* b b) (* four c))) b)))))
          (setf tau
                  (/ tau
                     (+ (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%)
                        (f2cl-lib:fsqrt
                         (+
                          (* (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%)
                             (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%))
                          tau)))))
          (setf dsigma (+ (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%) tau))
          (setf (f2cl-lib:fref delta-%data% (1) ((1 2)) delta-%offset%)
                  (- (+ del tau)))
          (setf (f2cl-lib:fref delta-%data% (2) ((1 2)) delta-%offset%)
                  (- tau))
          (setf (f2cl-lib:fref work-%data% (1) ((1 2)) work-%offset%)
                  (+ (f2cl-lib:fref d-%data% (1) ((1 2)) d-%offset%) tau
                     (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%)))
          (setf (f2cl-lib:fref work-%data% (2) ((1 2)) work-%offset%)
                  (+ (* two (f2cl-lib:fref d-%data% (2) ((1 2)) d-%offset%))
                     tau))))
        (go end_label)
       end_label
        (return (values nil nil nil nil nil dsigma nil))))))
```

———-

90.175 dlasd6

90.175.1 Axiom unit tests

— dlasd6.input —

```
)set break resume
)sys rm -f dlasd6.output
)spool dlasd6.output
)set message test on
)set message auto off
```

```
)clear all

)spool
)lisp (bye)
```

———-

90.175.2 Axiom help page

— dlasd6.help —

```
==================================================================
dlasd6 examples
==================================================================

==================================================================
Man Page Details
==================================================================

NAME
       DLASD6  -  the  SVD of an updated upper bidiagonal matrix B obtained by
       merging two smaller ones by appending a row

SYNOPSIS
       SUBROUTINE DLASD6( ICOMPQ, NL, NR, SQRE, D, VF, VL, ALPHA, BETA,  IDXQ,
                          PERM, GIVPTR, GIVCOL, LDGCOL, GIVNUM, LDGNUM, POLES,
                          DIFL, DIFR, Z, K, C, S, WORK, IWORK, INFO )

           INTEGER        GIVPTR, ICOMPQ, INFO, K,  LDGCOL,  LDGNUM,  NL,  NR,
                          SQRE

           DOUBLE         PRECISION ALPHA, BETA, C, S

           INTEGER        GIVCOL(  LDGCOL, * ), IDXQ( * ), IWORK( * ), PERM( *
                          )

           DOUBLE         PRECISION D( * ), DIFL( *  ),  DIFR(  *  ),  GIVNUM(
                          LDGNUM,  *  ), POLES( LDGNUM, * ), VF( * ), VL( * ),
                          WORK( * ), Z( * )

  Purpose
  =======

  DLASD6 computes the SVD of an updated upper bidiagonal matrix B
  obtained by merging two smaller ones by appending a row. This
  routine is used only for the problem which requires all singular
  values and optionally singular vector matrices in factored form.
  B is an N-by-M matrix with N = NL + NR + 1 and M = N + SQRE.
```

```
A related subroutine, DLASD1, handles the case in which all singular
values and singular vectors of the bidiagonal matrix are desired.

DLASD6 computes the SVD as follows:

              ( D1(in)  0    0     0 )
  B = U(in) * (   Z1'   a   Z2'    b ) * VT(in)
              (   0     0   D2(in) 0 )

    = U(out) * ( D(out) 0) * VT(out)

where Z' = (Z1' a Z2' b) = u' VT', and u is a vector of dimension M
with ALPHA and BETA in the NL+1 and NL+2 th entries and zeros
elsewhere; and the entry b is empty if SQRE = 0.

The singular values of B can be computed using D1, D2, the first
components of all the right singular vectors of the lower block, and
the last components of all the right singular vectors of the upper
block. These components are stored and updated in VF and VL,
respectively, in DLASD6. Hence U and VT are not explicitly
referenced.

The singular values are stored in D. The algorithm consists of two
stages:

      The first stage consists of deflating the size of the problem
      when there are multiple singular values or if there is a zero
      in the Z vector. For each such occurence the dimension of the
      secular equation problem is reduced by one. This stage is
      performed by the routine DLASD7.

      The second stage consists of calculating the updated
      singular values. This is done by finding the roots of the
      secular equation via the routine DLASD4 (as called by DLASD8).
      This routine also updates VF and VL and computes the distances
      between the updated singular values and the old singular
      values.

DLASD6 is called from DLASDA.

Arguments
=========

ICOMPQ (input) INTEGER
       Specifies whether singular vectors are to be computed in
       factored form:
       = 0: Compute singular values only.
       = 1: Compute singular vectors in factored form as well.

NL     (input) INTEGER
```

```
        The row dimension of the upper block.  NL >= 1.

NR      (input) INTEGER
        The row dimension of the lower block.  NR >= 1.

SQRE    (input) INTEGER
        = 0: the lower block is an NR-by-NR square matrix.
        = 1: the lower block is an NR-by-(NR+1) rectangular matrix.

        The bidiagonal matrix has row dimension N = NL + NR + 1,
        and column dimension M = N + SQRE.

D       (input/output) DOUBLE PRECISION array, dimension ( NL+NR+1 ).
        On entry D(1:NL,1:NL) contains the singular values of the
        upper block, and D(NL+2:N) contains the singular values
        of the lower block. On exit D(1:N) contains the singular
        values of the modified matrix.

VF      (input/output) DOUBLE PRECISION array, dimension ( M )
        On entry, VF(1:NL+1) contains the first components of all
        right singular vectors of the upper block; and VF(NL+2:M)
        contains the first components of all right singular vectors
        of the lower block. On exit, VF contains the first components
        of all right singular vectors of the bidiagonal matrix.

VL      (input/output) DOUBLE PRECISION array, dimension ( M )
        On entry, VL(1:NL+1) contains the  last components of all
        right singular vectors of the upper block; and VL(NL+2:M)
        contains the last components of all right singular vectors of
        the lower block. On exit, VL contains the last components of
        all right singular vectors of the bidiagonal matrix.

ALPHA   (input) DOUBLE PRECISION
        Contains the diagonal element associated with the added row.

BETA    (input) DOUBLE PRECISION
        Contains the off-diagonal element associated with the added
        row.

IDXQ    (output) INTEGER array, dimension ( N )
        This contains the permutation which will reintegrate the
        subproblem just solved back into sorted order, i.e.
        D( IDXQ( I = 1, N ) ) will be in ascending order.

PERM    (output) INTEGER array, dimension ( N )
        The permutations (from deflation and sorting) to be applied
        to each block. Not referenced if ICOMPQ = 0.

GIVPTR (output) INTEGER
        The number of Givens rotations which took place in this
```

```
       subproblem. Not referenced if ICOMPQ = 0.

GIVCOL (output) INTEGER array, dimension ( LDGCOL, 2 )
       Each pair of numbers indicates a pair of columns to take place
       in a Givens rotation. Not referenced if ICOMPQ = 0.

LDGCOL (input) INTEGER
       leading dimension of GIVCOL, must be at least N.

GIVNUM (output) DOUBLE PRECISION array, dimension ( LDGNUM, 2 )
       Each number indicates the C or S value to be used in the
       corresponding Givens rotation. Not referenced if ICOMPQ = 0.

LDGNUM (input) INTEGER
       The leading dimension of GIVNUM and POLES, must be at least N.

POLES  (output) DOUBLE PRECISION array, dimension ( LDGNUM, 2 )
       On exit, POLES(1,*) is an array containing the new singular
       values obtained from solving the secular equation, and
       POLES(2,*) is an array containing the poles in the secular
       equation. Not referenced if ICOMPQ = 0.

DIFL   (output) DOUBLE PRECISION array, dimension ( N )
       On exit, DIFL(I) is the distance between I-th updated
       (undeflated) singular value and the I-th (undeflated) old
       singular value.

DIFR   (output) DOUBLE PRECISION array,
                dimension ( LDGNUM, 2 ) if ICOMPQ = 1 and
                dimension ( N ) if ICOMPQ = 0.
       On exit, DIFR(I, 1) is the distance between I-th updated
       (undeflated) singular value and the I+1-th (undeflated) old
       singular value.

       If ICOMPQ = 1, DIFR(1:K,2) is an array containing the
       normalizing factors for the right singular vector matrix.

       See DLASD8 for details on DIFL and DIFR.

Z      (output) DOUBLE PRECISION array, dimension ( M )
       The first elements of this array contain the components
       of the deflation-adjusted updating row vector.

K      (output) INTEGER
       Contains the dimension of the non-deflated matrix,
       This is the order of the related secular equation. 1 <= K <=N.

C      (output) DOUBLE PRECISION
       C contains garbage if SQRE =0 and the C-value of a Givens
       rotation related to the right null space if SQRE = 1.
```

```
 S      (output) DOUBLE PRECISION
        S contains garbage if SQRE =0 and the S-value of a Givens
        rotation related to the right null space if SQRE = 1.

 WORK   (workspace) DOUBLE PRECISION array, dimension ( 4 * M )

 IWORK  (workspace) INTEGER array, dimension ( 3 * N )

 INFO   (output) INTEGER
         = 0:  successful exit.
         < 0:  if INFO = -i, the i-th argument had an illegal value.
         > 0:  if INFO = 1, an singular value did not converge

 Further Details
 ===============

 Based on contributions by
    Ming Gu and Huan Ren, Computer Science Division, University of
    California at Berkeley, USA
```

90.175.3 fortran code

— dlasd6.f —

```
      SUBROUTINE DLASD6( ICOMPQ, NL, NR, SQRE, D, VF, VL, ALPHA, BETA,
     $                   IDXQ, PERM, GIVPTR, GIVCOL, LDGCOL, GIVNUM,
     $                   LDGNUM, POLES, DIFL, DIFR, Z, K, C, S, WORK,
     $                   IWORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      INTEGER            GIVPTR, ICOMPQ, INFO, K, LDGCOL, LDGNUM, NL,
     $                   NR, SQRE
      DOUBLE PRECISION   ALPHA, BETA, C, S
*     ..
*     .. Array Arguments ..
      INTEGER            GIVCOL( LDGCOL, * ), IDXQ( * ), IWORK( * ),
     $                   PERM( * )
      DOUBLE PRECISION   D( * ), DIFL( * ), DIFR( * ),
     $                   GIVNUM( LDGNUM, * ), POLES( LDGNUM, * ),
```

```
     $                   VF( * ), VL( * ), WORK( * ), Z( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IDX, IDXC, IDXP, ISIGMA, IVFW, IVLW, IW, M,
     $                   N, N1, N2
      DOUBLE PRECISION   ORGNRM
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DLAMRG, DLASCL, DLASD7, DLASD8, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      N = NL + NR + 1
      M = N + SQRE
*
      IF( ( ICOMPQ.LT.0 ) .OR. ( ICOMPQ.GT.1 ) ) THEN
         INFO = -1
      ELSE IF( NL.LT.1 ) THEN
         INFO = -2
      ELSE IF( NR.LT.1 ) THEN
         INFO = -3
      ELSE IF( ( SQRE.LT.0 ) .OR. ( SQRE.GT.1 ) ) THEN
         INFO = -4
      ELSE IF( LDGCOL.LT.N ) THEN
         INFO = -14
      ELSE IF( LDGNUM.LT.N ) THEN
         INFO = -16
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASD6', -INFO )
         RETURN
      END IF
*
*     The following values are for bookkeeping purposes only.  They are
*     integer pointers which indicate the portion of the workspace
*     used by a particular array in DLASD7 and DLASD8.
*
```

```
      ISIGMA = 1
      IW = ISIGMA + N
      IVFW = IW + M
      IVLW = IVFW + M
*
      IDX = 1
      IDXC = IDX + N
      IDXP = IDXC + N
*
*     Scale.
*
      ORGNRM = MAX( ABS( ALPHA ), ABS( BETA ) )
      D( NL+1 ) = ZERO
      DO 10 I = 1, N
         IF( ABS( D( I ) ).GT.ORGNRM ) THEN
            ORGNRM = ABS( D( I ) )
         END IF
   10 CONTINUE
      CALL DLASCL( 'G', 0, 0, ORGNRM, ONE, N, 1, D, N, INFO )
      ALPHA = ALPHA / ORGNRM
      BETA = BETA / ORGNRM
*
*     Sort and Deflate singular values.
*
      CALL DLASD7( ICOMPQ, NL, NR, SQRE, K, D, Z, WORK( IW ), VF,
     $             WORK( IVFW ), VL, WORK( IVLW ), ALPHA, BETA,
     $             WORK( ISIGMA ), IWORK( IDX ), IWORK( IDXP ), IDXQ,
     $             PERM, GIVPTR, GIVCOL, LDGCOL, GIVNUM, LDGNUM, C, S,
     $             INFO )
*
*     Solve Secular Equation, compute DIFL, DIFR, and update VF, VL.
*
      CALL DLASD8( ICOMPQ, K, D, Z, VF, VL, DIFL, DIFR, LDGNUM,
     $             WORK( ISIGMA ), WORK( IW ), INFO )
*
*     Save the poles if ICOMPQ = 1.
*
      IF( ICOMPQ.EQ.1 ) THEN
         CALL DCOPY( K, D, 1, POLES( 1, 1 ), 1 )
         CALL DCOPY( K, WORK( ISIGMA ), 1, POLES( 1, 2 ), 1 )
      END IF
*
*     Unscale.
*
      CALL DLASCL( 'G', 0, 0, ONE, ORGNRM, N, 1, D, N, INFO )
*
*     Prepare the IDXQ sorting permutation.
*
      N1 = K
      N2 = N - K
```

```
      CALL DLAMRG( N1, N2, D, 1, -1, IDXQ )
*
      RETURN
*
*     End of DLASD6
*
      END
```

———-

90.175.4 lisp code

— **dlasd6.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dlasd6
         (icompq nl nr sqre d vf vl alpha beta idxq perm givptr givcol ldgcol
          givnum ldgnum poles difl difr z k c s work iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork givcol perm idxq)
             (type (double-float) s c beta alpha)
             (type (array double-float (*)) work z difr difl poles givnum vl vf
              d)
             (type (f2cl-lib:integer4) info k ldgnum ldgcol givptr sqre nr nl
              icompq))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (vf double-float vf-%data% vf-%offset%)
         (vl double-float vl-%data% vl-%offset%)
         (givnum double-float givnum-%data% givnum-%offset%)
         (poles double-float poles-%data% poles-%offset%)
         (difl double-float difl-%data% difl-%offset%)
         (difr double-float difr-%data% difr-%offset%)
         (z double-float z-%data% z-%offset%)
         (work double-float work-%data% work-%offset%)
         (idxq f2cl-lib:integer4 idxq-%data% idxq-%offset%)
         (perm f2cl-lib:integer4 perm-%data% perm-%offset%)
         (givcol f2cl-lib:integer4 givcol-%data% givcol-%offset%)
         (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
      (prog ((orgnrm 0.0d0) (i 0) (idx 0) (idxc 0) (idxp 0) (isigma 0) (ivfw 0)
             (ivlw 0) (iw 0) (m 0) (n 0) (n1 0) (n2 0))
        (declare (type (double-float) orgnrm)
                 (type (f2cl-lib:integer4) i idx idxc idxp isigma ivfw ivlw iw
                  m n n1 n2))
        (setf info 0)
```

```
(setf n (f2cl-lib:int-add nl nr 1))
(setf m (f2cl-lib:int-add n sqre))
(cond ((or (< icompq 0) (> icompq 1)) (setf info -1))
      ((< nl 1) (setf info -2)) ((< nr 1) (setf info -3))
      ((or (< sqre 0) (> sqre 1)) (setf info -4))
      ((< ldgcol n) (setf info -14)) ((< ldgnum n) (setf info -16)))
(cond
 ((/= info 0) (xerbla "DLASD6" (f2cl-lib:int-sub info))
  (go end_label)))
(setf isigma 1)
(setf iw (f2cl-lib:int-add isigma n))
(setf ivfw (f2cl-lib:int-add iw m))
(setf ivlw (f2cl-lib:int-add ivfw m))
(setf idx 1)
(setf idxc (f2cl-lib:int-add idx n))
(setf idxp (f2cl-lib:int-add idxc n))
(setf orgnrm (max (abs alpha) (abs beta)))
(setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add nl 1)) ((1 *))
                     d-%offset%)
        zero)
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (cond
     ((> (abs (f2cl-lib:fref d (i) ((1 *)))) orgnrm)
      (setf orgnrm
              (abs (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))))
   label10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dlascl "G" 0 0 orgnrm one n 1 d n info)
  (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
  (setf orgnrm var-3)
  (setf one var-4)
  (setf info var-9))
(setf alpha (/ alpha orgnrm))
(setf beta (/ beta orgnrm))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14 var-15 var-16 var-17 var-18 var-19
     var-20 var-21 var-22 var-23 var-24 var-25 var-26)
    (dlasd7 icompq nl nr sqre k d z
     (f2cl-lib:array-slice work-%data% double-float (iw) ((1 *))
                           work-%offset%)
     vf
     (f2cl-lib:array-slice work-%data% double-float (ivfw) ((1 *))
                           work-%offset%)
     vl
     (f2cl-lib:array-slice work-%data% double-float (ivlw) ((1 *))
                           work-%offset%)
```

```
      alpha beta
      (f2cl-lib:array-slice work-%data% double-float (isigma) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (idx) ((1 *))
                            iwork-%offset%)
      (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (idxp)
                            ((1 *)) iwork-%offset%)
      idxq perm givptr givcol ldgcol givnum ldgnum c s info)
  (declare
   (ignore var-5 var-6 var-7 var-8 var-9 var-10 var-11 var-14 var-15
    var-16 var-17 var-18 var-20 var-22))
  (when var-0 (setf icompq var-0))
  (when var-1 (setf nl var-1))
  (when var-2 (setf nr var-2))
  (when var-3 (setf sqre var-3))
  (when var-4 (setf k var-4))
  (when var-12 (setf alpha var-12))
  (when var-13 (setf beta var-13))
  (when var-19 (setf givptr var-19))
  (when var-21 (setf ldgcol var-21))
  (when var-23 (setf ldgnum var-23))
  (when var-24 (setf c var-24))
  (when var-25 (setf s var-25))
  (when var-26 (setf info var-26)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11)
    (dlasd8 icompq k d z vf vl difl difr ldgnum
     (f2cl-lib:array-slice work-%data% double-float (isigma) ((1 *))
                           work-%offset%)
     (f2cl-lib:array-slice work-%data% double-float (iw) ((1 *))
                           work-%offset%)
     info)
  (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-9 var-10))
  (when var-0 (setf icompq var-0))
  (when var-1 (setf k var-1))
  (when var-8 (setf ldgnum var-8))
  (when var-11 (setf info var-11)))
(cond ((/= info 0) (go end_label)))
(cond
 ((= icompq 1)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy k d 1
       (f2cl-lib:array-slice poles-%data% double-float (1 1)
                             ((1 ldgnum) (1 *)) poles-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf k var-0)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy k
```

```
            (f2cl-lib:array-slice work-%data% double-float (isigma) ((1 *))
                                  work-%offset%)
            1
            (f2cl-lib:array-slice poles-%data% double-float (1 2)
                                  ((1 ldgnum) (1 *)) poles-%offset%)
            1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf k var-0)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (dlascl "G" 0 0 one orgnrm n 1 d n info)
      (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
      (setf one var-3)
      (setf orgnrm var-4)
      (setf info var-9))
    (setf n1 k)
    (setf n2 (f2cl-lib:int-sub n k))
    (dlamrg n1 n2 d 1 -1 idxq)
    (go end_label)
   end_label
    (return
     (values icompq nl nr sqre nil nil nil alpha beta nil nil givptr nil
             ldgcol nil ldgnum nil nil nil nil k c s nil nil info))))))
```

———-

90.176 dlasd7

90.176.1 Axiom unit tests

— **dlasd7.input** —

```
)set break resume
)sys rm -f dlasd7.output
)spool dlasd7.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.176.2 Axiom help page

— dlasd7.help —

```
==================================================================
dlasd7 examples
==================================================================

==================================================================
Man Page Details
==================================================================

NAME
       DLASD7  - the two sets of singular values together into a single sorted
       set

SYNOPSIS
       SUBROUTINE DLASD7( ICOMPQ, NL, NR, SQRE, K, D, Z, ZW, VF, VFW, VL, VLW,
                          ALPHA,  BETA, DSIGMA, IDX, IDXP, IDXQ, PERM, GIVPTR,
                          GIVCOL, LDGCOL, GIVNUM, LDGNUM, C, S, INFO )

           INTEGER        GIVPTR, ICOMPQ, INFO, K,  LDGCOL,  LDGNUM,  NL,  NR,
                          SQRE

           DOUBLE         PRECISION ALPHA, BETA, C, S

           INTEGER        GIVCOL( LDGCOL, * ), IDX( * ), IDXP( * ), IDXQ( * ),
                          PERM( * )

           DOUBLE         PRECISION D( * ), DSIGMA( * ), GIVNUM( LDGNUM, *  ),
                          VF(  * ), VFW( * ), VL( * ), VLW( * ), Z( * ), ZW( *
                          )

  Purpose
  =======

  DLASD7 merges the two sets of singular values together into a single
  sorted set. Then it tries to deflate the size of the problem. There
  are two ways in which deflation can occur:  when two or more singular
  values are close together or if there is a tiny entry in the Z
  vector. For each such occurrence the order of the related
  secular equation problem is reduced by one.

  DLASD7 is called from DLASD6.

  Arguments
  =========

  ICOMPQ  (input) INTEGER
```

```
         Specifies whether singular vectors are to be computed
         in compact form, as follows:
         = 0: Compute singular values only.
         = 1: Compute singular vectors of upper
              bidiagonal matrix in compact form.

 NL      (input) INTEGER
         The row dimension of the upper block. NL >= 1.

 NR      (input) INTEGER
         The row dimension of the lower block. NR >= 1.

 SQRE    (input) INTEGER
         = 0: the lower block is an NR-by-NR square matrix.
         = 1: the lower block is an NR-by-(NR+1) rectangular matrix.

         The bidiagonal matrix has
         N = NL + NR + 1 rows and
         M = N + SQRE >= N columns.

 K       (output) INTEGER
         Contains the dimension of the non-deflated matrix, this is
         the order of the related secular equation. 1 <= K <=N.

 D       (input/output) DOUBLE PRECISION array, dimension ( N )
         On entry D contains the singular values of the two submatrices
         to be combined. On exit D contains the trailing (N-K) updated
         singular values (those which were deflated) sorted into
         increasing order.

 Z       (output) DOUBLE PRECISION array, dimension ( M )
         On exit Z contains the updating row vector in the secular
         equation.

 ZW      (workspace) DOUBLE PRECISION array, dimension ( M )
         Workspace for Z.

 VF      (input/output) DOUBLE PRECISION array, dimension ( M )
         On entry, VF(1:NL+1) contains the first components of all
         right singular vectors of the upper block; and VF(NL+2:M)
         contains the first components of all right singular vectors
         of the lower block. On exit, VF contains the first components
         of all right singular vectors of the bidiagonal matrix.

 VFW     (workspace) DOUBLE PRECISION array, dimension ( M )
         Workspace for VF.

 VL      (input/output) DOUBLE PRECISION array, dimension ( M )
         On entry, VL(1:NL+1) contains the  last components of all
         right singular vectors of the upper block; and VL(NL+2:M)
```

```
       contains the last components of all right singular vectors
       of the lower block. On exit, VL contains the last components
       of all right singular vectors of the bidiagonal matrix.

VLW    (workspace) DOUBLE PRECISION array, dimension ( M )
       Workspace for VL.

ALPHA  (input) DOUBLE PRECISION
       Contains the diagonal element associated with the added row.

BETA   (input) DOUBLE PRECISION
       Contains the off-diagonal element associated with the added
       row.

DSIGMA (output) DOUBLE PRECISION array, dimension ( N )
       Contains a copy of the diagonal elements (K-1 singular values
       and one zero) in the secular equation.

IDX    (workspace) INTEGER array, dimension ( N )
       This will contain the permutation used to sort the contents of
       D into ascending order.

IDXP   (workspace) INTEGER array, dimension ( N )
       This will contain the permutation used to place deflated
       values of D at the end of the array. On output IDXP(2:K)
       points to the nondeflated D-values and IDXP(K+1:N)
       points to the deflated singular values.

IDXQ   (input) INTEGER array, dimension ( N )
       This contains the permutation which separately sorts the two
       sub-problems in D into ascending order.  Note that entries in
       the first half of this permutation must first be moved one
       position backward; and entries in the second half
       must first have NL+1 added to their values.

PERM   (output) INTEGER array, dimension ( N )
       The permutations (from deflation and sorting) to be applied
       to each singular block. Not referenced if ICOMPQ = 0.

GIVPTR (output) INTEGER
       The number of Givens rotations which took place in this
       subproblem. Not referenced if ICOMPQ = 0.

GIVCOL (output) INTEGER array, dimension ( LDGCOL, 2 )
       Each pair of numbers indicates a pair of columns to take place
       in a Givens rotation. Not referenced if ICOMPQ = 0.

LDGCOL (input) INTEGER
       The leading dimension of GIVCOL, must be at least N.
```

```
 GIVNUM (output) DOUBLE PRECISION array, dimension ( LDGNUM, 2 )
        Each number indicates the C or S value to be used in the
        corresponding Givens rotation. Not referenced if ICOMPQ = 0.

 LDGNUM (input) INTEGER
        The leading dimension of GIVNUM, must be at least N.

 C      (output) DOUBLE PRECISION
        C contains garbage if SQRE =0 and the C-value of a Givens
        rotation related to the right null space if SQRE = 1.

 S      (output) DOUBLE PRECISION
        S contains garbage if SQRE =0 and the S-value of a Givens
        rotation related to the right null space if SQRE = 1.

 INFO   (output) INTEGER
        = 0:  successful exit.
        < 0:  if INFO = -i, the i-th argument had an illegal value.

 Further Details
 ===============

 Based on contributions by
    Ming Gu and Huan Ren, Computer Science Division, University of
    California at Berkeley, USA
```

———-

90.176.3 fortran code

— dlasd7.f —

```
      SUBROUTINE DLASD7( ICOMPQ, NL, NR, SQRE, K, D, Z, ZW, VF, VFW, VL,
     $                   VLW, ALPHA, BETA, DSIGMA, IDX, IDXP, IDXQ,
     $                   PERM, GIVPTR, GIVCOL, LDGCOL, GIVNUM, LDGNUM,
     $                   C, S, INFO )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Oak Ridge National Lab, Argonne National Lab,
*     Courant Institute, NAG Ltd., and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      INTEGER            GIVPTR, ICOMPQ, INFO, K, LDGCOL, LDGNUM, NL,
     $                   NR, SQRE
      DOUBLE PRECISION   ALPHA, BETA, C, S
*     ..
```

```
*     .. Array Arguments ..
      INTEGER            GIVCOL( LDGCOL, * ), IDX( * ), IDXP( * ),
     $                   IDXQ( * ), PERM( * )
      DOUBLE PRECISION   D( * ), DSIGMA( * ), GIVNUM( LDGNUM, * ),
     $                   VF( * ), VFW( * ), VL( * ), VLW( * ), Z( * ),
     $                   ZW( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE, TWO, EIGHT
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0, TWO = 2.0D+0,
     $                   EIGHT = 8.0D+0 )
*     ..
*     .. Local Scalars ..
*
      INTEGER            I, IDXI, IDXJ, IDXJP, J, JP, JPREV, K2, M, N,
     $                   NLP1, NLP2
      DOUBLE PRECISION   EPS, HLFTOL, TAU, TOL, Z1
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DLAMRG, DROT, XERBLA
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH, DLAPY2
      EXTERNAL           DLAMCH, DLAPY2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      N = NL + NR + 1
      M = N + SQRE
*
      IF( ( ICOMPQ.LT.0 ) .OR. ( ICOMPQ.GT.1 ) ) THEN
         INFO = -1
      ELSE IF( NL.LT.1 ) THEN
         INFO = -2
      ELSE IF( NR.LT.1 ) THEN
         INFO = -3
      ELSE IF( ( SQRE.LT.0 ) .OR. ( SQRE.GT.1 ) ) THEN
         INFO = -4
      ELSE IF( LDGCOL.LT.N ) THEN
         INFO = -22
      ELSE IF( LDGNUM.LT.N ) THEN
```

```
         INFO = -24
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASD7', -INFO )
         RETURN
      END IF
*
      NLP1 = NL + 1
      NLP2 = NL + 2
      IF( ICOMPQ.EQ.1 ) THEN
         GIVPTR = 0
      END IF
*
*     Generate the first part of the vector Z and move the singular
*     values in the first part of D one position backward.
*
      Z1 = ALPHA*VL( NLP1 )
      VL( NLP1 ) = ZERO
      TAU = VF( NLP1 )
      DO 10 I = NL, 1, -1
         Z( I+1 ) = ALPHA*VL( I )
         VL( I ) = ZERO
         VF( I+1 ) = VF( I )
         D( I+1 ) = D( I )
         IDXQ( I+1 ) = IDXQ( I ) + 1
   10 CONTINUE
      VF( 1 ) = TAU
*
*     Generate the second part of the vector Z.
*
      DO 20 I = NLP2, M
         Z( I ) = BETA*VF( I )
         VF( I ) = ZERO
   20 CONTINUE
*
*     Sort the singular values into increasing order
*
      DO 30 I = NLP2, N
         IDXQ( I ) = IDXQ( I ) + NLP1
   30 CONTINUE
*
*     DSIGMA, IDXC, IDXC, and ZW are used as storage space.
*
      DO 40 I = 2, N
         DSIGMA( I ) = D( IDXQ( I ) )
         ZW( I ) = Z( IDXQ( I ) )
         VFW( I ) = VF( IDXQ( I ) )
         VLW( I ) = VL( IDXQ( I ) )
   40 CONTINUE
*
```

```
      CALL DLAMRG( NL, NR, DSIGMA( 2 ), 1, 1, IDX( 2 ) )
*
      DO 50 I = 2, N
         IDXI = 1 + IDX( I )
         D( I ) = DSIGMA( IDXI )
         Z( I ) = ZW( IDXI )
         VF( I ) = VFW( IDXI )
         VL( I ) = VLW( IDXI )
   50 CONTINUE
*
*     Calculate the allowable deflation tolerence
*
      EPS = DLAMCH( 'Epsilon' )
      TOL = MAX( ABS( ALPHA ), ABS( BETA ) )
      TOL = EIGHT*EIGHT*EPS*MAX( ABS( D( N ) ), TOL )
*
*     There are 2 kinds of deflation -- first a value in the z-vector
*     is small, second two (or more) singular values are very close
*     together (their difference is small).
*
*     If the value in the z-vector is small, we simply permute the
*     array so that the corresponding singular value is moved to the
*     end.
*
*     If two values in the D-vector are close, we perform a two-sided
*     rotation designed to make one of the corresponding z-vector
*     entries zero, and then permute the array so that the deflated
*     singular value is moved to the end.
*
*     If there are multiple singular values then the problem deflates.
*     Here the number of equal singular values are found.  As each equal
*     singular value is found, an elementary reflector is computed to
*     rotate the corresponding singular subspace so that the
*     corresponding components of Z are zero in this new basis.
*
      K = 1
      K2 = N + 1
      DO 60 J = 2, N
         IF( ABS( Z( J ) ).LE.TOL ) THEN
*
*           Deflate due to small z component.
*
            K2 = K2 - 1
            IDXP( K2 ) = J
            IF( J.EQ.N )
     $         GO TO 100
         ELSE
            JPREV = J
            GO TO 70
         END IF
```

```
   60 CONTINUE
   70 CONTINUE
      J = JPREV
   80 CONTINUE
      J = J + 1
      IF( J.GT.N )
     $   GO TO 90
      IF( ABS( Z( J ) ).LE.TOL ) THEN
*
*        Deflate due to small z component.
*
         K2 = K2 - 1
         IDXP( K2 ) = J
      ELSE
*
*        Check if singular values are close enough to allow deflation.
*
         IF( ABS( D( J )-D( JPREV ) ).LE.TOL ) THEN
*
*           Deflation is possible.
*
            S = Z( JPREV )
            C = Z( J )
*
*           Find sqrt(a**2+b**2) without overflow or
*           destructive underflow.
*
            TAU = DLAPY2( C, S )
            Z( J ) = TAU
            Z( JPREV ) = ZERO
            C = C / TAU
            S = -S / TAU
*
*           Record the appropriate Givens rotation
*
            IF( ICOMPQ.EQ.1 ) THEN
               GIVPTR = GIVPTR + 1
               IDXJP = IDXQ( IDX( JPREV )+1 )
               IDXJ = IDXQ( IDX( J )+1 )
               IF( IDXJP.LE.NLP1 ) THEN
                  IDXJP = IDXJP - 1
               END IF
               IF( IDXJ.LE.NLP1 ) THEN
                  IDXJ = IDXJ - 1
               END IF
               GIVCOL( GIVPTR, 2 ) = IDXJP
               GIVCOL( GIVPTR, 1 ) = IDXJ
               GIVNUM( GIVPTR, 2 ) = C
               GIVNUM( GIVPTR, 1 ) = S
            END IF
```

```
            CALL DROT( 1, VF( JPREV ), 1, VF( J ), 1, C, S )
            CALL DROT( 1, VL( JPREV ), 1, VL( J ), 1, C, S )
            K2 = K2 - 1
            IDXP( K2 ) = JPREV
            JPREV = J
         ELSE
            K = K + 1
            ZW( K ) = Z( JPREV )
            DSIGMA( K ) = D( JPREV )
            IDXP( K ) = JPREV
            JPREV = J
         END IF
      END IF
      GO TO 80
   90 CONTINUE
*
*     Record the last singular value.
*
      K = K + 1
      ZW( K ) = Z( JPREV )
      DSIGMA( K ) = D( JPREV )
      IDXP( K ) = JPREV
*
  100 CONTINUE
*
*     Sort the singular values into DSIGMA. The singular values which
*     were not deflated go into the first K slots of DSIGMA, except
*     that DSIGMA(1) is treated separately.
*
      DO 110 J = 2, N
         JP = IDXP( J )
         DSIGMA( J ) = D( JP )
         VFW( J ) = VF( JP )
         VLW( J ) = VL( JP )
  110 CONTINUE
      IF( ICOMPQ.EQ.1 ) THEN
         DO 120 J = 2, N
            JP = IDXP( J )
            PERM( J ) = IDXQ( IDX( JP )+1 )
            IF( PERM( J ).LE.NLP1 ) THEN
               PERM( J ) = PERM( J ) - 1
            END IF
  120    CONTINUE
      END IF
*
*     The deflated singular values go back into the last N - K slots of
*     D.
*
      CALL DCOPY( N-K, DSIGMA( K+1 ), 1, D( K+1 ), 1 )
*
```

```
*     Determine DSIGMA(1), DSIGMA(2), Z(1), VF(1), VL(1), VF(M), and
*     VL(M).
*
      DSIGMA( 1 ) = ZERO
      HLFTOL = TOL / TWO
      IF( ABS( DSIGMA( 2 ) ).LE.HLFTOL )
     $   DSIGMA( 2 ) = HLFTOL
      IF( M.GT.N ) THEN
         Z( 1 ) = DLAPY2( Z1, Z( M ) )
         IF( Z( 1 ).LE.TOL ) THEN
            C = ONE
            S = ZERO
            Z( 1 ) = TOL
         ELSE
            C = Z1 / Z( 1 )
            S = -Z( M ) / Z( 1 )
         END IF
         CALL DROT( 1, VF( M ), 1, VF( 1 ), 1, C, S )
         CALL DROT( 1, VL( M ), 1, VL( 1 ), 1, C, S )
      ELSE
         IF( ABS( Z1 ).LE.TOL ) THEN
            Z( 1 ) = TOL
         ELSE
            Z( 1 ) = Z1
         END IF
      END IF
*
*     Restore Z, VF, and VL.
*
      CALL DCOPY( K-1, ZW( 2 ), 1, Z( 2 ), 1 )
      CALL DCOPY( N-1, VFW( 2 ), 1, VF( 2 ), 1 )
      CALL DCOPY( N-1, VLW( 2 ), 1, VL( 2 ), 1 )
*
      RETURN
*
*     End of DLASD7
*
      END
```

———-

90.176.4 lisp code

— dlasd7.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0) (two 2.0d0) (eight 8.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
```

```
          (type (double-float 1.0d0 1.0d0) one)
          (type (double-float 2.0d0 2.0d0) two)
          (type (double-float 8.0d0 8.0d0) eight)
          (ignorable zero one two eight))
(defun dlasd7
       (icompq nl nr sqre k d z zw vf vfw vl vlw alpha beta dsigma idx idxp
        idxq perm givptr givcol ldgcol givnum ldgnum c s info)
  (declare (type (array f2cl-lib:integer4 (*)) givcol perm idxq idxp idx)
           (type (double-float) s c beta alpha)
           (type (array double-float (*)) givnum dsigma vlw vl vfw vf zw z d)
           (type (f2cl-lib:integer4) info ldgnum ldgcol givptr k sqre nr nl
            icompq))
  (f2cl-lib:with-multi-array-data
      ((d double-float d-%data% d-%offset%)
       (z double-float z-%data% z-%offset%)
       (zw double-float zw-%data% zw-%offset%)
       (vf double-float vf-%data% vf-%offset%)
       (vfw double-float vfw-%data% vfw-%offset%)
       (vl double-float vl-%data% vl-%offset%)
       (vlw double-float vlw-%data% vlw-%offset%)
       (dsigma double-float dsigma-%data% dsigma-%offset%)
       (givnum double-float givnum-%data% givnum-%offset%)
       (idx f2cl-lib:integer4 idx-%data% idx-%offset%)
       (idxp f2cl-lib:integer4 idxp-%data% idxp-%offset%)
       (idxq f2cl-lib:integer4 idxq-%data% idxq-%offset%)
       (perm f2cl-lib:integer4 perm-%data% perm-%offset%)
       (givcol f2cl-lib:integer4 givcol-%data% givcol-%offset%))
    (prog ((eps 0.0d0) (hlftol 0.0d0) (tau 0.0d0) (tol 0.0d0) (z1 0.0d0)
           (i 0) (idxi 0) (idxj 0) (idxjp 0) (j 0) (jp 0) (jprev 0) (k2 0)
           (m 0) (n 0) (nlp1 0) (nlp2 0))
      (declare (type (double-float) eps hlftol tau tol z1)
               (type (f2cl-lib:integer4) i idxi idxj idxjp j jp jprev k2 m n
                nlp1 nlp2))
      (setf info 0)
      (setf n (f2cl-lib:int-add nl nr 1))
      (setf m (f2cl-lib:int-add n sqre))
      (cond ((or (< icompq 0) (> icompq 1)) (setf info -1))
            ((< nl 1) (setf info -2)) ((< nr 1) (setf info -3))
            ((or (< sqre 0) (> sqre 1)) (setf info -4))
            ((< ldgcol n) (setf info -22)) ((< ldgnum n) (setf info -24)))
      (cond
       ((/= info 0) (xerbla "DLASD7" (f2cl-lib:int-sub info))
        (go end_label)))
      (setf nlp1 (f2cl-lib:int-add nl 1))
      (setf nlp2 (f2cl-lib:int-add nl 2))
      (cond ((= icompq 1) (setf givptr 0)))
      (setf z1
              (* alpha (f2cl-lib:fref vl-%data% (nlp1) ((1 *)) vl-%offset%)))
      (setf (f2cl-lib:fref vl-%data% (nlp1) ((1 *)) vl-%offset%) zero)
      (setf tau (f2cl-lib:fref vf-%data% (nlp1) ((1 *)) vf-%offset%))
```

```
(f2cl-lib:fdo (i nl (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
              ((> i 1) nil)
  (tagbody
    (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         z-%offset%)
            (* alpha
               (f2cl-lib:fref vl-%data% (i) ((1 *)) vl-%offset%)))
    (setf (f2cl-lib:fref vl-%data% (i) ((1 *)) vl-%offset%) zero)
    (setf (f2cl-lib:fref vf-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         vf-%offset%)
            (f2cl-lib:fref vf-%data% (i) ((1 *)) vf-%offset%))
    (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         d-%offset%)
            (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
    (setf (f2cl-lib:fref idxq-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                         idxq-%offset%)
            (f2cl-lib:int-add
             (f2cl-lib:fref idxq-%data% (i) ((1 *)) idxq-%offset%) 1))
   label10))
(setf (f2cl-lib:fref vf-%data% (1) ((1 *)) vf-%offset%) tau)
(f2cl-lib:fdo (i nlp2 (f2cl-lib:int-add i 1))
              ((> i m) nil)
  (tagbody
    (setf (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
            (* beta (f2cl-lib:fref vf-%data% (i) ((1 *)) vf-%offset%)))
    (setf (f2cl-lib:fref vf-%data% (i) ((1 *)) vf-%offset%) zero)
   label20))
(f2cl-lib:fdo (i nlp2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref idxq-%data% (i) ((1 *)) idxq-%offset%)
            (f2cl-lib:int-add
             (f2cl-lib:fref idxq-%data% (i) ((1 *)) idxq-%offset%)
             nlp1))
   label30))
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref dsigma-%data% (i) ((1 *)) dsigma-%offset%)
            (f2cl-lib:fref d-%data% ((f2cl-lib:fref idxq (i) ((1 *))))
                           ((1 *)) d-%offset%))
    (setf (f2cl-lib:fref zw-%data% (i) ((1 *)) zw-%offset%)
            (f2cl-lib:fref z-%data% ((f2cl-lib:fref idxq (i) ((1 *))))
                           ((1 *)) z-%offset%))
    (setf (f2cl-lib:fref vfw-%data% (i) ((1 *)) vfw-%offset%)
            (f2cl-lib:fref vf-%data% ((f2cl-lib:fref idxq (i) ((1 *))))
                           ((1 *)) vf-%offset%))
    (setf (f2cl-lib:fref vlw-%data% (i) ((1 *)) vlw-%offset%)
            (f2cl-lib:fref vl-%data% ((f2cl-lib:fref idxq (i) ((1 *))))
                           ((1 *)) vl-%offset%))
```

```
    label40))
  (dlamrg nl nr
   (f2cl-lib:array-slice dsigma-%data% double-float (2) ((1 *))
                         dsigma-%offset%)
   1 1
   (f2cl-lib:array-slice idx-%data% f2cl-lib:integer4 (2) ((1 *))
                         idx-%offset%))
  (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf idxi
              (f2cl-lib:int-add 1
                                (f2cl-lib:fref idx-%data% (i) ((1 *))
                                               idx-%offset%)))
      (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
              (f2cl-lib:fref dsigma-%data% (idxi) ((1 *))
                             dsigma-%offset%))
      (setf (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
              (f2cl-lib:fref zw-%data% (idxi) ((1 *)) zw-%offset%))
      (setf (f2cl-lib:fref vf-%data% (i) ((1 *)) vf-%offset%)
              (f2cl-lib:fref vfw-%data% (idxi) ((1 *)) vfw-%offset%))
      (setf (f2cl-lib:fref vl-%data% (i) ((1 *)) vl-%offset%)
              (f2cl-lib:fref vlw-%data% (idxi) ((1 *)) vlw-%offset%))
    label50))
  (setf eps (dlamch "Epsilon"))
  (setf tol (max (abs alpha) (abs beta)))
  (setf tol
          (* eight eight eps
             (max (abs (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%))
                  tol)))
  (setf k 1)
  (setf k2 (f2cl-lib:int-add n 1))
  (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (cond
       ((<= (abs (f2cl-lib:fref z (j) ((1 *)))) tol)
        (setf k2 (f2cl-lib:int-sub k2 1))
        (setf (f2cl-lib:fref idxp-%data% (k2) ((1 *)) idxp-%offset%) j)
        (if (= j n)
            (go label100)))
       (t (setf jprev j) (go label70)))
    label60))
 label70
  (setf j jprev)
 label80
  (setf j (f2cl-lib:int-add j 1))
  (if (> j n)
      (go label90))
  (cond
```

```
((<= (abs (f2cl-lib:fref z (j) ((1 *)))) tol)
 (setf k2 (f2cl-lib:int-sub k2 1))
 (setf (f2cl-lib:fref idxp-%data% (k2) ((1 *)) idxp-%offset%) j))
(t
 (cond
  ((<=
    (abs
     (+ (f2cl-lib:fref d (j) ((1 *)))
        (- (f2cl-lib:fref d (jprev) ((1 *))))))
    tol)
   (setf s (f2cl-lib:fref z-%data% (jprev) ((1 *)) z-%offset%))
   (setf c (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%))
   (setf tau (dlapy2 c s))
   (setf (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%) tau)
   (setf (f2cl-lib:fref z-%data% (jprev) ((1 *)) z-%offset%) zero)
   (setf c (/ c tau)) (setf s (/ (- s) tau))
   (cond
    ((= icompq 1) (setf givptr (f2cl-lib:int-add givptr 1))
     (setf idxjp
             (f2cl-lib:fref idxq-%data%
                            ((f2cl-lib:int-add
                              (f2cl-lib:fref idx (jprev) ((1 *))) 1))
                            ((1 *)) idxq-%offset%))
     (setf idxj
             (f2cl-lib:fref idxq-%data%
                            ((f2cl-lib:int-add
                              (f2cl-lib:fref idx (j) ((1 *))) 1))
                            ((1 *)) idxq-%offset%))
     (cond ((<= idxjp nlp1) (setf idxjp (f2cl-lib:int-sub idxjp 1))))
     (cond ((<= idxj nlp1) (setf idxj (f2cl-lib:int-sub idxj 1))))
     (setf (f2cl-lib:fref givcol-%data% (givptr 2) ((1 ldgcol) (1 *))
                          givcol-%offset%)
             idxjp)
     (setf (f2cl-lib:fref givcol-%data% (givptr 1) ((1 ldgcol) (1 *))
                          givcol-%offset%)
             idxj)
     (setf (f2cl-lib:fref givnum-%data% (givptr 2) ((1 ldgnum) (1 *))
                          givnum-%offset%)
             c)
     (setf (f2cl-lib:fref givnum-%data% (givptr 1) ((1 ldgnum) (1 *))
                          givnum-%offset%)
             s)))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (drot 1
        (f2cl-lib:array-slice vf-%data% double-float (jprev) ((1 *))
                              vf-%offset%)
        1
        (f2cl-lib:array-slice vf-%data% double-float (j) ((1 *))
                              vf-%offset%)
        1 c s)
```

```
        (declare (ignore var-0 var-1 var-2 var-3 var-4))
        (when var-5 (setf c var-5))
        (when var-6 (setf s var-6)))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (drot 1
           (f2cl-lib:array-slice vl-%data% double-float (jprev) ((1 *))
                                 vl-%offset%)
           1
           (f2cl-lib:array-slice vl-%data% double-float (j) ((1 *))
                                 vl-%offset%)
           1 c s)
        (declare (ignore var-0 var-1 var-2 var-3 var-4))
        (when var-5 (setf c var-5))
        (when var-6 (setf s var-6)))
      (setf k2 (f2cl-lib:int-sub k2 1))
      (setf (f2cl-lib:fref idxp-%data% (k2) ((1 *)) idxp-%offset%) jprev)
      (setf jprev j))
     (t (setf k (f2cl-lib:int-add k 1))
      (setf (f2cl-lib:fref zw-%data% (k) ((1 *)) zw-%offset%)
              (f2cl-lib:fref z-%data% (jprev) ((1 *)) z-%offset%))
      (setf (f2cl-lib:fref dsigma-%data% (k) ((1 *)) dsigma-%offset%)
              (f2cl-lib:fref d-%data% (jprev) ((1 *)) d-%offset%))
      (setf (f2cl-lib:fref idxp-%data% (k) ((1 *)) idxp-%offset%) jprev)
      (setf jprev j)))))
 (go label80)
label90
 (setf k (f2cl-lib:int-add k 1))
 (setf (f2cl-lib:fref zw-%data% (k) ((1 *)) zw-%offset%)
         (f2cl-lib:fref z-%data% (jprev) ((1 *)) z-%offset%))
 (setf (f2cl-lib:fref dsigma-%data% (k) ((1 *)) dsigma-%offset%)
         (f2cl-lib:fref d-%data% (jprev) ((1 *)) d-%offset%))
 (setf (f2cl-lib:fref idxp-%data% (k) ((1 *)) idxp-%offset%) jprev)
label100
 (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (setf jp (f2cl-lib:fref idxp-%data% (j) ((1 *)) idxp-%offset%))
     (setf (f2cl-lib:fref dsigma-%data% (j) ((1 *)) dsigma-%offset%)
             (f2cl-lib:fref d-%data% (jp) ((1 *)) d-%offset%))
     (setf (f2cl-lib:fref vfw-%data% (j) ((1 *)) vfw-%offset%)
             (f2cl-lib:fref vf-%data% (jp) ((1 *)) vf-%offset%))
     (setf (f2cl-lib:fref vlw-%data% (j) ((1 *)) vlw-%offset%)
             (f2cl-lib:fref vl-%data% (jp) ((1 *)) vl-%offset%))
    label110))
 (cond
  ((= icompq 1)
   (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (setf jp (f2cl-lib:fref idxp-%data% (j) ((1 *)) idxp-%offset%))
```

```
      (setf (f2cl-lib:fref perm-%data% (j) ((1 *)) perm-%offset%)
              (f2cl-lib:fref idxq-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:fref idx (jp) ((1 *))) 1))
                             ((1 *)) idxq-%offset%))
      (cond
       ((<= (f2cl-lib:fref perm (j) ((1 *))) nlp1)
        (setf (f2cl-lib:fref perm-%data% (j) ((1 *)) perm-%offset%)
                (f2cl-lib:int-sub
                 (f2cl-lib:fref perm-%data% (j) ((1 *)) perm-%offset%)
                 1))))
     label120))))
(dcopy (f2cl-lib:int-sub n k)
 (f2cl-lib:array-slice dsigma-%data% double-float ((+ k 1)) ((1 *))
                       dsigma-%offset%)
 1
 (f2cl-lib:array-slice d-%data% double-float ((+ k 1)) ((1 *))
                       d-%offset%)
 1)
(setf (f2cl-lib:fref dsigma-%data% (1) ((1 *)) dsigma-%offset%) zero)
(setf hlftol (/ tol two))
(if (<= (abs (f2cl-lib:fref dsigma-%data% (2) ((1 *)) dsigma-%offset%))
        hlftol)
    (setf (f2cl-lib:fref dsigma-%data% (2) ((1 *)) dsigma-%offset%)
            hlftol))
(cond
 ((> m n)
  (setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)
          (dlapy2 z1 (f2cl-lib:fref z-%data% (m) ((1 *)) z-%offset%)))
  (cond
   ((<= (f2cl-lib:fref z (1) ((1 *))) tol) (setf c one) (setf s zero)
    (setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%) tol))
   (t (setf c (/ z1 (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)))
    (setf s
            (/ (- (f2cl-lib:fref z-%data% (m) ((1 *)) z-%offset%))
               (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)))))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (drot 1
       (f2cl-lib:array-slice vf-%data% double-float (m) ((1 *))
                             vf-%offset%)
       1
       (f2cl-lib:array-slice vf-%data% double-float (1) ((1 *))
                             vf-%offset%)
       1 c s)
    (declare (ignore var-0 var-1 var-2 var-3 var-4))
    (when var-5 (setf c var-5))
    (when var-6 (setf s var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (drot 1
       (f2cl-lib:array-slice vl-%data% double-float (m) ((1 *))
```

```
                                vl-%offset%)
              1
              (f2cl-lib:array-slice vl-%data% double-float (1) ((1 *))
                                vl-%offset%)
              1 c s)
           (declare (ignore var-0 var-1 var-2 var-3 var-4))
           (when var-5 (setf c var-5))
           (when var-6 (setf s var-6))))
         (t
          (cond
           ((<= (abs z1) tol)
            (setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%) tol))
           (t (setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%) z1)))))
        (dcopy (f2cl-lib:int-sub k 1)
         (f2cl-lib:array-slice zw-%data% double-float (2) ((1 *)) zw-%offset%)
         1 (f2cl-lib:array-slice z-%data% double-float (2) ((1 *)) z-%offset%)
         1)
        (dcopy (f2cl-lib:int-sub n 1)
         (f2cl-lib:array-slice vfw-%data% double-float (2) ((1 *))
                               vfw-%offset%)
         1
         (f2cl-lib:array-slice vf-%data% double-float (2) ((1 *)) vf-%offset%)
         1)
        (dcopy (f2cl-lib:int-sub n 1)
         (f2cl-lib:array-slice vlw-%data% double-float (2) ((1 *))
                               vlw-%offset%)
         1
         (f2cl-lib:array-slice vl-%data% double-float (2) ((1 *)) vl-%offset%)
         1)
        (go end_label)
       end_label
        (return
         (values nil nil nil nil k nil nil nil nil nil nil nil nil nil nil nil
                 nil nil nil givptr nil nil nil nil c s info))))))
```

———-

90.177 dlasd8

90.177.1 Axiom unit tests

— dlasd8.input —

```
)set break resume
)sys rm -f dlasd8.output
)spool dlasd8.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.177.2 Axiom help page

— dlasd8.help —

```
====================================================================
dlasd8 examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DLASD8 - the square roots of the roots of the secular equation,

SYNOPSIS
       SUBROUTINE DLASD8( ICOMPQ, K, D, Z, VF, VL, DIFL, DIFR, LDDIFR, DSIGMA,
                          WORK, INFO )

           INTEGER        ICOMPQ, INFO, K, LDDIFR

           DOUBLE         PRECISION D( * ), DIFL( *  ),  DIFR(  LDDIFR,  *  ),
                          DSIGMA( * ), VF( * ), VL( * ), WORK( * ), Z( * )

  Purpose
  =======

  DLASD8 finds the square roots of the roots of the secular equation,
  as defined by the values in DSIGMA and Z. It makes the appropriate
  calls to DLASD4, and stores, for each  element in D, the distance
  to its two nearest poles (elements in DSIGMA). It also updates
  the arrays VF and VL, the first and last components of all the
  right singular vectors of the original bidiagonal matrix.

  DLASD8 is called from DLASD6.

  Arguments
  =========
```

```
ICOMPQ  (input) INTEGER
        Specifies whether singular vectors are to be computed in
        factored form in the calling routine:
        = 0: Compute singular values only.
        = 1: Compute singular vectors in factored form as well.

K       (input) INTEGER
        The number of terms in the rational function to be solved
        by DLASD4.  K >= 1.

D       (output) DOUBLE PRECISION array, dimension ( K )
        On output, D contains the updated singular values.

Z       (input) DOUBLE PRECISION array, dimension ( K )
        The first K elements of this array contain the components
        of the deflation-adjusted updating row vector.

VF      (input/output) DOUBLE PRECISION array, dimension ( K )
        On entry, VF contains  information passed through DBEDE8.
        On exit, VF contains the first K components of the first
        components of all right singular vectors of the bidiagonal
        matrix.

VL      (input/output) DOUBLE PRECISION array, dimension ( K )
        On entry, VL contains  information passed through DBEDE8.
        On exit, VL contains the first K components of the last
        components of all right singular vectors of the bidiagonal
        matrix.

DIFL    (output) DOUBLE PRECISION array, dimension ( K )
        On exit, DIFL(I) = D(I) - DSIGMA(I).

DIFR    (output) DOUBLE PRECISION array,
                 dimension ( LDDIFR, 2 ) if ICOMPQ = 1 and
                 dimension ( K ) if ICOMPQ = 0.
        On exit, DIFR(I,1) = D(I) - DSIGMA(I+1), DIFR(K,1) is not
        defined and will not be referenced.

        If ICOMPQ = 1, DIFR(1:K,2) is an array containing the
        normalizing factors for the right singular vector matrix.

LDDIFR  (input) INTEGER
        The leading dimension of DIFR, must be at least K.

DSIGMA  (input) DOUBLE PRECISION array, dimension ( K )
        The first K elements of this array contain the old roots
        of the deflated updating problem.  These are the poles
        of the secular equation.

WORK    (workspace) DOUBLE PRECISION array, dimension at least 3 * K
```

```
 INFO    (output) INTEGER
         = 0:  successful exit.
         < 0:  if INFO = -i, the i-th argument had an illegal value.
         > 0:  if INFO = 1, an singular value did not converge

 Further Details
 ===============

 Based on contributions by
    Ming Gu and Huan Ren, Computer Science Division, University of
    California at Berkeley, USA
```

———-

90.177.3 fortran code

— **dlasd8.f** —

```
      SUBROUTINE DLASD8( ICOMPQ, K, D, Z, VF, VL, DIFL, DIFR, LDDIFR,
     $                   DSIGMA, WORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Oak Ridge National Lab, Argonne National Lab,
*     Courant Institute, NAG Ltd., and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      INTEGER            ICOMPQ, INFO, K, LDDIFR
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), DIFL( * ), DIFR( LDDIFR, * ),
     $                   DSIGMA( * ), VF( * ), VL( * ), WORK( * ),
     $                   Z( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IWK1, IWK2, IWK2I, IWK3, IWK3I, J
      DOUBLE PRECISION   DIFLJ, DIFRJ, DJ, DSIGJ, DSIGJP, RHO, TEMP
*     ..
*     .. External Subroutines ..
```

```
      EXTERNAL           DCOPY, DLASCL, DLASD4, DLASET, XERBLA
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DDOT, DLAMC3, DNRM2
      EXTERNAL           DDOT, DLAMC3, DNRM2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SIGN, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( ( ICOMPQ.LT.0 ) .OR. ( ICOMPQ.GT.1 ) ) THEN
         INFO = -1
      ELSE IF( K.LT.1 ) THEN
         INFO = -2
      ELSE IF( LDDIFR.LT.K ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASD8', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( K.EQ.1 ) THEN
         D( 1 ) = ABS( Z( 1 ) )
         DIFL( 1 ) = D( 1 )
         IF( ICOMPQ.EQ.1 ) THEN
            DIFL( 2 ) = ONE
            DIFR( 1, 2 ) = ONE
         END IF
         RETURN
      END IF
*
*     Modify values DSIGMA(i) to make sure all DSIGMA(i)-DSIGMA(j) can
*     be computed with high relative accuracy (barring over/underflow).
*     This is a problem on machines without a guard digit in
*     add/subtract (Cray XMP, Cray YMP, Cray C 90 and Cray 2).
*     The following code replaces DSIGMA(I) by 2*DSIGMA(I)-DSIGMA(I),
*     which on any of these machines zeros out the bottommost
*     bit of DSIGMA(I) if it is 1; this makes the subsequent
*     subtractions DSIGMA(I)-DSIGMA(J) unproblematic when cancellation
*     occurs. On binary machines with a guard digit (almost all
*     machines) it does not change DSIGMA(I) at all. On hexadecimal
*     and decimal machines with a guard digit, it slightly
```

```
*     changes the bottommost bits of DSIGMA(I). It does not account
*     for hexadecimal or decimal machines without guard digits
*     (we know of none). We use a subroutine call to compute
*     2*DLAMBDA(I) to prevent optimizing compilers from eliminating
*     this code.
*
      DO 10 I = 1, K
         DSIGMA( I ) = DLAMC3( DSIGMA( I ), DSIGMA( I ) ) - DSIGMA( I )
   10 CONTINUE
*
*     Book keeping.
*
      IWK1 = 1
      IWK2 = IWK1 + K
      IWK3 = IWK2 + K
      IWK2I = IWK2 - 1
      IWK3I = IWK3 - 1
*
*     Normalize Z.
*
      RHO = DNRM2( K, Z, 1 )
      CALL DLASCL( 'G', 0, 0, RHO, ONE, K, 1, Z, K, INFO )
      RHO = RHO*RHO
*
*     Initialize WORK(IWK3).
*
      CALL DLASET( 'A', K, 1, ONE, ONE, WORK( IWK3 ), K )
*
*     Compute the updated singular values, the arrays DIFL, DIFR,
*     and the updated Z.
*
      DO 40 J = 1, K
         CALL DLASD4( K, J, DSIGMA, Z, WORK( IWK1 ), RHO, D( J ),
     $                WORK( IWK2 ), INFO )
*
*        If the root finder fails, the computation is terminated.
*
         IF( INFO.NE.0 ) THEN
            RETURN
         END IF
         WORK( IWK3I+J ) = WORK( IWK3I+J )*WORK( J )*WORK( IWK2I+J )
         DIFL( J ) = -WORK( J )
         DIFR( J, 1 ) = -WORK( J+1 )
         DO 20 I = 1, J - 1
            WORK( IWK3I+I ) = WORK( IWK3I+I )*WORK( I )*
     $                        WORK( IWK2I+I ) / ( DSIGMA( I )-
     $                        DSIGMA( J ) ) / ( DSIGMA( I )+
     $                        DSIGMA( J ) )
   20    CONTINUE
         DO 30 I = J + 1, K
```

```
            WORK( IWK3I+I ) = WORK( IWK3I+I )*WORK( I )*
     $                        WORK( IWK2I+I ) / ( DSIGMA( I )-
     $                        DSIGMA( J ) ) / ( DSIGMA( I )+
     $                        DSIGMA( J ) )
   30    CONTINUE
   40 CONTINUE
*
*     Compute updated Z.
*
      DO 50 I = 1, K
         Z( I ) = SIGN( SQRT( ABS( WORK( IWK3I+I ) ) ), Z( I ) )
   50 CONTINUE
*
*     Update VF and VL.
*
      DO 80 J = 1, K
         DIFLJ = DIFL( J )
         DJ = D( J )
         DSIGJ = -DSIGMA( J )
         IF( J.LT.K ) THEN
            DIFRJ = -DIFR( J, 1 )
            DSIGJP = -DSIGMA( J+1 )
         END IF
         WORK( J ) = -Z( J ) / DIFLJ / ( DSIGMA( J )+DJ )
         DO 60 I = 1, J - 1
            WORK( I ) = Z( I ) / ( DLAMC3( DSIGMA( I ), DSIGJ )-DIFLJ )
     $                  / ( DSIGMA( I )+DJ )
   60    CONTINUE
         DO 70 I = J + 1, K
            WORK( I ) = Z( I ) / ( DLAMC3( DSIGMA( I ), DSIGJP )+DIFRJ )
     $                  / ( DSIGMA( I )+DJ )
   70    CONTINUE
         TEMP = DNRM2( K, WORK, 1 )
         WORK( IWK2I+J ) = DDOT( K, WORK, 1, VF, 1 ) / TEMP
         WORK( IWK3I+J ) = DDOT( K, WORK, 1, VL, 1 ) / TEMP
         IF( ICOMPQ.EQ.1 ) THEN
            DIFR( J, 2 ) = TEMP
         END IF
   80 CONTINUE
*
      CALL DCOPY( K, WORK( IWK2 ), 1, VF, 1 )
      CALL DCOPY( K, WORK( IWK3 ), 1, VL, 1 )
*
      RETURN
*
*     End of DLASD8
*
      END
```

———-

90.177.4 lisp code

— dlasd8.lisp —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dlasd8 (icompq k d z vf vl difl difr lddifr dsigma work info)
    (declare (type (array double-float (*)) work dsigma difr difl vl vf z d)
             (type (f2cl-lib:integer4) info lddifr k icompq))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (z double-float z-%data% z-%offset%)
         (vf double-float vf-%data% vf-%offset%)
         (vl double-float vl-%data% vl-%offset%)
         (difl double-float difl-%data% difl-%offset%)
         (difr double-float difr-%data% difr-%offset%)
         (dsigma double-float dsigma-%data% dsigma-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((diflj 0.0d0) (difrj 0.0d0) (dj 0.0d0) (dsigj 0.0d0)
             (dsigjp 0.0d0) (rho 0.0d0) (temp 0.0d0) (i 0) (iwk1 0) (iwk2 0)
             (iwk2i 0) (iwk3 0) (iwk3i 0) (j 0))
        (declare (type (double-float) diflj difrj dj dsigj dsigjp rho temp)
                 (type (f2cl-lib:integer4) i iwk1 iwk2 iwk2i iwk3 iwk3i j))
        (setf info 0)
        (cond ((or (< icompq 0) (> icompq 1)) (setf info -1))
              ((< k 1) (setf info -2)) ((< lddifr k) (setf info -9)))
        (cond
         ((/= info 0) (xerbla "DLASD8" (f2cl-lib:int-sub info))
          (go end_label)))
        (cond
         ((= k 1)
          (setf (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
                  (abs (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)))
          (setf (f2cl-lib:fref difl-%data% (1) ((1 *)) difl-%offset%)
                  (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%))
          (cond
           ((= icompq 1)
            (setf (f2cl-lib:fref difl-%data% (2) ((1 *)) difl-%offset%) one)
            (setf (f2cl-lib:fref difr-%data% (1 2) ((1 lddifr) (1 *))
                                 difr-%offset%)
                  one)))
          (go end_label)))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i k) nil)
          (tagbody
```

```
    (setf (f2cl-lib:fref dsigma-%data% (i) ((1 *)) dsigma-%offset%)
            (-
             (dlamc3
              (f2cl-lib:array-slice dsigma-%data% double-float (i)
                                    ((1 *)) dsigma-%offset%)
              (f2cl-lib:array-slice dsigma-%data% double-float (i)
                                    ((1 *)) dsigma-%offset%))
             (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                            dsigma-%offset%)))
   label10))
(setf iwk1 1)
(setf iwk2 (f2cl-lib:int-add iwk1 k))
(setf iwk3 (f2cl-lib:int-add iwk2 k))
(setf iwk2i (f2cl-lib:int-sub iwk2 1))
(setf iwk3i (f2cl-lib:int-sub iwk3 1))
(setf rho
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (dnrm2 k z 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf k var-0))
          ret-val))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dlascl "G" 0 0 rho one k 1 z k info)
  (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
  (setf rho var-3)
  (setf one var-4)
  (setf info var-9))
(setf rho (* rho rho))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlaset "A" k 1 one one
     (f2cl-lib:array-slice work-%data% double-float (iwk3) ((1 *))
                           work-%offset%)
     k)
  (declare (ignore var-0 var-2 var-5))
  (when var-1 (setf k var-1))
  (when var-3 (setf one var-3))
  (when var-4 (setf one var-4))
  (when var-6 (setf k var-6)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j k) nil)
  (tagbody
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dlasd4 k j dsigma z
         (f2cl-lib:array-slice work-%data% double-float (iwk1) ((1 *))
                               work-%offset%)
         rho (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (iwk2) ((1 *))
                               work-%offset%)
```

```
     info)
  (declare (ignore var-0 var-2 var-3 var-4 var-7))
  (setf j var-1)
  (setf rho var-5)
  (setf (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%) var-6)
  (setf info var-8))
(cond ((/= info 0) (go end_label)))
(setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add iwk3i j))
                     ((1 *)) work-%offset%)
        (*
         (f2cl-lib:fref work-%data% ((f2cl-lib:int-add iwk3i j))
                        ((1 *)) work-%offset%)
         (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
         (f2cl-lib:fref work-%data% ((f2cl-lib:int-add iwk2i j))
                        ((1 *)) work-%offset%)))
(setf (f2cl-lib:fref difl-%data% (j) ((1 *)) difl-%offset%)
        (- (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)))
(setf (f2cl-lib:fref difr-%data% (j 1) ((1 lddifr) (1 *))
                     difr-%offset%)
        (-
         (f2cl-lib:fref work-%data% ((f2cl-lib:int-add j 1))
                        ((1 *)) work-%offset%)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add iwk3i i))
                         ((1 *)) work-%offset%)
            (/
             (/
              (*
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add iwk3i i)) ((1 *))
                              work-%offset%)
               (f2cl-lib:fref work-%data% (i) ((1 *))
                              work-%offset%)
               (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add iwk2i i)) ((1 *))
                              work-%offset%))
              (-
               (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                              dsigma-%offset%)
               (f2cl-lib:fref dsigma-%data% (j) ((1 *))
                              dsigma-%offset%)))
             (+
              (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                             dsigma-%offset%)
              (f2cl-lib:fref dsigma-%data% (j) ((1 *))
                             dsigma-%offset%))))
   label20))
(f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
```

```
                    ((> i k) nil)
          (tagbody
            (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add iwk3i i))
                                 ((1 *)) work-%offset%)
                  (/
                   (/
                    (*
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add iwk3i i)) ((1 *))
                                    work-%offset%)
                     (f2cl-lib:fref work-%data% (i) ((1 *))
                                    work-%offset%)
                     (f2cl-lib:fref work-%data%
                                    ((f2cl-lib:int-add iwk2i i)) ((1 *))
                                    work-%offset%))
                    (-
                     (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                                    dsigma-%offset%)
                     (f2cl-lib:fref dsigma-%data% (j) ((1 *))
                                    dsigma-%offset%)))
                   (+
                    (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                                   dsigma-%offset%)
                    (f2cl-lib:fref dsigma-%data% (j) ((1 *))
                                   dsigma-%offset%))))
          label30))
      label40))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i k) nil)
    (tagbody
      (setf (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
              (f2cl-lib:sign
               (f2cl-lib:fsqrt
                (abs
                 (f2cl-lib:fref work-%data% ((f2cl-lib:int-add iwk3i i))
                                ((1 *)) work-%offset%)))
               (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)))
      label50))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j k) nil)
    (tagbody
      (setf diflj (f2cl-lib:fref difl-%data% (j) ((1 *)) difl-%offset%))
      (setf dj (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%))
      (setf dsigj
              (-
               (f2cl-lib:fref dsigma-%data% (j) ((1 *))
                              dsigma-%offset%)))
      (cond
       ((< j k)
        (setf difrj
```

```
          (-
           (f2cl-lib:fref difr-%data% (j 1) ((1 lddifr) (1 *))
                          difr-%offset%)))
  (setf dsigjp
          (-
           (f2cl-lib:fref dsigma-%data% ((f2cl-lib:int-add j 1))
                          ((1 *)) dsigma-%offset%)))))
(setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
        (/
         (/ (- (f2cl-lib:fref z-%data% (j) ((1 *)) z-%offset%))
            diflj)
         (+
          (f2cl-lib:fref dsigma-%data% (j) ((1 *)) dsigma-%offset%)
          dj)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (/
             (/ (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                (-
                 (multiple-value-bind (ret-val var-0 var-1)
                     (dlamc3
                      (f2cl-lib:array-slice dsigma-%data%
                                            double-float (i)
                                            ((1 *))
                                            dsigma-%offset%)
                      dsigj)
                   (declare (ignore var-0))
                   (when var-1 (setf dsigj var-1))
                   ret-val)
                 diflj))
             (+
              (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                             dsigma-%offset%)
              dj)))
   label60))
(f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
              ((> i k) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (/
             (/ (f2cl-lib:fref z-%data% (i) ((1 *)) z-%offset%)
                (+
                 (multiple-value-bind (ret-val var-0 var-1)
                     (dlamc3
                      (f2cl-lib:array-slice dsigma-%data%
                                            double-float (i)
                                            ((1 *))
                                            dsigma-%offset%)
```

```
                          dsigjp)
                        (declare (ignore var-0))
                        (when var-1 (setf dsigjp var-1))
                        ret-val)
                      difrj))
                  (+
                   (f2cl-lib:fref dsigma-%data% (i) ((1 *))
                                  dsigma-%offset%)
                   dj)))
       label70))
    (setf temp
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (dnrm2 k work 1)
              (declare (ignore var-1 var-2))
              (when var-0 (setf k var-0))
              ret-val))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add iwk2i j))
                         ((1 *)) work-%offset%)
            (/
             (multiple-value-bind
                 (ret-val var-0 var-1 var-2 var-3 var-4)
                 (ddot k work 1 vf 1)
               (declare (ignore var-1 var-2 var-3 var-4))
               (when var-0 (setf k var-0))
               ret-val)
             temp))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add iwk3i j))
                         ((1 *)) work-%offset%)
            (/
             (multiple-value-bind
                 (ret-val var-0 var-1 var-2 var-3 var-4)
                 (ddot k work 1 vl 1)
               (declare (ignore var-1 var-2 var-3 var-4))
               (when var-0 (setf k var-0))
               ret-val)
             temp))
    (cond
     ((= icompq 1)
      (setf (f2cl-lib:fref difr-%data% (j 2) ((1 lddifr) (1 *))
                           difr-%offset%)
            temp)))
   label80))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dcopy k
     (f2cl-lib:array-slice work-%data% double-float (iwk2) ((1 *))
                           work-%offset%)
     1 vf 1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf k var-0)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
```

```
          (dcopy k
           (f2cl-lib:array-slice work-%data% double-float (iwk3) ((1 *))
                                 work-%offset%)
           1 vl 1)
        (declare (ignore var-1 var-2 var-3 var-4))
        (when var-0 (setf k var-0)))
      (go end_label)
     end_label
      (return (values nil k nil nil nil nil nil nil nil nil nil info))))))
```

———-

90.178 dlasda

90.178.1 Axiom unit tests

— **dlasda.input** —

```
)set break resume
)sys rm -f dlasda.output
)spool dlasda.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.178.2 Axiom help page

— **dlasda.help** —

```
==========================================================================
dlasda examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

NAME
```

```
      DLASDA  -  divide  and  conquer  approach, DLASDA computes the singular
      value decomposition (SVD) of a real upper bidiagonal  N-by-M  matrix  B
      with diagonal D and offdiagonal E, where M = N + SQRE

SYNOPSIS
      SUBROUTINE DLASDA( ICOMPQ,  SMLSIZ, N, SQRE, D, E, U, LDU, VT, K, DIFL,
                         DIFR,  Z,  POLES,  GIVPTR,  GIVCOL,  LDGCOL,   PERM,
                         GIVNUM, C, S, WORK, IWORK, INFO )

          INTEGER        ICOMPQ, INFO, LDGCOL, LDU, N, SMLSIZ, SQRE

          INTEGER        GIVCOL(  LDGCOL,  * ), GIVPTR( * ), IWORK( * ), K( *
                         ), PERM( LDGCOL, * )

          DOUBLE         PRECISION C( * ), D( * ), DIFL( LDU, * ), DIFR( LDU,
                         * ), E( * ), GIVNUM( LDU, * ), POLES( LDU, * ), S( *
                         ), U( LDU, * ), VT( LDU, * ), WORK( * ), Z( LDU, * )

  Purpose
  =======

  Using a divide and conquer approach, DLASDA computes the singular
  value decomposition (SVD) of a real upper bidiagonal N-by-M matrix
  B with diagonal D and offdiagonal E, where M = N + SQRE. The
  algorithm computes the singular values in the SVD B = U * S * VT.
  The orthogonal matrices U and VT are optionally computed in
  compact form.

  A related subroutine, DLASD0, computes the singular values and
  the singular vectors in explicit form.

  Arguments
  =========

  ICOMPQ (input) INTEGER
         Specifies whether singular vectors are to be computed
         in compact form, as follows
         = 0: Compute singular values only.
         = 1: Compute singular vectors of upper bidiagonal
              matrix in compact form.

  SMLSIZ (input) INTEGER
         The maximum size of the subproblems at the bottom of the
         computation tree.

  N      (input) INTEGER
         The row dimension of the upper bidiagonal matrix. This is
         also the dimension of the main diagonal array D.

  SQRE   (input) INTEGER
```

```
        Specifies the column dimension of the bidiagonal matrix.
        = 0: The bidiagonal matrix has column dimension M = N;
        = 1: The bidiagonal matrix has column dimension M = N + 1.

 D      (input/output) DOUBLE PRECISION array, dimension ( N )
        On entry D contains the main diagonal of the bidiagonal
        matrix. On exit D, if INFO = 0, contains its singular values.

 E      (input) DOUBLE PRECISION array, dimension ( M-1 )
        Contains the subdiagonal entries of the bidiagonal matrix.
        On exit, E has been destroyed.

 U      (output) DOUBLE PRECISION array,
        dimension ( LDU, SMLSIZ ) if ICOMPQ = 1, and not referenced
        if ICOMPQ = 0. If ICOMPQ = 1, on exit, U contains the left
        singular vector matrices of all subproblems at the bottom
        level.

 LDU    (input) INTEGER, LDU = > N.
        The leading dimension of arrays U, VT, DIFL, DIFR, POLES,
        GIVNUM, and Z.

 VT     (output) DOUBLE PRECISION array,
        dimension ( LDU, SMLSIZ+1 ) if ICOMPQ = 1, and not referenced
        if ICOMPQ = 0. If ICOMPQ = 1, on exit, VT' contains the right
        singular vector matrices of all subproblems at the bottom
        level.

 K      (output) INTEGER array,
        dimension ( N ) if ICOMPQ = 1 and dimension 1 if ICOMPQ = 0.
        If ICOMPQ = 1, on exit, K(I) is the dimension of the I-th
        secular equation on the computation tree.

 DIFL   (output) DOUBLE PRECISION array, dimension ( LDU, NLVL ),
        where NLVL = floor(log_2 (N/SMLSIZ))).

 DIFR   (output) DOUBLE PRECISION array,
                 dimension ( LDU, 2 * NLVL ) if ICOMPQ = 1 and
                 dimension ( N ) if ICOMPQ = 0.
        If ICOMPQ = 1, on exit, DIFL(1:N, I) and DIFR(1:N, 2 * I - 1)
        record distances between singular values on the I-th
        level and singular values on the (I -1)-th level, and
        DIFR(1:N, 2 * I ) contains the normalizing factors for
        the right singular vector matrix. See DLASD8 for details.

 Z      (output) DOUBLE PRECISION array,
                 dimension ( LDU, NLVL ) if ICOMPQ = 1 and
                 dimension ( N ) if ICOMPQ = 0.
        The first K elements of Z(1, I) contain the components of
        the deflation-adjusted updating row vector for subproblems
```

```
       on the I-th level.

POLES  (output) DOUBLE PRECISION array,
       dimension ( LDU, 2 * NLVL ) if ICOMPQ = 1, and not referenced
       if ICOMPQ = 0. If ICOMPQ = 1, on exit, POLES(1, 2*I - 1) and
       POLES(1, 2*I) contain  the new and old singular values
       involved in the secular equations on the I-th level.

GIVPTR (output) INTEGER array,
       dimension ( N ) if ICOMPQ = 1, and not referenced if
       ICOMPQ = 0. If ICOMPQ = 1, on exit, GIVPTR( I ) records
       the number of Givens rotations performed on the I-th
       problem on the computation tree.

GIVCOL (output) INTEGER array,
       dimension ( LDGCOL, 2 * NLVL ) if ICOMPQ = 1, and not
       referenced if ICOMPQ = 0. If ICOMPQ = 1, on exit, for each I,
       GIVCOL(1, 2 *I - 1) and GIVCOL(1, 2 *I) record the locations
       of Givens rotations performed on the I-th level on the
       computation tree.

LDGCOL (input) INTEGER, LDGCOL = > N.
       The leading dimension of arrays GIVCOL and PERM.

PERM   (output) INTEGER array,
       dimension ( LDGCOL, NLVL ) if ICOMPQ = 1, and not referenced
       if ICOMPQ = 0. If ICOMPQ = 1, on exit, PERM(1, I) records
       permutations done on the I-th level of the computation tree.

GIVNUM (output) DOUBLE PRECISION array,
       dimension ( LDU,  2 * NLVL ) if ICOMPQ = 1, and not
       referenced if ICOMPQ = 0. If ICOMPQ = 1, on exit, for each I,
       GIVNUM(1, 2 *I - 1) and GIVNUM(1, 2 *I) record the C- and S-
       values of Givens rotations performed on the I-th level on
       the computation tree.

C      (output) DOUBLE PRECISION array,
       dimension ( N ) if ICOMPQ = 1, and dimension 1 if ICOMPQ = 0.
       If ICOMPQ = 1 and the I-th subproblem is not square, on exit,
       C( I ) contains the C-value of a Givens rotation related to
       the right null space of the I-th subproblem.

S      (output) DOUBLE PRECISION array, dimension ( N ) if
       ICOMPQ = 1, and dimension 1 if ICOMPQ = 0. If ICOMPQ = 1
       and the I-th subproblem is not square, on exit, S( I )
       contains the S-value of a Givens rotation related to
       the right null space of the I-th subproblem.

WORK   (workspace) DOUBLE PRECISION array, dimension
       (6 * N + (SMLSIZ + 1)*(SMLSIZ + 1)).
```

```
  IWORK  (workspace) INTEGER array.
         Dimension must be at least (7 * N).

  INFO   (output) INTEGER
          = 0:  successful exit.
          < 0:  if INFO = -i, the i-th argument had an illegal value.
          > 0:  if INFO = 1, an singular value did not converge

  Further Details
  ===============

  Based on contributions by
     Ming Gu and Huan Ren, Computer Science Division, University of
     California at Berkeley, USA
```

———-

90.178.3 fortran code

— dlasda.f —

```
      SUBROUTINE DLASDA( ICOMPQ, SMLSIZ, N, SQRE, D, E, U, LDU, VT, K,
     $                   DIFL, DIFR, Z, POLES, GIVPTR, GIVCOL, LDGCOL,
     $                   PERM, GIVNUM, C, S, WORK, IWORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1999
*
*     .. Scalar Arguments ..
      INTEGER            ICOMPQ, INFO, LDGCOL, LDU, N, SMLSIZ, SQRE
*     ..
*     .. Array Arguments ..
      INTEGER            GIVCOL( LDGCOL, * ), GIVPTR( * ), IWORK( * ),
     $                   K( * ), PERM( LDGCOL, * )
      DOUBLE PRECISION   C( * ), D( * ), DIFL( LDU, * ), DIFR( LDU, * ),
     $                   E( * ), GIVNUM( LDU, * ), POLES( LDU, * ),
     $                   S( * ), U( LDU, * ), VT( LDU, * ), WORK( * ),
     $                   Z( LDU, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
```

```
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, I1, IC, IDXQ, IDXQI, IM1, INODE, ITEMP, IWK,
     $                   J, LF, LL, LVL, LVL2, M, NCC, ND, NDB1, NDIML,
     $                   NDIMR, NL, NLF, NLP1, NLVL, NR, NRF, NRP1, NRU,
     $                   NWORK1, NWORK2, SMLSZP, SQREI, VF, VFI, VL, VLI
      DOUBLE PRECISION   ALPHA, BETA
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DLASD6, DLASDQ, DLASDT, DLASET, XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
*
      IF( ( ICOMPQ.LT.0 ) .OR. ( ICOMPQ.GT.1 ) ) THEN
         INFO = -1
      ELSE IF( SMLSIZ.LT.3 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( ( SQRE.LT.0 ) .OR. ( SQRE.GT.1 ) ) THEN
         INFO = -4
      ELSE IF( LDU.LT.( N+SQRE ) ) THEN
         INFO = -8
      ELSE IF( LDGCOL.LT.N ) THEN
         INFO = -17
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASDA', -INFO )
         RETURN
      END IF
*
      M = N + SQRE
*
*     If the input matrix is too small, call DLASDQ to find the SVD.
*
      IF( N.LE.SMLSIZ ) THEN
         IF( ICOMPQ.EQ.0 ) THEN
            CALL DLASDQ( 'U', SQRE, N, 0, 0, 0, D, E, VT, LDU, U, LDU,
     $                   U, LDU, WORK, INFO )
         ELSE
            CALL DLASDQ( 'U', SQRE, N, M, N, 0, D, E, VT, LDU, U, LDU,
     $                   U, LDU, WORK, INFO )
         END IF
         RETURN
      END IF
```

```
*
*     Book-keeping and  set up the computation tree.
*
      INODE = 1
      NDIML = INODE + N
      NDIMR = NDIML + N
      IDXQ = NDIMR + N
      IWK = IDXQ + N
*
      NCC = 0
      NRU = 0
*
      SMLSZP = SMLSIZ + 1
      VF = 1
      VL = VF + M
      NWORK1 = VL + M
      NWORK2 = NWORK1 + SMLSZP*SMLSZP
*
      CALL DLASDT( N, NLVL, ND, IWORK( INODE ), IWORK( NDIML ),
     $             IWORK( NDIMR ), SMLSIZ )
*
*     for the nodes on bottom level of the tree, solve
*     their subproblems by DLASDQ.
*
      NDB1 = ( ND+1 ) / 2
      DO 30 I = NDB1, ND
*
*        IC : center row of each node
*        NL : number of rows of left  subproblem
*        NR : number of rows of right subproblem
*        NLF: starting row of the left   subproblem
*        NRF: starting row of the right  subproblem
*
         I1 = I - 1
         IC = IWORK( INODE+I1 )
         NL = IWORK( NDIML+I1 )
         NLP1 = NL + 1
         NR = IWORK( NDIMR+I1 )
         NLF = IC - NL
         NRF = IC + 1
         IDXQI = IDXQ + NLF - 2
         VFI = VF + NLF - 1
         VLI = VL + NLF - 1
         SQREI = 1
         IF( ICOMPQ.EQ.0 ) THEN
            CALL DLASET( 'A', NLP1, NLP1, ZERO, ONE, WORK( NWORK1 ),
     $                   SMLSZP )
            CALL DLASDQ( 'U', SQREI, NL, NLP1, NRU, NCC, D( NLF ),
     $                   E( NLF ), WORK( NWORK1 ), SMLSZP,
     $                   WORK( NWORK2 ), NL, WORK( NWORK2 ), NL,
```

```
$                   WORK( NWORK2 ), INFO )
         ITEMP = NWORK1 + NL*SMLSZP
         CALL DCOPY( NLP1, WORK( NWORK1 ), 1, WORK( VFI ), 1 )
         CALL DCOPY( NLP1, WORK( ITEMP ), 1, WORK( VLI ), 1 )
      ELSE
         CALL DLASET( 'A', NL, NL, ZERO, ONE, U( NLF, 1 ), LDU )
         CALL DLASET( 'A', NLP1, NLP1, ZERO, ONE, VT( NLF, 1 ), LDU )
         CALL DLASDQ( 'U', SQREI, NL, NLP1, NL, NCC, D( NLF ),
$                   E( NLF ), VT( NLF, 1 ), LDU, U( NLF, 1 ), LDU,
$                   U( NLF, 1 ), LDU, WORK( NWORK1 ), INFO )
         CALL DCOPY( NLP1, VT( NLF, 1 ), 1, WORK( VFI ), 1 )
         CALL DCOPY( NLP1, VT( NLF, NLP1 ), 1, WORK( VLI ), 1 )
      END IF
      IF( INFO.NE.0 ) THEN
         RETURN
      END IF
      DO 10 J = 1, NL
         IWORK( IDXQI+J ) = J
10    CONTINUE
      IF( ( I.EQ.ND ) .AND. ( SQRE.EQ.0 ) ) THEN
         SQREI = 0
      ELSE
         SQREI = 1
      END IF
      IDXQI = IDXQI + NLP1
      VFI = VFI + NLP1
      VLI = VLI + NLP1
      NRP1 = NR + SQREI
      IF( ICOMPQ.EQ.0 ) THEN
         CALL DLASET( 'A', NRP1, NRP1, ZERO, ONE, WORK( NWORK1 ),
$                   SMLSZP )
         CALL DLASDQ( 'U', SQREI, NR, NRP1, NRU, NCC, D( NRF ),
$                   E( NRF ), WORK( NWORK1 ), SMLSZP,
$                   WORK( NWORK2 ), NR, WORK( NWORK2 ), NR,
$                   WORK( NWORK2 ), INFO )
         ITEMP = NWORK1 + ( NRP1-1 )*SMLSZP
         CALL DCOPY( NRP1, WORK( NWORK1 ), 1, WORK( VFI ), 1 )
         CALL DCOPY( NRP1, WORK( ITEMP ), 1, WORK( VLI ), 1 )
      ELSE
         CALL DLASET( 'A', NR, NR, ZERO, ONE, U( NRF, 1 ), LDU )
         CALL DLASET( 'A', NRP1, NRP1, ZERO, ONE, VT( NRF, 1 ), LDU )
         CALL DLASDQ( 'U', SQREI, NR, NRP1, NR, NCC, D( NRF ),
$                   E( NRF ), VT( NRF, 1 ), LDU, U( NRF, 1 ), LDU,
$                   U( NRF, 1 ), LDU, WORK( NWORK1 ), INFO )
         CALL DCOPY( NRP1, VT( NRF, 1 ), 1, WORK( VFI ), 1 )
         CALL DCOPY( NRP1, VT( NRF, NRP1 ), 1, WORK( VLI ), 1 )
      END IF
      IF( INFO.NE.0 ) THEN
         RETURN
      END IF
```

```
         DO 20 J = 1, NR
            IWORK( IDXQI+J ) = J
   20    CONTINUE
   30 CONTINUE
*
*     Now conquer each subproblem bottom-up.
*
      J = 2**NLVL
      DO 50 LVL = NLVL, 1, -1
         LVL2 = LVL*2 - 1
*
*        Find the first node LF and last node LL on
*        the current level LVL.
*
         IF( LVL.EQ.1 ) THEN
            LF = 1
            LL = 1
         ELSE
            LF = 2**( LVL-1 )
            LL = 2*LF - 1
         END IF
         DO 40 I = LF, LL
            IM1 = I - 1
            IC = IWORK( INODE+IM1 )
            NL = IWORK( NDIML+IM1 )
            NR = IWORK( NDIMR+IM1 )
            NLF = IC - NL
            NRF = IC + 1
            IF( I.EQ.LL ) THEN
               SQREI = SQRE
            ELSE
               SQREI = 1
            END IF
            VFI = VF + NLF - 1
            VLI = VL + NLF - 1
            IDXQI = IDXQ + NLF - 1
            ALPHA = D( IC )
            BETA = E( IC )
            IF( ICOMPQ.EQ.0 ) THEN
               CALL DLASD6( ICOMPQ, NL, NR, SQREI, D( NLF ),
     $                      WORK( VFI ), WORK( VLI ), ALPHA, BETA,
     $                      IWORK( IDXQI ), PERM, GIVPTR( 1 ), GIVCOL,
     $                      LDGCOL, GIVNUM, LDU, POLES, DIFL, DIFR, Z,
     $                      K( 1 ), C( 1 ), S( 1 ), WORK( NWORK1 ),
     $                      IWORK( IWK ), INFO )
            ELSE
               J = J - 1
               CALL DLASD6( ICOMPQ, NL, NR, SQREI, D( NLF ),
     $                      WORK( VFI ), WORK( VLI ), ALPHA, BETA,
     $                      IWORK( IDXQI ), PERM( NLF, LVL ),
```

```
     $                     GIVPTR( J ), GIVCOL( NLF, LVL2 ), LDGCOL,
     $                     GIVNUM( NLF, LVL2 ), LDU,
     $                     POLES( NLF, LVL2 ), DIFL( NLF, LVL ),
     $                     DIFR( NLF, LVL2 ), Z( NLF, LVL ), K( J ),
     $                     C( J ), S( J ), WORK( NWORK1 ),
     $                     IWORK( IWK ), INFO )
            END IF
            IF( INFO.NE.0 ) THEN
               RETURN
            END IF
   40    CONTINUE
   50 CONTINUE
*
      RETURN
*
*     End of DLASDA
*
      END
```

———-

90.178.4 lisp code

— dlasda.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dlasda
         (icompq smlsiz n sqre d e u ldu vt k difl difr z poles givptr givcol
          ldgcol perm givnum c s work iwork info)
    (declare (type (array f2cl-lib:integer4 (*)) iwork perm givcol givptr k)
             (type (array double-float (*)) work s c givnum poles z difr difl
              vt u e d)
             (type (f2cl-lib:integer4) info ldgcol ldu sqre n smlsiz icompq))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%)
         (u double-float u-%data% u-%offset%)
         (vt double-float vt-%data% vt-%offset%)
         (difl double-float difl-%data% difl-%offset%)
         (difr double-float difr-%data% difr-%offset%)
         (z double-float z-%data% z-%offset%)
         (poles double-float poles-%data% poles-%offset%)
         (givnum double-float givnum-%data% givnum-%offset%)
         (c double-float c-%data% c-%offset%)
```

```
    (s double-float s-%data% s-%offset%)
    (work double-float work-%data% work-%offset%)
    (k f2cl-lib:integer4 k-%data% k-%offset%)
    (givptr f2cl-lib:integer4 givptr-%data% givptr-%offset%)
    (givcol f2cl-lib:integer4 givcol-%data% givcol-%offset%)
    (perm f2cl-lib:integer4 perm-%data% perm-%offset%)
    (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
  (prog ((alpha 0.0d0) (beta 0.0d0) (i 0) (i1 0) (ic 0) (idxq 0) (idxqi 0)
         (im1 0) (inode 0) (itemp 0) (iwk 0) (j 0) (lf 0) (ll 0) (lvl 0)
         (lvl2 0) (m 0) (ncc 0) (nd 0) (ndb1 0) (ndiml 0) (ndimr 0) (nl 0)
         (nlf 0) (nlp1 0) (nlvl 0) (nr 0) (nrf 0) (nrp1 0) (nru 0)
         (nwork1 0) (nwork2 0) (smlszp 0) (sqrei 0) (vf 0) (vfi 0) (vl 0)
         (vli 0))
    (declare (type (double-float) alpha beta)
             (type (f2cl-lib:integer4) i i1 ic idxq idxqi im1 inode itemp
              iwk j lf ll lvl lvl2 m ncc nd ndb1 ndiml ndimr nl nlf nlp1
              nlvl nr nrf nrp1 nru nwork1 nwork2 smlszp sqrei vf vfi vl
              vli))
    (setf info 0)
    (cond ((or (< icompq 0) (> icompq 1)) (setf info -1))
          ((< smlsiz 3) (setf info -2)) ((< n 0) (setf info -3))
          ((or (< sqre 0) (> sqre 1)) (setf info -4))
          ((< ldu (f2cl-lib:int-add n sqre)) (setf info -8))
          ((< ldgcol n) (setf info -17)))
    (cond
     ((/= info 0) (xerbla "DLASDA" (f2cl-lib:int-sub info))
      (go end_label)))
    (setf m (f2cl-lib:int-add n sqre))
    (cond
     ((<= n smlsiz)
      (cond
       ((= icompq 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11 var-12 var-13 var-14 var-15)
            (dlasdq "U" sqre n 0 0 0 d e vt ldu u ldu u ldu work info)
          (declare
           (ignore var-0 var-3 var-4 var-5 var-6 var-7 var-8 var-10 var-12
            var-14))
          (when var-1 (setf sqre var-1))
          (when var-2 (setf n var-2))
          (when var-9 (setf ldu var-9))
          (when var-11 (setf ldu var-11))
          (when var-13 (setf ldu var-13))
          (when var-15 (setf info var-15))))
       (t
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11 var-12 var-13 var-14 var-15)
            (dlasdq "U" sqre n m n 0 d e vt ldu u ldu u ldu work info)
```

```
      (declare
       (ignore var-0 var-5 var-6 var-7 var-8 var-10 var-12 var-14))
      (when var-1 (setf sqre var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf m var-3))
      (when var-4 (setf n var-4))
      (when var-9 (setf ldu var-9))
      (when var-11 (setf ldu var-11))
      (when var-13 (setf ldu var-13))
      (when var-15 (setf info var-15)))))
  (go end_label)))
(setf inode 1)
(setf ndiml (f2cl-lib:int-add inode n))
(setf ndimr (f2cl-lib:int-add ndiml n))
(setf idxq (f2cl-lib:int-add ndimr n))
(setf iwk (f2cl-lib:int-add idxq n))
(setf ncc 0)
(setf nru 0)
(setf smlszp (f2cl-lib:int-add smlsiz 1))
(setf vf 1)
(setf vl (f2cl-lib:int-add vf m))
(setf nwork1 (f2cl-lib:int-add vl m))
(setf nwork2
        (f2cl-lib:int-add nwork1 (f2cl-lib:int-mul smlszp smlszp)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (dlasdt n nlvl nd
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (inode)
                           ((1 *)) iwork-%offset%)
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (ndiml)
                           ((1 *)) iwork-%offset%)
     (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4 (ndimr)
                           ((1 *)) iwork-%offset%)
     smlsiz)
  (declare (ignore var-3 var-4 var-5))
  (when var-0 (setf n var-0))
  (when var-1 (setf nlvl var-1))
  (when var-2 (setf nd var-2))
  (when var-6 (setf smlsiz var-6)))
(setf ndb1 (the f2cl-lib:integer4 (truncate (+ nd 1) 2)))
(f2cl-lib:fdo (i ndb1 (f2cl-lib:int-add i 1))
              ((> i nd) nil)
  (tagbody
    (setf i1 (f2cl-lib:int-sub i 1))
    (setf ic
            (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add inode i1))
                           ((1 *)) iwork-%offset%))
    (setf nl
            (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add ndiml i1))
                           ((1 *)) iwork-%offset%))
    (setf nlp1 (f2cl-lib:int-add nl 1))
```

```
(setf nr
        (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add ndimr i1))
                       ((1 *)) iwork-%offset%))
(setf nlf (f2cl-lib:int-sub ic nl))
(setf nrf (f2cl-lib:int-add ic 1))
(setf idxqi (f2cl-lib:int-sub (f2cl-lib:int-add idxq nlf) 2))
(setf vfi (f2cl-lib:int-sub (f2cl-lib:int-add vf nlf) 1))
(setf vli (f2cl-lib:int-sub (f2cl-lib:int-add vl nlf) 1))
(setf sqrei 1)
(cond
 ((= icompq 0)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" nlp1 nlp1 zero one
       (f2cl-lib:array-slice work-%data% double-float (nwork1)
                             ((1 *)) work-%offset%)
       smlszp)
    (declare (ignore var-0 var-5))
    (when var-1 (setf nlp1 var-1))
    (when var-2 (setf nlp1 var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf smlszp var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (dlasdq "U" sqrei nl nlp1 nru ncc
       (f2cl-lib:array-slice d-%data% double-float (nlf) ((1 *))
                             d-%offset%)
       (f2cl-lib:array-slice e-%data% double-float (nlf) ((1 *))
                             e-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (nwork1)
                             ((1 *)) work-%offset%)
       smlszp
       (f2cl-lib:array-slice work-%data% double-float (nwork2)
                             ((1 *)) work-%offset%)
       nl
       (f2cl-lib:array-slice work-%data% double-float (nwork2)
                             ((1 *)) work-%offset%)
       nl
       (f2cl-lib:array-slice work-%data% double-float (nwork2)
                             ((1 *)) work-%offset%)
       info)
    (declare (ignore var-0 var-6 var-7 var-8 var-10 var-12 var-14))
    (when var-1 (setf sqrei var-1))
    (when var-2 (setf nl var-2))
    (when var-3 (setf nlp1 var-3))
    (when var-4 (setf nru var-4))
    (when var-5 (setf ncc var-5))
    (when var-9 (setf smlszp var-9))
    (when var-11 (setf nl var-11))
```

```
    (when var-13 (setf nl var-13))
    (when var-15 (setf info var-15)))
  (setf itemp
          (f2cl-lib:int-add nwork1 (f2cl-lib:int-mul nl smlszp)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy nlp1
       (f2cl-lib:array-slice work-%data% double-float (nwork1)
                             ((1 *)) work-%offset%)
       1
       (f2cl-lib:array-slice work-%data% double-float (vfi) ((1 *))
                             work-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf nlp1 var-0)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy nlp1
       (f2cl-lib:array-slice work-%data% double-float (itemp)
                             ((1 *)) work-%offset%)
       1
       (f2cl-lib:array-slice work-%data% double-float (vli) ((1 *))
                             work-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf nlp1 var-0))))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" nl nl zero one
       (f2cl-lib:array-slice u-%data% double-float (nlf 1)
                             ((1 ldu) (1 *)) u-%offset%)
       ldu)
    (declare (ignore var-0 var-5))
    (when var-1 (setf nl var-1))
    (when var-2 (setf nl var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" nlp1 nlp1 zero one
       (f2cl-lib:array-slice vt-%data% double-float (nlf 1)
                             ((1 ldu) (1 *)) vt-%offset%)
       ldu)
    (declare (ignore var-0 var-5))
    (when var-1 (setf nlp1 var-1))
    (when var-2 (setf nlp1 var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
```

```
        (dlasdq "U" sqrei nl nlp1 nl ncc
         (f2cl-lib:array-slice d-%data% double-float (nlf) ((1 *))
                               d-%offset%)
         (f2cl-lib:array-slice e-%data% double-float (nlf) ((1 *))
                               e-%offset%)
         (f2cl-lib:array-slice vt-%data% double-float (nlf 1)
                               ((1 ldu) (1 *)) vt-%offset%)
         ldu
         (f2cl-lib:array-slice u-%data% double-float (nlf 1)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu
         (f2cl-lib:array-slice u-%data% double-float (nlf 1)
                               ((1 ldu) (1 *)) u-%offset%)
         ldu
         (f2cl-lib:array-slice work-%data% double-float (nwork1)
                               ((1 *)) work-%offset%)
         info)
      (declare (ignore var-0 var-6 var-7 var-8 var-10 var-12 var-14))
      (when var-1 (setf sqrei var-1))
      (when var-2 (setf nl var-2))
      (when var-3 (setf nlp1 var-3))
      (when var-4 (setf nl var-4))
      (when var-5 (setf ncc var-5))
      (when var-9 (setf ldu var-9))
      (when var-11 (setf ldu var-11))
      (when var-13 (setf ldu var-13))
      (when var-15 (setf info var-15)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy nlp1
         (f2cl-lib:array-slice vt-%data% double-float (nlf 1)
                               ((1 ldu) (1 *)) vt-%offset%)
         1
         (f2cl-lib:array-slice work-%data% double-float (vfi) ((1 *))
                               work-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf nlp1 var-0)))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy nlp1
         (f2cl-lib:array-slice vt-%data% double-float (nlf nlp1)
                               ((1 ldu) (1 *)) vt-%offset%)
         1
         (f2cl-lib:array-slice work-%data% double-float (vli) ((1 *))
                               work-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf nlp1 var-0)))))
  (cond ((/= info 0) (go end_label)))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j nl) nil)
```

```
  (tagbody
    (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add idxqi j))
                         ((1 *)) iwork-%offset%)
          j)
   label10))
(cond ((and (= i nd) (= sqre 0)) (setf sqrei 0))
      (t (setf sqrei 1)))
(setf idxqi (f2cl-lib:int-add idxqi nlp1))
(setf vfi (f2cl-lib:int-add vfi nlp1))
(setf vli (f2cl-lib:int-add vli nlp1))
(setf nrp1 (f2cl-lib:int-add nr sqrei))
(cond
 ((= icompq 0)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" nrp1 nrp1 zero one
       (f2cl-lib:array-slice work-%data% double-float (nwork1)
                             ((1 *)) work-%offset%)
       smlszp)
    (declare (ignore var-0 var-5))
    (when var-1 (setf nrp1 var-1))
    (when var-2 (setf nrp1 var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf smlszp var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14 var-15)
      (dlasdq "U" sqrei nr nrp1 nru ncc
       (f2cl-lib:array-slice d-%data% double-float (nrf) ((1 *))
                             d-%offset%)
       (f2cl-lib:array-slice e-%data% double-float (nrf) ((1 *))
                             e-%offset%)
       (f2cl-lib:array-slice work-%data% double-float (nwork1)
                             ((1 *)) work-%offset%)
       smlszp
       (f2cl-lib:array-slice work-%data% double-float (nwork2)
                             ((1 *)) work-%offset%)
       nr
       (f2cl-lib:array-slice work-%data% double-float (nwork2)
                             ((1 *)) work-%offset%)
       nr
       (f2cl-lib:array-slice work-%data% double-float (nwork2)
                             ((1 *)) work-%offset%)
       info)
    (declare (ignore var-0 var-6 var-7 var-8 var-10 var-12 var-14))
    (when var-1 (setf sqrei var-1))
    (when var-2 (setf nr var-2))
    (when var-3 (setf nrp1 var-3))
    (when var-4 (setf nru var-4))
    (when var-5 (setf ncc var-5))
```

```
    (when var-9 (setf smlszp var-9))
    (when var-11 (setf nr var-11))
    (when var-13 (setf nr var-13))
    (when var-15 (setf info var-15)))
  (setf itemp
          (f2cl-lib:int-add nwork1
                            (f2cl-lib:int-mul
                             (f2cl-lib:int-sub nrp1 1) smlszp)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy nrp1
       (f2cl-lib:array-slice work-%data% double-float (nwork1)
                             ((1 *)) work-%offset%)
       1
       (f2cl-lib:array-slice work-%data% double-float (vfi) ((1 *))
                             work-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf nrp1 var-0)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dcopy nrp1
       (f2cl-lib:array-slice work-%data% double-float (itemp)
                             ((1 *)) work-%offset%)
       1
       (f2cl-lib:array-slice work-%data% double-float (vli) ((1 *))
                             work-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf nrp1 var-0))))
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" nr nr zero one
       (f2cl-lib:array-slice u-%data% double-float (nrf 1)
                             ((1 ldu) (1 *)) u-%offset%)
       ldu)
    (declare (ignore var-0 var-5))
    (when var-1 (setf nr var-1))
    (when var-2 (setf nr var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldu var-6)))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlaset "A" nrp1 nrp1 zero one
       (f2cl-lib:array-slice vt-%data% double-float (nrf 1)
                             ((1 ldu) (1 *)) vt-%offset%)
       ldu)
    (declare (ignore var-0 var-5))
    (when var-1 (setf nrp1 var-1))
    (when var-2 (setf nrp1 var-2))
    (when var-3 (setf zero var-3))
    (when var-4 (setf one var-4))
```

```
  (when var-6 (setf ldu var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15)
    (dlasdq "U" sqrei nr nrp1 nr ncc
     (f2cl-lib:array-slice d-%data% double-float (nrf) ((1 *))
                           d-%offset%)
     (f2cl-lib:array-slice e-%data% double-float (nrf) ((1 *))
                           e-%offset%)
     (f2cl-lib:array-slice vt-%data% double-float (nrf 1)
                           ((1 ldu) (1 *)) vt-%offset%)
     ldu
     (f2cl-lib:array-slice u-%data% double-float (nrf 1)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu
     (f2cl-lib:array-slice u-%data% double-float (nrf 1)
                           ((1 ldu) (1 *)) u-%offset%)
     ldu
     (f2cl-lib:array-slice work-%data% double-float (nwork1)
                           ((1 *)) work-%offset%)
     info)
  (declare (ignore var-0 var-6 var-7 var-8 var-10 var-12 var-14))
  (when var-1 (setf sqrei var-1))
  (when var-2 (setf nr var-2))
  (when var-3 (setf nrp1 var-3))
  (when var-4 (setf nr var-4))
  (when var-5 (setf ncc var-5))
  (when var-9 (setf ldu var-9))
  (when var-11 (setf ldu var-11))
  (when var-13 (setf ldu var-13))
  (when var-15 (setf info var-15)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dcopy nrp1
     (f2cl-lib:array-slice vt-%data% double-float (nrf 1)
                           ((1 ldu) (1 *)) vt-%offset%)
     1
     (f2cl-lib:array-slice work-%data% double-float (vfi) ((1 *))
                           work-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf nrp1 var-0)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dcopy nrp1
     (f2cl-lib:array-slice vt-%data% double-float (nrf nrp1)
                           ((1 ldu) (1 *)) vt-%offset%)
     1
     (f2cl-lib:array-slice work-%data% double-float (vli) ((1 *))
                           work-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-4))
```

```
            (when var-0 (setf nrp1 var-0)))))
        (cond ((/= info 0) (go end_label)))
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j nr) nil)
          (tagbody
            (setf (f2cl-lib:fref iwork-%data% ((f2cl-lib:int-add idxqi j))
                                 ((1 *)) iwork-%offset%)
                  j)
           label20))
       label30))
    (setf j (expt 2 nlvl))
    (f2cl-lib:fdo (lvl nlvl (f2cl-lib:int-add lvl (f2cl-lib:int-sub 1)))
                  ((> lvl 1) nil)
      (tagbody
        (setf lvl2 (f2cl-lib:int-sub (f2cl-lib:int-mul lvl 2) 1))
        (cond ((= lvl 1) (setf lf 1) (setf ll 1))
              (t (setf lf (expt 2 (f2cl-lib:int-sub lvl 1)))
               (setf ll (f2cl-lib:int-sub (f2cl-lib:int-mul 2 lf) 1))))
        (f2cl-lib:fdo (i lf (f2cl-lib:int-add i 1))
                      ((> i ll) nil)
          (tagbody
            (setf im1 (f2cl-lib:int-sub i 1))
            (setf ic
                    (f2cl-lib:fref iwork-%data%
                                   ((f2cl-lib:int-add inode im1)) ((1 *))
                                   iwork-%offset%))
            (setf nl
                    (f2cl-lib:fref iwork-%data%
                                   ((f2cl-lib:int-add ndiml im1)) ((1 *))
                                   iwork-%offset%))
            (setf nr
                    (f2cl-lib:fref iwork-%data%
                                   ((f2cl-lib:int-add ndimr im1)) ((1 *))
                                   iwork-%offset%))
            (setf nlf (f2cl-lib:int-sub ic nl))
            (setf nrf (f2cl-lib:int-add ic 1))
            (cond ((= i ll) (setf sqrei sqre)) (t (setf sqrei 1)))
            (setf vfi (f2cl-lib:int-sub (f2cl-lib:int-add vf nlf) 1))
            (setf vli (f2cl-lib:int-sub (f2cl-lib:int-add vl nlf) 1))
            (setf idxqi (f2cl-lib:int-sub (f2cl-lib:int-add idxq nlf) 1))
            (setf alpha (f2cl-lib:fref d-%data% (ic) ((1 *)) d-%offset%))
            (setf beta (f2cl-lib:fref e-%data% (ic) ((1 *)) e-%offset%))
            (cond
             ((= icompq 0)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                   var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
                   var-17 var-18 var-19 var-20 var-21 var-22 var-23 var-24
                   var-25)
                  (dlasd6 icompq nl nr sqrei
```

```
        (f2cl-lib:array-slice d-%data% double-float (nlf)
                              ((1 *)) d-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (vfi)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (vli)
                              ((1 *)) work-%offset%)
        alpha beta
        (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                              (idxqi) ((1 *)) iwork-%offset%)
        perm
        (f2cl-lib:fref givptr-%data% (1) ((1 *))
                       givptr-%offset%)
        givcol ldgcol givnum ldu poles difl difr z
        (f2cl-lib:fref k-%data% (1) ((1 *)) k-%offset%)
        (f2cl-lib:fref c-%data% (1) ((1 *)) c-%offset%)
        (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%)
        (f2cl-lib:array-slice work-%data% double-float (nwork1)
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                              (iwk) ((1 *)) iwork-%offset%)
        info)
    (declare
     (ignore var-4 var-5 var-6 var-9 var-10 var-12 var-14
      var-16 var-17 var-18 var-19 var-23 var-24))
    (setf icompq var-0)
    (setf nl var-1)
    (setf nr var-2)
    (setf sqrei var-3)
    (setf alpha var-7)
    (setf beta var-8)
    (setf (f2cl-lib:fref givptr-%data% (1) ((1 *))
                         givptr-%offset%)
            var-11)
    (setf ldgcol var-13)
    (setf ldu var-15)
    (setf (f2cl-lib:fref k-%data% (1) ((1 *)) k-%offset%)
            var-20)
    (setf (f2cl-lib:fref c-%data% (1) ((1 *)) c-%offset%)
            var-21)
    (setf (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%)
            var-22)
    (setf info var-25)))
  (t (setf j (f2cl-lib:int-sub j 1))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
        var-17 var-18 var-19 var-20 var-21 var-22 var-23 var-24
        var-25)
       (dlasd6 icompq nl nr sqrei
        (f2cl-lib:array-slice d-%data% double-float (nlf)
```

```
                        ((1 *)) d-%offset%)
   (f2cl-lib:array-slice work-%data% double-float (vfi)
                        ((1 *)) work-%offset%)
   (f2cl-lib:array-slice work-%data% double-float (vli)
                        ((1 *)) work-%offset%)
   alpha beta
   (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                        (idxqi) ((1 *)) iwork-%offset%)
   (f2cl-lib:array-slice perm-%data% f2cl-lib:integer4
                        (nlf lvl) ((1 ldgcol) (1 *))
                        perm-%offset%)
   (f2cl-lib:fref givptr-%data% (j) ((1 *))
                  givptr-%offset%)
   (f2cl-lib:array-slice givcol-%data% f2cl-lib:integer4
                        (nlf lvl2) ((1 ldgcol) (1 *))
                        givcol-%offset%)
   ldgcol
   (f2cl-lib:array-slice givnum-%data% double-float
                        (nlf lvl2) ((1 ldu) (1 *))
                        givnum-%offset%)
   ldu
   (f2cl-lib:array-slice poles-%data% double-float
                        (nlf lvl2) ((1 ldu) (1 *))
                        poles-%offset%)
   (f2cl-lib:array-slice difl-%data% double-float (nlf lvl)
                        ((1 ldu) (1 *)) difl-%offset%)
   (f2cl-lib:array-slice difr-%data% double-float
                        (nlf lvl2) ((1 ldu) (1 *))
                        difr-%offset%)
   (f2cl-lib:array-slice z-%data% double-float (nlf lvl)
                        ((1 ldu) (1 *)) z-%offset%)
   (f2cl-lib:fref k-%data% (j) ((1 *)) k-%offset%)
   (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
   (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
   (f2cl-lib:array-slice work-%data% double-float (nwork1)
                        ((1 *)) work-%offset%)
   (f2cl-lib:array-slice iwork-%data% f2cl-lib:integer4
                        (iwk) ((1 *)) iwork-%offset%)
   info)
(declare
 (ignore var-4 var-5 var-6 var-9 var-10 var-12 var-14
  var-16 var-17 var-18 var-19 var-23 var-24))
(setf icompq var-0)
(setf nl var-1)
(setf nr var-2)
(setf sqrei var-3)
(setf alpha var-7)
(setf beta var-8)
(setf (f2cl-lib:fref givptr-%data% (j) ((1 *))
                     givptr-%offset%)
```

```
                        var-11)
                    (setf ldgcol var-13)
                    (setf ldu var-15)
                    (setf (f2cl-lib:fref k-%data% (j) ((1 *)) k-%offset%)
                        var-20)
                    (setf (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%)
                        var-21)
                    (setf (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%)
                        var-22)
                    (setf info var-25))))
                (cond ((/= info 0) (go end_label)))
               label40))
            label50))
         (go end_label)
        end_label
         (return
          (values icompq smlsiz n sqre nil nil nil ldu nil nil nil nil nil nil
                  nil nil ldgcol nil nil nil nil nil nil info))))))
```

———-

90.179 dlasdq

90.179.1 Axiom unit tests

— **dlasdq.input** —

```
)set break resume
)sys rm -f dlasdq.output
)spool dlasdq.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.179.2 Axiom help page

— **dlasdq.help** —

```
==================================================================
```

```
dlasdq examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DLASDQ  -  the  singular  value decomposition (SVD) of a real (upper or
       lower) bidiagonal matrix with diagonal D and offdiagonal E,  accumulat-
       ing the transformations if desired

SYNOPSIS
       SUBROUTINE DLASDQ( UPLO,  SQRE,  N,  NCVT, NRU, NCC, D, E, VT, LDVT, U,
                          LDU, C, LDC, WORK, INFO )

           CHARACTER      UPLO

           INTEGER        INFO, LDC, LDU, LDVT, N, NCC, NCVT, NRU, SQRE

           DOUBLE         PRECISION C( LDC, * ), D( * ), E( * ), U( LDU, *  ),
                          VT( LDVT, * ), WORK( * )

  Purpose
  =======

  DLASDQ computes the singular value decomposition (SVD) of a real
  (upper or lower) bidiagonal matrix with diagonal D and offdiagonal
  E, accumulating the transformations if desired. Letting B denote
  the input bidiagonal matrix, the algorithm computes orthogonal
  matrices Q and P such that B = Q * S * P' (P' denotes the transpose
  of P). The singular values S are overwritten on D.

  The input matrix U  is changed to U  * Q  if desired.
  The input matrix VT is changed to P' * VT if desired.
  The input matrix C  is changed to Q' * C  if desired.

  See "Computing  Small Singular Values of Bidiagonal Matrices With
  Guaranteed High Relative Accuracy," by J. Demmel and W. Kahan,
  LAPACK Working Note #3, for a detailed description of the algorithm.

  Arguments
  =========

  UPLO  (input) CHARACTER*1
        On entry, UPLO specifies whether the input bidiagonal matrix
        is upper or lower bidiagonal, and wether it is square are
        not.
           UPLO = 'U' or 'u'   B is upper bidiagonal.
           UPLO = 'L' or 'l'   B is lower bidiagonal.
```

```
SQRE  (input) INTEGER
      = 0: then the input matrix is N-by-N.
      = 1: then the input matrix is N-by-(N+1) if UPLU = 'U' and
           (N+1)-by-N if UPLU = 'L'.

      The bidiagonal matrix has
      N = NL + NR + 1 rows and
      M = N + SQRE >= N columns.

N     (input) INTEGER
      On entry, N specifies the number of rows and columns
      in the matrix. N must be at least 0.

NCVT  (input) INTEGER
      On entry, NCVT specifies the number of columns of
      the matrix VT. NCVT must be at least 0.

NRU   (input) INTEGER
      On entry, NRU specifies the number of rows of
      the matrix U. NRU must be at least 0.

NCC   (input) INTEGER
      On entry, NCC specifies the number of columns of
      the matrix C. NCC must be at least 0.

D     (input/output) DOUBLE PRECISION array, dimension (N)
      On entry, D contains the diagonal entries of the
      bidiagonal matrix whose SVD is desired. On normal exit,
      D contains the singular values in ascending order.

E     (input/output) DOUBLE PRECISION array.
      dimension is (N-1) if SQRE = 0 and N if SQRE = 1.
      On entry, the entries of E contain the offdiagonal entries
      of the bidiagonal matrix whose SVD is desired. On normal
      exit, E will contain 0. If the algorithm does not converge,
      D and E will contain the diagonal and superdiagonal entries
      of a bidiagonal matrix orthogonally equivalent to the one
      given as input.

VT    (input/output) DOUBLE PRECISION array, dimension (LDVT, NCVT)
      On entry, contains a matrix which on exit has been
      premultiplied by P', dimension N-by-NCVT if SQRE = 0
      and (N+1)-by-NCVT if SQRE = 1 (not referenced if NCVT=0).

LDVT  (input) INTEGER
      On entry, LDVT specifies the leading dimension of VT as
      declared in the calling (sub) program. LDVT must be at
      least 1. If NCVT is nonzero LDVT must also be at least N.
```

```
 U     (input/output) DOUBLE PRECISION array, dimension (LDU, N)
       On entry, contains a  matrix which on exit has been
       postmultiplied by Q, dimension NRU-by-N if SQRE = 0
       and NRU-by-(N+1) if SQRE = 1 (not referenced if NRU=0).

 LDU   (input) INTEGER
       On entry, LDU  specifies the leading dimension of U as
       declared in the calling (sub) program. LDU must be at
       least max( 1, NRU ) .

 C     (input/output) DOUBLE PRECISION array, dimension (LDC, NCC)
       On entry, contains an N-by-NCC matrix which on exit
       has been premultiplied by Q'  dimension N-by-NCC if SQRE = 0
       and (N+1)-by-NCC if SQRE = 1 (not referenced if NCC=0).

 LDC   (input) INTEGER
       On entry, LDC  specifies the leading dimension of C as
       declared in the calling (sub) program. LDC must be at
       least 1. If NCC is nonzero, LDC must also be at least N.

 WORK  (workspace) DOUBLE PRECISION array, dimension (4*N)
       Workspace. Only referenced if one of NCVT, NRU, or NCC is
       nonzero, and if N is at least 2.

 INFO  (output) INTEGER
       On exit, a value of 0 indicates a successful exit.
       If INFO < 0, argument number -INFO is illegal.
       If INFO > 0, the algorithm did not converge, and INFO
       specifies how many superdiagonals did not converge.

 Further Details
 ===============

 Based on contributions by
    Ming Gu and Huan Ren, Computer Science Division, University of
    California at Berkeley, USA
```

———-

90.179.3 fortran code

— dlasdq.f —

```
      SUBROUTINE DLASDQ( UPLO, SQRE, N, NCVT, NRU, NCC, D, E, VT, LDVT,
     $                   U, LDU, C, LDC, WORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.0) --
```

```
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1999
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            INFO, LDC, LDU, LDVT, N, NCC, NCVT, NRU, SQRE
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   C( LDC, * ), D( * ), E( * ), U( LDU, * ),
     $                   VT( LDVT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ROTATE
      INTEGER            I, ISUB, IUPLO, J, NP1, SQRE1
      DOUBLE PRECISION   CS, R, SMIN, SN
*     ..
*     .. External Subroutines ..
      EXTERNAL           DBDSQR, DLARTG, DLASR, DSWAP, XERBLA
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters.
*
      INFO = 0
      IUPLO = 0
      IF( LSAME( UPLO, 'U' ) )
     $   IUPLO = 1
      IF( LSAME( UPLO, 'L' ) )
     $   IUPLO = 2
      IF( IUPLO.EQ.0 ) THEN
         INFO = -1
      ELSE IF( ( SQRE.LT.0 ) .OR. ( SQRE.GT.1 ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
```

```
      ELSE IF( NCVT.LT.0 ) THEN
         INFO = -4
      ELSE IF( NRU.LT.0 ) THEN
         INFO = -5
      ELSE IF( NCC.LT.0 ) THEN
         INFO = -6
      ELSE IF( ( NCVT.EQ.0 .AND. LDVT.LT.1 ) .OR.
     $         ( NCVT.GT.0 .AND. LDVT.LT.MAX( 1, N ) ) ) THEN
         INFO = -10
      ELSE IF( LDU.LT.MAX( 1, NRU ) ) THEN
         INFO = -12
      ELSE IF( ( NCC.EQ.0 .AND. LDC.LT.1 ) .OR.
     $         ( NCC.GT.0 .AND. LDC.LT.MAX( 1, N ) ) ) THEN
         INFO = -14
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASDQ', -INFO )
         RETURN
      END IF
      IF( N.EQ.0 )
     $   RETURN
*
*     ROTATE is true if any singular vectors desired, false otherwise
*
      ROTATE = ( NCVT.GT.0 ) .OR. ( NRU.GT.0 ) .OR. ( NCC.GT.0 )
      NP1 = N + 1
      SQRE1 = SQRE
*
*     If matrix non-square upper bidiagonal, rotate to be lower
*     bidiagonal.  The rotations are on the right.
*
      IF( ( IUPLO.EQ.1 ) .AND. ( SQRE1.EQ.1 ) ) THEN
         DO 10 I = 1, N - 1
            CALL DLARTG( D( I ), E( I ), CS, SN, R )
            D( I ) = R
            E( I ) = SN*D( I+1 )
            D( I+1 ) = CS*D( I+1 )
            IF( ROTATE ) THEN
               WORK( I ) = CS
               WORK( N+I ) = SN
            END IF
   10    CONTINUE
         CALL DLARTG( D( N ), E( N ), CS, SN, R )
         D( N ) = R
         E( N ) = ZERO
         IF( ROTATE ) THEN
            WORK( N ) = CS
            WORK( N+N ) = SN
         END IF
         IUPLO = 2
```

```
         SQRE1 = 0
*
*        Update singular vectors if desired.
*
         IF( NCVT.GT.0 )
     $      CALL DLASR( 'L', 'V', 'F', NP1, NCVT, WORK( 1 ),
     $                  WORK( NP1 ), VT, LDVT )
      END IF
*
*     If matrix lower bidiagonal, rotate to be upper bidiagonal
*     by applying Givens rotations on the left.
*
      IF( IUPLO.EQ.2 ) THEN
         DO 20 I = 1, N - 1
            CALL DLARTG( D( I ), E( I ), CS, SN, R )
            D( I ) = R
            E( I ) = SN*D( I+1 )
            D( I+1 ) = CS*D( I+1 )
            IF( ROTATE ) THEN
               WORK( I ) = CS
               WORK( N+I ) = SN
            END IF
   20    CONTINUE
*
*        If matrix (N+1)-by-N lower bidiagonal, one additional
*        rotation is needed.
*
         IF( SQRE1.EQ.1 ) THEN
            CALL DLARTG( D( N ), E( N ), CS, SN, R )
            D( N ) = R
            IF( ROTATE ) THEN
               WORK( N ) = CS
               WORK( N+N ) = SN
            END IF
         END IF
*
*        Update singular vectors if desired.
*
         IF( NRU.GT.0 ) THEN
            IF( SQRE1.EQ.0 ) THEN
               CALL DLASR( 'R', 'V', 'F', NRU, N, WORK( 1 ),
     $                     WORK( NP1 ), U, LDU )
            ELSE
               CALL DLASR( 'R', 'V', 'F', NRU, NP1, WORK( 1 ),
     $                     WORK( NP1 ), U, LDU )
            END IF
         END IF
         IF( NCC.GT.0 ) THEN
            IF( SQRE1.EQ.0 ) THEN
               CALL DLASR( 'L', 'V', 'F', N, NCC, WORK( 1 ),
```

```
     $                     WORK( NP1 ), C, LDC )
            ELSE
               CALL DLASR( 'L', 'V', 'F', NP1, NCC, WORK( 1 ),
     $                     WORK( NP1 ), C, LDC )
            END IF
         END IF
      END IF
*
*     Call DBDSQR to compute the SVD of the reduced real
*     N-by-N upper bidiagonal matrix.
*
      CALL DBDSQR( 'U', N, NCVT, NRU, NCC, D, E, VT, LDVT, U, LDU, C,
     $             LDC, WORK, INFO )
*
*     Sort the singular values into ascending order (insertion sort on
*     singular values, but only one transposition per singular vector)
*
      DO 40 I = 1, N
*
*        Scan for smallest D(I).
*
         ISUB = I
         SMIN = D( I )
         DO 30 J = I + 1, N
            IF( D( J ).LT.SMIN ) THEN
               ISUB = J
               SMIN = D( J )
            END IF
   30    CONTINUE
         IF( ISUB.NE.I ) THEN
*
*           Swap singular values and vectors.
*
            D( ISUB ) = D( I )
            D( I ) = SMIN
            IF( NCVT.GT.0 )
     $         CALL DSWAP( NCVT, VT( ISUB, 1 ), LDVT, VT( I, 1 ), LDVT )
            IF( NRU.GT.0 )
     $         CALL DSWAP( NRU, U( 1, ISUB ), 1, U( 1, I ), 1 )
            IF( NCC.GT.0 )
     $         CALL DSWAP( NCC, C( ISUB, 1 ), LDC, C( I, 1 ), LDC )
         END IF
   40 CONTINUE
*
      RETURN
*
*     End of DLASDQ
*
      END
```

———-

90.179.4 lisp code

— dlasdq.lisp —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun dlasdq (uplo sqre n ncvt nru ncc d e vt ldvt u ldu c ldc work info)
    (declare (type (array double-float (*)) work c u vt e d)
             (type (f2cl-lib:integer4) info ldc ldu ldvt ncc nru ncvt n sqre)
             (type (string *) uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (d double-float d-%data% d-%offset%)
         (e double-float e-%data% e-%offset%)
         (vt double-float vt-%data% vt-%offset%)
         (u double-float u-%data% u-%offset%)
         (c double-float c-%data% c-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((cs 0.0d0) (r 0.0d0) (smin 0.0d0) (sn 0.0d0) (i 0) (isub 0)
             (iuplo 0) (j 0) (np1 0) (sqre1 0) (rotate nil))
        (declare (type (double-float) cs r smin sn)
                 (type (f2cl-lib:integer4) i isub iuplo j np1 sqre1)
                 (type f2cl-lib:logical rotate))
        (setf info 0)
        (setf iuplo 0)
        (if (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "U")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)
            (setf iuplo 1))
        (if (multiple-value-bind (ret-val var-0 var-1)
                (lsame uplo "L")
              (declare (ignore var-1))
              (when var-0 (setf uplo var-0))
              ret-val)
            (setf iuplo 2))
        (cond ((= iuplo 0) (setf info -1))
              ((or (< sqre 0) (> sqre 1)) (setf info -2))
              ((< n 0) (setf info -3)) ((< ncvt 0) (setf info -4))
              ((< nru 0) (setf info -5)) ((< ncc 0) (setf info -6))
              ((or (and (= ncvt 0) (< ldvt 1))
                   (and (> ncvt 0)
                        (< ldvt
                           (max (the f2cl-lib:integer4 1)
```

```
                          (the f2cl-lib:integer4 n)))))
        (setf info -10))
       ((< ldu
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nru)))
        (setf info -12))
       ((or (and (= ncc 0) (< ldc 1))
            (and (> ncc 0)
                 (< ldc
                    (max (the f2cl-lib:integer4 1)
                         (the f2cl-lib:integer4 n)))))
        (setf info -14)))
(cond
 ((/= info 0) (xerbla "DLASDQ" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(setf rotate (or (> ncvt 0) (> nru 0) (> ncc 0)))
(setf np1 (f2cl-lib:int-add n 1))
(setf sqre1 sqre)
(cond
 ((and (= iuplo 1) (= sqre1 1))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dlartg (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
           (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) cs sn r)
        (declare (ignore var-0 var-1))
        (setf cs var-2)
        (setf sn var-3)
        (setf r var-4))
      (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) r)
      (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
              (* sn
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                ((1 *)) d-%offset%)))
      (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                           d-%offset%)
              (* cs
                 (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                ((1 *)) d-%offset%)))
      (cond
       (rotate
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) cs)
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                             ((1 *)) work-%offset%)
                sn)))
    label10))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dlartg (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
```

```
        (f2cl-lib:fref e-%data% (n) ((1 *)) e-%offset%) cs sn r)
      (declare (ignore var-0 var-1))
      (setf cs var-2)
      (setf sn var-3)
      (setf r var-4))
    (setf (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) r)
    (setf (f2cl-lib:fref e-%data% (n) ((1 *)) e-%offset%) zero)
    (cond
     (rotate
      (setf (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%) cs)
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n n)) ((1 *))
                           work-%offset%)
            sn)))
    (setf iuplo 2) (setf sqre1 0)
    (if (> ncvt 0)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (dlasr "L" "V" "F" np1 ncvt
             (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                                   work-%offset%)
             (f2cl-lib:array-slice work-%data% double-float (np1) ((1 *))
                                   work-%offset%)
             vt ldvt)
          (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
          (when var-3 (setf np1 var-3))
          (when var-4 (setf ncvt var-4))
          (when var-8 (setf ldvt var-8))))))
  (cond
   ((= iuplo 2)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (dlartg (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
             (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%) cs sn r)
          (declare (ignore var-0 var-1))
          (setf cs var-2)
          (setf sn var-3)
          (setf r var-4))
        (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) r)
        (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                (* sn
                   (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                  ((1 *)) d-%offset%)))
        (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1)) ((1 *))
                             d-%offset%)
                (* cs
                   (f2cl-lib:fref d-%data% ((f2cl-lib:int-add i 1))
                                  ((1 *)) d-%offset%)))
        (cond
```

```
      (rotate
       (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) cs)
       (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n i))
                            ((1 *)) work-%offset%)
             sn)))
   label20))
(cond
 ((= sqre1 1)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (dlartg (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
       (f2cl-lib:fref e-%data% (n) ((1 *)) e-%offset%) cs sn r)
    (declare (ignore var-0 var-1))
    (setf cs var-2)
    (setf sn var-3)
    (setf r var-4))
  (setf (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%) r)
  (cond
   (rotate
    (setf (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%) cs)
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add n n)) ((1 *))
                         work-%offset%)
          sn)))))
(cond
 ((> nru 0)
  (cond
   ((= sqre1 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dlasr "R" "V" "F" nru n
         (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                               work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (np1) ((1 *))
                               work-%offset%)
         u ldu)
      (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
      (when var-3 (setf nru var-3))
      (when var-4 (setf n var-4))
      (when var-8 (setf ldu var-8))))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dlasr "R" "V" "F" nru np1
         (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                               work-%offset%)
         (f2cl-lib:array-slice work-%data% double-float (np1) ((1 *))
                               work-%offset%)
         u ldu)
      (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
      (when var-3 (setf nru var-3))
      (when var-4 (setf np1 var-4))
```

```
        (when var-8 (setf ldu var-8)))))))
  (cond
   ((> ncc 0)
    (cond
     ((= sqre1 0)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dlasr "L" "V" "F" n ncc
           (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                                 work-%offset%)
           (f2cl-lib:array-slice work-%data% double-float (np1) ((1 *))
                                 work-%offset%)
           c ldc)
        (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
        (when var-3 (setf n var-3))
        (when var-4 (setf ncc var-4))
        (when var-8 (setf ldc var-8))))
     (t
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dlasr "L" "V" "F" np1 ncc
           (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                                 work-%offset%)
           (f2cl-lib:array-slice work-%data% double-float (np1) ((1 *))
                                 work-%offset%)
           c ldc)
        (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
        (when var-3 (setf np1 var-3))
        (when var-4 (setf ncc var-4))
        (when var-8 (setf ldc var-8)))))))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
     var-11 var-12 var-13 var-14)
    (dbdsqr "U" n ncvt nru ncc d e vt ldvt u ldu c ldc work info)
  (declare (ignore var-0 var-5 var-6 var-7 var-9 var-11 var-13))
  (setf n var-1)
  (setf ncvt var-2)
  (setf nru var-3)
  (setf ncc var-4)
  (setf ldvt var-8)
  (setf ldu var-10)
  (setf ldc var-12)
  (setf info var-14))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf isub i)
    (setf smin (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
    (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                  ((> j n) nil)
```

```
          (tagbody
            (cond
             ((< (f2cl-lib:fref d (j) ((1 *))) smin) (setf isub j)
              (setf smin (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%))))
           label30))
        (cond
         ((/= isub i)
          (setf (f2cl-lib:fref d-%data% (isub) ((1 *)) d-%offset%)
                  (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
          (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) smin)
          (if (> ncvt 0)
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (dswap ncvt
                   (f2cl-lib:array-slice vt-%data% double-float (isub 1)
                                         ((1 ldvt) (1 *)) vt-%offset%)
                   ldvt
                   (f2cl-lib:array-slice vt-%data% double-float (i 1)
                                         ((1 ldvt) (1 *)) vt-%offset%)
                   ldvt)
                (declare (ignore var-1 var-3))
                (when var-0 (setf ncvt var-0))
                (when var-2 (setf ldvt var-2))
                (when var-4 (setf ldvt var-4))))
          (if (> nru 0)
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (dswap nru
                   (f2cl-lib:array-slice u-%data% double-float (1 isub)
                                         ((1 ldu) (1 *)) u-%offset%)
                   1
                   (f2cl-lib:array-slice u-%data% double-float (1 i)
                                         ((1 ldu) (1 *)) u-%offset%)
                   1)
                (declare (ignore var-1 var-2 var-3 var-4))
                (when var-0 (setf nru var-0))))
          (if (> ncc 0)
              (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
                  (dswap ncc
                   (f2cl-lib:array-slice c-%data% double-float (isub 1)
                                         ((1 ldc) (1 *)) c-%offset%)
                   ldc
                   (f2cl-lib:array-slice c-%data% double-float (i 1)
                                         ((1 ldc) (1 *)) c-%offset%)
                   ldc)
                (declare (ignore var-1 var-3))
                (when var-0 (setf ncc var-0))
                (when var-2 (setf ldc var-2))
                (when var-4 (setf ldc var-4))))))
    label40))
 (go end_label)
end_label
```

```
        (return
         (values uplo nil n ncvt nru ncc nil nil nil ldvt nil ldu nil ldc nil
                 info))))))
```

———-

90.180 dlasdt

90.180.1 Axiom unit tests

— **dlasdt.input** —

```
)set break resume
)sys rm -f dlasdt.output
)spool dlasdt.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.180.2 Axiom help page

— **dlasdt.help** —

```
======================================================================
dlasdt examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DLASDT - a tree of subproblems for bidiagonal divide and conquer

SYNOPSIS
       SUBROUTINE DLASDT( N, LVL, ND, INODE, NDIML, NDIMR, MSUB )

           INTEGER        LVL, MSUB, N, ND
```

```
         INTEGER        INODE( * ), NDIML( * ), NDIMR( * )

Purpose
=======

DLASDT creates a tree of subproblems for bidiagonal divide and
conquer.

Arguments
=========

 N      (input) INTEGER
        On entry, the number of diagonal elements of the
        bidiagonal matrix.

 LVL    (output) INTEGER
        On exit, the number of levels on the computation tree.

 ND     (output) INTEGER
        On exit, the number of nodes on the tree.

 INODE  (output) INTEGER array, dimension ( N )
        On exit, centers of subproblems.

 NDIML  (output) INTEGER array, dimension ( N )
        On exit, row dimensions of left children.

 NDIMR  (output) INTEGER array, dimension ( N )
        On exit, row dimensions of right children.

 MSUB   (input) INTEGER.
        On entry, the maximum row dimension each subproblem at the
        bottom of the tree can be of.

Further Details
===============

Based on contributions by
   Ming Gu and Huan Ren, Computer Science Division, University of
   California at Berkeley, USA
```

———-

90.180.3 fortran code

— dlasdt.f —

```
      SUBROUTINE DLASDT( N, LVL, ND, INODE, NDIML, NDIMR, MSUB )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      INTEGER            LVL, MSUB, N, ND
*     ..
*     .. Array Arguments ..
      INTEGER            INODE( * ), NDIML( * ), NDIMR( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IL, IR, LLST, MAXN, NCRNT, NLVL
      DOUBLE PRECISION   TEMP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, INT, LOG, MAX
*     ..
*     .. Executable Statements ..
*
*     Find the number of levels on the tree.
*
      MAXN = MAX( 1, N )
      TEMP = LOG( DBLE( MAXN ) / DBLE( MSUB+1 ) ) / LOG( TWO )
      LVL = INT( TEMP ) + 1
*
      I = N / 2
      INODE( 1 ) = I + 1
      NDIML( 1 ) = I
      NDIMR( 1 ) = N - I - 1
      IL = 0
      IR = 1
      LLST = 1
      DO 20 NLVL = 1, LVL - 1
*
*        Constructing the tree at (NLVL+1)-st level. The number of
*        nodes created on this level is LLST * 2.
*
         DO 10 I = 0, LLST - 1
            IL = IL + 2
            IR = IR + 2
```

```
            NCRNT = LLST + I
            NDIML( IL ) = NDIML( NCRNT ) / 2
            NDIMR( IL ) = NDIML( NCRNT ) - NDIML( IL ) - 1
            INODE( IL ) = INODE( NCRNT ) - NDIMR( IL ) - 1
            NDIML( IR ) = NDIMR( NCRNT ) / 2
            NDIMR( IR ) = NDIMR( NCRNT ) - NDIML( IR ) - 1
            INODE( IR ) = INODE( NCRNT ) + NDIML( IR ) + 1
   10    CONTINUE
         LLST = LLST*2
   20 CONTINUE
      ND = LLST*2 - 1
*
      RETURN
*
*     End of DLASDT
*
      END
```

———-

90.180.4 lisp code

— dlasdt.lisp —

```
(let* ((two 2.0d0))
  (declare (type (double-float 2.0d0 2.0d0) two)
           (ignorable two))
  (defun dlasdt (n lvl nd inode ndiml ndimr msub)
    (declare (type (array f2cl-lib:integer4 (*)) ndimr ndiml inode)
             (type (f2cl-lib:integer4) msub nd lvl n))
    (f2cl-lib:with-multi-array-data
        ((inode f2cl-lib:integer4 inode-%data% inode-%offset%)
         (ndiml f2cl-lib:integer4 ndiml-%data% ndiml-%offset%)
         (ndimr f2cl-lib:integer4 ndimr-%data% ndimr-%offset%))
      (prog ((temp 0.0d0) (i 0) (il 0) (ir 0) (llst 0) (maxn 0) (ncrnt 0)
             (nlvl 0))
        (declare (type (double-float) temp)
                 (type (f2cl-lib:integer4) i il ir llst maxn ncrnt nlvl))
        (setf maxn (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf temp
                (/
                 (f2cl-lib:flog
                  (/ (f2cl-lib:dble maxn)
                     (f2cl-lib:dble (f2cl-lib:int-add msub 1))))
                 (f2cl-lib:flog two)))
        (setf lvl (f2cl-lib:int-add (f2cl-lib:int temp) 1))
        (setf i (the f2cl-lib:integer4 (truncate n 2)))
```

```
(setf (f2cl-lib:fref inode-%data% (1) ((1 *)) inode-%offset%)
        (f2cl-lib:int-add i 1))
(setf (f2cl-lib:fref ndiml-%data% (1) ((1 *)) ndiml-%offset%) i)
(setf (f2cl-lib:fref ndimr-%data% (1) ((1 *)) ndimr-%offset%)
        (f2cl-lib:int-sub n i 1))
(setf il 0)
(setf ir 1)
(setf llst 1)
(f2cl-lib:fdo (nlvl 1 (f2cl-lib:int-add nlvl 1))
              ((> nlvl (f2cl-lib:int-add lvl (f2cl-lib:int-sub 1))) nil)
  (tagbody
    (f2cl-lib:fdo (i 0 (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add llst (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf il (f2cl-lib:int-add il 2))
        (setf ir (f2cl-lib:int-add ir 2))
        (setf ncrnt (f2cl-lib:int-add llst i))
        (setf (f2cl-lib:fref ndiml-%data% (il) ((1 *)) ndiml-%offset%)
                (the f2cl-lib:integer4
                     (truncate
                      (f2cl-lib:fref ndiml-%data% (ncrnt) ((1 *))
                                     ndiml-%offset%)
                      2)))
        (setf (f2cl-lib:fref ndimr-%data% (il) ((1 *)) ndimr-%offset%)
                (f2cl-lib:int-sub
                 (f2cl-lib:fref ndiml-%data% (ncrnt) ((1 *))
                                ndiml-%offset%)
                 (f2cl-lib:fref ndiml-%data% (il) ((1 *))
                                ndiml-%offset%)
                 1))
        (setf (f2cl-lib:fref inode-%data% (il) ((1 *)) inode-%offset%)
                (f2cl-lib:int-sub
                 (f2cl-lib:fref inode-%data% (ncrnt) ((1 *))
                                inode-%offset%)
                 (f2cl-lib:fref ndimr-%data% (il) ((1 *))
                                ndimr-%offset%)
                 1))
        (setf (f2cl-lib:fref ndiml-%data% (ir) ((1 *)) ndiml-%offset%)
                (the f2cl-lib:integer4
                     (truncate
                      (f2cl-lib:fref ndimr-%data% (ncrnt) ((1 *))
                                     ndimr-%offset%)
                      2)))
        (setf (f2cl-lib:fref ndimr-%data% (ir) ((1 *)) ndimr-%offset%)
                (f2cl-lib:int-sub
                 (f2cl-lib:fref ndimr-%data% (ncrnt) ((1 *))
                                ndimr-%offset%)
                 (f2cl-lib:fref ndiml-%data% (ir) ((1 *))
                                ndiml-%offset%)
```

```
                        1))
                (setf (f2cl-lib:fref inode-%data% (ir) ((1 *)) inode-%offset%)
                        (f2cl-lib:int-add
                         (f2cl-lib:fref inode-%data% (ncrnt) ((1 *))
                                        inode-%offset%)
                         (f2cl-lib:fref ndiml-%data% (ir) ((1 *))
                                        ndiml-%offset%)
                         1))
               label10))
            (setf llst (f2cl-lib:int-mul llst 2))
           label20))
        (setf nd (f2cl-lib:int-sub (f2cl-lib:int-mul llst 2) 1))
        (go end_label)
       end_label
        (return (values nil lvl nd nil nil nil nil))))))
```

———-

90.181 dlaset

90.181.1 Axiom unit tests

— **dlaset.input** —

```
)set break resume
)sys rm -f dlaset.output
)spool dlaset.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.181.2 Axiom help page

— **dlaset.help** —

```
========================================================================
dlaset examples
========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

NAME
       DLASET  -  an  m-by-n matrix A to BETA on the diagonal and ALPHA on the
       offdiagonals

SYNOPSIS
       SUBROUTINE DLASET( UPLO, M, N, ALPHA, BETA, A, LDA )

           CHARACTER      UPLO

           INTEGER        LDA, M, N

           DOUBLE         PRECISION ALPHA, BETA

           DOUBLE         PRECISION A( LDA, * )

  Purpose
  =======

  DLASET initializes an m-by-n matrix A to BETA on the diagonal and
  ALPHA on the offdiagonals.

  Arguments
  =========

  UPLO    (input) CHARACTER*1
          Specifies the part of the matrix A to be set.
          = 'U':      Upper triangular part is set; the strictly lower
                      triangular part of A is not changed.
          = 'L':      Lower triangular part is set; the strictly upper
                      triangular part of A is not changed.
          Otherwise:  All of the matrix A is set.

  M       (input) INTEGER
          The number of rows of the matrix A.  M >= 0.

  N       (input) INTEGER
          The number of columns of the matrix A.  N >= 0.

  ALPHA   (input) DOUBLE PRECISION
          The constant to which the offdiagonal elements are to be set.

  BETA    (input) DOUBLE PRECISION
          The constant to which the diagonal elements are to be set.

  A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
```

```
         On exit, the leading m-by-n submatrix of A is set as follows:

         if UPLO = 'U', A(i,j) = ALPHA, 1<=i<=j-1, 1<=j<=n,
         if UPLO = 'L', A(i,j) = ALPHA, j+1<=i<=m, 1<=j<=n,
         otherwise,     A(i,j) = ALPHA, 1<=i<=m, 1<=j<=n, i.ne.j,

         and, for all UPLO, A(i,i) = BETA, 1<=i<=min(m,n).

 LDA     (input) INTEGER
         The leading dimension of the array A.  LDA >= max(1,M).
```

———-

90.181.3 fortran code

— dlaset.f —

```
      SUBROUTINE DLASET( UPLO, M, N, ALPHA, BETA, A, LDA )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1992
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            LDA, M, N
      DOUBLE PRECISION   ALPHA, BETA
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * )
*     ..
*
* =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MIN
*     ..
*     .. Executable Statements ..
*
```

```
      IF( LSAME( UPLO, 'U' ) ) THEN
*
*        Set the strictly upper triangular or trapezoidal part of the
*        array to ALPHA.
*
         DO 20 J = 2, N
            DO 10 I = 1, MIN( J-1, M )
               A( I, J ) = ALPHA
   10       CONTINUE
   20    CONTINUE
*
      ELSE IF( LSAME( UPLO, 'L' ) ) THEN
*
*        Set the strictly lower triangular or trapezoidal part of the
*        array to ALPHA.
*
         DO 40 J = 1, MIN( M, N )
            DO 30 I = J + 1, M
               A( I, J ) = ALPHA
   30       CONTINUE
   40    CONTINUE
*
      ELSE
*
*        Set the leading m-by-n submatrix to ALPHA.
*
         DO 60 J = 1, N
            DO 50 I = 1, M
               A( I, J ) = ALPHA
   50       CONTINUE
   60    CONTINUE
      END IF
*
*     Set the first min(M,N) diagonal elements to BETA.
*
      DO 70 I = 1, MIN( M, N )
         A( I, I ) = BETA
   70 CONTINUE
*
      RETURN
*
*     End of DLASET
*
      END
```

90.181.4 lisp code

— dlaset.lisp —

```
(defun dlaset (uplo m n alpha beta a lda)
  (declare (type (array double-float (*)) a)
           (type (double-float) beta alpha)
           (type (f2cl-lib:integer4) lda n m)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a double-float a-%data% a-%offset%))
    (prog ((i 0) (j 0))
      (declare (type (f2cl-lib:integer4) j i))
      (cond
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)
        (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i
                              (min
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                               (the f2cl-lib:integer4 m)))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                      alpha)
               label10))
           label20)))
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j
                          (min (the f2cl-lib:integer4 m)
                               (the f2cl-lib:integer4 n)))
                       nil)
          (tagbody
            (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
```

```
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                        alpha)
               label30))
           label40)))
       (t
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                        alpha)
               label50))
           label60))))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (min (the f2cl-lib:integer4 m)
                             (the f2cl-lib:integer4 n)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%) beta)
         label70))
      (go end_label)
     end_label
      (return (values uplo nil nil nil nil nil nil)))))
```

———-

90.182 dlasq1

90.182.1 Axiom unit tests

— **dlasq1.input** —

```
)set break resume
)sys rm -f dlasq1.output
)spool dlasq1.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.182.2 Axiom help page

— dlasq1.help —

```
==================================================================
dlasq1 examples
==================================================================

==================================================================
Man Page Details
==================================================================

NAME
       DLASQ1  -  the  singular values of a real N-by-N bidiagonal matrix with
       diagonal D and off-diagonal E

SYNOPSIS
       SUBROUTINE DLASQ1( N, D, E, WORK, INFO )

           INTEGER        INFO, N

           DOUBLE         PRECISION D( * ), E( * ), WORK( * )

  Purpose
  =======

  DLASQ1 computes the singular values of a real N-by-N bidiagonal
  matrix with diagonal D and off-diagonal E. The singular values
  are computed to high relative accuracy, in the absence of
  denormalization, underflow and overflow. The algorithm was first
  presented in

  "Accurate singular values and differential qd algorithms" by K. V.
  Fernando and B. N. Parlett, Numer. Math., Vol-67, No. 2, pp. 191-230,
  1994,

  and the present implementation is described in "An implementation of
  the dqds Algorithm (Positive Case)", LAPACK Working Note.

  Arguments
  =========

  N     (input) INTEGER
        The number of rows and columns in the matrix. N >= 0.

  D     (input/output) DOUBLE PRECISION array, dimension (N)
        On entry, D contains the diagonal elements of the
        bidiagonal matrix whose SVD is desired. On normal exit,
        D contains the singular values in decreasing order.
```

```
E     (input/output) DOUBLE PRECISION array, dimension (N)
      On entry, elements E(1:N-1) contain the off-diagonal elements
      of the bidiagonal matrix whose SVD is desired.
      On exit, E is overwritten.

WORK  (workspace) DOUBLE PRECISION array, dimension (4*N)

INFO  (output) INTEGER
      = 0: successful exit
      < 0: if INFO = -i, the i-th argument had an illegal value
      > 0: the algorithm failed
           = 1, a split was marked by a positive value in E
           = 2, current block of Z not diagonalized after 30*N
                iterations (in inner while loop)
           = 3, termination criterion of outer while loop not met
                (program created more than N unreduced blocks)
```

———-

90.182.3 fortran code

— dlasq1.f —

```
      SUBROUTINE DLASQ1( N, D, E, WORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1999
*
*     .. Scalar Arguments ..
      INTEGER            INFO, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * ), E( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, IINFO
      DOUBLE PRECISION   EPS, SCALE, SAFMIN, SIGMN, SIGMX
```

```
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLAS2, DLASQ2, DLASRT, XERBLA
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -2
         CALL XERBLA( 'DLASQ1', -INFO )
         RETURN
      ELSE IF( N.EQ.0 ) THEN
         RETURN
      ELSE IF( N.EQ.1 ) THEN
         D( 1 ) = ABS( D( 1 ) )
         RETURN
      ELSE IF( N.EQ.2 ) THEN
         CALL DLAS2( D( 1 ), E( 1 ), D( 2 ), SIGMN, SIGMX )
         D( 1 ) = SIGMX
         D( 2 ) = SIGMN
         RETURN
      END IF
*
*     Estimate the largest singular value.
*
      SIGMX = ZERO
      DO 10 I = 1, N - 1
         D( I ) = ABS( D( I ) )
         SIGMX = MAX( SIGMX, ABS( E( I ) ) )
   10 CONTINUE
      D( N ) = ABS( D( N ) )
*
*     Early return if SIGMX is zero (matrix is already diagonal).
*
      IF( SIGMX.EQ.ZERO ) THEN
         CALL DLASRT( 'D', N, D, IINFO )
         RETURN
      END IF
*
      DO 20 I = 1, N
         SIGMX = MAX( SIGMX, D( I ) )
   20 CONTINUE
*
```

```
*     Copy D and E into WORK (in the Z format) and scale (squaring the
*     input data makes scaling by a power of the radix pointless).
*
      EPS = DLAMCH( 'Precision' )
      SAFMIN = DLAMCH( 'Safe minimum' )
      SCALE = SQRT( EPS / SAFMIN )
      CALL DCOPY( N, D, 1, WORK( 1 ), 2 )
      CALL DCOPY( N-1, E, 1, WORK( 2 ), 2 )
      CALL DLASCL( 'G', 0, 0, SIGMX, SCALE, 2*N-1, 1, WORK, 2*N-1,
     $             IINFO )
*
*     Compute the q's and e's.
*
      DO 30 I = 1, 2*N - 1
         WORK( I ) = WORK( I )**2
   30 CONTINUE
      WORK( 2*N ) = ZERO
*
      CALL DLASQ2( N, WORK, INFO )
*
      IF( INFO.EQ.0 ) THEN
         DO 40 I = 1, N
            D( I ) = SQRT( WORK( I ) )
   40    CONTINUE
         CALL DLASCL( 'G', 0, 0, SCALE, SIGMX, N, 1, D, N, IINFO )
      END IF
*
      RETURN
*
*     End of DLASQ1
*
      END
```

———-

90.182.4 lisp code

— dlasq1.lisp —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun dlasq1 (n d e work info)
    (declare (type (array double-float (*)) work e d)
             (type (f2cl-lib:integer4) info n))
    (f2cl-lib:with-multi-array-data
        ((d double-float d-%data% d-%offset%)
```

```
    (e double-float e-%data% e-%offset%)
    (work double-float work-%data% work-%offset%))
  (prog ((eps 0.0d0) (scale 0.0d0) (safmin 0.0d0) (sigmn 0.0d0)
         (sigmx 0.0d0) (i 0) (iinfo 0))
    (declare (type (double-float) eps scale safmin sigmn sigmx)
             (type (f2cl-lib:integer4) i iinfo))
    (setf info 0)
    (cond
     ((< n 0) (setf info -1) (xerbla "DLASQ1" (f2cl-lib:int-sub info))
      (go end_label))
     ((= n 0) (go end_label))
     ((= n 1)
      (setf (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
              (abs (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)))
      (go end_label))
     ((= n 2)
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (dlas2 (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%)
           (f2cl-lib:fref e-%data% (1) ((1 *)) e-%offset%)
           (f2cl-lib:fref d-%data% (2) ((1 *)) d-%offset%) sigmn sigmx)
        (declare (ignore var-0 var-1 var-2))
        (setf sigmn var-3)
        (setf sigmx var-4))
      (setf (f2cl-lib:fref d-%data% (1) ((1 *)) d-%offset%) sigmx)
      (setf (f2cl-lib:fref d-%data% (2) ((1 *)) d-%offset%) sigmn)
      (go end_label)))
    (setf sigmx zero)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add n (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                (abs (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
        (setf sigmx
                (max sigmx
                     (abs
                      (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%))))
       label10))
    (setf (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)
            (abs (f2cl-lib:fref d-%data% (n) ((1 *)) d-%offset%)))
    (cond
     ((= sigmx zero)
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (dlasrt "D" n d iinfo)
        (declare (ignore var-0 var-2))
        (when var-1 (setf n var-1))
        (when var-3 (setf iinfo var-3)))
      (go end_label)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
```

```
    (setf sigmx
            (max sigmx
                 (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)))
   label20))
(setf eps (dlamch "Precision"))
(setf safmin (dlamch "Safe minimum"))
(setf scale (f2cl-lib:fsqrt (/ eps safmin)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dcopy n d 1
     (f2cl-lib:array-slice work-%data% double-float (1) ((1 *))
                           work-%offset%)
     2)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf n var-0)))
(dcopy (f2cl-lib:int-sub n 1) e 1
 (f2cl-lib:array-slice work-%data% double-float (2) ((1 *))
                       work-%offset%)
 2)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dlascl "G" 0 0 sigmx scale
     (f2cl-lib:int-sub (f2cl-lib:int-mul 2 n) 1) 1 work
     (f2cl-lib:int-sub (f2cl-lib:int-mul 2 n) 1) iinfo)
  (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
  (setf sigmx var-3)
  (setf scale var-4)
  (setf iinfo var-9))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i
                  (f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                    (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
            (expt (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  2))
   label30))
(setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-mul 2 n)) ((1 *))
                     work-%offset%)
        zero)
(multiple-value-bind (var-0 var-1 var-2)
    (dlasq2 n work info)
  (declare (ignore var-1))
  (when var-0 (setf n var-0))
  (when var-2 (setf info var-2)))
(cond
 ((= info 0)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
```

```
          (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                  (f2cl-lib:fsqrt
                   (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)))
        label40))
     (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (dlascl "G" 0 0 scale sigmx n 1 d n iinfo)
        (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
        (setf scale var-3)
        (setf sigmx var-4)
        (setf iinfo var-9)))
    ((= info 2)
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                  (f2cl-lib:fsqrt
                   (f2cl-lib:fref work-%data%
                                  ((f2cl-lib:int-sub (f2cl-lib:int-mul 2 i)
                                                     1))
                                  ((1 *)) work-%offset%)))
          (setf (f2cl-lib:fref e-%data% (i) ((1 *)) e-%offset%)
                  (f2cl-lib:fsqrt
                   (f2cl-lib:fref work-%data% ((f2cl-lib:int-mul 2 i))
                                  ((1 *)) work-%offset%)))
        label100000))
     (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (dlascl "G" 0 0 scale sigmx n 1 d n iinfo)
        (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
        (setf scale var-3)
        (setf sigmx var-4)
        (setf iinfo var-9))
     (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
          (dlascl "G" 0 0 scale sigmx n 1 e n iinfo)
        (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
        (setf scale var-3)
        (setf sigmx var-4)
        (setf iinfo var-9))))
  (go end_label)
 end_label
  (return (values n nil nil nil info))))))
```

———-

90.183 dlasq2

90.183.1 Axiom unit tests

— dlasq2.input —

```
)set break resume
)sys rm -f dlasq2.output
)spool dlasq2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.183.2 Axiom help page

— dlasq2.help —

```
======================================================================
dlasq2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DLASQ2  - all the eigenvalues of the symmetric positive definite tridi-
       agonal matrix associated with the qd array Z to high relative  accuracy
       are  computed to high relative accuracy, in the absence of denormaliza-
       tion, underflow and overflow

SYNOPSIS
       SUBROUTINE DLASQ2( N, Z, INFO )

           INTEGER        INFO, N

           DOUBLE         PRECISION Z( * )

  Purpose
  =======
```

```
DLASQ2 computes all the eigenvalues of the symmetric positive
definite tridiagonal matrix associated with the qd array Z to high
relative accuracy are computed to high relative accuracy, in the
absence of denormalization, underflow and overflow.

To see the relation of Z to the tridiagonal matrix, let L be a
unit lower bidiagonal matrix with subdiagonals Z(2,4,6,,..) and
let U be an upper bidiagonal matrix with 1's above and diagonal
Z(1,3,5,,..). The tridiagonal is L*U or, if you prefer, the
symmetric tridiagonal to which it is similar.

Note : DLASQ2 defines a logical variable, IEEE, which is true
on machines which follow ieee-754 floating-point standard in their
handling of infinities and NaNs, and false otherwise. This variable
is passed to DLASQ3.

Arguments
=========

N     (input) INTEGER
      The number of rows and columns in the matrix. N >= 0.

Z     (workspace) DOUBLE PRECISION array, dimension ( 4*N )
      On entry Z holds the qd array. On exit, entries 1 to N hold
      the eigenvalues in decreasing order, Z( 2*N+1 ) holds the
      trace, and Z( 2*N+2 ) holds the sum of the eigenvalues. If
      N > 2, then Z( 2*N+3 ) holds the iteration count, Z( 2*N+4 )
      holds NDIVS/NIN^2, and Z( 2*N+5 ) holds the percentage of
      shifts that failed.

INFO  (output) INTEGER
      = 0: successful exit
      < 0: if the i-th argument is a scalar and had an illegal
           value, then INFO = -i, if the i-th argument is an
           array and the j-entry had an illegal value, then
           INFO = -(i*100+j)
      > 0: the algorithm failed
            = 1, a split was marked by a positive value in E
            = 2, current block of Z not diagonalized after 30*N
                 iterations (in inner while loop)
            = 3, termination criterion of outer while loop not met
                 (program created more than N unreduced blocks)

Further Details
===============
Local Variables: I0:N0 defines a current unreduced segment of Z.
The shifts are accumulated in SIGMA. Iteration count is in ITER.
Ping-pong is controlled by PP (alternates between 0 and 1).
```

———-

90.183.3 fortran code

— **dlasq2.f** —

```
      SUBROUTINE DLASQ2( N, Z, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1999
*
*     .. Scalar Arguments ..
      INTEGER            INFO, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   Z( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   CBIAS
      PARAMETER          ( CBIAS = 1.50D0 )
      DOUBLE PRECISION   ZERO, HALF, ONE, TWO, FOUR, HUNDRD
      PARAMETER          ( ZERO = 0.0D0, HALF = 0.5D0, ONE = 1.0D0,
     $                     TWO = 2.0D0, FOUR = 4.0D0, HUNDRD = 100.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            IEEE
      INTEGER            I0, I4, IINFO, IPN4, ITER, IWHILA, IWHILB, K,
     $                   N0, NBIG, NDIV, NFAIL, PP, SPLT
      DOUBLE PRECISION   D, DESIG, DMIN, E, EMAX, EMIN, EPS, OLDEMN,
     $                   QMAX, QMIN, S, SAFMIN, SIGMA, T, TEMP, TOL,
     $                   TOL2, TRACE, ZMAX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASQ3, DLASRT, XERBLA
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH, ILAENV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, MAX, MIN, SQRT
*     ..
```

```
*     .. Executable Statements ..
*
*     Test the input arguments.
*     (in case DLASQ2 is not called by DLASQ1)
*
      INFO = 0
      EPS = DLAMCH( 'Precision' )
      SAFMIN = DLAMCH( 'Safe minimum' )
      TOL = EPS*HUNDRD
      TOL2 = TOL**2
*
      IF( N.LT.0 ) THEN
         INFO = -1
         CALL XERBLA( 'DLASQ2', 1 )
         RETURN
      ELSE IF( N.EQ.0 ) THEN
         RETURN
      ELSE IF( N.EQ.1 ) THEN
*
*        1-by-1 case.
*
         IF( Z( 1 ).LT.ZERO ) THEN
            INFO = -201
            CALL XERBLA( 'DLASQ2', 2 )
         END IF
         RETURN
      ELSE IF( N.EQ.2 ) THEN
*
*        2-by-2 case.
*
         IF( Z( 2 ).LT.ZERO .OR. Z( 3 ).LT.ZERO ) THEN
            INFO = -2
            CALL XERBLA( 'DLASQ2', 2 )
            RETURN
         ELSE IF( Z( 3 ).GT.Z( 1 ) ) THEN
            D = Z( 3 )
            Z( 3 ) = Z( 1 )
            Z( 1 ) = D
         END IF
         Z( 5 ) = Z( 1 ) + Z( 2 ) + Z( 3 )
         IF( Z( 2 ).GT.Z( 3 )*TOL2 ) THEN
            T = HALF*( ( Z( 1 )-Z( 3 ) )+Z( 2 ) )
            S = Z( 3 )*( Z( 2 ) / T )
            IF( S.LE.T ) THEN
               S = Z( 3 )*( Z( 2 ) / ( T*( ONE+SQRT( ONE+S / T ) ) ) )
            ELSE
               S = Z( 3 )*( Z( 2 ) / ( T+SQRT( T )*SQRT( T+S ) ) )
            END IF
            T = Z( 1 ) + ( S+Z( 2 ) )
            Z( 3 ) = Z( 3 )*( Z( 1 ) / T )
```

```
            Z( 1 ) = T
         END IF
         Z( 2 ) = Z( 3 )
         Z( 6 ) = Z( 2 ) + Z( 1 )
         RETURN
      END IF
*
*     Check for negative data and compute sums of q's and e's.
*
      Z( 2*N ) = ZERO
      EMIN = Z( 2 )
      QMAX = ZERO
      ZMAX = ZERO
      D = ZERO
      E = ZERO
*
      DO 10 K = 1, 2*( N-1 ), 2
         IF( Z( K ).LT.ZERO ) THEN
            INFO = -( 200+K )
            CALL XERBLA( 'DLASQ2', 2 )
            RETURN
         ELSE IF( Z( K+1 ).LT.ZERO ) THEN
            INFO = -( 200+K+1 )
            CALL XERBLA( 'DLASQ2', 2 )
            RETURN
         END IF
         D = D + Z( K )
         E = E + Z( K+1 )
         QMAX = MAX( QMAX, Z( K ) )
         EMIN = MIN( EMIN, Z( K+1 ) )
         ZMAX = MAX( QMAX, ZMAX, Z( K+1 ) )
   10 CONTINUE
      IF( Z( 2*N-1 ).LT.ZERO ) THEN
         INFO = -( 200+2*N-1 )
         CALL XERBLA( 'DLASQ2', 2 )
         RETURN
      END IF
      D = D + Z( 2*N-1 )
      QMAX = MAX( QMAX, Z( 2*N-1 ) )
      ZMAX = MAX( QMAX, ZMAX )
*
*     Check for diagonality.
*
      IF( E.EQ.ZERO ) THEN
         DO 20 K = 2, N
            Z( K ) = Z( 2*K-1 )
   20    CONTINUE
         CALL DLASRT( 'D', N, Z, IINFO )
         Z( 2*N-1 ) = D
         RETURN
```

```
      END IF
*
      TRACE = D + E
*
*     Check for zero data.
*
      IF( TRACE.EQ.ZERO ) THEN
         Z( 2*N-1 ) = ZERO
         RETURN
      END IF
*
*     Check whether the machine is IEEE conformable.
*
      IEEE = ILAENV( 10, 'DLASQ2', 'N', 1, 2, 3, 4 ).EQ.1 .AND.
     $       ILAENV( 11, 'DLASQ2', 'N', 1, 2, 3, 4 ).EQ.1
*
*     Rearrange data for locality: Z=(q1,qq1,e1,ee1,q2,qq2,e2,ee2,...).
*
      DO 30 K = 2*N, 2, -2
         Z( 2*K ) = ZERO
         Z( 2*K-1 ) = Z( K )
         Z( 2*K-2 ) = ZERO
         Z( 2*K-3 ) = Z( K-1 )
   30 CONTINUE
*
      I0 = 1
      N0 = N
*
*     Reverse the qd-array, if warranted.
*
      IF( CBIAS*Z( 4*I0-3 ).LT.Z( 4*N0-3 ) ) THEN
         IPN4 = 4*( I0+N0 )
         DO 40 I4 = 4*I0, 2*( I0+N0-1 ), 4
            TEMP = Z( I4-3 )
            Z( I4-3 ) = Z( IPN4-I4-3 )
            Z( IPN4-I4-3 ) = TEMP
            TEMP = Z( I4-1 )
            Z( I4-1 ) = Z( IPN4-I4-5 )
            Z( IPN4-I4-5 ) = TEMP
   40    CONTINUE
      END IF
*
*     Initial split checking via dqd and Li's test.
*
      PP = 0
*
      DO 80 K = 1, 2
*
         D = Z( 4*N0+PP-3 )
         DO 50 I4 = 4*( N0-1 ) + PP, 4*I0 + PP, -4
```

```
            IF( Z( I4-1 ).LE.TOL2*D ) THEN
               Z( I4-1 ) = -ZERO
               D = Z( I4-3 )
            ELSE
               D = Z( I4-3 )*( D / ( D+Z( I4-1 ) ) )
            END IF
   50    CONTINUE
*
*        dqd maps Z to ZZ plus Li's test.
*
         EMIN = Z( 4*I0+PP+1 )
         D = Z( 4*I0+PP-3 )
         DO 60 I4 = 4*I0 + PP, 4*( N0-1 ) + PP, 4
            Z( I4-2*PP-2 ) = D + Z( I4-1 )
            IF( Z( I4-1 ).LE.TOL2*D ) THEN
               Z( I4-1 ) = -ZERO
               Z( I4-2*PP-2 ) = D
               Z( I4-2*PP ) = ZERO
               D = Z( I4+1 )
            ELSE IF( SAFMIN*Z( I4+1 ).LT.Z( I4-2*PP-2 ) .AND.
     $               SAFMIN*Z( I4-2*PP-2 ).LT.Z( I4+1 ) ) THEN
               TEMP = Z( I4+1 ) / Z( I4-2*PP-2 )
               Z( I4-2*PP ) = Z( I4-1 )*TEMP
               D = D*TEMP
            ELSE
               Z( I4-2*PP ) = Z( I4+1 )*( Z( I4-1 ) / Z( I4-2*PP-2 ) )
               D = Z( I4+1 )*( D / Z( I4-2*PP-2 ) )
            END IF
            EMIN = MIN( EMIN, Z( I4-2*PP ) )
   60    CONTINUE
         Z( 4*N0-PP-2 ) = D
*
*        Now find qmax.
*
         QMAX = Z( 4*I0-PP-2 )
         DO 70 I4 = 4*I0 - PP + 2, 4*N0 - PP - 2, 4
            QMAX = MAX( QMAX, Z( I4 ) )
   70    CONTINUE
*
*        Prepare for the next iteration on K.
*
         PP = 1 - PP
   80 CONTINUE
*
      ITER = 2
      NFAIL = 0
      NDIV = 2*( N0-I0 )
*
      DO 140 IWHILA = 1, N + 1
         IF( N0.LT.1 )
```

```
     $      GO TO 150
*
*        While array unfinished do
*
*        E(N0) holds the value of SIGMA when submatrix in I0:N0
*        splits from the rest of the array, but is negated.
*
         DESIG = ZERO
         IF( N0.EQ.N ) THEN
            SIGMA = ZERO
         ELSE
            SIGMA = -Z( 4*N0-1 )
         END IF
         IF( SIGMA.LT.ZERO ) THEN
            INFO = 1
            RETURN
         END IF
*
*        Find last unreduced submatrix's top index I0, find QMAX and
*        EMIN. Find Gershgorin-type bound if Q's much greater than E's.
*
         EMAX = ZERO
         IF( N0.GT.I0 ) THEN
            EMIN = ABS( Z( 4*N0-5 ) )
         ELSE
            EMIN = ZERO
         END IF
         QMIN = Z( 4*N0-3 )
         QMAX = QMIN
         DO 90 I4 = 4*N0, 8, -4
            IF( Z( I4-5 ).LE.ZERO )
     $         GO TO 100
            IF( QMIN.GE.FOUR*EMAX ) THEN
               QMIN = MIN( QMIN, Z( I4-3 ) )
               EMAX = MAX( EMAX, Z( I4-5 ) )
            END IF
            QMAX = MAX( QMAX, Z( I4-7 )+Z( I4-5 ) )
            EMIN = MIN( EMIN, Z( I4-5 ) )
   90    CONTINUE
         I4 = 4
*
  100    CONTINUE
         I0 = I4 / 4
*
*        Store EMIN for passing to DLASQ3.
*
         Z( 4*N0-1 ) = EMIN
*
*        Put -(initial shift) into DMIN.
*
```

```
         DMIN = -MAX( ZERO, QMIN-TWO*SQRT( QMIN )*SQRT( EMAX ) )
*
*        Now I0:N0 is unreduced. PP = 0 for ping, PP = 1 for pong.
*
         PP = 0
*
         NBIG = 30*( N0-I0+1 )
         DO 120 IWHILB = 1, NBIG
            IF( I0.GT.N0 )
     $         GO TO 130
*
*           While submatrix unfinished take a good dqds step.
*
            CALL DLASQ3( I0, N0, Z, PP, DMIN, SIGMA, DESIG, QMAX, NFAIL,
     $                   ITER, NDIV, IEEE )
*
            PP = 1 - PP
*
*           When EMIN is very small check for splits.
*
            IF( PP.EQ.0 .AND. N0-I0.GE.3 ) THEN
               IF( Z( 4*N0 ).LE.TOL2*QMAX .OR.
     $             Z( 4*N0-1 ).LE.TOL2*SIGMA ) THEN
                  SPLT = I0 - 1
                  QMAX = Z( 4*I0-3 )
                  EMIN = Z( 4*I0-1 )
                  OLDEMN = Z( 4*I0 )
                  DO 110 I4 = 4*I0, 4*( N0-3 ), 4
                     IF( Z( I4 ).LE.TOL2*Z( I4-3 ) .OR.
     $                   Z( I4-1 ).LE.TOL2*SIGMA ) THEN
                        Z( I4-1 ) = -SIGMA
                        SPLT = I4 / 4
                        QMAX = ZERO
                        EMIN = Z( I4+3 )
                        OLDEMN = Z( I4+4 )
                     ELSE
                        QMAX = MAX( QMAX, Z( I4+1 ) )
                        EMIN = MIN( EMIN, Z( I4-1 ) )
                        OLDEMN = MIN( OLDEMN, Z( I4 ) )
                     END IF
  110             CONTINUE
                  Z( 4*N0-1 ) = EMIN
                  Z( 4*N0 ) = OLDEMN
                  I0 = SPLT + 1
               END IF
            END IF
*
  120    CONTINUE
*
         INFO = 2
```

```
         RETURN
*
*        end IWHILB
*
  130    CONTINUE
*
  140 CONTINUE
*
      INFO = 3
      RETURN
*
*     end IWHILA
*
  150 CONTINUE
*
*     Move q's to the front.
*
      DO 160 K = 2, N
         Z( K ) = Z( 4*K-3 )
  160 CONTINUE
*
*     Sort and compute sum of eigenvalues.
*
      CALL DLASRT( 'D', N, Z, IINFO )
*
      E = ZERO
      DO 170 K = N, 1, -1
         E = E + Z( K )
  170 CONTINUE
*
*     Store trace, sum(eigenvalues) and information on performance.
*
      Z( 2*N+1 ) = TRACE
      Z( 2*N+2 ) = E
      Z( 2*N+3 ) = DBLE( ITER )
      Z( 2*N+4 ) = DBLE( NDIV ) / DBLE( N**2 )
      Z( 2*N+5 ) = HUNDRD*NFAIL / DBLE( ITER )
      RETURN
*
*     End of DLASQ2
*
      END
```

———-

90.183.4 lisp code

— dlasq2.lisp —

```
(let* ((cbias 1.5d0)
       (zero 0.0d0)
       (half 0.5d0)
       (one 1.0d0)
       (two 2.0d0)
       (four 4.0d0)
       (hundrd 100.0d0))
  (declare (type (double-float 1.5d0 1.5d0) cbias)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 0.5d0 0.5d0) half)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 4.0d0 4.0d0) four)
           (type (double-float 100.0d0 100.0d0) hundrd)
           (ignorable cbias zero half one two four hundrd))
  (defun dlasq2 (n z info)
    (declare (type (array double-float (*)) z)
             (type (f2cl-lib:integer4) info n))
    (f2cl-lib:with-multi-array-data
        ((z double-float z-%data% z-%offset%))
      (prog ((d 0.0d0) (dee 0.0d0) (deemin 0.0d0) (desig 0.0d0) (dmin 0.0d0)
             (f2cl-lib:dmin1 0.0d0) (dmin2 0.0d0) (dn 0.0d0) (dn1 0.0d0)
             (dn2 0.0d0) (e 0.0d0) (emax 0.0d0) (emin 0.0d0) (eps 0.0d0)
             (g 0.0d0) (oldemn 0.0d0) (qmax 0.0d0) (qmin 0.0d0) (s 0.0d0)
             (safmin 0.0d0) (sigma 0.0d0) (t$ 0.0d0) (tau 0.0d0) (temp 0.0d0)
             (tol 0.0d0) (tol2 0.0d0) (trace$ 0.0d0) (zmax 0.0d0) (tempe 0.0d0)
             (tempq 0.0d0) (i0 0) (i1 0) (i4 0) (iinfo 0) (ipn4 0) (iter 0)
             (iwhila 0) (iwhilb 0) (k 0) (kmin 0) (n0 0) (n1 0) (nbig 0)
             (ndiv 0) (nfail 0) (pp 0) (splt 0) (ttype 0) (ieee nil))
        (declare
         (type (double-float) d dee deemin desig dmin f2cl-lib:dmin1 dmin2 dn
          dn1 dn2 e emax emin eps g oldemn qmax qmin s safmin sigma t$ tau temp
          tol tol2 trace$ zmax tempe tempq)
         (type (f2cl-lib:integer4) i0 i1 i4 iinfo ipn4 iter iwhila iwhilb k
          kmin n0 n1 nbig ndiv nfail pp splt ttype)
         (type f2cl-lib:logical ieee))
        (setf info 0)
        (setf eps (dlamch "Precision"))
        (setf safmin (dlamch "Safe minimum"))
        (setf tol (* eps hundrd))
        (setf tol2 (expt tol 2))
        (cond ((< n 0) (setf info -1) (xerbla "DLASQ2" 1) (go end_label))
              ((= n 0) (go end_label))
              ((= n 1)
               (cond
                ((< (f2cl-lib:fref z (1) ((1 *))) zero) (setf info -201)
                 (xerbla "DLASQ2" 2)))
               (go end_label))
```

```
((= n 2)
 (cond
  ((or (< (f2cl-lib:fref z (2) ((1 *))) zero)
       (< (f2cl-lib:fref z (3) ((1 *))) zero))
   (setf info -2) (xerbla "DLASQ2" 2) (go end_label))
  ((> (f2cl-lib:fref z (3) ((1 *)))
      (f2cl-lib:fref z (1) ((1 *))))
   (setf d (f2cl-lib:fref z-%data% (3) ((1 *)) z-%offset%))
   (setf (f2cl-lib:fref z-%data% (3) ((1 *)) z-%offset%)
           (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%))
   (setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%) d)))
 (setf (f2cl-lib:fref z-%data% (5) ((1 *)) z-%offset%)
         (+ (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)
            (f2cl-lib:fref z-%data% (2) ((1 *)) z-%offset%)
            (f2cl-lib:fref z-%data% (3) ((1 *)) z-%offset%)))
 (cond
  ((> (f2cl-lib:fref z (2) ((1 *)))
      (* (f2cl-lib:fref z (3) ((1 *))) tol2))
   (setf t$
           (* half
              (+
               (- (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)
                  (f2cl-lib:fref z-%data% (3) ((1 *))
                                 z-%offset%))
               (f2cl-lib:fref z-%data% (2) ((1 *)) z-%offset%))))
   (setf s
           (* (f2cl-lib:fref z-%data% (3) ((1 *)) z-%offset%)
              (/ (f2cl-lib:fref z-%data% (2) ((1 *)) z-%offset%)
                 t$)))
   (cond
    ((<= s t$)
     (setf s
             (* (f2cl-lib:fref z-%data% (3) ((1 *)) z-%offset%)
                (/
                 (f2cl-lib:fref z-%data% (2) ((1 *)) z-%offset%)
                 (* t$
                    (+ one
                       (f2cl-lib:fsqrt (+ one (/ s t$)))))))))
    (t
     (setf s
             (* (f2cl-lib:fref z-%data% (3) ((1 *)) z-%offset%)
                (/
                 (f2cl-lib:fref z-%data% (2) ((1 *)) z-%offset%)
                 (+ t$
                    (* (f2cl-lib:fsqrt t$)
                       (f2cl-lib:fsqrt (+ t$ s)))))))))
   (setf t$
           (+ (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)
              (+ s
                 (f2cl-lib:fref z-%data% (2) ((1 *))
```

```
                                    z-%offset%))))
          (setf (f2cl-lib:fref z-%data% (3) ((1 *)) z-%offset%)
                  (* (f2cl-lib:fref z-%data% (3) ((1 *)) z-%offset%)
                     (/ (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)
                        t$)))
          (setf (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%) t$)))
        (setf (f2cl-lib:fref z-%data% (2) ((1 *)) z-%offset%)
                (f2cl-lib:fref z-%data% (3) ((1 *)) z-%offset%))
        (setf (f2cl-lib:fref z-%data% (6) ((1 *)) z-%offset%)
                (+ (f2cl-lib:fref z-%data% (2) ((1 *)) z-%offset%)
                   (f2cl-lib:fref z-%data% (1) ((1 *)) z-%offset%)))
        (go end_label)))
(setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-mul 2 n)) ((1 *))
                     z-%offset%)
        zero)
(setf emin (f2cl-lib:fref z-%data% (2) ((1 *)) z-%offset%))
(setf qmax zero)
(setf zmax zero)
(setf d zero)
(setf e zero)
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 2))
              ((> k
                  (f2cl-lib:int-mul 2
                                    (f2cl-lib:int-add n
                                                      (f2cl-lib:int-sub
                                                       1))))
               nil)
  (tagbody
    (cond
     ((< (f2cl-lib:fref z (k) ((1 *))) zero)
      (setf info (f2cl-lib:int-sub (f2cl-lib:int-add 200 k)))
      (xerbla "DLASQ2" 2) (go end_label))
     ((< (f2cl-lib:fref z ((f2cl-lib:int-add k 1)) ((1 *))) zero)
      (setf info (f2cl-lib:int-sub (f2cl-lib:int-add 200 k 1)))
      (xerbla "DLASQ2" 2) (go end_label)))
    (setf d (+ d (f2cl-lib:fref z-%data% (k) ((1 *)) z-%offset%)))
    (setf e
            (+ e
               (f2cl-lib:fref z-%data% ((f2cl-lib:int-add k 1)) ((1 *))
                              z-%offset%)))
    (setf qmax
            (max qmax (f2cl-lib:fref z-%data% (k) ((1 *)) z-%offset%)))
    (setf emin
            (min emin
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-add k 1))
                                ((1 *)) z-%offset%)))
    (setf zmax
            (max qmax zmax
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-add k 1))
                                ((1 *)) z-%offset%)))
```

```
   label10))
(cond
 ((<
   (f2cl-lib:fref z
                  ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n)
                                     (f2cl-lib:int-sub 1)))
                  ((1 *)))
   zero)
  (setf info
          (f2cl-lib:int-sub
           (f2cl-lib:int-sub
            (f2cl-lib:int-add 200 (f2cl-lib:int-mul 2 n)) 1)))
  (xerbla "DLASQ2" 2) (go end_label)))
(setf d
        (+ d
           (f2cl-lib:fref z-%data%
                          ((f2cl-lib:int-sub (f2cl-lib:int-mul 2 n) 1))
                          ((1 *)) z-%offset%)))
(setf qmax
        (max qmax
             (f2cl-lib:fref z-%data%
                            ((f2cl-lib:int-sub (f2cl-lib:int-mul 2 n)
                                               1))
                            ((1 *)) z-%offset%)))
(setf zmax (max qmax zmax))
(cond
 ((= e zero)
  (f2cl-lib:fdo (k 2 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (setf (f2cl-lib:fref z-%data% (k) ((1 *)) z-%offset%)
              (f2cl-lib:fref z-%data%
                             ((f2cl-lib:int-sub (f2cl-lib:int-mul 2 k)
                                                1))
                             ((1 *)) z-%offset%))
     label20))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dlasrt "D" n z iinfo)
    (declare (ignore var-0 var-2))
    (when var-1 (setf n var-1))
    (when var-3 (setf iinfo var-3)))
  (setf (f2cl-lib:fref z-%data%
                       ((f2cl-lib:int-sub (f2cl-lib:int-mul 2 n) 1))
                       ((1 *)) z-%offset%)
          d)
  (go end_label)))
(setf trace$ (+ d e))
(cond
 ((= trace$ zero)
  (setf (f2cl-lib:fref z-%data%
```

```
                        ((f2cl-lib:int-sub (f2cl-lib:int-mul 2 n) 1))
                        ((1 *)) z-%offset%)
          zero)
  (go end_label)))
(setf ieee
        (and (= (ilaenv 10 "DLASQ2" "N" 1 2 3 4) 1)
             (= (ilaenv 11 "DLASQ2" "N" 1 2 3 4) 1)))
(f2cl-lib:fdo (k (f2cl-lib:int-mul 2 n)
               (f2cl-lib:int-add k (f2cl-lib:int-sub 2)))
              ((> k 2) nil)
  (tagbody
    (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-mul 2 k)) ((1 *))
                         z-%offset%)
            zero)
    (setf (f2cl-lib:fref z-%data%
                         ((f2cl-lib:int-sub (f2cl-lib:int-mul 2 k) 1))
                         ((1 *)) z-%offset%)
            (f2cl-lib:fref z-%data% (k) ((1 *)) z-%offset%))
    (setf (f2cl-lib:fref z-%data%
                         ((f2cl-lib:int-sub (f2cl-lib:int-mul 2 k) 2))
                         ((1 *)) z-%offset%)
            zero)
    (setf (f2cl-lib:fref z-%data%
                         ((f2cl-lib:int-sub (f2cl-lib:int-mul 2 k) 3))
                         ((1 *)) z-%offset%)
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub k 1)) ((1 *))
                           z-%offset%))
   label30))
(setf i0 1)
(setf n0 n)
(cond
 ((<
   (* cbias
      (f2cl-lib:fref z
                     ((f2cl-lib:int-add (f2cl-lib:int-mul 4 i0)
                                        (f2cl-lib:int-sub 3)))
                     ((1 *))))
   (f2cl-lib:fref z
                  ((f2cl-lib:int-add (f2cl-lib:int-mul 4 n0)
                                     (f2cl-lib:int-sub 3)))
                  ((1 *))))
  (setf ipn4 (f2cl-lib:int-mul 4 (f2cl-lib:int-add i0 n0)))
  (f2cl-lib:fdo (i4 (f2cl-lib:int-mul 4 i0) (f2cl-lib:int-add i4 4))
                ((> i4
                    (f2cl-lib:int-mul 2
                                      (f2cl-lib:int-add i0 n0
                                                        (f2cl-lib:int-sub
                                                         1))))
                 nil)
    (tagbody
```

```
        (setf temp
                (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 3)) ((1 *))
                               z-%offset%))
        (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 3)) ((1 *))
                             z-%offset%)
                (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub ipn4 i4 3))
                               ((1 *)) z-%offset%))
        (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub ipn4 i4 3))
                             ((1 *)) z-%offset%)
                temp)
        (setf temp
                (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 1)) ((1 *))
                               z-%offset%))
        (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 1)) ((1 *))
                             z-%offset%)
                (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub ipn4 i4 5))
                               ((1 *)) z-%offset%))
        (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub ipn4 i4 5))
                             ((1 *)) z-%offset%)
                temp)
      label40))))
(setf pp 0)
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k 2) nil)
  (tagbody
    (setf d
            (f2cl-lib:fref z-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add (f2cl-lib:int-mul 4 n0)
                                               pp)
                             3))
                           ((1 *)) z-%offset%))
    (f2cl-lib:fdo (i4
                   (f2cl-lib:int-add
                    (f2cl-lib:int-mul 4
                                      (f2cl-lib:int-add n0
                                                        (f2cl-lib:int-sub
                                                         1)))
                    pp)
                   (f2cl-lib:int-add i4 (f2cl-lib:int-sub 4)))
                  ((> i4 (f2cl-lib:int-add (f2cl-lib:int-mul 4 i0) pp))
                   nil)
      (tagbody
        (cond
         ((<=
           (f2cl-lib:fref z
                          ((f2cl-lib:int-add i4 (f2cl-lib:int-sub 1)))
                          ((1 *)))
           (* tol2 d))
          (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 1))
```

```
                              ((1 *)) z-%offset%)
                 (- zero))
        (setf d
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 3))
                                ((1 *)) z-%offset%)))
       (t
        (setf d
                 (*
                  (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 3))
                                 ((1 *)) z-%offset%)
                  (/ d
                     (+ d
                        (f2cl-lib:fref z-%data%
                                       ((f2cl-lib:int-sub i4 1))
                                       ((1 *)) z-%offset%)))))))
   label50))
(setf emin
        (f2cl-lib:fref z-%data%
                       ((f2cl-lib:int-add (f2cl-lib:int-mul 4 i0)
                                          pp 1))
                       ((1 *)) z-%offset%))
(setf d
        (f2cl-lib:fref z-%data%
                       ((f2cl-lib:int-sub
                         (f2cl-lib:int-add (f2cl-lib:int-mul 4 i0)
                                           pp)
                         3))
                       ((1 *)) z-%offset%))
(f2cl-lib:fdo (i4 (f2cl-lib:int-add (f2cl-lib:int-mul 4 i0) pp)
               (f2cl-lib:int-add i4 4))
              ((> i4
                  (f2cl-lib:int-add
                   (f2cl-lib:int-mul 4
                                     (f2cl-lib:int-add n0
                                                       (f2cl-lib:int-sub
                                                        1)))
                   pp))
               nil)
  (tagbody
    (setf (f2cl-lib:fref z-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add i4
                                             (f2cl-lib:int-mul -1 2
                                                               pp))
                           2))
                         ((1 *)) z-%offset%)
          (+ d
             (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 1))
                            ((1 *)) z-%offset%)))
    (cond
```

```
((<=
  (f2cl-lib:fref z
                 ((f2cl-lib:int-add i4 (f2cl-lib:int-sub 1)))
                 ((1 *)))
  (* tol2 d))
 (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 1))
                      ((1 *)) z-%offset%)
        (- zero))
 (setf (f2cl-lib:fref z-%data%
                      ((f2cl-lib:int-sub
                        (f2cl-lib:int-add i4
                                          (f2cl-lib:int-mul -1
                                                            2
                                                            pp))
                        2))
                      ((1 *)) z-%offset%)
        d)
 (setf (f2cl-lib:fref z-%data%
                      ((f2cl-lib:int-add i4
                                         (f2cl-lib:int-mul -1
                                                           2
                                                           pp)))
                      ((1 *)) z-%offset%)
        zero)
 (setf d
        (f2cl-lib:fref z-%data% ((f2cl-lib:int-add i4 1))
                        ((1 *)) z-%offset%)))
((and
  (<
   (* safmin
      (f2cl-lib:fref z ((f2cl-lib:int-add i4 1)) ((1 *))))
   (f2cl-lib:fref z
                  ((f2cl-lib:int-add i4
                                     (f2cl-lib:int-mul -1 2
                                                       pp)
                                     (f2cl-lib:int-sub 2)))
                  ((1 *))))
  (<
   (* safmin
      (f2cl-lib:fref z
                     ((f2cl-lib:int-add i4
                                        (f2cl-lib:int-mul -1 2
                                                          pp)
                                        (f2cl-lib:int-sub 2)))
                     ((1 *))))
   (f2cl-lib:fref z ((f2cl-lib:int-add i4 1)) ((1 *)))))
 (setf temp
        (/
         (f2cl-lib:fref z-%data% ((f2cl-lib:int-add i4 1))
                         ((1 *)) z-%offset%)
```

```
          (f2cl-lib:fref z-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add i4
                                             (f2cl-lib:int-mul
                                              -1 2 pp))
                           2))
                         ((1 *)) z-%offset%)))
 (setf (f2cl-lib:fref z-%data%
                      ((f2cl-lib:int-add i4
                                         (f2cl-lib:int-mul -1
                                                           2
                                                           pp)))
                      ((1 *)) z-%offset%)
         (*
          (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 1))
                         ((1 *)) z-%offset%)
          temp))
 (setf d (* d temp)))
(t
 (setf (f2cl-lib:fref z-%data%
                      ((f2cl-lib:int-add i4
                                         (f2cl-lib:int-mul -1
                                                           2
                                                           pp)))
                      ((1 *)) z-%offset%)
         (*
          (f2cl-lib:fref z-%data% ((f2cl-lib:int-add i4 1))
                         ((1 *)) z-%offset%)
          (/
           (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 1))
                          ((1 *)) z-%offset%)
           (f2cl-lib:fref z-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add i4
                                              (f2cl-lib:int-mul
                                               -1 2 pp))
                            2))
                          ((1 *)) z-%offset%))))
 (setf d
         (*
          (f2cl-lib:fref z-%data% ((f2cl-lib:int-add i4 1))
                         ((1 *)) z-%offset%)
          (/ d
             (f2cl-lib:fref z-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-add i4
                                                (f2cl-lib:int-mul
                                                 -1 2 pp))
                              2))
                            ((1 *)) z-%offset%))))))
```

```
            (setf emin
                    (min emin
                         (f2cl-lib:fref z-%data%
                                        ((f2cl-lib:int-add i4
                                                           (f2cl-lib:int-mul
                                                            -1 2 pp)))
                                        ((1 *)) z-%offset%)))
           label60))
        (setf (f2cl-lib:fref z-%data%
                             ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 n0) pp
                                                2))
                             ((1 *)) z-%offset%)
                d)
        (setf qmax
                (f2cl-lib:fref z-%data%
                               ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 i0)
                                                  pp 2))
                               ((1 *)) z-%offset%))
        (f2cl-lib:fdo (i4
                       (f2cl-lib:int-add (f2cl-lib:int-mul 4 i0)
                                         (f2cl-lib:int-sub pp) 2)
                       (f2cl-lib:int-add i4 4))
                      ((> i4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 4 n0)
                                            (f2cl-lib:int-sub pp)
                                            (f2cl-lib:int-sub 2)))
                       nil)
          (tagbody
            (setf qmax
                    (max qmax
                         (f2cl-lib:fref z-%data% (i4) ((1 *)) z-%offset%)))
           label70))
        (setf pp (f2cl-lib:int-sub 1 pp))
       label80))
    (setf ttype 0)
    (setf f2cl-lib:dmin1 zero)
    (setf dmin2 zero)
    (setf dn zero)
    (setf dn1 zero)
    (setf dn2 zero)
    (setf g zero)
    (setf tau zero)
    (setf iter 2)
    (setf nfail 0)
    (setf ndiv (f2cl-lib:int-mul 2 (f2cl-lib:int-sub n0 i0)))
    (f2cl-lib:fdo (iwhila 1 (f2cl-lib:int-add iwhila 1))
                  ((> iwhila (f2cl-lib:int-add n 1)) nil)
      (tagbody
        (if (< n0 1)
            (go label170))
```

```
(setf desig zero)
(cond ((= n0 n) (setf sigma zero))
      (t
       (setf sigma
               (-
                (f2cl-lib:fref z-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-mul 4 n0) 1))
                               ((1 *)) z-%offset%)))))
(cond ((< sigma zero) (setf info 1) (go end_label)))
(setf emax zero)
(cond
 ((> n0 i0)
  (setf emin
          (abs
           (f2cl-lib:fref z-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-mul 4 n0) 5))
                          ((1 *)) z-%offset%))))
 (t (setf emin zero)))
(setf qmin
        (f2cl-lib:fref z-%data%
                       ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 n0)
                                          3))
                       ((1 *)) z-%offset%))
(setf qmax qmin)
(f2cl-lib:fdo (i4 (f2cl-lib:int-mul 4 n0)
               (f2cl-lib:int-add i4 (f2cl-lib:int-sub 4)))
              ((> i4 8) nil)
  (tagbody
    (if (<=
         (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 5)) ((1 *))
                        z-%offset%)
         zero)
        (go label100))
    (cond
     ((>= qmin (* four emax))
      (setf qmin
              (min qmin
                   (f2cl-lib:fref z-%data%
                                  ((f2cl-lib:int-sub i4 3)) ((1 *))
                                  z-%offset%)))
      (setf emax
              (max emax
                   (f2cl-lib:fref z-%data%
                                  ((f2cl-lib:int-sub i4 5)) ((1 *))
                                  z-%offset%)))))
    (setf qmax
            (max qmax
                 (+
```

```
                    (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 7))
                                   ((1 *)) z-%offset%)
                    (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 5))
                                   ((1 *)) z-%offset%))))
     (setf emin
             (min emin
                  (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 5))
                                 ((1 *)) z-%offset%)))
    label90))
  (setf i4 4)
 label100
  (setf i0 (the f2cl-lib:integer4 (truncate i4 4)))
  (setf pp 0)
  (cond
   ((> (f2cl-lib:int-add n0 (f2cl-lib:int-sub i0)) 1)
    (setf dee
            (f2cl-lib:fref z-%data%
                           ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 i0)
                                              3))
                           ((1 *)) z-%offset%))
    (setf deemin dee) (setf kmin i0)
    (f2cl-lib:fdo (i4 (f2cl-lib:int-add (f2cl-lib:int-mul 4 i0) 1)
                   (f2cl-lib:int-add i4 4))
                  ((> i4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 4 n0)
                                        (f2cl-lib:int-sub 3)))
                   nil)
     (tagbody
       (setf dee
               (* (f2cl-lib:fref z-%data% (i4) ((1 *)) z-%offset%)
                  (/ dee
                     (+ dee
                        (f2cl-lib:fref z-%data%
                                       ((f2cl-lib:int-sub i4 2))
                                       ((1 *)) z-%offset%)))))
       (cond
        ((<= dee deemin) (setf deemin dee)
         (setf kmin (the f2cl-lib:integer4 (truncate (+ i4 3) 4)))))
      label110))
    (cond
     ((and
       (<
        (f2cl-lib:int-mul
         (f2cl-lib:int-add kmin (f2cl-lib:int-sub i0)) 2)
        (f2cl-lib:int-add n0 (f2cl-lib:int-sub kmin)))
       (<= deemin
           (* half
              (f2cl-lib:fref z
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-mul 4 n0)
```

```
                        (f2cl-lib:int-sub 3)))
                      ((1 *))))))
(setf ipn4 (f2cl-lib:int-mul 4 (f2cl-lib:int-add i0 n0)))
(setf pp 2)
(f2cl-lib:fdo (i4 (f2cl-lib:int-mul 4 i0)
               (f2cl-lib:int-add i4 4))
              ((> i4
                  (f2cl-lib:int-mul 2
                                    (f2cl-lib:int-add i0 n0
                                                      (f2cl-lib:int-sub
                                                       1))))
               nil)
  (tagbody
    (setf temp
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 3))
                           ((1 *)) z-%offset%))
    (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 3))
                         ((1 *)) z-%offset%)
            (f2cl-lib:fref z-%data%
                           ((f2cl-lib:int-sub ipn4 i4 3))
                           ((1 *)) z-%offset%))
    (setf (f2cl-lib:fref z-%data%
                         ((f2cl-lib:int-sub ipn4 i4 3)) ((1 *))
                         z-%offset%)
            temp)
    (setf temp
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 2))
                           ((1 *)) z-%offset%))
    (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 2))
                         ((1 *)) z-%offset%)
            (f2cl-lib:fref z-%data%
                           ((f2cl-lib:int-sub ipn4 i4 2))
                           ((1 *)) z-%offset%))
    (setf (f2cl-lib:fref z-%data%
                         ((f2cl-lib:int-sub ipn4 i4 2)) ((1 *))
                         z-%offset%)
            temp)
    (setf temp
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 1))
                           ((1 *)) z-%offset%))
    (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 1))
                         ((1 *)) z-%offset%)
            (f2cl-lib:fref z-%data%
                           ((f2cl-lib:int-sub ipn4 i4 5))
                           ((1 *)) z-%offset%))
    (setf (f2cl-lib:fref z-%data%
                         ((f2cl-lib:int-sub ipn4 i4 5)) ((1 *))
                         z-%offset%)
            temp)
    (setf temp
```

```
                (f2cl-lib:fref z-%data% (i4) ((1 *)) z-%offset%))
        (setf (f2cl-lib:fref z-%data% (i4) ((1 *)) z-%offset%)
                (f2cl-lib:fref z-%data%
                               ((f2cl-lib:int-sub ipn4 i4 4))
                               ((1 *)) z-%offset%))
        (setf (f2cl-lib:fref z-%data%
                             ((f2cl-lib:int-sub ipn4 i4 4)) ((1 *))
                             z-%offset%)
                temp)
       label120))))))
(setf dmin
        (-
         (max zero
              (+ qmin
                 (* (- two) (f2cl-lib:fsqrt qmin)
                    (f2cl-lib:fsqrt emax))))))
(setf nbig
        (f2cl-lib:int-mul 100
                          (f2cl-lib:int-add
                           (f2cl-lib:int-sub n0 i0) 1)))
(f2cl-lib:fdo (iwhilb 1 (f2cl-lib:int-add iwhilb 1))
              ((> iwhilb nbig) nil)
  (tagbody
    (if (> i0 n0)
        (go label150))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
         var-17 var-18 var-19)
        (dlasq3 i0 n0 z pp dmin sigma desig qmax nfail iter ndiv
         ieee ttype f2cl-lib:dmin1 dmin2 dn dn1 dn2 g tau)
      (declare (ignore var-2 var-13))
      (when var-0 (setf i0 var-0))
      (when var-1 (setf n0 var-1))
      (when var-3 (setf pp var-3))
      (when var-4 (setf dmin var-4))
      (when var-5 (setf sigma var-5))
      (when var-6 (setf desig var-6))
      (when var-7 (setf qmax var-7))
      (when var-8 (setf nfail var-8))
      (when var-9 (setf iter var-9))
      (when var-10 (setf ndiv var-10))
      (when var-11 (setf ieee var-11))
      (when var-12 (setf ttype var-12))
      (when var-14 (setf dmin2 var-14))
      (when var-15 (setf dn var-15))
      (when var-16 (setf dn1 var-16))
      (when var-17 (setf dn2 var-17))
      (when var-18 (setf g var-18))
      (when var-19 (setf tau var-19)))
```

```
(setf pp (f2cl-lib:int-sub 1 pp))
(cond
 ((and (= pp 0)
       (>= (f2cl-lib:int-add n0 (f2cl-lib:int-sub i0)) 3))
  (cond
   ((or
     (<= (f2cl-lib:fref z ((f2cl-lib:int-mul 4 n0)) ((1 *)))
         (* tol2 qmax))
     (<=
      (f2cl-lib:fref z
                     ((f2cl-lib:int-add (f2cl-lib:int-mul 4 n0)
                                        (f2cl-lib:int-sub 1)))
                     ((1 *)))
      (* tol2 sigma)))
    (setf splt (f2cl-lib:int-sub i0 1))
    (setf qmax
            (f2cl-lib:fref z-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-mul 4 i0) 3))
                           ((1 *)) z-%offset%))
    (setf emin
            (f2cl-lib:fref z-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-mul 4 i0) 1))
                           ((1 *)) z-%offset%))
    (setf oldemn
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-mul 4 i0))
                           ((1 *)) z-%offset%))
    (f2cl-lib:fdo (i4 (f2cl-lib:int-mul 4 i0)
                   (f2cl-lib:int-add i4 4))
                  ((> i4
                      (f2cl-lib:int-mul 4
                                        (f2cl-lib:int-add n0
                                                          (f2cl-lib:int-sub
                                                           3))))
                   nil)
      (tagbody
        (cond
         ((or
           (<= (f2cl-lib:fref z (i4) ((1 *)))
               (* tol2
                  (f2cl-lib:fref z
                                 ((f2cl-lib:int-add i4
                                                    (f2cl-lib:int-sub
                                                     3)))
                                 ((1 *)))))
           (<=
            (f2cl-lib:fref z
                           ((f2cl-lib:int-add i4
                                              (f2cl-lib:int-sub
```

```
                                                  1)))
                                  ((1 *)))
                        (* tol2 sigma)))
                      (setf (f2cl-lib:fref z-%data%
                                           ((f2cl-lib:int-sub i4 1))
                                           ((1 *)) z-%offset%)
                              (- sigma))
                      (setf splt (the f2cl-lib:integer4 (truncate i4 4)))
                      (setf qmax zero)
                      (setf emin
                              (f2cl-lib:fref z-%data%
                                             ((f2cl-lib:int-add i4 3))
                                             ((1 *)) z-%offset%))
                      (setf oldemn
                              (f2cl-lib:fref z-%data%
                                             ((f2cl-lib:int-add i4 4))
                                             ((1 *)) z-%offset%)))
                     (t
                      (setf qmax
                              (max qmax
                                   (f2cl-lib:fref z-%data%
                                                  ((f2cl-lib:int-add i4 1))
                                                  ((1 *)) z-%offset%)))
                      (setf emin
                              (min emin
                                   (f2cl-lib:fref z-%data%
                                                  ((f2cl-lib:int-sub i4 1))
                                                  ((1 *)) z-%offset%)))
                      (setf oldemn
                              (min oldemn
                                   (f2cl-lib:fref z-%data% (i4) ((1 *))
                                                  z-%offset%)))))
                 label130))
              (setf (f2cl-lib:fref z-%data%
                                   ((f2cl-lib:int-sub
                                     (f2cl-lib:int-mul 4 n0) 1))
                                   ((1 *)) z-%offset%)
                      emin)
              (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-mul 4 n0))
                                   ((1 *)) z-%offset%)
                      oldemn)
              (setf i0 (f2cl-lib:int-add splt 1))))))
        label140))
      (setf info 2)
      (setf i1 i0)
      (setf n1 n0)
    label145
      (setf tempq
              (f2cl-lib:fref z-%data%
                             ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 i0)
```

```
                                          3))
                        ((1 *)) z-%offset%))
(setf (f2cl-lib:fref z-%data%
                     ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 i0) 3))
                     ((1 *)) z-%offset%)
        (+
         (f2cl-lib:fref z-%data%
                        ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 i0)
                                           3))
                        ((1 *)) z-%offset%)
         sigma))
(f2cl-lib:fdo (k (f2cl-lib:int-add i0 1) (f2cl-lib:int-add k 1))
              ((> k n0) nil)
  (tagbody
    (setf tempe
            (f2cl-lib:fref z-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-mul 4 k) 5))
                           ((1 *)) z-%offset%))
    (setf (f2cl-lib:fref z-%data%
                         ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 k)
                                            5))
                         ((1 *)) z-%offset%)
            (*
             (f2cl-lib:fref z-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-mul 4 k) 5))
                            ((1 *)) z-%offset%)
             (/ tempq
                (f2cl-lib:fref z-%data%
                               ((f2cl-lib:int-sub
                                 (f2cl-lib:int-mul 4 k) 7))
                               ((1 *)) z-%offset%))))
    (setf tempq
            (f2cl-lib:fref z-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-mul 4 k) 3))
                           ((1 *)) z-%offset%))
    (setf (f2cl-lib:fref z-%data%
                         ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 k)
                                            3))
                         ((1 *)) z-%offset%)
            (-
             (+
              (f2cl-lib:fref z-%data%
                             ((f2cl-lib:int-sub
                               (f2cl-lib:int-mul 4 k) 3))
                             ((1 *)) z-%offset%)
              sigma tempe)
             (f2cl-lib:fref z-%data%
```

```
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-mul 4 k) 5))
                                ((1 *)) z-%offset%)))
   label100000))
(cond
 ((> i1 1)
  (tagbody
    (setf n1 (f2cl-lib:int-sub i1 1))
   label100001
    (if (not
         (and (>= i1 2)
              (>=
               (f2cl-lib:fref z-%data%
                              ((f2cl-lib:int-sub
                                (f2cl-lib:int-mul 4 i1) 5))
                              ((1 *)) z-%offset%)
               zero)))
        (go label100002))
    (setf i1 (f2cl-lib:int-sub i1 1))
    (go label100001)
   label100002
    (setf sigma
            (-
             (f2cl-lib:fref z-%data%
                            ((f2cl-lib:int-sub
                              (f2cl-lib:int-mul 4 n1) 1))
                            ((1 *)) z-%offset%)))
    (go label145))))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (setf (f2cl-lib:fref z-%data%
                         ((f2cl-lib:int-sub (f2cl-lib:int-mul 2 k)
                                            1))
                         ((1 *)) z-%offset%)
            (f2cl-lib:fref z-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-mul 4 k) 3))
                           ((1 *)) z-%offset%))
    (cond
     ((< k n0)
      (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-mul 2 k))
                           ((1 *)) z-%offset%)
              (f2cl-lib:fref z-%data%
                             ((f2cl-lib:int-sub
                               (f2cl-lib:int-mul 4 k) 1))
                             ((1 *)) z-%offset%)))
     (t
      (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-mul 2 k))
                           ((1 *)) z-%offset%)
```

```
                    (coerce (the f2cl-lib:integer4 0) 'double-float))))
        label100003))
     (go end_label)
    label150
    label160))
 (setf info 3)
 (go end_label)
label170
 (f2cl-lib:fdo (k 2 (f2cl-lib:int-add k 1))
               ((> k n) nil)
   (tagbody
     (setf (f2cl-lib:fref z-%data% (k) ((1 *)) z-%offset%)
             (f2cl-lib:fref z-%data%
                            ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 k)
                                               3))
                            ((1 *)) z-%offset%))
    label180))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (dlasrt "D" n z iinfo)
   (declare (ignore var-0 var-2))
   (when var-1 (setf n var-1))
   (when var-3 (setf iinfo var-3)))
 (setf e zero)
 (f2cl-lib:fdo (k n (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
               ((> k 1) nil)
   (tagbody
     (setf e (+ e (f2cl-lib:fref z-%data% (k) ((1 *)) z-%offset%)))
    label190))
 (setf (f2cl-lib:fref z-%data%
                      ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) 1))
                      ((1 *)) z-%offset%)
         trace$)
 (setf (f2cl-lib:fref z-%data%
                      ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) 2))
                      ((1 *)) z-%offset%)
         e)
 (setf (f2cl-lib:fref z-%data%
                      ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) 3))
                      ((1 *)) z-%offset%)
         (f2cl-lib:dble iter))
 (setf (f2cl-lib:fref z-%data%
                      ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) 4))
                      ((1 *)) z-%offset%)
         (/ (f2cl-lib:dble ndiv) (f2cl-lib:dble (expt n 2))))
 (setf (f2cl-lib:fref z-%data%
                      ((f2cl-lib:int-add (f2cl-lib:int-mul 2 n) 5))
                      ((1 *)) z-%offset%)
         (/ (* hundrd nfail) (f2cl-lib:dble iter)))
 (go end_label)
end_label
```

```
        (return (values n nil info))))))
```

———-

90.184 dlasq3

90.184.1 Axiom unit tests

— **dlasq3.input** —

```
)set break resume
)sys rm -f dlasq3.output
)spool dlasq3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.184.2 Axiom help page

— **dlasq3.help** —

```
======================================================================
dlasq3 examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DLASQ3 - for deflation, computes a shift (TAU) and calls dqds

SYNOPSIS
       SUBROUTINE DLASQ3( I0,  N0,  Z,  PP,  DMIN,  SIGMA, DESIG, QMAX, NFAIL,
                          ITER, NDIV, IEEE )

           LOGICAL        IEEE
```

```
      INTEGER        I0, ITER, N0, NDIV, NFAIL, PP

      DOUBLE         PRECISION DESIG, DMIN, QMAX, SIGMA

      DOUBLE         PRECISION Z( * )
```

```
Purpose
=======

DLASQ3 checks for deflation, computes a shift (TAU) and calls dqds.
In case of failure it changes shifts, and tries again until output
is positive.

Arguments
=========

I0     (input) INTEGER
       First index.

N0     (input) INTEGER
       Last index.

Z      (input) DOUBLE PRECISION array, dimension ( 4*N )
       Z holds the qd array.

PP     (input) INTEGER
       PP=0 for ping, PP=1 for pong.

DMIN   (output) DOUBLE PRECISION
       Minimum value of d.

SIGMA  (output) DOUBLE PRECISION
       Sum of shifts used in current segment.

DESIG  (input/output) DOUBLE PRECISION
       Lower order part of SIGMA

QMAX   (input) DOUBLE PRECISION
       Maximum value of q.

NFAIL  (output) INTEGER
       Number of times shift was too big.

ITER   (output) INTEGER
       Number of iterations.

NDIV   (output) INTEGER
       Number of divisions.

TTYPE  (output) INTEGER
```

```
        Shift type.

  IEEE   (input) LOGICAL
         Flag for IEEE or non IEEE arithmetic (passed to DLASQ5).
```

90.184.3 fortran code

— dlasq3.f —

```
      SUBROUTINE DLASQ3( I0, N0, Z, PP, DMIN, SIGMA, DESIG, QMAX, NFAIL,
     $                   ITER, NDIV, IEEE )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     May 17, 2000
*
*     .. Scalar Arguments ..
      LOGICAL            IEEE
      INTEGER            I0, ITER, N0, NDIV, NFAIL, PP
      DOUBLE PRECISION   DESIG, DMIN, QMAX, SIGMA
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   Z( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   CBIAS
      PARAMETER          ( CBIAS = 1.50D0 )
      DOUBLE PRECISION   ZERO, QURTR, HALF, ONE, TWO, HUNDRD
      PARAMETER          ( ZERO = 0.0D0, QURTR = 0.250D0, HALF = 0.5D0,
     $                     ONE = 1.0D0, TWO = 2.0D0, HUNDRD = 100.0D0 )
*     ..
*     .. Local Scalars ..
      INTEGER            IPN4, J4, NOIN, NN, TTYPE
      DOUBLE PRECISION   DMIN1, DMIN2, DN, DN1, DN2, EPS, S, SAFMIN, T,
     $                   TAU, TEMP, TOL, TOL2
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLASQ4, DLASQ5, DLASQ6
*     ..
*     .. External Function ..
      DOUBLE PRECISION   DLAMCH
```

```
      EXTERNAL           DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MIN, SQRT
*     ..
*     .. Save statement ..
      SAVE               TTYPE
      SAVE               DMIN1, DMIN2, DN, DN1, DN2, TAU
*     ..
*     .. Data statement ..
      DATA               TTYPE / 0 /
      DATA               DMIN1 / ZERO /, DMIN2 / ZERO /, DN / ZERO /,
     $                   DN1 / ZERO /, DN2 / ZERO /, TAU / ZERO /
*     ..
*     .. Executable Statements ..
*
      NOIN = NO
      EPS = DLAMCH( 'Precision' )
      SAFMIN = DLAMCH( 'Safe minimum' )
      TOL = EPS*HUNDRD
      TOL2 = TOL**2
*
*     Check for deflation.
*
   10 CONTINUE
*
      IF( NO.LT.IO )
     $   RETURN
      IF( NO.EQ.IO )
     $   GO TO 20
      NN = 4*NO + PP
      IF( NO.EQ.( IO+1 ) )
     $   GO TO 40
*
*     Check whether E(NO-1) is negligible, 1 eigenvalue.
*
      IF( Z( NN-5 ).GT.TOL2*( SIGMA+Z( NN-3 ) ) .AND.
     $    Z( NN-2*PP-4 ).GT.TOL2*Z( NN-7 ) )
     $   GO TO 30
*
   20 CONTINUE
*
      Z( 4*NO-3 ) = Z( 4*NO+PP-3 ) + SIGMA
      NO = NO - 1
      GO TO 10
*
*     Check  whether E(NO-2) is negligible, 2 eigenvalues.
*
   30 CONTINUE
*
```

```
      IF( Z( NN-9 ).GT.TOL2*SIGMA .AND.
     $    Z( NN-2*PP-8 ).GT.TOL2*Z( NN-11 ) )
     $   GO TO 50
*
   40 CONTINUE
*
      IF( Z( NN-3 ).GT.Z( NN-7 ) ) THEN
         S = Z( NN-3 )
         Z( NN-3 ) = Z( NN-7 )
         Z( NN-7 ) = S
      END IF
      IF( Z( NN-5 ).GT.Z( NN-3 )*TOL2 ) THEN
         T = HALF*( ( Z( NN-7 )-Z( NN-3 ) )+Z( NN-5 ) )
         S = Z( NN-3 )*( Z( NN-5 ) / T )
         IF( S.LE.T ) THEN
            S = Z( NN-3 )*( Z( NN-5 ) /
     $          ( T*( ONE+SQRT( ONE+S / T ) ) ) )
         ELSE
            S = Z( NN-3 )*( Z( NN-5 ) / ( T+SQRT( T )*SQRT( T+S ) ) )
         END IF
         T = Z( NN-7 ) + ( S+Z( NN-5 ) )
         Z( NN-3 ) = Z( NN-3 )*( Z( NN-7 ) / T )
         Z( NN-7 ) = T
      END IF
      Z( 4*N0-7 ) = Z( NN-7 ) + SIGMA
      Z( 4*N0-3 ) = Z( NN-3 ) + SIGMA
      N0 = N0 - 2
      GO TO 10
*
   50 CONTINUE
*
*     Reverse the qd-array, if warranted.
*
      IF( DMIN.LE.ZERO .OR. N0.LT.N0IN ) THEN
         IF( CBIAS*Z( 4*I0+PP-3 ).LT.Z( 4*N0+PP-3 ) ) THEN
            IPN4 = 4*( I0+N0 )
            DO 60 J4 = 4*I0, 2*( I0+N0-1 ), 4
               TEMP = Z( J4-3 )
               Z( J4-3 ) = Z( IPN4-J4-3 )
               Z( IPN4-J4-3 ) = TEMP
               TEMP = Z( J4-2 )
               Z( J4-2 ) = Z( IPN4-J4-2 )
               Z( IPN4-J4-2 ) = TEMP
               TEMP = Z( J4-1 )
               Z( J4-1 ) = Z( IPN4-J4-5 )
               Z( IPN4-J4-5 ) = TEMP
               TEMP = Z( J4 )
               Z( J4 ) = Z( IPN4-J4-4 )
               Z( IPN4-J4-4 ) = TEMP
   60       CONTINUE
```

```
            IF( N0-I0.LE.4 ) THEN
               Z( 4*N0+PP-1 ) = Z( 4*I0+PP-1 )
               Z( 4*N0-PP ) = Z( 4*I0-PP )
            END IF
            DMIN2 = MIN( DMIN2, Z( 4*N0+PP-1 ) )
            Z( 4*N0+PP-1 ) = MIN( Z( 4*N0+PP-1 ), Z( 4*I0+PP-1 ),
     $                            Z( 4*I0+PP+3 ) )
            Z( 4*N0-PP ) = MIN( Z( 4*N0-PP ), Z( 4*I0-PP ),
     $                          Z( 4*I0-PP+4 ) )
            QMAX = MAX( QMAX, Z( 4*I0+PP-3 ), Z( 4*I0+PP+1 ) )
            DMIN = -ZERO
         END IF
      END IF
*
   70 CONTINUE
*
      IF( DMIN.LT.ZERO .OR. SAFMIN*QMAX.LT.MIN( Z( 4*N0+PP-1 ),
     $    Z( 4*N0+PP-9 ), DMIN2+Z( 4*N0-PP ) ) ) THEN
*
*        Choose a shift.
*
         CALL DLASQ4( I0, N0, Z, PP, N0IN, DMIN, DMIN1, DMIN2, DN, DN1,
     $                DN2, TAU, TTYPE )
*
*        Call dqds until DMIN > 0.
*
   80    CONTINUE
*
         CALL DLASQ5( I0, N0, Z, PP, TAU, DMIN, DMIN1, DMIN2, DN,
     $                DN1, DN2, IEEE )
*
         NDIV = NDIV + ( N0-I0+2 )
         ITER = ITER + 1
*
*        Check status.
*
         IF( DMIN.GE.ZERO .AND. DMIN1.GT.ZERO ) THEN
*
*           Success.
*
            GO TO 100
*
         ELSE IF( DMIN.LT.ZERO .AND. DMIN1.GT.ZERO .AND.
     $            Z( 4*( N0-1 )-PP ).LT.TOL*( SIGMA+DN1 ) .AND.
     $            ABS( DN ).LT.TOL*SIGMA ) THEN
*
*           Convergence hidden by negative DN.
*
            Z( 4*( N0-1 )-PP+2 ) = ZERO
            DMIN = ZERO
```

```
            GO TO 100
         ELSE IF( DMIN.LT.ZERO ) THEN
*
*           TAU too big. Select new TAU and try again.
*
            NFAIL = NFAIL + 1
            IF( TTYPE.LT.-22 ) THEN
*
*              Failed twice. Play it safe.
*
               TAU = ZERO
            ELSE IF( DMIN1.GT.ZERO ) THEN
*
*              Late failure. Gives excellent shift.
*
               TAU = ( TAU+DMIN )*( ONE-TWO*EPS )
               TTYPE = TTYPE - 11
            ELSE
*
*              Early failure. Divide by 4.
*
               TAU = QURTR*TAU
               TTYPE = TTYPE - 12
            END IF
            GO TO 80
         ELSE IF( DMIN.NE.DMIN ) THEN
*
*           NaN.
*
            TAU = ZERO
            GO TO 80
         ELSE
*
*           Possible underflow. Play it safe.
*
            GO TO 90
         END IF
      END IF
*
*     Risk of underflow.
*
   90 CONTINUE
      CALL DLASQ6( I0, N0, Z, PP, DMIN, DMIN1, DMIN2, DN, DN1, DN2 )
      NDIV = NDIV + ( N0-I0+2 )
      ITER = ITER + 1
      TAU = ZERO
*
  100 CONTINUE
      IF( TAU.LT.SIGMA ) THEN
         DESIG = DESIG + TAU
```

```
         T = SIGMA + DESIG
         DESIG = DESIG - ( T-SIGMA )
      ELSE
         T = SIGMA + TAU
         DESIG = SIGMA - ( T-TAU ) + DESIG
      END IF
      SIGMA = T
*
      RETURN
*
*     End of DLASQ3
*
      END
```

———-

90.184.4 lisp code

— **dlasq3.lisp** —

```
(let* ((cbias 1.5d0)
       (zero 0.0d0)
       (qurtr 0.25d0)
       (half 0.5d0)
       (one 1.0d0)
       (two 2.0d0)
       (hundrd 100.0d0))
  (declare (type (double-float 1.5d0 1.5d0) cbias)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 0.25d0 0.25d0) qurtr)
           (type (double-float 0.5d0 0.5d0) half)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 100.0d0 100.0d0) hundrd)
           (ignorable cbias zero qurtr half one two hundrd))
  (defun dlasq3
         (i0 n0 z pp dmin sigma desig qmax nfail iter ndiv ieee ttype dmin1
          dmin2 dn dn1 dn2 g tau)
    (declare (type f2cl-lib:logical ieee)
             (type (double-float) tau g dn2 dn1 dn dmin2 dmin1 qmax desig sigma
              dmin)
             (type (array double-float (*)) z)
             (type (f2cl-lib:integer4) ttype ndiv iter nfail pp n0 i0))
    (f2cl-lib:with-multi-array-data
        ((z double-float z-%data% z-%offset%))
      (prog ((eps 0.0d0) (s 0.0d0) (t$ 0.0d0) (temp 0.0d0) (tol 0.0d0)
             (tol2 0.0d0) (ipn4 0) (j4 0) (n0in 0) (nn 0))
```

```
 (declare (type (double-float) eps s t$ temp tol tol2)
          (type (f2cl-lib:integer4) ipn4 j4 n0in nn))
 (setf n0in n0)
 (setf eps (dlamch "Precision"))
 (setf tol (* eps hundrd))
 (setf tol2 (expt tol 2))
label10
 (if (< n0 i0)
     (go end_label))
 (if (= n0 i0)
     (go label20))
 (setf nn (f2cl-lib:int-add (f2cl-lib:int-mul 4 n0) pp))
 (if (= n0 (f2cl-lib:int-add i0 1))
     (go label40))
 (if (and
      (>
       (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 5)) ((1 *))
                      z-%offset%)
       (* tol2
          (+ sigma
             (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 3)) ((1 *))
                            z-%offset%))))
      (>
       (f2cl-lib:fref z-%data%
                      ((f2cl-lib:int-sub
                        (f2cl-lib:int-add nn (f2cl-lib:int-mul -1 2 pp))
                        4))
                      ((1 *)) z-%offset%)
       (* tol2
          (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 7)) ((1 *))
                         z-%offset%))))
     (go label30))
label20
 (setf (f2cl-lib:fref z-%data%
                      ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 n0) 3))
                      ((1 *)) z-%offset%)
         (+
          (f2cl-lib:fref z-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add (f2cl-lib:int-mul 4 n0) pp)
                           3))
                         ((1 *)) z-%offset%)
          sigma))
 (setf n0 (f2cl-lib:int-sub n0 1))
 (go label10)
label30
 (if (and
      (>
       (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 9)) ((1 *))
                      z-%offset%)
```

```
        (* tol2 sigma))
       (>
        (f2cl-lib:fref z-%data%
                       ((f2cl-lib:int-sub
                         (f2cl-lib:int-add nn (f2cl-lib:int-mul -1 2 pp))
                         8))
                       ((1 *)) z-%offset%)
        (* tol2
           (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 11)) ((1 *))
                          z-%offset%))))
      (go label50))
 label40
  (cond
   ((>
     (f2cl-lib:fref z ((f2cl-lib:int-add nn (f2cl-lib:int-sub 3)))
                    ((1 *)))
     (f2cl-lib:fref z ((f2cl-lib:int-add nn (f2cl-lib:int-sub 7)))
                    ((1 *))))
    (setf s
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 3)) ((1 *))
                           z-%offset%))
    (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 3)) ((1 *))
                         z-%offset%)
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 7)) ((1 *))
                           z-%offset%))
    (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 7)) ((1 *))
                         z-%offset%)
            s)))
  (setf t$
          (* half
             (+
              (-
               (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 7)) ((1 *))
                              z-%offset%)
               (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 3)) ((1 *))
                              z-%offset%))
              (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 5)) ((1 *))
                             z-%offset%))))
  (cond
   ((and
     (>
      (f2cl-lib:fref z ((f2cl-lib:int-add nn (f2cl-lib:int-sub 5)))
                     ((1 *)))
      (*
       (f2cl-lib:fref z ((f2cl-lib:int-add nn (f2cl-lib:int-sub 3)))
                      ((1 *)))
       tol2))
     (/= t$ zero))
    (setf s
            (*
```

```
                (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 3)) ((1 *))
                               z-%offset%)
                (/
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 5)) ((1 *))
                                z-%offset%)
                 t$)))
        (cond
         ((<= s t$)
          (setf s
                  (*
                   (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 3)) ((1 *))
                                  z-%offset%)
                   (/
                    (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 5)) ((1 *))
                                   z-%offset%)
                    (* t$ (+ one (f2cl-lib:fsqrt (+ one (/ s t$)))))))))
         (t
          (setf s
                  (*
                   (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 3)) ((1 *))
                                  z-%offset%)
                   (/
                    (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 5)) ((1 *))
                                   z-%offset%)
                    (+ t$
                       (* (f2cl-lib:fsqrt t$)
                          (f2cl-lib:fsqrt (+ t$ s)))))))))
        (setf t$
                (+
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 7)) ((1 *))
                                z-%offset%)
                 (+ s
                    (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 5)) ((1 *))
                                   z-%offset%))))
        (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 3)) ((1 *))
                             z-%offset%)
                (*
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 3)) ((1 *))
                                z-%offset%)
                 (/
                  (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 7)) ((1 *))
                                 z-%offset%)
                  t$)))
        (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 7)) ((1 *))
                             z-%offset%)
                t$)))
      (setf (f2cl-lib:fref z-%data%
                           ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 n0) 7))
                           ((1 *)) z-%offset%)
              (+
```

```
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 7)) ((1 *))
                           z-%offset%)
            sigma))
    (setf (f2cl-lib:fref z-%data%
                         ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 n0) 3))
                         ((1 *)) z-%offset%)
          (+
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 3)) ((1 *))
                           z-%offset%)
            sigma))
    (setf n0 (f2cl-lib:int-sub n0 2))
    (go label10)
   label50
    (if (= pp 2)
        (setf pp 0))
    (cond
     ((or (<= dmin zero) (< n0 n0in))
      (cond
       ((<
         (* cbias
            (f2cl-lib:fref z
                           ((f2cl-lib:int-add (f2cl-lib:int-mul 4 i0) pp
                                              (f2cl-lib:int-sub 3)))
                           ((1 *))))
         (f2cl-lib:fref z
                        ((f2cl-lib:int-add (f2cl-lib:int-mul 4 n0) pp
                                           (f2cl-lib:int-sub 3)))
                        ((1 *))))
        (setf ipn4 (f2cl-lib:int-mul 4 (f2cl-lib:int-add i0 n0)))
        (f2cl-lib:fdo (j4 (f2cl-lib:int-mul 4 i0) (f2cl-lib:int-add j4 4))
                      ((> j4
                          (f2cl-lib:int-mul 2
                                            (f2cl-lib:int-add i0 n0
                                                              (f2cl-lib:int-sub
                                                               1))))
                       nil)
         (tagbody
           (setf temp
                   (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 3))
                                  ((1 *)) z-%offset%))
           (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 3)) ((1 *))
                                z-%offset%)
                   (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub ipn4 j4 3))
                                  ((1 *)) z-%offset%))
           (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub ipn4 j4 3))
                                ((1 *)) z-%offset%)
                   temp)
           (setf temp
                   (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                                  ((1 *)) z-%offset%))
```

```
    (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2)) ((1 *))
                         z-%offset%)
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub ipn4 j4 2))
                           ((1 *)) z-%offset%))
    (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub ipn4 j4 2))
                         ((1 *)) z-%offset%)
            temp)
    (setf temp
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 1))
                           ((1 *)) z-%offset%))
    (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 1)) ((1 *))
                         z-%offset%)
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub ipn4 j4 5))
                           ((1 *)) z-%offset%))
    (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub ipn4 j4 5))
                         ((1 *)) z-%offset%)
            temp)
    (setf temp (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%))
    (setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub ipn4 j4 4))
                           ((1 *)) z-%offset%))
    (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub ipn4 j4 4))
                         ((1 *)) z-%offset%)
            temp)
   label60))
(cond
 ((<= (f2cl-lib:int-add n0 (f2cl-lib:int-sub i0)) 4)
  (setf (f2cl-lib:fref z-%data%
                       ((f2cl-lib:int-sub
                         (f2cl-lib:int-add (f2cl-lib:int-mul 4 n0)
                                           pp)
                         1))
                       ((1 *)) z-%offset%)
          (f2cl-lib:fref z-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add
                            (f2cl-lib:int-mul 4 i0) pp)
                           1))
                         ((1 *)) z-%offset%))
  (setf (f2cl-lib:fref z-%data%
                       ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 n0)
                                          pp))
                       ((1 *)) z-%offset%)
          (f2cl-lib:fref z-%data%
                         ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 i0)
                                            pp))
                         ((1 *)) z-%offset%))))
(setf dmin2
        (min dmin2
             (f2cl-lib:fref z-%data%
```

```
                                ((f2cl-lib:int-sub
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-mul 4 n0) pp)
                                  1))
                                ((1 *)) z-%offset%)))
(setf (f2cl-lib:fref z-%data%
                     ((f2cl-lib:int-sub
                       (f2cl-lib:int-add (f2cl-lib:int-mul 4 n0)
                                         pp)
                       1))
                     ((1 *)) z-%offset%)
        (min
         (f2cl-lib:fref z-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add (f2cl-lib:int-mul 4 n0)
                                            pp)
                          1))
                        ((1 *)) z-%offset%)
         (f2cl-lib:fref z-%data%
                        ((f2cl-lib:int-sub
                          (f2cl-lib:int-add (f2cl-lib:int-mul 4 i0)
                                            pp)
                          1))
                        ((1 *)) z-%offset%)
         (f2cl-lib:fref z-%data%
                        ((f2cl-lib:int-add (f2cl-lib:int-mul 4 i0)
                                           pp 3))
                        ((1 *)) z-%offset%)))
(setf (f2cl-lib:fref z-%data%
                     ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 n0)
                                        pp))
                     ((1 *)) z-%offset%)
        (min
         (f2cl-lib:fref z-%data%
                        ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 n0)
                                           pp))
                        ((1 *)) z-%offset%)
         (f2cl-lib:fref z-%data%
                        ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 i0)
                                           pp))
                        ((1 *)) z-%offset%)
         (f2cl-lib:fref z-%data%
                        ((f2cl-lib:int-add
                          (f2cl-lib:int-sub (f2cl-lib:int-mul 4 i0)
                                            pp)
                          4))
                        ((1 *)) z-%offset%)))
(setf qmax
        (max qmax
             (f2cl-lib:fref z-%data%
```

```
                                        ((f2cl-lib:int-sub
                                          (f2cl-lib:int-add
                                           (f2cl-lib:int-mul 4 i0) pp)
                                          3))
                                        ((1 *)) z-%offset%)
                        (f2cl-lib:fref z-%data%
                                        ((f2cl-lib:int-add
                                          (f2cl-lib:int-mul 4 i0) pp 1))
                                        ((1 *)) z-%offset%)))
        (setf dmin (- zero))))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
         var-11 var-12 var-13)
        (dlasq4 i0 n0 z pp n0in dmin dmin1 dmin2 dn dn1 dn2 tau ttype g)
      (declare (ignore var-2 var-6))
      (when var-0 (setf i0 var-0))
      (when var-1 (setf n0 var-1))
      (when var-3 (setf pp var-3))
      (when var-4 (setf n0in var-4))
      (when var-5 (setf dmin var-5))
      (when var-7 (setf dmin2 var-7))
      (when var-8 (setf dn var-8))
      (when var-9 (setf dn1 var-9))
      (when var-10 (setf dn2 var-10))
      (when var-11 (setf tau var-11))
      (when var-12 (setf ttype var-12))
      (when var-13 (setf g var-13)))
  label70
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
         var-11 var-12 var-13)
        (dlasq5 i0 n0 z pp tau sigma dmin dmin1 dmin2 dn dn1 dn2 ieee eps)
      (declare (ignore var-2 var-7))
      (when var-0 (setf i0 var-0))
      (when var-1 (setf n0 var-1))
      (when var-3 (setf pp var-3))
      (when var-4 (setf tau var-4))
      (when var-5 (setf sigma var-5))
      (when var-6 (setf dmin var-6))
      (when var-8 (setf dmin2 var-8))
      (when var-9 (setf dn var-9))
      (when var-10 (setf dn1 var-10))
      (when var-11 (setf dn2 var-11))
      (when var-12 (setf ieee var-12))
      (when var-13 (setf eps var-13)))
    (setf ndiv
            (f2cl-lib:int-add ndiv
                              (f2cl-lib:int-add (f2cl-lib:int-sub n0 i0)
                                                2)))
    (setf iter (f2cl-lib:int-add iter 1))
```

```
 (cond ((and (>= dmin zero) (>= dmin1 zero)) (go label90))
       ((and (< dmin zero) (> dmin1 zero)
             (<
              (f2cl-lib:fref z
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-mul 4
                                                 (f2cl-lib:int-add n0
                                                                   (f2cl-lib:int-sub
                                                                    1)))
                               (f2cl-lib:int-sub pp)))
                             ((1 *)))
              (* tol (+ sigma dn1)))
             (< (abs dn) (* tol sigma)))
        (setf (f2cl-lib:fref z-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-sub
                                (f2cl-lib:int-mul 4
                                                  (f2cl-lib:int-sub n0
                                                                    1))
                                pp)
                               2))
                             ((1 *)) z-%offset%)
                zero)
        (setf dmin zero) (go label90))
       ((< dmin zero) (setf nfail (f2cl-lib:int-add nfail 1))
        (cond ((< ttype (f2cl-lib:int-sub 22)) (setf tau zero))
              ((> dmin1 zero)
               (setf tau (* (+ tau dmin) (- one (* two eps))))
               (setf ttype (f2cl-lib:int-sub ttype 11)))
              (t (setf tau (* qurtr tau))
               (setf ttype (f2cl-lib:int-sub ttype 12))))
        (go label70))
       ((multiple-value-bind (ret-val var-0)
            (disnan dmin)
          (declare (ignore))
          (setf dmin var-0)
          ret-val)
        (cond ((= tau zero) (go label80))
              (t (setf tau zero) (go label70))))
       (t (go label80)))
label80
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
     (dlasq6 i0 n0 z pp dmin dmin1 dmin2 dn dn1 dn2)
   (declare (ignore var-2 var-5))
   (when var-0 (setf i0 var-0))
   (when var-1 (setf n0 var-1))
   (when var-3 (setf pp var-3))
   (when var-4 (setf dmin var-4))
   (when var-6 (setf dmin2 var-6))
```

```
          (when var-7 (setf dn var-7))
          (when var-8 (setf dn1 var-8))
          (when var-9 (setf dn2 var-9)))
        (setf ndiv
                (f2cl-lib:int-add ndiv
                                  (f2cl-lib:int-add (f2cl-lib:int-sub n0 i0)
                                                    2)))
        (setf iter (f2cl-lib:int-add iter 1))
        (setf tau zero)
       label90
        (cond
         ((< tau sigma) (setf desig (+ desig tau)) (setf t$ (+ sigma desig))
          (setf desig (- desig (- t$ sigma))))
         (t (setf t$ (+ sigma tau))
          (setf desig (+ (- sigma (- t$ tau)) desig))))
        (setf sigma t$)
        (go end_label)
       end_label
        (return
         (values i0 n0 nil pp dmin sigma desig qmax nfail iter ndiv ieee ttype
                 nil dmin2 dn dn1 dn2 g tau))))))
```

———-

90.185 dlasq4

90.185.1 Axiom unit tests

— **dlasq4.input** —

```
)set break resume
)sys rm -f dlasq4.output
)spool dlasq4.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.185.2 Axiom help page

— dlasq4.help —

```
=====================================================================
dlasq4 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

NAME
       DLASQ4  -  an approximation TAU to the smallest eigenvalue using values
       of d from the previous transform

SYNOPSIS
       SUBROUTINE DLASQ4( I0, N0, Z, PP, N0IN, DMIN, DMIN1,  DMIN2,  DN,  DN1,
                          DN2, TAU, TTYPE )

           INTEGER        I0, N0, N0IN, PP, TTYPE

           DOUBLE         PRECISION DMIN, DMIN1, DMIN2, DN, DN1, DN2, TAU

           DOUBLE         PRECISION Z( * )

  Purpose
  =======

  DLASQ4 computes an approximation TAU to the smallest eigenvalue
  using values of d from the previous transform.

  I0    (input) INTEGER
        First index.

  N0    (input) INTEGER
        Last index.

  Z     (input) DOUBLE PRECISION array, dimension ( 4*N )
        Z holds the qd array.

  PP    (input) INTEGER
        PP=0 for ping, PP=1 for pong.

  N0IN  (input) INTEGER
        The value of N0 at start of EIGTEST.

  DMIN  (input) DOUBLE PRECISION
        Minimum value of d.

  DMIN1 (input) DOUBLE PRECISION
        Minimum value of d, excluding D( N0 ).
```

```
DMIN2 (input) DOUBLE PRECISION
      Minimum value of d, excluding D( N0 ) and D( N0-1 ).

DN    (input) DOUBLE PRECISION
      d(N)

DN1   (input) DOUBLE PRECISION
      d(N-1)

DN2   (input) DOUBLE PRECISION
      d(N-2)

TAU   (output) DOUBLE PRECISION
      This is the shift.

TTYPE (output) INTEGER
      Shift type.

Further Details
===============
CNST1 = 9/16
```

———-

90.185.3 fortran code

— dlasq4.f —

```
      SUBROUTINE DLASQ4( I0, N0, Z, PP, N0IN, DMIN, DMIN1, DMIN2, DN,
     $                   DN1, DN2, TAU, TTYPE )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1999
*
*     .. Scalar Arguments ..
      INTEGER            I0, N0, N0IN, PP, TTYPE
      DOUBLE PRECISION   DMIN, DMIN1, DMIN2, DN, DN1, DN2, TAU
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   Z( * )
*     ..
*
*  =====================================================================
*
```

```
*     .. Parameters ..
      DOUBLE PRECISION   CNST1, CNST2, CNST3
      PARAMETER          ( CNST1 = 0.5630D0, CNST2 = 1.010D0,
     $                   CNST3 = 1.050D0 )
      DOUBLE PRECISION   QURTR, THIRD, HALF, ZERO, ONE, TWO, HUNDRD
      PARAMETER          ( QURTR = 0.250D0, THIRD = 0.3330D0,
     $                   HALF = 0.50D0, ZERO = 0.0D0, ONE = 1.0D0,
     $                   TWO = 2.0D0, HUNDRD = 100.0D0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I4, NN, NP
      DOUBLE PRECISION   A2, B1, B2, G, GAM, GAP1, GAP2, S
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN, SQRT
*     ..
*     .. Save statement ..
      SAVE               G
*     ..
*     .. Data statement ..
      DATA               G / ZERO /
*     ..
*     .. Executable Statements ..
*
*     A negative DMIN forces the shift to take that absolute value
*     TTYPE records the type of shift.
*
      IF( DMIN.LE.ZERO ) THEN
         TAU = -DMIN
         TTYPE = -1
         RETURN
      END IF
*
      NN = 4*N0 + PP
      IF( N0IN.EQ.N0 ) THEN
*
*        No eigenvalues deflated.
*
         IF( DMIN.EQ.DN .OR. DMIN.EQ.DN1 ) THEN
*
            B1 = SQRT( Z( NN-3 ) )*SQRT( Z( NN-5 ) )
            B2 = SQRT( Z( NN-7 ) )*SQRT( Z( NN-9 ) )
            A2 = Z( NN-7 ) + Z( NN-5 )
*
*           Cases 2 and 3.
*
            IF( DMIN.EQ.DN .AND. DMIN1.EQ.DN1 ) THEN
               GAP2 = DMIN2 - A2 - DMIN2*QURTR
               IF( GAP2.GT.ZERO .AND. GAP2.GT.B2 ) THEN
                  GAP1 = A2 - DN - ( B2 / GAP2 )*B2
```

```
               ELSE
                  GAP1 = A2 - DN - ( B1+B2 )
               END IF
               IF( GAP1.GT.ZERO .AND. GAP1.GT.B1 ) THEN
                  S = MAX( DN-( B1 / GAP1 )*B1, HALF*DMIN )
                  TTYPE = -2
               ELSE
                  S = ZERO
                  IF( DN.GT.B1 )
     $               S = DN - B1
                  IF( A2.GT.( B1+B2 ) )
     $               S = MIN( S, A2-( B1+B2 ) )
                  S = MAX( S, THIRD*DMIN )
                  TTYPE = -3
               END IF
            ELSE
*
*              Case 4.
*
               TTYPE = -4
               S = QURTR*DMIN
               IF( DMIN.EQ.DN ) THEN
                  GAM = DN
                  A2 = ZERO
                  IF( Z( NN-5 ) .GT. Z( NN-7 ) )
     $               RETURN
                  B2 = Z( NN-5 ) / Z( NN-7 )
                  NP = NN - 9
               ELSE
                  NP = NN - 2*PP
                  B2 = Z( NP-2 )
                  GAM = DN1
                  IF( Z( NP-4 ) .GT. Z( NP-2 ) )
     $               RETURN
                  A2 = Z( NP-4 ) / Z( NP-2 )
                  IF( Z( NN-9 ) .GT. Z( NN-11 ) )
     $               RETURN
                  B2 = Z( NN-9 ) / Z( NN-11 )
                  NP = NN - 13
               END IF
*
*              Approximate contribution to norm squared from I < NN-1.
*
               A2 = A2 + B2
               DO 10 I4 = NP, 4*I0 - 1 + PP, -4
                  IF( B2.EQ.ZERO )
     $               GO TO 20
                  B1 = B2
                  IF( Z( I4 ) .GT. Z( I4-2 ) )
     $               RETURN
```

```
                 B2 = B2*( Z( I4 ) / Z( I4-2 ) )
                 A2 = A2 + B2
                 IF( HUNDRD*MAX( B2, B1 ).LT.A2 .OR. CNST1.LT.A2 )
     $              GO TO 20
   10          CONTINUE
   20          CONTINUE
               A2 = CNST3*A2
*
*              Rayleigh quotient residual bound.
*
               IF( A2.LT.CNST1 )
     $            S = GAM*( ONE-SQRT( A2 ) ) / ( ONE+A2 )
            END IF
         ELSE IF( DMIN.EQ.DN2 ) THEN
*
*           Case 5.
*
            TTYPE = -5
            S = QURTR*DMIN
*
*           Compute contribution to norm squared from I > NN-2.
*
            NP = NN - 2*PP
            B1 = Z( NP-2 )
            B2 = Z( NP-6 )
            GAM = DN2
            IF( Z( NP-8 ).GT.B2 .OR. Z( NP-4 ).GT.B1 )
     $         RETURN
            A2 = ( Z( NP-8 ) / B2 )*( ONE+Z( NP-4 ) / B1 )
*
*           Approximate contribution to norm squared from I < NN-2.
*
            IF( N0-I0.GT.2 ) THEN
               B2 = Z( NN-13 ) / Z( NN-15 )
               A2 = A2 + B2
               DO 30 I4 = NN - 17, 4*I0 - 1 + PP, -4
                  IF( B2.EQ.ZERO )
     $               GO TO 40
                  B1 = B2
                  IF( Z( I4 ) .GT. Z( I4-2 ) )
     $               RETURN
                  B2 = B2*( Z( I4 ) / Z( I4-2 ) )
                  A2 = A2 + B2
                  IF( HUNDRD*MAX( B2, B1 ).LT.A2 .OR. CNST1.LT.A2 )
     $               GO TO 40
   30          CONTINUE
   40          CONTINUE
               A2 = CNST3*A2
            END IF
*
```

```
            IF( A2.LT.CNST1 )
     $         S = GAM*( ONE-SQRT( A2 ) ) / ( ONE+A2 )
         ELSE
*
*           Case 6, no information to guide us.
*
            IF( TTYPE.EQ.-6 ) THEN
               G = G + THIRD*( ONE-G )
            ELSE IF( TTYPE.EQ.-18 ) THEN
               G = QURTR*THIRD
            ELSE
               G = QURTR
            END IF
            S = G*DMIN
            TTYPE = -6
         END IF
*
      ELSE IF( NOIN.EQ.( NO+1 ) ) THEN
*
*        One eigenvalue just deflated. Use DMIN1, DN1 for DMIN and DN.
*
         IF( DMIN1.EQ.DN1 .AND. DMIN2.EQ.DN2 ) THEN
*
*           Cases 7 and 8.
*
            TTYPE = -7
            S = THIRD*DMIN1
            IF( Z( NN-5 ).GT.Z( NN-7 ) )
     $         RETURN
            B1 = Z( NN-5 ) / Z( NN-7 )
            B2 = B1
            IF( B2.EQ.ZERO )
     $         GO TO 60
            DO 50 I4 = 4*N0 - 9 + PP, 4*I0 - 1 + PP, -4
               A2 = B1
               IF( Z( I4 ).GT.Z( I4-2 ) )
     $            RETURN
               B1 = B1*( Z( I4 ) / Z( I4-2 ) )
               B2 = B2 + B1
               IF( HUNDRD*MAX( B1, A2 ).LT.B2 )
     $            GO TO 60
   50       CONTINUE
   60       CONTINUE
            B2 = SQRT( CNST3*B2 )
            A2 = DMIN1 / ( ONE+B2**2 )
            GAP2 = HALF*DMIN2 - A2
            IF( GAP2.GT.ZERO .AND. GAP2.GT.B2*A2 ) THEN
               S = MAX( S, A2*( ONE-CNST2*A2*( B2 / GAP2 )*B2 ) )
            ELSE
               S = MAX( S, A2*( ONE-CNST2*B2 ) )
```

```
               TTYPE = -8
            END IF
         ELSE
*
*           Case 9.
*
            S = QURTR*DMIN1
            IF( DMIN1.EQ.DN1 )
     $         S = HALF*DMIN1
            TTYPE = -9
         END IF
*
      ELSE IF( NOIN.EQ.( N0+2 ) ) THEN
*
*        Two eigenvalues deflated. Use DMIN2, DN2 for DMIN and DN.
*
*        Cases 10 and 11.
*
         IF( DMIN2.EQ.DN2 .AND. TWO*Z( NN-5 ).LT.Z( NN-7 ) ) THEN
            TTYPE = -10
            S = THIRD*DMIN2
            IF( Z( NN-5 ).GT.Z( NN-7 ) )
     $         RETURN
            B1 = Z( NN-5 ) / Z( NN-7 )
            B2 = B1
            IF( B2.EQ.ZERO )
     $         GO TO 80
            DO 70 I4 = 4*N0 - 9 + PP, 4*I0 - 1 + PP, -4
               IF( Z( I4 ).GT.Z( I4-2 ) )
     $            RETURN
               B1 = B1*( Z( I4 ) / Z( I4-2 ) )
               B2 = B2 + B1
               IF( HUNDRD*B1.LT.B2 )
     $            GO TO 80
   70       CONTINUE
   80       CONTINUE
            B2 = SQRT( CNST3*B2 )
            A2 = DMIN2 / ( ONE+B2**2 )
            GAP2 = Z( NN-7 ) + Z( NN-9 ) -
     $             SQRT( Z( NN-11 ) )*SQRT( Z( NN-9 ) ) - A2
            IF( GAP2.GT.ZERO .AND. GAP2.GT.B2*A2 ) THEN
               S = MAX( S, A2*( ONE-CNST2*A2*( B2 / GAP2 )*B2 ) )
            ELSE
               S = MAX( S, A2*( ONE-CNST2*B2 ) )
            END IF
         ELSE
            S = QURTR*DMIN2
            TTYPE = -11
         END IF
      ELSE IF( NOIN.GT.( N0+2 ) ) THEN
```

```
*
*        Case 12, more than two eigenvalues deflated. No information.
*
         S = ZERO
         TTYPE = -12
      END IF
*
      TAU = S
      RETURN
*
*     End of DLASQ4
*
      END
```

———-

90.185.4 lisp code

— dlasq4.lisp —

```
(let* ((cnst1 0.563d0)
       (cnst2 1.01d0)
       (cnst3 1.05d0)
       (qurtr 0.25d0)
       (third$ 0.333d0)
       (half 0.5d0)
       (zero 0.0d0)
       (one 1.0d0)
       (two 2.0d0)
       (hundrd 100.0d0))
  (declare (type (double-float 0.563d0 0.563d0) cnst1)
           (type (double-float 1.01d0 1.01d0) cnst2)
           (type (double-float 1.05d0 1.05d0) cnst3)
           (type (double-float 0.25d0 0.25d0) qurtr)
           (type (double-float 0.333d0 0.333d0) third$)
           (type (double-float 0.5d0 0.5d0) half)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 100.0d0 100.0d0) hundrd)
           (ignorable cnst1 cnst2 cnst3 qurtr third$ half zero one two hundrd))
  (defun dlasq4 (i0 n0 z pp n0in dmin dmin1 dmin2 dn dn1 dn2 tau ttype g)
    (declare (type (double-float) g tau dn2 dn1 dn dmin2 dmin1 dmin)
             (type (array double-float (*)) z)
             (type (f2cl-lib:integer4) ttype n0in pp n0 i0))
    (f2cl-lib:with-multi-array-data
        ((z double-float z-%data% z-%offset%))
```

```
(prog ((a2 0.0d0) (b1 0.0d0) (b2 0.0d0) (gam 0.0d0) (gap1 0.0d0)
       (gap2 0.0d0) (s 0.0d0) (i4 0) (nn 0) (np 0))
  (declare (type (double-float) a2 b1 b2 gam gap1 gap2 s)
           (type (f2cl-lib:integer4) i4 nn np))
  (cond
   ((<= dmin zero) (setf tau (- dmin)) (setf ttype -1) (go end_label)))
  (setf nn (f2cl-lib:int-add (f2cl-lib:int-mul 4 n0) pp))
  (cond
   ((= n0in n0)
    (cond
     ((or (= dmin dn) (= dmin dn1))
      (setf b1
              (*
               (f2cl-lib:fsqrt
                (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 3)) ((1 *))
                               z-%offset%))
               (f2cl-lib:fsqrt
                (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 5)) ((1 *))
                               z-%offset%))))
      (setf b2
              (*
               (f2cl-lib:fsqrt
                (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 7)) ((1 *))
                               z-%offset%))
               (f2cl-lib:fsqrt
                (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 9)) ((1 *))
                               z-%offset%))))
      (setf a2
              (+
               (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 7)) ((1 *))
                              z-%offset%)
               (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 5)) ((1 *))
                              z-%offset%)))
      (cond
       ((and (= dmin dn) (= dmin1 dn1))
        (setf gap2 (- dmin2 a2 (* dmin2 qurtr)))
        (cond
         ((and (> gap2 zero) (> gap2 b2))
          (setf gap1 (- a2 dn (* (/ b2 gap2) b2))))
         (t (setf gap1 (- a2 dn (+ b1 b2)))))
        (cond
         ((and (> gap1 zero) (> gap1 b1))
          (setf s (max (- dn (* (/ b1 gap1) b1)) (* half dmin)))
          (setf ttype -2))
         (t (setf s zero)
          (if (> dn b1)
              (setf s (- dn b1)))
          (if (> a2 (+ b1 b2))
              (setf s (min s (- a2 (+ b1 b2)))))
          (setf s (max s (* third$ dmin))) (setf ttype -3))))
```

```
(t
 (tagbody
   (setf ttype -4)
   (setf s (* qurtr dmin))
   (cond
    ((= dmin dn) (setf gam dn) (setf a2 zero)
     (if (>
          (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 5))
                         ((1 *)) z-%offset%)
          (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 7))
                         ((1 *)) z-%offset%))
         (go end_label))
     (setf b2
             (/
              (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 5))
                             ((1 *)) z-%offset%)
              (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 7))
                             ((1 *)) z-%offset%)))
     (setf np (f2cl-lib:int-sub nn 9)))
    (t (setf np (f2cl-lib:int-sub nn (f2cl-lib:int-mul 2 pp)))
     (setf b2
             (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub np 2))
                            ((1 *)) z-%offset%))
     (setf gam dn1)
     (if (>
          (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub np 4))
                         ((1 *)) z-%offset%)
          (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub np 2))
                         ((1 *)) z-%offset%))
         (go end_label))
     (setf a2
             (/
              (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub np 4))
                             ((1 *)) z-%offset%)
              (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub np 2))
                             ((1 *)) z-%offset%)))
     (if (>
          (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 9))
                         ((1 *)) z-%offset%)
          (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 11))
                         ((1 *)) z-%offset%))
         (go end_label))
     (setf b2
             (/
              (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 9))
                             ((1 *)) z-%offset%)
              (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 11))
                             ((1 *)) z-%offset%)))
     (setf np (f2cl-lib:int-sub nn 13))))
   (setf a2 (+ a2 b2))
```

```
        (f2cl-lib:fdo (i4 np (f2cl-lib:int-add i4 (f2cl-lib:int-sub 4)))
                      ((> i4
                          (f2cl-lib:int-add (f2cl-lib:int-mul 4 i0)
                                            (f2cl-lib:int-sub 1) pp))
                       nil)
          (tagbody
            (if (= b2 zero)
                (go label20))
            (setf b1 b2)
            (if (> (f2cl-lib:fref z-%data% (i4) ((1 *)) z-%offset%)
                   (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 2))
                                  ((1 *)) z-%offset%))
                (go end_label))
            (setf b2
                    (* b2
                       (/
                        (f2cl-lib:fref z-%data% (i4) ((1 *))
                                       z-%offset%)
                        (f2cl-lib:fref z-%data%
                                       ((f2cl-lib:int-sub i4 2))
                                       ((1 *)) z-%offset%))))
            (setf a2 (+ a2 b2))
            (if (or (< (* hundrd (max b2 b1)) a2) (< cnst1 a2))
                (go label20))
           label10))
       label20
        (setf a2 (* cnst3 a2))
        (if (< a2 cnst1)
            (setf s
                    (/ (* gam (- one (f2cl-lib:fsqrt a2)))
                       (+ one a2))))))))
    ((= dmin dn2) (setf ttype -5) (setf s (* qurtr dmin))
     (setf np (f2cl-lib:int-sub nn (f2cl-lib:int-mul 2 pp)))
     (setf b1
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub np 2)) ((1 *))
                           z-%offset%))
     (setf b2
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub np 6)) ((1 *))
                           z-%offset%))
     (setf gam dn2)
     (if (or
          (>
           (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub np 8)) ((1 *))
                          z-%offset%)
           b2)
          (>
           (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub np 4)) ((1 *))
                          z-%offset%)
           b1))
        (go end_label))
```

```
(setf a2
        (*
         (/
          (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub np 8)) ((1 *))
                         z-%offset%)
          b2)
         (+ one
            (/
             (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub np 4))
                            ((1 *)) z-%offset%)
             b1))))
(cond
 ((> (f2cl-lib:int-add n0 (f2cl-lib:int-sub i0)) 2)
  (tagbody
    (setf b2
            (/
             (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 13))
                            ((1 *)) z-%offset%)
             (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 15))
                            ((1 *)) z-%offset%)))
    (setf a2 (+ a2 b2))
    (f2cl-lib:fdo (i4 (f2cl-lib:int-add nn (f2cl-lib:int-sub 17))
                   (f2cl-lib:int-add i4 (f2cl-lib:int-sub 4)))
                  ((> i4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 4 i0)
                                        (f2cl-lib:int-sub 1) pp))
                   nil)
      (tagbody
        (if (= b2 zero)
            (go label40))
        (setf b1 b2)
        (if (> (f2cl-lib:fref z-%data% (i4) ((1 *)) z-%offset%)
               (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 2))
                              ((1 *)) z-%offset%))
            (go end_label))
        (setf b2
                (* b2
                   (/
                    (f2cl-lib:fref z-%data% (i4) ((1 *))
                                   z-%offset%)
                    (f2cl-lib:fref z-%data%
                                   ((f2cl-lib:int-sub i4 2))
                                   ((1 *)) z-%offset%))))
        (setf a2 (+ a2 b2))
        (if (or (< (* hundrd (max b2 b1)) a2) (< cnst1 a2))
            (go label40))
       label30))
   label40
    (setf a2 (* cnst3 a2)))))
(if (< a2 cnst1)
```

```
        (setf s (/ (* gam (- one (f2cl-lib:fsqrt a2))) (+ one a2)))))
    (t
     (cond
      ((= ttype (f2cl-lib:int-sub 6))
       (setf g (+ g (* third$ (- one g)))))
      ((= ttype (f2cl-lib:int-sub 18)) (setf g (* qurtr third$)))
      (t (setf g qurtr)))
     (setf s (* g dmin)) (setf ttype -6))))
  ((= n0in (f2cl-lib:int-add n0 1))
   (cond
    ((and (= dmin1 dn1) (= dmin2 dn2))
     (tagbody
       (setf ttype -7)
       (setf s (* third$ dmin1))
       (if (>
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 5)) ((1 *))
                           z-%offset%)
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 7)) ((1 *))
                           z-%offset%))
           (go end_label))
       (setf b1
               (/
                (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 5))
                               ((1 *)) z-%offset%)
                (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 7))
                               ((1 *)) z-%offset%)))
       (setf b2 b1)
       (if (= b2 zero)
           (go label60))
       (f2cl-lib:fdo (i4
                      (f2cl-lib:int-add (f2cl-lib:int-mul 4 n0)
                                        (f2cl-lib:int-sub 9) pp)
                      (f2cl-lib:int-add i4 (f2cl-lib:int-sub 4)))
                     ((> i4
                         (f2cl-lib:int-add (f2cl-lib:int-mul 4 i0)
                                           (f2cl-lib:int-sub 1) pp))
                      nil)
         (tagbody
           (setf a2 b1)
           (if (> (f2cl-lib:fref z-%data% (i4) ((1 *)) z-%offset%)
                  (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 2))
                                 ((1 *)) z-%offset%))
               (go end_label))
           (setf b1
                   (* b1
                      (/
                       (f2cl-lib:fref z-%data% (i4) ((1 *)) z-%offset%)
                       (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 2))
                                      ((1 *)) z-%offset%))))
           (setf b2 (+ b2 b1))
```

```
            (if (< (* hundrd (max b1 a2)) b2)
                (go label60))
           label50))
       label60
        (setf b2 (f2cl-lib:fsqrt (* cnst3 b2)))
        (setf a2 (/ dmin1 (+ one (expt b2 2))))
        (setf gap2 (- (* half dmin2) a2))
        (cond
         ((and (> gap2 zero) (> gap2 (* b2 a2)))
          (setf s
                  (max s
                       (* a2 (+ one (* (- cnst2) a2 (/ b2 gap2) b2))))))
         (t (setf s (max s (* a2 (- one (* cnst2 b2)))))
          (setf ttype -8)))))
      (t (setf s (* qurtr dmin1))
       (if (= dmin1 dn1)
           (setf s (* half dmin1)))
       (setf ttype -9))))
    ((= n0in (f2cl-lib:int-add n0 2))
     (cond
      ((and (= dmin2 dn2)
            (<
             (* two
                (f2cl-lib:fref z
                               ((f2cl-lib:int-add nn
                                                  (f2cl-lib:int-sub 5)))
                               ((1 *))))
             (f2cl-lib:fref z ((f2cl-lib:int-add nn (f2cl-lib:int-sub 7)))
                            ((1 *)))))
       (tagbody
        (setf ttype -10)
        (setf s (* third$ dmin2))
        (if (>
             (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 5)) ((1 *))
                            z-%offset%)
             (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 7)) ((1 *))
                            z-%offset%))
            (go end_label))
        (setf b1
                (/
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 5))
                                ((1 *)) z-%offset%)
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 7))
                                ((1 *)) z-%offset%)))
        (setf b2 b1)
        (if (= b2 zero)
            (go label80))
        (f2cl-lib:fdo (i4
                       (f2cl-lib:int-add (f2cl-lib:int-mul 4 n0)
                                         (f2cl-lib:int-sub 9) pp)
```

```
                        (f2cl-lib:int-add i4 (f2cl-lib:int-sub 4)))
                       ((> i4
                           (f2cl-lib:int-add (f2cl-lib:int-mul 4 i0)
                                             (f2cl-lib:int-sub 1) pp))
                        nil)
          (tagbody
            (if (> (f2cl-lib:fref z-%data% (i4) ((1 *)) z-%offset%)
                   (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 2))
                                  ((1 *)) z-%offset%))
                (go end_label))
            (setf b1
                    (* b1
                       (/
                        (f2cl-lib:fref z-%data% (i4) ((1 *)) z-%offset%)
                        (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub i4 2))
                                       ((1 *)) z-%offset%))))
            (setf b2 (+ b2 b1))
            (if (< (* hundrd b1) b2)
                (go label80))
           label70))
      label80
       (setf b2 (f2cl-lib:fsqrt (* cnst3 b2)))
       (setf a2 (/ dmin2 (+ one (expt b2 2))))
       (setf gap2
               (-
                (+
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 7))
                                ((1 *)) z-%offset%)
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 9))
                                ((1 *)) z-%offset%))
                (*
                 (f2cl-lib:fsqrt
                  (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 11))
                                 ((1 *)) z-%offset%))
                 (f2cl-lib:fsqrt
                  (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub nn 9))
                                 ((1 *)) z-%offset%)))
                a2))
       (cond
        ((and (> gap2 zero) (> gap2 (* b2 a2)))
         (setf s
                 (max s
                      (* a2 (+ one (* (- cnst2) a2 (/ b2 gap2) b2))))))
        (t (setf s (max s (* a2 (- one (* cnst2 b2)))))))))
    (t (setf s (* qurtr dmin2)) (setf ttype -11))))
  ((> n0in (f2cl-lib:int-add n0 2)) (setf s zero) (setf ttype -12)))
 (setf tau s)
 (go end_label)
end_label
 (return
```

```
        (values nil nil nil nil nil nil nil nil nil nil nil tau ttype g))))))
```

———-

90.186 dlasq5

90.186.1 Axiom unit tests

— **dlasq5.input** —

```
)set break resume
)sys rm -f dlasq5.output
)spool dlasq5.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.186.2 Axiom help page

— **dlasq5.help** —

```
====================================================================
dlasq5 examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DLASQ5  -  one  dqds  transform in ping-pong form, one version for IEEE
       machines another for non IEEE machines

SYNOPSIS
       SUBROUTINE DLASQ5( I0, N0, Z, PP, TAU, DMIN, DMIN1,  DMIN2,  DN,  DNM1,
                          DNM2, IEEE )

           LOGICAL        IEEE
```

```
      INTEGER        I0, N0, PP

      DOUBLE         PRECISION DMIN, DMIN1, DMIN2, DN, DNM1, DNM2, TAU

      DOUBLE         PRECISION Z( * )
```

Purpose
=======

DLASQ5 computes one dqds transform in ping-pong form, one
version for IEEE machines another for non IEEE machines.

Arguments
=========

```
I0    (input) INTEGER
      First index.

N0    (input) INTEGER
      Last index.

Z     (input) DOUBLE PRECISION array, dimension ( 4*N )
      Z holds the qd array. EMIN is stored in Z(4*N0) to avoid
      an extra argument.

PP    (input) INTEGER
      PP=0 for ping, PP=1 for pong.

TAU   (input) DOUBLE PRECISION
      This is the shift.

DMIN  (output) DOUBLE PRECISION
      Minimum value of d.

DMIN1 (output) DOUBLE PRECISION
      Minimum value of d, excluding D( N0 ).

DMIN2 (output) DOUBLE PRECISION
      Minimum value of d, excluding D( N0 ) and D( N0-1 ).

DN    (output) DOUBLE PRECISION
      d(N0), the last value of d.

DNM1  (output) DOUBLE PRECISION
      d(N0-1).

DNM2  (output) DOUBLE PRECISION
      d(N0-2).
```

```
 IEEE  (input) LOGICAL
       Flag for IEEE or non IEEE arithmetic.
```

———-

90.186.3 fortran code

— **dlasq5.f** —

```
      SUBROUTINE DLASQ5( I0, N0, Z, PP, TAU, DMIN, DMIN1, DMIN2, DN,
     $                   DNM1, DNM2, IEEE )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     May 17, 2000
*
*     .. Scalar Arguments ..
      LOGICAL            IEEE
      INTEGER            I0, N0, PP
      DOUBLE PRECISION   DMIN, DMIN1, DMIN2, DN, DNM1, DNM2, TAU
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   Z( * )
*     ..
*
*  =====================================================================
*
*     .. Parameter ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
*     ..
*     .. Local Scalars ..
      INTEGER            J4, J4P2
      DOUBLE PRECISION   D, EMIN, TEMP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MIN
*     ..
*     .. Executable Statements ..
*
      IF( ( N0-I0-1 ).LE.0 )
     $   RETURN
*
      J4 = 4*I0 + PP - 3
      EMIN = Z( J4+4 )
      D = Z( J4 ) - TAU
```

```
      DMIN = D
      DMIN1 = -Z( J4 )
*
      IF( IEEE ) THEN
*
*        Code for IEEE arithmetic.
*
         IF( PP.EQ.0 ) THEN
            DO 10 J4 = 4*I0, 4*( N0-3 ), 4
               Z( J4-2 ) = D + Z( J4-1 )
               TEMP = Z( J4+1 ) / Z( J4-2 )
               D = D*TEMP - TAU
               DMIN = MIN( DMIN, D )
               Z( J4 ) = Z( J4-1 )*TEMP
               EMIN = MIN( Z( J4 ), EMIN )
   10       CONTINUE
         ELSE
            DO 20 J4 = 4*I0, 4*( N0-3 ), 4
               Z( J4-3 ) = D + Z( J4 )
               TEMP = Z( J4+2 ) / Z( J4-3 )
               D = D*TEMP - TAU
               DMIN = MIN( DMIN, D )
               Z( J4-1 ) = Z( J4 )*TEMP
               EMIN = MIN( Z( J4-1 ), EMIN )
   20       CONTINUE
         END IF
*
*        Unroll last two steps.
*
         DNM2 = D
         DMIN2 = DMIN
         J4 = 4*( N0-2 ) - PP
         J4P2 = J4 + 2*PP - 1
         Z( J4-2 ) = DNM2 + Z( J4P2 )
         Z( J4 ) = Z( J4P2+2 )*( Z( J4P2 ) / Z( J4-2 ) )
         DNM1 = Z( J4P2+2 )*( DNM2 / Z( J4-2 ) ) - TAU
         DMIN = MIN( DMIN, DNM1 )
*
         DMIN1 = DMIN
         J4 = J4 + 4
         J4P2 = J4 + 2*PP - 1
         Z( J4-2 ) = DNM1 + Z( J4P2 )
         Z( J4 ) = Z( J4P2+2 )*( Z( J4P2 ) / Z( J4-2 ) )
         DN = Z( J4P2+2 )*( DNM1 / Z( J4-2 ) ) - TAU
         DMIN = MIN( DMIN, DN )
*
      ELSE
*
*        Code for non IEEE arithmetic.
*
```

```
      IF( PP.EQ.0 ) THEN
         DO 30 J4 = 4*I0, 4*( N0-3 ), 4
            Z( J4-2 ) = D + Z( J4-1 )
            IF( D.LT.ZERO ) THEN
               RETURN
            ELSE
               Z( J4 ) = Z( J4+1 )*( Z( J4-1 ) / Z( J4-2 ) )
               D = Z( J4+1 )*( D / Z( J4-2 ) ) - TAU
            END IF
            DMIN = MIN( DMIN, D )
            EMIN = MIN( EMIN, Z( J4 ) )
 30      CONTINUE
      ELSE
         DO 40 J4 = 4*I0, 4*( N0-3 ), 4
            Z( J4-3 ) = D + Z( J4 )
            IF( D.LT.ZERO ) THEN
               RETURN
            ELSE
               Z( J4-1 ) = Z( J4+2 )*( Z( J4 ) / Z( J4-3 ) )
               D = Z( J4+2 )*( D / Z( J4-3 ) ) - TAU
            END IF
            DMIN = MIN( DMIN, D )
            EMIN = MIN( EMIN, Z( J4-1 ) )
 40      CONTINUE
      END IF
*
*     Unroll last two steps.
*
      DNM2 = D
      DMIN2 = DMIN
      J4 = 4*( N0-2 ) - PP
      J4P2 = J4 + 2*PP - 1
      Z( J4-2 ) = DNM2 + Z( J4P2 )
      IF( DNM2.LT.ZERO ) THEN
         RETURN
      ELSE
         Z( J4 ) = Z( J4P2+2 )*( Z( J4P2 ) / Z( J4-2 ) )
         DNM1 = Z( J4P2+2 )*( DNM2 / Z( J4-2 ) ) - TAU
      END IF
      DMIN = MIN( DMIN, DNM1 )
*
      DMIN1 = DMIN
      J4 = J4 + 4
      J4P2 = J4 + 2*PP - 1
      Z( J4-2 ) = DNM1 + Z( J4P2 )
      IF( DNM1.LT.ZERO ) THEN
         RETURN
      ELSE
         Z( J4 ) = Z( J4P2+2 )*( Z( J4P2 ) / Z( J4-2 ) )
         DN = Z( J4P2+2 )*( DNM1 / Z( J4-2 ) ) - TAU
```

```
         END IF
         DMIN = MIN( DMIN, DN )
*
      END IF
*
      Z( J4+2 ) = DN
      Z( 4*N0-PP ) = EMIN
      RETURN
*
*     End of DLASQ5
*
      END
```

———-

90.186.4 lisp code

— **dlasq5.lisp** —

```
(let* ((zero 0.0d0) (half (coerce 0.5 'double-float)))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float) half)
           (ignorable zero half))
  (defun dlasq5 (i0 n0 z pp tau sigma dmin dmin1 dmin2 dn dnm1 dnm2 ieee eps)
    (declare (type f2cl-lib:logical ieee)
             (type (double-float) eps dnm2 dnm1 dn dmin2 dmin1 dmin sigma tau)
             (type (array double-float (*)) z)
             (type (f2cl-lib:integer4) pp n0 i0))
    (f2cl-lib:with-multi-array-data
        ((z double-float z-%data% z-%offset%))
      (prog ((d 0.0d0) (emin 0.0d0) (temp 0.0d0) (dthresh 0.0d0) (j4 0)
             (j4p2 0))
        (declare (type (double-float) d emin temp dthresh)
                 (type (f2cl-lib:integer4) j4 j4p2))
        (if (<= (f2cl-lib:int-sub n0 i0 1) 0)
            (go end_label))
        (setf dthresh (* eps (+ sigma tau)))
        (if (< tau (* dthresh half))
            (setf tau zero))
        (cond
         ((/= tau zero)
          (setf j4
                  (f2cl-lib:int-sub
                   (f2cl-lib:int-add (f2cl-lib:int-mul 4 i0) pp) 3))
          (setf emin
                  (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4 4)) ((1 *))
                                 z-%offset%))
```

```
(setf d (- (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%) tau))
(setf dmin d)
(setf dmin1 (- (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)))
(cond
 (ieee
  (cond
   ((= pp 0)
    (f2cl-lib:fdo (j4 (f2cl-lib:int-mul 4 i0) (f2cl-lib:int-add j4 4))
                  ((> j4
                      (f2cl-lib:int-mul 4
                                        (f2cl-lib:int-add n0
                                                          (f2cl-lib:int-sub
                                                           3))))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                             ((1 *)) z-%offset%)
                (+ d
                   (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 1))
                                  ((1 *)) z-%offset%)))
        (setf temp
                (/
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4 1))
                                ((1 *)) z-%offset%)
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                                ((1 *)) z-%offset%)))
        (setf d (- (* d temp) tau))
        (setf dmin (min dmin d))
        (setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
                (*
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 1))
                                ((1 *)) z-%offset%)
                 temp))
        (setf emin
                (min (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
                     emin))
       label10)))
   (t
    (f2cl-lib:fdo (j4 (f2cl-lib:int-mul 4 i0) (f2cl-lib:int-add j4 4))
                  ((> j4
                      (f2cl-lib:int-mul 4
                                        (f2cl-lib:int-add n0
                                                          (f2cl-lib:int-sub
                                                           3))))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 3))
                             ((1 *)) z-%offset%)
                (+ d
                   (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)))
```

```
      (setf temp
              (/
               (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4 2))
                              ((1 *)) z-%offset%)
               (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 3))
                              ((1 *)) z-%offset%)))
      (setf d (- (* d temp) tau))
      (setf dmin (min dmin d))
      (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 1))
                           ((1 *)) z-%offset%)
              (* (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
                 temp))
      (setf emin
              (min
               (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 1))
                              ((1 *)) z-%offset%)
               emin))
     label20))))
(setf dnm2 d) (setf dmin2 dmin)
(setf j4
        (f2cl-lib:int-sub
         (f2cl-lib:int-mul 4 (f2cl-lib:int-sub n0 2)) pp))
(setf j4p2
        (f2cl-lib:int-sub
         (f2cl-lib:int-add j4 (f2cl-lib:int-mul 2 pp)) 1))
(setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2)) ((1 *))
                     z-%offset%)
        (+ dnm2
           (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)))
(setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
        (*
         (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2))
                        ((1 *)) z-%offset%)
         (/ (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                           ((1 *)) z-%offset%))))
(setf dnm1
        (-
         (*
          (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2))
                         ((1 *)) z-%offset%)
          (/ dnm2
             (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                            ((1 *)) z-%offset%)))
         tau))
(setf dmin (min dmin dnm1)) (setf dmin1 dmin)
(setf j4 (f2cl-lib:int-add j4 4))
(setf j4p2
        (f2cl-lib:int-sub
         (f2cl-lib:int-add j4 (f2cl-lib:int-mul 2 pp)) 1))
```

```
(setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2)) ((1 *))
                     z-%offset%)
        (+ dnm1
           (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)))
(setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
        (*
         (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2))
                        ((1 *)) z-%offset%)
         (/ (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                           ((1 *)) z-%offset%))))
(setf dn
        (-
         (*
          (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2))
                         ((1 *)) z-%offset%)
          (/ dnm1
             (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                            ((1 *)) z-%offset%)))
         tau))
(setf dmin (min dmin dn)))
(t
 (cond
  ((= pp 0)
   (f2cl-lib:fdo (j4 (f2cl-lib:int-mul 4 i0) (f2cl-lib:int-add j4 4))
                 ((> j4
                     (f2cl-lib:int-mul 4
                                       (f2cl-lib:int-add n0
                                                         (f2cl-lib:int-sub
                                                          3))))
                  nil)
    (tagbody
      (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                           ((1 *)) z-%offset%)
               (+ d
                  (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 1))
                                 ((1 *)) z-%offset%)))
      (cond ((< d zero) (go end_label))
            (t
             (setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
                     (*
                      (f2cl-lib:fref z-%data%
                                     ((f2cl-lib:int-add j4 1))
                                     ((1 *)) z-%offset%)
                      (/
                       (f2cl-lib:fref z-%data%
                                      ((f2cl-lib:int-sub j4 1))
                                      ((1 *)) z-%offset%)
                       (f2cl-lib:fref z-%data%
                                      ((f2cl-lib:int-sub j4 2))
```

```
                                        ((1 *)) z-%offset%))))
                (setf d
                        (-
                         (*
                          (f2cl-lib:fref z-%data%
                                        ((f2cl-lib:int-add j4 1))
                                        ((1 *)) z-%offset%)
                          (/ d
                             (f2cl-lib:fref z-%data%
                                            ((f2cl-lib:int-sub j4 2))
                                            ((1 *)) z-%offset%)))
                         tau))))
        (setf dmin (min dmin d))
        (setf emin
                (min emin
                     (f2cl-lib:fref z-%data% (j4) ((1 *))
                                    z-%offset%)))
      label30)))
    (t
     (f2cl-lib:fdo (j4 (f2cl-lib:int-mul 4 i0) (f2cl-lib:int-add j4 4))
                   ((> j4
                       (f2cl-lib:int-mul 4
                                        (f2cl-lib:int-add n0
                                                          (f2cl-lib:int-sub
                                                           3))))
                    nil)
       (tagbody
        (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 3))
                             ((1 *)) z-%offset%)
                (+ d
                   (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)))
        (cond ((< d zero) (go end_label))
              (t
               (setf (f2cl-lib:fref z-%data%
                                    ((f2cl-lib:int-sub j4 1)) ((1 *))
                                    z-%offset%)
                       (*
                        (f2cl-lib:fref z-%data%
                                       ((f2cl-lib:int-add j4 2))
                                       ((1 *)) z-%offset%)
                        (/
                         (f2cl-lib:fref z-%data% (j4) ((1 *))
                                        z-%offset%)
                         (f2cl-lib:fref z-%data%
                                        ((f2cl-lib:int-sub j4 3))
                                        ((1 *)) z-%offset%))))
               (setf d
                       (-
                        (*
                         (f2cl-lib:fref z-%data%
```

```
                                        ((f2cl-lib:int-add j4 2))
                                        ((1 *)) z-%offset%)
                          (/ d
                             (f2cl-lib:fref z-%data%
                                            ((f2cl-lib:int-sub j4 3))
                                            ((1 *)) z-%offset%)))
                        tau))))
      (setf dmin (min dmin d))
      (setf emin
              (min emin
                   (f2cl-lib:fref z-%data%
                                  ((f2cl-lib:int-sub j4 1)) ((1 *))
                                  z-%offset%)))
     label40))))
(setf dnm2 d) (setf dmin2 dmin)
(setf j4
        (f2cl-lib:int-sub
         (f2cl-lib:int-mul 4 (f2cl-lib:int-sub n0 2)) pp))
(setf j4p2
        (f2cl-lib:int-sub
         (f2cl-lib:int-add j4 (f2cl-lib:int-mul 2 pp)) 1))
(setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2)) ((1 *))
                     z-%offset%)
        (+ dnm2
           (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)))
(cond ((< dnm2 zero) (go end_label))
      (t
       (setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
               (*
                (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2))
                               ((1 *)) z-%offset%)
                (/
                 (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                                ((1 *)) z-%offset%))))
       (setf dnm1
               (-
                (*
                 (f2cl-lib:fref z-%data%
                                ((f2cl-lib:int-add j4p2 2)) ((1 *))
                                z-%offset%)
                 (/ dnm2
                    (f2cl-lib:fref z-%data%
                                   ((f2cl-lib:int-sub j4 2))
                                   ((1 *)) z-%offset%)))
                tau))))
(setf dmin (min dmin dnm1)) (setf dmin1 dmin)
(setf j4 (f2cl-lib:int-add j4 4))
(setf j4p2
        (f2cl-lib:int-sub
```

```
            (f2cl-lib:int-add j4 (f2cl-lib:int-mul 2 pp)) 1))
   (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2)) ((1 *))
                        z-%offset%)
         (+ dnm1
            (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)))
   (cond ((< dnm1 zero) (go end_label))
         (t
          (setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
                  (*
                   (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2))
                                  ((1 *)) z-%offset%)
                   (/
                    (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)
                    (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                                   ((1 *)) z-%offset%))))
          (setf dn
                  (-
                   (*
                    (f2cl-lib:fref z-%data%
                                   ((f2cl-lib:int-add j4p2 2)) ((1 *))
                                   z-%offset%)
                    (/ dnm1
                       (f2cl-lib:fref z-%data%
                                      ((f2cl-lib:int-sub j4 2))
                                      ((1 *)) z-%offset%)))
                   tau))))
   (setf dmin (min dmin dn)))))
(t
 (setf j4
         (f2cl-lib:int-sub
          (f2cl-lib:int-add (f2cl-lib:int-mul 4 i0) pp) 3))
 (setf emin
         (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4 4)) ((1 *))
                        z-%offset%))
 (setf d (- (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%) tau))
 (setf dmin d)
 (setf dmin1 (- (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)))
 (cond
  (ieee
   (cond
    ((= pp 0)
     (f2cl-lib:fdo (j4 (f2cl-lib:int-mul 4 i0) (f2cl-lib:int-add j4 4))
                   ((> j4
                       (f2cl-lib:int-mul 4
                                         (f2cl-lib:int-add n0
                                                           (f2cl-lib:int-sub
                                                            3))))
                    nil)
       (tagbody
         (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
```

```
                                ((1 *)) z-%offset%)
                        (+ d
                           (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 1))
                                          ((1 *)) z-%offset%)))
                (setf temp
                        (/
                         (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4 1))
                                        ((1 *)) z-%offset%)
                         (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                                        ((1 *)) z-%offset%)))
                (setf d (- (* d temp) tau))
                (if (< d dthresh)
                    (setf d zero))
                (setf dmin (min dmin d))
                (setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
                        (*
                         (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 1))
                                        ((1 *)) z-%offset%)
                         temp))
                (setf emin
                        (min (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
                             emin))
            label50)))
        (t
         (f2cl-lib:fdo (j4 (f2cl-lib:int-mul 4 i0) (f2cl-lib:int-add j4 4))
                       ((> j4
                           (f2cl-lib:int-mul 4
                                             (f2cl-lib:int-add n0
                                                               (f2cl-lib:int-sub
                                                                3))))
                        nil)
           (tagbody
                (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 3))
                                      ((1 *)) z-%offset%)
                        (+ d
                           (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)))
                (setf temp
                        (/
                         (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4 2))
                                        ((1 *)) z-%offset%)
                         (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 3))
                                        ((1 *)) z-%offset%)))
                (setf d (- (* d temp) tau))
                (if (< d dthresh)
                    (setf d zero))
                (setf dmin (min dmin d))
                (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 1))
                                      ((1 *)) z-%offset%)
                        (* (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
                           temp))
```

```
      (setf emin
              (min
               (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 1))
                              ((1 *)) z-%offset%)
               emin))
     label60))))
(setf dnm2 d) (setf dmin2 dmin)
(setf j4
        (f2cl-lib:int-sub
         (f2cl-lib:int-mul 4 (f2cl-lib:int-sub n0 2)) pp))
(setf j4p2
        (f2cl-lib:int-sub
         (f2cl-lib:int-add j4 (f2cl-lib:int-mul 2 pp)) 1))
(setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2)) ((1 *))
                     z-%offset%)
        (+ dnm2
           (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)))
(setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
        (*
         (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2))
                        ((1 *)) z-%offset%)
         (/ (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                           ((1 *)) z-%offset%))))
(setf dnm1
        (-
         (*
          (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2))
                         ((1 *)) z-%offset%)
          (/ dnm2
             (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                            ((1 *)) z-%offset%)))
         tau))
(setf dmin (min dmin dnm1)) (setf dmin1 dmin)
(setf j4 (f2cl-lib:int-add j4 4))
(setf j4p2
        (f2cl-lib:int-sub
         (f2cl-lib:int-add j4 (f2cl-lib:int-mul 2 pp)) 1))
(setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2)) ((1 *))
                     z-%offset%)
        (+ dnm1
           (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)))
(setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
        (*
         (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2))
                        ((1 *)) z-%offset%)
         (/ (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)
            (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                           ((1 *)) z-%offset%))))
(setf dn
```

```
            (-
             (*
              (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2))
                             ((1 *)) z-%offset%)
              (/ dnm1
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                                ((1 *)) z-%offset%)))
             tau))
 (setf dmin (min dmin dn)))
(t
 (cond
  ((= pp 0)
   (f2cl-lib:fdo (j4 (f2cl-lib:int-mul 4 i0) (f2cl-lib:int-add j4 4))
                 ((> j4
                     (f2cl-lib:int-mul 4
                                       (f2cl-lib:int-add n0
                                                         (f2cl-lib:int-sub
                                                          3))))
                  nil)
     (tagbody
       (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                            ((1 *)) z-%offset%)
               (+ d
                  (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 1))
                                 ((1 *)) z-%offset%)))
       (cond ((< d zero) (go end_label))
             (t
              (setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
                      (*
                       (f2cl-lib:fref z-%data%
                                      ((f2cl-lib:int-add j4 1))
                                      ((1 *)) z-%offset%)
                       (/
                        (f2cl-lib:fref z-%data%
                                       ((f2cl-lib:int-sub j4 1))
                                       ((1 *)) z-%offset%)
                        (f2cl-lib:fref z-%data%
                                       ((f2cl-lib:int-sub j4 2))
                                       ((1 *)) z-%offset%))))
              (setf d
                      (-
                       (*
                        (f2cl-lib:fref z-%data%
                                       ((f2cl-lib:int-add j4 1))
                                       ((1 *)) z-%offset%)
                        (/ d
                           (f2cl-lib:fref z-%data%
                                          ((f2cl-lib:int-sub j4 2))
                                          ((1 *)) z-%offset%)))
                       tau))))
```

```
      (if (< d dthresh)
          (setf d zero))
      (setf dmin (min dmin d))
      (setf emin
              (min emin
                   (f2cl-lib:fref z-%data% (j4) ((1 *))
                                  z-%offset%)))
    label70)))
 (t
  (f2cl-lib:fdo (j4 (f2cl-lib:int-mul 4 i0) (f2cl-lib:int-add j4 4))
                ((> j4
                    (f2cl-lib:int-mul 4
                                      (f2cl-lib:int-add n0
                                                        (f2cl-lib:int-sub
                                                         3))))
                 nil)
    (tagbody
      (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 3))
                           ((1 *)) z-%offset%)
              (+ d
                 (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)))
      (cond ((< d zero) (go end_label))
            (t
             (setf (f2cl-lib:fref z-%data%
                                  ((f2cl-lib:int-sub j4 1)) ((1 *))
                                  z-%offset%)
                     (*
                      (f2cl-lib:fref z-%data%
                                     ((f2cl-lib:int-add j4 2))
                                     ((1 *)) z-%offset%)
                      (/
                       (f2cl-lib:fref z-%data% (j4) ((1 *))
                                      z-%offset%)
                       (f2cl-lib:fref z-%data%
                                      ((f2cl-lib:int-sub j4 3))
                                      ((1 *)) z-%offset%))))
             (setf d
                     (-
                      (*
                       (f2cl-lib:fref z-%data%
                                      ((f2cl-lib:int-add j4 2))
                                      ((1 *)) z-%offset%)
                       (/ d
                          (f2cl-lib:fref z-%data%
                                         ((f2cl-lib:int-sub j4 3))
                                         ((1 *)) z-%offset%)))
                      tau))))
      (if (< d dthresh)
          (setf d zero))
      (setf dmin (min dmin d))
```

```
      (setf emin
              (min emin
                   (f2cl-lib:fref z-%data%
                                  ((f2cl-lib:int-sub j4 1)) ((1 *))
                                  z-%offset%)))
     label80))))
(setf dnm2 d) (setf dmin2 dmin)
(setf j4
        (f2cl-lib:int-sub
         (f2cl-lib:int-mul 4 (f2cl-lib:int-sub n0 2)) pp))
(setf j4p2
        (f2cl-lib:int-sub
         (f2cl-lib:int-add j4 (f2cl-lib:int-mul 2 pp)) 1))
(setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2)) ((1 *))
                     z-%offset%)
        (+ dnm2
           (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)))
(cond ((< dnm2 zero) (go end_label))
      (t
       (setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
               (*
                (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2))
                               ((1 *)) z-%offset%)
                (/
                 (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                                ((1 *)) z-%offset%))))
       (setf dnm1
               (-
                (*
                 (f2cl-lib:fref z-%data%
                                ((f2cl-lib:int-add j4p2 2)) ((1 *))
                                z-%offset%)
                 (/ dnm2
                    (f2cl-lib:fref z-%data%
                                   ((f2cl-lib:int-sub j4 2))
                                   ((1 *)) z-%offset%)))
                tau))))
(setf dmin (min dmin dnm1)) (setf dmin1 dmin)
(setf j4 (f2cl-lib:int-add j4 4))
(setf j4p2
        (f2cl-lib:int-sub
         (f2cl-lib:int-add j4 (f2cl-lib:int-mul 2 pp)) 1))
(setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2)) ((1 *))
                     z-%offset%)
        (+ dnm1
           (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)))
(cond ((< dnm1 zero) (go end_label))
      (t
       (setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
```

```
                        (*
                         (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2))
                                        ((1 *)) z-%offset%)
                         (/
                          (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)
                          (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                                         ((1 *)) z-%offset%))))
                (setf dn
                        (-
                         (*
                          (f2cl-lib:fref z-%data%
                                         ((f2cl-lib:int-add j4p2 2)) ((1 *))
                                         z-%offset%)
                          (/ dnm1
                             (f2cl-lib:fref z-%data%
                                            ((f2cl-lib:int-sub j4 2))
                                            ((1 *)) z-%offset%)))
                         tau))))
            (setf dmin (min dmin dn))))))
        (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4 2)) ((1 *))
                              z-%offset%)
                dn)
        (setf (f2cl-lib:fref z-%data%
                              ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 n0) pp))
                              ((1 *)) z-%offset%)
                emin)
        (go end_label)
       end_label
        (return
         (values nil nil nil nil tau nil dmin dmin1 dmin2 dn dnm1 dnm2 nil
                 nil))))))
```

———-

90.187 dlasq6

90.187.1 Axiom unit tests

— dlasq6.input —

```
)set break resume
)sys rm -f dlasq6.output
)spool dlasq6.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

90.187.2 Axiom help page

— **dlasq6.help** —

```
=====================================================================
dlasq6 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

NAME
       DLASQ6  -  one  dqd  (shift equal to zero) transform in ping-pong form,
       with protection against underflow and overflow

SYNOPSIS
       SUBROUTINE DLASQ6( I0, N0, Z, PP, DMIN, DMIN1, DMIN2, DN, DNM1, DNM2 )

           INTEGER        I0, N0, PP

           DOUBLE         PRECISION DMIN, DMIN1, DMIN2, DN, DNM1, DNM2

           DOUBLE         PRECISION Z( * )

  Purpose
  =======

  DLASQ6 computes one dqd (shift equal to zero) transform in
  ping-pong form, with protection against underflow and overflow.

  Arguments
  =========

  I0    (input) INTEGER
        First index.

  N0    (input) INTEGER
        Last index.

  Z     (input) DOUBLE PRECISION array, dimension ( 4*N )
        Z holds the qd array. EMIN is stored in Z(4*N0) to avoid
```

```
        an extra argument.

  PP    (input) INTEGER
        PP=0 for ping, PP=1 for pong.

  DMIN  (output) DOUBLE PRECISION
        Minimum value of d.

  DMIN1 (output) DOUBLE PRECISION
        Minimum value of d, excluding D( N0 ).

  DMIN2 (output) DOUBLE PRECISION
        Minimum value of d, excluding D( N0 ) and D( N0-1 ).

  DN    (output) DOUBLE PRECISION
        d(N0), the last value of d.

  DNM1  (output) DOUBLE PRECISION
        d(N0-1).

  DNM2  (output) DOUBLE PRECISION
        d(N0-2).
```

———-

90.187.3 fortran code

— dlasq6.f —

```
      SUBROUTINE DLASQ6( I0, N0, Z, PP, DMIN, DMIN1, DMIN2, DN,
     $                   DNM1, DNM2 )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1999
*
*     .. Scalar Arguments ..
      INTEGER            I0, N0, PP
      DOUBLE PRECISION   DMIN, DMIN1, DMIN2, DN, DNM1, DNM2
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   Z( * )
*     ..
*
*  =====================================================================
*
```

```
*     .. Parameter ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
*     ..
*     .. Local Scalars ..
      INTEGER            J4, J4P2
      DOUBLE PRECISION   D, EMIN, SAFMIN, TEMP
*     ..
*     .. External Function ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MIN
*     ..
*     .. Executable Statements ..
*
      IF( ( N0-I0-1 ).LE.0 )
     $   RETURN
*
      SAFMIN = DLAMCH( 'Safe minimum' )
      J4 = 4*I0 + PP - 3
      EMIN = Z( J4+4 )
      D = Z( J4 )
      DMIN = D
*
      IF( PP.EQ.0 ) THEN
         DO 10 J4 = 4*I0, 4*( N0-3 ), 4
            Z( J4-2 ) = D + Z( J4-1 )
            IF( Z( J4-2 ).EQ.ZERO ) THEN
               Z( J4 ) = ZERO
               D = Z( J4+1 )
               DMIN = D
               EMIN = ZERO
            ELSE IF( SAFMIN*Z( J4+1 ).LT.Z( J4-2 ) .AND.
     $               SAFMIN*Z( J4-2 ).LT.Z( J4+1 ) ) THEN
               TEMP = Z( J4+1 ) / Z( J4-2 )
               Z( J4 ) = Z( J4-1 )*TEMP
               D = D*TEMP
            ELSE
               Z( J4 ) = Z( J4+1 )*( Z( J4-1 ) / Z( J4-2 ) )
               D = Z( J4+1 )*( D / Z( J4-2 ) )
            END IF
            DMIN = MIN( DMIN, D )
            EMIN = MIN( EMIN, Z( J4 ) )
   10    CONTINUE
      ELSE
         DO 20 J4 = 4*I0, 4*( N0-3 ), 4
            Z( J4-3 ) = D + Z( J4 )
            IF( Z( J4-3 ).EQ.ZERO ) THEN
```

```
               Z( J4-1 ) = ZERO
               D = Z( J4+2 )
               DMIN = D
               EMIN = ZERO
            ELSE IF( SAFMIN*Z( J4+2 ).LT.Z( J4-3 ) .AND.
     $              SAFMIN*Z( J4-3 ).LT.Z( J4+2 ) ) THEN
               TEMP = Z( J4+2 ) / Z( J4-3 )
               Z( J4-1 ) = Z( J4 )*TEMP
               D = D*TEMP
            ELSE
               Z( J4-1 ) = Z( J4+2 )*( Z( J4 ) / Z( J4-3 ) )
               D = Z( J4+2 )*( D / Z( J4-3 ) )
            END IF
            DMIN = MIN( DMIN, D )
            EMIN = MIN( EMIN, Z( J4-1 ) )
   20    CONTINUE
      END IF
*
*     Unroll last two steps.
*
      DNM2 = D
      DMIN2 = DMIN
      J4 = 4*( N0-2 ) - PP
      J4P2 = J4 + 2*PP - 1
      Z( J4-2 ) = DNM2 + Z( J4P2 )
      IF( Z( J4-2 ).EQ.ZERO ) THEN
         Z( J4 ) = ZERO
         DNM1 = Z( J4P2+2 )
         DMIN = DNM1
         EMIN = ZERO
      ELSE IF( SAFMIN*Z( J4P2+2 ).LT.Z( J4-2 ) .AND.
     $         SAFMIN*Z( J4-2 ).LT.Z( J4P2+2 ) ) THEN
         TEMP = Z( J4P2+2 ) / Z( J4-2 )
         Z( J4 ) = Z( J4P2 )*TEMP
         DNM1 = DNM2*TEMP
      ELSE
         Z( J4 ) = Z( J4P2+2 )*( Z( J4P2 ) / Z( J4-2 ) )
         DNM1 = Z( J4P2+2 )*( DNM2 / Z( J4-2 ) )
      END IF
      DMIN = MIN( DMIN, DNM1 )
*
      DMIN1 = DMIN
      J4 = J4 + 4
      J4P2 = J4 + 2*PP - 1
      Z( J4-2 ) = DNM1 + Z( J4P2 )
      IF( Z( J4-2 ).EQ.ZERO ) THEN
         Z( J4 ) = ZERO
         DN = Z( J4P2+2 )
         DMIN = DN
         EMIN = ZERO
```

```
      ELSE IF( SAFMIN*Z( J4P2+2 ).LT.Z( J4-2 ) .AND.
     $         SAFMIN*Z( J4-2 ).LT.Z( J4P2+2 ) ) THEN
         TEMP = Z( J4P2+2 ) / Z( J4-2 )
         Z( J4 ) = Z( J4P2 )*TEMP
         DN = DNM1*TEMP
      ELSE
         Z( J4 ) = Z( J4P2+2 )*( Z( J4P2 ) / Z( J4-2 ) )
         DN = Z( J4P2+2 )*( DNM1 / Z( J4-2 ) )
      END IF
      DMIN = MIN( DMIN, DN )
*
      Z( J4+2 ) = DN
      Z( 4*N0-PP ) = EMIN
      RETURN
*
*     End of DLASQ6
*
      END
```

———-

90.187.4 lisp code

— **dlasq6.lisp** —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun dlasq6 (i0 n0 z pp dmin dmin1 dmin2 dn dnm1 dnm2)
    (declare (type (double-float) dnm2 dnm1 dn dmin2 dmin1 dmin)
             (type (array double-float (*)) z)
             (type (f2cl-lib:integer4) pp n0 i0))
    (f2cl-lib:with-multi-array-data
        ((z double-float z-%data% z-%offset%))
      (prog ((d 0.0d0) (emin 0.0d0) (safmin 0.0d0) (temp 0.0d0) (j4 0)
             (j4p2 0))
        (declare (type (double-float) d emin safmin temp)
                 (type (f2cl-lib:integer4) j4 j4p2))
        (if (<= (f2cl-lib:int-sub n0 i0 1) 0)
            (go end_label))
        (setf safmin (dlamch "Safe minimum"))
        (setf j4
                (f2cl-lib:int-sub (f2cl-lib:int-add (f2cl-lib:int-mul 4 i0) pp)
                                  3))
        (setf emin
                (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4 4)) ((1 *))
                               z-%offset%))
```

```
(setf d (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%))
(setf dmin d)
(cond
 ((= pp 0)
  (f2cl-lib:fdo (j4 (f2cl-lib:int-mul 4 i0) (f2cl-lib:int-add j4 4))
                ((> j4
                    (f2cl-lib:int-mul 4
                                      (f2cl-lib:int-add n0
                                                        (f2cl-lib:int-sub
                                                         3))))
                 nil)
    (tagbody
      (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2)) ((1 *))
                           z-%offset%)
              (+ d
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 1))
                                ((1 *)) z-%offset%)))
      (cond
       ((=
         (f2cl-lib:fref z ((f2cl-lib:int-add j4 (f2cl-lib:int-sub 2)))
                        ((1 *)))
         zero)
        (setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%) zero)
        (setf d
                (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4 1))
                               ((1 *)) z-%offset%))
        (setf dmin d) (setf emin zero))
       ((and
         (<
          (* safmin
             (f2cl-lib:fref z ((f2cl-lib:int-add j4 1)) ((1 *))))
          (f2cl-lib:fref z ((f2cl-lib:int-add j4 (f2cl-lib:int-sub 2)))
                         ((1 *))))
         (<
          (* safmin
             (f2cl-lib:fref z
                            ((f2cl-lib:int-add j4
                                               (f2cl-lib:int-sub 2)))
                            ((1 *))))
          (f2cl-lib:fref z ((f2cl-lib:int-add j4 1)) ((1 *)))))
        (setf temp
                (/
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4 1))
                                ((1 *)) z-%offset%)
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                                ((1 *)) z-%offset%)))
        (setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
                (*
                 (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 1))
                                ((1 *)) z-%offset%)
```

```
                    temp))
            (setf d (* d temp)))
           (t
            (setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
                    (*
                     (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4 1))
                                    ((1 *)) z-%offset%)
                     (/
                      (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 1))
                                     ((1 *)) z-%offset%)
                      (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                                     ((1 *)) z-%offset%))))
            (setf d
                    (*
                     (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4 1))
                                    ((1 *)) z-%offset%)
                     (/ d
                        (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2))
                                       ((1 *)) z-%offset%))))))
          (setf dmin (min dmin d))
          (setf emin
                  (min emin
                       (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)))
        label10)))
     (t
      (f2cl-lib:fdo (j4 (f2cl-lib:int-mul 4 i0) (f2cl-lib:int-add j4 4))
                    ((> j4
                        (f2cl-lib:int-mul 4
                                          (f2cl-lib:int-add n0
                                                            (f2cl-lib:int-sub
                                                             3))))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 3)) ((1 *))
                               z-%offset%)
                  (+ d (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)))
          (cond
           ((=
             (f2cl-lib:fref z ((f2cl-lib:int-add j4 (f2cl-lib:int-sub 3)))
                            ((1 *)))
             zero)
            (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 1)) ((1 *))
                                 z-%offset%)
                    zero)
            (setf d
                    (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4 2))
                                   ((1 *)) z-%offset%))
            (setf dmin d) (setf emin zero))
           ((and
             (<
```

```
            (* safmin
               (f2cl-lib:fref z ((f2cl-lib:int-add j4 2)) ((1 *))))
            (f2cl-lib:fref z ((f2cl-lib:int-add j4 (f2cl-lib:int-sub 3)))
                           ((1 *))))
           (<
            (* safmin
               (f2cl-lib:fref z
                              ((f2cl-lib:int-add j4
                                                 (f2cl-lib:int-sub 3)))
                              ((1 *))))
            (f2cl-lib:fref z ((f2cl-lib:int-add j4 2)) ((1 *)))))
          (setf temp
                  (/
                   (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4 2))
                                  ((1 *)) z-%offset%)
                   (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 3))
                                  ((1 *)) z-%offset%)))
          (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 1)) ((1 *))
                               z-%offset%)
                  (* (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
                     temp))
          (setf d (* d temp)))
         (t
          (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 1)) ((1 *))
                               z-%offset%)
                  (*
                   (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4 2))
                                  ((1 *)) z-%offset%)
                   (/ (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
                      (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 3))
                                     ((1 *)) z-%offset%))))
          (setf d
                  (*
                   (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4 2))
                                  ((1 *)) z-%offset%)
                   (/ d
                      (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 3))
                                     ((1 *)) z-%offset%))))))
        (setf dmin (min dmin d))
        (setf emin
                (min emin
                     (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 1))
                                    ((1 *)) z-%offset%)))
      label20))))
(setf dnm2 d)
(setf dmin2 dmin)
(setf j4
        (f2cl-lib:int-sub (f2cl-lib:int-mul 4 (f2cl-lib:int-sub n0 2))
                          pp))
(setf j4p2
```

```
        (f2cl-lib:int-sub (f2cl-lib:int-add j4 (f2cl-lib:int-mul 2 pp))
                          1))
(setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2)) ((1 *))
                     z-%offset%)
        (+ dnm2 (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)))
(cond
 ((=
   (f2cl-lib:fref z ((f2cl-lib:int-add j4 (f2cl-lib:int-sub 2)))
                  ((1 *)))
   zero)
  (setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%) zero)
  (setf dnm1
          (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2)) ((1 *))
                         z-%offset%))
  (setf dmin dnm1) (setf emin zero))
 ((and
   (< (* safmin (f2cl-lib:fref z ((f2cl-lib:int-add j4p2 2)) ((1 *))))
      (f2cl-lib:fref z ((f2cl-lib:int-add j4 (f2cl-lib:int-sub 2)))
                     ((1 *))))
   (<
    (* safmin
       (f2cl-lib:fref z ((f2cl-lib:int-add j4 (f2cl-lib:int-sub 2)))
                      ((1 *))))
    (f2cl-lib:fref z ((f2cl-lib:int-add j4p2 2)) ((1 *)))))
  (setf temp
          (/
           (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2)) ((1 *))
                          z-%offset%)
           (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2)) ((1 *))
                          z-%offset%)))
  (setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
          (* (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%) temp))
  (setf dnm1 (* dnm2 temp)))
 (t
  (setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
          (*
           (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2)) ((1 *))
                          z-%offset%)
           (/ (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)
              (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2)) ((1 *))
                             z-%offset%))))
  (setf dnm1
          (*
           (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2)) ((1 *))
                          z-%offset%)
           (/ dnm2
              (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2)) ((1 *))
                             z-%offset%))))))
(setf dmin (min dmin dnm1))
(setf dmin1 dmin)
```

```
(setf j4 (f2cl-lib:int-add j4 4))
(setf j4p2
        (f2cl-lib:int-sub (f2cl-lib:int-add j4 (f2cl-lib:int-mul 2 pp))
                          1))
(setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2)) ((1 *))
                     z-%offset%)
        (+ dnm1 (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)))
(cond
 ((=
   (f2cl-lib:fref z ((f2cl-lib:int-add j4 (f2cl-lib:int-sub 2)))
                  ((1 *)))
   zero)
  (setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%) zero)
  (setf dn
          (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2)) ((1 *))
                         z-%offset%))
  (setf dmin dn) (setf emin zero))
 ((and
   (< (* safmin (f2cl-lib:fref z ((f2cl-lib:int-add j4p2 2)) ((1 *))))
      (f2cl-lib:fref z ((f2cl-lib:int-add j4 (f2cl-lib:int-sub 2)))
                     ((1 *))))
   (<
    (* safmin
       (f2cl-lib:fref z ((f2cl-lib:int-add j4 (f2cl-lib:int-sub 2)))
                      ((1 *))))
    (f2cl-lib:fref z ((f2cl-lib:int-add j4p2 2)) ((1 *)))))
  (setf temp
          (/
           (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2)) ((1 *))
                          z-%offset%)
           (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2)) ((1 *))
                          z-%offset%)))
  (setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
          (* (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%) temp))
  (setf dn (* dnm1 temp)))
 (t
  (setf (f2cl-lib:fref z-%data% (j4) ((1 *)) z-%offset%)
          (*
           (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2)) ((1 *))
                          z-%offset%)
           (/ (f2cl-lib:fref z-%data% (j4p2) ((1 *)) z-%offset%)
              (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2)) ((1 *))
                             z-%offset%))))
  (setf dn
          (*
           (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4p2 2)) ((1 *))
                          z-%offset%)
           (/ dnm1
              (f2cl-lib:fref z-%data% ((f2cl-lib:int-sub j4 2)) ((1 *))
                             z-%offset%))))))
```

```
        (setf dmin (min dmin dn))
        (setf (f2cl-lib:fref z-%data% ((f2cl-lib:int-add j4 2)) ((1 *))
                             z-%offset%)
                dn)
        (setf (f2cl-lib:fref z-%data%
                             ((f2cl-lib:int-sub (f2cl-lib:int-mul 4 n0) pp))
                             ((1 *)) z-%offset%)
                emin)
        (go end_label)
       end_label
        (return (values nil nil nil nil dmin dmin1 dmin2 dn dnm1 dnm2))))))
```

———-

90.188 dlasr

90.188.1 Axiom unit tests

— **dlasr.input** —

```
)set break resume
)sys rm -f dlasr.output
)spool dlasr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.188.2 Axiom help page

— **dlasr.help** —

```
======================================================================
dlasr examples
======================================================================

======================================================================
Man Page Details
======================================================================
```

```
NAME
       DLASR - a sequence of plane rotations to a real matrix A,

SYNOPSIS
       SUBROUTINE DLASR( SIDE, PIVOT, DIRECT, M, N, C, S, A, LDA )

           CHARACTER     DIRECT, PIVOT, SIDE

           INTEGER       LDA, M, N

           DOUBLE        PRECISION A( LDA, * ), C( * ), S( * )

  Purpose
  =======

  DLASR   performs the transformation

     A := P*A,   when SIDE = 'L' or 'l'  (  Left-hand side )

     A := A*P',  when SIDE = 'R' or 'r'  ( Right-hand side )

  where A is an m by n real matrix and P is an orthogonal matrix,
  consisting of a sequence of plane rotations determined by the
  parameters PIVOT and DIRECT as follows ( z = m when SIDE = 'L' or 'l'
  and z = n when SIDE = 'R' or 'r' ):

  When  DIRECT = 'F' or 'f'  ( Forward sequence ) then

     P = P( z - 1 )*...*P( 2 )*P( 1 ),

  and when DIRECT = 'B' or 'b'  ( Backward sequence ) then

     P = P( 1 )*P( 2 )*...*P( z - 1 ),

  where  P( k ) is a plane rotation matrix for the following planes:

     when  PIVOT = 'V' or 'v'  ( Variable pivot ),
        the plane ( k, k + 1 )

     when  PIVOT = 'T' or 't'  ( Top pivot ),
        the plane ( 1, k + 1 )

     when  PIVOT = 'B' or 'b'  ( Bottom pivot ),
        the plane ( k, z )

  c( k ) and s( k )  must contain the  cosine and sine that define the
  matrix  P( k ).  The two by two plane rotation part of the matrix
  P( k ), R( k ), is assumed to be of the form
```

```
   R( k ) = (  c( k )  s( k ) ).
            ( -s( k )  c( k ) )

This version vectorises across rows of the array A when SIDE = 'L'.

Arguments
=========

SIDE    (input) CHARACTER*1
        Specifies whether the plane rotation matrix P is applied to
        A on the left or the right.
        = 'L':  Left, compute A := P*A
        = 'R':  Right, compute A:= A*P'

DIRECT  (input) CHARACTER*1
        Specifies whether P is a forward or backward sequence of
        plane rotations.
        = 'F':  Forward, P = P( z - 1 )*...*P( 2 )*P( 1 )
        = 'B':  Backward, P = P( 1 )*P( 2 )*...*P( z - 1 )

PIVOT   (input) CHARACTER*1
        Specifies the plane for which P(k) is a plane rotation
        matrix.
        = 'V':  Variable pivot, the plane (k,k+1)
        = 'T':  Top pivot, the plane (1,k+1)
        = 'B':  Bottom pivot, the plane (k,z)

M       (input) INTEGER
        The number of rows of the matrix A.  If m <= 1, an immediate
        return is effected.

N       (input) INTEGER
        The number of columns of the matrix A.  If n <= 1, an
        immediate return is effected.

C, S    (input) DOUBLE PRECISION arrays, dimension
                (M-1) if SIDE = 'L'
                (N-1) if SIDE = 'R'
        c(k) and s(k) contain the cosine and sine that define the
        matrix P(k).  The two by two plane rotation part of the
        matrix P(k), R(k), is assumed to be of the form
        R( k ) = (  c( k )  s( k ) ).
                 ( -s( k )  c( k ) )

A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
        The m by n matrix A.  On exit, A is overwritten by P*A if
        SIDE = 'R' or by A*P' if SIDE = 'L'.

LDA     (input) INTEGER
        The leading dimension of the array A.  LDA >= max(1,M).
```

———-

90.188.3 fortran code

— **dlasr.f** —

```
      SUBROUTINE DLASR( SIDE, PIVOT, DIRECT, M, N, C, S, A, LDA )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1992
*
*     .. Scalar Arguments ..
      CHARACTER          DIRECT, PIVOT, SIDE
      INTEGER            LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), C( * ), S( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, INFO, J
      DOUBLE PRECISION   CTEMP, STEMP, TEMP
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
```

```
      IF( .NOT.( LSAME( SIDE, 'L' ) .OR. LSAME( SIDE, 'R' ) ) ) THEN
         INFO = 1
      ELSE IF( .NOT.( LSAME( PIVOT, 'V' ) .OR. LSAME( PIVOT,
     $         'T' ) .OR. LSAME( PIVOT, 'B' ) ) ) THEN
         INFO = 2
      ELSE IF( .NOT.( LSAME( DIRECT, 'F' ) .OR. LSAME( DIRECT, 'B' ) ) )
     $          THEN
         INFO = 3
      ELSE IF( M.LT.0 ) THEN
         INFO = 4
      ELSE IF( N.LT.0 ) THEN
         INFO = 5
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = 9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASR ', INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( ( M.EQ.0 ) .OR. ( N.EQ.0 ) )
     $   RETURN
      IF( LSAME( SIDE, 'L' ) ) THEN
*
*        Form  P * A
*
         IF( LSAME( PIVOT, 'V' ) ) THEN
            IF( LSAME( DIRECT, 'F' ) ) THEN
               DO 20 J = 1, M - 1
                  CTEMP = C( J )
                  STEMP = S( J )
                  IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                     DO 10 I = 1, N
                        TEMP = A( J+1, I )
                        A( J+1, I ) = CTEMP*TEMP - STEMP*A( J, I )
                        A( J, I ) = STEMP*TEMP + CTEMP*A( J, I )
   10                CONTINUE
                  END IF
   20          CONTINUE
            ELSE IF( LSAME( DIRECT, 'B' ) ) THEN
               DO 40 J = M - 1, 1, -1
                  CTEMP = C( J )
                  STEMP = S( J )
                  IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                     DO 30 I = 1, N
                        TEMP = A( J+1, I )
                        A( J+1, I ) = CTEMP*TEMP - STEMP*A( J, I )
                        A( J, I ) = STEMP*TEMP + CTEMP*A( J, I )
```

```
 30               CONTINUE
                END IF
 40          CONTINUE
          END IF
       ELSE IF( LSAME( PIVOT, 'T' ) ) THEN
          IF( LSAME( DIRECT, 'F' ) ) THEN
             DO 60 J = 2, M
                CTEMP = C( J-1 )
                STEMP = S( J-1 )
                IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                   DO 50 I = 1, N
                      TEMP = A( J, I )
                      A( J, I ) = CTEMP*TEMP - STEMP*A( 1, I )
                      A( 1, I ) = STEMP*TEMP + CTEMP*A( 1, I )
 50                CONTINUE
                END IF
 60          CONTINUE
          ELSE IF( LSAME( DIRECT, 'B' ) ) THEN
             DO 80 J = M, 2, -1
                CTEMP = C( J-1 )
                STEMP = S( J-1 )
                IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                   DO 70 I = 1, N
                      TEMP = A( J, I )
                      A( J, I ) = CTEMP*TEMP - STEMP*A( 1, I )
                      A( 1, I ) = STEMP*TEMP + CTEMP*A( 1, I )
 70                CONTINUE
                END IF
 80          CONTINUE
          END IF
       ELSE IF( LSAME( PIVOT, 'B' ) ) THEN
          IF( LSAME( DIRECT, 'F' ) ) THEN
             DO 100 J = 1, M - 1
                CTEMP = C( J )
                STEMP = S( J )
                IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                   DO 90 I = 1, N
                      TEMP = A( J, I )
                      A( J, I ) = STEMP*A( M, I ) + CTEMP*TEMP
                      A( M, I ) = CTEMP*A( M, I ) - STEMP*TEMP
 90                CONTINUE
                END IF
100          CONTINUE
          ELSE IF( LSAME( DIRECT, 'B' ) ) THEN
             DO 120 J = M - 1, 1, -1
                CTEMP = C( J )
                STEMP = S( J )
                IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                   DO 110 I = 1, N
                      TEMP = A( J, I )
```

```
                     A( J, I ) = STEMP*A( M, I ) + CTEMP*TEMP
                     A( M, I ) = CTEMP*A( M, I ) - STEMP*TEMP
  110             CONTINUE
                END IF
  120         CONTINUE
           END IF
        END IF
     ELSE IF( LSAME( SIDE, 'R' ) ) THEN
*
*       Form A * P'
*
        IF( LSAME( PIVOT, 'V' ) ) THEN
           IF( LSAME( DIRECT, 'F' ) ) THEN
              DO 140 J = 1, N - 1
                 CTEMP = C( J )
                 STEMP = S( J )
                 IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                    DO 130 I = 1, M
                       TEMP = A( I, J+1 )
                       A( I, J+1 ) = CTEMP*TEMP - STEMP*A( I, J )
                       A( I, J ) = STEMP*TEMP + CTEMP*A( I, J )
  130               CONTINUE
                 END IF
  140         CONTINUE
           ELSE IF( LSAME( DIRECT, 'B' ) ) THEN
              DO 160 J = N - 1, 1, -1
                 CTEMP = C( J )
                 STEMP = S( J )
                 IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                    DO 150 I = 1, M
                       TEMP = A( I, J+1 )
                       A( I, J+1 ) = CTEMP*TEMP - STEMP*A( I, J )
                       A( I, J ) = STEMP*TEMP + CTEMP*A( I, J )
  150               CONTINUE
                 END IF
  160         CONTINUE
           END IF
        ELSE IF( LSAME( PIVOT, 'T' ) ) THEN
           IF( LSAME( DIRECT, 'F' ) ) THEN
              DO 180 J = 2, N
                 CTEMP = C( J-1 )
                 STEMP = S( J-1 )
                 IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                    DO 170 I = 1, M
                       TEMP = A( I, J )
                       A( I, J ) = CTEMP*TEMP - STEMP*A( I, 1 )
                       A( I, 1 ) = STEMP*TEMP + CTEMP*A( I, 1 )
  170               CONTINUE
                 END IF
  180         CONTINUE
```

```
         ELSE IF( LSAME( DIRECT, 'B' ) ) THEN
            DO 200 J = N, 2, -1
               CTEMP = C( J-1 )
               STEMP = S( J-1 )
               IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                  DO 190 I = 1, M
                     TEMP = A( I, J )
                     A( I, J ) = CTEMP*TEMP - STEMP*A( I, 1 )
                     A( I, 1 ) = STEMP*TEMP + CTEMP*A( I, 1 )
 190              CONTINUE
               END IF
 200        CONTINUE
         END IF
      ELSE IF( LSAME( PIVOT, 'B' ) ) THEN
         IF( LSAME( DIRECT, 'F' ) ) THEN
            DO 220 J = 1, N - 1
               CTEMP = C( J )
               STEMP = S( J )
               IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                  DO 210 I = 1, M
                     TEMP = A( I, J )
                     A( I, J ) = STEMP*A( I, N ) + CTEMP*TEMP
                     A( I, N ) = CTEMP*A( I, N ) - STEMP*TEMP
 210              CONTINUE
               END IF
 220        CONTINUE
         ELSE IF( LSAME( DIRECT, 'B' ) ) THEN
            DO 240 J = N - 1, 1, -1
               CTEMP = C( J )
               STEMP = S( J )
               IF( ( CTEMP.NE.ONE ) .OR. ( STEMP.NE.ZERO ) ) THEN
                  DO 230 I = 1, M
                     TEMP = A( I, J )
                     A( I, J ) = STEMP*A( I, N ) + CTEMP*TEMP
                     A( I, N ) = CTEMP*A( I, N ) - STEMP*TEMP
 230              CONTINUE
               END IF
 240        CONTINUE
         END IF
      END IF
   END IF
*
   RETURN
*
*  End of DLASR
*
   END
```

———-

90.188.4 lisp code

— dlasr.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dlasr (side pivot direct m n c s a lda)
    (declare (type (array double-float (*)) a s c)
             (type (f2cl-lib:integer4) lda n m)
             (type (string *) direct pivot side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (pivot character pivot-%data% pivot-%offset%)
         (direct character direct-%data% direct-%offset%)
         (c double-float c-%data% c-%offset%)
         (s double-float s-%data% s-%offset%)
         (a double-float a-%data% a-%offset%))
      (prog ((ctemp 0.0d0) (stemp 0.0d0) (temp 0.0d0) (i 0) (info 0) (j 0))
        (declare (type (double-float) ctemp stemp temp)
                 (type (f2cl-lib:integer4) i info j))
        (setf info 0)
        (cond
         ((not
           (or
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame side "L")
              (declare (ignore var-1))
              (when var-0 (setf side var-0))
              ret-val)
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame side "R")
              (declare (ignore var-1))
              (when var-0 (setf side var-0))
              ret-val)))
          (setf info 1))
         ((not
           (or
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame pivot "V")
              (declare (ignore var-1))
              (when var-0 (setf pivot var-0))
              ret-val)
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame pivot "T")
              (declare (ignore var-1))
              (when var-0 (setf pivot var-0))
              ret-val)
```

```
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame pivot "B")
      (declare (ignore var-1))
      (when var-0 (setf pivot var-0))
      ret-val)))
  (setf info 2))
 ((not
   (or
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame direct "F")
      (declare (ignore var-1))
      (when var-0 (setf direct var-0))
      ret-val)
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame direct "B")
      (declare (ignore var-1))
      (when var-0 (setf direct var-0))
      ret-val)))
  (setf info 3))
 ((< m 0) (setf info 4)) ((< n 0) (setf info 5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info 9)))
(cond
 ((/= info 0)
  (multiple-value-bind (var-0 var-1)
      (xerbla "DLASR " info)
    (declare (ignore var-0))
    (when var-1 (setf info var-1)))
  (go end_label)))
(if (or (= m 0) (= n 0))
    (go end_label))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame side "L")
    (declare (ignore var-1))
    (when var-0 (setf side var-0))
    ret-val)
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame pivot "V")
      (declare (ignore var-1))
      (when var-0 (setf pivot var-0))
      ret-val)
    (cond
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame direct "F")
        (declare (ignore var-1))
        (when var-0 (setf direct var-0))
        ret-val)
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                ((> j (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (setf ctemp (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
      (setf stemp (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
      (cond
       ((or (/= ctemp one) (/= stemp zero))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf temp
                    (f2cl-lib:fref a-%data%
                                   ((f2cl-lib:int-add j 1) i)
                                   ((1 lda) (1 *)) a-%offset%))
            (setf (f2cl-lib:fref a-%data%
                                 ((f2cl-lib:int-add j 1) i)
                                 ((1 lda) (1 *)) a-%offset%)
                    (- (* ctemp temp)
                       (* stemp
                          (f2cl-lib:fref a-%data% (j i)
                                         ((1 lda) (1 *))
                                         a-%offset%))))
            (setf (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                 a-%offset%)
                    (+ (* stemp temp)
                       (* ctemp
                          (f2cl-lib:fref a-%data% (j i)
                                         ((1 lda) (1 *))
                                         a-%offset%))))
           label10))))
     label20)))
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame direct "B")
    (declare (ignore var-1))
    (when var-0 (setf direct var-0))
    ret-val)
   (f2cl-lib:fdo (j (f2cl-lib:int-add m (f2cl-lib:int-sub 1))
                  (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                 ((> j 1) nil)
    (tagbody
      (setf ctemp (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
      (setf stemp (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
      (cond
       ((or (/= ctemp one) (/= stemp zero))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf temp
                    (f2cl-lib:fref a-%data%
                                   ((f2cl-lib:int-add j 1) i)
```

```
                                  ((1 lda) (1 *)) a-%offset%))
                (setf (f2cl-lib:fref a-%data%
                                     ((f2cl-lib:int-add j 1) i)
                                     ((1 lda) (1 *)) a-%offset%)
                        (- (* ctemp temp)
                           (* stemp
                              (f2cl-lib:fref a-%data% (j i)
                                             ((1 lda) (1 *))
                                             a-%offset%))))
                (setf (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                     a-%offset%)
                        (+ (* stemp temp)
                           (* ctemp
                              (f2cl-lib:fref a-%data% (j i)
                                             ((1 lda) (1 *))
                                             a-%offset%))))
              label30))))
       label40)))))
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame pivot "T")
     (declare (ignore var-1))
     (when var-0 (setf pivot var-0))
     ret-val)
   (cond
    ((multiple-value-bind (ret-val var-0 var-1)
         (lsame direct "F")
       (declare (ignore var-1))
       (when var-0 (setf direct var-0))
       ret-val)
     (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                   ((> j m) nil)
       (tagbody
         (setf ctemp
                 (f2cl-lib:fref c-%data% ((f2cl-lib:int-sub j 1))
                                ((1 *)) c-%offset%))
         (setf stemp
                 (f2cl-lib:fref s-%data% ((f2cl-lib:int-sub j 1))
                                ((1 *)) s-%offset%))
         (cond
          ((or (/= ctemp one) (/= stemp zero))
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i n) nil)
             (tagbody
               (setf temp
                       (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                      a-%offset%))
               (setf (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                     a-%offset%)
                       (- (* ctemp temp)
                          (* stemp
```

```
                                (f2cl-lib:fref a-%data% (1 i)
                                               ((1 lda) (1 *))
                                               a-%offset%))))
                        (setf (f2cl-lib:fref a-%data% (1 i) ((1 lda) (1 *))
                                             a-%offset%)
                              (+ (* stemp temp)
                                 (* ctemp
                                    (f2cl-lib:fref a-%data% (1 i)
                                               ((1 lda) (1 *))
                                               a-%offset%))))
                      label50))))
                 label60)))
             ((multiple-value-bind (ret-val var-0 var-1)
                  (lsame direct "B")
                (declare (ignore var-1))
                (when var-0 (setf direct var-0))
                ret-val)
              (f2cl-lib:fdo (j m (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                            ((> j 2) nil)
                (tagbody
                  (setf ctemp
                          (f2cl-lib:fref c-%data% ((f2cl-lib:int-sub j 1))
                                         ((1 *)) c-%offset%))
                  (setf stemp
                          (f2cl-lib:fref s-%data% ((f2cl-lib:int-sub j 1))
                                         ((1 *)) s-%offset%))
                  (cond
                   ((or (/= ctemp one) (/= stemp zero))
                    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                  ((> i n) nil)
                      (tagbody
                        (setf temp
                                (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                               a-%offset%))
                        (setf (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                             a-%offset%)
                              (- (* ctemp temp)
                                 (* stemp
                                    (f2cl-lib:fref a-%data% (1 i)
                                               ((1 lda) (1 *))
                                               a-%offset%))))
                        (setf (f2cl-lib:fref a-%data% (1 i) ((1 lda) (1 *))
                                             a-%offset%)
                              (+ (* stemp temp)
                                 (* ctemp
                                    (f2cl-lib:fref a-%data% (1 i)
                                               ((1 lda) (1 *))
                                               a-%offset%))))
                      label70))))
                 label80)))))
```

```
((multiple-value-bind (ret-val var-0 var-1)
     (lsame pivot "B")
   (declare (ignore var-1))
   (when var-0 (setf pivot var-0))
   ret-val)
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame direct "F")
     (declare (ignore var-1))
     (when var-0 (setf direct var-0))
     ret-val)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
                  nil)
     (tagbody
       (setf ctemp (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
       (setf stemp (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
       (cond
        ((or (/= ctemp one) (/= stemp zero))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i n) nil)
           (tagbody
             (setf temp
                     (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                    a-%offset%))
             (setf (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                  a-%offset%)
                     (+
                      (* stemp
                         (f2cl-lib:fref a-%data% (m i)
                                        ((1 lda) (1 *)) a-%offset%))
                      (* ctemp temp)))
             (setf (f2cl-lib:fref a-%data% (m i) ((1 lda) (1 *))
                                  a-%offset%)
                     (-
                      (* ctemp
                         (f2cl-lib:fref a-%data% (m i)
                                        ((1 lda) (1 *)) a-%offset%))
                      (* stemp temp)))
            label90))))
     label100)))
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame direct "B")
     (declare (ignore var-1))
     (when var-0 (setf direct var-0))
     ret-val)
   (f2cl-lib:fdo (j (f2cl-lib:int-add m (f2cl-lib:int-sub 1))
                  (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                 ((> j 1) nil)
     (tagbody
```

```
            (setf ctemp (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
            (setf stemp (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
            (cond
             ((or (/= ctemp one) (/= stemp zero))
              (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                            ((> i n) nil)
                (tagbody
                  (setf temp
                          (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                         a-%offset%))
                  (setf (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                                       a-%offset%)
                          (+
                           (* stemp
                              (f2cl-lib:fref a-%data% (m i)
                                             ((1 lda) (1 *)) a-%offset%))
                           (* ctemp temp)))
                  (setf (f2cl-lib:fref a-%data% (m i) ((1 lda) (1 *))
                                       a-%offset%)
                          (-
                           (* ctemp
                              (f2cl-lib:fref a-%data% (m i)
                                             ((1 lda) (1 *)) a-%offset%))
                           (* stemp temp)))
                label110))))
        label120)))))))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame side "R")
   (declare (ignore var-1))
   (when var-0 (setf side var-0))
   ret-val)
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame pivot "V")
     (declare (ignore var-1))
     (when var-0 (setf pivot var-0))
     ret-val)
   (cond
    ((multiple-value-bind (ret-val var-0 var-1)
         (lsame direct "F")
       (declare (ignore var-1))
       (when var-0 (setf direct var-0))
       ret-val)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                    nil)
       (tagbody
         (setf ctemp (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
         (setf stemp (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
         (cond
```

```
        ((or (/= ctemp one) (/= stemp zero))
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf temp
                     (f2cl-lib:fref a-%data%
                                    (i (f2cl-lib:int-add j 1))
                                    ((1 lda) (1 *)) a-%offset%))
             (setf (f2cl-lib:fref a-%data%
                                  (i (f2cl-lib:int-add j 1))
                                  ((1 lda) (1 *)) a-%offset%)
                     (- (* ctemp temp)
                        (* stemp
                           (f2cl-lib:fref a-%data% (i j)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
             (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                     (+ (* stemp temp)
                        (* ctemp
                           (f2cl-lib:fref a-%data% (i j)
                                          ((1 lda) (1 *))
                                          a-%offset%))))
          label130))))
    label140)))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame direct "B")
   (declare (ignore var-1))
   (when var-0 (setf direct var-0))
   ret-val)
 (f2cl-lib:fdo (j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (setf ctemp (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
     (setf stemp (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
     (cond
      ((or (/= ctemp one) (/= stemp zero))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf temp
                   (f2cl-lib:fref a-%data%
                                  (i (f2cl-lib:int-add j 1))
                                  ((1 lda) (1 *)) a-%offset%))
           (setf (f2cl-lib:fref a-%data%
                                (i (f2cl-lib:int-add j 1))
                                ((1 lda) (1 *)) a-%offset%)
                   (- (* ctemp temp)
                      (* stemp
```

```
                                (f2cl-lib:fref a-%data% (i j)
                                               ((1 lda) (1 *))
                                               a-%offset%))))
                    (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                         a-%offset%)
                          (+ (* stemp temp)
                             (* ctemp
                                (f2cl-lib:fref a-%data% (i j)
                                               ((1 lda) (1 *))
                                               a-%offset%))))
                  label150))))
            label160)))))
      ((multiple-value-bind (ret-val var-0 var-1)
           (lsame pivot "T")
         (declare (ignore var-1))
         (when var-0 (setf pivot var-0))
         ret-val)
       (cond
        ((multiple-value-bind (ret-val var-0 var-1)
             (lsame direct "F")
           (declare (ignore var-1))
           (when var-0 (setf direct var-0))
           ret-val)
         (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (setf ctemp
                     (f2cl-lib:fref c-%data% ((f2cl-lib:int-sub j 1))
                                    ((1 *)) c-%offset%))
             (setf stemp
                     (f2cl-lib:fref s-%data% ((f2cl-lib:int-sub j 1))
                                    ((1 *)) s-%offset%))
             (cond
              ((or (/= ctemp one) (/= stemp zero))
               (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                             ((> i m) nil)
                 (tagbody
                   (setf temp
                           (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                          a-%offset%))
                   (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                        a-%offset%)
                           (- (* ctemp temp)
                              (* stemp
                                 (f2cl-lib:fref a-%data% (i 1)
                                                ((1 lda) (1 *))
                                                a-%offset%))))
                   (setf (f2cl-lib:fref a-%data% (i 1) ((1 lda) (1 *))
                                        a-%offset%)
                           (+ (* stemp temp)
```

```
                          (* ctemp
                             (f2cl-lib:fref a-%data% (i 1)
                                            ((1 lda) (1 *))
                                            a-%offset%))))
              label170))))
         label180)))
     ((multiple-value-bind (ret-val var-0 var-1)
          (lsame direct "B")
        (declare (ignore var-1))
        (when var-0 (setf direct var-0))
        ret-val)
      (f2cl-lib:fdo (j n (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                    ((> j 2) nil)
        (tagbody
          (setf ctemp
                  (f2cl-lib:fref c-%data% ((f2cl-lib:int-sub j 1))
                                 ((1 *)) c-%offset%))
          (setf stemp
                  (f2cl-lib:fref s-%data% ((f2cl-lib:int-sub j 1))
                                 ((1 *)) s-%offset%))
          (cond
           ((or (/= ctemp one) (/= stemp zero))
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i m) nil)
              (tagbody
                (setf temp
                        (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%))
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%)
                        (- (* ctemp temp)
                           (* stemp
                              (f2cl-lib:fref a-%data% (i 1)
                                             ((1 lda) (1 *))
                                             a-%offset%))))
                (setf (f2cl-lib:fref a-%data% (i 1) ((1 lda) (1 *))
                                     a-%offset%)
                        (+ (* stemp temp)
                           (* ctemp
                              (f2cl-lib:fref a-%data% (i 1)
                                             ((1 lda) (1 *))
                                             a-%offset%))))
              label190))))
         label200)))))
    ((multiple-value-bind (ret-val var-0 var-1)
         (lsame pivot "B")
       (declare (ignore var-1))
       (when var-0 (setf pivot var-0))
       ret-val)
     (cond
```

```
((multiple-value-bind (ret-val var-0 var-1)
     (lsame direct "F")
   (declare (ignore var-1))
   (when var-0 (setf direct var-0))
   ret-val)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (setf ctemp (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
     (setf stemp (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
     (cond
      ((or (/= ctemp one) (/= stemp zero))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
         (tagbody
           (setf temp
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%))
           (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                   (+
                    (* stemp
                       (f2cl-lib:fref a-%data% (i n)
                                      ((1 lda) (1 *)) a-%offset%))
                    (* ctemp temp)))
           (setf (f2cl-lib:fref a-%data% (i n) ((1 lda) (1 *))
                                a-%offset%)
                   (-
                    (* ctemp
                       (f2cl-lib:fref a-%data% (i n)
                                      ((1 lda) (1 *)) a-%offset%))
                    (* stemp temp)))
          label210))))
    label220)))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame direct "B")
   (declare (ignore var-1))
   (when var-0 (setf direct var-0))
   ret-val)
 (f2cl-lib:fdo (j (f2cl-lib:int-add n (f2cl-lib:int-sub 1))
                (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (setf ctemp (f2cl-lib:fref c-%data% (j) ((1 *)) c-%offset%))
     (setf stemp (f2cl-lib:fref s-%data% (j) ((1 *)) s-%offset%))
     (cond
      ((or (/= ctemp one) (/= stemp zero))
       (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                     ((> i m) nil)
```

```
                    (tagbody
                      (setf temp
                              (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                             a-%offset%))
                      (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                           a-%offset%)
                              (+
                               (* stemp
                                  (f2cl-lib:fref a-%data% (i n)
                                                 ((1 lda) (1 *)) a-%offset%))
                               (* ctemp temp)))
                      (setf (f2cl-lib:fref a-%data% (i n) ((1 lda) (1 *))
                                           a-%offset%)
                              (-
                               (* ctemp
                                  (f2cl-lib:fref a-%data% (i n)
                                                 ((1 lda) (1 *)) a-%offset%))
                               (* stemp temp)))
                     label230))))
             label240))))))))
      (go end_label)
     end_label
      (return (values side pivot direct nil nil nil nil nil nil))))))
```

———-

90.189 dlasrt

90.189.1 Axiom unit tests

— **dlasrt.input** —

```
)set break resume
)sys rm -f dlasrt.output
)spool dlasrt.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.189.2 Axiom help page

— dlasrt.help —

```
==================================================================
dlasrt examples
==================================================================

==================================================================
Man Page Details
==================================================================

NAME
       DLASRT - number in D in increasing order (if ID = 'I') or in decreasing
       order (if ID = 'D' )

SYNOPSIS
       SUBROUTINE DLASRT( ID, N, D, INFO )

           CHARACTER      ID

           INTEGER        INFO, N

           DOUBLE         PRECISION D( * )

  Purpose
  =======

  Sort the numbers in D in increasing order (if ID = 'I') or
  in decreasing order (if ID = 'D' ).

  Use Quick Sort, reverting to Insertion sort on arrays of
  size <= 20. Dimension of STACK limits N to about 2**32.

  Arguments
  =========

  ID      (input) CHARACTER*1
          = 'I': sort D in increasing order;
          = 'D': sort D in decreasing order.

  N       (input) INTEGER
          The length of the array D.

  D       (input/output) DOUBLE PRECISION array, dimension (N)
          On entry, the array to be sorted.
          On exit, D has been sorted into increasing order
          (D(1) <= ... <= D(N) ) or into decreasing order
          (D(1) >= ... >= D(N) ), depending on ID.
```

```
 INFO    (output) INTEGER
         = 0:  successful exit
         < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

90.189.3 fortran code

— **dlasrt.f** —

```
      SUBROUTINE DLASRT( ID, N, D, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     September 30, 1994
*
*     .. Scalar Arguments ..
      CHARACTER          ID
      INTEGER            INFO, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   D( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            SELECT
      PARAMETER          ( SELECT = 20 )
*     ..
*     .. Local Scalars ..
      INTEGER            DIR, ENDD, I, J, START, STKPNT
      DOUBLE PRECISION   D1, D2, D3, DMNMX, TMP
*     ..
*     .. Local Arrays ..
      INTEGER            STACK( 2, 32 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Executable Statements ..
```

```
*
*     Test the input paramters.
*
      INFO = 0
      DIR = -1
      IF( LSAME( ID, 'D' ) ) THEN
         DIR = 0
      ELSE IF( LSAME( ID, 'I' ) ) THEN
         DIR = 1
      END IF
      IF( DIR.EQ.-1 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DLASRT', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.LE.1 )
     $   RETURN
*
      STKPNT = 1
      STACK( 1, 1 ) = 1
      STACK( 2, 1 ) = N
   10 CONTINUE
      START = STACK( 1, STKPNT )
      ENDD = STACK( 2, STKPNT )
      STKPNT = STKPNT - 1
      IF( ENDD-START.LE.SELECT .AND. ENDD-START.GT.0 ) THEN
*
*        Do Insertion sort on D( START:ENDD )
*
         IF( DIR.EQ.0 ) THEN
*
*           Sort into decreasing order
*
            DO 30 I = START + 1, ENDD
               DO 20 J = I, START + 1, -1
                  IF( D( J ).GT.D( J-1 ) ) THEN
                     DMNMX = D( J )
                     D( J ) = D( J-1 )
                     D( J-1 ) = DMNMX
                  ELSE
                     GO TO 30
                  END IF
   20          CONTINUE
```

```
   30       CONTINUE
*
         ELSE
*
*           Sort into increasing order
*
            DO 50 I = START + 1, ENDD
               DO 40 J = I, START + 1, -1
                  IF( D( J ).LT.D( J-1 ) ) THEN
                     DMNMX = D( J )
                     D( J ) = D( J-1 )
                     D( J-1 ) = DMNMX
                  ELSE
                     GO TO 50
                  END IF
   40          CONTINUE
   50       CONTINUE
*
         END IF
*
      ELSE IF( ENDD-START.GT.SELECT ) THEN
*
*        Partition D( START:ENDD ) and stack parts, largest one first
*
*        Choose partition entry as median of 3
*
         D1 = D( START )
         D2 = D( ENDD )
         I = ( START+ENDD ) / 2
         D3 = D( I )
         IF( D1.LT.D2 ) THEN
            IF( D3.LT.D1 ) THEN
               DMNMX = D1
            ELSE IF( D3.LT.D2 ) THEN
               DMNMX = D3
            ELSE
               DMNMX = D2
            END IF
         ELSE
            IF( D3.LT.D2 ) THEN
               DMNMX = D2
            ELSE IF( D3.LT.D1 ) THEN
               DMNMX = D3
            ELSE
               DMNMX = D1
            END IF
         END IF
*
         IF( DIR.EQ.0 ) THEN
*
```

```
*          Sort into decreasing order
*
           I = START - 1
           J = ENDD + 1
   60      CONTINUE
   70      CONTINUE
           J = J - 1
           IF( D( J ).LT.DMNMX )
     $        GO TO 70
   80      CONTINUE
           I = I + 1
           IF( D( I ).GT.DMNMX )
     $        GO TO 80
           IF( I.LT.J ) THEN
              TMP = D( I )
              D( I ) = D( J )
              D( J ) = TMP
              GO TO 60
           END IF
           IF( J-START.GT.ENDD-J-1 ) THEN
              STKPNT = STKPNT + 1
              STACK( 1, STKPNT ) = START
              STACK( 2, STKPNT ) = J
              STKPNT = STKPNT + 1
              STACK( 1, STKPNT ) = J + 1
              STACK( 2, STKPNT ) = ENDD
           ELSE
              STKPNT = STKPNT + 1
              STACK( 1, STKPNT ) = J + 1
              STACK( 2, STKPNT ) = ENDD
              STKPNT = STKPNT + 1
              STACK( 1, STKPNT ) = START
              STACK( 2, STKPNT ) = J
           END IF
        ELSE
*
*          Sort into increasing order
*
           I = START - 1
           J = ENDD + 1
   90      CONTINUE
  100      CONTINUE
           J = J - 1
           IF( D( J ).GT.DMNMX )
     $        GO TO 100
  110      CONTINUE
           I = I + 1
           IF( D( I ).LT.DMNMX )
     $        GO TO 110
           IF( I.LT.J ) THEN
```

```
               TMP = D( I )
               D( I ) = D( J )
               D( J ) = TMP
               GO TO 90
            END IF
            IF( J-START.GT.ENDD-J-1 ) THEN
               STKPNT = STKPNT + 1
               STACK( 1, STKPNT ) = START
               STACK( 2, STKPNT ) = J
               STKPNT = STKPNT + 1
               STACK( 1, STKPNT ) = J + 1
               STACK( 2, STKPNT ) = ENDD
            ELSE
               STKPNT = STKPNT + 1
               STACK( 1, STKPNT ) = J + 1
               STACK( 2, STKPNT ) = ENDD
               STKPNT = STKPNT + 1
               STACK( 1, STKPNT ) = START
               STACK( 2, STKPNT ) = J
            END IF
         END IF
      END IF
      IF( STKPNT.GT.0 )
     $   GO TO 10
      RETURN
*
*     End of DLASRT
*
      END
```

———-

90.189.4 lisp code

— dlasrt.lisp —

```
(let* ((select 20))
  (declare (type (f2cl-lib:integer4 20 20) select)
           (ignorable select))
  (defun dlasrt (id n d info)
    (declare (type (array double-float (*)) d)
             (type (f2cl-lib:integer4) info n)
             (type (string *) id))
    (f2cl-lib:with-multi-array-data
        ((id character id-%data% id-%offset%)
         (d double-float d-%data% d-%offset%))
      (prog ((stack (make-array 64 :element-type 'f2cl-lib:integer4))
```

```
     (d1 0.0d0) (d2 0.0d0) (d3 0.0d0) (dmnmx 0.0d0) (tmp 0.0d0) (dir 0)
     (endd 0) (i 0) (j 0) (start 0) (stkpnt 0))
 (declare (type (array f2cl-lib:integer4 (64)) stack)
          (type (double-float) d1 d2 d3 dmnmx tmp)
          (type (f2cl-lib:integer4) dir endd i j start stkpnt))
 (setf info 0)
 (setf dir -1)
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame id "D")
     (declare (ignore var-1))
     (when var-0 (setf id var-0))
     ret-val)
   (setf dir 0))
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame id "I")
     (declare (ignore var-1))
     (when var-0 (setf id var-0))
     ret-val)
   (setf dir 1)))
 (cond ((= dir (f2cl-lib:int-sub 1)) (setf info -1))
       ((< n 0) (setf info -2)))
 (cond
  ((/= info 0) (xerbla "DLASRT" (f2cl-lib:int-sub info))
   (go end_label)))
 (if (<= n 1)
     (go end_label))
 (setf stkpnt 1)
 (setf (f2cl-lib:fref stack (1 1) ((1 2) (1 32))) 1)
 (setf (f2cl-lib:fref stack (2 1) ((1 2) (1 32))) n)
label10
 (setf start (f2cl-lib:fref stack (1 stkpnt) ((1 2) (1 32))))
 (setf endd (f2cl-lib:fref stack (2 stkpnt) ((1 2) (1 32))))
 (setf stkpnt (f2cl-lib:int-sub stkpnt 1))
 (cond
  ((and (<= (f2cl-lib:int-add endd (f2cl-lib:int-sub start)) select)
        (> (f2cl-lib:int-add endd (f2cl-lib:int-sub start)) 0))
   (cond
    ((= dir 0)
     (f2cl-lib:fdo (i (f2cl-lib:int-add start 1) (f2cl-lib:int-add i 1))
                   ((> i endd) nil)
       (tagbody
         (f2cl-lib:fdo (j i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((> j (f2cl-lib:int-add start 1)) nil)
           (tagbody
             (cond
              ((> (f2cl-lib:fref d (j) ((1 *)))
                  (f2cl-lib:fref d
                                 ((f2cl-lib:int-add j
                                                    (f2cl-lib:int-sub
```

```
                                                  1)))
                                  ((1 *))))
                (setf dmnmx
                        (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%))
                (setf (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                        (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub j 1))
                                        ((1 *)) d-%offset%))
                (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub j 1))
                                      ((1 *)) d-%offset%)
                        dmnmx))
               (t (go label30)))
             label20))
         label30)))
    (t
     (f2cl-lib:fdo (i (f2cl-lib:int-add start 1) (f2cl-lib:int-add i 1))
                   ((> i endd) nil)
       (tagbody
         (f2cl-lib:fdo (j i (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                       ((> j (f2cl-lib:int-add start 1)) nil)
           (tagbody
             (cond
              ((< (f2cl-lib:fref d (j) ((1 *)))
                  (f2cl-lib:fref d
                                  ((f2cl-lib:int-add j
                                                     (f2cl-lib:int-sub
                                                      1)))
                                  ((1 *))))
                (setf dmnmx
                        (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%))
                (setf (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%)
                        (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub j 1))
                                        ((1 *)) d-%offset%))
                (setf (f2cl-lib:fref d-%data% ((f2cl-lib:int-sub j 1))
                                      ((1 *)) d-%offset%)
                        dmnmx))
               (t (go label50)))
             label40))
         label50)))))
  ((> (f2cl-lib:int-add endd (f2cl-lib:int-sub start)) select)
   (setf d1 (f2cl-lib:fref d-%data% (start) ((1 *)) d-%offset%))
   (setf d2 (f2cl-lib:fref d-%data% (endd) ((1 *)) d-%offset%))
   (setf i (the f2cl-lib:integer4 (truncate (+ start endd) 2)))
   (setf d3 (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
   (cond
    ((< d1 d2)
     (cond ((< d3 d1) (setf dmnmx d1)) ((< d3 d2) (setf dmnmx d3))
           (t (setf dmnmx d2))))
    (t
     (cond ((< d3 d2) (setf dmnmx d2)) ((< d3 d1) (setf dmnmx d3))
           (t (setf dmnmx d1)))))
```

```
(cond
 ((= dir 0)
  (tagbody
    (setf i (f2cl-lib:int-sub start 1))
    (setf j (f2cl-lib:int-add endd 1))
   label60
   label70
    (setf j (f2cl-lib:int-sub j 1))
    (if (< (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%) dmnmx)
        (go label70))
   label80
    (setf i (f2cl-lib:int-add i 1))
    (if (> (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) dmnmx)
        (go label80))
    (cond
     ((< i j)
      (setf tmp (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
      (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
              (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%))
      (setf (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%) tmp)
      (go label60)))
    (cond
     ((> (f2cl-lib:int-add j (f2cl-lib:int-sub start))
         (f2cl-lib:int-add endd (f2cl-lib:int-sub j)
                           (f2cl-lib:int-sub 1)))
      (setf stkpnt (f2cl-lib:int-add stkpnt 1))
      (setf (f2cl-lib:fref stack (1 stkpnt) ((1 2) (1 32))) start)
      (setf (f2cl-lib:fref stack (2 stkpnt) ((1 2) (1 32))) j)
      (setf stkpnt (f2cl-lib:int-add stkpnt 1))
      (setf (f2cl-lib:fref stack (1 stkpnt) ((1 2) (1 32)))
              (f2cl-lib:int-add j 1))
      (setf (f2cl-lib:fref stack (2 stkpnt) ((1 2) (1 32))) endd))
     (t (setf stkpnt (f2cl-lib:int-add stkpnt 1))
      (setf (f2cl-lib:fref stack (1 stkpnt) ((1 2) (1 32)))
              (f2cl-lib:int-add j 1))
      (setf (f2cl-lib:fref stack (2 stkpnt) ((1 2) (1 32))) endd)
      (setf stkpnt (f2cl-lib:int-add stkpnt 1))
      (setf (f2cl-lib:fref stack (1 stkpnt) ((1 2) (1 32))) start)
      (setf (f2cl-lib:fref stack (2 stkpnt) ((1 2) (1 32))) j)))))
 (t
  (tagbody
    (setf i (f2cl-lib:int-sub start 1))
    (setf j (f2cl-lib:int-add endd 1))
   label90
   label100
    (setf j (f2cl-lib:int-sub j 1))
    (if (> (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%) dmnmx)
        (go label100))
   label110
    (setf i (f2cl-lib:int-add i 1))
```

```
            (if (< (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%) dmnmx)
                (go label110))
            (cond
             ((< i j)
              (setf tmp (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%))
              (setf (f2cl-lib:fref d-%data% (i) ((1 *)) d-%offset%)
                      (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%))
              (setf (f2cl-lib:fref d-%data% (j) ((1 *)) d-%offset%) tmp)
              (go label90)))
            (cond
             ((> (f2cl-lib:int-add j (f2cl-lib:int-sub start))
                 (f2cl-lib:int-add endd (f2cl-lib:int-sub j)
                                   (f2cl-lib:int-sub 1)))
              (setf stkpnt (f2cl-lib:int-add stkpnt 1))
              (setf (f2cl-lib:fref stack (1 stkpnt) ((1 2) (1 32))) start)
              (setf (f2cl-lib:fref stack (2 stkpnt) ((1 2) (1 32))) j)
              (setf stkpnt (f2cl-lib:int-add stkpnt 1))
              (setf (f2cl-lib:fref stack (1 stkpnt) ((1 2) (1 32)))
                      (f2cl-lib:int-add j 1))
              (setf (f2cl-lib:fref stack (2 stkpnt) ((1 2) (1 32))) endd))
             (t (setf stkpnt (f2cl-lib:int-add stkpnt 1))
              (setf (f2cl-lib:fref stack (1 stkpnt) ((1 2) (1 32)))
                      (f2cl-lib:int-add j 1))
              (setf (f2cl-lib:fref stack (2 stkpnt) ((1 2) (1 32))) endd)
              (setf stkpnt (f2cl-lib:int-add stkpnt 1))
              (setf (f2cl-lib:fref stack (1 stkpnt) ((1 2) (1 32))) start)
              (setf (f2cl-lib:fref stack (2 stkpnt) ((1 2) (1 32))) j))))))))
     (if (> stkpnt 0)
         (go label10))
     (go end_label)
    end_label
     (return (values id nil nil info))))))
```

———-

90.190 dlassq

90.190.1 Axiom unit tests

— **dlassq.input** —

```
)set break resume
)sys rm -f dlassq.output
)spool dlassq.output
)set message test on
)set message auto off
```

```
)clear all

)spool
)lisp (bye)
```

———-

90.190.2 Axiom help page

— **dlassq.help** —

```
====================================================================
dlassq examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DLASSQ  -  the  values  scl and smsq such that   ( scl**2 )*smsq = x( 1
       )**2 +...+ x( n )**2 + ( scale**2 )*sumsq,

SYNOPSIS
       SUBROUTINE DLASSQ( N, X, INCX, SCALE, SUMSQ )

           INTEGER        INCX, N

           DOUBLE         PRECISION SCALE, SUMSQ

           DOUBLE         PRECISION X( * )

  Purpose
  =======

  DLASSQ  returns the values  scl  and  smsq  such that

     ( scl**2 )*smsq = x( 1 )**2 +...+ x( n )**2 + ( scale**2 )*sumsq,

  where  x( i ) = X( 1 + ( i - 1 )*INCX ). The value of  sumsq  is
  assumed to be non-negative and  scl  returns the value

     scl = max( scale, abs( x( i ) ) ).

  scale and sumsq must be supplied in SCALE and SUMSQ and
  scl and smsq are overwritten on SCALE and SUMSQ respectively.

  The routine makes only one pass through the vector x.
```

```
Arguments
=========

N       (input) INTEGER
        The number of elements to be used from the vector X.

X       (input) DOUBLE PRECISION array, dimension (N)
        The vector for which a scaled sum of squares is computed.
           x( i )  = X( 1 + ( i - 1 )*INCX ), 1 <= i <= n.

INCX    (input) INTEGER
        The increment between successive values of the vector X.
        INCX > 0.

SCALE   (input/output) DOUBLE PRECISION
        On entry, the value  scale  in the equation above.
        On exit, SCALE is overwritten with  scl , the scaling factor
        for the sum of squares.

SUMSQ   (input/output) DOUBLE PRECISION
        On entry, the value  sumsq  in the equation above.
        On exit, SUMSQ is overwritten with  smsq , the basic sum of
        squares from which  scl  has been factored out.
```

———-

90.190.3 fortran code

— dlassq.f —

```
      SUBROUTINE DLASSQ( N, X, INCX, SCALE, SUMSQ )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      INTEGER            INCX, N
      DOUBLE PRECISION   SCALE, SUMSQ
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   X( * )
*     ..
*
* =====================================================================
```

```
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            IX
      DOUBLE PRECISION   ABSXI
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS
*     ..
*     .. Executable Statements ..
*
      IF( N.GT.0 ) THEN
         DO 10 IX = 1, 1 + ( N-1 )*INCX, INCX
            IF( X( IX ).NE.ZERO ) THEN
               ABSXI = ABS( X( IX ) )
               IF( SCALE.LT.ABSXI ) THEN
                  SUMSQ = 1 + SUMSQ*( SCALE / ABSXI )**2
                  SCALE = ABSXI
               ELSE
                  SUMSQ = SUMSQ + ( ABSXI / SCALE )**2
               END IF
            END IF
   10    CONTINUE
      END IF
      RETURN
*
*     End of DLASSQ
*
      END
```

———-

90.190.4 lisp code

— dlassq.lisp —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun dlassq (n x incx scale sumsq)
    (declare (type (double-float) sumsq scale)
             (type (array double-float (*)) x)
             (type (f2cl-lib:integer4) incx n))
    (f2cl-lib:with-multi-array-data
```

```
        ((x double-float x-%data% x-%offset%))
      (prog ((absxi 0.0d0) (ix 0))
        (declare (type (double-float) absxi)
                 (type (f2cl-lib:integer4) ix))
        (cond
         ((> n 0)
          (f2cl-lib:fdo (ix 1 (f2cl-lib:int-add ix incx))
                        ((> ix
                            (f2cl-lib:int-add 1
                                              (f2cl-lib:int-mul
                                               (f2cl-lib:int-add n
                                                                 (f2cl-lib:int-sub
                                                                  1))
                                               incx)))
                         nil)
            (tagbody
              (setf absxi
                      (abs (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%)))
              (cond
               ((or (> absxi zero)
                    (multiple-value-bind (ret-val var-0)
                        (disnan absxi)
                      (declare (ignore))
                      (setf absxi var-0)
                      ret-val))
                (cond
                 ((< scale absxi)
                  (setf sumsq (+ 1 (* sumsq (expt (/ scale absxi) 2))))
                  (setf scale absxi))
                 (t (setf sumsq (+ sumsq (expt (/ absxi scale) 2)))))))
             label10))))
        (go end_label)
       end_label
        (return (values nil nil nil scale sumsq))))))
```

———-

90.191 dlasv2

90.191.1 Axiom unit tests

— dlasv2.input —

```
)set break resume
)sys rm -f dlasv2.output
)spool dlasv2.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.191.2 Axiom help page

— dlasv2.help —

```
=====================================================================
dlasv2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

NAME
       DLASV2 - the singular value decomposition of a 2-by-2 triangular matrix
       [ F G ]  [ 0 H ]

SYNOPSIS
       SUBROUTINE DLASV2( F, G, H, SSMIN, SSMAX, SNR, CSR, SNL, CSL )

           DOUBLE         PRECISION CSL, CSR, F, G, H, SNL, SNR, SSMAX, SSMIN

  Purpose
  =======

  DLASV2 computes the singular value decomposition of a 2-by-2
  triangular matrix
     [  F   G  ]
     [  0   H  ].
  On return, abs(SSMAX) is the larger singular value, abs(SSMIN) is the
  smaller singular value, and (CSL,SNL) and (CSR,SNR) are the left and
  right singular vectors for abs(SSMAX), giving the decomposition

     [ CSL  SNL ] [  F   G  ] [ CSR -SNR ]  =  [ SSMAX   0   ]
     [-SNL  CSL ] [  0   H  ] [ SNR  CSR ]     [  0    SSMIN ].

  Arguments
  =========

  F       (input) DOUBLE PRECISION
```

```
        The (1,1) element of the 2-by-2 matrix.

G       (input) DOUBLE PRECISION
        The (1,2) element of the 2-by-2 matrix.

H       (input) DOUBLE PRECISION
        The (2,2) element of the 2-by-2 matrix.

SSMIN   (output) DOUBLE PRECISION
        abs(SSMIN) is the smaller singular value.

SSMAX   (output) DOUBLE PRECISION
        abs(SSMAX) is the larger singular value.

SNL     (output) DOUBLE PRECISION
CSL     (output) DOUBLE PRECISION
        The vector (CSL, SNL) is a unit left singular vector for the
        singular value abs(SSMAX).

SNR     (output) DOUBLE PRECISION
CSR     (output) DOUBLE PRECISION
        The vector (CSR, SNR) is a unit right singular vector for the
        singular value abs(SSMAX).

Further Details
===============

Any input parameter may be aliased with any output parameter.

Barring over/underflow and assuming a guard digit in subtraction, all
output quantities are correct to within a few units in the last
place (ulps).

In IEEE arithmetic, the code works correctly if one matrix element is
infinite.

Overflow will not occur unless the largest singular value itself
overflows or is within a few ulps of overflow. (On machines with
partial overflow, like the Cray, overflow may occur if the largest
singular value is within a factor of 2 of overflow.)

Underflow is harmless if underflow is gradual. Otherwise, results
may correspond to a matrix modified by perturbations of size near
the underflow threshold.
```

90.191.3 fortran code

— dlasv2.f —

```
      SUBROUTINE DLASV2( F, G, H, SSMIN, SSMAX, SNR, CSR, SNL, CSL )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1992
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   CSL, CSR, F, G, H, SNL, SNR, SSMAX, SSMIN
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D0 )
      DOUBLE PRECISION   HALF
      PARAMETER          ( HALF = 0.5D0 )
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D0 )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0D0 )
      DOUBLE PRECISION   FOUR
      PARAMETER          ( FOUR = 4.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            GASMAL, SWAP
      INTEGER            PMAX
      DOUBLE PRECISION   A, CLT, CRT, D, FA, FT, GA, GT, HA, HT, L, M,
     $                   MM, R, S, SLT, SRT, T, TEMP, TSIGN, TT
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, SIGN, SQRT
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. Executable Statements ..
*
      FT = F
      FA = ABS( FT )
      HT = H
      HA = ABS( H )
*
```

```
*     PMAX points to the maximum absolute element of matrix
*       PMAX = 1 if F largest in absolute values
*       PMAX = 2 if G largest in absolute values
*       PMAX = 3 if H largest in absolute values
*
      PMAX = 1
      SWAP = ( HA.GT.FA )
      IF( SWAP ) THEN
         PMAX = 3
         TEMP = FT
         FT = HT
         HT = TEMP
         TEMP = FA
         FA = HA
         HA = TEMP
*
*        Now FA .ge. HA
*
      END IF
      GT = G
      GA = ABS( GT )
      IF( GA.EQ.ZERO ) THEN
*
*        Diagonal matrix
*
         SSMIN = HA
         SSMAX = FA
         CLT = ONE
         CRT = ONE
         SLT = ZERO
         SRT = ZERO
      ELSE
         GASMAL = .TRUE.
         IF( GA.GT.FA ) THEN
            PMAX = 2
            IF( ( FA / GA ).LT.DLAMCH( 'EPS' ) ) THEN
*
*              Case of very large GA
*
               GASMAL = .FALSE.
               SSMAX = GA
               IF( HA.GT.ONE ) THEN
                  SSMIN = FA / ( GA / HA )
               ELSE
                  SSMIN = ( FA / GA )*HA
               END IF
               CLT = ONE
               SLT = HT / GT
               SRT = ONE
               CRT = FT / GT
```

```
         END IF
      END IF
      IF( GASMAL ) THEN
*
*        Normal case
*
         D = FA - HA
         IF( D.EQ.FA ) THEN
*
*           Copes with infinite F or H
*
            L = ONE
         ELSE
            L = D / FA
         END IF
*
*        Note that 0 .le. L .le. 1
*
         M = GT / FT
*
*        Note that abs(M) .le. 1/macheps
*
         T = TWO - L
*
*        Note that T .ge. 1
*
         MM = M*M
         TT = T*T
         S = SQRT( TT+MM )
*
*        Note that 1 .le. S .le. 1 + 1/macheps
*
         IF( L.EQ.ZERO ) THEN
            R = ABS( M )
         ELSE
            R = SQRT( L*L+MM )
         END IF
*
*        Note that 0 .le. R .le. 1 + 1/macheps
*
         A = HALF*( S+R )
*
*        Note that 1 .le. A .le. 1 + abs(M)
*
         SSMIN = HA / A
         SSMAX = FA*A
         IF( MM.EQ.ZERO ) THEN
*
*           Note that M is very tiny
*
```

```
            IF( L.EQ.ZERO ) THEN
               T = SIGN( TWO, FT )*SIGN( ONE, GT )
            ELSE
               T = GT / SIGN( D, FT ) + M / T
            END IF
         ELSE
            T = ( M / ( S+T )+M / ( R+L ) )*( ONE+A )
         END IF
         L = SQRT( T*T+FOUR )
         CRT = TWO / L
         SRT = T / L
         CLT = ( CRT+SRT*M ) / A
         SLT = ( HT / FT )*SRT / A
      END IF
   END IF
   IF( SWAP ) THEN
      CSL = SRT
      SNL = CRT
      CSR = SLT
      SNR = CLT
   ELSE
      CSL = CLT
      SNL = SLT
      CSR = CRT
      SNR = SRT
   END IF
*
*     Correct signs of SSMAX and SSMIN
*
   IF( PMAX.EQ.1 )
  $   TSIGN = SIGN( ONE, CSR )*SIGN( ONE, CSL )*SIGN( ONE, F )
   IF( PMAX.EQ.2 )
  $   TSIGN = SIGN( ONE, SNR )*SIGN( ONE, CSL )*SIGN( ONE, G )
   IF( PMAX.EQ.3 )
  $   TSIGN = SIGN( ONE, SNR )*SIGN( ONE, SNL )*SIGN( ONE, H )
   SSMAX = SIGN( SSMAX, TSIGN )
   SSMIN = SIGN( SSMIN, TSIGN*SIGN( ONE, F )*SIGN( ONE, H ) )
   RETURN
*
*     End of DLASV2
*
   END
```

———-

90.191.4 lisp code

— dlasv2.lisp —

```
(let* ((zero 0.0d0) (half 0.5d0) (one 1.0d0) (two 2.0d0) (four 4.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 0.5d0 0.5d0) half)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 4.0d0 4.0d0) four)
           (ignorable zero half one two four))
  (defun dlasv2 (f g h ssmin ssmax snr csr snl csl)
    (declare (type (double-float) csl snl csr snr ssmax ssmin h g f))
    (prog ((a 0.0d0) (clt 0.0d0) (crt 0.0d0) (d 0.0d0) (fa 0.0d0) (ft 0.0d0)
           (ga 0.0d0) (gt 0.0d0) (ha 0.0d0) (ht 0.0d0) (l 0.0d0) (m 0.0d0)
           (mm 0.0d0) (r 0.0d0) (s 0.0d0) (slt 0.0d0) (srt 0.0d0) (t$ 0.0d0)
           (temp 0.0d0) (tsign 0.0d0) (tt 0.0d0) (pmax 0) (gasmal nil)
           (swap nil))
      (declare
       (type (double-float) a clt crt d fa ft ga gt ha ht l m mm r s slt srt t$
        temp tsign tt)
       (type (f2cl-lib:integer4) pmax)
       (type f2cl-lib:logical gasmal swap))
      (setf ft f)
      (setf fa (abs ft))
      (setf ht h)
      (setf ha (abs h))
      (setf pmax 1)
      (setf swap (> ha fa))
      (cond
       (swap (setf pmax 3) (setf temp ft) (setf ft ht) (setf ht temp)
        (setf temp fa) (setf fa ha) (setf ha temp)))
      (setf gt g)
      (setf ga (abs gt))
      (cond
       ((= ga zero) (setf ssmin ha) (setf ssmax fa) (setf clt one)
        (setf crt one) (setf slt zero) (setf srt zero))
       (t (setf gasmal f2cl-lib:%true%)
        (cond
         ((> ga fa) (setf pmax 2)
          (cond
           ((< (f2cl-lib:f2cl/ fa ga) (dlamch "EPS"))
            (setf gasmal f2cl-lib:%false%) (setf ssmax ga)
            (cond ((> ha one) (setf ssmin (/ fa (/ ga ha))))
                  (t (setf ssmin (* (/ fa ga) ha))))
            (setf clt one) (setf slt (/ ht gt)) (setf srt one)
            (setf crt (/ ft gt))))))
        (cond
         (gasmal (setf d (- fa ha))
          (cond ((= d fa) (setf l one)) (t (setf l (/ d fa))))
          (setf m (/ gt ft)) (setf t$ (- two l)) (setf mm (* m m))
          (setf tt (* t$ t$)) (setf s (f2cl-lib:fsqrt (+ tt mm)))
```

```
        (cond ((= l zero) (setf r (abs m)))
              (t (setf r (f2cl-lib:fsqrt (+ (* l l) mm)))))
        (setf a (* half (+ s r))) (setf ssmin (/ ha a)) (setf ssmax (* fa a))
        (cond
         ((= mm zero)
          (cond
           ((= l zero)
            (setf t$ (* (f2cl-lib:sign two ft) (f2cl-lib:sign one gt))))
           (t (setf t$ (+ (/ gt (f2cl-lib:sign d ft)) (/ m t$))))))
         (t (setf t$ (* (+ (/ m (+ s t$)) (/ m (+ r l))) (+ one a)))))
        (setf l (f2cl-lib:fsqrt (+ (* t$ t$) four))) (setf crt (/ two l))
        (setf srt (/ t$ l)) (setf clt (/ (+ crt (* srt m)) a))
        (setf slt (/ (* (/ ht ft) srt) a))))))
    (cond (swap (setf csl srt) (setf snl crt) (setf csr slt) (setf snr clt))
          (t (setf csl clt) (setf snl slt) (setf csr crt) (setf snr srt)))
    (if (= pmax 1)
        (setf tsign
                (* (f2cl-lib:sign one csr) (f2cl-lib:sign one csl)
                   (f2cl-lib:sign one f))))
    (if (= pmax 2)
        (setf tsign
                (* (f2cl-lib:sign one snr) (f2cl-lib:sign one csl)
                   (f2cl-lib:sign one g))))
    (if (= pmax 3)
        (setf tsign
                (* (f2cl-lib:sign one snr) (f2cl-lib:sign one snl)
                   (f2cl-lib:sign one h))))
    (setf ssmax (f2cl-lib:sign ssmax tsign))
    (setf ssmin
            (f2cl-lib:sign ssmin
                           (* tsign (f2cl-lib:sign one f)
                              (f2cl-lib:sign one h))))
    (go end_label)
   end_label
    (return (values nil nil nil ssmin ssmax snr csr snl csl)))))
```

———-

90.192 dlaswp

90.192.1 Axiom unit tests

— dlaswp.input —

```
)set break resume
)sys rm -f dlaswp.output
```

```
)spool dlaswp.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

90.192.2 Axiom help page

— dlaswp.help —

```
=====================================================================
dlaswp examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

NAME
       DLASWP - a series of row interchanges on the matrix A

SYNOPSIS
       SUBROUTINE DLASWP( N, A, LDA, K1, K2, IPIV, INCX )

           INTEGER        INCX, K1, K2, LDA, N

           INTEGER        IPIV( * )

           DOUBLE         PRECISION A( LDA, * )

  Purpose
  =======

  DLASWP performs a series of row interchanges on the matrix A.
  One row interchange is initiated for each of rows K1 through K2 of A.

  Arguments
  =========

  N       (input) INTEGER
          The number of columns of the matrix A.

  A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
          On entry, the matrix of column dimension N to which the row
```

```
         interchanges will be applied.
         On exit, the permuted matrix.

 LDA     (input) INTEGER
         The leading dimension of the array A.

 K1      (input) INTEGER
         The first element of IPIV for which a row interchange will
         be done.

 K2      (input) INTEGER
         The last element of IPIV for which a row interchange will
         be done.

 IPIV    (input) INTEGER array, dimension (M*abs(INCX))
         The vector of pivot indices.  Only the elements in positions
         K1 through K2 of IPIV are accessed.
         IPIV(K) = L implies rows K and L are to be interchanged.

 INCX    (input) INTEGER
         The increment between successive values of IPIV.  If IPIV
         is negative, the pivots are applied in reverse order.

 Further Details
 ===============

 Modified by
  R. C. Whaley, Computer Science Dept., Univ. of Tenn., Knoxville, USA
```

90.192.3 fortran code

— dlaswp.f —

```
      SUBROUTINE DLASWP( N, A, LDA, K1, K2, IPIV, INCX )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      INTEGER            INCX, K1, K2, LDA, N
*     ..
*     .. Array Arguments ..
      INTEGER            IPIV( * )
```

```
      DOUBLE PRECISION   A( LDA, * )
*     ..
*
* =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, I1, I2, INC, IP, IX, IX0, J, K, N32
      DOUBLE PRECISION   TEMP
*     ..
*     .. Executable Statements ..
*
*     Interchange row I with row IPIV(I) for each of rows K1 through K2.
*
      IF( INCX.GT.0 ) THEN
         IX0 = K1
         I1 = K1
         I2 = K2
         INC = 1
      ELSE IF( INCX.LT.0 ) THEN
         IX0 = 1 + ( 1-K2 )*INCX
         I1 = K2
         I2 = K1
         INC = -1
      ELSE
         RETURN
      END IF
*
      N32 = ( N / 32 )*32
      IF( N32.NE.0 ) THEN
         DO 30 J = 1, N32, 32
            IX = IX0
            DO 20 I = I1, I2, INC
               IP = IPIV( IX )
               IF( IP.NE.I ) THEN
                  DO 10 K = J, J + 31
                     TEMP = A( I, K )
                     A( I, K ) = A( IP, K )
                     A( IP, K ) = TEMP
   10             CONTINUE
               END IF
               IX = IX + INCX
   20       CONTINUE
   30    CONTINUE
      END IF
      IF( N32.NE.N ) THEN
         N32 = N32 + 1
         IX = IX0
         DO 50 I = I1, I2, INC
            IP = IPIV( IX )
            IF( IP.NE.I ) THEN
```

```
               DO 40 K = N32, N
                  TEMP = A( I, K )
                  A( I, K ) = A( IP, K )
                  A( IP, K ) = TEMP
   40          CONTINUE
            END IF
            IX = IX + INCX
   50    CONTINUE
      END IF
*
      RETURN
*
*     End of DLASWP
*
      END
```

———-

90.192.4 lisp code

— dlaswp.lisp —

```
(defun dlaswp (n a lda k1 k2 ipiv incx)
  (declare (type (array f2cl-lib:integer4 (*)) ipiv)
           (type (array double-float (*)) a)
           (type (f2cl-lib:integer4) incx k2 k1 lda n))
  (f2cl-lib:with-multi-array-data
      ((a double-float a-%data% a-%offset%)
       (ipiv f2cl-lib:integer4 ipiv-%data% ipiv-%offset%))
    (prog ((temp 0.0d0) (i 0) (i1 0) (i2 0) (inc 0) (ip 0) (ix 0) (ix0 0) (j 0)
           (k 0) (n32 0))
      (declare (type (f2cl-lib:integer4) n32 k j ix0 ix ip inc i2 i1 i)
               (type (double-float) temp))
      (cond ((> incx 0) (setf ix0 k1) (setf i1 k1) (setf i2 k2) (setf inc 1))
            ((< incx 0)
             (setf ix0
                     (f2cl-lib:int-add 1
                                       (f2cl-lib:int-mul
                                        (f2cl-lib:int-sub 1 k2) incx)))
             (setf i1 k2) (setf i2 k1) (setf inc -1))
            (t (go end_label)))
      (setf n32 (* (the f2cl-lib:integer4 (truncate n 32)) 32))
      (cond
       ((/= n32 0)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 32))
                      ((> j n32) nil)
          (tagbody
```

```
        (setf ix ix0)
        (f2cl-lib:fdo (i i1 (f2cl-lib:int-add i inc))
                      ((> i i2) nil)
          (tagbody
            (setf ip
                    (f2cl-lib:fref ipiv-%data% (ix) ((1 *)) ipiv-%offset%))
            (cond
             ((/= ip i)
              (f2cl-lib:fdo (k j (f2cl-lib:int-add k 1))
                            ((> k (f2cl-lib:int-add j 31)) nil)
                (tagbody
                  (setf temp
                          (f2cl-lib:fref a-%data% (i k) ((1 lda) (1 *))
                                         a-%offset%))
                  (setf (f2cl-lib:fref a-%data% (i k) ((1 lda) (1 *))
                                       a-%offset%)
                          (f2cl-lib:fref a-%data% (ip k) ((1 lda) (1 *))
                                         a-%offset%))
                  (setf (f2cl-lib:fref a-%data% (ip k) ((1 lda) (1 *))
                                       a-%offset%)
                          temp)
                label10))))
            (setf ix (f2cl-lib:int-add ix incx))
           label20))
      label30))))
(cond
 ((/= n32 n) (setf n32 (f2cl-lib:int-add n32 1)) (setf ix ix0)
  (f2cl-lib:fdo (i i1 (f2cl-lib:int-add i inc))
                ((> i i2) nil)
    (tagbody
      (setf ip (f2cl-lib:fref ipiv-%data% (ix) ((1 *)) ipiv-%offset%))
      (cond
       ((/= ip i)
        (f2cl-lib:fdo (k n32 (f2cl-lib:int-add k 1))
                      ((> k n) nil)
          (tagbody
            (setf temp
                    (f2cl-lib:fref a-%data% (i k) ((1 lda) (1 *))
                                   a-%offset%))
            (setf (f2cl-lib:fref a-%data% (i k) ((1 lda) (1 *))
                                 a-%offset%)
                    (f2cl-lib:fref a-%data% (ip k) ((1 lda) (1 *))
                                   a-%offset%))
            (setf (f2cl-lib:fref a-%data% (ip k) ((1 lda) (1 *))
                                 a-%offset%)
                    temp)
           label40))))
      (setf ix (f2cl-lib:int-add ix incx))
     label50))))
(go end_label)
```

```
    end_label
     (return (values nil nil nil nil nil nil nil)))))
```

———-

90.193 dlasy2

90.193.1 Axiom unit tests

— **dlasy2.input** —

```
)set break resume
)sys rm -f dlasy2.output
)spool dlasy2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.193.2 Axiom help page

— **dlasy2.help** —

```
==================================================================
dlasy2 examples
==================================================================

==================================================================
Man Page Details
==================================================================

NAME
       DLASY2  -  for  the N1 by N2 matrix X, 1 <= N1,N2 <= 2, in   op(TL)*X +
       ISGN*X*op(TR) = SCALE*B,

SYNOPSIS
       SUBROUTINE DLASY2( LTRANL, LTRANR, ISGN, N1, N2, TL, LDTL, TR, LDTR, B,
                          LDB, SCALE, X, LDX, XNORM, INFO )
```

```
      LOGICAL        LTRANL, LTRANR

      INTEGER        INFO, ISGN, LDB, LDTL, LDTR, LDX, N1, N2

      DOUBLE         PRECISION SCALE, XNORM

      DOUBLE         PRECISION B( LDB, * ), TL( LDTL, * ), TR( LDTR, * ),
                     X( LDX, * )

Purpose
=======

DLASY2 solves for the N1 by N2 matrix X, 1 <= N1,N2 <= 2, in

       op(TL)*X + ISGN*X*op(TR) = SCALE*B,

where TL is N1 by N1, TR is N2 by N2, B is N1 by N2, and ISGN = 1 or
-1.  op(T) = T or T', where T' denotes the transpose of T.

Arguments
=========

LTRANL  (input) LOGICAL
        On entry, LTRANL specifies the op(TL):
           = .FALSE., op(TL) = TL,
           = .TRUE., op(TL) = TL'.

LTRANR  (input) LOGICAL
        On entry, LTRANR specifies the op(TR):
          = .FALSE., op(TR) = TR,
          = .TRUE., op(TR) = TR'.

ISGN    (input) INTEGER
        On entry, ISGN specifies the sign of the equation
        as described before. ISGN may only be 1 or -1.

N1      (input) INTEGER
        On entry, N1 specifies the order of matrix TL.
        N1 may only be 0, 1 or 2.

N2      (input) INTEGER
        On entry, N2 specifies the order of matrix TR.
        N2 may only be 0, 1 or 2.

TL      (input) DOUBLE PRECISION array, dimension (LDTL,2)
        On entry, TL contains an N1 by N1 matrix.

LDTL    (input) INTEGER
        The leading dimension of the matrix TL. LDTL >= max(1,N1).
```

```
TR      (input) DOUBLE PRECISION array, dimension (LDTR,2)
        On entry, TR contains an N2 by N2 matrix.

LDTR    (input) INTEGER
        The leading dimension of the matrix TR. LDTR >= max(1,N2).

B       (input) DOUBLE PRECISION array, dimension (LDB,2)
        On entry, the N1 by N2 matrix B contains the right-hand
        side of the equation.

LDB     (input) INTEGER
        The leading dimension of the matrix B. LDB >= max(1,N1).

SCALE   (output) DOUBLE PRECISION
        On exit, SCALE contains the scale factor. SCALE is chosen
        less than or equal to 1 to prevent the solution overflowing.

X       (output) DOUBLE PRECISION array, dimension (LDX,2)
        On exit, X contains the N1 by N2 solution.

LDX     (input) INTEGER
        The leading dimension of the matrix X. LDX >= max(1,N1).

XNORM   (output) DOUBLE PRECISION
        On exit, XNORM is the infinity-norm of the solution.

INFO    (output) INTEGER
        On exit, INFO is set to
           0: successful exit.
           1: TL and TR have too close eigenvalues, so TL or
              TR is perturbed to get a nonsingular equation.
        NOTE: In the interests of speed, this routine does not
              check the inputs for errors.
```

———-

90.193.3 fortran code

— dlasy2.f —

```
      SUBROUTINE DLASY2( LTRANL, LTRANR, ISGN, N1, N2, TL, LDTL, TR,
     $                   LDTR, B, LDB, SCALE, X, LDX, XNORM, INFO )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     October 31, 1992
```

```
*
*     .. Scalar Arguments ..
      LOGICAL            LTRANL, LTRANR
      INTEGER            INFO, ISGN, LDB, LDTL, LDTR, LDX, N1, N2
      DOUBLE PRECISION   SCALE, XNORM
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   B( LDB, * ), TL( LDTL, * ), TR( LDTR, * ),
     $                   X( LDX, * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   TWO, HALF, EIGHT
      PARAMETER          ( TWO = 2.0D+0, HALF = 0.5D+0, EIGHT = 8.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            BSWAP, XSWAP
      INTEGER            I, IP, IPIV, IPSV, J, JP, JPSV, K
      DOUBLE PRECISION   BET, EPS, GAM, L21, SGN, SMIN, SMLNUM, TAU1,
     $                   TEMP, U11, U12, U22, XMAX
*     ..
*     .. Local Arrays ..
      LOGICAL            BSWPIV( 4 ), XSWPIV( 4 )
      INTEGER            JPIV( 4 ), LOCL21( 4 ), LOCU12( 4 ),
     $                   LOCU22( 4 )
      DOUBLE PRECISION   BTMP( 4 ), T16( 4, 4 ), TMP( 4 ), X2( 2 )
*     ..
*     .. External Functions ..
      INTEGER            IDAMAX
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           IDAMAX, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DCOPY, DSWAP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX
*     ..
*     .. Data statements ..
      DATA               LOCU12 / 3, 4, 1, 2 / , LOCL21 / 2, 1, 4, 3 / ,
     $                   LOCU22 / 4, 3, 2, 1 /
      DATA               XSWPIV / .FALSE., .FALSE., .TRUE., .TRUE. /
      DATA               BSWPIV / .FALSE., .TRUE., .FALSE., .TRUE. /
*     ..
*     .. Executable Statements ..
*
```

```
*     Do not check the input parameters for errors
*
      INFO = 0
*
*     Quick return if possible
*
      IF( N1.EQ.0 .OR. N2.EQ.0 )
     $   RETURN
*
*     Set constants to control overflow
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' ) / EPS
      SGN = ISGN
*
      K = N1 + N1 + N2 - 2
      GO TO ( 10, 20, 30, 50 )K
*
*     1 by 1: TL11*X + SGN*X*TR11 = B11
*
   10 CONTINUE
      TAU1 = TL( 1, 1 ) + SGN*TR( 1, 1 )
      BET = ABS( TAU1 )
      IF( BET.LE.SMLNUM ) THEN
         TAU1 = SMLNUM
         BET = SMLNUM
         INFO = 1
      END IF
*
      SCALE = ONE
      GAM = ABS( B( 1, 1 ) )
      IF( SMLNUM*GAM.GT.BET )
     $   SCALE = ONE / GAM
*
      X( 1, 1 ) = ( B( 1, 1 )*SCALE ) / TAU1
      XNORM = ABS( X( 1, 1 ) )
      RETURN
*
*     1 by 2:
*     TL11*[X11 X12] + ISGN*[X11 X12]*op[TR11 TR12]  = [B11 B12]
*                                       [TR21 TR22]
*
   20 CONTINUE
*
      SMIN = MAX( EPS*MAX( ABS( TL( 1, 1 ) ), ABS( TR( 1, 1 ) ),
     $       ABS( TR( 1, 2 ) ), ABS( TR( 2, 1 ) ), ABS( TR( 2, 2 ) ) ),
     $       SMLNUM )
      TMP( 1 ) = TL( 1, 1 ) + SGN*TR( 1, 1 )
      TMP( 4 ) = TL( 1, 1 ) + SGN*TR( 2, 2 )
      IF( LTRANR ) THEN
```

```
         TMP( 2 ) = SGN*TR( 2, 1 )
         TMP( 3 ) = SGN*TR( 1, 2 )
      ELSE
         TMP( 2 ) = SGN*TR( 1, 2 )
         TMP( 3 ) = SGN*TR( 2, 1 )
      END IF
      BTMP( 1 ) = B( 1, 1 )
      BTMP( 2 ) = B( 1, 2 )
      GO TO 40
*
*     2 by 1:
*          op[TL11 TL12]*[X11] + ISGN* [X11]*TR11  = [B11]
*            [TL21 TL22] [X21]         [X21]         [B21]
*
   30 CONTINUE
      SMIN = MAX( EPS*MAX( ABS( TR( 1, 1 ) ), ABS( TL( 1, 1 ) ),
     $       ABS( TL( 1, 2 ) ), ABS( TL( 2, 1 ) ), ABS( TL( 2, 2 ) ) ),
     $       SMLNUM )
      TMP( 1 ) = TL( 1, 1 ) + SGN*TR( 1, 1 )
      TMP( 4 ) = TL( 2, 2 ) + SGN*TR( 1, 1 )
      IF( LTRANL ) THEN
         TMP( 2 ) = TL( 1, 2 )
         TMP( 3 ) = TL( 2, 1 )
      ELSE
         TMP( 2 ) = TL( 2, 1 )
         TMP( 3 ) = TL( 1, 2 )
      END IF
      BTMP( 1 ) = B( 1, 1 )
      BTMP( 2 ) = B( 2, 1 )
   40 CONTINUE
*
*     Solve 2 by 2 system using complete pivoting.
*     Set pivots less than SMIN to SMIN.
*
      IPIV = IDAMAX( 4, TMP, 1 )
      U11 = TMP( IPIV )
      IF( ABS( U11 ).LE.SMIN ) THEN
         INFO = 1
         U11 = SMIN
      END IF
      U12 = TMP( LOCU12( IPIV ) )
      L21 = TMP( LOCL21( IPIV ) ) / U11
      U22 = TMP( LOCU22( IPIV ) ) - U12*L21
      XSWAP = XSWPIV( IPIV )
      BSWAP = BSWPIV( IPIV )
      IF( ABS( U22 ).LE.SMIN ) THEN
         INFO = 1
         U22 = SMIN
      END IF
      IF( BSWAP ) THEN
```

```
         TEMP = BTMP( 2 )
         BTMP( 2 ) = BTMP( 1 ) - L21*TEMP
         BTMP( 1 ) = TEMP
      ELSE
         BTMP( 2 ) = BTMP( 2 ) - L21*BTMP( 1 )
      END IF
      SCALE = ONE
      IF( ( TWO*SMLNUM )*ABS( BTMP( 2 ) ).GT.ABS( U22 ) .OR.
     $    ( TWO*SMLNUM )*ABS( BTMP( 1 ) ).GT.ABS( U11 ) ) THEN
         SCALE = HALF / MAX( ABS( BTMP( 1 ) ), ABS( BTMP( 2 ) ) )
         BTMP( 1 ) = BTMP( 1 )*SCALE
         BTMP( 2 ) = BTMP( 2 )*SCALE
      END IF
      X2( 2 ) = BTMP( 2 ) / U22
      X2( 1 ) = BTMP( 1 ) / U11 - ( U12 / U11 )*X2( 2 )
      IF( XSWAP ) THEN
         TEMP = X2( 2 )
         X2( 2 ) = X2( 1 )
         X2( 1 ) = TEMP
      END IF
      X( 1, 1 ) = X2( 1 )
      IF( N1.EQ.1 ) THEN
         X( 1, 2 ) = X2( 2 )
         XNORM = ABS( X( 1, 1 ) ) + ABS( X( 1, 2 ) )
      ELSE
         X( 2, 1 ) = X2( 2 )
         XNORM = MAX( ABS( X( 1, 1 ) ), ABS( X( 2, 1 ) ) )
      END IF
      RETURN
*
*     2 by 2:
*     op[TL11 TL12]*[X11 X12] +ISGN* [X11 X12]*op[TR11 TR12] = [B11 B12]
*       [TL21 TL22] [X21 X22]        [X21 X22]   [TR21 TR22]   [B21 B22]
*
*     Solve equivalent 4 by 4 system using complete pivoting.
*     Set pivots less than SMIN to SMIN.
*
   50 CONTINUE
      SMIN = MAX( ABS( TR( 1, 1 ) ), ABS( TR( 1, 2 ) ),
     $       ABS( TR( 2, 1 ) ), ABS( TR( 2, 2 ) ) )
      SMIN = MAX( SMIN, ABS( TL( 1, 1 ) ), ABS( TL( 1, 2 ) ),
     $       ABS( TL( 2, 1 ) ), ABS( TL( 2, 2 ) ) )
      SMIN = MAX( EPS*SMIN, SMLNUM )
      BTMP( 1 ) = ZERO
      CALL DCOPY( 16, BTMP, 0, T16, 1 )
      T16( 1, 1 ) = TL( 1, 1 ) + SGN*TR( 1, 1 )
      T16( 2, 2 ) = TL( 2, 2 ) + SGN*TR( 1, 1 )
      T16( 3, 3 ) = TL( 1, 1 ) + SGN*TR( 2, 2 )
      T16( 4, 4 ) = TL( 2, 2 ) + SGN*TR( 2, 2 )
      IF( LTRANL ) THEN
```

```
         T16( 1, 2 ) = TL( 2, 1 )
         T16( 2, 1 ) = TL( 1, 2 )
         T16( 3, 4 ) = TL( 2, 1 )
         T16( 4, 3 ) = TL( 1, 2 )
      ELSE
         T16( 1, 2 ) = TL( 1, 2 )
         T16( 2, 1 ) = TL( 2, 1 )
         T16( 3, 4 ) = TL( 1, 2 )
         T16( 4, 3 ) = TL( 2, 1 )
      END IF
      IF( LTRANR ) THEN
         T16( 1, 3 ) = SGN*TR( 1, 2 )
         T16( 2, 4 ) = SGN*TR( 1, 2 )
         T16( 3, 1 ) = SGN*TR( 2, 1 )
         T16( 4, 2 ) = SGN*TR( 2, 1 )
      ELSE
         T16( 1, 3 ) = SGN*TR( 2, 1 )
         T16( 2, 4 ) = SGN*TR( 2, 1 )
         T16( 3, 1 ) = SGN*TR( 1, 2 )
         T16( 4, 2 ) = SGN*TR( 1, 2 )
      END IF
      BTMP( 1 ) = B( 1, 1 )
      BTMP( 2 ) = B( 2, 1 )
      BTMP( 3 ) = B( 1, 2 )
      BTMP( 4 ) = B( 2, 2 )
*
*     Perform elimination
*
      DO 100 I = 1, 3
         XMAX = ZERO
         DO 70 IP = I, 4
            DO 60 JP = I, 4
               IF( ABS( T16( IP, JP ) ).GE.XMAX ) THEN
                  XMAX = ABS( T16( IP, JP ) )
                  IPSV = IP
                  JPSV = JP
               END IF
   60       CONTINUE
   70    CONTINUE
         IF( IPSV.NE.I ) THEN
            CALL DSWAP( 4, T16( IPSV, 1 ), 4, T16( I, 1 ), 4 )
            TEMP = BTMP( I )
            BTMP( I ) = BTMP( IPSV )
            BTMP( IPSV ) = TEMP
         END IF
         IF( JPSV.NE.I )
     $      CALL DSWAP( 4, T16( 1, JPSV ), 1, T16( 1, I ), 1 )
         JPIV( I ) = JPSV
         IF( ABS( T16( I, I ) ).LT.SMIN ) THEN
            INFO = 1
```

```
            T16( I, I ) = SMIN
         END IF
         DO 90 J = I + 1, 4
            T16( J, I ) = T16( J, I ) / T16( I, I )
            BTMP( J ) = BTMP( J ) - T16( J, I )*BTMP( I )
            DO 80 K = I + 1, 4
               T16( J, K ) = T16( J, K ) - T16( J, I )*T16( I, K )
   80       CONTINUE
   90    CONTINUE
  100 CONTINUE
      IF( ABS( T16( 4, 4 ) ).LT.SMIN )
     $   T16( 4, 4 ) = SMIN
      SCALE = ONE
      IF( ( EIGHT*SMLNUM )*ABS( BTMP( 1 ) ).GT.ABS( T16( 1, 1 ) ) .OR.
     $    ( EIGHT*SMLNUM )*ABS( BTMP( 2 ) ).GT.ABS( T16( 2, 2 ) ) .OR.
     $    ( EIGHT*SMLNUM )*ABS( BTMP( 3 ) ).GT.ABS( T16( 3, 3 ) ) .OR.
     $    ( EIGHT*SMLNUM )*ABS( BTMP( 4 ) ).GT.ABS( T16( 4, 4 ) ) ) THEN
         SCALE = ( ONE / EIGHT ) / MAX( ABS( BTMP( 1 ) ),
     $           ABS( BTMP( 2 ) ), ABS( BTMP( 3 ) ), ABS( BTMP( 4 ) ) )
         BTMP( 1 ) = BTMP( 1 )*SCALE
         BTMP( 2 ) = BTMP( 2 )*SCALE
         BTMP( 3 ) = BTMP( 3 )*SCALE
         BTMP( 4 ) = BTMP( 4 )*SCALE
      END IF
      DO 120 I = 1, 4
         K = 5 - I
         TEMP = ONE / T16( K, K )
         TMP( K ) = BTMP( K )*TEMP
         DO 110 J = K + 1, 4
            TMP( K ) = TMP( K ) - ( TEMP*T16( K, J ) )*TMP( J )
  110    CONTINUE
  120 CONTINUE
      DO 130 I = 1, 3
         IF( JPIV( 4-I ).NE.4-I ) THEN
            TEMP = TMP( 4-I )
            TMP( 4-I ) = TMP( JPIV( 4-I ) )
            TMP( JPIV( 4-I ) ) = TEMP
         END IF
  130 CONTINUE
      X( 1, 1 ) = TMP( 1 )
      X( 2, 1 ) = TMP( 2 )
      X( 1, 2 ) = TMP( 3 )
      X( 2, 2 ) = TMP( 4 )
      XNORM = MAX( ABS( TMP( 1 ) )+ABS( TMP( 3 ) ),
     $        ABS( TMP( 2 ) )+ABS( TMP( 4 ) ) )
      RETURN
*
*     End of DLASY2
*
      END
```

———-

90.193.4 lisp code

— dlasy2.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0) (two 2.0d0) (half 0.5d0) (eight 8.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 0.5d0 0.5d0) half)
           (type (double-float 8.0d0 8.0d0) eight)
           (ignorable zero one two half eight))
  (let ((bswpiv
         (make-array 4 :element-type 't :initial-contents '(nil t nil t)))
        (xswpiv
         (make-array 4 :element-type 't :initial-contents '(nil nil t t)))
        (locu22
         (make-array 4 :element-type 'f2cl-lib:integer4 :initial-contents
                     '(4 3 2 1)))
        (locl21
         (make-array 4 :element-type 'f2cl-lib:integer4 :initial-contents
                     '(2 1 4 3)))
        (locu12
         (make-array 4 :element-type 'f2cl-lib:integer4 :initial-contents
                     '(3 4 1 2))))
    (declare (type (array f2cl-lib:logical (4)) bswpiv xswpiv)
             (type (array f2cl-lib:integer4 (4)) locu22 locl21 locu12))
    (defun dlasy2
           (ltranl ltranr isgn n1 n2 tl ldtl tr ldtr b ldb$ scale x ldx xnorm
            info)
      (declare (type (double-float) xnorm scale)
               (type (array double-float (*)) x b tr tl)
               (type (f2cl-lib:integer4) info ldx ldb$ ldtr ldtl n2 n1 isgn)
               (type f2cl-lib:logical ltranr ltranl))
      (f2cl-lib:with-multi-array-data
          ((tl double-float tl-%data% tl-%offset%)
           (tr double-float tr-%data% tr-%offset%)
           (b double-float b-%data% b-%offset%)
           (x double-float x-%data% x-%offset%))
        (prog ((btmp (make-array 4 :element-type 'double-float))
               (t16 (make-array 16 :element-type 'double-float))
               (tmp (make-array 4 :element-type 'double-float))
               (x2 (make-array 2 :element-type 'double-float))
               (jpiv (make-array 4 :element-type 'f2cl-lib:integer4))
               (bet 0.0d0) (eps 0.0d0) (gam 0.0d0) (l21 0.0d0) (sgn 0.0d0)
```

```
      (smin 0.0d0) (smlnum 0.0d0) (tau1 0.0d0) (temp 0.0d0)
      (u11 0.0d0) (u12 0.0d0) (u22 0.0d0) (xmax 0.0d0) (i 0) (ip 0)
      (ipiv 0) (ipsv 0) (j 0) (jp 0) (jpsv 0) (k 0) (bswap nil)
      (xswap nil))
 (declare (type (array double-float (16)) t16)
          (type (array double-float (4)) btmp tmp)
          (type (array double-float (2)) x2)
          (type (array f2cl-lib:integer4 (4)) jpiv)
          (type (double-float) bet eps gam l21 sgn smin smlnum tau1
           temp u11 u12 u22 xmax)
          (type (f2cl-lib:integer4) i ip ipiv ipsv j jp jpsv k)
          (type f2cl-lib:logical bswap xswap))
 (setf info 0)
 (if (or (= n1 0) (= n2 0))
     (go end_label))
 (setf eps (dlamch "P"))
 (setf smlnum (/ (dlamch "S") eps))
 (setf sgn (coerce (the f2cl-lib:integer4 isgn) 'double-float))
 (setf k (f2cl-lib:int-sub (f2cl-lib:int-add n1 n1 n2) 2))
 (f2cl-lib:computed-goto (label10 label20 label30 label50) k)
label10
 (setf tau1
         (+
          (f2cl-lib:fref tl-%data% (1 1) ((1 ldtl) (1 *)) tl-%offset%)
          (* sgn
             (f2cl-lib:fref tr-%data% (1 1) ((1 ldtr) (1 *))
                            tr-%offset%))))
 (setf bet (abs tau1))
 (cond
  ((<= bet smlnum) (setf tau1 smlnum) (setf bet smlnum)
   (setf info 1)))
 (setf scale one)
 (setf gam
         (abs
          (f2cl-lib:fref b-%data% (1 1) ((1 ldb$) (1 *)) b-%offset%)))
 (if (> (* smlnum gam) bet)
     (setf scale (/ one gam)))
 (setf (f2cl-lib:fref x-%data% (1 1) ((1 ldx) (1 *)) x-%offset%)
         (/
          (*
           (f2cl-lib:fref b-%data% (1 1) ((1 ldb$) (1 *)) b-%offset%)
           scale)
          tau1))
 (setf xnorm
         (abs
          (f2cl-lib:fref x-%data% (1 1) ((1 ldx) (1 *)) x-%offset%)))
 (go end_label)
label20
 (setf smin
         (max
```

```
         (* eps
            (max
             (abs
              (f2cl-lib:fref tl-%data% (1 1) ((1 ldtl) (1 *))
                             tl-%offset%))
             (abs
              (f2cl-lib:fref tr-%data% (1 1) ((1 ldtr) (1 *))
                             tr-%offset%))
             (abs
              (f2cl-lib:fref tr-%data% (1 2) ((1 ldtr) (1 *))
                             tr-%offset%))
             (abs
              (f2cl-lib:fref tr-%data% (2 1) ((1 ldtr) (1 *))
                             tr-%offset%))
             (abs
              (f2cl-lib:fref tr-%data% (2 2) ((1 ldtr) (1 *))
                             tr-%offset%))))
         smlnum))
(setf (f2cl-lib:fref tmp (1) ((1 4)))
        (+
         (f2cl-lib:fref tl-%data% (1 1) ((1 ldtl) (1 *)) tl-%offset%)
         (* sgn
            (f2cl-lib:fref tr-%data% (1 1) ((1 ldtr) (1 *))
                           tr-%offset%))))
(setf (f2cl-lib:fref tmp (4) ((1 4)))
        (+
         (f2cl-lib:fref tl-%data% (1 1) ((1 ldtl) (1 *)) tl-%offset%)
         (* sgn
            (f2cl-lib:fref tr-%data% (2 2) ((1 ldtr) (1 *))
                           tr-%offset%))))
(cond
 (ltranr
  (setf (f2cl-lib:fref tmp (2) ((1 4)))
          (* sgn
             (f2cl-lib:fref tr-%data% (2 1) ((1 ldtr) (1 *))
                            tr-%offset%)))
  (setf (f2cl-lib:fref tmp (3) ((1 4)))
          (* sgn
             (f2cl-lib:fref tr-%data% (1 2) ((1 ldtr) (1 *))
                            tr-%offset%))))
 (t
  (setf (f2cl-lib:fref tmp (2) ((1 4)))
          (* sgn
             (f2cl-lib:fref tr-%data% (1 2) ((1 ldtr) (1 *))
                            tr-%offset%)))
  (setf (f2cl-lib:fref tmp (3) ((1 4)))
          (* sgn
             (f2cl-lib:fref tr-%data% (2 1) ((1 ldtr) (1 *))
                            tr-%offset%)))))
(setf (f2cl-lib:fref btmp (1) ((1 4)))
```

```
        (f2cl-lib:fref b-%data% (1 1) ((1 ldb$) (1 *)) b-%offset%))
 (setf (f2cl-lib:fref btmp (2) ((1 4)))
        (f2cl-lib:fref b-%data% (1 2) ((1 ldb$) (1 *)) b-%offset%))
 (go label40)
label30
 (setf smin
        (max
         (* eps
            (max
             (abs
              (f2cl-lib:fref tr-%data% (1 1) ((1 ldtr) (1 *))
                             tr-%offset%))
             (abs
              (f2cl-lib:fref tl-%data% (1 1) ((1 ldtl) (1 *))
                             tl-%offset%))
             (abs
              (f2cl-lib:fref tl-%data% (1 2) ((1 ldtl) (1 *))
                             tl-%offset%))
             (abs
              (f2cl-lib:fref tl-%data% (2 1) ((1 ldtl) (1 *))
                             tl-%offset%))
             (abs
              (f2cl-lib:fref tl-%data% (2 2) ((1 ldtl) (1 *))
                             tl-%offset%))))
         smlnum))
 (setf (f2cl-lib:fref tmp (1) ((1 4)))
        (+
         (f2cl-lib:fref tl-%data% (1 1) ((1 ldtl) (1 *)) tl-%offset%)
         (* sgn
            (f2cl-lib:fref tr-%data% (1 1) ((1 ldtr) (1 *))
                           tr-%offset%))))
 (setf (f2cl-lib:fref tmp (4) ((1 4)))
        (+
         (f2cl-lib:fref tl-%data% (2 2) ((1 ldtl) (1 *)) tl-%offset%)
         (* sgn
            (f2cl-lib:fref tr-%data% (1 1) ((1 ldtr) (1 *))
                           tr-%offset%))))
 (cond
  (ltranl
   (setf (f2cl-lib:fref tmp (2) ((1 4)))
          (f2cl-lib:fref tl-%data% (1 2) ((1 ldtl) (1 *))
                         tl-%offset%))
   (setf (f2cl-lib:fref tmp (3) ((1 4)))
          (f2cl-lib:fref tl-%data% (2 1) ((1 ldtl) (1 *))
                         tl-%offset%)))
  (t
   (setf (f2cl-lib:fref tmp (2) ((1 4)))
          (f2cl-lib:fref tl-%data% (2 1) ((1 ldtl) (1 *))
                         tl-%offset%))
   (setf (f2cl-lib:fref tmp (3) ((1 4)))
```

```
          (f2cl-lib:fref tl-%data% (1 2) ((1 ldtl) (1 *))
                         tl-%offset%))))
 (setf (f2cl-lib:fref btmp (1) ((1 4)))
        (f2cl-lib:fref b-%data% (1 1) ((1 ldb$) (1 *)) b-%offset%))
 (setf (f2cl-lib:fref btmp (2) ((1 4)))
        (f2cl-lib:fref b-%data% (2 1) ((1 ldb$) (1 *)) b-%offset%))
label40
 (setf ipiv (idamax 4 tmp 1))
 (setf u11 (f2cl-lib:fref tmp (ipiv) ((1 4))))
 (cond ((<= (abs u11) smin) (setf info 1) (setf u11 smin)))
 (setf u12
        (f2cl-lib:fref tmp ((f2cl-lib:fref locu12 (ipiv) ((1 4))))
                       ((1 4))))
 (setf l21
        (/
         (f2cl-lib:fref tmp ((f2cl-lib:fref locl21 (ipiv) ((1 4))))
                        ((1 4)))
         u11))
 (setf u22
        (-
         (f2cl-lib:fref tmp ((f2cl-lib:fref locu22 (ipiv) ((1 4))))
                        ((1 4)))
         (* u12 l21)))
 (setf xswap (f2cl-lib:fref xswpiv (ipiv) ((1 4))))
 (setf bswap (f2cl-lib:fref bswpiv (ipiv) ((1 4))))
 (cond ((<= (abs u22) smin) (setf info 1) (setf u22 smin)))
 (cond
  (bswap (setf temp (f2cl-lib:fref btmp (2) ((1 4))))
   (setf (f2cl-lib:fref btmp (2) ((1 4)))
          (- (f2cl-lib:fref btmp (1) ((1 4))) (* l21 temp)))
   (setf (f2cl-lib:fref btmp (1) ((1 4))) temp))
  (t
   (setf (f2cl-lib:fref btmp (2) ((1 4)))
          (- (f2cl-lib:fref btmp (2) ((1 4)))
             (* l21 (f2cl-lib:fref btmp (1) ((1 4)))))))
 (setf scale one)
 (cond
  ((or
    (> (* two smlnum (abs (f2cl-lib:fref btmp (2) ((1 4)))))
       (abs u22))
    (> (* two smlnum (abs (f2cl-lib:fref btmp (1) ((1 4)))))
       (abs u11)))
   (setf scale
          (/ half
             (max (abs (f2cl-lib:fref btmp (1) ((1 4))))
                  (abs (f2cl-lib:fref btmp (2) ((1 4)))))))
   (setf (f2cl-lib:fref btmp (1) ((1 4)))
          (* (f2cl-lib:fref btmp (1) ((1 4))) scale))
   (setf (f2cl-lib:fref btmp (2) ((1 4)))
          (* (f2cl-lib:fref btmp (2) ((1 4))) scale))))
```

```
 (setf (f2cl-lib:fref x2 (2) ((1 2)))
        (/ (f2cl-lib:fref btmp (2) ((1 4))) u22))
 (setf (f2cl-lib:fref x2 (1) ((1 2)))
        (- (/ (f2cl-lib:fref btmp (1) ((1 4))) u11)
           (* (/ u12 u11) (f2cl-lib:fref x2 (2) ((1 2))))))
 (cond
  (xswap (setf temp (f2cl-lib:fref x2 (2) ((1 2))))
   (setf (f2cl-lib:fref x2 (2) ((1 2)))
         (f2cl-lib:fref x2 (1) ((1 2))))
   (setf (f2cl-lib:fref x2 (1) ((1 2))) temp)))
 (setf (f2cl-lib:fref x-%data% (1 1) ((1 ldx) (1 *)) x-%offset%)
        (f2cl-lib:fref x2 (1) ((1 2))))
 (cond
  ((= n1 1)
   (setf (f2cl-lib:fref x-%data% (1 2) ((1 ldx) (1 *)) x-%offset%)
         (f2cl-lib:fref x2 (2) ((1 2))))
   (setf xnorm
         (+
          (abs
           (f2cl-lib:fref x-%data% (1 1) ((1 ldx) (1 *))
                          x-%offset%))
          (abs
           (f2cl-lib:fref x-%data% (1 2) ((1 ldx) (1 *))
                          x-%offset%)))))
  (t
   (setf (f2cl-lib:fref x-%data% (2 1) ((1 ldx) (1 *)) x-%offset%)
         (f2cl-lib:fref x2 (2) ((1 2))))
   (setf xnorm
         (max
          (abs
           (f2cl-lib:fref x-%data% (1 1) ((1 ldx) (1 *))
                          x-%offset%))
          (abs
           (f2cl-lib:fref x-%data% (2 1) ((1 ldx) (1 *))
                          x-%offset%))))))
 (go end_label)
label50
 (setf smin
        (max
         (abs
          (f2cl-lib:fref tr-%data% (1 1) ((1 ldtr) (1 *))
                         tr-%offset%))
         (abs
          (f2cl-lib:fref tr-%data% (1 2) ((1 ldtr) (1 *))
                         tr-%offset%))
         (abs
          (f2cl-lib:fref tr-%data% (2 1) ((1 ldtr) (1 *))
                         tr-%offset%))
         (abs
          (f2cl-lib:fref tr-%data% (2 2) ((1 ldtr) (1 *))
```

```
                        tr-%offset%))))
(setf smin
        (max smin
             (abs
              (f2cl-lib:fref tl-%data% (1 1) ((1 ldtl) (1 *))
                             tl-%offset%))
             (abs
              (f2cl-lib:fref tl-%data% (1 2) ((1 ldtl) (1 *))
                             tl-%offset%))
             (abs
              (f2cl-lib:fref tl-%data% (2 1) ((1 ldtl) (1 *))
                             tl-%offset%))
             (abs
              (f2cl-lib:fref tl-%data% (2 2) ((1 ldtl) (1 *))
                             tl-%offset%))))
(setf smin (max (* eps smin) smlnum))
(setf (f2cl-lib:fref btmp (1) ((1 4))) zero)
(dcopy 16 btmp 0 t16 1)
(setf (f2cl-lib:fref t16 (1 1) ((1 4) (1 4)))
        (+
         (f2cl-lib:fref tl-%data% (1 1) ((1 ldtl) (1 *)) tl-%offset%)
         (* sgn
            (f2cl-lib:fref tr-%data% (1 1) ((1 ldtr) (1 *))
                           tr-%offset%))))
(setf (f2cl-lib:fref t16 (2 2) ((1 4) (1 4)))
        (+
         (f2cl-lib:fref tl-%data% (2 2) ((1 ldtl) (1 *)) tl-%offset%)
         (* sgn
            (f2cl-lib:fref tr-%data% (1 1) ((1 ldtr) (1 *))
                           tr-%offset%))))
(setf (f2cl-lib:fref t16 (3 3) ((1 4) (1 4)))
        (+
         (f2cl-lib:fref tl-%data% (1 1) ((1 ldtl) (1 *)) tl-%offset%)
         (* sgn
            (f2cl-lib:fref tr-%data% (2 2) ((1 ldtr) (1 *))
                           tr-%offset%))))
(setf (f2cl-lib:fref t16 (4 4) ((1 4) (1 4)))
        (+
         (f2cl-lib:fref tl-%data% (2 2) ((1 ldtl) (1 *)) tl-%offset%)
         (* sgn
            (f2cl-lib:fref tr-%data% (2 2) ((1 ldtr) (1 *))
                           tr-%offset%))))
(cond
 (ltranl
  (setf (f2cl-lib:fref t16 (1 2) ((1 4) (1 4)))
          (f2cl-lib:fref tl-%data% (2 1) ((1 ldtl) (1 *))
                         tl-%offset%))
  (setf (f2cl-lib:fref t16 (2 1) ((1 4) (1 4)))
          (f2cl-lib:fref tl-%data% (1 2) ((1 ldtl) (1 *))
                         tl-%offset%))
```

```
    (setf (f2cl-lib:fref t16 (3 4) ((1 4) (1 4)))
            (f2cl-lib:fref tl-%data% (2 1) ((1 ldtl) (1 *))
                           tl-%offset%))
    (setf (f2cl-lib:fref t16 (4 3) ((1 4) (1 4)))
            (f2cl-lib:fref tl-%data% (1 2) ((1 ldtl) (1 *))
                           tl-%offset%)))
   (t
    (setf (f2cl-lib:fref t16 (1 2) ((1 4) (1 4)))
            (f2cl-lib:fref tl-%data% (1 2) ((1 ldtl) (1 *))
                           tl-%offset%))
    (setf (f2cl-lib:fref t16 (2 1) ((1 4) (1 4)))
            (f2cl-lib:fref tl-%data% (2 1) ((1 ldtl) (1 *))
                           tl-%offset%))
    (setf (f2cl-lib:fref t16 (3 4) ((1 4) (1 4)))
            (f2cl-lib:fref tl-%data% (1 2) ((1 ldtl) (1 *))
                           tl-%offset%))
    (setf (f2cl-lib:fref t16 (4 3) ((1 4) (1 4)))
            (f2cl-lib:fref tl-%data% (2 1) ((1 ldtl) (1 *))
                           tl-%offset%))))
  (cond
   (ltranr
    (setf (f2cl-lib:fref t16 (1 3) ((1 4) (1 4)))
            (* sgn
               (f2cl-lib:fref tr-%data% (1 2) ((1 ldtr) (1 *))
                              tr-%offset%)))
    (setf (f2cl-lib:fref t16 (2 4) ((1 4) (1 4)))
            (* sgn
               (f2cl-lib:fref tr-%data% (1 2) ((1 ldtr) (1 *))
                              tr-%offset%)))
    (setf (f2cl-lib:fref t16 (3 1) ((1 4) (1 4)))
            (* sgn
               (f2cl-lib:fref tr-%data% (2 1) ((1 ldtr) (1 *))
                              tr-%offset%)))
    (setf (f2cl-lib:fref t16 (4 2) ((1 4) (1 4)))
            (* sgn
               (f2cl-lib:fref tr-%data% (2 1) ((1 ldtr) (1 *))
                              tr-%offset%))))
   (t
    (setf (f2cl-lib:fref t16 (1 3) ((1 4) (1 4)))
            (* sgn
               (f2cl-lib:fref tr-%data% (2 1) ((1 ldtr) (1 *))
                              tr-%offset%)))
    (setf (f2cl-lib:fref t16 (2 4) ((1 4) (1 4)))
            (* sgn
               (f2cl-lib:fref tr-%data% (2 1) ((1 ldtr) (1 *))
                              tr-%offset%)))
    (setf (f2cl-lib:fref t16 (3 1) ((1 4) (1 4)))
            (* sgn
               (f2cl-lib:fref tr-%data% (1 2) ((1 ldtr) (1 *))
                              tr-%offset%)))
```

```
  (setf (f2cl-lib:fref t16 (4 2) ((1 4) (1 4)))
          (* sgn
             (f2cl-lib:fref tr-%data% (1 2) ((1 ldtr) (1 *))
                            tr-%offset%)))))
(setf (f2cl-lib:fref btmp (1) ((1 4)))
        (f2cl-lib:fref b-%data% (1 1) ((1 ldb$) (1 *)) b-%offset%))
(setf (f2cl-lib:fref btmp (2) ((1 4)))
        (f2cl-lib:fref b-%data% (2 1) ((1 ldb$) (1 *)) b-%offset%))
(setf (f2cl-lib:fref btmp (3) ((1 4)))
        (f2cl-lib:fref b-%data% (1 2) ((1 ldb$) (1 *)) b-%offset%))
(setf (f2cl-lib:fref btmp (4) ((1 4)))
        (f2cl-lib:fref b-%data% (2 2) ((1 ldb$) (1 *)) b-%offset%))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i 3) nil)
  (tagbody
    (setf xmax zero)
    (f2cl-lib:fdo (ip i (f2cl-lib:int-add ip 1))
                  ((> ip 4) nil)
      (tagbody
        (f2cl-lib:fdo (jp i (f2cl-lib:int-add jp 1))
                      ((> jp 4) nil)
          (tagbody
            (cond
             ((>= (abs (f2cl-lib:fref t16 (ip jp) ((1 4) (1 4))))
                  xmax)
              (setf xmax
                      (abs
                       (f2cl-lib:fref t16 (ip jp) ((1 4) (1 4)))))
              (setf ipsv ip) (setf jpsv jp)))
           label60))
       label70))
    (cond
     ((/= ipsv i)
      (dswap 4
       (f2cl-lib:array-slice t16 double-float (ipsv 1) ((1 4) (1 4)))
       4 (f2cl-lib:array-slice t16 double-float (i 1) ((1 4) (1 4)))
       4)
      (setf temp (f2cl-lib:fref btmp (i) ((1 4))))
      (setf (f2cl-lib:fref btmp (i) ((1 4)))
              (f2cl-lib:fref btmp (ipsv) ((1 4))))
      (setf (f2cl-lib:fref btmp (ipsv) ((1 4))) temp)))
    (if (/= jpsv i)
        (dswap 4
         (f2cl-lib:array-slice t16 double-float (1 jpsv)
                               ((1 4) (1 4)))
         1
         (f2cl-lib:array-slice t16 double-float (1 i) ((1 4) (1 4)))
         1))
    (setf (f2cl-lib:fref jpiv (i) ((1 4))) jpsv)
    (cond
```

```
      ((< (abs (f2cl-lib:fref t16 (i i) ((1 4) (1 4)))) smin)
       (setf info 1)
       (setf (f2cl-lib:fref t16 (i i) ((1 4) (1 4))) smin)))
     (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                   ((> j 4) nil)
       (tagbody
         (setf (f2cl-lib:fref t16 (j i) ((1 4) (1 4)))
                 (/ (f2cl-lib:fref t16 (j i) ((1 4) (1 4)))
                    (f2cl-lib:fref t16 (i i) ((1 4) (1 4)))))
         (setf (f2cl-lib:fref btmp (j) ((1 4)))
                 (- (f2cl-lib:fref btmp (j) ((1 4)))
                    (* (f2cl-lib:fref t16 (j i) ((1 4) (1 4)))
                       (f2cl-lib:fref btmp (i) ((1 4))))))
         (f2cl-lib:fdo (k (f2cl-lib:int-add i 1)
                        (f2cl-lib:int-add k 1))
                       ((> k 4) nil)
           (tagbody
             (setf (f2cl-lib:fref t16 (j k) ((1 4) (1 4)))
                     (- (f2cl-lib:fref t16 (j k) ((1 4) (1 4)))
                        (* (f2cl-lib:fref t16 (j i) ((1 4) (1 4)))
                           (f2cl-lib:fref t16 (i k) ((1 4) (1 4))))))
            label80))
        label90))
   label100))
(if (< (abs (f2cl-lib:fref t16 (4 4) ((1 4) (1 4)))) smin)
    (setf (f2cl-lib:fref t16 (4 4) ((1 4) (1 4))) smin))
(setf scale one)
(cond
 ((or
   (> (* eight smlnum (abs (f2cl-lib:fref btmp (1) ((1 4)))))
      (abs (f2cl-lib:fref t16 (1 1) ((1 4) (1 4)))))
   (> (* eight smlnum (abs (f2cl-lib:fref btmp (2) ((1 4)))))
      (abs (f2cl-lib:fref t16 (2 2) ((1 4) (1 4)))))
   (> (* eight smlnum (abs (f2cl-lib:fref btmp (3) ((1 4)))))
      (abs (f2cl-lib:fref t16 (3 3) ((1 4) (1 4)))))
   (> (* eight smlnum (abs (f2cl-lib:fref btmp (4) ((1 4)))))
      (abs (f2cl-lib:fref t16 (4 4) ((1 4) (1 4))))))
  (setf scale
          (/ (/ one eight)
             (max (abs (f2cl-lib:fref btmp (1) ((1 4))))
                  (abs (f2cl-lib:fref btmp (2) ((1 4))))
                  (abs (f2cl-lib:fref btmp (3) ((1 4))))
                  (abs (f2cl-lib:fref btmp (4) ((1 4)))))))
  (setf (f2cl-lib:fref btmp (1) ((1 4)))
          (* (f2cl-lib:fref btmp (1) ((1 4))) scale))
  (setf (f2cl-lib:fref btmp (2) ((1 4)))
          (* (f2cl-lib:fref btmp (2) ((1 4))) scale))
  (setf (f2cl-lib:fref btmp (3) ((1 4)))
          (* (f2cl-lib:fref btmp (3) ((1 4))) scale))
  (setf (f2cl-lib:fref btmp (4) ((1 4)))
```

```
          (* (f2cl-lib:fref btmp (4) ((1 4))) scale))))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i 4) nil)
  (tagbody
    (setf k (f2cl-lib:int-sub 5 i))
    (setf temp (/ one (f2cl-lib:fref t16 (k k) ((1 4) (1 4)))))
    (setf (f2cl-lib:fref tmp (k) ((1 4)))
            (* (f2cl-lib:fref btmp (k) ((1 4))) temp))
    (f2cl-lib:fdo (j (f2cl-lib:int-add k 1) (f2cl-lib:int-add j 1))
                  ((> j 4) nil)
      (tagbody
        (setf (f2cl-lib:fref tmp (k) ((1 4)))
                (- (f2cl-lib:fref tmp (k) ((1 4)))
                   (* temp (f2cl-lib:fref t16 (k j) ((1 4) (1 4)))
                      (f2cl-lib:fref tmp (j) ((1 4))))))
       label110))
   label120))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i 3) nil)
  (tagbody
    (cond
     ((/=
       (f2cl-lib:fref jpiv
                      ((f2cl-lib:int-add 4 (f2cl-lib:int-sub i)))
                      ((1 4)))
       (f2cl-lib:int-add 4 (f2cl-lib:int-sub i)))
      (setf temp
              (f2cl-lib:fref tmp ((f2cl-lib:int-sub 4 i)) ((1 4))))
      (setf (f2cl-lib:fref tmp ((f2cl-lib:int-sub 4 i)) ((1 4)))
              (f2cl-lib:fref tmp
                             ((f2cl-lib:fref jpiv
                                             ((f2cl-lib:int-sub 4 i))
                                             ((1 4))))
                             ((1 4))))
      (setf (f2cl-lib:fref tmp
                           ((f2cl-lib:fref jpiv
                                           ((f2cl-lib:int-sub 4 i))
                                           ((1 4))))
                           ((1 4)))
              temp)))
   label130))
(setf (f2cl-lib:fref x-%data% (1 1) ((1 ldx) (1 *)) x-%offset%)
        (f2cl-lib:fref tmp (1) ((1 4))))
(setf (f2cl-lib:fref x-%data% (2 1) ((1 ldx) (1 *)) x-%offset%)
        (f2cl-lib:fref tmp (2) ((1 4))))
(setf (f2cl-lib:fref x-%data% (1 2) ((1 ldx) (1 *)) x-%offset%)
        (f2cl-lib:fref tmp (3) ((1 4))))
(setf (f2cl-lib:fref x-%data% (2 2) ((1 ldx) (1 *)) x-%offset%)
        (f2cl-lib:fref tmp (4) ((1 4))))
(setf xnorm
```

```
                (max
                 (+ (abs (f2cl-lib:fref tmp (1) ((1 4))))
                    (abs (f2cl-lib:fref tmp (3) ((1 4)))))
                 (+ (abs (f2cl-lib:fref tmp (2) ((1 4))))
                    (abs (f2cl-lib:fref tmp (4) ((1 4)))))))
        (go end_label)
       end_label
        (return
         (values nil nil nil nil nil nil nil nil nil nil nil scale nil nil
                 xnorm info)))))))
```

———-

90.194 dorg2r

90.194.1 Axiom unit tests

— **dorg2r.input** —

```
)set break resume
)sys rm -f dorg2r.output
)spool dorg2r.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.194.2 Axiom help page

— **dorg2r.help** —

```
======================================================================
dorg2r examples
======================================================================

======================================================================
Man Page Details
======================================================================
```

```
NAME
       DORG2R - an m by n real matrix Q with orthonormal columns,

SYNOPSIS
       SUBROUTINE DORG2R( M, N, K, A, LDA, TAU, WORK, INFO )

           INTEGER        INFO, K, LDA, M, N

           DOUBLE         PRECISION A( LDA, * ), TAU( * ), WORK( * )

  Purpose
  =======

  DORG2R generates an m by n real matrix Q with orthonormal columns,
  which is defined as the first n columns of a product of k elementary
  reflectors of order m

        Q  =  H(1) H(2) . . . H(k)

  as returned by DGEQRF.

  Arguments
  =========

  M       (input) INTEGER
          The number of rows of the matrix Q. M >= 0.

  N       (input) INTEGER
          The number of columns of the matrix Q. M >= N >= 0.

  K       (input) INTEGER
          The number of elementary reflectors whose product defines the
          matrix Q. N >= K >= 0.

  A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
          On entry, the i-th column must contain the vector which
          defines the elementary reflector H(i), for i = 1,2,...,k, as
          returned by DGEQRF in the first k columns of its array
          argument A.
          On exit, the m-by-n matrix Q.

  LDA     (input) INTEGER
          The first dimension of the array A. LDA >= max(1,M).

  TAU     (input) DOUBLE PRECISION array, dimension (K)
          TAU(i) must contain the scalar factor of the elementary
          reflector H(i), as returned by DGEQRF.

  WORK    (workspace) DOUBLE PRECISION array, dimension (N)
```

```
INFO    (output) INTEGER
        = 0: successful exit
        < 0: if INFO = -i, the i-th argument has an illegal value
```

———-

90.194.3 fortran code

— **dorg2r.f** —

```
      SUBROUTINE DORG2R( M, N, K, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     February 29, 1992
*
*     .. Scalar Arguments ..
      INTEGER            INFO, K, LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, L
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARF, DSCAL, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 .OR. N.GT.M ) THEN
```

```
         INFO = -2
      ELSE IF( K.LT.0 .OR. K.GT.N ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DORG2R', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.LE.0 )
     $   RETURN
*
*     Initialise columns k+1:n to columns of the unit matrix
*
      DO 20 J = K + 1, N
         DO 10 L = 1, M
            A( L, J ) = ZERO
   10    CONTINUE
         A( J, J ) = ONE
   20 CONTINUE
*
      DO 40 I = K, 1, -1
*
*        Apply H(i) to A(i:m,i:n) from the left
*
         IF( I.LT.N ) THEN
            A( I, I ) = ONE
            CALL DLARF( 'Left', M-I+1, N-I, A( I, I ), 1, TAU( I ),
     $                  A( I, I+1 ), LDA, WORK )
         END IF
         IF( I.LT.M )
     $      CALL DSCAL( M-I, -TAU( I ), A( I+1, I ), 1 )
         A( I, I ) = ONE - TAU( I )
*
*        Set A(1:i-1,i) to zero
*
         DO 30 L = 1, I - 1
            A( L, I ) = ZERO
   30    CONTINUE
   40 CONTINUE
      RETURN
*
*     End of DORG2R
*
      END
```

———-

90.194.4 lisp code

— **dorg2r.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun dorg2r (m n k a lda tau work info)
    (declare (type (array double-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lda k n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((i 0) (j 0) (l 0))
        (declare (type (f2cl-lib:integer4) i j l))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((or (< n 0) (> n m)) (setf info -2))
              ((or (< k 0) (> k n)) (setf info -3))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -5)))
        (cond
         ((/= info 0) (xerbla "DORG2R" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (<= n 0)
            (go end_label))
        (f2cl-lib:fdo (j (f2cl-lib:int-add k 1) (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                          ((> l m) nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data% (l j) ((1 lda) (1 *)) a-%offset%)
                        zero)
               label10))
            (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
                    one)
           label20))
        (f2cl-lib:fdo (i k (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (cond
             ((< i n)
              (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
```

```
                one)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (dlarf "Left" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
               (f2cl-lib:int-sub n i)
               (f2cl-lib:array-slice a-%data% double-float (i i)
                                     ((1 lda) (1 *)) a-%offset%)
               1 (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%)
               (f2cl-lib:array-slice a-%data% double-float
                                     (i (f2cl-lib:int-add i 1))
                                     ((1 lda) (1 *)) a-%offset%)
               lda work)
            (declare
             (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
            (setf lda var-7))))
        (if (< i m)
            (dscal (f2cl-lib:int-sub m i)
             (- (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
             (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                                   ((1 lda) (1 *)) a-%offset%)
             1))
        (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                (- one
                   (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%)))
        (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                      ((> l (f2cl-lib:int-add i (f2cl-lib:int-sub 1))) nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data% (l i) ((1 lda) (1 *)) a-%offset%)
                    zero)
           label30))
       label40))
    (go end_label)
   end_label
    (return (values nil nil nil nil lda nil nil info))))))
```

———-

90.195 dorgbr

90.195.1 Axiom unit tests

— **dorgbr.input** —

```
)set break resume
)sys rm -f dorgbr.output
)spool dorgbr.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.195.2 Axiom help page

— **dorgbr.help** —

```
==================================================================
dorgbr examples
==================================================================

==================================================================
Man Page Details
==================================================================

NAME
       DORGBR  -  one  of the real orthogonal matrices Q or P**T determined by
       DGEBRD when reducing a real matrix A to bidiagonal form

SYNOPSIS
       SUBROUTINE DORGBR( VECT, M, N, K, A, LDA, TAU, WORK, LWORK, INFO )

           CHARACTER      VECT

           INTEGER        INFO, K, LDA, LWORK, M, N

           DOUBLE         PRECISION A( LDA, * ), TAU( * ), WORK( * )

  Purpose
  =======

  DORGBR generates one of the real orthogonal matrices Q or P**T
  determined by DGEBRD when reducing a real matrix A to bidiagonal
  form: A = Q * B * P**T.  Q and P**T are defined as products of
  elementary reflectors H(i) or G(i) respectively.

  If VECT = 'Q', A is assumed to have been an M-by-K matrix, and Q
  is of order M:
  if m >= k, Q = H(1) H(2) . . . H(k) and DORGBR returns the first n
  columns of Q, where m >= n >= k;
  if m < k, Q = H(1) H(2) . . . H(m-1) and DORGBR returns Q as an
  M-by-M matrix.
```

```
If VECT = 'P', A is assumed to have been a K-by-N matrix, and P**T
is of order N:
if k < n, P**T = G(k) . . . G(2) G(1) and DORGBR returns the first m
rows of P**T, where n >= m >= k;
if k >= n, P**T = G(n-1) . . . G(2) G(1) and DORGBR returns P**T as
an N-by-N matrix.

Arguments
=========

VECT    (input) CHARACTER*1
        Specifies whether the matrix Q or the matrix P**T is
        required, as defined in the transformation applied by DGEBRD:
        = 'Q':  generate Q;
        = 'P':  generate P**T.

M       (input) INTEGER
        The number of rows of the matrix Q or P**T to be returned.
        M >= 0.

N       (input) INTEGER
        The number of columns of the matrix Q or P**T to be returned.
        N >= 0.
        If VECT = 'Q', M >= N >= min(M,K);
        if VECT = 'P', N >= M >= min(N,K).

K       (input) INTEGER
        If VECT = 'Q', the number of columns in the original M-by-K
        matrix reduced by DGEBRD.
        If VECT = 'P', the number of rows in the original K-by-N
        matrix reduced by DGEBRD.
        K >= 0.

A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
        On entry, the vectors which define the elementary reflectors,
        as returned by DGEBRD.
        On exit, the M-by-N matrix Q or P**T.

LDA     (input) INTEGER
        The leading dimension of the array A. LDA >= max(1,M).

TAU     (input) DOUBLE PRECISION array, dimension
                              (min(M,K)) if VECT = 'Q'
                              (min(N,K)) if VECT = 'P'
        TAU(i) must contain the scalar factor of the elementary
        reflector H(i) or G(i), which determines Q or P**T, as
        returned by DGEBRD in its array argument TAUQ or TAUP.

WORK    (workspace/output) DOUBLE PRECISION array, dimension (LWORK)
```

```
         On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK   (input) INTEGER
         The dimension of the array WORK. LWORK >= max(1,min(M,N)).
         For optimum performance LWORK >= min(M,N)*NB, where NB
         is the optimal blocksize.

         If LWORK = -1, then a workspace query is assumed; the routine
         only calculates the optimal size of the WORK array, returns
         this value as the first entry of the WORK array, and no error
         message related to LWORK is issued by XERBLA.

 INFO    (output) INTEGER
         = 0:  successful exit
         < 0:  if INFO = -i, the i-th argument had an illegal value
```

90.195.3 fortran code

— dorgbr.f —

```
      SUBROUTINE DORGBR( VECT, M, N, K, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      CHARACTER          VECT
      INTEGER            INFO, K, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, WANTQ
      INTEGER            I, IINFO, J, LWKOPT, MN, NB
*     ..
```

```
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DORGLQ, DORGQR, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      WANTQ = LSAME( VECT, 'Q' )
      MN = MIN( M, N )
      LQUERY = ( LWORK.EQ.-1 )
      IF( .NOT.WANTQ .AND. .NOT.LSAME( VECT, 'P' ) ) THEN
         INFO = -1
      ELSE IF( M.LT.0 ) THEN
         INFO = -2
      ELSE IF( N.LT.0 .OR. ( WANTQ .AND. ( N.GT.M .OR. N.LT.MIN( M,
     $         K ) ) ) .OR. ( .NOT.WANTQ .AND. ( M.GT.N .OR. M.LT.
     $         MIN( N, K ) ) ) ) THEN
         INFO = -3
      ELSE IF( K.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -6
      ELSE IF( LWORK.LT.MAX( 1, MN ) .AND. .NOT.LQUERY ) THEN
         INFO = -9
      END IF
*
      IF( INFO.EQ.0 ) THEN
         IF( WANTQ ) THEN
            NB = ILAENV( 1, 'DORGQR', ' ', M, N, K, -1 )
         ELSE
            NB = ILAENV( 1, 'DORGLQ', ' ', M, N, K, -1 )
         END IF
         LWKOPT = MAX( 1, MN )*NB
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DORGBR', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
```

```
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      IF( WANTQ ) THEN
*
*        Form Q, determined by a call to DGEBRD to reduce an m-by-k
*        matrix
*
         IF( M.GE.K ) THEN
*
*           If m >= k, assume m >= n >= k
*
            CALL DORGQR( M, N, K, A, LDA, TAU, WORK, LWORK, IINFO )
*
         ELSE
*
*           If m < k, assume m = n
*
*           Shift the vectors which define the elementary reflectors one
*           column to the right, and set the first row and column of Q
*           to those of the unit matrix
*
            DO 20 J = M, 2, -1
               A( 1, J ) = ZERO
               DO 10 I = J + 1, M
                  A( I, J ) = A( I, J-1 )
   10          CONTINUE
   20       CONTINUE
            A( 1, 1 ) = ONE
            DO 30 I = 2, M
               A( I, 1 ) = ZERO
   30       CONTINUE
            IF( M.GT.1 ) THEN
*
*              Form Q(2:m,2:m)
*
               CALL DORGQR( M-1, M-1, M-1, A( 2, 2 ), LDA, TAU, WORK,
     $                      LWORK, IINFO )
            END IF
         END IF
      ELSE
*
*        Form P', determined by a call to DGEBRD to reduce a k-by-n
*        matrix
```

```
*
         IF( K.LT.N ) THEN
*
*           If k < n, assume k <= m <= n
*
            CALL DORGLQ( M, N, K, A, LDA, TAU, WORK, LWORK, IINFO )
*
         ELSE
*
*           If k >= n, assume m = n
*
*           Shift the vectors which define the elementary reflectors one
*           row downward, and set the first row and column of P' to
*           those of the unit matrix
*
            A( 1, 1 ) = ONE
            DO 40 I = 2, N
               A( I, 1 ) = ZERO
   40       CONTINUE
            DO 60 J = 2, N
               DO 50 I = J - 1, 2, -1
                  A( I, J ) = A( I-1, J )
   50          CONTINUE
               A( 1, J ) = ZERO
   60       CONTINUE
            IF( N.GT.1 ) THEN
*
*              Form P'(2:n,2:n)
*
               CALL DORGLQ( N-1, N-1, N-1, A( 2, 2 ), LDA, TAU, WORK,
     $                       LWORK, IINFO )
            END IF
         END IF
      END IF
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of DORGBR
*
      END
```

———-

90.195.4 lisp code

— dorgbr.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dorgbr (vect m n k a lda tau work lwork info)
    (declare (type (array double-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lwork lda k n m)
             (type (string *) vect))
    (f2cl-lib:with-multi-array-data
        ((vect character vect-%data% vect-%offset%)
         (a double-float a-%data% a-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((i 0) (iinfo 0) (j 0) (lwkopt 0) (mn 0) (lquery nil) (wantq nil))
        (declare (type (f2cl-lib:integer4) i iinfo j lwkopt mn)
                 (type f2cl-lib:logical lquery wantq))
        (setf info 0)
        (setf wantq
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame vect "Q")
                  (declare (ignore var-1))
                  (when var-0 (setf vect var-0))
                  ret-val))
        (setf mn (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond
         ((and (not wantq)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame vect "P")
                  (declare (ignore var-1))
                  (when var-0 (setf vect var-0))
                  ret-val)))
          (setf info -1))
         ((< m 0) (setf info -2))
         ((or (< n 0)
              (and wantq
                   (or (> n m)
                       (< n
                          (min (the f2cl-lib:integer4 m)
                               (the f2cl-lib:integer4 k)))))
              (and (not wantq)
                   (or (> m n)
                       (< m
                          (min (the f2cl-lib:integer4 n)
                               (the f2cl-lib:integer4 k))))))
          (setf info -3))
         ((< k 0) (setf info -4))
         ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
          (setf info -6))
```

```
 ((and
   (< lwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 mn)))
   (not lquery))
  (setf info -9)))
(cond
 ((= info 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'double-float))
  (cond
   (wantq
    (cond
     ((>= m k)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dorgqr m n k a lda tau work -1 iinfo)
        (declare (ignore var-3 var-5 var-6 var-7))
        (when var-0 (setf m var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf k var-2))
        (when var-4 (setf lda var-4))
        (when var-8 (setf iinfo var-8))))
     (t
      (cond
       ((> m 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (dorgqr (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
             (f2cl-lib:int-sub m 1)
             (f2cl-lib:array-slice a-%data% double-float (2 2)
                                   ((1 lda) (1 *)) a-%offset%)
             lda tau work -1 iinfo)
          (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
          (when var-4 (setf lda var-4))
          (when var-8 (setf iinfo var-8))))))))
   (t
    (cond
     ((< k n)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (dorglq m n k a lda tau work -1 iinfo)
        (declare (ignore var-3 var-5 var-6 var-7))
        (when var-0 (setf m var-0))
        (when var-1 (setf n var-1))
        (when var-2 (setf k var-2))
        (when var-4 (setf lda var-4))
        (when var-8 (setf iinfo var-8))))
     (t
      (cond
       ((> n 1)
        (multiple-value-bind
```

```
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (dorglq (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
               (f2cl-lib:int-sub n 1)
               (f2cl-lib:array-slice a-%data% double-float (2 2)
                                     ((1 lda) (1 *)) a-%offset%)
               lda tau work -1 iinfo)
            (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-7))
            (when var-4 (setf lda var-4))
            (when var-8 (setf iinfo var-8)))))))))
  (setf lwkopt
          (f2cl-lib:int
           (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)))
  (setf lwkopt
          (max (the f2cl-lib:integer4 lwkopt)
               (the f2cl-lib:integer4 mn)))))
(cond
 ((/= info 0) (xerbla "DORGBR" (f2cl-lib:int-sub info)) (go end_label))
 (lquery
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
  (go end_label)))
(cond
 ((or (= m 0) (= n 0))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'double-float))
  (go end_label)))
(cond
 (wantq
  (cond
   ((>= m k)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dorgqr m n k a lda tau work lwork iinfo)
      (declare (ignore var-3 var-5 var-6))
      (when var-0 (setf m var-0))
      (when var-1 (setf n var-1))
      (when var-2 (setf k var-2))
      (when var-4 (setf lda var-4))
      (when var-7 (setf lwork var-7))
      (when var-8 (setf iinfo var-8))))
   (t
    (f2cl-lib:fdo (j m (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                  ((> j 2) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *)) a-%offset%)
                zero)
        (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
```

```
                                        a-%offset%)
                              (f2cl-lib:fref a-%data% (i (f2cl-lib:int-sub j 1))
                                             ((1 lda) (1 *)) a-%offset%))
                 label10))
            label20))
        (setf (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)
                one)
        (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data% (i 1) ((1 lda) (1 *)) a-%offset%)
                    zero)
            label30))
        (cond
         ((> m 1)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (dorgqr (f2cl-lib:int-sub m 1) (f2cl-lib:int-sub m 1)
               (f2cl-lib:int-sub m 1)
               (f2cl-lib:array-slice a-%data% double-float (2 2)
                                     ((1 lda) (1 *)) a-%offset%)
               lda tau work lwork iinfo)
            (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6))
            (when var-4 (setf lda var-4))
            (when var-7 (setf lwork var-7))
            (when var-8 (setf iinfo var-8))))))))
(t
 (cond
  ((< k n)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (dorglq m n k a lda tau work lwork iinfo)
     (declare (ignore var-3 var-5 var-6))
     (when var-0 (setf m var-0))
     (when var-1 (setf n var-1))
     (when var-2 (setf k var-2))
     (when var-4 (setf lda var-4))
     (when var-7 (setf lwork var-7))
     (when var-8 (setf iinfo var-8))))
  (t
   (setf (f2cl-lib:fref a-%data% (1 1) ((1 lda) (1 *)) a-%offset%)
           one)
   (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf (f2cl-lib:fref a-%data% (i 1) ((1 lda) (1 *)) a-%offset%)
               zero)
       label40))
   (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
```

```
            (tagbody
              (f2cl-lib:fdo (i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))
                             (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                            ((> i 2) nil)
                (tagbody
                  (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                       a-%offset%)
                        (f2cl-lib:fref a-%data% ((f2cl-lib:int-sub i 1) j)
                                       ((1 lda) (1 *)) a-%offset%))
                label50))
              (setf (f2cl-lib:fref a-%data% (1 j) ((1 lda) (1 *)) a-%offset%)
                    zero)
            label60))
          (cond
           ((> n 1)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (dorglq (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1)
                 (f2cl-lib:int-sub n 1)
                 (f2cl-lib:array-slice a-%data% double-float (2 2)
                                       ((1 lda) (1 *)) a-%offset%)
                 lda tau work lwork iinfo)
              (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6))
              (when var-4 (setf lda var-4))
              (when var-7 (setf lwork var-7))
              (when var-8 (setf iinfo var-8)))))))))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
      (go end_label)
     end_label
      (return (values vect m n k nil lda nil nil lwork info))))))
```

———-

90.196 dorghr

90.196.1 Axiom unit tests

— **dorghr.input** —

```
)set break resume
)sys rm -f dorghr.output
)spool dorghr.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

90.196.2 Axiom help page

— **dorghr.help** —

```
=====================================================================
dorghr examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

NAME
       DORGHR  - a real orthogonal matrix Q which is defined as the product of
       IHI-ILO elementary reflectors of order N, as returned by DGEHRD

SYNOPSIS
       SUBROUTINE DORGHR( N, ILO, IHI, A, LDA, TAU, WORK, LWORK, INFO )

           INTEGER        IHI, ILO, INFO, LDA, LWORK, N

           DOUBLE         PRECISION A( LDA, * ), TAU( * ), WORK( * )

  Purpose
  =======

  DORGHR generates a real orthogonal matrix Q which is defined as the
  product of IHI-ILO elementary reflectors of order N, as returned by
  DGEHRD:

  Q = H(ilo) H(ilo+1) . . . H(ihi-1).

  Arguments
  =========

  N       (input) INTEGER
          The order of the matrix Q. N >= 0.

  ILO     (input) INTEGER
  IHI     (input) INTEGER
          ILO and IHI must have the same values as in the previous call
          of DGEHRD. Q is equal to the unit matrix except in the
```

```
         submatrix Q(ilo+1:ihi,ilo+1:ihi).
         1 <= ILO <= IHI <= N, if N > 0; ILO=1 and IHI=0, if N=0.

 A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
         On entry, the vectors which define the elementary reflectors,
         as returned by DGEHRD.
         On exit, the N-by-N orthogonal matrix Q.

 LDA     (input) INTEGER
         The leading dimension of the array A. LDA >= max(1,N).

 TAU     (input) DOUBLE PRECISION array, dimension (N-1)
         TAU(i) must contain the scalar factor of the elementary
         reflector H(i), as returned by DGEHRD.

 WORK    (workspace/output) DOUBLE PRECISION array, dimension (LWORK)
         On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK   (input) INTEGER
         The dimension of the array WORK. LWORK >= IHI-ILO.
         For optimum performance LWORK >= (IHI-ILO)*NB, where NB is
         the optimal blocksize.

         If LWORK = -1, then a workspace query is assumed; the routine
         only calculates the optimal size of the WORK array, returns
         this value as the first entry of the WORK array, and no error
         message related to LWORK is issued by XERBLA.

 INFO    (output) INTEGER
         = 0:  successful exit
         < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

90.196.3 fortran code

— dorghr.f —

```
      SUBROUTINE DORGHR( N, ILO, IHI, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      INTEGER            IHI, ILO, INFO, LDA, LWORK, N
```

```
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IINFO, J, LWKOPT, NB, NH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DORGQR, XERBLA
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NH = IHI - ILO
      LQUERY = ( LWORK.EQ.-1 )
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -2
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LWORK.LT.MAX( 1, NH ) .AND. .NOT.LQUERY ) THEN
         INFO = -8
      END IF
*
      IF( INFO.EQ.0 ) THEN
         NB = ILAENV( 1, 'DORGQR', ' ', NH, NH, NH, -1 )
         LWKOPT = MAX( 1, NH )*NB
         WORK( 1 ) = LWKOPT
      END IF
*
```

```
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DORGHR', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
*     Shift the vectors which define the elementary reflectors one
*     column to the right, and set the first ilo and the last n-ihi
*     rows and columns to those of the unit matrix
*
      DO 40 J = IHI, ILO + 1, -1
         DO 10 I = 1, J - 1
            A( I, J ) = ZERO
   10    CONTINUE
         DO 20 I = J + 1, IHI
            A( I, J ) = A( I, J-1 )
   20    CONTINUE
         DO 30 I = IHI + 1, N
            A( I, J ) = ZERO
   30    CONTINUE
   40 CONTINUE
      DO 60 J = 1, ILO
         DO 50 I = 1, N
            A( I, J ) = ZERO
   50    CONTINUE
         A( J, J ) = ONE
   60 CONTINUE
      DO 80 J = IHI + 1, N
         DO 70 I = 1, N
            A( I, J ) = ZERO
   70    CONTINUE
         A( J, J ) = ONE
   80 CONTINUE
*
      IF( NH.GT.0 ) THEN
*
*        Generate Q(ilo+1:ihi,ilo+1:ihi)
*
         CALL DORGQR( NH, NH, NH, A( ILO+1, ILO+1 ), LDA, TAU( ILO ),
     $                WORK, LWORK, IINFO )
      END IF
      WORK( 1 ) = LWKOPT
```

```
      RETURN
*
*     End of DORGHR
*
      END
```

———-

90.196.4 lisp code

— **dorghr.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dorghr (n ilo ihi a lda tau work lwork info)
    (declare (type (array double-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lwork lda ihi ilo n))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((i 0) (iinfo 0) (j 0) (lwkopt 0) (nb 0) (nh 0) (lquery nil))
        (declare (type (f2cl-lib:integer4) i iinfo j lwkopt nb nh)
                 (type f2cl-lib:logical lquery))
        (setf info 0)
        (setf nh (f2cl-lib:int-sub ihi ilo))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond ((< n 0) (setf info -1))
              ((or (< ilo 1)
                   (> ilo
                      (max (the f2cl-lib:integer4 1)
                           (the f2cl-lib:integer4 n))))
               (setf info -2))
              ((or
                (< ihi
                   (min (the f2cl-lib:integer4 ilo) (the f2cl-lib:integer4 n)))
                (> ihi n))
               (setf info -3))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -5))
              ((and
                (< lwork
                   (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nh)))
                (not lquery))
```

```
        (setf info -8)))
(cond
 ((= info 0)
  (setf nb
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (ilaenv 1 "DORGQR" " " nh nh nh -1)
            (declare (ignore var-0 var-1 var-2 var-6))
            (when var-3 (setf nh var-3))
            (when var-4 (setf nh var-4))
            (when var-5 (setf nh var-5))
            ret-val))
  (setf lwkopt
          (f2cl-lib:int-mul
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nh))
           nb))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))))
(cond
 ((/= info 0) (xerbla "DORGHR" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond
 ((= n 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'double-float))
  (go end_label)))
(f2cl-lib:fdo (j ihi (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j (f2cl-lib:int-add ilo 1)) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                zero)
       label10))
    (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                  ((> i ihi) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref a-%data% (i (f2cl-lib:int-sub j 1))
                               ((1 lda) (1 *)) a-%offset%))
       label20))
    (f2cl-lib:fdo (i (f2cl-lib:int-add ihi 1) (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                zero)
       label30))
   label40))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                ((> j ilo) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                  zero)
         label50))
      (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
              one)
     label60))
  (f2cl-lib:fdo (j (f2cl-lib:int-add ihi 1) (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                  zero)
         label70))
      (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
              one)
     label80))
  (cond
   ((> nh 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dorgqr nh nh nh
         (f2cl-lib:array-slice a-%data% double-float
                               ((+ ilo 1) (f2cl-lib:int-add ilo 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice tau-%data% double-float (ilo) ((1 *))
                               tau-%offset%)
         work lwork iinfo)
      (declare (ignore var-3 var-5 var-6))
      (when var-0 (setf nh var-0))
      (when var-1 (setf nh var-1))
      (when var-2 (setf nh var-2))
      (when var-4 (setf lda var-4))
      (when var-7 (setf lwork var-7))
      (when var-8 (setf iinfo var-8)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
  (go end_label)
 end_label
  (return (values nil nil nil nil lda nil nil lwork info))))))
```

———-

90.197 dorgl2

90.197.1 Axiom unit tests

— dorgl2.input —

```
)set break resume
)sys rm -f dorgl2.output
)spool dorgl2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.197.2 Axiom help page

— dorgl2.help —

```
======================================================================
dorgl2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DORGL2 - an m by n real matrix Q with orthonormal rows,

SYNOPSIS
       SUBROUTINE DORGL2( M, N, K, A, LDA, TAU, WORK, INFO )

           INTEGER        INFO, K, LDA, M, N

           DOUBLE         PRECISION A( LDA, * ), TAU( * ), WORK( * )

  Purpose
  =======

  DORGL2 generates an m by n real matrix Q with orthonormal rows,
  which is defined as the first m rows of a product of k elementary
  reflectors of order n
```

```
      Q  =  H(k) . . . H(2) H(1)

 as returned by DGELQF.

 Arguments
 =========

 M       (input) INTEGER
         The number of rows of the matrix Q. M >= 0.

 N       (input) INTEGER
         The number of columns of the matrix Q. N >= M.

 K       (input) INTEGER
         The number of elementary reflectors whose product defines the
         matrix Q. M >= K >= 0.

 A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
         On entry, the i-th row must contain the vector which defines
         the elementary reflector H(i), for i = 1,2,...,k, as returned
         by DGELQF in the first k rows of its array argument A.
         On exit, the m-by-n matrix Q.

 LDA     (input) INTEGER
         The first dimension of the array A. LDA >= max(1,M).

 TAU     (input) DOUBLE PRECISION array, dimension (K)
         TAU(i) must contain the scalar factor of the elementary
         reflector H(i), as returned by DGELQF.

 WORK    (workspace) DOUBLE PRECISION array, dimension (M)

 INFO    (output) INTEGER
         = 0: successful exit
         < 0: if INFO = -i, the i-th argument has an illegal value
```

90.197.3 fortran code

— dorgl2.f —

```
      SUBROUTINE DORGL2( M, N, K, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
```

```
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      INTEGER            INFO, K, LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, L
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARF, DSCAL, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.M ) THEN
         INFO = -2
      ELSE IF( K.LT.0 .OR. K.GT.M ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DORGL2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.LE.0 )
     $   RETURN
*
      IF( K.LT.M ) THEN
```

```
*
*        Initialise rows k+1:m to rows of the unit matrix
*
         DO 20 J = 1, N
            DO 10 L = K + 1, M
               A( L, J ) = ZERO
   10       CONTINUE
            IF( J.GT.K .AND. J.LE.M )
     $         A( J, J ) = ONE
   20    CONTINUE
      END IF
*
      DO 40 I = K, 1, -1
*
*        Apply H(i) to A(i:m,i:n) from the right
*
         IF( I.LT.N ) THEN
            IF( I.LT.M ) THEN
               A( I, I ) = ONE
               CALL DLARF( 'Right', M-I, N-I+1, A( I, I ), LDA,
     $                     TAU( I ), A( I+1, I ), LDA, WORK )
            END IF
            CALL DSCAL( N-I, -TAU( I ), A( I, I+1 ), LDA )
         END IF
         A( I, I ) = ONE - TAU( I )
*
*        Set A(i,1:i-1) to zero
*
         DO 30 L = 1, I - 1
            A( I, L ) = ZERO
   30    CONTINUE
   40 CONTINUE
      RETURN
*
*     End of DORGL2
*
      END
```

———-

90.197.4 lisp code

— dorgl2.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
```

```
          (ignorable one zero))
  (defun dorgl2 (m n k a lda tau work info)
    (declare (type (array double-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lda k n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((i 0) (j 0) (l 0))
        (declare (type (f2cl-lib:integer4) i j l))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((< n m) (setf info -2))
              ((or (< k 0) (> k m)) (setf info -3))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -5)))
        (cond
         ((/= info 0) (xerbla "DORGL2" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (<= m 0)
            (go end_label))
        (cond
         ((< k m)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j n) nil)
            (tagbody
              (f2cl-lib:fdo (l (f2cl-lib:int-add k 1) (f2cl-lib:int-add l 1))
                            ((> l m) nil)
                (tagbody
                  (setf (f2cl-lib:fref a-%data% (l j) ((1 lda) (1 *))
                                       a-%offset%)
                        zero)
                 label10))
              (if (and (> j k) (<= j m))
                  (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                       a-%offset%)
                        one))
             label20))))
        (f2cl-lib:fdo (i k (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                      ((> i 1) nil)
          (tagbody
            (cond
             ((< i n)
              (cond
               ((< i m)
                (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                      one)
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                    (dlarf "Right" (f2cl-lib:int-sub m i)
```

```
                    (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1)
                    (f2cl-lib:array-slice a-%data% double-float (i i)
                                          ((1 lda) (1 *)) a-%offset%)
                    lda (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%)
                    (f2cl-lib:array-slice a-%data% double-float ((+ i 1) i)
                                          ((1 lda) (1 *)) a-%offset%)
                    lda work)
                (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6 var-8))
                (setf lda var-4)
                (setf lda var-7))))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (dscal (f2cl-lib:int-sub n i)
                 (- (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
                 (f2cl-lib:array-slice a-%data% double-float
                                       (i (f2cl-lib:int-add i 1))
                                       ((1 lda) (1 *)) a-%offset%)
                 lda)
              (declare (ignore var-0 var-1 var-2))
              (when var-3 (setf lda var-3)))))
        (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                (- one
                   (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%)))
        (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                      ((> l (f2cl-lib:int-add i (f2cl-lib:int-sub 1))) nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data% (i l) ((1 lda) (1 *)) a-%offset%)
                    zero)
           label30))
       label40))
     (go end_label)
    end_label
     (return (values nil nil nil nil lda nil nil info))))))
```

———-

90.198 dorglq

90.198.1 Axiom unit tests

— dorglq.input —

```
)set break resume
)sys rm -f dorglq.output
)spool dorglq.output
)set message test on
)set message auto off
```

```
)clear all

)spool
)lisp (bye)
```

———-

90.198.2 Axiom help page

— **dorglq.help** —

```
=====================================================================
dorglq examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

NAME
       DORGLQ - an M-by-N real matrix Q with orthonormal rows,

SYNOPSIS
       SUBROUTINE DORGLQ( M, N, K, A, LDA, TAU, WORK, LWORK, INFO )

           INTEGER        INFO, K, LDA, LWORK, M, N

           DOUBLE         PRECISION A( LDA, * ), TAU( * ), WORK( * )

  Purpose
  =======

  DORGLQ generates an M-by-N real matrix Q with orthonormal rows,
  which is defined as the first M rows of a product of K elementary
  reflectors of order N

        Q  =  H(k) . . . H(2) H(1)

  as returned by DGELQF.

  Arguments
  =========

  M       (input) INTEGER
          The number of rows of the matrix Q. M >= 0.

  N       (input) INTEGER
          The number of columns of the matrix Q. N >= M.
```

```
K       (input) INTEGER
        The number of elementary reflectors whose product defines the
        matrix Q. M >= K >= 0.

A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
        On entry, the i-th row must contain the vector which defines
        the elementary reflector H(i), for i = 1,2,...,k, as returned
        by DGELQF in the first k rows of its array argument A.
        On exit, the M-by-N matrix Q.

LDA     (input) INTEGER
        The first dimension of the array A. LDA >= max(1,M).

TAU     (input) DOUBLE PRECISION array, dimension (K)
        TAU(i) must contain the scalar factor of the elementary
        reflector H(i), as returned by DGELQF.

WORK    (workspace/output) DOUBLE PRECISION array, dimension (LWORK)
        On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK   (input) INTEGER
        The dimension of the array WORK. LWORK >= max(1,M).
        For optimum performance LWORK >= M*NB, where NB is
        the optimal blocksize.

        If LWORK = -1, then a workspace query is assumed; the routine
        only calculates the optimal size of the WORK array, returns
        this value as the first entry of the WORK array, and no error
        message related to LWORK is issued by XERBLA.

INFO    (output) INTEGER
        = 0:  successful exit
        < 0:  if INFO = -i, the i-th argument has an illegal value
```

———-

90.198.3 fortran code

— **dorglq.f** —

```
      SUBROUTINE DORGLQ( M, N, K, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
```

```
*
*     .. Scalar Arguments ..
      INTEGER            INFO, K, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, J, KI, KK, L, LDWORK,
     $                   LWKOPT, NB, NBMIN, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARFB, DLARFT, DORGL2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NB = ILAENV( 1, 'DORGLQ', ' ', M, N, K, -1 )
      LWKOPT = MAX( 1, M )*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.M ) THEN
         INFO = -2
      ELSE IF( K.LT.0 .OR. K.GT.M ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LWORK.LT.MAX( 1, M ) .AND. .NOT.LQUERY ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
```

```
         CALL XERBLA( 'DORGLQ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.LE.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      NX = 0
      IWS = M
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'DORGLQ', ' ', M, N, K, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = M
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'DORGLQ', ' ', M, N, K, -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code after the last block.
*        The first kk rows are handled by the block method.
*
         KI = ( ( K-NX-1 ) / NB )*NB
         KK = MIN( K, KI+NB )
*
*        Set A(kk+1:m,1:kk) to zero.
*
         DO 20 J = 1, KK
            DO 10 I = KK + 1, M
```

```
               A( I, J ) = ZERO
   10       CONTINUE
   20    CONTINUE
      ELSE
         KK = 0
      END IF
*
*     Use unblocked code for the last or only block.
*
      IF( KK.LT.M )
     $   CALL DORGL2( M-KK, N-KK, K-KK, A( KK+1, KK+1 ), LDA,
     $                TAU( KK+1 ), WORK, IINFO )
*
      IF( KK.GT.0 ) THEN
*
*        Use blocked code
*
         DO 50 I = KI + 1, 1, -NB
            IB = MIN( NB, K-I+1 )
            IF( I+IB.LE.M ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i) H(i+1) . . . H(i+ib-1)
*
               CALL DLARFT( 'Forward', 'Rowwise', N-I+1, IB, A( I, I ),
     $                      LDA, TAU( I ), WORK, LDWORK )
*
*              Apply H' to A(i+ib:m,i:n) from the right
*
               CALL DLARFB( 'Right', 'Transpose', 'Forward', 'Rowwise',
     $                      M-I-IB+1, N-I+1, IB, A( I, I ), LDA, WORK,
     $                      LDWORK, A( I+IB, I ), LDA, WORK( IB+1 ),
     $                      LDWORK )
            END IF
*
*           Apply H' to columns i:n of current block
*
            CALL DORGL2( IB, N-I+1, IB, A( I, I ), LDA, TAU( I ), WORK,
     $                   IINFO )
*
*           Set columns 1:i-1 of current block to zero
*
            DO 40 J = 1, I - 1
               DO 30 L = I, I + IB - 1
                  A( L, J ) = ZERO
   30          CONTINUE
   40       CONTINUE
   50    CONTINUE
      END IF
*
```

```
      WORK( 1 ) = IWS
      RETURN
*
*     End of DORGLQ
*
      END
```

90.198.4 lisp code

— **dorglq.lisp** —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun dorglq (m n k a lda tau work lwork info)
    (declare (type (array double-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lwork lda k n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (j 0) (ki 0) (kk 0) (l 0)
             (ldwork 0) (lwkopt 0) (nb 0) (nbmin 0) (nx 0) (lquery nil))
        (declare
         (type (f2cl-lib:integer4) i ib iinfo iws j ki kk l ldwork lwkopt nb
          nbmin nx)
         (type f2cl-lib:logical lquery))
        (setf info 0)
        (setf nb
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (ilaenv 1 "DORGLQ" " " m n k -1)
                  (declare (ignore var-0 var-1 var-2 var-6))
                  (when var-3 (setf m var-3))
                  (when var-4 (setf n var-4))
                  (when var-5 (setf k var-5))
                  ret-val))
        (setf lwkopt
                (f2cl-lib:int-mul
                 (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)) nb))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond ((< m 0) (setf info -1)) ((< n m) (setf info -2))
              ((or (< k 0) (> k m)) (setf info -3))
```

```
      ((< lda
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -5))
      ((and
        (< lwork
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
        (not lquery))
       (setf info -8)))
(cond
 ((/= info 0) (xerbla "DORGLQ" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond
 ((<= m 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'double-float))
  (go end_label)))
(setf nbmin 2)
(setf nx 0)
(setf iws m)
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "DORGLQ" " " m n k -1)
                      (declare (ignore var-0 var-1 var-2 var-6))
                      (when var-3 (setf m var-3))
                      (when var-4 (setf n var-4))
                      (when var-5 (setf k var-5))
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork m) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4
                             var-5 var-6)
                            (ilaenv 2 "DORGLQ" " " m n k -1)
                          (declare (ignore var-0 var-1 var-2 var-6))
                          (when var-3 (setf m var-3))
                          (when var-4 (setf n var-4))
                          (when var-5 (setf k var-5))
                          ret-val)))))))))
```

```
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (setf ki (* (the f2cl-lib:integer4 (truncate (- k nx 1) nb)) nb))
  (setf kk
          (min (the f2cl-lib:integer4 k)
               (the f2cl-lib:integer4 (f2cl-lib:int-add ki nb))))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j kk) nil)
    (tagbody
      (f2cl-lib:fdo (i (f2cl-lib:int-add kk 1) (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                  zero)
         label10))
     label20)))
 (t (setf kk 0)))
(if (< kk m)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (dorgl2 (f2cl-lib:int-sub m kk) (f2cl-lib:int-sub n kk)
         (f2cl-lib:int-sub k kk)
         (f2cl-lib:array-slice a-%data% double-float
                               ((+ kk 1) (f2cl-lib:int-add kk 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice tau-%data% double-float ((+ kk 1))
                               ((1 *)) tau-%offset%)
         work iinfo)
      (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6))
      (setf lda var-4)
      (setf iinfo var-7)))
(cond
 ((> kk 0)
  (f2cl-lib:fdo (i (f2cl-lib:int-add ki 1)
                 (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
                ((> i 1) nil)
    (tagbody
      (setf ib
              (min (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
      (cond
       ((<= (f2cl-lib:int-add i ib) m)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (dlarft "Forward" "Rowwise"
             (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) ib
             (f2cl-lib:array-slice a-%data% double-float (i i)
```

```
                              ((1 lda) (1 *)) a-%offset%)
        lda
        (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                              tau-%offset%)
        work ldwork)
      (declare (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7))
      (setf lda var-5)
      (setf ldwork var-8))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13 var-14)
        (dlarfb "Right" "Transpose" "Forward" "Rowwise"
         (f2cl-lib:int-add (f2cl-lib:int-sub m i ib) 1)
         (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) ib
         (f2cl-lib:array-slice a-%data% double-float (i i)
                               ((1 lda) (1 *)) a-%offset%)
         lda work ldwork
         (f2cl-lib:array-slice a-%data% double-float ((+ i ib) i)
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice work-%data% double-float ((+ ib 1))
                               ((1 *)) work-%offset%)
         ldwork)
      (declare
       (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9
        var-11 var-13))
      (setf ib var-6)
      (setf lda var-8)
      (setf ldwork var-10)
      (setf lda var-12)
      (setf ldwork var-14))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (dorgl2 ib (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1) ib
       (f2cl-lib:array-slice a-%data% double-float (i i)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                             tau-%offset%)
       work iinfo)
    (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6))
    (setf lda var-4)
    (setf iinfo var-7))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (f2cl-lib:fdo (l i (f2cl-lib:int-add l 1))
                    ((> l
                        (f2cl-lib:int-add i ib
```

```
                                                    (f2cl-lib:int-sub 1)))
                                nil)
                     (tagbody
                       (setf (f2cl-lib:fref a-%data% (l j) ((1 lda) (1 *))
                                            a-%offset%)
                             zero)
                      label30))
                  label40))
              label50))))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 iws) 'double-float))
        (go end_label)
       end_label
        (return (values m n k nil lda nil nil nil info))))))
```

———-

90.199 dorgqr

90.199.1 Axiom unit tests

— **dorgqr.input** —

```
)set break resume
)sys rm -f dorgqr.output
)spool dorgqr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.199.2 Axiom help page

— **dorgqr.help** —

```
========================================================================
dorgqr examples
========================================================================
```

```
=====================================================================
Man Page Details
=====================================================================

NAME
       DORGQR - an M-by-N real matrix Q with orthonormal columns,

SYNOPSIS
       SUBROUTINE DORGQR( M, N, K, A, LDA, TAU, WORK, LWORK, INFO )

           INTEGER        INFO, K, LDA, LWORK, M, N

           DOUBLE         PRECISION A( LDA, * ), TAU( * ), WORK( * )

  Purpose
  =======

  DORGQR generates an M-by-N real matrix Q with orthonormal columns,
  which is defined as the first N columns of a product of K elementary
  reflectors of order M

        Q  =  H(1) H(2) . . . H(k)

  as returned by DGEQRF.

  Arguments
  =========

  M       (input) INTEGER
          The number of rows of the matrix Q. M >= 0.

  N       (input) INTEGER
          The number of columns of the matrix Q. M >= N >= 0.

  K       (input) INTEGER
          The number of elementary reflectors whose product defines the
          matrix Q. N >= K >= 0.

  A       (input/output) DOUBLE PRECISION array, dimension (LDA,N)
          On entry, the i-th column must contain the vector which
          defines the elementary reflector H(i), for i = 1,2,...,k, as
          returned by DGEQRF in the first k columns of its array
          argument A.
          On exit, the M-by-N matrix Q.

  LDA     (input) INTEGER
          The first dimension of the array A. LDA >= max(1,M).

  TAU     (input) DOUBLE PRECISION array, dimension (K)
          TAU(i) must contain the scalar factor of the elementary
```

```
         reflector H(i), as returned by DGEQRF.

 WORK    (workspace/output) DOUBLE PRECISION array, dimension (LWORK)
         On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK   (input) INTEGER
         The dimension of the array WORK. LWORK >= max(1,N).
         For optimum performance LWORK >= N*NB, where NB is the
         optimal blocksize.

         If LWORK = -1, then a workspace query is assumed; the routine
         only calculates the optimal size of the WORK array, returns
         this value as the first entry of the WORK array, and no error
         message related to LWORK is issued by XERBLA.

 INFO    (output) INTEGER
         = 0:  successful exit
         < 0:  if INFO = -i, the i-th argument has an illegal value
```

90.199.3 fortran code

— dorgqr.f —

```
      SUBROUTINE DORGQR( M, N, K, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      INTEGER            INFO, K, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
```

```
      INTEGER            I, IB, IINFO, IWS, J, KI, KK, L, LDWORK,
     $                   LWKOPT, NB, NBMIN, NX
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARFB, DLARFT, DORG2R, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NB = ILAENV( 1, 'DORGQR', ' ', M, N, K, -1 )
      LWKOPT = MAX( 1, N )*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 .OR. N.GT.M ) THEN
         INFO = -2
      ELSE IF( K.LT.0 .OR. K.GT.N ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DORGQR', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.LE.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      NX = 0
      IWS = N
```

```
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
*
         NX = MAX( 0, ILAENV( 3, 'DORGQR', ' ', M, N, K, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = N
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'DORGQR', ' ', M, N, K, -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
*        Use blocked code after the last block.
*        The first kk columns are handled by the block method.
*
         KI = ( ( K-NX-1 ) / NB )*NB
         KK = MIN( K, KI+NB )
*
*        Set A(1:kk,kk+1:n) to zero.
*
         DO 20 J = KK + 1, N
            DO 10 I = 1, KK
               A( I, J ) = ZERO
   10       CONTINUE
   20    CONTINUE
      ELSE
         KK = 0
      END IF
*
*     Use unblocked code for the last or only block.
*
      IF( KK.LT.N )
     $   CALL DORG2R( M-KK, N-KK, K-KK, A( KK+1, KK+1 ), LDA,
     $                TAU( KK+1 ), WORK, IINFO )
*
      IF( KK.GT.0 ) THEN
*
*        Use blocked code
```

```
*
         DO 50 I = KI + 1, 1, -NB
            IB = MIN( NB, K-I+1 )
            IF( I+IB.LE.N ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i) H(i+1) . . . H(i+ib-1)
*
               CALL DLARFT( 'Forward', 'Columnwise', M-I+1, IB,
     $                      A( I, I ), LDA, TAU( I ), WORK, LDWORK )
*
*              Apply H to A(i:m,i+ib:n) from the left
*
               CALL DLARFB( 'Left', 'No transpose', 'Forward',
     $                      'Columnwise', M-I+1, N-I-IB+1, IB,
     $                      A( I, I ), LDA, WORK, LDWORK, A( I, I+IB ),
     $                      LDA, WORK( IB+1 ), LDWORK )
            END IF
*
*           Apply H to rows i:m of current block
*
            CALL DORG2R( M-I+1, IB, IB, A( I, I ), LDA, TAU( I ), WORK,
     $                   IINFO )
*
*           Set rows 1:i-1 of current block to zero
*
            DO 40 J = I, I + IB - 1
               DO 30 L = 1, I - 1
                  A( L, J ) = ZERO
   30          CONTINUE
   40       CONTINUE
   50    CONTINUE
      END IF
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of DORGQR
*
      END
```

———-

90.199.4 lisp code

— dorgqr.lisp —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun dorgqr (m n k a lda tau work lwork info)
    (declare (type (array double-float (*)) work tau a)
             (type (f2cl-lib:integer4) info lwork lda k n m))
    (f2cl-lib:with-multi-array-data
        ((a double-float a-%data% a-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (j 0) (ki 0) (kk 0) (l 0)
             (ldwork 0) (lwkopt 0) (nb 0) (nbmin 0) (nx 0) (lquery nil))
        (declare
         (type (f2cl-lib:integer4) i ib iinfo iws j ki kk l ldwork lwkopt nb
          nbmin nx)
         (type f2cl-lib:logical lquery))
        (setf info 0)
        (setf nb
                (multiple-value-bind
                    (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (ilaenv 1 "DORGQR" " " m n k -1)
                  (declare (ignore var-0 var-1 var-2 var-6))
                  (when var-3 (setf m var-3))
                  (when var-4 (setf n var-4))
                  (when var-5 (setf k var-5))
                  ret-val))
        (setf lwkopt
                (f2cl-lib:int-mul
                 (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)) nb))
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond ((< m 0) (setf info -1)) ((or (< n 0) (> n m)) (setf info -2))
              ((or (< k 0) (> k n)) (setf info -3))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -5))
              ((and
                (< lwork
                   (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
                (not lquery))
               (setf info -8)))
        (cond
         ((/= info 0) (xerbla "DORGQR" (f2cl-lib:int-sub info)) (go end_label))
         (lquery (go end_label)))
        (cond
         ((<= n 0)
          (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                  (coerce (the f2cl-lib:integer4 1) 'double-float))
          (go end_label)))
```

```
(setf nbmin 2)
(setf nx 0)
(setf iws n)
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "DORGQR" " " m n k -1)
                      (declare (ignore var-0 var-1 var-2 var-6))
                      (when var-3 (setf m var-3))
                      (when var-4 (setf n var-4))
                      (when var-5 (setf k var-5))
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork n) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4
                             var-5 var-6)
                            (ilaenv 2 "DORGQR" " " m n k -1)
                          (declare (ignore var-0 var-1 var-2 var-6))
                          (when var-3 (setf m var-3))
                          (when var-4 (setf n var-4))
                          (when var-5 (setf k var-5))
                          ret-val)))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (setf ki (* (the f2cl-lib:integer4 (truncate (- k nx 1) nb)) nb))
  (setf kk
          (min (the f2cl-lib:integer4 k)
               (the f2cl-lib:integer4 (f2cl-lib:int-add ki nb))))
  (f2cl-lib:fdo (j (f2cl-lib:int-add kk 1) (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i kk) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%)
                zero)
         label10))
```

```
     label20)))
 (t (setf kk 0)))
(if (< kk n)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (dorg2r (f2cl-lib:int-sub m kk) (f2cl-lib:int-sub n kk)
         (f2cl-lib:int-sub k kk)
         (f2cl-lib:array-slice a-%data% double-float
                               ((+ kk 1) (f2cl-lib:int-add kk 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice tau-%data% double-float ((+ kk 1))
                               ((1 *)) tau-%offset%)
         work iinfo)
      (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6))
      (setf lda var-4)
      (setf iinfo var-7)))
(cond
 ((> kk 0)
  (f2cl-lib:fdo (i (f2cl-lib:int-add ki 1)
                 (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
                ((> i 1) nil)
    (tagbody
      (setf ib
              (min (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
      (cond
       ((<= (f2cl-lib:int-add i ib) n)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (dlarft "Forward" "Columnwise"
             (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
             (f2cl-lib:array-slice a-%data% double-float (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                   tau-%offset%)
             work ldwork)
          (declare (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7))
          (setf lda var-5)
          (setf ldwork var-8))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12 var-13 var-14)
            (dlarfb "Left" "No transpose" "Forward" "Columnwise"
             (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1) ib
             (f2cl-lib:array-slice a-%data% double-float (i i)
                                   ((1 lda) (1 *)) a-%offset%)
```

```
                lda work ldwork
                (f2cl-lib:array-slice a-%data% double-float
                                      (i (f2cl-lib:int-add i ib))
                                      ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice work-%data% double-float ((+ ib 1))
                                      ((1 *)) work-%offset%)
                ldwork)
            (declare
             (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9
              var-11 var-13))
            (setf ib var-6)
            (setf lda var-8)
            (setf ldwork var-10)
            (setf lda var-12)
            (setf ldwork var-14))))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
            (dorg2r (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib ib
             (f2cl-lib:array-slice a-%data% double-float (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                   tau-%offset%)
             work iinfo)
          (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6))
          (setf lda var-4)
          (setf iinfo var-7))
        (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                      ((> j (f2cl-lib:int-add i ib (f2cl-lib:int-sub 1)))
                       nil)
          (tagbody
            (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                          ((> l
                              (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data% (l j) ((1 lda) (1 *))
                                     a-%offset%)
                      zero)
                label30))
           label40))
       label50))))
 (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce (the f2cl-lib:integer4 iws) 'double-float))
 (go end_label)
end_label
 (return (values m n k nil lda nil nil nil info))))))
```

———-

90.200 dorm2r

90.200.1 Axiom unit tests

— **dorm2r.input** —

```
)set break resume
)sys rm -f dorm2r.output
)spool dorm2r.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.200.2 Axiom help page

— **dorm2r.help** —

```
======================================================================
dorm2r examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DORM2R  -  the  general real m by n matrix C with   Q * C if SIDE = 'L'
       and TRANS = 'N', or   Q'* C if SIDE = 'L' and TRANS = 'T', or   C  *  Q
       if  SIDE  =  'R' and TRANS = 'N', or   C * Q' if SIDE = 'R' and TRANS =
       'T',

SYNOPSIS
       SUBROUTINE DORM2R( SIDE, TRANS, M, N, K, A, LDA,  TAU,  C,  LDC,  WORK,
                          INFO )

           CHARACTER      SIDE, TRANS
```

```
      INTEGER        INFO, K, LDA, LDC, M, N

      DOUBLE         PRECISION  A( LDA, * ), C( LDC, * ), TAU( * ), WORK(
                     * )
```

Purpose
=======

DORM2R overwrites the general real m by n matrix C with

```
      Q * C  if SIDE = 'L' and TRANS = 'N', or

      Q'* C  if SIDE = 'L' and TRANS = 'T', or

      C * Q  if SIDE = 'R' and TRANS = 'N', or

      C * Q' if SIDE = 'R' and TRANS = 'T',
```

where Q is a real orthogonal matrix defined as the product of k elementary reflectors

```
      Q = H(1) H(2) . . . H(k)
```

as returned by DGEQRF. Q is of order m if SIDE = 'L' and of order n if SIDE = 'R'.

Arguments
=========

```
SIDE    (input) CHARACTER*1
        = 'L': apply Q or Q' from the Left
        = 'R': apply Q or Q' from the Right

TRANS   (input) CHARACTER*1
        = 'N': apply Q  (No transpose)
        = 'T': apply Q' (Transpose)

M       (input) INTEGER
        The number of rows of the matrix C. M >= 0.

N       (input) INTEGER
        The number of columns of the matrix C. N >= 0.

K       (input) INTEGER
        The number of elementary reflectors whose product defines
        the matrix Q.
        If SIDE = 'L', M >= K >= 0;
        if SIDE = 'R', N >= K >= 0.

A       (input) DOUBLE PRECISION array, dimension (LDA,K)
```

```
         The i-th column must contain the vector which defines the
         elementary reflector H(i), for i = 1,2,...,k, as returned by
         DGEQRF in the first k columns of its array argument A.
         A is modified by the routine but restored on exit.

 LDA     (input) INTEGER
         The leading dimension of the array A.
         If SIDE = 'L', LDA >= max(1,M);
         if SIDE = 'R', LDA >= max(1,N).

 TAU     (input) DOUBLE PRECISION array, dimension (K)
         TAU(i) must contain the scalar factor of the elementary
         reflector H(i), as returned by DGEQRF.

 C       (input/output) DOUBLE PRECISION array, dimension (LDC,N)
         On entry, the m by n matrix C.
         On exit, C is overwritten by Q*C or Q'*C or C*Q' or C*Q.

 LDC     (input) INTEGER
         The leading dimension of the array C. LDC >= max(1,M).

 WORK    (workspace) DOUBLE PRECISION array, dimension
                                  (N) if SIDE = 'L',
                                  (M) if SIDE = 'R'

 INFO    (output) INTEGER
         = 0: successful exit
         < 0: if INFO = -i, the i-th argument had an illegal value
```

90.200.3 fortran code

— dorm2r.f —

```
      SUBROUTINE DORM2R( SIDE, TRANS, M, N, K, A, LDA, TAU, C, LDC,
     $                   WORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     February 29, 1992
*
*     .. Scalar Arguments ..
      CHARACTER          SIDE, TRANS
      INTEGER            INFO, K, LDA, LDC, M, N
*     ..
```

```
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), C( LDC, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LEFT, NOTRAN
      INTEGER            I, I1, I2, I3, IC, JC, MI, NI, NQ
      DOUBLE PRECISION   AII
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARF, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LEFT = LSAME( SIDE, 'L' )
      NOTRAN = LSAME( TRANS, 'N' )
*
*     NQ is the order of Q
*
      IF( LEFT ) THEN
         NQ = M
      ELSE
         NQ = N
      END IF
      IF( .NOT.LEFT .AND. .NOT.LSAME( SIDE, 'R' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) ) THEN
         INFO = -2
      ELSE IF( M.LT.0 ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( K.LT.0 .OR. K.GT.NQ ) THEN
         INFO = -5
```

```
      ELSE IF( LDA.LT.MAX( 1, NQ ) ) THEN
         INFO = -7
      ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
         INFO = -10
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DORM2R', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 .OR. K.EQ.0 )
     $   RETURN
*
      IF( ( LEFT .AND. .NOT.NOTRAN ) .OR. ( .NOT.LEFT .AND. NOTRAN ) )
     $     THEN
         I1 = 1
         I2 = K
         I3 = 1
      ELSE
         I1 = K
         I2 = 1
         I3 = -1
      END IF
*
      IF( LEFT ) THEN
         NI = N
         JC = 1
      ELSE
         MI = M
         IC = 1
      END IF
*
      DO 10 I = I1, I2, I3
         IF( LEFT ) THEN
*
*           H(i) is applied to C(i:m,1:n)
*
            MI = M - I + 1
            IC = I
         ELSE
*
*           H(i) is applied to C(1:m,i:n)
*
            NI = N - I + 1
            JC = I
         END IF
*
*        Apply H(i)
```

```
*
         AII = A( I, I )
         A( I, I ) = ONE
         CALL DLARF( SIDE, MI, NI, A( I, I ), 1, TAU( I ), C( IC, JC ),
     $               LDC, WORK )
         A( I, I ) = AII
   10 CONTINUE
      RETURN
*
*     End of DORM2R
*
      END
```

———-

90.200.4 lisp code

— **dorm2r.lisp** —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dorm2r (side trans m n k a lda tau c ldc work info)
    (declare (type (array double-float (*)) work c tau a)
             (type (f2cl-lib:integer4) info ldc lda k n m)
             (type (string *) trans side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (trans character trans-%data% trans-%offset%)
         (a double-float a-%data% a-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (c double-float c-%data% c-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((aii 0.0d0) (i 0) (i1 0) (i2 0) (i3 0) (ic 0) (jc 0) (mi 0) (ni 0)
             (nq 0) (left nil) (notran nil))
        (declare (type (double-float) aii)
                 (type (f2cl-lib:integer4) i i1 i2 i3 ic jc mi ni nq)
                 (type f2cl-lib:logical left notran))
        (setf info 0)
        (setf left
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame side "L")
                  (declare (ignore var-1))
                  (when var-0 (setf side var-0))
                  ret-val))
        (setf notran
                (multiple-value-bind (ret-val var-0 var-1)
```

```
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond (left (setf nq m)) (t (setf nq n)))
(cond
 ((and (not left)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "R")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val)))
  (setf info -1))
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -2))
 ((< m 0) (setf info -3)) ((< n 0) (setf info -4))
 ((or (< k 0) (> k nq)) (setf info -5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nq)))
  (setf info -7))
 ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -10)))
(cond
 ((/= info 0) (xerbla "DORM2R" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= m 0) (= n 0) (= k 0))
    (go end_label))
(cond
 ((or (and left (not notran)) (and (not left) notran)) (setf i1 1)
  (setf i2 k) (setf i3 1))
 (t (setf i1 k) (setf i2 1) (setf i3 -1)))
(cond (left (setf ni n) (setf jc 1)) (t (setf mi m) (setf ic 1)))
(f2cl-lib:fdo (i i1 (f2cl-lib:int-add i i3))
              ((> i i2) nil)
  (tagbody
    (cond
     (left (setf mi (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
      (setf ic i))
     (t (setf ni (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
      (setf jc i)))
    (setf aii
            (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
            one)
```

```
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (dlarf side mi ni
                 (f2cl-lib:array-slice a-%data% double-float (i i)
                                       ((1 lda) (1 *)) a-%offset%)
                 1 (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%)
                 (f2cl-lib:array-slice c-%data% double-float (ic jc)
                                       ((1 ldc) (1 *)) c-%offset%)
                 ldc work)
              (declare (ignore var-3 var-4 var-5 var-6 var-8))
              (setf side var-0)
              (setf mi var-1)
              (setf ni var-2)
              (setf ldc var-7))
            (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                    aii)
          label10))
        (go end_label)
       end_label
        (return
         (values side trans nil nil nil nil nil nil nil ldc nil info))))))
```

———-

90.201 dormbr

90.201.1 Axiom unit tests

— **dormbr.input** —

```
)set break resume
)sys rm -f dormbr.output
)spool dormbr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.201.2 Axiom help page

— dormbr.help —

```
====================================================================
dormbr examples
====================================================================

====================================================================
Man Page Details
====================================================================

NAME
       DORMBR - = 'Q', DORMBR overwrites the general real M-by-N matrix C with
       SIDE = 'L' SIDE = 'R' TRANS = 'N'

SYNOPSIS
       SUBROUTINE DORMBR( VECT, SIDE, TRANS, M, N, K, A,  LDA,  TAU,  C,  LDC,
                          WORK, LWORK, INFO )

           CHARACTER      SIDE, TRANS, VECT

           INTEGER        INFO, K, LDA, LDC, LWORK, M, N

           DOUBLE         PRECISION  A( LDA, * ), C( LDC, * ), TAU( * ), WORK(
                          * )

  Purpose
  =======

  If VECT = 'Q', DORMBR overwrites the general real M-by-N matrix C
  with
                  SIDE = 'L'     SIDE = 'R'
  TRANS = 'N':      Q * C          C * Q
  TRANS = 'T':      Q**T * C       C * Q**T

  If VECT = 'P', DORMBR overwrites the general real M-by-N matrix C
  with
                  SIDE = 'L'     SIDE = 'R'
  TRANS = 'N':      P * C          C * P
  TRANS = 'T':      P**T * C       C * P**T

  Here Q and P**T are the orthogonal matrices determined by DGEBRD when
  reducing a real matrix A to bidiagonal form: A = Q * B * P**T. Q and
  P**T are defined as products of elementary reflectors H(i) and G(i)
  respectively.

  Let nq = m if SIDE = 'L' and nq = n if SIDE = 'R'. Thus nq is the
  order of the orthogonal matrix Q or P**T that is applied.

  If VECT = 'Q', A is assumed to have been an NQ-by-K matrix:
  if nq >= k, Q = H(1) H(2) . . . H(k);
```

```
if nq < k, Q = H(1) H(2) . . . H(nq-1).

If VECT = 'P', A is assumed to have been a K-by-NQ matrix:
if k < nq, P = G(1) G(2) . . . G(k);
if k >= nq, P = G(1) G(2) . . . G(nq-1).

Arguments
=========

VECT    (input) CHARACTER*1
        = 'Q': apply Q or Q**T;
        = 'P': apply P or P**T.

SIDE    (input) CHARACTER*1
        = 'L': apply Q, Q**T, P or P**T from the Left;
        = 'R': apply Q, Q**T, P or P**T from the Right.

TRANS   (input) CHARACTER*1
        = 'N':  No transpose, apply Q  or P;
        = 'T':  Transpose, apply Q**T or P**T.

M       (input) INTEGER
        The number of rows of the matrix C. M >= 0.

N       (input) INTEGER
        The number of columns of the matrix C. N >= 0.

K       (input) INTEGER
        If VECT = 'Q', the number of columns in the original
        matrix reduced by DGEBRD.
        If VECT = 'P', the number of rows in the original
        matrix reduced by DGEBRD.
        K >= 0.

A       (input) DOUBLE PRECISION array, dimension
                              (LDA,min(nq,K)) if VECT = 'Q'
                              (LDA,nq)        if VECT = 'P'
        The vectors which define the elementary reflectors H(i) and
        G(i), whose products determine the matrices Q and P, as
        returned by DGEBRD.

LDA     (input) INTEGER
        The leading dimension of the array A.
        If VECT = 'Q', LDA >= max(1,nq);
        if VECT = 'P', LDA >= max(1,min(nq,K)).

TAU     (input) DOUBLE PRECISION array, dimension (min(nq,K))
        TAU(i) must contain the scalar factor of the elementary
        reflector H(i) or G(i) which determines Q or P, as returned
        by DGEBRD in the array argument TAUQ or TAUP.
```

```
 C       (input/output) DOUBLE PRECISION array, dimension (LDC,N)
         On entry, the M-by-N matrix C.
         On exit, C is overwritten by Q*C or Q**T*C or C*Q**T or C*Q
         or P*C or P**T*C or C*P or C*P**T.

 LDC     (input) INTEGER
         The leading dimension of the array C. LDC >= max(1,M).

 WORK    (workspace/output) DOUBLE PRECISION array, dimension (LWORK)
         On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 LWORK   (input) INTEGER
         The dimension of the array WORK.
         If SIDE = 'L', LWORK >= max(1,N);
         if SIDE = 'R', LWORK >= max(1,M).
         For optimum performance LWORK >= N*NB if SIDE = 'L', and
         LWORK >= M*NB if SIDE = 'R', where NB is the optimal
         blocksize.

         If LWORK = -1, then a workspace query is assumed; the routine
         only calculates the optimal size of the WORK array, returns
         this value as the first entry of the WORK array, and no error
         message related to LWORK is issued by XERBLA.

 INFO    (output) INTEGER
         = 0:  successful exit
         < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

90.201.3 fortran code

— dormbr.f —

```
      SUBROUTINE DORMBR( VECT, SIDE, TRANS, M, N, K, A, LDA, TAU, C,
     $                   LDC, WORK, LWORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      CHARACTER          SIDE, TRANS, VECT
      INTEGER            INFO, K, LDA, LDC, LWORK, M, N
*     ..
```

```
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), C( LDC, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            APPLYQ, LEFT, LQUERY, NOTRAN
      CHARACTER          TRANST
      INTEGER            I1, I2, IINFO, LWKOPT, MI, NB, NI, NQ, NW
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DORMLQ, DORMQR, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      APPLYQ = LSAME( VECT, 'Q' )
      LEFT = LSAME( SIDE, 'L' )
      NOTRAN = LSAME( TRANS, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
*
*     NQ is the order of Q or P and NW is the minimum dimension of WORK
*
      IF( LEFT ) THEN
         NQ = M
         NW = N
      ELSE
         NQ = N
         NW = M
      END IF
      IF( .NOT.APPLYQ .AND. .NOT.LSAME( VECT, 'P' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.LEFT .AND. .NOT.LSAME( SIDE, 'R' ) ) THEN
         INFO = -2
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) ) THEN
         INFO = -3
      ELSE IF( M.LT.0 ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
```

```
         INFO = -5
      ELSE IF( K.LT.0 ) THEN
         INFO = -6
      ELSE IF( ( APPLYQ .AND. LDA.LT.MAX( 1, NQ ) ) .OR.
     $         ( .NOT.APPLYQ .AND. LDA.LT.MAX( 1, MIN( NQ, K ) ) ) )
     $          THEN
         INFO = -8
      ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
         INFO = -11
      ELSE IF( LWORK.LT.MAX( 1, NW ) .AND. .NOT.LQUERY ) THEN
         INFO = -13
      END IF
*
      IF( INFO.EQ.0 ) THEN
         IF( APPLYQ ) THEN
            IF( LEFT ) THEN
               NB = ILAENV( 1, 'DORMQR', SIDE // TRANS, M-1, N, M-1,
     $              -1 )
            ELSE
               NB = ILAENV( 1, 'DORMQR', SIDE // TRANS, M, N-1, N-1,
     $              -1 )
            END IF
         ELSE
            IF( LEFT ) THEN
               NB = ILAENV( 1, 'DORMLQ', SIDE // TRANS, M-1, N, M-1,
     $              -1 )
            ELSE
               NB = ILAENV( 1, 'DORMLQ', SIDE // TRANS, M, N-1, N-1,
     $              -1 )
            END IF
         END IF
         LWKOPT = MAX( 1, NW )*NB
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DORMBR', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      WORK( 1 ) = 1
      IF( M.EQ.0 .OR. N.EQ.0 )
     $   RETURN
*
      IF( APPLYQ ) THEN
*
```

```
*        Apply Q
*
         IF( NQ.GE.K ) THEN
*
*           Q was determined by a call to DGEBRD with nq >= k
*
            CALL DORMQR( SIDE, TRANS, M, N, K, A, LDA, TAU, C, LDC,
     $                   WORK, LWORK, IINFO )
         ELSE IF( NQ.GT.1 ) THEN
*
*           Q was determined by a call to DGEBRD with nq < k
*
            IF( LEFT ) THEN
               MI = M - 1
               NI = N
               I1 = 2
               I2 = 1
            ELSE
               MI = M
               NI = N - 1
               I1 = 1
               I2 = 2
            END IF
            CALL DORMQR( SIDE, TRANS, MI, NI, NQ-1, A( 2, 1 ), LDA, TAU,
     $                   C( I1, I2 ), LDC, WORK, LWORK, IINFO )
         END IF
      ELSE
*
*        Apply P
*
         IF( NOTRAN ) THEN
            TRANST = 'T'
         ELSE
            TRANST = 'N'
         END IF
         IF( NQ.GT.K ) THEN
*
*           P was determined by a call to DGEBRD with nq > k
*
            CALL DORMLQ( SIDE, TRANST, M, N, K, A, LDA, TAU, C, LDC,
     $                   WORK, LWORK, IINFO )
         ELSE IF( NQ.GT.1 ) THEN
*
*           P was determined by a call to DGEBRD with nq <= k
*
            IF( LEFT ) THEN
               MI = M - 1
               NI = N
               I1 = 2
               I2 = 1
```

```
            ELSE
               MI = M
               NI = N - 1
               I1 = 1
               I2 = 2
            END IF
            CALL DORMLQ( SIDE, TRANST, MI, NI, NQ-1, A( 1, 2 ), LDA,
     $                   TAU, C( I1, I2 ), LDC, WORK, LWORK, IINFO )
         END IF
      END IF
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of DORMBR
*
      END
```

———-

90.201.4 lisp code

— dormbr.lisp —

```
(defun dormbr (vect side trans m n k a lda tau c ldc work lwork info)
  (declare (type (array double-float (*)) work c tau a)
           (type (f2cl-lib:integer4) info lwork ldc lda k n m)
           (type (string *) trans side vect))
  (f2cl-lib:with-multi-array-data
      ((vect character vect-%data% vect-%offset%)
       (side character side-%data% side-%offset%)
       (trans character trans-%data% trans-%offset%)
       (a double-float a-%data% a-%offset%)
       (tau double-float tau-%data% tau-%offset%)
       (c double-float c-%data% c-%offset%)
       (work double-float work-%data% work-%offset%))
    (prog ((i1 0) (i2 0) (iinfo 0) (lwkopt 0) (mi 0) (nb 0) (ni 0) (nq 0)
           (nw 0)
           (transt
            (make-array '(1) :element-type 'character :initial-element #\ ))
           (applyq nil) (left nil) (lquery nil) (notran nil))
      (declare (type f2cl-lib:logical notran lquery left applyq)
               (type (string 1) transt)
               (type (f2cl-lib:integer4) nw nq ni nb mi lwkopt iinfo i2 i1))
      (setf info 0)
      (setf applyq
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame vect "Q")
```

```
          (declare (ignore var-1))
          (when var-0 (setf vect var-0))
          ret-val))
(setf left
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "L")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val))
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond (left (setf nq m) (setf nw n)) (t (setf nq n) (setf nw m)))
(cond
 ((and (not applyq)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame vect "P")
          (declare (ignore var-1))
          (when var-0 (setf vect var-0))
          ret-val)))
  (setf info -1))
 ((and (not left)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "R")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val)))
  (setf info -2))
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -3))
 ((< m 0) (setf info -4)) ((< n 0) (setf info -5))
 ((< k 0) (setf info -6))
 ((or
   (and applyq
        (< lda
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nq))))
   (and (not applyq)
        (< lda
```

```
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4
                        (min (the f2cl-lib:integer4 nq)
                             (the f2cl-lib:integer4 k)))))))
       (setf info -8))
      ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
       (setf info -11))
      ((and
        (< lwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nw)))
        (not lquery))
       (setf info -13)))
     (cond
      ((= info 0)
       (cond
        (applyq
         (cond
          (left
           (setf nb
                   (multiple-value-bind
                       (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                       (ilaenv 1 "DORMQR" (f2cl-lib:f2cl-// side trans)
                        (f2cl-lib:int-sub m 1) n (f2cl-lib:int-sub m 1) -1)
                     (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6))
                     (when var-4 (setf n var-4))
                     ret-val)))
          (t
           (setf nb
                   (multiple-value-bind
                       (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                       (ilaenv 1 "DORMQR" (f2cl-lib:f2cl-// side trans) m
                        (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1) -1)
                     (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6))
                     (when var-3 (setf m var-3))
                     ret-val)))))
        (t
         (cond
          (left
           (setf nb
                   (multiple-value-bind
                       (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                       (ilaenv 1 "DORMLQ" (f2cl-lib:f2cl-// side trans)
                        (f2cl-lib:int-sub m 1) n (f2cl-lib:int-sub m 1) -1)
                     (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6))
                     (when var-4 (setf n var-4))
                     ret-val)))
          (t
           (setf nb
                   (multiple-value-bind
                       (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                       (ilaenv 1 "DORMLQ" (f2cl-lib:f2cl-// side trans) m
```

```
                    (f2cl-lib:int-sub n 1) (f2cl-lib:int-sub n 1) -1)
                 (declare (ignore var-0 var-1 var-2 var-4 var-5 var-6))
                 (when var-3 (setf m var-3))
                 ret-val))))))
  (setf lwkopt
          (f2cl-lib:int-mul
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nw))
           nb))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))))
(cond
 ((/= info 0) (xerbla "DORMBR" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce (the f2cl-lib:integer4 1) 'double-float))
(if (or (= m 0) (= n 0))
    (go end_label))
(cond
 (applyq
  (cond
   ((>= nq k)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (dormqr side trans m n k a lda tau c ldc work lwork iinfo)
      (declare (ignore var-5 var-7 var-8 var-10))
      (when var-0 (setf side var-0))
      (when var-1 (setf trans var-1))
      (when var-2 (setf m var-2))
      (when var-3 (setf n var-3))
      (when var-4 (setf k var-4))
      (when var-6 (setf lda var-6))
      (when var-9 (setf ldc var-9))
      (when var-11 (setf lwork var-11))
      (when var-12 (setf iinfo var-12))))
   ((> nq 1)
    (cond
     (left (setf mi (f2cl-lib:int-sub m 1)) (setf ni n) (setf i1 2)
      (setf i2 1))
     (t (setf mi m) (setf ni (f2cl-lib:int-sub n 1)) (setf i1 1)
      (setf i2 2)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10 var-11 var-12)
        (dormqr side trans mi ni (f2cl-lib:int-sub nq 1)
         (f2cl-lib:array-slice a-%data% double-float (2 1)
                               ((1 lda) (1 *)) a-%offset%)
         lda tau
         (f2cl-lib:array-slice c-%data% double-float (i1 i2)
                               ((1 ldc) (1 *)) c-%offset%)
```

```
          ldc work lwork iinfo)
        (declare (ignore var-4 var-5 var-7 var-8 var-10))
        (when var-0 (setf side var-0))
        (when var-1 (setf trans var-1))
        (when var-2 (setf mi var-2))
        (when var-3 (setf ni var-3))
        (when var-6 (setf lda var-6))
        (when var-9 (setf ldc var-9))
        (when var-11 (setf lwork var-11))
        (when var-12 (setf iinfo var-12))))))
(t
 (cond (notran (f2cl-lib:f2cl-set-string transt "T" (string 1)))
       (t (f2cl-lib:f2cl-set-string transt "N" (string 1))))
 (cond
  ((> nq k)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12)
       (dormlq side transt m n k a lda tau c ldc work lwork iinfo)
     (declare (ignore var-5 var-7 var-8 var-10))
     (when var-0 (setf side var-0))
     (when var-1 (setf transt var-1))
     (when var-2 (setf m var-2))
     (when var-3 (setf n var-3))
     (when var-4 (setf k var-4))
     (when var-6 (setf lda var-6))
     (when var-9 (setf ldc var-9))
     (when var-11 (setf lwork var-11))
     (when var-12 (setf iinfo var-12))))
  ((> nq 1)
   (cond
    (left (setf mi (f2cl-lib:int-sub m 1)) (setf ni n) (setf i1 2)
     (setf i2 1))
    (t (setf mi m) (setf ni (f2cl-lib:int-sub n 1)) (setf i1 1)
     (setf i2 2)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12)
       (dormlq side transt mi ni (f2cl-lib:int-sub nq 1)
        (f2cl-lib:array-slice a-%data% double-float (1 2)
                              ((1 lda) (1 *)) a-%offset%)
        lda tau
        (f2cl-lib:array-slice c-%data% double-float (i1 i2)
                              ((1 ldc) (1 *)) c-%offset%)
        ldc work lwork iinfo)
     (declare (ignore var-4 var-5 var-7 var-8 var-10))
     (when var-0 (setf side var-0))
     (when var-1 (setf transt var-1))
     (when var-2 (setf mi var-2))
     (when var-3 (setf ni var-3))
```

```
            (when var-6 (setf lda var-6))
            (when var-9 (setf ldc var-9))
            (when var-11 (setf lwork var-11))
            (when var-12 (setf iinfo var-12)))))))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
      (go end_label)
     end_label
      (return
       (values vect side trans m n k nil lda nil nil ldc nil lwork info)))))
```

———-

90.202 dorml2

90.202.1 Axiom unit tests

— **dorml2.input** —

```
)set break resume
)sys rm -f dorml2.output
)spool dorml2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.202.2 Axiom help page

— **dorml2.help** —

```
==========================================================================
dorml2 examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
NAME
       DORML2  -  the  general real m by n matrix C with   Q * C if SIDE = 'L'
       and TRANS = 'N', or   Q'* C if SIDE = 'L' and TRANS = 'T', or   C  *  Q
       if  SIDE  =  'R' and TRANS = 'N', or   C * Q' if SIDE = 'R' and TRANS =
       'T',

SYNOPSIS
       SUBROUTINE DORML2( SIDE, TRANS, M, N, K, A, LDA,  TAU,  C,  LDC,  WORK,
                          INFO )

           CHARACTER      SIDE, TRANS

           INTEGER        INFO, K, LDA, LDC, M, N

           DOUBLE         PRECISION  A( LDA, * ), C( LDC, * ), TAU( * ), WORK(
                          * )

  Purpose
  =======

  DORML2 overwrites the general real m by n matrix C with

        Q * C  if SIDE = 'L' and TRANS = 'N', or

        Q'* C  if SIDE = 'L' and TRANS = 'T', or

        C * Q  if SIDE = 'R' and TRANS = 'N', or

        C * Q' if SIDE = 'R' and TRANS = 'T',

  where Q is a real orthogonal matrix defined as the product of k
  elementary reflectors

        Q = H(k) . . . H(2) H(1)

  as returned by DGELQF. Q is of order m if SIDE = 'L' and of order n
  if SIDE = 'R'.

  Arguments
  =========

  SIDE    (input) CHARACTER*1
          = 'L': apply Q or Q' from the Left
          = 'R': apply Q or Q' from the Right

  TRANS   (input) CHARACTER*1
          = 'N': apply Q  (No transpose)
          = 'T': apply Q' (Transpose)

  M       (input) INTEGER
```

```
         The number of rows of the matrix C. M >= 0.

N        (input) INTEGER
         The number of columns of the matrix C. N >= 0.

K        (input) INTEGER
         The number of elementary reflectors whose product defines
         the matrix Q.
         If SIDE = 'L', M >= K >= 0;
         if SIDE = 'R', N >= K >= 0.

A        (input) DOUBLE PRECISION array, dimension
                             (LDA,M) if SIDE = 'L',
                             (LDA,N) if SIDE = 'R'
         The i-th row must contain the vector which defines the
         elementary reflector H(i), for i = 1,2,...,k, as returned by
         DGELQF in the first k rows of its array argument A.
         A is modified by the routine but restored on exit.

LDA      (input) INTEGER
         The leading dimension of the array A. LDA >= max(1,K).

TAU      (input) DOUBLE PRECISION array, dimension (K)
         TAU(i) must contain the scalar factor of the elementary
         reflector H(i), as returned by DGELQF.

C        (input/output) DOUBLE PRECISION array, dimension (LDC,N)
         On entry, the m by n matrix C.
         On exit, C is overwritten by Q*C or Q'*C or C*Q' or C*Q.

LDC      (input) INTEGER
         The leading dimension of the array C. LDC >= max(1,M).

WORK     (workspace) DOUBLE PRECISION array, dimension
                                  (N) if SIDE = 'L',
                                  (M) if SIDE = 'R'

INFO     (output) INTEGER
         = 0: successful exit
         < 0: if INFO = -i, the i-th argument had an illegal value
```

———-

90.202.3 fortran code

— dorml2.f —

```
      SUBROUTINE DORML2( SIDE, TRANS, M, N, K, A, LDA, TAU, C, LDC,
     $                   WORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     February 29, 1992
*
*     .. Scalar Arguments ..
      CHARACTER          SIDE, TRANS
      INTEGER            INFO, K, LDA, LDC, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), C( LDC, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LEFT, NOTRAN
      INTEGER            I, I1, I2, I3, IC, JC, MI, NI, NQ
      DOUBLE PRECISION   AII
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARF, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LEFT = LSAME( SIDE, 'L' )
      NOTRAN = LSAME( TRANS, 'N' )
*
*     NQ is the order of Q
*
      IF( LEFT ) THEN
         NQ = M
      ELSE
```

```
         NQ = N
      END IF
      IF( .NOT.LEFT .AND. .NOT.LSAME( SIDE, 'R' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) ) THEN
         INFO = -2
      ELSE IF( M.LT.0 ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( K.LT.0 .OR. K.GT.NQ ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, K ) ) THEN
         INFO = -7
      ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
         INFO = -10
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DORML2', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 .OR. K.EQ.0 )
     $   RETURN
*
      IF( ( LEFT .AND. NOTRAN ) .OR. ( .NOT.LEFT .AND. .NOT.NOTRAN ) )
     $     THEN
         I1 = 1
         I2 = K
         I3 = 1
      ELSE
         I1 = K
         I2 = 1
         I3 = -1
      END IF
*
      IF( LEFT ) THEN
         NI = N
         JC = 1
      ELSE
         MI = M
         IC = 1
      END IF
*
      DO 10 I = I1, I2, I3
         IF( LEFT ) THEN
*
*           H(i) is applied to C(i:m,1:n)
```

```
*
           MI = M - I + 1
           IC = I
        ELSE
*
*          H(i) is applied to C(1:m,i:n)
*
           NI = N - I + 1
           JC = I
        END IF
*
*       Apply H(i)
*
        AII = A( I, I )
        A( I, I ) = ONE
        CALL DLARF( SIDE, MI, NI, A( I, I ), LDA, TAU( I ),
     $              C( IC, JC ), LDC, WORK )
        A( I, I ) = AII
   10 CONTINUE
      RETURN
*
*     End of DORML2
*
      END
```

———-

90.202.4 lisp code

— **dorml2.lisp** —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun dorml2 (side trans m n k a lda tau c ldc work info)
    (declare (type (array double-float (*)) work c tau a)
             (type (f2cl-lib:integer4) info ldc lda k n m)
             (type (string *) trans side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (trans character trans-%data% trans-%offset%)
         (a double-float a-%data% a-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (c double-float c-%data% c-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((aii 0.0d0) (i 0) (i1 0) (i2 0) (i3 0) (ic 0) (jc 0) (mi 0) (ni 0)
             (nq 0) (left nil) (notran nil))
```

```
(declare (type (double-float) aii)
         (type (f2cl-lib:integer4) i i1 i2 i3 ic jc mi ni nq)
         (type f2cl-lib:logical left notran))
(setf info 0)
(setf left
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "L")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val))
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(cond (left (setf nq m)) (t (setf nq n)))
(cond
 ((and (not left)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "R")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val)))
  (setf info -1))
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -2))
 ((< m 0) (setf info -3)) ((< n 0) (setf info -4))
 ((or (< k 0) (> k nq)) (setf info -5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 k)))
  (setf info -7))
 ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -10)))
(cond
 ((/= info 0) (xerbla "DORML2" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= m 0) (= n 0) (= k 0))
    (go end_label))
(cond
 ((or (and left notran) (and (not left) (not notran))) (setf i1 1)
  (setf i2 k) (setf i3 1))
 (t (setf i1 k) (setf i2 1) (setf i3 -1)))
(cond (left (setf ni n) (setf jc 1)) (t (setf mi m) (setf ic 1)))
```

```
(f2cl-lib:fdo (i i1 (f2cl-lib:int-add i i3))
              ((> i i2) nil)
  (tagbody
    (cond
     (left (setf mi (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
      (setf ic i))
     (t (setf ni (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
      (setf jc i)))
    (setf aii
            (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
            one)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (dlarf side mi ni
         (f2cl-lib:array-slice a-%data% double-float (i i)
                               ((1 lda) (1 *)) a-%offset%)
         lda (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%)
         (f2cl-lib:array-slice c-%data% double-float (ic jc)
                               ((1 ldc) (1 *)) c-%offset%)
         ldc work)
      (declare (ignore var-3 var-5 var-6 var-8))
      (setf side var-0)
      (setf mi var-1)
      (setf ni var-2)
      (setf lda var-4)
      (setf ldc var-7))
    (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
            aii)
   label10))
 (go end_label)
end_label
 (return
  (values side trans nil nil nil nil lda nil nil ldc nil info))))))
```

———-

90.203 dormlq

90.203.1 Axiom unit tests

— **dormlq.input** —

```
)set break resume
)sys rm -f dormlq.output
)spool dormlq.output
```

```
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.203.2 Axiom help page

— **dormlq.help** —

```
==================================================================
dormlq examples
==================================================================

==================================================================
Man Page Details
==================================================================

NAME
       DORMLQ  - the general real M-by-N matrix C with   SIDE = 'L' SIDE = 'R'
       TRANS = 'N'

SYNOPSIS
       SUBROUTINE DORMLQ( SIDE, TRANS, M, N, K, A, LDA,  TAU,  C,  LDC,  WORK,
                          LWORK, INFO )

           CHARACTER      SIDE, TRANS

           INTEGER        INFO, K, LDA, LDC, LWORK, M, N

           DOUBLE         PRECISION  A( LDA, * ), C( LDC, * ), TAU( * ), WORK(
                          * )

  Purpose
  =======

  DORMLQ overwrites the general real M-by-N matrix C with

                  SIDE = 'L'     SIDE = 'R'
  TRANS = 'N':      Q * C          C * Q
  TRANS = 'T':      Q**T * C       C * Q**T

  where Q is a real orthogonal matrix defined as the product of k
  elementary reflectors
```

```
      Q = H(k) . . . H(2) H(1)

as returned by DGELQF. Q is of order M if SIDE = 'L' and of order N
if SIDE = 'R'.

Arguments
=========

SIDE    (input) CHARACTER*1
        = 'L': apply Q or Q**T from the Left;
        = 'R': apply Q or Q**T from the Right.

TRANS   (input) CHARACTER*1
        = 'N':  No transpose, apply Q;
        = 'T':  Transpose, apply Q**T.

M       (input) INTEGER
        The number of rows of the matrix C. M >= 0.

N       (input) INTEGER
        The number of columns of the matrix C. N >= 0.

K       (input) INTEGER
        The number of elementary reflectors whose product defines
        the matrix Q.
        If SIDE = 'L', M >= K >= 0;
        if SIDE = 'R', N >= K >= 0.

A       (input) DOUBLE PRECISION array, dimension
                              (LDA,M) if SIDE = 'L',
                              (LDA,N) if SIDE = 'R'
        The i-th row must contain the vector which defines the
        elementary reflector H(i), for i = 1,2,...,k, as returned by
        DGELQF in the first k rows of its array argument A.
        A is modified by the routine but restored on exit.

LDA     (input) INTEGER
        The leading dimension of the array A. LDA >= max(1,K).

TAU     (input) DOUBLE PRECISION array, dimension (K)
        TAU(i) must contain the scalar factor of the elementary
        reflector H(i), as returned by DGELQF.

C       (input/output) DOUBLE PRECISION array, dimension (LDC,N)
        On entry, the M-by-N matrix C.
        On exit, C is overwritten by Q*C or Q**T*C or C*Q**T or C*Q.

LDC     (input) INTEGER
        The leading dimension of the array C. LDC >= max(1,M).
```

```
  WORK    (workspace/output) DOUBLE PRECISION array, dimension (LWORK)
          On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

  LWORK   (input) INTEGER
          The dimension of the array WORK.
          If SIDE = 'L', LWORK >= max(1,N);
          if SIDE = 'R', LWORK >= max(1,M).
          For optimum performance LWORK >= N*NB if SIDE = 'L', and
          LWORK >= M*NB if SIDE = 'R', where NB is the optimal
          blocksize.

          If LWORK = -1, then a workspace query is assumed; the routine
          only calculates the optimal size of the WORK array, returns
          this value as the first entry of the WORK array, and no error
          message related to LWORK is issued by XERBLA.

  INFO    (output) INTEGER
          = 0:  successful exit
          < 0:  if INFO = -i, the i-th argument had an illegal value
```

90.203.3 fortran code

— dormlq.f —

```
      SUBROUTINE DORMLQ( SIDE, TRANS, M, N, K, A, LDA, TAU, C, LDC,
     $                   WORK, LWORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      CHARACTER          SIDE, TRANS
      INTEGER            INFO, K, LDA, LDC, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), C( LDC, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            NBMAX, LDT
      PARAMETER          ( NBMAX = 64, LDT = NBMAX+1 )
```

```
*     ..
*     .. Local Scalars ..
      LOGICAL            LEFT, LQUERY, NOTRAN
      CHARACTER          TRANST
      INTEGER            I, I1, I2, I3, IB, IC, IINFO, IWS, JC, LDWORK,
     $                   LWKOPT, MI, NB, NBMIN, NI, NQ, NW
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   T( LDT, NBMAX )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARFB, DLARFT, DORML2, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LEFT = LSAME( SIDE, 'L' )
      NOTRAN = LSAME( TRANS, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
*
*     NQ is the order of Q and NW is the minimum dimension of WORK
*
      IF( LEFT ) THEN
         NQ = M
         NW = N
      ELSE
         NQ = N
         NW = M
      END IF
      IF( .NOT.LEFT .AND. .NOT.LSAME( SIDE, 'R' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) ) THEN
         INFO = -2
      ELSE IF( M.LT.0 ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( K.LT.0 .OR. K.GT.NQ ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, K ) ) THEN
```

```
         INFO = -7
      ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
         INFO = -10
      ELSE IF( LWORK.LT.MAX( 1, NW ) .AND. .NOT.LQUERY ) THEN
         INFO = -12
      END IF
*
      IF( INFO.EQ.0 ) THEN
*
*        Determine the block size.  NB may be at most NBMAX, where NBMAX
*        is used to define the local array T.
*
         NB = MIN( NBMAX, ILAENV( 1, 'DORMLQ', SIDE // TRANS, M, N, K,
     $        -1 ) )
         LWKOPT = MAX( 1, NW )*NB
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DORMLQ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 .OR. K.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      LDWORK = NW
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
         IWS = NW*NB
         IF( LWORK.LT.IWS ) THEN
            NB = LWORK / LDWORK
            NBMIN = MAX( 2, ILAENV( 2, 'DORMLQ', SIDE // TRANS, M, N, K,
     $              -1 ) )
         END IF
      ELSE
         IWS = NW
      END IF
*
      IF( NB.LT.NBMIN .OR. NB.GE.K ) THEN
*
*        Use unblocked code
*
         CALL DORML2( SIDE, TRANS, M, N, K, A, LDA, TAU, C, LDC, WORK,
```

```
     $                IINFO )
      ELSE
*
*        Use blocked code
*
         IF( ( LEFT .AND. NOTRAN ) .OR.
     $       ( .NOT.LEFT .AND. .NOT.NOTRAN ) ) THEN
            I1 = 1
            I2 = K
            I3 = NB
         ELSE
            I1 = ( ( K-1 ) / NB )*NB + 1
            I2 = 1
            I3 = -NB
         END IF
*
         IF( LEFT ) THEN
            NI = N
            JC = 1
         ELSE
            MI = M
            IC = 1
         END IF
*
         IF( NOTRAN ) THEN
            TRANST = 'T'
         ELSE
            TRANST = 'N'
         END IF
*
         DO 10 I = I1, I2, I3
            IB = MIN( NB, K-I+1 )
*
*           Form the triangular factor of the block reflector
*           H = H(i) H(i+1) . . . H(i+ib-1)
*
            CALL DLARFT( 'Forward', 'Rowwise', NQ-I+1, IB, A( I, I ),
     $                   LDA, TAU( I ), T, LDT )
            IF( LEFT ) THEN
*
*              H or H' is applied to C(i:m,1:n)
*
               MI = M - I + 1
               IC = I
            ELSE
*
*              H or H' is applied to C(1:m,i:n)
*
               NI = N - I + 1
               JC = I
```

```
            END IF
*
*           Apply H or H'
*
            CALL DLARFB( SIDE, TRANST, 'Forward', 'Rowwise', MI, NI, IB,
     $                   A( I, I ), LDA, T, LDT, C( IC, JC ), LDC, WORK,
     $                   LDWORK )
   10    CONTINUE
      END IF
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of DORMLQ
*
      END
```

———-

90.203.4 lisp code

— dormlq.lisp —

```
(let* ((nbmax 64) (ldt (+ nbmax 1)) (tsize (* ldt nbmax)))
  (declare (type (f2cl-lib:integer4 64 64) nbmax)
           (type (f2cl-lib:integer4) ldt)
           (type (f2cl-lib:integer4) tsize)
           (ignorable nbmax ldt tsize))
  (defun dormlq (side trans m n k a lda tau c ldc work lwork info)
    (declare (type (array double-float (*)) work c tau a)
             (type (f2cl-lib:integer4) info lwork ldc lda k n m)
             (type (string *) trans side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (trans character trans-%data% trans-%offset%)
         (a double-float a-%data% a-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (c double-float c-%data% c-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((i 0) (i1 0) (i2 0) (i3 0) (ib 0) (ic 0) (iinfo 0) (iwt 0) (jc 0)
             (ldwork 0) (lwkopt 0) (mi 0) (nb 0) (nbmin 0) (ni 0) (nq 0) (nw 0)
             (transt
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (left nil) (lquery nil) (notran nil))
        (declare
         (type (f2cl-lib:integer4) i i1 i2 i3 ib ic iinfo iwt jc ldwork lwkopt
          mi nb nbmin ni nq nw)
         (type (string 1) transt)
```

```
 (type f2cl-lib:logical left lquery notran))
(setf info 0)
(setf left
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "L")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val))
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond (left (setf nq m) (setf nw n)) (t (setf nq n) (setf nw m)))
(cond
 ((and (not left)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "R")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val)))
  (setf info -1))
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -2))
 ((< m 0) (setf info -3)) ((< n 0) (setf info -4))
 ((or (< k 0) (> k nq)) (setf info -5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 k)))
  (setf info -7))
 ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -10))
 ((and
   (< lwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nw)))
   (not lquery))
  (setf info -12)))
(cond
 ((= info 0)
  (setf nb
          (min (the f2cl-lib:integer4 nbmax)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
```

```
                          var-6)
                         (ilaenv 1 "DORMLQ"
                          (f2cl-lib:f2cl-// side trans) m n k -1)
                       (declare (ignore var-0 var-1 var-2 var-6))
                       (when var-3 (setf m var-3))
                       (when var-4 (setf n var-4))
                       (when var-5 (setf k var-5))
                       ret-val))))
  (setf lwkopt
          (f2cl-lib:int-add
           (f2cl-lib:int-mul
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nw))
            nb)
           tsize))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))))
(cond
 ((/= info 0) (xerbla "DORMLQ" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond
 ((or (= m 0) (= n 0) (= k 0))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'double-float))
  (go end_label)))
(setf nbmin 2)
(setf ldwork nw)
(cond
 ((and (> nb 1) (< nb k))
  (cond
   ((< lwork (f2cl-lib:int-add (f2cl-lib:int-mul nw nb) tsize))
    (setf nb (the f2cl-lib:integer4 (truncate (- lwork tsize) ldwork)))
    (setf nbmin
            (max (the f2cl-lib:integer4 2)
                 (the f2cl-lib:integer4
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
                          (ilaenv 2 "DORMLQ"
                           (f2cl-lib:f2cl-// side trans) m n k -1)
                        (declare (ignore var-0 var-1 var-2 var-6))
                        (when var-3 (setf m var-3))
                        (when var-4 (setf n var-4))
                        (when var-5 (setf k var-5))
                        ret-val))))))))
(cond
 ((or (< nb nbmin) (>= nb k))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11)
      (dorml2 side trans m n k a lda tau c ldc work iinfo)
```

```
    (declare (ignore var-2 var-3 var-4 var-5 var-7 var-8 var-10))
    (setf side var-0)
    (setf trans var-1)
    (setf lda var-6)
    (setf ldc var-9)
    (setf iinfo var-11)))
  (t (setf iwt (f2cl-lib:int-add 1 (f2cl-lib:int-mul nw nb)))
   (cond
    ((or (and left notran) (and (not left) (not notran))) (setf i1 1)
     (setf i2 k) (setf i3 nb))
    (t
     (setf i1
             (+ (* (the f2cl-lib:integer4 (truncate (- k 1) nb)) nb) 1))
     (setf i2 1) (setf i3 (f2cl-lib:int-sub nb))))
   (cond (left (setf ni n) (setf jc 1)) (t (setf mi m) (setf ic 1)))
   (cond (notran (f2cl-lib:f2cl-set-string transt "T" (string 1)))
         (t (f2cl-lib:f2cl-set-string transt "N" (string 1))))
   (f2cl-lib:fdo (i i1 (f2cl-lib:int-add i i3))
                 ((> i i2) nil)
     (tagbody
       (setf ib
               (min (the f2cl-lib:integer4 nb)
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
           (dlarft "Forward" "Rowwise"
            (f2cl-lib:int-add (f2cl-lib:int-sub nq i) 1) ib
            (f2cl-lib:array-slice a-%data% double-float (i i)
                                  ((1 lda) (1 *)) a-%offset%)
            lda
            (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                  tau-%offset%)
            (f2cl-lib:array-slice work-%data% double-float (iwt) ((1 *))
                                  work-%offset%)
            ldt)
         (declare (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7))
         (setf lda var-5)
         (setf ldt var-8))
       (cond
        (left (setf mi (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
         (setf ic i))
        (t (setf ni (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
         (setf jc i)))
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
            var-10 var-11 var-12 var-13 var-14)
           (dlarfb side transt "Forward" "Rowwise" mi ni ib
            (f2cl-lib:array-slice a-%data% double-float (i i)
                                  ((1 lda) (1 *)) a-%offset%)
```

```
                lda
                (f2cl-lib:array-slice work-%data% double-float (iwt) ((1 *))
                                      work-%offset%)
                ldt
                (f2cl-lib:array-slice c-%data% double-float (ic jc)
                                      ((1 ldc) (1 *)) c-%offset%)
                ldc work ldwork)
              (declare (ignore var-2 var-3 var-7 var-9 var-11 var-13))
              (setf side var-0)
              (setf transt var-1)
              (setf mi var-4)
              (setf ni var-5)
              (setf ib var-6)
              (setf lda var-8)
              (setf ldt var-10)
              (setf ldc var-12)
              (setf ldwork var-14))
            label10))))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
      (go end_label)
     end_label
      (return (values side trans m n k nil lda nil nil ldc nil nil info))))))
```

———-

90.204 dormqr

90.204.1 Axiom unit tests

— **dormqr.input** —

```
)set break resume
)sys rm -f dormqr.output
)spool dormqr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.204.2 Axiom help page

— dormqr.help —

```
==================================================================
dormqr examples
==================================================================

==================================================================
Man Page Details
==================================================================

NAME
       DORMQR  - the general real M-by-N matrix C with   SIDE = 'L' SIDE = 'R'
       TRANS = 'N'

SYNOPSIS
       SUBROUTINE DORMQR( SIDE, TRANS, M, N, K, A, LDA,  TAU,  C,  LDC,  WORK,
                          LWORK, INFO )

           CHARACTER      SIDE, TRANS

           INTEGER        INFO, K, LDA, LDC, LWORK, M, N

           DOUBLE         PRECISION  A( LDA, * ), C( LDC, * ), TAU( * ), WORK(
                          * )

  Purpose
  =======

  DORMQR overwrites the general real M-by-N matrix C with

                  SIDE = 'L'     SIDE = 'R'
  TRANS = 'N':      Q * C          C * Q
  TRANS = 'T':      Q**T * C       C * Q**T

  where Q is a real orthogonal matrix defined as the product of k
  elementary reflectors

        Q = H(1) H(2) . . . H(k)

  as returned by DGEQRF. Q is of order M if SIDE = 'L' and of order N
  if SIDE = 'R'.

  Arguments
  =========

  SIDE    (input) CHARACTER*1
          = 'L': apply Q or Q**T from the Left;
```

```
        = 'R': apply Q or Q**T from the Right.

TRANS   (input) CHARACTER*1
        = 'N':  No transpose, apply Q;
        = 'T':  Transpose, apply Q**T.

M       (input) INTEGER
        The number of rows of the matrix C. M >= 0.

N       (input) INTEGER
        The number of columns of the matrix C. N >= 0.

K       (input) INTEGER
        The number of elementary reflectors whose product defines
        the matrix Q.
        If SIDE = 'L', M >= K >= 0;
        if SIDE = 'R', N >= K >= 0.

A       (input) DOUBLE PRECISION array, dimension (LDA,K)
        The i-th column must contain the vector which defines the
        elementary reflector H(i), for i = 1,2,...,k, as returned by
        DGEQRF in the first k columns of its array argument A.
        A is modified by the routine but restored on exit.

LDA     (input) INTEGER
        The leading dimension of the array A.
        If SIDE = 'L', LDA >= max(1,M);
        if SIDE = 'R', LDA >= max(1,N).

TAU     (input) DOUBLE PRECISION array, dimension (K)
        TAU(i) must contain the scalar factor of the elementary
        reflector H(i), as returned by DGEQRF.

C       (input/output) DOUBLE PRECISION array, dimension (LDC,N)
        On entry, the M-by-N matrix C.
        On exit, C is overwritten by Q*C or Q**T*C or C*Q**T or C*Q.

LDC     (input) INTEGER
        The leading dimension of the array C. LDC >= max(1,M).

WORK    (workspace/output) DOUBLE PRECISION array, dimension (LWORK)
        On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

LWORK   (input) INTEGER
        The dimension of the array WORK.
        If SIDE = 'L', LWORK >= max(1,N);
        if SIDE = 'R', LWORK >= max(1,M).
        For optimum performance LWORK >= N*NB if SIDE = 'L', and
        LWORK >= M*NB if SIDE = 'R', where NB is the optimal
        blocksize.
```

```
         If LWORK = -1, then a workspace query is assumed; the routine
         only calculates the optimal size of the WORK array, returns
         this value as the first entry of the WORK array, and no error
         message related to LWORK is issued by XERBLA.

 INFO    (output) INTEGER
         = 0:  successful exit
         < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

90.204.3 fortran code

— **dormqr.f** —

```
      SUBROUTINE DORMQR( SIDE, TRANS, M, N, K, A, LDA, TAU, C, LDC,
     $                   WORK, LWORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      CHARACTER          SIDE, TRANS
      INTEGER            INFO, K, LDA, LDC, LWORK, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   A( LDA, * ), C( LDC, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            NBMAX, LDT
      PARAMETER          ( NBMAX = 64, LDT = NBMAX+1 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LEFT, LQUERY, NOTRAN
      INTEGER            I, I1, I2, I3, IB, IC, IINFO, IWS, JC, LDWORK,
     $                   LWKOPT, MI, NB, NBMIN, NI, NQ, NW
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   T( LDT, NBMAX )
*     ..
*     .. External Functions ..
```

```
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLARFB, DLARFT, DORM2R, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LEFT = LSAME( SIDE, 'L' )
      NOTRAN = LSAME( TRANS, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
*
*     NQ is the order of Q and NW is the minimum dimension of WORK
*
      IF( LEFT ) THEN
         NQ = M
         NW = N
      ELSE
         NQ = N
         NW = M
      END IF
      IF( .NOT.LEFT .AND. .NOT.LSAME( SIDE, 'R' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) ) THEN
         INFO = -2
      ELSE IF( M.LT.0 ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( K.LT.0 .OR. K.GT.NQ ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, NQ ) ) THEN
         INFO = -7
      ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
         INFO = -10
      ELSE IF( LWORK.LT.MAX( 1, NW ) .AND. .NOT.LQUERY ) THEN
         INFO = -12
      END IF
*
      IF( INFO.EQ.0 ) THEN
*
*        Determine the block size.  NB may be at most NBMAX, where NBMAX
*        is used to define the local array T.
```

```
*
         NB = MIN( NBMAX, ILAENV( 1, 'DORMQR', SIDE // TRANS, M, N, K,
     $        -1 ) )
         LWKOPT = MAX( 1, NW )*NB
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DORMQR', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 .OR. K.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      LDWORK = NW
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
         IWS = NW*NB
         IF( LWORK.LT.IWS ) THEN
            NB = LWORK / LDWORK
            NBMIN = MAX( 2, ILAENV( 2, 'DORMQR', SIDE // TRANS, M, N, K,
     $              -1 ) )
         END IF
      ELSE
         IWS = NW
      END IF
*
      IF( NB.LT.NBMIN .OR. NB.GE.K ) THEN
*
*        Use unblocked code
*
         CALL DORM2R( SIDE, TRANS, M, N, K, A, LDA, TAU, C, LDC, WORK,
     $                IINFO )
      ELSE
*
*        Use blocked code
*
         IF( ( LEFT .AND. .NOT.NOTRAN ) .OR.
     $       ( .NOT.LEFT .AND. NOTRAN ) ) THEN
            I1 = 1
            I2 = K
            I3 = NB
         ELSE
```

```
            I1 = ( ( K-1 ) / NB )*NB + 1
            I2 = 1
            I3 = -NB
         END IF
*
         IF( LEFT ) THEN
            NI = N
            JC = 1
         ELSE
            MI = M
            IC = 1
         END IF
*
         DO 10 I = I1, I2, I3
            IB = MIN( NB, K-I+1 )
*
*           Form the triangular factor of the block reflector
*           H = H(i) H(i+1) . . . H(i+ib-1)
*
            CALL DLARFT( 'Forward', 'Columnwise', NQ-I+1, IB, A( I, I ),
     $                   LDA, TAU( I ), T, LDT )
            IF( LEFT ) THEN
*
*              H or H' is applied to C(i:m,1:n)
*
               MI = M - I + 1
               IC = I
            ELSE
*
*              H or H' is applied to C(1:m,i:n)
*
               NI = N - I + 1
               JC = I
            END IF
*
*           Apply H or H'
*
            CALL DLARFB( SIDE, TRANS, 'Forward', 'Columnwise', MI, NI,
     $                   IB, A( I, I ), LDA, T, LDT, C( IC, JC ), LDC,
     $                   WORK, LDWORK )
   10    CONTINUE
      END IF
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of DORMQR
*
      END
```

———-

90.204.4 lisp code

— dormqr.lisp —

```
(let* ((nbmax 64) (ldt (+ nbmax 1)) (tsize (* ldt nbmax)))
  (declare (type (f2cl-lib:integer4 64 64) nbmax)
           (type (f2cl-lib:integer4) ldt)
           (type (f2cl-lib:integer4) tsize)
           (ignorable nbmax ldt tsize))
  (defun dormqr (side trans m n k a lda tau c ldc work lwork info)
    (declare (type (array double-float (*)) work c tau a)
             (type (f2cl-lib:integer4) info lwork ldc lda k n m)
             (type (string *) trans side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (trans character trans-%data% trans-%offset%)
         (a double-float a-%data% a-%offset%)
         (tau double-float tau-%data% tau-%offset%)
         (c double-float c-%data% c-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((i 0) (i1 0) (i2 0) (i3 0) (ib 0) (ic 0) (iinfo 0) (iwt 0) (jc 0)
             (ldwork 0) (lwkopt 0) (mi 0) (nb 0) (nbmin 0) (ni 0) (nq 0) (nw 0)
             (left nil) (lquery nil) (notran nil))
        (declare
         (type (f2cl-lib:integer4) i i1 i2 i3 ib ic iinfo iwt jc ldwork lwkopt
          mi nb nbmin ni nq nw)
         (type f2cl-lib:logical left lquery notran))
        (setf info 0)
        (setf left
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame side "L")
                  (declare (ignore var-1))
                  (when var-0 (setf side var-0))
                  ret-val))
        (setf notran
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame trans "N")
                  (declare (ignore var-1))
                  (when var-0 (setf trans var-0))
                  ret-val))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond (left (setf nq m) (setf nw n)) (t (setf nq n) (setf nw m)))
        (cond
         ((and (not left)
               (not
                (multiple-value-bind (ret-val var-0 var-1)
```

```
            (lsame side "R")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val)))
  (setf info -1))
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "T")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -2))
 ((< m 0) (setf info -3)) ((< n 0) (setf info -4))
 ((or (< k 0) (> k nq)) (setf info -5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nq)))
  (setf info -7))
 ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -10))
 ((and
   (< lwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nw)))
   (not lquery))
  (setf info -12)))
(cond
 ((= info 0)
  (setf nb
          (min (the f2cl-lib:integer4 nbmax)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 1 "DORMQR"
                         (f2cl-lib:f2cl-// side trans) m n k -1)
                      (declare (ignore var-0 var-1 var-2 var-6))
                      (when var-3 (setf m var-3))
                      (when var-4 (setf n var-4))
                      (when var-5 (setf k var-5))
                      ret-val))))
  (setf lwkopt
          (f2cl-lib:int-add
           (f2cl-lib:int-mul
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nw))
            nb)
           tsize))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))))
(cond
 ((/= info 0) (xerbla "DORMQR" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond
```

```
 ((or (= m 0) (= n 0) (= k 0))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce (the f2cl-lib:integer4 1) 'double-float))
  (go end_label)))
(setf nbmin 2)
(setf ldwork nw)
(cond
 ((and (> nb 1) (< nb k))
  (cond
   ((< lwork (f2cl-lib:int-add (f2cl-lib:int-mul nw nb) tsize))
    (setf nb (the f2cl-lib:integer4 (truncate (- lwork tsize) ldwork)))
    (setf nbmin
            (max (the f2cl-lib:integer4 2)
                 (the f2cl-lib:integer4
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
                          (ilaenv 2 "DORMQR"
                           (f2cl-lib:f2cl-// side trans) m n k -1)
                        (declare (ignore var-0 var-1 var-2 var-6))
                        (when var-3 (setf m var-3))
                        (when var-4 (setf n var-4))
                        (when var-5 (setf k var-5))
                        ret-val))))))))
(cond
 ((or (< nb nbmin) (>= nb k))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11)
      (dorm2r side trans m n k a lda tau c ldc work iinfo)
    (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-10))
    (setf side var-0)
    (setf trans var-1)
    (setf ldc var-9)
    (setf iinfo var-11)))
 (t (setf iwt (f2cl-lib:int-add 1 (f2cl-lib:int-mul nw nb)))
  (cond
   ((or (and left (not notran)) (and (not left) notran)) (setf i1 1)
    (setf i2 k) (setf i3 nb))
   (t
    (setf i1
            (+ (* (the f2cl-lib:integer4 (truncate (- k 1) nb)) nb) 1))
    (setf i2 1) (setf i3 (f2cl-lib:int-sub nb))))
  (cond (left (setf ni n) (setf jc 1)) (t (setf mi m) (setf ic 1)))
  (f2cl-lib:fdo (i i1 (f2cl-lib:int-add i i3))
                ((> i i2) nil)
    (tagbody
      (setf ib
              (min (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
```

```
                    (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (dlarft "Forward" "Columnwise"
             (f2cl-lib:int-add (f2cl-lib:int-sub nq i) 1) ib
             (f2cl-lib:array-slice a-%data% double-float (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice tau-%data% double-float (i) ((1 *))
                                   tau-%offset%)
             (f2cl-lib:array-slice work-%data% double-float (iwt) ((1 *))
                                   work-%offset%)
             ldt)
          (declare (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7))
          (setf lda var-5)
          (setf ldt var-8))
        (cond
         (left (setf mi (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
          (setf ic i))
         (t (setf ni (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
          (setf jc i)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
             var-10 var-11 var-12 var-13 var-14)
            (dlarfb side trans "Forward" "Columnwise" mi ni ib
             (f2cl-lib:array-slice a-%data% double-float (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice work-%data% double-float (iwt) ((1 *))
                                   work-%offset%)
             ldt
             (f2cl-lib:array-slice c-%data% double-float (ic jc)
                                   ((1 ldc) (1 *)) c-%offset%)
             ldc work ldwork)
          (declare (ignore var-2 var-3 var-7 var-9 var-11 var-13))
          (setf side var-0)
          (setf trans var-1)
          (setf mi var-4)
          (setf ni var-5)
          (setf ib var-6)
          (setf lda var-8)
          (setf ldt var-10)
          (setf ldc var-12)
          (setf ldwork var-14))
      label10))))
 (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce (the f2cl-lib:integer4 lwkopt) 'double-float))
 (go end_label)
end_label
 (return (values side trans m n k nil lda nil nil ldc nil nil info))))))
```

———-

90.205 dtrevc

90.205.1 Axiom unit tests

— **dtrevc.input** —

```
)set break resume
)sys rm -f dtrevc.output
)spool dtrevc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.205.2 Axiom help page

— **dtrevc.help** —

```
======================================================================
dtrevc examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       DTREVC  -  some  or all of the right and/or left eigenvectors of a real
       upper quasi-triangular matrix T

SYNOPSIS
       SUBROUTINE DTREVC( SIDE, HOWMNY, SELECT, N, T, LDT, VL, LDVL, VR, LDVR,
                          MM, M, WORK, INFO )

           CHARACTER      HOWMNY, SIDE
```

```
      INTEGER        INFO, LDT, LDVL, LDVR, M, MM, N

      LOGICAL        SELECT( * )

      DOUBLE         PRECISION T( LDT, * ), VL( LDVL, * ), VR( LDVR, * ),
                     WORK( * )
```

Purpose
=======

DTREVC computes some or all of the right and/or left eigenvectors of a real upper quasi-triangular matrix T.

The right eigenvector x and the left eigenvector y of T corresponding to an eigenvalue w are defined by:

```
             T*x = w*x,     y'*T = w*y'
```

where y' denotes the conjugate transpose of the vector y.

If all eigenvectors are requested, the routine may either return the matrices X and/or Y of right or left eigenvectors of T, or the products Q*X and/or Q*Y, where Q is an input orthogonal matrix. If T was obtained from the real-Schur factorization of an original matrix A = Q*T*Q', then Q*X and Q*Y are the matrices of right or left eigenvectors of A.

T must be in Schur canonical form (as returned by DHSEQR), that is, block upper triangular with 1-by-1 and 2-by-2 diagonal blocks; each 2-by-2 diagonal block has its diagonal elements equal and its off-diagonal elements of opposite sign. Corresponding to each 2-by-2 diagonal block is a complex conjugate pair of eigenvalues and eigenvectors; only one eigenvector of the pair is computed, namely the one corresponding to the eigenvalue with positive imaginary part.

Arguments
=========

```
SIDE    (input) CHARACTER*1
        = 'R':  compute right eigenvectors only;
        = 'L':  compute left eigenvectors only;
        = 'B':  compute both right and left eigenvectors.

HOWMNY  (input) CHARACTER*1
        = 'A':  compute all right and/or left eigenvectors;
        = 'B':  compute all right and/or left eigenvectors,
                and backtransform them using the input matrices
                supplied in VR and/or VL;
        = 'S':  compute selected right and/or left eigenvectors,
                specified by the logical array SELECT.
```

```
SELECT  (input/output) LOGICAL array, dimension (N)
        If HOWMNY = 'S', SELECT specifies the eigenvectors to be
        computed.
        If HOWMNY = 'A' or 'B', SELECT is not referenced.
        To select the real eigenvector corresponding to a real
        eigenvalue w(j), SELECT(j) must be set to .TRUE..  To select
        the complex eigenvector corresponding to a complex conjugate
        pair w(j) and w(j+1), either SELECT(j) or SELECT(j+1) must be
        set to .TRUE.; then on exit SELECT(j) is .TRUE. and
        SELECT(j+1) is .FALSE..

N       (input) INTEGER
        The order of the matrix T. N >= 0.

T       (input) DOUBLE PRECISION array, dimension (LDT,N)
        The upper quasi-triangular matrix T in Schur canonical form.

LDT     (input) INTEGER
        The leading dimension of the array T. LDT >= max(1,N).

VL      (input/output) DOUBLE PRECISION array, dimension (LDVL,MM)
        On entry, if SIDE = 'L' or 'B' and HOWMNY = 'B', VL must
        contain an N-by-N matrix Q (usually the orthogonal matrix Q
        of Schur vectors returned by DHSEQR).
        On exit, if SIDE = 'L' or 'B', VL contains:
        if HOWMNY = 'A', the matrix Y of left eigenvectors of T;
                         VL has the same quasi-lower triangular form
                         as T'. If T(i,i) is a real eigenvalue, then
                         the i-th column VL(i) of VL  is its
                         corresponding eigenvector. If T(i:i+1,i:i+1)
                         is a 2-by-2 block whose eigenvalues are
                         complex-conjugate eigenvalues of T, then
                         VL(i)+sqrt(-1)*VL(i+1) is the complex
                         eigenvector corresponding to the eigenvalue
                         with positive real part.
        if HOWMNY = 'B', the matrix Q*Y;
        if HOWMNY = 'S', the left eigenvectors of T specified by
                         SELECT, stored consecutively in the columns
                         of VL, in the same order as their
                         eigenvalues.
        A complex eigenvector corresponding to a complex eigenvalue
        is stored in two consecutive columns, the first holding the
        real part, and the second the imaginary part.
        If SIDE = 'R', VL is not referenced.

LDVL    (input) INTEGER
        The leading dimension of the array VL.  LDVL >= max(1,N) if
        SIDE = 'L' or 'B'; LDVL >= 1 otherwise.
```

```
VR      (input/output) DOUBLE PRECISION array, dimension (LDVR,MM)
        On entry, if SIDE = 'R' or 'B' and HOWMNY = 'B', VR must
        contain an N-by-N matrix Q (usually the orthogonal matrix Q
        of Schur vectors returned by DHSEQR).
        On exit, if SIDE = 'R' or 'B', VR contains:
        if HOWMNY = 'A', the matrix X of right eigenvectors of T;
                         VR has the same quasi-upper triangular form
                         as T. If T(i,i) is a real eigenvalue, then
                         the i-th column VR(i) of VR  is its
                         corresponding eigenvector. If T(i:i+1,i:i+1)
                         is a 2-by-2 block whose eigenvalues are
                         complex-conjugate eigenvalues of T, then
                         VR(i)+sqrt(-1)*VR(i+1) is the complex
                         eigenvector corresponding to the eigenvalue
                         with positive real part.
        if HOWMNY = 'B', the matrix Q*X;
        if HOWMNY = 'S', the right eigenvectors of T specified by
                         SELECT, stored consecutively in the columns
                         of VR, in the same order as their
                         eigenvalues.
        A complex eigenvector corresponding to a complex eigenvalue
        is stored in two consecutive columns, the first holding the
        real part and the second the imaginary part.
        If SIDE = 'L', VR is not referenced.

LDVR    (input) INTEGER
        The leading dimension of the array VR.  LDVR >= max(1,N) if
        SIDE = 'R' or 'B'; LDVR >= 1 otherwise.

MM      (input) INTEGER
        The number of columns in the arrays VL and/or VR. MM >= M.

M       (output) INTEGER
        The number of columns in the arrays VL and/or VR actually
        used to store the eigenvectors.
        If HOWMNY = 'A' or 'B', M is set to N.
        Each selected real eigenvector occupies one column and each
        selected complex eigenvector occupies two columns.

WORK    (workspace) DOUBLE PRECISION array, dimension (3*N)

INFO    (output) INTEGER
        = 0:  successful exit
        < 0:  if INFO = -i, the i-th argument had an illegal value

Further Details
===============

The algorithm used in this program is basically backward (forward)
substitution, with scaling to make the the code robust against
```

```
possible overflow.

Each eigenvector is normalized so that the element of largest
magnitude has magnitude 1; here the magnitude of a complex number
(x,y) is taken to be |x| + |y|.
```

———-

90.205.3 fortran code

— **dtrevc.f** —

```
      SUBROUTINE DTREVC( SIDE, HOWMNY, SELECT, N, T, LDT, VL, LDVL, VR,
     $                   LDVR, MM, M, WORK, INFO )
*
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      CHARACTER          HOWMNY, SIDE
      INTEGER            INFO, LDT, LDVL, LDVR, M, MM, N
*     ..
*     .. Array Arguments ..
      LOGICAL            SELECT( * )
      DOUBLE PRECISION   T( LDT, * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            ALLV, BOTHV, LEFTV, OVER, PAIR, RIGHTV, SOMEV
      INTEGER            I, IERR, II, IP, IS, J, J1, J2, JNXT, K, KI, N2
      DOUBLE PRECISION   BETA, BIGNUM, EMAX, OVFL, REC, REMAX, SCALE,
     $                   SMIN, SMLNUM, ULP, UNFL, VCRIT, VMAX, WI, WR,
     $                   XNORM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX
      DOUBLE PRECISION   DDOT, DLAMCH
```

```
      EXTERNAL           LSAME, IDAMAX, DDOT, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DAXPY, DCOPY, DGEMV, DLALN2, DSCAL, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   X( 2, 2 )
*     ..
*     .. Executable Statements ..
*
*     Decode and test the input parameters
*
      BOTHV = LSAME( SIDE, 'B' )
      RIGHTV = LSAME( SIDE, 'R' ) .OR. BOTHV
      LEFTV = LSAME( SIDE, 'L' ) .OR. BOTHV
*
      ALLV = LSAME( HOWMNY, 'A' )
      OVER = LSAME( HOWMNY, 'B' )
      SOMEV = LSAME( HOWMNY, 'S' )
*
      INFO = 0
      IF( .NOT.RIGHTV .AND. .NOT.LEFTV ) THEN
         INFO = -1
      ELSE IF( .NOT.ALLV .AND. .NOT.OVER .AND. .NOT.SOMEV ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDT.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDVL.LT.1 .OR. ( LEFTV .AND. LDVL.LT.N ) ) THEN
         INFO = -8
      ELSE IF( LDVR.LT.1 .OR. ( RIGHTV .AND. LDVR.LT.N ) ) THEN
         INFO = -10
      ELSE
*
*        Set M to the number of columns required to store the selected
*        eigenvectors, standardize the array SELECT if necessary, and
*        test MM.
*
         IF( SOMEV ) THEN
            M = 0
            PAIR = .FALSE.
            DO 10 J = 1, N
               IF( PAIR ) THEN
                  PAIR = .FALSE.
                  SELECT( J ) = .FALSE.
               ELSE
```

```
                 IF( J.LT.N ) THEN
                    IF( T( J+1, J ).EQ.ZERO ) THEN
                       IF( SELECT( J ) )
     $                    M = M + 1
                    ELSE
                       PAIR = .TRUE.
                       IF( SELECT( J ) .OR. SELECT( J+1 ) ) THEN
                          SELECT( J ) = .TRUE.
                          M = M + 2
                       END IF
                    END IF
                 ELSE
                    IF( SELECT( N ) )
     $                 M = M + 1
                 END IF
              END IF
   10       CONTINUE
         ELSE
            M = N
         END IF
*
         IF( MM.LT.M ) THEN
            INFO = -11
         END IF
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DTREVC', -INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Set the constants to control overflow.
*
      UNFL = DLAMCH( 'Safe minimum' )
      OVFL = ONE / UNFL
      CALL DLABAD( UNFL, OVFL )
      ULP = DLAMCH( 'Precision' )
      SMLNUM = UNFL*( N / ULP )
      BIGNUM = ( ONE-ULP ) / SMLNUM
*
*     Compute 1-norm of each column of strictly upper triangular
*     part of T to control overflow in triangular solver.
*
      WORK( 1 ) = ZERO
      DO 30 J = 2, N
         WORK( J ) = ZERO
```

```
         DO 20 I = 1, J - 1
            WORK( J ) = WORK( J ) + ABS( T( I, J ) )
   20    CONTINUE
   30 CONTINUE
*
*     Index IP is used to specify the real or complex eigenvalue:
*       IP = 0, real eigenvalue,
*            1, first of conjugate complex pair: (wr,wi)
*           -1, second of conjugate complex pair: (wr,wi)
*
      N2 = 2*N
*
      IF( RIGHTV ) THEN
*
*        Compute right eigenvectors.
*
         IP = 0
         IS = M
         DO 140 KI = N, 1, -1
*
            IF( IP.EQ.1 )
     $         GO TO 130
            IF( KI.EQ.1 )
     $         GO TO 40
            IF( T( KI, KI-1 ).EQ.ZERO )
     $         GO TO 40
            IP = -1
*
   40       CONTINUE
            IF( SOMEV ) THEN
               IF( IP.EQ.0 ) THEN
                  IF( .NOT.SELECT( KI ) )
     $               GO TO 130
               ELSE
                  IF( .NOT.SELECT( KI-1 ) )
     $               GO TO 130
               END IF
            END IF
*
*           Compute the KI-th eigenvalue (WR,WI).
*
            WR = T( KI, KI )
            WI = ZERO
            IF( IP.NE.0 )
     $         WI = SQRT( ABS( T( KI, KI-1 ) ) )*
     $              SQRT( ABS( T( KI-1, KI ) ) )
            SMIN = MAX( ULP*( ABS( WR )+ABS( WI ) ), SMLNUM )
*
            IF( IP.EQ.0 ) THEN
*
```

```
*              Real right eigenvector
*
               WORK( KI+N ) = ONE
*
*              Form right-hand side
*
               DO 50 K = 1, KI - 1
                  WORK( K+N ) = -T( K, KI )
   50          CONTINUE
*
*              Solve the upper quasi-triangular system:
*                 (T(1:KI-1,1:KI-1) - WR)*X = SCALE*WORK.
*
               JNXT = KI - 1
               DO 60 J = KI - 1, 1, -1
                  IF( J.GT.JNXT )
     $               GO TO 60
                  J1 = J
                  J2 = J
                  JNXT = J - 1
                  IF( J.GT.1 ) THEN
                     IF( T( J, J-1 ).NE.ZERO ) THEN
                        J1 = J - 1
                        JNXT = J - 2
                     END IF
                  END IF
*
                  IF( J1.EQ.J2 ) THEN
*
*                    1-by-1 diagonal block
*
                     CALL DLALN2( .FALSE., 1, 1, SMIN, ONE, T( J, J ),
     $                            LDT, ONE, ONE, WORK( J+N ), N, WR,
     $                            ZERO, X, 2, SCALE, XNORM, IERR )
*
*                    Scale X(1,1) to avoid overflow when updating
*                    the right-hand side.
*
                     IF( XNORM.GT.ONE ) THEN
                        IF( WORK( J ).GT.BIGNUM / XNORM ) THEN
                           X( 1, 1 ) = X( 1, 1 ) / XNORM
                           SCALE = SCALE / XNORM
                        END IF
                     END IF
*
*                    Scale if necessary
*
                     IF( SCALE.NE.ONE )
     $                  CALL DSCAL( KI, SCALE, WORK( 1+N ), 1 )
                     WORK( J+N ) = X( 1, 1 )
```

```
*
*                    Update right-hand side
*
                     CALL DAXPY( J-1, -X( 1, 1 ), T( 1, J ), 1,
     $                           WORK( 1+N ), 1 )
*
                  ELSE
*
*                    2-by-2 diagonal block
*
                     CALL DLALN2( .FALSE., 2, 1, SMIN, ONE,
     $                            T( J-1, J-1 ), LDT, ONE, ONE,
     $                            WORK( J-1+N ), N, WR, ZERO, X, 2,
     $                            SCALE, XNORM, IERR )
*
*                    Scale X(1,1) and X(2,1) to avoid overflow when
*                    updating the right-hand side.
*
                     IF( XNORM.GT.ONE ) THEN
                        BETA = MAX( WORK( J-1 ), WORK( J ) )
                        IF( BETA.GT.BIGNUM / XNORM ) THEN
                           X( 1, 1 ) = X( 1, 1 ) / XNORM
                           X( 2, 1 ) = X( 2, 1 ) / XNORM
                           SCALE = SCALE / XNORM
                        END IF
                     END IF
*
*                    Scale if necessary
*
                     IF( SCALE.NE.ONE )
     $                  CALL DSCAL( KI, SCALE, WORK( 1+N ), 1 )
                     WORK( J-1+N ) = X( 1, 1 )
                     WORK( J+N ) = X( 2, 1 )
*
*                    Update right-hand side
*
                     CALL DAXPY( J-2, -X( 1, 1 ), T( 1, J-1 ), 1,
     $                           WORK( 1+N ), 1 )
                     CALL DAXPY( J-2, -X( 2, 1 ), T( 1, J ), 1,
     $                           WORK( 1+N ), 1 )
                  END IF
   60          CONTINUE
*
*              Copy the vector x or Q*x to VR and normalize.
*
               IF( .NOT.OVER ) THEN
                  CALL DCOPY( KI, WORK( 1+N ), 1, VR( 1, IS ), 1 )
*
                  II = IDAMAX( KI, VR( 1, IS ), 1 )
                  REMAX = ONE / ABS( VR( II, IS ) )
```

```
                 CALL DSCAL( KI, REMAX, VR( 1, IS ), 1 )
*
                 DO 70 K = KI + 1, N
                    VR( K, IS ) = ZERO
   70            CONTINUE
              ELSE
                 IF( KI.GT.1 )
     $              CALL DGEMV( 'N', N, KI-1, ONE, VR, LDVR,
     $                          WORK( 1+N ), 1, WORK( KI+N ),
     $                          VR( 1, KI ), 1 )
*
                 II = IDAMAX( N, VR( 1, KI ), 1 )
                 REMAX = ONE / ABS( VR( II, KI ) )
                 CALL DSCAL( N, REMAX, VR( 1, KI ), 1 )
              END IF
*
           ELSE
*
*             Complex right eigenvector.
*
*             Initial solve
*               [ (T(KI-1,KI-1) T(KI-1,KI) ) - (WR + I* WI)]*X = 0.
*               [ (T(KI,KI-1)   T(KI,KI)   )               ]
*
              IF( ABS( T( KI-1, KI ) ).GE.ABS( T( KI, KI-1 ) ) ) THEN
                 WORK( KI-1+N ) = ONE
                 WORK( KI+N2 ) = WI / T( KI-1, KI )
              ELSE
                 WORK( KI-1+N ) = -WI / T( KI, KI-1 )
                 WORK( KI+N2 ) = ONE
              END IF
              WORK( KI+N ) = ZERO
              WORK( KI-1+N2 ) = ZERO
*
*             Form right-hand side
*
              DO 80 K = 1, KI - 2
                 WORK( K+N ) = -WORK( KI-1+N )*T( K, KI-1 )
                 WORK( K+N2 ) = -WORK( KI+N2 )*T( K, KI )
   80         CONTINUE
*
*             Solve upper quasi-triangular system:
*             (T(1:KI-2,1:KI-2) - (WR+i*WI))*X = SCALE*(WORK+i*WORK2)
*
              JNXT = KI - 2
              DO 90 J = KI - 2, 1, -1
                 IF( J.GT.JNXT )
     $              GO TO 90
                 J1 = J
                 J2 = J
```

```
                 JNXT = J - 1
                 IF( J.GT.1 ) THEN
                    IF( T( J, J-1 ).NE.ZERO ) THEN
                       J1 = J - 1
                       JNXT = J - 2
                    END IF
                 END IF
*
                 IF( J1.EQ.J2 ) THEN
*
*                   1-by-1 diagonal block
*
                    CALL DLALN2( .FALSE., 1, 2, SMIN, ONE, T( J, J ),
     $                           LDT, ONE, ONE, WORK( J+N ), N, WR, WI,
     $                           X, 2, SCALE, XNORM, IERR )
*
*                   Scale X(1,1) and X(1,2) to avoid overflow when
*                   updating the right-hand side.
*
                    IF( XNORM.GT.ONE ) THEN
                       IF( WORK( J ).GT.BIGNUM / XNORM ) THEN
                          X( 1, 1 ) = X( 1, 1 ) / XNORM
                          X( 1, 2 ) = X( 1, 2 ) / XNORM
                          SCALE = SCALE / XNORM
                       END IF
                    END IF
*
*                   Scale if necessary
*
                    IF( SCALE.NE.ONE ) THEN
                       CALL DSCAL( KI, SCALE, WORK( 1+N ), 1 )
                       CALL DSCAL( KI, SCALE, WORK( 1+N2 ), 1 )
                    END IF
                    WORK( J+N ) = X( 1, 1 )
                    WORK( J+N2 ) = X( 1, 2 )
*
*                   Update the right-hand side
*
                    CALL DAXPY( J-1, -X( 1, 1 ), T( 1, J ), 1,
     $                          WORK( 1+N ), 1 )
                    CALL DAXPY( J-1, -X( 1, 2 ), T( 1, J ), 1,
     $                          WORK( 1+N2 ), 1 )
*
                 ELSE
*
*                   2-by-2 diagonal block
*
                    CALL DLALN2( .FALSE., 2, 2, SMIN, ONE,
     $                           T( J-1, J-1 ), LDT, ONE, ONE,
     $                           WORK( J-1+N ), N, WR, WI, X, 2, SCALE,
```

```
     $                          XNORM, IERR )
*
*                    Scale X to avoid overflow when updating
*                    the right-hand side.
*
                     IF( XNORM.GT.ONE ) THEN
                        BETA = MAX( WORK( J-1 ), WORK( J ) )
                        IF( BETA.GT.BIGNUM / XNORM ) THEN
                           REC = ONE / XNORM
                           X( 1, 1 ) = X( 1, 1 )*REC
                           X( 1, 2 ) = X( 1, 2 )*REC
                           X( 2, 1 ) = X( 2, 1 )*REC
                           X( 2, 2 ) = X( 2, 2 )*REC
                           SCALE = SCALE*REC
                        END IF
                     END IF
*
*                    Scale if necessary
*
                     IF( SCALE.NE.ONE ) THEN
                        CALL DSCAL( KI, SCALE, WORK( 1+N ), 1 )
                        CALL DSCAL( KI, SCALE, WORK( 1+N2 ), 1 )
                     END IF
                     WORK( J-1+N ) = X( 1, 1 )
                     WORK( J+N ) = X( 2, 1 )
                     WORK( J-1+N2 ) = X( 1, 2 )
                     WORK( J+N2 ) = X( 2, 2 )
*
*                    Update the right-hand side
*
                     CALL DAXPY( J-2, -X( 1, 1 ), T( 1, J-1 ), 1,
     $                           WORK( 1+N ), 1 )
                     CALL DAXPY( J-2, -X( 2, 1 ), T( 1, J ), 1,
     $                           WORK( 1+N ), 1 )
                     CALL DAXPY( J-2, -X( 1, 2 ), T( 1, J-1 ), 1,
     $                           WORK( 1+N2 ), 1 )
                     CALL DAXPY( J-2, -X( 2, 2 ), T( 1, J ), 1,
     $                           WORK( 1+N2 ), 1 )
                  END IF
   90          CONTINUE
*
*              Copy the vector x or Q*x to VR and normalize.
*
               IF( .NOT.OVER ) THEN
                  CALL DCOPY( KI, WORK( 1+N ), 1, VR( 1, IS-1 ), 1 )
                  CALL DCOPY( KI, WORK( 1+N2 ), 1, VR( 1, IS ), 1 )
*
                  EMAX = ZERO
                  DO 100 K = 1, KI
                     EMAX = MAX( EMAX, ABS( VR( K, IS-1 ) )+
```

```
     $                     ABS( VR( K, IS ) ) )
  100            CONTINUE
*
                 REMAX = ONE / EMAX
                 CALL DSCAL( KI, REMAX, VR( 1, IS-1 ), 1 )
                 CALL DSCAL( KI, REMAX, VR( 1, IS ), 1 )
*
                 DO 110 K = KI + 1, N
                    VR( K, IS-1 ) = ZERO
                    VR( K, IS ) = ZERO
  110            CONTINUE
*
              ELSE
*
                 IF( KI.GT.2 ) THEN
                    CALL DGEMV( 'N', N, KI-2, ONE, VR, LDVR,
     $                          WORK( 1+N ), 1, WORK( KI-1+N ),
     $                          VR( 1, KI-1 ), 1 )
                    CALL DGEMV( 'N', N, KI-2, ONE, VR, LDVR,
     $                          WORK( 1+N2 ), 1, WORK( KI+N2 ),
     $                          VR( 1, KI ), 1 )
                 ELSE
                    CALL DSCAL( N, WORK( KI-1+N ), VR( 1, KI-1 ), 1 )
                    CALL DSCAL( N, WORK( KI+N2 ), VR( 1, KI ), 1 )
                 END IF
*
                 EMAX = ZERO
                 DO 120 K = 1, N
                    EMAX = MAX( EMAX, ABS( VR( K, KI-1 ) )+
     $                     ABS( VR( K, KI ) ) )
  120            CONTINUE
                 REMAX = ONE / EMAX
                 CALL DSCAL( N, REMAX, VR( 1, KI-1 ), 1 )
                 CALL DSCAL( N, REMAX, VR( 1, KI ), 1 )
              END IF
           END IF
*
           IS = IS - 1
           IF( IP.NE.0 )
     $        IS = IS - 1
  130      CONTINUE
           IF( IP.EQ.1 )
     $        IP = 0
           IF( IP.EQ.-1 )
     $        IP = 1
  140    CONTINUE
      END IF
*
      IF( LEFTV ) THEN
*
```

```
*        Compute left eigenvectors.
*
         IP = 0
         IS = 1
         DO 260 KI = 1, N
*
            IF( IP.EQ.-1 )
     $         GO TO 250
            IF( KI.EQ.N )
     $         GO TO 150
            IF( T( KI+1, KI ).EQ.ZERO )
     $         GO TO 150
            IP = 1
*
  150       CONTINUE
            IF( SOMEV ) THEN
               IF( .NOT.SELECT( KI ) )
     $            GO TO 250
            END IF
*
*           Compute the KI-th eigenvalue (WR,WI).
*
            WR = T( KI, KI )
            WI = ZERO
            IF( IP.NE.0 )
     $         WI = SQRT( ABS( T( KI, KI+1 ) ) )*
     $              SQRT( ABS( T( KI+1, KI ) ) )
            SMIN = MAX( ULP*( ABS( WR )+ABS( WI ) ), SMLNUM )
*
            IF( IP.EQ.0 ) THEN
*
*              Real left eigenvector.
*
               WORK( KI+N ) = ONE
*
*              Form right-hand side
*
               DO 160 K = KI + 1, N
                  WORK( K+N ) = -T( KI, K )
  160          CONTINUE
*
*              Solve the quasi-triangular system:
*                 (T(KI+1:N,KI+1:N) - WR)'*X = SCALE*WORK
*
               VMAX = ONE
               VCRIT = BIGNUM
*
               JNXT = KI + 1
               DO 170 J = KI + 1, N
                  IF( J.LT.JNXT )
```

```
     $               GO TO 170
                  J1 = J
                  J2 = J
                  JNXT = J + 1
                  IF( J.LT.N ) THEN
                     IF( T( J+1, J ).NE.ZERO ) THEN
                        J2 = J + 1
                        JNXT = J + 2
                     END IF
                  END IF
*
                  IF( J1.EQ.J2 ) THEN
*
*                    1-by-1 diagonal block
*
*                    Scale if necessary to avoid overflow when forming
*                    the right-hand side.
*
                     IF( WORK( J ).GT.VCRIT ) THEN
                        REC = ONE / VMAX
                        CALL DSCAL( N-KI+1, REC, WORK( KI+N ), 1 )
                        VMAX = ONE
                        VCRIT = BIGNUM
                     END IF
*
                     WORK( J+N ) = WORK( J+N ) -
     $                             DDOT( J-KI-1, T( KI+1, J ), 1,
     $                             WORK( KI+1+N ), 1 )
*
*                    Solve (T(J,J)-WR)'*X = WORK
*
                     CALL DLALN2( .FALSE., 1, 1, SMIN, ONE, T( J, J ),
     $                            LDT, ONE, ONE, WORK( J+N ), N, WR,
     $                            ZERO, X, 2, SCALE, XNORM, IERR )
*
*                    Scale if necessary
*
                     IF( SCALE.NE.ONE )
     $                  CALL DSCAL( N-KI+1, SCALE, WORK( KI+N ), 1 )
                     WORK( J+N ) = X( 1, 1 )
                     VMAX = MAX( ABS( WORK( J+N ) ), VMAX )
                     VCRIT = BIGNUM / VMAX
*
                  ELSE
*
*                    2-by-2 diagonal block
*
*                    Scale if necessary to avoid overflow when forming
*                    the right-hand side.
*
```

```
                  BETA = MAX( WORK( J ), WORK( J+1 ) )
                  IF( BETA.GT.VCRIT ) THEN
                     REC = ONE / VMAX
                     CALL DSCAL( N-KI+1, REC, WORK( KI+N ), 1 )
                     VMAX = ONE
                     VCRIT = BIGNUM
                  END IF
*
                  WORK( J+N ) = WORK( J+N ) -
     $                          DDOT( J-KI-1, T( KI+1, J ), 1,
     $                          WORK( KI+1+N ), 1 )
*
                  WORK( J+1+N ) = WORK( J+1+N ) -
     $                            DDOT( J-KI-1, T( KI+1, J+1 ), 1,
     $                            WORK( KI+1+N ), 1 )
*
*                 Solve
*                   [T(J,J)-WR   T(J,J+1)     ]'* X = SCALE*( WORK1 )
*                   [T(J+1,J)    T(J+1,J+1)-WR]             ( WORK2 )
*
                  CALL DLALN2( .TRUE., 2, 1, SMIN, ONE, T( J, J ),
     $                         LDT, ONE, ONE, WORK( J+N ), N, WR,
     $                         ZERO, X, 2, SCALE, XNORM, IERR )
*
*                 Scale if necessary
*
                  IF( SCALE.NE.ONE )
     $               CALL DSCAL( N-KI+1, SCALE, WORK( KI+N ), 1 )
                  WORK( J+N ) = X( 1, 1 )
                  WORK( J+1+N ) = X( 2, 1 )
*
                  VMAX = MAX( ABS( WORK( J+N ) ),
     $                    ABS( WORK( J+1+N ) ), VMAX )
                  VCRIT = BIGNUM / VMAX
*
               END IF
  170       CONTINUE
*
*           Copy the vector x or Q*x to VL and normalize.
*
            IF( .NOT.OVER ) THEN
               CALL DCOPY( N-KI+1, WORK( KI+N ), 1, VL( KI, IS ), 1 )
*
               II = IDAMAX( N-KI+1, VL( KI, IS ), 1 ) + KI - 1
               REMAX = ONE / ABS( VL( II, IS ) )
               CALL DSCAL( N-KI+1, REMAX, VL( KI, IS ), 1 )
*
               DO 180 K = 1, KI - 1
                  VL( K, IS ) = ZERO
  180          CONTINUE
```

```
*
               ELSE
*
                  IF( KI.LT.N )
     $               CALL DGEMV( 'N', N, N-KI, ONE, VL( 1, KI+1 ), LDVL,
     $                           WORK( KI+1+N ), 1, WORK( KI+N ),
     $                           VL( 1, KI ), 1 )
*
                  II = IDAMAX( N, VL( 1, KI ), 1 )
                  REMAX = ONE / ABS( VL( II, KI ) )
                  CALL DSCAL( N, REMAX, VL( 1, KI ), 1 )
*
               END IF
*
            ELSE
*
*              Complex left eigenvector.
*
*               Initial solve:
*                ((T(KI,KI)    T(KI,KI+1) )' - (WR - I* WI))*X = 0.
*                ((T(KI+1,KI) T(KI+1,KI+1))                )
*
               IF( ABS( T( KI, KI+1 ) ).GE.ABS( T( KI+1, KI ) ) ) THEN
                  WORK( KI+N ) = WI / T( KI, KI+1 )
                  WORK( KI+1+N2 ) = ONE
               ELSE
                  WORK( KI+N ) = ONE
                  WORK( KI+1+N2 ) = -WI / T( KI+1, KI )
               END IF
               WORK( KI+1+N ) = ZERO
               WORK( KI+N2 ) = ZERO
*
*              Form right-hand side
*
               DO 190 K = KI + 2, N
                  WORK( K+N ) = -WORK( KI+N )*T( KI, K )
                  WORK( K+N2 ) = -WORK( KI+1+N2 )*T( KI+1, K )
  190          CONTINUE
*
*              Solve complex quasi-triangular system:
*              ( T(KI+2,N:KI+2,N) - (WR-i*WI) )*X = WORK1+i*WORK2
*
               VMAX = ONE
               VCRIT = BIGNUM
*
               JNXT = KI + 2
               DO 200 J = KI + 2, N
                  IF( J.LT.JNXT )
     $               GO TO 200
                  J1 = J
```

```
                 J2 = J
                 JNXT = J + 1
                 IF( J.LT.N ) THEN
                    IF( T( J+1, J ).NE.ZERO ) THEN
                       J2 = J + 1
                       JNXT = J + 2
                    END IF
                 END IF
*
                 IF( J1.EQ.J2 ) THEN
*
*                   1-by-1 diagonal block
*
*                   Scale if necessary to avoid overflow when
*                   forming the right-hand side elements.
*
                    IF( WORK( J ).GT.VCRIT ) THEN
                       REC = ONE / VMAX
                       CALL DSCAL( N-KI+1, REC, WORK( KI+N ), 1 )
                       CALL DSCAL( N-KI+1, REC, WORK( KI+N2 ), 1 )
                       VMAX = ONE
                       VCRIT = BIGNUM
                    END IF
*
                    WORK( J+N ) = WORK( J+N ) -
     $                            DDOT( J-KI-2, T( KI+2, J ), 1,
     $                            WORK( KI+2+N ), 1 )
                    WORK( J+N2 ) = WORK( J+N2 ) -
     $                             DDOT( J-KI-2, T( KI+2, J ), 1,
     $                             WORK( KI+2+N2 ), 1 )
*
*                   Solve (T(J,J)-(WR-i*WI))*(X11+i*X12)= WK+I*WK2
*
                    CALL DLALN2( .FALSE., 1, 2, SMIN, ONE, T( J, J ),
     $                           LDT, ONE, ONE, WORK( J+N ), N, WR,
     $                           -WI, X, 2, SCALE, XNORM, IERR )
*
*                   Scale if necessary
*
                    IF( SCALE.NE.ONE ) THEN
                       CALL DSCAL( N-KI+1, SCALE, WORK( KI+N ), 1 )
                       CALL DSCAL( N-KI+1, SCALE, WORK( KI+N2 ), 1 )
                    END IF
                    WORK( J+N ) = X( 1, 1 )
                    WORK( J+N2 ) = X( 1, 2 )
                    VMAX = MAX( ABS( WORK( J+N ) ),
     $                      ABS( WORK( J+N2 ) ), VMAX )
                    VCRIT = BIGNUM / VMAX
*
                 ELSE
```

```
*
*                    2-by-2 diagonal block
*
*                    Scale if necessary to avoid overflow when forming
*                    the right-hand side elements.
*
                     BETA = MAX( WORK( J ), WORK( J+1 ) )
                     IF( BETA.GT.VCRIT ) THEN
                        REC = ONE / VMAX
                        CALL DSCAL( N-KI+1, REC, WORK( KI+N ), 1 )
                        CALL DSCAL( N-KI+1, REC, WORK( KI+N2 ), 1 )
                        VMAX = ONE
                        VCRIT = BIGNUM
                     END IF
*
                     WORK( J+N ) = WORK( J+N ) -
     $                              DDOT( J-KI-2, T( KI+2, J ), 1,
     $                              WORK( KI+2+N ), 1 )
*
                     WORK( J+N2 ) = WORK( J+N2 ) -
     $                               DDOT( J-KI-2, T( KI+2, J ), 1,
     $                               WORK( KI+2+N2 ), 1 )
*
                     WORK( J+1+N ) = WORK( J+1+N ) -
     $                                DDOT( J-KI-2, T( KI+2, J+1 ), 1,
     $                                WORK( KI+2+N ), 1 )
*
                     WORK( J+1+N2 ) = WORK( J+1+N2 ) -
     $                                 DDOT( J-KI-2, T( KI+2, J+1 ), 1,
     $                                 WORK( KI+2+N2 ), 1 )
*
*                    Solve 2-by-2 complex linear equation
*                      ([T(j,j)   T(j,j+1)  ]'-(wr-i*wi)*I)*X = SCALE*B
*                      ([T(j+1,j) T(j+1,j+1)]             )
*
                     CALL DLALN2( .TRUE., 2, 2, SMIN, ONE, T( J, J ),
     $                            LDT, ONE, ONE, WORK( J+N ), N, WR,
     $                            -WI, X, 2, SCALE, XNORM, IERR )
*
*                    Scale if necessary
*
                     IF( SCALE.NE.ONE ) THEN
                        CALL DSCAL( N-KI+1, SCALE, WORK( KI+N ), 1 )
                        CALL DSCAL( N-KI+1, SCALE, WORK( KI+N2 ), 1 )
                     END IF
                     WORK( J+N ) = X( 1, 1 )
                     WORK( J+N2 ) = X( 1, 2 )
                     WORK( J+1+N ) = X( 2, 1 )
                     WORK( J+1+N2 ) = X( 2, 2 )
                     VMAX = MAX( ABS( X( 1, 1 ) ), ABS( X( 1, 2 ) ),
```

```
     $                    ABS( X( 2, 1 ) ), ABS( X( 2, 2 ) ), VMAX )
                     VCRITT = BIGNUM / VMAX
*
                  END IF
  200          CONTINUE
*
*              Copy the vector x or Q*x to VL and normalize.
*
  210          CONTINUE
               IF( .NOT.OVER ) THEN
                  CALL DCOPY( N-KI+1, WORK( KI+N ), 1, VL( KI, IS ), 1 )
                  CALL DCOPY( N-KI+1, WORK( KI+N2 ), 1, VL( KI, IS+1 ),
     $                        1 )
*
                  EMAX = ZERO
                  DO 220 K = KI, N
                     EMAX = MAX( EMAX, ABS( VL( K, IS ) )+
     $                       ABS( VL( K, IS+1 ) ) )
  220             CONTINUE
                  REMAX = ONE / EMAX
                  CALL DSCAL( N-KI+1, REMAX, VL( KI, IS ), 1 )
                  CALL DSCAL( N-KI+1, REMAX, VL( KI, IS+1 ), 1 )
*
                  DO 230 K = 1, KI - 1
                     VL( K, IS ) = ZERO
                     VL( K, IS+1 ) = ZERO
  230             CONTINUE
               ELSE
                  IF( KI.LT.N-1 ) THEN
                     CALL DGEMV( 'N', N, N-KI-1, ONE, VL( 1, KI+2 ),
     $                           LDVL, WORK( KI+2+N ), 1, WORK( KI+N ),
     $                           VL( 1, KI ), 1 )
                     CALL DGEMV( 'N', N, N-KI-1, ONE, VL( 1, KI+2 ),
     $                           LDVL, WORK( KI+2+N2 ), 1,
     $                           WORK( KI+1+N2 ), VL( 1, KI+1 ), 1 )
                  ELSE
                     CALL DSCAL( N, WORK( KI+N ), VL( 1, KI ), 1 )
                     CALL DSCAL( N, WORK( KI+1+N2 ), VL( 1, KI+1 ), 1 )
                  END IF
*
                  EMAX = ZERO
                  DO 240 K = 1, N
                     EMAX = MAX( EMAX, ABS( VL( K, KI ) )+
     $                       ABS( VL( K, KI+1 ) ) )
  240             CONTINUE
                  REMAX = ONE / EMAX
                  CALL DSCAL( N, REMAX, VL( 1, KI ), 1 )
                  CALL DSCAL( N, REMAX, VL( 1, KI+1 ), 1 )
*
               END IF
```

```
*
           END IF
*
           IS = IS + 1
           IF( IP.NE.0 )
     $        IS = IS + 1
  250      CONTINUE
           IF( IP.EQ.-1 )
     $        IP = 0
           IF( IP.EQ.1 )
     $        IP = -1
*
  260    CONTINUE
*
      END IF
*
      RETURN
*
*     End of DTREVC
*
      END
```

———-

90.205.4 lisp code

— dtrevc.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun dtrevc (side howmny select n t$ ldt vl ldvl vr ldvr mm m work info)
    (declare (type (array double-float (*)) work vr vl t$)
             (type (f2cl-lib:integer4) info m mm ldvr ldvl ldt n)
             (type (array f2cl-lib:logical (*)) select)
             (type (string *) howmny side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (howmny character howmny-%data% howmny-%offset%)
         (select f2cl-lib:logical select-%data% select-%offset%)
         (t$ double-float t$-%data% t$-%offset%)
         (vl double-float vl-%data% vl-%offset%)
         (vr double-float vr-%data% vr-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((x (make-array 4 :element-type 'double-float)) (beta 0.0d0)
             (bignum 0.0d0) (emax 0.0d0) (ovfl 0.0d0) (rec 0.0d0) (remax 0.0d0)
```

```
     (scale 0.0d0) (smin 0.0d0) (smlnum 0.0d0) (ulp 0.0d0) (unfl 0.0d0)
     (vcrit 0.0d0) (vmax 0.0d0) (wi 0.0d0) (wr 0.0d0) (xnorm 0.0d0)
     (i 0) (ierr 0) (ii 0) (ip 0) (is 0) (j 0) (j1 0) (j2 0) (jnxt 0)
     (k 0) (ki 0) (n2 0) (allv nil) (bothv nil) (leftv nil) (over nil)
     (pair nil) (rightv nil) (somev nil))
(declare (type (array double-float (4)) x)
         (type (double-float) beta bignum emax ovfl rec remax scale
          smin smlnum ulp unfl vcrit vmax wi wr xnorm)
         (type (f2cl-lib:integer4) i ierr ii ip is j j1 j2 jnxt k ki
          n2)
         (type f2cl-lib:logical allv bothv leftv over pair rightv
          somev))
(setf bothv
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "B")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val))
(setf rightv
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame side "R")
           (declare (ignore var-1))
           (when var-0 (setf side var-0))
           ret-val)
         bothv))
(setf leftv
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame side "L")
           (declare (ignore var-1))
           (when var-0 (setf side var-0))
           ret-val)
         bothv))
(setf allv
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame howmny "A")
          (declare (ignore var-1))
          (when var-0 (setf howmny var-0))
          ret-val))
(setf over
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame howmny "B")
          (declare (ignore var-1))
          (when var-0 (setf howmny var-0))
          ret-val))
(setf somev
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame howmny "S")
          (declare (ignore var-1))
```

```
          (when var-0 (setf howmny var-0))
          ret-val))
(setf info 0)
(cond ((and (not rightv) (not leftv)) (setf info -1))
      ((and (not allv) (not over) (not somev)) (setf info -2))
      ((< n 0) (setf info -4))
      ((< ldt
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -6))
      ((or (< ldvl 1) (and leftv (< ldvl n))) (setf info -8))
      ((or (< ldvr 1) (and rightv (< ldvr n))) (setf info -10))
      (t
       (cond
        (somev (setf m 0) (setf pair f2cl-lib:%false%)
         (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                       ((> j n) nil)
           (tagbody
             (cond
              (pair (setf pair f2cl-lib:%false%)
               (setf (f2cl-lib:fref select-%data% (j) ((1 *))
                                    select-%offset%)
                     f2cl-lib:%false%))
              (t
               (cond
                ((< j n)
                 (cond
                  ((=
                    (f2cl-lib:fref t$ ((f2cl-lib:int-add j 1) j)
                                   ((1 ldt) (1 *)))
                    zero)
                   (if (f2cl-lib:fref select-%data% (j) ((1 *))
                                      select-%offset%)
                       (setf m (f2cl-lib:int-add m 1))))
                  (t (setf pair f2cl-lib:%true%)
                   (cond
                    ((or (f2cl-lib:fref select (j) ((1 *)))
                         (f2cl-lib:fref select ((f2cl-lib:int-add j 1))
                                        ((1 *))))
                     (setf (f2cl-lib:fref select-%data% (j) ((1 *))
                                          select-%offset%)
                           f2cl-lib:%true%)
                     (setf m (f2cl-lib:int-add m 2)))))))
                (t
                 (if (f2cl-lib:fref select-%data% (n) ((1 *))
                                    select-%offset%)
                     (setf m (f2cl-lib:int-add m 1)))))))
           label10)))
        (t (setf m n)))
       (cond ((< mm m) (setf info -11)))))
(cond
```

```
 ((/= info 0) (xerbla "DTREVC" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(setf unfl (dlamch "Safe minimum"))
(setf ovfl (/ one unfl))
(multiple-value-bind (var-0 var-1)
    (dlabad unfl ovfl)
  (declare (ignore))
  (when var-0 (setf unfl var-0))
  (when var-1 (setf ovfl var-1)))
(setf ulp (dlamch "Precision"))
(setf smlnum (* unfl (/ n ulp)))
(setf bignum (/ (- one ulp) smlnum))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) zero)
(f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%) zero)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                (+
                 (f2cl-lib:fref work-%data% (j) ((1 *)) work-%offset%)
                 (abs
                  (f2cl-lib:fref t$-%data% (i j) ((1 ldt) (1 *))
                                 t$-%offset%))))
       label20))
   label30))
(setf n2 (f2cl-lib:int-mul 2 n))
(cond
 (rightv (setf ip 0) (setf is m)
  (f2cl-lib:fdo (ki n (f2cl-lib:int-add ki (f2cl-lib:int-sub 1)))
                ((> ki 1) nil)
    (tagbody
      (if (= ip 1)
          (go label130))
      (if (= ki 1)
          (go label40))
      (if (=
           (f2cl-lib:fref t$-%data% (ki (f2cl-lib:int-sub ki 1))
                          ((1 ldt) (1 *)) t$-%offset%)
           zero)
          (go label40))
      (setf ip -1)
     label40
      (cond
       (somev
        (cond
```

```
   ((= ip 0)
    (if (not
         (f2cl-lib:fref select-%data% (ki) ((1 *))
                        select-%offset%))
        (go label130)))
   (t
    (if (not
         (f2cl-lib:fref select-%data% ((f2cl-lib:int-sub ki 1))
                        ((1 *)) select-%offset%))
        (go label130))))))
(setf wr
        (f2cl-lib:fref t$-%data% (ki ki) ((1 ldt) (1 *))
                       t$-%offset%))
(setf wi zero)
(if (/= ip 0)
    (setf wi
            (*
             (f2cl-lib:fsqrt
              (abs
               (f2cl-lib:fref t$-%data%
                              (ki (f2cl-lib:int-sub ki 1))
                              ((1 ldt) (1 *)) t$-%offset%)))
             (f2cl-lib:fsqrt
              (abs
               (f2cl-lib:fref t$-%data%
                              ((f2cl-lib:int-sub ki 1) ki)
                              ((1 ldt) (1 *)) t$-%offset%))))))
(setf smin (max (* ulp (+ (abs wr) (abs wi))) smlnum))
(cond
 ((= ip 0)
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ki n))
                       ((1 *)) work-%offset%)
          one)
  (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                ((> k (f2cl-lib:int-add ki (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k n))
                           ((1 *)) work-%offset%)
             (-
              (f2cl-lib:fref t$-%data% (k ki) ((1 ldt) (1 *))
                             t$-%offset%)))
     label50))
  (setf jnxt (f2cl-lib:int-sub ki 1))
  (f2cl-lib:fdo (j (f2cl-lib:int-add ki (f2cl-lib:int-sub 1))
                 (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                ((> j 1) nil)
    (tagbody
      (if (> j jnxt)
          (go label60))
```

```
(setf j1 j)
(setf j2 j)
(setf jnxt (f2cl-lib:int-sub j 1))
(cond
 ((> j 1)
  (cond
   ((/=
     (f2cl-lib:fref t$
                    (j
                     (f2cl-lib:int-add j
                                       (f2cl-lib:int-sub
                                        1)))
                    ((1 ldt) (1 *)))
     zero)
    (setf j1 (f2cl-lib:int-sub j 1))
    (setf jnxt (f2cl-lib:int-sub j 2))))))
(cond
 ((= j1 j2)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8 var-9 var-10 var-11 var-12 var-13 var-14
       var-15 var-16 var-17)
      (dlaln2 f2cl-lib:%false% 1 1 smin one
       (f2cl-lib:array-slice t$-%data% double-float (j j)
                             ((1 ldt) (1 *)) t$-%offset%)
       ldt one one
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ j n)) ((1 *))
                             work-%offset%)
       n wr zero x 2 scale xnorm ierr)
    (declare
     (ignore var-0 var-1 var-2 var-5 var-9 var-13 var-14))
    (when var-3 (setf smin var-3))
    (when var-4 (setf one var-4))
    (when var-6 (setf ldt var-6))
    (when var-7 (setf one var-7))
    (when var-8 (setf one var-8))
    (when var-10 (setf n var-10))
    (when var-11 (setf wr var-11))
    (when var-12 (setf zero var-12))
    (when var-15 (setf scale var-15))
    (when var-16 (setf xnorm var-16))
    (when var-17 (setf ierr var-17)))
  (cond
   ((> xnorm one)
    (cond
     ((> (f2cl-lib:fref work (j) ((1 *)))
         (f2cl-lib:f2cl/ bignum xnorm))
      (setf (f2cl-lib:fref x (1 1) ((1 2) (1 2)))
            (/ (f2cl-lib:fref x (1 1) ((1 2) (1 2)))
```

```
                xnorm))
        (setf scale (/ scale xnorm))))))
    (if (/= scale one)
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (dscal ki scale
             (f2cl-lib:array-slice work-%data% double-float
                                   ((+ 1 n)) ((1 *))
                                   work-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf ki var-0))
          (when var-1 (setf scale var-1))))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add j n))
                         ((1 *)) work-%offset%)
            (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
    (daxpy (f2cl-lib:int-sub j 1)
     (- (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
     (f2cl-lib:array-slice t$-%data% double-float (1 j)
                           ((1 ldt) (1 *)) t$-%offset%)
     1
     (f2cl-lib:array-slice work-%data% double-float ((+ 1 n))
                           ((1 *)) work-%offset%)
     1))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
         var-8 var-9 var-10 var-11 var-12 var-13 var-14
         var-15 var-16 var-17)
        (dlaln2 f2cl-lib:%false% 2 1 smin one
         (f2cl-lib:array-slice t$-%data% double-float
                               ((+ j (f2cl-lib:int-sub 1))
                                (f2cl-lib:int-sub j 1))
                               ((1 ldt) (1 *)) t$-%offset%)
         ldt one one
         (f2cl-lib:array-slice work-%data% double-float
                               ((+ j (f2cl-lib:int-sub 1) n))
                               ((1 *)) work-%offset%)
         n wr zero x 2 scale xnorm ierr)
      (declare
       (ignore var-0 var-1 var-2 var-5 var-9 var-13 var-14))
      (when var-3 (setf smin var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldt var-6))
      (when var-7 (setf one var-7))
      (when var-8 (setf one var-8))
      (when var-10 (setf n var-10))
      (when var-11 (setf wr var-11))
      (when var-12 (setf zero var-12))
      (when var-15 (setf scale var-15))
      (when var-16 (setf xnorm var-16))
```

```
  (when var-17 (setf ierr var-17)))
(cond
 ((> xnorm one)
  (setf beta
          (max
           (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-sub j 1))
                          ((1 *)) work-%offset%)
           (f2cl-lib:fref work-%data% (j) ((1 *))
                          work-%offset%)))
  (cond
   ((> beta (f2cl-lib:f2cl/ bignum xnorm))
    (setf (f2cl-lib:fref x (1 1) ((1 2) (1 2)))
            (/ (f2cl-lib:fref x (1 1) ((1 2) (1 2)))
               xnorm))
    (setf (f2cl-lib:fref x (2 1) ((1 2) (1 2)))
            (/ (f2cl-lib:fref x (2 1) ((1 2) (1 2)))
               xnorm))
    (setf scale (/ scale xnorm))))))
(if (/= scale one)
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal ki scale
         (f2cl-lib:array-slice work-%data% double-float
                               ((+ 1 n)) ((1 *))
                               work-%offset%)
         1)
      (declare (ignore var-2 var-3))
      (when var-0 (setf ki var-0))
      (when var-1 (setf scale var-1))))
(setf (f2cl-lib:fref work-%data%
                     ((f2cl-lib:int-add
                       (f2cl-lib:int-sub j 1) n))
                     ((1 *)) work-%offset%)
        (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
(setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add j n))
                     ((1 *)) work-%offset%)
        (f2cl-lib:fref x (2 1) ((1 2) (1 2))))
(daxpy (f2cl-lib:int-sub j 2)
 (- (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
 (f2cl-lib:array-slice t$-%data% double-float
                       (1 (f2cl-lib:int-sub j 1))
                       ((1 ldt) (1 *)) t$-%offset%)
 1
 (f2cl-lib:array-slice work-%data% double-float ((+ 1 n))
                       ((1 *)) work-%offset%)
 1)
(daxpy (f2cl-lib:int-sub j 2)
 (- (f2cl-lib:fref x (2 1) ((1 2) (1 2))))
 (f2cl-lib:array-slice t$-%data% double-float (1 j)
                       ((1 ldt) (1 *)) t$-%offset%)
```

```
        1
        (f2cl-lib:array-slice work-%data% double-float ((+ 1 n))
                              ((1 *)) work-%offset%)
        1)))
    label60))
  (cond
   ((not over)
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (dcopy ki
         (f2cl-lib:array-slice work-%data% double-float ((+ 1 n))
                               ((1 *)) work-%offset%)
         1
         (f2cl-lib:array-slice vr-%data% double-float (1 is)
                               ((1 ldvr) (1 *)) vr-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf ki var-0)))
    (setf ii
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (idamax ki
                 (f2cl-lib:array-slice vr-%data% double-float
                                       (1 is) ((1 ldvr) (1 *))
                                       vr-%offset%)
                 1)
              (declare (ignore var-1 var-2))
              (when var-0 (setf ki var-0))
              ret-val))
    (setf remax
            (/ one
               (abs
                (f2cl-lib:fref vr-%data% (ii is) ((1 ldvr) (1 *))
                               vr-%offset%))))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (dscal ki remax
         (f2cl-lib:array-slice vr-%data% double-float (1 is)
                               ((1 ldvr) (1 *)) vr-%offset%)
         1)
      (declare (ignore var-2 var-3))
      (when var-0 (setf ki var-0))
      (when var-1 (setf remax var-1)))
    (f2cl-lib:fdo (k (f2cl-lib:int-add ki 1)
                   (f2cl-lib:int-add k 1))
                  ((> k n) nil)
      (tagbody
        (setf (f2cl-lib:fref vr-%data% (k is) ((1 ldvr) (1 *))
                             vr-%offset%)
              zero)
       label70)))
   (t
    (if (> ki 1)
```

```
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
                     var-8 var-9 var-10)
                    (dgemv "N" n (f2cl-lib:int-sub ki 1) one vr ldvr
                     (f2cl-lib:array-slice work-%data% double-float
                                           ((+ 1 n)) ((1 *))
                                           work-%offset%)
                     1
                     (f2cl-lib:array-slice work-%data% double-float
                                           ((+ ki n)) ((1 *))
                                           work-%offset%)
                     (f2cl-lib:array-slice vr-%data% double-float (1 ki)
                                           ((1 ldvr) (1 *)) vr-%offset%)
                     1)
                  (declare
                   (ignore var-0 var-2 var-4 var-6 var-7 var-8 var-9
                    var-10))
                  (when var-1 (setf n var-1))
                  (when var-3 (setf one var-3))
                  (when var-5 (setf ldvr var-5))))
            (setf ii
                    (multiple-value-bind (ret-val var-0 var-1 var-2)
                        (idamax n
                         (f2cl-lib:array-slice vr-%data% double-float
                                               (1 ki) ((1 ldvr) (1 *))
                                               vr-%offset%)
                         1)
                      (declare (ignore var-1 var-2))
                      (when var-0 (setf n var-0))
                      ret-val))
            (setf remax
                    (/ one
                       (abs
                        (f2cl-lib:fref vr-%data% (ii ki) ((1 ldvr) (1 *))
                                       vr-%offset%))))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (dscal n remax
                 (f2cl-lib:array-slice vr-%data% double-float (1 ki)
                                       ((1 ldvr) (1 *)) vr-%offset%)
                 1)
              (declare (ignore var-2 var-3))
              (when var-0 (setf n var-0))
              (when var-1 (setf remax var-1))))))
        (t
         (cond
          ((>=
            (abs
             (f2cl-lib:fref t$
                            ((f2cl-lib:int-add ki (f2cl-lib:int-sub 1))
                             ki)
```

```
                    ((1 ldt) (1 *))))
    (abs
     (f2cl-lib:fref t$
                    (ki
                     (f2cl-lib:int-add ki (f2cl-lib:int-sub 1)))
                    ((1 ldt) (1 *)))))
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-sub ki 1) n))
                       ((1 *)) work-%offset%)
          one)
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ki n2))
                       ((1 *)) work-%offset%)
          (/ wi
             (f2cl-lib:fref t$-%data%
                            ((f2cl-lib:int-sub ki 1) ki)
                            ((1 ldt) (1 *)) t$-%offset%))))
 (t
  (setf (f2cl-lib:fref work-%data%
                       ((f2cl-lib:int-add
                         (f2cl-lib:int-sub ki 1) n))
                       ((1 *)) work-%offset%)
          (/ (- wi)
             (f2cl-lib:fref t$-%data%
                            (ki (f2cl-lib:int-sub ki 1))
                            ((1 ldt) (1 *)) t$-%offset%)))
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ki n2))
                       ((1 *)) work-%offset%)
          one)))
(setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ki n))
                     ((1 *)) work-%offset%)
        zero)
(setf (f2cl-lib:fref work-%data%
                     ((f2cl-lib:int-add (f2cl-lib:int-sub ki 1)
                                        n2))
                     ((1 *)) work-%offset%)
        zero)
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k (f2cl-lib:int-add ki (f2cl-lib:int-sub 2)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k n))
                         ((1 *)) work-%offset%)
            (*
             (-
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-sub ki 1) n))
                             ((1 *)) work-%offset%))
             (f2cl-lib:fref t$-%data%
```

```
                                (k (f2cl-lib:int-sub ki 1))
                                ((1 ldt) (1 *)) t$-%offset%)))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k n2))
                         ((1 *)) work-%offset%)
            (*
             (-
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add ki n2)) ((1 *))
                             work-%offset%))
             (f2cl-lib:fref t$-%data% (k ki) ((1 ldt) (1 *))
                            t$-%offset%)))
   label80))
(setf jnxt (f2cl-lib:int-sub ki 2))
(f2cl-lib:fdo (j (f2cl-lib:int-add ki (f2cl-lib:int-sub 2))
               (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j 1) nil)
  (tagbody
    (if (> j jnxt)
        (go label90))
    (setf j1 j)
    (setf j2 j)
    (setf jnxt (f2cl-lib:int-sub j 1))
    (cond
     ((> j 1)
      (cond
       ((/=
         (f2cl-lib:fref t$
                        (j
                         (f2cl-lib:int-add j
                                           (f2cl-lib:int-sub
                                            1)))
                        ((1 ldt) (1 *)))
         zero)
        (setf j1 (f2cl-lib:int-sub j 1))
        (setf jnxt (f2cl-lib:int-sub j 2))))))
    (cond
     ((= j1 j2)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
           var-8 var-9 var-10 var-11 var-12 var-13 var-14
           var-15 var-16 var-17)
          (dlaln2 f2cl-lib:%false% 1 2 smin one
           (f2cl-lib:array-slice t$-%data% double-float (j j)
                                 ((1 ldt) (1 *)) t$-%offset%)
           ldt one one
           (f2cl-lib:array-slice work-%data% double-float
                                 ((+ j n)) ((1 *))
                                 work-%offset%)
           n wr wi x 2 scale xnorm ierr)
        (declare
```

```
   (ignore var-0 var-1 var-2 var-5 var-9 var-13 var-14))
  (when var-3 (setf smin var-3))
  (when var-4 (setf one var-4))
  (when var-6 (setf ldt var-6))
  (when var-7 (setf one var-7))
  (when var-8 (setf one var-8))
  (when var-10 (setf n var-10))
  (when var-11 (setf wr var-11))
  (when var-12 (setf wi var-12))
  (when var-15 (setf scale var-15))
  (when var-16 (setf xnorm var-16))
  (when var-17 (setf ierr var-17)))
(cond
 ((> xnorm one)
  (cond
   ((> (f2cl-lib:fref work (j) ((1 *)))
       (f2cl-lib:f2cl/ bignum xnorm))
    (setf (f2cl-lib:fref x (1 1) ((1 2) (1 2)))
            (/ (f2cl-lib:fref x (1 1) ((1 2) (1 2)))
               xnorm))
    (setf (f2cl-lib:fref x (1 2) ((1 2) (1 2)))
            (/ (f2cl-lib:fref x (1 2) ((1 2) (1 2)))
               xnorm))
    (setf scale (/ scale xnorm))))))
(cond
 ((/= scale one)
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dscal ki scale
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ 1 n)) ((1 *))
                             work-%offset%)
       1)
    (declare (ignore var-2 var-3))
    (when var-0 (setf ki var-0))
    (when var-1 (setf scale var-1)))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dscal ki scale
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ 1 n2)) ((1 *))
                             work-%offset%)
       1)
    (declare (ignore var-2 var-3))
    (when var-0 (setf ki var-0))
    (when var-1 (setf scale var-1)))))
(setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add j n))
                     ((1 *)) work-%offset%)
        (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
(setf (f2cl-lib:fref work-%data%
                     ((f2cl-lib:int-add j n2)) ((1 *))
                     work-%offset%)
```

```
        (f2cl-lib:fref x (1 2) ((1 2) (1 2))))
 (daxpy (f2cl-lib:int-sub j 1)
  (- (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
  (f2cl-lib:array-slice t$-%data% double-float (1 j)
                        ((1 ldt) (1 *)) t$-%offset%)
  1
  (f2cl-lib:array-slice work-%data% double-float ((+ 1 n))
                        ((1 *)) work-%offset%)
  1)
 (daxpy (f2cl-lib:int-sub j 1)
  (- (f2cl-lib:fref x (1 2) ((1 2) (1 2))))
  (f2cl-lib:array-slice t$-%data% double-float (1 j)
                        ((1 ldt) (1 *)) t$-%offset%)
  1
  (f2cl-lib:array-slice work-%data% double-float
                        ((+ 1 n2)) ((1 *)) work-%offset%)
  1))
(t
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
      var-8 var-9 var-10 var-11 var-12 var-13 var-14
      var-15 var-16 var-17)
     (dlaln2 f2cl-lib:%false% 2 2 smin one
      (f2cl-lib:array-slice t$-%data% double-float
                            ((+ j (f2cl-lib:int-sub 1))
                             (f2cl-lib:int-sub j 1))
                            ((1 ldt) (1 *)) t$-%offset%)
      ldt one one
      (f2cl-lib:array-slice work-%data% double-float
                            ((+ j (f2cl-lib:int-sub 1) n))
                            ((1 *)) work-%offset%)
      n wr wi x 2 scale xnorm ierr)
   (declare
    (ignore var-0 var-1 var-2 var-5 var-9 var-13 var-14))
   (when var-3 (setf smin var-3))
   (when var-4 (setf one var-4))
   (when var-6 (setf ldt var-6))
   (when var-7 (setf one var-7))
   (when var-8 (setf one var-8))
   (when var-10 (setf n var-10))
   (when var-11 (setf wr var-11))
   (when var-12 (setf wi var-12))
   (when var-15 (setf scale var-15))
   (when var-16 (setf xnorm var-16))
   (when var-17 (setf ierr var-17)))
 (cond
  ((> xnorm one)
   (setf beta
           (max
            (f2cl-lib:fref work-%data%
```

```
                              ((f2cl-lib:int-sub j 1))
                              ((1 *)) work-%offset%)
                (f2cl-lib:fref work-%data% (j) ((1 *))
                              work-%offset%)))
  (cond
   ((> beta (f2cl-lib:f2cl/ bignum xnorm))
    (setf rec (/ one xnorm))
    (setf (f2cl-lib:fref x (1 1) ((1 2) (1 2)))
            (* (f2cl-lib:fref x (1 1) ((1 2) (1 2)))
               rec))
    (setf (f2cl-lib:fref x (1 2) ((1 2) (1 2)))
            (* (f2cl-lib:fref x (1 2) ((1 2) (1 2)))
               rec))
    (setf (f2cl-lib:fref x (2 1) ((1 2) (1 2)))
            (* (f2cl-lib:fref x (2 1) ((1 2) (1 2)))
               rec))
    (setf (f2cl-lib:fref x (2 2) ((1 2) (1 2)))
            (* (f2cl-lib:fref x (2 2) ((1 2) (1 2)))
               rec))
    (setf scale (* scale rec))))))
(cond
 ((/= scale one)
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dscal ki scale
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ 1 n)) ((1 *))
                             work-%offset%)
       1)
    (declare (ignore var-2 var-3))
    (when var-0 (setf ki var-0))
    (when var-1 (setf scale var-1)))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dscal ki scale
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ 1 n2)) ((1 *))
                             work-%offset%)
       1)
    (declare (ignore var-2 var-3))
    (when var-0 (setf ki var-0))
    (when var-1 (setf scale var-1)))))
(setf (f2cl-lib:fref work-%data%
                     ((f2cl-lib:int-add
                       (f2cl-lib:int-sub j 1) n))
                     ((1 *)) work-%offset%)
        (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
(setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add j n))
                     ((1 *)) work-%offset%)
        (f2cl-lib:fref x (2 1) ((1 2) (1 2))))
(setf (f2cl-lib:fref work-%data%
                     ((f2cl-lib:int-add
```

```
                                (f2cl-lib:int-sub j 1) n2))
                              ((1 *)) work-%offset%)
                      (f2cl-lib:fref x (1 2) ((1 2) (1 2))))
                (setf (f2cl-lib:fref work-%data%
                              ((f2cl-lib:int-add j n2)) ((1 *))
                              work-%offset%)
                      (f2cl-lib:fref x (2 2) ((1 2) (1 2))))
                (daxpy (f2cl-lib:int-sub j 2)
                 (- (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
                 (f2cl-lib:array-slice t$-%data% double-float
                                (1 (f2cl-lib:int-sub j 1))
                                ((1 ldt) (1 *)) t$-%offset%)
                 1
                 (f2cl-lib:array-slice work-%data% double-float ((+ 1 n))
                                ((1 *)) work-%offset%)
                 1)
                (daxpy (f2cl-lib:int-sub j 2)
                 (- (f2cl-lib:fref x (2 1) ((1 2) (1 2))))
                 (f2cl-lib:array-slice t$-%data% double-float (1 j)
                                ((1 ldt) (1 *)) t$-%offset%)
                 1
                 (f2cl-lib:array-slice work-%data% double-float ((+ 1 n))
                                ((1 *)) work-%offset%)
                 1)
                (daxpy (f2cl-lib:int-sub j 2)
                 (- (f2cl-lib:fref x (1 2) ((1 2) (1 2))))
                 (f2cl-lib:array-slice t$-%data% double-float
                                (1 (f2cl-lib:int-sub j 1))
                                ((1 ldt) (1 *)) t$-%offset%)
                 1
                 (f2cl-lib:array-slice work-%data% double-float
                                ((+ 1 n2)) ((1 *)) work-%offset%)
                 1)
                (daxpy (f2cl-lib:int-sub j 2)
                 (- (f2cl-lib:fref x (2 2) ((1 2) (1 2))))
                 (f2cl-lib:array-slice t$-%data% double-float (1 j)
                                ((1 ldt) (1 *)) t$-%offset%)
                 1
                 (f2cl-lib:array-slice work-%data% double-float
                                ((+ 1 n2)) ((1 *)) work-%offset%)
                 1)))
           label90))
        (cond
         ((not over)
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (dcopy ki
               (f2cl-lib:array-slice work-%data% double-float ((+ 1 n))
                                ((1 *)) work-%offset%)
               1
               (f2cl-lib:array-slice vr-%data% double-float
```

```
                            (1 (f2cl-lib:int-sub is 1))
                            ((1 ldvr) (1 *)) vr-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf ki var-0)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (dcopy ki
     (f2cl-lib:array-slice work-%data% double-float
                            ((+ 1 n2)) ((1 *)) work-%offset%)
     1
     (f2cl-lib:array-slice vr-%data% double-float (1 is)
                            ((1 ldvr) (1 *)) vr-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf ki var-0)))
(setf emax zero)
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k ki) nil)
  (tagbody
    (setf emax
            (max emax
                 (+
                  (abs
                   (f2cl-lib:fref vr-%data%
                                  (k (f2cl-lib:int-sub is 1))
                                  ((1 ldvr) (1 *))
                                  vr-%offset%))
                  (abs
                   (f2cl-lib:fref vr-%data% (k is)
                                  ((1 ldvr) (1 *))
                                  vr-%offset%)))))
   label100))
(setf remax (/ one emax))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (dscal ki remax
     (f2cl-lib:array-slice vr-%data% double-float
                            (1 (f2cl-lib:int-sub is 1))
                            ((1 ldvr) (1 *)) vr-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf ki var-0))
  (when var-1 (setf remax var-1)))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (dscal ki remax
     (f2cl-lib:array-slice vr-%data% double-float (1 is)
                            ((1 ldvr) (1 *)) vr-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf ki var-0))
  (when var-1 (setf remax var-1)))
```

```
 (f2cl-lib:fdo (k (f2cl-lib:int-add ki 1)
                (f2cl-lib:int-add k 1))
               ((> k n) nil)
   (tagbody
     (setf (f2cl-lib:fref vr-%data%
                          (k (f2cl-lib:int-sub is 1))
                          ((1 ldvr) (1 *)) vr-%offset%)
           zero)
     (setf (f2cl-lib:fref vr-%data% (k is) ((1 ldvr) (1 *))
                          vr-%offset%)
           zero)
    label110)))
(t
 (cond
  ((> ki 2)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (dgemv "N" n (f2cl-lib:int-sub ki 2) one vr ldvr
        (f2cl-lib:array-slice work-%data% double-float
                              ((+ 1 n)) ((1 *)) work-%offset%)
        1
        (f2cl-lib:array-slice work-%data% double-float
                              ((+ ki (f2cl-lib:int-sub 1) n))
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice vr-%data% double-float
                              (1 (f2cl-lib:int-sub ki 1))
                              ((1 ldvr) (1 *)) vr-%offset%)
        1)
     (declare
      (ignore var-0 var-2 var-4 var-6 var-7 var-8 var-9
       var-10))
     (when var-1 (setf n var-1))
     (when var-3 (setf one var-3))
     (when var-5 (setf ldvr var-5)))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (dgemv "N" n (f2cl-lib:int-sub ki 2) one vr ldvr
        (f2cl-lib:array-slice work-%data% double-float
                              ((+ 1 n2)) ((1 *))
                              work-%offset%)
        1
        (f2cl-lib:array-slice work-%data% double-float
                              ((+ ki n2)) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice vr-%data% double-float (1 ki)
                              ((1 ldvr) (1 *)) vr-%offset%)
        1)
     (declare
```

```
      (ignore var-0 var-2 var-4 var-6 var-7 var-8 var-9
       var-10))
     (when var-1 (setf n var-1))
     (when var-3 (setf one var-3))
     (when var-5 (setf ldvr var-5))))
  (t
   (multiple-value-bind (var-0 var-1 var-2 var-3)
       (dscal n
        (f2cl-lib:array-slice work-%data% double-float
                              ((+ ki (f2cl-lib:int-sub 1) n))
                              ((1 *)) work-%offset%)
        (f2cl-lib:array-slice vr-%data% double-float
                              (1 (f2cl-lib:int-sub ki 1))
                              ((1 ldvr) (1 *)) vr-%offset%)
        1)
     (declare (ignore var-1 var-2 var-3))
     (when var-0 (setf n var-0)))
   (multiple-value-bind (var-0 var-1 var-2 var-3)
       (dscal n
        (f2cl-lib:array-slice work-%data% double-float
                              ((+ ki n2)) ((1 *))
                              work-%offset%)
        (f2cl-lib:array-slice vr-%data% double-float (1 ki)
                              ((1 ldvr) (1 *)) vr-%offset%)
        1)
     (declare (ignore var-1 var-2 var-3))
     (when var-0 (setf n var-0)))))
(setf emax zero)
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (setf emax
            (max emax
                 (+
                  (abs
                   (f2cl-lib:fref vr-%data%
                                  (k (f2cl-lib:int-sub ki 1))
                                  ((1 ldvr) (1 *))
                                  vr-%offset%))
                  (abs
                   (f2cl-lib:fref vr-%data% (k ki)
                                  ((1 ldvr) (1 *))
                                  vr-%offset%)))))
   label120))
(setf remax (/ one emax))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (dscal n remax
     (f2cl-lib:array-slice vr-%data% double-float
                           (1 (f2cl-lib:int-sub ki 1))
                           ((1 ldvr) (1 *)) vr-%offset%)
```

```
                  1)
                (declare (ignore var-2 var-3))
                (when var-0 (setf n var-0))
                (when var-1 (setf remax var-1)))
              (multiple-value-bind (var-0 var-1 var-2 var-3)
                  (dscal n remax
                   (f2cl-lib:array-slice vr-%data% double-float (1 ki)
                                         ((1 ldvr) (1 *)) vr-%offset%)
                   1)
                (declare (ignore var-2 var-3))
                (when var-0 (setf n var-0))
                (when var-1 (setf remax var-1)))))))
        (setf is (f2cl-lib:int-sub is 1))
        (if (/= ip 0)
            (setf is (f2cl-lib:int-sub is 1)))
       label130
        (if (= ip 1)
            (setf ip 0))
        (if (= ip -1)
            (setf ip 1))
       label140))))
  (cond
   (leftv (setf ip 0) (setf is 1)
    (f2cl-lib:fdo (ki 1 (f2cl-lib:int-add ki 1))
                  ((> ki n) nil)
      (tagbody
        (if (= ip -1)
            (go label250))
        (if (= ki n)
            (go label150))
        (if (=
             (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add ki 1) ki)
                            ((1 ldt) (1 *)) t$-%offset%)
             zero)
            (go label150))
        (setf ip 1)
       label150
        (cond
         (somev
          (if (not
               (f2cl-lib:fref select-%data% (ki) ((1 *))
                              select-%offset%))
              (go label250))))
        (setf wr
                (f2cl-lib:fref t$-%data% (ki ki) ((1 ldt) (1 *))
                               t$-%offset%))
        (setf wi zero)
        (if (/= ip 0)
            (setf wi
                    (*
```

```
                (f2cl-lib:fsqrt
                 (abs
                  (f2cl-lib:fref t$-%data%
                                 (ki (f2cl-lib:int-add ki 1))
                                 ((1 ldt) (1 *)) t$-%offset%)))
                (f2cl-lib:fsqrt
                 (abs
                  (f2cl-lib:fref t$-%data%
                                 ((f2cl-lib:int-add ki 1) ki)
                                 ((1 ldt) (1 *)) t$-%offset%))))))
(setf smin (max (* ulp (+ (abs wr) (abs wi))) smlnum))
(cond
 ((= ip 0)
  (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ki n))
                       ((1 *)) work-%offset%)
        one)
  (f2cl-lib:fdo (k (f2cl-lib:int-add ki 1) (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k n))
                           ((1 *)) work-%offset%)
            (-
             (f2cl-lib:fref t$-%data% (ki k) ((1 ldt) (1 *))
                            t$-%offset%)))
     label160))
  (setf vmax one) (setf vcrit bignum)
  (setf jnxt (f2cl-lib:int-add ki 1))
  (f2cl-lib:fdo (j (f2cl-lib:int-add ki 1) (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (if (< j jnxt)
          (go label170))
      (setf j1 j)
      (setf j2 j)
      (setf jnxt (f2cl-lib:int-add j 1))
      (cond
       ((< j n)
        (cond
         ((/=
           (f2cl-lib:fref t$ ((f2cl-lib:int-add j 1) j)
                          ((1 ldt) (1 *)))
           zero)
          (setf j2 (f2cl-lib:int-add j 1))
          (setf jnxt (f2cl-lib:int-add j 2))))))
      (cond
       ((= j1 j2)
        (cond
         ((> (f2cl-lib:fref work (j) ((1 *))) vcrit)
          (setf rec (/ one vmax))
          (multiple-value-bind (var-0 var-1 var-2 var-3)
```

```
      (dscal (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1)
       rec
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ ki n)) ((1 *))
                             work-%offset%)
       1)
    (declare (ignore var-0 var-2 var-3))
    (when var-1 (setf rec var-1)))
  (setf vmax one) (setf vcrit bignum)))
(setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add j n))
                     ((1 *)) work-%offset%)
        (-
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add j n)) ((1 *))
                        work-%offset%)
         (ddot (f2cl-lib:int-sub j ki 1)
          (f2cl-lib:array-slice t$-%data% double-float
                                ((+ ki 1) j)
                                ((1 ldt) (1 *))
                                t$-%offset%)
          1
          (f2cl-lib:array-slice work-%data% double-float
                                ((+ ki 1 n)) ((1 *))
                                work-%offset%)
          1)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
     var-8 var-9 var-10 var-11 var-12 var-13 var-14
     var-15 var-16 var-17)
    (dlaln2 f2cl-lib:%false% 1 1 smin one
     (f2cl-lib:array-slice t$-%data% double-float (j j)
                           ((1 ldt) (1 *)) t$-%offset%)
     ldt one one
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ j n)) ((1 *))
                           work-%offset%)
     n wr zero x 2 scale xnorm ierr)
  (declare
   (ignore var-0 var-1 var-2 var-5 var-9 var-13 var-14))
  (when var-3 (setf smin var-3))
  (when var-4 (setf one var-4))
  (when var-6 (setf ldt var-6))
  (when var-7 (setf one var-7))
  (when var-8 (setf one var-8))
  (when var-10 (setf n var-10))
  (when var-11 (setf wr var-11))
  (when var-12 (setf zero var-12))
  (when var-15 (setf scale var-15))
  (when var-16 (setf xnorm var-16))
  (when var-17 (setf ierr var-17)))
```

```
 (if (/= scale one)
     (multiple-value-bind (var-0 var-1 var-2 var-3)
         (dscal
          (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1)
          scale
          (f2cl-lib:array-slice work-%data% double-float
                                ((+ ki n)) ((1 *))
                                work-%offset%)
          1)
       (declare (ignore var-0 var-2 var-3))
       (when var-1 (setf scale var-1))))
 (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add j n))
                      ((1 *)) work-%offset%)
         (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
 (setf vmax
         (max
          (abs
           (f2cl-lib:fref work-%data%
                          ((f2cl-lib:int-add j n)) ((1 *))
                          work-%offset%))
          vmax))
 (setf vcrit (/ bignum vmax)))
(t
 (setf beta
         (max
          (f2cl-lib:fref work-%data% (j) ((1 *))
                         work-%offset%)
          (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add j 1)) ((1 *))
                         work-%offset%)))
 (cond
  ((> beta vcrit) (setf rec (/ one vmax))
   (multiple-value-bind (var-0 var-1 var-2 var-3)
       (dscal (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1)
        rec
        (f2cl-lib:array-slice work-%data% double-float
                              ((+ ki n)) ((1 *))
                              work-%offset%)
        1)
     (declare (ignore var-0 var-2 var-3))
     (when var-1 (setf rec var-1)))
   (setf vmax one) (setf vcrit bignum)))
 (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add j n))
                      ((1 *)) work-%offset%)
         (-
          (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add j n)) ((1 *))
                         work-%offset%)
          (ddot (f2cl-lib:int-sub j ki 1)
           (f2cl-lib:array-slice t$-%data% double-float
```

```
                                ((+ ki 1) j)
                                ((1 ldt) (1 *))
                                t$-%offset%)
          1
          (f2cl-lib:array-slice work-%data% double-float
                                ((+ ki 1 n)) ((1 *))
                                work-%offset%)
          1)))
(setf (f2cl-lib:fref work-%data%
                     ((f2cl-lib:int-add j 1 n)) ((1 *))
                     work-%offset%)
        (-
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add j 1 n))
                        ((1 *)) work-%offset%)
         (ddot (f2cl-lib:int-sub j ki 1)
          (f2cl-lib:array-slice t$-%data% double-float
                                ((+ ki 1)
                                 (f2cl-lib:int-add j 1))
                                ((1 ldt) (1 *))
                                t$-%offset%)
          1
          (f2cl-lib:array-slice work-%data% double-float
                                ((+ ki 1 n)) ((1 *))
                                work-%offset%)
          1)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
     var-8 var-9 var-10 var-11 var-12 var-13 var-14
     var-15 var-16 var-17)
    (dlaln2 f2cl-lib:%true% 2 1 smin one
     (f2cl-lib:array-slice t$-%data% double-float (j j)
                           ((1 ldt) (1 *)) t$-%offset%)
     ldt one one
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ j n)) ((1 *))
                           work-%offset%)
     n wr zero x 2 scale xnorm ierr)
  (declare
   (ignore var-0 var-1 var-2 var-5 var-9 var-13 var-14))
  (when var-3 (setf smin var-3))
  (when var-4 (setf one var-4))
  (when var-6 (setf ldt var-6))
  (when var-7 (setf one var-7))
  (when var-8 (setf one var-8))
  (when var-10 (setf n var-10))
  (when var-11 (setf wr var-11))
  (when var-12 (setf zero var-12))
  (when var-15 (setf scale var-15))
  (when var-16 (setf xnorm var-16))
```

```
          (when var-17 (setf ierr var-17)))
        (if (/= scale one)
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (dscal
                 (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1)
                 scale
                 (f2cl-lib:array-slice work-%data% double-float
                                       ((+ ki n)) ((1 *))
                                       work-%offset%)
                 1)
              (declare (ignore var-0 var-2 var-3))
              (when var-1 (setf scale var-1))))
        (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add j n))
                             ((1 *)) work-%offset%)
                (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
        (setf (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add j 1 n)) ((1 *))
                             work-%offset%)
                (f2cl-lib:fref x (2 1) ((1 2) (1 2))))
        (setf vmax
                (max
                 (abs
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add j n)) ((1 *))
                                 work-%offset%))
                 (abs
                  (f2cl-lib:fref work-%data%
                                 ((f2cl-lib:int-add j 1 n))
                                 ((1 *)) work-%offset%))
                 vmax))
        (setf vcrit (/ bignum vmax))))
    label170))
  (cond
   ((not over)
    (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1)
     (f2cl-lib:array-slice work-%data% double-float ((+ ki n))
                           ((1 *)) work-%offset%)
     1
     (f2cl-lib:array-slice vl-%data% double-float (ki is)
                           ((1 ldvl) (1 *)) vl-%offset%)
     1)
    (setf ii
            (f2cl-lib:int-sub
             (f2cl-lib:int-add
              (idamax
               (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1)
               (f2cl-lib:array-slice vl-%data% double-float
                                     (ki is) ((1 ldvl) (1 *))
                                     vl-%offset%)
               1)
```

```
          ki)
         1))
 (setf remax
        (/ one
           (abs
            (f2cl-lib:fref vl-%data% (ii is) ((1 ldvl) (1 *))
                           vl-%offset%))))
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (dscal (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1) remax
      (f2cl-lib:array-slice vl-%data% double-float (ki is)
                            ((1 ldvl) (1 *)) vl-%offset%)
      1)
   (declare (ignore var-0 var-2 var-3))
   (when var-1 (setf remax var-1)))
 (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
               ((> k
                   (f2cl-lib:int-add ki (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (setf (f2cl-lib:fref vl-%data% (k is) ((1 ldvl) (1 *))
                          vl-%offset%)
           zero)
    label180)))
(t
 (if (< ki n)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
          var-8 var-9 var-10)
         (dgemv "N" n (f2cl-lib:int-sub n ki) one
          (f2cl-lib:array-slice vl-%data% double-float
                                (1 (f2cl-lib:int-add ki 1))
                                ((1 ldvl) (1 *)) vl-%offset%)
          ldvl
          (f2cl-lib:array-slice work-%data% double-float
                                ((+ ki 1 n)) ((1 *))
                                work-%offset%)
          1
          (f2cl-lib:array-slice work-%data% double-float
                                ((+ ki n)) ((1 *))
                                work-%offset%)
          (f2cl-lib:array-slice vl-%data% double-float (1 ki)
                                ((1 ldvl) (1 *)) vl-%offset%)
          1)
       (declare
        (ignore var-0 var-2 var-4 var-6 var-7 var-8 var-9
         var-10))
       (when var-1 (setf n var-1))
       (when var-3 (setf one var-3))
       (when var-5 (setf ldvl var-5))))
 (setf ii
```

```
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (idamax n
                 (f2cl-lib:array-slice vl-%data% double-float
                                       (1 ki) ((1 ldvl) (1 *))
                                       vl-%offset%)
                 1)
              (declare (ignore var-1 var-2))
              (when var-0 (setf n var-0))
              ret-val))
   (setf remax
           (/ one
              (abs
               (f2cl-lib:fref vl-%data% (ii ki) ((1 ldvl) (1 *))
                              vl-%offset%))))
   (multiple-value-bind (var-0 var-1 var-2 var-3)
       (dscal n remax
        (f2cl-lib:array-slice vl-%data% double-float (1 ki)
                              ((1 ldvl) (1 *)) vl-%offset%)
        1)
     (declare (ignore var-2 var-3))
     (when var-0 (setf n var-0))
     (when var-1 (setf remax var-1))))))
(t
 (cond
  ((>=
    (abs
     (f2cl-lib:fref t$ (ki (f2cl-lib:int-add ki 1))
                    ((1 ldt) (1 *))))
    (abs
     (f2cl-lib:fref t$ ((f2cl-lib:int-add ki 1) ki)
                    ((1 ldt) (1 *)))))
   (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ki n))
                        ((1 *)) work-%offset%)
           (/ wi
              (f2cl-lib:fref t$-%data%
                             (ki (f2cl-lib:int-add ki 1))
                             ((1 ldt) (1 *)) t$-%offset%)))
   (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ki 1 n2))
                        ((1 *)) work-%offset%)
           one))
  (t
   (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ki n))
                        ((1 *)) work-%offset%)
           one)
   (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ki 1 n2))
                        ((1 *)) work-%offset%)
           (/ (- wi)
              (f2cl-lib:fref t$-%data%
                             ((f2cl-lib:int-add ki 1) ki)
                             ((1 ldt) (1 *)) t$-%offset%)))))
```

```
(setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ki 1 n))
                     ((1 *)) work-%offset%)
        zero)
(setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add ki n2))
                     ((1 *)) work-%offset%)
        zero)
(f2cl-lib:fdo (k (f2cl-lib:int-add ki 2) (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k n))
                         ((1 *)) work-%offset%)
            (*
             (-
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add ki n)) ((1 *))
                             work-%offset%))
             (f2cl-lib:fref t$-%data% (ki k) ((1 ldt) (1 *))
                            t$-%offset%)))
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k n2))
                         ((1 *)) work-%offset%)
            (*
             (-
              (f2cl-lib:fref work-%data%
                             ((f2cl-lib:int-add ki 1 n2))
                             ((1 *)) work-%offset%))
             (f2cl-lib:fref t$-%data%
                            ((f2cl-lib:int-add ki 1) k)
                            ((1 ldt) (1 *)) t$-%offset%)))
   label190))
(setf vmax one) (setf vcrit bignum)
(setf jnxt (f2cl-lib:int-add ki 2))
(f2cl-lib:fdo (j (f2cl-lib:int-add ki 2) (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (if (< j jnxt)
        (go label200))
    (setf j1 j)
    (setf j2 j)
    (setf jnxt (f2cl-lib:int-add j 1))
    (cond
     ((< j n)
      (cond
       ((/=
         (f2cl-lib:fref t$ ((f2cl-lib:int-add j 1) j)
                        ((1 ldt) (1 *)))
         zero)
        (setf j2 (f2cl-lib:int-add j 1))
        (setf jnxt (f2cl-lib:int-add j 2))))))
    (cond
     ((= j1 j2)
```

```
(cond
 ((> (f2cl-lib:fref work (j) ((1 *))) vcrit)
  (setf rec (/ one vmax))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dscal (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1)
       rec
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ ki n)) ((1 *))
                             work-%offset%)
       1)
    (declare (ignore var-0 var-2 var-3))
    (when var-1 (setf rec var-1)))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dscal (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1)
       rec
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ ki n2)) ((1 *))
                             work-%offset%)
       1)
    (declare (ignore var-0 var-2 var-3))
    (when var-1 (setf rec var-1)))
  (setf vmax one) (setf vcrit bignum)))
(setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add j n))
                     ((1 *)) work-%offset%)
        (-
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add j n)) ((1 *))
                        work-%offset%)
         (ddot (f2cl-lib:int-sub j ki 2)
          (f2cl-lib:array-slice t$-%data% double-float
                                ((+ ki 2) j)
                                ((1 ldt) (1 *))
                                t$-%offset%)
          1
          (f2cl-lib:array-slice work-%data% double-float
                                ((+ ki 2 n)) ((1 *))
                                work-%offset%)
          1)))
(setf (f2cl-lib:fref work-%data%
                     ((f2cl-lib:int-add j n2)) ((1 *))
                     work-%offset%)
        (-
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add j n2)) ((1 *))
                        work-%offset%)
         (ddot (f2cl-lib:int-sub j ki 2)
          (f2cl-lib:array-slice t$-%data% double-float
                                ((+ ki 2) j)
                                ((1 ldt) (1 *))
                                t$-%offset%)
```

```
          1
          (f2cl-lib:array-slice work-%data% double-float
                                ((+ ki 2 n2)) ((1 *))
                                work-%offset%)
          1)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
     var-8 var-9 var-10 var-11 var-12 var-13 var-14
     var-15 var-16 var-17)
    (dlaln2 f2cl-lib:%false% 1 2 smin one
     (f2cl-lib:array-slice t$-%data% double-float (j j)
                           ((1 ldt) (1 *)) t$-%offset%)
     ldt one one
     (f2cl-lib:array-slice work-%data% double-float
                           ((+ j n)) ((1 *))
                           work-%offset%)
     n wr (- wi) x 2 scale xnorm ierr)
  (declare
   (ignore var-0 var-1 var-2 var-5 var-9 var-12 var-13
    var-14))
  (when var-3 (setf smin var-3))
  (when var-4 (setf one var-4))
  (when var-6 (setf ldt var-6))
  (when var-7 (setf one var-7))
  (when var-8 (setf one var-8))
  (when var-10 (setf n var-10))
  (when var-11 (setf wr var-11))
  (when var-15 (setf scale var-15))
  (when var-16 (setf xnorm var-16))
  (when var-17 (setf ierr var-17)))
(cond
 ((/= scale one)
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dscal (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1)
       scale
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ ki n)) ((1 *))
                             work-%offset%)
       1)
    (declare (ignore var-0 var-2 var-3))
    (when var-1 (setf scale var-1)))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dscal (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1)
       scale
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ ki n2)) ((1 *))
                             work-%offset%)
       1)
    (declare (ignore var-0 var-2 var-3))
    (when var-1 (setf scale var-1)))))
```

```
 (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add j n))
                      ((1 *)) work-%offset%)
        (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
 (setf (f2cl-lib:fref work-%data%
                      ((f2cl-lib:int-add j n2)) ((1 *))
                      work-%offset%)
        (f2cl-lib:fref x (1 2) ((1 2) (1 2))))
 (setf vmax
        (max
         (abs
          (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add j n)) ((1 *))
                         work-%offset%))
         (abs
          (f2cl-lib:fref work-%data%
                         ((f2cl-lib:int-add j n2))
                         ((1 *)) work-%offset%))
         vmax))
 (setf vcrit (/ bignum vmax)))
(t
 (setf beta
        (max
         (f2cl-lib:fref work-%data% (j) ((1 *))
                        work-%offset%)
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add j 1)) ((1 *))
                        work-%offset%)))
 (cond
  ((> beta vcrit) (setf rec (/ one vmax))
   (multiple-value-bind (var-0 var-1 var-2 var-3)
       (dscal (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1)
        rec
        (f2cl-lib:array-slice work-%data% double-float
                              ((+ ki n)) ((1 *))
                              work-%offset%)
        1)
     (declare (ignore var-0 var-2 var-3))
     (when var-1 (setf rec var-1)))
   (multiple-value-bind (var-0 var-1 var-2 var-3)
       (dscal (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1)
        rec
        (f2cl-lib:array-slice work-%data% double-float
                              ((+ ki n2)) ((1 *))
                              work-%offset%)
        1)
     (declare (ignore var-0 var-2 var-3))
     (when var-1 (setf rec var-1)))
   (setf vmax one) (setf vcrit bignum)))
 (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add j n))
                      ((1 *)) work-%offset%)
```

```
        (-
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add j n)) ((1 *))
                        work-%offset%)
         (ddot (f2cl-lib:int-sub j ki 2)
          (f2cl-lib:array-slice t$-%data% double-float
                                ((+ ki 2) j)
                                ((1 ldt) (1 *))
                                t$-%offset%)
          1
          (f2cl-lib:array-slice work-%data% double-float
                                ((+ ki 2 n)) ((1 *))
                                work-%offset%)
          1)))
(setf (f2cl-lib:fref work-%data%
                     ((f2cl-lib:int-add j n2)) ((1 *))
                     work-%offset%)
        (-
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add j n2)) ((1 *))
                        work-%offset%)
         (ddot (f2cl-lib:int-sub j ki 2)
          (f2cl-lib:array-slice t$-%data% double-float
                                ((+ ki 2) j)
                                ((1 ldt) (1 *))
                                t$-%offset%)
          1
          (f2cl-lib:array-slice work-%data% double-float
                                ((+ ki 2 n2)) ((1 *))
                                work-%offset%)
          1)))
(setf (f2cl-lib:fref work-%data%
                     ((f2cl-lib:int-add j 1 n)) ((1 *))
                     work-%offset%)
        (-
         (f2cl-lib:fref work-%data%
                        ((f2cl-lib:int-add j 1 n))
                        ((1 *)) work-%offset%)
         (ddot (f2cl-lib:int-sub j ki 2)
          (f2cl-lib:array-slice t$-%data% double-float
                                ((+ ki 2)
                                 (f2cl-lib:int-add j 1))
                                ((1 ldt) (1 *))
                                t$-%offset%)
          1
          (f2cl-lib:array-slice work-%data% double-float
                                ((+ ki 2 n)) ((1 *))
                                work-%offset%)
          1)))
(setf (f2cl-lib:fref work-%data%
```

```
                        ((f2cl-lib:int-add j 1 n2)) ((1 *))
                        work-%offset%)
                (-
                 (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add j 1 n2))
                                ((1 *)) work-%offset%)
                 (ddot (f2cl-lib:int-sub j ki 2)
                  (f2cl-lib:array-slice t$-%data% double-float
                                        ((+ ki 2)
                                         (f2cl-lib:int-add j 1))
                                        ((1 ldt) (1 *))
                                        t$-%offset%)
                  1
                  (f2cl-lib:array-slice work-%data% double-float
                                        ((+ ki 2 n2)) ((1 *))
                                        work-%offset%)
                  1)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
             var-8 var-9 var-10 var-11 var-12 var-13 var-14
             var-15 var-16 var-17)
            (dlaln2 f2cl-lib:%true% 2 2 smin one
             (f2cl-lib:array-slice t$-%data% double-float (j j)
                                   ((1 ldt) (1 *)) t$-%offset%)
             ldt one one
             (f2cl-lib:array-slice work-%data% double-float
                                   ((+ j n)) ((1 *))
                                   work-%offset%)
             n wr (- wi) x 2 scale xnorm ierr)
          (declare
           (ignore var-0 var-1 var-2 var-5 var-9 var-12 var-13
            var-14))
          (when var-3 (setf smin var-3))
          (when var-4 (setf one var-4))
          (when var-6 (setf ldt var-6))
          (when var-7 (setf one var-7))
          (when var-8 (setf one var-8))
          (when var-10 (setf n var-10))
          (when var-11 (setf wr var-11))
          (when var-15 (setf scale var-15))
          (when var-16 (setf xnorm var-16))
          (when var-17 (setf ierr var-17)))
        (cond
         ((/= scale one)
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (dscal (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1)
               scale
               (f2cl-lib:array-slice work-%data% double-float
                                     ((+ ki n)) ((1 *))
                                     work-%offset%)
```

```
                1)
              (declare (ignore var-0 var-2 var-3))
              (when var-1 (setf scale var-1)))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (dscal (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1)
                 scale
                 (f2cl-lib:array-slice work-%data% double-float
                                       ((+ ki n2)) ((1 *))
                                       work-%offset%)
                 1)
              (declare (ignore var-0 var-2 var-3))
              (when var-1 (setf scale var-1)))))
          (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add j n))
                                ((1 *)) work-%offset%)
                  (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
          (setf (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add j n2)) ((1 *))
                                work-%offset%)
                  (f2cl-lib:fref x (1 2) ((1 2) (1 2))))
          (setf (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add j 1 n)) ((1 *))
                                work-%offset%)
                  (f2cl-lib:fref x (2 1) ((1 2) (1 2))))
          (setf (f2cl-lib:fref work-%data%
                                ((f2cl-lib:int-add j 1 n2)) ((1 *))
                                work-%offset%)
                  (f2cl-lib:fref x (2 2) ((1 2) (1 2))))
          (setf vmax
                  (max (abs (f2cl-lib:fref x (1 1) ((1 2) (1 2))))
                       (abs (f2cl-lib:fref x (1 2) ((1 2) (1 2))))
                       (abs (f2cl-lib:fref x (2 1) ((1 2) (1 2))))
                       (abs (f2cl-lib:fref x (2 2) ((1 2) (1 2))))
                       vmax))
          (setf vcrit (/ bignum vmax))))
      label200))
    (cond
     ((not over)
      (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1)
       (f2cl-lib:array-slice work-%data% double-float ((+ ki n))
                             ((1 *)) work-%offset%)
       1
       (f2cl-lib:array-slice vl-%data% double-float (ki is)
                             ((1 ldvl) (1 *)) vl-%offset%)
       1)
      (dcopy (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1)
       (f2cl-lib:array-slice work-%data% double-float ((+ ki n2))
                             ((1 *)) work-%offset%)
       1
       (f2cl-lib:array-slice vl-%data% double-float
                             (ki (f2cl-lib:int-add is 1))
```

```
                        ((1 ldvl) (1 *)) vl-%offset%)
 1)
(setf emax zero)
(f2cl-lib:fdo (k ki (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (setf emax
            (max emax
                 (+
                  (abs
                   (f2cl-lib:fref vl-%data% (k is)
                                  ((1 ldvl) (1 *))
                                  vl-%offset%))
                  (abs
                   (f2cl-lib:fref vl-%data%
                                  (k (f2cl-lib:int-add is 1))
                                  ((1 ldvl) (1 *))
                                  vl-%offset%)))))
   label220))
(setf remax (/ one emax))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (dscal (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1) remax
     (f2cl-lib:array-slice vl-%data% double-float (ki is)
                           ((1 ldvl) (1 *)) vl-%offset%)
     1)
  (declare (ignore var-0 var-2 var-3))
  (when var-1 (setf remax var-1)))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (dscal (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1) remax
     (f2cl-lib:array-slice vl-%data% double-float
                           (ki (f2cl-lib:int-add is 1))
                           ((1 ldvl) (1 *)) vl-%offset%)
     1)
  (declare (ignore var-0 var-2 var-3))
  (when var-1 (setf remax var-1)))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k
                  (f2cl-lib:int-add ki (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref vl-%data% (k is) ((1 ldvl) (1 *))
                         vl-%offset%)
            zero)
    (setf (f2cl-lib:fref vl-%data%
                         (k (f2cl-lib:int-add is 1))
                         ((1 ldvl) (1 *)) vl-%offset%)
            zero)
   label230)))
(t
 (cond
```

```
((< ki (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10)
     (dgemv "N" n (f2cl-lib:int-sub n ki 1) one
      (f2cl-lib:array-slice vl-%data% double-float
                            (1 (f2cl-lib:int-add ki 2))
                            ((1 ldvl) (1 *)) vl-%offset%)
      ldvl
      (f2cl-lib:array-slice work-%data% double-float
                            ((+ ki 2 n)) ((1 *))
                            work-%offset%)
      1
      (f2cl-lib:array-slice work-%data% double-float
                            ((+ ki n)) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice vl-%data% double-float (1 ki)
                            ((1 ldvl) (1 *)) vl-%offset%)
      1)
   (declare
    (ignore var-0 var-2 var-4 var-6 var-7 var-8 var-9
     var-10))
   (when var-1 (setf n var-1))
   (when var-3 (setf one var-3))
   (when var-5 (setf ldvl var-5)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10)
     (dgemv "N" n (f2cl-lib:int-sub n ki 1) one
      (f2cl-lib:array-slice vl-%data% double-float
                            (1 (f2cl-lib:int-add ki 2))
                            ((1 ldvl) (1 *)) vl-%offset%)
      ldvl
      (f2cl-lib:array-slice work-%data% double-float
                            ((+ ki 2 n2)) ((1 *))
                            work-%offset%)
      1
      (f2cl-lib:array-slice work-%data% double-float
                            ((+ ki 1 n2)) ((1 *))
                            work-%offset%)
      (f2cl-lib:array-slice vl-%data% double-float
                            (1 (f2cl-lib:int-add ki 1))
                            ((1 ldvl) (1 *)) vl-%offset%)
      1)
   (declare
    (ignore var-0 var-2 var-4 var-6 var-7 var-8 var-9
     var-10))
   (when var-1 (setf n var-1))
   (when var-3 (setf one var-3))
   (when var-5 (setf ldvl var-5))))
```

```
 (t
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dscal n
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ ki n)) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice vl-%data% double-float (1 ki)
                             ((1 ldvl) (1 *)) vl-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3))
    (when var-0 (setf n var-0)))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (dscal n
       (f2cl-lib:array-slice work-%data% double-float
                             ((+ ki 1 n2)) ((1 *))
                             work-%offset%)
       (f2cl-lib:array-slice vl-%data% double-float
                             (1 (f2cl-lib:int-add ki 1))
                             ((1 ldvl) (1 *)) vl-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3))
    (when var-0 (setf n var-0)))))
(setf emax zero)
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (setf emax
            (max emax
                 (+
                  (abs
                   (f2cl-lib:fref vl-%data% (k ki)
                                  ((1 ldvl) (1 *))
                                  vl-%offset%))
                  (abs
                   (f2cl-lib:fref vl-%data%
                                  (k (f2cl-lib:int-add ki 1))
                                  ((1 ldvl) (1 *))
                                  vl-%offset%)))))
   label240))
(setf remax (/ one emax))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (dscal n remax
     (f2cl-lib:array-slice vl-%data% double-float (1 ki)
                           ((1 ldvl) (1 *)) vl-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf remax var-1)))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (dscal n remax
```

```
                    (f2cl-lib:array-slice vl-%data% double-float
                                          (1 (f2cl-lib:int-add ki 1))
                                          ((1 ldvl) (1 *)) vl-%offset%)
                    1)
                 (declare (ignore var-2 var-3))
                 (when var-0 (setf n var-0))
                 (when var-1 (setf remax var-1)))))))
            (setf is (f2cl-lib:int-add is 1))
            (if (/= ip 0)
                (setf is (f2cl-lib:int-add is 1)))
           label250
            (if (= ip -1)
                (setf ip 0))
            (if (= ip 1)
                (setf ip -1))
           label260))))
      (go end_label)
     end_label
      (return
       (values side howmny nil n nil ldt nil ldvl nil ldvr nil m nil
               info))))))
```

———-

90.206 dtrexc

90.206.1 Axiom unit tests

— **dtrexc.input** —

```
)set break resume
)sys rm -f dtrexc.output
)spool dtrexc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.206.2 Axiom help page

— dtrexc.help —

```
=====================================================================
dtrexc examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

NAME
       DTREXC - the real Schur factorization of a real matrix A = Q*T*Q**T, so
       that the diagonal block of T with row index IFST is moved to row ILST

SYNOPSIS
       SUBROUTINE DTREXC( COMPQ, N, T, LDT, Q, LDQ, IFST, ILST, WORK, INFO )

           CHARACTER      COMPQ

           INTEGER        IFST, ILST, INFO, LDQ, LDT, N

           DOUBLE         PRECISION Q( LDQ, * ), T( LDT, * ), WORK( * )

  Purpose
  =======

  DTREXC reorders the real Schur factorization of a real matrix
  A = Q*T*Q**T, so that the diagonal block of T with row index IFST is
  moved to row ILST.

  The real Schur form T is reordered by an orthogonal similarity
  transformation Z**T*T*Z, and optionally the matrix Q of Schur vectors
  is updated by postmultiplying it with Z.

  T must be in Schur canonical form (as returned by DHSEQR), that is,
  block upper triangular with 1-by-1 and 2-by-2 diagonal blocks; each
  2-by-2 diagonal block has its diagonal elements equal and its
  off-diagonal elements of opposite sign.

  Arguments
  =========

  COMPQ   (input) CHARACTER*1
          = 'V':  update the matrix Q of Schur vectors;
          = 'N':  do not update Q.

  N       (input) INTEGER
          The order of the matrix T. N >= 0.

  T       (input/output) DOUBLE PRECISION array, dimension (LDT,N)
```

```
         On entry, the upper quasi-triangular matrix T, in Schur
         Schur canonical form.
         On exit, the reordered upper quasi-triangular matrix, again
         in Schur canonical form.

 LDT     (input) INTEGER
         The leading dimension of the array T. LDT >= max(1,N).

 Q       (input/output) DOUBLE PRECISION array, dimension (LDQ,N)
         On entry, if COMPQ = 'V', the matrix Q of Schur vectors.
         On exit, if COMPQ = 'V', Q has been postmultiplied by the
         orthogonal transformation matrix Z which reorders T.
         If COMPQ = 'N', Q is not referenced.

 LDQ     (input) INTEGER
         The leading dimension of the array Q.  LDQ >= max(1,N).

 IFST    (input/output) INTEGER
 ILST    (input/output) INTEGER
         Specify the reordering of the diagonal blocks of T.
         The block with row index IFST is moved to row ILST, by a
         sequence of transpositions between adjacent blocks.
         On exit, if IFST pointed on entry to the second row of a
         2-by-2 block, it is changed to point to the first row; ILST
         always points to the first row of the block in its final
         position (which may differ from its input value by +1 or -1).
         1 <= IFST <= N; 1 <= ILST <= N.

 WORK    (workspace) DOUBLE PRECISION array, dimension (N)

 INFO    (output) INTEGER
         = 0:  successful exit
         < 0:  if INFO = -i, the i-th argument had an illegal value
         = 1:  two adjacent blocks were too close to swap (the problem
               is very ill-conditioned); T may have been partially
               reordered, and ILST points to the first row of the
               current position of the block being moved.
```

———-

90.206.3 fortran code

— **dtrexc.f** —

```
      SUBROUTINE DTREXC( COMPQ, N, T, LDT, Q, LDQ, IFST, ILST, WORK,
     $                   INFO )
*
```

```
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     March 31, 1993
*
*     .. Scalar Arguments ..
      CHARACTER          COMPQ
      INTEGER            IFST, ILST, INFO, LDQ, LDT, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   Q( LDQ, * ), T( LDT, * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            WANTQ
      INTEGER            HERE, NBF, NBL, NBNEXT
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLAEXC, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
*     .. Executable Statements ..
*
*     Decode and test the input arguments.
*
      INFO = 0
      WANTQ = LSAME( COMPQ, 'V' )
      IF( .NOT.WANTQ .AND. .NOT.LSAME( COMPQ, 'N' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDT.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDQ.LT.1 .OR. ( WANTQ .AND. LDQ.LT.MAX( 1, N ) ) ) THEN
         INFO = -6
      ELSE IF( IFST.LT.1 .OR. IFST.GT.N ) THEN
         INFO = -7
      ELSE IF( ILST.LT.1 .OR. ILST.GT.N ) THEN
```

```
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DTREXC', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.LE.1 )
     $   RETURN
*
*     Determine the first row of specified block
*     and find out it is 1 by 1 or 2 by 2.
*
      IF( IFST.GT.1 ) THEN
         IF( T( IFST, IFST-1 ).NE.ZERO )
     $      IFST = IFST - 1
      END IF
      NBF = 1
      IF( IFST.LT.N ) THEN
         IF( T( IFST+1, IFST ).NE.ZERO )
     $      NBF = 2
      END IF
*
*     Determine the first row of the final block
*     and find out it is 1 by 1 or 2 by 2.
*
      IF( ILST.GT.1 ) THEN
         IF( T( ILST, ILST-1 ).NE.ZERO )
     $      ILST = ILST - 1
      END IF
      NBL = 1
      IF( ILST.LT.N ) THEN
         IF( T( ILST+1, ILST ).NE.ZERO )
     $      NBL = 2
      END IF
*
      IF( IFST.EQ.ILST )
     $   RETURN
*
      IF( IFST.LT.ILST ) THEN
*
*        Update ILST
*
         IF( NBF.EQ.2 .AND. NBL.EQ.1 )
     $      ILST = ILST - 1
         IF( NBF.EQ.1 .AND. NBL.EQ.2 )
     $      ILST = ILST + 1
*
```

```
         HERE = IFST
*
   10    CONTINUE
*
*        Swap block with next one below
*
         IF( NBF.EQ.1 .OR. NBF.EQ.2 ) THEN
*
*           Current block either 1 by 1 or 2 by 2
*
            NBNEXT = 1
            IF( HERE+NBF+1.LE.N ) THEN
               IF( T( HERE+NBF+1, HERE+NBF ).NE.ZERO )
     $            NBNEXT = 2
            END IF
            CALL DLAEXC( WANTQ, N, T, LDT, Q, LDQ, HERE, NBF, NBNEXT,
     $                   WORK, INFO )
            IF( INFO.NE.0 ) THEN
               ILST = HERE
               RETURN
            END IF
            HERE = HERE + NBNEXT
*
*           Test if 2 by 2 block breaks into two 1 by 1 blocks
*
            IF( NBF.EQ.2 ) THEN
               IF( T( HERE+1, HERE ).EQ.ZERO )
     $            NBF = 3
            END IF
*
         ELSE
*
*           Current block consists of two 1 by 1 blocks each of which
*           must be swapped individually
*
            NBNEXT = 1
            IF( HERE+3.LE.N ) THEN
               IF( T( HERE+3, HERE+2 ).NE.ZERO )
     $            NBNEXT = 2
            END IF
            CALL DLAEXC( WANTQ, N, T, LDT, Q, LDQ, HERE+1, 1, NBNEXT,
     $                   WORK, INFO )
            IF( INFO.NE.0 ) THEN
               ILST = HERE
               RETURN
            END IF
            IF( NBNEXT.EQ.1 ) THEN
*
*              Swap two 1 by 1 blocks, no problems possible
*
```

```
               CALL DLAEXC( WANTQ, N, T, LDT, Q, LDQ, HERE, 1, NBNEXT,
     $                      WORK, INFO )
               HERE = HERE + 1
            ELSE
*
*              Recompute NBNEXT in case 2 by 2 split
*
               IF( T( HERE+2, HERE+1 ).EQ.ZERO )
     $            NBNEXT = 1
               IF( NBNEXT.EQ.2 ) THEN
*
*                 2 by 2 Block did not split
*
                  CALL DLAEXC( WANTQ, N, T, LDT, Q, LDQ, HERE, 1,
     $                         NBNEXT, WORK, INFO )
                  IF( INFO.NE.0 ) THEN
                     ILST = HERE
                     RETURN
                  END IF
                  HERE = HERE + 2
               ELSE
*
*                 2 by 2 Block did split
*
                  CALL DLAEXC( WANTQ, N, T, LDT, Q, LDQ, HERE, 1, 1,
     $                         WORK, INFO )
                  CALL DLAEXC( WANTQ, N, T, LDT, Q, LDQ, HERE+1, 1, 1,
     $                         WORK, INFO )
                  HERE = HERE + 2
               END IF
            END IF
         END IF
         IF( HERE.LT.ILST )
     $      GO TO 10
*
      ELSE
*
         HERE = IFST
   20    CONTINUE
*
*        Swap block with next one above
*
         IF( NBF.EQ.1 .OR. NBF.EQ.2 ) THEN
*
*           Current block either 1 by 1 or 2 by 2
*
            NBNEXT = 1
            IF( HERE.GE.3 ) THEN
               IF( T( HERE-1, HERE-2 ).NE.ZERO )
     $            NBNEXT = 2
```

```
            END IF
            CALL DLAEXC( WANTQ, N, T, LDT, Q, LDQ, HERE-NBNEXT, NBNEXT,
     $                   NBF, WORK, INFO )
            IF( INFO.NE.0 ) THEN
               ILST = HERE
               RETURN
            END IF
            HERE = HERE - NBNEXT
*
*           Test if 2 by 2 block breaks into two 1 by 1 blocks
*
            IF( NBF.EQ.2 ) THEN
               IF( T( HERE+1, HERE ).EQ.ZERO )
     $            NBF = 3
            END IF
*
         ELSE
*
*           Current block consists of two 1 by 1 blocks each of which
*           must be swapped individually
*
            NBNEXT = 1
            IF( HERE.GE.3 ) THEN
               IF( T( HERE-1, HERE-2 ).NE.ZERO )
     $            NBNEXT = 2
            END IF
            CALL DLAEXC( WANTQ, N, T, LDT, Q, LDQ, HERE-NBNEXT, NBNEXT,
     $                   1, WORK, INFO )
            IF( INFO.NE.0 ) THEN
               ILST = HERE
               RETURN
            END IF
            IF( NBNEXT.EQ.1 ) THEN
*
*              Swap two 1 by 1 blocks, no problems possible
*
               CALL DLAEXC( WANTQ, N, T, LDT, Q, LDQ, HERE, NBNEXT, 1,
     $                      WORK, INFO )
               HERE = HERE - 1
            ELSE
*
*              Recompute NBNEXT in case 2 by 2 split
*
               IF( T( HERE, HERE-1 ).EQ.ZERO )
     $            NBNEXT = 1
               IF( NBNEXT.EQ.2 ) THEN
*
*                 2 by 2 Block did not split
*
                  CALL DLAEXC( WANTQ, N, T, LDT, Q, LDQ, HERE-1, 2, 1,
```

```
     $                        WORK, INFO )
                 IF( INFO.NE.0 ) THEN
                    ILST = HERE
                    RETURN
                 END IF
                 HERE = HERE - 2
              ELSE
*
*                2 by 2 Block did split
*
                 CALL DLAEXC( WANTQ, N, T, LDT, Q, LDQ, HERE, 1, 1,
     $                        WORK, INFO )
                 CALL DLAEXC( WANTQ, N, T, LDT, Q, LDQ, HERE-1, 1, 1,
     $                        WORK, INFO )
                 HERE = HERE - 2
              END IF
           END IF
        END IF
        IF( HERE.GT.ILST )
     $      GO TO 20
      END IF
      ILST = HERE
*
      RETURN
*
*     End of DTREXC
*
      END
```

90.206.4 lisp code

— dtrexc.lisp —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun dtrexc (compq n t$ ldt q ldq ifst ilst work info)
    (declare (type (array double-float (*)) work q t$)
             (type (f2cl-lib:integer4) info ilst ifst ldq ldt n)
             (type (string *) compq))
    (f2cl-lib:with-multi-array-data
        ((compq character compq-%data% compq-%offset%)
         (t$ double-float t$-%data% t$-%offset%)
         (q double-float q-%data% q-%offset%)
         (work double-float work-%data% work-%offset%))
```

```
(prog ((here 0) (nbf 0) (nbl 0) (nbnext 0) (wantq nil))
  (declare (type (f2cl-lib:integer4) here nbf nbl nbnext)
           (type f2cl-lib:logical wantq))
  (setf info 0)
  (setf wantq
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame compq "V")
            (declare (ignore var-1))
            (when var-0 (setf compq var-0))
            ret-val))
  (cond
   ((and (not wantq)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame compq "N")
            (declare (ignore var-1))
            (when var-0 (setf compq var-0))
            ret-val)))
    (setf info -1))
   ((< n 0) (setf info -2))
   ((< ldt (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
    (setf info -4))
   ((or (< ldq 1)
        (and wantq
             (< ldq
                (max (the f2cl-lib:integer4 1)
                     (the f2cl-lib:integer4 n)))))
    (setf info -6))
   ((or (< ifst 1) (> ifst n)) (setf info -7))
   ((or (< ilst 1) (> ilst n)) (setf info -8)))
  (cond
   ((/= info 0) (xerbla "DTREXC" (f2cl-lib:int-sub info))
    (go end_label)))
  (if (<= n 1)
      (go end_label))
  (cond
   ((> ifst 1)
    (if (/=
         (f2cl-lib:fref t$-%data% (ifst (f2cl-lib:int-sub ifst 1))
                        ((1 ldt) (1 *)) t$-%offset%)
         zero)
        (setf ifst (f2cl-lib:int-sub ifst 1)))))
  (setf nbf 1)
  (cond
   ((< ifst n)
    (if (/=
         (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add ifst 1) ifst)
                        ((1 ldt) (1 *)) t$-%offset%)
         zero)
        (setf nbf 2))))
```

```
(cond
 ((> ilst 1)
  (if (/=
       (f2cl-lib:fref t$-%data% (ilst (f2cl-lib:int-sub ilst 1))
                      ((1 ldt) (1 *)) t$-%offset%)
       zero)
      (setf ilst (f2cl-lib:int-sub ilst 1)))))
(setf nbl 1)
(cond
 ((< ilst n)
  (if (/=
       (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add ilst 1) ilst)
                      ((1 ldt) (1 *)) t$-%offset%)
       zero)
      (setf nbl 2))))
(if (= ifst ilst)
    (go end_label))
(cond
 ((< ifst ilst)
  (tagbody
    (if (and (= nbf 2) (= nbl 1))
        (setf ilst (f2cl-lib:int-sub ilst 1)))
    (if (and (= nbf 1) (= nbl 2))
        (setf ilst (f2cl-lib:int-add ilst 1)))
    (setf here ifst)
   label10
    (cond
     ((or (= nbf 1) (= nbf 2)) (setf nbnext 1)
      (cond
       ((<= (f2cl-lib:int-add here nbf 1) n)
        (if (/=
             (f2cl-lib:fref t$-%data%
                            ((f2cl-lib:int-add here nbf 1)
                             (f2cl-lib:int-add here nbf))
                            ((1 ldt) (1 *)) t$-%offset%)
             zero)
            (setf nbnext 2))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10)
          (dlaexc wantq n t$ ldt q ldq here nbf nbnext work info)
        (declare (ignore var-2 var-4 var-9))
        (when var-0 (setf wantq var-0))
        (when var-1 (setf n var-1))
        (when var-3 (setf ldt var-3))
        (when var-5 (setf ldq var-5))
        (when var-6 (setf here var-6))
        (when var-7 (setf nbf var-7))
        (when var-8 (setf nbnext var-8))
        (when var-10 (setf info var-10)))
```

```
 (cond ((/= info 0) (setf ilst here) (go end_label)))
 (setf here (f2cl-lib:int-add here nbnext))
 (cond
  ((= nbf 2)
   (if (=
        (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add here 1) here)
                       ((1 ldt) (1 *)) t$-%offset%)
        zero)
       (setf nbf 3)))))
(t (setf nbnext 1)
 (cond
  ((<= (f2cl-lib:int-add here 3) n)
   (if (/=
        (f2cl-lib:fref t$-%data%
                       ((f2cl-lib:int-add here 3)
                        (f2cl-lib:int-add here 2))
                       ((1 ldt) (1 *)) t$-%offset%)
        zero)
       (setf nbnext 2))))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (dlaexc wantq n t$ ldt q ldq (f2cl-lib:int-add here 1) 1
      nbnext work info)
   (declare (ignore var-2 var-4 var-6 var-7 var-9))
   (when var-0 (setf wantq var-0))
   (when var-1 (setf n var-1))
   (when var-3 (setf ldt var-3))
   (when var-5 (setf ldq var-5))
   (when var-8 (setf nbnext var-8))
   (when var-10 (setf info var-10)))
 (cond ((/= info 0) (setf ilst here) (go end_label)))
 (cond
  ((= nbnext 1)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10)
       (dlaexc wantq n t$ ldt q ldq here 1 nbnext work info)
     (declare (ignore var-2 var-4 var-7 var-9))
     (when var-0 (setf wantq var-0))
     (when var-1 (setf n var-1))
     (when var-3 (setf ldt var-3))
     (when var-5 (setf ldq var-5))
     (when var-6 (setf here var-6))
     (when var-8 (setf nbnext var-8))
     (when var-10 (setf info var-10)))
   (setf here (f2cl-lib:int-add here 1)))
  (t
   (if (=
        (f2cl-lib:fref t$-%data%
```

```
                                ((f2cl-lib:int-add here 2)
                                 (f2cl-lib:int-add here 1))
                                ((1 ldt) (1 *)) t$-%offset%)
                 zero)
                (setf nbnext 1))
            (cond
             ((= nbnext 2)
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                   var-9 var-10)
                  (dlaexc wantq n t$ ldt q ldq here 1 nbnext work info)
                (declare (ignore var-2 var-4 var-7 var-9))
                (when var-0 (setf wantq var-0))
                (when var-1 (setf n var-1))
                (when var-3 (setf ldt var-3))
                (when var-5 (setf ldq var-5))
                (when var-6 (setf here var-6))
                (when var-8 (setf nbnext var-8))
                (when var-10 (setf info var-10)))
              (cond ((/= info 0) (setf ilst here) (go end_label)))
              (setf here (f2cl-lib:int-add here 2)))
             (t
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                   var-9 var-10)
                  (dlaexc wantq n t$ ldt q ldq here 1 1 work info)
                (declare (ignore var-2 var-4 var-7 var-8 var-9))
                (when var-0 (setf wantq var-0))
                (when var-1 (setf n var-1))
                (when var-3 (setf ldt var-3))
                (when var-5 (setf ldq var-5))
                (when var-6 (setf here var-6))
                (when var-10 (setf info var-10)))
              (multiple-value-bind
                  (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                   var-9 var-10)
                  (dlaexc wantq n t$ ldt q ldq (f2cl-lib:int-add here 1) 1
                   1 work info)
                (declare (ignore var-2 var-4 var-6 var-7 var-8 var-9))
                (when var-0 (setf wantq var-0))
                (when var-1 (setf n var-1))
                (when var-3 (setf ldt var-3))
                (when var-5 (setf ldq var-5))
                (when var-10 (setf info var-10)))
              (setf here (f2cl-lib:int-add here 2))))))))
    (if (< here ilst)
        (go label10))))
(t
 (tagbody
   (setf here ifst)
```

```
label20
 (cond
  ((or (= nbf 1) (= nbf 2)) (setf nbnext 1)
   (cond
    ((>= here 3)
     (if (/=
          (f2cl-lib:fref t$-%data%
                         ((f2cl-lib:int-sub here 1)
                          (f2cl-lib:int-sub here 2))
                         ((1 ldt) (1 *)) t$-%offset%)
          zero)
         (setf nbnext 2))))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (dlaexc wantq n t$ ldt q ldq (f2cl-lib:int-sub here nbnext)
        nbnext nbf work info)
     (declare (ignore var-2 var-4 var-6 var-9))
     (when var-0 (setf wantq var-0))
     (when var-1 (setf n var-1))
     (when var-3 (setf ldt var-3))
     (when var-5 (setf ldq var-5))
     (when var-7 (setf nbnext var-7))
     (when var-8 (setf nbf var-8))
     (when var-10 (setf info var-10)))
   (cond ((/= info 0) (setf ilst here) (go end_label)))
   (setf here (f2cl-lib:int-sub here nbnext))
   (cond
    ((= nbf 2)
     (if (=
          (f2cl-lib:fref t$-%data% ((f2cl-lib:int-add here 1) here)
                         ((1 ldt) (1 *)) t$-%offset%)
          zero)
         (setf nbf 3)))))
  (t (setf nbnext 1)
   (cond
    ((>= here 3)
     (if (/=
          (f2cl-lib:fref t$-%data%
                         ((f2cl-lib:int-sub here 1)
                          (f2cl-lib:int-sub here 2))
                         ((1 ldt) (1 *)) t$-%offset%)
          zero)
         (setf nbnext 2))))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (dlaexc wantq n t$ ldt q ldq (f2cl-lib:int-sub here nbnext)
        nbnext 1 work info)
     (declare (ignore var-2 var-4 var-6 var-8 var-9))
```

```
  (when var-0 (setf wantq var-0))
  (when var-1 (setf n var-1))
  (when var-3 (setf ldt var-3))
  (when var-5 (setf ldq var-5))
  (when var-7 (setf nbnext var-7))
  (when var-10 (setf info var-10)))
(cond ((/= info 0) (setf ilst here) (go end_label)))
(cond
 ((= nbnext 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (dlaexc wantq n t$ ldt q ldq here nbnext 1 work info)
    (declare (ignore var-2 var-4 var-8 var-9))
    (when var-0 (setf wantq var-0))
    (when var-1 (setf n var-1))
    (when var-3 (setf ldt var-3))
    (when var-5 (setf ldq var-5))
    (when var-6 (setf here var-6))
    (when var-7 (setf nbnext var-7))
    (when var-10 (setf info var-10)))
  (setf here (f2cl-lib:int-sub here 1)))
 (t
  (if (=
       (f2cl-lib:fref t$-%data% (here (f2cl-lib:int-sub here 1))
                      ((1 ldt) (1 *)) t$-%offset%)
       zero)
      (setf nbnext 1))
  (cond
   ((= nbnext 2)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10)
        (dlaexc wantq n t$ ldt q ldq (f2cl-lib:int-sub here 1) 2
         1 work info)
      (declare (ignore var-2 var-4 var-6 var-7 var-8 var-9))
      (when var-0 (setf wantq var-0))
      (when var-1 (setf n var-1))
      (when var-3 (setf ldt var-3))
      (when var-5 (setf ldq var-5))
      (when var-10 (setf info var-10)))
    (cond ((/= info 0) (setf ilst here) (go end_label)))
    (setf here (f2cl-lib:int-sub here 2)))
   (t
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10)
        (dlaexc wantq n t$ ldt q ldq here 1 1 work info)
      (declare (ignore var-2 var-4 var-7 var-8 var-9))
      (when var-0 (setf wantq var-0))
```

```
                  (when var-1 (setf n var-1))
                  (when var-3 (setf ldt var-3))
                  (when var-5 (setf ldq var-5))
                  (when var-6 (setf here var-6))
                  (when var-10 (setf info var-10)))
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                     var-9 var-10)
                    (dlaexc wantq n t$ ldt q ldq (f2cl-lib:int-sub here 1) 1
                     1 work info)
                  (declare (ignore var-2 var-4 var-6 var-7 var-8 var-9))
                  (when var-0 (setf wantq var-0))
                  (when var-1 (setf n var-1))
                  (when var-3 (setf ldt var-3))
                  (when var-5 (setf ldq var-5))
                  (when var-10 (setf info var-10)))
                (setf here (f2cl-lib:int-sub here 2))))))))
            (if (> here ilst)
                (go label20)))))
        (setf ilst here)
        (go end_label)
       end_label
        (return (values compq n nil ldt nil ldq ifst ilst nil info))))))
```

———-

90.207 dtrsna

90.207.1 Axiom unit tests

— **dtrsna.input** —

```
)set break resume
)sys rm -f dtrsna.output
)spool dtrsna.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.207.2 Axiom help page

— dtrsna.help —

```
==================================================================
dtrsna examples
==================================================================

==================================================================
Man Page Details
==================================================================

NAME
       DTRSNA  - reciprocal condition numbers for specified eigenvalues and/or
       right eigenvectors of a real upper quasi-triangular matrix T (or of any
       matrix Q*T*Q**T with Q orthogonal)

SYNOPSIS
       SUBROUTINE DTRSNA( JOB,  HOWMNY, SELECT, N, T, LDT, VL, LDVL, VR, LDVR,
                          S, SEP, MM, M, WORK, LDWORK, IWORK, INFO )

           CHARACTER      HOWMNY, JOB

           INTEGER        INFO, LDT, LDVL, LDVR, LDWORK, M, MM, N

           LOGICAL        SELECT( * )

           INTEGER        IWORK( * )

           DOUBLE         PRECISION S( * ), SEP( * ), T( LDT, * ), VL( LDVL, *
                          ), VR( LDVR, * ), WORK( LDWORK, * )

  Purpose
  =======

  DTRSNA estimates reciprocal condition numbers for specified
  eigenvalues and/or right eigenvectors of a real upper
  quasi-triangular matrix T (or of any matrix Q*T*Q**T with Q
  orthogonal).

  T must be in Schur canonical form (as returned by DHSEQR), that is,
  block upper triangular with 1-by-1 and 2-by-2 diagonal blocks; each
  2-by-2 diagonal block has its diagonal elements equal and its
  off-diagonal elements of opposite sign.

  Arguments
  =========

  JOB     (input) CHARACTER*1
```

```
        Specifies whether condition numbers are required for
        eigenvalues (S) or eigenvectors (SEP):
        = 'E': for eigenvalues only (S);
        = 'V': for eigenvectors only (SEP);
        = 'B': for both eigenvalues and eigenvectors (S and SEP).

HOWMNY  (input) CHARACTER*1
        = 'A': compute condition numbers for all eigenpairs;
        = 'S': compute condition numbers for selected eigenpairs
               specified by the array SELECT.

SELECT  (input) LOGICAL array, dimension (N)
        If HOWMNY = 'S', SELECT specifies the eigenpairs for which
        condition numbers are required. To select condition numbers
        for the eigenpair corresponding to a real eigenvalue w(j),
        SELECT(j) must be set to .TRUE.. To select condition numbers
        corresponding to a complex conjugate pair of eigenvalues w(j)
        and w(j+1), either SELECT(j) or SELECT(j+1) or both, must be
        set to .TRUE..
        If HOWMNY = 'A', SELECT is not referenced.

N       (input) INTEGER
        The order of the matrix T. N >= 0.

T       (input) DOUBLE PRECISION array, dimension (LDT,N)
        The upper quasi-triangular matrix T, in Schur canonical form.

LDT     (input) INTEGER
        The leading dimension of the array T. LDT >= max(1,N).

VL      (input) DOUBLE PRECISION array, dimension (LDVL,M)
        If JOB = 'E' or 'B', VL must contain left eigenvectors of T
        (or of any Q*T*Q**T with Q orthogonal), corresponding to the
        eigenpairs specified by HOWMNY and SELECT. The eigenvectors
        must be stored in consecutive columns of VL, as returned by
        DHSEIN or DTREVC.
        If JOB = 'V', VL is not referenced.

LDVL    (input) INTEGER
        The leading dimension of the array VL.
        LDVL >= 1; and if JOB = 'E' or 'B', LDVL >= N.

VR      (input) DOUBLE PRECISION array, dimension (LDVR,M)
        If JOB = 'E' or 'B', VR must contain right eigenvectors of T
        (or of any Q*T*Q**T with Q orthogonal), corresponding to the
        eigenpairs specified by HOWMNY and SELECT. The eigenvectors
        must be stored in consecutive columns of VR, as returned by
        DHSEIN or DTREVC.
        If JOB = 'V', VR is not referenced.
```

```
LDVR    (input) INTEGER
        The leading dimension of the array VR.
        LDVR >= 1; and if JOB = 'E' or 'B', LDVR >= N.

S       (output) DOUBLE PRECISION array, dimension (MM)
        If JOB = 'E' or 'B', the reciprocal condition numbers of the
        selected eigenvalues, stored in consecutive elements of the
        array. For a complex conjugate pair of eigenvalues two
        consecutive elements of S are set to the same value. Thus
        S(j), SEP(j), and the j-th columns of VL and VR all
        correspond to the same eigenpair (but not in general the
        j-th eigenpair, unless all eigenpairs are selected).
        If JOB = 'V', S is not referenced.

SEP     (output) DOUBLE PRECISION array, dimension (MM)
        If JOB = 'V' or 'B', the estimated reciprocal condition
        numbers of the selected eigenvectors, stored in consecutive
        elements of the array. For a complex eigenvector two
        consecutive elements of SEP are set to the same value. If
        the eigenvalues cannot be reordered to compute SEP(j), SEP(j)
        is set to 0; this can only occur when the true value would be
        very small anyway.
        If JOB = 'E', SEP is not referenced.

MM      (input) INTEGER
        The number of elements in the arrays S (if JOB = 'E' or 'B')
         and/or SEP (if JOB = 'V' or 'B'). MM >= M.

M       (output) INTEGER
        The number of elements of the arrays S and/or SEP actually
        used to store the estimated condition numbers.
        If HOWMNY = 'A', M is set to N.

WORK    (workspace) DOUBLE PRECISION array, dimension (LDWORK,N+1)
        If JOB = 'E', WORK is not referenced.

LDWORK  (input) INTEGER
        The leading dimension of the array WORK.
        LDWORK >= 1; and if JOB = 'V' or 'B', LDWORK >= N.

IWORK   (workspace) INTEGER array, dimension (N)
        If JOB = 'E', IWORK is not referenced.

INFO    (output) INTEGER
        = 0: successful exit
        < 0: if INFO = -i, the i-th argument had an illegal value

Further Details
===============
```

```
The reciprocal of the condition number of an eigenvalue lambda is
defined as

        S(lambda) = |v'*u| / (norm(u)*norm(v))

where u and v are the right and left eigenvectors of T corresponding
to lambda; v' denotes the conjugate-transpose of v, and norm(u)
denotes the Euclidean norm. These reciprocal condition numbers always
lie between zero (very badly conditioned) and one (very well
conditioned). If n = 1, S(lambda) is defined to be 1.

An approximate error bound for a computed eigenvalue W(i) is given by

                    EPS * norm(T) / S(i)

where EPS is the machine precision.

The reciprocal of the condition number of the right eigenvector u
corresponding to lambda is defined as follows. Suppose

            T = ( lambda  c  )
                (   0    T22 )

Then the reciprocal condition number is

        SEP( lambda, T22 ) = sigma-min( T22 - lambda*I )

where sigma-min denotes the smallest singular value. We approximate
the smallest singular value by the reciprocal of an estimate of the
one-norm of the inverse of T22 - lambda*I. If n = 1, SEP(1) is
defined to be abs(T(1,1)).

An approximate error bound for a computed right eigenvector VR(i)
is given by

                    EPS * norm(T) / SEP(i)
```

———-

90.207.3 fortran code

— dtrsna.f —

```
      SUBROUTINE DTRSNA( JOB, HOWMNY, SELECT, N, T, LDT, VL, LDVL, VR,
     $                   LDVR, S, SEP, MM, M, WORK, LDWORK, IWORK,
     $                   INFO )
*
```

```
*  -- LAPACK routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     September 30, 1994
*
*     .. Scalar Arguments ..
      CHARACTER          HOWMNY, JOB
      INTEGER            INFO, LDT, LDVL, LDVR, LDWORK, M, MM, N
*     ..
*     .. Array Arguments ..
      LOGICAL            SELECT( * )
      INTEGER            IWORK( * )
      DOUBLE PRECISION   S( * ), SEP( * ), T( LDT, * ), VL( LDVL, * ),
     $                   VR( LDVR, * ), WORK( LDWORK, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE, TWO
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0, TWO = 2.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            PAIR, SOMCON, WANTBH, WANTS, WANTSP
      INTEGER            I, IERR, IFST, ILST, J, K, KASE, KS, N2, NN
      DOUBLE PRECISION   BIGNUM, COND, CS, DELTA, DUMM, EPS, EST, LNRM,
     $                   MU, PROD, PROD1, PROD2, RNRM, SCALE, SMLNUM, SN
*     ..
*     .. Local Arrays ..
      DOUBLE PRECISION   DUMMY( 1 )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      DOUBLE PRECISION   DDOT, DLAMCH, DLAPY2, DNRM2
      EXTERNAL           LSAME, DDOT, DLAMCH, DLAPY2, DNRM2
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLACON, DLACPY, DLAQTR, DTREXC, XERBLA
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Decode and test the input parameters
*
      WANTBH = LSAME( JOB, 'B' )
      WANTS = LSAME( JOB, 'E' ) .OR. WANTBH
      WANTSP = LSAME( JOB, 'V' ) .OR. WANTBH
*
```

```
      SOMCON = LSAME( HOWMNY, 'S' )
*
      INFO = 0
      IF( .NOT.WANTS .AND. .NOT.WANTSP ) THEN
         INFO = -1
      ELSE IF( .NOT.LSAME( HOWMNY, 'A' ) .AND. .NOT.SOMCON ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDT.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDVL.LT.1 .OR. ( WANTS .AND. LDVL.LT.N ) ) THEN
         INFO = -8
      ELSE IF( LDVR.LT.1 .OR. ( WANTS .AND. LDVR.LT.N ) ) THEN
         INFO = -10
      ELSE
*
*        Set M to the number of eigenpairs for which condition numbers
*        are required, and test MM.
*
         IF( SOMCON ) THEN
            M = 0
            PAIR = .FALSE.
            DO 10 K = 1, N
               IF( PAIR ) THEN
                  PAIR = .FALSE.
               ELSE
                  IF( K.LT.N ) THEN
                     IF( T( K+1, K ).EQ.ZERO ) THEN
                        IF( SELECT( K ) )
     $                     M = M + 1
                     ELSE
                        PAIR = .TRUE.
                        IF( SELECT( K ) .OR. SELECT( K+1 ) )
     $                     M = M + 2
                     END IF
                  ELSE
                     IF( SELECT( N ) )
     $                  M = M + 1
                  END IF
               END IF
   10       CONTINUE
         ELSE
            M = N
         END IF
*
         IF( MM.LT.M ) THEN
            INFO = -13
         ELSE IF( LDWORK.LT.1 .OR. ( WANTSP .AND. LDWORK.LT.N ) ) THEN
            INFO = -16
```

```
         END IF
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'DTRSNA', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
      IF( N.EQ.1 ) THEN
         IF( SOMCON ) THEN
            IF( .NOT.SELECT( 1 ) )
     $         RETURN
         END IF
         IF( WANTS )
     $      S( 1 ) = ONE
         IF( WANTSP )
     $      SEP( 1 ) = ABS( T( 1, 1 ) )
         RETURN
      END IF
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' ) / EPS
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
*
      KS = 0
      PAIR = .FALSE.
      DO 60 K = 1, N
*
*        Determine whether T(k,k) begins a 1-by-1 or 2-by-2 block.
*
         IF( PAIR ) THEN
            PAIR = .FALSE.
            GO TO 60
         ELSE
            IF( K.LT.N )
     $         PAIR = T( K+1, K ).NE.ZERO
         END IF
*
*        Determine whether condition numbers are required for the k-th
*        eigenpair.
*
         IF( SOMCON ) THEN
            IF( PAIR ) THEN
```

```
               IF( .NOT.SELECT( K ) .AND. .NOT.SELECT( K+1 ) )
     $            GO TO 60
            ELSE
               IF( .NOT.SELECT( K ) )
     $            GO TO 60
            END IF
         END IF
*
         KS = KS + 1
*
         IF( WANTS ) THEN
*
*           Compute the reciprocal condition number of the k-th
*           eigenvalue.
*
            IF( .NOT.PAIR ) THEN
*
*              Real eigenvalue.
*
               PROD = DDOT( N, VR( 1, KS ), 1, VL( 1, KS ), 1 )
               RNRM = DNRM2( N, VR( 1, KS ), 1 )
               LNRM = DNRM2( N, VL( 1, KS ), 1 )
               S( KS ) = ABS( PROD ) / ( RNRM*LNRM )
            ELSE
*
*              Complex eigenvalue.
*
               PROD1 = DDOT( N, VR( 1, KS ), 1, VL( 1, KS ), 1 )
               PROD1 = PROD1 + DDOT( N, VR( 1, KS+1 ), 1, VL( 1, KS+1 ),
     $                 1 )
               PROD2 = DDOT( N, VL( 1, KS ), 1, VR( 1, KS+1 ), 1 )
               PROD2 = PROD2 - DDOT( N, VL( 1, KS+1 ), 1, VR( 1, KS ),
     $                 1 )
               RNRM = DLAPY2( DNRM2( N, VR( 1, KS ), 1 ),
     $                DNRM2( N, VR( 1, KS+1 ), 1 ) )
               LNRM = DLAPY2( DNRM2( N, VL( 1, KS ), 1 ),
     $                DNRM2( N, VL( 1, KS+1 ), 1 ) )
               COND = DLAPY2( PROD1, PROD2 ) / ( RNRM*LNRM )
               S( KS ) = COND
               S( KS+1 ) = COND
            END IF
         END IF
*
         IF( WANTSP ) THEN
*
*           Estimate the reciprocal condition number of the k-th
*           eigenvector.
*
*           Copy the matrix T to the array WORK and swap the diagonal
*           block beginning at T(k,k) to the (1,1) position.
```

```
*
            CALL DLACPY( 'Full', N, N, T, LDT, WORK, LDWORK )
            IFST = K
            ILST = 1
            CALL DTREXC( 'No Q', N, WORK, LDWORK, DUMMY, 1, IFST, ILST,
     $                   WORK( 1, N+1 ), IERR )
*
            IF( IERR.EQ.1 .OR. IERR.EQ.2 ) THEN
*
*              Could not swap because blocks not well separated
*
               SCALE = ONE
               EST = BIGNUM
            ELSE
*
*              Reordering successful
*
               IF( WORK( 2, 1 ).EQ.ZERO ) THEN
*
*                 Form C = T22 - lambda*I in WORK(2:N,2:N).
*
                  DO 20 I = 2, N
                     WORK( I, I ) = WORK( I, I ) - WORK( 1, 1 )
   20             CONTINUE
                  N2 = 1
                  NN = N - 1
               ELSE
*
*                 Triangularize the 2 by 2 block by unitary
*                 transformation U = [  cs   i*ss ]
*                                    [ i*ss   cs  ].
*                 such that the (1,1) position of WORK is complex
*                 eigenvalue lambda with positive imaginary part. (2,2)
*                 position of WORK is the complex eigenvalue lambda
*                 with negative imaginary  part.
*
                  MU = SQRT( ABS( WORK( 1, 2 ) ) )*
     $                 SQRT( ABS( WORK( 2, 1 ) ) )
                  DELTA = DLAPY2( MU, WORK( 2, 1 ) )
                  CS = MU / DELTA
                  SN = -WORK( 2, 1 ) / DELTA
*
*                 Form
*
*                 C' = WORK(2:N,2:N) + i*[rwork(1) ..... rwork(n-1) ]
*                                        [   mu                     ]
*                                        [         ..               ]
*                                        [             ..           ]
*                                        [                  mu      ]
*                 where C' is conjugate transpose of complex matrix C,
```

```
*                 and RWORK is stored starting in the N+1-st column of
*                 WORK.
*
                  DO 30 J = 3, N
                     WORK( 2, J ) = CS*WORK( 2, J )
                     WORK( J, J ) = WORK( J, J ) - WORK( 1, 1 )
   30             CONTINUE
                  WORK( 2, 2 ) = ZERO
*
                  WORK( 1, N+1 ) = TWO*MU
                  DO 40 I = 2, N - 1
                     WORK( I, N+1 ) = SN*WORK( 1, I+1 )
   40             CONTINUE
                  N2 = 2
                  NN = 2*( N-1 )
               END IF
*
*              Estimate norm(inv(C'))
*
               EST = ZERO
               KASE = 0
   50          CONTINUE
               CALL DLACON( NN, WORK( 1, N+2 ), WORK( 1, N+4 ), IWORK,
     $                      EST, KASE )
               IF( KASE.NE.0 ) THEN
                  IF( KASE.EQ.1 ) THEN
                     IF( N2.EQ.1 ) THEN
*
                        Real eigenvalue: solve C'*x = scale*c.
*
                        CALL DLAQTR( .TRUE., .TRUE., N-1, WORK( 2, 2 ),
     $                               LDWORK, DUMMY, DUMM, SCALE,
     $                               WORK( 1, N+4 ), WORK( 1, N+6 ),
     $                               IERR )
                     ELSE
*
                        Complex eigenvalue: solve
                        C'*(p+iq) = scale*(c+id) in real arithmetic.
*
                        CALL DLAQTR( .TRUE., .FALSE., N-1, WORK( 2, 2 ),
     $                               LDWORK, WORK( 1, N+1 ), MU, SCALE,
     $                               WORK( 1, N+4 ), WORK( 1, N+6 ),
     $                               IERR )
                     END IF
                  ELSE
                     IF( N2.EQ.1 ) THEN
*
                        Real eigenvalue: solve C*x = scale*c.
*
                        CALL DLAQTR( .FALSE., .TRUE., N-1, WORK( 2, 2 ),
```

```
     $                              LDWORK, DUMMY, DUMM, SCALE,
     $                              WORK( 1, N+4 ), WORK( 1, N+6 ),
     $                              IERR )
                     ELSE
*
*                       Complex eigenvalue: solve
*                       C*(p+iq) = scale*(c+id) in real arithmetic.
*
                        CALL DLAQTR( .FALSE., .FALSE., N-1,
     $                              WORK( 2, 2 ), LDWORK,
     $                              WORK( 1, N+1 ), MU, SCALE,
     $                              WORK( 1, N+4 ), WORK( 1, N+6 ),
     $                              IERR )
*
                        END IF
                   END IF
*
                   GO TO 50
                END IF
            END IF
*
            SEP( KS ) = SCALE / MAX( EST, SMLNUM )
            IF( PAIR )
     $         SEP( KS+1 ) = SEP( KS )
         END IF
*
         IF( PAIR )
     $      KS = KS + 1
*
   60 CONTINUE
      RETURN
*
*     End of DTRSNA
*
      END
```

———-

90.207.4 lisp code

— dtrsna.lisp —

```
(let* ((zero 0.0d0) (one 1.0d0) (two 2.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (ignorable zero one two))
```

```
(defun dtrsna
       (job howmny select n t$ ldt vl ldvl vr ldvr s sep mm m work ldwork
        iwork info)
  (declare (type (array f2cl-lib:integer4 (*)) iwork)
           (type (array double-float (*)) work sep s vr vl t$)
           (type (f2cl-lib:integer4) info ldwork m mm ldvr ldvl ldt n)
           (type (array f2cl-lib:logical (*)) select)
           (type (string *) howmny job))
  (f2cl-lib:with-multi-array-data
      ((job character job-%data% job-%offset%)
       (howmny character howmny-%data% howmny-%offset%)
       (select f2cl-lib:logical select-%data% select-%offset%)
       (t$ double-float t$-%data% t$-%offset%)
       (vl double-float vl-%data% vl-%offset%)
       (vr double-float vr-%data% vr-%offset%)
       (s double-float s-%data% s-%offset%)
       (sep double-float sep-%data% sep-%offset%)
       (work double-float work-%data% work-%offset%)
       (iwork f2cl-lib:integer4 iwork-%data% iwork-%offset%))
    (prog ((dummy (make-array 1 :element-type 'double-float))
           (isave (make-array 3 :element-type 'f2cl-lib:integer4))
           (bignum 0.0d0) (cond$ 0.0d0) (cs 0.0d0) (delta 0.0d0) (dumm 0.0d0)
           (eps 0.0d0) (est 0.0d0) (lnrm 0.0d0) (mu 0.0d0) (prod 0.0d0)
           (prod1 0.0d0) (prod2 0.0d0) (rnrm 0.0d0) (scale 0.0d0)
           (smlnum 0.0d0) (sn 0.0d0) (i 0) (ierr 0) (ifst 0) (ilst 0) (j 0)
           (k 0) (kase 0) (ks 0) (n2 0) (nn 0) (pair nil) (somcon nil)
           (wantbh nil) (wants nil) (wantsp nil))
      (declare (type (array double-float (1)) dummy)
               (type (array f2cl-lib:integer4 (3)) isave)
               (type (double-float) bignum cond$ cs delta dumm eps est lnrm
                mu prod prod1 prod2 rnrm scale smlnum sn)
               (type (f2cl-lib:integer4) i ierr ifst ilst j k kase ks n2 nn)
               (type f2cl-lib:logical pair somcon wantbh wants wantsp))
      (setf wantbh
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame job "B")
                (declare (ignore var-1))
                (when var-0 (setf job var-0))
                ret-val))
      (setf wants
              (or
               (multiple-value-bind (ret-val var-0 var-1)
                   (lsame job "E")
                 (declare (ignore var-1))
                 (when var-0 (setf job var-0))
                 ret-val)
               wantbh))
      (setf wantsp
              (or
               (multiple-value-bind (ret-val var-0 var-1)
```

```
                (lsame job "V")
              (declare (ignore var-1))
              (when var-0 (setf job var-0))
              ret-val)
            wantbh))
(setf somcon
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame howmny "S")
          (declare (ignore var-1))
          (when var-0 (setf howmny var-0))
          ret-val))
(setf info 0)
(cond ((and (not wants) (not wantsp)) (setf info -1))
      ((and
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame howmny "A")
           (declare (ignore var-1))
           (when var-0 (setf howmny var-0))
           ret-val))
        (not somcon))
       (setf info -2))
      ((< n 0) (setf info -4))
      ((< ldt
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -6))
      ((or (< ldvl 1) (and wants (< ldvl n))) (setf info -8))
      ((or (< ldvr 1) (and wants (< ldvr n))) (setf info -10))
      (t
       (cond
        (somcon (setf m 0) (setf pair f2cl-lib:%false%)
         (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
                       ((> k n) nil)
           (tagbody
             (cond (pair (setf pair f2cl-lib:%false%))
                   (t
                    (cond
                     ((< k n)
                      (cond
                       ((=
                         (f2cl-lib:fref t$ ((f2cl-lib:int-add k 1) k)
                                        ((1 ldt) (1 *)))
                         zero)
                        (if (f2cl-lib:fref select-%data% (k) ((1 *))
                                           select-%offset%)
                            (setf m (f2cl-lib:int-add m 1))))
                       (t (setf pair f2cl-lib:%true%)
                        (if (or
                             (f2cl-lib:fref select-%data% (k) ((1 *))
                                            select-%offset%)
```

```
                                (f2cl-lib:fref select-%data%
                                               ((f2cl-lib:int-add k 1))
                                               ((1 *)) select-%offset%))
                              (setf m (f2cl-lib:int-add m 2))))))
                        (t
                         (if (f2cl-lib:fref select-%data% (n) ((1 *))
                                            select-%offset%)
                             (setf m (f2cl-lib:int-add m 1)))))))
              label10)))
          (t (setf m n)))
         (cond ((< mm m) (setf info -13))
               ((or (< ldwork 1) (and wantsp (< ldwork n)))
                (setf info -16)))))
(cond
 ((/= info 0) (xerbla "DTRSNA" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(cond
 ((= n 1)
  (cond
   (somcon
    (if (not (f2cl-lib:fref select-%data% (1) ((1 *)) select-%offset%))
        (go end_label))))
  (if wants
      (setf (f2cl-lib:fref s-%data% (1) ((1 *)) s-%offset%) one))
  (if wantsp
      (setf (f2cl-lib:fref sep-%data% (1) ((1 *)) sep-%offset%)
              (abs
               (f2cl-lib:fref t$-%data% (1 1) ((1 ldt) (1 *))
                              t$-%offset%))))
  (go end_label)))
(setf eps (dlamch "P"))
(setf smlnum (/ (dlamch "S") eps))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf ks 0)
(setf pair f2cl-lib:%false%)
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (cond (pair (setf pair f2cl-lib:%false%) (go label60))
          (t
           (if (< k n)
               (setf pair
                       (coerce
```

```
                    (/=
                     (f2cl-lib:fref t$-%data%
                                    ((f2cl-lib:int-add k 1) k)
                                    ((1 ldt) (1 *)) t$-%offset%)
                     zero)
                    'f2cl-lib:logical)))))
(cond
 (somcon
  (cond
   (pair
    (if (and
         (not
          (f2cl-lib:fref select-%data% (k) ((1 *))
                         select-%offset%))
         (not
          (f2cl-lib:fref select-%data% ((f2cl-lib:int-add k 1))
                         ((1 *)) select-%offset%)))
        (go label60)))
   (t
    (if (not
         (f2cl-lib:fref select-%data% (k) ((1 *)) select-%offset%))
        (go label60))))))
(setf ks (f2cl-lib:int-add ks 1))
(cond
 (wants
  (cond
   ((not pair)
    (setf prod
            (multiple-value-bind
                (ret-val var-0 var-1 var-2 var-3 var-4)
                (ddot n
                 (f2cl-lib:array-slice vr-%data% double-float
                                       (1 ks) ((1 ldvr) (1 *))
                                       vr-%offset%)
                 1
                 (f2cl-lib:array-slice vl-%data% double-float
                                       (1 ks) ((1 ldvl) (1 *))
                                       vl-%offset%)
                 1)
              (declare (ignore var-1 var-2 var-3 var-4))
              (when var-0 (setf n var-0))
              ret-val))
    (setf rnrm
            (multiple-value-bind (ret-val var-0 var-1 var-2)
                (dnrm2 n
                 (f2cl-lib:array-slice vr-%data% double-float
                                       (1 ks) ((1 ldvr) (1 *))
                                       vr-%offset%)
                 1)
              (declare (ignore var-1 var-2))
```

```
          (when var-0 (setf n var-0))
          ret-val))
 (setf lnrm
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (dnrm2 n
             (f2cl-lib:array-slice vl-%data% double-float
                                   (1 ks) ((1 ldvl) (1 *))
                                   vl-%offset%)
             1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
 (setf (f2cl-lib:fref s-%data% (ks) ((1 *)) s-%offset%)
        (/ (abs prod) (* rnrm lnrm))))
(t
 (setf prod1
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot n
             (f2cl-lib:array-slice vr-%data% double-float
                                   (1 ks) ((1 ldvr) (1 *))
                                   vr-%offset%)
             1
             (f2cl-lib:array-slice vl-%data% double-float
                                   (1 ks) ((1 ldvl) (1 *))
                                   vl-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf n var-0))
          ret-val))
 (setf prod1
        (+ prod1
           (multiple-value-bind
               (ret-val var-0 var-1 var-2 var-3 var-4)
               (ddot n
                (f2cl-lib:array-slice vr-%data% double-float
                                      (1
                                       (f2cl-lib:int-add ks 1))
                                      ((1 ldvr) (1 *))
                                      vr-%offset%)
                1
                (f2cl-lib:array-slice vl-%data% double-float
                                      (1
                                       (f2cl-lib:int-add ks 1))
                                      ((1 ldvl) (1 *))
                                      vl-%offset%)
               1)
            (declare (ignore var-1 var-2 var-3 var-4))
            (when var-0 (setf n var-0))
            ret-val)))
```

```
(setf prod2
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4)
            (ddot n
             (f2cl-lib:array-slice vl-%data% double-float
                                   (1 ks) ((1 ldvl) (1 *))
                                   vl-%offset%)
             1
             (f2cl-lib:array-slice vr-%data% double-float
                                   (1 (f2cl-lib:int-add ks 1))
                                   ((1 ldvr) (1 *))
                                   vr-%offset%)
             1)
          (declare (ignore var-1 var-2 var-3 var-4))
          (when var-0 (setf n var-0))
          ret-val))
(setf prod2
        (- prod2
           (multiple-value-bind
               (ret-val var-0 var-1 var-2 var-3 var-4)
               (ddot n
                (f2cl-lib:array-slice vl-%data% double-float
                                      (1
                                       (f2cl-lib:int-add ks 1))
                                      ((1 ldvl) (1 *))
                                      vl-%offset%)
                1
                (f2cl-lib:array-slice vr-%data% double-float
                                      (1 ks) ((1 ldvr) (1 *))
                                      vr-%offset%)
                1)
             (declare (ignore var-1 var-2 var-3 var-4))
             (when var-0 (setf n var-0))
             ret-val)))
(setf rnrm
        (dlapy2
         (multiple-value-bind (ret-val var-0 var-1 var-2)
             (dnrm2 n
              (f2cl-lib:array-slice vr-%data% double-float
                                    (1 ks) ((1 ldvr) (1 *))
                                    vr-%offset%)
              1)
           (declare (ignore var-1 var-2))
           (when var-0 (setf n var-0))
           ret-val)
         (multiple-value-bind (ret-val var-0 var-1 var-2)
             (dnrm2 n
              (f2cl-lib:array-slice vr-%data% double-float
                                    (1 (f2cl-lib:int-add ks 1))
                                    ((1 ldvr) (1 *))
```

```
                                        vr-%offset%)
                    1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf n var-0))
                  ret-val)))
      (setf lnrm
              (dlapy2
               (multiple-value-bind (ret-val var-0 var-1 var-2)
                   (dnrm2 n
                    (f2cl-lib:array-slice vl-%data% double-float
                                          (1 ks) ((1 ldvl) (1 *))
                                          vl-%offset%)
                    1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf n var-0))
                  ret-val)
               (multiple-value-bind (ret-val var-0 var-1 var-2)
                   (dnrm2 n
                    (f2cl-lib:array-slice vl-%data% double-float
                                          (1 (f2cl-lib:int-add ks 1))
                                          ((1 ldvl) (1 *))
                                          vl-%offset%)
                    1)
                  (declare (ignore var-1 var-2))
                  (when var-0 (setf n var-0))
                  ret-val)))
      (setf cond$
              (/
               (multiple-value-bind (ret-val var-0 var-1)
                   (dlapy2 prod1 prod2)
                  (declare (ignore))
                  (when var-0 (setf prod1 var-0))
                  (when var-1 (setf prod2 var-1))
                  ret-val)
               (* rnrm lnrm)))
      (setf (f2cl-lib:fref s-%data% (ks) ((1 *)) s-%offset%) cond$)
      (setf (f2cl-lib:fref s-%data% ((f2cl-lib:int-add ks 1)) ((1 *))
                           s-%offset%)
              cond$)))))
  (cond
   (wantsp
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (dlacpy "Full" n n t$ ldt work ldwork)
      (declare (ignore var-0 var-3 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-4 (setf ldt var-4))
      (when var-6 (setf ldwork var-6)))
    (setf ifst k) (setf ilst 1)
    (multiple-value-bind
```

```
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
    (dtrexc "No Q" n work ldwork dummy 1 ifst ilst
     (f2cl-lib:array-slice work-%data% double-float
                           (1 (f2cl-lib:int-add n 1))
                           ((1 ldwork) (1 *)) work-%offset%)
     ierr)
  (declare (ignore var-0 var-2 var-4 var-5 var-8))
  (setf n var-1)
  (setf ldwork var-3)
  (setf ifst var-6)
  (setf ilst var-7)
  (setf ierr var-9))
(cond
 ((or (= ierr 1) (= ierr 2)) (setf scale one) (setf est bignum))
 (t
  (tagbody
    (cond
     ((= (f2cl-lib:fref work (2 1) ((1 ldwork) (1 *))) zero)
      (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i i)
                               ((1 ldwork) (1 *)) work-%offset%)
                  (-
                   (f2cl-lib:fref work-%data% (i i)
                                  ((1 ldwork) (1 *))
                                  work-%offset%)
                   (f2cl-lib:fref work-%data% (1 1)
                                  ((1 ldwork) (1 *))
                                  work-%offset%)))
          label20))
      (setf n2 1) (setf nn (f2cl-lib:int-sub n 1)))
     (t
      (setf mu
              (*
               (f2cl-lib:fsqrt
                (abs
                 (f2cl-lib:fref work-%data% (1 2)
                                ((1 ldwork) (1 *))
                                work-%offset%)))
               (f2cl-lib:fsqrt
                (abs
                 (f2cl-lib:fref work-%data% (2 1)
                                ((1 ldwork) (1 *))
                                work-%offset%)))))
      (setf delta
              (multiple-value-bind (ret-val var-0 var-1)
                  (dlapy2 mu
                   (f2cl-lib:array-slice work-%data% double-float
                                         (2 1) ((1 ldwork) (1 *))
```

```
                                    work-%offset%))
          (declare (ignore var-1))
          (when var-0 (setf mu var-0))
          ret-val))
(setf cs (/ mu delta))
(setf sn
        (/
         (-
          (f2cl-lib:fref work-%data% (2 1)
                         ((1 ldwork) (1 *)) work-%offset%))
         delta))
(f2cl-lib:fdo (j 3 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (2 j)
                         ((1 ldwork) (1 *)) work-%offset%)
            (* cs
               (f2cl-lib:fref work-%data% (2 j)
                              ((1 ldwork) (1 *))
                              work-%offset%)))
    (setf (f2cl-lib:fref work-%data% (j j)
                         ((1 ldwork) (1 *)) work-%offset%)
            (-
             (f2cl-lib:fref work-%data% (j j)
                            ((1 ldwork) (1 *))
                            work-%offset%)
             (f2cl-lib:fref work-%data% (1 1)
                            ((1 ldwork) (1 *))
                            work-%offset%)))
   label30))
(setf (f2cl-lib:fref work-%data% (2 2) ((1 ldwork) (1 *))
                     work-%offset%)
        zero)
(setf (f2cl-lib:fref work-%data% (1 (f2cl-lib:int-add n 1))
                     ((1 ldwork) (1 *)) work-%offset%)
        (* two mu))
(f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
              ((> i
                  (f2cl-lib:int-add n
                                    (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data%
                         (i (f2cl-lib:int-add n 1))
                         ((1 ldwork) (1 *)) work-%offset%)
            (* sn
               (f2cl-lib:fref work-%data%
                              (1 (f2cl-lib:int-add i 1))
                              ((1 ldwork) (1 *))
                              work-%offset%)))
```

```
      label40))
    (setf n2 2)
    (setf nn (f2cl-lib:int-mul 2 (f2cl-lib:int-sub n 1)))))
  (setf est zero)
  (setf kase 0)
 label50
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (dlacn2 nn
       (f2cl-lib:array-slice work-%data% double-float
                             (1 (f2cl-lib:int-add n 2))
                             ((1 ldwork) (1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% double-float
                             (1 (f2cl-lib:int-add n 4))
                             ((1 ldwork) (1 *)) work-%offset%)
       iwork est kase isave)
    (declare (ignore var-1 var-2 var-3 var-6))
    (when var-0 (setf nn var-0))
    (when var-4 (setf est var-4))
    (when var-5 (setf kase var-5)))
  (cond
   ((/= kase 0)
    (cond
     ((= kase 1)
      (cond
       ((= n2 1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
             var-8 var-9 var-10)
            (dlaqtr f2cl-lib:%true% f2cl-lib:%true%
             (f2cl-lib:int-sub n 1)
             (f2cl-lib:array-slice work-%data% double-float
                                   (2 2) ((1 ldwork) (1 *))
                                   work-%offset%)
             ldwork dummy dumm scale
             (f2cl-lib:array-slice work-%data% double-float
                                   (1 (f2cl-lib:int-add n 4))
                                   ((1 ldwork) (1 *))
                                   work-%offset%)
             (f2cl-lib:array-slice work-%data% double-float
                                   (1 (f2cl-lib:int-add n 6))
                                   ((1 ldwork) (1 *))
                                   work-%offset%)
             ierr)
          (declare
           (ignore var-0 var-1 var-2 var-3 var-5 var-8 var-9))
          (when var-4 (setf ldwork var-4))
          (when var-6 (setf dumm var-6))
          (when var-7 (setf scale var-7))
          (when var-10 (setf ierr var-10))))
```

```
    (t
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
          var-8 var-9 var-10)
         (dlaqtr f2cl-lib:%true% f2cl-lib:%false%
          (f2cl-lib:int-sub n 1)
          (f2cl-lib:array-slice work-%data% double-float
                                (2 2) ((1 ldwork) (1 *))
                                work-%offset%)
          ldwork
          (f2cl-lib:array-slice work-%data% double-float
                                (1 (f2cl-lib:int-add n 1))
                                ((1 ldwork) (1 *))
                                work-%offset%)
          mu scale
          (f2cl-lib:array-slice work-%data% double-float
                                (1 (f2cl-lib:int-add n 4))
                                ((1 ldwork) (1 *))
                                work-%offset%)
          (f2cl-lib:array-slice work-%data% double-float
                                (1 (f2cl-lib:int-add n 6))
                                ((1 ldwork) (1 *))
                                work-%offset%)
          ierr)
       (declare
        (ignore var-0 var-1 var-2 var-3 var-5 var-8 var-9))
       (when var-4 (setf ldwork var-4))
       (when var-6 (setf mu var-6))
       (when var-7 (setf scale var-7))
       (when var-10 (setf ierr var-10))))))
(t
 (cond
  ((= n2 1)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
        var-8 var-9 var-10)
       (dlaqtr f2cl-lib:%false% f2cl-lib:%true%
        (f2cl-lib:int-sub n 1)
        (f2cl-lib:array-slice work-%data% double-float
                              (2 2) ((1 ldwork) (1 *))
                              work-%offset%)
        ldwork dummy dumm scale
        (f2cl-lib:array-slice work-%data% double-float
                              (1 (f2cl-lib:int-add n 4))
                              ((1 ldwork) (1 *))
                              work-%offset%)
        (f2cl-lib:array-slice work-%data% double-float
                              (1 (f2cl-lib:int-add n 6))
                              ((1 ldwork) (1 *))
                              work-%offset%)
```

```
                        ierr)
                      (declare
                       (ignore var-0 var-1 var-2 var-3 var-5 var-8 var-9))
                      (when var-4 (setf ldwork var-4))
                      (when var-6 (setf dumm var-6))
                      (when var-7 (setf scale var-7))
                      (when var-10 (setf ierr var-10))))
                    (t
                     (multiple-value-bind
                         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
                          var-8 var-9 var-10)
                         (dlaqtr f2cl-lib:%false% f2cl-lib:%false%
                          (f2cl-lib:int-sub n 1)
                          (f2cl-lib:array-slice work-%data% double-float
                                                (2 2) ((1 ldwork) (1 *))
                                                work-%offset%)
                          ldwork
                          (f2cl-lib:array-slice work-%data% double-float
                                                (1 (f2cl-lib:int-add n 1))
                                                ((1 ldwork) (1 *))
                                                work-%offset%)
                          mu scale
                          (f2cl-lib:array-slice work-%data% double-float
                                                (1 (f2cl-lib:int-add n 4))
                                                ((1 ldwork) (1 *))
                                                work-%offset%)
                          (f2cl-lib:array-slice work-%data% double-float
                                                (1 (f2cl-lib:int-add n 6))
                                                ((1 ldwork) (1 *))
                                                work-%offset%)
                          ierr)
                      (declare
                       (ignore var-0 var-1 var-2 var-3 var-5 var-8 var-9))
                      (when var-4 (setf ldwork var-4))
                      (when var-6 (setf mu var-6))
                      (when var-7 (setf scale var-7))
                      (when var-10 (setf ierr var-10)))))))
                (go label50))))))
        (setf (f2cl-lib:fref sep-%data% (ks) ((1 *)) sep-%offset%)
                (/ scale (max est smlnum)))
        (if pair
            (setf (f2cl-lib:fref sep-%data% ((f2cl-lib:int-add ks 1))
                                 ((1 *)) sep-%offset%)
                  (f2cl-lib:fref sep-%data% (ks) ((1 *))
                                 sep-%offset%)))))
     (if pair
         (setf ks (f2cl-lib:int-add ks 1)))
    label60))
 (go end_label)
end_label
```

```
        (return
         (values job howmny nil n nil ldt nil nil nil nil nil nil nil m nil
                 ldwork nil info))))))
```

———-

90.208 ieeeck

90.208.1 Axiom unit tests

— **ieeeck.input** —

```
)set break resume
)sys rm -f ieeeck.output
)spool ieeeck.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.208.2 Axiom help page

— **ieeeck.help** —

```
======================================================================
ieeeck examples
======================================================================

======================================================================
Man Page Details
======================================================================

NAME
       IEEECK  -  called  from the ILAENV to verify that Infinity and possibly
       NaN arithmetic is safe (i.e

SYNOPSIS
       INTEGER FUNCTION IEEECK( ISPEC, ZERO, ONE )
```

```
          INTEGER      ISPEC

          REAL         ONE, ZERO

  Purpose
  =======

  IEEECK is called from the ILAENV to verify that Infinity and
  possibly NaN arithmetic is safe (i.e. will not trap).

  Arguments
  =========

  ISPEC   (input) INTEGER
          Specifies whether to test just for inifinity arithmetic
          or whether to test for infinity and NaN arithmetic.
          = 0: Verify infinity arithmetic only.
          = 1: Verify infinity and NaN arithmetic.

  ZERO    (input) REAL
          Must contain the value 0.0
          This is passed to prevent the compiler from optimizing
          away this code.

  ONE     (input) REAL
          Must contain the value 1.0
          This is passed to prevent the compiler from optimizing
          away this code.

  RETURN VALUE:  INTEGER
          = 0:  Arithmetic failed to produce the correct answers
          = 1:  Arithmetic produced the correct answers
```

90.208.3 fortran code

— ieeeck.f —

```
      INTEGER          FUNCTION IEEECK( ISPEC, ZERO, ONE )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1998
*
*     .. Scalar Arguments ..
```

```
      INTEGER            ISPEC
      REAL               ONE, ZERO
*     ..
*
*     .. Local Scalars ..
      REAL               NAN1, NAN2, NAN3, NAN4, NAN5, NAN6, NEGINF,
     $                   NEGZRO, NEWZRO, POSINF
*     ..
*     .. Executable Statements ..
      IEEECK = 1
*
      POSINF = ONE / ZERO
      IF( POSINF.LE.ONE ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      NEGINF = -ONE / ZERO
      IF( NEGINF.GE.ZERO ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      NEGZRO = ONE / ( NEGINF+ONE )
      IF( NEGZRO.NE.ZERO ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      NEGINF = ONE / NEGZRO
      IF( NEGINF.GE.ZERO ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      NEWZRO = NEGZRO + ZERO
      IF( NEWZRO.NE.ZERO ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      POSINF = ONE / NEWZRO
      IF( POSINF.LE.ONE ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      NEGINF = NEGINF*POSINF
      IF( NEGINF.GE.ZERO ) THEN
         IEEECK = 0
```

```
         RETURN
      END IF
*
      POSINF = POSINF*POSINF
      IF( POSINF.LE.ONE ) THEN
         IEEECK = 0
         RETURN
      END IF
*
*
*
*
*     Return if we were only asked to check infinity arithmetic
*
      IF( ISPEC.EQ.0 )
     $   RETURN
*
      NAN1 = POSINF + NEGINF
*
      NAN2 = POSINF / NEGINF
*
      NAN3 = POSINF / POSINF
*
      NAN4 = POSINF*ZERO
*
      NAN5 = NEGINF*NEGZRO
*
      NAN6 = NAN5*0.0
*
      IF( NAN1.EQ.NAN1 ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      IF( NAN2.EQ.NAN2 ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      IF( NAN3.EQ.NAN3 ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      IF( NAN4.EQ.NAN4 ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      IF( NAN5.EQ.NAN5 ) THEN
```

```
         IEEECK = 0
         RETURN
      END IF
*
      IF( NAN6.EQ.NAN6 ) THEN
         IEEECK = 0
         RETURN
      END IF
*
      RETURN
      END
```

———-

90.208.4 lisp code

— **ieeeck.lisp** —

```
(defun ieeeck (ispec zero one)
  (declare (type (single-float) one zero)
           (type (f2cl-lib:integer4) ispec))
  (prog ((nan1 0.0) (nan2 0.0) (nan3 0.0) (nan4 0.0) (nan5 0.0) (nan6 0.0)
         (neginf 0.0) (negzro 0.0) (newzro 0.0) (posinf 0.0) (ieeeck 0))
    (declare (type (f2cl-lib:integer4) ieeeck)
             (type (single-float) posinf newzro negzro neginf nan6 nan5 nan4
              nan3 nan2 nan1))
    (setf ieeeck 1)
    (setf posinf (/ one zero))
    (cond ((<= posinf one) (setf ieeeck 0) (go end_label)))
    (setf neginf (/ (- one) zero))
    (cond ((>= neginf zero) (setf ieeeck 0) (go end_label)))
    (setf negzro (/ one (+ neginf one)))
    (cond ((/= negzro zero) (setf ieeeck 0) (go end_label)))
    (setf neginf (/ one negzro))
    (cond ((>= neginf zero) (setf ieeeck 0) (go end_label)))
    (setf newzro (+ negzro zero))
    (cond ((/= newzro zero) (setf ieeeck 0) (go end_label)))
    (setf posinf (/ one newzro))
    (cond ((<= posinf one) (setf ieeeck 0) (go end_label)))
    (setf neginf (* neginf posinf))
    (cond ((>= neginf zero) (setf ieeeck 0) (go end_label)))
    (setf posinf (* posinf posinf))
    (cond ((<= posinf one) (setf ieeeck 0) (go end_label)))
    (if (= ispec 0)
        (go end_label))
    (setf nan1 (+ posinf neginf))
    (setf nan2 (/ posinf neginf))
```

```
    (setf nan3 (/ posinf posinf))
    (setf nan4 (* posinf zero))
    (setf nan5 (* neginf negzro))
    (setf nan6 (* nan5 zero))
    (cond ((= nan1 nan1) (setf ieeeck 0) (go end_label)))
    (cond ((= nan2 nan2) (setf ieeeck 0) (go end_label)))
    (cond ((= nan3 nan3) (setf ieeeck 0) (go end_label)))
    (cond ((= nan4 nan4) (setf ieeeck 0) (go end_label)))
    (cond ((= nan5 nan5) (setf ieeeck 0) (go end_label)))
    (cond ((= nan6 nan6) (setf ieeeck 0) (go end_label)))
    (go end_label)
   end_label
    (return (values ieeeck nil nil nil))))
```

———-

90.209 ilaenv

90.209.1 Axiom unit tests

— **ilaenv.input** —

```
)set break resume
)sys rm -f ilaenv.output
)spool ilaenv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.209.2 Axiom help page

— **ilaenv.help** —

```
=====================================================================
ilaenv examples
=====================================================================

=====================================================================
```

```
Man Page Details
=====================================================================

NAME
       ILAENV  -  called  from the LAPACK routines to choose problem-dependent
       parameters for the local environment

SYNOPSIS
       INTEGER FUNCTION ILAENV( ISPEC, NAME, OPTS, N1, N2, N3, N4 )

           CHARACTER*(  * ) NAME, OPTS

           INTEGER      ISPEC, N1, N2, N3, N4

  Purpose
  =======

  ILAENV is called from the LAPACK routines to choose problem-dependent
  parameters for the local environment.  See ISPEC for a description of
  the parameters.

  This version provides a set of parameters which should give good,
  but not optimal, performance on many of the currently available
  computers.  Users are encouraged to modify this subroutine to set
  the tuning parameters for their particular machine using the option
  and problem size information in the arguments.

  This routine will not function correctly if it is converted to all
  lower case.  Converting it to all upper case is allowed.

  Arguments
  =========

  ISPEC   (input) INTEGER
          Specifies the parameter to be returned as the value of
          ILAENV.
          = 1: the optimal blocksize; if this value is 1, an unblocked
               algorithm will give the best performance.
          = 2: the minimum block size for which the block routine
               should be used; if the usable block size is less than
               this value, an unblocked routine should be used.
          = 3: the crossover point (in a block routine, for N less
               than this value, an unblocked routine should be used)
          = 4: the number of shifts, used in the nonsymmetric
               eigenvalue routines
          = 5: the minimum column dimension for blocking to be used;
               rectangular blocks must have dimension at least k by m,
               where k is given by ILAENV(2,...) and m by ILAENV(5,...)
          = 6: the crossover point for the SVD (when reducing an m by n
               matrix to bidiagonal form, if max(m,n)/min(m,n) exceeds
```

```
             this value, a QR factorization is used first to reduce
             the matrix to a triangular form.)
        = 7: the number of processors
        = 8: the crossover point for the multishift QR and QZ methods
             for nonsymmetric eigenvalue problems.
        = 9: maximum size of the subproblems at the bottom of the
             computation tree in the divide-and-conquer algorithm
             (used by xGELSD and xGESDD)
        =10: ieee NaN arithmetic can be trusted not to trap
        =11: infinity arithmetic can be trusted not to trap

 NAME    (input) CHARACTER*(*)
         The name of the calling subroutine, in either upper case or
         lower case.

 OPTS    (input) CHARACTER*(*)
         The character options to the subroutine NAME, concatenated
         into a single character string.  For example, UPLO = 'U',
         TRANS = 'T', and DIAG = 'N' for a triangular routine would
         be specified as OPTS = 'UTN'.

 N1      (input) INTEGER
 N2      (input) INTEGER
 N3      (input) INTEGER
 N4      (input) INTEGER
         Problem dimensions for the subroutine NAME; these may not all
         be required.

(ILAENV) (output) INTEGER
         >= 0: the value of the parameter specified by ISPEC
         < 0:  if ILAENV = -k, the k-th argument had an illegal value.

 Further Details
 ===============

 The following conventions have been used when calling ILAENV from the
 LAPACK routines:
 1)  OPTS is a concatenation of all of the character options to
     subroutine NAME, in the same order that they appear in the
     argument list for NAME, even if they are not used in determining
     the value of the parameter specified by ISPEC.
 2)  The problem dimensions N1, N2, N3, N4 are specified in the order
     that they appear in the argument list for NAME.  N1 is used
     first, N2 second, and so on, and unused problem dimensions are
     passed a value of -1.
 3)  The parameter value returned by ILAENV is checked for validity in
     the calling subroutine.  For example, ILAENV is used to retrieve
     the optimal blocksize for STRTRI as follows:

     NB = ILAENV( 1, 'STRTRI', UPLO // DIAG, N, -1, -1, -1 )
```

```
      IF( NB.LE.1 ) NB = MAX( 1, N )
```

———-

90.209.3 fortran code

— **ilaenv.f** —

```
      INTEGER          FUNCTION ILAENV( ISPEC, NAME, OPTS, N1, N2, N3,
     $                 N4 )
*
*  -- LAPACK auxiliary routine (version 3.0) --
*     Univ. of Tennessee, Univ. of California Berkeley, NAG Ltd.,
*     Courant Institute, Argonne National Lab, and Rice University
*     June 30, 1999
*
*     .. Scalar Arguments ..
      CHARACTER*( * )    NAME, OPTS
      INTEGER            ISPEC, N1, N2, N3, N4
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            CNAME, SNAME
      CHARACTER*1        C1
      CHARACTER*2        C2, C4
      CHARACTER*3        C3
      CHARACTER*6        SUBNAM
      INTEGER            I, IC, IZ, NB, NBMIN, NX
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          CHAR, ICHAR, INT, MIN, REAL
*     ..
*     .. External Functions ..
      INTEGER            IEEECK
      EXTERNAL           IEEECK
*     ..
*     .. Executable Statements ..
*
      GO TO ( 100, 100, 100, 400, 500, 600, 700, 800, 900, 1000,
     $        1100 ) ISPEC
*
*     Invalid value for ISPEC
*
      ILAENV = -1
      RETURN
```

```
*
  100 CONTINUE
*
*     Convert NAME to upper case if the first character is lower case.
*
      ILAENV = 1
      SUBNAM = NAME
      IC = ICHAR( SUBNAM( 1:1 ) )
      IZ = ICHAR( 'Z' )
      IF( IZ.EQ.90 .OR. IZ.EQ.122 ) THEN
*
*        ASCII character set
*
         IF( IC.GE.97 .AND. IC.LE.122 ) THEN
            SUBNAM( 1:1 ) = CHAR( IC-32 )
            DO 10 I = 2, 6
               IC = ICHAR( SUBNAM( I:I ) )
               IF( IC.GE.97 .AND. IC.LE.122 )
     $            SUBNAM( I:I ) = CHAR( IC-32 )
   10       CONTINUE
         END IF
*
      ELSE IF( IZ.EQ.233 .OR. IZ.EQ.169 ) THEN
*
*        EBCDIC character set
*
         IF( ( IC.GE.129 .AND. IC.LE.137 ) .OR.
     $       ( IC.GE.145 .AND. IC.LE.153 ) .OR.
     $       ( IC.GE.162 .AND. IC.LE.169 ) ) THEN
            SUBNAM( 1:1 ) = CHAR( IC+64 )
            DO 20 I = 2, 6
               IC = ICHAR( SUBNAM( I:I ) )
               IF( ( IC.GE.129 .AND. IC.LE.137 ) .OR.
     $             ( IC.GE.145 .AND. IC.LE.153 ) .OR.
     $             ( IC.GE.162 .AND. IC.LE.169 ) )
     $            SUBNAM( I:I ) = CHAR( IC+64 )
   20       CONTINUE
         END IF
*
      ELSE IF( IZ.EQ.218 .OR. IZ.EQ.250 ) THEN
*
*        Prime machines:  ASCII+128
*
         IF( IC.GE.225 .AND. IC.LE.250 ) THEN
            SUBNAM( 1:1 ) = CHAR( IC-32 )
            DO 30 I = 2, 6
               IC = ICHAR( SUBNAM( I:I ) )
               IF( IC.GE.225 .AND. IC.LE.250 )
     $            SUBNAM( I:I ) = CHAR( IC-32 )
   30       CONTINUE
```

```
         END IF
      END IF
*
      C1 = SUBNAM( 1:1 )
      SNAME = C1.EQ.'S' .OR. C1.EQ.'D'
      CNAME = C1.EQ.'C' .OR. C1.EQ.'Z'
      IF( .NOT.( CNAME .OR. SNAME ) )
     $   RETURN
      C2 = SUBNAM( 2:3 )
      C3 = SUBNAM( 4:6 )
      C4 = C3( 2:3 )
*
      GO TO ( 110, 200, 300 ) ISPEC
*
  110 CONTINUE
*
*     ISPEC = 1:  block size
*
*     In these examples, separate code is provided for setting NB for
*     real and complex.  We assume that NB will take the same value in
*     single or double precision.
*
      NB = 1
*
      IF( C2.EQ.'GE' ) THEN
         IF( C3.EQ.'TRF' ) THEN
            IF( SNAME ) THEN
               NB = 64
            ELSE
               NB = 64
            END IF
         ELSE IF( C3.EQ.'QRF' .OR. C3.EQ.'RQF' .OR. C3.EQ.'LQF' .OR.
     $           C3.EQ.'QLF' ) THEN
            IF( SNAME ) THEN
               NB = 32
            ELSE
               NB = 32
            END IF
         ELSE IF( C3.EQ.'HRD' ) THEN
            IF( SNAME ) THEN
               NB = 32
            ELSE
               NB = 32
            END IF
         ELSE IF( C3.EQ.'BRD' ) THEN
            IF( SNAME ) THEN
               NB = 32
            ELSE
               NB = 32
            END IF
```

```
      ELSE IF( C3.EQ.'TRI' ) THEN
         IF( SNAME ) THEN
            NB = 64
         ELSE
            NB = 64
         END IF
      END IF
   ELSE IF( C2.EQ.'PO' ) THEN
      IF( C3.EQ.'TRF' ) THEN
         IF( SNAME ) THEN
            NB = 64
         ELSE
            NB = 64
         END IF
      END IF
   ELSE IF( C2.EQ.'SY' ) THEN
      IF( C3.EQ.'TRF' ) THEN
         IF( SNAME ) THEN
            NB = 64
         ELSE
            NB = 64
         END IF
      ELSE IF( SNAME .AND. C3.EQ.'TRD' ) THEN
         NB = 32
      ELSE IF( SNAME .AND. C3.EQ.'GST' ) THEN
         NB = 64
      END IF
   ELSE IF( CNAME .AND. C2.EQ.'HE' ) THEN
      IF( C3.EQ.'TRF' ) THEN
         NB = 64
      ELSE IF( C3.EQ.'TRD' ) THEN
         NB = 32
      ELSE IF( C3.EQ.'GST' ) THEN
         NB = 64
      END IF
   ELSE IF( SNAME .AND. C2.EQ.'OR' ) THEN
      IF( C3( 1:1 ).EQ.'G' ) THEN
         IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR.
$           C4.EQ.'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR.
$           C4.EQ.'BR' ) THEN
            NB = 32
         END IF
      ELSE IF( C3( 1:1 ).EQ.'M' ) THEN
         IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR.
$           C4.EQ.'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR.
$           C4.EQ.'BR' ) THEN
            NB = 32
         END IF
      END IF
   ELSE IF( CNAME .AND. C2.EQ.'UN' ) THEN
```

```
      IF( C3( 1:1 ).EQ.'G' ) THEN
         IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR.
     $       C4.EQ.'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR.
     $       C4.EQ.'BR' ) THEN
            NB = 32
         END IF
      ELSE IF( C3( 1:1 ).EQ.'M' ) THEN
         IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR.
     $       C4.EQ.'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR.
     $       C4.EQ.'BR' ) THEN
            NB = 32
         END IF
      END IF
   ELSE IF( C2.EQ.'GB' ) THEN
      IF( C3.EQ.'TRF' ) THEN
         IF( SNAME ) THEN
            IF( N4.LE.64 ) THEN
               NB = 1
            ELSE
               NB = 32
            END IF
         ELSE
            IF( N4.LE.64 ) THEN
               NB = 1
            ELSE
               NB = 32
            END IF
         END IF
      END IF
   ELSE IF( C2.EQ.'PB' ) THEN
      IF( C3.EQ.'TRF' ) THEN
         IF( SNAME ) THEN
            IF( N2.LE.64 ) THEN
               NB = 1
            ELSE
               NB = 32
            END IF
         ELSE
            IF( N2.LE.64 ) THEN
               NB = 1
            ELSE
               NB = 32
            END IF
         END IF
      END IF
   ELSE IF( C2.EQ.'TR' ) THEN
      IF( C3.EQ.'TRI' ) THEN
         IF( SNAME ) THEN
            NB = 64
         ELSE
```

```
               NB = 64
            END IF
         END IF
      ELSE IF( C2.EQ.'LA' ) THEN
         IF( C3.EQ.'UUM' ) THEN
            IF( SNAME ) THEN
               NB = 64
            ELSE
               NB = 64
            END IF
         END IF
      ELSE IF( SNAME .AND. C2.EQ.'ST' ) THEN
         IF( C3.EQ.'EBZ' ) THEN
            NB = 1
         END IF
      END IF
      ILAENV = NB
      RETURN
*
  200 CONTINUE
*
*     ISPEC = 2:  minimum block size
*
      NBMIN = 2
      IF( C2.EQ.'GE' ) THEN
         IF( C3.EQ.'QRF' .OR. C3.EQ.'RQF' .OR. C3.EQ.'LQF' .OR.
     $       C3.EQ.'QLF' ) THEN
            IF( SNAME ) THEN
               NBMIN = 2
            ELSE
               NBMIN = 2
            END IF
         ELSE IF( C3.EQ.'HRD' ) THEN
            IF( SNAME ) THEN
               NBMIN = 2
            ELSE
               NBMIN = 2
            END IF
         ELSE IF( C3.EQ.'BRD' ) THEN
            IF( SNAME ) THEN
               NBMIN = 2
            ELSE
               NBMIN = 2
            END IF
         ELSE IF( C3.EQ.'TRI' ) THEN
            IF( SNAME ) THEN
               NBMIN = 2
            ELSE
               NBMIN = 2
            END IF
```

```
         END IF
      ELSE IF( C2.EQ.'SY' ) THEN
         IF( C3.EQ.'TRF' ) THEN
            IF( SNAME ) THEN
               NBMIN = 8
            ELSE
               NBMIN = 8
            END IF
         ELSE IF( SNAME .AND. C3.EQ.'TRD' ) THEN
            NBMIN = 2
         END IF
      ELSE IF( CNAME .AND. C2.EQ.'HE' ) THEN
         IF( C3.EQ.'TRD' ) THEN
            NBMIN = 2
         END IF
      ELSE IF( SNAME .AND. C2.EQ.'OR' ) THEN
         IF( C3( 1:1 ).EQ.'G' ) THEN
            IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR.
     $          C4.EQ.'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR.
     $          C4.EQ.'BR' ) THEN
               NBMIN = 2
            END IF
         ELSE IF( C3( 1:1 ).EQ.'M' ) THEN
            IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR.
     $          C4.EQ.'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR.
     $          C4.EQ.'BR' ) THEN
               NBMIN = 2
            END IF
         END IF
      ELSE IF( CNAME .AND. C2.EQ.'UN' ) THEN
         IF( C3( 1:1 ).EQ.'G' ) THEN
            IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR.
     $          C4.EQ.'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR.
     $          C4.EQ.'BR' ) THEN
               NBMIN = 2
            END IF
         ELSE IF( C3( 1:1 ).EQ.'M' ) THEN
            IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR.
     $          C4.EQ.'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR.
     $          C4.EQ.'BR' ) THEN
               NBMIN = 2
            END IF
         END IF
      END IF
      ILAENV = NBMIN
      RETURN
*
  300 CONTINUE
*
*     ISPEC = 3:  crossover point
```

```
*
      NX = 0
      IF( C2.EQ.'GE' ) THEN
         IF( C3.EQ.'QRF' .OR. C3.EQ.'RQF' .OR. C3.EQ.'LQF' .OR.
     $       C3.EQ.'QLF' ) THEN
            IF( SNAME ) THEN
               NX = 128
            ELSE
               NX = 128
            END IF
         ELSE IF( C3.EQ.'HRD' ) THEN
            IF( SNAME ) THEN
               NX = 128
            ELSE
               NX = 128
            END IF
         ELSE IF( C3.EQ.'BRD' ) THEN
            IF( SNAME ) THEN
               NX = 128
            ELSE
               NX = 128
            END IF
         END IF
      ELSE IF( C2.EQ.'SY' ) THEN
         IF( SNAME .AND. C3.EQ.'TRD' ) THEN
            NX = 32
         END IF
      ELSE IF( CNAME .AND. C2.EQ.'HE' ) THEN
         IF( C3.EQ.'TRD' ) THEN
            NX = 32
         END IF
      ELSE IF( SNAME .AND. C2.EQ.'OR' ) THEN
         IF( C3( 1:1 ).EQ.'G' ) THEN
            IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR.
     $          C4.EQ.'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR.
     $          C4.EQ.'BR' ) THEN
               NX = 128
            END IF
         END IF
      ELSE IF( CNAME .AND. C2.EQ.'UN' ) THEN
         IF( C3( 1:1 ).EQ.'G' ) THEN
            IF( C4.EQ.'QR' .OR. C4.EQ.'RQ' .OR. C4.EQ.'LQ' .OR.
     $          C4.EQ.'QL' .OR. C4.EQ.'HR' .OR. C4.EQ.'TR' .OR.
     $          C4.EQ.'BR' ) THEN
               NX = 128
            END IF
         END IF
      END IF
      ILAENV = NX
      RETURN
```

```
*
  400 CONTINUE
*
*     ISPEC = 4:  number of shifts (used by xHSEQR)
*
      ILAENV = 6
      RETURN
*
  500 CONTINUE
*
*     ISPEC = 5:  minimum column dimension (not used)
*
      ILAENV = 2
      RETURN
*
  600 CONTINUE
*
*     ISPEC = 6:  crossover point for SVD (used by xGELSS and xGESVD)
*
      ILAENV = INT( REAL( MIN( N1, N2 ) )*1.6E0 )
      RETURN
*
  700 CONTINUE
*
*     ISPEC = 7:  number of processors (not used)
*
      ILAENV = 1
      RETURN
*
  800 CONTINUE
*
*     ISPEC = 8:  crossover point for multishift (used by xHSEQR)
*
      ILAENV = 50
      RETURN
*
  900 CONTINUE
*
*     ISPEC = 9:  maximum size of the subproblems at the bottom of the
*                 computation tree in the divide-and-conquer algorithm
*                 (used by xGELSD and xGESDD)
*
      ILAENV = 25
      RETURN
*
 1000 CONTINUE
*
*     ISPEC = 10: ieee NaN arithmetic can be trusted not to trap
*
C     ILAENV = 0
```

```
      ILAENV = 1
      IF( ILAENV.EQ.1 ) THEN
         ILAENV = IEEECK( 0, 0.0, 1.0 )
      END IF
      RETURN
*
 1100 CONTINUE
*
*     ISPEC = 11: infinity arithmetic can be trusted not to trap
*
C     ILAENV = 0
      ILAENV = 1
      IF( ILAENV.EQ.1 ) THEN
         ILAENV = IEEECK( 1, 0.0, 1.0 )
      END IF
      RETURN
*
*     End of ILAENV
*
      END
```

———-

90.209.4 lisp code

— ilaenv.lisp —

```
(defun ilaenv (ispec name opts n1 n2 n3 n4)
  (declare (type (string *) opts name)
           (type (f2cl-lib:integer4) n4 n3 n2 n1 ispec))
  (f2cl-lib:with-multi-array-data
      ((name character name-%data% name-%offset%)
       (opts character opts-%data% opts-%offset%))
    (prog ((c1 (make-array '(1) :element-type 'character :initial-element #\ ))
           (c2 (make-array '(2) :element-type 'character :initial-element #\ ))
           (c4 (make-array '(2) :element-type 'character :initial-element #\ ))
           (c3 (make-array '(3) :element-type 'character :initial-element #\ ))
           (subnam
            (make-array '(6) :element-type 'character :initial-element #\ ))
           (cname nil) (sname nil) (i 0) (ic 0) (iz 0) (nb 0) (nbmin 0) (nx 0)
           (ilaenv 0))
      (declare (type (f2cl-lib:integer4) ilaenv nx nbmin nb iz ic i)
               (type f2cl-lib:logical sname cname)
               (type (string 6) subnam)
               (type (string 3) c3)
               (type (string 2) c4 c2)
               (type (string 1) c1))
```

```
  (f2cl-lib:computed-goto
   (label10 label10 label10 label80 label90 label100 label110 label120
    label130 label140 label150 label160 label160 label160 label160
    label160)
   ispec)
  (setf ilaenv -1)
  (go end_label)
 label10
  (setf ilaenv 1)
  (f2cl-lib:f2cl-set-string subnam name (string 6))
  (setf ic (f2cl-lib:ichar (f2cl-lib:fref-string subnam (1 1))))
  (setf iz (f2cl-lib:ichar "Z"))
  (cond
   ((or (= iz 90) (= iz 122))
    (cond
     ((and (>= ic 97) (<= ic 122))
      (f2cl-lib:fset-string (f2cl-lib:fref-string subnam (1 1))
                            (f2cl-lib:fchar (f2cl-lib:int-sub ic 32)))
      (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                    ((> i 6) nil)
        (tagbody
          (setf ic (f2cl-lib:ichar (f2cl-lib:fref-string subnam (i i))))
          (if (and (>= ic 97) (<= ic 122))
              (f2cl-lib:fset-string (f2cl-lib:fref-string subnam (i i))
                                    (f2cl-lib:fchar
                                     (f2cl-lib:int-sub ic 32))))
        label20)))))
   ((or (= iz 233) (= iz 169))
    (cond
     ((or (and (>= ic 129) (<= ic 137)) (and (>= ic 145) (<= ic 153))
          (and (>= ic 162) (<= ic 169)))
      (f2cl-lib:fset-string (f2cl-lib:fref-string subnam (1 1))
                            (f2cl-lib:fchar (f2cl-lib:int-add ic 64)))
      (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
                    ((> i 6) nil)
        (tagbody
          (setf ic (f2cl-lib:ichar (f2cl-lib:fref-string subnam (i i))))
          (if (or (and (>= ic 129) (<= ic 137))
                  (and (>= ic 145) (<= ic 153))
                  (and (>= ic 162) (<= ic 169)))
              (f2cl-lib:fset-string (f2cl-lib:fref-string subnam (i i))
                                    (f2cl-lib:fchar
                                     (f2cl-lib:int-add ic 64))))
        label30)))))
   ((or (= iz 218) (= iz 250))
    (cond
     ((and (>= ic 225) (<= ic 250))
      (f2cl-lib:fset-string (f2cl-lib:fref-string subnam (1 1))
                            (f2cl-lib:fchar (f2cl-lib:int-sub ic 32)))
      (f2cl-lib:fdo (i 2 (f2cl-lib:int-add i 1))
```

```
                    ((> i 6) nil)
        (tagbody
          (setf ic (f2cl-lib:ichar (f2cl-lib:fref-string subnam (i i))))
          (if (and (>= ic 225) (<= ic 250))
              (f2cl-lib:fset-string (f2cl-lib:fref-string subnam (i i))
                                    (f2cl-lib:fchar
                                     (f2cl-lib:int-sub ic 32))))
         label40))))))
  (f2cl-lib:f2cl-set-string c1 (f2cl-lib:fref-string subnam (1 1))
                            (string 1))
  (setf sname (or (f2cl-lib:fstring-= c1 "S") (f2cl-lib:fstring-= c1 "D")))
  (setf cname (or (f2cl-lib:fstring-= c1 "C") (f2cl-lib:fstring-= c1 "Z")))
  (if (not (or cname sname))
      (go end_label))
  (f2cl-lib:f2cl-set-string c2 (f2cl-lib:fref-string subnam (2 3))
                            (string 2))
  (f2cl-lib:f2cl-set-string c3 (f2cl-lib:fref-string subnam (4 6))
                            (string 3))
  (f2cl-lib:f2cl-set-string c4 (f2cl-lib:fref-string c3 (2 3)) (string 2))
  (f2cl-lib:computed-goto (label50 label60 label70) ispec)
 label50
  (setf nb 1)
  (cond
   ((f2cl-lib:fstring-= c2 "GE")
    (cond
     ((f2cl-lib:fstring-= c3 "TRF")
      (cond (sname (setf nb 64)) (t (setf nb 64))))
     ((or (f2cl-lib:fstring-= c3 "QRF") (f2cl-lib:fstring-= c3 "RQF")
          (f2cl-lib:fstring-= c3 "LQF") (f2cl-lib:fstring-= c3 "QLF"))
      (cond (sname (setf nb 32)) (t (setf nb 32))))
     ((f2cl-lib:fstring-= c3 "HRD")
      (cond (sname (setf nb 32)) (t (setf nb 32))))
     ((f2cl-lib:fstring-= c3 "BRD")
      (cond (sname (setf nb 32)) (t (setf nb 32))))
     ((f2cl-lib:fstring-= c3 "TRI")
      (cond (sname (setf nb 64)) (t (setf nb 64))))))
   ((f2cl-lib:fstring-= c2 "PO")
    (cond
     ((f2cl-lib:fstring-= c3 "TRF")
      (cond (sname (setf nb 64)) (t (setf nb 64))))))
   ((f2cl-lib:fstring-= c2 "SY")
    (cond
     ((f2cl-lib:fstring-= c3 "TRF")
      (cond (sname (setf nb 64)) (t (setf nb 64))))
     ((and sname (f2cl-lib:fstring-= c3 "TRD")) (setf nb 32))
     ((and sname (f2cl-lib:fstring-= c3 "GST")) (setf nb 64))))
   ((and cname (f2cl-lib:fstring-= c2 "HE"))
    (cond ((f2cl-lib:fstring-= c3 "TRF") (setf nb 64))
          ((f2cl-lib:fstring-= c3 "TRD") (setf nb 32))
          ((f2cl-lib:fstring-= c3 "GST") (setf nb 64))))
```

```
((and sname (f2cl-lib:fstring-= c2 "OR"))
 (cond
  ((f2cl-lib:fstring-= (f2cl-lib:fref-string c3 (1 1)) "G")
   (cond
    ((or (f2cl-lib:fstring-= c4 "QR") (f2cl-lib:fstring-= c4 "RQ")
         (f2cl-lib:fstring-= c4 "LQ") (f2cl-lib:fstring-= c4 "QL")
         (f2cl-lib:fstring-= c4 "HR") (f2cl-lib:fstring-= c4 "TR")
         (f2cl-lib:fstring-= c4 "BR"))
     (setf nb 32))))
  ((f2cl-lib:fstring-= (f2cl-lib:fref-string c3 (1 1)) "M")
   (cond
    ((or (f2cl-lib:fstring-= c4 "QR") (f2cl-lib:fstring-= c4 "RQ")
         (f2cl-lib:fstring-= c4 "LQ") (f2cl-lib:fstring-= c4 "QL")
         (f2cl-lib:fstring-= c4 "HR") (f2cl-lib:fstring-= c4 "TR")
         (f2cl-lib:fstring-= c4 "BR"))
     (setf nb 32))))))
((and cname (f2cl-lib:fstring-= c2 "UN"))
 (cond
  ((f2cl-lib:fstring-= (f2cl-lib:fref-string c3 (1 1)) "G")
   (cond
    ((or (f2cl-lib:fstring-= c4 "QR") (f2cl-lib:fstring-= c4 "RQ")
         (f2cl-lib:fstring-= c4 "LQ") (f2cl-lib:fstring-= c4 "QL")
         (f2cl-lib:fstring-= c4 "HR") (f2cl-lib:fstring-= c4 "TR")
         (f2cl-lib:fstring-= c4 "BR"))
     (setf nb 32))))
  ((f2cl-lib:fstring-= (f2cl-lib:fref-string c3 (1 1)) "M")
   (cond
    ((or (f2cl-lib:fstring-= c4 "QR") (f2cl-lib:fstring-= c4 "RQ")
         (f2cl-lib:fstring-= c4 "LQ") (f2cl-lib:fstring-= c4 "QL")
         (f2cl-lib:fstring-= c4 "HR") (f2cl-lib:fstring-= c4 "TR")
         (f2cl-lib:fstring-= c4 "BR"))
     (setf nb 32))))))
((f2cl-lib:fstring-= c2 "GB")
 (cond
  ((f2cl-lib:fstring-= c3 "TRF")
   (cond (sname (cond ((<= n4 64) (setf nb 1)) (t (setf nb 32))))
         (t (cond ((<= n4 64) (setf nb 1)) (t (setf nb 32))))))))
((f2cl-lib:fstring-= c2 "PB")
 (cond
  ((f2cl-lib:fstring-= c3 "TRF")
   (cond (sname (cond ((<= n2 64) (setf nb 1)) (t (setf nb 32))))
         (t (cond ((<= n2 64) (setf nb 1)) (t (setf nb 32))))))))
((f2cl-lib:fstring-= c2 "TR")
 (cond
  ((f2cl-lib:fstring-= c3 "TRI")
   (cond (sname (setf nb 64)) (t (setf nb 64))))))
((f2cl-lib:fstring-= c2 "LA")
 (cond
  ((f2cl-lib:fstring-= c3 "UUM")
   (cond (sname (setf nb 64)) (t (setf nb 64))))))
```

```
    ((and sname (f2cl-lib:fstring-= c2 "ST"))
     (cond ((f2cl-lib:fstring-= c3 "EBZ") (setf nb 1))))
    ((f2cl-lib:fstring-= c2 "GG") (setf nb 32)
     (cond
      ((f2cl-lib:fstring-= c3 "HD3")
       (cond (sname (setf nb 32)) (t (setf nb 32)))))))
   (setf ilaenv nb)
   (go end_label)
  label60
   (setf nbmin 2)
   (cond
    ((f2cl-lib:fstring-= c2 "GE")
     (cond
      ((or (f2cl-lib:fstring-= c3 "QRF") (f2cl-lib:fstring-= c3 "RQF")
           (f2cl-lib:fstring-= c3 "LQF") (f2cl-lib:fstring-= c3 "QLF"))
       (cond (sname (setf nbmin 2)) (t (setf nbmin 2))))
      ((f2cl-lib:fstring-= c3 "HRD")
       (cond (sname (setf nbmin 2)) (t (setf nbmin 2))))
      ((f2cl-lib:fstring-= c3 "BRD")
       (cond (sname (setf nbmin 2)) (t (setf nbmin 2))))
      ((f2cl-lib:fstring-= c3 "TRI")
       (cond (sname (setf nbmin 2)) (t (setf nbmin 2))))))
    ((f2cl-lib:fstring-= c2 "SY")
     (cond
      ((f2cl-lib:fstring-= c3 "TRF")
       (cond (sname (setf nbmin 8)) (t (setf nbmin 8))))
      ((and sname (f2cl-lib:fstring-= c3 "TRD")) (setf nbmin 2))))
    ((and cname (f2cl-lib:fstring-= c2 "HE"))
     (cond ((f2cl-lib:fstring-= c3 "TRD") (setf nbmin 2))))
    ((and sname (f2cl-lib:fstring-= c2 "OR"))
     (cond
      ((f2cl-lib:fstring-= (f2cl-lib:fref-string c3 (1 1)) "G")
       (cond
        ((or (f2cl-lib:fstring-= c4 "QR") (f2cl-lib:fstring-= c4 "RQ")
             (f2cl-lib:fstring-= c4 "LQ") (f2cl-lib:fstring-= c4 "QL")
             (f2cl-lib:fstring-= c4 "HR") (f2cl-lib:fstring-= c4 "TR")
             (f2cl-lib:fstring-= c4 "BR"))
         (setf nbmin 2))))
      ((f2cl-lib:fstring-= (f2cl-lib:fref-string c3 (1 1)) "M")
       (cond
        ((or (f2cl-lib:fstring-= c4 "QR") (f2cl-lib:fstring-= c4 "RQ")
             (f2cl-lib:fstring-= c4 "LQ") (f2cl-lib:fstring-= c4 "QL")
             (f2cl-lib:fstring-= c4 "HR") (f2cl-lib:fstring-= c4 "TR")
             (f2cl-lib:fstring-= c4 "BR"))
         (setf nbmin 2))))))
    ((and cname (f2cl-lib:fstring-= c2 "UN"))
     (cond
      ((f2cl-lib:fstring-= (f2cl-lib:fref-string c3 (1 1)) "G")
       (cond
        ((or (f2cl-lib:fstring-= c4 "QR") (f2cl-lib:fstring-= c4 "RQ")
```

```
              (f2cl-lib:fstring-= c4 "LQ") (f2cl-lib:fstring-= c4 "QL")
              (f2cl-lib:fstring-= c4 "HR") (f2cl-lib:fstring-= c4 "TR")
              (f2cl-lib:fstring-= c4 "BR"))
         (setf nbmin 2))))
      ((f2cl-lib:fstring-= (f2cl-lib:fref-string c3 (1 1)) "M")
       (cond
        ((or (f2cl-lib:fstring-= c4 "QR") (f2cl-lib:fstring-= c4 "RQ")
              (f2cl-lib:fstring-= c4 "LQ") (f2cl-lib:fstring-= c4 "QL")
              (f2cl-lib:fstring-= c4 "HR") (f2cl-lib:fstring-= c4 "TR")
              (f2cl-lib:fstring-= c4 "BR"))
         (setf nbmin 2))))))
    ((f2cl-lib:fstring-= c2 "GG") (setf nbmin 2)
     (cond ((f2cl-lib:fstring-= c3 "HD3") (setf nbmin 2)))))
   (setf ilaenv nbmin)
   (go end_label)
  label70
   (setf nx 0)
   (cond
    ((f2cl-lib:fstring-= c2 "GE")
     (cond
      ((or (f2cl-lib:fstring-= c3 "QRF") (f2cl-lib:fstring-= c3 "RQF")
           (f2cl-lib:fstring-= c3 "LQF") (f2cl-lib:fstring-= c3 "QLF"))
       (cond (sname (setf nx 128)) (t (setf nx 128))))
      ((f2cl-lib:fstring-= c3 "HRD")
       (cond (sname (setf nx 128)) (t (setf nx 128))))
      ((f2cl-lib:fstring-= c3 "BRD")
       (cond (sname (setf nx 128)) (t (setf nx 128))))))
    ((f2cl-lib:fstring-= c2 "SY")
     (cond ((and sname (f2cl-lib:fstring-= c3 "TRD")) (setf nx 32))))
    ((and cname (f2cl-lib:fstring-= c2 "HE"))
     (cond ((f2cl-lib:fstring-= c3 "TRD") (setf nx 32))))
    ((and sname (f2cl-lib:fstring-= c2 "OR"))
     (cond
      ((f2cl-lib:fstring-= (f2cl-lib:fref-string c3 (1 1)) "G")
       (cond
        ((or (f2cl-lib:fstring-= c4 "QR") (f2cl-lib:fstring-= c4 "RQ")
              (f2cl-lib:fstring-= c4 "LQ") (f2cl-lib:fstring-= c4 "QL")
              (f2cl-lib:fstring-= c4 "HR") (f2cl-lib:fstring-= c4 "TR")
              (f2cl-lib:fstring-= c4 "BR"))
         (setf nx 128))))))
    ((and cname (f2cl-lib:fstring-= c2 "UN"))
     (cond
      ((f2cl-lib:fstring-= (f2cl-lib:fref-string c3 (1 1)) "G")
       (cond
        ((or (f2cl-lib:fstring-= c4 "QR") (f2cl-lib:fstring-= c4 "RQ")
              (f2cl-lib:fstring-= c4 "LQ") (f2cl-lib:fstring-= c4 "QL")
              (f2cl-lib:fstring-= c4 "HR") (f2cl-lib:fstring-= c4 "TR")
              (f2cl-lib:fstring-= c4 "BR"))
         (setf nx 128))))))
    ((f2cl-lib:fstring-= c2 "GG") (setf nx 128)
```

```
    (cond ((f2cl-lib:fstring-= c3 "HD3") (setf nx 128)))))
  (setf ilaenv nx)
  (go end_label)
label80
  (setf ilaenv 6)
  (go end_label)
label90
  (setf ilaenv 2)
  (go end_label)
label100
  (setf ilaenv
          (f2cl-lib:int
           (*
            (f2cl-lib:freal
             (min (the f2cl-lib:integer4 n1) (the f2cl-lib:integer4 n2)))
            1.6)))
  (go end_label)
label110
  (setf ilaenv 1)
  (go end_label)
label120
  (setf ilaenv 50)
  (go end_label)
label130
  (setf ilaenv 25)
  (go end_label)
label140
  (setf ilaenv 1)
  (cond ((= ilaenv 1) (setf ilaenv (ieeeck 1 0.0 1.0))))
  (go end_label)
label150
  (setf ilaenv 1)
  (cond ((= ilaenv 1) (setf ilaenv (ieeeck 0 0.0 1.0))))
  (go end_label)
label160
  (setf ilaenv
          (multiple-value-bind
              (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (iparmq ispec name opts n1 n2 n3 n4)
            (declare (ignore))
            (when var-0 (setf ispec var-0))
            (when var-1 (setf name var-1))
            (when var-2 (setf opts var-2))
            (when var-3 (setf n1 var-3))
            (when var-4 (setf n2 var-4))
            (when var-5 (setf n3 var-5))
            (when var-6 (setf n4 var-6))
            ret-val))
  (go end_label)
end_label
```

```
(return (values ilaenv ispec name opts n1 n2 n3 n4)))))
```

———-

90.210 ilazlc

90.210.1 Axiom unit tests

— **ilazlc.input** —

```
)set break resume
)sys rm -f ilazlc.output
)spool ilazlc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.210.2 Axiom help page

— **ilazlc.help** —

```
======================================================================
ilazlc examples
======================================================================

======================================================================
Man Page Details
======================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       INTEGER FUNCTION ILAZLC( M, N, A, LDA )

       .. Scalar Arguments ..
```

```
      INTEGER            M, N, LDA
      ..
      .. Array Arguments ..
      COMPLEX*16         A( LDA, * )
      ..


 Purpose:
 =============

   ILAZLC scans A for its last non-zero column.

 Arguments:
 ==========

  [in] M
          M is INTEGER
          The number of rows of the matrix A.

  [in] N
          N is INTEGER
          The number of columns of the matrix A.

  [in] A
          A is COMPLEX*16 array, dimension (LDA,N)
          The m by n matrix A.

  [in] LDA
          LDA is INTEGER
          The leading dimension of the array A. LDA >= max(1,M).
```

90.210.3 fortran code

— ilazlc.f —

```
*  =====================================================================
      INTEGER FUNCTION ILAZLC( M, N, A, LDA )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            M, N, LDA
```

```
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16       ZERO
      PARAMETER ( ZERO = (0.0D+0, 0.0D+0) )
*     ..
*     .. Local Scalars ..
      INTEGER I
*     ..
*     .. Executable Statements ..
*
*     Quick test for the common case where one corner is non-zero.
      IF( N.EQ.0 ) THEN
         ILAZLC = N
      ELSE IF( A(1, N).NE.ZERO .OR. A(M, N).NE.ZERO ) THEN
         ILAZLC = N
      ELSE
*     Now scan each column from the end, returning with the first non-zero.
         DO ILAZLC = N, 1, -1
            DO I = 1, M
               IF( A(I, ILAZLC).NE.ZERO ) RETURN
            END DO
         END DO
      END IF
      RETURN
      END
```

———-

90.210.4 lisp code

— ilazlc.lisp —

```
(let* ((zero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) zero)
           (ignorable zero))
  (defun ilazlc (m n a lda)
    (declare (type (array f2cl-lib:complex16 (*)) a)
             (type (f2cl-lib:integer4) lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%))
      (prog ((i 0) (ilazlc 0))
```

```
(declare (type (f2cl-lib:integer4) i ilazlc))
(cond ((= n 0) (setf ilazlc n))
      ((or (/= (f2cl-lib:fref a (1 n) ((1 lda) (1 *))) zero)
           (/= (f2cl-lib:fref a (m n) ((1 lda) (1 *))) zero))
       (setf ilazlc n))
      (t
       (f2cl-lib:fdo (ilazlc n
                      (f2cl-lib:int-add ilazlc (f2cl-lib:int-sub 1)))
                     ((> ilazlc 1) nil)
         (tagbody
           (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                         ((> i m) nil)
             (tagbody
               (if (/=
                    (f2cl-lib:fref a-%data% (i ilazlc) ((1 lda) (1 *))
                                   a-%offset%)
                    zero)
                   (go end_label))
              label100001))
          label100000))))
(go end_label)
end_label
(return (values ilazlc nil nil nil nil))))))
```

———-

90.211 ilazlr

90.211.1 Axiom unit tests

— **ilazlr.input** —

```
)set break resume
)sys rm -f ilazlr.output
)spool ilazlr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.211.2 Axiom help page

— ilazlr.help —

```
=====================================================================
ilazlr examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

  Definition:
  ===========

        INTEGER FUNCTION ILAZLR( M, N, A, LDA )

        .. Scalar Arguments ..
        INTEGER            M, N, LDA
        ..
        .. Array Arguments ..
        COMPLEX*16         A( LDA, * )
        ..


  Purpose:
  =============

 ILAZLR scans A for its last non-zero row.

  Arguments:
  ==========

   [in] M

          M is INTEGER
          The number of rows of the matrix A.

   [in] N
          N is INTEGER
          The number of columns of the matrix A.

   [in] A

          A is COMPLEX*16 array, dimension (LDA,N)
          The m by n matrix A.
```

```
[in] LDA

       LDA is INTEGER
       The leading dimension of the array A. LDA >= max(1,M).
```

———-

90.211.3 fortran code

— **ilazlr.f** —

```
*  =====================================================================
      INTEGER FUNCTION ILAZLR( M, N, A, LDA )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            M, N, LDA
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16       ZERO
      PARAMETER ( ZERO = (0.0D+0, 0.0D+0) )
*     ..
*     .. Local Scalars ..
      INTEGER I, J
*     ..
*     .. Executable Statements ..
*
*     Quick test for the common case where one corner is non-zero.
      IF( M.EQ.0 ) THEN
         ILAZLR = M
      ELSE IF( A(M, 1).NE.ZERO .OR. A(M, N).NE.ZERO ) THEN
         ILAZLR = M
      ELSE
*     Scan up each column tracking the last zero row seen.
         ILAZLR = 0
         DO J = 1, N
```

```
            I=M
            DO WHILE ((A(I,J).NE.ZERO).AND.(I.GE.1))
              I=I-1
              IF (I.EQ.0) THEN
                 EXIT
              END IF
            ENDDO
            ILAZLR = MAX( ILAZLR, I )
         END DO
      END IF
      RETURN
      END
```

———-

90.211.4 lisp code

— ilazlr.lisp —

```
(let* ((zero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) zero)
           (ignorable zero))
  (defun ilazlr (m n a lda)
    (declare (type (array f2cl-lib:complex16 (*)) a)
             (type (f2cl-lib:integer4) lda n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%))
      (prog ((i 0) (j 0) (ilazlr 0))
        (declare (type (f2cl-lib:integer4) i j ilazlr))
        (cond ((= m 0) (setf ilazlr m))
              ((or (/= (f2cl-lib:fref a (m 1) ((1 lda) (1 *))) zero)
                   (/= (f2cl-lib:fref a (m n) ((1 lda) (1 *))) zero))
               (setf ilazlr m))
              (t (setf ilazlr 0)
               (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                             ((> j n) nil)
                 (tagbody
                   (setf i m)
                  label100001
                   (if (not
                        (and
                         (=
                          (f2cl-lib:fref a-%data%
                                         ((max (the f2cl-lib:integer4 i)
                                               (the f2cl-lib:integer4 1))
                                          j)
                                         ((1 lda) (1 *)) a-%offset%)
```

```
                        zero)
                       (>= i 1)))
                     (go label100002))
                  (setf i (f2cl-lib:int-sub i 1))
                  (go label100001)
                label100002
                  (setf ilazlr
                          (max (the f2cl-lib:integer4 ilazlr)
                               (the f2cl-lib:integer4 i)))
                label100000))))
        (go end_label)
       end_label
        (return (values ilazlr nil nil nil nil))))))
```

———-

90.212 zgebak

90.212.1 Axiom unit tests

— **zgebak.input** —

```
)set break resume
)sys rm -f zgebak.output
)spool zgebak.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.212.2 Axiom help page

— **zgebak.help** —

```
=====================================================================
zgebak examples
=====================================================================

=====================================================================
```

```
Man Page Details
=====================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

  Definition:
  ===========

       SUBROUTINE ZGEBAK( JOB, SIDE, N, ILO, IHI, SCALE, M, V, LDV,
                          INFO )

       .. Scalar Arguments ..
       CHARACTER          JOB, SIDE
       INTEGER            IHI, ILO, INFO, LDV, M, N
       ..
       .. Array Arguments ..
       DOUBLE PRECISION   SCALE( * )
       COMPLEX*16         V( LDV, * )
       ..


  Purpose:
  =============

   ZGEBAK forms the right or left eigenvectors of a complex general
   matrix by backward transformation on the computed eigenvectors of the
   balanced matrix output by ZGEBAL.

  Arguments:
  ==========

  [in] JOB

          JOB is CHARACTER*1
          Specifies the type of backward transformation required:
          = 'N', do nothing, return immediately;
          = 'P', do backward transformation for permutation only;
          = 'S', do backward transformation for scaling only;
          = 'B', do backward transformations for both permutation and
                 scaling.
          JOB must be the same as the argument JOB supplied to ZGEBAL.

  [in] SIDE

          SIDE is CHARACTER*1
          = 'R':  V contains right eigenvectors;
          = 'L':  V contains left eigenvectors.

  [in] N
```

```
        N is INTEGER
        The number of rows of the matrix V.  N >= 0.

[in] ILO

        ILO is INTEGER

[in] IHI

        IHI is INTEGER
        The integers ILO and IHI determined by ZGEBAL.
        1 <= ILO <= IHI <= N, if N > 0; ILO=1 and IHI=0, if N=0.


[in] SCALE

        SCALE is DOUBLE PRECISION array, dimension (N)
        Details of the permutation and scaling factors, as returned
        by ZGEBAL.


[in] M

        M is INTEGER
        The number of columns of the matrix V.  M >= 0.

[in,out] V

        V is COMPLEX*16 array, dimension (LDV,M)
        On entry, the matrix of right or left eigenvectors to be
        transformed, as returned by ZHSEIN or ZTREVC.
        On exit, V is overwritten by the transformed eigenvectors.

[in] LDV

        LDV is INTEGER
        The leading dimension of the array V. LDV >= max(1,N).

[out] INFO

        INFO is INTEGER
        = 0:  successful exit
        < 0:  if INFO = -i, the i-th argument had an illegal value.
```

———-

90.212.3 fortran code

— zgebak.f —

```
*  =====================================================================
      SUBROUTINE ZGEBAK( JOB, SIDE, N, ILO, IHI, SCALE, M, V, LDV,
     $                   INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOB, SIDE
      INTEGER            IHI, ILO, INFO, LDV, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   SCALE( * )
      COMPLEX*16         V( LDV, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE
      PARAMETER          ( ONE = 1.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LEFTV, RIGHTV
      INTEGER            I, II, K
      DOUBLE PRECISION   S
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZDSCAL, ZSWAP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Decode and Test the input parameters
*
      RIGHTV = LSAME( SIDE, 'R' )
      LEFTV = LSAME( SIDE, 'L' )
```

```
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.RIGHTV .AND. .NOT.LEFTV ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -5
      ELSE IF( M.LT.0 ) THEN
         INFO = -7
      ELSE IF( LDV.LT.MAX( 1, N ) ) THEN
         INFO = -9
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEBAK', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
      IF( M.EQ.0 )
     $   RETURN
      IF( LSAME( JOB, 'N' ) )
     $   RETURN
*
      IF( ILO.EQ.IHI )
     $   GO TO 30
*
*     Backward balance
*
      IF( LSAME( JOB, 'S' ) .OR. LSAME( JOB, 'B' ) ) THEN
*
         IF( RIGHTV ) THEN
            DO 10 I = ILO, IHI
               S = SCALE( I )
               CALL ZDSCAL( M, S, V( I, 1 ), LDV )
   10       CONTINUE
         END IF
*
         IF( LEFTV ) THEN
            DO 20 I = ILO, IHI
               S = ONE / SCALE( I )
               CALL ZDSCAL( M, S, V( I, 1 ), LDV )
```

```
   20       CONTINUE
         END IF
*
      END IF
*
*     Backward permutation
*
*     For  I = ILO-1 step -1 until 1,
*              IHI+1 step 1 until N do --
*
   30 CONTINUE
      IF( LSAME( JOB, 'P' ) .OR. LSAME( JOB, 'B' ) ) THEN
         IF( RIGHTV ) THEN
            DO 40 II = 1, N
               I = II
               IF( I.GE.ILO .AND. I.LE.IHI )
     $            GO TO 40
               IF( I.LT.ILO )
     $            I = ILO - II
               K = SCALE( I )
               IF( K.EQ.I )
     $            GO TO 40
               CALL ZSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   40       CONTINUE
         END IF
*
         IF( LEFTV ) THEN
            DO 50 II = 1, N
               I = II
               IF( I.GE.ILO .AND. I.LE.IHI )
     $            GO TO 50
               IF( I.LT.ILO )
     $            I = ILO - II
               K = SCALE( I )
               IF( K.EQ.I )
     $            GO TO 50
               CALL ZSWAP( M, V( I, 1 ), LDV, V( K, 1 ), LDV )
   50       CONTINUE
         END IF
      END IF
*
      RETURN
*
*     End of ZGEBAK
*
      END
```

90.212.4 lisp code

— zgebak.lisp —

```
(let* ((one 1.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (ignorable one))
  (defun zgebak (job side n ilo ihi scale m v ldv info)
    (declare (type (array f2cl-lib:complex16 (*)) v)
             (type (array double-float (*)) scale)
             (type (f2cl-lib:integer4) info ldv m ihi ilo n)
             (type (string *) side job))
    (f2cl-lib:with-multi-array-data
        ((job character job-%data% job-%offset%)
         (side character side-%data% side-%offset%)
         (scale double-float scale-%data% scale-%offset%)
         (v f2cl-lib:complex16 v-%data% v-%offset%))
      (prog ((s 0.0d0) (i 0) (ii 0) (k 0) (leftv nil) (rightv nil))
        (declare (type (double-float) s)
                 (type (f2cl-lib:integer4) i ii k)
                 (type f2cl-lib:logical leftv rightv))
        (setf rightv
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame side "R")
                  (declare (ignore var-1))
                  (when var-0 (setf side var-0))
                  ret-val))
        (setf leftv
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame side "L")
                  (declare (ignore var-1))
                  (when var-0 (setf side var-0))
                  ret-val))
        (setf info 0)
        (cond
         ((and
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame job "N")
              (declare (ignore var-1))
              (when var-0 (setf job var-0))
              ret-val))
           (not
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame job "P")
              (declare (ignore var-1))
              (when var-0 (setf job var-0))
              ret-val))
           (not
```

```
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "S")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "B")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)))
  (setf info -1))
 ((and (not rightv) (not leftv)) (setf info -2))
 ((< n 0) (setf info -3))
 ((or (< ilo 1)
      (> ilo
         (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n))))
  (setf info -4))
 ((or
   (< ihi (min (the f2cl-lib:integer4 ilo) (the f2cl-lib:integer4 n)))
   (> ihi n))
  (setf info -5))
 ((< m 0) (setf info -7))
 ((< ldv (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -9)))
(cond
 ((/= info 0) (xerbla "ZGEBAK" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(if (= m 0)
    (go end_label))
(if (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "N")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)
    (go end_label))
(if (= ilo ihi)
    (go label30))
(cond
 ((or
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame job "S")
     (declare (ignore var-1))
     (when var-0 (setf job var-0))
     ret-val)
   (multiple-value-bind (ret-val var-0 var-1)
       (lsame job "B")
     (declare (ignore var-1))
```

```
        (when var-0 (setf job var-0))
        ret-val))
      (cond
       (rightv
        (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                      ((> i ihi) nil)
          (tagbody
            (setf s
                    (f2cl-lib:fref scale-%data% (i) ((1 *))
                                   scale-%offset%))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (zdscal m s
                 (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (i 1)
                                       ((1 ldv) (1 *)) v-%offset%)
                 ldv)
              (declare (ignore var-2))
              (when var-0 (setf m var-0))
              (when var-1 (setf s var-1))
              (when var-3 (setf ldv var-3)))
           label10))))
      (cond
       (leftv
        (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                      ((> i ihi) nil)
          (tagbody
            (setf s
                    (/ one
                       (f2cl-lib:fref scale-%data% (i) ((1 *))
                                      scale-%offset%)))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (zdscal m s
                 (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (i 1)
                                       ((1 ldv) (1 *)) v-%offset%)
                 ldv)
              (declare (ignore var-2))
              (when var-0 (setf m var-0))
              (when var-1 (setf s var-1))
              (when var-3 (setf ldv var-3)))
           label20))))))
    label30
     (cond
      ((or
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame job "P")
          (declare (ignore var-1))
          (when var-0 (setf job var-0))
          ret-val)
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame job "B")
          (declare (ignore var-1))
```

```
    (when var-0 (setf job var-0))
    ret-val))
 (cond
  (rightv
   (f2cl-lib:fdo (ii 1 (f2cl-lib:int-add ii 1))
                 ((> ii n) nil)
     (tagbody
       (setf i ii)
       (if (and (>= i ilo) (<= i ihi))
           (go label40))
       (if (< i ilo)
           (setf i (f2cl-lib:int-sub ilo ii)))
       (setf k
               (f2cl-lib:int
                (f2cl-lib:fref scale-%data% (i) ((1 *))
                               scale-%offset%)))
       (if (= k i)
           (go label40))
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (zswap m
            (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (i 1)
                                  ((1 ldv) (1 *)) v-%offset%)
            ldv
            (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (k 1)
                                  ((1 ldv) (1 *)) v-%offset%)
            ldv)
         (declare (ignore var-1 var-3))
         (when var-0 (setf m var-0))
         (when var-2 (setf ldv var-2))
         (when var-4 (setf ldv var-4)))
     label40))))
 (cond
  (leftv
   (f2cl-lib:fdo (ii 1 (f2cl-lib:int-add ii 1))
                 ((> ii n) nil)
     (tagbody
       (setf i ii)
       (if (and (>= i ilo) (<= i ihi))
           (go label50))
       (if (< i ilo)
           (setf i (f2cl-lib:int-sub ilo ii)))
       (setf k
               (f2cl-lib:int
                (f2cl-lib:fref scale-%data% (i) ((1 *))
                               scale-%offset%)))
       (if (= k i)
           (go label50))
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (zswap m
            (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (i 1)
```

```
                                        ((1 ldv) (1 *)) v-%offset%)
                    ldv
                    (f2cl-lib:array-slice v-%data% f2cl-lib:complex16 (k 1)
                                        ((1 ldv) (1 *)) v-%offset%)
                    ldv)
                (declare (ignore var-1 var-3))
                (when var-0 (setf m var-0))
                (when var-2 (setf ldv var-2))
                (when var-4 (setf ldv var-4)))
            label50))))))
  (go end_label)
 end_label
  (return (values job side nil nil nil nil m nil ldv info))))))
```

———-

90.213 zgebal

90.213.1 Axiom unit tests

— **zgebal.input** —

```
)set break resume
)sys rm -f zgebal.output
)spool zgebal.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.213.2 Axiom help page

— **zgebal.help** —

```
==================================================================
zgebal examples
==================================================================

==================================================================
```

```
Man Page Details
=====================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZGEBAL( JOB, N, A, LDA, ILO, IHI, SCALE, INFO )

       .. Scalar Arguments ..
       CHARACTER          JOB
       INTEGER            IHI, ILO, INFO, LDA, N
       ..
       .. Array Arguments ..
       DOUBLE PRECISION   SCALE( * )
       COMPLEX*16         A( LDA, * )
       ..


 Purpose:
 =============

  ZGEBAL balances a general complex matrix A.  This involves, first,
  permuting A by a similarity transformation to isolate eigenvalues
  in the first 1 to ILO-1 and last IHI+1 to N elements on the
  diagonal; and second, applying a diagonal similarity transformation
  to rows and columns ILO to IHI to make the rows and columns as
  close in norm as possible.  Both steps are optional.

  Balancing may reduce the 1-norm of the matrix, and improve the
  accuracy of the computed eigenvalues and/or eigenvectors.

 Arguments:
 ==========

   [in] JOB

          JOB is CHARACTER*1
          Specifies the operations to be performed on A:
          = 'N':  none:  simply set ILO = 1, IHI = N, SCALE(I) = 1.0
                  for i = 1,...,N;
          = 'P':  permute only;
          = 'S':  scale only;
          = 'B':  both permute and scale.

   [in] N

          N is INTEGER
```

```
          The order of the matrix A.  N >= 0.

   [in,out] A

          A is COMPLEX*16 array, dimension (LDA,N)
          On entry, the input matrix A.
          On exit,  A is overwritten by the balanced matrix.
          If JOB = 'N', A is not referenced.
          See Further Details.

   [in] LDA

          LDA is INTEGER
          The leading dimension of the array A.  LDA >= max(1,N).

   [out] ILO

   [out] IHI

          ILO and IHI are set to INTEGER such that on exit
          A(i,j) = 0 if i > j and j = 1,...,ILO-1 or I = IHI+1,...,N.
          If JOB = 'N' or 'S', ILO = 1 and IHI = N.

   [out] SCALE

          SCALE is DOUBLE PRECISION array, dimension (N)
          Details of the permutations and scaling factors applied to
          A.  If P(j) is the index of the row and column interchanged
          with row and column j and D(j) is the scaling factor
          applied to row and column j, then
          SCALE(j) = P(j)    for j = 1,...,ILO-1
                   = D(j)    for j = ILO,...,IHI
                   = P(j)    for j = IHI+1,...,N.
          The order in which the interchanges are made is N to IHI+1,
          then 1 to ILO-1.

   [out] INFO

          INFO is INTEGER
          = 0:  successful exit.
          < 0:  if INFO = -i, the i-th argument had an illegal value.

 Further Details:
 ====================

  The permutations consist of row and column interchanges which put
  the matrix in the form

             ( T1   X   Y  )
     P A P = (  0   B   Z  )
```

```
            (  0   0   T2 )

  where T1 and T2 are upper triangular matrices whose eigenvalues lie
  along the diagonal.  The column indices ILO and IHI mark the starting
  and ending columns of the submatrix B. Balancing consists of applying
  a diagonal similarity transformation inv(D) * B * D to make the
  1-norms of each row of B and its corresponding column nearly equal.
  The output matrix is

     ( T1     X*D          Y    )
     (  0  inv(D)*B*D  inv(D)*Z ).
     (  0      0           T2   )

  Information about the permutations P and the diagonal matrix D is
  returned in the vector SCALE.

  This subroutine is based on the EISPACK routine CBAL.

  Modified by Tzu-Yi Chen, Computer Science Division, University of
    California at Berkeley, USA
```

———-

90.213.3 fortran code

— zgebal.f —

```
*  =====================================================================
      SUBROUTINE ZGEBAL( JOB, N, A, LDA, ILO, IHI, SCALE, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          JOB
      INTEGER            IHI, ILO, INFO, LDA, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   SCALE( * )
      COMPLEX*16         A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
```

```
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      DOUBLE PRECISION   SCLFAC
      PARAMETER          ( SCLFAC = 2.0D+0 )
      DOUBLE PRECISION   FACTOR
      PARAMETER          ( FACTOR = 0.95D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOCONV
      INTEGER            I, ICA, IEXC, IRA, J, K, L, M
      DOUBLE PRECISION   C, CA, F, G, R, RA, S, SFMAX1, SFMAX2, SFMIN1,
     $                   SFMIN2
      COMPLEX*16         CDUM
*     ..
*     .. External Functions ..
      LOGICAL            DISNAN, LSAME
      INTEGER            IZAMAX
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DISNAN, LSAME, IZAMAX, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZDSCAL, ZSWAP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG, MAX, MIN
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( CDUM ) = ABS( DBLE( CDUM ) ) + ABS( DIMAG( CDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'N' ) .AND. .NOT.LSAME( JOB, 'P' ) .AND.
     $    .NOT.LSAME( JOB, 'S' ) .AND. .NOT.LSAME( JOB, 'B' ) ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -4
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEBAL', -INFO )
         RETURN
      END IF
*
```

```
      K = 1
      L = N
*
      IF( N.EQ.0 )
     $   GO TO 210
*
      IF( LSAME( JOB, 'N' ) ) THEN
         DO 10 I = 1, N
            SCALE( I ) = ONE
   10    CONTINUE
         GO TO 210
      END IF
*
      IF( LSAME( JOB, 'S' ) )
     $   GO TO 120
*
*     Permutation to isolate eigenvalues if possible
*
      GO TO 50
*
*     Row and column exchange.
*
   20 CONTINUE
      SCALE( M ) = J
      IF( J.EQ.M )
     $   GO TO 30
*
      CALL ZSWAP( L, A( 1, J ), 1, A( 1, M ), 1 )
      CALL ZSWAP( N-K+1, A( J, K ), LDA, A( M, K ), LDA )
*
   30 CONTINUE
      GO TO ( 40, 80 )IEXC
*
*     Search for rows isolating an eigenvalue and push them down.
*
   40 CONTINUE
      IF( L.EQ.1 )
     $   GO TO 210
      L = L - 1
*
   50 CONTINUE
      DO 70 J = L, 1, -1
*
         DO 60 I = 1, L
            IF( I.EQ.J )
     $         GO TO 60
            IF( DBLE( A( J, I ) ).NE.ZERO .OR. DIMAG( A( J, I ) ).NE.
     $          ZERO )GO TO 70
   60    CONTINUE
*
```

```
         M = L
         IEXC = 1
         GO TO 20
   70 CONTINUE
*
      GO TO 90
*
*     Search for columns isolating an eigenvalue and push them left.
*
   80 CONTINUE
      K = K + 1
*
   90 CONTINUE
      DO 110 J = K, L
*
         DO 100 I = K, L
            IF( I.EQ.J )
     $         GO TO 100
            IF( DBLE( A( I, J ) ).NE.ZERO .OR. DIMAG( A( I, J ) ).NE.
     $          ZERO )GO TO 110
  100    CONTINUE
*
         M = K
         IEXC = 2
         GO TO 20
  110 CONTINUE
*
  120 CONTINUE
      DO 130 I = K, L
         SCALE( I ) = ONE
  130 CONTINUE
*
      IF( LSAME( JOB, 'P' ) )
     $   GO TO 210
*
*     Balance the submatrix in rows K to L.
*
*     Iterative loop for norm reduction
*
      SFMIN1 = DLAMCH( 'S' ) / DLAMCH( 'P' )
      SFMAX1 = ONE / SFMIN1
      SFMIN2 = SFMIN1*SCLFAC
      SFMAX2 = ONE / SFMIN2
  140 CONTINUE
      NOCONV = .FALSE.
*
      DO 200 I = K, L
         C = ZERO
         R = ZERO
*
```

```
         DO 150 J = K, L
            IF( J.EQ.I )
     $         GO TO 150
            C = C + CABS1( A( J, I ) )
            R = R + CABS1( A( I, J ) )
  150    CONTINUE
         ICA = IZAMAX( L, A( 1, I ), 1 )
         CA = ABS( A( ICA, I ) )
         IRA = IZAMAX( N-K+1, A( I, K ), LDA )
         RA = ABS( A( I, IRA+K-1 ) )
*
*        Guard against zero C or R due to underflow.
*
         IF( C.EQ.ZERO .OR. R.EQ.ZERO )
     $      GO TO 200
         G = R / SCLFAC
         F = ONE
         S = C + R
  160    CONTINUE
         IF( C.GE.G .OR. MAX( F, C, CA ).GE.SFMAX2 .OR.
     $       MIN( R, G, RA ).LE.SFMIN2 )GO TO 170
            IF( DISNAN( C+F+CA+R+G+RA ) ) THEN
*
*           Exit if NaN to avoid infinite loop
*
            INFO = -3
            CALL XERBLA( 'ZGEBAL', -INFO )
            RETURN
         END IF
         F = F*SCLFAC
         C = C*SCLFAC
         CA = CA*SCLFAC
         R = R / SCLFAC
         G = G / SCLFAC
         RA = RA / SCLFAC
         GO TO 160
*
  170    CONTINUE
         G = C / SCLFAC
  180    CONTINUE
         IF( G.LT.R .OR. MAX( R, RA ).GE.SFMAX2 .OR.
     $       MIN( F, C, G, CA ).LE.SFMIN2 )GO TO 190
         F = F / SCLFAC
         C = C / SCLFAC
         G = G / SCLFAC
         CA = CA / SCLFAC
         R = R*SCLFAC
         RA = RA*SCLFAC
         GO TO 180
*
```

```
*        Now balance.
*
  190    CONTINUE
         IF( ( C+R ).GE.FACTOR*S )
     $      GO TO 200
         IF( F.LT.ONE .AND. SCALE( I ).LT.ONE ) THEN
            IF( F*SCALE( I ).LE.SFMIN1 )
     $         GO TO 200
         END IF
         IF( F.GT.ONE .AND. SCALE( I ).GT.ONE ) THEN
            IF( SCALE( I ).GE.SFMAX1 / F )
     $         GO TO 200
         END IF
         G = ONE / F
         SCALE( I ) = SCALE( I )*F
         NOCONV = .TRUE.
*
         CALL ZDSCAL( N-K+1, G, A( I, K ), LDA )
         CALL ZDSCAL( L, F, A( 1, I ), 1 )
*
  200 CONTINUE
*
      IF( NOCONV )
     $   GO TO 140
*
  210 CONTINUE
      ILO = K
      IHI = L
*
      RETURN
*
*     End of ZGEBAL
*
      END
```

———-

90.213.4 lisp code

— **zgebal.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0) (sclfac 2.0d0) (factor 0.95d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) sclfac)
           (type (double-float 0.95d0 0.95d0) factor)
           (ignorable zero one sclfac factor))
```

```
(defun zgebal (job n a lda ilo ihi scale info)
  (declare (type (array double-float (*)) scale)
           (type (array f2cl-lib:complex16 (*)) a)
           (type (f2cl-lib:integer4) info ihi ilo lda n)
           (type (string *) job))
  (f2cl-lib:with-multi-array-data
      ((job character job-%data% job-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (scale double-float scale-%data% scale-%offset%))
    (prog ((cdum #C(0.0d0 0.0d0)) (c 0.0d0) (ca 0.0d0) (f 0.0d0) (g 0.0d0)
           (r 0.0d0) (ra 0.0d0) (s 0.0d0) (sfmax1 0.0d0) (sfmax2 0.0d0)
           (sfmin1 0.0d0) (sfmin2 0.0d0) (i 0) (ica 0) (iexc 0) (ira 0) (j 0)
           (k 0) (l 0) (m 0) (noconv nil))
      (declare (type (f2cl-lib:complex16) cdum)
               (type (double-float) c ca f g r ra s sfmax1 sfmax2 sfmin1
                sfmin2)
               (type (f2cl-lib:integer4) i ica iexc ira j k l m)
               (type f2cl-lib:logical noconv))
      (setf info 0)
      (cond
       ((and
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame job "N")
            (declare (ignore var-1))
            (when var-0 (setf job var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame job "P")
            (declare (ignore var-1))
            (when var-0 (setf job var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame job "S")
            (declare (ignore var-1))
            (when var-0 (setf job var-0))
            ret-val))
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame job "B")
            (declare (ignore var-1))
            (when var-0 (setf job var-0))
            ret-val)))
        (setf info -1))
       ((< n 0) (setf info -2))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
        (setf info -4)))
      (cond
```

```
 ((/= info 0) (xerbla "ZGEBAL" (f2cl-lib:int-sub info))
  (go end_label)))
(setf k 1)
(setf l n)
(if (= n 0)
    (go label210))
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame job "N")
    (declare (ignore var-1))
    (when var-0 (setf job var-0))
    ret-val)
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%)
              one)
     label10))
  (go label210)))
(if (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "S")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val)
    (go label120))
(go label50)
label20
(setf (f2cl-lib:fref scale-%data% (m) ((1 *)) scale-%offset%)
        (coerce (the f2cl-lib:integer4 j) 'double-float))
(if (= j m)
    (go label30))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (zswap l
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 j)
                           ((1 lda) (1 *)) a-%offset%)
     1
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 m)
                           ((1 lda) (1 *)) a-%offset%)
     1)
  (declare (ignore var-1 var-2 var-3 var-4))
  (when var-0 (setf l var-0)))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (zswap (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (j k)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (m k)
                           ((1 lda) (1 *)) a-%offset%)
     lda)
  (declare (ignore var-0 var-1 var-3))
```

```
    (when var-2 (setf lda var-2))
    (when var-4 (setf lda var-4)))
label30
 (f2cl-lib:computed-goto (label40 label80) iexc)
label40
 (if (= l 1)
     (go label210))
 (setf l (f2cl-lib:int-sub l 1))
label50
 (f2cl-lib:fdo (j l (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
               ((> j 1) nil)
   (tagbody
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i l) nil)
       (tagbody
         (if (= i j)
             (go label60))
         (if (or
              (/=
               (f2cl-lib:dble
                (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                               a-%offset%))
               zero)
              (/=
               (f2cl-lib:dimag
                (f2cl-lib:fref a-%data% (j i) ((1 lda) (1 *))
                               a-%offset%))
               zero))
             (go label70))
        label60))
     (setf m l)
     (setf iexc 1)
     (go label20)
    label70))
 (go label90)
label80
 (setf k (f2cl-lib:int-add k 1))
label90
 (f2cl-lib:fdo (j k (f2cl-lib:int-add j 1))
               ((> j l) nil)
   (tagbody
     (f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
                   ((> i l) nil)
       (tagbody
         (if (= i j)
             (go label100))
         (if (or
              (/=
               (f2cl-lib:dble
                (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
```

```
                                a-%offset%))
                 zero)
                (/=
                 (f2cl-lib:dimag
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%))
                 zero))
               (go label110))
           label100))
       (setf m k)
       (setf iexc 2)
       (go label20)
      label110))
  label120
   (f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
                 ((> i l) nil)
     (tagbody
       (setf (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%) one)
      label130))
   (if (multiple-value-bind (ret-val var-0 var-1)
           (lsame job "P")
         (declare (ignore var-1))
         (when var-0 (setf job var-0))
         ret-val)
       (go label210))
   (setf sfmin1 (/ (dlamch "S") (dlamch "P")))
   (setf sfmax1 (/ one sfmin1))
   (setf sfmin2 (* sfmin1 sclfac))
   (setf sfmax2 (/ one sfmin2))
  label140
   (setf noconv f2cl-lib:%false%)
   (f2cl-lib:fdo (i k (f2cl-lib:int-add i 1))
                 ((> i l) nil)
     (tagbody
       (setf c
               (dznrm2 (f2cl-lib:int-add (f2cl-lib:int-sub l k) 1)
                (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (k i)
                                      ((1 lda) (1 *)) a-%offset%)
                1))
       (setf r
               (multiple-value-bind (ret-val var-0 var-1 var-2)
                   (dznrm2 (f2cl-lib:int-add (f2cl-lib:int-sub l k) 1)
                    (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                          (i k) ((1 lda) (1 *))
                                          a-%offset%)
                    lda)
                 (declare (ignore var-0 var-1))
                 (when var-2 (setf lda var-2))
                 ret-val))
       (setf ica
```

```
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (izamax l
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   (1 i) ((1 lda) (1 *))
                                   a-%offset%)
             1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf l var-0))
          ret-val))
 (setf ca
        (abs
         (f2cl-lib:fref a-%data% (ica i) ((1 lda) (1 *))
                        a-%offset%)))
 (setf ira
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (izamax (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   (i k) ((1 lda) (1 *))
                                   a-%offset%)
             lda)
          (declare (ignore var-0 var-1))
          (when var-2 (setf lda var-2))
          ret-val))
 (setf ra
        (abs
         (f2cl-lib:fref a-%data%
                        (i
                         (f2cl-lib:int-sub (f2cl-lib:int-add ira k)
                                           1))
                        ((1 lda) (1 *)) a-%offset%)))
 (if (or (= c zero) (= r zero))
     (go label200))
 (setf g (/ r sclfac))
 (setf f one)
 (setf s (+ c r))
label160
 (if (or (>= c g) (>= (max f c ca) sfmax2) (<= (min r g ra) sfmin2))
     (go label170))
 (cond
  ((disnan (+ c f ca r g ra)) (setf info -3)
   (xerbla "ZGEBAL" (f2cl-lib:int-sub info)) (go end_label)))
 (setf f (* f sclfac))
 (setf c (* c sclfac))
 (setf ca (* ca sclfac))
 (setf r (/ r sclfac))
 (setf g (/ g sclfac))
 (setf ra (/ ra sclfac))
 (go label160)
label170
 (setf g (/ c sclfac))
```

```
    label180
     (if (or (< g r) (>= (max r ra) sfmax2) (<= (min f c g ca) sfmin2))
         (go label190))
     (setf f (/ f sclfac))
     (setf c (/ c sclfac))
     (setf g (/ g sclfac))
     (setf ca (/ ca sclfac))
     (setf r (* r sclfac))
     (setf ra (* ra sclfac))
     (go label180)
    label190
     (if (>= (+ c r) (* factor s))
         (go label200))
     (cond
      ((and (< f one) (< (f2cl-lib:fref scale (i) ((1 *))) one))
       (if (<=
            (* f
               (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%))
            sfmin1)
           (go label200))))
     (cond
      ((and (> f one) (> (f2cl-lib:fref scale (i) ((1 *))) one))
       (if (>= (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%)
               (/ sfmax1 f))
           (go label200))))
     (setf g (/ one f))
     (setf (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%)
             (* (f2cl-lib:fref scale-%data% (i) ((1 *)) scale-%offset%)
                f))
     (setf noconv f2cl-lib:%true%)
     (multiple-value-bind (var-0 var-1 var-2 var-3)
         (zdscal (f2cl-lib:int-add (f2cl-lib:int-sub n k) 1) g
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i k)
                                ((1 lda) (1 *)) a-%offset%)
          lda)
       (declare (ignore var-0 var-2))
       (when var-1 (setf g var-1))
       (when var-3 (setf lda var-3)))
     (multiple-value-bind (var-0 var-1 var-2 var-3)
         (zdscal l f
          (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 i)
                                ((1 lda) (1 *)) a-%offset%)
          1)
       (declare (ignore var-2 var-3))
       (when var-0 (setf l var-0))
       (when var-1 (setf f var-1)))
    label200))
 (if noconv
     (go label140))
label210
```

```
        (setf ilo k)
        (setf ihi l)
        (go end_label)
       end_label
        (return (values job nil nil lda ilo ihi nil info))))))
```

———-

90.214 zgeev

90.214.1 Axiom unit tests

— **zgeev.input** —

```
)set break resume
)sys rm -f zgeev.output
)spool zgeev.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.214.2 Axiom help page

— **zgeev.help** —

```
======================================================================
zgeev examples
======================================================================

======================================================================
Man Page Details
======================================================================


ZGEEV computes the eigenvalues and, optionally, the left and/or right
eigenvectors for GE matrices

 Online html documentation available at
```

```
           http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZGEEV( JOBVL, JOBVR, N, A, LDA, W, VL, LDVL, VR, LDVR,
                         WORK, LWORK, RWORK, INFO )

       .. Scalar Arguments ..
       CHARACTER          JOBVL, JOBVR
       INTEGER            INFO, LDA, LDVL, LDVR, LWORK, N
       ..
       .. Array Arguments ..
       DOUBLE PRECISION   RWORK( * )
       COMPLEX*16         A( LDA, * ), VL( LDVL, * ), VR( LDVR, * ),
      $                   W( * ), WORK( * )
       ..


 Purpose:
 =============

  ZGEEV computes for an N-by-N complex nonsymmetric matrix A, the
  eigenvalues and, optionally, the left and/or right eigenvectors.

  The right eigenvector v(j) of A satisfies
                   A * v(j) = lambda(j) * v(j)
   where lambda(j) is its eigenvalue.
  The left eigenvector u(j) of A satisfies
              u(j)**H * A = lambda(j) * u(j)**H
   where u(j)**H denotes the conjugate transpose of u(j).

 The computed eigenvectors are normalized to have Euclidean norm
 equal to 1 and largest component real.

 Arguments:
 ==========

  [in] JOBVL

          JOBVL is CHARACTER*1
          = 'N': left eigenvectors of A are not computed;
          = 'V': left eigenvectors of are computed.

  [in] JOBVR

          JOBVR is CHARACTER*1
          = 'N': right eigenvectors of A are not computed;
          = 'V': right eigenvectors of A are computed.
```

```
[in] N

        N is INTEGER
        The order of the matrix A. N >= 0.

[in,out] A

        A is COMPLEX*16 array, dimension (LDA,N)
        On entry, the N-by-N matrix A.
        On exit, A has been overwritten.

[in] LDA

        LDA is INTEGER
        The leading dimension of the array A.  LDA >= max(1,N).

[out] W

        W is COMPLEX*16 array, dimension (N)
        W contains the computed eigenvalues.

[out] VL

        VL is COMPLEX*16 array, dimension (LDVL,N)
        If JOBVL = 'V', the left eigenvectors u(j) are stored one
        after another in the columns of VL, in the same order
        as their eigenvalues.
        If JOBVL = 'N', VL is not referenced.
        u(j) = VL(:,j), the j-th column of VL.

[in] LDVL

        LDVL is INTEGER
        The leading dimension of the array VL.  LDVL >= 1; if
        JOBVL = 'V', LDVL >= N.

[out] VR

        VR is COMPLEX*16 array, dimension (LDVR,N)
        If JOBVR = 'V', the right eigenvectors v(j) are stored one
        after another in the columns of VR, in the same order
        as their eigenvalues.
        If JOBVR = 'N', VR is not referenced.
        v(j) = VR(:,j), the j-th column of VR.

[in] LDVR

        LDVR is INTEGER
        The leading dimension of the array VR.  LDVR >= 1; if
        JOBVR = 'V', LDVR >= N.
```

```
[out] WORK

        WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
        On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

[in] LWORK

        LWORK is INTEGER
        The dimension of the array WORK.  LWORK >= max(1,2*N).
        For good performance, LWORK must generally be larger.

        If LWORK = -1, then a workspace query is assumed; the routine
        only calculates the optimal size of the WORK array, returns
        this value as the first entry of the WORK array, and no error
        message related to LWORK is issued by XERBLA.

[out] RWORK

        RWORK is DOUBLE PRECISION array, dimension (2*N)

[out] INFO

        INFO is INTEGER
        = 0:  successful exit
        < 0:  if INFO = -i, the i-th argument had an illegal value.
        > 0:  if INFO = i, the QR algorithm failed to compute all the
              eigenvalues, and no eigenvectors have been computed;
              elements and i+1:N of W contain eigenvalues which have
              converged.
```

90.214.3 fortran code

— zgeev.f —

```
*  ===================================================================
      SUBROUTINE ZGEEV( JOBVL, JOBVR, N, A, LDA, W, VL, LDVL, VR, LDVR,
     $                  WORK, LWORK, RWORK, INFO )
*
*  -- LAPACK driver routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
```

```
      CHARACTER          JOBVL, JOBVR
      INTEGER            INFO, LDA, LDVL, LDVR, LWORK, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   RWORK( * )
      COMPLEX*16         A( LDA, * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   W( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY, SCALEA, WANTVL, WANTVR
      CHARACTER          SIDE
      INTEGER            HSWORK, I, IBAL, IERR, IHI, ILO, IRWORK, ITAU,
     $                   IWRK, K, MAXWRK, MINWRK, NOUT
      DOUBLE PRECISION   ANRM, BIGNUM, CSCALE, EPS, SCL, SMLNUM
      COMPLEX*16         TMP
*     ..
*     .. Local Arrays ..
      LOGICAL            SELECT( 1 )
      DOUBLE PRECISION   DUM( 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, XERBLA, ZDSCAL, ZGEBAK, ZGEBAL, ZGEHRD,
     $                   ZHSEQR, ZLACPY, ZLASCL, ZSCAL, ZTREVC, ZUNGHR
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX, ILAENV
      DOUBLE PRECISION   DLAMCH, DZNRM2, ZLANGE
      EXTERNAL           LSAME, IDAMAX, ILAENV, DLAMCH, DZNRM2, ZLANGE
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, DCMPLX, DCONJG, DIMAG, MAX, SQRT
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LQUERY = ( LWORK.EQ.-1 )
      WANTVL = LSAME( JOBVL, 'V' )
      WANTVR = LSAME( JOBVR, 'V' )
      IF( ( .NOT.WANTVL ) .AND. ( .NOT.LSAME( JOBVL, 'N' ) ) ) THEN
         INFO = -1
```

```
      ELSE IF( ( .NOT.WANTVR ) .AND. ( .NOT.LSAME( JOBVR, 'N' ) ) ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LDVL.LT.1 .OR. ( WANTVL .AND. LDVL.LT.N ) ) THEN
         INFO = -8
      ELSE IF( LDVR.LT.1 .OR. ( WANTVR .AND. LDVR.LT.N ) ) THEN
         INFO = -10
      END IF
*
*     Compute workspace
*      (Note: Comments in the code beginning "Workspace:" describe the
*       minimal amount of workspace needed at that point in the code,
*       as well as the preferred amount for good performance.
*       CWorkspace refers to complex workspace, and RWorkspace to real
*       workspace. NB refers to the optimal block size for the
*       immediately following subroutine, as returned by ILAENV.
*       HSWORK refers to the workspace preferred by ZHSEQR, as
*       calculated below. HSWORK is computed assuming ILO=1 and IHI=N,
*       the worst case.)
*
      IF( INFO.EQ.0 ) THEN
         IF( N.EQ.0 ) THEN
            MINWRK = 1
            MAXWRK = 1
         ELSE
            MAXWRK = N + N*ILAENV( 1, 'ZGEHRD', ' ', N, 1, N, 0 )
            MINWRK = 2*N
            IF( WANTVL ) THEN
               MAXWRK = MAX( MAXWRK, N + ( N - 1 )*ILAENV( 1, 'ZUNGHR',
     $                       ' ', N, 1, N, -1 ) )
               CALL ZHSEQR( 'S', 'V', N, 1, N, A, LDA, W, VL, LDVL,
     $                WORK, -1, INFO )
            ELSE IF( WANTVR ) THEN
               MAXWRK = MAX( MAXWRK, N + ( N - 1 )*ILAENV( 1, 'ZUNGHR',
     $                       ' ', N, 1, N, -1 ) )
               CALL ZHSEQR( 'S', 'V', N, 1, N, A, LDA, W, VR, LDVR,
     $                WORK, -1, INFO )
            ELSE
               CALL ZHSEQR( 'E', 'N', N, 1, N, A, LDA, W, VR, LDVR,
     $                WORK, -1, INFO )
            END IF
            HSWORK = WORK( 1 )
            MAXWRK = MAX( MAXWRK, HSWORK, MINWRK )
         END IF
         WORK( 1 ) = MAXWRK
*
         IF( LWORK.LT.MINWRK .AND. .NOT.LQUERY ) THEN
```

```
            INFO = -12
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEEV ', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Get machine constants
*
      EPS = DLAMCH( 'P' )
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SQRT( SMLNUM ) / EPS
      BIGNUM = ONE / SMLNUM
*
*     Scale A if max element outside range [SMLNUM,BIGNUM]
*
      ANRM = ZLANGE( 'M', N, N, A, LDA, DUM )
      SCALEA = .FALSE.
      IF( ANRM.GT.ZERO .AND. ANRM.LT.SMLNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = SMLNUM
      ELSE IF( ANRM.GT.BIGNUM ) THEN
         SCALEA = .TRUE.
         CSCALE = BIGNUM
      END IF
      IF( SCALEA )
     $   CALL ZLASCL( 'G', 0, 0, ANRM, CSCALE, N, N, A, LDA, IERR )
*
*     Balance the matrix
*     (CWorkspace: none)
*     (RWorkspace: need N)
*
      IBAL = 1
      CALL ZGEBAL( 'B', N, A, LDA, ILO, IHI, RWORK( IBAL ), IERR )
*
*     Reduce to upper Hessenberg form
*     (CWorkspace: need 2*N, prefer N+N*NB)
*     (RWorkspace: none)
*
```

```
      ITAU = 1
      IWRK = ITAU + N
      CALL ZGEHRD( N, ILO, IHI, A, LDA, WORK( ITAU ), WORK( IWRK ),
     $             LWORK-IWRK+1, IERR )
*
      IF( WANTVL ) THEN
*
*        Want left eigenvectors
*        Copy Householder vectors to VL
*
         SIDE = 'L'
         CALL ZLACPY( 'L', N, N, A, LDA, VL, LDVL )
*
*        Generate unitary matrix in VL
*        (CWorkspace: need 2*N-1, prefer N+(N-1)*NB)
*        (RWorkspace: none)
*
         CALL ZUNGHR( N, ILO, IHI, VL, LDVL, WORK( ITAU ), WORK( IWRK ),
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VL
*        (CWorkspace: need 1, prefer HSWORK (see comments) )
*        (RWorkspace: none)
*
         IWRK = ITAU
         CALL ZHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, W, VL, LDVL,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
         IF( WANTVR ) THEN
*
*           Want left and right eigenvectors
*           Copy Schur vectors to VR
*
            SIDE = 'B'
            CALL ZLACPY( 'F', N, N, VL, LDVL, VR, LDVR )
         END IF
*
      ELSE IF( WANTVR ) THEN
*
*        Want right eigenvectors
*        Copy Householder vectors to VR
*
         SIDE = 'R'
         CALL ZLACPY( 'L', N, N, A, LDA, VR, LDVR )
*
*        Generate unitary matrix in VR
*        (CWorkspace: need 2*N-1, prefer N+(N-1)*NB)
*        (RWorkspace: none)
*
         CALL ZUNGHR( N, ILO, IHI, VR, LDVR, WORK( ITAU ), WORK( IWRK ),
```

```
     $                LWORK-IWRK+1, IERR )
*
*        Perform QR iteration, accumulating Schur vectors in VR
*        (CWorkspace: need 1, prefer HSWORK (see comments) )
*        (RWorkspace: none)
*
         IWRK = ITAU
         CALL ZHSEQR( 'S', 'V', N, ILO, IHI, A, LDA, W, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
*
      ELSE
*
*        Compute eigenvalues only
*        (CWorkspace: need 1, prefer HSWORK (see comments) )
*        (RWorkspace: none)
*
         IWRK = ITAU
         CALL ZHSEQR( 'E', 'N', N, ILO, IHI, A, LDA, W, VR, LDVR,
     $                WORK( IWRK ), LWORK-IWRK+1, INFO )
      END IF
*
*     If INFO > 0 from ZHSEQR, then quit
*
      IF( INFO.GT.0 )
     $   GO TO 50
*
      IF( WANTVL .OR. WANTVR ) THEN
*
*        Compute left and/or right eigenvectors
*        (CWorkspace: need 2*N)
*        (RWorkspace: need 2*N)
*
         IRWORK = IBAL + N
         CALL ZTREVC( SIDE, 'B', SELECT, N, A, LDA, VL, LDVL, VR, LDVR,
     $                N, NOUT, WORK( IWRK ), RWORK( IRWORK ), IERR )
      END IF
*
      IF( WANTVL ) THEN
*
*        Undo balancing of left eigenvectors
*        (CWorkspace: none)
*        (RWorkspace: need N)
*
         CALL ZGEBAK( 'B', 'L', N, ILO, IHI, RWORK( IBAL ), N, VL, LDVL,
     $                IERR )
*
*        Normalize left eigenvectors and make largest component real
*
         DO 20 I = 1, N
            SCL = ONE / DZNRM2( N, VL( 1, I ), 1 )
```

```
            CALL ZDSCAL( N, SCL, VL( 1, I ), 1 )
            DO 10 K = 1, N
               RWORK( IRWORK+K-1 ) = DBLE( VL( K, I ) )**2 +
     $                               DIMAG( VL( K, I ) )**2
   10       CONTINUE
            K = IDAMAX( N, RWORK( IRWORK ), 1 )
            TMP = DCONJG( VL( K, I ) ) / SQRT( RWORK( IRWORK+K-1 ) )
            CALL ZSCAL( N, TMP, VL( 1, I ), 1 )
            VL( K, I ) = DCMPLX( DBLE( VL( K, I ) ), ZERO )
   20    CONTINUE
      END IF
*
      IF( WANTVR ) THEN
*
*        Undo balancing of right eigenvectors
*        (CWorkspace: none)
*        (RWorkspace: need N)
*
         CALL ZGEBAK( 'B', 'R', N, ILO, IHI, RWORK( IBAL ), N, VR, LDVR,
     $                IERR )
*
*        Normalize right eigenvectors and make largest component real
*
         DO 40 I = 1, N
            SCL = ONE / DZNRM2( N, VR( 1, I ), 1 )
            CALL ZDSCAL( N, SCL, VR( 1, I ), 1 )
            DO 30 K = 1, N
               RWORK( IRWORK+K-1 ) = DBLE( VR( K, I ) )**2 +
     $                               DIMAG( VR( K, I ) )**2
   30       CONTINUE
            K = IDAMAX( N, RWORK( IRWORK ), 1 )
            TMP = DCONJG( VR( K, I ) ) / SQRT( RWORK( IRWORK+K-1 ) )
            CALL ZSCAL( N, TMP, VR( 1, I ), 1 )
            VR( K, I ) = DCMPLX( DBLE( VR( K, I ) ), ZERO )
   40    CONTINUE
      END IF
*
*     Undo scaling if necessary
*
   50 CONTINUE
      IF( SCALEA ) THEN
         CALL ZLASCL( 'G', 0, 0, CSCALE, ANRM, N-INFO, 1, W( INFO+1 ),
     $                MAX( N-INFO, 1 ), IERR )
         IF( INFO.GT.0 ) THEN
            CALL ZLASCL( 'G', 0, 0, CSCALE, ANRM, ILO-1, 1, W, N, IERR )
         END IF
      END IF
*
      WORK( 1 ) = MAXWRK
      RETURN
```

```
*
*     End of ZGEEV
*
      END
```

———-

90.214.4 lisp code

— **zgeev.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun zgeev (jobvl jobvr n a lda w vl ldvl vr ldvr work lwork rwork info)
    (declare (type (array double-float (*)) rwork)
             (type (array f2cl-lib:complex16 (*)) work vr vl w a)
             (type (f2cl-lib:integer4) info lwork ldvr ldvl lda n)
             (type (string *) jobvr jobvl))
    (f2cl-lib:with-multi-array-data
        ((jobvl character jobvl-%data% jobvl-%offset%)
         (jobvr character jobvr-%data% jobvr-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (w f2cl-lib:complex16 w-%data% w-%offset%)
         (vl f2cl-lib:complex16 vl-%data% vl-%offset%)
         (vr f2cl-lib:complex16 vr-%data% vr-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (prog ((dum (make-array 1 :element-type 'double-float))
             (select (make-array 1 :element-type 't)) (tmp #C(0.0d0 0.0d0))
             (anrm 0.0d0) (bignum 0.0d0) (cscale 0.0d0) (eps 0.0d0) (scl 0.0d0)
             (smlnum 0.0d0) (hswork 0) (i 0) (ibal 0) (ierr 0) (ihi 0) (ilo 0)
             (irwork 0) (itau 0) (iwrk 0) (k 0) (maxwrk 0) (minwrk 0) (nout 0)
             (side
              (make-array '(1) :element-type 'character :initial-element #\ ))
             (lquery nil) (scalea nil) (wantvl nil) (wantvr nil))
        (declare (type (array double-float (1)) dum)
                 (type (array f2cl-lib:logical (1)) select)
                 (type (f2cl-lib:complex16) tmp)
                 (type (double-float) anrm bignum cscale eps scl smlnum)
                 (type (f2cl-lib:integer4) hswork i ibal ierr ihi ilo irwork
                  itau iwrk k maxwrk minwrk nout)
                 (type (string 1) side)
                 (type f2cl-lib:logical lquery scalea wantvl wantvr))
        (setf info 0)
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
```

```
(setf wantvl
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvl "V")
          (declare (ignore var-1))
          (when var-0 (setf jobvl var-0))
          ret-val))
(setf wantvr
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvr "V")
          (declare (ignore var-1))
          (when var-0 (setf jobvr var-0))
          ret-val))
(cond
 ((and (not wantvl)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvl "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvl var-0))
          ret-val)))
  (setf info -1))
 ((and (not wantvr)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame jobvr "N")
          (declare (ignore var-1))
          (when var-0 (setf jobvr var-0))
          ret-val)))
  (setf info -2))
 ((< n 0) (setf info -3))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -5))
 ((or (< ldvl 1) (and wantvl (< ldvl n))) (setf info -8))
 ((or (< ldvr 1) (and wantvr (< ldvr n))) (setf info -10)))
(cond
 ((= info 0)
  (cond ((= n 0) (setf minwrk 1) (setf maxwrk 1))
        (t
         (setf maxwrk
                 (f2cl-lib:int-add n
                                   (f2cl-lib:int-mul n
                                                     (multiple-value-bind
                                                         (ret-val var-0
                                                          var-1 var-2
                                                          var-3 var-4
                                                          var-5 var-6)
                                                         (ilaenv 1
                                                          "ZGEHRD" " "
                                                          n 1 n 0)
                                                       (declare
```

```
                                                  (ignore var-0
                                                   var-1 var-2
                                                   var-4 var-6))
                                                 (setf n var-3)
                                                 (setf n var-5)
                                                 ret-val))))
(setf minwrk (f2cl-lib:int-mul 2 n))
(cond
 (wantvl
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add n
                                      (f2cl-lib:int-mul
                                       (f2cl-lib:int-sub n 1)
                                       (multiple-value-bind
                                           (ret-val var-0
                                            var-1 var-2 var-3
                                            var-4 var-5 var-6)
                                           (ilaenv 1 "ZUNGHR"
                                            " " n 1 n -1)
                                         (declare
                                          (ignore var-0 var-1
                                           var-2 var-4 var-6))
                                         (setf n var-3)
                                         (setf n var-5)
                                         ret-val))))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zhseqr "S" "V" n 1 n a lda w vl ldvl work -1 info)
    (declare
     (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10
      var-11))
    (when var-2 (setf n var-2))
    (when var-4 (setf n var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldvl var-9))
    (when var-12 (setf info var-12))))
 (wantvr
  (setf maxwrk
          (max (the f2cl-lib:integer4 maxwrk)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add n
                                      (f2cl-lib:int-mul
                                       (f2cl-lib:int-sub n 1)
                                       (multiple-value-bind
                                           (ret-val var-0
                                            var-1 var-2 var-3
                                            var-4 var-5 var-6)
```

```
                                                        (ilaenv 1 "ZUNGHR"
                                                         " " n 1 n -1)
                                                      (declare
                                                       (ignore var-0 var-1
                                                        var-2 var-4 var-6))
                                                      (setf n var-3)
                                                      (setf n var-5)
                                                      ret-val))))))
                 (multiple-value-bind
                     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                      var-9 var-10 var-11 var-12)
                     (zhseqr "S" "V" n 1 n a lda w vr ldvr work -1 info)
                   (declare
                    (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10
                     var-11))
                   (when var-2 (setf n var-2))
                   (when var-4 (setf n var-4))
                   (when var-6 (setf lda var-6))
                   (when var-9 (setf ldvr var-9))
                   (when var-12 (setf info var-12))))
                 (t
                  (multiple-value-bind
                      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                       var-9 var-10 var-11 var-12)
                      (zhseqr "E" "N" n 1 n a lda w vr ldvr work -1 info)
                   (declare
                    (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10
                     var-11))
                   (when var-2 (setf n var-2))
                   (when var-4 (setf n var-4))
                   (when var-6 (setf lda var-6))
                   (when var-9 (setf ldvr var-9))
                   (when var-12 (setf info var-12)))))
                (setf hswork
                        (f2cl-lib:int
                         (f2cl-lib:fref work-%data% (1) ((1 *))
                                        work-%offset%)))
                (setf maxwrk
                        (max (the f2cl-lib:integer4 maxwrk)
                             (the f2cl-lib:integer4 hswork)
                             (the f2cl-lib:integer4 minwrk)))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce maxwrk 'f2cl-lib:complex16))
  (cond ((and (< lwork minwrk) (not lquery)) (setf info -12)))))
(cond
 ((/= info 0) (xerbla "ZGEEV " (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(if (= n 0)
    (go end_label))
(setf eps (dlamch "P"))
```

```
(setf smlnum (dlamch "S"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ (f2cl-lib:fsqrt smlnum) eps))
(setf bignum (/ one smlnum))
(setf anrm
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5)
            (zlange "M" n n a lda dum)
          (declare (ignore var-0 var-3 var-5))
          (when var-1 (setf n var-1))
          (when var-2 (setf n var-2))
          (when var-4 (setf lda var-4))
          ret-val))
(setf scalea f2cl-lib:%false%)
(cond
 ((and (> anrm zero) (< anrm smlnum)) (setf scalea f2cl-lib:%true%)
  (setf cscale smlnum))
 ((> anrm bignum) (setf scalea f2cl-lib:%true%) (setf cscale bignum)))
(if scalea
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
        (zlascl "G" 0 0 anrm cscale n n a lda ierr)
      (declare (ignore var-0 var-1 var-2 var-7))
      (when var-3 (setf anrm var-3))
      (when var-4 (setf cscale var-4))
      (when var-5 (setf n var-5))
      (when var-6 (setf n var-6))
      (when var-8 (setf lda var-8))
      (when var-9 (setf ierr var-9))))
(setf ibal 1)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (zgebal "B" n a lda ilo ihi
     (f2cl-lib:array-slice rwork-%data% double-float (ibal) ((1 *))
                           rwork-%offset%)
     ierr)
  (declare (ignore var-0 var-1 var-2 var-6))
  (setf lda var-3)
  (setf ilo var-4)
  (setf ihi var-5)
  (setf ierr var-7))
(setf itau 1)
(setf iwrk (f2cl-lib:int-add itau n))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zgehrd n ilo ihi a lda
```

```
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
  (declare (ignore var-3 var-5 var-6 var-7))
  (when var-0 (setf n var-0))
  (when var-1 (setf ilo var-1))
  (when var-2 (setf ihi var-2))
  (when var-4 (setf lda var-4))
  (when var-8 (setf ierr var-8)))
(cond
 (wantvl (f2cl-lib:f2cl-set-string side "L" (string 1))
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
      (zlacpy "L" n n a lda vl ldvl)
    (declare (ignore var-0 var-3 var-5))
    (when var-1 (setf n var-1))
    (when var-2 (setf n var-2))
    (when var-4 (setf lda var-4))
    (when var-6 (setf ldvl var-6)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zunghr n ilo ihi vl ldvl
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
    (declare (ignore var-3 var-5 var-6 var-7))
    (when var-0 (setf n var-0))
    (when var-1 (setf ilo var-1))
    (when var-2 (setf ihi var-2))
    (when var-4 (setf ldvl var-4))
    (when var-8 (setf ierr var-8)))
  (setf iwrk itau)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (zhseqr "S" "V" n ilo ihi a lda w vl ldvl
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                             ((1 *)) work-%offset%)
       (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldvl var-9))
    (when var-12 (setf info var-12)))
  (cond
```

```
  (wantvr (f2cl-lib:f2cl-set-string side "B" (string 1))
   (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
       (zlacpy "F" n n vl ldvl vr ldvr)
     (declare (ignore var-0 var-3 var-5))
     (when var-1 (setf n var-1))
     (when var-2 (setf n var-2))
     (when var-4 (setf ldvl var-4))
     (when var-6 (setf ldvr var-6))))))
(wantvr (f2cl-lib:f2cl-set-string side "R" (string 1))
 (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
     (zlacpy "L" n n a lda vr ldvr)
   (declare (ignore var-0 var-3 var-5))
   (when var-1 (setf n var-1))
   (when var-2 (setf n var-2))
   (when var-4 (setf lda var-4))
   (when var-6 (setf ldvr var-6)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
     (zunghr n ilo ihi vr ldvr
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (itau)
                            ((1 *)) work-%offset%)
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) ierr)
   (declare (ignore var-3 var-5 var-6 var-7))
   (when var-0 (setf n var-0))
   (when var-1 (setf ilo var-1))
   (when var-2 (setf ihi var-2))
   (when var-4 (setf ldvr var-4))
   (when var-8 (setf ierr var-8)))
 (setf iwrk itau)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12)
     (zhseqr "S" "V" n ilo ihi a lda w vr ldvr
      (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                            ((1 *)) work-%offset%)
      (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
   (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
   (when var-2 (setf n var-2))
   (when var-3 (setf ilo var-3))
   (when var-4 (setf ihi var-4))
   (when var-6 (setf lda var-6))
   (when var-9 (setf ldvr var-9))
   (when var-12 (setf info var-12))))
(t (setf iwrk itau)
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10 var-11 var-12)
     (zhseqr "E" "N" n ilo ihi a lda w vr ldvr
```

```
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                              ((1 *)) work-%offset%)
        (f2cl-lib:int-add (f2cl-lib:int-sub lwork iwrk) 1) info)
    (declare (ignore var-0 var-1 var-5 var-7 var-8 var-10 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldvr var-9))
    (when var-12 (setf info var-12)))))
(if (> info 0)
    (go label50))
(cond
 ((or wantvl wantvr) (setf irwork (f2cl-lib:int-add ibal n))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (ztrevc side "B" select n a lda vl ldvl vr ldvr n nout
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwrk)
                             ((1 *)) work-%offset%)
       (f2cl-lib:array-slice rwork-%data% double-float (irwork) ((1 *))
                             rwork-%offset%)
       ierr)
    (declare (ignore var-1 var-2 var-4 var-6 var-8 var-12 var-13))
    (when var-0 (setf side var-0))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))
    (when var-7 (setf ldvl var-7))
    (when var-9 (setf ldvr var-9))
    (when var-10 (setf n var-10))
    (when var-11 (setf nout var-11))
    (when var-14 (setf ierr var-14)))))
(cond
 (wantvl
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zgebak "B" "L" n ilo ihi
       (f2cl-lib:array-slice rwork-%data% double-float (ibal) ((1 *))
                             rwork-%offset%)
       n vl ldvl ierr)
    (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7))
    (setf n var-6)
    (setf ldvl var-8)
    (setf ierr var-9))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf scl
              (/ one
                 (multiple-value-bind (ret-val var-0 var-1 var-2)
```

```
                (dznrm2 n
                 (f2cl-lib:array-slice vl-%data%
                                       f2cl-lib:complex16 (1 i)
                                       ((1 ldvl) (1 *))
                                       vl-%offset%)
                 1)
               (declare (ignore var-1 var-2))
               (when var-0 (setf n var-0))
               ret-val)))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (zdscal n scl
     (f2cl-lib:array-slice vl-%data% f2cl-lib:complex16 (1 i)
                           ((1 ldvl) (1 *)) vl-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf scl var-1)))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add irwork k) 1))
                         ((1 *)) rwork-%offset%)
            (+
             (expt
              (f2cl-lib:dble
               (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                              vl-%offset%))
              2)
             (expt
              (f2cl-lib:dimag
               (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                              vl-%offset%))
              2)))
   label10))
(setf k
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (idamax n
             (f2cl-lib:array-slice rwork-%data% double-float
                                   (irwork) ((1 *))
                                   rwork-%offset%)
             1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(setf tmp
        (coerce
         (/
          (f2cl-lib:dconjg
```

```
                    (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                                   vl-%offset%))
                   (f2cl-lib:fsqrt
                    (f2cl-lib:fref rwork-%data%
                                   ((f2cl-lib:int-sub
                                     (f2cl-lib:int-add irwork k) 1))
                                   ((1 *)) rwork-%offset%)))
                 'f2cl-lib:complex16))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (zscal n tmp
             (f2cl-lib:array-slice vl-%data% f2cl-lib:complex16 (1 i)
                                   ((1 ldvl) (1 *)) vl-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-1 (setf tmp var-1)))
        (setf (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                             vl-%offset%)
                (f2cl-lib:dcmplx
                 (f2cl-lib:dble
                  (f2cl-lib:fref vl-%data% (k i) ((1 ldvl) (1 *))
                                 vl-%offset%))
                 zero))
      label20))))
(cond
 (wantvr
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
      (zgebak "B" "R" n ilo ihi
       (f2cl-lib:array-slice rwork-%data% double-float (ibal) ((1 *))
                             rwork-%offset%)
       n vr ldvr ierr)
    (declare (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7))
    (setf n var-6)
    (setf ldvr var-8)
    (setf ierr var-9))
  (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                ((> i n) nil)
    (tagbody
      (setf scl
              (/ one
                 (multiple-value-bind (ret-val var-0 var-1 var-2)
                     (dznrm2 n
                      (f2cl-lib:array-slice vr-%data%
                                            f2cl-lib:complex16 (1 i)
                                            ((1 ldvr) (1 *))
                                            vr-%offset%)
                      1)
                   (declare (ignore var-1 var-2))
                   (when var-0 (setf n var-0))
```

```
             ret-val)))
(multiple-value-bind (var-0 var-1 var-2 var-3)
    (zdscal n scl
     (f2cl-lib:array-slice vr-%data% f2cl-lib:complex16 (1 i)
                           ((1 ldvr) (1 *)) vr-%offset%)
     1)
  (declare (ignore var-2 var-3))
  (when var-0 (setf n var-0))
  (when var-1 (setf scl var-1)))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k n) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add irwork k) 1))
                         ((1 *)) rwork-%offset%)
            (+
             (expt
              (f2cl-lib:dble
               (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                              vr-%offset%))
              2)
             (expt
              (f2cl-lib:dimag
               (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                              vr-%offset%))
              2)))
   label30))
(setf k
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (idamax n
             (f2cl-lib:array-slice rwork-%data% double-float
                                   (irwork) ((1 *))
                                   rwork-%offset%)
             1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(setf tmp
        (coerce
         (/
          (f2cl-lib:dconjg
           (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                          vr-%offset%))
          (f2cl-lib:fsqrt
           (f2cl-lib:fref rwork-%data%
                          ((f2cl-lib:int-sub
                            (f2cl-lib:int-add irwork k) 1))
                          ((1 *)) rwork-%offset%)))
         'f2cl-lib:complex16))
```

```
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (zscal n tmp
             (f2cl-lib:array-slice vr-%data% f2cl-lib:complex16 (1 i)
                                   ((1 ldvr) (1 *)) vr-%offset%)
             1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-1 (setf tmp var-1)))
        (setf (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                             vr-%offset%)
                (f2cl-lib:dcmplx
                 (f2cl-lib:dble
                  (f2cl-lib:fref vr-%data% (k i) ((1 ldvr) (1 *))
                                 vr-%offset%))
                 zero))
      label40))))
label50
 (cond
  (scalea
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
       (zlascl "G" 0 0 cscale anrm (f2cl-lib:int-sub n info) 1
        (f2cl-lib:array-slice w-%data% f2cl-lib:complex16 ((+ info 1))
                              ((1 *)) w-%offset%)
        (max (the f2cl-lib:integer4 (f2cl-lib:int-sub n info))
             (the f2cl-lib:integer4 1))
        ierr)
     (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7 var-8))
     (when var-3 (setf cscale var-3))
     (when var-4 (setf anrm var-4))
     (when var-9 (setf ierr var-9)))
   (cond
    ((> info 0)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9)
         (zlascl "G" 0 0 cscale anrm (f2cl-lib:int-sub ilo 1) 1 w n
          ierr)
       (declare (ignore var-0 var-1 var-2 var-5 var-6 var-7))
       (when var-3 (setf cscale var-3))
       (when var-4 (setf anrm var-4))
       (when var-8 (setf n var-8))
       (when var-9 (setf ierr var-9)))))))
 (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
         (coerce maxwrk 'f2cl-lib:complex16))
 (go end_label)
end_label
 (return
  (values jobvl jobvr n nil lda nil nil ldvl nil ldvr nil nil nil
          info))))))
```

———-

90.215 zgehd2

90.215.1 Axiom unit tests

— **zgehd2.input** —

```
)set break resume
)sys rm -f zgehd2.output
)spool zgehd2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.215.2 Axiom help page

— **zgehd2.help** —

```
======================================================================
zgehd2 examples
======================================================================

======================================================================
Man Page Details
======================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZGEHD2( N, ILO, IHI, A, LDA, TAU, WORK, INFO )

       .. Scalar Arguments ..
       INTEGER            IHI, ILO, INFO, LDA, N
       ..
       .. Array Arguments ..
```

```
      COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
      ..


 Purpose:
 =============

  ZGEHD2 reduces a complex general matrix A to upper Hessenberg form H
  by a unitary similarity transformation:  Q**H * A * Q = H .

 Arguments:
 ==========

  [in] N

          N is INTEGER
          The order of the matrix A.  N >= 0.

  [in] ILO

          ILO is INTEGER

  [in] IHI

          IHI is INTEGER

          It is assumed that A is already upper triangular in rows
          and columns 1:ILO-1 and IHI+1:N. ILO and IHI are normally
          set by a previous call to ZGEBAL; otherwise they should be
          set to 1 and N respectively. See Further Details.
          1 <= ILO <= IHI <= max(1,N).

  [in,out] A

          A is COMPLEX*16 array, dimension (LDA,N)
          On entry, the n by n general matrix to be reduced.
          On exit, the upper triangle and the first subdiagonal of A
          are overwritten with the upper Hessenberg matrix H, and the
          elements below the first subdiagonal, with the array TAU,
          represent the unitary matrix Q as a product of elementary
          reflectors. See Further Details.

  [in] LDA

          LDA is INTEGER
          The leading dimension of the array A.  LDA >= max(1,N).

  [out] TAU

          TAU is COMPLEX*16 array, dimension (N-1)
```

```
         The scalar factors of the elementary reflectors (see Further
         Details).

 [out] WORK

         WORK is COMPLEX*16 array, dimension (N)

 [out] INFO

         INFO is INTEGER
         = 0:  successful exit
         < 0:  if INFO = -i, the i-th argument had an illegal value.

Further Details:
=====================

 The matrix Q is represented as a product of (ihi-ilo) elementary
 reflectors

    Q = H(ilo) H(ilo+1) . . . H(ihi-1).

 Each H(i) has the form

    H(i) = I - tau * v * v**H

 where tau is a complex scalar, and v is a complex vector with
 v(1:i) = 0, v(i+1) = 1 and v(ihi+1:n) = 0; v(i+2:ihi) is stored on
 exit in A(i+2:ihi,i), and tau in TAU(i).

 The contents of A are illustrated by the following example, with
 n = 7, ilo = 2 and ihi = 6:

 on entry,                        on exit,

 ( a   a   a   a   a   a   a )    (  a   a   h   h   h   h   a )
 (     a   a   a   a   a   a )    (      a   h   h   h   h   a )
 (     a   a   a   a   a   a )    (      h   h   h   h   h   h )
 (     a   a   a   a   a   a )    (      v2  h   h   h   h   h )
 (     a   a   a   a   a   a )    (      v2  v3  h   h   h   h )
 (     a   a   a   a   a   a )    (      v2  v3  v4  h   h   h )
 (                         a )    (                          a )

 where a denotes an element of the original matrix A, h denotes a
 modified element of the upper Hessenberg matrix H, and vi denotes an
 element of the vector defining H(i).
```

90.215.3 fortran code

— zgehd2.f —

```
*  =====================================================================
      SUBROUTINE ZGEHD2( N, ILO, IHI, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            IHI, ILO, INFO, LDA, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      COMPLEX*16         ALPHA
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLARF, ZLARFG
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DCONJG, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input parameters
*
      INFO = 0
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -2
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      END IF
```

```
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZGEHD2', -INFO )
         RETURN
      END IF
*
      DO 10 I = ILO, IHI - 1
*
*        Compute elementary reflector H(i) to annihilate A(i+2:ihi,i)
*
         ALPHA = A( I+1, I )
         CALL ZLARFG( IHI-I, ALPHA, A( MIN( I+2, N ), I ), 1, TAU( I ) )
         A( I+1, I ) = ONE
*
*        Apply H(i) to A(1:ihi,i+1:ihi) from the right
*
         CALL ZLARF( 'Right', IHI, IHI-I, A( I+1, I ), 1, TAU( I ),
     $               A( 1, I+1 ), LDA, WORK )
*
*        Apply H(i)**H to A(i+1:ihi,i+1:n) from the left
*
         CALL ZLARF( 'Left', IHI-I, N-I, A( I+1, I ), 1,
     $               DCONJG( TAU( I ) ), A( I+1, I+1 ), LDA, WORK )
*
         A( I+1, I ) = ALPHA
   10 CONTINUE
*
      RETURN
*
*     End of ZGEHD2
*
      END
```

———-

90.215.4 lisp code

— **zgehd2.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (ignorable one))
  (defun zgehd2 (n ilo ihi a lda tau work info)
    (declare (type (array f2cl-lib:complex16 (*)) work tau a)
             (type (f2cl-lib:integer4) info lda ihi ilo n))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
```

```
    (work f2cl-lib:complex16 work-%data% work-%offset%))
  (prog ((alpha #C(0.0d0 0.0d0)) (i 0))
    (declare (type (f2cl-lib:complex16) alpha)
             (type (f2cl-lib:integer4) i))
    (setf info 0)
    (cond ((< n 0) (setf info -1))
          ((or (< ilo 1)
               (> ilo
                  (max (the f2cl-lib:integer4 1)
                       (the f2cl-lib:integer4 n))))
           (setf info -2))
          ((or
            (< ihi
               (min (the f2cl-lib:integer4 ilo) (the f2cl-lib:integer4 n)))
            (> ihi n))
           (setf info -3))
          ((< lda
              (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
           (setf info -5)))
    (cond
     ((/= info 0) (xerbla "ZGEHD2" (f2cl-lib:int-sub info))
      (go end_label)))
    (f2cl-lib:fdo (i ilo (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add ihi (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (setf alpha
                (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                               ((1 lda) (1 *)) a-%offset%))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
            (zlarfg (f2cl-lib:int-sub ihi i) alpha
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   ((min (f2cl-lib:int-add i 2) n) i)
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                                   ((1 *)) tau-%offset%))
          (declare (ignore var-0 var-2 var-3 var-4))
          (when var-1 (setf alpha var-1)))
        (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                             ((1 lda) (1 *)) a-%offset%)
              one)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (zlarf "Right" ihi (f2cl-lib:int-sub ihi i)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 ((+ i 1) i)
                                   ((1 lda) (1 *)) a-%offset%)
             1
             (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                                   ((1 *)) tau-%offset%)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
```

```
                                  (1 (f2cl-lib:int-add i 1))
                                  ((1 lda) (1 *)) a-%offset%)
                 lda work)
              (declare (ignore var-0 var-2 var-3 var-4 var-5 var-6 var-8))
              (when var-1 (setf ihi var-1))
              (when var-7 (setf lda var-7)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (zlarf "Left" (f2cl-lib:int-sub ihi i) (f2cl-lib:int-sub n i)
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 ((+ i 1) i)
                                  ((1 lda) (1 *)) a-%offset%)
                 1
                 (f2cl-lib:dconjg
                  (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                  ((+ i 1) (f2cl-lib:int-add i 1))
                                  ((1 lda) (1 *)) a-%offset%)
                 lda work)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
              (when var-7 (setf lda var-7)))
            (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add i 1) i)
                                 ((1 lda) (1 *)) a-%offset%)
                    alpha)
          label10))
        (go end_label)
      end_label
        (return (values nil nil ihi nil lda nil nil info))))))
```

———-

90.216 zhseqr

90.216.1 Axiom unit tests

— **zhseqr.input** —

```
)set break resume
)sys rm -f zhseqr.output
)spool zhseqr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.216.2 Axiom help page

See Braman [Bram02a, Bram02b]

— **zhseqr.help** —

```
==========================================================================
zhseqr examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZHSEQR( JOB, COMPZ, N, ILO, IHI, H, LDH, W, Z, LDZ,
                          WORK, LWORK, INFO )

       .. Scalar Arguments ..
       INTEGER            IHI, ILO, INFO, LDH, LDZ, LWORK, N
       CHARACTER          COMPZ, JOB
       ..
       .. Array Arguments ..
       COMPLEX*16         H( LDH, * ), W( * ), WORK( * ), Z( LDZ, * )
       ..


 Purpose:
 =============

    ZHSEQR computes the eigenvalues of a Hessenberg matrix H
    and, optionally, the matrices T and Z from the Schur decomposition
    H = Z T Z**H, where T is an upper triangular matrix (the
    Schur form), and Z is the unitary matrix of Schur vectors.

    Optionally Z may be postmultiplied into an input unitary
    matrix Q so that this routine can give the Schur factorization
    of a matrix A which has been reduced to the Hessenberg form H
    by the unitary matrix Q:  A = Q*H*Q**H = (QZ)*H*(QZ)**H.


 Arguments:
 ==========
```

```
[in] JOB

        JOB is CHARACTER*1
         = 'E':  compute eigenvalues only;
         = 'S':  compute eigenvalues and the Schur form T.

[in] COMPZ

        COMPZ is CHARACTER*1
         = 'N':  no Schur vectors are computed;
         = 'I':  Z is initialized to the unit matrix and the matrix Z
                 of Schur vectors of H is returned;
         = 'V':  Z must contain an unitary matrix Q on entry, and
                 the product Q*Z is returned.

[in] N

        N is INTEGER
         The order of the matrix H.  N .GE. 0.

[in] ILO

        ILO is INTEGER

[in] IHI

        IHI is INTEGER

         It is assumed that H is already upper triangular in rows
         and columns 1:ILO-1 and IHI+1:N. ILO and IHI are normally
         set by a previous call to ZGEBAL, and then passed to ZGEHRD
         when the matrix output by ZGEBAL is reduced to Hessenberg
         form. Otherwise ILO and IHI should be set to 1 and N
         respectively.  If N.GT.0, then 1.LE.ILO.LE.IHI.LE.N.
         If N = 0, then ILO = 1 and IHI = 0.

[in,out] H

        H is COMPLEX*16 array, dimension (LDH,N)
         On entry, the upper Hessenberg matrix H.
         On exit, if INFO = 0 and JOB = 'S', H contains the upper
         triangular matrix T from the Schur decomposition (the
         Schur form). If INFO = 0 and JOB = 'E', the contents of
         H are unspecified on exit.  (The output value of H when
         INFO.GT.0 is given under the description of INFO below.)

         Unlike earlier versions of ZHSEQR, this subroutine may
         explicitly H(i,j) = 0 for i.GT.j and j = 1, 2, ... ILO-1
         or j = IHI+1, IHI+2, ... N.
```

```
[in] LDH

        LDH is INTEGER
         The leading dimension of the array H. LDH .GE. max(1,N).

[out] W

        W is COMPLEX*16 array, dimension (N)
         The computed eigenvalues. If JOB = 'S', the eigenvalues are
         stored in the same order as on the diagonal of the Schur
         form returned in H, with W(i) = H(i,i).

[in,out] Z

        Z is COMPLEX*16 array, dimension (LDZ,N)
         If COMPZ = 'N', Z is not referenced.
         If COMPZ = 'I', on entry Z need not be set and on exit,
         if INFO = 0, Z contains the unitary matrix Z of the Schur
         vectors of H.  If COMPZ = 'V', on entry Z must contain an
         N-by-N matrix Q, which is assumed to be equal to the unit
         matrix except for the submatrix Z(ILO:IHI,ILO:IHI). On exit,
         if INFO = 0, Z contains Q*Z.
         Normally Q is the unitary matrix generated by ZUNGHR
         after the call to ZGEHRD which formed the Hessenberg matrix
         H. (The output value of Z when INFO.GT.0 is given under
         the description of INFO below.)

[in] LDZ

        LDZ is INTEGER
         The leading dimension of the array Z.  if COMPZ = 'I' or
         COMPZ = 'V', then LDZ.GE.MAX(1,N).  Otherwize, LDZ.GE.1.

[out] WORK

        WORK is COMPLEX*16 array, dimension (LWORK)
         On exit, if INFO = 0, WORK(1) returns an estimate of
         the optimal value for LWORK.

[in] LWORK

        LWORK is INTEGER
         The dimension of the array WORK.  LWORK .GE. max(1,N)
         is sufficient and delivers very good and sometimes
         optimal performance.  However, LWORK as large as 11*N
         may be required for optimal performance.  A workspace
         query is recommended to determine the optimal workspace
         size.
```

```
          If LWORK = -1, then ZHSEQR does a workspace query.
          In this case, ZHSEQR checks the input parameters and
          estimates the optimal workspace size for the given
          values of N, ILO and IHI.  The estimate is returned
          in WORK(1).  No error message related to LWORK is
          issued by XERBLA.  Neither H nor Z are accessed.

 [out] INFO

         INFO is INTEGER
            =  0:  successful exit
          .LT. 0:  if INFO = -i, the i-th argument had an illegal
                   value
          .GT. 0:  if INFO = i, ZHSEQR failed to compute all of
               the eigenvalues.  Elements 1:ilo-1 and i+1:n of WR
               and WI contain those eigenvalues which have been
               successfully computed.  (Failures are rare.)

               If INFO .GT. 0 and JOB = 'E', then on exit, the
               remaining unconverged eigenvalues are the eigen-
               values of the upper Hessenberg matrix rows and
               columns ILO through INFO of the final, output
               value of H.

               If INFO .GT. 0 and JOB   = 'S', then on exit

          (*)  (initial value of H)*U  = U*(final value of H)

               where U is a unitary matrix.  The final
               value of  H is upper Hessenberg and triangular in
               rows and columns INFO+1 through IHI.

               If INFO .GT. 0 and COMPZ = 'V', then on exit

                 (final value of Z)  =  (initial value of Z)*U

               where U is the unitary matrix in (*) (regard-
               less of the value of JOB.)

               If INFO .GT. 0 and COMPZ = 'I', then on exit
                     (final value of Z)  = U
               where U is the unitary matrix in (*) (regard-
               less of the value of JOB.)

               If INFO .GT. 0 and COMPZ = 'N', then Z is not
               accessed.


Further Details:
=====================
```

```
 Default values supplied by
 ILAENV(ISPEC,'ZHSEQR',JOB(:1)//COMPZ(:1),N,ILO,IHI,LWORK).
 It is suggested that these defaults be adjusted in order
 to attain best performance in each particular
 computational environment.

ISPEC=12: The ZLAHQR vs ZLAQR0 crossover point.
          Default: 75. (Must be at least 11.)

ISPEC=13: Recommended deflation window size.
          This depends on ILO, IHI and NS.  NS is the
          number of simultaneous shifts returned
          by ILAENV(ISPEC=15).  (See ISPEC=15 below.)
          The default for (IHI-ILO+1).LE.500 is NS.
          The default for (IHI-ILO+1).GT.500 is 3*NS/2.

ISPEC=14: Nibble crossover point. (See IPARMQ for
          details.)  Default: 14% of deflation window
          size.

ISPEC=15: Number of simultaneous shifts in a multishift
          QR iteration.

          If IHI-ILO+1 is ...

          greater than      ...but less    ... the
          or equal to ...      than        default is

               1               30          NS =   2(+)
              30               60          NS =   4(+)
              60              150          NS =  10(+)
             150              590          NS =  **
             590             3000          NS =  64
            3000             6000          NS = 128
            6000             infinity      NS = 256

      (+)  By default some or all matrices of this order
           are passed to the implicit double shift routine
           ZLAHQR and this parameter is ignored.  See
           ISPEC=12 above and comments in IPARMQ for
           details.

     (**)  The asterisks (**) indicate an ad-hoc
           function of N increasing from 10 to 64.

ISPEC=16: Select structured matrix multiply.
          If the number of simultaneous shifts (specified
          by ISPEC=15) is less than 14, then the default
          for ISPEC=16 is 0.  Otherwise the default for
```

```
                    ISPEC=16 is 2.
```

———-

90.216.3 fortran code

— **zhseqr.f** —

```
*  =====================================================================
      SUBROUTINE ZHSEQR( JOB, COMPZ, N, ILO, IHI, H, LDH, W, Z, LDZ,
     $                   WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            IHI, ILO, INFO, LDH, LDZ, LWORK, N
      CHARACTER          COMPZ, JOB
*     ..
*     .. Array Arguments ..
      COMPLEX*16         H( LDH, * ), W( * ), WORK( * ), Z( LDZ, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
*
*     ==== Matrices of order NTINY or smaller must be processed by
*     .    ZLAHQR because of insufficient subdiagonal scratch space.
*     .    (This is a hard limit.) ====
      INTEGER            NTINY
      PARAMETER          ( NTINY = 11 )
*
*     ==== NL allocates some local workspace to help small matrices
*     .    through a rare ZLAHQR failure.  NL .GT. NTINY = 11 is
*     .    required and NL .LE. NMIN = ILAENV(ISPEC=12,...) is recom-
*     .    mended.  (The default value of NMIN is 75.)  Using NL = 49
*     .    allows up to six simultaneous shifts and a 16-by-16
*     .    deflation window.  ====
      INTEGER            NL
      PARAMETER          ( NL = 49 )
      COMPLEX*16         ZERO, ONE
      PARAMETER          ( ZERO = ( 0.0d0, 0.0d0 ),
     $                   ONE = ( 1.0d0, 0.0d0 ) )
      DOUBLE PRECISION   RZERO
```

```
      PARAMETER          ( RZERO = 0.0d0 )
*     ..
*     .. Local Arrays ..
      COMPLEX*16         HL( NL, NL ), WORKL( NL )
*     ..
*     .. Local Scalars ..
      INTEGER            KBOT, NMIN
      LOGICAL            INITZ, LQUERY, WANTT, WANTZ
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      LOGICAL            LSAME
      EXTERNAL           ILAENV, LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZCOPY, ZLACPY, ZLAHQR, ZLAQR0, ZLASET
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, DCMPLX, MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     ==== Decode and check the input parameters. ====
*
      WANTT = LSAME( JOB, 'S' )
      INITZ = LSAME( COMPZ, 'I' )
      WANTZ = INITZ .OR. LSAME( COMPZ, 'V' )
      WORK( 1 ) = DCMPLX( DBLE( MAX( 1, N ) ), RZERO )
      LQUERY = LWORK.EQ.-1
*
      INFO = 0
      IF( .NOT.LSAME( JOB, 'E' ) .AND. .NOT.WANTT ) THEN
         INFO = -1
      ELSE IF( .NOT.LSAME( COMPZ, 'N' ) .AND. .NOT.WANTZ ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -3
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -5
      ELSE IF( LDH.LT.MAX( 1, N ) ) THEN
         INFO = -7
      ELSE IF( LDZ.LT.1 .OR. ( WANTZ .AND. LDZ.LT.MAX( 1, N ) ) ) THEN
         INFO = -10
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -12
      END IF
*
      IF( INFO.NE.0 ) THEN
```

```
*
*        ==== Quick return in case of invalid argument. ====
*
         CALL XERBLA( 'ZHSEQR', -INFO )
         RETURN
*
      ELSE IF( N.EQ.0 ) THEN
*
*        ==== Quick return in case N = 0; nothing to do. ====
*
         RETURN
*
      ELSE IF( LQUERY ) THEN
*
*        ==== Quick return in case of a workspace query ====
*
         CALL ZLAQR0( WANTT, WANTZ, N, ILO, IHI, H, LDH, W, ILO, IHI, Z,
     $               LDZ, WORK, LWORK, INFO )
*        ==== Ensure reported workspace size is backward-compatible with
*        .    previous LAPACK versions. ====
         WORK( 1 ) = DCMPLX( MAX( DBLE( WORK( 1 ) ), DBLE( MAX( 1,
     $              N ) ) ), RZERO )
         RETURN
*
      ELSE
*
*        ==== copy eigenvalues isolated by ZGEBAL ====
*
         IF( ILO.GT.1 )
     $      CALL ZCOPY( ILO-1, H, LDH+1, W, 1 )
         IF( IHI.LT.N )
     $      CALL ZCOPY( N-IHI, H( IHI+1, IHI+1 ), LDH+1, W( IHI+1 ), 1 )
*
*        ==== Initialize Z, if requested ====
*
         IF( INITZ )
     $      CALL ZLASET( 'A', N, N, ZERO, ONE, Z, LDZ )
*
*        ==== Quick return if possible ====
*
         IF( ILO.EQ.IHI ) THEN
            W( ILO ) = H( ILO, ILO )
            RETURN
         END IF
*
*        ==== ZLAHQR/ZLAQR0 crossover point ====
*
         NMIN = ILAENV( 12, 'ZHSEQR', JOB // COMPZ, N,
     $          ILO, IHI, LWORK )
         NMIN = MAX( NTINY, NMIN )
```

```
*
*        ==== ZLAQR0 for big matrices; ZLAHQR for small ones ====
*
         IF( N.GT.NMIN ) THEN
            CALL ZLAQR0( WANTT, WANTZ, N, ILO, IHI, H, LDH, W, ILO, IHI,
     $                   Z, LDZ, WORK, LWORK, INFO )
         ELSE
*
*           ==== Small matrix ====
*
            CALL ZLAHQR( WANTT, WANTZ, N, ILO, IHI, H, LDH, W, ILO, IHI,
     $                   Z, LDZ, INFO )
*
            IF( INFO.GT.0 ) THEN
*
*              ==== A rare ZLAHQR failure!  ZLAQR0 sometimes succeeds
*              .    when ZLAHQR fails. ====
*
               KBOT = INFO
*
               IF( N.GE.NL ) THEN
*
*                 ==== Larger matrices have enough subdiagonal scratch
*                 .    space to call ZLAQR0 directly. ====
*
                  CALL ZLAQR0( WANTT, WANTZ, N, ILO, KBOT, H, LDH, W,
     $                         ILO, IHI, Z, LDZ, WORK, LWORK, INFO )
*
               ELSE
*
*                 ==== Tiny matrices don't have enough subdiagonal
*                 .    scratch space to benefit from ZLAQR0.  Hence,
*                 .    tiny matrices must be copied into a larger
*                 .    array before calling ZLAQR0. ====
*
                  CALL ZLACPY( 'A', N, N, H, LDH, HL, NL )
                  HL( N+1, N ) = ZERO
                  CALL ZLASET( 'A', NL, NL-N, ZERO, ZERO, HL( 1, N+1 ),
     $                         NL )
                  CALL ZLAQR0( WANTT, WANTZ, NL, ILO, KBOT, HL, NL, W,
     $                         ILO, IHI, Z, LDZ, WORKL, NL, INFO )
                  IF( WANTT .OR. INFO.NE.0 )
     $               CALL ZLACPY( 'A', N, N, HL, NL, H, LDH )
               END IF
            END IF
         END IF
*
*        ==== Clear out the trash, if necessary. ====
*
         IF( ( WANTT .OR. INFO.NE.0 ) .AND. N.GT.2 )
```

```
     $      CALL ZLASET( 'L', N-2, N-2, ZERO, ZERO, H( 3, 1 ), LDH )
*
*        ==== Ensure reported workspace size is backward-compatible with
*        .    previous LAPACK versions. ====
*
         WORK( 1 ) = DCMPLX( MAX( DBLE( MAX( 1, N ) ),
     $               DBLE( WORK( 1 ) ) ), RZERO )
      END IF
*
*     ==== End of ZHSEQR ====
*
      END
```

———-

90.216.4 lisp code

— **zhseqr.lisp** —

```
(let* ((ntiny 11)
       (nl 49)
       (zero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (one (f2cl-lib:cmplx 1.0d0 0.0d0))
       (rzero 0.0d0))
  (declare (type (f2cl-lib:integer4 11 11) ntiny)
           (type (f2cl-lib:integer4 49 49) nl)
           (type (f2cl-lib:complex16) zero)
           (type (f2cl-lib:complex16) one)
           (type (double-float 0.0d0 0.0d0) rzero)
           (ignorable ntiny nl zero one rzero))
  (defun zhseqr (job compz n ilo ihi h ldh w z ldz work lwork info)
    (declare (type (array f2cl-lib:complex16 (*)) work z w h)
             (type (f2cl-lib:integer4) info lwork ldz ldh ihi ilo n)
             (type (string *) compz job))
    (f2cl-lib:with-multi-array-data
        ((job character job-%data% job-%offset%)
         (compz character compz-%data% compz-%offset%)
         (h f2cl-lib:complex16 h-%data% h-%offset%)
         (w f2cl-lib:complex16 w-%data% w-%offset%)
         (z f2cl-lib:complex16 z-%data% z-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%))
      (prog ((initz nil) (lquery nil) (wantt nil) (wantz nil) (kbot 0) (nmin 0)
             (hl
              (make-array (the fixnum (reduce #'* (list nl nl))) :element-type
                          'f2cl-lib:complex16))
             (workl (make-array nl :element-type 'f2cl-lib:complex16)))
        (declare (type f2cl-lib:logical initz lquery wantt wantz)
```

```
         (type (f2cl-lib:integer4) kbot nmin)
         (type (array f2cl-lib:complex16 (*)) hl workl))
(setf wantt
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame job "S")
          (declare (ignore var-1))
          (when var-0 (setf job var-0))
          ret-val))
(setf initz
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame compz "I")
          (declare (ignore var-1))
          (when var-0 (setf compz var-0))
          ret-val))
(setf wantz
        (or initz
            (multiple-value-bind (ret-val var-0 var-1)
                (lsame compz "V")
              (declare (ignore var-1))
              (when var-0 (setf compz var-0))
              ret-val)))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (f2cl-lib:dcmplx
         (f2cl-lib:dble
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
         rzero))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(setf info 0)
(cond
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame job "E")
      (declare (ignore var-1))
      (when var-0 (setf job var-0))
      ret-val))
   (not wantt))
  (setf info -1))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame compz "N")
      (declare (ignore var-1))
      (when var-0 (setf compz var-0))
      ret-val))
   (not wantz))
  (setf info -2))
 ((< n 0) (setf info -3))
 ((or (< ilo 1)
      (> ilo
```

```
        (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n))))
  (setf info -4))
 ((or
   (< ihi (min (the f2cl-lib:integer4 ilo) (the f2cl-lib:integer4 n)))
   (> ihi n))
  (setf info -5))
 ((< ldh (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7))
 ((or (< ldz 1)
      (and wantz
           (< ldz
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 n)))))
  (setf info -10))
 ((and
   (< lwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
   (not lquery))
  (setf info -12)))
(cond
 ((/= info 0) (xerbla "ZHSEQR" (f2cl-lib:int-sub info)) (go end_label))
 ((= n 0) (go end_label))
 (lquery
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (zlaqr0 wantt wantz n ilo ihi h ldh w ilo ihi z ldz work lwork
       info)
    (declare (ignore var-5 var-7 var-10 var-12))
    (when var-0 (setf wantt var-0))
    (when var-1 (setf wantz var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf ldh var-6))
    (when var-8 (setf ilo var-8))
    (when var-9 (setf ihi var-9))
    (when var-11 (setf ldz var-11))
    (when var-13 (setf lwork var-13))
    (when var-14 (setf info var-14)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:dcmplx
           (max
            (f2cl-lib:dble
             (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%))
            (f2cl-lib:dble
             (max (the f2cl-lib:integer4 1)
                  (the f2cl-lib:integer4 n))))
           rzero))
  (go end_label))
 (t
```

```
(if (> ilo 1)
    (zcopy (f2cl-lib:int-sub ilo 1) h (f2cl-lib:int-add ldh 1) w 1))
(if (< ihi n)
    (zcopy (f2cl-lib:int-sub n ihi)
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                           ((+ ihi 1) (f2cl-lib:int-add ihi 1))
                           ((1 ldh) (1 *)) h-%offset%)
     (f2cl-lib:int-add ldh 1)
     (f2cl-lib:array-slice w-%data% f2cl-lib:complex16 ((+ ihi 1))
                           ((1 *)) w-%offset%)
     1))
(if initz
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
        (zlaset "A" n n zero one z ldz)
      (declare (ignore var-0 var-5))
      (when var-1 (setf n var-1))
      (when var-2 (setf n var-2))
      (when var-3 (setf zero var-3))
      (when var-4 (setf one var-4))
      (when var-6 (setf ldz var-6))))
(cond
 ((= ilo ihi)
  (setf (f2cl-lib:fref w-%data% (ilo) ((1 *)) w-%offset%)
          (f2cl-lib:fref h-%data% (ilo ilo) ((1 ldh) (1 *))
                         h-%offset%))
  (go end_label)))
(setf nmin
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 12 "ZHSEQR"
             (f2cl-lib:f2cl-// (f2cl-lib:fref-string job (1 1))
                               (f2cl-lib:fref-string compz (1 1)))
             n ilo ihi lwork)
          (declare (ignore var-0 var-1 var-2))
          (setf n var-3)
          (setf ilo var-4)
          (setf ihi var-5)
          (setf lwork var-6)
          ret-val))
(setf nmin
        (max (the f2cl-lib:integer4 ntiny)
             (the f2cl-lib:integer4 nmin)))
(cond
 ((> n nmin)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (zlaqr0 wantt wantz n ilo ihi h ldh w ilo ihi z ldz work lwork
       info)
    (declare (ignore var-5 var-7 var-10 var-12))
```

```
    (when var-0 (setf wantt var-0))
    (when var-1 (setf wantz var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf ldh var-6))
    (when var-8 (setf ilo var-8))
    (when var-9 (setf ihi var-9))
    (when var-11 (setf ldz var-11))
    (when var-13 (setf lwork var-13))
    (when var-14 (setf info var-14))))
  (t
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10 var-11 var-12)
       (zlahqr wantt wantz n ilo ihi h ldh w ilo ihi z ldz info)
    (declare (ignore var-5 var-7 var-10))
    (when var-0 (setf wantt var-0))
    (when var-1 (setf wantz var-1))
    (when var-2 (setf n var-2))
    (when var-3 (setf ilo var-3))
    (when var-4 (setf ihi var-4))
    (when var-6 (setf ldh var-6))
    (when var-8 (setf ilo var-8))
    (when var-9 (setf ihi var-9))
    (when var-11 (setf ldz var-11))
    (when var-12 (setf info var-12)))
   (cond
    ((> info 0) (setf kbot info)
     (cond
      ((>= n nl)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
            var-9 var-10 var-11 var-12 var-13 var-14)
           (zlaqr0 wantt wantz n ilo kbot h ldh w ilo ihi z ldz work
            lwork info)
         (declare (ignore var-5 var-7 var-10 var-12))
         (when var-0 (setf wantt var-0))
         (when var-1 (setf wantz var-1))
         (when var-2 (setf n var-2))
         (when var-3 (setf ilo var-3))
         (when var-4 (setf kbot var-4))
         (when var-6 (setf ldh var-6))
         (when var-8 (setf ilo var-8))
         (when var-9 (setf ihi var-9))
         (when var-11 (setf ldz var-11))
         (when var-13 (setf lwork var-13))
         (when var-14 (setf info var-14))))
      (t
       (multiple-value-bind
```

```
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (zlacpy "A" n n h ldh hl nl)
              (declare (ignore var-0 var-3 var-5))
              (when var-1 (setf n var-1))
              (when var-2 (setf n var-2))
              (when var-4 (setf ldh var-4))
              (when var-6 (setf nl var-6)))
            (setf (f2cl-lib:fref hl ((f2cl-lib:int-add n 1) n)
                                 ((1 nl) (1 nl)))
                  zero)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                (zlaset "A" nl (f2cl-lib:int-sub nl n) zero zero
                 (f2cl-lib:array-slice hl f2cl-lib:complex16
                                       (1 (f2cl-lib:int-add n 1))
                                       ((1 nl) (1 nl)))
                 nl)
              (declare (ignore var-0 var-2 var-5))
              (when var-1 (setf nl var-1))
              (when var-3 (setf zero var-3))
              (when var-4 (setf zero var-4))
              (when var-6 (setf nl var-6)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10 var-11 var-12 var-13 var-14)
                (zlaqr0 wantt wantz nl ilo kbot hl nl w ilo ihi z ldz workl
                 nl info)
              (declare (ignore var-5 var-7 var-10 var-12))
              (when var-0 (setf wantt var-0))
              (when var-1 (setf wantz var-1))
              (when var-2 (setf nl var-2))
              (when var-3 (setf ilo var-3))
              (when var-4 (setf kbot var-4))
              (when var-6 (setf nl var-6))
              (when var-8 (setf ilo var-8))
              (when var-9 (setf ihi var-9))
              (when var-11 (setf ldz var-11))
              (when var-13 (setf nl var-13))
              (when var-14 (setf info var-14)))
            (if (or wantt (/= info 0))
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                    (zlacpy "A" n n hl nl h ldh)
                  (declare (ignore var-0 var-3 var-5))
                  (when var-1 (setf n var-1))
                  (when var-2 (setf n var-2))
                  (when var-4 (setf nl var-4))
                  (when var-6 (setf ldh var-6)))))))))
    (if (and (or wantt (/= info 0)) (> n 2))
        (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
                (zlaset "L" (f2cl-lib:int-sub n 2) (f2cl-lib:int-sub n 2)
                 zero zero
                 (f2cl-lib:array-slice h-%data% f2cl-lib:complex16 (3 1)
                                       ((1 ldh) (1 *)) h-%offset%)
                 ldh)
               (declare (ignore var-0 var-1 var-2 var-5))
               (when var-3 (setf zero var-3))
               (when var-4 (setf zero var-4))
               (when var-6 (setf ldh var-6))))
          (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (f2cl-lib:dcmplx
                 (max
                  (f2cl-lib:dble
                   (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
                  (f2cl-lib:dble
                   (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)))
                 rzero))))
      end_label
       (return
        (values job compz n ilo ihi nil ldh nil nil ldz nil lwork info))))))
```

———-

90.217 zlacgv

90.217.1 Axiom unit tests

— **zlacgv.input** —

```
)set break resume
)sys rm -f zlacgv.output
)spool zlacgv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.217.2 Axiom help page

— **zlacgv.help** —

```
====================================================================
zlacgv examples
====================================================================

====================================================================
Man Page Details
====================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZLACGV( N, X, INCX )

       .. Scalar Arguments ..
       INTEGER            INCX, N
       ..
       .. Array Arguments ..
       COMPLEX*16         X( * )
       ..


 Purpose:
 =============

 ZLACGV conjugates a complex vector of length N.

 Arguments:
 ==========

  [in] N

          N is INTEGER
          The length of the vector X.  N >= 0.

  [in,out] X

          X is COMPLEX*16 array, dimension
                         (1+(N-1)*abs(INCX))
          On entry, the vector of length N to be conjugated.
          On exit, X is overwritten with conjg(X).

  [in] INCX

          INCX is INTEGER
          The spacing between successive elements of X.
```

———-

90.217.3 fortran code

— **zlacgv.f** —

```
*  =====================================================================
      SUBROUTINE ZLACGV( N, X, INCX )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INCX, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         X( * )
*     ..
*
* =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, IOFF
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DCONJG
*     ..
*     .. Executable Statements ..
*
      IF( INCX.EQ.1 ) THEN
         DO 10 I = 1, N
            X( I ) = DCONJG( X( I ) )
   10    CONTINUE
      ELSE
         IOFF = 1
         IF( INCX.LT.0 )
     $      IOFF = 1 - ( N-1 )*INCX
         DO 20 I = 1, N
            X( IOFF ) = DCONJG( X( IOFF ) )
            IOFF = IOFF + INCX
   20    CONTINUE
      END IF
      RETURN
*
*     End of ZLACGV
*
```

```
      END
```

———-

90.217.4 lisp code

— zlacgv.lisp —

```
(defun zlacgv (n x incx)
  (declare (type (array f2cl-lib:complex16 (*)) x)
           (type (f2cl-lib:integer4) incx n))
  (f2cl-lib:with-multi-array-data
      ((x f2cl-lib:complex16 x-%data% x-%offset%))
    (prog ((i 0) (ioff 0))
      (declare (type (f2cl-lib:integer4) ioff i))
      (cond
       ((= incx 1)
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
                    (coerce
                     (f2cl-lib:dconjg
                      (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%))
                     'f2cl-lib:complex16))
           label10)))
       (t (setf ioff 1)
        (if (< incx 0)
            (setf ioff
                    (f2cl-lib:int-sub 1
                                      (f2cl-lib:int-mul (f2cl-lib:int-sub n 1)
                                                        incx))))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i n) nil)
          (tagbody
            (setf (f2cl-lib:fref x-%data% (ioff) ((1 *)) x-%offset%)
                    (coerce
                     (f2cl-lib:dconjg
                      (f2cl-lib:fref x-%data% (ioff) ((1 *)) x-%offset%))
                     'f2cl-lib:complex16))
            (setf ioff (f2cl-lib:int-add ioff incx))
           label20))))
      (go end_label)
     end_label
      (return (values nil nil nil)))))
```

———-

90.218 zlacpy

90.218.1 Axiom unit tests

— **zlacpy.input** —

```
)set break resume
)sys rm -f zlacpy.output
)spool zlacpy.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.218.2 Axiom help page

— **zlacpy.help** —

```
======================================================================
zlacpy examples
======================================================================

======================================================================
Man Page Details
======================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZLACPY( UPLO, M, N, A, LDA, B, LDB )

       .. Scalar Arguments ..
       CHARACTER          UPLO
       INTEGER            LDA, LDB, M, N
       ..
```

```
      .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), B( LDB, * )
      ..


Purpose:
=============

 ZLACPY copies all or part of a two-dimensional matrix A to another
 matrix B.


Arguments:
==========

 [in] UPLO

         UPLO is CHARACTER*1
         Specifies the part of the matrix A to be copied to B.
         = 'U':      Upper triangular part
         = 'L':      Lower triangular part
         Otherwise:  All of the matrix A

 [in] M

         M is INTEGER
         The number of rows of the matrix A.  M >= 0.

 [in] N

         N is INTEGER
         The number of columns of the matrix A.  N >= 0.

 [in] A

         A is COMPLEX*16 array, dimension (LDA,N)
         The m by n matrix A.  If UPLO = 'U', only the upper trapezium
         is accessed; if UPLO = 'L', only the lower trapezium is
         accessed.

 [in] LDA

         LDA is INTEGER
         The leading dimension of the array A.  LDA >= max(1,M).

 [out] B

         B is COMPLEX*16 array, dimension (LDB,N)
         On exit, B = A in the locations specified by UPLO.
```

```
[in] LDB

        LDB is INTEGER
        The leading dimension of the array B.  LDB >= max(1,M).
```

———-

90.218.3 fortran code

— **zlacpy.f** —

```
*  =====================================================================
      SUBROUTINE ZLACPY( UPLO, M, N, A, LDA, B, LDB )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            LDA, LDB, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), B( LDB, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MIN
*     ..
*     .. Executable Statements ..
*
      IF( LSAME( UPLO, 'U' ) ) THEN
         DO 20 J = 1, N
            DO 10 I = 1, MIN( J, M )
               B( I, J ) = A( I, J )
   10       CONTINUE
```

```
   20    CONTINUE
*
      ELSE IF( LSAME( UPLO, 'L' ) ) THEN
         DO 40 J = 1, N
            DO 30 I = J, M
               B( I, J ) = A( I, J )
   30       CONTINUE
   40    CONTINUE
*
      ELSE
         DO 60 J = 1, N
            DO 50 I = 1, M
               B( I, J ) = A( I, J )
   50       CONTINUE
   60    CONTINUE
      END IF
*
      RETURN
*
*     End of ZLACPY
*
      END
```

———-

90.218.4 lisp code

— zlacpy.lisp —

```
(defun zlacpy (uplo m n a lda b ldb$)
  (declare (type (array f2cl-lib:complex16 (*)) b a)
           (type (f2cl-lib:integer4) ldb$ lda n m)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (b f2cl-lib:complex16 b-%data% b-%offset%))
    (prog ((i 0) (j 0))
      (declare (type (f2cl-lib:integer4) j i))
      (cond
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
```

```
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i
                            (min (the f2cl-lib:integer4 j)
                                 (the f2cl-lib:integer4 m)))
                         nil)
            (tagbody
              (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)
                    (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%))
            label10))
         label20)))
    ((multiple-value-bind (ret-val var-0 var-1)
         (lsame uplo "L")
       (declare (ignore var-1))
       (when var-0 (setf uplo var-0))
       ret-val)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%))
           label30))
         label40)))
    (t
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j n) nil)
       (tagbody
         (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                       ((> i m) nil)
           (tagbody
             (setf (f2cl-lib:fref b-%data% (i j) ((1 ldb$) (1 *))
                                    b-%offset%)
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                    a-%offset%))
           label50))
         label60))))
  (go end_label)
 end_label
  (return (values uplo nil nil nil nil nil nil)))))
```

———-

90.219 zladiv

90.219.1 Axiom unit tests

— zladiv.input —

```
)set break resume
)sys rm -f zladiv.output
)spool zladiv.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.219.2 Axiom help page

— zladiv.help —

```
=====================================================================
zladiv examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       COMPLEX*16     FUNCTION ZLADIV( X, Y )

       .. Scalar Arguments ..
       COMPLEX*16         X, Y
       ..


 Purpose:
 =============
```

```
  ZLADIV := X / Y, where X and Y are complex.  The computation of X / Y
  will not overflow on an intermediary step unless the results
  overflows.

 Arguments:
 ==========

  [in] X

          X is COMPLEX*16

  [in] Y

          Y is COMPLEX*16
          The complex scalars X and Y.
```

90.219.3 fortran code

— zladiv.f —

```
*  =====================================================================
      COMPLEX*16     FUNCTION ZLADIV( X, Y )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX*16         X, Y
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      DOUBLE PRECISION   ZI, ZR
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLADIV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DBLE, DCMPLX, DIMAG
*     ..
*     .. Executable Statements ..
```

```
*
      CALL DLADIV( DBLE( X ), DIMAG( X ), DBLE( Y ), DIMAG( Y ), ZR,
     $             ZI )
      ZLADIV = DCMPLX( ZR, ZI )
*
      RETURN
*
*     End of ZLADIV
*
      END
```

———-

90.219.4 lisp code

— **zladiv.lisp** —

```
(defun zladiv (x y)
  (declare (type (f2cl-lib:complex16) y x))
  (prog ((zi 0.0d0) (zr 0.0d0) (zladiv #C(0.0d0 0.0d0)))
    (declare (type (f2cl-lib:complex16) zladiv)
             (type (double-float) zr zi))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5)
        (dladiv (f2cl-lib:dble x) (f2cl-lib:dimag x) (f2cl-lib:dble y)
         (f2cl-lib:dimag y) zr zi)
      (declare (ignore var-0 var-1 var-2 var-3))
      (when var-4 (setf zr var-4))
      (when var-5 (setf zi var-5)))
    (setf zladiv (f2cl-lib:dcmplx zr zi))
    (go end_label)
   end_label
    (return (values zladiv nil nil))))
```

———-

90.220 zlahqr

90.220.1 Axiom unit tests

— **zlahqr.input** —

```
)set break resume
```

```
)sys rm -f zlahqr.output
)spool zlahqr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.220.2 Axiom help page

— **zlahqr.help** —

```
==================================================================
zlahqr examples
==================================================================

==================================================================
Man Page Details
==================================================================

 Online html documentation available at
           http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZLAHQR( WANTT, WANTZ, N, ILO, IHI, H, LDH, W, ILOZ,
                          IHIZ, Z, LDZ, INFO )

       .. Scalar Arguments ..
       INTEGER            IHI, IHIZ, ILO, ILOZ, INFO, LDH, LDZ, N
       LOGICAL            WANTT, WANTZ
       ..
       .. Array Arguments ..
       COMPLEX*16         H( LDH, * ), W( * ), Z( LDZ, * )
       ..


 Purpose:
 =============

    ZLAHQR is an auxiliary routine called by CHSEQR to update the
    eigenvalues and Schur decomposition already computed by CHSEQR, by
    dealing with the Hessenberg submatrix in rows and columns ILO to
    IHI.
```

```
 Arguments:
 ==========

  [in] WANTT

          WANTT is LOGICAL
          = .TRUE. : the full Schur form T is required;
          = .FALSE.: only eigenvalues are required.

  [in] WANTZ

          WANTZ is LOGICAL
          = .TRUE. : the matrix of Schur vectors Z is required;
          = .FALSE.: Schur vectors are not required.

  [in] N

          N is INTEGER
          The order of the matrix H.  N >= 0.

  [in] ILO

          ILO is INTEGER

  [in] IHI

          IHI is INTEGER
          It is assumed that H is already upper triangular in rows and
          columns IHI+1:N, and that H(ILO,ILO-1) = 0 (unless ILO = 1).
          ZLAHQR works primarily with the Hessenberg submatrix in rows
          and columns ILO to IHI, but applies transformations to all of
          H if WANTT is .TRUE..
          1 <= ILO <= max(1,IHI); IHI <= N.

  [in,out] H

          H is COMPLEX*16 array, dimension (LDH,N)
          On entry, the upper Hessenberg matrix H.
          On exit, if INFO is zero and if WANTT is .TRUE., then H
          is upper triangular in rows and columns ILO:IHI.  If INFO
          is zero and if WANTT is .FALSE., then the contents of H
          are unspecified on exit.  The output state of H in case
          INF is positive is below under the description of INFO.

  [in] LDH

          LDH is INTEGER
          The leading dimension of the array H. LDH >= max(1,N).
```

```
[out] W

        W is COMPLEX*16 array, dimension (N)
        The computed eigenvalues ILO to IHI are stored in the
        corresponding elements of W. If WANTT is .TRUE., the
        eigenvalues are stored in the same order as on the diagonal
        of the Schur form returned in H, with W(i) = H(i,i).

[in] ILOZ

        ILOZ is INTEGER

[in] IHIZ

        IHIZ is INTEGER
        Specify the rows of Z to which transformations must be
        applied if WANTZ is .TRUE..
        1 <= ILOZ <= ILO; IHI <= IHIZ <= N.

[in,out] Z

        Z is COMPLEX*16 array, dimension (LDZ,N)
        If WANTZ is .TRUE., on entry Z must contain the current
        matrix Z of transformations accumulated by CHSEQR, and on
        exit Z has been updated; transformations are applied only to
        the submatrix Z(ILOZ:IHIZ,ILO:IHI).
        If WANTZ is .FALSE., Z is not referenced.

[in] LDZ

        LDZ is INTEGER
        The leading dimension of the array Z. LDZ >= max(1,N).

[out] INFO

        INFO is INTEGER
         =   0: successful exit
        .GT. 0: if INFO = i, ZLAHQR failed to compute all the
                eigenvalues ILO to IHI in a total of 30 iterations
                per eigenvalue; elements i+1:ihi of W contain
                those eigenvalues which have been successfully
                computed.

                If INFO .GT. 0 and WANTT is .FALSE., then on exit,
                the remaining unconverged eigenvalues are the
                eigenvalues of the upper Hessenberg matrix
                rows and columns ILO thorugh INFO of the final,
                output value of H.
```

```
                If INFO .GT. 0 and WANTT is .TRUE., then on exit
         (*)      (initial value of H)*U  = U*(final value of H)
                where U is an orthognal matrix.    The final
                value of H is upper Hessenberg and triangular in
                rows and columns INFO+1 through IHI.

                If INFO .GT. 0 and WANTZ is .TRUE., then on exit
                    (final value of Z)  = (initial value of Z)*U
                where U is the orthogonal matrix in (*)
                (regardless of the value of WANTT.)
```

———-

90.220.3 fortran code

— zlahqr.f —

```
*  =====================================================================
      SUBROUTINE ZLAHQR( WANTT, WANTZ, N, ILO, IHI, H, LDH, W, ILOZ,
     $                   IHIZ, Z, LDZ, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            IHI, IHIZ, ILO, ILOZ, INFO, LDH, LDZ, N
      LOGICAL            WANTT, WANTZ
*     ..
*     .. Array Arguments ..
      COMPLEX*16         H( LDH, * ), W( * ), Z( LDZ, * )
*     ..
*
*  =====================================================
*
*     .. Parameters ..
      INTEGER            ITMAX
      PARAMETER          ( ITMAX = 30 )
      COMPLEX*16         ZERO, ONE
      PARAMETER          ( ZERO = ( 0.0d0, 0.0d0 ),
     $                   ONE = ( 1.0d0, 0.0d0 ) )
      DOUBLE PRECISION   RZERO, RONE, HALF
      PARAMETER          ( RZERO = 0.0d0, RONE = 1.0d0, HALF = 0.5d0 )
      DOUBLE PRECISION   DAT1
      PARAMETER          ( DAT1 = 3.0d0 / 4.0d0 )
*     ..
```

```
*     .. Local Scalars ..
      COMPLEX*16         CDUM, H11, H11S, H22, SC, SUM, T, T1, TEMP, U,
     $                   V2, X, Y
      DOUBLE PRECISION   AA, AB, BA, BB, H10, H21, RTEMP, S, SAFMAX,
     $                   SAFMIN, SMLNUM, SX, T2, TST, ULP
      INTEGER            I, I1, I2, ITS, J, JHI, JLO, K, L, M, NH, NZ
*     ..
*     .. Local Arrays ..
      COMPLEX*16         V( 2 )
*     ..
*     .. External Functions ..
      COMPLEX*16         ZLADIV
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           ZLADIV, DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, ZCOPY, ZLARFG, ZSCAL
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DCONJG, DIMAG, MAX, MIN, SQRT
*     ..
*     .. Statement Function definitions ..
      CABS1( CDUM ) = ABS( DBLE( CDUM ) ) + ABS( DIMAG( CDUM ) )
*     ..
*     .. Executable Statements ..
*
      INFO = 0
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
      IF( ILO.EQ.IHI ) THEN
         W( ILO ) = H( ILO, ILO )
         RETURN
      END IF
*
*     ==== clear out the trash ====
      DO 10 J = ILO, IHI - 3
         H( J+2, J ) = ZERO
         H( J+3, J ) = ZERO
   10 CONTINUE
      IF( ILO.LE.IHI-2 )
     $   H( IHI, IHI-2 ) = ZERO
*     ==== ensure that subdiagonal entries are real ====
      IF( WANTT ) THEN
         JLO = 1
```

```
         JHI = N
      ELSE
         JLO = ILO
         JHI = IHI
      END IF
      DO 20 I = ILO + 1, IHI
         IF( DIMAG( H( I, I-1 ) ).NE.RZERO ) THEN
*           ==== The following redundant normalization
*           .    avoids problems with both gradual and
*           .    sudden underflow in ABS(H(I,I-1)) ====
            SC = H( I, I-1 ) / CABS1( H( I, I-1 ) )
            SC = DCONJG( SC ) / ABS( SC )
            H( I, I-1 ) = ABS( H( I, I-1 ) )
            CALL ZSCAL( JHI-I+1, SC, H( I, I ), LDH )
            CALL ZSCAL( MIN( JHI, I+1 )-JLO+1, DCONJG( SC ),
     $                  H( JLO, I ), 1 )
            IF( WANTZ )
     $         CALL ZSCAL( IHIZ-ILOZ+1, DCONJG( SC ), Z( ILOZ, I ), 1 )
         END IF
   20 CONTINUE
*
      NH = IHI - ILO + 1
      NZ = IHIZ - ILOZ + 1
*
*     Set machine-dependent constants for the stopping criterion.
*
      SAFMIN = DLAMCH( 'SAFE MINIMUM' )
      SAFMAX = RONE / SAFMIN
      CALL DLABAD( SAFMIN, SAFMAX )
      ULP = DLAMCH( 'PRECISION' )
      SMLNUM = SAFMIN*( DBLE( NH ) / ULP )
*
*     I1 and I2 are the indices of the first row and last column of H
*     to which transformations must be applied. If eigenvalues only are
*     being computed, I1 and I2 are set inside the main loop.
*
      IF( WANTT ) THEN
         I1 = 1
         I2 = N
      END IF
*
*     The main loop begins here. I is the loop index and decreases from
*     IHI to ILO in steps of 1. Each iteration of the loop works
*     with the active submatrix in rows and columns L to I.
*     Eigenvalues I+1 to IHI have already converged. Either L = ILO, or
*     H(L,L-1) is negligible so that the matrix splits.
*
      I = IHI
   30 CONTINUE
      IF( I.LT.ILO )
```

```
     $   GO TO 150
*
*     Perform QR iterations on rows and columns ILO to I until a
*     submatrix of order 1 splits off at the bottom because a
*     subdiagonal element has become negligible.
*
      L = ILO
      DO 130 ITS = 0, ITMAX
*
*        Look for a single small subdiagonal element.
*
         DO 40 K = I, L + 1, -1
            IF( CABS1( H( K, K-1 ) ).LE.SMLNUM )
     $         GO TO 50
            TST = CABS1( H( K-1, K-1 ) ) + CABS1( H( K, K ) )
            IF( TST.EQ.ZERO ) THEN
               IF( K-2.GE.ILO )
     $            TST = TST + ABS( DBLE( H( K-1, K-2 ) ) )
               IF( K+1.LE.IHI )
     $            TST = TST + ABS( DBLE( H( K+1, K ) ) )
            END IF
*           ==== The following is a conservative small subdiagonal
*           .    deflation criterion due to Ahues & Tisseur (LAWN 122,
*           .    1997). It has better mathematical foundation and
*           .    improves accuracy in some examples.  ====
            IF( ABS( DBLE( H( K, K-1 ) ) ).LE.ULP*TST ) THEN
               AB = MAX( CABS1( H( K, K-1 ) ), CABS1( H( K-1, K ) ) )
               BA = MIN( CABS1( H( K, K-1 ) ), CABS1( H( K-1, K ) ) )
               AA = MAX( CABS1( H( K, K ) ),
     $              CABS1( H( K-1, K-1 )-H( K, K ) ) )
               BB = MIN( CABS1( H( K, K ) ),
     $              CABS1( H( K-1, K-1 )-H( K, K ) ) )
               S = AA + AB
               IF( BA*( AB / S ).LE.MAX( SMLNUM,
     $             ULP*( BB*( AA / S ) ) ) )GO TO 50
            END IF
   40    CONTINUE
   50    CONTINUE
         L = K
         IF( L.GT.ILO ) THEN
*
*           H(L,L-1) is negligible
*
            H( L, L-1 ) = ZERO
         END IF
*
*        Exit from loop if a submatrix of order 1 has split off.
*
         IF( L.GE.I )
     $      GO TO 140
```

```
*
*       Now the active submatrix is in rows and columns L to I. If
*       eigenvalues only are being computed, only the active submatrix
*       need be transformed.
*
        IF( .NOT.WANTT ) THEN
           I1 = L
           I2 = I
        END IF
*
        IF( ITS.EQ.10 ) THEN
*
*          Exceptional shift.
*
           S = DAT1*ABS( DBLE( H( L+1, L ) ) )
           T = S + H( L, L )
        ELSE IF( ITS.EQ.20 ) THEN
*
*          Exceptional shift.
*
           S = DAT1*ABS( DBLE( H( I, I-1 ) ) )
           T = S + H( I, I )
        ELSE
*
*          Wilkinson's shift.
*
           T = H( I, I )
           U = SQRT( H( I-1, I ) )*SQRT( H( I, I-1 ) )
           S = CABS1( U )
           IF( S.NE.RZERO ) THEN
              X = HALF*( H( I-1, I-1 )-T )
              SX = CABS1( X )
              S = MAX( S, CABS1( X ) )
              Y = S*SQRT( ( X / S )**2+( U / S )**2 )
              IF( SX.GT.RZERO ) THEN
                 IF( DBLE( X / SX )*DBLE( Y )+DIMAG( X / SX )*
     $               DIMAG( Y ).LT.RZERO )Y = -Y
              END IF
              T = T - U*ZLADIV( U, ( X+Y ) )
           END IF
        END IF
*
*       Look for two consecutive small subdiagonal elements.
*
        DO 60 M = I - 1, L + 1, -1
*
*          Determine the effect of starting the single-shift QR
*          iteration at row M, and see if this would make H(M,M-1)
*          negligible.
*
```

```
            H11 = H( M, M )
            H22 = H( M+1, M+1 )
            H11S = H11 - T
            H21 = DBLE( H( M+1, M ) )
            S = CABS1( H11S ) + ABS( H21 )
            H11S = H11S / S
            H21 = H21 / S
            V( 1 ) = H11S
            V( 2 ) = H21
            H10 = DBLE( H( M, M-1 ) )
            IF( ABS( H10 )*ABS( H21 ).LE.ULP*
     $          ( CABS1( H11S )*( CABS1( H11 )+CABS1( H22 ) ) ) )
     $          GO TO 70
   60    CONTINUE
         H11 = H( L, L )
         H22 = H( L+1, L+1 )
         H11S = H11 - T
         H21 = DBLE( H( L+1, L ) )
         S = CABS1( H11S ) + ABS( H21 )
         H11S = H11S / S
         H21 = H21 / S
         V( 1 ) = H11S
         V( 2 ) = H21
   70    CONTINUE
*
*        Single-shift QR step
*
         DO 120 K = M, I - 1
*
*           The first iteration of this loop determines a reflection G
*           from the vector V and applies it from left and right to H,
*           thus creating a nonzero bulge below the subdiagonal.
*
*           Each subsequent iteration determines a reflection G to
*           restore the Hessenberg form in the (K-1)th column, and thus
*           chases the bulge one step toward the bottom of the active
*           submatrix.
*
*           V(2) is always real before the call to ZLARFG, and hence
*           after the call T2 ( = T1*V(2) ) is also real.
*
            IF( K.GT.M )
     $         CALL ZCOPY( 2, H( K, K-1 ), 1, V, 1 )
            CALL ZLARFG( 2, V( 1 ), V( 2 ), 1, T1 )
            IF( K.GT.M ) THEN
               H( K, K-1 ) = V( 1 )
               H( K+1, K-1 ) = ZERO
            END IF
            V2 = V( 2 )
            T2 = DBLE( T1*V2 )
```

```
*
*          Apply G from the left to transform the rows of the matrix
*          in columns K to I2.
*
           DO 80 J = K, I2
              SUM = DCONJG( T1 )*H( K, J ) + T2*H( K+1, J )
              H( K, J ) = H( K, J ) - SUM
              H( K+1, J ) = H( K+1, J ) - SUM*V2
   80      CONTINUE
*
*          Apply G from the right to transform the columns of the
*          matrix in rows I1 to min(K+2,I).
*
           DO 90 J = I1, MIN( K+2, I )
              SUM = T1*H( J, K ) + T2*H( J, K+1 )
              H( J, K ) = H( J, K ) - SUM
              H( J, K+1 ) = H( J, K+1 ) - SUM*DCONJG( V2 )
   90      CONTINUE
*
           IF( WANTZ ) THEN
*
*             Accumulate transformations in the matrix Z
*
              DO 100 J = ILOZ, IHIZ
                 SUM = T1*Z( J, K ) + T2*Z( J, K+1 )
                 Z( J, K ) = Z( J, K ) - SUM
                 Z( J, K+1 ) = Z( J, K+1 ) - SUM*DCONJG( V2 )
  100         CONTINUE
           END IF
*
           IF( K.EQ.M .AND. M.GT.L ) THEN
*
*             If the QR step was started at row M > L because two
*             consecutive small subdiagonals were found, then extra
*             scaling must be performed to ensure that H(M,M-1) remains
*             real.
*
              TEMP = ONE - T1
              TEMP = TEMP / ABS( TEMP )
              H( M+1, M ) = H( M+1, M )*DCONJG( TEMP )
              IF( M+2.LE.I )
     $           H( M+2, M+1 ) = H( M+2, M+1 )*TEMP
              DO 110 J = M, I
                 IF( J.NE.M+1 ) THEN
                    IF( I2.GT.J )
     $                 CALL ZSCAL( I2-J, TEMP, H( J, J+1 ), LDH )
                    CALL ZSCAL( J-I1, DCONJG( TEMP ), H( I1, J ), 1 )
                    IF( WANTZ ) THEN
                       CALL ZSCAL( NZ, DCONJG( TEMP ), Z( ILOZ, J ),
     $                             1 )
```

```
                  END IF
               END IF
  110          CONTINUE
            END IF
  120    CONTINUE
*
*        Ensure that H(I,I-1) is real.
*
         TEMP = H( I, I-1 )
         IF( DIMAG( TEMP ).NE.RZERO ) THEN
            RTEMP = ABS( TEMP )
            H( I, I-1 ) = RTEMP
            TEMP = TEMP / RTEMP
            IF( I2.GT.I )
     $         CALL ZSCAL( I2-I, DCONJG( TEMP ), H( I, I+1 ), LDH )
            CALL ZSCAL( I-I1, TEMP, H( I1, I ), 1 )
            IF( WANTZ ) THEN
               CALL ZSCAL( NZ, TEMP, Z( ILOZ, I ), 1 )
            END IF
         END IF
*
  130 CONTINUE
*
*     Failure to converge in remaining number of iterations
*
      INFO = I
      RETURN
*
  140 CONTINUE
*
*     H(I,I-1) is negligible: one eigenvalue has converged.
*
      W( I ) = H( I, I )
*
*     return to start of the main loop with new value of I.
*
      I = L - 1
      GO TO 30
*
  150 CONTINUE
      RETURN
*
*     End of ZLAHQR
*
      END
```

90.220.4 lisp code

— zlahqr.lisp —

```
(let* ((zero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (one (f2cl-lib:cmplx 1.0d0 0.0d0))
       (rzero 0.0d0)
       (rone 1.0d0)
       (half 0.5d0)
       (dat1 (f2cl-lib:f2cl/ 3.0d0 4.0d0)))
  (declare (type (f2cl-lib:complex16) zero)
           (type (f2cl-lib:complex16) one)
           (type (double-float 0.0d0 0.0d0) rzero)
           (type (double-float 1.0d0 1.0d0) rone)
           (type (double-float 0.5d0 0.5d0) half)
           (type (double-float) dat1)
           (ignorable zero one rzero rone half dat1))
  (defun zlahqr (wantt wantz n ilo ihi h ldh w iloz ihiz z ldz info)
    (declare (type (array f2cl-lib:complex16 (*)) z w h)
             (type (f2cl-lib:integer4) info ldz ihiz iloz ldh ihi ilo n)
             (type f2cl-lib:logical wantz wantt))
    (f2cl-lib:with-multi-array-data
        ((h f2cl-lib:complex16 h-%data% h-%offset%)
         (w f2cl-lib:complex16 w-%data% w-%offset%)
         (z f2cl-lib:complex16 z-%data% z-%offset%))
      (labels ((cabs1 (cdum)
                 (+ (abs (f2cl-lib:dble cdum)) (abs (f2cl-lib:dimag cdum)))))
        (declare
         (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
          cabs1))
        (prog ((v (make-array 2 :element-type 'f2cl-lib:complex16)) (i 0)
               (i1 0) (i2 0) (its 0) (itmax 0) (j 0) (jhi 0) (jlo 0) (k 0)
               (l 0) (m 0) (nh 0) (nz 0) (aa 0.0d0) (ab 0.0d0) (ba 0.0d0)
               (bb 0.0d0) (h10 0.0d0) (h21 0.0d0) (rtemp 0.0d0) (s 0.0d0)
               (safmax 0.0d0) (safmin 0.0d0) (smlnum 0.0d0) (sx 0.0d0)
               (t2 0.0d0) (tst 0.0d0) (ulp 0.0d0) (cdum #C(0.0d0 0.0d0))
               (h11 #C(0.0d0 0.0d0)) (h11s #C(0.0d0 0.0d0))
               (h22 #C(0.0d0 0.0d0)) (sc #C(0.0d0 0.0d0)) (sum #C(0.0d0 0.0d0))
               (t$ #C(0.0d0 0.0d0)) (t1 #C(0.0d0 0.0d0)) (temp #C(0.0d0 0.0d0))
               (u #C(0.0d0 0.0d0)) (v2 #C(0.0d0 0.0d0)) (x #C(0.0d0 0.0d0))
               (y #C(0.0d0 0.0d0)))
          (declare (type (array f2cl-lib:complex16 (2)) v)
                   (type (f2cl-lib:integer4) i i1 i2 its itmax j jhi jlo k l m
                    nh nz)
                   (type (double-float) aa ab ba bb h10 h21 rtemp s safmax
                    safmin smlnum sx t2 tst ulp)
                   (type (f2cl-lib:complex16) cdum h11 h11s h22 sc sum t$ t1
                    temp u v2 x y))
          (setf info 0)
```

```
(if (= n 0)
    (go end_label))
(cond
 ((= ilo ihi)
  (setf (f2cl-lib:fref w-%data% (ilo) ((1 *)) w-%offset%)
          (f2cl-lib:fref h-%data% (ilo ilo) ((1 ldh) (1 *))
                          h-%offset%))
  (go end_label)))
(f2cl-lib:fdo (j ilo (f2cl-lib:int-add j 1))
              ((> j (f2cl-lib:int-add ihi (f2cl-lib:int-sub 3))) nil)
  (tagbody
    (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add j 2) j)
                          ((1 ldh) (1 *)) h-%offset%)
            zero)
    (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add j 3) j)
                          ((1 ldh) (1 *)) h-%offset%)
            zero)
   label10))
(if (<= ilo (f2cl-lib:int-sub ihi 2))
    (setf (f2cl-lib:fref h-%data% (ihi (f2cl-lib:int-sub ihi 2))
                          ((1 ldh) (1 *)) h-%offset%)
            zero))
(cond (wantt (setf jlo 1) (setf jhi n))
      (t (setf jlo ilo) (setf jhi ihi)))
(f2cl-lib:fdo (i (f2cl-lib:int-add ilo 1) (f2cl-lib:int-add i 1))
              ((> i ihi) nil)
  (tagbody
    (cond
     ((/=
       (f2cl-lib:dimag
        (f2cl-lib:fref h
                        (i (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                        ((1 ldh) (1 *))))
       rzero)
      (setf sc
              (/
               (f2cl-lib:fref h-%data% (i (f2cl-lib:int-sub i 1))
                               ((1 ldh) (1 *)) h-%offset%)
               (cabs1
                (f2cl-lib:fref h-%data% (i (f2cl-lib:int-sub i 1))
                                ((1 ldh) (1 *)) h-%offset%))))
      (setf sc
              (coerce (/ (f2cl-lib:dconjg sc) (abs sc))
                      'f2cl-lib:complex16))
      (setf (f2cl-lib:fref h-%data% (i (f2cl-lib:int-sub i 1))
                            ((1 ldh) (1 *)) h-%offset%)
              (coerce
               (abs
                (f2cl-lib:fref h-%data% (i (f2cl-lib:int-sub i 1))
                                ((1 ldh) (1 *)) h-%offset%))
```

```
                 'f2cl-lib:complex16))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (zscal (f2cl-lib:int-add (f2cl-lib:int-sub jhi i) 1) sc
             (f2cl-lib:array-slice h-%data% f2cl-lib:complex16 (i i)
                                   ((1 ldh) (1 *)) h-%offset%)
             ldh)
          (declare (ignore var-0 var-2))
          (when var-1 (setf sc var-1))
          (when var-3 (setf ldh var-3)))
        (zscal
         (f2cl-lib:int-add
          (f2cl-lib:int-sub
           (min (the f2cl-lib:integer4 jhi)
                (the f2cl-lib:integer4 (f2cl-lib:int-add i 1)))
           jlo)
          1)
         (f2cl-lib:dconjg sc)
         (f2cl-lib:array-slice h-%data% f2cl-lib:complex16 (jlo i)
                               ((1 ldh) (1 *)) h-%offset%)
         1)
        (if wantz
            (zscal (f2cl-lib:int-add (f2cl-lib:int-sub ihiz iloz) 1)
             (f2cl-lib:dconjg sc)
             (f2cl-lib:array-slice z-%data% f2cl-lib:complex16 (iloz i)
                                   ((1 ldz) (1 *)) z-%offset%)
             1))))
    label20))
  (setf nh (f2cl-lib:int-add (f2cl-lib:int-sub ihi ilo) 1))
  (setf nz (f2cl-lib:int-add (f2cl-lib:int-sub ihiz iloz) 1))
  (setf safmin (dlamch "SAFE MINIMUM"))
  (setf safmax (/ rone safmin))
  (multiple-value-bind (var-0 var-1)
      (dlabad safmin safmax)
    (declare (ignore))
    (when var-0 (setf safmin var-0))
    (when var-1 (setf safmax var-1)))
  (setf ulp (dlamch "PRECISION"))
  (setf smlnum (* safmin (/ (f2cl-lib:dble nh) ulp)))
  (cond (wantt (setf i1 1) (setf i2 n)))
  (setf itmax
          (f2cl-lib:int-mul 30
                            (max (the f2cl-lib:integer4 10)
                                 (the f2cl-lib:integer4 nh))))
  (setf i ihi)
 label30
  (if (< i ilo)
      (go label150))
  (setf l ilo)
  (f2cl-lib:fdo (its 0 (f2cl-lib:int-add its 1))
                ((> its itmax) nil)
```

```
(tagbody
  (f2cl-lib:fdo (k i (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                ((> k (f2cl-lib:int-add l 1)) nil)
    (tagbody
      (if (<=
           (cabs1
            (f2cl-lib:fref h-%data% (k (f2cl-lib:int-sub k 1))
                           ((1 ldh) (1 *)) h-%offset%))
           smlnum)
          (go label50))
      (setf tst
              (+
               (cabs1
                (f2cl-lib:fref h-%data%
                               ((f2cl-lib:int-sub k 1)
                                (f2cl-lib:int-sub k 1))
                               ((1 ldh) (1 *)) h-%offset%))
               (cabs1
                (f2cl-lib:fref h-%data% (k k) ((1 ldh) (1 *))
                               h-%offset%))))
      (cond
       ((= tst zero)
        (if (>= (f2cl-lib:int-sub k 2) ilo)
            (setf tst
                    (+ tst
                       (abs
                        (f2cl-lib:dble
                         (f2cl-lib:fref h-%data%
                                        ((f2cl-lib:int-sub k 1)
                                         (f2cl-lib:int-sub k 2))
                                        ((1 ldh) (1 *))
                                        h-%offset%))))))
        (if (<= (f2cl-lib:int-add k 1) ihi)
            (setf tst
                    (+ tst
                       (abs
                        (f2cl-lib:dble
                         (f2cl-lib:fref h-%data%
                                        ((f2cl-lib:int-add k 1) k)
                                        ((1 ldh) (1 *))
                                        h-%offset%))))))))
      (cond
       ((<=
         (abs
          (f2cl-lib:dble
           (f2cl-lib:fref h
                          (k
                           (f2cl-lib:int-add k
                                             (f2cl-lib:int-sub 1)))
                          ((1 ldh) (1 *)))))
```

```
 (* ulp tst))
(setf ab
        (max
         (cabs1
          (f2cl-lib:fref h-%data%
                         (k (f2cl-lib:int-sub k 1))
                         ((1 ldh) (1 *)) h-%offset%))
         (cabs1
          (f2cl-lib:fref h-%data%
                         ((f2cl-lib:int-sub k 1) k)
                         ((1 ldh) (1 *)) h-%offset%))))
(setf ba
        (min
         (cabs1
          (f2cl-lib:fref h-%data%
                         (k (f2cl-lib:int-sub k 1))
                         ((1 ldh) (1 *)) h-%offset%))
         (cabs1
          (f2cl-lib:fref h-%data%
                         ((f2cl-lib:int-sub k 1) k)
                         ((1 ldh) (1 *)) h-%offset%))))
(setf aa
        (max
         (cabs1
          (f2cl-lib:fref h-%data% (k k) ((1 ldh) (1 *))
                         h-%offset%))
         (cabs1
          (-
           (f2cl-lib:fref h-%data%
                          ((f2cl-lib:int-sub k 1)
                           (f2cl-lib:int-sub k 1))
                          ((1 ldh) (1 *)) h-%offset%)
           (f2cl-lib:fref h-%data% (k k) ((1 ldh) (1 *))
                          h-%offset%)))))
(setf bb
        (min
         (cabs1
          (f2cl-lib:fref h-%data% (k k) ((1 ldh) (1 *))
                         h-%offset%))
         (cabs1
          (-
           (f2cl-lib:fref h-%data%
                          ((f2cl-lib:int-sub k 1)
                           (f2cl-lib:int-sub k 1))
                          ((1 ldh) (1 *)) h-%offset%)
           (f2cl-lib:fref h-%data% (k k) ((1 ldh) (1 *))
                          h-%offset%)))))
(setf s (+ aa ab))
(if (<= (* ba (/ ab s))
        (max smlnum (* ulp (* bb (/ aa s)))))
```

```
            (go label50))))
     label40))
  label50
   (setf l k)
   (cond
    ((> l ilo)
     (setf (f2cl-lib:fref h-%data% (l (f2cl-lib:int-sub l 1))
                          ((1 ldh) (1 *)) h-%offset%)
            zero)))
   (if (>= l i)
       (go label140))
   (cond ((not wantt) (setf i1 l) (setf i2 i)))
   (cond
    ((= its 10)
     (setf s
            (* dat1
               (abs
                (f2cl-lib:dble
                 (f2cl-lib:fref h-%data% ((f2cl-lib:int-add l 1) l)
                                ((1 ldh) (1 *)) h-%offset%)))))
     (setf t$
            (+ s
               (f2cl-lib:fref h-%data% (l l) ((1 ldh) (1 *))
                              h-%offset%))))
    ((= its 20)
     (setf s
            (* dat1
               (abs
                (f2cl-lib:dble
                 (f2cl-lib:fref h-%data% (i (f2cl-lib:int-sub i 1))
                                ((1 ldh) (1 *)) h-%offset%)))))
     (setf t$
            (+ s
               (f2cl-lib:fref h-%data% (i i) ((1 ldh) (1 *))
                              h-%offset%))))
    (t
     (setf t$
            (f2cl-lib:fref h-%data% (i i) ((1 ldh) (1 *))
                           h-%offset%))
     (setf u
            (*
             (f2cl-lib:fsqrt
              (f2cl-lib:fref h-%data% ((f2cl-lib:int-sub i 1) i)
                             ((1 ldh) (1 *)) h-%offset%))
             (f2cl-lib:fsqrt
              (f2cl-lib:fref h-%data% (i (f2cl-lib:int-sub i 1))
                             ((1 ldh) (1 *)) h-%offset%))))
     (setf s (cabs1 u))
     (cond
      ((/= s rzero)
```

```
    (setf x
            (* half
               (-
                (f2cl-lib:fref h-%data%
                               ((f2cl-lib:int-sub i 1)
                                (f2cl-lib:int-sub i 1))
                               ((1 ldh) (1 *)) h-%offset%)
                t$)))
    (setf sx (cabs1 x)) (setf s (max s (cabs1 x)))
    (setf y
            (* s
               (f2cl-lib:fsqrt
                (+ (expt (/ x s) 2) (expt (/ u s) 2)))))
    (cond
     ((> sx rzero)
      (if (<
           (+ (* (f2cl-lib:dble (/ x sx)) (f2cl-lib:dble y))
              (* (f2cl-lib:dimag (/ x sx)) (f2cl-lib:dimag y)))
           rzero)
          (setf y (- y)))))
    (setf t$ (- t$ (* u (zladiv u (+ x y))))))))
(f2cl-lib:fdo (m (f2cl-lib:int-add i (f2cl-lib:int-sub 1))
               (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
              ((> m (f2cl-lib:int-add l 1)) nil)
  (tagbody
    (setf h11
            (f2cl-lib:fref h-%data% (m m) ((1 ldh) (1 *))
                           h-%offset%))
    (setf h22
            (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-add m 1)
                            (f2cl-lib:int-add m 1))
                           ((1 ldh) (1 *)) h-%offset%))
    (setf h11s (- h11 t$))
    (setf h21
            (f2cl-lib:dble
             (f2cl-lib:fref h-%data% ((f2cl-lib:int-add m 1) m)
                            ((1 ldh) (1 *)) h-%offset%)))
    (setf s (+ (cabs1 h11s) (abs h21)))
    (setf h11s (/ h11s s))
    (setf h21 (/ h21 s))
    (setf (f2cl-lib:fref v (1) ((1 2))) h11s)
    (setf (f2cl-lib:fref v (2) ((1 2)))
            (coerce h21 'f2cl-lib:complex16))
    (setf h10
            (f2cl-lib:dble
             (f2cl-lib:fref h-%data% (m (f2cl-lib:int-sub m 1))
                            ((1 ldh) (1 *)) h-%offset%)))
    (if (<= (* (abs h10) (abs h21))
            (* ulp (* (cabs1 h11s) (+ (cabs1 h11) (cabs1 h22)))))
```

```
        (go label70))
    label60))
 (setf h11
        (f2cl-lib:fref h-%data% (l l) ((1 ldh) (1 *))
                       h-%offset%))
 (setf h22
        (f2cl-lib:fref h-%data%
                       ((f2cl-lib:int-add l 1)
                        (f2cl-lib:int-add l 1))
                       ((1 ldh) (1 *)) h-%offset%))
 (setf h11s (- h11 t$))
 (setf h21
        (f2cl-lib:dble
         (f2cl-lib:fref h-%data% ((f2cl-lib:int-add l 1) l)
                        ((1 ldh) (1 *)) h-%offset%)))
 (setf s (+ (cabs1 h11s) (abs h21)))
 (setf h11s (/ h11s s))
 (setf h21 (/ h21 s))
 (setf (f2cl-lib:fref v (1) ((1 2))) h11s)
 (setf (f2cl-lib:fref v (2) ((1 2)))
        (coerce h21 'f2cl-lib:complex16))
label70
 (f2cl-lib:fdo (k m (f2cl-lib:int-add k 1))
               ((> k (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (if (> k m)
         (zcopy 2
          (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                                (k (f2cl-lib:int-sub k 1))
                                ((1 ldh) (1 *)) h-%offset%)
          1 v 1))
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (zlarfg 2
          (f2cl-lib:array-slice v f2cl-lib:complex16 (1) ((1 2)))
          (f2cl-lib:array-slice v f2cl-lib:complex16 (2) ((1 2)))
          1 t1)
       (declare (ignore var-0 var-1 var-2 var-3))
       (when var-4 (setf t1 var-4)))
     (cond
      ((> k m)
       (setf (f2cl-lib:fref h-%data% (k (f2cl-lib:int-sub k 1))
                            ((1 ldh) (1 *)) h-%offset%)
             (f2cl-lib:fref v (1) ((1 2))))
       (setf (f2cl-lib:fref h-%data%
                            ((f2cl-lib:int-add k 1)
                             (f2cl-lib:int-sub k 1))
                            ((1 ldh) (1 *)) h-%offset%)
             zero)))
     (setf v2 (f2cl-lib:fref v (2) ((1 2))))
```

```
(setf t2 (f2cl-lib:dble (* t1 v2)))
(f2cl-lib:fdo (j k (f2cl-lib:int-add j 1))
              ((> j i2) nil)
  (tagbody
    (setf sum
            (+
             (* (f2cl-lib:dconjg t1)
                (f2cl-lib:fref h-%data% (k j) ((1 ldh) (1 *))
                               h-%offset%))
             (* t2
                (f2cl-lib:fref h-%data%
                               ((f2cl-lib:int-add k 1) j)
                               ((1 ldh) (1 *)) h-%offset%))))
    (setf (f2cl-lib:fref h-%data% (k j) ((1 ldh) (1 *))
                         h-%offset%)
            (-
             (f2cl-lib:fref h-%data% (k j) ((1 ldh) (1 *))
                            h-%offset%)
             sum))
    (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add k 1) j)
                         ((1 ldh) (1 *)) h-%offset%)
            (-
             (f2cl-lib:fref h-%data%
                            ((f2cl-lib:int-add k 1) j)
                            ((1 ldh) (1 *)) h-%offset%)
             (* sum v2)))
   label80))
(f2cl-lib:fdo (j i1 (f2cl-lib:int-add j 1))
              ((> j
                  (min
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add k 2))
                   (the f2cl-lib:integer4 i)))
               nil)
  (tagbody
    (setf sum
            (+
             (* t1
                (f2cl-lib:fref h-%data% (j k) ((1 ldh) (1 *))
                               h-%offset%))
             (* t2
                (f2cl-lib:fref h-%data%
                               (j (f2cl-lib:int-add k 1))
                               ((1 ldh) (1 *)) h-%offset%))))
    (setf (f2cl-lib:fref h-%data% (j k) ((1 ldh) (1 *))
                         h-%offset%)
            (-
             (f2cl-lib:fref h-%data% (j k) ((1 ldh) (1 *))
                            h-%offset%)
             sum))
```

```
    (setf (f2cl-lib:fref h-%data% (j (f2cl-lib:int-add k 1))
                         ((1 ldh) (1 *)) h-%offset%)
            (-
             (f2cl-lib:fref h-%data%
                            (j (f2cl-lib:int-add k 1))
                            ((1 ldh) (1 *)) h-%offset%)
             (* sum (f2cl-lib:dconjg v2))))
   label90))
(cond
 (wantz
  (f2cl-lib:fdo (j iloz (f2cl-lib:int-add j 1))
                ((> j ihiz) nil)
    (tagbody
      (setf sum
              (+
               (* t1
                  (f2cl-lib:fref z-%data% (j k)
                                 ((1 ldz) (1 *)) z-%offset%))
               (* t2
                  (f2cl-lib:fref z-%data%
                                 (j (f2cl-lib:int-add k 1))
                                 ((1 ldz) (1 *))
                                 z-%offset%))))
      (setf (f2cl-lib:fref z-%data% (j k) ((1 ldz) (1 *))
                           z-%offset%)
              (-
               (f2cl-lib:fref z-%data% (j k) ((1 ldz) (1 *))
                              z-%offset%)
               sum))
      (setf (f2cl-lib:fref z-%data%
                           (j (f2cl-lib:int-add k 1))
                           ((1 ldz) (1 *)) z-%offset%)
              (-
               (f2cl-lib:fref z-%data%
                              (j (f2cl-lib:int-add k 1))
                              ((1 ldz) (1 *)) z-%offset%)
               (* sum (f2cl-lib:dconjg v2))))
     label100))))
(cond
 ((and (= k m) (> m l)) (setf temp (- one t1))
  (setf temp (/ temp (abs temp)))
  (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add m 1) m)
                       ((1 ldh) (1 *)) h-%offset%)
          (*
           (f2cl-lib:fref h-%data% ((f2cl-lib:int-add m 1) m)
                          ((1 ldh) (1 *)) h-%offset%)
           (f2cl-lib:dconjg temp)))
  (if (<= (f2cl-lib:int-add m 2) i)
      (setf (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-add m 2)
```

```
                                    (f2cl-lib:int-add m 1))
                                   ((1 ldh) (1 *)) h-%offset%)
                          (*
                           (f2cl-lib:fref h-%data%
                                          ((f2cl-lib:int-add m 2)
                                           (f2cl-lib:int-add m 1))
                                          ((1 ldh) (1 *)) h-%offset%)
                           temp)))
                (f2cl-lib:fdo (j m (f2cl-lib:int-add j 1))
                              ((> j i) nil)
                  (tagbody
                    (cond
                     ((/= j (f2cl-lib:int-add m 1))
                      (if (> i2 j)
                          (multiple-value-bind (var-0 var-1 var-2 var-3)
                              (zscal (f2cl-lib:int-sub i2 j) temp
                               (f2cl-lib:array-slice h-%data%
                                                     f2cl-lib:complex16
                                                     (j
                                                      (f2cl-lib:int-add j
                                                                        1))
                                                     ((1 ldh) (1 *))
                                                     h-%offset%)
                               ldh)
                            (declare (ignore var-0 var-2))
                            (when var-1 (setf temp var-1))
                            (when var-3 (setf ldh var-3))))
                      (zscal (f2cl-lib:int-sub j i1) (f2cl-lib:dconjg temp)
                       (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                                             (i1 j) ((1 ldh) (1 *))
                                             h-%offset%)
                       1)
                      (cond
                       (wantz
                        (multiple-value-bind (var-0 var-1 var-2 var-3)
                            (zscal nz (f2cl-lib:dconjg temp)
                             (f2cl-lib:array-slice z-%data%
                                                   f2cl-lib:complex16
                                                   (iloz j) ((1 ldz) (1 *))
                                                   z-%offset%)
                             1)
                            (declare (ignore var-1 var-2 var-3))
                            (when var-0 (setf nz var-0)))))))
                 label110))))
          label120))
        (setf temp
                (f2cl-lib:fref h-%data% (i (f2cl-lib:int-sub i 1))
                               ((1 ldh) (1 *)) h-%offset%))
        (cond
         ((/= (f2cl-lib:dimag temp) rzero) (setf rtemp (abs temp))
```

```
        (setf (f2cl-lib:fref h-%data% (i (f2cl-lib:int-sub i 1))
                             ((1 ldh) (1 *)) h-%offset%)
                (coerce rtemp 'f2cl-lib:complex16))
        (setf temp (/ temp rtemp))
        (if (> i2 i)
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (zscal (f2cl-lib:int-sub i2 i) (f2cl-lib:dconjg temp)
                 (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                                       (i (f2cl-lib:int-add i 1))
                                       ((1 ldh) (1 *)) h-%offset%)
                 ldh)
              (declare (ignore var-0 var-1 var-2))
              (when var-3 (setf ldh var-3))))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (zscal (f2cl-lib:int-sub i i1) temp
             (f2cl-lib:array-slice h-%data% f2cl-lib:complex16 (i1 i)
                                   ((1 ldh) (1 *)) h-%offset%)
             1)
          (declare (ignore var-0 var-2 var-3))
          (when var-1 (setf temp var-1)))
        (cond
         (wantz
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (zscal nz temp
               (f2cl-lib:array-slice z-%data% f2cl-lib:complex16
                                     (iloz i) ((1 ldz) (1 *))
                                     z-%offset%)
               1)
            (declare (ignore var-2 var-3))
            (when var-0 (setf nz var-0))
            (when var-1 (setf temp var-1)))))))
    label130))
 (setf info i)
 (go end_label)
label140
 (setf (f2cl-lib:fref w-%data% (i) ((1 *)) w-%offset%)
         (f2cl-lib:fref h-%data% (i i) ((1 ldh) (1 *)) h-%offset%))
 (setf i (f2cl-lib:int-sub l 1))
 (go label30)
label150
 (go end_label)
end_label
 (return
  (values nil nil nil nil nil nil ldh nil nil nil nil nil info)))))))
```

———-

90.221 zlahr2

90.221.1 Axiom unit tests

— zlahr2.input —

```
)set break resume
)sys rm -f zlahr2.output
)spool zlahr2.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.221.2 Axiom help page

— zlahr2.help —

```
=====================================================================
zlahr2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZLAHR2( N, K, NB, A, LDA, TAU, T, LDT, Y, LDY )

       .. Scalar Arguments ..
       INTEGER            K, LDA, LDT, LDY, N, NB
       ..
       .. Array Arguments ..
       COMPLEX*16        A( LDA, * ), T( LDT, NB ), TAU( NB ),
      $                   Y( LDY, NB )
       ..
```

```
Purpose:
=============

 ZLAHR2 reduces the first NB columns of A complex general n-BY-(n-k+1)
 matrix A so that elements below the k-th subdiagonal are zero. The
 reduction is performed by an unitary similarity transformation
 Q**H * A * Q. The routine returns the matrices V and T which determine
 Q as a block reflector I - V*T*V**H, and also the matrix Y = A * V * T.

 This is an auxiliary routine called by ZGEHRD.

Arguments:
==========

 [in] N

         N is INTEGER
         The order of the matrix A.

 [in] K

         K is INTEGER
         The offset for the reduction. Elements below the k-th
         subdiagonal in the first NB columns are reduced to zero.
         K < N.

 [in] NB

         NB is INTEGER
         The number of columns to be reduced.

 [in,out] A

         A is COMPLEX*16 array, dimension (LDA,N-K+1)
         On entry, the n-by-(n-k+1) general matrix A.
         On exit, the elements on and above the k-th subdiagonal in
         the first NB columns are overwritten with the corresponding
         elements of the reduced matrix; the elements below the k-th
         subdiagonal, with the array TAU, represent the matrix Q as a
         product of elementary reflectors. The other columns of A are
         unchanged. See Further Details.

 [in] LDA

         LDA is INTEGER
         The leading dimension of the array A.  LDA >= max(1,N).

 [out] TAU
```

```
          TAU is COMPLEX*16 array, dimension (NB)
          The scalar factors of the elementary reflectors. See Further
          Details.

  [out] T

          T is COMPLEX*16 array, dimension (LDT,NB)
          The upper triangular matrix T.

  [in] LDT

          LDT is INTEGER
          The leading dimension of the array T.  LDT >= NB.

  [out] Y

          Y is COMPLEX*16 array, dimension (LDY,NB)
          The n-by-nb matrix Y.

  [in] LDY

          LDY is INTEGER
          The leading dimension of the array Y. LDY >= N.

 Further Details:
 ====================

  The matrix Q is represented as a product of nb elementary reflectors

     Q = H(1) H(2) . . . H(nb).

  Each H(i) has the form

     H(i) = I - tau * v * v**H

  where tau is a complex scalar, and v is a complex vector with
  v(1:i+k-1) = 0, v(i+k) = 1; v(i+k+1:n) is stored on exit in
  A(i+k+1:n,i), and tau in TAU(i).

  The elements of the vectors v together form the (n-k+1)-by-nb matrix
  V which is needed, with T and Y, to apply the transformation to the
  unreduced part of the matrix, using an update of the form:
  A := (I - V*T*V**H) * (A - Y*V**H).

  The contents of A on exit are illustrated by the following example
  with n = 7, k = 3 and nb = 2:

     ( a   a   a   a   a )
     ( a   a   a   a   a )
     ( a   a   a   a   a )
```

```
    ( h   h   a   a   a )
    ( v1  h   a   a   a )
    ( v1  v2  a   a   a )
    ( v1  v2  a   a   a )

 where a denotes an element of the original matrix A, h denotes a
 modified element of the upper Hessenberg matrix H, and vi denotes an
 element of the vector defining H(i).

 This subroutine is a slight modification of LAPACK-3.0's DLAHRD
 incorporating improvements proposed by Quintana-Orti and Van de
 Gejin. Note that the entries of A(1:K,2:NB) differ from those
 returned by the original LAPACK-3.0's DLAHRD routine. (This
 subroutine is not backward compatible with LAPACK-3.0's DLAHRD.)

References:
================

 Gregorio Quintana-Orti and Robert van de Geijn, "Improving the
 performance of reduction to Hessenberg form," ACM Transactions on
 Mathematical Software, 32(2):180-194, June 2006.
```

90.221.3 fortran code

— zlahr2.f —

```
*  =====================================================================
      SUBROUTINE ZLAHR2( N, K, NB, A, LDA, TAU, T, LDT, Y, LDY )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            K, LDA, LDT, LDY, N, NB
*     ..
*     .. Array Arguments ..
      COMPLEX*16        A( LDA, * ), T( LDT, NB ), TAU( NB ),
     $                   Y( LDY, NB )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
```

```
      COMPLEX*16        ZERO, ONE
      PARAMETER          ( ZERO = ( 0.0D+0, 0.0D+0 ),
     $                     ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I
      COMPLEX*16        EI
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZAXPY, ZCOPY, ZGEMM, ZGEMV, ZLACPY,
     $                   ZLARFG, ZSCAL, ZTRMM, ZTRMV, ZLACGV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MIN
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( N.LE.1 )
     $   RETURN
*
      DO 10 I = 1, NB
         IF( I.GT.1 ) THEN
*
*           Update A(K+1:N,I)
*
*           Update I-th column of A - Y * V**H
*
            CALL ZLACGV( I-1, A( K+I-1, 1 ), LDA )
            CALL ZGEMV( 'NO TRANSPOSE', N-K, I-1, -ONE, Y(K+1,1), LDY,
     $                  A( K+I-1, 1 ), LDA, ONE, A( K+1, I ), 1 )
            CALL ZLACGV( I-1, A( K+I-1, 1 ), LDA )
*
*           Apply I - V * T**H * V**H to this column (call it b) from the
*           left, using the last column of T as workspace
*
*           Let  V = ( V1 )   and   b = ( b1 )   (first I-1 rows)
*                    ( V2 )             ( b2 )
*
*           where V1 is unit lower triangular
*
*           w := V1**H * b1
*
            CALL ZCOPY( I-1, A( K+1, I ), 1, T( 1, NB ), 1 )
            CALL ZTRMV( 'Lower', 'Conjugate transpose', 'UNIT',
     $                  I-1, A( K+1, 1 ),
     $                  LDA, T( 1, NB ), 1 )
*
*           w := w + V2**H * b2
```

```
*
            CALL ZGEMV( 'Conjugate transpose', N-K-I+1, I-1,
     $                  ONE, A( K+I, 1 ),
     $                  LDA, A( K+I, I ), 1, ONE, T( 1, NB ), 1 )
*
*           w := T**H * w
*
            CALL ZTRMV( 'Upper', 'Conjugate transpose', 'NON-UNIT',
     $                  I-1, T, LDT,
     $                  T( 1, NB ), 1 )
*
*           b2 := b2 - V2*w
*
            CALL ZGEMV( 'NO TRANSPOSE', N-K-I+1, I-1, -ONE,
     $                  A( K+I, 1 ),
     $                  LDA, T( 1, NB ), 1, ONE, A( K+I, I ), 1 )
*
*           b1 := b1 - V1*w
*
            CALL ZTRMV( 'Lower', 'NO TRANSPOSE',
     $                  'UNIT', I-1,
     $                  A( K+1, 1 ), LDA, T( 1, NB ), 1 )
            CALL ZAXPY( I-1, -ONE, T( 1, NB ), 1, A( K+1, I ), 1 )
*
            A( K+I-1, I-1 ) = EI
         END IF
*
*        Generate the elementary reflector H(I) to annihilate
*        A(K+I+1:N,I)
*
         CALL ZLARFG( N-K-I+1, A( K+I, I ), A( MIN( K+I+1, N ), I ), 1,
     $               TAU( I ) )
         EI = A( K+I, I )
         A( K+I, I ) = ONE
*
*        Compute  Y(K+1:N,I)
*
         CALL ZGEMV( 'NO TRANSPOSE', N-K, N-K-I+1,
     $              ONE, A( K+1, I+1 ),
     $              LDA, A( K+I, I ), 1, ZERO, Y( K+1, I ), 1 )
         CALL ZGEMV( 'Conjugate transpose', N-K-I+1, I-1,
     $              ONE, A( K+I, 1 ), LDA,
     $              A( K+I, I ), 1, ZERO, T( 1, I ), 1 )
         CALL ZGEMV( 'NO TRANSPOSE', N-K, I-1, -ONE,
     $              Y( K+1, 1 ), LDY,
     $              T( 1, I ), 1, ONE, Y( K+1, I ), 1 )
         CALL ZSCAL( N-K, TAU( I ), Y( K+1, I ), 1 )
*
*        Compute T(1:I,I)
*
```

```
         CALL ZSCAL( I-1, -TAU( I ), T( 1, I ), 1 )
         CALL ZTRMV( 'Upper', 'No Transpose', 'NON-UNIT',
     $               I-1, T, LDT,
     $               T( 1, I ), 1 )
         T( I, I ) = TAU( I )
*
   10 CONTINUE
      A( K+NB, NB ) = EI
*
*     Compute Y(1:K,1:NB)
*
      CALL ZLACPY( 'ALL', K, NB, A( 1, 2 ), LDA, Y, LDY )
      CALL ZTRMM( 'RIGHT', 'Lower', 'NO TRANSPOSE',
     $            'UNIT', K, NB,
     $            ONE, A( K+1, 1 ), LDA, Y, LDY )
      IF( N.GT.K+NB )
     $   CALL ZGEMM( 'NO TRANSPOSE', 'NO TRANSPOSE', K,
     $               NB, N-K-NB, ONE,
     $               A( 1, 2+NB ), LDA, A( K+1+NB, 1 ), LDA, ONE, Y,
     $               LDY )
      CALL ZTRMM( 'RIGHT', 'Upper', 'NO TRANSPOSE',
     $            'NON-UNIT', K, NB,
     $            ONE, T, LDT, Y, LDY )
*
      RETURN
*
*     End of ZLAHR2
*
      END
```

———-

90.221.4 lisp code

— zlahr2.lisp —

```
(let* ((zero (f2cl-lib:cmplx 0.0d0 0.0d0)) (one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) zero)
           (type (f2cl-lib:complex16) one)
           (ignorable zero one))
  (defun zlahr2 (n k nb a lda tau t$ ldt y ldy)
    (declare (type (array f2cl-lib:complex16 (*)) y t$ tau a)
             (type (f2cl-lib:integer4) ldy ldt lda nb k n))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
         (t$ f2cl-lib:complex16 t$-%data% t$-%offset%)
```

```
    (y f2cl-lib:complex16 y-%data% y-%offset%))
  (prog ((ei #C(0.0d0 0.0d0)) (i 0))
    (declare (type (f2cl-lib:complex16) ei)
             (type (f2cl-lib:integer4) i))
    (if (<= n 1)
        (go end_label))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i nb) nil)
      (tagbody
        (cond
         ((> i 1)
          (zlacgv (f2cl-lib:int-sub i 1)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                 ((+ k i (f2cl-lib:int-sub 1)) 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
               var-10)
              (zgemv "NO TRANSPOSE" (f2cl-lib:int-sub n k)
               (f2cl-lib:int-sub i 1) (- one)
               (f2cl-lib:array-slice y-%data% f2cl-lib:complex16
                                     ((+ k 1) 1) ((1 ldy) (1 nb))
                                     y-%offset%)
               ldy
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                     ((+ k i (f2cl-lib:int-sub 1)) 1)
                                     ((1 lda) (1 *)) a-%offset%)
               lda one
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                     ((+ k 1) i) ((1 lda) (1 *))
                                     a-%offset%)
               1)
            (declare
             (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-9 var-10))
            (when var-5 (setf ldy var-5))
            (when var-7 (setf lda var-7))
            (when var-8 (setf one var-8)))
          (zlacgv (f2cl-lib:int-sub i 1)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                 ((+ k i (f2cl-lib:int-sub 1)) 1)
                                 ((1 lda) (1 *)) a-%offset%)
           lda)
          (zcopy (f2cl-lib:int-sub i 1)
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 ((+ k 1) i)
                                 ((1 lda) (1 *)) a-%offset%)
           1
           (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 nb)
                                 ((1 ldt) (1 nb)) t$-%offset%)
           1)
```

```
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (ztrmv "Lower" "Conjugate transpose" "UNIT"
     (f2cl-lib:int-sub i 1)
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                           ((+ k 1) 1) ((1 lda) (1 *))
                           a-%offset%)
     lda
     (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 nb)
                           ((1 ldt) (1 nb)) t$-%offset%)
     1)
  (declare (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7))
  (when var-5 (setf lda var-5)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgemv "Conjugate transpose"
     (f2cl-lib:int-add (f2cl-lib:int-sub n k i) 1)
     (f2cl-lib:int-sub i 1) one
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                           ((+ k i) 1) ((1 lda) (1 *))
                           a-%offset%)
     lda
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                           ((+ k i) i) ((1 lda) (1 *))
                           a-%offset%)
     1 one
     (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 nb)
                           ((1 ldt) (1 nb)) t$-%offset%)
     1)
  (declare
   (ignore var-0 var-1 var-2 var-4 var-6 var-7 var-9 var-10))
  (when var-3 (setf one var-3))
  (when var-5 (setf lda var-5))
  (when var-8 (setf one var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
    (ztrmv "Upper" "Conjugate transpose" "NON-UNIT"
     (f2cl-lib:int-sub i 1) t$ ldt
     (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 nb)
                           ((1 ldt) (1 nb)) t$-%offset%)
     1)
  (declare (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7))
  (when var-5 (setf ldt var-5)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgemv "NO TRANSPOSE"
     (f2cl-lib:int-add (f2cl-lib:int-sub n k i) 1)
     (f2cl-lib:int-sub i 1) (- one)
```

```
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                              ((+ k i) 1) ((1 lda) (1 *))
                              a-%offset%)
        lda
        (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 nb)
                              ((1 ldt) (1 nb)) t$-%offset%)
        1 one
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                              ((+ k i) i) ((1 lda) (1 *))
                              a-%offset%)
        1)
      (declare
       (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7 var-9
        var-10))
      (when var-5 (setf lda var-5))
      (when var-8 (setf one var-8)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (ztrmv "Lower" "NO TRANSPOSE" "UNIT" (f2cl-lib:int-sub i 1)
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                              ((+ k 1) 1) ((1 lda) (1 *))
                              a-%offset%)
        lda
        (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 nb)
                              ((1 ldt) (1 nb)) t$-%offset%)
        1)
      (declare (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7))
      (when var-5 (setf lda var-5)))
    (zaxpy (f2cl-lib:int-sub i 1) (- one)
     (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 nb)
                           ((1 ldt) (1 nb)) t$-%offset%)
     1
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 ((+ k 1) i)
                           ((1 lda) (1 *)) a-%offset%)
     1)
    (setf (f2cl-lib:fref a-%data%
                         ((f2cl-lib:int-sub (f2cl-lib:int-add k i) 1)
                          (f2cl-lib:int-sub i 1))
                         ((1 lda) (1 *)) a-%offset%)
          ei)))
  (zlarfg (f2cl-lib:int-add (f2cl-lib:int-sub n k i) 1)
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 ((+ k i) i)
                         ((1 lda) (1 *)) a-%offset%)
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                         ((min (f2cl-lib:int-add k i 1) n) i)
                         ((1 lda) (1 *)) a-%offset%)
   1
   (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i) ((1 nb))
                         tau-%offset%))
  (setf ei
```

```
        (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k i) i)
                       ((1 lda) (1 *)) a-%offset%))
(setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k i) i)
                     ((1 lda) (1 *)) a-%offset%)
        one)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgemv "NO TRANSPOSE" (f2cl-lib:int-sub n k)
     (f2cl-lib:int-add (f2cl-lib:int-sub n k i) 1) one
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                           ((+ k 1) (f2cl-lib:int-add i 1))
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 ((+ k i) i)
                           ((1 lda) (1 *)) a-%offset%)
     1 zero
     (f2cl-lib:array-slice y-%data% f2cl-lib:complex16 ((+ k 1) i)
                           ((1 ldy) (1 nb)) y-%offset%)
     1)
  (declare
   (ignore var-0 var-1 var-2 var-4 var-6 var-7 var-9 var-10))
  (when var-3 (setf one var-3))
  (when var-5 (setf lda var-5))
  (when var-8 (setf zero var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgemv "Conjugate transpose"
     (f2cl-lib:int-add (f2cl-lib:int-sub n k i) 1)
     (f2cl-lib:int-sub i 1) one
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 ((+ k i) 1)
                           ((1 lda) (1 *)) a-%offset%)
     lda
     (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 ((+ k i) i)
                           ((1 lda) (1 *)) a-%offset%)
     1 zero
     (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 i)
                           ((1 ldt) (1 nb)) t$-%offset%)
     1)
  (declare
   (ignore var-0 var-1 var-2 var-4 var-6 var-7 var-9 var-10))
  (when var-3 (setf one var-3))
  (when var-5 (setf lda var-5))
  (when var-8 (setf zero var-8)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (zgemv "NO TRANSPOSE" (f2cl-lib:int-sub n k)
     (f2cl-lib:int-sub i 1) (- one)
```

```
          (f2cl-lib:array-slice y-%data% f2cl-lib:complex16 ((+ k 1) 1)
                                ((1 ldy) (1 nb)) y-%offset%)
          ldy
          (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 i)
                                ((1 ldt) (1 nb)) t$-%offset%)
          1 one
          (f2cl-lib:array-slice y-%data% f2cl-lib:complex16 ((+ k 1) i)
                                ((1 ldy) (1 nb)) y-%offset%)
          1)
        (declare
         (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7 var-9 var-10))
        (when var-5 (setf ldy var-5))
        (when var-8 (setf one var-8)))
      (zscal (f2cl-lib:int-sub n k)
       (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i) ((1 nb))
                             tau-%offset%)
       (f2cl-lib:array-slice y-%data% f2cl-lib:complex16 ((+ k 1) i)
                             ((1 ldy) (1 nb)) y-%offset%)
       1)
      (zscal (f2cl-lib:int-sub i 1)
       (- (f2cl-lib:fref tau-%data% (i) ((1 nb)) tau-%offset%))
       (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 i)
                             ((1 ldt) (1 nb)) t$-%offset%)
       1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (ztrmv "Upper" "No Transpose" "NON-UNIT" (f2cl-lib:int-sub i 1)
           t$ ldt
           (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 i)
                                 ((1 ldt) (1 nb)) t$-%offset%)
           1)
        (declare (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7))
        (when var-5 (setf ldt var-5)))
      (setf (f2cl-lib:fref t$-%data% (i i) ((1 ldt) (1 nb)) t$-%offset%)
              (f2cl-lib:fref tau-%data% (i) ((1 nb)) tau-%offset%))
    label10))
  (setf (f2cl-lib:fref a-%data% ((f2cl-lib:int-add k nb) nb)
                        ((1 lda) (1 *)) a-%offset%)
          ei)
  (zlacpy "ALL" k nb
   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 2)
                          ((1 lda) (1 *)) a-%offset%)
   lda y ldy)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ztrmm "RIGHT" "Lower" "NO TRANSPOSE" "UNIT" k nb one
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 ((+ k 1) 1)
                             ((1 lda) (1 *)) a-%offset%)
       lda y ldy)
```

```
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf k var-4))
    (when var-5 (setf nb var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf lda var-8))
    (when var-10 (setf ldy var-10)))
  (if (> n (f2cl-lib:int-add k nb))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12)
          (zgemm "NO TRANSPOSE" "NO TRANSPOSE" k nb
           (f2cl-lib:int-sub n k nb) one
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                 (1 (f2cl-lib:int-add 2 nb))
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                 ((+ k 1 nb) 1) ((1 lda) (1 *))
                                 a-%offset%)
           lda one y ldy)
        (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
        (when var-2 (setf k var-2))
        (when var-3 (setf nb var-3))
        (when var-5 (setf one var-5))
        (when var-7 (setf lda var-7))
        (when var-9 (setf lda var-9))
        (when var-10 (setf one var-10))
        (when var-12 (setf ldy var-12))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ztrmm "RIGHT" "Upper" "NO TRANSPOSE" "NON-UNIT" k nb one t$ ldt y
       ldy)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf k var-4))
    (when var-5 (setf nb var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf ldt var-8))
    (when var-10 (setf ldy var-10)))
  (go end_label)
 end_label
  (return (values nil k nb nil lda nil nil ldt nil ldy))))))
```

———-

90.222 zlange

90.222.1 Axiom unit tests

— zlange.input —

```
)set break resume
)sys rm -f zlange.output
)spool zlange.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.222.2 Axiom help page

— zlange.help —

```
======================================================================
zlange examples
======================================================================

======================================================================
Man Page Details
======================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       DOUBLE PRECISION FUNCTION ZLANGE( NORM, M, N, A, LDA, WORK )

      .. Scalar Arguments ..
       CHARACTER          NORM
       INTEGER            LDA, M, N
       ..
       .. Array Arguments ..
       DOUBLE PRECISION   WORK( * )
       COMPLEX*16         A( LDA, * )
       ..
```

```
Purpose:
=============

ZLANGE  returns the value of the one norm,  or the Frobenius norm, or
the  infinity norm,  or the  element of  largest absolute value  of a
complex matrix A.


   ZLANGE = ( max(abs(A(i,j))), NORM = 'M' or 'm'
            (
            ( norm1(A),         NORM = '1', 'O' or 'o'
            (
            ( normI(A),         NORM = 'I' or 'i'
            (
            ( normF(A),         NORM = 'F', 'f', 'E' or 'e'

where  norm1  denotes the  one norm of a matrix (maximum column sum),
normI  denotes the  infinity norm  of a matrix  (maximum row sum) and
normF  denotes the  Frobenius norm of a matrix (square root of sum of
squares).  Note that  max(abs(A(i,j)))  is not a consistent matrix norm.

Arguments:
==========

 [in] NORM

         NORM is CHARACTER*1
         Specifies the value to be returned in ZLANGE as described
         above.

 [in] M

         M is INTEGER
         The number of rows of the matrix A.  M >= 0.  When M = 0,
         ZLANGE is set to zero.

 [in] N

         N is INTEGER
         The number of columns of the matrix A.  N >= 0.  When N = 0,
         ZLANGE is set to zero.

 [in] A

         A is COMPLEX*16 array, dimension (LDA,N)
         The m by n matrix A.

 [in] LDA
```

```
          LDA is INTEGER
          The leading dimension of the array A.  LDA >= max(M,1).

  [out] WORK

          WORK is DOUBLE PRECISION array, dimension (MAX(1,LWORK)),
          where LWORK >= M when NORM = 'I'; otherwise, WORK is not
          referenced.
```

———-

90.222.3 fortran code

— **zlange.f** —

```
*  =====================================================================
      DOUBLE PRECISION FUNCTION ZLANGE( NORM, M, N, A, LDA, WORK )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          NORM
      INTEGER            LDA, M, N
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   WORK( * )
      COMPLEX*16         A( LDA, * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J
      DOUBLE PRECISION   SCALE, SUM, VALUE
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
```

```
*     .. External Subroutines ..
      EXTERNAL           ZLASSQ
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN, SQRT
*     ..
*     .. Executable Statements ..
*
      IF( MIN( M, N ).EQ.0 ) THEN
         VALUE = ZERO
      ELSE IF( LSAME( NORM, 'M' ) ) THEN
*
*        Find max(abs(A(i,j))).
*
         VALUE = ZERO
         DO 20 J = 1, N
            DO 10 I = 1, M
               VALUE = MAX( VALUE, ABS( A( I, J ) ) )
   10       CONTINUE
   20    CONTINUE
      ELSE IF( ( LSAME( NORM, 'O' ) ) .OR. ( NORM.EQ.'1' ) ) THEN
*
*        Find norm1(A).
*
         VALUE = ZERO
         DO 40 J = 1, N
            SUM = ZERO
            DO 30 I = 1, M
               SUM = SUM + ABS( A( I, J ) )
   30       CONTINUE
            VALUE = MAX( VALUE, SUM )
   40    CONTINUE
      ELSE IF( LSAME( NORM, 'I' ) ) THEN
*
*        Find normI(A).
*
         DO 50 I = 1, M
            WORK( I ) = ZERO
   50    CONTINUE
         DO 70 J = 1, N
            DO 60 I = 1, M
               WORK( I ) = WORK( I ) + ABS( A( I, J ) )
   60       CONTINUE
   70    CONTINUE
         VALUE = ZERO
         DO 80 I = 1, M
            VALUE = MAX( VALUE, WORK( I ) )
   80    CONTINUE
      ELSE IF( ( LSAME( NORM, 'F' ) ) .OR. ( LSAME( NORM, 'E' ) ) ) THEN
*
```

```
*        Find normF(A).
*
         SCALE = ZERO
         SUM = ONE
         DO 90 J = 1, N
            CALL ZLASSQ( M, A( 1, J ), 1, SCALE, SUM )
   90    CONTINUE
         VALUE = SCALE*SQRT( SUM )
      END IF
*
      ZLANGE = VALUE
      RETURN
*
*     End of ZLANGE
*
      END
```

———-

90.222.4 lisp code

— zlange.lisp —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun zlange (norm m n a lda work)
    (declare (type (array double-float (*)) work)
             (type (array f2cl-lib:complex16 (*)) a)
             (type (f2cl-lib:integer4) lda n m)
             (type (string *) norm))
    (f2cl-lib:with-multi-array-data
        ((norm character norm-%data% norm-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (work double-float work-%data% work-%offset%))
      (prog ((scale 0.0d0) (sum 0.0d0) (value 0.0d0) (temp 0.0d0) (i 0) (j 0)
             (zlange 0.0d0))
        (declare (type (f2cl-lib:integer4) i j)
                 (type (double-float) scale sum value temp zlange))
        (cond
         ((= (min (the f2cl-lib:integer4 m) (the f2cl-lib:integer4 n)) 0)
          (setf value zero))
         ((multiple-value-bind (ret-val var-0 var-1)
              (lsame norm "M")
            (declare (ignore var-1))
            (when var-0 (setf norm var-0))
```

```
    ret-val)
  (setf value zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf temp
                  (abs
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)))
          (if (or (< value temp)
                  (multiple-value-bind (ret-val var-0)
                      (disnan temp)
                    (declare (ignore))
                    (when var-0 (setf temp var-0))
                    ret-val))
              (setf value temp))
        label10))
     label20)))
((or
  (multiple-value-bind (ret-val var-0 var-1)
      (lsame norm "O")
    (declare (ignore var-1))
    (when var-0 (setf norm var-0))
    ret-val)
  (f2cl-lib:fstring-= norm "1"))
  (setf value zero)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (setf sum zero)
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf sum
                  (+ sum
                     (abs
                      (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                     a-%offset%))))
        label30))
      (if (or (< value sum)
              (multiple-value-bind (ret-val var-0)
                  (disnan sum)
                (declare (ignore))
                (when var-0 (setf sum var-0))
                ret-val))
          (setf value sum))
     label40)))
```

```
((multiple-value-bind (ret-val var-0 var-1)
     (lsame norm "I")
   (declare (ignore var-1))
   (when var-0 (setf norm var-0))
   ret-val)
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i m) nil)
   (tagbody
     (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%) zero)
    label50))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i m) nil)
       (tagbody
         (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                 (+
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%)
                  (abs
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%))))
        label60))
    label70))
 (setf value zero)
 (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
               ((> i m) nil)
   (tagbody
     (setf temp (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%))
     (if (or (< value temp)
             (multiple-value-bind (ret-val var-0)
                 (disnan temp)
               (declare (ignore))
               (when var-0 (setf temp var-0))
               ret-val))
         (setf value temp))
    label80)))
((or
  (multiple-value-bind (ret-val var-0 var-1)
      (lsame norm "F")
    (declare (ignore var-1))
    (when var-0 (setf norm var-0))
    ret-val)
  (multiple-value-bind (ret-val var-0 var-1)
      (lsame norm "E")
    (declare (ignore var-1))
    (when var-0 (setf norm var-0))
    ret-val))
 (setf scale zero) (setf sum one)
```

```
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
              (zlassq m
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 j)
                                     ((1 lda) (1 *)) a-%offset%)
               1 scale sum)
            (declare (ignore var-1 var-2))
            (when var-0 (setf m var-0))
            (when var-3 (setf scale var-3))
            (when var-4 (setf sum var-4)))
         label90))
      (setf value (* scale (f2cl-lib:fsqrt sum)))))
    (setf zlange value)
    (go end_label)
   end_label
    (return (values zlange norm m nil nil nil nil))))))
```

———-

90.223 zlaqr0

See Braman [Bram02a, Bram02b]

90.223.1 Axiom unit tests

— **zlaqr0.input** —

```
)set break resume
)sys rm -f zlaqr0.output
)spool zlaqr0.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.223.2 Axiom help page

— zlaqr0.help —

```
=====================================================================
zlaqr0 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZLAQR0( WANTT, WANTZ, N, ILO, IHI, H, LDH, W, ILOZ,
                          IHIZ, Z, LDZ, WORK, LWORK, INFO )

       .. Scalar Arguments ..
       INTEGER            IHI, IHIZ, ILO, ILOZ, INFO, LDH, LDZ, LWORK, N
       LOGICAL            WANTT, WANTZ
       ..
       .. Array Arguments ..
       COMPLEX*16         H( LDH, * ), W( * ), WORK( * ), Z( LDZ, * )
       ..


 Purpose:
 =============

    ZLAQR0 computes the eigenvalues of a Hessenberg matrix H
    and, optionally, the matrices T and Z from the Schur decomposition
    H = Z T Z**H, where T is an upper triangular matrix (the
    Schur form), and Z is the unitary matrix of Schur vectors.

    Optionally Z may be postmultiplied into an input unitary
    matrix Q so that this routine can give the Schur factorization
    of a matrix A which has been reduced to the Hessenberg form H
    by the unitary matrix Q:  A = Q*H*Q**H = (QZ)*H*(QZ)**H.

 Arguments:
 ==========

  [in] WANTT

          WANTT is LOGICAL
```

```
        = .TRUE. : the full Schur form T is required;
        = .FALSE.: only eigenvalues are required.

[in] WANTZ

        WANTZ is LOGICAL
        = .TRUE. : the matrix of Schur vectors Z is required;
        = .FALSE.: Schur vectors are not required.

[in] N

        N is INTEGER
         The order of the matrix H.  N .GE. 0.

[in] ILO

        ILO is INTEGER

[in] IHI

        IHI is INTEGER

         It is assumed that H is already upper triangular in rows
         and columns 1:ILO-1 and IHI+1:N and, if ILO.GT.1,
         H(ILO,ILO-1) is zero. ILO and IHI are normally set by a
         previous call to ZGEBAL, and then passed to ZGEHRD when the
         matrix output by ZGEBAL is reduced to Hessenberg form.
         Otherwise, ILO and IHI should be set to 1 and N,
         respectively.  If N.GT.0, then 1.LE.ILO.LE.IHI.LE.N.
         If N = 0, then ILO = 1 and IHI = 0.

[in,out] H

        H is COMPLEX*16 array, dimension (LDH,N)
         On entry, the upper Hessenberg matrix H.
         On exit, if INFO = 0 and WANTT is .TRUE., then H
         contains the upper triangular matrix T from the Schur
         decomposition (the Schur form). If INFO = 0 and WANT is
         .FALSE., then the contents of H are unspecified on exit.
         (The output value of H when INFO.GT.0 is given under the
         description of INFO below.)

         This subroutine may explicitly set H(i,j) = 0 for i.GT.j and
         j = 1, 2, ... ILO-1 or j = IHI+1, IHI+2, ... N.

[in] LDH

        LDH is INTEGER
         The leading dimension of the array H. LDH .GE. max(1,N).
```

```
[out] W

        W is COMPLEX*16 array, dimension (N)
         The computed eigenvalues of H(ILO:IHI,ILO:IHI) are stored
         in W(ILO:IHI). If WANTT is .TRUE., then the eigenvalues are
         stored in the same order as on the diagonal of the Schur
         form returned in H, with W(i) = H(i,i).

[in] ILOZ

        ILOZ is INTEGER

[in] IHIZ

        IHIZ is INTEGER
         Specify the rows of Z to which transformations must be
         applied if WANTZ is .TRUE..
         1 .LE. ILOZ .LE. ILO; IHI .LE. IHIZ .LE. N.

[in,out] Z

        Z is COMPLEX*16 array, dimension (LDZ,IHI)
         If WANTZ is .FALSE., then Z is not referenced.
         If WANTZ is .TRUE., then Z(ILO:IHI,ILOZ:IHIZ) is
         replaced by Z(ILO:IHI,ILOZ:IHIZ)*U where U is the
         orthogonal Schur factor of H(ILO:IHI,ILO:IHI).
         (The output value of Z when INFO.GT.0 is given under
         the description of INFO below.)

[in] LDZ

        LDZ is INTEGER
         The leading dimension of the array Z.  if WANTZ is .TRUE.
         then LDZ.GE.MAX(1,IHIZ).  Otherwize, LDZ.GE.1.

[out] WORK

        WORK is COMPLEX*16 array, dimension LWORK
         On exit, if LWORK = -1, WORK(1) returns an estimate of
         the optimal value for LWORK.

[in] LWORK

        LWORK is INTEGER
         The dimension of the array WORK.  LWORK .GE. max(1,N)
         is sufficient, but LWORK typically as large as 6*N may
         be required for optimal performance.  A workspace query
         to determine the optimal workspace size is recommended.

         If LWORK = -1, then ZLAQR0 does a workspace query.
```

```
          In this case, ZLAQR0 checks the input parameters and
          estimates the optimal workspace size for the given
          values of N, ILO and IHI.  The estimate is returned
          in WORK(1).  No error message related to LWORK is
          issued by XERBLA.  Neither H nor Z are accessed.

  [out] INFO

         INFO is INTEGER
            =  0:  successful exit
          .GT. 0:  if INFO = i, ZLAQR0 failed to compute all of
               the eigenvalues.  Elements 1:ilo-1 and i+1:n of WR
               and WI contain those eigenvalues which have been
               successfully computed.  (Failures are rare.)

               If INFO .GT. 0 and WANT is .FALSE., then on exit,
               the remaining unconverged eigenvalues are the eigen-
               values of the upper Hessenberg matrix rows and
               columns ILO through INFO of the final, output
               value of H.

               If INFO .GT. 0 and WANTT is .TRUE., then on exit

          (*)  (initial value of H)*U  = U*(final value of H)

               where U is a unitary matrix.  The final
               value of  H is upper Hessenberg and triangular in
               rows and columns INFO+1 through IHI.

               If INFO .GT. 0 and WANTZ is .TRUE., then on exit

                 (final value of Z(ILO:IHI,ILOZ:IHIZ)
                  =  (initial value of Z(ILO:IHI,ILOZ:IHIZ)*U

               where U is the unitary matrix in (*) (regard-
               less of the value of WANTT.)

               If INFO .GT. 0 and WANTZ is .FALSE., then Z is not
               accessed.
```

———-

90.223.3 fortran code

— zlaqr0.f —

```
*  =================================================================
```

```
      SUBROUTINE ZLAQR0( WANTT, WANTZ, N, ILO, IHI, H, LDH, W, ILOZ,
     $                   IHIZ, Z, LDZ, WORK, LWORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            IHI, IHIZ, ILO, ILOZ, INFO, LDH, LDZ, LWORK, N
      LOGICAL            WANTT, WANTZ
*     ..
*     .. Array Arguments ..
      COMPLEX*16         H( LDH, * ), W( * ), WORK( * ), Z( LDZ, * )
*     ..
*
*  ================================================================
*
*     .. Parameters ..
*
*     ==== Matrices of order NTINY or smaller must be processed by
*     .    ZLAHQR because of insufficient subdiagonal scratch space.
*     .    (This is a hard limit.) ====
      INTEGER            NTINY
      PARAMETER          ( NTINY = 11 )
*
*     ==== Exceptional deflation windows:  try to cure rare
*     .    slow convergence by varying the size of the
*     .    deflation window after KEXNW iterations. ====
      INTEGER            KEXNW
      PARAMETER          ( KEXNW = 5 )
*
*     ==== Exceptional shifts: try to cure rare slow convergence
*     .    with ad-hoc exceptional shifts every KEXSH iterations.
*     .    ====
      INTEGER            KEXSH
      PARAMETER          ( KEXSH = 6 )
*
*     ==== The constant WILK1 is used to form the exceptional
*     .    shifts. ====
      DOUBLE PRECISION   WILK1
      PARAMETER          ( WILK1 = 0.75d0 )
      COMPLEX*16         ZERO, ONE
      PARAMETER          ( ZERO = ( 0.0d0, 0.0d0 ),
     $                   ONE = ( 1.0d0, 0.0d0 ) )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0d0 )
*     ..
*     .. Local Scalars ..
      COMPLEX*16         AA, BB, CC, CDUM, DD, DET, RTDISC, SWAP, TR2
```

```
      DOUBLE PRECISION   S
      INTEGER            I, INF, IT, ITMAX, K, KACC22, KBOT, KDU, KS,
     $                   KT, KTOP, KU, KV, KWH, KWTOP, KWV, LD, LS,
     $                   LWKOPT, NDEC, NDFL, NH, NHO, NIBBLE, NMIN, NS,
     $                   NSMAX, NSR, NVE, NW, NWMAX, NWR, NWUPBD
      LOGICAL            SORTED
      CHARACTER          JBCMPZ*2
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Local Arrays ..
      COMPLEX*16         ZDUM( 1, 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZLACPY, ZLAHQR, ZLAQR3, ZLAQR4, ZLAQR5
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DCMPLX, DIMAG, INT, MAX, MIN, MOD,
     $                   SQRT
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( CDUM ) = ABS( DBLE( CDUM ) ) + ABS( DIMAG( CDUM ) )
*     ..
*     .. Executable Statements ..
      INFO = 0
*
*     ==== Quick return for N = 0: nothing to do. ====
*
      IF( N.EQ.0 ) THEN
         WORK( 1 ) = ONE
         RETURN
      END IF
*
      IF( N.LE.NTINY ) THEN
*
*        ==== Tiny matrices must use ZLAHQR. ====
*
         LWKOPT = 1
         IF( LWORK.NE.-1 )
     $      CALL ZLAHQR( WANTT, WANTZ, N, ILO, IHI, H, LDH, W, ILOZ,
     $                   IHIZ, Z, LDZ, INFO )
      ELSE
*
*        ==== Use small bulge multi-shift QR with aggressive early
*        .    deflation on larger-than-tiny matrices. ====
```

```
*
*        ==== Hope for the best. ====
*
         INFO = 0
*
*        ==== Set up job flags for ILAENV. ====
*
         IF( WANTT ) THEN
            JBCMPZ( 1: 1 ) = 'S'
         ELSE
            JBCMPZ( 1: 1 ) = 'E'
         END IF
         IF( WANTZ ) THEN
            JBCMPZ( 2: 2 ) = 'V'
         ELSE
            JBCMPZ( 2: 2 ) = 'N'
         END IF
*
*        ==== NWR = recommended deflation window size.  At this
*        .    point,  N .GT. NTINY = 11, so there is enough
*        .    subdiagonal workspace for NWR.GE.2 as required.
*        .    (In fact, there is enough subdiagonal space for
*        .    NWR.GE.3.) ====
*
         NWR = ILAENV( 13, 'ZLAQR0', JBCMPZ, N, ILO, IHI, LWORK )
         NWR = MAX( 2, NWR )
         NWR = MIN( IHI-ILO+1, ( N-1 ) / 3, NWR )
*
*        ==== NSR = recommended number of simultaneous shifts.
*        .    At this point N .GT. NTINY = 11, so there is at
*        .    enough subdiagonal workspace for NSR to be even
*        .    and greater than or equal to two as required. ====
*
         NSR = ILAENV( 15, 'ZLAQR0', JBCMPZ, N, ILO, IHI, LWORK )
         NSR = MIN( NSR, ( N+6 ) / 9, IHI-ILO )
         NSR = MAX( 2, NSR-MOD( NSR, 2 ) )
*
*        ==== Estimate optimal workspace ====
*
*        ==== Workspace query call to ZLAQR3 ====
*
         CALL ZLAQR3( WANTT, WANTZ, N, ILO, IHI, NWR+1, H, LDH, ILOZ,
     $                IHIZ, Z, LDZ, LS, LD, W, H, LDH, N, H, LDH, N, H,
     $                LDH, WORK, -1 )
*
*        ==== Optimal workspace = MAX(ZLAQR5, ZLAQR3) ====
*
         LWKOPT = MAX( 3*NSR / 2, INT( WORK( 1 ) ) )
*
*        ==== Quick return in case of workspace query. ====
```

```
*
      IF( LWORK.EQ.-1 ) THEN
         WORK( 1 ) = DCMPLX( LWKOPT, 0 )
         RETURN
      END IF
*
*     ==== ZLAHQR/ZLAQR0 crossover point ====
*
      NMIN = ILAENV( 12, 'ZLAQR0', JBCMPZ, N, ILO, IHI, LWORK )
      NMIN = MAX( NTINY, NMIN )
*
*     ==== Nibble crossover point ====
*
      NIBBLE = ILAENV( 14, 'ZLAQR0', JBCMPZ, N, ILO, IHI, LWORK )
      NIBBLE = MAX( 0, NIBBLE )
*
*     ==== Accumulate reflections during ttswp?  Use block
*     .    2-by-2 structure during matrix-matrix multiply? ====
*
      KACC22 = ILAENV( 16, 'ZLAQR0', JBCMPZ, N, ILO, IHI, LWORK )
      KACC22 = MAX( 0, KACC22 )
      KACC22 = MIN( 2, KACC22 )
*
*     ==== NWMAX = the largest possible deflation window for
*     .    which there is sufficient workspace. ====
*
      NWMAX = MIN( ( N-1 ) / 3, LWORK / 2 )
      NW = NWMAX
*
*     ==== NSMAX = the Largest number of simultaneous shifts
*     .    for which there is sufficient workspace. ====
*
      NSMAX = MIN( ( N+6 ) / 9, 2*LWORK / 3 )
      NSMAX = NSMAX - MOD( NSMAX, 2 )
*
*     ==== NDFL: an iteration count restarted at deflation. ====
*
      NDFL = 1
*
*     ==== ITMAX = iteration limit ====
*
      ITMAX = MAX( 30, 2*KEXSH )*MAX( 10, ( IHI-ILO+1 ) )
*
*     ==== Last row and column in the active block ====
*
      KBOT = IHI
*
*     ==== Main Loop ====
*
      DO 70 IT = 1, ITMAX
```

```
*
*          ==== Done when KBOT falls below ILO ====
*
           IF( KBOT.LT.ILO )
     $        GO TO 80
*
*          ==== Locate active block ====
*
           DO 10 K = KBOT, ILO + 1, -1
              IF( H( K, K-1 ).EQ.ZERO )
     $           GO TO 20
   10      CONTINUE
           K = ILO
   20      CONTINUE
           KTOP = K
*
*          ==== Select deflation window size:
*          .    Typical Case:
*          .      If possible and advisable, nibble the entire
*          .      active block.  If not, use size MIN(NWR,NWMAX)
*          .      or MIN(NWR+1,NWMAX) depending upon which has
*          .      the smaller corresponding subdiagonal entry
*          .      (a heuristic).
*          .
*          .    Exceptional Case:
*          .      If there have been no deflations in KEXNW or
*          .      more iterations, then vary the deflation window
*          .      size.   At first, because, larger windows are,
*          .      in general, more powerful than smaller ones,
*          .      rapidly increase the window to the maximum possible.
*          .      Then, gradually reduce the window size. ====
*
           NH = KBOT - KTOP + 1
           NWUPBD = MIN( NH, NWMAX )
           IF( NDFL.LT.KEXNW ) THEN
              NW = MIN( NWUPBD, NWR )
           ELSE
              NW = MIN( NWUPBD, 2*NW )
           END IF
           IF( NW.LT.NWMAX ) THEN
              IF( NW.GE.NH-1 ) THEN
                 NW = NH
              ELSE
                 KWTOP = KBOT - NW + 1
                 IF( CABS1( H( KWTOP, KWTOP-1 ) ).GT.
     $               CABS1( H( KWTOP-1, KWTOP-2 ) ) )NW = NW + 1
              END IF
           END IF
           IF( NDFL.LT.KEXNW ) THEN
              NDEC = -1
```

```
            ELSE IF( NDEC.GE.0 .OR. NW.GE.NWUPBD ) THEN
               NDEC = NDEC + 1
               IF( NW-NDEC.LT.2 )
     $            NDEC = 0
               NW = NW - NDEC
            END IF
*
*           ==== Aggressive early deflation:
*           .    split workspace under the subdiagonal into
*           .      - an nw-by-nw work array V in the lower
*           .        left-hand-corner,
*           .      - an NW-by-at-least-NW-but-more-is-better
*           .        (NW-by-NHO) horizontal work array along
*           .        the bottom edge,
*           .      - an at-least-NW-but-more-is-better (NHV-by-NW)
*           .        vertical work array along the left-hand-edge.
*           .        ====
*
            KV = N - NW + 1
            KT = NW + 1
            NHO = ( N-NW-1 ) - KT + 1
            KWV = NW + 2
            NVE = ( N-NW ) - KWV + 1
*
*           ==== Aggressive early deflation ====
*
            CALL ZLAQR3( WANTT, WANTZ, N, KTOP, KBOT, NW, H, LDH, ILOZ,
     $                   IHIZ, Z, LDZ, LS, LD, W, H( KV, 1 ), LDH, NHO,
     $                   H( KV, KT ), LDH, NVE, H( KWV, 1 ), LDH, WORK,
     $                   LWORK )
*
*           ==== Adjust KBOT accounting for new deflations. ====
*
            KBOT = KBOT - LD
*
*           ==== KS points to the shifts. ====
*
            KS = KBOT - LS + 1
*
*           ==== Skip an expensive QR sweep if there is a (partly
*           .    heuristic) reason to expect that many eigenvalues
*           .    will deflate without it.  Here, the QR sweep is
*           .    skipped if many eigenvalues have just been deflated
*           .    or if the remaining active block is small.
*
            IF( ( LD.EQ.0 ) .OR. ( ( 100*LD.LE.NW*NIBBLE ) .AND. ( KBOT-
     $          KTOP+1.GT.MIN( NMIN, NWMAX ) ) ) ) THEN
*
*              ==== NS = nominal number of simultaneous shifts.
*              .    This may be lowered (slightly) if ZLAQR3
```

```
*              .    did not provide that many shifts. ====
*
               NS = MIN( NSMAX, NSR, MAX( 2, KBOT-KTOP ) )
               NS = NS - MOD( NS, 2 )
*
*              ==== If there have been no deflations
*              .    in a multiple of KEXSH iterations,
*              .    then try exceptional shifts.
*              .    Otherwise use shifts provided by
*              .    ZLAQR3 above or from the eigenvalues
*              .    of a trailing principal submatrix. ====
*
               IF( MOD( NDFL, KEXSH ).EQ.0 ) THEN
                  KS = KBOT - NS + 1
                  DO 30 I = KBOT, KS + 1, -2
                     W( I ) = H( I, I ) + WILK1*CABS1( H( I, I-1 ) )
                     W( I-1 ) = W( I )
   30             CONTINUE
               ELSE
*
                  ==== Got NS/2 or fewer shifts? Use ZLAQR4 or
*                 .    ZLAHQR on a trailing principal submatrix to
*                 .    get more. (Since NS.LE.NSMAX.LE.(N+6)/9,
*                 .    there is enough space below the subdiagonal
*                 .    to fit an NS-by-NS scratch array.) ====
*
                  IF( KBOT-KS+1.LE.NS / 2 ) THEN
                     KS = KBOT - NS + 1
                     KT = N - NS + 1
                     CALL ZLACPY( 'A', NS, NS, H( KS, KS ), LDH,
     $                            H( KT, 1 ), LDH )
                     IF( NS.GT.NMIN ) THEN
                        CALL ZLAQR4( .false., .false., NS, 1, NS,
     $                               H( KT, 1 ), LDH, W( KS ), 1, 1,
     $                               ZDUM, 1, WORK, LWORK, INF )
                     ELSE
                        CALL ZLAHQR( .false., .false., NS, 1, NS,
     $                               H( KT, 1 ), LDH, W( KS ), 1, 1,
     $                               ZDUM, 1, INF )
                     END IF
                     KS = KS + INF
*
                     ==== In case of a rare QR failure use
*                    .    eigenvalues of the trailing 2-by-2
*                    .    principal submatrix.  Scale to avoid
*                    .    overflows, underflows and subnormals.
*                    .    (The scale factor S can not be zero,
*                    .    because H(KBOT,KBOT-1) is nonzero.) ====
*
                     IF( KS.GE.KBOT ) THEN
```

```
                     S = CABS1( H( KBOT-1, KBOT-1 ) ) +
     $                   CABS1( H( KBOT, KBOT-1 ) ) +
     $                   CABS1( H( KBOT-1, KBOT ) ) +
     $                   CABS1( H( KBOT, KBOT ) )
                     AA = H( KBOT-1, KBOT-1 ) / S
                     CC = H( KBOT, KBOT-1 ) / S
                     BB = H( KBOT-1, KBOT ) / S
                     DD = H( KBOT, KBOT ) / S
                     TR2 = ( AA+DD ) / TWO
                     DET = ( AA-TR2 )*( DD-TR2 ) - BB*CC
                     RTDISC = SQRT( -DET )
                     W( KBOT-1 ) = ( TR2+RTDISC )*S
                     W( KBOT ) = ( TR2-RTDISC )*S
*
                     KS = KBOT - 1
                  END IF
               END IF
*
               IF( KBOT-KS+1.GT.NS ) THEN
*
*                 ==== Sort the shifts (Helps a little) ====
*
                  SORTED = .false.
                  DO 50 K = KBOT, KS + 1, -1
                     IF( SORTED )
     $                  GO TO 60
                     SORTED = .true.
                     DO 40 I = KS, K - 1
                        IF( CABS1( W( I ) ).LT.CABS1( W( I+1 ) ) )
     $                       THEN
                           SORTED = .false.
                           SWAP = W( I )
                           W( I ) = W( I+1 )
                           W( I+1 ) = SWAP
                        END IF
   40                CONTINUE
   50             CONTINUE
   60             CONTINUE
               END IF
            END IF
*
*           ==== If there are only two shifts, then use
*           .    only one.  ====
*
            IF( KBOT-KS+1.EQ.2 ) THEN
               IF( CABS1( W( KBOT )-H( KBOT, KBOT ) ).LT.
     $             CABS1( W( KBOT-1 )-H( KBOT, KBOT ) ) ) THEN
                  W( KBOT-1 ) = W( KBOT )
               ELSE
                  W( KBOT ) = W( KBOT-1 )
```

```
                 END IF
              END IF
*
*             ==== Use up to NS of the the smallest magnatiude
*             .    shifts.  If there aren't NS shifts available,
*             .    then use them all, possibly dropping one to
*             .    make the number of shifts even. ====
*
              NS = MIN( NS, KBOT-KS+1 )
              NS = NS - MOD( NS, 2 )
              KS = KBOT - NS + 1
*
*             ==== Small-bulge multi-shift QR sweep:
*             .    split workspace under the subdiagonal into
*             .    - a KDU-by-KDU work array U in the lower
*             .      left-hand-corner,
*             .    - a KDU-by-at-least-KDU-but-more-is-better
*             .      (KDU-by-NHo) horizontal work array WH along
*             .      the bottom edge,
*             .    - and an at-least-KDU-but-more-is-better-by-KDU
*             .      (NVE-by-KDU) vertical work WV arrow along
*             .      the left-hand-edge. ====
*
              KDU = 3*NS - 3
              KU = N - KDU + 1
              KWH = KDU + 1
              NHO = ( N-KDU+1-4 ) - ( KDU+1 ) + 1
              KWV = KDU + 4
              NVE = N - KDU - KWV + 1
*
*             ==== Small-bulge multi-shift QR sweep ====
*
              CALL ZLAQR5( WANTT, WANTZ, KACC22, N, KTOP, KBOT, NS,
     $                     W( KS ), H, LDH, ILOZ, IHIZ, Z, LDZ, WORK,
     $                     3, H( KU, 1 ), LDH, NVE, H( KWV, 1 ), LDH,
     $                     NHO, H( KU, KWH ), LDH )
           END IF
*
*          ==== Note progress (or the lack of it). ====
*
           IF( LD.GT.0 ) THEN
              NDFL = 1
           ELSE
              NDFL = NDFL + 1
           END IF
*
*          ==== End of main loop ====
   70    CONTINUE
*
*        ==== Iteration limit exceeded.  Set INFO to show where
```

```
*        .    the problem occurred and exit. ====
*
         INFO = KBOT
   80    CONTINUE
      END IF
*
*     ==== Return the optimal value of LWORK. ====
*
      WORK( 1 ) = DCMPLX( LWKOPT, 0 )
*
*     ==== End of ZLAQR0 ====
*
      END
```

———-

90.223.4 lisp code

— **zlaqr0.lisp** —

```
(let* ((ntiny 11)
       (kexnw 5)
       (kexsh 6)
       (wilk1 0.75d0)
       (zero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (one (f2cl-lib:cmplx 1.0d0 0.0d0))
       (two 2.0d0))
  (declare (type (f2cl-lib:integer4 11 11) ntiny)
           (type (f2cl-lib:integer4 5 5) kexnw)
           (type (f2cl-lib:integer4 6 6) kexsh)
           (type (double-float 0.75d0 0.75d0) wilk1)
           (type (f2cl-lib:complex16) zero)
           (type (f2cl-lib:complex16) one)
           (type (double-float 2.0d0 2.0d0) two)
           (ignorable ntiny kexnw kexsh wilk1 zero one two))
  (defun zlaqr0 (wantt wantz n ilo ihi h ldh w iloz ihiz z ldz work lwork info)
    (declare (type (array f2cl-lib:complex16 (*)) work z w h)
             (type (f2cl-lib:integer4) info lwork ldz ihiz iloz ldh ihi ilo n)
             (type f2cl-lib:logical wantz wantt))
    (f2cl-lib:with-multi-array-data
        ((h f2cl-lib:complex16 h-%data% h-%offset%)
         (w f2cl-lib:complex16 w-%data% w-%offset%)
         (z f2cl-lib:complex16 z-%data% z-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%))
      (labels ((cabs1 (cdum)
                 (+ (abs (f2cl-lib:dble cdum)) (abs (f2cl-lib:dimag cdum)))))
        (declare
```

```
 (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
  cabs1))
(prog ((zdum (make-array 1 :element-type 'f2cl-lib:complex16))
       (jbcmpz
        (make-array '(2) :element-type 'character :initial-element
                    #\ ))
       (sorted nil) (i 0) (inf 0) (it 0) (itmax 0) (k 0) (kacc22 0)
       (kbot 0) (kdu 0) (ks 0) (kt 0) (ktop 0) (ku 0) (kv 0) (kwh 0)
       (kwtop 0) (kwv 0) (ld 0) (ls 0) (lwkopt 0) (ndec 0) (ndfl 0)
       (nh 0) (nho 0) (nibble 0) (nmin 0) (ns 0) (nsmax 0) (nsr 0)
       (nve 0) (nw 0) (nwmax 0) (nwr 0) (nwupbd 0) (s 0.0d0)
       (aa #C(0.0d0 0.0d0)) (bb #C(0.0d0 0.0d0)) (cc #C(0.0d0 0.0d0))
       (cdum #C(0.0d0 0.0d0)) (dd #C(0.0d0 0.0d0))
       (det #C(0.0d0 0.0d0)) (rtdisc #C(0.0d0 0.0d0))
       (swap #C(0.0d0 0.0d0)) (tr2 #C(0.0d0 0.0d0)))
  (declare (type (array f2cl-lib:complex16 (1)) zdum)
           (type (string 2) jbcmpz)
           (type f2cl-lib:logical sorted)
           (type (f2cl-lib:integer4) i inf it itmax k kacc22 kbot kdu
            ks kt ktop ku kv kwh kwtop kwv ld ls lwkopt ndec ndfl nh
            nho nibble nmin ns nsmax nsr nve nw nwmax nwr nwupbd)
           (type (double-float) s)
           (type (f2cl-lib:complex16) aa bb cc cdum dd det rtdisc swap
            tr2))
  (setf info 0)
  (cond
   ((= n 0)
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) one)
    (go end_label)))
  (cond
   ((<= n ntiny) (setf lwkopt 1)
    (if (/= lwork -1)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12)
            (zlahqr wantt wantz n ilo ihi h ldh w iloz ihiz z ldz info)
          (declare
           (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-8
            var-9 var-10 var-11))
          (setf ldh var-6)
          (setf info var-12))))
   (t
    (tagbody
      (setf info 0)
      (cond
       (wantt
        (f2cl-lib:fset-string (f2cl-lib:fref-string jbcmpz (1 1)) "S"))
       (t
        (f2cl-lib:fset-string (f2cl-lib:fref-string jbcmpz (1 1))
                              "E")))
```

```
(cond
 (wantz
  (f2cl-lib:fset-string (f2cl-lib:fref-string jbcmpz (2 2)) "V"))
 (t
  (f2cl-lib:fset-string (f2cl-lib:fref-string jbcmpz (2 2))
                        "N")))
(setf nwr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 13 "ZLAQR0" jbcmpz n ilo ihi lwork)
          (declare (ignore var-0 var-1))
          (setf jbcmpz var-2)
          (setf n var-3)
          (setf ilo var-4)
          (setf ihi var-5)
          (setf lwork var-6)
          ret-val))
(setf nwr
        (max (the f2cl-lib:integer4 2)
             (the f2cl-lib:integer4 nwr)))
(setf nwr
        (min (f2cl-lib:int-add (f2cl-lib:int-sub ihi ilo) 1)
             (the f2cl-lib:integer4 (truncate (- n 1) 3)) nwr))
(setf nsr
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 15 "ZLAQR0" jbcmpz n ilo ihi lwork)
          (declare (ignore var-0 var-1))
          (setf jbcmpz var-2)
          (setf n var-3)
          (setf ilo var-4)
          (setf ihi var-5)
          (setf lwork var-6)
          ret-val))
(setf nsr
        (min nsr (the f2cl-lib:integer4 (truncate (+ n 6) 9))
             (f2cl-lib:int-sub ihi ilo)))
(setf nsr
        (max (the f2cl-lib:integer4 2)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-sub nsr (mod nsr 2)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17
     var-18 var-19 var-20 var-21 var-22 var-23 var-24)
    (zlaqr3 wantt wantz n ilo ihi (f2cl-lib:int-add nwr 1) h ldh
     iloz ihiz z ldz ls ld w h ldh n h ldh n h ldh work -1)
  (declare
   (ignore var-5 var-6 var-10 var-14 var-15 var-18 var-21 var-23
    var-24))
```

```
  (when var-0 (setf wantt var-0))
  (when var-1 (setf wantz var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf ilo var-3))
  (when var-4 (setf ihi var-4))
  (when var-7 (setf ldh var-7))
  (when var-8 (setf iloz var-8))
  (when var-9 (setf ihiz var-9))
  (when var-11 (setf ldz var-11))
  (when var-12 (setf ls var-12))
  (when var-13 (setf ld var-13))
  (when var-16 (setf ldh var-16))
  (when var-17 (setf n var-17))
  (when var-19 (setf ldh var-19))
  (when var-20 (setf n var-20))
  (when var-22 (setf ldh var-22)))
(setf lwkopt
        (max (the f2cl-lib:integer4 (truncate (* 3 nsr) 2))
             (f2cl-lib:int
              (f2cl-lib:fref work-%data% (1) ((1 *))
                             work-%offset%))))
(cond
 ((= lwork (f2cl-lib:int-sub 1))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:dcmplx lwkopt 0))
  (go end_label)))
(setf nmin
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 12 "ZLAQR0" jbcmpz n ilo ihi lwork)
          (declare (ignore var-0 var-1))
          (setf jbcmpz var-2)
          (setf n var-3)
          (setf ilo var-4)
          (setf ihi var-5)
          (setf lwork var-6)
          ret-val))
(setf nmin
        (max (the f2cl-lib:integer4 ntiny)
             (the f2cl-lib:integer4 nmin)))
(setf nibble
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 14 "ZLAQR0" jbcmpz n ilo ihi lwork)
          (declare (ignore var-0 var-1))
          (setf jbcmpz var-2)
          (setf n var-3)
          (setf ilo var-4)
          (setf ihi var-5)
          (setf lwork var-6)
```

```
          ret-val))
(setf nibble
        (max (the f2cl-lib:integer4 0)
             (the f2cl-lib:integer4 nibble)))
(setf kacc22
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 16 "ZLAQR0" jbcmpz n ilo ihi lwork)
          (declare (ignore var-0 var-1))
          (setf jbcmpz var-2)
          (setf n var-3)
          (setf ilo var-4)
          (setf ihi var-5)
          (setf lwork var-6)
          ret-val))
(setf kacc22
        (max (the f2cl-lib:integer4 0)
             (the f2cl-lib:integer4 kacc22)))
(setf kacc22
        (min (the f2cl-lib:integer4 2)
             (the f2cl-lib:integer4 kacc22)))
(setf nwmax
        (min (the f2cl-lib:integer4 (truncate (- n 1) 3))
             (the f2cl-lib:integer4 (truncate lwork 2))))
(setf nw nwmax)
(setf nsmax
        (min (the f2cl-lib:integer4 (truncate (+ n 6) 9))
             (the f2cl-lib:integer4 (truncate (* 2 lwork) 3))))
(setf nsmax (f2cl-lib:int-sub nsmax (mod nsmax 2)))
(setf ndfl 1)
(setf itmax
        (f2cl-lib:int-mul
         (max (the f2cl-lib:integer4 30)
              (the f2cl-lib:integer4 (f2cl-lib:int-mul 2 kexsh)))
         (max (the f2cl-lib:integer4 10)
              (the f2cl-lib:integer4
                   (f2cl-lib:int-add (f2cl-lib:int-sub ihi ilo)
                                     1)))))
(setf kbot ihi)
(f2cl-lib:fdo (it 1 (f2cl-lib:int-add it 1))
              ((> it itmax) nil)
  (tagbody
    (if (< kbot ilo)
        (go label80))
    (f2cl-lib:fdo (k kbot
                   (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                  ((> k (f2cl-lib:int-add ilo 1)) nil)
      (tagbody
        (if (=
             (f2cl-lib:fref h-%data% (k (f2cl-lib:int-sub k 1))
```

```
                          ((1 ldh) (1 *)) h-%offset%)
         zero)
        (go label20))
    label10))
 (setf k ilo)
label20
 (setf ktop k)
 (setf nh (f2cl-lib:int-add (f2cl-lib:int-sub kbot ktop) 1))
 (setf nwupbd
        (min (the f2cl-lib:integer4 nh)
             (the f2cl-lib:integer4 nwmax)))
 (cond
  ((< ndfl kexnw)
   (setf nw
           (min (the f2cl-lib:integer4 nwupbd)
                (the f2cl-lib:integer4 nwr))))
  (t
   (setf nw
           (min (the f2cl-lib:integer4 nwupbd)
                (the f2cl-lib:integer4
                     (f2cl-lib:int-mul 2 nw))))))
 (cond
  ((< nw nwmax)
   (cond
    ((>= nw (f2cl-lib:int-add nh (f2cl-lib:int-sub 1)))
     (setf nw nh))
    (t
     (setf kwtop
             (f2cl-lib:int-add (f2cl-lib:int-sub kbot nw) 1))
     (if (>
          (cabs1
           (f2cl-lib:fref h-%data%
                          (kwtop (f2cl-lib:int-sub kwtop 1))
                          ((1 ldh) (1 *)) h-%offset%))
          (cabs1
           (f2cl-lib:fref h-%data%
                          ((f2cl-lib:int-sub kwtop 1)
                           (f2cl-lib:int-sub kwtop 2))
                          ((1 ldh) (1 *)) h-%offset%)))
         (setf nw (f2cl-lib:int-add nw 1)))))))
 (cond ((< ndfl kexnw) (setf ndec -1))
       ((or (>= ndec 0) (>= nw nwupbd))
        (setf ndec (f2cl-lib:int-add ndec 1))
        (if (< (f2cl-lib:int-sub nw ndec) 2)
            (setf ndec 0))
        (setf nw (f2cl-lib:int-sub nw ndec))))
 (setf kv (f2cl-lib:int-add (f2cl-lib:int-sub n nw) 1))
 (setf kt (f2cl-lib:int-add nw 1))
 (setf nho (f2cl-lib:int-add (f2cl-lib:int-sub n nw 1 kt) 1))
 (setf kwv (f2cl-lib:int-add nw 2))
```

```
(setf nve (f2cl-lib:int-add (f2cl-lib:int-sub n nw kwv) 1))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17 var-18 var-19 var-20 var-21 var-22 var-23 var-24)
    (zlaqr3 wantt wantz n ktop kbot nw h ldh iloz ihiz z ldz
     ls ld w
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16 (kv 1)
                           ((1 ldh) (1 *)) h-%offset%)
     ldh nho
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                           (kv kt) ((1 ldh) (1 *))
                           h-%offset%)
     ldh nve
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                           (kwv 1) ((1 ldh) (1 *))
                           h-%offset%)
     ldh work lwork)
  (declare
   (ignore var-6 var-10 var-14 var-15 var-18 var-21 var-23))
  (when var-0 (setf wantt var-0))
  (when var-1 (setf wantz var-1))
  (when var-2 (setf n var-2))
  (when var-3 (setf ktop var-3))
  (when var-4 (setf kbot var-4))
  (when var-5 (setf nw var-5))
  (when var-7 (setf ldh var-7))
  (when var-8 (setf iloz var-8))
  (when var-9 (setf ihiz var-9))
  (when var-11 (setf ldz var-11))
  (when var-12 (setf ls var-12))
  (when var-13 (setf ld var-13))
  (when var-16 (setf ldh var-16))
  (when var-17 (setf nho var-17))
  (when var-19 (setf ldh var-19))
  (when var-20 (setf nve var-20))
  (when var-22 (setf ldh var-22))
  (when var-24 (setf lwork var-24)))
(setf kbot (f2cl-lib:int-sub kbot ld))
(setf ks (f2cl-lib:int-add (f2cl-lib:int-sub kbot ls) 1))
(cond
 ((or (= ld 0)
      (and
       (<= (f2cl-lib:int-mul 100 ld)
           (f2cl-lib:int-mul nw nibble))
       (> (f2cl-lib:int-add kbot (f2cl-lib:int-sub ktop) 1)
          (min (the f2cl-lib:integer4 nmin)
               (the f2cl-lib:integer4 nwmax)))))
  (setf ns
          (min (the f2cl-lib:integer4 nsmax)
```

```
                (the f2cl-lib:integer4 nsr)
                (the f2cl-lib:integer4
                     (max (the f2cl-lib:integer4 2)
                          (the f2cl-lib:integer4
                               (f2cl-lib:int-sub kbot
                                                 ktop))))))
(setf ns (f2cl-lib:int-sub ns (mod ns 2)))
(cond
 ((= (mod ndfl kexsh) 0)
  (setf ks (f2cl-lib:int-add (f2cl-lib:int-sub kbot ns) 1))
  (f2cl-lib:fdo (i kbot
                 (f2cl-lib:int-add i (f2cl-lib:int-sub 2)))
                ((> i (f2cl-lib:int-add ks 1)) nil)
    (tagbody
      (setf (f2cl-lib:fref w-%data% (i) ((1 *)) w-%offset%)
              (+
               (f2cl-lib:fref h-%data% (i i)
                              ((1 ldh) (1 *)) h-%offset%)
               (* wilk1
                  (cabs1
                   (f2cl-lib:fref h-%data%
                                  (i
                                   (f2cl-lib:int-sub i 1))
                                  ((1 ldh) (1 *))
                                  h-%offset%)))))
      (setf (f2cl-lib:fref w-%data%
                           ((f2cl-lib:int-sub i 1)) ((1 *))
                           w-%offset%)
              (f2cl-lib:fref w-%data% (i) ((1 *))
                             w-%offset%))
     label30)))
 (t
  (cond
   ((<= (f2cl-lib:int-add kbot (f2cl-lib:int-sub ks) 1)
        (f2cl-lib:f2cl/ ns 2))
    (setf ks
            (f2cl-lib:int-add (f2cl-lib:int-sub kbot ns)
                              1))
    (setf kt (f2cl-lib:int-add (f2cl-lib:int-sub n ns) 1))
    (zlacpy "A" ns ns
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                           (ks ks) ((1 ldh) (1 *))
                           h-%offset%)
     ldh
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                           (kt 1) ((1 ldh) (1 *))
                           h-%offset%)
     ldh)
    (cond
     ((> ns nmin)
```

```
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8 var-9 var-10 var-11 var-12 var-13 var-14)
      (zlaqr4 f2cl-lib:%false% f2cl-lib:%false% ns 1 ns
       (f2cl-lib:array-slice h-%data%
                             f2cl-lib:complex16 (kt 1)
                             ((1 ldh) (1 *))
                             h-%offset%)
       ldh
       (f2cl-lib:array-slice w-%data%
                             f2cl-lib:complex16 (ks)
                             ((1 *)) w-%offset%)
       1 1 zdum 1 work lwork inf)
    (declare
     (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-9
      var-10 var-11 var-12))
    (when var-2 (setf ns var-2))
    (when var-4 (setf ns var-4))
    (when var-6 (setf ldh var-6))
    (when var-13 (setf lwork var-13))
    (when var-14 (setf inf var-14))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
       var-8 var-9 var-10 var-11 var-12)
      (zlahqr f2cl-lib:%false% f2cl-lib:%false% ns 1 ns
       (f2cl-lib:array-slice h-%data%
                             f2cl-lib:complex16 (kt 1)
                             ((1 ldh) (1 *))
                             h-%offset%)
       ldh
       (f2cl-lib:array-slice w-%data%
                             f2cl-lib:complex16 (ks)
                             ((1 *)) w-%offset%)
       1 1 zdum 1 inf)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7
      var-8 var-9 var-10 var-11))
    (setf ldh var-6)
    (setf inf var-12))))
(setf ks (f2cl-lib:int-add ks inf))
(cond
 ((>= ks kbot)
  (setf s
          (+
           (cabs1
            (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-sub kbot 1)
                            (f2cl-lib:int-sub kbot 1))
                           ((1 ldh) (1 *)) h-%offset%))
```

```
          (cabs1
           (f2cl-lib:fref h-%data%
                          (kbot
                           (f2cl-lib:int-sub kbot 1))
                          ((1 ldh) (1 *)) h-%offset%))
          (cabs1
           (f2cl-lib:fref h-%data%
                          ((f2cl-lib:int-sub kbot 1)
                           kbot)
                          ((1 ldh) (1 *)) h-%offset%))
          (cabs1
           (f2cl-lib:fref h-%data% (kbot kbot)
                          ((1 ldh) (1 *))
                          h-%offset%))))
(setf aa
        (/
         (f2cl-lib:fref h-%data%
                        ((f2cl-lib:int-sub kbot 1)
                         (f2cl-lib:int-sub kbot 1))
                        ((1 ldh) (1 *)) h-%offset%)
         s))
(setf cc
        (/
         (f2cl-lib:fref h-%data%
                        (kbot
                         (f2cl-lib:int-sub kbot 1))
                        ((1 ldh) (1 *)) h-%offset%)
         s))
(setf bb
        (/
         (f2cl-lib:fref h-%data%
                        ((f2cl-lib:int-sub kbot 1)
                         kbot)
                        ((1 ldh) (1 *)) h-%offset%)
         s))
(setf dd
        (/
         (f2cl-lib:fref h-%data% (kbot kbot)
                        ((1 ldh) (1 *)) h-%offset%)
         s))
(setf tr2 (/ (+ aa dd) two))
(setf det (- (* (- aa tr2) (- dd tr2)) (* bb cc)))
(setf rtdisc (f2cl-lib:fsqrt (- det)))
(setf (f2cl-lib:fref w-%data%
                     ((f2cl-lib:int-sub kbot 1))
                     ((1 *)) w-%offset%)
        (* (+ tr2 rtdisc) s))
(setf (f2cl-lib:fref w-%data% (kbot) ((1 *))
                     w-%offset%)
        (* (- tr2 rtdisc) s))
```

```
      (setf ks (f2cl-lib:int-sub kbot 1))))))
    (cond
     ((> (f2cl-lib:int-add kbot (f2cl-lib:int-sub ks) 1) ns)
      (tagbody
        (setf sorted f2cl-lib:%false%)
        (f2cl-lib:fdo (k kbot
                       (f2cl-lib:int-add k
                                         (f2cl-lib:int-sub
                                          1)))
                      ((> k (f2cl-lib:int-add ks 1)) nil)
          (tagbody
            (if sorted
                (go label60))
            (setf sorted f2cl-lib:%true%)
            (f2cl-lib:fdo (i ks (f2cl-lib:int-add i 1))
                          ((> i
                              (f2cl-lib:int-add k
                                                (f2cl-lib:int-sub
                                                 1)))
                           nil)
              (tagbody
                (cond
                 ((< (cabs1 (f2cl-lib:fref w (i) ((1 *))))
                     (cabs1
                      (f2cl-lib:fref w
                                     ((f2cl-lib:int-add i 1))
                                     ((1 *)))))
                  (setf sorted f2cl-lib:%false%)
                  (setf swap
                          (f2cl-lib:fref w-%data% (i) ((1 *))
                                         w-%offset%))
                  (setf (f2cl-lib:fref w-%data% (i) ((1 *))
                                        w-%offset%)
                          (f2cl-lib:fref w-%data%
                                         ((f2cl-lib:int-add i
                                                            1))
                                         ((1 *)) w-%offset%))
                  (setf (f2cl-lib:fref w-%data%
                                       ((f2cl-lib:int-add i
                                                          1))
                                       ((1 *)) w-%offset%)
                          swap)))
              label40))
          label50))
      label60)))))
  (cond
   ((= (f2cl-lib:int-add kbot (f2cl-lib:int-sub ks) 1) 2)
    (cond
     ((<
       (cabs1
```

```
      (+ (f2cl-lib:fref w (kbot) ((1 *)))
         (-
          (f2cl-lib:fref h (kbot kbot) ((1 ldh) (1 *))))))
     (cabs1
      (+
       (f2cl-lib:fref w
                      ((f2cl-lib:int-add kbot
                                         (f2cl-lib:int-sub
                                          1)))
                      ((1 *)))
       (- (f2cl-lib:fref h (kbot kbot) ((1 ldh) (1 *)))))))
    (setf (f2cl-lib:fref w-%data%
                         ((f2cl-lib:int-sub kbot 1))
                         ((1 *)) w-%offset%)
            (f2cl-lib:fref w-%data% (kbot) ((1 *))
                           w-%offset%)))
   (t
    (setf (f2cl-lib:fref w-%data% (kbot) ((1 *))
                         w-%offset%)
            (f2cl-lib:fref w-%data%
                           ((f2cl-lib:int-sub kbot 1))
                           ((1 *)) w-%offset%))))))
(setf ns
        (min (the f2cl-lib:integer4 ns)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add
                   (f2cl-lib:int-sub kbot ks) 1))))
(setf ns (f2cl-lib:int-sub ns (mod ns 2)))
(setf ks (f2cl-lib:int-add (f2cl-lib:int-sub kbot ns) 1))
(setf kdu (f2cl-lib:int-sub (f2cl-lib:int-mul 3 ns) 3))
(setf ku (f2cl-lib:int-add (f2cl-lib:int-sub n kdu) 1))
(setf kwh (f2cl-lib:int-add kdu 1))
(setf nho
        (f2cl-lib:int-add
         (f2cl-lib:int-sub
          (f2cl-lib:int-add (f2cl-lib:int-sub n kdu) 1) 4
          (f2cl-lib:int-add kdu 1))
         1))
(setf kwv (f2cl-lib:int-add kdu 4))
(setf nve
        (f2cl-lib:int-add (f2cl-lib:int-sub n kdu kwv) 1))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17 var-18 var-19 var-20 var-21 var-22 var-23)
    (zlaqr5 wantt wantz kacc22 n ktop kbot ns
     (f2cl-lib:array-slice w-%data% f2cl-lib:complex16 (ks)
                           ((1 *)) w-%offset%)
     h ldh iloz ihiz z ldz work 3
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
```

```
                                    (ku 1) ((1 ldh) (1 *))
                                    h-%offset%)
                      ldh nve
                      (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                                    (kwv 1) ((1 ldh) (1 *))
                                    h-%offset%)
                      ldh nho
                      (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                                    (ku kwh) ((1 ldh) (1 *))
                                    h-%offset%)
                      ldh)
                    (declare
                     (ignore var-7 var-8 var-12 var-14 var-15 var-16 var-19
                      var-22))
                    (when var-0 (setf wantt var-0))
                    (when var-1 (setf wantz var-1))
                    (when var-2 (setf kacc22 var-2))
                    (when var-3 (setf n var-3))
                    (when var-4 (setf ktop var-4))
                    (when var-5 (setf kbot var-5))
                    (when var-6 (setf ns var-6))
                    (when var-9 (setf ldh var-9))
                    (when var-10 (setf iloz var-10))
                    (when var-11 (setf ihiz var-11))
                    (when var-13 (setf ldz var-13))
                    (when var-17 (setf ldh var-17))
                    (when var-18 (setf nve var-18))
                    (when var-20 (setf ldh var-20))
                    (when var-21 (setf nho var-21))
                    (when var-23 (setf ldh var-23)))))
                (cond ((> ld 0) (setf ndfl 1))
                      (t (setf ndfl (f2cl-lib:int-add ndfl 1))))
               label70))
            (setf info kbot)
           label80)))
         (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (f2cl-lib:dcmplx lwkopt 0))
        end_label
         (return
          (values wantt wantz n ilo ihi nil ldh nil iloz ihiz nil ldz nil
                  lwork info)))))))
```

———-

90.224 zlaqr1

90.224.1 Axiom unit tests

— zlaqr1.input —

```
)set break resume
)sys rm -f zlaqr1.output
)spool zlaqr1.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.224.2 Axiom help page

— zlaqr1.help —

```
======================================================================
zlaqr1 examples
======================================================================

======================================================================
Man Page Details
======================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZLAQR1( N, H, LDH, S1, S2, V )

       .. Scalar Arguments ..
       COMPLEX*16         S1, S2
       INTEGER            LDH, N
       ..
       .. Array Arguments ..
       COMPLEX*16         H( LDH, * ), V( * )
       ..
```

```
Purpose:
=============

     Given a 2-by-2 or 3-by-3 matrix H, ZLAQR1 sets v to a
     scalar multiple of the first column of the product

     (*)  K = (H - s1*I)*(H - s2*I)

     scaling to avoid overflows and most underflows.

     This is useful for starting double implicit shift bulges
     in the QR algorithm.

Arguments:
==========

 [in] N

         N is integer
             Order of the matrix H. N must be either 2 or 3.

 [in] H

         H is COMPLEX*16 array of dimension (LDH,N)
             The 2-by-2 or 3-by-3 matrix H in (*).

 [in] LDH

         LDH is integer
             The leading dimension of H as declared in
             the calling procedure.  LDH.GE.N

 [in] S1

         S1 is COMPLEX*16

 [in] S2

         S2 is COMPLEX*16

         S1 and S2 are the shifts defining K in (*) above.

 [out] V

         V is COMPLEX*16 array of dimension N
             A scalar multiple of the first column of the
             matrix K in (*).
```

———-

90.224.3 fortran code

— zlaqr1.f —

```
*  ====================================================================
      SUBROUTINE ZLAQR1( N, H, LDH, S1, S2, V )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      COMPLEX*16         S1, S2
      INTEGER            LDH, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         H( LDH, * ), V( * )
*     ..
*
*  ================================================================
*
*     .. Parameters ..
      COMPLEX*16         ZERO
      PARAMETER          ( ZERO = ( 0.0d0, 0.0d0 ) )
      DOUBLE PRECISION   RZERO
      PARAMETER          ( RZERO = 0.0d0 )
*     ..
*     .. Local Scalars ..
      COMPLEX*16         CDUM, H21S, H31S
      DOUBLE PRECISION   S
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( CDUM ) = ABS( DBLE( CDUM ) ) + ABS( DIMAG( CDUM ) )
*     ..
*     .. Executable Statements ..
      IF( N.EQ.2 ) THEN
         S = CABS1( H( 1, 1 )-S2 ) + CABS1( H( 2, 1 ) )
         IF( S.EQ.RZERO ) THEN
            V( 1 ) = ZERO
```

```
         V( 2 ) = ZERO
      ELSE
         H21S = H( 2, 1 ) / S
         V( 1 ) = H21S*H( 1, 2 ) + ( H( 1, 1 )-S1 )*
     $              ( ( H( 1, 1 )-S2 ) / S )
         V( 2 ) = H21S*( H( 1, 1 )+H( 2, 2 )-S1-S2 )
      END IF
    ELSE
      S = CABS1( H( 1, 1 )-S2 ) + CABS1( H( 2, 1 ) ) +
     $      CABS1( H( 3, 1 ) )
      IF( S.EQ.ZERO ) THEN
         V( 1 ) = ZERO
         V( 2 ) = ZERO
         V( 3 ) = ZERO
      ELSE
         H21S = H( 2, 1 ) / S
         H31S = H( 3, 1 ) / S
         V( 1 ) = ( H( 1, 1 )-S1 )*( ( H( 1, 1 )-S2 ) / S ) +
     $              H( 1, 2 )*H21S + H( 1, 3 )*H31S
         V( 2 ) = H21S*( H( 1, 1 )+H( 2, 2 )-S1-S2 ) + H( 2, 3 )*H31S
         V( 3 ) = H31S*( H( 1, 1 )+H( 3, 3 )-S1-S2 ) + H21S*H( 3, 2 )
      END IF
    END IF
    END
```

———-

90.224.4 lisp code

— zlaqr1.lisp —

```
(let* ((zero (f2cl-lib:cmplx 0.0d0 0.0d0)) (rzero 0.0d0))
  (declare (type (f2cl-lib:complex16) zero)
           (type (double-float 0.0d0 0.0d0) rzero)
           (ignorable zero rzero))
  (defun zlaqr1 (n h ldh s1 s2 v)
    (declare (type (f2cl-lib:complex16) s2 s1)
             (type (array f2cl-lib:complex16 (*)) v h)
             (type (f2cl-lib:integer4) ldh n))
    (f2cl-lib:with-multi-array-data
        ((h f2cl-lib:complex16 h-%data% h-%offset%)
         (v f2cl-lib:complex16 v-%data% v-%offset%))
      (labels ((cabs1 (cdum)
                 (+ (abs (f2cl-lib:dble cdum)) (abs (f2cl-lib:dimag cdum)))))
        (declare
         (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
          cabs1))
```

```
(prog ((s 0.0d0) (cdum #C(0.0d0 0.0d0)) (h21s #C(0.0d0 0.0d0))
       (h31s #C(0.0d0 0.0d0)))
  (declare (type (double-float) s)
           (type (f2cl-lib:complex16) cdum h21s h31s))
  (cond
   ((= n 2)
    (setf s
            (+
             (cabs1
              (-
               (f2cl-lib:fref h-%data% (1 1) ((1 ldh) (1 *))
                              h-%offset%)
               s2))
             (cabs1
              (f2cl-lib:fref h-%data% (2 1) ((1 ldh) (1 *))
                             h-%offset%))))
    (cond
     ((= s rzero)
      (setf (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%) zero)
      (setf (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%) zero))
     (t
      (setf h21s
              (/
               (f2cl-lib:fref h-%data% (2 1) ((1 ldh) (1 *))
                              h-%offset%)
               s))
      (setf (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%)
              (+
               (* h21s
                  (f2cl-lib:fref h-%data% (1 2) ((1 ldh) (1 *))
                                 h-%offset%))
               (*
                (-
                 (f2cl-lib:fref h-%data% (1 1) ((1 ldh) (1 *))
                                h-%offset%)
                 s1)
                (/
                 (-
                  (f2cl-lib:fref h-%data% (1 1) ((1 ldh) (1 *))
                                 h-%offset%)
                  s2)
                 s))))
      (setf (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%)
              (* h21s
                 (-
                  (+
                   (f2cl-lib:fref h-%data% (1 1) ((1 ldh) (1 *))
                                  h-%offset%)
                   (f2cl-lib:fref h-%data% (2 2) ((1 ldh) (1 *))
                                  h-%offset%))
```

```
                s1 s2))))))
        (t
         (setf s
                 (+
                  (cabs1
                   (-
                    (f2cl-lib:fref h-%data% (1 1) ((1 ldh) (1 *))
                                   h-%offset%)
                    s2))
                  (cabs1
                   (f2cl-lib:fref h-%data% (2 1) ((1 ldh) (1 *))
                                  h-%offset%))
                  (cabs1
                   (f2cl-lib:fref h-%data% (3 1) ((1 ldh) (1 *))
                                  h-%offset%))))
         (cond
          ((= s zero)
           (setf (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%) zero)
           (setf (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%) zero)
           (setf (f2cl-lib:fref v-%data% (3) ((1 *)) v-%offset%) zero))
          (t
           (setf h21s
                   (/
                    (f2cl-lib:fref h-%data% (2 1) ((1 ldh) (1 *))
                                   h-%offset%)
                    s))
           (setf h31s
                   (/
                    (f2cl-lib:fref h-%data% (3 1) ((1 ldh) (1 *))
                                   h-%offset%)
                    s))
           (setf (f2cl-lib:fref v-%data% (1) ((1 *)) v-%offset%)
                   (+
                    (*
                     (-
                      (f2cl-lib:fref h-%data% (1 1) ((1 ldh) (1 *))
                                     h-%offset%)
                      s1)
                     (/
                      (-
                       (f2cl-lib:fref h-%data% (1 1) ((1 ldh) (1 *))
                                      h-%offset%)
                       s2)
                      s))
                    (*
                     (f2cl-lib:fref h-%data% (1 2) ((1 ldh) (1 *))
                                    h-%offset%)
                     h21s)
                    (*
                     (f2cl-lib:fref h-%data% (1 3) ((1 ldh) (1 *))
```

```
                                h-%offset%)
                     h31s)))
            (setf (f2cl-lib:fref v-%data% (2) ((1 *)) v-%offset%)
                    (+
                     (* h21s
                        (-
                         (+
                          (f2cl-lib:fref h-%data% (1 1) ((1 ldh) (1 *))
                                         h-%offset%)
                          (f2cl-lib:fref h-%data% (2 2) ((1 ldh) (1 *))
                                         h-%offset%))
                         s1 s2))
                     (*
                      (f2cl-lib:fref h-%data% (2 3) ((1 ldh) (1 *))
                                     h-%offset%)
                      h31s)))
            (setf (f2cl-lib:fref v-%data% (3) ((1 *)) v-%offset%)
                    (+
                     (* h31s
                        (-
                         (+
                          (f2cl-lib:fref h-%data% (1 1) ((1 ldh) (1 *))
                                         h-%offset%)
                          (f2cl-lib:fref h-%data% (3 3) ((1 ldh) (1 *))
                                         h-%offset%))
                         s1 s2))
                     (* h21s
                        (f2cl-lib:fref h-%data% (3 2) ((1 ldh) (1 *))
                                       h-%offset%))))))))
        end_label
         (return (values nil nil nil nil nil nil)))))))
```

———-

90.225 zlaqr2

90.225.1 Axiom unit tests

— **zlaqr2.input** —

```
)set break resume
)sys rm -f zlaqr2.output
)spool zlaqr2.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

90.225.2 Axiom help page

— **zlaqr2.help** —

```
=====================================================================
zlaqr2 examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZLAQR2( WANTT, WANTZ, N, KTOP, KBOT, NW, H, LDH, ILOZ,
                          IHIZ, Z, LDZ, NS, ND, SH, V, LDV, NH, T, LDT,
                          NV, WV, LDWV, WORK, LWORK )

       .. Scalar Arguments ..
       INTEGER            IHIZ, ILOZ, KBOT, KTOP, LDH, LDT, LDV, LDWV,
      $                   LDZ, LWORK, N, ND, NH, NS, NV, NW
       LOGICAL            WANTT, WANTZ
       ..
       .. Array Arguments ..
       COMPLEX*16         H( LDH, * ), SH( * ), T( LDT, * ), V( LDV, * ),
      $                   WORK( * ), WV( LDWV, * ), Z( LDZ, * )
       ..

 Purpose:
 =============

    ZLAQR2 is identical to ZLAQR3 except that it avoids
    recursion by calling ZLAHQR instead of ZLAQR4.

    Aggressive early deflation:

    ZLAQR2 accepts as input an upper Hessenberg matrix
    H and performs an unitary similarity transformation
```

```
   designed to detect and deflate fully converged eigenvalues from
   a trailing principal submatrix.  On output H has been over-
   written by a new Hessenberg matrix that is a perturbation of
   an unitary similarity transformation of H.  It is to be
   hoped that the final version of H has many zero subdiagonal
   entries.

Arguments:
==========

 [in] WANTT

         WANTT is LOGICAL
         If .TRUE., then the Hessenberg matrix H is fully updated
         so that the triangular Schur factor may be
         computed (in cooperation with the calling subroutine).
         If .FALSE., then only enough of H is updated to preserve
         the eigenvalues.

 [in] WANTZ

         WANTZ is LOGICAL
         If .TRUE., then the unitary matrix Z is updated so
         so that the unitary Schur factor may be computed
         (in cooperation with the calling subroutine).
         If .FALSE., then Z is not referenced.

 [in] N

         N is INTEGER
         The order of the matrix H and (if WANTZ is .TRUE.) the
         order of the unitary matrix Z.

 [in] KTOP

         KTOP is INTEGER
         It is assumed that either KTOP = 1 or H(KTOP,KTOP-1)=0.
         KBOT and KTOP together determine an isolated block
         along the diagonal of the Hessenberg matrix.

 [in] KBOT

         KBOT is INTEGER
         It is assumed without a check that either
         KBOT = N or H(KBOT+1,KBOT)=0.  KBOT and KTOP together
         determine an isolated block along the diagonal of the
         Hessenberg matrix.

 [in] NW
```

```
        NW is INTEGER
        Deflation window size.  1 .LE. NW .LE. (KBOT-KTOP+1).

[in,out] H

        H is COMPLEX*16 array, dimension (LDH,N)
        On input the initial N-by-N section of H stores the
        Hessenberg matrix undergoing aggressive early deflation.
        On output H has been transformed by a unitary
        similarity transformation, perturbed, and the returned
        to Hessenberg form that (it is to be hoped) has some
        zero subdiagonal entries.

[in] LDH

        LDH is integer
        Leading dimension of H just as declared in the calling
        subroutine.  N .LE. LDH

[in] ILOZ

        ILOZ is INTEGER

[in] IHIZ

        IHIZ is INTEGER
        Specify the rows of Z to which transformations must be
        applied if WANTZ is .TRUE.. 1 .LE. ILOZ .LE. IHIZ .LE. N.

[in,out] Z

        Z is COMPLEX*16 array, dimension (LDZ,N)
        IF WANTZ is .TRUE., then on output, the unitary
        similarity transformation mentioned above has been
        accumulated into Z(ILOZ:IHIZ,ILO:IHI) from the right.
        If WANTZ is .FALSE., then Z is unreferenced.

[in] LDZ

        LDZ is integer
        The leading dimension of Z just as declared in the
        calling subroutine.  1 .LE. LDZ.

[out] NS

        NS is integer
        The number of unconverged (ie approximate) eigenvalues
        returned in SR and SI that may be used as shifts by the
        calling subroutine.
```

```
[out] ND

        ND is integer
        The number of converged eigenvalues uncovered by this
        subroutine.

[out] SH

        SH is COMPLEX*16 array, dimension KBOT
        On output, approximate eigenvalues that may
        be used for shifts are stored in SH(KBOT-ND-NS+1)
        through SR(KBOT-ND).  Converged eigenvalues are
        stored in SH(KBOT-ND+1) through SH(KBOT).

[out] V

        V is COMPLEX*16 array, dimension (LDV,NW)
        An NW-by-NW work array.

[in] LDV

        LDV is integer scalar
        The leading dimension of V just as declared in the
        calling subroutine.  NW .LE. LDV

[in] NH

        NH is integer scalar
        The number of columns of T.  NH.GE.NW.

[out] T

        T is COMPLEX*16 array, dimension (LDT,NW)

[in] LDT

        LDT is integer
        The leading dimension of T just as declared in the
        calling subroutine.  NW .LE. LDT

[in] NV

        NV is integer
        The number of rows of work array WV available for
        workspace.  NV.GE.NW.

[out] WV

        WV is COMPLEX*16 array, dimension (LDWV,NW)
```

```
[in] LDWV

        LDWV is integer
        The leading dimension of W just as declared in the
        calling subroutine.  NW .LE. LDV

[out] WORK

        WORK is COMPLEX*16 array, dimension LWORK.
        On exit, WORK(1) is set to an estimate of the optimal value
        of LWORK for the given values of N, NW, KTOP and KBOT.

[in] LWORK

        LWORK is integer
        The dimension of the work array WORK.  LWORK = 2*NW
        suffices, but greater efficiency may result from larger
        values of LWORK.

        If LWORK = -1, then a workspace query is assumed; ZLAQR2
        only estimates the optimal workspace size for the given
        values of N, NW, KTOP and KBOT.  The estimate is returned
        in WORK(1).  No error message related to LWORK is issued
        by XERBLA.  Neither H nor Z are accessed.
```

———-

90.225.3 fortran code

— zlaqr2.f —

```
*  =====================================================================
      SUBROUTINE ZLAQR2( WANTT, WANTZ, N, KTOP, KBOT, NW, H, LDH, ILOZ,
     $                   IHIZ, Z, LDZ, NS, ND, SH, V, LDV, NH, T, LDT,
     $                   NV, WV, LDWV, WORK, LWORK )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            IHIZ, ILOZ, KBOT, KTOP, LDH, LDT, LDV, LDWV,
     $                   LDZ, LWORK, N, ND, NH, NS, NV, NW
      LOGICAL            WANTT, WANTZ
*     ..
*     .. Array Arguments ..
```

```
      COMPLEX*16         H( LDH, * ), SH( * ), T( LDT, * ), V( LDV, * ),
     $                   WORK( * ), WV( LDWV, * ), Z( LDZ, * )
*     ..
*
*  ================================================================
*
*     .. Parameters ..
      COMPLEX*16         ZERO, ONE
      PARAMETER          ( ZERO = ( 0.0d0, 0.0d0 ),
     $                   ONE = ( 1.0d0, 0.0d0 ) )
      DOUBLE PRECISION   RZERO, RONE
      PARAMETER          ( RZERO = 0.0d0, RONE = 1.0d0 )
*     ..
*     .. Local Scalars ..
      COMPLEX*16         BETA, CDUM, S, TAU
      DOUBLE PRECISION   FOO, SAFMAX, SAFMIN, SMLNUM, ULP
      INTEGER            I, IFST, ILST, INFO, INFQR, J, JW, KCOL, KLN,
     $                   KNT, KROW, KWTOP, LTOP, LWK1, LWK2, LWKOPT
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, ZCOPY, ZGEHRD, ZGEMM, ZLACPY, ZLAHQR,
     $                   ZLARF, ZLARFG, ZLASET, ZTREXC, ZUNMHR
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DCMPLX, DCONJG, DIMAG, INT, MAX, MIN
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( CDUM ) = ABS( DBLE( CDUM ) ) + ABS( DIMAG( CDUM ) )
*     ..
*     .. Executable Statements ..
*
*     ==== Estimate optimal workspace. ====
*
      JW = MIN( NW, KBOT-KTOP+1 )
      IF( JW.LE.2 ) THEN
         LWKOPT = 1
      ELSE
*
*        ==== Workspace query call to ZGEHRD ====
*
         CALL ZGEHRD( JW, 1, JW-1, T, LDT, WORK, WORK, -1, INFO )
         LWK1 = INT( WORK( 1 ) )
*
```

```
*        ==== Workspace query call to ZUNMHR ====
*
         CALL ZUNMHR( 'R', 'N', JW, JW, 1, JW-1, T, LDT, WORK, V, LDV,
     $                WORK, -1, INFO )
         LWK2 = INT( WORK( 1 ) )
*
*        ==== Optimal workspace ====
*
         LWKOPT = JW + MAX( LWK1, LWK2 )
      END IF
*
*     ==== Quick return in case of workspace query. ====
*
      IF( LWORK.EQ.-1 ) THEN
         WORK( 1 ) = DCMPLX( LWKOPT, 0 )
         RETURN
      END IF
*
*     ==== Nothing to do ...
*     ... for an empty active block ... ====
      NS = 0
      ND = 0
      WORK( 1 ) = ONE
      IF( KTOP.GT.KBOT )
     $   RETURN
*     ... nor for an empty deflation window. ====
      IF( NW.LT.1 )
     $   RETURN
*
*     ==== Machine constants ====
*
      SAFMIN = DLAMCH( 'SAFE MINIMUM' )
      SAFMAX = RONE / SAFMIN
      CALL DLABAD( SAFMIN, SAFMAX )
      ULP = DLAMCH( 'PRECISION' )
      SMLNUM = SAFMIN*( DBLE( N ) / ULP )
*
*     ==== Setup deflation window ====
*
      JW = MIN( NW, KBOT-KTOP+1 )
      KWTOP = KBOT - JW + 1
      IF( KWTOP.EQ.KTOP ) THEN
         S = ZERO
      ELSE
         S = H( KWTOP, KWTOP-1 )
      END IF
*
      IF( KBOT.EQ.KWTOP ) THEN
*
*        ==== 1-by-1 deflation window: not much to do ====
```

```
*
         SH( KWTOP ) = H( KWTOP, KWTOP )
         NS = 1
         ND = 0
         IF( CABS1( S ).LE.MAX( SMLNUM, ULP*CABS1( H( KWTOP,
     $       KWTOP ) ) ) ) THEN
            NS = 0
            ND = 1
            IF( KWTOP.GT.KTOP )
     $         H( KWTOP, KWTOP-1 ) = ZERO
         END IF
         WORK( 1 ) = ONE
         RETURN
      END IF
*
*     ==== Convert to spike-triangular form.  (In case of a
*     .    rare QR failure, this routine continues to do
*     .    aggressive early deflation using that part of
*     .    the deflation window that converged using INFQR
*     .    here and there to keep track.) ====
*
      CALL ZLACPY( 'U', JW, JW, H( KWTOP, KWTOP ), LDH, T, LDT )
      CALL ZCOPY( JW-1, H( KWTOP+1, KWTOP ), LDH+1, T( 2, 1 ), LDT+1 )
*
      CALL ZLASET( 'A', JW, JW, ZERO, ONE, V, LDV )
      CALL ZLAHQR( .true., .true., JW, 1, JW, T, LDT, SH( KWTOP ), 1,
     $             JW, V, LDV, INFQR )
*
*     ==== Deflation detection loop ====
*
      NS = JW
      ILST = INFQR + 1
      DO 10 KNT = INFQR + 1, JW
*
*        ==== Small spike tip deflation test ====
*
         FOO = CABS1( T( NS, NS ) )
         IF( FOO.EQ.RZERO )
     $      FOO = CABS1( S )
         IF( CABS1( S )*CABS1( V( 1, NS ) ).LE.MAX( SMLNUM, ULP*FOO ) )
     $        THEN
*
*           ==== One more converged eigenvalue ====
*
            NS = NS - 1
         ELSE
*
*           ==== One undeflatable eigenvalue.  Move it up out of the
*           .    way.   (ZTREXC can not fail in this case.) ====
*
```

```
            IFST = NS
            CALL ZTREXC( 'V', JW, T, LDT, V, LDV, IFST, ILST, INFO )
            ILST = ILST + 1
         END IF
   10 CONTINUE
*
*        ==== Return to Hessenberg form ====
*
      IF( NS.EQ.0 )
     $   S = ZERO
*
      IF( NS.LT.JW ) THEN
*
         ==== sorting the diagonal of T improves accuracy for
*        .    graded matrices.  ====
*
         DO 30 I = INFQR + 1, NS
            IFST = I
            DO 20 J = I + 1, NS
               IF( CABS1( T( J, J ) ).GT.CABS1( T( IFST, IFST ) ) )
     $            IFST = J
   20       CONTINUE
            ILST = I
            IF( IFST.NE.ILST )
     $         CALL ZTREXC( 'V', JW, T, LDT, V, LDV, IFST, ILST, INFO )
   30    CONTINUE
      END IF
*
*     ==== Restore shift/eigenvalue array from T ====
*
      DO 40 I = INFQR + 1, JW
         SH( KWTOP+I-1 ) = T( I, I )
   40 CONTINUE
*
*
      IF( NS.LT.JW .OR. S.EQ.ZERO ) THEN
         IF( NS.GT.1 .AND. S.NE.ZERO ) THEN
*
            ==== Reflect spike back into lower triangle ====
*
            CALL ZCOPY( NS, V, LDV, WORK, 1 )
            DO 50 I = 1, NS
               WORK( I ) = DCONJG( WORK( I ) )
   50       CONTINUE
            BETA = WORK( 1 )
            CALL ZLARFG( NS, BETA, WORK( 2 ), 1, TAU )
            WORK( 1 ) = ONE
*
            CALL ZLASET( 'L', JW-2, JW-2, ZERO, ZERO, T( 3, 1 ), LDT )
*
```

```
            CALL ZLARF( 'L', NS, JW, WORK, 1, DCONJG( TAU ), T, LDT,
     $                  WORK( JW+1 ) )
            CALL ZLARF( 'R', NS, NS, WORK, 1, TAU, T, LDT,
     $                  WORK( JW+1 ) )
            CALL ZLARF( 'R', JW, NS, WORK, 1, TAU, V, LDV,
     $                  WORK( JW+1 ) )
*
            CALL ZGEHRD( JW, 1, NS, T, LDT, WORK, WORK( JW+1 ),
     $                   LWORK-JW, INFO )
         END IF
*
*        ==== Copy updated reduced window into place ====
*
         IF( KWTOP.GT.1 )
     $      H( KWTOP, KWTOP-1 ) = S*DCONJG( V( 1, 1 ) )
         CALL ZLACPY( 'U', JW, JW, T, LDT, H( KWTOP, KWTOP ), LDH )
         CALL ZCOPY( JW-1, T( 2, 1 ), LDT+1, H( KWTOP+1, KWTOP ),
     $               LDH+1 )
*
*        ==== Accumulate orthogonal matrix in order update
*        .    H and Z, if requested.  ====
*
         IF( NS.GT.1 .AND. S.NE.ZERO )
     $      CALL ZUNMHR( 'R', 'N', JW, NS, 1, NS, T, LDT, WORK, V, LDV,
     $                   WORK( JW+1 ), LWORK-JW, INFO )
*
*        ==== Update vertical slab in H ====
*
         IF( WANTT ) THEN
            LTOP = 1
         ELSE
            LTOP = KTOP
         END IF
         DO 60 KROW = LTOP, KWTOP - 1, NV
            KLN = MIN( NV, KWTOP-KROW )
            CALL ZGEMM( 'N', 'N', KLN, JW, JW, ONE, H( KROW, KWTOP ),
     $                  LDH, V, LDV, ZERO, WV, LDWV )
            CALL ZLACPY( 'A', KLN, JW, WV, LDWV, H( KROW, KWTOP ), LDH )
   60    CONTINUE
*
*        ==== Update horizontal slab in H ====
*
         IF( WANTT ) THEN
            DO 70 KCOL = KBOT + 1, N, NH
               KLN = MIN( NH, N-KCOL+1 )
               CALL ZGEMM( 'C', 'N', JW, KLN, JW, ONE, V, LDV,
     $                     H( KWTOP, KCOL ), LDH, ZERO, T, LDT )
               CALL ZLACPY( 'A', JW, KLN, T, LDT, H( KWTOP, KCOL ),
     $                      LDH )
   70       CONTINUE
```

```
         END IF
*
*        ==== Update vertical slab in Z ====
*
         IF( WANTZ ) THEN
            DO 80 KROW = ILOZ, IHIZ, NV
               KLN = MIN( NV, IHIZ-KROW+1 )
               CALL ZGEMM( 'N', 'N', KLN, JW, JW, ONE, Z( KROW, KWTOP ),
     $                     LDZ, V, LDV, ZERO, WV, LDWV )
               CALL ZLACPY( 'A', KLN, JW, WV, LDWV, Z( KROW, KWTOP ),
     $                      LDZ )
   80       CONTINUE
         END IF
      END IF
*
*     ==== Return the number of deflations ... ====
*
      ND = JW - NS
*
*     ==== ... and the number of shifts. (Subtracting
*     .    INFQR from the spike length takes care
*     .    of the case of a rare QR failure while
*     .    calculating eigenvalues of the deflation
*     .    window.)  ====
*
      NS = NS - INFQR
*
*      ==== Return optimal workspace. ====
*
      WORK( 1 ) = DCMPLX( LWKOPT, 0 )
*
*     ==== End of ZLAQR2 ====
*
      END
```

———-

90.225.4 lisp code

— zlaqr2.lisp —

```
(let* ((zero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (one (f2cl-lib:cmplx 1.0d0 0.0d0))
       (rzero 0.0d0)
       (rone 1.0d0))
  (declare (type (f2cl-lib:complex16) zero)
           (type (f2cl-lib:complex16) one)
```

```
          (type (double-float 0.0d0 0.0d0) rzero)
          (type (double-float 1.0d0 1.0d0) rone)
          (ignorable zero one rzero rone))
 (defun zlaqr2
        (wantt wantz n ktop kbot nw h ldh iloz ihiz z ldz ns nd sh v ldv nh t$
         ldt nv wv ldwv work lwork)
   (declare (type (array f2cl-lib:complex16 (*)) work wv t$ v sh z h)
            (type (f2cl-lib:integer4) lwork ldwv nv ldt nh ldv nd ns ldz ihiz
             iloz ldh nw kbot ktop n)
            (type f2cl-lib:logical wantz wantt))
   (f2cl-lib:with-multi-array-data
       ((h f2cl-lib:complex16 h-%data% h-%offset%)
        (z f2cl-lib:complex16 z-%data% z-%offset%)
        (sh f2cl-lib:complex16 sh-%data% sh-%offset%)
        (v f2cl-lib:complex16 v-%data% v-%offset%)
        (t$ f2cl-lib:complex16 t$-%data% t$-%offset%)
        (wv f2cl-lib:complex16 wv-%data% wv-%offset%)
        (work f2cl-lib:complex16 work-%data% work-%offset%))
     (labels ((cabs1 (cdum)
                (+ (abs (f2cl-lib:dble cdum)) (abs (f2cl-lib:dimag cdum)))))
       (declare
        (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
         cabs1))
       (prog ((i 0) (ifst 0) (ilst 0) (info 0) (infqr 0) (j 0) (jw 0) (kcol 0)
              (kln 0) (knt 0) (krow 0) (kwtop 0) (ltop 0) (lwk1 0) (lwk2 0)
              (lwkopt 0) (foo 0.0d0) (safmax 0.0d0) (safmin 0.0d0)
              (smlnum 0.0d0) (ulp 0.0d0) (beta #C(0.0d0 0.0d0))
              (cdum #C(0.0d0 0.0d0)) (s #C(0.0d0 0.0d0))
              (tau #C(0.0d0 0.0d0)))
         (declare
          (type (f2cl-lib:integer4) i ifst ilst info infqr j jw kcol kln knt
           krow kwtop ltop lwk1 lwk2 lwkopt)
          (type (double-float) foo safmax safmin smlnum ulp)
          (type (f2cl-lib:complex16) beta cdum s tau))
         (setf jw
                 (min (the f2cl-lib:integer4 nw)
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add (f2cl-lib:int-sub kbot ktop)
                                             1))))
         (cond ((<= jw 2) (setf lwkopt 1))
               (t
                (multiple-value-bind
                    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                    (zgehrd jw 1 (f2cl-lib:int-sub jw 1) t$ ldt work work -1
                     info)
                  (declare (ignore var-1 var-2 var-3 var-5 var-6 var-7))
                  (setf jw var-0)
                  (setf ldt var-4)
                  (setf info var-8))
                (setf lwk1
```

```
                (f2cl-lib:int
                 (f2cl-lib:fref work-%data% (1) ((1 *))
                                work-%offset%)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12 var-13)
            (zunmhr "R" "N" jw jw 1 (f2cl-lib:int-sub jw 1) t$ ldt
             work v ldv work -1 info)
          (declare
           (ignore var-0 var-1 var-4 var-5 var-6 var-8 var-9 var-11
            var-12))
          (when var-2 (setf jw var-2))
          (when var-3 (setf jw var-3))
          (when var-7 (setf ldt var-7))
          (when var-10 (setf ldv var-10))
          (when var-13 (setf info var-13)))
        (setf lwk2
                (f2cl-lib:int
                 (f2cl-lib:fref work-%data% (1) ((1 *))
                                work-%offset%)))
        (setf lwkopt
                (f2cl-lib:int-add jw
                                  (max (the f2cl-lib:integer4 lwk1)
                                       (the f2cl-lib:integer4
                                            lwk2))))))
(cond
 ((= lwork (f2cl-lib:int-sub 1))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:dcmplx lwkopt 0))
  (go end_label)))
(setf ns 0)
(setf nd 0)
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) one)
(if (> ktop kbot)
    (go end_label))
(if (< nw 1)
    (go end_label))
(setf safmin (dlamch "SAFE MINIMUM"))
(setf safmax (/ rone safmin))
(multiple-value-bind (var-0 var-1)
    (dlabad safmin safmax)
  (declare (ignore))
  (when var-0 (setf safmin var-0))
  (when var-1 (setf safmax var-1)))
(setf ulp (dlamch "PRECISION"))
(setf smlnum (* safmin (/ (f2cl-lib:dble n) ulp)))
(setf jw
        (min (the f2cl-lib:integer4 nw)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub kbot ktop)
```

```
                                    1))))
(setf kwtop (f2cl-lib:int-add (f2cl-lib:int-sub kbot jw) 1))
(cond ((= kwtop ktop) (setf s zero))
      (t
       (setf s
               (f2cl-lib:fref h-%data%
                              (kwtop (f2cl-lib:int-sub kwtop 1))
                              ((1 ldh) (1 *)) h-%offset%))))
(cond
 ((= kbot kwtop)
  (setf (f2cl-lib:fref sh-%data% (kwtop) ((1 *)) sh-%offset%)
          (f2cl-lib:fref h-%data% (kwtop kwtop) ((1 ldh) (1 *))
                         h-%offset%))
  (setf ns 1) (setf nd 0)
  (cond
   ((<= (cabs1 s)
        (max smlnum
             (* ulp
                (cabs1
                 (f2cl-lib:fref h (kwtop kwtop) ((1 ldh) (1 *)))))))
    (setf ns 0) (setf nd 1)
    (if (> kwtop ktop)
        (setf (f2cl-lib:fref h-%data%
                             (kwtop (f2cl-lib:int-sub kwtop 1))
                             ((1 ldh) (1 *)) h-%offset%)
                zero))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) one)
  (go end_label)))
(zlacpy "U" jw jw
 (f2cl-lib:array-slice h-%data% f2cl-lib:complex16 (kwtop kwtop)
                       ((1 ldh) (1 *)) h-%offset%)
 ldh t$ ldt)
(zcopy (f2cl-lib:int-sub jw 1)
 (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                       ((+ kwtop 1) kwtop) ((1 ldh) (1 *))
                       h-%offset%)
 (f2cl-lib:int-add ldh 1)
 (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (2 1)
                       ((1 ldt) (1 *)) t$-%offset%)
 (f2cl-lib:int-add ldt 1))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "A" jw jw zero one v ldv)
  (declare (ignore var-0 var-5))
  (when var-1 (setf jw var-1))
  (when var-2 (setf jw var-2))
  (when var-3 (setf zero var-3))
  (when var-4 (setf one var-4))
  (when var-6 (setf ldv var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
```

```
     var-10 var-11 var-12)
    (zlahqr f2cl-lib:%true% f2cl-lib:%true% jw 1 jw t$ ldt
     (f2cl-lib:array-slice sh-%data% f2cl-lib:complex16 (kwtop)
                           ((1 *)) sh-%offset%)
     1 jw v ldv infqr)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-8 var-9
    var-10 var-11))
  (setf ldt var-6)
  (setf infqr var-12))
(setf ns jw)
(setf ilst (f2cl-lib:int-add infqr 1))
(f2cl-lib:fdo (knt (f2cl-lib:int-add infqr 1)
               (f2cl-lib:int-add knt 1))
              ((> knt jw) nil)
  (tagbody
    (setf foo
            (cabs1
             (f2cl-lib:fref t$-%data% (ns ns) ((1 ldt) (1 *))
                            t$-%offset%)))
    (if (= foo rzero)
        (setf foo (cabs1 s)))
    (cond
     ((<=
       (* (cabs1 s) (cabs1 (f2cl-lib:fref v (1 ns) ((1 ldv) (1 *)))))
       (max smlnum (* ulp foo)))
      (setf ns (f2cl-lib:int-sub ns 1)))
     (t (setf ifst ns)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (ztrexc "V" jw t$ ldt v ldv ifst ilst info)
        (declare (ignore var-0 var-2 var-4))
        (when var-1 (setf jw var-1))
        (when var-3 (setf ldt var-3))
        (when var-5 (setf ldv var-5))
        (when var-6 (setf ifst var-6))
        (when var-7 (setf ilst var-7))
        (when var-8 (setf info var-8)))
      (setf ilst (f2cl-lib:int-add ilst 1))))
   label10))
(if (= ns 0)
    (setf s zero))
(cond
 ((< ns jw)
  (f2cl-lib:fdo (i (f2cl-lib:int-add infqr 1) (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
      (setf ifst i)
      (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                    ((> j ns) nil)
```

```
          (tagbody
            (if (>
                 (cabs1
                  (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                                 t$-%offset%))
                 (cabs1
                  (f2cl-lib:fref t$-%data% (ifst ifst) ((1 ldt) (1 *))
                                 t$-%offset%)))
                (setf ifst j))
           label20))
        (setf ilst i)
        (if (/= ifst ilst)
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (ztrexc "V" jw t$ ldt v ldv ifst ilst info)
              (declare (ignore var-0 var-2 var-4))
              (when var-1 (setf jw var-1))
              (when var-3 (setf ldt var-3))
              (when var-5 (setf ldv var-5))
              (when var-6 (setf ifst var-6))
              (when var-7 (setf ilst var-7))
              (when var-8 (setf info var-8))))
       label30))))
  (f2cl-lib:fdo (i (f2cl-lib:int-add infqr 1) (f2cl-lib:int-add i 1))
                ((> i jw) nil)
    (tagbody
      (setf (f2cl-lib:fref sh-%data%
                           ((f2cl-lib:int-sub
                             (f2cl-lib:int-add kwtop i) 1))
                           ((1 *)) sh-%offset%)
            (f2cl-lib:fref t$-%data% (i i) ((1 ldt) (1 *))
                           t$-%offset%))
     label40))
  (cond
   ((or (< ns jw) (= s zero))
    (cond
     ((and (> ns 1) (/= s zero))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (zcopy ns v ldv work 1)
        (declare (ignore var-1 var-3 var-4))
        (when var-0 (setf ns var-0))
        (when var-2 (setf ldv var-2)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i ns) nil)
        (tagbody
          (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                  (coerce
                   (f2cl-lib:dconjg
                    (f2cl-lib:fref work-%data% (i) ((1 *))
                                   work-%offset%))
```

```
              'f2cl-lib:complex16))
   label50))
(setf beta (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (zlarfg ns beta
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (2)
                           ((1 *)) work-%offset%)
     1 tau)
  (declare (ignore var-2 var-3))
  (when var-0 (setf ns var-0))
  (when var-1 (setf beta var-1))
  (when var-4 (setf tau var-4)))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) one)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "L" (f2cl-lib:int-sub jw 2) (f2cl-lib:int-sub jw 2)
     zero zero
     (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (3 1)
                           ((1 ldt) (1 *)) t$-%offset%)
     ldt)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zlarf "L" ns jw work 1 (f2cl-lib:dconjg tau) t$ ldt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ jw 1)) ((1 *)) work-%offset%))
  (declare (ignore var-0 var-3 var-4 var-5 var-6 var-8))
  (when var-1 (setf ns var-1))
  (when var-2 (setf jw var-2))
  (when var-7 (setf ldt var-7)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zlarf "R" ns ns work 1 tau t$ ldt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ jw 1)) ((1 *)) work-%offset%))
  (declare (ignore var-0 var-3 var-4 var-6 var-8))
  (when var-1 (setf ns var-1))
  (when var-2 (setf ns var-2))
  (when var-5 (setf tau var-5))
  (when var-7 (setf ldt var-7)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zlarf "R" jw ns work 1 tau v ldv
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ jw 1)) ((1 *)) work-%offset%))
  (declare (ignore var-0 var-3 var-4 var-6 var-8))
  (when var-1 (setf jw var-1))
  (when var-2 (setf ns var-2))
```

```
    (when var-5 (setf tau var-5))
    (when var-7 (setf ldv var-7)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
      (zgehrd jw 1 ns t$ ldt work
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             ((+ jw 1)) ((1 *)) work-%offset%)
       (f2cl-lib:int-sub lwork jw) info)
    (declare (ignore var-1 var-3 var-5 var-6 var-7))
    (setf jw var-0)
    (setf ns var-2)
    (setf ldt var-4)
    (setf info var-8))))
(if (> kwtop 1)
    (setf (f2cl-lib:fref h-%data%
                         (kwtop (f2cl-lib:int-sub kwtop 1))
                         ((1 ldh) (1 *)) h-%offset%)
            (* s
               (f2cl-lib:dconjg
                (f2cl-lib:fref v-%data% (1 1) ((1 ldv) (1 *))
                               v-%offset%)))))
(zlacpy "U" jw jw t$ ldt
 (f2cl-lib:array-slice h-%data% f2cl-lib:complex16 (kwtop kwtop)
                       ((1 ldh) (1 *)) h-%offset%)
 ldh)
(zcopy (f2cl-lib:int-sub jw 1)
 (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (2 1)
                       ((1 ldt) (1 *)) t$-%offset%)
 (f2cl-lib:int-add ldt 1)
 (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                       ((+ kwtop 1) kwtop) ((1 ldh) (1 *))
                       h-%offset%)
 (f2cl-lib:int-add ldh 1))
(if (and (> ns 1) (/= s zero))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13)
        (zunmhr "R" "N" jw ns 1 ns t$ ldt work v ldv
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               ((+ jw 1)) ((1 *)) work-%offset%)
         (f2cl-lib:int-sub lwork jw) info)
      (declare
       (ignore var-0 var-1 var-4 var-6 var-8 var-9 var-11 var-12))
      (when var-2 (setf jw var-2))
      (when var-3 (setf ns var-3))
      (when var-5 (setf ns var-5))
      (when var-7 (setf ldt var-7))
      (when var-10 (setf ldv var-10))
      (when var-13 (setf info var-13))))
(cond (wantt (setf ltop 1)) (t (setf ltop ktop)))
```

```
(f2cl-lib:fdo (krow ltop (f2cl-lib:int-add krow nv))
              ((> krow
                  (f2cl-lib:int-add kwtop (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf kln
            (min (the f2cl-lib:integer4 nv)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-sub kwtop krow))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (zgemm "N" "N" kln jw jw one
         (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                               (krow kwtop) ((1 ldh) (1 *))
                               h-%offset%)
         ldh v ldv zero wv ldwv)
      (declare (ignore var-0 var-1 var-6 var-8 var-11))
      (when var-2 (setf kln var-2))
      (when var-3 (setf jw var-3))
      (when var-4 (setf jw var-4))
      (when var-5 (setf one var-5))
      (when var-7 (setf ldh var-7))
      (when var-9 (setf ldv var-9))
      (when var-10 (setf zero var-10))
      (when var-12 (setf ldwv var-12)))
    (zlacpy "A" kln jw wv ldwv
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16 (krow kwtop)
                           ((1 ldh) (1 *)) h-%offset%)
     ldh)
   label60))
(cond
 (wantt
  (f2cl-lib:fdo (kcol (f2cl-lib:int-add kbot 1)
                 (f2cl-lib:int-add kcol nh))
                ((> kcol n) nil)
    (tagbody
      (setf kln
              (min (the f2cl-lib:integer4 nh)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub n kcol)
                                          1))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
           var-9 var-10 var-11 var-12)
          (zgemm "C" "N" jw kln jw one v ldv
           (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                                 (kwtop kcol) ((1 ldh) (1 *))
                                 h-%offset%)
           ldh zero t$ ldt)
```

```
            (declare (ignore var-0 var-1 var-6 var-8 var-11))
            (when var-2 (setf jw var-2))
            (when var-3 (setf kln var-3))
            (when var-4 (setf jw var-4))
            (when var-5 (setf one var-5))
            (when var-7 (setf ldv var-7))
            (when var-9 (setf ldh var-9))
            (when var-10 (setf zero var-10))
            (when var-12 (setf ldt var-12)))
          (zlacpy "A" jw kln t$ ldt
           (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                                 (kwtop kcol) ((1 ldh) (1 *))
                                 h-%offset%)
           ldh)
         label70))))
  (cond
   (wantz
    (f2cl-lib:fdo (krow iloz (f2cl-lib:int-add krow nv))
                  ((> krow ihiz) nil)
      (tagbody
        (setf kln
                (min (the f2cl-lib:integer4 nv)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add
                           (f2cl-lib:int-sub ihiz krow) 1))))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12)
            (zgemm "N" "N" kln jw jw one
             (f2cl-lib:array-slice z-%data% f2cl-lib:complex16
                                   (krow kwtop) ((1 ldz) (1 *))
                                   z-%offset%)
             ldz v ldv zero wv ldwv)
          (declare (ignore var-0 var-1 var-6 var-8 var-11))
          (when var-2 (setf kln var-2))
          (when var-3 (setf jw var-3))
          (when var-4 (setf jw var-4))
          (when var-5 (setf one var-5))
          (when var-7 (setf ldz var-7))
          (when var-9 (setf ldv var-9))
          (when var-10 (setf zero var-10))
          (when var-12 (setf ldwv var-12)))
        (zlacpy "A" kln jw wv ldwv
         (f2cl-lib:array-slice z-%data% f2cl-lib:complex16
                               (krow kwtop) ((1 ldz) (1 *))
                               z-%offset%)
         ldz)
       label80))))))
(setf nd (f2cl-lib:int-sub jw ns))
(setf ns (f2cl-lib:int-sub ns infqr))
```

```
          (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                  (f2cl-lib:dcmplx lwkopt 0))
        end_label
          (return
           (values nil nil nil nil nil nil nil ldh nil nil nil ldz ns nd nil
                   nil ldv nil nil ldt nil nil ldwv nil nil)))))))
```

———-

90.226 zlaqr3

90.226.1 Axiom unit tests

— **zlaqr3.input** —

```
)set break resume
)sys rm -f zlaqr3.output
)spool zlaqr3.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.226.2 Axiom help page

— **zlaqr3.help** —

```
======================================================================
zlaqr3 examples
======================================================================

======================================================================
Man Page Details
======================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
```

```
===========

      SUBROUTINE ZLAQR3( WANTT, WANTZ, N, KTOP, KBOT, NW, H, LDH, ILOZ,
                         IHIZ, Z, LDZ, NS, ND, SH, V, LDV, NH, T, LDT,
                         NV, WV, LDWV, WORK, LWORK )

      .. Scalar Arguments ..
      INTEGER            IHIZ, ILOZ, KBOT, KTOP, LDH, LDT, LDV, LDWV,
     $                   LDZ, LWORK, N, ND, NH, NS, NV, NW
      LOGICAL            WANTT, WANTZ
      ..
      .. Array Arguments ..
      COMPLEX*16         H( LDH, * ), SH( * ), T( LDT, * ), V( LDV, * ),
     $                   WORK( * ), WV( LDWV, * ), Z( LDZ, * )
      ..


Purpose:
=============

   Aggressive early deflation:

   ZLAQR3 accepts as input an upper Hessenberg matrix
   H and performs an unitary similarity transformation
   designed to detect and deflate fully converged eigenvalues from
   a trailing principal submatrix.  On output H has been over-
   written by a new Hessenberg matrix that is a perturbation of
   an unitary similarity transformation of H.  It is to be
   hoped that the final version of H has many zero subdiagonal
   entries.

Arguments:
==========

 [in] WANTT

         WANTT is LOGICAL
         If .TRUE., then the Hessenberg matrix H is fully updated
         so that the triangular Schur factor may be
         computed (in cooperation with the calling subroutine).
         If .FALSE., then only enough of H is updated to preserve
         the eigenvalues.

 [in] WANTZ

         WANTZ is LOGICAL
         If .TRUE., then the unitary matrix Z is updated so
         so that the unitary Schur factor may be computed
         (in cooperation with the calling subroutine).
         If .FALSE., then Z is not referenced.
```

```
[in] N

        N is INTEGER
        The order of the matrix H and (if WANTZ is .TRUE.) the
        order of the unitary matrix Z.

[in] KTOP

        KTOP is INTEGER
        It is assumed that either KTOP = 1 or H(KTOP,KTOP-1)=0.
        KBOT and KTOP together determine an isolated block
        along the diagonal of the Hessenberg matrix.

[in] KBOT

        KBOT is INTEGER
        It is assumed without a check that either
        KBOT = N or H(KBOT+1,KBOT)=0.  KBOT and KTOP together
        determine an isolated block along the diagonal of the
        Hessenberg matrix.

[in] NW

        NW is INTEGER
        Deflation window size.  1 .LE. NW .LE. (KBOT-KTOP+1).

[in,out] H

        H is COMPLEX*16 array, dimension (LDH,N)
        On input the initial N-by-N section of H stores the
        Hessenberg matrix undergoing aggressive early deflation.
        On output H has been transformed by a unitary
        similarity transformation, perturbed, and the returned
        to Hessenberg form that (it is to be hoped) has some
        zero subdiagonal entries.

[in] LDH

        LDH is integer
        Leading dimension of H just as declared in the calling
        subroutine.  N .LE. LDH

[in] ILOZ

        ILOZ is INTEGER

[in] IHIZ

        IHIZ is INTEGER
```

```
        Specify the rows of Z to which transformations must be
        applied if WANTZ is .TRUE.. 1 .LE. ILOZ .LE. IHIZ .LE. N.

[in,out] Z

        Z is COMPLEX*16 array, dimension (LDZ,N)
        IF WANTZ is .TRUE., then on output, the unitary
        similarity transformation mentioned above has been
        accumulated into Z(ILOZ:IHIZ,ILO:IHI) from the right.
        If WANTZ is .FALSE., then Z is unreferenced.

[in] LDZ

        LDZ is integer
        The leading dimension of Z just as declared in the
        calling subroutine.  1 .LE. LDZ.

[out] NS

        NS is integer
        The number of unconverged (ie approximate) eigenvalues
        returned in SR and SI that may be used as shifts by the
        calling subroutine.

[out] ND

        ND is integer
        The number of converged eigenvalues uncovered by this
        subroutine.

[out] SH

        SH is COMPLEX*16 array, dimension KBOT
        On output, approximate eigenvalues that may
        be used for shifts are stored in SH(KBOT-ND-NS+1)
        through SR(KBOT-ND).  Converged eigenvalues are
        stored in SH(KBOT-ND+1) through SH(KBOT).

[out] V

        V is COMPLEX*16 array, dimension (LDV,NW)
        An NW-by-NW work array.

[in] LDV

        LDV is integer scalar
        The leading dimension of V just as declared in the
        calling subroutine.  NW .LE. LDV

[in] NH
```

```
        NH is integer scalar
        The number of columns of T.  NH.GE.NW.

[out] T

        T is COMPLEX*16 array, dimension (LDT,NW)

[in] LDT

        LDT is integer
        The leading dimension of T just as declared in the
        calling subroutine.  NW .LE. LDT

[in] NV

        NV is integer
        The number of rows of work array WV available for
        workspace.  NV.GE.NW.

[out] WV

        WV is COMPLEX*16 array, dimension (LDWV,NW)

[in] LDWV

        LDWV is integer
        The leading dimension of W just as declared in the
        calling subroutine.  NW .LE. LDV

[out] WORK

        WORK is COMPLEX*16 array, dimension LWORK.
        On exit, WORK(1) is set to an estimate of the optimal value
        of LWORK for the given values of N, NW, KTOP and KBOT.

[in] LWORK

        LWORK is integer
        The dimension of the work array WORK.  LWORK = 2*NW
        suffices, but greater efficiency may result from larger
        values of LWORK.

        If LWORK = -1, then a workspace query is assumed; ZLAQR3
        only estimates the optimal workspace size for the given
        values of N, NW, KTOP and KBOT.  The estimate is returned
        in WORK(1).  No error message related to LWORK is issued
        by XERBLA.  Neither H nor Z are accessed.
```

———-

90.226.3 fortran code

— **zlaqr3.f** —

```
*  ====================================================================
      SUBROUTINE ZLAQR3( WANTT, WANTZ, N, KTOP, KBOT, NW, H, LDH, ILOZ,
     $                   IHIZ, Z, LDZ, NS, ND, SH, V, LDV, NH, T, LDT,
     $                   NV, WV, LDWV, WORK, LWORK )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            IHIZ, ILOZ, KBOT, KTOP, LDH, LDT, LDV, LDWV,
     $                   LDZ, LWORK, N, ND, NH, NS, NV, NW
      LOGICAL            WANTT, WANTZ
*     ..
*     .. Array Arguments ..
      COMPLEX*16         H( LDH, * ), SH( * ), T( LDT, * ), V( LDV, * ),
     $                   WORK( * ), WV( LDWV, * ), Z( LDZ, * )
*     ..
*
*  ================================================================
*
*     .. Parameters ..
      COMPLEX*16         ZERO, ONE
      PARAMETER          ( ZERO = ( 0.0d0, 0.0d0 ),
     $                   ONE = ( 1.0d0, 0.0d0 ) )
      DOUBLE PRECISION   RZERO, RONE
      PARAMETER          ( RZERO = 0.0d0, RONE = 1.0d0 )
*     ..
*     .. Local Scalars ..
      COMPLEX*16         BETA, CDUM, S, TAU
      DOUBLE PRECISION   FOO, SAFMAX, SAFMIN, SMLNUM, ULP
      INTEGER            I, IFST, ILST, INFO, INFQR, J, JW, KCOL, KLN,
     $                   KNT, KROW, KWTOP, LTOP, LWK1, LWK2, LWK3,
     $                   LWKOPT, NMIN
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      INTEGER            ILAENV
      EXTERNAL           DLAMCH, ILAENV
*     ..
*     .. External Subroutines ..
```

```
      EXTERNAL           DLABAD, ZCOPY, ZGEHRD, ZGEMM, ZLACPY, ZLAHQR,
     $                   ZLAQR4, ZLARF, ZLARFG, ZLASET, ZTREXC, ZUNMHR
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DCMPLX, DCONJG, DIMAG, INT, MAX, MIN
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( CDUM ) = ABS( DBLE( CDUM ) ) + ABS( DIMAG( CDUM ) )
*     ..
*     .. Executable Statements ..
*
*     ==== Estimate optimal workspace. ====
*
      JW = MIN( NW, KBOT-KTOP+1 )
      IF( JW.LE.2 ) THEN
         LWKOPT = 1
      ELSE
*
*        ==== Workspace query call to ZGEHRD ====
*
         CALL ZGEHRD( JW, 1, JW-1, T, LDT, WORK, WORK, -1, INFO )
         LWK1 = INT( WORK( 1 ) )
*
*        ==== Workspace query call to ZUNMHR ====
*
         CALL ZUNMHR( 'R', 'N', JW, JW, 1, JW-1, T, LDT, WORK, V, LDV,
     $                WORK, -1, INFO )
         LWK2 = INT( WORK( 1 ) )
*
*        ==== Workspace query call to ZLAQR4 ====
*
         CALL ZLAQR4( .true., .true., JW, 1, JW, T, LDT, SH, 1, JW, V,
     $                LDV, WORK, -1, INFQR )
         LWK3 = INT( WORK( 1 ) )
*
*        ==== Optimal workspace ====
*
         LWKOPT = MAX( JW+MAX( LWK1, LWK2 ), LWK3 )
      END IF
*
*     ==== Quick return in case of workspace query. ====
*
      IF( LWORK.EQ.-1 ) THEN
         WORK( 1 ) = DCMPLX( LWKOPT, 0 )
         RETURN
      END IF
*
```

```
*     ==== Nothing to do ...
*     ... for an empty active block ... ====
      NS = 0
      ND = 0
      WORK( 1 ) = ONE
      IF( KTOP.GT.KBOT )
     $   RETURN
*     ... nor for an empty deflation window. ====
      IF( NW.LT.1 )
     $   RETURN
*
*     ==== Machine constants ====
*
      SAFMIN = DLAMCH( 'SAFE MINIMUM' )
      SAFMAX = RONE / SAFMIN
      CALL DLABAD( SAFMIN, SAFMAX )
      ULP = DLAMCH( 'PRECISION' )
      SMLNUM = SAFMIN*( DBLE( N ) / ULP )
*
*     ==== Setup deflation window ====
*
      JW = MIN( NW, KBOT-KTOP+1 )
      KWTOP = KBOT - JW + 1
      IF( KWTOP.EQ.KTOP ) THEN
         S = ZERO
      ELSE
         S = H( KWTOP, KWTOP-1 )
      END IF
*
      IF( KBOT.EQ.KWTOP ) THEN
*
*        ==== 1-by-1 deflation window: not much to do ====
*
         SH( KWTOP ) = H( KWTOP, KWTOP )
         NS = 1
         ND = 0
         IF( CABS1( S ).LE.MAX( SMLNUM, ULP*CABS1( H( KWTOP,
     $       KWTOP ) ) ) ) THEN
            NS = 0
            ND = 1
            IF( KWTOP.GT.KTOP )
     $         H( KWTOP, KWTOP-1 ) = ZERO
         END IF
         WORK( 1 ) = ONE
         RETURN
      END IF
*
*     ==== Convert to spike-triangular form.  (In case of a
*     .    rare QR failure, this routine continues to do
*     .    aggressive early deflation using that part of
```

```
*     .    the deflation window that converged using INFQR
*     .    here and there to keep track.) ====
*
      CALL ZLACPY( 'U', JW, JW, H( KWTOP, KWTOP ), LDH, T, LDT )
      CALL ZCOPY( JW-1, H( KWTOP+1, KWTOP ), LDH+1, T( 2, 1 ), LDT+1 )
*
      CALL ZLASET( 'A', JW, JW, ZERO, ONE, V, LDV )
      NMIN = ILAENV( 12, 'ZLAQR3', 'SV', JW, 1, JW, LWORK )
      IF( JW.GT.NMIN ) THEN
         CALL ZLAQR4( .true., .true., JW, 1, JW, T, LDT, SH( KWTOP ), 1,
     $                JW, V, LDV, WORK, LWORK, INFQR )
      ELSE
         CALL ZLAHQR( .true., .true., JW, 1, JW, T, LDT, SH( KWTOP ), 1,
     $                JW, V, LDV, INFQR )
      END IF
*
*     ==== Deflation detection loop ====
*
      NS = JW
      ILST = INFQR + 1
      DO 10 KNT = INFQR + 1, JW
*
*        ==== Small spike tip deflation test ====
*
         FOO = CABS1( T( NS, NS ) )
         IF( FOO.EQ.RZERO )
     $      FOO = CABS1( S )
         IF( CABS1( S )*CABS1( V( 1, NS ) ).LE.MAX( SMLNUM, ULP*FOO ) )
     $        THEN
*
*           ==== One more converged eigenvalue ====
*
            NS = NS - 1
         ELSE
*
*           ==== One undeflatable eigenvalue.  Move it up out of the
*           .    way.   (ZTREXC can not fail in this case.) ====
*
            IFST = NS
            CALL ZTREXC( 'V', JW, T, LDT, V, LDV, IFST, ILST, INFO )
            ILST = ILST + 1
         END IF
   10 CONTINUE
*
*        ==== Return to Hessenberg form ====
*
      IF( NS.EQ.0 )
     $   S = ZERO
*
      IF( NS.LT.JW ) THEN
```

```
*
*        ==== sorting the diagonal of T improves accuracy for
*        .    graded matrices.  ====
*
         DO 30 I = INFQR + 1, NS
            IFST = I
            DO 20 J = I + 1, NS
               IF( CABS1( T( J, J ) ).GT.CABS1( T( IFST, IFST ) ) )
     $            IFST = J
   20       CONTINUE
            ILST = I
            IF( IFST.NE.ILST )
     $         CALL ZTREXC( 'V', JW, T, LDT, V, LDV, IFST, ILST, INFO )
   30    CONTINUE
      END IF
*
*     ==== Restore shift/eigenvalue array from T ====
*
      DO 40 I = INFQR + 1, JW
         SH( KWTOP+I-1 ) = T( I, I )
   40 CONTINUE
*
*
      IF( NS.LT.JW .OR. S.EQ.ZERO ) THEN
         IF( NS.GT.1 .AND. S.NE.ZERO ) THEN
*
*           ==== Reflect spike back into lower triangle ====
*
            CALL ZCOPY( NS, V, LDV, WORK, 1 )
            DO 50 I = 1, NS
               WORK( I ) = DCONJG( WORK( I ) )
   50       CONTINUE
            BETA = WORK( 1 )
            CALL ZLARFG( NS, BETA, WORK( 2 ), 1, TAU )
            WORK( 1 ) = ONE
*
            CALL ZLASET( 'L', JW-2, JW-2, ZERO, ZERO, T( 3, 1 ), LDT )
*
            CALL ZLARF( 'L', NS, JW, WORK, 1, DCONJG( TAU ), T, LDT,
     $                  WORK( JW+1 ) )
            CALL ZLARF( 'R', NS, NS, WORK, 1, TAU, T, LDT,
     $                  WORK( JW+1 ) )
            CALL ZLARF( 'R', JW, NS, WORK, 1, TAU, V, LDV,
     $                  WORK( JW+1 ) )
*
            CALL ZGEHRD( JW, 1, NS, T, LDT, WORK, WORK( JW+1 ),
     $                   LWORK-JW, INFO )
         END IF
*
*        ==== Copy updated reduced window into place ====
```

```
*
         IF( KWTOP.GT.1 )
     $      H( KWTOP, KWTOP-1 ) = S*DCONJG( V( 1, 1 ) )
         CALL ZLACPY( 'U', JW, JW, T, LDT, H( KWTOP, KWTOP ), LDH )
         CALL ZCOPY( JW-1, T( 2, 1 ), LDT+1, H( KWTOP+1, KWTOP ),
     $               LDH+1 )
*
*        ==== Accumulate orthogonal matrix in order update
*        .    H and Z, if requested.  ====
*
         IF( NS.GT.1 .AND. S.NE.ZERO )
     $      CALL ZUNMHR( 'R', 'N', JW, NS, 1, NS, T, LDT, WORK, V, LDV,
     $                   WORK( JW+1 ), LWORK-JW, INFO )
*
*        ==== Update vertical slab in H ====
*
         IF( WANTT ) THEN
            LTOP = 1
         ELSE
            LTOP = KTOP
         END IF
         DO 60 KROW = LTOP, KWTOP - 1, NV
            KLN = MIN( NV, KWTOP-KROW )
            CALL ZGEMM( 'N', 'N', KLN, JW, JW, ONE, H( KROW, KWTOP ),
     $                  LDH, V, LDV, ZERO, WV, LDWV )
            CALL ZLACPY( 'A', KLN, JW, WV, LDWV, H( KROW, KWTOP ), LDH )
   60    CONTINUE
*
*        ==== Update horizontal slab in H ====
*
         IF( WANTT ) THEN
            DO 70 KCOL = KBOT + 1, N, NH
               KLN = MIN( NH, N-KCOL+1 )
               CALL ZGEMM( 'C', 'N', JW, KLN, JW, ONE, V, LDV,
     $                     H( KWTOP, KCOL ), LDH, ZERO, T, LDT )
               CALL ZLACPY( 'A', JW, KLN, T, LDT, H( KWTOP, KCOL ),
     $                      LDH )
   70       CONTINUE
         END IF
*
*        ==== Update vertical slab in Z ====
*
         IF( WANTZ ) THEN
            DO 80 KROW = ILOZ, IHIZ, NV
               KLN = MIN( NV, IHIZ-KROW+1 )
               CALL ZGEMM( 'N', 'N', KLN, JW, JW, ONE, Z( KROW, KWTOP ),
     $                     LDZ, V, LDV, ZERO, WV, LDWV )
               CALL ZLACPY( 'A', KLN, JW, WV, LDWV, Z( KROW, KWTOP ),
     $                      LDZ )
   80       CONTINUE
```

```
         END IF
      END IF
*
*     ==== Return the number of deflations ... ====
*
      ND = JW - NS
*
*     ==== ... and the number of shifts. (Subtracting
*     .    INFQR from the spike length takes care
*     .    of the case of a rare QR failure while
*     .    calculating eigenvalues of the deflation
*     .    window.)  ====
*
      NS = NS - INFQR
*
*      ==== Return optimal workspace. ====
*
      WORK( 1 ) = DCMPLX( LWKOPT, 0 )
*
*     ==== End of ZLAQR3 ====
*
      END
```

———-

90.226.4 lisp code

— zlaqr3.lisp —

```
(let* ((zero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (one (f2cl-lib:cmplx 1.0d0 0.0d0))
       (rzero 0.0d0)
       (rone 1.0d0))
  (declare (type (f2cl-lib:complex16) zero)
           (type (f2cl-lib:complex16) one)
           (type (double-float 0.0d0 0.0d0) rzero)
           (type (double-float 1.0d0 1.0d0) rone)
           (ignorable zero one rzero rone))
  (defun zlaqr3
         (wantt wantz n ktop kbot nw h ldh iloz ihiz z ldz ns nd sh v ldv nh t$
          ldt nv wv ldwv work lwork)
    (declare (type (array f2cl-lib:complex16 (*)) work wv t$ v sh z h)
             (type (f2cl-lib:integer4) lwork ldwv nv ldt nh ldv nd ns ldz ihiz
              iloz ldh nw kbot ktop n)
             (type f2cl-lib:logical wantz wantt))
    (f2cl-lib:with-multi-array-data
        ((h f2cl-lib:complex16 h-%data% h-%offset%)
```

```
    (z f2cl-lib:complex16 z-%data% z-%offset%)
    (sh f2cl-lib:complex16 sh-%data% sh-%offset%)
    (v f2cl-lib:complex16 v-%data% v-%offset%)
    (t$ f2cl-lib:complex16 t$-%data% t$-%offset%)
    (wv f2cl-lib:complex16 wv-%data% wv-%offset%)
    (work f2cl-lib:complex16 work-%data% work-%offset%))
  (labels ((cabs1 (cdum)
             (+ (abs (f2cl-lib:dble cdum)) (abs (f2cl-lib:dimag cdum)))))
    (declare
     (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
      cabs1))
    (prog ((i 0) (ifst 0) (ilst 0) (info 0) (infqr 0) (j 0) (jw 0) (kcol 0)
           (kln 0) (knt 0) (krow 0) (kwtop 0) (ltop 0) (lwk1 0) (lwk2 0)
           (lwk3 0) (lwkopt 0) (nmin 0) (foo 0.0d0) (safmax 0.0d0)
           (safmin 0.0d0) (smlnum 0.0d0) (ulp 0.0d0) (beta #C(0.0d0 0.0d0))
           (cdum #C(0.0d0 0.0d0)) (s #C(0.0d0 0.0d0))
           (tau #C(0.0d0 0.0d0)))
     (declare
      (type (f2cl-lib:integer4) i ifst ilst info infqr j jw kcol kln knt
       krow kwtop ltop lwk1 lwk2 lwk3 lwkopt nmin)
      (type (double-float) foo safmax safmin smlnum ulp)
      (type (f2cl-lib:complex16) beta cdum s tau))
     (setf jw
             (min (the f2cl-lib:integer4 nw)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add (f2cl-lib:int-sub kbot ktop)
                                         1))))
     (cond ((<= jw 2) (setf lwkopt 1))
           (t
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
                (zgehrd jw 1 (f2cl-lib:int-sub jw 1) t$ ldt work work -1
                 info)
              (declare (ignore var-1 var-2 var-3 var-5 var-6 var-7))
              (setf jw var-0)
              (setf ldt var-4)
              (setf info var-8))
            (setf lwk1
                    (f2cl-lib:int
                     (f2cl-lib:fref work-%data% (1) ((1 *))
                                    work-%offset%)))
            (multiple-value-bind
                (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
                 var-9 var-10 var-11 var-12 var-13)
                (zunmhr "R" "N" jw jw 1 (f2cl-lib:int-sub jw 1) t$ ldt
                 work v ldv work -1 info)
              (declare
               (ignore var-0 var-1 var-4 var-5 var-6 var-8 var-9 var-11
                var-12))
              (when var-2 (setf jw var-2))
```

```
          (when var-3 (setf jw var-3))
          (when var-7 (setf ldt var-7))
          (when var-10 (setf ldv var-10))
          (when var-13 (setf info var-13)))
        (setf lwk2
                (f2cl-lib:int
                 (f2cl-lib:fref work-%data% (1) ((1 *))
                                work-%offset%)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12 var-13 var-14)
            (zlaqr4 f2cl-lib:%true% f2cl-lib:%true% jw 1 jw t$ ldt sh
             1 jw v ldv work -1 infqr)
          (declare
           (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10 var-12
            var-13))
          (when var-2 (setf jw var-2))
          (when var-4 (setf jw var-4))
          (when var-6 (setf ldt var-6))
          (when var-9 (setf jw var-9))
          (when var-11 (setf ldv var-11))
          (when var-14 (setf infqr var-14)))
        (setf lwk3
                (f2cl-lib:int
                 (f2cl-lib:fref work-%data% (1) ((1 *))
                                work-%offset%)))
        (setf lwkopt
                (max
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-add jw
                                        (max
                                         (the f2cl-lib:integer4 lwk1)
                                         (the f2cl-lib:integer4
                                              lwk2))))
                 (the f2cl-lib:integer4 lwk3)))))
(cond
 ((= lwork (f2cl-lib:int-sub 1))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:dcmplx lwkopt 0))
  (go end_label)))
(setf ns 0)
(setf nd 0)
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) one)
(if (> ktop kbot)
    (go end_label))
(if (< nw 1)
    (go end_label))
(setf safmin (dlamch "SAFE MINIMUM"))
(setf safmax (/ rone safmin))
(multiple-value-bind (var-0 var-1)
```

```
    (dlabad safmin safmax)
  (declare (ignore))
  (when var-0 (setf safmin var-0))
  (when var-1 (setf safmax var-1)))
(setf ulp (dlamch "PRECISION"))
(setf smlnum (* safmin (/ (f2cl-lib:dble n) ulp)))
(setf jw
        (min (the f2cl-lib:integer4 nw)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add (f2cl-lib:int-sub kbot ktop)
                                    1))))
(setf kwtop (f2cl-lib:int-add (f2cl-lib:int-sub kbot jw) 1))
(cond ((= kwtop ktop) (setf s zero))
      (t
       (setf s
               (f2cl-lib:fref h-%data%
                              (kwtop (f2cl-lib:int-sub kwtop 1))
                              ((1 ldh) (1 *)) h-%offset%))))
(cond
 ((= kbot kwtop)
  (setf (f2cl-lib:fref sh-%data% (kwtop) ((1 *)) sh-%offset%)
          (f2cl-lib:fref h-%data% (kwtop kwtop) ((1 ldh) (1 *))
                         h-%offset%))
  (setf ns 1) (setf nd 0)
  (cond
   ((<= (cabs1 s)
        (max smlnum
             (* ulp
                (cabs1
                 (f2cl-lib:fref h (kwtop kwtop) ((1 ldh) (1 *)))))))
    (setf ns 0) (setf nd 1)
    (if (> kwtop ktop)
        (setf (f2cl-lib:fref h-%data%
                             (kwtop (f2cl-lib:int-sub kwtop 1))
                             ((1 ldh) (1 *)) h-%offset%)
                zero))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) one)
  (go end_label)))
(zlacpy "U" jw jw
 (f2cl-lib:array-slice h-%data% f2cl-lib:complex16 (kwtop kwtop)
                       ((1 ldh) (1 *)) h-%offset%)
 ldh t$ ldt)
(zcopy (f2cl-lib:int-sub jw 1)
 (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                       ((+ kwtop 1) kwtop) ((1 ldh) (1 *))
                       h-%offset%)
 (f2cl-lib:int-add ldh 1)
 (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (2 1)
                       ((1 ldt) (1 *)) t$-%offset%)
 (f2cl-lib:int-add ldt 1))
```

```
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "A" jw jw zero one v ldv)
  (declare (ignore var-0 var-5))
  (when var-1 (setf jw var-1))
  (when var-2 (setf jw var-2))
  (when var-3 (setf zero var-3))
  (when var-4 (setf one var-4))
  (when var-6 (setf ldv var-6)))
(setf nmin
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 12 "ZLAQR3" "SV" jw 1 jw lwork)
          (declare (ignore var-0 var-1 var-2 var-4))
          (setf jw var-3)
          (setf jw var-5)
          (setf lwork var-6)
          ret-val))
(cond
 ((> jw nmin)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12 var-13 var-14)
      (zlaqr4 f2cl-lib:%true% f2cl-lib:%true% jw 1 jw t$ ldt
       (f2cl-lib:array-slice sh-%data% f2cl-lib:complex16 (kwtop)
                             ((1 *)) sh-%offset%)
       1 jw v ldv work lwork infqr)
    (declare
     (ignore var-0 var-1 var-3 var-5 var-7 var-8 var-10 var-12))
    (when var-2 (setf jw var-2))
    (when var-4 (setf jw var-4))
    (when var-6 (setf ldt var-6))
    (when var-9 (setf jw var-9))
    (when var-11 (setf ldv var-11))
    (when var-13 (setf lwork var-13))
    (when var-14 (setf infqr var-14))))
 (t
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11 var-12)
      (zlahqr f2cl-lib:%true% f2cl-lib:%true% jw 1 jw t$ ldt
       (f2cl-lib:array-slice sh-%data% f2cl-lib:complex16 (kwtop)
                             ((1 *)) sh-%offset%)
       1 jw v ldv infqr)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-8 var-9
      var-10 var-11))
    (setf ldt var-6)
    (setf infqr var-12))))
(setf ns jw)
(setf ilst (f2cl-lib:int-add infqr 1))
```

```
(f2cl-lib:fdo (knt (f2cl-lib:int-add infqr 1)
               (f2cl-lib:int-add knt 1))
              ((> knt jw) nil)
  (tagbody
    (setf foo
            (cabs1
             (f2cl-lib:fref t$-%data% (ns ns) ((1 ldt) (1 *))
                            t$-%offset%)))
    (if (= foo rzero)
        (setf foo (cabs1 s)))
    (cond
     ((<=
       (* (cabs1 s) (cabs1 (f2cl-lib:fref v (1 ns) ((1 ldv) (1 *)))))
       (max smlnum (* ulp foo)))
      (setf ns (f2cl-lib:int-sub ns 1)))
     (t (setf ifst ns)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (ztrexc "V" jw t$ ldt v ldv ifst ilst info)
        (declare (ignore var-0 var-2 var-4))
        (when var-1 (setf jw var-1))
        (when var-3 (setf ldt var-3))
        (when var-5 (setf ldv var-5))
        (when var-6 (setf ifst var-6))
        (when var-7 (setf ilst var-7))
        (when var-8 (setf info var-8)))
      (setf ilst (f2cl-lib:int-add ilst 1))))
   label10))
(if (= ns 0)
    (setf s zero))
(cond
 ((< ns jw)
  (f2cl-lib:fdo (i (f2cl-lib:int-add infqr 1) (f2cl-lib:int-add i 1))
                ((> i ns) nil)
    (tagbody
      (setf ifst i)
      (f2cl-lib:fdo (j (f2cl-lib:int-add i 1) (f2cl-lib:int-add j 1))
                    ((> j ns) nil)
        (tagbody
          (if (>
               (cabs1
                (f2cl-lib:fref t$-%data% (j j) ((1 ldt) (1 *))
                               t$-%offset%))
               (cabs1
                (f2cl-lib:fref t$-%data% (ifst ifst) ((1 ldt) (1 *))
                               t$-%offset%)))
              (setf ifst j))
         label20))
      (setf ilst i)
      (if (/= ifst ilst)
```

```
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (ztrexc "V" jw t$ ldt v ldv ifst ilst info)
            (declare (ignore var-0 var-2 var-4))
            (when var-1 (setf jw var-1))
            (when var-3 (setf ldt var-3))
            (when var-5 (setf ldv var-5))
            (when var-6 (setf ifst var-6))
            (when var-7 (setf ilst var-7))
            (when var-8 (setf info var-8))))
     label30))))
(f2cl-lib:fdo (i (f2cl-lib:int-add infqr 1) (f2cl-lib:int-add i 1))
              ((> i jw) nil)
  (tagbody
    (setf (f2cl-lib:fref sh-%data%
                         ((f2cl-lib:int-sub
                           (f2cl-lib:int-add kwtop i) 1))
                         ((1 *)) sh-%offset%)
            (f2cl-lib:fref t$-%data% (i i) ((1 ldt) (1 *))
                           t$-%offset%))
   label40))
(cond
 ((or (< ns jw) (= s zero))
  (cond
   ((and (> ns 1) (/= s zero))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zcopy ns v ldv work 1)
      (declare (ignore var-1 var-3 var-4))
      (when var-0 (setf ns var-0))
      (when var-2 (setf ldv var-2)))
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i ns) nil)
      (tagbody
        (setf (f2cl-lib:fref work-%data% (i) ((1 *)) work-%offset%)
                (coerce
                 (f2cl-lib:dconjg
                  (f2cl-lib:fref work-%data% (i) ((1 *))
                                 work-%offset%))
                 'f2cl-lib:complex16))
       label50))
    (setf beta (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zlarfg ns beta
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (2)
                               ((1 *)) work-%offset%)
         1 tau)
      (declare (ignore var-2 var-3))
      (when var-0 (setf ns var-0))
      (when var-1 (setf beta var-1))
      (when var-4 (setf tau var-4)))
```

```
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) one)
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "L" (f2cl-lib:int-sub jw 2) (f2cl-lib:int-sub jw 2)
     zero zero
     (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (3 1)
                           ((1 ldt) (1 *)) t$-%offset%)
     ldt)
  (declare (ignore var-0 var-1 var-2 var-5))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldt var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zlarf "L" ns jw work 1 (f2cl-lib:dconjg tau) t$ ldt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ jw 1)) ((1 *)) work-%offset%))
  (declare (ignore var-0 var-3 var-4 var-5 var-6 var-8))
  (when var-1 (setf ns var-1))
  (when var-2 (setf jw var-2))
  (when var-7 (setf ldt var-7)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zlarf "R" ns ns work 1 tau t$ ldt
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ jw 1)) ((1 *)) work-%offset%))
  (declare (ignore var-0 var-3 var-4 var-6 var-8))
  (when var-1 (setf ns var-1))
  (when var-2 (setf ns var-2))
  (when var-5 (setf tau var-5))
  (when var-7 (setf ldt var-7)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zlarf "R" jw ns work 1 tau v ldv
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ jw 1)) ((1 *)) work-%offset%))
  (declare (ignore var-0 var-3 var-4 var-6 var-8))
  (when var-1 (setf jw var-1))
  (when var-2 (setf ns var-2))
  (when var-5 (setf tau var-5))
  (when var-7 (setf ldv var-7)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
    (zgehrd jw 1 ns t$ ldt work
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                           ((+ jw 1)) ((1 *)) work-%offset%)
     (f2cl-lib:int-sub lwork jw) info)
  (declare (ignore var-1 var-3 var-5 var-6 var-7))
  (setf jw var-0)
  (setf ns var-2)
  (setf ldt var-4)
```

```
    (setf info var-8))))
(if (> kwtop 1)
    (setf (f2cl-lib:fref h-%data%
                         (kwtop (f2cl-lib:int-sub kwtop 1))
                         ((1 ldh) (1 *)) h-%offset%)
            (* s
               (f2cl-lib:dconjg
                (f2cl-lib:fref v-%data% (1 1) ((1 ldv) (1 *))
                               v-%offset%)))))
(zlacpy "U" jw jw t$ ldt
 (f2cl-lib:array-slice h-%data% f2cl-lib:complex16 (kwtop kwtop)
                       ((1 ldh) (1 *)) h-%offset%)
 ldh)
(zcopy (f2cl-lib:int-sub jw 1)
 (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (2 1)
                       ((1 ldt) (1 *)) t$-%offset%)
 (f2cl-lib:int-add ldt 1)
 (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                       ((+ kwtop 1) kwtop) ((1 ldh) (1 *))
                       h-%offset%)
 (f2cl-lib:int-add ldh 1))
(if (and (> ns 1) (/= s zero))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12 var-13)
        (zunmhr "R" "N" jw ns 1 ns t$ ldt work v ldv
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               ((+ jw 1)) ((1 *)) work-%offset%)
         (f2cl-lib:int-sub lwork jw) info)
      (declare
       (ignore var-0 var-1 var-4 var-6 var-8 var-9 var-11 var-12))
      (when var-2 (setf jw var-2))
      (when var-3 (setf ns var-3))
      (when var-5 (setf ns var-5))
      (when var-7 (setf ldt var-7))
      (when var-10 (setf ldv var-10))
      (when var-13 (setf info var-13))))
(cond (wantt (setf ltop 1)) (t (setf ltop ktop)))
(f2cl-lib:fdo (krow ltop (f2cl-lib:int-add krow nv))
              ((> krow
                  (f2cl-lib:int-add kwtop (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf kln
            (min (the f2cl-lib:integer4 nv)
                 (the f2cl-lib:integer4
                      (f2cl-lib:int-sub kwtop krow))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
```

```
          (zgemm "N" "N" kln jw jw one
           (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                                 (krow kwtop) ((1 ldh) (1 *))
                                 h-%offset%)
           ldh v ldv zero wv ldwv)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf kln var-2))
        (when var-3 (setf jw var-3))
        (when var-4 (setf jw var-4))
        (when var-5 (setf one var-5))
        (when var-7 (setf ldh var-7))
        (when var-9 (setf ldv var-9))
        (when var-10 (setf zero var-10))
        (when var-12 (setf ldwv var-12)))
      (zlacpy "A" kln jw wv ldwv
       (f2cl-lib:array-slice h-%data% f2cl-lib:complex16 (krow kwtop)
                             ((1 ldh) (1 *)) h-%offset%)
       ldh)
     label60))
  (cond
   (wantt
    (f2cl-lib:fdo (kcol (f2cl-lib:int-add kbot 1)
                   (f2cl-lib:int-add kcol nh))
                  ((> kcol n) nil)
      (tagbody
        (setf kln
                (min (the f2cl-lib:integer4 nh)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add (f2cl-lib:int-sub n kcol)
                                            1))))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12)
            (zgemm "C" "N" jw kln jw one v ldv
             (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                                   (kwtop kcol) ((1 ldh) (1 *))
                                   h-%offset%)
             ldh zero t$ ldt)
          (declare (ignore var-0 var-1 var-6 var-8 var-11))
          (when var-2 (setf jw var-2))
          (when var-3 (setf kln var-3))
          (when var-4 (setf jw var-4))
          (when var-5 (setf one var-5))
          (when var-7 (setf ldv var-7))
          (when var-9 (setf ldh var-9))
          (when var-10 (setf zero var-10))
          (when var-12 (setf ldt var-12)))
        (zlacpy "A" jw kln t$ ldt
         (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                               (kwtop kcol) ((1 ldh) (1 *))
```

```
                                h-%offset%)
          ldh)
        label70))))
    (cond
     (wantz
      (f2cl-lib:fdo (krow iloz (f2cl-lib:int-add krow nv))
                    ((> krow ihiz) nil)
        (tagbody
          (setf kln
                  (min (the f2cl-lib:integer4 nv)
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add
                             (f2cl-lib:int-sub ihiz krow) 1))))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
               var-9 var-10 var-11 var-12)
              (zgemm "N" "N" kln jw jw one
               (f2cl-lib:array-slice z-%data% f2cl-lib:complex16
                                     (krow kwtop) ((1 ldz) (1 *))
                                     z-%offset%)
               ldz v ldv zero wv ldwv)
            (declare (ignore var-0 var-1 var-6 var-8 var-11))
            (when var-2 (setf kln var-2))
            (when var-3 (setf jw var-3))
            (when var-4 (setf jw var-4))
            (when var-5 (setf one var-5))
            (when var-7 (setf ldz var-7))
            (when var-9 (setf ldv var-9))
            (when var-10 (setf zero var-10))
            (when var-12 (setf ldwv var-12)))
          (zlacpy "A" kln jw wv ldwv
           (f2cl-lib:array-slice z-%data% f2cl-lib:complex16
                                 (krow kwtop) ((1 ldz) (1 *))
                                 z-%offset%)
           ldz)
        label80))))))
  (setf nd (f2cl-lib:int-sub jw ns))
  (setf ns (f2cl-lib:int-sub ns infqr))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:dcmplx lwkopt 0))
end_label
  (return
   (values nil nil nil nil nil nil nil ldh nil nil nil ldz ns nd nil
           nil ldv nil nil ldt nil nil ldwv nil lwork)))))))
```

———-

90.227 zlaqr4

See Braman [Bram02a, Bram02b]

90.227.1 Axiom unit tests

— zlaqr4.input —

```
)set break resume
)sys rm -f zlaqr4.output
)spool zlaqr4.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.227.2 Axiom help page

— zlaqr4.help —

```
======================================================================
zlaqr4 examples
======================================================================

======================================================================
Man Page Details
======================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZLAQR4( WANTT, WANTZ, N, ILO, IHI, H, LDH, W, ILOZ,
                          IHIZ, Z, LDZ, WORK, LWORK, INFO )

       .. Scalar Arguments ..
       INTEGER            IHI, IHIZ, ILO, ILOZ, INFO, LDH, LDZ, LWORK, N
       LOGICAL            WANTT, WANTZ
       ..
```

```
      .. Array Arguments ..
      COMPLEX*16         H( LDH, * ), W( * ), WORK( * ), Z( LDZ, * )
      ..


Purpose:
=============

   ZLAQR4 implements one level of recursion for ZLAQR0.
   It is a complete implementation of the small bulge multi-shift
   QR algorithm.  It may be called by ZLAQR0 and, for large enough
   deflation window size, it may be called by ZLAQR3.  This
   subroutine is identical to ZLAQR0 except that it calls ZLAQR2
   instead of ZLAQR3.

   ZLAQR4 computes the eigenvalues of a Hessenberg matrix H
   and, optionally, the matrices T and Z from the Schur decomposition
   H = Z T Z**H, where T is an upper triangular matrix (the
   Schur form), and Z is the unitary matrix of Schur vectors.

   Optionally Z may be postmultiplied into an input unitary
   matrix Q so that this routine can give the Schur factorization
   of a matrix A which has been reduced to the Hessenberg form H
   by the unitary matrix Q:  A = Q*H*Q**H = (QZ)*H*(QZ)**H.

Arguments:
==========

 [in] WANTT

         WANTT is LOGICAL
         = .TRUE. : the full Schur form T is required;
         = .FALSE.: only eigenvalues are required.

 [in] WANTZ

         WANTZ is LOGICAL
         = .TRUE. : the matrix of Schur vectors Z is required;
         = .FALSE.: Schur vectors are not required.

 [in] N

         N is INTEGER
          The order of the matrix H.  N .GE. 0.

 [in] ILO

         ILO is INTEGER

 [in] IHI
```

```
        IHI is INTEGER
         It is assumed that H is already upper triangular in rows
         and columns 1:ILO-1 and IHI+1:N and, if ILO.GT.1,
         H(ILO,ILO-1) is zero. ILO and IHI are normally set by a
         previous call to ZGEBAL, and then passed to ZGEHRD when the
         matrix output by ZGEBAL is reduced to Hessenberg form.
         Otherwise, ILO and IHI should be set to 1 and N,
         respectively.  If N.GT.0, then 1.LE.ILO.LE.IHI.LE.N.
         If N = 0, then ILO = 1 and IHI = 0.

[in,out] H

        H is COMPLEX*16 array, dimension (LDH,N)
         On entry, the upper Hessenberg matrix H.
         On exit, if INFO = 0 and WANTT is .TRUE., then H
         contains the upper triangular matrix T from the Schur
         decomposition (the Schur form). If INFO = 0 and WANT is
         .FALSE., then the contents of H are unspecified on exit.
         (The output value of H when INFO.GT.0 is given under the
         description of INFO below.)

         This subroutine may explicitly set H(i,j) = 0 for i.GT.j and
         j = 1, 2, ... ILO-1 or j = IHI+1, IHI+2, ... N.

[in] LDH

        LDH is INTEGER
         The leading dimension of the array H. LDH .GE. max(1,N).

[out] W

        W is COMPLEX*16 array, dimension (N)
         The computed eigenvalues of H(ILO:IHI,ILO:IHI) are stored
         in W(ILO:IHI). If WANTT is .TRUE., then the eigenvalues are
         stored in the same order as on the diagonal of the Schur
         form returned in H, with W(i) = H(i,i).

[in] ILOZ

        ILOZ is INTEGER

[in] IHIZ

        IHIZ is INTEGER
         Specify the rows of Z to which transformations must be
         applied if WANTZ is .TRUE..
         1 .LE. ILOZ .LE. ILO; IHI .LE. IHIZ .LE. N.

[in,out] Z
```

```
        Z is COMPLEX*16 array, dimension (LDZ,IHI)
         If WANTZ is .FALSE., then Z is not referenced.
         If WANTZ is .TRUE., then Z(ILO:IHI,ILOZ:IHIZ) is
         replaced by Z(ILO:IHI,ILOZ:IHIZ)*U where U is the
         orthogonal Schur factor of H(ILO:IHI,ILO:IHI).
         (The output value of Z when INFO.GT.0 is given under
         the description of INFO below.)

[in] LDZ

        LDZ is INTEGER
         The leading dimension of the array Z.  if WANTZ is .TRUE.
         then LDZ.GE.MAX(1,IHIZ).  Otherwize, LDZ.GE.1.

[out] WORK

        WORK is COMPLEX*16 array, dimension LWORK
         On exit, if LWORK = -1, WORK(1) returns an estimate of
         the optimal value for LWORK.

[in] LWORK

        LWORK is INTEGER
         The dimension of the array WORK.  LWORK .GE. max(1,N)
         is sufficient, but LWORK typically as large as 6*N may
         be required for optimal performance.  A workspace query
         to determine the optimal workspace size is recommended.

         If LWORK = -1, then ZLAQR4 does a workspace query.
         In this case, ZLAQR4 checks the input parameters and
         estimates the optimal workspace size for the given
         values of N, ILO and IHI.  The estimate is returned
         in WORK(1).  No error message related to LWORK is
         issued by XERBLA.  Neither H nor Z are accessed.

[out] INFO

        INFO is INTEGER
           =  0:  successful exit
         .GT. 0:  if INFO = i, ZLAQR4 failed to compute all of
              the eigenvalues.  Elements 1:ilo-1 and i+1:n of WR
              and WI contain those eigenvalues which have been
              successfully computed.  (Failures are rare.)

              If INFO .GT. 0 and WANT is .FALSE., then on exit,
              the remaining unconverged eigenvalues are the eigen-
              values of the upper Hessenberg matrix rows and
              columns ILO through INFO of the final, output
              value of H.
```

```
             If INFO .GT. 0 and WANTT is .TRUE., then on exit

        (*)  (initial value of H)*U  = U*(final value of H)

             where U is a unitary matrix.  The final
             value of  H is upper Hessenberg and triangular in
             rows and columns INFO+1 through IHI.

             If INFO .GT. 0 and WANTZ is .TRUE., then on exit

               (final value of Z(ILO:IHI,ILOZ:IHIZ)
                =  (initial value of Z(ILO:IHI,ILOZ:IHIZ)*U

             where U is the unitary matrix in (*) (regard-
             less of the value of WANTT.)

             If INFO .GT. 0 and WANTZ is .FALSE., then Z is not
             accessed.
```

90.227.3 fortran code

— zlaqr4.f —

```
*  =====================================================================
      SUBROUTINE ZLAQR4( WANTT, WANTZ, N, ILO, IHI, H, LDH, W, ILOZ,
     $                   IHIZ, Z, LDZ, WORK, LWORK, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            IHI, IHIZ, ILO, ILOZ, INFO, LDH, LDZ, LWORK, N
      LOGICAL            WANTT, WANTZ
*     ..
*     .. Array Arguments ..
      COMPLEX*16         H( LDH, * ), W( * ), WORK( * ), Z( LDZ, * )
*     ..
*
*  ================================================================
*
*     .. Parameters ..
*
```

```
*     ==== Matrices of order NTINY or smaller must be processed by
*     .    ZLAHQR because of insufficient subdiagonal scratch space.
*     .    (This is a hard limit.) ====
      INTEGER            NTINY
      PARAMETER          ( NTINY = 11 )
*
*     ==== Exceptional deflation windows:  try to cure rare
*     .    slow convergence by varying the size of the
*     .    deflation window after KEXNW iterations. ====
      INTEGER            KEXNW
      PARAMETER          ( KEXNW = 5 )
*
*     ==== Exceptional shifts: try to cure rare slow convergence
*     .    with ad-hoc exceptional shifts every KEXSH iterations.
*     .    ====
      INTEGER            KEXSH
      PARAMETER          ( KEXSH = 6 )
*
*     ==== The constant WILK1 is used to form the exceptional
*     .    shifts. ====
      DOUBLE PRECISION   WILK1
      PARAMETER          ( WILK1 = 0.75d0 )
      COMPLEX*16         ZERO, ONE
      PARAMETER          ( ZERO = ( 0.0d0, 0.0d0 ),
     $                   ONE = ( 1.0d0, 0.0d0 ) )
      DOUBLE PRECISION   TWO
      PARAMETER          ( TWO = 2.0d0 )
*     ..
*     .. Local Scalars ..
      COMPLEX*16         AA, BB, CC, CDUM, DD, DET, RTDISC, SWAP, TR2
      DOUBLE PRECISION   S
      INTEGER            I, INF, IT, ITMAX, K, KACC22, KBOT, KDU, KS,
     $                   KT, KTOP, KU, KV, KWH, KWTOP, KWV, LD, LS,
     $                   LWKOPT, NDEC, NDFL, NH, NHO, NIBBLE, NMIN, NS,
     $                   NSMAX, NSR, NVE, NW, NWMAX, NWR, NWUPBD
      LOGICAL            SORTED
      CHARACTER          JBCMPZ*2
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Local Arrays ..
      COMPLEX*16         ZDUM( 1, 1 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZLACPY, ZLAHQR, ZLAQR2, ZLAQR5
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DCMPLX, DIMAG, INT, MAX, MIN, MOD,
```

```
     $                   SQRT
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( CDUM ) = ABS( DBLE( CDUM ) ) + ABS( DIMAG( CDUM ) )
*     ..
*     .. Executable Statements ..
      INFO = 0
*
*     ==== Quick return for N = 0: nothing to do. ====
*
      IF( N.EQ.0 ) THEN
         WORK( 1 ) = ONE
         RETURN
      END IF
*
      IF( N.LE.NTINY ) THEN
*
*        ==== Tiny matrices must use ZLAHQR. ====
*
         LWKOPT = 1
         IF( LWORK.NE.-1 )
     $      CALL ZLAHQR( WANTT, WANTZ, N, ILO, IHI, H, LDH, W, ILOZ,
     $                   IHIZ, Z, LDZ, INFO )
      ELSE
*
*        ==== Use small bulge multi-shift QR with aggressive early
*        .    deflation on larger-than-tiny matrices. ====
*
*        ==== Hope for the best. ====
*
         INFO = 0
*
*        ==== Set up job flags for ILAENV. ====
*
         IF( WANTT ) THEN
            JBCMPZ( 1: 1 ) = 'S'
         ELSE
            JBCMPZ( 1: 1 ) = 'E'
         END IF
         IF( WANTZ ) THEN
            JBCMPZ( 2: 2 ) = 'V'
         ELSE
            JBCMPZ( 2: 2 ) = 'N'
         END IF
*
*        ==== NWR = recommended deflation window size.  At this
*        .    point,  N .GT. NTINY = 11, so there is enough
```

```
*        .    subdiagonal workspace for NWR.GE.2 as required.
*        .    (In fact, there is enough subdiagonal space for
*        .    NWR.GE.3.) ====
*
         NWR = ILAENV( 13, 'ZLAQR4', JBCMPZ, N, ILO, IHI, LWORK )
         NWR = MAX( 2, NWR )
         NWR = MIN( IHI-ILO+1, ( N-1 ) / 3, NWR )
*
*        ==== NSR = recommended number of simultaneous shifts.
*        .    At this point N .GT. NTINY = 11, so there is at
*        .    enough subdiagonal workspace for NSR to be even
*        .    and greater than or equal to two as required. ====
*
         NSR = ILAENV( 15, 'ZLAQR4', JBCMPZ, N, ILO, IHI, LWORK )
         NSR = MIN( NSR, ( N+6 ) / 9, IHI-ILO )
         NSR = MAX( 2, NSR-MOD( NSR, 2 ) )
*
*        ==== Estimate optimal workspace ====
*
*        ==== Workspace query call to ZLAQR2 ====
*
         CALL ZLAQR2( WANTT, WANTZ, N, ILO, IHI, NWR+1, H, LDH, ILOZ,
     $               IHIZ, Z, LDZ, LS, LD, W, H, LDH, N, H, LDH, N, H,
     $               LDH, WORK, -1 )
*
*        ==== Optimal workspace = MAX(ZLAQR5, ZLAQR2) ====
*
         LWKOPT = MAX( 3*NSR / 2, INT( WORK( 1 ) ) )
*
*        ==== Quick return in case of workspace query. ====
*
         IF( LWORK.EQ.-1 ) THEN
            WORK( 1 ) = DCMPLX( LWKOPT, 0 )
            RETURN
         END IF
*
*        ==== ZLAHQR/ZLAQR0 crossover point ====
*
         NMIN = ILAENV( 12, 'ZLAQR4', JBCMPZ, N, ILO, IHI, LWORK )
         NMIN = MAX( NTINY, NMIN )
*
*        ==== Nibble crossover point ====
*
         NIBBLE = ILAENV( 14, 'ZLAQR4', JBCMPZ, N, ILO, IHI, LWORK )
         NIBBLE = MAX( 0, NIBBLE )
*
*        ==== Accumulate reflections during ttswp?  Use block
*        .    2-by-2 structure during matrix-matrix multiply? ====
*
         KACC22 = ILAENV( 16, 'ZLAQR4', JBCMPZ, N, ILO, IHI, LWORK )
```

```
         KACC22 = MAX( 0, KACC22 )
         KACC22 = MIN( 2, KACC22 )
*
*        ==== NWMAX = the largest possible deflation window for
*        .    which there is sufficient workspace. ====
*
         NWMAX = MIN( ( N-1 ) / 3, LWORK / 2 )
         NW = NWMAX
*
*        ==== NSMAX = the Largest number of simultaneous shifts
*        .    for which there is sufficient workspace. ====
*
         NSMAX = MIN( ( N+6 ) / 9, 2*LWORK / 3 )
         NSMAX = NSMAX - MOD( NSMAX, 2 )
*
*        ==== NDFL: an iteration count restarted at deflation. ====
*
         NDFL = 1
*
*        ==== ITMAX = iteration limit ====
*
         ITMAX = MAX( 30, 2*KEXSH )*MAX( 10, ( IHI-ILO+1 ) )
*
*        ==== Last row and column in the active block ====
*
         KBOT = IHI
*
*        ==== Main Loop ====
*
         DO 70 IT = 1, ITMAX
*
*           ==== Done when KBOT falls below ILO ====
*
            IF( KBOT.LT.ILO )
     $         GO TO 80
*
*           ==== Locate active block ====
*
            DO 10 K = KBOT, ILO + 1, -1
               IF( H( K, K-1 ).EQ.ZERO )
     $            GO TO 20
   10       CONTINUE
            K = ILO
   20       CONTINUE
            KTOP = K
*
*           ==== Select deflation window size:
*           .    Typical Case:
*           .      If possible and advisable, nibble the entire
*           .      active block.  If not, use size MIN(NWR,NWMAX)
```

```
*           .      or MIN(NWR+1,NWMAX) depending upon which has
*           .      the smaller corresponding subdiagonal entry
*           .      (a heuristic).
*           .
*           .    Exceptional Case:
*           .      If there have been no deflations in KEXNW or
*           .      more iterations, then vary the deflation window
*           .      size.   At first, because, larger windows are,
*           .      in general, more powerful than smaller ones,
*           .      rapidly increase the window to the maximum possible.
*           .      Then, gradually reduce the window size. ====
*
            NH = KBOT - KTOP + 1
            NWUPBD = MIN( NH, NWMAX )
            IF( NDFL.LT.KEXNW ) THEN
               NW = MIN( NWUPBD, NWR )
            ELSE
               NW = MIN( NWUPBD, 2*NW )
            END IF
            IF( NW.LT.NWMAX ) THEN
               IF( NW.GE.NH-1 ) THEN
                  NW = NH
               ELSE
                  KWTOP = KBOT - NW + 1
                  IF( CABS1( H( KWTOP, KWTOP-1 ) ).GT.
     $                CABS1( H( KWTOP-1, KWTOP-2 ) ) )NW = NW + 1
               END IF
            END IF
            IF( NDFL.LT.KEXNW ) THEN
               NDEC = -1
            ELSE IF( NDEC.GE.0 .OR. NW.GE.NWUPBD ) THEN
               NDEC = NDEC + 1
               IF( NW-NDEC.LT.2 )
     $            NDEC = 0
               NW = NW - NDEC
            END IF
*
            ==== Aggressive early deflation:
*           .    split workspace under the subdiagonal into
*           .      - an nw-by-nw work array V in the lower
*           .        left-hand-corner,
*           .      - an NW-by-at-least-NW-but-more-is-better
*           .        (NW-by-NHO) horizontal work array along
*           .        the bottom edge,
*           .      - an at-least-NW-but-more-is-better (NHV-by-NW)
*           .        vertical work array along the left-hand-edge.
*           .        ====
*
            KV = N - NW + 1
            KT = NW + 1
```

```
      NHO = ( N-NW-1 ) - KT + 1
      KWV = NW + 2
      NVE = ( N-NW ) - KWV + 1
*
*     ==== Aggressive early deflation ====
*
      CALL ZLAQR2( WANTT, WANTZ, N, KTOP, KBOT, NW, H, LDH, ILOZ,
     $             IHIZ, Z, LDZ, LS, LD, W, H( KV, 1 ), LDH, NHO,
     $             H( KV, KT ), LDH, NVE, H( KWV, 1 ), LDH, WORK,
     $             LWORK )
*
*     ==== Adjust KBOT accounting for new deflations. ====
*
      KBOT = KBOT - LD
*
*     ==== KS points to the shifts. ====
*
      KS = KBOT - LS + 1
*
*     ==== Skip an expensive QR sweep if there is a (partly
*     .    heuristic) reason to expect that many eigenvalues
*     .    will deflate without it.  Here, the QR sweep is
*     .    skipped if many eigenvalues have just been deflated
*     .    or if the remaining active block is small.
*
      IF( ( LD.EQ.0 ) .OR. ( ( 100*LD.LE.NW*NIBBLE ) .AND. ( KBOT-
     $     KTOP+1.GT.MIN( NMIN, NWMAX ) ) ) ) THEN
*
*        ==== NS = nominal number of simultaneous shifts.
*        .    This may be lowered (slightly) if ZLAQR2
*        .    did not provide that many shifts. ====
*
         NS = MIN( NSMAX, NSR, MAX( 2, KBOT-KTOP ) )
         NS = NS - MOD( NS, 2 )
*
*        ==== If there have been no deflations
*        .    in a multiple of KEXSH iterations,
*        .    then try exceptional shifts.
*        .    Otherwise use shifts provided by
*        .    ZLAQR2 above or from the eigenvalues
*        .    of a trailing principal submatrix. ====
*
         IF( MOD( NDFL, KEXSH ).EQ.0 ) THEN
            KS = KBOT - NS + 1
            DO 30 I = KBOT, KS + 1, -2
               W( I ) = H( I, I ) + WILK1*CABS1( H( I, I-1 ) )
               W( I-1 ) = W( I )
   30       CONTINUE
         ELSE
*
```

```
*                 ==== Got NS/2 or fewer shifts? Use ZLAHQR
*                 .    on a trailing principal submatrix to
*                 .    get more. (Since NS.LE.NSMAX.LE.(N+6)/9,
*                 .    there is enough space below the subdiagonal
*                 .    to fit an NS-by-NS scratch array.) ====
*
                  IF( KBOT-KS+1.LE.NS / 2 ) THEN
                     KS = KBOT - NS + 1
                     KT = N - NS + 1
                     CALL ZLACPY( 'A', NS, NS, H( KS, KS ), LDH,
     $                            H( KT, 1 ), LDH )
                     CALL ZLAHQR( .false., .false., NS, 1, NS,
     $                            H( KT, 1 ), LDH, W( KS ), 1, 1, ZDUM,
     $                            1, INF )
                     KS = KS + INF
*
*                    ==== In case of a rare QR failure use
*                    .    eigenvalues of the trailing 2-by-2
*                    .    principal submatrix.  Scale to avoid
*                    .    overflows, underflows and subnormals.
*                    .    (The scale factor S can not be zero,
*                    .    because H(KBOT,KBOT-1) is nonzero.) ====
*
                     IF( KS.GE.KBOT ) THEN
                        S = CABS1( H( KBOT-1, KBOT-1 ) ) +
     $                      CABS1( H( KBOT, KBOT-1 ) ) +
     $                      CABS1( H( KBOT-1, KBOT ) ) +
     $                      CABS1( H( KBOT, KBOT ) )
                        AA = H( KBOT-1, KBOT-1 ) / S
                        CC = H( KBOT, KBOT-1 ) / S
                        BB = H( KBOT-1, KBOT ) / S
                        DD = H( KBOT, KBOT ) / S
                        TR2 = ( AA+DD ) / TWO
                        DET = ( AA-TR2 )*( DD-TR2 ) - BB*CC
                        RTDISC = SQRT( -DET )
                        W( KBOT-1 ) = ( TR2+RTDISC )*S
                        W( KBOT ) = ( TR2-RTDISC )*S
*
                        KS = KBOT - 1
                     END IF
                  END IF
*
                  IF( KBOT-KS+1.GT.NS ) THEN
*
*                    ==== Sort the shifts (Helps a little) ====
*
                     SORTED = .false.
                     DO 50 K = KBOT, KS + 1, -1
                        IF( SORTED )
     $                     GO TO 60
```

```
                     SORTED = .true.
                     DO 40 I = KS, K - 1
                        IF( CABS1( W( I ) ).LT.CABS1( W( I+1 ) ) )
     $                       THEN
                           SORTED = .false.
                           SWAP = W( I )
                           W( I ) = W( I+1 )
                           W( I+1 ) = SWAP
                        END IF
   40                CONTINUE
   50             CONTINUE
   60             CONTINUE
                  END IF
               END IF
*
*              ==== If there are only two shifts, then use
*              .    only one.  ====
*
               IF( KBOT-KS+1.EQ.2 ) THEN
                  IF( CABS1( W( KBOT )-H( KBOT, KBOT ) ).LT.
     $                CABS1( W( KBOT-1 )-H( KBOT, KBOT ) ) ) THEN
                     W( KBOT-1 ) = W( KBOT )
                  ELSE
                     W( KBOT ) = W( KBOT-1 )
                  END IF
               END IF
*
*              ==== Use up to NS of the the smallest magnatiude
*              .    shifts.  If there aren't NS shifts available,
*              .    then use them all, possibly dropping one to
*              .    make the number of shifts even. ====
*
               NS = MIN( NS, KBOT-KS+1 )
               NS = NS - MOD( NS, 2 )
               KS = KBOT - NS + 1
*
*              ==== Small-bulge multi-shift QR sweep:
*              .    split workspace under the subdiagonal into
*              .    - a KDU-by-KDU work array U in the lower
*              .      left-hand-corner,
*              .    - a KDU-by-at-least-KDU-but-more-is-better
*              .      (KDU-by-NHo) horizontal work array WH along
*              .      the bottom edge,
*              .    - and an at-least-KDU-but-more-is-better-by-KDU
*              .      (NVE-by-KDU) vertical work WV arrow along
*              .      the left-hand-edge. ====
*
               KDU = 3*NS - 3
               KU = N - KDU + 1
               KWH = KDU + 1
```

```
               NHO = ( N-KDU+1-4 ) - ( KDU+1 ) + 1
               KWV = KDU + 4
               NVE = N - KDU - KWV + 1
*
*              ==== Small-bulge multi-shift QR sweep ====
*
               CALL ZLAQR5( WANTT, WANTZ, KACC22, N, KTOP, KBOT, NS,
     $                      W( KS ), H, LDH, ILOZ, IHIZ, Z, LDZ, WORK,
     $                      3, H( KU, 1 ), LDH, NVE, H( KWV, 1 ), LDH,
     $                      NHO, H( KU, KWH ), LDH )
            END IF
*
*           ==== Note progress (or the lack of it). ====
*
            IF( LD.GT.0 ) THEN
               NDFL = 1
            ELSE
               NDFL = NDFL + 1
            END IF
*
*           ==== End of main loop ====
   70    CONTINUE
*
*        ==== Iteration limit exceeded.  Set INFO to show where
*        .    the problem occurred and exit. ====
*
         INFO = KBOT
   80    CONTINUE
      END IF
*
*     ==== Return the optimal value of LWORK. ====
*
      WORK( 1 ) = DCMPLX( LWKOPT, 0 )
*
*     ==== End of ZLAQR4 ====
*
      END
```

———-

90.227.4 lisp code

— zlaqr4.lisp —

```
(let* ((ntiny 11)
       (kexnw 5)
       (kexsh 6)
```

```
      (wilk1 0.75d0)
      (zero (f2cl-lib:cmplx 0.0d0 0.0d0))
      (one (f2cl-lib:cmplx 1.0d0 0.0d0))
      (two 2.0d0))
 (declare (type (f2cl-lib:integer4 11 11) ntiny)
          (type (f2cl-lib:integer4 5 5) kexnw)
          (type (f2cl-lib:integer4 6 6) kexsh)
          (type (double-float 0.75d0 0.75d0) wilk1)
          (type (f2cl-lib:complex16) zero)
          (type (f2cl-lib:complex16) one)
          (type (double-float 2.0d0 2.0d0) two)
          (ignorable ntiny kexnw kexsh wilk1 zero one two))
 (defun zlaqr4 (wantt wantz n ilo ihi h ldh w iloz ihiz z ldz work lwork info)
   (declare (type (array f2cl-lib:complex16 (*)) work z w h)
            (type (f2cl-lib:integer4) info lwork ldz ihiz iloz ldh ihi ilo n)
            (type f2cl-lib:logical wantz wantt))
   (f2cl-lib:with-multi-array-data
       ((h f2cl-lib:complex16 h-%data% h-%offset%)
        (w f2cl-lib:complex16 w-%data% w-%offset%)
        (z f2cl-lib:complex16 z-%data% z-%offset%)
        (work f2cl-lib:complex16 work-%data% work-%offset%))
     (labels ((cabs1 (cdum)
                (+ (abs (f2cl-lib:dble cdum)) (abs (f2cl-lib:dimag cdum)))))
       (declare
        (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
         cabs1))
       (prog ((zdum (make-array 1 :element-type 'f2cl-lib:complex16))
              (jbcmpz
               (make-array '(2) :element-type 'character :initial-element
                           #\ ))
              (sorted nil) (i 0) (inf 0) (it 0) (itmax 0) (k 0) (kacc22 0)
              (kbot 0) (kdu 0) (ks 0) (kt 0) (ktop 0) (ku 0) (kv 0) (kwh 0)
              (kwtop 0) (kwv 0) (ld 0) (ls 0) (lwkopt 0) (ndec 0) (ndfl 0)
              (nh 0) (nho 0) (nibble 0) (nmin 0) (ns 0) (nsmax 0) (nsr 0)
              (nve 0) (nw 0) (nwmax 0) (nwr 0) (nwupbd 0) (s 0.0d0)
              (aa #C(0.0d0 0.0d0)) (bb #C(0.0d0 0.0d0)) (cc #C(0.0d0 0.0d0))
              (cdum #C(0.0d0 0.0d0)) (dd #C(0.0d0 0.0d0))
              (det #C(0.0d0 0.0d0)) (rtdisc #C(0.0d0 0.0d0))
              (swap #C(0.0d0 0.0d0)) (tr2 #C(0.0d0 0.0d0)))
         (declare (type (array f2cl-lib:complex16 (1)) zdum)
                  (type (string 2) jbcmpz)
                  (type f2cl-lib:logical sorted)
                  (type (f2cl-lib:integer4) i inf it itmax k kacc22 kbot kdu
                   ks kt ktop ku kv kwh kwtop kwv ld ls lwkopt ndec ndfl nh
                   nho nibble nmin ns nsmax nsr nve nw nwmax nwr nwupbd)
                  (type (double-float) s)
                  (type (f2cl-lib:complex16) aa bb cc cdum dd det rtdisc swap
                   tr2))
         (setf info 0)
         (cond
```

```
  ((= n 0)
   (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%) one)
   (go end_label)))
 (cond
  ((<= n ntiny) (setf lwkopt 1)
   (if (/= lwork -1)
       (multiple-value-bind
           (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
            var-9 var-10 var-11 var-12)
           (zlahqr wantt wantz n ilo ihi h ldh w iloz ihiz z ldz info)
         (declare
          (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-8
           var-9 var-10 var-11))
         (setf ldh var-6)
         (setf info var-12))))
  (t
   (tagbody
     (setf info 0)
     (cond
      (wantt
       (f2cl-lib:fset-string (f2cl-lib:fref-string jbcmpz (1 1)) "S"))
      (t
       (f2cl-lib:fset-string (f2cl-lib:fref-string jbcmpz (1 1))
                             "E")))
     (cond
      (wantz
       (f2cl-lib:fset-string (f2cl-lib:fref-string jbcmpz (2 2)) "V"))
      (t
       (f2cl-lib:fset-string (f2cl-lib:fref-string jbcmpz (2 2))
                             "N")))
     (setf nwr
             (multiple-value-bind
                 (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                 (ilaenv 13 "ZLAQR4" jbcmpz n ilo ihi lwork)
               (declare (ignore var-0 var-1))
               (setf jbcmpz var-2)
               (setf n var-3)
               (setf ilo var-4)
               (setf ihi var-5)
               (setf lwork var-6)
               ret-val))
     (setf nwr
             (max (the f2cl-lib:integer4 2)
                  (the f2cl-lib:integer4 nwr)))
     (setf nwr
             (min (f2cl-lib:int-add (f2cl-lib:int-sub ihi ilo) 1)
                  (the f2cl-lib:integer4 (truncate (- n 1) 3)) nwr))
     (setf nsr
             (multiple-value-bind
                 (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
```

```
            (ilaenv 15 "ZLAQR4" jbcmpz n ilo ihi lwork)
          (declare (ignore var-0 var-1))
          (setf jbcmpz var-2)
          (setf n var-3)
          (setf ilo var-4)
          (setf ihi var-5)
          (setf lwork var-6)
          ret-val))
(setf nsr
        (min nsr (the f2cl-lib:integer4 (truncate (+ n 6) 9))
             (f2cl-lib:int-sub ihi ilo)))
(setf nsr
        (max (the f2cl-lib:integer4 2)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-sub nsr (mod nsr 2)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10 var-11 var-12 var-13 var-14 var-15 var-16 var-17
     var-18 var-19 var-20 var-21 var-22 var-23 var-24)
    (zlaqr2 wantt wantz n ilo ihi (f2cl-lib:int-add nwr 1) h ldh
     iloz ihiz z ldz ls ld w h ldh n h ldh n h ldh work -1)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8 var-9
    var-10 var-14 var-15 var-17 var-18 var-20 var-21 var-23
    var-24))
  (setf ldh var-7)
  (setf ldz var-11)
  (setf ls var-12)
  (setf ld var-13)
  (setf ldh var-16)
  (setf ldh var-19)
  (setf ldh var-22))
(setf lwkopt
        (max (the f2cl-lib:integer4 (truncate (* 3 nsr) 2))
             (f2cl-lib:int
              (f2cl-lib:fref work-%data% (1) ((1 *))
                             work-%offset%))))
(cond
 ((= lwork (f2cl-lib:int-sub 1))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (f2cl-lib:dcmplx lwkopt 0))
  (go end_label)))
(setf nmin
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 12 "ZLAQR4" jbcmpz n ilo ihi lwork)
          (declare (ignore var-0 var-1))
          (setf jbcmpz var-2)
          (setf n var-3)
          (setf ilo var-4)
```

```
          (setf ihi var-5)
          (setf lwork var-6)
          ret-val))
(setf nmin
        (max (the f2cl-lib:integer4 ntiny)
             (the f2cl-lib:integer4 nmin)))
(setf nibble
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 14 "ZLAQR4" jbcmpz n ilo ihi lwork)
          (declare (ignore var-0 var-1))
          (setf jbcmpz var-2)
          (setf n var-3)
          (setf ilo var-4)
          (setf ihi var-5)
          (setf lwork var-6)
          ret-val))
(setf nibble
        (max (the f2cl-lib:integer4 0)
             (the f2cl-lib:integer4 nibble)))
(setf kacc22
        (multiple-value-bind
            (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
            (ilaenv 16 "ZLAQR4" jbcmpz n ilo ihi lwork)
          (declare (ignore var-0 var-1))
          (setf jbcmpz var-2)
          (setf n var-3)
          (setf ilo var-4)
          (setf ihi var-5)
          (setf lwork var-6)
          ret-val))
(setf kacc22
        (max (the f2cl-lib:integer4 0)
             (the f2cl-lib:integer4 kacc22)))
(setf kacc22
        (min (the f2cl-lib:integer4 2)
             (the f2cl-lib:integer4 kacc22)))
(setf nwmax
        (min (the f2cl-lib:integer4 (truncate (- n 1) 3))
             (the f2cl-lib:integer4 (truncate lwork 2))))
(setf nw nwmax)
(setf nsmax
        (min (the f2cl-lib:integer4 (truncate (+ n 6) 9))
             (the f2cl-lib:integer4 (truncate (* 2 lwork) 3))))
(setf nsmax (f2cl-lib:int-sub nsmax (mod nsmax 2)))
(setf ndfl 1)
(setf itmax
        (f2cl-lib:int-mul
         (max (the f2cl-lib:integer4 30)
              (the f2cl-lib:integer4 (f2cl-lib:int-mul 2 kexsh)))
```

```
          (max (the f2cl-lib:integer4 10)
               (the f2cl-lib:integer4
                    (f2cl-lib:int-add (f2cl-lib:int-sub ihi ilo)
                                      1)))))
(setf kbot ihi)
(f2cl-lib:fdo (it 1 (f2cl-lib:int-add it 1))
              ((> it itmax) nil)
  (tagbody
    (if (< kbot ilo)
        (go label80))
    (f2cl-lib:fdo (k kbot
                   (f2cl-lib:int-add k (f2cl-lib:int-sub 1)))
                  ((> k (f2cl-lib:int-add ilo 1)) nil)
      (tagbody
        (if (=
             (f2cl-lib:fref h-%data% (k (f2cl-lib:int-sub k 1))
                            ((1 ldh) (1 *)) h-%offset%)
             zero)
            (go label20))
       label10))
    (setf k ilo)
   label20
    (setf ktop k)
    (setf nh (f2cl-lib:int-add (f2cl-lib:int-sub kbot ktop) 1))
    (setf nwupbd
            (min (the f2cl-lib:integer4 nh)
                 (the f2cl-lib:integer4 nwmax)))
    (cond
     ((< ndfl kexnw)
      (setf nw
              (min (the f2cl-lib:integer4 nwupbd)
                   (the f2cl-lib:integer4 nwr))))
     (t
      (setf nw
              (min (the f2cl-lib:integer4 nwupbd)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-mul 2 nw))))))
    (cond
     ((< nw nwmax)
      (cond
       ((>= nw (f2cl-lib:int-add nh (f2cl-lib:int-sub 1)))
        (setf nw nh))
       (t
        (setf kwtop
                (f2cl-lib:int-add (f2cl-lib:int-sub kbot nw) 1))
        (if (>
             (cabs1
              (f2cl-lib:fref h-%data%
                             (kwtop (f2cl-lib:int-sub kwtop 1))
                             ((1 ldh) (1 *)) h-%offset%))
```

```
          (cabs1
           (f2cl-lib:fref h-%data%
                          ((f2cl-lib:int-sub kwtop 1)
                           (f2cl-lib:int-sub kwtop 2))
                          ((1 ldh) (1 *)) h-%offset%)))
         (setf nw (f2cl-lib:int-add nw 1)))))))
(cond ((< ndfl kexnw) (setf ndec -1))
      ((or (>= ndec 0) (>= nw nwupbd))
       (setf ndec (f2cl-lib:int-add ndec 1))
       (if (< (f2cl-lib:int-sub nw ndec) 2)
           (setf ndec 0))
       (setf nw (f2cl-lib:int-sub nw ndec))))
(setf kv (f2cl-lib:int-add (f2cl-lib:int-sub n nw) 1))
(setf kt (f2cl-lib:int-add nw 1))
(setf nho (f2cl-lib:int-add (f2cl-lib:int-sub n nw 1 kt) 1))
(setf kwv (f2cl-lib:int-add nw 2))
(setf nve (f2cl-lib:int-add (f2cl-lib:int-sub n nw kwv) 1))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17 var-18 var-19 var-20 var-21 var-22 var-23 var-24)
    (zlaqr2 wantt wantz n ktop kbot nw h ldh iloz ihiz z ldz
     ls ld w
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16 (kv 1)
                           ((1 ldh) (1 *)) h-%offset%)
     ldh nho
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                           (kv kt) ((1 ldh) (1 *))
                           h-%offset%)
     ldh nve
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                           (kwv 1) ((1 ldh) (1 *))
                           h-%offset%)
     ldh work lwork)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8
    var-9 var-10 var-14 var-15 var-17 var-18 var-20 var-21
    var-23 var-24))
  (setf ldh var-7)
  (setf ldz var-11)
  (setf ls var-12)
  (setf ld var-13)
  (setf ldh var-16)
  (setf ldh var-19)
  (setf ldh var-22))
(setf kbot (f2cl-lib:int-sub kbot ld))
(setf ks (f2cl-lib:int-add (f2cl-lib:int-sub kbot ls) 1))
(cond
 ((or (= ld 0)
      (and
```

```
     (<= (f2cl-lib:int-mul 100 ld)
         (f2cl-lib:int-mul nw nibble))
     (> (f2cl-lib:int-add kbot (f2cl-lib:int-sub ktop) 1)
        (min (the f2cl-lib:integer4 nmin)
             (the f2cl-lib:integer4 nwmax)))))
(setf ns
        (min (the f2cl-lib:integer4 nsmax)
             (the f2cl-lib:integer4 nsr)
             (the f2cl-lib:integer4
                  (max (the f2cl-lib:integer4 2)
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-sub kbot
                                              ktop))))))
(setf ns (f2cl-lib:int-sub ns (mod ns 2)))
(cond
 ((= (mod ndfl kexsh) 0)
  (setf ks (f2cl-lib:int-add (f2cl-lib:int-sub kbot ns) 1))
  (f2cl-lib:fdo (i kbot
                 (f2cl-lib:int-add i (f2cl-lib:int-sub 2)))
                ((> i (f2cl-lib:int-add ks 1)) nil)
    (tagbody
      (setf (f2cl-lib:fref w-%data% (i) ((1 *)) w-%offset%)
              (+
               (f2cl-lib:fref h-%data% (i i)
                              ((1 ldh) (1 *)) h-%offset%)
               (* wilk1
                  (cabs1
                   (f2cl-lib:fref h-%data%
                                  (i
                                   (f2cl-lib:int-sub i 1))
                                  ((1 ldh) (1 *))
                                  h-%offset%)))))
      (setf (f2cl-lib:fref w-%data%
                           ((f2cl-lib:int-sub i 1)) ((1 *))
                           w-%offset%)
              (f2cl-lib:fref w-%data% (i) ((1 *))
                             w-%offset%))
     label30)))
 (t
  (cond
   ((<= (f2cl-lib:int-add kbot (f2cl-lib:int-sub ks) 1)
        (f2cl-lib:f2cl/ ns 2))
    (setf ks
            (f2cl-lib:int-add (f2cl-lib:int-sub kbot ns)
                              1))
    (setf kt (f2cl-lib:int-add (f2cl-lib:int-sub n ns) 1))
    (zlacpy "A" ns ns
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                           (ks ks) ((1 ldh) (1 *))
                           h-%offset%)
```

```
 ldh
 (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                       (kt 1) ((1 ldh) (1 *))
                       h-%offset%)
 ldh)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
     var-8 var-9 var-10 var-11 var-12)
    (zlahqr f2cl-lib:%false% f2cl-lib:%false% ns 1 ns
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                           (kt 1) ((1 ldh) (1 *))
                           h-%offset%)
     ldh
     (f2cl-lib:array-slice w-%data% f2cl-lib:complex16
                           (ks) ((1 *)) w-%offset%)
     1 1 zdum 1 inf)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7
    var-8 var-9 var-10 var-11))
  (setf ldh var-6)
  (setf inf var-12))
(setf ks (f2cl-lib:int-add ks inf))
(cond
 ((>= ks kbot)
  (setf s
          (+
           (cabs1
            (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-sub kbot 1)
                            (f2cl-lib:int-sub kbot 1))
                           ((1 ldh) (1 *)) h-%offset%))
           (cabs1
            (f2cl-lib:fref h-%data%
                           (kbot
                            (f2cl-lib:int-sub kbot 1))
                           ((1 ldh) (1 *)) h-%offset%))
           (cabs1
            (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-sub kbot 1)
                            kbot)
                           ((1 ldh) (1 *)) h-%offset%))
           (cabs1
            (f2cl-lib:fref h-%data% (kbot kbot)
                           ((1 ldh) (1 *))
                           h-%offset%))))
  (setf aa
          (/
           (f2cl-lib:fref h-%data%
                          ((f2cl-lib:int-sub kbot 1)
                           (f2cl-lib:int-sub kbot 1))
```

```
                                ((1 ldh) (1 *)) h-%offset%)
                         s))
                  (setf cc
                         (/
                          (f2cl-lib:fref h-%data%
                                ((1 ldh) (1 *)) h-%offset%)
                          s))
                  (setf bb
                         (/
                          (f2cl-lib:fref h-%data%
                                ((f2cl-lib:int-sub kbot 1)
                                 kbot)
                                ((1 ldh) (1 *)) h-%offset%)
                          s))
                  (setf dd
                         (/
                          (f2cl-lib:fref h-%data% (kbot kbot)
                                ((1 ldh) (1 *)) h-%offset%)
                          s))
                  (setf tr2 (/ (+ aa dd) two))
                  (setf det (- (* (- aa tr2) (- dd tr2)) (* bb cc)))
                  (setf rtdisc (f2cl-lib:fsqrt (- det)))
                  (setf (f2cl-lib:fref w-%data%
                                ((f2cl-lib:int-sub kbot 1))
                                ((1 *)) w-%offset%)
                         (* (+ tr2 rtdisc) s))
                  (setf (f2cl-lib:fref w-%data% (kbot) ((1 *))
                                w-%offset%)
                         (* (- tr2 rtdisc) s))
                  (setf ks (f2cl-lib:int-sub kbot 1))))))
              (cond
               ((> (f2cl-lib:int-add kbot (f2cl-lib:int-sub ks) 1) ns)
                (tagbody
                  (setf sorted f2cl-lib:%false%)
                  (f2cl-lib:fdo (k kbot
                                 (f2cl-lib:int-add k
                                                   (f2cl-lib:int-sub
                                                    1)))
                                ((> k (f2cl-lib:int-add ks 1)) nil)
                    (tagbody
                      (if sorted
                          (go label60))
                      (setf sorted f2cl-lib:%true%)
                      (f2cl-lib:fdo (i ks (f2cl-lib:int-add i 1))
                                    ((> i
                                        (f2cl-lib:int-add k
                                                          (f2cl-lib:int-sub
                                                           1)))
```

```
                                 nil)
                        (tagbody
                          (cond
                           ((< (cabs1 (f2cl-lib:fref w (i) ((1 *))))
                               (cabs1
                                (f2cl-lib:fref w
                                               ((f2cl-lib:int-add i 1))
                                               ((1 *)))))
                            (setf sorted f2cl-lib:%false%)
                            (setf swap
                                    (f2cl-lib:fref w-%data% (i) ((1 *))
                                                   w-%offset%))
                            (setf (f2cl-lib:fref w-%data% (i) ((1 *))
                                                 w-%offset%)
                                    (f2cl-lib:fref w-%data%
                                                   ((f2cl-lib:int-add i
                                                                      1))
                                                   ((1 *)) w-%offset%))
                            (setf (f2cl-lib:fref w-%data%
                                                 ((f2cl-lib:int-add i
                                                                    1))
                                                 ((1 *)) w-%offset%)
                                    swap)))
                         label40))
                     label50))
                 label60)))))
            (cond
             ((= (f2cl-lib:int-add kbot (f2cl-lib:int-sub ks) 1) 2)
              (cond
               ((<
                 (cabs1
                  (+ (f2cl-lib:fref w (kbot) ((1 *)))
                     (-
                      (f2cl-lib:fref h (kbot kbot) ((1 ldh) (1 *))))))
                 (cabs1
                  (+
                   (f2cl-lib:fref w
                                  ((f2cl-lib:int-add kbot
                                                     (f2cl-lib:int-sub
                                                      1)))
                                  ((1 *)))
                   (- (f2cl-lib:fref h (kbot kbot) ((1 ldh) (1 *)))))))
                (setf (f2cl-lib:fref w-%data%
                                     ((f2cl-lib:int-sub kbot 1))
                                     ((1 *)) w-%offset%)
                        (f2cl-lib:fref w-%data% (kbot) ((1 *))
                                       w-%offset%)))
               (t
                (setf (f2cl-lib:fref w-%data% (kbot) ((1 *))
                                     w-%offset%)
```

```
            (f2cl-lib:fref w-%data%
                           ((f2cl-lib:int-sub kbot 1))
                           ((1 *)) w-%offset%))))))
(setf ns
        (min (the f2cl-lib:integer4 ns)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add
                   (f2cl-lib:int-sub kbot ks) 1))))
(setf ns (f2cl-lib:int-sub ns (mod ns 2)))
(setf ks (f2cl-lib:int-add (f2cl-lib:int-sub kbot ns) 1))
(setf kdu (f2cl-lib:int-sub (f2cl-lib:int-mul 3 ns) 3))
(setf ku (f2cl-lib:int-add (f2cl-lib:int-sub n kdu) 1))
(setf kwh (f2cl-lib:int-add kdu 1))
(setf nho
        (f2cl-lib:int-add
         (f2cl-lib:int-sub
          (f2cl-lib:int-add (f2cl-lib:int-sub n kdu) 1) 4
          (f2cl-lib:int-add kdu 1))
         1))
(setf kwv (f2cl-lib:int-add kdu 4))
(setf nve
        (f2cl-lib:int-add (f2cl-lib:int-sub n kdu kwv) 1))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
     var-9 var-10 var-11 var-12 var-13 var-14 var-15 var-16
     var-17 var-18 var-19 var-20 var-21 var-22 var-23)
    (zlaqr5 wantt wantz kacc22 n ktop kbot ns
     (f2cl-lib:array-slice w-%data% f2cl-lib:complex16 (ks)
                           ((1 *)) w-%offset%)
     h ldh iloz ihiz z ldz work 3
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                           (ku 1) ((1 ldh) (1 *))
                           h-%offset%)
     ldh nve
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                           (kwv 1) ((1 ldh) (1 *))
                           h-%offset%)
     ldh nho
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                           (ku kwh) ((1 ldh) (1 *))
                           h-%offset%)
     ldh)
  (declare
   (ignore var-7 var-8 var-12 var-14 var-15 var-16 var-19
    var-22))
  (when var-0 (setf wantt var-0))
  (when var-1 (setf wantz var-1))
  (when var-2 (setf kacc22 var-2))
  (when var-3 (setf n var-3))
  (when var-4 (setf ktop var-4))
```

```
                    (when var-5 (setf kbot var-5))
                    (when var-6 (setf ns var-6))
                    (when var-9 (setf ldh var-9))
                    (when var-10 (setf iloz var-10))
                    (when var-11 (setf ihiz var-11))
                    (when var-13 (setf ldz var-13))
                    (when var-17 (setf ldh var-17))
                    (when var-18 (setf nve var-18))
                    (when var-20 (setf ldh var-20))
                    (when var-21 (setf nho var-21))
                    (when var-23 (setf ldh var-23)))))
                (cond ((> ld 0) (setf ndfl 1))
                      (t (setf ndfl (f2cl-lib:int-add ndfl 1))))
               label70))
            (setf info kbot)
           label80)))
         (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                 (f2cl-lib:dcmplx lwkopt 0))
       end_label
        (return
         (values wantt wantz n ilo ihi nil ldh nil iloz ihiz nil ldz nil
                 lwork info)))))))
```

———-

90.228 zlaqr5

See Braman [Bram02a, Bram02b]

90.228.1 Axiom unit tests

— **zlaqr5.input** —

```
)set break resume
)sys rm -f zlaqr5.output
)spool zlaqr5.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.228.2 Axiom help page

— zlaqr5.help —

```
====================================================================
zlaqr5 examples
====================================================================

====================================================================
Man Page Details
====================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZLAQR5( WANTT, WANTZ, KACC22, N, KTOP, KBOT, NSHFTS, S,
                          H, LDH, ILOZ, IHIZ, Z, LDZ, V, LDV, U, LDU, NV,
                          WV, LDWV, NH, WH, LDWH )

       .. Scalar Arguments ..
       INTEGER            IHIZ, ILOZ, KACC22, KBOT, KTOP, LDH, LDU, LDV,
      $                   LDWH, LDWV, LDZ, N, NH, NSHFTS, NV
       LOGICAL            WANTT, WANTZ
       ..
       .. Array Arguments ..
       COMPLEX*16         H( LDH, * ), S( * ), U( LDU, * ), V( LDV, * ),
      $                   WH( LDWH, * ), WV( LDWV, * ), Z( LDZ, * )
       ..


 Purpose:
 =============

    ZLAQR5, called by ZLAQR0, performs a
    single small-bulge multi-shift QR sweep.

 Arguments:
 ==========

  [in] WANTT

          WANTT is logical scalar
             WANTT = .true. if the triangular Schur factor
             is being computed.  WANTT is set to .false. otherwise.

  [in] WANTZ
```

```
        WANTZ is logical scalar
           WANTZ = .true. if the unitary Schur factor is being
           computed.  WANTZ is set to .false. otherwise.

[in] KACC22

        KACC22 is integer with value 0, 1, or 2.
           Specifies the computation mode of far-from-diagonal
           orthogonal updates.
      = 0: ZLAQR5 does not accumulate reflections and does not
           use matrix-matrix multiply to update far-from-diagonal
           matrix entries.
      = 1: ZLAQR5 accumulates reflections and uses matrix-matrix
           multiply to update the far-from-diagonal matrix entries.
      = 2: ZLAQR5 accumulates reflections, uses matrix-matrix
           multiply to update the far-from-diagonal matrix entries,
           and takes advantage of 2-by-2 block structure during
           matrix multiplies.

[in] N

        N is integer scalar
           N is the order of the Hessenberg matrix H upon which this
           subroutine operates.

[in] KTOP

        KTOP is integer scalar

[in] KBOT

        KBOT is integer scalar
           These are the first and last rows and columns of an
           isolated diagonal block upon which the QR sweep is to be
           applied. It is assumed without a check that
                     either KTOP = 1  or   H(KTOP,KTOP-1) = 0
           and
                     either KBOT = N  or   H(KBOT+1,KBOT) = 0.

[in] NSHFTS

        NSHFTS is integer scalar
           NSHFTS gives the number of simultaneous shifts.  NSHFTS
           must be positive and even.

[in,out] S

        S is COMPLEX*16 array of size (NSHFTS)
           S contains the shifts of origin that define the multi-
```

```
           shift QR sweep.  On output S may be reordered.

 [in,out] H

        H is COMPLEX*16 array of size (LDH,N)
           On input H contains a Hessenberg matrix.  On output a
           multi-shift QR sweep with shifts SR(J)+i*SI(J) is applied
           to the isolated diagonal block in rows and columns KTOP
           through KBOT.

 [in] LDH

        LDH is integer scalar
           LDH is the leading dimension of H just as declared in the
           calling procedure.  LDH.GE.MAX(1,N).

 [in] ILOZ

        ILOZ is INTEGER

 [in] IHIZ

        IHIZ is INTEGER
           Specify the rows of Z to which transformations must be
           applied if WANTZ is .TRUE.. 1 .LE. ILOZ .LE. IHIZ .LE. N

 [in,out] Z

        Z is COMPLEX*16 array of size (LDZ,IHI)
           If WANTZ = .TRUE., then the QR Sweep unitary
           similarity transformation is accumulated into
           Z(ILOZ:IHIZ,ILO:IHI) from the right.
           If WANTZ = .FALSE., then Z is unreferenced.

 [in] LDZ

        LDZ is integer scalar
           LDA is the leading dimension of Z just as declared in
           the calling procedure. LDZ.GE.N.

 [out] V

        V is COMPLEX*16 array of size (LDV,NSHFTS/2)

 [in] LDV

        LDV is integer scalar
           LDV is the leading dimension of V as declared in the
           calling procedure.  LDV.GE.3.
```

```
[out] U

        U is COMPLEX*16 array of size
           (LDU,3*NSHFTS-3)

[in] LDU

        LDU is integer scalar
           LDU is the leading dimension of U just as declared in the
           in the calling subroutine.  LDU.GE.3*NSHFTS-3.

[in] NH

        NH is integer scalar
           NH is the number of columns in array WH available for
           workspace. NH.GE.1.

[out] WH

        WH is COMPLEX*16 array of size (LDWH,NH)

[in] LDWH

        LDWH is integer scalar
           Leading dimension of WH just as declared in the
           calling procedure.  LDWH.GE.3*NSHFTS-3.

[in] NV

        NV is integer scalar
           NV is the number of rows in WV agailable for workspace.
           NV.GE.1.

[out] WV

        WV is COMPLEX*16 array of size
           (LDWV,3*NSHFTS-3)

[in] LDWV

        LDWV is integer scalar
           LDWV is the leading dimension of WV as declared in the
           in the calling subroutine.  LDWV.GE.NV.
```

90.228.3 fortran code

— zlaqr5.f —

```
*  =====================================================================
      SUBROUTINE ZLAQR5( WANTT, WANTZ, KACC22, N, KTOP, KBOT, NSHFTS, S,
     $                   H, LDH, ILOZ, IHIZ, Z, LDZ, V, LDV, U, LDU, NV,
     $                   WV, LDWV, NH, WH, LDWH )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            IHIZ, ILOZ, KACC22, KBOT, KTOP, LDH, LDU, LDV,
     $                   LDWH, LDWV, LDZ, N, NH, NSHFTS, NV
      LOGICAL            WANTT, WANTZ
*     ..
*     .. Array Arguments ..
      COMPLEX*16         H( LDH, * ), S( * ), U( LDU, * ), V( LDV, * ),
     $                   WH( LDWH, * ), WV( LDWV, * ), Z( LDZ, * )
*     ..
*
*  ================================================================
*     .. Parameters ..
      COMPLEX*16         ZERO, ONE
      PARAMETER          ( ZERO = ( 0.0d0, 0.0d0 ),
     $                   ONE = ( 1.0d0, 0.0d0 ) )
      DOUBLE PRECISION   RZERO, RONE
      PARAMETER          ( RZERO = 0.0d0, RONE = 1.0d0 )
*     ..
*     .. Local Scalars ..
      COMPLEX*16         ALPHA, BETA, CDUM, REFSUM
      DOUBLE PRECISION   H11, H12, H21, H22, SAFMAX, SAFMIN, SCL,
     $                   SMLNUM, TST1, TST2, ULP
      INTEGER            I2, I4, INCOL, J, J2, J4, JBOT, JCOL, JLEN,
     $                   JROW, JTOP, K, K1, KDU, KMS, KNZ, KRCOL, KZS,
     $                   M, M22, MBOT, MEND, MSTART, MTOP, NBMPS, NDCOL,
     $                   NS, NU
      LOGICAL            ACCUM, BLK22, BMP22
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           DLAMCH
*     ..
*     .. Intrinsic Functions ..
*
      INTRINSIC          ABS, DBLE, DCONJG, DIMAG, MAX, MIN, MOD
```

```
*     ..
*     .. Local Arrays ..
      COMPLEX*16         VT( 3 )
*     ..
*     .. External Subroutines ..
      EXTERNAL           DLABAD, ZGEMM, ZLACPY, ZLAQR1, ZLARFG, ZLASET,
     $                   ZTRMM
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( CDUM ) = ABS( DBLE( CDUM ) ) + ABS( DIMAG( CDUM ) )
*     ..
*     .. Executable Statements ..
*
*     ==== If there are no shifts, then there is nothing to do. ====
*
      IF( NSHFTS.LT.2 )
     $   RETURN
*
*     ==== If the active block is empty or 1-by-1, then there
*     .    is nothing to do. ====
*
      IF( KTOP.GE.KBOT )
     $   RETURN
*
*     ==== NSHFTS is supposed to be even, but if it is odd,
*     .    then simply reduce it by one.  ====
*
      NS = NSHFTS - MOD( NSHFTS, 2 )
*
*     ==== Machine constants for deflation ====
*
      SAFMIN = DLAMCH( 'SAFE MINIMUM' )
      SAFMAX = RONE / SAFMIN
      CALL DLABAD( SAFMIN, SAFMAX )
      ULP = DLAMCH( 'PRECISION' )
      SMLNUM = SAFMIN*( DBLE( N ) / ULP )
*
*     ==== Use accumulated reflections to update far-from-diagonal
*     .    entries ? ====
*
      ACCUM = ( KACC22.EQ.1 ) .OR. ( KACC22.EQ.2 )
*
*     ==== If so, exploit the 2-by-2 block structure? ====
*
      BLK22 = ( NS.GT.2 ) .AND. ( KACC22.EQ.2 )
*
*     ==== clear trash ====
```

```
*
      IF( KTOP+2.LE.KBOT )
     $   H( KTOP+2, KTOP ) = ZERO
*
*     ==== NBMPS = number of 2-shift bulges in the chain ====
*
      NBMPS = NS / 2
*
*     ==== KDU = width of slab ====
*
      KDU = 6*NBMPS - 3
*
*     ==== Create and chase chains of NBMPS bulges ====
*
      DO 210 INCOL = 3*( 1-NBMPS ) + KTOP - 1, KBOT - 2, 3*NBMPS - 2
         NDCOL = INCOL + KDU
         IF( ACCUM )
     $      CALL ZLASET( 'ALL', KDU, KDU, ZERO, ONE, U, LDU )
*
*        ==== Near-the-diagonal bulge chase.  The following loop
*        .    performs the near-the-diagonal part of a small bulge
*        .    multi-shift QR sweep.  Each 6*NBMPS-2 column diagonal
*        .    chunk extends from column INCOL to column NDCOL
*        .    (including both column INCOL and column NDCOL). The
*        .    following loop chases a 3*NBMPS column long chain of
*        .    NBMPS bulges 3*NBMPS-2 columns to the right.  (INCOL
*        .    may be less than KTOP and and NDCOL may be greater than
*        .    KBOT indicating phantom columns from which to chase
*        .    bulges before they are actually introduced or to which
*        .    to chase bulges beyond column KBOT.)  ====
*
         DO 140 KRCOL = INCOL, MIN( INCOL+3*NBMPS-3, KBOT-2 )
*
*           ==== Bulges number MTOP to MBOT are active double implicit
*           .    shift bulges.  There may or may not also be small
*           .    2-by-2 bulge, if there is room.  The inactive bulges
*           .    (if any) must wait until the active bulges have moved
*           .    down the diagonal to make room.  The phantom matrix
*           .    paradigm described above helps keep track.  ====
*
            MTOP = MAX( 1, ( ( KTOP-1 )-KRCOL+2 ) / 3+1 )
            MBOT = MIN( NBMPS, ( KBOT-KRCOL ) / 3 )
            M22 = MBOT + 1
            BMP22 = ( MBOT.LT.NBMPS ) .AND. ( KRCOL+3*( M22-1 ) ).EQ.
     $              ( KBOT-2 )
*
*           ==== Generate reflections to chase the chain right
*           .    one column.  (The minimum value of K is KTOP-1.) ====
*
            DO 10 M = MTOP, MBOT
```

```
               K = KRCOL + 3*( M-1 )
               IF( K.EQ.KTOP-1 ) THEN
                  CALL ZLAQR1( 3, H( KTOP, KTOP ), LDH, S( 2*M-1 ),
     $                         S( 2*M ), V( 1, M ) )
                  ALPHA = V( 1, M )
                  CALL ZLARFG( 3, ALPHA, V( 2, M ), 1, V( 1, M ) )
               ELSE
                  BETA = H( K+1, K )
                  V( 2, M ) = H( K+2, K )
                  V( 3, M ) = H( K+3, K )
                  CALL ZLARFG( 3, BETA, V( 2, M ), 1, V( 1, M ) )
*
*                 ==== A Bulge may collapse because of vigilant
*                 .    deflation or destructive underflow.  In the
*                 .    underflow case, try the two-small-subdiagonals
*                 .    trick to try to reinflate the bulge.  ====
*
                  IF( H( K+3, K ).NE.ZERO .OR. H( K+3, K+1 ).NE.
     $                ZERO .OR. H( K+3, K+2 ).EQ.ZERO ) THEN
*
*                    ==== Typical case: not collapsed (yet). ====
*
                     H( K+1, K ) = BETA
                     H( K+2, K ) = ZERO
                     H( K+3, K ) = ZERO
                  ELSE
*
*                    ==== Atypical case: collapsed.  Attempt to
*                    .    reintroduce ignoring H(K+1,K) and H(K+2,K).
*                    .    If the fill resulting from the new
*                    .    reflector is too large, then abandon it.
*                    .    Otherwise, use the new one. ====
*
                     CALL ZLAQR1( 3, H( K+1, K+1 ), LDH, S( 2*M-1 ),
     $                            S( 2*M ), VT )
                     ALPHA = VT( 1 )
                     CALL ZLARFG( 3, ALPHA, VT( 2 ), 1, VT( 1 ) )
                     REFSUM = DCONJG( VT( 1 ) )*
     $                       ( H( K+1, K )+DCONJG( VT( 2 ) )*
     $                       H( K+2, K ) )
*
                     IF( CABS1( H( K+2, K )-REFSUM*VT( 2 ) )+
     $                   CABS1( REFSUM*VT( 3 ) ).GT.ULP*
     $                   ( CABS1( H( K, K ) )+CABS1( H( K+1,
     $                   K+1 ) )+CABS1( H( K+2, K+2 ) ) ) ) THEN
*
*                       ==== Starting a new bulge here would
*                       .    create non-negligible fill.  Use
*                       .    the old one with trepidation. ====
*
```

```
                     H( K+1, K ) = BETA
                     H( K+2, K ) = ZERO
                     H( K+3, K ) = ZERO
                  ELSE
*
*                    ==== Stating a new bulge here would
*                    .    create only negligible fill.
*                    .    Replace the old reflector with
*                    .    the new one. ====
*
                     H( K+1, K ) = H( K+1, K ) - REFSUM
                     H( K+2, K ) = ZERO
                     H( K+3, K ) = ZERO
                     V( 1, M ) = VT( 1 )
                     V( 2, M ) = VT( 2 )
                     V( 3, M ) = VT( 3 )
                  END IF
               END IF
            END IF
   10       CONTINUE
*
*           ==== Generate a 2-by-2 reflection, if needed. ====
*
            K = KRCOL + 3*( M22-1 )
            IF( BMP22 ) THEN
               IF( K.EQ.KTOP-1 ) THEN
                  CALL ZLAQR1( 2, H( K+1, K+1 ), LDH, S( 2*M22-1 ),
     $                         S( 2*M22 ), V( 1, M22 ) )
                  BETA = V( 1, M22 )
                  CALL ZLARFG( 2, BETA, V( 2, M22 ), 1, V( 1, M22 ) )
               ELSE
                  BETA = H( K+1, K )
                  V( 2, M22 ) = H( K+2, K )
                  CALL ZLARFG( 2, BETA, V( 2, M22 ), 1, V( 1, M22 ) )
                  H( K+1, K ) = BETA
                  H( K+2, K ) = ZERO
               END IF
            END IF
*
*           ==== Multiply H by reflections from the left ====
*
            IF( ACCUM ) THEN
               JBOT = MIN( NDCOL, KBOT )
            ELSE IF( WANTT ) THEN
               JBOT = N
            ELSE
               JBOT = KBOT
            END IF
            DO 30 J = MAX( KTOP, KRCOL ), JBOT
               MEND = MIN( MBOT, ( J-KRCOL+2 ) / 3 )
```

```
               DO 20 M = MTOP, MEND
                  K = KRCOL + 3*( M-1 )
                  REFSUM = DCONJG( V( 1, M ) )*
     $                    ( H( K+1, J )+DCONJG( V( 2, M ) )*
     $                    H( K+2, J )+DCONJG( V( 3, M ) )*H( K+3, J ) )
                  H( K+1, J ) = H( K+1, J ) - REFSUM
                  H( K+2, J ) = H( K+2, J ) - REFSUM*V( 2, M )
                  H( K+3, J ) = H( K+3, J ) - REFSUM*V( 3, M )
   20          CONTINUE
   30       CONTINUE
            IF( BMP22 ) THEN
               K = KRCOL + 3*( M22-1 )
               DO 40 J = MAX( K+1, KTOP ), JBOT
                  REFSUM = DCONJG( V( 1, M22 ) )*
     $                    ( H( K+1, J )+DCONJG( V( 2, M22 ) )*
     $                    H( K+2, J ) )
                  H( K+1, J ) = H( K+1, J ) - REFSUM
                  H( K+2, J ) = H( K+2, J ) - REFSUM*V( 2, M22 )
   40          CONTINUE
            END IF
*
*           ==== Multiply H by reflections from the right.
*           .    Delay filling in the last row until the
*           .    vigilant deflation check is complete. ====
*
            IF( ACCUM ) THEN
               JTOP = MAX( KTOP, INCOL )
            ELSE IF( WANTT ) THEN
               JTOP = 1
            ELSE
               JTOP = KTOP
            END IF
            DO 80 M = MTOP, MBOT
               IF( V( 1, M ).NE.ZERO ) THEN
                  K = KRCOL + 3*( M-1 )
                  DO 50 J = JTOP, MIN( KBOT, K+3 )
                     REFSUM = V( 1, M )*( H( J, K+1 )+V( 2, M )*
     $                       H( J, K+2 )+V( 3, M )*H( J, K+3 ) )
                     H( J, K+1 ) = H( J, K+1 ) - REFSUM
                     H( J, K+2 ) = H( J, K+2 ) -
     $                             REFSUM*DCONJG( V( 2, M ) )
                     H( J, K+3 ) = H( J, K+3 ) -
     $                             REFSUM*DCONJG( V( 3, M ) )
   50             CONTINUE
*
                  IF( ACCUM ) THEN
*
*                    ==== Accumulate U. (If necessary, update Z later
*                    .    with with an efficient matrix-matrix
*                    .    multiply.) ====
```

```
*
                  KMS = K - INCOL
                  DO 60 J = MAX( 1, KTOP-INCOL ), KDU
                     REFSUM = V( 1, M )*( U( J, KMS+1 )+V( 2, M )*
     $                        U( J, KMS+2 )+V( 3, M )*U( J, KMS+3 ) )
                     U( J, KMS+1 ) = U( J, KMS+1 ) - REFSUM
                     U( J, KMS+2 ) = U( J, KMS+2 ) -
     $                               REFSUM*DCONJG( V( 2, M ) )
                     U( J, KMS+3 ) = U( J, KMS+3 ) -
     $                               REFSUM*DCONJG( V( 3, M ) )
   60                CONTINUE
                  ELSE IF( WANTZ ) THEN
*
*                 ==== U is not accumulated, so update Z
*                 .    now by multiplying by reflections
*                 .    from the right. ====
*
                  DO 70 J = ILOZ, IHIZ
                     REFSUM = V( 1, M )*( Z( J, K+1 )+V( 2, M )*
     $                        Z( J, K+2 )+V( 3, M )*Z( J, K+3 ) )
                     Z( J, K+1 ) = Z( J, K+1 ) - REFSUM
                     Z( J, K+2 ) = Z( J, K+2 ) -
     $                             REFSUM*DCONJG( V( 2, M ) )
                     Z( J, K+3 ) = Z( J, K+3 ) -
     $                             REFSUM*DCONJG( V( 3, M ) )
   70                CONTINUE
                  END IF
               END IF
   80       CONTINUE
*
*           ==== Special case: 2-by-2 reflection (if needed) ====
*
            K = KRCOL + 3*( M22-1 )
            IF( BMP22 ) THEN
               IF ( V( 1, M22 ).NE.ZERO ) THEN
                  DO 90 J = JTOP, MIN( KBOT, K+3 )
                     REFSUM = V( 1, M22 )*( H( J, K+1 )+V( 2, M22 )*
     $                        H( J, K+2 ) )
                     H( J, K+1 ) = H( J, K+1 ) - REFSUM
                     H( J, K+2 ) = H( J, K+2 ) -
     $                             REFSUM*DCONJG( V( 2, M22 ) )
   90             CONTINUE
*
                  IF( ACCUM ) THEN
                     KMS = K - INCOL
                     DO 100 J = MAX( 1, KTOP-INCOL ), KDU
                        REFSUM = V( 1, M22 )*( U( J, KMS+1 )+
     $                        V( 2, M22 )*U( J, KMS+2 ) )
                        U( J, KMS+1 ) = U( J, KMS+1 ) - REFSUM
                        U( J, KMS+2 ) = U( J, KMS+2 ) -
```

```
     $                                REFSUM*DCONJG( V( 2, M22 ) )
  100               CONTINUE
                 ELSE IF( WANTZ ) THEN
                    DO 110 J = ILOZ, IHIZ
                       REFSUM = V( 1, M22 )*( Z( J, K+1 )+V( 2, M22 )*
     $                         Z( J, K+2 ) )
                       Z( J, K+1 ) = Z( J, K+1 ) - REFSUM
                       Z( J, K+2 ) = Z( J, K+2 ) -
     $                               REFSUM*DCONJG( V( 2, M22 ) )
  110               CONTINUE
                 END IF
              END IF
           END IF
*
*          ==== Vigilant deflation check ====
*
           MSTART = MTOP
           IF( KRCOL+3*( MSTART-1 ).LT.KTOP )
     $        MSTART = MSTART + 1
           MEND = MBOT
           IF( BMP22 )
     $        MEND = MEND + 1
           IF( KRCOL.EQ.KBOT-2 )
     $        MEND = MEND + 1
           DO 120 M = MSTART, MEND
              K = MIN( KBOT-1, KRCOL+3*( M-1 ) )
*
*             ==== The following convergence test requires that
*             .    the tradition small-compared-to-nearby-diagonals
*             .    criterion and the Ahues & Tisseur (LAWN 122, 1997)
*             .    criteria both be satisfied.  The latter improves
*             .    accuracy in some examples. Falling back on an
*             .    alternate convergence criterion when TST1 or TST2
*             .    is zero (as done here) is traditional but probably
*             .    unnecessary. ====
*
              IF( H( K+1, K ).NE.ZERO ) THEN
                 TST1 = CABS1( H( K, K ) ) + CABS1( H( K+1, K+1 ) )
                 IF( TST1.EQ.RZERO ) THEN
                    IF( K.GE.KTOP+1 )
     $                 TST1 = TST1 + CABS1( H( K, K-1 ) )
                    IF( K.GE.KTOP+2 )
     $                 TST1 = TST1 + CABS1( H( K, K-2 ) )
                    IF( K.GE.KTOP+3 )
     $                 TST1 = TST1 + CABS1( H( K, K-3 ) )
                    IF( K.LE.KBOT-2 )
     $                 TST1 = TST1 + CABS1( H( K+2, K+1 ) )
                    IF( K.LE.KBOT-3 )
     $                 TST1 = TST1 + CABS1( H( K+3, K+1 ) )
                    IF( K.LE.KBOT-4 )
```

```
     $                 TST1 = TST1 + CABS1( H( K+4, K+1 ) )
                 END IF
                 IF( CABS1( H( K+1, K ) ).LE.MAX( SMLNUM, ULP*TST1 ) )
     $                THEN
                    H12 = MAX( CABS1( H( K+1, K ) ),
     $                     CABS1( H( K, K+1 ) ) )
                    H21 = MIN( CABS1( H( K+1, K ) ),
     $                     CABS1( H( K, K+1 ) ) )
                    H11 = MAX( CABS1( H( K+1, K+1 ) ),
     $                     CABS1( H( K, K )-H( K+1, K+1 ) ) )
                    H22 = MIN( CABS1( H( K+1, K+1 ) ),
     $                     CABS1( H( K, K )-H( K+1, K+1 ) ) )
                    SCL = H11 + H12
                    TST2 = H22*( H11 / SCL )
*
                    IF( TST2.EQ.RZERO .OR. H21*( H12 / SCL ).LE.
     $                  MAX( SMLNUM, ULP*TST2 ) )H( K+1, K ) = ZERO
                 END IF
              END IF
  120      CONTINUE
*
*          ==== Fill in the last row of each bulge. ====
*
           MEND = MIN( NBMPS, ( KBOT-KRCOL-1 ) / 3 )
           DO 130 M = MTOP, MEND
              K = KRCOL + 3*( M-1 )
              REFSUM = V( 1, M )*V( 3, M )*H( K+4, K+3 )
              H( K+4, K+1 ) = -REFSUM
              H( K+4, K+2 ) = -REFSUM*DCONJG( V( 2, M ) )
              H( K+4, K+3 ) = H( K+4, K+3 ) -
     $                        REFSUM*DCONJG( V( 3, M ) )
  130      CONTINUE
*
*          ==== End of near-the-diagonal bulge chase. ====
*
  140    CONTINUE
*
*        ==== Use U (if accumulated) to update far-from-diagonal
*        .    entries in H.  If required, use U to update Z as
*        .    well. ====
*
         IF( ACCUM ) THEN
            IF( WANTT ) THEN
               JTOP = 1
               JBOT = N
            ELSE
               JTOP = KTOP
               JBOT = KBOT
            END IF
            IF( ( .NOT.BLK22 ) .OR. ( INCOL.LT.KTOP ) .OR.
```

```
     $          ( NDCOL.GT.KBOT ) .OR. ( NS.LE.2 ) ) THEN
*
*              ==== Updates not exploiting the 2-by-2 block
*              .    structure of U.  K1 and NU keep track of
*              .    the location and size of U in the special
*              .    cases of introducing bulges and chasing
*              .    bulges off the bottom.  In these special
*              .    cases and in case the number of shifts
*              .    is NS = 2, there is no 2-by-2 block
*              .    structure to exploit.  ====
*
               K1 = MAX( 1, KTOP-INCOL )
               NU = ( KDU-MAX( 0, NDCOL-KBOT ) ) - K1 + 1
*
*              ==== Horizontal Multiply ====
*
               DO 150 JCOL = MIN( NDCOL, KBOT ) + 1, JBOT, NH
                  JLEN = MIN( NH, JBOT-JCOL+1 )
                  CALL ZGEMM( 'C', 'N', NU, JLEN, NU, ONE, U( K1, K1 ),
     $                        LDU, H( INCOL+K1, JCOL ), LDH, ZERO, WH,
     $                        LDWH )
                  CALL ZLACPY( 'ALL', NU, JLEN, WH, LDWH,
     $                         H( INCOL+K1, JCOL ), LDH )
  150          CONTINUE
*
*              ==== Vertical multiply ====
*
               DO 160 JROW = JTOP, MAX( KTOP, INCOL ) - 1, NV
                  JLEN = MIN( NV, MAX( KTOP, INCOL )-JROW )
                  CALL ZGEMM( 'N', 'N', JLEN, NU, NU, ONE,
     $                        H( JROW, INCOL+K1 ), LDH, U( K1, K1 ),
     $                        LDU, ZERO, WV, LDWV )
                  CALL ZLACPY( 'ALL', JLEN, NU, WV, LDWV,
     $                         H( JROW, INCOL+K1 ), LDH )
  160          CONTINUE
*
*              ==== Z multiply (also vertical) ====
*
               IF( WANTZ ) THEN
                  DO 170 JROW = ILOZ, IHIZ, NV
                     JLEN = MIN( NV, IHIZ-JROW+1 )
                     CALL ZGEMM( 'N', 'N', JLEN, NU, NU, ONE,
     $                           Z( JROW, INCOL+K1 ), LDZ, U( K1, K1 ),
     $                           LDU, ZERO, WV, LDWV )
                     CALL ZLACPY( 'ALL', JLEN, NU, WV, LDWV,
     $                            Z( JROW, INCOL+K1 ), LDZ )
  170             CONTINUE
               END IF
            ELSE
*
```

```
*              ==== Updates exploiting U's 2-by-2 block structure.
*              .    (I2, I4, J2, J4 are the last rows and columns
*              .    of the blocks.) ====
*
               I2 = ( KDU+1 ) / 2
               I4 = KDU
               J2 = I4 - I2
               J4 = KDU
*
*              ==== KZS and KNZ deal with the band of zeros
*              .    along the diagonal of one of the triangular
*              .    blocks. ====
*
               KZS = ( J4-J2 ) - ( NS+1 )
               KNZ = NS + 1
*
*              ==== Horizontal multiply ====
*
               DO 180 JCOL = MIN( NDCOL, KBOT ) + 1, JBOT, NH
                  JLEN = MIN( NH, JBOT-JCOL+1 )
*
*                 ==== Copy bottom of H to top+KZS of scratch ====
*                  (The first KZS rows get multiplied by zero.) ====
*
                  CALL ZLACPY( 'ALL', KNZ, JLEN, H( INCOL+1+J2, JCOL ),
     $                         LDH, WH( KZS+1, 1 ), LDWH )
*
*                 ==== Multiply by U21**H ====
*
                  CALL ZLASET( 'ALL', KZS, JLEN, ZERO, ZERO, WH, LDWH )
                  CALL ZTRMM( 'L', 'U', 'C', 'N', KNZ, JLEN, ONE,
     $                        U( J2+1, 1+KZS ), LDU, WH( KZS+1, 1 ),
     $                        LDWH )
*
*                 ==== Multiply top of H by U11**H ====
*
                  CALL ZGEMM( 'C', 'N', I2, JLEN, J2, ONE, U, LDU,
     $                        H( INCOL+1, JCOL ), LDH, ONE, WH, LDWH )
*
*                 ==== Copy top of H to bottom of WH ====
*
                  CALL ZLACPY( 'ALL', J2, JLEN, H( INCOL+1, JCOL ), LDH,
     $                         WH( I2+1, 1 ), LDWH )
*
*                 ==== Multiply by U21**H ====
*
                  CALL ZTRMM( 'L', 'L', 'C', 'N', J2, JLEN, ONE,
     $                        U( 1, I2+1 ), LDU, WH( I2+1, 1 ), LDWH )
*
*                 ==== Multiply by U22 ====
```

```
*
                 CALL ZGEMM( 'C', 'N', I4-I2, JLEN, J4-J2, ONE,
     $                       U( J2+1, I2+1 ), LDU,
     $                       H( INCOL+1+J2, JCOL ), LDH, ONE,
     $                       WH( I2+1, 1 ), LDWH )
*
*                ==== Copy it back ====
*
                 CALL ZLACPY( 'ALL', KDU, JLEN, WH, LDWH,
     $                        H( INCOL+1, JCOL ), LDH )
  180         CONTINUE
*
              ==== Vertical multiply ====
*
              DO 190 JROW = JTOP, MAX( INCOL, KTOP ) - 1, NV
                 JLEN = MIN( NV, MAX( INCOL, KTOP )-JROW )
*
*                ==== Copy right of H to scratch (the first KZS
*                .    columns get multiplied by zero) ====
*
                 CALL ZLACPY( 'ALL', JLEN, KNZ, H( JROW, INCOL+1+J2 ),
     $                        LDH, WV( 1, 1+KZS ), LDWV )
*
*                ==== Multiply by U21 ====
*
                 CALL ZLASET( 'ALL', JLEN, KZS, ZERO, ZERO, WV, LDWV )
                 CALL ZTRMM( 'R', 'U', 'N', 'N', JLEN, KNZ, ONE,
     $                       U( J2+1, 1+KZS ), LDU, WV( 1, 1+KZS ),
     $                       LDWV )
*
*                ==== Multiply by U11 ====
*
                 CALL ZGEMM( 'N', 'N', JLEN, I2, J2, ONE,
     $                       H( JROW, INCOL+1 ), LDH, U, LDU, ONE, WV,
     $                       LDWV )
*
*                ==== Copy left of H to right of scratch ====
*
                 CALL ZLACPY( 'ALL', JLEN, J2, H( JROW, INCOL+1 ), LDH,
     $                        WV( 1, 1+I2 ), LDWV )
*
*                ==== Multiply by U21 ====
*
                 CALL ZTRMM( 'R', 'L', 'N', 'N', JLEN, I4-I2, ONE,
     $                       U( 1, I2+1 ), LDU, WV( 1, 1+I2 ), LDWV )
*
*                ==== Multiply by U22 ====
*
                 CALL ZGEMM( 'N', 'N', JLEN, I4-I2, J4-J2, ONE,
     $                       H( JROW, INCOL+1+J2 ), LDH,
```

```
     $                       U( J2+1, I2+1 ), LDU, ONE, WV( 1, 1+I2 ),
     $                       LDWV )
*
*                 ==== Copy it back ====
*
                  CALL ZLACPY( 'ALL', JLEN, KDU, WV, LDWV,
     $                        H( JROW, INCOL+1 ), LDH )
  190          CONTINUE
*
               ==== Multiply Z (also vertical) ====
*
               IF( WANTZ ) THEN
                  DO 200 JROW = ILOZ, IHIZ, NV
                     JLEN = MIN( NV, IHIZ-JROW+1 )
*
                     ==== Copy right of Z to left of scratch (first
*                    .     KZS columns get multiplied by zero) ====
*
                     CALL ZLACPY( 'ALL', JLEN, KNZ,
     $                            Z( JROW, INCOL+1+J2 ), LDZ,
     $                            WV( 1, 1+KZS ), LDWV )
*
                     ==== Multiply by U12 ====
*
                     CALL ZLASET( 'ALL', JLEN, KZS, ZERO, ZERO, WV,
     $                            LDWV )
                     CALL ZTRMM( 'R', 'U', 'N', 'N', JLEN, KNZ, ONE,
     $                           U( J2+1, 1+KZS ), LDU, WV( 1, 1+KZS ),
     $                           LDWV )
*
                     ==== Multiply by U11 ====
*
                     CALL ZGEMM( 'N', 'N', JLEN, I2, J2, ONE,
     $                           Z( JROW, INCOL+1 ), LDZ, U, LDU, ONE,
     $                           WV, LDWV )
*
                     ==== Copy left of Z to right of scratch ====
*
                     CALL ZLACPY( 'ALL', JLEN, J2, Z( JROW, INCOL+1 ),
     $                            LDZ, WV( 1, 1+I2 ), LDWV )
*
                     ==== Multiply by U21 ====
*
                     CALL ZTRMM( 'R', 'L', 'N', 'N', JLEN, I4-I2, ONE,
     $                           U( 1, I2+1 ), LDU, WV( 1, 1+I2 ),
     $                           LDWV )
*
                     ==== Multiply by U22 ====
*
                     CALL ZGEMM( 'N', 'N', JLEN, I4-I2, J4-J2, ONE,
```

```
     $                          Z( JROW, INCOL+1+J2 ), LDZ,
     $                          U( J2+1, I2+1 ), LDU, ONE,
     $                          WV( 1, 1+I2 ), LDWV )
*
*                    ==== Copy the result back to Z ====
*
                     CALL ZLACPY( 'ALL', JLEN, KDU, WV, LDWV,
     $                            Z( JROW, INCOL+1 ), LDZ )
  200             CONTINUE
               END IF
            END IF
         END IF
  210 CONTINUE
*
*     ==== End of ZLAQR5 ====
*
      END
```

———-

90.228.4 lisp code

— **zlaqr5.lisp** —

```
(let* ((zero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (one (f2cl-lib:cmplx 1.0d0 0.0d0))
       (rzero 0.0d0)
       (rone 1.0d0))
  (declare (type (f2cl-lib:complex16) zero)
           (type (f2cl-lib:complex16) one)
           (type (double-float 0.0d0 0.0d0) rzero)
           (type (double-float 1.0d0 1.0d0) rone)
           (ignorable zero one rzero rone))
  (defun zlaqr5
         (wantt wantz kacc22 n ktop kbot nshfts s h ldh iloz ihiz z ldz v ldv u
          ldu nv wv ldwv nh wh ldwh)
    (declare (type (array f2cl-lib:complex16 (*)) wh wv u v z h s)
             (type (f2cl-lib:integer4) ldwh nh ldwv nv ldu ldv ldz ihiz iloz
              ldh nshfts kbot ktop n kacc22)
             (type f2cl-lib:logical wantz wantt))
    (f2cl-lib:with-multi-array-data
        ((s f2cl-lib:complex16 s-%data% s-%offset%)
         (h f2cl-lib:complex16 h-%data% h-%offset%)
         (z f2cl-lib:complex16 z-%data% z-%offset%)
         (v f2cl-lib:complex16 v-%data% v-%offset%)
         (u f2cl-lib:complex16 u-%data% u-%offset%)
         (wv f2cl-lib:complex16 wv-%data% wv-%offset%)
```

```
    (wh f2cl-lib:complex16 wh-%data% wh-%offset%))
  (labels ((cabs1 (cdum)
             (+ (abs (f2cl-lib:dble cdum)) (abs (f2cl-lib:dimag cdum)))))
    (declare
     (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
      cabs1))
    (prog ((vt (make-array 3 :element-type 'f2cl-lib:complex16))
           (accum nil) (blk22 nil) (bmp22 nil) (i2 0) (i4 0) (incol 0)
           (j 0) (j2 0) (j4 0) (jbot 0) (jcol 0) (jlen 0) (jrow 0) (jtop 0)
           (k 0) (k1 0) (kdu 0) (kms 0) (knz 0) (krcol 0) (kzs 0) (m 0)
           (m22 0) (mbot 0) (mend 0) (mstart 0) (mtop 0) (nbmps 0)
           (ndcol 0) (ns 0) (nu 0) (h11 0.0d0) (h12 0.0d0) (h21 0.0d0)
           (h22 0.0d0) (safmax 0.0d0) (safmin 0.0d0) (scl 0.0d0)
           (smlnum 0.0d0) (tst1 0.0d0) (tst2 0.0d0) (ulp 0.0d0)
           (alpha #C(0.0d0 0.0d0)) (beta #C(0.0d0 0.0d0))
           (cdum #C(0.0d0 0.0d0)) (refsum #C(0.0d0 0.0d0)))
     (declare (type (array f2cl-lib:complex16 (3)) vt)
              (type f2cl-lib:logical accum blk22 bmp22)
              (type (f2cl-lib:integer4) i2 i4 incol j j2 j4 jbot jcol jlen
               jrow jtop k k1 kdu kms knz krcol kzs m m22 mbot mend mstart
               mtop nbmps ndcol ns nu)
              (type (double-float) h11 h12 h21 h22 safmax safmin scl
               smlnum tst1 tst2 ulp)
              (type (f2cl-lib:complex16) alpha beta cdum refsum))
     (if (< nshfts 2)
         (go end_label))
     (if (>= ktop kbot)
         (go end_label))
     (setf ns (f2cl-lib:int-sub nshfts (mod nshfts 2)))
     (setf safmin (dlamch "SAFE MINIMUM"))
     (setf safmax (/ rone safmin))
     (multiple-value-bind (var-0 var-1)
         (dlabad safmin safmax)
       (declare (ignore))
       (when var-0 (setf safmin var-0))
       (when var-1 (setf safmax var-1)))
     (setf ulp (dlamch "PRECISION"))
     (setf smlnum (* safmin (/ (f2cl-lib:dble n) ulp)))
     (setf accum (or (= kacc22 1) (= kacc22 2)))
     (setf blk22 (and (> ns 2) (= kacc22 2)))
     (if (<= (f2cl-lib:int-add ktop 2) kbot)
         (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add ktop 2) ktop)
                              ((1 ldh) (1 *)) h-%offset%)
               zero))
     (setf nbmps (the f2cl-lib:integer4 (truncate ns 2)))
     (setf kdu (f2cl-lib:int-sub (f2cl-lib:int-mul 6 nbmps) 3))
     (f2cl-lib:fdo (incol
                    (f2cl-lib:int-add
                     (f2cl-lib:int-mul 3
                                       (f2cl-lib:int-add 1
```

```
                                                    (f2cl-lib:int-sub
                                                     nbmps)))
                 ktop (f2cl-lib:int-sub 1))
                (f2cl-lib:int-add incol
                                  (f2cl-lib:int-add
                                   (f2cl-lib:int-mul 3 nbmps)
                                   (f2cl-lib:int-sub 2))))
               ((> incol (f2cl-lib:int-add kbot (f2cl-lib:int-sub 2)))
                nil)
(tagbody
  (setf ndcol (f2cl-lib:int-add incol kdu))
  (if accum
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
          (zlaset "ALL" kdu kdu zero one u ldu)
        (declare (ignore var-0 var-5))
        (when var-1 (setf kdu var-1))
        (when var-2 (setf kdu var-2))
        (when var-3 (setf zero var-3))
        (when var-4 (setf one var-4))
        (when var-6 (setf ldu var-6))))
  (f2cl-lib:fdo (krcol incol (f2cl-lib:int-add krcol 1))
                ((> krcol
                    (min
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add incol
                                            (f2cl-lib:int-mul 3
                                                              nbmps)
                                            (f2cl-lib:int-sub 3)))
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add kbot
                                            (f2cl-lib:int-sub
                                             2)))))
                 nil)
    (tagbody
      (setf mtop
              (max 1
                   (+
                    (the f2cl-lib:integer4
                         (truncate (+ (- ktop 1 krcol) 2) 3))
                    1)))
      (setf mbot
              (min nbmps
                   (the f2cl-lib:integer4
                        (truncate (- kbot krcol) 3))))
      (setf m22 (f2cl-lib:int-add mbot 1))
      (setf bmp22
              (and (< mbot nbmps)
                   (=
                    (f2cl-lib:int-add krcol
```

```
                                (f2cl-lib:int-mul 3
                                                  (f2cl-lib:int-sub
                                                   m22 1)))
              (f2cl-lib:int-sub kbot 2))))
(f2cl-lib:fdo (m mtop (f2cl-lib:int-add m 1))
              ((> m mbot) nil)
  (tagbody
    (setf k
            (f2cl-lib:int-add krcol
                              (f2cl-lib:int-mul 3
                                                (f2cl-lib:int-sub
                                                 m 1))))
    (cond
     ((= k (f2cl-lib:int-add ktop (f2cl-lib:int-sub 1)))
      (zlaqr1 3
       (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                             (ktop ktop) ((1 ldh) (1 *))
                             h-%offset%)
       ldh
       (f2cl-lib:fref s-%data%
                      ((f2cl-lib:int-sub
                        (f2cl-lib:int-mul 2 m) 1))
                      ((1 *)) s-%offset%)
       (f2cl-lib:fref s-%data% ((f2cl-lib:int-mul 2 m))
                      ((1 *)) s-%offset%)
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             (1 m) ((1 ldv) (1 *))
                             v-%offset%))
      (setf alpha
              (f2cl-lib:fref v-%data% (1 m) ((1 ldv) (1 *))
                             v-%offset%))
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (zlarfg 3 alpha
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                                 (2 m) ((1 ldv) (1 *))
                                 v-%offset%)
           1
           (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                                 (1 m) ((1 ldv) (1 *))
                                 v-%offset%))
        (declare (ignore var-0 var-2 var-3 var-4))
        (when var-1 (setf alpha var-1))))
     (t
      (setf beta
              (f2cl-lib:fref h-%data%
                             ((f2cl-lib:int-add k 1) k)
                             ((1 ldh) (1 *)) h-%offset%))
      (setf (f2cl-lib:fref v-%data% (2 m) ((1 ldv) (1 *))
                           v-%offset%)
              (f2cl-lib:fref h-%data%
```

```
                        ((f2cl-lib:int-add k 2) k)
                        ((1 ldh) (1 *)) h-%offset%))
(setf (f2cl-lib:fref v-%data% (3 m) ((1 ldv) (1 *))
                      v-%offset%)
        (f2cl-lib:fref h-%data%
                        ((f2cl-lib:int-add k 3) k)
                        ((1 ldh) (1 *)) h-%offset%))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (zlarfg 3 beta
     (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                           (2 m) ((1 ldv) (1 *))
                           v-%offset%)
     1
     (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                           (1 m) ((1 ldv) (1 *))
                           v-%offset%))
  (declare (ignore var-0 var-2 var-3 var-4))
  (when var-1 (setf beta var-1)))
(cond
 ((or
   (/=
    (f2cl-lib:fref h ((f2cl-lib:int-add k 3) k)
                   ((1 ldh) (1 *)))
    zero)
   (/=
    (f2cl-lib:fref h
                   ((f2cl-lib:int-add k 3)
                    (f2cl-lib:int-add k 1))
                   ((1 ldh) (1 *)))
    zero)
   (=
    (f2cl-lib:fref h
                   ((f2cl-lib:int-add k 3)
                    (f2cl-lib:int-add k 2))
                   ((1 ldh) (1 *)))
    zero))
  (setf (f2cl-lib:fref h-%data%
                        ((f2cl-lib:int-add k 1) k)
                        ((1 ldh) (1 *)) h-%offset%)
          beta)
  (setf (f2cl-lib:fref h-%data%
                        ((f2cl-lib:int-add k 2) k)
                        ((1 ldh) (1 *)) h-%offset%)
          zero)
  (setf (f2cl-lib:fref h-%data%
                        ((f2cl-lib:int-add k 3) k)
                        ((1 ldh) (1 *)) h-%offset%)
          zero))
 (t
  (zlaqr1 3
```

```
 (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                       ((+ k 1)
                        (f2cl-lib:int-add k 1))
                       ((1 ldh) (1 *)) h-%offset%)
 ldh
 (f2cl-lib:fref s-%data%
                ((f2cl-lib:int-sub
                  (f2cl-lib:int-mul 2 m) 1))
                ((1 *)) s-%offset%)
 (f2cl-lib:fref s-%data% ((f2cl-lib:int-mul 2 m))
                ((1 *)) s-%offset%)
 vt)
(setf alpha (f2cl-lib:fref vt (1) ((1 3))))
(multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
    (zlarfg 3 alpha
     (f2cl-lib:array-slice vt f2cl-lib:complex16 (2)
                           ((1 3)))
     1
     (f2cl-lib:array-slice vt f2cl-lib:complex16 (1)
                           ((1 3))))
  (declare (ignore var-0 var-2 var-3 var-4))
  (when var-1 (setf alpha var-1)))
(setf refsum
        (*
         (f2cl-lib:dconjg
          (f2cl-lib:fref vt (1) ((1 3))))
         (+
          (f2cl-lib:fref h-%data%
                         ((f2cl-lib:int-add k 1) k)
                         ((1 ldh) (1 *)) h-%offset%)
          (*
           (f2cl-lib:dconjg
            (f2cl-lib:fref vt (2) ((1 3))))
           (f2cl-lib:fref h-%data%
                          ((f2cl-lib:int-add k 2) k)
                          ((1 ldh) (1 *))
                          h-%offset%)))))
(cond
 ((>
   (+
    (cabs1
     (+
      (f2cl-lib:fref h ((f2cl-lib:int-add k 2) k)
                     ((1 ldh) (1 *)))
      (* -1 refsum (f2cl-lib:fref vt (2) ((1 3))))))
    (cabs1
     (* refsum (f2cl-lib:fref vt (3) ((1 3))))))
   (* ulp
      (+
       (cabs1
```

```
                    (f2cl-lib:fref h (k k) ((1 ldh) (1 *))))
                   (cabs1
                    (f2cl-lib:fref h
                                   ((f2cl-lib:int-add k 1)
                                    (f2cl-lib:int-add k 1))
                                   ((1 ldh) (1 *))))
                   (cabs1
                    (f2cl-lib:fref h
                                   ((f2cl-lib:int-add k 2)
                                    (f2cl-lib:int-add k 2))
                                   ((1 ldh) (1 *)))))))
            (setf (f2cl-lib:fref h-%data%
                                 ((f2cl-lib:int-add k 1) k)
                                 ((1 ldh) (1 *)) h-%offset%)
                    beta)
            (setf (f2cl-lib:fref h-%data%
                                 ((f2cl-lib:int-add k 2) k)
                                 ((1 ldh) (1 *)) h-%offset%)
                    zero)
            (setf (f2cl-lib:fref h-%data%
                                 ((f2cl-lib:int-add k 3) k)
                                 ((1 ldh) (1 *)) h-%offset%)
                    zero))
           (t
            (setf (f2cl-lib:fref h-%data%
                                 ((f2cl-lib:int-add k 1) k)
                                 ((1 ldh) (1 *)) h-%offset%)
                    (-
                     (f2cl-lib:fref h-%data%
                                    ((f2cl-lib:int-add k 1) k)
                                    ((1 ldh) (1 *)) h-%offset%)
                     refsum))
            (setf (f2cl-lib:fref h-%data%
                                 ((f2cl-lib:int-add k 2) k)
                                 ((1 ldh) (1 *)) h-%offset%)
                    zero)
            (setf (f2cl-lib:fref h-%data%
                                 ((f2cl-lib:int-add k 3) k)
                                 ((1 ldh) (1 *)) h-%offset%)
                    zero)
            (setf (f2cl-lib:fref v-%data% (1 m) ((1 ldv) (1 *))
                                 v-%offset%)
                    (f2cl-lib:fref vt (1) ((1 3))))
            (setf (f2cl-lib:fref v-%data% (2 m) ((1 ldv) (1 *))
                                 v-%offset%)
                    (f2cl-lib:fref vt (2) ((1 3))))
            (setf (f2cl-lib:fref v-%data% (3 m) ((1 ldv) (1 *))
                                 v-%offset%)
                    (f2cl-lib:fref vt (3) ((1 3)))))))))
label10))
```

```
(setf k
        (f2cl-lib:int-add krcol
                          (f2cl-lib:int-mul 3
                                            (f2cl-lib:int-sub
                                             m22 1))))
(cond
 (bmp22
  (cond
   ((= k (f2cl-lib:int-add ktop (f2cl-lib:int-sub 1)))
    (zlaqr1 2
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                           ((+ k 1) (f2cl-lib:int-add k 1))
                           ((1 ldh) (1 *)) h-%offset%)
     ldh
     (f2cl-lib:fref s-%data%
                    ((f2cl-lib:int-sub
                      (f2cl-lib:int-mul 2 m22) 1))
                    ((1 *)) s-%offset%)
     (f2cl-lib:fref s-%data% ((f2cl-lib:int-mul 2 m22))
                    ((1 *)) s-%offset%)
     (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                           (1 m22) ((1 ldv) (1 *))
                           v-%offset%))
    (setf beta
            (f2cl-lib:fref v-%data% (1 m22) ((1 ldv) (1 *))
                           v-%offset%))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zlarfg 2 beta
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                               (2 m22) ((1 ldv) (1 *))
                               v-%offset%)
         1
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                               (1 m22) ((1 ldv) (1 *))
                               v-%offset%))
      (declare (ignore var-0 var-2 var-3 var-4))
      (when var-1 (setf beta var-1))))
   (t
    (setf beta
            (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-add k 1) k)
                           ((1 ldh) (1 *)) h-%offset%))
    (setf (f2cl-lib:fref v-%data% (2 m22) ((1 ldv) (1 *))
                         v-%offset%)
            (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-add k 2) k)
                           ((1 ldh) (1 *)) h-%offset%))
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zlarfg 2 beta
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
```

```
                                (2 m22) ((1 ldv) (1 *))
                                v-%offset%)
         1
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                                (1 m22) ((1 ldv) (1 *))
                                v-%offset%))
      (declare (ignore var-0 var-2 var-3 var-4))
      (when var-1 (setf beta var-1)))
    (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add k 1) k)
                         ((1 ldh) (1 *)) h-%offset%)
            beta)
    (setf (f2cl-lib:fref h-%data% ((f2cl-lib:int-add k 2) k)
                         ((1 ldh) (1 *)) h-%offset%)
            zero)))))
(cond
 (accum
  (setf jbot
          (min (the f2cl-lib:integer4 ndcol)
               (the f2cl-lib:integer4 kbot))))
 (wantt (setf jbot n)) (t (setf jbot kbot)))
(f2cl-lib:fdo (j
               (max (the f2cl-lib:integer4 ktop)
                    (the f2cl-lib:integer4 krcol))
               (f2cl-lib:int-add j 1))
              ((> j jbot) nil)
  (tagbody
    (setf mend
            (min mbot
                 (the f2cl-lib:integer4
                      (truncate (+ (- j krcol) 2) 3))))
    (f2cl-lib:fdo (m mtop (f2cl-lib:int-add m 1))
                  ((> m mend) nil)
      (tagbody
        (setf k
                (f2cl-lib:int-add krcol
                                  (f2cl-lib:int-mul 3
                                                    (f2cl-lib:int-sub
                                                     m 1))))
        (setf refsum
                (*
                 (f2cl-lib:dconjg
                  (f2cl-lib:fref v-%data% (1 m)
                                 ((1 ldv) (1 *)) v-%offset%))
                 (+
                  (f2cl-lib:fref h-%data%
                                 ((f2cl-lib:int-add k 1) j)
                                 ((1 ldh) (1 *)) h-%offset%)
                  (*
                   (f2cl-lib:dconjg
                    (f2cl-lib:fref v-%data% (2 m)
```

```
                                  ((1 ldv) (1 *))
                                  v-%offset%))
                     (f2cl-lib:fref h-%data%
                                  ((f2cl-lib:int-add k 2) j)
                                  ((1 ldh) (1 *))
                                  h-%offset%))
                    (*
                     (f2cl-lib:dconjg
                      (f2cl-lib:fref v-%data% (3 m)
                                  ((1 ldv) (1 *))
                                  v-%offset%))
                     (f2cl-lib:fref h-%data%
                                  ((f2cl-lib:int-add k 3) j)
                                  ((1 ldh) (1 *))
                                  h-%offset%)))))
            (setf (f2cl-lib:fref h-%data%
                                 ((f2cl-lib:int-add k 1) j)
                                 ((1 ldh) (1 *)) h-%offset%)
                  (-
                   (f2cl-lib:fref h-%data%
                                  ((f2cl-lib:int-add k 1) j)
                                  ((1 ldh) (1 *)) h-%offset%)
                   refsum))
            (setf (f2cl-lib:fref h-%data%
                                 ((f2cl-lib:int-add k 2) j)
                                 ((1 ldh) (1 *)) h-%offset%)
                  (-
                   (f2cl-lib:fref h-%data%
                                  ((f2cl-lib:int-add k 2) j)
                                  ((1 ldh) (1 *)) h-%offset%)
                   (* refsum
                      (f2cl-lib:fref v-%data% (2 m)
                                  ((1 ldv) (1 *))
                                  v-%offset%))))
            (setf (f2cl-lib:fref h-%data%
                                 ((f2cl-lib:int-add k 3) j)
                                 ((1 ldh) (1 *)) h-%offset%)
                  (-
                   (f2cl-lib:fref h-%data%
                                  ((f2cl-lib:int-add k 3) j)
                                  ((1 ldh) (1 *)) h-%offset%)
                   (* refsum
                      (f2cl-lib:fref v-%data% (3 m)
                                  ((1 ldv) (1 *))
                                  v-%offset%))))
        label20))
    label30))
(cond
 (bmp22
  (setf k
```

```
            (f2cl-lib:int-add krcol
                              (f2cl-lib:int-mul 3
                                                (f2cl-lib:int-sub
                                                 m22 1))))
  (f2cl-lib:fdo (j
                 (max
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add k 1))
                  (the f2cl-lib:integer4 ktop))
                 (f2cl-lib:int-add j 1))
                ((> j jbot) nil)
    (tagbody
      (setf refsum
              (*
               (f2cl-lib:dconjg
                (f2cl-lib:fref v-%data% (1 m22)
                               ((1 ldv) (1 *)) v-%offset%))
               (+
                (f2cl-lib:fref h-%data%
                               ((f2cl-lib:int-add k 1) j)
                               ((1 ldh) (1 *)) h-%offset%)
                (*
                 (f2cl-lib:dconjg
                  (f2cl-lib:fref v-%data% (2 m22)
                                 ((1 ldv) (1 *)) v-%offset%))
                 (f2cl-lib:fref h-%data%
                                ((f2cl-lib:int-add k 2) j)
                                ((1 ldh) (1 *))
                                h-%offset%)))))
      (setf (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-add k 1) j)
                           ((1 ldh) (1 *)) h-%offset%)
              (-
               (f2cl-lib:fref h-%data%
                              ((f2cl-lib:int-add k 1) j)
                              ((1 ldh) (1 *)) h-%offset%)
               refsum))
      (setf (f2cl-lib:fref h-%data%
                           ((f2cl-lib:int-add k 2) j)
                           ((1 ldh) (1 *)) h-%offset%)
              (-
               (f2cl-lib:fref h-%data%
                              ((f2cl-lib:int-add k 2) j)
                              ((1 ldh) (1 *)) h-%offset%)
               (* refsum
                  (f2cl-lib:fref v-%data% (2 m22)
                                 ((1 ldv) (1 *))
                                 v-%offset%))))
    label40))))
(cond
```

```
 (accum
  (setf jtop
          (max (the f2cl-lib:integer4 ktop)
               (the f2cl-lib:integer4 incol))))
 (wantt (setf jtop 1)) (t (setf jtop ktop)))
(f2cl-lib:fdo (m mtop (f2cl-lib:int-add m 1))
              ((> m mbot) nil)
  (tagbody
    (cond
     ((/= (f2cl-lib:fref v (1 m) ((1 ldv) (1 *))) zero)
      (setf k
              (f2cl-lib:int-add krcol
                                (f2cl-lib:int-mul 3
                                                  (f2cl-lib:int-sub
                                                   m 1))))
      (f2cl-lib:fdo (j jtop (f2cl-lib:int-add j 1))
                    ((> j
                        (min (the f2cl-lib:integer4 kbot)
                             (the f2cl-lib:integer4
                                  (f2cl-lib:int-add k 3))))
                     nil)
        (tagbody
          (setf refsum
                  (*
                   (f2cl-lib:fref v-%data% (1 m)
                                  ((1 ldv) (1 *)) v-%offset%)
                   (+
                    (f2cl-lib:fref h-%data%
                                   (j (f2cl-lib:int-add k 1))
                                   ((1 ldh) (1 *))
                                   h-%offset%)
                    (*
                     (f2cl-lib:fref v-%data% (2 m)
                                    ((1 ldv) (1 *))
                                    v-%offset%)
                     (f2cl-lib:fref h-%data%
                                    (j
                                     (f2cl-lib:int-add k 2))
                                    ((1 ldh) (1 *))
                                    h-%offset%))
                    (*
                     (f2cl-lib:fref v-%data% (3 m)
                                    ((1 ldv) (1 *))
                                    v-%offset%)
                     (f2cl-lib:fref h-%data%
                                    (j
                                     (f2cl-lib:int-add k 3))
                                    ((1 ldh) (1 *))
                                    h-%offset%)))))
          (setf (f2cl-lib:fref h-%data%
```

```
                                (j (f2cl-lib:int-add k 1))
                                ((1 ldh) (1 *)) h-%offset%)
                        (-
                         (f2cl-lib:fref h-%data%
                                        (j (f2cl-lib:int-add k 1))
                                        ((1 ldh) (1 *)) h-%offset%)
                         refsum))
                (setf (f2cl-lib:fref h-%data%
                                (j (f2cl-lib:int-add k 2))
                                ((1 ldh) (1 *)) h-%offset%)
                        (-
                         (f2cl-lib:fref h-%data%
                                        (j (f2cl-lib:int-add k 2))
                                        ((1 ldh) (1 *)) h-%offset%)
                         (* refsum
                            (f2cl-lib:dconjg
                             (f2cl-lib:fref v-%data% (2 m)
                                            ((1 ldv) (1 *))
                                            v-%offset%)))))
                (setf (f2cl-lib:fref h-%data%
                                (j (f2cl-lib:int-add k 3))
                                ((1 ldh) (1 *)) h-%offset%)
                        (-
                         (f2cl-lib:fref h-%data%
                                        (j (f2cl-lib:int-add k 3))
                                        ((1 ldh) (1 *)) h-%offset%)
                         (* refsum
                            (f2cl-lib:dconjg
                             (f2cl-lib:fref v-%data% (3 m)
                                            ((1 ldv) (1 *))
                                            v-%offset%)))))
           label50))
        (cond
         (accum (setf kms (f2cl-lib:int-sub k incol))
          (f2cl-lib:fdo (j
                         (max (the f2cl-lib:integer4 1)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add ktop
                                                     (f2cl-lib:int-sub
                                                      incol))))
                         (f2cl-lib:int-add j 1))
                        ((> j kdu) nil)
                (tagbody
                  (setf refsum
                          (*
                           (f2cl-lib:fref v-%data% (1 m)
                                          ((1 ldv) (1 *))
                                          v-%offset%)
                           (+
                            (f2cl-lib:fref u-%data%
```

```
                                (j
                                 (f2cl-lib:int-add kms
                                                   1))
                                ((1 ldu) (1 *))
                                u-%offset%)
                         (*
                          (f2cl-lib:fref v-%data% (2 m)
                                         ((1 ldv) (1 *))
                                         v-%offset%)
                          (f2cl-lib:fref u-%data%
                                         (j
                                          (f2cl-lib:int-add kms
                                                            2))
                                         ((1 ldu) (1 *))
                                         u-%offset%))
                         (*
                          (f2cl-lib:fref v-%data% (3 m)
                                         ((1 ldv) (1 *))
                                         v-%offset%)
                          (f2cl-lib:fref u-%data%
                                         (j
                                          (f2cl-lib:int-add kms
                                                            3))
                                         ((1 ldu) (1 *))
                                         u-%offset%)))))
(setf (f2cl-lib:fref u-%data%
                     (j (f2cl-lib:int-add kms 1))
                     ((1 ldu) (1 *)) u-%offset%)
        (-
         (f2cl-lib:fref u-%data%
                        (j
                         (f2cl-lib:int-add kms
                                           1))
                        ((1 ldu) (1 *))
                        u-%offset%)
         refsum))
(setf (f2cl-lib:fref u-%data%
                     (j (f2cl-lib:int-add kms 2))
                     ((1 ldu) (1 *)) u-%offset%)
        (-
         (f2cl-lib:fref u-%data%
                        (j
                         (f2cl-lib:int-add kms
                                           2))
                        ((1 ldu) (1 *))
                        u-%offset%)
         (* refsum
            (f2cl-lib:dconjg
             (f2cl-lib:fref v-%data% (2 m)
                            ((1 ldv) (1 *))
```

```
                                    v-%offset%)))))
      (setf (f2cl-lib:fref u-%data%
                           (j (f2cl-lib:int-add kms 3))
                           ((1 ldu) (1 *)) u-%offset%)
              (-
               (f2cl-lib:fref u-%data%
                              (j
                               (f2cl-lib:int-add kms
                                                 3))
                              ((1 ldu) (1 *))
                              u-%offset%)
               (* refsum
                  (f2cl-lib:dconjg
                   (f2cl-lib:fref v-%data% (3 m)
                                  ((1 ldv) (1 *))
                                  v-%offset%)))))
    label60)))
(wantz
 (f2cl-lib:fdo (j iloz (f2cl-lib:int-add j 1))
               ((> j ihiz) nil)
   (tagbody
     (setf refsum
             (*
              (f2cl-lib:fref v-%data% (1 m)
                             ((1 ldv) (1 *))
                             v-%offset%)
              (+
               (f2cl-lib:fref z-%data%
                              (j
                               (f2cl-lib:int-add k 1))
                              ((1 ldz) (1 *))
                              z-%offset%)
               (*
                (f2cl-lib:fref v-%data% (2 m)
                               ((1 ldv) (1 *))
                               v-%offset%)
                (f2cl-lib:fref z-%data%
                               (j
                                (f2cl-lib:int-add k
                                                  2))
                               ((1 ldz) (1 *))
                               z-%offset%))
               (*
                (f2cl-lib:fref v-%data% (3 m)
                               ((1 ldv) (1 *))
                               v-%offset%)
                (f2cl-lib:fref z-%data%
                               (j
                                (f2cl-lib:int-add k
                                                  3))
```

```
                                        ((1 ldz) (1 *))
                                        z-%offset%)))))
                    (setf (f2cl-lib:fref z-%data%
                                         (j (f2cl-lib:int-add k 1))
                                         ((1 ldz) (1 *)) z-%offset%)
                          (-
                           (f2cl-lib:fref z-%data%
                                          (j
                                           (f2cl-lib:int-add k 1))
                                          ((1 ldz) (1 *))
                                          z-%offset%)
                           refsum))
                    (setf (f2cl-lib:fref z-%data%
                                         (j (f2cl-lib:int-add k 2))
                                         ((1 ldz) (1 *)) z-%offset%)
                          (-
                           (f2cl-lib:fref z-%data%
                                          (j
                                           (f2cl-lib:int-add k 2))
                                          ((1 ldz) (1 *))
                                          z-%offset%)
                           (* refsum
                              (f2cl-lib:dconjg
                               (f2cl-lib:fref v-%data% (2 m)
                                              ((1 ldv) (1 *))
                                              v-%offset%)))))
                    (setf (f2cl-lib:fref z-%data%
                                         (j (f2cl-lib:int-add k 3))
                                         ((1 ldz) (1 *)) z-%offset%)
                          (-
                           (f2cl-lib:fref z-%data%
                                          (j
                                           (f2cl-lib:int-add k 3))
                                          ((1 ldz) (1 *))
                                          z-%offset%)
                           (* refsum
                              (f2cl-lib:dconjg
                               (f2cl-lib:fref v-%data% (3 m)
                                              ((1 ldv) (1 *))
                                              v-%offset%)))))
                 label70))))))
      label80))
    (setf k
            (f2cl-lib:int-add krcol
                              (f2cl-lib:int-mul 3
                                                (f2cl-lib:int-sub
                                                 m22 1))))
    (cond
     (bmp22
      (cond
```

```
((/= (f2cl-lib:fref v (1 m22) ((1 ldv) (1 *))) zero)
 (f2cl-lib:fdo (j jtop (f2cl-lib:int-add j 1))
               ((> j
                   (min (the f2cl-lib:integer4 kbot)
                        (the f2cl-lib:integer4
                             (f2cl-lib:int-add k 3))))
                nil)
   (tagbody
     (setf refsum
             (*
              (f2cl-lib:fref v-%data% (1 m22)
                             ((1 ldv) (1 *)) v-%offset%)
              (+
               (f2cl-lib:fref h-%data%
                              (j (f2cl-lib:int-add k 1))
                              ((1 ldh) (1 *)) h-%offset%)
               (*
                (f2cl-lib:fref v-%data% (2 m22)
                               ((1 ldv) (1 *)) v-%offset%)
                (f2cl-lib:fref h-%data%
                               (j (f2cl-lib:int-add k 2))
                               ((1 ldh) (1 *))
                               h-%offset%)))))
     (setf (f2cl-lib:fref h-%data%
                          (j (f2cl-lib:int-add k 1))
                          ((1 ldh) (1 *)) h-%offset%)
             (-
              (f2cl-lib:fref h-%data%
                             (j (f2cl-lib:int-add k 1))
                             ((1 ldh) (1 *)) h-%offset%)
              refsum))
     (setf (f2cl-lib:fref h-%data%
                          (j (f2cl-lib:int-add k 2))
                          ((1 ldh) (1 *)) h-%offset%)
             (-
              (f2cl-lib:fref h-%data%
                             (j (f2cl-lib:int-add k 2))
                             ((1 ldh) (1 *)) h-%offset%)
              (* refsum
                 (f2cl-lib:dconjg
                  (f2cl-lib:fref v-%data% (2 m22)
                                 ((1 ldv) (1 *))
                                 v-%offset%)))))
   label90))
 (cond
  (accum (setf kms (f2cl-lib:int-sub k incol))
   (f2cl-lib:fdo (j
                  (max (the f2cl-lib:integer4 1)
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add ktop
```

```
                                                (f2cl-lib:int-sub
                                                 incol))))
                 (f2cl-lib:int-add j 1))
                ((> j kdu) nil)
  (tagbody
    (setf refsum
            (*
             (f2cl-lib:fref v-%data% (1 m22)
                            ((1 ldv) (1 *)) v-%offset%)
             (+
              (f2cl-lib:fref u-%data%
                             (j
                              (f2cl-lib:int-add kms 1))
                             ((1 ldu) (1 *))
                             u-%offset%)
              (*
               (f2cl-lib:fref v-%data% (2 m22)
                              ((1 ldv) (1 *))
                              v-%offset%)
               (f2cl-lib:fref u-%data%
                              (j
                               (f2cl-lib:int-add kms
                                                 2))
                              ((1 ldu) (1 *))
                              u-%offset%)))))
    (setf (f2cl-lib:fref u-%data%
                         (j (f2cl-lib:int-add kms 1))
                         ((1 ldu) (1 *)) u-%offset%)
            (-
             (f2cl-lib:fref u-%data%
                            (j
                             (f2cl-lib:int-add kms 1))
                            ((1 ldu) (1 *)) u-%offset%)
             refsum))
    (setf (f2cl-lib:fref u-%data%
                         (j (f2cl-lib:int-add kms 2))
                         ((1 ldu) (1 *)) u-%offset%)
            (-
             (f2cl-lib:fref u-%data%
                            (j
                             (f2cl-lib:int-add kms 2))
                            ((1 ldu) (1 *)) u-%offset%)
             (* refsum
                (f2cl-lib:dconjg
                 (f2cl-lib:fref v-%data% (2 m22)
                                ((1 ldv) (1 *))
                                v-%offset%)))))
   label100)))
(wantz
 (f2cl-lib:fdo (j iloz (f2cl-lib:int-add j 1))
```

```
                    ((> j ihiz) nil)
          (tagbody
            (setf refsum
                    (*
                     (f2cl-lib:fref v-%data% (1 m22)
                                    ((1 ldv) (1 *)) v-%offset%)
                     (+
                      (f2cl-lib:fref z-%data%
                                     (j (f2cl-lib:int-add k 1))
                                     ((1 ldz) (1 *))
                                     z-%offset%)
                      (*
                       (f2cl-lib:fref v-%data% (2 m22)
                                      ((1 ldv) (1 *))
                                      v-%offset%)
                       (f2cl-lib:fref z-%data%
                                      (j
                                       (f2cl-lib:int-add k 2))
                                      ((1 ldz) (1 *))
                                      z-%offset%)))))
            (setf (f2cl-lib:fref z-%data%
                                 (j (f2cl-lib:int-add k 1))
                                 ((1 ldz) (1 *)) z-%offset%)
                    (-
                     (f2cl-lib:fref z-%data%
                                    (j (f2cl-lib:int-add k 1))
                                    ((1 ldz) (1 *)) z-%offset%)
                     refsum))
            (setf (f2cl-lib:fref z-%data%
                                 (j (f2cl-lib:int-add k 2))
                                 ((1 ldz) (1 *)) z-%offset%)
                    (-
                     (f2cl-lib:fref z-%data%
                                    (j (f2cl-lib:int-add k 2))
                                    ((1 ldz) (1 *)) z-%offset%)
                     (* refsum
                        (f2cl-lib:dconjg
                         (f2cl-lib:fref v-%data% (2 m22)
                                        ((1 ldv) (1 *))
                                        v-%offset%)))))
          label110))))))))
(setf mstart mtop)
(if (<
     (f2cl-lib:int-add krcol
                       (f2cl-lib:int-mul 3
                                         (f2cl-lib:int-sub
                                          mstart 1)))
     ktop)
    (setf mstart (f2cl-lib:int-add mstart 1)))
(setf mend mbot)
```

```
(if bmp22
    (setf mend (f2cl-lib:int-add mend 1)))
(if (= krcol (f2cl-lib:int-sub kbot 2))
    (setf mend (f2cl-lib:int-add mend 1)))
(f2cl-lib:fdo (m mstart (f2cl-lib:int-add m 1))
              ((> m mend) nil)
  (tagbody
    (setf k
            (min
             (the f2cl-lib:integer4
                  (f2cl-lib:int-sub kbot 1))
             (the f2cl-lib:integer4
                  (f2cl-lib:int-add krcol
                                    (f2cl-lib:int-mul 3
                                                      (f2cl-lib:int-sub
                                                       m
                                                       1))))))
    (cond
     ((/=
       (f2cl-lib:fref h ((f2cl-lib:int-add k 1) k)
                      ((1 ldh) (1 *)))
       zero)
      (setf tst1
              (+
               (cabs1
                (f2cl-lib:fref h-%data% (k k) ((1 ldh) (1 *))
                               h-%offset%))
               (cabs1
                (f2cl-lib:fref h-%data%
                               ((f2cl-lib:int-add k 1)
                                (f2cl-lib:int-add k 1))
                               ((1 ldh) (1 *)) h-%offset%))))
      (cond
       ((= tst1 rzero)
        (if (>= k (f2cl-lib:int-add ktop 1))
            (setf tst1
                    (+ tst1
                       (cabs1
                        (f2cl-lib:fref h-%data%
                                       (k
                                        (f2cl-lib:int-sub k
                                                          1))
                                       ((1 ldh) (1 *))
                                       h-%offset%)))))
        (if (>= k (f2cl-lib:int-add ktop 2))
            (setf tst1
                    (+ tst1
                       (cabs1
                        (f2cl-lib:fref h-%data%
                                       (k
```

```
                                    (f2cl-lib:int-sub k
                                                      2))
                                   ((1 ldh) (1 *))
                                   h-%offset%)))))
          (if (>= k (f2cl-lib:int-add ktop 3))
              (setf tst1
                      (+ tst1
                         (cabs1
                          (f2cl-lib:fref h-%data%
                                         (k
                                          (f2cl-lib:int-sub k
                                                            3))
                                         ((1 ldh) (1 *))
                                         h-%offset%)))))
          (if (<= k (f2cl-lib:int-sub kbot 2))
              (setf tst1
                      (+ tst1
                         (cabs1
                          (f2cl-lib:fref h-%data%
                                         ((f2cl-lib:int-add k
                                                            2)
                                          (f2cl-lib:int-add k
                                                            1))
                                         ((1 ldh) (1 *))
                                         h-%offset%)))))
          (if (<= k (f2cl-lib:int-sub kbot 3))
              (setf tst1
                      (+ tst1
                         (cabs1
                          (f2cl-lib:fref h-%data%
                                         ((f2cl-lib:int-add k
                                                            3)
                                          (f2cl-lib:int-add k
                                                            1))
                                         ((1 ldh) (1 *))
                                         h-%offset%)))))
          (if (<= k (f2cl-lib:int-sub kbot 4))
              (setf tst1
                      (+ tst1
                         (cabs1
                          (f2cl-lib:fref h-%data%
                                         ((f2cl-lib:int-add k
                                                            4)
                                          (f2cl-lib:int-add k
                                                            1))
                                         ((1 ldh) (1 *))
                                         h-%offset%)))))))
      (cond
       ((<=
         (cabs1
```

```
  (f2cl-lib:fref h ((f2cl-lib:int-add k 1) k)
                 ((1 ldh) (1 *))))
 (max smlnum (* ulp tst1)))
(setf h12
        (max
         (cabs1
          (f2cl-lib:fref h-%data%
                         ((f2cl-lib:int-add k 1) k)
                         ((1 ldh) (1 *)) h-%offset%))
         (cabs1
          (f2cl-lib:fref h-%data%
                         (k (f2cl-lib:int-add k 1))
                         ((1 ldh) (1 *))
                         h-%offset%))))
(setf h21
        (min
         (cabs1
          (f2cl-lib:fref h-%data%
                         ((f2cl-lib:int-add k 1) k)
                         ((1 ldh) (1 *)) h-%offset%))
         (cabs1
          (f2cl-lib:fref h-%data%
                         (k (f2cl-lib:int-add k 1))
                         ((1 ldh) (1 *))
                         h-%offset%))))
(setf h11
        (max
         (cabs1
          (f2cl-lib:fref h-%data%
                         ((f2cl-lib:int-add k 1)
                          (f2cl-lib:int-add k 1))
                         ((1 ldh) (1 *)) h-%offset%))
         (cabs1
          (-
           (f2cl-lib:fref h-%data% (k k)
                          ((1 ldh) (1 *)) h-%offset%)
           (f2cl-lib:fref h-%data%
                          ((f2cl-lib:int-add k 1)
                           (f2cl-lib:int-add k 1))
                          ((1 ldh) (1 *))
                          h-%offset%)))))
(setf h22
        (min
         (cabs1
          (f2cl-lib:fref h-%data%
                         ((f2cl-lib:int-add k 1)
                          (f2cl-lib:int-add k 1))
                         ((1 ldh) (1 *)) h-%offset%))
         (cabs1
          (-
```

```
                    (f2cl-lib:fref h-%data% (k k)
                                   ((1 ldh) (1 *)) h-%offset%)
                    (f2cl-lib:fref h-%data%
                                   ((f2cl-lib:int-add k 1)
                                    (f2cl-lib:int-add k 1))
                                   ((1 ldh) (1 *))
                                   h-%offset%)))))
        (setf scl (+ h11 h12))
        (setf tst2 (* h22 (/ h11 scl)))
        (if (or (= tst2 rzero)
                (<= (* h21 (/ h12 scl))
                    (max smlnum (* ulp tst2))))
            (setf (f2cl-lib:fref h-%data%
                                 ((f2cl-lib:int-add k 1) k)
                                 ((1 ldh) (1 *)) h-%offset%)
                  zero))))))
   label120))
(setf mend
        (min nbmps
             (the f2cl-lib:integer4
                  (truncate (- kbot krcol 1) 3))))
(f2cl-lib:fdo (m mtop (f2cl-lib:int-add m 1))
              ((> m mend) nil)
  (tagbody
    (setf k
            (f2cl-lib:int-add krcol
                              (f2cl-lib:int-mul 3
                                                (f2cl-lib:int-sub
                                                 m 1))))
    (setf refsum
            (*
             (f2cl-lib:fref v-%data% (1 m) ((1 ldv) (1 *))
                            v-%offset%)
             (f2cl-lib:fref v-%data% (3 m) ((1 ldv) (1 *))
                            v-%offset%)
             (f2cl-lib:fref h-%data%
                            ((f2cl-lib:int-add k 4)
                             (f2cl-lib:int-add k 3))
                            ((1 ldh) (1 *)) h-%offset%)))
    (setf (f2cl-lib:fref h-%data%
                         ((f2cl-lib:int-add k 4)
                          (f2cl-lib:int-add k 1))
                         ((1 ldh) (1 *)) h-%offset%)
            (- refsum))
    (setf (f2cl-lib:fref h-%data%
                         ((f2cl-lib:int-add k 4)
                          (f2cl-lib:int-add k 2))
                         ((1 ldh) (1 *)) h-%offset%)
            (* (- refsum)
               (f2cl-lib:dconjg
```

```
                    (f2cl-lib:fref v-%data% (2 m) ((1 ldv) (1 *))
                                   v-%offset%))))
          (setf (f2cl-lib:fref h-%data%
                               ((f2cl-lib:int-add k 4)
                                (f2cl-lib:int-add k 3))
                               ((1 ldh) (1 *)) h-%offset%)
                (-
                 (f2cl-lib:fref h-%data%
                                ((f2cl-lib:int-add k 4)
                                 (f2cl-lib:int-add k 3))
                                ((1 ldh) (1 *)) h-%offset%)
                 (* refsum
                    (f2cl-lib:dconjg
                     (f2cl-lib:fref v-%data% (3 m)
                                    ((1 ldv) (1 *))
                                    v-%offset%)))))
        label130))
    label140))
 (cond
  (accum
   (cond (wantt (setf jtop 1) (setf jbot n))
         (t (setf jtop ktop) (setf jbot kbot)))
   (cond
    ((or (not blk22) (< incol ktop) (> ndcol kbot) (<= ns 2))
     (setf k1
             (max (the f2cl-lib:integer4 1)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-sub ktop incol))))
     (setf nu
             (f2cl-lib:int-add
              (f2cl-lib:int-sub kdu
                                (max (the f2cl-lib:integer4 0)
                                     (the f2cl-lib:integer4
                                          (f2cl-lib:int-sub ndcol
                                                            kbot)))
                                k1)
              1))
     (f2cl-lib:fdo (jcol
                    (f2cl-lib:int-add
                     (min (the f2cl-lib:integer4 ndcol)
                          (the f2cl-lib:integer4 kbot))
                     1)
                    (f2cl-lib:int-add jcol nh))
                   ((> jcol jbot) nil)
       (tagbody
         (setf jlen
                 (min (the f2cl-lib:integer4 nh)
                      (the f2cl-lib:integer4
                           (f2cl-lib:int-add
                            (f2cl-lib:int-sub jbot jcol) 1))))
```

```
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
           var-8 var-9 var-10 var-11 var-12)
          (zgemm "C" "N" nu jlen nu one
           (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                                 (k1 k1) ((1 ldu) (1 *))
                                 u-%offset%)
           ldu
           (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                                 ((+ incol k1) jcol)
                                 ((1 ldh) (1 *)) h-%offset%)
           ldh zero wh ldwh)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf nu var-2))
        (when var-3 (setf jlen var-3))
        (when var-4 (setf nu var-4))
        (when var-5 (setf one var-5))
        (when var-7 (setf ldu var-7))
        (when var-9 (setf ldh var-9))
        (when var-10 (setf zero var-10))
        (when var-12 (setf ldwh var-12)))
      (zlacpy "ALL" nu jlen wh ldwh
       (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                             ((+ incol k1) jcol)
                             ((1 ldh) (1 *)) h-%offset%)
       ldh)
     label150))
  (f2cl-lib:fdo (jrow jtop (f2cl-lib:int-add jrow nv))
                ((> jrow
                    (f2cl-lib:int-add
                     (max (the f2cl-lib:integer4 ktop)
                          (the f2cl-lib:integer4 incol))
                     (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (setf jlen
              (min (the f2cl-lib:integer4 nv)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-sub
                         (max (the f2cl-lib:integer4 ktop)
                              (the f2cl-lib:integer4 incol))
                         jrow))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
           var-8 var-9 var-10 var-11 var-12)
          (zgemm "N" "N" jlen nu nu one
           (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                                 (jrow
                                  (f2cl-lib:int-add incol k1))
                                 ((1 ldh) (1 *)) h-%offset%)
```

```
          ldh
          (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                                (k1 k1) ((1 ldu) (1 *))
                                u-%offset%)
          ldu zero wv ldwv)
        (declare (ignore var-0 var-1 var-6 var-8 var-11))
        (when var-2 (setf jlen var-2))
        (when var-3 (setf nu var-3))
        (when var-4 (setf nu var-4))
        (when var-5 (setf one var-5))
        (when var-7 (setf ldh var-7))
        (when var-9 (setf ldu var-9))
        (when var-10 (setf zero var-10))
        (when var-12 (setf ldwv var-12)))
      (zlacpy "ALL" jlen nu wv ldwv
       (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                             (jrow (f2cl-lib:int-add incol k1))
                             ((1 ldh) (1 *)) h-%offset%)
       ldh)
     label160))
  (cond
   (wantz
    (f2cl-lib:fdo (jrow iloz (f2cl-lib:int-add jrow nv))
                  ((> jrow ihiz) nil)
      (tagbody
        (setf jlen
                (min (the f2cl-lib:integer4 nv)
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add
                           (f2cl-lib:int-sub ihiz jrow) 1))))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
             var-8 var-9 var-10 var-11 var-12)
            (zgemm "N" "N" jlen nu nu one
             (f2cl-lib:array-slice z-%data% f2cl-lib:complex16
                                   (jrow
                                    (f2cl-lib:int-add incol
                                                      k1))
                                   ((1 ldz) (1 *)) z-%offset%)
             ldz
             (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                                   (k1 k1) ((1 ldu) (1 *))
                                   u-%offset%)
             ldu zero wv ldwv)
          (declare (ignore var-0 var-1 var-6 var-8 var-11))
          (when var-2 (setf jlen var-2))
          (when var-3 (setf nu var-3))
          (when var-4 (setf nu var-4))
          (when var-5 (setf one var-5))
          (when var-7 (setf ldz var-7))
```

```
          (when var-9 (setf ldu var-9))
          (when var-10 (setf zero var-10))
          (when var-12 (setf ldwv var-12)))
        (zlacpy "ALL" jlen nu wv ldwv
         (f2cl-lib:array-slice z-%data% f2cl-lib:complex16
                               (jrow
                                (f2cl-lib:int-add incol k1))
                               ((1 ldz) (1 *)) z-%offset%)
         ldz)
       label170)))))
(t (setf i2 (the f2cl-lib:integer4 (truncate (+ kdu 1) 2)))
 (setf i4 kdu) (setf j2 (f2cl-lib:int-sub i4 i2))
 (setf j4 kdu)
 (setf kzs (f2cl-lib:int-sub j4 j2 (f2cl-lib:int-add ns 1)))
 (setf knz (f2cl-lib:int-add ns 1))
 (f2cl-lib:fdo (jcol
                (f2cl-lib:int-add
                 (min (the f2cl-lib:integer4 ndcol)
                      (the f2cl-lib:integer4 kbot))
                 1)
                (f2cl-lib:int-add jcol nh))
               ((> jcol jbot) nil)
   (tagbody
     (setf jlen
             (min (the f2cl-lib:integer4 nh)
                  (the f2cl-lib:integer4
                       (f2cl-lib:int-add
                        (f2cl-lib:int-sub jbot jcol) 1))))
     (zlacpy "ALL" knz jlen
      (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                            ((+ incol 1 j2) jcol)
                            ((1 ldh) (1 *)) h-%offset%)
      ldh
      (f2cl-lib:array-slice wh-%data% f2cl-lib:complex16
                            ((+ kzs 1) 1) ((1 ldwh) (1 *))
                            wh-%offset%)
      ldwh)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
         (zlaset "ALL" kzs jlen zero zero wh ldwh)
       (declare (ignore var-0 var-5))
       (when var-1 (setf kzs var-1))
       (when var-2 (setf jlen var-2))
       (when var-3 (setf zero var-3))
       (when var-4 (setf zero var-4))
       (when var-6 (setf ldwh var-6)))
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
          var-8 var-9 var-10)
         (ztrmm "L" "U" "C" "N" knz jlen one
```

```
      (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                            ((+ j2 1)
                             (f2cl-lib:int-add 1 kzs))
                            ((1 ldu) (1 *)) u-%offset%)
      ldu
      (f2cl-lib:array-slice wh-%data% f2cl-lib:complex16
                            ((+ kzs 1) 1) ((1 ldwh) (1 *))
                            wh-%offset%)
      ldwh)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf knz var-4))
  (when var-5 (setf jlen var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldu var-8))
  (when var-10 (setf ldwh var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
     var-8 var-9 var-10 var-11 var-12)
    (zgemm "C" "N" i2 jlen j2 one u ldu
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                            ((+ incol 1) jcol)
                            ((1 ldh) (1 *)) h-%offset%)
     ldh one wh ldwh)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf i2 var-2))
  (when var-3 (setf jlen var-3))
  (when var-4 (setf j2 var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldu var-7))
  (when var-9 (setf ldh var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf ldwh var-12)))
(zlacpy "ALL" j2 jlen
 (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                       ((+ incol 1) jcol) ((1 ldh) (1 *))
                       h-%offset%)
 ldh
 (f2cl-lib:array-slice wh-%data% f2cl-lib:complex16
                       ((+ i2 1) 1) ((1 ldwh) (1 *))
                       wh-%offset%)
 ldwh)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
     var-8 var-9 var-10)
    (ztrmm "L" "L" "C" "N" j2 jlen one
     (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                            (1 (f2cl-lib:int-add i2 1))
                            ((1 ldu) (1 *)) u-%offset%)
     ldu
     (f2cl-lib:array-slice wh-%data% f2cl-lib:complex16
```

```
                                  ((+ i2 1) 1) ((1 ldwh) (1 *))
                                  wh-%offset%)
            ldwh)
        (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
        (when var-4 (setf j2 var-4))
        (when var-5 (setf jlen var-5))
        (when var-6 (setf one var-6))
        (when var-8 (setf ldu var-8))
        (when var-10 (setf ldwh var-10)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
           var-8 var-9 var-10 var-11 var-12)
          (zgemm "C" "N" (f2cl-lib:int-sub i4 i2) jlen
           (f2cl-lib:int-sub j4 j2) one
           (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                                  ((+ j2 1)
                                   (f2cl-lib:int-add i2 1))
                                  ((1 ldu) (1 *)) u-%offset%)
           ldu
           (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                                  ((+ incol 1 j2) jcol)
                                  ((1 ldh) (1 *)) h-%offset%)
           ldh one
           (f2cl-lib:array-slice wh-%data% f2cl-lib:complex16
                                  ((+ i2 1) 1) ((1 ldwh) (1 *))
                                  wh-%offset%)
           ldwh)
        (declare
         (ignore var-0 var-1 var-2 var-4 var-6 var-8 var-11))
        (when var-3 (setf jlen var-3))
        (when var-5 (setf one var-5))
        (when var-7 (setf ldu var-7))
        (when var-9 (setf ldh var-9))
        (when var-10 (setf one var-10))
        (when var-12 (setf ldwh var-12)))
      (zlacpy "ALL" kdu jlen wh ldwh
       (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                             ((+ incol 1) jcol) ((1 ldh) (1 *))
                             h-%offset%)
       ldh)
     label180))
  (f2cl-lib:fdo (jrow jtop (f2cl-lib:int-add jrow nv))
                ((> jrow
                    (f2cl-lib:int-add
                     (max (the f2cl-lib:integer4 incol)
                          (the f2cl-lib:integer4 ktop))
                     (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (setf jlen
```

```
        (min (the f2cl-lib:integer4 nv)
             (the f2cl-lib:integer4
                  (f2cl-lib:int-sub
                   (max (the f2cl-lib:integer4 incol)
                        (the f2cl-lib:integer4 ktop))
                   jrow))))
(zlacpy "ALL" jlen knz
 (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                       (jrow
                        (f2cl-lib:int-add incol 1 j2))
                       ((1 ldh) (1 *)) h-%offset%)
 ldh
 (f2cl-lib:array-slice wv-%data% f2cl-lib:complex16
                       (1 (f2cl-lib:int-add 1 kzs))
                       ((1 ldwv) (1 *)) wv-%offset%)
 ldwv)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
    (zlaset "ALL" jlen kzs zero zero wv ldwv)
  (declare (ignore var-0 var-5))
  (when var-1 (setf jlen var-1))
  (when var-2 (setf kzs var-2))
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwv var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
     var-8 var-9 var-10)
    (ztrmm "R" "U" "N" "N" jlen knz one
     (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                           ((+ j2 1)
                            (f2cl-lib:int-add 1 kzs))
                           ((1 ldu) (1 *)) u-%offset%)
     ldu
     (f2cl-lib:array-slice wv-%data% f2cl-lib:complex16
                           (1 (f2cl-lib:int-add 1 kzs))
                           ((1 ldwv) (1 *)) wv-%offset%)
     ldwv)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf jlen var-4))
  (when var-5 (setf knz var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldu var-8))
  (when var-10 (setf ldwv var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
     var-8 var-9 var-10 var-11 var-12)
    (zgemm "N" "N" jlen i2 j2 one
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                           (jrow
```

```
                              (f2cl-lib:int-add incol 1))
                             ((1 ldh) (1 *)) h-%offset%)
     ldh u ldu one wv ldwv)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf jlen var-2))
  (when var-3 (setf i2 var-3))
  (when var-4 (setf j2 var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldh var-7))
  (when var-9 (setf ldu var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf ldwv var-12)))
(zlacpy "ALL" jlen j2
 (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                       (jrow (f2cl-lib:int-add incol 1))
                       ((1 ldh) (1 *)) h-%offset%)
 ldh
 (f2cl-lib:array-slice wv-%data% f2cl-lib:complex16
                       (1 (f2cl-lib:int-add 1 i2))
                       ((1 ldwv) (1 *)) wv-%offset%)
 ldwv)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
     var-8 var-9 var-10)
    (ztrmm "R" "L" "N" "N" jlen (f2cl-lib:int-sub i4 i2)
     one
     (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                           (1 (f2cl-lib:int-add i2 1))
                           ((1 ldu) (1 *)) u-%offset%)
     ldu
     (f2cl-lib:array-slice wv-%data% f2cl-lib:complex16
                           (1 (f2cl-lib:int-add 1 i2))
                           ((1 ldwv) (1 *)) wv-%offset%)
     ldwv)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9))
  (when var-4 (setf jlen var-4))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldu var-8))
  (when var-10 (setf ldwv var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
     var-8 var-9 var-10 var-11 var-12)
    (zgemm "N" "N" jlen (f2cl-lib:int-sub i4 i2)
     (f2cl-lib:int-sub j4 j2) one
     (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                           (jrow
                            (f2cl-lib:int-add incol 1
                                              j2))
                           ((1 ldh) (1 *)) h-%offset%)
```

```
            ldh
            (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                                  ((+ j2 1)
                                   (f2cl-lib:int-add i2 1))
                                  ((1 ldu) (1 *)) u-%offset%)
            ldu one
            (f2cl-lib:array-slice wv-%data% f2cl-lib:complex16
                                  (1 (f2cl-lib:int-add 1 i2))
                                  ((1 ldwv) (1 *)) wv-%offset%)
            ldwv)
          (declare
           (ignore var-0 var-1 var-3 var-4 var-6 var-8 var-11))
          (when var-2 (setf jlen var-2))
          (when var-5 (setf one var-5))
          (when var-7 (setf ldh var-7))
          (when var-9 (setf ldu var-9))
          (when var-10 (setf one var-10))
          (when var-12 (setf ldwv var-12)))
        (zlacpy "ALL" jlen kdu wv ldwv
         (f2cl-lib:array-slice h-%data% f2cl-lib:complex16
                               (jrow (f2cl-lib:int-add incol 1))
                               ((1 ldh) (1 *)) h-%offset%)
         ldh)
       label190))
    (cond
     (wantz
      (f2cl-lib:fdo (jrow iloz (f2cl-lib:int-add jrow nv))
                    ((> jrow ihiz) nil)
        (tagbody
          (setf jlen
                  (min (the f2cl-lib:integer4 nv)
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add
                             (f2cl-lib:int-sub ihiz jrow) 1))))
          (zlacpy "ALL" jlen knz
           (f2cl-lib:array-slice z-%data% f2cl-lib:complex16
                                 (jrow
                                  (f2cl-lib:int-add incol 1 j2))
                                 ((1 ldz) (1 *)) z-%offset%)
           ldz
           (f2cl-lib:array-slice wv-%data% f2cl-lib:complex16
                                 (1 (f2cl-lib:int-add 1 kzs))
                                 ((1 ldwv) (1 *)) wv-%offset%)
           ldwv)
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6)
              (zlaset "ALL" jlen kzs zero zero wv ldwv)
            (declare (ignore var-0 var-5))
            (when var-1 (setf jlen var-1))
            (when var-2 (setf kzs var-2))
```

```
  (when var-3 (setf zero var-3))
  (when var-4 (setf zero var-4))
  (when var-6 (setf ldwv var-6)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
     var-8 var-9 var-10)
    (ztrmm "R" "U" "N" "N" jlen knz one
     (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                           ((+ j2 1)
                            (f2cl-lib:int-add 1 kzs))
                           ((1 ldu) (1 *)) u-%offset%)
     ldu
     (f2cl-lib:array-slice wv-%data% f2cl-lib:complex16
                           (1 (f2cl-lib:int-add 1 kzs))
                           ((1 ldwv) (1 *))
                           wv-%offset%)
     ldwv)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf jlen var-4))
  (when var-5 (setf knz var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldu var-8))
  (when var-10 (setf ldwv var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
     var-8 var-9 var-10 var-11 var-12)
    (zgemm "N" "N" jlen i2 j2 one
     (f2cl-lib:array-slice z-%data% f2cl-lib:complex16
                           (jrow
                            (f2cl-lib:int-add incol 1))
                           ((1 ldz) (1 *)) z-%offset%)
     ldz u ldu one wv ldwv)
  (declare (ignore var-0 var-1 var-6 var-8 var-11))
  (when var-2 (setf jlen var-2))
  (when var-3 (setf i2 var-3))
  (when var-4 (setf j2 var-4))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldz var-7))
  (when var-9 (setf ldu var-9))
  (when var-10 (setf one var-10))
  (when var-12 (setf ldwv var-12)))
(zlacpy "ALL" jlen j2
 (f2cl-lib:array-slice z-%data% f2cl-lib:complex16
                       (jrow
                        (f2cl-lib:int-add incol 1))
                       ((1 ldz) (1 *)) z-%offset%)
 ldz
 (f2cl-lib:array-slice wv-%data% f2cl-lib:complex16
                       (1 (f2cl-lib:int-add 1 i2))
```

```
                      ((1 ldwv) (1 *)) wv-%offset%)
 ldwv)
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
     var-8 var-9 var-10)
    (ztrmm "R" "L" "N" "N" jlen
     (f2cl-lib:int-sub i4 i2) one
     (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                           (1 (f2cl-lib:int-add i2 1))
                           ((1 ldu) (1 *)) u-%offset%)
     ldu
     (f2cl-lib:array-slice wv-%data% f2cl-lib:complex16
                           (1 (f2cl-lib:int-add 1 i2))
                           ((1 ldwv) (1 *))
                           wv-%offset%)
     ldwv)
  (declare
   (ignore var-0 var-1 var-2 var-3 var-5 var-7 var-9))
  (when var-4 (setf jlen var-4))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldu var-8))
  (when var-10 (setf ldwv var-10)))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
     var-8 var-9 var-10 var-11 var-12)
    (zgemm "N" "N" jlen (f2cl-lib:int-sub i4 i2)
     (f2cl-lib:int-sub j4 j2) one
     (f2cl-lib:array-slice z-%data% f2cl-lib:complex16
                           (jrow
                            (f2cl-lib:int-add incol 1
                                              j2))
                           ((1 ldz) (1 *)) z-%offset%)
     ldz
     (f2cl-lib:array-slice u-%data% f2cl-lib:complex16
                           ((+ j2 1)
                            (f2cl-lib:int-add i2 1))
                           ((1 ldu) (1 *)) u-%offset%)
     ldu one
     (f2cl-lib:array-slice wv-%data% f2cl-lib:complex16
                           (1 (f2cl-lib:int-add 1 i2))
                           ((1 ldwv) (1 *))
                           wv-%offset%)
     ldwv)
  (declare
   (ignore var-0 var-1 var-3 var-4 var-6 var-8 var-11))
  (when var-2 (setf jlen var-2))
  (when var-5 (setf one var-5))
  (when var-7 (setf ldz var-7))
  (when var-9 (setf ldu var-9))
  (when var-10 (setf one var-10))
```

```
                        (when var-12 (setf ldwv var-12)))
                      (zlacpy "ALL" jlen kdu wv ldwv
                       (f2cl-lib:array-slice z-%data% f2cl-lib:complex16
                                             (jrow
                                              (f2cl-lib:int-add incol 1))
                                             ((1 ldz) (1 *)) z-%offset%)
                       ldz)
                     label200))))))))
            label210))
        end_label
         (return
          (values nil nil nil nil nil nil nil nil nil ldh nil nil nil ldz nil
                  nil nil ldu nil nil ldwv nil nil ldwh)))))))
```

———-

90.229 zlarfb

90.229.1 Axiom unit tests

— **zlarfb.input** —

```
)set break resume
)sys rm -f zlarfb.output
)spool zlarfb.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.229.2 Axiom help page

— **zlarfb.help** —

```
======================================================================
zlarfb examples
======================================================================

======================================================================
```

```
Man Page Details
=====================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZLARFB( SIDE, TRANS, DIRECT, STOREV, M, N, K, V, LDV,
                          T, LDT, C, LDC, WORK, LDWORK )

       .. Scalar Arguments ..
       CHARACTER          DIRECT, SIDE, STOREV, TRANS
       INTEGER            K, LDC, LDT, LDV, LDWORK, M, N
       ..
       .. Array Arguments ..
       COMPLEX*16         C( LDC, * ), T( LDT, * ), V( LDV, * ),
      $                   WORK( LDWORK, * )
       ..


 Purpose:
 =============

 ZLARFB applies a complex block reflector H or its transpose H**H to a
 complex M-by-N matrix C, from either the left or the right.


 Arguments:
 ==========

  [in] SIDE

          SIDE is CHARACTER*1
          = 'L': apply H or H**H from the Left
          = 'R': apply H or H**H from the Right

  [in] TRANS

          TRANS is CHARACTER*1
          = 'N': apply H (No transpose)
          = 'C': apply H**H (Conjugate transpose)

  [in] DIRECT

          DIRECT is CHARACTER*1
          Indicates how H is formed from a product of elementary
          reflectors
          = 'F': H = H(1) H(2) . . . H(k) (Forward)
```

```
        = 'B': H = H(k) . . . H(2) H(1) (Backward)

[in] STOREV

        STOREV is CHARACTER*1
        Indicates how the vectors which define the elementary
        reflectors are stored:
        = 'C': Columnwise
        = 'R': Rowwise

[in] M

        M is INTEGER
        The number of rows of the matrix C.

[in] N

        N is INTEGER
        The number of columns of the matrix C.

[in] K

        K is INTEGER
        The order of the matrix T (= the number of elementary
        reflectors whose product defines the block reflector).

[in] V

        V is COMPLEX*16 array, dimension
                              (LDV,K) if STOREV = 'C'
                              (LDV,M) if STOREV = 'R' and SIDE = 'L'
                              (LDV,N) if STOREV = 'R' and SIDE = 'R'
        See Further Details.

[in] LDV

        LDV is INTEGER
        The leading dimension of the array V.
        If STOREV = 'C' and SIDE = 'L', LDV >= max(1,M);
        if STOREV = 'C' and SIDE = 'R', LDV >= max(1,N);
        if STOREV = 'R', LDV >= K.

[in] T

        T is COMPLEX*16 array, dimension (LDT,K)
        The triangular K-by-K matrix T in the representation of the
        block reflector.

[in] LDT
```

```
          LDT is INTEGER
          The leading dimension of the array T. LDT >= K.

  [in,out] C

          C is COMPLEX*16 array, dimension (LDC,N)
          On entry, the M-by-N matrix C.
          On exit, C is overwritten by H*C or H**H*C or C*H or C*H**H.

  [in] LDC

          LDC is INTEGER
          The leading dimension of the array C. LDC >= max(1,M).

  [out] WORK

          WORK is COMPLEX*16 array, dimension (LDWORK,K)

  [in] LDWORK

          LDWORK is INTEGER
          The leading dimension of the array WORK.
          If SIDE = 'L', LDWORK >= max(1,N);
          if SIDE = 'R', LDWORK >= max(1,M).

Further Details:
=====================

  The shape of the matrix V and the storage of the vectors which define
  the H(i) is best illustrated by the following example with n = 5 and
  k = 3. The elements equal to 1 are not stored; the corresponding
  array elements are modified but restored on exit. The rest of the
  array is not used.

  DIRECT = 'F' and STOREV = 'C':         DIRECT = 'F' and STOREV = 'R':

               V = (  1       )                 V = (  1 v1 v1 v1 v1 )
                   ( v1  1    )                     (     1 v2 v2 v2 )
                   ( v1 v2  1 )                     (        1 v3 v3 )
                   ( v1 v2 v3 )
                   ( v1 v2 v3 )

  DIRECT = 'B' and STOREV = 'C':         DIRECT = 'B' and STOREV = 'R':

               V = ( v1 v2 v3 )                 V = ( v1 v1  1       )
                   ( v1 v2 v3 )                     ( v2 v2 v2  1    )
                   (  1 v2 v3 )                     ( v3 v3 v3 v3  1 )
                   (     1 v3 )
                   (        1 )
```

———-

90.229.3 fortran code

— **zlarfb.f** —

```
*  =====================================================================
      SUBROUTINE ZLARFB( SIDE, TRANS, DIRECT, STOREV, M, N, K, V, LDV,
     $                   T, LDT, C, LDC, WORK, LDWORK )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          DIRECT, SIDE, STOREV, TRANS
      INTEGER            K, LDC, LDT, LDV, LDWORK, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         C( LDC, * ), T( LDT, * ), V( LDV, * ),
     $                   WORK( LDWORK, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      CHARACTER          TRANST
      INTEGER            I, J, LASTV, LASTC
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAZLR, ILAZLC
      EXTERNAL           LSAME, ILAZLR, ILAZLC
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZCOPY, ZGEMM, ZLACGV, ZTRMM
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DCONJG
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
```

```
*
      IF( M.LE.0 .OR. N.LE.0 )
     $   RETURN
*
      IF( LSAME( TRANS, 'N' ) ) THEN
         TRANST = 'C'
      ELSE
         TRANST = 'N'
      END IF
*
      IF( LSAME( STOREV, 'C' ) ) THEN
*
         IF( LSAME( DIRECT, 'F' ) ) THEN
*
*           Let  V =  ( V1 )    (first K rows)
*                     ( V2 )
*           where  V1  is unit lower triangular.
*
            IF( LSAME( SIDE, 'L' ) ) THEN
*
*              Form  H * C  or  H**H * C  where  C = ( C1 )
*                                                    ( C2 )
*
               LASTV = MAX( K, ILAZLR( M, K, V, LDV ) )
               LASTC = ILAZLC( LASTV, N, C, LDC )
*
*              W := C**H * V  =  (C1**H * V1 + C2**H * V2)  (stored in WORK)
*
*              W := C1**H
*
               DO 10 J = 1, K
                  CALL ZCOPY( LASTC, C( J, 1 ), LDC, WORK( 1, J ), 1 )
                  CALL ZLACGV( LASTC, WORK( 1, J ), 1 )
   10          CONTINUE
*
*              W := W * V1
*
               CALL ZTRMM( 'Right', 'Lower', 'No transpose', 'Unit',
     $              LASTC, K, ONE, V, LDV, WORK, LDWORK )
               IF( LASTV.GT.K ) THEN
*
*                 W := W + C2**H *V2
*
                  CALL ZGEMM( 'Conjugate transpose', 'No transpose',
     $                 LASTC, K, LASTV-K, ONE, C( K+1, 1 ), LDC,
     $                 V( K+1, 1 ), LDV, ONE, WORK, LDWORK )
               END IF
*
*              W := W * T**H  or  W * T
*
```

```
               CALL ZTRMM( 'Right', 'Upper', TRANST, 'Non-unit',
     $              LASTC, K, ONE, T, LDT, WORK, LDWORK )
*
*              C := C - V * W**H
*
               IF( M.GT.K ) THEN
*
*                 C2 := C2 - V2 * W**H
*
                  CALL ZGEMM( 'No transpose', 'Conjugate transpose',
     $                 LASTV-K, LASTC, K,
     $                 -ONE, V( K+1, 1 ), LDV, WORK, LDWORK,
     $                 ONE, C( K+1, 1 ), LDC )
               END IF
*
*              W := W * V1**H
*
               CALL ZTRMM( 'Right', 'Lower', 'Conjugate transpose',
     $              'Unit', LASTC, K, ONE, V, LDV, WORK, LDWORK )
*
*              C1 := C1 - W**H
*
               DO 30 J = 1, K
                  DO 20 I = 1, LASTC
                     C( J, I ) = C( J, I ) - DCONJG( WORK( I, J ) )
   20             CONTINUE
   30          CONTINUE
*
            ELSE IF( LSAME( SIDE, 'R' ) ) THEN
*
*              Form  C * H  or  C * H**H  where  C = ( C1  C2 )
*
               LASTV = MAX( K, ILAZLR( N, K, V, LDV ) )
               LASTC = ILAZLR( M, LASTV, C, LDC )
*
*              W := C * V  =  (C1*V1 + C2*V2)  (stored in WORK)
*
*              W := C1
*
               DO 40 J = 1, K
                  CALL ZCOPY( LASTC, C( 1, J ), 1, WORK( 1, J ), 1 )
   40          CONTINUE
*
*              W := W * V1
*
               CALL ZTRMM( 'Right', 'Lower', 'No transpose', 'Unit',
     $              LASTC, K, ONE, V, LDV, WORK, LDWORK )
               IF( LASTV.GT.K ) THEN
*
*                 W := W + C2 * V2
```

```
*
                 CALL ZGEMM( 'No transpose', 'No transpose',
     $                LASTC, K, LASTV-K,
     $                ONE, C( 1, K+1 ), LDC, V( K+1, 1 ), LDV,
     $                ONE, WORK, LDWORK )
              END IF
*
*             W := W * T  or  W * T**H
*
              CALL ZTRMM( 'Right', 'Upper', TRANS, 'Non-unit',
     $             LASTC, K, ONE, T, LDT, WORK, LDWORK )
*
*             C := C - W * V**H
*
              IF( LASTV.GT.K ) THEN
*
*                C2 := C2 - W * V2**H
*
                 CALL ZGEMM( 'No transpose', 'Conjugate transpose',
     $                LASTC, LASTV-K, K,
     $                -ONE, WORK, LDWORK, V( K+1, 1 ), LDV,
     $                ONE, C( 1, K+1 ), LDC )
              END IF
*
*             W := W * V1**H
*
              CALL ZTRMM( 'Right', 'Lower', 'Conjugate transpose',
     $             'Unit', LASTC, K, ONE, V, LDV, WORK, LDWORK )
*
*             C1 := C1 - W
*
              DO 60 J = 1, K
                 DO 50 I = 1, LASTC
                    C( I, J ) = C( I, J ) - WORK( I, J )
   50            CONTINUE
   60         CONTINUE
           END IF
*
        ELSE
*
*          Let  V =  ( V1 )
*                    ( V2 )    (last K rows)
*          where  V2  is unit upper triangular.
*
           IF( LSAME( SIDE, 'L' ) ) THEN
*
*             Form  H * C  or  H**H * C  where  C = ( C1 )
*                                                   ( C2 )
*
              LASTV = MAX( K, ILAZLR( M, K, V, LDV ) )
```

```
               LASTC = ILAZLC( LASTV, N, C, LDC )
*
*              W := C**H * V  =  (C1**H * V1 + C2**H * V2)  (stored in WORK)
*
*              W := C2**H
*
               DO 70 J = 1, K
                  CALL ZCOPY( LASTC, C( LASTV-K+J, 1 ), LDC,
     $                 WORK( 1, J ), 1 )
                  CALL ZLACGV( LASTC, WORK( 1, J ), 1 )
   70          CONTINUE
*
*              W := W * V2
*
               CALL ZTRMM( 'Right', 'Upper', 'No transpose', 'Unit',
     $              LASTC, K, ONE, V( LASTV-K+1, 1 ), LDV,
     $              WORK, LDWORK )
               IF( LASTV.GT.K ) THEN
*
*                 W := W + C1**H*V1
*
                  CALL ZGEMM( 'Conjugate transpose', 'No transpose',
     $                 LASTC, K, LASTV-K,
     $                 ONE, C, LDC, V, LDV,
     $                 ONE, WORK, LDWORK )
               END IF
*
*              W := W * T**H  or  W * T
*
               CALL ZTRMM( 'Right', 'Lower', TRANST, 'Non-unit',
     $              LASTC, K, ONE, T, LDT, WORK, LDWORK )
*
*              C := C - V * W**H
*
               IF( LASTV.GT.K ) THEN
*
*                 C1 := C1 - V1 * W**H
*
                  CALL ZGEMM( 'No transpose', 'Conjugate transpose',
     $                 LASTV-K, LASTC, K,
     $                 -ONE, V, LDV, WORK, LDWORK,
     $                 ONE, C, LDC )
               END IF
*
*              W := W * V2**H
*
               CALL ZTRMM( 'Right', 'Upper', 'Conjugate transpose',
     $              'Unit', LASTC, K, ONE, V( LASTV-K+1, 1 ), LDV,
     $              WORK, LDWORK )
*
```

```
*             C2 := C2 - W**H
*
              DO 90 J = 1, K
                 DO 80 I = 1, LASTC
                    C( LASTV-K+J, I ) = C( LASTV-K+J, I ) -
     $                              DCONJG( WORK( I, J ) )
   80            CONTINUE
   90         CONTINUE
*
           ELSE IF( LSAME( SIDE, 'R' ) ) THEN
*
*             Form  C * H  or  C * H**H  where  C = ( C1  C2 )
*
              LASTV = MAX( K, ILAZLR( N, K, V, LDV ) )
              LASTC = ILAZLR( M, LASTV, C, LDC )
*
*             W := C * V  =  (C1*V1 + C2*V2)  (stored in WORK)
*
*             W := C2
*
              DO 100 J = 1, K
                 CALL ZCOPY( LASTC, C( 1, LASTV-K+J ), 1,
     $                WORK( 1, J ), 1 )
  100         CONTINUE
*
*             W := W * V2
*
              CALL ZTRMM( 'Right', 'Upper', 'No transpose', 'Unit',
     $             LASTC, K, ONE, V( LASTV-K+1, 1 ), LDV,
     $             WORK, LDWORK )
              IF( LASTV.GT.K ) THEN
*
*                W := W + C1 * V1
*
                 CALL ZGEMM( 'No transpose', 'No transpose',
     $                LASTC, K, LASTV-K,
     $                ONE, C, LDC, V, LDV, ONE, WORK, LDWORK )
              END IF
*
*             W := W * T  or  W * T**H
*
              CALL ZTRMM( 'Right', 'Lower', TRANS, 'Non-unit',
     $             LASTC, K, ONE, T, LDT, WORK, LDWORK )
*
*             C := C - W * V**H
*
              IF( LASTV.GT.K ) THEN
*
*                C1 := C1 - W * V1**H
*
```

```
                 CALL ZGEMM( 'No transpose', 'Conjugate transpose',
     $                LASTC, LASTV-K, K, -ONE, WORK, LDWORK, V, LDV,
     $                ONE, C, LDC )
              END IF
*
*             W := W * V2**H
*
              CALL ZTRMM( 'Right', 'Upper', 'Conjugate transpose',
     $             'Unit', LASTC, K, ONE, V( LASTV-K+1, 1 ), LDV,
     $             WORK, LDWORK )
*
*             C2 := C2 - W
*
              DO 120 J = 1, K
                 DO 110 I = 1, LASTC
                    C( I, LASTV-K+J ) = C( I, LASTV-K+J )
     $                   - WORK( I, J )
  110            CONTINUE
  120         CONTINUE
           END IF
        END IF
*
      ELSE IF( LSAME( STOREV, 'R' ) ) THEN
*
        IF( LSAME( DIRECT, 'F' ) ) THEN
*
*          Let  V =  ( V1  V2 )    (V1: first K columns)
*          where  V1  is unit upper triangular.
*
           IF( LSAME( SIDE, 'L' ) ) THEN
*
*             Form  H * C  or  H**H * C  where  C = ( C1 )
*                                                   ( C2 )
*
              LASTV = MAX( K, ILAZLC( K, M, V, LDV ) )
              LASTC = ILAZLC( LASTV, N, C, LDC )
*
*             W := C**H * V**H  =  (C1**H * V1**H + C2**H * V2**H) (stored in WORK)
*
*             W := C1**H
*
              DO 130 J = 1, K
                 CALL ZCOPY( LASTC, C( J, 1 ), LDC, WORK( 1, J ), 1 )
                 CALL ZLACGV( LASTC, WORK( 1, J ), 1 )
  130         CONTINUE
*
*             W := W * V1**H
*
              CALL ZTRMM( 'Right', 'Upper', 'Conjugate transpose',
     $                    'Unit', LASTC, K, ONE, V, LDV, WORK, LDWORK )
```

```
               IF( LASTV.GT.K ) THEN
*
*                 W := W + C2**H*V2**H
*
                  CALL ZGEMM( 'Conjugate transpose',
     $                 'Conjugate transpose', LASTC, K, LASTV-K,
     $                 ONE, C( K+1, 1 ), LDC, V( 1, K+1 ), LDV,
     $                 ONE, WORK, LDWORK )
               END IF
*
*              W := W * T**H  or  W * T
*
               CALL ZTRMM( 'Right', 'Upper', TRANST, 'Non-unit',
     $             LASTC, K, ONE, T, LDT, WORK, LDWORK )
*
*              C := C - V**H * W**H
*
               IF( LASTV.GT.K ) THEN
*
*                 C2 := C2 - V2**H * W**H
*
                  CALL ZGEMM( 'Conjugate transpose',
     $                 'Conjugate transpose', LASTV-K, LASTC, K,
     $                 -ONE, V( 1, K+1 ), LDV, WORK, LDWORK,
     $                 ONE, C( K+1, 1 ), LDC )
               END IF
*
*              W := W * V1
*
               CALL ZTRMM( 'Right', 'Upper', 'No transpose', 'Unit',
     $             LASTC, K, ONE, V, LDV, WORK, LDWORK )
*
*              C1 := C1 - W**H
*
               DO 150 J = 1, K
                  DO 140 I = 1, LASTC
                     C( J, I ) = C( J, I ) - DCONJG( WORK( I, J ) )
  140             CONTINUE
  150          CONTINUE
*
            ELSE IF( LSAME( SIDE, 'R' ) ) THEN
*
*              Form  C * H  or  C * H**H  where  C = ( C1  C2 )
*
               LASTV = MAX( K, ILAZLC( K, N, V, LDV ) )
               LASTC = ILAZLR( M, LASTV, C, LDC )
*
*              W := C * V**H  =  (C1*V1**H + C2*V2**H)  (stored in WORK)
*
*              W := C1
```

```
*
               DO 160 J = 1, K
                  CALL ZCOPY( LASTC, C( 1, J ), 1, WORK( 1, J ), 1 )
  160          CONTINUE
*
*              W := W * V1**H
*
               CALL ZTRMM( 'Right', 'Upper', 'Conjugate transpose',
     $                     'Unit', LASTC, K, ONE, V, LDV, WORK, LDWORK )
               IF( LASTV.GT.K ) THEN
*
*                 W := W + C2 * V2**H
*
                  CALL ZGEMM( 'No transpose', 'Conjugate transpose',
     $                 LASTC, K, LASTV-K, ONE, C( 1, K+1 ), LDC,
     $                 V( 1, K+1 ), LDV, ONE, WORK, LDWORK )
               END IF
*
*              W := W * T  or  W * T**H
*
               CALL ZTRMM( 'Right', 'Upper', TRANS, 'Non-unit',
     $              LASTC, K, ONE, T, LDT, WORK, LDWORK )
*
*              C := C - W * V
*
               IF( LASTV.GT.K ) THEN
*
*                 C2 := C2 - W * V2
*
                  CALL ZGEMM( 'No transpose', 'No transpose',
     $                 LASTC, LASTV-K, K,
     $                 -ONE, WORK, LDWORK, V( 1, K+1 ), LDV,
     $                 ONE, C( 1, K+1 ), LDC )
               END IF
*
*              W := W * V1
*
               CALL ZTRMM( 'Right', 'Upper', 'No transpose', 'Unit',
     $              LASTC, K, ONE, V, LDV, WORK, LDWORK )
*
*              C1 := C1 - W
*
               DO 180 J = 1, K
                  DO 170 I = 1, LASTC
                     C( I, J ) = C( I, J ) - WORK( I, J )
  170             CONTINUE
  180          CONTINUE
*
            END IF
*
```

```
         ELSE
*
*           Let  V =  ( V1  V2 )    (V2: last K columns)
*           where  V2  is unit lower triangular.
*
            IF( LSAME( SIDE, 'L' ) ) THEN
*
*              Form  H * C  or  H**H * C  where  C = ( C1 )
*                                                    ( C2 )
*
               LASTV = MAX( K, ILAZLC( K, M, V, LDV ) )
               LASTC = ILAZLC( LASTV, N, C, LDC )
*
*              W := C**H * V**H  =  (C1**H * V1**H + C2**H * V2**H) (stored in WORK)
*
*              W := C2**H
*
               DO 190 J = 1, K
                  CALL ZCOPY( LASTC, C( LASTV-K+J, 1 ), LDC,
     $                 WORK( 1, J ), 1 )
                  CALL ZLACGV( LASTC, WORK( 1, J ), 1 )
  190          CONTINUE
*
*              W := W * V2**H
*
               CALL ZTRMM( 'Right', 'Lower', 'Conjugate transpose',
     $              'Unit', LASTC, K, ONE, V( 1, LASTV-K+1 ), LDV,
     $              WORK, LDWORK )
               IF( LASTV.GT.K ) THEN
*
*                 W := W + C1**H * V1**H
*
                  CALL ZGEMM( 'Conjugate transpose',
     $                 'Conjugate transpose', LASTC, K, LASTV-K,
     $                 ONE, C, LDC, V, LDV, ONE, WORK, LDWORK )
               END IF
*
*              W := W * T**H  or  W * T
*
               CALL ZTRMM( 'Right', 'Lower', TRANST, 'Non-unit',
     $              LASTC, K, ONE, T, LDT, WORK, LDWORK )
*
*              C := C - V**H * W**H
*
               IF( LASTV.GT.K ) THEN
*
*                 C1 := C1 - V1**H * W**H
*
                  CALL ZGEMM( 'Conjugate transpose',
     $                 'Conjugate transpose', LASTV-K, LASTC, K,
```

```
     $                 -ONE, V, LDV, WORK, LDWORK, ONE, C, LDC )
               END IF
*
*              W := W * V2
*
               CALL ZTRMM( 'Right', 'Lower', 'No transpose', 'Unit',
     $              LASTC, K, ONE, V( 1, LASTV-K+1 ), LDV,
     $              WORK, LDWORK )
*
*              C2 := C2 - W**H
*
               DO 210 J = 1, K
                  DO 200 I = 1, LASTC
                     C( LASTV-K+J, I ) = C( LASTV-K+J, I ) -
     $                               DCONJG( WORK( I, J ) )
  200             CONTINUE
  210          CONTINUE
*
            ELSE IF( LSAME( SIDE, 'R' ) ) THEN
*
*              Form  C * H  or  C * H**H  where  C = ( C1  C2 )
*
               LASTV = MAX( K, ILAZLC( K, N, V, LDV ) )
               LASTC = ILAZLR( M, LASTV, C, LDC )
*
*              W := C * V**H  =  (C1*V1**H + C2*V2**H)  (stored in WORK)
*
*              W := C2
*
               DO 220 J = 1, K
                  CALL ZCOPY( LASTC, C( 1, LASTV-K+J ), 1,
     $                 WORK( 1, J ), 1 )
  220          CONTINUE
*
*              W := W * V2**H
*
               CALL ZTRMM( 'Right', 'Lower', 'Conjugate transpose',
     $              'Unit', LASTC, K, ONE, V( 1, LASTV-K+1 ), LDV,
     $              WORK, LDWORK )
               IF( LASTV.GT.K ) THEN
*
*                 W := W + C1 * V1**H
*
                  CALL ZGEMM( 'No transpose', 'Conjugate transpose',
     $                 LASTC, K, LASTV-K, ONE, C, LDC, V, LDV, ONE,
     $                 WORK, LDWORK )
               END IF
*
*              W := W * T  or  W * T**H
*
```

```
               CALL ZTRMM( 'Right', 'Lower', TRANS, 'Non-unit',
     $              LASTC, K, ONE, T, LDT, WORK, LDWORK )
*
*              C := C - W * V
*
               IF( LASTV.GT.K ) THEN
*
*                 C1 := C1 - W * V1
*
                  CALL ZGEMM( 'No transpose', 'No transpose',
     $                 LASTC, LASTV-K, K, -ONE, WORK, LDWORK, V, LDV,
     $                 ONE, C, LDC )
               END IF
*
*              W := W * V2
*
               CALL ZTRMM( 'Right', 'Lower', 'No transpose', 'Unit',
     $              LASTC, K, ONE, V( 1, LASTV-K+1 ), LDV,
     $              WORK, LDWORK )
*
*              C1 := C1 - W
*
               DO 240 J = 1, K
                  DO 230 I = 1, LASTC
                     C( I, LASTV-K+J ) = C( I, LASTV-K+J )
     $                    - WORK( I, J )
  230             CONTINUE
  240          CONTINUE
*
            END IF
*
         END IF
      END IF
*
      RETURN
*
*     End of ZLARFB
*
      END
```

———-

90.229.4 lisp code

— zlarfb.lisp —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)))
```

```
(declare (type (f2cl-lib:complex16) one)
         (ignorable one))
(defun zlarfb (side trans direct storev m n k v ldv t$ ldt c ldc work ldwork)
  (declare (type (array f2cl-lib:complex16 (*)) work c t$ v)
           (type (f2cl-lib:integer4) ldwork ldc ldt ldv k n m)
           (type (string *) storev direct trans side))
  (f2cl-lib:with-multi-array-data
      ((side character side-%data% side-%offset%)
       (trans character trans-%data% trans-%offset%)
       (direct character direct-%data% direct-%offset%)
       (storev character storev-%data% storev-%offset%)
       (v f2cl-lib:complex16 v-%data% v-%offset%)
       (t$ f2cl-lib:complex16 t$-%data% t$-%offset%)
       (c f2cl-lib:complex16 c-%data% c-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%))
    (prog ((i 0) (j 0)
           (transt
            (make-array '(1) :element-type 'character :initial-element #\ )))
      (declare (type (f2cl-lib:integer4) i j)
               (type (string 1) transt))
      (if (or (<= m 0) (<= n 0))
          (go end_label))
      (cond
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)
        (f2cl-lib:f2cl-set-string transt "C" (string 1)))
       (t (f2cl-lib:f2cl-set-string transt "N" (string 1))))
      (cond
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame storev "C")
          (declare (ignore var-1))
          (when var-0 (setf storev var-0))
          ret-val)
        (cond
         ((multiple-value-bind (ret-val var-0 var-1)
              (lsame direct "F")
            (declare (ignore var-1))
            (when var-0 (setf direct var-0))
            ret-val)
          (cond
           ((multiple-value-bind (ret-val var-0 var-1)
                (lsame side "L")
              (declare (ignore var-1))
              (when var-0 (setf side var-0))
              ret-val)
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j k) nil)
```

```
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zcopy n
         (f2cl-lib:array-slice c-%data% f2cl-lib:complex16 (j 1)
                               ((1 ldc) (1 *)) c-%offset%)
         ldc
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (1 j) ((1 ldwork) (1 *))
                               work-%offset%)
         1)
      (declare (ignore var-1 var-3 var-4))
      (when var-0 (setf n var-0))
      (when var-2 (setf ldc var-2)))
    (zlacgv n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (1 j)
                           ((1 ldwork) (1 *)) work-%offset%)
     1)
   label10))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "Right" "Lower" "No transpose" "Unit" n k one v ldv
     work ldwork)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> m k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "Conjugate transpose" "No transpose" n k
       (f2cl-lib:int-sub m k) one
       (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                             ((+ k 1) 1) ((1 ldc) (1 *))
                             c-%offset%)
       ldc
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             ((+ k 1) 1) ((1 ldv) (1 *))
                             v-%offset%)
       ldv one work ldwork)
    (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf k var-3))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldc var-7))
    (when var-9 (setf ldv var-9))
```

```
    (when var-10 (setf one var-10))
    (when var-12 (setf ldwork var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "Right" "Upper" transt "Non-unit" n k one t$ ldt work
     ldwork)
  (declare (ignore var-0 var-1 var-3 var-7 var-9))
  (when var-2 (setf transt var-2))
  (when var-4 (setf n var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> m k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "No transpose" "Conjugate transpose"
       (f2cl-lib:int-sub m k) n k (- one)
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             ((+ k 1) 1) ((1 ldv) (1 *))
                             v-%offset%)
       ldv work ldwork one
       (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                             ((+ k 1) 1) ((1 ldc) (1 *))
                             c-%offset%)
       ldc)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
    (when var-3 (setf n var-3))
    (when var-4 (setf k var-4))
    (when var-7 (setf ldv var-7))
    (when var-9 (setf ldwork var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldc var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "Right" "Lower" "Conjugate transpose" "Unit" n k one v
     ldv work ldwork)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j k) nil)
  (tagbody
```

```
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref c-%data% (j i) ((1 ldc) (1 *))
                               c-%offset%)
                (-
                 (f2cl-lib:fref c-%data% (j i) ((1 ldc) (1 *))
                                c-%offset%)
                 (f2cl-lib:dconjg
                  (f2cl-lib:fref work-%data% (i j)
                                 ((1 ldwork) (1 *))
                                 work-%offset%))))
         label20))
    label30)))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame side "R")
   (declare (ignore var-1))
   (when var-0 (setf side var-0))
   ret-val)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j k) nil)
   (tagbody
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (zcopy m
          (f2cl-lib:array-slice c-%data% f2cl-lib:complex16 (1 j)
                                ((1 ldc) (1 *)) c-%offset%)
          1
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                (1 j) ((1 ldwork) (1 *))
                                work-%offset%)
          1)
       (declare (ignore var-1 var-2 var-3 var-4))
       (when var-0 (setf m var-0)))
    label40))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (ztrmm "Right" "Lower" "No transpose" "Unit" m k one v ldv
      work ldwork)
   (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
   (when var-4 (setf m var-4))
   (when var-5 (setf k var-5))
   (when var-6 (setf one var-6))
   (when var-8 (setf ldv var-8))
   (when var-10 (setf ldwork var-10)))
 (cond
  ((> n k)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10 var-11 var-12)
```

```
      (zgemm "No transpose" "No transpose" m k
       (f2cl-lib:int-sub n k) one
       (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                             (1 (f2cl-lib:int-add k 1))
                             ((1 ldc) (1 *)) c-%offset%)
       ldc
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             ((+ k 1) 1) ((1 ldv) (1 *))
                             v-%offset%)
       ldv one work ldwork)
    (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf k var-3))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldc var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldwork var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "Right" "Upper" trans "Non-unit" m k one t$ ldt work
     ldwork)
  (declare (ignore var-0 var-1 var-3 var-7 var-9))
  (when var-2 (setf trans var-2))
  (when var-4 (setf m var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> n k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "No transpose" "Conjugate transpose" m
       (f2cl-lib:int-sub n k) k (- one) work ldwork
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             ((+ k 1) 1) ((1 ldv) (1 *))
                             v-%offset%)
       ldv one
       (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                             (1 (f2cl-lib:int-add k 1))
                             ((1 ldc) (1 *)) c-%offset%)
       ldc)
    (declare (ignore var-0 var-1 var-3 var-5 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-4 (setf k var-4))
    (when var-7 (setf ldwork var-7))
    (when var-9 (setf ldv var-9))
```

```
        (when var-10 (setf one var-10))
        (when var-12 (setf ldc var-12)))))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (ztrmm "Right" "Lower" "Conjugate transpose" "Unit" m k one v
         ldv work ldwork)
      (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
      (when var-4 (setf m var-4))
      (when var-5 (setf k var-5))
      (when var-6 (setf one var-6))
      (when var-8 (setf ldv var-8))
      (when var-10 (setf ldwork var-10)))
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j k) nil)
      (tagbody
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                 c-%offset%)
                  (-
                   (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                  c-%offset%)
                   (f2cl-lib:fref work-%data% (i j)
                                  ((1 ldwork) (1 *))
                                  work-%offset%)))
           label50))
       label60)))))
(t
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame side "L")
     (declare (ignore var-1))
     (when var-0 (setf side var-0))
     ret-val)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j k) nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (zcopy n
            (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                  ((+ m (f2cl-lib:int-sub k) j) 1)
                                  ((1 ldc) (1 *)) c-%offset%)
            ldc
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                  (1 j) ((1 ldwork) (1 *))
                                  work-%offset%)
            1)
         (declare (ignore var-1 var-3 var-4))
```

```
      (when var-0 (setf n var-0))
      (when var-2 (setf ldc var-2)))
    (zlacgv n
     (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (1 j)
                           ((1 ldwork) (1 *)) work-%offset%)
     1)
   label70))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "Right" "Upper" "No transpose" "Unit" n k one
     (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                           ((+ m (f2cl-lib:int-sub k) 1) 1)
                           ((1 ldv) (1 *)) v-%offset%)
     ldv work ldwork)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> m k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "Conjugate transpose" "No transpose" n k
       (f2cl-lib:int-sub m k) one c ldc v ldv one work ldwork)
    (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf k var-3))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldc var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldwork var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "Right" "Lower" transt "Non-unit" n k one t$ ldt work
     ldwork)
  (declare (ignore var-0 var-1 var-3 var-7 var-9))
  (when var-2 (setf transt var-2))
  (when var-4 (setf n var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> m k)
```

```
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "No transpose" "Conjugate transpose"
       (f2cl-lib:int-sub m k) n k (- one) v ldv work ldwork one c
       ldc)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
    (when var-3 (setf n var-3))
    (when var-4 (setf k var-4))
    (when var-7 (setf ldv var-7))
    (when var-9 (setf ldwork var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldc var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "Right" "Upper" "Conjugate transpose" "Unit" n k one
     (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                           ((+ m (f2cl-lib:int-sub k) 1) 1)
                           ((1 ldv) (1 *)) v-%offset%)
     ldv work ldwork)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j k) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data%
                             ((f2cl-lib:int-add
                               (f2cl-lib:int-sub m k) j)
                              i)
                             ((1 ldc) (1 *)) c-%offset%)
              (-
               (f2cl-lib:fref c-%data%
                              ((f2cl-lib:int-add
                                (f2cl-lib:int-sub m k) j)
                               i)
                              ((1 ldc) (1 *)) c-%offset%)
               (f2cl-lib:dconjg
                (f2cl-lib:fref work-%data% (i j)
                               ((1 ldwork) (1 *))
                               work-%offset%))))
      label80))
  label90)))
```

```
((multiple-value-bind (ret-val var-0 var-1)
     (lsame side "R")
   (declare (ignore var-1))
   (when var-0 (setf side var-0))
   ret-val)
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j k) nil)
   (tagbody
     (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
         (zcopy m
          (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                (1
                                 (f2cl-lib:int-add
                                  (f2cl-lib:int-sub n k) j))
                                ((1 ldc) (1 *)) c-%offset%)
          1
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                (1 j) ((1 ldwork) (1 *))
                                work-%offset%)
          1)
       (declare (ignore var-1 var-2 var-3 var-4))
       (when var-0 (setf m var-0)))
    label100))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
      var-10)
     (ztrmm "Right" "Upper" "No transpose" "Unit" m k one
      (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                            ((+ n (f2cl-lib:int-sub k) 1) 1)
                            ((1 ldv) (1 *)) v-%offset%)
      ldv work ldwork)
   (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
   (when var-4 (setf m var-4))
   (when var-5 (setf k var-5))
   (when var-6 (setf one var-6))
   (when var-8 (setf ldv var-8))
   (when var-10 (setf ldwork var-10)))
 (cond
  ((> n k)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
        var-9 var-10 var-11 var-12)
       (zgemm "No transpose" "No transpose" m k
        (f2cl-lib:int-sub n k) one c ldc v ldv one work ldwork)
     (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
     (when var-2 (setf m var-2))
     (when var-3 (setf k var-3))
     (when var-5 (setf one var-5))
     (when var-7 (setf ldc var-7))
     (when var-9 (setf ldv var-9))
```

```
    (when var-10 (setf one var-10))
    (when var-12 (setf ldwork var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "Right" "Lower" trans "Non-unit" m k one t$ ldt work
     ldwork)
  (declare (ignore var-0 var-1 var-3 var-7 var-9))
  (when var-2 (setf trans var-2))
  (when var-4 (setf m var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> n k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "No transpose" "Conjugate transpose" m
       (f2cl-lib:int-sub n k) k (- one) work ldwork v ldv one c
       ldc)
    (declare (ignore var-0 var-1 var-3 var-5 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-4 (setf k var-4))
    (when var-7 (setf ldwork var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldc var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "Right" "Upper" "Conjugate transpose" "Unit" m k one
     (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                           ((+ n (f2cl-lib:int-sub k) 1) 1)
                           ((1 ldv) (1 *)) v-%offset%)
     ldv work ldwork)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf m var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j k) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data%
```

```
                                    (i
                                     (f2cl-lib:int-add
                                      (f2cl-lib:int-sub n k) j))
                                    ((1 ldc) (1 *)) c-%offset%)
                        (-
                         (f2cl-lib:fref c-%data%
                                        (i
                                         (f2cl-lib:int-add
                                          (f2cl-lib:int-sub n k) j))
                                        ((1 ldc) (1 *)) c-%offset%)
                         (f2cl-lib:fref work-%data% (i j)
                                        ((1 ldwork) (1 *))
                                        work-%offset%)))
            label110))
        label120)))))))
((multiple-value-bind (ret-val var-0 var-1)
     (lsame storev "R")
   (declare (ignore var-1))
   (when var-0 (setf storev var-0))
   ret-val)
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame direct "F")
     (declare (ignore var-1))
     (when var-0 (setf direct var-0))
     ret-val)
   (cond
    ((multiple-value-bind (ret-val var-0 var-1)
         (lsame side "L")
       (declare (ignore var-1))
       (when var-0 (setf side var-0))
       ret-val)
     (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                   ((> j k) nil)
       (tagbody
         (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
             (zcopy n
              (f2cl-lib:array-slice c-%data% f2cl-lib:complex16 (j 1)
                                    ((1 ldc) (1 *)) c-%offset%)
              ldc
              (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                    (1 j) ((1 ldwork) (1 *))
                                    work-%offset%)
              1)
           (declare (ignore var-1 var-3 var-4))
           (when var-0 (setf n var-0))
           (when var-2 (setf ldc var-2)))
         (zlacgv n
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (1 j)
                                ((1 ldwork) (1 *)) work-%offset%)
```

```
     1)
   label130))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "Right" "Upper" "Conjugate transpose" "Unit" n k one v
     ldv work ldwork)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> m k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "Conjugate transpose" "Conjugate transpose" n k
       (f2cl-lib:int-sub m k) one
       (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                             ((+ k 1) 1) ((1 ldc) (1 *))
                             c-%offset%)
       ldc
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             (1 (f2cl-lib:int-add k 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv one work ldwork)
    (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf k var-3))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldc var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldwork var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "Right" "Upper" transt "Non-unit" n k one t$ ldt work
     ldwork)
  (declare (ignore var-0 var-1 var-3 var-7 var-9))
  (when var-2 (setf transt var-2))
  (when var-4 (setf n var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> m k)
```

```
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (zgemm "Conjugate transpose" "Conjugate transpose"
         (f2cl-lib:int-sub m k) n k (- one)
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                               (1 (f2cl-lib:int-add k 1))
                               ((1 ldv) (1 *)) v-%offset%)
         ldv work ldwork one
         (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                               ((+ k 1) 1) ((1 ldc) (1 *))
                               c-%offset%)
         ldc)
      (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
      (when var-3 (setf n var-3))
      (when var-4 (setf k var-4))
      (when var-7 (setf ldv var-7))
      (when var-9 (setf ldwork var-9))
      (when var-10 (setf one var-10))
      (when var-12 (setf ldc var-12)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ztrmm "Right" "Upper" "No transpose" "Unit" n k one v ldv
       work ldwork)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf k var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf ldv var-8))
    (when var-10 (setf ldwork var-10)))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j k) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref c-%data% (j i) ((1 ldc) (1 *))
                               c-%offset%)
                  (-
                   (f2cl-lib:fref c-%data% (j i) ((1 ldc) (1 *))
                                  c-%offset%)
                   (f2cl-lib:dconjg
                    (f2cl-lib:fref work-%data% (i j)
                                   ((1 ldwork) (1 *))
                                   work-%offset%))))
          label140))
     label150)))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame side "R")
```

```
  (declare (ignore var-1))
  (when var-0 (setf side var-0))
  ret-val)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j k) nil)
  (tagbody
    (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
        (zcopy m
         (f2cl-lib:array-slice c-%data% f2cl-lib:complex16 (1 j)
                               ((1 ldc) (1 *)) c-%offset%)
         1
         (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (1 j) ((1 ldwork) (1 *))
                               work-%offset%)
         1)
      (declare (ignore var-1 var-2 var-3 var-4))
      (when var-0 (setf m var-0)))
   label160))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "Right" "Upper" "Conjugate transpose" "Unit" m k one v
     ldv work ldwork)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf m var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> n k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "No transpose" "Conjugate transpose" m k
       (f2cl-lib:int-sub n k) one
       (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                             (1 (f2cl-lib:int-add k 1))
                             ((1 ldc) (1 *)) c-%offset%)
       ldc
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             (1 (f2cl-lib:int-add k 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv one work ldwork)
    (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf k var-3))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldc var-7))
    (when var-9 (setf ldv var-9))
```

```
    (when var-10 (setf one var-10))
    (when var-12 (setf ldwork var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "Right" "Upper" trans "Non-unit" m k one t$ ldt work
     ldwork)
  (declare (ignore var-0 var-1 var-3 var-7 var-9))
  (when var-2 (setf trans var-2))
  (when var-4 (setf m var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> n k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "No transpose" "No transpose" m
       (f2cl-lib:int-sub n k) k (- one) work ldwork
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             (1 (f2cl-lib:int-add k 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv one
       (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                             (1 (f2cl-lib:int-add k 1))
                             ((1 ldc) (1 *)) c-%offset%)
       ldc)
    (declare (ignore var-0 var-1 var-3 var-5 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-4 (setf k var-4))
    (when var-7 (setf ldwork var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldc var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "Right" "Upper" "No transpose" "Unit" m k one v ldv
     work ldwork)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf m var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j k) nil)
  (tagbody
```

```
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i m) nil)
          (tagbody
            (setf (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                 c-%offset%)
                  (-
                   (f2cl-lib:fref c-%data% (i j) ((1 ldc) (1 *))
                                  c-%offset%)
                   (f2cl-lib:fref work-%data% (i j)
                                  ((1 ldwork) (1 *))
                                  work-%offset%)))
           label170))
      label180)))))
(t
 (cond
  ((multiple-value-bind (ret-val var-0 var-1)
       (lsame side "L")
     (declare (ignore var-1))
     (when var-0 (setf side var-0))
     ret-val)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j k) nil)
     (tagbody
       (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
           (zcopy n
            (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                  ((+ m (f2cl-lib:int-sub k) j) 1)
                                  ((1 ldc) (1 *)) c-%offset%)
            ldc
            (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                  (1 j) ((1 ldwork) (1 *))
                                  work-%offset%)
            1)
         (declare (ignore var-1 var-3 var-4))
         (when var-0 (setf n var-0))
         (when var-2 (setf ldc var-2)))
       (zlacgv n
        (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (1 j)
                              ((1 ldwork) (1 *)) work-%offset%)
        1)
      label190))
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
        var-10)
       (ztrmm "Right" "Lower" "Conjugate transpose" "Unit" n k one
        (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                              (1
                               (f2cl-lib:int-add
                                (f2cl-lib:int-sub m k) 1))
                              ((1 ldv) (1 *)) v-%offset%)
```

```
      ldv work ldwork)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf n var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> m k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "Conjugate transpose" "Conjugate transpose" n k
       (f2cl-lib:int-sub m k) one c ldc v ldv one work ldwork)
    (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
    (when var-2 (setf n var-2))
    (when var-3 (setf k var-3))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldc var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldwork var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "Right" "Lower" transt "Non-unit" n k one t$ ldt work
     ldwork)
  (declare (ignore var-0 var-1 var-3 var-7 var-9))
  (when var-2 (setf transt var-2))
  (when var-4 (setf n var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> m k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "Conjugate transpose" "Conjugate transpose"
       (f2cl-lib:int-sub m k) n k (- one) v ldv work ldwork one c
       ldc)
    (declare (ignore var-0 var-1 var-2 var-5 var-6 var-8 var-11))
    (when var-3 (setf n var-3))
    (when var-4 (setf k var-4))
    (when var-7 (setf ldv var-7))
    (when var-9 (setf ldwork var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldc var-12)))))
(multiple-value-bind
```

```
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10)
      (ztrmm "Right" "Lower" "No transpose" "Unit" n k one
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             (1
                              (f2cl-lib:int-add
                               (f2cl-lib:int-sub m k) 1))
                             ((1 ldv) (1 *)) v-%offset%)
       ldv work ldwork)
    (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
    (when var-4 (setf n var-4))
    (when var-5 (setf k var-5))
    (when var-6 (setf one var-6))
    (when var-8 (setf ldv var-8))
    (when var-10 (setf ldwork var-10)))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j k) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf (f2cl-lib:fref c-%data%
                               ((f2cl-lib:int-add
                                 (f2cl-lib:int-sub m k) j)
                                i)
                               ((1 ldc) (1 *)) c-%offset%)
                  (-
                   (f2cl-lib:fref c-%data%
                                  ((f2cl-lib:int-add
                                    (f2cl-lib:int-sub m k) j)
                                   i)
                                  ((1 ldc) (1 *)) c-%offset%)
                   (f2cl-lib:dconjg
                    (f2cl-lib:fref work-%data% (i j)
                                   ((1 ldwork) (1 *))
                                   work-%offset%))))
          label200))
     label210)))
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame side "R")
    (declare (ignore var-1))
    (when var-0 (setf side var-0))
    ret-val)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j k) nil)
    (tagbody
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (zcopy m
           (f2cl-lib:array-slice c-%data% f2cl-lib:complex16
                                 (1
```

```
                                (f2cl-lib:int-add
                                 (f2cl-lib:int-sub n k) j))
                               ((1 ldc) (1 *)) c-%offset%)
          1
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                               (1 j) ((1 ldwork) (1 *))
                               work-%offset%)
          1)
       (declare (ignore var-1 var-2 var-3 var-4))
       (when var-0 (setf m var-0)))
   label220))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "Right" "Lower" "Conjugate transpose" "Unit" m k one
     (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                           (1
                            (f2cl-lib:int-add
                             (f2cl-lib:int-sub n k) 1))
                           ((1 ldv) (1 *)) v-%offset%)
     ldv work ldwork)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf m var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> n k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "No transpose" "Conjugate transpose" m k
       (f2cl-lib:int-sub n k) one c ldc v ldv one work ldwork)
    (declare (ignore var-0 var-1 var-4 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-3 (setf k var-3))
    (when var-5 (setf one var-5))
    (when var-7 (setf ldc var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldwork var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "Right" "Lower" trans "Non-unit" m k one t$ ldt work
     ldwork)
  (declare (ignore var-0 var-1 var-3 var-7 var-9))
  (when var-2 (setf trans var-2))
  (when var-4 (setf m var-4))
```

```
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldt var-8))
  (when var-10 (setf ldwork var-10)))
(cond
 ((> n k)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10 var-11 var-12)
      (zgemm "No transpose" "No transpose" m
       (f2cl-lib:int-sub n k) k (- one) work ldwork v ldv one c
       ldc)
    (declare (ignore var-0 var-1 var-3 var-5 var-6 var-8 var-11))
    (when var-2 (setf m var-2))
    (when var-4 (setf k var-4))
    (when var-7 (setf ldwork var-7))
    (when var-9 (setf ldv var-9))
    (when var-10 (setf one var-10))
    (when var-12 (setf ldc var-12)))))
(multiple-value-bind
    (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
     var-10)
    (ztrmm "Right" "Lower" "No transpose" "Unit" m k one
     (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                           (1
                            (f2cl-lib:int-add
                             (f2cl-lib:int-sub n k) 1))
                           ((1 ldv) (1 *)) v-%offset%)
     ldv work ldwork)
  (declare (ignore var-0 var-1 var-2 var-3 var-7 var-9))
  (when var-4 (setf m var-4))
  (when var-5 (setf k var-5))
  (when var-6 (setf one var-6))
  (when var-8 (setf ldv var-8))
  (when var-10 (setf ldwork var-10)))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j k) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i m) nil)
      (tagbody
        (setf (f2cl-lib:fref c-%data%
                             (i
                              (f2cl-lib:int-add
                               (f2cl-lib:int-sub n k) j))
                             ((1 ldc) (1 *)) c-%offset%)
                (-
                 (f2cl-lib:fref c-%data%
                                (i
                                 (f2cl-lib:int-add
```

```
                                                  (f2cl-lib:int-sub n k) j))
                                                ((1 ldc) (1 *)) c-%offset%)
                                (f2cl-lib:fref work-%data% (i j)
                                               ((1 ldwork) (1 *))
                                               work-%offset%)))
                    label230))
                label240))))))))
        (go end_label)
       end_label
        (return
         (values side trans direct storev m n k nil ldv nil ldt nil ldc nil
                 ldwork))))))
```

———-

90.230 zlarf

90.230.1 Axiom unit tests

— **zlarf.input** —

```
)set break resume
)sys rm -f zlarf.output
)spool zlarf.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.230.2 Axiom help page

— **zlarf.help** —

```
======================================================================
zlarf examples
======================================================================

======================================================================
Man Page Details
```

```
=====================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZLARF( SIDE, M, N, V, INCV, TAU, C, LDC, WORK )

       .. Scalar Arguments ..
       CHARACTER          SIDE
       INTEGER            INCV, LDC, M, N
       COMPLEX*16         TAU
       ..
       .. Array Arguments ..
       COMPLEX*16         C( LDC, * ), V( * ), WORK( * )
       ..


 Purpose:
 =============

 ZLARF applies a complex elementary reflector H to a complex M-by-N
 matrix C, from either the left or the right. H is represented in the
 form

       H = I - tau * v * v**H

 where tau is a complex scalar and v is a complex vector.

 If tau = 0, then H is taken to be the unit matrix.

 To apply H**H, supply conjg(tau) instead
 tau.

 Arguments:
 ==========

  [in] SIDE

          SIDE is CHARACTER*1
          = 'L': form  H * C
          = 'R': form  C * H

  [in] M

          M is INTEGER
          The number of rows of the matrix C.
```

```
[in] N

        N is INTEGER
        The number of columns of the matrix C.

[in] V

        V is COMPLEX*16 array, dimension
                   (1 + (M-1)*abs(INCV)) if SIDE = 'L'
                or (1 + (N-1)*abs(INCV)) if SIDE = 'R'
        The vector v in the representation of H. V is not used if
        TAU = 0.

[in] INCV

        INCV is INTEGER
        The increment between elements of v. INCV <> 0.

[in] TAU

        TAU is COMPLEX*16
        The value tau in the representation of H.

[in,out] C

        C is COMPLEX*16 array, dimension (LDC,N)
        On entry, the M-by-N matrix C.
        On exit, C is overwritten by the matrix H * C if SIDE = 'L',
        or C * H if SIDE = 'R'.

[in] LDC

        LDC is INTEGER
        The leading dimension of the array C. LDC >= max(1,M).

[out] WORK

        WORK is COMPLEX*16 array, dimension
                       (N) if SIDE = 'L'
                    or (M) if SIDE = 'R'
```

———-

90.230.3 fortran code

— zlarf.f —

```
*  =====================================================================
      SUBROUTINE ZLARF( SIDE, M, N, V, INCV, TAU, C, LDC, WORK )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          SIDE
      INTEGER            INCV, LDC, M, N
      COMPLEX*16         TAU
*     ..
*     .. Array Arguments ..
      COMPLEX*16         C( LDC, * ), V( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE, ZERO
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ),
     $                   ZERO = ( 0.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            APPLYLEFT
      INTEGER            I, LASTV, LASTC
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZGEMV, ZGERC
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAZLR, ILAZLC
      EXTERNAL           LSAME, ILAZLR, ILAZLC
*     ..
*     .. Executable Statements ..
*
      APPLYLEFT = LSAME( SIDE, 'L' )
      LASTV = 0
      LASTC = 0
      IF( TAU.NE.ZERO ) THEN
*     Set up variables for scanning V.  LASTV begins pointing to the end
*     of V.
         IF( APPLYLEFT ) THEN
            LASTV = M
         ELSE
            LASTV = N
         END IF
         IF( INCV.GT.0 ) THEN
```

```
            I = 1 + (LASTV-1) * INCV
         ELSE
            I = 1
         END IF
*     Look for the last non-zero row in V.
         DO WHILE( LASTV.GT.0 .AND. V( I ).EQ.ZERO )
            LASTV = LASTV - 1
            I = I - INCV
         END DO
         IF( APPLYLEFT ) THEN
*     Scan for the last non-zero column in C(1:lastv,:).
            LASTC = ILAZLC(LASTV, N, C, LDC)
         ELSE
*     Scan for the last non-zero row in C(:,1:lastv).
            LASTC = ILAZLR(M, LASTV, C, LDC)
         END IF
      END IF
*     Note that lastc.eq.0 renders the BLAS operations null; no special
*     case is needed at this level.
      IF( APPLYLEFT ) THEN
*
*        Form  H * C
*
         IF( LASTV.GT.0 ) THEN
*
*           w(1:lastc,1) := C(1:lastv,1:lastc)**H * v(1:lastv,1)
*
            CALL ZGEMV( 'Conjugate transpose', LASTV, LASTC, ONE,
     $           C, LDC, V, INCV, ZERO, WORK, 1 )
*
*           C(1:lastv,1:lastc) := C(...) - v(1:lastv,1) * w(1:lastc,1)**H
*
            CALL ZGERC( LASTV, LASTC, -TAU, V, INCV, WORK, 1, C, LDC )
         END IF
      ELSE
*
*        Form  C * H
*
         IF( LASTV.GT.0 ) THEN
*
*           w(1:lastc,1) := C(1:lastc,1:lastv) * v(1:lastv,1)
*
            CALL ZGEMV( 'No transpose', LASTC, LASTV, ONE, C, LDC,
     $           V, INCV, ZERO, WORK, 1 )
*
*           C(1:lastc,1:lastv) := C(...) - w(1:lastc,1) * v(1:lastv,1)**H
*
            CALL ZGERC( LASTC, LASTV, -TAU, WORK, 1, V, INCV, C, LDC )
         END IF
      END IF
```

```
      RETURN
*
*     End of ZLARF
*
      END
```

———-

90.230.4 lisp code

— **zlarf.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)) (zero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (type (f2cl-lib:complex16) zero)
           (ignorable one zero))
  (defun zlarf (side m n v incv tau c ldc work)
    (declare (type (f2cl-lib:complex16) tau)
             (type (array f2cl-lib:complex16 (*)) work c v)
             (type (f2cl-lib:integer4) ldc incv n m)
             (type (string *) side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (v f2cl-lib:complex16 v-%data% v-%offset%)
         (c f2cl-lib:complex16 c-%data% c-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%))
      (prog ((i 0) (lastv 0) (lastc 0) (applyleft nil))
        (declare (type (f2cl-lib:integer4) i lastv lastc)
                 (type f2cl-lib:logical applyleft))
        (setf applyleft
                (multiple-value-bind (ret-val var-0 var-1)
                    (lsame side "L")
                  (declare (ignore var-1))
                  (when var-0 (setf side var-0))
                  ret-val))
        (setf lastv 0)
        (setf lastc 0)
        (cond
         ((/= tau zero)
          (tagbody
            (cond (applyleft (setf lastv m)) (t (setf lastv n)))
            (cond
             ((> incv 0)
              (setf i
                      (f2cl-lib:int-add 1
                                        (f2cl-lib:int-mul
                                         (f2cl-lib:int-sub lastv 1) incv))))
```

```
     (t (setf i 1)))
   label100000
    (if (not
         (and (> lastv 0)
              (= (f2cl-lib:fref v-%data% (i) ((1 *)) v-%offset%)
                 zero)))
        (go label100001))
    (setf lastv (f2cl-lib:int-sub lastv 1))
    (setf i (f2cl-lib:int-sub i incv))
    (go label100000)
   label100001
    (cond (applyleft (setf lastc (ilazlc lastv n c ldc)))
          (t (setf lastc (ilazlr m lastv c ldc)))))))
(cond
 (applyleft
  (cond
   ((> lastv 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (zgemv "Conjugate transpose" lastv lastc one c ldc v incv zero
         work 1)
      (declare (ignore var-0 var-4 var-6 var-9 var-10))
      (when var-1 (setf lastv var-1))
      (when var-2 (setf lastc var-2))
      (when var-3 (setf one var-3))
      (when var-5 (setf ldc var-5))
      (when var-7 (setf incv var-7))
      (when var-8 (setf zero var-8)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (zgerc lastv lastc (- tau) v incv work 1 c ldc)
      (declare (ignore var-2 var-3 var-5 var-6 var-7))
      (when var-0 (setf lastv var-0))
      (when var-1 (setf lastc var-1))
      (when var-4 (setf incv var-4))
      (when var-8 (setf ldc var-8))))))
 (t
  (cond
   ((> lastv 0)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
         var-10)
        (zgemv "No transpose" lastc lastv one c ldc v incv zero work 1)
      (declare (ignore var-0 var-4 var-6 var-9 var-10))
      (when var-1 (setf lastc var-1))
      (when var-2 (setf lastv var-2))
      (when var-3 (setf one var-3))
      (when var-5 (setf ldc var-5))
      (when var-7 (setf incv var-7))
```

```
            (when var-8 (setf zero var-8)))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
              (zgerc lastc lastv (- tau) work 1 v incv c ldc)
            (declare (ignore var-2 var-3 var-4 var-5 var-7))
            (when var-0 (setf lastc var-0))
            (when var-1 (setf lastv var-1))
            (when var-6 (setf incv var-6))
            (when var-8 (setf ldc var-8)))))))
      (go end_label)
     end_label
      (return (values side nil nil nil incv nil nil ldc nil))))))
```

———-

90.231 zlarfg

90.231.1 Axiom unit tests

— **zlarfg.input** —

```
)set break resume
)sys rm -f zlarfg.output
)spool zlarfg.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.231.2 Axiom help page

— **zlarfg.help** —

```
======================================================================
zlarfg examples
======================================================================

======================================================================
Man Page Details
```

```
=====================================================================


 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZLARFG( N, ALPHA, X, INCX, TAU )

       .. Scalar Arguments ..
       INTEGER            INCX, N
       COMPLEX*16         ALPHA, TAU
       ..
       .. Array Arguments ..
       COMPLEX*16         X( * )
       ..


 Purpose:
 =============

 ZLARFG generates a complex elementary reflector H of order n, such
 that

       H**H * ( alpha ) = ( beta ),   H**H * H = I.
              (   x   )   (   0  )

 where alpha and beta are scalars, with beta real, and x is an
 (n-1)-element complex vector. H is represented in the form

       H = I - tau * ( 1 ) * ( 1 v**H ) ,
                     ( v )

 where tau is a complex scalar and v is a complex (n-1)-element
 vector. Note that H is not hermitian.

 If the elements of x are all zero and alpha is real, then tau = 0
 and H is taken to be the unit matrix.

 Otherwise  1 <= real(tau) <= 2  and  abs(tau-1) <= 1 .

 Arguments:
 ==========

  [in] N

          N is INTEGER
          The order of the elementary reflector.
```

```
[in,out] ALPHA

        ALPHA is COMPLEX*16
        On entry, the value alpha.
        On exit, it is overwritten with the value beta.

[in,out] X

        X is COMPLEX*16 array, dimension
                       (1+(N-2)*abs(INCX))
        On entry, the vector x.
        On exit, it is overwritten with the vector v.

[in] INCX

        INCX is INTEGER
        The increment between elements of X. INCX > 0.

[out] TAU

        TAU is COMPLEX*16
        The value tau.
```

———-

90.231.3 fortran code

— zlzrfg.f —

```
*  ===================================================================
      SUBROUTINE ZLARFG( N, ALPHA, X, INCX, TAU )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INCX, N
      COMPLEX*16         ALPHA, TAU
*     ..
*     .. Array Arguments ..
      COMPLEX*16         X( * )
*     ..
*
*  ===================================================================
```

```
*
*     .. Parameters ..
      DOUBLE PRECISION   ONE, ZERO
      PARAMETER          ( ONE = 1.0D+0, ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            J, KNT
      DOUBLE PRECISION   ALPHI, ALPHR, BETA, RSAFMN, SAFMIN, XNORM
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH, DLAPY3, DZNRM2
      COMPLEX*16         ZLADIV
      EXTERNAL           DLAMCH, DLAPY3, DZNRM2, ZLADIV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DCMPLX, DIMAG, SIGN
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZDSCAL, ZSCAL
*     ..
*     .. Executable Statements ..
*
      IF( N.LE.0 ) THEN
         TAU = ZERO
         RETURN
      END IF
*
      XNORM = DZNRM2( N-1, X, INCX )
      ALPHR = DBLE( ALPHA )
      ALPHI = DIMAG( ALPHA )
*
      IF( XNORM.EQ.ZERO .AND. ALPHI.EQ.ZERO ) THEN
*
*        H  =  I
*
         TAU = ZERO
      ELSE
*
*        general case
*
         BETA = -SIGN( DLAPY3( ALPHR, ALPHI, XNORM ), ALPHR )
         SAFMIN = DLAMCH( 'S' ) / DLAMCH( 'E' )
         RSAFMN = ONE / SAFMIN
*
         KNT = 0
         IF( ABS( BETA ).LT.SAFMIN ) THEN
*
*           XNORM, BETA may be inaccurate; scale X and recompute them
*
   10       CONTINUE
```

```
            KNT = KNT + 1
            CALL ZDSCAL( N-1, RSAFMN, X, INCX )
            BETA = BETA*RSAFMN
            ALPHI = ALPHI*RSAFMN
            ALPHR = ALPHR*RSAFMN
            IF( ABS( BETA ).LT.SAFMIN )
     $         GO TO 10
*
*           New BETA is at most 1, at least SAFMIN
*
            XNORM = DZNRM2( N-1, X, INCX )
            ALPHA = DCMPLX( ALPHR, ALPHI )
            BETA = -SIGN( DLAPY3( ALPHR, ALPHI, XNORM ), ALPHR )
         END IF
         TAU = DCMPLX( ( BETA-ALPHR ) / BETA, -ALPHI / BETA )
         ALPHA = ZLADIV( DCMPLX( ONE ), ALPHA-BETA )
         CALL ZSCAL( N-1, ALPHA, X, INCX )
*
*        If ALPHA is subnormal, it may lose relative accuracy
*
         DO 20 J = 1, KNT
            BETA = BETA*SAFMIN
 20      CONTINUE
         ALPHA = BETA
      END IF
*
      RETURN
*
*     End of ZLARFG
*
      END
```

———-

90.231.4 lisp code

— **zlarfg.lisp** —

```
(let* ((one 1.0d0) (zero 0.0d0))
  (declare (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (ignorable one zero))
  (defun zlarfg (n alpha x incx tau)
    (declare (type (array f2cl-lib:complex16 (*)) x)
             (type (f2cl-lib:complex16) tau alpha)
             (type (f2cl-lib:integer4) incx n))
    (f2cl-lib:with-multi-array-data
```

```
  ((x f2cl-lib:complex16 x-%data% x-%offset%))
(prog ((alphi 0.0d0) (alphr 0.0d0) (beta 0.0d0) (rsafmn 0.0d0)
       (safmin 0.0d0) (xnorm 0.0d0) (j 0) (knt 0))
  (declare (type (double-float) alphi alphr beta rsafmn safmin xnorm)
           (type (f2cl-lib:integer4) j knt))
  (cond
   ((<= n 0) (setf tau (coerce zero 'f2cl-lib:complex16))
    (go end_label)))
  (setf xnorm
          (multiple-value-bind (ret-val var-0 var-1 var-2)
              (dznrm2 (f2cl-lib:int-sub n 1) x incx)
            (declare (ignore var-0 var-1))
            (when var-2 (setf incx var-2))
            ret-val))
  (setf alphr (f2cl-lib:dble alpha))
  (setf alphi (f2cl-lib:dimag alpha))
  (cond
   ((and (= xnorm zero) (= alphi zero))
    (setf tau (coerce zero 'f2cl-lib:complex16)))
   (t
    (setf beta
            (-
             (f2cl-lib:sign
              (multiple-value-bind (ret-val var-0 var-1 var-2)
                  (dlapy3 alphr alphi xnorm)
                (declare (ignore))
                (when var-0 (setf alphr var-0))
                (when var-1 (setf alphi var-1))
                (when var-2 (setf xnorm var-2))
                ret-val)
              alphr)))
    (setf safmin (/ (dlamch "S") (dlamch "E")))
    (setf rsafmn (/ one safmin)) (setf knt 0)
    (cond
     ((< (abs beta) safmin)
      (tagbody
       label10
        (setf knt (f2cl-lib:int-add knt 1))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (zdscal (f2cl-lib:int-sub n 1) rsafmn x incx)
          (declare (ignore var-0 var-2))
          (when var-1 (setf rsafmn var-1))
          (when var-3 (setf incx var-3)))
        (setf beta (* beta rsafmn))
        (setf alphi (* alphi rsafmn))
        (setf alphr (* alphr rsafmn))
        (if (< (abs beta) safmin)
            (go label10))
        (setf xnorm
                (multiple-value-bind (ret-val var-0 var-1 var-2)
```

```
                        (dznrm2 (f2cl-lib:int-sub n 1) x incx)
                      (declare (ignore var-0 var-1))
                      (when var-2 (setf incx var-2))
                      ret-val))
            (setf alpha (f2cl-lib:dcmplx alphr alphi))
            (setf beta
                    (-
                     (f2cl-lib:sign
                      (multiple-value-bind (ret-val var-0 var-1 var-2)
                          (dlapy3 alphr alphi xnorm)
                        (declare (ignore))
                        (when var-0 (setf alphr var-0))
                        (when var-1 (setf alphi var-1))
                        (when var-2 (setf xnorm var-2))
                        ret-val)
                      alphr))))))
          (setf tau
                  (f2cl-lib:dcmplx (/ (- beta alphr) beta) (/ (- alphi) beta)))
          (setf alpha (zladiv (f2cl-lib:dcmplx one) (- alpha beta)))
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (zscal (f2cl-lib:int-sub n 1) alpha x incx)
            (declare (ignore var-0 var-2))
            (when var-1 (setf alpha var-1))
            (when var-3 (setf incx var-3)))
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j knt) nil)
            (tagbody (setf beta (* beta safmin)) label20))
          (setf alpha (coerce beta 'f2cl-lib:complex16))))
        (go end_label)
      end_label
        (return (values nil alpha nil incx tau))))))
```

———-

90.232 zlarft

90.232.1 Axiom unit tests

— **zlarft.input** —

```
)set break resume
)sys rm -f zlarft.output
)spool zlarft.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

90.232.2 Axiom help page

— **zlarft.help** —

```
==========================================================================
zlarft examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZLARFT( DIRECT, STOREV, N, K, V, LDV, TAU, T, LDT )

       .. Scalar Arguments ..
       CHARACTER          DIRECT, STOREV
       INTEGER            K, LDT, LDV, N
       ..
       .. Array Arguments ..
       COMPLEX*16         T( LDT, * ), TAU( * ), V( LDV, * )
       ..


 Purpose:
 =============

 ZLARFT forms the triangular factor T of a complex block reflector H
 of order n, which is defined as a product of k elementary reflectors.

 If DIRECT = 'F', H = H(1) H(2) . . . H(k) and T is upper triangular;

 If DIRECT = 'B', H = H(k) . . . H(2) H(1) and T is lower triangular.

 If STOREV = 'C', the vector which defines the elementary reflector
 H(i) is stored in the i-th column of the array V, and
```

```
   H  =  I - V * T * V**H

If STOREV = 'R', the vector which defines the elementary reflector
H(i) is stored in the i-th row of the array V, and

   H  =  I - V**H * T * V

Arguments:
==========

 [in] DIRECT

         DIRECT is CHARACTER*1
         Specifies the order in which the elementary reflectors are
         multiplied to form the block reflector:
         = 'F': H = H(1) H(2) . . . H(k) (Forward)
         = 'B': H = H(k) . . . H(2) H(1) (Backward)

 [in] STOREV

         STOREV is CHARACTER*1
         Specifies how the vectors which define the elementary
         reflectors are stored (see also Further Details):
         = 'C': columnwise
         = 'R': rowwise

 [in] N

         N is INTEGER
         The order of the block reflector H. N >= 0.

 [in] K

         K is INTEGER
         The order of the triangular factor T (= the number of
         elementary reflectors). K >= 1.

 [in,out] V

         V is COMPLEX*16 array, dimension
                             (LDV,K) if STOREV = 'C'
                             (LDV,N) if STOREV = 'R'
         The matrix V. See further details.

 [in] LDV

         LDV is INTEGER
         The leading dimension of the array V.
         If STOREV = 'C', LDV >= max(1,N); if STOREV = 'R', LDV >= K.
```

```
 [in] TAU

         TAU is COMPLEX*16 array, dimension (K)
         TAU(i) must contain the scalar factor of the elementary
         reflector H(i).

 [out] T

         T is COMPLEX*16 array, dimension (LDT,K)
         The k by k triangular factor T of the block reflector.
         If DIRECT = 'F', T is upper triangular; if DIRECT = 'B', T is
         lower triangular. The rest of the array is not used.

 [in] LDT

         LDT is INTEGER
         The leading dimension of the array T. LDT >= K.

Further Details:
=====================

 The shape of the matrix V and the storage of the vectors which define
 the H(i) is best illustrated by the following example with n = 5 and
 k = 3. The elements equal to 1 are not stored; the corresponding
 array elements are modified but restored on exit. The rest of the
 array is not used.

 DIRECT = 'F' and STOREV = 'C':         DIRECT = 'F' and STOREV = 'R':

              V = (  1       )                 V = (  1 v1 v1 v1 v1 )
                  ( v1  1    )                     (     1 v2 v2 v2 )
                  ( v1 v2  1 )                     (        1 v3 v3 )
                  ( v1 v2 v3 )
                  ( v1 v2 v3 )

 DIRECT = 'B' and STOREV = 'C':         DIRECT = 'B' and STOREV = 'R':

              V = ( v1 v2 v3 )                 V = ( v1 v1  1       )
                  ( v1 v2 v3 )                     ( v2 v2 v2  1    )
                  (  1 v2 v3 )                     ( v3 v3 v3 v3  1 )
                  (     1 v3 )
                  (        1 )
```

90.232.3 fortran code

— zlarft.f —

```
*  =====================================================================
      SUBROUTINE ZLARFT( DIRECT, STOREV, N, K, V, LDV, TAU, T, LDT )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          DIRECT, STOREV
      INTEGER            K, LDT, LDV, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         T( LDT, * ), TAU( * ), V( LDV, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE, ZERO
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ),
     $                   ZERO = ( 0.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, PREVLASTV, LASTV
      COMPLEX*16         VII
*     ..
*     .. External Subroutines ..
      EXTERNAL           ZGEMV, ZLACGV, ZTRMV
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Executable Statements ..
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
      IF( LSAME( DIRECT, 'F' ) ) THEN
         PREVLASTV = N
         DO 20 I = 1, K
            PREVLASTV = MAX( PREVLASTV, I )
```

```
            IF( TAU( I ).EQ.ZERO ) THEN
*
*              H(i)  =  I
*
               DO 10 J = 1, I
                  T( J, I ) = ZERO
   10          CONTINUE
            ELSE
*
*              general case
*
               VII = V( I, I )
               V( I, I ) = ONE
               IF( LSAME( STOREV, 'C' ) ) THEN
!                 Skip any trailing zeros.
                  DO LASTV = N, I+1, -1
                     IF( V( LASTV, I ).NE.ZERO ) EXIT
                  END DO
                  J = MIN( LASTV, PREVLASTV )
*
*                 T(1:i-1,i) := - tau(i) * V(i:j,1:i-1)**H * V(i:j,i)
*
                  CALL ZGEMV( 'Conjugate transpose', J-I+1, I-1,
     $                        -TAU( I ), V( I, 1 ), LDV, V( I, I ), 1,
     $                        ZERO, T( 1, I ), 1 )
               ELSE
!                 Skip any trailing zeros.
                  DO LASTV = N, I+1, -1
                     IF( V( I, LASTV ).NE.ZERO ) EXIT
                  END DO
                  J = MIN( LASTV, PREVLASTV )
*
*                 T(1:i-1,i) := - tau(i) * V(1:i-1,i:j) * V(i,i:j)**H
*
                  IF( I.LT.J )
     $               CALL ZLACGV( J-I, V( I, I+1 ), LDV )
                  CALL ZGEMV( 'No transpose', I-1, J-I+1, -TAU( I ),
     $                        V( 1, I ), LDV, V( I, I ), LDV, ZERO,
     $                        T( 1, I ), 1 )
                  IF( I.LT.J )
     $               CALL ZLACGV( J-I, V( I, I+1 ), LDV )
               END IF
               V( I, I ) = VII
*
*              T(1:i-1,i) := T(1:i-1,1:i-1) * T(1:i-1,i)
*
               CALL ZTRMV( 'Upper', 'No transpose', 'Non-unit', I-1, T,
     $                     LDT, T( 1, I ), 1 )
               T( I, I ) = TAU( I )
               IF( I.GT.1 ) THEN
```

```
                 PREVLASTV = MAX( PREVLASTV, LASTV )
              ELSE
                 PREVLASTV = LASTV
              END IF
            END IF
   20    CONTINUE
      ELSE
         PREVLASTV = 1
         DO 40 I = K, 1, -1
            IF( TAU( I ).EQ.ZERO ) THEN
*
*             H(i)  =  I
*
              DO 30 J = I, K
                 T( J, I ) = ZERO
   30         CONTINUE
            ELSE
*
*             general case
*
              IF( I.LT.K ) THEN
                 IF( LSAME( STOREV, 'C' ) ) THEN
                    VII = V( N-K+I, I )
                    V( N-K+I, I ) = ONE
!                   Skip any leading zeros.
                    DO LASTV = 1, I-1
                       IF( V( LASTV, I ).NE.ZERO ) EXIT
                    END DO
                    J = MAX( LASTV, PREVLASTV )
*
*                   T(i+1:k,i) :=
*                           - tau(i) * V(j:n-k+i,i+1:k)**H * V(j:n-k+i,i)
*
                    CALL ZGEMV( 'Conjugate transpose', N-K+I-J+1, K-I,
     $                          -TAU( I ), V( J, I+1 ), LDV, V( J, I ),
     $                          1, ZERO, T( I+1, I ), 1 )
                    V( N-K+I, I ) = VII
                 ELSE
                    VII = V( I, N-K+I )
                    V( I, N-K+I ) = ONE
!                   Skip any leading zeros.
                    DO LASTV = 1, I-1
                       IF( V( I, LASTV ).NE.ZERO ) EXIT
                    END DO
                    J = MAX( LASTV, PREVLASTV )
*
*                   T(i+1:k,i) :=
*                           - tau(i) * V(i+1:k,j:n-k+i) * V(i,j:n-k+i)**H
*
                    CALL ZLACGV( N-K+I-1-J+1, V( I, J ), LDV )
```

```
                    CALL ZGEMV( 'No transpose', K-I, N-K+I-J+1,
     $                   -TAU( I ), V( I+1, J ), LDV, V( I, J ), LDV,
     $                   ZERO, T( I+1, I ), 1 )
                    CALL ZLACGV( N-K+I-1-J+1, V( I, J ), LDV )
                    V( I, N-K+I ) = VII
                 END IF
*
*                T(i+1:k,i) := T(i+1:k,i+1:k) * T(i+1:k,i)
*
                 CALL ZTRMV( 'Lower', 'No transpose', 'Non-unit', K-I,
     $                       T( I+1, I+1 ), LDT, T( I+1, I ), 1 )
                 IF( I.GT.1 ) THEN
                    PREVLASTV = MIN( PREVLASTV, LASTV )
                 ELSE
                    PREVLASTV = LASTV
                 END IF
              END IF
              T( I, I ) = TAU( I )
           END IF
   40    CONTINUE
      END IF
      RETURN
*
*     End of ZLARFT
*
      END
```

———-

90.232.4 lisp code

— **zlarft.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)) (zero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (type (f2cl-lib:complex16) zero)
           (ignorable one zero))
  (defun zlarft (direct storev n k v ldv tau t$ ldt)
    (declare (type (array f2cl-lib:complex16 (*)) t$ tau v)
             (type (f2cl-lib:integer4) ldt ldv k n)
             (type (string *) storev direct))
    (f2cl-lib:with-multi-array-data
        ((direct character direct-%data% direct-%offset%)
         (storev character storev-%data% storev-%offset%)
         (v f2cl-lib:complex16 v-%data% v-%offset%)
         (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
         (t$ f2cl-lib:complex16 t$-%data% t$-%offset%))
```

```
(prog ((i 0) (j 0) (prevlastv 0) (lastv 0))
  (declare (type (f2cl-lib:integer4) i j prevlastv lastv))
  (if (= n 0)
      (go end_label))
  (cond
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame direct "F")
      (declare (ignore var-1))
      (when var-0 (setf direct var-0))
      ret-val)
    (setf prevlastv n)
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i k) nil)
      (tagbody
        (setf prevlastv
                (max (the f2cl-lib:integer4 prevlastv)
                     (the f2cl-lib:integer4 i)))
        (cond
         ((= (f2cl-lib:fref tau (i) ((1 *))) zero)
          (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                        ((> j i) nil)
            (tagbody
              (setf (f2cl-lib:fref t$-%data% (j i) ((1 ldt) (1 *))
                                    t$-%offset%)
                    zero)
             label100001)))
         (t
          (cond
           ((multiple-value-bind (ret-val var-0 var-1)
                (lsame storev "C")
              (declare (ignore var-1))
              (when var-0 (setf storev var-0))
              ret-val)
            (f2cl-lib:fdo (lastv n
                           (f2cl-lib:int-add lastv (f2cl-lib:int-sub 1)))
                          ((> lastv (f2cl-lib:int-add i 1)) nil)
              (tagbody
                (if (/=
                     (f2cl-lib:fref v-%data% (lastv i) ((1 ldv) (1 *))
                                    v-%offset%)
                     zero)
                    (go f2cl-lib::exit))
               label100002))
            (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                          ((> j
                              (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref t$-%data% (j i) ((1 ldt) (1 *))
                                      t$-%offset%)
```

```
                (*
                 (-
                  (f2cl-lib:fref tau-%data% (i) ((1 *))
                                 tau-%offset%))
                 (f2cl-lib:conjg
                  (f2cl-lib:fref v-%data% (i j) ((1 ldv) (1 *))
                                 v-%offset%))))
    label100003))
  (setf j
          (min (the f2cl-lib:integer4 lastv)
               (the f2cl-lib:integer4 prevlastv)))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (zgemv "Conjugate transpose" (f2cl-lib:int-sub j i)
       (f2cl-lib:int-sub i 1)
       (- (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             ((+ i 1) 1) ((1 ldv) (1 *))
                             v-%offset%)
       ldv
       (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                             ((+ i 1) i) ((1 ldv) (1 *))
                             v-%offset%)
       1 one
       (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 i)
                             ((1 ldt) (1 *)) t$-%offset%)
       1)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7 var-9
      var-10))
    (when var-5 (setf ldv var-5))
    (when var-8 (setf one var-8))))
 (t
  (f2cl-lib:fdo (lastv n
                 (f2cl-lib:int-add lastv (f2cl-lib:int-sub 1)))
                ((> lastv (f2cl-lib:int-add i 1)) nil)
    (tagbody
      (if (/=
           (f2cl-lib:fref v-%data% (i lastv) ((1 ldv) (1 *))
                          v-%offset%)
           zero)
          (go f2cl-lib::exit))
    label100004))
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j
                     (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                 nil)
    (tagbody
      (setf (f2cl-lib:fref t$-%data% (j i) ((1 ldt) (1 *))
```

```
                                t$-%offset%)
                        (*
                         (-
                          (f2cl-lib:fref tau-%data% (i) ((1 *))
                                         tau-%offset%))
                         (f2cl-lib:fref v-%data% (j i) ((1 ldv) (1 *))
                                        v-%offset%)))
            label100005))
        (setf j
                (min (the f2cl-lib:integer4 lastv)
                     (the f2cl-lib:integer4 prevlastv)))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12)
            (zgemm "N" "C" (f2cl-lib:int-sub i 1) 1
             (f2cl-lib:int-sub j i)
             (- (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
             (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                                   (1 (f2cl-lib:int-add i 1))
                                   ((1 ldv) (1 *)) v-%offset%)
             ldv
             (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                                   (i (f2cl-lib:int-add i 1))
                                   ((1 ldv) (1 *)) v-%offset%)
             ldv one
             (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 i)
                                   ((1 ldt) (1 *)) t$-%offset%)
             ldt)
          (declare
           (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8
            var-11))
          (when var-7 (setf ldv var-7))
          (when var-9 (setf ldv var-9))
          (when var-10 (setf one var-10))
          (when var-12 (setf ldt var-12)))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
          (ztrmv "Upper" "No transpose" "Non-unit"
           (f2cl-lib:int-sub i 1) t$ ldt
           (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 i)
                                 ((1 ldt) (1 *)) t$-%offset%)
           1)
        (declare (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7))
        (when var-5 (setf ldt var-5)))
      (setf (f2cl-lib:fref t$-%data% (i i) ((1 ldt) (1 *))
                           t$-%offset%)
            (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
      (cond
       ((> i 1)
        (setf prevlastv
```

```
                  (max (the f2cl-lib:integer4 prevlastv)
                       (the f2cl-lib:integer4 lastv))))
        (t (setf prevlastv lastv)))))
    label100000)))
(t (setf prevlastv 1)
 (f2cl-lib:fdo (i k (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
               ((> i 1) nil)
   (tagbody
     (cond
      ((= (f2cl-lib:fref tau (i) ((1 *))) zero)
       (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                     ((> j k) nil)
         (tagbody
           (setf (f2cl-lib:fref t$-%data% (j i) ((1 ldt) (1 *))
                                t$-%offset%)
                 zero)
          label100007)))
      (t
       (cond
        ((< i k)
         (cond
          ((multiple-value-bind (ret-val var-0 var-1)
               (lsame storev "C")
             (declare (ignore var-1))
             (when var-0 (setf storev var-0))
             ret-val)
           (f2cl-lib:fdo (lastv 1 (f2cl-lib:int-add lastv 1))
                         ((> lastv
                             (f2cl-lib:int-add i
                                               (f2cl-lib:int-sub 1)))
                          nil)
             (tagbody
               (if (/=
                    (f2cl-lib:fref v-%data% (lastv i) ((1 ldv) (1 *))
                                   v-%offset%)
                    zero)
                   (go f2cl-lib::exit))
              label100008))
           (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                            (f2cl-lib:int-add j 1))
                         ((> j k) nil)
             (tagbody
               (setf (f2cl-lib:fref t$-%data% (j i) ((1 ldt) (1 *))
                                    t$-%offset%)
                     (*
                      (-
                       (f2cl-lib:fref tau-%data% (i) ((1 *))
                                      tau-%offset%))
                      (f2cl-lib:conjg
                       (f2cl-lib:fref v-%data%
```

```
                                ((f2cl-lib:int-add
                                  (f2cl-lib:int-sub n k) i)
                                 j)
                                ((1 ldv) (1 *)) v-%offset%))))
   label100009))
 (setf j
         (max (the f2cl-lib:integer4 lastv)
              (the f2cl-lib:integer4 prevlastv)))
 (multiple-value-bind
     (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
      var-9 var-10)
     (zgemv "Conjugate transpose"
      (f2cl-lib:int-sub
       (f2cl-lib:int-add (f2cl-lib:int-sub n k) i) j)
      (f2cl-lib:int-sub k i)
      (-
       (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
      (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                            (j (f2cl-lib:int-add i 1))
                            ((1 ldv) (1 *)) v-%offset%)
      ldv
      (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                            (j i) ((1 ldv) (1 *))
                            v-%offset%)
      1 one
      (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16
                            ((+ i 1) i) ((1 ldt) (1 *))
                            t$-%offset%)
      1)
   (declare
    (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7 var-9
     var-10))
   (when var-5 (setf ldv var-5))
   (when var-8 (setf one var-8))))
(t
 (f2cl-lib:fdo (lastv 1 (f2cl-lib:int-add lastv 1))
               ((> lastv
                   (f2cl-lib:int-add i
                                     (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (if (/=
          (f2cl-lib:fref v-%data% (i lastv) ((1 ldv) (1 *))
                         v-%offset%)
          zero)
         (go f2cl-lib::exit))
   label100010))
 (f2cl-lib:fdo (j (f2cl-lib:int-add i 1)
                (f2cl-lib:int-add j 1))
               ((> j k) nil)
```

```
      (tagbody
        (setf (f2cl-lib:fref t$-%data% (j i) ((1 ldt) (1 *))
                             t$-%offset%)
                (*
                 (-
                  (f2cl-lib:fref tau-%data% (i) ((1 *))
                                 tau-%offset%))
                 (f2cl-lib:fref v-%data%
                                (j
                                 (f2cl-lib:int-add
                                  (f2cl-lib:int-sub n k) i))
                                ((1 ldv) (1 *)) v-%offset%)))
       label100011))
    (setf j
            (max (the f2cl-lib:integer4 lastv)
                 (the f2cl-lib:integer4 prevlastv)))
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
         var-9 var-10 var-11 var-12)
        (zgemm "N" "C" (f2cl-lib:int-sub k i) 1
         (f2cl-lib:int-sub
          (f2cl-lib:int-add (f2cl-lib:int-sub n k) i) j)
         (-
          (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                               ((+ i 1) j) ((1 ldv) (1 *))
                               v-%offset%)
         ldv
         (f2cl-lib:array-slice v-%data% f2cl-lib:complex16
                               (i j) ((1 ldv) (1 *))
                               v-%offset%)
         ldv one
         (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16
                               ((+ i 1) i) ((1 ldt) (1 *))
                               t$-%offset%)
         ldt)
      (declare
       (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8
        var-11))
      (when var-7 (setf ldv var-7))
      (when var-9 (setf ldv var-9))
      (when var-10 (setf one var-10))
      (when var-12 (setf ldt var-12)))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
      (ztrmv "Lower" "No transpose" "Non-unit"
       (f2cl-lib:int-sub k i)
       (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16
                             ((+ i 1) (f2cl-lib:int-add i 1))
                             ((1 ldt) (1 *)) t$-%offset%)
```

```
                    ldt
                    (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16
                                          ((+ i 1) i) ((1 ldt) (1 *))
                                          t$-%offset%)
                    1)
                  (declare
                   (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7))
                  (when var-5 (setf ldt var-5)))
                (cond
                 ((> i 1)
                  (setf prevlastv
                          (min (the f2cl-lib:integer4 prevlastv)
                               (the f2cl-lib:integer4 lastv))))
                 (t (setf prevlastv lastv)))))
              (setf (f2cl-lib:fref t$-%data% (i i) ((1 ldt) (1 *))
                                   t$-%offset%)
                    (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))))
          label100006))))
     (go end_label)
    end_label
     (return (values direct storev nil nil nil ldv nil nil ldt))))))
```

———-

90.233 zlartg

90.233.1 Axiom unit tests

— **zlartg.input** —

```
)set break resume
)sys rm -f zlartg.output
)spool zlartg.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.233.2 Axiom help page

— zlartg.help —

```
=====================================================================
zlartg examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZLARTG( F, G, CS, SN, R )

       .. Scalar Arguments ..
       DOUBLE PRECISION   CS
       COMPLEX*16         F, G, R, SN
       ..


 Purpose:
 =============

 ZLARTG generates a plane rotation so that

    [  CS  SN  ]     [ F ]     [ R ]
    [  __      ]  .  [   ]  =  [   ]   where CS**2 + |SN|**2 = 1.
    [ -SN  CS  ]     [ G ]     [ 0 ]

 This is a faster version of the BLAS1 routine ZROTG, except for
 the following differences:
    F and G are unchanged on return.
    If G=0, then CS=1 and SN=0.
    If F=0, then CS=0 and SN is chosen so that R is real.

 Arguments:
 ==========

  [in] F

          F is COMPLEX*16
          The first component of vector to be rotated.
```

```
 [in] G

         G is COMPLEX*16
         The second component of vector to be rotated.

 [out] CS

         CS is DOUBLE PRECISION
         The cosine of the rotation.

 [out] SN

         SN is COMPLEX*16
         The sine of the rotation.

 [out] R

         R is COMPLEX*16
         The nonzero component of the rotated vector.


Further Details:
=====================

 3-5-96 - Modified with a new algorithm by W. Kahan and J. Demmel

 This version has a few statements commented out for thread safety
 (machine parameters are computed on each entry). 10 feb 03, SJH.
```

90.233.3 fortran code

— zlartg.f —

```
*  =====================================================================
      SUBROUTINE ZLARTG( F, G, CS, SN, R )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      DOUBLE PRECISION   CS
      COMPLEX*16         F, G, R, SN
*     ..
```

```
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   TWO, ONE, ZERO
      PARAMETER          ( TWO = 2.0D+0, ONE = 1.0D+0, ZERO = 0.0D+0 )
      COMPLEX*16         CZERO
      PARAMETER          ( CZERO = ( 0.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
*     LOGICAL            FIRST
      INTEGER            COUNT, I
      DOUBLE PRECISION   D, DI, DR, EPS, F2, F2S, G2, G2S, SAFMIN,
     $                   SAFMN2, SAFMX2, SCALE
      COMPLEX*16         FF, FS, GS
*     ..
*     .. External Functions ..
      DOUBLE PRECISION   DLAMCH, DLAPY2
      EXTERNAL           DLAMCH, DLAPY2
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DCMPLX, DCONJG, DIMAG, INT, LOG,
     $                   MAX, SQRT
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   ABS1, ABSSQ
*     ..
*     .. Save statement ..
*     SAVE               FIRST, SAFMX2, SAFMIN, SAFMN2
*     ..
*     .. Data statements ..
*     DATA               FIRST / .TRUE. /
*     ..
*     .. Statement Function definitions ..
      ABS1( FF ) = MAX( ABS( DBLE( FF ) ), ABS( DIMAG( FF ) ) )
      ABSSQ( FF ) = DBLE( FF )**2 + DIMAG( FF )**2
*     ..
*     .. Executable Statements ..
*
*     IF( FIRST ) THEN
         SAFMIN = DLAMCH( 'S' )
         EPS = DLAMCH( 'E' )
         SAFMN2 = DLAMCH( 'B' )**INT( LOG( SAFMIN / EPS ) /
     $            LOG( DLAMCH( 'B' ) ) / TWO )
         SAFMX2 = ONE / SAFMN2
*        FIRST = .FALSE.
*     END IF
      SCALE = MAX( ABS1( F ), ABS1( G ) )
      FS = F
      GS = G
```

```
      COUNT = 0
      IF( SCALE.GE.SAFMX2 ) THEN
   10    CONTINUE
         COUNT = COUNT + 1
         FS = FS*SAFMN2
         GS = GS*SAFMN2
         SCALE = SCALE*SAFMN2
         IF( SCALE.GE.SAFMX2 )
     $      GO TO 10
      ELSE IF( SCALE.LE.SAFMN2 ) THEN
         IF( G.EQ.CZERO ) THEN
            CS = ONE
            SN = CZERO
            R = F
            RETURN
         END IF
   20    CONTINUE
         COUNT = COUNT - 1
         FS = FS*SAFMX2
         GS = GS*SAFMX2
         SCALE = SCALE*SAFMX2
         IF( SCALE.LE.SAFMN2 )
     $      GO TO 20
      END IF
      F2 = ABSSQ( FS )
      G2 = ABSSQ( GS )
      IF( F2.LE.MAX( G2, ONE )*SAFMIN ) THEN
*
*        This is a rare case: F is very small.
*
         IF( F.EQ.CZERO ) THEN
            CS = ZERO
            R = DLAPY2( DBLE( G ), DIMAG( G ) )
*           Do complex/real division explicitly with two real divisions
            D = DLAPY2( DBLE( GS ), DIMAG( GS ) )
            SN = DCMPLX( DBLE( GS ) / D, -DIMAG( GS ) / D )
            RETURN
         END IF
         F2S = DLAPY2( DBLE( FS ), DIMAG( FS ) )
*        G2 and G2S are accurate
*        G2 is at least SAFMIN, and G2S is at least SAFMN2
         G2S = SQRT( G2 )
*        Error in CS from underflow in F2S is at most
*        UNFL / SAFMN2 .lt. sqrt(UNFL*EPS) .lt. EPS
*        If MAX(G2,ONE)=G2, then F2 .lt. G2*SAFMIN,
*        and so CS .lt. sqrt(SAFMIN)
*        If MAX(G2,ONE)=ONE, then F2 .lt. SAFMIN
*        and so CS .lt. sqrt(SAFMIN)/SAFMN2 = sqrt(EPS)
*        Therefore, CS = F2S/G2S / sqrt( 1 + (F2S/G2S)**2 ) = F2S/G2S
         CS = F2S / G2S
```

```
*        Make sure abs(FF) = 1
*        Do complex/real division explicitly with 2 real divisions
         IF( ABS1( F ).GT.ONE ) THEN
            D = DLAPY2( DBLE( F ), DIMAG( F ) )
            FF = DCMPLX( DBLE( F ) / D, DIMAG( F ) / D )
         ELSE
            DR = SAFMX2*DBLE( F )
            DI = SAFMX2*DIMAG( F )
            D = DLAPY2( DR, DI )
            FF = DCMPLX( DR / D, DI / D )
         END IF
         SN = FF*DCMPLX( DBLE( GS ) / G2S, -DIMAG( GS ) / G2S )
         R = CS*F + SN*G
      ELSE
*
*        This is the most common case.
*        Neither F2 nor F2/G2 are less than SAFMIN
*        F2S cannot overflow, and it is accurate
*
         F2S = SQRT( ONE+G2 / F2 )
*        Do the F2S(real)*FS(complex) multiply with two real multiplies
         R = DCMPLX( F2S*DBLE( FS ), F2S*DIMAG( FS ) )
         CS = ONE / F2S
         D = F2 + G2
*        Do complex/real division explicitly with two real divisions
         SN = DCMPLX( DBLE( R ) / D, DIMAG( R ) / D )
         SN = SN*DCONJG( GS )
         IF( COUNT.NE.0 ) THEN
            IF( COUNT.GT.0 ) THEN
               DO 30 I = 1, COUNT
                  R = R*SAFMX2
   30          CONTINUE
            ELSE
               DO 40 I = 1, -COUNT
                  R = R*SAFMN2
   40          CONTINUE
            END IF
         END IF
      END IF
      RETURN
*
*     End of ZLARTG
*
      END
```

90.233.4 lisp code

— zlartg.lisp —

```
(let* ((two 2.0d0)
       (one 1.0d0)
       (zero 0.0d0)
       (czero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (double-float 2.0d0 2.0d0) two)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 0.0d0 0.0d0) zero)
           (type (f2cl-lib:complex16) czero)
           (ignorable two one zero czero))
  (defun zlartg (f g cs sn r)
    (declare (type (double-float) cs)
             (type (f2cl-lib:complex16) r sn g f))
    (labels ((abs1 (ff)
               (max (abs (f2cl-lib:dble ff)) (abs (f2cl-lib:dimag ff))))
             (abssq (ff)
               (+ (expt (f2cl-lib:dble ff) 2) (expt (f2cl-lib:dimag ff) 2))))
      (declare
       (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
        abs1))
      (declare
       (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
        abssq))
      (prog ((ff #C(0.0d0 0.0d0)) (fs #C(0.0d0 0.0d0)) (gs #C(0.0d0 0.0d0))
             (d 0.0d0) (di 0.0d0) (dr 0.0d0) (eps 0.0d0) (f2 0.0d0) (f2s 0.0d0)
             (g2 0.0d0) (g2s 0.0d0) (safmin 0.0d0) (safmn2 0.0d0)
             (safmx2 0.0d0) (scale 0.0d0) (i 0) (count$ 0))
        (declare (type (f2cl-lib:complex16) ff fs gs)
                 (type (double-float) d di dr eps f2 f2s g2 g2s safmin safmn2
                  safmx2 scale)
                 (type (f2cl-lib:integer4) count$ i))
        (setf safmin (dlamch "S"))
        (setf eps (dlamch "E"))
        (setf safmn2
                (expt (dlamch "B")
                      (f2cl-lib:int
                       (/
                        (/ (f2cl-lib:flog (/ safmin eps))
                           (f2cl-lib:flog (dlamch "B")))
                        two))))
        (setf safmx2 (/ one safmn2))
        (setf scale (max (abs1 f) (abs1 g)))
        (setf fs f)
        (setf gs g)
        (setf count$ 0)
        (cond
```

```
 ((>= scale safmx2)
  (tagbody
   label10
    (setf count$ (f2cl-lib:int-add count$ 1))
    (setf fs (* fs safmn2))
    (setf gs (* gs safmn2))
    (setf scale (* scale safmn2))
    (if (>= scale safmx2)
        (go label10))))
 ((<= scale safmn2)
  (tagbody
    (cond
     ((or (= g czero) (disnan (abs g))) (setf cs one) (setf sn czero)
      (setf r f) (go end_label)))
   label20
    (setf count$ (f2cl-lib:int-sub count$ 1))
    (setf fs (* fs safmx2))
    (setf gs (* gs safmx2))
    (setf scale (* scale safmx2))
    (if (<= scale safmn2)
        (go label20)))))
(setf f2 (abssq fs))
(setf g2 (abssq gs))
(cond
 ((<= f2 (* (max g2 one) safmin))
  (cond
   ((= f czero) (setf cs zero)
    (setf r
            (coerce (dlapy2 (f2cl-lib:dble g) (f2cl-lib:dimag g))
                    'f2cl-lib:complex16))
    (setf d (dlapy2 (f2cl-lib:dble gs) (f2cl-lib:dimag gs)))
    (setf sn
            (f2cl-lib:dcmplx (/ (f2cl-lib:dble gs) d)
                             (/ (- (f2cl-lib:dimag gs)) d)))
    (go end_label)))
  (setf f2s (dlapy2 (f2cl-lib:dble fs) (f2cl-lib:dimag fs)))
  (setf g2s (f2cl-lib:fsqrt g2)) (setf cs (/ f2s g2s))
  (cond
   ((> (abs1 f) one)
    (setf d (dlapy2 (f2cl-lib:dble f) (f2cl-lib:dimag f)))
    (setf ff
            (f2cl-lib:dcmplx (/ (f2cl-lib:dble f) d)
                             (/ (f2cl-lib:dimag f) d))))
   (t (setf dr (* safmx2 (f2cl-lib:dble f)))
    (setf di (* safmx2 (f2cl-lib:dimag f)))
    (setf d
            (multiple-value-bind (ret-val var-0 var-1)
                (dlapy2 dr di)
              (declare (ignore))
              (when var-0 (setf dr var-0))
```

```
                    (when var-1 (setf di var-1))
                    ret-val))
          (setf ff (f2cl-lib:dcmplx (/ dr d) (/ di d)))))
        (setf sn
                (* ff
                   (f2cl-lib:dcmplx (/ (f2cl-lib:dble gs) g2s)
                                    (/ (- (f2cl-lib:dimag gs)) g2s))))
        (setf r (+ (* cs f) (* sn g))))
       (t (setf f2s (f2cl-lib:fsqrt (+ one (/ g2 f2))))
        (setf r
                (f2cl-lib:dcmplx (* f2s (f2cl-lib:dble fs))
                                 (* f2s (f2cl-lib:dimag fs))))
        (setf cs (/ one f2s)) (setf d (+ f2 g2))
        (setf sn
                (f2cl-lib:dcmplx (/ (f2cl-lib:dble r) d)
                                 (/ (f2cl-lib:dimag r) d)))
        (setf sn (* sn (f2cl-lib:dconjg gs)))
        (cond
         ((/= count$ 0)
          (cond
           ((> count$ 0)
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i count$) nil)
              (tagbody (setf r (* r safmx2)) label30)))
           (t
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i (f2cl-lib:int-sub count$)) nil)
              (tagbody (setf r (* r safmn2)) label40))))))))
     (go end_label)
    end_label
     (return (values nil nil cs sn r))))))
```

———-

90.234 zlascl

90.234.1 Axiom unit tests

— **zlascl.input** —

```
)set break resume
)sys rm -f zlascl.output
)spool zlascl.output
)set message test on
)set message auto off
)clear all
```

```
)spool
)lisp (bye)
```

———-

90.234.2 Axiom help page

— **zlascl.help** —

```
=====================================================================
zlascl examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZLASCL( TYPE, KL, KU, CFROM, CTO, M, N, A, LDA, INFO )

       .. Scalar Arguments ..
       CHARACTER          TYPE
       INTEGER            INFO, KL, KU, LDA, M, N
       DOUBLE PRECISION   CFROM, CTO
       ..
       .. Array Arguments ..
       COMPLEX*16         A( LDA, * )
       ..


 Purpose:
 =============

 ZLASCL multiplies the M by N complex matrix A by the real scalar
 CTO/CFROM.  This is done without over/underflow as long as the final
 result CTO*A(I,J)/CFROM does not over/underflow. TYPE specifies that
 A may be full, upper triangular, lower triangular, upper Hessenberg,
 or banded.

 Arguments:
 ==========
```

```
[in] TYPE

        TYPE is CHARACTER*1
        TYPE indices the storage type of the input matrix.
        = 'G':  A is a full matrix.
        = 'L':  A is a lower triangular matrix.
        = 'U':  A is an upper triangular matrix.
        = 'H':  A is an upper Hessenberg matrix.
        = 'B':  A is a symmetric band matrix with lower bandwidth KL
                and upper bandwidth KU and with the only the lower
                half stored.
        = 'Q':  A is a symmetric band matrix with lower bandwidth KL
                and upper bandwidth KU and with the only the upper
                half stored.
        = 'Z':  A is a band matrix with lower bandwidth KL and upper
                bandwidth KU. See ZGBTRF for storage details.

[in] KL

        KL is INTEGER
        The lower bandwidth of A.  Referenced only if TYPE = 'B',
        'Q' or 'Z'.

[in] KU

        KU is INTEGER
        The upper bandwidth of A.  Referenced only if TYPE = 'B',
        'Q' or 'Z'.

[in] CFROM

        CFROM is DOUBLE PRECISION

[in] CTO

        CTO is DOUBLE PRECISION

        The matrix A is multiplied by CTO/CFROM. A(I,J) is computed
        without over/underflow if the final result CTO*A(I,J)/CFROM
        can be represented without over/underflow.  CFROM must be
        nonzero.

[in] M

        M is INTEGER
        The number of rows of the matrix A.  M >= 0.

[in] N

        N is INTEGER
```

```
          The number of columns of the matrix A.  N >= 0.

  [in,out] A

          A is COMPLEX*16 array, dimension (LDA,N)
          The matrix to be multiplied by CTO/CFROM.  See TYPE for the
          storage type.

  [in] LDA

          LDA is INTEGER
          The leading dimension of the array A.  LDA >= max(1,M).

  [out] INFO

          INFO is INTEGER
          0  - successful exit
          <0 - if INFO = -i, the i-th argument had an illegal value.
```

90.234.3 fortran code

— zlascl.f —

```
*  =====================================================================
      SUBROUTINE ZLASCL( TYPE, KL, KU, CFROM, CTO, M, N, A, LDA, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          TYPE
      INTEGER            INFO, KL, KU, LDA, M, N
      DOUBLE PRECISION   CFROM, CTO
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D0, ONE = 1.0D0 )
```

```
*     ..
*     .. Local Scalars ..
      LOGICAL            DONE
      INTEGER            I, ITYPE, J, K1, K2, K3, K4
      DOUBLE PRECISION   BIGNUM, CFROM1, CFROMC, CTO1, CTOC, MUL, SMLNUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME, DISNAN
      DOUBLE PRECISION   DLAMCH
      EXTERNAL           LSAME, DLAMCH, DISNAN
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, MAX, MIN
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
*
      IF( LSAME( TYPE, 'G' ) ) THEN
         ITYPE = 0
      ELSE IF( LSAME( TYPE, 'L' ) ) THEN
         ITYPE = 1
      ELSE IF( LSAME( TYPE, 'U' ) ) THEN
         ITYPE = 2
      ELSE IF( LSAME( TYPE, 'H' ) ) THEN
         ITYPE = 3
      ELSE IF( LSAME( TYPE, 'B' ) ) THEN
         ITYPE = 4
      ELSE IF( LSAME( TYPE, 'Q' ) ) THEN
         ITYPE = 5
      ELSE IF( LSAME( TYPE, 'Z' ) ) THEN
         ITYPE = 6
      ELSE
         ITYPE = -1
      END IF
*
      IF( ITYPE.EQ.-1 ) THEN
         INFO = -1
      ELSE IF( CFROM.EQ.ZERO .OR. DISNAN(CFROM) ) THEN
         INFO = -4
      ELSE IF( DISNAN(CTO) ) THEN
         INFO = -5
      ELSE IF( M.LT.0 ) THEN
         INFO = -6
      ELSE IF( N.LT.0 .OR. ( ITYPE.EQ.4 .AND. N.NE.M ) .OR.
```

```
     $         ( ITYPE.EQ.5 .AND. N.NE.M ) ) THEN
         INFO = -7
      ELSE IF( ITYPE.LE.3 .AND. LDA.LT.MAX( 1, M ) ) THEN
         INFO = -9
      ELSE IF( ITYPE.GE.4 ) THEN
         IF( KL.LT.0 .OR. KL.GT.MAX( M-1, 0 ) ) THEN
            INFO = -2
         ELSE IF( KU.LT.0 .OR. KU.GT.MAX( N-1, 0 ) .OR.
     $           ( ( ITYPE.EQ.4 .OR. ITYPE.EQ.5 ) .AND. KL.NE.KU ) )
     $            THEN
            INFO = -3
         ELSE IF( ( ITYPE.EQ.4 .AND. LDA.LT.KL+1 ) .OR.
     $           ( ITYPE.EQ.5 .AND. LDA.LT.KU+1 ) .OR.
     $           ( ITYPE.EQ.6 .AND. LDA.LT.2*KL+KU+1 ) ) THEN
            INFO = -9
         END IF
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZLASCL', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 .OR. M.EQ.0 )
     $   RETURN
*
*     Get machine parameters
*
      SMLNUM = DLAMCH( 'S' )
      BIGNUM = ONE / SMLNUM
*
      CFROMC = CFROM
      CTOC = CTO
*
   10 CONTINUE
      CFROM1 = CFROMC*SMLNUM
      IF( CFROM1.EQ.CFROMC ) THEN
!        CFROMC is an inf.  Multiply by a correctly signed zero for
!        finite CTOC, or a NaN if CTOC is infinite.
         MUL = CTOC / CFROMC
         DONE = .TRUE.
         CTO1 = CTOC
      ELSE
         CTO1 = CTOC / BIGNUM
         IF( CTO1.EQ.CTOC ) THEN
!           CTOC is either 0 or an inf.  In both cases, CTOC itself
!           serves as the correct multiplication factor.
            MUL = CTOC
```

```
            DONE = .TRUE.
            CFROMC = ONE
         ELSE IF( ABS( CFROM1 ).GT.ABS( CTOC ) .AND. CTOC.NE.ZERO ) THEN
            MUL = SMLNUM
            DONE = .FALSE.
            CFROMC = CFROM1
         ELSE IF( ABS( CTO1 ).GT.ABS( CFROMC ) ) THEN
            MUL = BIGNUM
            DONE = .FALSE.
            CTOC = CTO1
         ELSE
            MUL = CTOC / CFROMC
            DONE = .TRUE.
         END IF
      END IF
*
      IF( ITYPE.EQ.0 ) THEN
*
*        Full matrix
*
         DO 30 J = 1, N
            DO 20 I = 1, M
               A( I, J ) = A( I, J )*MUL
   20       CONTINUE
   30    CONTINUE
*
      ELSE IF( ITYPE.EQ.1 ) THEN
*
*        Lower triangular matrix
*
         DO 50 J = 1, N
            DO 40 I = J, M
               A( I, J ) = A( I, J )*MUL
   40       CONTINUE
   50    CONTINUE
*
      ELSE IF( ITYPE.EQ.2 ) THEN
*
*        Upper triangular matrix
*
         DO 70 J = 1, N
            DO 60 I = 1, MIN( J, M )
               A( I, J ) = A( I, J )*MUL
   60       CONTINUE
   70    CONTINUE
*
      ELSE IF( ITYPE.EQ.3 ) THEN
*
*        Upper Hessenberg matrix
*
```

```
         DO 90 J = 1, N
            DO 80 I = 1, MIN( J+1, M )
               A( I, J ) = A( I, J )*MUL
   80       CONTINUE
   90    CONTINUE
*
      ELSE IF( ITYPE.EQ.4 ) THEN
*
*        Lower half of a symmetric band matrix
*
         K3 = KL + 1
         K4 = N + 1
         DO 110 J = 1, N
            DO 100 I = 1, MIN( K3, K4-J )
               A( I, J ) = A( I, J )*MUL
  100       CONTINUE
  110    CONTINUE
*
      ELSE IF( ITYPE.EQ.5 ) THEN
*
*        Upper half of a symmetric band matrix
*
         K1 = KU + 2
         K3 = KU + 1
         DO 130 J = 1, N
            DO 120 I = MAX( K1-J, 1 ), K3
               A( I, J ) = A( I, J )*MUL
  120       CONTINUE
  130    CONTINUE
*
      ELSE IF( ITYPE.EQ.6 ) THEN
*
*        Band matrix
*
         K1 = KL + KU + 2
         K2 = KL + 1
         K3 = 2*KL + KU + 1
         K4 = KL + KU + 1 + M
         DO 150 J = 1, N
            DO 140 I = MAX( K1-J, K2 ), MIN( K3, K4-J )
               A( I, J ) = A( I, J )*MUL
  140       CONTINUE
  150    CONTINUE
*
      END IF
*
      IF( .NOT.DONE )
     $   GO TO 10
*
      RETURN
```

```
*
*     End of ZLASCL
*
      END
```

———-

90.234.4 lisp code

— **zlascl.lisp** —

```
(let* ((zero 0.0d0) (one 1.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (ignorable zero one))
  (defun zlascl (type kl ku cfrom cto m n a lda info)
    (declare (type (array f2cl-lib:complex16 (*)) a)
             (type (double-float) cto cfrom)
             (type (f2cl-lib:integer4) info lda n m ku kl)
             (type (string *) type))
    (f2cl-lib:with-multi-array-data
        ((type character type-%data% type-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%))
      (prog ((bignum 0.0d0) (cfrom1 0.0d0) (cfromc 0.0d0) (cto1 0.0d0)
             (ctoc 0.0d0) (mul 0.0d0) (smlnum 0.0d0) (i 0) (itype 0) (j 0)
             (k1 0) (k2 0) (k3 0) (k4 0) (done nil))
        (declare
         (type (double-float) bignum cfrom1 cfromc cto1 ctoc mul smlnum)
         (type (f2cl-lib:integer4) i itype j k1 k2 k3 k4)
         (type f2cl-lib:logical done))
        (setf info 0)
        (cond
         ((multiple-value-bind (ret-val var-0 var-1)
              (lsame type "G")
            (declare (ignore var-1))
            (when var-0 (setf type var-0))
            ret-val)
          (setf itype 0))
         ((multiple-value-bind (ret-val var-0 var-1)
              (lsame type "L")
            (declare (ignore var-1))
            (when var-0 (setf type var-0))
            ret-val)
          (setf itype 1))
         ((multiple-value-bind (ret-val var-0 var-1)
              (lsame type "U")
            (declare (ignore var-1))
```

```
      (when var-0 (setf type var-0))
      ret-val)
    (setf itype 2))
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame type "H")
      (declare (ignore var-1))
      (when var-0 (setf type var-0))
      ret-val)
    (setf itype 3))
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame type "B")
      (declare (ignore var-1))
      (when var-0 (setf type var-0))
      ret-val)
    (setf itype 4))
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame type "Q")
      (declare (ignore var-1))
      (when var-0 (setf type var-0))
      ret-val)
    (setf itype 5))
   ((multiple-value-bind (ret-val var-0 var-1)
        (lsame type "Z")
      (declare (ignore var-1))
      (when var-0 (setf type var-0))
      ret-val)
    (setf itype 6))
   (t (setf itype -1)))
  (cond ((= itype (f2cl-lib:int-sub 1)) (setf info -1))
        ((or (= cfrom zero)
             (multiple-value-bind (ret-val var-0)
                 (disnan cfrom)
               (declare (ignore))
               (when var-0 (setf cfrom var-0))
               ret-val))
         (setf info -4))
        ((multiple-value-bind (ret-val var-0)
             (disnan cto)
           (declare (ignore))
           (when var-0 (setf cto var-0))
           ret-val)
         (setf info -5))
        ((< m 0) (setf info -6))
        ((or (< n 0) (and (= itype 4) (/= n m))
             (and (= itype 5) (/= n m)))
         (setf info -7))
        ((and (<= itype 3)
              (< lda
                 (max (the f2cl-lib:integer4 1)
                      (the f2cl-lib:integer4 m))))
```

```
          (setf info -9))
         ((>= itype 4)
          (cond
           ((or (< kl 0)
                (> kl
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add m (f2cl-lib:int-sub 1)))
                    (the f2cl-lib:integer4 0))))
            (setf info -2))
           ((or (< ku 0)
                (> ku
                   (max
                    (the f2cl-lib:integer4
                         (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                    (the f2cl-lib:integer4 0)))
                (and (or (= itype 4) (= itype 5)) (/= kl ku)))
            (setf info -3))
           ((or (and (= itype 4) (< lda (f2cl-lib:int-add kl 1)))
                (and (= itype 5) (< lda (f2cl-lib:int-add ku 1)))
                (and (= itype 6)
                     (< lda
                        (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))))
            (setf info -9)))))
 (cond
  ((/= info 0) (xerbla "ZLASCL" (f2cl-lib:int-sub info))
   (go end_label)))
 (if (or (= n 0) (= m 0))
     (go end_label))
 (setf smlnum (dlamch "S"))
 (setf bignum (/ one smlnum))
 (setf cfromc cfrom)
 (setf ctoc cto)
label10
 (setf cfrom1 (* cfromc smlnum))
 (cond
  ((= cfrom1 cfromc) (setf mul (/ ctoc cfromc))
   (setf done f2cl-lib:%true%) (setf cto1 ctoc))
  (t (setf cto1 (/ ctoc bignum))
   (cond
    ((= cto1 ctoc) (setf mul ctoc) (setf done f2cl-lib:%true%)
     (setf cfromc one))
    ((and (> (abs cfrom1) (abs ctoc)) (/= ctoc zero)) (setf mul smlnum)
     (setf done f2cl-lib:%false%) (setf cfromc cfrom1))
    ((> (abs cto1) (abs cfromc)) (setf mul bignum)
     (setf done f2cl-lib:%false%) (setf ctoc cto1))
    (t (setf mul (/ ctoc cfromc)) (setf done f2cl-lib:%true%)))))
 (cond
  ((= itype 0)
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
```

```
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                  (*
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                   mul))
         label20))
     label30)))
 ((= itype 1)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i j (f2cl-lib:int-add i 1))
                    ((> i m) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                  (*
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                   mul))
         label40))
     label50)))
 ((= itype 2)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (min (the f2cl-lib:integer4 j)
                             (the f2cl-lib:integer4 m)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                  (*
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                   mul))
         label60))
     label70)))
 ((= itype 3)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
```

```
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (min
                         (the f2cl-lib:integer4 (f2cl-lib:int-add j 1))
                         (the f2cl-lib:integer4 m)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                  (*
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                   mul))
         label80))
    label90)))
((= itype 4) (setf k3 (f2cl-lib:int-add kl 1))
 (setf k4 (f2cl-lib:int-add n 1))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                   ((> i
                       (min (the f2cl-lib:integer4 k3)
                            (the f2cl-lib:integer4
                                 (f2cl-lib:int-add k4
                                                   (f2cl-lib:int-sub
                                                    j)))))
                    nil)
       (tagbody
         (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%)
                 (*
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%)
                  mul))
        label100))
    label110)))
((= itype 5) (setf k1 (f2cl-lib:int-add ku 2))
 (setf k3 (f2cl-lib:int-add ku 1))
 (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
               ((> j n) nil)
   (tagbody
     (f2cl-lib:fdo (i
                    (max
                     (the f2cl-lib:integer4
                          (f2cl-lib:int-add k1 (f2cl-lib:int-sub j)))
                     (the f2cl-lib:integer4 1))
                    (f2cl-lib:int-add i 1))
                   ((> i k3) nil)
       (tagbody
```

```
            (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%)
                  (*
                   (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                  a-%offset%)
                   mul))
          label120))
      label130)))
  ((= itype 6) (setf k1 (f2cl-lib:int-add kl ku 2))
   (setf k2 (f2cl-lib:int-add kl 1))
   (setf k3 (f2cl-lib:int-add (f2cl-lib:int-mul 2 kl) ku 1))
   (setf k4 (f2cl-lib:int-add kl ku 1 m))
   (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                 ((> j n) nil)
     (tagbody
       (f2cl-lib:fdo (i
                      (max
                       (the f2cl-lib:integer4
                            (f2cl-lib:int-add k1 (f2cl-lib:int-sub j)))
                       (the f2cl-lib:integer4 k2))
                      (f2cl-lib:int-add i 1))
                     ((> i
                         (min (the f2cl-lib:integer4 k3)
                              (the f2cl-lib:integer4
                                   (f2cl-lib:int-add k4
                                                     (f2cl-lib:int-sub
                                                      j)))))
                      nil)
         (tagbody
           (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                a-%offset%)
                 (*
                  (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                                 a-%offset%)
                  mul))
          label140))
      label150))))
 (if (not done)
     (go label10))
 (go end_label)
end_label
 (return (values type nil nil cfrom cto nil nil nil nil info))))))
```

———-

90.235 zlaset

90.235.1 Axiom unit tests

— zlaset.input —

```
)set break resume
)sys rm -f zlaset.output
)spool zlaset.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.235.2 Axiom help page

— zlaset.help —

```
==================================================================
zlaset examples
==================================================================

==================================================================
Man Page Details
==================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZLASET( UPLO, M, N, ALPHA, BETA, A, LDA )

       .. Scalar Arguments ..
       CHARACTER          UPLO
       INTEGER            LDA, M, N
       COMPLEX*16         ALPHA, BETA
       ..
       .. Array Arguments ..
       COMPLEX*16         A( LDA, * )
       ..
```

```
Purpose:
=============

ZLASET initializes a 2-D array A to BETA on the diagonal and
ALPHA on the offdiagonals.

Arguments:
==========

 [in] UPLO

         UPLO is CHARACTER*1
         Specifies the part of the matrix A to be set.
         = 'U':      Upper triangular part is set. The lower triangle
                     is unchanged.
         = 'L':      Lower triangular part is set. The upper triangle
                     is unchanged.
         Otherwise:  All of the matrix A is set.

 [in] M

         M is INTEGER
         On entry, M specifies the number of rows of A.

 [in] N

         N is INTEGER
         On entry, N specifies the number of columns of A.

 [in] ALPHA

         ALPHA is COMPLEX*16
         All the offdiagonal array elements are set to ALPHA.

 [in] BETA

         BETA is COMPLEX*16
         All the diagonal array elements are set to BETA.

 [in,out] A

         A is COMPLEX*16 array, dimension (LDA,N)
         On entry, the m by n matrix A.
         On exit, A(i,j) = ALPHA, 1 <= i <= m, 1 <= j <= n, i.ne.j;
                  A(i,i) = BETA , 1 <= i <= min(m,n)

 [in] LDA
```

```
         LDA is INTEGER
         The leading dimension of the array A.  LDA >= max(1,M).
```

———-

90.235.3 fortran code

— **zlaset.f** —

```
*  =====================================================================
      SUBROUTINE ZLASET( UPLO, M, N, ALPHA, BETA, A, LDA )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          UPLO
      INTEGER            LDA, M, N
      COMPLEX*16         ALPHA, BETA
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, J
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MIN
*     ..
*     .. Executable Statements ..
*
      IF( LSAME( UPLO, 'U' ) ) THEN
*
*        Set the diagonal to BETA and the strictly upper triangular
*        part of the array to ALPHA.
*
         DO 20 J = 2, N
            DO 10 I = 1, MIN( J-1, M )
```

```
               A( I, J ) = ALPHA
   10       CONTINUE
   20    CONTINUE
         DO 30 I = 1, MIN( N, M )
            A( I, I ) = BETA
   30    CONTINUE
*
      ELSE IF( LSAME( UPLO, 'L' ) ) THEN
*
*        Set the diagonal to BETA and the strictly lower triangular
*        part of the array to ALPHA.
*
         DO 50 J = 1, MIN( M, N )
            DO 40 I = J + 1, M
               A( I, J ) = ALPHA
   40       CONTINUE
   50    CONTINUE
         DO 60 I = 1, MIN( N, M )
            A( I, I ) = BETA
   60    CONTINUE
*
      ELSE
*
*        Set the array to BETA on the diagonal and ALPHA on the
*        offdiagonal.
*
         DO 80 J = 1, N
            DO 70 I = 1, M
               A( I, J ) = ALPHA
   70       CONTINUE
   80    CONTINUE
         DO 90 I = 1, MIN( M, N )
            A( I, I ) = BETA
   90    CONTINUE
      END IF
*
      RETURN
*
*     End of ZLASET
*
      END
```

———-

90.235.4 lisp code

— zlaset.lisp —

```
(defun zlaset (uplo m n alpha beta a lda)
  (declare (type (array f2cl-lib:complex16 (*)) a)
           (type (f2cl-lib:complex16) beta alpha)
           (type (f2cl-lib:integer4) lda n m)
           (type (string *) uplo))
  (f2cl-lib:with-multi-array-data
      ((uplo character uplo-%data% uplo-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%))
    (prog ((i 0) (j 0))
      (declare (type (f2cl-lib:integer4) j i))
      (cond
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "U")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)
        (f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                          ((> i
                              (min
                               (the f2cl-lib:integer4
                                    (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
                               (the f2cl-lib:integer4 m)))
                           nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                        alpha)
              label10))
          label20))
        (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                      ((> i
                          (min (the f2cl-lib:integer4 n)
                               (the f2cl-lib:integer4 m)))
                       nil)
          (tagbody
            (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                    beta)
          label30)))
       ((multiple-value-bind (ret-val var-0 var-1)
            (lsame uplo "L")
          (declare (ignore var-1))
          (when var-0 (setf uplo var-0))
          ret-val)
        (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                      ((> j
                          (min (the f2cl-lib:integer4 m)
                               (the f2cl-lib:integer4 n)))
                       nil)
```

```
        (tagbody
          (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                      alpha)
             label40))
         label50))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (min (the f2cl-lib:integer4 n)
                             (the f2cl-lib:integer4 m)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                  beta)
         label60)))
     (t
      (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                    ((> j n) nil)
        (tagbody
          (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                        ((> i m) nil)
            (tagbody
              (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                      alpha)
             label70))
         label80))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i
                        (min (the f2cl-lib:integer4 m)
                             (the f2cl-lib:integer4 n)))
                     nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                  beta)
         label90))))
    (go end_label)
   end_label
    (return (values uplo nil nil nil nil nil nil)))))
```

———-

90.236 zlassq

90.236.1 Axiom unit tests

— **zlassq.input** —

```
)set break resume
)sys rm -f zlassq.output
)spool zlassq.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.236.2 Axiom help page

— **zlassq.help** —

```
==================================================================
zlassq examples
==================================================================

==================================================================
Man Page Details
==================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZLASSQ( N, X, INCX, SCALE, SUMSQ )

       .. Scalar Arguments ..
       INTEGER            INCX, N
       DOUBLE PRECISION   SCALE, SUMSQ
       ..
       .. Array Arguments ..
       COMPLEX*16         X( * )
       ..
```

```
Purpose:
=============

ZLASSQ returns the values scl and ssq such that

   ( scl**2 )*ssq = x( 1 )**2 +...+ x( n )**2 + ( scale**2 )*sumsq,

where x( i ) = abs( X( 1 + ( i - 1 )*INCX ) ). The value of sumsq is
assumed to be at least unity and the value of ssq will then satisfy

   1.0 .le. ssq .le. ( sumsq + 2*n ).

scale is assumed to be non-negative and scl returns the value

   scl = max( scale, abs( real( x( i ) ) ), abs( aimag( x( i ) ) ) ),
          i

scale and sumsq must be supplied in SCALE and SUMSQ respectively.
SCALE and SUMSQ are overwritten by scl and ssq respectively.

The routine makes only one pass through the vector X.

Arguments:
==========

 [in] N

         N is INTEGER
         The number of elements to be used from the vector X.

 [in] X

         X is COMPLEX*16 array, dimension (N)
         The vector x as described above.
            x( i )  = X( 1 + ( i - 1 )*INCX ), 1 <= i <= n.

 [in] INCX

         INCX is INTEGER
         The increment between successive values of the vector X.
         INCX > 0.

 [in,out] SCALE

         SCALE is DOUBLE PRECISION
         On entry, the value  scale  in the equation above.
         On exit, SCALE is overwritten with the value  scl .

 [in,out] SUMSQ
```

```
         SUMSQ is DOUBLE PRECISION
         On entry, the value  sumsq  in the equation above.
         On exit, SUMSQ is overwritten with the value  ssq .
```

———-

90.236.3 fortran code

— **zlassq.f** —

```
*  =====================================================================
      SUBROUTINE ZLASSQ( N, X, INCX, SCALE, SUMSQ )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INCX, N
      DOUBLE PRECISION   SCALE, SUMSQ
*     ..
*     .. Array Arguments ..
      COMPLEX*16         X( * )
*     ..
*
* =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO
      PARAMETER          ( ZERO = 0.0D+0 )
*     ..
*     .. Local Scalars ..
      INTEGER            IX
      DOUBLE PRECISION   TEMP1
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DIMAG
*     ..
*     .. Executable Statements ..
*
      IF( N.GT.0 ) THEN
         DO 10 IX = 1, 1 + ( N-1 )*INCX, INCX
            IF( DBLE( X( IX ) ).NE.ZERO ) THEN
               TEMP1 = ABS( DBLE( X( IX ) ) )
               IF( SCALE.LT.TEMP1 ) THEN
```

```
                 SUMSQ = 1 + SUMSQ*( SCALE / TEMP1 )**2
                 SCALE = TEMP1
              ELSE
                 SUMSQ = SUMSQ + ( TEMP1 / SCALE )**2
              END IF
           END IF
           IF( DIMAG( X( IX ) ).NE.ZERO ) THEN
              TEMP1 = ABS( DIMAG( X( IX ) ) )
              IF( SCALE.LT.TEMP1 ) THEN
                 SUMSQ = 1 + SUMSQ*( SCALE / TEMP1 )**2
                 SCALE = TEMP1
              ELSE
                 SUMSQ = SUMSQ + ( TEMP1 / SCALE )**2
              END IF
           END IF
   10    CONTINUE
      END IF
*
      RETURN
*
*     End of ZLASSQ
*
      END
```

———-

90.236.4 lisp code

— **zlassq.lisp** —

```
(let* ((zero 0.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (ignorable zero))
  (defun zlassq (n x incx scale sumsq)
    (declare (type (double-float) sumsq scale)
             (type (array f2cl-lib:complex16 (*)) x)
             (type (f2cl-lib:integer4) incx n))
    (f2cl-lib:with-multi-array-data
        ((x f2cl-lib:complex16 x-%data% x-%offset%))
      (prog ((temp1 0.0d0) (ix 0))
        (declare (type (double-float) temp1)
                 (type (f2cl-lib:integer4) ix))
        (cond
         ((> n 0)
          (f2cl-lib:fdo (ix 1 (f2cl-lib:int-add ix incx))
                        ((> ix
                            (f2cl-lib:int-add 1
```

```
                                        (f2cl-lib:int-mul
                                         (f2cl-lib:int-add n
                                                           (f2cl-lib:int-sub
                                                            1))
                                         incx)))
                    nil)
          (tagbody
            (setf temp1
                    (abs
                     (f2cl-lib:dble
                      (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%))))
            (cond
             ((or (> temp1 zero)
                  (multiple-value-bind (ret-val var-0)
                      (disnan temp1)
                    (declare (ignore))
                    (when var-0 (setf temp1 var-0))
                    ret-val))
              (cond
               ((< scale temp1)
                (setf sumsq (+ 1 (* sumsq (expt (/ scale temp1) 2))))
                (setf scale temp1))
               (t (setf sumsq (+ sumsq (expt (/ temp1 scale) 2)))))))
            (setf temp1
                    (abs
                     (f2cl-lib:dimag
                      (f2cl-lib:fref x-%data% (ix) ((1 *)) x-%offset%))))
            (cond
             ((or (> temp1 zero)
                  (multiple-value-bind (ret-val var-0)
                      (disnan temp1)
                    (declare (ignore))
                    (when var-0 (setf temp1 var-0))
                    ret-val))
              (cond
               ((< scale temp1)
                (setf sumsq (+ 1 (* sumsq (expt (/ scale temp1) 2))))
                (setf scale temp1))
               (t (setf sumsq (+ sumsq (expt (/ temp1 scale) 2)))))))
          label10))))
 (go end_label)
end_label
 (return (values nil nil nil scale sumsq))))))
```

———-

90.237 zlatrs

90.237.1 Axiom unit tests

— zlatrs.input —

```
)set break resume
)sys rm -f zlatrs.output
)spool zlatrs.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.237.2 Axiom help page

— zlatrs.help —

```
======================================================================
zlatrs examples
======================================================================

======================================================================
Man Page Details
======================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZLATRS( UPLO, TRANS, DIAG, NORMIN, N, A, LDA, X, SCALE,
                          CNORM, INFO )

       .. Scalar Arguments ..
       CHARACTER          DIAG, NORMIN, TRANS, UPLO
       INTEGER            INFO, LDA, N
       DOUBLE PRECISION   SCALE
       ..
       .. Array Arguments ..
       DOUBLE PRECISION   CNORM( * )
```

```
      COMPLEX*16         A( LDA, * ), X( * )
      ..


Purpose:
=============

ZLATRS solves one of the triangular systems

   A * x = s*b,  A**T * x = s*b,  or  A**H * x = s*b,

with scaling to prevent overflow.  Here A is an upper or lower
triangular matrix, A**T denotes the transpose of A, A**H denotes the
conjugate transpose of A, x and b are n-element vectors, and s is a
scaling factor, usually less than or equal to 1, chosen so that the
components of x will be less than the overflow threshold.  If the
unscaled problem will not cause overflow, the Level 2 BLAS routine
ZTRSV is called. If the matrix A is singular (A(j,j) = 0 for some j),
then s is set to 0 and a non-trivial solution to A*x = 0 is returned.

Arguments:
==========

 [in] UPLO

         UPLO is CHARACTER*1
         Specifies whether the matrix A is upper or lower triangular.
         = 'U':  Upper triangular
         = 'L':  Lower triangular

 [in] TRANS

         TRANS is CHARACTER*1
         Specifies the operation applied to A.
         = 'N':  Solve A * x = s*b     (No transpose)
         = 'T':  Solve A**T * x = s*b  (Transpose)
         = 'C':  Solve A**H * x = s*b  (Conjugate transpose)

 [in] DIAG

         DIAG is CHARACTER*1
         Specifies whether or not the matrix A is unit triangular.
         = 'N':  Non-unit triangular
         = 'U':  Unit triangular

 [in] NORMIN

         NORMIN is CHARACTER*1
         Specifies whether CNORM has been set or not.
         = 'Y':  CNORM contains the column norms on entry
```

```
        = 'N':  CNORM is not set on entry.  On exit, the norms will
                be computed and stored in CNORM.

[in] N

        N is INTEGER
        The order of the matrix A.  N >= 0.

[in] A

        A is COMPLEX*16 array, dimension (LDA,N)
        The triangular matrix A.  If UPLO = 'U', the leading n by n
        upper triangular part of the array A contains the upper
        triangular matrix, and the strictly lower triangular part of
        A is not referenced.  If UPLO = 'L', the leading n by n lower
        triangular part of the array A contains the lower triangular
        matrix, and the strictly upper triangular part of A is not
        referenced.  If DIAG = 'U', the diagonal elements of A are
        also not referenced and are assumed to be 1.

[in] LDA

        LDA is INTEGER
        The leading dimension of the array A.  LDA >= max (1,N).

[in,out] X

        X is COMPLEX*16 array, dimension (N)
        On entry, the right hand side b of the triangular system.
        On exit, X is overwritten by the solution vector x.

[out] SCALE

        SCALE is DOUBLE PRECISION
        The scaling factor s for the triangular system
           A * x = s*b,  A**T * x = s*b,  or  A**H * x = s*b.
        If SCALE = 0, the matrix A is singular or badly scaled, and
        the vector x is an exact or approximate solution to A*x = 0.

[in,out] CNORM

        CNORM is or output) DOUBLE PRECISION array, dimension (N)

        If NORMIN = 'Y', CNORM is an input argument and CNORM(j)
        contains the norm of the off-diagonal part of the j-th column
        of A.  If TRANS = 'N', CNORM(j) must be greater than or equal
        to the infinity-norm, and if TRANS = 'T' or 'C', CNORM(j)
        must be greater than or equal to the 1-norm.

        If NORMIN = 'N', CNORM is an output argument and CNORM(j)
```

```
         returns the 1-norm of the offdiagonal part of the j-th column
         of A.

 [out] INFO

         INFO is INTEGER
         = 0:  successful exit
         < 0:  if INFO = -k, the k-th argument had an illegal value

Further Details:
=====================

 A rough bound on x is computed; if that is less than overflow, ZTRSV
 is called, otherwise, specific code is used which checks for possible
 overflow or divide-by-zero at every operation.

 A columnwise scheme is used for solving A*x = b.  The basic algorithm
 if A is lower triangular is

      x[1:n] := b[1:n]
      for j = 1, ..., n
           x(j) := x(j) / A(j,j)
           x[j+1:n] := x[j+1:n] - x(j) * A[j+1:n,j]
      end

 Define bounds on the components of x after j iterations of the loop:
    M(j) = bound on x[1:j]
    G(j) = bound on x[j+1:n]
 Initially, let M(0) = 0 and G(0) = max{x(i), i=1,...,n}.

 Then for iteration j+1 we have
    M(j+1) <= G(j) / | A(j+1,j+1) |
    G(j+1) <= G(j) + M(j+1) * | A[j+2:n,j+1] |
           <= G(j) ( 1 + CNORM(j+1) / | A(j+1,j+1) | )

 where CNORM(j+1) is greater than or equal to the infinity-norm of
 column j+1 of A, not counting the diagonal.  Hence

    G(j) <= G(0) product ( 1 + CNORM(i) / | A(i,i) | )
                 1<=i<=j
 and

    |x(j)| <= ( G(0) / |A(j,j)| ) product ( 1 + CNORM(i) / |A(i,i)| )
                                   1<=i< j

 Since |x(j)| <= M(j), we use the Level 2 BLAS routine ZTRSV if the
 reciprocal of the largest M(j), j=1,..,n, is larger than
 max(underflow, 1/overflow).

 The bound on x(j) is also used to determine when a step in the
```

```
columnwise method can be performed without fear of overflow.  If
the computed bound is greater than a large constant, x is scaled to
prevent overflow, but if the bound overflows, x is set to 0, x(j) to
1, and scale to 0, and a non-trivial solution to A*x = 0 is found.

Similarly, a row-wise scheme is used to solve A**T *x = b  or
A**H *x = b.  The basic algorithm for A upper triangular is

     for j = 1, ..., n
          x(j) := ( b(j) - A[1:j-1,j]' * x[1:j-1] ) / A(j,j)
     end

We simultaneously compute two bounds
     G(j) = bound on ( b(i) - A[1:i-1,i]' * x[1:i-1] ), 1<=i<=j
     M(j) = bound on x(i), 1<=i<=j

The initial values are G(0) = 0, M(0) = max{b(i), i=1,..,n}, and we
add the constraint G(j) >= G(j-1) and M(j) >= M(j-1) for j >= 1.
Then the bound on x(j) is

     M(j) <= M(j-1) * ( 1 + CNORM(j) ) / | A(j,j) |

          <= M(0) * product ( ( 1 + CNORM(i) ) / |A(i,i)| )
                    1<=i<=j

and we can safely call ZTRSV if 1/M(n) and 1/G(n) are both greater
than max(underflow, 1/overflow).
```

———-

90.237.3 fortran code

— **zlatrs.f** —

```
*  =====================================================================
      SUBROUTINE ZLATRS( UPLO, TRANS, DIAG, NORMIN, N, A, LDA, X, SCALE,
     $                   CNORM, INFO )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          DIAG, NORMIN, TRANS, UPLO
      INTEGER            INFO, LDA, N
      DOUBLE PRECISION   SCALE
```

```
*     ..
*     .. Array Arguments ..
      DOUBLE PRECISION   CNORM( * )
      COMPLEX*16         A( LDA, * ), X( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, HALF, ONE, TWO
      PARAMETER          ( ZERO = 0.0D+0, HALF = 0.5D+0, ONE = 1.0D+0,
     $                   TWO = 2.0D+0 )
*     ..
*     .. Local Scalars ..
      LOGICAL            NOTRAN, NOUNIT, UPPER
      INTEGER            I, IMAX, J, JFIRST, JINC, JLAST
      DOUBLE PRECISION   BIGNUM, GROW, REC, SMLNUM, TJJ, TMAX, TSCAL,
     $                   XBND, XJ, XMAX
      COMPLEX*16         CSUMJ, TJJS, USCAL, ZDUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IDAMAX, IZAMAX
      DOUBLE PRECISION   DLAMCH, DZASUM
      COMPLEX*16         ZDOTC, ZDOTU, ZLADIV
      EXTERNAL           LSAME, IDAMAX, IZAMAX, DLAMCH, DZASUM, ZDOTC,
     $                   ZDOTU, ZLADIV
*     ..
*     .. External Subroutines ..
      EXTERNAL           DSCAL, XERBLA, ZAXPY, ZDSCAL, ZTRSV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DCMPLX, DCONJG, DIMAG, MAX, MIN
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1, CABS2
*     ..
*     .. Statement Function definitions ..
      CABS1( ZDUM ) = ABS( DBLE( ZDUM ) ) + ABS( DIMAG( ZDUM ) )
      CABS2( ZDUM ) = ABS( DBLE( ZDUM ) / 2.D0 ) +
     $                ABS( DIMAG( ZDUM ) / 2.D0 )
*     ..
*     .. Executable Statements ..
*
      INFO = 0
      UPPER = LSAME( UPLO, 'U' )
      NOTRAN = LSAME( TRANS, 'N' )
      NOUNIT = LSAME( DIAG, 'N' )
*
*     Test the input parameters.
```

```
*
      IF( .NOT.UPPER .AND. .NOT.LSAME( UPLO, 'L' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'T' ) .AND. .NOT.
     $         LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( .NOT.NOUNIT .AND. .NOT.LSAME( DIAG, 'U' ) ) THEN
         INFO = -3
      ELSE IF( .NOT.LSAME( NORMIN, 'Y' ) .AND. .NOT.
     $         LSAME( NORMIN, 'N' ) ) THEN
         INFO = -4
      ELSE IF( N.LT.0 ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -7
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZLATRS', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Determine machine dependent parameters to control overflow.
*
      SMLNUM = DLAMCH( 'Safe minimum' )
      BIGNUM = ONE / SMLNUM
      CALL DLABAD( SMLNUM, BIGNUM )
      SMLNUM = SMLNUM / DLAMCH( 'Precision' )
      BIGNUM = ONE / SMLNUM
      SCALE = ONE
*
      IF( LSAME( NORMIN, 'N' ) ) THEN
*
*        Compute the 1-norm of each column, not including the diagonal.
*
         IF( UPPER ) THEN
*
*           A is upper triangular.
*
            DO 10 J = 1, N
               CNORM( J ) = DZASUM( J-1, A( 1, J ), 1 )
   10       CONTINUE
         ELSE
*
*           A is lower triangular.
*
```

```
            DO 20 J = 1, N - 1
               CNORM( J ) = DZASUM( N-J, A( J+1, J ), 1 )
   20       CONTINUE
            CNORM( N ) = ZERO
         END IF
      END IF
*
*     Scale the column norms by TSCAL if the maximum element in CNORM is
*     greater than BIGNUM/2.
*
      IMAX = IDAMAX( N, CNORM, 1 )
      TMAX = CNORM( IMAX )
      IF( TMAX.LE.BIGNUM*HALF ) THEN
         TSCAL = ONE
      ELSE
         TSCAL = HALF / ( SMLNUM*TMAX )
         CALL DSCAL( N, TSCAL, CNORM, 1 )
      END IF
*
*     Compute a bound on the computed solution vector to see if the
*     Level 2 BLAS routine ZTRSV can be used.
*
      XMAX = ZERO
      DO 30 J = 1, N
         XMAX = MAX( XMAX, CABS2( X( J ) ) )
   30 CONTINUE
      XBND = XMAX
*
      IF( NOTRAN ) THEN
*
*        Compute the growth in A * x = b.
*
         IF( UPPER ) THEN
            JFIRST = N
            JLAST = 1
            JINC = -1
         ELSE
            JFIRST = 1
            JLAST = N
            JINC = 1
         END IF
*
         IF( TSCAL.NE.ONE ) THEN
            GROW = ZERO
            GO TO 60
         END IF
*
         IF( NOUNIT ) THEN
*
*           A is non-unit triangular.
```

```
*
*          Compute GROW = 1/G(j) and XBND = 1/M(j).
*          Initially, G(0) = max{x(i), i=1,...,n}.
*
           GROW = HALF / MAX( XBND, SMLNUM )
           XBND = GROW
           DO 40 J = JFIRST, JLAST, JINC
*
*             Exit the loop if the growth factor is too small.
*
              IF( GROW.LE.SMLNUM )
     $           GO TO 60
*
              TJJS = A( J, J )
              TJJ = CABS1( TJJS )
*
              IF( TJJ.GE.SMLNUM ) THEN
*
*                M(j) = G(j-1) / abs(A(j,j))
*
                 XBND = MIN( XBND, MIN( ONE, TJJ )*GROW )
              ELSE
*
*                M(j) could overflow, set XBND to 0.
*
                 XBND = ZERO
              END IF
*
              IF( TJJ+CNORM( J ).GE.SMLNUM ) THEN
*
*                G(j) = G(j-1)*( 1 + CNORM(j) / abs(A(j,j)) )
*
                 GROW = GROW*( TJJ / ( TJJ+CNORM( J ) ) )
              ELSE
*
*                G(j) could overflow, set GROW to 0.
*
                 GROW = ZERO
              END IF
   40      CONTINUE
           GROW = XBND
        ELSE
*
*          A is unit triangular.
*
*          Compute GROW = 1/G(j), where G(0) = max{x(i), i=1,...,n}.
*
           GROW = MIN( ONE, HALF / MAX( XBND, SMLNUM ) )
           DO 50 J = JFIRST, JLAST, JINC
*
```

```
*             Exit the loop if the growth factor is too small.
*
              IF( GROW.LE.SMLNUM )
     $           GO TO 60
*
*             G(j) = G(j-1)*( 1 + CNORM(j) )
*
              GROW = GROW*( ONE / ( ONE+CNORM( J ) ) )
   50       CONTINUE
         END IF
   60    CONTINUE
*
      ELSE
*
*        Compute the growth in A**T * x = b  or  A**H * x = b.
*
         IF( UPPER ) THEN
            JFIRST = 1
            JLAST = N
            JINC = 1
         ELSE
            JFIRST = N
            JLAST = 1
            JINC = -1
         END IF
*
         IF( TSCAL.NE.ONE ) THEN
            GROW = ZERO
            GO TO 90
         END IF
*
         IF( NOUNIT ) THEN
*
*           A is non-unit triangular.
*
*           Compute GROW = 1/G(j) and XBND = 1/M(j).
*           Initially, M(0) = max{x(i), i=1,...,n}.
*
            GROW = HALF / MAX( XBND, SMLNUM )
            XBND = GROW
            DO 70 J = JFIRST, JLAST, JINC
*
*             Exit the loop if the growth factor is too small.
*
              IF( GROW.LE.SMLNUM )
     $           GO TO 90
*
*             G(j) = max( G(j-1), M(j-1)*( 1 + CNORM(j) ) )
*
              XJ = ONE + CNORM( J )
```

```
               GROW = MIN( GROW, XBND / XJ )
*
               TJJS = A( J, J )
               TJJ = CABS1( TJJS )
*
               IF( TJJ.GE.SMLNUM ) THEN
*
*                 M(j) = M(j-1)*( 1 + CNORM(j) ) / abs(A(j,j))
*
                  IF( XJ.GT.TJJ )
     $               XBND = XBND*( TJJ / XJ )
               ELSE
*
*                 M(j) could overflow, set XBND to 0.
*
                  XBND = ZERO
               END IF
   70       CONTINUE
            GROW = MIN( GROW, XBND )
         ELSE
*
*          A is unit triangular.
*
*          Compute GROW = 1/G(j), where G(0) = max{x(i), i=1,...,n}.
*
            GROW = MIN( ONE, HALF / MAX( XBND, SMLNUM ) )
            DO 80 J = JFIRST, JLAST, JINC
*
*             Exit the loop if the growth factor is too small.
*
               IF( GROW.LE.SMLNUM )
     $            GO TO 90
*
               G(j) = ( 1 + CNORM(j) )*G(j-1)
*
               XJ = ONE + CNORM( J )
               GROW = GROW / XJ
   80       CONTINUE
         END IF
   90    CONTINUE
      END IF
*
      IF( ( GROW*TSCAL ).GT.SMLNUM ) THEN
*
*        Use the Level 2 BLAS solve if the reciprocal of the bound on
*        elements of X is not too small.
*
         CALL ZTRSV( UPLO, TRANS, DIAG, N, A, LDA, X, 1 )
      ELSE
*
```

```
*        Use a Level 1 BLAS solve, scaling intermediate results.
*
         IF( XMAX.GT.BIGNUM*HALF ) THEN
*
*           Scale X so that its components are less than or equal to
*           BIGNUM in absolute value.
*
            SCALE = ( BIGNUM*HALF ) / XMAX
            CALL ZDSCAL( N, SCALE, X, 1 )
            XMAX = BIGNUM
         ELSE
            XMAX = XMAX*TWO
         END IF
*
         IF( NOTRAN ) THEN
*
*           Solve A * x = b
*
            DO 120 J = JFIRST, JLAST, JINC
*
*              Compute x(j) = b(j) / A(j,j), scaling x if necessary.
*
               XJ = CABS1( X( J ) )
               IF( NOUNIT ) THEN
                  TJJS = A( J, J )*TSCAL
               ELSE
                  TJJS = TSCAL
                  IF( TSCAL.EQ.ONE )
     $               GO TO 110
               END IF
               TJJ = CABS1( TJJS )
               IF( TJJ.GT.SMLNUM ) THEN
*
*                    abs(A(j,j)) > SMLNUM:
*
                  IF( TJJ.LT.ONE ) THEN
                     IF( XJ.GT.TJJ*BIGNUM ) THEN
*
*                          Scale x by 1/b(j).
*
                        REC = ONE / XJ
                        CALL ZDSCAL( N, REC, X, 1 )
                        SCALE = SCALE*REC
                        XMAX = XMAX*REC
                     END IF
                  END IF
                  X( J ) = ZLADIV( X( J ), TJJS )
                  XJ = CABS1( X( J ) )
               ELSE IF( TJJ.GT.ZERO ) THEN
*
```

```
*                    0 < abs(A(j,j)) <= SMLNUM:
*
                  IF( XJ.GT.TJJ*BIGNUM ) THEN
*
*                      Scale x by (1/abs(x(j)))*abs(A(j,j))*BIGNUM
*                      to avoid overflow when dividing by A(j,j).
*
                     REC = ( TJJ*BIGNUM ) / XJ
                     IF( CNORM( J ).GT.ONE ) THEN
*
*                        Scale by 1/CNORM(j) to avoid overflow when
*                        multiplying x(j) times column j.
*
                      REC = REC / CNORM( J )
                     END IF
                     CALL ZDSCAL( N, REC, X, 1 )
                     SCALE = SCALE*REC
                     XMAX = XMAX*REC
                  END IF
                  X( J ) = ZLADIV( X( J ), TJJS )
                  XJ = CABS1( X( J ) )
               ELSE
*
*                    A(j,j) = 0:  Set x(1:n) = 0, x(j) = 1, and
*                    scale = 0, and compute a solution to A*x = 0.
*
                  DO 100 I = 1, N
                     X( I ) = ZERO
  100             CONTINUE
                  X( J ) = ONE
                  XJ = ONE
                  SCALE = ZERO
                  XMAX = ZERO
               END IF
  110          CONTINUE
*
*              Scale x if necessary to avoid overflow when adding a
*              multiple of column j of A.
*
               IF( XJ.GT.ONE ) THEN
                  REC = ONE / XJ
                  IF( CNORM( J ).GT.( BIGNUM-XMAX )*REC ) THEN
*
*                    Scale x by 1/(2*abs(x(j))).
*
                     REC = REC*HALF
                     CALL ZDSCAL( N, REC, X, 1 )
                     SCALE = SCALE*REC
                  END IF
               ELSE IF( XJ*CNORM( J ).GT.( BIGNUM-XMAX ) ) THEN
```

```
*
*                Scale x by 1/2.
*
                 CALL ZDSCAL( N, HALF, X, 1 )
                 SCALE = SCALE*HALF
              END IF
*
              IF( UPPER ) THEN
                 IF( J.GT.1 ) THEN
*
*                   Compute the update
*                      x(1:j-1) := x(1:j-1) - x(j) * A(1:j-1,j)
*
                    CALL ZAXPY( J-1, -X( J )*TSCAL, A( 1, J ), 1, X,
     $                           1 )
                    I = IZAMAX( J-1, X, 1 )
                    XMAX = CABS1( X( I ) )
                 END IF
              ELSE
                 IF( J.LT.N ) THEN
*
*                   Compute the update
*                      x(j+1:n) := x(j+1:n) - x(j) * A(j+1:n,j)
*
                    CALL ZAXPY( N-J, -X( J )*TSCAL, A( J+1, J ), 1,
     $                           X( J+1 ), 1 )
                    I = J + IZAMAX( N-J, X( J+1 ), 1 )
                    XMAX = CABS1( X( I ) )
                 END IF
              END IF
  120       CONTINUE
*
         ELSE IF( LSAME( TRANS, 'T' ) ) THEN
*
*          Solve A**T * x = b
*
           DO 170 J = JFIRST, JLAST, JINC
*
*             Compute x(j) = b(j) - sum A(k,j)*x(k).
*                                   k<>j
*
              XJ = CABS1( X( J ) )
              USCAL = TSCAL
              REC = ONE / MAX( XMAX, ONE )
              IF( CNORM( J ).GT.( BIGNUM-XJ )*REC ) THEN
*
*                If x(j) could overflow, scale x by 1/(2*XMAX).
*
                 REC = REC*HALF
                 IF( NOUNIT ) THEN
```

```
                  TJJS = A( J, J )*TSCAL
               ELSE
                  TJJS = TSCAL
               END IF
               TJJ = CABS1( TJJS )
               IF( TJJ.GT.ONE ) THEN
*
*                    Divide by A(j,j) when scaling x if A(j,j) > 1.
*
                  REC = MIN( ONE, REC*TJJ )
                  USCAL = ZLADIV( USCAL, TJJS )
               END IF
               IF( REC.LT.ONE ) THEN
                  CALL ZDSCAL( N, REC, X, 1 )
                  SCALE = SCALE*REC
                  XMAX = XMAX*REC
               END IF
            END IF
*
            CSUMJ = ZERO
            IF( USCAL.EQ.DCMPLX( ONE ) ) THEN
*
*              If the scaling needed for A in the dot product is 1,
*              call ZDOTU to perform the dot product.
*
               IF( UPPER ) THEN
                  CSUMJ = ZDOTU( J-1, A( 1, J ), 1, X, 1 )
               ELSE IF( J.LT.N ) THEN
                  CSUMJ = ZDOTU( N-J, A( J+1, J ), 1, X( J+1 ), 1 )
               END IF
            ELSE
*
*              Otherwise, use in-line code for the dot product.
*
               IF( UPPER ) THEN
                  DO 130 I = 1, J - 1
                     CSUMJ = CSUMJ + ( A( I, J )*USCAL )*X( I )
  130             CONTINUE
               ELSE IF( J.LT.N ) THEN
                  DO 140 I = J + 1, N
                     CSUMJ = CSUMJ + ( A( I, J )*USCAL )*X( I )
  140             CONTINUE
               END IF
            END IF
*
            IF( USCAL.EQ.DCMPLX( TSCAL ) ) THEN
*
*              Compute x(j) := ( x(j) - CSUMJ ) / A(j,j) if 1/A(j,j)
*              was not used to scale the dotproduct.
*
```

```
                 X( J ) = X( J ) - CSUMJ
                 XJ = CABS1( X( J ) )
                 IF( NOUNIT ) THEN
                    TJJS = A( J, J )*TSCAL
                 ELSE
                    TJJS = TSCAL
                    IF( TSCAL.EQ.ONE )
     $                 GO TO 160
                 END IF
*
*                   Compute x(j) = x(j) / A(j,j), scaling if necessary.
*
                 TJJ = CABS1( TJJS )
                 IF( TJJ.GT.SMLNUM ) THEN
*
*                      abs(A(j,j)) > SMLNUM:
*
                    IF( TJJ.LT.ONE ) THEN
                       IF( XJ.GT.TJJ*BIGNUM ) THEN
*
*                            Scale X by 1/abs(x(j)).
*
                          REC = ONE / XJ
                          CALL ZDSCAL( N, REC, X, 1 )
                          SCALE = SCALE*REC
                          XMAX = XMAX*REC
                       END IF
                    END IF
                    X( J ) = ZLADIV( X( J ), TJJS )
                 ELSE IF( TJJ.GT.ZERO ) THEN
*
*                      0 < abs(A(j,j)) <= SMLNUM:
*
                    IF( XJ.GT.TJJ*BIGNUM ) THEN
*
*                          Scale x by (1/abs(x(j)))*abs(A(j,j))*BIGNUM.
*
                       REC = ( TJJ*BIGNUM ) / XJ
                       CALL ZDSCAL( N, REC, X, 1 )
                       SCALE = SCALE*REC
                       XMAX = XMAX*REC
                    END IF
                    X( J ) = ZLADIV( X( J ), TJJS )
                 ELSE
*
*                      A(j,j) = 0:  Set x(1:n) = 0, x(j) = 1, and
*                      scale = 0 and compute a solution to A**T *x = 0.
*
                    DO 150 I = 1, N
                       X( I ) = ZERO
```

```
  150               CONTINUE
                    X( J ) = ONE
                    SCALE = ZERO
                    XMAX = ZERO
                 END IF
  160            CONTINUE
              ELSE
*
*                Compute x(j) := x(j) / A(j,j) - CSUMJ if the dot
*                product has already been divided by 1/A(j,j).
*
                 X( J ) = ZLADIV( X( J ), TJJS ) - CSUMJ
              END IF
              XMAX = MAX( XMAX, CABS1( X( J ) ) )
  170       CONTINUE
*
         ELSE
*
*          Solve A**H * x = b
*
           DO 220 J = JFIRST, JLAST, JINC
*
*             Compute x(j) = b(j) - sum A(k,j)*x(k).
*                                   k<>j
*
              XJ = CABS1( X( J ) )
              USCAL = TSCAL
              REC = ONE / MAX( XMAX, ONE )
              IF( CNORM( J ).GT.( BIGNUM-XJ )*REC ) THEN
*
*                If x(j) could overflow, scale x by 1/(2*XMAX).
*
                 REC = REC*HALF
                 IF( NOUNIT ) THEN
                    TJJS = DCONJG( A( J, J ) )*TSCAL
                 ELSE
                    TJJS = TSCAL
                 END IF
                 TJJ = CABS1( TJJS )
                 IF( TJJ.GT.ONE ) THEN
*
*                      Divide by A(j,j) when scaling x if A(j,j) > 1.
*
                    REC = MIN( ONE, REC*TJJ )
                    USCAL = ZLADIV( USCAL, TJJS )
                 END IF
                 IF( REC.LT.ONE ) THEN
                    CALL ZDSCAL( N, REC, X, 1 )
                    SCALE = SCALE*REC
                    XMAX = XMAX*REC
```

```
               END IF
            END IF
*
            CSUMJ = ZERO
            IF( USCAL.EQ.DCMPLX( ONE ) ) THEN
*
*              If the scaling needed for A in the dot product is 1,
*              call ZDOTC to perform the dot product.
*
               IF( UPPER ) THEN
                  CSUMJ = ZDOTC( J-1, A( 1, J ), 1, X, 1 )
               ELSE IF( J.LT.N ) THEN
                  CSUMJ = ZDOTC( N-J, A( J+1, J ), 1, X( J+1 ), 1 )
               END IF
            ELSE
*
*              Otherwise, use in-line code for the dot product.
*
               IF( UPPER ) THEN
                  DO 180 I = 1, J - 1
                     CSUMJ = CSUMJ + ( DCONJG( A( I, J ) )*USCAL )*
     $                       X( I )
  180             CONTINUE
               ELSE IF( J.LT.N ) THEN
                  DO 190 I = J + 1, N
                     CSUMJ = CSUMJ + ( DCONJG( A( I, J ) )*USCAL )*
     $                       X( I )
  190             CONTINUE
               END IF
            END IF
*
            IF( USCAL.EQ.DCMPLX( TSCAL ) ) THEN
*
*              Compute x(j) := ( x(j) - CSUMJ ) / A(j,j) if 1/A(j,j)
*              was not used to scale the dotproduct.
*
               X( J ) = X( J ) - CSUMJ
               XJ = CABS1( X( J ) )
               IF( NOUNIT ) THEN
                  TJJS = DCONJG( A( J, J ) )*TSCAL
               ELSE
                  TJJS = TSCAL
                  IF( TSCAL.EQ.ONE )
     $               GO TO 210
               END IF
*
*                 Compute x(j) = x(j) / A(j,j), scaling if necessary.
*
               TJJ = CABS1( TJJS )
               IF( TJJ.GT.SMLNUM ) THEN
```

```
*
*                     abs(A(j,j)) > SMLNUM:
*
                   IF( TJJ.LT.ONE ) THEN
                      IF( XJ.GT.TJJ*BIGNUM ) THEN
*
*                           Scale X by 1/abs(x(j)).
*
                         REC = ONE / XJ
                         CALL ZDSCAL( N, REC, X, 1 )
                         SCALE = SCALE*REC
                         XMAX = XMAX*REC
                      END IF
                   END IF
                   X( J ) = ZLADIV( X( J ), TJJS )
                ELSE IF( TJJ.GT.ZERO ) THEN
*
*                     0 < abs(A(j,j)) <= SMLNUM:
*
                   IF( XJ.GT.TJJ*BIGNUM ) THEN
*
*                         Scale x by (1/abs(x(j)))*abs(A(j,j))*BIGNUM.
*
                      REC = ( TJJ*BIGNUM ) / XJ
                      CALL ZDSCAL( N, REC, X, 1 )
                      SCALE = SCALE*REC
                      XMAX = XMAX*REC
                   END IF
                   X( J ) = ZLADIV( X( J ), TJJS )
                ELSE
*
*                     A(j,j) = 0:  Set x(1:n) = 0, x(j) = 1, and
*                     scale = 0 and compute a solution to A**H *x = 0.
*
                   DO 200 I = 1, N
                      X( I ) = ZERO
  200              CONTINUE
                   X( J ) = ONE
                   SCALE = ZERO
                   XMAX = ZERO
                END IF
  210           CONTINUE
             ELSE
*
*                Compute x(j) := x(j) / A(j,j) - CSUMJ if the dot
*                product has already been divided by 1/A(j,j).
*
                X( J ) = ZLADIV( X( J ), TJJS ) - CSUMJ
             END IF
             XMAX = MAX( XMAX, CABS1( X( J ) ) )
```

```
  220       CONTINUE
         END IF
         SCALE = SCALE / TSCAL
      END IF
*
*     Scale the column norms by 1/TSCAL for return.
*
      IF( TSCAL.NE.ONE ) THEN
         CALL DSCAL( N, ONE / TSCAL, CNORM, 1 )
      END IF
*
      RETURN
*
*     End of ZLATRS
*
      END
```

———-

90.237.4 lisp code

— **zlatrs.lisp** —

```
(let* ((zero 0.0d0) (half 0.5d0) (one 1.0d0) (two 2.0d0))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 0.5d0 0.5d0) half)
           (type (double-float 1.0d0 1.0d0) one)
           (type (double-float 2.0d0 2.0d0) two)
           (ignorable zero half one two))
  (defun zlatrs (uplo trans diag normin n a lda x scale cnorm info)
    (declare (type (array double-float (*)) cnorm)
             (type (double-float) scale)
             (type (array f2cl-lib:complex16 (*)) x a)
             (type (f2cl-lib:integer4) info lda n)
             (type (string *) normin diag trans uplo))
    (f2cl-lib:with-multi-array-data
        ((uplo character uplo-%data% uplo-%offset%)
         (trans character trans-%data% trans-%offset%)
         (diag character diag-%data% diag-%offset%)
         (normin character normin-%data% normin-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (x f2cl-lib:complex16 x-%data% x-%offset%)
         (cnorm double-float cnorm-%data% cnorm-%offset%))
      (labels ((cabs1 (zdum)
                 (+ (abs (f2cl-lib:dble zdum)) (abs (f2cl-lib:dimag zdum))))
               (cabs2 (zdum)
                 (+ (abs (/ (f2cl-lib:dble zdum) 2.0d0))
```

```
            (abs (/ (f2cl-lib:dimag zdum) 2.0d0)))))
(declare
 (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
  cabs1))
(declare
 (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
  cabs2))
(prog ((csumj #C(0.0d0 0.0d0)) (tjjs #C(0.0d0 0.0d0))
       (uscal #C(0.0d0 0.0d0)) (zdum #C(0.0d0 0.0d0)) (bignum 0.0d0)
       (grow 0.0d0) (rec 0.0d0) (smlnum 0.0d0) (tjj 0.0d0) (tmax 0.0d0)
       (tscal 0.0d0) (xbnd 0.0d0) (xj 0.0d0) (xmax 0.0d0) (i 0)
       (imax 0) (j 0) (jfirst 0) (jinc 0) (jlast 0) (notran nil)
       (nounit nil) (upper nil))
  (declare (type (f2cl-lib:complex16) csumj tjjs uscal zdum)
           (type (double-float) bignum grow rec smlnum tjj tmax tscal
            xbnd xj xmax)
           (type (f2cl-lib:integer4) i imax j jfirst jinc jlast)
           (type f2cl-lib:logical notran nounit upper))
  (setf info 0)
  (setf upper
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "U")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val))
  (setf notran
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame trans "N")
            (declare (ignore var-1))
            (when var-0 (setf trans var-0))
            ret-val))
  (setf nounit
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame diag "N")
            (declare (ignore var-1))
            (when var-0 (setf diag var-0))
            ret-val))
  (cond
   ((and (not upper)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame uplo "L")
            (declare (ignore var-1))
            (when var-0 (setf uplo var-0))
            ret-val)))
    (setf info -1))
   ((and (not notran)
         (not
          (multiple-value-bind (ret-val var-0 var-1)
              (lsame trans "T")
```

```
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
        (not
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -2))
 ((and (not nounit)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame diag "U")
          (declare (ignore var-1))
          (when var-0 (setf diag var-0))
          ret-val)))
  (setf info -3))
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame normin "Y")
      (declare (ignore var-1))
      (when var-0 (setf normin var-0))
      ret-val))
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame normin "N")
      (declare (ignore var-1))
      (when var-0 (setf normin var-0))
      ret-val)))
  (setf info -4))
 ((< n 0) (setf info -5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -7)))
(cond
 ((/= info 0) (xerbla "ZLATRS" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(setf smlnum (dlamch "Safe minimum"))
(setf bignum (/ one smlnum))
(multiple-value-bind (var-0 var-1)
    (dlabad smlnum bignum)
  (declare (ignore))
  (when var-0 (setf smlnum var-0))
  (when var-1 (setf bignum var-1)))
(setf smlnum (/ smlnum (dlamch "Precision")))
(setf bignum (/ one smlnum))
(setf scale one)
```

```
(cond
 ((multiple-value-bind (ret-val var-0 var-1)
      (lsame normin "N")
    (declare (ignore var-1))
    (when var-0 (setf normin var-0))
    ret-val)
  (cond
   (upper
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j n) nil)
      (tagbody
        (setf (f2cl-lib:fref cnorm-%data% (j) ((1 *)) cnorm-%offset%)
                (dzasum (f2cl-lib:int-sub j 1)
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                       (1 j) ((1 lda) (1 *))
                                       a-%offset%)
                 1))
       label10)))
   (t
    (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                  ((> j (f2cl-lib:int-add n (f2cl-lib:int-sub 1)))
                   nil)
      (tagbody
        (setf (f2cl-lib:fref cnorm-%data% (j) ((1 *)) cnorm-%offset%)
                (dzasum (f2cl-lib:int-sub n j)
                 (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                       ((+ j 1) j) ((1 lda) (1 *))
                                       a-%offset%)
                 1))
       label20))
    (setf (f2cl-lib:fref cnorm-%data% (n) ((1 *)) cnorm-%offset%)
            zero)))))
(setf imax
        (multiple-value-bind (ret-val var-0 var-1 var-2)
            (idamax n cnorm 1)
          (declare (ignore var-1 var-2))
          (when var-0 (setf n var-0))
          ret-val))
(setf tmax
        (f2cl-lib:fref cnorm-%data% (imax) ((1 *)) cnorm-%offset%))
(cond ((<= tmax (* bignum half)) (setf tscal one))
      (t (setf tscal (/ half (* smlnum tmax)))
       (multiple-value-bind (var-0 var-1 var-2 var-3)
           (dscal n tscal cnorm 1)
         (declare (ignore var-2 var-3))
         (when var-0 (setf n var-0))
         (when var-1 (setf tscal var-1)))))
(setf xmax zero)
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j n) nil)
```

```
  (tagbody
    (setf xmax
            (max xmax
                 (cabs2
                  (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))))
   label30))
(setf xbnd xmax)
(cond
 (notran
  (tagbody
    (cond (upper (setf jfirst n) (setf jlast 1) (setf jinc -1))
          (t (setf jfirst 1) (setf jlast n) (setf jinc 1)))
    (cond ((/= tscal one) (setf grow zero) (go label60)))
    (cond
     (nounit (setf grow (/ half (max xbnd smlnum))) (setf xbnd grow)
      (f2cl-lib:fdo (j jfirst (f2cl-lib:int-add j jinc))
                    ((> j jlast) nil)
        (tagbody
          (if (<= grow smlnum)
              (go label60))
          (setf tjjs
                  (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                 a-%offset%))
          (setf tjj (cabs1 tjjs))
          (cond
           ((>= tjj smlnum)
            (setf xbnd (min xbnd (* (min one tjj) grow))))
           (t (setf xbnd zero)))
          (cond
           ((>= (+ tjj (f2cl-lib:fref cnorm (j) ((1 *)))) smlnum)
            (setf grow
                    (* grow
                       (/ tjj
                          (+ tjj
                             (f2cl-lib:fref cnorm-%data% (j) ((1 *))
                                            cnorm-%offset%))))))
           (t (setf grow zero)))
         label40))
      (setf grow xbnd))
     (t (setf grow (min one (/ half (max xbnd smlnum))))
      (f2cl-lib:fdo (j jfirst (f2cl-lib:int-add j jinc))
                    ((> j jlast) nil)
        (tagbody
          (if (<= grow smlnum)
              (go label60))
          (setf grow
                  (* grow
                     (/ one
                        (+ one
                           (f2cl-lib:fref cnorm-%data% (j) ((1 *))
```

```
                                                cnorm-%offset%)))))
            label50))))
      label60))
    (t
     (tagbody
       (cond (upper (setf jfirst 1) (setf jlast n) (setf jinc 1))
             (t (setf jfirst n) (setf jlast 1) (setf jinc -1)))
       (cond ((/= tscal one) (setf grow zero) (go label90)))
       (cond
        (nounit (setf grow (/ half (max xbnd smlnum))) (setf xbnd grow)
         (f2cl-lib:fdo (j jfirst (f2cl-lib:int-add j jinc))
                       ((> j jlast) nil)
           (tagbody
             (if (<= grow smlnum)
                 (go label90))
             (setf xj
                     (+ one
                        (f2cl-lib:fref cnorm-%data% (j) ((1 *))
                                       cnorm-%offset%)))
             (setf grow (min grow (/ xbnd xj)))
             (setf tjjs
                     (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                    a-%offset%))
             (setf tjj (cabs1 tjjs))
             (cond
              ((>= tjj smlnum)
               (if (> xj tjj)
                   (setf xbnd (* xbnd (/ tjj xj)))))
              (t (setf xbnd zero)))
            label70))
         (setf grow (min grow xbnd)))
        (t (setf grow (min one (/ half (max xbnd smlnum))))
         (f2cl-lib:fdo (j jfirst (f2cl-lib:int-add j jinc))
                       ((> j jlast) nil)
           (tagbody
             (if (<= grow smlnum)
                 (go label90))
             (setf xj
                     (+ one
                        (f2cl-lib:fref cnorm-%data% (j) ((1 *))
                                       cnorm-%offset%)))
             (setf grow (/ grow xj))
            label80))))
      label90)))
   (cond
    ((> (* grow tscal) smlnum)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
         (ztrsv uplo trans diag n a lda x 1)
       (declare (ignore var-4 var-6 var-7))
```

```
    (when var-0 (setf uplo var-0))
    (when var-1 (setf trans var-1))
    (when var-2 (setf diag var-2))
    (when var-3 (setf n var-3))
    (when var-5 (setf lda var-5))))
  (t
   (cond
    ((> xmax (* bignum half)) (setf scale (/ (* bignum half) xmax))
     (multiple-value-bind (var-0 var-1 var-2 var-3)
         (zdscal n scale x 1)
       (declare (ignore var-2 var-3))
       (when var-0 (setf n var-0))
       (when var-1 (setf scale var-1)))
     (setf xmax bignum))
    (t (setf xmax (* xmax two))))
   (cond
    (notran
     (f2cl-lib:fdo (j jfirst (f2cl-lib:int-add j jinc))
                   ((> j jlast) nil)
       (tagbody
         (setf xj
                 (cabs1
                  (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
         (cond
          (nounit
           (setf tjjs
                   (*
                    (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                   a-%offset%)
                    tscal)))
          (t (setf tjjs (coerce tscal 'f2cl-lib:complex16))
           (if (= tscal one)
               (go label110))))
         (setf tjj (cabs1 tjjs))
         (cond
          ((> tjj smlnum)
           (cond
            ((< tjj one)
             (cond
              ((> xj (* tjj bignum)) (setf rec (/ one xj))
               (multiple-value-bind (var-0 var-1 var-2 var-3)
                   (zdscal n rec x 1)
                 (declare (ignore var-2 var-3))
                 (when var-0 (setf n var-0))
                 (when var-1 (setf rec var-1)))
               (setf scale (* scale rec)) (setf xmax (* xmax rec))))))
           (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                   (zladiv
                    (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                    tjjs))
```

```
   (setf xj
           (cabs1
            (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))))
  ((> tjj zero)
   (cond
    ((> xj (* tjj bignum)) (setf rec (/ (* tjj bignum) xj))
     (cond
      ((> (f2cl-lib:fref cnorm (j) ((1 *))) one)
       (setf rec
               (/ rec
                  (f2cl-lib:fref cnorm-%data% (j) ((1 *))
                                 cnorm-%offset%)))))
     (multiple-value-bind (var-0 var-1 var-2 var-3)
         (zdscal n rec x 1)
       (declare (ignore var-2 var-3))
       (when var-0 (setf n var-0))
       (when var-1 (setf rec var-1)))
     (setf scale (* scale rec)) (setf xmax (* xmax rec))))
   (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
           (zladiv
            (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
            tjjs))
   (setf xj
           (cabs1
            (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))))
  (t
   (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                 ((> i n) nil)
     (tagbody
       (setf (f2cl-lib:fref x-%data% (i) ((1 *)) x-%offset%)
               (coerce zero 'f2cl-lib:complex16))
      label100))
   (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
           (coerce one 'f2cl-lib:complex16))
   (setf xj one) (setf scale zero) (setf xmax zero)))
label110
 (cond
  ((> xj one) (setf rec (/ one xj))
   (cond
    ((> (f2cl-lib:fref cnorm (j) ((1 *)))
        (* (+ bignum (- xmax)) rec))
     (setf rec (* rec half))
     (multiple-value-bind (var-0 var-1 var-2 var-3)
         (zdscal n rec x 1)
       (declare (ignore var-2 var-3))
       (when var-0 (setf n var-0))
       (when var-1 (setf rec var-1)))
     (setf scale (* scale rec)))))
  ((> (* xj (f2cl-lib:fref cnorm (j) ((1 *))))
      (+ bignum (- xmax)))
```

```
                (multiple-value-bind (var-0 var-1 var-2 var-3)
                    (zdscal n half x 1)
                  (declare (ignore var-2 var-3))
                  (when var-0 (setf n var-0))
                  (when var-1 (setf half var-1)))
                (setf scale (* scale half))))
              (cond
               (upper
                (cond
                 ((> j 1)
                  (zaxpy (f2cl-lib:int-sub j 1)
                   (* (- (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                      tscal)
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (1 j)
                                         ((1 lda) (1 *)) a-%offset%)
                   1 x 1)
                  (setf i (izamax (f2cl-lib:int-sub j 1) x 1))
                  (setf xmax
                          (cabs1
                           (f2cl-lib:fref x-%data% (i) ((1 *))
                                          x-%offset%))))))
               (t
                (cond
                 ((< j n)
                  (zaxpy (f2cl-lib:int-sub n j)
                   (* (- (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%))
                      tscal)
                   (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                         ((+ j 1) j) ((1 lda) (1 *))
                                         a-%offset%)
                   1
                   (f2cl-lib:array-slice x-%data% f2cl-lib:complex16
                                         ((+ j 1)) ((1 *)) x-%offset%)
                   1)
                  (setf i
                          (f2cl-lib:int-add j
                                            (izamax (f2cl-lib:int-sub n j)
                                             (f2cl-lib:array-slice x-%data%
                                                                   f2cl-lib:complex16
                                                                   ((+ j
                                                                       1))
                                                                   ((1 *))
                                                                   x-%offset%)
                                             1)))
                  (setf xmax
                          (cabs1
                           (f2cl-lib:fref x-%data% (i) ((1 *))
                                          x-%offset%)))))))
            label120)))
        ((multiple-value-bind (ret-val var-0 var-1)
```

```
      (lsame trans "T")
    (declare (ignore var-1))
    (when var-0 (setf trans var-0))
    ret-val)
  (f2cl-lib:fdo (j jfirst (f2cl-lib:int-add j jinc))
                ((> j jlast) nil)
    (tagbody
      (setf xj
              (cabs1
               (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
      (setf uscal (coerce tscal 'f2cl-lib:complex16))
      (setf rec (/ one (max xmax one)))
      (cond
       ((> (f2cl-lib:fref cnorm (j) ((1 *)))
           (* (+ bignum (- xj)) rec))
        (setf rec (* rec half))
        (cond
         (nounit
          (setf tjjs
                  (*
                   (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                  a-%offset%)
                   tscal)))
         (t (setf tjjs (coerce tscal 'f2cl-lib:complex16))))
        (setf tjj (cabs1 tjjs))
        (cond
         ((> tjj one) (setf rec (min one (* rec tjj)))
          (setf uscal (zladiv uscal tjjs))))
        (cond
         ((< rec one)
          (multiple-value-bind (var-0 var-1 var-2 var-3)
              (zdscal n rec x 1)
            (declare (ignore var-2 var-3))
            (when var-0 (setf n var-0))
            (when var-1 (setf rec var-1)))
          (setf scale (* scale rec)) (setf xmax (* xmax rec))))))
      (setf csumj (coerce zero 'f2cl-lib:complex16))
      (cond
       ((= uscal (f2cl-lib:dcmplx one))
        (cond
         (upper
          (setf csumj
                  (zdotu (f2cl-lib:int-sub j 1)
                   (f2cl-lib:array-slice a-%data%
                                         f2cl-lib:complex16 (1 j)
                                         ((1 lda) (1 *))
                                         a-%offset%)
                   1 x 1)))
         ((< j n)
          (setf csumj
```

```
            (zdotu (f2cl-lib:int-sub n j)
             (f2cl-lib:array-slice a-%data%
                                   f2cl-lib:complex16
                                   ((+ j 1) j)
                                   ((1 lda) (1 *))
                                   a-%offset%)
             1
             (f2cl-lib:array-slice x-%data%
                                   f2cl-lib:complex16
                                   ((+ j 1)) ((1 *))
                                   x-%offset%)
             1)))))
 (t
  (cond
   (upper
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i
                      (f2cl-lib:int-add j
                                        (f2cl-lib:int-sub
                                         1)))
                   nil)
      (tagbody
        (setf csumj
                (+ csumj
                   (*
                    (f2cl-lib:fref a-%data% (i j)
                                   ((1 lda) (1 *))
                                   a-%offset%)
                    uscal
                    (f2cl-lib:fref x-%data% (i) ((1 *))
                                   x-%offset%))))
       label130)))
   ((< j n)
    (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                   (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf csumj
                (+ csumj
                   (*
                    (f2cl-lib:fref a-%data% (i j)
                                   ((1 lda) (1 *))
                                   a-%offset%)
                    uscal
                    (f2cl-lib:fref x-%data% (i) ((1 *))
                                   x-%offset%))))
       label140))))))
(cond
 ((= uscal (f2cl-lib:dcmplx tscal))
  (tagbody
```

```
(setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
        (-
         (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
         csumj))
(setf xj
        (cabs1
         (f2cl-lib:fref x-%data% (j) ((1 *))
                        x-%offset%)))
(cond
 (nounit
  (setf tjjs
          (*
           (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                          a-%offset%)
           tscal)))
 (t (setf tjjs (coerce tscal 'f2cl-lib:complex16))
  (if (= tscal one)
      (go label160))))
(setf tjj (cabs1 tjjs))
(cond
 ((> tjj smlnum)
  (cond
   ((< tjj one)
    (cond
     ((> xj (* tjj bignum)) (setf rec (/ one xj))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (zdscal n rec x 1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf n var-0))
        (when var-1 (setf rec var-1)))
      (setf scale (* scale rec))
      (setf xmax (* xmax rec))))))
  (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
          (zladiv
           (f2cl-lib:fref x-%data% (j) ((1 *))
                          x-%offset%)
           tjjs)))
 ((> tjj zero)
  (cond
   ((> xj (* tjj bignum))
    (setf rec (/ (* tjj bignum) xj))
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zdscal n rec x 1)
      (declare (ignore var-2 var-3))
      (when var-0 (setf n var-0))
      (when var-1 (setf rec var-1)))
    (setf scale (* scale rec)) (setf xmax (* xmax rec))))
  (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
          (zladiv
           (f2cl-lib:fref x-%data% (j) ((1 *))
```

```
                                        x-%offset%)
                          tjjs)))
                     (t
                      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                                    ((> i n) nil)
                        (tagbody
                          (setf (f2cl-lib:fref x-%data% (i) ((1 *))
                                                x-%offset%)
                                  (coerce zero 'f2cl-lib:complex16))
                         label150))
                      (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                              (coerce one 'f2cl-lib:complex16))
                      (setf scale zero) (setf xmax zero)))
                  label160))
                (t
                 (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                         (-
                          (zladiv
                           (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                           tjjs)
                          csumj))))
             (setf xmax
                     (max xmax
                          (cabs1
                           (f2cl-lib:fref x-%data% (j) ((1 *))
                                          x-%offset%))))
            label170)))
        (t
         (f2cl-lib:fdo (j jfirst (f2cl-lib:int-add j jinc))
                       ((> j jlast) nil)
           (tagbody
             (setf xj
                     (cabs1
                      (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)))
             (setf uscal (coerce tscal 'f2cl-lib:complex16))
             (setf rec (/ one (max xmax one)))
             (cond
              ((> (f2cl-lib:fref cnorm (j) ((1 *)))
                  (* (+ bignum (- xj)) rec))
               (setf rec (* rec half))
               (cond
                (nounit
                 (setf tjjs
                         (coerce
                          (*
                           (f2cl-lib:dconjg
                            (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *))
                                           a-%offset%))
                           tscal)
                          'f2cl-lib:complex16)))
```

```
   (t (setf tjjs (coerce tscal 'f2cl-lib:complex16))))
  (setf tjj (cabs1 tjjs))
  (cond
   ((> tjj one) (setf rec (min one (* rec tjj)))
    (setf uscal (zladiv uscal tjjs))))
  (cond
   ((< rec one)
    (multiple-value-bind (var-0 var-1 var-2 var-3)
        (zdscal n rec x 1)
      (declare (ignore var-2 var-3))
      (when var-0 (setf n var-0))
      (when var-1 (setf rec var-1)))
    (setf scale (* scale rec)) (setf xmax (* xmax rec))))))
(setf csumj (coerce zero 'f2cl-lib:complex16))
(cond
 ((= uscal (f2cl-lib:dcmplx one))
  (cond
   (upper
    (setf csumj
            (zdotc (f2cl-lib:int-sub j 1)
             (f2cl-lib:array-slice a-%data%
                                   f2cl-lib:complex16 (1 j)
                                   ((1 lda) (1 *))
                                   a-%offset%)
             1 x 1)))
   ((< j n)
    (setf csumj
            (zdotc (f2cl-lib:int-sub n j)
             (f2cl-lib:array-slice a-%data%
                                   f2cl-lib:complex16
                                   ((+ j 1) j)
                                   ((1 lda) (1 *))
                                   a-%offset%)
             1
             (f2cl-lib:array-slice x-%data%
                                   f2cl-lib:complex16
                                   ((+ j 1)) ((1 *))
                                   x-%offset%)
             1)))))
 (t
  (cond
   (upper
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i
                      (f2cl-lib:int-add j
                                        (f2cl-lib:int-sub
                                         1)))
                   nil)
      (tagbody
        (setf csumj
```

```
                (+ csumj
                   (*
                    (f2cl-lib:dconjg
                     (f2cl-lib:fref a-%data% (i j)
                                    ((1 lda) (1 *))
                                    a-%offset%))
                    uscal
                    (f2cl-lib:fref x-%data% (i) ((1 *))
                                   x-%offset%))))
        label180)))
    ((< j n)
     (f2cl-lib:fdo (i (f2cl-lib:int-add j 1)
                    (f2cl-lib:int-add i 1))
                   ((> i n) nil)
       (tagbody
         (setf csumj
                (+ csumj
                   (*
                    (f2cl-lib:dconjg
                     (f2cl-lib:fref a-%data% (i j)
                                    ((1 lda) (1 *))
                                    a-%offset%))
                    uscal
                    (f2cl-lib:fref x-%data% (i) ((1 *))
                                   x-%offset%))))
        label190))))))
(cond
 ((= uscal (f2cl-lib:dcmplx tscal))
  (tagbody
    (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
            (-
             (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
             csumj))
    (setf xj
            (cabs1
             (f2cl-lib:fref x-%data% (j) ((1 *))
                            x-%offset%)))
    (cond
     (nounit
      (setf tjjs
              (coerce
               (*
                (f2cl-lib:dconjg
                 (f2cl-lib:fref a-%data% (j j)
                                ((1 lda) (1 *)) a-%offset%))
                tscal)
               'f2cl-lib:complex16)))
     (t (setf tjjs (coerce tscal 'f2cl-lib:complex16))
      (if (= tscal one)
          (go label210))))
```

```
  (setf tjj (cabs1 tjjs))
  (cond
   ((> tjj smlnum)
    (cond
     ((< tjj one)
      (cond
       ((> xj (* tjj bignum)) (setf rec (/ one xj))
        (multiple-value-bind (var-0 var-1 var-2 var-3)
            (zdscal n rec x 1)
          (declare (ignore var-2 var-3))
          (when var-0 (setf n var-0))
          (when var-1 (setf rec var-1)))
        (setf scale (* scale rec))
        (setf xmax (* xmax rec))))))
    (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
            (zladiv
             (f2cl-lib:fref x-%data% (j) ((1 *))
                            x-%offset%)
             tjjs)))
   ((> tjj zero)
    (cond
     ((> xj (* tjj bignum))
      (setf rec (/ (* tjj bignum) xj))
      (multiple-value-bind (var-0 var-1 var-2 var-3)
          (zdscal n rec x 1)
        (declare (ignore var-2 var-3))
        (when var-0 (setf n var-0))
        (when var-1 (setf rec var-1)))
      (setf scale (* scale rec)) (setf xmax (* xmax rec))))
    (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
            (zladiv
             (f2cl-lib:fref x-%data% (j) ((1 *))
                            x-%offset%)
             tjjs)))
   (t
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref x-%data% (i) ((1 *))
                             x-%offset%)
              (coerce zero 'f2cl-lib:complex16))
       label200))
    (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
            (coerce one 'f2cl-lib:complex16))
    (setf scale zero) (setf xmax zero)))
  label210))
(t
 (setf (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
        (-
         (zladiv
```

```
                              (f2cl-lib:fref x-%data% (j) ((1 *)) x-%offset%)
                              tjjs)
                             csumj))))
                    (setf xmax
                            (max xmax
                                 (cabs1
                                  (f2cl-lib:fref x-%data% (j) ((1 *))
                                                 x-%offset%))))
                   label220))))
            (setf scale (/ scale tscal))))
          (cond
           ((/= tscal one)
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (dscal n (/ one tscal) cnorm 1)
              (declare (ignore var-1 var-2 var-3))
              (when var-0 (setf n var-0)))))
          (go end_label)
         end_label
          (return
           (values uplo trans diag normin n nil lda nil scale nil info)))))))
```

———-

90.238 zrot

90.238.1 Axiom unit tests

— **zrot.input** —

```
)set break resume
)sys rm -f zrot.output
)spool zrot.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.238.2 Axiom help page

— **zrot.help** —

```
====================================================================
zrot examples
====================================================================

====================================================================
Man Page Details
====================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZROT( N, CX, INCX, CY, INCY, C, S )

       .. Scalar Arguments ..
       INTEGER            INCX, INCY, N
       DOUBLE PRECISION   C
       COMPLEX*16         S
       ..
       .. Array Arguments ..
       COMPLEX*16         CX( * ), CY( * )
       ..


 Purpose:
 =============

 ZROT  applies a plane rotation, where the cos (C) is real and the
 sin (S) is complex, and the vectors CX and CY are complex.

 Arguments:
 ==========

  [in] N

          N is INTEGER
          The number of elements in the vectors CX and CY.

  [in,out] CX

          CX is COMPLEX*16 array, dimension (N)
          On input, the vector X.
          On output, CX is overwritten with C*X + S*Y.

  [in] INCX

          INCX is INTEGER
          The increment between successive values of CY.  INCX <> 0.
```

```
[in,out] CY

        CY is COMPLEX*16 array, dimension (N)
        On input, the vector Y.
        On output, CY is overwritten with -CONJG(S)*X + C*Y.

[in] INCY

        INCY is INTEGER
        The increment between successive values of CY.  INCX <> 0.

[in] C

        C is DOUBLE PRECISION

[in] S

        S is COMPLEX*16
        C and S define a rotation
           [  C          S  ]
           [ -conjg(S)   C  ]
        where C*C + S*CONJG(S) = 1.0.
```

———-

90.238.3 fortran code

— **zrot.f** —

```
*  =====================================================================
      SUBROUTINE ZROT( N, CX, INCX, CY, INCY, C, S )
*
*  -- LAPACK auxiliary routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INCX, INCY, N
      DOUBLE PRECISION   C
      COMPLEX*16         S
*     ..
*     .. Array Arguments ..
      COMPLEX*16         CX( * ), CY( * )
*     ..
*
```

```
* =====================================================================
*
*     .. Local Scalars ..
      INTEGER            I, IX, IY
      COMPLEX*16         STEMP
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DCONJG
*     ..
*     .. Executable Statements ..
*
      IF( N.LE.0 )
     $   RETURN
      IF( INCX.EQ.1 .AND. INCY.EQ.1 )
     $   GO TO 20
*
*     Code for unequal increments or equal increments not equal to 1
*
      IX = 1
      IY = 1
      IF( INCX.LT.0 )
     $   IX = ( -N+1 )*INCX + 1
      IF( INCY.LT.0 )
     $   IY = ( -N+1 )*INCY + 1
      DO 10 I = 1, N
         STEMP = C*CX( IX ) + S*CY( IY )
         CY( IY ) = C*CY( IY ) - DCONJG( S )*CX( IX )
         CX( IX ) = STEMP
         IX = IX + INCX
         IY = IY + INCY
   10 CONTINUE
      RETURN
*
*     Code for both increments equal to 1
*
   20 CONTINUE
      DO 30 I = 1, N
         STEMP = C*CX( I ) + S*CY( I )
         CY( I ) = C*CY( I ) - DCONJG( S )*CX( I )
         CX( I ) = STEMP
   30 CONTINUE
      RETURN
      END
```

90.238.4 lisp code

— zrot.lisp —

```
(defun zrot (n cx incx cy incy c s)
  (declare (type (f2cl-lib:complex16) s)
           (type (double-float) c)
           (type (array f2cl-lib:complex16 (*)) cy cx)
           (type (f2cl-lib:integer4) incy incx n))
  (f2cl-lib:with-multi-array-data
      ((cx f2cl-lib:complex16 cx-%data% cx-%offset%)
       (cy f2cl-lib:complex16 cy-%data% cy-%offset%))
    (prog ((stemp #C(0.0d0 0.0d0)) (i 0) (ix 0) (iy 0))
      (declare (type (f2cl-lib:integer4) iy ix i)
               (type (f2cl-lib:complex16) stemp))
      (if (<= n 0)
          (go end_label))
      (if (and (= incx 1) (= incy 1))
          (go label20))
      (setf ix 1)
      (setf iy 1)
      (if (< incx 0)
          (setf ix
                  (f2cl-lib:int-add
                   (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incx) 1)))
      (if (< incy 0)
          (setf iy
                  (f2cl-lib:int-add
                   (f2cl-lib:int-mul (f2cl-lib:int-sub 1 n) incy) 1)))
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf stemp
                  (+ (* c (f2cl-lib:fref cx-%data% (ix) ((1 *)) cx-%offset%))
                     (* s (f2cl-lib:fref cy-%data% (iy) ((1 *)) cy-%offset%))))
          (setf (f2cl-lib:fref cy-%data% (iy) ((1 *)) cy-%offset%)
                  (- (* c (f2cl-lib:fref cy-%data% (iy) ((1 *)) cy-%offset%))
                     (* (f2cl-lib:dconjg s)
                        (f2cl-lib:fref cx-%data% (ix) ((1 *)) cx-%offset%))))
          (setf (f2cl-lib:fref cx-%data% (ix) ((1 *)) cx-%offset%) stemp)
          (setf ix (f2cl-lib:int-add ix incx))
          (setf iy (f2cl-lib:int-add iy incy))
         label10))
      (go end_label)
     label20
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i n) nil)
        (tagbody
          (setf stemp
```

```
                (+ (* c (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%))
                   (* s (f2cl-lib:fref cy-%data% (i) ((1 *)) cy-%offset%))))
        (setf (f2cl-lib:fref cy-%data% (i) ((1 *)) cy-%offset%)
                (- (* c (f2cl-lib:fref cy-%data% (i) ((1 *)) cy-%offset%))
                   (* (f2cl-lib:dconjg s)
                      (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%))))
        (setf (f2cl-lib:fref cx-%data% (i) ((1 *)) cx-%offset%) stemp)
       label30))
     (go end_label)
    end_label
     (return (values nil nil nil nil nil nil nil)))))
```

———-

90.239 ztrevc

90.239.1 Axiom unit tests

— **ztrevc.input** —

```
)set break resume
)sys rm -f ztrevc.output
)spool ztrevc.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.239.2 Axiom help page

— **ztrevc.help** —

```
==========================================================================
ztrevc examples
==========================================================================

==========================================================================
Man Page Details
==========================================================================
```

```
Online html documentation available at
           http://www.netlib.org/lapack/explore-html/

Definition:
===========

      SUBROUTINE ZTREVC( SIDE, HOWMNY, SELECT, N, T, LDT, VL, LDVL, VR,
                         LDVR, MM, M, WORK, RWORK, INFO )

      .. Scalar Arguments ..
      CHARACTER          HOWMNY, SIDE
      INTEGER            INFO, LDT, LDVL, LDVR, M, MM, N
      ..
      .. Array Arguments ..
      LOGICAL            SELECT( * )
      DOUBLE PRECISION   RWORK( * )
      COMPLEX*16         T( LDT, * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   WORK( * )
      ..


Purpose:
=============

ZTREVC computes some or all of the right and/or left eigenvectors of
a complex upper triangular matrix T.
Matrices of this type are produced by the Schur factorization of
a complex general matrix:  A = Q*T*Q**H, as computed by ZHSEQR.

The right eigenvector x and the left eigenvector y of T corresponding
to an eigenvalue w are defined by:

             T*x = w*x,     (y**H)*T = w*(y**H)

where y**H denotes the conjugate transpose of the vector y.
The eigenvalues are not input to this routine, but are read directly
from the diagonal of T.

This routine returns the matrices X and/or Y of right and left
eigenvectors of T, or the products Q*X and/or Q*Y, where Q is an
input matrix.  If Q is the unitary factor that reduces a matrix A to
Schur form T, then Q*X and Q*Y are the matrices of right and left
eigenvectors of A.

Arguments:
==========

 [in] SIDE
```

```
        SIDE is CHARACTER*1
        = 'R':  compute right eigenvectors only;
        = 'L':  compute left eigenvectors only;
        = 'B':  compute both right and left eigenvectors.

[in] HOWMNY

        HOWMNY is CHARACTER*1
        = 'A':  compute all right and/or left eigenvectors;
        = 'B':  compute all right and/or left eigenvectors,
                backtransformed using the matrices supplied in
                VR and/or VL;
        = 'S':  compute selected right and/or left eigenvectors,
                as indicated by the logical array SELECT.

[in] SELECT

        SELECT is LOGICAL array, dimension (N)
        If HOWMNY = 'S', SELECT specifies the eigenvectors to be
        computed.
        The eigenvector corresponding to the j-th eigenvalue is
        computed if SELECT(j) = .TRUE..
        Not referenced if HOWMNY = 'A' or 'B'.

[in] N

        N is INTEGER
        The order of the matrix T. N >= 0.

[in,out] T

        T is COMPLEX*16 array, dimension (LDT,N)
        The upper triangular matrix T.  T is modified, but restored
        on exit.

[in] LDT

        LDT is INTEGER
        The leading dimension of the array T. LDT >= max(1,N).

[in,out] VL

        VL is COMPLEX*16 array, dimension (LDVL,MM)
        On entry, if SIDE = 'L' or 'B' and HOWMNY = 'B', VL must
        contain an N-by-N matrix Q (usually the unitary matrix Q of
        Schur vectors returned by ZHSEQR).
        On exit, if SIDE = 'L' or 'B', VL contains:
        if HOWMNY = 'A', the matrix Y of left eigenvectors of T;
        if HOWMNY = 'B', the matrix Q*Y;
        if HOWMNY = 'S', the left eigenvectors of T specified by
```

```
                         SELECT, stored consecutively in the columns
                         of VL, in the same order as their
                         eigenvalues.
        Not referenced if SIDE = 'R'.

[in] LDVL

        LDVL is INTEGER
        The leading dimension of the array VL.  LDVL >= 1, and if
        SIDE = 'L' or 'B', LDVL >= N.

[in,out] VR

        VR is COMPLEX*16 array, dimension (LDVR,MM)
        On entry, if SIDE = 'R' or 'B' and HOWMNY = 'B', VR must
        contain an N-by-N matrix Q (usually the unitary matrix Q of
        Schur vectors returned by ZHSEQR).
        On exit, if SIDE = 'R' or 'B', VR contains:
        if HOWMNY = 'A', the matrix X of right eigenvectors of T;
        if HOWMNY = 'B', the matrix Q*X;
        if HOWMNY = 'S', the right eigenvectors of T specified by
                         SELECT, stored consecutively in the columns
                         of VR, in the same order as their
                         eigenvalues.
        Not referenced if SIDE = 'L'.

[in] LDVR

        LDVR is INTEGER
        The leading dimension of the array VR.  LDVR >= 1, and if
        SIDE = 'R' or 'B'; LDVR >= N.

[in] MM

        MM is INTEGER
        The number of columns in the arrays VL and/or VR. MM >= M.

[out] M

        M is INTEGER
        The number of columns in the arrays VL and/or VR actually
        used to store the eigenvectors.  If HOWMNY = 'A' or 'B', M
        is set to N.  Each selected eigenvector occupies one
        column.

[out] WORK

        WORK is COMPLEX*16 array, dimension (2*N)

[out] RWORK
```

```
          RWORK is DOUBLE PRECISION array, dimension (N)

  [out] INFO

          INFO is INTEGER
          = 0:  successful exit
          < 0:  if INFO = -i, the i-th argument had an illegal value

 Further Details:
 ====================

  The algorithm used in this program is basically backward (forward)
  substitution, with scaling to make the the code robust against
  possible overflow.

  Each eigenvector is normalized so that the element of largest
  magnitude has magnitude 1; here the magnitude of a complex number
  (x,y) is taken to be |x| + |y|.
```

———-

90.239.3 fortran code

— **ztrevc.f** —

```
*  =====================================================================
      SUBROUTINE ZTREVC( SIDE, HOWMNY, SELECT, N, T, LDT, VL, LDVL, VR,
     $                   LDVR, MM, M, WORK, RWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          HOWMNY, SIDE
      INTEGER            INFO, LDT, LDVL, LDVR, M, MM, N
*     ..
*     .. Array Arguments ..
      LOGICAL            SELECT( * )
      DOUBLE PRECISION   RWORK( * )
      COMPLEX*16         T( LDT, * ), VL( LDVL, * ), VR( LDVR, * ),
     $                   WORK( * )
*     ..
*
*  =====================================================================
```

```
*
*     .. Parameters ..
      DOUBLE PRECISION   ZERO, ONE
      PARAMETER          ( ZERO = 0.0D+0, ONE = 1.0D+0 )
      COMPLEX*16         CMZERO, CMONE
      PARAMETER          ( CMZERO = ( 0.0D+0, 0.0D+0 ),
     $                   CMONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            ALLV, BOTHV, LEFTV, OVER, RIGHTV, SOMEV
      INTEGER            I, II, IS, J, K, KI
      DOUBLE PRECISION   OVFL, REMAX, SCALE, SMIN, SMLNUM, ULP, UNFL
      COMPLEX*16         CDUM
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            IZAMAX
      DOUBLE PRECISION   DLAMCH, DZASUM
      EXTERNAL           LSAME, IZAMAX, DLAMCH, DZASUM
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZCOPY, ZDSCAL, ZGEMV, ZLATRS
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          ABS, DBLE, DCMPLX, DCONJG, DIMAG, MAX
*     ..
*     .. Statement Functions ..
      DOUBLE PRECISION   CABS1
*     ..
*     .. Statement Function definitions ..
      CABS1( CDUM ) = ABS( DBLE( CDUM ) ) + ABS( DIMAG( CDUM ) )
*     ..
*     .. Executable Statements ..
*
*     Decode and test the input parameters
*
      BOTHV = LSAME( SIDE, 'B' )
      RIGHTV = LSAME( SIDE, 'R' ) .OR. BOTHV
      LEFTV = LSAME( SIDE, 'L' ) .OR. BOTHV
*
      ALLV = LSAME( HOWMNY, 'A' )
      OVER = LSAME( HOWMNY, 'B' )
      SOMEV = LSAME( HOWMNY, 'S' )
*
*     Set M to the number of columns required to store the selected
*     eigenvectors.
*
      IF( SOMEV ) THEN
         M = 0
         DO 10 J = 1, N
```

```
            IF( SELECT( J ) )
     $         M = M + 1
   10    CONTINUE
      ELSE
         M = N
      END IF
*
      INFO = 0
      IF( .NOT.RIGHTV .AND. .NOT.LEFTV ) THEN
         INFO = -1
      ELSE IF( .NOT.ALLV .AND. .NOT.OVER .AND. .NOT.SOMEV ) THEN
         INFO = -2
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( LDT.LT.MAX( 1, N ) ) THEN
         INFO = -6
      ELSE IF( LDVL.LT.1 .OR. ( LEFTV .AND. LDVL.LT.N ) ) THEN
         INFO = -8
      ELSE IF( LDVR.LT.1 .OR. ( RIGHTV .AND. LDVR.LT.N ) ) THEN
         INFO = -10
      ELSE IF( MM.LT.M ) THEN
         INFO = -11
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZTREVC', -INFO )
         RETURN
      END IF
*
*     Quick return if possible.
*
      IF( N.EQ.0 )
     $   RETURN
*
*     Set the constants to control overflow.
*
      UNFL = DLAMCH( 'Safe minimum' )
      OVFL = ONE / UNFL
      CALL DLABAD( UNFL, OVFL )
      ULP = DLAMCH( 'Precision' )
      SMLNUM = UNFL*( N / ULP )
*
*     Store the diagonal elements of T in working array WORK.
*
      DO 20 I = 1, N
         WORK( I+N ) = T( I, I )
   20 CONTINUE
*
*     Compute 1-norm of each column of strictly upper triangular
*     part of T to control overflow in triangular solver.
*
```

```
      RWORK( 1 ) = ZERO
      DO 30 J = 2, N
         RWORK( J ) = DZASUM( J-1, T( 1, J ), 1 )
   30 CONTINUE
*
      IF( RIGHTV ) THEN
*
*        Compute right eigenvectors.
*
         IS = M
         DO 80 KI = N, 1, -1
*
            IF( SOMEV ) THEN
               IF( .NOT.SELECT( KI ) )
     $            GO TO 80
            END IF
            SMIN = MAX( ULP*( CABS1( T( KI, KI ) ) ), SMLNUM )
*
            WORK( 1 ) = CMONE
*
*           Form right-hand side.
*
            DO 40 K = 1, KI - 1
               WORK( K ) = -T( K, KI )
   40       CONTINUE
*
*           Solve the triangular system:
*              (T(1:KI-1,1:KI-1) - T(KI,KI))*X = SCALE*WORK.
*
            DO 50 K = 1, KI - 1
               T( K, K ) = T( K, K ) - T( KI, KI )
               IF( CABS1( T( K, K ) ).LT.SMIN )
     $            T( K, K ) = SMIN
   50       CONTINUE
*
            IF( KI.GT.1 ) THEN
               CALL ZLATRS( 'Upper', 'No transpose', 'Non-unit', 'Y',
     $                    KI-1, T, LDT, WORK( 1 ), SCALE, RWORK,
     $                    INFO )
               WORK( KI ) = SCALE
            END IF
*
*           Copy the vector x or Q*x to VR and normalize.
*
            IF( .NOT.OVER ) THEN
               CALL ZCOPY( KI, WORK( 1 ), 1, VR( 1, IS ), 1 )
*
               II = IZAMAX( KI, VR( 1, IS ), 1 )
               REMAX = ONE / CABS1( VR( II, IS ) )
               CALL ZDSCAL( KI, REMAX, VR( 1, IS ), 1 )
```

```
*
               DO 60 K = KI + 1, N
                  VR( K, IS ) = CMZERO
   60          CONTINUE
            ELSE
               IF( KI.GT.1 )
     $            CALL ZGEMV( 'N', N, KI-1, CMONE, VR, LDVR, WORK( 1 ),
     $                        1, DCMPLX( SCALE ), VR( 1, KI ), 1 )
*
               II = IZAMAX( N, VR( 1, KI ), 1 )
               REMAX = ONE / CABS1( VR( II, KI ) )
               CALL ZDSCAL( N, REMAX, VR( 1, KI ), 1 )
            END IF
*
*           Set back the original diagonal elements of T.
*
            DO 70 K = 1, KI - 1
               T( K, K ) = WORK( K+N )
   70       CONTINUE
*
            IS = IS - 1
   80    CONTINUE
      END IF
*
      IF( LEFTV ) THEN
*
*        Compute left eigenvectors.
*
         IS = 1
         DO 130 KI = 1, N
*
            IF( SOMEV ) THEN
               IF( .NOT.SELECT( KI ) )
     $            GO TO 130
            END IF
            SMIN = MAX( ULP*( CABS1( T( KI, KI ) ) ), SMLNUM )
*
            WORK( N ) = CMONE
*
*           Form right-hand side.
*
            DO 90 K = KI + 1, N
               WORK( K ) = -DCONJG( T( KI, K ) )
   90       CONTINUE
*
*           Solve the triangular system:
*              (T(KI+1:N,KI+1:N) - T(KI,KI))**H * X = SCALE*WORK.
*
            DO 100 K = KI + 1, N
               T( K, K ) = T( K, K ) - T( KI, KI )
```

```
               IF( CABS1( T( K, K ) ).LT.SMIN )
     $            T( K, K ) = SMIN
  100       CONTINUE
*
            IF( KI.LT.N ) THEN
               CALL ZLATRS( 'Upper', 'Conjugate transpose', 'Non-unit',
     $                      'Y', N-KI, T( KI+1, KI+1 ), LDT,
     $                      WORK( KI+1 ), SCALE, RWORK, INFO )
               WORK( KI ) = SCALE
            END IF
*
*           Copy the vector x or Q*x to VL and normalize.
*
            IF( .NOT.OVER ) THEN
               CALL ZCOPY( N-KI+1, WORK( KI ), 1, VL( KI, IS ), 1 )
*
               II = IZAMAX( N-KI+1, VL( KI, IS ), 1 ) + KI - 1
               REMAX = ONE / CABS1( VL( II, IS ) )
               CALL ZDSCAL( N-KI+1, REMAX, VL( KI, IS ), 1 )
*
               DO 110 K = 1, KI - 1
                  VL( K, IS ) = CMZERO
  110          CONTINUE
            ELSE
               IF( KI.LT.N )
     $            CALL ZGEMV( 'N', N, N-KI, CMONE, VL( 1, KI+1 ), LDVL,
     $                        WORK( KI+1 ), 1, DCMPLX( SCALE ),
     $                        VL( 1, KI ), 1 )
*
               II = IZAMAX( N, VL( 1, KI ), 1 )
               REMAX = ONE / CABS1( VL( II, KI ) )
               CALL ZDSCAL( N, REMAX, VL( 1, KI ), 1 )
            END IF
*
*           Set back the original diagonal elements of T.
*
            DO 120 K = KI + 1, N
               T( K, K ) = WORK( K+N )
  120       CONTINUE
*
            IS = IS + 1
  130    CONTINUE
      END IF
*
      RETURN
*
*     End of ZTREVC
*
      END
```

———-

90.239.4 lisp code

— ztrevc.lisp —

```
(let* ((zero 0.0d0)
       (one 1.0d0)
       (cmzero (f2cl-lib:cmplx 0.0d0 0.0d0))
       (cmone (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (double-float 0.0d0 0.0d0) zero)
           (type (double-float 1.0d0 1.0d0) one)
           (type (f2cl-lib:complex16) cmzero)
           (type (f2cl-lib:complex16) cmone)
           (ignorable zero one cmzero cmone))
  (defun ztrevc
         (side howmny select n t$ ldt vl ldvl vr ldvr mm m work rwork info)
    (declare (type (array double-float (*)) rwork)
             (type (array f2cl-lib:complex16 (*)) work vr vl t$)
             (type (f2cl-lib:integer4) info m mm ldvr ldvl ldt n)
             (type (array f2cl-lib:logical (*)) select)
             (type (string *) howmny side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (howmny character howmny-%data% howmny-%offset%)
         (select f2cl-lib:logical select-%data% select-%offset%)
         (t$ f2cl-lib:complex16 t$-%data% t$-%offset%)
         (vl f2cl-lib:complex16 vl-%data% vl-%offset%)
         (vr f2cl-lib:complex16 vr-%data% vr-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%)
         (rwork double-float rwork-%data% rwork-%offset%))
      (labels ((cabs1 (cdum)
                 (+ (abs (f2cl-lib:dble cdum)) (abs (f2cl-lib:dimag cdum)))))
        (declare
         (ftype (function (f2cl-lib:complex16) (values double-float &rest t))
          cabs1))
        (prog ((cdum #C(0.0d0 0.0d0)) (ovfl 0.0d0) (remax 0.0d0) (scale 0.0d0)
               (smin 0.0d0) (smlnum 0.0d0) (ulp 0.0d0) (unfl 0.0d0) (i 0)
               (ii 0) (is 0) (j 0) (k 0) (ki 0) (allv nil) (bothv nil)
               (leftv nil) (over nil) (rightv nil) (somev nil))
          (declare (type (f2cl-lib:complex16) cdum)
                   (type (double-float) ovfl remax scale smin smlnum ulp unfl)
                   (type (f2cl-lib:integer4) i ii is j k ki)
                   (type f2cl-lib:logical allv bothv leftv over rightv somev))
          (setf bothv
                  (multiple-value-bind (ret-val var-0 var-1)
                      (lsame side "B")
                    (declare (ignore var-1))
```

```
          (when var-0 (setf side var-0))
          ret-val))
(setf rightv
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame side "R")
           (declare (ignore var-1))
           (when var-0 (setf side var-0))
           ret-val)
         bothv))
(setf leftv
        (or
         (multiple-value-bind (ret-val var-0 var-1)
             (lsame side "L")
           (declare (ignore var-1))
           (when var-0 (setf side var-0))
           ret-val)
         bothv))
(setf allv
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame howmny "A")
          (declare (ignore var-1))
          (when var-0 (setf howmny var-0))
          ret-val))
(setf over
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame howmny "B")
          (declare (ignore var-1))
          (when var-0 (setf howmny var-0))
          ret-val))
(setf somev
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame howmny "S")
          (declare (ignore var-1))
          (when var-0 (setf howmny var-0))
          ret-val))
(cond
 (somev (setf m 0)
  (f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (if (f2cl-lib:fref select-%data% (j) ((1 *)) select-%offset%)
          (setf m (f2cl-lib:int-add m 1)))
     label10)))
 (t (setf m n)))
(setf info 0)
(cond ((and (not rightv) (not leftv)) (setf info -1))
      ((and (not allv) (not over) (not somev)) (setf info -2))
      ((< n 0) (setf info -4))
      ((< ldt
```

```
          (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
       (setf info -6))
      ((or (< ldvl 1) (and leftv (< ldvl n))) (setf info -8))
      ((or (< ldvr 1) (and rightv (< ldvr n))) (setf info -10))
      ((< mm m) (setf info -11)))
(cond
 ((/= info 0) (xerbla "ZTREVC" (f2cl-lib:int-sub info))
  (go end_label)))
(if (= n 0)
    (go end_label))
(setf unfl (dlamch "Safe minimum"))
(setf ovfl (/ one unfl))
(multiple-value-bind (var-0 var-1)
    (dlabad unfl ovfl)
  (declare (ignore))
  (when var-0 (setf unfl var-0))
  (when var-1 (setf ovfl var-1)))
(setf ulp (dlamch "Precision"))
(setf smlnum (* unfl (/ n ulp)))
(f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
              ((> i n) nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% ((f2cl-lib:int-add i n)) ((1 *))
                         work-%offset%)
            (f2cl-lib:fref t$-%data% (i i) ((1 ldt) (1 *))
                           t$-%offset%))
   label20))
(setf (f2cl-lib:fref rwork-%data% (1) ((1 *)) rwork-%offset%) zero)
(f2cl-lib:fdo (j 2 (f2cl-lib:int-add j 1))
              ((> j n) nil)
  (tagbody
    (setf (f2cl-lib:fref rwork-%data% (j) ((1 *)) rwork-%offset%)
            (dzasum (f2cl-lib:int-sub j 1)
             (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 j)
                                   ((1 ldt) (1 *)) t$-%offset%)
             1))
   label30))
(cond
 (rightv (setf is m)
  (f2cl-lib:fdo (ki n (f2cl-lib:int-add ki (f2cl-lib:int-sub 1)))
                ((> ki 1) nil)
    (tagbody
      (cond
       (somev
        (if (not
             (f2cl-lib:fref select-%data% (ki) ((1 *))
                            select-%offset%))
            (go label80))))
      (setf smin
              (max
```

```
         (* ulp
            (cabs1
             (f2cl-lib:fref t$-%data% (ki ki) ((1 ldt) (1 *))
                            t$-%offset%)))
         smlnum))
(setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        cmone)
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k (f2cl-lib:int-add ki (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
            (-
             (f2cl-lib:fref t$-%data% (k ki) ((1 ldt) (1 *))
                            t$-%offset%)))
   label40))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k (f2cl-lib:int-add ki (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref t$-%data% (k k) ((1 ldt) (1 *))
                         t$-%offset%)
            (-
             (f2cl-lib:fref t$-%data% (k k) ((1 ldt) (1 *))
                            t$-%offset%)
             (f2cl-lib:fref t$-%data% (ki ki) ((1 ldt) (1 *))
                            t$-%offset%)))
    (if (<
         (cabs1
          (f2cl-lib:fref t$-%data% (k k) ((1 ldt) (1 *))
                         t$-%offset%))
         smin)
        (setf (f2cl-lib:fref t$-%data% (k k) ((1 ldt) (1 *))
                             t$-%offset%)
                (coerce smin 'f2cl-lib:complex16)))
   label50))
(cond
 ((> ki 1)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (zlatrs "Upper" "No transpose" "Non-unit" "Y"
       (f2cl-lib:int-sub ki 1) t$ ldt
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (1)
                             ((1 *)) work-%offset%)
       scale rwork info)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9))
    (setf ldt var-6)
    (setf scale var-8)
```

```
    (setf info var-10))
  (setf (f2cl-lib:fref work-%data% (ki) ((1 *)) work-%offset%)
          (coerce scale 'f2cl-lib:complex16))))
(cond
 ((not over)
  (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
      (zcopy ki
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (1)
                             ((1 *)) work-%offset%)
       1
       (f2cl-lib:array-slice vr-%data% f2cl-lib:complex16
                             (1 is) ((1 ldvr) (1 *))
                             vr-%offset%)
       1)
    (declare (ignore var-1 var-2 var-3 var-4))
    (when var-0 (setf ki var-0)))
  (setf ii
          (multiple-value-bind (ret-val var-0 var-1 var-2)
              (izamax ki
               (f2cl-lib:array-slice vr-%data%
                                     f2cl-lib:complex16 (1 is)
                                     ((1 ldvr) (1 *))
                                     vr-%offset%)
               1)
            (declare (ignore var-1 var-2))
            (when var-0 (setf ki var-0))
            ret-val))
  (setf remax
          (/ one
             (cabs1
              (f2cl-lib:fref vr-%data% (ii is) ((1 ldvr) (1 *))
                             vr-%offset%))))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (zdscal ki remax
       (f2cl-lib:array-slice vr-%data% f2cl-lib:complex16
                             (1 is) ((1 ldvr) (1 *))
                             vr-%offset%)
       1)
    (declare (ignore var-2 var-3))
    (when var-0 (setf ki var-0))
    (when var-1 (setf remax var-1)))
  (f2cl-lib:fdo (k (f2cl-lib:int-add ki 1)
                 (f2cl-lib:int-add k 1))
                ((> k n) nil)
    (tagbody
      (setf (f2cl-lib:fref vr-%data% (k is) ((1 ldvr) (1 *))
                           vr-%offset%)
            cmzero)
     label60)))
 (t
```

```
  (if (> ki 1)
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
           var-8 var-9 var-10)
          (zgemv "N" n (f2cl-lib:int-sub ki 1) cmone vr ldvr
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                 (1) ((1 *)) work-%offset%)
           1 (f2cl-lib:dcmplx scale)
           (f2cl-lib:array-slice vr-%data% f2cl-lib:complex16
                                 (1 ki) ((1 ldvr) (1 *))
                                 vr-%offset%)
           1)
        (declare
         (ignore var-0 var-2 var-4 var-6 var-7 var-8 var-9
          var-10))
        (when var-1 (setf n var-1))
        (when var-3 (setf cmone var-3))
        (when var-5 (setf ldvr var-5))))
  (setf ii
          (multiple-value-bind (ret-val var-0 var-1 var-2)
              (izamax n
               (f2cl-lib:array-slice vr-%data%
                                     f2cl-lib:complex16 (1 ki)
                                     ((1 ldvr) (1 *))
                                     vr-%offset%)
               1)
            (declare (ignore var-1 var-2))
            (when var-0 (setf n var-0))
            ret-val))
  (setf remax
          (/ one
             (cabs1
              (f2cl-lib:fref vr-%data% (ii ki) ((1 ldvr) (1 *))
                             vr-%offset%))))
  (multiple-value-bind (var-0 var-1 var-2 var-3)
      (zdscal n remax
       (f2cl-lib:array-slice vr-%data% f2cl-lib:complex16
                             (1 ki) ((1 ldvr) (1 *))
                             vr-%offset%)
       1)
    (declare (ignore var-2 var-3))
    (when var-0 (setf n var-0))
    (when var-1 (setf remax var-1)))))
(f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
              ((> k (f2cl-lib:int-add ki (f2cl-lib:int-sub 1)))
               nil)
  (tagbody
    (setf (f2cl-lib:fref t$-%data% (k k) ((1 ldt) (1 *))
                         t$-%offset%)
            (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k n))
```

```
                                  ((1 *)) work-%offset%))
            label70))
          (setf is (f2cl-lib:int-sub is 1))
         label80))))
     (cond
      (leftv (setf is 1)
       (f2cl-lib:fdo (ki 1 (f2cl-lib:int-add ki 1))
                     ((> ki n) nil)
         (tagbody
           (cond
            (somev
             (if (not
                  (f2cl-lib:fref select-%data% (ki) ((1 *))
                                 select-%offset%))
                 (go label130))))
           (setf smin
                 (max
                  (* ulp
                     (cabs1
                      (f2cl-lib:fref t$-%data% (ki ki) ((1 ldt) (1 *))
                                     t$-%offset%)))
                  smlnum))
           (setf (f2cl-lib:fref work-%data% (n) ((1 *)) work-%offset%)
                 cmone)
           (f2cl-lib:fdo (k (f2cl-lib:int-add ki 1) (f2cl-lib:int-add k 1))
                         ((> k n) nil)
             (tagbody
               (setf (f2cl-lib:fref work-%data% (k) ((1 *)) work-%offset%)
                     (coerce
                      (-
                       (f2cl-lib:dconjg
                        (f2cl-lib:fref t$-%data% (ki k) ((1 ldt) (1 *))
                                       t$-%offset%)))
                      'f2cl-lib:complex16))
            label90))
           (f2cl-lib:fdo (k (f2cl-lib:int-add ki 1) (f2cl-lib:int-add k 1))
                         ((> k n) nil)
             (tagbody
               (setf (f2cl-lib:fref t$-%data% (k k) ((1 ldt) (1 *))
                                    t$-%offset%)
                     (-
                      (f2cl-lib:fref t$-%data% (k k) ((1 ldt) (1 *))
                                     t$-%offset%)
                      (f2cl-lib:fref t$-%data% (ki ki) ((1 ldt) (1 *))
                                     t$-%offset%)))
               (if (<
                    (cabs1
                     (f2cl-lib:fref t$-%data% (k k) ((1 ldt) (1 *))
                                    t$-%offset%))
                    smin)
```

```
        (setf (f2cl-lib:fref t$-%data% (k k) ((1 ldt) (1 *))
                             t$-%offset%)
              (coerce smin 'f2cl-lib:complex16)))
   label100))
(cond
 ((< ki n)
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
       var-9 var-10)
      (zlatrs "Upper" "Conjugate transpose" "Non-unit" "Y"
       (f2cl-lib:int-sub n ki)
       (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16
                             ((+ ki 1) (f2cl-lib:int-add ki 1))
                             ((1 ldt) (1 *)) t$-%offset%)
       ldt
       (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                             ((+ ki 1)) ((1 *)) work-%offset%)
       scale rwork info)
    (declare
     (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9))
    (setf ldt var-6)
    (setf scale var-8)
    (setf info var-10))
  (setf (f2cl-lib:fref work-%data% (ki) ((1 *)) work-%offset%)
        (coerce scale 'f2cl-lib:complex16))))
(cond
 ((not over)
  (zcopy (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1)
   (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (ki)
                         ((1 *)) work-%offset%)
   1
   (f2cl-lib:array-slice vl-%data% f2cl-lib:complex16 (ki is)
                         ((1 ldvl) (1 *)) vl-%offset%)
   1)
  (setf ii
        (f2cl-lib:int-sub
         (f2cl-lib:int-add
          (izamax
           (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1)
           (f2cl-lib:array-slice vl-%data% f2cl-lib:complex16
                                 (ki is) ((1 ldvl) (1 *))
                                 vl-%offset%)
           1)
          ki)
         1))
  (setf remax
        (/ one
           (cabs1
            (f2cl-lib:fref vl-%data% (ii is) ((1 ldvl) (1 *))
                           vl-%offset%))))
```

```
 (multiple-value-bind (var-0 var-1 var-2 var-3)
     (zdscal (f2cl-lib:int-add (f2cl-lib:int-sub n ki) 1)
      remax
      (f2cl-lib:array-slice vl-%data% f2cl-lib:complex16
                            (ki is) ((1 ldvl) (1 *))
                            vl-%offset%)
      1)
   (declare (ignore var-0 var-2 var-3))
   (when var-1 (setf remax var-1)))
 (f2cl-lib:fdo (k 1 (f2cl-lib:int-add k 1))
               ((> k
                   (f2cl-lib:int-add ki (f2cl-lib:int-sub 1)))
                nil)
   (tagbody
     (setf (f2cl-lib:fref vl-%data% (k is) ((1 ldvl) (1 *))
                          vl-%offset%)
           cmzero)
    label110)))
(t
 (if (< ki n)
     (multiple-value-bind
         (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7
          var-8 var-9 var-10)
         (zgemv "N" n (f2cl-lib:int-sub n ki) cmone
          (f2cl-lib:array-slice vl-%data% f2cl-lib:complex16
                                (1 (f2cl-lib:int-add ki 1))
                                ((1 ldvl) (1 *)) vl-%offset%)
          ldvl
          (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                ((+ ki 1)) ((1 *))
                                work-%offset%)
          1 (f2cl-lib:dcmplx scale)
          (f2cl-lib:array-slice vl-%data% f2cl-lib:complex16
                                (1 ki) ((1 ldvl) (1 *))
                                vl-%offset%)
          1)
       (declare
        (ignore var-0 var-2 var-4 var-6 var-7 var-8 var-9
         var-10))
       (when var-1 (setf n var-1))
       (when var-3 (setf cmone var-3))
       (when var-5 (setf ldvl var-5))))
 (setf ii
         (multiple-value-bind (ret-val var-0 var-1 var-2)
             (izamax n
              (f2cl-lib:array-slice vl-%data%
                                    f2cl-lib:complex16 (1 ki)
                                    ((1 ldvl) (1 *))
                                    vl-%offset%)
              1)
```

```
                    (declare (ignore var-1 var-2))
                    (when var-0 (setf n var-0))
                    ret-val))
            (setf remax
                  (/ one
                     (cabs1
                      (f2cl-lib:fref vl-%data% (ii ki) ((1 ldvl) (1 *))
                                     vl-%offset%))))
            (multiple-value-bind (var-0 var-1 var-2 var-3)
                (zdscal n remax
                 (f2cl-lib:array-slice vl-%data% f2cl-lib:complex16
                                       (1 ki) ((1 ldvl) (1 *))
                                       vl-%offset%)
                 1)
              (declare (ignore var-2 var-3))
              (when var-0 (setf n var-0))
              (when var-1 (setf remax var-1)))))
          (f2cl-lib:fdo (k (f2cl-lib:int-add ki 1) (f2cl-lib:int-add k 1))
                        ((> k n) nil)
            (tagbody
              (setf (f2cl-lib:fref t$-%data% (k k) ((1 ldt) (1 *))
                                   t$-%offset%)
                    (f2cl-lib:fref work-%data% ((f2cl-lib:int-add k n))
                                   ((1 *)) work-%offset%))
             label120))
          (setf is (f2cl-lib:int-add is 1))
         label130))))
  (go end_label)
 end_label
  (return
   (values side howmny nil n nil ldt nil ldvl nil ldvr nil m nil nil
           info)))))))
```

———-

90.240 ztrexc

90.240.1 Axiom unit tests

— **ztrexc.input** —

```
)set break resume
)sys rm -f ztrexc.output
)spool ztrexc.output
)set message test on
)set message auto off
```

```
)clear all

)spool
)lisp (bye)
```

———-

90.240.2 Axiom help page

— **ztrexc.help** —

```
==================================================================
ztrexc examples
==================================================================

==================================================================
Man Page Details
==================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZTREXC( COMPQ, N, T, LDT, Q, LDQ, IFST, ILST, INFO )

       .. Scalar Arguments ..
       CHARACTER          COMPQ
       INTEGER            IFST, ILST, INFO, LDQ, LDT, N
       ..
       .. Array Arguments ..
       COMPLEX*16         Q( LDQ, * ), T( LDT, * )
       ..


 Purpose:
 =============

 ZTREXC reorders the Schur factorization of a complex matrix
 A = Q*T*Q**H, so that the diagonal element of T with row index IFST
 is moved to row ILST.

 The Schur form T is reordered by a unitary similarity transformation
 Z**H*T*Z, and optionally the matrix Q of Schur vectors is updated by
 postmultplying it with Z.

 Arguments:
```

```
==========

 [in] COMPQ

         COMPQ is CHARACTER*1
         = 'V':  update the matrix Q of Schur vectors;
         = 'N':  do not update Q.

 [in] N

         N is INTEGER
         The order of the matrix T. N >= 0.

 [in,out] T

         T is COMPLEX*16 array, dimension (LDT,N)
         On entry, the upper triangular matrix T.
         On exit, the reordered upper triangular matrix.

 [in] LDT

         LDT is INTEGER
         The leading dimension of the array T. LDT >= max(1,N).

 [in,out] Q

         Q is COMPLEX*16 array, dimension (LDQ,N)
         On entry, if COMPQ = 'V', the matrix Q of Schur vectors.
         On exit, if COMPQ = 'V', Q has been postmultiplied by the
         unitary transformation matrix Z which reorders T.
         If COMPQ = 'N', Q is not referenced.

 [in] LDQ

         LDQ is INTEGER
         The leading dimension of the array Q.  LDQ >= max(1,N).

 [in] IFST

         IFST is INTEGER

 [in] ILST

         ILST is INTEGER

         Specify the reordering of the diagonal elements of T:
         The element with row index IFST is moved to row ILST by a
         sequence of transpositions between adjacent elements.
         1 <= IFST <= N; 1 <= ILST <= N.
```

```
[out] INFO

        INFO is INTEGER
        = 0:  successful exit
        < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

90.240.3 fortran code

— **ztrexc.f** —

```
*  =====================================================================
      SUBROUTINE ZTREXC( COMPQ, N, T, LDT, Q, LDQ, IFST, ILST, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          COMPQ
      INTEGER            IFST, ILST, INFO, LDQ, LDT, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         Q( LDQ, * ), T( LDT, * )
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            WANTQ
      INTEGER            K, M1, M2, M3
      DOUBLE PRECISION   CS
      COMPLEX*16         SN, T11, T22, TEMP
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLARTG, ZROT
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DCONJG, MAX
*     ..
*     .. Executable Statements ..
```

```
*
*     Decode and test the input parameters.
*
      INFO = 0
      WANTQ = LSAME( COMPQ, 'V' )
      IF( .NOT.LSAME( COMPQ, 'N' ) .AND. .NOT.WANTQ ) THEN
         INFO = -1
      ELSE IF( N.LT.0 ) THEN
         INFO = -2
      ELSE IF( LDT.LT.MAX( 1, N ) ) THEN
         INFO = -4
      ELSE IF( LDQ.LT.1 .OR. ( WANTQ .AND. LDQ.LT.MAX( 1, N ) ) ) THEN
         INFO = -6
      ELSE IF( IFST.LT.1 .OR. IFST.GT.N ) THEN
         INFO = -7
      ELSE IF( ILST.LT.1 .OR. ILST.GT.N ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZTREXC', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.EQ.1 .OR. IFST.EQ.ILST )
     $   RETURN
*
      IF( IFST.LT.ILST ) THEN
*
*        Move the IFST-th diagonal element forward down the diagonal.
*
         M1 = 0
         M2 = -1
         M3 = 1
      ELSE
*
*        Move the IFST-th diagonal element backward up the diagonal.
*
         M1 = -1
         M2 = 0
         M3 = -1
      END IF
*
      DO 10 K = IFST + M1, ILST + M2, M3
*
*        Interchange the k-th and (k+1)-th diagonal elements.
*
         T11 = T( K, K )
         T22 = T( K+1, K+1 )
```

```
*
*        Determine the transformation to perform the interchange.
*
         CALL ZLARTG( T( K, K+1 ), T22-T11, CS, SN, TEMP )
*
*        Apply transformation to the matrix T.
*
         IF( K+2.LE.N )
     $      CALL ZROT( N-K-1, T( K, K+2 ), LDT, T( K+1, K+2 ), LDT, CS,
     $                 SN )
         CALL ZROT( K-1, T( 1, K ), 1, T( 1, K+1 ), 1, CS,
     $              DCONJG( SN ) )
*
         T( K, K ) = T22
         T( K+1, K+1 ) = T11
*
         IF( WANTQ ) THEN
*
*           Accumulate transformation in the matrix Q.
*
            CALL ZROT( N, Q( 1, K ), 1, Q( 1, K+1 ), 1, CS,
     $                 DCONJG( SN ) )
         END IF
*
   10 CONTINUE
*
      RETURN
*
*     End of ZTREXC
*
      END
```

———-

90.240.4 lisp code

— ztrexc.lisp —

```
(defun ztrexc (compq n t$ ldt q ldq ifst ilst info)
  (declare (type (array f2cl-lib:complex16 (*)) q t$)
           (type (f2cl-lib:integer4) info ilst ifst ldq ldt n)
           (type (string *) compq))
  (f2cl-lib:with-multi-array-data
      ((compq character compq-%data% compq-%offset%)
       (t$ f2cl-lib:complex16 t$-%data% t$-%offset%)
       (q f2cl-lib:complex16 q-%data% q-%offset%))
    (prog ((sn #C(0.0d0 0.0d0)) (t11 #C(0.0d0 0.0d0)) (t22 #C(0.0d0 0.0d0))
```

```
     (temp #C(0.0d0 0.0d0)) (cs 0.0d0) (k 0) (m1 0) (m2 0) (m3 0)
     (wantq nil))
(declare (type f2cl-lib:logical wantq)
         (type (f2cl-lib:integer4) m3 m2 m1 k)
         (type (double-float) cs)
         (type (f2cl-lib:complex16) temp t22 t11 sn))
(setf info 0)
(setf wantq
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame compq "V")
          (declare (ignore var-1))
          (when var-0 (setf compq var-0))
          ret-val))
(cond
 ((and
   (not
    (multiple-value-bind (ret-val var-0 var-1)
        (lsame compq "N")
      (declare (ignore var-1))
      (when var-0 (setf compq var-0))
      ret-val))
   (not wantq))
  (setf info -1))
 ((< n 0) (setf info -2))
 ((< ldt (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
  (setf info -4))
 ((or (< ldq 1)
      (and wantq
           (< ldq
              (max (the f2cl-lib:integer4 1)
                   (the f2cl-lib:integer4 n)))))
  (setf info -6))
 ((or (< ifst 1) (> ifst n)) (setf info -7))
 ((or (< ilst 1) (> ilst n)) (setf info -8)))
(cond
 ((/= info 0) (xerbla "ZTREXC" (f2cl-lib:int-sub info)) (go end_label)))
(if (or (= n 1) (= ifst ilst))
    (go end_label))
(cond ((< ifst ilst) (setf m1 0) (setf m2 -1) (setf m3 1))
      (t (setf m1 -1) (setf m2 0) (setf m3 -1)))
(f2cl-lib:fdo (k (f2cl-lib:int-add ifst m1) (f2cl-lib:int-add k m3))
              ((> k (f2cl-lib:int-add ilst m2)) nil)
  (tagbody
    (setf t11
            (f2cl-lib:fref t$-%data% (k k) ((1 ldt) (1 *)) t$-%offset%))
    (setf t22
            (f2cl-lib:fref t$-%data%
                           ((f2cl-lib:int-add k 1)
                            (f2cl-lib:int-add k 1))
                           ((1 ldt) (1 *)) t$-%offset%))
```

```
      (multiple-value-bind (var-0 var-1 var-2 var-3 var-4)
          (zlartg
           (f2cl-lib:fref t$-%data% (k (f2cl-lib:int-add k 1))
                          ((1 ldt) (1 *)) t$-%offset%)
           (- t22 t11) cs sn temp)
        (declare (ignore var-0 var-1))
        (setf cs var-2)
        (setf sn var-3)
        (setf temp var-4))
      (if (<= (f2cl-lib:int-add k 2) n)
          (zrot (f2cl-lib:int-sub n k 1)
           (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16
                                 (k (f2cl-lib:int-add k 2)) ((1 ldt) (1 *))
                                 t$-%offset%)
           ldt
           (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16
                                 ((+ k 1) (f2cl-lib:int-add k 2))
                                 ((1 ldt) (1 *)) t$-%offset%)
           ldt cs sn))
      (zrot (f2cl-lib:int-sub k 1)
       (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16 (1 k)
                             ((1 ldt) (1 *)) t$-%offset%)
       1
       (f2cl-lib:array-slice t$-%data% f2cl-lib:complex16
                             (1 (f2cl-lib:int-add k 1)) ((1 ldt) (1 *))
                             t$-%offset%)
       1 cs (f2cl-lib:dconjg sn))
      (setf (f2cl-lib:fref t$-%data% (k k) ((1 ldt) (1 *)) t$-%offset%)
            t22)
      (setf (f2cl-lib:fref t$-%data%
                           ((f2cl-lib:int-add k 1) (f2cl-lib:int-add k 1))
                           ((1 ldt) (1 *)) t$-%offset%)
            t11)
      (cond
       (wantq
        (zrot n
         (f2cl-lib:array-slice q-%data% f2cl-lib:complex16 (1 k)
                               ((1 ldq) (1 *)) q-%offset%)
         1
         (f2cl-lib:array-slice q-%data% f2cl-lib:complex16
                               (1 (f2cl-lib:int-add k 1)) ((1 ldq) (1 *))
                               q-%offset%)
         1 cs (f2cl-lib:dconjg sn))))
     label10))
  (go end_label)
 end_label
  (return (values compq nil nil nil nil nil nil nil info)))))
```

———-

90.241 zung2r

90.241.1 Axiom unit tests

— zung2r.input —

```
)set break resume
)sys rm -f zung2r.output
)spool zung2r.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.241.2 Axiom help page

— zung2r.help —

```
======================================================================
zung2r examples
======================================================================

======================================================================
Man Page Details
======================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZUNG2R( M, N, K, A, LDA, TAU, WORK, INFO )

       .. Scalar Arguments ..
       INTEGER            INFO, K, LDA, M, N
       ..
       .. Array Arguments ..
       COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
       ..
```

```
Purpose:
=============

ZUNG2R generates an m by n complex matrix Q with orthonormal columns,
which is defined as the first n columns of a product of k elementary
reflectors of order m

      Q  =  H(1) H(2) . . . H(k)

as returned by ZGEQRF.

Arguments:
==========

 [in] M

         M is INTEGER
         The number of rows of the matrix Q. M >= 0.

 [in] N

         N is INTEGER
         The number of columns of the matrix Q. M >= N >= 0.

 [in] K

         K is INTEGER
         The number of elementary reflectors whose product defines the
         matrix Q. N >= K >= 0.

 [in,out] A

         A is COMPLEX*16 array, dimension (LDA,N)
         On entry, the i-th column must contain the vector which
         defines the elementary reflector H(i), for i = 1,2,...,k, as
         returned by ZGEQRF in the first k columns of its array
         argument A.
         On exit, the m by n matrix Q.

 [in] LDA

         LDA is INTEGER
         The first dimension of the array A. LDA >= max(1,M).

 [in] TAU

         TAU is COMPLEX*16 array, dimension (K)
         TAU(i) must contain the scalar factor of the elementary
         reflector H(i), as returned by ZGEQRF.
```

```
[out] WORK

        WORK is COMPLEX*16 array, dimension (N)

[out] INFO

        INFO is INTEGER
        = 0: successful exit
        < 0: if INFO = -i, the i-th argument has an illegal value
```

———-

90.241.3 fortran code

— **zung2r.f** —

```
*  =====================================================================
      SUBROUTINE ZUNG2R( M, N, K, A, LDA, TAU, WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, K, LDA, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE, ZERO
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ),
     $                   ZERO = ( 0.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      INTEGER            I, J, L
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLARF, ZSCAL
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX
*     ..
```

```
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 .OR. N.GT.M ) THEN
         INFO = -2
      ELSE IF( K.LT.0 .OR. K.GT.N ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZUNG2R', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.LE.0 )
     $   RETURN
*
*     Initialise columns k+1:n to columns of the unit matrix
*
      DO 20 J = K + 1, N
         DO 10 L = 1, M
            A( L, J ) = ZERO
   10    CONTINUE
         A( J, J ) = ONE
   20 CONTINUE
*
      DO 40 I = K, 1, -1
*
*        Apply H(i) to A(i:m,i:n) from the left
*
         IF( I.LT.N ) THEN
            A( I, I ) = ONE
            CALL ZLARF( 'Left', M-I+1, N-I, A( I, I ), 1, TAU( I ),
     $                  A( I, I+1 ), LDA, WORK )
         END IF
         IF( I.LT.M )
     $      CALL ZSCAL( M-I, -TAU( I ), A( I+1, I ), 1 )
         A( I, I ) = ONE - TAU( I )
*
*        Set A(1:i-1,i) to zero
*
         DO 30 L = 1, I - 1
            A( L, I ) = ZERO
```

```
   30    CONTINUE
   40 CONTINUE
      RETURN
*
*     End of ZUNG2R
*
      END
```

———-

90.241.4 lisp code

— **zung2r.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)) (zero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (type (f2cl-lib:complex16) zero)
           (ignorable one zero))
  (defun zung2r (m n k a lda tau work info)
    (declare (type (array f2cl-lib:complex16 (*)) work tau a)
             (type (f2cl-lib:integer4) info lda k n m))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%))
      (prog ((i 0) (j 0) (l 0))
        (declare (type (f2cl-lib:integer4) i j l))
        (setf info 0)
        (cond ((< m 0) (setf info -1)) ((or (< n 0) (> n m)) (setf info -2))
              ((or (< k 0) (> k n)) (setf info -3))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
               (setf info -5)))
        (cond
         ((/= info 0) (xerbla "ZUNG2R" (f2cl-lib:int-sub info))
          (go end_label)))
        (if (<= n 0)
            (go end_label))
        (f2cl-lib:fdo (j (f2cl-lib:int-add k 1) (f2cl-lib:int-add j 1))
                      ((> j n) nil)
          (tagbody
            (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                          ((> l m) nil)
              (tagbody
                (setf (f2cl-lib:fref a-%data% (l j) ((1 lda) (1 *)) a-%offset%)
                        zero)
               label10))
```

```
      (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
              one)
    label20))
  (f2cl-lib:fdo (i k (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                ((> i 1) nil)
    (tagbody
      (cond
       ((< i n)
        (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
                one)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (zlarf "Left" (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:int-sub n i)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             1 (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%)
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   (i (f2cl-lib:int-add i 1))
                                   ((1 lda) (1 *)) a-%offset%)
             lda work)
          (declare
           (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-8))
          (setf lda var-7))))
      (if (< i m)
          (zscal (f2cl-lib:int-sub m i)
           (- (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 ((+ i 1) i)
                                 ((1 lda) (1 *)) a-%offset%)
           1))
      (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
              (- one
                 (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%)))
      (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                    ((> l (f2cl-lib:int-add i (f2cl-lib:int-sub 1))) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (l i) ((1 lda) (1 *)) a-%offset%)
                  zero)
         label30))
    label40))
  (go end_label)
end_label
  (return (values nil nil nil nil lda nil nil info))))))
```

———-

90.242 zunghr

90.242.1 Axiom unit tests

— zunghr.input —

```
)set break resume
)sys rm -f zunghr.output
)spool zunghr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.242.2 Axiom help page

— zunghr.help —

```
======================================================================
zunghr examples
======================================================================

======================================================================
Man Page Details
======================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZUNGHR( N, ILO, IHI, A, LDA, TAU, WORK, LWORK, INFO )

       .. Scalar Arguments ..
       INTEGER            IHI, ILO, INFO, LDA, LWORK, N
       ..
       .. Array Arguments ..
       COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
       ..
```

```
Purpose:
=============

ZUNGHR generates a complex unitary matrix Q which is defined as the
product of IHI-ILO elementary reflectors of order N, as returned by
ZGEHRD:

Q = H(ilo) H(ilo+1) . . . H(ihi-1).

Arguments:
==========

 [in] N

         N is INTEGER
         The order of the matrix Q. N >= 0.

 [in] ILO

         ILO is INTEGER

 [in] IHI

         IHI is INTEGER

         ILO and IHI must have the same values as in the previous call
         of ZGEHRD. Q is equal to the unit matrix except in the
         submatrix Q(ilo+1:ihi,ilo+1:ihi).
         1 <= ILO <= IHI <= N, if N > 0; ILO=1 and IHI=0, if N=0.

 [in,out] A

         A is COMPLEX*16 array, dimension (LDA,N)
         On entry, the vectors which define the elementary reflectors,
         as returned by ZGEHRD.
         On exit, the N-by-N unitary matrix Q.

 [in] LDA

         LDA is INTEGER
         The leading dimension of the array A. LDA >= max(1,N).

 [in] TAU

         TAU is COMPLEX*16 array, dimension (N-1)
         TAU(i) must contain the scalar factor of the elementary
         reflector H(i), as returned by ZGEHRD.

 [out] WORK
```

```
         WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
         On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

  [in] LWORK

         LWORK is INTEGER
         The dimension of the array WORK. LWORK >= IHI-ILO.
         For optimum performance LWORK >= (IHI-ILO)*NB, where NB is
         the optimal blocksize.

         If LWORK = -1, then a workspace query is assumed; the routine
         only calculates the optimal size of the WORK array, returns
         this value as the first entry of the WORK array, and no error
         message related to LWORK is issued by XERBLA.

  [out] INFO

         INFO is INTEGER
         = 0:  successful exit
         < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

90.242.3 fortran code

— zunghr.f —

```
*  =====================================================================
      SUBROUTINE ZUNGHR( N, ILO, IHI, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            IHI, ILO, INFO, LDA, LWORK, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ZERO, ONE
      PARAMETER          ( ZERO = ( 0.0D+0, 0.0D+0 ),
```

```
     $                   ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IINFO, J, LWKOPT, NB, NH
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZUNGQR
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NH = IHI - ILO
      LQUERY = ( LWORK.EQ.-1 )
      IF( N.LT.0 ) THEN
         INFO = -1
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, N ) ) THEN
         INFO = -2
      ELSE IF( IHI.LT.MIN( ILO, N ) .OR. IHI.GT.N ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, N ) ) THEN
         INFO = -5
      ELSE IF( LWORK.LT.MAX( 1, NH ) .AND. .NOT.LQUERY ) THEN
         INFO = -8
      END IF
*
      IF( INFO.EQ.0 ) THEN
         NB = ILAENV( 1, 'ZUNGQR', ' ', NH, NH, NH, -1 )
         LWKOPT = MAX( 1, NH )*NB
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZUNGHR', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
```

```
      IF( N.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
*     Shift the vectors which define the elementary reflectors one
*     column to the right, and set the first ilo and the last n-ihi
*     rows and columns to those of the unit matrix
*
      DO 40 J = IHI, ILO + 1, -1
         DO 10 I = 1, J - 1
            A( I, J ) = ZERO
   10    CONTINUE
         DO 20 I = J + 1, IHI
            A( I, J ) = A( I, J-1 )
   20    CONTINUE
         DO 30 I = IHI + 1, N
            A( I, J ) = ZERO
   30    CONTINUE
   40 CONTINUE
      DO 60 J = 1, ILO
         DO 50 I = 1, N
            A( I, J ) = ZERO
   50    CONTINUE
         A( J, J ) = ONE
   60 CONTINUE
      DO 80 J = IHI + 1, N
         DO 70 I = 1, N
            A( I, J ) = ZERO
   70    CONTINUE
         A( J, J ) = ONE
   80 CONTINUE
*
      IF( NH.GT.0 ) THEN
*
*        Generate Q(ilo+1:ihi,ilo+1:ihi)
*
         CALL ZUNGQR( NH, NH, NH, A( ILO+1, ILO+1 ), LDA, TAU( ILO ),
     $                WORK, LWORK, IINFO )
      END IF
      WORK( 1 ) = LWKOPT
      RETURN
*
*     End of ZUNGHR
*
      END
```

90.242.4 lisp code

— zunghr.lisp —

```
(let* ((zero (f2cl-lib:cmplx 0.0d0 0.0d0)) (one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) zero)
           (type (f2cl-lib:complex16) one)
           (ignorable zero one))
  (defun zunghr (n ilo ihi a lda tau work lwork info)
    (declare (type (array f2cl-lib:complex16 (*)) work tau a)
             (type (f2cl-lib:integer4) info lwork lda ihi ilo n))
    (f2cl-lib:with-multi-array-data
        ((a f2cl-lib:complex16 a-%data% a-%offset%)
         (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%))
      (prog ((i 0) (iinfo 0) (j 0) (lwkopt 0) (nb 0) (nh 0) (lquery nil))
        (declare (type (f2cl-lib:integer4) i iinfo j lwkopt nb nh)
                 (type f2cl-lib:logical lquery))
        (setf info 0)
        (setf nh (f2cl-lib:int-sub ihi ilo))
        (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
        (cond ((< n 0) (setf info -1))
              ((or (< ilo 1)
                   (> ilo
                      (max (the f2cl-lib:integer4 1)
                           (the f2cl-lib:integer4 n))))
               (setf info -2))
              ((or
                (< ihi
                   (min (the f2cl-lib:integer4 ilo) (the f2cl-lib:integer4 n)))
                (> ihi n))
               (setf info -3))
              ((< lda
                  (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
               (setf info -5))
              ((and
                (< lwork
                   (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nh)))
                (not lquery))
               (setf info -8)))
        (cond
         ((= info 0)
          (setf nb
                  (multiple-value-bind
                      (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                      (ilaenv 1 "ZUNGQR" " " nh nh nh -1)
                    (declare (ignore var-0 var-1 var-2 var-6))
                    (setf nh var-3)
                    (setf nh var-4)
```

```
            (setf nh var-5)
            ret-val))
  (setf lwkopt
          (f2cl-lib:int-mul
           (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nh))
           nb))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwkopt 'f2cl-lib:complex16))))
(cond
 ((/= info 0) (xerbla "ZUNGHR" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond
 ((= n 0)
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce 1 'f2cl-lib:complex16))
  (go end_label)))
(f2cl-lib:fdo (j ihi (f2cl-lib:int-add j (f2cl-lib:int-sub 1)))
              ((> j (f2cl-lib:int-add ilo 1)) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i (f2cl-lib:int-add j (f2cl-lib:int-sub 1))) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                zero)
       label10))
    (f2cl-lib:fdo (i (f2cl-lib:int-add j 1) (f2cl-lib:int-add i 1))
                  ((> i ihi) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                (f2cl-lib:fref a-%data% (i (f2cl-lib:int-sub j 1))
                               ((1 lda) (1 *)) a-%offset%))
       label20))
    (f2cl-lib:fdo (i (f2cl-lib:int-add ihi 1) (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                zero)
       label30))
   label40))
(f2cl-lib:fdo (j 1 (f2cl-lib:int-add j 1))
              ((> j ilo) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                zero)
       label50))
    (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
            one)
```

```
   label60))
 (f2cl-lib:fdo (j (f2cl-lib:int-add ihi 1) (f2cl-lib:int-add j 1))
               ((> j n) nil)
  (tagbody
    (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                  ((> i n) nil)
      (tagbody
        (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *)) a-%offset%)
                zero)
       label70))
    (setf (f2cl-lib:fref a-%data% (j j) ((1 lda) (1 *)) a-%offset%)
            one)
   label80))
 (cond
  ((> nh 0)
   (multiple-value-bind
       (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
       (zungqr nh nh nh
        (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                              ((+ ilo 1) (f2cl-lib:int-add ilo 1))
                              ((1 lda) (1 *)) a-%offset%)
        lda
        (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (ilo)
                              ((1 *)) tau-%offset%)
        work lwork iinfo)
    (declare (ignore var-3 var-5 var-6))
    (when var-0 (setf nh var-0))
    (when var-1 (setf nh var-1))
    (when var-2 (setf nh var-2))
    (when var-4 (setf lda var-4))
    (when var-7 (setf lwork var-7))
    (when var-8 (setf iinfo var-8)))))
 (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
        (coerce lwkopt 'f2cl-lib:complex16))
 (go end_label)
end_label
 (return (values nil nil nil nil lda nil nil lwork info))))))
```

———-

90.243 zungqr

90.243.1 Axiom unit tests

— **zungqr.input** —

```
)set break resume
)sys rm -f zungqr.output
)spool zungqr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.243.2 Axiom help page

— **zungqr.help** —

```
=====================================================================
zungqr examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZUNGQR( M, N, K, A, LDA, TAU, WORK, LWORK, INFO )

       .. Scalar Arguments ..
       INTEGER            INFO, K, LDA, LWORK, M, N
       ..
       .. Array Arguments ..
       COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
       ..


 Purpose:
 =============

 ZUNGQR generates an M-by-N complex matrix Q with orthonormal columns,
 which is defined as the first N columns of a product of K elementary
 reflectors of order M

       Q  =  H(1) H(2) . . . H(k)
```

```
as returned by ZGEQRF.

Arguments:
==========

 [in] M

         M is INTEGER
         The number of rows of the matrix Q. M >= 0.

 [in] N

         N is INTEGER
         The number of columns of the matrix Q. M >= N >= 0.

 [in] K

         K is INTEGER
         The number of elementary reflectors whose product defines the
         matrix Q. N >= K >= 0.

 [in,out] A

         A is COMPLEX*16 array, dimension (LDA,N)
         On entry, the i-th column must contain the vector which
         defines the elementary reflector H(i), for i = 1,2,...,k, as
         returned by ZGEQRF in the first k columns of its array
         argument A.
         On exit, the M-by-N matrix Q.

 [in] LDA

         LDA is INTEGER
         The first dimension of the array A. LDA >= max(1,M).

 [in] TAU

         TAU is COMPLEX*16 array, dimension (K)
         TAU(i) must contain the scalar factor of the elementary
         reflector H(i), as returned by ZGEQRF.

 [out] WORK

         WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
         On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

 [in] LWORK

         LWORK is INTEGER
```

```
         The dimension of the array WORK. LWORK >= max(1,N).
         For optimum performance LWORK >= N*NB, where NB is the
         optimal blocksize.

         If LWORK = -1, then a workspace query is assumed; the routine
         only calculates the optimal size of the WORK array, returns
         this value as the first entry of the WORK array, and no error
         message related to LWORK is issued by XERBLA.

  [out] INFO

         INFO is INTEGER
         = 0:  successful exit
         < 0:  if INFO = -i, the i-th argument has an illegal value
```

———-

90.243.3 fortran code

— **zungqr.f** —

```
*  =====================================================================
      SUBROUTINE ZUNGQR( M, N, K, A, LDA, TAU, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      INTEGER            INFO, K, LDA, LWORK, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      COMPLEX*16         ZERO
      PARAMETER          ( ZERO = ( 0.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LQUERY
      INTEGER            I, IB, IINFO, IWS, J, KI, KK, L, LDWORK,
     $                   LWKOPT, NB, NBMIN, NX
*     ..
```

```
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLARFB, ZLARFT, ZUNG2R
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. External Functions ..
      INTEGER            ILAENV
      EXTERNAL           ILAENV
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NB = ILAENV( 1, 'ZUNGQR', ' ', M, N, K, -1 )
      LWKOPT = MAX( 1, N )*NB
      WORK( 1 ) = LWKOPT
      LQUERY = ( LWORK.EQ.-1 )
      IF( M.LT.0 ) THEN
         INFO = -1
      ELSE IF( N.LT.0 .OR. N.GT.M ) THEN
         INFO = -2
      ELSE IF( K.LT.0 .OR. K.GT.N ) THEN
         INFO = -3
      ELSE IF( LDA.LT.MAX( 1, M ) ) THEN
         INFO = -5
      ELSE IF( LWORK.LT.MAX( 1, N ) .AND. .NOT.LQUERY ) THEN
         INFO = -8
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZUNGQR', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( N.LE.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      NX = 0
      IWS = N
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
*
*        Determine when to cross over from blocked to unblocked code.
```

```
*
         NX = MAX( 0, ILAENV( 3, 'ZUNGQR', ' ', M, N, K, -1 ) )
         IF( NX.LT.K ) THEN
*
*           Determine if workspace is large enough for blocked code.
*
            LDWORK = N
            IWS = LDWORK*NB
            IF( LWORK.LT.IWS ) THEN
*
*              Not enough workspace to use optimal NB:  reduce NB and
*              determine the minimum value of NB.
*
               NB = LWORK / LDWORK
               NBMIN = MAX( 2, ILAENV( 2, 'ZUNGQR', ' ', M, N, K, -1 ) )
            END IF
         END IF
      END IF
*
      IF( NB.GE.NBMIN .AND. NB.LT.K .AND. NX.LT.K ) THEN
*
         Use blocked code after the last block.
         The first kk columns are handled by the block method.
*
         KI = ( ( K-NX-1 ) / NB )*NB
         KK = MIN( K, KI+NB )
*
         Set A(1:kk,kk+1:n) to zero.
*
         DO 20 J = KK + 1, N
            DO 10 I = 1, KK
               A( I, J ) = ZERO
   10       CONTINUE
   20    CONTINUE
      ELSE
         KK = 0
      END IF
*
      Use unblocked code for the last or only block.
*
      IF( KK.LT.N )
     $   CALL ZUNG2R( M-KK, N-KK, K-KK, A( KK+1, KK+1 ), LDA,
     $                TAU( KK+1 ), WORK, IINFO )
*
      IF( KK.GT.0 ) THEN
*
         Use blocked code
*
         DO 50 I = KI + 1, 1, -NB
            IB = MIN( NB, K-I+1 )
```

```
            IF( I+IB.LE.N ) THEN
*
*              Form the triangular factor of the block reflector
*              H = H(i) H(i+1) . . . H(i+ib-1)
*
               CALL ZLARFT( 'Forward', 'Columnwise', M-I+1, IB,
     $                      A( I, I ), LDA, TAU( I ), WORK, LDWORK )
*
*              Apply H to A(i:m,i+ib:n) from the left
*
               CALL ZLARFB( 'Left', 'No transpose', 'Forward',
     $                      'Columnwise', M-I+1, N-I-IB+1, IB,
     $                      A( I, I ), LDA, WORK, LDWORK, A( I, I+IB ),
     $                      LDA, WORK( IB+1 ), LDWORK )
            END IF
*
*           Apply H to rows i:m of current block
*
            CALL ZUNG2R( M-I+1, IB, IB, A( I, I ), LDA, TAU( I ), WORK,
     $                   IINFO )
*
*           Set rows 1:i-1 of current block to zero
*
            DO 40 J = I, I + IB - 1
               DO 30 L = 1, I - 1
                  A( L, J ) = ZERO
   30          CONTINUE
   40       CONTINUE
   50    CONTINUE
      END IF
*
      WORK( 1 ) = IWS
      RETURN
*
*     End of ZUNGQR
*
      END
```

———-

90.243.4 lisp code

— **zungqr.lisp** —

```
(let* ((zero (f2cl-lib:cmplx 0.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) zero)
           (ignorable zero))
```

```
(defun zungqr (m n k a lda tau work lwork info)
  (declare (type (array f2cl-lib:complex16 (*)) work tau a)
           (type (f2cl-lib:integer4) info lwork lda k n m))
  (f2cl-lib:with-multi-array-data
      ((a f2cl-lib:complex16 a-%data% a-%offset%)
       (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%))
    (prog ((i 0) (ib 0) (iinfo 0) (iws 0) (j 0) (ki 0) (kk 0) (l 0)
           (ldwork 0) (lwkopt 0) (nb 0) (nbmin 0) (nx 0) (lquery nil))
      (declare
       (type (f2cl-lib:integer4) i ib iinfo iws j ki kk l ldwork lwkopt nb
        nbmin nx)
       (type f2cl-lib:logical lquery))
      (setf info 0)
      (setf nb
              (multiple-value-bind
                  (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                  (ilaenv 1 "ZUNGQR" " " m n k -1)
                (declare (ignore var-0 var-1 var-2 var-6))
                (setf m var-3)
                (setf n var-4)
                (setf k var-5)
                ret-val))
      (setf lwkopt
              (f2cl-lib:int-mul
               (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)) nb))
      (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
              (coerce lwkopt 'f2cl-lib:complex16))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond ((< m 0) (setf info -1)) ((or (< n 0) (> n m)) (setf info -2))
            ((or (< k 0) (> k n)) (setf info -3))
            ((< lda
                (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
             (setf info -5))
            ((and
              (< lwork
                 (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 n)))
              (not lquery))
             (setf info -8)))
      (cond
       ((/= info 0) (xerbla "ZUNGQR" (f2cl-lib:int-sub info)) (go end_label))
       (lquery (go end_label)))
      (cond
       ((<= n 0)
        (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
                (coerce 1 'f2cl-lib:complex16))
        (go end_label)))
      (setf nbmin 2)
      (setf nx 0)
      (setf iws n)
```

```
(cond
 ((and (> nb 1) (< nb k))
  (setf nx
          (max (the f2cl-lib:integer4 0)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
                        (ilaenv 3 "ZUNGQR" " " m n k -1)
                      (declare (ignore var-0 var-1 var-2 var-6))
                      (setf m var-3)
                      (setf n var-4)
                      (setf k var-5)
                      ret-val))))
  (cond
   ((< nx k) (setf ldwork n) (setf iws (f2cl-lib:int-mul ldwork nb))
    (cond
     ((< lwork iws)
      (setf nb (the f2cl-lib:integer4 (truncate lwork ldwork)))
      (setf nbmin
              (max (the f2cl-lib:integer4 2)
                   (the f2cl-lib:integer4
                        (multiple-value-bind
                            (ret-val var-0 var-1 var-2 var-3 var-4
                             var-5 var-6)
                            (ilaenv 2 "ZUNGQR" " " m n k -1)
                          (declare (ignore var-0 var-1 var-2 var-6))
                          (setf m var-3)
                          (setf n var-4)
                          (setf k var-5)
                          ret-val)))))))))
(cond
 ((and (>= nb nbmin) (< nb k) (< nx k))
  (setf ki (* (the f2cl-lib:integer4 (truncate (- k nx 1) nb)) nb))
  (setf kk
          (min (the f2cl-lib:integer4 k)
               (the f2cl-lib:integer4 (f2cl-lib:int-add ki nb))))
  (f2cl-lib:fdo (j (f2cl-lib:int-add kk 1) (f2cl-lib:int-add j 1))
                ((> j n) nil)
    (tagbody
      (f2cl-lib:fdo (i 1 (f2cl-lib:int-add i 1))
                    ((> i kk) nil)
        (tagbody
          (setf (f2cl-lib:fref a-%data% (i j) ((1 lda) (1 *))
                               a-%offset%)
                zero)
         label10))
     label20)))
 (t (setf kk 0)))
(if (< kk n)
```

```
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
        (zung2r (f2cl-lib:int-sub m kk) (f2cl-lib:int-sub n kk)
         (f2cl-lib:int-sub k kk)
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                               ((+ kk 1) (f2cl-lib:int-add kk 1))
                               ((1 lda) (1 *)) a-%offset%)
         lda
         (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 ((+ kk 1))
                               ((1 *)) tau-%offset%)
         work iinfo)
      (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6))
      (setf lda var-4)
      (setf iinfo var-7)))
(cond
 ((> kk 0)
  (f2cl-lib:fdo (i (f2cl-lib:int-add ki 1)
                 (f2cl-lib:int-add i (f2cl-lib:int-sub nb)))
                ((> i 1) nil)
    (tagbody
      (setf ib
              (min (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
      (cond
       ((<= (f2cl-lib:int-add i ib) n)
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
            (zlarft "Forward" "Columnwise"
             (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda
             (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                                   ((1 *)) tau-%offset%)
             work ldwork)
          (declare (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7))
          (setf lda var-5)
          (setf ldwork var-8))
        (multiple-value-bind
            (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8
             var-9 var-10 var-11 var-12 var-13 var-14)
            (zlarfb "Left" "No transpose" "Forward" "Columnwise"
             (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1)
             (f2cl-lib:int-add (f2cl-lib:int-sub n i ib) 1) ib
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                   ((1 lda) (1 *)) a-%offset%)
             lda work ldwork
             (f2cl-lib:array-slice a-%data% f2cl-lib:complex16
                                   (i (f2cl-lib:int-add i ib))
```

```
                                  ((1 lda) (1 *)) a-%offset%)
                lda
                (f2cl-lib:array-slice work-%data% f2cl-lib:complex16
                                  ((+ ib 1)) ((1 *)) work-%offset%)
                ldwork)
              (declare
               (ignore var-0 var-1 var-2 var-3 var-4 var-5 var-7 var-9
                var-11 var-13))
              (setf ib var-6)
              (setf lda var-8)
              (setf ldwork var-10)
              (setf lda var-12)
              (setf ldwork var-14))))
          (multiple-value-bind
              (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7)
              (zung2r (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1) ib ib
               (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                 ((1 lda) (1 *)) a-%offset%)
               lda
               (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                                 ((1 *)) tau-%offset%)
               work iinfo)
            (declare (ignore var-0 var-1 var-2 var-3 var-5 var-6))
            (setf lda var-4)
            (setf iinfo var-7))
          (f2cl-lib:fdo (j i (f2cl-lib:int-add j 1))
                        ((> j (f2cl-lib:int-add i ib (f2cl-lib:int-sub 1)))
                         nil)
            (tagbody
              (f2cl-lib:fdo (l 1 (f2cl-lib:int-add l 1))
                            ((> l
                                (f2cl-lib:int-add i (f2cl-lib:int-sub 1)))
                             nil)
                (tagbody
                  (setf (f2cl-lib:fref a-%data% (l j) ((1 lda) (1 *))
                                  a-%offset%)
                        zero)
                label30))
             label40))
         label50))))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce iws 'f2cl-lib:complex16))
  (go end_label)
 end_label
  (return (values m n k nil lda nil nil nil info))))))
```

———-

90.244 zunm2r

90.244.1 Axiom unit tests

— zunm2r.input —

```
)set break resume
)sys rm -f zunm2r.output
)spool zunm2r.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.244.2 Axiom help page

— zunm2r.help —

```
=====================================================================
zunm2r examples
=====================================================================

=====================================================================
Man Page Details
=====================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZUNM2R( SIDE, TRANS, M, N, K, A, LDA, TAU, C, LDC,
                          WORK, INFO )

       .. Scalar Arguments ..
       CHARACTER          SIDE, TRANS
       INTEGER            INFO, K, LDA, LDC, M, N
       ..
       .. Array Arguments ..
       COMPLEX*16         A( LDA, * ), C( LDC, * ), TAU( * ), WORK( * )
       ..
```

```
Purpose:
=============

ZUNM2R overwrites the general complex m-by-n matrix C with

      Q * C  if SIDE = 'L' and TRANS = 'N', or

      Q**H* C  if SIDE = 'L' and TRANS = 'C', or

      C * Q  if SIDE = 'R' and TRANS = 'N', or

      C * Q**H if SIDE = 'R' and TRANS = 'C',

where Q is a complex unitary matrix defined as the product of k
elementary reflectors

      Q = H(1) H(2) . . . H(k)

as returned by ZGEQRF. Q is of order m if SIDE = 'L' and of order n
if SIDE = 'R'.

Arguments:
==========

 [in] SIDE

         SIDE is CHARACTER*1
         = 'L': apply Q or Q**H from the Left
         = 'R': apply Q or Q**H from the Right

 [in] TRANS

         TRANS is CHARACTER*1
         = 'N': apply Q  (No transpose)
         = 'C': apply Q**H (Conjugate transpose)

 [in] M

         M is INTEGER
         The number of rows of the matrix C. M >= 0.

 [in] N

         N is INTEGER
         The number of columns of the matrix C. N >= 0.

 [in] K
```

```
        K is INTEGER
        The number of elementary reflectors whose product defines
        the matrix Q.
        If SIDE = 'L', M >= K >= 0;
        if SIDE = 'R', N >= K >= 0.

[in] A

        A is COMPLEX*16 array, dimension (LDA,K)
        The i-th column must contain the vector which defines the
        elementary reflector H(i), for i = 1,2,...,k, as returned by
        ZGEQRF in the first k columns of its array argument A.
        A is modified by the routine but restored on exit.

[in] LDA

        LDA is INTEGER
        The leading dimension of the array A.
        If SIDE = 'L', LDA >= max(1,M);
        if SIDE = 'R', LDA >= max(1,N).

[in] TAU

        TAU is COMPLEX*16 array, dimension (K)
        TAU(i) must contain the scalar factor of the elementary
        reflector H(i), as returned by ZGEQRF.

[in,out] C

        C is COMPLEX*16 array, dimension (LDC,N)
        On entry, the m-by-n matrix C.
        On exit, C is overwritten by Q*C or Q**H*C or C*Q**H or C*Q.

[in] LDC

        LDC is INTEGER
        The leading dimension of the array C. LDC >= max(1,M).

[out] WORK

        WORK is COMPLEX*16 array, dimension
                                 (N) if SIDE = 'L',
                                 (M) if SIDE = 'R'

[out] INFO

        INFO is INTEGER
        = 0: successful exit
        < 0: if INFO = -i, the i-th argument had an illegal value
```

———-

90.244.3 fortran code

— **zunm2r.f** —

```
*  ===================================================================
      SUBROUTINE ZUNM2R( SIDE, TRANS, M, N, K, A, LDA, TAU, C, LDC,
     $                   WORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          SIDE, TRANS
      INTEGER            INFO, K, LDA, LDC, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), C( LDC, * ), TAU( * ), WORK( * )
*     ..
*
*  ===================================================================
*
*     .. Parameters ..
      COMPLEX*16         ONE
      PARAMETER          ( ONE = ( 1.0D+0, 0.0D+0 ) )
*     ..
*     .. Local Scalars ..
      LOGICAL            LEFT, NOTRAN
      INTEGER            I, I1, I2, I3, IC, JC, MI, NI, NQ
      COMPLEX*16         AII, TAUI
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      EXTERNAL           LSAME
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLARF
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          DCONJG, MAX
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
```

```
      INFO = 0
      LEFT = LSAME( SIDE, 'L' )
      NOTRAN = LSAME( TRANS, 'N' )
*
*     NQ is the order of Q
*
      IF( LEFT ) THEN
         NQ = M
      ELSE
         NQ = N
      END IF
      IF( .NOT.LEFT .AND. .NOT.LSAME( SIDE, 'R' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( M.LT.0 ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( K.LT.0 .OR. K.GT.NQ ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, NQ ) ) THEN
         INFO = -7
      ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
         INFO = -10
      END IF
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZUNM2R', -INFO )
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 .OR. K.EQ.0 )
     $   RETURN
*
      IF( ( LEFT .AND. .NOT.NOTRAN .OR. .NOT.LEFT .AND. NOTRAN ) ) THEN
         I1 = 1
         I2 = K
         I3 = 1
      ELSE
         I1 = K
         I2 = 1
         I3 = -1
      END IF
*
      IF( LEFT ) THEN
         NI = N
         JC = 1
      ELSE
```

```
         MI = M
         IC = 1
      END IF
*
      DO 10 I = I1, I2, I3
         IF( LEFT ) THEN
*
*           H(i) or H(i)**H is applied to C(i:m,1:n)
*
            MI = M - I + 1
            IC = I
         ELSE
*
*           H(i) or H(i)**H is applied to C(1:m,i:n)
*
            NI = N - I + 1
            JC = I
         END IF
*
*        Apply H(i) or H(i)**H
*
         IF( NOTRAN ) THEN
            TAUI = TAU( I )
         ELSE
            TAUI = DCONJG( TAU( I ) )
         END IF
         AII = A( I, I )
         A( I, I ) = ONE
         CALL ZLARF( SIDE, MI, NI, A( I, I ), 1, TAUI, C( IC, JC ), LDC,
     $               WORK )
         A( I, I ) = AII
   10 CONTINUE
      RETURN
*
*     End of ZUNM2R
*
      END
```

———-

90.244.4 lisp code

— **zunm2r.lisp** —

```
(let* ((one (f2cl-lib:cmplx 1.0d0 0.0d0)))
  (declare (type (f2cl-lib:complex16) one)
           (ignorable one))
```

```
(defun zunm2r (side trans m n k a lda tau c ldc work info)
  (declare (type (array f2cl-lib:complex16 (*)) work c tau a)
           (type (f2cl-lib:integer4) info ldc lda k n m)
           (type (string *) trans side))
  (f2cl-lib:with-multi-array-data
      ((side character side-%data% side-%offset%)
       (trans character trans-%data% trans-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
       (c f2cl-lib:complex16 c-%data% c-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%))
    (prog ((aii #C(0.0d0 0.0d0)) (taui #C(0.0d0 0.0d0)) (i 0) (i1 0) (i2 0)
           (i3 0) (ic 0) (jc 0) (mi 0) (ni 0) (nq 0) (left nil) (notran nil))
      (declare (type (f2cl-lib:complex16) aii taui)
               (type (f2cl-lib:integer4) i i1 i2 i3 ic jc mi ni nq)
               (type f2cl-lib:logical left notran))
      (setf info 0)
      (setf left
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame side "L")
                (declare (ignore var-1))
                (when var-0 (setf side var-0))
                ret-val))
      (setf notran
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "N")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val))
      (cond (left (setf nq m)) (t (setf nq n)))
      (cond
       ((and (not left)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame side "R")
                (declare (ignore var-1))
                (when var-0 (setf side var-0))
                ret-val)))
        (setf info -1))
       ((and (not notran)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame trans "C")
                (declare (ignore var-1))
                (when var-0 (setf trans var-0))
                ret-val)))
        (setf info -2))
       ((< m 0) (setf info -3)) ((< n 0) (setf info -4))
       ((or (< k 0) (> k nq)) (setf info -5))
       ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nq)))
```

```
  (setf info -7))
 ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -10)))
(cond
 ((/= info 0) (xerbla "ZUNM2R" (f2cl-lib:int-sub info))
  (go end_label)))
(if (or (= m 0) (= n 0) (= k 0))
    (go end_label))
(cond
 ((or (and left (not notran)) (and (not left) notran)) (setf i1 1)
  (setf i2 k) (setf i3 1))
 (t (setf i1 k) (setf i2 1) (setf i3 -1)))
(cond (left (setf ni n) (setf jc 1)) (t (setf mi m) (setf ic 1)))
(f2cl-lib:fdo (i i1 (f2cl-lib:int-add i i3))
              ((> i i2) nil)
  (tagbody
    (cond
     (left (setf mi (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
      (setf ic i))
     (t (setf ni (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
      (setf jc i)))
    (cond
     (notran
      (setf taui (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%)))
     (t
      (setf taui
              (coerce
               (f2cl-lib:dconjg
                (f2cl-lib:fref tau-%data% (i) ((1 *)) tau-%offset%))
               'f2cl-lib:complex16))))
    (setf aii
            (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%))
    (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
            one)
    (multiple-value-bind
        (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
        (zlarf side mi ni
         (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                               ((1 lda) (1 *)) a-%offset%)
         1 taui
         (f2cl-lib:array-slice c-%data% f2cl-lib:complex16 (ic jc)
                               ((1 ldc) (1 *)) c-%offset%)
         ldc work)
      (declare (ignore var-1 var-2 var-3 var-4 var-5 var-6 var-8))
      (setf side var-0)
      (setf ldc var-7))
    (setf (f2cl-lib:fref a-%data% (i i) ((1 lda) (1 *)) a-%offset%)
            aii)
   label10))
(go end_label)
```

```
      end_label
       (return
        (values side trans nil nil nil nil nil nil nil ldc nil info))))))
```

———-

90.245 zunmhr

90.245.1 Axiom unit tests

— **zunmhr.input** —

```
)set break resume
)sys rm -f zunmhr.output
)spool zunmhr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.245.2 Axiom help page

— **zunmhr.help** —

```
======================================================================
zunmhr examples
======================================================================

======================================================================
Man Page Details
======================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZUNMHR( SIDE, TRANS, M, N, ILO, IHI, A, LDA, TAU, C,
```

```
                         LDC, WORK, LWORK, INFO )

      .. Scalar Arguments ..
      CHARACTER          SIDE, TRANS
      INTEGER            IHI, ILO, INFO, LDA, LDC, LWORK, M, N
      ..
      .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), C( LDC, * ), TAU( * ), WORK( * )
      ..


Purpose:
=============

ZUNMHR overwrites the general complex M-by-N matrix C with

                SIDE = 'L'     SIDE = 'R'
TRANS = 'N':      Q * C          C * Q
TRANS = 'C':      Q**H * C       C * Q**H

where Q is a complex unitary matrix of order nq, with nq = m if
SIDE = 'L' and nq = n if SIDE = 'R'. Q is defined as the product of
IHI-ILO elementary reflectors, as returned by ZGEHRD:

Q = H(ilo) H(ilo+1) . . . H(ihi-1).

Arguments:
==========

 [in] SIDE

         SIDE is CHARACTER*1
         = 'L': apply Q or Q**H from the Left;
         = 'R': apply Q or Q**H from the Right.

 [in] TRANS

         TRANS is CHARACTER*1
         = 'N': apply Q  (No transpose)
         = 'C': apply Q**H (Conjugate transpose)

 [in] M

         M is INTEGER
         The number of rows of the matrix C. M >= 0.

 [in] N

         N is INTEGER
         The number of columns of the matrix C. N >= 0.
```

```
[in] ILO

        ILO is INTEGER

[in] IHI

        IHI is INTEGER

        ILO and IHI must have the same values as in the previous call
        of ZGEHRD. Q is equal to the unit matrix except in the
        submatrix Q(ilo+1:ihi,ilo+1:ihi).
        If SIDE = 'L', then 1 <= ILO <= IHI <= M, if M > 0, and
        ILO = 1 and IHI = 0, if M = 0;
        if SIDE = 'R', then 1 <= ILO <= IHI <= N, if N > 0, and
        ILO = 1 and IHI = 0, if N = 0.

[in] A

        A is COMPLEX*16 array, dimension
                             (LDA,M) if SIDE = 'L'
                             (LDA,N) if SIDE = 'R'
        The vectors which define the elementary reflectors, as
        returned by ZGEHRD.

[in] LDA

        LDA is INTEGER
        The leading dimension of the array A.
        LDA >= max(1,M) if SIDE = 'L'; LDA >= max(1,N) if SIDE = 'R'.

[in] TAU

        TAU is COMPLEX*16 array, dimension
                             (M-1) if SIDE = 'L'
                             (N-1) if SIDE = 'R'
        TAU(i) must contain the scalar factor of the elementary
        reflector H(i), as returned by ZGEHRD.

[in,out] C

        C is COMPLEX*16 array, dimension (LDC,N)
        On entry, the M-by-N matrix C.
        On exit, C is overwritten by Q*C or Q**H*C or C*Q**H or C*Q.

[in] LDC

        LDC is INTEGER
        The leading dimension of the array C. LDC >= max(1,M).
```

```
[out] WORK

        WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
        On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

[in] LWORK

        LWORK is INTEGER
        The dimension of the array WORK.
        If SIDE = 'L', LWORK >= max(1,N);
        if SIDE = 'R', LWORK >= max(1,M).
        For optimum performance LWORK >= N*NB if SIDE = 'L', and
        LWORK >= M*NB if SIDE = 'R', where NB is the optimal
        blocksize.

        If LWORK = -1, then a workspace query is assumed; the routine
        only calculates the optimal size of the WORK array, returns
        this value as the first entry of the WORK array, and no error
        message related to LWORK is issued by XERBLA.

[out] INFO

        INFO is INTEGER
        = 0:  successful exit
        < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

90.245.3 fortran code

— zunmhr.f —

```
*  =====================================================================
      SUBROUTINE ZUNMHR( SIDE, TRANS, M, N, ILO, IHI, A, LDA, TAU, C,
     $                   LDC, WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          SIDE, TRANS
      INTEGER            IHI, ILO, INFO, LDA, LDC, LWORK, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), C( LDC, * ), TAU( * ), WORK( * )
```

```
*     ..
*
*  =====================================================================
*
*     .. Local Scalars ..
      LOGICAL            LEFT, LQUERY
      INTEGER            I1, I2, IINFO, LWKOPT, MI, NB, NH, NI, NQ, NW
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZUNMQR
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      NH = IHI - ILO
      LEFT = LSAME( SIDE, 'L' )
      LQUERY = ( LWORK.EQ.-1 )
*
*     NQ is the order of Q and NW is the minimum dimension of WORK
*
      IF( LEFT ) THEN
         NQ = M
         NW = N
      ELSE
         NQ = N
         NW = M
      END IF
      IF( .NOT.LEFT .AND. .NOT.LSAME( SIDE, 'R' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.LSAME( TRANS, 'N' ) .AND. .NOT.LSAME( TRANS, 'C' ) )
     $          THEN
         INFO = -2
      ELSE IF( M.LT.0 ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( ILO.LT.1 .OR. ILO.GT.MAX( 1, NQ ) ) THEN
         INFO = -5
      ELSE IF( IHI.LT.MIN( ILO, NQ ) .OR. IHI.GT.NQ ) THEN
         INFO = -6
```

```
      ELSE IF( LDA.LT.MAX( 1, NQ ) ) THEN
         INFO = -8
      ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
         INFO = -11
      ELSE IF( LWORK.LT.MAX( 1, NW ) .AND. .NOT.LQUERY ) THEN
         INFO = -13
      END IF
*
      IF( INFO.EQ.0 ) THEN
         IF( LEFT ) THEN
            NB = ILAENV( 1, 'ZUNMQR', SIDE // TRANS, NH, N, NH, -1 )
         ELSE
            NB = ILAENV( 1, 'ZUNMQR', SIDE // TRANS, M, NH, NH, -1 )
         END IF
         LWKOPT = MAX( 1, NW )*NB
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZUNMHR', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 .OR. NH.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      IF( LEFT ) THEN
         MI = NH
         NI = N
         I1 = ILO + 1
         I2 = 1
      ELSE
         MI = M
         NI = NH
         I1 = 1
         I2 = ILO + 1
      END IF
*
      CALL ZUNMQR( SIDE, TRANS, MI, NI, NH, A( ILO+1, ILO ), LDA,
     $             TAU( ILO ), C( I1, I2 ), LDC, WORK, LWORK, IINFO )
*
      WORK( 1 ) = LWKOPT
      RETURN
*
```

```
*     End of ZUNMHR
*
      END
```

———-

90.245.4 lisp code

— **zunmhr.lisp** —

```
(defun zunmhr (side trans m n ilo ihi a lda tau c ldc work lwork info)
  (declare (type (array f2cl-lib:complex16 (*)) work c tau a)
           (type (f2cl-lib:integer4) info lwork ldc lda ihi ilo n m)
           (type (string *) trans side))
  (f2cl-lib:with-multi-array-data
      ((side character side-%data% side-%offset%)
       (trans character trans-%data% trans-%offset%)
       (a f2cl-lib:complex16 a-%data% a-%offset%)
       (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
       (c f2cl-lib:complex16 c-%data% c-%offset%)
       (work f2cl-lib:complex16 work-%data% work-%offset%))
    (prog ((i1 0) (i2 0) (iinfo 0) (lwkopt 0) (mi 0) (nb 0) (nh 0) (ni 0)
           (nq 0) (nw 0) (left nil) (lquery nil))
      (declare (type f2cl-lib:logical lquery left)
               (type (f2cl-lib:integer4) nw nq ni nh nb mi lwkopt iinfo i2 i1))
      (setf info 0)
      (setf nh (f2cl-lib:int-sub ihi ilo))
      (setf left
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame side "L")
                (declare (ignore var-1))
                (when var-0 (setf side var-0))
                ret-val))
      (setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
      (cond (left (setf nq m) (setf nw n)) (t (setf nq n) (setf nw m)))
      (cond
       ((and (not left)
             (not
              (multiple-value-bind (ret-val var-0 var-1)
                  (lsame side "R")
                (declare (ignore var-1))
                (when var-0 (setf side var-0))
                ret-val)))
        (setf info -1))
       ((and
         (not
          (multiple-value-bind (ret-val var-0 var-1)
```

```
          (lsame trans "N")
        (declare (ignore var-1))
        (when var-0 (setf trans var-0))
        ret-val))
    (not
     (multiple-value-bind (ret-val var-0 var-1)
          (lsame trans "C")
        (declare (ignore var-1))
        (when var-0 (setf trans var-0))
        ret-val)))
   (setf info -2))
  ((< m 0) (setf info -3)) ((< n 0) (setf info -4))
  ((or (< ilo 1)
       (> ilo (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nq))))
   (setf info -5))
  ((or
    (< ihi (min (the f2cl-lib:integer4 ilo) (the f2cl-lib:integer4 nq)))
    (> ihi nq))
   (setf info -6))
  ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nq)))
   (setf info -8))
  ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
   (setf info -11))
  ((and
    (< lwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nw)))
    (not lquery))
   (setf info -13)))
 (cond
  ((= info 0)
   (cond
    (left
     (setf nb
             (multiple-value-bind
                 (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                 (ilaenv 1 "ZUNMQR" (f2cl-lib:f2cl-// side trans) nh n nh
                  -1)
               (declare (ignore var-0 var-1 var-2 var-6))
               (setf nh var-3)
               (setf n var-4)
               (setf nh var-5)
               ret-val)))
    (t
     (setf nb
             (multiple-value-bind
                 (ret-val var-0 var-1 var-2 var-3 var-4 var-5 var-6)
                 (ilaenv 1 "ZUNMQR" (f2cl-lib:f2cl-// side trans) m nh nh
                  -1)
               (declare (ignore var-0 var-1 var-2 var-6))
               (setf m var-3)
               (setf nh var-4)
```

```
                (setf nh var-5)
                ret-val))))
    (setf lwkopt
            (f2cl-lib:int-mul
             (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nw))
             nb))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce lwkopt 'f2cl-lib:complex16))))
  (cond
   ((/= info 0) (xerbla "ZUNMHR" (f2cl-lib:int-sub info)) (go end_label))
   (lquery (go end_label)))
  (cond
   ((or (= m 0) (= n 0) (= nh 0))
    (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
            (coerce 1 'f2cl-lib:complex16))
    (go end_label)))
  (cond
   (left (setf mi nh) (setf ni n) (setf i1 (f2cl-lib:int-add ilo 1))
    (setf i2 1))
   (t (setf mi m) (setf ni nh) (setf i1 1)
    (setf i2 (f2cl-lib:int-add ilo 1))))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9 var-10
       var-11 var-12)
      (zunmqr side trans mi ni nh
       (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 ((+ ilo 1) ilo)
                             ((1 lda) (1 *)) a-%offset%)
       lda
       (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (ilo) ((1 *))
                             tau-%offset%)
       (f2cl-lib:array-slice c-%data% f2cl-lib:complex16 (i1 i2)
                             ((1 ldc) (1 *)) c-%offset%)
       ldc work lwork iinfo)
    (declare (ignore var-5 var-7 var-8 var-10))
    (when var-0 (setf side var-0))
    (when var-1 (setf trans var-1))
    (when var-2 (setf mi var-2))
    (when var-3 (setf ni var-3))
    (when var-4 (setf nh var-4))
    (when var-6 (setf lda var-6))
    (when var-9 (setf ldc var-9))
    (when var-11 (setf lwork var-11))
    (when var-12 (setf iinfo var-12)))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwkopt 'f2cl-lib:complex16))
  (go end_label)
 end_label
  (return
   (values side trans m n nil nil nil lda nil nil ldc nil lwork info)))))
```

———-

90.246 zunmqr

90.246.1 Axiom unit tests

— **zunmqr.input** —

```
)set break resume
)sys rm -f zunmqr.output
)spool zunmqr.output
)set message test on
)set message auto off
)clear all

)spool
)lisp (bye)
```

———-

90.246.2 Axiom help page

— **zunmqr.help** —

```
======================================================================
zunmqr examples
======================================================================

======================================================================
Man Page Details
======================================================================

 Online html documentation available at
            http://www.netlib.org/lapack/explore-html/

 Definition:
 ===========

       SUBROUTINE ZUNMQR( SIDE, TRANS, M, N, K, A, LDA, TAU, C, LDC,
                          WORK, LWORK, INFO )

       .. Scalar Arguments ..
       CHARACTER          SIDE, TRANS
       INTEGER            INFO, K, LDA, LDC, LWORK, M, N
```

```
      ..
      .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), C( LDC, * ), TAU( * ), WORK( * )
      ..


Purpose:
=============

ZUNMQR overwrites the general complex M-by-N matrix C with

                SIDE = 'L'     SIDE = 'R'
TRANS = 'N':      Q * C          C * Q
TRANS = 'C':      Q**H * C       C * Q**H

where Q is a complex unitary matrix defined as the product of k
elementary reflectors

      Q = H(1) H(2) . . . H(k)

as returned by ZGEQRF. Q is of order M if SIDE = 'L' and of order N
if SIDE = 'R'.

Arguments:
==========

 [in] SIDE

         SIDE is CHARACTER*1
         = 'L': apply Q or Q**H from the Left;
         = 'R': apply Q or Q**H from the Right.

 [in] TRANS

         TRANS is CHARACTER*1
         = 'N':  No transpose, apply Q;
         = 'C':  Conjugate transpose, apply Q**H.

 [in] M

         M is INTEGER
         The number of rows of the matrix C. M >= 0.

 [in] N

         N is INTEGER
         The number of columns of the matrix C. N >= 0.

 [in] K
```

```
        K is INTEGER
        The number of elementary reflectors whose product defines
        the matrix Q.
        If SIDE = 'L', M >= K >= 0;
        if SIDE = 'R', N >= K >= 0.

[in] A

        A is COMPLEX*16 array, dimension (LDA,K)
        The i-th column must contain the vector which defines the
        elementary reflector H(i), for i = 1,2,...,k, as returned by
        ZGEQRF in the first k columns of its array argument A.
        A is modified by the routine but restored on exit.

[in] LDA

        LDA is INTEGER
        The leading dimension of the array A.
        If SIDE = 'L', LDA >= max(1,M);
        if SIDE = 'R', LDA >= max(1,N).

[in] TAU

        TAU is COMPLEX*16 array, dimension (K)
        TAU(i) must contain the scalar factor of the elementary
        reflector H(i), as returned by ZGEQRF.

[in,out] C

        C is COMPLEX*16 array, dimension (LDC,N)
        On entry, the M-by-N matrix C.
        On exit, C is overwritten by Q*C or Q**H*C or C*Q**H or C*Q.

[in] LDC

        LDC is INTEGER
        The leading dimension of the array C. LDC >= max(1,M).

[out] WORK

        WORK is COMPLEX*16 array, dimension (MAX(1,LWORK))
        On exit, if INFO = 0, WORK(1) returns the optimal LWORK.

[in] LWORK

        LWORK is INTEGER
        The dimension of the array WORK.
        If SIDE = 'L', LWORK >= max(1,N);
        if SIDE = 'R', LWORK >= max(1,M).
        For optimum performance LWORK >= N*NB if SIDE = 'L', and
```

```
         LWORK >= M*NB if SIDE = 'R', where NB is the optimal
         blocksize.

         If LWORK = -1, then a workspace query is assumed; the routine
         only calculates the optimal size of the WORK array, returns
         this value as the first entry of the WORK array, and no error
         message related to LWORK is issued by XERBLA.

  [out] INFO

         INFO is INTEGER
         = 0:  successful exit
         < 0:  if INFO = -i, the i-th argument had an illegal value
```

———-

90.246.3 fortran code

— **zunmqr.f** —

```
*  =====================================================================
      SUBROUTINE ZUNMQR( SIDE, TRANS, M, N, K, A, LDA, TAU, C, LDC,
     $                   WORK, LWORK, INFO )
*
*  -- LAPACK computational routine (version 3.4.0) --
*  -- LAPACK is a software package provided by Univ. of Tennessee,    --
*  -- Univ. of California Berkeley, Univ. of Colorado Denver and NAG Ltd..--
*     November 2011
*
*     .. Scalar Arguments ..
      CHARACTER          SIDE, TRANS
      INTEGER            INFO, K, LDA, LDC, LWORK, M, N
*     ..
*     .. Array Arguments ..
      COMPLEX*16         A( LDA, * ), C( LDC, * ), TAU( * ), WORK( * )
*     ..
*
*  =====================================================================
*
*     .. Parameters ..
      INTEGER            NBMAX, LDT
      PARAMETER          ( NBMAX = 64, LDT = NBMAX+1 )
*     ..
*     .. Local Scalars ..
      LOGICAL            LEFT, LQUERY, NOTRAN
      INTEGER            I, I1, I2, I3, IB, IC, IINFO, IWS, JC, LDWORK,
     $                   LWKOPT, MI, NB, NBMIN, NI, NQ, NW
```

```
*     ..
*     .. Local Arrays ..
      COMPLEX*16         T( LDT, NBMAX )
*     ..
*     .. External Functions ..
      LOGICAL            LSAME
      INTEGER            ILAENV
      EXTERNAL           LSAME, ILAENV
*     ..
*     .. External Subroutines ..
      EXTERNAL           XERBLA, ZLARFB, ZLARFT, ZUNM2R
*     ..
*     .. Intrinsic Functions ..
      INTRINSIC          MAX, MIN
*     ..
*     .. Executable Statements ..
*
*     Test the input arguments
*
      INFO = 0
      LEFT = LSAME( SIDE, 'L' )
      NOTRAN = LSAME( TRANS, 'N' )
      LQUERY = ( LWORK.EQ.-1 )
*
*     NQ is the order of Q and NW is the minimum dimension of WORK
*
      IF( LEFT ) THEN
         NQ = M
         NW = N
      ELSE
         NQ = N
         NW = M
      END IF
      IF( .NOT.LEFT .AND. .NOT.LSAME( SIDE, 'R' ) ) THEN
         INFO = -1
      ELSE IF( .NOT.NOTRAN .AND. .NOT.LSAME( TRANS, 'C' ) ) THEN
         INFO = -2
      ELSE IF( M.LT.0 ) THEN
         INFO = -3
      ELSE IF( N.LT.0 ) THEN
         INFO = -4
      ELSE IF( K.LT.0 .OR. K.GT.NQ ) THEN
         INFO = -5
      ELSE IF( LDA.LT.MAX( 1, NQ ) ) THEN
         INFO = -7
      ELSE IF( LDC.LT.MAX( 1, M ) ) THEN
         INFO = -10
      ELSE IF( LWORK.LT.MAX( 1, NW ) .AND. .NOT.LQUERY ) THEN
         INFO = -12
      END IF
```

```
*
      IF( INFO.EQ.0 ) THEN
*
*        Determine the block size.  NB may be at most NBMAX, where NBMAX
*        is used to define the local array T.
*
         NB = MIN( NBMAX, ILAENV( 1, 'ZUNMQR', SIDE // TRANS, M, N, K,
     $        -1 ) )
         LWKOPT = MAX( 1, NW )*NB
         WORK( 1 ) = LWKOPT
      END IF
*
      IF( INFO.NE.0 ) THEN
         CALL XERBLA( 'ZUNMQR', -INFO )
         RETURN
      ELSE IF( LQUERY ) THEN
         RETURN
      END IF
*
*     Quick return if possible
*
      IF( M.EQ.0 .OR. N.EQ.0 .OR. K.EQ.0 ) THEN
         WORK( 1 ) = 1
         RETURN
      END IF
*
      NBMIN = 2
      LDWORK = NW
      IF( NB.GT.1 .AND. NB.LT.K ) THEN
         IWS = NW*NB
         IF( LWORK.LT.IWS ) THEN
            NB = LWORK / LDWORK
            NBMIN = MAX( 2, ILAENV( 2, 'ZUNMQR', SIDE // TRANS, M, N, K,
     $              -1 ) )
         END IF
      ELSE
         IWS = NW
      END IF
*
      IF( NB.LT.NBMIN .OR. NB.GE.K ) THEN
*
*        Use unblocked code
*
         CALL ZUNM2R( SIDE, TRANS, M, N, K, A, LDA, TAU, C, LDC, WORK,
     $               IINFO )
      ELSE
*
*        Use blocked code
*
         IF( ( LEFT .AND. .NOT.NOTRAN ) .OR.
```

```
     $       ( .NOT.LEFT .AND. NOTRAN ) ) THEN
            I1 = 1
            I2 = K
            I3 = NB
         ELSE
            I1 = ( ( K-1 ) / NB )*NB + 1
            I2 = 1
            I3 = -NB
         END IF
*
         IF( LEFT ) THEN
            NI = N
            JC = 1
         ELSE
            MI = M
            IC = 1
         END IF
*
         DO 10 I = I1, I2, I3
            IB = MIN( NB, K-I+1 )
*
*           Form the triangular factor of the block reflector
*           H = H(i) H(i+1) . . . H(i+ib-1)
*
            CALL ZLARFT( 'Forward', 'Columnwise', NQ-I+1, IB, A( I, I ),
     $                   LDA, TAU( I ), T, LDT )
            IF( LEFT ) THEN
*
*              H or H**H is applied to C(i:m,1:n)
*
               MI = M - I + 1
               IC = I
            ELSE
*
*              H or H**H is applied to C(1:m,i:n)
*
               NI = N - I + 1
               JC = I
            END IF
*
*           Apply H or H**H
*
            CALL ZLARFB( SIDE, TRANS, 'Forward', 'Columnwise', MI, NI,
     $                   IB, A( I, I ), LDA, T, LDT, C( IC, JC ), LDC,
     $                   WORK, LDWORK )
   10    CONTINUE
      END IF
      WORK( 1 ) = LWKOPT
      RETURN
*
```

```
*     End of ZUNMQR
*
      END
```

———-

```
Warning:  Types of argument 0 in call to ZUNM2R do not match.
  Declared type: (SIMPLE-ARRAY CHARACTER (*))
  Argument type: (STRING 1)
Warning:  Types of argument 1 in call to ZUNM2R do not match.
  Declared type: (SIMPLE-ARRAY CHARACTER (*))
  Argument type: (STRING 1)
Warning:  Types of argument 0 in call to ZLARFB do not match.
  Declared type: (SIMPLE-ARRAY CHARACTER (*))
  Argument type: (STRING 1)
Warning:  Types of argument 1 in call to ZLARFB do not match.
  Declared type: (SIMPLE-ARRAY CHARACTER (*))
  Argument type: (STRING 1)
```

90.246.4 lisp code

— **zunmqr.lisp** —

```
(let* ((nbmax 64) (ldt (+ nbmax 1)) (tsize (* ldt nbmax)))
  (declare (type (f2cl-lib:integer4 64 64) nbmax)
           (type (f2cl-lib:integer4) ldt)
           (type (f2cl-lib:integer4) tsize)
           (ignorable nbmax ldt tsize))
  (defun zunmqr (side trans m n k a lda tau c ldc work lwork info)
    (declare (type (array f2cl-lib:complex16 (*)) work c tau a)
             (type (f2cl-lib:integer4) info lwork ldc lda k n m)
             (type (string *) trans side))
    (f2cl-lib:with-multi-array-data
        ((side character side-%data% side-%offset%)
         (trans character trans-%data% trans-%offset%)
         (a f2cl-lib:complex16 a-%data% a-%offset%)
         (tau f2cl-lib:complex16 tau-%data% tau-%offset%)
         (c f2cl-lib:complex16 c-%data% c-%offset%)
         (work f2cl-lib:complex16 work-%data% work-%offset%))
      (prog ((i 0) (i1 0) (i2 0) (i3 0) (ib 0) (ic 0) (iinfo 0) (iwt 0) (jc 0)
             (ldwork 0) (lwkopt 0) (mi 0) (nb 0) (nbmin 0) (ni 0) (nq 0) (nw 0)
             (left nil) (lquery nil) (notran nil))
        (declare
         (type (f2cl-lib:integer4) i i1 i2 i3 ib ic iinfo iwt jc ldwork lwkopt
          mi nb nbmin ni nq nw)
         (type f2cl-lib:logical left lquery notran))
```

```
(setf info 0)
(setf left
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "L")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val))
(setf notran
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "N")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val))
(setf lquery (coerce (= lwork -1) 'f2cl-lib:logical))
(cond (left (setf nq m) (setf nw n)) (t (setf nq n) (setf nw m)))
(cond
 ((and (not left)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame side "R")
          (declare (ignore var-1))
          (when var-0 (setf side var-0))
          ret-val)))
  (setf info -1))
 ((and (not notran)
       (not
        (multiple-value-bind (ret-val var-0 var-1)
            (lsame trans "C")
          (declare (ignore var-1))
          (when var-0 (setf trans var-0))
          ret-val)))
  (setf info -2))
 ((< m 0) (setf info -3)) ((< n 0) (setf info -4))
 ((or (< k 0) (> k nq)) (setf info -5))
 ((< lda (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nq)))
  (setf info -7))
 ((< ldc (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 m)))
  (setf info -10))
 ((and
   (< lwork (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nw)))
   (not lquery))
  (setf info -12)))
(cond
 ((= info 0)
  (setf nb
          (min (the f2cl-lib:integer4 nbmax)
               (the f2cl-lib:integer4
                    (multiple-value-bind
                        (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                         var-6)
```

```
                          (ilaenv 1 "ZUNMQR"
                           (f2cl-lib:f2cl-// side trans) m n k -1)
                        (declare (ignore var-0 var-1 var-2 var-6))
                        (setf m var-3)
                        (setf n var-4)
                        (setf k var-5)
                        ret-val))))
  (setf lwkopt
          (f2cl-lib:int-add
           (f2cl-lib:int-mul
            (max (the f2cl-lib:integer4 1) (the f2cl-lib:integer4 nw))
            nb)
           tsize))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce lwkopt 'f2cl-lib:complex16))))
(cond
 ((/= info 0) (xerbla "ZUNMQR" (f2cl-lib:int-sub info)) (go end_label))
 (lquery (go end_label)))
(cond
 ((or (= m 0) (= n 0) (= k 0))
  (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
          (coerce 1 'f2cl-lib:complex16))
  (go end_label)))
(setf nbmin 2)
(setf ldwork nw)
(cond
 ((and (> nb 1) (< nb k))
  (cond
   ((< lwork (f2cl-lib:int-add (f2cl-lib:int-mul nw nb) tsize))
    (setf nb (the f2cl-lib:integer4 (truncate (- lwork tsize) ldwork)))
    (setf nbmin
            (max (the f2cl-lib:integer4 2)
                 (the f2cl-lib:integer4
                      (multiple-value-bind
                          (ret-val var-0 var-1 var-2 var-3 var-4 var-5
                           var-6)
                          (ilaenv 2 "ZUNMQR"
                           (f2cl-lib:f2cl-// side trans) m n k -1)
                        (declare (ignore var-0 var-1 var-2 var-6))
                        (setf m var-3)
                        (setf n var-4)
                        (setf k var-5)
                        ret-val))))))))
(cond
 ((or (< nb nbmin) (>= nb k))
  (multiple-value-bind
      (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
       var-10 var-11)
      (zunm2r side trans m n k a lda tau c ldc work iinfo)
    (declare (ignore var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-10))
```

```
    (setf side var-0)
    (setf trans var-1)
    (setf ldc var-9)
    (setf iinfo var-11)))
 (t (setf iwt (f2cl-lib:int-add 1 (f2cl-lib:int-mul nw nb)))
  (cond
   ((or (and left (not notran)) (and (not left) notran)) (setf i1 1)
    (setf i2 k) (setf i3 nb))
   (t
    (setf i1
            (+ (* (the f2cl-lib:integer4 (truncate (- k 1) nb)) nb) 1))
    (setf i2 1) (setf i3 (f2cl-lib:int-sub nb))))
  (cond (left (setf ni n) (setf jc 1)) (t (setf mi m) (setf ic 1)))
  (f2cl-lib:fdo (i i1 (f2cl-lib:int-add i i3))
                ((> i i2) nil)
    (tagbody
      (setf ib
              (min (the f2cl-lib:integer4 nb)
                   (the f2cl-lib:integer4
                        (f2cl-lib:int-add (f2cl-lib:int-sub k i) 1))))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8)
          (zlarft "Forward" "Columnwise"
           (f2cl-lib:int-add (f2cl-lib:int-sub nq i) 1) ib
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice tau-%data% f2cl-lib:complex16 (i)
                                 ((1 *)) tau-%offset%)
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwt)
                                 ((1 *)) work-%offset%)
           ldt)
        (declare (ignore var-0 var-1 var-2 var-3 var-4 var-6 var-7))
        (setf lda var-5)
        (setf ldt var-8))
      (cond
       (left (setf mi (f2cl-lib:int-add (f2cl-lib:int-sub m i) 1))
        (setf ic i))
       (t (setf ni (f2cl-lib:int-add (f2cl-lib:int-sub n i) 1))
        (setf jc i)))
      (multiple-value-bind
          (var-0 var-1 var-2 var-3 var-4 var-5 var-6 var-7 var-8 var-9
           var-10 var-11 var-12 var-13 var-14)
          (zlarfb side trans "Forward" "Columnwise" mi ni ib
           (f2cl-lib:array-slice a-%data% f2cl-lib:complex16 (i i)
                                 ((1 lda) (1 *)) a-%offset%)
           lda
           (f2cl-lib:array-slice work-%data% f2cl-lib:complex16 (iwt)
                                 ((1 *)) work-%offset%)
           ldt
```

```
              (f2cl-lib:array-slice c-%data% f2cl-lib:complex16 (ic jc)
                                    ((1 ldc) (1 *)) c-%offset%)
              ldc work ldwork)
           (declare (ignore var-2 var-3 var-7 var-9 var-11 var-13))
           (setf side var-0)
           (setf trans var-1)
           (setf mi var-4)
           (setf ni var-5)
           (setf ib var-6)
           (setf lda var-8)
           (setf ldt var-10)
           (setf ldc var-12)
           (setf ldwork var-14))
      label10))))
 (setf (f2cl-lib:fref work-%data% (1) ((1 *)) work-%offset%)
         (coerce lwkopt 'f2cl-lib:complex16))
 (go end_label)
end_label
 (return (values side trans m n k nil lda nil nil ldc nil nil info))))))
```

———-

Chapter 91

LAPACK tests

```
;;;
;;; Simple tests for selected LAPACK routines.
;;;
;;; $Id$
;;;

(in-package "LAPACK")

;; Convert the eigenvalues returned by DGEEV into an array
(defun make-eigval (wr wi)
  (let ((e-val (make-array (length wr))))
    (map-into e-val #'(lambda (r i)
       ;; Do we really want to do this?  Should we
       ;; just make all of the eigenvalues complex?
       (if (zerop i)
   r
   (complex r i)))
      wr wi)
    e-val))

;; Convert the eigenvalues returned by DGEEV into a more typical
;; matrix form.
(defun make-eigvec (n vr wi)
  (let ((evec (make-array (list n n))))
    (do ((col 0 (incf col))
 (posn 0))
((>= col n))
      (cond ((zerop (aref wi col))
     (dotimes (row n)
       (setf (aref evec row col) (aref vr posn))
       (incf posn)))
    (t
```

```
     (dotimes (row n)
       (let* ((next-posn (+ posn n))
      (val+ (complex (aref vr posn) (aref vr next-posn)))
      (val- (conjugate val+)))
 (setf (aref evec row col) val+)
 (setf (aref evec row (1+ col)) val-)
 (incf posn)))
     ;; Skip over the next column, which we've already used
     (incf col)
     (incf posn n))))
    evec))

;; Expected results from
;;   http://www.nag.co.uk/lapack-ex/examples/results/dgeev-ex.r
;;
;; DGEEV Example Program Results
;;
;;  Eigenvalue( 1) =  7.9948E-01
;;
;;  Eigenvector( 1)
;;  -6.5509E-01
;;  -5.2363E-01
;;   5.3622E-01
;;  -9.5607E-02
;;
;;  Eigenvalue( 2) = (-9.9412E-02, 4.0079E-01)
;;
;;  Eigenvector( 2)
;;  (-1.9330E-01, 2.5463E-01)
;;  ( 2.5186E-01,-5.2240E-01)
;;  ( 9.7182E-02,-3.0838E-01)
;;  ( 6.7595E-01, 0.0000E+00)
;;
;;  Eigenvalue( 3) = (-9.9412E-02,-4.0079E-01)
;;
;;  Eigenvector( 3)
;;  (-1.9330E-01,-2.5463E-01)
;;  ( 2.5186E-01, 5.2240E-01)
;;  ( 9.7182E-02, 3.0838E-01)
;;  ( 6.7595E-01,-0.0000E+00)
;;
;;  Eigenvalue( 4) = -1.0066E-01
;;
;;  Eigenvector( 4)
;;   1.2533E-01
;;   3.3202E-01
;;   5.9384E-01
;;   7.2209E-01
;;
(defun print-dgeev-results (e-val e-vec)
```

```
  (format t "~2%DGEEV Example Program Results~%")
  (let ((n (length e-val)))
    (dotimes (k n)
      (format t "Eigenvalue(~D) = ~A~%" k (aref e-val k))
      (format t "~%Eigenvector(~D)~%" k)
      (dotimes (row n)
(format t "~A~%" (aref e-vec row k)))
      (terpri))))

(defun check-eigen-val-vec (n e-val e-vec true-val true-vec &key (tol 1d-14))
  (flet ((relerr-ok (est true)
   (let* ((re (/ (abs (- est true))
 (abs true)))
  (ok (<= re tol)))
     ;; Return NIL if it's ok.  Otherwise return a list to
     ;; indicate what failed.
     (unless ok
       (format t "est  = ~S~%true = ~S~%  rel  = ~S~%"
       est true re)
       (list est true re)))))
    (or (relerr-ok (aref e-val n) true-val)
(dotimes (k n t)
  (let ((res (relerr-ok (aref e-vec k n) (aref true-vec k))))
    (when res
      (return res)))))))

;; DGEEV example based on the example from
;; http://www.nag.co.uk/lapack-ex/node87.html
(defun test-dgeev ()
  ;; The matrix is
  ;;
  ;;  0.35  0.45 -0.14 -0.17
  ;;  0.09  0.07 -0.54  0.35
  ;; -0.44 -0.33 -0.03  0.17
  ;;  0.25 -0.32 -0.13  0.11
  ;;
  ;; Recall that Fortran arrays are column-major order!
  (let* ((n 4)
 (a-mat (make-array (* n n) :element-type 'double-float
    :initial-contents '(0.35d0 0.09d0 -0.44d0 0.25d0
0.45d0 0.07d0 -0.33d0 -0.32d0
-0.14d0 -0.54d0 -0.03d0 -0.13d0
-0.17d0 0.35d0 0.17d0 0.11d0)))
 (wr (make-array n :element-type 'double-float))
 (wi (make-array n :element-type 'double-float))
 (vl (make-array 0 :element-type 'double-float))
 (vr (make-array (* n n) :element-type 'double-float))
 (lwork 660)
 (work (make-array lwork :element-type 'double-float)))
    (multiple-value-bind (z-jobvl z-jobvr z-n z-a z-lda z-wr z-wi z-vl z-ldvl
```

```
                          z-vr z-ldvr z-work z-lwork info)
(dgeev "N" "V" n a-mat n wr wi vl n vr n work lwork 0)
      (declare (ignore z-jobvl z-jobvr z-n z-a z-lda z-wr z-wi z-vl z-ldvl z-vr
       z-ldvr z-work z-lwork))
      (let ((e-val (make-eigval wr wi))
    (e-vec (make-eigvec n vr wi)))
;; Display solution
(cond ((zerop info)
       (print-dgeev-results e-val
    e-vec))
      (t
       (format t "Failure in DGEEV.  INFO = ~D~%" info)))
;; Display workspace info
(format t "Optimum workspace required = ~D~%" (truncate (aref work 0)))
(format t "Workspace provided = ~D~%" lwork)

(values e-val e-vec)))))

(rt:deftest dgeev.1
    (multiple-value-bind (e-val e-vec)
(test-dgeev)
      (list (check-eigen-val-vec 0 e-val e-vec
 0.799482122586210d0
 #(-0.6550887675124076d0
   -0.5236294609021240d0
   0.5362184613722345d0
   -0.0956067782012298d0))
    (check-eigen-val-vec 1 e-val e-vec
 #c(-0.0994124532950747d0 0.4007924719897546d0)
 #(#c(-0.193301548264222d0 0.254631571927584d0)
   #c(0.251856531726740d0 -0.522404734711629d0)
   #c(0.097182458443282d0 -0.308383755897228d0)
   #c(0.675954054254748 0d0)))
    (check-eigen-val-vec 2 e-val e-vec
 #c(-0.0994124532950747d0 -0.4007924719897546d0)
 #(#c(-0.193301548264222d0 -0.254631571927584d0)
   #c(0.251856531726740d0 0.522404734711629d0)
   #c(0.097182458443282d0 0.308383755897228d0)
   #c(0.675954054254748 0d0)))
    (check-eigen-val-vec 3 e-val e-vec
 -0.100657215996059d0
 #(0.125332697230903d0
   0.332022215571751d0
   0.593837759557331d0
   0.722087029862455d0
   -0.6550887675124076d0))))
  (t t t t))

;; Expected results http://www.nag.co.uk/lapack-ex/examples/results/dgeevx-ex.r
;;
```

```
;; DGEEVX Example Program Results
;;
;;  Eigenvalue( 1) =  7.9948E-01
;;
;;  Reciprocal condition number =  9.9E-01
;;  Error bound                 =  1.3E-16
;;
;;  Eigenvector( 1)
;;  -6.5509E-01
;;  -5.2363E-01
;;   5.3622E-01
;;  -9.5607E-02
;;
;;  Reciprocal condition number =  8.2E-01
;;  Error bound                 =  1.6E-16
;;
;;  Eigenvalue( 2) = (-9.9412E-02, 4.0079E-01)
;;
;;  Reciprocal condition number =  7.0E-01
;;  Error bound                 =  1.8E-16
;;
;;  Eigenvector( 2)
;;  (-1.9330E-01, 2.5463E-01)
;;  ( 2.5186E-01,-5.2240E-01)
;;  ( 9.7182E-02,-3.0838E-01)
;;  ( 6.7595E-01, 0.0000E+00)
;;
;;  Reciprocal condition number =  4.0E-01
;;  Error bound                 =  3.3E-16
;;
;;  Eigenvalue( 3) = (-9.9412E-02,-4.0079E-01)
;;
;;  Reciprocal condition number =  7.0E-01
;;  Error bound                 =  1.8E-16
;;
;;  Eigenvector( 3)
;;  (-1.9330E-01,-2.5463E-01)
;;  ( 2.5186E-01, 5.2240E-01)
;;  ( 9.7182E-02, 3.0838E-01)
;;  ( 6.7595E-01,-0.0000E+00)
;;
;;  Reciprocal condition number =  4.0E-01
;;  Error bound                 =  3.3E-16
;;
;;  Eigenvalue( 4) = -1.0066E-01
;;
;;  Reciprocal condition number =  5.7E-01
;;  Error bound                 =  2.3E-16
;;
;;  Eigenvector( 4)
```

```
;;   1.2533E-01
;;   3.3202E-01
;;   5.9384E-01
;;   7.2209E-01
;;
;;  Reciprocal condition number =  3.1E-01
;;  Error bound                 =  4.2E-16
;;
(defun print-dgeevx-results (tol e-val e-vec rconde rcondv)
  (format t "~2%DGEEVX Example Program Results~%")
  (let ((n (length e-val)))
    (dotimes (k n)
      (format t "Eigenvalue(~D) = ~A~%" k (aref e-val k))
      (let ((rcnd (aref rconde k)))
(format t "Reciprocal condition number = ~A~%" rcnd)
(if (plusp rcnd)
    (format t "Error bound = ~A~%" (/ tol rcnd))
    (format t "Error bound is infinite~%")))

      (format t "~%Eigenvector(~D)~%" k)
      (dotimes (row n)
(format t "~A~%" (aref e-vec row k)))
      (let ((rcnd (aref rcondv k)))
(format t "Reciprocal condition number = ~A~%" rcnd)
(if (plusp rcnd)
    (format t "Error bound = ~A~%" (/ tol rcnd))
    (format t "Error bound is infinity~%")))
      (terpri))))

(defun test-dgeevx ()
  (let* ((n 4)
 (a-mat (make-array (* n n) :element-type 'double-float
    :initial-contents '(0.35d0 0.09d0 -0.44d0 0.25d0
0.45d0 0.07d0 -0.33d0 -0.32d0
-0.14d0 -0.54d0 -0.03d0 -0.13d0
-0.17d0 0.35d0 0.17d0 0.11d0)))
 (wr (make-array n :element-type 'double-float))
 (wi (make-array n :element-type 'double-float))
 (vl (make-array (* n n) :element-type 'double-float))
 (vr (make-array (* n n) :element-type 'double-float))
 (scale (make-array n :element-type 'double-float))
 (rconde (make-array n :element-type 'double-float))
 (rcondv (make-array n :element-type 'double-float))
 (lwork 660)
 (work (make-array lwork :element-type 'double-float))
 (iwork (make-array (- (* n 2) 2) :element-type 'f2cl-lib::integer4)))
    (multiple-value-bind (z-balanc z-jobvl z-jobvr z-sense z-n z-a z-lda z-wr
                          z-wi z-vl z-ldvl z-vr z-ldvr ilo ihi z-scale abnrm
                          z-rconde z-rcondv z-work z-lwork z-iwork info)
(dgeevx "Balance" "Vectors (left)" "Vectors (right)"
```

```
"Both reciprocal condition numbers"
n a-mat n wr wi vl n vr n 0 0 scale 0d0 rconde rcondv
work lwork iwork 0)
      (declare (ignore z-balanc z-jobvl z-jobvr z-sense z-n z-a z-lda z-wr
                       z-wi z-vl z-ldvl z-vr z-ldvr z-scale z-rconde z-rcondv
                       z-work z-lwork z-iwork))
      ;; Display solution
      (cond ((zerop info)
     (let* ((eps (dlamch "Eps"))
    (tol (* eps abnrm)))
       (print-dgeevx-results tol
     (make-eigval wr wi)
     (make-eigvec n vr wi)
     rconde rcondv)))
    (t
     (format t "Failure in DGEEV.  INFO = ~D~%" info)))
      ;; Display workspace info
      (format t "Optimum workspace required = ~D~%" (truncate (aref work 0)))
      (format t "Workspace provided = ~D~%" lwork))))

;; Expected results (from
;;    http://www.nag.co.uk/lapack-ex/examples/results/dgesv-ex.r)
;; Solution
;;         1.0000    -1.0000     3.0000    -5.0000
;;
;;  Details of factorization
;;              1          2          3          4
;;  1      5.2500    -2.9500    -0.9500    -3.8000
;;  2      0.3429     3.8914     2.3757     0.4129
;;  3      0.3010    -0.4631    -1.5139     0.2948
;;  4     -0.2114    -0.3299     0.0047     0.1314
;;
;;  Pivot indices
;;              2          2          3          4
;;
(defun print-dgesv-results (n a b ipiv)
  (format t "~2%DGESV Example Program Results~%")
  (format t "Solution~%")
  (dotimes (k n)
    (format t "~21,14e " (aref b k)))
  (format t "~&Details of factorization~%")
  (dotimes (r n)
    (dotimes (c n)
      (format t "~21,14e" (aref a (+ r (* c n)))))
    (terpri))
  (format t "Pivot indices~%")
  (dotimes (k n)
    (format t " ~d" (aref ipiv k)))
  (terpri))
```

```
(defun test-dgesv ()
  ;;
  ;; Matrix A:
  ;;  1.80   2.88   2.05  -0.89
  ;;  5.25  -2.95  -0.95  -3.80
  ;;  1.58  -2.69  -2.90  -1.04
  ;; -1.11  -0.66  -0.59   0.80
  ;;
  ;; RHS:
  ;; 9.52  24.35   0.77  -6.22
  (let* ((n 4)
 (a-mat (make-array (* n n) :element-type 'double-float
    :initial-contents '(1.80d0 5.25d0 1.58d0 -1.11d0
2.88d0 -2.95d0 -2.69d0 -0.66d0
2.05d0 -0.95d0 -2.90d0 -0.59d0
-0.89d0 -3.80d0 -1.04d0 0.8d0)))
 (b (make-array n :element-type 'double-float
:initial-contents '(9.52d0 24.35d0 0.77d0 -6.22d0)))
 (ipiv (make-array n :element-type 'f2cl-lib:integer4)))
    (multiple-value-bind (z-n z-nrhs z-a z-lda z-ipiv z-b z-ldb info)
(dgesv n 1 a-mat n ipiv b n 0)
      (declare (ignore z-n z-nrhs z-a z-lda z-ipiv z-b z-ldb))
      ;; Display solution
      (cond ((zerop info)
     (print-dgesv-results n a-mat b ipiv))
    (t
     (format t "The (~D, ~D) element of the factor U is zero~%"
                         info info))))))

;; Expected results (from )
;;
;; It seems, however, that the result from that page are wrong.  At
;; least they seem wrong when I run the actual test program.  The main
;; difference is that the singular vectors have the signs of some
;; entries wrong.
;;
;; The result below is what the test program actually produces.

;; DGESDD Example Program Results
;;
;;  Singular values
;;      9.9966  3.6831  1.3569  0.5000
;;  Left singular vectors
;;           1       2       3       4
;;  1  -0.1921  0.8030 -0.0041  0.5642
;;  2   0.8794  0.3926  0.0752 -0.2587
;;  3  -0.2140  0.2980 -0.7827 -0.5027
;;  4   0.3795 -0.3351 -0.6178  0.6017
;;
;;  Right singular vectors by row (first m rows of V**T)
```

```
;;           1       2       3       4       5       6
;;  1  -0.2774 -0.2020 -0.2918  0.0938  0.4213 -0.7816
;;  2   0.6003  0.0301 -0.3348  0.3699 -0.5266 -0.3353
;;  3   0.1277 -0.2805 -0.6453 -0.6781 -0.0413  0.1645
;;  4  -0.1323 -0.7034 -0.1906  0.5399  0.0575  0.3957
;;
;;  Error estimate for the singular values
;;         1.1E-15
;;
;;  Error estimates for the left singular vectors
;;         1.8E-16    4.8E-16    1.3E-15    1.3E-15
;;
;;  Error estimates for the right singular vectors
;;         1.8E-16    4.8E-16    1.3E-15    2.2E-15
;;
(defun print-dgesdd-results (m n s u a)
  (format t "~2%DGESDD Example Program Results~%")
  (format t "Singular values~%")
  (dotimes (k m)
    (format t "~20,14e" (aref s k)))
  (format t "~2%Left singular vectors~%")
  (dotimes (r m)
    (dotimes (c m)
      (format t "~16,7e" (aref u (+ r (* c m)))))
    (terpri))
  (format t "~%Right singular vectors (first m rows of V**T)~%")
  (dotimes (r m)
    (dotimes (c n)
      (format t "~16,7e" (aref a (+ r (* c m)))))
    (terpri))
  ;; Compute error estimates for the singular vectors
  (let ((serrbd (* (aref s 0) (dlamch "Eps")))
(rcondu (make-array m :element-type 'double-float))
(rcondv (make-array m :element-type 'double-float))
(uerrbd (make-array m :element-type 'double-float))
(verrbd (make-array m :element-type 'double-float)))
    (ddisna "Left" m n s rcondu 0)
    (ddisna "Right" m n s rcondv 0)
    (dotimes (k m)
      (setf (aref uerrbd k) (/ serrbd (aref rcondu k)))
      (setf (aref verrbd k) (/ serrbd (aref rcondv k))))
    (format t "Error estimate for the singular values~%")
    (format t "~20,15g~%" serrbd)
    (format t "~%~%Error estimates for the left singular values~%")
    (format t "~{~15,4e~^ ~}~%" (coerce uerrbd 'list))
    (format t "~%~%Error estimates for the right singular values~%")
    (format t "~{~15,4e~^ ~}~%" (coerce verrbd 'list))))

(defun test-dgesdd ()
  ;;
```

```
  ;; Matrix A:
  ;;  2.27   0.28  -0.48   1.07  -2.35   0.62
  ;; -1.54  -1.67  -3.09   1.22   2.93  -7.39
  ;;  1.15   0.94   0.99   0.79  -1.45   1.03
  ;; -1.94  -0.78  -0.21   0.63   2.30  -2.57
  (let* ((m 4) ; rows
 (n 6) ; cols
 (a-mat (make-array (* m n) :element-type 'double-float
    :initial-contents '(2.27d0 -1.54d0 1.15d0 -1.94d0
0.28d0 -1.67d0 0.94d0 -0.78d0
-0.48d0 -3.09d0 0.99d0 -0.21d0
1.07d0 1.22d0 0.79d0 0.63d0
-2.35d0 2.93d0 -1.45d0 2.30d0
0.62d0 -7.39d0 1.03d0 -2.57d0)))
 (s (make-array (min m n) :element-type 'double-float))
 (u (make-array (* m (min m n)):element-type 'double-float))
 (vt (make-array (* n n) :element-type 'double-float))
 (lwork 1000)
 (work (make-array lwork :element-type 'double-float))
 (iwork (make-array (* 8 (min m n)) :element-type 'f2cl-lib:integer4)))
    (multiple-value-bind (z-jobz z-m z-n z-a z-lda z-s z-u z-ldu z-vt z-ldvt
                          z-work z-lwork z-iwork info)
(dgesdd "Overwrite A by transpose(V)"
                m n a-mat m s u m vt n work lwork iwork 0)
      (declare (ignore z-jobz z-m z-n z-a z-lda z-s z-u z-ldu z-vt z-ldvt
                       z-work z-lwork z-iwork ))
      ;; Display solution
      (cond ((zerop info)
     (print-dgesdd-results m n s u a-mat))
    (t
     (format t "Failure in DGESDD.  Info = ~D~%" info)))
      (format t "Optimum workspace required = ~D~%" (truncate (aref work 0)))
      (format t "Workspace provided = ~D~%" lwork))))

;; Expected results (from
;;     http://www.nag.co.uk/lapack-ex/examples/results/dgesvd-ex.r)
;; DGESVD Example Program Results
;;
;;  Singular values
;;      9.9966  3.6831  1.3569  0.5000
;;  Left singular vectors (first n columns of U)
;;           1       2       3       4
;;  1  -0.2774 -0.6003 -0.1277  0.1323
;;  2  -0.2020 -0.0301  0.2805  0.7034
;;  3  -0.2918  0.3348  0.6453  0.1906
;;  4   0.0938 -0.3699  0.6781 -0.5399
;;  5   0.4213  0.5266  0.0413 -0.0575
;;  6  -0.7816  0.3353 -0.1645 -0.3957
;;
;;  Right singular vectors by row (V**T)
```

```
;;           1       2       3       4
;;  1  -0.1921  0.8794 -0.2140  0.3795
;;  2  -0.8030 -0.3926 -0.2980  0.3351
;;  3   0.0041 -0.0752  0.7827  0.6178
;;  4  -0.5642  0.2587  0.5027 -0.6017
;;
;;  Error estimate for the singular values
;;         1.1E-15
;;
;;  Error estimates for the left singular vectors
;;         1.8E-16    4.8E-16    1.3E-15    2.2E-15
;;
;;  Error estimates for the right singular vectors
;;         1.8E-16    4.8E-16    1.3E-15    1.3E-15
;;
(defun print-dgesvd-results (m n s vt a)
  (format t "~2%DGESVD Example Program Results~%")
  (format t "Singular values~%")
  (dotimes (k n)
    (format t "~20,14e" (aref s k)))
  (format t "~2%Left singular vectors~%")
  (dotimes (r m)
    (dotimes (c n)
      (format t "~16,7e" (aref a (+ r (* c m)))))
    (terpri))
  (format t "~%Right singular vectors (first m rows of V**T)~%")
  (dotimes (r n)
    (dotimes (c n)
      (format t "~16,7e" (aref vt (+ r (* c n)))))
    (terpri))
  ;; Compute error estimates for the singular vectors
  (let ((serrbd (* (aref s 0) (dlamch "Eps")))
(rcondu (make-array n :element-type 'double-float))
(rcondv (make-array n :element-type 'double-float))
(uerrbd (make-array n :element-type 'double-float))
(verrbd (make-array n :element-type 'double-float)))
    (ddisna "Left" m n s rcondu 0)
    (ddisna "Right" m n s rcondv 0)
    (dotimes (k n)
      (setf (aref uerrbd k) (/ serrbd (aref rcondu k)))
      (setf (aref verrbd k) (/ serrbd (aref rcondv k))))
    (format t "Error estimate for the singular values~%")
    (format t "~20,15g~%" serrbd)
    (format t "~%~%Error estimates for the left singular values~%")
    (format t "~{~15,4e~^ ~}~%" (coerce uerrbd 'list))
    (format t "~%~%Error estimates for the right singular values~%")
    (format t "~{~15,4e~^ ~}~%" (coerce verrbd 'list))))

(defun test-dgesvd ()
  ;;
```

```
  ;; Matrix A:
  ;;     2.27  -1.54   1.15  -1.94
  ;;     0.28  -1.67   0.94  -0.78
  ;;    -0.48  -3.09   0.99  -0.21
  ;;     1.07   1.22   0.79   0.63
  ;;    -2.35   2.93  -1.45   2.30
  ;;     0.62  -7.39   1.03  -2.57
  (let* ((m 6) ; rows
 (n 4) ; cols
 (a-mat (make-array (* m n) :element-type 'double-float
    :initial-contents '(2.27d0 0.28d0 -0.48d0 1.07d0 -2.35d0 0.62d0
-1.54d0 -1.67d0 -3.09d0 1.22d0 2.93d0 -7.39d0
1.15d0 0.94d0 0.99d0 0.79d0 -1.45d0 1.03d0
    -1.94d0 -0.78d0 -0.21d0 0.63d0 2.30d0 -2.57d0)))
 (s (make-array (min m n) :element-type 'double-float))
 (u (make-array (* m (min m n)):element-type 'double-float))
 (vt (make-array (* n n) :element-type 'double-float))
 (lwork (+ 10 (* 4 8)
   (* 64 (+ 10 8))))
 (work (make-array lwork :element-type 'double-float)))
    (multiple-value-bind (z-jobz z-jobvt z-m z-n z-a z-lda z-s z-u z-ldu z-vt
                          z-ldvt z-work z-lwork info)
(dgesvd "Overwrite A by U" "Singular vectors (V)"
m n a-mat m s u m vt n work lwork 0)
      (declare (ignore z-jobz z-jobvt z-m z-n z-a z-lda z-s z-u z-ldu z-vt
                       z-ldvt z-work z-lwork))
      ;; Display solution
      (cond ((zerop info)
     (print-dgesvd-results m n s vt a-mat))
    (t
     (format t "Failure in DGESDD.  Info = ~D~%" info)))
      (format t "Optimum workspace required = ~D~%" (truncate (aref work 0)))
      (format t "Workspace provided = ~D~%" lwork))))

(defun make-complex-eigvec (n vr)
  (make-array (list n n)
      :displaced-to vr
      :element-type (array-element-type vr)))

(defun print-zgeev-results (e-val e-vec)
  (format t "~2%ZGEEV Example Program Results~%")
  (let ((n (length e-val)))
    (dotimes (k n)
      (format t "Eigenvalue(~D) = ~A~%" k (aref e-val k))
      (format t "~%Eigenvector(~D)~%" k)
      (dotimes (row n)
(format t "~A~%" (aref e-vec row k)))
      (terpri))))

(defun test-zgeev ()
```

```
  ;; The matrix is
  ;;
  ;; #c(-3.97, -5.04)  #c(-4.11,  3.70)  #c(-0.34,  1.01)  #c( 1.29, -0.86)
  ;; #c( 0.34, -1.50)  #c( 1.52, -0.43)  #c( 1.88, -5.38)  #c( 3.36,  0.65)
  ;; #c( 3.31, -3.85)  #c( 2.50,  3.45)  #c( 0.88, -1.08)  #c( 0.64, -1.48)
  ;; #c(-1.10,  0.82)  #c( 1.81, -1.59)  #c( 3.25,  1.33)  #c( 1.57, -3.44)
  ;;
  ;; Recall that Fortran arrays are column-major order!
  (let* ((n 4)
 (a-mat (make-array (* n n)
    :element-type '(complex double-float)
    :initial-contents '(#c(-3.97d0 -5.04d0)
#c( 0.34d0 -1.50d0)
#c( 3.31d0 -3.85d0)
#c(-1.10d0  0.82d0)
#c(-4.11d0  3.70d0)
#c( 1.52d0 -0.43d0)
#c( 2.50d0  3.45d0)
#c( 1.81d0 -1.59d0)
#c(-0.34d0  1.01d0)
#c( 1.88d0 -5.38d0)
#c( 0.88d0 -1.08d0)
#c( 3.25d0  1.33d0)
#c( 1.29d0 -0.86d0)
#c( 3.36d0  0.65d0)
#c( 0.64d0 -1.48d0)
#c( 1.57d0 -3.44d0))))
 (lwork 660)
 (w (make-array n :element-type '(complex double-float)))
 (rw (make-array lwork :element-type 'double-float))
 (vl (make-array 0 :element-type '(complex double-float)))
 (vr (make-array (* n n) :element-type '(complex double-float)))
 (work (make-array lwork :element-type '(complex double-float))))
    (multiple-value-bind (z-jobvl z-jobvr z-n z-a z-lda z-w z-vl z-ldvl z-vr
z-ldvr z-work z-lwork z-rwork info)
(zgeev "N" "V" n a-mat n w vl n vr n work lwork rw 0)
      (declare (ignore z-jobvl z-jobvr z-n z-a z-lda z-w z-vl z-ldvl z-vr
       z-ldvr z-work z-lwork z-rwork))
      ;; Display solution
      (cond ((zerop info)
     (print-zgeev-results w
  (make-complex-eigvec n vr)))
    (t
     (format t "Failure in DGEEV.  INFO = ~D~%" info)))
      ;; Display workspace info
      (format t "Optimum workspace required = ~D~%"
                (truncate (realpart (aref work 0))))
      (format t "Workspace provided = ~D~%" lwork))))

(defun do-all-lapack-tests ()
```

```
  (test-dgeev)
  (test-dgeevx)
  (test-dgesv)
  (test-dgesdd)
  (test-dgesvd)
  (test-zgeev))

;;; $Log$
;;; Revision 1.11  2006/12/01 04:29:29  rtoy
;;; Create packages for BLAS and LAPACK routines.
;;;
;;; blas.system:
;;; o Converted files are in the BLAS package.
;;; o Add blas-package defsystem to load the package definition.
;;;
;;; lapack.system:
;;; o Converted files are in the LAPACK package.
;;; o Add lapack-package defsystem to load the package definition.
;;;
;;; lapack/lapack-tests.lisp:
;;; o Tests are in the LAPACK package
;;;
;;; Revision 1.10  2006/11/28 15:49:01  rtoy
;;; Print out short title for each test.
;;;
;;; Revision 1.9  2006/11/27 22:22:23  rtoy
;;; Add expected results.
;;;
;;; Revision 1.8  2006/11/27 20:04:33  rtoy
;;; Add DGESVD and update files and tests appropriately.
;;;
;;; Revision 1.7  2006/11/27 15:23:29  rtoy
;;; Add function to run all the tests.
;;;
;;; Revision 1.6  2006/11/26 23:26:47  rtoy
;;; packages/lapack.system:
;;; o Add DGESDD and dependencies
;;; o Add DDISNA to compute condition number of singular vectors
;;;
;;; packages/lapack/.cvsignore:
;;; o Ignore new generated Lisp files.
;;;
;;; packages/lapack/lapack-tests.lisp:
;;; o Add test for DGESDD
;;;
;;; Revision 1.5  2006/11/26 14:26:42  rtoy
;;; Add expected results for DGESV.
;;;
;;; Revision 1.4  2006/11/26 14:24:46  rtoy
;;; packages/lapack.system:
```

```
;;; o DGESV and dependencies
;;;
;;; packages/.cvsignore:
;;; o Ignore generated dgesv.lisp and dependencies
;;;
;;; packages/lapack/lapack-tests.lisp:
;;; o Test routine for DGESV
;;;
;;; Revision 1.3  2006/11/26 05:31:16  rtoy
;;; packages/lapack.system:
;;; o Add DGEEVX and dependencies
;;;
;;; packages/lapack/lapack-tests.lisp:
;;; o Add test for DGEEVX
;;; o Add comments
;;;
;;; packages/lapack/dgeevx.f:
;;; packages/lapack/dlacon.f:
;;; packages/lapack/dlaexc.f:
;;; packages/lapack/dlaqtr.f:
;;; packages/lapack/dlasy2.f:
;;; packages/lapack/dtrexc.f:
;;; packages/lapack/dtrsna.f:
;;; o New files for DGEEVX and dependencies.
;;;
;;; Revision 1.2  2006/11/26 04:53:22  rtoy
;;; Add comments
;;;
;;; Revision 1.1  2006/11/26 04:51:05  rtoy
;;; packages/lapack.system:
;;; o Add defsystem for LAPACK tests
;;;
;;; packages/lapack/lapack-tests.lisp:
;;; o Add simple tests for LAPACK.  (Currently only DGEEV).
;;;
```

Chapter 92

ISAAC Secure Random Number Generation

Optimised Common Lisp implementation of Bob Jenkins' ISAAC-32 algorithm as implemented by Doug Hoyte (see license.isaac)

Indirection, Shift, Accumulate, Add, and Count. More details and the C reference implementations can be found here:

ISAAC: a fast cryptographic random number generator `http://burtleburtle.net/bob/rand/isaacafa.html`. Also see the analysis at `http://burtleburtle.net/bob/rand/isaac.html`.

Compare the code below with the C implementation at `http://burtleburtle.net/bob/c/rand.c`

This lisp implementation is roughly as fast as Jenkins' optimised rand.c when compiled with a good native-code lisp compiler. It also performs well when byte-code compiled.

ISAAC is the third of three cryptograpically secure random number generators. It was designed the criteria

- Deducing the internal state from the results should be intractable
- Be as fast as possible and optimized for speed
- Be crptographically secure.
- No biases should be detectable for the entire cycle length
- There should be no short cycles at all
- Orderly states should become disorderly quickly.

The random values in **randrsl** are selected from the array **randmem**

92.0.5 Axiom help page

— isaac.help —

```
====================================================================
issac examples
====================================================================

USAGE:

First, create an isaac context. There are three functions that do this:

   initKernelSeed => <isaac context>
     *RECOMMENDED* Seeds with values from /dev/arandom on BSD
     or /dev/urandom on Linux. Reads 1024 bytes from the device.

   initCommonLispRandomSeed => <isaac context>
     Seeds with values from your Common Lisp implementation's
     random function. Consumes 256 32-bit values from #'random.

   initNullSeed => <isaac context>
     Seeds with all 0s. Always results in the same stream.
     For comparing with Jenkins' reference implementations.

 These are functions you can pass an isaac context to. They will modify
 the isaac context and return a random value:

   rand32 <isaac context> => <random 32-bit value>
     Uses the ISAAC-32 algorithm to generate a new random value.

   randBits <isaac context> <N> => <random N-bit value>
     Uses the ISAAC-32 algorithm to generate random values between
     0 and (1- (expt 2 N)). This function always consumes one or more
     ISAAC-32 words. Note that the N parameter is different from
     the CL random function parameter.   Examples:
      (randBits ctx 1) => [0,1] (consumes 1 ISAAC-32 word)
      (randBits ctx 2) => [0,1,2,3] (ditto)
      (randBits ctx 3) => [0,1,2,3,4,5,6,7] (ditto)
      (randBits ctx 32) => [0,1,...,(1- (expt 2 32))] (ditto)
      (randBits ctx 33) => [0,1,...,(1- (expt 2 33))] (consumes 2 words)
      (randBits ctx 512) => [0,1,...,(1- (expt 2 512))] (consumes 16 words)


QUICK RECIPE:

 Generate a 128-bit session ID as a 0-padded hexadecimal string:

   (defvar my-isaac-ctx (initKernelSeed))
   (format nil "~32,'0x" (randBits my-isaac-ctx 128))
```

```
 => "078585213B0EF01B1B9BECB291EF38F0"


FAQ:
    Q) My Common Lisp implementation already uses the Mersenne Twister,
       what are the advantages of ISAAC?

    A1) The Mersenne Twister is not a cryptographic PRNG. This means that it
        is possible for someone to predict future values based on previously
        observed values (just over 600 of them). As such, MT is particularly
        undesirable for things like web session IDs. You can still use MT for
        crypto, but you must use a cryptographic hash function on the
        MT output.

    A2) isaac.lisp appears to be roughly as fast as the Mersenne Twister
        #'random of CMUCL 19d on x86 before even considering the
        above-mentioned hash function overhead requirement of MT.

    A3) isaac.lisp is not implemented as an x86 VOP like CMUCL's Mersenne
        Twister, but instead in 100% standard ANSI Common Lisp (except for the
        kernel seed interface). This should mean comparable performance on all
        architectures targeted by your lisp compiler. The non-x86 MT
        implementation is apparently an order-of-magnitude slower.

    Q) How "random" can I expect these values to be?

    A) Very. From Bob Jenkins' website: "Cycles are guaranteed to be
       at least (expt 2 40) values long, and they are (expt 2 8295)
       values long on average. The results are uniformly distributed,
       unbiased, and unpredictable unless you know the seed. [...] Why
       not use RC4? RC4 is three times slower, more biased, has a
       shorter minimum and average cycle length, and is
       proprietary. No way is known to break either RC4 or ISAAC; both
       are immune to Gaussian elimination."


        ________-
```

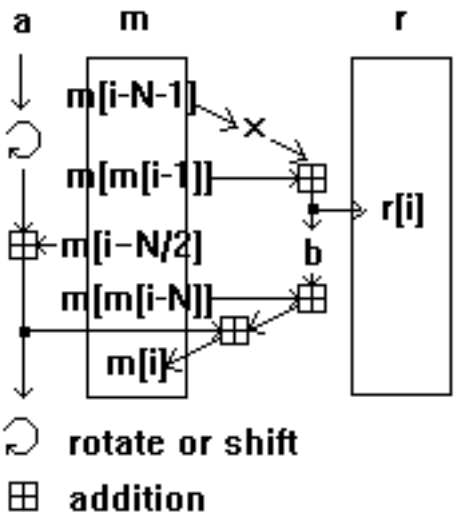

92.0.6 defstruct isaacCtx

randrsl, the result array, is memory of size $246 \cdot 32$ bits. **randmem**, an internal memory of the same size as randrsl. **a** is an accumulator, a single value. **b** is the previous result. **c** is a counter, a 32 bit value.

The random values in randrsl are selected from the array randmem and two elements of randmem are swapped for every byte reported. The security stems from the difficulty of knowing where anyh values is in randmem, and from the difficulty of knowing which locations in randmem are used in selecting each value in the sequence.

The swap of a known position with an unknown position thwarts Gaussian elimination. Gauss can only be applied when we know that a value is used in two equations. Since every swap might change any value, we never know when a position's value changes, so there is no way to be sure that one equation is using the same value as another.

```
#define RANDSIZL   (8)
#define RANDSIZ    (1<<RANDSIZL)

/* context of random number generator */
struct randctx
{
  ub4 randcnt;
  ub4 randrsl[RANDSIZ];
  ub4 randmem[RANDSIZ];
  ub4 randa;
  ub4 randb;
  ub4 randc;
};
typedef  struct randctx  randctx;
```

— defstruct isaacCtx —

```
(defstruct isaacctx
  (randcnt 0 :type (unsigned-byte 32))
  (randrsl
   (make-array 256 :element-type '(unsigned-byte 32) :initial-element 0)
           :type (simple-array (unsigned-byte 32) (256)))
  (randmem
   (make-array 256 :element-type '(unsigned-byte 32) :initial-element 0)
           :type (simple-array (unsigned-byte 32) (256)))
  (a 0 :type (unsigned-byte 32))
  (b 0 :type (unsigned-byte 32))
  (c 0 :type (unsigned-byte 32)))
```

———-

```
#define ind(mm,x)  (*(ub4 *)((ub1 *)(mm) + ((x) & ((RANDSIZ-1)<<2))))
#define rngstep(mix,a,b,mm,m,m2,r,x) \
{ \
  x = *m;  \
  a = (a^(mix)) + *(m2++); \
  *(m++) = y = ind(mm,x) + a + b; \
  *(r++) = b = ind(mm,y>>RANDSIZL) + x; \
}

void     isaac(ctx)
randctx *ctx;
{
   register ub4 a,b,x,y,*m,*mm,*m2,*r,*mend;
   mm=ctx->randmem; r=ctx->randrsl;
   a = ctx->randa; b = ctx->randb + (++ctx->randc);
   for (m = mm, mend = m2 = m+(RANDSIZ/2); m<mend; )
   {
      rngstep( a<<13, a, b, mm, m, m2, r, x);
      rngstep( a>>6 , a, b, mm, m, m2, r, x);
      rngstep( a<<2 , a, b, mm, m, m2, r, x);
      rngstep( a>>16, a, b, mm, m, m2, r, x);
   }
   for (m2 = mm; m2<mend; )
   {
      rngstep( a<<13, a, b, mm, m, m2, r, x);
      rngstep( a>>6 , a, b, mm, m, m2, r, x);
      rngstep( a<<2 , a, b, mm, m, m2, r, x);
      rngstep( a>>16, a, b, mm, m, m2, r, x);
   }
   ctx->randb = b; ctx->randa = a;
}
```

92.0.7 defun generateNextIsaacBlock

Compare this to the funtion **isaac** in the original C source code.

— **defun generateNextIsaacBlock** —

```
(defun generateNextIsaacBlock (ctx)
  (declare (optimize (speed 3) (safety 0)))
  (incf (isaacCtx-c ctx))
  (incf (isaacCtx-b ctx) (isaacCtx-c ctx))
  (loop for i from 0 below 256 do
    (setf (isaacCtx-a ctx)
          (logxor (isaacCtx-a ctx)
                  (logand #xFFFFFFFF
                    (the (unsigned-byte 32)
                      (ash (isaacCtx-a ctx)
                           (ecase (logand i 3)
                             ((0) 13)
                             ((1) -6)
                             ((2) 2)
                             ((3) -16)))))))
    (setf (isaacCtx-a ctx)
          (logand #xFFFFFFFF
                  (+ (isaacCtx-a ctx)
                     (aref (isaacCtx-randmem ctx) (logand (+ i 128) #xFF)))))
    (let* ((x (aref (isaacCtx-randmem ctx) i))
           (y (logand #xFFFFFFFF
                      (+ (aref (isaacCtx-randmem ctx)
                               (logand (ash x -2) #xFF))
                         (isaacCtx-a ctx)
                         (isaacCtx-b ctx)))))
      (setf (aref (isaacCtx-randmem ctx) i) y)
      (setf (isaacCtx-b ctx)
            (logand #xFFFFFFFF
                    (+ (aref (isaacCtx-randmem ctx)
                             (logand (ash y -10) #xFF)) x)))
      (setf (aref (isaacCtx-randrsl ctx) i) (isaacCtx-b ctx)))))
```

———-

```
/*
------------------------------------------------------------------------------
 Call rand(/o_ randctx *r _o/) to retrieve a single 32-bit random value
------------------------------------------------------------------------------
*/
#define rand(r) \
   (!(r)->randcnt-- ? \
     (isaac(r), (r)->randcnt=RANDSIZ-1, (r)->randrsl[(r)->randcnt]) : \
     (r)->randrsl[(r)->randcnt])
```

92.0.8 defun rand32

This is a user-level function to return a random value.

— **defun rand32** —

```
(defun rand32 (ctx)
  (let ((c (isaacCtx-randcnt ctx)))
    (declare (optimize (speed 3) (safety 0)))
    (decf (isaacCtx-randcnt ctx))
    (if (zerop c)
      (progn
        (generateNextIsaacBlock ctx)
        (setf (isaacCtx-randcnt ctx) 255)
        (aref (isaacCtx-randrsl ctx) 255))
      (aref (isaacCtx-randrsl ctx) (isaacCtx-randcnt ctx)))))
```

———-

92.0.9 defun randBits

This is a user-level function to return n random bits.

— **defun randBits** —

```
(defun randBits (ctx n)
  (let ((v 0))
    (loop while (> n 0) do
      (setq v (logior (ash v (min n 32))
                      (logand (1- (ash 1 (min n 32)))
                              (rand32 ctx))))
      (decf n (min n 32)))
    v))
```

———-

92.0.10 defmacro incfWrap32

This simple macro does additon mod 2^32, imitating a 32 bit register

— **defmacro incfWrap32** —

```
(defmacro incfWrap32 (a b)
  `(setf ,a (logand #xFFFFFFFF (+ ,a ,b))))
```

———-

```
#define mix(a,b,c,d,e,f,g,h) \
{ \
   a^=b<<11; d+=a; b+=c; \
   b^=c>>2;  e+=b; c+=d; \
   c^=d<<8;  f+=c; d+=e; \
   d^=e>>16; g+=d; e+=f; \
   e^=f<<10; h+=e; f+=g; \
   f^=g>>4;  a+=f; g+=h; \
   g^=h<<8;  b+=g; h+=a; \
   h^=a>>9;  c+=h; a+=b; \
}
```

92.0.11 defmacro mix

Because Lisp uses multi-precision integers we need to mask all arithmetic to 32 bits. Compare this with the C version of mix.

— defmacro mix —

```
(defmacro mix (a b c d e f g h)
  `(progn
     (setf ,a (logxor ,a (logand #xFFFFFFFF (ash ,b 11))))
     (incfWrap32 ,d ,a)
     (incfWrap32 ,b ,c)
     (setf ,b (logxor ,b (logand #xFFFFFFFF (ash ,c -2))))
     (incfWrap32 ,e ,b)
     (incfWrap32 ,c ,d)
     (setf ,c (logxor ,c (logand #xFFFFFFFF (ash ,d 8))))
     (incfWrap32 ,f ,c)
     (incfWrap32 ,d ,e)
     (setf ,d (logxor ,d (logand #xFFFFFFFF (ash ,e -16))))
     (incfWrap32 ,g ,d)
     (incfWrap32 ,e ,f)
     (setf ,e (logxor ,e (logand #xFFFFFFFF (ash ,f 10))))
     (incfWrap32 ,h ,e)
     (incfWrap32 ,f ,g)
     (setf ,f (logxor ,f (logand #xFFFFFFFF (ash ,g -4))))
     (incfWrap32 ,a ,f)
     (incfWrap32 ,g ,h)
     (setf ,g (logxor ,g (logand #xFFFFFFFF (ash ,h 8))))
     (incfWrap32 ,b ,g)
     (incfWrap32 ,h ,a)
     (setf ,h (logxor ,h (logand #xFFFFFFFF (ash ,a -9))))
     (incfWrap32 ,c ,h)
     (incfWrap32 ,a ,b)))
```

———-

```
/* if (flag==TRUE), then use the contents of randrsl[] to initialize mm[]. */
```

```
void randinit(ctx, flag)
randctx *ctx;
word     flag;
{
   word i;
   ub4 a,b,c,d,e,f,g,h;
   ub4 *m,*r;
   ctx->randa = ctx->randb = ctx->randc = 0;
   m=ctx->randmem;
   r=ctx->randrsl;
   a=b=c=d=e=f=g=h=0x9e3779b9;  /* the golden ratio */

   for (i=0; i<4; ++i)          /* scramble it */
   {
     mix(a,b,c,d,e,f,g,h);
   }

   if (flag)
   {
     /* initialize using the contents of r[] as the seed */
     for (i=0; i<RANDSIZ; i+=8)
     {
       a+=r[i  ]; b+=r[i+1]; c+=r[i+2]; d+=r[i+3];
       e+=r[i+4]; f+=r[i+5]; g+=r[i+6]; h+=r[i+7];
       mix(a,b,c,d,e,f,g,h);
       m[i  ]=a; m[i+1]=b; m[i+2]=c; m[i+3]=d;
       m[i+4]=e; m[i+5]=f; m[i+6]=g; m[i+7]=h;
     }
     /* do a second pass to make all of the seed affect all of m */
     for (i=0; i<RANDSIZ; i+=8)
     {
       a+=m[i  ]; b+=m[i+1]; c+=m[i+2]; d+=m[i+3];
       e+=m[i+4]; f+=m[i+5]; g+=m[i+6]; h+=m[i+7];
       mix(a,b,c,d,e,f,g,h);
       m[i  ]=a; m[i+1]=b; m[i+2]=c; m[i+3]=d;
       m[i+4]=e; m[i+5]=f; m[i+6]=g; m[i+7]=h;
     }
   }
   else
   {
     /* fill in m[] with messy stuff */
     for (i=0; i<RANDSIZ; i+=8)
     {
       mix(a,b,c,d,e,f,g,h);
       m[i  ]=a; m[i+1]=b; m[i+2]=c; m[i+3]=d;
       m[i+4]=e; m[i+5]=f; m[i+6]=g; m[i+7]=h;
     }
   }

   isaac(ctx);            /* fill in the first set of results */
```

```
   ctx->randcnt=RANDSIZ;  /* prepare to use the first set of results */
}
```

92.0.12 defun scramble

Compare this with the **randinit** routine in the original C code. Note that the **flag** value, when set to TRUE, in the C code would initialize the array with "messy stuff". Instead this is handled by which randomization initialization routine was called.

— **defun scramble** —

```
(defun scramble (ctx)
  (let (a b c d e f g h) ; a thru h are a pseudo-256 byte register
    (setf a #x9e3779b9  b a  c a  d a  e a  f a  g a  h a) ; golden ratio
    (loop for i from 0 below 4 do
      (mix a b c d e f g h))
    ;; Pass #1 initialize using the contents of randrsl as the seed
    (loop for i from 0 below 256 by 8 do
      (incfWrap32 a (aref (isaacCtx-randrsl ctx) (+ i 0)))
      (incfWrap32 b (aref (isaacCtx-randrsl ctx) (+ i 1)))
      (incfWrap32 c (aref (isaacCtx-randrsl ctx) (+ i 2)))
      (incfWrap32 d (aref (isaacCtx-randrsl ctx) (+ i 3)))
      (incfWrap32 e (aref (isaacCtx-randrsl ctx) (+ i 4)))
      (incfWrap32 f (aref (isaacCtx-randrsl ctx) (+ i 5)))
      (incfWrap32 g (aref (isaacCtx-randrsl ctx) (+ i 6)))
      (incfWrap32 h (aref (isaacCtx-randrsl ctx) (+ i 7)))
      (mix a b c d e f g h)
      (setf (aref (isaacCtx-randmem ctx) (+ i 0)) a)
      (setf (aref (isaacCtx-randmem ctx) (+ i 1)) b)
      (setf (aref (isaacCtx-randmem ctx) (+ i 2)) c)
      (setf (aref (isaacCtx-randmem ctx) (+ i 3)) d)
      (setf (aref (isaacCtx-randmem ctx) (+ i 4)) e)
      (setf (aref (isaacCtx-randmem ctx) (+ i 5)) f)
      (setf (aref (isaacCtx-randmem ctx) (+ i 6)) g)
      (setf (aref (isaacCtx-randmem ctx) (+ i 7)) h))
    ;; Pass #2 do a second pass to make all of the seed affect all of randmem
    (loop for i from 0 below 256 by 8 do
      (incfWrap32 a (aref (isaacCtx-randmem ctx) (+ i 0)))
      (incfWrap32 b (aref (isaacCtx-randmem ctx) (+ i 1)))
      (incfWrap32 c (aref (isaacCtx-randmem ctx) (+ i 2)))
      (incfWrap32 d (aref (isaacCtx-randmem ctx) (+ i 3)))
      (incfWrap32 e (aref (isaacCtx-randmem ctx) (+ i 4)))
      (incfWrap32 f (aref (isaacCtx-randmem ctx) (+ i 5)))
      (incfWrap32 g (aref (isaacCtx-randmem ctx) (+ i 6)))
      (incfWrap32 h (aref (isaacCtx-randmem ctx) (+ i 7)))
      (mix a b c d e f g h)
      (setf (aref (isaacCtx-randmem ctx) (+ i 0)) a)
      (setf (aref (isaacCtx-randmem ctx) (+ i 1)) b)
      (setf (aref (isaacCtx-randmem ctx) (+ i 2)) c)
```

```
      (setf (aref (isaacCtx-randmem ctx) (+ i 3)) d)
      (setf (aref (isaacCtx-randmem ctx) (+ i 4)) e)
      (setf (aref (isaacCtx-randmem ctx) (+ i 5)) f)
      (setf (aref (isaacCtx-randmem ctx) (+ i 6)) g)
      (setf (aref (isaacCtx-randmem ctx) (+ i 7)) h))
    ;; fill in the first set of results
    (generateNextIsaacBlock ctx)
    ;; prepare to use the first set of results
    (setf (isaacCtx-randcnt ctx) 256)
    ctx))
```

———-

Initialization is not specified in the original algorithm. Here we provide three possible sources.

- from system random seed sources (either /dev/arandom or /dev/urandom)
- from Common Lisp's random function
- from an all-0's seed

92.0.13 defun initKernelSeed

We try to initialize from system random sources.

— **defun initKernelSeed** —

```
(defun initKernelSeed ()
  (let ((ctx (make-isaacCtx)))
    (or
      (ignore-errors
        (with-open-file
         (r "/dev/arandom" :direction :input :element-type '(unsigned-byte 32))
          #1=(loop for i from 0 below 256 do
               (setf (aref (isaacCtx-randrsl ctx) i) (read-byte r)))
          t))
      (ignore-errors
        (with-open-file
         (r "/dev/urandom" :direction :input :element-type '(unsigned-byte 32))
          #1#
          t))
      (error "couldn't open /dev/arandom or /dev/urandom"))
    (scramble ctx)))
```

———-

92.0.14 defun initCommonLispRandomSeed

We initialize from Common Lisp's random function.

— **defun initCommonLispRandomSeed** —

```
(defun initCommonLispRandomSeed ()
  (let ((ctx (make-isaacCtx)))
    (loop for i from 0 below 256 do
      (setf (aref (isaacCtx-randrsl ctx) i)
            (random (ash 1 32))))
    (scramble ctx)))
```

———-

92.0.15 defun initNullSeed

We initialize the seed to 0. This is useful for repeatable testing.

— **defun initNullSeed** —

```
(defun initNullSeed ()
  (let ((ctx (make-isaacCtx)))
    (scramble ctx)))
```

———-

```
int main()
{
  ub4 i,j;
  randctx ctx;
  ctx.randa=ctx.randb=ctx.randc=(ub4)0;
  for (i=0; i<256; ++i) ctx.randrsl[i]=(ub4)0;
  randinit(&ctx, TRUE);
  for (i=0; i<2; ++i)
  {
    isaac(&ctx);
    for (j=0; j<256; ++j)
    {
      printf("%.8lx",ctx.randrsl[j]);
      if ((j&7)==7) printf("\n");
    }
  }
}
```

and the lisp equivalent test function

```
(defun jenkins-output (filename)
```

```
(let ((ctx (initNullSeed)))
  (with-open-file (o filename :direction :output :if-exists :supersede)
    (loop for i from 0 below 2 do
      (generateNextIsaacBlock ctx)
      (loop for j from 0 below 256 do
        (format o "~(~8,'0x~)" (aref (isaacCtx-randrsl ctx) j))
        (if (= 7 (logand j 7)) (terpri o)))))))
```

Chapter 93

Chunk collections

— BLAS FORTRAN —

```
\getchunk{dcabs1.f}
\getchunk{lsame.f}
\getchunk{xerbla.f}
\getchunk{dasum.f}
\getchunk{daxpy.f}
\getchunk{dcopy.f}
\getchunk{ddot.f}
\getchunk{dnrm2.f}
\getchunk{drotg.f}
\getchunk{drot.f}
\getchunk{dscal.f}
\getchunk{dswap.f}
\getchunk{dzasum.f}
\getchunk{dznrm2.f}
\getchunk{icamax.f}
\getchunk{idamax.f}
\getchunk{isamax.f}
\getchunk{izamax.f}
\getchunk{zaxpy.f}
\getchunk{zcopy.f}
\getchunk{zdotc.f}
\getchunk{zdotu.f}
\getchunk{zdscal.f}
\getchunk{zrotg.f}
\getchunk{zscal.f}
\getchunk{zswap.f}
\getchunk{dgbmv.f}
\getchunk{dgemv.f}
\getchunk{dger.f}
```

```
\getchunk{dsbmv.f}
\getchunk{dspmv.f}
\getchunk{dspr2.f}
\getchunk{dspr.f}
\getchunk{dsymv.f}
\getchunk{dsyr2.f}
\getchunk{dsyr.f}
\getchunk{dtbmv.f}
\getchunk{dtbsv.f}
\getchunk{dtpmv.f}
\getchunk{dtpsv.f}
\getchunk{dtrmv.f}
\getchunk{dtrsv.f}
\getchunk{zgbmv.f}
\getchunk{zgemv.f}
\getchunk{zgerc.f}
\getchunk{zgeru.f}
\getchunk{zhbmv.f}
\getchunk{zhemv.f}
\getchunk{zher2.f}
\getchunk{zher.f}
\getchunk{zhpmv.f}
\getchunk{zhpr2.f}
\getchunk{zhpr.f}
\getchunk{ztbmv.f}
\getchunk{ztbsv.f}
\getchunk{ztpmv.f}
\getchunk{ztpsv.f}
\getchunk{ztrmv.f}
\getchunk{ztrsv.f}
\getchunk{dgemm.f}
\getchunk{dsymm.f}
\getchunk{dsyr2k.f}
\getchunk{dsyrk.f}
\getchunk{dtrmm.f}
\getchunk{dtrsm.f}
\getchunk{zgemm.f}
\getchunk{zhemm.f}
\getchunk{zher2k.f}
\getchunk{zherk.f}
\getchunk{zsymm.f}
\getchunk{zsyr2k.f}
\getchunk{zsyrk.f}
\getchunk{ztrmm.f}
\getchunk{ztrsm.f}
```

— LAPACK FORTRAN —

```
\getchunk{dbdsdc.f}
\getchunk{dbdsqr.f}
\getchunk{ddisna.f}
\getchunk{dgebak.f}
\getchunk{dgebal.f}
\getchunk{dgebd2.f}
\getchunk{dgebrd.f}
\getchunk{dgeev.f}
\getchunk{dgeevx.f}
\getchunk{dgehd2.f}
\getchunk{dgehrd.f}
\getchunk{dgelq2.f}
\getchunk{dgelqf.f}
\getchunk{dgeqr2.f}
\getchunk{dgeqrf.f}
\getchunk{dgesdd.f}
\getchunk{dgesvd.f}
\getchunk{dgesv.f}
\getchunk{dgetf2.f}
\getchunk{dgetrf.f}
\getchunk{dgetrs.f}
\getchunk{dhseqr.f}
\getchunk{disnan.f}
\getchunk{dlabad.f}
\getchunk{dlabrd.f}
\getchunk{dlacon.f}
\getchunk{dlacpy.f}
\getchunk{dladiv.f}
\getchunk{dlaed6.f}
\getchunk{dlaexc.f}
\getchunk{dlahqr.f}
\getchunk{dlahrd.f}
\getchunk{dlaisnan.f}
\getchunk{dlaln2.f}
\getchunk{dlamch.f}
\getchunk{dlamc1.f}
\getchunk{dlamc2.f}
\getchunk{dlamc3.f}
\getchunk{dlamc4.f}
\getchunk{dlamc5.f}
\getchunk{dlamrg.f}
\getchunk{dlange.f}
\getchunk{dlanhs.f}
\getchunk{dlanst.f}
\getchunk{dlanv2.f}
\getchunk{dlapy2.f}
\getchunk{dlapy3.f}
\getchunk{dlaqtr.f}
\getchunk{dlarfb.f}
\getchunk{dlarfg.f}
```

```
\getchunk{dlarf.f}
\getchunk{dlarft.f}
\getchunk{dlarfx.f}
\getchunk{dlartg.f}
\getchunk{dlas2.f}
\getchunk{dlascl.f}
\getchunk{dlasd0.f}
\getchunk{dlasd1.f}
\getchunk{dlasd2.f}
\getchunk{dlasd3.f}
\getchunk{dlasd4.f}
\getchunk{dlasd5.f}
\getchunk{dlasd6.f}
\getchunk{dlasd7.f}
\getchunk{dlasd8.f}
\getchunk{dlasda.f}
\getchunk{dlasdq.f}
\getchunk{dlasdt.f}
\getchunk{dlaset.f}
\getchunk{dlasq1.f}
\getchunk{dlasq2.f}
\getchunk{dlasq3.f}
\getchunk{dlasq4.f}
\getchunk{dlasq5.f}
\getchunk{dlasq6.f}
\getchunk{dlasr.f}
\getchunk{dlasrt.f}
\getchunk{dlassq.f}
\getchunk{dlasv2.f}
\getchunk{dlaswp.f}
\getchunk{dlasy2.f}
\getchunk{dorg2r.f}
\getchunk{dorgbr.f}
\getchunk{dorghr.f}
\getchunk{dorgl2.f}
\getchunk{dorglq.f}
\getchunk{dorgqr.f}
\getchunk{dorm2r.f}
\getchunk{dormbr.f}
\getchunk{dorml2.f}
\getchunk{dormlq.f}
\getchunk{dormqr.f}
\getchunk{dtrevc.f}
\getchunk{dtrexc.f}
\getchunk{dtrsna.f}
\getchunk{ieeeck.f}
\getchunk{ilaenv.f}
\getchunk{ilazlc.f}
\getchunk{ilazlr.f}
\getchunk{zgebak.f}
```

```
\getchunk{zgebal.f}
\getchunk{zgeev.f}
\getchunk{zgehd2.f}
\getchunk{zgehrd.f}
\getchunk{zhseqr.f}
\getchunk{zlacgv.f}
\getchunk{zlacpy.f}
\getchunk{zladiv.f}
\getchunk{zlahqr.f}
\getchunk{zlahr2.f}
\getchunk{zlange.f}
\getchunk{zlaqr0.f}
\getchunk{zlaqr1.f}
\getchunk{zlaqr2.f}
\getchunk{zlaqr3.f}
\getchunk{zlaqr4.f}
\getchunk{zlaqr5.f}
\getchunk{zlarfb.f}
\getchunk{zlarf.f}
\getchunk{zlzrfg.f}
\getchunk{zlarft.f}
\getchunk{zlartg.f}
\getchunk{zlascl.f}
\getchunk{zlaset.f}
\getchunk{zlassq.f}
\getchunk{zlatrs.f}
\getchunk{zrot.f}
\getchunk{ztrevc.f}
\getchunk{ztrexc.f}
\getchunk{zung2r.f}
\getchunk{zunghr.f}
\getchunk{zungqr.f}
\getchunk{zunm2r.f}
\getchunk{zunmhr.f}
\getchunk{zunmqr.f}
```

———-

— Numerics —

```
(in-package "BOOT")

\getchunk{BLAS 1 dcabs1}
\getchunk{BLAS 1 dasum}
\getchunk{BLAS 1 daxpy}
\getchunk{BLAS 1 dcopy}
\getchunk{BLAS 1 ddot}
\getchunk{BLAS 1 dnrm2}
\getchunk{BLAS 1 drotg}
```

```
\getchunk{BLAS 1 drot}
\getchunk{BLAS 1 dscal}
\getchunk{BLAS 1 dswap}
\getchunk{BLAS 1 dzasum}
\getchunk{BLAS 1 dznrm2}
\getchunk{BLAS 1 icamax}
\getchunk{BLAS 1 idamax}
\getchunk{BLAS 1 isamax}
\getchunk{BLAS 1 izamax}
\getchunk{BLAS 1 zaxpy}
```

———-

— untested —

```
\getchunk{BLAS lsame}
\getchunk{BLAS xerbla}

\getchunk{BLAS 1 zcopy}
\getchunk{BLAS 1 zdotc}
\getchunk{BLAS 1 zdotu}
\getchunk{BLAS 1 zdscal}
\getchunk{BLAS 1 zrotg}
\getchunk{BLAS 1 zscal}
\getchunk{BLAS 1 zswap}

\getchunk{BLAS 2 dgbmv}
\getchunk{BLAS 2 dgemv}
\getchunk{BLAS 2 dger}
\getchunk{BLAS 2 dsbmv}
\getchunk{BLAS 2 dspmv}
\getchunk{BLAS 2 dspr2}
\getchunk{BLAS 2 dspr}
\getchunk{BLAS 2 dsymv}
\getchunk{BLAS 2 dsyr2}
\getchunk{BLAS 2 dsyr}
\getchunk{BLAS 2 dtbmv}
\getchunk{BLAS 2 dtbsv}
\getchunk{BLAS 2 dtpmv}
\getchunk{BLAS 2 dtpsv}
\getchunk{BLAS 2 dtrmv}
\getchunk{BLAS 2 dtrsv}
\getchunk{BLAS 2 zgbmv}
\getchunk{BLAS 2 zgemv}
\getchunk{BLAS 2 zgerc}
\getchunk{BLAS 2 zgeru}
\getchunk{BLAS 2 zhbmv}
\getchunk{BLAS 2 zhemv}
\getchunk{BLAS 2 zher2}
```

```
\getchunk{BLAS 2 zher}
\getchunk{BLAS 2 zhpmv}
\getchunk{BLAS 2 zhpr2}
\getchunk{BLAS 2 zhpr}
\getchunk{BLAS 2 ztbmv}
\getchunk{BLAS 2 ztbsv}
\getchunk{BLAS 2 ztpmv}
\getchunk{BLAS 2 ztpsv}
\getchunk{BLAS 2 ztrmv}
\getchunk{BLAS 2 ztrsv}

\getchunk{BLAS 3 dgemm}
\getchunk{BLAS 3 dsymm}
\getchunk{BLAS 3 dsyr2k}
\getchunk{BLAS 3 dsyrk}
\getchunk{BLAS 3 dtrmm}
\getchunk{BLAS 3 dtrsm}
\getchunk{BLAS 3 zgemm}
\getchunk{BLAS 3 zhemm}
\getchunk{BLAS 3 zher2k}
\getchunk{BLAS 3 zherk}
\getchunk{BLAS 3 zsymm}
\getchunk{BLAS 3 zsyr2k}
\getchunk{BLAS 3 zsyrk}
\getchunk{BLAS 3 ztrmm}
\getchunk{BLAS 3 ztrsm}

\getchunk{LAPACK dbdsdc}
\getchunk{LAPACK dbdsqr}
\getchunk{LAPACK ddisna}
\getchunk{LAPACK dgebak}
\getchunk{LAPACK dgebal}
\getchunk{LAPACK dgebd2}
\getchunk{LAPACK dgebrd}
\getchunk{LAPACK dgeev}
\getchunk{LAPACK dgeevx}
\getchunk{LAPACK dgehd2}
\getchunk{LAPACK dgehrd}
\getchunk{LAPACK dgelq2}
\getchunk{LAPACK dgelqf}
\getchunk{LAPACK dgeqr2}
\getchunk{LAPACK dgeqrf}
\getchunk{LAPACK dgesdd}
\getchunk{LAPACK dgesvd}
\getchunk{LAPACK dgesv}
\getchunk{LAPACK dgetf2}
\getchunk{LAPACK dgetrf}
\getchunk{LAPACK dgetrs}
\getchunk{LAPACK dhseqr}
\getchunk{LAPACK disnan}
```

```
\getchunk{LAPACK dlabad}
\getchunk{LAPACK dlabrd}
\getchunk{LAPACK dlacon}
\getchunk{LAPACK dlacpy}
\getchunk{LAPACK dladiv}
\getchunk{LAPACK dlaed6}
\getchunk{LAPACK dlaexc}
\getchunk{LAPACK dlahqr}
\getchunk{LAPACK dlahrd}
\getchunk{LAPACK dlaisnan}
\getchunk{LAPACK dlaln2}
\getchunk{LAPACK dlamch}
\getchunk{LAPACK dlamc1}
\getchunk{LAPACK dlamc2}
\getchunk{LAPACK dlamc3}
\getchunk{LAPACK dlamc4}
\getchunk{LAPACK dlamc5}
\getchunk{LAPACK dlamrg}
\getchunk{LAPACK dlange}
\getchunk{LAPACK dlanhs}
\getchunk{LAPACK dlanst}
\getchunk{LAPACK dlanv2}
\getchunk{LAPACK dlapy2}
\getchunk{LAPACK dlapy3}
\getchunk{LAPACK dlaqtr}
\getchunk{LAPACK dlarfb}
\getchunk{LAPACK dlarfg}
\getchunk{LAPACK dlarf}
\getchunk{LAPACK dlarft}
\getchunk{LAPACK dlarfx}
\getchunk{LAPACK dlartg}
\getchunk{LAPACK dlas2}
\getchunk{LAPACK dlascl}
\getchunk{LAPACK dlasd0}
\getchunk{LAPACK dlasd1}
\getchunk{LAPACK dlasd2}
\getchunk{LAPACK dlasd3}
\getchunk{LAPACK dlasd4}
\getchunk{LAPACK dlasd5}
\getchunk{LAPACK dlasd6}
\getchunk{LAPACK dlasd7}
\getchunk{LAPACK dlasd8}
\getchunk{LAPACK dlasda}
\getchunk{LAPACK dlasdq}
\getchunk{LAPACK dlasdt}
\getchunk{LAPACK dlaset}
\getchunk{LAPACK dlasq1}
\getchunk{LAPACK dlasq2}
\getchunk{LAPACK dlasq3}
\getchunk{LAPACK dlasq4}
```

```
\getchunk{LAPACK dlasq5}
\getchunk{LAPACK dlasq6}
\getchunk{LAPACK dlasr}
\getchunk{LAPACK dlasrt}
\getchunk{LAPACK dlassq}
\getchunk{LAPACK dlasv2}
\getchunk{LAPACK dlaswp}
\getchunk{LAPACK dlasy2}
\getchunk{LAPACK dorg2r}
\getchunk{LAPACK dorgbr}
\getchunk{LAPACK dorghr}
\getchunk{LAPACK dorgl2}
\getchunk{LAPACK dorglq}
\getchunk{LAPACK dorgqr}
\getchunk{LAPACK dorm2r}
\getchunk{LAPACK dormbr}
\getchunk{LAPACK dorml2}
\getchunk{LAPACK dormlq}
\getchunk{LAPACK dormqr}
\getchunk{LAPACK dtrevc}
\getchunk{LAPACK dtrexc}
\getchunk{LAPACK dtrsna}
\getchunk{LAPACK ieeeck}
\getchunk{LAPACK ilaenv}
\getchunk{LAPACK ilazlc}
\getchunk{LAPACK ilazlr}
\getchunk{LAPACK zgebak}
\getchunk{LAPACK zgebal}
\getchunk{LAPACK zgeev}
\getchunk{LAPACK zgehd2}
\getchunk{LAPACK zgehrd}
\getchunk{LAPACK zhseqr}
\getchunk{LAPACK zlacgv}
\getchunk{LAPACK zlacpy}
\getchunk{LAPACK zladiv}
\getchunk{LAPACK zlahqr}
\getchunk{LAPACK zlahr2}
\getchunk{LAPACK zlange}
\getchunk{LAPACK zlaqr0}
\getchunk{LAPACK zlaqr1}
\getchunk{LAPACK zlaqr2}
\getchunk{LAPACK zlaqr3}
\getchunk{LAPACK zlaqr4}
\getchunk{LAPACK zlaqr5}
\getchunk{LAPACK zlarfb}
\getchunk{LAPACK zlarf}
\getchunk{LAPACK zlarfg}
\getchunk{LAPACK zlarft}
\getchunk{LAPACK zlartg}
\getchunk{LAPACK zlascl}
```

```
\getchunk{LAPACK zlaset}
\getchunk{LAPACK zlassq}
\getchunk{LAPACK zlatrs}
\getchunk{LAPACK zrot}
\getchunk{LAPACK ztrevc}
\getchunk{LAPACK ztrexc}
\getchunk{LAPACK zung2r}
\getchunk{LAPACK zunghr}
\getchunk{LAPACK zungqr}
\getchunk{LAPACK zunm2r}
\getchunk{LAPACK zunmhr}
\getchunk{LAPACK zunmqr}
\getchunk{defstruct isaacCtx}
\getchunk{defun generateNextIsaacBlock}
\getchunk{defun rand32}
\getchunk{defun randBits}
\getchunk{defmacro incfWrap32}
\getchunk{defmacro mix}
\getchunk{defun scramble}
\getchunk{defun initKernelSeed}
\getchunk{defun initNullSeed}
```

Bibliography

[Acto70] F.S. Acton. *Numerical Methods that (Usually) Work*. Harper and Row, New York, USA, 1970.

[Acto96] F.S. Acton. *Real Computing Made Real: Preventing Errors in Scientific and Engineering Calculations*. Princeton University Press, Princeton, N.J. USA, 1996, 0-691-03663-2.

[Ahre15] Peter Ahrens, Hong Diep Nguyen, and James Demmel. Efficient reproducible floating point summation and blas. technical report 229, University of California, Berkeley, December 2015.

Abstract: We define reproducibility to mean getting bitwise identical results from multiple runs of the same program, perhaps with different hardware resources or other changes that should ideally not change the answer. Many users depend on reproducibility for debugging or correctness. However, dynamic scheduling of parallel computing resources, combined with nonassociativity of floating point addition, makes attaining reproducibility a challenge even for simple operations like summing a vector of numbers, or more complicated operations like Basic Linear Algebra Subprograms (BLAS). We describe an algorithm that computes a reproducible sum of floating point numbers independent of the order of summation. The algorithm depends only on a subset of the IEEE Floating Point Standard 754-2008. It is communication-optimal, in the sense that it does just one pass over the data in the sequential case, or one reduction operation in the parallel case, requiring an "accumulator" represented by just 6 floating point words (more can be used if higher precision is desired). Th arithmetic code with a 6-word accumulator is $7n$ floating point additions to sum n words, and (in IEEEE double precision) the final error bound can be up to 10^8 times smaller than the error bound for conventional summation. We describe the basic summation algorithm, the software infrastructure used to build reproducible BLAS (ReproBLAS), and performance results. For example, when computing the dot product of 4096 double precision floating point numbers, we get a $4x$ slowdown compared to Intel

Math Kernel Library (MKL) running on an Intel Core i7-2600 CPU operating at 3.4 GHz and 256 KB L2 Cache.

Link: http://www.eecs.berkeley.edu/Pubs/TechRpts/2015/EECS-2015-229.pdf

[Ande99] E. Anderson, Z. Bai, S. Bischof, S. Blackford, J. Demmel, J. J. Dongarra, J. DuCroz, A. Greenbaum, S. Hammarling, and D. C. McKenney, A. Sorensen. *LAPACK Users' Guide*. SIAM, 1999, 0-89871-447-8.

Link: http://www.netlib.org/lapack/lug/

[Bind02] D. Bindel, J. Demmel, W. Kahan, and O. Marques. On computing givens rotations reliably and efficiently. *ACM Trans. Math. Software*, 28:206–238, 2002.

[Blac97] L. S. Blackford, A. Cleary, J. Demmel, I. Dhillon, J. J. Dongarra, S. Hammarling, A. Petitet, H. Ren, K. Stanley, and R. C. Whaley. Practical experience in the numerical dangers of heterogeneous computing. *ACM Trans. Math. Software*, 23:133–147, 1997.

[Bram02a] Karen Braman, Ralph Byers, and Roy Mathias. The multi-shift qr algorithm part i: Maintaining well focused shifts, and level 3 performance. *SIAM Journal of Matrix Analysis*, 23:929–947, 2002.

Abstract: This paper presents a small-bulge multishift variation of the multishift QR algorithm that avoids the phenomenon of shift blurring, which retards convergence and limits the number of simultaneous shifts. It replaces the large diagonal bulge in the multishift QR sweep with a chain of many small bulges. The small-bulge multishift QR sweep admits nearly any number of simultaneous shifts – even hundreds – without adverse effects on the convergence rate. With enough simultaneous shifts, the small-bulge multishift QR algorithm takes advantage of the level 3 BLAS, which is a special advantage for computers with advanced architectures.

[Bram02b] Karen Braman, Ralph Byers, and Roy Mathias. The multi-shift qr algorithm part ii: Aggressive early deflation. *SIAM Journal of Matrix Analysis*, 23:948–973, 2002.

Abstract: Aggressive early deflation is a QR algorithm strategy that takes advantage of matrix perturbations outside of the subdiagonal entries of the Hessenberg QR iterate. It identifies and deflates converged eigenvalues long before the classic small-subdiagonal strategy would. The new deflation strategy enhances the performance of conventional large-bulge multishift QR algorithms, but it is particularly effective in combination with the small-bulge multishift QR algorithm. The small-bulge multishift QR sweep with aggressive early deflation maintains a high rate of execution of floating

point operations while significantly reducing the number of operations required.

[Bran92] R. W. Brankin, I. Gladwell, and L. F. Shampine. Rksuite: A suite of runge-kutta codes for the initial value problem for odes. Technical Report Softreport 92-S1, Southern Methodist University, Dept of Math., 1992.

[Bran93] R. W. Brankin, I. Gladwell, and L. F. Shampine. Rksuite: A suite of explicit runge-kutta codes. *Contributions ot Numerical Mathematics*, pages 41–53, 1993.

[Bran97] R. W. Brankin and I. Gladwell. rksuite_90: Fortran 90 software for ordinary differential equation initial-value problems. *ACM Trans. Math. Software*, 23:402–415, 1997.

[Chai96] F. Chaitin-Chatelin and V. Fraysse. *Lectures on Finite Precision Computations*. SIAM, 1996, 0-89871-358-7.

[Chan83] T. F. Chan, G. H. Golub, and R. J. LeVeque. Algorithms for computing the sample variance: Analysis and recommendations. *The American Statistician*, 37:242–247, 1983.

[Chel06] Srinivas Chellappa, Franz Franchetti, and Markus Puschel. How to write fast numerical code: A small introduction, 2006.

Abstract: The complexity of modern computing platforms has made it extremely difficult to write numerical code that achieves the best possible performance. Straightforward implementations based on algorithms that minimize the operations count often fall short in performance by at least one order of magnitude. This tutorial introduces the reader to a set of general techniques to improve the performance of numerical code, focusing on optimizations for the computer's memory hierarchy. Further, program generators are discussed as a way to reduce the implementation and optimization effort. Two running examples are used to demonstrate these techniques: matrix-matrix multiplication and the discrete Fourier transform.

Link: https://users.ece.cmu.edu/~franzf/papers/gttse07.pdf

Algebra:
(p82) package BLAS1 BlasLevelOne

[Cool03] R. Cools and A. Haegemans. Algorithm 824: Cubpack: A package for automatic cubature; framework description. *ACM Trans. Math. Software*, 29:287–296, 2003.

[Coxx00] M. G. Cox, M. P. Dainton, and P. M. Harris. Testing spreadsheets and other packages used in metrology: Testing functions for the calculation of standard deviation. Technical Report NPL Report CMSC07/00, National Physical Lab, Teddington, Middlesex UK, 2000.

[Demm05] James Demmel, Yozo Hida, W. Kahan, Xiaoye S. Li, Soni Mukherjee, and E. Jason Riedy. Error bounds from extra precise iterative refinement. Technical Report 165, Univerity of California, Berkeley, 2005.

Abstract: We present the design and testing of an algorithm for iterative refinement of the solution of linear equations, were the residual is computed with extra precision. This algorithm was originally proposed in the 1960s as a means to compute very accurate solutions to all but the most ill-conditioned linear systems of equations. However two obstacles hae until now prevented its adoption in standard subroutine libraries like LAPACK: (1) There was no standard way to access the higher precision arithmetic needed to compute residuals, and (2) it was unclear how to compute a reliable error bound for the computed solution. The completion of the new BLAS Technical Forum Standard has recently removed the first obstacle. To overcome the second obstacle, we show how a single application of iterative refindment can be used to copute an error bound in any norm at small cost, and use this to compute both an error bound in the usual infinity norm, and a componentwise relative error bound. We report extensive test results on over 6.2 million matrices of dimension 5, 10, 100, and 1000. As long as a normwise (resp. componentwise) condition number computed by the algorithm is less than $1/max(10, \sqrt{n})\epsilon_w$, the computed normwise (resp. componentwise) error bound is at most $2max(10, \sqrt{n})\epsilon_w$, and indeed bounds the true error. Here, n is the matrix dimension and ϵ_w is a single precision roundoff error. For worse conditioned problems, we get similarly small correct error bounds in over 89.4% of cases.

Link: http://www.netlib.org/lapack/lawnspdf/lawn165.pdf

[Demm08] James Demmel, Mark Hoemmen, Yozo Hida, and E. Jason Riedy. Non-negative diagonals and high performance on low-profile matrices from householder qr. Technical Report 203, Univerity of California, Berkeley, 2008.

Abstract: The Householder reflections used in LAPACK's QR factorization leave positive and negative real entries along R's diagonal. This is sufficient for most applications of QR factorizations, but a few require that R have a non-negative diagonal. This note provides a new Householder generation routine to produce a non-negative diagonal. Additionally, we find that scanning for trailing zeros in the generated reflections leads to large performance improvements when

applying reflections with many trailing zeros. Factoring low-profile matrices, those with non-zero entries mostly near the diagonal (e.g band matrices), now requires far fewer operations. For example, QR factorization of matrices with profile width b that are stored densely in an n matrix improves form $O(n^3)$ to $O(n^2 + nb^2)$.

Link: http://www.netlib.org/lapack/lawnspdf/lawn203.pdf

[Demm88] James Demmel and W. Kahan. Computing small singular values of bidiagonal matrices with guaranteed high relative accuracy. Technical Report 326, New York University, 1988.

Abstract: Computing the singular values of a bidiagonal matrix is the final phase of the standard algorithm for the singular value decomposition of a general matrix. We present a new algorithm which computes all the singular values of a bidiagonal matrix to high relative accuracy independent of their magnitudes. In contrast, the standard algorithm for bidiagonal matrices may compute small singular values with no relative accuracy at all. Numerical experiments show that the new algorithm is comparable in speed to the standard algorithm, and frequently faster. We also show how to accurately compute tiny eigenvalues of some classes of symmetric trigiagonal matrices using the same technique.

[Dong88a] J. J. Dongarra, J. DuCroz, S. Hammarling, and R. J. Hanson. Corrigenda: 'an extended set of fortran basic linear algebra subprograms. *ACM Trans. Math. Software*, 14:399, 1988.

[Dong90] J. Dongarra, J. DuCroz, I. S. Duff, and S. Hammarling. A set of level 3 basic linear algebra subprograms. *ACM Trans. Math. Software*, 16:1–28, 1990.

[Dubr83] A. A. Dubrulle. A class of numerical methods for the computation of pythagorean sums. *IBM J. Res. Develop.*, 27(6):582–589, 1983.

[Elmr00] E. Elmroth and F. G. Gustavson. Applying recursion to serial and parallel qr factorization leads to better performance. *IBM Journal of Research and Development*, 44(4):605–624, July 2000.

Abstract: We present new recursive serial and parallel algorithms for QR factorization of an m by n matrix. They improve performance. The recursion leads to an automatic variable blocking, and it also replaces a Level 2 part in a standard aglorithm with Level 3 operations. However, there are significant additional costs for creating and performing the updates, which prohibit the efficient use of the recursion for large n. We present a quantitative analysis of these extra costs. This analysis leads us to introduce a hybrid recursive algorithm that outperforms the LAPACK algorithm DGEQRF by about 20% for large square matrices up to almost a factor of 3 for

tall thin matrices. Uniprocessor performance results are presented for two IBM RS/6000 SP nodes – a 120-Mhz IBM POWER2 node and one processor of a four-way 332-Mhz IBM PowerPC 604e SMP node. The hybrid recursive algorithm reaches more than 90% of the theoretical peak performance of the POWER2 node. Compared to standard block algorithms, the recursive approach also shows a significant advantage in the automatic tuning obtained from its automatic variable blocking. A successful parallel implementation on a four-way 332-MHz IBM PPC604e SMP node based on dynamic load balancing is presented. For two, three, and four processors it shows speedups of up to 1.97, 299, and 3.97.

Link: http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.33.1820

[Fern92] K. Vince Fernando and Beresford N. Parlett. Accurate singular values and differential qd algorithms. Technical Report PAM-554, University of California, Berkeley, 1992.

Abstract: We have discovered a new implementaiton of the qd algorithm that has a far wider domain of stability than Rutishauser's version. Our algorithm was developed from an examination of the LR-Cholesky transformation and can be adapted to parallel computation in stark contrast to traditional qd. Our algorithm also yields useful a posteriori upper and lower bounds on the smallest singular value of a bidiagonal matrix. The zero-shift bidiagonal QR of Demmel and Kahan computes the smallest singlar values to maximal relative accuracy and the others to maximal absolute accuracy with little or no degradation in efficiency when compared with the LINPACK code. Our algorithm obtains maximal relative accuracy for all the singluar values and runs at least four times faster than the LINPACK code.

Link: http://www.dtic.mil/dtic/tr/fulltext/u2/a256582.pdf

[Fors69] G. E. Forsythe. What is a satisfactory quadratic equation solver. In *Constructive Aspects of the Fundamental Theorem of Algebra*, pages 53–61. Wiley, 1969.

[Fors70] G. E. Forsythe. Pitfalls in computations, or why a math book isn't enough. *Amer. Math. Monthly*, 9:931–995, 1970.

[Foxx71] L. Fox. How to get meaningless answers in scientific computations (and what to do about it). *IMA Bulletin*, 7:296–302, 1971.

[Give54] W. Givens. Numerical computation of the characteristic values of a real symmetric matrix. Technical Report ORNL-1574, Oak Ridge National Laboratory, 1954.

[Golu65] G.H. Golub. Numerical methods for solving linear least squares problems. *Numer. Math.*, 7:206–216, 1965.

[Golu96] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 1996, 978-0-8018-5414-9.

[Gust16] John Gustafson. A radical approach to computation with real numbers.

Abstract: If we are willing to give up compatibility with IEEE 754 floats and design a number format with goals appropriate to 2016, we can achieve several goals simultaneously: Extremely high energy efficiency and information-per-bit, no penalty for decimal operations instead of binary, rigorous bounds on answers without the overly pessimistic bounds produced by interval methods, and unprecedented high speed up to some precision. This approach extends the ideas of unum arithmetic introduced two years ago by breaking completely from the IEEE float-type format, resulting in fixed bit size values, fixed execution time, no exception values or 'gradual underflow' issues, no wasted bit patterns, and no redundant representations (like 'negative zero'). As an example of the power of this format, a difficult 12-dimensional nonlinear robotic kinematics problem that has defied solvers to date is quickly solvable with absolute bounds. Also unlike interval methods, it becomes possible to operate on arbitrary disconnected subsets of the real number line with the same speed as operating on a simple bound.

Link: http://www.johngustafson.net/presentations/Multicore2016-JLG.pdf

[Hamm05] Sven Hammarling. *An Introduction to the Quality of Computed Solutions*. SIAM, 2005.

Link: http://eprints.ma.man.ac.uk/101/

[Harg02] G. Hargreaves. Interval analysis in matlab. Master's thesis, University of Manchester, Dept. of Mathematics, 2002.

[High02] Nicholas J. Higham. *Accuracy and stability of numerical algorithms*. SIAM, 2002, 0-89871-521-0.

[High05] D. J. Higham and N. J. Higham. *MATLAB Guide*. SIAM, 2002, 0-89871-521-0.

[High86] Nicholas J. Higham. Efficient algorithms for computing the condition number of a tridiagonal matrix. *SIAM J. Sci. Stat. Comput.*, 7(1), January 1986.

Abstract: Let A be a tridiagonal matrix of order n. We show that it is possible to compute $||A^{-1}||_\infty$ and hence $\text{cond}_\infty(A)$ in $O(n)$ operations. Several algorithms which perform this task are given and

their numerical properties are investigated. If A is also positive definite then $||A^{-1}||_\infty$ can be computed as the norm of the solution to a positive definite tridiagonal linear system whose coefficient matrix is closely related to A. We show how this computation can be carried out in parallel with the solution of a linear system $Ax = b$. In particular we describe some simple modifications to the LINPACK routine SPTSL which enable this routine to compute $\text{cond}_1(A)$, efficiently, in addtion to solving $Ax = b$.

[High98] Nicholas J. Higham. Can you 'count' on your computer?, 1998.

Link: http://www.maths.man.ac.uk/~higham/talks/

[Hous81] Alston S. Householder. *Principles of Numerical Analysis*. Dover Publications, Mineola, NY, 1981, 0-486-45312-X.

[IEEE85] IEEE. Ansi/ieee standard for binary floating point arithmetic: Std 754-1985, 1985.

[IEEE87] IEEE. Ansi/ieee standard for radix independent floating point arithmetic: Std 854-1987, 1987.

[Isaa94] E. Isaacson and H. B. Keller. *Analysis of Numerical Methods*. Dover, 1994, 0-486-68029-0.

[Knus98] L. Knüsel. On the accuracy of statistical distributions in microsoft excel 97. *Comput. Statist. Data Anal.*, 26:375–377, 1998.

[Krei05] V. Kreinovich. Interval cmoputations, 2005.

Link: http://www.cs.utep.edu/interval-comp/

[LAPA99] Anderson E. et al. Lapack user's guide third addition, August 1999.

Link: http://www.netlib.org/lapack/lug/

[Laws79] C.L. Lawson, Hanson R.J., D.R. Kincaid, and F.T. Krogh. Algorithm 539: Basic linear algebra subprograms for fortran usage. *ACM Transactions on Mathematical Software*, 5(3):308–323, September 1979.

[Laws79a] C. L. Lawson, R. J. Hanson, D.R. Kincaid, and F.T. Krogh. Basic linear algebra subprograms for fortran usage. *Transactions on Mathematical Software*, 5(3):308–325, September 1979.

Abstract: A package of 38 low level subprograms for many of the basic operations of numerical linear algebra is presented. The package is intended to be used with Fortran. The operations in the package include dot product, elementary vector operations, Givens transformations, vector copy and swap, vector norm, vector scaling, and the determination of the index of the vector component of largest magnitude.

[Laws95] C. L. Lawson and R. J. Hanson. *Solving Least Squares Problems*. SIAM, 1995, 0-89871-356-0.

[Mart68] R. S. Martin and J. H. Wilkinson. Similarity reduction ofa general matrix to hessenberg form. *Numer. Math.*, 12:349–368, 1968.

[Math05] MathWorks. Matlab.

Link: http://www.mathworks.com

[Mccu02] B. D. McCullough and B. Wilson. On the accuracy of statistical procedures in microsoft excel 2000 and excel xp. *Comput. Statist. Data Anal.*, 40:713–721, 2002.

[Mccu99] B. D. McCullough and B. Wilson. On the accuracy of statistical procedures in microsoft excel 97. *Comput. Statist. Data Anal.*, 31:27–37, 1999.

[Metc04] M. Metcalf, J. K. Reid, and M. Cohen. *Fortran 95/2003 Explained*. Oxford University Press, 2004, 0-19-852693-8.

[Metc96] M. Metcalf and J. K. Reid. *Fortran 90/95 Explained*. Oxford University Press, 1996.

[Mole71] C.B. Moler and G.W. Stewart. An algorithm for generalized matrix eigenvalue problems. *SIAM J. Numer. Anal*, 10:241–256, 1973.

Abstract: A new method, called the QZ algorithm, is presented for the solution of the matrix eigenvalue problem $Ax = \lambda Bx$ with general square matrices A and B. Particluar attention is paid to the degeracies which result when B is singular. No inversions of B or its submatrices are used. The algorithm is a generalization of the QR algorithm, and reduces to it when $B = I$. A Fortran program and some illustrative examples are included.

[Mole83] C. Moler and D. Morrison. Replacing square roots by pythagorena sums. *IBM J. Res. Develop.*, 27(6):577–581, 1983.

[Moor79] R. E. Moore. *methods and Applications of Interval Analysis*. SIAM, 1979.

[NAGa05] Numerical Algorithms Group. The nag library, 2005.

Link: http://www.nag.co.uk/numeric

[NAGb05] Numerical Algorithms Group. The nag fortran library manual, 2005.

Link: http://www.nag.co.uk/numeric/fl/manual/html/FLlibrarymanual.asp

[Over01] M. L. Overton. *Numerical Computing with IEEE Floating Point Arithmetic*. SIAM, 2001, 0-89871-482-6.

[Pies83] R. Piessens, E. de Doncker-Kapenga, C. W. Überhuber, and D. K. Kahaner. *QUADPACK - A Subroutine Package for Automatic Integration.* Springer-Verlag, 1983.

[Prie04] D. M. Priest. Efficient scaling for complex division. *ACM Trans. Math. Software*, 30:389–401, 2004.

[Rump99] S. M. Rump. Intlab - interval laboratory. In *Developments in Reliable Computing*, pages 77–104. Kluwer Academic, 1999.

[Sham92] L. F. Shampine and I. Gladwell. The next generation of runge-kutta codes. In *Computational Ordinary Differential Equations*, pages 145–164. Oxford University Press, 1992.

[Smit62] R. L. Smith. Algorithm 116: Complex division. *Communs. Ass. comput. Mach.*, 5:435, 1962.

[Stew85] G. W. Stewart. A note on complex division. *ACM Trans. Math. Software*, 11:238–241, 1985.

[Stew90] G. W. Stewart and J. Sun. *Matrix Perturbation Theory.* Academic Press, 1990.

[Stew98] G. W. Stewart. *Matrix Algorithms: Basic Decompositions, volume I.* SIAM, 1998, 0-89871-414-1.

[Sutt13] Brian D. Sutton. Computing the complete cs decomposition. *Numerical Algorithms*, 50:33–65, February 2013.

Abstract: An algorithm is developed to compute the complete CS decomposition (CSD) of a partitioned unitary matrix. Although the existence of the CSD has been recognized since 1977, prior algorithms compute only a reduced version (the 2-by-1 CSD) that is equivalent to two simultaneous singular value decompositions. The algorithm presented here computes the complete 2-by-2 CSD, which requires the simultaneous diagonalization of all four blocks of a unitary matrix partitioned into a 2-by-2 block structure. The algorithm appears to be the only fully specified algorithm available. The computation occurs in two phases. In the first phase, the unitary matrix is reduced to bidiagonal block form, as described by Sutton and Edelman. In the second phase, the blocks are simultaneously diagonalized using techniques from bidiagonal SVD algorithms of Golub, Kahan, and Demmel. The algorithm has a number of desirable numerical features.

Link: http://arxiv.org/pdf/0707.1838v3.pdf

[Turi48] A. M. Turing. Rounding-off errors in matrix processes. *Q. J. Mech. Appl. Math.*, 1:287–308, 1948.

[Vign93] J. Vignes. A stochastic arithmetic for reliable scientific computation. *Math. and Comp. in Sim.*, 25:233–261, 1993.

[Wilk60] J. H. Wilkinson. Error analysis of floating-point computation. *Numer. Math.*, 2:319–340, 1960.

[Wilk61] J. H. Wilkinson. Error analysis of diret methods of matrix inversion. *J. ACM*, 8:281–330, 1961.

[Wilk63] J. H. Wilkinson. *Rounding Erroors in Algebraic Processes.* Notes on Applied Science, No. 32. HMSO, 1963.

[Wilk65] J. H. Wilkinson. *The Algebraic Eigenvalue Problem.* Oxford University Press, 1965.

[Wilk71] J. H. Wilkinson. *Handbook for Automatic Computation, V2, Linear Algebra.* Springer-Verlag, 1971.

[Wilk84] J. H. Wilkinson. The perfidious polynomial. In *Studies in Numerical Analysis*, volume 24, pages 1–28, 1984.

[Wilk85] J. H. Wilkinson. The state of the art in error analysis. *NAG Newsletter*, 2/85:5–28, 1985.

[Wilk86] J. H. Wilkinson. Error analysis revisited. *IMA Bulletin*, 22:192–200, 1986.

[blas01] Jack Dongarra and et al. Basic linear algebra subprograms technical (blast) forum standard, 2001.

Link: http://www.netlib.org/blas/blast-forum/blas-report.pdf

Index

BLAS1 Spad code, 82
BlasLevelOne, 82

caxpy.f, 346
cbbcsd.f, 13632
cbdsqr.f, 13714
ccopy.f, 350
cdotc.f, 353
cdotu.f, 356
cgbbrd.f, 8002, 13764
cgbcon.f, 8013
cgbequ.f, 8024
cgbequb.f, 8035
cgbmv.f, 818
cgbrfs.f, 8047
cgbrfsx.f, 8069
cgbsv.f, 7426
cgbsvx.f, 7435
cgbsvxx.f, 7461
cgbtf2.f, 8078
cgbtrf.f, 8087
cgbtrs.f, 8110
cgebak.f, 6173
cgebal.f, 6184
cgebd2.f, 6199
cgebrd.f, 6211
cgecon.f, 6222
cgeequ.f, 6231
cgeequb.f, 6241
cgees.f, 2896
cgeesx.f, 2905
cgeev.f, 2914
cgeevx.f, 2938
cgehd2.f, 6251
cgehrd.f, 6258
cgejsv.f, 3980
cgelq2.f, 6271
cgelqf.f, 6277
cgels.f, 2158
cgelsd.f, 2179
cgelss.f, 2214
cgelsy.f, 2258
cgemm.f, 1486
cgemqrt.f, 6286
cgemv.f, 832
cgeql2.f, 6296
cgeqlf.f, 6302
cgeqp3.f, 6312
cgeqr2.f, 6324
cgeqr2p.f, 6330
cgeqrf.f, 6336
cgeqrfp.f, 6345
cgeqrt.f, 6354
cgeqrt2.f, 6361
cgeqrt3.f, 6368
cgerc.f, 843
cgerfs.f, 6379
cgerfsx.f, 6399
cgerq2.f, 6408
cgerqf.f, 6415
cgeru.f, 849
cgesc2.f, 7237
cgesdd.f, 4077
cgesv.f, 2276
cgesvd.f, 4227
cgesvdx.f, 4429
cgesvj.f, 6427
cgesvx.f, 2285
cgesvxx.f, 2309
cgetc2.f, 7243
cgetf2.f, 6500
cgetrf.f, 6507
cgetrf2.f, 6515
cgetri.f, 6524

cgetrs.f, 6534
cggbak.f, 8121
cggbal.f, 8133
cgges.f, 2965
cgges3.f, 2977
cggesx.f, 2989
cggev.f, 3001
cggev3.f, 3025
cggevx.f, 3052
cgtcon.f, 12269
cgtrfs.f, 12278
cgtsv.f, 12117
cgtsvx.f, 12128
cgttrf.f, 12296
cgttrs.f, 12304
cgtts2.f, 12311
char-equal
    calledby dbdsdc, 13800
chbmv.f, 857
chemm.f, 1505
chemv.f, 869
cher.f, 881
cher2.f, 892
cher2k.f, 1519
cherk.f, 1538
chgeqz.f, 6544
chpmv.f, 904
chpr.f, 916
chpr2.f, 928
cla_gbamv.f, 8162
cla_gbrcond_c.f, 8177
cla_gbrcond_x.f, 8190
cla_gbrfsx_extended.f, 8206
cla_gbrpvgrw.f, 8223
cla_geamv.f, 6585
cla_gercond_c.f, 6598
cla_gercond_x.f, 6610
cla_gerfsx_extended.f, 6624
cla_gerpvgrw.f, 6641
cla_porcond_c.f, 11775
cla_porcond_x.f, 11787
cla_porfsx_extended.f, 11803
cla_porpvgrw.f, 11819
cla_syamv.f, 10088
cla_syrcond_c.f, 10104
cla_syrcond_x.f, 10116
cla_syrfsx_extended.f, 10131
cla_syrpvgrw.f, 10149
claesy.f, 11187
clangb.f, 8533
clange.f, 7251
clansy.f, 11193
claqgb.f, 8543
claqge.f, 7259
claqsy.f, 11203
clasyf.f, 10164
clasyf_rook.f, 10199
Condition Number, 1799
cpocon.f, 11827
cpoequ.f, 11835
cpoequb.f, 11841
cporfs.f, 11848
cporfsx.f, 11869
cposv.f, 11415
cposvx.f, 11424
cposvxx.f, 11442
cpotf2.f, 11877
cpotrf.f, 11886
cpotrf2.f, 11897
cpotri.f, 11905
cpotrs.f, 11910
cptcon.f, 12541
cpteqr.f, 12548
cptrfs.f, 12556
cptsv.f, 12417
cptsvx.f, 12423
cpttrf.f, 12574
cpttrs.f, 12582
cptts2.f, 12589
crotg.f, 359
cscal.f, 362
csrot.f, 366
csscal.f, 369
cswap.f, 372
csycon.f, 10240
csycon_rook.f, 10247
csyconv.f, 10254
csyequb.f, 10269
csymm.f, 1555
csymv.f, 11210
csyr.f, 11222
csyr2k.f, 1569

csyrfs.f, 10286
csyrfsx.f, 10307
csyrk.f, 1583
csysv.f, 8628
csysv_rook.f, 8636
csysvx.f, 8645
csysvxx.f, 8661
csyswapr.f, 11230
csytf2.f, 10317
csytf2_rook.f, 10340
csytrf.f, 10371
csytrf_rook.f, 10383
csytri.f, 10393
csytri2.f, 10411
csytri2x.f, 10417
csytri_rook.f, 10450
csytrs.f, 10471
csytrs2.f, 10492
csytrs_rook.f, 10507
ctbmv.f, 942
ctbsv.f, 961
ctgevc.f, 6647
ctgex2.f, 7266
ctgexc.f, 6677
ctgsy2.f, 11236
ctgsyl.f, 10532
ctpmv.f, 980
ctpsv.f, 998
ctrmm.f, 1597
ctrmv.f, 1016
ctrsm.f, 1618
ctrsv.f, 1033
ctrsyl.f, 10559
cungbr.f, 8229

dasum.f, 168
daxpy.f, 180
dbdsdc
    calls char-equal, 13800
    calls dcopy, 13800
    calls dlamch, 13800
    calls dlanst, 13800
    calls dlartg, 13800
    calls dlascl, 13800
    calls dlasd0, 13800
    calls dlasda, 13800
    calls dlasdq, 13800
    calls dlaset, 13800
    calls dlasr, 13800
    calls dswap, 13800
    calls ilaenv, 13800
    calls xerbla, 13800
dbdsdc.f, 13800
dbdsqr.f, 13829
dcabs1.f, 188
dcopy
    calledby dbdsdc, 13800
dcopy.f, 196
ddisna.f, 13879
ddot.f, 205
defmacro
    incfWrap32, 16017
    mix, 16018
defstruct
    isaacCtx, 16014
defun
    generateNextIsaacBlock, 16016
    initCommonLispRandomSeed, 16022
    initKernelSeed, 16021
    initNullSeed, 16022
    rand32, 16017
    randBits, 16017
    scramble, 16020
dgbbrd.f, 7534
dgbcon.f, 7558
dgbequ.f, 7569
dgbequb.f, 7580
dgbmv.f, 614
dgbrfs.f, 7591
dgbrfsx.f, 7613
dgbsv.f, 7318
dgbsvx.f, 7327
dgbsvxx.f, 7353
dgbtf2.f, 7622
dgbtrf.f, 7631
dgbtrs.f, 7654
dgebak.f, 4955, 13890
dgebal.f, 4961, 13902
dgebd2.f, 4969, 13921
dgebrd.f, 4976, 13934
dgecon.f, 4982
dgeequ.f, 4991

dgeequb.f, 5000
dgees.f, 2511
dgeesx.f, 2522
dgeev.f, 2533, 13947
dgeevx.f, 2545, 13979
dgehd2.f, 5010, 14014
dgehrd.f, 5015, 14022
dgejsv.f, 3279
dgelq2.f, 5021, 14036
dgelqf.f, 5025, 14043
dgels.f, 1812
dgelsd.f, 1833
dgelss.f, 1868
dgelsy.f, 1910
dgemm.f, 1390
dgemqrt.f, 5031
dgemv.f, 626
dgeql2.f, 5041
dgeqlf.f, 5047
dgeqp3.f, 5057
dgeqr2.f, 5069, 14054
dgeqr2p.f, 5075
dgeqrf.f, 5081, 14061
dgeqrfp.f, 5090
dgeqrt.f, 5099
dgeqrt2.f, 5105
dgeqrt3.f, 5114
dger.f, 637
dgerfs.f, 5124
dgerfsx.f, 5144
dgerq2.f, 5153
dgerqf.f, 5159
dgesc2.f, 7103
dgesdd.f, 3376, 14073
dgesv.f, 1929, 14466
dgesvd.f, 3404, 14202
dgesvdx.f, 3496
dgesvj.f, 5171
dgesvx.f, 1936
dgesvxx.f, 1960
dgetc2.f, 7109
dgetf2.f, 5269, 14472
dgetrf.f, 5273, 14480
dgetrf2.f, 5281
dgetri.f, 5290
dgetrs.f, 5300, 14490
dggbak.f, 7663
dggbal.f, 7675
dgges.f, 2558
dgges3.f, 2571
dggesx.f, 2585
dggev.f, 2599
dggev3.f, 2625
dggevx.f, 2654
dgtcon.f, 12161
dgtrfs.f, 12170
dgtsv.f, 12061
dgtsvx.f, 12079
dgttrf.f, 12187
dgttrs.f, 12195
dgtts2.f, 12202
dhgeqz.f, 5307
dhseqr.f, 14500
disnan.f, 12656, 14519
dla_gbamv.f, 7702
dla_gbrcond.f, 7717
dla_gbrfsx_extended.f, 7735
dla_gbrpvgrw.f, 7752
dla_geamv.f, 5377
dla_gercond.f, 5390
dla_gerfsx_extended.f, 5406
dla_gerpvgrw.f, 5422
dla_porcond.f, 11513
dla_porfsx_extended.f, 11531
dla_porpvgrw.f, 11547
dla_syamv.f, 8928
dla_syrcond.f, 8943
dla_syrfsx_extended.f, 8961
dla_syrpvgrw.f, 8978
dlabad.f, 12658, 14522
dlabrd.f, 14527
dlacon.f, 14550
dlacpy.f, 12661, 14562
dladiv.f, 12664, 14567
dlae2.f, 12670
dlaebz.f, 12678
dlaed6.f, 14574
dlaev2.f, 12697
dlaexc.f, 14593
dlagts.f, 12704
dlahqr.f, 14619
dlahrd.f, 14651

dlaisnan.f, 12717, 14659
dlaln2.f, 14663
dlamc1.f, 12719, 14694
dlamc2.f, 12721, 14701
dlamc3.f, 12722, 14710
dlamc4.f, 14713
dlamc5.f, 14717
dlamch
    calledby dbdsdc, 13800
dlamch.f, 12724, 14687
dlamrg.f, 14723
dlaneg.f, 12727
dlangb.f, 8497
dlange.f, 7117, 14727
dlanhs.f, 14734
dlanst
    calledby dbdsdc, 13800
dlanst.f, 12735, 14742
dlansy.f, 10995
dlanv2.f, 14748
dlapy2.f, 12738, 14755
dlapy3.f, 12741, 14758
dlaqgb.f, 8507
dlaqge.f, 7121
dlaqsy.f, 11005
dlaqtr.f, 14762
dlarf.f, 14833
dlarfb.f, 14796
dlarfg.f, 14828
dlarft.f, 14840
dlarfx.f, 14851
dlarnv.f, 12744
dlarra.f, 12749
dlarrb.f, 12756
dlarrc.f, 12767
dlarrd.f, 12777
dlarre.f, 12807
dlarrf.f, 12840
dlarrj.f, 12856
dlarrk.f, 12868
dlarrr.f, 12874
dlartg
    calledby dbdsdc, 13800
dlartg.f, 12880, 14892
dlartgp.f, 12885
dlaruv.f, 12891
dlas2.f, 12908, 14897
dlascl
    calledby dbdsdc, 13800
dlascl.f, 12912, 14902
dlasd0
    calledby dbdsdc, 13800
dlasd0.f, 12920, 14915
dlasd1.f, 12927, 14926
dlasd2.f, 12935, 14936
dlasd3.f, 12946, 14956
dlasd4.f, 12954, 14975
dlasd5.f, 12974, 15021
dlasd6.f, 12982, 15032
dlasd7.f, 12990, 15042
dlasd8.f, 12999, 15058
dlasda
    calledby dbdsdc, 13800
dlasda.f, 13007, 15072
dlasdq
    calledby dbdsdc, 13800
dlasdq.f, 13016, 15092
dlasdt.f, 13022, 15104
dlaset
    calledby dbdsdc, 13800
dlaset.f, 13026, 15110
dlasq1.f, 15115
dlasq2.f, 15123
dlasq3.f, 15152
dlasq4.f, 15168
dlasq5.f, 15184
dlasq6.f, 15201
dlasr
    calledby dbdsdc, 13800
dlasr.f, 13031, 15213
dlasrt.f, 15231
dlassq.f, 13038, 15241
dlasv2.f, 13042, 15246
dlaswp.f, 15253
dlasy2.f, 11012, 15259
dlasyf.f, 8992
dlasyf_rook.f, 9026
dnrm2.f, 212
dorg2r.f, 15279
dorgbr.f, 7758, 15285
dorghr.f, 15295
dorgl2.f, 15302

dorglq.f, 15308
dorgqr.f, 15318
dorm2r.f, 15328
dormbr.f, 15336
dorml2.f, 15347
dormlq.f, 15355
dormqr.f, 15366
dpocon.f, 11554
dpoequ.f, 11563
dpoequb.f, 11569
dporfs.f, 11576
dporfsx.f, 11596
dposv.f, 11319
dposvx.f, 11328
dposvxx.f, 11346
dpotf2.f, 11605
dpotrf.f, 11613
dpotrf2.f, 11623
dpotri.f, 11631
dpotrs.f, 11636
dptcon.f, 12449
dpteqr.f, 12456
dptrfs.f, 12464
dptsv.f, 12389
dptsvx.f, 12395
dpttrf.f, 12477
dpttrs.f, 12484
dptts2.f, 12489
drot.f, 223
drotg.f, 239
drotm.f, 248
drotmg.f, 255
dsbmv.f, 644
dscal.f, 264
dsdot.f, 270
dsecnd.f, 13047
dsgesv.f, 1977
dspmv.f, 657
dsposv.f, 11359
dspr.f, 669
dspr2.f, 679
dswap
    calledby dbdsdc, 13800
dswap.f, 276
dsycon.f, 9065
dsycon_rook.f, 9072
dsyconv.f, 9080
dsyequb.f, 9095
dsyev.f, 8723
dsyevd.f, 8734
dsyevr.f, 8748
dsyevx.f, 8770
dsygs2.f, 9110
dsygst.f, 9124
dsygv.f, 8790
dsygvd.f, 8801
dsygvx.f, 8813
dsymm.f, 1404
dsymv.f, 690
dsyr.f, 702
dsyr2.f, 711
dsyr2k.f, 1418
dsyrfs.f, 9143
dsyrfsx.f, 9163
dsyrk.f, 1433
dsysv.f, 8570
dsysv_rook.f, 8578
dsysvx.f, 8587
dsyswapr.f, 11020
dsytd2.f, 9173
dsytf2.f, 9184
dsytf2_rook.f, 9207
dsytrd.f, 9228
dsytrf.f, 9241
dsytrf_rook.f, 9252
dsytri.f, 9262
dsytri2.f, 9280
dsytri2x.f, 9286
dsytri_rook.f, 9319
dsytrs.f, 9340
dsytrs2.f, 9360
dsytrs_rook.f, 9376
dtbmv.f, 723
dtbsv.f, 740
dtgevc.f, 5429
dtgex2.f, 7129
dtgexc.f, 5489
dtgsy2.f, 11027
dtgsyl.f, 9400
dtpmv.f, 756
dtpsv.f, 772
dtrevc.f, 15378

dtrexc.f, 15434
dtrmm.f, 1447
dtrmv.f, 788
dtrsm.f, 1465
dtrsna.f, 15451
dtrsv.f, 283, 802
dtrsyl.f, 9430
dzasum.f, 297
dznrm2.f, 304

Euclidean Norm, 1800

generateNextIsaacBlock, 16016
    defun, 16016

icamax.f, 312
idamax.f, 322
ieeeck.f, 13049, 15472
iladlc.f, 13053
iladlr.f, 13057
ilaenv
    calledby dbdsdc, 13800
ilaenv.f, 13061, 15479
ilaver.f, 13073
ilazlc.f, 15496
ilazlr.f, 15500
incfWrap32, 16017
    defmacro, 16017
initCommonLispRandomSeed, 16022
    defun, 16022
initKernelSeed, 16021
    defun, 16021
initNullSeed, 16022
    defun, 16022
iparmq.f, 13078
isaacCtx, 16014
    defstruct, 16014
isamax.f, 331
izamax.f, 338

lsame.f, 89, 13086
lsamen.f, 13090

mix, 16018
    defmacro, 16018

rand32, 16017
    defun, 16017
randBits, 16017
    defun, 16017

sasum.f, 96
saxpy.f, 100
scabs1.f, 104
scasum.f, 106
scnrm2.f, 110
scopy.f, 113
scramble, 16020
    defun, 16020
sdot.f, 118
sdsdot.f, 122
second.f, 13093
sgbbrd.f, 7766
sgbcon.f, 7790
sgbequ.f, 7801
sgbequb.f, 7812
sgbmv.f, 430
sgbrfs.f, 7823
sgbrfsx.f, 7845
sgbsv.f, 7372
sgbsvx.f, 7381
sgbsvxx.f, 7407
sgbtf2.f, 7854
sgbtrf.f, 7862
sgbtrs.f, 7885
sgebak.f, 5583
sgebal.f, 5594
sgebd2.f, 5608
sgebrd.f, 5619
sgecon.f, 5630
sgeequ.f, 5639
sgeequb.f, 5648
sgees.f, 2689
sgeesx.f, 2700
sgeev.f, 2711
sgeevx.f, 2738
sgehd2.f, 5658
sgehrd.f, 5665
sgejsv.f, 3539
sgelq2.f, 5677
sgelqf.f, 5683
sgels.f, 1992
sgelsd.f, 2013

sgelss.f, 2047
sgelsy.f, 2089
sgemm.f, 1302
sgemqrt.f, 5693
sgemv.f, 441
sgeql2.f, 5702
sgeqlf.f, 5708
sgeqp3.f, 5718
sgeqr2.f, 5731
sgeqr2p.f, 5736
sgeqrf.f, 5742
sgeqrfp.f, 5751
sgeqrt.f, 5760
sgeqrt2.f, 5767
sgeqrt3.f, 5774
sger.f, 451
sgerfs.f, 5785
sgerfsx.f, 5805
sgerq2.f, 5814
sgerqf.f, 5820
sgesc2.f, 7171
sgesdd.f, 3635
sgesv.f, 2108
sgesvd.f, 3736
sgesvdx.f, 3937
sgesvj.f, 5832
sgesvx.f, 2117
sgesvxx.f, 2141
sgetc2.f, 7177
sgetf2.f, 5929
sgetrf.f, 5936
sgetrf2.f, 5944
sgetri.f, 5953
sgetrs.f, 5963
sggbak.f, 7895
sggbal.f, 7906
sgges.f, 2769
sgges3.f, 2782
sggesx.f, 2796
sggev.f, 2809
sggev3.f, 2835
sggevx.f, 2864
sgtcon.f, 12215
sgtrfs.f, 12224
sgtsv.f, 12089
sgtsvx.f, 12107
sgttrf.f, 12241
sgttrs.f, 12249
sgtts2.f, 12256
shgeqz.f, 5973
sisnan.f, 13094
sla_gbamv.f, 7934
sla_gbrcond.f, 7948
sla_gbrfsx_extended.f, 7967
sla_gbrpvgrw.f, 7983
sla_geamv.f, 6042
sla_gercond.f, 6055
sla_gerfsx_extended.f, 6072
sla_gerpvgrw.f, 6088
sla_porcond.f, 11645
sla_porfsx_extended.f, 11663
sla_porpvgrw.f, 11678
sla_syamv.f, 9506
sla_syrcond.f, 9521
sla_syrfsx_extended.f, 9539
sla_syrpvgrw.f, 9556
slabad.f, 13097
slacpy.f, 13100
sladiv.f, 13105
slae2.f, 13114
slaebz.f, 13122
slaev2.f, 13141
slag2d.f, 13147
slagts.f, 13152
slaisnan.f, 13164
slamc1.f, 13167
slamc2.f, 13169
slamc3.f, 13170
slamch.f, 13172
slaneg.f, 13174
slangb.f, 8515
slange.f, 7185
slanst.f, 13182
slansy.f, 11085
slapy2.f, 13188
slapy3.f, 13191
slaqgb.f, 8524
slaqge.f, 7192
slaqsy.f, 11095
slarnv.f, 13195
slarra.f, 13200
slarrb.f, 13207

slarrc.f, 13218
slarrd.f, 13228
slarre.f, 13258
slarrf.f, 13290
slarrj.f, 13307
slarrk.f, 13319
slarrr.f, 13325
slartg.f, 13331
slartgp.f, 13337
slaruv.f, 13343
slas2.f, 13360
slascl.f, 13365
slasd0.f, 13378
slasd1.f, 13390
slasd2.f, 13401
slasd3.f, 13422
slasd4.f, 13442
slasd5.f, 13490
slasd6.f, 13502
slasd7.f, 13513
slasd8.f, 13530
slasda.f, 13545
slasdq.f, 13566
slasdt.f, 13579
slaset.f, 13585
slasr.f, 13592
slassq.f, 13611
slasv2.f, 13616
slasy2.f, 11101
slasyf.f, 9570
slasyf_rook.f, 9604
snrm2.f, 127
sorgbr.f, 7989
spocon.f, 11686
spoequ.f, 11695
spoequb.f, 11700
sporfs.f, 11707
sporfsx.f, 11728
sposv.f, 11375
sposvx.f, 11384
sposvxx.f, 11402
spotf2.f, 11736
spotrf.f, 11744
spotrf2.f, 11755
spotri.f, 11763
spotrs.f, 11768
sptcon.f, 12495
spteqr.f, 12502
sptrfs.f, 12510
sptsv.f, 12403
sptsvx.f, 12409
spttrf.f, 12523
spttrs.f, 12530
sptts2.f, 12535
srot.f, 130
srotg.f, 134
srotm.f, 137
srotmg.f, 144
ssbmv.f, 458
sscal.f, 151
sspmv.f, 470
sspr.f, 482
sspr2.f, 490
sswap.f, 155
ssycon.f, 9643
ssycon_rook.f, 9650
ssyconv.f, 9657
ssyequb.f, 9672
ssyev.f, 8825
ssyevd.f, 8836
ssyevr.f, 8850
ssyevx.f, 8872
ssygs2.f, 9688
ssygst.f, 9701
ssygv.f, 8892
ssygvd.f, 8902
ssygvx.f, 8914
ssymm.f, 1315
ssymv.f, 500
ssyr.f, 511
ssyr2.f, 519
ssyr2k.f, 1328
ssyrfs.f, 9720
ssyrfsx.f, 9741
ssyrk.f, 1341
ssysv.f, 8600
ssysv_rook.f, 8608
ssysvx.f, 8617
ssyswapr.f, 11121
ssytd2.f, 9751
ssytf2.f, 9762
ssytf2_rook.f, 9784

ssytrd.f, 9814
ssytrf.f, 9827
ssytrf_rook.f, 9838
ssytri.f, 9848
ssytri2.f, 9866
ssytri2x.f, 9872
ssytri_rook.f, 9905
ssytrs.f, 9926
ssytrs2.f, 9946
ssytrs_rook.f, 9962
stbmv.f, 530
stbsv.f, 545
stgevc.f, 6095
stgex2.f, 7200
stgexc.f, 6153
stgsy2.f, 11128
stgsyl.f, 9986
stpmv.f, 559
stpsv.f, 573
strmm.f, 1354
strmv.f, 587
strsm.f, 1371
strsv.f, 600
strsyl.f, 10013

Tridiagonal matrices, 1799
tstiee.f, 13623

xerbla
    calledby dbdsdc, 13800
xerbla.f, 92
xerbla_array.f, 13625

zaxpy.f, 384
zcgesv.f, 2326
zcopy.f, 395
zcposv.f, 11457
zdotc.f, 399
zdotu.f, 404
zdrot.f, 408
zdscal.f, 412
zgbbrd.f, 8242
zgbcon.f, 8268
zgbequ.f, 8280
zgbequb.f, 8290
zgbmv.f, 1052
zgbrfs.f, 8302
zgbrfsx.f, 8324
zgbsv.f, 7480
zgbsvx.f, 7489
zgbsvxx.f, 7515
zgbtf2.f, 8334
zgbtrf.f, 8342
zgbtrs.f, 8365
zgebak.f, 6685, 15505
zgebal.f, 6691, 15515
zgebd2.f, 6699
zgebrd.f, 6712
zgecon.f, 6722
zgeequ.f, 6732
zgeequb.f, 6741
zgees.f, 3084
zgeesx.f, 3093
zgeev.f, 3102, 15530
zgeevx.f, 3128
zgehd2.f, 6752, 15552
zgehrd.f, 6757
zgejsv.f, 4473
zgelq2.f, 6763
zgelqf.f, 6769
zgels.f, 2342
zgelsd.f, 2363
zgelss.f, 2398
zgelsy.f, 2441
zgemm.f, 1642
zgemqrt.f, 6779
zgemv.f, 1066
zgeql2.f, 6789
zgeqlf.f, 6795
zgeqp3.f, 6804
zgeqr2.f, 6817
zgeqr2p.f, 6823
zgeqrf.f, 6829
zgeqrfp.f, 6838
zgeqrt.f, 6847
zgeqrt2.f, 6853
zgeqrt3.f, 6861
zgerc.f, 1077
zgerfs.f, 6872
zgerfsx.f, 6893
zgerq2.f, 6902
zgerqf.f, 6908

zgeru.f, 1084
zgesc2.f, 7279
zgesdd.f, 4571
zgesv.f, 2460
zgesvd.f, 4715
zgesvdx.f, 4916
zgesvj.f, 5509
zgesvx.f, 2468
zgesvxx.f, 2493
zgetc2.f, 7285
zgetf2.f, 6917
zgetrf.f, 6924
zgetrf2.f, 6933
zgetri.f, 6942
zgetrs.f, 6952
zggbak.f, 8377
zggbal.f, 8389
zgges.f, 3156
zgges3.f, 3168
zggesx.f, 3180
zggev.f, 3192
zggev3.f, 3216
zggevx.f, 3243
zgtcon.f, 12329
zgtrfs.f, 12338
zgtsv.f, 12139
zgtsvx.f, 12150
zgttrf.f, 12356
zgttrs.f, 12364
zgtts2.f, 12371
zhbmv.f, 1092
zhemm.f, 1662
zhemv.f, 1105
zher.f, 1117
zher2.f, 1129
zher2k.f, 1677
zherk.f, 1697
zhgeqz.f, 6961
zhpmv.f, 1144
zhpr.f, 1156
zhpr2.f, 1169
zhseqr.f, 15561
zla_gbamv.f, 8417
zla_gbrcond_c.f, 8432
zla_gbrcond_x.f, 8446
zla_gbrfsx_extended.f, 8462
zla_gbrpvgrw.f, 8479
zla_geamv.f, 7003
zla_gercond_c.f, 7016
zla_gercond_x.f, 7028
zla_gerfsx_extended.f, 7042
zla_gerpvgrw.f, 7059
zla_porcond_c.f, 11917
zla_porcond_x.f, 11930
zla_porfsx_extended.f, 11945
zla_porpvgrw.f, 11961
zla_syamv.f, 10580
zla_syrcond_c.f, 10595
zla_syrcond_x.f, 10608
zla_syrfsx_extended.f, 10624
zla_syrpvgrw.f, 10641
zlacgv.f, 15573
zlacpy.f, 15577
zladiv.f, 15581
zlaesy.f, 11253
zlahqr.f, 15586
zlahr2.f, 15609
zlangb.f, 8551
zlange.f, 7293, 15621
zlansy.f, 11259
zlaqgb.f, 8561
zlaqge.f, 7297
zlaqr0.f, 15630
zlaqr1.f, 15655
zlaqr2.f, 15664
zlaqr3.f, 15685
zlaqr4.f, 15706
zlaqr5.f, 15732
zlaqsy.f, 11269
zlarf.f, 15817
zlarfb.f, 15783
zlarft.f, 15832
zlartg.f, 15844
zlascl.f, 15853
zlaset.f, 15866
zlassq.f, 15872
zlasyf.f, 10656
zlasyf_rook.f, 10673
zlatrs.f, 15879
zlzrfg.f, 15824
zpocon.f, 11969
zpoequ.f, 11978

zpoequb.f, 11984
zporfs.f, 11991
zporfsx.f, 12012
zposv.f, 11472
zposvx.f, 11480
zposvxx.f, 11499
zpotf2.f, 12020
zpotrf.f, 12029
zpotrf2.f, 12039
zpotri.f, 12048
zpotrs.f, 12053
zptcon.f, 12599
zpteqr.f, 12606
zptrfs.f, 12614
zptsv.f, 12433
zptsvx.f, 12440
zpttrf.f, 12632
zpttrs.f, 12641
zptts2.f, 12647
zrot.f, 15912
zrotg.f, 417
zscal.f, 420
zswap.f, 423
zsycon.f, 10689
zsycon_rook.f, 10697
zsyconv.f, 10704
zsyequb.f, 10719
zsymm.f, 1716
zsymv.f, 11276
zsyr.f, 11288
zsyr2k.f, 1730
zsyrfs.f, 10736
zsyrfsx.f, 10757
zsyrk.f, 1745
zsysv.f, 8676
zsysv_rook.f, 8684
zsysvx.f, 8693
zsysvxx.f, 8709
zsyswapr.f, 11296
zsytf2.f, 10767
zsytf2_rook.f, 10779
zsytrf.f, 10786
zsytrf_rook.f, 10798
zsytri.f, 10808
zsytri2.f, 10826
zsytri2x.f, 10832
zsytri_rook.f, 10865
zsytrs.f, 10887
zsytrs2.f, 10907
zsytrs_rook.f, 10923
ztbmv.f, 1185
ztbsv.f, 1205
ztgevc.f, 7065
ztgex2.f, 7305
ztgexc.f, 7095
ztgsy2.f, 11302
ztgsyl.f, 10947
ztpmv.f, 1225
ztpsv.f, 1245
ztrevc.f, 15919
ztrexc.f, 15937
ztrmm.f, 1758
ztrmv.f, 1264
ztrsm.f, 1779
ztrsv.f, 1282
ztrsyl.f, 10975
zung2r.f, 15944
zungbr.f, 8485
zunghr.f, 15950
zungqr.f, 15958
zunm2r.f, 15969
zunmhr.f, 15977
zunmqr.f, 15986